

Document Number: N5054
Date: 2026-07-16
Revises: N5050
Reply to: Thomas Köppe
Google DeepMind
cxxeditor@gmail.com

Working Draft

Programming Languages — C++

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomatting.

Contents

Foreword	x
Introduction	xi
1 Scope	1
2 Normative references	2
3 Terms and definitions	3
4 General principles	10
4.1 Implementation compliance	10
4.2 Structure of this document	12
4.3 Syntax notation	12
5 Lexical conventions	13
5.1 Separate translation	13
5.2 Phases of translation	13
5.3 Characters	15
5.4 Comments	17
5.5 Preprocessing tokens	17
5.6 Header names	18
5.7 Preprocessing numbers	19
5.8 Operators and punctuators	19
5.9 Alternative tokens	20
5.10 Tokens	20
5.11 Identifiers	20
5.12 Keywords	21
5.13 Literals	22
6 Basics	32
6.1 Preamble	32
6.2 Declarations and definitions	33
6.3 One-definition rule	35
6.4 Scope	40
6.5 Name lookup	46
6.6 Splice specifiers	59
6.7 Program and linkage	60
6.8 Memory and objects	64
6.9 Types	80
6.10 Program execution	90
6.11 Contract assertions	102
7 Expressions	105
7.1 Preamble	105
7.2 Properties of expressions	106
7.3 Standard conversions	109
7.4 Usual arithmetic conversions	115
7.5 Primary expressions	115
7.6 Compound expressions	139
7.7 Constant evaluation	177

8	Statements	192
8.1	Preamble	192
8.2	Label	193
8.3	Expression statement	193
8.4	Compound statement or block	193
8.5	Selection statements	194
8.6	Iteration statements	196
8.7	Expansion statements	198
8.8	Jump statements	201
8.9	Assertion statement	203
8.10	Declaration statement	203
8.11	Ambiguity resolution	204
9	Declarations	206
9.1	Preamble	206
9.2	Specifiers	209
9.3	Declarators	227
9.4	Function contract specifiers	246
9.5	Initializers	249
9.6	Function definitions	268
9.7	Structured binding declarations	274
9.8	Enumerations	276
9.9	Namespaces	279
9.10	The <code>using</code> declaration	284
9.11	The <code>asm</code> declaration	289
9.12	Linkage specifications	290
9.13	Attributes	292
10	Modules	302
10.1	Module units and purviews	302
10.2	Export declaration	303
10.3	Import declaration	306
10.4	Global module fragment	307
10.5	Private module fragment	309
10.6	Instantiation context	310
10.7	Reachability	311
11	Classes	314
11.1	Preamble	314
11.2	Properties of classes	315
11.3	Class names	316
11.4	Class members	318
11.5	Unions	341
11.6	Local class declarations	343
11.7	Derived classes	344
11.8	Member access control	351
11.9	Initialization	362
11.10	Comparisons	374
12	Overloading	377
12.1	Preamble	377
12.2	Overload resolution	377
12.3	Address of an overload set	406
12.4	Overloaded operators	407
12.5	Built-in operators	412
12.6	User-defined literals	414

13	Templates	416
13.1	Preamble	416
13.2	Template parameters	418
13.3	Names of template specializations	422
13.4	Template arguments	426
13.5	Template constraints	432
13.6	Type equivalence	440
13.7	Template declarations	441
13.8	Name resolution	463
13.9	Template instantiation and specialization	480
13.10	Function template specializations	492
14	Exception handling	514
14.1	Preamble	514
14.2	Throwing an exception	515
14.3	Stack unwinding	517
14.4	Handling an exception	518
14.5	Exception specifications	519
14.6	Special functions	521
15	Preprocessing directives	523
15.1	Preamble	523
15.2	Conditional inclusion	526
15.3	Source file inclusion	528
15.4	Resource inclusion	529
15.5	Module directive	534
15.6	Header unit importation	535
15.7	Macro replacement	537
15.8	Line control	542
15.9	Diagnostic directives	543
15.10	Pragma directive	543
15.11	Null directive	543
15.12	Predefined macro names	543
15.13	Pragma operator	546
16	Library introduction	547
16.1	General	547
16.2	The C standard library	548
16.3	Method of description	548
16.4	Library-wide requirements	556
17	Language support library	579
17.1	General	579
17.2	Common definitions	579
17.3	Implementation properties	583
17.4	Arithmetic types	597
17.5	Startup and termination	599
17.6	Dynamic memory management	600
17.7	Type identification	607
17.8	Source location	610
17.9	Exception handling	612
17.10	Contract-violation handling	616
17.11	Initializer lists	618
17.12	Comparisons	619
17.13	Coroutines	628
17.14	Other runtime support	632
17.15	C headers	634

18 Concepts library	637
18.1 General	637
18.2 Equality preservation	637
18.3 Header <code><concepts></code> synopsis	638
18.4 Language-related concepts	640
18.5 Comparison concepts	645
18.6 Object concepts	648
18.7 Callable concepts	648
19 Diagnostics library	650
19.1 General	650
19.2 Exception classes	650
19.3 Assertions	653
19.4 Error numbers	654
19.5 System error support	655
19.6 Stacktrace	664
19.7 Debugging	670
20 Memory management library	672
20.1 General	672
20.2 Memory	672
20.3 Pointer tagging	694
20.4 Smart pointers	697
20.5 Types for composite class design	726
20.6 Memory resources	737
20.7 Class template <code>scoped_allocator_adaptor</code>	745
21 Metaprogramming library	750
21.1 General	750
21.2 Compile-time integer sequences	750
21.3 Metaprogramming and type traits	751
21.4 Reflection	779
21.5 Compile-time rational arithmetic	817
22 General utilities library	821
22.1 General	821
22.2 Utility components	821
22.3 Pairs	832
22.4 Tuples	838
22.5 Optional objects	852
22.6 Variants	871
22.7 Storage for any type	883
22.8 Expected objects	888
22.9 Bitsets	910
22.10 Function objects	916
22.11 Bit manipulation	952
22.12 Header <code><stdbit.h></code> synopsis	957
23 Containers library	960
23.1 General	960
23.2 Requirements	960
23.3 Sequence containers	995
23.4 Associative containers	1042
23.5 Unordered associative containers	1063
23.6 Container adaptors	1089
23.7 Views	1138

24 Iterators library	1192
24.1 General	1192
24.2 Header <code><iterator></code> synopsis	1192
24.3 Iterator requirements	1200
24.4 Iterator primitives	1221
24.5 Iterator adaptors	1225
24.6 Stream iterators	1250
24.7 Range access	1256
25 Ranges library	1259
25.1 General	1259
25.2 Header <code><ranges></code> synopsis	1259
25.3 Range access	1268
25.4 Range requirements	1273
25.5 Range utilities	1276
25.6 Range factories	1285
25.7 Range adaptors	1298
25.8 Range generators	1420
26 Algorithms library	1426
26.1 General	1426
26.2 Algorithms requirements	1426
26.3 Parallel algorithms	1428
26.4 Header <code><algorithm></code> synopsis	1432
26.5 Algorithm result types	1486
26.6 Non-modifying sequence operations	1489
26.7 Mutating sequence operations	1509
26.8 Sorting and related operations	1532
26.9 Header <code><numeric></code> synopsis	1565
26.10 Generalized numeric operations	1569
26.11 Specialized <code><memory></code> algorithms	1579
26.12 Specialized <code><random></code> algorithms	1586
26.13 C library algorithms	1587
27 Strings library	1589
27.1 General	1589
27.2 Character traits	1589
27.3 String view classes	1594
27.4 String classes	1604
27.5 Null-terminated sequence utilities	1632
28 Text processing library	1634
28.1 General	1634
28.2 Primitive numeric conversions	1634
28.3 Localization library	1636
28.4 Text encodings identification	1677
28.5 Formatting	1684
28.6 Regular expressions library	1713
28.7 Null-terminated sequence utilities	1746
29 Numerics library	1751
29.1 General	1751
29.2 Numeric type requirements	1751
29.3 The floating-point environment	1751
29.4 Complex numbers	1752
29.5 Random number generation	1759
29.6 Numeric arrays	1805
29.7 Mathematical functions for floating-point types	1824
29.8 Numbers	1837

29.9	Basic linear algebra algorithms	1837
29.10	Data-parallel types	1890
29.11	C compatibility	1940
30	Time library	1941
30.1	General	1941
30.2	Header <code><chrono></code> synopsis	1941
30.3	<i>Cpp17Clock</i> requirements	1955
30.4	Time-related traits	1956
30.5	Class template <code>duration</code>	1957
30.6	Class template <code>time_point</code>	1964
30.7	Clocks	1967
30.8	The civil calendar	1978
30.9	Class template <code>hh_mm_ss</code>	2007
30.10	12/24 hours functions	2009
30.11	Time zones	2010
30.12	Formatting	2023
30.13	Parsing	2027
30.14	Hash support	2031
30.15	Header <code><ctime></code> synopsis	2032
31	Input/output library	2033
31.1	General	2033
31.2	Iostreams requirements	2033
31.3	Forward declarations	2034
31.4	Standard iostream objects	2036
31.5	Iostreams base classes	2038
31.6	Stream buffers	2053
31.7	Formatting and manipulators	2061
31.8	String-based streams	2088
31.9	Span-based streams	2104
31.10	File-based streams	2112
31.11	Synchronized output streams	2124
31.12	File systems	2129
31.13	C library files	2176
32	Concurrency support library	2180
32.1	General	2180
32.2	Requirements	2180
32.3	Stop tokens	2183
32.4	Threads	2192
32.5	Atomic operations	2201
32.6	Mutual exclusion	2244
32.7	Condition variables	2262
32.8	Semaphore	2269
32.9	Coordination types	2271
32.10	Futures	2274
32.11	Safe reclamation	2288
33	Execution control library	2295
33.1	General	2295
33.2	Queries and queryables	2296
33.3	Asynchronous operations	2296
33.4	Header <code><execution></code> synopsis	2298
33.5	Queries	2305
33.6	Schedulers	2308
33.7	Receivers	2309
33.8	Operation states	2311
33.9	Senders	2311

33.10	Completion signatures	2358
33.11	Queryable utilities	2359
33.12	Execution contexts	2361
33.13	Coroutine utilities	2363
33.14	Execution scope utilities	2375
33.15	Parallel scheduler	2381
33.16	Namespace <code>parallel_scheduler_replacement</code>	2382
Annex A	Grammar summary	2386
A.1	General	2386
A.2	Keywords	2386
A.3	Lexical conventions	2386
A.4	Basics	2391
A.5	Expressions	2391
A.6	Statements	2396
A.7	Declarations	2397
A.8	Modules	2404
A.9	Classes	2405
A.10	Overloading	2406
A.11	Templates	2407
A.12	Exception handling	2408
A.13	Preprocessing directives	2408
Annex B	Implementation quantities	2411
Annex C	Compatibility	2413
C.1	C++ and ISO C++ 2026	2413
C.2	C++ and ISO C++ 2023	2414
C.3	C++ and ISO C++ 2020	2418
C.4	C++ and ISO C++ 2017	2423
C.5	C++ and ISO C++ 2014	2431
C.6	C++ and ISO C++ 2011	2435
C.7	C++ and ISO C++ 2003	2437
C.8	C++ and C	2443
C.9	C standard library	2453
Annex D	Compatibility features	2455
D.1	General	2455
D.2	Non-local use of TU-local entities	2455
D.3	Implicit capture of <code>*this</code> by reference	2455
D.4	Deprecated <code>volatile</code> types	2455
D.5	Non-comma-separated ellipsis parameters	2456
D.6	Implicit declaration of copy functions	2456
D.7	Redeclaration of <code>static constexpr</code> data members	2456
D.8	Literal operator function declarations using an identifier	2456
D.9	<code>template</code> keyword before qualified names	2457
D.10	<code>has_denorm</code> members in <code>numeric_limits</code>	2457
D.11	Deprecated C macros	2457
D.12	Deprecated error numbers	2457
D.13	Deprecated type traits	2458
D.14	Relational operators	2459
D.15	Tuple	2459
D.16	Variant	2460
D.17	Deprecated <code>vector<bool, Allocator></code> swap	2460
D.18	Deprecated <code>iterator</code> class template	2460
D.19	Deprecated <code>move_iterator</code> access	2461
D.20	Deprecated locale category facets	2461
D.21	Deprecated formatting	2461
D.22	Deprecated time formatting	2462

D.23	Deprecated file systems	2462
D.24	Deprecated atomic operations	2463
D.25	Deprecated <code>signed char</code> and <code>unsigned char</code> extraction	2464
D.26	Deprecated <code>signed char</code> and <code>unsigned char</code> insertion	2465
Annex E	Conformance with UAX #31	2466
E.1	General	2466
E.2	R1 Default identifiers	2466
E.3	R2 Immutable identifiers	2466
E.4	R3 Pattern_White_Space and Pattern_Syntax characters	2466
E.5	R4 Equivalent normalized identifiers	2467
E.6	R5 Equivalent case-insensitive identifiers	2467
E.7	R6 Filtered normalized identifiers	2467
E.8	R7 Filtered case-insensitive identifiers	2467
E.9	R8 Hashtag identifiers	2467
Annex F	Core undefined behavior	2468
F.1	General	2468
F.2	Clause 6: Basics	2468
F.3	Clause 7: Expressions	2475
F.4	Clause 8: Statements	2484
F.5	Clause 9: Declarations	2485
F.6	Clause 11: Classes	2489
F.7	Clause 13: Templates	2493
F.8	Clause 14: Exception handling	2493
Annex G	Ill-formed, no diagnostic required	2494
G.1	General	2494
G.2	Clause 5: Lexical conventions	2494
G.3	Clause 6: Basics	2494
G.4	Clause 7: Expressions	2497
G.5	Clause 8: Statements	2498
G.6	Clause 9: Declarations	2498
G.7	Clause 10: Modules	2500
G.8	Clause 11: Classes	2501
G.9	Clause 12: Overloading	2502
G.10	Clause 13: Templates	2502
G.11	Clause 15: Preprocessing directives	2506
Bibliography		2508
Cross-references		2509
Cross-references from ISO C++ 2017		2540
Index		2544
Index of grammar productions		2583
Index of library headers		2590
Index of library names		2592
Index of library concepts		2715
Index of implementation-defined behavior		2721

Foreword

[This page is intentionally left blank.]

Introduction

Clauses and subclauses in this document are annotated with a so-called stable name, presented in square brackets next to the (sub)clause heading (such as “[lex.token]” for [5.10](#), “Tokens”). Stable names aid in the discussion and evolution of this document by serving as stable references to subclauses across editions that are unaffected by changes of subclause numbering.

Aspects of the language syntax of C++ are distinguished typographically by the use of *italic*, *sans-serif* type or **constant width** type to avoid ambiguities; see [4.3](#).

Programming languages — C++

1 Scope

[intro.scope]

- ¹ This document specifies requirements for implementations of C++, which is a general-purpose programming language. The first such requirement is that an implementation implements the language, so this document also defines C++. Other requirements and relaxations of the first requirement appear at various places within this document.

2 Normative references

[intro.refs]

¹ The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- (1.1) — ISO/IEC 2382, *Information technology — Vocabulary*
- (1.2) — ISO 8601-1:2019, *Date and time — Representations for information interchange — Part 1: Basic rules*
- (1.3) — ISO/IEC 9899:2024, *Information technology — Programming languages — C*
- (1.4) — ISO/IEC/IEEE 9945:2009, *Information Technology — Portable Operating System Interface (POSIX®)*¹
Base Specifications, Issue 7
- (1.5) — ISO/IEC/IEEE 9945:2009/Cor 1:2013, *Information Technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7 — Technical Corrigendum 1*
- (1.6) — ISO/IEC/IEEE 9945:2009/Cor 2:2017, *Information Technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7 — Technical Corrigendum 2*
- (1.7) — ISO/IEC 60559:2020, *Information technology — Microprocessor Systems — Floating-Point arithmetic*
- (1.8) — ISO 80000-2:2019, *Quantities and units — Part 2: Mathematics*
- (1.9) — Ecma International, *ECMAScript*² *Language Specification*, Standard Ecma-262, third edition, 1999.
- (1.10) — The Unicode Consortium. *The Unicode Standard*, Version 15.1. Available from: <https://www.unicode.org/versions/Unicode15.1.0/>

1) POSIX® is a registered trademark of the Institute of Electrical and Electronic Engineers, Inc. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.

2) ECMAScript® is a registered trademark of Ecma International. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.

3 Terms and definitions

[intro.defs]

- ¹ For the purposes of this document, the terms and definitions given in ISO/IEC 2382, ISO 80000-2:2019, and the following apply.
- ² ISO and IEC maintain terminology databases for use in standardization at the following addresses:
- (2.1) — ISO Online browsing platform: available at <https://www.iso.org/obp>
 - (2.2) — IEC Electropedia: available at <https://www.electropedia.org/>

3.1 [defns.access]

access

⟨execution-time action⟩ read or modify the value of an object

[*Note 1 to entry:* Only glvalues of scalar type (6.9.1) can be used to access objects. Reads of scalar objects are described in 7.3.2 and modifications of scalar objects are described in 7.6.19, 7.6.1.6, and 7.6.2.3. Attempts to read or modify an object of class type typically invoke a constructor (11.4.5) or assignment operator (11.4.6); such invocations do not themselves constitute accesses, although they may involve accesses of scalar subobjects. — *end note*]

3.2 [defns.argument]

argument

⟨function call expression⟩ expression or *braced-init-list* in the comma-separated list bounded by the parentheses

3.3 [defns.argument.macro]

argument

⟨function-like macro⟩ sequence of preprocessing tokens in the comma-separated list bounded by the parentheses

3.4 [defns.argument.throw]

argument

⟨throw expression⟩ operand of **throw**

3.5 [defns.argument.templ]

argument

⟨template instantiation⟩ *constant-expression*, *type-id*, or *id-expression* in the comma-separated list bounded by the angle brackets

3.6 [defns.block]

block

⟨execution⟩ wait for some condition (other than for the implementation to execute the execution steps of the thread of execution) to be satisfied before continuing execution past the blocking operation

3.7 [defns.block.stmt]

block

⟨statement⟩ compound statement

3.8 [defns.c.lib]

C standard library

library described in ISO/IEC 9899:2024, Clause 7

[*Note 1 to entry:* With the qualifications noted in Clause 17 through Clause 33 and in C.9, the C standard library is a subset of the C++ standard library. — *end note*]

3.9 [defns.character]

character

⟨library⟩ object which, when treated sequentially, can represent text

[*Note 1 to entry:* The term does not mean only `char`, `char8_t`, `char16_t`, `char32_t`, and `wchar_t` objects (6.9.3), but any value that can be represented by a type that provides the definitions specified in Clause 27, 28.3, Clause 31, or 28.6. — *end note*]

3.10**[defns.character.container]****character container type***<library>* class or a type used to represent a *character* (3.9)

[*Note 1 to entry:* It is used for one of the template parameters of `char_traits` and the class templates which use that, such as the string, `iostream`, and regular expression class templates. — *end note*]

3.11**[defns.regex.collating.element]****collating element**sequence of one or more *characters* (3.9) within the current locale that collate as if they were a single character**3.12****[defns.component]****component***<library>* group of library entities directly related as members, *parameters* (3.38), or return types

[*Note 1 to entry:* For example, the class template `basic_string` and the non-member function templates that operate on strings are referred to as the string component. — *end note*]

3.13**[defns.cond.suppl]****conditionally-supported**

program construct that an implementation is not required to support

[*Note 1 to entry:* Each implementation documents all conditionally-supported constructs that it does not support. — *end note*]

3.14**[defns.const.eval]****constant evaluation**

evaluation that is performed as part of evaluating an expression as a core constant expression (7.7.2)

3.15**[defns.const.subexpr]****constant subexpression**expression whose evaluation as subexpression of a *conditional-expression* CE would not prevent CE from being a core constant expression**3.16****[defns.deadlock]****deadlock**

<library> situation wherein one or more threads are unable to continue execution because each is *blocked* (3.6) waiting for one or more of the others to satisfy some condition

3.17**[defns.default.behavior.impl]****default behavior***<library implementation>* specific behavior provided by the implementation, within the scope of the *required behavior* (3.49)**3.18****[defns.diagnostic]****diagnostic message**

message belonging to an implementation-defined subset of the implementation's output messages

3.19**[defns.dynamic.type]****dynamic type***<glvalue>* type of the most derived object to which the glvalue refers

[*Example 1:* If a pointer (9.3.4.2) `p` whose type is “pointer to class B” is pointing to a base class subobject of class B, whose most derived object is of class D, derived from B (11.7), the dynamic type of the expression `*p` is “D”. References (9.3.4.3) are treated similarly. — *end example*]

3.20**[defns.dynamic.type.prvalue]****dynamic type***<prvalue>* *static type* (3.62) of the prvalue expression

3.21**[defns.erroneous]****erroneous behavior**

well-defined behavior that the implementation is recommended to diagnose

[*Note 1 to entry:* Erroneous behavior is always the consequence of incorrect program code. Implementations are allowed, but not required, to diagnose it (4.1.1). Evaluation of a constant expression (7.7.2) never exhibits behavior specified as erroneous in Clause 4 through Clause 15. — *end note*]

3.22**[defns.expression.equivalent]****expression-equivalent**

⟨library⟩ expressions that all have the same effects, either are all potentially-throwing or are all not potentially-throwing, and either are all *constant subexpressions* (3.15) or are all not constant subexpressions

[*Example 1:* For a value *x* of type *int* and a function *f* that accepts integer arguments, the expressions *f*(*x* + 2), *f*(2 + *x*), and *f*(1 + *x* + 1) are expression-equivalent. — *end example*]

3.23**[defns.regex.finite.state.machine]****finite state machine**

⟨regular expression⟩ unspecified data structure that is used to represent a *regular expression* (3.47), and which permits efficient matches against the regular expression to be obtained

3.24**[defns.regex.format.specifier]****format specifier**

⟨regular expression⟩ sequence of one or more *characters* (3.9) that is expected to be replaced with some part of a *regular expression* (3.47) match

3.25**[defns.handler]****handler function**

⟨library⟩ non-reserved function whose definition may be provided by a C++ program

[*Note 1 to entry:* A C++ program may designate a handler function at various points in its execution by supplying a pointer to the function when calling any of the library functions that install handler functions (see Clause 17). — *end note*]

3.26**[defns.ill.formed]****ill-formed program**

program that is not well-formed (3.69)

3.27**[defns.impl.defined]****implementation-defined behavior**

behavior, for a *well-formed program* (3.69) construct and correct data, that depends on the implementation and that each implementation documents

3.28**[defns.order.ptr]****implementation-defined strict total order over pointers**

⟨library⟩ implementation-defined strict total ordering over all pointer values such that the ordering is consistent with the partial order imposed by the built-in operators <, >, <=, >=, and <=>

3.29**[defns.impl.limits]****implementation limit**

restriction imposed upon programs by the implementation

3.30**[defns.locale.specific]****locale-specific behavior**

behavior that depends on local conventions of nationality, culture, and language that each implementation documents

3.31**[defns.regex.matched]****matched**

⟨regular expression⟩ condition when a sequence of zero or more *characters* (3.9) correspond to a sequence of characters defined by the pattern

3.32 [defns.modifier]**modifier function**

⟨library⟩ class member function other than a constructor, assignment operator, or destructor that alters the state of an object of the class

3.33 [defns.move.assign]**move assignment**

⟨library⟩ assignment of an rvalue of some object type to a modifiable lvalue of the same type

3.34 [defns.move.constr]**move construction**

⟨library⟩ direct-initialization of an object of some type with an rvalue of the same type

3.35 [defns.nonconst.libcall]**non-constant library call**

invocation of a library function that, as part of evaluating any expression E, prevents E from being a core constant expression

3.36 [defns.ntcts]**NTCTS**

⟨library⟩ sequence of values that have *character* (3.9) type that precede the terminating null character type value `charT()`

3.37 [defns.observer]**observer function**

⟨library⟩ class member function that accesses the state of an object of the class but does not alter that state

[*Note 1 to entry:* Observer functions are specified as `const` member functions. — *end note*]

3.38 [defns.parameter]**parameter**

⟨function or catch clause⟩ object or reference declared as part of a function declaration or definition or in the catch clause of an exception handler that acquires a value on entry to the function or handler

3.39 [defns.parameter.macro]**parameter**

⟨function-like macro⟩ identifier from the comma-separated list bounded by the parentheses immediately following the macro name

3.40 [defns.parameter.templ]**parameter**

⟨template⟩ member of a *template-parameter-list*

3.41 [defns.regex.primary.equivalence.class]**primary equivalence class**

⟨regular expression⟩ set of one or more *characters* (3.9) which share the same primary sort key: that is the sort key weighting that depends only upon character shape, and not accents, case, or locale-specific tailorings

3.42 [defns.prog.def.spec]**program-defined specialization**

⟨library⟩ explicit template specialization or partial specialization that is not part of the C++ standard library and not defined by the implementation

3.43 [defns.prog.def.type]**program-defined type**

⟨library⟩ non-closure class type or enumeration type that is not part of the C++ standard library and not defined by the implementation, or a closure type of a non-implementation-provided lambda expression, or an instantiation of a *program-defined specialization* (3.42)

[*Note 1 to entry:* Types defined by the implementation include extensions (4.1) and internal types used by the library. — *end note*]

3.44**[defns.projection]****projection**

⟨library⟩ transformation that an algorithm applies before inspecting the values of elements

[*Example 1*:

```
std::pair<int, std::string_view> pairs[] = {{2, "foo"}, {1, "bar"}, {0, "baz"}};
std::ranges::sort(pairs, std::ranges::less{}, [](auto const& p) { return p.first; });
```

sorts the pairs in increasing order of their **first** members:

```
{{0, "baz"}, {1, "bar"}, {2, "foo"}}
```

— *end example*]

3.45**[defns.impl.prop]****property of the implementation**

behavior, for a well-formed program (3.69) construct and correct data, that depends on the implementation

3.46**[defns.referenceable]****referenceable type**

type that is either an object type, a function type that does not have cv-qualifiers or a *ref-qualifier*, or a reference type

[*Note 1 to entry*: The term describes a type to which a reference can be created, including reference types. — *end note*]

3.47**[defns.regex.regular.expression]****regular expression**

pattern that selects specific strings from a set of *character* (3.9) strings

3.48**[defns.replacement]****replacement function**

⟨library⟩ non-reserved function whose definition is provided by a C++ program

[*Note 1 to entry*: Only one definition for such a function is in effect for the duration of the program's execution, as the result of creating the program (5.2) and resolving the definitions of all translation units (6.7). — *end note*]

3.49**[defns.required.behavior]****required behavior**

⟨library⟩ description of *replacement function* (3.48) and *handler function* (3.25) semantics applicable to both the behavior provided by the implementation and the behavior of any such function definition in the program

[*Note 1 to entry*: If such a function defined in a C++ program fails to meet the required behavior when it executes, the behavior is undefined. — *end note*]

3.50**[defns.reserved.function]****reserved function**

⟨library⟩ function, specified as part of the C++ standard library, that is defined by the implementation

[*Note 1 to entry*: If a C++ program provides a definition for any reserved function, the results are undefined. — *end note*]

3.51**[defns.undefined.runtime]****runtime-undefined behavior**

behavior that is undefined except when it occurs during constant evaluation

[*Note 1 to entry*: During constant evaluation,

- it is implementation-defined whether runtime-undefined behavior results in the expression being deemed non-constant (as specified in 7.7.2) and
- runtime-undefined behavior has no other effect.

— *end note*]

3.52**[defns.signature]****signature**

⟨function⟩ name, parameter-type-list, and enclosing namespace

[*Note 1 to entry*: Signatures are used as a basis for name mangling and linking. — *end note*]

- 3.53** [defns.signature.friend]
signature
 ⟨non-template friend function with trailing *requires-clause*⟩ name, parameter-type-list, enclosing class, and trailing *requires-clause*
- 3.54** [defns.signature.templ]
signature
 ⟨function template⟩ name, parameter-type-list, enclosing namespace, return type, *signature* (3.60) of the *template-head*, and trailing *requires-clause* (if any)
- 3.55** [defns.signature.templ.friend]
signature
 ⟨friend function template with constraint involving enclosing template parameters⟩ name, parameter-type-list, return type, enclosing class, *signature* (3.60) of the *template-head*, and trailing *requires-clause* (if any)
- 3.56** [defns.signature.spec]
signature
 ⟨function template specialization⟩ *signature* (3.54) of the template of which it is a specialization and its template *arguments* (3.5) (whether explicitly specified or deduced)
- 3.57** [defns.signature.member]
signature
 ⟨class member function⟩ name, parameter-type-list, class of which the function is a member, *cv*-qualifiers (if any), *ref-qualifier* (if any), and trailing *requires-clause* (if any)
- 3.58** [defns.signature.member.templ]
signature
 ⟨class member function template⟩ name, parameter-type-list, class of which the function is a member, *cv*-qualifiers (if any), *ref-qualifier* (if any), return type (if any), *signature* (3.60) of the *template-head*, and trailing *requires-clause* (if any)
- 3.59** [defns.signature.member.spec]
signature
 ⟨class member function template specialization⟩ *signature* (3.58) of the member function template of which it is a specialization and its template arguments (whether explicitly specified or deduced)
- 3.60** [defns.signature.template.head]
signature
 ⟨*template-head*⟩ template *parameter* (3.40) list, excluding template parameter names and default *arguments* (3.5), and *requires-clause* (if any)
- 3.61** [defns.stable]
stable algorithm
 ⟨library⟩ algorithm that preserves, as appropriate to the particular algorithm, the order of elements
 [Note 1 to entry: Requirements for stable algorithms are given in 16.4.6.8. — end note]
- 3.62** [defns.static.type]
static type
 type of an expression resulting from analysis of the program without considering execution semantics
 [Note 1 to entry: The static type of an expression depends only on the form of the program in which the expression appears, and does not change while the program is executing. — end note]
- 3.63** [defns.regex.subexpression]
sub-expression
 ⟨regular expression⟩ subset of a *regular expression* (3.47) that has been marked by parentheses
- 3.64** [defns.traits]
traits class
 ⟨library⟩ class that encapsulates a set of types and functions necessary for class templates and function templates to manipulate objects of types for which they are instantiated

3.65**[defns.unblock]****unblock**

satisfy a condition that one or more *blocked* (3.6) threads of execution are waiting for

3.66**[defns.undefined]****undefined behavior**

behavior for which this document imposes no requirements

[*Note 1 to entry:* Undefined behavior may be expected when this document omits any explicit definition of behavior or when a program uses an incorrect construct or invalid data. Permissible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a *diagnostic message* (3.18)), to terminating a translation or execution (with the issuance of a diagnostic message). Many incorrect program constructs do not engender undefined behavior; they are required to be diagnosed. Evaluation of a constant expression (7.7.2) never exhibits behavior explicitly specified as undefined in [Clause 4](#) through [Clause 15](#). — *end note*]

3.67**[defns.unspecified]****unspecified behavior**

behavior, for a *well-formed program* (3.69) construct and correct data, that depends on the implementation

[*Note 1 to entry:* The implementation is not required to document which behavior occurs. The range of possible behaviors is usually delineated by this document. — *end note*]

3.68**[defns.valid]****valid but unspecified state**

⟨library⟩ value of an object that is not specified except that the object's invariants are met and operations on the object behave as specified for its type

[*Example 1:* If an object `x` of type `std::vector<int>` is in a valid but unspecified state, `x.empty()` can be called unconditionally, and `x.front()` can be called only if `x.empty()` returns `false`. — *end example*]

3.69**[defns.well.formed]****well-formed program**

C++ program constructed according to the syntax and semantic rules

4 General principles

[intro]

4.1 Implementation compliance

[intro.compliance]

4.1.1 General

[intro.compliance.general]

- ¹ The set of *diagnosable rules* consists of all syntactic and semantic rules in this document except for those rules containing an explicit notation that “no diagnostic is required” or which are described as resulting in “undefined behavior”.
 - ² Although this document states only requirements on C++ implementations, those requirements are often easier to understand if they are phrased as requirements on programs, parts of programs, or execution of programs. Such requirements have the following meaning:
 - (2.1) — If a program contains no violations of the rules in [Clause 5](#) through [Clause 33](#) as well as those specified in [Annex D](#), a conforming implementation shall accept and correctly execute³ that program, except when the implementation’s limitations (see below) are exceeded.
 - (2.2) — If a program contains a violation of a rule for which no diagnostic is required, this document places no requirement on implementations with respect to that program.
 - (2.3) — Otherwise, if a program contains
 - (2.3.1) — a violation of any diagnosable rule,
 - (2.3.2) — a preprocessing translation unit with a `#warning` preprocessing directive ([15.9](#)),
 - (2.3.3) — an occurrence of a construct described in this document as “conditionally-supported” when the implementation does not support that construct, or
 - (2.3.4) — a contract assertion ([6.11.2](#)) evaluated with a checking semantic in a manifestly constant-evaluated context ([7.7.7](#)) resulting in a contract violation,
 a conforming implementation shall issue at least one diagnostic message.
- [*Note 1*: During template argument deduction and substitution, certain constructs that in other contexts require a diagnostic are treated differently; see [13.10.3](#). — *end note*]
- Furthermore, a conforming implementation shall not accept
- (2.4) — a preprocessing translation unit containing a `#error` preprocessing directive ([15.9](#)),
 - (2.5) — a translation unit with a `static_assert-declaration` that fails ([9.1](#)), or
 - (2.6) — a contract assertion evaluated with a terminating semantic ([6.11.2](#)) in a manifestly constant-evaluated context ([7.7.7](#)) resulting in a contract violation.
- ³ For classes and class templates, the library Clauses specify partial definitions. Private members ([11.8](#)) are not specified, but each implementation shall supply them to complete the definitions according to the description in the library Clauses.
 - ⁴ For functions, function templates, objects, and values, the library Clauses specify declarations. Implementations shall supply definitions consistent with the descriptions in the library Clauses.
 - ⁵ A C++ translation unit ([5.2](#)) obtains access to the names defined in the library by including the appropriate standard library header or importing the appropriate standard library named header unit ([16.4.3.2](#)).
 - ⁶ The templates, classes, functions, and objects in the library have external linkage ([6.7](#)). The implementation provides definitions for standard library entities, as necessary, while combining translation units to form a complete C++ program ([5.2](#)).
 - ⁷ An implementation is either a *hosted implementation* or a *freestanding implementation*. A freestanding implementation is one in which execution may take place without the benefit of an operating system. A hosted implementation supports all the facilities described in this document, while a freestanding implementation supports the entire C++ language described in [Clause 5](#) through [Clause 15](#) and the subset of the library facilities described in [16.4.2.5](#).

³) “Correct execution” can include undefined behavior and erroneous behavior, depending on the data being processed; see [Clause 3](#) and [6.10.1](#).

- 8 It is implementation-defined whether the implementation is a *hardened implementation*. If it is a hardened implementation, violating a hardened precondition results in a contract violation (16.3.2.4).
- 9 *Recommended practice*: An implementation is encouraged to document its limitations in the size or complexity of the programs it can successfully process, if possible and where known. Annex B lists some quantities that can be subject to limitations and a potential minimum supported value for each quantity.
- 10 A conforming implementation may use an implementation-defined version of the Unicode Standard that is a later version than the one referenced in Clause 2.
- 11 A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any well-formed program. Implementations are required to diagnose programs that use such extensions that are ill-formed according to this document. Having done so, however, they can compile and execute such programs.
- 12 Each implementation shall include documentation that identifies all conditionally-supported constructs that it does not support and defines all locale-specific characteristics.⁴

4.1.2 Abstract machine

[intro.abstract]

- 1 The semantic descriptions in this document define a parameterized nondeterministic abstract machine. This document places no requirement on the structure of conforming implementations. In particular, they need not copy or emulate the structure of the abstract machine. Rather, conforming implementations are required to emulate (only) the observable behavior of the abstract machine as explained below.⁵
- 2 Certain aspects and operations of the abstract machine constitute the parameters of the abstract machine and are described in this document as implementation-defined behavior (for example, `sizeof(int)`) or as properties of the implementation (for example, padding in class types). For implementation-defined behavior, each implementation shall include documentation describing its characteristics and behavior in these respects.⁶ Such documentation shall define the instance of the abstract machine that corresponds to that implementation (referred to as the “corresponding instance” below).
- 3 Certain other aspects and operations of the abstract machine are described in this document as unspecified behavior (for example, order of evaluation of arguments in a function call (7.6.1.3)). Where possible, this document defines a set of allowable behaviors. These define the nondeterministic aspects of the abstract machine. An instance of the abstract machine can thus have more than one possible execution for a given program and a given input.
- 4 Certain other operations are described in this document as undefined behavior (for example, the effect of attempting to modify a `const` object).
- 5 Certain events in the execution of a program are termed *observable checkpoints*.

[Note 1: A call to `std::observable_checkpoint` (22.2.9) is an observable checkpoint, as are certain parts of the evaluation of contract assertions (6.11). — end note]

- 6 The *defined prefix* of an execution comprises the operations O for which for every undefined operation U there is an observable checkpoint C such that O happens before C and C happens before U .

[Note 2: The undefined behavior that arises from a data race (6.10.2.2) occurs on all participating threads. — end note]

A conforming implementation executing a well-formed program shall produce the observable behavior of the defined prefix of one of the possible executions of the corresponding instance of the abstract machine with the same program and the same input. If the selected execution contains an undefined operation, the implementation executing that program with that input may produce arbitrary additional observable behavior afterwards. If the execution of an operation is specified as having erroneous behavior, the implementation is permitted to issue a diagnostic and is permitted to terminate the execution of the program.

- 7 *Recommended practice*: An implementation should issue a diagnostic when such an operation is executed.

[Note 3: An implementation can issue a diagnostic if it can determine that erroneous behavior is reachable under an implementation-specific set of assumptions about the program behavior, which can result in false positives. — end note]

⁴) This documentation also defines implementation-defined behavior; see 4.1.2.

⁵) This provision is sometimes called the “as-if” rule, because an implementation is free to disregard any requirement of this document as long as the result is *as if* the requirement had been obeyed, as far as can be determined from the observable behavior of the program. For instance, an actual implementation need not evaluate part of an expression if it can deduce that its value is not used and that no side effects affecting the observable behavior of the program are produced.

⁶) This documentation also includes conditionally-supported constructs and locale-specific behavior. See 4.1.1.

⁸ The following specify the *observable behavior* of the program:

- (8.1) — Accesses through volatile glvalues are evaluated strictly according to the rules of the abstract machine.
- (8.2) — Data is delivered to the host environment to be written into files (SEE ALSO: ISO/IEC 9899:2024, 7.23.3).

[*Note 4*: Delivering such data is followed by an observable checkpoint (31.13.1). Not all host environments provide access to file contents before program termination. — *end note*]

- (8.3) — The input and output dynamics of interactive devices shall take place in such a fashion that prompting output is actually delivered before a program waits for input. What constitutes an interactive device is implementation-defined.

[*Note 5*: More stringent correspondences between abstract and actual semantics can be defined by each implementation. — *end note*]

4.2 Structure of this document

[intro.structure]

- ¹ Clause 5 through Clause 15 describe the C++ programming language. That description includes detailed syntactic specifications in a form described in 4.3. For convenience, Annex A repeats all such syntactic specifications.
- ² Clause 17 through Clause 33 and Annex D (the *library clauses*) describe the C++ standard library. That description includes detailed descriptions of the entities and macros that constitute the library, in a form described in Clause 16.
- ³ Annex B recommends lower bounds on the capacity of conforming implementations.
- ⁴ Annex C summarizes the evolution of C++ since its first published description, and explains in detail the differences between C++ and C. Certain features of C++ exist solely for compatibility purposes; Annex D describes those features.

4.3 Syntax notation

[syntax]

- ¹ In the syntax notation used in this document, syntactic categories are indicated by *italic*, *sans-serif* type, and literal words and characters in **constant width** type. Alternatives are listed on separate lines except in a few cases where a long set of alternatives is marked by the phrase “one of”. If the text of an alternative is too long to fit on a line, the text is continued on subsequent lines indented from the first one. An optional terminal or non-terminal symbol is indicated by the subscript “*opt*”, so

{ *expression_{opt}* }

indicates an optional expression enclosed in braces.

- ² Names for syntactic categories have generally been chosen according to the following rules:

- (2.1) — *X-name* is a use of an identifier in a context that determines its meaning (e.g., *class-name*, *typedef-name*).
- (2.2) — *X-id* is an identifier with no context-dependent meaning (e.g., *qualified-id*).
- (2.3) — *X-seq* is one or more *X*’s without intervening delimiters (e.g., *declaration-seq* is a sequence of *declarations*).
- (2.4) — *X-list* is one or more *X*’s separated by intervening commas (e.g., *identifier-list* is a sequence of *identifiers* separated by commas).

5 Lexical conventions

[lex]

5.1 Separate translation

[lex.separate]

- ¹ The text of the program is kept in units called *source files* in this document. A source file together with all the headers (16.4.2.3) and source files included (15.3) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (15.2) preprocessing directives, as modified by the implementation-defined behavior of any conditionally-supported-directives (15.1) and pragmas (15.10), if any, is called a *preprocessing translation unit*.

[Note 1: A C++ program need not all be translated at the same time. Translation units can be separately translated and then later linked to produce an executable program (6.7). — end note]

[Note 2: Source files, translation units, and translated translation units need not necessarily be stored as files, nor need there be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation. — end note]

5.2 Phases of translation

[lex.phases]

- ¹ The precedence among the syntax rules of translation is specified by the following phases.⁷
1. An implementation shall support input files that are a sequence of UTF-8 code units (UTF-8 files). It may also support an implementation-defined set of other kinds of input files, and, if so, the kind of an input file is determined in an implementation-defined manner that includes a means of designating input files as UTF-8 files, independent of their content.

[Note 1: In other words, recognizing the U+FEFF BYTE ORDER MARK is not sufficient. — end note]

If an input file is determined to be a UTF-8 file, then it shall be a well-formed UTF-8 code unit sequence and it is decoded to produce a sequence of Unicode⁸ scalar values. A sequence of translation character set elements (5.3.1) is then formed by mapping each Unicode scalar value to the corresponding translation character set element. In the resulting sequence, each pair of characters in the input sequence consisting of U+000D CARRIAGE RETURN followed by U+000A LINE FEED, as well as each U+000D CARRIAGE RETURN not immediately followed by a U+000A LINE FEED, is replaced by a single new-line character.

For any other kind of input file supported by the implementation, characters are mapped, in an implementation-defined manner, to a sequence of translation character set elements, representing end-of-line indicators as new-line characters.

2. If the first translation character is U+FEFF BYTE ORDER MARK, it is deleted. Each sequence comprising a backslash character (\) immediately followed by zero or more whitespace characters other than new-line followed by a new-line character is deleted, splicing physical source lines to form *logical source lines*. Only the last backslash on any physical source line is eligible for being part of such a splice.

[Note 2: Line splicing can form a *universal-character-name* (5.3.1). — end note]

A source file that is not empty and that (after splicing) does not end in a new-line character is processed as if an additional new-line character were appended to the file.

3. The source file is decomposed into preprocessing tokens (5.5) and sequences of whitespace characters (including comments). A source file shall not end in a partial preprocessing token or in a partial comment.⁹ Each comment (5.4) is replaced by one U+0020 SPACE character. New-line characters are retained. Whether each nonempty sequence of whitespace characters other than new-line is retained or replaced by one U+0020 SPACE character is unspecified. As characters from the source file are consumed to form the next preprocessing token (i.e., not being consumed as part of a comment or other forms of whitespace), except when matching a *c-char-sequence*, *s-char-sequence*, *r-char-sequence*, *h-char-sequence*, or *q-char-sequence*, *universal-character-names* are recognized (5.3.2) and replaced by the designated

⁷ Implementations behave as if these separate phases occur, although in practice different phases can be folded together.

⁸ Unicode® is a registered trademark of Unicode, Inc. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.

⁹ A partial preprocessing token would arise from a source file ending in the first portion of a multi-character token that requires a terminating sequence of characters, such as a *header-name* that is missing the closing " or >. A partial comment would arise from a source file ending with an unclosed /* comment.

element of the translation character set (5.3.1). The process of dividing a source file's characters into preprocessing tokens is context-dependent.

[*Example 1*: See the handling of < within a `#include` preprocessing directive (5.6, 15.3). — *end example*]

4. The source file is analyzed as a *preprocessing-file* (15.1). Preprocessing directives (Clause 15) are executed, macro invocations are expanded (15.7), and `_Pragma` unary operator expressions are executed (15.13). A `#include` preprocessing directive (15.3) causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted. Whitespace characters separating preprocessing tokens are no longer significant.
5. For a sequence of two or more adjacent *string-literal* preprocessing tokens, a common *encoding-prefix* is determined as specified in 5.13.5. Each such *string-literal* preprocessing token is then considered to have that common *encoding-prefix*. Then, adjacent *string-literal* preprocessing tokens are concatenated (5.13.5).
6. Each preprocessing token is converted into a token (5.10).
7. The tokens constitute a *translation unit* and are syntactically and semantically analyzed as a *translation-unit* (6.7) and translated.

[*Note 3*: The process of analyzing and translating the tokens can occasionally result in one token being replaced by a sequence of other tokens (13.3). — *end note*]

It is implementation-defined whether the sources for module units and header units on which the current translation unit has an interface dependency (10.1, 10.3) are required to be available.

[*Note 4*: Previously translated translation units can be preserved individually or in libraries. The separate translation units of a program communicate (6.7) by (for example) calls to functions whose names have external or module linkage, manipulation of variables whose names have external or module linkage, or manipulation of data files. — *end note*]

While the tokens constituting translation units are being analyzed and translated, required instantiations are performed.

[*Note 5*: This can include instantiations which have been explicitly requested (13.9.3). — *end note*]

The contexts from which instantiations may be performed are determined by their respective points of instantiation (13.8.4.1).

[*Note 6*: Other requirements in this document can further constrain the context from which an instantiation can be performed. For example, a `constexpr` function template specialization might have a point of instantiation at the end of a translation unit, but its use in certain constant expressions could require that it be instantiated at an earlier point (13.9.2). — *end note*]

The program is ill-formed if any instantiation fails.

During the analysis and translation of tokens, each manifestly constant-evaluated expression is evaluated (7.7.7). The values of these expressions affect the types named and reflection values used in the program and thus the syntactic analysis. Their evaluation also can produce side effects that affect that analysis. Constructs appearing at a program point *P* are analyzed in a context where each side effect of evaluating an expression *E* as a full-expression is complete if and only if

- (1.1) — *E* is the expression corresponding to a *constexpr-block-declaration* (9.1), and
- (1.2) — either that *constexpr-block-declaration* or the template definition from which it is instantiated is reachable from (10.7)
 - (1.2.1) — *P*, or
 - (1.2.2) — the point immediately following the *class-specifier* of the outermost class for which *P* is in a complete-class context (11.4.1).

[*Example 2*:

```
class S {
    class Incomplete;

    class Inner {
        void fn() {
            /* p1 */ Incomplete i;    // OK
        }
    } /* p2 */ ;
}
```

```

consteval {
    define_aggregate(~~Incomplete, {});
}
} /* p3 */ ;

```

Constructs at p_1 are analyzed in a context where the side effect of the call to `define_aggregate` is complete because

- (1.3) — the call appears in a `consteval` block, and
- (1.4) — p_1 is in a complete-class context of **S** and the `consteval` block is reachable from p_3 .
— *end example*]

8. Translated translation units are combined, and all external entity references are resolved (6.7). Library components are linked to satisfy external references to entities not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

5.3 Characters

[lex.char]

5.3.1 Character sets

[lex.charset]

- 1 The *translation character set* consists of the following elements:

- (1.1) — each abstract character assigned a code point in the Unicode codespace as specified in the Unicode Standard, and
- (1.2) — a distinct character for each Unicode scalar value not assigned to an abstract character.

[Note 1: Unicode code points are integers in the range [0, 10FFFF] (hexadecimal). A surrogate code point is a value in the range [D800, DFFF] (hexadecimal). A Unicode scalar value is any code point that is not a surrogate code point. — *end note*]

- 2 The *basic character set* is a subset of the translation character set, consisting of 99 characters as specified in Table 1. In this document, glyphs are often used to identify elements of the basic character set.

[Note 2: Unicode short names are given only as a means to identifying the character; the numerical value has no other meaning in this context. — *end note*]

- 3 The *basic literal character set* consists of all characters of the basic character set, plus the control characters specified in Table 2.
- 4 A *code unit* is an integer value of character type (6.9.3). Characters in a *character-literal* other than a multicharacter or non-encodable character literal or in a *string-literal* are encoded as a sequence of one or more code units, as determined by the *encoding-prefix* (5.13.3, 5.13.5); this is termed the respective *literal encoding*. The *ordinary literal encoding* is the encoding applied to an ordinary character or string literal. The *wide literal encoding* is the encoding applied to a wide character or string literal.
- 5 A literal encoding or a locale-specific encoding of one of the execution character sets (16.3.3.3.4) encodes each element of the basic literal character set as a single code unit with non-negative value, distinct from the code unit for any other such element.

[Note 3: A character not in the basic literal character set can be encoded with more than one code unit; the value of such a code unit can be the same as that of a code unit for an element of the basic literal character set. — *end note*]

The U+0000 NULL character is encoded as the value 0. No other element of the translation character set is encoded with a code unit of value 0. The code unit value of each decimal digit character after the digit 0 (U+0030) is one greater than the value of the previous. The ordinary and wide literal encodings are otherwise implementation-defined. For a UTF-8, UTF-16, or UTF-32 literal, the implementation shall encode the Unicode scalar value corresponding to each character of the translation character set as specified in the Unicode Standard for the respective Unicode encoding form.

5.3.2 Universal character names

[lex.universal.char]

n-char:

any member of the translation character set except the U+007D RIGHT CURLY BRACKET or new-line character

n-char-sequence:

n-char *n-char-sequence*_{opt}

named-universal-character:

\N{ *n-char-sequence* }

Table 1 — Basic character set [tab:lex.charset.basic]

character		glyph
U+0009	CHARACTER TABULATION	
U+000B	LINE TABULATION	
U+000C	FORM FEED	
U+0020	SPACE	
U+000A	LINE FEED	new-line
U+0021	EXCLAMATION MARK	!
U+0022	QUOTATION MARK	"
U+0023	NUMBER SIGN	#
U+0024	DOLLAR SIGN	\$
U+0025	PERCENT SIGN	%
U+0026	AMPERSAND	&
U+0027	APOSTROPHE	'
U+0028	LEFT PARENTHESIS	(
U+0029	RIGHT PARENTHESIS	)
U+002A	ASTERISK	*
U+002B	PLUS SIGN	+
U+002C	COMMA	,
U+002D	HYPHEN-MINUS	-
U+002E	FULL STOP	.
U+002F	SOLIDUS	/
U+0030 .. U+0039	DIGIT ZERO .. NINE	0 1 2 3 4 5 6 7 8 9
U+003A	COLON	:
U+003B	SEMICOLON	;
U+003C	LESS-THAN SIGN	<
U+003D	EQUALS SIGN	=
U+003E	GREATER-THAN SIGN	>
U+003F	QUESTION MARK	?
U+0040	COMMERCIAL AT	@
U+0041 .. U+005A	LATIN CAPITAL LETTER A .. Z	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
U+005B	LEFT SQUARE BRACKET	[
U+005C	REVERSE SOLIDUS	\
U+005D	RIGHT SQUARE BRACKET	]
U+005E	CIRCUMFLEX ACCENT	^
U+005F	LOW LINE	_
U+0060	GRAVE ACCENT	`
U+0061 .. U+007A	LATIN SMALL LETTER A .. Z	a b c d e f g h i j k l m n o p q r s t u v w x y z
U+007B	LEFT CURLY BRACKET	{
U+007C	VERTICAL LINE	
U+007D	RIGHT CURLY BRACKET	}
U+007E	TILDE	~

Table 2 — Additional control characters in the basic literal character set [tab:lex.charset.literal]

character	
U+0000	NULL
U+0007	ALERT
U+0008	BACKSPACE
U+000D	CARRIAGE RETURN

hex-quad:
hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

simple-hexadecimal-digit-sequence:
hexadecimal-digit simple-hexadecimal-digit-sequence_{opt}

universal-character-name:
\u hex-quad
\U hex-quad hex-quad
\u{ simple-hexadecimal-digit-sequence }
named-universal-character

- ¹ The *universal-character-name* construct provides a way to name any element in the translation character set using just the basic character set. If a *universal-character-name* outside the *c-char-sequence* or *s-char-sequence* of a *character-literal* or *string-literal* (in either case, including within a *user-defined-literal*) corresponds to a control character or to a character in the basic character set, the program is ill-formed.

[*Note 1*: A sequence of characters resembling a *universal-character-name* in an *r-char-sequence* (5.13.5) does not form a *universal-character-name*. — end note]

- ² A *universal-character-name* of the form *\u hex-quad*, *\U hex-quad hex-quad*, or *\u{simple-hexadecimal-digit-sequence}* designates the character in the translation character set whose Unicode scalar value is the hexadecimal number represented by the sequence of *hexadecimal-digits* in the *universal-character-name*. The program is ill-formed if that number is not a Unicode scalar value.
- ³ A *universal-character-name* that is a *named-universal-character* designates the corresponding character in the Unicode Standard (chapter 4.8 Name) if the *n-char-sequence* is equal to its character name or to one of its character name aliases; otherwise, the program is ill-formed.

[*Note 2*: These aliases are listed in the Unicode Character Database's `NameAliases.txt`. None of these names or aliases have leading or trailing spaces. — end note]

5.4 Comments

[lex.comment]

- ¹ The characters */** start a comment, which terminates with the characters **/*. These comments do not nest. The characters *//* start a comment, which terminates immediately before the next new-line character.

[*Note 1*: The comment characters *//*, */**, and **/* have no special meaning within a *//* comment and are treated just like other characters. Similarly, the comment characters *//* and */** have no special meaning within a */** comment. — end note]

5.5 Preprocessing tokens

[lex.pptoken]

preprocessing-token:
header-name
import-keyword
module-keyword
export-keyword
identifier
pp-number
character-literal
user-defined-character-literal
string-literal
user-defined-string-literal
preprocessing-op-or-punc
 each non-whitespace character that cannot be one of the above

- ¹ A preprocessing token is the minimal lexical element of the language in translation phases 3 through 5. The categories of preprocessing token are: header names, placeholder tokens produced by preprocessing `import` and `module` directives (*import-keyword*, *module-keyword*, and *export-keyword*), identifiers, preprocessing numbers, character literals (including user-defined character literals), string literals (including user-defined string literals), preprocessing operators and punctuators, and single non-whitespace characters that do not lexically match the other preprocessing token categories. If a U+0027 APOSTROPHE, a U+0022 QUOTATION MARK, or any character not in the basic character set matches the last category, the program is ill-formed.
- ² Preprocessing tokens can be separated by whitespace; this consists of comments (5.4), or whitespace characters (U+0020 SPACE, U+0009 CHARACTER TABULATION, new-line, U+000B LINE TABULATION, and U+000C FORM FEED), or both.

[*Note 1:* As described in [Clause 15](#), in certain circumstances during translation phase 4, whitespace (or the absence thereof) serves as more than preprocessing token separation. Whitespace can appear within a preprocessing token only as part of a header name or between the quotation characters in a character literal or string literal. — *end note*]

- 3 Each preprocessing token that is converted to a token ([5.10](#)) shall have the lexical form of a keyword, an identifier, a literal, a header name, or an operator or punctuation.
- 4 The *import-keyword* is produced by processing an **import** directive ([15.6](#)), the *module-keyword* is produced by preprocessing a **module** directive ([15.5](#)), and the *export-keyword* is produced by preprocessing either of the previous two directives.

[*Note 2:* None has any observable spelling. — *end note*]

- 5 If the input stream has been parsed into preprocessing tokens up to a given character:

- (5.1) — If the next character begins a sequence of characters that could be the prefix and initial double quote of a raw string literal, such as **R**", the next preprocessing token shall be a raw string literal. Between the initial and final double quote characters of the raw string, any transformations performed in phase 2 (line splicing) are reverted; this reversion is applied before any *d-char*, *r-char*, or delimiting parenthesis is identified. The raw string literal is defined as the shortest sequence of characters that matches the raw-string pattern

encoding-prefix_{opt} R raw-string

- (5.2) — Otherwise, if the next three characters are **<::** and the subsequent character is neither **:** nor **>**, the **<** is treated as a preprocessing token by itself and not as the first character of the alternative token **<:**.
- (5.3) — Otherwise, if the next three characters are **[::** and the subsequent character is not **:**, or if the next three characters are **[>**, the **[** is treated as a preprocessing token by itself and not as the first character of the preprocessing token **[:**.

[*Note 3:* The tokens **[:** and **:]** cannot be composed from digraphs. — *end note*]

- (5.4) — Otherwise, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token, even if that would cause further lexical analysis to fail, except that

- (5.4.1) — a *string-literal* token is never formed when a *header-name* token can be formed, and
- (5.4.2) — a *header-name* ([5.6](#)) is only formed
 - (5.4.2.1) — immediately after the **include** or **embed** preprocessing token in a **#include** ([15.3](#)) or **#embed** ([15.4](#)) directive, respectively, or
 - (5.4.2.2) — immediately after an **import** preprocessing token that is at the start of a logical source line, or
 - (5.4.2.3) — immediately after a preprocessing token sequence of **__has_include** or **__has_embed** immediately followed by **(** in a **#if**, **#elif**, or **#embed** directive ([15.2](#), [15.4](#)).

A preprocessing token is considered to be *immediately after* another preprocessing token if the preprocessing tokens are on the same logical source line and there are no intervening preprocessing tokens.

- 6 [*Example 1:*

```
#define R "x"
const char* s = R"y";           // ill-formed raw string, not "x" "y"
```

— *end example*]

- 7 [*Example 2:* The program fragment **0xe+foo** is parsed as a preprocessing number token (one that is not a valid *integer-literal* or *floating-point-literal* token), even though a parse as three preprocessing tokens **0xe**, **+**, and **foo** can produce a valid expression (for example, if **foo** is a macro defined as **1**). Similarly, the program fragment **1E1** is parsed as a preprocessing number (one that is a valid *floating-point-literal* token), whether or not **E** is a macro name. — *end example*]

- 8 [*Example 3:* The program fragment **x+++++y** is parsed as **x ++ ++ + y**, which, if **x** and **y** have integral types, violates a constraint on increment operators, even though the parse **x ++ + ++ y** can yield a correct expression. — *end example*]

5.6 Header names

[lex.header]

header-name:

< *h-char-sequence* **>**
" *q-char-sequence* **"**

h-char-sequence:
h-char h-char-sequence_{opt}

h-char:
 any member of the translation character set except new-line and U+003E GREATER-THAN SIGN

q-char-sequence:
q-char q-char-sequence_{opt}

q-char:
 any member of the translation character set except new-line and U+0022 QUOTATION MARK

- ¹ The sequences in both forms of *header-names* are mapped in an implementation-defined manner to headers or to external source file names as specified in 15.3.

[Note 1: Header name preprocessing tokens appear only within a `#include` preprocessing directive, a `__has_include` preprocessing expression, or after certain occurrences of an `import` token (see 5.5). — end note]

- ² The appearance of either of the characters ' or \ or of either of the character sequences /* or // in a *q-char-sequence* or an *h-char-sequence* is conditionally-supported with implementation-defined semantics, as is the appearance of the character " in an *h-char-sequence*.

[Note 2: Thus, a sequence of characters that resembles an escape sequence can result in an error, be interpreted as the character corresponding to the escape sequence, or have a completely different meaning, depending on the implementation. — end note]

5.7 Preprocessing numbers

[lex.ppnumber]

pp-number:
digit
. *digit*
pp-number identifier-continue
pp-number ' digit
pp-number ' nondigit
pp-number e sign
pp-number E sign
pp-number p sign
pp-number P sign
pp-number .

- ¹ Preprocessing number tokens lexically include all *integer-literal* tokens (5.13.2) and all *floating-point-literal* tokens (5.13.4).
- ² A preprocessing number does not have a type or a value; it acquires both after a successful conversion to an *integer-literal* token or a *floating-point-literal* token.

5.8 Operators and punctuators

[lex.operators]

- ¹ The lexical representation of C++ programs includes a number of preprocessing tokens that are used in the syntax of the preprocessor or are converted into tokens for operators and punctuators:

preprocessing-op-or-punc:
preprocessing-operator
operator-or-punctuator

preprocessing-operator: one of

%: %::

operator-or-punctuator: one of

{	}	[	]	(	)	[:	:]		
<%	%>	<:	:>	;	:	...			
?	::	.	.*	->	->*	^^	~		
!	+	-	*	/	%	^	&		
=	+=	-=	*=	/=	%=	^=	&=	=	
==	!=	<	>	<=	>=	<=>	&&		
<<	>>	<<=	>>=	++	--	,			
and	or	xor	not	bitand	bitor	compl			
and_eq	or_eq	xor_eq	not_eq						

Each *operator-or-punctuator* is converted to a single token in translation phase 6 (5.2).

5.9 Alternative tokens

[lex.digraph]

- ¹ Alternative token representations are provided for some operators and punctuators.¹⁰
- ² In all respects of the language, each alternative token behaves the same, respectively, as its primary token, except for its spelling.

[*Note 1*: The “stringized” values (15.7.3) of [and <: are different, maintaining the source spelling. — *end note*]

The set of alternative tokens is defined in Table 3.

Table 3 — Alternative tokens [tab:lex.digraph]

Alternative	Primary	Alternative	Primary	Alternative	Primary
<%	{	and	&&	and_eq	&=
%>	}	bitor		or_eq	=
<:	[	or		xor_eq	^=
:>	]	xor	^	not	!
%:	#	compl	~	not_eq	!=
%::	##	bitand	&		

5.10 Tokens

[lex.token]

token:

identifier

keyword

literal

header-name

operator-or-punctuator

- ¹ There are six kinds of tokens: identifiers, keywords, literals, header names, operators, and other separators.

5.11 Identifiers

[lex.name]

identifier:

identifier-start

identifier identifier-continue

identifier-start:

nondigit

an element of the translation character set with the Unicode property XID_Start or the Unicode property ID_Compat_Math_Start

identifier-continue:

digit

nondigit

an element of the translation character set with the Unicode property XID_Continue or the Unicode property ID_Compat_Math_Continue

nondigit: one of

a b c d e f g h i j k l m

n o p q r s t u v w x y z

A B C D E F G H I J K L M

N O P Q R S T U V W X Y Z _

digit: one of

0 1 2 3 4 5 6 7 8 9

- ¹ [*Note 1*: The character properties XID_Start and XID_Continue are described by UAX #44 of the Unicode Standard.¹¹ — *end note*]

¹⁰ These include “digraphs” and additional reserved words. The term “digraph” (token consisting of two characters) is not perfectly descriptive, since one of the alternative *preprocessing-tokens* is %:: and of course several primary tokens contain two characters. Nonetheless, those alternative tokens that aren’t lexical keywords are colloquially known as “digraphs”.

¹¹ On systems in which linkers cannot accept extended characters, an encoding of the *universal-character-name* can be used in forming valid external identifiers. For example, some otherwise unused character or sequence of characters can be used to encode the \u in a *universal-character-name*. Extended characters can produce a long external identifier, but C++ does not place a translation limit on significant characters for external identifiers.

The program is ill-formed if an *identifier* does not conform to Normalization Form C as specified in the Unicode Standard.

[Note 2: Identifiers are case-sensitive. — end note]

[Note 3: Annex E compares the requirements of UAX #31 of the Unicode Standard with the C++ rules for identifiers. — end note]

[Note 4: In translation phase 4, *identifier* also includes those *preprocessing-tokens* (5.5) differentiated as keywords (5.12) in the later translation phase 7 (5.10). — end note]

- ² The identifiers in Table 4 have a special meaning when appearing in a certain context. When referred to in the grammar, these identifiers are used explicitly rather than using the *identifier* grammar production. Unless otherwise specified, any ambiguity as to whether a given *identifier* has a special meaning is resolved to interpret the token as a regular *identifier*.

Table 4 — Identifiers with special meaning [tab:lex.name.special]

final	import	module	override	post	pre
-------	--------	--------	----------	------	-----

³ [Note 5: Identifiers with special meaning cannot be used as macro names (15.7.1). — end note]

- ⁴ In addition, some identifiers appearing as a *token* or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required (G.2.1).

- (4.1) — Each identifier that contains a double underscore `__` or begins with an underscore followed by an uppercase letter, other than those specified in this document (for example, `__cplusplus` (15.12)), is reserved to the implementation for any use.
- (4.2) — Each identifier that begins with an underscore is reserved to the implementation for use as a name in the global namespace.

5.12 Keywords

[lex.key]

keyword:

any identifier listed in Table 5

import-keyword

module-keyword

export-keyword

- ¹ The identifiers shown in Table 5 are reserved for use as keywords (that is, they are unconditionally treated as keywords in phases 6 and 7) except in an *attribute-token* (9.13.1).

[Note 1: The `register` keyword is unused but is reserved for future use. — end note]

Table 5 — Keywords [tab:lex.key]

alignas	constinit	extern	protected	throw
alignof	const_cast	false	public	true
asm	continue	float	register	try
auto	contract_assert	for	reinterpret_cast	typedef
bool	co_await	friend	requires	typeid
break	co_return	goto	return	typename
case	co_yield	if	short	union
catch	decltype	inline	signed	unsigned
char	default	int	sizeof	using
char8_t	delete	long	static	virtual
char16_t	do	mutable	static_assert	void
char32_t	double	namespace	static_cast	volatile
class	dynamic_cast	new	struct	wchar_t
concept	else	noexcept	switch	while
const	enum	nullptr	template	
constexpr	explicit	operator	this	
constexpr	export	private	thread_local	

² [Note 2: Keywords cannot be used as macro names (15.7.1). — end note]

³ Furthermore, the alternative representations shown in Table 6 for certain operators and punctuators (5.9) are reserved and shall not be used otherwise.

Table 6 — Alternative representations [tab:lex.key.digraph]

and	and_eq	bitand	bitor	compl	not
not_eq	or	or_eq	xor	xor_eq	

5.13 Literals

[lex.literal]

5.13.1 Kinds of literals

[lex.literal.kinds]

¹ There are several kinds of literals.

literal:

integer-literal
character-literal
floating-point-literal
string-literal
boolean-literal
pointer-literal
user-defined-literal

[Note 1: When appearing as an *expression*, a literal has a type and a value category (7.5.2). — end note]

5.13.2 Integer literals

[lex.icon]

integer-literal:

binary-literal integer-suffix_{opt}
octal-literal integer-suffix_{opt}
decimal-literal integer-suffix_{opt}
hexadecimal-literal integer-suffix_{opt}

binary-literal:

0b *binary-digit*
 0B *binary-digit*
binary-literal ' _{opt} *binary-digit*

octal-literal:

0
octal-literal ' _{opt} *octal-digit*

decimal-literal:

nonzero-digit
decimal-literal ' _{opt} *digit*

hexadecimal-literal:

hexadecimal-prefix hexadecimal-digit-sequence

binary-digit: one of

0 1

octal-digit: one of

0 1 2 3 4 5 6 7

nonzero-digit: one of

1 2 3 4 5 6 7 8 9

hexadecimal-prefix: one of

0x 0X

hexadecimal-digit-sequence:

hexadecimal-digit
hexadecimal-digit-sequence ' _{opt} *hexadecimal-digit*

hexadecimal-digit: one of

0 1 2 3 4 5 6 7 8 9
 a b c d e f
 A B C D E F

integer-suffix:
 unsigned-suffix long-suffix_{opt}
 unsigned-suffix long-long-suffix_{opt}
 unsigned-suffix size-suffix_{opt}
 long-suffix unsigned-suffix_{opt}
 long-long-suffix unsigned-suffix_{opt}
 size-suffix unsigned-suffix_{opt}

unsigned-suffix: one of
 u U

long-suffix: one of
 l L

long-long-suffix: one of
 ll LL

size-suffix: one of
 z Z

- ¹ In an *integer-literal*, the sequence of *binary-digits*, *octal-digits*, *digits*, or *hexadecimal-digits* is interpreted as a base *N* integer as shown in Table 7; the lexically first digit of the sequence of digits is the most significant.
 [Note 1: The prefix and any optional separating single quotes are ignored when determining the value. — end note]

Table 7 — Base of *integer-literals* [tab:lex.icon.base]

Kind of <i>integer-literal</i>	base <i>N</i>
<i>binary-literal</i>	2
<i>octal-literal</i>	8
<i>decimal-literal</i>	10
<i>hexadecimal-literal</i>	16

- ² The *hexadecimal-digits* **a** through **f** and **A** through **F** have decimal values ten through fifteen.
 [Example 1: The number twelve can be written 12, 014, 0XC, or 0b1100. The *integer-literals* 1048576, 1'048'576, 0X100000, 0x10'0000, and 0'004'000'000 all have the same value. — end example]
- ³ The type of an *integer-literal* is the first type in the list in Table 8 corresponding to its optional *integer-suffix* in which its value can be represented.
- ⁴ Except for *integer-literals* containing a *size-suffix*, if the value of an *integer-literal* cannot be represented by any type in its list and an extended integer type (6.9.3) can represent its value, it may have that extended integer type. If all of the types in the list for the *integer-literal* are signed, the extended integer type is signed. If all of the types in the list for the *integer-literal* are unsigned, the extended integer type is unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. If an *integer-literal* cannot be represented by any of the allowed types, the program is ill-formed.
 [Note 2: An *integer-literal* with a **z** or **Z** suffix is ill-formed if it cannot be represented by `std::size_t`. — end note]

5.13.3 Character literals

[lex.ccon]

character-literal:
 encoding-prefix_{opt} ' *c-char-sequence* '

encoding-prefix: one of
 u8 u U L

c-char-sequence:
 c-char c-char-sequence_{opt}

c-char:
 basic-c-char
 escape-sequence
 universal-character-name

basic-c-char:
 any member of the translation character set except the U+0027 APOSTROPHE,
 U+005C REVERSE SOLIDUS, or new-line character

Table 8 — Types of *integer-literals* [tab:lex.icon.type]

<i>integer-suffix</i>	<i>decimal-literal</i>	<i>integer-literal</i> other than <i>decimal-literal</i>
none	int long int long long int	int unsigned int long int unsigned long int long long int unsigned long long int
u or U	unsigned int unsigned long int unsigned long long int	unsigned int unsigned long int unsigned long long int
l or L	long int long long int	long int unsigned long int long long int unsigned long long int
Both u or U and l or L	unsigned long int unsigned long long int	unsigned long int unsigned long long int
ll or LL	long long int	long long int unsigned long long int
Both u or U and ll or LL	unsigned long long int	unsigned long long int
z or Z	the signed integer type corresponding to <code>std::size_t</code> (17.2.4)	the signed integer type corresponding to <code>std::size_t</code> <code>std::size_t</code>
Both u or U and z or Z	<code>std::size_t</code>	<code>std::size_t</code>

escape-sequence:

simple-escape-sequence
numeric-escape-sequence
conditional-escape-sequence

simple-escape-sequence:

\ *simple-escape-sequence-char*

simple-escape-sequence-char: one of

' " ? \ a b f n r t v

numeric-escape-sequence:

octal-escape-sequence
hexadecimal-escape-sequence

simple-octal-digit-sequence:

octal-digit *simple-octal-digit-sequence*_{opt}

octal-escape-sequence:

\ *octal-digit*
\ *octal-digit* *octal-digit*
\ *octal-digit* *octal-digit* *octal-digit*
\ o{ *simple-octal-digit-sequence* }

hexadecimal-escape-sequence:

\ x *simple-hexadecimal-digit-sequence*
\ x{ *simple-hexadecimal-digit-sequence* }

conditional-escape-sequence:

\ *conditional-escape-sequence-char*

conditional-escape-sequence-char:

any member of the basic character set that is not an *octal-digit*, a *simple-escape-sequence-char*, or the characters N, o, u, U, or x

- ¹ A *multicharacter literal* is a *character-literal* whose *c-char-sequence* consists of more than one *c-char*. A multicharacter literal shall not have an *encoding-prefix*. If a multicharacter literal contains a *c-char* that is not encodable as a single code unit in the ordinary literal encoding, the program is ill-formed. Multicharacter literals are conditionally-supported.
- ² The kind of a *character-literal*, its type, and its associated character encoding (5.3.1) are determined by its *encoding-prefix* and its *c-char-sequence* as defined by Table 9.

Table 9 — Character literals [tab:lex.ccon.literal]

Encoding prefix	Kind	Type	Associated character encoding	Example
none	<i>ordinary character literal</i>	char	ordinary literal encoding	'v'
	<i>multicharacter literal</i>	int		'abcd'
L	<i>wide character literal</i>	wchar_t	wide literal encoding	L'w'
u8	<i>UTF-8 character literal</i>	char8_t	UTF-8	u8'x'
u	<i>UTF-16 character literal</i>	char16_t	UTF-16	u'y'
U	<i>UTF-32 character literal</i>	char32_t	UTF-32	U'z'

- ³ In translation phase 4, the value of a *character-literal* is determined using the range of representable values of the *character-literal*'s type in translation phase 7. A multicharacter literal has an implementation-defined value. The value of any other kind of *character-literal* is determined as follows:
- (3.1) — A *character-literal* with a *c-char-sequence* consisting of a single *basic-c-char*, *simple-escape-sequence*, or *universal-character-name* is the code unit value of the specified character as encoded in the literal's associated character encoding. If the specified character lacks representation in the literal's associated character encoding or if it cannot be encoded as a single code unit, then the program is ill-formed.
- (3.2) — A *character-literal* with a *c-char-sequence* consisting of a single *numeric-escape-sequence* has a value as follows:
- (3.2.1) — Let v be the integer value represented by the octal number comprising the sequence of *octal-digits* in an *octal-escape-sequence* or by the hexadecimal number comprising the sequence of *hexadecimal-digits* in a *hexadecimal-escape-sequence*.
- (3.2.2) — If v does not exceed the range of representable values of the *character-literal*'s type, then the value is v .
- (3.2.3) — Otherwise, if the *character-literal*'s *encoding-prefix* is absent or L, and v does not exceed the range of representable values of the corresponding unsigned type for the underlying type of the *character-literal*'s type, then the value is the unique value of the *character-literal*'s type T that is congruent to v modulo 2^N , where N is the width of T.
- (3.2.4) — Otherwise, the program is ill-formed.
- (3.3) — A *character-literal* with a *c-char-sequence* consisting of a single *conditional-escape-sequence* is conditionally-supported and has an implementation-defined value.
- ⁴ The character specified by a *simple-escape-sequence* is specified in Table 10.

[Note 1: Using an escape sequence for a question mark is supported for compatibility with C++ 2014 and C. — end note]

5.13.4 Floating-point literals

[lex.fcon]

floating-point-literal:

decimal-floating-point-literal

hexadecimal-floating-point-literal

decimal-floating-point-literal:

fractional-constant *exponent-part*_{opt} *floating-point-suffix*_{opt}

digit-sequence *exponent-part* *floating-point-suffix*_{opt}

hexadecimal-floating-point-literal:

hexadecimal-prefix *hexadecimal-fractional-constant* *binary-exponent-part* *floating-point-suffix*_{opt}

hexadecimal-prefix *hexadecimal-digit-sequence* *binary-exponent-part* *floating-point-suffix*_{opt}

Table 10 — Simple escape sequences [tab:lex.ccon.esc]

character	<i>simple-escape-sequence</i>
U+000A LINE FEED	<code>\n</code>
U+0009 CHARACTER TABULATION	<code>\t</code>
U+000B LINE TABULATION	<code>\v</code>
U+0008 BACKSPACE	<code>\b</code>
U+000D CARRIAGE RETURN	<code>\r</code>
U+000C FORM FEED	<code>\f</code>
U+0007 ALERT	<code>\a</code>
U+005C REVERSE SOLIDUS	<code>\\</code>
U+003F QUESTION MARK	<code>\?</code>
U+0027 APOSTROPHE	<code>\'</code>
U+0022 QUOTATION MARK	<code>\"</code>

fractional-constant:

*digit-sequence*_{opt} . *digit-sequence*
digit-sequence .

hexadecimal-fractional-constant:

*hexadecimal-digit-sequence*_{opt} . *hexadecimal-digit-sequence*
hexadecimal-digit-sequence .

exponent-part:

e *sign*_{opt} *digit-sequence*
E *sign*_{opt} *digit-sequence*

binary-exponent-part:

p *sign*_{opt} *digit-sequence*
P *sign*_{opt} *digit-sequence*

sign: one of

+ -

digit-sequence:

digit
digit-sequence ' *opt* *digit*

floating-point-suffix: one of

f l f16 f32 f64 f128 bf16 F L F16 F32 F64 F128 BF16

- ¹ The type of a *floating-point-literal* (6.9.3, 6.9.4) is determined by its *floating-point-suffix* as specified in Table 11.
[Note 1: The floating-point suffixes f16, f32, f64, f128, bf16, F16, F32, F64, F128, and BF16 are conditionally-supported. See 6.9.4. — end note]

Table 11 — Types of *floating-point-literals* [tab:lex.fcon.type]

<i>floating-point-suffix</i>	type
none	double
f or F	float
l or L	long double
f16 or F16	std::float16_t
f32 or F32	std::float32_t
f64 or F64	std::float64_t
f128 or F128	std::float128_t
bf16 or BF16	std::bfloat16_t

- ² The *significand* of a *floating-point-literal* is the *fractional-constant* or *digit-sequence* of a *decimal-floating-point-literal* or the *hexadecimal-fractional-constant* or *hexadecimal-digit-sequence* of a *hexadecimal-floating-point-literal*. In the *significand*, the sequence of *digits* or *hexadecimal-digits* and optional period are interpreted as a base *N* real number *s*, where *N* is 10 for a *decimal-floating-point-literal* and 16 for a *hexadecimal-floating-point-literal*.

[Note 2: Any optional separating single quotes are ignored when determining the value. — end note]

If an *exponent-part* or *binary-exponent-part* is present, the exponent e of the *floating-point-literal* is the result of interpreting the sequence of an optional *sign* and the *digits* as a base 10 integer. Otherwise, the exponent e is 0. The scaled value of the literal is $s \times 10^e$ for a *decimal-floating-point-literal* and $s \times 2^e$ for a *hexadecimal-floating-point-literal*.

[Example 1: The *floating-point-literals* 49.625 and 0xC.68p+2 have the same value. The *floating-point-literals* 1.602'176'565e-19 and 1.602176565e-19 have the same value. — end example]

- ³ If the scaled value is not in the range of representable values for its type, the program is ill-formed. Otherwise, the value of a *floating-point-literal* is the scaled value if representable, else the next greater or lesser value of the type, chosen in an implementation-defined manner.

[Example 2: The following example assumes that `std::float32_t` is supported (6.9.4).

```
std::float32_t x = 0.0f32;           // value 0 is exactly representable
std::float32_t y = 0.1f32;           // rounded to one of two values nearest to 0.1
std::float32_t z = 1e1000000000f32; // either greatest finite value or positive infinity
```

— end example]

5.13.5 String literals

[lex.string]

string-literal:

*encoding-prefix*_{opt} *plain-string-literal*
*encoding-prefix*_{opt} R *raw-string*

plain-string-literal:

" *s-char-sequence*_{opt} "

s-char-sequence:

s-char *s-char-sequence*_{opt}

s-char:

basic-s-char
escape-sequence
universal-character-name

basic-s-char:

any member of the translation character set except the U+0022 QUOTATION MARK,
U+005C REVERSE SOLIDUS, or new-line character

raw-string:

" *d-char-sequence*_{opt} (*r-char-sequence*_{opt}) *d-char-sequence*_{opt} "

r-char-sequence:

r-char *r-char-sequence*_{opt}

r-char:

any member of the translation character set, except a U+0029 RIGHT PARENTHESIS followed by
the initial *d-char-sequence* (which may be empty) followed by a U+0022 QUOTATION MARK

d-char-sequence:

d-char *d-char-sequence*_{opt}

d-char:

any member of the basic character set except:
U+0020 SPACE, U+0028 LEFT PARENTHESIS, U+0029 RIGHT PARENTHESIS, U+005C REVERSE SOLIDUS,
U+0009 CHARACTER TABULATION, U+000B LINE TABULATION, U+000C FORM FEED, and new-line

- ¹ The kind of a *string-literal*, its type, and its associated character encoding (5.3.1) are determined by its encoding prefix and sequence of *s-chars* or *r-chars* as defined by Table 12 where n is the number of encoded code units that would result from an evaluation of the *string-literal* (see below).
- ² A *string-literal* that has an R in the prefix is a *raw string literal*. The *d-char-sequence* serves as a delimiter. The terminating *d-char-sequence* of a *raw-string* is the same sequence of characters as the initial *d-char-sequence*. A *d-char-sequence* shall consist of at most 16 characters.
- ³ [Note 1: The characters '(' and ')' can appear in a *raw-string*. Thus, `R"delimiter((a|b))delimiter"` is equivalent to `"(a|b)"`. — end note]
- ⁴ [Note 2: A source-file new-line in a raw string literal results in a new-line in the resulting execution string literal. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

Table 12 — String literals [tab:lex.string.literal]

Enco- ding prefix	Kind	Type	Associated character encoding	Examples
none	<i>ordinary string literal</i>	array of <i>n</i> const char	ordinary literal encoding	"ordinary string" R"(ordinary raw string)"
L	<i>wide string literal</i>	array of <i>n</i> const wchar_t	wide literal encoding	L"wide string" LR"w(wide raw string)w"
u8	<i>UTF-8 string literal</i>	array of <i>n</i> const char8_t	UTF-8	u8"UTF-8 string" u8R"x(UTF-8 raw string)x"
u	<i>UTF-16 string literal</i>	array of <i>n</i> const char16_t	UTF-16	u"UTF-16 string" uR"y(UTF-16 raw string)y"
U	<i>UTF-32 string literal</i>	array of <i>n</i> const char32_t	UTF-32	U"UTF-32 string" UR"z(UTF-32 raw string)z"

```
const char* p = R"(a\
b
c)";
assert(std::strcmp(p, "a\\nb\\nc") == 0);
```

— end note]

5 [Example 1: The raw string

```
R"a(
)\
a"
)a"
```

is equivalent to "\n)\\na\"n". The raw string

```
R"(x = "\"y\"")"
```

is equivalent to "x = \"\\\"y\\\"\"". — end example]

6 Ordinary string literals and UTF-8 string literals are also referred to as *narrow string literals*.

7 The *string-literals* in any sequence of adjacent *string-literals* shall have at most one unique *encoding-prefix* among them. The common *encoding-prefix* of the sequence is that *encoding-prefix*, if any.

[Note 3: A *string-literal*'s rawness has no effect on the determination of the common *encoding-prefix*. — end note]

8 In translation phase 5 (5.2), adjacent *string-literals* are concatenated. The lexical structure and grouping of the contents of the individual *string-literals* is retained.

[Example 2:

```
"\xA" "B"
```

represents the code unit '\xA' and the character 'B' after concatenation (and not the single code unit '\xAB'). Similarly,

```
R"(\u00)" "41"
```

represents six characters, starting with a backslash and ending with the digit 1 (and not the single character 'A' specified by a *universal-character-name*).

Table 13 has some examples of valid concatenations. — end example]

Table 13 — String literal concatenations [tab:lex.string.concat]

Source	Means	Source	Means	Source	Means
u"a" u"b"	u"ab"	U"a" U"b"	U"ab"	L"a" L"b"	L"ab"
u"a" "b"	u"ab"	U"a" "b"	U"ab"	L"a" "b"	L"ab"
"a" u"b"	u"ab"	"a" U"b"	U"ab"	"a" L"b"	L"ab"

9 Evaluating a *string-literal* results in a string literal object with static storage duration (6.8.6).

[*Note 4*: String literal objects are potentially non-unique (6.8.2). Whether successive evaluations of a *string-literal* yield the same or a different object is unspecified. — *end note*]

[*Note 5*: The effect of attempting to modify a string literal object is undefined. — *end note*]

- 10 String literal objects are initialized with the sequence of code unit values corresponding to the *string-literal*'s sequence of *s-chars* (originally from non-raw string literals) and *r-chars* (originally from raw string literals), plus a terminating U+0000 NULL character, in order as follows:

- (10.1) — The sequence of characters denoted by each contiguous sequence of *basic-s-chars*, *r-chars*, *simple-escape-sequences* (5.13.3), and *universal-character-names* (5.3.1) is encoded to a code unit sequence using the *string-literal*'s associated character encoding. If a character lacks representation in the associated character encoding, then the program is ill-formed.

[*Note 6*: No character lacks representation in any Unicode encoding form. — *end note*]

When encoding a stateful character encoding, implementations should encode the first such sequence beginning with the initial encoding state and encode subsequent sequences beginning with the final encoding state of the prior sequence.

[*Note 7*: The encoded code unit sequence can differ from the sequence of code units that would be obtained by encoding each character independently. — *end note*]

- (10.2) — Each *numeric-escape-sequence* (5.13.3) contributes a single code unit with a value as follows:
- (10.2.1) — Let v be the integer value represented by the octal number comprising the sequence of *octal-digits* in an *octal-escape-sequence* or by the hexadecimal number comprising the sequence of *hexadecimal-digits* in a *hexadecimal-escape-sequence*.
- (10.2.2) — If v does not exceed the range of representable values of the *string-literal*'s array element type, then the value is v .
- (10.2.3) — Otherwise, if the *string-literal*'s *encoding-prefix* is absent or L, and v does not exceed the range of representable values of the corresponding unsigned type for the underlying type of the *string-literal*'s array element type, then the value is the unique value of the *string-literal*'s array element type T that is congruent to v modulo 2^N , where N is the width of T.
- (10.2.4) — Otherwise, the program is ill-formed.

When encoding a stateful character encoding, these sequences should have no effect on encoding state.

- (10.3) — Each *conditional-escape-sequence* (5.13.3) contributes an implementation-defined code unit sequence. When encoding a stateful character encoding, it is implementation-defined what effect these sequences have on encoding state.

5.13.6 Unevaluated strings

[lex.string.uneval]

unevaluated-string:
string-literal

- 1 An *unevaluated-string* shall have no *encoding-prefix*.
- 2 Each *universal-character-name* and each *simple-escape-sequence* in an *unevaluated-string* is replaced by the member of the translation character set it denotes. An *unevaluated-string* that contains a *numeric-escape-sequence* or a *conditional-escape-sequence* is ill-formed.
- 3 An *unevaluated-string* is never evaluated and its interpretation depends on the context in which it appears.

5.13.7 Boolean literals

[lex.bool]

boolean-literal:
false
true

- 1 The Boolean literals are the keywords **false** and **true**. Such literals have type **bool**.

5.13.8 Pointer literals

[lex.nullptr]

pointer-literal:
nullptr

- 1 The pointer literal is the keyword **nullptr**. It has type **std::nullptr_t**.

[*Note 1: `std::nullptr_t` is a distinct type that is neither a pointer type nor a pointer-to-member type; rather, a prvalue of this type is a null pointer constant and can be converted to a null pointer value or null member pointer value. See 7.3.12 and 7.3.13. — end note*]

5.13.9 User-defined literals

[lex.ext]

user-defined-literal:

user-defined-integer-literal
user-defined-floating-point-literal
user-defined-string-literal
user-defined-character-literal

user-defined-integer-literal:

decimal-literal ud-suffix
octal-literal ud-suffix
hexadecimal-literal ud-suffix
binary-literal ud-suffix

user-defined-floating-point-literal:

fractional-constant exponent-part_{opt} ud-suffix
digit-sequence exponent-part ud-suffix
hexadecimal-prefix hexadecimal-fractional-constant binary-exponent-part ud-suffix
hexadecimal-prefix hexadecimal-digit-sequence binary-exponent-part ud-suffix

user-defined-string-literal:

string-literal ud-suffix

user-defined-character-literal:

character-literal ud-suffix

ud-suffix:

identifier

- ¹ If a token matches both *user-defined-literal* and another *literal* kind, it is treated as the latter.

[*Example 1: `123_km` is a *user-defined-literal*, but `12LL` is an *integer-literal*. — end example*]

The syntactic non-terminal preceding the *ud-suffix* in a *user-defined-literal* is taken to be the longest sequence of characters that could match that non-terminal.

- ² A *user-defined-literal* is treated as a call to a literal operator or literal operator template (12.6). To determine the form of this call for a given *user-defined-literal* *L* with *ud-suffix* *X*, first let *S* be the set of declarations found by unqualified lookup for the *literal-operator-id* whose literal suffix identifier is *X* (6.5.3). *S* shall not be empty.
- ³ If *L* is a *user-defined-integer-literal*, let *n* be the literal without its *ud-suffix*. If *S* contains a literal operator with parameter type `unsigned long long`, the literal *L* is treated as a call of the form

`operator ""X(nULL)`

Otherwise, *S* shall contain a raw literal operator or a numeric literal operator template (12.6) but not both. If *S* contains a raw literal operator, the literal *L* is treated as a call of the form

`operator ""X("n")`

Otherwise (*S* contains a numeric literal operator template), *L* is treated as a call of the form

`operator ""X<'c1', 'c2', ... 'ck'>()`

where *n* is the source character sequence *c₁c₂...c_k*.

[*Note 1: The sequence *c₁c₂...c_k* can only contain characters from the basic character set. — end note*]

- ⁴ If *L* is a *user-defined-floating-point-literal*, let *f* be the literal without its *ud-suffix*. If *S* contains a literal operator with parameter type `long double`, the literal *L* is treated as a call of the form

`operator ""X(fL)`

Otherwise, *S* shall contain a raw literal operator or a numeric literal operator template (12.6) but not both. If *S* contains a raw literal operator, the literal *L* is treated as a call of the form

`operator ""X("f")`

Otherwise (*S* contains a numeric literal operator template), *L* is treated as a call of the form

`operator ""X<'c1', 'c2', ... 'ck'>()`

where f is the source character sequence $c_1c_2...c_k$.

[*Note 2*: The sequence $c_1c_2...c_k$ can only contain characters from the basic character set. — *end note*]

- ⁵ If L is a *user-defined-string-literal*, let str be the literal without its *ud-suffix* and let len be the number of code units in str (i.e., its length excluding the terminating null character). If S contains a literal operator template with a constant template parameter for which str is a well-formed *template-argument*, the literal L is treated as a call of the form

```
operator ""X<str>()
```

Otherwise, the literal L is treated as a call of the form

```
operator ""X(str, len)
```

- ⁶ If L is a *user-defined-character-literal*, let ch be the literal without its *ud-suffix*. S shall contain a literal operator (12.6) whose only parameter has the type of ch and the literal L is treated as a call of the form

```
operator ""X(ch)
```

- ⁷ [*Example 2*:

```
long double operator ""_w(long double);
std::string operator ""_w(const char16_t*, std::size_t);
unsigned operator ""_w(const char*);
int main() {
    1.2_w;           // calls operator ""_w(1.2L)
    u"one"_w;        // calls operator ""_w(u"one", 3)
    12_w;            // calls operator ""_w("12")
    "two"_w;         // error: no applicable literal operator
}
```

— *end example*]

- ⁸ In translation phase 5 (5.2), adjacent *string-literals* are concatenated and *user-defined-string-literals* are considered *string-literals* for that purpose. During concatenation, *ud-suffixes* are removed and ignored and the concatenation process occurs as described in 5.13.5. At the end of phase 5, if a *string-literal* is the result of a concatenation involving at least one *user-defined-string-literal*, all the participating *user-defined-string-literals* shall have the same *ud-suffix* and that suffix is applied to the result of the concatenation.

- ⁹ [*Example 3*:

```
int main() {
    L"A" "B" "C"_x; // OK, same as L"ABC"_x
    "P"_x "Q" "R"_y; // error: two different ud-suffixes
}
```

— *end example*]

6 Basics

[basic]

6.1 Preamble

[basic.pre]

¹ [Note 1: This Clause presents the basic concepts of the C++ language. It explains the difference between an object and a name and how they relate to the value categories for expressions. It introduces the concepts of a declaration and a definition and presents C++'s notion of type, scope, linkage, and storage duration. The mechanisms for starting and terminating a program are discussed. Finally, this Clause presents the fundamental types of the language and lists the ways of constructing compound types from these. — end note]

² [Note 2: This Clause does not cover concepts that affect only a single part of the language. Such concepts are discussed in the relevant Clauses. — end note]

³ A *name* is

- (3.1) — an *identifier* token (5.10, 5.11) other than
 - (3.1.1) — the *identifier* of a *label* (8.2) or *literal-operator-id* (12.6),
 - (3.1.2) — the *identifier* following a *goto* in a *jump-statement* (8.8.1),
 - (3.1.3) — any *identifier* in a *module-name* (10.1) or *attribute-token* (9.13.1), or
- (3.2) — a *conversion-function-id* (11.4.8.3),
- (3.3) — an *operator-function-id* (12.4), or
- (3.4) — a *literal-operator-id* (12.6).

⁴ Two names are *the same* if

- (4.1) — they are *identifiers* composed of the same character sequence, or
- (4.2) — they are *conversion-function-ids* formed with equivalent (13.7.7.2) types, or
- (4.3) — they are *operator-function-ids* formed with the same operator, or
- (4.4) — they are *literal-operator-ids* formed with the same literal suffix identifier.

⁵ Every name is introduced by a *declaration*, which is a

- (5.1) — *name-declaration*, *block-declaration*, or *member-declaration* (9.1, 11.4),
- (5.2) — *init-declarator* (9.3),
- (5.3) — *identifier* in a structured binding declaration (9.7),
- (5.4) — *identifier* in a *result-name-introducer* in a postcondition assertion (9.4.2),
- (5.5) — *init-capture* (7.5.6.3),
- (5.6) — *condition* with a *declarator* (8.1),
- (5.7) — *member-declarator* (11.4),
- (5.8) — *using-declarator* (9.10),
- (5.9) — *parameter-declaration* (9.3.4.6, 13.2),
- (5.10) — *type-parameter* (13.2),
- (5.11) — *type-tt-parameter* (13.2),
- (5.12) — *variable-tt-parameter* (13.2),
- (5.13) — *concept-tt-parameter* (13.2),
- (5.14) — *elaborated-type-specifier* that introduces a name (9.2.9.5),
- (5.15) — *class-specifier* (11.1),
- (5.16) — *enum-specifier* or *enumerator-definition* (9.8.1),
- (5.17) — *exception-declaration* (14.1), or
- (5.18) — implicit declaration of an injected-class-name (11.1).

[Note 3: The term *declaration* is not a synonym for the grammar non-terminal *declaration* (9.1). — end note]

[*Note 4*: The interpretation of a *for-range-declaration* produces one or more of the above (8.6.5). — *end note*]

- ⁶ [*Note 5*: Some names denote types or templates. In general, whenever a name is encountered it is necessary to look it up (6.5) to determine whether that name denotes one of these entities before continuing to parse the program that contains it. — *end note*]

⁷ A *variable* is introduced by the declaration *D* of

(7.1) — a reference other than a non-static data member or

(7.2) — an object,

where *D* is not the *parameter-declaration* of a *template-parameter*.

- ⁸ An *entity* is a variable, structured binding, result binding, function, enumerator, type, type alias, non-static data member, bit-field, template, namespace, namespace alias, template parameter, function parameter, or *init-capture*. The *underlying entity* of an entity is that entity unless otherwise specified. A name *denotes* the underlying entity of the entity declared by each declaration that introduces the name.

[*Note 6*: Type aliases and namespace aliases have underlying entities that are distinct from themselves. — *end note*]

- ⁹ A *local entity* is a variable with automatic storage duration (6.8.6.4), a structured binding (9.7) whose corresponding variable is such an entity, a result binding (9.4.2), or the `*this` object (7.5.3).

- ¹⁰ A name used in more than one translation unit can potentially refer to the same entity in these translation units depending on the linkage (6.7) of the name specified in each translation unit.

6.2 Declarations and definitions

[**basic.def**]

- ¹ A declaration (6.1) may (re)introduce one or more names and/or entities into a translation unit. If so, the declaration specifies the interpretation and semantic properties of these names. A declaration of an entity *X* is a *redeclaration* of *X* if another declaration of *X* is reachable from it (10.7); otherwise, it is a *first declaration*.

[*Note 1*: A declaration can also have effects including:

(1.1) — a static assertion (9.1),

(1.2) — controlling template instantiation (13.9.3),

(1.3) — guiding template argument deduction for constructors (13.7.2.3),

(1.4) — use of attributes (9.13), and

(1.5) — nothing (in the case of an *empty-declaration*).

— *end note*]

- ² Each entity declared by a declaration is also *defined* by that declaration unless:

(2.1) — it declares a function without specifying the function's body (9.6),

(2.2) — it contains the `extern` specifier (9.2.2) or a *linkage-specification*¹² (9.12) and neither an *initializer* nor a *function-body*,

(2.3) — it declares a non-inline static data member in a class definition (11.4, 11.4.9),

(2.4) — it declares a static data member outside a class definition and the variable was defined within the class with the `constexpr` specifier (11.4.9.3) (this usage is deprecated; see D.7),

(2.5) — it is an *elaborated-type-specifier* (11.3),

(2.6) — it is an *opaque-enum-declaration* (9.8.1),

(2.7) — it is a *template-parameter* (13.2),

(2.8) — it is a *parameter-declaration* (9.3.4.6) in a function declarator that is not the *declarator* of a *function-definition*,

(2.9) — it is a `typedef` declaration (9.2.4),

(2.10) — it is an *alias-declaration* (9.2.4),

(2.11) — it is a *namespace-alias-definition* (9.9.3),

(2.12) — it is a *using-declaration* (9.10),

¹² Appearing inside the brace-enclosed *declaration-seq* in a *linkage-specification* does not affect whether a declaration is a definition.

- (2.13) — it is a *deduction-guide* (13.7.2.3),
- (2.14) — it is a *static_assert-declaration* (9.1),
- (2.15) — it is a *constexpr-block-declaration*,
- (2.16) — it is an *attribute-declaration* (9.1),
- (2.17) — it is an *empty-declaration* (9.1),
- (2.18) — it is a *using-directive* (9.9.4),
- (2.19) — it is a *using-enum-declaration* (9.8.2),
- (2.20) — it is a *template-declaration* (13.1) whose *template-head* is not followed by either a *concept-definition* or a *declaration* that defines a function, a class, a variable, or a static data member,
- (2.21) — it is an explicit instantiation declaration (13.9.3), or
- (2.22) — it is an *explicit-specialization* (13.9.4) whose *declaration* is not a definition.

A declaration is said to be a *definition* of each entity that it defines.

[Example 1: All but one of the following are definitions:

```
int a;                // defines a
extern const int c = 1; // defines c
int f(int x) { return x+a; } // defines f and defines x
struct S { int a; int b; }; // defines S, S::a, and S::b
struct X {            // defines X
    int x;             // defines non-static data member x
    static int y;      // declares static data member y
    X(): x(0) { }      // defines a constructor of X
};
int X::y = 1;         // defines X::y
enum { up, down };    // defines up and down
namespace N { int d; } // defines N and N::d
X anX;                // defines anX
```

whereas these are just declarations:

```
extern int a;          // declares a
extern const int c;    // declares c
int f(int);            // declares f
struct S;              // declares S
typedef int Int;        // declares Int
namespace N1 = N;      // declares N1
extern X anotherX;     // declares anotherX
using N::d;            // declares d
```

— end example]

- ³ [Note 2: In some circumstances, C++ implementations implicitly define the default constructor (11.4.5.2), copy constructor (11.4.5.3), copy assignment operator, move assignment operator (11.4.6), or destructor (11.4.7) member functions. — end note]

[Example 2: Given

```
#include <string>

struct C {
    std::string s;           // std::string is the standard library class (27.4)
};

int main() {
    C a;
    C b = a;
    b = a;
}
```

the implementation will implicitly define functions to make the definition of `C` equivalent to

```
struct C {
    std::string s;
    C() : s() { }
```

```

C(const C& x): s(x.s) { }
C(C&& x): s(static_cast<std::string&&>(x.s)) { }
// : s(std::move(x.s)) { }
C& operator=(const C& x) { s = x.s; return *this; }
C& operator=(C&& x) { s = static_cast<std::string&&>(x.s); return *this; }
// { s = std::move(x.s); return *this; }
~C() { }
};

```

— end example]

⁴ [Note 3: A class name can also be implicitly declared by an *elaborated-type-specifier* (9.2.9.5). — end note]

⁵ In the definition of an object, the type of that object shall not be an incomplete type (6.9.1), an abstract class type (11.7.4), or a (possibly multidimensional) array thereof.

6.3 One-definition rule

[basic.def.odr]

¹ Each of the following is termed a *definable item*:

- (1.1) — a class type (Clause 11),
- (1.2) — an enumeration type (9.8.1),
- (1.3) — a function (9.3.4.6),
- (1.4) — a variable (6.1),
- (1.5) — a templated entity (13.1),
- (1.6) — a default argument for a parameter (for a function in a given scope) (9.3.4.7), or
- (1.7) — a default template argument (13.2).

² No translation unit shall contain more than one definition of any definable item.

³ An expression or conversion is *potentially evaluated* unless it is an unevaluated operand (7.2.3), a subexpression thereof, or a conversion in an initialization or conversion sequence in such a context. The set of *potential results* of an expression E is defined as follows:

- (3.1) — If E is an *id-expression* (7.5.5) or a *splice-expression* (7.5.9), the set contains only E .
- (3.2) — If E is a subscripting operation (7.6.1.2) with an array operand, the set contains the potential results of that operand.
- (3.3) — If E is a class member access expression (7.6.1.5) of the form $E_1 . \text{template}_{opt} E_2$, where E_2 designates a non-static data member or a direct base class relationship, the set contains the potential results of E_1 .
- (3.4) — If E is a class member access expression naming a static data member, the set contains the *id-expression* designating the data member.
- (3.5) — If E is a pointer-to-member expression (7.6.4) of the form $E_1 . * E_2$, the set contains the potential results of E_1 .
- (3.6) — If E has the form (E_1) , the set contains the potential results of E_1 .
- (3.7) — If E is a glvalue conditional expression (7.6.16), the set is the union of the sets of potential results of the second and third operands.
- (3.8) — If E is a comma expression (7.6.20), the set contains the potential results of the right operand.
- (3.9) — Otherwise, the set is empty.

[Note 1: This set is a (possibly-empty) set of *id-expressions* and *splice-expressions*, each of which is either E or a subexpression of E .

[Example 1: In the following example, the set of potential results of the initializer of `n` contains the first `S::x` subexpression, but not the second `S::x` subexpression. The set of potential results of the initializer of `o` contains the subexpression `[:^S::x:]`.

```

struct S { static const int x = 0; };
const int &f(const int &r);
int n = b ? (1, S::x)           // S::x is not odr-used here
          : f(S::x);           // S::x is odr-used here, so a definition is required
int o = [:^S::x:];

```

— end example]

— end note]

⁴ A function is *named by* an expression or conversion as follows:

- (4.1) — A function is named by an expression or conversion if it is the selected member of an overload set (6.5, 12.2, 12.3) in an overload resolution performed as part of forming that expression or conversion, and either it is not a pure virtual function or the expression is an *id-expression* naming the function with an explicitly qualified name that does not form a pointer to member (7.6.2.2).

[Note 2: This covers taking the address of functions (7.3.4, 7.6.2.2), calls to named functions (7.6.1.3), operator overloading (Clause 12), user-defined conversions (11.4.8.3), allocation functions for *new-expressions* (7.6.2.8), as well as non-default initialization (9.5). A constructor selected to copy or move an object of class type is considered to be named by an expression or conversion even if the call is actually elided by the implementation (11.9.6).

— end note]

- (4.2) — A deallocation function for a class is named by a *new-expression* if it is the single matching deallocation function for the allocation function selected by overload resolution, as specified in 7.6.2.8.

- (4.3) — A deallocation function for a class is named by a *delete-expression* if it is the selected usual deallocation function as specified in 7.6.2.9 and 11.4.11.

⁵ A variable is named by an expression if the expression is an *id-expression* or *splice-expression* (7.5.9) that designates it. A variable *x* that is named by a potentially evaluated expression *N* that appears at a point *P* is *odr-used* by *N* unless

- (5.1) — *x* is a reference that is usable in constant expressions at *P* (7.7.4), or

- (5.2) — *N* is an element of the set of potential results of an expression *E*, where

- (5.2.1) — *E* is a discarded-value expression (7.2.3) to which the lvalue-to-rvalue conversion is not applied, or

- (5.2.2) — *x* is a non-volatile object that is usable in constant expressions at *P* and has no mutable subobjects, and

- (5.2.2.1) — *E* is a class member access expression (7.6.1.5) naming a non-static data member of reference type and whose object expression has non-volatile-qualified type, or

- (5.2.2.2) — the lvalue-to-rvalue conversion (7.3.2) is applied to *E* and *E* has non-volatile-qualified non-class type.

[Example 2:

```
int f(int);
int g(int&);
struct A {
    int x;
};
struct B {
    int& r;
};
int h(bool cond) {
    constexpr A a = {1};
    constexpr const volatile A& r = a;    // odr-uses a
    int _ = f(cond ? a.x : r.x);          // does not odr-use a or r
    int x, y;
    constexpr B b1 = {x}, b2 = {y};       // odr-uses x and y
    int _ = g(cond ? b1.r : b2.r);         // does not odr-use b1 or b2
    int _ = ((cond ? x : y), 0);           // does not odr-use x or y
    return [] {
        return b1.r;                      // error: b1 is odr-used here because the object
                                           // referred to by b1.r is not constexpr-referenceable here
    };
}
```

— end example]

⁶ A structured binding is named by an expression if that expression is either an *id-expression* or a *splice-expression* that designates that structured binding. A structured binding is *odr-used* if it is named by a potentially evaluated expression.

⁷ ***this** is *odr-used* if **this** appears as a potentially evaluated expression (including as the result of any implicit transformation to a class member access expression (7.5.5.1)).

- ⁸ A virtual member function is odr-used if it is not pure. A function is odr-used if it is named by a potentially evaluated expression or conversion. A non-placement allocation or deallocation function for a class is odr-used by the definition of a constructor of that class. A non-placement deallocation function for a class is odr-used by the definition of the destructor of that class, or by being selected by the lookup at the point of definition of a virtual destructor (11.4.7).¹³
- ⁹ An assignment operator function in a class is odr-used by an implicitly-defined copy assignment or move assignment function for another class as specified in 11.4.6. A constructor for a class is odr-used as specified in 9.5. A destructor for a class is odr-used if it is potentially invoked (11.4.7).
- ¹⁰ A local entity (6.1) is *odr-usable* in a scope (6.4.1) if
- (10.1) — either the local entity is not **this*, or an enclosing class or non-lambda function parameter scope exists and, if the innermost such scope is a function parameter scope, it corresponds to a non-static member function, and
 - (10.2) — for each intervening scope (6.4.1) between the point at which the entity is introduced and the scope (where **this* is considered to be introduced within the innermost enclosing class or non-lambda function parameter scope), either
 - (10.2.1) — the intervening scope is a block scope,
 - (10.2.2) — the intervening scope is a contract-assertion scope (6.4.10),
 - (10.2.3) — the intervening scope is the function parameter scope of a *lambda-expression* or *requires-expression*, or
 - (10.2.4) — the intervening scope is the lambda scope of a *lambda-expression* that has a *simple-capture* naming the entity or has a *capture-default*, and the block scope of the *lambda-expression* is also an intervening scope.

If a local entity is odr-used in a scope in which it is not odr-usable, the program is ill-formed.

[Example 3:

```
void f(int n) {
    [] { n = 1; };           // error: n is not odr-usable due to intervening lambda-expression
    struct A {
        void f() { n = 2; }   // error: n is not odr-usable due to intervening function parameter scope
    };
    void g(int = n);          // error: n is not odr-usable due to intervening function parameter scope
    [=](int k = n) {};        // error: n is not odr-usable due to being
                              // outside the block scope of the lambda-expression
    [&] { [n]{ return n; }; }; // OK
}
```

— end example]

¹¹ [Example 4:

```
void g() {
    constexpr int x = 1;
    auto lambda = [] <typename T, int = ((T)x, 0)> {}; // OK
    lambda.operator()<int, 1>(); // OK, does not consider x at all
    lambda.operator()<int>();    // OK, does not odr-use x
    lambda.operator()<const int&>(); // error: odr-uses x from a context where x is not odr-usable
}

void h() {
    constexpr int x = 1;
    auto lambda = [] <typename T> { (T)x; }; // OK
    lambda.operator()<int>(); // OK, does not odr-use x
    lambda.operator()<void>(); // OK, does not odr-use x
    lambda.operator()<const int&>(); // error: odr-uses x from a context where x is not odr-usable
}
```

— end example]

¹³ An implementation is not required to call allocation and deallocation functions from constructors or destructors; however, this is a permissible implementation technique.

- 12 Every program shall contain at least one definition of every function or variable that is odr-used in that program outside of a discarded statement (8.5.2); no diagnostic required (G.3.2). The definition can appear explicitly in the program, it can be found in the standard or a user-defined library, or (when appropriate) it is implicitly defined (see 11.4.5.2, 11.4.5.3, 11.4.7, and 11.4.6).

[Example 5:

```
auto f() {
    struct A {};
    return A{};
}
decltype(f()) g();
auto x = g();
```

A program containing this translation unit is ill-formed because `g` is odr-used but not defined, and cannot be defined in any other translation unit because the local class `A` cannot be named outside this translation unit. — end example]

- 13 A *definition domain* is a *private-module-fragment* or the portion of a translation unit excluding its *private-module-fragment* (if any). A definition of an inline function or variable shall be reachable from the end of every definition domain in which it is odr-used outside of a discarded statement.
- 14 A definition of a class shall be reachable in every context in which the class is used in a way that requires the class type to be complete.

[Example 6: The following complete translation unit is well-formed, even though it never defines `X`:

```
struct X;                // declare X as a struct type
struct X* x1;            // use X in pointer formation
X* x2;                   // use X in pointer formation
```

— end example]

[Note 3: The rules for declarations and expressions describe in which contexts complete class types are required. A class type `T` must be complete if

- (14.1) — an object of type `T` is defined (6.2), or
- (14.2) — a non-static class data member of type `T` is declared (11.4), or
- (14.3) — `T` is used as the allocated type or array element type in a *new-expression* (7.6.2.8), or
- (14.4) — an lvalue-to-rvalue conversion is applied to a glvalue referring to an object of type `T` (7.3.2), or
- (14.5) — an expression is converted (either implicitly or explicitly) to type `T` (7.3, 7.6.1.4, 7.6.1.7, 7.6.1.9, 7.6.3), or
- (14.6) — an expression that is not a null pointer constant, and has type other than *cv void**, is converted to the type pointer to `T` or reference to `T` using a standard conversion (7.3), a `dynamic_cast` (7.6.1.7) or a `static_cast` (7.6.1.9), or
- (14.7) — a class member access operator is applied to an expression of type `T` (7.6.1.5), or
- (14.8) — the `typeid` operator (7.6.1.8) or the `sizeof` operator (7.6.2.5) is applied to an operand of type `T`, or
- (14.9) — a function with a return type or argument type of type `T` is defined (6.2) or called (7.6.1.3), or
- (14.10) — a class with a base class of type `T` is defined (11.7), or
- (14.11) — an lvalue of type `T` is assigned to (7.6.19), or
- (14.12) — the type `T` is the subject of an `alignof` expression (7.6.2.6), or
- (14.13) — an *exception-declaration* has type `T`, reference to `T`, or pointer to `T` (14.4).

— end note]

- 15 If a definable item `D` is defined in a translation unit by an injected declaration `X` (7.7.6) and another translation unit contains a definition of `D`, that definition shall be an injected declaration having the same characteristic sequence as `X`; a diagnostic is required only if `D` is attached to a named module and a prior definition is reachable at the point where a later definition occurs (G.3.3).
- 16 For any other definable item `D` with definitions in multiple translation units,
- (16.1) — if `D` is a non-inline non-templated function or variable (G.3.4), or
 - (16.2) — if the definitions in different translation units do not satisfy the following requirements (G.3.5),

the program is ill-formed; a diagnostic is required only if the definable item is attached to a named module and a prior definition is reachable at the point where a later definition occurs. Given such an item, for all definitions of `D`, or, if `D` is an unnamed enumeration, for all definitions of `D` that are reachable at any given program point, the following requirements shall be satisfied.

- (16.3) — Each such definition shall not be attached to a named module (10.1).
 - (16.4) — Each such definition shall consist of the same sequence of tokens, where the definition of a closure type is considered to consist of the sequence of tokens of the corresponding *lambda-expression*.
 - (16.5) — In each such definition, corresponding names, looked up according to 6.5, shall denote the same entity, after overload resolution (12.2) and after matching of partial template specializations (13.7.6.2), except that a name can refer to
 - (16.5.1) — a non-volatile const object with internal or no linkage if the object
 - (16.5.1.1) — has the same literal type in all definitions of D,
 - (16.5.1.2) — is initialized with a constant expression (7.7.3),
 - (16.5.1.3) — is not odr-used in any definition of D, and
 - (16.5.1.4) — has the same value in all definitions of D,
 - or
 - (16.5.2) — a reference with internal or no linkage initialized with a constant expression such that the reference refers to the same object or function in all definitions of D.
 - (16.6) — In each such definition, except within the default arguments and default template arguments of D, corresponding *lambda-expressions* shall have the same closure type (see below).
 - (16.7) — In each such definition, corresponding entities shall have the same language linkage.
 - (16.8) — In each such definition, const objects with static or thread storage duration shall be constant-initialized if the object is constant-initialized in any such definition.
 - (16.9) — In each such definition, corresponding manifestly constant-evaluated expressions that are not value-dependent shall have the same value (7.7, 13.8.3.4).
 - (16.10) — In each such definition, the overloaded operators referred to, the implicit calls to conversion functions, constructors, operator new functions and operator delete functions, shall refer to the same function.
 - (16.11) — In each such definition, a default argument used by an (implicit or explicit) function call or a default template argument used by an (implicit or explicit) *template-id*, *simple-template-id*, or *splice-specialization-specifier* is treated as if its token sequence were present in the definition of D; that is, the default argument or default template argument is subject to the requirements described in this paragraph (recursively).
 - (16.12) — In each such definition, corresponding *reflect-expressions* (7.6.2.10) compute equivalent values (7.6.10).
- 17 For the purposes of the preceding requirements:
- (17.1) — If D is a class with an implicitly-declared constructor (11.4.5.2, 11.4.5.3), it is as if the constructor was implicitly defined in every translation unit where it is odr-used, and the implicit definition in every translation unit shall call the same constructor for a subobject of D.

[Example 7:

```
// translation unit 1:
struct X {
    X(int, int);
    X(int, int, int);
};
X::X(int, int = 0) { }
class D {
    X x = 0;
};
D d1;                                // X(int, int) called by D()

// translation unit 2:
struct X {
    X(int, int);
    X(int, int, int);
};
X::X(int, int = 0, int = 0) { }
```

```

class D {
    X x = 0;
};
D d2;                                     // X(int, int, int) called by D();
                                           // D()'s implicit definition violates the ODR
— end example]

```

(17.2) — If *D* is a class with a defaulted three-way comparison operator function (11.10.3), it is as if the operator was implicitly defined in every translation unit where it is odr-used, and the implicit definition in every translation unit shall call the same comparison operators for each subobject of *D*.

(17.3) — If *D* is a template and is defined in more than one translation unit, the requirements apply both to names from the template's enclosing scope used in the template definition, and also to dependent names at the point of instantiation (13.8.3).

18 These requirements also apply to corresponding entities defined within each definition of *D* (including the closure types of *lambda-expressions*, but excluding entities defined within default arguments or default template arguments of either *D* or an entity not defined within *D*). For each such entity and for *D* itself, the behavior is as if there is a single entity with a single definition, including in the application of these requirements to other entities.

[Note 4: The entity is still declared in multiple translation units, and 6.7 still applies to these declarations. In particular, *lambda-expressions* (7.5.6) appearing in the type of *D* can result in the different declarations having distinct types, and *lambda-expressions* appearing in a default argument of *D* might still denote different types in different translation units. — end note]

19 [Example 8:

```

inline void f(bool cond, void (*p)()) {
    if (cond) f(false, []{});
}
inline void g(bool cond, void (*p)() = []{}) {
    if (cond) g(false);
}
struct X {
    void h(bool cond, void (*p)() = []{}) {
        if (cond) h(false);
    }
};

```

If the definition of *f* appears in multiple translation units, the behavior of the program is as if there is only one definition of *f*. If the definition of *g* appears in multiple translation units, the program is ill-formed (no diagnostic required) because each such definition uses a default argument that refers to a distinct *lambda-expression* closure type. The definition of *X* can appear in multiple translation units of a valid program; the *lambda-expressions* defined within the default argument of *X::h* within the definition of *X* denote the same closure type in each translation unit. — end example]

20 If, at any point in the program, there is more than one reachable unnamed enumeration definition in the same scope that have the same first enumerator name and do not have typedef names for linkage purposes (9.8.1), those unnamed enumeration types shall be the same; no diagnostic required (G.3.6).

6.4 Scope

[basic.scope]

6.4.1 General

[basic.scope.scope]

1 The declarations in a program appear in a number of *scopes* that are in general discontinuous. The *global scope* contains the entire program; every other scope *S* is introduced by a declaration, *parameter-declaration-clause*, *statement*, *handler*, *lambda-expression*, or contract assertion (as described in the following subclauses of 6.4) appearing in another scope, which thereby contains *S*. An *enclosing scope* at a program point is any scope that contains it; the smallest such scope is said to be the *immediate scope* at that point. A scope *intervenes* between a program point *P* and a scope *S* (that does not contain *P*) if it is or contains *S* but does not contain *P*.

2 Unless otherwise specified:

- (2.1) — The smallest scope that contains a scope *S* is the *parent scope* of *S*.
- (2.2) — No two declarations (re)introduce the same entity.
- (2.3) — A declaration *inhabits* the immediate scope at its locus (6.4.2).

- (2.4) — A declaration's *target scope* is the scope it inhabits.
- (2.5) — Any names (re)introduced by a declaration are *bound* to it in its target scope.

The *host scope* of a declaration is the inhabited scope if that scope is a block scope and the target scope otherwise. An entity *belongs* to a scope *S* if *S* is the target scope of a declaration of the entity.

[Note 1: Special cases include that:

- (2.6) — Template parameter scopes are parents only to other template parameter scopes (6.4.9).
- (2.7) — Corresponding declarations with appropriate linkage declare the same entity (6.7).
- (2.8) — The *declaration* of a *template-declaration* inhabits the same scope as the *template-declaration*.
- (2.9) — Friend declarations and declarations of template specializations do not bind names (9.3.4); those with qualified names target a specified scope, and other friend declarations and certain *elaborated-type-specifiers* (9.2.9.5) target a larger enclosing scope.
- (2.10) — Block-scope extern or function declarations target a larger enclosing scope but bind a name in their immediate scope (9.3.4.1).
- (2.11) — The names of unscoped enumerators are bound in the two innermost enclosing scopes (9.8.1).
- (2.12) — A class's name is also bound in its own scope (11.1).
- (2.13) — The names of the members of an anonymous union are bound in the union's parent scope (11.5.2).

— *end note*]

³ Two non-static member functions have *corresponding object parameters* if

- (3.1) — exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters (9.3.4.6), after removing references, are the same, or
- (3.2) — their object parameters have the same type.

Two non-static member function templates have *corresponding object parameters* if

- (3.3) — exactly one is an implicit object member function with no *ref-qualifier* and the types of their object parameters, after removing any references, are equivalent, or
- (3.4) — the types of their object parameters are equivalent.

Two function templates have *corresponding signatures* if their *template-parameter-lists* have the same length, their corresponding *template-parameters* are equivalent, they have equivalent non-object-parameter-type-lists and return types (if any), and, if both are non-static members, they have corresponding object parameters.

⁴ Two declarations *correspond* if they (re)introduce the same name, both declare constructors, or both declare destructors, unless

- (4.1) — either is a *using-declarator*, or
- (4.2) — one declares a type (not a type alias) and the other declares a variable, non-static data member other than of an anonymous union (11.5.2), enumerator, function, or function template, or
- (4.3) — each declares a function or function template and they do not declare corresponding overloads.

Two function or function template declarations declare *corresponding overloads* if

- (4.4) — both declare functions with the same non-object-parameter-type-list,¹⁴ equivalent (13.7.7.2) trailing *requires-clauses* (if any, except as specified in 13.7.5), and, if both are non-static members, they have corresponding object parameters, or
- (4.5) — both declare function templates with corresponding signatures and equivalent *template-heads* and trailing *requires-clauses* (if any).

[Note 2: Declarations can correspond even if neither binds a name.

[Example 1:

```
struct A {
    friend void f();    // #1
};
struct B {
    friend void f() {}  // corresponds to, and defines, #1
};
```

¹⁴ An implicit object parameter (12.2.2) is not part of the parameter-type-list.

— end example]

— end note]

[Example 2:

```
typedef int Int;
enum E : int { a };
void f(int);           // #1
void f(Int) {};        // defines #1
void f(E) {};          // OK, another overload

struct X {
    static void f();
    void f() const;    // error: redeclaration
    void g();
    void g() const;    // OK
    void g() &;        // error: redeclaration

    void h(this X&, int);
    void h(int) &&;    // OK, another overload
    void j(this const X&);
    void j() const &; // error: redeclaration
    void k();
    void k(this X&);  // error: redeclaration
};
```

— end example]

- 5 A declaration is *name-independent* if its name is `_` (U+005F LOW LINE) and it declares
- (5.1) — a variable, other than a function parameter, with automatic storage duration,
 - (5.2) — a structured binding with no *storage-class-specifier* and not inhabiting a namespace scope,
 - (5.3) — a result binding (9.4.2),
 - (5.4) — the variable introduced by an *init-capture*, or
 - (5.5) — a non-static data member of other than an anonymous union.

Recommended practice: Implementations should not emit a warning that a name-independent declaration is used or unused.

- 6 Two declarations *potentially conflict* if they correspond and cause their shared name to denote different entities (6.7). The program is ill-formed if, in any scope, a name is bound to two declarations *A* and *B* that potentially conflict and *A* precedes *B* (6.5), unless *B* is name-independent.

[Note 3: An *id-expression* that names a unique name-independent declaration is usable until an additional declaration of the same name is introduced in the same scope (6.5.1). — end note]

[Note 4: Overload resolution can consider potentially conflicting declarations found in multiple scopes (e.g., via *using-directives* or for operator functions), in which case it is often ambiguous. — end note]

[Example 3:

```
void f() {
    int x,y;
    void x();    // error: different entity for x
    int y;       // error: redefinition
}

enum { f };      // error: different entity for ::f
namespace A {};
namespace B = A;
namespace B = A; // OK, no effect
namespace B = B; // OK, no effect
namespace A = B; // OK, no effect
namespace B {};  // error: different entity for B

void g() {
    int _;
    _ = 0;        // OK
    int _;        // OK, name-independent declaration
```

```

    _ = 0;                // error: two non-function declarations in the lookup set
}
void h () {
    int _;                // #1
    _ ++;                 // OK
    static int _;         // error: conflicts with #1 because static variables are not name-independent
}

```

— end example]

- ⁷ A declaration is *nominable* in a class, class template, or namespace *E* at a point *P* if it precedes *P*, it does not inhabit a block scope, and its target scope is the scope associated with *E* or, if *E* is a namespace, any element of the inline namespace set of *E* (9.9.2).

[Example 4:

```

namespace A {
    void f() {void g();}
    inline namespace B {
        struct S {
            friend void h();
            static int i;
        };
    }
}

```

At the end of this example, the declarations of *f*, *B*, *S*, and *h* are nominable in *A*, but those of *g* and *i* are not. — end example]

- ⁸ When instantiating a templated entity (13.1), any scope *S* introduced by any part of the template definition is considered to be introduced by the instantiated entity and to contain the instantiations of any declarations that inhabit *S*.

6.4.2 Point of declaration

[basic.scope.pdecl]

- ¹ The *locus* of a declaration (6.1) that is a declarator is immediately after the complete declarator (9.3).

[Example 1:

```

unsigned char x = 12;
{ unsigned char x = x; }

```

Here, the initialization of the second *x* has undefined behavior, because the initializer accesses the second *x* outside its lifetime (6.8.4). — end example]

- ² [Note 1: A name from an outer scope remains visible up to the locus of the declaration that hides it.

[Example 2:

```

const int i = 2;
{ int i[i]; }

```

declares a block-scope array of two integers. — end example]

— end note]

- ³ The locus of a *class-specifier* is immediately after the *identifier* or *simple-template-id* (if any) in its *class-head* (11.1). The locus of an *enum-specifier* is immediately after its *enum-head*; the locus of an *opaque-enum-declaration* is immediately after it (9.8.1). The locus of an *alias-declaration* is immediately after it.
- ⁴ The locus of a *using-declarator* that does not name a constructor is immediately after the *using-declarator* (9.10).
- ⁵ The locus of an *enumerator-definition* is immediately after it.

[Example 3:

```

const int x = 12;
{ enum { x = x }; }

```

Here, the enumerator *x* is initialized with the value of the constant *x*, namely 12. — end example]

- ⁶ [Note 2: After the declaration of a class member, the member name can be found in the scope of its class even if the class is an incomplete class.

[Example 4:

```

struct X {
    enum E { z = 16 };
    int b[X::z];          // OK
};
— end example]
— end note]

```

- 7 The locus of an *elaborated-type-specifier* that is a declaration (9.2.9.5) is immediately after it.
- 8 The locus of an injected-class-name declaration (11.1) is immediately following the opening brace of the class definition.
- 9 The locus of the implicit declaration of a function-local predefined variable (9.6.1) is immediately before the *function-body* of its function's definition.
- 10 The locus of the declaration of a structured binding (9.7) is immediately after the *identifier-list* of the structured binding declaration.
- 11 The locus of a *for-range-declaration* of a range-based **for** statement (8.6.5) is immediately after the *for-range-initializer*. The locus of a *for-range-declaration* of an *expansion-statement* (8.7) is immediately after the *expansion-initializer*.
- 12 The locus of a *template-parameter* is immediately after it.

[Example 5:

```

typedef unsigned char T;
template<class T
    = T                // lookup finds the typedef-name
    , T                // lookup finds the template parameter
    N = 0> struct A { };
— end example]

```

- 13 The locus of a *result-name-introducer* (9.4.2) is immediately after it.
- 14 The locus of a *concept-definition* is immediately after its *concept-name* (13.7.9).
[Note 3: The *constraint-expression* cannot use the *concept-name*. — end note]
- 15 The locus of a *namespace-definition* with an *identifier* is immediately after the *identifier*.
[Note 4: An identifier is invented for an *unnamed-namespace-definition* (9.9.2.2). — end note]
- 16 [Note 5: Friend declarations can introduce functions or classes that belong to the nearest enclosing namespace or block scope, but they do not bind names anywhere (11.8.4). Function declarations at block scope and variable declarations with the **extern** specifier at block scope declare entities that belong to the nearest enclosing namespace, but they do not bind names in it. — end note]
- 17 [Note 6: For point of instantiation of a template, see 13.8.4.1. — end note]

6.4.3 Block scope

[basic.scope.block]

- 1 Each
 - (1.1) — selection, iteration, or expansion statement (8.5, 8.6, 8.7),
 - (1.2) — substatement of such a statement,
 - (1.3) — *handler* (14.1), or
 - (1.4) — compound statement (8.4) that is not the *compound-statement* of a *handler*

introduces a *block scope* that includes that statement or *handler*.

[Note 1: A substatement that is also a block has only one scope. — end note]

A variable that belongs to a block scope is a *block variable*.

[Example 1:

```

int i = 42;
int a[10];

for (int i = 0; i < 10; i++)
    a[i] = i;

```

```
int j = i;           // j = 42
— end example]
```

- ² If a declaration that is not a name-independent declaration and that binds a name in the block scope *S* of a
- (2.1) — *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
 - (2.2) — substatement of a selection, iteration, or expansion statement that is not itself a selection, iteration, or expansion statement, or
 - (2.3) — *handler* of a *function-try-block*

potentially conflicts with a declaration whose target scope is the parent scope of *S*, the program is ill-formed.

[Example 2:

```
if (int x = f()) {
    int x;           // error: redeclaration of x
}
else {
    int x;           // error: redeclaration of x
}
— end example]
```

6.4.4 Function parameter scope

[basic.scope.param]

- ¹ A *parameter-declaration-clause* *P* introduces a *function parameter scope* that includes *P*.

[Note 1: A function parameter cannot be used for its value within the *parameter-declaration-clause* (9.3.4.7). — end note]

- (1.1) — If *P* is associated with a *declarator* and is preceded by a (possibly-parenthesized) *nopttr-declarator* of the form *declarator-id attribute-specifier-seq_{opt}*, its scope extends to the end of the nearest enclosing *init-declarator*, *member-declarator*, *declarator* of a *parameter-declaration* or a *nodeclspec-function-declaration*, or *function-definition*, but does not include the locus of the associated *declarator*.

[Note 2: In this case, *P* declares the parameters of a function (or a function or template parameter declared with function type). A member function's parameter scope is nested within its class's scope. — end note]

- (1.2) — If *P* is associated with a *lambda-declarator*, its scope extends to the end of the *compound-statement* of the *lambda-expression*.
- (1.3) — If *P* is associated with a *requirement-parameter-list*, its scope extends to the end of the *requirement-body* of the *requires-expression*.
- (1.4) — If *P* is associated with a *deduction-guide*, its scope extends to the end of the *deduction-guide*.

6.4.5 Lambda scope

[basic.scope.lambda]

A *lambda-expression* *E* introduces a *lambda scope* that starts immediately after the *lambda-introducer* of *E* and extends to the end of the *compound-statement* of *E*.

6.4.6 Namespace scope

[basic.scope.namespace]

- ¹ Any *namespace-definition* for a namespace *N* introduces a *namespace scope* that includes the *namespace-body* for every *namespace-definition* for *N*. For each non-friend redeclaration or specialization whose target scope is or is contained by the scope, the portion after the *declarator-id*, *class-head-name*, or *enum-head-name* is also included in the scope. The global scope is the namespace scope of the global namespace (9.9).

[Example 1:

```
namespace Q {
    namespace V { void f(); }
    void V::f() {           // in the scope of V
        void h();           // declares Q::V::h
    }
}
— end example]
```

6.4.7 Class scope

[basic.scope.class]

- ¹ Any declaration of a class or class template *C* introduces a *class scope* that includes the *member-specification* of the *class-specifier* for *C* (if any). For each non-friend redeclaration or specialization whose target scope is or is contained by the scope, the portion after the *declarator-id*, *class-head-name*, or *enum-head-name* is also included in the scope.

[Note 1: Lookup from a program point before the *class-specifier* of a class will find no bindings in the class scope.

[Example 1:

```
template<class D>
struct B {
    D::type x;           // #1
};

struct A { using type = int; };
struct C : A, B<C> {}; // error at #1: C::type not found
```

— end example]

— end note]

6.4.8 Enumeration scope

[basic.scope.enum]

- ¹ Any declaration of an enumeration *E* introduces an *enumeration scope* that includes the *enumerator-list* of the *enum-specifier* for *E* (if any).

6.4.9 Template parameter scope

[basic.scope.temp]

- ¹ Each *type-tt-parameter*, *variable-tt-parameter*, and *concept-tt-parameter* introduces a *template parameter scope* that includes the *template-head* of the *template-parameter*.
- ² Each *template-declaration* *D* introduces a template parameter scope that extends from the beginning of its *template-parameter-list* to the end of the *template-declaration*. Any declaration outside the *template-parameter-list* that would inhabit that scope instead inhabits the same scope as *D*. The parent scope of any scope *S* that is not a template parameter scope is the smallest scope that contains *S* and is not a template parameter scope.

[Note 1: Therefore, only template parameters belong to a template parameter scope, and only template parameter scopes have a template parameter scope as a parent scope. — end note]

6.4.10 Contract-assertion scope

[basic.scope.contract]

- ¹ Each contract assertion (6.11) *C* introduces a *contract-assertion scope* that includes *C*.
- ² If a *result-name-introducer* (9.4.2) that is not name-independent (6.4.1) and whose enclosing postcondition assertion is associated with a function *F* potentially conflicts with a declaration whose target scope is
 - (2.1) — the function parameter scope of *F* or
 - (2.2) — if associated with a *lambda-declarator*, the nearest enclosing lambda scope of the postcondition assertion (7.5.6),

the program is ill-formed.

6.5 Name lookup

[basic.lookup]

6.5.1 General

[basic.lookup.general]

- ¹ *Name lookup* associates the use of a name with a set of declarations (6.2) of that name. The name lookup rules apply uniformly to all names (including *typedef-names* (9.2.4), *namespace-names* (9.9), and *class-names* (11.3)) wherever the grammar allows such names in the context discussed by a particular rule. Unless otherwise specified, the program is ill-formed if no declarations are found. If the declarations found by name lookup all denote functions or function templates, the declarations are said to form an *overload set*. Otherwise, if the declarations found by name lookup do not all denote the same entity, they are *ambiguous* and the program is ill-formed. Overload resolution (12.2, 12.3) takes place after name lookup has succeeded. The access rules (11.8) are considered only once name lookup and function overload resolution (if applicable) have succeeded. Only after name lookup, function overload resolution (if applicable) and access checking have succeeded are the semantic properties introduced by the declarations used in further processing.

- ² There is a *program point* before the first token of the translation unit, at least one between every pair of adjacent tokens, and at least one after the last token of the translation unit.

A program point *P* is said to *follow* any declaration in the same translation unit whose locus (6.4.2) is before *P*.

[Note 1: The declaration might appear in a scope that does not contain *P*. — end note]

A declaration *X* *precedes* a program point *P* in a translation unit *L* if *P* follows *X*, *X* inhabits a class scope and is reachable from *P*, or else *X* appears in a translation unit *D* and

- (2.1) — *P* follows a *module-import-declaration* or *module-declaration* that imports *D* (directly or indirectly), and
- (2.2) — *X* appears after the *module-declaration* in *D* (if any) and before the *private-module-fragment* in *D* (if any), and
- (2.3) — either *X* is exported or else *D* and *L* are part of the same module and *X* does not inhabit a namespace with internal linkage or declare a name with internal linkage.

[Note 2: Names declared by a *using-declaration* have no linkage. — end note]

[Note 3: A *module-import-declaration* imports both the named translation unit(s) and any modules named by exported *module-import-declarations* within them, recursively.

[Example 1:

Translation unit #1:

```
export module Q;
export int sq(int i) { return i*i; }
```

Translation unit #2:

```
export module R;
export import Q;
```

Translation unit #3:

```
import R;
int main() { return sq(9); }    // OK, sq from module Q
```

— end example]

— end note]

- ³ A *single search* in a scope *S* for a name *N* from a program point *P* finds all declarations that precede *P* to which any name that is the same as *N* (6.1) is bound in *S*. If any such declaration is a *using-declarator* whose terminal name (7.5.5.2) is not dependent (13.8.3.2), it is replaced by the declarations named by the *using-declarator* (9.10).
- ⁴ In certain contexts, only certain kinds of declarations are included. After any such restriction, any declarations of classes or enumerations are discarded if any other declarations are found.

[Note 4: A type (but not a type alias or template) is therefore hidden by any other entity in its scope. — end note]

However, if a lookup is *type-only*, only declarations of types and templates whose specializations are types are considered; furthermore, if declarations of a type alias and of its underlying entity are found, the declaration of the type alias is discarded instead of the type declaration.

6.5.2 Member name lookup

[class.member.lookup]

- ¹ A *search* in a scope *X* for a name *M* from a program point *P* is a single search in *X* for *M* from *P* unless *X* is the scope of a class or class template *T*, in which case the following steps define the result of the search.

[Note 1: The result differs only if *M* is a *conversion-function-id* or if the single search would find nothing. — end note]

- ² The *lookup set* for a name *N* in a class or class template *C*, called *S(N, C)*, consists of two component sets: the *declaration set*, a set of members named *N*; and the *subobject set*, a set of subobjects where declarations of these members were found (possibly via *using-declarations*). In the declaration set, type declarations (including injected-class-names) are replaced by the types they designate. *S(N, C)* is calculated as follows:
- ³ The declaration set is the result of a single search in the scope of *C* for *N* from immediately after the *class-specifier* of *C* if *P* is in a complete-class context of *C* or from *P* otherwise. If the resulting declaration set is not empty, the subobject set contains *C* itself, and calculation is complete.

- ⁴ Otherwise (i.e., C does not contain a declaration of N or the resulting declaration set is empty), $S(N, C)$ is initially empty. Calculate the lookup set for N in each direct non-dependent (13.8.3.2) base class subobject B_i , and merge each such lookup set $S(N, B_i)$ in turn into $S(N, C)$.

[Note 2: If C is incomplete, only base classes whose *base-specifier* appears before P are considered. If C is an instantiated class, its base classes are not dependent. — end note]

- ⁵ The following steps define the result of merging lookup set $S(N, B_i)$ into the intermediate $S(N, C)$:
- (5.1) — If each of the subobject members of $S(N, B_i)$ is a base class subobject of at least one of the subobject members of $S(N, C)$, or if $S(N, B_i)$ is empty, $S(N, C)$ is unchanged and the merge is complete. Conversely, if each of the subobject members of $S(N, C)$ is a base class subobject of at least one of the subobject members of $S(N, B_i)$, or if $S(N, C)$ is empty, the new $S(N, C)$ is a copy of $S(N, B_i)$.
 - (5.2) — Otherwise, if the declaration sets of $S(N, B_i)$ and $S(N, C)$ differ, the merge is ambiguous: the new $S(N, C)$ is a lookup set with an invalid declaration set and the union of the subobject sets. In subsequent merges, an invalid declaration set is considered different from any other.
 - (5.3) — Otherwise, the new $S(N, C)$ is a lookup set with the shared set of declarations and the union of the subobject sets.
- ⁶ The result of the search is the declaration set of $S(M, T)$. If it is an invalid set, the program is ill-formed. If it differs from the result of a search in T for M in a complete-class context (11.4) of T , the program is ill-formed, no diagnostic required (G.3.9).

[Example 1:

```

struct A { int x; };           // S(x,A) = { { A::x }, { A } }
struct B { float x; };        // S(x,B) = { { B::x }, { B } }
struct C: public A, public B { }; // S(x,C) = { invalid, { A in C, B in C } }
struct D: public virtual C { }; // S(x,D) = S(x,C)
struct E: public virtual C { char x; }; // S(x,E) = { { E::x }, { E } }
struct F: public D, public E { }; // S(x,F) = S(x,E)
int main() {
    F f;
    f.x = 0;                   // OK, lookup finds E::x
}

```

$S(x, F)$ is unambiguous because the **A** and **B** base class subobjects of **D** are also base class subobjects of **E**, so $S(x, D)$ is discarded in the first merge step. — end example]

- ⁷ If M is a non-dependent *conversion-function-id*, conversion function templates that are members of T are considered. For each such template F , the lookup set $S(t, T)$ is constructed, considering a function template declaration to have the name t only if it corresponds to a declaration of F (6.4.1). The members of the declaration set of each such lookup set, which shall not be an invalid set, are included in the result.

[Note 3: Overload resolution will discard those that cannot convert to the type specified by M (13.10.4). — end note]

- ⁸ [Note 4: A static member, a nested type or an enumerator defined in a base class **T** can unambiguously be found even if an object has more than one base class subobject of type **T**. Two base class subobjects share the non-static member subobjects of their common virtual base classes. — end note]

[Example 2:

```

struct V {
    int v;
};
struct A {
    int a;
    static int s;
    enum { e };
};
struct B : A, virtual V { };
struct C : A, virtual V { };
struct D : B, C { };

void f(D* pd) {
    pd->v++;           // OK, only one v (virtual)
    pd->s++;           // OK, only one s (static)
    int i = pd->e;     // OK, only one e (enumerator)
}

```

```
    pd->a++;           // error: ambiguous: two as in D
}
```

— end example]

- ⁹ [Note 5: When virtual base classes are used, a hidden declaration can be reached along a path through the subobject lattice that does not pass through the hiding declaration. This is not an ambiguity. The identical use with non-virtual base classes is an ambiguity; in that case there is no unique instance of the name that hides all the others. — end note]

[Example 3:

```
struct V { int f(); int x; };
struct W { int g(); int y; };
struct B : virtual V, W {
    int f(); int x;
    int g(); int y;
};
struct C : virtual V, W { };
struct D : B, C { void glorp(); };
```

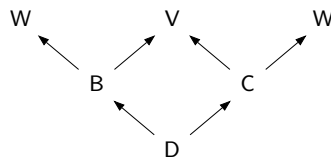


Figure 1 — Name lookup [fig:class.lookup]

As illustrated in Figure 1, the names declared in V and the left-hand instance of W are hidden by those in B, but the names declared in the right-hand instance of W are not hidden at all.

```
void D::glorp() {
    x++;           // OK, B::x hides V::x
    f();           // OK, B::f() hides V::f()
    y++;           // error: B::y and C's W::y
    g();           // error: B::g() and C's W::g()
}
```

— end example]

- ¹⁰ An explicit or implicit conversion from a pointer to or an expression designating an object of a derived class to a pointer or reference to one of its base classes shall unambiguously refer to a unique object representing the base class.

[Example 4:

```
struct V { };
struct A { };
struct B : A, virtual V { };
struct C : A, virtual V { };
struct D : B, C { };

void g() {
    D d;
    B* pb = &d;
    A* pa = &d;           // error: ambiguous: C's A or B's A?
    V* pv = &d;           // OK, only one V subobject
}
```

— end example]

- ¹¹ [Note 6: Even if the result of name lookup is unambiguous, use of a name found in multiple subobjects might still be ambiguous (7.3.13, 7.6.1.5, 11.8.3). — end note]

[Example 5:

```

struct B1 {
    void f();
    static void f(int);
    int i;
};
struct B2 {
    void f(double);
};
struct I1: B1 { };
struct I2: B1 { };

struct D: I1, I2, B2 {
    using B1::f;
    using B2::f;
    void g() {
        f();           // Ambiguous conversion of this
        f(0);          // Unambiguous (static)
        f(0.0);         // Unambiguous (only one B2)
        int B1::* mpB1 = &D::i; // Unambiguous
        int D::* mpD = &D::i;   // Ambiguous conversion
    }
};

```

— end example]

6.5.3 Unqualified name lookup

[basic.lookup.unqual]

- ¹ A *using-directive* is *active* in a scope *S* at a program point *P* if it precedes *P* and inhabits either *S* or the scope of a namespace nominated by a *using-directive* that is active in *S* at *P*.
- ² An *unqualified search* in a scope *S* from a program point *P* includes the results of searches from *P* in
 - (2.1) — *S*, and
 - (2.2) — for any scope *U* that contains *P* and is or is contained by *S*, each namespace contained by *S* that is nominated by a *using-directive* that is active in *U* at *P*.

If no declarations are found, the results of the unqualified search are the results of an unqualified search in the parent scope of *S*, if any, from *P*.

[Note 1: When a class scope is searched, the scopes of its base classes are also searched (6.5.2). If it inherits from a single base, it is as if the scope of the base immediately contains the scope of the derived class. Template parameter scopes that are associated with one scope in the chain of parents are also considered (13.8.2). — end note]

- ³ *Unqualified name lookup* from a program point performs an unqualified search in its immediate scope.
- ⁴ An *unqualified name* is a name that does not immediately follow a *nested-name-specifier* or the `.` or `->` in a class member access expression (7.6.1.5), possibly after a `template` keyword or `~`. Unless otherwise specified, such a name undergoes unqualified name lookup from the point where it appears.
- ⁵ An unqualified name that is a component name (7.5.5.2) of a *type-specifier* or *ptr-operator* of a *conversion-type-id* is looked up in the same fashion as the *conversion-function-id* in which it appears. If that lookup finds nothing, it undergoes unqualified name lookup; in each case, only names that denote types or templates whose specializations are types are considered.

[Example 1:

```

struct T1 { struct U { int i; }; };
struct T2 { };
struct U1 {};
struct U2 {};

struct B {
    using T = T1;
    using U = U1;
    operator U1 T1::*();
    operator U1 T2::*();
    operator U2 T1::*();
};

```

```

    operator U2 T2::*();
};

template<class X, class T>
int g() {
    using U = U2;
    X().operator U T::*();           // #1, searches for T in the scope of X first
    X().operator U decltype(T)::*(); // #2
    return 0;
}
int x = g<B, T2>();                 // #1 calls B::operator U1 T1::*
                                    // #2 calls B::operator U1 T2::*

```

— end example]

- ⁶ In a friend declaration *declarator* whose *declarator-id* is a *qualified-id* whose *nested-name-specifier* designates a class or namespace *S* (7.5.5.3), lookup for an unqualified name that appears after the *declarator-id* performs a search in the scope associated with *S*. If that lookup finds nothing, it undergoes unqualified name lookup.

[Example 2:

```

using I = int;
using D = double;
namespace A {
    inline namespace N {using C = char; }
    using F = float;
    void f(I);
    void f(D);
    void f(C);
    void f(F);
}
struct X0 {using F = float; };
struct W {
    using D = void;
    struct X : X0 {
        void g(I);
        void g(::D);
        void g(F);
    };
};
namespace B {
    typedef short I, F;
    class Y {
        friend void A::f(I);           // error: no void A::f(short)
        friend void A::f(D);           // OK
        friend void A::f(C);           // error: A::N::C not found
        friend void A::f(F);           // OK
        friend void W::X::g(I);         // error: no void X::g(short)
        friend void W::X::g(D);         // OK
        friend void W::X::g(F);         // OK
    };
}

```

— end example]

6.5.4 Argument-dependent name lookup

[basic.lookup.argdep]

- ¹ When the *postfix-expression* in a function call (7.6.1.3) is an *unqualified-id*, and unqualified lookup (6.5.3) for the name in the *unqualified-id* does not find any

- (1.1) — declaration of a class member, or
- (1.2) — function declaration inhabiting a block scope, or
- (1.3) — declaration not of a function or function template

then lookup for the name also includes the result of *argument-dependent lookup* in a set of associated namespaces that depends on the types of the arguments (and for type template template arguments, the namespace of the template argument), as specified below.

[Example 1:

```
namespace N {
    struct S { };
    void f(S);
}

void g() {
    N::S s;
    f(s);           // OK, calls N::f
    (f)(s);         // error: N::f not considered; parentheses prevent argument-dependent lookup
}
```

— end example]

- ² [Note 1: For purposes of determining (during parsing) whether an expression is a *postfix-expression* for a function call, the usual name lookup rules apply. In some cases a name followed by < is treated as a *template-name* even though name lookup did not find the declaration of a template (see 13.3). For example,

```
int h;
void g();
namespace N {
    struct A {};
    template <class T> int f(T);
    template <class T> int g(T);
    template <class T> int h(T);
}

int x = f<N::A>(N::A()); // OK, lookup of f finds nothing, f treated as template name
int y = g<N::A>(N::A()); // OK, lookup of g finds a function, g treated as template name
int z = h<N::A>(N::A()); // error: h< does not begin a template-id
```

The rules have no effect on the syntactic interpretation of an expression. For example,

```
typedef int f;
namespace N {
    struct A {
        friend void f(A &);
        operator int();
        void g(A a) {
            int i = f(a); // f is the typedef, not the friend function: equivalent to int(a)
        }
    };
}
```

Because the expression is not a function call, argument-dependent name lookup does not apply and the friend function `f` is not found. — end note]

- ³ For each argument type `T` in the function call, there is a set of zero or more *associated entities* to be considered. The set of entities is determined entirely by the types of the function arguments (and any type template arguments). Any *typedef-names* and *using-declarations* used to specify the types do not contribute to this set. The set of entities is determined in the following way:

- (3.1) — If `T` is `std::meta::info` (21.4.1), its associated set of entities is the singleton containing the enumeration type `std::meta::operators` (21.4.5).

[Note 2: The `std::meta::info` type is a type alias, so an explicit rule is needed to associate calls whose arguments are reflections with the namespace `std::meta`. — end note]

- (3.2) — If `T` is any other fundamental type, its associated set of entities is empty.

- (3.3) — If `T` is a class type (including unions), its associated entities are: the class itself; the class of which it is a member, if any; and, if it is a complete type, its direct and indirect base classes. Furthermore, if `T` is a class template specialization, its associated entities also include: the entities associated with the types of the template arguments provided for template type parameters; the templates used as type template arguments; and the classes of which any member templates used as type template arguments are members.

[Note 3: Constant template arguments, variable template arguments, and concept template arguments do not contribute to the set of associated entities. — end note]

- (3.4) — If *T* is an enumeration type, its associated entities are *T* and, if it is a class member, the member's class.
- (3.5) — If *T* is a pointer to *U* or an array of *U*, its associated entities are those associated with *U*.
- (3.6) — If *T* is a function type, its associated entities are those associated with the function parameter types and those associated with the return type.
- (3.7) — If *T* is a pointer to a member function of a class *X*, its associated entities are those associated with the function parameter types and return type, together with those associated with *X*.
- (3.8) — If *T* is a pointer to a data member of class *X*, its associated entities are those associated with the member type together with those associated with *X*.

In addition, if the argument is an overload set or the address of such a set, its associated entities are the union of those associated with each of the members of the set, i.e., the entities associated with its parameter types and return type. Additionally, if the aforementioned overload set is named with a *template-id*, its associated entities also include its template arguments and those associated with its type template arguments.

- ⁴ The *associated namespaces* for a call are the innermost enclosing non-inline namespaces for its associated entities as well as every element of the inline namespace set (9.9.2) of those namespaces. Argument-dependent lookup for a name *N* finds all declarations of functions and function templates named *N* that

- (4.1) — are found by a search for *N* in any associated namespace, or
- (4.2) — are declared as a friend (11.8.4) of any class with a reachable definition in the set of associated entities, or
- (4.3) — are exported, are attached to a named module *M* (10.2), do not appear in the translation unit containing the point of the lookup, and have the same innermost enclosing non-inline namespace scope as a declaration of an associated entity attached to *M* (6.7).

If *N* is a dependent name (13.8.3, 13.8.4.2), the above lookup is also performed from each point in the instantiation context (10.6) of the lookup, additionally ignoring any declaration that appears in another translation unit, is attached to the global module, and is either discarded (10.4) or has internal linkage.

- ⁵ [Example 2:

Translation unit #1:

```
export module M;
namespace R {
    export struct X {};
    export void f(X);
}
namespace S {
    export void f(R::X, R::X);
}
```

Translation unit #2:

```
export module N;
import M;
export R::X make();
namespace R { static int g(X); }
export template<typename T, typename U> void apply(T t, U u) {
    f(t, u);
    g(t);
}
```

Translation unit #3:

```
module Q;
import N;
namespace S {
    struct Z { template<typename T> operator T(); };
}
void test() {
    auto x = make();           // OK, decltype(x) is R::X in module M
    R::f(x);                   // error: R and R::f are not visible here
    f(x);                      // OK, calls R::f from interface of M
    f(x, S::Z());              // error: S::f in module M not considered
```

```

    apply(x, S::Z());
    }
    — end example]

```

// even though S is an associated namespace
// error: S::f is visible in instantiation context, but
// R::g has internal linkage and cannot be used outside TU #2

- ⁶ [Note 4: The set of associated namespaces can include namespaces already considered by ordinary unqualified lookup.
— end note]

[Example 3:

```

namespace NS {
    class T { };
    void f(T);
    void g(T, int);
}
NS::T parm;
void g(NS::T, float);
int main() {
    f(parm);
    extern void g(NS::T, float);
    g(parm, 1);
}

```

// OK, calls NS::f
// OK, calls g(NS::T, float)

— end example]

6.5.5 Qualified name lookup

[basic.lookup.qual]

6.5.5.1 General

[basic.lookup.qual.general]

- ¹ Lookup of an *identifier* followed by a `::` scope resolution operator considers only namespaces, types, and templates whose specializations are types. If a name, *template-id*, *splice-scope-specifier*, or *computed-type-specifier* is followed by a `::`, it shall either be a dependent *splice-scope-specifier* (13.8.3.5) or it shall designate a namespace, class, enumeration, or dependent type, and the `::` is never interpreted as a complete *nested-name-specifier*.

[Example 1:

```

class A {
public:
    static int n;
};
int main() {
    int A;
    A::n = 42;
    A b;
}
template<int> struct B : A {};
namespace N {
    template<int> void B();
    int f() {
        return B<0>::n;
    }
}

```

// OK
// error: A does not name a type
// error: N::B<0> is not a type

— end example]

- ² A *member-qualified name* is the (unique) component name (7.5.5.2), if any, of

(2.1) — an *unqualified-id* or

(2.2) — a *nested-name-specifier* of the form *type-name ::* or *namespace-name ::*

in the *id-expression* of a class member access expression (7.6.1.5). A *qualified name* is

(2.3) — a member-qualified name or

(2.4) — the terminal name of

(2.4.1) — a *qualified-id*,

(2.4.2) — a *using-declarator*,

- (2.4.3) — a *typename-specifier*,
- (2.4.4) — a *qualified-namespace-specifier*, or
- (2.4.5) — a *nested-name-specifier*, *reflection-name*, *elaborated-type-specifier*, or *class-or-decltype* that has a *nested-name-specifier* (7.5.5.3).

The *lookup context* of a member-qualified name is the type of its associated object expression (considered dependent if the object expression is type-dependent). The lookup context of any other qualified name is the type, template, or namespace nominated by the preceding *nested-name-specifier*.

[*Note 1*: When parsing a class member access, the name following the `->` or `.` is a qualified name even though it is not yet known of which kind. — *end note*]

[*Example 2*: In

```
N::C::m.Base::f()
```

`Base` is a member-qualified name; the other qualified names are `C`, `m`, and `f`. — *end example*]

- 3 *Qualified name lookup* in a class, namespace, or enumeration performs a search of the scope associated with it (6.5.2) except as specified below. Unless otherwise specified, a qualified name undergoes qualified name lookup in its lookup context from the point where it appears unless the lookup context either is dependent and is not the current instantiation (13.8.3.2) or is not a class or class template. If nothing is found by qualified lookup for a member-qualified name that is the terminal name (7.5.5.2) of a *nested-name-specifier* and is not dependent, it undergoes unqualified lookup.

[*Note 2*: During lookup for a template specialization, no names are dependent. — *end note*]

[*Example 3*:

```
int f();
struct A {
    int B, C;
    template<int> using D = void;
    using T = void;
    void f();
};
using B = A;
template<int> using C = A;
template<int> using D = A;
template<int> using X = A;

template<class T>
void g(T *p) {
    p->X<0>::f();           // as instantiated for g<A>:
                           // error: A::X not found in ((p->X) < 0) > ::f()
    p->template X<0>::f();   // OK, ::X found in definition context
    p->B::f();              // OK, non-type A::B ignored
    p->template C<0>::f();   // error: A::C is not a template
    p->template D<0>::f();   // error: A::D<0> is not a class type
    p->T::f();              // error: A::T is not a class type
}
template void g(A*);
```

— *end example*]

- 4 If a qualified name *Q* follows a `~`:

- (4.1) — If *Q* is a member-qualified name, it undergoes unqualified lookup as well as qualified lookup.
- (4.2) — Otherwise, its *nested-name-specifier* *N* shall nominate a type. If *N* has another *nested-name-specifier* *S*, *Q* is looked up as if its lookup context were that nominated by *S*.
- (4.3) — Otherwise, if the terminal name of *N* is a member-qualified name *M*, *Q* is looked up as if `~Q` appeared in place of *M* (as above).
- (4.4) — Otherwise, *Q* undergoes unqualified lookup.
- (4.5) — Each lookup for *Q* considers only types (if *Q* is not followed by a `<`) and templates whose specializations are types. If it finds nothing or is ambiguous, it is discarded.
- (4.6) — The *type-name* that is or contains *Q* shall refer to its (original) lookup context (ignoring cv-qualification) under the interpretation established by at least one (successful) lookup performed.

[Example 4:

```

struct C {
    typedef int I;
};
typedef int I1, I2;
extern int* p;
extern int* q;
void f() {
    p->C::I::~~I();           // I is looked up in the scope of C
    q->I1::~~I2();           // I2 is found by unqualified lookup
}
struct A {
    ~A();
};
typedef A AB;
int main() {
    AB* p;
    p->AB::~~AB();           // explicitly calls the destructor for A
}

```

— end example]

6.5.5.2 Class members

[class.qual]

- ¹ In a lookup for a qualified name *N* whose lookup context is a class *C* in which function names are not ignored,¹⁵

- (1.1) — if the search finds the injected-class-name of *C* (11.1), or
- (1.2) — if *N* is dependent and is the terminal name of a *using-declarator* (9.10) that names a constructor,

N is instead considered to name the constructor of class *C*. Such a constructor name shall be used only in the *declarator-id* of a (friend) declaration of a constructor or in a *using-declaration*.

[Example 1:

```

struct A { A(); };
struct B: public A { B(); };

A::A() { }
B::B() { }

B::A ba;           // object of type A
A::A a;            // error: A::A is not a type name
struct A::A a2;     // object of type A

```

— end example]

6.5.5.3 Namespace members

[namespace.qual]

- ¹ Qualified name lookup in a namespace *N* additionally searches every element of the inline namespace set of *N* (9.9.2). If nothing is found, the results of the lookup are the results of qualified name lookup in each namespace nominated by a *using-directive* that precedes the point of the lookup and inhabits *N* or an element of *N*'s inline namespace set.

[Note 1: If a *using-directive* refers to a namespace that has already been considered, it does not affect the result.
— end note]

[Example 1:

```

int x;
namespace Y {
    void f(float);
    void h(int);
}

```

¹⁵ Lookups in which function names are ignored include names appearing in a *nested-name-specifier*, an *elaborated-type-specifier*, or a *base-specifier*.

```

namespace Z {
    void h(double);
}

namespace A {
    using namespace Y;
    void f(int);
    void g(int);
    int i;
}

namespace B {
    using namespace Z;
    void f(char);
    int i;
}

namespace AB {
    using namespace A;
    using namespace B;
    void g();
}

void h()
{
    AB::g();           // g is declared directly in AB, therefore S is {AB::g()} and AB::g() is chosen

    AB::f(1);          // f is not declared directly in AB so the rules are applied recursively to A and B;
                      // namespace Y is not searched and Y::f(float) is not considered;
                      // S is {A::f(int), B::f(char)} and overload resolution chooses A::f(int)

    AB::f('c');        // as above but resolution chooses B::f(char)

    AB::x++;           // x is not declared directly in AB, and is not declared in A or B, so the rules
                      // are applied recursively to Y and Z, S is {} so the program is ill-formed

    AB::i++;           // i is not declared directly in AB so the rules are applied recursively to A and B,
                      // S is {A::i, B::i} so the use is ambiguous and the program is ill-formed

    AB::h(16.8);        // h is not declared directly in AB and not declared directly in A or B so the rules
                      // are applied recursively to Y and Z, S is {Y::h(int), Z::h(double)} and
                      // overload resolution chooses Z::h(double)
}

```

— end example]

² [Note 2: The same declaration found more than once is not an ambiguity (because it is still a unique declaration).

[Example 2:

```

namespace A {
    int a;
}

namespace B {
    using namespace A;
}

namespace C {
    using namespace A;
}

namespace BC {
    using namespace B;
    using namespace C;
}

```



```
void f()
{
    BC::a++;           // OK, S is {A::a, A::a}
}
```

```
namespace D {
    using A::a;
}
```

```
namespace BD {
    using namespace B;
    using namespace D;
}
```

```
void g()
{
    BD::a++;           // OK, S is {A::a, A::a}
}
```

— end example]

— end note]

- ³ [Example 3: Because each referenced namespace is searched at most once, the following is well-defined:

```
namespace B {
    int b;
}
```

```
namespace A {
    using namespace B;
    int a;
}
```

```
namespace B {
    using namespace A;
}
```

```
void f()
{
    A::a++;           // OK, a declared directly in A, S is {A::a}
    B::a++;           // OK, both A and B searched (once), S is {A::a}
    A::b++;           // OK, both A and B searched (once), S is {B::b}
    B::b++;           // OK, b declared directly in B, S is {B::b}
}
```

— end example]

- ⁴ [Note 3: Class and enumeration declarations are not discarded because of other declarations found in other searches.

— end note]

[Example 4:

```
namespace A {
    struct x { };
    int x;
    int y;
}
```

```
namespace B {
    struct y { };
}
```

```
namespace C {
    using namespace A;
    using namespace B;
    int i = C::x;     // OK, A::x (of type int)
    int j = C::y;     // ambiguous, A::y or B::y
}
```

— end example]

6.5.6 Elaborated type specifiers

[basic.lookup.elab]

- ¹ If the *class-key* or *enum* keyword in an *elaborated-type-specifier* is followed by an *identifier* that is not followed by `::`, lookup for the *identifier* is type-only (6.5.1).

[Note 1: In general, the recognition of an *elaborated-type-specifier* depends on the following tokens. If the *identifier* is followed by `::`, see 6.5.5. — end note]

- ² If the terminal name of an *elaborated-type-specifier* is a qualified name, lookup for it is type-only. If the name lookup does not find a previously declared *type-name*, the *elaborated-type-specifier* is ill-formed.
- ³ [Example 1:

```
struct Node {
    struct Node* Next;           // OK, refers to injected-class-name Node
    struct Data* Data;           // OK, declares type Data at global scope and member Data
};

struct Data {
    struct Node* Node;           // OK, refers to Node at global scope
    friend struct ::Glob;        // error: Glob is not declared, cannot introduce a qualified type (9.2.9.5)
    friend struct Glob;          // OK, refers to (as yet) undeclared Glob at global scope.
    /* ... */
};

struct Base {
    struct Data;                 // OK, declares nested Data
    struct ::Data* thatData;     // OK, refers to ::Data
    struct Base::Data* thisData; // OK, refers to nested Data
    friend class ::Data;         // OK, global Data is a friend
    friend class Data;           // OK, nested Data is a friend
    struct Data { /* ... */ };   // Defines nested Data
};

struct Data;                    // OK, redeclares Data at global scope
struct ::Data;                  // error: cannot introduce a qualified type (9.2.9.5)
struct Base::Data;              // error: cannot introduce a qualified type (9.2.9.5)
struct Base::Datum;             // error: Datum undefined
struct Base::Data* pBase;       // OK, refers to nested Data
```

— end example]

6.5.7 Using-directives and namespace aliases

[basic.lookup.udir]

- ¹ In a *using-directive* or *namespace-alias-definition*, during the lookup for a *namespace-name* or for a name in a *nested-name-specifier* only namespace names are considered.

6.6 Splice specifiers

[basic.splice]

splice-specifier:

[`:` *constant-expression* `:`]

splice-specialization-specifier:

splice-specifier < *template-argument-list*_{opt} >

- ¹ The *constant-expression* of a *splice-specifier* shall be a converted constant expression of type `std::meta::info` (7.7.3). A *splice-specifier* whose converted *constant-expression* represents a construct *X* is said to *designate* either
 - (1.1) — the underlying entity of *X* if *X* is an entity (6.1), or
 - (1.2) — *X* otherwise.

[Note 1: A *splice-specifier* is dependent if the converted *constant-expression* is value-dependent (13.8.3.5). — end note]

- ² A non-dependent *splice-specifier* of a *splice-specialization-specifier* shall designate a template.
- ³ [Note 2: A < following a *splice-specifier* is interpreted as the delimiter of a *template-argument-list* when the *splice-specifier* is preceded by the keyword `template` or the keyword `typename`, or when it appears in a type-only context (13.3).

[Example 1:

```
constexpr int v = 1;
template<int V> struct TCls {
    static constexpr int s = V + 1;
};

using alias = [::~TCls:]<([::v:])>;
// OK, a splice-specialization-specifier with a parenthesized splice-expression as a template argument

static_assert(alias::s == 2);

auto o1 = [::~TCls:]<([::v:])>(); // error: < means less than
auto o2 = typename [::~TCls:]<([::v:])>(); // OK, o2 is an object of type TCls<1>

constexpr int bad_splice(std::meta::info v) {
    return [::v:]; // error: v is not constant
}
```

— end example]

— end note]

6.7 Program and linkage

[basic.link]

- ¹ A *program* consists of one or more translation units (5.2) that are translated and linked together. A translation unit consists of a sequence of declarations.

translation-unit:

declaration-seq_{opt}

global-module-fragment_{opt} module-declaration declaration-seq_{opt} private-module-fragment_{opt}

- ² A name has *external linkage*, *module linkage*, *internal linkage*, or *no linkage*, as determined by the rules below.

[Note 1: All declarations of an entity with a name with internal linkage appear in the same translation unit. All declarations of an entity with module linkage are attached to the same module. — end note]

- ³ The name of an entity that belongs to a namespace scope (6.4.6) has internal linkage if it is the name of

- (3.1) — a variable, variable template, function, or function template that is explicitly declared **static**; or
- (3.2) — a non-template variable of non-volatile const-qualified type, unless
 - (3.2.1) — it is declared in the purview of a module interface unit (outside the *private-module-fragment*, if any) or module partition, or
 - (3.2.2) — it is explicitly declared **extern**, or
 - (3.2.3) — it is inline, or
 - (3.2.4) — it was previously declared and the prior declaration did not have internal linkage; or
- (3.3) — a data member of an anonymous union.

[Note 2: An instantiated variable template that has const-qualified type can have external or module linkage, even if not declared **extern**. — end note]

- ⁴ An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. The name of an entity that belongs to a namespace scope, that has not been given internal linkage above, and that is the name of

- (4.1) — a variable; or
- (4.2) — a function; or
- (4.3) — a named class (11.1), or an unnamed class defined in a typedef declaration in which the class has the typedef name for linkage purposes (9.2.4); or
- (4.4) — a named enumeration (9.8.1), or an unnamed enumeration defined in a typedef declaration in which the enumeration has the typedef name for linkage purposes (9.2.4); or
- (4.5) — an unnamed enumeration that has an enumerator as a name for linkage purposes (9.8.1); or
- (4.6) — a template

has its linkage determined as follows:

- (4.7) — if the entity is a function or function template first declared in a friend declaration and that declaration is a definition and the enclosing class is defined within an *export-declaration*, the name has the same linkage, if any, as the name of the enclosing class (11.8.4);
- (4.8) — otherwise, if the entity is a function or function template declared in a friend declaration and a corresponding non-friend declaration is reachable, the name has the linkage determined from that prior declaration,
- (4.9) — otherwise, if the enclosing namespace has internal linkage, the name has internal linkage;
- (4.10) — otherwise, if the declaration of the name is attached to a named module (10.1) and is not exported (10.2), the name has module linkage;
- (4.11) — otherwise, the name has external linkage.

⁵ In addition, a member function, a static data member, a named class or enumeration that inhabits a class scope, or an unnamed class or enumeration defined in a typedef declaration that inhabits a class scope such that the class or enumeration has the typedef name for linkage purposes (9.2.4), has the same linkage, if any, as the name of the class of which it is a member.

⁶ [Example 1:

```
static void f();
extern "C" void h();
static int i = 0;           // #1
void q() {
    extern void f();        // internal linkage
    extern void g();        // ::g, external linkage
    extern void h();        // C language linkage
    int i;                 // #2: i has no linkage
    {
        extern void f();    // internal linkage
        extern int i;       // #3: internal linkage
    }
}
```

Even though the declaration at line #2 hides the declaration at line #1, the declaration at line #3 still redeclares #1 and receives internal linkage. — end example]

- ⁷ Names not covered by these rules have no linkage. Moreover, except as noted, a name declared at block scope (6.4.3) has no linkage.
- ⁸ Two declarations of entities declare the same entity if, considering declarations of unnamed types to introduce their names for linkage purposes, if any (9.2.4, 9.8.1), they correspond (6.4.1), have the same target scope that is not a function or template parameter scope, neither is a name-independent declaration, and either
 - (8.1) — they appear in the same translation unit, or
 - (8.2) — they both declare type aliases or namespace aliases that have the same underlying entity, or
 - (8.3) — they both declare names with module or external linkage and are attached to the same module.

[Note 3: There are other circumstances in which declarations declare the same entity (9.12, 13.6, 13.7.6). — end note]

⁹ If a declaration *H* that declares a name with internal linkage precedes a declaration *D* in another translation unit *U* and would declare the same entity as *D* if it appeared in *U*, the program is ill-formed.

[Note 4: Such an *H* can appear only in a header unit. — end note]

¹⁰ [Note 5: If two declarations correspond but are attached to different modules, the program is ill-formed if one precedes the other (6.4.1). — end note]

[Example 2:

```
"decls.h":
int f();           // #1, attached to the global module
int g();           // #2, attached to the global module
```

Module interface of M:

```
module;
```

```
#include "decls.h"
export module M;
export using ::f;    // OK, does not declare an entity, exports #1
int g();             // error: corresponds to #2, but attached to M
export int h();       // #3
export int k();       // #4
```

Other translation unit:

```
import M;
static int h();      // error: conflicts with #3
int k();              // error: conflicts with #4
```

— end example]

As a consequence of these rules, all declarations of an entity are attached to the same module; the entity is said to be *attached* to that module.

11 For any two declarations of an entity *E*:

- (11.1) — If one declares *E* to be a variable or function, the other shall declare *E* as one of the same type.
- (11.2) — If one declares *E* to be an enumerator, the other shall do so.
- (11.3) — If one declares *E* to be a namespace, the other shall do so.
- (11.4) — If one declares *E* to be a type, the other shall declare *E* to be a type of the same kind (9.2.9.5).
- (11.5) — If one declares *E* to be a class template, the other shall do so with the same kind and an equivalent *template-head* (13.7.7.2).
[Note 6: The declarations can supply different default template arguments. — end note]
- (11.6) — If one declares *E* to be a function template or a (partial specialization of a) variable template, the other shall declare *E* to be one with an equivalent *template-head* and type.
- (11.7) — If one declares *E* to be an alias template, the other shall declare *E* to be one with an equivalent *template-head* and *defining-type-id*.
- (11.8) — If one declares *E* to be a concept, the other shall do so.

Types are compared after all adjustments of types (during which type aliases (9.2.4) are replaced by the types they denote); declarations for an array object can specify array types that differ by the presence or absence of a major array bound (9.3.4.5). No diagnostic is required if neither declaration is reachable from the other (G.3.1).

[Example 3:

```
int f(int x, int x);    // error: different entities for x
void g();               // #1
void g(int);            // OK, different entity from #1
int g();                // error: same entity as #1 with different type
void h();               // #2
namespace h {}          // error: same entity as #2, but not a function
```

— end example]

12 [Note 7: Linkage to non-C++ declarations can be achieved using a *linkage-specification* (9.12). — end note]

13 A declaration *D* names an entity *E* if

- (13.1) — *D* contains a *lambda-expression* whose closure type is *E*,
- (13.2) — *D* contains a *reflect-expression* or a *splice-specifier* that, respectively, represents or designates *E*,
- (13.3) — *D* is an injected declaration (7.7.6) whose characteristic sequence contains a reflection that represents a data member description (*T*, *N*, *A*, *W*, *NUA*, *ANN*) (11.4.1) for which *T* is *E*,
- (13.4) — *E* is not a function or function template and *D* contains an *id-expression*, *type-specifier*, *nested-name-specifier*, *template-name*, or *concept-name* denoting *E*, or
- (13.5) — *E* is a function or function template and *D* contains an expression that names *E* (6.3) or an *id-expression* that refers to a set of overloads that contains *E*.

[Note 8: Non-dependent names in an instantiated declaration do not refer to a set of overloads (13.8). — end note]

14 A declaration is an *exposure* if it either names a TU-local entity (defined below), ignoring

- (14.1) — the *function-body* for a non-inline function or function template (but not the deduced return type for a (possibly instantiated) definition of a function with a declared return type that uses a placeholder type (9.2.9.7)),
- (14.2) — the *initializer* for a variable or variable template (but not the variable's type),
- (14.3) — friend declarations in a class definition, and
- (14.4) — any reference to a non-volatile const object or reference with internal or no linkage initialized with a constant expression that is not an odr-use (6.3),

or defines a constexpr variable initialized to a TU-local value (defined below).

[Note 9: An inline function template can be an exposure even though certain explicit specializations of it would be usable in other translation units. — end note]

15 An entity is *TU-local* if it is

- (15.1) — a type, type alias, namespace, namespace alias, function, variable, or template that
 - (15.1.1) — has a name with internal linkage, or
 - (15.1.2) — does not have a name with linkage and is declared, or introduced by a *lambda-expression*, within the definition of a TU-local entity,
- (15.2) — a type with no name that is defined outside a *class-specifier*, function body, or *initializer* or is introduced by a *defining-type-specifier* that is used to declare only TU-local entities,
- (15.3) — a specialization of a TU-local template,
- (15.4) — a specialization of a template with any TU-local template argument, or
- (15.5) — a specialization of a template whose (possibly instantiated) declaration is an exposure.

[Note 10: A specialization can be produced by implicit or explicit instantiation. — end note]

16 A value or object is *TU-local* if either

- (16.1) — it is of TU-local type,
- (16.2) — it is, or is a pointer to, a TU-local function or the object associated with a TU-local variable,
- (16.3) — it is an object of class or array type and any of its subobjects or any of the objects or functions to which its non-static data members of reference type refer is TU-local and is usable in constant expressions, or
- (16.4) — it is a reflection value (6.9.3) that represents
 - (16.4.1) — an entity, value, or object that is TU-local,
 - (16.4.2) — an annotation (9.13.12),
 - (16.4.3) — a direct base class relationship (D, B) (11.7.1) for which either D or B is TU-local, or
 - (16.4.4) — a data member description (T, N, A, W, NUA, ANN) (11.4.1) for which T is TU-local.

17 If a (possibly instantiated) declaration of, or a deduction guide for, a non-TU-local entity in a module interface unit (outside the *private-module-fragment*, if any) or module partition (10.1) is an exposure, the program is ill-formed. Such a declaration in any other context is deprecated (D.2).

18 If a declaration that appears in one translation unit names a TU-local entity declared in another translation unit that is not a header unit, the program is ill-formed. A declaration instantiated for a template specialization (13.9) appears at the point of instantiation of the specialization (13.8.4.1).

19 [Example 4:

Translation unit #1:

```
export module A;
static void f() {}
inline void it() { f(); }           // error: is an exposure of f
static inline void its() { f(); }   // OK
template<int> void g() { its(); }    // OK
template void g<0>();

decltype(f) *fp;                   // error: f (though not its type) is TU-local
auto &fr = f;                       // OK
```

```

constexpr auto &fr2 = fr;           // error: is an exposure of f
constexpr static auto fp2 = fr;     // OK

struct S { void (&ref)(); } s{f};    // OK, value is TU-local
constexpr extern struct W { S &s; } wrap{s}; // OK, value is not TU-local

static auto x = []{f();};           // OK
auto x2 = x;                        // error: the closure type is TU-local
int y = ([]{f();})(), 0);           // error: the closure type is not TU-local
int y2 = (x, 0);                    // OK

namespace N {
    struct A {};
    void adl(A);
    static void adl(int);
}
void adl(double);

inline void h(auto x) { adl(x); }    // OK, but certain specializations are exposures

constexpr std::meta::info r1 = ^g<0>; // OK
namespace N2 {
    static constexpr std::meta::info r2 = ^g<1>; // OK, r2 is TU-local
}
constexpr std::meta::info r3 = ^f;      // error: r3 is an exposure of f

constexpr auto ctx = std::meta::access_context::current();
constexpr std::meta::info r4 =
    std::meta::members_of(^N2, ctx)[0]; // error: r4 is an exposure of N2::r2

```

Translation unit #2:

```

module A;
void other() {
    g<0>(); // OK, specialization is explicitly instantiated
    g<1>(); // error: instantiation uses TU-local its
    h(N::A{}); // error: overload set contains TU-local N::adl(int)
    h(0); // OK, calls adl(double)
    adl(N::A{}); // OK; N::adl(int) not found, calls N::adl(N::A)
    fr(); // OK, calls f
    constexpr auto ptr = fr; // error: fr is not usable in constant expressions here
}

```

— end example]

6.8 Memory and objects

[basic.memobj]

6.8.1 Memory model

[intro.memory]

- ¹ The fundamental storage unit in the C++ memory model is the *byte*. A byte is at least large enough to contain the ordinary literal encoding of any element of the basic literal character set (5.3.1) and the eight-bit code units of the Unicode UTF-8 encoding form and is composed of a contiguous sequence of bits, the number of which is implementation-defined.

[Note 1: See the macro `CHAR_BIT` in the header `<climits>` (17.3.6). — end note]

The memory available to a C++ program consists of one or more sequences of contiguous bytes. Every byte has a unique address.

- ² [Note 2: The representation of types is described in 6.9.1. — end note]
- ³ A *memory location* is the storage occupied by the object representation of either an object of scalar type that is not a bit-field or a maximal sequence of adjacent bit-fields all having nonzero width.

[Note 3: Various features of the language, such as references and virtual functions, might involve additional memory locations that are not accessible to programs but are managed by the implementation. — end note]

Two or more threads of execution (6.10.2) can access separate memory locations without interfering with each other.

⁴ [Note 4: Thus a bit-field and an adjacent non-bit-field are in separate memory locations, and therefore can be concurrently updated by two threads of execution without interference. The same applies to two bit-fields, if one is declared inside a nested struct declaration and the other is not, or if the two are separated by a zero-length bit-field declaration, or if they are separated by a non-bit-field declaration. It is not safe to concurrently update two bit-fields in the same struct if all fields between them are also bit-fields of nonzero width. — end note]

⁵ [Example 1: A class declared as

```
struct {
    char a;
    int b:5,
    c:11,
    :0,
    d:8;
    struct {int ee:8;} e;
};
```

contains four separate memory locations: The member **a** and bit-fields **d** and **e.ee** each occupy separate memory locations, and can be modified concurrently without interfering with each other. The bit-fields **b** and **c** together occupy the fourth memory location. The bit-fields **b** and **c** cannot be concurrently modified, but **b** and **a**, for example, can be. — end example]

6.8.2 Object model

[intro.object]

¹ The constructs in a C++ program create, destroy, refer to, access, and manipulate objects. An *object* is created by a definition (6.2), by a *new-expression* (7.6.2.8), by an operation that implicitly creates objects (see below), when implicitly changing the active member of a union (11.5), or when a temporary object is created (7.3.5, 6.8.7). An object occupies a region of storage in its period of construction (11.9.5), throughout its lifetime (6.8.4), and in its period of destruction (11.9.5).

[Note 1: A function is not an object, regardless of whether or not it occupies storage in the way that objects do. — end note]

The properties of an object are determined when the object is created. An object can have a name (6.1). An object has a storage duration (6.8.6) which influences its lifetime (6.8.4). An object has a type (6.9).

[Note 2: Some objects are polymorphic (11.7.3); the implementation generates information associated with each such object that makes it possible to determine that object's type during program execution. — end note]

² Objects can contain other objects, called *subobjects*. A subobject can be a *member subobject* (11.4), a *base class subobject* (11.7), or an array element. An object that is not a subobject of any other object is called a *complete object*. If an object is created in storage associated with a subobject *e* (which may or may not be within its lifetime), the created object is a subobject of *e*'s containing object if

- (2.1) — the lifetime of *e*'s containing object has begun and not ended, and
- (2.2) — the storage for the new object exactly overlays the storage location associated with *e*, and
- (2.3) — *e* is not a potentially-overlapping subobject, and
- (2.4) — the new object is of the same type as *e* (ignoring cv-qualification).

In this case, *e* and the created object are *corresponding direct subobjects*.

³ If a complete object is created (7.6.2.8) in storage associated with another object *e* of type “array of *N* unsigned char” or of type “array of *N* std::byte” (17.2.1), that array *provides storage* for the created object if

- (3.1) — the lifetime of *e* has begun and not ended, and
- (3.2) — the storage for the new object fits entirely within *e*, and
- (3.3) — there is no array object that satisfies these constraints nested within *e*.

[Note 3: If that portion of the array previously provided storage for another object, the lifetime of that object ends because its storage was reused (6.8.4). — end note]

[Example 1:

```
// assumes that sizeof(int) is equal to 4

template<typename ...T>
struct AlignedUnion {
    alignas(T...) unsigned char data[max(sizeof(T)...)];
```



```

};
int f() {
    AlignedUnion<int, char> au;
    int *p = new (au.data) int;           // OK, au.data provides storage
    char *c = new (au.data) char();       // OK, ends lifetime of *p
    char *d = new (au.data + 1) char();
    return *c + *d;                       // OK
}

struct A { unsigned char a[32]; };
struct B { unsigned char b[16]; };
alignas(int) A a;
B *b = new (a.a + 8) B;                  // a.a provides storage for *b
int *p = new (b->b + 4) int;              // b->b provides storage for *p
                                           // a.a does not provide storage for *p (directly),
                                           // but *p is nested within a (see below)

```

— end example]

4 An object *a* is *nested within* another object *b* if

- (4.1) — *a* is a subobject of *b*, or
- (4.2) — *b* provides storage for *a*, or
- (4.3) — there exists an object *c* where *a* is nested within *c*, and *c* is nested within *b*.

5 For every object *x*, there is some object called the *complete object of x*, determined as follows:

- (5.1) — If *x* is a complete object, then the complete object of *x* is itself.
- (5.2) — Otherwise, the complete object of *x* is the complete object of the (unique) object that contains *x*.

6 If a complete object, a member subobject, or an array element is of class type, its type is considered the *most derived class*, to distinguish it from the class type of any base class subobject; an object of a most derived class type or of a non-class type is called a *most derived object*.

7 A *potentially-overlapping subobject* is either:

- (7.1) — a base class subobject, or
- (7.2) — a non-static data member declared with the `no_unique_address` attribute (9.13.11).

8 An object has nonzero size if it

- (8.1) — is not a potentially-overlapping subobject, or
- (8.2) — is not of class type, or
- (8.3) — is of a class type with virtual member functions or virtual base classes, or
- (8.4) — has subobjects of nonzero size or unnamed bit-fields of nonzero length.

Otherwise, if the object is a base class subobject of a standard-layout class type with no non-static data members, it has zero size. Otherwise, the circumstances under which the object has zero size are implementation-defined. Unless it is a bit-field (11.4.10), an object with nonzero size shall occupy one or more bytes of storage, including every byte that is occupied in full or in part by any of its subobjects. An object of trivially copyable or standard-layout type (6.9.1) shall occupy contiguous bytes of storage.

9 An object is a *potentially non-unique object* if it is

- (9.1) — a string literal object (5.13.5),
- (9.2) — the backing array of an initializer list (9.5.4), or
- (9.3) — a template parameter object of array type (21.4.3), or
- (9.4) — a subobject thereof.

10 Unless an object is a bit-field or a subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects with overlapping lifetimes that are not bit-fields may have the same address if

- (10.1) — one is nested within the other,
- (10.2) — they are both nested within some complete object *o*, exactly one is a subobject of *o*, and the subobject is of zero size,

- (10.3) — they are both subobjects of the same complete object, at least one is a subobject of zero size, and they are not of similar types (7.3.6), or
- (10.4) — they are both potentially non-unique objects;

otherwise, they have distinct addresses and occupy disjoint bytes of storage.

[Example 2:

```
static const char test1 = 'x';
static const char test2 = 'x';
const bool b = &test1 != &test2;           // always true

static const char (&r) [] = "x";
static const char *s = "x";
static std::initializer_list<char> il = { 'x' };
const bool b2 = r != il.begin();           // unspecified result
const bool b3 = r != s;                   // unspecified result
const bool b4 = il.begin() != &test1;     // always true
const bool b5 = r != &test1;             // always true
```

— end example]

The address of a subobject of zero size is the address of an unspecified byte of storage occupied by the complete object of that subobject.

- 11 A *union elemental subobject* is a direct member of a union or an element of an array that is a union elemental subobject. An *inactive union elemental subobject* is a union elemental subobject that is not within its lifetime.
- 12 The *constituent values* of an object *o* are
 - (12.1) — if *o* has scalar type, the value of *o*;
 - (12.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union elemental subobjects.

The *constituent references* of an object *o* are

- (12.3) — any direct members of *o* that have reference type, and
- (12.4) — the constituent references of any direct subobjects of *o* other than inactive union elemental subobjects.

[Example 3:

```
struct A {
    struct X {
        int i;
        int j;
    };

    struct Y {
        X x1;
        X x2;
    };

    union {
        int i;
        int arr[4];
        Y y;
    };
};

constexpr A v1;           // OK, no constituent values
constexpr A v2{.i=1};     // OK, the constituent values are {v2.i}
constexpr A v3 = []{
    A a;
    std::start_lifetime(a.arr); // OK, arr is now the active element of the union
    new (&a.arr[1]) int(1);
    a.arr[2] = 2;
    return a;
}();                       // OK, the constituent values are {v3.arr[1], v3.arr[2]}
```

```
constexpr A v4 = []{
    A a;
    a.y.x1.i = 1;
    a.y.x2.j = 2;
    return a;
}();
```

*// error: the constituent values include v4.y.x1.j and v4.y.x2.i
// which have erroneous value*

— end example]

- ¹³ Some operations are described as *implicitly creating objects* within a specified region of storage. For each operation that is specified as implicitly creating objects, that operation implicitly creates and starts the lifetime of zero or more objects of implicit-lifetime types (6.9.1) in its specified region of storage if doing so would result in the program having defined behavior. If no such set of objects would give the program defined behavior (F.2.1), the behavior of the program is undefined. If multiple such sets of objects would give the program defined behavior, it is unspecified which such set of objects is created.

[Note 4: Such operations do not start the lifetimes of subobjects of such objects that are not themselves of implicit-lifetime types. — end note]

- ¹⁴ Further, after implicitly creating objects within a specified region of storage, some operations are described as producing a pointer to a *suitable created object*. These operations select one of the implicitly-created objects whose address is the address of the start of the region of storage, and produce a pointer value that points to that object, if that value would result in the program having defined behavior. If no such pointer value would give the program defined behavior, the behavior of the program is undefined (F.2.2). If multiple such pointer values would give the program defined behavior, it is unspecified which such pointer value is produced.

- ¹⁵ [Example 4:

```
#include <cstdlib>
struct X { int a, b; };
X *make_x() {
    // The call to std::malloc implicitly creates an object of type X
    // and its subobjects a and b, and returns a pointer to that X object
    // (or an object that is pointer-interconvertible (6.9.5) with it),
    // in order to give the subsequent class member access operations
    // defined behavior.
    X *p = (X*)std::malloc(sizeof(struct X));
    p->a = 1;
    p->b = 2;
    return p;
}
```

— end example]

- ¹⁶ Except during constant evaluation, an operation that begins the lifetime of an array of `unsigned char` or `std::byte` implicitly creates objects within the region of storage occupied by the array.

[Note 5: The array object provides storage for these objects. — end note]

Except during constant evaluation, any implicit or explicit invocation of a function named `operator new` or `operator new[]` implicitly creates objects in the returned region of storage and returns a pointer to a suitable created object.

[Note 6: Some functions in the C++ standard library implicitly create objects (20.2.6, 20.2.12, 20.6.2.2, 22.11.3, 27.5.1). — end note]

6.8.3 Alignment

[basic.align]

- ¹ Object types have *alignment requirements* (6.9.3, 6.9.5) which place restrictions on the addresses at which an object of that type may be allocated. An *alignment* is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated. An object type imposes an alignment requirement on every object of that type; stricter alignment can be requested using the *alignment-specifier* (9.13.2). Attempting to create an object (6.8.2) in storage that does not meet the alignment requirements of the object's type is undefined behavior (F.2.3).
- ² A *fundamental alignment* is represented by an alignment less than or equal to the greatest alignment supported by the implementation in all contexts, which is equal to `alignof(std::max_align_t)` (17.2). The alignment

required for a type may be different when it is used as the type of a complete object and when it is used as the type of a subobject.

[*Example 1:*

```
struct B { long double d; };
struct D : virtual B { char c; };
```

When D is the type of a complete object, it will have a subobject of type B, so it must be aligned appropriately for a `long double`. If D appears as a subobject of another object that also has B as a virtual base class, the B subobject might be part of a different subobject, reducing the alignment requirements on the D subobject. — *end example*]

The result of the `alignof` operator reflects the alignment requirement of the type in the complete-object case.

- ³ An *extended alignment* is represented by an alignment greater than `alignof(std::max_align_t)`. It is implementation-defined whether any extended alignments are supported and the contexts in which they are supported (9.13.2). A type having an extended alignment requirement is an *over-aligned type*.

[*Note 1:* Every over-aligned type is or contains a class type to which extended alignment applies (possibly through a non-static data member). — *end note*]

A *new-extended alignment* is represented by an alignment greater than `__STDCPP_DEFAULT_NEW_ALIGNMENT__` (15.12).

- ⁴ Alignments are represented as values of the type `std::size_t`. Valid alignments include only those values returned by an `alignof` expression for the fundamental types plus an additional implementation-defined set of values, which may be empty. Every alignment value shall be a non-negative integral power of two.
- ⁵ Alignments have an order from *weaker* to *stronger* or *stricter* alignments. Stricter alignments have larger alignment values. An address that satisfies an alignment requirement also satisfies any weaker valid alignment requirement.
- ⁶ The alignment requirement of a complete type can be queried using an `alignof` expression (7.6.2.6). Furthermore, the narrow character types (6.9.3) shall have the weakest alignment requirement.

[*Note 2:* The type `unsigned char` can be used as the element type of an array providing aligned storage (9.13.2). — *end note*]

- ⁷ Comparing alignments is meaningful and provides the obvious results:

- (7.1) — Two alignments are equal when their numeric values are equal.
- (7.2) — Two alignments are different when their numeric values are not equal.
- (7.3) — When an alignment is larger than another it represents a stricter alignment.

- ⁸ [*Note 3:* The runtime pointer alignment function (20.2.5) can be used to obtain an aligned pointer within a buffer; an *alignment-specifier* (9.13.2) can be used to align storage explicitly. — *end note*]

- ⁹ If a request for a specific extended alignment in a specific context is not supported by an implementation, the program is ill-formed.

6.8.4 Lifetime

[**basic.life**]

- ¹ In this subclause, “before” and “after” refer to the “happens before” relation (6.10.2).

- ² The *lifetime* of an object or reference is a runtime property of the object or reference. A variable is said to have *vacuous initialization* if it is default-initialized, no other initialization is performed, and, if it is of class type or a (possibly multidimensional) array thereof, a trivial constructor of that class type is selected for the default-initialization. The lifetime of an object of type T begins when:

- (2.1) — storage with the proper alignment and size for type T is obtained, and
- (2.2) — its initialization (if any) is complete (including vacuous initialization) (9.5, 11.9.3),

except that if the object is a union member or subobject thereof, its lifetime only begins if that union member is the initialized member in the union (9.5.2, 11.9.3), or as described in 11.5, 11.4.5.3, 11.4.6, and 20.2.6, and except as described in 20.2.10.2. The lifetime of an object *o* of type T ends when:

- (2.3) — if T is a non-class type, the object is destroyed, or
- (2.4) — if T is a class type, the destructor call starts, or
- (2.5) — the storage which the object occupies is released, or is reused by an object that is not nested within *o* (6.8.2).

When evaluating a *new-expression*, storage is considered reused after it is returned from the allocation function, but before the evaluation of the *new-initializer* (7.6.2.8).

[Example 1:

```
struct S {
    int m;
};

void f() {
    S x{1};
    new(&x) S(x.m);    // undefined behavior
}
```

— end example]

- 3 The lifetime of a reference begins when its initialization is complete. The lifetime of a reference ends as if it were a scalar object requiring storage.

4 [Note 1: 11.9.3 describes the lifetime of base and member subobjects. — end note]

- 5 The properties ascribed to objects and references throughout this document apply for a given object or reference only during its lifetime.

[Note 2: In particular, before the lifetime of an object starts and after its lifetime ends there are significant restrictions on the use of the object, as described below, in 11.9.3, and in 11.9.5. Also, the behavior of an object under construction and destruction can differ from the behavior of an object whose lifetime has started and not ended. 11.9.3 and 11.9.5 describe the behavior of an object during its periods of construction and destruction. — end note]

- 6 A program may end the lifetime of an object of class type without invoking the destructor, by reusing or releasing the storage as described above.

[Note 3: A *delete-expression* (7.6.2.9) invokes the destructor prior to releasing the storage. — end note]

In this case, the destructor is not implicitly invoked.

[Note 4: The correct behavior of a program often depends on the destructor being invoked for each object of class type. — end note]

- 7 Before the lifetime of an object has started but after the storage which the object will occupy has been allocated¹⁶ or after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any pointer that represents the address of the storage location where the object will be or was located may be used but only in limited ways. For an object under construction or destruction, see 11.9.5. Otherwise, such a pointer refers to allocated storage (6.8.6.5.2), and using the pointer as if the pointer were of type `void*` is well-defined. Indirection through such a pointer is permitted but the resulting lvalue may only be used in limited ways, as described below. The program has undefined behavior if:

- (7.1) — the pointer is used as the operand of a *delete-expression* (F.2.4),
- (7.2) — the pointer is used to access a non-static data member or call a non-static member function of the object (F.2.5), or
- (7.3) — the pointer is converted (7.3.12, 7.6.1.9) to a pointer to a virtual base class (F.2.6) or to a base class thereof, or
- (7.4) — the pointer is used as the operand of a `dynamic_cast` (7.6.1.7) (F.2.7).

[Example 2:

```
#include <cstdlib>

struct B {
    virtual void f();
    void mutate();
    virtual ~B();
};

struct D1 : B { void f(); };
struct D2 : B { void f(); };
```

¹⁶) For example, before the dynamic initialization of an object with static storage duration (6.10.3.3).

```

void B::mutate() {
    new (this) D2;    // reuses storage — ends the lifetime of *this
    f();             // undefined behavior
    ... = this;      // OK, this points to valid memory
}

void g() {
    void* p = std::malloc(sizeof(D1) + sizeof(D2));
    B* pb = new (p) D1;
    pb->mutate();
    *pb;             // OK, pb points to valid memory
    void* q = pb;    // OK, pb points to valid memory
    pb->f();          // undefined behavior: lifetime of *pb has ended
}

```

— end example]

- 8 Similarly, before the lifetime of an object has started but after the storage which the object will occupy has been allocated or after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any glvalue that refers to the original object may be used but only in limited ways. For an object under construction or destruction, see 11.9.5. Otherwise, such a glvalue refers to allocated storage (6.8.6.5.2), and using the properties of the glvalue that do not depend on its value is well-defined. The program has undefined behavior if:

- (8.1) — the glvalue is used to access the object (F.2.8), or
- (8.2) — the glvalue is used to call a non-static member function of the object (F.2.9), or
- (8.3) — the glvalue is bound to a reference to a virtual base class (9.5.4) (F.2.10), or
- (8.4) — the glvalue is used as the operand of a `dynamic_cast` (7.6.1.7) or as the operand of `typeid` (F.2.11).

[Note 5: Therefore, undefined behavior results if an object that is being constructed in one thread is referenced from another thread without adequate synchronization. — end note]

- 9 An object o_1 is *transparently replaceable* by an object o_2 if either

- (9.1) — o_1 and o_2 are complete objects for which:
 - (9.1.1) — o_1 is not `const`,
 - (9.1.2) — the storage that o_2 occupies exactly overlays the storage that o_1 occupied, and
 - (9.1.3) — o_1 and o_2 are of the same type (ignoring the top-level cv-qualifiers), or
- (9.2) — o_1 and o_2 are corresponding direct subobjects (6.8.2) for which:
 - (9.2.1) — the complete object of o_1 is not `const` or
 - (9.2.2) — o_1 is a mutable member subobject or a subobject thereof.

- 10 After the lifetime of an object has ended and before the storage which the object occupied is reused or released, if a new object is created at the storage location which the original object occupied and the original object was transparently replaceable by the new object, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object.

[Example 3:

```

struct C {
    int i;
    void f();
    const C& operator=( const C& );
};

const C& C::operator=( const C& other) {
    if ( this != &other ) {
        this->~C();           // lifetime of *this ends
        new (this) C(other);  // new object of type C created
        f();                 // well-defined
    }
    return *this;
}

```

```

C c1;
C c2;
c1 = c2;           // well-defined
c1.f();            // well-defined; c1 refers to a new object of type C
— end example]

```

[*Note 6*: If these conditions are not met, a pointer to the new object can be obtained from a pointer that represents the address of its storage by calling `std::launder` (17.6.5). — *end note*]

- ¹¹ If a program ends the lifetime of an object of type *T* with static (6.8.6.2), thread (6.8.6.3), or automatic (6.8.6.4) storage duration and if *T* has a non-trivial destructor,¹⁷ and another object of the original type does not occupy that same storage location when the implicit destructor call takes place, the behavior of the program is undefined (F.2.12). This is true even if the block is exited with an exception.

[*Example 4*:

```

class T { };
struct B {
    ~B();
};

void h() {
    B b;
    new (&b) T;
}
// undefined behavior at block exit

```

— *end example*]

- ¹² Creating a new object within the storage that a `const`, complete object with static, thread, or automatic storage duration occupies, or within the storage that such a `const` object used to occupy before its lifetime ended, results in undefined behavior (F.2.13).

[*Example 5*:

```

struct B {
    B();
    ~B();
};

const B b;

void h() {
    b.~B();
    new (const_cast<B*>(&b)) const B;    // undefined behavior
}

```

— *end example*]

6.8.5 Indeterminate and erroneous values

[**basic.indet**]

- ¹ When storage for an object with automatic or dynamic storage duration is obtained, the bytes comprising the storage for the object have the following initial value:
- (1.1) — If the object has dynamic storage duration, or is the object associated with a variable or function parameter whose first declaration is marked with the `[[indeterminate]]` attribute (9.13.6), the bytes have *indeterminate values*;
 - (1.2) — otherwise, the bytes have *erroneous values*, where each value is determined by the implementation independently of the state of the program.

If no initialization is performed for an object (including subobjects), such a byte retains its initial value until that value is replaced (9.5.1, 7.6.19). If any bit in the value representation has an indeterminate value, the object has an indeterminate value; otherwise, if any bit in the value representation has an erroneous value, the object has an erroneous value.

[*Note 1*: Lvalue-to-rvalue conversion has undefined behavior if the erroneous value of an object is not valid for its type (7.3.2). — *end note*]

¹⁷⁾ That is, an object for which a destructor will be called implicitly—upon exit from the block for an object with automatic storage duration, upon exit from the thread for an object with thread storage duration, or upon exit from the program for an object with static storage duration.

[*Note 2:* Objects with static or thread storage duration are zero-initialized, see 6.10.3.2. — *end note*]

- ² If any operand of a built-in operator that produces a prvalue is evaluated, is not a discarded-value expression (7.2.3), and produces an erroneous value, then the value produced by that operator is erroneous. Except in the following cases, if an indeterminate value is produced by an evaluation, the behavior is undefined (F.2.14), and if an erroneous value is produced by an evaluation, the behavior is erroneous and the result of the evaluation is that erroneous value:

- (2.1) — If an indeterminate or erroneous value of unsigned ordinary character type (6.9.3) or `std::byte` type (17.2.1) is produced by the evaluation of:
 - (2.1.1) — the second or third operand of a conditional expression (7.6.16),
 - (2.1.2) — the right operand of a comma expression (7.6.20),
 - (2.1.3) — the operand of a cast or conversion (7.3.9, 7.6.1.4, 7.6.1.9, 7.6.3) to an unsigned ordinary character type or `std::byte` type (17.2.1), or
 - (2.1.4) — a discarded-value expression (7.2.3),
 then the result of the operation is an indeterminate value or that erroneous value, respectively.
- (2.2) — If an indeterminate or erroneous value of unsigned ordinary character type or `std::byte` type is produced by the evaluation of the right operand of a simple assignment operator (7.6.19) whose first operand is an lvalue of unsigned ordinary character type or `std::byte` type, an indeterminate value or that erroneous value, respectively, replaces the value of the object referred to by the left operand.
- (2.3) — If an indeterminate or erroneous value of unsigned ordinary character type is produced by the evaluation of the initialization expression when initializing an object of unsigned ordinary character type, that object is initialized to an indeterminate value or that erroneous value, respectively.
- (2.4) — If an indeterminate value of unsigned ordinary character type or `std::byte` type is produced by the evaluation of the initialization expression when initializing an object of `std::byte` type, that object is initialized to an indeterminate value or that erroneous value, respectively.

Converting an indeterminate or erroneous value of unsigned ordinary character type or `std::byte` type produces an indeterminate or erroneous value, respectively. In the latter case, the result of the conversion is the value of the converted operand.

[*Example 1:*

```
int f(bool b) {
    unsigned char *c = new unsigned char;
    unsigned char d = *c;           // OK, d has an indeterminate value
    int e = d;                      // undefined behavior
    return b ? d : 0;               // undefined behavior if b is true
}

int g(bool b) {
    unsigned char c;
    unsigned char d = c;           // no erroneous behavior, but d has an erroneous value

    assert(c == d);                // holds, both integral promotions have erroneous behavior

    int e = d;                     // erroneous behavior
    return b ? d : 0;              // erroneous behavior if b is true
}

void h() {
    int d1, d2;

    int e1 = d1;                   // erroneous behavior
    int e2 = d1;                   // erroneous behavior

    assert(e1 == e2);              // holds
    assert(e1 == d1);              // holds, erroneous behavior
    assert(e2 == d1);              // holds, erroneous behavior

    std::memcpy(&d2, &d1, sizeof(int)); // no erroneous behavior, but d2 has an erroneous value
```



```

    assert(e1 == d2);           // holds, erroneous behavior
    assert(e2 == d2);           // holds, erroneous behavior
}
— end example]

```

6.8.6 Storage duration

[basic.stc]

6.8.6.1 General

[basic.stc.general]

- ¹ The *storage duration* is the property of an object that defines the minimum potential lifetime of the storage containing the object. The storage duration is determined by the construct used to create the object and is one of the following:
 - (1.1) — static storage duration
 - (1.2) — thread storage duration
 - (1.3) — automatic storage duration
 - (1.4) — dynamic storage duration

[Note 1: After the duration of a region of storage has ended, the use of pointers to that region of storage is limited (6.9.5). — end note]
- ² Static, thread, and automatic storage durations are associated with objects introduced by declarations (6.2) and with temporary objects (6.8.7). The dynamic storage duration is associated with objects created by a *new-expression* (7.6.2.8) or with implicitly created objects (6.8.2).
- ³ The storage duration categories apply to references as well.
- ⁴ The storage duration of subobjects and reference members is that of their complete object (6.8.2).

6.8.6.2 Static storage duration

[basic.stc.static]

- ¹ All variables which
 - (1.1) — do not have thread storage duration and
 - (1.2) — belong to a namespace scope (6.4.6) or are first declared with the **static** or **extern** keywords (9.2.2) have *static storage duration*. The storage for these entities lasts for the duration of the program (6.10.3.2, 6.10.3.4).
- ² If a variable with static storage duration has initialization or a destructor with side effects, it shall not be eliminated even if it appears to be unused, except that a class object or its copy/move may be eliminated as specified in 11.9.6.
- ³ [Note 1: The keyword **static** can be used to declare a block variable (6.4.3) with static storage duration; 8.10 and 6.10.3.4 describe the initialization and destruction of such variables. The keyword **static** applied to a class data member in a class definition gives the data member static storage duration (11.4.9.3). — end note]

6.8.6.3 Thread storage duration

[basic.stc.thread]

- ¹ All variables declared with the **thread_local** keyword have *thread storage duration*. The storage for these entities lasts for the duration of the thread in which they are created. There is a distinct object or reference per thread, and use of the declared name refers to the entity associated with the current thread.
- ² [Note 1: A variable with thread storage duration is initialized as specified in 6.10.3.2, 6.10.3.3, and 8.10 and, if constructed, is destroyed on thread exit (6.10.3.4). — end note]

6.8.6.4 Automatic storage duration

[basic.stc.auto]

- ¹ Variables that belong to a block scope and are not explicitly declared **static**, **thread_local**, or **extern** have *automatic storage duration*. The storage for such variables lasts until the block in which they are created exits.

[Note 1: These variables are initialized and destroyed as described in 8.10. — end note]

Variables that belong to a parameter scope also have automatic storage duration. The storage for a function parameter lasts until immediately after its destruction (7.6.1.3).
- ² If a variable with automatic storage duration has initialization or a destructor with side effects, an implementation shall not destroy it before the end of its block nor eliminate it as an optimization, even if it appears to be unused, except that a class object or its copy/move may be eliminated as specified in 11.9.6.

6.8.6.5 Dynamic storage duration

[basic.stc.dynamic]

6.8.6.5.1 General

[basic.stc.dynamic.general]

- ¹ Objects can be created dynamically during program execution (6.10.1), using *new-expressions* (7.6.2.8), and destroyed using *delete-expressions* (7.6.2.9). A C++ implementation provides access to, and management of, dynamic storage via the global *allocation functions* `operator new` and `operator new[]` and the global *deallocation functions* `operator delete` and `operator delete[]`.

[Note 1: The non-allocating forms described in 17.6.3.4 do not perform allocation or deallocation. — end note]

- ² The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (9.6.5). The following allocation and deallocation functions (17.6) are implicitly declared in global scope in each translation unit of a program.

```
void* operator new(std::size_t);
void* operator new(std::size_t, std::align_val_t);

void operator delete(void*) noexcept;
void operator delete(void*, std::size_t) noexcept;
void operator delete(void*, std::align_val_t) noexcept;
void operator delete(void*, std::size_t, std::align_val_t) noexcept;

void* operator new[](std::size_t);
void* operator new[](std::size_t, std::align_val_t);

void operator delete[](void*) noexcept;
void operator delete[](void*, std::size_t) noexcept;
void operator delete[](void*, std::align_val_t) noexcept;
void operator delete[](void*, std::size_t, std::align_val_t) noexcept;
```

These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`.

[Note 2: The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression*, or function call that refers to one of these functions without importing or including the header `<new>` (17.6.2) or importing a C++ library module (16.4.2.4) is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` is ill-formed unless a standard library declaration (17.2.1, 17.6.2, 16.4.2.4) of that name precedes (6.5.1) the use of that name. — end note]

Allocation and/or deallocation functions may also be declared and defined for any class (11.4.11).

- ³ If the behavior of an allocation or deallocation function does not satisfy the semantic constraints specified in 6.8.6.5.2 and 6.8.6.5.3, the behavior is undefined (F.2.15).

6.8.6.5.2 Allocation functions

[basic.stc.dynamic.allocation]

- ¹ An allocation function that is not a class member function shall belong to the global scope and not have a name with internal linkage. The return type shall be “pointer to void”. The first parameter shall have type `std::size_t` (17.2). The first parameter shall not have an associated default argument (9.3.4.7). The value of the first parameter is interpreted as the requested size of the allocation. An allocation function can be a function template. Such a template shall declare its return type and first parameter as specified above (that is, template parameter types shall not be used in the return type and first parameter type). Allocation function templates shall have two or more parameters.
- ² An allocation function attempts to allocate the requested amount of storage. If it is successful, it returns the address of the start of a block of storage whose length in bytes is at least as large as the requested size. The order, contiguity, and initial value of storage allocated by successive calls to an allocation function are unspecified. Even if the size of the space requested is zero, the request can fail. If the request succeeds, the value returned by a replaceable allocation function is a non-null pointer value (6.9.5) `p0` different from any previously returned value `p1`, unless that value `p1` was subsequently passed to a replaceable deallocation function. Furthermore, for the library allocation functions in 17.6.3.2 and 17.6.3.3, `p0` represents the address of a block of storage disjoint from the storage for any other object accessible to the caller. The effect of indirecting through a pointer returned from a request for zero size is undefined (F.2.16).¹⁸

¹⁸ The intent is to have `operator new()` implementable by calling `std::malloc()` or `std::calloc()`, so the rules are substantially the same. C++ differs from C in requiring a zero request to return a non-null pointer.

- ³ For an allocation function other than a reserved placement allocation function (17.6.3.4), the pointer returned on a successful call shall represent the address of storage that is aligned as follows:
- (3.1) — If the allocation function takes an argument of type `std::align_val_t`, the storage will have the alignment specified by the value of this argument.
 - (3.2) — Otherwise, if the allocation function is named `operator new[]`, the storage is aligned for any object that does not have new-extended alignment (6.8.3) and is no larger than the requested size.
 - (3.3) — Otherwise, the storage is aligned for any object that does not have new-extended alignment and is of the requested size.

- ⁴ An allocation function that fails to allocate storage can invoke the currently installed new-handler function (17.6.4.3), if any.

[Note 1: A program-supplied allocation function can obtain the currently installed `new_handler` using the `std::get_new_handler` function (17.6.4.5). — end note]

An allocation function that has a non-throwing exception specification (14.5) indicates failure by returning a null pointer value. Any other allocation function never returns a null pointer value and indicates failure only by throwing an exception (14.2) of a type that would match a handler (14.4) of type `std::bad_alloc` (17.6.4.1).

- ⁵ A global allocation function is only called as the result of a new expression (7.6.2.8), or called directly using the function call syntax (7.6.1.3), or called indirectly to allocate storage for a coroutine state (9.6.4), or called indirectly through calls to the functions in the C++ standard library.

[Note 2: In particular, a global allocation function is not called to allocate storage for objects with static storage duration (6.8.6.2), for objects or references with thread storage duration (6.8.6.3), for objects of type `std::type_info` (7.6.1.8), for an object of type `std::contracts::contract_violation` when a contract violation occurs (6.11.2), or for an exception object (14.2). — end note]

6.8.6.5.3 Deallocation functions

[basic.stc.dynamic.deallocation]

- ¹ A deallocation function that is not a class member function shall belong to the global scope and not have a name with internal linkage.
- ² A deallocation function is a *destroying operator delete* if it has at least two parameters and its second parameter is of type `std::destroying_delete_t`. A destroying operator delete shall be a class member function named `operator delete`.

[Note 1: Array deletion cannot use a destroying operator delete. — end note]

- ³ Each deallocation function shall return `void`. A deallocation function shall not have a potentially throwing exception specification (14.5). If the function is a destroying operator delete declared in class type `C`, the type of its first parameter shall be “pointer to `C`”; otherwise, the type of its first parameter shall be “pointer to `void`”. A deallocation function may have more than one parameter. A *usual deallocation function* is a deallocation function whose parameters after the first are

- (3.1) — optionally, a parameter of type `std::destroying_delete_t`, then
- (3.2) — optionally, a parameter of type `std::size_t`,¹⁹ then
- (3.3) — optionally, a parameter of type `std::align_val_t`.

A destroying operator delete shall be a usual deallocation function. A deallocation function may be an instance of a function template. Neither the first parameter nor the return type shall depend on a template parameter. A deallocation function template shall have two or more function parameters. A template instance is never a usual deallocation function, regardless of its signature.

- ⁴ The value of the first argument supplied to a deallocation function may be a null pointer value; if so, and if the deallocation function is one supplied in the standard library, the call has no effect.
- ⁵ If the argument given to a deallocation function in the standard library is a pointer that is not the null pointer value (6.9.5), the deallocation function shall deallocate the storage referenced by the pointer, ending the duration of the region of storage.

6.8.7 Temporary objects

[class.temporary]

- ¹ A *temporary object* is an object created

¹⁹ The global `operator delete(void*, std::size_t)` precludes use of an allocation function `void operator new(std::size_t, std::size_t)` as a placement allocation function (C.6.3).

- (1.1) — when a prvalue is converted to an xvalue (7.3.5) and
- (1.2) — when needed by the implementation to pass or return an object of suitable type (see below).

Even when the creation of the temporary object is unevaluated (7.2.3), all the semantic restrictions shall be respected as if the temporary object had been created and later destroyed.

[*Note 1:* This includes accessibility (11.8) and whether it is deleted, for the constructor selected and for the destructor. However, in the special case of the operand of a *decltype-specifier* (9.2.9.6), no temporary is introduced, so the foregoing does not apply to such a prvalue. — *end note*]

- 2 The materialization of a temporary object is generally delayed as long as possible in order to avoid creating unnecessary temporary objects.

[*Note 2:* Temporary objects are materialized:

- (2.1) — when binding a reference to a prvalue (9.5.4, 7.6.1.4, 7.6.1.7, 7.6.1.9, 7.6.1.11, 7.6.3),
- (2.2) — when performing certain member accesses on a class prvalue (7.6.1.5, 7.6.4),
- (2.3) — when invoking an implicit object member function on a class prvalue (7.6.1.3),
- (2.4) — when performing an array-to-pointer conversion or subscripting on an array prvalue (7.3.3, 7.6.1.2),
- (2.5) — when initializing an object of type `std::initializer_list<T>` from a *braced-init-list* (9.5.5),
- (2.6) — for certain unevaluated operands (7.6.1.8, 7.6.2.5), and
- (2.7) — when a prvalue that has type other than *cv void* appears as a discarded-value expression (7.2.3).

— *end note*]

[*Example 1:* Consider the following code:

```
class X {
public:
    X(int);
    X(const X&);
    X& operator=(const X&);
    ~X();
};

class Y {
public:
    Y(int);
    Y(Y&&);
    ~Y();
};

X f(X);
Y g(Y);

void h() {
    X a(1);
    X b = f(X(2));
    Y c = g(Y(3));
    a = f(a);
}
```

`X(2)` is constructed in the space used to hold `f()`'s argument and `Y(3)` is constructed in the space used to hold `g()`'s argument. Likewise, `f()`'s result is constructed directly in `b` and `g()`'s result is constructed directly in `c`. On the other hand, the expression `a = f(a)` requires a temporary for the result of `f(a)`, which is materialized so that the reference parameter of `X::operator=(const X&)` can bind to it. — *end example*]

- 3 When an object of type `X` is passed to or returned from a potentially evaluated function call, if `X` is
 - (3.1) — a scalar type or
 - (3.2) — a class type that has at least one eligible copy or move constructor (11.4.4), where each such constructor is trivial, and the destructor of `X` is either trivial or deleted,
 implementations are permitted to create temporary objects to hold the function parameter or result object, as follows:
 - (3.3) — The first such temporary object is constructed from the function argument or return value, respectively.

- (3.4) — Each successive temporary object is initialized from the previous one as if by direct-initialization if **X** is a scalar type, otherwise by using an eligible trivial constructor.
- (3.5) — The function parameter or return object is initialized from the final temporary as if by direct-initialization if **X** is a scalar type, otherwise by using an eligible trivial constructor.

(In all cases, the eligible constructor is used even if that constructor is inaccessible or would not be selected by overload resolution to perform a copy or move of the object).

[*Note 3*: This latitude is granted to allow objects to be passed to or returned from functions in registers. — *end note*]

- 4 Temporary objects are destroyed as the last step in evaluating the full-expression (6.10.1) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.
- 5 Temporary objects are destroyed at a different point than the end of the full-expression in the following contexts: The first context is when a default constructor is called to initialize an element of an array with no corresponding initializer (9.5). The second context is when a copy constructor is called to copy an element of an array while the entire array is copied (7.5.6.3, 11.4.5.3). In either case, if the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.
- 6 The third context is when a reference binds to a temporary object.²⁰ The temporary object to which the reference is bound or the temporary object that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference if the glvalue to which the reference is bound was obtained through one of the following:
 - (6.1) — a temporary materialization conversion (7.3.5),
 - (6.2) — (*expression*), where *expression* is one of these expressions,
 - (6.3) — subscripting (7.6.1.2) of an array operand, where that operand is one of these expressions,
 - (6.4) — a class member access (7.6.1.5) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member (11.4.1) of non-reference type or a direct base class relationship (11.7.1),
 - (6.5) — a pointer-to-member operation (7.6.4) using the .* operator where the left operand is one of these expressions and the right operand is a pointer to data member of non-reference type,
 - (6.6) — a
 - (6.6.1) — `const_cast` (7.6.1.11),
 - (6.6.2) — `static_cast` (7.6.1.9),
 - (6.6.3) — `dynamic_cast` (7.6.1.7), or
 - (6.6.4) — `reinterpret_cast` (7.6.1.10)
 converting, without a user-defined conversion, a glvalue operand that is one of these expressions to a glvalue that refers to the object designated by the operand, or to its complete object or a subobject thereof,
 - (6.7) — an explicit type conversion (functional notation) (7.6.1.4) to a reference type whose initializer is a *braced-init-list* where the reference is
 - (6.7.1) — bound directly to the glvalue result of one of these expressions (necessarily the sole element of the *braced-init-list*) or
 - (6.7.2) — bound to the result of a temporary materialization conversion,
 - (6.8) — a conditional expression (7.6.16) that is a glvalue where the second or third operand is one of these expressions, or
 - (6.9) — a comma expression (7.6.20) that is a glvalue where the right operand is one of these expressions.

[*Example 2*:

```
template<typename T> using id = T;
```

²⁰) The same rules apply to initialization of an `initializer_list` object (9.5.5) with its underlying temporary array.

```

int i = 1;
int&& a = id<int[3]>{1, 2, 3}[i];           // temporary array has same lifetime as a
const int& b = static_cast<const int&>(0); // temporary int has same lifetime as b
int&& c = cond ? id<int[3]>{1, 2, 3}[i] : static_cast<int&&>(0);
                                           // exactly one of the two temporaries is lifetime-extended

```

— end example]

[Note 4: An explicit type conversion (7.6.1.4, 7.6.3) is interpreted as a sequence of elementary casts, covered above.

[Example 3:

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

— end example]

— end note]

[Note 5: If a temporary object has a reference member initialized by another temporary object, lifetime extension applies recursively to such a member's initializer.

[Example 4:

```

struct S {
    const int& m;
};
const S& s = S{1};           // both S and int temporaries have lifetime of s

```

— end example]

— end note]

The exceptions to this lifetime rule are:

- (6.10) — A temporary object bound to a reference parameter in a function call (7.6.1.3) persists until the completion of the full-expression containing the call.
- (6.11) — A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized *expression-list* (9.5) persists until the completion of the full-expression containing the *expression-list*.
- (6.12) — A temporary bound to a reference in a *new-initializer* (7.6.2.8) persists until the completion of the full-expression containing the *new-initializer*.

[Note 6: This might introduce a dangling reference. — end note]

[Example 5:

```

struct S { int mi; const std::pair<int,int>& mp; };
S a { 1, {2,3} };
S* p = new S{ 1, {2,3} };           // creates dangling reference

```

— end example]

- 7 The fourth context is when a temporary object is created in the *for-range-initializer* of a range-based **for** statement. If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.
- 8 The fifth context is when a temporary object is created in an element *E* of the *expansion-init-list* of an enumerating expansion statement (8.7). If such a temporary object would otherwise be destroyed at the end of the full-expression of *E*, the object persists for the lifetime of the *for-range-declaration* initialized from *E*.
- 9 The sixth context is when a temporary object is created in the *expansion-initializer* of an iterating or destructuring expansion statement. If such a temporary object would otherwise be destroyed at the end of that *expansion-initializer*, the object persists for the lifetime of the reference initialized by the *expansion-initializer*, if any.
- 10 The seventh context is when a temporary object is created in a structured binding declaration (9.7). Any temporary objects introduced by the *initializers* for the variables with unique names are destroyed at the end of the structured binding declaration.
- 11 Let *x* and *y* each be either a temporary object whose lifetime is not extended, or a function parameter. If the lifetimes of *x* and *y* end at the end of the same full-expression, and *x* is initialized before *y*, then the destruction of *y* is sequenced before that of *x*. If the lifetime of two or more temporaries with lifetimes extending beyond the full-expressions in which they were created ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the

destruction of such temporaries shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (6.8.6.2, 6.8.6.3, 6.8.6.4); that is, if `obj1` is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before `obj1` is destroyed; if `obj2` is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after `obj2` is destroyed.

¹² [Example 6:

```
struct S {
    S();
    S(int);
    friend S operator+(const S&, const S&);
    ~S();
};
S obj1;
const S& cr = S(16)+S(23);
S obj2;
```

The expression `S(16) + S(23)` creates three temporaries: a first temporary `T1` to hold the result of the expression `S(16)`, a second temporary `T2` to hold the result of the expression `S(23)`, and a third temporary `T3` to hold the result of the addition of these two expressions. The temporary `T3` is then bound to the reference `cr`. It is unspecified whether `T1` or `T2` is created first. On an implementation where `T1` is created before `T2`, `T2` shall be destroyed before `T1`. The temporaries `T1` and `T2` are bound to the reference parameters of `operator+`; these temporaries are destroyed at the end of the full-expression containing the call to `operator+`. The temporary `T3` bound to the reference `cr` is destroyed at the end of `cr`'s lifetime, that is, at the end of the program. In addition, the order in which `T3` is destroyed takes into account the destruction order of other objects with static storage duration. That is, because `obj1` is constructed before `T3`, and `T3` is constructed before `obj2`, `obj2` shall be destroyed before `T3`, and `T3` shall be destroyed before `obj1`. — end example]

6.9 Types

[basic.types]

6.9.1 General

[basic.types.general]

- ¹ [Note 1: 6.9 and the subclauses thereof impose requirements on implementations regarding the representation of types. There are two kinds of types: fundamental types and compound types. Types describe objects (6.8.2), references (9.3.4.3), or functions (9.3.4.6). — end note]
- ² The *object representation* of a complete object type `T` is the sequence of N `unsigned char` objects taken up by a non-bit-field complete object of type `T`, where N equals `sizeof(T)`. The *value representation* of a type `T` is the set of bits in the object representation of `T` that participate in representing a value of type `T`. The object and value representation of a non-bit-field complete object are the bytes and bits, respectively, of the object corresponding to the object and value representation of its type. The object representation of a bit-field object is the sequence of N bits taken up by the object, where N is the width of the bit-field (11.4.10). The value representation of a bit-field object is the set of bits in the object representation that participate in representing its value. Bits in the object representation of a type or object that are not part of the value representation are *padding bits*.²¹
- ³ A class that has been declared but not defined, an enumeration type in certain contexts (9.8.1), or an array of unknown bound or of incomplete element type, is an *incompletely-defined object type*.²² Incompletely-defined object types and `cv void` are *incomplete types* (6.9.3).
- [Note 2: Objects cannot be defined to have an incomplete type (6.2). — end note]
- ⁴ A class type (such as “`class X`”) can be incomplete at one point in a translation unit and complete later on; the type “`class X`” is the same type at both points. The declared type of an array object can be an array of incomplete class type and therefore incomplete; if the class type is completed later on in the translation unit, the array type becomes complete; the array type at those two points is the same type. The declared type of an array object can be an array of unknown bound and therefore be incomplete at one point in a translation unit and complete later on; the array types at those two points (“array of unknown bound of `T`” and “array of `N T`”) are different types.

[Note 3: The type of a pointer or reference to array of unknown bound permanently points to or refers to an incomplete type. An array of unknown bound named by a `typedef` declaration permanently refers to an incomplete type. In either case, the array type cannot be completed. — end note]

²¹) The intent is that the memory model of C++ is compatible with that of the C programming language.

²²) The size and layout of an instance of an incompletely-defined object type is unknown.

[Example 1:

```

class X;                                // X is an incomplete type
extern X* xp;                            // xp is a pointer to an incomplete type
extern int arr[];                        // the type of arr is incomplete
typedef int UNKA[];                      // UNKA is an incomplete type
UNKA* arrp;                             // arrp is a pointer to an incomplete type
UNKA** arrpp;

void foo() {
    xp++;                                // error: X is incomplete
    arrp++;                             // error: incomplete type
    arrpp++;                            // OK, sizeof(UNKA*) is known
}

struct X { int i; };                    // now X is a complete type
int arr[10];                            // now the type of arr is complete

X x;
void bar() {
    xp = &x;                            // OK, type is "pointer to X"
    arrp = &arr;                        // OK, qualification conversion (7.3.6)
    xp++;                                // OK, X is complete
    arrp++;                             // error: UNKA can't be completed
}

```

— end example]

5 [Note 4: The rules for declarations and expressions describe in which contexts incomplete types are prohibited. — end note]

6 An *object type* is a (possibly cv-qualified) type that is not a function type, not a reference type, and not cv void.

7 Arithmetic types (6.9.3), enumeration types, pointer types, pointer-to-member types (6.9.5), `std::meta::info`, `std::nullptr_t`, and cv-qualified (6.9.6) versions of these types are collectively called *scalar types*. Scalar types, trivially copyable class types (11.2), arrays of such types, and cv-qualified versions of these types are collectively called *trivially copyable types*. Scalar types, standard-layout class types (11.2), arrays of such types, and cv-qualified versions of these types are collectively called *standard-layout types*. Scalar types, implicit-lifetime class types (11.2), array types, and cv-qualified versions of these types are collectively called *implicit-lifetime types*.

8 A type is a *literal type* if it is:

- (8.1) — cv void; or
- (8.2) — a scalar type; or
- (8.3) — a reference type; or
- (8.4) — an array of literal type; or
- (8.5) — a possibly cv-qualified class type (Clause 11) that has all of the following properties:
 - (8.5.1) — it has a constexpr destructor (9.2.6),
 - (8.5.2) — all of its non-variant non-static data members and base classes are of non-volatile literal types, and
 - (8.5.3) — it
 - (8.5.3.1) — is a closure type (7.5.6.2),
 - (8.5.3.2) — is an aggregate union type that has either no variant members or at least one variant member of non-volatile literal type,
 - (8.5.3.3) — is a non-union aggregate type for which each of its anonymous union members satisfies the above requirements for an aggregate union type, or
 - (8.5.3.4) — has at least one constexpr constructor or constructor template (possibly inherited (9.10) from a base class) that is not a copy or move constructor.

[Note 5: A literal type is one for which it might be possible to create an object within a constant expression. It is not a guarantee that it is possible to create such an object, nor is it a guarantee that any object of that type will be usable in a constant expression. — end note]

- ⁹ Two types *cv1* T1 and *cv2* T2 are *layout-compatible types* if T1 and T2 are the same type, layout-compatible enumerations (9.8.1), or layout-compatible standard-layout class types (11.4).

6.9.2 Trivially copyable types

[basic.types.trivial]

- ¹ Each trivially copyable type T has an implementation-defined set of discrete *values*. Each possible value representation of an object of type T corresponds to a distinct implementation-defined subset of this set. The union of these subsets is the set of values; for scalar types other than object pointer types, each such subset contains no more than one value. Certain operations cause an object to *acquire* a value representation, in which case the object's value is replaced with an unspecified member of the corresponding subset that would result in the program having defined behavior, if any.
- ² [Note 1: A single subset for a pointer type can contain pointers to multiple objects in each of several regions of storage whose durations are disjoint. — end note]
- ³ If an object of such a type T is not a potentially-overlapping subobject, whether or not the object holds a valid value of type T, the underlying bytes (6.8.1) making up the object can be copied into an array of `char`, `unsigned char`, or `std::byte` (17.2.1).²³ If the content of that array is copied back into the object, the object acquires its original value representation.

[Example 1:

```
constexpr std::size_t N = sizeof(T);
char buf[N];
T obj;                                // obj initialized to its original value
std::memcpy(buf, &obj, N);           // between these two calls to std::memcpy, obj might be modified
std::memcpy(&obj, buf, N);           // at this point, each subobject of obj of scalar type holds its original value
```

— end example]

- ⁴ For two distinct such objects `obj1` and `obj2`, if the underlying bytes (6.8.1) making up `obj1` are copied into `obj2`,²⁴ `obj2` acquires the value representation of `obj1`.

[Example 2:

```
T* t1p;
T* t2p;
// provided that t2p points to an initialized object ...
std::memcpy(t1p, t2p, sizeof(T));
// at this point, every subobject of trivially copyable type in *t1p contains
// the same value as the corresponding subobject in *t2p
```

— end example]

6.9.3 Fundamental types

[basic.fundamental]

- ¹ There are five *standard signed integer types*: “signed char”, “short int”, “int”, “long int”, and “long long int”. In this list, each type provides at least as much storage as those preceding it in the list. There may also be implementation-defined *extended signed integer types*. The standard and extended signed integer types are collectively called *signed integer types*. The range of representable values for a signed integer type is -2^{N-1} to $2^{N-1} - 1$ (inclusive), where N is called the *width* of the type.

[Note 1: Plain `ints` are intended to have the natural width suggested by the architecture of the execution environment; the other signed integer types are provided to meet special needs. — end note]

- ² For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: “unsigned char”, “unsigned short int”, “unsigned int”, “unsigned long int”, and “unsigned long long int”. Likewise, for each of the extended signed integer types, there exists a corresponding *extended unsigned integer type*. The standard and extended unsigned integer types are collectively called *unsigned integer types*. An unsigned integer type has the same width N as the corresponding signed integer type. The range of representable values for the unsigned type is 0 to $2^N - 1$ (inclusive); arithmetic for the unsigned type is performed modulo 2^N .

²³ By using, for example, the library functions (16.4.2.3) `std::memcpy` or `std::memmove`.

²⁴ By using, for example, the library functions (16.4.2.3) `std::memcpy` or `std::memmove`.

[Note 2: Unsigned arithmetic does not overflow. Overflow for signed arithmetic yields undefined behavior (7.1). — end note]

- ³ An unsigned integer type has the same object representation, value representation, and alignment requirements (6.8.3) as the corresponding signed integer type. For each value x of a signed integer type, the value of the corresponding unsigned integer type congruent to x modulo 2^N has the same value of corresponding bits in its value representation.²⁵

[Example 1: The value -1 of a signed integer type has the same representation as the largest value of the corresponding unsigned type. — end example]

Table 14 — Minimum width [tab:basic.fundamental.width]

Type	Minimum width N
signed char	8
short int	16
int	16
long int	32
long long int	64

- ⁴ The width of each standard signed integer type shall not be less than the values specified in Table 14. The value representation of a signed or unsigned integer type comprises N bits, where N is the respective width. Each set of values for any padding bits (6.9.1) in the object representation are alternative representations of the value specified by the value representation.

[Note 3: Padding bits have unspecified value, but cannot cause traps. In contrast, see ISO/IEC 9899:2024 6.2.6.2. — end note]

[Note 4: The signed and unsigned integer types satisfy the constraints given in ISO/IEC 9899:2024 5.3.5.3.2. — end note]

Except as specified above, the width of a signed or unsigned integer type is implementation-defined.

- ⁵ Each value x of an unsigned integer type with width N has a unique representation $x = x_0 2^0 + x_1 2^1 + \dots + x_{N-1} 2^{N-1}$, where each coefficient x_i is either 0 or 1; this is called the *base-2 representation* of x . The base-2 representation of a value of signed integer type is the base-2 representation of the congruent value of the corresponding unsigned integer type. The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, and the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*.
- ⁶ A fundamental type specified to have a signed or unsigned integer type as its *underlying type* has the same object representation, value representation, alignment requirements (6.8.3), and range of representable values as the underlying type. Further, each value has the same representation in both types.
- ⁷ Type `char` is a distinct type that has an implementation-defined choice of “signed char” or “unsigned char” as its underlying type. The three types `char`, `signed char`, and `unsigned char` are collectively called *ordinary character types*. The ordinary character types and `char8_t` are collectively called *narrow character types*. For narrow character types, each possible bit pattern of the object representation represents a distinct value.

[Note 5: This requirement does not hold for other types. — end note]

[Note 6: A bit-field of narrow character type whose width is larger than the width of that type has padding bits; see 6.9.1. — end note]

- ⁸ Type `wchar_t` is a distinct type that has an implementation-defined signed or unsigned integer type as its underlying type.
- ⁹ Type `char8_t` is a distinct type whose underlying type is `unsigned char`. Types `char16_t` and `char32_t` are distinct types whose underlying types are `std::uint_least16_t` and `std::uint_least32_t`, respectively, in `<stdint>` (17.4.1).
- ¹⁰ Type `bool` is a distinct type that has the same object representation, value representation, and alignment requirements as an implementation-defined unsigned integer type. The values of type `bool` are `true` and `false`.

²⁵) This is also known as two’s complement representation.

[Note 7: There are no **signed**, **unsigned**, **short**, or **long bool** types or values. — end note]

- 11 The types **char**, **wchar_t**, **char8_t**, **char16_t**, and **char32_t** are collectively called *character types*. The character types, **bool**, the signed and unsigned integer types, and cv-qualified versions (6.9.6) thereof, are collectively termed *integral types*. A synonym for integral type is *integer type*.

[Note 8: Enumerations (9.8.1) are not integral; however, unscoped enumerations can be promoted to integral types as specified in 7.3.7. — end note]

- 12 The three distinct types **float**, **double**, and **long double** can represent floating-point numbers. The type **double** provides at least as much precision as **float**, and the type **long double** provides at least as much precision as **double**. The set of values of the type **float** is a subset of the set of values of the type **double**; the set of values of the type **double** is a subset of the set of values of the type **long double**. The types **float**, **double**, and **long double**, and cv-qualified versions (6.9.6) thereof, are collectively termed *standard floating-point types*. An implementation may also provide additional types that represent floating-point values and define them (and cv-qualified versions thereof) to be *extended floating-point types*. The standard and extended floating-point types are collectively termed *floating-point types*.

[Note 9: Any additional implementation-specific types representing floating-point values that are not defined by the implementation to be extended floating-point types are not considered to be floating-point types, and this document imposes no requirements on them or their interactions with floating-point types. — end note]

Except as specified in 6.9.4, the object and value representations and accuracy of operations of floating-point types are implementation-defined.

- 13 The minimum range of representable values for a floating-point type is the most negative finite floating-point number representable in that type through the most positive finite floating-point number representable in that type. In addition, if negative infinity is representable in a type, the range of representable values of that type is extended to all negative real numbers; likewise, if positive infinity is representable in a type, the range of representable values of that type is extended to all positive real numbers.

[Note 10: Since negative and positive infinity are representable in ISO/IEC 60559 formats, all real numbers (but not infinity or NaN) lie within the range of representable values of a floating-point type adhering to ISO/IEC 60559. — end note]

- 14 Integral and floating-point types are collectively termed *arithmetic types*.

[Note 11: Properties of the arithmetic types, such as their minimum and maximum representable value, can be queried using the facilities in the standard library headers `<limits>` (17.3.3), `<climits>` (17.3.6), and `<cfloat>` (17.3.7). — end note]

- 15 A type *cv void* is an incomplete type that cannot be completed; such a type has an empty set of values.

[Note 12: An expression of type *cv void* can be used as

- (15.1) — the *expression* of an expression statement (8.3),
- (15.2) — the operand of a **return** statement (8.8.4) for a function with the return type *cv void*,
- (15.3) — an operand of a comma expression (7.6.20),
- (15.4) — the operand of a parenthesized expression (7.5.4),
- (15.5) — a requirement in a *requires-expression* (7.5.8.1),
- (15.6) — the second or third operand of **?:** (7.6.16),
- (15.7) — the operand of a **typeid** expression (7.6.1.8),
- (15.8) — the operand of a **noexcept** operator (7.6.2.7),
- (15.9) — the operand of a **decltype** specifier (9.2.9.6), or
- (15.10) — the operand of an explicit conversion to type *cv void* (7.6.1.4, 7.6.1.9, 7.6.3).

— end note]

- 16 The types denoted by *cv std::nullptr_t* are distinct types. A prvalue of type **std::nullptr_t** is a null pointer constant (7.3.12). Such values participate in the pointer and the pointer-to-member conversions (7.3.12, 7.3.13). The size (7.6.2.5) and alignment requirement (6.8.3) of the type **std::nullptr_t** are those of the type “pointer to void”.

[Note 13: The value representation can comprise no bits (7.3.2). — end note]

- 17 A value of type **std::meta::info** is called a *reflection*. There exists a unique *null reflection*; every other reflection is a representation of

- (17.1) — a value of scalar type (13.2),
- (17.2) — an object with static storage duration (6.8.6),
- (17.3) — a variable (6.1),
- (17.4) — a structured binding (9.7),
- (17.5) — a function (9.3.4.6),
- (17.6) — a function parameter,
- (17.7) — an enumerator (9.8.1),
- (17.8) — an annotation (9.13.1),
- (17.9) — a type alias (9.2.4),
- (17.10) — a type (6.9),
- (17.11) — a class member (11.4),
- (17.12) — an unnamed bit-field (11.4.10),
- (17.13) — a class template (13.1),
- (17.14) — a function template,
- (17.15) — a variable template,
- (17.16) — an alias template (13.7.8),
- (17.17) — a concept (13.7.9),
- (17.18) — a namespace alias (9.9.3),
- (17.19) — a namespace (9.9.1),
- (17.20) — a direct base class relationship (11.7.1), or
- (17.21) — a data member description (11.4.1).

A reflection is said to *represent* the corresponding construct.

[*Note 14:* A reflection of a value can be produced by library functions such as `std::meta::constant_of` and `std::meta::reflect_constant`. — *end note*]

[*Example 2:*

```
int arr[] = {1, 2, 3};
auto [a1, a2, a3] = arr;
[[=1]] void fn(int n);
enum Enum { A };
using Alias = int;
struct B {};
struct S : B {
    int mem;
    int : 0;
};
template<auto> struct TCls {};
template<auto> void TFn();
template<auto> int TVar;
template<auto N> using TAlias = TCls<N>;
template<auto> concept Concept = requires { true; };
namespace NS {};
namespace NSAlias = NS;

constexpr auto ctx = std::meta::access_context::current();

constexpr auto r1 = std::meta::reflect_constant(42);           // represents int value of 42
constexpr auto r2 = std::meta::reflect_object(arr[1]);        // represents int object
constexpr auto r3 = ^^arr;                                     // represents a variable
constexpr auto r4 = ^^a3;                                       // represents a structured binding
constexpr auto r5 = ^^fn;                                       // represents a function
constexpr auto r6 = std::meta::parameters_of(^^fn)[0];        // represents a function parameter
constexpr auto r7 = ^^Enum::A;                                  // represents an enumerator
```

```

constexpr auto r8 = std::meta::annotations_of(^fn)[0]; // represents an annotation
constexpr auto r9 = ^Alias; // represents a type alias
constexpr auto r10 = ^S; // represents a type
constexpr auto r11 = ^S::mem; // represents a class member
constexpr auto r12 = std::meta::members_of(^S, ctx)[1]; // represents an unnamed bit-field
constexpr auto r13 = ^TClS; // represents a class template
constexpr auto r14 = ^TFn; // represents a function template
constexpr auto r15 = ^TVar; // represents a variable template
constexpr auto r16 = ^TAlias; // represents an alias template
constexpr auto r17 = ^Concept; // represents a concept
constexpr auto r18 = ^NSAlias; // represents a namespace alias
constexpr auto r19 = ^NS; // represents a namespace
constexpr auto r20 = std::meta::bases_of(^S, ctx)[0]; // represents a direct base class relationship
constexpr auto r21 =
    std::meta::data_member_spec(^int, {.name="member"}); // represents a data member description
— end example]

```

- 18 *Recommended practice:* Implementations should not represent other constructs specified in this document, such as *using-declarators*, partial template specializations, attributes, placeholder types, statements, or expressions, as values of type `std::meta::info`.

[Note 15: Future revisions of this document can specify semantics for reflections representing any such constructs. — end note]

- 19 The types described in this subclause are called *fundamental types*.

[Note 16: Even if the implementation defines two or more fundamental types to have the same value representation, they are nevertheless different types. — end note]

6.9.4 Optional extended floating-point types

[basic.extended.fp]

- 1 If the implementation supports an extended floating-point type (6.9.3) whose properties are specified by the ISO/IEC 60559 floating-point interchange format binary16, then the type alias `std::float16_t` is declared in the header `<stdfloat>` (17.4.2) and names such a type, the macro `__STDCPP_FLOAT16_T__` is defined (15.12), and the floating-point literal suffixes `f16` and `F16` are supported (5.13.4).
- 2 If the implementation supports an extended floating-point type whose properties are specified by the ISO/IEC 60559 floating-point interchange format binary32, then the type alias `std::float32_t` is declared in the header `<stdfloat>` and names such a type, the macro `__STDCPP_FLOAT32_T__` is defined, and the floating-point literal suffixes `f32` and `F32` are supported.
- 3 If the implementation supports an extended floating-point type whose properties are specified by the ISO/IEC 60559 floating-point interchange format binary64, then the type alias `std::float64_t` is declared in the header `<stdfloat>` and names such a type, the macro `__STDCPP_FLOAT64_T__` is defined, and the floating-point literal suffixes `f64` and `F64` are supported.
- 4 If the implementation supports an extended floating-point type whose properties are specified by the ISO/IEC 60559 floating-point interchange format binary128, then the type alias `std::float128_t` is declared in the header `<stdfloat>` and names such a type, the macro `__STDCPP_FLOAT128_T__` is defined, and the floating-point literal suffixes `f128` and `F128` are supported.
- 5 If the implementation supports an extended floating-point type with the properties, as specified by ISO/IEC 60559, of radix (b) of 2, storage width in bits (k) of 16, precision in bits (p) of 8, maximum exponent ($emax$) of 127, and exponent field width in bits (w) of 8, then the type alias `std::bfloat16_t` is declared in the header `<stdfloat>` and names such a type, the macro `__STDCPP_BFLOAT16_T__` is defined, and the floating-point literal suffixes `bf16` and `BF16` are supported.
- 6 [Note 1: A summary of the parameters for each type is given in Table 15. The precision p includes the implicit 1 bit at the beginning of the significand, so the storage used for the significand is $p - 1$ bits. ISO/IEC 60559 does not assign a name for a type having the parameters specified for `std::bfloat16_t`. — end note]
- 7 *Recommended practice:* Any names that the implementation provides for the extended floating-point types described in this subsection that are in addition to the names declared in the `<stdfloat>` header should be chosen to increase compatibility and interoperability with the interchange floating types `_Float16`, `_Float32`, `_Float64`, and `_Float128` defined in ISO/IEC 9899:2024 H.2.2.

Table 15 — Properties of named extended floating-point types [tab:basic.extended.fp]

Parameter	float16_t	float32_t	float64_t	float128_t	bfloat16_t
ISO/IEC 60559 name	binary16	binary32	binary64	binary128	
<i>k</i> , storage width in bits	16	32	64	128	16
<i>p</i> , precision in bits	11	24	53	113	8
<i>emax</i> , maximum exponent	15	127	1023	16383	127
<i>w</i> , exponent field width in bits	5	8	11	15	8

6.9.5 Compound types

[basic.compound]

¹ Compound types can be constructed in the following ways:

- (1.1) — *arrays* of objects of a given type, 9.3.4.5;
- (1.2) — *functions*, which have parameters of given types and return `void` or a result of a given type, 9.3.4.6;
- (1.3) — *pointers* to *cv* `void` or objects or functions (including static members of classes) of a given type, 9.3.4.2;
- (1.4) — *references* to objects or functions of a given type, 9.3.4.3. There are two types of references:
 - (1.4.1) — lvalue reference
 - (1.4.2) — rvalue reference
- (1.5) — *classes* containing a sequence of class members (Clause 11, 11.4), and a set of restrictions on the access to these entities (11.8);
- (1.6) — *unions*, which are classes capable of containing objects of different types at different times, 11.5;
- (1.7) — *enumerations*, which comprise a set of named constant values, 9.8.1;
- (1.8) — *pointers to non-static class members*,²⁶ which identify members of a given type within objects of a given class, 9.3.4.4. Pointers to data members and pointers to member functions are collectively called *pointer-to-member* types.

² These methods of constructing types can be applied recursively; restrictions are mentioned in 9.3.4. Constructing a type such that the number of bytes in its object representation exceeds the maximum value representable in the type `std::size_t` (17.2) is ill-formed.

³ The type of a pointer to *cv* `void` or a pointer to an object type is called an *object pointer type*.

[Note 1: A pointer to `void` does not have a pointer-to-object type, however, because `void` is not an object type. — end note]

The type of a pointer that can designate a function is called a *function pointer type*. A pointer to an object of type `T` is referred to as a “pointer to `T`”.

[Example 1: A pointer to an object of type `int` is referred to as “pointer to `int`” and a pointer to an object of class `X` is called a “pointer to `X`”. — end example]

Except for pointers to static members, text referring to “pointers” does not apply to pointers to members. Pointers to incomplete types are allowed although there are restrictions on what can be done with them (6.9.1). Every value of pointer type is one of the following:

- (3.1) — a *pointer to* an object or function (the pointer is said to *point* to the object or function), or
- (3.2) — a *pointer past the end of* an object (7.6.6), or
- (3.3) — the *null pointer value* for that type, or
- (3.4) — an *invalid pointer value*.

A value of a pointer type that is a pointer to or past the end of an object *represents the address* of the first byte in memory (6.8.1) occupied by the object²⁷ or the first byte in memory after the end of the storage occupied by the object, respectively.

[Note 2: A pointer past the end of an object (7.6.6) is not considered to point to an unrelated object of the object’s type, even if the unrelated object is located at that address. — end note]

²⁶) Static class members are objects or functions, and pointers to them are ordinary pointers to objects or functions.

²⁷) For an object that is not within its lifetime, this is the first byte in memory that it will occupy or used to occupy.

For purposes of pointer arithmetic (7.6.6) and comparison (7.6.9, 7.6.10), a pointer past the end of the last element of an array x of n elements is considered to be equivalent to a pointer to a hypothetical array element n of x , and an object of type T that is not an array element is considered to belong to an array with one element of type T . The value representation of pointer types is implementation-defined. Pointers to layout-compatible types shall have the same value representation and alignment requirements (6.8.3).

[Note 3: Pointers to over-aligned types (6.8.3) have no special representation, but their range of valid values is restricted by the extended alignment requirement. — end note]

- 4 A pointer value pointing to a potentially non-unique object O (6.8.2) is *associated with* the evaluation of
- (4.1) — the *string-literal* (5.13.5) that resulted in the string literal object,
 - (4.2) — the initializer list (9.5.5) that resulted in the backing array, or
 - (4.3) — the initialization of the template parameter object (13.4.3, 21.4.3)

that is O or of which O is a subobject.

[Note 4: A pointer value obtained by pointer arithmetic (7.6.6) from a pointer value associated with an evaluation E is also associated with E . — end note]

- 5 If an evaluation produces or causes an object to have (6.9.2) a pointer value to or past the end of an object O and happens before the beginning of the duration of the region of storage for O , the behavior is undefined.

[Note 5: Relaxed atomic operations (32.5.4) can produce such values. Conversions from integers avoid producing them (7.6.1.10). — end note]

- 6 A pointer value P is *valid in the context of* an evaluation E if P is a pointer to function or a null pointer value, or if it is a pointer to or past the end of an object O and E happens after the beginning and happens before the end of the duration of the region of storage for O . If a pointer value P is used in an evaluation E and P is not valid in the context of E :

- (6.1) — If E is an indirection (7.6.2.2), the behavior is undefined (F.2.17).
- (6.2) — If E either
 - (6.2.1) — is a boolean conversion (7.3.15) or
 - (6.2.2) — is performed by a unary $+$, additive, three-way comparison, relational, or equality operator (7.6),
 the behavior is implementation-defined.

[Note 6: P can be valid in the context of E even if it points to a type unrelated to that of O or if O is not within its lifetime, although further restrictions apply to such pointer values (6.8.4, 7.2.1, 7.6.6). — end note]

- 7 Two objects a and b are *pointer-interconvertible* if

- (7.1) — they are the same object, or
- (7.2) — one is a union object and the other is a non-static data member of that object (11.5), or
- (7.3) — one is a standard-layout class object and the other is the first non-static data member of that object or any base class subobject of that object (11.4), or
- (7.4) — there exists an object c such that a and c are pointer-interconvertible, and c and b are pointer-interconvertible.

If two objects are pointer-interconvertible, then they have the same address, and it is possible to obtain a pointer to one from a pointer to the other via a `reinterpret_cast` (7.6.1.10).

[Note 7: An array object and its first element are not pointer-interconvertible, even though they have the same address. — end note]

- 8 A byte of storage b is *reachable through* a pointer value that points to an object x if there is an object y , pointer-interconvertible with x , such that b is within the storage occupied by y , or the immediately-enclosing array object if y is an array element.
- 9 A pointer to *cv void* can be used to point to objects of unknown type. Such a pointer shall be able to hold any object pointer. An object of type “pointer to *cv void*” shall have the same representation and alignment requirements as an object of type “pointer to *cv char*”.

6.9.6 CV-qualifiers

[basic.type.qualifier]

- 1 Each type other than a function or reference type is part of a group of four distinct, but related, types: a *cv-unqualified* version, a *const-qualified* version, a *volatile-qualified* version, and a *const-volatile-qualified*

version. The types in each such group shall have the same representation and alignment requirements (6.8.3).²⁸ A function or reference type is always cv-unqualified.

- (1.1) — A *const object* is an object of type `const T` or a non-mutable subobject of a *const object*.
- (1.2) — A *volatile object* is an object of type `volatile T` or a subobject of a *volatile object*.
- (1.3) — A *const volatile object* is an object of type `const volatile T`, a non-mutable subobject of a *const volatile object*, a *const* subobject of a *volatile object*, or a non-mutable *volatile* subobject of a *const object*.

[Note 1: The type of an object (6.8.2) includes the *cv-qualifiers* specified in the *decl-specifier-seq* (9.2), *declarator* (9.3), *type-id* (9.3.2), or *new-type-id* (7.6.2.8) when the object is created. — end note]

- ² Except for array types, a compound type (6.9.5) is not cv-qualified by the cv-qualifiers (if any) of the types from which it is compounded.
- ³ An array type whose elements are cv-qualified is also considered to have the same cv-qualifications as its elements.

[Note 2: Cv-qualifiers applied to an array type attach to the underlying element type, so the notation “*cv T*”, where *T* is an array type, refers to an array whose elements are so-qualified (9.3.4.5). — end note]

[Example 1:

```
typedef char CA[5];
typedef const char CC;
CC arr1[5] = { 0 };
const CA arr2 = { 0 };
```

The type of both `arr1` and `arr2` is “array of 5 `const char`”, and the array type is considered to be *const*-qualified. — end example]

- ⁴ [Note 3: See 9.3.4.6 and 12.2.2 regarding function types that have *cv-qualifiers*. — end note]
- ⁵ There is a partial ordering on cv-qualifiers, so that a type can be said to be *more cv-qualified* than another. Table 16 shows the relations that constitute this ordering.

Table 16 — Relations on *const* and *volatile* [tab:basic.type.qualifier.rel]

<i>no cv-qualifier</i>	<	<code>const</code>
<i>no cv-qualifier</i>	<	<code>volatile</code>
<i>no cv-qualifier</i>	<	<code>const volatile</code>
<code>const</code>	<	<code>const volatile</code>
<code>volatile</code>	<	<code>const volatile</code>

- ⁶ In this document, the notation *cv* (or *cv1*, *cv2*, etc.), used in the description of types, represents an arbitrary set of cv-qualifiers, i.e., one of {`const`}, {`volatile`}, {`const`, `volatile`}, or the empty set. For a type *cv T*, the *top-level cv-qualifiers* of that type are those denoted by *cv*.

[Example 2: The type corresponding to the *type-id* `const int&` has no top-level cv-qualifiers. The type corresponding to the *type-id* `volatile int * const` has the top-level cv-qualifier `const`. For a class type *C*, the type corresponding to the *type-id* `void (C::* volatile)(int) const` has the top-level cv-qualifier `volatile`. — end example]

6.9.7 Conversion ranks

[conv.rank]

- ¹ Every integer type has an *integer conversion rank* defined as follows:

- (1.1) — No two signed integer types have the same rank, even if they have the same representation.
- (1.2) — The rank of a signed integer type is greater than the rank of any signed integer type with a smaller width.
- (1.3) — The rank of `long long int` is greater than the rank of `long int`, which is greater than the rank of `int`, which is greater than the rank of `short int`, which is greater than the rank of `signed char`.
- (1.4) — The rank of any unsigned integer type equals the rank of the corresponding signed integer type.

²⁸) The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and non-static data members of unions.

- (1.5) — The rank of any standard integer type is greater than the rank of any extended integer type with the same width.
- (1.6) — The rank of `bool` is less than the rank of all other integer types.
- (1.7) — The ranks of the character types equal the ranks of their corresponding underlying types (6.9.3).
- (1.8) — The rank of any extended signed integer type relative to another extended signed integer type with the same width is implementation-defined, but still subject to the other rules for determining the integer conversion rank.
- (1.9) — For all integer types `T1`, `T2`, and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` has greater rank than `T3`.

[*Note 1:* The integer conversion rank is used in the definition of the integral promotions (7.3.7) and the usual arithmetic conversions (7.4). — *end note*]

² Every floating-point type has a *floating-point conversion rank* defined as follows:

- (2.1) — The rank of a floating-point type `T` is greater than the rank of any floating-point type whose set of values is a proper subset of the set of values of `T`.
- (2.2) — The rank of `long double` is greater than the rank of `double`, which is greater than the rank of `float`.
- (2.3) — Two extended floating-point types with the same set of values have equal ranks.
- (2.4) — An extended floating-point type with the same set of values as exactly one cv-unqualified standard floating-point type has a rank equal to the rank of that standard floating-point type.
- (2.5) — An extended floating-point type with the same set of values as more than one cv-unqualified standard floating-point type has a rank equal to the rank of `double`.

[*Note 2:* The treatment of `std::float64_t` differs from that of the analogous `_Float64` in C, for example on platforms where all of `long double`, `double`, and `std::float64_t` have the same set of values (see ISO/IEC 9899:2024 H.4.3). — *end note*]

[*Note 3:* The conversion ranks of floating-point types `T1` and `T2` are unordered if the set of values of `T1` is neither a subset nor a superset of the set of values of `T2`. This can happen when one type has both a larger range and a lower precision than the other. — *end note*]

³ Floating-point types that have equal floating-point conversion ranks are ordered by floating-point conversion subrank. The subrank forms a total order among types with equal ranks. The types `std::float16_t`, `std::float32_t`, `std::float64_t`, and `std::float128_t` (17.4.2) have a greater conversion subrank than any standard floating-point type with equal conversion rank. Otherwise, the conversion subrank order is implementation-defined.

⁴ [*Note 4:* The floating-point conversion rank and subrank are used in the definition of the usual arithmetic conversions (7.4). — *end note*]

6.10 Program execution

[basic.exec]

6.10.1 Sequential execution

[intro.execution]

- ¹ An instance of each object with automatic storage duration (6.8.6.4) is associated with each entry into its block. Such an object exists and retains its last-stored value during the execution of the block and while the block is suspended (by a call of a function, suspension of a coroutine (7.6.2.4), or receipt of a signal).
- ² A *constituent expression* is defined as follows:
 - (2.1) — The constituent expression of an expression is that expression.
 - (2.2) — The constituent expression of a conversion is the corresponding implicit function call, if any, or the converted expression otherwise.
 - (2.3) — The constituent expressions of a *braced-init-list* or of a (possibly parenthesized) *expression-list* are the constituent expressions of the elements of the respective list.
 - (2.4) — The constituent expressions of a *brace-or-equal-initializer* of the form `= initializer-clause` are the constituent expressions of the *initializer-clause*.

[*Example 1:*

```
struct A { int x; };
struct B { int y; struct A a; };
B b = { 5, { 1+1 } };
```

The constituent expressions of the *initializer* used for the initialization of **b** are **5** and **1+1**. — *end example*]

³ The *immediate subexpressions* of an expression *E* are

- (3.1) — the constituent expressions of *E*'s operands (7.2),
- (3.2) — any function call that *E* implicitly invokes,
- (3.3) — if *E* is a *lambda-expression* (7.5.6), the initialization of the entities captured by copy and the constituent expressions of the *initializer* of the *init-captures*,
- (3.4) — if *E* is a function call (7.6.1.3) or implicitly invokes a function, the constituent expressions of each default argument (9.3.4.7) used in the call, and
- (3.5) — if *E* performs aggregate initialization (9.5.2), the constituent expressions of each default member initializer (11.4) used in the initialization.

⁴ A *subexpression* of an expression *E* is an immediate subexpression of *E* or a subexpression of an immediate subexpression of *E*.

[*Note 1*: Expressions appearing in the *compound-statement* of a *lambda-expression* are not subexpressions of the *lambda-expression*. — *end note*]

The *potentially-evaluated subexpressions* of an expression, conversion, or *initializer* *E* are

- (4.1) — the constituent expressions of *E* and
- (4.2) — the subexpressions thereof that are not subexpressions of a nested unevaluated operand (7.2.3).

⁵ A *full-expression* is

- (5.1) — an unevaluated operand (7.2.3),
- (5.2) — a *constant-expression* (7.7.2),
- (5.3) — an immediate invocation (7.7.5),
- (5.4) — an *init-declarator* (9.3) (including such introduced by a structured binding (9.7)) or a *mem-initializer* (11.9.3), including the constituent expressions of the *initializer*,
- (5.5) — an invocation of a destructor generated at the end of the lifetime of an object other than a temporary object (6.8.7) whose lifetime has not been extended,
- (5.6) — the predicate of a contract assertion (6.11), or
- (5.7) — an expression that is not a subexpression of another expression and that is not otherwise part of a full-expression.

If a language construct is defined to produce an implicit call of a function, a use of the language construct is considered to be an expression for the purposes of this definition. Conversions applied to the result of an expression in order to satisfy the requirements of the language construct in which the expression appears are also considered to be part of the full-expression. For an *initializer*, performing the initialization of the entity (including evaluating default member initializers of an aggregate) is also considered part of the full-expression.

[*Example 2*:

```

struct S {
    S(int i): I(i) { }           // full-expression is initialization of I
    int& v() { return I; }
    ~S() noexcept(false) { }
private:
    int I;
};

S s1(1);                        // full-expression comprises call of S::S(int)
void f() {
    S s2 = 2;                   // full-expression comprises call of S::S(int)
    if (S(3).v())               // full-expression includes lvalue-to-rvalue and int to bool conversions,
                                // performed before temporary is deleted at end of full-expression
    { }
    bool b = noexcept(S(4));    // exception specification of destructor of S considered for noexcept

    // full-expression is destruction of s2 at end of block
}

```

```

struct B {
    B(S = S(0));
};
B b[2] = { B(), B() };           // full-expression is the entire initialization
                                // including the destruction of temporaries

```

— end example]

⁶ [Note 2: The evaluation of a full-expression can include the evaluation of subexpressions that are not lexically part of the full-expression. For example, subexpressions involved in evaluating default arguments (9.3.4.7) are considered to be created in the expression that calls the function, not the expression that defines the default argument. — end note]

⁷ Reading an object designated by a **volatile** glvalue (7.2.1), modifying an object, producing an injected declaration (7.7.6), calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution or translation environment. *Evaluation* of an expression (or a subexpression) in general includes both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access through a volatile glvalue is evaluated, the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the **volatile** access may not have completed yet.

⁸ *Sequenced before* is an asymmetric, transitive, pair-wise relation between evaluations executed by a single thread (6.10.2), which induces a partial order among those evaluations. Given any two evaluations *A* and *B*, if *A* is sequenced before *B* (or, equivalently, *B* is *sequenced after A*), then the execution of *A* shall precede the execution of *B*. If *A* is not sequenced before *B* and *B* is not sequenced before *A*, then *A* and *B* are *unsequenced*.

[Note 3: The execution of unsequenced evaluations can overlap. — end note]

Evaluations *A* and *B* are *indeterminately sequenced* when either *A* is sequenced before *B* or *B* is sequenced before *A*, but it is unspecified which.

[Note 4: Indeterminately sequenced evaluations cannot overlap, but either can be executed first. — end note]

An expression *X* is said to be sequenced before an expression *Y* if every value computation and every side effect associated with the expression *X* is sequenced before every value computation and every side effect associated with the expression *Y*.

⁹ Every value computation and side effect associated with a full-expression is sequenced before every value computation and side effect associated with the next full-expression to be evaluated.²⁹

¹⁰ Except where noted, evaluations of operands of individual operators and of subexpressions of individual expressions are unsequenced.

[Note 5: In an expression that is evaluated more than once during the execution of a program, unsequenced and indeterminately sequenced evaluations of its subexpressions need not be performed consistently in different evaluations. — end note]

The value computations of the operands of an operator are sequenced before the value computation of the result of the operator. The behavior is undefined (6.10.2) (F.2.18) if

(10.1) — a side effect on a memory location (6.8.1) or

(10.2) — starting or ending the lifetime of an object in a memory location

is unsequenced relative to

(10.3) — another side effect on the same memory location,

(10.4) — starting or ending the lifetime of an object occupying storage that overlaps with the memory location,
or

(10.5) — a value computation using the value of any object in the same memory location,

and the two evaluations are not potentially concurrent (6.10.2).

[Note 6: Starting the lifetime of an object in a memory location can end the lifetime of objects in other memory locations (6.8.4). — end note]

[Note 7: The next subclause imposes similar, but more complex restrictions on potentially concurrent computations. — end note]

²⁹ As specified in 6.8.7, after a full-expression is evaluated, a sequence of zero or more invocations of destructor functions for temporary objects takes place, usually in reverse order of the construction of each temporary object.

[Example 3:

```
void g(int i) {
    i = 7, i++, i++;           // i becomes 9

    i = i++ + 1;              // the value of i is incremented
    i = i++ + i;              // undefined behavior
    i = i + 1;                // the value of i is incremented

    union U { int x, y; } u;
    (u.x = 1, 0) + (u.y = 2, 0); // undefined behavior
}
```

— end example]

- 11 When invoking a function f (whether or not the function is inline), every argument expression and the postfix expression designating f are sequenced before every precondition assertion of the function call (7.6.1.3), which in turn are sequenced before every expression or statement in the body of f , which in turn are sequenced before every postcondition assertion of the function call.

- 12 For each

- (12.1) — function invocation,
- (12.2) — evaluation of an *await-expression* (7.6.2.4), or
- (12.3) — evaluation of a *throw-expression* (7.6.18)

F , each evaluation that does not occur within F but is evaluated on the same thread and as part of the same signal handler (if any) is either sequenced before all evaluations that occur within F or sequenced after all evaluations that occur within F ;³⁰ if F invokes or resumes a coroutine (7.6.2.4), only evaluations subsequent to the previous suspension (if any) and prior to the next suspension (if any) are considered to occur within F .

- 13 Several contexts in C++ cause evaluation of a function call, even though no corresponding function call syntax appears in the translation unit.

[Example 4: Evaluation of a *new-expression* invokes one or more allocation and constructor functions; see 7.6.2.8. For another example, invocation of a conversion function (11.4.8.3) can arise in contexts in which no function call syntax appears. — end example]

- 14 The sequencing constraints on the execution of the called function (as described above) are features of the function calls as evaluated, regardless of the syntax of the expression that calls the function.

- 15 If a signal handler is executed as a result of a call to the `std::raise` function, then the execution of the handler is sequenced after the invocation of the `std::raise` function and before its return.

[Note 8: When a signal is received for another reason, the execution of the signal handler is usually unsequenced with respect to the rest of the program. — end note]

- 16 During the evaluation of an expression as a core constant expression (7.7.2), evaluations of operands of individual operators and of subexpressions of individual expressions that are otherwise either unsequenced or indeterminately sequenced are evaluated in lexical order. For a function call (7.6.1.3), any default arguments used are evaluated in the order as if an argument were present for the corresponding parameter.

6.10.2 Multi-threaded executions and data races

[intro.multithread]

6.10.2.1 General

[intro.multithread.general]

- 1 A *thread of execution* (also known as a *thread*) is a single flow of control within a program, including the initial invocation of a specific top-level function, and recursively including every function invocation subsequently executed by the thread.

[Note 1: When one thread creates another, the initial call to the top-level function of the new thread is executed by the new thread, not by the creating thread. — end note]

Every thread in a program can potentially use every object and function in a program.³¹ Under a hosted implementation, a C++ program can have more than one thread running concurrently. The execution of each thread proceeds as defined by the remainder of this document. The execution of the entire program consists of an execution of all of its threads.

³⁰) In other words, function executions do not interleave with each other.

³¹) An object with automatic or thread storage duration (6.8.6) is associated with one specific thread, and can be accessed by a different thread only indirectly through a pointer or reference (6.9.5).

[*Note 2*: Usually the execution can be viewed as an interleaving of all its threads. However, some kinds of atomic operations, for example, allow executions inconsistent with a simple interleaving, as described below. — *end note*]

Under a freestanding implementation, it is implementation-defined whether a program can have more than one thread of execution.

- 2 For a signal handler that is not executed as a result of a call to the `std::raise` function, it is unspecified which thread of execution contains the signal handler invocation.

6.10.2.2 Data races

[**intro.races**]

- 1 The value of an object visible to a thread *T* at a particular point is the initial value of the object, a value assigned to the object by *T*, or a value assigned to the object by another thread, according to the rules below.

[*Note 1*: In some cases, there might instead be undefined behavior. Much of this subclause is motivated by the desire to support atomic operations with explicit and detailed visibility constraints. However, it also implicitly supports a simpler view for more restricted programs. — *end note*]

- 2 Two expression evaluations *conflict* if one of them

- (2.1) — modifies (3.1) a memory location (6.8.1) or
 - (2.2) — starts or ends the lifetime of an object in a memory location
- and the other one

- (2.3) — reads or modifies the same memory location or
- (2.4) — starts or ends the lifetime of an object occupying storage that overlaps with the memory location.

[*Note 2*: A modification can still conflict even if it does not alter the value of any bits. — *end note*]

- 3 The library defines a number of atomic operations (32.5) and operations on mutexes (Clause 32) that are specially identified as synchronization operations. These operations play a special role in making assignments in one thread visible to another. A synchronization operation on one or more memory locations is either an acquire operation, a release operation, or both an acquire and release operation. A synchronization operation without an associated memory location is a fence and can be either an acquire fence, a release fence, or both an acquire and release fence. In addition, there are relaxed atomic operations, which are not synchronization operations, and atomic read-modify-write operations, which have special characteristics.

[*Note 3*: For example, a call that acquires a mutex will perform an acquire operation on the locations comprising the mutex. Correspondingly, a call that releases the same mutex will perform a release operation on those same locations. Informally, performing a release operation on *A* forces prior side effects on other memory locations to become visible to other threads that later perform a consume or an acquire operation on *A*. “Relaxed” atomic operations are not synchronization operations even though, like synchronization operations, they cannot contribute to data races. — *end note*]

- 4 All modifications to a particular atomic object *M* occur in some particular total order, called the *modification order* of *M*.

[*Note 4*: There is a separate order for each atomic object. There is no requirement that these can be combined into a single total order for all objects. In general this will be impossible since different threads can observe modifications to different objects in inconsistent orders. — *end note*]

- 5 A *release sequence* headed by a release operation *A* on an atomic object *M* is a maximal contiguous subsequence of side effects in the modification order of *M*, where the first operation is *A*, and every subsequent operation is an atomic read-modify-write operation.

- 6 Certain library calls *synchronize with* other library calls performed by another thread. For example, an atomic store-release synchronizes with a load-acquire that takes its value from the store (32.5.4).

[*Note 5*: Except in the specified cases, reading a later value does not necessarily ensure visibility as described below. Such a requirement would sometimes interfere with efficient implementation. — *end note*]

[*Note 6*: The specifications of the synchronization operations define when one reads the value written by another. For atomic objects, the definition is clear. All operations on a given mutex occur in a single total order. Each mutex acquisition “reads the value written” by the last mutex release. — *end note*]

- 7 An evaluation *A* *happens before* an evaluation *B* (or, equivalently, *B* *happens after* *A*) if either

- (7.1) — *A* is sequenced before *B*, or
- (7.2) — *A* synchronizes with *B*, or
- (7.3) — *A* happens before *X* and *X* happens before *B*.

[Note 7: An evaluation does not happen before itself. — end note]

8 An evaluation *A* *strongly happens before* an evaluation *D* if, either

(8.1) — *A* is sequenced before *D*, or

(8.2) — *A* synchronizes with *D*, and both *A* and *D* are sequentially consistent atomic operations (32.5.4), or

(8.3) — there are evaluations *B* and *C* such that *A* is sequenced before *B*, *B* happens before *C*, and *C* is sequenced before *D*, or

(8.4) — there is an evaluation *B* such that *A* strongly happens before *B*, and *B* strongly happens before *D*.

[Note 8: Informally, if *A* strongly happens before *B*, then *A* appears to be evaluated before *B* in all contexts. — end note]

9 A *visible side effect* *A* on a scalar object or bit-field *M* with respect to a value computation *B* of *M* satisfies the conditions:

(9.1) — *A* happens before *B* and

(9.2) — there is no other side effect *X* on *M* such that *A* happens before *X* and *X* happens before *B*.

The value of a non-atomic scalar object or bit-field *M*, as determined by evaluation *B*, is the value stored by the visible side effect *A*.

[Note 9: If there is ambiguity about which side effect to a non-atomic object or bit-field is visible, then the behavior is either unspecified or undefined. — end note]

[Note 10: This states that operations on ordinary objects are not visibly reordered. This is not actually detectable without data races, but is needed to ensure that data races, as defined below, and with suitable restrictions on the use of atomics, correspond to data races in a simple interleaved (sequentially consistent) execution. — end note]

10 The value of an atomic object *M*, as determined by evaluation *B*, is the value stored by some unspecified side effect *A* that modifies *M*, where *B* does not happen before *A*.

[Note 11: The set of such side effects is also restricted by the rest of the rules described here, and in particular, by the coherence requirements below. — end note]

11 If an operation *A* that modifies an atomic object *M* happens before an operation *B* that modifies *M*, then *A* is earlier than *B* in the modification order of *M*.

[Note 12: This requirement is known as write-write coherence. — end note]

12 If a value computation *A* of an atomic object *M* happens before a value computation *B* of *M*, and *A* takes its value from a side effect *X* on *M*, then the value computed by *B* is either the value stored by *X* or the value stored by a side effect *Y* on *M*, where *Y* follows *X* in the modification order of *M*.

[Note 13: This requirement is known as read-read coherence. — end note]

13 If a value computation *A* of an atomic object *M* happens before an operation *B* that modifies *M*, then *A* takes its value from a side effect *X* on *M*, where *X* precedes *B* in the modification order of *M*.

[Note 14: This requirement is known as read-write coherence. — end note]

14 If a side effect *X* on an atomic object *M* happens before a value computation *B* of *M*, then the evaluation *B* takes its value from *X* or from a side effect *Y* that follows *X* in the modification order of *M*.

[Note 15: This requirement is known as write-read coherence. — end note]

15 [Note 16: The four preceding coherence requirements effectively disallow compiler reordering of atomic operations to a single object, even if both operations are relaxed loads. This effectively makes the cache coherence guarantee provided by most hardware available to C++ atomic operations. — end note]

16 [Note 17: The value observed by a load of an atomic depends on the “happens before” relation, which depends on the values observed by loads of atomics. The intended reading is that there must exist an association of atomic loads with modifications they observe that, together with suitably chosen modification orders and the “happens before” relation derived as described above, satisfy the resulting constraints as imposed here. — end note]

17 Two actions are *potentially concurrent* if

(17.1) — they are performed by different threads, or

(17.2) — they are unsequenced, at least one is performed by a signal handler, and they are not both performed by the same signal handler invocation.

The execution of a program contains a *data race* if it contains two potentially concurrent conflicting actions, at least one of which is not atomic, and neither happens before the other, except for the special case for signal handlers described below. Any such data race results in undefined behavior (F.2.19).

[Note 18: It can be shown that programs that correctly use mutexes and `memory_order::seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as “sequential consistency”. However, this applies only to data-race-free programs, and data-race-free programs cannot observe most program transformations that do not change single-threaded program semantics. In fact, most single-threaded program transformations remain possible, since any program that behaves differently as a result has undefined behavior. — end note]

- 18 Two accesses to the same non-bit-field object of type `volatile std::sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. For each signal handler invocation, evaluations performed by the thread invoking a signal handler can be divided into two groups *A* and *B*, such that no evaluations in *B* happen before evaluations in *A*, and the evaluations of such `volatile std::sig_atomic_t` objects take values as though all evaluations in *A* happened before the execution of the signal handler and the execution of the signal handler happened before all evaluations in *B*.

- 19 [Note 19: Compiler transformations that introduce assignments to a potentially shared memory location that would not be modified by the abstract machine are generally precluded by this document, since such an assignment might overwrite another assignment by a different thread in cases in which an abstract machine execution would not have encountered a data race. This includes implementations of data member assignment that overwrite adjacent members in separate memory locations. Reordering of atomic loads in cases in which the atomics in question might alias is also generally precluded, since this could violate the coherence rules. — end note]

- 20 [Note 20: It is possible that transformations that introduce a speculative read of a potentially shared memory location do not preserve the semantics of the C++ program as defined in this document, since they potentially introduce a data race. However, they are typically valid in the context of an optimizing compiler that targets a specific machine with well-defined semantics for data races. They would be invalid for a hypothetical machine that is not tolerant of races or provides hardware race detection. — end note]

6.10.2.3 Forward progress

[intro.progress]

- 1 The implementation may assume (F.2.20) that any thread will eventually do one of the following:

- (1.1) — terminate,
- (1.2) — invoke the function `std::this_thread::yield` (32.4.5),
- (1.3) — make a call to a library I/O function,
- (1.4) — perform an access through a volatile glvalue,
- (1.5) — perform an atomic or synchronization operation other than an atomic modify-write operation (32.5.4), or
- (1.6) — continue execution of a trivial infinite loop (8.6.1).

[Note 1: This is intended to allow compiler transformations such as removal, merging, and reordering of empty loops, even when termination cannot be proven. An affordance is made for trivial infinite loops, which cannot be removed nor reordered. — end note]

- 2 Executions of atomic functions that are either defined to be lock-free (32.5.10) or indicated as lock-free (32.5.5) are *lock-free executions*.

- (2.1) — If there is only one thread that is not blocked (3.6) in a standard library function, a lock-free execution in that thread shall complete.

[Note 2: Concurrently executing threads might prevent progress of a lock-free execution. For example, this situation can occur with load-locked store-conditional implementations. This property is sometimes termed obstruction-free. — end note]

- (2.2) — When one or more lock-free executions run concurrently, at least one should complete.

[Note 3: It is difficult for some implementations to provide absolute guarantees to this effect, since repeated and particularly inopportune interference from other threads could prevent forward progress, e.g., by repeatedly stealing a cache line for unrelated purposes between load-locked and store-conditional instructions. For implementations that follow this recommendation and ensure that such effects cannot indefinitely delay progress under expected operating conditions, such anomalies can therefore safely be ignored by programmers. Outside this document, this property is sometimes termed lock-free. — end note]

³ During the execution of a thread of execution, each of the following is termed an *execution step*:

- (3.1) — termination of the thread of execution,
- (3.2) — performing an access through a volatile glvalue,
- (3.3) — completion of a call to a library I/O function, or
- (3.4) — completion of an atomic or synchronization operation other than an atomic modify-write operation (32.5.4).

⁴ An invocation of a standard library function that blocks (3.6) is considered to continuously execute execution steps while waiting for the condition that it blocks on to be satisfied.

[*Example 1*: A library I/O function that blocks until the I/O operation is complete can be considered to continuously check whether the operation is complete. Each such check consists of one or more execution steps, for example using observable behavior of the abstract machine. — *end example*]

⁵ A thread of execution *makes progress* when an execution step occurs or a lock-free execution does not complete because there are other concurrent threads that are not blocked in a standard library function (see above).

⁶ For a thread of execution providing *concurrent forward progress guarantees*, the implementation ensures that the thread will eventually make progress for as long as it has not terminated.

[*Note 4*: This applies regardless of whether or not other threads of execution (if any) have been or are making progress. To eventually fulfill this requirement means that this will happen in an unspecified but finite amount of time. — *end note*]

⁷ It is implementation-defined whether the implementation-created thread of execution that executes `main` (6.10.3.1) and the threads of execution created by `std::thread` (32.4.3) or `std::jthread` (32.4.4) provide concurrent forward progress guarantees. General-purpose implementations should provide these guarantees.

⁸ For a thread of execution providing *parallel forward progress guarantees*, the implementation is not required to ensure that the thread will eventually make progress if it has not yet executed any execution step; once this thread has executed a step, it provides concurrent forward progress guarantees.

⁹ [*Note 5*: This does not specify a requirement for when to start this thread of execution, which will typically be specified by the entity that creates this thread of execution. For example, a thread of execution that provides concurrent forward progress guarantees and executes tasks from a set of tasks in an arbitrary order, one after the other, satisfies the requirements of parallel forward progress for these tasks. — *end note*]

¹⁰ For a thread of execution providing *weakly parallel forward progress guarantees*, the implementation does not ensure that the thread will eventually make progress.

¹¹ [*Note 6*: Threads of execution providing weakly parallel forward progress guarantees cannot be expected to make progress regardless of whether other threads make progress or not; however, blocking with forward progress guarantee delegation, as defined below, can be used to ensure that such threads of execution make progress eventually. — *end note*]

¹² Concurrent forward progress guarantees are stronger than parallel forward progress guarantees, which in turn are stronger than weakly parallel forward progress guarantees.

[*Note 7*: For example, some kinds of synchronization between threads of execution might only make progress if the respective threads of execution provide parallel forward progress guarantees, but will fail to make progress under weakly parallel guarantees. — *end note*]

¹³ When a thread of execution *P* is specified to *block with forward progress guarantee delegation* on the completion of a set *S* of threads of execution, then throughout the whole time of *P* being blocked on *S*, the implementation shall ensure that the forward progress guarantees provided by at least one thread of execution in *S* is at least as strong as *P*'s forward progress guarantees.

[*Note 8*: It is unspecified which thread or threads of execution in *S* are chosen and for which number of execution steps. The strengthening is not permanent and not necessarily in place for the rest of the lifetime of the affected thread of execution. As long as *P* is blocked, the implementation has to eventually select and potentially strengthen a thread of execution in *S*. — *end note*]

Once a thread of execution in *S* terminates, it is removed from *S*. Once *S* is empty, *P* is unblocked.

¹⁴ [*Note 9*: A thread of execution *B* thus can temporarily provide an effectively stronger forward progress guarantee for a certain amount of time, due to a second thread of execution *A* being blocked on it with forward progress guarantee delegation. In turn, if *B* then blocks with forward progress guarantee delegation on *C*, this can also temporarily provide a stronger forward progress guarantee to *C*. — *end note*]

- ¹⁵ [Note 10: If all threads of execution in *S* finish executing (e.g., they terminate and do not use blocking synchronization incorrectly), then *P*'s execution of the operation that blocks with forward progress guarantee delegation will not result in *P*'s progress guarantee being effectively weakened. — end note]
- ¹⁶ [Note 11: This does not remove any constraints regarding blocking synchronization for threads of execution providing parallel or weakly parallel forward progress guarantees because the implementation is not required to strengthen a particular thread of execution whose too-weak progress guarantee is preventing overall progress. — end note]
- ¹⁷ An implementation should ensure that the last value (in modification order) assigned by an atomic or synchronization operation will become visible to all other threads in a finite period of time.

6.10.3 Start and termination

[basic.start]

6.10.3.1 main function

[basic.start.main]

- ¹ A program shall contain exactly one function called **main** that belongs to the global scope. Executing a program starts a main thread of execution (6.10.2, 32.4) in which the **main** function is invoked. It is implementation-defined whether a program in a freestanding environment is required to define a **main** function.
- [Note 1: In a freestanding environment, startup and termination is implementation-defined; startup contains the execution of constructors for non-local objects with static storage duration; termination contains the execution of destructors for objects with static storage duration. — end note]
- ² An implementation shall not predefine the **main** function. Its type shall have C++ language linkage and it shall have a declared return type of type **int**, but otherwise its type is implementation-defined. An implementation shall allow both
- (2.1) — an “optionally **noexcept** function of () returning **int**” and
 - (2.2) — an “optionally **noexcept** function of (**int**, pointer to pointer to **char**) returning **int**”
- as the type of **main** (9.3.4.6). In the latter form, for purposes of exposition, the first function parameter is called **argc** and the second function parameter is called **argv**, where **argc** shall be the number of arguments passed to the program from the environment in which the program is run. If **argc** is nonzero these arguments shall be supplied in **argv**[0] through **argv**[**argc** - 1] as pointers to the initial characters of null-terminated multibyte strings (NTMBSS) (16.3.3.3.4.3) and **argv**[0] shall be the pointer to the initial character of an NTMBSS that represents the name used to invoke the program or "". The value of **argc** shall be non-negative. The value of **argv**[**argc**] shall be 0.
- Recommended practice:* Any further (optional) parameters should be added after **argv**.
- ³ The function **main** shall not be named by an expression. The linkage (6.7) of **main** is implementation-defined. A program that defines **main** as deleted or that declares **main** to be **inline**, **static**, **constexpr**, or **constexpr** is ill-formed. The function **main** shall not be a coroutine (9.6.4). The **main** function shall not be declared with a *linkage-specification* (9.12) other than "C++". A program that declares
- (3.1) — a variable **main** that belongs to the global scope, or
 - (3.2) — a function **main** that belongs to the global scope and is attached to a named module, or
 - (3.3) — a function template **main** that belongs to the global scope, or
 - (3.4) — an entity named **main** with C language linkage (in any namespace)
- is ill-formed. The name **main** is not otherwise reserved.
- [Example 1: Member functions, classes, and enumerations can be called **main**, as can entities in other namespaces. — end example]
- ⁴ Terminating the program without leaving the current block (e.g., by calling the function **std::exit(int)** (17.5)) does not destroy any objects with automatic storage duration (11.4.7). If **std::exit** is invoked during the destruction of an object with static or thread storage duration, the program has undefined behavior (F.2.21).
- ⁵ A **return** statement (8.8.4) in **main** has the effect of leaving the **main** function (destroying any objects with automatic storage duration and evaluating any postcondition assertions of **main**) and calling **std::exit** with the return value as the argument. If control flows off the end of the *compound-statement* of **main**, the effect is equivalent to a **return** with operand 0 (see also 14.4).

6.10.3.2 Static initialization**[basic.start.static]**

¹ Variables with static storage duration are initialized as a consequence of program initiation. Variables with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.

² *Constant initialization* is performed if a variable with static or thread storage duration is constant-initialized (7.7.4). If constant initialization is not performed, a variable with static storage duration (6.8.6.2) or thread storage duration (6.8.6.3) is zero-initialized (9.5). Together, zero-initialization and constant initialization are called *static initialization*; all other initialization is *dynamic initialization*. All static initialization strongly happens before (6.10.2.2) any dynamic initialization.

[Note 1: The dynamic initialization of non-block variables is described in 6.10.3.3; that of static block variables is described in 8.10. — end note]

³ An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically, provided that

- (3.1) — the dynamic version of the initialization does not change the value of any other object of static or thread storage duration prior to its initialization, and
- (3.2) — the static version of the initialization produces the same value in the initialized variable as would be produced by the dynamic initialization if all variables not required to be initialized statically were initialized dynamically.

[Note 2: As a consequence, if the initialization of an object `obj1` refers to an object `obj2` potentially requiring dynamic initialization and defined later in the same translation unit, it is unspecified whether the value of `obj2` used will be the value of the fully initialized `obj2` (because `obj2` was statically initialized) or will be the value of `obj2` merely zero-initialized. For example:

```
inline double fd() { return 1.0; }
extern double d1;
double d2 = d1;
double d1 = fd();
```

Both `d1` and `d2` can be initialized either statically or dynamically. If `d1` is initialized statically and `d2` dynamically, both variables are initialized to 1.0; in all other cases, `d2` is initialized to 0.0. — end note]

6.10.3.3 Dynamic initialization of non-block variables**[basic.start.dynamic]**

¹ Dynamic initialization of a non-block variable with static storage duration is unordered if the variable is an implicitly or explicitly instantiated specialization, is partially-ordered if the variable is an inline variable that is not an implicitly or explicitly instantiated specialization, and otherwise is ordered.

[Note 1: A non-inline explicit specialization of a templated variable has ordered initialization. — end note]

² A declaration `D` is *appearance-ordered* before a declaration `E` if

- (2.1) — `D` appears in the same translation unit as `E`, or
- (2.2) — the translation unit containing `E` has an interface dependency (10.3) on the translation unit containing `D`,

in either case prior to `E`.

³ Dynamic initialization of non-block variables `V` and `W` with static storage duration is ordered as follows:

- (3.1) — If `V` and `W` have ordered initialization and the definition of `V` is appearance-ordered before the definition of `W`, or if `V` has partially-ordered initialization, `W` does not have unordered initialization, and for every definition `E` of `W` there exists a definition `D` of `V` such that `D` is appearance-ordered before `E`, then
 - (3.1.1) — if the program does not start a thread (6.10.2) other than the main thread (6.10.3.1) or `V` and `W` have ordered initialization and they are defined in the same translation unit, the initialization of `V` is sequenced before the initialization of `W`;
 - (3.1.2) — otherwise, the initialization of `V` strongly happens before the initialization of `W`.
- (3.2) — Otherwise, if the program starts a thread other than the main thread before either `V` or `W` is initialized, it is unspecified in which threads the initializations of `V` and `W` occur; the initializations are unsequenced if they occur in the same thread.
- (3.3) — Otherwise, the initializations of `V` and `W` are indeterminately sequenced.

[*Note 2*: This definition permits initialization of a sequence of ordered variables concurrently with another sequence. — *end note*]

- 4 A *non-initialization odr-use* is an odr-use (6.3) not caused directly or indirectly by the initialization of a non-block static or thread storage duration variable.
- 5 It is implementation-defined whether the dynamic initialization of a non-block non-inline variable with static storage duration is sequenced before the first statement of `main` or is deferred. If it is deferred, it strongly happens before any non-initialization odr-use of any non-inline function or non-inline variable defined in the same translation unit as the variable to be initialized.³² It is implementation-defined in which threads and at which points in the program such deferred dynamic initialization occurs.

Recommended practice: An implementation should choose such points in a way that allows the programmer to avoid deadlocks.

[*Example 1*:

```
// - File 1 -
#include "a.h"
#include "b.h"
B b;
A::A() {
    b.Use();
}

// - File 2 -
#include "a.h"
A a;

// - File 3 -
#include "a.h"
#include "b.h"
extern A a;
extern B b;

int main() {
    a.Use();
    b.Use();
}
```

It is implementation-defined whether either `a` or `b` is initialized before `main` is entered or whether the initializations are delayed until `a` is first odr-used in `main`. In particular, if `a` is initialized before `main` is entered, it is not guaranteed that `b` will be initialized before it is odr-used by the initialization of `a`, that is, before `A::A` is called. If, however, `a` is initialized at some point after the first statement of `main`, `b` will be initialized prior to its use in `A::A`. — *end example*]

- 6 It is implementation-defined whether the dynamic initialization of a non-block inline variable with static storage duration is sequenced before the first statement of `main` or is deferred. If it is deferred, it strongly happens before any non-initialization odr-use of that variable. It is implementation-defined in which threads and at which points in the program such deferred dynamic initialization occurs.
- 7 It is implementation-defined whether the dynamic initialization of a non-block non-inline variable with thread storage duration is sequenced before the first statement of the initial function of a thread or is deferred. If it is deferred, the initialization associated with the entity for thread *t* is sequenced before the first non-initialization odr-use by *t* of any non-inline variable with thread storage duration and dynamic initialization defined in the same translation unit as the variable to be initialized. It is implementation-defined in which threads and at which points in the program such deferred dynamic initialization occurs.
- 8 If the initialization of a non-block variable with static or thread storage duration exits via an exception, the function `std::terminate` is called (14.6.2).

6.10.3.4 Termination

[**basic.start.term**]

- 1 Constructed complete objects (9.5) with static storage duration are destroyed and functions registered with `std::atexit` are called as part of a call to `std::exit` (17.5). The call to `std::exit` is sequenced before the destructions and the registered functions.

³² A non-block variable with static storage duration having initialization with side effects is initialized in this case, even if it is not itself odr-used (6.3, 6.8.6.2).

[*Note 1*: Returning from `main` invokes `std::exit` (6.10.3.1). — *end note*]

- 2 Constructed complete objects with thread storage duration within a given thread are destroyed as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The destruction of those constructed objects is sequenced before releasing the storage for any object with thread storage duration within that thread (6.8.6.3). The destruction of those constructed objects strongly happens before destroying any object with static storage duration.
- 3 The destruction of a complete object with thread storage duration within a given thread and having constant destruction (7.7.7) is sequenced after the destruction of any other complete object with thread storage duration within the thread. The destruction of a complete object with static storage duration and having constant destruction is sequenced after the destruction of any other complete object with static storage duration and after any call to a function passed to `std::atexit`. The sequencing rules in the remainder of this subclause apply only to complete objects not having constant destruction.
- 4 If the deemed construction (9.5.1) of a complete object with static storage duration strongly happens before that of another, the completion of the destruction of the second is sequenced before the initiation of the destruction of the first. If the deemed construction of a complete object with thread storage duration is sequenced before that of another, the completion of the destruction of the second is sequenced before the initiation of the destruction of the first. If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. If the destruction of an object with static or thread storage duration exits via an exception, the function `std::terminate` is called (14.6.2).

[*Example 1*: In the following program, the elements of `a` are destroyed, followed by `dt`, and finally by the two `BTemp` objects:

```
struct DTemp { ~DTemp(); };
struct Temp {
    ~Temp() {
        static DTemp dt;
    }
};
struct BTemp {
    ~BTemp();
};
struct A {
    const BTemp &tb;
    ~A();
};
A a[] = { (Temp(), BTemp()), BTemp() };

int main() {}
```

If the array `a` were an object with automatic storage duration, the `BTemp` temporaries would be destroyed as each element of the array is destroyed (6.8.7). — *end example*]

- 5 If a function contains a block variable of static or thread storage duration that has been destroyed and the function is called during the destruction of an object with static or thread storage duration, the program has undefined behavior (F.2.22) if the flow of control passes through the definition of the previously destroyed block variable.

[*Note 2*: Likewise, the behavior is undefined if the block variable is used indirectly (e.g., through a pointer) after its destruction. — *end note*]

- 6 If the deemed construction of a complete object with static storage duration strongly happens before a call to `std::atexit` (see <cstdlib>, 17.5), the call to the function passed to `std::atexit` is sequenced before the initiation of the destruction of the object. If a call to `std::atexit` strongly happens before the deemed construction of a complete object with static storage duration, the completion of the destruction of the object is sequenced before the call to the function passed to `std::atexit`. If a call to `std::atexit` strongly happens before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call is sequenced before the call to the function passed to the first `std::atexit` call.
- 7 If there is a use of a standard library object or function not permitted within signal handlers (17.14) that does not happen before (6.10.2) completion of destruction of objects with static storage duration and execution of `std::atexit` registered functions (17.5), the program has undefined behavior

[*Note 3*: If there is a use of an object with static storage duration that does not happen before the object's destruction, the program has undefined behavior. Terminating every thread before a call to `std::exit` or the exit from `main`

is sufficient, but not necessary, to satisfy these requirements. These requirements permit thread managers as static-storage-duration objects. — *end note*

- 8 Calling the function `std::abort()` declared in `<cstdlib>` (17.2.2) terminates the program without executing any destructors and without calling the functions passed to `std::atexit()` or `std::at_quick_exit()`.

6.11 Contract assertions [basic.contract]

6.11.1 General [basic.contract.general]

- 1 *Contract assertions* allow the programmer to specify properties of the state of the program that are expected to hold at certain points during execution. Contract assertions are introduced by *precondition-specifiers*, *postcondition-specifiers* (9.4.1), and *assertion-statements* (8.9).
- 2 Each contract assertion has a *contract-assertion predicate*, which is an expression of type `bool`.
[*Note 1*: The value of the predicate is used to identify program states that are expected. — *end note*]
- 3 An invocation of the macro `va_start` (17.14.2) shall not be a subexpression of the predicate of a contract assertion, no diagnostic required (G.3.7).
- 4 [*Note 2*: Within the predicate of a contract assertion, *id-expressions* referring to variables declared outside the contract assertion are `const` (7.5.5.2), `this` is a pointer to `const` (7.5.3), and the result object can be named if a *result-name-introducer* (9.4.2) has been specified. — *end note*]

6.11.2 Evaluation [basic.contract.eval]

- 1 An evaluation of a contract assertion uses one of the following four *evaluation semantics*: *ignore*, *observe*, *enforce*, or *quick-enforce*. Observe, enforce, and quick-enforce are *checking semantics*; enforce and quick-enforce are *terminating semantics*.
- 2 It is implementation-defined which evaluation semantic is used for any given evaluation of a contract assertion.
[*Note 1*: The range and flexibility of available choices of evaluation semantics depends on the implementation and need not allow all four evaluation semantics as possibilities. The evaluation semantics can differ for different evaluations of the same contract assertion, including evaluations during constant evaluation. — *end note*]
- 3 *Recommended practice*: An implementation should provide the option to translate a program such that all evaluations of contract assertions use the ignore semantic as well as the option to translate a program such that all evaluations of contract assertions use the enforce semantic. By default, evaluations of contract assertions should use the enforce semantic.
- 4 The evaluation of a contract assertion using the ignore semantic has no effect.
[*Note 2*: The predicate is potentially evaluated (6.3), but not evaluated. — *end note*]
- 5 The evaluation *A* of a contract assertion using a checking semantic determines the value of the predicate. It is unspecified whether the predicate is evaluated. Let *B* be the value that would result from evaluating the predicate.

[*Note 3*: To determine whether a predicate would evaluate to `true` or `false`, an alternative evaluation that produces the same value as the predicate but has no side effects can occur.

[*Example 1*:

```
struct S {
    mutable int g = 5;
} s;
void f()
    pre(( s.g++, false ));    // #1
void g()
{
    f(); // Increment of s.g might not occur, even if #1 uses a checking semantic.
}
```

— *end example*

— *end note*

- 6 There is an observable checkpoint (4.1.2) *C* that happens before *A* such that any other evaluation that happens before *A* also happens before *C*.
- 7 A *contract violation* occurs when
- (7.1) — *B* is `false`,

- (7.2) — the evaluation of the predicate exits via an exception, or
- (7.3) — the evaluation of the predicate is performed in a context that is manifestly constant-evaluated (7.7.7) and the predicate is not a core constant expression.

[Note 4: If *B* is **true**, no contract violation occurs and control flow continues normally after the point of evaluation of the contract assertion. The evaluation of the predicate can fail to produce a value without causing a contract violation, for example, by calling **longjmp** (17.14.3) or terminating the program. — end note]

- 8 If a contract violation occurs in a context that is manifestly constant-evaluated (7.7.7), and the evaluation semantic is a terminating semantic, the program is ill-formed.

[Note 5: A diagnostic is produced if the evaluation semantic is observe (4.1). — end note]

[Note 6: Different evaluation semantics chosen for the same contract assertion in different translation units can result in violations of the one-definition rule (6.3) when a contract assertion has side effects that alter the value produced by a constant expression.

[Example 2:

```
constexpr int f(int i)
{
    contract_assert(++const_cast<int&>(i), true);
    return i;
}
inline void g()
{
    int a[f(1)]; // size dependent on the evaluation semantic of contract_assert above
}
```

— end example]

— end note]

- 9 When the program is *contract-terminated*, it is implementation-defined (depending on context) whether
 - (9.1) — **std::terminate** is called,
 - (9.2) — **std::abort** is called, or
 - (9.3) — execution is terminated.

[Note 7: No further execution steps occur (6.10.2.3). — end note]

[Note 8: Performing the actions of **std::terminate** or **std::abort** without actually making a library call is a conforming implementation of contract-termination (4.1.2). — end note]

- 10 If a contract violation occurs in a context that is not manifestly constant-evaluated and the evaluation semantic is quick-enforce, the program is contract-terminated.
- 11 If a contract violation occurs in a context that is not manifestly constant-evaluated and the evaluation semantic is enforce or observe, the contract-violation handler (6.11.3) is invoked with an lvalue referring to an object *v* of type **const std::contracts::contract_violation** (17.10.3) containing information about the contract violation. Storage for *v* is allocated in an unspecified manner except as noted in 6.8.6.5.2. The lifetime of *v* persists for the duration of the invocation of the contract-violation handler.
- 12 If the contract violation occurred because the evaluation of the predicate exited via an exception, the contract-violation handler is invoked from within an active implicit handler for that exception (14.4). If the contract-violation handler returns normally and the evaluation semantic is observe, that implicit handler is no longer considered active.

[Note 9: The exception can be inspected or rethrown within the contract-violation handler. — end note]

- 13 If the contract-violation handler returns normally and the evaluation semantic is enforce, the program is contract-terminated; if violation occurred as the result of an uncaught exception from the evaluation of the predicate, the implicit handler remains active when contract termination occurs.
- 14 [Note 10: If the contract-violation handler returns normally and the evaluation semantic is observe, control flow continues normally after the point of evaluation of the contract assertion. — end note]
- 15 There is an observable checkpoint (4.1.2) *C* that happens after the contract-violation handler returns normally such that any other evaluation that happens after the contract-violation handler returns also happens after *C*.

16 [Note 11: The terminating semantics terminate the program if execution would otherwise continue normally past a contract violation: the enforce semantic provides the opportunity to log information about the contract violation before terminating the program or to throw an exception to avoid termination, and the quick-enforce semantic is intended to terminate the program as soon as possible as well as to minimize the impact of contract checks on the generated code size. Conversely, the observe semantic provides the opportunity to log information about the contract violation without having to terminate the program. — end note]

17 If a contract-violation handler invoked from the evaluation of a function contract assertion (9.4.1) exits via an exception, the behavior is as if the function body exits via that same exception.

[Note 12: A *function-try-block* (14.1) is the function body when present and thus does not have an opportunity to catch the exception. If the function has a non-throwing exception specification, the function `std::terminate` is invoked (14.6.2). — end note]

[Note 13: If a contract-violation handler invoked from an *assertion-statement* (8.9) exits via an exception, the search for a handler continues from the execution of that statement. — end note]

18 To *evaluate in sequence* a list R of contract assertions:

- (18.1) — Construct a list of contract assertions S such that
 - (18.1.1) — all elements of R are in S ,
 - (18.1.2) — each element of R may be repeated an implementation-defined number of times within S , and
 - (18.1.3) — if a contract assertion A precedes another contract assertion B in R , then the first occurrence of A precedes the first occurrence of B in S .
- (18.2) — Evaluate each element of S such that, if a contract assertion A precedes a contract assertion B in S , then the evaluation of A is sequenced before the evaluation of B .

[Example 3:

```
void f(int i)
{
    contract_assert(i > 0);    // #1
    contract_assert(i < 10);   // #2
    // valid sequence of evaluations: #1 #2
    // valid sequence of evaluations: #1 #1 #2 #2
    // valid sequence of evaluations: #1 #2 #1 #2
    // valid sequence of evaluations: #1 #2 #2 #1
    // invalid sequence of evaluations: #2 #1
}
```

— end example]

19 *Recommended practice:* An implementation should provide an option to perform a specified number of repeated evaluations for contract assertions. By default, no repeated evaluations should be performed.

6.11.3 Contract-violation handler

[basic.contract.handler]

1 The *contract-violation handler* of a program is a function named `::handle_contract_violation`. The contract-violation handler shall have a single parameter of type “lvalue reference to `const std::contracts::contract_violation`” and shall return `void`. The contract-violation handler may have a non-throwing exception specification. The implementation shall provide a definition of the contract-violation handler, called the *default contract-violation handler*.

[Note 1: No declaration for the default contract-violation handler is provided by any standard library header. — end note]

2 *Recommended practice:* The default contract-violation handler should produce diagnostic output that suitably formats the most relevant contents of the `std::contracts::contract_violation` object, rate-limited for potentially repeated violations of observed contract assertions, and then return normally.

3 It is implementation-defined whether the contract-violation handler is replaceable (9.6.5). If the contract-violation handler is not replaceable, a declaration of a replacement function for the contract-violation handler is ill-formed, no diagnostic required (G.3.8).

7 Expressions

[expr]

7.1 Preamble

[expr.pre]

- ¹ [Note 1: [Clause 7](#) defines the syntax, order of evaluation, and meaning of expressions.³³ An expression is a sequence of operators and operands that specifies a computation. An expression can result in a value and can cause side effects. — end note]
- ² [Note 2: Operators can be overloaded, that is, given meaning when applied to expressions of class type ([Clause 11](#)) or enumeration type ([9.8.1](#)). Uses of overloaded operators are transformed into function calls as described in [12.4](#). Overloaded operators obey the rules for syntax and evaluation order specified in [7.6](#), but the requirements of operand type and value category are replaced by the rules for function call. Relations between operators, such as `++a` meaning `a += 1`, are not guaranteed for overloaded operators ([12.4](#)). — end note]
- ³ Subclause [7.6](#) defines the effects of operators when applied to types for which they have not been overloaded. Operator overloading shall not modify the rules for the *built-in operators*, that is, for operators applied to types for which they are defined by this Standard. However, these built-in operators participate in overload resolution, and as part of that process user-defined conversions will be considered where necessary to convert the operands to types appropriate for the built-in operator. If a built-in operator is selected, such conversions will be applied to the operands before the operation is considered further according to the rules in [7.6](#); see [12.2.2.3](#), [12.5](#).

⁴ An *arithmetic expression* is

- (4.1) — a unary plus or minus ([7.6.2.2](#)),
- (4.2) — an addition ([7.6.6](#)),
- (4.3) — a subtraction ([7.6.1.2](#)), or
- (4.4) — a multiplication, division, or remainder ([7.6.5](#))

expression where every (possibly converted) operand is of arithmetic type.

[Note 3: There exist non-arithmetic expressions such as compound assignment ([7.6.19](#)) which are defined in terms of arithmetic expressions. — end note]

The behavior of evaluating an arithmetic expression is undefined ([F.3.1](#)) if the mathematical result is neither

- (4.5) — in the range of representable values for its type nor
- (4.6) — a negative infinity, positive infinity, or NaN that is among the values of the type.

[Note 4: If the operands are of a type that adheres to ISO/IEC 60559, division by zero is the only case where an arithmetic expression has undefined behavior ([7.6.5](#)). However, some well-defined arithmetic expressions are not core constant expressions ([7.7.2](#)).

[Example 1: The following example assumes that `std::float32_t` is supported ([6.9.4](#)).

```
constexpr std::float32_t min = std::numeric_limits<std::float32_t>::min(); // OK
constexpr std::float32_t max = std::numeric_limits<std::float32_t>::max(); // OK
constexpr std::float32_t inf = std::numeric_limits<std::float32_t>::infinity(); // OK
constexpr std::float32_t nan = std::numeric_limits<std::float32_t>::quiet_NaN(); // OK

// Furthermore, if arithmetic expressions with operands of type std::float32_t
// behave as specified in ISO/IEC 60559 for binary32:
constexpr std::float32_t inf2 = inf * 2; // OK, also positive infinity
constexpr std::float32_t zero = min / max; // OK, result cannot be represented, and is rounded to zero
constexpr std::float32_t oflo = max * 2; // error: non-finite result but operands are finite (7.7.2)
constexpr std::float32_t nan2 = nan * 2; // OK, propagating a NaN
constexpr std::float32_t udef = inf * 0; // error: result is NaN but neither operand is NaN (7.7.2)
constexpr std::float32_t div0 = max / 0; // error: division by zero is undefined (7.6.5, 7.7.2)
```

— end example]

— end note]

³³) The precedence of operators is not directly specified, but it can be derived from the syntax.

[*Note 5*: Treatment of division by zero, forming a remainder using a zero divisor, and all floating-point exceptions varies among machines, and is sometimes adjustable by a library function. — *end note*]

- ⁵ [*Note 6*: The implementation can regroup operators according to the usual mathematical rules only where the operators really are associative or commutative.³⁴ For example, in the following fragment

```
int a, b;
/* ... */
a = a + 32760 + b + 5;
```

the expression statement behaves exactly the same as

```
a = (((a + 32760) + b) + 5);
```

due to the associativity and precedence of these operators. Thus, the result of the sum ($a + 32760$) is next added to b , and that result is then added to 5 which results in the value assigned to a . On a machine in which overflows produce an exception and in which the range of values representable by an `int` is $[-32768, +32767]$, the implementation cannot rewrite this expression as

```
a = ((a + b) + 32765);
```

since if the values for a and b were, respectively, -32754 and -15 , the sum $a + b$ would produce an exception while the original expression would not; nor can the expression be rewritten as either

```
a = ((a + 32765) + b);
```

or

```
a = (a + (b + 32765));
```

since the values for a and b might have been, respectively, 4 and -8 or -17 and 12. However on a machine in which overflows do not produce an exception and in which the results of overflows are reversible, the above expression statement can be rewritten by the implementation in any of the above ways because the same result will occur. — *end note*]

- ⁶ The values of the floating-point operands and the results of floating-point expressions may be represented in greater precision and range than that required by the type; the types are not changed thereby.³⁵

7.2 Properties of expressions

[**expr.prop**]

7.2.1 Value category

[**basic.lval**]

- ¹ Expressions are categorized according to the taxonomy in Figure 2.

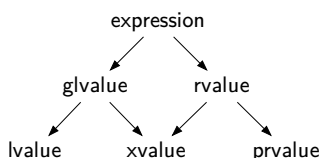


Figure 2 — Expression category taxonomy [fig:basic.lval]

- (1.1) — A *glvalue* is an expression whose evaluation determines the identity of an object, function, non-static data member, or a direct base class relationship.
 - (1.2) — A *prvalue* is an expression whose evaluation initializes an object or computes the value of an operand of an operator, as specified by the context in which it appears, or an expression that has type *cv void*.
 - (1.3) — An *xvalue* is a *glvalue* that denotes an object whose resources can be reused (usually because it is near the end of its lifetime).
 - (1.4) — An *lvalue* is a *glvalue* that is not an *xvalue*.
 - (1.5) — An *rvalue* is a *prvalue* or an *xvalue*.
- ² Every expression belongs to exactly one of the fundamental categories in this taxonomy: *lvalue*, *xvalue*, or *prvalue*. This property of an expression is called its *value category*.

³⁴) Overloaded operators are never assumed to be associative or commutative.

³⁵) The cast and assignment operators must still perform their specific conversions as described in 7.6.1.4, 7.6.3, 7.6.1.9 and 7.6.19.

[*Note 1*: The discussion of each built-in operator in 7.6 indicates the category of the value it yields and the value categories of the operands it expects. For example, the built-in assignment operators expect that the left operand is an lvalue and that the right operand is a prvalue and yield an lvalue as the result. User-defined operators are functions, and the categories of values they expect and yield are determined by their parameter and return types. — *end note*]

3 [*Note 2*: Historically, lvalues and rvalues were so-called because they could appear on the left- and right-hand side of an assignment (although this is no longer generally true); glvalues are “generalized” lvalues, prvalues are “pure” rvalues, and xvalues are “eXpiring” lvalues. Despite their names, these terms apply to expressions, not values. — *end note*]

4 [*Note 3*: An expression is an xvalue if it is:

- (4.1) — a move-eligible *id-expression* (7.5.5.2) or *splice-expression* (7.5.9),
- (4.2) — the result of calling a function, whether implicitly or explicitly, whose return type is an rvalue reference to object type (7.6.1.3),
- (4.3) — a cast to an rvalue reference to object type (7.6.1.4, 7.6.1.7, 7.6.1.9, 7.6.1.10, 7.6.1.11, 7.6.3),
- (4.4) — a subscripting operation with an xvalue array operand (7.6.1.2),
- (4.5) — a class member access expression designating a non-static data member of non-reference type in which the object expression is an xvalue (7.6.1.5), or
- (4.6) — a `.*` pointer-to-member expression in which the first operand is an xvalue and the second operand is a pointer to data member (7.6.4).

In general, the effect of this rule is that named rvalue references are treated as lvalues and unnamed rvalue references to objects are treated as xvalues; rvalue references to functions are treated as lvalues whether named or not. — *end note*]

[*Example 1*:

```
struct A {
    int m;
};
A&& operator+(A, A);
A&& f();

A a;
A&& ar = static_cast<A&&>(a);
```

The expressions `f()`, `f().m`, `static_cast<A&&>(a)`, and `a + a` are xvalues. The expression `ar` is an lvalue. — *end example*]

5 The *result* of a glvalue is the entity denoted by the expression. The *result* of a prvalue is the value that the expression stores into its context; a prvalue that has type `cv void` has no result. A prvalue whose result is the value *V* is sometimes said to have or name the value *V*. The *result object* of a prvalue is the object initialized by the prvalue; a prvalue that has type `cv void` has no result object.

[*Note 4*: Except when the prvalue is the operand of a *decltype-specifier*, a prvalue of object type always has a result object. For a discarded prvalue that has type other than `cv void`, a temporary object is materialized; see 7.2.3. — *end note*]

6 Whenever a glvalue appears as an operand of an operator that requires a prvalue for that operand, the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), or function-to-pointer (7.3.4) standard conversions are applied to convert the expression to a prvalue.

[*Note 5*: An attempt to bind an rvalue reference to an lvalue is not such a context; see 9.5.4. — *end note*]

[*Note 6*: Because `cv`-qualifiers are removed from the type of an expression of non-class type when the expression is converted to a prvalue, an lvalue of type `const int` can, for example, be used where a prvalue of type `int` is required. — *end note*]

[*Note 7*: There are no prvalue bit-fields; if a bit-field is converted to a prvalue (7.3.2), a prvalue of the type of the bit-field is created, which might then be promoted (7.3.7). — *end note*]

7 Unless otherwise specified (7.6.1.10, 7.6.1.11), whenever a prvalue that is not the result of the lvalue-to-rvalue conversion (7.3.2) appears as an operand of an operator, the temporary materialization conversion (7.3.5) is applied to convert the expression to an xvalue.

8 [*Note 8*: The discussion of reference initialization in 9.5.4 and of temporaries in 6.8.7 indicates the behavior of lvalues and rvalues in other significant contexts. — *end note*]

- ⁹ Unless otherwise indicated (9.2.9.6), a prvalue shall always have complete type or the `void` type; if it has a class type or (possibly multidimensional) array of class type, that class shall not be an abstract class (11.7.4). A glvalue shall not have type `cv void`.

[Note 9: A glvalue can have complete or incomplete non-void type. Class and array prvalues can have cv-qualified types; other prvalues always have cv-unqualified types. See 7.2.2. — end note]

- ¹⁰ An lvalue is *modifiable* unless its type is const-qualified or is a function type.

[Note 10: A program that attempts to modify an object through a nonmodifiable lvalue or through an rvalue is ill-formed (7.6.19, 7.6.1.6, 7.6.2.3). — end note]

- ¹¹ An object of dynamic type T_{obj} is *type-accessible* through a glvalue of type T_{ref} if T_{ref} is similar (7.3.6) to:

- (11.1) — T_{obj} ,
- (11.2) — a type that is the signed or unsigned type corresponding to T_{obj} , or
- (11.3) — a `char`, `unsigned char`, or `std::byte` type.

If a program attempts to access (3.1) the stored value of an object through a glvalue through which it is not type-accessible, the behavior is undefined (F.3.2).³⁶ If a program invokes a defaulted copy/move constructor or copy/move assignment operator for a union of type U with a glvalue argument that does not denote an object of type `cv U` within its lifetime, the behavior is undefined (F.3.3).

[Note 11: In C, an entire object of structure type can be accessed, e.g., using assignment. By contrast, C++ has no notion of accessing an object of class type through an lvalue of class type. — end note]

7.2.2 Type

[expr.type]

- ¹ If an expression initially has the type “reference to T ” (9.3.4.3, 9.5.4), the type is adjusted to T prior to any further analysis; the value category of the expression is not altered. Let X be the object or function denoted by the reference. If a pointer to X would be valid in the context of the evaluation of the expression (6.9.3), the result designates X ; otherwise, the behavior is undefined (F.3.4).

[Note 1: Before the lifetime of the reference has started or after it has ended, the behavior is undefined (see 6.8.4). — end note]

- ² If a prvalue initially has the type “`cv T`”, where T is a cv-unqualified non-class, non-array type, the type of the expression is adjusted to T prior to any further analysis.
- ³ The *composite pointer type* of two operands $p1$ and $p2$ having types $T1$ and $T2$, respectively, where at least one is a pointer or pointer-to-member type or `std::nullptr_t`, is:

- (3.1) — if both $p1$ and $p2$ are null pointer constants, `std::nullptr_t`;
- (3.2) — if either $p1$ or $p2$ is a null pointer constant, $T2$ or $T1$, respectively;
- (3.3) — if $T1$ or $T2$ is “pointer to *cv1* `void`” and the other type is “pointer to *cv2* T ”, where T is an object type or `void`, “pointer to *cv12* `void`”, where *cv12* is the union of *cv1* and *cv2*;
- (3.4) — if $T1$ or $T2$ is “pointer to `noexcept` function” and the other type is “pointer to function”, where the function types are otherwise the same, “pointer to function”;
- (3.5) — if $T1$ is “pointer to $C1$ ” and $T2$ is “pointer to $C2$ ”, where $C1$ is reference-related to $C2$ or $C2$ is reference-related to $C1$ (9.5.4), the qualification-combined type (7.3.6) of $T1$ and $T2$ or the qualification-combined type of $T2$ and $T1$, respectively;
- (3.6) — if $T1$ or $T2$ is “pointer to member of $C1$ of type function”, the other type is “pointer to member of $C2$ of type `noexcept` function”, and $C1$ is reference-related to $C2$ or $C2$ is reference-related to $C1$ (9.5.4), where the function types are otherwise the same, “pointer to member of $C2$ of type function” or “pointer to member of $C1$ of type function”, respectively;
- (3.7) — if $T1$ is “pointer to member of $C1$ of type *cv1* U ” and $T2$ is “pointer to member of $C2$ of type *cv2* U ”, for some non-function type U , where $C1$ is reference-related to $C2$ or $C2$ is reference-related to $C1$ (9.5.4), the qualification-combined type of $T2$ and $T1$ or the qualification-combined type of $T1$ and $T2$, respectively;
- (3.8) — if $T1$ and $T2$ are similar types (7.3.6), the qualification-combined type of $T1$ and $T2$;
- (3.9) — otherwise, a program that necessitates the determination of a composite pointer type is ill-formed.

³⁶) The intent of this list is to specify those circumstances in which an object can or cannot be aliased.

[Example 1:

```
typedef void *p;
typedef const int *q;
typedef int **pi;
typedef const int **pci;
```

The composite pointer type of `p` and `q` is “pointer to `const void`”; the composite pointer type of `pi` and `pci` is “pointer to `const` pointer to `const int`”. — end example]

7.2.3 Context dependence

[expr.context]

- ¹ In some contexts, *unevaluated operands* appear (7.5.8.2, 7.5.8.4, 7.6.1.8, 7.6.2.5, 7.6.2.7, 7.6.2.10, 9.2.9.6, 13.1, 13.7.9). An unevaluated operand is not evaluated.

[Note 1: In an unevaluated operand, a non-static class member can be named (7.5.5) and naming of objects or functions does not, by itself, require that a definition be provided (6.3). An unevaluated operand is considered a full-expression (6.10.1). — end note]

- ² In some contexts, an expression only appears for its side effects. Such an expression is called a *discarded-value expression*. The array-to-pointer (7.3.3) and function-to-pointer (7.3.4) standard conversions are not applied. The lvalue-to-rvalue conversion (7.3.2) is applied if and only if the expression is a glvalue of volatile-qualified type and it is one of the following:

- (2.1) — (*expression*), where *expression* is one of these expressions,
- (2.2) — *id-expression* (7.5.5),
- (2.3) — *splice-expression* (7.5.9),
- (2.4) — subscripting (7.6.1.2),
- (2.5) — class member access (7.6.1.5),
- (2.6) — indirection (7.6.2.2),
- (2.7) — pointer-to-member operation (7.6.4),
- (2.8) — conditional expression (7.6.16) where both the second and the third operands are one of these expressions, or
- (2.9) — comma expression (7.6.20) where the right operand is one of these expressions.

[Note 2: Using an overloaded operator causes a function call; the above covers only operators with built-in meaning. — end note]

The temporary materialization conversion (7.3.5) is applied if the (possibly converted) expression is a prvalue of object type.

[Note 3: If the original expression is an lvalue of class type, it must have a volatile copy constructor to initialize the temporary object that is the result object of the temporary materialization conversion. — end note]

The expression is evaluated and its result (if any) is discarded.

7.3 Standard conversions

[conv]

7.3.1 General

[conv.general]

- ¹ Standard conversions are implicit conversions with built-in meaning. 7.3 enumerates the full set of such conversions. A *standard conversion sequence* is a sequence of standard conversions in the following order:

- (1.1) — Zero or one conversion from the following set: lvalue-to-rvalue conversion, array-to-pointer conversion, and function-to-pointer conversion.
- (1.2) — Zero or one conversion from the following set: integral promotions, floating-point promotion, integral conversions, floating-point conversions, floating-integral conversions, pointer conversions, pointer-to-member conversions, and boolean conversions.
- (1.3) — Zero or one function pointer conversion.
- (1.4) — Zero or one qualification conversion.

[Note 1: A standard conversion sequence can be empty, i.e., it can consist of no conversions. — end note]

A standard conversion sequence will be applied to an expression if necessary to convert it to an expression having a required destination type and value category.

² [Note 2: Expressions with a given type will be implicitly converted to other types in several contexts:

- (2.1) — When used as operands of operators. The operator’s requirements for its operands dictate the destination type (7.6).
- (2.2) — When used in the condition of an **if** statement (8.5.2) or iteration statement (8.6). The destination type is **bool**.
- (2.3) — When used in the expression of a **switch** statement (8.5.3). The destination type is integral.
- (2.4) — When used as the source expression for an initialization (which includes use as an argument in a function call and use as the expression in a **return** statement). The type of the entity being initialized is (generally) the destination type. See 9.5, 9.5.4.

— end note]

³ An expression E can be *implicitly converted* to a type T if and only if the declaration $T\ t = E$; is well-formed, for some invented temporary variable t (9.5).

⁴ Certain language constructs require that an expression be converted to a Boolean value. An expression E appearing in such a context is said to be *contextually converted to bool* and is well-formed if and only if the declaration $\text{bool } t(E)$; is well-formed, for some invented temporary variable t (9.5).

⁵ Certain language constructs require conversion to a value having one of a specified set of types appropriate to the construct. An expression E of class type C appearing in such a context is said to be *contextually implicitly converted* to a specified type T and is well-formed if and only if E can be implicitly converted to a type T that is determined as follows: C is searched for non-explicit conversion functions whose return type is $cv\ T$ or reference to $cv\ T$ such that T is allowed by the context. There shall be exactly one such T .

⁶ The effect of any implicit conversion is the same as performing the corresponding declaration and initialization and then using the temporary variable as the result of the conversion. The result is an lvalue if T is an lvalue reference type or an rvalue reference to function type (9.3.4.3), an xvalue if T is an rvalue reference to object type, and a prvalue otherwise. The expression E is used as a glvalue if and only if the initialization uses it as a glvalue.

⁷ [Note 3: For class types, user-defined conversions are considered as well; see 11.4.8. In general, an implicit conversion sequence (12.2.4.2) consists of a standard conversion sequence followed by a user-defined conversion followed by another standard conversion sequence. — end note]

⁸ [Note 4: There are some contexts where certain conversions are suppressed. For example, the lvalue-to-rvalue conversion is not done on the operand of the unary **&** operator. Specific exceptions are given in the descriptions of those operators and contexts. — end note]

7.3.2 Lvalue-to-rvalue conversion

[conv.lval]

¹ A glvalue (7.2.1) of a non-function, non-array type T can be converted to a prvalue.³⁷ If T is an incomplete type, a program that necessitates this conversion is ill-formed. If T is a non-class type, the type of the prvalue is the cv-unqualified version of T . Otherwise, the type of the prvalue is T .³⁸

² When an lvalue-to-rvalue conversion is applied to an expression E , and either

- (2.1) — E is not potentially evaluated, or
- (2.2) — the evaluation of E results in the evaluation of a member E_x of the set of potential results of E , and E_x names a variable x that is not odr-used by E_x (6.3),

the value contained in the referenced object is not accessed.

[Example 1:

```
struct S { int n; };
auto f() {
    S x { 1 };
    constexpr S y { 2 };
    return [&](bool b) { return (b ? y : x).n; };
}
auto g = f();
int m = g(false);    // undefined behavior: access of x.n outside its lifetime
int n = g(true);     // OK, does not access y.n
```

³⁷ For historical reasons, this conversion is called the “lvalue-to-rvalue” conversion, even though that name does not accurately reflect the taxonomy of expressions described in 7.2.1.

³⁸ In C++ class and array prvalues can have cv-qualified types. This differs from C, in which non-lvalues never have cv-qualified types.

— *end example*]

³ The result of the conversion is determined according to the following rules:

- (3.1) — If T is `cv std::nullptr_t`, the result is a null pointer constant (7.3.12).

[*Note 1*: Since the conversion does not access the object to which the glvalue refers, there is no side effect even if T is volatile-qualified (6.10.1), and the glvalue can refer to an inactive member of a union (11.5). — *end note*]

- (3.2) — Otherwise, if T has a class type, the conversion copy-initializes the result object from the glvalue.

- (3.3) — Otherwise, if the bits in the value representation of the object to which the glvalue refers are not valid for the object's type, the behavior is undefined (F.3.5).

[*Example 2*:

```
bool f() {
    bool b = true;
    char c = 42;
    memcpy(&b, &c, 1);
    return b;          // undefined behavior if 42 is not a valid value representation for bool
}
```

— *end example*]

- (3.4) — Otherwise, the object indicated by the glvalue is read (3.1). Let V be the value contained in the object. The prvalue result is

- (3.4.1) — the value of type T congruent (6.9.3) to V if T is an integer type,

- (3.4.2) — the result of `reinterpret_cast<T>(reinterpret_cast<std::uintptr_t>(V))` if T is volatile-qualified and V is a pointer value that is not valid in the context of the evaluation (6.9.5), and

- (3.4.3) — V otherwise.

⁴ [*Note 2*: See also 7.2.1. — *end note*]

7.3.3 Array-to-pointer conversion

[conv.array]

- ¹ An expression E of type “array of N T ” or “array of unknown bound of T ” can be converted to a prvalue of type “pointer to T ”. If E is a prvalue, the temporary materialization conversion (7.3.5) is applied. If the result of E (possibly converted) is an object whose type is similar to the type of E , the result is a pointer to the first element of the array; otherwise, the behavior is undefined.

7.3.4 Function-to-pointer conversion

[conv.func]

- ¹ An lvalue of function type T can be converted to a prvalue of type “pointer to T ”. The result is a pointer to the function.³⁹

7.3.5 Temporary materialization conversion

[conv.rval]

- ¹ A prvalue of type T can be converted to an xvalue of type T . This conversion initializes a temporary object (6.8.7) of type T from the prvalue by evaluating the prvalue with the temporary object as its result object, and produces an xvalue denoting the temporary object. T shall be a complete type.

[*Note 1*: If T is a class type (or array thereof), it must have an accessible and non-deleted destructor; see 11.4.7. — *end note*]

[*Example 1*:

```
struct X { int n; };
int k = X().n;          // OK, X() prvalue is converted to xvalue
```

— *end example*]

7.3.6 Qualification conversions

[conv.qual]

- ¹ A *qualification-decomposition* of a type T is a sequence of cv_i and P_i such that T is

“ $cv_0 P_0 cv_1 P_1 \cdots cv_{n-1} P_{n-1} cv_n U$ ” for $n \geq 0$,

³⁹) This conversion never applies to non-static member functions because an lvalue that refers to a non-static member function cannot be obtained.

where each cv_i is a set of cv-qualifiers (6.9.6), and each P_i is “pointer to” (9.3.4.2), “pointer to member of class C_i of type” (9.3.4.4), “array of N_i ”, or “array of unknown bound of” (9.3.4.5). If P_i designates an array, the cv-qualifiers cv_{i+1} on the element type are also taken as the cv-qualifiers cv_i of the array.

[Example 1: The type denoted by the *type-id* `const int **` has three qualification-decompositions, taking U as “`int`”, as “pointer to `const int`”, and as “pointer to pointer to `const int`”. — end example]

- 2 Two types $T1$ and $T2$ are *similar* if they have qualification-decompositions with the same n such that corresponding P_i components are either the same or one is “array of N_i ” and the other is “array of unknown bound of”, and the types denoted by U are the same.
- 3 The *qualification-combined type* of two types $T1$ and $T2$ is the type $T3$ similar to $T1$ whose qualification-decomposition is such that:
 - (3.1) — for every $i > 0$, cv_i^3 is the union of cv_i^1 and cv_i^2 ,
 - (3.2) — if either P_i^1 or P_i^2 is “array of unknown bound of”, P_i^3 is “array of unknown bound of”, otherwise it is P_i^1 , and
 - (3.3) — if the resulting cv_i^3 is different from cv_i^1 or cv_i^2 , or the resulting P_i^3 is different from P_i^1 or P_i^2 , then `const` is added to every cv_k^3 for $0 < k < i$,

where cv_i^j and P_i^j are the components of the qualification-decomposition of Tj . A prvalue of type $T1$ can be converted to type $T2$ if the qualification-combined type of $T1$ and $T2$ is $T2$.

[Note 1: If a program could assign a pointer of type `T**` to a pointer of type `const T**` (that is, if line #1 below were allowed), a program could inadvertently modify a `const` object (as it is done on line #2). For example,

```
int main() {
    const char c = 'c';
    char* pc;
    const char** pcc = &pc;      // #1: not allowed
    *pcc = &c;
    *pc = 'C';                  // #2: modifies a const object
}
```

— end note]

[Note 2: Given similar types $T1$ and $T2$, this construction ensures that both can be converted to the qualification-combined type of $T1$ and $T2$. — end note]

- 4 [Note 3: A prvalue of type “pointer to $cv1$ T ” can be converted to a prvalue of type “pointer to $cv2$ T ” if “ $cv2$ T ” is more cv-qualified than “ $cv1$ T ”. A prvalue of type “pointer to member of X of type $cv1$ T ” can be converted to a prvalue of type “pointer to member of X of type $cv2$ T ” if “ $cv2$ T ” is more cv-qualified than “ $cv1$ T ”. — end note]
- 5 [Note 4: Function types (including those used in pointer-to-member-function types) are never cv-qualified (9.3.4.6). — end note]

7.3.7 Integral promotions

[conv.prom]

- 1 For the purposes of 7.3.7, a *converted bit-field* is a prvalue that is the result of an lvalue-to-rvalue conversion (7.3.2) applied to a bit-field (11.4.10).
- 2 A prvalue that is not a converted bit-field and has an integer type other than `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t` whose integer conversion rank (6.9.7) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.
- 3 A prvalue of an unscoped enumeration type whose underlying type is not fixed can be converted to a prvalue of the first of the following types that can represent all the values of the enumeration (9.8.1): `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of the enumeration, a prvalue of an unscoped enumeration type can be converted to a prvalue of the extended integer type with lowest integer conversion rank (6.9.7) greater than the rank of `long long` in which all the values of the enumeration can be represented. If there are two such extended types, the signed one is chosen.
- 4 A prvalue of an unscoped enumeration type whose underlying type is fixed (9.8.1) can be converted to a prvalue of its underlying type. Moreover, if integral promotion can be applied to its underlying type, a prvalue of an unscoped enumeration type whose underlying type is fixed can also be converted to a prvalue of the promoted underlying type.

[Note 1: A converted bit-field of enumeration type is treated as any other value of that type for promotion purposes. — end note]

- 5 A converted bit-field of integral type can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field.
- 6 A prvalue of type `char8_t`, `char16_t`, `char32_t`, or `wchar_t` (6.9.3) (including a converted bit-field that was not already promoted to `int` or `unsigned int` according to the rules above) can be converted to a prvalue of the first of the following types that can represent all the values of its underlying type: `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, `unsigned long long int`, or its underlying type.
- 7 A prvalue of type `bool` can be converted to a prvalue of type `int`, with `false` becoming zero and `true` becoming one.
- 8 These conversions are called *integral promotions*.

7.3.8 Floating-point promotion

[conv.fpprom]

- 1 A prvalue of type `float` can be converted to a prvalue of type `double`. The value is unchanged.
- 2 This conversion is called *floating-point promotion*.

7.3.9 Integral conversions

[conv.integral]

- 1 A prvalue of an integer type can be converted to a prvalue of another integer type. A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.
- 2 If the destination type is `bool`, see 7.3.15. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.
- 3 Otherwise, the result is the unique value of the destination type that is congruent to the source integer modulo 2^N , where N is the width of the destination type.
- 4 The conversions allowed as integral promotions are excluded from the set of integral conversions.

7.3.10 Floating-point conversions

[conv.double]

- 1 A prvalue of floating-point type can be converted to a prvalue of another floating-point type with a greater or equal conversion rank (6.9.7). A prvalue of standard floating-point type can be converted to a prvalue of another standard floating-point type.
- 2 If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined (F.3.6).
- 3 The conversions allowed as floating-point promotions are excluded from the set of floating-point conversions.

7.3.11 Floating-integral conversions

[conv.fpoint]

- 1 A prvalue of a floating-point type can be converted to a prvalue of an integer type. The conversion truncates; that is, the fractional part is discarded. The behavior is undefined (F.3.7) if the truncated value cannot be represented in the destination type.

[Note 1: If the destination type is `bool`, see 7.3.15. — end note]

- 2 A prvalue of an integer type or of an unscoped enumeration type can be converted to a prvalue of a floating-point type. The result is exact if possible. If the value being converted is in the range of values that can be represented but the value cannot be represented exactly, it is an implementation-defined choice of either the next lower or higher representable value.

[Note 2: Loss of precision occurs if the integral value cannot be represented exactly as a value of the floating-point type. — end note]

If the value being converted is outside the range of values that can be represented, the behavior is undefined (F.3.8). If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.

7.3.12 Pointer conversions

[conv.ptr]

- 1 A *null pointer constant* is an integer literal (5.13.2) with value zero or a prvalue of type `std::nullptr_t`. A null pointer constant can be converted to a pointer type; the result is the null pointer value of that type (6.9.5)

and is distinguishable from every other value of object pointer or function pointer type. Such a conversion is called a *null pointer conversion*. The conversion of a null pointer constant to a pointer to cv-qualified type is a single conversion, and not the sequence of a pointer conversion followed by a qualification conversion (7.3.6). A null pointer constant of integral type can be converted to a prvalue of type `std::nullptr_t`.

[Note 1: The resulting prvalue is not a null pointer value. — end note]

- ² A prvalue of type “pointer to cv T”, where T is an object type, can be converted to a prvalue of type “pointer to cv void”. The pointer value (6.9.5) is unchanged by this conversion.
- ³ A prvalue v of type “pointer to cv D”, where D is a complete class type, can be converted to a prvalue of type “pointer to cv B”, where B is a base class (11.7) of D. If B is an inaccessible (11.8) or ambiguous (6.5.2) base class of D, a program that necessitates this conversion is ill-formed. If v is a null pointer value, the result is a null pointer value. Otherwise, if B is a virtual base class of D or is a base class of a virtual base class of D and v does not point to an object whose type is similar (7.3.6) to D and that is within its lifetime or within its period of construction or destruction (11.9.5), the behavior is undefined (F.3.9). Otherwise, the result is a pointer to the base class subobject of the derived class object.

7.3.13 Pointer-to-member conversions

[conv.mem]

- ¹ A null pointer constant (7.3.12) can be converted to a pointer-to-member type; the result is the *null member pointer value* of that type and is distinguishable from any pointer to member not created from a null pointer constant. Such a conversion is called a *null member pointer conversion*. The conversion of a null pointer constant to a pointer to member of cv-qualified type is a single conversion, and not the sequence of a pointer-to-member conversion followed by a qualification conversion (7.3.6).
- ² A prvalue of type “pointer to member of B of type cv T”, where B is a class type, can be converted to a prvalue of type “pointer to member of D of type cv T”, where D is a complete class derived (11.7) from B. If B is an inaccessible (11.8), ambiguous (6.5.2), or virtual (11.7.2) base class of D, or a base class of a virtual base class of D, a program that necessitates this conversion is ill-formed. If class D does not contain the original member and is not a base class of the class containing the original member, the behavior is undefined (F.3.10). Otherwise, the result of the conversion refers to the same member as the pointer to member before the conversion took place, but it refers to the base class member as if it were a member of the derived class. The result refers to the member in D’s instance of B. Since the result has type “pointer to member of D of type cv T”, indirection through it with a D object is valid. The result is the same as if indirecting through the pointer to member of B with the B subobject of D. The null member pointer value is converted to the null member pointer value of the destination type.⁴⁰

7.3.14 Function pointer conversions

[conv.fctptr]

- ¹ A prvalue of type “pointer to `noexcept` function” can be converted to a prvalue of type “pointer to function”. The result is a pointer to the function. A prvalue of type “pointer to member of type `noexcept` function” can be converted to a prvalue of type “pointer to member of type function”. The result designates the member function.

[Example 1:

```
void (*p)();
void (**pp)() noexcept = &p;    // error: cannot convert to pointer to noexcept function

struct S { typedef void (*p)(); operator p(); };
void (*q)() noexcept = S();    // error: cannot convert to pointer to noexcept function
```

— end example]

7.3.15 Boolean conversions

[conv.bool]

- ¹ A prvalue of arithmetic, unscoped enumeration, pointer, or pointer-to-member type can be converted to a prvalue of type `bool`. A zero value, null pointer value, or null member pointer value is converted to `false`; any other value is converted to `true`.

⁴⁰ The rule for conversion of pointers to members (from pointer to member of base to pointer to member of derived) appears inverted compared to the rule for pointers to objects (from pointer to derived to pointer to base) (7.3.12, 11.7). This inversion is necessary to ensure type safety. Note that a pointer to member is not an object pointer or a function pointer and the rules for conversions of such pointers do not apply to pointers to members. In particular, a pointer to member cannot be converted to a `void*`.

7.4 Usual arithmetic conversions

[expr.arith.conv]

¹ Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:

- (1.1) — The lvalue-to-rvalue conversion (7.3.2) is applied to each operand and the resulting prvalues are used in place of the original operands for the remainder of this section.
- (1.2) — If either operand is of scoped enumeration type (9.8.1), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.
- (1.3) — Otherwise, if one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, the expression is ill-formed.
- (1.4) — Otherwise, if either operand is of floating-point type, the following rules are applied:
 - (1.4.1) — If both operands have the same type, no further conversion is performed.
 - (1.4.2) — Otherwise, if one of the operands is of a non-floating-point type, that operand is converted to the type of the operand with the floating-point type.
 - (1.4.3) — Otherwise, if the floating-point conversion ranks (6.9.7) of the types of the operands are ordered but not equal, then the operand of the type with the lesser floating-point conversion rank is converted to the type of the other operand.
 - (1.4.4) — Otherwise, if the floating-point conversion ranks of the types of the operands are equal, then the operand with the lesser floating-point conversion subrank (6.9.7) is converted to the type of the other operand.
 - (1.4.5) — Otherwise, the expression is ill-formed.
- (1.5) — Otherwise, each operand is converted to a common type C. The integral promotion rules (7.3.7) are used to determine a type T1 and type T2 for each operand.⁴¹ Then the following rules are applied to determine C:
 - (1.5.1) — If T1 and T2 are the same type, C is that type.
 - (1.5.2) — Otherwise, if T1 and T2 are both signed integer types or are both unsigned integer types, C is the type with greater rank.
 - (1.5.3) — Otherwise, let U be the unsigned integer type and S be the signed integer type.
 - (1.5.3.1) — If U has rank greater than or equal to the rank of S, C is U.
 - (1.5.3.2) — Otherwise, if S can represent all of the values of U, C is S.
 - (1.5.3.3) — Otherwise, C is the unsigned integer type corresponding to S.

7.5 Primary expressions

[expr.prim]

7.5.1 Grammar

[expr.prim.grammar]

```

primary-expression:
    literal
    this
    ( expression )
    id-expression
    lambda-expression
    fold-expression
    requires-expression
    splice-expression
  
```

7.5.2 Literals

[expr.prim.literal]

¹ The type of a *literal* is determined based on its form as specified in 5.13. A *string-literal* is an lvalue designating a corresponding string literal object (5.13.5), a *user-defined-literal* has the same value category as the corresponding operator call expression described in 5.13.9, and any other *literal* is a prvalue.

⁴¹ As a consequence, operands of type `bool`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, or of enumeration type are converted to some integral type.

7.5.3 This

[expr.prim.this]

- ¹ The keyword **this** names a pointer to the object for which an implicit object member function (11.4.3) is invoked or a non-static data member's initializer (11.4) is evaluated.
- ² The *current class* at a program point is the class associated with the innermost class scope containing that point.

[Note 1: A *lambda-expression* does not introduce a class scope. — end note]

- ³ If the expression **this** appears within the predicate of a contract assertion (6.11.1) (including as the result of an implicit transformation (7.5.5.1) and including in the bodies of nested *lambda-expressions*) and the current class encloses the contract assertion, **const** is combined with the *cv-qualifier-seq* used to generate the resulting type (see below).
- ⁴ If a declaration declares a member function or member function template of a class **X**, the expression **this** is a prvalue of type “pointer to *cv-qualifier-seq* **X**” wherever **X** is the current class between the optional *cv-qualifier-seq* and the end of the *function-definition*, *member-declarator*, or *declarator*. It shall not appear within the declaration of a static or explicit object member function of the current class (although its type and value category are defined within such member functions as they are within an implicit object member function).

[Note 2: This is because declaration matching does not occur until the complete declarator is known. — end note]

[Note 3: In a *trailing-return-type*, the class being defined is not required to be complete for purposes of class member access (7.6.1.5). Class members declared later are not visible.

[Example 1:

```
struct A {
    char g();
    template<class T> auto f(T t) -> decltype(t + g())
    { return t + g(); }
};
template auto A::f(int t) -> decltype(t + g());
```

— end example]

— end note]

- ⁵ Otherwise, if a *member-declarator* declares a non-static data member (11.4) of a class **X**, the expression **this** is a prvalue of type “pointer to **X**” wherever **X** is the current class within the optional default member initializer (11.4).
- ⁶ The expression **this** shall not appear in any other context.

[Example 2:

```
class Outer {
    int a[sizeof(*this)];           // error: not inside a member function
    unsigned int sz = sizeof(*this); // OK, in default member initializer

    void f() {
        int b[sizeof(*this)];      // OK

        struct Inner {
            int c[sizeof(*this)];   // error: not inside a member function of Inner
        };
    }
};
```

— end example]

7.5.4 Parentheses

[expr.prim.paren]

- ¹ A parenthesized expression (*E*) is a primary expression whose type, result, and value category are identical to those of *E*. The parenthesized expression can be used in exactly the same contexts as those where *E* can be used, and with the same meaning, except as otherwise indicated.

7.5.5 Names

[expr.prim.id]

7.5.5.1 General

[expr.prim.id.general]

id-expression:
 unqualified-id
 qualified-id
 pack-index-expression

- ¹ An *id-expression* is a restricted form of a *primary-expression*.

[Note 1: An *id-expression* can appear after . and -> operators (7.6.1.5). — end note]

- ² If an *id-expression* *E* denotes a non-static non-type member of some class *C* at a point where the current class (7.5.3) is *X* and

- (2.1) — *E* is potentially evaluated or *C* is *X* or a base class of *X*, and
- (2.2) — *E* is not the *id-expression* of a class member access expression (7.6.1.5), and
- (2.3) — *E* is not the *id-expression* of a *reflect-expression* (7.6.2.10), and
- (2.4) — if *E* is a *qualified-id*, *E* is not the un-parenthesized operand of the unary & operator (7.6.2.2),

the *id-expression* is transformed into a class member access expression using (**this*) as the object expression. If this transformation occurs in the predicate of a precondition assertion of a constructor of *X* or a postcondition assertion of a destructor of *X*, the expression is ill-formed.

[Note 2: If *C* is not *X* or a base class of *X*, the class member access expression is ill-formed. Also, if the *id-expression* occurs within a static or explicit object member function, the class member access is ill-formed. — end note]

This transformation does not apply in the template definition context (13.8.3.2).

[Example 1:

```
struct C {
    bool b;
    C() pre(b)           // error
        pre(&this->b)     // OK
        pre(sizeof(b) > 0); // OK, b is not potentially evaluated.
};
```

— end example]

- ³ If an *id-expression* *E* denotes a variant member *M* of an anonymous union (11.5.2) *U*:

- (3.1) — If *U* is a non-static data member, *E* refers to *M* as a member of the lookup context of the terminal name of *E* (after any implicit transformation to a class member access expression).

[Example 2: *o.u.x* is interpreted as *o.u.x*, where *u* names the anonymous union member. — end example]

- (3.2) — Otherwise, *E* is interpreted as a class member access (7.6.1.5) that designates the member subobject *M* of the anonymous union variable for *U*.

[Note 3: Under this interpretation, *E* no longer denotes a non-static data member. — end note]

[Example 3: *N::x* is interpreted as *N::u.x*, where *u* names the anonymous union variable. — end example]

- ⁴ An *id-expression* or *splice-expression* that designates a non-static data member or implicit object member function of a class can only be used:

- (4.1) — as part of a class member access (after any implicit transformation (see above)) in which the object expression refers to the member's class or a class derived from that class, or
- (4.2) — to form a pointer to member (7.6.2.2), or
- (4.3) — if that *id-expression* or *splice-expression* designates a non-static data member and it appears in an unevaluated operand.

[Example 4:

```
struct S {
    int m;
};
int i = sizeof(S::m);           // OK
int j = sizeof(S::m + 42);      // OK
int S::*k = &[::~S::m];        // OK
```

— end example]

- ⁵ A *splice-expression* that designates a direct base class relationship shall appear only as the second operand of a class member access.
- ⁶ For an *id-expression* that denotes an overload set, overload resolution is performed to select a unique function (12.2, 12.3).

[Note 4: A program cannot refer to a function with a trailing *requires-clause* whose *constraint-expression* is not satisfied, because such functions are never selected by overload resolution.

[Example 5:

```
template<typename T> struct A {
    static void f(int) requires false;
};

void g() {
    A<int>::f(0);                // error: cannot call f
    void (*p1)(int) = A<int>::f; // error: cannot take the address of f
    decltype(A<int>::f)* p2 = nullptr; // error: the type decltype(A<int>::f) is invalid
}
```

In each case, the constraints of **f** are not satisfied. In the declaration of **p2**, those constraints need to be satisfied even though **f** is an unevaluated operand (7.2.3). — end example]

— end note]

7.5.5.2 Unqualified names

[expr.prim.id.unqual]

unqualified-id:

- identifier*
- operator-function-id*
- conversion-function-id*
- literal-operator-id*
- ~ type-name*
- ~ computed-type-specifier*
- template-id*

- ¹ An *identifier* is only an *id-expression* if it has been suitably declared (Clause 9) or if it appears as part of a *declarator-id* (9.3).

[Note 1: For *operator-function-ids*, see 12.4; for *conversion-function-ids*, see 11.4.8.3; for *literal-operator-ids*, see 12.6; for *template-ids*, see 13.3. A *type-name* or *computed-type-specifier* prefixed by *~* denotes the destructor of the type so named; see 7.5.5.5. — end note]

- ² A *component name* of an *unqualified-id* *U* is

- (2.1) — *U* if it is a name or
- (2.2) — the component name of the *template-id* or *type-name* of *U*, if any.

[Note 2: Other constructs that contain names to look up can have several component names (7.5.5.3, 9.2.9.3, 9.2.9.5, 9.3.4.4, 9.10, 13.2, 13.3, 13.8). — end note]

The *terminal name* of a construct is the component name of that construct that appears lexically last.

- ³ The result is the entity denoted by the *unqualified-id* (6.5.3).

- ⁴ If

- (4.1) — the *unqualified-id* appears in a *lambda-expression* at program point *P*,
- (4.2) — the entity is a local entity (6.1) or a variable declared by an *init-capture* (7.5.6.3),
- (4.3) — naming the entity within the *compound-statement* of the innermost enclosing *lambda-expression* of *P*, but not in an unevaluated operand, would refer to an entity captured by copy in some intervening *lambda-expression*, and
- (4.4) — *P* is in the function parameter scope, but not the *parameter-declaration-clause*, of the innermost such *lambda-expression* *E*,

then the type of the expression is the type of a class member access expression (7.6.1.5) naming the non-static data member that would be declared for such a capture in the object parameter (9.3.4.6) of the function call operator of *E*.

[Note 3: If *E* is not declared **mutable**, the type of such an identifier will typically be **const** qualified. — end note]

[Example 1:

```
void f() {
    float x, &r = x;

    [=]() -> decltype((x)) {           // lambda returns float const& because this lambda is not mutable and
                                        // x is an lvalue
        decltype(x) y1;                // y1 has type float
        decltype((x)) y2 = y1;         // y2 has type float const&
        decltype(r) r1 = y1;           // r1 has type float&
        decltype((r)) r2 = y2;         // r2 has type float const&
        return y2;
    };

    [=](decltype((x)) y) {
        decltype((x)) z = x;           // OK, y has type float&, z has type float const&
    };

    [=] {
        [](decltype((x)) y) {};        // OK, lambda takes a parameter of type float const&

        [x=1](decltype((x)) y) {
            decltype((x)) z = x;        // OK, y has type int&, z has type int const&
        };
    };
}
```

— end example]

5 Otherwise, if the *unqualified-id* names a coroutine parameter, the type of the expression is that of the copy of the parameter (9.6.4), and the result is that copy.

6 Otherwise, if the *unqualified-id* names a result binding (9.4.2) attached to a function with return type U,

(6.1) — if U is “reference to T”, then the type of the expression is **const** T;

(6.2) — otherwise, the type of the expression is **const** U.

7 Otherwise, if the *unqualified-id* appears in the predicate of a contract assertion *C* (6.11) and the entity is

(7.1) — a variable declared outside of *C* of object type T,

(7.2) — a variable or template parameter declared outside of *C* of type “reference to T”, or

(7.3) — a structured binding of type T whose corresponding variable is declared outside of *C*,

then the type of the expression is **const** T.

8 [Example 2:

```
int n = 0;
struct X { bool m(); };

struct Y {
    int z = 0;

    void f(int i, int* p, int& r, X x, X* px)
    pre (++n)           // error: attempting to modify const lvalue
    pre (++i)           // error: attempting to modify const lvalue
    pre (++(*p))        // OK
    pre (++r)           // error: attempting to modify const lvalue
    pre (x.m())         // error: calling non-const member function
    pre (px->m())        // OK
    pre ([=,&i,*this] mutable {
        ++n;           // error: attempting to modify const lvalue
        ++i;           // error: attempting to modify const lvalue
        ++p;           // OK, refers to member of closure type
        ++r;           // OK, refers to non-reference member of closure type
        ++this->z;       // OK, captured *this
        ++z;           // OK, captured *this
        int j = 17;
    });
};
```

```

[&]{
    int k = 34;
    ++i;    // error: attempting to modify const lvalue
    ++j;    // OK
    ++k;    // OK
}();
return true;
}());

template <int N, int& R, int* P>
void g()
    pre(++N)        // error: attempting to modify prvalue
    pre(++R)        // error: attempting to modify const lvalue
    pre(++(*P));    // OK

int h()
    post(r : ++r)    // error: attempting to modify const lvalue
    post(r: [=] mutable {
        ++r;        // OK, refers to member of closure type
        return true;
    }());

int& k()
    post(r : ++r);  // error: attempting to modify const lvalue
};
— end example]

```

- 9 Otherwise, if the entity is a template parameter object for a template parameter of type *T* (13.2), the type of the expression is **const T**.
- 10 In all other cases, the type of the expression is the type of the entity.
- 11 [Note 4: The type will be adjusted as described in 7.2.2 if it is cv-qualified or is a reference type. — end note]
- 12 The expression is an xvalue if it is move-eligible (see below); an lvalue if the entity is a function, variable, structured binding (9.7), result binding (9.4.2), data member, or template parameter object; and a prvalue otherwise (7.2.1); it is a bit-field if the identifier designates a bit-field.
- 13 If an *id-expression* *E* appears in the predicate of a function contract assertion attached to a function *f* and denotes a function parameter of *f* and the implementation introduces any temporary objects to hold the value of that parameter as specified in 6.8.7,
 - (13.1) — if the contract assertion is a precondition assertion and the evaluation of the precondition assertion is sequenced before the initialization of the parameter object, *E* refers to the most recently initialized such temporary object, and
 - (13.2) — if the contract assertion is a postcondition assertion, it is unspecified whether *E* refers to one of the temporary objects or the parameter object; the choice is consistent within a single evaluation of a postcondition assertion.
- 14 If an *id-expression* *E* names a result binding in a postcondition assertion and the implementation introduces any temporary objects to hold the result object as specified in 6.8.7, and the postcondition assertion is sequenced before the initialization of the result object (7.6.1.3), *E* refers to the most recently initialized such temporary object.
- 15 An *implicitly movable entity* is a variable with automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. An *id-expression* or *splice-expression* (7.5.9) is *move-eligible* if
 - (15.1) — it designates an implicitly movable entity,
 - (15.2) — it is the (possibly parenthesized) operand of a **return** (8.8.4) or **co_return** (8.8.5) statement or of a *throw-expression* (7.6.18), and
 - (15.3) — each intervening scope between the declaration of the entity and the innermost enclosing scope of the expression is a block scope and, for a *throw-expression*, is not the block scope of a *try-block* or *function-try-block*.

7.5.5.3 Qualified names

[expr.prim.id.qual]

qualified-id:
nested-name-specifier *template*_{opt} *unqualified-id*

nested-name-specifier:
`::`
type-name `::`
namespace-name `::`
computed-type-specifier `::`
splice-scope-specifier `::`
nested-name-specifier *identifier* `::`
nested-name-specifier *template*_{opt} *simple-template-id* `::`

splice-scope-specifier:
splice-specifier
*template*_{opt} *splice-specialization-specifier*

- ¹ The component names of a *qualified-id* are those of its *nested-name-specifier* and *unqualified-id*. The component names of a *nested-name-specifier* are its *identifier* (if any) and those of its *type-name*, *namespace-name*, *simple-template-id*, and/or *nested-name-specifier*.
- ² A *splice-specifier* or *splice-specialization-specifier* that is not followed by `::` is never interpreted as part of a *splice-scope-specifier*. The keyword `template` may only be omitted from the form *template*_{opt} *splice-specialization-specifier* `::` when the *splice-specialization-specifier* is preceded by `typename`.

[Example 1:

```
template<int V>
struct TCls {
    static constexpr int s = V;
    using type = int;
};

int v1 = [::TCls<1>::]::s;
int v2 = template [::TCls:]<2>::s;           // OK, template binds to splice-scope-specifier
typename [::TCls:]<3>::type v3 = 3;         // OK, typename binds to the qualified name
template [::TCls:]<3>::type v4 = 4;         // OK, template binds to the splice-scope-specifier
typename template [::TCls:]<3>::type v5 = 5; // OK, same as v3
[::TCls:]<3>::type v6 = 6;                  // error: unexpected <
```

— end example]

- ³ A *nested-name-specifier* is *declarative* if it is part of
 - (3.1) — a *class-head-name* (11.1),
 - (3.2) — an *enum-head-name* (9.8.1),
 - (3.3) — a *qualified-id* that is the *id-expression* of a *declarator-id* (9.3.1),
 - (3.4) — an *elaborated-type-specifier* of an explicit instantiation (13.9.3), or
 - (3.5) — a declarative *nested-name-specifier*.

A declarative *nested-name-specifier* shall not have a *computed-type-specifier* or a *splice-scope-specifier*. A declaration that uses a declarative *nested-name-specifier* shall be a friend declaration or inhabit a scope that contains the entity being redeclared or specialized.

- ⁴ The entity designated by a *nested-name-specifier* is determined as follows:
 - (4.1) — The *nested-name-specifier* `::` designates the global namespace.
 - (4.2) — A *nested-name-specifier* with a *computed-type-specifier* designates the same type designated by the *computed-type-specifier*, which shall be a class or enumeration type.
 - (4.3) — For a *nested-name-specifier* of the form *splice-specifier* `::`, the *splice-specifier* shall designate a class or enumeration type or a namespace. The *nested-name-specifier* designates the same entity as the *splice-specifier*.
 - (4.4) — For a *nested-name-specifier* of the form *template*_{opt} *splice-specialization-specifier* `::`, the *splice-specifier* of the *splice-specialization-specifier* shall designate a class template or an alias template *T*. Letting *S* be the specialization of *T* corresponding to the template argument list of the *splice-specialization-specifier*,

S shall either be a class template specialization or an alias template specialization that denotes a class or enumeration type. The *nested-name-specifier* designates the underlying entity of S .

- (4.5) — If a *nested-name-specifier* N is declarative and has a *simple-template-id* with a template argument list A that involves a template parameter, let T be the template nominated by N without A . T shall be a class template.
- (4.5.1) — If A is the template argument list (13.4) of the corresponding *template-head* H (13.7.3), N designates the primary template of T ; H shall be equivalent to the *template-head* of T (13.7.7.2).
- (4.5.2) — Otherwise, N designates the partial specialization (13.7.6) of T whose template argument list is equivalent to A (13.7.7.2); the program is ill-formed if no such partial specialization exists.
- (4.6) — Any other *nested-name-specifier* designates the entity denotes by its *type-name*, *namespace-name*, *identifier*, or *simple-template-id*. If the *nested-name-specifier* is not declarative, the entity shall not be a template.
- ⁵ A *qualified-id* shall not be of the form *nested-name-specifier* `templateopt ~ computed-type-specifier` nor of the form *computed-type-specifier* `:: ~ type-name`.
- ⁶ The result of a *qualified-id* Q is the entity it denotes (6.5.5).
- ⁷ If Q appears in the predicate of a contract assertion C (6.11) and the entity is
 - (7.1) — a variable declared outside of C of object type T ,
 - (7.2) — a variable declared outside of C of type “reference to T ”, or
 - (7.3) — a structured binding of type T whose corresponding variable is declared outside of C ,
 then the type of the expression is `const T`.
- ⁸ Otherwise, the type of the expression is the type of the result.
- ⁹ The result is an lvalue if the member is
 - (9.1) — a function other than a non-static member function,
 - (9.2) — a non-static member function if Q is the operand of a unary `&` operator,
 - (9.3) — a variable,
 - (9.4) — a structured binding (9.7), or
 - (9.5) — a data member,
 and a prvalue otherwise.

7.5.5.4 Pack indexing expression

[expr.prim.pack.index]

pack-index-expression:
id-expression ... [*constant-expression*]

- ¹ The *id-expression* P in a *pack-index-expression* shall be an *identifier* that denotes a pack.
- ² The *constant-expression* shall be a converted constant expression (7.7.3) of type `std::size_t` whose value V , termed the index, is such that $0 \leq V < \text{sizeof} \dots (P)$.
- ³ A *pack-index-expression* is a pack expansion (13.7.4).
- ⁴ [Note 1: A *pack-index-expression* denotes the V^{th} element of the pack. — end note]

7.5.5.5 Destruction

[expr.prim.id.dtor]

- ¹ An *id-expression* that denotes the destructor of a type T names the destructor of T if T is a class type (11.4.7), otherwise the *id-expression* is said to name a *pseudo-destructor*.
- ² If the *id-expression* names a pseudo-destructor, T shall be a scalar type and the *id-expression* shall appear as the right operand of a class member access (7.6.1.5) that forms the *postfix-expression* of a function call (7.6.1.3).

[Note 1: Such a call ends the lifetime of the object (7.6.1.3, 6.8.4). — end note]

- ³ [Example 1:

```
struct C { };
void f() {
    C * pc = new C;
    using C2 = C;
    pc->C::~~C2();    // OK, destroys *pc
```

```

C().C::~~C();           // undefined behavior: temporary of type C destroyed twice
using T = int;
0 .T::~~T();           // OK, no effect
0.T::~~T();            // error: 0.T is a user-defined-floating-point-literal (5.13.9)
}

```

— end example]

7.5.6 Lambda expressions

[expr.prim.lambda]

7.5.6.1 General

[expr.prim.lambda.general]

lambda-expression:

lambda-introducer *attribute-specifier-seq*_{opt} *lambda-declarator* *compound-statement*
lambda-introducer < *template-parameter-list* > *requires-clause*_{opt} *attribute-specifier-seq*_{opt}
lambda-declarator *compound-statement*

lambda-introducer:

[*lambda-capture*_{opt}]

lambda-declarator:

lambda-specifier-seq *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt} *trailing-return-type*_{opt}
*function-contract-specifier-seq*_{opt}
noexcept-specifier *attribute-specifier-seq*_{opt} *trailing-return-type*_{opt} *function-contract-specifier-seq*_{opt}
*trailing-return-type*_{opt} *function-contract-specifier-seq*_{opt}
(*parameter-declaration-clause*) *lambda-specifier-seq*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt}
*trailing-return-type*_{opt} *requires-clause*_{opt} *function-contract-specifier-seq*_{opt}

lambda-specifier:

`constexpr`
`constexpr`
`mutable`
`static`

lambda-specifier-seq:

lambda-specifier *lambda-specifier-seq*_{opt}

- ¹ A *lambda-expression* provides a concise way to create a simple function object.

[Example 1:

```

#include <algorithm>
#include <cmath>
void absort(float* x, unsigned N) {
    std::sort(x, x + N, [](float a, float b) { return std::abs(a) < std::abs(b); });
}

```

— end example]

- ² A *lambda-expression* is a prvalue whose result object is called the *closure object*.

[Note 1: A closure object behaves like a function object (22.10). — end note]

- ³ An ambiguity can arise because a *requires-clause* can end in an *attribute-specifier-seq*, which collides with the *attribute-specifier-seq* in *lambda-expression*. In such cases, any attributes are treated as *attribute-specifier-seq* in *lambda-expression*.

[Note 2: Such ambiguous cases cannot have valid semantics because the constraint expression would not have type `bool`.

[Example 2:

```

auto x = []<class T> requires T::operator int [[some_attribute]] (int) { }

```

— end example]

— end note]

- ⁴ A *lambda-specifier-seq* shall contain at most one of each *lambda-specifier* and shall not contain both `constexpr` and `constexpr`. If the *lambda-declarator* contains an explicit object parameter (9.3.4.6), then no *lambda-specifier* in the *lambda-specifier-seq* shall be `mutable` or `static`. The *lambda-specifier-seq* shall not contain both `mutable` and `static`. If the *lambda-specifier-seq* contains `static`, there shall be no *lambda-capture*.

[Note 3: The trailing *requires-clause* is described in 9.3. — end note]

- ⁵ A *lambda-expression's parameter-declaration-clause* is the *parameter-declaration-clause* of the *lambda-expression's lambda-declarator*, if any, or empty otherwise. If the *lambda-declarator* does not include a *trailing-return-type*, it is considered to be `-> auto`.

[Note 4: In that case, the return type is deduced from `return` statements as described in 9.2.9.7. — end note]

[Example 3:

```
auto x1 = [](int i) { return i; };           // OK, return type is int
auto x2 = []{ return { 1, 2 }; };           // error: deducing return type from braced-init-list
int j;
auto x3 = [&]()->auto&& { return j; };        // OK, return type is int&
```

— end example]

- ⁶ A lambda is a *generic lambda* if the *lambda-expression* has any generic parameter type placeholders (9.2.9.7), or if the lambda has a *template-parameter-list*.

[Example 4:

```
auto x = [](int i, auto a) { return i; };    // OK, a generic lambda
auto y = [](this auto self, int i) { return i; }; // OK, a generic lambda
auto z = []<class T>(int i) { return i; };    // OK, a generic lambda
```

— end example]

7.5.6.2 Closure types

[`expr.prim.lambda.closure`]

- 1 The type of a *lambda-expression* (which is also the type of the closure object) is a unique, unnamed non-union class type, called the *closure type*, whose properties are described below.
- 2 The closure type is incomplete until the end of its corresponding *compound-statement*.
- 3 The closure type is declared in the smallest block scope, class scope, or namespace scope that contains the corresponding *lambda-expression*.

[Note 1: This determines the set of namespaces and classes associated with the closure type (6.5.4). The parameter types of a *lambda-declarator* do not affect these associated namespaces and classes. — end note]

- 4 The closure type is not an aggregate type (9.5.2) and is not `final` (11.1); it is a structural type (13.2) if and only if the lambda has no *lambda-capture*. An implementation may define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:

- (4.1) — the size and/or alignment of the closure type,
- (4.2) — whether the closure type is trivially copyable (11.2), or
- (4.3) — whether the closure type is a standard-layout class (11.2).

An implementation shall not add members of rvalue reference type to the closure type.

- ⁵ The closure type for a *lambda-expression* has a public inline function call operator (for a non-generic lambda) or function call operator template (for a generic lambda) (12.4.4) whose parameters and return type are those of the *lambda-expression's parameter-declaration-clause* and *trailing-return-type* respectively, and whose *template-parameter-list* consists of the specified *template-parameter-list*, if any. The function call operator or the function call operator template are direct members of the closure type. The *requires-clause* of the function call operator template is the *requires-clause* immediately following `< template-parameter-list >`, if any. The trailing *requires-clause* of the function call operator or operator template is the *requires-clause* of the *lambda-declarator*, if any.

[Note 2: The function call operator template for a generic lambda can be an abbreviated function template (9.3.4.6). — end note]

[Example 1:

```
auto glambda = [](auto a, auto&& b) { return a < b; };
bool b = glambda(3, 3.14);                                     // OK

auto vglambda = [](auto printer) {
    return [=](auto&& ... ts) {
        printer(std::forward<decltype(ts)>(ts)...);
    };
};                                                             // OK, ts is a function parameter pack
```

```

    return [=]() {
        printer(ts ...);
    };
};
auto p = vglambda( [](auto v1, auto v2, auto v3)
    { std::cout << v1 << v2 << v3; } );
auto q = p(1, 'a', 3.14); // OK, outputs 1a3.14
q(); // OK, outputs 1a3.14

auto fact = [](this auto self, int n) -> int { // OK, explicit object parameter
    return (n <= 1) ? 1 : n * self(n-1);
};
std::cout << fact(5); // OK, outputs 120
— end example]

```

- 6 Given a lambda with a *lambda-capture*, the type of the explicit object parameter, if any, of the lambda's function call operator (possibly instantiated from a function call operator template) shall be either:

- (6.1) — the closure type,
- (6.2) — a class type publicly and unambiguously derived from the closure type, or
- (6.3) — a reference to a possibly cv-qualified such type.

[Example 2:

```

struct C {
    template <typename T>
    C(T);
};

void func(int i) {
    int x = [=](this auto&&) { return i; }(); // OK
    int y = [=](this C) { return i; }(); // error
    int z = [](this C) { return 42; }(); // OK
}

```

— end example]

- 7 The function call operator or operator template is a static member function or static member function template (11.4.9.2) if the *lambda-expression's parameter-declaration-clause* is followed by **static**. Otherwise, it is a non-static member function or member function template (11.4.3) that is declared **const** (11.4.3) if and only if the *lambda-expression's parameter-declaration-clause* is not followed by **mutable** and the *lambda-declarator* does not contain an explicit object parameter. It is neither virtual nor declared **volatile**. Any *noexcept-specifier* or *function-contract-specifier* (9.4.1) specified on a *lambda-expression* applies to the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-declarator* appertains to the type of the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-expression* preceding a *lambda-declarator* appertains to the corresponding function call operator or operator template. The function call operator or any given operator template specialization is a **constexpr** function if either the corresponding *lambda-expression's parameter-declaration-clause* is followed by **constexpr** or **constexpr**, or it is **constexpr-suitable** (9.2.6). It is an immediate function (9.2.6) if the corresponding *lambda-expression's parameter-declaration-clause* is followed by **constexpr**.

[Example 3:

```

auto ID = [](auto a) { return a; };
static_assert(ID(3) == 3); // OK

struct NonLiteral {
    NonLiteral(int n) : n(n) { }
    int n;
};
static_assert(ID(NonLiteral{3}).n == 3); // error

```

— end example]

- 8 [Example 4:

```

auto monoid = [](auto v) { return [=] { return v; }; };

```

```

auto add = [](auto m1) constexpr {
    auto ret = m1();
    return [=](auto m2) mutable {
        auto m1val = m1();
        auto plus = [=](auto m2val) mutable constexpr
            { return m1val += m2val; };
        ret = plus(m2());
        return monoid(ret);
    };
};
constexpr auto zero = monoid(0);
constexpr auto one = monoid(1);
static_assert(add(one)(zero)() == one());           // OK

// Since two below is not declared constexpr, an evaluation of its constexpr member function call operator
// cannot perform an lvalue-to-rvalue conversion on one of its subobjects (that represents its capture)
// in a constant expression.
auto two = monoid(2);
assert(two() == 2); // OK, not a constant expression.
static_assert(add(one)(one)() == two());           // error: two() is not a constant expression
static_assert(add(one)(one)() == monoid(2)());     // OK
— end example]

```

- ⁹ [Note 3: The function call operator or operator template can be constrained (13.5.3) by a *type-constraint* (13.2), a *requires-clause* (13.1), or a trailing *requires-clause* (9.3).

[Example 5:

```

template <typename T> concept C1 = /* ... */;
template <std::size_t N> concept C2 = /* ... */;
template <typename A, typename B> concept C3 = /* ... */;

auto f = [<typename T1, C1 T2> requires C2<sizeof(T1) + sizeof(T2)>
    (T1 a1, T1 b1, T2 a2, auto a3, auto a4) requires C3<decltype(a4), T2> {
    // T2 is constrained by a type-constraint.
    // T1 and T2 are constrained by a requires-clause, and
    // T2 and the type of a4 are constrained by a trailing requires-clause.
};

```

— end example]

— end note]

- ¹⁰ If all potential references to a local entity implicitly captured by a *lambda-expression* *L* occur within the function contract assertions (9.4.1) of the call operator or operator template of *L* or within *assertion-statements* (8.9) within the body of *L*, the program is ill-formed.

[Note 4: Adding a contract assertion to an existing C++ program cannot cause additional captures. — end note]

[Example 6:

```

static int i = 0;

void test() {
    auto f1 = [=] pre(i > 0) {}; // OK, no local entities are captured.

    int i = 1;
    auto f2 = [=] pre(i > 0) {}; // error: cannot implicitly capture i here
    auto f3 = [i] pre(i > 0) {}; // OK, i is captured explicitly.

    auto f4 = [=] {
        contract_assert(i > 0); // error: cannot implicitly capture i here
    };

    auto f5 = [=] {
        contract_assert(i > 0); // OK, i is referenced elsewhere.
        (void)i;
    };
}

```

```

auto f6 = [=] pre(                                // #1
    []{
        bool x = true;
        return [=]{ return x; }();              // OK, #1 captures nothing.
    }()) {};

bool y = true;
auto f7 = [=] pre([=]{ return y; }());           // error: outer capture of y is invalid.
}
— end example]

```

- 11 The closure type for a non-generic *lambda-expression* with no *lambda-capture* and no explicit object parameter (9.3.4.6) whose constraints (if any) are satisfied has a conversion function to pointer to function with C++ language linkage (9.12) having the same parameter and return types as the closure type’s function call operator. The conversion is to “pointer to **noexcept** function” if the function call operator has a non-throwing exception specification. If the function call operator is a static member function, then the value returned by this conversion function is a pointer to the function call operator. Otherwise, the value returned by this conversion function is a pointer to a function *F* that, when invoked, has the same effect as invoking the closure type’s function call operator on a default-constructed instance of the closure type. *F* is a *constexpr* function if the function call operator is a *constexpr* function and is an immediate function if the function call operator is an immediate function.
- 12 For a generic lambda with no *lambda-capture* and no explicit object parameter (9.3.4.6), the closure type has a conversion function template to pointer to function. The conversion function template has the same invented template parameter list, and the pointer to function has the same parameter types, as the function call operator template. The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization.
- 13 [Note 5: If the generic lambda has no *trailing-return-type* or the *trailing-return-type* contains a placeholder type, return type deduction of the corresponding function call operator template specialization has to be done. The corresponding specialization is that instantiation of the function call operator template with the same template arguments as those deduced for the conversion function template. Consider the following:

```

auto glambda = [](auto a) { return a; };
int (*fp)(int) = glambda;

```

The behavior of the conversion function of *glambda* above is like that of the following conversion function:

```

struct Closure {
    template<class T> auto operator()(T t) const { /* ... */ }
    template<class T> static auto lambda_call_operator_invoker(T a) {
        // forwards execution to operator()(a) and therefore has
        // the same return type deduced
        /* ... */
    }
    template<class T> using fp_ptr_t =
        decltype(lambda_call_operator_invoker(declval<T>())) (*)(T);

    template<class T> operator fp_ptr_t<T>() const
        { return &lambda_call_operator_invoker; }
};

```

— end note]

[Example 7:

```

void f1(int (*)(int)) { }
void f2(char (*)(int)) { }

void g(int (*)(int)) { }      // #1
void g(char (*)(char)) { }    // #2

void h(int (*)(int)) { }      // #3
void h(char (*)(int)) { }     // #4

auto glambda = [](auto a) { return a; };
f1(glambda);                  // OK

```

```

f2(glambda);           // error: ID is not convertible
g(glambda);           // error: ambiguous
h(glambda);           // OK, calls #3 since it is convertible from ID
int& (*fpi)(int*) = [] (auto* a) -> auto& { return *a; }; // OK
— end example]

```

- 14 If the function call operator template is a static member function template, then the value returned by any given specialization of this conversion function template is a pointer to the corresponding function call operator template specialization. Otherwise, the value returned by any given specialization of this conversion function template is a pointer to a function *F* that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization on a default-constructed instance of the closure type. *F* is a constexpr function if the corresponding specialization is a constexpr function and *F* is an immediate function if the function call operator template specialization is an immediate function.

[Note 6: This will result in the implicit instantiation of the generic lambda's body. The instantiated generic lambda's return type and parameter types need to match the return type and parameter types of the pointer to function. — end note]

[Example 8:

```

auto GL = [] (auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL;           // OK, through conversion function template
GL_int(3);                         // OK, same as GL(3)
— end example]

```

- 15 The conversion function or conversion function template is public, constexpr, non-virtual, non-explicit, const, and has a non-throwing exception specification (14.5).

[Example 9:

```

auto Fwd = [] (int (*fp)(int), auto a) { return fp(a); };
auto C = [] (auto a) { return a; };

static_assert(Fwd(C,3) == 3); // OK

// No specialization of the function call operator template can be constexpr (due to the local static).
auto NC = [] (auto a) { static int s; return a; };
static_assert(Fwd(NC,3) == 3); // error
— end example]

```

- 16 The *lambda-expression's compound-statement* yields the *function-body* (9.6) of the function call operator, but it is not within the scope of the closure type.

[Example 10:

```

struct S1 {
    int x, y;
    int operator()(int);
    void f() {
        [=]() -> int {
            return operator()(this->x + y); // equivalent to S1::operator()(this->x + (*this).y)
                                           // this has type S1*
        };
    }
};
— end example]

```

Unless the *compound-statement* is that of a *consteval-block-declaration* (9.1), a variable `__func__` is implicitly defined at the beginning of the *compound-statement* of the *lambda-expression*, with semantics as described in 9.6.1.

- 17 The closure type associated with a *lambda-expression* has no default constructor if the *lambda-expression* has a *lambda-capture* and a defaulted default constructor otherwise. It has a defaulted copy constructor and a defaulted move constructor (11.4.5.3). It has a deleted copy assignment operator if the *lambda-expression* has a *lambda-capture* and defaulted copy and move assignment operators otherwise (11.4.6).

[Note 7: These special member functions are implicitly defined as usual, which can result in them being defined as deleted. — end note]

- 18 The closure type associated with a *lambda-expression* has an implicitly-declared destructor (11.4.7).
- 19 A member of a closure type shall not be explicitly instantiated (13.9.3), explicitly specialized (13.9.4), or named in a friend declaration (11.8.4).

7.5.6.3 Captures

[expr.prim.lambda.capture]

```

lambda-capture:
    capture-default
    capture-list
    capture-default , capture-list

capture-default:
    &
    =

capture-list:
    capture
    capture-list , capture

capture:
    simple-capture
    init-capture

simple-capture:
    identifier ...opt
    & identifier ...opt
    this
    * this

init-capture:
    ...opt identifier initializer
    & ...opt identifier initializer

```

- 1 The body of a *lambda-expression* may refer to local entities of enclosing scopes by capturing those entities, as described below.
- 2 If a *lambda-capture* includes a *capture-default* that is `&`, no identifier in a *simple-capture* of that *lambda-capture* shall be preceded by `&`. If a *lambda-capture* includes a *capture-default* that is `=`, each *simple-capture* of that *lambda-capture* shall be of the form “& *identifier* ..._{opt}”, “*this*”, or “* *this*”.

[Note 1: The form [`&`,*this*] is redundant but accepted for compatibility with C++ 2014. — end note]

Ignoring appearances in *initializers* of *init-captures*, an identifier or `this` shall not appear more than once in a *lambda-capture*.

[Example 1:

```

struct S2 { void f(int i); };
void S2::f(int i) {
    [&, i]{ };           // OK
    [&, this, i]{ };     // OK, equivalent to [&, i]
    [&, &i]{ };          // error: i preceded by & when & is the default
    [=, *this]{ };       // OK
    [=, this]{ };        // OK, equivalent to [=]
    [i, i]{ };           // error: i repeated
    [this, *this]{ };    // error: this appears twice
}

```

— end example]

- 3 A *lambda-expression* shall not have a *capture-default* or *simple-capture* in its *lambda-introducer* unless
- (3.1) — its innermost enclosing scope is a block scope (6.4.3),
 - (3.2) — it appears within a default member initializer and its innermost enclosing scope is the corresponding class scope (6.4.7), or
 - (3.3) — it appears within a contract assertion and its innermost enclosing scope is the corresponding contract-assertion scope (6.4.10).
- 4 The *identifier* in a *simple-capture* shall denote a local entity (6.5.3, 6.1). The *simple-captures* `this` and `*this` denote the local entity `*this`. An entity that is designated by a *simple-capture* is said to be *explicitly captured*.

- ⁵ If an *identifier* in a *capture* appears as the *declarator-id* of a parameter of the *lambda-declarator*'s *parameter-declaration-clause* or as the name of a template parameter of the *lambda-expression*'s *template-parameter-list*, the program is ill-formed.

[Example 2:

```
void f() {
    int x = 0;
    auto g = [x](int x) { return 0; };    // error: parameter and capture have the same name
    auto h = [y = 0]<typename y>(y) { return 0; };    // error: template parameter and capture
                                                    // have the same name
}
```

— end example]

- ⁶ An *init-capture* inhabits the lambda scope (6.4.5) of the *lambda-expression*. An *init-capture* without ellipsis behaves as if it declares and explicitly captures a variable of the form “`auto init-capture ;`”, except that:
- (6.1) — if the capture is by copy (see below), the non-static data member declared for the capture and the variable are treated as two different ways of referring to the same object, which has the lifetime of the non-static data member, and no additional copy and destruction is performed, and
 - (6.2) — if the capture is by reference, the variable's lifetime ends when the closure object's lifetime ends.

[Note 2: This enables an *init-capture* like “`x = std::move(x)`”; the second “`x`” must bind to a declaration in the surrounding context. — end note]

[Example 3:

```
int x = 4;
auto y = [&r = x, x = x+1]()->int {
    r += 2;
    return x+2;
}();    // Updates ::x to 6, and initializes y to 7.

auto z = [a = 42](int a) { return 1; };    // error: parameter and conceptual local variable
                                           // have the same name

auto counter = [i=0]() mutable -> decltype(i) {    // OK, returns int
    return i++;
};
```

— end example]

- ⁷ For the purposes of lambda capture, an expression potentially references local entities as follows:
- (7.1) — An *id-expression* that names a local entity potentially references that entity; an *id-expression* that names one or more non-static class members and does not form a pointer to member (7.6.2.2) potentially references `*this`.
 [Note 3: This occurs even if overload resolution selects a static member function for the *id-expression*. — end note]
 - (7.2) — A `this` expression potentially references `*this`.
 - (7.3) — A *lambda-expression* potentially references the local entities named by its *simple-captures*.

If an expression potentially references a local entity within a scope in which it is odr-usable (6.3), and the expression would be potentially evaluated if the effect of any enclosing `typeid` expressions (7.6.1.8) were ignored, the entity is said to be *implicitly captured* by each intervening *lambda-expression* with an associated *capture-default* that does not explicitly capture it. The implicit capture of `*this` is deprecated when the *capture-default* is `=`; see D.3.

[Example 4:

```
void f(int, const int (&)[2] = {});    // #1
void f(const int&, const int (&)[1]);    // #2
void test() {
    const int x = 17;
    auto g = [](auto a) {
        f(x);    // OK, calls #1, does not capture x
    };
}
```

```

auto g1 = [=](auto a) {
    f(x);                      // OK, calls #1, captures x
};

auto g2 = [=](auto a) {
    int selector[sizeof(a) == 1 ? 1 : 2]{};
    f(x, selector);           // OK, captures x, can call #1 or #2
};

auto g3 = [=](auto a) {
    typeid(a + x);             // captures x regardless of whether a + x is an unevaluated operand
};
}

```

Within `g1`, an implementation can optimize away the capture of `x` as it is not odr-used. — *end example*]

[*Note 4*: The set of captured entities is determined syntactically, and entities are implicitly captured even if the expression denoting a local entity is within a discarded statement (8.5.2).

[*Example 5*:

```

template<bool B>
void f(int n) {
    [=](auto a) {
        if constexpr (B && sizeof(a) > 4) {
            (void)n;           // captures n regardless of the value of B and sizeof(int)
        }
    }(0);
}

```

— *end example*]

— *end note*]

- ⁸ An entity is *captured* if it is captured explicitly or implicitly. An entity captured by a *lambda-expression* is odr-used (6.3) by the *lambda-expression*.

[*Note 5*: As a consequence, if a *lambda-expression* explicitly captures an entity that is not odr-usable, the program is ill-formed (6.3). — *end note*]

[*Example 6*:

```

void f1(int i) {
    int const N = 20;
    auto m1 = [=]{
        int const M = 30;
        auto m2 = [i]{
            int x[N][M];
            x[0][0] = i;
        };
    };
    struct s1 {
        int f;
        void work(int n) {
            int m = n*n;
            int j = 40;
            auto m3 = [this,m] {
                auto m4 = [&,j] {
                    int x = n;
                    x += m;
                    x += i;
                    x += f;
                };
            };
        }
    };
}

```

// OK, N and M are not odr-used
// OK, i is explicitly captured by m2 and implicitly captured by m1

// error: j not odr-usable due to intervening lambda m3
// error: n is odr-used but not odr-usable due to intervening lambda m3
// OK, m implicitly captured by m4 and explicitly captured by m3
// error: i is odr-used but not odr-usable
// due to intervening function and class scopes
// OK, this captured implicitly by m4 and explicitly by m3

```

struct s2 {
    double ohseven = .007;
    auto f() {
        return [this] {
            return [*this] {
                return ohseven;    // OK
            };
        }();
    }
    auto g() {
        return [] {
            return [*this] { };    // error: *this not captured by outer lambda-expression
        }();
    }
};

```

— end example]

- ⁹ [Note 6: Because local entities are not odr-usable within a default argument (6.3), a *lambda-expression* appearing in a default argument cannot implicitly or explicitly capture any local entity. Such a *lambda-expression* can still have an *init-capture* if any full-expression in its *initializer* satisfies the constraints of an expression appearing in a default argument (9.3.4.7). — end note]

[Example 7:

```

void f2() {
    int i = 1;
    void g1(int = ([i]{ return i; }())());    // error
    void g2(int = ([i]{ return 0; }())());    // error
    void g3(int = ([=]{ return i; }())());    // error
    void g4(int = ([=]{ return 0; }())());    // OK
    void g5(int = ([ ]{ return sizeof i; }())());    // OK
    void g6(int = ([x=1]{ return x; }())());    // OK
    void g7(int = ([x=i]{ return x; }())());    // error
}

```

— end example]

- ¹⁰ An entity is *captured by copy* if

- (10.1) — it is implicitly captured, the *capture-default* is `=`, and the captured entity is not `*this`, or
- (10.2) — it is explicitly captured with a capture that is not of the form `this`, `& identifier ...opt`, or `& ...opt identifier initializer`.

For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The type of such a data member is the referenced type if the entity is a reference to an object, an lvalue reference to the referenced function type if the entity is a reference to a function, or the type of the corresponding captured entity otherwise. A member of an anonymous union shall not be captured by copy.

- ¹¹ Every *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use (6.3) of an entity captured by copy is transformed into an access to the corresponding unnamed data member of the closure type.

[Note 7: An *id-expression* that is not an odr-use refers to the original entity, never to a member of the closure type. However, such an *id-expression* can still cause the implicit capture of the entity. — end note]

If `*this` is captured by copy, each expression that odr-uses `*this` is transformed to instead refer to the corresponding unnamed data member of the closure type.

[Example 8:

```

void f(const int*);
void g() {
    const int N = 10;
    [=] {
        int arr[N];    // OK, not an odr-use, refers to variable with automatic storage duration
        f(&N);          // OK, causes N to be captured; &N points to
                        // the corresponding member of the closure type
    };
}

```

— end example]

- ¹² An entity is *captured by reference* if it is implicitly or explicitly captured but not captured by copy. It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference. If declared, such non-static data members shall be of literal type.

[Example 9:

// The inner closure type must be a literal type regardless of how reference captures are represented.

```
static_assert([](int n) { return [&n] { return ++n; }(); }(3) == 4);
```

— end example]

A bit-field or a member of an anonymous union shall not be captured by reference.

- ¹³ An *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use of a reference captured by reference refers to the entity to which the captured reference is bound and not to the captured reference.

[Note 8: The validity of such captures is determined by the lifetime of the object to which the reference refers, not by the lifetime of the reference itself. — end note]

[Example 10:

```
auto h(int &r) {
    return [&] {
        ++r;           // Valid after h returns if the lifetime of the
                       // object to which r is bound has not ended
    };
}
```

— end example]

- ¹⁴ If a *lambda-expression* *m2* captures an entity and that entity is captured by an immediately enclosing *lambda-expression* *m1*, then *m2*'s capture is transformed as follows:

- (14.1) — If *m1* captures the entity by copy, *m2* captures the corresponding non-static data member of *m1*'s closure type; if *m1* is not *mutable*, the non-static data member is considered to be *const-qualified*.
- (14.2) — If *m1* captures the entity by reference, *m2* captures the same entity captured by *m1*.

[Example 11: The nested *lambda-expressions* and invocations below will output 123234.

```
int a = 1, b = 1, c = 1;
auto m1 = [a, &b, &c]() mutable {
    auto m2 = [a, b, &c]() mutable {
        std::cout << a << b << c;
        a = 4; b = 4; c = 4;
    };
    a = 3; b = 3; c = 3;
    m2();
};
a = 2; b = 2; c = 2;
m1();
std::cout << a << b << c;
```

— end example]

- ¹⁵ The non-static data members in the closure type are declared in an unspecified order, except that the members introduced for explicit captures are declared in an order consistent with that of those captures.
- ¹⁶ The entities that are captured by copy are used to direct-initialize each corresponding non-static data member of the resulting closure object. The non-static data members corresponding to the *init-captures* and the (reference) variables declared by *init-captures* without corresponding non-static data members are initialized as indicated by the corresponding *initializer* (which may be copy- or direct-initialization). (For array members, the array elements are direct-initialized in increasing subscript order.) These initializations are performed when the *lambda-expression* is evaluated, in an order consistent with the (partially unspecified) order in which the non-static data members are declared and with the order of explicit captures.

[Note 9: This ensures that the destructions will occur in the reverse order of the constructions. — end note]

[Example 12:

```
void f(std::vector<int> &&v) {
    [size = v.size(), own = std::move(v)] {};
}
```

Since `size` is initialized before `own`, it contains the original size of `v`. — *end example*

- ¹⁷ [Note 10: If a non-reference entity is implicitly or explicitly captured by reference, invoking the function call operator of the corresponding *lambda-expression* after the lifetime of the entity has ended is likely to result in undefined behavior. — *end note*]

- ¹⁸ A *simple-capture* containing an ellipsis is a pack expansion (13.7.4). An *init-capture* containing an ellipsis is a pack expansion that declares an *init-capture* pack (13.7.4).

[Example 13:

```
template<class... Args>
void f(Args... args) {
    auto lm = [&, args...] { return g(args...); };
    lm();

    auto lm2 = [...xs=std::move(args)] { return g(xs...); };
    lm2();
}
```

— *end example*]

7.5.7 Fold expressions

[expr.prim.fold]

- ¹ A fold expression performs a fold of a pack (13.7.4) over a binary operator.

fold-expression:

```
( cast-expression fold-operator ... )
( ... fold-operator cast-expression )
( cast-expression fold-operator ... fold-operator cast-expression )
```

fold-operator: one of

```
+ - * / % ^ & | << >>
+= -= *= /= %= ^= &= |= <<= >>= =
== != < > <= >= && || , .* ->*
```

- ² An expression of the form `(... op e)` where *op* is a *fold-operator* is called a *unary left fold*. An expression of the form `(e op ...)` where *op* is a *fold-operator* is called a *unary right fold*. Unary left folds and unary right folds are collectively called *unary folds*. In a unary fold, the *cast-expression* shall contain an unexpanded pack (13.7.4).
- ³ An expression of the form `(e1 op1 ... op2 e2)` where *op1* and *op2* are *fold-operators* is called a *binary fold*. In a binary fold, *op1* and *op2* shall be the same *fold-operator*, and either *e1* shall contain an unexpanded pack or *e2* shall contain an unexpanded pack, but not both. If *e2* contains an unexpanded pack, the expression is called a *binary left fold*. If *e1* contains an unexpanded pack, the expression is called a *binary right fold*.

[Example 1:

```
template<typename ...Args>
bool f(Args ...args) {
    return (true && ... && args); // OK
}

template<typename ...Args>
bool f(Args ...args) {
    return (args + ... + args); // error: both operands contain unexpanded packs
}
```

— *end example*]

- ⁴ A fold expression is a pack expansion.

7.5.8 Requires expressions

[expr.prim.req]

7.5.8.1 General

[expr.prim.req.general]

- ¹ A *requires-expression* provides a concise way to express requirements on template arguments that can be checked by name lookup (6.5) or by checking properties of types and expressions.

```

requires-expression:
    requires requirement-parameter-listopt requirement-body

requirement-parameter-list:
    ( parameter-declaration-clause )

requirement-body:
    { requirement-seq }

requirement-seq:
    requirement requirement-seqopt

requirement:
    simple-requirement
    type-requirement
    compound-requirement
    nested-requirement

```

² A *requires-expression* is a prvalue of type `bool` whose value is described below.

³ [Example 1: A common use of *requires-expressions* is to define requirements in concepts such as the one below:

```

template<typename T>
concept R = requires (T t) {
    typename T::type;
    {t} -> std::convertible_to<const typename T::type&>;
};

```

A *requires-expression* can also be used in a *requires-clause* (13.1) as a way of writing ad hoc constraints on template arguments such as the one below:

```

template<typename T>
requires requires (T x) { x + x; }
T add(T a, T b) { return a + b; }

```

The first `requires` introduces the *requires-clause*, and the second introduces the *requires-expression*. — end example]

⁴ A *requires-expression* may introduce local parameters using a *parameter-declaration-clause*. A local parameter of a *requires-expression* shall not have a default argument. The type of such a parameter is determined as specified for a function parameter in 9.3.4.6. These parameters have no linkage, storage, or lifetime; they are only used as notation for the purpose of defining *requirements*. The *parameter-declaration-clause* of a *requirement-parameter-list* shall not terminate with an ellipsis.

[Example 2:

```

template<typename T>
concept C = requires(T t, ...) {    // error: terminates with an ellipsis
    t;
};
template<typename T>
concept C2 = requires(T p[2]) {
    (decltype(p))nullptr;          // OK, p has type "pointer to T"
};

```

— end example]

⁵ The substitution of template arguments into a *requires-expression* can result in the formation of invalid types or expressions in the immediate context of its *requirements* (13.10.3.1) or the violation of the semantic constraints of those *requirements*. In such cases, the *requires-expression* evaluates to `false`; it does not cause the program to be ill-formed. The substitution and semantic constraint checking proceeds in lexical order and stops when a condition that determines the result of the *requires-expression* is encountered. If substitution (if any) and semantic constraint checking succeed, the *requires-expression* evaluates to `true`.

[Note 1: If a *requires-expression* contains invalid types or expressions in its *requirements*, and it does not appear within the declaration of a templated entity, then the program is ill-formed. — end note]

If the substitution of template arguments into a *requirement* would always result in a substitution failure, the program is ill-formed; no diagnostic required (G.4.1).

[Example 3:

```

template<typename T> concept C =
requires {
    new decltype((void)T{});    // ill-formed, no diagnostic required

```

```
};
```

— end example]

7.5.8.2 Simple requirements

[expr.prim.req.simple]

simple-requirement:
expression ;

- ¹ A *simple-requirement* asserts the validity of an *expression*. The *expression* is an unevaluated operand.

[Note 1: The enclosing *requires-expression* will evaluate to **false** if substitution of template arguments into the *expression* fails. — end note]

[Example 1:

```
template<typename T> concept C =
  requires (T a, T b) {
    a + b;           // C<T> is true if a + b is a valid expression
  };
```

— end example]

- ² A *requirement* that starts with a **requires** token is never interpreted as a *simple-requirement*.

[Note 2: This simplifies distinguishing between a *simple-requirement* and a *nested-requirement*. — end note]

7.5.8.3 Type requirements

[expr.prim.req.type]

type-requirement:
 typename *nested-name-specifier*_{opt} *type-name* ;
 typename *splice-specifier* ;
 typename *splice-specialization-specifier* ;

- ¹ A *type-requirement* asserts the validity of a type. The component names of a *type-requirement* are those of its *nested-name-specifier* (if any) and *type-name* (if any).

[Note 1: The enclosing *requires-expression* will evaluate to **false** if substitution of template arguments fails. — end note]

[Example 1:

```
template<typename T, typename T::type = 0> struct S;
template<typename T> using Ref = T&;

template<typename T> concept C = requires {
  typename T::inner;           // required nested member name
  typename S<T>;               // required valid (13.3) template-id; fails if T::type does not exist as a type
                               // to which 0 can be implicitly converted
  typename Ref<T>;             // required alias template substitution, fails if T is void
  typename [:T::r1:];          // fails if T::r1 is not a reflection of a type
  typename [:T::r2:]<int>;      // fails if T::r2 is not a reflection of a template Z for which Z<int> is a type
};
```

— end example]

- ² A *type-requirement* that names a class template specialization does not require that type to be complete (6.9.1).

7.5.8.4 Compound requirements

[expr.prim.req.compound]

compound-requirement:
 { *expression* } *noexcept-specifier*_{opt} *return-type-requirement*_{opt} ;
return-type-requirement:
 -> *type-constraint*

- ¹ A *compound-requirement* asserts properties of the *expression E*. The *expression* is an unevaluated operand. Substitution of template arguments (if any) and verification of semantic properties proceed in the following order:

- (1.1) — Substitution of template arguments (if any) into the *expression* is performed.
- (1.2) — If the *noexcept-specifier* (14.5) is present, then:
 - (1.2.1) — Substitution of template arguments (if any) into its associated *constant-expression* is performed.

- (1.2.2) — If the value of the converted *constant-expression* is **true**, *E* shall not be a potentially-throwing expression.
- (1.3) — If the *return-type-requirement* is present, then:
 - (1.3.1) — Substitution of template arguments (if any) into the *return-type-requirement* is performed.
 - (1.3.2) — The immediately-declared constraint (13.2) of the *type-constraint* for `decltype((E))` shall be satisfied.

[*Example 1*: Given concepts *C* and *D*,

```
requires {
  { E1 } -> C;
  { E2 } -> D<A1, ..., An>;
};
```

is equivalent to

```
requires {
  E1; requires C<decltype((E1))>;
  E2; requires D<decltype((E2)), A1, ..., An>;
};
```

(including in the case where *n* is zero). — *end example*]

[*Example 2*:

```
template<typename T> concept C1 = requires(T x) {
  {x++};
};
```

The *compound-requirement* in *C1* requires that `x++` is a valid expression. It is equivalent to the *simple-requirement* `x++;`.

```
template<typename T> concept C2 = requires(T x) {
  {*x} -> std::same_as<typename T::inner>;
};
```

The *compound-requirement* in *C2* requires that `*x` is a valid expression, that `typename T::inner` is a valid type, and that `std::same_as<decltype((*x)), typename T::inner>` is satisfied.

```
template<typename T> concept C3 =
  requires(T x) {
    {g(x)} noexcept;
  };
```

The *compound-requirement* in *C3* requires that `g(x)` is a valid expression and that `g(x)` is non-throwing. — *end example*]

7.5.8.5 Nested requirements

[**expr.prim.req.nested**]

nested-requirement:

requires *constraint-expression* ;

- ¹ A *nested-requirement* can be used to specify additional constraints in terms of local parameters. The *constraint-expression* shall be satisfied (13.5.3) by the substituted template arguments, if any. Substitution of template arguments into a *nested-requirement* does not result in substitution into the *constraint-expression* other than as specified in 13.5.2.

[*Example 1*:

```
template<typename U> concept C = sizeof(U) == 1;

template<typename T> concept D = requires (T t) {
  requires C<decltype (+t)>;
};
```

`D<T>` is satisfied if `sizeof(decltype (+t)) == 1` (13.5.2.3). — *end example*]

7.5.9 Expression splicing

[**expr.prim.splice**]

splice-expression:

splice-specifier

template *splice-specifier*

template *splice-specialization-specifier*

- ¹ A *splice-specifier* or *splice-specialization-specifier* immediately followed by `::` or preceded by **typename** is never interpreted as part of a *splice-expression*.

[Example 1:

```
struct S { static constexpr int a = 1; };
template<typename> struct TCls { static constexpr int b = 2; };

constexpr int c = [::S]::a;           // OK, [::S] is not an expression
constexpr int d = template [::TCls:]<int>::b; // OK, template [::TCls:]<int> is not an expression
template<auto V> constexpr int e = [V]; // OK
constexpr int f = template [::e:]<S::a>; // OK

constexpr auto g = typename [::int:](42); // OK, typename [::int:] is a splice-type-specifier

constexpr auto h = ^g;
constexpr auto i = e<[::h]>;           // error: unparenthesized splice-expression used as template argument
constexpr auto j = e<([::h])>;        // OK
```

— end example]

- ² For a *splice-expression* *E* of the form *splice-specifier*, let *S* be the construct designated by *splice-specifier*.

- (2.1) — The expression is ill-formed if *S* is
 - (2.1.1) — a constructor,
 - (2.1.2) — a destructor,
 - (2.1.3) — an unnamed bit-field, or
 - (2.1.4) — a local entity (6.1) such that
 - (2.1.4.1) — there is a lambda scope that intervenes between the expression and the point at which *S* was introduced and
 - (2.1.4.2) — the expression would be potentially evaluated if the effect of any enclosing **typeid** expressions (7.6.1.8) were ignored.
- (2.2) — Otherwise, if *S* is a function *F*, the expression denotes an overload set containing all declarations of *F* that precede either the expression or the point immediately following the *class-specifier* of the outermost class for which the expression is in a complete-class context; overload resolution is performed (12.2, 12.3).
- (2.3) — Otherwise, if *S* is an object or a non-static data member, the expression is an lvalue designating *S*. The expression has the same type as that of *S*, and is a bit-field if and only if *S* is a bit-field.

[Note 1: The implicit transformation whereby an *id-expression* denoting a non-static member becomes a class member access (7.5.5) does not apply to a *splice-expression*. — end note]
- (2.4) — Otherwise, if *S* is a direct base class relationship (D, B), the expression is an lvalue designating *S*. The expression has the type B.
- (2.5) — Otherwise, if *S* is a variable or a structured binding, *S* shall either have static or thread storage duration or shall inhabit a scope enclosing the expression. The expression *E* is an lvalue referring to the object or function *X* associated with or referenced by *S*, and is a bit-field if and only if *X* is a bit-field. If *E* appears in the predicate of a contract assertion *C* (6.11) and *S* is
 - (2.5.1) — a variable declared outside of *C* of object type *T*,
 - (2.5.2) — a variable declared outside of *C* of type “reference to *T*”, or
 - (2.5.3) — a structured binding of type *T* whose corresponding variable is declared outside of *C*,
 then the type of *E* is **const T**, otherwise *E* has the same type as that of *S*.

[Note 2: The type of a *splice-expression* designating a variable or structured binding of reference type will be adjusted to a non-reference type (7.2.2). — end note]
- (2.6) — Otherwise, if *S* is a value or an enumerator, the expression is a prvalue that computes *S* and whose type is the same as that of *S*.
- (2.7) — Otherwise, the expression is ill-formed.

- ³ For a *splice-expression* of the form **template** *splice-specifier*, the *splice-specifier* shall designate a function template *T* that is not a constructor template. The expression denotes an overload set containing all declarations of *T* that precede either the expression or the point immediately following the *class-specifier* of the outermost class for which the expression is in a complete-class context; overload resolution is performed.

[*Note 3*: During overload resolution, candidate function templates undergo template argument deduction and the resulting specializations are considered as candidate functions. — *end note*]

- ⁴ For a *splice-expression* of the form **template** *splice-specialization-specifier*, the *splice-specifier* of the *splice-specialization-specifier* shall designate a template *T*.
- (4.1) — If *T* is a function template, the expression denotes an overload set containing all declarations of *T* that precede either the expression or the point immediately following the *class-specifier* of the outermost class for which the expression is in a complete-class context; overload resolution is performed (12.2, 12.3).
- (4.2) — Otherwise, if *T* is a variable template, let *S* be the specialization of *T* corresponding to the template argument list of the *splice-specialization-specifier*. The expression is an lvalue referring to the object associated with *S* and has the same type as that of *S*.
- (4.3) — Otherwise, the expression is ill-formed.

[*Note 4*: Class members are accessible from any point when designated by *splice-expressions* (11.8.3). A class member access expression (7.6.1.5) whose right operand is a *splice-expression* is ill-formed if the left operand (considered as a pointer) cannot be implicitly converted to a pointer to the designating class of the right operand. — *end note*]

7.6 Compound expressions

[**expr.compound**]

7.6.1 Postfix expressions

[**expr.post**]

7.6.1.1 General

[**expr.post.general**]

- ¹ Postfix expressions group left-to-right.

postfix-expression:

```

primary-expression
postfix-expression [ expression-listopt ]
postfix-expression ( expression-listopt )
simple-type-specifier ( expression-listopt )
typename-specifier ( expression-listopt )
simple-type-specifier braced-init-list
typename-specifier braced-init-list
postfix-expression . templateopt id-expression
postfix-expression . splice-expression
postfix-expression -> templateopt id-expression
postfix-expression -> splice-expression
postfix-expression ++
postfix-expression --
dynamic_cast < type-id > ( expression )
static_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
const_cast < type-id > ( expression )
typeid ( expression )
typeid ( type-id )

```

expression-list:

initializer-list

- ² [*Note 1*: The > token following the *type-id* in a **dynamic_cast**, **static_cast**, **reinterpret_cast**, or **const_cast** can be the product of replacing a >> token by two consecutive > tokens (13.3). — *end note*]

7.6.1.2 Subscripting

[**expr.sub**]

- ¹ A *subscript expression* is a postfix expression followed by square brackets containing a possibly empty, comma-separated list of *initializer-clauses* that constitute the arguments to the subscript operator. The *postfix-expression* and the initialization of the object parameter (9.3.4.6) of any applicable subscript operator function (12.4.5) is sequenced before each expression in the *expression-list* and also before any default argument (9.3.4.7). The initialization of a non-object parameter of a subscript operator function *S*, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other non-object parameter of *S*.

- ² With the built-in subscript operator, an *expression-list* shall be present, consisting of a single *assignment-expression*. One of the expressions shall be a glvalue of type “array of T” or a prvalue of type “pointer to T” and the other shall be a prvalue of unscoped enumeration or integral type. The result is of type “T”. The type “T” shall be a completely-defined object type.⁴² The expression E1[E2] is identical (by definition) to *((E1)+(E2)), except that in the case of an array operand, the result is an lvalue if that operand is an lvalue and an xvalue otherwise.

- ³ [Note 1: Despite its asymmetric appearance, subscripting is a commutative operation except for sequencing. See 7.6.2 and 7.6.6 for details of * and + and 9.3.4.5 for details of array types. — end note]

7.6.1.3 Function call

[expr.call]

- ¹ A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of *initializer-clauses* which constitute the arguments to the function.

[Note 1: If the postfix expression is a function name, the appropriate function and the validity of the call are determined according to the rules in 12.2. — end note]

The postfix expression shall have function type or function pointer type. For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3.4) is suppressed on the postfix expression), or a prvalue of function pointer type.

- ² If the selected function is non-virtual, or if the *id-expression* in the class member access expression is a *qualified-id*, that function is called. Otherwise, its final overrider (11.7.3) in the dynamic type of the object expression is called; such a call is referred to as a *virtual function call*.

[Note 2: The dynamic type is the type of the object referred to by the current value of the object expression. 11.9.5 describes the behavior of virtual function calls when the object expression refers to an object under construction or destruction. — end note]

- ³ [Note 3: If a function name is used, and name lookup (6.5) does not find a declaration of that name, the program is ill-formed. No function is implicitly declared by such a call. — end note]

- ⁴ If the *postfix-expression* names a destructor or pseudo-destructor (7.5.5.5), the type of the function call expression is `void`; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the `virtual` keyword), even if the type of the function actually called is different. If the *postfix-expression* names a pseudo-destructor (in which case the *postfix-expression* is a possibly-parenthesized class member access), the function call destroys the object of scalar type denoted by the object expression of the class member access (7.6.1.5, 6.8.4).

- ⁵ The precondition assertions of a function call are, in order:

- (5.1) — if the *postfix-expression* is a (possibly implicit) class member access expression and the call is a virtual function call, the precondition assertions (9.4.1) of the statically chosen function;
- (5.2) — the precondition assertions of the function actually called.

The postcondition assertions of a function call are, in order:

- (5.3) — the postcondition assertions of the function actually called;
- (5.4) — if the *postfix-expression* is a (possibly implicit) class member access expression and the call is a virtual function call, the postcondition assertions of the statically chosen function.

[Example 1:

```
struct X1 { virtual void f()          pre(a) post(b) {} };
struct X2 { virtual void f()          pre(c) post(d) {} };
struct Y : X1 { void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
    Z z;
    z.f(); // asserts g, h
    static_cast<Y*>(&z)->f(); // asserts e, g, h, f

    X1& x1ref = z;
    X2& x2ref = z;
```

⁴²) This is true even if the subscript operator is used in the following common idiom: `&x[0]`.

```

x1ref.f();           // asserts a, g, h, b
x2ref.f();           // asserts c, g, h, d
x1ref.X1::f();        // asserts a, b
x1ref.[::~X1::f]();   // asserts a, g, h, b

void (X1::*pmf)() = &X1::f;
(x1ref.*pmf)();       // asserts g, h
}

```

— end example]

- 6 A type T_{call} is *call-compatible* with a function type T_{func} if T_{call} is the same type as T_{func} or if the type “pointer to T_{func} ” can be converted to type “pointer to T_{call} ” via a function pointer conversion (7.3.14). Calling a function through an expression whose function type is not call-compatible with the type of the called function’s definition results in undefined behavior (F.3.11).

[Note 4: This requirement allows the case when the expression has the type of a potentially-throwing function, but the called function has a non-throwing exception specification, and the function types are otherwise the same. — end note]

- 7 When a function is called, each parameter (9.3.4.6) is initialized (9.5, 11.4.5.3) with its corresponding argument, and each precondition assertion of the function call is evaluated (9.4.1). If the function is an explicit object member function and there is an implied object argument (12.2.2.2.2), the list of provided arguments is preceded by the implied object argument for the purposes of this correspondence. If there is no corresponding argument, the default argument for the parameter is used.

[Example 2:

```

template<typename ...T> int f(int n = 0, T ...t);
int x = f<int>();           // error: no argument for second function parameter

```

— end example]

If the function is an implicit object member function, the object expression of the class member access shall be a glvalue and the implicit object parameter of the function (12.2.2) is initialized with that glvalue, converted as if by an explicit type conversion (7.6.3).

[Note 5: There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See 6.5.2, 11.8.3, and 7.6.1.5. — end note]

When a function is called, the type of any parameter shall not be a class type that is either incomplete or abstract.

[Note 6: This still allows a parameter to be a pointer or reference to such a type. However, it prevents a passed-by-value parameter to have an incomplete or abstract class type. — end note]

It is implementation-defined whether a parameter is destroyed when the function in which it is defined exits (8.8.4, 14.3, 7.6.2.4) or at the end of the enclosing full-expression; parameters are always destroyed in the reverse order of their construction. The initialization and destruction of each parameter occurs within the context of the full-expression (6.10.1) where the function call appears.

[Example 3: The access (11.8.1) of the constructor, conversion functions, or destructor is checked at the point of call. If a constructor or destructor for a function parameter throws an exception, any *function-try-block* (14.1) of the called function with a handler that can handle the exception is not considered. — end example]

- 8 The *postfix-expression* is sequenced before each *initializer-clause* in the *expression-list* and any default argument. The initialization of a parameter or, if the implementation introduces any temporary objects to hold the values of function parameters (6.8.7), the initialization of those temporaries, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other parameter. These evaluations are sequenced before the evaluation of the precondition assertions of the function call, which are evaluated in sequence (9.4.1). For any temporaries introduced to hold the values of function parameters, the initialization of the parameter objects from those temporaries is indeterminately sequenced with respect to the evaluation of each precondition assertion.

[Note 7: All side effects of argument evaluations are sequenced before the function is entered (see 6.10.1). — end note]

[Example 4:

```

void f() {
    std::string s = "but I have heard it works even if you don't believe in it";
    s.replace(0, 4, "").replace(s.find("even"), 4, "only").replace(s.find(" don't"), 6, "");
}

```

```
    assert(s == "I have heard it works only if you believe in it");    // OK
}
```

— end example]

[Note 8: If an operator function is invoked using operator notation, argument evaluation is sequenced as specified for the built-in operator; see 12.2.2.3. — end note]

[Example 5:

```
struct S {
    S(int);
};
int operator<<(S, int);
int i, j;
int x = S(i=1) << (i=2);
int y = operator<<(S(j=1), j=2);
```

After performing the initializations, the value of *i* is 2 (see 7.6.7), but it is unspecified whether the value of *j* is 1 or 2. — end example]

- 9 The result of a function call is the result of the possibly-converted operand of the **return** statement (8.8.4) that transferred control out of the called function (if any), except in a virtual function call if the return type of the final overrider is different from the return type of the statically chosen function, the value returned from the final overrider is converted to the return type of the statically chosen function.
- 10 When the called function exits normally (8.8.4, 7.6.2.4), all postcondition assertions of the function call are evaluated in sequence (9.4.1). If the implementation introduces any temporary objects to hold the result value as specified in 6.8.7, the evaluation of each postcondition assertion is indeterminately sequenced with respect to the initialization of any of those temporaries or the result object. These evaluations, in turn, are sequenced before the destruction of any function parameters.
- 11 [Note 9: A function can change the values of its non-const parameters, but these changes cannot affect the values of the arguments except where a parameter is of a reference type (9.3.4.3); if the reference is to a const-qualified type, **const_cast** needs to be used to cast away the constness in order to modify the argument's value. Where a parameter is of **const** reference type a temporary object is introduced if needed (9.2.9, 5.13, 5.13.5, 9.3.4.5, 6.8.7). In addition, it is possible to modify the values of non-constant objects through pointer parameters. — end note]
- 12 A function can be declared to accept fewer arguments (by declaring default arguments (9.3.4.7)) or more arguments (by using the ellipsis, ..., or a function parameter pack (9.3.4.6)) than the number of parameters in the function definition (9.6).
[Note 10: This implies that, except where the ellipsis (...) or a function parameter pack is used, a parameter is available for each argument. — end note]
- 13 When there is no parameter for a given argument, the argument is passed in such a way that the receiving function can obtain the value of the argument by invoking **va_arg** (17.14).

[Note 11: This paragraph does not apply to arguments passed to a function parameter pack. Function parameter packs are expanded during template instantiation (13.7.4), thus each such argument has a corresponding parameter when a function template specialization is actually called. — end note]

The lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions are performed on the argument expression. An argument that has type *cv std::nullptr_t* is converted to type **void*** (7.3.12). After these conversions, if the argument does not have arithmetic, enumeration, pointer, pointer-to-member, or class type, the program is ill-formed. Passing a potentially evaluated argument of a scoped enumeration type (9.8.1) or of a class type (Clause 11) having an eligible non-trivial copy constructor (11.4.4, 11.4.5.3), an eligible non-trivial move constructor, or a non-trivial destructor (11.4.7), with no corresponding parameter, is conditionally-supported with implementation-defined semantics. If the argument has integral or enumeration type that is subject to the integral promotions (7.3.7), or a floating-point type that is subject to the floating-point promotion (7.3.8), the value of the argument is converted to the promoted type before the call. These promotions are referred to as the *default argument promotions*.

- 14 Recursive calls are permitted, except to the **main** function (6.10.3.1).
- 15 A function call is an lvalue if the result type is an lvalue reference type or an rvalue reference to function type, an xvalue if the result type is an rvalue reference to object type, and a prvalue otherwise. If it is a non-void prvalue, the type of the function call expression shall be complete, except as specified in 9.2.9.6.

7.6.1.4 Explicit type conversion (functional notation)

[expr.type.conv]

- ¹ A *simple-type-specifier* (9.2.9.3) or *typename-specifier* (13.8) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.2.2.9) for the remainder of this subclause. Otherwise, if the type contains a placeholder type, it is replaced by the type determined by placeholder type deduction (9.2.9.7.2). Let T denote the resulting type. Then:
- (1.1) — If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (7.6.3).
 - (1.2) — Otherwise, if T is *cv void*, the initializer shall be `()` or `{}` (after pack expansion, if any), and the expression is a prvalue of type *void* that performs no initialization.
 - (1.3) — Otherwise, if T is a reference type, the expression has the same effect as direct-initializing an invented variable *t* of type T from the initializer and then using *t* as the result of the expression; the result is an lvalue if T is an lvalue reference type or an rvalue reference to function type and an xvalue otherwise.
 - (1.4) — Otherwise, the expression is a prvalue of type T whose result object is direct-initialized (9.5) with the initializer.

If the initializer is a parenthesized optional *expression-list*, T shall not be an array type.

[Example 1:

```

struct A {};
void f(A&);           // #1
void f(A&&);           // #2
A& g();
void h() {
    f(g());           // calls #1
    f(A(g()));        // calls #2 with a temporary object
    f(auto(g()));     // calls #2 with a temporary object
}

```

— end example]

7.6.1.5 Class member access

[expr.ref]

- ¹ A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword `template`, and then followed by an *id-expression* or a *splice-expression*, is a postfix expression.

[Note 1: If the keyword `template` is used and followed by an *id-expression*, the unqualified name is considered to refer to a template (13.3). If a *simple-template-id* results and is followed by a `::`, the *id-expression* is a *qualified-id*. — end note]

- ² For a dot that is followed by an expression that designates a static member or an enumerator, the first expression is a discarded-value expression (7.2.3); if the expression after the dot designates a non-static data member (11.4.1) or a direct base class relationship (11.7.1), the first expression shall be a glvalue. A postfix expression that is followed by an arrow shall be a prvalue having pointer type. The expression *E1*->*E2* is converted to the equivalent form `(*(E1)).E2`; the remainder of 7.6.1.5 will address only the form using a dot.⁴³
- ³ The postfix expression before the dot is evaluated;⁴⁴ the result of that evaluation, together with the *id-expression* or *splice-expression*, determines the result of the entire postfix expression.
- ⁴ Abbreviating *postfix-expression*.*id-expression* or *postfix-expression*.*splice-expression* as *E1*.*E2*, *E1* is called the *object expression*. If the object expression is of scalar type, *E2* shall name the pseudo-destructor of that same type (ignoring cv-qualifications) and *E1*.*E2* is a prvalue of type “function of () returning *void*”.

[Note 2: This value can only be used for a notional function call (7.5.5.5). — end note]

- ⁵ Otherwise, the object expression shall be of class type. The class type shall be complete unless the class member access appears in the definition of that class.

[Note 3: The program is ill-formed if the result differs from that when the class is complete (6.5.2). — end note]

[Note 4: 6.5.5 describes how names are looked up after the `.` and `->` operators. — end note]

⁴³ Note that `(*(E1))` is an lvalue.

⁴⁴ If the class member access expression is evaluated, the subexpression evaluation happens even if the result is unnecessary to determine the value of the entire postfix expression, for example if the *id-expression* denotes a static member.

- ⁶ If *E2* is a *splice-expression*, then let *T1* be the type of *E1*. *E2* shall designate either a member of *T1* or a direct base class relationship (*T1*, *B*).
- ⁷ If *E2* designates a bit-field, *E1.E2* is a bit-field. The type and value category of *E1.E2* are determined as follows. In the remainder of 7.6.1.5, *cq* represents either **const** or the absence of **const** and *vq* represents either **volatile** or the absence of **volatile**. *cv* represents an arbitrary set of cv-qualifiers, as defined in 6.9.6.
- ⁸ If *E2* designates an entity that is declared to have type “reference to *T*”, then *E1.E2* is an lvalue of type *T*. In that case, if *E2* designates a static data member, *E1.E2* designates the object or function to which the reference is bound, otherwise *E1.E2* designates the object or function to which the corresponding reference member of *E1* is bound. Otherwise, one of the following rules applies.
- (8.1) — If *E2* designates a static data member and the type of *E2* is *T*, then *E1.E2* is an lvalue; the expression designates the named member of the class. The type of *E1.E2* is *T*.
 - (8.2) — Otherwise, if *E2* designates a non-static data member and the type of *E1* is “*cq1 vq1 X*”, and the type of *E2* is “*cq2 vq2 T*”, the expression designates the corresponding member subobject of the object designated by *E1*. If *E1* is an lvalue, then *E1.E2* is an lvalue; otherwise *E1.E2* is an xvalue. Let the notation *vq12* stand for the “union” of *vq1* and *vq2*; that is, if *vq1* or *vq2* is **volatile**, then *vq12* is **volatile**. Similarly, let the notation *cq12* stand for the “union” of *cq1* and *cq2*; that is, if *cq1* or *cq2* is **const**, then *cq12* is **const**. If the entity designated by *E2* is declared to be a **mutable** member, then the type of *E1.E2* is “*vq12 T*”. If the entity designated by *E2* is not declared to be a **mutable** member, then the type of *E1.E2* is “*cq12 vq12 T*”.
 - (8.3) — Otherwise, if *E2* denotes an overload set, the expression shall be the (possibly-parenthesized) left-hand operand of a member function call (7.6.1.3), and function overload resolution (12.2) is used to select the function to which *E2* refers. The type of *E1.E2* is the type of *E2* and *E1.E2* refers to the function referred to by *E2*.
 - (8.3.1) — If *E2* refers to a static member function, *E1.E2* is an lvalue.
 - (8.3.2) — Otherwise (when *E2* refers to a non-static member function), *E1.E2* is a prvalue.
[Note 5: Any redundant set of parentheses surrounding the expression is ignored (7.5.4). — end note]
 - (8.4) — Otherwise, if *E2* designates a nested type, the expression *E1.E2* is ill-formed.
 - (8.5) — Otherwise, if *E2* designates a member enumerator and the type of *E2* is *T*, the expression *E1.E2* is a prvalue of type *T* whose value is the value of the enumerator.
 - (8.6) — Otherwise, if *E2* designates a direct base class relationship (*D*, *B*) and *D* is either the cv-unqualified class type of *E1* or a base class thereof, let *cv* be the cv-qualification of the type of *E1*. *E1* is implicitly converted to the type “reference to *cv D*” (where the reference is an lvalue reference if *E1* is an lvalue and an rvalue reference otherwise) and the expression designates the direct base class subobject of type *B* of the object designated by the converted *E1*. If *E1* is an lvalue, then *E1.E2* is an lvalue; otherwise, *E1.E2* is an xvalue. The type of *E1.E2* is *cv B*.

[Example 1:

```

struct B {
    int b;
};
struct C : B {
    int get() const { return b; }
};
struct D : B, C { };

constexpr int f() {
    D d = {1, {}};

    // b unambiguously refers to the direct base class of type B,
    // not the indirect base class of type B
    B& b = d.[ : std::meta::bases_of(^D, std::meta::access_context::current())[0] : ];
    b.b += 10;
    return 10 * b.b + d.get();
}
static_assert(f() == 110);
— end example]
```

(8.7) — Otherwise, the program is ill-formed.

- ⁹ If E2 designates a non-static member (possibly after overload resolution), the program is ill-formed if the class of which E2 designates a direct member is an ambiguous base (6.5.2) of the designating class (11.8.3) of E2.

[Note 6: The program is also ill-formed if the naming class is an ambiguous base of the class type of the object expression; see 11.8.3. — end note]

- ¹⁰ If E2 designates a non-static member (possibly after overload resolution) or direct base class relationship and the result of E1 is an object whose type is not similar (7.3.6) to the type of E1, the behavior is undefined (F.3.12).

[Example 2:

```
struct A { int i; };
struct B { int j; };
struct D : A, B {};
void f() {
    D d;
    static_cast<B&>(d).j;           // OK, object expression designates the B subobject of d
    reinterpret_cast<B&>(d).j;      // undefined behavior
}
```

— end example]

7.6.1.6 Increment and decrement

[**expr.post.incr**]

- ¹ The value of a postfix ++ expression is the value obtained by applying the lvalue-to-rvalue conversion (7.3.2) to its operand.

[Note 1: The value obtained is a copy of the original value. — end note]

The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than *cv* bool, or a pointer to a complete object type. An operand with volatile-qualified type is deprecated; see D.4. The value of the operand object is modified (3.1) as if it were the operand of the prefix ++ operator (7.6.2.3). The value computation of the ++ expression is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of postfix ++ is a single evaluation.

[Note 2: Therefore, a function call cannot intervene between the lvalue-to-rvalue conversion and the side effect associated with any single postfix ++ operator. — end note]

The result is a prvalue. The type of the result is the cv-unqualified version of the type of the operand.

- ² The operand of postfix -- is decremented analogously to the postfix ++ operator.

[Note 3: For prefix increment and decrement, see 7.6.2.3. — end note]

7.6.1.7 Dynamic cast

[**expr.dynamic.cast**]

- ¹ The result of the expression `dynamic_cast<T>(v)` is the result of converting the expression `v` to type `T`. `T` shall be a pointer or reference to a complete class type, or “pointer to *cv* void”. The `dynamic_cast` operator shall not cast away constness (7.6.1.11).
- ² If `T` is a pointer type, `v` shall be a prvalue of a pointer to complete class type, and the result is a prvalue of type `T`. If `T` is an lvalue reference type, `v` shall be an lvalue of a complete class type, and the result is an lvalue of the type referred to by `T`. If `T` is an rvalue reference type, `v` shall be a glvalue having a complete class type, and the result is an xvalue of the type referred to by `T`.
- ³ If the type of `v` is the same as `T` (ignoring cv-qualifications), the result is `v` (converted if necessary).
- ⁴ If `T` is “pointer to *cv1* B” and `v` has type “pointer to *cv2* D” such that B is a base class of D, the result is a pointer to the unique B subobject of the D object pointed to by `v`, or a null pointer value if `v` is a null pointer value. Similarly, if `T` is “reference to *cv1* B” and `v` has type *cv2* D such that B is a base class of D, the result is the unique B subobject of the D object referred to by `v`.⁴⁵ In both the pointer and reference cases, the program is ill-formed if B is an inaccessible or ambiguous base class of D.

[Example 1:

```
struct B { };
struct D : B { };
```

⁴⁵ The most derived object (6.8.2) pointed or referred to by `v` can contain other B objects as base classes, but these are ignored.


```
void foo(D* dp) {
    B* bp = dynamic_cast<B*>(dp);    // equivalent to B* bp = dp;
}
```

— end example]

- 5 Otherwise, *v* shall be a pointer to or a glvalue of a polymorphic type (11.7.3).
- 6 If *v* is a null pointer value, the result is a null pointer value.
- 7 If *v* has type “pointer to *cv* *U*” and *v* does not point to an object whose type is similar (7.3.6) to *U* and that is within its lifetime or within its period of construction or destruction (11.9.5), the behavior is undefined (F.3.13). If *v* is a glvalue of type *U* and *v* does not refer to an object whose type is similar to *U* and that is within its lifetime or within its period of construction or destruction, the behavior is undefined (F.3.14).
- 8 If *T* is “pointer to *cv* void”, then the result is a pointer to the most derived object pointed to by *v*. Otherwise, a runtime check is applied to see if the object pointed or referred to by *v* can be converted to the type pointed or referred to by *T*.
- 9 Let *C* be the class type to which *T* points or refers. The runtime check logically executes as follows:
 - (9.1) — If, in the most derived object pointed (referred) to by *v*, *v* points (refers) to a public base class subobject of a *C* object, and if only one object of type *C* is derived from the subobject pointed (referred) to by *v*, the result points (refers) to that *C* object.
 - (9.2) — Otherwise, if *v* points (refers) to a public base class subobject of the most derived object, and the type of the most derived object has a base class, of type *C*, that is unambiguous and public, the result points (refers) to the *C* subobject of the most derived object.
 - (9.3) — Otherwise, the runtime check *fails*.
- 10 The value of a failed cast to pointer type is the null pointer value of the required result type. A failed cast to reference type throws an exception (14.2) of a type that would match a handler (14.4) of type `std::bad_cast` (17.7.4).

[Example 2:

```
class A { virtual void f(); };
class B { virtual void g(); };
class D : public virtual A, private B { };
void g() {
    D d;
    B* bp = (B*)&d;                // cast needed to break protection
    A* ap = &d;                    // public derivation, no cast needed
    D& dr = dynamic_cast<D&>(*bp); // fails
    ap = dynamic_cast<A*>(bp);      // fails
    bp = dynamic_cast<B*>(ap);      // fails
    ap = dynamic_cast<A*>(&d);       // succeeds
    bp = dynamic_cast<B*>(&d);       // ill-formed (not a runtime check)
}

class E : public D, public B { };
class F : public E, public D { };
void h() {
    F f;
    A* ap = &f;                    // succeeds: finds unique A
    D* dp = dynamic_cast<D*>(ap);   // fails: yields null; f has two D subobjects
    E* ep = (E*)ap;                // error: cast from virtual base
    E* ep1 = dynamic_cast<E*>(ap);  // succeeds
}
```

— end example]

[Note 1: Subclause 11.9.5 describes the behavior of a `dynamic_cast` applied to an object under construction or destruction. — end note]

7.6.1.8 Type identification

[**expr.typeid**]

- 1 The result of a `typeid` expression is an lvalue of static type `const std::type_info` (17.7.3) and dynamic type `const std::type_info` or `const name` where *name* is an implementation-defined class publicly derived from

`std::type_info` which preserves the behavior described in 17.7.3.⁴⁶ The lifetime of the object referred to by the lvalue extends to the end of the program. Whether or not the destructor is called for the `std::type_info` object at the end of the program is unspecified.

- ² If the type of the *expression* or *type-id* operand is a (possibly cv-qualified) class type or a reference to (possibly cv-qualified) class type, that class shall be completely defined.
- ³ If an *expression* operand of `typeid` is a possibly-parenthesized *unary-expression* whose *unary-operator* is `*` and whose operand evaluates to a null pointer value (6.9.5), the `typeid` expression throws an exception (14.2) of a type that would match a handler of type `std::bad_typeid` (17.7.5).

[Note 1: In other contexts, evaluating such a *unary-expression* results in undefined behavior (7.6.2.2). — end note]

- ⁴ When `typeid` is applied to a glvalue whose type is a polymorphic class type (11.7.3), the result refers to a `std::type_info` object representing the type of the most derived object (6.8.2) (that is, the dynamic type) to which the glvalue refers.
- ⁵ When `typeid` is applied to an expression other than a glvalue of a polymorphic class type, the result refers to a `std::type_info` object representing the static type of the expression. Lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) conversions are not applied to the expression. If the expression is a prvalue, the temporary materialization conversion (7.3.5) is applied. The expression is an unevaluated operand (7.2.3).
- ⁶ When `typeid` is applied to a *type-id*, the result refers to a `std::type_info` object representing the type of the *type-id*. If the type of the *type-id* is a reference to a possibly cv-qualified type, the result of the `typeid` expression refers to a `std::type_info` object representing the cv-unqualified referenced type.

[Note 2: The *type-id* cannot denote a function type with a *cv-qualifier-seq* or a *ref-qualifier* (9.3.4.6). — end note]

- ⁷ If the type of the expression or *type-id* is a cv-qualified type, the result of the `typeid` expression refers to a `std::type_info` object representing the cv-unqualified type.

[Example 1:

```
class D { /* ... */ };
D d1;
const D d2;

typeid(d1) == typeid(d2);      // yields true
typeid(D)  == typeid(const D); // yields true
typeid(D)  == typeid(d2);     // yields true
typeid(D)  == typeid(const D&); // yields true
```

— end example]

- ⁸ The type `std::type_info` (17.7.3) is not predefined; if a standard library declaration (17.7.2, 16.4.2.4) of `std::type_info` does not precede (6.5.1) a `typeid` expression, the program is ill-formed.
- ⁹ [Note 3: Subclause 11.9.5 describes the behavior of `typeid` applied to an object under construction or destruction. — end note]

7.6.1.9 Static cast

[`expr.static.cast`]

- ¹ The result of the expression `static_cast<T>(v)` is the result of converting the expression `v` to type `T`. If `T` is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if `T` is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue.
- ² An lvalue of type “*cv1* B”, where B is a class type, can be cast to type “reference to *cv2* D”, where D is a complete class derived (11.7) from B, if *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*. If B is a virtual base class of D or a base class of a virtual base class of D, or if no valid standard conversion from “pointer to D” to “pointer to B” exists (7.3.12), the program is ill-formed. An xvalue of type “*cv1* B” can be cast to type “rvalue reference to *cv2* D” with the same constraints as for an lvalue of type “*cv1* B”. If the object of type “*cv1* B” is actually a base class subobject of an object of type D, the result refers to the enclosing object of type D. Otherwise, the behavior is undefined (F.3.15).

[Example 1:

```
struct B { };
struct D : public B { };
D d;
```

⁴⁶) The recommended name for such a class is `extended_type_info`.

```
B &br = d;
```

```
static_cast<D&>(br);           // produces lvalue denoting the original d object
```

— end example]

- 3 An lvalue of type T1 can be cast to type “rvalue reference to T2” if T2 is reference-compatible with T1 (9.5.4). If the value is not a bit-field, the result refers to the object or the specified base class subobject thereof; otherwise, the lvalue-to-rvalue conversion (7.3.2) is applied to the bit-field and the resulting prvalue is used as the operand of the `static_cast` for the remainder of this subclause. If T2 is an inaccessible (11.8) or ambiguous (6.5.2) base class of T1, a program that necessitates such a cast is ill-formed.
- 4 Any expression can be explicitly converted to type `cv void`, in which case the operand is a discarded-value expression (7.2).

[Note 1: Such a `static_cast` has no result as it is a prvalue of type `void`; see 7.2.1. — end note]

[Note 2: However, if the value is in a temporary object (6.8.7), the destructor for that object is not executed until the usual time, and the value of the object is preserved for the purpose of executing the destructor. — end note]

- 5 Otherwise, an expression *E* can be explicitly converted to a type T if there is an implicit conversion sequence (12.2.4.2) from *E* to T, if overload resolution for a direct-initialization (9.5) of an object or reference of type T from *E* would find at least one viable function (12.2.3), or if T is an aggregate type (9.5.2) having a first element *x* and there is an implicit conversion sequence from *E* to the type of *x*. If T is a reference type, the effect is the same as performing the declaration and initialization

```
T t(E);
```

for some invented temporary variable *t* (9.5) and then using the temporary variable as the result of the conversion. Otherwise, the result object is direct-initialized from *E*.

[Note 3: The conversion is ill-formed when attempting to convert an expression of class type to an inaccessible or ambiguous base class. — end note]

[Note 4: If T is “array of unknown bound of U”, this direct-initialization defines the type of the expression as U[1]. — end note]

- 6 Otherwise, the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) conversions are applied to the operand, and the conversions that can be performed using `static_cast` are listed below. No other conversion can be performed using `static_cast`.
- 7 A value of a scoped enumeration type (9.8.1) can be explicitly converted to an integral type; the result is the same as that of converting to the enumeration’s underlying type and then to the destination type. A value of a scoped enumeration type can also be explicitly converted to a floating-point type; the result is the same as that of converting from the original value to the floating-point type.
- 8 A value of integral or enumeration type can be explicitly converted to a complete enumeration type. If the enumeration type has a fixed underlying type, the value is first converted to that type by integral promotion (7.3.7) or integral conversion (7.3.9), if necessary, and then to the enumeration type. If the enumeration type does not have a fixed underlying type, the value is unchanged if the original value is within the range of the enumeration values (9.8.1), and otherwise, the behavior is undefined (F.3.16). A value of floating-point type can also be explicitly converted to an enumeration type. The resulting value is the same as converting the original value to the underlying type of the enumeration (7.3.11), and subsequently to the enumeration type.
- 9 A prvalue of floating-point type can be explicitly converted to any other floating-point type. If the source value can be exactly represented in the destination type, the result of the conversion has that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined (F.3.17).
- 10 A prvalue of type “pointer to *cv1* B”, where B is a class type, can be converted to a prvalue of type “pointer to *cv2* D”, where D is a complete class derived (11.7) from B, if *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*. If B is a virtual base class of D or a base class of a virtual base class of D, or if no valid standard conversion from “pointer to D” to “pointer to B” exists (7.3.12), the program is ill-formed. The null pointer value (6.9.5) is converted to the null pointer value of the destination type. If the prvalue of type “pointer to *cv1* B” points to a B that is actually a base class subobject of an object of type D, the resulting pointer points to the enclosing object of type D. Otherwise, the behavior is undefined (F.3.18).

- ¹¹ A prvalue of type “pointer to member of D of type *cv1* T” can be converted to a prvalue of type “pointer to member of B of type *cv2* T”, where D is a complete class type and B is a base class (11.7) of D, if *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*.

[Note 5: Function types (including those used in pointer-to-member-function types) are never cv-qualified (9.3.4.6). — end note]

If no valid standard conversion from “pointer to member of B of type T” to “pointer to member of D of type T” exists (7.3.13), the program is ill-formed. The null member pointer value (7.3.13) is converted to the null member pointer value of the destination type. If class B contains the original member, or is a base class of the class containing the original member, the resulting pointer to member points to the original member. Otherwise, the behavior is undefined (F.3.19).

[Note 6: Although class B need not contain the original member, the dynamic type of the object with which indirection through the pointer to member is performed must contain the original member; see 7.6.4. — end note]

- ¹² A prvalue of type “pointer to *cv1* void” can be converted to a prvalue of type “pointer to *cv2* T”, where T is an object type and *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*. If the original pointer value represents the address A of a byte in memory and A does not satisfy the alignment requirement of T, then the resulting pointer value (6.9.5) is unspecified. Otherwise, if the original pointer value points to an object *a*, and there is an object *b* of type similar to T that is pointer-interconvertible (6.9.5) with *a*, the result is a pointer to *b*. Otherwise, the pointer value is unchanged by the conversion.

[Example 2:

```
T* p1 = new T;
const T* p2 = static_cast<const T*>(static_cast<void*>(p1));
bool b = p1 == p2; // b will have the value true.
```

— end example]

7.6.1.10 Reinterpret cast

[**expr.reinterpret.cast**]

- ¹ The result of the expression **reinterpret_cast**<T>(v) is the result of converting the expression v to type T. If T is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions are performed on the expression v. Conversions that can be performed explicitly using **reinterpret_cast** are listed below. No other conversion can be performed explicitly using **reinterpret_cast**.
- ² The **reinterpret_cast** operator shall not cast away constness (7.6.1.11). An expression of integral, enumeration, pointer, or pointer-to-member type can be explicitly converted to its own type; such a cast yields the value of its operand.
- ³ [Note 1: The mapping performed by **reinterpret_cast** might, or might not, produce a representation different from the original value. — end note]
- ⁴ A pointer can be explicitly converted to any integral type large enough to distinguish all value representations of its type. The mapping function is implementation-defined.

[Note 2: It is intended to be unsurprising to those who know the addressing structure of the underlying machine. — end note]

A value of type **std::nullptr_t** can be converted to an integral type; the conversion has the same meaning and validity as a conversion of (void*)0 to the integral type.

[Note 3: A **reinterpret_cast** cannot be used to convert a value of any type to the type **std::nullptr_t**. — end note]

- ⁵ A value of integral type or enumeration type can be explicitly converted to a pointer. If the value is one that can be produced by converting one or more pointer values (6.9.5) to an integral type, the result is an unspecified choice among all such values that would result in the program having defined behavior. If no such value exists, the behavior is undefined.

[Note 4: It is possible for the result to be an invalid pointer value or to not be valid in the context of the conversion (6.9.5) because it points to an object in a region of storage whose duration has ended or has not yet begun. — end note]

Otherwise, the result is implementation-defined.

[Note 5: It can be an invalid pointer value. — end note]

- ⁶ A function pointer can be explicitly converted to a function pointer of a different type. The function pointer value is unchanged by the conversion.

[Note 6: The effect of calling a function through a pointer to a function type (9.3.4.6) that is not call-compatible with the type used in the definition of the function is undefined (7.6.1.3). — end note]

- 7 An object pointer can be explicitly converted to an object pointer of a different type.⁴⁷ When a prvalue v of object pointer type is converted to the object pointer type “pointer to cv T ”, the result is `static_cast<cv T*>(static_cast<cv void*>(v))`.

[Note 7: Converting a pointer of type “pointer to T_1 ” that points to an object of type T_1 to the type “pointer to T_2 ” (where T_2 is an object type and the alignment requirements of T_2 are no stricter than those of T_1) and back to its original type yields the original pointer value. — end note]

- 8 Converting a function pointer to an object pointer type or vice versa is conditionally-supported. The meaning of such a conversion is implementation-defined, except that if an implementation supports conversions in both directions, converting a prvalue of one type to the other type and back, possibly with different cv -qualification, shall yield the original pointer value.

- 9 The null pointer value (6.9.5) is converted to the null pointer value of the destination type.

[Note 8: A null pointer constant of type `std::nullptr_t` cannot be converted to a pointer type, and a null pointer constant of integral type is not necessarily converted to a null pointer value. — end note]

- 10 A prvalue of type “pointer to member of X of type T_1 ” can be explicitly converted to a prvalue of a different type “pointer to member of Y of type T_2 ” if T_1 and T_2 are both function types or both object types.⁴⁸ The null member pointer value (7.3.13) is converted to the null member pointer value of the destination type. The result of this conversion is unspecified, except in the following cases:

- (10.1) — Converting a prvalue of type “pointer to member function” to a different pointer-to-member-function type and back to its original type yields the original pointer-to-member value.
- (10.2) — Converting a prvalue of type “pointer to data member of X of type T_1 ” to the type “pointer to data member of Y of type T_2 ” (where the alignment requirements of T_2 are no stricter than those of T_1) and back to its original type yields the original pointer-to-member value.

- 11 If v is a glvalue of type T_1 , designating an object or function x , it can be cast to the type “reference to T_2 ” if an expression of type “pointer to T_1 ” can be explicitly converted to the type “pointer to T_2 ” using a `reinterpret_cast`. The result is that of `*reinterpret_cast<T2*>(p)` where p is a pointer to x of type “pointer to T_1 ”.

[Note 9: No temporary is materialized (7.3.5) or created, no copy is made, and no constructors (11.4.5) or conversion functions (11.4.8) are called.⁴⁹ — end note]

7.6.1.11 Const cast

[`expr.const.cast`]

- 1 The result of the expression `const_cast<T>(v)` is of type T . If T is an lvalue reference to object type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions are performed on the expression v . The temporary materialization conversion (7.3.5) is not performed on v , other than as specified below. Conversions that can be performed explicitly using `const_cast` are listed below. No other conversion shall be performed explicitly using `const_cast`.
- 2 [Note 1: Subject to the restrictions in this subclause, an expression can be cast to its own type using a `const_cast` operator. — end note]
- 3 For two similar object pointer or pointer to data member types T_1 and T_2 (7.3.6), a prvalue of type T_1 can be explicitly converted to the type T_2 using a `const_cast` if, considering the qualification-decompositions of both types, each P_i^1 is the same as P_i^2 for all i . If v is a null pointer or null member pointer, the result is a null pointer or null member pointer, respectively. Otherwise, the result points to or past the end of the same object, or points to the same member, respectively, as v .
- 4 For two object types T_1 and T_2 , if a pointer to T_1 can be explicitly converted to the type “pointer to T_2 ” using a `const_cast`, then the following conversions can also be made:
- (4.1) — an lvalue of type T_1 can be explicitly converted to an lvalue of type T_2 using the cast `const_cast<T2&>`;

⁴⁷) The types can have different cv -qualifiers, subject to the overall restriction that a `reinterpret_cast` cannot cast away constness.

⁴⁸) T_1 and T_2 can have different cv -qualifiers, subject to the overall restriction that a `reinterpret_cast` cannot cast away constness.

⁴⁹) This is sometimes referred to as a type pun when the result refers to the same object as the source glvalue.

- (4.2) — a glvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast `const_cast<T2&&>`; and
- (4.3) — if T1 is a class or array type, a prvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast `const_cast<T2&&>`. The temporary materialization conversion is performed on v.

The result refers to the same object as the (possibly converted) operand.

[Example 1:

```
typedef int *A[3];           // array of 3 pointer to int
typedef const int *const CA[3]; // array of 3 const pointer to const int

auto &&r2 = const_cast<A&&>(CA{}); // OK, temporary materialization conversion is performed
```

— end example]

- 5 [Note 2: Depending on the type of the object, a write operation through the pointer, lvalue or pointer to data member resulting from a `const_cast` that casts away a const-qualifier⁵⁰ can produce undefined behavior (9.2.9.2). — end note]
- 6 A conversion from a type T1 to a type T2 *casts away constness* if T1 and T2 are different, there is a qualification-decomposition (7.3.6) of T1 yielding n such that T2 has a qualification-decomposition of the form
- $$cv_0^2 P_0^2 cv_1^2 P_1^2 \cdots cv_{n-1}^2 P_{n-1}^2 cv_n^2 U_2,$$
- and there is no qualification conversion that converts T1 to
- $$cv_0^2 P_0^1 cv_1^2 P_1^1 \cdots cv_{n-1}^2 P_{n-1}^1 cv_n^2 U_1.$$
- 7 Casting from an lvalue of type T1 to an lvalue of type T2 using an lvalue reference cast or casting from an expression of type T1 to an xvalue of type T2 using an rvalue reference cast casts away constness if a cast from a prvalue of type “pointer to T1” to the type “pointer to T2” casts away constness.
- 8 [Note 3: Some conversions which involve only changes in cv-qualification cannot be done using `const_cast`. For instance, conversions between pointers to functions are not covered because such conversions lead to values whose use causes undefined behavior. For the same reasons, conversions between pointers to member functions, and in particular, the conversion from a pointer to a const member function to a pointer to a non-const member function, are not covered. — end note]

7.6.2 Unary expressions

[expr.unary]

7.6.2.1 General

[expr.unary.general]

- 1 Expressions with unary operators group right-to-left.

unary-expression:

```
postfix-expression
unary-operator cast-expression
++ cast-expression
-- cast-expression
await-expression
sizeof unary-expression
sizeof ( nofun-type-id )
sizeof ... ( identifier )
alignof ( nofun-type-id )
noexcept-expression
new-expression
delete-expression
reflect-expression
```

unary-operator: one of

* & + - ! ~

7.6.2.2 Unary operators

[expr.unary.op]

- 1 The unary * operator performs *indirection*. Its operand shall be a prvalue of type “pointer to T”, where T is an object or function type. The operator yields an lvalue of type T. If the operand points to an object or function, the result denotes that object or function; otherwise, the behavior is undefined except as specified in 7.6.1.8 (F.3.20).

⁵⁰) `const_cast` is not limited to conversions that cast away a const-qualifier.

[Note 1: Indirection through a pointer to an out-of-lifetime object is valid (6.8.4). — end note]

[Note 2: Indirection through a pointer to an incomplete type (other than *cv void*) is valid. The lvalue thus obtained can be used in limited ways (to initialize a reference, for example); this lvalue must not be converted to a prvalue, see 7.3.2. — end note]

2 Each of the following unary operators yields a prvalue.

3 The operand of the unary `&` operator shall be an lvalue of some type *T*.

- (3.1) — If the operand is a *qualified-id* or *splice-expression* designating a non-static member *m*, other than an explicit object member function, *m* shall be a direct member of some class *C* that is not an anonymous union. The result has type “pointer to member of class *C* of type *T*” and designates *C::m*.

[Note 3: A *qualified-id* that names a member of a namespace-scope anonymous union is considered to be a class member access expression (7.5.5.1) and cannot be used to form a pointer to member. — end note]

- (3.2) — Otherwise, the result has type “pointer to *T*” and points to the designated object (6.8.1) or function (6.9.5). If the operand designates an explicit object member function (9.3.4.6), the operand shall be a *qualified-id* or a *splice-expression*.

[Note 4: In particular, taking the address of a variable of type “*cv T*” yields a pointer of type “pointer to *cv T*”. — end note]

[Example 1:

```
struct A { int i; };
struct B : A { };
... &B::i ...      // has type int A::*
int a;
int* p1 = &a;
int* p2 = p1 + 1;    // defined behavior
bool b = p2 > p1;    // defined behavior, with value true
```

— end example]

[Note 5: A pointer to member formed from a **mutable** non-static data member (9.2.2) does not reflect the **mutable** specifier associated with the non-static data member. — end note]

- 4 A pointer to member is only formed when an explicit `&` is used and its operand is a *qualified-id* or *splice-expression* not enclosed in parentheses.

[Note 6: That is, the expression `&(qualified-id)`, where the *qualified-id* is enclosed in parentheses, does not form an expression of type “pointer to member”. Neither does *qualified-id*, because there is no implicit conversion from a *qualified-id* for a non-static member function to the type “pointer to member function” as there is from an lvalue of function type to the type “pointer to function” (7.3.4). Nor is *&unqualified-id* a pointer to member, even within the scope of the *unqualified-id*’s class. — end note]

- 5 If `&` is applied to an lvalue of incomplete class type and the complete type declares `operator&()`, it is unspecified whether the operator has the built-in meaning or the operator function is called. The operand of `&` shall not be a bit-field.

- 6 [Note 7: The address of an overload set (Clause 12) can be taken only in a context that uniquely determines which function is referred to (see 12.3). Since the context can affect whether the operand is a static or non-static member function, the context can also affect whether the expression has type “pointer to function” or “pointer to member function”. — end note]

- 7 The operand of the unary `+` operator shall be a prvalue of arithmetic, unscoped enumeration, or pointer type and the result is the value of the argument. Integral promotion is performed on integral or enumeration operands. The type of the result is the type of the promoted operand.

- 8 The operand of the unary `-` operator shall be a prvalue of arithmetic or unscoped enumeration type and the result is the negative of its operand. Integral promotion is performed on integral or enumeration operands. The negative of an unsigned quantity is computed by subtracting its value from 2^n , where n is the number of bits in the promoted operand. The type of the result is the type of the promoted operand.

[Note 8: The result is the two’s complement of the operand (where operand and result are considered as unsigned). — end note]

- 9 The operand of the logical negation operator `!` is contextually converted to **bool** (7.3); its value is **true** if the converted operand is **false** and **false** otherwise. The type of the result is **bool**.

- 10 The operand of the `~` operator shall be a prvalue of integral or unscoped enumeration type. Integral promotions are performed. The type of the result is the type of the promoted operand. Given the coefficients

x_i of the base-2 representation (6.9.3) of the promoted operand x , the coefficient r_i of the base-2 representation of the result r is 1 if x_i is 0, and 0 otherwise.

[Note 9: The result is the ones' complement of the operand (where operand and result are considered as unsigned). — end note]

There is an ambiguity in the grammar when $\sim$ is followed by a *type-name* or *computed-type-specifier*. The ambiguity is resolved by treating $\sim$ as the operator rather than as the start of an *unqualified-id* naming a destructor.

[Note 10: Because the grammar does not permit an operator to follow the `.`, `->`, or `::` tokens, a $\sim$ followed by a *type-name* or *computed-type-specifier* in a member access expression or *qualified-id* is unambiguously parsed as a destructor name. — end note]

7.6.2.3 Increment and decrement

[expr.pre.incr]

- ¹ The operand of prefix `++` or `--` shall not be of type *cv bool*. An operand with volatile-qualified type is deprecated; see D.4. The expression `++x` is otherwise equivalent to `x+=1` and the expression `--x` is otherwise equivalent to `x-=1` (7.6.19).

[Note 1: For postfix increment and decrement, see 7.6.1.6. — end note]

7.6.2.4 Await

[expr.await]

- ¹ The `co_await` expression is used to suspend evaluation of a coroutine (9.6.4) while awaiting completion of the computation represented by the operand expression. Suspending the evaluation of a coroutine transfers control to its caller or resumer.

await-expression:
`co_await cast-expression`

- ² An *await-expression* shall appear only as a potentially evaluated expression within the *compound-statement* of a *function-body* or *lambda-expression*, in either case outside of a *handler* (14.1). In a *declaration-statement* or in the *simple-declaration* (if any) of an *init-statement*, an *await-expression* shall appear only in an *initializer* of that *declaration-statement* or *simple-declaration*. An *await-expression* shall not appear in a default argument (9.3.4.7). An *await-expression* shall not appear in the initializer of a block variable with static or thread storage duration. An *await-expression* shall not be a potentially-evaluated subexpression of the predicate of a contract assertion (6.11). A context within a function where an *await-expression* can appear is called a *suspension context* of the function.
- ³ Evaluation of an *await-expression* involves the following auxiliary types, expressions, and objects:
 - (3.1) — p is an lvalue naming the promise object (9.6.4) of the enclosing coroutine and P is the type of that object.
 - (3.2) — Unless the *await-expression* was implicitly produced by a *yield-expression* (7.6.17), an initial await expression, or a final await expression (9.6.4), a search is performed for the name `await_transform` in the scope of P (6.5.2). If this search is performed and finds at least one declaration, then a is `p.await_transform(cast-expression)`; otherwise, a is the *cast-expression*.
 - (3.3) — o is determined by enumerating the applicable `operator co_await` functions for an argument a (12.2.2.3), and choosing the best one through overload resolution (12.2). If overload resolution is ambiguous, the program is ill-formed. If no viable functions are found, o is a . Otherwise, o is a call to the selected function with the argument a . If o would be a prvalue, the temporary materialization conversion (7.3.5) is applied.
 - (3.4) — e is an lvalue referring to the result of evaluating the (possibly-converted) o .
 - (3.5) — h is an object of type `std::coroutine_handle<P>` referring to the enclosing coroutine.
 - (3.6) — *await-ready* is the expression `e.await_ready()`, contextually converted to `bool`.
 - (3.7) — *await-suspend* is the expression `e.await_suspend(h)`, which shall be a prvalue of type `void`, `bool`, or `std::coroutine_handle<Z>` for some type Z .
 - (3.8) — *await-resume* is the expression `e.await_resume()`.
- ⁴ The *await-expression* has the same type and value category as the *await-resume* expression.
- ⁵ The *await-expression* evaluates the (possibly-converted) o expression and the *await-ready* expression, then:
 - (5.1) — If the result of *await-ready* is `false`, the coroutine is considered suspended. Then:

- (5.1.1) — If the type of *await-suspend* is `std::coroutine_handle<Z>`, *await-suspend.resume()* is evaluated.
 [Note 1: This resumes the coroutine referred to by the result of *await-suspend*. Any number of coroutines can be successively resumed in this fashion, eventually returning control flow to the current coroutine caller or resumer (9.6.4). — end note]
- (5.1.2) — Otherwise, if the type of *await-suspend* is `bool`, *await-suspend* is evaluated, and the coroutine is resumed if the result is `false`.
- (5.1.3) — Otherwise, *await-suspend* is evaluated.

If the evaluation of *await-suspend* exits via an exception, the exception is caught, the coroutine is resumed, and the exception is immediately rethrown (14.2). Otherwise, control flow returns to the current coroutine caller or resumer (9.6.4) without exiting any scopes (8.8). The point in the coroutine immediately prior to control returning to its caller or resumer is a coroutine *suspend point*.

- (5.2) — If the result of *await-ready* is `true`, or when the coroutine is resumed other than by rethrowing an exception from *await-suspend*, the *await-resume* expression is evaluated, and its result is the result of the *await-expression*.

[Note 2: With respect to sequencing, an *await-expression* is indivisible (6.10.1). — end note]

⁶ [Example 1:

```
template <typename T>
struct my_future {
    /* ... */
    bool await_ready();
    void await_suspend(std::coroutine_handle<>);
    T await_resume();
};

template <class Rep, class Period>
auto operator co_await(std::chrono::duration<Rep, Period> d) {
    struct awaiter {
        std::chrono::system_clock::duration duration;
        /* ... */
        awaiter(std::chrono::system_clock::duration d) : duration(d) {}
        bool await_ready() const { return duration.count() <= 0; }
        void await_resume() {}
        void await_suspend(std::coroutine_handle<> h) { /* ... */ }
    };
    return awaiter{d};
}

using namespace std::chrono;

my_future<int> h();

my_future<void> g() {
    std::cout << "just about to go to sleep...\n";
    co_await 10ms;
    std::cout << "resumed\n";
    co_await h();
}

auto f(int x = co_await h());    // error: await-expression outside of function suspension context
int a[] = { co_await h() };    // error: await-expression outside of function suspension context
```

— end example]

7.6.2.5 Sizeof

[`expr.sizeof`]

- ¹ The `sizeof` operator yields the number of bytes occupied by a non-potentially-overlapping object of the type of its operand. The operand is either an expression, which is an unevaluated operand (7.2.3), or a parenthesized *nofun-type-id*. The `sizeof` operator shall not be applied to an expression that has function or incomplete type, to the parenthesized name of such types, or to a glvalue that designates a bit-field. The result of `sizeof` applied to any of the narrow character types is 1. The result of `sizeof` applied to any other fundamental type (6.9.3) is implementation-defined.

[Note 1: In particular, the values of `sizeof(bool)`, `sizeof(char16_t)`, `sizeof(char32_t)`, and `sizeof(wchar_t)` are implementation-defined.⁵¹ — end note]

[Note 2: See 6.8.1 for the definition of byte and 6.9.1 for the definition of object representation. — end note]

- ² When applied to a reference type, the result is the size of the referenced type. When applied to a class, the result is the number of bytes in an object of that class including any padding required for placing objects of that type in an array. The amount and placement of padding in a class type is a property of the implementation. The result of applying `sizeof` to a potentially-overlapping subobject is the size of the type, not the size of the subobject.⁵² When applied to an array, the result is the total number of bytes in the array. This implies that the size of an array of n elements is n times the size of an element.
- ³ The lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions are not applied to the operand of `sizeof`. If the operand is a prvalue, the temporary materialization conversion (7.3.5) is applied.
- ⁴ The *identifier* in a `sizeof...` expression shall name a pack. The `sizeof...` operator yields the number of elements in the pack (13.7.4). A `sizeof...` expression is a pack expansion (13.7.4).

[Example 1:

```
template<class... Types>
struct count {
    static constexpr std::size_t value = sizeof...(Types);
};
```

— end example]

- ⁵ The result of `sizeof` and `sizeof...` is a prvalue of type `std::size_t`.

[Note 3: A `sizeof` expression is an integral constant expression (7.7.3). The *typedef-name* `std::size_t` is declared in the standard header `<cstddef>` (17.2.1, 17.2.4). — end note]

7.6.2.6 Alignof

[**expr.alignof**]

- ¹ An `alignof` expression yields the alignment requirement of its operand type. The operand shall be a *nofun-type-id* representing a complete object type, or an array thereof, or a reference to one of those types.
- ² The result is a prvalue of type `std::size_t`.
[Note 1: An `alignof` expression is an integral constant expression (7.7.3). The *typedef-name* `std::size_t` is declared in the standard header `<cstddef>` (17.2.1, 17.2.4). — end note]
- ³ When `alignof` is applied to a reference type, the result is the alignment of the referenced type. When `alignof` is applied to an array type, the result is the alignment of the element type.

7.6.2.7 noexcept operator

[**expr.unary.noexcept**]

noexcept-expression:
`noexcept ( expression )`

- ¹ The operand of the `noexcept` operator is an unevaluated operand (7.2.3). If the operand is a prvalue, the temporary materialization conversion (7.3.5) is applied.
- ² The result of the `noexcept` operator is a prvalue of type `bool`. The result is `false` if the full-expression of the operand is potentially-throwing (14.5), and `true` otherwise.

[Note 1: A *noexcept-expression* is an integral constant expression (7.7.3). — end note]

7.6.2.8 New

[**expr.new**]

- ¹ The *new-expression* attempts to create an object of the *nofun-type-id* or *new-type-id* (9.3.2) to which it is applied. The type of that object is the *allocated type*. This type shall be a complete object type (6.9.1), but not an abstract class type (11.7.4) or array thereof (6.8.2).

[Note 1: Because references are not objects, references cannot be created by *new-expressions*. — end note]

[Note 2: The *nofun-type-id* can be a cv-qualified type, in which case the object created by the *new-expression* has a cv-qualified type. — end note]

⁵¹ `sizeof(bool)` is not required to be 1.

⁵² The actual size of a potentially-overlapping subobject can be less than the result of applying `sizeof` to the subobject, due to virtual base classes and less strict padding requirements on potentially-overlapping subobjects.

```

new-expression:
    ::opt new new-placementopt new-type-id new-initializeropt
    ::opt new new-placementopt ( nofun-type-id ) new-initializeropt

new-placement:
    ( expression-list )

new-type-id:
    type-specifier-seq new-declaratoropt

new-declarator:
    ptr-operator new-declaratoropt
    noptr-new-declarator

noptr-new-declarator:
    [ expressionopt ] attribute-specifier-seqopt
    noptr-new-declarator [ constant-expression ] attribute-specifier-seqopt

new-initializer:
    ( expression-listopt )
    braced-init-list

```

- ² If a placeholder type (9.2.9.7) or a placeholder for a deduced class type (9.2.9.8) appears in the *type-specifier-seq* of a *new-type-id* or *nofun-type-id* of a *new-expression*, the allocated type is deduced as follows: Let *init* be the *new-initializer*, if any, and *T* be the *new-type-id* or *nofun-type-id* of the *new-expression*, then the allocated type is the type deduced for the variable *x* in the invented declaration (9.2.9.7):

```
T x init ;
```

[Example 1:

```

new auto(1);           // allocated type is int
auto x = new auto('a'); // allocated type is char, x is of type char*

```

```

template<class T> struct A { A(T, T); };
auto y = new A{1, 2};           // allocated type is A<int>

```

— end example]

- ³ The *new-type-id* in a *new-expression* is the longest possible sequence of *new-declarators*.

[Note 3: This prevents ambiguities between the declarator operators *&*, *&&*, ***, and *[]* and their expression counterparts. — end note]

[Example 2:

```
new int * i;           // syntax error: parsed as (new int*) i, not as (new int)*i
```

The *** is the pointer declarator and not the multiplication operator. — end example]

- ⁴ [Note 4: Parentheses in a *new-type-id* of a *new-expression* can have surprising effects.

[Example 3:

```
new int(*[10])();           // error
```

is ill-formed because the binding is

```
(new int) (*[10])();           // error
```

Instead, the explicitly parenthesized version of the **new** operator can be used to create objects of compound types (6.9.5):

```
new (int (*[10]))();
```

allocates an array of 10 pointers to functions (taking no argument and returning **int**). — end example]

— end note]

- ⁵ The *attribute-specifier-seq* in a *noptr-new-declarator* appertains to the associated array type.
- ⁶ Every *constant-expression* in a *noptr-new-declarator* shall be a converted constant expression (7.7.3) of type **std::size_t** and its value shall be greater than zero.

[Example 4: Given the definition `int n = 42`, `new float[n][5]` is well-formed (because *n* is the *expression* of a *noptr-new-declarator*), but `new float[5][n]` is ill-formed (because *n* is not a constant expression). Furthermore, `new float[0]` is well-formed (because 0 is the *expression* of a *noptr-new-declarator*, where a value of zero results in the allocation of an array with no elements), but `new float[n][0]` is ill-formed (because 0 is the *constant-expression* of a *noptr-new-declarator*, where only values greater than zero are allowed). — end example]

⁷ If the *nofun-type-id* or *new-type-id* denotes an array type of unknown bound (9.3.4.5), the *new-initializer* shall not be omitted; the allocated object is an array with *n* elements, where *n* is determined from the number of initial elements supplied in the *new-initializer* (9.5.2, 9.5.3).

⁸ If the *expression* in a *nopt-new-declarator* is present, it is implicitly converted to `std::size_t`. The value of the *expression* is invalid if

- (8.1) — the *expression* is of non-class type and its value before converting to `std::size_t` is less than zero;
- (8.2) — the *expression* is of class type and its value before application of the second standard conversion (12.2.4.2.3)⁵³ is less than zero;
- (8.3) — its value is such that the size of the allocated object would exceed the implementation-defined limit (Annex B); or
- (8.4) — the *new-initializer* is a *braced-init-list* or a parenthesized *expression-list* and the number of array elements for which initializers are provided (including the terminating `'\0'` in a *string-literal* (5.13.5)) exceeds the number of elements to initialize.

If the value of the *expression* is invalid after converting to `std::size_t`:

- (8.5) — if the *expression* is a potentially evaluated core constant expression, the program is ill-formed;
- (8.6) — otherwise, an allocation function is not called; instead
- (8.6.1) — if the allocation function that would have been called has a non-throwing exception specification (14.5), the value of the *new-expression* is the null pointer value of the required result type;
- (8.6.2) — otherwise, the *new-expression* terminates by throwing an exception of a type that would match a handler (14.4) of type `std::bad_array_new_length` (17.6.4.2).

When the value of the *expression* is zero, the allocation function is called to allocate an array with no elements.

⁹ If the allocated type is an array, the *new-initializer* is a *braced-init-list* or a parenthesized *expression-list*, and the *expression* is potentially evaluated and not a core constant expression, the semantic constraints of initializing a hypothetical element of the array are checked as follows:

- (9.1) — If the *new-initializer* is a *braced-init-list*, the hypothetical element is copy-initialized from an empty initializer list (9.5.5).
- (9.2) — Otherwise, the hypothetical element is value-initialized (9.5.1).

[Note 5: The array can contain more elements than there are elements in the *new-initializer*, requiring initialization of the remainder of the array elements as appropriate. — end note]

¹⁰ Objects created by a *new-expression* have dynamic storage duration (6.8.6.5).

[Note 6: The lifetime of such an object is not necessarily restricted to the scope in which it is created. — end note]

¹¹ When the allocated type is “array of *N* *T*” (that is, the *nopt-new-declarator* syntax is used or the *new-type-id* or *nofun-type-id* denotes an array type), the *new-expression* yields a prvalue of type “pointer to *T*” that points to the initial element (if any) of the array. Otherwise, let *T* be the allocated type; the *new-expression* is a prvalue of type “pointer to *T*” that points to the object created.

[Note 7: Both `new int` and `new int[10]` have type `int*` and the type of `new int[i][10]` is `int (*)[10]`. — end note]

¹² A *new-expression* may obtain storage for the object by calling an allocation function (6.8.6.5.2). If the *new-expression* terminates by throwing an exception, it may release storage by calling a deallocation function (6.8.6.5.3). If the allocated type is a non-array type, the allocation function’s name is **operator new** and the deallocation function’s name is **operator delete**. If the allocated type is an array type, the allocation function’s name is **operator new[]** and the deallocation function’s name is **operator delete[]**.

[Note 8: An implementation is expected to provide default definitions for the global allocation functions (6.8.6.5, 17.6.3.2, 17.6.3.3). A C++ program can provide alternative definitions of these functions (16.4.5.6) and/or class-specific versions (11.4.11). The set of allocation and deallocation functions that can be called by a *new-expression* can include functions that do not perform allocation or deallocation; for example, see 17.6.3.4. — end note]

¹³ If the *new-expression* does not begin with a unary `::` operator and the allocated type is a class type *T* or array thereof, a search is performed for the allocation function’s name in the scope of *T* (6.5.2). Otherwise, or if nothing is found, the allocation function’s name is looked up by searching for it in the global scope.

⁵³ If the conversion function returns a signed integer type, the second standard conversion converts to the unsigned type `std::size_t` and thus thwarts any attempt to detect a negative value afterwards.

- 14 An implementation is allowed to omit a call to a replaceable global allocation function (17.6.3.2, 17.6.3.3). When it does so, the storage is instead provided by the implementation or provided by extending the allocation of another *new-expression*.
- 15 During an evaluation of a constant expression, a call to a replaceable allocation function is always omitted (7.7.2).
- 16 The implementation may extend the allocation of a *new-expression* **e1** to provide storage for a *new-expression* **e2** if the following would be true were the allocation not extended:
- (16.1) — the evaluation of **e1** is sequenced before the evaluation of **e2**, and
 - (16.2) — **e2** is evaluated whenever **e1** obtains storage, and
 - (16.3) — both **e1** and **e2** invoke the same replaceable global allocation function, and
 - (16.4) — if the allocation function invoked by **e1** and **e2** is throwing, any exceptions thrown in the evaluation of either **e1** or **e2** would be first caught in the same handler, and
 - (16.5) — the pointer values produced by **e1** and **e2** are operands to evaluated *delete-expressions*, and
 - (16.6) — the evaluation of **e2** is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by **e1**.

[Example 5:

```
void can_merge(int x) {
    // These allocations are safe for merging:
    std::unique_ptr<char[]> a{new (std::nothrow) char[8]};
    std::unique_ptr<char[]> b{new (std::nothrow) char[8]};
    std::unique_ptr<char[]> c{new (std::nothrow) char[x]};

    g(a.get(), b.get(), c.get());
}

void cannot_merge(int x) {
    std::unique_ptr<char[]> a{new char[8]};
    try {
        // Merging this allocation would change its catch handler.
        std::unique_ptr<char[]> b{new char[x]};
    } catch (const std::bad_alloc& e) {
        std::cerr << "Allocation failed: " << e.what() << std::endl;
        throw;
    }
}
```

— end example]

- 17 When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type `std::size_t`. That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array and the allocation function is not a non-allocating form (17.6.3.4). For arrays of `char`, `unsigned char`, and `std::byte`, the difference between the result of the *new-expression* and the address returned by the allocation function shall be an integral multiple of the strictest fundamental alignment requirement (6.8.3) of any object type whose size is no greater than the size of the array being created.
- [Note 9: Because allocation functions are assumed to return pointers to storage that is appropriately aligned for objects of any type with fundamental alignment, this constraint on array allocation overhead permits the common idiom of allocating character arrays into which objects of other types will later be placed. — end note]
- 18 When a *new-expression* calls an allocation function and that allocation has been extended, the size argument to the allocation call shall be no greater than the sum of the sizes for the omitted calls as specified above, plus the size for the extended call had it not been extended, plus any padding necessary to align the allocated objects within the allocated memory.
- 19 The *new-placement* syntax is used to supply additional arguments to an allocation function; such an expression is called a *placement new-expression*.
- 20 Overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. If the type of the allocated object

has new-extended alignment, the next argument is the type's alignment, and has type `std::align_val_t`. If the *new-placement* syntax is used, the *initializer-clauses* in its *expression-list* are the succeeding arguments. If no matching function is found then

- (20.1) — if the allocated object type has new-extended alignment, the alignment argument is removed from the argument list;
- (20.2) — otherwise, an argument that is the type's alignment and has type `std::align_val_t` is added into the argument list immediately after the first argument;

and then overload resolution is performed again.

21 [Example 6:

- (21.1) — `new T` results in one of the following calls:

```
operator new(sizeof(T))
operator new(sizeof(T), std::align_val_t(alignof(T)))
```
- (21.2) — `new(2,f) T` results in one of the following calls:

```
operator new(sizeof(T), 2, f)
operator new(sizeof(T), std::align_val_t(alignof(T)), 2, f)
```
- (21.3) — `new T[5]` results in one of the following calls:

```
operator new[](sizeof(T) * 5 + x)
operator new[](sizeof(T) * 5 + x, std::align_val_t(alignof(T)))
```
- (21.4) — `new(2,f) T[5]` results in one of the following calls:

```
operator new[](sizeof(T) * 5 + x, 2, f)
operator new[](sizeof(T) * 5 + x, std::align_val_t(alignof(T)), 2, f)
```

Here, each instance of *x* is a non-negative unspecified value representing array allocation overhead; the result of the *new-expression* will be offset by this amount from the value returned by `operator new[]`. This overhead may be applied in all array *new-expressions*, including those referencing a placement allocation function, except when referencing the library function `operator new[](std::size_t, void*)`. The amount of overhead may vary from one invocation of `new` to another. — end example]

- 22 [Note 10: Unless an allocation function has a non-throwing exception specification (14.5), it indicates failure to allocate storage by throwing a `std::bad_alloc` exception (6.8.6.5.2, 14.2, 17.6.4.1); it returns a non-null pointer otherwise. If the allocation function has a non-throwing exception specification, it returns null to indicate failure to allocate storage and a non-null pointer otherwise. — end note]

If the allocation function is a non-allocating form (17.6.3.4) that returns null, the behavior is undefined (F.3.21). Otherwise, if the allocation function returns null, initialization shall not be done, the deallocation function shall not be called, and the value of the *new-expression* shall be null.

- 23 [Note 11: When the allocation function returns a value other than null, it must be a pointer to a block of storage in which space for the object has been reserved. The block of storage is assumed to be appropriately aligned (6.8.3) and of the requested size. The address of the created object will not necessarily be the same as that of the block if the object is an array. — end note]

24 A *new-expression* that creates an object of type *T* initializes that object as follows:

- (24.1) — If the *new-initializer* is omitted, the object is default-initialized (9.5).
 [Note 12: If no initialization is performed, the object has an indeterminate value. — end note]
- (24.2) — Otherwise, the *new-initializer* is interpreted according to the initialization rules of 9.5 for direct-initialization.

25 The invocation of the allocation function is sequenced before the evaluations of expressions in the *new-initializer*. Initialization of the allocated object is sequenced before the value computation of the *new-expression*.

26 If the *new-expression* creates an array of objects of class type, the destructor is potentially invoked (11.4.7).

27 If any part of the object initialization described above⁵⁴ terminates by throwing an exception and a suitable deallocation function can be found, the deallocation function is called to free the memory in which the object was being constructed, after which the exception continues to propagate in the context of the *new-expression*. If no unambiguous matching deallocation function can be found, propagating the exception does not cause the object's memory to be freed.

⁵⁴) This can include evaluating a *new-initializer* and/or calling a constructor.

[*Note 13*: This is appropriate when the called allocation function does not allocate memory; otherwise, it is likely to result in a memory leak. — *end note*]

- 28 If the *new-expression* does not begin with a unary `::` operator and the allocated type is a class type `T` or an array thereof, a search is performed for the deallocation function's name in the scope of `T`. Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope.
- 29 For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function (7.6.2.9). For a placement allocation function, the selection process is described below. In any case, the matching deallocation function (if any) shall be non-deleted and accessible from the point where the *new-expression* appears.
- 30 For a placement allocation function, the matching deallocation function is selected as follows:
- (30.1) — Each candidate that is a function template is replaced by the function template specializations (if any) generated using template argument deduction (13.10.4, 13.10.3.2) with arguments as specified below.
 - (30.2) — Each candidate whose parameter-type-list is not identical to that of the allocation function, ignoring their respective first parameters, is removed from the set of candidates.
 - (30.3) — Each candidate whose associated constraints (if any) are not satisfied (13.5.2) is removed from the set of candidates.
 - (30.4) — If exactly one function remains, that function is selected.
 - (30.5) — Otherwise, no deallocation function is selected.

If a usual deallocation function is selected, the program is ill-formed.

[*Note 14*: A deallocation function with an additional trailing parameter compared to the allocation function is never matched, even if a default argument is provided. — *end note*]

31 [*Example 7*:

```
struct S {
    // Placement allocation function:
    static void* operator new(std::size_t, std::size_t);

    // Usual (non-placement) deallocation function:
    static void operator delete(void*, std::size_t);
};

S* p = new (0) S;    // error: non-placement deallocation function matches
                    // placement allocation function
```

— *end example*]

[*Example 8*:

```
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A& al);           // #1

template<int = 0>
void operator delete(void* p, A& al);               // #2

A al;
T* p = new (al) T;                                 // OK, uses #1 and #2.
```

— *end example*]

[*Example 9*:

```
template<int I>
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A<0>& al);        // #1

template<int I>
void operator delete(void* p, A<I>& al);
void operator delete(void* p, A<0>& al);
```

```
A<0> al;
T* p = new (al) T;      // OK, uses #1. No deallocation function is selected (two candidates remain).
— end example]
```

[Example 10:

```
template<int I>
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A<0>& al);    // #1

template<int I> requires (I > 0)
void operator delete(void* p, A<I>& al);
void operator delete(void* p, A<0>& al);        // #2

A<0> al;
T* p = new (al) T;                            // OK, uses #1 and #2.
— end example]
```

- ³² If a *new-expression* calls a deallocation function, it passes the value returned from the allocation function call as the first argument of type `void*`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the *new-placement* syntax. If the implementation is allowed to introduce a temporary object or make a copy of any argument as part of the call to the allocation function, it is unspecified whether the same object is used in the call to both the allocation and deallocation functions.

7.6.2.9 Delete

[**expr.delete**]

- ¹ The *delete-expression* operator destroys a most derived object (6.8.2) or array created by a *new-expression*.

```
delete-expression:
    ::opt delete cast-expression
    ::opt delete [ ] cast-expression
```

The first alternative is a *single-object delete expression*, and the second is an *array delete expression*. Whenever the **delete** keyword is immediately followed by empty square brackets, it shall be interpreted as the second alternative.⁵⁵ If the operand is of class type, it is contextually implicitly converted (7.3) to a pointer to object type and the converted operand is used in place of the original operand for the remainder of this subclause. Otherwise, it shall be a prvalue of pointer to object type. The *delete-expression* has type `void`.

- ² In a single-object delete expression, the value of the operand of **delete** may be a null pointer value, a pointer value that resulted from a previous non-array *new-expression*, or a pointer to a base class subobject of an object created by such a *new-expression*. If not, the behavior is undefined (F.3.22). In an array delete expression, the value of the operand of **delete** may be a null pointer value or a pointer value that resulted from a previous array *new-expression* whose allocation function was not a non-allocating form (17.6.3.4).⁵⁶ If not, the behavior is undefined (F.3.23).

[Note 1: This means that the syntax of the *delete-expression* must match the type of the object allocated by **new**, not the syntax of the *new-expression*. — end note]

[Note 2: A pointer to a **const** type can be the operand of a *delete-expression*; it is not necessary to cast away the constness (7.6.1.11) of the pointer expression before it is used as the operand of the *delete-expression*. — end note]

- ³ In a single-object delete expression, if the static type of the object to be deleted is not similar (7.3.6) to its dynamic type and the selected deallocation function (see below) is not a destroying operator delete, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined (F.3.24). In an array delete expression, if the dynamic type of the object to be deleted is not similar to its static type, the behavior is undefined (F.3.25).
- ⁴ If the object being deleted has incomplete class type at the point of deletion, the program is ill-formed.
- ⁵ If the value of the operand of the *delete-expression* is not a null pointer value and the selected deallocation function (see below) is not a destroying operator delete, evaluating the *delete-expression* invokes the destructor

⁵⁵) A *lambda-expression* with a *lambda-introducer* that consists of empty square brackets can follow the **delete** keyword if the *lambda-expression* is enclosed in parentheses.

⁵⁶) For nonzero-length arrays, this is the same as a pointer to the first element of the array created by that *new-expression*. Zero-length arrays do not have a first element.

(if any) for the object or the elements of the array being deleted. The destructor shall be accessible from the point where the *delete-expression* appears. In the case of an array, the elements are destroyed in order of decreasing address (that is, in reverse order of the completion of their constructor; see 11.9.3).

6 If the value of the operand of the *delete-expression* is not a null pointer value, then:

- (6.1) — If the allocation call for the *new-expression* for the object to be deleted was not omitted and the allocation was not extended (7.6.2.8), the *delete-expression* shall call a deallocation function (6.8.6.5.3). The value returned from the allocation call of the *new-expression* shall be passed as the first argument to the deallocation function.
- (6.2) — Otherwise, if the allocation was extended or was provided by extending the allocation of another *new-expression*, and the *delete-expression* for every other pointer value produced by a *new-expression* that had storage provided by the extended *new-expression* has been evaluated, the *delete-expression* shall call a deallocation function. The value returned from the allocation call of the extended *new-expression* shall be passed as the first argument to the deallocation function.
- (6.3) — Otherwise, the *delete-expression* will not call a deallocation function.

[Note 3: The deallocation function is called regardless of whether the destructor for the object or some element of the array throws an exception. — end note]

If the value of the operand of the *delete-expression* is a null pointer value, it is unspecified whether a deallocation function will be called as described above.

7 If a deallocation function is called, it is **operator delete** for a single-object delete expression or **operator delete[]** for an array delete expression.

[Note 4: An implementation provides default definitions of the global deallocation functions (17.6.3.2, 17.6.3.3). A C++ program can provide alternative definitions of these functions (16.4.5.6), and/or class-specific versions (11.4.11). — end note]

8 If the keyword **delete** in a *delete-expression* is not preceded by the unary **::** operator and the type of the operand is a pointer to a (possibly cv-qualified) class type **T** or (possibly multidimensional) array thereof:

- (8.1) — For a single-object delete expression, if the operand is a pointer to *cv T* and **T** has a virtual destructor, the deallocation function is the one selected at the point of definition of the dynamic type's virtual destructor (11.4.7).
- (8.2) — Otherwise, a search is performed for the deallocation function's name in the scope of **T**.

Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope. In any case, any declarations other than of usual deallocation functions (6.8.6.5.3) are discarded.

[Note 5: If only a placement deallocation function is found in a class, the program is ill-formed because the lookup set is empty (6.5). — end note]

9 The deallocation function to be called is selected as follows:

- (9.1) — If any of the deallocation functions is a destroying operator delete, all deallocation functions that are not destroying operator deletes are eliminated from further consideration.
- (9.2) — If the type has new-extended alignment, a function with a parameter of type **std::align_val_t** is preferred; otherwise a function without such a parameter is preferred. If any preferred functions are found, all non-preferred functions are eliminated from further consideration.
- (9.3) — If exactly one function remains, that function is selected and the selection process terminates.
- (9.4) — If the deallocation functions belong to a class scope, the one without a parameter of type **std::size_t** is selected.
- (9.5) — If the type is complete and if, for an array delete expression only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multidimensional) array thereof, the function with a parameter of type **std::size_t** is selected.
- (9.6) — Otherwise, it is unspecified whether a deallocation function with a parameter of type **std::size_t** is selected.

Unless the deallocation function is selected at the point of definition of the dynamic type's virtual destructor, the selected deallocation function shall be accessible from the point where the *delete-expression* appears.

10 For a single-object delete expression, the deleted object is the object *A* pointed to by the operand if the static type of *A* does not have a virtual destructor, and the most-derived object of *A* otherwise.

[*Note 6*: If the deallocation function is not a destroying operator delete and the deleted object is not the most derived object in the former case, the behavior is undefined, as stated above. — *end note*]

For an array delete expression, the deleted object is the array object. When a *delete-expression* is executed, the selected deallocation function shall be called with the address of the deleted object in a single-object delete expression, or the address of the deleted object suitably adjusted for the array allocation overhead (7.6.2.8) in an array delete expression, as its first argument.

[*Note 7*: Any cv-qualifiers in the type of the deleted object are ignored when forming this argument. — *end note*]

If a destroying operator delete is used, an unspecified value is passed as the argument corresponding to the parameter of type `std::destroying_delete_t`. If a deallocation function with a parameter of type `std::align_val_t` is used, the alignment of the type of the deleted object is passed as the corresponding argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the deleted object in a single-object delete expression, or of the array plus allocation overhead in an array delete expression, is passed as the corresponding argument.

[*Note 8*: If this results in a call to a replaceable deallocation function, and either the first argument was not the result of a prior call to a replaceable allocation function or the second or third argument was not the corresponding argument in said call, the behavior is undefined (17.6.3.2, 17.6.3.3). — *end note*]

7.6.2.10 The reflection operator

[`expr.reflect`]

```
reflect-expression:
    ^^ ::
    ^^ reflection-name
    ^^ type-id
    ^^ id-expression

reflection-name:
    nested-name-specifieropt identifier
    nested-name-specifier template identifier
```

- ¹ The unary ^^ operator, called the *reflection operator*, yields a prvalue of type `std::meta::info` (6.9.3).

[*Note 1*: This document places no restriction on representing, by reflections, constructs not described by this document or using the names of such constructs as operands of *reflect-expressions*. — *end note*]

- ² The component names of a *reflection-name* are those of its *nested-name-specifier* (if any) and its *identifier*. The terminal name of a *reflection-name* of the form *nested-name-specifier* **template** *identifier* shall denote a template.
- ³ A *reflect-expression* is parsed as the longest possible sequence of tokens that could syntactically form a *reflect-expression*. An unparenthesized *reflect-expression* that represents a template shall not be followed by <.

[*Example 1*:

```
static_assert(std::meta::is_type(^int()));    // ^^ applies to the type-id int()

template<bool> struct X {};
constexpr bool operator<(std::meta::info, X<false>) { return false; }
constexpr void g(std::meta::info r, X<false> xv) {
    r == ^int && true;           // error: ^^ applies to the type-id int&&
    r == ^int & true;           // error: ^^ applies to the type-id int&
    r == (^int) && true;        // OK
    r == ^int &&&& true;         // error: int &&&& is not a valid type-id
    ^^X < xv;                   // error: reflect-expression that represents a template is followed by <
    (^X) < xv;                  // OK
    ^^X<true> < xv;            // OK
}
```

— *end example*]

- ⁴ A *reflect-expression* of the form ^^:: represents the global namespace.
- ⁵ If a *reflect-expression* *R* matches the form ^^*reflection-name*, it is interpreted as such; the *identifier* is looked up and the representation of *R* is determined as follows:
- (5.1) — If lookup finds a declaration that replaced a *using-declarator* during a single search (6.5.1, 9.10), *R* is ill-formed.

[Example 2:

```
struct A { struct S {}; };
struct B : A { using A::S; };
constexpr std::meta::info r1 = ^B::S; // error: A::S found through using-declarator

struct C : virtual B { struct S {}; };
struct D : virtual B, C {};
D::S s; // OK, names C::S per 6.5.2
constexpr std::meta::info r2 = ^D::S; // OK, result C::S not found through using-declarator
— end example]
```

- (5.2) — Otherwise, if lookup finds a namespace alias (9.9.3), all declarations found by name lookup shall have the same target scope, and *R* represents the namespace alias.
- (5.3) — Otherwise, if lookup finds a namespace (9.9), *R* represents that namespace.
- (5.4) — Otherwise, if lookup finds a concept (13.7.9), *R* represents the denoted concept.
- (5.5) — Otherwise, if lookup finds a template (13.3), the representation of *R* is determined as follows:
 - (5.5.1) — If lookup finds an injected-class-name (11.1), then:
 - (5.5.1.1) — If the *reflection-name* is of the form *nested-name-specifier* **template** *identifier*, then *R* represents the class template named by the injected-class-name.
 - (5.5.1.2) — Otherwise, the injected-class-name shall be unambiguous when considered as a *type-name* and *R* represents the class template specialization so named.
 - (5.5.2) — Otherwise, if lookup finds an overload set, that overload set shall contain only declarations of a unique function template *F*; *R* represents *F*.
 - (5.5.3) — Otherwise, if lookup finds a class template, variable template, or alias template, *R* represents that template.
- [Note 2: Lookup never finds a partial or explicit specialization. — end note]
- (5.6) — Otherwise, if lookup finds a type alias:
 - (5.6.1) — If any declaration found by name lookup is of a template parameter *T*, *R* represents the underlying entity of *T*.
 - (5.6.2) — Otherwise, all declarations found by name lookup shall have the same target scope, and *R* represents the type alias.

[Example 3:

```
namespace A {
    using T = int;
}
namespace B {
    using T = int;
}
using namespace A;
using namespace B;
auto r = ^T; // error: lookup finds type aliases with different target scopes
— end example]
```

- (5.7) — Otherwise, if lookup finds a class or an enumeration, *R* represents the denoted type.
- (5.8) — Otherwise, if lookup finds a class member of an anonymous union (11.5.2), *R* represents that class member.
- (5.9) — Otherwise, the *reflection-name* shall be an *id-expression* *I* and *R* is ^*I* (see below).

⁶ A *reflect-expression* *R* of the form ^*type-id* represents an entity determined as follows:

- (6.1) — If the *type-id* designates a placeholder type (9.2.9.7.1), *R* is ill-formed.
- (6.2) — Otherwise, if the *type-id* is of the form *nested-name-specifier*_{opt} **template**_{opt} *simple-template-id* and whose terminal name is a *template-name* that names an alias template (13.7.8), *R* represents the type alias so named.
- (6.3) — Otherwise, *R* represents the type denoted by the *type-id*.

⁷ A *reflect-expression* R of the form $\text{\textasciitilde}\text{\textasciitilde}id\text{-expression}$ represents an entity determined as follows:

- (7.1) — If the *id-expression* denotes
 - (7.1.1) — a variable declared by an *init-capture* (7.5.6.3),
 - (7.1.2) — a function-local predefined variable (9.6.1),
 - (7.1.3) — a local parameter introduced by a *requires-expression* (7.5.8), or
 - (7.1.4) — a local entity E (6.1) for which a lambda scope intervenes between the point at which E was introduced and R ,
 then R is ill-formed.
 - (7.2) — Otherwise, if the *id-expression* denotes an overload set S , the expression $\&id\text{-expression}$ shall be well-formed when considered as an unevaluated operand, except that the function F selected as described in 12.3 may be deleted (9.6.3); R represents F .
 - (7.3) — Otherwise, if the *id-expression* denotes a variable, structured binding, enumerator, or non-static data member, R represents that entity.
 - (7.4) — Otherwise, R is ill-formed.
- [Note 3: This includes *unqualified-ids* that name a constant template parameter and *pack-index-expressions*. — end note]

The *id-expression* of a *reflect-expression* is an unevaluated operand (7.2.3).

[Example 4:

```
template<typename T> void fn() requires (^^T != ^^int);
template<typename T> void fn() requires (^^T == ^^int);
template<typename T> void fn() requires (sizeof(T) == sizeof(int));

constexpr std::meta::info a = ^^fn<char>;           // OK
constexpr std::meta::info b = ^^fn<int>;           // error: ambiguous

constexpr std::meta::info c = ^^std::vector;        // OK

template<typename T>
struct S {
    static constexpr std::meta::info r = ^^T;
    using type = T;
};
static_assert(S<int>::r == ^^int);
static_assert(^^S<int>::type != ^^int);

typedef struct X {} Y;
typedef struct Z {} Z;
constexpr std::meta::info e = ^^Y;                 // OK, represents the type alias Y
constexpr std::meta::info f = ^^Z;                 // OK, represents the type alias Z, not the type (6.5.1)
```

— end example]

7.6.3 Explicit type conversion (cast notation)

[expr.cast]

- ¹ The result of the expression $(T) \text{ cast-expression}$ is of type T . The result is an lvalue if T is an lvalue reference type or an rvalue reference to function type and an xvalue if T is an rvalue reference to object type; otherwise the result is a prvalue.

[Note 1: If T is a non-class type that is cv-qualified, the *cv-qualifiers* are discarded when determining the type of the resulting prvalue; see 7.2. — end note]

- ² An explicit type conversion can be expressed using functional notation (7.6.1.4), a type conversion operator (`dynamic_cast`, `static_cast`, `reinterpret_cast`, `const_cast`), or the *cast* notation.

cast-expression:
 unary-expression
 (*nofun-type-id*) *cast-expression*

- ³ Any type conversion not mentioned below and not explicitly defined by the user (11.4.8) is ill-formed.
- ⁴ The conversions performed by

- (4.1) — a `const_cast` (7.6.1.11),
- (4.2) — a `static_cast` (7.6.1.9),
- (4.3) — a `static_cast` followed by a `const_cast`,
- (4.4) — a `reinterpret_cast` (7.6.1.10), or
- (4.5) — a `reinterpret_cast` followed by a `const_cast`,

can be performed using the cast notation of explicit type conversion. The same semantic restrictions and behaviors apply, with the exception that in performing a `static_cast` in the following situations the conversion is valid even if the base class is inaccessible:

- (4.6) — a pointer to an object of derived class type or an lvalue or rvalue of derived class type may be explicitly converted to a pointer or reference to an unambiguous base class type, respectively;
- (4.7) — a pointer to member of derived class type may be explicitly converted to a pointer to member of an unambiguous non-virtual base class type;
- (4.8) — a pointer to an object of an unambiguous non-virtual base class type, a glvalue of an unambiguous non-virtual base class type, or a pointer to member of an unambiguous non-virtual base class type may be explicitly converted to a pointer, a reference, or a pointer to member of a derived class type, respectively.

If a conversion can be interpreted in more than one of the ways listed above, the interpretation that appears first in the list is used, even if a cast resulting from that interpretation is ill-formed. If a `static_cast` followed by a `const_cast` is used and the conversion can be interpreted in more than one way as such, the conversion is ill-formed.

[Example 1:

```
struct A { };
struct I1 : A { };
struct I2 : A { };
struct D : I1, I2 { };
A* foo( D* p ) {
    return (A*)( p );           // ill-formed static_cast interpretation
}

int*** ptr = 0;
auto t = (int const*const*const*)ptr;    // OK, const_cast interpretation

struct S {
    operator const int*();
    operator volatile int*();
};
int *p = (int*)S();           // error: two possible interpretations using static_cast followed by const_cast
```

— end example]

- 5 The operand of a cast using the cast notation can be a prvalue of type “pointer to incomplete class type”. The destination type of a cast using the cast notation can be “pointer to incomplete class type”. If both the operand and destination types are class types and one or both are incomplete, it is unspecified whether the `static_cast` or the `reinterpret_cast` interpretation is used, even if there is an inheritance relationship between the two classes.

[Note 2: For example, if the classes were defined later in the translation unit, a multi-pass compiler could validly interpret a cast between pointers to the classes as if the class types were complete at the point of the cast. — end note]

7.6.4 Pointer-to-member operators

[expr.mptr.oper]

- 1 The pointer-to-member operators `->*` and `.*` group left-to-right.

```
pm-expression:
    cast-expression
    pm-expression .* cast-expression
    pm-expression ->* cast-expression
```


- ² The binary operator `.*` binds its second operand, which shall be a prvalue of type “pointer to member of T” to its first operand, which shall be a glvalue of class T or of a class of which T is an unambiguous and accessible base class. The result is an object or a function of the type specified by the second operand.
- ³ The binary operator `->*` binds its second operand, which shall be a prvalue of type “pointer to member of T” to its first operand, which shall be of type “pointer to U” where U is either T or a class of which T is an unambiguous and accessible base class. The expression `E1->*E2` is converted into the equivalent form `(*(E1)).*E2`.
- ⁴ Abbreviating *pm-expression.*cast-expression* as `E1.*E2`, E1 is called the *object expression*. If the result of E1 is an object whose type is not similar to the type of E2, or whose most derived object does not contain the member to which E2 refers, the behavior is undefined (F.3.26). The expression E1 is sequenced before the expression E2.
- ⁵ The restrictions on cv-qualification, and the manner in which the cv-qualifiers of the operands are combined to produce the cv-qualifiers of the result, are the same as the rules for `E1.E2` given in 7.6.1.5.

[Note 1: It is not possible to use a pointer to member that refers to a **mutable** member to modify a **const** class object. For example,

```
struct S {
    S() : i(0) { }
    mutable int i;
};
void f()
{
    const S cs;
    int S::* pm = &S::i;           // pm refers to mutable member S::i
    cs.*pm = 88;                   // error: cs is a const object
}
```

— end note]

- ⁶ If the result of `.*` or `->*` is a function, then that result can be used only as the operand for the function call operator `()`.

[Example 1:

```
(ptr_to_obj->*ptr_to_mfct)(10);
```

calls the member function denoted by `ptr_to_mfct` for the object pointed to by `ptr_to_obj`. — end example]

In a `.*` expression whose object expression is an rvalue, the program is ill-formed if the second operand is a pointer to member function whose *ref-qualifier* is `&`, unless its *cv-qualifier-seq* is `const`. In a `.*` expression whose object expression is an lvalue, the program is ill-formed if the second operand is a pointer to member function whose *ref-qualifier* is `&&`. The result of a `.*` expression whose second operand is a pointer to a data member is an lvalue if the first operand is an lvalue and an xvalue otherwise. The result of a `.*` expression whose second operand is a pointer to a member function is a prvalue. If the second operand is the null member pointer value (7.3.13), the behavior is undefined (F.3.27).

7.6.5 Multiplicative operators

[expr.mul]

- ¹ The multiplicative operators `*`, `/`, and `%` group left-to-right.

multiplicative-expression:

```
pm-expression
multiplicative-expression * pm-expression
multiplicative-expression / pm-expression
multiplicative-expression % pm-expression
```

- ² The operands of `*` and `/` shall have arithmetic or unscoped enumeration type; the operands of `%` shall have integral or unscoped enumeration type. The usual arithmetic conversions (7.4) are performed on the operands and determine the type of the result.
- ³ The binary `*` operator indicates multiplication.
- ⁴ The binary `/` operator yields the quotient, and the binary `%` operator yields the remainder from the division of the first expression by the second. If the second operand of `/` or `%` is zero, the behavior is undefined (F.3.28). For integral operands, the `/` operator yields the algebraic quotient with any fractional part discarded;⁵⁷ if the

⁵⁷) This is often called truncation towards zero.

quotient a/b is representable in the type of the result, $(a/b)*b + a\%b$ is equal to a ; otherwise, the behavior of both a/b and $a\%b$ is undefined (F.3.29).

7.6.6 Additive operators

[expr.add]

- ¹ The additive operators $+$ and $-$ group left-to-right. Each operand shall be a prvalue. If both operands have arithmetic or unscoped enumeration type, the usual arithmetic conversions (7.4) are performed. Otherwise, if one operand has arithmetic or unscoped enumeration type, integral promotion is applied (7.3.7) to that operand. A converted or promoted operand is used in place of the corresponding original operand for the remainder of this section.

additive-expression:

multiplicative-expression

additive-expression $+$ *multiplicative-expression*

additive-expression $-$ *multiplicative-expression*

For addition, either both operands shall have arithmetic type, or one operand shall be a pointer to a completely-defined object type and the other shall have integral type.

- ² For subtraction, one of the following shall hold:

- (2.1) — both operands have arithmetic type; or
 - (2.2) — both operands are pointers to similar (7.3.6) complete object types; or
 - (2.3) — the left operand is a pointer to a completely-defined object type and the right operand has integral type.
- ³ The result of the binary $+$ operator is the sum of the operands. The result of the binary $-$ operator is the difference resulting from the subtraction of the second operand from the first.
 - ⁴ When an expression J that has integral type is added to or subtracted from an expression P of pointer type, the result has the type of P .
 - (4.1) — If P evaluates to a null pointer value and J evaluates to 0, the result is a null pointer value.
 - (4.2) — Otherwise, if P points to a (possibly-hypothetical) array element i of an array object x with n elements (9.3.4.5),⁵⁸ the expressions $P + J$ and $J + P$ (where J has the value j) point to the (possibly-hypothetical) array element $i + j$ of x if $0 \leq i + j \leq n$ and the expression $P - J$ points to the (possibly-hypothetical) array element $i - j$ of x if $0 \leq i - j \leq n$.
 - (4.3) — Otherwise, the behavior is undefined (F.3.30).

[Note 1: Adding a value other than 0 or 1 to a pointer to a base class subobject, a member subobject, or a complete object results in undefined behavior. — end note]

- ⁵ The result of subtracting two pointer expressions P and Q is a prvalue of type `std::ptrdiff_t` (17.2.4).

- (5.1) — If P and Q both evaluate to null pointer values, the value is 0.
- (5.2) — Otherwise, if P and Q point to, respectively, array elements i and j of the same array object x , the expression $P - Q$ has the value $i - j$.

[Note 2: If the value $i - j$ is not in the range of representable values of type `std::ptrdiff_t`, the behavior is undefined (7.1). — end note]

- (5.3) — Otherwise, the behavior is undefined (F.3.31).

- ⁶ For addition or subtraction, if the expressions P or Q have type “pointer to *cv* T ”, where T and the array element type are not similar (7.3.6), the behavior is undefined (F.3.32).

[Example 1:

```
int arr[5] = {1, 2, 3, 4, 5};
unsigned int *p = reinterpret_cast<unsigned int*>(arr + 1);
unsigned int k = *p;           // OK, value of k is 2 (7.3.2)
unsigned int *q = p + 1;       // undefined behavior: p points to an int, not an unsigned int object
```

— end example]

⁵⁸ As specified in 6.9.5, an object that is not an array element is considered to belong to a single-element array for this purpose and a pointer past the last element of an array of n elements is considered to be equivalent to a pointer to a hypothetical array element n for this purpose.

7.6.7 Shift operators

[expr.shift]

- 1 The shift operators << and >> group left-to-right.

```

shift-expression:
    additive-expression
    shift-expression << additive-expression
    shift-expression >> additive-expression

```

The operands shall be prvalues of integral or unscoped enumeration type and integral promotions are performed. The type of the result is that of the promoted left operand. The behavior is undefined (F.3.33) if the right operand is negative, or greater than or equal to the width of the promoted left operand.

- 2 The value of $E1 \ll E2$ is the unique value congruent to $E1 \times 2^{E2}$ modulo 2^N , where N is the width of the type of the result.

[Note 1: $E1$ is left-shifted $E2$ bit positions; vacated bits are zero-filled. — end note]

- 3 The value of $E1 \gg E2$ is $E1/2^{E2}$, rounded towards negative infinity.

[Note 2: $E1$ is right-shifted $E2$ bit positions. Right-shift on signed integral types is an arithmetic right shift, which performs sign-extension. — end note]

- 4 The expression $E1$ is sequenced before the expression $E2$.

7.6.8 Three-way comparison operator

[expr.spaceship]

- 1 The three-way comparison operator groups left-to-right.

```

compare-expression:
    shift-expression
    compare-expression <=> shift-expression

```

- 2 The expression $p \leq q$ is a prvalue indicating whether p is less than, equal to, greater than, or incomparable with q .

- 3 If one of the operands is of type `bool` and the other is not, the program is ill-formed.

- 4 If both operands have arithmetic types, or one operand has integral type and the other operand has unscoped enumeration type, the usual arithmetic conversions (7.4) are applied to the operands. Then:

- (4.1) — If a narrowing conversion (9.5.5) is required, other than from an integral type to a floating-point type, the program is ill-formed.
- (4.2) — Otherwise, if the operands have integral type, the result is of type `std::strong_ordering`. The result is `std::strong_ordering::equal` if both operands are arithmetically equal, `std::strong_ordering::less` if the first operand is arithmetically less than the second operand, and `std::strong_ordering::greater` otherwise.
- (4.3) — Otherwise, the operands have floating-point type, and the result is of type `std::partial_ordering`. The expression $a \leq b$ yields `std::partial_ordering::less` if a is less than b , `std::partial_ordering::greater` if a is greater than b , `std::partial_ordering::equivalent` if a is equivalent to b , and `std::partial_ordering::unordered` otherwise.

- 5 If both operands have the same enumeration type E , the operator yields the result of converting the operands to the underlying type of E and applying $\leq$ to the converted operands.

- 6 If at least one of the operands is of object pointer type and the other operand is of object pointer or array type, array-to-pointer conversions (7.3.3), pointer conversions (7.3.12), and qualification conversions (7.3.6) are performed on both operands to bring them to their composite pointer type (7.2.2). After the conversions, the operands shall have the same type.

[Note 1: If both of the operands are arrays, array-to-pointer conversions (7.3.3) are not applied. — end note]

In this case, $p \leq q$ is of type `std::strong_ordering` and the result is defined by the following rules:

- (6.1) — If two pointer operands p and q compare equal (7.6.10), $p \leq q$ yields `std::strong_ordering::equal`;
 - (6.2) — otherwise, if p and q compare unequal, $p \leq q$ yields `std::strong_ordering::less` if q compares greater than p and `std::strong_ordering::greater` if p compares greater than q (7.6.9);
 - (6.3) — otherwise, the result is unspecified.
- 7 Otherwise, the program is ill-formed.

- ⁸ The three comparison category types (17.12.2) (the types `std::strong_ordering`, `std::weak_ordering`, and `std::partial_ordering`) are not predefined; if a standard library declaration (17.12.1, 16.4.2.4) of such a class type does not precede (6.5.1) a use of that type — even an implicit use in which the type is not named (e.g., via the `auto` specifier (9.2.9.7) in a defaulted three-way comparison (11.10.3) or use of the built-in operator) — the program is ill-formed.

7.6.9 Relational operators

[expr.rel]

- ¹ The relational operators group left-to-right.

[Example 1: `a<b<c` means `(a<b)<c` and *not* `(a<b)&&(b<c)`. — end example]

relational-expression:

```
compare-expression
relational-expression < compare-expression
relational-expression > compare-expression
relational-expression <= compare-expression
relational-expression >= compare-expression
```

The lvalue-to-rvalue (7.3.2) and function-to-pointer (7.3.4) standard conversions are performed on the operands. If one of the operands is a pointer, the array-to-pointer conversion (7.3.3) is performed on the other operand.

- ² The converted operands shall have arithmetic, enumeration, or pointer type. The operators `<` (less than), `>` (greater than), `<=` (less than or equal to), and `>=` (greater than or equal to) all yield `false` or `true`. The type of the result is `bool`.
- ³ The usual arithmetic conversions (7.4) are performed on operands of arithmetic or enumeration type. If both converted operands are pointers, pointer conversions (7.3.12), function pointer conversions (7.3.14), and qualification conversions (7.3.6) are performed to bring them to their composite pointer type (7.2.2). After conversions, the operands shall have the same type.
- ⁴ The result of comparing unequal pointers to objects⁵⁹ is defined in terms of a partial order consistent with the following rules:
- (4.1) — If two pointers point to different elements of the same array, or to subobjects thereof, the pointer to the element with the higher subscript is required to compare greater.
 - (4.2) — If two pointers point to different non-static data members of the same object, or to subobjects of such members, recursively, the pointer to the later declared member is required to compare greater provided neither member is a subobject of zero size and their class is not a union.
 - (4.3) — Otherwise, neither pointer is required to compare greater than the other.
- ⁵ If two operands `p` and `q` compare equal (7.6.10), `p<=q` and `p>=q` both yield `true` and `p<q` and `p>q` both yield `false`. Otherwise, if a pointer to object `p` compares greater than a pointer `q`, `p>=q`, `p>q`, `q<=p`, and `q<p` all yield `true` and `p<=q`, `p<q`, `q>=p`, and `q>p` all yield `false`. Otherwise, the result of each of the operators is unspecified.
- [Note 1: A relational operator applied to unequal function pointers yields an unspecified result. A pointer value of type “pointer to *cv* void” can point to an object (6.9.5). — end note]
- ⁶ If both operands (after conversions) are of arithmetic or enumeration type, each of the operators shall yield `true` if the specified relationship is true and `false` if it is false.

7.6.10 Equality operators

[expr.eq]

equality-expression:

```
relational-expression
equality-expression == relational-expression
equality-expression != relational-expression
```

- ¹ The `==` (equal to) and the `!=` (not equal to) operators group left-to-right. The lvalue-to-rvalue (7.3.2) and function-to-pointer (7.3.4) standard conversions are performed on the operands. If one of the operands is a pointer or a null pointer constant (7.3.12), the array-to-pointer conversion (7.3.3) is performed on the other operand.

⁵⁹ As specified in 6.9.5, an object that is not an array element is considered to belong to a single-element array for this purpose and a pointer past the last element of an array of *n* elements is considered to be equivalent to a pointer to a hypothetical array element *n* for this purpose.

² The converted operands shall have scalar type. The operators `==` and `!=` both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.

³ Any two pointer values or two pointer-to-member values either compare equal or compare unequal.

[*Note 1*: Repeated comparisons are consistent so long as neither value is an invalid pointer value. — *end note*]

⁴ If at least one of the converted operands is a pointer, pointer conversions (7.3.12), function pointer conversions (7.3.14), and qualification conversions (7.3.6) are performed on both operands to bring them to their composite pointer type (7.2.2). Comparing pointers is defined as follows:

- (4.1) — If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object,⁶⁰ the result of the comparison is unspecified.
- (4.2) — Otherwise, if the pointers are both null, both point to the same function, or both represent the same address (6.9.5), they compare equal.
- (4.3) — Otherwise, the pointers compare unequal.

⁵ If at least one of the operands is a pointer to member, pointer-to-member conversions (7.3.13), function pointer conversions (7.3.14), and qualification conversions (7.3.6) are performed on both operands to bring them to their composite pointer type (7.2.2). Comparing pointers to members is defined as follows:

- (5.1) — If two pointers to members are both the null member pointer value, they compare equal.
- (5.2) — If only one of two pointers to members is the null member pointer value, they compare unequal.
- (5.3) — If either is a pointer to a virtual member function, the result is unspecified.
- (5.4) — If one refers to a member of class C1 and the other refers to a member of a different class C2, where neither is a base class of the other, the result is unspecified.

[*Example 1*:

```
struct A {};
struct B : A { int x; };
struct C : A { int x; };

int A::*bx = (int(A::*))&B::x;
int A::*cx = (int(A::*))&C::x;

bool b1 = (bx == cx);    // unspecified
— end example]
```

- (5.5) — If both refer to (possibly different) members of the same union (11.5), they compare equal.
- (5.6) — Otherwise, two pointers to members compare equal if they would refer to the same member of the same most derived object (6.8.2) or the same subobject if indirection with a hypothetical object of the associated class type were performed, otherwise they compare unequal.

[*Example 2*:

```
struct B {
    int f();
};
struct L : B { };
struct R : B { };
struct D : L, R { };

int (B::*pb)() = &B::f;
int (L::*pl)() = pb;
int (R::*pr)() = pb;
int (D::*pdl)() = pl;
int (D::*pdr)() = pr;
bool x = (pdl == pdr);    // false
bool y = (pb == pl);     // true
— end example]
```

⁶⁰ As specified in 6.9.5, an object that is not an array element is considered to belong to a single-element array for this purpose.

- ⁶ Two operands of type `std::nullptr_t` or one operand of type `std::nullptr_t` and the other a null pointer constant compare equal.
- ⁷ If both operands are of type `std::meta::info`, they compare equal if both operands
- (7.1) — are null reflection values,
 - (7.2) — represent values that are template-argument-equivalent (13.6),
 - (7.3) — represent the same object,
 - (7.4) — represent the same entity,
 - (7.5) — represent the same annotation (9.13.12),
 - (7.6) — represent the same direct base class relationship, or
 - (7.7) — represent equal data member descriptions (11.4.1),
- and they compare unequal otherwise.
- ⁸ If two operands compare equal, the result is `true` for the `==` operator and `false` for the `!=` operator. If two operands compare unequal, the result is `false` for the `==` operator and `true` for the `!=` operator.
- ⁹ If both operands are of arithmetic or enumeration type, the usual arithmetic conversions (7.4) are performed on both operands; each of the operators shall yield `true` if the specified relationship is true and `false` if it is false.

7.6.11 Bitwise AND operator

[`expr.bit.and`]

and-expression:
equality-expression
and-expression & equality-expression

- ¹ The `&` operator groups left-to-right. The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4) are performed. Given the coefficients x_i and y_i of the base-2 representation (6.9.3) of the converted operands `x` and `y`, the coefficient r_i of the base-2 representation of the result `r` is 1 if both x_i and y_i are 1, and 0 otherwise.

[*Note 1*: The result is the bitwise AND function of the operands. — *end note*]

7.6.12 Bitwise exclusive OR operator

[`expr.xor`]

exclusive-or-expression:
and-expression
exclusive-or-expression ^ and-expression

- ¹ The `^` operator groups left-to-right. The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4) are performed. Given the coefficients x_i and y_i of the base-2 representation (6.9.3) of the converted operands `x` and `y`, the coefficient r_i of the base-2 representation of the result `r` is 1 if either (but not both) of x_i and y_i is 1, and 0 otherwise.

[*Note 1*: The result is the bitwise exclusive OR function of the operands. — *end note*]

7.6.13 Bitwise inclusive OR operator

[`expr.or`]

inclusive-or-expression:
exclusive-or-expression
inclusive-or-expression | exclusive-or-expression

- ¹ The `|` operator groups left-to-right. The operands shall be of integral or unscoped enumeration type. The usual arithmetic conversions (7.4) are performed. Given the coefficients x_i and y_i of the base-2 representation (6.9.3) of the converted operands `x` and `y`, the coefficient r_i of the base-2 representation of the result `r` is 1 if at least one of x_i and y_i is 1, and 0 otherwise.

[*Note 1*: The result is the bitwise inclusive OR function of the operands. — *end note*]

7.6.14 Logical AND operator

[`expr.log.and`]

logical-and-expression:
inclusive-or-expression
logical-and-expression && inclusive-or-expression

- ¹ The `&&` operator groups left-to-right. The operands are both contextually converted to `bool` (7.3). The result is `true` if both operands are `true` and `false` otherwise. Unlike `&`, `&&` guarantees left-to-right evaluation: the second operand is not evaluated if the first operand is `false`.
- ² The result is a `bool`. If the second expression is evaluated, the first expression is sequenced before the second expression (6.10.1).

7.6.15 Logical OR operator

[`expr.log.or`]

logical-or-expression:

logical-and-expression

logical-or-expression `||` *logical-and-expression*

- ¹ The `||` operator groups left-to-right. The operands are both contextually converted to `bool` (7.3). The result is `true` if either of its operands is `true`, and `false` otherwise. Unlike `|`, `||` guarantees left-to-right evaluation; moreover, the second operand is not evaluated if the first operand evaluates to `true`.
- ² The result is a `bool`. If the second expression is evaluated, the first expression is sequenced before the second expression (6.10.1).

7.6.16 Conditional operator

[`expr.cond`]

conditional-expression:

logical-or-expression

logical-or-expression `?` *expression* `:` *assignment-expression*

- ¹ Conditional expressions group right-to-left. The first expression is contextually converted to `bool` (7.3). It is evaluated and if it is `true`, the result of the conditional expression is the value of the second expression, otherwise that of the third expression. Only one of the second and third expressions is evaluated. The first expression is sequenced before the second or third expression (6.10.1).
- ² If either the second or the third operand has type `void`, one of the following shall hold:
 - (2.1) — The second or the third operand (but not both) is a (possibly parenthesized) *throw-expression* (7.6.18); the result is of the type and value category of the other. The *conditional-expression* is a bit-field if that operand is a bit-field.
 - (2.2) — Both the second and the third operands have type `void`; the result is of type `void` and is a prvalue.

[*Note 1:* This includes the case where both operands are *throw-expressions*. — *end note*]
- ³ Otherwise, if the second and third operand are glvalue bit-fields of the same value category and of types *cv1* T and *cv2* T, respectively, the operands are considered to be of type *cv* T for the remainder of this subclause, where *cv* is the union of *cv1* and *cv2*.
- ⁴ Otherwise, if the second and third operand have different types and either has (possibly cv-qualified) class type, or if both are glvalues of the same value category and the same type except for cv-qualification, an attempt is made to form an implicit conversion sequence (12.2.4.2) from each of those operands to the type of the other.

[*Note 2:* Properties such as access, whether an operand is a bit-field, or whether a conversion function is deleted are ignored for that determination. — *end note*]

Attempts are made to form an implicit conversion sequence from an operand expression E1 of type T1 to a target type related to the type T2 of the operand expression E2 as follows:

- (4.1) — If E2 is an lvalue, the target type is “lvalue reference to T2”, but an implicit conversion sequence can only be formed if the reference would bind directly (9.5.4) to a glvalue.
- (4.2) — If E2 is an xvalue, the target type is “rvalue reference to T2”, but an implicit conversion sequence can only be formed if the reference would bind directly.
- (4.3) — If E2 is a prvalue or if neither of the conversion sequences above can be formed and at least one of the operands has (possibly cv-qualified) class type:
 - (4.3.1) — if T1 and T2 are the same class type (ignoring cv-qualification):
 - (4.3.1.1) — if T2 is at least as cv-qualified as T1, the target type is T2,
 - (4.3.1.2) — otherwise, no conversion sequence is formed for this operand;
 - (4.3.2) — otherwise, if T2 is a base class of T1, the target type is *cv1* T2, where *cv1* denotes the cv-qualifiers of T1;

- (4.3.3) — otherwise, the target type is the type that E2 would have after applying the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions.

Using this process, it is determined whether an implicit conversion sequence can be formed from the second operand to the target type determined for the third operand, and vice versa, with the following outcome:

- (4.4) — If both sequences can be formed, or one can be formed but it is the ambiguous conversion sequence, the program is ill-formed.
- (4.5) — If no conversion sequence can be formed, the operands are left unchanged and further checking is performed as described below.
- (4.6) — Otherwise, if exactly one conversion sequence can be formed, that conversion is applied to the chosen operand and the converted operand is used in place of the original operand for the remainder of this subclause.

[*Note 3*: The conversion might be ill-formed even if an implicit conversion sequence could be formed. — *end note*]

- ⁵ If the second and third operands are glvalues of the same value category and have the same type, the result is of that type and value category and it is a bit-field if the second or the third operand is a bit-field, or if both are bit-fields.
- ⁶ Otherwise, the result is a prvalue. If the second and third operands do not have the same type, and either has (possibly cv-qualified) class type, overload resolution is used to determine the conversions (if any) to be applied to the operands (12.2.2.3, 12.5). If the overload resolution fails, the program is ill-formed. Otherwise, the conversions thus determined are applied, and the converted operands are used in place of the original operands for the remainder of this subclause.
- ⁷ Array-to-pointer (7.3.3) and function-to-pointer (7.3.4) standard conversions are performed on the second and third operands. After those conversions, one of the following shall hold:
- (7.1) — The second and third operands have the same type; the result is of that type and the result is copy-initialized using the selected operand.
- (7.2) — The second and third operands have arithmetic or enumeration type; the usual arithmetic conversions (7.4) are performed to bring them to a common type, and the result is of that type.
- (7.3) — One or both of the second and third operands have pointer type; lvalue-to-rvalue (7.3.2), pointer (7.3.12), function pointer (7.3.14), and qualification conversions (7.3.6) are performed to bring them to their composite pointer type (7.2.2). The result is of the composite pointer type.
- (7.4) — One or both of the second and third operands have pointer-to-member type; lvalue-to-rvalue (7.3.2), pointer to member (7.3.13), function pointer (7.3.14), and qualification conversions (7.3.6) are performed to bring them to their composite pointer type (7.2.2). The result is of the composite pointer type.
- (7.5) — Both the second and third operands have type `std::nullptr_t` or one has that type and the other is a null pointer constant. The result is of type `std::nullptr_t`.

7.6.17 Yielding a value

[**expr.yield**]

yield-expression:
 `co_yield assignment-expression`
 `co_yield braced-init-list`

- ¹ A *yield-expression* shall appear only within a suspension context of a function (7.6.2.4). Let *e* be the operand of the *yield-expression* and *p* be an lvalue naming the promise object of the enclosing coroutine (9.6.4), then the *yield-expression* is equivalent to the expression `co_await p.yield_value(e)`.

[*Example 1*:

```
template <typename T>
struct my_generator {
    struct promise_type {
        T current_value;
        /* ... */
        auto yield_value(T v) {
            current_value = std::move(v);
            return std::suspend_always{};
        }
    };
};
```

```

    struct iterator { /* ... */ };
    iterator begin();
    iterator end();
};

my_generator<pair<int,int>> g1() {
    for (int i = 0; i < 10; ++i) co_yield {i,i};
}
my_generator<pair<int,int>> g2() {
    for (int i = 0; i < 10; ++i) co_yield make_pair(i,i);
}

auto f(int x = co_yield 5);    // error: yield-expression outside of function suspension context
int a[] = { co_yield 1 };    // error: yield-expression outside of function suspension context

int main() {
    auto r1 = g1();
    auto r2 = g2();
    assert(std::equal(r1.begin(), r1.end(), r2.begin(), r2.end()));
}
— end example]

```

7.6.18 Throwing an exception

[expr.throw]

throw-expression:
throw assignment-expression_{opt}

- ¹ A *throw-expression* is of type `void`.
- ² A *throw-expression* with an operand throws an exception (14.2). The array-to-pointer (7.3.3) and function-to-pointer (7.3.4) standard conversions are performed on the operand. The type of the exception object is determined by removing any top-level *cv-qualifiers* from the type of the (possibly converted) operand. The exception object is copy-initialized (9.5.1) from the (possibly converted) operand.
- ³ A *throw-expression* with no operand rethrows the currently handled exception (14.4). If no exception is presently being handled, the function `std::terminate` is invoked (14.6.2). Otherwise, the exception is reactivated with the existing exception object; no new exception object is created. The exception is no longer considered to be caught.

[Example 1: An exception handler that cannot completely handle the exception itself can be written like this:

```

try {
    // ...
} catch (...) {    // catch all exceptions
    // respond (partially) to exception
    throw;          // pass the exception to some other handler
}
— end example]

```

7.6.19 Assignment and compound assignment operators

[expr.assign]

- ¹ The assignment operator (`=`) and the compound assignment operators all group right-to-left. All require a modifiable lvalue as their left operand; their result is an lvalue of the type of the left operand, referring to the left operand. The result in all cases is a bit-field if the left operand is a bit-field. In all cases, the assignment is sequenced after the value computation of the right and left operands, and before the value computation of the assignment expression. The right operand is sequenced before the left operand. With respect to an indeterminately-sequenced function call, the operation of a compound assignment is a single evaluation.

[Note 1: Therefore, a function call cannot intervene between the lvalue-to-rvalue conversion and the side effect associated with any single compound assignment operator. — end note]

assignment-expression:
conditional-expression
yield-expression
throw-expression
logical-or-expression assignment-operator initializer-clause

assignment-operator: one of

= *= /= %= += -= >>= <<= &= ^= |=

- 2 In simple assignment (=), let *V* be the result of the right operand; the object referred to by the left operand is modified (3.1) by replacing its value with
 - (2.1) — the value congruent (6.9.3) to *V* if the object is of integer type,
 - (2.2) — the result of `reinterpret_cast<T>(reinterpret_cast<std::uintptr_t>(V))` if the left operand is a volatile-qualified glvalue of pointer type *T* and *V* is a pointer value that is not valid in the context of the evaluation (6.9.5), and
 - (2.3) — *V* otherwise.
- 3 If the right operand is an expression, it is implicitly converted (7.3) to the cv-unqualified type of the left operand.
- 4 When the left operand of an assignment operator is a bit-field that cannot represent the value of the expression, the resulting value of the bit-field is implementation-defined.
- 5 An assignment whose left operand is of a volatile-qualified type is deprecated (D.4) unless the (possibly parenthesized) assignment is a discarded-value expression or an unevaluated operand (7.2.3).
- 6 The behavior of an expression of the form *E1 op* = *E2* is equivalent to *E1* = *E1 op* *E2* except that *E1* is evaluated only once.

[*Note 2*: The object designated by *E1* is accessed twice. — *end note*]

For += and -=, *E1* shall either have arithmetic type or be a pointer to a possibly cv-qualified completely-defined object type. In all other cases, *E1* shall have arithmetic type.
- 7 If the value being stored in an object is read via another object that overlaps in any way the storage of the first object, then the overlap shall be exact and the two objects shall have the same type, otherwise the behavior is undefined (F.3.34).

[*Note 3*: This restriction applies to the relationship between the left and right sides of the assignment operation; it is not a statement about how the target of the assignment can be aliased in general. See 7.2.1. — *end note*]
- 8 A *braced-init-list* *B* may appear on the right-hand side of
 - (8.1) — an assignment to a scalar of type *T*, in which case *B* shall have at most a single element. The meaning of *x* = *B* is *x* = *t*, where *t* is an invented temporary variable declared and initialized as *T t* = *B*.
 - (8.2) — an assignment to an object of class type, in which case *B* is passed as the argument to the assignment operator function selected by overload resolution (12.4.3.2, 12.2).

[*Example 1*:

```
complex<double> z;
z = { 1, 2 };           // meaning z.operator=({1,2})
z += { 1, 2 };          // meaning z.operator+=({1,2})
int a, b;
a = b = { 1 };          // meaning a=b=1;
a = { 1 } = b;          // syntax error
```

— *end example*]

7.6.20 Comma operator

[*expr.comma*]

- 1 The comma operator groups left-to-right.

expression:

assignment-expression

expression , *assignment-expression*

A pair of expressions separated by a comma is evaluated left-to-right; the left expression is a discarded-value expression (7.2). The left expression is sequenced before the right expression (6.10.1). The type and value of the result are the type and value of the right operand; the result is of the same value category as its right operand, and is a bit-field if its right operand is a bit-field.

- 2 [*Note 1*: In contexts where the comma token is given special meaning (e.g., function calls (7.6.1.3), subscript expressions (7.6.1.2), lists of initializers (9.5), or *template-argument-lists* (13.3)), the comma operator as described in this subclause can appear only in parentheses.

[Example 1:

```
f(a, (t=3, t+2), c);
```

has three arguments, the second of which has the value 5. — end example]

— end note]

7.7 Constant evaluation

[expr.const]

7.7.1 General

[expr.const.general]

constant-expression:

conditional-expression

- ¹ Certain contexts require expressions that satisfy additional requirements as detailed in 7.7; other contexts have different semantics depending on whether or not an expression satisfies these requirements. Expressions that satisfy these requirements, assuming that copy elision (11.9.6) is not performed, are called constant expressions.

[Note 1: Certain constant expressions are evaluated during translation (5.2). — end note]

- ² *Recommended practice:* Implementations should provide consistent results of floating-point evaluations, irrespective of whether the evaluation is performed during translation or during program execution.

[Note 2: Since this document imposes no restrictions on the accuracy of floating-point operations, it is unspecified whether the evaluation of a floating-point expression during translation yields the same result as the evaluation of the same expression (or the same operations on the same values) during program execution.

[Example 1:

```
bool f() {
    char array[1 + int(1 + 0.2 - 0.1 - 0.1)]; // Must be evaluated during translation
    int size = 1 + int(1 + 0.2 - 0.1 - 0.1); // May be evaluated at runtime
    return sizeof(array) == size;
}
```

It is unspecified whether the value of `f()` will be `true` or `false`. — end example]

— end note]

7.7.2 Core constant expressions

[expr.const.core]

- ¹ A type has *constexpr-unknown* representation if it

- (1.1) — is a union,
- (1.2) — is `std::meta::type_info`,
- (1.3) — is a pointer or pointer-to-member type,
- (1.4) — is volatile-qualified,
- (1.5) — is a class type with a non-static data member of reference type, or
- (1.6) — has a base class or a non-static member whose type has *constexpr-unknown* representation.

- ² An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine (6.10.1), would evaluate one of the following:

- (2.1) — `this` (7.5.3), except
 - (2.1.1) — in a *constexpr* function (9.2.6) that is being evaluated as part of *E* or
 - (2.1.2) — when appearing as the *postfix-expression* of an implicit or explicit class member access expression (7.6.1.5);
- (2.2) — a control flow that passes through a declaration of a block variable (6.4.3) with static (6.8.6.2) or thread (6.8.6.3) storage duration, unless that variable is usable in constant expressions;

[Example 1:

```
constexpr char test() {
    static const int x = 5;
    static constexpr char c[] = "Hello World";
    return *(c + x);
}
static_assert(' ' == test());
```

— *end example*]

- (2.3) — an invocation of a non-constexpr function;⁶¹
- (2.4) — an invocation of an undefined constexpr function;
- (2.5) — an invocation of an instantiated constexpr function that is not constexpr-suitable;
- (2.6) — an invocation of a virtual function (11.7.3) for an object whose dynamic type is constexpr-unknown;
- (2.7) — an expression that would exceed the implementation-defined limits (see Annex B);
- (2.8) — an operation that would have undefined or erroneous behavior as specified in Clause 4 through Clause 15;⁶²
- (2.9) — an arithmetic expression (7.1) where
 - (2.9.1) — all operands are finite and the result is not finite,
 - (2.9.2) — any operand is a signaling NaN, or
 - (2.9.3) — no operand is NaN and the result is NaN;
- (2.10) — an lvalue-to-rvalue conversion (7.3.2) unless it is applied to
 - (2.10.1) — a glvalue of type `cv std::nullptr_t`,
 - (2.10.2) — a non-volatile glvalue that refers to an object that is usable in constant expressions, or
 - (2.10.3) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of *E*;
- (2.11) — an lvalue-to-rvalue conversion that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;
- (2.12) — an lvalue-to-rvalue conversion that is applied to an object with an indeterminate value (6.8.5);
- (2.13) — an invocation of an implicitly-defined copy/move constructor or copy/move assignment operator for a union whose active member (if any) is mutable, unless the lifetime of the union object began within the evaluation of *E*;
- (2.14) — in a *lambda-expression*, a reference to **this** or to a variable with automatic storage duration defined outside that *lambda-expression*, where the reference would be an odr-use (6.3, 7.5.6);

[*Example 2*:

```
void g() {
    const int n = 0;
    [=] {
        constexpr int i = n;           // OK, n is not odr-used here
        constexpr int j = *&n;         // error: &n would be an odr-use of n
    };
}
```

— *end example*]

[*Note 1*: If the odr-use occurs in an invocation of a function call operator of a closure type, it no longer refers to **this** or to an enclosing variable with automatic storage duration due to the transformation (7.5.6.3) of the *id-expression* into an access of the corresponding data member.

[*Example 3*:

```
auto monad = [](auto v) { return [=] { return v; }; };
auto bind = [](auto m) {
    return [=](auto fvm) { return fvm(m()); };
};

// OK to capture objects with automatic storage duration created during constant expression evaluation.
static_assert(bind(monad(2))(monad()) == monad(2));
```

— *end example*]

— *end note*]

⁶¹) Overload resolution (12.2) is applied as usual.

⁶²) This includes, for example, signed integer overflow (7.1), certain pointer arithmetic (7.6.6), division by zero (7.6.5), or certain shift operations (7.6.7).

- (2.15) — a conversion from a prvalue P of type “pointer to *cv* void” to a type “*cv1* pointer to T”, where T is not *cv2* void, unless P is a null pointer value or points to an object whose type is similar to T;
- (2.16) — a `reinterpret_cast` (7.6.1.10);
- (2.17) — pointer arithmetic (7.6.6) where one (possibly converted) operand points to the first element of an array of unknown bound and the other (possibly converted) operand is of integral type with non-zero value;
- (2.18) — a modification of an object (7.6.19, 7.6.1.6, 7.6.2.3) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of *E*;
- (2.19) — an invocation of a destructor (11.4.7) or a function call whose *postfix-expression* names a pseudo-destructor (7.6.1.3), in either case for an object whose lifetime did not begin within the evaluation of *E*;
- (2.20) — a *new-expression* (7.6.2.8), unless either
 - (2.20.1) — the selected allocation function is a replaceable global allocation function (17.6.3.2, 17.6.3.3) and the allocated storage is deallocated within the evaluation of *E*, or
 - (2.20.2) — the selected allocation function is a non-allocating form (17.6.3.4) with an allocated type T, where
 - (2.20.2.1) — the placement argument to the *new-expression* points to an object that is transparently replaceable (6.8.4) by the object created by the *new-expression* or, if T is an array type, to the first element of such an object, and
 - (2.20.2.2) — the placement argument points to storage whose duration began within the evaluation of *E*;
- (2.21) — a *delete-expression* (7.6.2.9), unless it deallocates a region of storage allocated within the evaluation of *E*;
- (2.22) — a call to an instance of `std::allocator<T>::allocate` (20.2.10.2), unless the allocated storage is deallocated within the evaluation of *E*;
- (2.23) — a call to an instance of `std::allocator<T>::deallocate` (20.2.10.2), unless it deallocates a region of storage allocated within the evaluation of *E*;
- (2.24) — a construction of an exception object, unless the exception object and all of its implicit copies created by invocations of `std::current_exception` or `std::rethrow_exception` (17.9.7) are destroyed within the evaluation of *E*;
- (2.25) — a *throw-expression* (7.6.18) with no operand, unless there is a currently handled exception whose exception object was constructed within the evaluation of *E*;
- (2.26) — an *await-expression* (7.6.2.4);
- (2.27) — a *yield-expression* (7.6.17);
- (2.28) — a three-way comparison (7.6.8), relational (7.6.9), or equality (7.6.10) operator where the result is unspecified;
- (2.29) — an equality operator comparing pointers to potentially non-unique objects, if the pointer values of the two operands are associated with different evaluations (6.9.5) and either they can both point to the same offset within the same potentially non-unique object or one of them points to an object whose type has `constexpr-unknown` representation;

[Example 4:

```
constexpr const char *f() { return "foo"; }

constexpr bool b1 = +"foo" == "foo";           // error: non-constant
constexpr bool b2 = f() == f();                 // error: non-constant
constexpr const char *p = f();
constexpr bool b3 = p == p;                     // OK, value of b3 is true
constexpr bool b4 = "xfoo" + 1 == "foo\0y";     // error: non-constant; string literal
                                                // object could contain "xfoo\0y"
constexpr bool b5 = "foo" == "bar" + 0;         // OK, value of b5 is false
constexpr bool b6 = (const char*)"foo" == "oo"; // OK, value of b6 is false; offsets would
                                                // be different in a merged string literal object

constexpr std::initializer_list<int *> il1 = { (int *)nullptr };
constexpr std::initializer_list<unsigned long> il2 = { 0 };
constexpr bool b7 = il1.begin() == (void *)il2.begin(); // error: non-constant
```

— *end example*]

- (2.30) — a `dynamic_cast` (7.6.1.7) or `typeid` (7.6.1.8) expression on a glvalue that refers to an object whose dynamic type is `constexpr-unknown`;
- (2.31) — a `dynamic_cast` (7.6.1.7) expression, `typeid` (7.6.1.8) expression, or `new-expression` (7.6.2.8) that would throw an exception where no definition of the exception type is reachable;
- (2.32) — an expression that would produce an injected declaration (see below), unless E is the corresponding expression of a *constexpr-block-declaration* (9.1);
- (2.33) — an *asm-declaration* (9.11);
- (2.34) — an invocation of the `va_arg` macro (17.14.2);
- (2.35) — a non-constant library call (3.35); or
- (2.36) — a `goto` statement (8.8.6).

[*Note 2:* A `goto` statement introduced by equivalence (Clause 8) is not in scope. For example, a `while` statement (8.6.2) can be executed during constant evaluation. — *end note*]

- ³ It is implementation-defined whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E has runtime-undefined behavior.
- ⁴ It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate
- (4.1) — an operation that has undefined behavior as specified in Clause 16 through Clause 33 or
- (4.2) — an invocation of the `va_start` macro (17.14.2).

⁵ [*Example 5:*

```

int x;                                // not constant
struct A {
    constexpr A(bool b) : m(b?42:x) { }
    int m;
};
constexpr int v = A(true).m;           // OK, constructor call initializes m with the value 42

constexpr int w = A(false).m;          // error: initializer for m is x, which is non-constant

constexpr int f1(int k) {
    constexpr int x = k;               // error: x is not initialized by a constant expression
                                        // because lifetime of k began outside the initializer of x

    return x;
}
constexpr int f2(int k) {
    int x = k;                         // OK, not required to be a constant expression
                                        // because x is not constexpr

    return x;
}

constexpr int incr(int &n) {
    return ++n;
}

constexpr int g(int k) {
    constexpr int x = incr(k);          // error: incr(k) is not a core constant expression
                                        // because lifetime of k began outside the expression incr(k)

    return x;
}
constexpr int h(int k) {
    int x = incr(k);                   // OK, incr(k) is not required to be a core constant expression
    return x;
}
constexpr int y = h(1);                // OK, initializes y with the value 2
                                        // h(1) is a core constant expression because
                                        // the lifetime of k begins inside h(1)

```

— *end example*]

⁶ For the purposes of determining whether an expression E is a core constant expression, the evaluation of the body of a member function of `std::allocator<T>` as defined in 20.2.10.2, is ignored.

⁷ For the purposes of determining whether E is a core constant expression, the evaluation of a call to a trivial copy/move constructor or copy/move assignment operator of a union is considered to copy/move the active member of the union, if any.

[Note 3: The copy/move of the active member is trivial. — end note]

⁸ For the purposes of determining whether E is a core constant expression, the evaluation of an *id-expression* that names a structured binding v (9.7) has the following semantics:

- (8.1) — If v is an lvalue referring to the object bound to an invented reference r , the behavior is as if r were nominated.
- (8.2) — Otherwise, if v names an array element or class member, the behavior is that of evaluating $e[i]$ or $e.m$, respectively, where e is the name of the variable initialized from the initializer of the structured binding declaration, and i is the index of the element referred to by v or m is the name of the member referred to by v , respectively.

[Example 6:

```
#include <tuple>
struct S {
    mutable int m;
    constexpr S(int m): m(m) {}
    virtual int g() const;
};
void f(std::tuple<S&> t) {
    auto [r] = t;
    static_assert(r.g() >= 0);           // error: dynamic type is constexpr-unknown
    constexpr auto [m] = S(1);
    static_assert(m == 1);               // error: lvalue-to-rvalue conversion on mutable
                                         // subobject e.m, where e is a constexpr object of type S

    using A = int[2];
    constexpr auto [v0, v1] = A{2, 3};
    static_assert(v0 + v1 == 5);         // OK, equivalent to e[0] + e[1] where e is a constexpr array
}
```

— end example]

⁹ During the evaluation of an expression E as a core constant expression, all *id-expressions*, *splice-expressions*, and uses of `*this` that refer to an object or reference whose lifetime did not begin within the evaluation of E are treated as referring to a specific instance of that object or reference whose lifetime and that of all subobjects (including all union members) includes the entire constant evaluation. For such an object that is not usable in constant expressions, the dynamic type of the object is *constexpr-unknown*. For such a reference that is not usable in constant expressions, the reference is treated as binding to an unspecified object of the referenced type whose lifetime and that of all subobjects includes the entire constant evaluation and whose dynamic type is *constexpr-unknown*.

[Example 7:

```
template <typename T, size_t N>
constexpr size_t array_size(T (&)[N]) {
    return N;
}

void use_array(int const (&gold_medal_mel)[2]) {
    constexpr auto gold = array_size(gold_medal_mel); // OK
}

constexpr auto olympic_mile() {
    const int ledecky = 1500;
    return []{ return ledecky; };
}
static_assert(olympic_mile()() == 1500); // OK

struct Swim {
    constexpr int phelps() { return 28; }
}
```

```

    virtual constexpr int lochte() { return 12; }
    int coughlin = 12;
};

constexpr int how_many(Swim& swam) {
    Swim* p = &swam;
    return (p + 1 - 1)->phelps();
}

void splash(Swim& swam) {
    static_assert(swam.phelps() == 28);           // OK
    static_assert((&swam)->phelps() == 28);      // OK

    Swim* pswam = &swam;
    static_assert(pswam->phelps() == 28);         // error: lvalue-to-rvalue conversion on a pointer
                                                    // not usable in constant expressions

    static_assert(how_many(swam) == 28);         // OK
    static_assert(Swim().lochte() == 12);        // OK

    static_assert(swam.lochte() == 12);          // error: invoking virtual function on reference
                                                    // with constexpr-unknown dynamic type

    static_assert(swam.coughlin == 12);          // error: lvalue-to-rvalue conversion on an object
                                                    // not usable in constant expressions
}

extern Swim dc;
extern Swim& trident;

constexpr auto& sandeno = typeid(dc);           // OK, can only be typeid(Swim)
constexpr auto& gallagher = typeid(trident);    // error: constexpr-unknown dynamic type
— end example]

```

7.7.3 Constant expressions

[expr.const.const]

- ¹ A *consteval-only* value is either
- (1.1) — a reflection value (6.9.3) that is not the null reflection value or
 - (1.2) — a pointer or pointer-to-member that points to an immediate function or to or past the end of an immediate object.
- ² An object is an *immediate object* if its complete object has
- (2.1) — a constituent value that is consteval-only or
 - (2.2) — a constituent reference that refers to an immediate object or immediate function.
- ³ Every immediate object shall be
- (3.1) — the object associated with a constexpr variable or a subobject thereof,
 - (3.2) — a template parameter object (13.2) or a subject thereof, or
 - (3.3) — an object whose lifetime begins and ends during the evaluation of a core constant expression.

[Example 1:

```

consteval int plus1(int x) { return x + 1; }
template <auto V> struct C {};

auto a = plus1;           // error: immediate object not associated with constexpr variable
constexpr auto b = plus1; // OK
auto c = C<plus1>();       // OK

auto d = ^^int;           // error: immediate object not associated with constexpr variable
auto e = C<^^char>();      // OK
— end example]

```

Each expression *E* that odr-uses a variable that declares or refers to an immediate object shall be in an immediate function context; a diagnostic is required only if either

- (3.4) — the innermost declaration that contains *E* or
- (3.5) — the defining declaration of the variable

is reachable from the other.

- 4 A core constant expression is a *constant expression* unless it is a prvalue whose result object (7.2.1) has a constituent value that is an invalid pointer value (6.9.5) or is indeterminate or erroneous (6.8.5)
- 5 A constant expression is an *immediate constant expression* if it is either
 - (5.1) — a glvalue that refers to an immediate object or an immediate function, or
 - (5.2) — a prvalue whose result object (if any) is an immediate object.

[*Note 1:* A glvalue core constant expression that either refers to or points to an unspecified object is not a constant expression. — *end note*]

[*Example 2:*

```
consteval int f() { return 42; }
consteval auto g() { return f; }
consteval int h(int (*p)() = g()) { return p(); }
constexpr int r = h();           // OK
constexpr auto e = g();          // error: a pointer to an immediate function is
                                  // not a permitted result of a constant expression
```

```
struct S {
    int x;
    constexpr S() {}
};
int i() {
    constexpr S s;                // error: s.x has erroneous value
}
```

— *end example*]

- 6 An *integral constant expression* is an expression of integral or unscoped enumeration type, implicitly converted to a prvalue, where the converted expression is a core constant expression.

[*Note 2:* Such expressions can be used as bit-field lengths (11.4.10), as enumerator initializers if the underlying type is not fixed (9.8.1), and as alignments (9.13.2). — *end note*]

- 7 If an expression of literal class type is used in a context where an integral constant expression is required, then that expression is contextually implicitly converted (7.3) to an integral or unscoped enumeration type and the selected conversion function shall be `constexpr`.

[*Example 3:*

```
struct A {
    constexpr A(int i) : val(i) { }
    constexpr operator int() const { return val; }
    constexpr operator long() const { return 42; }
private:
    int val;
};
constexpr A a = alignof(int);
alignas(a) int n;           // error: ambiguous conversion
struct B { int n : a; };    // error: ambiguous conversion
```

— *end example*]

- 8 A *converted constant expression* of type *T* is an expression, implicitly converted to type *T*, where the converted expression is a constant expression and the implicit conversion sequence contains only
 - (8.1) — user-defined conversions,
 - (8.2) — lvalue-to-rvalue conversions (7.3.2),
 - (8.3) — array-to-pointer conversions (7.3.3),
 - (8.4) — function-to-pointer conversions (7.3.4),

- (8.5) — qualification conversions (7.3.6),
- (8.6) — integral promotions (7.3.7),
- (8.7) — integral conversions (7.3.9) other than narrowing conversions (9.5.5),
- (8.8) — floating-point promotions (7.3.8),
- (8.9) — floating-point conversions (7.3.10) where the source value can be represented exactly in the destination type,
- (8.10) — null pointer conversions (7.3.12) from `std::nullptr_t`,
- (8.11) — null member pointer conversions (7.3.13) from `std::nullptr_t`, and
- (8.12) — function pointer conversions (7.3.14),

and where the reference binding (if any) binds directly.

[Note 3: Such expressions can be used in **new** expressions (7.6.2.8), as case expressions (8.5.3), as enumerator initializers if the underlying type is fixed (9.8.1), as array bounds (9.3.4.5), as constant template arguments (13.4), and as the constant expression of a *splice-specifier* (6.6). — end note]

- ⁹ A *contextually converted constant expression of type `bool`* is an expression, contextually converted to `bool` (7.3), where the converted expression is a constant expression and the conversion sequence contains only the conversions above.

7.7.4 Constant initialization

[**expr.const.init**]

- ¹ The constituent values and constituent references of a variable **x** are defined as follows:

- (1.1) — If **x** declares an object, the constituent values and references of that object (6.8.2) are constituent values and references of **x**.
- (1.2) — If **x** declares a reference, that reference is a constituent reference of **x**.

For any constituent reference **r** of a variable **x**, if **r** is bound to a temporary object or subobject thereof whose lifetime is extended to that of **r**, the constituent values and references of that temporary object are also constituent values and references of **x**, recursively.

- ² An object *o* is *constexpr-referenceable* from a point *P* if

- (2.1) — *o* has static storage duration, or
- (2.2) — *o* has automatic storage duration, and, letting **v** denote
 - (2.2.1) — the variable corresponding to *o*'s complete object or
 - (2.2.2) — the variable to whose lifetime that of *o* is extended,
 the smallest scope enclosing **v** and the smallest scope enclosing *P* that are neither
 - (2.2.3) — block scopes nor
 - (2.2.4) — function parameter scopes associated with a *requirement-parameter-list*
 are the same function parameter scope.

[Example 1:

```
struct A {
    int m;
    const int& r;
};
void f() {
    static int sx;
    thread_local int tx;           // tx is never constexpr-referenceable
    int ax;
    A aa = {1, 2};
    static A sa = {3, 4};
    // The objects sx, ax, and aa.m, sa.m, and the temporaries to which aa.r and sa.r are bound, are constexpr-referenceable.
    auto lambda = [] {
        int ay;
```

```

    // The objects sx, sa.m, and ay (but not ax or aa), and the
    // temporary to which sa.r is bound, are constexpr-referenceable.
};
}

```

— end example]

- 3 An object or reference **x** is *constexpr-representable* at a point *P* if, for each constituent value of **x** that points to or past an object *o*, and for each constituent reference of **x** that refers to an object *o*, *o* is *constexpr-referenceable* from *P*.
- 4 A variable **v** is *constant-initializable* if
 - (4.1) — the full-expression of its initialization is a constant expression when interpreted as a *constant-expression* with all contract assertions using the ignore evaluation semantic (6.11.2),

[Note 1: In the course of this determination, `std::is_constant_evaluated()` (21.3.11) has the value `true`. — end note]

[Note 2: Furthermore, if the initialization is manifestly constant-evaluated (7.7.7), its evaluation during translation can still evaluate contract assertions with other evaluation semantics, resulting in a diagnostic or ill-formed program if a contract violation occurs. — end note]
 - (4.2) — immediately after the initializing declaration of **v**, the object or reference **x** declared by **v** is *constexpr-representable*, and
 - (4.3) — if **x** has static or thread storage duration, **x** is *constexpr-representable* at the nearest point whose immediate scope is a namespace scope that follows the initializing declaration of **v**.
- 5 A constant-initializable variable is *constant-initialized* if either it has an initializer or its type is *const-default-constructible* (9.5.1).

[Example 2:

```

void f() {
    int ax = 0;           // ax is constant-initialized
    thread_local int tx = 0; // tx is constant-initialized
    static int sx;        // sx is not constant-initialized
    static int& rss = sx;  // rss is constant-initialized
    static int& rst = tx;  // rst is not constant-initialized
    static int& rsa = ax;  // rsa is not constant-initialized
    thread_local int& rts = sx; // rts is constant-initialized
    thread_local int& rtt = tx; // rtt is not constant-initialized
    thread_local int& rta = ax; // rta is not constant-initialized
    int& ras = sx;         // ras is constant-initialized
    int& rat = tx;         // rat is not constant-initialized
    int& raa = ax;         // raa is constant-initialized
}

```

— end example]

- 6 A variable is *potentially-constant* if it is *constexpr* or it has reference or non-volatile *const*-qualified integral or enumeration type.
- 7 A variable *V* is *usable in constant expressions* at a point *P* if *V* is *constant-initialized* and *potentially-constant*, *V*'s initializing declaration *D* is *reachable* from *P*, and
 - (7.1) — *V* is *constexpr*,
 - (7.2) — *V* is not initialized to a TU-local value, or
 - (7.3) — *P* is in the same translation unit as *D*.
- 8 An object or reference is *potentially usable in constant expressions* at point *P* if it is
 - (8.1) — the object or reference declared by a variable that is *usable in constant expressions* at *P*,
 - (8.2) — a temporary object of non-volatile *const*-qualified literal type whose lifetime is extended (6.8.7) to that of a variable that is *usable in constant expressions* at *P*,
 - (8.3) — a template parameter object (13.2),
 - (8.4) — a string literal object (5.13.5),
 - (8.5) — a non-mutable subobject of any of the above, or

(8.6) — a reference member of any of the above.

- ⁹ An object or reference is *usable in constant expressions* at point *P* if it is an object or reference that is potentially usable in constant expressions at *P* and is constexpr-representable at *P*.

[Example 3:

```
struct A {
    int* const & r;
};
void f(int x) {
    constexpr A a = {&x};
    static_assert(a.r == &x);           // OK
    [&] {
        static_assert(a.r != nullptr); // error: a.r is not usable in constant expressions at this point
    }();
}
```

— end example]

7.7.5 Immediate functions

[expr.const.imm]

- ¹ An expression or conversion is in an *immediate function context* if it is potentially evaluated and either:

- (1.1) — its innermost enclosing non-block scope is a function parameter scope of an immediate function,
- (1.2) — it is a subexpression of a manifestly constant-evaluated expression or conversion (7.7.7), or
- (1.3) — its enclosing statement is enclosed (8.1) by the *compound-statement* of a consteval if statement (8.5.2).

An invocation is an *immediate invocation* if it is a potentially evaluated explicit or implicit invocation of an immediate function and is not in an immediate function context. An aggregate initialization is an immediate invocation if it evaluates a default member initializer that has a subexpression that is an immediate-escalating expression.

- ² A potentially evaluated expression or conversion is *immediate-escalating* if it is neither initially in an immediate function context nor a subexpression of an immediate invocation, and

- (2.1) — it is an immediate constant expression or

[Example 1:

```
consteval std::meta::info refl() { return ~int; }
template <auto F> constexpr void ex() { auto x = F(); }
auto p = &ex<refl>;           // error: immediate function because F()
                             // is an immediate constant expression
```

— end example]

- (2.2) — it is an immediate invocation that is not a constant expression.

[Example 2:

```
consteval int id(int x) { return x; }
template <auto F> constexpr auto apply_to(int i) { return F(i); }
auto p = &apply_to<id>;       // error: immediate function because F(i)
                             // is not a constant expression
```

— end example]

- ³ An *immediate-escalating* function is

- (3.1) — the call operator of a lambda that is not declared with the **consteval** specifier,
- (3.2) — a non-user-provided defaulted function that is not declared with the **consteval** specifier, or
- (3.3) — a function that is not a prospective destructor and that results from the instantiation of a templated entity defined with the **constexpr** specifier.

An immediate-escalating expression shall appear only in an immediate-escalating function.

- ⁴ An *immediate function* is a function that is either

- (4.1) — declared with the **consteval** specifier, or
- (4.2) — an immediate-escalating function *F* whose function body contains an immediate-escalating expression whose innermost enclosing non-block scope is *F*'s function parameter scope.

[*Note 1*: Default member initializers used to initialize a base or member subobject (11.9.3) are considered to be part of the function body (9.6.1). — *end note*]

⁵ [Example 3:

```

constexpr int id(int i) { return i; }
constexpr char id(char c) { return c; }

template<class T>
constexpr int f(T t) {
    return t + id(t);
}

auto a = &f<char>;           // OK, f<char> is not an immediate function
auto b = &f<int>;            // error: f<int> is an immediate function

static_assert(f(3) == 6);    // OK

template<class T>
constexpr int g(T t) {      // g<int> is not an immediate function
    return t + id(42);      // because id(42) is already a constant
}

template<class T, class F>
constexpr bool is_not(T t, F f) {
    return not f(t);
}

constexpr bool is_even(int i) { return i % 2 == 0; }

static_assert(is_not(5, is_even));    // OK

int x = 0;

template<class T>
constexpr T h(T t = id(x)) {         // h<int> is not an immediate function
                                     // id(x) is not evaluated when parsing the default argument (9.3.4.7, 13.9.2)
    return t;
}

template<class T>
constexpr T hh() {                   // hh<int> is an immediate function because of the invocation
    return h<T>();                   // of the immediate function id in the default argument of h<int>
}

int i = hh<int>();                    // error: hh<int>() is an immediate-escalating expression
                                     // outside of an immediate-escalating function

struct A {
    int x;
    int y = id(x);
};

template<class T>
constexpr int k(int) {               // k<int> is not an immediate function because A(42) is a
    return A(42).y;                  // constant expression and thus not immediate-escalating
}

constexpr int l(int c) pre(c >= 2) {
    return (c % 2 == 0) ? c / 0 : c;
}

const int i0 = l(0);                 // dynamic initialization; contract violation or undefined behavior
const int i1 = l(1);                 // static initialization; value of 1 or contract violation at compile time
const int i2 = l(2);                 // dynamic initialization; undefined behavior

```



```
const int i3 = 1(3);    // static initialization; value of 3
— end example]
```

7.7.6 Reflection

[expr.const.reflect]

- ¹ The evaluation of an expression can introduce one or more *injected declarations*. The evaluation is said to *produce* the declarations.

[Note 1: An invocation of the library function template `std::meta::define_aggregate` produces an injected declaration (21.4.17). — end note]

Each such declaration has

- (1.1) — an associated *synthesized point*, which follows the last non-synthesized program point in the translation unit containing that declaration, and
- (1.2) — an associated *characteristic sequence* of values.

[Note 2: Special rules concerning reachability apply to synthesized points (10.7). — end note]

[Note 3: The program is ill-formed if injected declarations with different characteristic sequences define the same entity in different translation units (6.3). — end note]

- ² A member of an entity defined by an injected declaration shall not have a name reserved to the implementation (5.11); no diagnostic is required.
- ³ Let *C* be a *consteval-block-declaration*, the evaluation of whose corresponding expression produces an injected declaration for an entity *E*. The program is ill-formed if either
 - (3.1) — *C* is enclosed by a scope associated with *E* or
 - (3.2) — letting *P* be a point whose immediate scope is that to which *E* belongs, there is a function parameter scope or class scope that encloses exactly one of *C* or *P*.

- ⁴ [Example 1:

```
struct S0 {
    consteval {
        std::meta::define_aggregate(^^S0, {}); // error: scope associated with S0 encloses the consteval block
    }
};

struct S1;
consteval { std::meta::define_aggregate(^^S1, {}); } // OK

template <std::meta::info R> consteval void tfn1() {
    std::meta::define_aggregate(R, {});
}

struct S2;
consteval { tfn1(^^S2>()); } // OK

template <std::meta::info R> consteval void tfn2() {
    consteval { std::meta::define_aggregate(R, {}); }
}

struct S3;
consteval { tfn2(^^S3>()); }
// error: function parameter scope of tfn2<^^S3> intervenes between the declaration of S3
// and the consteval block that produces the injected declaration

template <typename> struct TCls {
    struct S4;
    static void sfm() requires ([]) {
        consteval { std::meta::define_aggregate(^^S4, {}); }
        return true;
    }() { }
};

consteval { TCls<void>::sfm(); } // error: TCls<void>::S4 is not enclosed by requires-clause lambda
```

```

struct S5;
struct Cls {
    consteval { std::meta::define_aggregate(^^S5, {}); } // error: S5 is not enclosed by class Cls
};

struct S6;
consteval {
    struct S7; // #1
               // local class

    std::meta::define_aggregate(^^S7, {}); // error: consteval block #1 does not enclose itself,
                                           // but encloses S7

    struct S8; // local class
    consteval { // #2
        std::meta::define_aggregate(^^S6, {}); // error: consteval block #1 encloses
                                                // consteval block #2 but not S6

        std::meta::define_aggregate(^^S8, {}); // OK, consteval block #1 encloses both #2 and S8
    }
}

```

— end example]

- ⁵ During an evaluation V (6.10.1) of an expression, conversion, or initialization E as a core constant expression, the *point of evaluation of E during V* is the program point P determined as follows:

- (5.1) — If E is a potentially-evaluated subexpression of a default member initializer I , and V is the evaluation of E in the evaluation of I as an immediate subexpression of a (possibly aggregate) initialization, then P is the point of evaluation of that initialization.

[Note 4: For example, E can be an immediate invocation in a default member initializer used by an aggregate initialization appearing at P . — end note]

- (5.2) — Otherwise, if E is a potentially-evaluated subexpression of a default argument A (9.3.4.7), and V is the evaluation of E in the evaluation of A as an immediate subexpression of a function call (7.6.1.3), then P is the point of evaluation of that function call.
- (5.3) — Otherwise, P is the point at which E appears.

During the evaluation V of an expression E as a core constant expression, the *evaluation context* of an evaluation X during V is the set C of program points determined as follows:

- (5.4) — If X occurs during the evaluation Y of a manifestly constant-evaluated expression, where Y occurs during V , then C is the evaluation context of X during Y .
- (5.5) — Otherwise, C contains
- (5.5.1) — the point of evaluation of E during V and each synthesized point in the instantiation context thereof and
- (5.5.2) — each synthesized point corresponding to an injected declaration produced by any evaluation executed during V that is sequenced before X (6.10.1).

[Note 5: The evaluation context determines the behavior of certain functions used for reflection (21.4). — end note]

[Example 2:

```

struct S;
consteval std::size_t f(int p) {
    constexpr std::size_t r = /* Q */
        std::meta::is_complete_type(^^S) ? 1 : 2; // #1
    if (!std::meta::is_complete_type(^^S)) { // #2
        std::meta::define_aggregate(^^S, {});
    }
    return (p > 0) ? f(p - 1) : r;
}

consteval {
    if (f(1) != 2) {
        throw; // OK, not evaluated
    }
}

```

```

    }
}

```

During each evaluation of `std::meta::is_complete_type(^S)` at #1 (21.4.7) that is executed during the evaluation of `f(1) != 2`, the evaluation context contains `Q`, but does not contain the synthesized point associated with the injected declaration of `S`. However, the synthesized point is in the evaluation context of `std::meta::is_complete_type(^S)` at #2 during the evaluation of `f(0)`. — *end example*

7.7.7 Further definitions

[expr.const.defns]

¹ An expression or conversion is *manifestly constant-evaluated* if it is

- (1.1) — a *constant-expression*, or
- (1.2) — the condition of a *constexpr if* statement (8.5.2), or
- (1.3) — the expression corresponding to a *constexpr-block-declaration* (9.1), or
- (1.4) — an immediate invocation, or
- (1.5) — the result of substitution into an atomic constraint expression to determine whether it is satisfied (13.5.2.3), or
- (1.6) — the initializer of a variable that is usable in constant expressions or has constant initialization (6.10.3.2).⁶³

[Example 1:

```

template<bool> struct X {};
X<std::is_constant_evaluated()> x;           // type X<true>
int y;
const int a = std::is_constant_evaluated() ? y : 1; // dynamic initialization to 1
double z[a];                                   // error: a is not usable
                                              // in constant expressions

const int b = std::is_constant_evaluated() ? 2 : y; // static initialization to 2
int c = y + (std::is_constant_evaluated() ? 2 : y); // dynamic initialization to y+y

constexpr int f() {
    const int n = std::is_constant_evaluated() ? 13 : 17; // n is 13
    int m = std::is_constant_evaluated() ? 13 : 17;       // m can be 13 or 17 (see below)
    char arr[n] = {};                                     // char[13]
    return m + sizeof(arr);
}
int p = f();                                              // m is 13; initialized to 26
int q = p + f();                                          // m is 17 for this call; initialized to 56

```

— *end example*]

[Note 1: Except for a *static_assert-message*, a manifestly constant-evaluated expression is evaluated even in an unevaluated operand (7.2.3). — *end note*]

² An expression or conversion is *potentially constant evaluated* if it is:

- (2.1) — a manifestly constant-evaluated expression,
- (2.2) — a potentially evaluated expression (6.3),
- (2.3) — an immediate subexpression of a *braced-init-list*,⁶⁴
- (2.4) — an expression of the form *& cast-expression* that occurs within a templated entity,⁶⁵ or
- (2.5) — a potentially-evaluated subexpression (6.10.1) of one of the above.

A function or variable is *needed for constant evaluation* if it is:

- (2.6) — a *constexpr* function that is named by an expression (6.3) that is potentially constant evaluated, or
- (2.7) — a potentially-constant variable named by a potentially constant evaluated expression.

³ An object `a` is said to have *constant destruction* if

- (3.1) — it is not of class type nor (possibly multidimensional) array thereof, or

⁶³) Testing this condition can involve a notional evaluation of its initializer, with evaluations of contract assertions using the ignore evaluation semantic (6.11.2), as described above.

⁶⁴) In some cases, constant evaluation is needed to determine whether a narrowing conversion is performed (9.5.5).

⁶⁵) In some cases, constant evaluation is needed to determine whether such an expression is value-dependent (13.8.3.4).

- (3.2) — it is of class type or (possibly multidimensional) array thereof, that class type has a constexpr destructor (9.2.6), and for a hypothetical expression E whose only effect is to destroy $\mathbf{a}$, E would be a core constant expression if the lifetime of $\mathbf{a}$ and its non-mutable subobjects (but not its mutable subobjects) were considered to start within E .

8 Statements

[stmt]

8.1 Preamble

[stmt.pre]

- ¹ Except as indicated, statements are executed in sequence (6.10.1).

statement:

labeled-statement
attribute-specifier-seq_{opt} expression-statement
attribute-specifier-seq_{opt} compound-statement
attribute-specifier-seq_{opt} selection-statement
attribute-specifier-seq_{opt} iteration-statement
attribute-specifier-seq_{opt} expansion-statement
attribute-specifier-seq_{opt} jump-statement
attribute-specifier-seq_{opt} assertion-statement
declaration-statement
attribute-specifier-seq_{opt} try-block

init-statement:

expression-statement
simple-declaration
alias-declaration

condition:

expression
condition-declaration

condition-declaration:

attribute-specifier-seq_{opt} decl-specifier-seq declarator brace-or-equal-initializer
structured-binding-declaration initializer

for-range-declaration:

attribute-specifier-seq_{opt} decl-specifier-seq declarator
structured-binding-declaration

for-range-initializer:

expr-or-braced-init-list

The optional *attribute-specifier-seq* appertains to the respective statement. See 9.3.4 for the optional *attribute-specifier-seq* in a *for-range-declaration*.

- ² A *substatement* of a *statement* is one of the following:

- (2.1) — for a *labeled-statement*, its *statement*,
- (2.2) — for a *compound-statement*, any *statement* of its *statement-seq*,
- (2.3) — for a *selection-statement*, any of its *statements* or *compound-statements* (but not its *init-statement*),
- (2.4) — for an *iteration-statement*, its *statement* (but not an *init-statement*), or
- (2.5) — for an *expansion-statement*, its *compound-statement* (but not an *init-statement*).

[Note 1: The *compound-statement* of a *lambda-expression* is not a substatement of the *statement* (if any) in which the *lambda-expression* lexically appears. — end note]

- ³ A *statement S1* *encloses* a *statement S2* if

- (3.1) — *S2* is a substatement of *S1*,
- (3.2) — *S1* is a *selection-statement*, *iteration-statement*, or *expansion-statement*, and *S2* is the *init-statement* of *S1*,
- (3.3) — *S1* is a *try-block* and *S2* is its *compound-statement* or any of the *compound-statements* of its *handlers*, or
- (3.4) — *S1* encloses a *statement S3* and *S3* encloses *S2*.

A *statement S1* is *enclosed by* a *statement S2* if *S2* encloses *S1*.

- ⁴ The rules for *conditions* apply both to *selection-statements* (8.5) and to the **for** and **while** statements (8.6). If a *structured-binding-declaration* appears in a *condition*, the *condition* is a structured binding declaration (9.1).

A *condition* that is neither an *expression* nor a structured binding declaration is a declaration (Clause 9). The *declarator* shall not specify a function or an array. The *decl-specifier-seq* shall not define a class or enumeration. If the **auto type-specifier** appears in the *decl-specifier-seq*, the type of the identifier being declared is deduced from the initializer as described in 9.2.9.7.

- ⁵ The *decision variable* of a *condition* that is neither an *expression* nor a structured binding declaration is the declared variable. The decision variable of a *condition* that is a structured binding declaration is specified in 9.7.
- ⁶ The value of a *condition* that is not an *expression* in a statement other than a **switch** statement is the value of the decision variable contextually converted to **bool** (7.3). If that conversion is ill-formed, the program is ill-formed. The value of a *condition* that is an *expression* is the value of the expression, contextually converted to **bool** for statements other than **switch**; if that conversion is ill-formed, the program is ill-formed. The value of the condition will be referred to as simply “the condition” where the usage is unambiguous.
- ⁷ If a *condition* can be syntactically resolved as either an *expression* or a *condition-declaration*, it is interpreted as the latter.
- ⁸ For any *condition* or *for-range-declaration* *D*, each *decl-specifier* in the *decl-specifier-seq* of *D*, including that of any *structured-binding-declaration* of *D*, shall be either a *type-specifier* or **constexpr**.

8.2 Label

[stmt.label]

- ¹ A label can be added to a statement or used anywhere in a *compound-statement*.

```
label:
    attribute-specifier-seqopt identifier :
    attribute-specifier-seqopt case constant-expression :
    attribute-specifier-seqopt default :

labeled-statement:
    label statement
```

The optional *attribute-specifier-seq* appertains to the label. The only use of a label with an *identifier* is as the target of a **goto**. No two labels in a function shall have the same *identifier*. A label can be used in a **goto** statement before its introduction.

- ² A *labeled-statement* whose *label* is a **case** or **default** label shall be enclosed by (8.1) a **switch** statement (8.5.3).
- ³ A *control-flow-limited statement* is a statement **S** for which:
 - (3.1) — a **case** or **default** label appearing within **S** shall be associated with a **switch** statement (8.5.3) within **S**, and
 - (3.2) — a label declared in **S** shall only be referred to by a statement (8.8.6) in **S**.
- ⁴ An identifier label shall not be enclosed by an *expansion-statement* (8.7).

8.3 Expression statement

[stmt.expr]

- ¹ Expression statements have the form

```
expression-statement:
    expressionopt ;
```

The expression is a discarded-value expression (7.2.3). All side effects from an expression statement are completed before the next statement is executed. An expression statement with the *expression* missing is called a *null statement*.

[Note 1: Most statements are expression statements — usually assignments or function calls. A null statement is useful to supply a null body to an iteration statement such as a **while** statement (8.6.2). — end note]

8.4 Compound statement or block

[stmt.block]

- ¹ A *compound statement* (also known as a block) groups a sequence of statements into a single statement.

```
compound-statement:
    { statement-seqopt label-seqopt }

statement-seq:
    statement statement-seqopt
```

label-seq:
label label-seq_{opt}

A label at the end of a *compound-statement* is treated as if it were followed by a null statement.

- ² [Note 1: A compound statement defines a block scope (6.4). — end note]

8.5 Selection statements

[stmt.select]

8.5.1 General

[stmt.select.general]

- ¹ Selection statements choose one of several flows of control.

selection-statement:
 if *constexpr_{opt}* (*init-statement_{opt}* *condition*) *statement*
 if *constexpr_{opt}* (*init-statement_{opt}* *condition*) *statement* else *statement*
 if !_{*opt*} *constexpr* *compound-statement*
 if !_{*opt*} *constexpr* *compound-statement* else *statement*
 switch (*init-statement_{opt}* *condition*) *statement*

See 9.3.4 for the optional *attribute-specifier-seq* in a condition.

[Note 1: An *init-statement* ends with a semicolon. — end note]

- ² [Note 2: Each *selection-statement* and each substatement of a *selection-statement* has a block scope (6.4.3). — end note]

8.5.2 The if statement

[stmt.if]

- ¹ If the condition (8.1) yields **true**, the first substatement is executed. If the **else** part of the selection statement is present and the condition yields **false**, the second substatement is executed. If the first substatement is reached via a label, the condition is not evaluated and the second substatement is not executed. In the second form of if statement (the one including **else**), if the first substatement is also an if statement then that inner if statement shall contain an **else** part.⁶⁶
- ² If the if statement is of the form if *constexpr*, the value of the condition is contextually converted to **bool** and the converted expression shall be a constant expression (7.7.3); this form is called a *constexpr if* statement. If the value of the converted condition is **false**, the first substatement is a *discarded statement*, otherwise the second substatement, if present, is a discarded statement. During the instantiation of an enclosing templated entity (13.1), if the condition is not value-dependent after its instantiation, the discarded substatement (if any) is not instantiated. Each substatement of a *constexpr if* statement is a control-flow-limited statement (8.2).

[Example 1:

```
if constexpr (sizeof(int[2])) {}           // OK, narrowing allowed
```

— end example]

[Note 1: Odr-uses (6.3) in a discarded statement do not require an entity to be defined. — end note]

[Example 2:

```
template<typename T, typename ... Rest> void g(T&& p, Rest&& ...rs) {
    // ... handle p

    if constexpr (sizeof...(rs) > 0)
        g(rs...);           // never instantiated with an empty argument list
}
```

```
extern int x;               // no definition of x required
```

```
int f() {
    if constexpr (true)
        return 0;
    else if (x)
        return x;
    else
        return -x;
}
```

— end example]

⁶⁶) In other words, the **else** is associated with the nearest un-elsed **if**.

- ³ An **if** statement of the form

```
if constexpropt ( init-statement condition ) statement
```

is equivalent to

```
{
    init-statement
    if constexpropt ( condition ) statement
}
```

and an **if** statement of the form

```
if constexpropt ( init-statement condition ) statement else statement
```

is equivalent to

```
{
    init-statement
    if constexpropt ( condition ) statement else statement
}
```

except that the *init-statement* is in the same scope as the *condition*.

- ⁴ An **if** statement of the form **if constexpr** is called a *constexpr if statement*. The *statement*, if any, in a **constexpr if** statement shall be a *compound-statement*.

[Example 3:

```
constexpr void f(bool b) {
    if (true)
        if constexpr { }
        else ;           // error: not a compound-statement; else not associated with outer if
}
```

— end example]

- ⁵ If a **constexpr if** statement is evaluated in a context that is manifestly constant-evaluated (7.7.7), the first substatement is executed.

[Note 2: The first substatement is an immediate function context. — end note]

Otherwise, if the **else** part of the selection statement is present, then the second substatement is executed. Each substatement of a **constexpr if** statement is a control-flow-limited statement (8.2).

- ⁶ An **if** statement of the form

```
if ! constexpr compound-statement
```

is not itself a **constexpr if** statement, but is equivalent to the **constexpr if** statement

```
if constexpr { } else compound-statement
```

An **if** statement of the form

```
if ! constexpr compound-statement1 else statement2
```

is not itself a **constexpr if** statement, but is equivalent to the **constexpr if** statement

```
if constexpr statement2 else compound-statement1
```

8.5.3 The **switch** statement

[stmt.switch]

- ¹ The **switch** statement causes control to be transferred to one of several statements depending on the value of a condition.
- ² If the *condition* is an *expression*, the value of the condition is the value of the *expression*; otherwise, it is the value of the decision variable. The value of the condition shall be of integral type, enumeration type, or class type. If of class type, the condition is contextually implicitly converted (7.3) to an integral or enumeration type. If the (possibly converted) type is subject to integral promotions (7.3.7), the condition is converted to the promoted type. Any statement within the **switch** statement can be labeled with one or more case labels as follows:

```
case constant-expression :
```

where the *constant-expression* shall be a converted constant expression (7.7.3) of the adjusted type of the switch condition. No two of the case constants in the same switch shall have the same value after conversion.

- ³ There shall be at most one label of the form

default :

within a **switch** statement.

- 4 Switch statements can be nested; a **case** or **default** label is associated with the smallest switch enclosing it.
- 5 When the **switch** statement is executed, its condition is evaluated. If one of the case constants has the same value as the condition, control is passed to the statement following the matched case label. If no case constant matches the condition, and if there is a **default** label, control passes to the statement labeled by the default label. If no case matches and if there is no **default** then none of the statements in the switch is executed.
- 6 **case** and **default** labels in themselves do not alter the flow of control, which continues unimpeded across such labels. To exit from a switch, see **break**, 8.8.2.

[Note 1: Usually, the substatement that is the subject of a switch is compound and **case** and **default** labels appear on the top-level statements contained within the (compound) substatement, but this is not required. Declarations can appear in the substatement of a **switch** statement. — end note]

- 7 A **switch** statement of the form

```
switch ( init-statement condition ) statement
```

is equivalent to

```
{
    init-statement
    switch ( condition ) statement
}
```

except that the *init-statement* is in the same scope as the *condition*.

8.6 Iteration statements

[stmt.iter]

8.6.1 General

[stmt.iter.general]

- 1 Iteration statements specify looping.

iteration-statement:

```
while ( condition ) statement
do statement while ( expression ) ;
for ( init-statement conditionopt ; expressionopt ) statement
for ( init-statementopt for-range-declaration : for-range-initializer ) statement
```

[Note 1: An *init-statement* ends with a semicolon. — end note]

- 2 The substatement in an *iteration-statement* implicitly defines a block scope (6.4) which is entered and exited each time through the loop. If the substatement in an *iteration-statement* is a single statement and not a *compound-statement*, it is as if it was rewritten to be a *compound-statement* containing the original statement.

[Example 1:

```
while (--x >= 0)
    int i;
```

can be equivalently rewritten as

```
while (--x >= 0) {
    int i;
}
```

Thus after the **while** statement, *i* is no longer in scope. — end example]

- 3 A *trivially empty iteration statement* is an iteration statement matching one of the following forms:

- (3.1) — **while** (*expression*) ;
- (3.2) — **while** (*expression*) { }
- (3.3) — **do** ; **while** (*expression*) ;
- (3.4) — **do** { } **while** (*expression*) ;
- (3.5) — **for** (*init-statement expression_{opt}* ;) ;
- (3.6) — **for** (*init-statement expression_{opt}* ;) { }

The *controlling expression* of a trivially empty iteration statement is the *expression* of a **while**, **do**, or **for** statement (or **true**, if the **for** statement has no *expression*). A *trivial infinite loop* is a trivially empty iteration statement for which the converted controlling expression is a constant expression, when interpreted

as a *constant-expression* (7.7.3), and evaluates to **true**. The *statement* of a trivial infinite loop is replaced with a call to the function `std::this_thread::yield` (32.4.5); it is implementation-defined whether this replacement occurs on freestanding implementations.

[Note 2: In a freestanding environment, concurrent forward progress is not guaranteed; such systems therefore require explicit cooperation. A call to `yield` can add implicit cooperation where none is otherwise intended. — end note]

8.6.2 The while statement

[**stmt.while**]

- ¹ In the **while** statement, the substatement is executed repeatedly until the value of the condition (8.1) becomes **false**. The test takes place before each execution of the substatement.
- ² A **while** statement is equivalent to

```

    label :
    {
        if ( condition ) {
            statement
            goto label ;
        }
    }

```

[Note 1: The variable created in the condition is destroyed and created with each iteration of the loop.

[Example 1:

```

struct A {
    int val;
    A(int i) : val(i) { }
    ~A() { }
    operator bool() { return val != 0; }
};
int i = 1;
while (A a = i) {
    // ...
    i = 0;
}

```

In the while-loop, the constructor and destructor are each called twice, once for the condition that succeeds and once for the condition that fails. — end example]

— end note]

8.6.3 The do statement

[**stmt.do**]

- ¹ The expression is contextually converted to **bool** (7.3); if that conversion is ill-formed, the program is ill-formed.
- ² In the **do** statement, the substatement is executed repeatedly until the value of the expression becomes **false**. The test takes place after each execution of the statement.

8.6.4 The for statement

[**stmt.for**]

- ¹ The **for** statement

```

for ( init-statement conditionopt ; expressionopt ) statement

```

is equivalent to

```

{
    init-statement
    while ( condition ) {
        statement
        expression ;
    }
}

```

except that the *init-statement* is in the same scope as the *condition*, and except that a **continue** in *statement* (not enclosed in another iteration statement) will execute *expression* before re-evaluating *condition*.

[Note 1: Thus the first statement specifies initialization for the loop; the condition (8.1) specifies a test, sequenced before each iteration, such that the loop is exited when the condition becomes **false**; the expression often specifies incrementing that is sequenced after each iteration. — end note]

- ² Either or both of the *condition* and the *expression* can be omitted. A missing *condition* makes the implied **while** clause equivalent to **while (true)**.

8.6.5 The range-based for statement

[stmt.ranged]

- ¹ The range-based for statement

```
for ( init-statementopt for-range-declaration : for-range-initializer ) statement
```

is equivalent to

```
{
    init-statementopt
    auto &&range = for-range-initializer ;
    auto begin = begin-expr ;
    auto end = end-expr ;
    for ( ; begin != end; ++begin ) {
        for-range-declaration = * begin ;
        statement
    }
}
```

where

- (1.1) — if the *for-range-initializer* is an *expression*, it is regarded as if it were surrounded by parentheses (so that a comma operator cannot be reinterpreted as delimiting two *init-declarators*);
 - (1.2) — *range*, *begin*, and *end* are variables defined for exposition only; and
 - (1.3) — *begin-expr* and *end-expr* are determined as follows:
 - (1.3.1) — if the type of *range* is a reference to an array type R, *begin-expr* and *end-expr* are *range* and *range* + N, respectively, where N is the array bound. If R is an array of unknown bound or an array of incomplete type, the program is ill-formed;
 - (1.3.2) — if the type of *range* is a reference to a class type C, and searches in the scope of C (6.5.2) for the names *begin* and *end* each find at least one declaration, *begin-expr* and *end-expr* are *range*.*begin*() and *range*.*end*(), respectively;
 - (1.3.3) — otherwise, *begin-expr* and *end-expr* are *begin*(*range*) and *end*(*range*), respectively, where *begin* and *end* undergo argument-dependent lookup (6.5.4).
- [Note 1: Ordinary unqualified lookup (6.5.3) is not performed. — end note]

[Example 1:

```
int array[5] = { 1, 2, 3, 4, 5 };
for (int& x : array)
    x *= 2;
```

— end example]

[Note 2: The lifetime of some temporaries in the *for-range-initializer* is extended to cover the entire loop (6.8.7). — end note]

[Example 2:

```
using T = std::list<int>;
const T& f1(const T& t) { return t; }
const T& f2(T t)        { return t; }
T g();

void foo() {
    for (auto e : f1(g())) {} // OK, lifetime of return value of g() extended
    for (auto e : f2(g())) {} // undefined behavior
}
```

— end example]

8.7 Expansion statements

[stmt.expand]

- ¹ Expansion statements specify repeated instantiations (13.7.1) of their substatement.

expansion-statement:

```
template for ( init-statementopt for-range-declaration : expansion-initializer ) compound-statement
```

expansion-initializer:
expression
expansion-init-list

expansion-init-list:
{ *expression-list* ,_{opt} }
{ }

² The *compound-statement* of an *expansion-statement* is a control-flow-limited statement (8.2).

³ For an expression *E*, let the expressions *begin-expr* and *end-expr* be determined as specified in 8.6.5. An expression is *expansion-iterable* if it does not have array type and either

(3.1) — *begin-expr* and *end-expr* are of the form *E.begin()* and *E.end()*, or

(3.2) — argument-dependent lookups for *begin*(*E*) and for *end*(*E*) each find at least one viable candidate (12.2.3).

⁴ An expansion statement is

(4.1) — an *enumerating expansion statement* if its *expansion-initializer* is of the form *expansion-init-list*;

(4.2) — otherwise, an *iterating expansion statement* if its *expansion-initializer* is an *expansion-iterable* expression;

(4.3) — otherwise, a *destructuring expansion statement*.

⁵ An expansion statement *S* is equivalent to a *compound-statement* containing instantiations of the *for-range-declaration* (including its implied initialization), together with the *compound-statement* of *S*, as follows:

(5.1) — If *S* is an enumerating expansion statement, *S* is equivalent to:

```
{
    init-statement
    S0
    :
    SN-1
}
```

where *N* is the number of elements in the *expression-list*, *S_i* is

```
{
    for-range-declaration = Ei;
    compound-statement
}
```

and *E_i* is the *i*th element of the *expression-list*.

(5.2) — Otherwise, if *S* is an iterating expansion statement, *S* is equivalent to:

```
{
    init-statement
    constexpropt decltype(auto) range = (expansion-initializer);
    constexpropt auto begin = begin-expr; // see 8.6.5
    S0
    :
    SN-1
}
```

where *N* is the result of evaluating the expression

```
[&] consteval {
    std::ptrdiff_t result = 0;
    auto b = begin-expr;
    auto e = end-expr;
    for (; b != e; ++b) ++result;
    return result;
}() // distance from begin to end
```

and *S_i* is

```
{
    constexpropt auto iter = begin + decltype(begin - begin){i};
    for-range-declaration = *iter;
```

```

    compound-statement
}

```

The variables *range*, *begin*, and *iter* are defined for exposition only. The keyword `constexpr` is present in the declarations of *range*, *begin*, and *iter* if and only if `constexpr` is one of the *decl-specifiers* of the *decl-specifier-seq* of the *for-range-declaration*. The identifier *i* is considered to be a prvalue of type `std::ptrdiff_t`; the program is ill-formed if *i* is not representable as such a value.

- (5.3) — Otherwise, *S* is a destructuring expansion statement and, if *N* is 0, *S* is equivalent to:

```

{
    init-statement
    constexpropt auto&& range = expansion-initializer;
}

```

otherwise, *S* is equivalent to:

```

{
    init-statement
    constexpropt auto&& [u0, u1, ..., uN-1] = expansion-initializer;
    S0
    :
    SN-1
}

```

where *N* is the structured binding size of the type of the *expansion-initializer* and *S_i* is

```

{
    for-range-declaration = vi;
    compound-statement
}

```

If the *expansion-initializer* is an lvalue, then *v_i* is *u_i*; otherwise, *v_i* is `static_cast<decltype(ui)&&>(ui)`. The keyword `constexpr` is present in the *structured-binding-declaration* of *u₀*, *u₁*, ..., *u_{N-1}* if and only if `constexpr` is one of the *decl-specifiers* of the *decl-specifier-seq* of the *for-range-declaration*.

- ⁶ [Example 1:

```

constexpr int f(auto const&... Containers) {
    int result = 0;
    template for (auto const& c : {Containers...}) {           // OK, enumerating expansion statement
        result += c[0];
    }
    return result;
}
constexpr int c1[] = {1, 2, 3};
constexpr int c2[] = {4, 3, 2, 1};
static_assert(f(c1, c2) == 5);

```

— end example]

- ⁷ [Example 2:

```

constexpr int f() {
    constexpr std::array<int, 3> arr {1, 2, 3};
    int result = 0;
    template for (constexpr int s : arr) {                     // OK, iterating expansion statement
        result += sizeof(char[s]);
    }
    return result;
}
static_assert(f() == 6);

```

— end example]

- ⁸ [Example 3:

```

struct S {
    int i;
    short s;
};

```

```

consteval long f(S s) {
    long result = 0;
    template for (auto x : s) {
        result += sizeof(x);
    }
    return result;
}
static_assert(f(S{}) == sizeof(int) + sizeof(short));
— end example]

```

// OK, destructuring expansion statement

8.8 Jump statements

[stmt.jump]

8.8.1 General

[stmt.jump.general]

- ¹ Jump statements unconditionally transfer control.

jump-statement:
 break ;
 continue ;
 return *expr-or-braced-init-list*_{opt} ;
coroutine-return-statement
 goto *identifier* ;

- ² [Note 1: On exit from a scope (however accomplished), objects with automatic storage duration (6.8.6.4) that have been constructed in that scope are destroyed in the reverse order of their construction (8.10). For temporaries, see 6.8.7. However, the program can be terminated (by calling `std::exit()` or `std::abort()` (17.5), for example) without destroying objects with automatic storage duration. — end note]

[Note 2: A suspension of a coroutine (7.6.2.4) is not considered to be an exit from a scope. — end note]

8.8.2 The break statement

[stmt.break]

- ¹ A **break** statement shall be enclosed by (8.1) an *iteration-statement* (8.6), an *expansion-statement* (8.7), or a **switch** statement (8.5.3). The **break** statement causes termination of the innermost such enclosing statement; control passes to the statement following the terminated statement, if any.

8.8.3 The continue statement

[stmt.cont]

- ¹ A **continue** statement shall be enclosed by (8.1) an *iteration-statement* or an *expansion-statement*. If the innermost enclosing such statement *X* is an *iteration-statement* (8.6), the **continue** statement causes control to pass to the end of the *statement* of *X*. Otherwise, control passes to the end of the *compound-statement* of the current *S_i* (8.7).

8.8.4 The return statement

[stmt.return]

- ¹ A function returns control to its caller by the **return** statement.
- ² The *expr-or-braced-init-list* of a **return** statement is called its operand. A **return** statement with no operand shall be used only in a function whose return type is *cv void*, a constructor (11.4.5), or a destructor (11.4.7). A **return** statement with an operand of type *void* shall be used only in a function that has a *cv void* return type. A **return** statement with any other operand shall be used only in a function that has a return type other than *cv void*; the **return** statement initializes the returned reference or prvalue result object of the (explicit or implicit) function call by copy-initialization (9.5) from the operand.

[Note 1: A constructor or destructor does not have a return type. — end note]

[Note 2: A **return** statement can involve an invocation of a constructor to perform a copy or move of the operand if it is not a prvalue or if its type differs from the return type of the function. A copy operation associated with a **return** statement can be elided or converted to a move operation if an automatic storage duration variable is returned (11.9.6). — end note]

- ³ The destructor for the result object is potentially invoked (11.4.7, 14.3).

[Example 1:

```

class A {
    ~A() {}
};
A f() { return A(); } // error: destructor of A is private (even though it is never invoked)
— end example]

```


⁴ Flowing off the end of a constructor, a destructor, or a non-coroutine function with a *cv* void return type is equivalent to a **return** with no operand. Otherwise, flowing off the end of a function that is neither **main** (6.10.3.1) nor a coroutine (9.6.4) results in undefined behavior (F.4.1).

⁵ The copy-initialization of the result of the call is sequenced before the destruction of temporaries at the end of the full-expression established by the operand of the **return** statement, which, in turn, is sequenced before the destruction of local variables (8.8) of the block enclosing the **return** statement.

[Note 3: These operations are sequenced before the destruction of local variables in each remaining enclosing block of the function (8.10), which, in turn, is sequenced before the evaluation of postcondition assertions of the function (9.4.1), which, in turn, is sequenced before the destruction of function parameters (7.6.1.3). — end note]

⁶ In a function other than an invented function for **std::is_convertible** (21.3.7), a **return** statement that binds

(6.1) — a returned reference or

(6.2) — a constituent reference (6.8.2) of a returned object

to a temporary expression (6.8.7) is ill-formed.

[Example 2:

```
auto&& f1() {
    return 42;           // ill-formed
}
const double& f2() {
    static int x = 42;
    return x;           // ill-formed
}
auto&& id(auto&& r) {
    return static_cast<decltype(r)&&>(r);
}
auto&& f3() {
    return id(42);       // OK, but probably a bug
}
```

— end example]

8.8.5 The **co_return** statement

[stmt.return.coroutine]

coroutine-return-statement:

co_return *expr-or-braced-init-list*_{opt} ;

¹ A **co_return** statement transfers control to the caller or resumer of a coroutine (9.6.4). A coroutine shall not enclose a **return** statement (8.8.4).

[Note 1: For this determination, it is irrelevant whether the **return** statement is enclosed by a discarded statement (8.5.2). — end note]

² The *expr-or-braced-init-list* of a **co_return** statement is called its operand. Let *p* be an lvalue naming the coroutine promise object (9.6.4). A **co_return** statement is equivalent to:

{ *S*; goto *final-suspend*; }

where *final-suspend* is the exposition-only label defined in 9.6.4 and *S* is defined as follows:

(2.1) — If the operand is a *braced-init-list* or an expression of non-void type, *S* is *p.return_value(expr-or-braced-init-list)*. The expression *S* shall be a prvalue of type **void**.

(2.2) — Otherwise, *S* is the *compound-statement* { *expression*_{opt} ; *p.return_void()*; }. The expression *p.return_void()* shall be a prvalue of type **void**.

³ If overload resolution for *p.return_void()* succeeds, flowing off the end of a coroutine's *function-body* is equivalent to a **co_return** with no operand; otherwise flowing off the end of a coroutine's *function-body* results in undefined behavior (F.4.2).

8.8.6 The **goto** statement

[stmt.goto]

¹ The **goto** statement unconditionally transfers control to the statement labeled by the identifier. The identifier shall be a label (8.2) located in the current function.

8.9 Assertion statement

[stmt.contract.assert]

assertion-statement:

`contract_assert attribute-specifier-seqopt ( conditional-expression ) ;`

- ¹ An *assertion-statement* introduces a contract assertion (6.11). The optional *attribute-specifier-seq* appertains to the introduced contract assertion.
- ² The predicate (6.11.1) of an *assertion-statement* is its *conditional-expression* contextually converted to `bool`.
- ³ The evaluation of consecutive *assertion-statements* is an evaluation in sequence (6.11.2) of the contract assertions introduced by those *assertion-statements*.

[Note 1: A sequence of *assertion-statements* can thus be repeatedly evaluated as a group.

[Example 1:

```
int f(int i)
{
    contract_assert(i == 0); // #1
    contract_assert(i >= 0); // #2
    return 0;
}
int g = f(0); // can evaluate #1, #2, #1, #2
— end example]
— end note]
```

8.10 Declaration statement

[stmt.dcl]

- ¹ A declaration statement introduces one or more new names into a block; it has the form

declaration-statement:

block-declaration

[Note 1: If an identifier introduced by a declaration was previously declared in an outer block, the outer declaration is hidden for the remainder of the block (6.5.3), after which it resumes its force. — end note]

- ² A block variable with automatic storage duration (6.8.6.4) is *active* everywhere in the scope to which it belongs after its *init-declarator*. Upon each transfer of control (including sequential execution of statements, but excluding function calls) within a function from point *P* to point *Q*, all block variables with automatic storage duration that are active at *P* and not at *Q* are destroyed in the reverse order of their construction. Then, all block variables with automatic storage duration that are active at *Q* but not at *P* are initialized in declaration order; unless all such variables have vacuous initialization (6.8.4), the transfer of control shall not be a jump.⁶⁷ When a *declaration-statement* is executed, *P* and *Q* are the points immediately before and after it; when a function returns, *Q* is after its body.

[Example 1:

```
void f() {
    // ...
    goto lx;           // error: jump into scope of a
    // ...
ly:
    X a = 1;
    // ...
lx:
    goto ly;           // OK, jump implies destructor call for a followed by
                       // construction again immediately following label ly
}
— end example]
```

- ³ Dynamic initialization of a block variable with static storage duration (6.8.6.2) or thread storage duration (6.8.6.3) is performed the first time control passes through its declaration; such a variable is considered initialized upon the completion of its initialization. If the initialization exits by throwing an exception, the initialization is not complete, so it will be tried again the next time control enters the declaration. If control enters the declaration concurrently while the variable is being initialized, the concurrent execution shall wait for completion of the initialization.

⁶⁷) The transfer from the condition of a `switch` statement to a `case` label is considered a jump in this respect.

[*Note 2*: A conforming implementation cannot introduce any deadlock around execution of the initializer. Deadlocks might still be caused by the program logic; the implementation need only avoid deadlocks due to its own synchronization operations. — *end note*]

If control re-enters the declaration recursively while the variable is being initialized, the behavior is undefined (F.4.3).

[*Example 2*:

```
int foo(int i) {
    static int s = foo(2*i);    // undefined behavior: recursive call
    return i+1;
}
```

— *end example*]

- ⁴ An object associated with a block variable with static or thread storage duration will be destroyed if and only if it was constructed.

[*Note 3*: 6.10.3.4 describes the order in which such objects are destroyed. — *end note*]

8.11 Ambiguity resolution

[stmt.ambig]

- ¹ There is an ambiguity in the grammar involving *expression-statements* and *declarations*: An *expression-statement* with a function-style explicit type conversion (7.6.1.4) as its leftmost subexpression can be indistinguishable from a *declaration* where the first *declarator* starts with a (. In those cases the *statement* is considered a *declaration*, except as specified below.
- ² [*Note 1*: If the *statement* cannot syntactically be a *declaration*, there is no ambiguity, so this rule does not apply. In some cases, the whole *statement* needs to be examined to determine whether this is the case. This resolves the meaning of many examples.

[*Example 1*: Assuming T is a *simple-type-specifier* (9.2.9.3),

```
T(a)->m = 7;    // expression-statement
T(a)++;         // expression-statement
T(a,5)<<c;       // expression-statement

T(*d)(int);     // declaration
T(e)[5];        // declaration
T(f) = { 1, 2 }; // declaration
T(*g)(double(3)); // declaration
```

In the last example above, g, which is a pointer to T, is initialized to double(3). This is of course ill-formed for semantic reasons, but that does not affect the syntactic analysis. — *end example*]

The remaining cases are *declarations*.

[*Example 2*:

```
class T {
    // ...
public:
    T();
    T(int);
    T(int, int);
};
T(a);           // declaration
T(*b)();        // declaration
T(c)=7;         // declaration
T(d),e,f=3;     // declaration
extern int h;
T(g)(h,2);      // declaration
```

— *end example*]

— *end note*]

- ³ The disambiguation is purely syntactic; that is, the meaning of the names occurring in such a statement, beyond whether they are *type-names* or not, is not generally used in or changed by the disambiguation. Class templates are instantiated as necessary to determine if a qualified name is a *type-name*. Disambiguation precedes parsing, and a statement disambiguated as a declaration may be an ill-formed declaration. If, during

parsing, lookup finds that a name in a template argument is bound to (part of) the declaration being parsed, the program is ill-formed. No diagnostic is required (G.5.1).

[Example 3:

```
struct T1 {
    T1 operator()(int x) { return T1(x); }
    int operator=(int x) { return x; }
    T1(int) { }
};
struct T2 { T2(int) { } };
int a, ((*b)(T2))(int), c, d;

void f() {
    // disambiguation requires this to be parsed as a declaration:
    T1(a) = 3,
    T2(4),           // T2 will be declared as a variable of type T1, but this will not
    ((*b)(T2(c)))(int(d)); // allow the last part of the declaration to parse properly,
                        // since it depends on T2 being a type-name
}
```

— end example]

- ⁴ A syntactically ambiguous statement that can syntactically be a *declaration* with an outermost *declarator* with a *trailing-return-type* is considered a *declaration* only if it starts with **auto**.

[Example 4:

```
struct M;
struct S {
    S* operator()();
    int N;
    int M;

    void mem(S s) {
        auto(s)()->M;           // expression, S::M hides ::M
    }
};

void f(S s) {
    {
        auto(s)()->N;           // expression
        auto(s)()->M;           // function declaration
    }
    {
        S(s)()->N;               // expression
        S(s)()->M;               // expression
    }
}
```

— end example]

9 Declarations

[dcl]

9.1 Preamble

[dcl.pre]

- ¹ Declarations generally specify how names are to be interpreted. Declarations have the form

```

declaration-seq:
    declaration declaration-seqopt

declaration:
    name-declaration
    special-declaration

name-declaration:
    block-declaration
    nodeclspec-function-declaration
    function-definition
    friend-type-declaration
    template-declaration
    deduction-guide
    linkage-specification
    namespace-definition
    empty-declaration
    attribute-declaration
    module-import-declaration

special-declaration:
    explicit-instantiation
    explicit-specialization
    export-declaration

block-declaration:
    simple-declaration
    asm-declaration
    namespace-alias-definition
    using-declaration
    using-enum-declaration
    using-directive
    static_assert-declaration
    constexpr-block-declaration
    alias-declaration
    opaque-enum-declaration

nodeclspec-function-declaration:
    attribute-specifier-seqopt declarator ;

alias-declaration:
    using identifier attribute-specifier-seqopt = defining-type-id ;

sb-identifier:
    ...opt identifier attribute-specifier-seqopt

sb-identifier-list:
    sb-identifier
    sb-identifier-list , sb-identifier

structured-binding-declaration:
    attribute-specifier-seqopt decl-specifier-seq ref-qualifieropt [ sb-identifier-list ]

simple-declaration:
    decl-specifier-seq init-declarator-listopt ;
    attribute-specifier-seq decl-specifier-seq init-declarator-list ;
    structured-binding-declaration initializer ;

static_assert-message:
    unevaluated-string
    constant-expression

```

```

static_assert-declaration:
    static_assert ( constant-expression ) ;
    static_assert ( constant-expression , static_assert-message ) ;

constexpr-block-declaration:
    constexpr compound-statement

empty-declaration:
    ;

attribute-declaration:
    attribute-specifier-seq ;

```

[Note 1: *asm-declarations* are described in 9.11, and *linkage-specifications* are described in 9.12; *function-definitions* are described in 9.6 and *template-declarations* and *deduction-guides* are described in 13.7.2.3; *namespace-definitions* are described in 9.9.2, *using-declarations* are described in 9.10 and *using-directives* are described in 9.9.4. — end note]

- 2 Certain declarations contain one or more scopes (6.4.1). Unless otherwise stated, utterances in Clause 9 about components in, of, or contained by a declaration or subcomponent thereof refer only to those components of the declaration that are *not* nested within scopes nested within the declaration.
- 3 If a *name-declaration* matches the syntactic requirements of *friend-type-declaration*, it is a *friend-type-declaration*.
- 4 A *simple-declaration* or *nodeclspec-function-declaration* of the form

```
attribute-specifier-seqopt decl-specifier-seqopt init-declarator-listopt ;
```

is divided into three parts. Attributes are described in 9.13. *decl-specifiers*, the principal components of a *decl-specifier-seq*, are described in 9.2. *declarators*, the components of an *init-declarator-list*, are described in 9.3. The *attribute-specifier-seq* appertains to each of the entities declared by the *declarators* of the *init-declarator-list*.

[Note 2: In the declaration for an entity, attributes appertaining to that entity can appear at the start of the declaration and after the *declarator-id* for that declaration. — end note]

[Example 1:

```
[[noreturn]] void f [[noreturn]] ();    // OK
— end example]
```

- 5 If a *declarator-id* is a name, the *init-declarator* and (hence) the declaration introduce that name.

[Note 3: Otherwise, the *declarator-id* is a *qualified-id* or names a destructor or its *unqualified-id* is a *template-id* and no name is introduced. — end note]

The *defining-type-specifiers* (9.2.9) in the *decl-specifier-seq* and the recursive *declarator* structure describe a type (9.3.4), which is then associated with the *declarator-id*.

- 6 In a *simple-declaration*, the optional *init-declarator-list* can be omitted only when declaring a class (11.1) or enumeration (9.8.1), that is, when the *decl-specifier-seq* contains either a *class-specifier*, an *elaborated-type-specifier* with a *class-key* (11.3), or an *enum-specifier*. In these cases and whenever a *class-specifier* or *enum-specifier* is present in the *decl-specifier-seq*, the identifiers in these specifiers are also declared (as *class-names*, *enum-names*, or *enumerators*, depending on the syntax). In such cases, the *decl-specifier-seq* shall (re)introduce one or more names into the program.

[Example 2:

```
enum { };          // error
typedef class { }; // error
— end example]
```

- 7 A *simple-declaration* or a *condition* with a *structured-binding-declaration* is called a *structured binding declaration* (9.7). Each *decl-specifier* in the *decl-specifier-seq* shall be *constexpr*, *constexpr*, *static*, *thread_local*, *auto* (9.2.9.7), or a *cv-qualifier*. The declaration shall contain at most one *sb-identifier* whose *identifier* is preceded by an ellipsis. If the declaration contains any such *sb-identifier*, it shall declare a templated entity (13.1).

[Example 3:

```
template<class T> concept C = true;
C auto [x, y] = std::pair{1, 2};    // error: constrained placeholder-type-specifier
                                   // not permitted for structured bindings
— end example]
```

The *initializer* shall be of the form “= *assignment-expression*”, of the form “{ *assignment-expression* }”, or of the form “(*assignment-expression*)”. If the *structured-binding-declaration* appears as a *condition*, the *assignment-expression* shall be of non-union class type. Otherwise, the *assignment-expression* shall be of array or non-union class type.

- ⁸ If the *decl-specifier-seq* contains the **typedef** specifier, the declaration is a *typedef declaration* and each *declarator-id* is declared to be a *typedef-name* (9.2.4).

[Note 4: Such a *declarator-id* is an *identifier* (11.4.8.3). — end note]

Otherwise, if the type associated with a *declarator-id* is a function type (9.3.4.6), the declaration is a *function declaration*. Otherwise, if the type associated with a *declarator-id* is an object or reference type, the declaration is an *object declaration*. Otherwise, the program is ill-formed.

[Example 4:

```
int f(), x;           // OK, function declaration for f and object declaration for x
extern void g(),      // OK, function declaration for g
y;                   // error: void is not an object type
```

— end example]

- ⁹ An object definition causes storage of appropriate size and alignment to be reserved and any appropriate initialization (9.5) to be done.
- ¹⁰ Syntactic components beyond those found in the general form of *simple-declaration* are added to a function declaration to make a *function-definition*. A token sequence starting with { or = is treated as a *function-body* (9.6.1) if the type of the *declarator-id* (9.3.4.1) is a function type, and is otherwise treated as a *brace-or-equal-initializer* (9.5.1).

[Note 5: If the declaration acquires a function type through template instantiation, the program is ill-formed; see 13.9.1. The function type of a function definition cannot be specified with a *typedef-name* (9.3.4.6). — end note]

- ¹¹ A *nodeclspec-function-declaration* shall declare a constructor, destructor, or conversion function.

[Note 6: Because a member function cannot be subject to a non-defining declaration outside of a class definition (11.4.2), a *nodeclspec-function-declaration* can only be used in a *template-declaration* (13.1), *explicit-instantiation* (13.9.3), or *explicit-specialization* (13.9.4). — end note]

- ¹² If a *static_assert-message* matches the syntactic requirements of *unevaluated-string*, it is an *unevaluated-string* and the text of the *static_assert-message* is the text of the *unevaluated-string*. Otherwise, a *static_assert-message* shall be an expression *M* such that

(12.1) — the expression *M.size()* is implicitly convertible to the type **std::size_t**, and

(12.2) — the expression *M.data()* is implicitly convertible to the type “pointer to **const char**”.

- ¹³ In a *static_assert-declaration*, the *constant-expression E* is contextually converted to **bool** and the converted expression shall be a constant expression (7.7.3). If the value of the expression *E* when so converted is **true** or the expression is evaluated in the context of a template definition, the declaration has no effect and the *static_assert-message* is an unevaluated operand (7.2.3). Otherwise, the *static_assert-declaration fails* and

(13.1) — the program is ill-formed, and

(13.2) — if the *static_assert-message* is a *constant-expression M*,

(13.2.1) — *M.size()* shall be a converted constant expression of type **std::size_t** and let *N* denote the value of that expression,

(13.2.2) — *M.data()*, implicitly converted to the type “pointer to **const char**”, shall be a core constant expression and let *D* denote the converted expression,

(13.2.3) — for each *i* where $0 \leq i < N$, *D[i]* shall be an integral constant expression, and

(13.2.4) — the text of the *static_assert-message* is formed by the sequence of *N* code units, starting at *D*, of the ordinary literal encoding (5.3.1).

- ¹⁴ *Recommended practice*: When a *static_assert-declaration* fails, the resulting diagnostic message should include the text of the *static_assert-message*, if one is supplied.

[Example 5:

```
static_assert(sizeof(int) == sizeof(void*), "wrong pointer size");
static_assert(sizeof(int[2]));           // OK, narrowing allowed
```



```

template <class T>
void f(T t) {
    if constexpr (sizeof(T) == sizeof(int)) {
        use(t);
    } else {
        static_assert(false, "must be int-sized");
    }
}

void g(char c) {
    f(0);           // OK
    f(c);           // error on implementations where sizeof(int) > 1: must be int-sized
}

```

— end example]

- 15 For a *constexpr-block-declaration* D , the expression E corresponding to D is:

```
[] static constexpr -> void compound-statement ()
```

E shall be a constant expression (7.7.3).

[Note 7: The evaluation of the expression corresponding to a *constexpr-block-declaration* (5.2) can produce injected declarations (7.7.6) as side effects. — end note]

[Example 6:

```

struct S;
constexpr {
    std::meta::define_aggregate(~~S, {});    // OK

    template<class T>
    struct X { };                          // error: local templates are not allowed

    template<class T>
    concept C = true;                      // error: local concepts are not allowed

    return;                                // OK
}

```

— end example]

- 16 An *empty-declaration* has no effect.
- 17 Except where otherwise specified, the meaning of an *attribute-declaration* is implementation-defined.

9.2 Specifiers

[dcl.spec]

9.2.1 General

[dcl.spec.general]

- 1 The specifiers that can be used in a declaration are

decl-specifier:

```

storage-class-specifier
defining-type-specifier
function-specifier
friend
typedef
constexpr
constexpr
constexpr
inline

```

decl-specifier-seq:

```

decl-specifier attribute-specifier-seqopt
decl-specifier decl-specifier-seq

```

The optional *attribute-specifier-seq* in a *decl-specifier-seq* appertains to the type determined by the preceding *decl-specifiers* (9.3.4). The *attribute-specifier-seq* affects the type only for the declaration it appears in, not other declarations involving the same type.

- 2 At most one of each of the *decl-specifiers* **friend**, **typedef**, or **inline** shall appear in a *decl-specifier-seq*. At most one of the **constexpr**, **constexpr**, and **constexpr** keywords shall appear in a *decl-specifier-seq*.

- ³ If a *type-name* is encountered while parsing a *decl-specifier-seq*, it is interpreted as part of the *decl-specifier-seq* if and only if there is no previous *defining-type-specifier* other than a *cv-qualifier* in the *decl-specifier-seq*. The sequence shall be self-consistent as described below.

[Example 1:

```
typedef char* Pc;
static Pc;           // error: name missing
```

Here, the declaration `static Pc` is ill-formed because no name was specified for the static variable of type `Pc`. To get a variable called `Pc`, a *type-specifier* (other than `const` or `volatile`) has to be present to indicate that the *typedef-name* `Pc` is the name being (re)declared, rather than being part of the *decl-specifier* sequence. For another example,

```
void f(const Pc);           // void f(char* const) (not const char*)
void g(const int Pc);       // void g(const int)
```

— end example]

- ⁴ [Note 1: Since `signed`, `unsigned`, `long`, and `short` by default imply `int`, a *type-name* appearing after one of those specifiers is treated as the name being (re)declared.

[Example 2:

```
void h(unsigned Pc);       // void h(unsigned int)
void k(unsigned int Pc);   // void k(unsigned int)
```

— end example]

— end note]

9.2.2 Storage class specifiers

[dcl.stc]

- ¹ The storage class specifiers are

```
storage-class-specifier:
    static
    thread_local
    extern
    mutable
```

At most one *storage-class-specifier* shall appear in a given *decl-specifier-seq*, except that `thread_local` may appear with `static` or `extern`. If `thread_local` appears in any declaration of a variable it shall be present in all declarations of that entity. If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* of the *simple-declaration* or the *member-declarator-list* of the *member-declaration* shall not be empty (except for an anonymous union declared in a namespace scope (11.5.2)). The *storage-class-specifier* applies to the name declared by each *init-declarator* in the list and not to any names declared by other specifiers.

[Note 1: See 13.9.4 and 13.9.3 for restrictions in explicit specializations and explicit instantiations, respectively. — end note]

- ² [Note 2: A variable declared without a *storage-class-specifier* at block scope or declared as a function parameter has automatic storage duration by default (6.8.6.4). — end note]
- ³ The `thread_local` specifier indicates that the named entity has thread storage duration (6.8.6.3). It shall be applied only to the declaration of a variable of namespace or block scope, to a structured binding declaration (9.7), or to the declaration of a static data member. When `thread_local` is applied to a variable of block scope the *storage-class-specifier* `static` is implied if no other *storage-class-specifier* appears in the *decl-specifier-seq*.
- ⁴ The `static` specifier shall be applied only to the declaration of a variable or function, to a structured binding declaration (9.7), or to the declaration of an anonymous union (11.5.2). There can be no `static` function declarations within a block, nor any `static` function parameters. A `static` specifier used in the declaration of a variable declares the variable to have static storage duration (6.8.6.2), unless accompanied by the `thread_local` specifier, which declares the variable to have thread storage duration (6.8.6.3). A `static` specifier can be used in declarations of class members; 11.4.9 describes its effect. For the linkage of a name declared with a `static` specifier, see 6.7.
- ⁵ The `extern` specifier shall be applied only to the declaration of a variable or function. The `extern` specifier shall not be used in the declaration of a class member or function parameter. For the linkage of a name declared with an `extern` specifier, see 6.7.

[Note 3: The **extern** keyword can also be used in *explicit-instantiations* and *linkage-specifications*, but it is not a *storage-class-specifier* in such contexts. — end note]

- 6 All declarations for a given entity shall give its name the same linkage.

[Note 4: The linkage given by some declarations is affected by previous declarations. Overloads are distinct entities. — end note]

[Example 1:

```
static char* f();           // f() has internal linkage
char* f()                  // f() still has internal linkage
{ /* ... */ }

char* g();                 // g() has external linkage
static char* g()           // error: inconsistent linkage
{ /* ... */ }

void h();
inline void h();           // external linkage

inline void l();
void l();                  // external linkage

inline void m();
extern void m();           // external linkage

static void n();
inline void n();           // internal linkage

static int a;
int a;                     // a has internal linkage
                        // error: two definitions

static int b;
extern int b;              // b has internal linkage
                        // b still has internal linkage

int c;
static int c;              // c has external linkage
                        // error: inconsistent linkage

extern int d;
static int d;              // d has external linkage
                        // error: inconsistent linkage
```

— end example]

- 7 The name of a declared but undefined class can be used in an **extern** declaration. Such a declaration can only be used in ways that do not require a complete class type.

[Example 2:

```
struct S;
extern S a;
extern S f();
extern void g(S);

void h() {
    g(a);                  // error: S is incomplete
    f();                   // error: S is incomplete
}
```

— end example]

- 8 The **mutable** specifier shall appear only in the declaration of a non-static data member (11.4) whose type is neither **const**-qualified nor a reference type.

[Example 3:

```
class X {
    mutable const int* p;   // OK
    mutable int* const q;  // error
};
```

— end example]

- ⁹ [Note 5: The **mutable** specifier on a class data member nullifies a **const** specifier applied to the containing class object and permits modification of the mutable class member even though the rest of the object is **const** (6.9.6, 9.2.9.2). — end note]

9.2.3 Function specifiers

[dcl.fct.spec]

- ¹ A *function-specifier* can be used only in a function declaration. At most one *explicit-specifier* and at most one **virtual** keyword shall appear in a *decl-specifier-seq*.

```
function-specifier:
    virtual
    explicit-specifier

explicit-specifier:
    explicit ( constant-expression )
    explicit
```

- ² The **virtual** specifier shall be used only in the initial declaration of a non-static member function; see 11.7.3.
- ³ An *explicit-specifier* shall be used only in the declaration of a constructor or conversion function within its class definition; see 11.4.8.2 and 11.4.8.3.
- ⁴ In an *explicit-specifier*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type **bool** (7.7.3). The *explicit-specifier* **explicit** without a *constant-expression* is equivalent to the *explicit-specifier* **explicit(true)**. If the constant expression evaluates to **true**, the function is explicit. Otherwise, the function is not explicit. A (token that follows **explicit** is parsed as part of the *explicit-specifier*.

[Example 1:

```
struct S {
    explicit(sizeof(char[2])) S(char);    // error: narrowing conversion of value 2 to type bool
    explicit(sizeof(char)) S(bool);      // OK, conversion of value 1 to type bool is non-narrowing
};
```

— end example]

9.2.4 The typedef specifier

[dcl.typedef]

- ¹ Declarations containing the *decl-specifier* **typedef** declare *type aliases*. The **typedef** specifier shall not be combined in a *decl-specifier-seq* with any other kind of specifier except a *defining-type-specifier*, and it shall not be used in the *decl-specifier-seq* of a *parameter-declaration* (9.3.4.6) nor in the *decl-specifier-seq* of a *function-definition* (9.6). If a **typedef** specifier appears in a declaration without a *declarator*, the program is ill-formed.

```
typedef-name:
    identifier
    simple-template-id
```

A name declared with the **typedef** specifier becomes a *typedef-name*. The underlying entity of the type alias is the type associated with the *identifier* (9.3) or *simple-template-id* (13.1). A *typedef-name* does not introduce a new type the way a class declaration (11.3) or enum declaration (9.8.1) does.

[Example 1: After

```
typedef int MILES, *KCLICKSP;
```

the constructions

```
MILES distance;
extern KCLICKSP metricp;
```

are all correct declarations; the type of **distance** is **int** and that of **metricp** is “pointer to **int**”. — end example]

- ² A type alias can also be declared by an *alias-declaration*. The *identifier* following the **using** keyword is not looked up; it becomes the *typedef-name* of a type alias and the optional *attribute-specifier-seq* following the *identifier* appertains to that type alias. Such a type alias has the same semantics as if it were introduced by the **typedef** specifier.

[Example 2:

```
using handler_t = void (*)(int);
extern handler_t ignore;
extern void (*ignore)(int);    // redeclare ignore
```

```
template<class T> struct P { };
using cell = P<cell*>;           // error: cell not found (6.4.2)
— end example]
```

The *defining-type-specifier-seq* of the *defining-type-id* shall not define a class or enumeration if the *alias-declaration* is the *declaration* of a *template-declaration*.

- ³ A *simple-template-id* is only a *typedef-name* if its *template-name* names an alias template or a type template template parameter.

[*Note 1*: A *simple-template-id* that names a class template specialization is a *class-name* (11.3). If a *typedef-name* is used to identify the subject of an *elaborated-type-specifier* (9.2.9.5), a class definition (Clause 11), a constructor declaration (11.4.5), or a destructor declaration (11.4.7), the program is ill-formed. — end note]

[*Example 3*:

```
struct S {
    S();
    ~S();
};

typedef struct S T;

S a = T();           // OK
struct T * p;         // error
```

— end example]

- ⁴ An unnamed class or enumeration *C* defined in a typedef declaration has the first *typedef-name* declared by the declaration to be of type *C* as its *typedef name for linkage purposes* (6.7).

[*Note 2*: A typedef declaration involving a *lambda-expression* does not itself define the associated closure type, and so the closure type is not given a typedef name for linkage purposes. — end note]

[*Example 4*:

```
typedef struct { } *ps, S;       // S is the typedef name for linkage purposes
typedef decltype([]{}) C;       // the closure type has no typedef name for linkage purposes
```

— end example]

- ⁵ An unnamed class with a typedef name for linkage purposes shall not

- (5.1) — declare any members other than non-static data members, member enumerations, or member classes,
- (5.2) — have any base classes or default member initializers, or
- (5.3) — contain a *lambda-expression*,

and all member classes shall also satisfy these requirements (recursively).

[*Example 5*:

```
typedef struct {
    int f() {}
} X;           // error: struct with typedef name for linkage has member functions
```

— end example]

9.2.5 The friend specifier

[dcl.friend]

- ¹ The **friend** specifier is used to specify access to class members; see 11.8.4.

9.2.6 The constexpr and consteval specifiers

[dcl.constexpr]

- ¹ The **constexpr** specifier shall be applied only to the definition of a variable or variable template, a structured binding declaration, or the declaration of a function or function template. The **constexpr** specifier shall be applied only to the declaration of a function or function template. A function or static data member declared with the **constexpr** or **constexpr** specifier on its first declaration is implicitly an inline function or variable (9.2.8). If any declaration of a function or function template has a **constexpr** or **constexpr** specifier, then all its declarations shall contain the same specifier.

[*Note 1*: An explicit specialization can differ from the template declaration with respect to the **constexpr** or **constexpr** specifier. — end note]

[*Note 2*: Function parameters cannot be declared **constexpr**. — end note]

[Example 1:

```
constexpr void square(int &x); // OK, declaration
constexpr int bufsz = 1024;   // OK, definition
constexpr struct pixel {      // error: pixel is a type
    int x;
    int y;
    constexpr pixel(int);      // OK, declaration
};
constexpr pixel::pixel(int a)
    : x(a), y(x)               // OK, definition
    { square(x); }
constexpr pixel small(2);      // error: square not defined, so small(2)
                                // not constant (7.7.2) so constexpr not satisfied

constexpr void square(int &x) { // OK, definition
    x *= x;
}
constexpr pixel large(4);      // OK, square defined
int next(constexpr int x) {    // error: not for parameters
    return x + 1;
}
extern constexpr int memsz;    // error: not a definition
```

— end example]

- ² A `constexpr` or `constexpr` specifier used in the declaration of a function declares that function to be a *constexpr function*.

[Note 3: A function declared with the `constexpr` specifier is an immediate function (7.7.5). — end note]

A destructor, an allocation function, or a deallocation function shall not be declared with the `constexpr` specifier.

- ³ A function is *constexpr-suitable* if it is not a coroutine (9.6.4).

Except for instantiated `constexpr` functions, non-templated `constexpr` functions shall be *constexpr-suitable*.

[Example 2:

```
constexpr int square(int x)
{ return x * x; } // OK
constexpr long long_max()
{ return 2147483647; } // OK
constexpr int abs(int x) {
    if (x < 0)
        x = -x;
    return x; // OK
}
constexpr int constant_non_42(int n) { // OK
    if (n == 42) {
        static int value = n;
        return value;
    }
    return n;
}
constexpr int uninit() {
    struct { int a; } s;
    return s.a; // error: uninitialized read of s.a
}
constexpr int prev(int x)
{ return --x; } // OK
constexpr int g(int x, int n) { // OK
    int r = 1;
    while (--n > 0) r *= x;
    return r;
}
```

— end example]

- ⁴ An invocation of a constexpr function in a given context produces the same result as an invocation of an equivalent non-constexpr function in the same context in all respects except that

- (4.1) — an invocation of a constexpr function can appear in a constant expression (7.7.2) and
- (4.2) — copy elision is not performed in a constant expression (11.9.6).

[*Note 4:* Declaring a function constexpr can change whether an expression is a constant expression. This can indirectly cause calls to `std::is_constant_evaluated` within an invocation of the function to produce a different value. — *end note*]

[*Note 5:* It is possible to write a constexpr function for which no invocation satisfies the requirements of a core constant expression. — *end note*]

- ⁵ The `constexpr` and `constexpr` specifiers have no effect on the type of a constexpr function.

[*Example 3:*

```
constexpr int bar(int x, int y)           // OK
{ return x + y + x*y; }
// ...
int bar(int x, int y)                     // error: redefinition of bar
{ return x * 2 + 3 * y; }
```

— *end example*]

- ⁶ A `constexpr` specifier used in an object declaration declares the object as `const`. Such an object shall have literal type and shall be initialized. A `constexpr` variable shall be constant-initializable (7.7.4). A `constexpr` variable that is an object, as well as any temporary to which a `constexpr` reference is bound, shall have constant destruction.

[*Example 4:*

```
struct pixel {
    int x, y;
};
constexpr pixel ur = { 1294, 1024 };      // OK
constexpr pixel origin;                  // error: initializer missing

namespace N {
    void f() {
        int x;
        constexpr int& ar = x;           // OK
        static constexpr int& sr = x;     // error: x is not constexpr-representable
                                           // at the point indicated below
    }
    // immediate scope here is that of N
}
```

— *end example*]

9.2.7 The `constinit` specifier

[`dcl.constinit`]

- ¹ The `constinit` specifier shall be applied only to a declaration of a variable with static or thread storage duration or to a structured binding declaration (9.7).

[*Note 1:* A structured binding declaration introduces a uniquely named variable, to which the `constinit` specifier applies. — *end note*]

If the specifier is applied to any declaration of a variable, it shall be applied to the initializing declaration. No diagnostic is required if no `constinit` declaration is reachable at the point of the initializing declaration (G.6.1).

- ² If a variable declared with the `constinit` specifier has dynamic initialization (6.10.3.3), the program is ill-formed, even if the implementation would perform that initialization as a static initialization (6.10.3.2).

[*Note 2:* The `constinit` specifier ensures that the variable is initialized during static initialization. — *end note*]

- ³ [*Example 1:*

```
const char * g() { return "dynamic initialization"; }
constexpr const char * f(bool p) { return p ? "constant initializer" : g(); }
constinit const char * c = f(true);      // OK
constinit const char * d = f(false);     // error
```


— *end example*]

9.2.8 The inline specifier

[dcl.inline]

- ¹ The **inline** specifier shall be applied only to the declaration of a function or variable. The **inline** specifier shall not appear on a block scope declaration or on the declaration of a function parameter. If the **inline** specifier is used in a friend function declaration, that declaration shall be a definition or the function shall have previously been declared inline.
- ² A function declaration (9.3.4.6, 11.4.2, 11.8.4) with an **inline** specifier declares an *inline function*. A variable declaration with an **inline** specifier declares an *inline variable*.

[*Note 1*: An inline function or variable with external or module linkage can be defined in multiple translation units (6.3), but is one entity with one address. A type or **static** variable defined in the body of such a function is therefore a single entity. — *end note*]

[*Note 2*: The **inline** keyword has no effect on the linkage of a function. In certain cases, an inline function cannot use names with internal linkage; see 6.7. — *end note*]

- ³ The **inline** specifier indicates to the implementation that inline substitution of the function body at the point of call is to be preferred to the usual function call mechanism. An implementation is not required to perform this inline substitution at the point of call; however, even if this inline substitution is omitted, the other rules for inline functions specified in this subclause shall still be respected.
- ⁴ If a definition of a function or variable is reachable at the point of its first declaration as inline, the program is ill-formed. If a function or variable with external or module linkage is declared inline in one definition domain, an inline declaration of it shall be reachable from the end of every definition domain in which it is declared; no diagnostic is required (G.6.2).

[*Note 3*: A call to an inline function or a use of an inline variable can be encountered before its definition becomes reachable in a translation unit. — *end note*]

- ⁵ If an inline function or variable that is attached to a named module is declared in a definition domain, it shall be defined in that domain.

[*Note 4*: A constexpr function (9.2.6) is implicitly inline. In the global module, a function defined within a class definition is implicitly inline (11.4.2, 11.8.4). — *end note*]

9.2.9 Type specifiers

[dcl.type]

9.2.9.1 General

[dcl.type.general]

- ¹ The type-specifiers are

type-specifier:

simple-type-specifier
elaborated-type-specifier
typename-specifier
cv-qualifier

type-specifier-seq:

type-specifier attribute-specifier-seq_{opt}
type-specifier type-specifier-seq

defining-type-specifier:

type-specifier
class-specifier
enum-specifier

defining-type-specifier-seq:

defining-type-specifier attribute-specifier-seq_{opt}
defining-type-specifier defining-type-specifier-seq

The optional *attribute-specifier-seq* in a *type-specifier-seq* or a *defining-type-specifier-seq* appertains to the type denoted by the preceding *type-specifiers* or *defining-type-specifiers* (9.3.4). The *attribute-specifier-seq* affects the type only for the declaration it appears in, not other declarations involving the same type.

- ² As a general rule, at most one *defining-type-specifier* is allowed in the complete *decl-specifier-seq* of a *declaration* or in a *defining-type-specifier-seq*, and at most one *type-specifier* is allowed in a *type-specifier-seq*. The only exceptions to this rule are the following:

(2.1) — **const** can be combined with any type specifier except itself.

- (2.2) — **volatile** can be combined with any type specifier except itself.
 - (2.3) — **signed** or **unsigned** can be combined with **char**, **long**, **short**, or **int**.
 - (2.4) — **short** or **long** can be combined with **int**.
 - (2.5) — **long** can be combined with **double**.
 - (2.6) — **long** can be combined with **long**.
- 3 Except in a declaration of a constructor, destructor, or conversion function, at least one *defining-type-specifier* that is not a *cv-qualifier* shall appear in a complete *type-specifier-seq* or a complete *decl-specifier-seq*.
- 4 [Note 1: *enum-specifiers*, *class-specifiers*, and *typename-specifiers* are discussed in 9.8.1, Clause 11, and 13.8, respectively. The remaining *type-specifiers* are discussed in the rest of 9.2.9. — end note]

9.2.9.2 The *cv-qualifiers*

[dcl.type.cv]

- 1 There are two *cv-qualifiers*, **const** and **volatile**. Each *cv-qualifier* shall appear at most once in a *cv-qualifier-seq*. If a *cv-qualifier* appears in a *decl-specifier-seq*, the *init-declarator-list* of the *simple-declaration* or the *member-declarator-list* of the *member-declaration* shall not be empty.

[Note 1: 6.9.6 and 9.3.4.6 describe how *cv-qualifiers* affect object and function types. — end note]

Redundant *cv-qualifications* are ignored.

[Note 2: For example, these could be introduced by typedefs. — end note]

- 2 [Note 3: Declaring a variable **const** can affect its linkage (9.2.2) and its usability in constant expressions (7.7.4). As described in 9.5, the definition of an object or subobject of *const*-qualified type must specify an initializer or be subject to default-initialization. — end note]

- 3 A pointer or reference to a *cv*-qualified type need not actually point or refer to a *cv*-qualified object, but it is treated as if it does; a *const*-qualified access path cannot be used to modify an object even if the object referenced is a non-*const* object and can be modified through some other access path.

[Note 4: *Cv-qualifiers* are supported by the type system so that they cannot be subverted without casting (7.6.1.11). — end note]

- 4 Any attempt to modify (7.6.19, 7.6.1.6, 7.6.2.3) a *const* object (6.9.6) during its lifetime (6.8.4) results in undefined behavior (F.5.1).

[Example 1:

```

const int ci = 3;           // cv-qualified (initialized as required)
ci = 4;                    // error: attempt to modify const

int i = 2;                 // not cv-qualified
const int* cip;            // pointer to const int
cip = &i;                  // OK, cv-qualified access path to unqualified
*cip = 4;                  // error: attempt to modify through ptr to const

int* ip;
ip = const_cast<int*>(cip); // cast needed to convert const int* to int*
*ip = 4;                  // defined: *ip points to i, a non-const object

const int* ciq = new const int (3); // initialized as required
int* iq = const_cast<int*>(ciq);    // cast required
*iq = 4;                          // undefined behavior: modifies a const object

```

For another example,

```

struct X {
    mutable int i;
    int j;
};
struct Y {
    X x;
    Y();
};

const Y y;
y.x.i++; // well-formed: mutable member can be modified
y.x.j++; // error: const-qualified member modified

```

```

Y* p = const_cast<Y*>(&y);           // cast away const-ness of y
p->x.i = 99;                         // well-formed: mutable member can be modified
p->x.j = 99;                         // undefined behavior: modifies a const subobject
— end example]

```

- ⁵ The semantics of an access through a volatile glvalue are implementation-defined. If an attempt is made to access an object defined with a volatile-qualified type through the use of a non-volatile glvalue, the behavior is undefined (F.5.2).
- ⁶ [Note 5: **volatile** is a hint to the implementation to avoid aggressive optimization involving the object because it is possible for the value of the object to change by means undetectable by an implementation. Furthermore, for some implementations, **volatile** can indicate that special hardware instructions are needed to access the object. See 6.10.1 for detailed semantics. In general, the semantics of **volatile** are intended to be the same in C++ as they are in C. — end note]

9.2.9.3 Simple type specifiers

[dcl.type.simple]

- ¹ The simple type specifiers are

```

simple-type-specifier:
    nested-name-specifieropt type-name
    nested-name-specifier template simple-template-id
    computed-type-specifier
    placeholder-type-specifier
    nested-name-specifieropt simple-template-name
    pack-index-template-name
    char
    char8_t
    char16_t
    char32_t
    wchar_t
    bool
    short
    int
    long
    signed
    unsigned
    float
    double
    void

type-name:
    class-name
    enum-name
    typedef-name

computed-type-specifier:
    decltype-specifier
    pack-index-specifier
    splice-type-specifier

```

- ² The component names of a *simple-type-specifier* are those of its *nested-name-specifier*, *type-name*, *simple-template-id*, *simple-template-name*, and/or *type-constraint* (if it is a *placeholder-type-specifier*). The component name of a *type-name* is the first name in it.
- ³ A *placeholder-type-specifier* is a placeholder for a type to be deduced (9.2.9.7). A *type-specifier* is a placeholder for a deduced class type (9.2.9.8) if
- (3.1) — it is of the form **typename**_{opt} *nested-name-specifier*_{opt} *simple-template-name*, or
 - (3.2) — it is of the form **typename**_{opt} *pack-index-template-name*, or
 - (3.3) — it is of the form **typename**_{opt} *splice-specifier* and the *splice-specifier* designates a class template or alias template.

The *nested-name-specifier* or *splice-specifier*, if any, shall be non-dependent and the *simple-template-name*, *pack-index-template-name*, or *splice-specifier* shall designate a deducible template. A *deducible template* is

- (3.4) — a class template,
- (3.5) — a type template template parameter, or

- (3.6) — an alias template **A** whose *defining-type-id* is of the form

`typenameopt nested-name-specifieropt templateopt simple-template-id`

where the *nested-name-specifier* (if any) is non-dependent and the *template-name* of the *simple-template-id* names a deducible template other than a type template template parameter of **A**.

[*Note 1*: An injected-class-name is never interpreted as a *template-name* in contexts where class template argument deduction would be performed (13.8.2). — *end note*]

The other *simple-type-specifiers* specify either a previously-declared type, a type determined from an expression, or one of the fundamental types (6.9.3). Table 17 summarizes the valid combinations of *simple-type-specifiers* and the types they specify.

Table 17 — *simple-type-specifiers* and the types they specify [tab:dcl.type.simple]

Specifier(s)	Type
<i>type-name</i>	the type named
<i>simple-template-id</i>	the type as defined in 13.3
<i>decltype-specifier</i>	the type as defined in 9.2.9.6
<i>pack-index-specifier</i>	the type as defined in 9.2.9.4
<i>placeholder-type-specifier</i>	the type as defined in 9.2.9.7
<i>template-name</i>	the type as defined in 9.2.9.8
<i>splice-type-specifier</i>	the type as defined in 9.2.9.9
<code>char</code>	"char"
<code>unsigned char</code>	"unsigned char"
<code>signed char</code>	"signed char"
<code>char8_t</code>	"char8_t"
<code>char16_t</code>	"char16_t"
<code>char32_t</code>	"char32_t"
<code>bool</code>	"bool"
<code>unsigned</code>	"unsigned int"
<code>unsigned int</code>	"unsigned int"
<code>signed</code>	"int"
<code>signed int</code>	"int"
<code>int</code>	"int"
<code>unsigned short int</code>	"unsigned short int"
<code>unsigned short</code>	"unsigned short int"
<code>unsigned long int</code>	"unsigned long int"
<code>unsigned long</code>	"unsigned long int"
<code>unsigned long long int</code>	"unsigned long long int"
<code>unsigned long long</code>	"unsigned long long int"
<code>signed long int</code>	"long int"
<code>signed long</code>	"long int"
<code>signed long long int</code>	"long long int"
<code>signed long long</code>	"long long int"
<code>long long int</code>	"long long int"
<code>long long</code>	"long long int"
<code>long int</code>	"long int"
<code>long</code>	"long int"
<code>signed short int</code>	"short int"
<code>signed short</code>	"short int"
<code>short int</code>	"short int"
<code>short</code>	"short int"
<code>wchar_t</code>	"wchar_t"
<code>float</code>	"float"
<code>double</code>	"double"
<code>long double</code>	"long double"
<code>void</code>	"void"

- ⁴ When multiple *simple-type-specifiers* are allowed, they can be freely intermixed with other *decl-specifiers* in any order.

[*Note 2*: It is implementation-defined whether objects of `char` type are represented as signed or unsigned quantities. The `signed` specifier forces `char` objects to be signed; it is redundant in other contexts. — *end note*]

9.2.9.4 Pack indexing specifier

[`dcl.type.pack.index`]

pack-index-specifier:

`typedef-name ... [ constant-expression ]`

- ¹ The *typedef-name* *P* in a *pack-index-specifier* shall be an *identifier* that denotes a pack.
- ² The *constant-expression* shall be a converted constant expression (7.7.3) of type `std::size_t` whose value *V*, termed the index, is such that $0 \leq V < \text{sizeof} \dots (P)$.
- ³ A *pack-index-specifier* is a pack expansion (13.7.4).
- ⁴ [*Note 1*: The *pack-index-specifier* denotes the type of the V^{th} element of the pack. — *end note*]

9.2.9.5 Elaborated type specifiers

[`dcl.type.elab`]

elaborated-type-specifier:

`class-key attribute-specifier-seqopt nested-name-specifieropt identifier`

`class-key simple-template-id`

`class-key nested-name-specifier templateopt simple-template-id`

`enum nested-name-specifieropt identifier`

- ¹ The component names of an *elaborated-type-specifier* are its *identifier* (if any) and those of its *nested-name-specifier* and *simple-template-id* (if any).
- ² If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization (13.9.4), a partial specialization (13.7.6), an explicit instantiation (13.9.3), or it has one of the following forms:

`class-key attribute-specifier-seqopt identifier ;`

`class-key attribute-specifier-seqopt simple-template-id ;`

In the first case, the *elaborated-type-specifier* declares the *identifier* as a *class-name*. The second case shall appear only in an *explicit-specialization* (13.9.4) or in a *template-declaration* (where it declares a partial specialization). The *attribute-specifier-seq*, if any, appertains to the class or template being declared.

- ³ Otherwise, an *elaborated-type-specifier* *E* shall not have an *attribute-specifier-seq*. If *E* contains an *identifier* but no *nested-name-specifier* and (unqualified) lookup for the *identifier* finds nothing, *E* shall not be introduced by the `enum` keyword and declares the *identifier* as a *class-name*. The target scope of *E* is the nearest enclosing namespace or block scope.
- ⁴ A *friend-type-specifier* that is an *elaborated-type-specifier* shall have one of the following forms:

`class-key nested-name-specifieropt identifier`

`class-key simple-template-id`

`class-key nested-name-specifier templateopt simple-template-id`

Any unqualified lookup for the *identifier* (in the first case) does not consider scopes that contain the nearest enclosing namespace or block scope; no name is bound.

[*Note 1*: A *using-directive* in the target scope is ignored if it refers to a namespace not contained by that scope. — *end note*]

- ⁵ [*Note 2*: 6.5.6 describes how name lookup proceeds in an *elaborated-type-specifier*. An *elaborated-type-specifier* can be used to refer to a previously declared *class-name* or *enum-name* even if the name has been hidden by a non-type declaration. — *end note*]
- ⁶ If the *identifier* or *simple-template-id* in an *elaborated-type-specifier* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name* (9.2.9.3). If the *identifier* or *simple-template-id* resolves to a *typedef-name* (9.2.4, 13.3), the *elaborated-type-specifier* is ill-formed.

[*Note 3*: This implies that, within a class template with a template *type-parameter* *T*, the declaration

`friend class T;`

is ill-formed. However, the similar declaration `friend T;` is well-formed (11.8.4). — *end note*]

- ⁷ The *class-key* or **enum** keyword present in an *elaborated-type-specifier* shall agree in kind with the declaration to which the name in the *elaborated-type-specifier* refers. This rule also applies to the form of *elaborated-type-specifier* that declares a *class-name* or friend class since it can be construed as referring to the definition of the class. Thus, in any *elaborated-type-specifier*, the **enum** keyword shall be used to refer to an enumeration (9.8.1), the **union** *class-key* shall be used to refer to a union (11.5), and either the **class** or **struct** *class-key* shall be used to refer to a non-union class (11.1).

[Example 1:

```
enum class E { a, b };
enum E x = E::a;           // OK
struct S { } s;
class S* p = &s;           // OK
```

— end example]

9.2.9.6 Decltype specifiers

[dcl.type.decltype]

decltype-specifier:

`decltype ( expression )`

- ¹ For an expression *E*, the type denoted by `decltype(E)` is defined as follows:

- (1.1) — if *E* is an unparenthesized *id-expression* naming a structured binding (9.7), `decltype(E)` is the referenced type as given in the specification of the structured binding declaration;
- (1.2) — otherwise, if *E* is an unparenthesized *id-expression* naming a constant template parameter (13.2), `decltype(E)` is the type of the template parameter after performing any necessary type deduction (9.2.9.7, 9.2.9.8);
- (1.3) — otherwise, if *E* is an unparenthesized *id-expression* or an unparenthesized class member access (7.6.1.5), `decltype(E)` is the type of the entity named by *E*. If there is no such entity, the program is ill-formed;
- (1.4) — otherwise, if *E* is an unparenthesized *splice-expression*, `decltype(E)` is the type of the entity, object, or value designated by the *splice-specifier* of *E*;
- (1.5) — otherwise, if *E* is an xvalue, `decltype(E)` is `T&&`, where *T* is the type of *E*;
- (1.6) — otherwise, if *E* is an lvalue, `decltype(E)` is `T&`, where *T* is the type of *E*;
- (1.7) — otherwise, `decltype(E)` is the type of *E*.

The operand of the `decltype` specifier is an unevaluated operand (7.2.3).

[Example 1:

```
const int&& foo();
int i;
struct A { double x; };
const A* a = new A();
decltype(foo()) x1 = 17;      // type is const int&&
decltype(i) x2;              // type is int
decltype(a->x) x3;           // type is double
decltype((a->x)) x4 = x3;     // type is const double&
decltype([:~x1:]) x5 = 18;   // type is const int&&
decltype([[:~x1:]]) x6 = 19; // type is const int&

void f() {
    [](auto ...pack) {
        decltype(pack...[0]) x7; // type is int
        decltype((pack...[0])) x8; // type is int&
    }(0);
}
```

— end example]

[Note 1: The rules for determining types involving `decltype(auto)` are specified in 9.2.9.7. — end note]

- ² If the operand of a *decltype-specifier* is a prvalue and is not a (possibly parenthesized) immediate invocation (7.7.5), the temporary materialization conversion is not applied (7.3.5) and no result object is provided for the prvalue. The type of the prvalue may be incomplete or an abstract class type.

[*Note 2*: As a result, storage is not allocated for the prvalue and it is not destroyed. Thus, a class type is not instantiated as a result of being the type of a function call in this context. In this context, the common purpose of writing the expression is merely to refer to its type. In that sense, a *decltype-specifier* is analogous to a use of a *typedef-name*, so the usual reasons for requiring a complete type do not apply. In particular, it is not necessary to allocate storage for a temporary object or to enforce the semantic constraints associated with invoking the type's destructor. — *end note*]

[*Note 3*: Unlike the preceding rule, parentheses have no special meaning in this context. — *end note*]

[*Example 2*:

```
template<class T> struct A { ~A() = delete; };
template<class T> auto h()
    -> A<T>;
template<class T> auto i(T)      // identity
    -> T;
template<class T> auto f(T)      // #1
    -> decltype(i(h<T>()));      // forces completion of A<T> and implicitly uses A<T>::~~A()
                                // for the temporary introduced by the use of h().
                                // (A temporary is not introduced as a result of the use of i().)
template<class T> auto f(T)      // #2
    -> void;
auto g() -> void {
    f(42);                      // OK, calls #2. (#1 is not a viable candidate: type deduction
                                // fails (13.10.3) because A<int>::~~A() is implicitly used in its
                                // decltype-specifier)
}
template<class T> auto q(T)
    -> decltype(h<T>());          // does not force completion of A<T>; A<T>::~~A() is not implicitly
                                // used within the context of this decltype-specifier
void r() {
    q(42);                      // error: deduction against q succeeds, so overload resolution selects
                                // the specialization "q(T) -> decltype(h<T>())" with T=int;
                                // the return type is A<int>, so a temporary is introduced and its
                                // destructor is used, so the program is ill-formed
}
```

— *end example*]

9.2.9.7 Placeholder type specifiers

[dcl.spec.auto]

9.2.9.7.1 General

[dcl.spec.auto.general]

placeholder-type-specifier:
*type-constraint*_{opt} auto
*type-constraint*_{opt} decltype (auto)

¹ A *placeholder-type-specifier* designates a placeholder type that will be replaced later, typically by deduction from an initializer.

² The type of a *parameter-declaration* of a

- (2.1) — function declaration (9.3.4.6),
- (2.2) — *lambda-expression* (7.5.6), or
- (2.3) — *template-parameter* (13.2)

can be declared using a *placeholder-type-specifier* of the form *type-constraint*_{opt} auto. The placeholder type shall appear as one of the *decl-specifiers* in the *decl-specifier-seq* or as one of the *type-specifiers* in a *trailing-return-type* that specifies the type that replaces such a *decl-specifier* (see below); the placeholder type is a *generic parameter type placeholder* of the function declaration, *lambda-expression*, or *template-parameter*, respectively.

[*Note 1*: Having a generic parameter type placeholder signifies that the function is an abbreviated function template (9.3.4.6) or the lambda is a generic lambda (7.5.6). — *end note*]

³ A placeholder type can appear in the *decl-specifier-seq* for a function declarator that includes a *trailing-return-type* (9.3.4.6).

- ⁴ A placeholder type can appear in the *decl-specifier-seq* or *type-specifier-seq* in the declared return type of a function declarator that declares a function other than a conversion function (11.4.8.3); the return type of the function is deduced from non-discarded **return** statements, if any, in the body of the function (8.5.2).
- ⁵ The type of a variable declared using a placeholder type is deduced from its initializer. This use is allowed in an initializing declaration (9.5) of a variable. The placeholder type shall appear as one of the *decl-specifiers* in the *decl-specifier-seq* or as one of the *type-specifiers* in a *trailing-return-type* that specifies the type that replaces such a *decl-specifier*; the *decl-specifier-seq* shall be followed by one or more *declarators*, each of which shall be followed by a non-empty *initializer*.

[Example 1:

```

auto x = 5;           // OK, x has type int
const auto *v = &x, u = 6; // OK, v has type const int*, u has type const int
static auto y = 0.0; // OK, y has type double
auto int r;          // error: auto is not a storage-class-specifier
auto f() -> int;      // OK, f returns int
auto g() { return 0.0; } // OK, g returns double
auto (*fp)() -> auto = f; // OK
auto h();             // OK, h's return type will be deduced when it is defined

```

— end example]

The **auto** *type-specifier* can also be used to introduce a structured binding declaration (9.7).

- ⁶ A placeholder type can also be used in the *type-specifier-seq* of the *new-type-id* or in the *nofun-type-id* of a *new-expression* (7.6.2.8). In such a *nofun-type-id*, the placeholder type shall appear as one of the *type-specifiers* in the *type-specifier-seq* or as one of the *type-specifiers* in a *trailing-return-type* that specifies the type that replaces such a *type-specifier*.
- ⁷ The **auto** *type-specifier* can also be used as the *simple-type-specifier* in an explicit type conversion (functional notation) (7.6.1.4).
- ⁸ A program that uses a placeholder type in a context not explicitly allowed in 9.2.9.7 is ill-formed.
- ⁹ If the *init-declarator-list* contains more than one *init-declarator*, they shall all form declarations of variables. The type of each declared variable is determined by placeholder type deduction (9.2.9.7.2), and if the type that replaces the placeholder type is not the same in each deduction, the program is ill-formed.

[Example 2:

```

auto x = 5, *y = &x; // OK, auto is int
auto a = 5, b = { 1, 2 }; // error: different types for auto

```

— end example]

- ¹⁰ If a function with a declared return type that contains a placeholder type has multiple non-discarded **return** statements, the return type is deduced for each such **return** statement. If the type deduced is not the same in each deduction, the program is ill-formed.
- ¹¹ If a function with a declared return type that uses a placeholder type has no non-discarded **return** statements, the return type is deduced as though from a **return** statement with no operand at the closing brace of the function body.

[Example 3:

```

auto f() { } // OK, return type is void
auto* g() { } // error: cannot deduce auto* from void()

```

— end example]

- ¹² An exported function with a declared return type that uses a placeholder type shall be defined in the translation unit containing its exported declaration, outside the *private-module-fragment* (if any).

[Note 2: The deduced return type cannot have a name with internal linkage (6.7). — end note]

- ¹³ If a variable or function with an undeduced placeholder type is named by an expression (6.3), the program is ill-formed. Once a non-discarded **return** statement has been seen in a function, however, the return type deduced from that statement can be used in the rest of the function, including in other **return** statements.

[Example 4:

```

auto n = n; // error: n's initializer refers to n
auto f();
void g() { &f; } // error: f's return type is unknown

```

```

auto sum(int i) {
    if (i == 1)
        return i;                // sum's return type is int
    else
        return sum(i-1)+i;        // OK, sum's return type has been deduced
}

```

— end example]

- 14 A result binding never has an undeduced placeholder type (9.4.2).

[Example 5:

```

auto f()
    post(r : r == 7) // OK
{
    return 7;
}

```

— end example]

- 15 Return type deduction for a templated function with a placeholder in its declared type occurs when the definition is instantiated even if the function body contains a **return** statement with a non-type-dependent operand.

[Note 3: Therefore, any use of a specialization of the function template will cause an implicit instantiation. Any errors that arise from this instantiation are not in the immediate context of the function type and can result in the program being ill-formed (13.10.3). — end note]

[Example 6:

```

template <class T> auto f(T t) { return t; } // return type deduced at instantiation time
typedef decltype(f(1)) fint_t; // instantiates f<int> to deduce return type
template<class T> auto f(T* t) { return *t; }
void g() { int (*p)(int*) = &f; } // instantiates both fs to determine return types,
// chooses second

```

— end example]

- 16 If a function or function template *F* has a declared return type that uses a placeholder type, redeclarations or specializations of *F* shall use that placeholder type, not a deduced type; otherwise, they shall not use a placeholder type.

[Example 7:

```

auto f();
auto f() { return 42; } // return type is int
auto f(); // OK
int f(); // error: auto and int don't match
decltype(auto) f(); // error: auto and decltype(auto) don't match

template <typename T> auto g(T t) { return t; } // #1
template auto g(int); // OK, return type is int
template char g(char); // error: no matching template
template<> auto g(double); // OK, forward declaration with unknown return type

template <class T> T g(T t) { return t; } // OK, not functionally equivalent to #1
template char g(char); // OK, now there is a matching template
template auto g(float); // still matches #1

void h() { return g(42); } // error: ambiguous

template <typename T> struct A {
    friend T frf(T);
};
auto frf(int i) { return i; } // not a friend of A<int>
extern int v;
auto v = 17; // OK, redeclares v
struct S {
    static int i;
};

```

```
auto S::i = 23;                                // OK
— end example]
```

- 17 A function declared with a return type that uses a placeholder type shall not be **virtual** (11.7.3).
- 18 A function declared with a return type that uses a placeholder type shall not be a coroutine (9.6.4).
- 19 An explicit instantiation declaration (13.9.3) does not cause the instantiation of an entity declared using a placeholder type, but it also does not prevent that entity from being instantiated as needed to determine its type.

[Example 8:

```
template <typename T> auto f(T t) { return t; }
extern template auto f(int);           // does not instantiate f<int>
int (*p)(int) = f;                     // instantiates f<int> to determine its return type, but an explicit
                                        // instantiation definition is still required somewhere in the program
```

— end example]

9.2.9.7.2 Placeholder type deduction

[**dcl.type.auto.deduct**]

- 1 Placeholder type deduction is the process by which a type containing a placeholder type is replaced by a deduced type.
- 2 A type *T* containing a placeholder type, and a corresponding *initializer-clause E*, are determined as follows:
- (2.1) — For a non-discarded **return** statement that occurs in a function declared with a return type that contains a placeholder type, *T* is the declared return type.
- (2.1.1) — If the **return** statement has no operand, then *E* is **void()**.
- (2.1.2) — If the operand is a *braced-init-list* (9.5.5), the program is ill-formed.
- (2.1.3) — If the operand is an *expression X* that is not an *assignment-expression*, *E* is (*X*).
- [Note 1: A comma expression (7.6.20) is not an *assignment-expression*. — end note]
- (2.1.4) — Otherwise, *E* is the operand of the **return** statement.
- If *E* has type **void**, *T* shall be either *type-constraint_{opt} decltype(auto)* or *cv type-constraint_{opt} auto*.
- (2.2) — For a variable declared with a type that contains a placeholder type, *T* is the declared type of the variable.
- (2.2.1) — If the initializer of the variable is a *brace-or-equal-initializer* of the form **= initializer-clause**, *E* is the *initializer-clause*.
- (2.2.2) — If the initializer is a *braced-init-list*, it shall consist of a single brace-enclosed *assignment-expression* and *E* is the *assignment-expression*.
- (2.2.3) — If the initializer is a parenthesized *expression-list*, the *expression-list* shall be a single *assignment-expression* and *E* is the *assignment-expression*.
- (2.3) — For an explicit type conversion (7.6.1.4), *T* is the specified type, which shall be **auto**.
- (2.3.1) — If the initializer is a *braced-init-list*, it shall consist of a single brace-enclosed *assignment-expression* and *E* is the *assignment-expression*.
- (2.3.2) — If the initializer is a parenthesized *expression-list*, the *expression-list* shall be a single *assignment-expression* and *E* is the *assignment-expression*.
- (2.4) — For a constant template parameter declared with a type that contains a placeholder type, *T* is the declared type of the constant template parameter and *E* is the corresponding template argument.

T shall not be an array type.

- 3 If the *placeholder-type-specifier* is of the form *type-constraint_{opt} auto*, the deduced type *T'* replacing *T* is determined using the rules for template argument deduction. If the initialization is copy-list-initialization, a declaration of **std::initializer_list** shall precede (6.5.1) the *placeholder-type-specifier*. Obtain *P* from *T* by replacing the occurrence of *type-constraint_{opt} auto* either with a new invented type template parameter *U* or, if the initialization is copy-list-initialization, with **std::initializer_list<U>**. If *E* is a value synthesized for a constant template parameter of type **decltype(auto)** (13.7.7.3), the declaration is ill-formed. Otherwise, deduce a value for *U* using the rules of template argument deduction from a function call (13.10.3.2), where

P is a function template parameter type and the corresponding argument is *E*. If the deduction fails, the declaration is ill-formed. Otherwise, *T'* is obtained by substituting the deduced *U* into *P*.

[Example 1:

```
auto x1 = { 1, 2 };           // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 };        // error: cannot deduce element type
auto x3{ 1, 2 };             // error: not a single element
auto x4 = { 3 };             // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };                // decltype(x5) is int
```

— end example]

[Example 2:

```
const auto &i = expr;
```

The type of *i* is the deduced type of the parameter *u* in the call *f*(*expr*) of the following invented function template:

```
template <class U> void f(const U& u);
```

— end example]

- ⁴ If the *placeholder-type-specifier* is of the form *type-constraint*_{opt} *decltype*(*auto*), *T* shall be the placeholder alone. The type deduced for *T* is determined as described in 9.2.9.6, as though *E* had been the operand of the *decltype*.

[Example 3:

```
int i;
int&& f();
auto      x2a(i);           // decltype(x2a) is int
decltype(auto) x2d(i);      // decltype(x2d) is int
auto      x3a = i;          // decltype(x3a) is int
decltype(auto) x3d = i;     // decltype(x3d) is int
auto      x4a = (i);        // decltype(x4a) is int
decltype(auto) x4d = (i);    // decltype(x4d) is int&
auto      x5a = f();        // decltype(x5a) is int
decltype(auto) x5d = f();    // decltype(x5d) is int&&
auto      x6a = { 1, 2 };   // decltype(x6a) is std::initializer_list<int>
decltype(auto) x6d = { 1, 2 }; // error: { 1, 2 } is not an expression
auto      *x7a = &i;        // decltype(x7a) is int*
decltype(auto)*x7d = &i;    // error: declared type is not plain decltype(auto)
auto f1(int x) -> decltype((x)) { return (x); } // return type is int&
auto f2(int x) -> decltype(auto) { return (x); } // return type is int&&
```

— end example]

- ⁵ For a *placeholder-type-specifier* with a *type-constraint*, the immediately-declared constraint (13.2) of the *type-constraint* for the type deduced for the placeholder shall be satisfied.

9.2.9.8 Deduced class template specialization types

[*dcl.type.class.deduct*]

- ¹ If a placeholder for a deduced class type appears as a *decl-specifier* in the *decl-specifier-seq* of an initializing declaration (9.5) of a variable, the declared type of the variable shall be *cv T*, where *T* is the placeholder.

[Example 1:

```
template <class ...T> struct A {
    A(T...) {}
};
A x[29]{};           // error: no declarator operators allowed
const A& y{};        // error: no declarator operators allowed
```

— end example]

The placeholder is replaced by the return type of the function selected by overload resolution for class template deduction (12.2.2.9). If the *decl-specifier-seq* is followed by an *init-declarator-list* or *member-declarator-list* containing more than one *declarator*, the type that replaces the placeholder shall be the same in each deduction.

- ² A placeholder for a deduced class type can also be used in the *type-specifier-seq* in the *new-type-id* or *nofun-type-id* of a *new-expression* (7.6.2.8), as the *simple-type-specifier* in an explicit type conversion (functional

notation) (7.6.1.4), or as the *type-specifier* in the *parameter-declaration* of a *template-parameter* (13.2). A placeholder for a deduced class type shall not appear in any other context.

³ [Example 2:

```
template<class T> struct container {
    container(T t) {}
    template<class Iter> container(Iter beg, Iter end);
};
template<class Iter>
container(Iter b, Iter e) -> container<typename std::iterator_traits<Iter>::value_type>;
std::vector<double> v = { /* ... */ };

container c(7); // OK, deduces int for T
auto d = container(v.begin(), v.end()); // OK, deduces double for T
container e{5, 6}; // error: int is not an iterator
```

— end example]

9.2.9.9 Type splicing

[dcl.type.splice]

splice-type-specifier:

`typenameopt splice-specifier`

`typenameopt splice-specialization-specifier`

- ¹ A *splice-specifier* or *splice-specialization-specifier* immediately followed by `::` is never interpreted as part of a *splice-type-specifier*. A *splice-specifier* or *splice-specialization-specifier* not preceded by `typename` is only interpreted as a *splice-type-specifier* within a type-only context (13.8.1).

[Example 1:

```
template<std::meta::info R> void tfn() {
    typename [R::]::type m; // OK, typename applies to the qualified name
}

struct S { using type = int; };
void fn() {
    [::S::type:] *var; // error: [::S::type:] is an expression
    typename [::S::type:] *var; // OK, declares variable with type int*
}

using alias = [::S::type]; // OK, type-only context
```

— end example]

- ² For a *splice-type-specifier* of the form `typenameopt splice-specifier`, the *splice-specifier* shall designate a type, a class template, or an alias template. The *splice-type-specifier* designates the same entity as the *splice-specifier*.
- ³ For a *splice-type-specifier* of the form `typenameopt splice-specialization-specifier`, the *splice-specifier* of the *splice-specialization-specifier* shall designate a template T that is either a class template or an alias template. The *splice-type-specifier* designates the specialization of T corresponding to the template argument list of the *splice-specialization-specifier*.

9.3 Declarators

[dcl.decl]

9.3.1 General

[dcl.decl.general]

- ¹ A declarator declares a single variable, function, or type, within a declaration. The *init-declarator-list* appearing in a *simple-declaration* is a comma-separated sequence of declarators, each of which can have an initializer.

init-declarator-list:

init-declarator

init-declarator-list , *init-declarator*

init-declarator:

declarator initializer

declarator requires-clause_{opt} function-contract-specifier-seq_{opt}

- ² In all contexts, a *declarator* is interpreted as given below. Where an *abstract-declarator* can be used (or omitted) in place of a *declarator* (9.3.4.6, 14.1), it is as if a unique identifier were included in the appropriate place (9.3.2). The preceding specifiers indicate the type, storage duration, linkage, or other properties of

the entity or entities being declared. Each declarator specifies one entity and (optionally) names it and/or modifies the type of the specifiers with operators such as * (pointer to) and () (function returning).

[Note 1: An *init-declarator* can also specify an initializer (9.5). — end note]

- ³ Each *init-declarator* of a *simple-declaration* or *member-declarator* of a *member-declaration* is analyzed separately as if it were in a *simple-declaration* or *member-declaration* by itself.

[Note 2: A declaration with several declarators is usually equivalent to the corresponding sequence of declarations each with a single declarator. That is,

```
T D1, D2, ... Dn;
```

is usually equivalent to

```
T D1; T D2; ... T Dn;
```

where T is a *decl-specifier-seq* and each Di is an *init-declarator* or *member-declarator*. One exception is when a name introduced by one of the *declarators* hides a type name used by the *decl-specifiers*, so that when the same *decl-specifiers* are used in a subsequent declaration, they do not have the same meaning, as in

```
struct S { /* ... */ };
S S, T;           // declare two instances of struct S
```

which is not equivalent to

```
struct S { /* ... */ };
S S;
S T;              // error
```

Another exception is when T is **auto** (9.2.9.7), for example:

```
auto i = 1, j = 2.0; // error: deduced types for i and j do not match
```

as opposed to

```
auto i = 1;          // OK, i deduced to have type int
auto j = 2.0;        // OK, j deduced to have type double
```

— end note]

- ⁴ The *initializer* of an *init-declarator* shall not be present unless the declarator declares a variable and the host scope (6.4.1) of the declaration is the same as its target scope.
- ⁵ The optional *requires-clause* of an *init-declarator* or *member-declarator* shall not be present unless the declarator declares a templated function (13.1). When present after a declarator, the *requires-clause* is called the *trailing requires-clause*. The trailing *requires-clause* introduces the *constraint-expression* that results from interpreting its *constraint-logical-or-expression* as a *constraint-expression*.

[Example 1:

```
void f1(int a) requires true;           // error: non-templated function
template<typename T>
    auto f2(T a) -> bool requires true; // OK
template<typename T>
    auto f3(T a) requires true -> bool;  // error: requires-clause precedes trailing-return-type
void (*pf)() requires true;             // error: constraint on a variable
void g(int (*)() requires true);         // error: constraint on a parameter-declaration

auto* p = new void(*) (char) requires true; // error: not a function declaration
```

— end example]

- ⁶ The optional *function-contract-specifier-seq* (9.4.1) of an *init-declarator* shall not be present unless the *declarator* declares a function.
- ⁷ Declarators have the syntax

```
declarator:
    ptr-declarator
    noptr-declarator parameters-and-qualifiers trailing-return-type

ptr-declarator:
    noptr-declarator
    ptr-operator ptr-declarator
```

noPtr-declarator:
declarator-id *attribute-specifier-seq*_{opt}
noPtr-declarator *parameters-and-qualifiers*
noPtr-declarator [*constant-expression*_{opt}] *attribute-specifier-seq*_{opt}
(*ptr-declarator*)

parameters-and-qualifiers:
(*parameter-declaration-clause*) *cv-qualifier-seq*_{opt}
*ref-qualifier*_{opt} *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt}

trailing-return-type:
→ *noFun-type-id*

ptr-operator:
* *attribute-specifier-seq*_{opt} *cv-qualifier-seq*_{opt}
& *attribute-specifier-seq*_{opt}
&& *attribute-specifier-seq*_{opt}
nested-name-specifier * *attribute-specifier-seq*_{opt} *cv-qualifier-seq*_{opt}

cv-qualifier-seq:
cv-qualifier *cv-qualifier-seq*_{opt}

cv-qualifier:
const
volatile

ref-qualifier:
&
&&

declarator-id:
..._{opt} *id-expression*

9.3.2 Type names

[dcl.name]

- ¹ A type can be named with a *type-id*, *noFun-type-id*, or *new-type-id* (7.6.2.8), each of which is syntactically a declaration for a variable or (only for a *type-id*) function of that type that omits the name of the entity.

noFun-type-id:
type-specifier-seq *noFun-declarator*_{opt}

noFun-declarator:
ptr-noFun-declarator
noPtr-noFun-declarator *parameters-and-qualifiers* *trailing-return-type*

ptr-noFun-declarator:
noPtr-noFun-declarator
ptr-operator *ptr-noFun-declarator*_{opt}

noPtr-noFun-declarator:
noPtr-noFun-declarator *parameters-and-qualifiers*
*noPtr-noFun-declarator*_{opt} [*constant-expression*_{opt}] *attribute-specifier-seq*_{opt}
(*ptr-noFun-declarator*)

type-id:
type-specifier-seq *abstract-declarator*_{opt}

defining-type-id:
defining-type-specifier-seq *abstract-declarator*_{opt}

abstract-declarator:
ptr-abstract-declarator
*noPtr-abstract-declarator*_{opt} *parameters-and-qualifiers* *trailing-return-type*
abstract-pack-declarator

ptr-abstract-declarator:
noPtr-abstract-declarator
ptr-operator *ptr-abstract-declarator*_{opt}

noPtr-abstract-declarator:
*noPtr-abstract-declarator*_{opt} *parameters-and-qualifiers*
*noPtr-abstract-declarator*_{opt} [*constant-expression*_{opt}] *attribute-specifier-seq*_{opt}
(*ptr-abstract-declarator*)


```

abstract-pack-declarator:
    noptr-abstract-pack-declarator
    ptr-operator abstract-pack-declarator
noptr-abstract-pack-declarator:
    noptr-abstract-pack-declarator parameters-and-qualifiers
...

```

It is possible to identify uniquely the location in the *abstract-declarator* or *nofun-declarator* where the identifier would appear if the construction were a declarator in a declaration. The named type is then the same as the type of the hypothetical identifier.

[Example 1:

```

int                // int i
int *              // int *pi
int *[3]           // int *p[3]
int (*)[3]         // int (*pi)[3]
int *()           // int *f()
int *(double)     // int (*pf)(double)

```

name respectively the types “*int*”, “pointer to *int*”, “array of 3 pointers to *int*”, “pointer to array of 3 *int*”, “function of (no parameters) returning pointer to *int*”, and “pointer to a function of (*double*) returning *int*”. — end example]

- ² [Note 1: A type can also be named by a *typedef-name*, which is introduced by a *typedef* declaration or *alias-declaration* (9.2.4). — end note]

9.3.3 Ambiguity resolution

[dcl.ambig.res]

- ¹ The ambiguity arising from the similarity between a function-style cast and a declaration mentioned in 8.11 can also occur in the context of a declaration. In that context, the choice is between an object declaration with a function-style cast as the initializer and a declaration involving a function declarator with a redundant set of parentheses around a parameter name. Just as for the ambiguities mentioned in 8.11, the resolution is to consider any construct, such as the potential parameter declaration, that could possibly be a declaration to be a declaration. However, a construct that can syntactically be a *declaration* whose outermost *declarator* would match the grammar of a *declarator* with a *trailing-return-type* is a declaration only if it starts with **auto**.

[Note 1: A declaration can be explicitly disambiguated by adding parentheses around the argument. The ambiguity can be avoided by use of copy-initialization or list-initialization syntax, or by use of a non-function-style cast. — end note]

[Example 1:

```

struct S {
    S(int);
};
typedef struct BB { int C[2]; } *B, C;

void foo(double a) {
    S v(int(a));           // function declaration
    S w(int());            // function declaration
    S x((int(a)));         // object declaration
    S y((int)a);           // object declaration
    S z = int(a);          // object declaration
    S a(B()->C);           // object declaration
    S b(auto()->C);        // function declaration
}

```

— end example]

- ² An ambiguity can arise from the similarity between a function-style cast and a *type-id*. The resolution is that any construct that could possibly be a *type-id* in its syntactic context shall be considered a *type-id*.

[Note 2: No such ambiguity can arise between an expression and a *nofun-type-id*. — end note]

However, a construct that can syntactically be a *type-id* whose outermost *abstract-declarator* would match the grammar of an *abstract-declarator* with a *trailing-return-type* is considered a *type-id* only if it starts with **auto**.

[Example 2:

```

template <class T> struct X {};
template <int N> struct Y {};

```

```

X<int()> a;           // type-id
X<int(1)> b;           // expression (ill-formed)
Y<int()> c;           // type-id (ill-formed)
Y<int(1)> d;           // expression

void foo(signed char a) {
    sizeof(int());    // expression
    sizeof(int(a));    // expression
    sizeof(int(unsigned(a))); // expression
    (int())+1;         // expression
    (int(a))+1;        // expression
    (int(unsigned(a)))+1; // expression
}

typedef struct BB { int C[2]; } *B, C;
void g() {
    sizeof(B()->C[1]); // OK, sizeof(expression)
    sizeof(auto()->C[1]); // error: ill-formed expression
}

```

— end example]

- ³ Another ambiguity arises in a *parameter-declaration-clause* when a *type-name* is nested in parentheses. In this case, the choice is between the declaration of a parameter of type pointer to function and the declaration of a parameter with redundant parentheses around the *declarator-id*. The resolution is to consider the *type-name* as a *simple-type-specifier* rather than a *declarator-id*.

[Example 3:

```

class C { };
void f(int(C)) { }           // void f(int(*fp)(C c)) { }
                             // not: void f(int C) { }

int g(C);

void foo() {
    f(1);                    // error: cannot convert 1 to function pointer
    f(g);                    // OK
}

```

For another example,

```

class C { };
void h(int *(C[10]));        // void h(int *(*_fp)(C _parm[10]));
                             // not: void h(int *C[10]);

```

— end example]

9.3.4 Meaning of declarators

[dcl.meaning]

9.3.4.1 General

[dcl.meaning.general]

- ¹ A declarator contains exactly one *declarator-id*; it names the entity that is declared. If the *unqualified-id* occurring in a *declarator-id* is a *template-id*, the declarator shall appear in the *declaration* of a *template-declaration* (13.7), *explicit-specialization* (13.9.4), or *explicit-instantiation* (13.9.3).

[Note 1: An *unqualified-id* that is not an *identifier* is used to declare certain functions (11.4.8.3, 11.4.7, 12.4, 12.6).
— end note]

The optional *attribute-specifier-seq* following a *declarator-id* appertains to the entity that is declared.

- ² If the declaration is a friend declaration:

- (2.1) — The *declarator* does not bind a name.
- (2.2) — If the *id-expression* *E* in the *declarator-id* of the *declarator* is a *qualified-id* or a *template-id*:
 - (2.2.1) — If the friend declaration is not a template declaration, then in the lookup for the terminal name of *E*:
 - (2.2.1.1) — if the *unqualified-id* in *E* is a *template-id*, all function declarations are discarded;

- (2.2.1.2) — otherwise, if the *declarator* corresponds (6.4.1) to any declaration found of a non-template function, all function template declarations are discarded;
- (2.2.1.3) — each remaining function template is replaced with the specialization chosen by deduction from the friend declaration (13.10.3.7) or discarded if deduction fails.
- (2.2.2) — The *declarator* shall correspond to one or more declarations found by the lookup; they shall all have the same target scope, and the target scope of the *declarator* is that scope.
- (2.3) — Otherwise, the terminal name of *E* is not looked up. The declaration's target scope is the innermost enclosing namespace scope; if the declaration is contained by a block scope, the declaration shall correspond to a reachable (10.7) declaration that inhabits the innermost block scope.

3 Otherwise:

- (3.1) — If the *id-expression* in the *declarator-id* of the *declarator* is a *qualified-id* *Q*, let *S* be its lookup context (6.5.5); the declaration shall inhabit a namespace scope.
- (3.2) — Otherwise, let *S* be the entity associated with the scope inhabited by the *declarator*.
- (3.3) — If the *declarator* declares an explicit instantiation or a partial or explicit specialization, the *declarator* does not bind a name. If it declares a class member, the terminal name of the *declarator-id* is not looked up; otherwise, only those lookup results that are nominable in *S* are considered when identifying any function template specialization being declared (13.10.3.7).

[Example 1:

```
namespace N {
    inline namespace O {
        template<class T> void f(T);           // #1
        template<class T> void g(T) {}
    }
    namespace P {
        template<class T> void f(T*);          // #2, more specialized than #1
        template<class> int g;
    }
    using P::f,P::g;
}
template<> void N::f(int*) {}                 // OK, #2 is not nominable
template void N::g(int);                     // error: lookup is ambiguous
```

— end example]

- (3.4) — Otherwise, the terminal name of the *declarator-id* is not looked up. If it is a qualified name, the *declarator* shall correspond to one or more declarations nominable in *S*; all the declarations shall have the same target scope and the target scope of the *declarator* is that scope.

[Example 2:

```
namespace Q {
    namespace V {
        void f();
    }
    void V::f() { /* ... */ }                // OK
    void V::g() { /* ... */ }                // error: g() is not yet a member of V
    namespace V {
        void g();
    }
}

namespace R {
    void Q::V::g() { /* ... */ }             // error: R doesn't enclose Q
}
```

— end example]

- (3.5) — If the declaration inhabits a block scope *S* and declares a function (9.3.4.6) or uses the **extern** specifier, the declaration shall not be attached to a named module (10.1); its target scope is the innermost enclosing namespace scope, but the name is bound in *S*.

[Example 3:

```

namespace X {
    void p() {
        q();                // error: q not yet declared
        extern void q();     // q is a member of namespace X
        extern void r();     // r is a member of namespace X
    }

    void middle() {
        q();                // error: q not found
    }

    void q() { /* ... */ }   // definition of X::q
}

void q() { /* ... */ }     // some other, unrelated q
void X::r() { /* ... */ }  // error: r cannot be declared by qualified-id
— end example]

```

⁴ A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, `constexpr`, `constinit`, or `typedef` specifier or an *explicit-specifier* applies directly to each *declarator-id* in a *simple-declaration* or *member-declaration*; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

⁵ Thus, (for each *declarator*) a declaration has the form

T D

where T is of the form *attribute-specifier-seq_{opt}* *decl-specifier-seq* and D is a *declarator*. Following is a recursive procedure for determining the type specified for the contained *declarator-id* by such a declaration.

⁶ First, the *decl-specifier-seq* determines a type. In a declaration

T D

the *decl-specifier-seq* T determines the type T.

[Example 4: In the declaration

`int unsigned i;`

the type specifiers `int unsigned` determine the type “`unsigned int`” (9.2.9.3). — end example]

⁷ In a declaration *attribute-specifier-seq_{opt}* T D where D is an unadorned *declarator-id*, the type of the declared entity is “T”.

⁸ In a declaration T D where D has the form

(D1)

the type of the contained *declarator-id* is the same as that of the contained *declarator-id* in the declaration

T D1

Parentheses do not alter the type of the embedded *declarator-id*, but they can alter the binding of complex declarators.

9.3.4.2 Pointers

[dcl.ptr]

¹ In a declaration T D where D has the form

* *attribute-specifier-seq_{opt}* *cv-qualifier-seq_{opt}* D1

and the type of the contained *declarator-id* in the declaration T D1 is “*derived-declarator-type-list* T”, the type of the *declarator-id* in D is “*derived-declarator-type-list* *cv-qualifier-seq* pointer to T”. The *cv-qualifiers* apply to the pointer and not to the object pointed to. Similarly, the optional *attribute-specifier-seq* (9.13.1) appertains to the pointer and not to the object pointed to.

² [Example 1: The declarations

`const int ci = 10, *pc = &ci, *const cpc = pc, **ppc;`
`int i, *p, *const cp = &i;`

declare `ci`, a constant integer; `pc`, a pointer to a constant integer; `cpc`, a constant pointer to a constant integer; `ppc`, a pointer to a pointer to a constant integer; `i`, an integer; `p`, a pointer to integer; and `cp`, a constant pointer to integer.

The value of `ci`, `cpc`, and `cp` cannot be changed after initialization. The value of `pc` can be changed, and so can the object pointed to by `cp`. Examples of some correct operations are

```
i = ci;
*cp = ci;
pc++;
pc = cpc;
pc = p;
ppc = &pc;
```

Examples of ill-formed operations are

```
ci = 1;           // error
ci++;            // error
*pc = 2;         // error
cp = &ci;        // error
cpc++;          // error
p = pc;         // error
ppc = &p;        // error
```

Each is unacceptable because it would either change the value of an object declared `const` or allow it to be changed through a cv-unqualified pointer later, for example:

```
*ppc = &ci;      // OK, but would make p point to ci because of previous error
*p = 5;          // clobber ci
```

See also 7.6.19 and 9.5. — end example]

- ³ [Note 1: Forming a pointer to reference type is ill-formed; see 9.3.4.3. Forming a function pointer type is ill-formed if the function type has *cv-qualifiers* or a *ref-qualifier*; see 9.3.4.6. Since the address of a bit-field (11.4.10) cannot be taken, a pointer can never point to a bit-field. — end note]

9.3.4.3 References

[dcl.ref]

- ¹ In a declaration `T D` where `D` has either of the forms

```
& attribute-specifier-seqopt D1
&& attribute-specifier-seqopt D1
```

and the type of the contained *declarator-id* in the declaration `T D1` is “*derived-declarator-type-list T*”, the type of the *declarator-id* in `D` is “*derived-declarator-type-list reference to T*”. The optional *attribute-specifier-seq* appertains to the reference type. Cv-qualified references are ill-formed except when the cv-qualifiers are introduced through the use of a *typedef-name* (9.2.4, 13.2) or *decltype-specifier* (9.2.9.6), in which case the cv-qualifiers are ignored.

[Example 1:

```
typedef int& A;
const A aref = 3; // error: lvalue reference to non-const initialized with rvalue
```

The type of `aref` is “lvalue reference to `int`”, not “lvalue reference to `const int`”. — end example]

[Note 1: A reference can be thought of as a name of an object. — end note]

Forming the type “reference to cv void” is ill-formed.

- ² A reference type that is declared using `&` is called an *lvalue reference*, and a reference type that is declared using `&&` is called an *rvalue reference*. Lvalue references and rvalue references are distinct types. Except where explicitly noted, they are semantically equivalent and commonly referred to as references.
- ³ [Example 2:

```
void f(double& a) { a += 3.14; }
// ...
double d = 0;
f(d);
```

declares `a` to be a reference parameter of `f` so the call `f(d)` will add 3.14 to `d`.

```
int v[20];
// ...
int& g(int i) { return v[i]; }
// ...
g(3) = 7;
```

declares the function `g()` to return a reference to an integer so `g(3)=7` will assign 7 to the fourth element of the array `v`. For another example,

```
struct link {
    link* next;
};

link* first;

void h(link*& p) { // p is a reference to pointer
    p->next = first;
    first = p;
    p = 0;
}

void k() {
    link* q = new link;
    h(q);
}
```

declares `p` to be a reference to a pointer to `link` so `h(q)` will leave `q` with the value zero. See also 9.5.4. — *end example*

- 4 It is unspecified whether or not a reference requires storage (6.8.6).
- 5 There shall be no references to references, no arrays of references, and no pointers to references. The declaration of a reference shall contain an *initializer* (9.5.4) except when the declaration contains an explicit **extern** specifier (9.2.2), is a class member (11.4) declaration within a class definition, or is the declaration of a parameter or a return type (9.3.4.6); see 6.2.
- 6 Attempting to bind a reference to a function where the converted initializer is a glvalue whose type is not call-compatible (7.6.1.3) with the type of the function's definition results in undefined behavior (F.5.3). Attempting to bind a reference to an object where the converted initializer is a glvalue through which the object is not type-accessible (7.2.1) results in undefined behavior (F.5.4).

[*Note 2:* The object designated by such a glvalue can be outside its lifetime (6.8.4). Because a null pointer value or a pointer past the end of an object does not point to an object, a reference in a well-defined program cannot refer to such things; see 7.6.2.2. As described in 11.4.10, a reference cannot be bound directly to a bit-field. — *end note*]

The behavior of an evaluation of a reference (7.5.5, 7.6.1.5) that does not happen after (6.10.2.2) the initialization of the reference is undefined (F.5.5).

[*Example 3:*

```
int &f(int&);
int &g();
extern int &ir3;
int *ip = 0;
int &ir1 = *ip;           // undefined behavior: null pointer
int &ir2 = f(ir3);        // undefined behavior: ir3 not yet initialized
int &ir3 = g();
int &ir4 = f(ir4);        // undefined behavior: ir4 used in its own initializer

char x alignas(int);
int &ir5 = *reinterpret_cast<int *>(&x); // undefined behavior: initializer refers to char object
```

— *end example*]

- 7 If a *typedef-name* (9.2.4, 13.2) or a *decltype-specifier* (9.2.9.6) denotes a type `TR` that is a reference to a type `T`, an attempt to create the type “lvalue reference to *cv* `TR`” creates the type “lvalue reference to `T`”, while an attempt to create the type “rvalue reference to *cv* `TR`” creates the type `TR`.

[*Note 3:* This rule is known as reference collapsing. — *end note*]

[*Example 4:*

```
int i;
typedef int& LRI;
typedef int&& RRI;

LRI& r1 = i;           // r1 has the type int&
```

```

const LRI& r2 = i;           // r2 has the type int&
const LRI&& r3 = i;          // r3 has the type int&

RRI& r4 = i;                 // r4 has the type int&
RRI&& r5 = 5;                 // r5 has the type int&&

decltype(r2)& r6 = i;         // r6 has the type int&
decltype(r2)&& r7 = i;        // r7 has the type int&

```

— end example]

- ⁸ [Note 4: Forming a reference to function type is ill-formed if the function type has *cv-qualifiers* or a *ref-qualifier*; see 9.3.4.6. — end note]

9.3.4.4 Pointers to members

[dcl.mptr]

- ¹ The component names of a *ptr-operator* are those of its *nested-name-specifier*, if any.
- ² In a declaration `T D` where `D` has the form

nested-name-specifier * *attribute-specifier-seq*_{opt} *cv-qualifier-seq*_{opt} `D1`

and the *nested-name-specifier* designates a class, and the type of the contained *declarator-id* in the declaration `T D1` is “*derived-declarator-type-list T*”, the type of the *declarator-id* in `D` is “*derived-declarator-type-list cv-qualifier-seq* pointer to member of class *nested-name-specifier* of type *T*”. The optional *attribute-specifier-seq* (9.13.1) appertains to the pointer-to-member. The *nested-name-specifier* shall not designate an anonymous union.

- ³ [Example 1:

```

struct X {
    void f(int);
    int a;
};
struct Y;

int X::* pmi = &X::a;
void (X::* pmf)(int) = &X::f;
double X::* pmd;
char Y::* pmc;

```

declares `pmi`, `pmf`, `pmd` and `pmc` to be a pointer to a member of `X` of type `int`, a pointer to a member of `X` of type `void(int)`, a pointer to a member of `X` of type `double` and a pointer to a member of `Y` of type `char` respectively. The declaration of `pmd` is well-formed even though `X` has no members of type `double`. Similarly, the declaration of `pmc` is well-formed even though `Y` is an incomplete type. `pmi` and `pmf` can be used like this:

```

X obj;
// ...
obj.*pmi = 7;           // assign 7 to an integer member of obj
(obj.*pmf)(7);          // call a function member of obj with the argument 7

```

— end example]

- ⁴ A pointer to member shall not point to a static member of a class (11.4.9), a member with reference type, or “*cv void*”.
- ⁵ [Note 1: See also 7.6.2 and 7.6.4. The type “pointer to member” is distinct from the type “pointer”, that is, a pointer to member is declared only by the pointer-to-member declarator syntax, and never by the pointer declarator syntax. There is no “reference-to-member” type in C++. — end note]

9.3.4.5 Arrays

[dcl.array]

- ¹ In a declaration `T D` where `D` has the form

`D1` [*constant-expression*_{opt}] *attribute-specifier-seq*_{opt}

and the type of the contained *declarator-id* in the declaration `T D1` is “*derived-declarator-type-list T*”, the type of the *declarator-id* in `D` is “*derived-declarator-type-list array of N T*”. The *constant-expression* shall be a converted constant expression of type `std::size_t` (7.7.3). Its value `N` specifies the *array bound*, i.e., the number of elements in the array; `N` shall be greater than zero.

- ² In a declaration `T D` where `D` has the form

`D1` [] *attribute-specifier-seq*_{opt}

and the type of the contained *declarator-id* in the declaration **T D1** is “*derived-declarator-type-list T*”, the type of the *declarator-id* in **D** is “*derived-declarator-type-list array of unknown bound of T*”, except as specified below.

- ³ A type of the form “array of **N U**” or “array of unknown bound of **U**” is an *array type*. The optional *attribute-specifier-seq* appertains to the array type.
- ⁴ **U** is called the array *element type*; this type shall not be a reference type, a function type, an array of unknown bound, or *cv void*.

[*Note 1*: An array can be constructed from one of the fundamental types (except **void**), from a pointer, from a pointer to member, from a class, from an enumeration type, or from an array of known bound. — *end note*]

[*Example 1*:

```
float fa[17], *afp[17];
```

declares an array of **float** numbers and an array of pointers to **float** numbers. — *end example*]

- ⁵ Any type of the form “*cv-qualifier-seq* array of **N U**” is adjusted to “array of **N cv-qualifier-seq U**”, and similarly for “array of unknown bound of **U**”.

[*Example 2*:

```
typedef int A[5], AA[2][3];
typedef const A CA;           // type is “array of 5 const int”
typedef const AA CAA;         // type is “array of 2 array of 3 const int”
```

— *end example*]

[*Note 2*: An “array of **N cv-qualifier-seq U**” has *cv-qualified type*; see 6.9.6. — *end note*]

- ⁶ An object of type “array of **N U**” consists of a contiguously allocated non-empty set of **N** subobjects of type **U**, known as the *elements* of the array, and numbered 0 to **N**-1. The element numbered 0 is termed the *first element* of the array.
- ⁷ In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (9.3.4.6). An array bound may also be omitted when an object (but not a non-static data member) of array type is initialized and the declarator is followed by an initializer (9.5, 11.4, 7.6.1.4, 7.6.2.8). In these cases, the array bound is calculated from the number of initial elements (say, **N**) supplied (9.5.2), and the type of the array is “array of **N U**”.
- ⁸ Furthermore, if there is a reachable declaration of the entity that specifies a bound and has the same host scope (6.4.1), an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

[*Example 3*:

```
extern int x[10];
struct S {
    static int y[10];
};

int x[];           // OK, bound is 10
int S::y[];        // OK, bound is 10

void f() {
    extern int x[];
    int i = sizeof(x); // error: incomplete object type
}

namespace A { extern int z[3]; }
int A::z[] = {};    // OK, defines an array of 3 elements
```

— *end example*]

- ⁹ [*Note 3*: When several “array of” specifications are adjacent, a multidimensional array type is created; only the first of the constant expressions that specify the bounds of the arrays can be omitted.

[*Example 4*:

```
int x3d[3][5][7];
```

declares an array of three elements, each of which is an array of five elements, each of which is an array of seven integers. The overall array can be viewed as a three-dimensional array of integers, with rank $3 \times 5 \times 7$. Any of the

expressions `x3d`, `x3d[i]`, `x3d[i][j]`, `x3d[i][j][k]` can reasonably appear in an expression. The expression `x3d[i]` is equivalent to `*(x3d + i)`; in that expression, `x3d` is subject to the array-to-pointer conversion (7.3.3) and is first converted to a pointer to a 2-dimensional array with rank 5×7 that points to the first element of `x3d`. Then `i` is added, which on typical implementations involves multiplying `i` by the length of the object to which the pointer points, which is `sizeof(int) × 5 × 7`. The result of the addition and indirection is an lvalue denoting the i^{th} array element of `x3d` (an array of five arrays of seven integers). If there is another subscript, the same argument applies again, so `x3d[i][j]` is an lvalue denoting the j^{th} array element of the i^{th} array element of `x3d` (an array of seven integers), and `x3d[i][j][k]` is an lvalue denoting the k^{th} array element of the j^{th} array element of the i^{th} array element of `x3d` (an integer). — *end example*

The first subscript in the declaration helps determine the amount of storage consumed by an array but plays no other part in subscript calculations. — *end note*

¹⁰ [Note 4: Conversions affecting expressions of array type are described in 7.3.3. — *end note*]

¹¹ [Note 5: The subscript operator can be overloaded for a class (12.4.5). For the operator’s built-in meaning, see 7.6.1.2. — *end note*]

9.3.4.6 Functions

[**dcl.fct**]

¹ In a declaration `T D` where `T` may be empty and `D` has the form

`D1 (parameter-declaration-clause) cv-qualifier-seqopt
ref-qualifieropt noexcept-specifieropt attribute-specifier-seqopt trailing-return-typeopt`

a *derived-declarator-type-list* is determined as follows:

- (1.1) — If the *unqualified-id* of the *declarator-id* is a *conversion-function-id*, the *derived-declarator-type-list* is empty.
- (1.2) — Otherwise, the *derived-declarator-type-list* is as appears in the type “*derived-declarator-type-list* `T`” of the contained *declarator-id* in the declaration `T D1`.

The declared return type `U` of the function type is determined as follows:

- (1.3) — If the *trailing-return-type* is present, `T` shall be the single *type-specifier* `auto`, and `U` is the type specified by the *trailing-return-type*.
- (1.4) — Otherwise, if the declaration declares a conversion function, see 11.4.8.3.
- (1.5) — Otherwise, `U` is `T`.

The type of the *declarator-id* in `D` is “*derived-declarator-type-list* `noexceptopt` function of parameter-type-list *cv-qualifier-seq_{opt}* *ref-qualifier_{opt}* returning `U`”, where

- (1.6) — the parameter-type-list is derived from the *parameter-declaration-clause* as described below and
- (1.7) — the optional `noexcept` is present if and only if the exception specification (14.5) is non-throwing.

Such a type is a *function type*.⁶⁸ The optional *attribute-specifier-seq* appertains to the function type.

parameter-declaration-clause:

...
parameter-declaration-list_{opt}
parameter-declaration-list , ...
parameter-declaration-list ...

parameter-declaration-list:

parameter-declaration
parameter-declaration-list , *parameter-declaration*

parameter-declaration:

attribute-specifier-seq_{opt} *this_{opt}* *decl-specifier-seq* *declarator*
attribute-specifier-seq_{opt} *decl-specifier-seq* *declarator* = *initializer-clause*
attribute-specifier-seq_{opt} *this_{opt}* *decl-specifier-seq* *abstract-declarator_{opt}*
attribute-specifier-seq_{opt} *decl-specifier-seq* *abstract-declarator_{opt}* = *initializer-clause*

The optional *attribute-specifier-seq* in a *parameter-declaration* appertains to the parameter.

² The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called.

[Note 1: The *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 7.6.1.3. — *end note*]

⁶⁸) As indicated by syntax, cv-qualifiers are a significant component in function return types.

If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed non-object parameter whose type is the same as (13.7.7.2) `void` is interpreted as an empty parameter list. Except for this special case, a parameter shall not have type *cv void*. A parameter with volatile-qualified type is deprecated; see D.4.

[Example 1:

```
template<class T>
struct S {
    void f(std::void_t<T*>);    // #1
};
S<int> x;                      // error: #1 has a parameter of type void

template<class T>
struct S2 {
    void f(std::void_t<int*>);  // #2
};
S2<int> y;                     // OK, #2 is a zero-parameter function
```

— end example]

- ³ If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (13.7.4), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where “...” is not part of an *abstract-declarator*, “...” is synonymous with “, ...”. A *parameter-declaration-clause* of the form *parameter-declaration-list* ... is deprecated (D.5).

[Example 2: The declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a `const char*`. — end example]

[Note 2: The standard header `<stdarg.h>` (17.14.2) contains a mechanism for accessing arguments passed using the ellipsis (see 7.6.1.3 and 17.14). — end note]

- ⁴ The type of a function is determined using the following rules. The type of each parameter (including function parameter packs) is determined from its own *parameter-declaration* (9.3). After determining the type of each parameter, any parameter of type “array of T” or of function type T is adjusted to be “pointer to T”. After producing the list of parameter types, any top-level *cv-qualifiers* modifying a parameter type are deleted when forming the function type. The resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function’s *parameter-type-list*.

[Note 3: This transformation does not affect the types of the parameters. For example, `int(*) (const int p, decltype(p)*)` and `int(*) (int, const int*)` are identical types. — end note]

[Example 3:

```
void f(char*);                // #1
void f(char[]) {}            // defines #1
void f(const char*) {}       // OK, another overload
void f(char *const) {}       // error: redefines #1

void g(char(*)[2]);          // #2
void g(char[3][2]) {}        // defines #2
void g(char[3][3]) {}        // OK, another overload

void h(int x(const int));    // #3
void h(int (*)(int)) {}      // defines #3
```

— end example]

- ⁵ A function with a parameter-type-list that has an ellipsis is termed a *vararg function*.
- ⁶ An *explicit-object-parameter-declaration* is a *parameter-declaration* with a `this` specifier. An explicit-object-parameter-declaration shall appear only as the first *parameter-declaration* of a *parameter-declaration-list* of one of:

- (6.1) — a declaration of a member function or member function template (11.4), or
- (6.2) — an explicit instantiation (13.9.3) or explicit specialization (13.9.4) of a templated member function, or
- (6.3) — a *lambda-declarator* (7.5.6).

A *member-declarator* with an explicit-object-parameter-declaration shall not include a *ref-qualifier* or a *cv-qualifier-seq* and shall not be declared **static** or **virtual**.

[Example 4:

```
struct C {
    void f(this C& self);
    template <typename Self> void g(this Self&& self, int);

    void h(this C) const;           // error: const not allowed here
};

void test(C c) {
    c.f();                         // OK, calls C::f
    c.g(42);                       // OK, calls C::g<C&>
    std::move(c).g(42);           // OK, calls C::g<C>
}
```

— end example]

- 7 A function parameter declared with an explicit-object-parameter-declaration is an *explicit object parameter*. An explicit object parameter shall not be a function parameter pack (13.7.4). An *explicit object member function* is a non-static member function with an explicit object parameter. An *implicit object member function* is a non-static member function without an explicit object parameter.
- 8 The *object parameter* of a non-static member function is either the explicit object parameter or the implicit object parameter (12.2.2).
- 9 A *non-object parameter* is a function parameter that is not the explicit object parameter. The *non-object-parameter-type-list* of a member function is the parameter-type-list of that function with the explicit object parameter, if any, omitted.

[Note 4: The non-object-parameter-type-list consists of the adjusted types of all the non-object parameters. — end note]

- 10 A function type with a *cv-qualifier-seq* or a *ref-qualifier* (including a type denoted by *typedef-name* (9.2.4, 13.2)) shall appear only as:

- (10.1) — the function type for a non-static member function,
- (10.2) — the function type to which a pointer to member refers,
- (10.3) — the top-level function type of a function typedef declaration or *alias-declaration*,
- (10.4) — the *type-id* in the default argument of a *type-parameter* (13.2),
- (10.5) — the *type-id* of a *template-argument* for a *type-parameter* (13.4.2), or
- (10.6) — the operand of a *reflect-expression* (7.6.2.10).

[Example 5:

```
typedef int FIC(int) const;
FIC f;                               // error: does not declare a member function
struct S {
    FIC f;                           // OK
};
FIC S::*pm = &S::f;                 // OK
constexpr std::meta::info yeti = ~void(int) const &; // OK
```

— end example]

- 11 The effect of a *cv-qualifier-seq* in a function declarator is not the same as adding cv-qualification on top of the function type. In the latter case, the cv-qualifiers are ignored.

[Note 5: A function type that has a *cv-qualifier-seq* is not a cv-qualified type; there are no cv-qualified function types. — end note]

[Example 6:

```
typedef void F();
struct S {
    const F f;           // OK, equivalent to: void f();
};
```

— end example]

- 12 The return type, the parameter-type-list, the *ref-qualifier*, the *cv-qualifier-seq*, and the exception specification, but not the default arguments (9.3.4.7) or the trailing *requires-clause* (9.3), are part of the function type.

[Note 6: Function types are checked during the assignments and initializations of pointers to functions, references to functions, and pointers to member functions. — end note]

- 13 [Example 7: The declaration

```
int fseek(FILE*, long, int);
```

declares a function taking three arguments of the specified types, and returning `int` (9.2.9). — end example]

- 14 [Note 7: A single name can be used for several different functions in a single scope; this is function overloading (Clause 12). — end note]

- 15 The return type shall be a non-array object type, a reference type, or *cv void*.

[Note 8: An array of placeholder type is considered an array type. — end note]

- 16 A volatile-qualified return type is deprecated; see D.4.

- 17 Types shall not be defined in return or parameter types.

- 18 A typedef of function type may be used to declare a function but shall not be used to define a function (9.6).

[Example 8:

```
typedef void F();
F fv;           // OK, equivalent to void fv();
F fv { }        // error
void fv() { }    // OK, definition of fv
```

— end example]

- 19 An identifier can optionally be provided as a parameter name; if present in a function definition (9.6), it names a parameter.

[Note 9: In particular, parameter names are also optional in function definitions and names used for a parameter in different declarations and the definition of a function need not be the same. — end note]

- 20 [Example 9: The declaration

```
int i,
    *pi,
    f(),
    *fpi(int),
    (*pif)(const char*, const char*),
    (*fpif(int))(int);
```

declares an integer `i`, a pointer `pi` to an integer, a function `f` taking no arguments and returning an integer, a function `fpi` taking an integer argument and returning a pointer to an integer, a pointer `pif` to a function which takes two pointers to constant characters and returns an integer, a function `fpif` taking an integer argument and returning a pointer to a function that takes an integer argument and returns an integer. It is especially useful to compare `fpi` and `pif`. The binding of `*fpi(int)` is `*(fpi(int))`, so the declaration suggests, and the same construction in an expression requires, the calling of a function `fpi`, and then using indirection through the (pointer) result to yield an integer. In the declarator `(*pif)(const char*, const char*)`, the extra parentheses are necessary to indicate that indirection through a pointer to a function yields a function, which is then called. — end example]

[Note 10: Typedefs and *trailing-return-types* are sometimes convenient when the return type of a function is complex. For example, the function `fpif` above can be declared

```
typedef int IFUNC(int);
IFUNC* fpif(int);
```

or

```
auto fpif(int)->int(*) (int);
```

A *trailing-return-type* is most useful for a type that would be more complicated to specify before the *declarator-id*:

```
template <class T, class U> auto add(T t, U u) -> decltype(t + u);
```

rather than

```
template <class T, class U> decltype((*T*)0) + ((*U*)0) add(T t, U u);
```

— end note]

- 21 A *non-template function* is a function that is not a function template specialization.

[Note 11: A function template is not a function. — end note]

- 22 An *abbreviated function template* is a function declaration that has one or more generic parameter type placeholders (9.2.9.7). An abbreviated function template is equivalent to a function template (13.7.7) whose *template-parameter-list* includes one invented *type-parameter* for each generic parameter type placeholder of the function declaration, in order of appearance. For a *placeholder-type-specifier* of the form *auto*, the invented parameter is an unconstrained *type-parameter*. For a *placeholder-type-specifier* of the form *type-constraint auto*, the invented parameter is a *type-parameter* with that *type-constraint*. The invented *type-parameter* declares a template parameter pack if the corresponding *parameter-declaration* declares a function parameter pack. If the placeholder contains `decltype(auto)`, the program is ill-formed. The adjusted function parameters of an abbreviated function template are derived from the *parameter-declaration-clause* by replacing each occurrence of a placeholder with the name of the corresponding invented *type-parameter*.

[Example 10:

```
template<typename T>      concept C1 = /* ... */;
template<typename T>      concept C2 = /* ... */;
template<typename... Ts> concept C3 = /* ... */;

void g1(const C1 auto*, C2 auto&);
void g2(C1 auto&...);
void g3(C3 auto...);
void g4(C3 auto);
```

The declarations above are functionally equivalent (but not equivalent) to their respective declarations below:

```
template<C1 T, C2 U> void g1(const T*, U&);
template<C1... Ts>  void g2(Ts&...);
template<C3... Ts>  void g3(Ts...);
template<C3 T>      void g4(T);
```

Abbreviated function templates can be specialized like all function templates.

```
template<> void g1<int>(const int*, const double&); // OK, specialization of g1<int, const double>
```

— end example]

- 23 An abbreviated function template can have a *template-head*. The invented *type-parameters* are appended to the *template-parameter-list* after the explicitly declared *template-parameters*.

[Example 11:

```
template<typename> concept C = /* ... */;

template <typename T, C U>
    void g(T x, U y, C auto z);
```

This is functionally equivalent to each of the following two declarations.

```
template<typename T, C U, C W>
    void g(T x, U y, W z);

template<typename T, typename U, typename W>
    requires C<U> && C<W>
    void g(T x, U y, W z);
```

— end example]

- 24 A function declaration at block scope shall not declare an abbreviated function template.
- 25 A *declarator-id* or *abstract-declarator* containing an ellipsis shall only be used in a *parameter-declaration*. When it is part of a *parameter-declaration-clause*, the *parameter-declaration* declares a function parameter pack (13.7.4). Otherwise, the *parameter-declaration* is part of a *template-parameter-list* and declares a template parameter pack; see 13.2. A function parameter pack is a pack expansion (13.7.4).

[Example 12:

```
template<typename... T> void f(T (* ...t)(int, int));
```

```

int add(int, int);
float subtract(int, int);

void g() {
    f(add, subtract);
}

```

— end example]

- ²⁶ There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.⁶⁹

9.3.4.7 Default arguments

[dcl.fct.default]

- ¹ If an *initializer-clause* is specified in a *parameter-declaration* that is not a *template-parameter* (13.2), this *initializer-clause* is used as a default argument.

[Note 1: Default arguments will be used in calls where trailing arguments are missing (7.6.1.3). — end note]

- ² [Example 1: The declaration

```
void point(int = 3, int = 4);
```

declares a function that can be called with zero, one, or two arguments of type `int`. It can be called in any of these ways:

```
point(1,2); point(1); point();
```

The last two calls are equivalent to `point(1,4)` and `point(3,4)`, respectively. — end example]

- ³ A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or *lambda-declarator*. A default argument shall not be specified for a function parameter pack. A default argument shall not occur within a *declarator* or *abstract-declarator* of a *parameter-declaration*.⁷⁰
- ⁴ For non-template functions, default arguments can be added in later declarations of a function that have the same host scope. Declarations that have different host scopes have completely distinct sets of default arguments. That is, declarations in inner scopes do not acquire default arguments from declarations in outer scopes, and vice versa. In a given function declaration, each parameter subsequent to a parameter with a default argument shall have a default argument supplied in this or a previous declaration, unless the parameter was expanded from a parameter pack, or shall be a function parameter pack.

[Note 2: A default argument cannot be redefined by a later declaration (not even to the same value) (6.3). — end note]

[Example 2:

```

void g(int = 0, ...);           // OK, ellipsis is not a parameter so it can follow
                                // a parameter with a default argument

void f(int, int);
void f(int, int = 7);
void h() {
    f(3);                       // OK, calls f(3, 7)
    void f(int = 1, int);       // error: does not use default from surrounding scope
}

void m() {
    void f(int, int);           // has no defaults
    f(4);                       // error: wrong number of arguments
    void f(int, int = 5);       // OK
    f(4);                       // OK, calls f(4, 5);
    void f(int, int = 5);       // error: cannot redefine, even to same value
}

void n() {
    f(6);                       // OK, calls f(6, 7)
}

```

⁶⁹ One can explicitly disambiguate the parse either by introducing a comma (so the ellipsis will be parsed as part of the *parameter-declaration-clause*) or by introducing a name for the parameter (so the ellipsis will be parsed as part of the *declarator-id*).

⁷⁰ This means that default arguments cannot appear, for example, in declarations of pointers to functions, references to functions, or `typedef` declarations.


```

template<class ... T> struct C {
    void f(int n = 0, T...);
};
C<int> c;                                // OK, instantiates declaration void C::f(int n = 0, int)
— end example]

```

For a given inline function defined in different translation units, the accumulated sets of default arguments at the end of the translation units shall be the same; no diagnostic is required (G.6.3). If a friend declaration *D* specifies a default argument expression, that declaration shall be a definition and there shall be no other declaration of the function or function template which is reachable from *D* or from which *D* is reachable.

- ⁵ The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (9.5). The names in the default argument are looked up, and the semantic constraints are checked, at the point where the default argument appears, except that an immediate invocation (7.7.5) that is a potentially-evaluated subexpression (6.10.1) of the *initializer-clause* in a *parameter-declaration* is neither evaluated nor checked for whether it is a constant expression at that point. Name lookup and checking of semantic constraints for default arguments of templated functions are performed as described in 13.9.2.

[Example 3: In the following code, *g* will be called with the value *f*(2):

```

int a = 1;
int f(int);
int g(int x = f(a));                // default argument: f(::a)

void h() {
    a = 2;
    {
        int a = 3;
        g();                        // g(f(::a))
    }
}
— end example]

```

[Note 3: A default argument is a complete-class context (11.4). Access checking applies to names in default arguments as described in 11.8. — end note]

- ⁶ Except for member functions of templated classes, the default arguments in a member function definition that appears outside of the class definition are added to the set of default arguments provided by the member function declaration in the class definition; the program is ill-formed if a default constructor (11.4.5.2), copy or move constructor (11.4.5.3), or copy or move assignment operator (11.4.6) is so declared. Default arguments for a member function of a templated class shall be specified on the initial declaration of the member function within the templated class.

[Example 4:

```

class C {
    void f(int i = 3);
    void g(int i, int j = 99);
};

void C::f(int i = 3) {}              // error: default argument already specified in class scope
void C::g(int i = 88, int j) {}     // in this translation unit, C::g can be called with no arguments
— end example]

```

- ⁷ [Note 4: A local variable cannot be odr-used (6.3) in a default argument. — end note]

[Example 5:

```

void f() {
    int i;
    extern void g(int x = i);        // error
    extern void h(int x = sizeof(i)); // OK
    // ...
}
— end example]

```

- ⁸ [Note 5: The keyword *this* cannot appear in a default argument of a member function; see 7.5.3.

[Example 6:

```
class A {
    void f(A* p = this) { }           // error
};
— end example]
— end note]
```

- 9 A default argument is evaluated each time the function is called with no argument for the corresponding parameter. A parameter shall not appear as a potentially evaluated expression in a default argument.

[Note 6: Parameters of a function declared before a default argument are in scope and can hide namespace and class member names. — end note]

[Example 7:

```
int a;
int f(int a, int b = a);              // error: parameter a used as default argument
typedef int I;
int g(float I, int b = I(2));         // error: parameter I found
int h(int a, int b = sizeof(a));      // OK, unevaluated operand (7.2.3)
— end example]
```

A non-static member shall not be designated in a default argument unless

- (9.1) — it is designated by the *id-expression* or *splice-expression* of a class member access expression (7.6.1.5),
- (9.2) — it is designated by an expression used to form a pointer to member (7.6.2.2), or
- (9.3) — it appears as the operand of a *reflect-expression* (7.6.2.10).

[Example 8: The declaration of `X::mem1()` in the following example is ill-formed because no object is supplied for the non-static member `X::a` used as an initializer.

```
int b;
class X {
    int a;
    int mem1(int i = a);              // error: non-static member a used as default argument
    int mem2(int i = b);              // OK, use X::b
    consteval void mem3(std::meta::info r = ^^a) {} // OK
    int mem4(int i = [::^^a:]);       // error: non-static member a designated in default argument
    static int b;
};
```

The declaration of `X::mem2()` is meaningful, however, since no object is needed to access the static member `X::b`. Classes, objects, and members are described in [Clause 11](#). — end example]

A default argument is not part of the type of a function.

[Example 9:

```
int f(int = 0);

void h() {
    int j = f(1);
    int k = f();                      // OK, means f(0)
}

int (*p1)(int) = &f;
int (*p2)() = &f;                    // error: type mismatch
— end example]
```

[Note 7: When an overload set contains a declaration of a function whose host scope is *S*, any default argument associated with any reachable declaration whose host scope is *S* is available to the call (12.2.3). — end note]

[Note 8: The candidate might have been found through a *using-declarator* from which the declaration that provides the default argument is not reachable. — end note]

- 10 A virtual function call (11.7.3) uses the default arguments in the declaration of the virtual function determined by the static type of the pointer or reference denoting the object. An overriding function in a derived class does not acquire default arguments from the function it overrides.

[Example 10:

```

struct A {
    virtual void f(int a = 7);
};
struct B : public A {
    void f(int a);
};
void m() {
    B* pb = new B;
    A* pa = pb;
    pa->f();           // OK, calls pa->B::f(7)
    pb->f();           // error: wrong number of arguments for B::f()
}

```

— end example]

9.4 Function contract specifiers

[dcl.contract]

9.4.1 General

[dcl.contract.func]

function-contract-specifier-seq:

function-contract-specifier function-contract-specifier-seq_{opt}

function-contract-specifier:

precondition-specifier

postcondition-specifier

precondition-specifier:

pre attribute-specifier-seq_{opt} (conditional-expression)

postcondition-specifier:

post attribute-specifier-seq_{opt} (result-name-introducer_{opt} conditional-expression)

- ¹ A *function contract assertion* is a contract assertion (6.11.1) associated with a function. A *precondition-specifier* introduces a *precondition assertion*, which is a function contract assertion associated with entering a function. A *postcondition-specifier* introduces a *postcondition assertion*, which is a function contract assertion associated with exiting a function normally.

[Note 1: A postcondition assertion is not associated with exiting a function in any other fashion, such as via an exception (7.6.18) or via a call to `longjmp` (17.14.3). — end note]

- ² The predicate (6.11.1) of a function contract assertion is its *conditional-expression* contextually converted to `bool`.
- ³ Each *function-contract-specifier* of a *function-contract-specifier-seq* (if any) of an unspecified first declaration (6.2) of a function introduces a corresponding function contract assertion for that function. The optional *attribute-specifier-seq* following `pre` or `post` appertains to the introduced contract assertion.

[Note 2: The *function-contract-specifier-seq* of a *lambda-declarator* applies to the function call operator or operator template of the corresponding closure type (7.5.6.2). — end note]

- ⁴ A declaration *D* of a function or function template *f* that is not a first declaration shall have either no *function-contract-specifier-seq* or the same *function-contract-specifier-seq* (see below) as any first declaration *F* reachable from *D*. If *D* and *F* are in different translation units, a diagnostic is required only if *D* is attached to a named module. If a declaration *F*₁ is a first declaration of **f** in one translation unit and a declaration *F*₂ is a first declaration of **f** in another translation unit, *F*₁ and *F*₂ shall specify the same *function-contract-specifier-seq*, no diagnostic required (G.6.4).
- ⁵ A *function-contract-specifier-seq* *S*₁ is the same as a *function-contract-specifier-seq* *S*₂ if *S*₁ and *S*₂ consist of the same *function-contract-specifiers* in the same order. A *function-contract-specifier* *C*₁ on a function declaration *D*₁ is the same as a *function-contract-specifier* *C*₂ on a function declaration *D*₂ if

(5.1) — their predicates *P*₁ and *P*₂ would satisfy the one-definition rule (6.3) if placed in function definitions on the declarations *D*₁ and *D*₂, respectively, except for

(5.1.1) — renaming of the parameters of *f*,

(5.1.2) — renaming of template parameters of a template enclosing *f*, and

(5.1.3) — renaming of the result binding (9.4.2), if any,

and, if D_1 and D_2 are in different translation units, corresponding entities defined within each predicate behave as if there is a single entity with a single definition, and

- (5.2) — both C_1 and C_2 specify a *result-name-introducer* or neither do.

If this condition is not met solely due to the comparison of two *lambda-expressions* that are contained within P_1 and P_2 , no diagnostic is required.

[Note 3: Equivalent *function-contract-specifier-seqs* apply to all uses and definitions of a function across all translation units. — end note]

[Example 1:

```
bool b1, b2;

void f() pre (b1) pre ([]{ return b2; }());
void f(); // OK, function-contract-specifiers omitted
void f() pre (b1) pre ([]{ return b2; }()); // ill-formed, no diagnostic required: closures have
// different types
void f() pre (b1); // error: function-contract-specifiers only partially repeated

int g() post(r : b1);
int g() post(b1); // error: mismatched result-name-introducer presence

namespace N {
    void h() pre (b1);
    bool b1;
    void h() pre (b1); // error: function-contract-specifiers differ according to
// the one-definition rule (6.3).
}
```

— end example]

- 6 A deleted function (9.6.3), or a function defaulted on its first declaration (9.6.2) shall not have a *function-contract-specifier-seq*.
- 7 If the predicate of a postcondition assertion of a function f odr-uses (6.3) a non-reference parameter of f , that parameter and the corresponding parameter on all declarations of f shall have **const** type.

[Note 4: This requirement applies even to declarations that do not specify the *postcondition-specifier*. Parameters with array or function type will decay to non-**const** types even if a **const** qualifier is present.

[Example 2:

```
int f(const int i[10])
    post(r : r == i[0]); // error: i has type const int * (not int* const).
```

— end example]

— end note]

- 8 [Note 5: The function contract assertions of a function are evaluated even when invoked indirectly, such as through a pointer to function or a pointer to member function. A pointer to function, pointer to member function, or function type alias cannot have a *function-contract-specifier-seq* associated directly with it. — end note]

- 9 The function contract assertions of a function are considered to be *needed* (13.9.2) when

- (9.1) — the function is odr-used (6.3) or
- (9.2) — the function is defined.

[Note 6: Overload resolution does not consider *function-contract-specifiers* (13.10.3, 13.9.2).

[Example 3:

```
template <typename T> void f(T t) pre( t == "" );
template <typename T> void f(T&& t);
void g()
{
    f(5); // error: ambiguous
}
```

— end example]

— end note]

9.4.2 Referring to the result object

[dcl.contract.res]

*attributed-identifier:**identifier attribute-specifier-seq_{opt}**result-name-introducer:**attributed-identifier :*

- ¹ The *result-name-introducer* of a *postcondition-specifier* is a declaration. The *result-name-introducer* introduces the *identifier* as the name of a *result binding* of the associated function. If a postcondition assertion has a *result-name-introducer* and the return type of the function is *cv void*, the program is ill-formed. A result binding denotes the object or reference returned by invocation of that function. The type of a result binding is the return type of its associated function. The optional *attribute-specifier-seq* of the *attributed-identifier* in the *result-name-introducer* appertains to the result binding so introduced.

[Note 1: An *id-expression* that names a result binding is a **const** lvalue (7.5.5.2). — end note]

[Example 1:

```
int f()
  post(r : r == 1)
{
  return 1;
}
int i = f();    // Postcondition check succeeds.
```

— end example]

[Example 2:

```
struct A {};
struct B {
  B() {}
  B(const B&) {}
};

template <typename T>
T f(T* const ptr)
  post(r: &r == ptr)
{
  return {};
}

int main() {
  A a = f(&a);    // The postcondition check can fail if the implementation introduces
                  // a temporary for the return value (6.8.7).
  B b = f(&b);    // The postcondition check succeeds, no temporary is introduced.
}
```

— end example]

- ² When the declared return type of a non-templated function contains a placeholder type, a *postcondition-specifier* with a *result-name-introducer* shall be present only on a definition.

[Example 3:

```
auto g(auto&)
  post (r: r >= 0);    // OK, g is a template.

auto h()
  post (r: r >= 0);    // error: cannot name the return value

auto k()
  post (r: r >= 0)    // OK
{
  return 0;
}
```

— end example]

9.5 Initializers

[dcl.init]

9.5.1 General

[dcl.init.general]

- ¹ The process of initialization described in 9.5 applies to all initializations regardless of syntactic context, including the initialization of a function parameter (7.6.1.3), the initialization of a return value (8.8.4), or when an initializer follows a declarator.

```

initializer:
    brace-or-equal-initializer
    ( expression-list )

brace-or-equal-initializer:
    = initializer-clause
    braced-init-list

initializer-clause:
    assignment-expression
    braced-init-list

braced-init-list:
    { initializer-list ,opt }
    { designated-initializer-list ,opt }
    { }

initializer-list:
    initializer-clause . . .opt
    initializer-list , initializer-clause . . .opt

designated-only-initializer-list:
    designated-initializer-clause
    designated-only-initializer-list , designated-initializer-clause

designated-initializer-list:
    designated-only-initializer-list
    initializer-list , designated-only-initializer-list

designated-initializer-clause:
    designator brace-or-equal-initializer

designator:
    . identifier

expr-or-braced-init-list:
    expression
    braced-init-list

```

[Note 1: The rules in 9.5 apply even if the grammar permits only the *brace-or-equal-initializer* form of *initializer* in a given context. — end note]

- ² Except for objects declared with the `constexpr` specifier, for which see 9.2.6, an *initializer* in the definition of a variable can consist of arbitrary expressions involving literals and previously declared variables and functions, regardless of the variable's storage duration.

[Example 1:

```

int f(int);
int a = 2;
int b = f(a);
int c(b);

```

— end example]

- ³ [Note 2: Default arguments are more restricted; see 9.3.4.7. — end note]

- ⁴ [Note 3: The order of initialization of variables with static storage duration is described in 6.10.3 and 8.10. — end note]

- ⁵ To *zero-initialize* an object or reference of type T means:

- (5.1) — if T is `std::meta::info`, the object is initialized to a null reflection value;
- (5.2) — if T is any other scalar type (6.9.1), the object is initialized to the value obtained by converting the integer literal 0 (zero) to T;⁷¹

⁷¹) As specified in 7.3.12, converting an integer literal whose value is 0 to a pointer type results in a null pointer value.

- (5.3) — if *T* is a (possibly cv-qualified) non-union class type, its padding bits (6.9.1) are initialized to zero bits and each non-static data member, each non-virtual base class subobject, and, if the object is not a base class subobject, each virtual base class subobject is zero-initialized;
- (5.4) — if *T* is a (possibly cv-qualified) union type, its padding bits (6.9.1) are initialized to zero bits and the object's first non-static named data member is zero-initialized;
- (5.5) — if *T* is an array type, each element is zero-initialized;
- (5.6) — if *T* is a reference type, no initialization is performed.

⁶ To *default-initialize* an object of type *T* means:

- (6.1) — If *T* is a (possibly cv-qualified) class type (Clause 11), constructors are considered. The applicable constructors are enumerated (12.2.2.4), and the best one for the *initializer* () is chosen through overload resolution (12.2). The constructor thus selected is called, with an empty argument list, to initialize the object.
- (6.2) — If *T* is an array type, the semantic constraints of default-initializing a hypothetical element shall be met and each element is default-initialized.
- (6.3) — If *T* is `std::meta::info`, the object is zero-initialized.
- (6.4) — Otherwise, no initialization is performed.

⁷ A type *cv T* is *const-default-constructible* if

- (7.1) — *T* is `std::meta::info`;
- (7.2) — *T* is `std::nullptr_t`;
- (7.3) — default-initialization of *T* would invoke a user-provided constructor of *T* (not inherited from a base class);
- (7.4) — *T* is a class type where
 - (7.4.1) — each direct non-variant non-static data member of *T* has a default member initializer or is of const-default-constructible type,
 - (7.4.2) — if *T* is a union with at least one non-static data member, exactly one variant member has a default member initializer,
 - (7.4.3) — if *T* is not a union, the type of each anonymous union member is const-default-constructible, and
 - (7.4.4) — each potentially constructed base class of *T* is const-default-constructible; or
- (7.5) — *T* is an array of const-default-constructible type.

If a program calls for the default-initialization of an object of a const-qualified type *T*, *T* shall be a const-default-constructible type.

⁸ To *value-initialize* an object of type *T* means:

- (8.1) — If *T* is a (possibly cv-qualified) class type (Clause 11), then let *C* be the constructor selected to default-initialize the object, if any. If *C* is not user-provided, the object is first zero-initialized. In all cases, the object is then default-initialized.
- (8.2) — If *T* is an array type, the semantic constraints of value-initializing a hypothetical element shall be met and each element is value-initialized.
- (8.3) — Otherwise, the object is zero-initialized.

⁹ A program that calls for default-initialization or value-initialization of an entity of reference type is ill-formed.

¹⁰ [Note 4: For every object with static storage duration, static initialization (6.10.3.2) is performed at program startup before any other initialization takes place. In some cases, additional initialization is done later. — end note]

¹¹ If no initializer is specified for an object, the object is default-initialized.

¹² If the entity being initialized does not have class or array type, the *expression-list* in a parenthesized initializer shall be a single expression.

¹³ The initialization that occurs in the = form of a *brace-or-equal-initializer* or *condition* (8.5), as well as in argument passing, function return, throwing an exception (14.2), handling an exception (14.4), and aggregate member initialization other than by a *designated-initializer-clause* (9.5.2), is called *copy-initialization*.

[Note 5: Copy-initialization can invoke a move (11.4.5.3). — end note]

14 The initialization that occurs

- (14.1) — for an *initializer* that is a parenthesized *expression-list* or a *braced-init-list*,
- (14.2) — for a *new-initializer* (7.6.2.8),
- (14.3) — in a `static_cast` expression (7.6.1.9),
- (14.4) — in a functional notation type conversion (7.6.1.4), and
- (14.5) — in the *braced-init-list* form of a *condition*

is called *direct-initialization*.

15 The semantics of initializers are as follows. The *destination type* is the cv-unqualified type of the object or reference being initialized and the *source type* is the type of the initializer expression. If the initializer is not a single (possibly parenthesized) expression, the source type is not defined.

- (15.1) — If the initializer is a (non-parenthesized) *braced-init-list* or is `= braced-init-list`, the object or reference is list-initialized (9.5.5).
- (15.2) — If the destination type is a reference type, see 9.5.4.
- (15.3) — If the destination type is an array of characters, an array of `char8_t`, an array of `char16_t`, an array of `char32_t`, or an array of `wchar_t`, and the initializer is a *string-literal*, see 9.5.3.
- (15.4) — If the initializer is `()`, the object is value-initialized.

[Note 6: Since `()` is not permitted by the syntax for *initializer*,

```
X a();
```

is not the declaration of an object of class `X`, but the declaration of a function taking no arguments and returning an `X`. The form `()` can appear in certain other initialization contexts (7.6.2.8, 7.6.1.4, 11.9.3). — end note]

- (15.5) — Otherwise, if the destination type is an array, the object is initialized as follows. The *initializer* shall be of the form `( expression-list )`. Let $x_1, \dots, x_k$ be the elements of the *expression-list*. If the destination type is an array of unknown bound, it is defined as having k elements. Let n denote the array size after this potential adjustment. If k is greater than n , the program is ill-formed. Otherwise, the i^{th} array element is copy-initialized with x_i for each $1 \leq i \leq k$, and value-initialized for each $k < i \leq n$. For each $1 \leq i < j \leq n$, every value computation and side effect associated with the initialization of the i^{th} element of the array is sequenced before those associated with the initialization of the j^{th} element.
- (15.6) — Otherwise, if the destination type is a class type:
 - (15.6.1) — If the initializer expression is a prvalue and the cv-unqualified version of the source type is the same as the destination type, the initializer expression is used to initialize the destination object.

[Example 2: `T x = T(T());` value-initializes `x` (7.2.1, 7.6.1.4). — end example]
 - (15.6.2) — Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same as or is derived from the class of the destination type, constructors are considered. The applicable constructors are enumerated (12.2.2.4), and the best one is chosen through overload resolution (12.2). Then:
 - (15.6.2.1) — If overload resolution is successful, the selected constructor is called to initialize the object, with the initializer expression or *expression-list* as its argument(s).
 - (15.6.2.2) — Otherwise, if no constructor is viable, the destination type is an aggregate class, and the initializer is a parenthesized *expression-list*, the object is initialized as follows. Let $e_1, \dots, e_n$ be the elements of the aggregate (9.5.2). Let $x_1, \dots, x_k$ be the elements of the *expression-list*. If k is greater than n , the program is ill-formed. The element e_i is copy-initialized with x_i for $1 \leq i \leq k$. The remaining elements are initialized with their default member initializers, if any, and otherwise are value-initialized. For each $1 \leq i < j \leq n$, every value computation and side effect associated with the initialization of e_i is sequenced before those associated with the initialization of e_j .

[Note 7: By contrast with direct-list-initialization, narrowing conversions (9.5.5) can appear, designators are not permitted, a temporary object bound to a reference does not have its lifetime extended (6.8.7), and there is no brace elision.

[Example 3:

```
struct A {
    int a;
```

```

    int&& r;
};

int f();
int n = 10;

A a1{1, f()};           // OK, lifetime is extended
A a2{1, f()};           // well-formed, but dangling reference
A a3{1.0, 1};           // error: narrowing conversion
A a4{1.0, 1};           // well-formed, but dangling reference
A a5{1.0, std::move(n)}; // OK
— end example]
— end note]

```

(15.6.2.3) — Otherwise, the initialization is ill-formed.

(15.6.3) — Otherwise (i.e., for the remaining copy-initialization cases), user-defined conversions that can convert from the source type to the destination type or (when a conversion function is used) to a derived class thereof are enumerated as described in 12.2.2.5, and the best one is chosen through overload resolution (12.2). If the conversion cannot be done or is ambiguous, the initialization is ill-formed. The function selected is called with the initializer expression as its argument; if the function is a constructor, the call is a prvalue of the cv-unqualified version of the destination type whose result object is initialized by the constructor. The call is used to direct-initialize, according to the rules above, the object that is the destination of the copy-initialization.

(15.7) — Otherwise, if the source type is a (possibly cv-qualified) class type, conversion functions are considered. The applicable conversion functions are enumerated (12.2.2.6), and the best one is chosen through overload resolution (12.2). The user-defined conversion so selected is called to convert the initializer expression into the object being initialized. If the conversion cannot be done or is ambiguous, the initialization is ill-formed.

(15.8) — Otherwise, if the initialization is direct-initialization, the source type is `std::nullptr_t`, and the destination type is `bool`, the initial value of the object being initialized is `false`.

(15.9) — Otherwise, the initial value of the object being initialized is the (possibly converted) value of the initializer expression. A standard conversion sequence (7.3) is used to convert the initializer expression to a prvalue of the destination type; no user-defined conversions are considered. If the conversion cannot be done, the initialization is ill-formed. When initializing a bit-field with a value that it cannot represent, the resulting value of the bit-field is implementation-defined.

[Note 8: An expression of type “*cv1* T” can initialize an object of type “*cv2* T” independently of the cv-qualifiers *cv1* and *cv2*.

```

    int a;
    const int b = a;
    int c = b;
— end note]

```

16 An immediate invocation (7.7.5) that is not evaluated where it appears (9.3.4.7, 11.4.1) is evaluated and checked for whether it is a constant expression at the point where the enclosing *initializer* is used in a function call, a constructor definition, or an aggregate initialization.

17 An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4).

18 Initialization includes the evaluation of all subexpressions of each *initializer-clause* of the initializer (possibly nested within *braced-init-lists*) and the creation of any temporary objects for function arguments or return values (6.8.7).

19 If the destination type is not an aggregate and the initializer is a parenthesized *expression-list*, the expressions are evaluated in the order specified for function calls (7.6.1.3).

20 The same *identifier* shall not appear in multiple *designators* of a *designated-initializer-list*.

21 The *deemed construction* of an object occurs when its initialization completes; for the purposes of 6.10.3.4, if the initialization is a full-expression, deemed construction occurs when the evaluation of that full-expression completes. The object is deemed to be constructed, even if the object is of non-class type or no constructor of the object’s class is invoked for the initialization.

[*Note 9*: Such an object might have been value-initialized or initialized by aggregate initialization (9.5.2) or by an inherited constructor (11.9.4). — *end note*]

Destroying an object of class type invokes the destructor of the class. Destroying a scalar type has no effect other than ending the lifetime of the object (6.8.4). Destroying an array destroys each element in reverse subscript order.

- 22 A declaration that specifies the initialization of a variable, whether from an explicit initializer or by default-initialization, is called the *initializing declaration* of that variable.

[*Note 10*: In most cases this is the defining declaration (6.2) of the variable, but the initializing declaration of a non-inline static data member (11.4.9.3) can be the declaration within the class definition and not the definition (if any) outside it. — *end note*]

9.5.2 Aggregates

[dcl.init.aggr]

- 1 An *aggregate* is an array or a class (Clause 11) with

- (1.1) — no user-declared or inherited constructors (11.4.5),
- (1.2) — no private or protected direct non-static data members (11.8),
- (1.3) — no private or protected direct base classes (11.8.3), and
- (1.4) — no virtual functions (11.7.3) or virtual base classes (11.7.2).

[*Note 1*: Aggregate initialization does not allow accessing protected and private base class' members, including constructors. — *end note*]

- 2 The *elements* of an aggregate are:

- (2.1) — for an array, the array elements in increasing subscript order, or
- (2.2) — for a class, the direct base classes in declaration order, followed by the direct non-static data members (11.4) that are not members of an anonymous union, in declaration order.

- 3 The *associated element* of a member M of an aggregate T is:

- (3.1) — M if M is an element of T;
- (3.2) — otherwise, the element of T that contains M.

[*Example 1*:

```
struct A {
    int a1;
    union {
        int a2;
        char a3;
    };
};

struct B : A {
    int b1;
    union {
        double b2;
    };
};
```

The associated element of each of the members A::a1, A::a2, and A::a3 of B is A. The associated element of the member B::b1 of B is itself. The associated element of the member B::b2 of B is the anonymous union containing it. — *end example*]

- 4 When an aggregate is initialized by an initializer list as specified in 9.5.5, the elements of the initializer list are taken as initializers for the elements of the aggregate. The *explicitly initialized elements* of the aggregate are determined as follows:

- (4.1) — If the initializer list is a brace-enclosed *designated-initializer-list*, the aggregate shall be of class type C. For each *designator* within the *designated-only-initializer-list* of the *designated-initializer-list*, the lookup set for the *identifier* in C (6.5.2) shall comprise
 - (4.1.1) — a declaration set consisting of a single non-static data member and

- (4.1.2) — a subobject set containing only one subobject, whose type shall be an aggregate base of *C*. An aggregate *B* is an *aggregate base* of a class *D* if it is *D* or a direct base class of an aggregate base of *D*.

That non-static data member is *designated* by the *identifier*. Each *initializer-clause* of the *initializer-list*, if any, shall appertain (see below) to a (direct) base class subobject of *C*. The explicitly initialized elements of the aggregate include the associated elements of each member *M* of *C* for which *M* is designated by an *identifier* in a *designated-initializer-clause*.

- (4.2) — If the *braced-init-list* has an *initializer-list* (possibly within a *designated-initializer-list*) the explicitly initialized elements of the aggregate include those for which an element of the initializer list appertains to the aggregate element or to a subobject thereof (see below).
- (4.3) — No other elements of the aggregate are explicitly initialized.

[*Note 2*: The initializer {} does not explicitly initialize any elements of the aggregate. — *end note*]

If any element of the aggregate is explicitly initialized per both an *initializer-list* and a *designated-only-initializer-list*, the program is ill-formed.

[*Example 2*:

```
struct A { int a1, a2; };
struct B : A { int b; };
struct C : A { int a1; };
```

```
A v0 = A{1, .a2=2};           // error: initializer-clause appertains to non-static data member
B v1 = B{.a1=1, .b=2};       // the explicitly initialized elements are [A, B::b]
B v2 = B{.a1=1, .a2=2, .b=3}; // the explicitly initialized elements are [A, B::b]
B v3 = B{A{1, 2}, .b=3};     // the explicitly initialized elements are [A, B::b]
B v4 = B{A{}, .a2=1, .b=3};  // error: A initialized two different ways
C v5 = C{.a1=4};             // the explicitly initialized elements are [C::a1]
```

— *end example*]

- 5 For each explicitly initialized element:

- (5.1) — If the element is an anonymous union member and is explicitly initialized per a *designated-only-initializer-list*, the element is initialized by the *braced-init-list* { *D* }, where *D* is the *designated-initializer-clause* whose associated element is the anonymous union member. There shall be only one such *designated-initializer-clause*.

[*Example 3*:

```
struct C {
    union {
        int a;
        const char* p;
    };
    int x;
} c = { .a = 1, .x = 3 };
```

initializes *c.a* with 1 and *c.x* with 3. — *end example*]

- (5.2) — Otherwise, if the element is explicitly initialized per a *designated-only-initializer-list*, then:

- (5.2.1) — If the element is a (direct) non-static data member, then the element is initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*. If that initializer is of the form = *assignment-expression* and a narrowing conversion (9.5.5) is required to convert the expression, the program is ill-formed.

[*Note 3*: The form of the initializer determines whether copy-initialization or direct-initialization is performed. — *end note*]

- (5.2.2) — Otherwise, the element is a base class subobject *B*, and is copy-initialized from a brace-enclosed *designated-initializer-list* consisting of all of the *designated-initializer-clauses* whose associated element is *B*, in order.

[*Example 4*:

```
struct A { int a; };
struct B : A { int b; };
struct C : B { int c; };
```

```
// the A element is initialized from {.a=1}
B x = B{.a=1};

// the B element is initialized from {.a=2, .b=3}
// which leads to its A element being initialized from {.a=2}
C y = C{.a=2, .b=3, .c=4};

— end example]
```

- (5.3) — Otherwise, element is explicitly initialized per an *initializer-list*. If an *initializer-clause* appertains to the aggregate element, then the aggregate element is copy-initialized from the *initializer-clause*. Otherwise, the aggregate element is copy-initialized from a brace-enclosed *initializer-list* consisting of all of the *initializer-clauses* that appertain to subobjects of the aggregate element, in the order of appearance.

[*Note 4*: If an initializer is itself an initializer list, the element is list-initialized, which will result in a recursive application of the rules in this subclause if the element is an aggregate. — end note]

[*Example 5*:

```
struct A {
    int x;
    struct B {
        int i;
        int j;
    } b;
} a = { 1, { 2, 3 } };
```

initializes `a.x` with 1, `a.b.i` with 2, `a.b.j` with 3.

```
struct base1 { int b1, b2 = 42; };
struct base2 {
    base2() {
        b3 = 42;
    }
    int b3;
};
struct derived : base1, base2 {
    int d;
};
```

```
derived d1{{1, 2}, {}, 4};
derived d2{{}, {}, 4};
```

initializes `d1.b1` with 1, `d1.b2` with 2, `d1.b3` with 42, `d1.d` with 4, and `d2.b1` with 0, `d2.b2` with 42, `d2.b3` with 42, `d2.d` with 4. — end example]

- ⁶ For a non-union aggregate, each element that is not an explicitly initialized element is initialized as follows:
- (6.1) — If the element has a default member initializer (11.4), the element is initialized from that initializer.
- (6.2) — Otherwise, if the element is not a reference, the element is copy-initialized from an empty initializer list (9.5.5).
- (6.3) — Otherwise, the program is ill-formed.

If the aggregate is a union and the initializer list is empty, then

- (6.4) — if any variant member has a default member initializer, that member is initialized from its default member initializer;
- (6.5) — otherwise, the first member of the union (if any) is copy-initialized from an empty initializer list.

⁷ [*Example 6*:

```
struct S { int a; const char* b; int c; int d = b[a]; };
S ss = { 1, "asdf" };
```

initializes `ss.a` with 1, `ss.b` with "asdf", `ss.c` with the value of an expression of the form `int{}` (that is, 0), and `ss.d` with the value of `ss.b[ss.a]` (that is, 's').

```
struct A {
    string a;
    int b = 42;
```

```
    int c = -1;
};
```

A{.c=21} has the following steps:

- (7.1) — Initialize **a** with {}
- (7.2) — Initialize **b** with = 42
- (7.3) — Initialize **c** with = 21

— end example]

- 8 The initializations of the elements of the aggregate are evaluated in the element order. That is, all value computations and side effects associated with a given element are sequenced before those of any element that follows it in order.
- 9 An aggregate that is a class can also be initialized with a single expression not enclosed in braces, as described in 9.5.
- 10 The destructor for each element of class type other than an anonymous union member is potentially invoked (11.4.7) from the context where the aggregate initialization occurs.
[Note 5: This provision ensures that destructors can be called for fully-constructed subobjects in case an exception is thrown (14.3). — end note]
- 11 The number of elements (9.3.4.5) in an array of unknown bound initialized with a brace-enclosed *initializer-list* is the number of explicitly initialized elements of the array.

[Example 7:

```
int x[] = { 1, 3, 5 };
```

declares and initializes **x** as a one-dimensional array that has three elements since no size was specified and there are three initializers. — end example]

[Example 8: In

```
struct X { int i, j, k; };
X a[] = { 1, 2, 3, 4, 5, 6 };
X b[2] = { { 1, 2, 3 }, { 4, 5, 6 } };
```

a and **b** have the same value. — end example]

An array of unknown bound shall not be initialized with an empty *braced-init-list* {}. ⁷²

[Note 6: A default member initializer does not determine the bound for a member array of unknown bound. Since the default member initializer is ignored if a suitable *mem-initializer* is present (11.9.3), the default member initializer is not considered to initialize the array of unknown bound.

[Example 9:

```
struct S {
    int y[] = { 0 };           // error: non-static data member of incomplete type
};
```

— end example]

— end note]

- 12 [Note 7: Static data members, non-static data members of anonymous union members, and unnamed bit-fields are not considered elements of the aggregate.

[Example 10:

```
struct A {
    int i;
    static int s;
    int j;
    int :17;
    int k;
} a = { 1, 2, 3 };
```

Here, the second initializer 2 initializes **a.j** and not the static data member **A::s**, and the third initializer 3 initializes **a.k** and not the unnamed bit-field before it. — end example]

— end note]

⁷²⁾ The syntax provides for empty *braced-init-lists*, but nonetheless C++ does not have zero length arrays.

- 13 If a member has a default member initializer and a potentially-evaluated subexpression thereof is an aggregate initialization that would use that default member initializer, the program is ill-formed.

[Example 11:

```
struct A;
extern A a;
struct A {
    const A& a1 { A{a,a} };           // OK
    const A& a2 { A{} };               // error
};
A a{a,a};                             // OK

struct B {
    int n = B{}.n;                     // error
};
```

— end example]

- 14 When initializing a multidimensional array, the *initializer-clauses* initialize the elements with the last (rightmost) index of the array varying the fastest (9.3.4.5).

[Example 12:

```
int x[2][2] = { 3, 1, 4, 2 };
```

initializes `x[0][0]` to 3, `x[0][1]` to 1, `x[1][0]` to 4, and `x[1][1]` to 2. On the other hand,

```
float y[4][3] = {
    { 1 }, { 2 }, { 3 }, { 4 }
};
```

initializes the first column of `y` (regarded as a two-dimensional array) and leaves the rest zero. — end example]

- 15 Each *initializer-clause* in a *braced-init-list* is said to *appertain* to an element of the aggregate being initialized or to an element of one of its subaggregates. Considering the sequence of *initializer-clauses*, and the sequence of aggregate elements initially formed as the sequence of elements of the aggregate being initialized and potentially modified as described below, each *initializer-clause* appertains to the corresponding aggregate element if

- (15.1) — the aggregate element is not an aggregate, or
- (15.2) — the *initializer-clause* begins with a left brace, or
- (15.3) — the *initializer-clause* is an expression and an implicit conversion sequence can be formed that converts the expression to the type of the aggregate element, or
- (15.4) — the aggregate element is an aggregate that itself has no aggregate elements.

Otherwise, the aggregate element is an aggregate and that subaggregate is replaced in the list of aggregate elements by the sequence of its own aggregate elements, and the appertainment analysis resumes with the first such element and the same *initializer-clause*.

[Note 8: These rules apply recursively to the aggregate's subaggregates.

[Example 13: In

```
struct S1 { int a, b; };
struct S2 { S1 s, t; };

S2 x[2] = { 1, 2, 3, 4, 5, 6, 7, 8 };
S2 y[2] = {
    {
        { 1, 2 },
        { 3, 4 }
    },
    {
        { 5, 6 },
        { 7, 8 }
    }
};
```

`x` and `y` have the same value. — end example]

— end note]

This process continues until all *initializer-clauses* have been exhausted. If any *initializer-clause* remains that does not appertain to an element of the aggregate or one of its subaggregates, the program is ill-formed.

[Example 14:

```
char cv[4] = { 'a', 's', 'd', 'f', 0 };    // error: too many initializers
```

— end example]

16 [Example 15:

```
float y[4][3] = {
    { 1, 3, 5 },
    { 2, 4, 6 },
    { 3, 5, 7 },
};
```

is a completely-braced initialization: 1, 3, and 5 initialize the first row of the array `y[0]`, namely `y[0][0]`, `y[0][1]`, and `y[0][2]`. Likewise the next two lines initialize `y[1]` and `y[2]`. The initializer ends early and therefore `y[3]`'s elements are initialized as if explicitly initialized with an expression of the form `float()`, that is, are initialized with 0.0. In the following example, braces in the *initializer-list* are elided; however the *initializer-list* has the same effect as the completely-braced *initializer-list* of the above example,

```
float y[4][3] = {
    1, 3, 5, 2, 4, 6, 3, 5, 7
};
```

The initializer for `y` begins with a left brace, but the one for `y[0]` does not, therefore three elements from the list are used. Likewise the next three are taken successively for `y[1]` and `y[2]`. — end example]

17 [Note 9: The initializer for an empty subaggregate is needed if any initializers are provided for subsequent elements.

[Example 16:

```
struct S { } s;
struct A {
    S s1;
    int i1;
    S s2;
    int i2;
    S s3;
    int i3;
} a = {
    { },           // Required initialization
    0,
    s,             // Required initialization
    0
};               // Initialization not required for A::s3 because A::i3 is also not initialized
```

— end example]

— end note]

18 [Example 17:

```
struct A {
    int i;
    operator int();
};
struct B {
    A a1, a2;
    int z;
};
A a;
B b = { 4, a, a };
```

Braces are elided around the *initializer-clause* for `b.a1.i`. `b.a1.i` is initialized with 4, `b.a2` is initialized with `a`, `b.z` is initialized with whatever `a.operator int()` returns. — end example]

19 [Note 10: An aggregate array or an aggregate class can contain elements of a class type with a user-declared constructor (11.4.5). Initialization of these aggregate objects is described in 11.9.2. — end note]

²⁰ [Note 11: Whether the initialization of aggregates with static storage duration is static or dynamic is specified in 6.10.3.2, 6.10.3.3, and 8.10. — end note]

²¹ When a union is initialized with an initializer list, there shall not be more than one explicitly initialized element.

[Example 18:

```
union u { int a; const char* b; };
u a = { 1 };
u b = a;
u c = 1;                                // error
u d = { 0, "asdf" };                    // error
u e = { "asdf" };                        // error
u f = { .b = "asdf" };
u g = { .a = 1, .b = "asdf" };          // error
```

— end example]

²² [Note 12: As described above, the braces around the *initializer-clause* for a union member can be omitted if the union is a member of another aggregate. — end note]

9.5.3 Character arrays

[dcl.init.string]

¹ An array of ordinary character type (6.9.3), `char8_t` array, `char16_t` array, `char32_t` array, or `wchar_t` array may be initialized by an ordinary string literal, UTF-8 string literal, UTF-16 string literal, UTF-32 string literal, or wide string literal, respectively, or by an appropriately-typed *string-literal* enclosed in braces (5.13.5). Additionally, an array of `char` or `unsigned char` may be initialized by a UTF-8 string literal, or by such a string literal enclosed in braces. Successive characters of the value of the *string-literal* initialize the elements of the array, with an integral conversion (7.3.9) if necessary for the source and destination value.

[Example 1:

```
char msg[] = "Syntax error on line %s\n";
```

shows a character array whose members are initialized with a *string-literal*. Note that because `'\n'` is a single character and because a trailing `'\0'` is appended, `sizeof(msg)` is 25. — end example]

² There shall not be more initializers than there are array elements.

[Example 2:

```
char cv[4] = "asdf";                    // error
```

is ill-formed since there is no space for the implied trailing `'\0'`. — end example]

³ If there are fewer initializers than there are array elements, each element not explicitly initialized shall be zero-initialized (9.5).

9.5.4 References

[dcl.init.ref]

¹ A variable whose declared type is “reference to T” (9.3.4.3) shall be initialized.

[Example 1:

```
int g(int) noexcept;
void f() {
    int i;
    int& r = i;                        // r refers to i
    r = 1;                             // the value of i becomes 1
    int* p = &r;                       // p points to i
    int& rr = r;                       // rr refers to what r refers to, that is, to i
    int (&rg)(int) = g;                // rg refers to the function g
    rg(i);                             // calls function g
    int a[3];
    int (&ra)[3] = a;                  // ra refers to the array a
    ra[1] = i;                         // modifies a[1]
}
```

— end example]

² A reference cannot be changed to refer to another object after initialization.

[Note 1: Assignment to a reference assigns to the object referred to by the reference (7.6.19). — end note]

Argument passing (7.6.1.3) and function value return (8.8.4) are initializations.

- ³ The initializer can be omitted for a reference only in a parameter declaration (9.3.4.6), in the declaration of a function return type, in the declaration of a class member within its class definition (11.4), and where the **extern** specifier is explicitly used.

[Example 2:

```
int& r1;                // error: initializer missing
extern int& r2;          // OK
— end example]
```

- ⁴ Given types “*cv1* T1” and “*cv2* T2”, “*cv1* T1” is *reference-related* to “*cv2* T2” if T1 is similar (7.3.6) to T2, or T1 is a base class of T2. “*cv1* T1” is *reference-compatible* with “*cv2* T2” if a prvalue of type “pointer to *cv2* T2” can be converted to the type “pointer to *cv1* T1” via a standard conversion sequence (7.3). In all cases where the reference-compatible relationship of two types is used to establish the validity of a reference binding and the standard conversion sequence would be ill-formed, a program that necessitates such a binding is ill-formed.

- ⁵ A reference to type “*cv1* T1” is initialized by an expression of type “*cv2* T2” as follows:

- (5.1) — If the reference is an lvalue reference and the initializer expression

- (5.1.1) — is an lvalue (but is not a bit-field), and “*cv1* T1” is reference-compatible with “*cv2* T2”, or

- (5.1.2) — has a class type (i.e., T2 is a class type), where T1 is not reference-related to T2, and can be converted to an lvalue of type “*cv3* T3”, where “*cv1* T1” is reference-compatible with “*cv3* T3”⁷³ (this conversion is selected by enumerating the applicable conversion functions (12.2.2.7) and choosing the best one through overload resolution (12.2)),

then the reference binds to the initializer expression lvalue in the first case and to the lvalue result of the conversion in the second case (or, in either case, to the appropriate base class subobject of the object).

[Note 2: The usual lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions are not needed, and therefore are suppressed, when such direct bindings to lvalues are done. — end note]

[Example 3:

```
double d = 2.0;
double& rd = d;          // rd refers to d
const double& rcd = d;   // rcd refers to d

struct A { };
struct B : A { operator int&(); } b;
A& ra = b;               // ra refers to A subobject in b
const A& rca = b;         // rca refers to A subobject in b
int& ir = B();            // ir refers to the result of B::operator int&
— end example]
```

- (5.2) — Otherwise, if the reference is an lvalue reference to a type that is not const-qualified or is volatile-qualified, the program is ill-formed.

[Example 4:

```
double& rd2 = 2.0;        // error: not an lvalue and reference not const
int i = 2;
double& rd3 = i;          // error: type mismatch and reference not const
— end example]
```

- (5.3) — Otherwise, if the initializer expression

- (5.3.1) — is an rvalue (but not a bit-field) or an lvalue of function type and “*cv1* T1” is reference-compatible with “*cv2* T2”, or

- (5.3.2) — has a class type (i.e., T2 is a class type), where T1 is not reference-related to T2, and can be converted to an rvalue of type “*cv3* T3” or an lvalue of function type “*cv3* T3”, where “*cv1* T1” is reference-compatible with “*cv3* T3” (see 12.2.2.7),

⁷³) This requires a conversion function (11.4.8.3) returning a reference type.

then the initializer expression in the first case and the converted expression in the second case is called the converted initializer. If the converted initializer is a prvalue, let its type be denoted by T4; the temporary materialization conversion (7.3.5) is applied, considering the type of the prvalue to be “*cv1* T4” (7.3.6). In any case, the reference binds to the resulting glvalue (or to an appropriate base class subobject).

[Example 5:

```

struct A { };
struct B : A { } b;
extern B f();
const A& rca2 = f();           // binds to the A subobject of the B rvalue.
A&& rra = f();                 // same as above
struct X {
    operator B();
    operator int&();
} x;
const A& r = x;                // binds to the A subobject of the result of the conversion
int i2 = 42;
int&& rri = static_cast<int&&>(i2); // binds directly to i2
B&& rrb = x;                    // binds directly to the result of operator B

constexpr int f() {
    const int &x = 42;
    const_cast<int &>(x) = 1;    // undefined behavior
    return x;
}
constexpr int z = f();         // error: not a constant expression

typedef int *AP[3];             // array of 3 pointer to int
typedef const int *const ACPC[3]; // array of 3 const pointer to const int
ACPC &&r = AP{};                // binds directly

```

— end example]

- (5.4) — Otherwise, T1 shall not be reference-related to T2.
- (5.4.1) — If T1 or T2 is a class type, user-defined conversions are considered using the rules for copy-initialization of an object of type “*cv1* T1” by user-defined conversion (9.5, 12.2.2.5, 12.2.2.6); the program is ill-formed if the corresponding non-reference copy-initialization would be ill-formed. The result *E* of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference using the form (*E*). For this direct-initialization, user-defined conversions are not considered.
- (5.4.2) — Otherwise, the initializer expression is implicitly converted to a prvalue of type “T1”. The temporary materialization conversion is applied, considering the type of the prvalue to be “*cv1* T1”, and the reference is bound to the result.

[Example 6:

```

struct Banana { };
struct Enigma { operator const Banana(); };
struct Alaska { operator Banana&(); };
void enigmatic() {
    typedef const Banana ConstBanana;
    Banana &&banana1 = ConstBanana(); // error
    Banana &&banana2 = Enigma();      // error
    Banana &&banana3 = Alaska();      // error
}

const double& rcd2 = 2;             // rcd2 refers to temporary with type const double and value 2.0
double&& rrd = 2;                   // rrd refers to temporary with value 2.0
const volatile int cvi = 1;
const int& r2 = cvi;                // error: cv-qualifier dropped
struct A { operator volatile int&(); } a;
const int& r3 = a;                  // error: cv-qualifier dropped

```

```

double d2 = 1.0;
double&& rrd2 = d2;
struct X { operator int&(); };
int&& rri2 = X();

int i3 = 2;
double&& rrd3 = i3;
— end example]

```

// from result of conversion function
// error: initializer is lvalue of reference-related type
// error: result of conversion function is
// lvalue of reference-related type
// rrd3 refers to temporary with value 2.0

In all cases except the last (i.e., implicitly converting the initializer expression to the referenced type), the reference is said to *bind directly* to the initializer expression.

⁶ [Note 3: 6.8.7 describes the lifetime of temporaries bound to references. — end note]

9.5.5 List-initialization

[dcl.init.list]

¹ *List-initialization* is initialization of an object or reference from a *braced-init-list*. Such an initializer is called an *initializer list*, and the comma-separated *initializer-clauses* of the *initializer-list* or *designated-initializer-clauses* of the *designated-initializer-list* are called the *elements* of the initializer list. An initializer list may be empty. List-initialization can occur in direct-initialization or copy-initialization contexts; list-initialization in a direct-initialization context is called *direct-list-initialization* and list-initialization in a copy-initialization context is called *copy-list-initialization*. Direct-initialization that is not list-initialization is called *direct-non-list-initialization*.

[Note 1: List-initialization can be used

- (1.1) — as the initializer in a variable definition (9.5),
- (1.2) — as the initializer in a *new-expression* (7.6.2.8),
- (1.3) — in a **return** statement (8.8.4),
- (1.4) — as a *for-range-initializer* (8.6),
- (1.5) — as a function argument (7.6.1.3),
- (1.6) — as a template argument (13.4.3),
- (1.7) — as a subscript (7.6.1.2),
- (1.8) — as an argument to a constructor invocation (9.5, 7.6.1.4),
- (1.9) — as an initializer for a non-static data member (11.4),
- (1.10) — in a *mem-initializer* (11.9.3), or
- (1.11) — on the right-hand side of an assignment (7.6.19).

[Example 1:

```

int a = {1};
std::complex<double> z{1,2};
new std::vector<std::string>{"once", "upon", "a", "time"}; // 4 string elements
f( {"Nicholas","Annemarie"} ); // pass list of two elements
return { "Norah" }; // return list of one element
int* e {}; // initialization to zero / null pointer
x = double{1}; // explicitly construct a double
std::map<std::string,int> anim = { {"bear",4}, {"cassowary",2}, {"tiger",7} };

```

— end example]

— end note]

² A constructor is an *initializer-list constructor* if its first parameter is of type `std::initializer_list<E>` or reference to `cv std::initializer_list<E>` for some type `E`, and either there are no other parameters or else all other parameters have default arguments (9.3.4.7).

[Note 2: Initializer-list constructors are favored over other constructors in list-initialization (12.2.2.8). Passing an initializer list as the argument to the constructor template `template<class T> C(T)` of a class `C` does not create an initializer-list constructor, because an initializer list argument causes the corresponding parameter to be a non-deduced context (13.10.3.2). — end note]

The template `std::initializer_list` is not predefined; if a standard library declaration (17.11.2, 16.4.2.4) of `std::initializer_list` is not reachable from (10.7) a use of `std::initializer_list` — even an implicit use in which the type is not named (9.2.9.7) — the program is ill-formed.

³ List-initialization of an object or reference of type *cv* T is defined as follows:

- (3.1) — If the *braced-init-list* contains a *designated-initializer-list* and T is not a reference type, T shall be an aggregate class. The associated elements of the non-static data members designated by the ordered *identifiers* in the *designators* of the *designated-initializer-list* shall be in non-decreasing element order. Aggregate initialization is performed (9.5.2).

[Example 2:

```

struct A { int x; int y; int z; };
A a{.y = 2, .x = 1};           // error: designator order does not match declaration order
A b{.x = 1, .z = 2};           // OK, b.y initialized to 0

struct B : A { int q; };
B e{.x = 1, .q = 3};           // OK, e.y and e.z initialized to 0
B f{.q = 3, .x = 1};           // error: designator order does not match declaration order

struct C { int p; int x; };
struct D : A, C { };
D g{.y=1, .p=2};               // OK
D h{.x=2};                     // error: ambiguous lookup for x

struct NonAggr { int na; NonAggr(int); };
struct E : NonAggr { int e; };
E i{.na=1, .e=2};              // error: the lookup set for na finds NonAggr,
                                // which is not an aggregate base of E

```

— end example]

- (3.2) — If T is an aggregate class and the initializer list has a single element of type *cv1* U, where U is T or a class derived from T, the object is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization).
- (3.3) — Otherwise, if T is a character array and the initializer list has a single element that is an appropriately-typed *string-literal* (9.5.3), initialization is performed as described in that subclause.
- (3.4) — Otherwise, if T is an aggregate, aggregate initialization is performed (9.5.2).

[Example 3:

```

double ad[] = { 1, 2.0 };      // OK
int ai[] = { 1, 2.0 };         // error: narrowing

struct S2 {
    int m1;
    double m2, m3;
};
S2 s21 = { 1, 2, 3.0 };        // OK
S2 s22 { 1.0, 2, 3 };          // error: narrowing
S2 s23 { };                   // OK, default to 0,0,0

```

— end example]

- (3.5) — Otherwise, if the initializer list has no elements and T is a class type with a default constructor, the object is value-initialized.
- (3.6) — Otherwise, if T is a specialization of `std::initializer_list`, the object is constructed as described below.
- (3.7) — Otherwise, if T is a class type, constructors are considered. The applicable constructors are enumerated and the best one is chosen through overload resolution (12.2, 12.2.2.8). If a narrowing conversion (see below) is required to convert any of the arguments, the program is ill-formed.

[Example 4:

```

struct S {
    S(std::initializer_list<double>); // #1
    S(std::initializer_list<int>);    // #2
    S(std::initializer_list<S>);      // #3
    S();                             // #4

```

```

    // ...
};
S s1 = { 1.0, 2.0, 3.0 };           // invoke #1
S s2 = { 1, 2, 3 };                 // invoke #2
S s3{s2};                           // invoke #3 (not the copy constructor)
S s4 = { };                         // invoke #4
— end example]

```

[Example 5:

```

struct Map {
    Map(std::initializer_list<std::pair<std::string,int>>);
};
Map ship = {{"Sophie",14}, {"Surprise",28}};
— end example]

```

[Example 6:

```

struct S {
    // no initializer-list constructors
    S(int, double, double);           // #1
    S();                             // #2
    // ...
};
S s1 = { 1, 2, 3.0 };                // OK, invoke #1
S s2 { 1.0, 2, 3 };                  // error: narrowing
S s3 { };                           // OK, invoke #2
— end example]

```

- (3.8) — Otherwise, if T is an enumeration with a fixed underlying type (9.8.1) U, the *initializer-list* has a single element v of scalar type, v can be implicitly converted to U, and the initialization is direct-list-initialization, the object is initialized with the value T(v) (7.6.1.4); if a narrowing conversion is required to convert v to U, the program is ill-formed.

[Example 7:

```

enum byte : unsigned char { };
byte b { 42 };                       // OK
byte c = { 42 };                     // error
byte d = byte{ 42 };                 // OK; same value as b
byte e { -1 };                       // error

struct A { byte b; };
A a1 = { { 42 } };                   // error
A a2 = { byte{ 42 } };               // OK

void f(byte);
f({ 42 });                           // error

enum class Handle : uint32_t { Invalid = 0 };
Handle h { 42 };                     // OK
— end example]

```

- (3.9) — Otherwise, if the initializer list is not a *designated-initializer-list* and has a single element of type E and either T is not a reference type or its referenced type is reference-related to E, the object or reference is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization); if a narrowing conversion (see below) is required to convert the element to T, the program is ill-formed.

[Example 8:

```

int x1 {2};                          // OK
int x2 {2.0};                        // error: narrowing
— end example]

```

- (3.10) — Otherwise, if T is a reference type, a prvalue is generated. The prvalue initializes its result object by copy-list-initialization from the initializer list. The prvalue is then used to direct-initialize the reference.

The type of the prvalue is the type referenced by T, unless T is “reference to array of unknown bound of U”, in which case the type of the prvalue is the type of x in the declaration U x[] H, where H is the initializer list.

[Note 3: As usual, the binding will fail and the program is ill-formed if the reference type is an lvalue reference to a non-const type. — end note]

[Example 9:

```

struct S {
    S(std::initializer_list<double>); // #1
    S(const std::string&);           // #2
    // ...
};
const S& r1 = { 1, 2, 3.0 };        // OK, invoke #1
const S& r2 { "Spinach" };          // OK, invoke #2
S& r3 = { 1, 2, 3 };                // error: initializer is not an lvalue
const int& i1 = { 1 };              // OK
const int& i2 = { 1.1 };            // error: narrowing
const int (&iar)[2] = { 1, 2 };     // OK, iar is bound to temporary array

struct A { } a;
struct B { explicit B(const A&); };
const B& b2{a};                     // error: cannot copy-list-initialize B temporary from A

struct C { int x; };
C&& c = { .x = 1 };                 // OK

```

— end example]

- (3.11) — Otherwise, if the initializer list has no elements, the object is value-initialized.

[Example 10:

```

int** pp {};                        // initialized to null pointer

```

— end example]

- (3.12) — Otherwise, the program is ill-formed.

[Example 11:

```

struct A { int i; int j; };
A a1 { 1, 2 };                     // aggregate initialization
A a2 { 1.2 };                       // error: narrowing
struct B {
    B(std::initializer_list<int>);
};
B b1 { 1, 2 };                      // creates initializer_list<int> and calls constructor
B b2 { 1, 2.0 };                    // error: narrowing
struct C {
    C(int i, double j);
};
C c1 = { 1, 2.2 };                  // calls constructor with arguments (1, 2.2)
C c2 = { 1.1, 2 };                  // error: narrowing

int j { 1 };                        // initialize to 1
int k { };                          // initialize to 0

```

— end example]

- ⁴ Within the *initializer-list* of a *braced-init-list*, the *initializer-clauses*, including any that result from pack expansions (13.7.4), are evaluated in the order in which they appear. That is, every value computation and side effect associated with a given *initializer-clause* is sequenced before every value computation and side effect associated with any *initializer-clause* that follows it in the comma-separated list of the *initializer-list*. If the *initializer-clauses* of the *initializer-list* are interpreted as arguments of a constructor call (12.2.2.8), any default arguments used are evaluated in the order as if an *initializer-clause* were present for the corresponding parameter.

[*Note 4*: This evaluation ordering holds regardless of the semantics of the initialization; for example, it applies when the elements of the *initializer-list* are interpreted as arguments of a constructor call, even though ordinarily there are no sequencing constraints on the arguments of a call. — *end note*]

[*Example 12*:

```
#include <iostream>
struct I {
    I(int x) { std::cout << x; }
};
struct S {
    S(I, I = 2) {}
};
int main() {
    S(1, 2); std::cout << '\n';    // unspecified order; prints 12 or 21
    S{1, 2}; std::cout << '\n';    // prints 12
    S{1};                          // prints 12
}
```

— *end example*]

- ⁵ An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation generated and materialized (7.3.5) a prvalue of type “array of N `const E`”, where N is the number of elements in the initializer list; this is called the initializer list’s *backing array*. Each element of the backing array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array.

[*Note 5*: A constructor or conversion function selected for the copy needs to be accessible (11.8) in the context of the initializer list. — *end note*]

If a narrowing conversion is required to initialize any of the elements, the program is ill-formed.

[*Note 6*: Backing arrays are potentially non-unique objects (6.8.2). — *end note*]

- ⁶ The backing array has the same lifetime as any other temporary object (6.8.7), except that initializing an `initializer_list` object from the array extends the lifetime of the array exactly like binding a reference to a temporary.

[*Example 13*:

```
void f(std::initializer_list<double> il);
void g(float x) {
    f({1, x, 3});
}
void h() {
    f({1, 2, 3});
}

struct A {
    mutable int i;
};
void q(std::initializer_list<A>);
void r() {
    q({A{1}, A{2}, A{3}});
}
```

The initialization will be implemented in a way roughly equivalent to this:

```
void g(float x) {
    const double __a[3] = {double{1}, double{x}, double{3}};    // backing array
    f(std::initializer_list<double>(__a, __a+3));
}
void h() {
    static constexpr double __b[3] = {double{1}, double{2}, double{3}};    // backing array
    f(std::initializer_list<double>(__b, __b+3));
}
void r() {
    const A __c[3] = {A{1}, A{2}, A{3}};    // backing array
    q(std::initializer_list<A>(__c, __c+3));
}
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers, and with the understanding that `__b` does not outlive the call to `f`. — *end example*]

[*Example 14*:

```
typedef std::complex<double> cmplx;
std::vector<cmplx> v1 = { 1, 2, 3 };

void f() {
    std::vector<cmplx> v2{ 1, 2, 3 };
    std::initializer_list<int> i3 = { 1, 2, 3 };
}

struct A {
    std::initializer_list<int> i4;
    A() : i4{ 1, 2, 3 } {}           // ill-formed, would create a dangling reference
};
```

For `v1` and `v2`, the `initializer_list` object is a parameter in a function call, so the array created for `{ 1, 2, 3 }` has full-expression lifetime. For `i3`, the `initializer_list` object is a variable, so the array persists for the lifetime of the variable. For `i4`, the `initializer_list` object is initialized in the constructor's *ctor-initializer* as if by binding a temporary array to a reference member, so the program is ill-formed (11.9.3). — *end example*]

⁷ A *narrowing conversion* is an implicit conversion

- (7.1) — from a floating-point type to an integer type, or
- (7.2) — from a floating-point type `T` to another floating-point type whose floating-point conversion rank is neither greater than nor equal to that of `T`, except where the result of the conversion is a constant expression and either its value is finite and the conversion did not overflow, or the values before and after the conversion are not finite, or
- (7.3) — from an integer type or unscoped enumeration type to a floating-point type, except where the source is a constant expression and the actual value after conversion will fit into the target type and will produce the original value when converted back to the original type, or
- (7.4) — from an integer type or unscoped enumeration type to an integer type that cannot represent all the values of the original type, except where
 - (7.4.1) — the source is a bit-field whose width *w* is less than that of its type (or, for an enumeration type, its underlying type) and the target type can represent all the values of a hypothetical extended integer type with width *w* and with the same signedness as the original type or
 - (7.4.2) — the source is a constant expression whose value after integral promotions will fit into the target type, or
- (7.5) — from a pointer type or a pointer-to-member type to `bool`.

[*Note 7*: As indicated above, such conversions are not allowed at the top level in list-initializations. — *end note*]

[*Example 15*:

```
int x = 999;           // x is not a constant expression
const int y = 999;
const int z = 99;
char c1 = x;           // OK, though it potentially narrows (in this case, it does narrow)
char c2{x};            // error: potentially narrows
char c3{y};            // error: narrows (assuming char is 8 bits)
char c4{z};            // OK, no narrowing needed
unsigned char uc1 = {5}; // OK, no narrowing needed
unsigned char uc2 = {-1}; // error: narrows
unsigned int ui1 = {-1}; // error: narrows
signed int si1 =
    { (unsigned int)-1 }; // error: narrows
int ii = {2.0};         // error: narrows
float f1 { x };         // error: potentially narrows
float f2 { 7 };         // OK, 7 can be exactly represented as a float
bool b = {"meow"};      // error: narrows
int f(int);
int a[] = { 2, f(2), f(2.0) }; // OK, the double-to-int conversion is not at the top level
```

— end example]

9.6 Function definitions

[dcl.fct.def]

9.6.1 General

[dcl.fct.def.general]

- ¹ Function definitions have the form

```
function-definition:
    attribute-specifier-seqopt decl-specifier-seqopt declarator virt-specifier-seqopt
    function-contract-specifier-seqopt function-body
    attribute-specifier-seqopt decl-specifier-seqopt declarator requires-clause
    function-contract-specifier-seqopt function-body

function-body:
    ctor-initializeropt compound-statement
    function-try-block
    = default ;
    deleted-function-body

deleted-function-body:
    = delete ;
    = delete ( unevaluated-string ) ;
```

Any informal reference to the body of a function should be interpreted as a reference to the non-terminal *function-body*, including, for a constructor, default member initializers or default initialization used to initialize a base or member subobject in the absence of a *mem-initializer-id* (11.9.3). The optional *attribute-specifier-seq* in a *function-definition* appertains to the function. A *function-definition* with a *virt-specifier-seq* shall be a *member-declaration* (11.4). A *function-definition* with a *requires-clause* shall define a templated function.

- ² In a *function-definition*, either *void declarator ;* or *declarator ;* shall be a well-formed function declaration as described in 9.3.4.6. A function shall be defined only in namespace or class scope. The type of a parameter or the return type for a function definition shall not be a (possibly cv-qualified) class type that is incomplete or abstract within the function body unless the function is deleted (9.6.3).
- ³ [Example 1: A simple example of a complete function definition is

```
int max(int a, int b, int c) {
    int m = (a > b) ? a : b;
    return (m > c) ? m : c;
}
```

Here *int* is the *decl-specifier-seq*; *max(int a, int b, int c)* is the *declarator*; { /* ... */ } is the *function-body*. — end example]

- ⁴ A *ctor-initializer* is used only in a constructor; see 11.4.5 and 11.9.
- ⁵ [Note 1: A *cv-qualifier-seq* affects the type of *this* in the body of a member function; see 7.5.3. — end note]
- ⁶ [Note 2: Unused parameters need not be named. For example,

```
void print(int a, int) {
    std::printf("a = %d\n",a);
}
```

— end note]

- ⁷ A *function-local predefined variable* is a variable with static storage duration that is implicitly defined in a function parameter scope.
- ⁸ The function-local predefined variable `__func__` is defined as if a definition of the form

```
static const char __func__[ ] = "function-name";
```

had been provided, where *function-name* is an implementation-defined string. It is unspecified whether such a variable has an address distinct from that of any other object in the program.⁷⁴

[Example 2:

```
struct S {
    S() : s(__func__) { }           // OK
    const char* s;
};
```

⁷⁴ Implementations are permitted to provide additional predefined variables with names that are reserved to the implementation (5.11). If a predefined variable is not odr-used (6.3), its string value need not be present in the program image.

```
void f(const char* s = __func__); // error: __func__ is undeclared
— end example]
```

9.6.2 Explicitly-defaulted functions

[dcl.fct.def.default]

- ¹ A function definition whose *function-body* is of the form `= default` ; is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

- (1.1) — be a special member function (11.4.4), comparison operator function (12.4.3, 11.10.1), or postfix increment or decrement operator function (12.4.7.2), and
- (1.2) — not have default arguments (9.3.4.7).

- ² The program is ill-formed if an explicitly defaulted special member function F_1 of class C differs from the corresponding special member function F_2 that would have been implicitly declared other than as follows:

- (2.1) — if F_1 is an assignment operator, it may have the *& ref-qualifier*;
- (2.2) — if F_2 is an assignment operator (which has an implicit object parameter of type “lvalue reference to C ”), F_1 may have an explicit object parameter of type “lvalue reference to C ”, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
- (2.3) — F_1 and F_2 may have differing exception specifications;
- (2.4) — if F_2 has a non-object parameter of type “lvalue reference to `const C`”, the corresponding non-object parameter of F_1 may be of type “lvalue reference to C ”; and
- (2.5) — if F_2 has a non-object parameter of type “lvalue reference to C ”, the corresponding non-object parameter of F_1 may be of type “lvalue reference to `const C`”; in this case only, F_1 is defined as deleted if it is explicitly defaulted on its first declaration and the program is ill-formed otherwise.

- ³ A function explicitly defaulted on its first declaration is implicitly inline (9.2.8), and is implicitly constexpr (9.2.6) if it is constexpr-suitable.

[Note 1: Other defaulted functions are not implicitly constexpr. — end note]

- ⁴ [Example 1:

```
struct S {
    S(int a = 0) = default;           // error: default argument
    void operator=(const S&) = default; // error: non-matching return type
    ~S() noexcept(false) = default;   // OK, despite mismatched exception specification
private:
    int i;
    S(S&);                           // OK, private copy constructor
};
S::S(S&) = default;                  // OK, defines copy constructor

struct T {
    T();
    T(T &&) noexcept(false);
};
struct U {
    T t;
    U();
    U(U &&) noexcept = default;
};
U u1;
U u2 = static_cast<U&&>(u1);          // OK, calls std::terminate if T::T(T&&) throws
```

— end example]

- ⁵ Explicitly-defaulted functions and implicitly-declared functions are collectively called *defaulted* functions, and the implementation shall provide implicit definitions for them (11.4.5, 11.4.7, 11.4.5.3, 11.4.6) as described below, including possibly defining them as deleted. A defaulted prospective destructor (11.4.7) that is not a destructor is defined as deleted. A defaulted special member function that is neither a prospective destructor nor an eligible special member function (11.4.4) is defined as deleted. A function is *user-provided* if it is user-declared and not explicitly defaulted or deleted on its first declaration. A user-provided explicitly-defaulted function (i.e., explicitly defaulted after its first declaration) is implicitly defined at the point where it is explicitly defaulted; if such a function is implicitly defined as deleted, the program is ill-formed.

[*Note 2*: Declaring a function as defaulted after its first declaration can provide efficient execution and concise definition while enabling a stable binary interface to an evolving code base. — *end note*]

A non-user-provided defaulted function (i.e., implicitly declared or explicitly defaulted in the class) that is not defined as deleted is implicitly defined when it is odr-used (6.3) or needed for constant evaluation (7.7.7).

[*Note 3*: The implicit definition of a non-user-provided defaulted function does not bind any names. — *end note*]

⁶ [*Example 2*:

```
struct trivial {
    trivial() = default;
    trivial(const trivial&) = default;
    trivial(trivial&&) = default;
    trivial& operator=(const trivial&) = default;
    trivial& operator=(trivial&&) = default;
    ~trivial() = default;
};

struct nontrivial1 {
    nontrivial1();
};
nontrivial1::nontrivial1() = default;    // not first declaration
```

— *end example*]

9.6.3 Deleted definitions

[dcl.fct.def.delete]

¹ A *deleted definition* of a function is a function definition whose *function-body* is a *deleted-function-body* or an explicitly-defaulted definition of the function where the function is defined as deleted. A *deleted function* is a function with a deleted definition or a function that is implicitly defined as deleted.

² A construct that designates a deleted function implicitly or explicitly, other than to declare it or to appear as the operand of a *reflect-expression* (7.6.2.10), is ill-formed.

Recommended practice: The resulting diagnostic message should include the text of the *unevaluated-string*, if one is supplied.

[*Note 1*: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially evaluated. For an overload set, only the function selected by overload resolution is referenced. The implicit odr-use (6.3) of a virtual function does not, by itself, constitute a reference. The *unevaluated-string*, if present, can be used to explain the rationale for deletion and/or to suggest an alternative. — *end note*]

³ [*Example 1*: One can prevent default initialization and initialization by non-doubles with

```
struct onlydouble {
    onlydouble() = delete;                // OK, but redundant
    template<class T>
        onlydouble(T) = delete;
    onlydouble(double);
};
```

— *end example*]

[*Example 2*: One can prevent use of a class in certain *new-expressions* by using deleted definitions of a user-declared `operator new` for that class.

```
struct sometype {
    void* operator new(std::size_t) = delete;
    void* operator new[](std::size_t) = delete;
};
sometype* p = new sometype;    // error: deleted class operator new
sometype* q = new sometype[3]; // error: deleted class operator new[]
```

— *end example*]

[*Example 3*: One can make a class uncopyable, i.e., move-only, by using deleted definitions of the copy constructor and copy assignment operator, and then providing defaulted definitions of the move constructor and move assignment operator.

```
struct moveonly {
    moveonly() = default;
```

```

    moveonly(const moveonly&) = delete;
    moveonly(moveonly&&) = default;
    moveonly& operator=(const moveonly&) = delete;
    moveonly& operator=(moveonly&&) = default;
    ~moveonly() = default;
};
moveonly* p;
moveonly q(*p);           // error: deleted copy constructor
— end example]

```

- 4 A deleted function is implicitly an inline function (9.2.8).

[Note 2: The one-definition rule (6.3) applies to deleted definitions. — end note]

A deleted definition of a function shall be the first declaration of the function or, for an explicit specialization of a function template, the first declaration of that specialization. An implicitly declared allocation or deallocation function (6.8.6.5) shall not be defined as deleted.

[Example 4:

```

    struct sometype {
        sometype();
    };
    sometype::sometype() = delete; // error: not first declaration
— end example]

```

9.6.4 Coroutine definitions

[dcl.fct.def.coroutine]

- ¹ A function is a *coroutine* if its *function-body* encloses a *coroutine-return-statement* (8.8.5), an *await-expression* (7.6.2.4), or a *yield-expression* (7.6.17). The *parameter-declaration-clause* of the coroutine shall not terminate with an ellipsis that is not part of a *parameter-declaration*.

- ² [Example 1:

```

    task<int> f();

    task<void> g1() {
        int i = co_await f();
        std::cout << "f() => " << i << std::endl;
    }

    template <typename... Args>
    task<void> g2(Args&&...) { // OK, ellipsis is a pack expansion
        int i = co_await f();
        std::cout << "f() => " << i << std::endl;
    }

    task<void> g3(int a, ...) { // error: variable parameter list not allowed
        int i = co_await f();
        std::cout << "f() => " << i << std::endl;
    }

```

— end example]

- ³ The *promise type* of a coroutine is `std::coroutine_traits<R, P1, ..., Pn>::promise_type`, where R is the return type of the function, and P₁...P_n is the sequence of types of the non-object function parameters, preceded by the type of the object parameter (9.3.4.6) if the coroutine is a non-static member function. The promise type shall be a class type.
- ⁴ In the following, p_i is an lvalue of type P_i, where p₁ denotes the object parameter and p_{i+1} denotes the ith non-object function parameter for an implicit object member function, and p_i denotes the ith function parameter otherwise. For an implicit object member function, q₁ is an lvalue that denotes `*this`; any other q_i is an lvalue that denotes the parameter copy corresponding to p_i, as described below.
- ⁵ A coroutine behaves as if the top-level cv-qualifiers in all *parameter-declarations* in the declarator of its *function-definition* were removed and its *function-body* were replaced by the following *replacement body*:


```

{
    promise-type promise promise-constructor-arguments ;
    try {
        co_await promise.initial_suspend() ;
        function-body
    } catch ( ... ) {
        if (!initial-await-resume-called)
            throw ;
        promise.unhandled_exception() ;
    }
    final-suspend :
        co_await promise.final_suspend() ;
}

```

where

- (5.1) — the *await-expression* containing the call to `initial_suspend` is the *initial await expression*, and
 - (5.2) — the *await-expression* containing the call to `final_suspend` is the *final await expression*, and
 - (5.3) — *initial-await-resume-called* is initially **false** and is set to **true** immediately before the evaluation of the *await-resume* expression (7.6.2.4) of the initial *await expression*, and
 - (5.4) — *promise-type* denotes the promise type, and
 - (5.5) — the object denoted by the exposition-only name *promise* is the *promise object* of the coroutine, and
 - (5.6) — the label denoted by the name *final-suspend* is defined for exposition only (8.8.5), and
 - (5.7) — *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list $q_1 \dots q_n$. If a viable constructor is found (12.2.3), then *promise-constructor-arguments* is $(q_1, \dots, q_n)$, otherwise *promise-constructor-arguments* is empty, and
 - (5.8) — a coroutine is suspended at the *initial suspend point* if it is suspended at the initial *await expression*, and
 - (5.9) — a coroutine is suspended at a *final suspend point* if it is suspended
 - (5.9.1) — at a final *await expression* or
 - (5.9.2) — due to an exception exiting from `unhandled_exception()`.
- ⁶ [Note 1: An odr-use of a non-reference parameter in a postcondition assertion of a coroutine is ill-formed (9.4.1). — end note]
- ⁷ The expression `promise.get_return_object()` is used to initialize the returned reference or prvalue result object of a call to a coroutine. The call to `get_return_object` is sequenced before the call to `initial_suspend` and is invoked at most once.
- ⁸ A suspended coroutine can be resumed to continue execution by invoking a resumption member function (17.13.4.6) of a coroutine handle (17.13.4) that refers to the coroutine. The evaluation that invoked a resumption member function is called the *resumer*. Invoking a resumption member function for a coroutine that is not suspended results in undefined behavior (F.5.6).
- ⁹ An implementation may need to allocate additional storage for a coroutine. This storage is known as the *coroutine state* and is obtained by calling a non-array allocation function (6.8.6.5.2) as part of the replacement body. The allocation function's name is looked up by searching for it in the scope of the promise type.
- (9.1) — If the search finds any declarations, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and is a prvalue of type `std::size_t`. The lvalues $p_1 \dots p_n$ with their original types (including cv-qualifiers) are the successive arguments. If no viable function is found (12.2.3), overload resolution is performed again on a function call created by passing just the amount of space required as a prvalue of type `std::size_t`.
 - (9.2) — If the search finds no declarations, a search is performed in the global scope. Overload resolution is performed on a function call created by passing the amount of space required as a prvalue of type `std::size_t`.
- ¹⁰ If a search for the name `get_return_object_on_allocation_failure` in the scope of the promise type (6.5.2) finds any declarations, then the result of a call to an allocation function used to obtain storage for the coroutine state is assumed to return `nullptr` if it fails to obtain storage, and if a global allocation function

is selected, the `::operator new(size_t, nothrow_t)` form is used. The allocation function used in this case shall have a non-throwing *noexcept-specifier*. If the allocation function returns `nullptr`, the coroutine transfers control to the caller of the coroutine and the return value is obtained by a call to `T::get_return_object_on_allocation_failure()`, where `T` is the promise type.

[Example 2:

```
#include <iostream>
#include <coroutine>

// ::operator new(size_t, nothrow_t) will be used if allocation is needed
struct generator {
    struct promise_type;
    using handle = std::coroutine_handle<promise_type>;
    struct promise_type {
        int current_value;
        static auto get_return_object_on_allocation_failure() { return generator{nullptr}; }
        auto get_return_object() { return generator{handle::from_promise(*this)}; }
        auto initial_suspend() { return std::suspend_always{}; }
        auto final_suspend() noexcept { return std::suspend_always{}; }
        void unhandled_exception() { std::terminate(); }
        void return_void() {}
        auto yield_value(int value) {
            current_value = value;
            return std::suspend_always{};
        }
    };
    bool move_next() { return coro ? (coro.resume(), !coro.done()) : false; }
    int current_value() { return coro.promise().current_value; }
    generator(generator const&) = delete;
    generator(generator && rhs) : coro(rhs.coro) { rhs.coro = nullptr; }
    ~generator() { if (coro) coro.destroy(); }
private:
    generator(handle h) : coro(h) {}
    handle coro;
};

generator f() { co_yield 1; co_yield 2; }
int main() {
    auto g = f();
    while (g.move_next()) std::cout << g.current_value() << std::endl;
}
```

— end example]

- 11 The coroutine state is destroyed when control flows off the end of the coroutine or the `destroy` member function (17.13.4.6) of a coroutine handle (17.13.4) that refers to the coroutine is invoked. In the latter case, control in the coroutine is considered to be transferred out of the function (8.10). The storage for the coroutine state is released by calling a non-array deallocation function (6.8.6.5.3). If `destroy` is called for a coroutine that is not suspended, the program has undefined behavior (F.5.7).
- 12 The deallocation function's name is looked up by searching for it in the scope of the promise type. If nothing is found, a search is performed in the global scope. If both a usual deallocation function with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter are found, then the selected deallocation function shall be the one with two parameters. Otherwise, the selected deallocation function shall be the function with one parameter. If no usual deallocation function is found, the program is ill-formed. The selected deallocation function shall be called with the address of the block of storage to be reclaimed as its first argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the block is passed as the corresponding argument.
- 13 When a coroutine is invoked, a copy is created for each coroutine parameter at the beginning of the replacement body. For a parameter whose original declaration specified the type *cv T*,
 - (13.1) — if *T* is a reference type, the copy is a reference of type *cv T* bound to the same object as a parameter;
 - (13.2) — otherwise, the copy is a variable of type *cv T* with automatic storage duration that is direct-initialized from an xvalue of type *T* referring to the parameter.

[*Note 2*: An identifier in the *function-body* that names one of these parameters refers to the created copy, not the original parameter (7.5.5.2). — *end note*]

The initialization and destruction of each parameter copy occurs in the context of the called coroutine. Initializations of parameter copies are sequenced before the call to the coroutine promise constructor and indeterminately sequenced with respect to each other. The lifetime of parameter copies ends immediately after the lifetime of the coroutine promise object ends.

[*Note 3*: If a coroutine has a parameter passed by reference, resuming the coroutine after the lifetime of the entity referred to by that parameter has ended is likely to result in undefined behavior. — *end note*]

- 14 If the evaluation of the expression `promise.unhandled_exception()` exits via an exception, the coroutine is considered suspended at the final suspend point and the exception propagates to the caller or resumer.
- 15 The expression `co_await promise.final_suspend()` shall not be potentially-throwing (14.5).

9.6.5 Replaceable function definitions

[**dcl.fct.def.replace**]

- 1 Certain functions for which a definition is supplied by the implementation are *replaceable*. A C++ program may provide a definition with the signature of a replaceable function, called a *replacement function*. The replacement function is used instead of the default version supplied by the implementation. Such replacement occurs prior to program startup (6.3, 6.10.3). A declaration of the replacement function
 - (1.1) — shall not be inline,
 - (1.2) — shall be attached to the global module,
 - (1.3) — shall have C++ language linkage,
 - (1.4) — shall have the same return type as the replaceable function, and
 - (1.5) — if the function is declared in a standard library header, shall be such that it would be valid as a redeclaration of the declaration in that header;

no diagnostic is required (G.6.5).

[*Note 1*: The one-definition rule (6.3) applies to the definitions of a replaceable function provided by the program. The implementation-supplied function definition is an otherwise-unnamed function with no linkage. — *end note*]

9.7 Structured binding declarations

[**dcl.struct.bind**]

- 1 A structured binding declaration introduces the *identifiers* $v_0, v_1, v_2, \dots, v_{N-1}$ of the *sb-identifier-list* as names. An *sb-identifier* that contains an ellipsis introduces a structured binding pack (13.7.4). A *structured binding* is either an *sb-identifier* that does not contain an ellipsis or an element of a structured binding pack. The optional *attribute-specifier-seq* of an *sb-identifier* appertains to the associated structured bindings. Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq* and *S* consist of each *decl-specifier* of the *decl-specifier-seq* that is *constexpr*, *constinit*, or a *storage-class-specifier*. A *cv* that includes *volatile* is deprecated; see D.4. First, a variable with a unique name *e* is introduced. If the *assignment-expression* in the *initializer* has array type *cv1 A* and no *ref-qualifier* is present, *e* is defined by

attribute-specifier-seq_{opt} *S* *cv* *A* *e* ;

and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the *initializer*. Otherwise, *e* is defined as-if by

attribute-specifier-seq_{opt} *decl-specifier-seq* *ref-qualifier_{opt}* *e* *initializer* ;

where the declaration is never interpreted as a function declaration and the parts of the declaration other than the *declarator-id* are taken from the corresponding structured binding declaration. The type of the *id-expression* *e* is called *E*.

[*Note 1*: *E* is never a reference type (7.2). — *end note*]

- 2 The *structured binding size* of *E*, as defined below, is the number of structured bindings that need to be introduced by the structured binding declaration. If there is no structured binding pack, then the number of elements in the *sb-identifier-list* shall be equal to the structured binding size of *E*. Otherwise, the number of non-pack elements shall be no more than the structured binding size of *E*; the number of elements of the structured binding pack is the structured binding size of *E* less the number of non-pack elements in the *sb-identifier-list*.
- 3 Let *SB_i* denote the *i*th structured binding in the structured binding declaration after expanding the structured binding pack, if any.

[Note 2: If there is no structured binding pack, then SB_i denotes v_i . — end note]

[Example 1:

```
struct C { int x, y, z; };

template<class T>
void now_i_know_my() {
    auto [a, b, c] = C();           // OK,  $SB_0$  is a,  $SB_1$  is b, and  $SB_2$  is c
    auto [d, ...e] = C();           // OK,  $SB_0$  is d, the pack e ( $v_1$ ) contains two structured bindings:  $SB_1$  and  $SB_2$ 
    auto [...f, g] = C();           // OK, the pack f ( $v_0$ ) contains two structured bindings:  $SB_0$  and  $SB_1$ , and  $SB_2$ 
    is g
    auto [h, i, j, ...k] = C();      // OK, the pack k is empty
    auto [l, m, n, o, ...p] = C();  // error: structured binding size is too small
}
```

— end example]

- 4 If a structured binding declaration appears as a *condition*, the decision variable (8.1) of the condition is *e*.
- 5 If the *initializer* refers to one of the names introduced by the structured binding declaration, the program is ill-formed.
- 6 E shall not be an array type of unknown bound. If E is any other array type with element type T, the structured binding size of E is equal to the number of elements of E. Each SB_i is the name of an lvalue that refers to the element *i* of the array and whose type is T; the referenced type is T.

[Note 3: The top-level cv-qualifiers of T are *cv*. — end note]

[Example 2:

```
auto f() -> int(&)[2];
auto [ x, y ] = f();           // x and y refer to elements in a copy of the array return value
auto& [ xr, yr ] = f();        // xr and yr refer to elements in the array referred to by f's return value

auto g() -> int(&)[4];

template<size_t N>
void h(int (&arr)[N]) {
    auto [a, ...b, c] = arr;    // a names the first element of the array, b is a pack referring to the second and
                                // third elements, and c names the fourth element
    auto& [...e] = arr;         // e is a pack referring to the four elements of the array
}

void call_h() {
    h(g());
}
```

— end example]

- 7 Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named *value*, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression whose value is non-negative; the structured binding size of E is equal to that value. Let *i* be an index prvalue of type `std::size_t` corresponding to SB_i . If a search for the name *get* in the scope of E (6.5.2) finds at least one declaration that is a function template whose first template parameter is a constant template parameter, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)`, where *get* undergoes argument-dependent lookup (6.5.4). In either case, `get<i>` is interpreted as a *template-id*.

[Note 4: Ordinary unqualified lookup (6.5.3) is not performed. — end note]

In either case, *e* is an lvalue if the type of the entity *e* is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<i, E>::type` and the type U_i defined as T_i if the initializer is a prvalue, as “lvalue reference to T_i ” if the initializer is an lvalue, or as “rvalue reference to T_i ” otherwise, variables are introduced with unique names r_i as follows:

$S \ U_i \ r_i = \text{initializer};$

Each SB_i is the name of an lvalue of type T_i that refers to the object bound to r_i ; the referenced type is T_i . The initialization of *e* and any conversion of *e* considered as a decision variable (8.1) is sequenced before the initialization of any r_i . The initialization of each r_i is sequenced before the initialization of any r_j where $i < j$.

- ⁸ Otherwise, all of *E*'s non-static data members shall be direct members of *E* or of the same base class of *E*, well-formed when named as *e.name* in the context of the structured binding, *E* shall not have an anonymous union member, and the structured binding size of *E* is equal to the number of non-static data members of *E*. Designating the non-static data members of *E* as *m*₀, *m*₁, *m*₂, ... (in declaration order), each *SB*_{*i*} is the name of an lvalue that refers to the member *m*_{*i*} of *e* and whose type is that of *e.m*_{*i*} (7.6.1.5); the referenced type is the declared type of *m*_{*i*} if that type is a reference type, or the type of *e.m*_{*i*} otherwise. The lvalue is a bit-field if that member is a bit-field.

[Example 3:

```
struct S { mutable int x1 : 2; volatile double y1; };
S f();
const auto [ x, y ] = f();
```

The type of the *id-expression* *x* is “int”, the type of the *id-expression* *y* is “const volatile double”. — end example]

9.8 Enumerations

[enum]

9.8.1 Enumeration declarations

[dcl.enum]

- ¹ An enumeration is a distinct type (6.9.5) with named constants. Its name becomes an *enum-name* within its scope.

```
enum-name:
    identifier

enum-specifier:
    enum-head { enumerator-listopt }
    enum-head { enumerator-list , }

enum-head:
    enum-key attribute-specifier-seqopt enum-head-nameopt enum-baseopt

enum-head-name:
    nested-name-specifieropt identifier

opaque-enum-declaration:
    enum-key attribute-specifier-seqopt enum-head-name enum-baseopt ;

enum-key:
    enum
    enum class
    enum struct

enum-base:
    : type-specifier-seq

enumerator-list:
    enumerator-definition
    enumerator-list , enumerator-definition

enumerator-definition:
    enumerator
    enumerator = constant-expression

enumerator:
    identifier attribute-specifier-seqopt
```

The optional *attribute-specifier-seq* in the *enum-head* and the *opaque-enum-declaration* appertains to the enumeration; the attributes in that *attribute-specifier-seq* are thereafter considered attributes of the enumeration whenever it is named. A : following “enum *nested-name-specifier*_{opt} *identifier*” within the *decl-specifier-seq* of a *member-declaration* is parsed as part of an *enum-base*.

[Note 1: This resolves a potential ambiguity between the declaration of an enumeration with an *enum-base* and the declaration of an unnamed bit-field of enumeration type.

[Example 1:

```
struct S {
    enum E : int {};
    enum E : int {};           // error: redeclaration of enumeration
};
```

— end example]

— end note]

The *identifier* in an *enum-head-name* is not looked up and is introduced by the *enum-specifier* or *opaque-enum-declaration*. If the *enum-head-name* of an *opaque-enum-declaration* contains a *nested-name-specifier*, the declaration shall be an explicit specialization (13.9.4).

- 2 The enumeration type declared with an *enum-key* of only **enum** is an *unscoped enumeration*, and its *enumerators* are *unscoped enumerators*. The *enum-keys* **enum class** and **enum struct** are semantically equivalent; an enumeration type declared with one of these is a *scoped enumeration*, and its *enumerators* are *scoped enumerators*. The optional *enum-head-name* shall not be omitted in the declaration of a *scoped enumeration*. The *type-specifier-seq* of an *enum-base* shall name an integral type; any cv-qualification is ignored. An *opaque-enum-declaration* declaring an unscoped enumeration shall not omit the *enum-base*. The identifiers in an *enumerator-list* are declared as constants, and can appear wherever constants are required. The same identifier shall not appear as the name of multiple enumerators in an *enumerator-list*. An *enumerator-definition* with **=** gives the associated *enumerator* the value indicated by the *constant-expression*. An *enumerator-definition* without **=** gives the associated *enumerator* the value zero if it is the first *enumerator-definition*, and the value of the previous *enumerator* increased by one otherwise.

[Example 2:

```
enum { a, b, c=0 };
enum { d, e, f=e+2 };
```

defines **a**, **c**, and **d** to be zero, **b** and **e** to be 1, and **f** to be 3. — end example]

The optional *attribute-specifier-seq* in an *enumerator* appertains to that enumerator.

- 3 An *opaque-enum-declaration* is either a redeclaration of an enumeration in the current scope or a declaration of a new enumeration.

[Note 2: An enumeration declared by an *opaque-enum-declaration* has a fixed underlying type and is a complete type. The list of enumerators can be provided in a later redeclaration with an *enum-specifier*. — end note]

A *scoped enumeration* shall not be later redeclared as *unscoped* or with a different underlying type. An *unscoped enumeration* shall not be later redeclared as *scoped* and each redeclaration shall include an *enum-base* specifying the same underlying type as in the original declaration.

- 4 If an *enum-head-name* contains a *nested-name-specifier*, the enclosing *enum-specifier* or *opaque-enum-declaration* *D* shall not inhabit a class scope and shall correspond to one or more declarations nominable in the class, class template, or namespace to which the *nested-name-specifier* refers (6.4.1). All those declarations shall have the same target scope; the target scope of *D* is that scope.
- 5 Each enumeration defines a type that is different from all other types. Each enumeration also has an *underlying type*. The underlying type can be explicitly specified using an *enum-base*. For a *scoped enumeration* type, the underlying type is **int** if it is not explicitly specified. In both of these cases, the underlying type is said to be *fixed*. Following the closing brace of an *enum-specifier*, each enumerator has the type of its enumeration. If the underlying type is fixed, the type of each enumerator prior to the closing brace is the underlying type and the *constant-expression* in the *enumerator-definition* shall be a converted constant expression of the underlying type (7.7.3). If the underlying type is not fixed, the type of each enumerator prior to the closing brace is determined as follows:
 - (5.1) — If an initializer is specified for an enumerator, the *constant-expression* shall be an integral constant expression (7.7.3). If the expression has unscoped enumeration type, the enumerator has the underlying type of that enumeration type, otherwise it has the same type as the expression.
 - (5.2) — If no initializer is specified for the first enumerator, its type is an unspecified signed integral type.
 - (5.3) — Otherwise the type of the enumerator is the same as that of the preceding enumerator unless the incremented value is not representable in that type, in which case the type is an unspecified integral type sufficient to contain the incremented value. If no such type exists, the program is ill-formed.
- 6 An enumeration whose underlying type is fixed is an incomplete type until immediately after its *enum-base* (if any), at which point it becomes a complete type. An enumeration whose underlying type is not fixed is an incomplete type until the closing **}** of its *enum-specifier*, at which point it becomes a complete type.
- 7 For an enumeration whose underlying type is not fixed, the underlying type is an integral type that can represent all the enumerator values defined in the enumeration. If no integral type can represent all the enumerator values, the enumeration is ill-formed. It is implementation-defined which integral type is used as the underlying type except that the underlying type shall not be larger than **int** unless the value of an

enumerator cannot fit in an `int` or `unsigned int`. If the *enumerator-list* is empty, the underlying type is as if the enumeration had a single enumerator with value 0.

- 8 For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, the values of the enumeration are the values representable by a hypothetical integer type with minimal width M such that all enumerators can be represented. The width of the smallest bit-field large enough to hold all the values of the enumeration type is M . It is possible to define an enumeration that has values not defined by any of its enumerators. If the *enumerator-list* is empty, the values of the enumeration are as if the enumeration had a single enumerator with value 0.⁷⁵
- 9 An enumeration has the same size, value representation, and alignment requirements (6.8.3) as its underlying type. Furthermore, each value of an enumeration has the same representation as the corresponding value of the underlying type.
- 10 Two enumeration types are *layout-compatible enumerations* if they have the same underlying type and the same values.
- 11 The value of an enumerator or an object of an unscoped enumeration type is converted to an integer by integral promotion (7.3.7).

[Example 3:

```
enum color { red, yellow, green=20, blue };
color col = red;
color* cp = &col;
if (*cp == blue)           // ...
```

makes `color` a type describing various colors, and then declares `col` as an object of that type, and `cp` as a pointer to an object of that type. The possible values of an object of type `color` are `red`, `yellow`, `green`, `blue`; these values can be converted to the integral values 0, 1, 20, and 21. Since enumerations are distinct types, objects of type `color` can be assigned only values of type `color`.

```
color c = 1;                // error: type mismatch, no conversion from int to color
int i = yellow;             // OK, yellow converted to integral value 1, integral promotion
```

Note that this implicit `enum` to `int` conversion is not provided for a scoped enumeration:

```
enum class Col { red, yellow, green };
int x = Col::red;           // error: no Col to int conversion
Col y = Col::red;
if (y) { }                 // error: no Col to bool conversion
```

— end example]

- 12 The name of each unscoped enumerator is also bound in the scope that immediately contains the *enum-specifier*. An unnamed enumeration that does not have a typedef name for linkage purposes (9.2.4) and that has a first enumerator is denoted, for linkage purposes (6.7), by its underlying type and its first enumerator; such an enumeration is said to have an enumerator as a name for linkage purposes.

[Note 3: Each unnamed enumeration with no enumerators is a distinct type. — end note]

[Example 4:

```
enum direction { left='l', right='r' };

void g() {
    direction d;           // OK
    d = left;              // OK
    d = direction::right;  // OK
}

enum class altitude { high='h', low='l' };

void h() {
    altitude a;            // OK
    a = high;              // error: high not in scope
    a = altitude::low;     // OK
}
```

⁷⁵) This set of values is used to define promotion and conversion semantics for the enumeration type. It does not preclude an expression of enumeration type from having a value that falls outside this range.

— end example]

9.8.2 The using enum declaration

[enum.udecl]

using-enum-declaration:

using enum *using-enum-declarator* ;

using-enum-declarator:

*nested-name-specifier*_{opt} *identifier*

*nested-name-specifier*_{opt} *simple-template-id*

splice-type-specifier

- ¹ A *using-enum-declarator* of the form *splice-type-specifier* designates the same type designated by the *splice-type-specifier*. Any other *using-enum-declarator* names the set of declarations found by type-only lookup (6.5.1) for the *using-enum-declarator* (6.5.3, 6.5.5). The *using-enum-declarator* shall designate a non-dependent type with a reachable *enum-specifier*.

[Example 1:

```
enum E { x };
void f() {
    int E;
    using enum E;           // OK
}
using F = E;
using enum F;              // OK
template<class T> using EE = T;
void g() {
    using enum EE<E>;       // OK
}
```

— end example]

- ² A *using-enum-declaration* is equivalent to a *using-declaration* for each enumerator.
- ³ [Note 1: A *using-enum-declaration* in class scope makes the enumerators of the named enumeration available via member lookup.

[Example 2:

```
enum class fruit { orange, apple };
struct S {
    using enum fruit;       // OK, introduces orange and apple into S
};
void f() {
    S s;
    s.orange;              // OK, names fruit::orange
    S::orange;             // OK, names fruit::orange
}
```

— end example]

— end note]

- ⁴ [Note 2: Two *using-enum-declarations* that introduce two enumerators of the same name conflict.

[Example 3:

```
enum class fruit { orange, apple };
enum class color { red, orange };
void f() {
    using enum fruit;       // OK
    using enum color;       // error: color::orange and fruit::orange conflict
}
```

— end example]

— end note]

9.9 Namespaces

[basic.namespace]

9.9.1 General

[basic.namespace.general]

- ¹ A namespace is an optionally-named entity whose scope can contain declarations of any kind of entity. The name of a namespace can be used to access entities that belong to that namespace; that is, the *members* of

the namespace. Unlike other entities, the definition of a namespace can be split over several parts of one or more translation units and modules.

- ² [Note 1: A *namespace-definition* is exported if it contains any *export-declarations* (10.2). A namespace is never attached to a named module and never has a name with module linkage. — end note]

[Example 1:

```
export module M;
namespace N1 {}           // N1 is not exported
export namespace N2 {}    // N2 is exported
namespace N3 { export int n; } // N3 is exported
```

— end example]

- ³ There is a *global namespace* with no declaration; see 6.4.6. The global namespace belongs to the global scope; it is not an unnamed namespace (9.9.2.2).

[Note 2: Lacking a declaration, it cannot be found by name lookup. — end note]

9.9.2 Namespace definition

[namespace.def]

9.9.2.1 General

[namespace.def.general]

```
namespace-name:
    identifier
    namespace-alias

namespace-definition:
    named-namespace-definition
    unnamed-namespace-definition
    nested-namespace-definition

named-namespace-definition:
    inlineopt namespace attribute-specifier-seqopt identifier { namespace-body }

unnamed-namespace-definition:
    inlineopt namespace attribute-specifier-seqopt { namespace-body }

nested-namespace-definition:
    namespace enclosing-namespace-specifier :: inlineopt identifier { namespace-body }

enclosing-namespace-specifier:
    identifier
    enclosing-namespace-specifier :: inlineopt identifier

namespace-body:
    declaration-seqopt
```

- ¹ Every *namespace-definition* shall inhabit a namespace scope (6.4.6).
- ² In a *named-namespace-definition* *D*, the *identifier* is the name of the namespace. The *identifier* is looked up by searching for it in the scopes of the namespace *A* in which *D* appears and of every element of the inline namespace set of *A*. If the lookup finds a *namespace-definition* for a namespace *N*, *D* extends *N*, and the target scope of *D* is the scope to which *N* belongs. If the lookup finds nothing, the *identifier* is introduced as a *namespace-name* into *A*.
- ³ Because a *namespace-definition* contains *declarations* in its *namespace-body* and a *namespace-definition* is itself a *declaration*, it follows that *namespace-definitions* can be nested.

[Example 1:

```
namespace Outer {
    int i;
    namespace Inner {
        void f() { i++; }    // Outer::i
        int i;
        void g() { i++; }    // Inner::i
    }
}
```

— end example]

- ⁴ If the optional initial **inline** keyword appears in a *namespace-definition* for a particular namespace, that namespace is declared to be an *inline namespace*. The **inline** keyword may be used on a *namespace-definition*

that extends a namespace only if it was previously used on the *namespace-definition* that initially declared the *namespace-name* for that namespace.

- ⁵ The optional *attribute-specifier-seq* in a *named-namespace-definition* appertains to the namespace being defined or extended.
- ⁶ Members of an inline namespace can be used in most respects as though they were members of the innermost enclosing namespace. Specifically, the inline namespace and its enclosing namespace are both added to the set of associated namespaces used in argument-dependent lookup (6.5.4) whenever one of them is, and a *using-directive* (9.9.4) that names the inline namespace is implicitly inserted into the enclosing namespace as for an unnamed namespace (9.9.2.2). Furthermore, each member of the inline namespace can subsequently be partially specialized (13.7.6), explicitly instantiated (13.9.3), or explicitly specialized (13.9.4) as though it were a member of the enclosing namespace. Finally, looking up a name in the enclosing namespace via explicit qualification (6.5.5.3) will include members of the inline namespace even if there are declarations of that name in the enclosing namespace.
- ⁷ These properties are transitive: if a namespace N contains an inline namespace M, which in turn contains an inline namespace O, then the members of O can be used as though they were members of M or N. The *inline namespace set* of N is the transitive closure of all inline namespaces in N.
- ⁸ A *nested-namespace-definition* with an *enclosing-namespace-specifier* E, *identifier* I and *namespace-body* B is equivalent to

```
namespace E { inlineopt namespace I { B } }
```

where the optional `inline` is present if and only if the *identifier* I is preceded by `inline`.

[Example 2:

```
namespace A::inline B::C {
    int i;
}
```

The above has the same effect as:

```
namespace A {
    inline namespace B {
        namespace C {
            int i;
        }
    }
}
```

— end example]

9.9.2.2 Unnamed namespaces

[namespace.unnamed]

- ¹ An *unnamed-namespace-definition* behaves as if it were replaced by

```
inlineopt namespace unique { /* empty body */ }
using namespace unique ;
namespace unique { namespace-body }
```

where `inline` appears if and only if it appears in the *unnamed-namespace-definition* and all occurrences of *unique* in a translation unit are replaced by the same identifier, and this identifier differs from all other identifiers in the program. The optional *attribute-specifier-seq* in the *unnamed-namespace-definition* appertains to *unique*.

[Example 1:

```
namespace { int i; }           // unique::i
void f() { i++; }              // unique::i++

namespace A {
    namespace {
        int i;                 // A::unique::i
        int j;                 // A::unique::j
    }
    void g() { i++; }           // A::unique::i++
}
```

```

using namespace A;
void h() {
    i++;                // error: unique::i or A::unique::i
    A::i++;             // A::unique::i
    j++;               // A::unique::j
}

```

— end example]

9.9.3 Namespace alias

[namespace.alias]

- ¹ A *namespace-alias-definition* declares a *namespace alias* according to the following grammar:

```

namespace-alias:
    identifier

namespace-alias-definition:
    namespace identifier = qualified-namespace-specifier ;
    namespace identifier = splice-specifier ;

qualified-namespace-specifier:
    nested-name-specifieropt namespace-name

```

- ² The *splice-specifier* (if any) shall designate a namespace that is not the global namespace.
- ³ The *identifier* in a *namespace-alias-definition* becomes a *namespace-alias*.
- ⁴ The underlying entity (6.1) of the namespace alias is the namespace either denoted by the *qualified-namespace-specifier* or designated by the *splice-specifier*.

[Note 1: When looking up a *namespace-name* in a *namespace-alias-definition*, only namespace names are considered, see 6.5.7. — end note]

9.9.4 Using namespace directive

[namespace.udir]

```

using-directive:
    attribute-specifier-seqopt using namespace nested-name-specifieropt namespace-name ;
    attribute-specifier-seqopt using namespace splice-specifier ;

```

- ¹ The *splice-specifier* (if any) shall designate a namespace that is not the global namespace. The *nested-name-specifier*, *namespace-name*, and *splice-specifier* shall not be dependent.
- ² A *using-directive* shall not appear in class scope, but may appear in namespace scope or in block scope.

[Note 1: When looking up a *namespace-name* in a *using-directive*, only namespace names are considered, see 6.5.7. — end note]

The optional *attribute-specifier-seq* appertains to the *using-directive*.

- ³ [Note 2: A *using-directive* makes the names in the designated namespace usable in the scope in which the *using-directive* appears after the *using-directive* (6.5.3, 6.5.5.3). During unqualified name lookup, the names appear as if they were declared in the nearest enclosing namespace which contains both the *using-directive* and the designated namespace. — end note]

- ⁴ [Note 3: A *using-directive* does not introduce any names. — end note]

[Example 1:

```

namespace A {
    int i;
    namespace B {
        namespace C {
            int i;
        }
        using namespace A::B::C;
        void f1() {
            i = 5;          // OK, C::i visible in B and hides A::i
        }
    }
    namespace D {
        using namespace B;
        using namespace C;
        void f2() {
            i = 5;          // ambiguous, B::C::i or A::i?
        }
    }
}

```

```

    }
}
void f3() {
    i = 5;           // uses A::i
}
}
void f4() {
    i = 5;           // error: neither i is visible
}

```

— end example]

- ⁵ [Note 4: A *using-directive* is transitive: if a scope contains a *using-directive* that designates a namespace that itself contains *using-directives*, the namespaces designated by those *using-directives* are also eligible to be considered. — end note]

[Example 2:

```

namespace M {
    int i;
}

namespace N {
    int i;
    using namespace M;
}

void f() {
    using namespace N;
    i = 7;           // error: both M::i and N::i are visible
}

```

For another example,

```

namespace A {
    int i;
}
namespace B {
    int i;
    int j;
    namespace C {
        namespace D {
            using namespace A;
            int j;
            int k;
            int a = i; // B::i hides A::i
        }
        using namespace D;
        int k = 89;    // no problem yet
        int l = k;     // ambiguous: C::k or D::k
        int m = i;     // B::i hides A::i
        int n = j;     // D::j hides B::j
    }
}

```

— end example]

- ⁶ [Note 5: Declarations in a namespace that appear after a *using-directive* for that namespace can be found through that *using-directive* after they appear. — end note]
- ⁷ [Note 6: If name lookup finds a declaration for a name in two different namespaces, and the declarations do not declare the same entity and do not declare functions or function templates, the use of the name is ill-formed (6.5). In particular, the name of a variable, function or enumerator does not hide the name of a class or enumeration declared in a different namespace. For example,

```

namespace A {
    class X { };
    extern "C"   int g();
    extern "C++" int h();
}

```

```

namespace B {
    void X(int);
    extern "C"    int g();
    extern "C++"  int h(int);
}
using namespace A;
using namespace B;

void f() {
    X(1);          // error: name X found in two namespaces
    g();           // OK, name g refers to the same entity
    h();           // OK, overload resolution selects A::h
}

```

— end note]

- ⁸ [Note 7: The order in which namespaces are considered and the relationships among the namespaces implied by the *using-directives* do not affect overload resolution. Neither is any function excluded because another has the same signature, even if one is in a namespace reachable through *using-directives* in the namespace of the other.⁷⁶ — end note]

[Example 3:

```

namespace D {
    int d1;
    void f(char);
}
using namespace D;

int d1;          // OK, no conflict with D::d1

namespace E {
    int e;
    void f(int);
}

namespace D {    // namespace extension
    int d2;
    using namespace E;
    void f(int);
}

void f() {
    d1++;          // error: ambiguous ::d1 or D::d1?
    ::d1++;        // OK
    D::d1++;       // OK
    d2++;          // OK, D::d2
    e++;           // OK, E::e
    f(1);          // error: ambiguous: D::f(int) or E::f(int)?
    f('a');        // OK, D::f(char)
}

```

— end example]

9.10 The using declaration

[namespace.udecl]

```

using-declaration:
    using using-declarator-list ;

using-declarator-list:
    using-declarator ...opt
    using-declarator-list , using-declarator ...opt

using-declarator:
    typenameopt nested-name-specifier unqualified-id

```

⁷⁶) During name lookup in a class hierarchy, some ambiguities can be resolved by considering whether one member hides the other along some paths (6.5.2). There is no such disambiguation when considering the set of names found as a result of following *using-directives*.

¹ The component names of a *using-declarator* are those of its *nested-name-specifier* and *unqualified-id*. Each *using-declarator* in a *using-declaration*⁷⁷ names the set of declarations found by lookup (6.5.5) for the *using-declarator*, except that class and enumeration declarations that would be discarded are merely ignored when checking for ambiguity (6.5), conversion function templates with a dependent return type are ignored, and certain functions are hidden as described below. If the terminal name of the *using-declarator* is dependent (13.8.3.2), the *using-declarator* is considered to name a constructor if and only if the *nested-name-specifier* has a terminal name that is the same as the *unqualified-id*. If the lookup in any instantiation finds that a *using-declarator* that is not considered to name a constructor does do so, or that a *using-declarator* that is considered to name a constructor does not, the program is ill-formed.

² If the *using-declarator* names a constructor, it declares that the class *inherits* the named set of constructor declarations from the nominated base class.

[Note 1: Otherwise, the *unqualified-id* in the *using-declarator* is bound to the *using-declarator*, which is replaced during name lookup with the declarations it names (6.5). If such a declaration is of an enumeration, the names of its enumerators are not bound. For the keyword `typename`, see 13.8. — end note]

³ In a *using-declaration* used as a *member-declaration*, each *using-declarator* shall either name an enumerator or have a *nested-name-specifier* naming a base class of the current class (7.5.3).

[Example 1:

```
enum class button { up, down };
struct S {
    using button::up;
    button b = up;           // OK
};
```

— end example]

If a *using-declarator* names a constructor, its *nested-name-specifier* shall name a direct base class of the current class. If the immediate (class) scope is associated with a class template, it shall derive from the specified base class or have at least one dependent base class.

[Example 2:

```
struct B {
    void f(char);
    enum E { e };
    union { int x; };
};

struct C {
    int f();
};

struct D : B {
    using B::f;           // OK, B is a base of D
    using B::e;           // OK, e is an enumerator of base B
    using B::x;           // OK, x is a union member of base B
    using C::f;           // error: C isn't a base of D
    void f(int) { f('c'); } // calls B::f(char)
    void g(int) { g('c'); } // recursively calls D::g(int)
};
template <typename... bases>
struct X : bases... {
    using bases::f...;
};
X<B, C> x;               // OK, B::f and C::f named
```

— end example]

⁴ [Note 2: Since destructors do not have names, a *using-declaration* cannot refer to a destructor for a base class. — end note]

⁷⁷ A *using-declaration* with more than one *using-declarator* is equivalent to a corresponding sequence of *using-declarations* with one *using-declarator* each.

[*Note 3:* A *using-declarator* that names a member function of a base class does not suppress the implicit declaration of a special member function in the derived class, even if their signatures are the same (11.4.5.2, 11.4.5.3, 11.4.6).
— *end note*]

- ⁵ A *using-declaration* shall not name a *template-id*.

[*Example 3:*

```
struct A {
    template <class T> void f(T);
    template <class T> struct X { };
};
struct B : A {
    using A::f<double>;           // error
    using A::X<int>;             // error
};
```

— *end example*]

- ⁶ A *using-declaration* shall not name a namespace.

- ⁷ A *using-declaration* that names a class member other than an enumerator shall be a *member-declaration*.

[*Example 4:*

```
struct X {
    int i;
    static int s;
};

void f() {
    using X::i;                 // error: X::i is a class member and this is not a member declaration.
    using X::s;                 // error: X::s is a class member and this is not a member declaration.
}
```

— *end example*]

- ⁸ If a declaration is named by two *using-declarators* that inhabit the same class scope, the program is ill-formed.

[*Example 5:*

```
struct C {
    int i;
};

struct D1 : C { };
struct D2 : C { };

struct D3 : D1, D2 {
    using D1::i;                // OK, equivalent to using C::i
    using D1::i;                // error: duplicate
    using D2::i;                // error: duplicate, also names C::i
};
```

— *end example*]

- ⁹ [*Note 4:* A *using-declarator* whose *nested-name-specifier* names a namespace does not name declarations added to the namespace after it. Thus, additional overloads added after the *using-declaration* are ignored, but default function arguments (9.3.4.7), default template arguments (13.2), and template specializations (13.7.6, 13.9.4) are considered.
— *end note*]

[*Example 6:*

```
namespace A {
    void f(int);
}

using A::f;                    // f is a synonym for A::f; that is, for A::f(int).
namespace A {
    void f(char);
}
```

```

void foo() {
    f('a');           // calls f(int), even though f(char) exists.
}

void bar() {
    using A::f;        // f is a synonym for A::f; that is, for A::f(int) and A::f(char).
    f('a');           // calls f(char)
}

```

— end example]

- 10 If a declaration named by a *using-declaration* that inhabits the target scope of another declaration *B* potentially conflicts with it (6.4.1), and either is reachable from the other, the program is ill-formed unless *B* is name-independent and the *using-declaration* precedes *B*.

[Example 7:

```

int _;
void f() {
    int _;           // B
    _ = 0;
    using ::_;       // error: using-declaration does not precede B
}

```

— end example]

If two declarations named by *using-declarations* that inhabit the same scope potentially conflict, either is reachable from the other, and they do not both declare functions or function templates, the program is ill-formed.

[Note 5: Overload resolution possibly cannot distinguish between conflicting function declarations. — end note]

[Example 8:

```

namespace A {
    int x;
    int f(int);
    int g;
    void h();
}

namespace B {
    int i;
    struct g { };
    struct x { };
    void f(int);
    void f(double);
    void g(char);           // OK, hides struct g
}

void func() {
    int i;
    using B::i;             // error: conflicts
    void f(char);
    using B::f;             // OK, each f is a function
    using A::f;             // OK, but interferes with B::f(int)
    f(1);                   // error: ambiguous
    static_cast<int*>(int)>(f)(1); // OK, calls A::f
    f(3.5);                 // calls B::f(double)
    using B::g;
    g('a');                 // calls B::g(char)
    struct g g1;            // g1 has class type B::g
    using A::g;             // error: conflicts with B::g
    void h();
    using A::h;             // error: conflicts
    using B::x;
    using A::x;             // OK, hides struct B::x
    using A::x;             // OK, does not conflict with previous using A::x
    x = 99;                 // assigns to A::x
}

```

```
    struct x x1;                      // x1 has class type B::x
}
```

— end example]

- ¹¹ The set of declarations named by a *using-declarator* that inhabits a class *C* does not include member functions and member function templates of a base class that, when considered as members of *C*, correspond to a declaration of a function or function template in *C*.

[Example 9:

```
struct B {
    virtual void f(int);
    virtual void f(char);
    void g(int);
    void h(int);
    void i();
    void j();
};

struct D : B {
    using B::f;
    void f(int);           // OK, D::f(int) overrides B::f(int);

    using B::g;
    void g(char);          // OK

    using B::h;
    void h(int);           // OK, D::h(int) hides B::h(int)

    using B::i;
    void i(this B &);      // OK

    using B::j;
    void j(this D &);      // OK, D::j() hides B::j()
};

void k(D* p)
{
    p->f(1);                // calls D::f(int)
    p->f('a');              // calls B::f(char)
    p->g(1);                // calls B::g(int)
    p->g('a');              // calls D::g(char)
    p->i();                 // calls B::i, because B::i as a member of D is a better match than D::i
    p->j();                 // calls D::j
}

struct B1 {
    B1(int);
};

struct B2 {
    B2(int);
};

struct D1 : B1, B2 {
    using B1::B1;
    using B2::B2;
};
D1 d1(0);                 // error: ambiguous

struct D2 : B1, B2 {
    using B1::B1;
    using B2::B2;
    D2(int);               // OK, D2::D2(int) hides B1::B1(int) and B2::B2(int)
};
```

```
D2 d2(0);           // calls D2::D2(int)
```

— end example]

- ¹² [Note 6: For the purpose of forming a set of candidates during overload resolution, the functions named by a *using-declaration* in a derived class are treated as though they were direct members of the derived class. In particular, the implicit object parameter is treated as if it were a reference to the derived class rather than to the base class (12.2.2). This has no effect on the type of the function, and in all other respects the function remains part of the base class. — end note]

- ¹³ Constructors that are named by a *using-declaration* are treated as though they were constructors of the derived class when looking up the constructors of the derived class (6.5.5.2) or forming a set of overload candidates (12.2.2.4, 12.2.2.5, 12.2.2.8).

[Note 7: If such a constructor is selected to perform the initialization of an object of class type, all subobjects other than the base class from which the constructor originated are implicitly initialized (11.9.4). A constructor of a derived class is sometimes preferred to a constructor of a base class if they would otherwise be ambiguous (12.2.4). — end note]

- ¹⁴ In a *using-declarator* that does not name a constructor, every declaration named shall be accessible. In a *using-declarator* that names a constructor, no access check is performed.

- ¹⁵ [Note 8: Because a *using-declarator* designates a base class member (and not a member subobject or a member function of a base class subobject), a *using-declarator* cannot be used to resolve inherited member ambiguities.

[Example 10:

```
struct A { int x(); };
struct B : A { };
struct C : A {
    using A::x;
    int x(int);
};

struct D : B, C {
    using C::x;
    int x(double);
};

int f(D* d) {
    return d->x();    // error: overload resolution selects A::x, but A is an ambiguous base class
}
```

— end example]

— end note]

- ¹⁶ A *using-declaration* has the usual accessibility for a *member-declaration*. Base-class constructors considered because of a *using-declarator* are accessible if they would be accessible when used to construct an object of the base class; the accessibility of the *using-declaration* is ignored.

[Example 11:

```
class A {
private:
    void f(char);
public:
    void f(int);
protected:
    void g();
};

class B : public A {
    using A::f;    // error: A::f(char) is inaccessible
public:
    using A::g;    // B::g is a public synonym for A::g
};
```

— end example]

9.11 The asm declaration

[dcl.asm]

- ¹ An **asm** declaration has the form

```
asm-declaration:
    attribute-specifier-seqopt asm ( balanced-token-seq ) ;
```

The **asm** declaration is conditionally-supported; any restrictions on the *balanced-token-seq* and its meaning are implementation-defined. The optional *attribute-specifier-seq* in an *asm-declaration* appertains to the **asm** declaration.

[*Note 1*: Typically it is used to pass information through the implementation to an assembler. — *end note*]

9.12 Linkage specifications

[**dcl.link**]

- ¹ All functions and variables whose names have external linkage and all function types have a *language linkage*.

[*Note 1*: Some of the properties associated with an entity with language linkage are specific to each implementation and are not described here. For example, a particular language linkage might be associated with a particular form of representing names of objects and functions with external linkage, or with a particular calling convention, etc. — *end note*]

The default language linkage of all function types, functions, and variables is C++ language linkage. Two function types with different language linkages are distinct types even if they are otherwise identical.

- ² Linkage (6.7) between C++ and non-C++ code fragments can be achieved using a *linkage-specification*:

```
linkage-specification:
    extern unevaluated-string { declaration-seqopt }
    extern unevaluated-string name-declaration
```

The *unevaluated-string* indicates the required language linkage.

[*Note 2*: Escape sequences and *universal-character-names* have been replaced (5.13.6). — *end note*]

This document specifies the semantics for the *unevaluated-strings* "C" and "C++". Use of an *unevaluated-string* other than "C" or "C++" is conditionally-supported, with implementation-defined semantics.

[*Note 3*: Therefore, a *linkage-specification* with a language linkage that is unknown to the implementation requires a diagnostic. — *end note*]

Recommended practice: The spelling of the language linkage should be taken from the document defining that language. For example, *Ada* (not *ADA*) and *Fortran* or *FORTRAN*, depending on the vintage.

- ³ Every implementation shall provide for linkage to the C programming language, "C", and C++, "C++".

[*Example 1*:

```
complex sqrt(complex);           // C++ language linkage by default
extern "C" {
    double sqrt(double);         // C language linkage
}
```

— *end example*]

- ⁴ A *module-import-declaration* appearing in a linkage specification with other than C++ language linkage is conditionally-supported with implementation-defined semantics.

- ⁵ Linkage specifications nest. When linkage specifications nest, the innermost one determines the language linkage.

[*Note 4*: A linkage specification does not establish a scope. — *end note*]

A *linkage-specification* shall inhabit a namespace scope. In a *linkage-specification*, the specified language linkage applies to the function types of all function declarators and to all functions and variables whose names have external linkage.

[*Example 2*:

```
extern "C"                       // f1 and its function type have C language linkage;
    void f1(void(*pf)(int));      // pf is a pointer to a C function

extern "C" typedef void FUNC();
FUNC f2;                          // f2 has C++ language linkage and
                                   // its type has C language linkage

extern "C" FUNC f3;               // f3 and its type have C language linkage
```

```

void (*pf2)(FUNC*);           // the variable pf2 has C++ language linkage; its type
                               // is "pointer to C++ function that takes one parameter of type
                               // pointer to C function"

extern "C" {
    static void f4();          // the name of the function f4 has internal linkage,
                               // so f4 has no language linkage; its type has C language linkage
}

extern "C" void f5() {
    extern void f4();           // OK, name linkage (internal) and function type linkage (C language linkage)
                               // obtained from previous declaration.
}

extern void f4();              // OK, name linkage (internal) and function type linkage (C language linkage)
                               // obtained from previous declaration.

void f6() {
    extern void f4();           // OK, name linkage (internal) and function type linkage (C language linkage)
                               // obtained from previous declaration.
}

```

— end example]

A C language linkage is ignored in determining the language linkage of class members, friend functions with a trailing *requires-clause*, and the function type of non-static class member functions.

[Example 3:

```

extern "C" typedef void FUNC_c();

class C {
    void mf1(FUNC_c*);          // the function mf1 and its type have C++ language linkage;
                               // the parameter has type "pointer to C function"

    FUNC_c mf2;                 // the function mf2 and its type have C++ language linkage

    static FUNC_c* q;           // the data member q has C++ language linkage;
                               // its type is "pointer to C function"
};

extern "C" {
    class X {
        void mf();              // the function mf and its type have C++ language linkage
        void mf2(void(*)());    // the function mf2 has C++ language linkage;
                               // the parameter has type "pointer to C function"
    };
}

```

— end example]

- ⁶ If two declarations of an entity give it different language linkages, the program is ill-formed; no diagnostic is required if neither declaration is reachable from the other (G.6.6). A redeclaration of an entity without a linkage specification inherits the language linkage of the entity and (if applicable) its type.
- ⁷ Two declarations declare the same entity if they (re)introduce the same name, one declares a function or variable with C language linkage, and the other declares such an entity or declares a variable that belongs to the global scope.

[Example 4:

```

int x;
namespace A {
    extern "C" int f();
    extern "C" int g() { return 1; }
    extern "C" int h();
    extern "C" int x();          // error: same name as global-space object x
}

```

```

namespace B {
    extern "C" int f();           // A::f and B::f refer to the same function
    extern "C" int g() { return 1; } // error: the function g with C language linkage has two definitions
}

int A::f() { return 98; }        // definition for the function f with C language linkage
extern "C" int h() { return 97; } // definition for the function h with C language linkage
                                // A::h and ::h refer to the same function

```

— end example]

- ⁸ A declaration directly contained in a *linkage-specification* is treated as if it contains the **extern** specifier (9.2.2) for the purpose of determining the linkage of the declared name and whether it is a definition. Such a declaration shall not have a *storage-class-specifier*.

[Example 5:

```

extern "C" double f();
static double f();           // error
extern "C" int i;             // declaration
extern "C" {
    int i;                    // definition
}
extern "C" static void g();    // error

```

— end example]

- ⁹ [Note 5: Because the language linkage is part of a function type, when indirecting through a pointer to C function, the function to which the resulting lvalue refers is considered a C function. — end note]
- ¹⁰ Linkage from C++ to entities defined in other languages and to entities defined in C++ from other languages is implementation-defined and language-dependent. Only where the object layout strategies of two language implementations are similar enough can such linkage be achieved.

9.13 Attributes

[dcl.attr]

9.13.1 Attribute syntax and semantics

[dcl.attr.grammar]

- ¹ Attributes and annotations specify additional information for various source constructs such as types, variables, names, contract assertions, blocks, or translation units.

```

attribute-specifier-seq:
    attribute-specifier attribute-specifier-seqopt

attribute-specifier:
    [ [ attribute-using-prefixopt attribute-list ] ]
    [ [ annotation-list ] ]
    alignment-specifier

alignment-specifier:
    alignas ( nofun-type-id ...opt )
    alignas ( constant-expression ...opt )

attribute-using-prefix:
    using attribute-namespace :

attribute-list:
    attributeopt
    attribute-list , attributeopt
    attribute ...
    attribute-list , attribute ...

annotation-list:
    annotation ...opt
    annotation-list , annotation ...opt

attribute:
    attribute-token attribute-argument-clauseopt

annotation:
    = constant-expression

```



```

attribute-token:
    identifier
    attribute-scoped-token

attribute-scoped-token:
    attribute-namespace :: identifier

attribute-namespace:
    identifier

attribute-argument-clause:
    ( balanced-token-seqopt )

balanced-token-seq:
    balanced-token balanced-token-seqopt

balanced-token:
    ( balanced-token-seqopt )
    [ balanced-token-seqopt ]
    { balanced-token-seqopt }
    [: balanced-token-seqopt :]
    any token other than (, ), [, ], {, }, [:, or :]

```

- ² If an *attribute-specifier* contains an *attribute-using-prefix*, the *attribute-list* following that *attribute-using-prefix* shall not contain an *attribute-scoped-token* and every *attribute-token* in that *attribute-list* is treated as if its *identifier* were prefixed with *N::*, where *N* is the *attribute-namespace* specified in the *attribute-using-prefix*.

[*Note 1*: This rule imposes no constraints on how an *attribute-using-prefix* affects the tokens in an *attribute-argument-clause*. — end note]

[*Example 1*:

```

[[using CC: opt(1), debug]]           // same as [[CC::opt(1), CC::debug]]
void f() {}
[[using CC: opt(1)]] [[CC::debug]]    // same as [[CC::opt(1)]] [[CC::debug]]
void g() {}
[[using CC: CC::opt(1)]]              // error: cannot combine using and scoped attribute token
void h() {}

```

— end example]

- ³ [*Note 2*: For each individual attribute, the form of the *balanced-token-seq* will be specified. — end note]
- ⁴ In an *attribute-list*, an ellipsis may appear only if that *attribute*'s specification permits it. An *attribute* followed by an ellipsis is a pack expansion (13.7.4). An *attribute-specifier* that contains an *attribute-list* with no *attributes* has no effect. The order in which the *attribute-tokens* appear in an *attribute-list* is not significant. If a keyword (5.12) or an alternative token (5.9) that satisfies the syntactic requirements of an *identifier* (5.11) is contained in an *attribute-token*, it is considered an identifier. No name lookup (6.5) is performed on any of the identifiers contained in an *attribute-token*. The *attribute-token* determines additional requirements on the *attribute-argument-clause* (if any).
- ⁵ [*Note 3*: Unless otherwise specified, an *attribute-token* specified in this document cannot be used as a macro name (15.7.1). — end note]
- ⁶ An *annotation* followed by an ellipsis is a pack expansion (13.7.4).
- ⁷ Each *attribute-specifier-seq* is said to *appertain* to some entity or statement, identified by the syntactic context where it appears (Clause 8, Clause 9, 9.3). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* or *alignment-specifier* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.8.4), that declaration shall be a definition.

[*Note 4*: An *attribute-specifier-seq* cannot appertain to an explicit instantiation (13.9.3). — end note]

- ⁸ For an *attribute-token* (including an *attribute-scoped-token*) not specified in this document, the behavior is implementation-defined; any such *attribute-token* that is not recognized by the implementation is ignored.

[*Note 5*: A program is ill-formed if it contains an *attribute* specified in 9.13 that violates the rules specifying to which entity or statement the attribute can apply or the syntax rules for the attribute's *attribute-argument-clause*, if any. — end note]

[*Note 6*: The *attributes* specified in 9.13 have optional semantics: given a well-formed program, removing all instances of any one of those *attributes* results in a program whose set of possible executions (4.1.2) for a given input is a subset

of those of the original program for the same input, absent implementation-defined guarantees with respect to that *attribute*. — *end note*

An *attribute-token* is reserved for future standardization if

- (8.1) — it is not an *attribute-scoped-token* and is not specified in this document, or
- (8.2) — it is an *attribute-scoped-token* and its *attribute-namespace* is **std** followed by zero or more digits.

Each implementation should choose a distinctive name for the *attribute-namespace* in an *attribute-scoped-token*.

- 9 Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier* or within the *balanced-token-seq* of an *attribute-argument-clause*.

[*Note 7*: If two consecutive left square brackets appear where an *attribute-specifier* is not allowed, the program is ill-formed even if the brackets match an alternative grammar production. — *end note*]

[*Example 2*:

```
int p[10];
void f() {
    int x = 42, y[5];
    int(p[[x] { return x; }()]); // error: invalid attribute on a nested declarator-id and
                                // not a function-style cast of an element of p.
    y[[] { return 2; }()] = 2;   // error even though attributes are not allowed in this context.
    int i [[vendor::attr([[]])]]; // well-formed implementation-defined attribute.
}
```

— *end example*]

9.13.2 Alignment specifier

[**dcl.align**]

- 1 An *alignment-specifier* may be applied to a variable or to a class data member, but it shall not be applied to a bit-field, a function parameter, or an *exception-declaration* (14.4). An *alignment-specifier* may also be applied to the declaration of a class (in an *elaborated-type-specifier* (9.2.9.5) or *class-head* (Clause 11), respectively). An *alignment-specifier* with an ellipsis is a pack expansion (13.7.4).
- 2 When the *alignment-specifier* is of the form **alignas**(*constant-expression*):
 - (2.1) — the *constant-expression* shall be an integral constant expression
 - (2.2) — if the constant expression does not evaluate to an alignment value (6.8.3), or evaluates to an extended alignment and the implementation does not support that alignment in the context of the declaration, the program is ill-formed.
- 3 An *alignment-specifier* of the form **alignas**(*nofun-type-id*) has the same effect as **alignas**(**alignof**(*nofun-type-id*)) (7.6.2.6).
- 4 The alignment requirement of an entity is the strictest nonzero alignment specified by its *alignment-specifiers*, if any; otherwise, the *alignment-specifiers* have no effect.
- 5 The combined effect of all *alignment-specifiers* in a declaration shall not specify an alignment that is less strict than the alignment that would be required for the entity being declared if all *alignment-specifiers* appertaining to that entity were omitted.

[*Example 1*:

```
struct alignas(8) S {};
struct alignas(1) U {
    S s;
}; // error: U specifies an alignment that is less strict than if the alignas(1) were omitted.
```

— *end example*]

- 6 If the defining declaration of an entity has an *alignment-specifier*, any non-defining declaration of that entity shall either specify equivalent alignment or have no *alignment-specifier*. Conversely, if any declaration of an entity has an *alignment-specifier*, every defining declaration of that entity shall specify an equivalent alignment. No diagnostic is required (G.6.7) if declarations of an entity have different *alignment-specifiers* in different translation units.

[*Example 2*:

```
// Translation unit #1:
struct S { int x; } s, *p = &s;
```

```
// Translation unit #2:
struct alignas(16) S;           // ill-formed, no diagnostic required: definition of S lacks alignment
extern S* p;
— end example]
```

- 7 [Example 3: An aligned buffer with an alignment requirement of **A** and holding **N** elements of type **T** can be declared as:

```
alignas(T) alignas(A) T buffer[N];
```

Specifying **alignas(T)** ensures that the final requested alignment will not be weaker than **alignof(T)**, and therefore the program will not be ill-formed. — end example]

- 8 [Example 4:

```
alignas(double) void f();           // error: alignment applied to function
alignas(double) unsigned char c[sizeof(double)]; // array of characters, suitably aligned for a double
extern unsigned char c[sizeof(double)]; // no alignas necessary
alignas(float)
extern unsigned char c[sizeof(double)]; // error: different alignment in declaration
— end example]
```

9.13.3 Assumption attribute

[dcl.attr.assume]

- 1 The *attribute-token* **assume** may be applied to a null statement; such a statement is an *assumption*. An *attribute-argument-clause* shall be present and shall have the form:

(*conditional-expression*)

The expression is contextually converted to **bool** (7.3.1). The expression is not evaluated. If the converted expression would evaluate to **true** at the point where the assumption appears, the assumption has no effect. Otherwise, evaluation of the assumption has runtime-undefined behavior (F.5.8).

- 2 [Note 1: The expression is potentially evaluated (6.3). The use of assumptions is intended to allow implementations to analyze the form of the expression and deduce information used to optimize the program. Implementations are not required to deduce any information from any particular assumption. It is expected that the value of a *has-attribute-expression* for the **assume** attribute is 0 if an implementation does not attempt to deduce any such information from assumptions. — end note]

- 3 [Example 1:

```
int divide_by_32(int x) {
    [[assume(x >= 0)]];
    return x/32;           // The instructions produced for the division
                          // may omit handling of negative values.
}
int f(int y) {
    [[assume(++y == 43)]]; // y is not incremented
    return y;             // statement may be replaced with return 42;
}
— end example]
```

9.13.4 Deprecated attribute

[dcl.attr.deprecated]

- 1 The *attribute-token* **deprecated** can be used to mark names and entities whose use is still allowed, but is discouraged for some reason.

[Note 1: In particular, **deprecated** is appropriate for names and entities that are deemed obsolescent or unsafe. — end note]

An *attribute-argument-clause* may be present and, if present, it shall have the form:

(*unevaluated-string*)

[Note 2: The *unevaluated-string* in the *attribute-argument-clause* can be used to explain the rationale for deprecation and/or to suggest a replacing entity. — end note]

- 2 The attribute may be applied to the declaration of a class, a type alias, a variable, a non-static data member, a function, a namespace, an enumeration, an enumerator, a concept, or a template specialization.
- 3 An entity declared without the **deprecated** attribute can later be redeclared with the attribute and vice-versa.

[*Note 3*: Thus, an entity initially declared without the attribute can be marked as deprecated by a subsequent redeclaration. However, after an entity is marked as deprecated, later redeclarations do not un-deprecate the entity. — *end note*]

Redeclarations using different forms of the attribute (with or without the *attribute-argument-clause* or with different *attribute-argument-clauses*) are allowed.

- ⁴ *Recommended practice*: Implementations should use the **deprecated** attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute. The diagnostic message should include the text provided within the *attribute-argument-clause* of any **deprecated** attribute applied to the name or entity. The value of a *has-attribute-expression* for the **deprecated** attribute should be 0 unless the implementation can issue such diagnostic messages.

9.13.5 Fallthrough attribute

[**dcl.attr.fallthrough**]

- ¹ The *attribute-token* **fallthrough** may be applied to a null statement (8.3); such a statement is a *fallthrough statement*. No *attribute-argument-clause* shall be present. A fallthrough statement may only appear within an enclosing **switch** statement (8.5.3). The next statement that would be executed after a fallthrough statement shall be a labeled statement whose label is a case label or default label for the same **switch** statement and, if the fallthrough statement is contained in an iteration statement, the next statement shall be part of the same execution of the substatement of the innermost enclosing iteration statement. The program is ill-formed if there is no such statement. The innermost enclosing **switch** statement of a fallthrough statement *S* shall be contained in the innermost enclosing expansion statement (8.7) of *S*, if any.
- ² *Recommended practice*: The use of a fallthrough statement should suppress a warning that an implementation might otherwise issue for a case or default label that is reachable from another case or default label along some path of execution. The value of a *has-attribute-expression* for the **fallthrough** attribute should be 0 if the attribute does not cause suppression of such warnings. Implementations should issue a warning if a fallthrough statement is not dynamically reachable.
- ³ [*Example 1*:

```
void f(int n) {
    void g(), h(), i();
    switch (n) {
        case 1:
        case 2:
            g();
            [[fallthrough]];
        case 3:                                     // warning on fallthrough discouraged
            do {
                [[fallthrough]];                     // error: next statement is not part of the same substatement execution
            } while (false);
        case 6:
            do {
                [[fallthrough]];                     // error: next statement is not part of the same substatement execution
            } while (n--);
        case 7:
            while (false) {
                [[fallthrough]];                     // error: next statement is not part of the same substatement execution
            }
        case 5:
            h();
        case 4:                                     // implementation may warn on fallthrough
            i();
            [[fallthrough]];                         // error
    }
}
```

— *end example*]

9.13.6 Indeterminate storage

[**dcl.attr.indet**]

- ¹ The *attribute-token* **indeterminate** may be applied to the definition of a block variable with automatic storage duration or to a *parameter-declaration* of a function declaration. No *attribute-argument-clause* shall be present. The attribute specifies that the storage of an object with automatic storage duration is initially indeterminate rather than erroneous (6.8.5).

- ² If a function parameter is declared with the `indeterminate` attribute, it shall be so declared in the first declaration of its function. If a function parameter is declared with the `indeterminate` attribute in the first declaration of its function in one translation unit and the same function is declared without the `indeterminate` attribute on the same parameter in its first declaration in another translation unit, the program is ill-formed, no diagnostic required (G.6.8).

- ³ [Note 1: Reading from an uninitialized variable that is marked `[[indeterminate]]` can cause undefined behavior.

```
void f(int);
void g() {
    int x [[indeterminate]], y;
    f(y);           // erroneous behavior (6.8.5)
    f(x);           // undefined behavior
}

struct T {
    T() {}
    int x;
};
int h(T t [[indeterminate]]) {
    f(t.x);         // undefined behavior when called below
    return 0;
}
int _ = h(T());
— end note]
```

9.13.7 Likelihood attributes

[dcl.attr.likelihood]

- ¹ The *attribute-tokens* `likely` and `unlikely` may be applied to labels or statements. No *attribute-argument-clause* shall be present. The *attribute-token* `likely` shall not appear in an *attribute-specifier-seq* that contains the *attribute-token* `unlikely`.

- ² [Note 1: The use of the `likely` attribute is intended to allow implementations to optimize for the case where paths of execution including it are arbitrarily more likely than any alternative path of execution that does not include such an attribute on a statement or label. The use of the `unlikely` attribute is intended to allow implementations to optimize for the case where paths of execution including it are arbitrarily more unlikely than any alternative path of execution that does not include such an attribute on a statement or label. It is expected that the value of a *has-attribute-expression* for the `likely` and `unlikely` attributes is 0 if the implementation does not attempt to use these attributes for such optimizations. A path of execution includes a label if and only if it contains a jump to that label. — end note]

[Note 2: Excessive usage of either of these attributes is liable to result in performance degradation. — end note]

- ³ [Example 1:

```
void g(int);
int f(int n) {
    if (n > 5) [[unlikely]] {           // n > 5 is considered to be arbitrarily unlikely
        g(0);
        return n * 2 + 1;
    }

    switch (n) {
    case 1:
        g(1);
        [[fallthrough]];

    [[likely]] case 2:                  // n == 2 is considered to be arbitrarily more
        g(2);                          // likely than any other value of n
        break;
    }
    return 3;
}
```

— end example]

9.13.8 Maybe unused attribute

[dcl.attr.unused]

- 1 The *attribute-token* **maybe_unused** indicates that a name, label, or entity is possibly intentionally unused. No *attribute-argument-clause* shall be present.
- 2 The attribute may be applied to the declaration of a class, type alias, variable (including a structured binding declaration), structured binding, result binding (9.4.2), non-static data member, function, enumeration, or enumerator, or to an *identifier* label (8.2).
- 3 A name or entity declared without the **maybe_unused** attribute can later be redeclared with the attribute and vice versa. An entity is considered marked after the first declaration that marks it.
- 4 *Recommended practice:* For an entity marked **maybe_unused**, implementations should not emit a warning that the entity or its structured bindings (if any) are used or unused. For a structured binding declaration not marked **maybe_unused**, implementations should not emit such a warning unless all of its structured bindings are unused. For a label to which **maybe_unused** is applied, implementations should not emit a warning that the label is used or unused. The value of a *has-attribute-expression* for the **maybe_unused** attribute should be 0 if the attribute does not cause suppression of such warnings.

5 [Example 1:

```
[[maybe_unused]] void f([[maybe_unused]] bool thing1,
                        [[maybe_unused]] bool thing2) {
    [[maybe_unused]] bool b = thing1 && thing2;
    assert(b);
#ifdef NDEBUG
    goto x;
#endif
    [[maybe_unused]] x:
}
```

Implementations should not warn that **b** or **x** is unused, whether or not **NDEBUG** is defined. — end example]

9.13.9 Nodiscard attribute

[dcl.attr.nodiscard]

- 1 The *attribute-token* **nodiscard** may be applied to a function or a lambda call operator or to the declaration of a class or enumeration. An *attribute-argument-clause* may be present and, if present, shall have the form:

(*unevaluated-string*)

- 2 A name or entity declared without the **nodiscard** attribute can later be redeclared with the attribute and vice-versa.

[Note 1: Thus, an entity initially declared without the attribute can be marked as **nodiscard** by a subsequent redeclaration. However, after an entity is marked as **nodiscard**, later redeclarations do not remove the **nodiscard** from the entity. — end note]

Redeclarations using different forms of the attribute (with or without the *attribute-argument-clause* or with different *attribute-argument-clauses*) are allowed.

- 3 A *nodiscard type* is a (possibly cv-qualified) class or enumeration type marked **nodiscard** in a reachable declaration. A *nodiscard call* is either
 - (3.1) — a function call expression (7.6.1.3) that calls a function declared **nodiscard** in a reachable declaration or whose return type is a *nodiscard type*, or
 - (3.2) — an explicit type conversion (7.6.1.4, 7.6.1.9, 7.6.3) that constructs an object through a constructor declared **nodiscard** in a reachable declaration, or that initializes an object of a *nodiscard type*.
- 4 *Recommended practice:* Appearance of a *nodiscard call* as a potentially evaluated discarded-value expression (7.2) of non-void type is discouraged unless explicitly cast to **void**. Implementations should issue a warning in such cases. The value of a *has-attribute-expression* for the **nodiscard** attribute should be 0 unless the implementation can issue such warnings.

[Note 2: This is typically because discarding the return value of a *nodiscard call* has surprising consequences. — end note]

The *unevaluated-string* in a **nodiscard attribute-argument-clause** should be used in the message of the warning as the rationale for why the result should not be discarded.

5 [Example 1:

```
struct [[nodiscard]] my_scopeguard { /* ... */ };
```

```

struct my_unique {
    my_unique() = default;                                // does not acquire resource
    [[nodiscard]] my_unique(int fd) { /* ... */ }          // acquires resource
    ~my_unique() noexcept { /* ... */ }                   // releases resource, if any
    /* ... */
};
struct [[nodiscard]] error_info { /* ... */ };
error_info enable_missile_safety_mode();
void launch_missiles();
void test_missiles() {
    my_scopeguard();                                     // warning encouraged
    (void)my_scopeguard(),                               // warning not encouraged, cast to void
    launch_missiles();                                   // comma operator, statement continues
    my_unique(42);                                       // warning encouraged
    my_unique();                                         // warning not encouraged
    enable_missile_safety_mode(); // warning encouraged
    launch_missiles();
}
error_info &foo();
void f() { foo(); }                                     // warning not encouraged: not a nodiscard call, because neither
                                                         // the (reference) return type nor the function is declared nodiscard

```

— end example]

9.13.10 Noreturn attribute

[dcl.attr.noreturn]

- ¹ The *attribute-token* **noreturn** specifies that a function does not return. No *attribute-argument-clause* shall be present. The attribute may be applied to a function or a lambda call operator. The first declaration of a function shall specify the **noreturn** attribute if any declaration of that function specifies the **noreturn** attribute. If a function is declared with the **noreturn** attribute in one translation unit and the same function is declared without the **noreturn** attribute in another translation unit, the program is ill-formed, no diagnostic required (G.6.9).
- ² If a function **f** is invoked where **f** was previously declared with the **noreturn** attribute and that invocation eventually returns, the behavior is runtime-undefined (F.5.9).

[Note 1: The function can terminate by throwing an exception. — end note]

- ³ *Recommended practice:* Implementations should issue a warning if a function marked **[[noreturn]]** might return. The value of a *has-attribute-expression* for the **noreturn** attribute should be 0 unless the implementation can issue such warnings.

- ⁴ [Example 1:

```

[[ noreturn ]] void f() {
    throw "error";                                     // OK
}

[[ noreturn ]] void q(int i) { // behavior is undefined if called with an argument <= 0
    if (i > 0)
        throw "positive";
}

```

— end example]

9.13.11 No unique address attribute

[dcl.attr.nouniqueaddr]

- ¹ The *attribute-token* **no_unique_address** specifies that a non-static data member is a potentially-overlapping subobject (6.8.2). No *attribute-argument-clause* shall be present. The attribute may appertain to a non-static data member other than a bit-field.
- ² [Note 1: The non-static data member can share the address of another non-static data member or that of a base class, and any padding that would normally be inserted at the end of the object can be reused as storage for other members. — end note]

Recommended practice: The value of a *has-attribute-expression* for the **no_unique_address** attribute should be 0 for a given implementation unless this attribute can cause a potentially-overlapping subobject to have zero size.

[Example 1:

```
template<typename Key, typename Value,
        typename Hash, typename Pred, typename Allocator>
class hash_map {
    [[no_unique_address]] Hash hasher;
    [[no_unique_address]] Pred pred;
    [[no_unique_address]] Allocator alloc;
    Bucket *buckets;
    // ...
public:
    // ...
};
```

Here, `hasher`, `pred`, and `alloc` could have the same address as `buckets` if their respective types are all empty. — *end example*

9.13.12 Annotations

[**dcl.attr.annotation**]

- ¹ An annotation may be applied to a *base-specifier* or to a declaration *D* of a type, type alias, variable, function, function parameter of non-void type, namespace, enumerator, or non-static data member, unless

- (1.1) — the host scope of *X* differs from its target scope or
- (1.2) — *X* is a non-defining friend declaration,

where *X* is

- (1.3) — *D'* if *D* is a function parameter declaration in a function declarator (9.3.4.6) of a function declaration *D'* and
- (1.4) — *D* otherwise.

[Note 1: An annotation on a *parameter-declaration* in a function definition applies to both the function parameter and the variable.

[Example 1:

```
void f([[=1]] int x);
void f([[=2]] int y) {
    constexpr auto rp = parameters_of(^f)[0];
    constexpr auto ry = variable_of(rp);
    static_assert(ry == ^y);

    static_assert(annotations_of(rp).size() == 2);    // both [1, 2]
    static_assert(annotations_of(ry).size() == 1);    // just [2]
    static_assert(annotations_of(rp)[1] == annotations_of(ry)[0]);
}
```

— *end example*

— *end note*

- ² Let *E* be the expression `std::meta::reflect_constant(constant-expression)`. *E* shall be a constant expression; the result of *E* is the *underlying constant* of the annotation.
- ³ Each *annotation* or instantiation thereof produces a unique annotation.

[Example 2:

```
[[=1]] void f();
[[=2, =3, =2]] void g();
void g [[=4, =2]] ();
```

`f` has one annotation and `g` has five annotations. These can be queried with metafunctions such as `std::meta::annotations_of` (21.4.13). — *end example*

[Example 3:

```
template<int> int x [[=1]];
static_assert(annotations_of(^x<0>) != annotations_of(^x<1>));    // OK
```

— *end example*

- ⁴ Substituting into an *annotation* is not in the immediate context.

[*Example 4:*

```
template<class T>
  [[=T::type()]] void f(T t);

void f(int);

void g() {
  f(0);           // OK
  f('0');        // error, substituting into the annotation results in an invalid expression
}
```

— *end example*]

10 Modules

[module]

10.1 Module units and purviews

[module.unit]

module-declaration:
`export-keywordopt module-keyword module-name module-partitionopt attribute-specifier-seqopt ;`
module-name:
`module-name-qualifieropt identifier`
module-partition:
`: module-name-qualifieropt identifier`
module-name-qualifier:
`identifier .`
`module-name-qualifier identifier .`

- ¹ A *module unit* is a translation unit that contains a *module-declaration*. A *named module* is the collection of module units with the same *module-name*. The identifiers `module` and `import` shall not appear as *identifiers* in a *module-name* or *module-partition*. All *module-names* either beginning with an *identifier* consisting of `std` followed by zero or more *digits* or containing a reserved identifier (5.11) are reserved and shall not be specified in a *module-declaration*; no diagnostic is required (G.7.1). If any *identifier* in a reserved *module-name* is a reserved identifier, the module name is reserved for use by C++ implementations; otherwise it is reserved for future standardization. The optional *attribute-specifier-seq* appertains to the *module-declaration*.
- ² A *module interface unit* is a module unit whose *module-declaration* starts with *export-keyword*; any other module unit is a *module implementation unit*. A named module shall contain exactly one module interface unit with no *module-partition*, known as the *primary module interface unit* of the module; no diagnostic is required (G.7.2).
- ³ A *module partition* is a module unit whose *module-declaration* contains a *module-partition*. A named module shall not contain multiple module partitions with the same *module-partition*. All module partitions of a module that are module interface units shall be directly or indirectly exported by the primary module interface unit (10.3). No diagnostic is required for a violation of these rules (G.7.3).

[Note 1: Module partitions can be imported only by other module units in the same module. The division of a module into module units is not visible outside the module. — end note]

- ⁴ [Example 1:

Translation unit #1:

```
export module A;
export import :Foo;
export int baz();
```

Translation unit #2:

```
export module A:Foo;
import :Internals;
export int foo() { return 2 * (bar() + 1); }
```

Translation unit #3:

```
module A:Internals;
int bar();
```

Translation unit #4:

```
module A;
import :Internals;
int bar() { return baz() - 10; }
int baz() { return 30; }
```

Module A contains four translation units:

- (4.1) — a primary module interface unit,
- (4.2) — a module partition `A:Foo`, which is a module interface unit forming part of the interface of module A,

- (4.3) — a module partition **A:Internals**, which does not contribute to the external interface of module **A**, and
- (4.4) — a module implementation unit providing a definition of **bar** and **baz**, which cannot be imported because it does not have a partition name.
— *end example*]
- 5 A *module unit purview* is the sequence of *tokens* starting at the *module-declaration* and extending to the end of the translation unit. The *purview* of a named module **M** is the set of module unit purviews of **M**'s module units.
- 6 The *global module* is the collection of all *global-module-fragments* and all translation units that are not module units. Declarations appearing in such a context are said to be in the *purview* of the global module.
[*Note 2*: The global module has no name, no module interface unit, and is not introduced by any *module-declaration*.
— *end note*]
- 7 A *module* is either a named module or the global module. A declaration is *attached* to a module as follows:
 - (7.1) — If the declaration is a non-dependent friend declaration that nominates a function with a *declarator-id* that is a *qualified-id* or *template-id* or that nominates a class other than with an *elaborated-type-specifier* with neither a *nested-name-specifier* nor a *simple-template-id*, it is attached to the module to which the friend is attached (6.7).
 - (7.2) — Otherwise, if the declaration
 - (7.2.1) — declares a namespace whose name has external linkage,
 - (7.2.2) — declares a type alias,
 - (7.2.3) — declares a namespace alias, or
 - (7.2.4) — appears within a *linkage-specification* (9.12)
 it is attached to the global module.
 - (7.3) — Otherwise, the declaration is attached to the module in whose purview it appears.
- 8 A *module-declaration* that contains neither an *export-keyword* nor a *module-partition* implicitly imports the primary module interface unit of the module as if by a *module-import-declaration*.

[*Example 2*:

Translation unit #1:

```
module B:Y;           // does not implicitly import B
int y();
```

Translation unit #2:

```
export module B;
import :Y;           // OK, does not create interface dependency cycle
int n = y();
```

Translation unit #3:

```
module B:X1;         // does not implicitly import B
int &a = n;           // error: n not visible here
```

Translation unit #4:

```
module B:X2;         // does not implicitly import B
import B;
int &b = n;           // OK
```

Translation unit #5:

```
module B;             // implicitly imports B
int &c = n;           // OK
```

— *end example*]

10.2 Export declaration

[**module.interface**]

export-declaration:

```
export name-declaration
export { declaration-seqopt }
export-keyword module-import-declaration
```

- ¹ An *export-declaration* shall inhabit a namespace scope and appear in the purview of a module interface unit. An *export-declaration* shall not appear directly or indirectly within an unnamed namespace or a *private-module-fragment*. An *export-declaration* has the declarative effects of its *name-declaration*, *declaration-seq* (if any), or *module-import-declaration*. The *name-declaration* of an *export-declaration* shall not declare a partial specialization (13.7.1). The *declaration-seq* of an *export-declaration* shall not contain an *export-declaration* or *module-import-declaration*.

[Note 1: An *export-declaration* does not establish a scope. — end note]

- ² A declaration is *exported* if it is declared within an *export-declaration* and inhabits a namespace scope or it is
- (2.1) — a *namespace-definition* that contains an exported declaration, or
- (2.2) — a declaration within a header unit (10.3) that introduces at least one name.
- ³ If an exported declaration is not within a header unit, it shall not declare a name with internal linkage.
- ⁴ [Example 1:

Source file "a.h":

```
export int x;
```

Translation unit #1:

```
module;
#include "a.h"                // error: declaration of x is not in the
                              // purview of a module interface unit

export module M;
export namespace {}          // error: namespace has internal linkage
namespace {
    export int a2;            // error: export of name with internal linkage
}
export static int b;          // error: b explicitly declared static
export int f();               // OK
export namespace N { }       // OK
export using namespace N;     // OK
```

— end example]

- ⁵ If an exported declaration is a *using-declaration* (9.10) and is not within a header unit, all entities named by the *using-declarators* (if any) shall either be a type alias or have been introduced with a name having external linkage.

[Example 2:

Source file "b.h":

```
int f();
```

Importable header "c.h":

```
int g();
```

Translation unit #1:

```
export module X;
export int h();
```

Translation unit #2:

```
module;
#include "b.h"
export module M;
import "c.h";
import X;
export using ::f, ::g, ::h;    // OK
struct S;
export using ::S;              // error: S has module linkage
namespace N {
    export int h();
    static int h(int);         // #1
}
export using N::h;             // error: #1 has internal linkage
```

— end example]

[Note 2: The underlying entity of an exported type alias need not have a name with external linkage.

[Example 3:

```
export module M;
struct S;
export using T = S;           // OK, exports name T denoting type S
```

— end example]

— end note]

- ⁶ A redeclaration of an entity *X* is implicitly exported if *X* was introduced by an exported declaration; otherwise it shall not be exported unless it is a type alias, a namespace, or a namespace alias.

[Example 4:

```
export module M;
struct S { int n; };
typedef S S;
export typedef S S;           // OK
export struct S;               // error: exported declaration follows non-exported declaration
namespace N {                  // external linkage, attached to global module, not exported
    void f();
}
namespace N {                  // OK, exported namespace redeclaring non-exported namespace
    export void g();
}
```

— end example]

- ⁷ [Note 3: Names introduced by exported declarations never have module linkage. They have external linkage, no linkage, or (in the case of header units) internal linkage; see 6.7. Namespace-scope declarations exported by a module can be found by name lookup in any translation unit importing that module (6.5). Class and enumeration member names can be found by name lookup in any context in which a definition of the type is reachable. — end note]

[Example 5:

Interface unit of M:

```
export module M;
export struct X {
    static void f();
    struct Y { };
};

namespace {
    struct S { };
}
export void f(S);              // OK
struct T { };
export T id(T);                // OK

export struct A;               // A exported as incomplete

export auto rootFinder(double a) {
    return [=](double x) { return (x + a/x)/2; };
}

export const int n = 5;        // OK, n has external linkage
```

Implementation unit of M:

```
module M;
struct A {
    int value;
};
```

Main program:

```
import M;
```

```

int main() {
    X::f();           // OK, X is exported and definition of X is reachable
    X::Y y;          // OK, X::Y is exported as a complete type
    auto f = rootFinder(2); // OK
    return A{45}.value; // error: A is incomplete
}

```

— end example]

- ⁸ [Note 4: Declarations in an exported *namespace-definition* or in an exported *linkage-specification* (9.12) are exported and subject to the rules of exported declarations.

[Example 6:

```

export module M;
int g;
export namespace N {
    int x;           // OK
    using ::g;       // error: ::g has module linkage
}

```

— end example]

— end note]

10.3 Import declaration

[module.import]

module-import-declaration:

```

import-keyword module-name attribute-specifier-seqopt ;
import-keyword module-partition attribute-specifier-seqopt ;
import-keyword header-name attribute-specifier-seqopt ;

```

- ¹ A *module-import-declaration* shall inhabit the global namespace scope. In a module unit, all *module-import-declarations* and *export-declarations* exporting *module-import-declarations* shall appear before all other declarations in the *declaration-seq* of the *translation-unit* and of the *private-module-fragment* (if any). The optional *attribute-specifier-seq* appertains to the *module-import-declaration*.
- ² A *module-import-declaration* *imports* a set of translation units determined as described below.
- [Note 1: Namespace-scope declarations exported by the imported translation units can be found by name lookup (6.5) in the importing translation unit and declarations within the imported translation units become reachable (10.7) in the importing translation unit after the import declaration. — end note]
- ³ A *module-import-declaration* that specifies a *module-name* M imports all module interface units of M.
- ⁴ A *module-import-declaration* that specifies a *module-partition* shall only appear after the *module-declaration* in a module unit of some module M. Such a declaration imports the so-named module partition of M.
- ⁵ A *module-import-declaration* that specifies a *header-name* H imports a synthesized *header unit*, which is a translation unit formed by applying phases 1 to 7 of translation (5.2) to the source file or header nominated by H, which shall not contain a *module-declaration*.

[Note 2: A header unit is a separate translation unit with an independent set of defined macros. All declarations within a header unit are implicitly exported (10.2), and are attached to the global module (10.1). — end note]

An *importable header* is a member of an implementation-defined set of headers that includes all importable C++ library headers (16.4.2.3). H shall identify an importable header. Given two such *module-import-declarations*:

- (5.1) — if their *header-names* identify different headers or source files (15.3), they import distinct header units;
- (5.2) — otherwise, if they appear in the same translation unit, they import the same header unit;
- (5.3) — otherwise, it is unspecified whether they import the same header unit.

[Note 3: It is therefore possible that multiple copies exist of entities declared with internal linkage in an importable header. — end note]

[Note 4: A *module-import-declaration* nominating a *header-name* is also recognized by the preprocessor, and results in macros defined at the end of phase 4 of translation of the header unit being made visible as described in 15.6. Any other *module-import-declaration* does not make macros visible. — end note]

- ⁶ A declaration of a name with internal linkage is permitted within a header unit despite all declarations being implicitly exported (10.2).

[Note 5: A definition that appears in multiple translation units cannot in general refer to such names (6.3). — end note]

A header unit shall not contain a definition of a non-inline function or variable whose name has external linkage.

- ⁷ When a *module-import-declaration* imports a translation unit *T*, it also imports all translation units imported by exported *module-import-declarations* in *T*; such translation units are said to be *exported* by *T*. Additionally, when a *module-import-declaration* in a module unit of some module *M* imports another module unit *U* of *M*, it also imports all translation units imported by non-exported *module-import-declarations* in the module unit purview of *U*.⁷⁸ These rules can in turn lead to the importation of yet more translation units.

[Note 6: Such indirect importation does not make macros available, because a translation unit is a sequence of tokens in translation phase 7 (5.2). Macros can be made available by directly importing header units as described in 15.6. — end note]

- ⁸ A module implementation unit shall not be exported.

[Example 1:

Translation unit #1:

```
module M:Part;
```

Translation unit #2:

```
export module M;
export import :Part;    // error: exported partition :Part is an implementation unit
```

— end example]

- ⁹ A module implementation unit of a module *M* that is not a module partition shall not contain a *module-import-declaration* nominating *M*.

[Example 2:

```
module M;
import M;                // error: cannot import M in its own unit
```

— end example]

- ¹⁰ A translation unit has an *interface dependency* on a translation unit *U* if it contains a declaration (possibly a *module-declaration*) that imports *U* or if it has an interface dependency on a translation unit that has an interface dependency on *U*. A translation unit shall not have an interface dependency on itself.

[Example 3:

Interface unit of M1:

```
export module M1;
import M2;
```

Interface unit of M2:

```
export module M2;
import M3;
```

Interface unit of M3:

```
export module M3;
import M1;                // error: cyclic interface dependency M3 → M1 → M2 → M3
```

— end example]

10.4 Global module fragment

[module.global.frag]

global-module-fragment:

module-keyword ; *declaration-seq_{opt}*

- ¹ A *global-module-fragment* specifies the contents of the *global module fragment* for a module unit. The global module fragment can be used to provide declarations that are attached to the global module and usable within the module unit.
- ² A declaration *D* is *decl-reachable* from a declaration *S* in the same translation unit if
- (2.1) — *D* does not declare a function or function template and *S* contains an *id-expression*, *namespace-name*, *type-name*, *template-name*, or *concept-name* naming *D*, or

⁷⁸) This is consistent with the lookup rules for imported names (6.5).

- (2.2) — D declares a function or function template that is named by an expression (6.3) appearing in S , or
- (2.3) — S contains a dependent call E (13.8.3) and D is found by any name lookup performed for an expression synthesized from E by replacing each type-dependent argument or operand with a value of a placeholder type with no associated namespaces or entities, or
 [Note 1: This includes the lookup for `operator==` performed when considering rewriting an `!=` expression, the lookup for `operator<=>` performed when considering rewriting a relational comparison, and the lookup for `operator!=` when considering whether an `operator==` is a rewrite target. — end note]
- (2.4) — S contains an expression that takes the address of an overload set (12.3) that contains D and for which the target type is dependent, or
- (2.5) — there exists a declaration M that is not a *namespace-definition* for which M is decl-reachable from S and either
 - (2.5.1) — D is decl-reachable from M , or
 - (2.5.2) — D and M declare the same entity, and D neither is a friend declaration nor inhabits a block scope, or
 - (2.5.3) — D declares a namespace N and M is a member of N , or
 - (2.5.4) — one of D and M declares a class or class template C and the other declares a member or friend of C , or
 - (2.5.5) — one of D and M declares an enumeration E and the other declares an enumerator of E , or
 - (2.5.6) — D declares a function or variable and M is declared in D ,⁷⁹ or
 - (2.5.7) — one of D and M declares a template and the other declares a partial or explicit specialization or an implicit or explicit instantiation of that template, or
 - (2.5.8) — M declares a class template and D is a deduction guide for that template, or
 - (2.5.9) — one of D and M declares a class or enumeration type and the other introduces a typedef name for linkage purposes for that type.

In this determination, it is unspecified

- (2.6) — whether a reference to an *alias-declaration*, *typedef* declaration, *using-declaration*, or *namespace-alias-definition* is replaced by the declarations they name prior to this determination,
- (2.7) — whether a *simple-template-id* that does not denote a dependent type and whose *template-name* names an alias template is replaced by its denoted type prior to this determination,
- (2.8) — whether a *decltype-specifier* that does not denote a dependent type is replaced by its denoted type prior to this determination,
- (2.9) — whether a non-value-dependent constant expression is replaced by the result of constant evaluation prior to this determination, and
- (2.10) — whether a *splice-expression*, a *splice-type-specifier*, a *splice-scope-specifier*, or any *splice-specifier* or *splice-specialization-specifier* outside of the preceding is replaced in any non-dependent context by the construct that it designates prior to this determination.

- ³ A declaration D in a global module fragment of a module unit is *discarded* if D is not decl-reachable from any *declaration* in the *declaration-seq* of the *translation-unit*.

[Note 2: A discarded declaration is neither reachable nor visible to name lookup outside the module unit, nor in template instantiations whose points of instantiation (13.8.4.1) are outside the module unit, even when the instantiation context (10.6) includes the module unit. — end note]

- ⁴ [Example 1:

```

const int size = 2;
int ary1[size];           // unspecified whether size is decl-reachable from ary1
constexpr int identity(int x) { return x; }
int ary2[identity(2)];     // unspecified whether identity is decl-reachable from ary2

template<typename> struct S;
template<typename, int> struct S2;
constexpr int g(int);

```

⁷⁹) A declaration can appear within a *lambda-expression* in the initializer of a variable.


```

template<typename T, int N>
S<S2<T, g(N)>> f();           // S, S2, g, and :: are decl-reachable from f

template<int N>
void h() noexcept(g(N) == N); // g and :: are decl-reachable from h
— end example]

```

⁵ [Example 2:

Source file "foo.h":

```

namespace N {
    struct X {};
    int d();
    int e();
    inline int f(X, int = d()) { return e(); }
    int g(X);
    int h(X);
}

```

Module M interface:

```

module;
#include "foo.h"
export module M;
template<typename T> int use_f() {
    N::X x;           // N::X, N, and :: are decl-reachable from use_f
    return f(x, 123); // N::f is decl-reachable from use_f,
                     // N::e is indirectly decl-reachable from use_f
                     // because it is decl-reachable from N::f, and
                     // N::d is decl-reachable from use_f
                     // because it is decl-reachable from N::f
                     // even though it is not used in this call
}
template<typename T> int use_g() {
    N::X x;           // N::X, N, and :: are decl-reachable from use_g
    return g((T()), x); // N::g is not decl-reachable from use_g
}
template<typename T> int use_h() {
    N::X x;           // N::X, N, and :: are decl-reachable from use_h
    return h((T()), x); // N::h is not decl-reachable from use_h, but
                       // N::h is decl-reachable from use_h<int>
}
int k = use_h<int>();
// use_h<int> is decl-reachable from k, so
// N::h is decl-reachable from k

```

Module M implementation:

```

module M;
int a = use_f<int>(); // OK
int b = use_g<int>(); // error: no viable function for call to g;
                     // g is not decl-reachable from purview of
                     // module M's interface, so is discarded

int c = use_h<int>(); // OK
— end example]

```

10.5 Private module fragment

[module.private.frag]

private-module-fragment:

module-keyword : private ; *declaration-seq_{opt}*

- ¹ A *private-module-fragment* shall appear only in a primary module interface unit (10.1). A module unit with a *private-module-fragment* shall be the only module unit of its module; no diagnostic is required (G.7.4).
- ² [Note 1: A *private-module-fragment* ends the portion of the module interface unit that can affect the behavior of other translation units. A *private-module-fragment* allows a module to be represented as a single translation unit without making all of the contents of the module reachable to importers. The presence of a *private-module-fragment* affects:

- (2.1) — the point by which the definition of an inline function or variable is required (9.2.8),
- (2.2) — the point by which the definition of an exported function with a placeholder return type is required (9.2.9.7),
- (2.3) — whether a declaration is required not to be an exposure (6.7),
- (2.4) — where definitions for inline functions and templates must appear (6.3, 9.2.8, 13.1),
- (2.5) — the instantiation contexts of templates instantiated before it (10.6), and
- (2.6) — the reachability of declarations within it (10.7).

— *end note*

3 [Example 1:

```
export module A;
export inline void fn_e();           // error: exported inline function fn_e not defined
                                     // before private module fragment

inline void fn_m();                 // error: non-exported inline function fn_m not defined
static void fn_s();

export struct X;
export void g(X *x) {
    fn_s();                         // OK, call to static function in same translation unit
}
export X *factory();                // OK

module :private;
struct X {};                       // definition not reachable from importers of A
X *factory() {
    return new X ();
}
void fn_e() {}
void fn_m() {}
void fn_s() {}
```

— *end example*

10.6 Instantiation context

[module.context]

- 1 The *instantiation context* is a set of points within the program that determines which declarations are found by argument-dependent name lookup (6.5.4) and which are reachable (10.7) in the context of a particular declaration or template instantiation.
- 2 During the implicit definition of a defaulted function (11.4.4, 11.10.1), the instantiation context contains each point in the instantiation context from the definition of the class and each point in the instantiation context of the program construct that resulted in the implicit definition of the defaulted function.
- 3 During the implicit instantiation of a template whose point of instantiation is specified as that of an enclosing specialization (13.8.4.1), the instantiation context contains each point in the instantiation context of the enclosing specialization and, if the template is defined in a module interface unit of a module *M* and the point of instantiation is not in a module interface unit of *M*, the point at the end of the *declaration-seq* of the primary module interface unit of *M* (prior to the *private-module-fragment*, if any).
- 4 During the implicit instantiation of a template that is implicitly instantiated because it is referenced from within the implicit definition of a defaulted function, the instantiation context contains each point in the instantiation context of the defaulted function.
- 5 During the instantiation of any other template specialization, the instantiation context contains the point of instantiation of the template.
- 6 During the implicit instantiation of any construct that resulted from the evaluation of an expression as a core constant expression, the instantiation context contains each point in the evaluation context (7.7.6).
 [Note 1: Implicit instantiations can result from invocations of library functions (21.4). The evaluation context can include synthesized points associated with injected declarations produced by `std::meta::define_aggregate` (21.4.17).
 — *end note*
- 7 In any other case, the instantiation context at a point within the program contains that point.
- 8 The instantiation context contains only the points specified above.

⁹ [Example 1:

Translation unit #1:

```
export module stuff;
export template<typename T, typename U> void foo(T, U u) { auto v = u; }
export template<typename T, typename U> void bar(T, U u) { auto v = *u; }
```

Translation unit #2:

```
export module M1;
import "defn.h";          // provides struct X {};
import stuff;
export template<typename T> void f(T t) {
    X x;
    foo(t, x);
}
```

Translation unit #3:

```
export module M2;
import "decl.h";          // provides struct X; (not a definition)
import stuff;
export template<typename T> void g(T t) {
    X *x;
    bar(t, x);
}
```

Translation unit #4:

```
import M1;
import M2;
void test() {
    f(0);
    g(0);
}
```

The call to `f(0)` is valid; the instantiation context of `foo<int, X>` comprises

- (9.1) — the point at the end of translation unit #1,
- (9.2) — the point at the end of translation unit #2, and
- (9.3) — the point of the call to `f(0)`,

so the definition of `X` is reachable (10.7).

It is unspecified whether the call to `g(0)` is valid: the instantiation context of `bar<int, X>` comprises

- (9.4) — the point at the end of translation unit #1,
- (9.5) — the point at the end of translation unit #3, and
- (9.6) — the point of the call to `g(0)`,

so the definition of `X` need not be reachable, as described in 10.7. — end example]

10.7 Reachability

[module.reach]

- ¹ A translation unit *U* is *necessarily reachable* from a point *P* if *U* is a module interface unit on which the translation unit containing *P* has an interface dependency, or the translation unit containing *P* imports *U*, in either case prior to *P* (10.3).

[Note 1: While module interface units are reachable even when they are only transitively imported via a non-exported import declaration, namespace-scope names from such module interface units are not found by name lookup (6.5). — end note]

- ² All translation units that are necessarily reachable are *reachable*. Additional translation units on which the point within the program has an interface dependency may be considered reachable, but it is unspecified which are and under what circumstances.⁸⁰

[Note 2: It is advisable to avoid depending on the reachability of any additional translation units in programs intending to be portable. — end note]

⁸⁰) Implementations are therefore not required to prevent the semantic effects of additional translation units involved in the compilation from being observed.

³ A declaration *D* is *reachable* from a point *P* if

- (3.1) — *P* is a non-synthesized point and
 - (3.1.1) — *D* appears prior to *P* in the same translation unit, or
 - (3.1.2) — *D* is not discarded (10.4), appears in a translation unit that is reachable from *P*, and does not appear within a *private-module-fragment*; or
- (3.2) — *D* is the injected declaration for which *P* is the corresponding synthesized point.

[Example 1:

```
class Incomplete;

consteval {
  int n = nonstatic_data_members_of(
    define_aggregate(~~Incomplete, {data_member_spec(~~int, {.name="x"})}),
    std::meta::access_context::current()
  ).size();

  Incomplete y;          // error: type of y is incomplete
}
/* P */
```

The value of *n* is 1. The member `Incomplete::x` members-of-precedes (21.4.11) the synthesized point *P* associated with the injected declaration produced by the call to `define_aggregate`. — end example]

⁴ A declaration is *reachable* if it is reachable from any point in the instantiation context (10.6).

[Note 3: Whether a declaration is exported has no bearing on whether it is reachable. — end note]

⁵ The accumulated properties of all reachable declarations of an entity within a context determine the behavior of the entity within that context.

[Note 4: These reachable semantic properties include type completeness, type definitions, initializers, default arguments of functions or template declarations, attributes, names bound, etc. Since default arguments are evaluated in the context of the call expression, the reachable semantic properties of the corresponding parameter types apply in that context.

[Example 2:

Translation unit #1:

```
export module M:A;
export struct B;
```

Translation unit #2:

```
module M:B;
struct B {
  operator int();
};
```

Translation unit #3:

```
module M:C;
import :A;
B b1;          // error: no reachable definition of struct B
```

Translation unit #4:

```
export module M;
export import :A;
import :B;
B b2;
export void f(B b = B());
```

Translation unit #5:

```
import M;
B b3;          // error: no reachable definition of struct B
void g() { f(); } // error: no reachable definition of struct B
```

— end example]

— *end note*]

- ⁶ [*Note 5*: Declarations of an entity can be reachable even where they cannot be found by name lookup. — *end note*]

[*Example 3*:

Translation unit #1:

```
export module A;
struct X {};
export using Y = X;
```

Translation unit #2:

```
import A;
Y y;           // OK, definition of X is reachable
X x;           // error: X not visible to unqualified lookup
```

— *end example*]

11 Classes

[class]

11.1 Preamble

[class.pre]

- ¹ A class is a type. Its name becomes a *class-name* (11.3) within its scope.

class-name:
identifier
simple-template-id

A *class-specifier* or an *elaborated-type-specifier* (9.2.9.5) is used to make a *class-name*. An object of a class consists of a (possibly empty) sequence of members and base class objects.

class-specifier:
class-head { *member-specification*_{opt} }

class-head:
class-key *attribute-specifier-seq*_{opt} *class-head-name* *class-property-specifier-seq*_{opt} *base-clause*_{opt}
class-key *attribute-specifier-seq*_{opt} *base-clause*_{opt}

class-head-name:
*nested-name-specifier*_{opt} *class-name*

class-property-specifier-seq:
class-property-specifier *class-property-specifier-seq*_{opt}

class-property-specifier:
final

class-key:
class
struct
union

A class declaration where the *class-name* in the *class-head-name* is a *simple-template-id* shall be an explicit specialization (13.9.4) or a partial specialization (13.7.6). A *class-specifier* whose *class-head* omits the *class-head-name* defines an *unnamed class*.

[Note 1: An unnamed class thus can't be **final**. — end note]

Otherwise, the *class-name* is an *identifier*; it is not looked up, and the *class-specifier* introduces it.

- ² The component name of the *class-name* is also bound in the scope of the class (template) itself; this is known as the *injected-class-name*. For purposes of access checking, the injected-class-name is treated as if it were a public member name. A *class-specifier* is commonly referred to as a *class definition*. A class is considered defined after the closing brace of its *class-specifier* has been seen even though its member functions are in general not yet defined. The optional *attribute-specifier-seq* appertains to the class; the attributes in the *attribute-specifier-seq* are thereafter considered attributes of the class whenever it is named.
- ³ If a *class-head-name* contains a *nested-name-specifier*, the *class-specifier* shall not inhabit a class scope. If its *class-name* is an *identifier*, the *class-specifier* shall correspond to one or more declarations nominable in the class, class template, or namespace to which the *nested-name-specifier* refers; they shall all have the same target scope, and the target scope of the *class-specifier* is that scope.

[Example 1:

```
namespace N {
    template<class>
    struct A {
        struct B;
    };
}
using N::A;
template<class T> struct A<T>::B {};    // OK
template<> struct A<void> {};          // OK
```

— end example]

- ⁴ [Note 2: The *class-key* determines whether the class is a union (11.5) and whether access is public or private by default (11.8). A union holds the value of at most one data member at a time. — end note]
- ⁵ Each *class-property-specifier* shall appear at most once within a single *class-property-specifier-seq*. Whenever a *class-key* is followed by a *class-head-name*, the identifier **final**, and a colon or left brace, the identifier is interpreted as a *class-property-specifier*.

[Example 2:

```
struct A;
struct A final {};           // OK, definition of struct A,
                             // not value-initialization of variable final

struct X {
    struct C { constexpr operator int() { return 5; } };
    struct B final : C{};    // OK, definition of nested class B,
                             // not declaration of a bit-field member final
};
```

— end example]

- ⁶ If a class is marked with the *class-property-specifier* **final** and that class appears as a *class-or-decltype* in a *base-clause* (11.7), the program is ill-formed.
- ⁷ [Note 3: Complete objects of class type have nonzero size. Base class subobjects and members declared with the **no_unique_address** attribute (9.13.11) are not so constrained. — end note]
- ⁸ [Note 4: Class objects can be assigned (12.4.3.2, 11.4.6), passed as arguments to functions (9.5, 11.4.5.3), and returned by functions (except objects of classes for which copying or moving has been restricted; see 9.6.3 and 11.8). Other plausible operators, such as equality comparison, can be defined by the user; see 12.4. — end note]

11.2 Properties of classes

[**class.prop**]

- ¹ A *trivially copyable class* is a class:

- (1.1) — that has at least one eligible copy constructor, move constructor, copy assignment operator, or move assignment operator (11.4.4, 11.4.5.3, 11.4.6),
- (1.2) — where each eligible copy constructor, move constructor, copy assignment operator, and move assignment operator is trivial, and
- (1.3) — that has a trivial, non-deleted destructor (11.4.7).

- ² A class **S** is a *standard-layout class* if it:

- (2.1) — has no non-static data members of type non-standard-layout class (or array of such types) or reference,
- (2.2) — has no virtual functions (11.7.3) and no virtual base classes (11.7.2),
- (2.3) — has the same access control (11.8) for all non-static data members,
- (2.4) — has no non-standard-layout base classes,
- (2.5) — has at most one base class subobject of any given type,
- (2.6) — has all non-static data members and bit-fields in the class and its base classes first declared in the same class, and
- (2.7) — has no element of the set $M(\mathbf{S})$ of types as a base class, where for any type **X**, $M(\mathbf{X})$ is defined as follows.⁸¹

[Note 1: $M(\mathbf{X})$ is the set of the types of all non-base-class subobjects that can be at a zero offset in **X**. — end note]

- (2.7.1) — If **X** is a non-union class type with no non-static data members, the set $M(\mathbf{X})$ is empty.
- (2.7.2) — If **X** is a non-union class type with a non-static data member of type $\mathbf{X}_0$ that is either of zero size or is the first non-static data member of **X** (where said member may be an anonymous union), the set $M(\mathbf{X})$ consists of $\mathbf{X}_0$ and the elements of $M(\mathbf{X}_0)$.
- (2.7.3) — If **X** is a union type, the set $M(\mathbf{X})$ is the union of all $M(\mathbf{U}_i)$ and the set containing all $\mathbf{U}_i$, where each $\mathbf{U}_i$ is the type of the i^{th} non-static data member of **X**.

⁸¹) This ensures that two subobjects that have the same class type and that belong to the same most derived object are not allocated at the same address (7.6.10).

- (2.7.4) — If X is an array type with element type X_e , the set $M(X)$ consists of X_e and the elements of $M(X_e)$.
- (2.7.5) — If X is a non-class, non-array type, the set $M(X)$ is empty.

³ [Example 1:

```
struct B { int i; };           // standard-layout class
struct C : B { };             // standard-layout class
struct D : C { };             // standard-layout class
struct E : D { char : 4; };    // not a standard-layout class

struct Q {};
struct S : Q { };
struct T : Q { };
struct U : S, T { };           // not a standard-layout class
```

— end example]

- ⁴ A *standard-layout struct* is a standard-layout class defined with the *class-key* **struct** or the *class-key* **class**. A *standard-layout union* is a standard-layout class defined with the *class-key* **union**.
- ⁵ [Note 2: Standard-layout classes are useful for communicating with code written in other programming languages. Their layout is specified in 11.4.1 and 7.6.9. — end note]

⁶ [Example 2:

```
struct N {                     // neither trivially copyable nor standard-layout
    int i;
    int j;
    virtual ~N();
};

struct T {                     // trivially copyable but not standard-layout
    int i;
private:
    int j;
};

struct SL {                    // standard-layout but not trivially copyable
    int i;
    int j;
    ~SL();
};

struct POD {                   // both trivially copyable and standard-layout
    int i;
    int j;
};
```

— end example]

- ⁷ [Note 3: Aggregates of class type are described in 9.5.2. — end note]

⁸ A class **S** is an *implicit-lifetime class* if

- (8.1) — it is an aggregate whose destructor is not user-provided or
- (8.2) — it has at least one trivial eligible constructor and a trivial, non-deleted destructor.

11.3 Class names

[class.name]

¹ A class definition introduces a new type.

[Example 1:

```
struct X { int a; };
struct Y { int a; };
X a1;
Y a2;
int a3;
```

declares three variables of three different types. This implies that

```
a1 = a2;                       // error: Y assigned to X
```



```
a1 = a3;                                // error: int assigned to X
```

are type mismatches, and that

```
int f(X);
int f(Y);
```

declare overloads (Clause 12) named `f` and not simply a single function `f` twice. For the same reason,

```
struct S { int a; };
struct S { int a; };                    // error: double definition
```

is ill-formed because it defines `S` twice. — end example]

- ² [Note 1: It can be necessary to use an *elaborated-type-specifier* to refer to a class that belongs to a scope in which its name is also bound to a variable, function, or enumerator (6.5.6).

[Example 2:

```
struct stat {
    // ...
};

stat gstat;                            // use plain stat to define variable

int stat(struct stat*);                // stat now also names a function

void f() {
    struct stat* ps;                   // struct prefix needed to name struct stat
    stat(ps);                          // call stat function
}
```

— end example]

An *elaborated-type-specifier* can also be used to declare an *identifier* as a *class-name*.

[Example 3:

```
struct s { int a; };

void g() {
    struct s;                          // hide global struct s with a block-scope declaration
    s* p;                              // refer to local struct s
    struct s { char* p; };             // define local struct s
    struct s;                          // redeclaration, has no effect
}
```

— end example]

Such declarations allow definition of classes that refer to each other.

[Example 4:

```
class Vector;

class Matrix {
    // ...
    friend Vector operator*(const Matrix&, const Vector&);
};

class Vector {
    // ...
    friend Vector operator*(const Matrix&, const Vector&);
};
```

Declaration of friends is described in 11.8.4, operator functions in 12.4. — end example]

— end note]

- ³ [Note 2: An *elaborated-type-specifier* (9.2.9.5) can also be used as a *type-specifier* as part of a declaration. It differs from a class declaration in that it can refer to an existing class of the given name. — end note]

[Example 5:

```
struct s { int a; };
```

```
void g(int s) {
    struct s* p = new struct s;    // global s
    p->a = s;                      // parameter s
}
```

— end example]

- ⁴ [Note 3: The declaration of a class name takes effect immediately after the *identifier* is seen in the class definition or *elaborated-type-specifier*.

[Example 6:

```
class A * A;
```

first specifies **A** to be the name of a class and then redefines it as the name of a pointer to an object of that class. This means that the elaborated form **class A** must be used to refer to the class. Such artistry with names can be confusing and is best avoided. — end example]

— end note]

- ⁵ A *simple-template-id* is only a *class-name* if its *template-name* names a class template.

11.4 Class members

[class.mem]

11.4.1 General

[class.mem.general]

member-specification:

member-declaration *member-specification*_{opt}

access-specifier : *member-specification*_{opt}

member-declaration:

*attribute-specifier-seq*_{opt} *decl-specifier-seq*_{opt} *member-declarator-list*_{opt} ;

function-definition

friend-type-declaration

using-declaration

using-enum-declaration

static_assert-declaration

constexpr-block-declaration

template-declaration

explicit-specialization

deduction-guide

alias-declaration

opaque-enum-declaration

empty-declaration

member-declarator-list:

member-declarator

member-declarator-list , *member-declarator*

member-declarator:

declarator *virt-specifier-seq*_{opt} *function-contract-specifier-seq*_{opt} *pure-specifier*_{opt}

declarator *requires-clause* *function-contract-specifier-seq*_{opt}

declarator *brace-or-equal-initializer*

*identifier*_{opt} *attribute-specifier-seq*_{opt} : *constant-expression* *brace-or-equal-initializer*_{opt}

virt-specifier-seq:

virt-specifier *virt-specifier-seq*_{opt}

virt-specifier:

override

final

pure-specifier:

= 0

friend-type-declaration:

friend *friend-type-specifier-list* ;

friend-type-specifier-list:

friend-type-specifier ..._{opt}

friend-type-specifier-list , *friend-type-specifier* ..._{opt}

friend-type-specifier:
simple-type-specifier
elaborated-type-specifier
typename-specifier

- ¹ In the absence of a *virt-specifier-seq*, the token sequence = 0 is treated as a *pure-specifier* if the type of the *declarator-id* (9.3.4.1) is a function type, and is otherwise treated as a *brace-or-equal-initializer*.

[*Note 1:* If the member declaration acquires a function type through template instantiation, the program is ill-formed; see 13.9.1. — *end note*]

- ² The optional *function-contract-specifier-seq* (9.4.1) in a *member-declarator* shall be present only if the *declarator* declares a function.
- ³ The *member-specification* in a class definition declares the full set of members of the class; no member can be added elsewhere. A *direct member* of a class *X* is a member of *X* that was first declared within the *member-specification* of *X*, including anonymous union members (11.5.2) and direct members thereof. Members of a class are data members, member functions (11.4.2), nested types, enumerators, and member templates (13.7.3) and specializations thereof.

[*Note 2:* A specialization of a static data member template is a static data member. A specialization of a member function template is a member function. A specialization of a member class template is a nested class. — *end note*]

- ⁴ A *member-declaration* does not itself declare new members of the class if it is

- (4.1) — a friend declaration (11.8.4),
- (4.2) — a *deduction-guide* (13.7.2.3),
- (4.3) — a *template-declaration* whose *declaration* is one of the above,
- (4.4) — a *static_assert-declaration*,
- (4.5) — a *constexpr-block-declaration*,
- (4.6) — a *using-declaration* (9.10), or
- (4.7) — an *empty-declaration*.

For any other *member-declaration*, each declared entity that is not an unnamed bit-field (11.4.10) is a member of the class, and each such *member-declaration* shall either declare at least one member name of the class or declare at least one unnamed bit-field. A *user-declared* entity is a direct member or a friend that, in either case, is declared by a *member-declaration*.

- ⁵ A *data member* is either a non-function member introduced by a *member-declarator* or an anonymous union member. A *member function* is a member that is a function. Nested types are classes (11.3, 11.4.12) and enumerations (9.8.1) declared in the class and arbitrary types declared as members by use of a typedef declaration (9.2.4) or *alias-declaration*. The enumerators of an unscoped enumeration (9.8.1) defined in the class are members of the class.
- ⁶ A data member or member function may be declared **static** in its *member-declaration*, in which case it is a *static member* (see 11.4.9) (a *static data member* (11.4.9.3) or *static member function* (11.4.9.2), respectively) of the class. Any other data member or member function is a *non-static member* (a *non-static data member* or *non-static member function* (11.4.3), respectively).
- ⁷ Every object of class type has a unique member subobject corresponding to each of its direct non-static data members. If any non-static data member of a class *C* is of reference type, then let *D* be an invented class that is identical to *C* except that each non-static member of *D* corresponding to a member of *C* of type “reference to *T*” instead has type “pointer to *T*”. Every member subobject of a complete object of type *C* has the same size, alignment, and offset as that of the corresponding subobject of a complete object of type *D*. The size and alignment of *C* are the same as the size and alignment of *D*.
- ⁸ A member shall not be declared twice in the *member-specification*, except that
- (8.1) — a nested class or member class template can be declared and then later defined, and
 - (8.2) — an enumeration can be introduced with an *opaque-enum-declaration* and later redeclared with an *enum-specifier*.

[*Note 3:* A single name can denote several member functions provided their types are sufficiently different (6.4.1). — *end note*]

⁹ A redeclaration of a class member outside its class definition shall be a definition, an explicit specialization, or an explicit instantiation (13.9.4, 13.9.3). The member shall not be a non-static data member.

¹⁰ A *complete-class context* of a class (template) is a

- (10.1) — function body (9.6.1),
- (10.2) — default argument (9.3.4.7),
- (10.3) — default template argument (13.2),
- (10.4) — *noexcept-specifier* (14.5),
- (10.5) — *function-contract-specifier* (9.4.1), or
- (10.6) — default member initializer

within the *member-specification* of the class or class template.

[*Note 4*: A complete-class context of a nested class is also a complete-class context of any enclosing class, if the nested class is defined within the *member-specification* of the enclosing class. — *end note*]

¹¹ A class *C* is complete at a program point *P* if the definition of *C* is reachable from *P* (10.7) or if *P* is in a complete-class context of *C*. Otherwise, *C* is incomplete at *P*.

¹² If a *member-declaration* matches the syntactic requirements of *friend-type-declaration*, it is a *friend-type-declaration*.

¹³ In a *member-declarator*, an = immediately following the *declarator* is interpreted as introducing a *pure-specifier* if the *declarator-id* has function type, otherwise it is interpreted as introducing a *brace-or-equal-initializer*.

[*Example 1*:

```
struct S {
    using T = void();
    T * p = 0;           // OK, brace-or-equal-initializer
    virtual T f = 0;     // OK, pure-specifier
};
```

— *end example*]

¹⁴ In a *member-declarator* for a bit-field, the *constant-expression* is parsed as the longest sequence of tokens that could syntactically form a *constant-expression*.

[*Example 2*:

```
int a;
const int b = 0;
struct S {
    int x1 : 8 = 42;           // OK, "= 42" is brace-or-equal-initializer
    int x2 : 8 { 42 };         // OK, "{ 42 }" is brace-or-equal-initializer
    int y1 : true ? 8 : a = 42; // OK, brace-or-equal-initializer is absent
    int y2 : true ? 8 : b = 42; // error: cannot assign to const int
    int y3 : (true ? 8 : b) = 42; // OK, "= 42" is brace-or-equal-initializer
    int z : 1 || new int { 0 }; // OK, brace-or-equal-initializer is absent
};
```

— *end example*]

¹⁵ A *brace-or-equal-initializer* shall appear only in the declaration of a data member. (For static data members, see 11.4.9.3; for non-static data members, see 11.9.3 and 9.5.2). A *brace-or-equal-initializer* for a non-static data member specifies a *default member initializer* for the member, and shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor. An immediate invocation (7.7.5) that is a potentially-evaluated subexpression (6.10.1) of a default member initializer is neither evaluated nor checked for whether it is a constant expression at the point where the subexpression appears.

¹⁶ A member shall not be declared with the *extern storage-class-specifier*. Within a class definition, a member shall not be declared with the *thread_local storage-class-specifier* unless also declared *static*.

¹⁷ The *decl-specifier-seq* may be omitted in constructor, destructor, and conversion function declarations only; when declaring another kind of member the *decl-specifier-seq* shall contain a *type-specifier* that is not a *cv-qualifier*. The *member-declarator-list* can be omitted only after a *class-specifier* or an *enum-specifier* or in a

friend declaration (11.8.4). A *pure-specifier* shall be used only in the declaration of a virtual function (11.7.3) that is not a friend declaration.

18 The optional *attribute-specifier-seq* in a *member-declaration* appertains to each of the entities declared by the *member-declarators*; it shall not appear if the optional *member-declarator-list* is omitted.

19 A *virt-specifier-seq* shall contain at most one of each *virt-specifier*. A *virt-specifier-seq* shall appear only in the first declaration of a virtual member function (11.7.3).

20 The type of a non-static data member shall not be an incomplete type (6.9.1), an abstract class type (11.7.4), or a (possibly multidimensional) array thereof.

[Note 5: In particular, a class *C* cannot contain a non-static member of class *C*, but it can contain a pointer or reference to an object of class *C*. — end note]

21 [Note 6: See 7.5.5 for restrictions on the use of non-static data members and non-static member functions. — end note]

22 [Note 7: The type of a non-static member function is an ordinary function type, and the type of a non-static data member is an ordinary object type. There are no special member function types or data member types. — end note]

23 [Example 3: A simple example of a class definition is

```
struct tnode {
    char tword[20];
    int count;
    tnode* left;
    tnode* right;
};
```

which contains an array of twenty characters, an integer, and two pointers to objects of the same type. Once this definition has been given, the declaration

```
tnode s, *sp;
```

declares *s* to be a *tnode* and *sp* to be a pointer to a *tnode*. With these declarations, *sp->count* refers to the *count* member of the object to which *sp* points; *s.left* refers to the *left* subtree pointer of the object *s*; and *s.right->tword[0]* refers to the initial character of the *tword* member of the *right* subtree of *s*. — end example]

24 [Note 8: Non-variant non-static data members of non-zero size (6.8.2) are allocated so that later members have higher addresses within a class object (7.6.9). Implementation alignment requirements can cause two adjacent members not to be allocated immediately after each other; so can requirements for space for managing virtual functions (11.7.3) and virtual base classes (11.7.2). — end note]

25 If *T* is the name of a class, then each of the following shall have a name different from *T*:

(25.1) — every static data member of class *T*;

(25.2) — every member function of class *T*;

[Note 9: This restriction does not apply to constructors, which do not have names (11.4.5). — end note]

(25.3) — every member of class *T* that is itself a type;

(25.4) — every member template of class *T*;

(25.5) — every enumerator of every member of class *T* that is an unscoped enumeration type; and

(25.6) — every member of every anonymous union that is a member of class *T*.

26 In addition, if class *T* has a user-declared constructor (11.4.5), every non-static data member of class *T* shall have a name different from *T*.

27 The *common initial sequence* of two standard-layout struct (11.2) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that

(27.1) — corresponding entities have layout-compatible types (6.9),

(27.2) — corresponding entities have the same alignment requirements (6.8.3),

(27.3) — if a *has-attribute-expression* (15.2) is not 0 for the *no_unique_address* attribute, then neither entity is declared with the *no_unique_address* attribute (9.13.11), and

(27.4) — either both entities are bit-fields with the same width or neither is a bit-field.

[Example 4:

```
struct A { int a; char b; };
```

```

struct B { const int b1; volatile char b2; };
struct C { int c; unsigned : 0; char b; };
struct D { int d; char b : 4; };
struct E { unsigned int e; char b; };

```

The common initial sequence of **A** and **B** comprises all members of either class. The common initial sequence of **A** and **C** and of **A** and **D** comprises the first member in each case. The common initial sequence of **A** and **E** is empty. — *end example*

- 28 Two standard-layout struct (11.2) types are *layout-compatible classes* if their common initial sequence comprises all members and bit-fields of both classes (6.9).
- 29 Two standard-layout unions are layout-compatible if they have the same number of non-static data members and corresponding non-static data members (in any order) have layout-compatible types (6.9.1).
- 30 In a standard-layout union with an active member (11.5) of struct type **T1**, it is permitted to read a non-static data member **m** of another union member of struct type **T2** provided **m** is part of the common initial sequence of **T1** and **T2**; the behavior is as if the corresponding member of **T1** were nominated.

[Example 5:

```

struct T1 { int a, b; };
struct T2 { int c; double d; };
union U { T1 t1; T2 t2; };
int f() {
    U u = { { 1, 2 } };    // active member is t1
    return u.t2.c;         // OK, as if u.t1.a were nominated
}

```

— *end example*]

[Note 10: Reading a volatile object through a glvalue of non-volatile type has undefined behavior (9.2.9.2). — *end note*]

- 31 If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member if that member is not a bit-field. Its address is also the same as the address of each of its base class subobjects.

[Note 11: There can therefore be unnamed padding within a standard-layout struct object inserted by an implementation, but not at its beginning, as necessary to achieve appropriate alignment. — *end note*]

[Note 12: The object and its first subobject are pointer-interconvertible (6.9.5, 7.6.1.9). — *end note*]

- 32 A *data member description* is a sextuple (T , N , A , W , NUA , ANN) describing the potential declaration of a non-static data member where

- (32.1) — T is a type,
- (32.2) — N is an *identifier* or $\perp$,
- (32.3) — A is an alignment or $\perp$,
- (32.4) — W is a bit-field width or $\perp$,
- (32.5) — NUA is a boolean value, and
- (32.6) — ANN is a sequence of reflections representing either values or template parameter objects.

Two data member descriptions are equal if each of their respective components are the same entity, the same identifier, the same value, the same sequence, or both $\perp$.

[Note 13: The components of a data member description describe a data member such that

- (32.7) — its type is specified using the type given by T ,
- (32.8) — it is declared with the name given by N if N is not $\perp$ and is otherwise unnamed,
- (32.9) — it is declared with the *alignment-specifier* (9.13.2) given by `alignas( $A$ )` if A is not $\perp$ and is otherwise declared without an *alignment-specifier*,
- (32.10) — it is a bit-field (11.4.10) with the width given by W if W is not $\perp$ and is otherwise not a bit-field, and
- (32.11) — it is declared with the attribute `[[no_unique_address]]` (9.13.11) if NUA is true and is otherwise declared without that attribute.

Data member descriptions are represented by reflections (6.9.3) returned by `std::meta::data_member_spec` (21.4.17) and can be reified as data members of a class using `std::meta::define_aggregate`. — *end note*

11.4.2 Member functions

[class.mfct]

- ¹ If a member function is attached to the global module and is defined (9.6) in its class definition, it is inline (9.2.8).

[Note 1: A member function is also inline if it is declared `inline`, `constexpr`, or `constexpr`. — end note]

- ² [Example 1:

```
struct X {
    typedef int T;
    static T count;
    void f(T);
};
void X::f(T t = count) { }
```

The definition of the member function `f` of class `X` inhabits the global scope; the notation `X::f` indicates that the function `f` is a member of class `X` and in the scope of class `X`. In the function definition, the parameter type `T` refers to the typedef member `T` declared in class `X` and the default argument `count` refers to the static data member `count` declared in class `X`. — end example]

- ³ Member functions of a local class shall be defined inline in their class definition, if they are defined at all.
- ⁴ [Note 2: A member function can be declared (but not defined) using a typedef for a function type. The resulting member function has exactly the same type as it would have if the function declarator were provided explicitly, see 9.3.4.6 and 13.4.

[Example 2:

```
typedef void fv();
typedef void fvc() const;
struct S {
    fv memfunc1;          // equivalent to: void memfunc1();
    void memfunc2();
    fvc memfunc3;         // equivalent to: void memfunc3() const;
};
fv S::* pmfv1 = &S::memfunc1;
fv S::* pmfv2 = &S::memfunc2;
fvc S::* pmfv3 = &S::memfunc3;
```

— end example]

— end note]

11.4.3 Non-static member functions

[class.mfct.non.static]

- ¹ A non-static member function may be called for an object of its class type, or for an object of a class derived (11.7) from its class type, using the class member access syntax (7.6.1.5, 12.2.2.2). A non-static member function may also be called directly using the function call syntax (7.6.1.3, 12.2.2.2) from within its class or a class derived from its class, or a member thereof, as described below.
- ² [Note 1: An implicit object member function can be declared with *cv-qualifiers*, which affect the type of the `this` pointer (7.5.3), and/or a *ref-qualifier* (9.3.4.6); both affect overload resolution (12.2.2). — end note]
- ³ An implicit object member function may be declared virtual (11.7.3) or pure virtual (11.7.4).

11.4.4 Special member functions

[special]

- ¹ Default constructors (11.4.5.2), copy constructors, move constructors (11.4.5.3), copy assignment operators, move assignment operators (11.4.6), and prospective destructors (11.4.7) are *special member functions*.

[Note 1: The implementation will implicitly declare these member functions for some class types when the program does not explicitly declare them. The implementation will implicitly define them as needed (9.6.2). — end note]

An implicitly-declared special member function is declared at the closing `}` of the *class-specifier*. Programs shall not define implicitly-declared special member functions.

- ² Programs may explicitly refer to implicitly-declared special member functions.

[Example 1: A program may explicitly call or form a pointer to member to an implicitly-declared special member function.

```
struct A { };          // implicitly declared A::operator=
```

```

struct B : A {
    B& operator=(const B &);
};
B& B::operator=(const B& s) {
    this->A::operator=(s);    // well-formed
    return *this;
}

```

— end example]

- 3 [Note 2: The special member functions affect the way objects of class type are created, copied, moved, and destroyed, and how values can be converted to values of other types. Often such special member functions are called implicitly. — end note]

- 4 Special member functions obey the usual access rules (11.8).

[Example 2: Declaring a constructor protected ensures that only derived classes and friends can create objects using it. — end example]

- 5 Two special member functions are of the same kind if

- (5.1) — they are both default constructors,
- (5.2) — they are both copy or move constructors with the same first parameter type, or
- (5.3) — they are both copy or move assignment operators with the same first parameter type and the same *cv-qualifiers* and *ref-qualifier*, if any.

- 6 An *eligible special member function* is a special member function for which:

- (6.1) — the function is not deleted,
- (6.2) — the associated constraints (13.5), if any, are satisfied, and
- (6.3) — no special member function of the same kind whose associated constraints, if any, are satisfied is more constrained (13.5.5).

- 7 For a class, its direct non-static data members, its non-virtual direct base classes, and, if the class is not abstract (11.7.4), its virtual base classes are called its *potentially constructed subobjects*.

11.4.5 Constructors

[class.ctor]

11.4.5.1 General

[class.ctor.general]

- 1 A *declarator* declares a *constructor* if it is a function declarator (9.3.4.6) of the form

ptr-declarator (*parameter-declaration-clause*) *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt}

where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

- (1.1) — in a friend declaration (11.8.4), the *id-expression* is a *qualified-id* that names a constructor (6.5.5.2);
- (1.2) — otherwise, in a *member-declaration* that belongs to the *member-specification* of a class or class template, the *id-expression* is the injected-class-name (11.1) of the immediately-enclosing entity;
- (1.3) — otherwise, the *id-expression* is a *qualified-id* whose *unqualified-id* is the injected-class-name of its lookup context.

Constructors do not have names. In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be **friend**, **inline**, **constexpr**, **constexpr**, or an *explicit-specifier*.

[Example 1:

```

struct S {
    S();    // declares the constructor
};

```

```

S::S() { }    // defines the constructor

```

— end example]

- 2 A constructor is used to initialize objects of its class type.

[Note 1: Because constructors do not have names, they are never found during unqualified name lookup; however an explicit type conversion using the functional notation (7.6.1.4) will cause a constructor to be called to initialize an object. The syntax looks like an explicit call of the constructor. — end note]

[Example 2:

```
complex zz = complex(1,2.3);
cprint( complex(7.8,1.2) );
```

— end example]

[Note 2: For initialization of objects of class type see 11.9. — end note]

[Note 3: 6.8.7 describes the lifetime of temporary objects. — end note]

[Note 4: Explicit constructor calls do not yield lvalues, see 7.2.1. — end note]

³ [Note 5: Some language constructs have special semantics when used during construction; see 11.9.3 and 11.9.5. — end note]

⁴ A constructor can be invoked for a **const**, **volatile** or **const volatile** object. **const** and **volatile** semantics (9.2.9.2) are not applied on an object under construction. They come into effect when the constructor for the most derived object (6.8.2) ends.

⁵ The address of a constructor shall not be taken.

[Note 6: A **return** statement in the body of a constructor cannot specify a return value (8.8.4). — end note]

⁶ A constructor shall not be a coroutine.

⁷ A constructor shall not have an explicit object parameter (9.3.4.6).

11.4.5.2 Default constructors

[class.default.ctor]

¹ A *default constructor* for a class **X** is a constructor of class **X** for which each parameter that is not a function parameter pack has a default argument (including the case of a constructor with no parameters). If a class does not have a user-declared constructor or constructor template, and the class is not an anonymous union, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.6). An implicitly-declared default constructor is an inline public member of its class.

² A defaulted default constructor for class **X** is defined as deleted if

- (2.1) — any non-static data member with no default member initializer (11.4) is of reference type,
- (2.2) — **X** is a non-union class and any non-variant non-static data member of const-qualified type (or possibly multidimensional array thereof) with no *brace-or-equal-initializer* is not const-default-constructible (9.5),
- (2.3) — any non-variant potentially constructed subobject, except for a non-static data member with a *brace-or-equal-initializer*, has class type **M** (or possibly multidimensional array thereof) and overload resolution (12.2) as applied to find **M**'s corresponding constructor does not result in a usable candidate (12.2.1), or
- (2.4) — any potentially constructed subobject *S* has class type **M** (or possibly multidimensional array thereof), **M** has a destructor that is deleted or inaccessible from the defaulted default constructor, and either *S* is non-variant or *S* has a default member initializer.

³ A default constructor for a class **X** is *trivial* if it is not user-provided and if

- (3.1) — **X** has no virtual functions (11.7.3) and no virtual base classes (11.7.2), and
- (3.2) — no non-static data member of **X** has a default member initializer (11.4), and
- (3.3) — all the direct base classes of **X** have trivial default constructors, and
- (3.4) — either **X** is a union or for all the non-variant non-static data members of **X** that are of class type (or array thereof), each such class has a trivial default constructor.

Otherwise, the default constructor is *non-trivial*.

⁴ An implicitly-defined (9.6.2) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no *ctor-initializer* (11.9.3) and an empty *compound-statement*. If that user-written default constructor would be ill-formed, the program is ill-formed. The implicitly-defined default constructor is *constexpr*. Before the defaulted default constructor for a class is implicitly defined, all the non-user-provided default constructors for its base classes and its non-static data members are implicitly defined.

[Note 1: An implicitly-declared default constructor has an exception specification (14.5). An explicitly-defaulted definition might have an implicit exception specification, see 9.6. — end note]

⁵ [Note 2: A default constructor is implicitly invoked to initialize a class object when no initializer is specified (9.5.1). Such a default constructor needs to be accessible (11.8). — end note]

- ⁶ [Note 3: 11.9.3 describes the order in which constructors for base classes and non-static data members are called and describes how arguments can be specified for the calls to these constructors. — end note]

11.4.5.3 Copy/move constructors

[class.copy.ctor]

- ¹ A non-template constructor for class **X** is a copy constructor if its first parameter is of type **X&**, **const X&**, **volatile X&** or **const volatile X&**, and either there are no other parameters or else all other parameters have default arguments (9.3.4.7).

[Example 1: **X::X(const X&)** and **X::X(X&,int=1)** are copy constructors.

```
struct X {
    X(int);
    X(const X&, int = 1);
};
X a(1);           // calls X(int);
X b(a, 0);        // calls X(const X&, int);
X c = b;          // calls X(const X&, int);
```

— end example]

- ² A non-template constructor for class **X** is a move constructor if its first parameter is of type **X&&**, **const X&&**, **volatile X&&**, or **const volatile X&&**, and either there are no other parameters or else all other parameters have default arguments (9.3.4.7).

[Example 2: **Y::Y(Y&&)** is a move constructor.

```
struct Y {
    Y(const Y&);
    Y(Y&&);
};
extern Y f(int);
Y d(f(1));        // calls Y(Y&&)
Y e = d;           // calls Y(const Y&)
```

— end example]

- ³ [Note 1: All forms of copy/move constructor can be declared for a class.

[Example 3:

```
struct X {
    X(const X&);
    X(X&);           // OK
    X(X&&);
    X(const X&&);    // OK, but possibly not sensible
};
```

— end example]

— end note]

- ⁴ [Note 2: If a class **X** only has a copy constructor with a parameter of type **X&**, an initializer of type **const X** or **volatile X** cannot initialize an object of type **cv X**.

[Example 4:

```
struct X {
    X();             // default constructor
    X(X&);           // copy constructor with a non-const parameter
};
const X cx;
X x = cx;           // error: X::X(X&) cannot copy cx into x
```

— end example]

— end note]

- ⁵ A declaration of a constructor for a class **X** is ill-formed if its first parameter is of type **cv X** and either there are no other parameters or else all other parameters have default arguments. A member function template is never instantiated to produce such a constructor signature.

[Example 5:

```
struct S {
    template<typename T> S(T);
```

```

    S();
};

S g;

void h() {
    S a(g);           // does not instantiate the member template to produce S::S<S>(S);
                      // uses the implicitly declared copy constructor
}
— end example]

```

- 6 If the class does not have a user-declared copy constructor and the class is not an anonymous union, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defaulted (9.6). The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor (D.6).

- 7 The implicitly-declared copy constructor for a class **X** will have the form

```
X::X(const X&)
```

if each potentially constructed subobject of a class type **M** (or array thereof) has a copy constructor whose first parameter is of type `const M&` or `const volatile M&`.⁸² Otherwise, the implicitly-declared copy constructor will have the form

```
X::X(X&)
```

- 8 If a class **X** does not have a user-declared move constructor, a non-explicit one will be implicitly declared as defaulted if and only if

- (8.1) — **X** is not an anonymous union,
- (8.2) — **X** does not have a user-declared copy constructor,
- (8.3) — **X** does not have a user-declared copy assignment operator,
- (8.4) — **X** does not have a user-declared move assignment operator, and
- (8.5) — **X** does not have a user-declared destructor.

[Note 3: When the move constructor is not implicitly declared or explicitly supplied, expressions that otherwise would have invoked the move constructor might instead invoke a copy constructor. — end note]

- 9 The implicitly-declared move constructor for class **X** will have the form

```
X::X(X&&)
```

- 10 An implicitly-declared copy/move constructor is an inline public member of its class. A defaulted copy/move constructor for a class **X** is defined as deleted (9.6.3) if **X** has:

- (10.1) — a potentially constructed subobject of type **M** (or possibly multidimensional array thereof) for which overload resolution (12.2), as applied to find **M**'s corresponding constructor, either does not result in a usable candidate (12.2.1) or, in the case of a variant member, selects a non-trivial function,
- (10.2) — any potentially constructed subobject of class type **M** (or possibly multidimensional array thereof) where **M** has a destructor that is deleted or inaccessible from the defaulted constructor, or,
- (10.3) — for the copy constructor, a non-static data member of rvalue reference type.

[Note 4: A defaulted move constructor that is defined as deleted is ignored by overload resolution (12.2, 12.3). Such a constructor would otherwise interfere with initialization from an rvalue which can use the copy constructor instead. — end note]

- 11 [Note 5: A using-declaration in a derived class **C** that names a constructor from a base class never suppresses the implicit declaration of a copy/move constructor of **C**, even if the base class constructor would be a copy or move constructor if declared as a member of **C**. — end note]

- 12 A copy/move constructor for class **X** is trivial if it is not user-provided and if

- (12.1) — class **X** has no virtual functions (11.7.3) and no virtual base classes (11.7.2), and

⁸²) This implies that the reference parameter of the implicitly-declared copy constructor cannot bind to a `volatile` lvalue; see C.8.7.

- (12.2) — the constructor selected to copy/move each direct base class subobject is a direct member of that base class and is trivial, and
- (12.3) — for each non-static data member of **X** that is of class type (or array thereof), the constructor selected to copy/move that member is trivial;

otherwise the copy/move constructor is *non-trivial*.

- 13 [Note 6: The copy/move constructor is implicitly defined even if the implementation elided its odr-use (6.3, 6.8.7). — end note]

The implicitly-defined (9.6.2) constructor is constexpr.

- 14 Before the defaulted copy/move constructor for a class is implicitly defined, all non-user-provided copy/move constructors for its potentially constructed subobjects are implicitly defined.

[Note 7: An implicitly-declared copy/move constructor has an implied exception specification (14.5). — end note]

- 15 The implicitly-defined copy/move constructor for a non-union class **X** performs a memberwise copy/move of its bases and members.

[Note 8: Default member initializers of non-static data members are ignored. — end note]

The order of initialization is the same as the order of initialization of bases and members in a user-defined constructor (see 11.9.3). Let **x** be either the parameter of the constructor or, for the move constructor, an xvalue referring to the parameter. Each base or non-static data member is copied/moved in the manner appropriate to its type:

- (15.1) — if the member is an array, each element is direct-initialized with the corresponding subobject of **x**;
- (15.2) — if a member **m** has rvalue reference type **T&&**, it is direct-initialized with `static_cast<T&&>(x.m)`;
- (15.3) — otherwise, the base or member is direct-initialized with the corresponding base or member of **x**.

Virtual base class subobjects shall be initialized only once by the implicitly-defined copy/move constructor (see 11.9.3).

- 16 The implicitly-defined copy/move constructor for a union **X** copies the object representation (6.9.1) of **X**. For each object nested within (6.8.2) the object that is the source of the copy, a corresponding object *o* nested within the destination is identified (if the object is a subobject) or created (otherwise), and the lifetime of *o* begins before the copy is performed.

11.4.6 Copy/move assignment operator

[class.copy.assign]

- 1 A user-declared *copy* assignment operator **X::operator=** is a non-static non-template member function of class **X** with exactly one non-object parameter of type **X**, **X&**, **const X&**, **volatile X&**, or **const volatile X&**.⁸³

[Note 1: More than one form of copy assignment operator can be declared for a class. — end note]

[Note 2: If a class **X** only has a copy assignment operator with a non-object parameter of type **X&**, an expression of type **const X** cannot be assigned to an object of type **X**.

[Example 1:

```
struct X {
    X();
    X& operator=(X&);
};
const X cx;
X x;
void f() {
    x = cx;           // error: X::operator=(X&) cannot assign cx into x
}
```

— end example]

— end note]

⁸³) Because a template assignment operator or an assignment operator taking an rvalue reference parameter is never a copy assignment operator, the presence of such an assignment operator does not suppress the implicit declaration of a copy assignment operator. Such assignment operators participate in overload resolution with other assignment operators, including copy assignment operators, and, if selected, will be used to assign an object.

- ² If the class does not have a user-declared copy assignment operator and the class is not an anonymous union, one is declared *implicitly*. If the class has a user-declared move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defaulted (9.6). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor (D.6). The implicitly-declared copy assignment operator for a class **X** will have the form

```
X&& X::operator=(const X&)
```

if

- (2.1) — each direct base class **B** of **X** has a copy assignment operator whose non-object parameter is of type **const B&**, **const volatile B&**, or **B**, and
- (2.2) — for all the non-static data members of **X** that are of a class type **M** (or array thereof), each such class type has a copy assignment operator whose non-object parameter is of type **const M&**, **const volatile M&**, or **M**.⁸⁴

Otherwise, the implicitly-declared copy assignment operator will have the form

```
X&& X::operator=(X&)
```

- ³ A user-declared move assignment operator **X::operator=** is a non-static non-template member function of class **X** with exactly one non-object parameter of type **X&&**, **const X&&**, **volatile X&&**, or **const volatile X&&**.

[Note 3: More than one form of move assignment operator can be declared for a class. — end note]

- ⁴ If the definition of a class **X** does not explicitly declare a move assignment operator, one will be implicitly declared as defaulted if and only if

- (4.1) — **X** is not an anonymous union,
- (4.2) — **X** does not have a user-declared copy constructor,
- (4.3) — **X** does not have a user-declared move constructor,
- (4.4) — **X** does not have a user-declared copy assignment operator, and
- (4.5) — **X** does not have a user-declared destructor.

[Example 2: The class definition

```
struct S {  
    int a;  
    S& operator=(const S&) = default;  
};
```

will not have a default move assignment operator implicitly declared because the copy assignment operator has been user-declared. The move assignment operator may be explicitly defaulted.

```
struct S {  
    int a;  
    S& operator=(const S&) = default;  
    S& operator=(S&&) = default;  
};
```

— end example]

- ⁵ The implicitly-declared move assignment operator for a class **X** will have the form

```
X&& X::operator=(X&&)
```

- ⁶ The implicitly-declared copy/move assignment operator for class **X** has the return type **X&**. An implicitly-declared copy/move assignment operator is an inline public member of its class.

- ⁷ A defaulted copy/move assignment operator for class **X** is defined as deleted if **X** has:

- (7.1) — a non-static data member of **const** non-class type (or possibly multidimensional array thereof), or
- (7.2) — a non-static data member of reference type, or
- (7.3) — a direct non-static data member of class type **M** (or possibly multidimensional array thereof) or a direct base class **M** that cannot be copied/moved because overload resolution (12.2), as applied to find **M**'s

⁸⁴) This implies that the reference parameter of the implicitly-declared copy assignment operator cannot bind to a **volatile lvalue**; see C.8.7.

corresponding assignment operator, either does not result in a usable candidate (12.2.1) or, in the case of a variant member, selects a non-trivial function.

[Note 4: A defaulted move assignment operator that is defined as deleted is ignored by overload resolution (12.2, 12.3). — end note]

- ⁸ Because a copy/move assignment operator is implicitly declared for a class if not declared by the user, a base class copy/move assignment operator is always hidden by the corresponding assignment operator of a derived class (12.4.3.2).

[Note 5: A *using-declaration* in a derived class *C* that names an assignment operator from a base class never suppresses the implicit declaration of an assignment operator of *C*, even if the base class assignment operator would be a copy or move assignment operator if declared as a member of *C*. — end note]

- ⁹ A copy/move assignment operator for class *X* is trivial if it is not user-provided and if

- (9.1) — class *X* has no virtual functions (11.7.3) and no virtual base classes (11.7.2), and
- (9.2) — the assignment operator selected to copy/move each direct base class subobject is a direct member of that base class and is trivial, and
- (9.3) — for each non-static data member of *X* that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial;

otherwise the copy/move assignment operator is *non-trivial*.

- ¹⁰ An implicitly-defined (9.6.2) copy/move assignment operator is constexpr.

- ¹¹ Before the defaulted copy/move assignment operator for a class is implicitly defined, all non-user-provided copy/move assignment operators for its direct base classes and its non-static data members are implicitly defined.

[Note 6: An implicitly-declared copy/move assignment operator has an implied exception specification (14.5). — end note]

- ¹² The implicitly-defined copy/move assignment operator for a non-union class *X* performs memberwise copy/move assignment of its subobjects. The direct base classes of *X* are assigned first, in the order of their declaration in the *base-specifier-list*, and then the immediate non-static data members of *X* are assigned, in the order in which they were declared in the class definition. Let *x* be either the parameter of the function or, for the move operator, an xvalue referring to the parameter. Each subobject is assigned in the manner appropriate to its type:

- (12.1) — if the subobject is of class type, as if by a call to `operator=` with the subobject as the object expression and the corresponding subobject of *x* as a single function argument (as if by explicit qualification; that is, ignoring any possible virtual overriding functions in more derived classes);
- (12.2) — if the subobject is an array, each element is assigned, in the manner appropriate to the element type;
- (12.3) — if the subobject is of scalar type, the built-in assignment operator is used.

It is unspecified whether subobjects representing virtual base classes are assigned more than once by the implicitly-defined copy/move assignment operator.

[Example 3:

```
struct V { };
struct A : virtual V { };
struct B : virtual V { };
struct C : B, A { };
```

It is unspecified whether the virtual base class subobject *V* is assigned twice by the implicitly-defined copy/move assignment operator for *C*. — end example]

- ¹³ The implicitly-defined copy/move assignment operator for a union *X* copies the object representation (6.9.1) of *X*. If the source and destination of the assignment are not the same object, then for each object nested within (6.8.2) the object that is the source of the copy, a corresponding object *o* nested within the destination is created, and the lifetime of *o* begins before the copy is performed.

- ¹⁴ The implicitly-defined copy/move assignment operator for a class returns the object for which the assignment operator is invoked, that is, the object assigned to.

11.4.7 Destructors

[class.dtor]

- ¹ A declaration whose *declarator-id* has an *unqualified-id* that begins with a `~` declares a *prospective destructor*; its *declarator* shall be a function declarator (9.3.4.6) of the form

ptr-declarator (*parameter-declaration-clause*) *noexcept-specifier*_{opt} *attribute-specifier-seq*_{opt}

where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

- (1.1) — in a *member-declaration* that belongs to the *member-specification* of a class or class template but is not a friend declaration (11.8.4), the *id-expression* is `~class-name` and the *class-name* is the injected-class-name (11.1) of the immediately-enclosing entity or
- (1.2) — otherwise, the *id-expression* is *nested-name-specifier* `~class-name` and the *class-name* is the injected-class-name of the class nominated by the *nested-name-specifier*.

A prospective destructor shall take no arguments (9.3.4.6). Each *decl-specifier* of the *decl-specifier-seq* of a prospective destructor declaration (if any) shall be **friend**, **inline**, **virtual**, or **constexpr**.

- ² If a class has no user-declared prospective destructor and the class is not an anonymous union, a prospective destructor is implicitly declared as defaulted (9.6). An implicitly-declared prospective destructor is an inline public member of its class.
- ³ An implicitly-declared prospective destructor for a class **X** will have the form

`~X()`

- ⁴ At the end of the definition of a class other than an anonymous union, overload resolution is performed among the prospective destructors declared in that class with an empty argument list to select the *destructor* for the class, also known as the *selected destructor*. The program is ill-formed if overload resolution fails. Destructor selection does not constitute a reference to, or odr-use (6.3) of, the selected destructor, and in particular, the selected destructor may be deleted (9.6.3).
- ⁵ The address of a destructor shall not be taken.

[Note 1: A **return** statement in the body of a destructor cannot specify a return value (8.8.4). — end note]

A destructor can be invoked for a **const**, **volatile** or **const volatile** object. **const** and **volatile** semantics (9.2.9.2) are not applied on an object under destruction. They stop being in effect when the destructor for the most derived object (6.8.2) starts.

- ⁶ [Note 2: A declaration of a destructor that does not have a *noexcept-specifier* has the same exception specification as if it had been implicitly declared (14.5). — end note]

- ⁷ A defaulted destructor for a class **X** is defined as deleted if

- (7.1) — **X** is a non-union class and any non-variant potentially constructed subobject has class type **M** (or possibly multidimensional array thereof) where **M** has a destructor that is deleted or is inaccessible from the defaulted destructor,
- (7.2) — **X** is a union and
 - (7.2.1) — overload resolution to select a constructor to default-initialize an object of type **X** either fails or selects a constructor that is either deleted or not trivial, or
 - (7.2.2) — **X** has a variant member **V** of class type **M** (or possibly multi-dimensional array thereof) where **V** has a default member initializer and **M** has a destructor that is non-trivial, or,
- (7.3) — for a virtual destructor, lookup of the non-array deallocation function results in an ambiguity or in a function that is deleted or inaccessible from the defaulted destructor.

- ⁸ A destructor for a class **X** is trivial if it is not user-provided and if

- (8.1) — the destructor is not virtual,
- (8.2) — all of the direct base classes of **X** have trivial destructors, and
- (8.3) — either **X** is a union or for all of the non-variant non-static data members of **X** that are of class type (or array thereof), each such class has a trivial destructor.

Otherwise, the destructor is *non-trivial*.

- ⁹ A defaulted destructor is a **constexpr** destructor.

10 Before a defaulted destructor for a class is implicitly defined, all the non-user-provided destructors for its base classes and its non-static data members are implicitly defined.

11 A prospective destructor can be declared **virtual** (11.7.3) and with a *pure-specifier* (11.7.4). If the destructor of a class is virtual and any objects of that class or any derived class are created in the program, the destructor shall be defined.

12 [Note 3: Some language constructs have special semantics when used during destruction; see 11.9.5. — end note]

13 After executing the body of the destructor and destroying any objects with automatic storage duration allocated within the body, a destructor for class **X** calls the destructors for **X**'s direct non-variant non-static data members other than anonymous unions, the destructors for **X**'s non-virtual direct base classes and, if **X** is the most derived class (11.9.3), its destructor calls the destructors for **X**'s virtual base classes. All destructors are called as if they were referenced with a qualified name, that is, ignoring any possible virtual overriding destructors in more derived classes. Bases and members are destroyed in the reverse order of the completion of their constructor (see 11.9.3).

[Note 4: A **return** statement (8.8.4) in a destructor might not directly return to the caller; before transferring control to the caller, the destructors for the members and bases are called. — end note]

Destructors for elements of an array are called in reverse order of their construction (see 11.9).

14 A destructor is invoked implicitly

- (14.1) — for a constructed object with static storage duration (6.8.6.2) at program termination (6.10.3.4),
- (14.2) — for a constructed object with thread storage duration (6.8.6.3) at thread exit,
- (14.3) — for a constructed object with automatic storage duration (6.8.6.4) when the block in which an object is created exits (8.10),
- (14.4) — for a constructed temporary object when its lifetime ends (7.3.5, 6.8.7).

In each case, the context of the invocation is the context of the construction of the object. A destructor may also be invoked implicitly through use of a *delete-expression* (7.6.2.9) for a constructed object allocated by a *new-expression* (7.6.2.8); the context of the invocation is the *delete-expression*.

[Note 5: An array of class type contains several subobjects for each of which the destructor is invoked. — end note]

A destructor can also be invoked explicitly. A destructor is *potentially invoked* if it is invoked or as specified in 7.6.2.8, 8.8.4, 9.5.2, 11.9.3, and 14.2. A program is ill-formed if a destructor that is potentially invoked is deleted or not accessible from the context of the invocation.

15 At the point of definition of a virtual destructor (including an implicit definition), the non-array deallocation function is determined as if for the expression **delete this** appearing in a non-virtual destructor of the destructor's class (see 7.6.2.9). If the lookup fails or if the deallocation function has a deleted definition (9.6), the program is ill-formed.

[Note 6: This assures that a deallocation function corresponding to the dynamic type of an object is available for the *delete-expression* (11.4.11). — end note]

16 In an explicit destructor call, the destructor is specified by a *~* followed by a *type-name* or *computed-type-specifier* that denotes the destructor's class type. The invocation of a destructor is subject to the usual rules for member functions (11.4.2); that is, if the object is not of the destructor's class type and not of a class derived from the destructor's class type (including when the destructor is invoked via a null pointer value), the program has undefined behavior.

[Note 7: Invoking **delete** on a null pointer does not call the destructor; see 7.6.2.9. — end note]

[Example 1:

```
struct B {
    virtual ~B() { }
};
struct D : B {
    ~D() { }
};

D D_object;
typedef B B_alias;
B* B_ptr = &D_object;
```



```

void f() {
    D_object.B::~~B();           // calls B's destructor
    B_ptr->~B();                 // calls D's destructor
    B_ptr->~B_alias();           // calls D's destructor
    B_ptr->B_alias::~~B();        // calls B's destructor
    B_ptr->B_alias::~~B_alias(); // calls B's destructor
}

```

— end example]

[Note 8: An explicit destructor call must always be written using a member access operator (7.6.1.5) or a *qualified-id* (7.5.5.3); in particular, the *unary-expression* `~X()` in a member function is not an explicit destructor call (7.6.2.2). — end note]

- 17 [Note 9: Explicit calls of destructors are rarely needed. One use of such calls is for objects placed at specific addresses using a placement *new-expression*. Such use of explicit placement and destruction of objects can be necessary to cope with dedicated hardware resources and for writing memory management facilities.

[Example 2:

```

void* operator new(std::size_t, void* p) { return p; }
struct X {
    X(int);
    ~X();
};
void f(X* p);

void g() {                               // rare, specialized use:
    char* buf = new char[sizeof(X)];
    X* p = new(buf) X(222);              // use buf[] and initialize
    f(p);
    p->X::~~X();                          // cleanup
}

```

— end example]

— end note]

- 18 Once a destructor is invoked for an object, the object's lifetime ends; the behavior is undefined (F.6.1) if the destructor is invoked for an object whose lifetime has ended (6.8.4).

[Example 3: If the destructor for an object with automatic storage duration is explicitly invoked, and the block is subsequently left in a manner that would ordinarily invoke implicit destruction of the object, the behavior is undefined. — end example]

- 19 [Note 10: The notation for explicit call of a destructor can be used for any scalar type name (7.5.5.5). Allowing this makes it possible to write code without having to know if a destructor exists for a given type. For example:

```

typedef int I;
I* p;
p->I::~~I();

```

— end note]

- 20 A destructor shall not be a coroutine.

11.4.8 Conversions

[class.conv]

11.4.8.1 General

[class.conv.general]

- 1 Type conversions of class objects can be specified by constructors and by conversion functions. These conversions are called *user-defined conversions* and are used for implicit type conversions (7.3), for initialization (9.5), and for explicit type conversions (7.6.1.4, 7.6.3, 7.6.1.9).
- 2 User-defined conversions are applied only where they are unambiguous (6.5.2, 11.4.8.3). Conversions obey the access control rules (11.8). Access control is applied after ambiguity resolution (6.5).
- 3 [Note 1: See 12.2 for a discussion of the use of conversions in function calls. — end note]
- 4 At most one user-defined conversion (constructor or conversion function) is implicitly applied to a single value.

[Example 1:

```

struct X {
    operator int();
};

struct Y {
    operator X();
};

Y a;
int b = a;           // error: no viable conversion (a.operator X().operator int() not considered)
int c = X(a);        // OK, a.operator X().operator int()
— end example]

```

11.4.8.2 Conversion by constructor

[class.conv.ctor]

- ¹ A constructor that is not explicit (9.2.3) specifies a conversion from the types of its parameters (if any) to the type of its class.

[Example 1:

```

struct X {
    X(int);
    X(const char*, int = 0);
    X(int, int);
};

void f(X arg) {
    X a = 1;           // a = X(1)
    X b = "Jessie";    // b = X("Jessie",0)
    a = 2;             // a = X(2)
    f(3);              // f(X(3))
    f({1, 2});         // f(X(1,2))
}

```

— end example]

- ² [Note 1: An explicit constructor constructs objects just like non-explicit constructors, but does so only where the direct-initialization syntax (9.5) or where casts (7.6.1.9, 7.6.3) are explicitly used; see also 12.2.2.5. A default constructor can be an explicit constructor; such a constructor will be used to perform default-initialization or value-initialization (9.5).

[Example 2:

```

struct Z {
    explicit Z();
    explicit Z(int);
    explicit Z(int, int);
};

Z a;                // OK, default-initialization performed
Z b{};              // OK, direct initialization syntax used
Z c = {};           // error: copy-list-initialization
Z a1 = 1;           // error: no implicit conversion
Z a3 = Z(1);        // OK, direct initialization syntax used
Z a2(1);            // OK, direct initialization syntax used
Z* p = new Z(1);    // OK, direct initialization syntax used
Z a4 = (Z)1;        // OK, explicit cast used
Z a5 = static_cast<Z>(1); // OK, explicit cast used
Z a6 = { 3, 4 };    // error: no implicit conversion

```

— end example]

— end note]

11.4.8.3 Conversion functions

[class.conv.fct]

```

conversion-function-id:
    operator conversion-type-id

conversion-type-id:
    type-specifier-seq conversion-declaratoropt

```

conversion-declarator:

ptr-operator conversion-declarator_{opt}

- ¹ A declaration whose *declarator-id* has an *unqualified-id* that is a *conversion-function-id* declares a *conversion function*; its *declarator* shall be a function declarator (9.3.4.6) of the form

noptr-declarator parameters-and-qualifiers

where the *noptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

- (1.1) — in a *member-declaration* that belongs to the *member-specification* of a class or class template but is not a friend declaration (11.8.4), the *id-expression* is a *conversion-function-id*;
 - (1.2) — otherwise, the *id-expression* is a *qualified-id* whose *unqualified-id* is a *conversion-function-id*.
- ² A conversion function shall have no non-object parameters and shall be a non-static member function of a class or class template X; its declared return type is the *conversion-type-id* and it specifies a conversion from X to the type specified by the *conversion-type-id*, interpreted as a *type-id* (9.3.2). A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall not be a *defining-type-specifier*.
- ³ [Note 1: A conversion function is never invoked for implicit or explicit conversions of an object to the same object type (or a reference to it), to a base class of that type (or a reference to it), or to *cv void*. Even though never directly called to perform a conversion, such conversion functions can be declared and can potentially be reached through a call to a virtual conversion function in a base class. — end note]

[Example 1:

```
struct X {
    operator int();
    operator auto() -> short;    // error: trailing return type
};

void f(X a) {
    int i = int(a);
    i = (int)a;
    i = a;
}
```

In all three cases the value assigned will be converted by `X::operator int()`. — end example]

- ⁴ A conversion function may be explicit (9.2.3), in which case it is only considered as a user-defined conversion for direct-initialization (9.5). Otherwise, user-defined conversions are not restricted to use in assignments and initializations.

[Example 2:

```
class Y { };
struct Z {
    explicit operator Y() const;
};

void h(Z z) {
    Y y1(z);           // OK, direct-initialization
    Y y2 = z;          // error: no conversion function candidate for copy-initialization
    Y y3 = (Y)z;       // OK, cast notation
}

void g(X a, X b) {
    int i = (a) ? 1+a : 0;
    int j = (a&&b) ? a+b : i;
    if (a) {
    }
}
```

— end example]

- ⁵ The *conversion-type-id* shall not represent a function type nor an array type. The *conversion-type-id* in a *conversion-function-id* is the longest sequence of tokens that could possibly form a *conversion-type-id*.

[Note 2: This prevents ambiguities between the declarator operator `*` and its expression counterparts.

[Example 3:

```
&ac.operator int*i; // syntax error:
                      // parsed as: &(ac.operator int *)i
                      // not as: &(ac.operator int)*i
```

The `*` is the pointer declarator and not the multiplication operator. — end example]

This rule also prevents ambiguities for attributes.

[Example 4:

```
operator int [[noreturn]] (); // error: noreturn attribute applied to a type
— end example]
— end note]
```

- ⁶ [Note 3: A conversion function in a derived class hides only conversion functions in base classes that convert to the same type. A conversion function template with a dependent return type hides only templates in base classes that correspond to it (6.5.2); otherwise, it hides and is hidden as a non-template function. Function overload resolution (12.2.4) selects the best conversion function to perform the conversion.

[Example 5:

```
struct X {
    operator int();
};

struct Y : X {
    operator char();
};

void f(Y& a) {
    if (a) { // error: ambiguous between X::operator int() and Y::operator char()
    }
}
— end example]
— end note]
```

- ⁷ Conversion functions can be virtual.
- ⁸ A conversion function template shall not have a deduced return type (9.2.9.7).

[Example 6:

```
struct S {
    template<class T>
    operator auto() const { return 1.2; } // error: conversion function template
};
— end example]
```

11.4.9 Static members

[class.static]

11.4.9.1 General

[class.static.general]

- ¹ A static member `s` of class `X` may be referred to using the *qualified-id* expression `X::s`; it is not necessary to use the class member access syntax (7.6.1.5) to refer to a static member. A static member may be referred to using the class member access syntax, in which case the object expression is evaluated.

[Example 1:

```
struct process {
    static void reschedule();
};

process& g();

void f() {
    process::reschedule(); // OK, no object necessary
    g().reschedule();      // g() is called
}
— end example]
```

- ² Static members obey the usual class member access rules (11.8). When used in the declaration of a class member, the **static** specifier shall only be used in the member declarations that appear within the *member-specification* of the class definition.

[Note 1: It cannot be specified in member declarations that appear in namespace scope. — end note]

11.4.9.2 Static member functions

[class.static.mfct]

- ¹ [Note 1: The rules described in 11.4.2 apply to static member functions. — end note]
- ² [Note 2: A static member function does not have a **this** pointer (7.5.3). A static member function cannot be qualified with **const**, **volatile**, or **virtual** (9.3.4.6). — end note]

11.4.9.3 Static data members

[class.static.data]

- ¹ A static data member is not part of the subobjects of a class. If a static data member is declared **thread_local** there is one copy of the member per thread. If a static data member is not declared **thread_local** there is one copy of the data member that is shared by all the objects of the class.
- ² A static data member shall not be **mutable** (9.2.2). A static data member shall not be a direct member (11.4) of an unnamed (11.1) or local (11.6) class or of a (possibly indirectly) nested class (11.4.12) thereof.
- ³ The declaration of a non-inline static data member in its class definition is not a definition and may be of an incomplete type other than *cv void*.

[Note 1: The *initializer* in the definition of a static data member is in the scope of its class (6.4.7). — end note]

[Example 1:

```
class process {
    static process* run_chain;
    static process* running;
};

process* process::running = get_main();
process* process::run_chain = running;
```

The definition of the static data member **run_chain** of class **process** inhabits the global scope; the notation **process::run_chain** indicates that the member **run_chain** is a member of class **process** and in the scope of class **process**. In the static data member definition, the *initializer* expression refers to the static data member **running** of class **process**. — end example]

[Note 2: Once the static data member has been defined, it exists even if no objects of its class have been created.

[Example 2: In the example above, **run_chain** and **running** exist even if no objects of class **process** are created by the program. — end example]

The initialization and destruction of static data members is described in 6.10.3.2, 6.10.3.3, and 6.10.3.4. — end note]

- ⁴ If a non-volatile non-inline **const** static data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (7.7.3). The member shall still be defined in a namespace scope if it is odr-used (6.3) in the program and the namespace scope definition shall not contain an *initializer*. The declaration of an inline static data member (which is a definition) may specify a *brace-or-equal-initializer*. If the member is declared with the **constexpr** specifier, it may be redeclared in namespace scope with no *initializer* (this usage is deprecated; see D.7). Declarations of other static data members shall not specify a *brace-or-equal-initializer*.
- ⁵ [Note 3: There is exactly one definition of a static data member that is odr-used (6.3) in a valid program. — end note]
- ⁶ [Note 4: Static data members of a class in namespace scope have the linkage of the name of the class (6.7). — end note]

11.4.10 Bit-fields

[class.bit]

- ¹ A *member-declarator* of the form

identifier_{opt} attribute-specifier-seq_{opt} : constant-expression brace-or-equal-initializer_{opt}

specifies a bit-field. The optional *attribute-specifier-seq* appertains to the entity being declared. A bit-field shall not be a static member. A bit-field shall have integral or (possibly cv-qualified) enumeration type; the bit-field semantic property is not part of the type of the class member. The *constant-expression* shall be an integral constant expression with a value greater than or equal to zero and is called the *width* of the bit-field.

If the width of a bit-field is larger than the width of the bit-field's type (or, in case of an enumeration type, of its underlying type), the extra bits are padding bits (6.9.1). Allocation of bit-fields within a class object is implementation-defined. Alignment of bit-fields is implementation-defined. Bit-fields are packed into some addressable allocation unit.

[*Note 1*: Bit-fields straddle allocation units on some machines and not on others. Bit-fields are assigned right-to-left on some machines, left-to-right on others. — *end note*]

- ² A declaration for a bit-field that omits the *identifier* declares an *unnamed bit-field*. Unnamed bit-fields are not members and cannot be initialized. An unnamed bit-field shall not be declared with a cv-qualified type.

[*Note 2*: An unnamed bit-field is useful for padding to conform to externally-imposed layouts. — *end note*]

As a special case, an unnamed bit-field with a width of zero specifies alignment of the next bit-field at an allocation unit boundary. Only when declaring an unnamed bit-field may the width be zero.

- ³ The address-of operator & shall not be applied to a bit-field, so there are no pointers to bit-fields. A non-const reference shall not bind to a bit-field (9.5.4).

[*Note 3*: If the initializer for a reference of type `const T&` is an lvalue that refers to a bit-field, the reference is bound to a temporary initialized to hold the value of the bit-field; the reference is not bound to the bit-field directly. See 9.5.4. — *end note*]

- ⁴ If a value of integral type (other than `bool`) is stored into a bit-field of width N and the value would be representable in a hypothetical signed or unsigned integer type with width N and the same signedness as the bit-field's type, the original value and the value of the bit-field compare equal. If the value `true` or `false` is stored into a bit-field of type `bool` of any size (including a one bit bit-field), the original `bool` value and the value of the bit-field compare equal. If a value of an enumeration type is stored into a bit-field of the same type and the width is large enough to hold all the values of that enumeration type (9.8.1), the original value and the value of the bit-field compare equal.

[*Example 1*:

```
enum BOOL { FALSE=0, TRUE=1 };
struct A {
    BOOL b:1;
};
A a;
void f() {
    a.b = TRUE;
    if (a.b == TRUE)           // yields true
        { /* ... */ }
}
```

— *end example*]

11.4.11 Allocation and deallocation functions

[`class.free`]

- ¹ Any allocation function for a class `T` is a static member (even if not explicitly declared `static`).

- ² [*Example 1*:

```
class Arena;
struct B {
    void* operator new(std::size_t, Arena*);
};
struct D1 : B {
};

Arena* ap;
void foo(int i) {
    new (ap) D1;           // calls B::operator new(std::size_t, Arena*)
    new D1[i];             // calls ::operator new[](std::size_t)
    new D1;                // error: ::operator new(std::size_t) hidden
}
```

— *end example*]

- ³ Any deallocation function for a class `X` is a static member (even if not explicitly declared `static`).

[Example 2:

```
class X {
    void operator delete(void*);
    void operator delete[](void*, std::size_t);
};

class Y {
    void operator delete(void*, std::size_t);
    void operator delete[](void*);
};
```

— end example]

- 4 Since member allocation and deallocation functions are **static** they cannot be virtual.

[Note 1: However, when the *cast-expression* of a *delete-expression* refers to an object of class type with a virtual destructor, because the deallocation function is chosen by the destructor of the dynamic type of the object, the effect is the same in that case.

[Example 3:

```
struct B {
    virtual ~B();
    void operator delete(void*, std::size_t);
};

struct D : B {
    void operator delete(void*);
};

struct E : B {
    void log_deletion();
    void operator delete(E *p, std::destroying_delete_t) {
        p->log_deletion();
        p->~E();
        ::operator delete(p);
    }
};

void f() {
    B* bp = new D;
    delete bp;           // 1: uses D::operator delete(void*)
    bp = new E;
    delete bp;           // 2: uses E::operator delete(E*, std::destroying_delete_t)
}
```

Here, storage for the object of class D is deallocated by **D::operator delete()**, and the object of class E is destroyed and its storage is deallocated by **E::operator delete()**, due to the virtual destructor. — end example]

— end note]

[Note 2: Virtual destructors have no effect on the deallocation function actually called when the *cast-expression* of a *delete-expression* refers to an array of objects of class type.

[Example 4:

```
struct B {
    virtual ~B();
    void operator delete[](void*, std::size_t);
};

struct D : B {
    void operator delete[](void*, std::size_t);
};

void f(int i) {
    D* dp = new D[i];
    delete [] dp;        // uses D::operator delete[](void*, std::size_t)
    B* bp = new D[i];
```

```

    delete[] bp;      // undefined behavior
}
— end example]
— end note]

```

- ⁵ Access to the deallocation function is checked statically, even if a different one is actually executed.

[*Example 5*: For the call on line “// 1” above, if `B::operator delete()` had been private, the delete expression would have been ill-formed. — end example]

- ⁶ [*Note 3*: If a deallocation function has no explicit *noexcept-specifier*, it has a non-throwing exception specification (14.5). — end note]

11.4.12 Nested class declarations

[**class.nest**]

- ¹ A class can be declared within another class. A class declared within another is called a *nested class*.

[*Note 1*: See 7.5.5 for restrictions on the use of non-static data members and non-static member functions. — end note]

[*Example 1*:

```

int x;
int y;

struct enclose {
    int x;
    static int s;

    struct inner {
        void f(int i) {
            int a = sizeof(x);      // OK, operand of sizeof is an unevaluated operand
            x = i;                  // error: assign to enclose::x
            s = i;                  // OK, assign to enclose::s
            ::x = i;                // OK, assign to global x
            y = i;                  // OK, assign to global y
        }
        void g(enclose* p, int i) {
            p->x = i;                // OK, assign to enclose::x
        }
    };
};

inner* p = 0;                      // error: inner not found
— end example]

```

- ² [*Note 2*: Nested classes can be defined either in the enclosing class or in an enclosing namespace; member functions and static data members of a nested class can be defined either in the nested class or in an enclosing namespace scope.

[*Example 2*:

```

struct enclose {
    struct inner {
        static int x;
        void f(int i);
    };
};

int enclose::inner::x = 1;

void enclose::inner::f(int i) { /* ... */ }

class E {
    class I1;                // forward declaration of nested class
    class I2;
    class I1 { };            // definition of nested class
};
class E::I2 { };             // definition of nested class

```


— *end example*]

— *end note*]

- ³ A friend function (11.8.4) defined within a nested class has no special access rights to members of an enclosing class.

11.5 Unions

[class.union]

11.5.1 General

[class.union.general]

- ¹ A *union* is a class defined with the *class-key union*.
- ² In a union, a non-static data member is *active* if its name refers to an object whose lifetime has begun and has not ended (6.8.4). At most one of the non-static data members of an object of union type can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time.

[*Note 1*: One special guarantee is made in order to simplify the use of unions: If a standard-layout union contains several standard-layout structs that share a common initial sequence (11.4), and if a non-static data member of an object of this standard-layout union type is active and is one of the standard-layout structs, the common initial sequence of any of the standard-layout struct members can be inspected; see 11.4. — *end note*]

- ³ The size of a union is sufficient to contain the largest of its non-static data members. Each non-static data member is allocated as if it were the sole member of a non-union class.

[*Note 2*: A union object and its non-static data members are pointer-interconvertible (6.9.5, 7.6.1.9). As a consequence, all non-static data members of a union object have the same address. — *end note*]

- ⁴ A union can have member functions (including constructors and destructors), but it shall not have virtual (11.7.3) functions. A union shall not have base classes. A union shall not be used as a base class. If a union contains a non-static data member of reference type, the program is ill-formed.

[*Note 3*: If any non-static data member of a union has a non-trivial copy constructor, move constructor (11.4.5.3), copy assignment operator, or move assignment operator (11.4.6), the corresponding member function of the union must be user-provided or it will be implicitly deleted (9.6.3) for the union.

[*Example 1*: Consider the following union:

```
union U {
    int i;
    float f;
    std::string s;
};
```

Since `std::string` (27.4) declares non-trivial versions of all of the special member functions, `U` will have an implicitly deleted copy/move constructor and copy/move assignment operator. The default constructor and destructor of `U` are both trivial even though `std::string` has a non-trivial default constructor and a non-trivial destructor. — *end example*]

— *end note*]

- ⁵ When the left operand of an assignment operator involves a member access expression (7.6.1.5) that nominates a union member, it may begin the lifetime of that union member, as described below. For an expression `E`, define the set $S(E)$ of subexpressions of `E` as follows:
- (5.1) — If `E` is of the form `A.B`, $S(E)$ contains the elements of $S(A)$, and also contains `A.B` if `B` names a union member of a non-class, non-array type, or of a class type with a trivial default constructor that is not deleted, or an array of such types.
 - (5.2) — If `E` is of the form `A[B]` and is interpreted as a built-in array subscripting operator, $S(E)$ is $S(A)$ if `A` is of array type, $S(B)$ if `B` is of array type, and empty otherwise.
 - (5.3) — Otherwise, $S(E)$ is empty.

In an assignment expression of the form `E1 = E2` that uses either the built-in assignment operator (7.6.19) or a trivial assignment operator (11.4.6), for each element `X` of $S(E1)$ and each anonymous union member `X` (11.5.2) that is a member of a union and has such an element as an immediate subobject (recursively), if modification of `X` would have undefined behavior under 6.8.4, an object of the type of `X` is implicitly created in the nominated storage; no initialization is performed and the beginning of its lifetime is sequenced after the value computation of the left and right operands and before the assignment.

[*Note 4*: This ends the lifetime of the previously-active member of the union, if any (6.8.4). — *end note*]

[Example 2:

```
union A { int x; int y[4]; };
struct B { A a; };
union C { B b; int k; };
int f() {
    C c;                // does not start lifetime of any union member
    c.b.a.y[3] = 4;      // OK, S(c.b.a.y[3]) contains c.b and c.b.a.y;
                        // creates objects to hold union members c.b and c.b.a.y
    return c.b.a.y[3];   // OK, c.b.a.y refers to newly created object (see 6.8.4)
}

struct X { const int a; int b; };
union Y { X x; int k; };
void g() {
    Y y = { { 1, 2 } }; // OK, y.x is active union member (11.4)
    int n = y.x.a;
    y.k = 4;             // OK, ends lifetime of y.x, y.k is active member of union
    y.x.b = n;           // undefined behavior: y.x.b modified outside its lifetime,
                        // S(y.x.b) is empty because X's default constructor is deleted,
                        // so union member y.x's lifetime does not implicitly start
}
```

— end example]

- ⁶ [Note 5: In cases where the above rule does not apply, the active member of a union can only be changed by the use of a placement *new-expression*. — end note]

[Example 3: Consider an object *u* of a **union** type *U* having non-static data members *m* of type *M* and *n* of type *N*. If *M* has a non-trivial destructor and *N* has a non-trivial constructor (for instance, if they declare or inherit virtual functions), the active member of *u* can be safely switched from *m* to *n* using the destructor and placement *new-expression* as follows:

```
u.m.~M();
new (&u.n) N;
```

— end example]

11.5.2 Anonymous unions

[class.union.anon]

- ¹ A union of the form

```
union { member-specification } ;
```

is called an *anonymous union*; it defines an unnamed type and an unnamed object of that type called an *anonymous union member* if it is a non-static data member or an *anonymous union variable* otherwise. Each object of such an unnamed type shall be such an unnamed object. Each *member-declaration* in the *member-specification* of an anonymous union shall define one or more public non-static data members, be an *empty-declaration*, or be a *static_assert-declaration*. Nested types (including closure types (7.5.6.2) and anonymous unions) and functions shall not be declared within an anonymous union. The names of the members of an anonymous union are bound in the scope inhabited by the union declaration.

[Example 1:

```
void f() {
    union { int a; const char* p; };
    a = 1;
    p = "Jennifer";
}
```

Here *a* and *p* are used like ordinary (non-member) variables, but since they are union members they have the same address. — end example]

- ² An anonymous union declared in the scope of a namespace with external linkage shall use the *storage-class-specifier* **static**. Anonymous unions declared at block scope shall not use a *storage-class-specifier* that is not permitted in the declaration of a block variable. An anonymous union declaration at class scope shall not have a *storage-class-specifier*.
- ³ [Note 1: A union for which objects, pointers, or references are declared is not an anonymous union.

[Example 2:

```
void f() {
    union { int aa; char* p; } obj, *ptr = &obj;
    aa = 1;           // error
    ptr->aa = 1;       // OK
}
```

The assignment to plain `aa` is ill-formed since the member name is not visible outside the union, and even if it were visible, it is not associated with any particular object. — *end example*]

— *end note*]

[Note 2: Initialization of unions with no user-declared constructors is described in 9.5.2. — *end note*]

- ⁴ A *union-like class* is a union or a class that has an anonymous union as a direct member. A union-like class `X` has a set of *variant members*. If `X` is a union, a non-static data member of `X` that is not an anonymous union is a variant member of `X`. In addition, a non-static data member of an anonymous union that is a member of `X` is also a variant member of `X`. At most one variant member of a union may have a default member initializer.

[Example 3:

```
union U {
    int x = 0;
    union {
        int k;
    };
    union {
        int z;
        int y = 1;    // error: initialization for second variant member of U
    };
};
```

— *end example*]

11.6 Local class declarations

[class.local]

- ¹ A class can be declared within a function definition; such a class is called a *local class*.

[Note 1: A declaration in a local class cannot odr-use (6.3) a local entity from an enclosing scope. — *end note*]

[Example 1:

```
int x;
void f() {
    static int s;
    int x;
    const int N = 5;
    extern int q();
    int arr[2];
    auto [y, z] = arr;

    struct local {
        int g() { return x; }    // error: odr-use of non-odr-usable variable x
        int h() { return s; }    // OK
        int k() { return ::x; }  // OK
        int l() { return q(); }  // OK
        int m() { return N; }    // OK, not an odr-use
        int* n() { return &N; }  // error: odr-use of non-odr-usable variable N
        int p() { return y; }    // error: odr-use of non-odr-usable structured binding y
    };
}
```

```
local* p = 0;    // error: local not found
```

— *end example*]

- ² An enclosing function has no special access to members of the local class; it obeys the usual access rules (11.8). Member functions of a local class shall be defined within their class definition, if they are defined at all.

- ³ A class nested within a local class is a local class. A member of a local class **X** shall be declared only in the definition of **X** or, if the member is a nested class, in the nearest enclosing block scope of **X**.
- ⁴ [Note 2: A local class cannot have static data members (11.4.9.3). — end note]

11.7 Derived classes

[class.derived]

11.7.1 General

[class.derived.general]

- ¹ A list of base classes can be specified in a class definition using the notation:

```

base-clause:
    : base-specifier-list

base-specifier-list:
    base-specifier ...opt
    base-specifier-list , base-specifier ...opt

base-specifier:
    attribute-specifier-seqopt class-or-decltype
    attribute-specifier-seqopt virtual access-specifieropt class-or-decltype
    attribute-specifier-seqopt access-specifier virtualopt class-or-decltype

class-or-decltype:
    nested-name-specifieropt type-name
    nested-name-specifier template simple-template-id
    computed-type-specifier

access-specifier:
    private
    protected
    public

```

The optional *attribute-specifier-seq* appertains to the *base-specifier*.

- ² The component names of a *class-or-decltype* are those of its *nested-name-specifier*, *type-name*, and/or *simple-template-id*. A *class-or-decltype* shall denote a (possibly cv-qualified) class type that is not an incompletely defined class (11.4); any cv-qualifiers are ignored. The class denoted by the *class-or-decltype* of a *base-specifier* is called a *direct base class* for the class being defined; for each such *base-specifier*, the corresponding *direct base class relationship* is the ordered pair (D, B) where D is the class being defined and B is the direct base class. The lookup for the component name of the *type-name* or *simple-template-id* is type-only (6.5). A class B is a base class of a class D if it is a direct base class of D or a direct base class of one of D's base classes. A class is an *indirect base class* of another if it is a base class but not a direct base class. A class is said to be (directly or indirectly) *derived* from its (direct or indirect) base classes.

[Note 1: See 11.8 for the meaning of *access-specifier*. — end note]

Members of a base class are also members of the derived class.

[Note 2: Constructors of a base class can be explicitly inherited (9.10). Base class members can be referred to in expressions in the same manner as other members of the derived class, unless their names are hidden or ambiguous (6.5.2). The scope resolution operator :: (7.5.5.3) can be used to refer to a direct or indirect base member explicitly, even if it is hidden in the derived class. A derived class can itself serve as a base class subject to access control; see 11.8.3. A pointer to a derived class can be implicitly converted to a pointer to an accessible unambiguous base class (7.3.12). An lvalue of a derived class type can be bound to a reference to an accessible unambiguous base class (9.5.4). — end note]

- ³ The *base-specifier-list* specifies the type of the *base class subobjects* contained in an object of the derived class type.

[Example 1:

```

struct Base {
    int a, b, c;
};

struct Derived : Base {
    int b;
};

struct Derived2 : Derived {
    int c;
};

```

Here, an object of class **Derived2** will have a subobject of class **Derived** which in turn will have a subobject of class **Base**. — *end example*

- 4 A *base-specifier* followed by an ellipsis is a pack expansion (13.7.4).
- 5 The order in which the base class subobjects are allocated in the most derived object (6.8.2) is unspecified.
- [*Note 3*: A derived class and its base class subobjects can be represented by a directed acyclic graph (DAG) where an arrow means “directly derived from” (see Figure 3). An arrow need not have a physical representation in memory. A DAG of subobjects is often referred to as a “subobject lattice”. — *end note*]

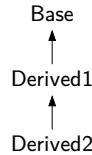


Figure 3 — Directed acyclic graph [fig:class.dag]

- 6 [*Note 4*: Initialization of objects representing base classes can be specified in constructors; see 11.9.3. — *end note*]
- 7 [*Note 5*: A base class subobject can have a layout different from the layout of a most derived object of the same type. A base class subobject can have a polymorphic behavior (11.9.5) different from the polymorphic behavior of a most derived object of the same type. A base class subobject can be of zero size; however, two subobjects that have the same class type and that belong to the same most derived object cannot be allocated at the same address (6.8.2). — *end note*]

11.7.2 Multiple base classes

[class.mi]

- 1 A class can be derived from any number of base classes.

[*Note 1*: The use of more than one direct base class is often called multiple inheritance. — *end note*]

[*Example 1*:

```

class A { /* ... */ };
class B { /* ... */ };
class C { /* ... */ };
class D : public A, public B, public C { /* ... */ };
  
```

— *end example*]

- 2 [*Note 2*: The order of derivation is not significant except as specified by the semantics of initialization by constructor (11.9.3), cleanup (11.4.7), and storage layout (11.4, 11.8.2). — *end note*]
- 3 A class shall not be specified as a direct base class of a derived class more than once.

[*Note 3*: A class can be an indirect base class more than once and can be a direct and an indirect base class. There are limited things that can be done with such a class; lookup that finds its non-static data members and member functions in the scope of the derived class will be ambiguous. However, the static members, enumerations and types can be unambiguously referred to. — *end note*]

[*Example 2*:

```

class X { /* ... */ };
class Y : public X, public X { /* ... */ };           // error
class L { public: int next; /* ... */ };
class A : public L { /* ... */ };
class B : public L { /* ... */ };
class C : public A, public B { void f(); /* ... */ }; // well-formed
class D : public A, public L { void f(); /* ... */ }; // well-formed
  
```

— *end example*]

- 4 A base class specifier that does not contain the keyword **virtual** specifies a *non-virtual base class*. A base class specifier that contains the keyword **virtual** specifies a *virtual base class*. For each distinct occurrence of a non-virtual base class in the class lattice of the most derived class, the most derived object (6.8.2) shall contain a corresponding distinct base class subobject of that type. For each distinct base class that is specified virtual, the most derived object shall contain a single base class subobject of that type.

- ⁵ [Note 4: For an object of class type **C**, each distinct occurrence of a (non-virtual) base class **L** in the class lattice of **C** corresponds one-to-one with a distinct **L** subobject within the object of type **C**. Given the class **C** defined above, an object of class **C** will have two subobjects of class **L** as shown in Figure 4.

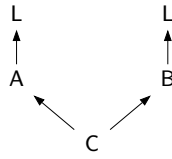


Figure 4 — Non-virtual base [fig:class.nonvirt]

In such lattices, explicit qualification can be used to specify which subobject is meant. The body of function **C::f** can refer to the member **next** of each **L** subobject:

```
void C::f() { A::next = B::next; } // well-formed
```

Without the **A::** or **B::** qualifiers, the definition of **C::f** above would be ill-formed because of ambiguity (6.5.2). — end note]

- ⁶ [Note 5: In contrast, consider the case with a virtual base class:

```
class V { /* ... */ };
class A : virtual public V { /* ... */ };
class B : virtual public V { /* ... */ };
class C : public A, public B { /* ... */ };
```

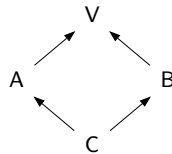


Figure 5 — Virtual base [fig:class.virt]

For an object **c** of class type **C**, a single subobject of type **V** is shared by every base class subobject of **c** that has a virtual base class of type **V**. Given the class **C** defined above, an object of class **C** will have one subobject of class **V**, as shown in Figure 5. — end note]

- ⁷ [Note 6: A class can have both virtual and non-virtual base classes of a given type.

```
class B { /* ... */ };
class X : virtual public B { /* ... */ };
class Y : virtual public B { /* ... */ };
class Z : public B { /* ... */ };
class AA : public X, public Y, public Z { /* ... */ };
```

For an object of class **AA**, all virtual occurrences of base class **B** in the class lattice of **AA** correspond to a single **B** subobject within the object of type **AA**, and every other occurrence of a (non-virtual) base class **B** in the class lattice of **AA** corresponds one-to-one with a distinct **B** subobject within the object of type **AA**. Given the class **AA** defined above, class **AA** has two subobjects of class **B**: **Z**'s **B** and the virtual **B** shared by **X** and **Y**, as shown in Figure 6.

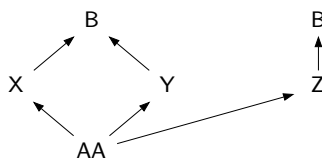


Figure 6 — Virtual and non-virtual base [fig:class.virtnonvirt]

— end note]

11.7.3 Virtual functions

[class.virtual]

- ¹ A non-static member function is a *virtual function* if it is first declared with the keyword `virtual` or if it overrides a virtual member function declared in a base class (see below).⁸⁵

[Note 1: Virtual functions support dynamic binding and object-oriented programming. — end note]

A class with a virtual member function is called a *polymorphic class*.⁸⁶

- ² If a virtual member function F is declared in a class B , and, in a class D derived (directly or indirectly) from B , a declaration of a member function G corresponds (6.4.1) to a declaration of F , ignoring trailing *requires-clauses*, then G *overrides*⁸⁷ F . For convenience, we say that any virtual function overrides itself. A virtual member function V of a class object S is a *final overrider* unless the most derived class (6.8.2) of which S is a base class subobject (if any) has another member function that overrides V . In a derived class, if a virtual member function of a base class subobject has more than one final overrider, the program is ill-formed.

[Example 1:

```
struct A {
    virtual void f();
};
struct B : virtual A {
    virtual void f();
};
struct C : B , virtual A {
    using A::f;
};

void foo() {
    C c;
    c.f();           // calls B::f, the final overrider
    c.C::f();        // calls A::f because of the using-declaration
}
```

— end example]

[Example 2:

```
struct A { virtual void f(); };
struct B : A { };
struct C : A { void f(); };
struct D : B, C { };           // OK, A::f and C::f are the final overriders
                                // for the B and C subobjects, respectively
```

— end example]

- ³ [Note 2: A virtual member function does not have to be visible to be overridden, for example,

```
struct B {
    virtual void f();
};
struct D : B {
    void f(int);
};
struct D2 : D {
    void f();
};
```

the function `f(int)` in class `D` hides the virtual function `f()` in its base class `B`; `D::f(int)` is not a virtual function. However, `f()` declared in class `D2` has the same name and the same parameter list as `B::f()`, and therefore is a virtual function that overrides the function `B::f()` even though `B::f()` is not visible in class `D2`. — end note]

⁸⁵) The use of the `virtual` specifier in the declaration of an overriding function is valid but redundant (has empty semantics).

⁸⁶) If all virtual functions are immediate functions, the class is still polymorphic even if its internal representation does not otherwise require any additions for that polymorphic behavior.

⁸⁷) A function with the same name but a different parameter list (Clause 12) as a virtual function is not necessarily virtual and does not override. Access control (11.8) is not considered in determining overriding.

- ⁴ If a virtual function `f` in some class `B` is marked with the *virt-specifier* `final` and in a class `D` derived from `B` a function `D::f` overrides `B::f`, the program is ill-formed.

[Example 3:

```
struct B {
    virtual void f() const final;
};

struct D : B {
    void f() const;    // error: D::f attempts to override final B::f
};
```

— end example]

- ⁵ If a virtual function is marked with the *virt-specifier* `override` and does not override a member function of a base class, the program is ill-formed.

[Example 4:

```
struct B {
    virtual void f(int);
};

struct D : B {
    virtual void f(long) override;    // error: wrong signature overriding B::f
    virtual void f(int) override;    // OK
};
```

— end example]

- ⁶ A virtual function shall not have a trailing *requires-clause* (9.3).

[Example 5:

```
template<typename T>
struct A {
    virtual void f() requires true;    // error: virtual function cannot be constrained (13.5.3)
};
```

— end example]

- ⁷ The *ref-qualifier*, or lack thereof, of an overriding function shall be the same as that of the overridden function.
- ⁸ The return type of an overriding function shall be either identical to the return type of the overridden function or *covariant* with the classes of the functions. If a function `D::f` overrides a function `B::f`, the return types of the functions are covariant if they satisfy the following criteria:
- (8.1) — both are pointers to classes, both are lvalue references to classes, or both are rvalue references to classes⁸⁸
 - (8.2) — the class in the return type of `B::f` is the same class as the class in the return type of `D::f`, or is an unambiguous and accessible direct or indirect base class of the class in the return type of `D::f`
 - (8.3) — both pointers or references have the same cv-qualification and the class type in the return type of `D::f` has the same cv-qualification as or less cv-qualification than the class type in the return type of `B::f`.
- ⁹ If the class type in the covariant return type of `D::f` differs from that of `B::f`, the class type in the return type of `D::f` shall be complete at the locus (6.4.2) of the overriding declaration or shall be the class type `D`. When the overriding function is called as the final overrider of the overridden function, its result is converted to the type returned by the (statically chosen) overridden function (7.6.1.3).

[Example 6:

```
class B { };
class D : private B { friend class Derived; };
struct Base {
    virtual void vf1();
    virtual void vf2();
    virtual void vf3();
    virtual B*   vf4();
    virtual B*   vf5();
};
```

⁸⁸) Multi-level pointers to classes or references to multi-level pointers to classes are not allowed.


```

    void f();
};

struct No_good : public Base {
    D* vf4();           // error: B (base class of D) inaccessible
};

class A;
struct Derived : public Base {
    void vf1();         // virtual and overrides Base::vf1()
    void vf2(int);      // not virtual, hides Base::vf2()
    char vf3();         // error: invalid difference in return type only
    D* vf4();           // OK, returns pointer to derived class
    A* vf5();           // error: returns pointer to incomplete class
    void f();
};

void g() {
    Derived d;
    Base* bp = &d;      // standard conversion:
                        // Derived* to Base*

    bp->vf1();           // calls Derived::vf1()
    bp->vf2();           // calls Base::vf2()
    bp->f();             // calls Base::f() (not virtual)
    B* p = bp->vf4();    // calls Derived::vf4() and converts the
                        // result to B*

    Derived* dp = &d;
    D* q = dp->vf4();    // calls Derived::vf4() and does not
                        // convert the result to B*
    dp->vf2();           // error: argument mismatch
}

```

— end example]

- 10 [Note 3: The interpretation of the call of a virtual function depends on the type of the object for which it is called (the dynamic type), whereas the interpretation of a call of a non-virtual member function depends only on the type of the pointer or reference denoting that object (the static type) (7.6.1.3). — end note]
- 11 [Note 4: The **virtual** specifier implies membership, so a virtual function cannot be a non-member (9.2.3) function. Nor can a virtual function be a static member, since a virtual function call relies on a specific object for determining which function to invoke. A virtual function declared in one class can be declared a friend (11.8.4) in another class. — end note]
- 12 A virtual function declared in a class shall be defined, or declared pure (11.7.4) in that class, or both; no diagnostic is required (6.3) (G.8.2).
- 13 [Example 7: Here are some uses of virtual functions with multiple base classes:

```

struct A {
    virtual void f();
};

struct B1 : A {           // note non-virtual derivation
    void f();
};

struct B2 : A {
    void f();
};

struct D : B1, B2 {       // D has two separate A subobjects
};

void foo() {
    D d;
    // A* ap = &d; // would be ill-formed: ambiguous
    B1* b1p = &d;
}

```

```

A*   ap = b1p;
D*   dp = &d;
ap->f();           // calls D::B1::f
dp->f();           // error: ambiguous
}

```

In class D above there are two occurrences of class A and hence two occurrences of the virtual member function A::f. The final overrider of B1::A::f is B1::f and the final overrider of B2::A::f is B2::f. — *end example*

- 14 [Example 8: The following example shows a function that does not have a unique final overrider:

```

struct A {
    virtual void f();
};

struct VB1 : virtual A {           // note virtual derivation
    void f();
};

struct VB2 : virtual A {
    void f();
};

struct Error : VB1, VB2 {         // error
};

struct Okay : VB1, VB2 {
    void f();
};

```

Both VB1::f and VB2::f override A::f but there is no overrider of both of them in class Error. This example is therefore ill-formed. Class Okay is well-formed, however, because Okay::f is a final overrider. — *end example*

- 15 [Example 9: The following example uses the well-formed classes from above.

```

struct VB1a : virtual A {         // does not declare f
};

struct Da : VB1a, VB2 {
};

void foe() {
    VB1a* vb1ap = new Da;
    vb1ap->f();                   // calls VB2::f
}

```

— *end example*

- 16 Explicit qualification with the scope operator (7.5.5.3) suppresses the virtual call mechanism.

[Example 10:

```

class B { public: virtual void f(); };
class D : public B { public: void f(); };

void D::f() { /* ... */ B::f(); }

```

Here, the function call in D::f really does call B::f and not D::f. — *end example*

- 17 A deleted function (9.6) shall not override a function that is not deleted. Likewise, a function that is not deleted shall not override a deleted function.

- 18 A non-immediate virtual function shall not be overridden by an immediate virtual function. An immediate virtual function shall not be overridden by a non-immediate virtual function.

11.7.4 Abstract classes

[class.abstract]

- 1 [Note 1: The abstract class mechanism supports the notion of a general concept, such as a **shape**, of which only more concrete variants, such as **circle** and **square**, can actually be used. An abstract class can also be used to define an interface for which derived classes provide a variety of implementations. — *end note*

- ² A virtual function is specified as a *pure virtual function* by using a *pure-specifier* (11.4) in the function declaration in the class definition.

[Note 2: Such a function might be inherited: see below. — end note]

A class is an *abstract class* if it has at least one pure virtual function.

[Note 3: An abstract class can be used only as a base class of some other class; no objects of an abstract class can be created except as subobjects of a class derived from it (6.2, 11.4). — end note]

A pure virtual function need be defined only if called with, or as if with (11.4.7), the *qualified-id* syntax (7.5.5.3).

[Example 1:

```
class point { /* ... */ };
class shape {                      // abstract class
public:
    point center;
    point where() { return center; }
    void move(point p) { center=p; draw(); }
    virtual void rotate(int) = 0; // pure virtual
    virtual void draw() = 0;      // pure virtual
};
```

— end example]

[Note 4: A function declaration cannot provide both a *pure-specifier* and a definition. — end note]

[Example 2:

```
struct C {
    virtual void f() = 0 { };      // error
};
```

— end example]

- ³ [Note 5: An abstract class type cannot be used as a parameter or return type of a function being defined (9.3.4.6) or called (7.6.1.3), except as specified in 9.2.9.3. Further, an abstract class type cannot be used as the type of an explicit type conversion (7.6.1.9, 7.6.1.10, 7.6.1.11), because the resulting prvalue would be of abstract class type (7.2.1). However, pointers and references to abstract class types can appear in such contexts. — end note]
- ⁴ A class is abstract if it has at least one pure virtual function for which the final overrider is pure virtual.

[Example 3:

```
class ab_circle : public shape {
    int radius;
public:
    void rotate(int) { }
    // ab_circle::draw() is a pure virtual
};
```

Since `shape::draw()` is a pure virtual function `ab_circle::draw()` is a pure virtual by default. The alternative declaration,

```
class circle : public shape {
    int radius;
public:
    void rotate(int) { }
    void draw();                      // a definition is required somewhere
};
```

would make class `circle` non-abstract and a definition of `circle::draw()` must be provided. — end example]

- ⁵ [Note 6: An abstract class can be derived from a class that is not abstract, and a pure virtual function can override a virtual function which is not pure. — end note]
- ⁶ Member functions can be called from a constructor (or destructor) of an abstract class; the effect of making a virtual call (11.7.3) to a pure virtual function directly or indirectly for the object being created (or destroyed) from such a constructor (or destructor) is undefined (F.6.2).

11.8 Member access control

[class.access]

11.8.1 General

[class.access.general]

- ¹ A member of a class can be

- (1.1) — private, that is, it can be named only by members and friends of the class in which it is declared;
- (1.2) — protected, that is, it can be named only by members and friends of the class in which it is declared, by classes derived from that class, and by their friends (see 11.8.5); or
- (1.3) — public, that is, it can be named anywhere without access restriction.

[*Note 1:* A constructor or destructor can be named by an expression (6.3) even though it has no name. — *end note*]

- 2 A member of a class can also access all the members to which the class has access. A local class of a member function may access the same members that the member function itself may access.⁸⁹
- 3 Members of a class defined with the keyword **class** are private by default. Members of a class defined with the keywords **struct** or **union** are public by default.

[*Example 1:*

```
class X {
    int a;           // X::a is private by default
};

struct S {
    int a;           // S::a is public by default
};
```

— *end example*]

- 4 Access control is applied uniformly to declarations and expressions.

[*Note 2:* Access control applies to members nominated by friend declarations (11.8.4) and *using-declarations* (9.10). — *end note*]

When a *using-declarator* is named, access control is applied to it, not to the declarations that replace it. For an overload set, access control is applied only to the function selected by overload resolution.

[*Example 2:*

```
struct S {
    void f(int);
private:
    void f(double);
};

void g(S* sp) {
    sp->f(2);        // OK, access control applied after overload resolution
}
```

— *end example*]

[*Note 3:* Because access control applies to the declarations named, if access control is applied to a type alias, only the accessibility of the typedef or alias declaration itself is considered. The accessibility of the underlying entity is not considered.

[*Example 3:*

```
class A {
    class B { };
public:
    typedef B BB;
};

void f() {
    A::BB x;         // OK, typedef A::BB is public
    A::B y;          // access error, A::B is private
}
```

— *end example*]

— *end note*]

- 5 [*Note 4:* Access control does not prevent members from being found by name lookup or implicit conversions to base classes from being considered. — *end note*]

⁸⁹⁾ Access permissions are thus transitive and cumulative to nested and local classes.

The interpretation of a given construct is established without regard to access control. If the interpretation established makes use of inaccessible members or base classes, the construct is ill-formed.

- ⁶ All access controls in 11.8 affect the ability to name a class member from the declaration of a particular entity, including parts of the declaration preceding the name of the entity being declared and, if the entity is a class, the definitions of members of the class appearing outside the class's *member-specification*.

[Note 5: This access also applies to implicit references to constructors, conversion functions, and destructors. — end note]

- ⁷ [Example 4:

```
class A {
    typedef int I;    // private member
    I f() pre(A::x > 0);
    friend I g(I) post(A::x <= 0);
    static I x;
    template<int> struct Q;
    template<int> friend struct R;
protected:
    struct B { };
};

A::I A::f() pre(A::x > 0) { return 0; }
A::I g(A::I p = A::x) post(A::x <= 0);
A::I g(A::I p) { return 0; }
A::I A::x = 0;
template<A::I> struct A::Q { };
template<A::I> struct R { };

struct D: A::B, A { };
```

Here, all the uses of `A::I` are well-formed because `A::f`, `A::x`, and `A::Q` are members of class `A` and `g` and `R` are friends of class `A`. This implies, for example, that access checking on the first use of `A::I` must be deferred until it is determined that this use of `A::I` is as the return type of a member of class `A`. Similarly, the use of `A::B` as a *base-specifier* is well-formed because `D` is derived from `A`, so checking of *base-specifiers* must be deferred until the entire *base-specifier-list* has been seen. — end example]

- ⁸ Access is checked for a default argument (9.3.4.7) at the point of declaration, rather than at any points of use of the default argument. Access checking for default arguments in function templates and in member functions of class templates is performed as described in 13.9.2.
- ⁹ Access for a default *template-argument* (13.2) is checked in the context in which it appears rather than at any points of use of it.

[Example 5:

```
class B { };
template <class T> class C {
protected:
    typedef T TT;
};

template <class U, class V = typename U::TT>
class D : public U { };

D <C<B>> >* d;    // access error, C::TT is protected
— end example]
```

11.8.2 Access specifiers

[class.access.spec]

- ¹ Member declarations can be labeled by an *access-specifier* (11.7):

access-specifier : *member-specification*_{opt}

An *access-specifier* specifies the access rules for members following it until the end of the class or until another *access-specifier* is encountered.

[Example 1:

```

class X {
    int a;           // X::a is private by default: class used
public:
    int b;           // X::b is public
    int c;           // X::c is public
};

```

— end example]

- ² Any number of access specifiers is allowed and no particular order is required.

[Example 2:

```

struct S {
    int a;           // S::a is public by default: struct used
protected:
    int b;           // S::b is protected
private:
    int c;           // S::c is private
public:
    int d;           // S::d is public
};

```

— end example]

- ³ When a member is redeclared within its class definition, the access specified at its redeclaration shall be the same as at its initial declaration.

[Example 3:

```

struct S {
    class A;
    enum E : int;
private:
    class A { };           // error: cannot change access
    enum E: int { e0 };     // error: cannot change access
};

```

— end example]

- ⁴ [Note 1: In a derived class, the lookup of a base class name will find the injected-class-name instead of the name of the base class in the scope in which it was declared. The injected-class-name might be less accessible than the name of the base class in the scope in which it was declared. — end note]

[Example 4:

```

class A { };
class B : private A { };
class C : public B {
    A* p;           // error: injected-class-name A is inaccessible
    ::A* q;         // OK
};

```

— end example]

11.8.3 Accessibility of base classes and base class members [class.access.base]

- ¹ If a class is declared to be a base class (11.7) for another class using the **public** access specifier, the public members of the base class are accessible as public members of the derived class and protected members of the base class are accessible as protected members of the derived class. If a class is declared to be a base class for another class using the **protected** access specifier, the public and protected members of the base class are accessible as protected members of the derived class. If a class is declared to be a base class for another class using the **private** access specifier, the public and protected members of the base class are accessible as private members of the derived class.⁹⁰
- ² In the absence of an *access-specifier* for a base class, **public** is assumed when the derived class is defined with the *class-key* **struct** and **private** is assumed when the class is defined with the *class-key* **class**.

⁹⁰) As specified previously in 11.8, private members of a base class remain inaccessible even to derived classes unless friend declarations within the base class definition are used to grant access explicitly.

[Example 1:

```
class B { /* ... */ };
class D1 : private B { /* ... */ };
class D2 : public B { /* ... */ };
class D3 : B { /* ... */ };           // B private by default
struct D4 : public B { /* ... */ };
struct D5 : private B { /* ... */ };
struct D6 : B { /* ... */ };         // B public by default
class D7 : protected B { /* ... */ };
struct D8 : protected B { /* ... */ };
```

Here B is a public base of D2, D4, and D6, a private base of D1, D3, and D5, and a protected base of D7 and D8. — end example]

- ³ [Note 1: A member of a private base class can be inaccessible as inherited, but accessible directly. Because of the rules on pointer conversions (7.3.12) and explicit casts (7.6.1.4, 7.6.1.9, 7.6.3), a conversion from a pointer to a derived class to a pointer to an inaccessible base class can be ill-formed if an implicit conversion is used, but well-formed if an explicit cast is used.

[Example 2:

```
class B {
public:
    int mi;           // non-static member
    static int si;    // static member
};
class D : private B {
};
class DD : public D {
    void f();
};

void DD::f() {
    mi = 3;           // error: mi is private in D
    si = 3;           // error: si is private in D
    ::B b;
    b.mi = 3;         // OK (b.mi is different from this->mi)
    b.si = 3;         // OK (b.si is different from this->si)
    ::B::si = 3;      // OK
    ::B* bp1 = this;  // error: B is a private base class
    ::B* bp2 = (::B*)this; // OK with cast
    bp2->mi = 3;       // OK, access through a pointer to B.
}
```

— end example]

— end note]

- ⁴ A base class B of N is *accessible* at R, if
- (4.1) — an invented public member of B would be a public member of N, or
 - (4.2) — R occurs in a direct member or friend of class N, and an invented public member of B would be a private or protected member of N, or
 - (4.3) — R occurs in a direct member or friend of a class P derived from N, and an invented public member of B would be a private or protected member of P, or
 - (4.4) — there exists a class S such that B is a base class of S accessible at R and S is a base class of N accessible at R.

[Example 3:

```
class B {
public:
    int m;
};

class S: private B {
    friend class N;
};
```

```

class N: private S {
    void f() {
        B* p = this;    // OK because class S satisfies the fourth condition above: B is a base class of N
                        // accessible in f() because B is an accessible base class of S and S is an accessible
                        // base class of N.
    }
};

```

— end example]

- ⁵ If a base class is accessible, one can implicitly convert a pointer to a derived class to a pointer to that base class (7.3.12, 7.3.13).

[Note 2: It follows that members and friends of a class *X* can implicitly convert an *X** to a pointer to a private or protected immediate base class of *X*. — end note]

An expression *E* that designates a member *m* has a *designating class* that affects the access to *m*. This designating class is either

- (5.1) — the innermost class of which *m* is directly a member if *E* is a *splice-expression* or
- (5.2) — the class in whose scope name lookup performed a search that found *m* otherwise.

[Note 3: This class can be explicit, e.g., when a *qualified-id* is used, or implicit, e.g., when a class member access operator (7.6.1.5) is used (including cases where an implicit “*this->*” is added). If both a class member access operator and a *qualified-id* are used to name the member (as in *p->T::m*), the class designating the member is the class designated by the *nested-name-specifier* of the *qualified-id* (that is, *T*). — end note]

A member *m* is accessible at the point *R* when designated in class *N* if

- (5.3) — *m* is designated by a *splice-expression*, or
- (5.4) — *m* as a member of *N* is public, or
- (5.5) — *m* as a member of *N* is private, and *R* occurs in a direct member or friend of class *N*, or
- (5.6) — *m* as a member of *N* is protected, and *R* occurs in a direct member or friend of class *N*, or in a member of a class *P* derived from *N*, where *m* as a member of *P* is public, private, or protected, or
- (5.7) — there exists a base class *B* of *N* that is accessible at *R*, and *m* is accessible at *R* when designated in class *B*.

[Example 4:

```

class B;
class A {
private:
    int i;
    friend void f(B*);
};
class B : public A { };
void f(B* p) {
    p->i = 1;    // OK, B* can be implicitly converted to A*, and f has access to i in A
}

```

— end example]

- ⁶ If a class member access operator, including an implicit “*this->*”, is used to access a non-static data member or non-static member function, the reference is ill-formed if the left operand (considered as a pointer in the “.” operator case) cannot be implicitly converted to a pointer to the designating class of the right operand.

[Note 4: This requirement is in addition to the requirement that the member be accessible as designated. — end note]

11.8.4 Friends

[class.friend]

- ¹ A friend of a class is a function or class that is given permission to name the private and protected members of the class. A class specifies its friends, if any, by way of friend declarations. Such declarations give special access rights to the friends, but they do not make the nominated friends members of the befriending class.

[Example 1: The following example illustrates the differences between members and friends:

```

class X {
    int a;
    friend void friend_set(X*, int);
}

```



```

public:
    void member_set(int);
};

void friend_set(X* p, int i) { p->a = i; }
void X::member_set(int i) { a = i; }

void f() {
    X obj;
    friend_set(&obj,10);
    obj.member_set(10);
}

```

— end example]

- ² Declaring a class to be a friend implies that private and protected members of the class granting friendship can be named in the *base-specifiers* and member declarations of the befriended class.

[Example 2:

```

class A {
    class B { };
    friend class X;
};

struct X : A::B {
    A::B mx;           // OK, A::B accessible to friend
    class Y {           // OK, A::B accessible to member of friend
        A::B my;
    };
};

```

— end example]

[Example 3:

```

class X {
    enum { a=100 };
    friend class Y;
};

class Y {
    int v[X::a];       // OK, Y is a friend of X
};

class Z {
    int v[X::a];       // error: X::a is private
};

```

— end example]

- ³ A friend declaration that does not declare a function shall be a *friend-type-declaration*.

[Note 1: A friend declaration can be the *declaration* in a *template-declaration* (13.1, 13.7.5). — end note]

If a *friend-type-specifier* in a friend declaration designates a (possibly cv-qualified) class type, that class is declared as a friend; otherwise, the *friend-type-specifier* is ignored.

[Example 4:

```

class C;
typedef C Ct;
class E;

class X1 {
    friend C;           // OK, class C is a friend
};

class X2 {
    friend Ct;          // OK, class C is a friend
    friend D;           // error: D not found
};

```

```

    friend class D;                // OK, elaborated-type-specifier declares new class
};

template <typename ... Ts> class R {
    friend Ts...;
};

template <class... Ts, class... Us>
class R<R<Ts...>, R<Us...>> {
    friend Ts::Nested..., Us...;
};

R<C> rc;                          // class C is a friend of R<C>
R<C, E> rce;                      // classes C and E are friends of R<C, E>
R<int> Ri;                        // OK, "friend int;" is ignored

struct E { struct Nested; };

R<R<E>, R<C, int>>> rr;           // E::Nested and C are friends of R<R<E>, R<C, int>>>
— end example]

```

- ⁴ [Note 2: A friend declaration refers to an entity, not (all overloads of) a name. A member function of a class X can be a friend of a class Y.

[Example 5:

```

class Y {
    friend char* X::foo(int);
    friend X::X(char);          // constructors can be friends
    friend X::~X();             // destructors can be friends
};

```

— end example]

— end note]

- ⁵ A function may be defined in a friend declaration of a class if and only if the class is a non-local class (11.6) and the function name is unqualified.

[Example 6:

```

class M {
    friend void f() { }          // definition of global f, a friend of M,
                                // not the definition of a member function
};

```

— end example]

- ⁶ Such a function is implicitly an inline (9.2.8) function if it is attached to the global module.

[Note 3: If a friend function is defined outside a class, it is not in the scope of the class. — end note]

- ⁷ No *storage-class-specifier* shall appear in the *decl-specifier-seq* of a friend declaration.

- ⁸ A member nominated by a friend declaration shall be accessible in the class containing the friend declaration. The meaning of the friend declaration is the same whether the friend declaration appears in the private, protected, or public (11.4) portion of the class *member-specification*.

- ⁹ Friendship is neither inherited nor transitive.

[Example 7:

```

class A {
    friend class B;
    int a;
};

class B {
    friend class C;
};

```

```

class C {
    void f(A* p) {
        p->a++;          // error: C is not a friend of A despite being a friend of a friend
    }
};

class D : public B {
    void f(A* p) {
        p->a++;          // error: D is not a friend of A despite being derived from a friend
    }
};
— end example]

```

¹⁰ [Note 4: A friend declaration never binds any names (9.3.4, 9.2.9.5). — end note]

[Example 8:

```

// Assume f and g have not yet been declared.
void h(int);
template <class T> void f2(T);
namespace A {
    class X {
        friend void f(X);          // A::f(X) is a friend
        class Y {
            friend void g();        // A::g is a friend
            friend void h(int);     // A::h is a friend
            // ::h not considered
            friend void f2<>(int);   // ::f2<>(int) is a friend
        };
    };

    // A::f, A::g and A::h are not visible here
    X x;
    void g() { f(x); }             // definition of A::g
    void f(X) { /* ... */ }        // definition of A::f
    void h(int) { /* ... */ }      // definition of A::h
    // A::f, A::g and A::h are visible here and known to be friends
}

using A::x;

void h() {
    A::f(x);
    A::X::f(x);                   // error: f is not a member of A::X
    A::X::Y::g();                 // error: g is not a member of A::X::Y
}

```

— end example]

[Example 9:

```

class X;
void a();
void f() {
    class Y;
    extern void b();
    class A {
        friend class X;           // OK, but X is a local class, not ::X
        friend class Y;           // OK
        friend class Z;           // OK, introduces local class Z
        friend void a();          // error, ::a is not considered
        friend void b();          // OK
        friend void c();          // error
    };
    X* px;                        // OK, but ::X is found
    Z* pz;                        // error: no Z is found
}

```

— end example]

11.8.5 Protected member access

[class.protected]

- ¹ An additional access check beyond those described earlier in 11.8 is applied when a non-static data member or non-static member function is a protected member of its designating class (11.8.3) and is not designated by a *splice-expression*.⁹¹ As described earlier, access to a protected member is granted because the reference occurs in a friend or direct member of some class C. If the access is to form a pointer to member (7.6.2.2), the *nested-name-specifier* shall designate C or a class derived from C. Otherwise, if the access involves a (possibly implicit) object expression (7.5.5.1, 7.6.1.5), the class of the object expression shall be C or a class derived from C.

[Example 1:

```
class B {
protected:
    int i;
    static int j;
};

class D1 : public B {
};

class D2 : public B {
    friend void fr(B*,D1*,D2*);
    void mem(B*,D1*);
};

void fr(B* pb, D1* p1, D2* p2) {
    pb->i = 1;           // error
    p1->i = 2;           // error
    p2->i = 3;           // OK (access through a D2)
    p2->B::i = 4;        // OK (access through a D2, even though designating class is B)
    int B::* pmi_B = &B::i; // error
    int B::* pmi_B2 = &D2::i; // OK (type of &D2::i is int B::*)
    B::j = 5;           // error: not a friend of designating class B
    D2::j = 6;          // OK (because refers to static member)
}

void D2::mem(B* pb, D1* p1) {
    pb->i = 1;           // error
    p1->i = 2;           // error
    i = 3;              // OK (access through this)
    B::i = 4;           // OK (access through this, qualification ignored)
    int B::* pmi_B = &B::i; // error
    int B::* pmi_B2 = &D2::i; // OK
    j = 5;              // OK (because j refers to static member)
    B::j = 6;           // OK (because B::j refers to static member)
}

void g(B* pb, D1* p1, D2* p2) {
    pb->i = 1;           // error
    p1->i = 2;           // error
    p2->i = 3;           // error
}
```

— end example]

11.8.6 Access to virtual functions

[class.access.virt]

- ¹ The access rules (11.8) for a virtual function are determined by its declaration and are not affected by the rules for a function that later overrides it.

⁹¹) This additional check does not apply to other members, e.g., static data members or enumerator member constants.

[Example 1:

```
class B {
public:
    virtual int f();
};

class D : public B {
private:
    int f();
};

void f() {
    D d;
    B* pb = &d;
    D* pd = &d;

    pb->f();           // OK, B::f() is public, D::f() is invoked
    pd->f();           // error: D::f() is private
}
```

— end example]

- ² Access is checked at the call point using the type of the expression used to denote the object for which the member function is called (B* in the example above). The access of the member function in the class in which it was defined (D in the example above) is in general not known.

11.8.7 Multiple access

[class.paths]

- ¹ If a declaration can be reached by several paths through a multiple inheritance graph, the access is that of the path that gives most access.

[Example 1:

```
class W { public: void f(); };
class A : private virtual W { };
class B : public virtual W { };
class C : public A, public B {
    void f() { W::f(); }    // OK
};
```

Since W::f() is available to C::f() along the public path through B, access is allowed. — end example]

11.8.8 Nested classes

[class.access.nest]

- ¹ A nested class is a member and as such has the same access rights as any other member. The members of an enclosing class have no special access to members of a nested class; the usual access rules (11.8) shall be obeyed.

[Example 1:

```
class E {
    int x;
    class B { };

    class I {
        B b;           // OK, E::I can access E::B
        int y;
        void f(E* p, int i) {
            p->x = i;    // OK, E::I can access E::x
        }
    };

    int g(I* p) {
        return p->y;    // error: I::y is private
    }
};
```

— end example]

11.9 Initialization

[class.init]

11.9.1 General

[class.init.general]

- ¹ When no initializer is specified for an object of (possibly cv-qualified) class type (or array thereof), or the initializer has the form `()`, the object is initialized as specified in 9.5.
- ² An object of class type (or array thereof) can be explicitly initialized; see 11.9.2 and 11.9.3.
- ³ When an array of class objects is initialized (either explicitly or implicitly) and the elements are initialized by constructor, the constructor shall be called for each element of the array, following the subscript order; see 9.3.4.5.

[Note 1: Destructors for the array elements are called in reverse order of their construction. — end note]

11.9.2 Explicit initialization

[class.expl.init]

- ¹ An object of class type can be initialized with a parenthesized *expression-list*, where the *expression-list* is construed as an argument list for a constructor that is called to initialize the object. Alternatively, a single *assignment-expression* can be specified as an *initializer* using the `=` form of initialization. Either direct-initialization semantics or copy-initialization semantics apply; see 9.5.

[Example 1:

```
struct complex {
    complex();
    complex(double);
    complex(double,double);
};

complex sqrt(complex,complex);

complex a(1);           // initialized by calling complex(double) with argument 1
complex b = a;          // initialized as a copy of a
complex c = complex(1,2); // initialized by calling complex(double,double) with arguments 1 and 2
complex d = sqrt(b,c);  // initialized by calling sqrt(complex,complex) with d as its result object
complex e;              // initialized by calling complex()
complex f = 3;          // initialized by calling complex(double) with argument 3
complex g = { 1, 2 };   // initialized by calling complex(double, double) with arguments 1 and 2
```

— end example]

[Note 1: Overloading of the assignment operator (12.4.3.2) has no effect on initialization. — end note]

- ² An object of class type can also be initialized by a *braced-init-list*. List-initialization semantics apply; see 9.5 and 9.5.5.

[Example 2:

```
complex v[6] = { 1, complex(1,2), complex(), 2 };
```

Here, `complex::complex(double)` is called for the initialization of `v[0]` and `v[3]`, `complex::complex(double, double)` is called for the initialization of `v[1]`, `complex::complex()` is called for the initialization of `v[2]`, `v[4]`, and `v[5]`. For another example,

```
struct X {
    int i;
    float f;
    complex c;
} x = { 99, 88.8, 77.7 };
```

Here, `x.i` is initialized with 99, `x.f` is initialized with 88.8, and `complex::complex(double)` is called for the initialization of `x.c`. — end example]

[Note 2: Braces can be elided in the *initializer-list* for any aggregate, even if the aggregate has members of a class type with user-defined type conversions; see 9.5.2. — end note]

- ³ [Note 3: If `T` is a class type with no default constructor, any initializing declaration of an object of type `T` (or array thereof) is ill-formed if no *initializer* is explicitly specified (see 11.9 and 9.5). — end note]
- ⁴ [Note 4: The order in which objects with static or thread storage duration are initialized is described in 6.10.3.3 and 8.10. — end note]

11.9.3 Initializing bases and members

[class.base.init]

- ¹ In the definition of a constructor for a class, initializers for direct and virtual base class subobjects and non-static data members can be specified by a *ctor-initializer*, which has the form

```

ctor-initializer:
    : mem-initializer-list

mem-initializer-list:
    mem-initializer ... opt
    mem-initializer-list , mem-initializer ... opt

mem-initializer:
    mem-initializer-id ( expression-listopt )
    mem-initializer-id braced-init-list

mem-initializer-id:
    class-or-decltype
    identifier

```

- ² Lookup for an unqualified name in a *mem-initializer-id* ignores the constructor's function parameter scope.

[*Note 1*: If the constructor's class contains a member with the same name as a direct or virtual base class of the class, a *mem-initializer-id* naming the member or base class and composed of a single identifier refers to the class member. A *mem-initializer-id* for the hidden base class can be specified using a qualified name. — *end note*]

Unless the *mem-initializer-id* names the constructor's class, a direct non-static data member of the constructor's class, or a direct or virtual base of that class, the *mem-initializer* is ill-formed.

- ³ A *mem-initializer-list* can initialize a base class using any *class-or-decltype* that denotes that base class type.

[*Example 1*:

```

struct A { A(); };
typedef A global_A;
struct B { };
struct C: public A, public B { C(); };
C::C(): global_A() { }           // mem-initializer for base A

```

— *end example*]

- ⁴ If a *mem-initializer-id* is ambiguous because it designates both a direct non-virtual base class and an indirect virtual base class, the *mem-initializer* is ill-formed.

[*Example 2*:

```

struct A { A(); };
struct B: public virtual A { };
struct C: public A, public B { C(); };
C::C(): A() { }                 // error: which A?

```

— *end example*]

- ⁵ A *ctor-initializer* may initialize a variant member of the constructor's class. If a *ctor-initializer* specifies more than one *mem-initializer* for the same member or for the same base class, the *ctor-initializer* is ill-formed.

- ⁶ A *mem-initializer-list* can delegate to another constructor of the constructor's class using any *class-or-decltype* that denotes the constructor's class itself. If a *mem-initializer-id* designates the constructor's class, it shall be the only *mem-initializer*; the constructor is a *delegating constructor*, and the constructor selected by the *mem-initializer* is the *target constructor*. The target constructor is selected by overload resolution. Once the target constructor returns, the body of the delegating constructor is executed. If a constructor delegates to itself directly or indirectly, the program is ill-formed, no diagnostic required (G.8.1).

[*Example 3*:

```

struct C {
    C( int ) { }                // #1: non-delegating constructor
    C(): C(42) { }              // #2: delegates to #1
    C( char c ) : C(42.0) { }   // #3: ill-formed due to recursion with #4
    C( double d ) : C('a') { } // #4: ill-formed due to recursion with #3
};

```

— *end example*]

- ⁷ The *expression-list* or *braced-init-list* in a *mem-initializer* is used to initialize the designated subobject (or, in the case of a delegating constructor, the complete class object) according to the initialization rules of 9.5 for direct-initialization.

[Example 4:

```
struct B1 { B1(int); /* ... */ };
struct B2 { B2(int); /* ... */ };
struct D : B1, B2 {
    D(int);
    B1 b;
    const int c;
};
```

```
D::D(int a) : B2(a+1), B1(a+2), c(a+3), b(a+4) { /* ... */ }
D d(10);
```

— end example]

[Note 2: The initialization performed by each *mem-initializer* constitutes a full-expression (6.10.1). Any expression in a *mem-initializer* is evaluated as part of the full-expression that performs the initialization. — end note]

A *mem-initializer* where the *mem-initializer-id* denotes a virtual base class is ignored during execution of a constructor of any class that is not the most derived class.

- ⁸ A temporary expression bound to a reference member in a *mem-initializer* is ill-formed.

[Example 5:

```
struct A {
    A() : v(42) { }    // error
    const int& v;
};
```

— end example]

- ⁹ In a non-delegating constructor other than an implicitly-defined copy/move constructor (11.4.5.3), if a given potentially constructed subobject is not designated by a *mem-initializer-id* (including the case where there is no *mem-initializer-list* because the constructor has no *ctor-initializer*), then

- (9.1) — if the entity is a non-static data member that has a default member initializer (11.4) and either
 - (9.1.1) — the constructor's class is a union (11.5), and no other variant member of that union is designated by a *mem-initializer-id* or
 - (9.1.2) — the constructor's class is not a union, and, if the entity is a member of an anonymous union, no other member of that union is designated by a *mem-initializer-id*,
 the entity is initialized from its default member initializer as specified in 9.5;
- (9.2) — otherwise, if the entity is an anonymous union or a variant member (11.5.2), no initialization is performed;
- (9.3) — otherwise, the entity is default-initialized (9.5).

[Note 3: An abstract class (11.7.4) is never a most derived class, thus its constructors never initialize virtual base classes, therefore the corresponding *mem-initializers* can be omitted. — end note]

[Note 4: After the call to a constructor for class **X** for an object with automatic or dynamic storage duration has completed, if the constructor was not invoked as part of value-initialization and a member of **X** is neither initialized nor given a value during execution of the *compound-statement* of the body of the constructor, the member has an indeterminate or erroneous value (6.8.5). — end note]

[Example 6:

```
struct A {
    A();
};

struct B {
    B(int);
};
```



```

struct C {
    C() { }           // initializes members as follows:
    A a;              // OK, calls A::A()
    const B b;        // error: B has no default constructor
    int i;             // OK, i has indeterminate or erroneous value
    int j = 5;        // OK, j has the value 5
};

```

— end example]

10 An attempt to initialize more than one non-static data member of a union renders the program ill-formed.

11 An object's initialization is considered complete when a non-delegating constructor for that object returns.

[Note 5: Therefore, an object's lifetime can begin (6.8.4) before all delegating constructors have completed. — end note]

12 If a given non-static data member has both a default member initializer and a *mem-initializer*, the initialization specified by the *mem-initializer* is performed, and the non-static data member's default member initializer is ignored.

[Example 7: Given

```

struct A {
    int i = /* some integer expression with side effects */ ;
    A(int arg) : i(arg) { }
    // ...
};

```

the `A(int)` constructor will simply initialize `i` to the value of `arg`, and the side effects in `i`'s default member initializer will not take place. — end example]

13 A temporary expression bound to a reference member from a default member initializer is ill-formed.

[Example 8:

```

struct A {
    A() = default;      // OK
    A(int v) : v(v) { } // OK
    const int& v = 42;   // OK
};
A a1;                  // error: ill-formed binding of temporary to reference
A a2(1);               // OK, unfortunately

```

— end example]

14 In a non-delegating constructor, the destructor for each potentially constructed subobject of class type is potentially invoked (11.4.7).

[Note 6: This provision ensures that destructors can be called for fully-constructed subobjects in case an exception is thrown (14.3). — end note]

15 In a non-delegating constructor, initialization proceeds in the following order:

- (15.1) — First, and only for the constructor of the most derived class (6.8.2), virtual base classes are initialized in the order they appear on a depth-first left-to-right traversal of the directed acyclic graph of base classes, where “left-to-right” is the order of appearance of the base classes in the derived class *base-specifier-list*.
- (15.2) — Then, direct base classes are initialized in declaration order as they appear in the *base-specifier-list* (regardless of the order of the *mem-initializers*).
- (15.3) — Then, non-static data members are initialized in the order they were declared in the class definition (again regardless of the order of the *mem-initializers*).
- (15.4) — Finally, the *compound-statement* of the constructor body is executed.

[Note 7: The declaration order is mandated to ensure that base and member subobjects are destroyed in the reverse order of initialization. — end note]

16 [Example 9:

```

struct V {
    V();
    V(int);
};

```

```

struct A : virtual V {
    A();
    A(int);
};

struct B : virtual V {
    B();
    B(int);
};

struct C : A, B, virtual V {
    C();
    C(int);
};

A::A(int i) : V(i) { /* ... */ }
B::B(int i) { /* ... */ }
C::C(int i) { /* ... */ }

V v(1);           // use V(int)
A a(2);           // use V(int)
B b(3);           // use V()
C c(4);           // use V()

```

— end example]

- 17 [Note 8: The *expression-list* or *braced-init-list* of a *mem-initializer* is in the function parameter scope of the constructor and can use **this** to refer to the object being initialized. — end note]

[Example 10:

```

class X {
    int a;
    int b;
    int i;
    int j;
public:
    const int& r;
    X(int i): r(a), b(i), i(i), j(this->i) { }
};

```

initializes **X::r** to refer to **X::a**, initializes **X::b** with the value of the constructor parameter **i**, initializes **X::i** with the value of the constructor parameter **i**, and initializes **X::j** with the value of **X::i**; this takes place each time an object of class **X** is created. — end example]

- 18 Member functions (including virtual member functions, 11.7.3) can be called for an object under construction or destruction. Similarly, an object under construction or destruction can be the operand of the **typeid** operator (7.6.1.8) or of a **dynamic_cast** (7.6.1.7). However, if these operations are performed during evaluation of
- (18.1) — a *ctor-initializer* (or in a function called directly or indirectly from a *ctor-initializer*) before all the *mem-initializers* for base classes have completed,
 - (18.2) — a precondition assertion of a constructor, or
 - (18.3) — a postcondition assertion of a destructor (9.4.1),

the program has undefined behavior (F.6.3).

[Example 11:

```

class A {
public:
    A(int);
};

class B : public A {
    int j;
public:
    int f();
};

```

```

    B() : A(f()),      // undefined behavior: calls member function but base A not yet initialized
    j(f()) { }        // well-defined: bases are all initialized
};

class C {
public:
    C(int);
};

class D : public B, C {
    int i;
public:
    D() : C(f()),      // undefined behavior: calls member function but base C not yet initialized
    i(f()) { }        // well-defined: bases are all initialized
};

```

— end example]

¹⁹ [Note 9: 11.9.5 describes the results of virtual function calls, typeid and dynamic_casts during construction for the well-defined cases; that is, describes the polymorphic behavior of an object under construction. — end note]

²⁰ A mem-initializer followed by an ellipsis is a pack expansion (13.7.4) that initializes the base classes specified by a pack expansion in the base-specifier-list for the class.

[Example 12:

```

template<class... Mixins>
class X : public Mixins... {
public:
    X(const Mixins&... mixins) : Mixins(mixins)... { }
};

```

— end example]

11.9.4 Initialization by inherited constructor

[class.inhctor.init]

¹ When a constructor for type B is invoked to initialize an object of a different type D (that is, when the constructor was inherited (9.10)), initialization proceeds as if a defaulted default constructor were used to initialize the D object and each base class subobject from which the constructor was inherited, except that the B subobject is initialized by the inherited constructor if the base class subobject were to be initialized as part of the D object (11.9.3). The invocation of the inherited constructor, including the evaluation of any arguments, is omitted if the B subobject is not to be initialized as part of the D object. The complete initialization is considered to be a single function call; in particular, unless omitted, the initialization of the inherited constructor's parameters is sequenced before the initialization of any part of the D object.

[Example 1:

```

struct B1 {
    B1(int, ...) { }
};

struct B2 {
    B2(double) { }
};

int get();

struct D1 : B1 {
    using B1::B1;      // inherits B1(int, ...)
    int x;
    int y = get();
};

void test() {
    D1 d(2, 3, 4);      // OK, B1 is initialized by calling B1(2, 3, 4),
                        // then d.x is default-initialized (no initialization is performed),
                        // then d.y is initialized by calling get()
    D1 e;               // error: D1 has no default constructor
}

```

```

}

struct D2 : B2 {
    using B2::B2;
    B1 b;
};

D2 f(1.0);           // error: B1 has no default constructor

struct W { W(int); };
struct X : virtual W { using W::W; X() = delete; };
struct Y : X { using X::X; };
struct Z : Y, virtual W { using Y::Y; };
Z z(0);              // OK, initialization of Y does not invoke default constructor of X

template<class T> struct Log : T {
    using T::T;       // inherits all constructors from class T
    ~Log() { std::clog << "Destroying wrapper" << std::endl; }
};

```

Class template Log wraps any class and forwards all of its constructors, while writing a message to the standard log whenever an object of class Log is destroyed. — *end example*

[Example 2:

```

struct V { V() = default; V(int); };
struct Q { Q(); };
struct A : virtual V, Q {
    using V::V;
    A() = delete;
};
int bar() { return 42; }
struct B : A {
    B() : A(bar()) {} // OK
};
struct C : B {};
void foo() { C c; } // bar is not invoked, because the V subobject is not initialized as part of B

```

— *end example*

- ² If the constructor was inherited from multiple base class subobjects of type B, the program is ill-formed.

[Example 3:

```

struct A { A(int); };
struct B : A { using A::A; };

struct C1 : B { using B::B; };
struct C2 : B { using B::B; };

struct D1 : C1, C2 {
    using C1::C1;
    using C2::C2;
};

struct V1 : virtual B { using B::B; };
struct V2 : virtual B { using B::B; };

struct D2 : V1, V2 {
    using V1::V1;
    using V2::V2;
};

D1 d1(0);           // error: ambiguous
D2 d2(0);           // OK, initializes virtual B base class, which initializes the A base class
                    // then initializes the V1 and V2 base classes as if by a defaulted default constructor

```

```

struct M { M(); M(int); };
struct N : M { using M::M; };
struct O : M {};
struct P : N, O { using N::N; using O::O; };
P p(0);           // OK, use M(0) to initialize N's base class,
                  // use M() to initialize O's base class

```

— end example]

- ³ When an object is initialized by an inherited constructor, initialization of the object is complete when the initialization of all subobjects is complete.

11.9.5 Construction and destruction

[class.cdtor]

- ¹ For an object with a non-trivial constructor, referring to any non-static member or base class of the object before the constructor begins execution results in undefined behavior (F.6.4). For an object with a non-trivial destructor, referring to any non-static member or base class of the object after the destructor finishes execution results in undefined behavior (F.6.5).

[Example 1:

```

struct X { int i; };
struct Y : X { Y(); };           // non-trivial
struct A { int a; };
struct B : public A { int j; Y y; }; // non-trivial

extern B bobj;
B* pb = &bobj;                   // OK
int* p1 = &bobj.a;               // undefined behavior: refers to base class member
int* p2 = &bobj.y.i;             // undefined behavior: refers to member's member

A* pa = &bobj;                   // undefined behavior: upcast to a base class type
B bobj;                          // definition of bobj

extern X xobj;
int* p3 = &xobj.i;               // OK, all constructors of X are trivial
X xobj;

```

For another example,

```

struct W { int j; };
struct X : public virtual W { };
struct Y {
    int* p;
    X x;
    Y() : p(&x.j) { // undefined, x is not yet constructed
    }
};

```

— end example]

- ² During the initialization of an object, if the value of any of its subobjects or any element of its object representation is accessed through a glvalue that is not obtained, directly or indirectly, from the constructor's `this` pointer, the value thus obtained is unspecified.

[Example 2:

```

struct C;
void no_opt(C*);

struct C {
    int c;
    C() : c(0) { no_opt(this); }
};

const C cobj;

void no_opt(C* cptr) {
    int i = cobj.c * 100; // value of cobj.c is unspecified
    cptr->c = 1;
}

```

```

    cout << cobj.c * 100           // value of cobj.c is unspecified
    << '\n';
}

extern struct D d;
struct D {
    D(int a) : a(a), b(d.a) {}
    int a, b;
};
D d = D(1);                       // value of d.b is unspecified
— end example]

```

- ³ To explicitly or implicitly convert a pointer (a glvalue) referring to an object of class *X* to a pointer (reference) to a direct or indirect base class *B* of *X*, the construction of *X* and the construction of all of its direct or indirect bases that directly or indirectly derive from *B* shall have started and the destruction of these classes shall not have completed, otherwise the conversion results in undefined behavior (F.6.6). To form a pointer to (or access the value of) a direct non-static member of an object *obj*, the construction of *obj* shall have started and its destruction shall not have completed, otherwise the computation of the pointer value (or accessing the member value) results in undefined behavior (F.6.7).

[Example 3:

```

struct A { };
struct B : virtual A { };
struct C : B { };
struct D : virtual A { D(A*); };
struct X { X(A*); };

struct E : C, D, X {
    E() : D(this),           // undefined behavior: upcast from E* to A* might use path E* → D* → A*
                                // but D is not constructed

                                // “D((C*)this)” would be defined: E* → C* is defined because E() has started,
                                // and C* → A* is defined because C is fully constructed

    X(this) {}               // defined: upon construction of X, C/B/D/A sublattice is fully constructed
};
— end example]

```

- ⁴ Member functions, including virtual functions (11.7.3), can be called during construction or destruction (11.9.3). When a virtual function is called directly or indirectly from a constructor or from a destructor, including during the construction or destruction of the class’s non-static data members, or during the evaluation of a postcondition assertion of a constructor or a precondition assertion of a destructor (9.4.1), and the object to which the call applies is the object (call it *x*) under construction or destruction, the function called is the final overrider in the constructor’s or destructor’s class and not one overriding it in a more-derived class. If the virtual function call uses an explicit class member access (7.6.1.5) and the object expression refers to the complete object of *x* or one of that object’s base class subobjects but not *x* or one of its base class subobjects, the behavior is undefined (F.6.8).

[Example 4:

```

struct V {
    virtual void f();
    virtual void g();
};

struct A : virtual V {
    virtual void f();
};

struct B : virtual V {
    virtual void g();
    B(V*, A*);
};

```

```

struct D : A, B {
    virtual void f();
    virtual void g();
    D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
    f();           // calls V::f, not A::f
    g();           // calls B::g, not D::g
    v->g();         // v is base of B, the call is well-defined, calls B::g
    a->f();         // undefined behavior: a's type not a base of B
}

```

— end example]

- ⁵ The `typeid` operator (7.6.1.8) can be used during construction or destruction (11.9.3). When `typeid` is used in a constructor (including the *mem-initializer* or default member initializer (11.4) for a non-static data member) or in a destructor, or used in a function called (directly or indirectly) from a constructor or destructor, if the operand of `typeid` refers to the object under construction or destruction, `typeid` yields the `std::type_info` object representing the constructor or destructor's class. If the operand of `typeid` refers to the object under construction or destruction and the static type of the operand is neither the constructor or destructor's class nor one of its bases, the behavior is undefined (F.6.9).
- ⁶ `dynamic_casts` (7.6.1.7) can be used during construction or destruction (11.9.3). When a `dynamic_cast` is used in a constructor (including the *mem-initializer* or default member initializer for a non-static data member) or in a destructor, or used in a function called (directly or indirectly) from a constructor or destructor, if the operand of the `dynamic_cast` refers to the object under construction or destruction, this object is considered to be a most derived object that has the type of the constructor or destructor's class. If the operand of the `dynamic_cast` refers to the object under construction or destruction and the static type of the operand is not a pointer to or object of the constructor or destructor's own class or one of its bases, the `dynamic_cast` results in undefined behavior (F.6.10).

[Example 5:

```

struct V {
    virtual void f();
};

struct A : virtual V { };

struct B : virtual V {
    B(V*, A*);
};

struct D : A, B {
    D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
    typeid(*this);           // type_info for B
    typeid(*v);              // well-defined: *v has type V, a base of B yields type_info for B
    typeid(*a);              // undefined behavior: type A not a base of B
    dynamic_cast<B*>(v);      // well-defined: v of type V*, V base of B results in B*
    dynamic_cast<B*>(a);      // undefined behavior: a has type A*, A not a base of B
}

```

— end example]

11.9.6 Copy/move elision

[class.copy.elision]

- ¹ When certain criteria are met, an implementation is allowed to omit the creation of a class object from a source object of the same type (ignoring cv-qualification), even if the selected constructor and/or the destructor for the object have side effects. In such cases, the implementation treats the source and target of the omitted initialization as simply two different ways of referring to the same object. If the first parameter of the selected constructor is an rvalue reference to the object's type, the destruction of that object occurs

when the target would have been destroyed; otherwise, the destruction occurs at the later of the times when the two objects would have been destroyed without the optimization.

[*Note 1*: Because only one object is destroyed instead of two, and the creation of one object is omitted, there is still one object destroyed for each one constructed. — *end note*]

This elision of object creation, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

- (1.1) — in a **return** statement (8.8.4) in a function with a class return type, when the *expression* is the name of a non-volatile object *o* with automatic storage duration (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler* (14.4)), the copy-initialization of the result object can be omitted by constructing *o* directly into the function call's result object;
- (1.2) — in a *throw-expression* (7.6.18), when the operand is the name of a non-volatile object *o* with automatic storage duration (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler*) that belongs to a scope that does not contain the innermost enclosing *compound-statement* associated with a *try-block* (if there is one), the copy-initialization of the exception object can be omitted by constructing *o* directly into the exception object;
- (1.3) — in a coroutine (9.6.4), a copy of a coroutine parameter can be omitted and references to that copy replaced with references to the corresponding parameter if the meaning of the program will be unchanged except for the execution of a constructor and destructor for the parameter copy object;
- (1.4) — when the *exception-declaration* of a *handler* (14.4) declares an object *o*, the copy-initialization of *o* can be omitted by treating the *exception-declaration* as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the *exception-declaration*.

[*Note 2*: There cannot be a move from the exception object because it is always an lvalue. — *end note*]

Copy elision is not permitted where an expression is evaluated in a context requiring a constant expression (7.7.3) and in constant initialization (6.10.3.2).

[*Note 3*: It is possible that copy elision is performed if the same expression is evaluated in another context. — *end note*]

² [*Example 1*:

```
class Thing {
public:
    Thing();
    ~Thing();
    Thing(const Thing&);
};

Thing f() {
    Thing t;
    return t;
}

Thing t2 = f();

struct A {
    void *p;
    constexpr A(): p(this) {}
};

constexpr A g() {
    A loc;
    return loc;
}

constexpr A a;           // well-formed, a.p points to a
constexpr A b = g();     // error: b.p would be dangling (7.7.3)

void h() {
    A c = g();           // well-formed, c.p can point to c or be dangling
}
```


Here the criteria for elision can eliminate the copying of the object `t` with automatic storage duration into the result object for the function call `f()`, which is the non-local object `t2`. Effectively, the construction of `t` can be viewed as directly initializing `t2`, and that object's destruction will occur at program exit. Adding a move constructor to `Thing` has the same effect, but it is the move construction from the object with automatic storage duration to `t2` that is elided. — *end example*]

³ [Example 2:

```
class Thing {
public:
    Thing();
    ~Thing();
    Thing(Thing&&);
private:
    Thing(const Thing&);
};

Thing f(bool b) {
    Thing t;
    if (b)
        throw t;          // OK, Thing(Thing&&) used (or elided) to throw t
    return t;              // OK, Thing(Thing&&) used (or elided) to return t
}

Thing t2 = f(false);      // OK, no extra copy/move performed, t2 constructed by call to f

struct Weird {
    Weird();
    Weird(Weird&);
};

Weird g(bool b) {
    static Weird w1;
    Weird w2;
    if (b)
        return w1;       // OK, uses Weird(Weird&)
    else
        return w2;       // error: w2 in this context is an xvalue
}

int& h(bool b, int i) {
    static int s;
    if (b)
        return s;        // OK
    else
        return i;        // error: i is an xvalue
}

decltype(auto) h2(Thing t) {
    return t;            // OK, t is an xvalue and h2's return type is Thing
}

decltype(auto) h3(Thing t) {
    return (t);          // OK, (t) is an xvalue and h3's return type is Thing&&
}

— end example]
```

⁴ [Example 3:

```
template<class T> void g(const T&);

template<class T> void f() {
    T x;
    try {
        T y;
        try { g(x); }
    }
}
```

```

    catch (...) {
        if (/*...*/)
            throw x;          // does not move
            throw y;          // moves
    }
    g(y);
} catch (...) {
    g(x);
    g(y);                    // error: y is not in scope
}
}
— end example]

```

11.10 Comparisons

[class.compare]

11.10.1 Defaulted comparison operator functions

[class.compare.default]

¹ A defaulted comparison operator function (12.4.3) shall be a non-template function that

- (1.1) — is a non-static member or friend of some class *C*,
- (1.2) — is defined as defaulted in *C* or in a context where *C* is complete, and
- (1.3) — either has two parameters of type `const C&` or two parameters of type *C*, where the implicit object parameter (if any) is considered to be the first parameter.

Such a comparison operator function is termed a defaulted comparison operator function for class *C*. Name lookups and access checks in the implicit definition (9.6.2) of a comparison operator function are performed from a context equivalent to its *function-body*. A definition of a comparison operator as defaulted that appears in a class shall be the first declaration of that function.

[Example 1:

```

struct S;
bool operator==(S, S) = default;          // error: S is not complete
struct S {
    friend bool operator==(S, const S&) = default; // error: parameters of different types
};
enum E { };
bool operator==(E, E) = default;          // error: not a member or friend of a class
— end example]

```

- ² A defaulted `<=>` or `==` operator function for class *C* is defined as deleted if any non-static data member of *C* is of reference type or *C* has variant members (11.5.2).
- ³ A binary operator expression *a* @ *b* is *usable* if either
 - (3.1) — *a* or *b* is of class or enumeration type and overload resolution (12.2) as applied to *a* @ *b* results in a usable candidate, or
 - (3.2) — neither *a* nor *b* is of class or enumeration type and *a* @ *b* is a valid expression.
- ⁴ If the *member-specification* does not explicitly declare any member or friend named `operator==`, an `==` operator function is declared implicitly for each three-way comparison operator function defined as defaulted in the *member-specification*, with the same access and *function-definition* and in the same class scope as the respective three-way comparison operator function, except that the return type is replaced with `bool` and the *declarator-id* is replaced with `operator==`.

[Note 1: Such an implicitly-declared `==` operator for a class *X* is defined as defaulted in the definition of *X* and has the same *parameter-declaration-clause* and trailing *requires-clause* as the respective three-way comparison operator. It is declared with `friend`, `virtual`, `constexpr`, or `consteval` if the three-way comparison operator function is so declared. If the three-way comparison operator function has no *noexcept-specifier*, the implicitly-declared `==` operator function has an implicit exception specification (14.5) that can differ from the implicit exception specification of the three-way comparison operator function. — end note]

[Example 2:

```

template<typename T> struct X {
    friend constexpr std::partial_ordering operator<=>(X, X) requires (sizeof(T) != 1) = default;
    // implicitly declares: friend constexpr bool operator==(X, X) requires (sizeof(T) != 1) = default;
};

```

```
[[nodiscard]] virtual std::strong_ordering operator<=>(const X&) const = default;
// implicitly declares: [[nodiscard]] virtual bool operator==(const X&) const = default;
};
— end example]
```

[Note 2: The == operator function is declared implicitly even if the defaulted three-way comparison operator function is defined as deleted. — end note]

- ⁵ The direct base class subobjects of *C*, in the order of their declaration in the *base-specifier-list* of *C*, followed by the non-static data members of *C*, in the order of their declaration in the *member-specification* of *C*, form a list of subobjects. In that list, any subobject of array type is recursively expanded to the sequence of its elements, in the order of increasing subscript. Let x_i be an lvalue denoting the i^{th} element in the expanded list of subobjects for an object *x* (of length *n*), where x_i is formed by a sequence of derived-to-base conversions (12.2.4.2), class member access expressions (7.6.1.5), and array subscript expressions (7.6.1.2) applied to *x*.

11.10.2 Equality operator

[class.eq]

- ¹ A defaulted equality operator function (12.4.3) shall have a declared return type `bool`.
- ² A defaulted == operator function for a class *C* is defined as deleted unless, for each x_i in the expanded list of subobjects for an object *x* of type *C*, $x_i == x_i$ is usable (11.10.1).
- ³ The return value of a defaulted == operator function with parameters *x* and *y* is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for *x* and *y* (in increasing index order) until the first index *i* where $x_i == y_i$ yields a result value which, when contextually converted to `bool`, yields `false`. The return value is `false` if such an index exists and `true` otherwise.
- ⁴ [Example 1:

```
struct D {
    int i;
    friend bool operator==(const D& x, const D& y) = default;
                                // OK, returns x.i == y.i
};
— end example]
```

11.10.3 Three-way comparison

[class.spaceship]

- ¹ The *synthesized three-way comparison* of type *R* (17.12.2) of glvalues *a* and *b* of the same type is defined as follows:
- (1.1) — If $a <=> b$ is usable (11.10.1) and can be explicitly converted to *R* using `static_cast`, `static_cast<R>(a <=> b)`.
- (1.2) — Otherwise, if $a <=> b$ is usable or overload resolution for $a <=> b$ is performed and finds at least one viable candidate, the synthesized three-way comparison is not defined.
- (1.3) — Otherwise, if *R* is not a comparison category type, or either the expression $a == b$ or the expression $a < b$ is not usable, the synthesized three-way comparison is not defined.
- (1.4) — Otherwise, if *R* is `strong_ordering`, then
- ```
 a == b ? strong_ordering::equal :
 a < b ? strong_ordering::less :
 strong_ordering::greater
```
- (1.5) — Otherwise, if *R* is `weak_ordering`, then
- ```
    a == b ? weak_ordering::equivalent :
    a < b  ? weak_ordering::less      :
            weak_ordering::greater
```
- (1.6) — Otherwise (when *R* is `partial_ordering`),
- ```
 a == b ? partial_ordering::equivalent :
 a < b ? partial_ordering::less :
 b < a ? partial_ordering::greater :
 partial_ordering::unordered
```

[Note 1: A synthesized three-way comparison is ill-formed if overload resolution finds usable candidates that do not otherwise meet the requirements implied by the defined expression. — end note]

- <sup>2</sup> Let **R** be the declared return type of a defaulted three-way comparison operator function, and let  $x_i$  be the elements of the expanded list of subobjects for an object **x** of type **C**.
- (2.1) — If **R** is **auto**, then let  $cv_i R_i$  be the type of the expression  $x_i <=> x_i$ . The operator function is defined as deleted if that expression is not usable or if  $R_i$  is not a comparison category type (17.12.2.1) for any  $i$ . The return type is deduced as the common comparison type (see below) of  $R_0, R_1, \dots, R_{n-1}$ .
  - (2.2) — Otherwise, **R** shall not contain a placeholder type. If the synthesized three-way comparison of type **R** between any objects  $x_i$  and  $x_i$  is not defined, the operator function is defined as deleted.
- <sup>3</sup> The return value of type **R** of the defaulted three-way comparison operator function with parameters **x** and **y** of the same type is determined by comparing corresponding elements  $x_i$  and  $y_i$  in the expanded lists of subobjects for **x** and **y** (in increasing index order) until the first index  $i$  where the synthesized three-way comparison of type **R** between  $x_i$  and  $y_i$  yields a result value  $v_i$  where  $v_i \neq 0$ , contextually converted to **bool**, yields **true**. The return value is a copy of  $v_i$  if such an index exists and `static_cast<R>(std::strong_ordering::equal)` otherwise.
- <sup>4</sup> The *common comparison type* **U** of a possibly-empty list of  $n$  comparison category types  $T_0, T_1, \dots, T_{n-1}$  is defined as follows:
- (4.1) — If at least one  $T_i$  is `std::partial_ordering`, **U** is `std::partial_ordering` (17.12.2.2).
  - (4.2) — Otherwise, if at least one  $T_i$  is `std::weak_ordering`, **U** is `std::weak_ordering` (17.12.2.3).
  - (4.3) — Otherwise, **U** is `std::strong_ordering` (17.12.2.4).
- [Note 2: In particular, this is the result when  $n$  is 0. — end note]

#### 11.10.4 Secondary comparison operators

[class.compare.secondary]

- <sup>1</sup> A *secondary comparison operator* is a relational operator (7.6.9) or the `!=` operator. A defaulted operator function (12.4.3) for a secondary comparison operator `@` shall have a declared return type **bool**.
- <sup>2</sup> The operator function with parameters **x** and **y** is defined as deleted if
- (2.1) — a first overload resolution (12.2), as applied to  $x @ y$ ,
    - (2.1.1) — does not result in a usable candidate, or
    - (2.1.2) — the selected candidate is not a rewritten candidate, or
  - (2.2) — a second overload resolution for the expression resulting from the interpretation of  $x @ y$  using the selected rewritten candidate (12.2.2.3) does not result in a usable candidate (for example, that expression might be  $(x <=> y) @ 0$ ), or
  - (2.3) —  $x @ y$  cannot be implicitly converted to **bool**.

In any of the two overload resolutions above, the defaulted operator function is not considered as a candidate for the `@` operator. Otherwise, the operator function yields  $x @ y$ .

<sup>3</sup> [Example 1:

```
struct HasNoLessThan { };

struct C {
 friend HasNoLessThan operator<=>(const C&, const C&);
 bool operator<(const C&) const = default; // OK, function is deleted
};
```

— end example]

# 12 Overloading

[over]

## 12.1 Preamble

[over.pre]

- <sup>1</sup> [Note 1: Each of two or more entities with the same name in the same scope, which must be functions or function templates, is commonly called an “overload”. — end note]
- <sup>2</sup> When a function is designated in a call, which function declaration is being referenced and the validity of the call are determined by comparing the types of the arguments at the point of use with the types of the parameters in the declarations in the overload set. This function selection process is called *overload resolution* and is defined in 12.2.

[Note 2: Overload sets are formed by *id-expressions* naming functions and function templates and by *splice-expressions* designating entities of the same kinds. — end note]

[Example 1:

```
double abs(double);
int abs(int);

abs(1); // calls abs(int);
abs(1.0); // calls abs(double);
```

— end example]

## 12.2 Overload resolution

[over.match]

### 12.2.1 General

[over.match.general]

- <sup>1</sup> Overload resolution is a mechanism for selecting the best function to call given a list of expressions that are to be the arguments of the call and a set of *candidate functions* that can be called based on the context of the call. The selection criteria for the best function are the number of arguments, how well the arguments match the parameter-type-list of the candidate function, how well (for non-static member functions) the object matches the object parameter, and certain other properties of the candidate function.
 

[Note 1: The function selected by overload resolution is not guaranteed to be appropriate for the context. Other restrictions, such as the accessibility of the function, can make its use in the calling context ill-formed. — end note]
- <sup>2</sup> Overload resolution selects the function to call in seven distinct contexts within the language:
  - (2.1) — invocation of a function named in the function call syntax (12.2.2.2.2);
  - (2.2) — invocation of a function call operator, a pointer-to-function conversion function, a reference-to-pointer-to-function conversion function, or a reference-to-function conversion function on a class object named in the function call syntax (12.2.2.2.3);
  - (2.3) — invocation of the operator referenced in an expression (12.2.2.3);
  - (2.4) — invocation of a constructor for default- or direct-initialization (9.5) of a class object (12.2.2.4);
  - (2.5) — invocation of a user-defined conversion for copy-initialization (9.5) of a class object (12.2.2.5);
  - (2.6) — invocation of a conversion function for initialization of an object of a non-class type from an expression of class type (12.2.2.6); and
  - (2.7) — invocation of a conversion function for conversion in which a reference (9.5.4) will be directly bound (12.2.2.7).

Each of these contexts defines the set of candidate functions and the list of arguments in its own unique way. But, once the candidate functions and argument lists have been identified, the selection of the best function is the same in all cases:

- (2.8) — First, a subset of the candidate functions (those that have the proper number of arguments and meet certain other conditions) is selected to form a set of viable functions (12.2.3).
- (2.9) — Then the best viable function is selected based on the implicit conversion sequences (12.2.4.2) needed to match each argument to the corresponding parameter of each viable function.

- <sup>3</sup> If a best viable function exists and is unique, overload resolution succeeds and produces it as the result. Otherwise overload resolution fails and the invocation is ill-formed. When overload resolution succeeds, and the best viable function is not accessible (11.8) in the context in which it is used, the program is ill-formed.
- <sup>4</sup> Overload resolution results in a *usable candidate* if overload resolution succeeds and the selected candidate is either not a function (12.5), or is a function that is not deleted and is accessible from the context in which overload resolution was performed.

## 12.2.2 Candidate functions and argument lists

[over.match.funcs]

### 12.2.2.1 General

[over.match.funcs.general]

- <sup>1</sup> The subclauses of 12.2.2 describe the set of candidate functions and the argument list submitted to overload resolution in each context in which overload resolution is used. The source transformations and constructions defined in these subclauses are only for the purpose of describing the overload resolution process. An implementation is not required to use such transformations and constructions.
- <sup>2</sup> The set of candidate functions can contain both member and non-member functions to be resolved against the same argument list. If a member function is
  - (2.1) — an implicit object member function that is not a constructor, or
  - (2.2) — a static member function and the argument list includes an implied object argument,
 it is considered to have an extra first parameter, called the *implicit object parameter*, which represents the object for which the member function has been called.
- <sup>3</sup> Similarly, when appropriate, the context can construct an argument list that contains an *implied object argument* as the first argument in the list to denote the object to be operated on.
- <sup>4</sup> For implicit object member functions, the type of the implicit object parameter is
  - (4.1) — “lvalue reference to *cv* X” for functions declared without a *ref-qualifier* or with the *& ref-qualifier*
  - (4.2) — “rvalue reference to *cv* X” for functions declared with the *&& ref-qualifier*

where X is the class of which the function is a direct member and *cv* is the cv-qualification on the member function declaration.

[*Example 1*: For a **const** member function of class X, the extra parameter is assumed to have type “lvalue reference to **const** X”. — *end example*]

For conversion functions that are implicit object member functions, the function is considered to be a member of the class of the implied object argument for the purpose of defining the type of the implicit object parameter. For non-conversion functions that are implicit object member functions nominated by a *using-declaration* in a derived class, the function is considered to be a member of the derived class for the purpose of defining the type of the implicit object parameter. For static member functions, the implicit object parameter is considered to match any object (since if the function is selected, the object is discarded).

[*Note 1*: No actual type is established for the implicit object parameter of a static member function, and no attempt will be made to determine a conversion sequence for that parameter (12.2.4). — *end note*]

- <sup>5</sup> During overload resolution, the implied object argument is indistinguishable from other arguments. The implicit object parameter, however, retains its identity since no user-defined conversions can be applied to achieve a type match with it. For implicit object member functions declared without a *ref-qualifier*, even if the implicit object parameter is not const-qualified, an rvalue can be bound to the parameter as long as in all other respects the argument can be converted to the type of the implicit object parameter.

[*Note 2*: The fact that such an argument is an rvalue does not affect the ranking of implicit conversion sequences (12.2.4.3). — *end note*]

- <sup>6</sup> Because other than in list-initialization only one user-defined conversion is allowed in an implicit conversion sequence, special rules apply when selecting the best user-defined conversion (12.2.4, 12.2.4.2).

[*Example 2*:

```
class T {
public:
 T();
};
```

```

class C : T {
public:
 C(int);
};
T a = 1; // error: no viable conversion (T(C(1)) not considered)
— end example]

```

- 7 In each case where conversion functions of a class *S* are considered for initializing an object or reference of type *T*, the candidate functions include the result of a search for the *conversion-function-id operator T* in *S*.  
[*Note 3*: This search can find a specialization of a conversion function template (6.5). — end note]

Each such case also defines sets of *permissible types* for explicit and non-explicit conversion functions; each (non-template) conversion function that

- (7.1) — is a non-hidden member of *S*,
- (7.2) — yields a permissible type, and,
- (7.3) — for the former set, is non-explicit

is also a candidate function. If initializing an object, for any permissible type *cv U*, any *cv2 U*, *cv2 U&*, or *cv2 U&&* is also a permissible type. If the set of permissible types for explicit conversion functions is empty, any candidates that are explicit are discarded.

- 8 In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction (13.10.4, 13.10.3). If a constructor template or conversion function template has an *explicit-specifier* whose *constant-expression* is value-dependent (13.8.3), template argument deduction is performed first and then, if the context admits only candidates that are not explicit and the generated specialization is explicit (9.2.3), it will be removed from the candidate set. Those candidates are then handled as candidate functions in the usual way.<sup>92</sup> A given name can refer to, or a conversion can consider, one or more function templates as well as a set of non-template functions. In such a case, the candidate functions generated from each function template are combined with the set of non-template candidate functions.
- 9 A defaulted move special member function (11.4.5.3, 11.4.6) that is defined as deleted is excluded from the set of candidate functions in all contexts. A constructor inherited from class type *C* (11.9.4) that has a first parameter of type “reference to *cv1 P*” (including such a constructor instantiated from a template) is excluded from the set of candidate functions when constructing an object of type *cv2 D* if the argument list has exactly one argument and *C* is reference-related to *P* and *P* is reference-related to *D*.

[*Example 3*:

```

struct A {
 A(); // #1
 A(A &&); // #2
 template<typename T> A(T &&); // #3
};
struct B : A {
 using A::A;
 B(const B &); // #4
 B(B &&) = default; // #5, implicitly deleted

 struct X { X(X &&) = delete; } x;
};
extern B b1;
B b2 = static_cast<B&&>(b1); // calls #4: #1 is not viable, #2, #3, and #5 are not candidates
struct C { operator B&&(); };
B b3 = C(); // calls #4
— end example]

```

<sup>92</sup>) The process of argument deduction fully determines the parameter types of the function template specializations, i.e., the parameters of function template specializations contain no template parameter types. Therefore, except where specified otherwise, function template specializations and non-template functions (9.3.4.6) are treated equivalently for the remainder of overload resolution.

**12.2.2.2 Function call syntax****[over.match.call]****12.2.2.2.1 General****[over.match.call.general]**

- <sup>1</sup> In a function call (7.6.1.3)

*postfix-expression* ( *expression-list<sub>opt</sub>* )

if the *postfix-expression* names at least one function or function template, overload resolution is applied as specified in 12.2.2.2.2. If the *postfix-expression* denotes an object of class type, overload resolution is applied as specified in 12.2.2.2.3.

- <sup>2</sup> If the *postfix-expression* is the address of an overload set, overload resolution is applied using that set as described above.

[Note 1: No implied object argument is added in this case. — end note]

If the function selected by overload resolution is an implicit object member function, the program is ill-formed.

[Note 2: The resolution of the address of an overload set in other contexts is described in 12.3. — end note]

**12.2.2.2.2 Call to designated function****[over.call.func]**

- <sup>1</sup> Of interest in 12.2.2.2.2 are only those function calls in which the *postfix-expression* ultimately contains an *id-expression* or *splice-expression* that designates one or more functions. Such a *postfix-expression*, perhaps nested arbitrarily deep in parentheses, has one of the following forms:

*postfix-expression*:

*postfix-expression* . *id-expression*

*postfix-expression* . *splice-expression*

*postfix-expression* -> *id-expression*

*postfix-expression* -> *splice-expression*

*id-expression*

*splice-expression*

These represent two syntactic subcategories of function calls: qualified function calls and unqualified function calls.

- <sup>2</sup> In qualified function calls, the function is designated by an *id-expression* or *splice-expression* *E* preceded by an -> or . operator. Since the construct *A*->*B* is generally equivalent to (\**A*).*B*, the rest of Clause 12 assumes, without loss of generality, that all member function calls have been normalized to the form that uses an object and the . operator. Furthermore, Clause 12 assumes that the *postfix-expression* that is the left operand of the . operator has type “*cv T*” where *T* denotes a class.<sup>93</sup> The set of candidate functions either is the set found by name lookup (6.5.2) if *E* is an *id-expression* or is the set determined as specified in 7.5.9 if *E* is a *splice-expression*. The argument list is the *expression-list* in the call augmented by the addition of the left operand of the . operator in the normalized member function call as the implied object argument (12.2.2).
- <sup>3</sup> In unqualified function calls, the function is designated by an *id-expression* or a *splice-expression* *E*. The set of candidate functions either is the set found by name lookup (6.5) if *E* is an *id-expression* or is the set determined as specified in 7.5.9 if *E* is a *splice-expression*. The set of candidate functions consists either entirely of non-member functions or entirely of member functions of some class *T*. In the former case or if *E* is either a *splice-expression* or the address of an overload set, the argument list is the same as the *expression-list* in the call. Otherwise, the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If the current class is, or is derived from, *T*, and the keyword **this** (7.5.3) refers to it,
- (3.1) — if the unqualified function call appears in a precondition assertion of a constructor or a postcondition assertion of a destructor and overload resolution selects a non-static member function, the call is ill-formed;
- (3.2) — otherwise, the implied object argument is (\***this**).
- Otherwise,
- (3.3) — if overload resolution selects a non-static member function, the call is ill-formed;
- (3.4) — otherwise, a contrived object of type *T* becomes the implied object argument.<sup>94</sup>

<sup>93</sup> Note that *cv*-qualifiers on the type of objects are significant in overload resolution for both glvalue and class prvalue objects.

<sup>94</sup> An implied object argument is contrived to correspond to the implicit object parameter attributed to member functions during overload resolution. It is not used in the call to the selected function. Since the member functions all have the same implicit object parameter, the contrived object will not be the cause to select or reject a function.



[Example 1:

```

struct C {
 bool a();
 void b() {
 a(); // OK, (*this).a()
 }

 void c(this const C&); // #1
 void c() &; // #2
 static void c(int = 0); // #3

 void d() {
 c(); // error: ambiguous between #2 and #3
 (C::c)(); // error: as above
 (&(C::c))(); // error: cannot resolve address of overloaded this->C::c (12.3)
 (&C::c)(C{}); // selects #1
 (&C::c)(*this); // error: selects #2, and is ill-formed (12.2.2.1)
 (&C::c)(); // selects #3
 }

 void f(this const C&);
 void g() const {
 f(); // OK, (*this).f()
 f(*this); // error: no viable candidate for (*this).f(*this)
 this->f(); // OK
 }

 static void h() {
 f(); // error: contrived object argument, but overload resolution
 // picked a non-static member function
 f(C{}); // error: no viable candidate
 C{}.f(); // OK
 }

 void k(this int);
 operator int() const;
 void m(this const C& c) {
 c.k(); // OK
 }

 C()
 pre(a()) // error: implied this in constructor precondition
 pre(this->a()) // OK
 post(a()); // OK
 ~C()
 pre(a()) // OK
 post(a()) // error: implied this in destructor postcondition
 post(this->a()); // OK
};

```

— end example]

### 12.2.2.2.3 Call to object of class type

[over.call.object]

- <sup>1</sup> If the *postfix-expression* E in the function call syntax evaluates to a class object of type “*cv T*”, then the set of candidate functions includes at least the function call operators of T. The function call operators of T are the results of a search for the name `operator()` in the scope of T.
- <sup>2</sup> In addition, for each non-explicit conversion function declared in T of the form

`operator conversion-type-id ( ) cv-qualifier-seqopt ref-qualifieropt noexcept-specifieropt attribute-specifier-seqopt ;`

where the optional *cv-qualifier-seq* is the same cv-qualification as, or a greater cv-qualification than, *cv*, and where *conversion-type-id* denotes the type “pointer to function of ( $P_1, \dots, P_n$ ) returning  $R$ ”, or the type “reference to pointer to function of ( $P_1, \dots, P_n$ ) returning  $R$ ”, or the type “reference to function of ( $P_1, \dots, P_n$ ) returning  $R$ ”, a *surrogate call function* with the unique name *call-function* and having a declaration of the form

```
R call-function (conversion-type-id F, P1 a1, ..., Pn an);
```

is also considered as a candidate function.

[*Note 1*: If a surrogate call function is selected by overload resolution, the behavior is as described in 12.4.4. — *end note*]

Similarly, surrogate call functions are added to the set of candidate functions for each non-explicit conversion function declared in a base class of  $T$  provided the function is not hidden within  $T$  by another intervening declaration.<sup>95</sup>

- <sup>3</sup> The argument list submitted to overload resolution consists of the argument expressions present in the function call syntax preceded by the implied object argument ( $E$ ).

[*Note 2*: When comparing the call against the function call operators, the implied object argument is compared against the object parameter of the function call operator. When comparing the call against a surrogate call function, the implied object argument is compared against the first parameter of the surrogate call function. — *end note*]

[*Example 1*:

```
int f1(int);
int f2(float);
typedef int (*fp1)(int);
typedef int (*fp2)(float);
struct A {
 operator fp1() { return f1; }
 operator fp2() { return f2; }
} a;
int i = a(1); // calls f1 via pointer returned from conversion function
```

— *end example*]

### 12.2.2.3 Operators in expressions

[over.match.oper]

- <sup>1</sup> If no operand of an operator in an expression has a type that is a class or an enumeration, the operator is assumed to be a built-in operator and interpreted according to 7.6.

[*Note 1*: Because `.`, `.*`, and `::` cannot be overloaded, these operators are always built-in operators interpreted according to 7.6. `?:` cannot be overloaded, but the rules in this subclause are used to determine the conversions to be applied to the second and third operands when they have class or enumeration type (7.6.16). — *end note*]

[*Example 1*:

```
struct String {
 String (const String&);
 String (const char*);
 operator const char* ();
};
String operator + (const String&, const String&);

void f() {
 const char* p= "one" + "two"; // error: cannot add two pointers; overloaded operator+ not considered
 // because neither operand has class or enumeration type
 int I = 1 + 1; // always evaluates to 2 even if class or enumeration types exist
 // that would perform the operation.
}
```

— *end example*]

- <sup>2</sup> If either operand has a type that is a class or an enumeration, a user-defined operator function can be declared that implements this operator or a user-defined conversion can be necessary to convert the operand to a type that is appropriate for a built-in operator. In this case, overload resolution is used to determine which

<sup>95</sup>) Note that this construction can yield candidate call functions that cannot be differentiated one from the other by overload resolution because they have identical declarations or differ only in their return type. The call will be ambiguous if overload resolution cannot select a match to the call that is uniquely better than such undifferentiable functions.

operator function or built-in operator is to be invoked to implement the operator. Therefore, the operator notation is first transformed to the equivalent function-call notation as summarized in Table 18 (where @ denotes one of the operators covered in the specified subclause). However, the operands are sequenced in the order prescribed for the built-in operator (7.6).

Table 18 — Relationship between operator and function call notation [tab:over.match.oper]

| Subclause | Expression | As member function | As non-member function |
|-----------|------------|--------------------|------------------------|
| 12.4.2    | @a         | (a).operator@ ( )  | operator@(a)           |
| 12.4.3    | a@b        | (a).operator@ (b)  | operator@(a, b)        |
| 12.4.3.2  | a=b        | (a).operator= (b)  |                        |
| 12.4.5    | a[b]       | (a).operator[] (b) |                        |
| 12.4.6    | a->        | (a).operator-> ( ) |                        |
| 12.4.7    | a@         | (a).operator@ (0)  | operator@(a, 0)        |

<sup>3</sup> For a unary operator @ with an operand of type *cv1* T1, and for a binary operator @ with a left operand of type *cv1* T1 and a right operand of type *cv2* T2, four sets of candidate functions, designated *member candidates*, *non-member candidates*, *built-in candidates*, and *rewritten candidates*, are constructed as follows:

- (3.1) — If T1 is a complete class type or a class currently being defined, the set of member candidates is the result of a search for **operator@** in the scope of T1; otherwise, the set of member candidates is empty.
- (3.2) — For the operators =, [], or ->, the set of non-member candidates is empty; otherwise, it includes the result of unqualified lookup for **operator@** in the rewritten function call (6.5.3, 6.5.4), ignoring all member functions. However, if no operand has a class type, only those non-member functions in the lookup set that have a first parameter of type T1 or “reference to *cv* T1”, when T1 is an enumeration type, or (if there is a right operand) a second parameter of type T2 or “reference to *cv* T2”, when T2 is an enumeration type, are candidate functions.
- (3.3) — For the operator ,, the unary operator &, or the operator ->, the built-in candidates set is empty. For all other operators, the built-in candidates include all of the candidate operator functions defined in 12.5 that, compared to the given operator,
  - (3.3.1) — have the same operator name, and
  - (3.3.2) — accept the same number of operands, and
  - (3.3.3) — accept operand types to which the given operand or operands can be converted according to 12.2.4.2, and
  - (3.3.4) — do not have the same parameter-type-list as any non-member candidate or rewritten non-member candidate that is not a function template specialization.
- (3.4) — The rewritten candidate set is determined as follows:
  - (3.4.1) — For the relational (7.6.9) operators, the rewritten candidates include all non-rewritten candidates for the expression **x <=> y**.
  - (3.4.2) — For the relational (7.6.9) and three-way comparison (7.6.8) operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression **y <=> x**.
  - (3.4.3) — For the != operator (7.6.10), the rewritten candidates include all non-rewritten candidates for the expression **x == y** that are rewrite targets with first operand x (see below).
  - (3.4.4) — For the equality operators, the rewritten candidates also include a synthesized candidate, with the order of the two parameters reversed, for each non-rewritten candidate for the expression **y == x** that is a rewrite target with first operand y.
  - (3.4.5) — For all other operators, the rewritten candidate set is empty.

[Note 2: A candidate synthesized from a member candidate has its object parameter as the second parameter, thus implicit conversions are considered for the first, but not for the second, parameter. — end note]

<sup>4</sup> A non-template function or function template F named **operator==** is a rewrite target with first operand o unless a search for the name **operator!=** in the scope S from the instantiation context of the operator expression finds a function or function template that would correspond (6.4.1) to F if its name were **operator==**,

where  $S$  is the scope of the class type of  $o$  if  $F$  is a class member, and the namespace scope of which  $F$  is a member otherwise. A function template specialization named `operator==` is a rewrite target if its function template is a rewrite target.

[Example 2:

```

struct A {};
template<typename T> bool operator==(A, T); // #1
bool a1 = 0 == A(); // OK, calls reversed #1
template<typename T> bool operator!=(A, T);
bool a2 = 0 == A(); // error, #1 is not a rewrite target

struct B {
 bool operator==(const B&); // #2
};
struct C : B {
 C();
 C(B);
 bool operator!=(const B&); // #3
};
bool c1 = B() == C(); // OK, calls #2; reversed #2 is not a candidate
// because search for operator!= in C finds #3
bool c2 = C() == B(); // error: ambiguous between #2 found when searching C and
// reversed #2 found when searching B

struct D {};
template<typename T> bool operator==(D, T); // #4
inline namespace N {
 template<typename T> bool operator!=(D, T); // #5
}
bool d1 = 0 == D(); // OK, calls reversed #4; #5 does not forbid #4 as a rewrite target

```

— end example]

- 5 For the first parameter of the built-in assignment operators, only standard conversion sequences (12.2.4.2.2) are considered.
- 6 For all other operators, no such restrictions apply.
- 7 The set of candidate functions for overload resolution for some operator  $@$  is the union of the member candidates, the non-member candidates, the built-in candidates, and the rewritten candidates for that operator  $@$ .
- 8 The argument list contains all of the operands of the operator. The best function from the set of candidate functions is selected according to 12.2.3 and 12.2.4.<sup>96</sup>

[Example 3:

```

struct A {
 operator int();
};
A operator+(const A&, const A&);
void m() {
 A a, b;
 a + b; // operator+(a, b) chosen over int(a) + int(b)
}

```

— end example]

- 9 If a rewritten `operator<=>` candidate is selected by overload resolution for an operator  $@$ ,  $x @ y$  is interpreted as  $0 @ (y <=> x)$  if the selected candidate is a synthesized candidate with reversed order of parameters, or  $(x <=> y) @ 0$  otherwise, using the selected rewritten `operator<=>` candidate. Rewritten candidates for the operator  $@$  are not considered in the context of the resulting expression.
- 10 If a rewritten `operator==` candidate is selected by overload resolution for an operator  $@$ , its return type shall be *cv bool*, and  $x @ y$  is interpreted as:

<sup>96</sup>) If the set of candidate functions is empty, overload resolution is unsuccessful.

- (10.1) — if @ is != and the selected candidate is a synthesized candidate with reversed order of parameters, !(y == x),
- (10.2) — otherwise, if @ is !=, !(x == y),
- (10.3) — otherwise (when @ is ==), y == x,

in each case using the selected rewritten `operator==` candidate.

- <sup>11</sup> If a built-in candidate is selected by overload resolution, the operands of class type are converted to the types of the corresponding parameters of the selected operation function, except that the second standard conversion sequence of a user-defined conversion sequence (12.2.4.2.3) is not applied. Then the operator is treated as the corresponding built-in operator and interpreted according to 7.6.

[Example 4:

```
struct X {
 operator double();
};

struct Y {
 operator int*();
};

int *a = Y() + 100.0; // error: pointer arithmetic requires integral operand
int *b = Y() + X(); // error: pointer arithmetic requires integral operand
```

— end example]

- <sup>12</sup> The second operand of `operator->` is ignored in selecting an `operator->` function, and is not an argument when the `operator->` function is called. When `operator->` returns, the `operator->` is applied to the value returned, with the original second operand.<sup>97</sup>
- <sup>13</sup> If the operator is the operator `,`, the unary operator `&`, or the operator `->`, and there are no viable functions, then the operator is assumed to be the built-in operator and interpreted according to 7.6.
- <sup>14</sup> [Note 3: The lookup rules for operators in expressions are different than the lookup rules for operator function names in a function call, as shown in the following example:

```
struct A { };
void operator + (A, A);

struct B {
 void operator + (B);
 void f ();
};

A a;

void B::f() {
 operator+ (a,a); // error: global operator hidden by member
 a + a; // OK, calls global operator+
}
```

— end note]

#### 12.2.2.4 Initialization by constructor

[over.match.ctor]

- <sup>1</sup> When objects of class type are direct-initialized (9.5), copy-initialized from an expression of the same or a derived class type (9.5), or default-initialized (9.5), overload resolution selects the constructor. For direct-initialization or default-initialization (including default-initialization in the context of copy-list-initialization), the candidate functions are all the constructors of the class of the object being initialized. Otherwise, the candidate functions are all the non-explicit constructors (11.4.8.2) of that class. The argument list is the *expression-list* or *assignment-expression* of the *initializer*; for default-initialization, the argument list is empty. For default-initialization in the context of copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed.

<sup>97</sup> If the value returned by the `operator->` function has class type, this can result in selecting and calling another `operator->` function. The process repeats until an `operator->` function returns a value of non-class type.

### 12.2.2.5 Copy-initialization of class by user-defined conversion [over.match.copy]

- 1 Under the conditions specified in 9.5, as part of a copy-initialization of an object of class type, a user-defined conversion can be invoked to convert an initializer expression to the type of the object being initialized. Overload resolution is used to select the user-defined conversion to be invoked.

[Note 1: The conversion performed for indirect binding to a reference to a possibly cv-qualified class type is determined in terms of a corresponding non-reference copy-initialization. — end note]

Assuming that “*cv1* T” is the type of the object being initialized, with T a class type, the candidate functions are selected as follows:

- (1.1) — The non-explicit constructors (11.4.8.2) of T are candidate functions.
- (1.2) — When the type of the initializer expression is a class type “*cv* S”, conversion functions are considered. The permissible types for non-explicit conversion functions are T and any class derived from T. When initializing a temporary object (11.4) to be bound to the first parameter of a constructor where the parameter is of type “reference to *cv2* T” and the constructor is called with a single argument in the context of direct-initialization of an object of type “*cv3* T”, the permissible types for explicit conversion functions are the same; otherwise there are none.
- 2 In both cases, the argument list has one argument, which is the initializer expression.

[Note 2: This argument will be compared against the first parameter of the constructors and against the object parameter of the conversion functions. — end note]

### 12.2.2.6 Initialization by conversion function [over.match.conv]

- 1 Under the conditions specified in 9.5, as part of an initialization of an object of non-class type, a conversion function can be invoked to convert an initializer expression of class type to the type of the object being initialized. Overload resolution is used to select the conversion function to be invoked. Assuming that “*cv* T” is the type of the object being initialized, the candidate functions are selected as follows:

- (1.1) — The permissible types for non-explicit conversion functions are those that can be converted to type T via a standard conversion sequence (12.2.4.2.2). For direct-initialization, the permissible types for explicit conversion functions are those that can be converted to type T with a (possibly trivial) qualification conversion (7.3.6); otherwise there are none.
- 2 The argument list has one argument, which is the initializer expression.

[Note 1: This argument will be compared against the object parameter of the conversion functions. — end note]

### 12.2.2.7 Initialization by conversion function for direct reference binding [over.match.ref]

- 1 Under the conditions specified in 9.5.4, a reference can be bound directly to the result of applying a conversion function to an initializer expression. Overload resolution is used to select the conversion function to be invoked. Assuming that “reference to *cv1* T” is the type of the reference being initialized, the candidate functions are selected as follows:

- (1.1) — Let *R* be the set of all
  - (1.1.1) — lvalue reference types (when converting to an lvalue) and
  - (1.1.2) — non-reference types and rvalue reference types (when converting to an rvalue or an lvalue of function type).

The permissible types for non-explicit conversion functions are the members of *R* having the form “*cv* T2” or “reference to *cv2* T2” where “*cv1* T” is reference-compatible (9.5.4) with “*cv2* T2”. For direct-initialization, the permissible types for explicit conversion functions are the members of *R* having the form “*cv* T2” or “reference to *cv2* T2” where T2 can be converted to type T with a (possibly trivial) qualification conversion (7.3.6); otherwise there are none.

- 2 The argument list has one argument, which is the initializer expression.

[Note 1: This argument will be compared against the object parameter of the conversion functions. — end note]

### 12.2.2.8 Initialization by list-initialization [over.match.list]

- 1 When objects of non-aggregate class type T are list-initialized such that 9.5.5 specifies that overload resolution is performed according to the rules in this subclause or when forming a list-initialization sequence according to 12.2.4.2.6, overload resolution selects the constructor in two phases:

- (1.1) — If the initializer list is not empty or *T* has no default constructor, overload resolution is first performed where the candidate functions are the initializer-list constructors (9.5.5) of the class *T* and the argument list consists of the initializer list as a single argument.
- (1.2) — Otherwise, or if no viable initializer-list constructor is found, overload resolution is performed again, where the candidate functions are all the constructors of the class *T* and the argument list consists of the elements of the initializer list.

In copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed.

[*Note 1*: This differs from other situations (12.2.2.4, 12.2.2.5), where only non-explicit constructors are considered for copy-initialization. This restriction only applies if this initialization is part of the final result of overload resolution. — *end note*]

#### 12.2.2.9 Class template argument deduction

[over.match.class.deduct]

- <sup>1</sup> When resolving a placeholder for a deduced class type (9.2.9.8) where the *template-name* or *splice-type-specifier* designates a primary class template *C*, a set of functions and function templates, called the guides of *C*, is formed comprising:

- (1.1) — If *C* is defined, for each constructor of *C*, a function template with the following properties:
  - (1.1.1) — The template parameters are the template parameters of *C* followed by the template parameters (including default template arguments) of the constructor, if any.
  - (1.1.2) — The associated constraints (13.5.3) are the conjunction of the associated constraints of *C* and the associated constraints of the constructor, if any.
 

[*Note 1*: A *constraint-expression* in the *template-head* of *C* is checked for satisfaction before any constraints from the *template-head* or trailing *requires-clause* of the constructor. — *end note*]
  - (1.1.3) — The *parameter-declaration-clause* is that of the constructor.
  - (1.1.4) — The return type is the class template specialization designated by *C* and template arguments corresponding to the template parameters of *C*.
- (1.2) — If *C* is not defined or does not declare any constructors, an additional function template derived as above from a hypothetical constructor *C*() .
- (1.3) — An additional function template derived as above from a hypothetical constructor *C*(*C*) , called the *copy deduction candidate*.
- (1.4) — For each *deduction-guide*, a function or function template with the following properties:
  - (1.4.1) — The *template-head*, if any, and *parameter-declaration-clause* are those of the *deduction-guide*.
  - (1.4.2) — The return type is the *simple-template-id* of the *deduction-guide*.

In addition, if *C* is defined and its definition satisfies the conditions for an aggregate class (9.5.2) with the assumption that any dependent base class has no virtual functions and no virtual base classes, and the initializer is a non-empty *braced-init-list* or parenthesized *expression-list*, and there are no *deduction-guides* for *C*, the set contains an additional function template, called the *aggregate deduction candidate*, defined as follows. Let  $x_1, \dots, x_n$  be the elements of the *initializer-list* or *designated-initializer-list* of the *braced-init-list*, or of the *expression-list*. For each  $x_i$ , let  $e_i$  be the corresponding aggregate element of *C* or of one of its (possibly recursive) subaggregates that would be initialized by  $x_i$  (9.5.2) if

- (1.5) — brace elision is not considered for any aggregate element that has
  - (1.5.1) — a dependent non-array type,
  - (1.5.2) — an array type with a value-dependent bound, or
  - (1.5.3) — an array type with a dependent array element type and  $x_i$  is a string literal; and
- (1.6) — each non-trailing aggregate element that is a pack expansion is assumed to correspond to no elements of the initializer list, and
- (1.7) — a trailing aggregate element that is a pack expansion is assumed to correspond to all remaining elements of the initializer list (if any).

If there is no such aggregate element  $e_i$  for any  $x_i$ , the aggregate deduction candidate is not added to the set. The aggregate deduction candidate is derived as above from a hypothetical constructor *C*(*T*<sub>1</sub>, ..., *T*<sub>*n*</sub>), where

- (1.8) — if  $e_i$  is of array type and  $x_i$  is a *braced-init-list*, *T*<sub>*i*</sub> is an rvalue reference to the declared type of  $e_i$ , and



- (1.9) — if  $e_i$  is of array type and  $x_i$  is a *string-literal*,  $T_i$  is an lvalue reference to the const-qualified declared type of  $e_i$ , and
- (1.10) — otherwise,  $T_i$  is the declared type of  $e_i$ ,

except that additional parameter packs of the form  $P_j \dots$  are inserted into the parameter list in their original aggregate element position corresponding to each non-trailing aggregate element of type  $P_j$  that was skipped because it was a parameter pack, and the trailing sequence of parameters corresponding to a trailing aggregate element that is a pack expansion (if any) is replaced by a single parameter of the form  $T_n \dots$ . In addition, if  $C$  is defined and inherits constructors (9.10) from a direct base class denoted in the *base-specifier-list* by a *class-or-decltype*  $B$ , let  $A$  be an alias template whose template parameter list is that of  $C$  and whose *defining-type-id* is  $B$ . If  $A$  is a deducible template (9.2.9.3), the set contains the guides of  $A$  with the return type  $R$  of each guide replaced with `typename CC<R>::type` given a class template

```
template <typename> class CC;
```

whose primary template is not defined and with a single partial specialization whose template parameter list is that of  $A$  and whose template argument list is a specialization of  $A$  with the template argument list of  $A$  (13.8.3.2) having a member typedef `type` designating a template specialization with the template argument list of  $A$  but with  $C$  as the template.

[Note 2: Equivalently, the template parameter list of the specialization is that of  $C$ , the template argument list of the specialization is  $B$ , and the member typedef names  $C$  with the template argument list of  $C$ . — end note]

<sup>2</sup> [Example 1:

```
template <typename T> struct B {
 B(T);
};
template <typename T> struct C : public B<T> {
 using B<T>::B;
};
template <typename T> struct D : public B<T> {};

C c(42); // OK, deduces C<int>
D d(42); // error: deduction failed, no inherited deduction guides
B(int) -> B<char>;
C c2(42); // OK, deduces C<char>

template <typename T> struct E : public B<int> {
 using B<int>::B;
};

E e(42); // error: deduction failed, arguments of E cannot be deduced from introduced guides

template <typename T, typename U, typename V> struct F {
 F(T, U, V);
};
template <typename T, typename U> struct G : F<U, T, int> {
 using G::F::F;
}

G g(true, 'a', 1); // OK, deduces G<char, bool>

template<class T, std::size_t N>
struct H {
 T array[N];
};
template<class T, std::size_t N>
struct I {
 volatile T array[N];
};
template<std::size_t N>
struct J {
 unsigned char array[N];
};
```



```

H h = { "abc" }; // OK, deduces H<char, 4> (not T = const char)
I i = { "def" }; // OK, deduces I<char, 4>
J j = { "ghi" }; // error: cannot bind reference to array of unsigned char to array of char in deduction
— end example]

```

- 3 When resolving a placeholder for a deduced class type where the *template-name* designates a type template parameter P, the type template template argument for P shall be a deducible template. Let A be an alias template whose template parameter list is that of P and whose *defining-type-id* is a *simple-template-id* whose *template-name* designates the type template template argument and whose *template-argument-list* is the template argument list of P. The alias template A is then used instead of the original *template-name* to resolve the placeholder.
- 4 When resolving a placeholder for a deduced class type (9.2.9.3) where the *template-name* or *splice-type-specifier* designates an alias template A, the *defining-type-id* of A must be of the form

`typenameopt nested-name-specifieropt templateopt simple-template-id`

as specified in 9.2.9.3. The guides of A are the set of functions or function templates formed as follows. For each function or function template f in the guides of the template named by the *simple-template-id* of the *defining-type-id*, the template arguments of the return type of f are deduced from the *defining-type-id* of A according to the process in 13.10.3.6 with the exception that deduction does not fail if not all template arguments are deduced. If deduction fails for another reason, proceed with an empty set of deduced template arguments. Let g denote the result of substituting these deductions into f. If substitution succeeds, form a function or function template f' with the following properties and add it to the set of guides of A:

- (4.1) — The function type of f' is the function type of g.
- (4.2) — If f is a function template, f' is a function template whose template parameter list consists of all the template parameters of A (including their default template arguments) that appear in the above deductions or (recursively) in their default template arguments, followed by the template parameters of f that were not deduced (including their default template arguments), otherwise f' is not a function template.
- (4.3) — The associated constraints (13.5.3) are the conjunction of the associated constraints of g and a constraint that is satisfied if and only if the arguments of A are deducible (see below) from the return type.
- (4.4) — If f is a copy deduction candidate, then f' is considered to be so as well.
- (4.5) — If f was generated from a *deduction-guide* (13.7.2.3), then f' is considered to be so as well.
- (4.6) — The *explicit-specifier* of f' is the *explicit-specifier* of g (if any).

- 5 The arguments of a template A are said to be deducible from a type T if, given a class template

```
template <typename> class AA;
```

with a single partial specialization whose template parameter list is that of A and whose template argument list is a specialization of A with the template argument list of A (13.8.3.2), AA<T> matches the partial specialization.

- 6 Initialization and overload resolution are performed as described in 9.5 and 12.2.2.4, 12.2.2.5, or 12.2.2.8 (as appropriate for the type of initialization performed) for an object of a hypothetical class type, where the guides of the template named by the placeholder are considered to be the constructors of that class type for the purpose of forming an overload set, and the initializer is provided by the context in which class template argument deduction was performed. The following exceptions apply:
  - (6.1) — The first phase in 12.2.2.8 (considering initializer-list constructors) is omitted if the initializer list consists of a single expression of type cv U, where U is, or is derived from, a specialization of the class template directly or indirectly named by the placeholder.
  - (6.2) — During template argument deduction for the aggregate deduction candidate, the number of elements in a trailing parameter pack is only deduced from the number of remaining function arguments if it is not otherwise deduced.

If the function or function template was generated from a constructor or *deduction-guide* that had an *explicit-specifier*, each such notional constructor is considered to have that same *explicit-specifier*. All such notional constructors are considered to be public members of the hypothetical class type.

- 7 [Example 2:

```

template <class T> struct A {
 explicit A(const T&, ...) noexcept; // #1
 A(T&&, ...); // #2
};

int i;
A a1 = { i, i }; // error: explicit constructor #1 selected in copy-list-initialization during deduction,
 // cannot deduce from non-forwarding rvalue reference in #2

A a2{i, i}; // OK, #1 deduces to A<int> and also initializes
A a3{0, i}; // OK, #2 deduces to A<int> and also initializes
A a4 = {0, i}; // OK, #2 deduces to A<int> and also initializes

template <class T> A(const T&, const T&) -> A<T&>; // #3
template <class T> explicit A(T&&, T&&) -> A<T>; // #4

A a5 = {0, 1}; // error: explicit deduction guide #4 selected in copy-list-initialization during deduction
A a6{0,1}; // OK, #4 deduces to A<int> and #2 initializes
A a7 = {0, i}; // error: #3 deduces to A<int&>, #1 and #2 declare same constructor
A a8{0,i}; // error: #3 deduces to A<int&>, #1 and #2 declare same constructor

template <class T> struct B {
 template <class U> using TA = T;
 template <class U> B(U, TA<U>);
};

B b{(int*)0, (char*)0}; // OK, deduces B<char*>

template <typename T>
struct S {
 T x;
 T y;
};

template <typename T>
struct C {
 S<T> s;
 T t;
};

template <typename T>
struct D {
 S<int> s;
 T t;
};

C c1 = {1, 2}; // error: deduction failed
C c2 = {1, 2, 3}; // error: deduction failed
C c3 = {{1u, 2u}, 3}; // OK, deduces C<int>

D d1 = {1, 2}; // error: deduction failed
D d2 = {1, 2, 3}; // OK, braces elided, deduces D<int>

template <typename T>
struct E {
 T t;
 decltype(t) t2;
};

E e1 = {1, 2}; // OK, deduces E<int>

template <typename... T>
struct Types {};

```

```

template <typename... T>
struct F : Types<T..., T... {}>;

struct X {};
struct Y {};
struct Z {};
struct W { operator Y(); };

F f1 = {Types<X, Y, Z>{}, {}, {}}; // OK, F<X, Y, Z> deduced
F f2 = {Types<X, Y, Z>{}, X{}, Y{}}; // OK, F<X, Y, Z> deduced
F f3 = {Types<X, Y, Z>{}, X{}, W{}}; // error: conflicting types deduced; operator Y not considered
— end example]

```

8 [Example 3:

```

template <class T, class U> struct C {
 C(T, U); // #1
};
template<class T, class U>
 C(T, U) -> C<T, std::type_identity_t<U>>; // #2

template<class V> using A = C<V *, V *>;
template<std::integral W> using B = A<W>;

int i{};
double d{};
A a1(&i, &i); // deduces A<int>
A a2(i, i); // error: cannot deduce V * from i
A a3(&i, &d); // error: #1: cannot deduce (V*, V*) from (int *, double *)
 // #2: cannot deduce A<V> from C<int *, double *>
B b1(&i, &i); // deduces B<int>
B b2(&d, &d); // error: cannot deduce B<W> from C<double *, double *>

```

Possible exposition-only implementation of the above procedure:

```

// The following concept ensures a specialization of A is deduced.
template <class> class AA;
template <class V> class AA<A<V>> { };
template <class T> concept deduces_A = requires { sizeof(AA<T>); };

// f1 is formed from the constructor #1 of C, generating the following function template
template<class T, class U>
 auto f1(T, U) -> C<T, U>;

// Deducing arguments for C<T, U> from C<V *, V*> deduces T as V * and U as V *;
// f1' is obtained by transforming f1 as described by the above procedure.
template<class V> requires deduces_A<C<V *, V *>>
 auto f1_prime(V *, V*) -> C<V *, V *>;

// f2 is formed from the deduction-guide #2 of C
template<class T, class U> auto f2(T, U) -> C<T, std::type_identity_t<U>>;

// Deducing arguments for C<T, std::type_identity_t<U>> from C<V *, V*> deduces T as V *;
// f2' is obtained by transforming f2 as described by the above procedure.
template<class V, class U>
 requires deduces_A<C<V *, std::type_identity_t<U>>>
 auto f2_prime(V *, U) -> C<V *, std::type_identity_t<U>>;

// The following concept ensures a specialization of B is deduced.
template <class> class BB;
template <class V> class BB<B<V>> { };
template <class T> concept deduces_B = requires { sizeof(BB<T>); };

```

// The guides for B derived from the above f1' and f2' for A are as follows:

```
template<std::integral W>
 requires deduces_A<C<W *, W *>> && deduces_B<C<W *, W *>>
 auto f1_prime_for_B(W *, W *) -> C<W *, W *>;

template<std::integral W, class U>
 requires deduces_A<C<W *, std::type_identity_t<U>>> &&
 deduces_B<C<W *, std::type_identity_t<U>>>
 auto f2_prime_for_B(W *, U) -> C<W *, std::type_identity_t<U>>;
```

— end example]

9 [Example 4:

```
template<typename ... Ts>
struct Y {
 Y();
 Y(Ts ...);
};

template<typename T = char> class X>
void f() {
 X x; // OK, deduces Y<char>
 X x0{}; // OK, deduces Y<char>
 X x1{1}; // OK, deduces Y<int>
 X x2{1, 2}; // error: cannot deduce X<T> from Y<int, int>
}

template void f<Y>();
```

— end example]

### 12.2.3 Viable functions

[over.match.viable]

<sup>1</sup> From the set of candidate functions constructed for a given context (12.2.2), a set of viable functions is chosen, from which the best function will be selected by comparing argument conversion sequences and associated constraints (13.5.3) for the best fit (12.2.4). The selection of viable functions considers associated constraints, if any, and relationships between arguments and function parameters other than the ranking of conversion sequences.

<sup>2</sup> First, to be a viable function, a candidate function shall have enough parameters to agree in number with the arguments in the list.

- (2.1) — If there are  $m$  arguments in the list, all candidate functions having exactly  $m$  parameters are viable.
- (2.2) — A candidate function having fewer than  $m$  parameters is viable only if it has an ellipsis in its parameter list (9.3.4.6). For the purposes of overload resolution, any argument for which there is no corresponding parameter is considered to “match the ellipsis” (12.2.4.2.4).
- (2.3) — A candidate function  $C$  having more than  $m$  parameters is viable only if the set of scopes  $G$ , as defined below, is not empty.  $G$  consists of every scope  $X$  that satisfies all of the following:
  - (2.3.1) — There is a declaration of  $C$ , whose host scope is  $X$ , considered by the overload resolution.
  - (2.3.2) — For every  $k^{\text{th}}$  parameter  $P$  where  $k > m$ , there is a reachable declaration, whose host scope is  $X$ , that specifies a default argument (9.3.4.7) for  $P$ .

If  $C$  is selected as the best viable function (12.2.4):

- (2.3.3) —  $G$  shall contain exactly one scope (call it  $S$ ).
- (2.3.4) — If the candidates are denoted by a *splice-expression*, then  $S$  shall not be a block scope.
- (2.3.5) — The default arguments used in the call to  $C$  are the default arguments specified by the reachable declarations whose host scope is  $S$ .

For the purposes of overload resolution, the parameter list is truncated on the right, so that there are exactly  $m$  parameters.

[Example 1:

```
namespace A {
 extern "C" void f(int, int = 5);
 extern "C" void f(int = 6, int);
}
```

```

namespace B {
 extern "C" void f(int, int = 7);
}

void use() {
 [::A::f:](3, 4); // OK, default argument was not used for viability
 [::A::f:](3); // error: default argument provided by declarations from two scopes
 [::A::f:](); // OK, default arguments provided by declarations in the scope of A

 using A::f;
 using B::f;
 f(3, 4); // OK, default argument was not used for viability
 f(3); // error: default argument provided by declaration from two scopes
 f(); // OK, default arguments provided by declarations in the scope of A

 void g(int = 8);
 g(); // OK
 [::g:](); // error: host scope is block scope
}

void h(int = 7);
constexpr std::meta::info r = h;
void poison() {
 void h(int = 8);
 h(); // OK, calls h(8)
 [::h:](); // error: default argument provided by declarations from two scopes
}
void call_h() {
 [::h:](); // error: default argument provided by declarations from two scopes
 [r:](); // error: default argument provided by declarations from two scopes
}

template<typename... Ts>
int k(int = 3, Ts...);
int i = k<int>(); // error: no default argument for the second parameter
int j = k<>(); // OK

```

— end example]

- 3 Second, for a function to be viable, if it has associated constraints (13.5.3), those constraints shall be satisfied (13.5.2).
- 4 Third, for  $F$  to be a viable function, there shall exist for each argument an implicit conversion sequence (12.2.4.2) that converts that argument to the corresponding parameter of  $F$ . If the parameter has reference type, the implicit conversion sequence includes the operation of binding the reference, and the fact that an lvalue reference to non-const cannot bind to an rvalue and that an rvalue reference cannot bind to an lvalue can affect the viability of the function (see 12.2.4.2.5).

## 12.2.4 Best viable function

[over.match.best]

### 12.2.4.1 General

[over.match.best.general]

- 1 Define  $ICS^i(F)$  as the implicit conversion sequence that converts the  $i^{\text{th}}$  argument in the list to the type of the  $i^{\text{th}}$  parameter of viable function  $F$ . 12.2.4.2 defines the implicit conversion sequences and 12.2.4.3 defines what it means for one implicit conversion sequence to be a better conversion sequence or worse conversion sequence than another.
- 2 Given these definitions, a viable function  $F_1$  is defined to be a *better* function than another viable function  $F_2$  if for all arguments  $i$ ,  $ICS^i(F_1)$  is not a worse conversion sequence than  $ICS^i(F_2)$ , and then
  - (2.1) — for some argument  $j$ ,  $ICS^j(F_1)$  is a better conversion sequence than  $ICS^j(F_2)$ , or, if not that,
  - (2.2) — the context is an initialization by user-defined conversion (see 9.5, 12.2.2.6, and 12.2.2.7) and the standard conversion sequence from the result of  $F_1$  to the destination type (i.e., the type of the entity being initialized) is a better conversion sequence than the standard conversion sequence from the result of  $F_2$  to the destination type

[Example 1:

```
struct A {
 A();
 operator int();
 operator double();
} a;
int i = a; // a.operator int() followed by no conversion is better than
 // a.operator double() followed by a conversion to int
float x = a; // ambiguous: both possibilities require conversions,
 // and neither is better than the other
```

— end example]

or, if not that,

- (2.3) — the context is an initialization by conversion function for direct reference binding (12.2.2.7) of a reference to function type, the return type of  $F_1$  is the same kind of reference (lvalue or rvalue) as the reference being initialized, and the return type of  $F_2$  is not

[Example 2:

```
template <class T> struct A {
 operator T&(); // #1
 operator T&&(); // #2
};
typedef int Fn();
A<Fn> a;
Fn& lf = a; // calls #1
Fn&& rf = a; // calls #2
```

— end example]

or, if not that,

- (2.4) —  $F_1$  is not a function template specialization and  $F_2$  is a function template specialization, or, if not that,  
 (2.5) —  $F_1$  and  $F_2$  are function template specializations, and the function template for  $F_1$  is more specialized than the template for  $F_2$  according to the partial ordering rules described in 13.7.7.3, or, if not that,  
 (2.6) —  $F_1$  and  $F_2$  are non-template functions and  $F_1$  is more partial-ordering-constrained than  $F_2$  (13.5.5)

[Example 3:

```
template <typename T = int>
struct S {
 constexpr void f(); // #1
 constexpr void f(this S&) requires true; // #2
};

void test() {
 S<> s;
 s.f(); // calls #2
}
```

— end example]

or, if not that,

- (2.7) —  $F_1$  is a constructor for a class D,  $F_2$  is a constructor for a base class B of D, and for all arguments the corresponding parameters of  $F_1$  and  $F_2$  have the same type

[Example 4:

```
struct A {
 A(int = 0);
};

struct B: A {
 using A::A;
 B();
};
```

```
int main() {
 B b; // OK, B::B()
}
```

— end example]

or, if not that,

- (2.8) —  $F_2$  is a rewritten candidate (12.2.2.3) and  $F_1$  is not

[Example 5:

```
struct S {
 friend auto operator<=>(const S&, const S&) = default; // #1
 friend bool operator<(const S&, const S&); // #2
};
bool b = S() < S(); // calls #2
```

— end example]

or, if not that,

- (2.9) —  $F_1$  and  $F_2$  are rewritten candidates, and  $F_2$  is a synthesized candidate with reversed order of parameters and  $F_1$  is not

[Example 6:

```
struct S {
 friend std::weak_ordering operator<=>(const S&, int); // #1
 friend std::weak_ordering operator<=>(int, const S&); // #2
};
bool b = 1 < S(); // calls #2
```

— end example]

or, if not that,

- (2.10) —  $F_1$  and  $F_2$  are generated from class template argument deduction (12.2.2.9) for a class D, and  $F_2$  is generated from inheriting constructors from a base class of D while  $F_1$  is not, and for each explicit function argument, the corresponding parameters of  $F_1$  and  $F_2$  are either both ellipses or have the same type, or, if not that,
- (2.11) —  $F_1$  is generated from a *deduction-guide* (12.2.2.9) and  $F_2$  is not, or, if not that,
- (2.12) —  $F_1$  is the copy deduction candidate (12.2.2.9) and  $F_2$  is not, or, if not that,
- (2.13) —  $F_1$  is generated from a non-template constructor and  $F_2$  is generated from a constructor template.

[Example 7:

```
template <class T> struct A {
 using value_type = T;
 A(value_type); // #1
 A(const A&); // #2
 A(T, T, int); // #3
 template<class U>
 A(int, T, U); // #4
 // #5 is the copy deduction candidate, A(A)
};

A x(1, 2, 3); // uses #3, generated from a non-template constructor

template <class T>
A(T) -> A<T>; // #6, less specialized than #5

A a(42); // uses #6 to deduce A<int> and #1 to initialize
A b = a; // uses #5 to deduce A<int> and #2 to initialize

template <class T>
A(A<T>) -> A<A<T>>; // #7, as specialized as #5

A b2 = a; // uses #7 to deduce A<A<int>> and #1 to initialize
```

— end example]

- <sup>3</sup> If there is exactly one viable function that is a better function than all other viable functions, then it is the one selected by overload resolution; otherwise the call is ill-formed.<sup>98</sup>

[Example 8:

```
void Fcn(const int*, short);
void Fcn(int*, int);

int i;
short s = 0;

void f() {
 Fcn(&i, s); // is ambiguous because &i → int* is better than &i → const int*
 // but s → short is also better than s → int

 Fcn(&i, 1L); // calls Fcn(int*, int), because &i → int* is better than &i → const int*
 // and 1L → short and 1L → int are indistinguishable

 Fcn(&i, 'c'); // calls Fcn(int*, int), because &i → int* is better than &i → const int*
 // and 'c' → int is better than 'c' → short
}
```

— end example]

- <sup>4</sup> [Note 1: If the best viable function was made viable by one or more default arguments, additional requirements apply (12.2.3). — end note]

#### 12.2.4.2 Implicit conversion sequences

[over.best.ics]

##### 12.2.4.2.1 General

[over.best.ics.general]

- <sup>1</sup> An *implicit conversion sequence* is a sequence of conversions used to convert an argument in a function call to the type of the corresponding parameter of the function being called. The sequence of conversions is an implicit conversion as defined in 7.3, which means it is governed by the rules for initialization of an object or reference by a single expression (9.5, 9.5.4).
- <sup>2</sup> Implicit conversion sequences are concerned only with the type, cv-qualification, and value category of the argument and how these are converted to match the corresponding properties of the parameter.

[Note 1: Other properties, such as the lifetime, storage duration, linkage, alignment, accessibility of the argument, whether the argument is a bit-field, and whether a function is deleted (9.6.3), are ignored. So, although an implicit conversion sequence can be defined for a given argument-parameter pair, the conversion from the argument to the parameter might still be ill-formed in the final analysis. — end note]

- <sup>3</sup> A well-formed implicit conversion sequence is one of the following forms:

- (3.1) — a standard conversion sequence (12.2.4.2.2),
- (3.2) — a user-defined conversion sequence (12.2.4.2.3), or
- (3.3) — an ellipsis conversion sequence (12.2.4.2.4).

- <sup>4</sup> However, if the target is

- (4.1) — the first parameter of a constructor or
- (4.2) — the object parameter of a user-defined conversion function

and the constructor or user-defined conversion function is a candidate by

- (4.3) — 12.2.2.4, when the argument is the temporary in the second step of a class copy-initialization,
- (4.4) — 12.2.2.5, 12.2.2.6, or 12.2.2.7 (in all cases), or
- (4.5) — the second phase of 12.2.2.8 when the initializer list has exactly one element that is itself an initializer list, and the target is the first parameter of a constructor of class X, and the conversion is to X or reference to cv X,

user-defined conversion sequences are not considered.

<sup>98</sup>) The algorithm for selecting the best viable function is linear in the number of viable functions. Run a simple tournament to find a function W that is not worse than any opponent it faced. Although it is possible that another function F that W did not face is at least as good as W, F cannot be the best function because at some point in the tournament F encountered another function G such that F was not better than G. Hence, either W is the best function or there is no best function. So, make a second pass over the viable functions to verify that W is better than all other functions.



[*Note 2:* These rules prevent more than one user-defined conversion from being applied during overload resolution, thereby avoiding infinite recursion. — *end note*]

[*Example 1:*

```
struct Y { Y(int); };
struct A { operator int(); };
Y y1 = A(); // error: A::operator int() is not a candidate

struct X { X(); };
struct B { operator X(); };
B b;
X x{b}; // error: B::operator X() is not a candidate
```

— *end example*]

- 5 For the case where the parameter type is a reference, see 12.2.4.2.5.
- 6 When the parameter type is not a reference, the implicit conversion sequence models a copy-initialization of the parameter from the argument expression. The implicit conversion sequence is the one required to convert the argument expression to a prvalue of the type of the parameter.

[*Note 3:* When the parameter has a class type, this is a conceptual conversion defined for the purposes of Clause 12; the actual initialization is defined in terms of constructors and is not a conversion. — *end note*]

- 7 When the cv-unqualified version of the type of the argument expression is the same as the parameter type, the implicit conversion sequence is an identity conversion. When the parameter has a class type and the argument expression has a (possibly cv-qualified) derived class type, the implicit conversion sequence is a derived-to-base conversion from the derived class to the base class. A derived-to-base conversion has Conversion rank (12.2.4.2.2).

[*Note 4:* There is no such standard conversion; this derived-to-base conversion exists only in the description of implicit conversion sequences. — *end note*]

[*Example 2:* An implicit conversion sequence from an argument of type `const A` to a parameter of type `A` can be formed, even if overload resolution for copy-initialization of `A` from the argument would not find a viable function (12.2.2.4, 12.2.3). The implicit conversion sequence for that case is the identity sequence; it contains no “conversion” from `const A` to `A`. — *end example*]

- 8 When the parameter is the implicit object parameter of a static member function, the implicit conversion sequence is a standard conversion sequence that is neither better nor worse than any other standard conversion sequence.
  - 9 In all contexts, when converting to the implicit object parameter or when converting to the left operand of an assignment operation only standard conversion sequences are allowed.
- [*Note 5:* When a conversion to the explicit object parameter occurs, it can include user-defined conversion sequences. — *end note*]
- 10 If no conversions are required to match an argument to a parameter type, the implicit conversion sequence is the standard conversion sequence consisting of the identity conversion (12.2.4.2.2).
  - 11 If no sequence of conversions can be found to convert an argument to a parameter type, an implicit conversion sequence cannot be formed.
  - 12 If there are multiple well-formed implicit conversion sequences converting the argument to the parameter type, the implicit conversion sequence associated with the parameter is defined to be the unique conversion sequence designated the *ambiguous conversion sequence*. For the purpose of ranking implicit conversion sequences as described in 12.2.4.3, the ambiguous conversion sequence is treated as a user-defined conversion sequence that is indistinguishable from any other user-defined conversion sequence.

[*Note 6:* This rule prevents a function from becoming non-viable because of an ambiguous conversion sequence for one of its parameters.

[*Example 3:*

```
class B;
class A { A (B&); };
class B { operator A (); };
class C { C (B&); };
void f(A) { }
void f(C) { }
B b;
```

```

f(b); // error: ambiguous because there is a conversion $b \rightarrow C$ (via constructor)
 // and an (ambiguous) conversion $b \rightarrow A$ (via constructor or conversion function)

void f(B) { }
f(b); // OK, unambiguous
— end example]
— end note]

```

If a function that uses the ambiguous conversion sequence is selected as the best viable function, the call will be ill-formed because the conversion of one of the arguments in the call is ambiguous.

- <sup>13</sup> The three forms of implicit conversion sequences mentioned above are defined in the following subclauses.

#### 12.2.4.2.2 Standard conversion sequences

[over.ics.scs]

- <sup>1</sup> [Table 19](#) summarizes the conversions defined in [7.3](#) and partitions them into four disjoint categories: Lvalue Transformation, Qualification Adjustment, Promotion, and Conversion.

[*Note 1*: These categories are orthogonal with respect to value category, cv-qualification, and data representation: the Lvalue Transformations do not change the cv-qualification or data representation of the type; the Qualification Adjustments do not change the value category or data representation of the type; and the Promotions and Conversions do not change the value category or cv-qualification of the type. — end note]

- <sup>2</sup> [*Note 2*: As described in [7.3](#), a standard conversion sequence either is the Identity conversion by itself (that is, no conversion) or consists of one to three conversions from the other four categories. If there are two or more conversions in the sequence, the conversions are applied in the canonical order: **Lvalue Transformation, Promotion or Conversion, Qualification Adjustment**. — end note]
- <sup>3</sup> Each conversion in [Table 19](#) also has an associated rank (Exact Match, Promotion, or Conversion). These are used to rank standard conversion sequences ([12.2.4.3](#)). The rank of a conversion sequence is determined by considering the rank of each conversion in the sequence and the rank of any reference binding ([12.2.4.2.5](#)). If any of those has Conversion rank, the sequence has Conversion rank; otherwise, if any of those has Promotion rank, the sequence has Promotion rank; otherwise, the sequence has Exact Match rank.

**Table 19 — Conversions** [tab:over.ics.scs]

| Conversion                     | Category                 | Rank                  | Subclause              |
|--------------------------------|--------------------------|-----------------------|------------------------|
| No conversions required        | Identity                 | Exact Match           |                        |
| Lvalue-to-rvalue conversion    | Lvalue Transformation    |                       | <a href="#">7.3.2</a>  |
| Array-to-pointer conversion    |                          |                       | <a href="#">7.3.3</a>  |
| Function-to-pointer conversion |                          |                       | <a href="#">7.3.4</a>  |
| Qualification conversions      | Qualification Adjustment |                       | <a href="#">7.3.6</a>  |
| Function pointer conversion    |                          |                       | <a href="#">7.3.14</a> |
| Integral promotions            | Promotion                | Promotion             | <a href="#">7.3.7</a>  |
| Floating-point promotion       |                          | <a href="#">7.3.8</a> |                        |
| Integral conversions           | Conversion               | Conversion            | <a href="#">7.3.9</a>  |
| Floating-point conversions     |                          |                       | <a href="#">7.3.10</a> |
| Floating-integral conversions  |                          |                       | <a href="#">7.3.11</a> |
| Pointer conversions            |                          |                       | <a href="#">7.3.12</a> |
| Pointer-to-member conversions  |                          |                       | <a href="#">7.3.13</a> |
| Boolean conversions            |                          |                       | <a href="#">7.3.15</a> |

#### 12.2.4.2.3 User-defined conversion sequences

[over.ics.user]

- <sup>1</sup> A *user-defined conversion sequence* consists of an initial standard conversion sequence followed by a user-defined conversion ([11.4.8](#)) followed by a second standard conversion sequence. If the user-defined conversion is specified by a constructor ([11.4.8.2](#)), the initial standard conversion sequence converts the source type to the type of the first parameter of that constructor. If the user-defined conversion is specified by a conversion function ([11.4.8.3](#)), the initial standard conversion sequence converts the source type to the type of the object parameter of that conversion function.
- <sup>2</sup> The second standard conversion sequence converts the result of the user-defined conversion to the target type for the sequence; any reference binding is included in the second standard conversion sequence. Since an implicit conversion sequence is an initialization, the special rules for initialization by user-defined conversion

apply when selecting the best user-defined conversion for a user-defined conversion sequence (see 12.2.4 and 12.2.4.2).

- 3 If the user-defined conversion is specified by a specialization of a conversion function template, the second standard conversion sequence shall have Exact Match rank.
- 4 A conversion of an expression of class type to the same class type is given Exact Match rank, and a conversion of an expression of class type to a base class of that type is given Conversion rank, in spite of the fact that a constructor (i.e., a user-defined conversion function) is called for those cases.

#### 12.2.4.2.4 Ellipsis conversion sequences

[over.ics.ellipsis]

- 1 An ellipsis conversion sequence occurs when an argument in a function call is matched with the ellipsis parameter specification of the function called (see 7.6.1.3).

#### 12.2.4.2.5 Reference binding

[over.ics.ref]

- 1 When a parameter of type “reference to *cv* T” binds directly (9.5.4) to an argument expression:
  - (1.1) — If the argument expression has a type D that is a derived class of T, the implicit conversion sequence is a derived-to-base conversion from D to T (12.2.4.2).
  - (1.2) — Otherwise, if the type of the argument is possibly cv-qualified T, or if T is an array type of unknown bound with element type U and the argument has an array type of known bound whose element type is possibly cv-qualified U, the implicit conversion sequence is the identity conversion.
  - (1.3) — Otherwise, if T is a function type, the implicit conversion sequence is a function pointer conversion.
  - (1.4) — Otherwise, the implicit conversion sequence is a qualification conversion.

[Example 1:

```
struct A {};
struct B : public A {} b;
int f(A&);
int f(B&);
int i = f(b); // calls f(B&), an exact match, rather than f(A&), a conversion

void g() noexcept;
int h(void (&)() noexcept); // #1
int h(void (&)()); // #2
int j = h(g); // calls #1, an exact match, rather than #2, a function pointer conversion
```

— end example]

If the parameter binds directly to the result of applying a conversion function to the argument expression, the implicit conversion sequence is a user-defined conversion sequence (12.2.4.2.3) whose second standard conversion sequence is determined by the above rules.

- 2 When a parameter of reference type is not bound directly to an argument expression, the conversion sequence is the one required to convert the argument expression to the referenced type according to 12.2.4.2. Conceptually, this conversion sequence corresponds to copy-initializing a temporary of the referenced type with the argument expression. Any difference in top-level cv-qualification is subsumed by the initialization itself and does not constitute a conversion.
- 3 Except for an implicit object parameter, for which see 12.2.2, an implicit conversion sequence cannot be formed if it requires binding an lvalue reference other than a reference to a non-volatile **const** type to an rvalue or binding an rvalue reference to an lvalue of object type.

[Note 1: This means, for example, that a candidate function cannot be a viable function if it has a non-**const** lvalue reference parameter (other than the implicit object parameter) and the corresponding argument would require a temporary to be created to initialize the lvalue reference (see 9.5.4). — end note]

- 4 Other restrictions on binding a reference to a particular argument that are not based on the types of the reference and the argument do not affect the formation of an implicit conversion sequence, however.

[Example 2: A function with an “lvalue reference to **int**” parameter can be a viable candidate even if the corresponding argument is an **int** bit-field. The formation of implicit conversion sequences treats the **int** bit-field as an **int** lvalue and finds an exact match with the parameter. If the function is selected by overload resolution, the call will nonetheless be ill-formed because of the prohibition on binding a non-**const** lvalue reference to a bit-field (9.5.4). — end example]

## 12.2.4.2.6 List-initialization sequence

[over.ics.list]

- <sup>1</sup> When an argument is an initializer list (9.5.5), it is not an expression and special rules apply for converting it to a parameter type.
- <sup>2</sup> If the initializer list is a *designated-initializer-list* and the parameter is not a reference, a conversion is only possible if the parameter has an aggregate type that can be initialized from the initializer list according to the rules for aggregate initialization (9.5.2), in which case the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.

[Note 1: Aggregate initialization does not require that the members are declared in designation order. If, after overload resolution, the order does not match for the selected overload, the initialization of the parameter will be ill-formed (9.5.5).

[Example 1:

```
struct A { int x, y; };
struct B { int y, x; };
void f(A a, int); // #1
void f(B b, ...); // #2
void g(A a); // #3
void g(B b); // #4
void h() {
 f({.x = 1, .y = 2}, 0); // OK; calls #1
 f({.y = 2, .x = 1}, 0); // error: selects #1, initialization of a fails
 // due to non-matching member order (9.5.5)
 g({.x = 1, .y = 2}); // error: ambiguous between #3 and #4
}
```

— end example]

— end note]

- <sup>3</sup> Otherwise, if the parameter type is an aggregate class **X** and the initializer list has a single element of type *cv* **U**, where **U** is **X** or a class derived from **X**, the implicit conversion sequence is the one required to convert the element to the parameter type.
- <sup>4</sup> Otherwise, if the parameter type is a character array<sup>99</sup> and the initializer list has a single element that is an appropriately-typed *string-literal* (9.5.3), the implicit conversion sequence is the identity conversion.
- <sup>5</sup> Otherwise, if the parameter type is `std::initializer_list<X>` and all the elements of the initializer list can be implicitly converted to **X**, the implicit conversion sequence is the worst conversion necessary to convert an element of the list to **X**, or if the initializer list has no elements, the identity conversion. This conversion can be a user-defined conversion even in the context of a call to an initializer-list constructor.

[Example 2:

```
void f(std::initializer_list<int>);
f({}); // OK, f(initializer_list<int>) identity conversion
f({1,2,3}); // OK, f(initializer_list<int>) identity conversion
f({'a','b'}); // OK, f(initializer_list<int>) integral promotion
f({1.0}); // error: narrowing

struct A {
 A(std::initializer_list<double>); // #1
 A(std::initializer_list<std::complex<double>>); // #2
 A(std::initializer_list<std::string>); // #3
};
A a{ 1.0,2.0 }; // OK, uses #1

void g(A);
g({ "foo", "bar" }); // OK, uses #3

typedef int IA[3];
void h(const IA&);
h({ 1, 2, 3 }); // OK, identity conversion
```

— end example]

<sup>99</sup> Since there are no parameters of array type, this will only occur as the referenced type of a reference parameter.

- <sup>6</sup> Otherwise, if the parameter type is “array of N X” or “array of unknown bound of X”, if there exists an implicit conversion sequence from each element of the initializer list (and from {} in the former case if N exceeds the number of elements in the initializer list) to X, the implicit conversion sequence is the worst such implicit conversion sequence.
- <sup>7</sup> Otherwise, if the parameter is a non-aggregate class X and overload resolution per 12.2.2.8 chooses a single best constructor C of X to perform the initialization of an object of type X from the argument initializer list:
- (7.1) — If C is not an initializer-list constructor and the initializer list has a single element of type cv U, where U is X or a class derived from X, the implicit conversion sequence has Exact Match rank if U is X, or Conversion rank if U is derived from X.
  - (7.2) — Otherwise, the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.

If multiple constructors are viable but none is better than the others, the implicit conversion sequence is the ambiguous conversion sequence. User-defined conversions are allowed for conversion of the initializer list elements to the constructor parameter types except as noted in 12.2.4.2.

[Example 3:

```

struct A {
 A(std::initializer_list<int>);
};
void f(A);
f({ 'a', 'b' }); // OK, f(A(std::initializer_list<int>)) user-defined conversion

struct B {
 B(int, double);
};
void g(B);
g({ 'a', 'b' }); // OK, g(B(int, double)) user-defined conversion
g({ 1.0, 1.0 }); // error: narrowing

void f(B);
f({ 'a', 'b' }); // error: ambiguous f(A) or f(B)

struct C {
 C(std::string);
};
void h(C);
h({"foo"}); // OK, h(C(std::string("foo")))

struct D {
 D(A, C);
};
void i(D);
i({ {1,2}, {"bar"} }); // OK, i(D(A(std::initializer_list<int>{1,2}), C(std::string("bar"))))

```

— end example]

- <sup>8</sup> Otherwise, if the parameter has an aggregate type which can be initialized from the initializer list according to the rules for aggregate initialization (9.5.2), the implicit conversion sequence is a user-defined conversion sequence whose second standard conversion sequence is an identity conversion.

[Example 4:

```

struct A {
 int m1;
 double m2;
};

void f(A);
f({ 'a', 'b' }); // OK, f(A(int,double)) user-defined conversion
f({ 1.0 }); // error: narrowing

```

— end example]

- <sup>9</sup> Otherwise, if the parameter is a reference, see 12.2.4.2.5.

[Note 2: The rules in this subclause will apply for initializing the underlying temporary for the reference. — end note]

[Example 5:

```
struct A {
 int m1;
 double m2;
};

void f(const A&);
f({ 'a', 'b' }); // OK, f(A(int,double)) user-defined conversion
f({ 1.0 }); // error: narrowing

void g(const double &);
g({1}); // same conversion as int to double
```

— end example]

<sup>10</sup> Otherwise, if the parameter type is not a class:

- (10.1) — if the initializer list has one element that is not itself an initializer list, the implicit conversion sequence is the one required to convert the element to the parameter type;

[Example 6:

```
void f(int);
f({ 'a' }); // OK, same conversion as char to int
f({ 1.0 }); // error: narrowing
```

— end example]

- (10.2) — if the initializer list has no elements, the implicit conversion sequence is the identity conversion.

[Example 7:

```
void f(int);
f({ }); // OK, identity conversion
```

— end example]

<sup>11</sup> In all cases other than those enumerated above, no conversion is possible.

#### 12.2.4.3 Ranking implicit conversion sequences

[over.ics.rank]

<sup>1</sup> This subclause defines a partial ordering of implicit conversion sequences based on the relationships *better conversion sequence* and *worse conversion sequence*. If an implicit conversion sequence S1 is defined by these rules to be a better conversion sequence than S2, then it is also the case that S2 is a *worse conversion sequence* than S1. If conversion sequence S1 is neither better than nor worse than conversion sequence S2, S1 and S2 are said to be *indistinguishable conversion sequences*.

<sup>2</sup> When comparing the basic forms of implicit conversion sequences (as defined in 12.2.4.2)

- (2.1) — a standard conversion sequence (12.2.4.2.2) is a better conversion sequence than a user-defined conversion sequence or an ellipsis conversion sequence, and
- (2.2) — a user-defined conversion sequence (12.2.4.2.3) is a better conversion sequence than an ellipsis conversion sequence (12.2.4.2.4).

<sup>3</sup> Two implicit conversion sequences of the same form are indistinguishable conversion sequences unless one of the following rules applies:

- (3.1) — List-initialization sequence L1 is a better conversion sequence than list-initialization sequence L2 if
- (3.1.1) — L1 converts to `std::initializer_list<X>` for some X and L2 does not, or, if not that,
- (3.1.2) — L1 and L2 convert to arrays of the same element type, and either the number of elements  $n_1$  initialized by L1 is less than the number of elements  $n_2$  initialized by L2, or  $n_1 = n_2$  and L2 converts to an array of unknown bound and L1 does not,

even if one of the other rules in this paragraph would otherwise apply.

[Example 1:

```
void f1(int); // #1
void f1(std::initializer_list<long>); // #2
void g1() { f1({42}); } // chooses #2
```

```

void f2(std::pair<const char*, const char*>); // #3
void f2(std::initializer_list<std::string>); // #4
void g2() { f2({"foo","bar"}); } // chooses #4

```

— end example]

[Example 2:

```

void f(int (&&[])); // #1
void f(double (&&[])); // #2
void f(int (&&)[2]); // #3

f({1}); // Calls #1: Better than #2 due to conversion, better than #3 due to bounds
f({1.0}); // Calls #2: Identity conversion is better than floating-integral conversion
f({1.0, 2.0}); // Calls #2: Identity conversion is better than floating-integral conversion
f({1, 2}); // Calls #3: Converting to array of known bound is better than to unknown bound,
 // and an identity conversion is better than floating-integral conversion

```

— end example]

(3.2) — Standard conversion sequence **S1** is a better conversion sequence than standard conversion sequence **S2** if

- (3.2.1) — **S1** is a proper subsequence of **S2** (comparing the conversion sequences in the canonical form defined by 12.2.4.2.2, excluding any Lvalue Transformation; the identity conversion sequence is considered to be a subsequence of any non-identity conversion sequence) or, if not that,
- (3.2.2) — the rank of **S1** is better than the rank of **S2**, or **S1** and **S2** have the same rank and are distinguishable by the rules in the paragraph below, or, if not that,
- (3.2.3) — **S1** and **S2** include reference bindings (9.5.4) and neither refers to an implicit object parameter of a non-static member function declared without a *ref-qualifier*, and **S1** binds an rvalue reference to an rvalue and **S2** binds an lvalue reference

[Example 3:

```

int i;
int f1();
int&& f2();
int g(const int&);
int g(const int&&);
int j = g(i); // calls g(const int&)
int k = g(f1()); // calls g(const int&&)
int l = g(f2()); // calls g(const int&&)

struct A {
 A& operator<<(int);
 void p() &;
 void p() &&;
};
A& operator<<(A&&, char);
A() << 1; // calls A::operator<<(int)
A() << 'c'; // calls operator<<(A&&, char)
A a;
a << 1; // calls A::operator<<(int)
a << 'c'; // calls A::operator<<(int)
A().p(); // calls A::p()&
a.p(); // calls A::p()&

```

— end example]

or, if not that,

- (3.2.4) — **S1** and **S2** include reference bindings (9.5.4) and **S1** binds an lvalue reference to an lvalue of function type and **S2** binds an rvalue reference to an lvalue of function type

[Example 4:

```

int f(void(&)()); // #1
int f(void(&&)()); // #2
void g();
int i1 = f(g); // calls #1

```

— *end example*]

or, if not that,

- (3.2.5) — S1 and S2 differ only in their qualification conversion (7.3.6) and yield similar types T1 and T2, respectively (where a standard conversion sequence that is a reference binding is considered to yield the cv-unqualified referenced type), where T1 and T2 are not the same type, and `const` T2 is reference-compatible with T1 (9.5.4)

[*Example 5:*

```
int f(const volatile int *);
int f(const int *);
int i;
int j = f(&i); // calls f(const int*)
int g(const int*);
int g(const volatile int* const&);
int* p;
int k = g(p); // calls g(const int*)
```

— *end example*]

or, if not that,

- (3.2.6) — S1 and S2 bind “reference to T1” and “reference to T2”, respectively (9.5.4), where T1 and T2 are not the same type, and T2 is reference-compatible with T1

[*Example 6:*

```
int f(const int &);
int f(int &);
int g(const int &);
int g(int);

int i;
int j = f(i); // calls f(int &)
int k = g(i); // ambiguous

struct X {
 void f() const;
 void f();
};
void g(const X& a, X b) {
 a.f(); // calls X::f() const
 b.f(); // calls X::f()
}

int h(int (&)[]);
int h(int (&)[1]);
void g2() {
 int a[1];
 h(a); // calls h(int (&)[1])
}
```

— *end example*]

or, if not that,

- (3.2.7) — S1 and S2 bind the same reference type “reference to T” and have source types V1 and V2, respectively, where the standard conversion sequence from V1\* to T\* is better than the standard conversion sequence from V2\* to T\*.

[*Example 7:*

```
struct Z {};

struct A {
 operator Z&();
 operator const Z&(); // #1
};
```



```

struct B {
 operator Z();
 operator const Z&&(); // #2
};

const Z& r1 = A(); // OK, uses #1
const Z&& r2 = B(); // OK, uses #2
— end example]

```

- (3.3) — User-defined conversion sequence U1 is a better conversion sequence than another user-defined conversion sequence U2 if they contain the same user-defined conversion function or constructor or they initialize the same class in an aggregate initialization and in either case the second standard conversion sequence of U1 is better than the second standard conversion sequence of U2.

[Example 8:

```

struct A {
 operator short();
} a;
int f(int);
int f(float);
int i = f(a); // calls f(int), because short → int is
 // better than short → float.
— end example]

```

- <sup>4</sup> Standard conversion sequences are ordered by their ranks: an Exact Match is a better conversion than a Promotion, which is a better conversion than a Conversion. Two conversion sequences with the same rank are indistinguishable unless one of the following rules applies:

- (4.1) — A conversion that does not convert a pointer or a pointer to member to `bool` is better than one that does.
- (4.2) — A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.
- (4.3) — A conversion in either direction between floating-point type FP1 and floating-point type FP2 is better than a conversion in the same direction between FP1 and arithmetic type T3 if
- (4.3.1) — the floating-point conversion rank (6.9.7) of FP1 is equal to the rank of FP2, and
- (4.3.2) — T3 is not a floating-point type, or T3 is a floating-point type whose rank is not equal to the rank of FP1, or the floating-point conversion subrank (6.9.7) of FP2 is greater than the subrank of T3.

[Example 9:

```

int f(std::float32_t);
int f(std::float64_t);
int f(long long);
float x;
std::float16_t y;
int i = f(x); // calls f(std::float32_t) on implementations where
 // float and std::float32_t have equal conversion ranks
int j = f(y); // error: ambiguous, no equal conversion rank
— end example]

```

- (4.4) — If class B is derived directly or indirectly from class A, conversion of `B*` to `A*` is better than conversion of `B*` to `void*`, and conversion of `A*` to `void*` is better than conversion of `B*` to `void*`.
- (4.5) — If class B is derived directly or indirectly from class A and class C is derived directly or indirectly from B,
- (4.5.1) — conversion of `C*` to `B*` is better than conversion of `C*` to `A*`,

[Example 10:

```

struct A {};
struct B : public A {};
struct C : public B {};
C* pc;
int f(A*);
int f(B*);
int i = f(pc); // calls f(B*)

```

— *end example*]

- (4.5.2) — conversion of  $A::*$  to  $B::*$  is better than conversion of  $A::*$  to  $C::*$ ,
- (4.5.3) — conversion of  $C$  to  $B$  is better than conversion of  $C$  to  $A$ ,
- (4.5.4) — conversion of  $B*$  to  $A*$  is better than conversion of  $C*$  to  $A*$ ,
- (4.5.5) — conversion of  $B::*$  to  $C::*$  is better than conversion of  $A::*$  to  $C::*$ , and
- (4.5.6) — conversion of  $B$  to  $A$  is better than conversion of  $C$  to  $A$ .

[*Note 1*: Compared conversion sequences will have different source types only in the context of comparing the second standard conversion sequence of an initialization by user-defined conversion (see 12.2.4); in all other contexts, the source types will be the same and the target types will be different. — *end note*]

## 12.3 Address of an overload set

[over.over]

- 1 An expression that designates an overload set  $S$  and that appears without arguments is resolved to a function, a pointer to function, or a pointer to member function for a specific function that is chosen from a set of functions selected from  $S$  determined based on the target type required in the context (if any), as described below. The target can be

- (1.1) — an object or reference being initialized (9.5, 9.5.4, 9.5.5),
- (1.2) — the left side of an assignment (7.6.19),
- (1.3) — a parameter of a function (7.6.1.3),
- (1.4) — a parameter of a user-defined operator (12.4),
- (1.5) — the return value of a function, operator function, or conversion (8.8.4),
- (1.6) — an explicit type conversion (7.6.1.4, 7.6.1.9, 7.6.3), or
- (1.7) — a constant template parameter (13.4.3).

If the target type contains a placeholder type, placeholder type deduction is performed (9.2.9.7.2), and the remainder of this subclause uses the target type so deduced. The expression can be preceded by the `&` operator.

[*Note 1*: Any redundant set of parentheses surrounding the function name is ignored (7.5.4). — *end note*]

- 2 If there is no target, all non-template functions named are selected. Otherwise, a non-template function with type  $F$  is selected for the function type  $FT$  of the target type if  $F$  (after possibly applying the function pointer conversion (7.3.14)) is identical to  $FT$ .

[*Note 2*: That is, the class of which the function is a member is ignored when matching a pointer-to-member-function type. — *end note*]

- 3 The specialization, if any, generated by template argument deduction (13.10.4, 13.10.3.3, 13.10.2) for each function template named is added to the set of selected functions considered.
- 4 Non-member functions, static member functions, and explicit object member functions match targets of function pointer type or reference to function type. Implicit object member functions match targets of pointer-to-member-function type.

[*Note 3*: If an implicit object member function is chosen, the result can be used only to form a pointer to member (7.6.2.2). — *end note*]

- 5 All functions with associated constraints that are not satisfied (13.5.3) are eliminated from the set of selected functions. If more than one function in the set remains, all function template specializations in the set are eliminated if the set also contains a function that is not a function template specialization. Any given non-template function  $F0$  is eliminated if the set contains a second non-template function that is more partial-ordering-constrained than  $F0$  (13.5.5). Any given function template specialization  $F1$  is eliminated if the set contains a second function template specialization whose function template is more specialized than the function template of  $F1$  according to the partial ordering rules of 13.7.7.3. After such eliminations, if any, there shall remain exactly one selected function.

- 6 [*Example 1*:

```
int f(double);
int f(int);
int (*pfd)(double) = &f; // selects f(double)
int (*pfi)(int) = &f; // selects f(int)
```

```

int (*pfe)(...) = &f; // error: type mismatch
int (&rfi)(int) = f; // selects f(int)
int (&rfd)(double) = f; // selects f(double)
void g() {
 (int (*)(int))&f; // cast expression as selector
}

```

The initialization of `pfe` is ill-formed because no `f()` with type `int(...)` has been declared, and not because of any ambiguity. — *end example*

[Example 2:

```

struct X {
 int f(int);
 static int f(long);
};

int (X::*p1)(int) = &X::f; // OK
int (*p2)(int) = &X::f; // error: mismatch
int (*p3)(long) = &X::f; // OK
int (X::*p4)(long) = &X::f; // error: mismatch
int (X::*p5)(int) = &(X::f); // error: wrong syntax for
 // pointer to member
int (*p6)(long) = &(X::f); // OK

```

— *end example*

[Example 3:

```

template<bool B> struct X {
 void f(short) requires B;
 void f(long);
 template<typename> void g(short) requires B;
 template<typename> void g(long);
};

void test() {
 &X<true>::f; // error: ambiguous; constraints are not considered
 &X<true>::g<int>; // error: ambiguous; constraints are not considered
}

```

— *end example*

<sup>7</sup> [Note 4: If `f` and `g` are both overload sets, the Cartesian product of possibilities is considered to resolve `f(&g)`, or the equivalent expression `f(g)`. — *end note*

<sup>8</sup> [Note 5: Even if `B` is a public base of `D`, we have

```

D* f();
B* (*p1)() = &f; // error

void g(D*);
void (*p2)(B*) = &g; // error

```

— *end note*

## 12.4 Overloaded operators

[over.oper]

### 12.4.1 General

[over.oper.general]

<sup>1</sup> A declaration whose *declarator-id* is an *operator-function-id* shall declare a function or function template or an explicit instantiation or specialization of a function template. A function so declared is an *operator function*. A function template so declared is an *operator function template*. A specialization of an operator function template is also an operator function. An operator function is said to *implement* the operator named in its *operator-function-id*.

*operator-function-id*:  
operator operator

*operator*: one of

|                  |                       |                       |                        |                           |                    |                    |                        |
|------------------|-----------------------|-----------------------|------------------------|---------------------------|--------------------|--------------------|------------------------|
| <code>new</code> | <code>delete</code>   | <code>new[]</code>    | <code>delete[]</code>  | <code>co_await ( )</code> | <code>[]</code>    | <code>-&gt;</code> | <code>-&gt;*</code>    |
| <code>~</code>   | <code>!</code>        | <code>+</code>        | <code>-</code>         | <code>*</code>            | <code>/</code>     | <code>%</code>     | <code>&amp;</code>     |
| <code> </code>   | <code>=</code>        | <code>+=</code>       | <code>-=</code>        | <code>*=</code>           | <code>/=</code>    | <code>%=</code>    | <code>&amp;=</code>    |
| <code> =</code>  | <code>==</code>       | <code>!=</code>       | <code>&lt;</code>      | <code>&gt;</code>         | <code>&lt;=</code> | <code>&gt;=</code> | <code>&lt;=&gt;</code> |
| <code>  </code>  | <code>&lt;&lt;</code> | <code>&gt;&gt;</code> | <code>&lt;&lt;=</code> | <code>&gt;&gt;=</code>    | <code>++</code>    | <code>--</code>    | <code>,</code>         |

[*Note 1*: The operators `new[]`, `delete[]`, `()`, and `[]` are formed from more than one token. The latter two operators are function call (7.6.1.3) and subscripting (7.6.1.2). — *end note*]

- 2 Both the unary and binary forms of

`+`      `-`      `*`      `&`

can be overloaded.

- 3 [*Note 2*: The following operators cannot be overloaded:

`.`      `.*`      `::`      `?:`

nor can the preprocessing symbols `#` (15.7.3) and `##` (15.7.4). — *end note*]

- 4 Operator functions are usually not called directly; instead they are invoked to evaluate the operators they implement (12.4.2 – 12.4.7). They can be explicitly called, however, using the *operator-function-id* as the name of the function in the function call syntax (7.6.1.3).

[*Example 1*:

```
complex z = a.operator+(b); // complex z = a+b;
void* p = operator new(sizeof(int)*n);
```

— *end example*]

- 5 The allocation and deallocation functions, `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`, are described completely in 6.8.6.5. The attributes and restrictions found in the rest of 12.4 do not apply to them unless explicitly stated in 6.8.6.5.
- 6 The `co_await` operator is described completely in 7.6.2.4. The attributes and restrictions found in the rest of 12.4 do not apply to it unless explicitly stated in 7.6.2.4.
- 7 An operator function shall have at least one function parameter or implicit object parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration. It is not possible to change the precedence, grouping, or number of operands of operators. The meaning of the operators `=`, (unary) `&`, and `,` (comma), predefined for each type, can be changed for specific class types by defining operator functions that implement these operators. Likewise, the meaning of the operators (unary) `&` and `,` (comma) can be changed for specific enumeration types. Operator functions are inherited in the same manner as other base class functions.
- 8 An operator function shall be a prefix unary, binary, function call, subscripting, class member access, increment, or decrement operator function.
- 9 [*Note 3*: The identities among certain predefined operators applied to fundamental types (for example, `++a`  $\equiv$  `a+=1`) need not hold for operator functions. Some predefined operators, such as `+=`, require an operand to be an lvalue when applied to fundamental types; this is not required by operator functions. — *end note*]
- 10 An operator function cannot have default arguments (9.3.4.7), except where explicitly stated below. Operator functions cannot have more or fewer parameters than the number required for the corresponding operator, as described in the rest of 12.4.
- 11 Operators not mentioned explicitly in subclauses 12.4.3.2 through 12.4.7 act as ordinary unary and binary operators obeying the rules of 12.4.2 or 12.4.3.

## 12.4.2 Unary operators

[over.unary]

- 1 A *prefix unary operator function* is a function named `operator@` for a prefix *unary-operator* `@` (7.6.2.2) that is either a non-static member function (11.4.2) with no non-object parameters or a non-member function with one parameter. For a *unary-expression* of the form `@ cast-expression`, the operator function is selected by overload resolution (12.2.2.3). If a member function is selected, the expression is interpreted as

`cast-expression . operator @ ( )`

Otherwise, if a non-member function is selected, the expression is interpreted as

`operator @ ( cast-expression )`

[*Note 1*: The operators `++` and `--` (7.6.2.3) are described in 12.4.7. — *end note*]

- <sup>2</sup> [Note 2: The unary and binary forms of the same operator have the same name. Consequently, a unary operator can hide a binary operator from an enclosing scope, and vice versa. — end note]

### 12.4.3 Binary operators

[over.binary]

#### 12.4.3.1 General

[over.binary.general]

- <sup>1</sup> A *binary operator function* is a function named `operator@` for a binary operator `@` that is either a non-static member function (11.4.2) with one non-object parameter or a non-member function with two parameters. For an expression `x @ y` with subexpressions `x` and `y`, the operator function is selected by overload resolution (12.2.2.3). If a member function is selected, the expression is interpreted as

$$x . \text{operator } @ ( y )$$

Otherwise, if a non-member function is selected, the expression is interpreted as

$$\text{operator } @ ( x , y )$$

- <sup>2</sup> An *equality operator function* is an operator function for an equality operator (7.6.10). A *relational operator function* is an operator function for a relational operator (7.6.9). A *three-way comparison operator function* is an operator function for the three-way comparison operator (7.6.8). A *comparison operator function* is an equality operator function, a relational operator function, or a three-way comparison operator function.

#### 12.4.3.2 Simple assignment

[over.assign]

- <sup>1</sup> A *simple assignment operator function* is a binary operator function named `operator=`. A simple assignment operator function shall be a non-static member function.

[Note 1: Because only standard conversion sequences are considered when converting to the left operand of an assignment operation (12.2.4.2), an expression `x = y` with a subexpression `x` of class type is always interpreted as `x.operator=(y)`. — end note]

- <sup>2</sup> [Note 2: Since a copy assignment operator is implicitly declared for a class if not declared by the user (11.4.6), a base class assignment operator function is always hidden by the copy assignment operator function of the derived class. — end note]

- <sup>3</sup> [Note 3: Any assignment operator function, even the copy and move assignment operators, can be virtual. For a derived class `D` with a base class `B` for which a virtual copy/move assignment has been declared, the copy/move assignment operator in `D` does not override `B`'s virtual copy/move assignment operator.

[Example 1:

```
struct B {
 virtual int operator= (int);
 virtual B& operator= (const B&);
};
struct D : B {
 virtual int operator= (int);
 virtual D& operator= (const B&);
};

D dobj1;
D dobj2;
B* bptr = &dobj1;
void f() {
 bptr->operator=(99); // calls D::operator=(int)
 *bptr = 99; // ditto
 bptr->operator=(dobj2); // calls D::operator=(const B&)
 *bptr = dobj2; // ditto
 dobj1 = dobj2; // calls implicitly-declared D::operator=(const D&)
}
```

— end example]

— end note]

### 12.4.4 Function call

[over.call]

- <sup>1</sup> A *function call operator function* is a function named `operator()` that is a member function with an arbitrary number of parameters. It may have default arguments. For an expression of the form

$$\text{postfix-expression } ( \text{expression-list}_{\text{opt}} )$$

where the *postfix-expression* is of class type, the operator function is selected by overload resolution (12.2.2.3). If a surrogate call function is selected, let *e* be the result of invoking the corresponding conversion operator function on the *postfix-expression*;

the expression is interpreted as

$$e \text{ ( expression-list}_{opt} \text{ )}$$

Otherwise, the expression is interpreted as

$$\text{postfix-expression} . \text{operator} \text{ ( ) ( expression-list}_{opt} \text{ )}$$

### 12.4.5 Subscripting

[over.sub]

- <sup>1</sup> A *subscripting operator function* is a member function named `operator[]` with an arbitrary number of parameters. It may have default arguments. For an expression of the form

$$\text{postfix-expression [ expression-list}_{opt} \text{ ]}$$

the operator function is selected by overload resolution (12.2.2.3). If a member function is selected, the expression is interpreted as

$$\text{postfix-expression} . \text{operator} \text{ [ ] ( expression-list}_{opt} \text{ )}$$

- <sup>2</sup> [Example 1:

```
struct X {
 Z operator[](std::initializer_list<int>);
 Z operator[](auto...);
};
X x;
x[{1,2,3}] = 7; // OK, meaning x.operator[]({1,2,3})
x[1,2,3] = 7; // OK, meaning x.operator[](1,2,3)
int a[10];
a[{1,2,3}] = 7; // error: built-in subscript operator
a[1,2,3] = 7; // error: built-in subscript operator
```

— end example]

### 12.4.6 Class member access

[over.ref]

- <sup>1</sup> A *class member access operator function* is a function named `operator->` that is a non-static member function taking no non-object parameters. For an expression of the form

$$\text{postfix-expression} \rightarrow \text{template}_{opt} \text{ id-expression}$$

the operator function is selected by overload resolution (12.2.2.3), and the expression is interpreted as

$$\text{( postfix-expression} . \text{operator} \rightarrow \text{ ( ) )} \rightarrow \text{template}_{opt} \text{ id-expression}$$

Analogously, for an expression of the form

$$\text{postfix-expression} \rightarrow \text{splice-expression}$$

the operator function is selected by overload resolution, and the expression is interpreted as

$$\text{( postfix-expression} . \text{operator} \rightarrow \text{ ( ) )} \rightarrow \text{splice-expression}$$

### 12.4.7 Increment and decrement

[over.inc]

#### 12.4.7.1 General

[over.inc.general]

- <sup>1</sup> An *increment operator function* is a function named `operator++`. If this function is a non-static member function with no non-object parameters, or a non-member function with one parameter, it defines the prefix increment operator `++` for objects of that type. If the function is a non-static member function with one non-object parameter (which shall be of type `int`) or a non-member function with two parameters (the second of which shall be of type `int`), it defines the postfix increment operator `++` for objects of that type. When the postfix increment is called as a result of using the `++` operator, the `int` argument will have value zero.<sup>100</sup>

[Example 1:

```
struct X {
 X& operator++(); // prefix ++a
 X operator++(int); // postfix a++
};
```

<sup>100</sup> Calling `operator++` explicitly, as in expressions like `a.operator++(2)`, has no special properties: The argument to `operator++` is 2.

```

struct Y { };
Y& operator++(Y&); // prefix ++b
Y operator++(Y&, int); // postfix b++

void f(X a, Y b) {
 ++a; // a.operator++();
 a++; // a.operator++(0);
 ++b; // operator++(b);
 b++; // operator++(b, 0);

 a.operator++(); // explicit call: like ++a;
 a.operator++(0); // explicit call: like a++;
 operator++(b); // explicit call: like ++b;
 operator++(b, 0); // explicit call: like b++;
}

```

— end example]

- <sup>2</sup> A *decrement operator function* is a function named `operator--` and is handled analogously to an increment operator function.

#### 12.4.7.2 Defaulted function

[over.inc.default]

- <sup>1</sup> A defaulted postfix increment or decrement operator function for a type `C` shall be a non-template function that

- (1.1) — has a first parameter of type “reference to `C`” or a first parameter of type “reference to volatile `C`”, where the implicit object parameter (if any) is considered to be the first parameter,
- (1.2) — is defined as defaulted in `C` or in a context where `C` is complete, and
- (1.3) — has a declared return type of `C`.

[Note 1: A postfix increment or decrement operator always has a parameter of type `int` (12.4.7.1). — end note]

Name lookups and access checks in the implicit definition (9.6.2) of a defaulted postfix increment or decrement operator function are performed from a context equivalent to its *function-body*. A definition of a postfix increment or decrement operator as defaulted that appears in a class shall be the first declaration of that function.

[Example 1:

```

struct S;
S operator++(S&, int) = default; // error: S is not complete

```

— end example]

- <sup>2</sup> A defaulted postfix increment or decrement operator function for a type `C` is defined as deleted if
- (2.1) — `C` is a class type for which overload resolution (12.2), as applied to direct-initialization of a variable of type `C` from an lvalue of type `C`, does not result in a usable candidate,
  - (2.2) — `C` has a destructor that is deleted or inaccessible from the context of the *function-body*, or
  - (2.3) — for an lvalue `c` of type `C`, overload resolution as applied to `++c` for a postfix increment operator function or `--c` for a postfix decrement operator function does not result in a usable candidate.
- <sup>3</sup> The implicit definition of a defaulted postfix increment or decrement operator function `F` that is not defined as deleted for a type `C` is equivalent to

```

C tmp(c);
++c;
return tmp;

```

for a postfix increment operator function, or

```

C tmp(c);
--c;
return tmp;

```

for a postfix decrement operator function, where `tmp` is a variable defined for exposition only, and `c` is an lvalue that denotes `*this` if `F` is an implicit object member function, or the first parameter of `F` otherwise.

<sup>4</sup> [Example 2:

```

struct S{
 S(const S&) = default;
 S& operator++() { return *this; }
};
S operator++(S, int) = default; // error: incorrect parameter type

struct T{
 T operator++(int) = default; // OK, defined as deleted
};

enum class E1 {};
E1 operator++(E1&, int) = default; // OK, defined as deleted; built-in operator++
 // does not exist for enumeration types

enum E2 {};
E2& operator++(E2& e);
E2 operator++(E2&, int) = default; // OK, not deleted
— end example]
```

## 12.5 Built-in operators

[over.built]

- <sup>1</sup> The candidate operator functions that represent the built-in operators defined in 7.6 are specified in this subclause. These candidate functions participate in the operator overload resolution process as described in 12.2.2.3 and are used for no other purpose.

[Note 1: Because built-in operators take only operands with non-class type, and operator overload resolution occurs only when an operand expression originally has class or enumeration type, operator overload resolution can resolve to a built-in operator only when an operand has a class type that has a user-defined conversion to a non-class type appropriate for the operator, or when an operand has an enumeration type that can be converted to a type appropriate for the operator. Also note that some of the candidate operator functions given in this subclause are more permissive than the built-in operators themselves. As described in 12.2.2.3, after a built-in operator is selected by overload resolution the expression is subject to the requirements for the built-in operator given in 7.6, and therefore to any additional semantic constraints given there. In some cases, user-written candidates with the same name and parameter types as a built-in candidate operator function cause the built-in operator function to not be included in the set of candidate functions. — end note]

- <sup>2</sup> In this subclause, the term *promoted integral type* is used to refer to those cv-unqualified integral types which are preserved by integral promotion (7.3.7) (including e.g., `int` and `long` but excluding e.g., `char`).

[Note 2: In all cases where a promoted integral type is required, an operand of unscoped enumeration type will be acceptable by way of the integral promotions. — end note]

- <sup>3</sup> In the remainder of this subclause, *vq* represents either `volatile` or no cv-qualifier.
- <sup>4</sup> For every pair (*T*, *vq*), where *T* is a cv-unqualified arithmetic type other than `bool` or a cv-unqualified pointer to (possibly cv-qualified) object type, there exist candidate operator functions of the form

```

vq T& operator++(vq T&);
T operator++(vq T&, int);
vq T& operator--(vq T&);
T operator--(vq T&, int);
```

- <sup>5</sup> For every (possibly cv-qualified) object type *T* and for every function type *T* that has neither *cv-qualifiers* nor a *ref-qualifier*, there exist candidate operator functions of the form

```
T& operator*(T*);
```

- <sup>6</sup> For every type *T* there exist candidate operator functions of the form

```
T* operator+(T*);
```

- <sup>7</sup> For every cv-unqualified floating-point or promoted integral type *T*, there exist candidate operator functions of the form

```
T operator+(T);
T operator-(T);
```

- <sup>8</sup> For every promoted integral type *T*, there exist candidate operator functions of the form

```
T operator~(T);
```



- <sup>9</sup> For every quintuple ( $C1$ ,  $C2$ ,  $T$ ,  $cv1$ ,  $cv2$ ), where  $C2$  is a class type,  $C1$  is the same type as  $C2$  or is a derived class of  $C2$ , and  $T$  is an object type or a function type, there exist candidate operator functions of the form

```
cv12 T& operator->*(cv1 C1*, cv2 T C2::*);
```

where  $cv12$  is the union of  $cv1$  and  $cv2$ . The return type is shown for exposition only; see 7.6.4 for the determination of the operator's result type.

- <sup>10</sup> For every pair of types  $L$  and  $R$ , where each of  $L$  and  $R$  is a cv-unqualified floating-point or promoted integral type, there exist candidate operator functions of the form

```
LR operator*(L, R);
LR operator/(L, R);
LR operator+(L, R);
LR operator-(L, R);
bool operator==(L, R);
bool operator!=(L, R);
bool operator<(L, R);
bool operator>(L, R);
bool operator<=(L, R);
bool operator>=(L, R);
```

where  $LR$  is the result of the usual arithmetic conversions (7.4) between types  $L$  and  $R$ .

- <sup>11</sup> For every cv-unqualified integral type  $T$  there exists a candidate operator function of the form

```
std::strong_ordering operator<=>(T, T);
```

- <sup>12</sup> For every pair of cv-unqualified floating-point types  $L$  and  $R$ , there exists a candidate operator function of the form

```
std::partial_ordering operator<=>(L, R);
```

- <sup>13</sup> For every cv-qualified or cv-unqualified object type  $T$  there exist candidate operator functions of the form

```
T* operator+(T*, std::ptrdiff_t);
T& operator[](T*, std::ptrdiff_t);
T* operator-(T*, std::ptrdiff_t);
T* operator+(std::ptrdiff_t, T*);
T& operator[](std::ptrdiff_t, T*);
```

- <sup>14</sup> For every  $T$ , where  $T$  is a pointer to object type, there exist candidate operator functions of the form

```
std::ptrdiff_t operator-(T, T);
```

- <sup>15</sup> For every  $T$ , where  $T$  is an enumeration type or a pointer type, there exist candidate operator functions of the form

```
bool operator==(T, T);
bool operator!=(T, T);
bool operator<(T, T);
bool operator>(T, T);
bool operator<=(T, T);
bool operator>=(T, T);
R operator<=>(T, T);
```

where  $R$  is the result type specified in 7.6.8.

- <sup>16</sup> For every  $T$ , where  $T$  is a pointer-to-member type, `std::meta::info`, or `std::nullptr_t`, there exist candidate operator functions of the form

```
bool operator==(T, T);
bool operator!=(T, T);
```

- <sup>17</sup> For every pair of promoted integral types  $L$  and  $R$ , there exist candidate operator functions of the form

```
LR operator%(L, R);
LR operator&(L, R);
LR operator^(L, R);
LR operator|(L, R);
L operator<<(L, R);
L operator>>(L, R);
```

where  $LR$  is the result of the usual arithmetic conversions (7.4) between types  $L$  and  $R$ .

- 18 For every triple  $(L, vq, R)$ , where  $L$  is a cv-unqualified arithmetic type and  $R$  is a cv-unqualified floating-point or promoted integral type, there exist candidate operator functions of the form

```

vq L& operator=(vq L&, R);
vq L& operator*=(vq L&, R);
vq L& operator/=(vq L&, R);
vq L& operator+=(vq L&, R);
vq L& operator-=(vq L&, R);

```

- 19 For every pair  $(T, vq)$ , where  $T$  is any type, there exist candidate operator functions of the form

```

T*vq& operator=(T*vq&, T*);

```

- 20 For every pair  $(T, vq)$ , where  $T$  is an enumeration or pointer-to-member type, there exist candidate operator functions of the form

```

vq T& operator=(vq T&, T);

```

- 21 For every pair  $(T, vq)$ , where  $T$  is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form

```

T*vq& operator+=(T*vq&, std::ptrdiff_t);
T*vq& operator-=(T*vq&, std::ptrdiff_t);

```

- 22 For every triple  $(L, vq, R)$ , where  $L$  is a cv-unqualified integral type and  $R$  is a promoted integral type, there exist candidate operator functions of the form

```

vq L& operator%=(vq L&, R);
vq L& operator<<=(vq L&, R);
vq L& operator>>=(vq L&, R);
vq L& operator&=(vq L&, R);
vq L& operator^=(vq L&, R);
vq L& operator|=(vq L&, R);

```

- 23 There also exist candidate operator functions of the form

```

bool operator!(bool);
bool operator&&(bool, bool);
bool operator||(bool, bool);

```

- 24 For every pair of types  $L$  and  $R$ , where each of  $L$  and  $R$  is a cv-unqualified floating-point or promoted integral type, there exist candidate operator functions of the form

```

LR operator?:(bool, L, R);

```

where  $LR$  is the result of the usual arithmetic conversions (7.4) between types  $L$  and  $R$ .

[Note 3: As with all these descriptions of candidate functions, this declaration serves only to describe the built-in operator for purposes of overload resolution. The operator “?:” cannot be overloaded. — end note]

- 25 For every type  $T$ , where  $T$  is a pointer, pointer-to-member, or scoped enumeration type, there exist candidate operator functions of the form

```

T operator?:(bool, T, T);

```

## 12.6 User-defined literals

[over.literal]

```

literal-operator-id:
 operator unevaluated-string identifier
 operator user-defined-string-literal

```

- 1 The *user-defined-string-literal* in a *literal-operator-id* shall have no *encoding-prefix*. The *unevaluated-string* or *user-defined-string-literal* shall be empty. The *ud-suffix* of the *user-defined-string-literal* or the *identifier* in a *literal-operator-id* is called a *literal suffix identifier*. The first form of *literal-operator-id* is deprecated (D.8). Some literal suffix identifiers are reserved for future standardization; see 16.4.5.3.6. A declaration whose *literal-operator-id* uses such a literal suffix identifier is ill-formed, no diagnostic required (G.9.1).
- 2 A declaration whose *declarator-id* is a *literal-operator-id* shall declare a function or function template that belongs to a namespace (it could be a friend function (11.8.4)) or an explicit instantiation or specialization of a function template. A function declared with a *literal-operator-id* is a *literal operator*. A function template declared with a *literal-operator-id* is a *literal operator template*.
- 3 The declaration of a literal operator shall have a *parameter-declaration-clause* equivalent to one of the following:

```

const char*
unsigned long long int
long double
char
wchar_t
char8_t
char16_t
char32_t
const char*, std::size_t
const wchar_t*, std::size_t
const char8_t*, std::size_t
const char16_t*, std::size_t
const char32_t*, std::size_t

```

If a parameter has a default argument (9.3.4.7), the program is ill-formed.

- 4 A *raw literal operator* is a literal operator with a single parameter whose type is `const char*`.
- 5 A *numeric literal operator template* is a literal operator template whose *template-parameter-list* has a single *template-parameter* that is a constant template parameter pack (13.7.4) with element type `char`. A *string literal operator template* is a literal operator template whose *template-parameter-list* comprises a single *parameter-declaration* that declares a constant template parameter of class type. The declaration of a literal operator template shall have an empty *parameter-declaration-clause* and shall declare either a numeric literal operator template or a string literal operator template.
- 6 Literal operators and literal operator templates shall not have C language linkage.
- 7 [Note 1: Literal operators and literal operator templates are usually invoked implicitly through user-defined literals (5.13.9). However, except for the constraints described above, they are ordinary namespace-scope functions and function templates. In particular, they are looked up like ordinary functions and function templates and they follow the same overload resolution rules. Also, they can be declared `inline` or `constexpr`, they can have internal, module, or external linkage, they can be called explicitly, their addresses can be taken, etc. — end note]
- 8 [Example 1:

```

void operator ""_km(long double); // OK
string operator ""_i18n(const char*, std::size_t); // OK, deprecated
template <char...> double operator ""_u03C0(); // OK, UCN for lowercase pi
float operator ""_e(const char*); // OK
float operator ""_E(const char*); // ill-formed, no diagnostic required:
 // reserved literal suffix (16.4.5.3.6, 5.13.9)

double operator ""_Bq(long double); // OK, does not use the reserved identifier _Bq (5.11)
double operator ""_Bq(long double); // ill-formed, no diagnostic required:
 // uses the reserved identifier _Bq (5.11)

float operator ""_B(const char*); // error: non-empty string-literal
string operator ""_5X(const char*, std::size_t); // error: invalid literal suffix identifier
double operator ""_miles(double); // error: invalid parameter-declaration-clause
template <char...> int operator ""_j(const char*); // error: invalid parameter-declaration-clause
extern "C" void operator ""_m(long double); // error: C language linkage

```

— end example]

# 13 Templates

[temp]

## 13.1 Preamble

[temp.pre]

- <sup>1</sup> A *template* defines a family of classes, functions, or variables, an alias for a family of types, or a concept.

```

template-declaration:
 template-head declaration
 template-head concept-definition

template-head:
 template < template-parameter-list > requires-clauseopt

template-parameter-list:
 template-parameter
 template-parameter-list , template-parameter

requires-clause:
 requires constraint-logical-or-expression

constraint-logical-or-expression:
 constraint-logical-and-expression
 constraint-logical-or-expression || constraint-logical-and-expression

constraint-logical-and-expression:
 primary-expression
 constraint-logical-and-expression && primary-expression

```

[Note 1: The > token following the *template-parameter-list* of a *template-declaration* can be the product of replacing a >> token by two consecutive > tokens (13.3). — end note]

- <sup>2</sup> The *declaration* in a *template-declaration* (if any) shall
- (2.1) — declare or define a function, a class, or a variable, or
  - (2.2) — define a member function, a member class, a member enumeration, or a static data member of a class template or of a class nested within a class template, or
  - (2.3) — define a member template of a class or class template, or
  - (2.4) — be a *friend-type-declaration*, or
  - (2.5) — be a *deduction-guide*, or
  - (2.6) — be an *alias-declaration*.

It shall not be a *linkage-specification*.

- <sup>3</sup> A *template-declaration* is a *declaration*. A declaration introduced by a template declaration of a variable is a *variable template*. A variable template at class scope is a *static data member template*.

[Example 1:

```

template<class T>
constexpr T pi = T(3.1415926535897932385L);
template<class T>
T circular_area(T r) {
 return pi<T> * r * r;
}
struct matrix_constants {
 template<class T>
 using pauli = hermitian_matrix<T, 2>;
 template<class T>
 constexpr static pauli<T> sigma1 = { { 0, 1 }, { 1, 0 } };
 template<class T>
 constexpr static pauli<T> sigma2 = { { 0, -1i }, { 1i, 0 } };
 template<class T>
 constexpr static pauli<T> sigma3 = { { 1, 0 }, { 0, -1 } };
};

```

— end example]

- <sup>4</sup> [Note 2: A *template-declaration* can appear only as a namespace scope or class scope declaration. — end note]

Its *declaration* shall not be an *export-declaration*. In a function template declaration, the *unqualified-id* of the *declarator-id* shall be a name.

[Note 3: A class or variable template declaration of a *simple-template-id* declares a partial specialization (13.7.6). — end note]

- <sup>5</sup> In a *template-declaration*, explicit specialization, or explicit instantiation, the *init-declarator-list* in the declaration shall contain at most one declarator. When such a declaration is used to declare a class template, no declarator is permitted. In a *template-declaration* whose *declaration* is a *friend-type-declaration*, the *friend-type-declaration* shall be of the form

```
friend friend-type-specifier ;
```

[Example 2:

```
template<class ...>
struct C {
 struct Nested { };
};

template<class ... Us>
struct S {
 template <typename ...Ts>
 friend class C<Ts>::Nested...; // error: pack expansion is not a friend-type-specifier
 friend class C<Us>::Nested...; // OK, not a template-declaration
};
```

— end example]

- <sup>6</sup> A specialization (explicit or implicit) of one template is distinct from all specializations of any other template.

[Note 4: Default arguments for function templates and for member functions of class templates are considered definitions for the purpose of template instantiation (13.7) and must obey the one-definition rule (6.3). — end note]

- <sup>7</sup> [Note 5: A template cannot have the same name as any other name bound in the same scope (6.4.1), except that a function template can share a name with *using-declarators*, a type, non-template functions (9.3.4.6) and/or function templates (13.10.4). Specializations, including partial specializations (13.7.6), do not reintroduce or bind names. Their target scope is the target scope of the primary template, so all specializations of a template belong to the same scope as it does.

[Example 3:

```
void f() {}
class f {}; // OK
namespace N {
 void f(int) {}
}
using N::f; // OK
template<typename> void f(long) {} // #1, OK
template<typename> void f(long) {} // error: redefinition of #1
template<typename> void f(long long) {} // OK
template<> void f<int>(long long) {} // OK, doesn't bind a name
```

— end example]

— end note]

- <sup>8</sup> An entity is *templated* if it is

- (8.1) — a template,
- (8.2) — an entity defined (6.2) or created (6.8.7) within the *for-range-declaration* or *compound-statement* of an *expansion-statement* (8.7),
- (8.3) — an entity defined or created in a templated entity,
- (8.4) — a member of a templated entity,
- (8.5) — an enumerator for an enumeration that is a templated entity, or
- (8.6) — the closure type of a *lambda-expression* (7.5.6.2) appearing in the declaration of a templated entity.

[Note 6: A local class, a local or block variable, or a friend function defined in a templated entity is a templated entity. — end note]

A *templated function* is a function template or a function that is templated. A *templated class* is a class template or a class that is templated. A *templated variable* is a variable template or a variable that is templated.

- <sup>9</sup> A *template-declaration* is written in terms of its template parameters. The optional *requires-clause* following a *template-parameter-list* allows the specification of constraints (13.5.3) on template arguments (13.4). The *requires-clause* introduces the *constraint-expression* that results from interpreting the *constraint-logical-or-expression* as a *constraint-expression*. The *constraint-logical-or-expression* of a *requires-clause* is an unevaluated operand (7.2.3).

[Note 7: The expression in a *requires-clause* uses a restricted grammar to avoid ambiguities. Parentheses can be used to specify arbitrary expressions in a *requires-clause*.

[Example 4:

```
template<int N> requires N == sizeof new unsigned short
int f(); // error: parentheses required around == expression
```

— end example]

— end note]

- <sup>10</sup> A definition of a function template, member function of a class template, variable template, or static data member of a class template shall be reachable from the end of every definition domain (6.3) in which it is implicitly instantiated (13.9.2) unless the corresponding specialization is explicitly instantiated (13.9.3) in some translation unit; no diagnostic is required (G.10.1).

## 13.2 Template parameters

[temp.param]

- <sup>1</sup> The syntax for *template-parameters* is:

*template-parameter*:

*type-parameter*  
*parameter-declaration*  
*type-tt-parameter*  
*variable-tt-parameter*  
*concept-tt-parameter*

*type-parameter*:

*type-parameter-key* ... *opt identifier<sub>opt</sub>*  
*type-parameter-key identifier<sub>opt</sub>* = *type-id*  
*type-constraint* ... *opt identifier<sub>opt</sub>*  
*type-constraint identifier<sub>opt</sub>* = *type-id*

*type-parameter-key*:

*class*  
*typename*

*type-constraint*:

*nested-name-specifier<sub>opt</sub>* *concept-name*  
*nested-name-specifier<sub>opt</sub>* *concept-name* < *template-argument-list<sub>opt</sub>* >

*type-tt-parameter*:

*template-head type-parameter-key* ... *opt identifier<sub>opt</sub>*  
*template-head type-parameter-key identifier<sub>opt</sub>* *type-tt-parameter-default*

*type-tt-parameter-default*:

= *nested-name-specifier<sub>opt</sub>* *template-name*  
 = *nested-name-specifier* *template* *template-name*

*variable-tt-parameter*:

*template-head auto* ... *opt identifier<sub>opt</sub>*  
*template-head auto identifier<sub>opt</sub>* = *nested-name-specifier<sub>opt</sub>* *template-name*

*concept-tt-parameter*:

*template* < *template-parameter-list* > *concept* ... *opt identifier<sub>opt</sub>*  
*template* < *template-parameter-list* > *concept identifier<sub>opt</sub>* = *nested-name-specifier<sub>opt</sub>* *template-name*

The component names of a *type-constraint* are its *concept-name* and those of its *nested-name-specifier* (if any).

[Note 1: The > token following the *template-parameter-list* of a *type-tt-parameter*, *variable-tt-parameter*, or *concept-tt-parameter* can be the product of replacing a >> token by two consecutive > tokens (13.3). — end note]

<sup>2</sup> A template parameter is of one of the following kinds:

- (2.1) — A *type template parameter* is a template parameter introduced by a *type-parameter*.
- (2.2) — A *constant template parameter* is a template parameter introduced by a *parameter-declaration*.
- (2.3) — A *type template template parameter* is a template parameter introduced by a *type-tt-parameter*.
- (2.4) — A *variable template template parameter* is a template parameter introduced by a *variable-tt-parameter*.
- (2.5) — A *concept template parameter* is a template parameter introduced by a *concept-tt-parameter*.

<sup>3</sup> Type template template parameters, variable template template parameters, and concept template parameters are collectively referred to as *template template parameters*.

<sup>4</sup> The *nested-name-specifier* of a *type-constraint*, if any, shall not be dependent.

<sup>5</sup> A concept template parameter shall not have associated constraints (13.5.3).

<sup>6</sup> If a *template-parameter* is a *parameter-declaration* that declares a pack (9.3.4.6), or otherwise has an ellipsis prior to its optional *identifier*, then the *template-parameter* declares a template parameter pack (13.7.4). A template parameter pack that is a *parameter-declaration* whose type contains one or more unexpanded packs is a pack expansion. Similarly, a template parameter pack that is a template template parameter with a *template-parameter-list* containing one or more unexpanded packs is a pack expansion. A type parameter pack with a *type-constraint* that contains an unexpanded parameter pack is a pack expansion. A template parameter pack that is a pack expansion shall not expand a template parameter pack declared in the same *template-parameter-list*.

[Example 1:

```
template <class... Types> // Types is a template type parameter pack
 class Tuple; // but not a pack expansion

template <class T, int... Dims> // Dims is a constant template parameter pack
 struct multi_array; // but not a pack expansion

template <class... T>
 struct value_holder {
 template <T... Values> struct apply { }; // Values is a constant template parameter pack
 }; // and a pack expansion

template <class... T, T... Values> // error: Values expands template type parameter
 struct static_array; // pack T within the same template parameter list
```

— end example]

<sup>7</sup> There is no semantic difference between **class** and **typename** in a *type-parameter-key*. **typename** followed by an *unqualified-id* names a template type parameter. **typename** followed by a *qualified-id* denotes the type in a *parameter-declaration*. A *template-parameter* of the form **class identifier** is a *type-parameter*.

[Example 2:

```
class T { /* ... */ };
int i;

template<class T, T i> void f(T t) {
 T t1 = i; // template parameters T and i
 ::T t2 = ::i; // global namespace members T and i
}
```

Here, the template **f** has a type template parameter called **T**, rather than an unnamed constant template parameter of class **T**. — end example]

The *parameter-declaration* of a *template-parameter* shall not have a *storage-class-specifier*. Types shall not be defined in a template parameter declaration.

<sup>8</sup> The *identifier* in a *template-parameter* denoting a type or template is not looked up. An *identifier* that does not follow an ellipsis is defined to be

- (8.1) — a *typedef-name* for a *type-parameter*,
- (8.2) — a *template-name* for a *variable-tt-parameter*,
- (8.3) — a *template-name* for a *type-tt-parameter*, or



- (8.4) — a *concept-name* for a *concept-tt-parameter*,  
in the scope of the template declaration.

- 9 A *type-constraint* *Q* that designates a concept *C* can be used to constrain a contextually-determined type or template type parameter pack *T* with a *constraint-expression* *E* defined as follows. If *Q* is of the form *C*<*A*<sub>1</sub>, ..., *A*<sub>*n*</sub>>, then let *E'* be *C*<*T*, *A*<sub>1</sub>, ..., *A*<sub>*n*</sub>>. Otherwise, let *E'* be *C*<*T*>. If *T* is not a pack, then *E* is *E'*, otherwise *E* is (*E'* && ...). This *constraint-expression* *E* is called the *immediately-declared constraint* of *Q* for *T*. The concept designated by a *type-constraint* shall be a type concept (13.7.9).
- 10 A *type-parameter* that starts with a *type-constraint* introduces the immediately-declared constraint of the *type-constraint* for the parameter.

[Example 3:

```
template<typename T> concept C1 = true;
template<typename... Ts> concept C2 = true;
template<typename T, typename U> concept C3 = true;

template<C1 T> struct s1; // associates C1<T>
template<C1... T> struct s2; // associates (C1<T> && ...)
template<C2... T> struct s3; // associates (C2<T> && ...)
template<C3<int> T> struct s4; // associates C3<T, int>
template<C3<int>... T> struct s5; // associates (C3<T, int> && ...)
```

— end example]

- 11 A constant template parameter shall have one of the following (possibly cv-qualified) types:
- (11.1) — a structural type (see below),
- (11.2) — a type that contains a placeholder type (9.2.9.7), or
- (11.3) — a placeholder for a deduced class type (9.2.9.8).

The top-level *cv-qualifiers* on the *template-parameter* are ignored when determining its type.

- 12 A *structural type* is one of the following:

- (12.1) — a scalar type, or
- (12.2) — an lvalue reference type, or
- (12.3) — a literal class type with the following properties:
- (12.3.1) — all base classes and non-static data members are public and non-mutable and
- (12.3.2) — the types of all base classes and non-static data members are structural types or (possibly multidimensional) arrays thereof.

- 13 Certain constructs refer to template parameter objects, which are distinct objects with static storage duration and non-volatile const type. No two such objects have template-argument-equivalent values (13.6). An *id-expression* naming a constant template parameter of class type *T* denotes the template parameter object of type *const T*, which is template-argument-equivalent to the corresponding template argument after it has been converted to the type of the template parameter (13.4.3).

[Note 2: There can be template parameter objects of array type (21.4.3), but such an object is never denoted by an *id-expression* that names a constant template parameter. Such an object can have an address that is not unique among all other in-lifetime objects (6.8.2). — end note]

[Note 3: If an *id-expression* names a non-reference constant template parameter, then it is a prvalue if it has non-class type. Otherwise, if it is of class type *T*, it is an lvalue and has type *const T* (7.5.5.2). — end note]

[Example 4:

```
using X = int;
struct A {};
template<const X& x, int i, A a> void f() {
 i++; // error: change of template parameter value

 &x; // OK
 &i; // error: address of non-reference template parameter
 &a; // OK
 int& ri = i; // error: attempt to bind non-const reference to temporary
 const int& cri = i; // OK, const reference binds to temporary
```



```
 const A& ra = a; // OK, const reference binds to a template parameter object
}
```

— end example]

- 14 [Note 4: A constant template parameter cannot be declared to have type *cv void*.

[Example 5:

```
 template<void v> class X; // error
 template<void* pv> class Y; // OK
```

— end example]

— end note]

- 15 A constant template parameter of type “array of T” or of function type T is adjusted to be of type “pointer to T”.

[Example 6:

```
 template<int* a> struct R { /* ... */ };
 template<int b[5]> struct S { /* ... */ };
 int p;
 R<&p> w; // OK
 S<&p> x; // OK due to parameter adjustment
 int v[5];
 R<v> y; // OK due to implicit argument conversion
 S<v> z; // OK due to both adjustment and conversion
```

— end example]

- 16 A constant template parameter declared with a type that contains a placeholder type with a *type-constraint* introduces the immediately-declared constraint of the *type-constraint* for the invented type corresponding to the placeholder (9.3.4.6).

- 17 A *default template argument* is a template argument (13.4) specified after = in a *template-parameter*. A default template argument shall not be specified for a template parameter pack (13.7.4). A default template argument may be specified in a template declaration. A default template argument shall not be specified in the *template-parameter-lists* of the definition of a member of a class template that appears outside of the member’s class. A default template argument shall not be specified in a friend class template declaration. If a friend function template declaration *D* specifies a default template argument, that declaration shall be a definition and there shall be no other declaration of the function template which is reachable from *D* or from which *D* is reachable.

- 18 The set of default template arguments available for use is obtained by merging the default arguments from all prior declarations of the template in the same way default function arguments are (9.3.4.7).

[Example 7:

```
 template<class T1, class T2 = int> class A;
 template<class T1 = int, class T2> class A;
```

is equivalent to

```
 template<class T1 = int, class T2 = int> class A;
```

— end example]

- 19 If a *template-parameter* of a class template, variable template, or alias template has a default template argument, each subsequent *template-parameter* shall either have a default template argument supplied or declare a template parameter pack. If a *template-parameter* of a primary class template, primary variable template, or alias template declares a template parameter pack, it shall be the last *template-parameter*. If a *template-parameter* of a function template declares a template parameter pack, it shall not be followed by another *template-parameter* unless that template parameter is deducible from the parameter-type-list (9.3.4.6) of the function template or has a default argument (13.10.3). A template parameter of a deduction guide template (13.7.2.3) that does not have a default argument shall be deducible from the parameter-type-list of the deduction guide template.

[Example 8:

```
 template<class T1 = int, class T2> class B; // error
```

```
// U can be neither deduced from the parameter-type-list nor specified
template<class... T, class... U> void f() { } // error
template<class... T, class U> void g() { } // error
```

— end example]

- <sup>20</sup> When parsing a default template argument for a constant template parameter, the first non-nested > is taken as the end of the *template-parameter-list* rather than a greater-than operator.

[Example 9:

```
template<int i = 3 > 4 > // syntax error
class X { /* ... */ };

template<int i = (3 > 4) > // OK
class Y { /* ... */ };
```

— end example]

- <sup>21</sup> A *template-parameter* of a template *template-parameter* is permitted to have a default template argument. When such default arguments are specified, they apply to the template *template-parameter* in the scope of the template *template-parameter*.

[Example 10:

```
template <template <class TT = float> class T> struct A {
 inline void f();
 inline void g();
};
template <template <class TT> class T> void A<T>::f() {
 T<> t; // error: TT has no default template argument
}
template <template <class TT = char> class T> void A<T>::g() {
 T<> t; // OK, T<char>
}
```

— end example]

The associated constraints of a template template parameter shall not contain a concept-dependent constraint (13.5.2.4).

[Example 11:

```
template<
 template<typename> concept C,
 template<C> class TT // error: C forms a concept-dependent constraint
>
struct A {};
```

— end example]

### 13.3 Names of template specializations

[temp.names]

- <sup>1</sup> A template specialization (13.9) can be referred to by a *template-id*:

```
simple-template-id:
 template-name < template-argument-listopt >

template-id:
 simple-template-id
 operator-function-id < template-argument-listopt >
 literal-operator-id < template-argument-listopt >

template-name:
 simple-template-name
 pack-index-template-name

pack-index-template-name:
 simple-template-name ... [constant-expression]

simple-template-name:
 identifier
```

*template-argument-list*:  
*template-argument* ...<sub>opt</sub>  
*template-argument-list* , *template-argument* ...<sub>opt</sub>

*template-argument*:  
*template-argument-name*  
*constant-expression*  
*type-id*  
*braced-init-list*

*template-argument-name*:  
*nested-name-specifier*<sub>opt</sub> *identifier*  
*nested-name-specifier* *template* *identifier*

- 2 The component name of a *simple-template-id*, *template-id*, or *template-name* is the first name in it.
- 3 The *simple-template-name* *P* in a *pack-index-template-name* shall denote a pack.
- 4 The *constant-expression* shall be a converted constant expression (7.7.3) of type `std::size_t` whose value *V*, termed the index, is such that  $0 \leq V < \text{sizeof} \dots(P)$ .
- 5 A *pack-index-template-name* is a pack expansion (13.7.4).
- 6 [Note 1: The *pack-index-template-name* denotes the  $V^{\text{th}}$  *template-name* of the pack. — end note]
- 7 A *<* is interpreted as the delimiter of a *template-argument-list* if
  - (7.1) — it follows a *splice-specifier* that either
    - (7.1.1) — appears in a type-only context or
    - (7.1.2) — is preceded by `template` or `typename`, or
  - (7.2) — it follows a *pack-index-template-name*, or
  - (7.3) — it follows a name that is not a *conversion-function-id* and
    - (7.3.1) — that follows the keyword `template` or a `~` after a *nested-name-specifier* or in a class member access expression, or
    - (7.3.2) — for which name lookup finds the injected-class-name of a class template or finds any declaration of a template, or
    - (7.3.3) — that is an unqualified name for which name lookup either finds one or more functions or finds nothing, or
    - (7.3.4) — that is a terminal name in a *using-declarator* (9.10), in a *declarator-id* (9.3.4), or in a type-only context other than a *nested-name-specifier* (13.8).

[Note 2: If the name is an *identifier*, it is then interpreted as a *template-name*. The keyword `template` is used to indicate that a dependent qualified name (13.8.3.2) denotes a template where an expression might appear. — end note]

[Example 1:

```
struct X {
 template<std::size_t> X* alloc();
 template<std::size_t> static X* adjust();
};
template<class T> void f(T* p) {
 T* p1 = p->alloc<200>(); // error: < means less than
 T* p2 = p->template alloc<200>(); // OK, < starts template argument list
 T::adjust<100>(); // error: < means less than
 T::template adjust<100>(); // OK, < starts template argument list

 static constexpr std::meta::info r = ~T::adjust;
 T* p3 = [:r:]<200>(); // error: < means less than
 T* p4 = template [:r:]<200>(); // OK, < starts template argument list
}
```

— end example]

- <sup>8</sup> When parsing a *template-argument-list*, the first non-nested `>`<sup>101</sup> is taken as the ending delimiter rather than a greater-than operator. Similarly, the first non-nested `>>` is treated as two consecutive but distinct `>` tokens, the first of which is taken as the end of the *template-argument-list* and completes the *template-id* or *splice-specialization-specifier*.

[*Note 3*: The second `>` token produced by this replacement rule can terminate an enclosing *template-id* or *splice-specialization-specifier* construct or it can be part of a different construct (e.g., a cast). — *end note*]

[*Example 2*:

```
template<int i> class X { /* ... */ };

X< 1>2 > x1; // syntax error
X<(1>2)> x2; // OK

template<class T> class Y { /* ... */ };
Y<X<1>> x3; // OK, same as Y<X<1> > x3;
Y<X<6>>1>> x4; // syntax error
Y<X<(6>>1)>> x5; // OK
```

— *end example*]

- <sup>9</sup> The keyword `template` shall not appear immediately after a declarative *nested-name-specifier* (7.5.5.3).
- <sup>10</sup> The *constant-expression* of a *template-argument* shall not be an unparenthesized *splice-expression*.

[*Example 3*:

```
template<int> struct S { };
constexpr int k = 5;
constexpr std::meta::info r = ^^k;
S<[:r:]> s1; // error: unparenthesized splice-expression used as template argument
S<(:[:r:])> s2; // OK
S<[:r:] + 1> s3; // OK
```

— *end example*]

- <sup>11</sup> A name prefixed by the keyword `template` shall be followed by a template argument list or refer to a class template or an alias template. The latter case is deprecated (D.9). The keyword `template` shall not appear immediately before a `~` token (as to name a destructor).

[*Note 4*: The keyword `template` cannot be applied to non-template members of class templates. — *end note*]

[*Note 5*: As is the case with the `typename` prefix, the `template` prefix is well-formed even when lookup for the name would already find a template. — *end note*]

[*Example 4*:

```
template <class T> struct A {
 void f(int);
 template <class U> void f(U);
};

template <class T> void f(T t) {
 A<T> a;
 a.template f<>(t); // OK, calls template
 a.template f(t); // error: not a template-id
}

template <class T> struct B {
 template <class T2> struct C { };
};

// deprecated: T::C is assumed to name a class template:
template <class T, template <class X> class TT = T::template C> struct D { };
D<B<int> > db;
```

— *end example*]

<sup>101</sup>) A `>` that encloses the *type-id* of a `dynamic_cast`, `static_cast`, `reinterpret_cast` or `const_cast`, or which encloses the *template-arguments* of a subsequent *template-id* or *splice-specialization-specifier*, is considered nested for the purpose of this description.

<sup>12</sup> The component names of a *template-argument-name* are those of its *nested-name-specifier* (if any) and its *identifier*.

<sup>13</sup> A *template-id* or *splice-specialization-specifier* is *valid* if

- (13.1) — there are at most as many arguments as there are parameters or a parameter is a template parameter pack (13.7.4),
- (13.2) — there is an argument for each non-deducible non-pack parameter that does not have a default *template-argument*,
- (13.3) — each *template-argument* matches the corresponding template parameter (13.4),
- (13.4) — substitution of each template argument into the following template parameters (if any) succeeds, and
- (13.5) — if the *template-id* or *splice-specialization-specifier* is non-dependent, the associated constraints are satisfied as specified in the next paragraph.

A *simple-template-id* or *splice-specialization-specifier* shall be valid unless its respective *template-name* or *splice-specifier* designates a function template (13.10.3).

[Example 5:

```
template<class T, T::type n = 0> class X;
struct S {
 using type = int;
};
using T1 = X<S, int, int>; // error: too many arguments
using T2 = X<>; // error: no default argument for first template parameter
using T3 = X<1>; // error: value 1 does not match type-parameter
using T4 = X<int>; // error: substitution failure for second template parameter
using T5 = X<S>; // OK
```

— end example]

<sup>14</sup> When the *template-name* of a *simple-template-id* or the *splice-specifier* of a *splice-specialization-specifier* designates a constrained non-function template or a constrained template template parameter, and all *template-arguments* in the *simple-template-id* or *splice-specialization-specifier* are non-dependent (13.8.3.7), the associated constraints (13.5.3) of the constrained template shall be satisfied (13.5.2).

[Example 6:

```
template<typename T> concept C1 = sizeof(T) != sizeof(int);

template<C1 T> struct S1 { };
template<C1 T> using Ptr = T*;

S1<int>* p; // error: constraints not satisfied
Ptr<int> p; // error: constraints not satisfied

template<typename T>
struct S2 { Ptr<int> x; }; // ill-formed, no diagnostic required

template<typename T>
struct S3 { Ptr<T> x; }; // OK, satisfaction is not required

S3<int> x; // error: constraints not satisfied

template<template<C1 T> class X>
struct S4 {
 X<int> x; // ill-formed, no diagnostic required
};

template<typename T> concept C2 = sizeof(T) == 1;

template<C2 T> struct S { };

template struct S<char[2]>; // error: constraints not satisfied
template<> struct S<char[2]> { }; // error: constraints not satisfied
```

— end example]

- <sup>15</sup> A *concept-id* is a *simple-template-id* where the *template-name* is a *concept-name*. A *concept-id* is a prvalue of type `bool`, and does not name a template specialization. A *concept-id* evaluates to `true` if the concept's normalized *constraint-expression* (13.5.3) is satisfied (13.5.2) by the specified template arguments and `false` otherwise.

[Note 6: Since a *constraint-expression* is an unevaluated operand, a *concept-id* appearing in a *constraint-expression* is not evaluated except as necessary to determine whether the normalized constraints are satisfied. — end note]

[Example 7:

```
template<typename T> concept C = true;
static_assert(C<int>); // OK
```

— end example]

## 13.4 Template arguments

[temp.arg]

### 13.4.1 General

[temp.arg.general]

- <sup>1</sup> The type and form of each *template-argument* specified in a *template-id* or in a *splice-specialization-specifier* shall match the type and form specified for the corresponding parameter declared by the template in its *template-parameter-list*. When the parameter declared by the template is a template parameter pack (13.7.4), it will correspond to zero or more *template-arguments*.

[Example 1:

```
template<class T> class Array {
 T* v;
 int sz;
public:
 explicit Array(int);
 T& operator[](int);
 T& elem(int i) { return v[i]; }
};

Array<int> v1(20);
typedef std::complex<double> dcomplex; // std::complex is a standard library template
Array<dcomplex> v2(30);
Array<dcomplex> v3(40);

void bar() {
 v1[3] = 7;
 v2[3] = v3.elem(4) = dcomplex(7,8);
}
```

— end example]

- <sup>2</sup> The template argument list of a *template-head* is a template argument list in which the  $n^{\text{th}}$  template argument has the value of the  $n^{\text{th}}$  template parameter of the *template-head*. If the  $n^{\text{th}}$  template parameter is a template parameter pack (13.7.4), the  $n^{\text{th}}$  template argument is a pack expansion whose pattern is the name of the template parameter pack.
- <sup>3</sup> If a *template-argument* *A* matches the form *template-argument-name*, it is interpreted as such; the *identifier* is looked up and its meaning is determined as follows:
- (3.1) — If lookup finds an injected-class-name (13.8.2), then:
    - (3.1.1) — When *A* is for a type template template parameter, *A* denotes the corresponding class template.
    - (3.1.2) — Otherwise, it denotes a *type-name*.
  - (3.2) — Otherwise, if lookup finds a template, *A* denotes that template.
  - (3.3) — Otherwise, if lookup finds a type alias or a type, *A* denotes the underlying type and is interpreted as a *type-id*.
  - (3.4) — Otherwise, *A* is interpreted as an expression.
- <sup>4</sup> In a *template-argument*, an ambiguity between a *type-id* and an expression is resolved to a *type-id*, regardless of the form of the corresponding *template-parameter*.<sup>102</sup>

<sup>102</sup> There is no such ambiguity in a default *template-argument* because the form of the *template-parameter* determines the allowable forms of the *template-argument*.

[Example 2:

```
template<class T> void f();
template<int I> void f();

void g() {
 f<int>(); // int() is a type-id: call the first f()
}
```

— end example]

- <sup>5</sup> [Note 1: Names used in a *template-argument* are subject to access control where they appear. Because a template parameter is not a class member, no access control applies where the template parameter is used. — end note]

[Example 3:

```
template<class T> class X {
 static T t;
};

class Y {
private:
 struct S { /* ... */ };
 X<S> x; // OK, S is accessible
 // X<Y::S> has a static member of type Y::S
 // OK, even though Y::S is private
};

X<Y::S> y; // error: S not accessible
```

— end example]

For a template argument that is a class type or a class template, the template definition has no special access rights to the members of the template argument.

[Example 4:

```
template <template <class TT> class T> class A {
 typename T<int>::S s;
};

template <class U> class B {
private:
 struct S { /* ... */ };
};

A b; // error: A has no access to B::S
```

— end example]

- <sup>6</sup> When template argument packs or default template arguments are used, a *template-argument* list can be empty. In that case the empty <> brackets shall still be used as the *template-argument-list*.

[Example 5:

```
template<class T = char> class String;
String<>* p; // OK, String<char>
String* q; // syntax error
template<class ... Elements> class Tuple;
Tuple<>* t; // OK, Elements is empty
Tuple* u; // syntax error
```

— end example]

- <sup>7</sup> An explicit destructor call (11.4.7) for an object that has a type that is a class template specialization may explicitly specify the *template-arguments*.

[Example 6:

```
template<class T> struct A {
 ~A();
};
void f(A<int>* p, A<int>* q) {
 p->A<int>::~~A(); // OK, destructor call
```

```
 q->A<int>::~~A<int>(); // OK, destructor call
}
```

— end example]

- <sup>8</sup> If the use of a template argument gives rise to an ill-formed construct in the instantiation of a template specialization, the program is ill-formed.
- <sup>9</sup> When name lookup for the component name of a *template-id* finds an overload set, both non-template functions in the overload set and function templates in the overload set for which the *template-arguments* do not match the *template-parameters* are ignored.

[Note 2: If none of the function templates have matching *template-parameters*, the program is ill-formed. — end note]

- <sup>10</sup> When a *simple-template-id* or *splice-specialization-specifier* does not designate a function, a default *template-argument* is implicitly instantiated (13.9.2) when the value of that default argument is needed.

[Example 7:

```
template<typename T, typename U = int> struct S { };
S<bool>* p; // the type of p is S<bool, int>*
```

The default argument for U is instantiated to form the type S<bool, int>\*. — end example]

- <sup>11</sup> A *template-argument* followed by an ellipsis is a pack expansion (13.7.4).

### 13.4.2 Type template arguments

[temp.arg.type]

- <sup>1</sup> A *template-argument* for a type template parameter shall be a *type-id*.

<sup>2</sup> [Example 1:

```
template <class T> class X { };
template <class T> void f(T t) { }
struct { } unnamed_obj;

void f() {
 struct A { };
 enum { e1 };
 typedef struct { } B;
 B b;
 X<A> x1; // OK
 X<A*> x2; // OK
 X x3; // OK
 f(e1); // OK
 f(unnamed_obj); // OK
 f(b); // OK
}
```

— end example]

[Note 1: A template type argument can be an incomplete type (6.9.1). — end note]

### 13.4.3 Constant template arguments

[temp.arg.nontype]

- <sup>1</sup> A template argument *E* for a constant template parameter with declared type *T* shall be such that the invented declaration

```
T x = E ;
```

satisfies the semantic constraints for the definition of a **constexpr** variable with static storage duration (9.2.6). If *T* contains a placeholder type (9.2.9.7) or a placeholder for a deduced class type (9.2.9.8), the type of the parameter is deduced from the above declaration.

[Note 1: *E* is a *template-argument* or (for a default template argument) an *initializer-clause*. — end note]

If the parameter type thus deduced is not permitted for a constant template parameter (13.2), the program is ill-formed.

- <sup>2</sup> The value of a constant template parameter *P* of (possibly deduced) type *T* is determined from its template argument *A* as follows. If *T* is not a class type and *A* is not a *braced-init-list*, *A* shall be a converted constant expression (7.7.3) of type *T*; the value of *P* is *A* (as converted).
- <sup>3</sup> Otherwise, a temporary variable

```
constexpr T v = A;
```



is introduced. The lifetime of *v* ends immediately after initializing it and any template parameter object (see below). For each such variable, the *id-expression* *v* is termed a *candidate initializer*.

- 4 If *T* is a class type, a template parameter object (13.2) exists that is constructed so as to be template-argument-equivalent to *v*; *P* denotes that template parameter object. *P* is copy-initialized from an unspecified candidate initializer that is template-argument-equivalent to *v*. If, for the initialization from any candidate initializer,

- (4.1) — the initialization would be ill-formed, or
- (4.2) — the full-expression of an invented *init-declarator* for the initialization would not be a constant expression when interpreted as a *constant-expression* (7.7.3), or
- (4.3) — the initialization would cause *P* to not be template-argument-equivalent (13.6) to *v*,  
the program is ill-formed.

- 5 Otherwise, the value of *P* is that of *v*.

- 6 For a constant template parameter of reference or pointer type, or for each non-static data member of reference or pointer type in a constant template parameter of class type or subobject thereof, the reference or pointer value shall not refer or point to (respectively):

- (6.1) — a temporary object (6.8.7),
- (6.2) — a string literal object (5.13.5),
- (6.3) — the result of a *typeid* expression (7.6.1.8),
- (6.4) — a predefined `__func__` variable (9.6.1), or
- (6.5) — a subobject (6.8.2) of one of the above.

- 7 [Example 1:

```
template <int& r> class A{};
extern int x;
A<x> a; // OK
void f(int p) {
 constexpr int& r = p; // OK
 A<r> a; // error: a static constexpr int& variable cannot be initialized to refer to p here
}
```

— end example]

- 8 [Example 2:

```
template<const int* pci> struct X { /* ... */ };
int ai[10];
X<ai> xi; // array to pointer and qualification conversions

struct Y { /* ... */ };
template<const Y& b> struct Z { /* ... */ };
Y y;
Z<y> z; // no conversion, but note extra cv-qualification

template<int (&pa)[5]> struct W { /* ... */ };
int b[5];
W w; // no conversion

void f(char);
void f(int);

template<void (*pf)(int)> struct A { /* ... */ };

A<&f> a; // selects f(int)

template<auto n> struct B { /* ... */ };
B<5> b1; // OK, template parameter type is int
B<'a'> b2; // OK, template parameter type is char
B<2.5> b3; // OK, template parameter type is double
B<void(0)> b4; // error: template parameter type cannot be void
```

```

template<int i> struct C { /* ... */ };
C<{ 42 }> c1; // OK

struct J1 {
 J1 *self = this;
};
B<J1{}> j1; // error: initialization of template parameter object is not a constant expression

struct J2 {
 J2 *self = this;
 constexpr J2() {}
 constexpr J2(const J2&) {}
};
B<J2{}> j2; // error: template parameter object not template-argument-equivalent to introduced temporary
— end example]

```

- <sup>9</sup> [Note 2: A *string-literal* (5.13.5) is not an acceptable *template-argument* for a constant template parameter of non-class type.

[Example 3:

```

template<class T, T p> class X {
 /* ... */
};

X<const char*, "Studebaker"> x; // error: string literal object as template-argument
X<const char*, "Knope" + 1> x2; // error: subobject of string literal object as template-argument

const char p[] = "Vivisectionist";
X<const char*, p> y; // OK

struct A {
 constexpr A(const char*) {}
};

X<A, "Pyrophoricity"> z; // OK, string-literal is a constructor argument to A

```

— end example]

— end note]

- <sup>10</sup> [Note 3: A temporary object is not an acceptable *template-argument* when the corresponding template parameter has reference type.

[Example 4:

```

template<const int& CRI> struct B { /* ... */ };

B<1> b1; // error: temporary would be required for template argument

int c = 1;
B<c> b2; // OK

struct X { int n; };
struct Y { const int &r; };
template<Y y> struct C { /* ... */ };
C<Y{X{1}.n}> c; // error: subobject of temporary object used to initialize
 // reference member of template parameter

```

— end example]

— end note]

#### 13.4.4 Template template arguments

[temp.arg.template]

- <sup>1</sup> A *template-argument* for a template template parameter shall be the name of a template. For a *type-tt-parameter*, the name shall denote a class template, alias template, or type template template parameter. For a *variable-tt-parameter*, the name shall denote a variable template or variable template template parameter. For a *concept-tt-parameter*, the name shall denote a concept or concept template parameter. Only primary templates are considered when matching the template template argument with the corresponding parameter;

partial specializations are not considered even if their parameter lists match that of the template template parameter.

- <sup>2</sup> Any partial specializations (13.7.6) associated with the primary template are considered when a specialization based on the template template parameter is instantiated. If a specialization is not reachable from the point of instantiation, and it would have been selected had it been reachable, the program is ill-formed, no diagnostic required (G.10.2).

[Example 1:

```
template<class T> class A { // primary template
 int x;
};
template<class T> class A<T*> { // partial specialization
 long x;
};
template<template<class U> class V> class C {
 V<int> y;
 V<int*> z;
};
C<A> c; // V<int> within C<A> uses the primary template, so c.y.x has type int
 // V<int*> within C<A> uses the partial specialization, so c.z.x has type long
```

— end example]

- <sup>3</sup> A template template parameter P and a *template-argument* A are *compatible* if

- (3.1) — A denotes a class template or an alias template and P is a type template template parameter,
- (3.2) — A denotes a variable template and P is a variable template template parameter, or
- (3.3) — A denotes a concept and P is a concept template parameter.

- <sup>4</sup> A template *template-argument* A matches a template template parameter P when A and P are compatible and P is at least as specialized as A, ignoring constraints on A if P is unconstrained. If P contains a template parameter pack, then A also matches P if each of A's template parameters matches the corresponding template parameter declared in the *template-head* of P. Two template parameters match if they are of the same kind, for constant template parameters, their types are equivalent (13.7.7.2), and for template template parameters, each of their corresponding template parameters matches, recursively. When P's *template-head* contains a *template-parameter* that declares a template parameter pack (13.7.4), the template parameter pack will match zero or more template parameters or template parameter packs declared in the *template-head* of A with the same type and form as the template parameter pack declared in P (ignoring whether those template parameters are template parameter packs).

[Example 2:

```
template<class T> class A { /* ... */ };
template<class T, class U = T> class B { /* ... */ };
template<class ... Types> class C { /* ... */ };
template<auto n> class D { /* ... */ };
template<template<class> class P> class X { /* ... */ };
template<template<class ...> class Q> class Y { /* ... */ };
template<template<int> class R> class Z { /* ... */ };

X<A> xa; // OK
X xb; // OK
X<C> xc; // OK
Y<A> ya; // OK
Y yb; // OK
Y<C> yc; // OK
Z<D> zd; // OK
```

— end example]

[Example 3:

```
template <class T> struct eval;

template <template <class, class...> class TT, class T1, class... Rest>
struct eval<TT<T1, Rest...>> { };
```

```

template <class T1> struct A;
template <class T1, class T2> struct B;
template <int N> struct C;
template <class T1, int N> struct D;
template <class T1, class T2, int N = 17> struct E;

eval<A<int>> eA; // OK, matches partial specialization of eval
eval<B<int, float>> eB; // OK, matches partial specialization of eval
eval<C<17>> eC; // error: C does not match TT in partial specialization
eval<D<int, 17>> eD; // error: D does not match TT in partial specialization
eval<E<int, float>> eE; // error: E does not match TT in partial specialization

```

— end example]

[Example 4:

```

template<typename T> concept C = requires (T t) { t.f(); };
template<typename T> concept D = C<T> && requires (T t) { t.g(); };

template<template<C> class P> struct S { };

template<C> struct X { };
template<D> struct Y { };
template<typename T> struct Z { };

S<X> s1; // OK, X and P have equivalent constraints
S<Y> s2; // error: P is not at least as specialized as Y
S<Z> s3; // OK, P is at least as specialized as Z

```

— end example]

- <sup>5</sup> A template template parameter *P* is at least as specialized as a template *template-argument* *A* if, given the following rewrite to two function templates, the function template corresponding to *P* is at least as specialized as the function template corresponding to *A* according to the partial ordering rules for function templates (13.7.7.3). Given an invented class template *X* with the *template-head* of *A* (including default arguments and *requires-clause*, if any):

- (5.1) — Each of the two function templates has the same template parameters and *requires-clause* (if any), respectively, as *P* or *A*.
- (5.2) — Each function template has a single function parameter whose type is a specialization of *X* with template arguments corresponding to the template parameters from the respective function template where, for each *template-parameter* *PP* in the *template-head* of the function template, a corresponding *template-argument* *AA* is formed. If *PP* declares a template parameter pack, then *AA* is the pack expansion *PP...* (13.7.4); otherwise, *AA* is an *id-expression* denoting *PP*.

If the rewrite produces an invalid type, then *P* is not at least as specialized as *A*.

## 13.5 Template constraints

[temp.constr]

### 13.5.1 General

[temp.constr.general]

- <sup>1</sup> [Note 1: Subclause 13.5 defines the meaning of constraints on template arguments. The abstract syntax and satisfaction rules are defined in 13.5.2. Constraints are associated with declarations in 13.5.3. Declarations are partially ordered by their associated constraints (13.5.5). — end note]

### 13.5.2 Constraints

[temp.constr.constr]

#### 13.5.2.1 General

[temp.constr.constr.general]

- <sup>1</sup> A *constraint* is a sequence of logical operations and operands that specifies requirements on template arguments. The operands of a logical operation are constraints. There are five different kinds of constraints:

- (1.1) — conjunctions (13.5.2.2),
- (1.2) — disjunctions (13.5.2.2),
- (1.3) — atomic constraints (13.5.2.3),
- (1.4) — concept-dependent constraints (13.5.2.4), and
- (1.5) — fold expanded constraints (13.5.2.5).

- <sup>2</sup> In order for a constrained template to be instantiated (13.9), its associated constraints (13.5.3) shall be satisfied as described in the following subclauses.

[*Note 1*: Forming the name of a specialization of a class template, a variable template, or an alias template (13.3) requires the satisfaction of its constraints. Overload resolution (12.2.3) requires the satisfaction of constraints on functions and function templates. — *end note*]

### 13.5.2.2 Logical operations

[temp.constr.op]

- <sup>1</sup> There are two binary logical operations on constraints: conjunction and disjunction.

[*Note 1*: These logical operations have no corresponding C++ syntax. For the purpose of exposition, conjunction is spelled using the symbol  $\wedge$  and disjunction is spelled using the symbol  $\vee$ . The operands of these operations are called the left and right operands. In the constraint  $A \wedge B$ ,  $A$  is the left operand, and  $B$  is the right operand. — *end note*]

- <sup>2</sup> A *conjunction* is a constraint taking two operands. To determine if a conjunction is *satisfied*, the satisfaction of the first operand is checked. If that is not satisfied, the conjunction is not satisfied. Otherwise, the conjunction is satisfied if and only if the second operand is satisfied.
- <sup>3</sup> A *disjunction* is a constraint taking two operands. To determine if a disjunction is *satisfied*, the satisfaction of the first operand is checked. If that is satisfied, the disjunction is satisfied. Otherwise, the disjunction is satisfied if and only if the second operand is satisfied.

- <sup>4</sup> [*Example 1*:

```
template<typename T>
constexpr bool get_value() { return T::value; }

template<typename T>
requires (sizeof(T) > 1) && (get_value<T>())
void f(T); // has associated constraint sizeof(T) > 1 ^ get_value<T>()

void f(int);

f('a'); // OK, calls f(int)
```

In the satisfaction of the associated constraints (13.5.3) of `f`, the constraint `sizeof(char) > 1` is not satisfied; the second operand is not checked for satisfaction. — *end example*]

- <sup>5</sup> [*Note 2*: A logical negation expression (7.6.2.2) is an atomic constraint; the negation operator is not treated as a logical operation on constraints. As a result, distinct negation *constraint-expressions* that are equivalent under 13.7.7.2 do not subsume one another under 13.5.5. Furthermore, if substitution to determine whether an atomic constraint is satisfied (13.5.2.3) encounters a substitution failure, the constraint is not satisfied, regardless of the presence of a negation operator.

[*Example 2*:

```
template <class T> concept sad = false;

template <class T> int f1(T) requires (!sad<T>);
template <class T> int f1(T) requires (!sad<T>) && true;
int i1 = f1(42); // ambiguous, !sad<T> atomic constraint expressions (13.5.2.3)
 // are not formed from the same expression

template <class T> concept not_sad = !sad<T>;
template <class T> int f2(T) requires not_sad<T>;
template <class T> int f2(T) requires not_sad<T> && true;
int i2 = f2(42); // OK, !sad<T> atomic constraint expressions both come from not_sad

template <class T> int f3(T) requires (!sad<typename T::type>);
int i3 = f3(42); // error: associated constraints not satisfied due to substitution failure

template <class T> concept sad_nested_type = sad<typename T::type>;
template <class T> int f4(T) requires (!sad_nested_type<T>);
int i4 = f4(42); // OK, substitution failure contained within sad_nested_type
```

Here, `requires (!sad<typename T::type>)` requires that there is a nested `type` that is not `sad`, whereas `requires (!sad_nested_type<T>)` requires that there is no `sad` nested `type`. — *end example*]

— *end note*]

## 13.5.2.3 Atomic constraints

[temp.constr.atomic]

- <sup>1</sup> An *atomic constraint* is formed from an expression *E* and a mapping from the template parameters that appear within *E* to template arguments that are formed via substitution during constraint normalization in the declaration of a constrained entity (and, therefore, can involve the unsubstituted template parameters of the constrained entity), called the *parameter mapping* (13.5.3).

[Note 1: Atomic constraints are formed by constraint normalization (13.5.4). *E* is never a logical AND expression (7.6.14) nor a logical OR expression (7.6.15). — end note]

- <sup>2</sup> Two atomic constraints, *e*<sub>1</sub> and *e*<sub>2</sub>, are *identical* if they are formed from the same appearance of the same *expression* and if, given a hypothetical template *A* whose *template-parameter-list* consists of *template-parameters* corresponding and equivalent (13.7.7.2) to those mapped by the parameter mappings of the expression, a *template-id* naming *A* whose *template-arguments* are the targets of the parameter mapping of *e*<sub>1</sub> is the same (13.6) as a *template-id* naming *A* whose *template-arguments* are the targets of the parameter mapping of *e*<sub>2</sub>.

[Note 2: The comparison of parameter mappings of atomic constraints operates in a manner similar to that of declaration matching with alias template substitution (13.7.8).

[Example 1:

```
template <unsigned N> constexpr bool Atomic = true;
template <unsigned N> concept C = Atomic<N>;
template <unsigned N> concept Add1 = C<N + 1>;
template <unsigned N> concept AddOne = C<N + 1>;
template <unsigned M> void f()
 requires Add1<2 * M>;
template <unsigned M> int f()
 requires AddOne<2 * M> && true;

int x = f<0>(); // OK, the atomic constraints from concept C in both fs are Atomic<N>
 // with mapping similar to N ↦ 2 * M + 1

template <unsigned N> struct WrapN;
template <unsigned N> using Add1Ty = WrapN<N + 1>;
template <unsigned N> using AddOneTy = WrapN<N + 1>;
template <unsigned M> void g(Add1Ty<2 * M> *);
template <unsigned M> void g(AddOneTy<2 * M> *);

void h() {
 g<0>(nullptr); // OK, there is only one g
}
```

— end example]

As specified in 13.7.7.2, if the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required (G.10.3).

[Example 2:

```
template <unsigned N> void f2()
 requires Add1<2 * N>;
template <unsigned N> int f2()
 requires Add1<N * 2> && true;
void h2() {
 f2<0>(); // ill-formed, no diagnostic required:
 // requires determination of subsumption between atomic constraints that are
 // functionally equivalent but not equivalent
}
```

— end example]

— end note]

- <sup>3</sup> To determine if an atomic constraint is *satisfied*, the parameter mapping and template arguments are first substituted into its expression. If substitution results in an invalid type or expression in the immediate context of the atomic constraint (13.10.3.1), the constraint is not satisfied. Otherwise, the lvalue-to-rvalue conversion (7.3.2) is performed if necessary, and *E* shall be a constant expression of type *bool*. The constraint is satisfied if and only if evaluation of *E* results in *true*. If, at different points in the program, the satisfaction

result is different for identical atomic constraints and template arguments, the program is ill-formed, no diagnostic required (G.10.4).

[Example 3:

```
template<typename T> concept C =
 sizeof(T) == 4 && !true; // requires atomic constraints sizeof(T) == 4 and !true

template<typename T> struct S {
 constexpr operator bool() const { return true; }
};

template<typename T> requires (S<T>{})
void f(T); // #1
void f(int); // #2

void g() {
 f(0); // error: expression S<int>{} does not have type bool
} // while checking satisfaction of deduced arguments of #1;
 // call is ill-formed even though #2 is a better match
```

— end example]

### 13.5.2.4 Concept-dependent constraints

[temp.constr.concept]

- <sup>1</sup> A *concept-dependent constraint* **CD** is an atomic constraint whose expression is a concept-id **CI** whose *concept-name* names a dependent concept named **C**.
- <sup>2</sup> To determine if **CD** is *satisfied*, the parameter mapping and template arguments are first substituted into **C**. If substitution results in an invalid concept-id in the immediate context of the constraint (13.10.3.1), the constraint is not satisfied. Otherwise, let **CI'** be the normal form (13.5.4) of the concept-id after substitution of **C**.

[Note 1: Normalization of **CI** can be ill-formed with no diagnostic required. — end note]

- <sup>3</sup> To form **CI''**, each appearance of **C**'s template parameters in the parameter mappings of the atomic constraints (including concept-dependent constraints) in **CI'** is substituted with their respective arguments from the parameter mapping of **CD** and the arguments of **CI**.
- <sup>4</sup> **CD** is satisfied if **CI''** is satisfied.

[Note 2: Checking whether **CI''** is satisfied can lead to further normalization of concept-dependent constraints. — end note]

[Example 1:

```
template<typename>
concept C = true;

template<typename T, template<typename> concept CC>
concept D = CC<T>;

template<typename U,
 template<typename> concept CT,
 template<typename, template<typename> concept> concept CU>
int f() requires CU<U, CT>;
int i = f<int, C, D>();
```

In this example, the associated constraints of **f** consist of a concept-dependent constraint whose expression is the concept-id **CU<U, CT>** with the mapping  $U \mapsto U, CT \mapsto CT, CU \mapsto CU$ . The result of substituting **D** into this expression is **D<U, CT>**. We consider the normal form of the resulting concept-id, which is **CC<T>** with the mapping  $T \mapsto U, CC \mapsto CT$ . By recursion, **C** is substituted into **CC<T>**, and the result is normalized to the atomic constraint **true**, which is satisfied.  
— end example]

### 13.5.2.5 Fold expanded constraint

[temp.constr.fold]

- <sup>1</sup> A *fold expanded constraint* is formed from a constraint **C** and a *fold-operator* which can either be **&&** or **||**. A fold expanded constraint is a pack expansion (13.7.4). Let **N** be the number of elements in the pack expansion parameters (13.7.4).



- <sup>2</sup> A fold expanded constraint whose *fold-operator* is `&&` is satisfied if it is a valid pack expansion and if  $N = 0$  or if for each  $i$  where  $0 \leq i < N$  in increasing order,  $C$  is satisfied when replacing each pack expansion parameter with the corresponding  $i^{\text{th}}$  element. No substitution takes place for any  $i$  greater than the smallest  $i$  for which the constraint is not satisfied.
- <sup>3</sup> A fold expanded constraint whose *fold-operator* is `||` is satisfied if it is a valid pack expansion,  $N > 0$ , and if for  $i$  where  $0 \leq i < N$  in increasing order, there is a smallest  $i$  for which  $C$  is satisfied when replacing each pack expansion parameter with the corresponding  $i^{\text{th}}$  element. No substitution takes place for any  $i$  greater than the smallest  $i$  for which the constraint is satisfied.
- <sup>4</sup> [Note 1: If the pack expansion expands packs of different size, then it is invalid and the fold expanded constraint is not satisfied. — end note]
- <sup>5</sup> Two fold expanded constraints are *compatible for subsumption* if their respective constraints both contain an equivalent unexpanded pack (13.7.7.2).

### 13.5.3 Constrained declarations

[temp.constr.decl]

- <sup>1</sup> A template declaration (13.1) or templated function declaration (9.3.4.6) can be constrained by the use of a *requires-clause*. This allows the specification of constraints for that declaration as an expression:

*constraint-expression*:  
*logical-or-expression*

- <sup>2</sup> Constraints can also be associated with a declaration through the use of *type-constraints* in a *template-parameter-list* or parameter-type-list. Each of these forms introduces additional *constraint-expressions* that are used to constrain the declaration.
- <sup>3</sup> A declaration's *associated constraints* are defined as follows:
  - (3.1) — If there are no introduced *constraint-expressions*, the declaration has no associated constraints.
  - (3.2) — Otherwise, if there is a single introduced *constraint-expression*, the associated constraints are the normal form (13.5.4) of that expression.
  - (3.3) — Otherwise, the associated constraints are the normal form of a logical AND expression (7.6.14) whose operands are in the following order:
    - (3.3.1) — the *constraint-expression* introduced by each *type-constraint* (13.2) in the declaration's *template-parameter-list*, in order of appearance, and
    - (3.3.2) — the *constraint-expression* introduced by a *requires-clause* following a *template-parameter-list* (13.1), and
    - (3.3.3) — the *constraint-expression* introduced by each *type-constraint* in the parameter-type-list of a function declaration, and
    - (3.3.4) — the *constraint-expression* introduced by a trailing *requires-clause* (9.3) of a function declaration (9.3.4.6).

The formation of the associated constraints establishes the order in which constraints are instantiated when checking for satisfaction (13.5.2).

[Example 1:

```
template<typename T> concept C = true;

template<C T> void f1(T);
template<typename T> requires C<T> void f2(T);
template<typename T> void f3(T) requires C<T>;
```

The functions f1, f2, and f3 have the associated constraint C<T>.

```
template<typename T> concept C1 = true;
template<typename T> concept C2 = sizeof(T) > 0;

template<C1 T> void f4(T) requires C2<T>;
template<typename T> requires C1<T> && C2<T> void f5(T);
```

The associated constraints of f4 and f5 are C1<T>  $\wedge$  C2<T>.

```
template<C1 T> requires C2<T> void f6();
template<C2 T> requires C1<T> void f7();
```

The associated constraints of f6 are C1<T>  $\wedge$  C2<T>, and those of f7 are C2<T>  $\wedge$  C1<T>. — end example]



- <sup>4</sup> When determining whether a given introduced *constraint-expression*  $C_1$  of a declaration in an instantiated specialization of a templated class is equivalent (13.7.7.2) to the corresponding *constraint-expression*  $C_2$  of a declaration outside the class body,  $C_1$  is instantiated. If the instantiation results in an invalid expression, the *constraint-expressions* are not equivalent.

[Note 1: This can happen when determining which member template is specialized by an explicit specialization declaration. — end note]

[Example 2:

```
template <class T> concept C = true;
template <class T> struct A {
 template <class U> U f(U) requires C<typename T::type>; // #1
 template <class U> U f(U) requires C<T>; // #2
};

template <> template <class U>
U A<int>::f(U u) requires C<int> { return u; } // OK, specializes #2
```

Substituting `int` for `T` in `C<typename T::type>` produces an invalid expression, so the specialization does not match #1. Substituting `int` for `T` in `C<T>` produces `C<int>`, which is equivalent to the *constraint-expression* for the specialization, so it does match #2. — end example]

### 13.5.4 Constraint normalization

[temp.constr.normal]

- <sup>1</sup> The *normal form* of an *expression*  $E$  is a constraint (13.5.2) that is defined as follows:

- (1.1) — The normal form of an expression  $( E )$  is the normal form of  $E$ .
- (1.2) — The normal form of an expression  $E1 \ || \ E2$  is the disjunction (13.5.2.2) of the normal forms of  $E1$  and  $E2$ .
- (1.3) — The normal form of an expression  $E1 \ \&\& \ E2$  is the conjunction of the normal forms of  $E1$  and  $E2$ .
- (1.4) — For a concept-id  $C<A_1, A_2, \dots, A_n>$  termed  $CI$ :
  - (1.4.1) — If  $C$  names a dependent concept, the normal form of  $CI$  is a concept-dependent constraint whose concept-id is  $CI$  and whose parameter mapping is the identity mapping.
  - (1.4.2) — Otherwise, to form  $CE$ , any non-dependent concept template argument  $A_i$  is substituted into the *constraint-expression* of  $C$ . If any such substitution results in an invalid concept-id, the program is ill-formed; no diagnostic is required. The normal form of  $CI$  is the result of substituting, in the normal form  $N$  of  $CE$ , appearances of  $C$ 's template parameters in the parameter mappings of the atomic constraints in  $N$  with their respective arguments from  $CI$ . If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required.

[Example 1:

```
template<typename T> concept A = T::value || true;
template<typename U> concept B = A<U*>;
template<typename V> concept C = B<V&>;
```

Normalization of  $B$ 's *constraint-expression* is valid and results in  $T::value$  (with the mapping  $T \mapsto U^*$ )  $\vee$  `true` (with an empty mapping), despite the expression  $T::value$  being ill-formed for a pointer type  $T$ . Normalization of  $C$ 's *constraint-expression* results in the program being ill-formed, because it would form the invalid type  $V\&^*$  in the parameter mapping. — end example]

- (1.5) — For a *fold-operator*  $Op$  (7.5.7) that is either `&&` or `||`:
  - (1.5.1) — The normal form of an expression  $( \dots Op E )$  is the normal form of  $( E Op \dots )$ .
  - (1.5.2) — The normal form of an expression  $( E1 Op \dots Op E2 )$  is the normal form of
    - (1.5.2.1) —  $( E1 Op \dots ) Op E2$  if  $E1$  contains an unexpanded pack, or
    - (1.5.2.2) —  $E1 Op ( E2 Op \dots )$  otherwise.
  - (1.5.3) — The normal form of an expression  $F$  of the form  $( E Op \dots )$  is as follows:
    - If  $E$  contains an unexpanded concept template parameter pack, it shall not contain an unexpanded template parameter pack of another kind. Let  $E'$  be the normal form of  $E$ .
    - (1.5.3.1) — If  $E$  contains an unexpanded concept template parameter pack  $P_k$  that has corresponding template arguments in the parameter mapping of any atomic constraint (including concept-dependent constraints) of  $E'$ , the number of arguments specified for all such  $P_k$  shall be the same

number  $N$ . The normal form of  $F$  is the normal form of  $E_0 \text{ Op } \dots \text{ Op } E_{N-1}$  after substituting in  $E_i$  the respective  $i^{\text{th}}$  concept argument of each  $P_k$ . If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required (G.10.5).

(1.5.3.2) — Otherwise, the normal form of  $F$  is a fold expanded constraint (13.5.2.5) whose constraint is  $E'$  and whose *fold-operator* is  $\text{Op}$ .

(1.6) — The normal form of any other expression  $E$  is the atomic constraint whose expression is  $E$  and whose parameter mapping is the identity mapping.

2 The process of obtaining the normal form of a *constraint-expression* is called *normalization*.

[Note 1: Normalization of *constraint-expressions* is performed when determining the associated constraints (13.5.2) of a declaration and when evaluating the value of an *id-expression* that names a concept specialization (7.5.5). — end note]

3 [Example 2:

```
template<typename T> concept C1 = sizeof(T) == 1;
template<typename T> concept C2 = C1<T> && 1 == 2;
template<typename T> concept C3 = requires { typename T::type; };
template<typename T> concept C4 = requires (T x) { ++x; };

template<C2 U> void f1(U); // #1
template<C3 U> void f2(U); // #2
template<C4 U> void f3(U); // #3
```

The associated constraints of #1 are `sizeof(T) == 1` (with mapping  $T \mapsto U$ )  $\wedge$  `1 == 2`.

The associated constraints of #2 are `requires { typename T::type; }` (with mapping  $T \mapsto U$ ).

The associated constraints of #3 are `requires (T x) { ++x; }` (with mapping  $T \mapsto U$ ). — end example]

[Example 3:

```
template<typename T>
concept C = true;
template<typename T, template<typename> concept CT>
concept CC = CT<T>;

template<typename U,
 template<typename, template<typename> concept> concept CT>
void f() requires CT<U*, C>;
template<typename U>
void g() requires CC<U*, C>;
```

The normal form of the associated constraints of `f` is the concept-dependent constraint `CT<T, C>`.

The normal form of the associated constraints of `g` is the atomic constraint `true`. — end example]

[Example 4:

```
template<typename T>
concept A = true;
template<typename T>
concept B = A<T> && true; // B subsumes A
template<typename T>
concept C = true;
template<typename T>
concept D = C<T> && true; // D subsumes C

template<typename T, template<typename> concept... CTs>
concept all_of = (CTs<T> && ...);

template<typename T> requires all_of<T, A, C>
constexpr int f(T) { return 1; } // #1
template<typename T> requires all_of<T, B, D>
constexpr int f(T) { return 2; } // #2

static_assert(f(1) == 2); // ok
```

The normal form of `all_of<T, A, C>` is the conjunction of the normal forms of `A<T>` and `C<T>`.

Similarly, the normal form of `all_of<T, B, D>` is the conjunction of the normal forms of `B<T>` and `D<T>`.

#2 therefore is more constrained than #1. — end example]

[Example 5:

```
template<typename T, template<typename> concept>
struct wrapper {};

template<typename... T, template<typename> concept... CTs>
int f(wrapper<T, CTs>...) requires (CTs<T> && ...); // error: fold expression contains
// different kinds of template parameters
```

— end example]

### 13.5.5 Partial ordering by constraints

[temp.constr.order]

- <sup>1</sup> A constraint  $P$  *subsumes* a constraint  $Q$  if and only if, for every disjunctive clause  $P_i$  in the disjunctive normal form<sup>103</sup> of  $P$ ,  $P_i$  subsumes every conjunctive clause  $Q_j$  in the conjunctive normal form<sup>104</sup> of  $Q$ , where
  - (1.1) — a disjunctive clause  $P_i$  subsumes a conjunctive clause  $Q_j$  if and only if there exists an atomic constraint  $P_{ia}$  in  $P_i$  for which there exists an atomic constraint  $Q_{jb}$  in  $Q_j$  such that  $P_{ia}$  subsumes  $Q_{jb}$ ,
  - (1.2) — an atomic constraint  $A$  subsumes another atomic constraint  $B$  if and only if  $A$  and  $B$  are identical using the rules described in 13.5.2.3, and
  - (1.3) — a fold expanded constraint  $A$  subsumes another fold expanded constraint  $B$  if they are compatible for subsumption, have the same *fold-operator*, and the constraint of  $A$  subsumes that of  $B$ .

[Example 1: Let  $A$  and  $B$  be atomic constraints (13.5.2.3). The constraint  $A \wedge B$  subsumes  $A$ , but  $A$  does not subsume  $A \wedge B$ . The constraint  $A$  subsumes  $A \vee B$ , but  $A \vee B$  does not subsume  $A$ . Also note that every constraint subsumes itself. — end example]
- <sup>2</sup> [Note 1: The subsumption relation defines a partial ordering on constraints. This partial ordering is used to determine
  - (2.1) — the best viable candidate of non-template functions (12.2.4),
  - (2.2) — the address of a non-template function (12.3),
  - (2.3) — the matching of template template arguments (13.4.4),
  - (2.4) — the partial ordering of class template specializations (13.7.6.3), and
  - (2.5) — the partial ordering of function templates (13.7.7.3).

— end note]
- <sup>3</sup> The associated constraints  $C$  of a declaration  $D$  are *eligible for subsumption* unless  $C$  contains a concept-dependent constraint.
- <sup>4</sup> A declaration  $D1$  is *at least as constrained* as a declaration  $D2$  if
  - (4.1) —  $D1$  and  $D2$  are both constrained declarations and  $D1$ 's associated constraints are eligible for subsumption and subsume those of  $D2$ ; or
  - (4.2) —  $D2$  has no associated constraints.
- <sup>5</sup> A declaration  $D1$  is *more constrained* than another declaration  $D2$  when  $D1$  is at least as constrained as  $D2$ , and  $D2$  is not at least as constrained as  $D1$ .

[Example 2:

```
template<typename T> concept C1 = requires(T t) { --t; };
template<typename T> concept C2 = C1<T> && requires(T t) { *t; };

template<C1 T> void f(T); // #1
template<C2 T> void f(T); // #2
template<typename T> void g(T); // #3
template<C1 T> void g(T); // #4

f(0); // selects #1
f((int*)0); // selects #2
g(true); // selects #3 because C1<bool> is not satisfied
```

103) A constraint is in disjunctive normal form when it is a disjunction of clauses where each clause is a conjunction of fold expanded or atomic constraints. For atomic constraints  $A$ ,  $B$ , and  $C$ , the disjunctive normal form of the constraint  $A \wedge (B \vee C)$  is  $(A \wedge B) \vee (A \wedge C)$ . Its disjunctive clauses are  $(A \wedge B)$  and  $(A \wedge C)$ .

104) A constraint is in conjunctive normal form when it is a conjunction of clauses where each clause is a disjunction of fold expanded or atomic constraints. For atomic constraints  $A$ ,  $B$ , and  $C$ , the constraint  $A \wedge (B \vee C)$  is in conjunctive normal form. Its conjunctive clauses are  $A$  and  $(B \vee C)$ .

```

 g(0); // selects #4
— end example]
[Example 3:
 template<template<typename T> concept CT, typename T>
 struct S {};
 template<typename T>
 concept A = true;

 template<template<typename T> concept X, typename T>
 int f(S<X, T>) requires A<T> { return 42; } // #1
 template<template<typename T> concept X, typename T>
 int f(S<X, T>) requires X<T> { return 43; } // #2

 f(S<A, int>{}); // ok, select #1 because #2 is not eligible for subsumption
— end example]

```

- <sup>6</sup> A non-template function F1 is *more partial-ordering-constrained* than a non-template function F2 if
- (6.1) — they have the same non-object-parameter-type-lists (9.3.4.6), and
  - (6.2) — if they are member functions, both are direct members of the same class, and
  - (6.3) — if both are non-static member functions, they have the same types for their object parameters, and
  - (6.4) — the declaration of F1 is more constrained than the declaration of F2.

### 13.6 Type equivalence

[temp.type]

- <sup>1</sup> Two *template-ids* are the same if

- (1.1) — their *template-names*, *operator-function-ids*, or *literal-operator-ids* refer to the same template, and
- (1.2) — their corresponding type *template-arguments* are the same type, and
- (1.3) — the template parameter values determined by their corresponding constant template arguments (13.4.3) are template-argument-equivalent (see below), and
- (1.4) — their corresponding template *template-arguments* refer to the same template.

Two *template-ids* that are the same refer to the same class, function, or variable.

- <sup>2</sup> Two values are *template-argument-equivalent* if they are of the same type and

- (2.1) — they are of integral type and their values are the same, or
- (2.2) — they are of floating-point type and their values are identical, or
- (2.3) — they are of type `std::nullptr_t`, or
- (2.4) — they are of type `std::meta::info` and their values compare equal (7.6.10), or
- (2.5) — they are of enumeration type and their values are the same,<sup>105</sup> or
- (2.6) — they are of pointer type and they have the same pointer value, or
- (2.7) — they are of pointer-to-member type and they refer to the same class member or are both the null member pointer value, or
- (2.8) — they are of reference type and they refer to the same object or function, or
- (2.9) — they are of array type and their corresponding elements are either both within their lifetimes and template-argument-equivalent or both not within their lifetimes,<sup>106</sup> or
- (2.10) — they are of union type and either they both have no active member or they have the same active member and their active members are template-argument-equivalent, or
- (2.11) — they are of a closure type (7.5.6.2), or
- (2.12) — they are of class type and their corresponding direct subobjects and reference members are template-argument-equivalent.

<sup>105</sup>) The identity of enumerators is not preserved.

<sup>106</sup>) An array as a *template-parameter* decays to a pointer.

3 [Example 1:

```
template<class E, int size> class buffer { /* ... */ };
buffer<char,2*512> x;
buffer<char,1024> y;
```

declares `x` and `y` to be of the same type, and

```
template<class T, void(*err_fct)()> class list { /* ... */ };
list<int,&error_handler1> x1;
list<int,&error_handler2> x2;
list<int,&error_handler2> x3;
list<char,&error_handler2> x4;
```

declares `x2` and `x3` to be of the same type. Their type differs from the types of `x1` and `x4`.

```
template<class T> struct X { };
template<class> struct Y { };
template<class T> using Z = Y<T>;
X<Y<int>> > y;
X<Z<int>> > z;
```

declares `y` and `z` to be of the same type. — end example]

4 If an expression *e* is type-dependent (13.8.3.3), `decltype(e)` denotes a unique dependent type. Two such *decltype-specifiers* refer to the same type only if their *expressions* are equivalent (13.7.7.2).

[Note 1: However, such a type might be aliased, e.g., by a *typedef-name*. — end note]

5 For a type template parameter pack `T`, `T... [constant-expression]` denotes a unique dependent type. Two such *pack-index-specifiers* refer to the same type only if

(5.1) — their *typedef-names* refer to the same template parameter pack and

(5.2) — if neither of their *constant-expressions* is value-dependent, then they have the same value, otherwise their *constant-expressions* are equivalent (13.7.7.2).

6 Two *pack-index-template-names* refer to the same template only if

(6.1) — their *simple-template-names* refer to the same template parameter pack and

(6.2) — if neither of their *constant-expressions* is value-dependent, then they have the same value, otherwise their *constant-expressions* are equivalent (13.7.7.2).

## 13.7 Template declarations

[temp.decls]

### 13.7.1 General

[temp.decls.general]

1 The template parameters of a template are specified in the angle bracket enclosed list that immediately follows the keyword `template`.

2 A *primary template* declaration is one in which the name of the template is not followed by a *template-argument-list*. The template argument list of a primary template is the template argument list of its *template-head* (13.4). A template declaration in which the name of the template is followed by a *template-argument-list* is a partial specialization (13.7.6) of the template named in the declaration, which shall be a class or variable template.

3 A *separately instantiated construct* of a templated function `F` is a

(3.1) — default argument,

(3.2) — *noexcept-specifier*, or

(3.3) — *function-contract-specifier*

of `F`. For purposes of name lookup and instantiation, separately instantiated constructs, *type-constraints*, and *requires-clauses* (13.1) are considered definitions; each separately instantiated construct, *type-constraint*, or *requires-clause* is a separate definition which is unrelated to the templated function definition or to any other separately instantiated constructs, *type-constraints*, or *requires-clauses*. For the purpose of instantiation, the substatements of a `constexpr` if statement (8.5.2) are considered definitions. For the purpose of name lookup and instantiation, the *compound-statement* of an *expansion-statement* is considered a template definition.

4 Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template.

## 13.7.2 Class templates

[temp.class]

### 13.7.2.1 General

[temp.class.general]

- <sup>1</sup> A *class template* defines the layout and operations for an unbounded set of related types.
- <sup>2</sup> [Example 1: It is possible for a single class template `List` to provide an unbounded set of class definitions: one class `List<T>` for every type `T`, each describing a linked list of elements of type `T`. Similarly, a class template `Array` describing a contiguous, dynamic array can be defined like this:

```
template<class T> class Array {
 T* v;
 int sz;
public:
 explicit Array(int);
 T& operator[](int);
 T& elem(int i) { return v[i]; }
};
```

The prefix `template<class T>` specifies that a template is being declared and that a *type-name* `T` can be used in the declaration. In other words, `Array` is a parameterized type with `T` as its parameter. — end example]

- <sup>3</sup> [Note 1: When a member of a class template is defined outside of the class template definition, the member definition is defined as a template definition with the *template-head* equivalent to that of the class template. The names of the template parameters used in the definition of the member can differ from the template parameter names used in the class template definition. The class template name in the member definition is followed by the template argument list of the *template-head* (13.4).

[Example 2:

```
template<class T1, class T2> struct A {
 void f1();
 void f2();
};

template<class T2, class T1> void A<T2,T1>::f1() { } // OK
template<class T2, class T1> void A<T1,T2>::f2() { } // error

template<class ... Types> struct B {
 void f3();
 void f4();
};

template<class ... Types> void B<Types ...>::f3() { } // OK
template<class ... Types> void B<Types>::f4() { } // error

template<typename T> concept C = true;
template<typename T> concept D = true;

template<C T> struct S {
 void f();
 void g();
 void h();
 template<D U> struct Inner;
};

template<C A> void S<A>::f() { } // OK, template-heads match
template<typename T> void S<T>::g() { } // error: no matching declaration for S<T>

template<typename T> requires C<T> // ill-formed, no diagnostic required: template-heads are
void S<T>::h() { } // functionally equivalent but not equivalent

template<C X> template<D Y>
struct S<X>::Inner { }; // OK
```

— end example]

— end note]

- <sup>4</sup> In a partial specialization, explicit specialization or explicit instantiation of a class template, the *class-key* shall agree in kind with the original class template declaration (9.2.9.5).

### 13.7.2.2 Member functions of class templates

[temp.mem.func]

- <sup>1</sup> A member function of a class template may be defined outside of the class template definition in which it is declared.

[Example 1:

```
template<class T> class Array {
 T* v;
 int sz;
public:
 explicit Array(int);
 T& operator[](int);
 T& elem(int i) { return v[i]; }
};
```

declares three member functions of a class template. The subscript function can be defined like this:

```
template<class T> T& Array<T>::operator[](int i) {
 if (i<0 || sz<=i) error("Array: range error");
 return v[i];
}
```

A constrained member function can be defined out of line:

```
template<typename T> concept C = requires {
 typename T::type;
};

template<typename T> struct S {
 void f() requires C<T>;
 void g() requires C<T>;
};

template<typename T>
void S<T>::f() requires C<T> { } // OK
template<typename T>
void S<T>::g() { } // error: no matching function in S<T>
```

— end example]

- <sup>2</sup> The *template-arguments* for a member function of a class template are determined by the *template-arguments* of the type of the object for which the member function is called.

[Example 2: The *template-argument* for `Array<T>::operator[]` will be determined by the `Array` to which the subscripting operation is applied.

```
Array<int> v1(20);
Array<dcomplex> v2(30);

v1[3] = 7; // Array<int>::operator[]
v2[3] = dcomplex(7,8); // Array<dcomplex>::operator[]
```

— end example]

### 13.7.2.3 Deduction guides

[temp.deduct.guide]

- <sup>1</sup> Deduction guides are used when a *template-name* or *splice-type-specifier* appears as a type specifier for a deduced class type (9.2.9.8). Deduction guides are not found by name lookup. Instead, when performing class template argument deduction (12.2.2.9), all reachable deduction guides declared for the class template are considered.

*deduction-guide:*

*explicit-specifier<sub>opt</sub> simple-template-name ( parameter-declaration-clause ) -> simple-template-id requires-clause<sub>opt</sub> ;*

- <sup>2</sup> [Example 1:

```
template<class T, class D = int>
struct S {
 T data;
};
```

```

template<class U>
S(U) -> S<typename U::type>;

struct A {
 using type = short;
 operator type();
};
S x{A()}; // x is of type S<short, int>
— end example]

```

- <sup>3</sup> The same restrictions apply to the *parameter-declaration-clause* of a deduction guide as in a function declaration (9.3.4.6), except that a generic parameter type placeholder (9.2.9.7) shall not appear in the *parameter-declaration-clause* of a deduction guide. The *simple-template-id* shall name a class template specialization. The *simple-template-name* shall be the same *identifier* as the *template-name* of the *simple-template-id*. A *deduction-guide* shall inhabit the scope to which the corresponding class template belongs and, for a member class template, have the same access. Two deduction guide declarations for the same class template shall not have equivalent *parameter-declaration-clauses* if either is reachable from the other.

#### 13.7.2.4 Member classes of class templates

[temp.mem.class]

- <sup>1</sup> A member class of a class template may be defined outside the class template definition in which it is declared.

[Note 1: The member class must be defined before its first use that requires an instantiation (13.9.2). For example,

```

template<class T> struct A {
 class B;
};
A<int>::B* b1; // OK, requires A to be defined but not A::B
template<class T> class A<T>::B { };
A<int>::B b2; // OK, requires A::B to be defined
— end note]

```

#### 13.7.2.5 Static data members of class templates

[temp.static]

- <sup>1</sup> A definition for a static data member or static data member template may be provided in a namespace scope enclosing the definition of the static member's class template.

[Example 1:

```

template<class T> class X {
 static T s;
};
template<class T> T X<T>::s = 0;

struct limits {
 template<class T>
 static const T min; // declaration
};

template<class T>
const T limits::min = { }; // definition
— end example]

```

- <sup>2</sup> An explicit specialization of a static data member declared as an array of unknown bound can have a different bound from its definition, if any.

[Example 2:

```

template <class T> struct A {
 static int i[];
};
template <class T> int A<T>::i[4]; // 4 elements
template <> int A<int>::i[] = { 1 }; // OK, 1 element
— end example]

```

#### 13.7.2.6 Enumeration members of class templates

[temp.mem.enum]

- <sup>1</sup> An enumeration member of a class template may be defined outside the class template definition.



[Example 1:

```
template<class T> struct A {
 enum E : T;
};
template<class T> enum A<T>::E : T { e1, e2 };
A<int>::E e = A<int>::e1;
```

— end example]

### 13.7.3 Member templates

[temp.mem]

- <sup>1</sup> A template can be declared within a class or class template; such a template is called a member template. A member template can be defined within or outside its class definition or class template definition. A member template of a class template that is defined outside of its class template definition shall be specified with a *template-head* equivalent to that of the class template followed by a *template-head* equivalent to that of the member template (13.7.7.2).

[Example 1:

```
template<class T> struct string {
 template<class T2> int compare(const T2&);
 template<class T2> string(const string<T2>& s) { /* ... */ }
};

template<class T> template<class T2> int string<T>::compare(const T2& s) {
}
```

— end example]

[Example 2:

```
template<typename T> concept C1 = true;
template<typename T> concept C2 = sizeof(T) <= 4;

template<C1 T> struct S {
 template<C2 U> void f(U);
 template<C2 U> void g(U);
};

template<C1 T> template<C2 U>
void S<T>::f(U) { } // OK
template<C1 T> template<typename U>
void S<T>::g(U) { } // error: no matching function in S<T>
```

— end example]

- <sup>2</sup> A local class of non-closure type shall not have member templates. Access control rules (11.8) apply to member template names. A destructor shall not be a member template. A non-template member function (9.3.4.6) with a given name and type and a member function template of the same name, which could be used to generate a specialization of the same type, can both be declared in a class. When both exist, a use of that name and type refers to the non-template member unless an explicit template argument list is supplied.

[Example 3:

```
template <class T> struct A {
 void f(int);
 template <class T2> void f(T2);
};

template <> void A<int>::f(int) { } // non-template member function
template <> template <> void A<int>::f<>(int) { } // member function template specialization

int main() {
 A<char> ac;
 ac.f(1); // non-template
 ac.f('c'); // template
 ac.f<>(1); // template
}
```

— end example]

- <sup>3</sup> A member function template shall not be declared **virtual**.

[Example 4:

```
template <class T> struct AA {
 template <class C> virtual void g(C); // error
 virtual void f(); // OK
};
```

— end example]

- <sup>4</sup> A specialization of a member function template does not override a virtual function from a base class.

[Example 5:

```
class B {
 virtual void f(int);
};

class D : public B {
 template <class T> void f(T); // does not override B::f(int)
 void f(int i) { f<>(i); } // overriding function that calls the function template specialization
};
```

— end example]

- <sup>5</sup> [Note 1: A specialization of a conversion function template is named in the same way as a non-template conversion function that converts to the same type (11.4.8.3).

[Example 6:

```
struct A {
 template <class T> operator T*();
};
template <class T> A::operator T*() { return 0; }
template <> A::operator char*() { return 0; } // specialization
template A::operator void*(); // explicit instantiation

int main() {
 A a;
 int* ip;
 ip = a.operator int*(); // explicit call to template operator A::operator int*()
}
```

— end example]

An expression designating a particular specialization of a conversion function template can only be formed with a *splice-expression*. There is no analogous syntax to form a *template-id* (13.3) for such a function by providing an explicit template argument list (13.10.2). — end note]

### 13.7.4 Variadic templates

[temp.variadic]

- <sup>1</sup> A *template parameter pack* is a template parameter that accepts zero or more template arguments.

[Example 1:

```
template<class ... Types> struct Tuple { };

Tuple<> t0; // Types contains no arguments
Tuple<int> t1; // Types contains one argument: int
Tuple<int, float> t2; // Types contains two arguments: int and float
Tuple<0> error; // error: 0 is not a type
```

— end example]

- <sup>2</sup> A *function parameter pack* is a function parameter that accepts zero or more function arguments.

[Example 2:

```
template<class ... Types> void f(Types ... args);

f(); // args contains no arguments
f(1); // args contains one argument: int
f(2, 1.0); // args contains two arguments: int and double
```

— end example]

- <sup>3</sup> An *init-capture pack* is a lambda capture that introduces an *init-capture* for each of the elements in the pack expansion of its *initializer*.

[Example 3:

```
template <typename... Args>
void foo(Args... args) {
 [...xs=args]{
 bar(xs...); // xs is an init-capture pack
 };
}

foo(); // xs contains zero init-captures
foo(1); // xs contains one init-capture
```

— end example]

- <sup>4</sup> A *structured binding pack* is an *sb-identifier* that introduces zero or more structured bindings (9.7).

[Example 4:

```
auto foo() -> int(&)[2];

template <class T>
void g() {
 auto [...a] = foo(); // a is a structured binding pack containing two elements
 auto [b, c, ...d] = foo(); // d is a structured binding pack containing zero elements
}
```

— end example]

- <sup>5</sup> A *pack* is a template parameter pack, a function parameter pack, an *init-capture* pack, or a structured binding pack. The number of elements of a template parameter pack or a function parameter pack is the number of arguments provided for the parameter pack. The number of elements of an *init-capture* pack is the number of elements in the pack expansion of its *initializer*.

- <sup>6</sup> A *pack expansion* consists of a *pattern* and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

- (6.1) — In a function parameter pack (9.3.4.6); the pattern is the *parameter-declaration* without the ellipsis.
- (6.2) — In a *using-declaration* (9.10); the pattern is a *using-declarator*.
- (6.3) — In a *friend-type-declaration* (11.4.1); the pattern is a *friend-type-specifier*.
- (6.4) — In a template parameter pack that is a pack expansion (13.2):
  - (6.4.1) — if the template parameter pack is a *parameter-declaration*; the pattern is the *parameter-declaration* without the ellipsis;
  - (6.4.2) — if the template parameter pack is a *type-parameter*; the pattern is the corresponding *type-parameter* without the ellipsis;
  - (6.4.3) — if the template parameter pack is a template template parameter; the pattern is the corresponding *type-tt-parameter*, *variable-tt-parameter*, or *concept-tt-parameter* without the ellipsis.
- (6.5) — In an *initializer-list* (9.5); the pattern is an *initializer-clause*.
- (6.6) — In a *base-specifier-list* (11.7); the pattern is a *base-specifier*.
- (6.7) — In a *mem-initializer-list* (11.9.3) for a *mem-initializer* whose *mem-initializer-id* denotes a base class; the pattern is the *mem-initializer*.
- (6.8) — In a *template-argument-list* (13.4); the pattern is a *template-argument*.
- (6.9) — In an *attribute-list* (9.13.1); the pattern is an *attribute*.
- (6.10) — In an *annotation-list* (9.13.1); the pattern is an *annotation*.
- (6.11) — In an *alignment-specifier* (9.13.2); the pattern is the *alignment-specifier* without the ellipsis.
- (6.12) — In a *capture-list* (7.5.6.3); the pattern is the *capture* without the ellipsis.
- (6.13) — In a *sizeof... expression* (7.6.2.5); the pattern is an *identifier*.
- (6.14) — In a *pack-index-expression*; the pattern is an *identifier*.

- (6.15) — In a *pack-index-specifier*; the pattern is a *typedef-name*.
- (6.16) — In a *pack-index-template-name*; the pattern is a *simple-template-name*.
- (6.17) — In a *fold-expression* (7.5.7); the pattern is the *cast-expression* that contains an unexpanded pack.
- (6.18) — In a fold expanded constraint (13.5.2.5); the pattern is the constraint of that fold expanded constraint.

[Example 5:

```
template<class ... Types> void f(Types ... rest);
template<class ... Types> void g(Types ... rest) {
 f(&rest ...); // “&rest ...” is a pack expansion; “&rest” is its pattern
}
```

— end example]

- 7 For the purpose of determining whether a pack satisfies a rule regarding entities other than packs, the pack is considered to be the entity that would result from an instantiation of the pattern in which it appears.
- 8 A pack whose name appears within the pattern of a pack expansion is expanded by that pack expansion. An appearance of the name of a pack is only expanded by the innermost enclosing pack expansion. The pattern of a pack expansion shall name one or more packs that are not expanded by a nested pack expansion; such packs are called *unexpanded packs* in the pattern. All of the packs expanded by a pack expansion shall have the same number of arguments specified. An appearance of a name of a pack that is not expanded is ill-formed.

[Example 6:

```
template<typename...> struct Tuple {};
template<typename T1, typename T2> struct Pair {};

template<class ... Args1> struct zip {
 template<class ... Args2> struct with {
 typedef Tuple<Pair<Args1, Args2> ... > type;
 };
};

typedef zip<short, int>::with<unsigned short, unsigned>::type T1;
// T1 is Tuple<Pair<short, unsigned short>, Pair<int, unsigned>>
typedef zip<short>::with<unsigned short, unsigned>::type T2;
// error: different number of arguments specified for Args1 and Args2

template<class ... Args>
void g(Args ... args) { // OK, Args is expanded by the function parameter pack args
 f(const_cast<const Args*>(&args)...); // OK, “Args” and “args” are expanded
 f(5 ...); // error: pattern does not contain any packs
 f(args); // error: pack “args” is not expanded
 f(h(args ...) + args ...); // OK, first “args” expanded within h,
 // second “args” expanded within f
}
```

— end example]

- 9 The instantiation of a pack expansion considers items  $E_1, E_2, \dots, E_N$ , where  $N$  is the number of elements in the pack expansion parameters. Each  $E_i$  is generated by instantiating the pattern and replacing each pack expansion parameter with its  $i^{\text{th}}$  element. Such an element, in the context of the instantiation, is interpreted as follows:

- (9.1) — if the pack is a template parameter pack, the element is
  - (9.1.1) — a *typedef-name* for a type template parameter pack,
  - (9.1.2) — an *id-expression* for a constant template parameter pack, or
  - (9.1.3) — a *template-name* for a template template parameter pack
 designating the  $i^{\text{th}}$  corresponding type, constant, or template template argument;
- (9.2) — if the pack is a function parameter pack, the element is an *id-expression* designating the  $i^{\text{th}}$  function parameter that resulted from instantiation of the function parameter pack declaration;

- (9.3) — if the pack is an *init-capture* pack, the element is an *id-expression* designating the variable introduced by the  $i^{\text{th}}$  *init-capture* that resulted from instantiation of the *init-capture* pack declaration; otherwise
- (9.4) — if the pack is a structured binding pack, the element is an *id-expression* designating the  $i^{\text{th}}$  structured binding in the pack that resulted from the structured binding declaration.

When  $N$  is zero, the instantiation of a pack expansion does not alter the syntactic interpretation of the enclosing construct, even in cases where omitting the pack expansion entirely would otherwise be ill-formed or would result in an ambiguity in the grammar.

- 10 The instantiation of a **sizeof**... expression (7.6.2.5) produces an integral constant with value  $N$ .
- 11 When instantiating a *pack-index-expression*  $P$ , let  $K$  be the index of  $P$ . The instantiation of  $P$  is the *id-expression*  $E_K$ .
- 12 When instantiating a *pack-index-specifier*  $P$ , let  $K$  be the index of  $P$ . The instantiation of  $P$  is the *typedef-name*  $E_K$ .
- 13 When instantiating a *pack-index-template-name*  $P$ , let  $K$  be the index of  $P$ . The instantiation of  $P$  is the *simple-template-name*  $E_K$ .
- 14 The instantiation of an *alignment-specifier* with an ellipsis produces  $E_1 E_2 \dots E_N$ .
- 15 The instantiation of a *fold-expression* (7.5.7) produces:
- (15.1) — ( ( ( $E_1$  *op*  $E_2$ ) *op*  $\dots$ ) *op*  $E_N$  ) for a unary left fold,
- (15.2) — (  $E_1$  *op* ( $\dots$  *op* ( $E_{N-1}$  *op*  $E_N$ )) ) for a unary right fold,
- (15.3) — ( ((( $E$  *op*  $E_1$ ) *op*  $E_2$ ) *op*  $\dots$ ) *op*  $E_N$  ) for a binary left fold, and
- (15.4) — (  $E_1$  *op* ( $\dots$  *op* ( $E_{N-1}$  *op* ( $E_N$  *op*  $E$ ))) ) for a binary right fold.

In each case, *op* is the *fold-operator*. For a binary fold,  $E$  is generated by instantiating the *cast-expression* that did not contain an unexpanded pack.

[Example 7:

```
template<typename ...Args>
bool all(Args ...args) { return (... && args); }

bool b = all(true, true, true, false);
```

Within the instantiation of **all**, the returned expression expands to ((**true** && **true**) && **true**) && **false**, which evaluates to **false**. — end example]

If  $N$  is zero for a unary fold, the value of the expression is shown in Table 20; if the operator is not listed in Table 20, the instantiation is ill-formed.

Table 20 — Value of folding empty sequences [tab:temp.fold.empty]

| Operator | Value when pack is empty |
|----------|--------------------------|
| &&       | true                     |
|          | false                    |
| ,        | void()                   |

- 16 A fold expanded constraint is not instantiated (13.5.2.5).
- 17 The instantiation of any other pack expansion produces a list of elements  $E_1, E_2, \dots, E_N$ .

[Note 1: The variety of list varies with the context: *expression-list*, *base-specifier-list*, *template-argument-list*, etc. — end note]

When  $N$  is zero, the instantiation of the expansion produces an empty list.

[Example 8:

```
template<class... T> struct X : T... { };
template<class... T> void f(T... values) {
 X<T...> x(values...);
}
```

```
template void f<>(); // OK, X<> has no base classes
 // x is a variable of type X<> that is value-initialized
```

— end example]

### 13.7.5 Friends

[temp.friend]

- <sup>1</sup> A friend of a class or class template can be a function template or class template, a specialization of a function template or class template, or a non-template function or class.

[Example 1:

```
template<class T> class task;
template<class T> task<T>* preempt(task<T>*);

template<class T> class task {
 friend void next_time();
 friend void process(task<T>*);
 friend task<T>* preempt<T>(task<T>*);
 template<class C> friend int func(C);

 friend class task<int>;
 template<class P> friend class frd;
};
```

Here, each specialization of the `task` class template has the function `next_time` as a friend; because `process` does not have explicit *template-arguments*, each specialization of the `task` class template has an appropriately typed function `process` as a friend, and this friend is not a function template specialization; because the friend `preempt` has an explicit *template-argument* `T`, each specialization of the `task` class template has the appropriate specialization of the function template `preempt` as a friend; and each specialization of the `task` class template has all specializations of the function template `func` as friends. Similarly, each specialization of the `task` class template has the class template specialization `task<int>` as a friend, and has all specializations of the class template `frd` as friends. — end example]

- <sup>2</sup> Friend classes, class templates, functions, or function templates can be declared within a class template. When a template is instantiated, its friend declarations are found by name lookup as if the specialization had been explicitly declared at its point of instantiation.

[Note 1: They can introduce entities that belong to an enclosing namespace scope (9.3.4), in which case they are attached to the same module as the class template (10.1). — end note]

- <sup>3</sup> A friend template may be declared within a class or class template. A friend function template may be defined within a class or class template, but a friend class template may not be defined in a class or class template. In these cases, all specializations of the friend class or friend function template are friends of the class or class template granting friendship.

[Example 2:

```
class A {
 template<class T> friend class B; // OK
 template<class T> friend void f(T) { /* ... */ } // OK
};
```

— end example]

- <sup>4</sup> A template friend declaration specifies that all specializations of that template, whether they are implicitly instantiated (13.9.2), partially specialized (13.7.6) or explicitly specialized (13.9.4), are friends of the class containing the template friend declaration.

[Example 3:

```
class X {
 template<class T> friend struct A;
 class Y { };
};

template<class T> struct A { X::Y ab; }; // OK
template<class T> struct A<T*> { X::Y ab; }; // OK
```

— end example]

- <sup>5</sup> A template friend declaration may declare a member of a dependent type to be a friend. The friend declaration shall declare a function or specify a type with an *elaborated-type-specifier*, in either case with a

*nested-name-specifier* ending with a *simple-template-id*, *C*, whose *template-name* names a class template. The template parameters of the template friend declaration shall be deducible from *C* (13.10.3.6). In this case, a member of a specialization *S* of the class template is a friend of the class granting friendship if deduction of the template parameters of *C* from *S* succeeds, and substituting the deduced template arguments into the friend declaration produces a declaration that corresponds to the member of the specialization.

[Example 4:

```
template<class T> struct A {
 struct B { };
 void f();
 struct D {
 void g();
 };
 T h();
 template<T U> T i();
};

template<> struct A<int> {
 struct B { };
 int f();
 struct D {
 void g();
 };
 template<int U> int i();
};

template<> struct A<float*> {
 int *h();
};

class C {
 template<class T> friend struct A<T>::B; // grants friendship to A<int>::B even though
 // it is not a specialization of A<T>::B
 template<class T> friend void A<T>::f(); // does not grant friendship to A<int>::f()
 // because its return type does not match
 template<class T> friend void A<T>::D::g(); // error: A<T>::D does not end with a simple-template-id
 template<class T> friend int *A<T*>::h(); // grants friendship to A<int*>::h() and A<float*>::h()
 template<class T> template<T U> // grants friendship to instantiations of A<T>::i() and
 friend T A<T>::i(); // to A<int>::i(), and thereby to all specializations
 // of those function templates
};
```

— end example]

- <sup>6</sup> A friend template shall not be declared in a local class.
- <sup>7</sup> Friend declarations shall not declare partial specializations.

[Example 5:

```
template<class T> class A { };
class X {
 template<class T> friend class A<T*>; // error
};
```

— end example]

- <sup>8</sup> When a friend declaration refers to a specialization of a function template, the function parameter declarations shall not include default arguments, nor shall the `inline`, `constexpr`, or `consteval` specifiers be used in such a declaration.
- <sup>9</sup> A non-template friend declaration with a *requires-clause* shall be a definition. A friend function template with a constraint that depends on a template parameter from an enclosing template shall be a definition. Such a constrained friend function or function template declaration does not declare the same function or function template as a declaration in any other scope.

## 13.7.6 Partial specialization

[temp.spec.partial]

### 13.7.6.1 General

[temp.spec.partial.general]

- <sup>1</sup> A partial specialization of a template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization

(13.7.6.2). A declaration of the primary template shall precede any partial specialization of that template. A partial specialization shall be reachable from any use of a template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation; no diagnostic is required (G.10.6).

- <sup>2</sup> Two partial specialization declarations declare the same entity if they are partial specializations of the same template and have equivalent *template-heads* and template argument lists (13.7.7.2). Each partial specialization is a distinct template.

<sup>3</sup> [Example 1:

```
template<class T1, class T2, int I> class A { };
template<class T, int I> class A<T, T*, I> { };
template<class T1, class T2, int I> class A<T1*, T2, I> { };
template<class T> class A<int, T*, 5> { };
template<class T1, class T2, int I> class A<T1, T2*, I> { };
```

The first declaration declares the primary (unspecialized) class template. The second and subsequent declarations declare partial specializations of the primary template. — end example]

- <sup>4</sup> A partial specialization may be constrained (13.5).

[Example 2:

```
template<typename T> concept C = true;

template<typename T> struct X { };
template<typename T> struct X<T*> { }; // #1
template<C T> struct X<T> { }; // #2
```

Both partial specializations are more specialized than the primary template. #1 is more specialized because the deduction of its template arguments from the template argument list of the class template specialization succeeds, while the reverse does not. #2 is more specialized because the template arguments are equivalent, but the partial specialization is more constrained (13.5.5). — end example]

- <sup>5</sup> The template argument list of a partial specialization is the *template-argument-list* following the name of the template.
- <sup>6</sup> A partial specialization may be declared in any scope in which the corresponding primary template may be defined (9.3.4, 11.4, 13.7.3).

[Example 3:

```
template<class T> struct A {
 struct C {
 template<class T2> struct B { };
 template<class T2> struct B<T2**> { }; // partial specialization #1
 };
};

// partial specialization of A<T>::C::B<T2>
template<class T> template<class T2>
 struct A<T>::C::B<T2*> { }; // #2

A<short>::C::B<int*> absip; // uses partial specialization #2
```

— end example]

- <sup>7</sup> Partial specialization declarations do not introduce a name. Instead, when the primary template name is used, any reachable partial specializations of the primary template are also considered.

[Note 1: One consequence is that a *using-declaration* which refers to a class template does not restrict the set of partial specializations that are found through the *using-declaration*. — end note]

[Example 4:

```
namespace N {
 template<class T1, class T2> class A { }; // primary template
}

using N::A; // refers to the primary template
```



```

namespace N {
 template<class T> class A<T, T*> { }; // partial specialization
}

A<int,int*> a; // uses the partial specialization, which is found through the using-declaration
 // which refers to the primary template

```

— end example]

- <sup>8</sup> A constant template argument is non-specialized if it is the name of a constant template parameter. All other constant template arguments are specialized.
- <sup>9</sup> Within the argument list of a partial specialization, the following restrictions apply:
- (9.1) — The type of a template parameter corresponding to a specialized constant template argument shall not be dependent on a parameter of the partial specialization.

[Example 5:

```

template <class T, T t> struct C {};
template <class T> struct C<T, 1>; // error

template< int X, int (*array_ptr)[X] > class A {};
int array[5];
template< int X > class A<X,&array> { }; // error

```

— end example]

- (9.2) — The partial specialization shall be more specialized than the primary template (13.7.6.3).
- (9.3) — The template parameter list of a partial specialization shall not contain default template argument values.<sup>107</sup>
- (9.4) — An argument shall not contain an unexpanded pack. If an argument is a pack expansion (13.7.4), it shall be the last argument in the template argument list.

- <sup>10</sup> The usual access checking rules do not apply to non-dependent names used to specify template arguments of the *simple-template-id* of the partial specialization.

[Note 2: The template arguments can be private types or objects that would normally not be accessible. Dependent names cannot be checked when declaring the partial specialization, but will be checked when substituting into the partial specialization. — end note]

### 13.7.6.2 Matching of partial specializations

[temp.spec.partial.match]

- <sup>1</sup> When a template is used in a context that requires an instantiation of the template, it is necessary to determine whether the instantiation is to be generated using the primary template or one of the partial specializations. This is done by matching the template arguments of the template specialization with the template argument lists of the partial specializations.
- (1.1) — If exactly one matching partial specialization is found, the instantiation is generated from that partial specialization.
- (1.2) — If more than one matching partial specialization is found, the partial order rules (13.7.6.3) are used to determine whether one of the partial specializations is more specialized than the others. If such a partial specialization exists, the instantiation is generated from that partial specialization; otherwise, the use of the template is ambiguous and the program is ill-formed.
- (1.3) — If no matches are found, the instantiation is generated from the primary template.
- <sup>2</sup> A partial specialization matches a given actual template argument list if the template arguments of the partial specialization can be deduced from the actual template argument list (13.10.3), and the deduced template arguments satisfy the associated constraints of the partial specialization, if any (13.5.3).

[Example 1:

```

template<class T1, class T2, int I> class A { }; // #1
template<class T, int I> class A<T, T*, I> { }; // #2
template<class T1, class T2, int I> class A<T1*, T2, I> { }; // #3
template<class T> class A<int, T*, 5> { }; // #4
template<class T1, class T2, int I> class A<T1, T2*, I> { }; // #5

```

<sup>107)</sup> There is no context in which they would be used.

```

A<int, int, 1> a1; // uses #1
A<int, int*, 1> a2; // uses #2, T is int, I is 1
A<int, char*, 5> a3; // uses #4, T is char
A<int, char*, 1> a4; // uses #5, T1 is int, T2 is char, I is 1
A<int*, int*, 2> a5; // ambiguous: matches #3 and #5

```

— end example]

[Example 2:

```

template<typename T> concept C = requires (T t) { t.f(); };

template<typename T> struct S { }; // #1
template<C T> struct S<T> { }; // #2

struct Arg { void f(); };

S<int> s1; // uses #1; the constraints of #2 are not satisfied
S<Arg> s2; // uses #2; both constraints are satisfied but #2 is more specialized

```

— end example]

- <sup>3</sup> If the template arguments of a partial specialization cannot be deduced because of the structure of its *template-parameter-list* and the *template-id*, the program is ill-formed.

[Example 3:

```

template <int I, int J> struct A {};
template <int I> struct A<I+5, I*2> {}; // error

template <int I> struct A<I, I> {}; // OK

template <int I, int J, int K> struct B {};
template <int I> struct B<I, I*2, 2> {}; // OK

```

— end example]

- <sup>4</sup> In a name that refers to a specialization of a class or variable template (e.g., **A<int, int, 1>**), the argument list shall match the template parameter list of the primary template. The template arguments of a partial specialization are deduced from the arguments of the primary template.

### 13.7.6.3 Partial ordering of partial specializations

[temp.spec.partial.order]

- <sup>1</sup> For two partial specializations, the first is *more specialized* than the second if, given the following rewrite to two function templates, the first function template is more specialized than the second according to the ordering rules for function templates (13.7.7.3):

- (1.1) — Each of the two function templates has the same template parameters and associated constraints (13.5.3) as the corresponding partial specialization.
- (1.2) — Each function template has a single function parameter whose type is a class template specialization where the template arguments are the corresponding template parameters from the function template for each template argument in the *template-argument-list* of the *simple-template-id* of the partial specialization.

- <sup>2</sup> [Example 1:

```

template<int I, int J, class T> class X { };
template<int I, int J> class X<I, J, int> { }; // #1
template<int I> class X<I, I, int> { }; // #2

template<int I0, int J0> void f(X<I0, J0, int>); // A
template<int I0> void f(X<I0, I0, int>); // B

template <auto v> class Y { };
template <auto* p> class Y<p> { }; // #3
template <auto** pp> class Y<pp> { }; // #4

template <auto* p0> void g(Y<p0>); // C
template <auto** pp0> void g(Y<pp0>); // D

```

According to the ordering rules for function templates, the function template *B* is more specialized than the function template *A* and the function template *D* is more specialized than the function template *C*. Therefore, the partial specialization #2 is more specialized than the partial specialization #1 and the partial specialization #4 is more specialized than the partial specialization #3. — *end example*]

[*Example 2:*

```
template<typename T> concept C = requires (T t) { t.f(); };
template<typename T> concept D = C<T> && requires (T t) { t.f(); };

template<typename T> class S { };
template<C T> class S<T> { }; // #1
template<D T> class S<T> { }; // #2

template<C T> void f(S<T>); // A
template<D T> void f(S<T>); // B
```

The partial specialization #2 is more specialized than #1 because B is more specialized than A. — *end example*]

#### 13.7.6.4 Members of class template partial specializations [temp.spec.partial.member]

- <sup>1</sup> The members of the class template partial specialization are unrelated to the members of the primary template. Class template partial specialization members that are used in a way that requires a definition shall be defined; the definitions of members of the primary template are never used as definitions for members of a class template partial specialization. An explicit specialization of a member of a class template partial specialization is declared in the same way as an explicit specialization of a member of the primary template.

[*Example 1:*

```
// primary class template
template<class T, int I> struct A {
 void f();
};

// member of primary class template
template<class T, int I> void A<T,I>::f() { }

// class template partial specialization
template<class T> struct A<T,2> {
 void f();
 void g();
 void h();
};

// member of class template partial specialization
template<class T> void A<T,2>::g() { }

// explicit specialization
template<> void A<char,2>::h() { }

int main() {
 A<char,0> a0;
 A<char,2> a2;
 a0.f(); // OK, uses definition of primary template's member
 a2.g(); // OK, uses definition of partial specialization's member
 a2.h(); // OK, uses definition of explicit specialization's member
 a2.f(); // error: no definition of f for A<T,2>; the primary template is not used here
}
```

— *end example*]

- <sup>2</sup> If a member template of a class template is partially specialized, the member template partial specializations are member templates of the enclosing class template; if the enclosing class template is instantiated (13.9.2, 13.9.3), a declaration for every member template partial specialization is also instantiated as part of creating the members of the class template specialization. If the primary member template is explicitly specialized for a given (implicit) specialization of the enclosing class template, the partial specializations of the member template are ignored for this specialization of the enclosing class template. If a partial specialization of the member template is explicitly specialized for a given (implicit) specialization of the enclosing class template,

the primary member template and its other partial specializations are still considered for this specialization of the enclosing class template.

[*Example 2:*

```
template<class T> struct A {
 template<class T2> struct B {};
 template<class T2> struct B<T2*> {};
};

template<> template<class T2> struct A<short>::B {};

A<char>::B<int*> abcip;
A<short>::B<int*> absip;
A<char>::B<int> abci;
```

// #1  
// #2  
  
// #3  
  
// uses #2  
// uses #3  
// uses #1

— *end example*]

### 13.7.7 Function templates

[temp.fct]

#### 13.7.7.1 General

[temp.fct.general]

- <sup>1</sup> A function template defines an unbounded set of related functions.

[*Example 1:* A family of sort functions can be declared like this:

```
template<class T> class Array { };
template<class T> void sort(Array<T>&);
```

— *end example*]

- <sup>2</sup> [Note 1: A function template can have the same name as other function templates and non-template functions (9.3.4.6) in the same scope. — *end note*]

A non-template function is not related to a function template (i.e., it is never considered to be a specialization), even if it has the same name and type as a potentially generated function template specialization.<sup>108</sup>

#### 13.7.7.2 Function template overloading

[temp.over.link]

- <sup>1</sup> It is possible to overload function templates so that two different function template specializations have the same type.

[*Example 1:*

|                                                                                                                                |                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <pre>// translation unit 1: template&lt;class T&gt; void f(T*); void g(int* p) {     f(p); // calls f&lt;int&gt;(int*) }</pre> | <pre>// translation unit 2: template&lt;class T&gt; void f(T); void h(int* p) {     f(p); // calls f&lt;int*&gt;(int*) }</pre> |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|

— *end example*]

- <sup>2</sup> Such specializations are distinct functions and do not violate the one-definition rule (6.3).
- <sup>3</sup> The signature of a function template is defined in Clause 3. The names of the template parameters are significant only for establishing the relationship between the template parameters and the rest of the signature.

[Note 1: Two distinct function templates can have identical function return types and function parameter lists, even if overload resolution alone cannot distinguish them.

```
template<class T> void f();
template<int I> void f(); // OK, overloads the first template
 // distinguishable with an explicit template argument list
```

— *end note*]

- <sup>4</sup> When an expression that references a template parameter is used in the function parameter list or the return type in the declaration of a function template, the expression that references the template parameter is part of the signature of the function template. This is necessary to permit a declaration of a function template in one translation unit to be linked with another declaration of the function template in another translation

<sup>108)</sup> That is, declarations of non-template functions do not merely guide overload resolution of function template specializations with the same name. If such a non-template function is odr-used (6.3) in a program, it must be defined; it will not be implicitly instantiated using the function template definition.

unit and, conversely, to ensure that function templates that are intended to be distinct are not linked with one another.

[Example 2:

```
template <int I, int J> A<I+J> f(A<I>, A<J>); // #1
template <int K, int L> A<K+L> f(A<K>, A<L>); // same as #1
template <int I, int J> A<I-J> f(A<I>, A<J>); // different from #1
```

— end example]

[Note 2: Most expressions that use template parameters use constant template parameters, but it is possible for an expression to reference a type parameter. For example, a template type parameter can be used in the `sizeof` operator. — end note]

- <sup>5</sup> Two expressions involving template parameters are considered *equivalent* if two function definitions containing the expressions would satisfy the one-definition rule (6.3), except that the tokens used to name the template parameters may differ as long as a token used to name a template parameter in one expression is replaced by another token that names the same template parameter in the other expression. Two unevaluated operands that do not involve template parameters are considered equivalent if two function definitions containing the expressions would satisfy the one-definition rule, except that the tokens used to name types and declarations may differ as long as they name the same entities, and the tokens used to form concept-ids (13.3) may differ as long as the two *template-ids* are the same (13.6).

[Note 3: For instance, `A<42>` and `A<40+2>` name the same type. — end note]

Two *lambda-expressions* are never considered equivalent.

[Note 4: The intent is to avoid *lambda-expressions* appearing in the signature of a function template with external linkage. — end note]

For determining whether two dependent names (13.8.3) are equivalent, only the name itself is considered, not the result of name lookup.

[Note 5: If such a dependent name is unqualified, it is looked up from a first declaration of the function template (13.8.1). — end note]

[Example 3:

```
template <int I, int J> void f(A<I+J>); // #1
template <int K, int L> void f(A<K+L>); // same as #1

template <class T> decltype(g(T())) h();
int g(int);
template <class T> decltype(g(T())) h() // redeclaration of h() uses the earlier lookup...
{ return g(T()); } // ... although the lookup here does find g(int)
int i = h<int>(); // template argument substitution fails; g(int)
 // not considered at the first declaration of h()
```

// ill-formed, no diagnostic required: the two expressions are functionally equivalent but not equivalent

```
template <int N> void foo(const char (*s)[[]{}], N)];
template <int N> void foo(const char (*s)[[]{}], N)];
```

// two different declarations because the non-dependent portions are not considered equivalent

```
template <class T> void spam(decltype([]{}) (*s)[sizeof(T)]);
template <class T> void spam(decltype([]{}) (*s)[sizeof(T)]);
```

— end example]

Two potentially evaluated expressions involving template parameters that are not equivalent are *functionally equivalent* if, for any given set of template arguments, the evaluation of the expression results in the same value. Two unevaluated operands that are not equivalent are functionally equivalent if, for any given set of template arguments, the expressions perform the same operations in the same order with the same entities.

[Note 6: For instance, one could have redundant parentheses. — end note]

[Example 4:

```
template<int I> concept C = true;
template<typename T> struct A {
 void f() requires C<42>; // #1
 void f() requires true; // OK, different functions
};
```

— *end example*]

- 6 Two *template-heads* are *equivalent* if their *template-parameter-lists* have the same length, corresponding *template-parameters* are equivalent and are both declared with *type-constraints* that are equivalent if either *template-parameter* is declared with a *type-constraint*, and if either *template-head* has a *requires-clause*, they both have *requires-clauses* and the corresponding *constraint-expressions* are equivalent. Two *template-parameters* are *equivalent* under the following conditions:

- (6.1) — they declare template parameters of the same kind,
- (6.2) — if either declares a template parameter pack, they both do,
- (6.3) — if they declare constant template parameters, they have equivalent types ignoring the use of *type-constraints* for placeholder types, and
- (6.4) — if they declare template template parameters, their *template-heads* are equivalent.

When determining whether types or *type-constraints* are equivalent, the rules above are used to compare expressions involving template parameters. Two *template-heads* are *functionally equivalent* if they accept and are satisfied by (13.5.2) the same set of template argument lists.

- 7 If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required (G.10.7). Furthermore, if two declarations *A* and *B* of function templates

- (7.1) — introduce the same name,
- (7.2) — have corresponding signatures (6.4.1),
- (7.3) — would declare the same entity, when considering *A* and *B* to correspond in that determination (6.7), and
- (7.4) — accept and are satisfied by the same set of template argument lists,

but do not correspond, the program is ill-formed, no diagnostic required.

- 8 [Note 7: This rule guarantees that equivalent declarations will be linked with one another, while not requiring implementations to use heroic efforts to guarantee that functionally equivalent declarations will be treated as distinct. For example, the last two declarations are functionally equivalent and would cause a program to be ill-formed:

```
// guaranteed to be the same
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+10>);

// guaranteed to be different
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+11>);

// ill-formed, no diagnostic required
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+1+2+3+4>);
```

— *end note*]

### 13.7.7.3 Partial ordering of function templates

[temp.func.order]

- 1 If multiple function templates share a name, the use of that name can be ambiguous because template argument deduction (13.10.3) may identify a specialization for more than one function template. *Partial ordering* of overloaded function template declarations is used in the following contexts to select the function template to which a function template specialization refers:

- (1.1) — during overload resolution for a call to a function template specialization (12.2.4);
- (1.2) — when the address of a function template specialization is taken;
- (1.3) — when a placement deallocation function that is a function template specialization is selected to match a placement allocation function (6.8.6.5.3, 7.6.2.8);
- (1.4) — when a friend function declaration (13.7.5), an explicit instantiation (13.9.3) or an explicit specialization (13.9.4) refers to a function template specialization.

- 2 Partial ordering selects which of two function templates is more specialized than the other by transforming each template in turn (see next paragraph) and performing template argument deduction using the function type. The deduction process determines whether one of the templates is more specialized than the other.

If so, the more specialized template is the one chosen by the partial ordering process. If both deductions succeed, the partial ordering selects the more constrained template (if one exists) as determined below.

- 3 To produce the transformed template, for each type, constant, type template, variable template, or concept template parameter (including template parameter packs (13.7.4) thereof) synthesize a unique type, value, class template, variable template, or concept, respectively, and substitute it for each occurrence of that parameter in the function type of the template. The type replacing a placeholder in the type of the value synthesized for a constant template parameter is a unique synthesized type.
- 4 A synthesized template has the same *template-head* as its corresponding template template parameter.
- 5 Each function template  $M$  that is a member function is considered to have a new first parameter of type  $X(M)$ , described below, inserted in its function parameter list. If exactly one of the function templates was considered by overload resolution via a rewritten candidate (12.2.2.3) with a reversed order of parameters, then the order of the function parameters in its transformed template is reversed. For a function template  $M$  with cv-qualifiers  $cv$  that is a member of a class  $A$ :
  - (5.1) — The type  $X(M)$  is “rvalue reference to  $cv$   $A$ ” if the optional *ref-qualifier* of  $M$  is `&&` or if  $M$  has no *ref-qualifier* and the positionally-corresponding parameter of the other transformed template has rvalue reference type; if this determination depends recursively upon whether  $X(M)$  is an rvalue reference type, it is not considered to have rvalue reference type.
  - (5.2) — Otherwise,  $X(M)$  is “lvalue reference to  $cv$   $A$ ”.

[*Note 1*: This allows a non-static member to be ordered with respect to a non-member function and for the results to be equivalent to the ordering of two equivalent non-members. — *end note*]

[*Example 1*:

```
struct A { };
template<class T> struct B {
 template<class R> int operator*(R&); // #1
};

template<class T, class R> int operator*(T&, R&); // #2

// The declaration of B::operator* is transformed into the equivalent of
// template<class R> int operator*(B<A>&, R&); // #1a

int main() {
 A a;
 B<A> b;
 b * a; // calls #1
}
```

— *end example*]

- 6 Using the transformed function template’s function type, perform type deduction against the other template as described in 13.10.3.5.

[*Example 2*:

```
template<class T> struct A { A(); };

template<class T> void f(T);
template<class T> void f(T*);
template<class T> void f(const T*);

template<class T> void g(T);
template<class T> void g(T&);

template<class T> void h(const T&);
template<class T> void h(A<T>&);

void m() {
 const int* p;
 f(p); // f(const T*) is more specialized than f(T) or f(T*)
 float x;
 g(x); // ambiguous: g(T) or g(T&)
```



```

A<int> z;
h(z); // overload resolution selects h(A<T>&)
const A<int> z2;
h(z2); // h(const T&) is called because h(A<T>&) is not callable
}

```

— end example]

- <sup>7</sup> [Note 2: Since, in a call context, such type deduction considers only parameters for which there are explicit call arguments, some parameters are ignored (namely, function parameter packs, parameters with default arguments, and ellipsis parameters).

[Example 3:

```

template<class T> void f(T); // #1
template<class T> void f(T*, int=1); // #2
template<class T> void g(T); // #3
template<class T> void g(T*, ...); // #4

int main() {
 int* ip;
 f(ip); // calls #2
 g(ip); // calls #4
}

```

— end example]

[Example 4:

```

template<class T, class U> struct A { };

template<class T, class U> void f(U, A<U, T>* p = 0); // #1
template<class T, class U> void f(U, A<U, U>* p = 0); // #2
template<class T> void g(T, T = T()); // #3
template<class T, class... U> void g(T, U ...); // #4

void h() {
 f<int>(42, (A<int, int>*)0); // calls #2
 f<int>(42); // error: ambiguous
 g(42); // error: ambiguous
}

```

— end example]

[Example 5:

```

template<class T, class... U> void f(T, U...); // #1
template<class T> void f(T); // #2
template<class T, class... U> void g(T*, U...); // #3
template<class T> void g(T); // #4

void h(int i) {
 f(&i); // OK, calls #2
 g(&i); // OK, calls #3
}

```

— end example]

— end note]

- <sup>8</sup> If deduction against the other template succeeds for both transformed templates, constraints can be considered as follows:
- (8.1) — If their *template-parameter-lists* (possibly including *template-parameters* invented for an abbreviated function template (9.3.4.6)) or function parameter lists differ in length, neither template is more specialized than the other.
  - (8.2) — Otherwise:
    - (8.2.1) — If exactly one of the templates was considered by overload resolution via a rewritten candidate with reversed order of parameters:
      - (8.2.1.1) — If, for either template, some of the template parameters are not deducible from their function parameters, neither template is more specialized than the other.



- (8.2.1.2) — If there is either no reordering or more than one reordering of the associated *template-parameter-list* such that
  - (8.2.1.2) — the corresponding *template-parameters* of the *template-parameter-lists* are equivalent and
  - (8.2.1.2) — the function parameters that positionally correspond between the two templates are of the same type,
 neither template is more specialized than the other.
- (8.2.2) — Otherwise, if the corresponding *template-parameters* of the *template-parameter-lists* are not equivalent (13.7.7.2) or if the function parameters that positionally correspond between the two templates are not of the same type, neither template is more specialized than the other.
- (8.3) — Otherwise, if the context in which the partial ordering is done is that of a call to a conversion function and the return types of the templates are not the same, then neither template is more specialized than the other.
- (8.4) — Otherwise, if one template is more constrained than the other (13.5.5), the more constrained template is more specialized than the other.
- (8.5) — Otherwise, neither template is more specialized than the other.

[Example 6:

```
template <typename> constexpr bool True = true;
template <typename T> concept C = True<T>;

void f(C auto &, auto &) = delete;
template <C Q> void f(Q &, C auto &);

void g(struct A *ap, struct B *bp) {
 f(*ap, *bp); // OK, can use different methods to produce template parameters
}

template <typename T, typename U> struct X {};

template <typename T, C U, typename V> bool operator==(X<T, U>, V) = delete;
template <C T, C U, C V> bool operator==(T, X<U, V>);

void h() {
 X<void *, int>{} == 0; // OK, correspondence of [T, U, V] and [U, V, T]
}
```

— end example]

### 13.7.8 Alias templates

[temp.alias]

- <sup>1</sup> A *template-declaration* in which the *declaration* is an *alias-declaration* (9.1) declares the *identifier* to be an *alias template*. An alias template is a name for a family of types. The name of the alias template is a *simple-template-name*.

<sup>2</sup> A

- (2.1) — *template-id* that is not the operand of a *reflect-expression* or
- (2.2) — *splice-specialization-specifier*

that designates the specialization of an alias template is equivalent to the associated type obtained by substitution of its *template-arguments* for the *template-parameters* in the *defining-type-id* of the alias template. Any other *template-id* that names a specialization of an alias template is a *typedef-name* for a type alias; such a *template-id* is ill-formed if forming the associated type results in substitution failure.

[Note 1: The alias template name is not deduced from such a type (13.10.3.6). — end note]

[Example 1:

```
template<class T> struct Alloc { /* ... */ };
template<class T> using Vec = vector<T, Alloc<T>>;
Vec<int> v; // same as vector<int, Alloc<int>> v;
```

```

template<class T>
void process(Vec<T>& v)
{ /* ... */ }

template<class T>
void process(vector<T, Alloc<T>>& w)
{ /* ... */ } // error: redefinition

template<template<class> class TT>
void f(TT<int>);

f(v); // error: Vec not deduced

template<template<class, class> class TT>
void g(TT<int, Alloc<int>>);
g(v); // OK, TT = vector
— end example]

```

- <sup>3</sup> However, if the *template-id* is dependent, subsequent template argument substitution still applies to the *template-id*.

[Example 2:

```

template<typename...> using void_t = void;
template<typename T> void_t<typename T::foo> f();
f<int>(); // error: int does not have a nested type foo
— end example]

```

- <sup>4</sup> The *defining-type-id* in an alias template declaration shall not refer to the alias template being declared. The type produced by an alias template specialization shall not directly or indirectly make use of that specialization.

[Example 3:

```

template <class T> struct A;
template <class T> using B = typename A<T>::U;
template <class T> struct A {
 typedef B<T> U;
};
B<short> b; // error: instantiation of B<short> uses own type via A<short>::U
— end example]

```

- <sup>5</sup> The type of a *lambda-expression* appearing in an alias template declaration is different between instantiations of that template, even when the *lambda-expression* is not dependent.

[Example 4:

```

template <class T>
using A = decltype([] { }); // A<int> and A<char> refer to different closure types
— end example]

```

### 13.7.9 Concept definitions

[temp.concept]

- <sup>1</sup> A *concept* is a template that defines constraints on its template arguments.

*concept-definition*:

concept *concept-name* *attribute-specifier-seq<sub>opt</sub>* = *constraint-expression* ;

*concept-name*:

*identifier*

- <sup>2</sup> A *concept-definition* declares a concept. Its *identifier* becomes a *concept-name* referring to that concept within its scope. The optional *attribute-specifier-seq* appertains to the concept.

[Example 1:

```

template<typename T>
concept C = requires(T x) {
 { x == x } -> std::convertible_to<bool>;
};

```

```
template<typename T>
 requires C<T> // C constrains f1(T) in constraint-expression
T f1(T x) { return x; }
```

```
template<C T> // C, as a type-constraint, constrains f2(T)
T f2(T x) { return x; }
```

— end example]

<sup>3</sup> A *concept-definition* shall inhabit a namespace scope (6.4.6).

<sup>4</sup> A concept shall not have associated constraints (13.5.3).

<sup>5</sup> A concept is not instantiated (13.9).

[Note 1: A concept-id (13.3) is evaluated as an expression. A concept cannot be explicitly instantiated (13.9.3), explicitly specialized (13.9.4), or partially specialized (13.7.6). — end note]

<sup>6</sup> The *constraint-expression* of a *concept-definition* is an unevaluated operand (7.2.3).

<sup>7</sup> The first declared template parameter of a concept definition is its *prototype parameter*. A *type concept* is a concept whose prototype parameter is a type template parameter.

## 13.8 Name resolution

[temp.res]

### 13.8.1 General

[temp.res.general]

<sup>1</sup> A name that appears in a declaration *D* of a template *T* is looked up from where it appears in an unspecified declaration of *T* that either is *D* itself or is reachable from *D* and from which no other declaration of *T* that contains the usage of the name is reachable. If the name is dependent (as specified in 13.8.3), it is looked up for each specialization (after substitution) because the lookup depends on a template parameter.

[Note 1: Some dependent names are also looked up during parsing to determine that they are dependent or to interpret following < tokens. Uses of other names might be type-dependent or value-dependent (13.8.3.3, 13.8.3.4). A *using-declarator* is never dependent in a specialization and is therefore replaced during lookup for that specialization (6.5). — end note]

[Example 1:

```
struct A { operator int(); };
template<class B, class T>
struct D : B {
 T get() { return operator T(); } // conversion-function-id is dependent
};
int f(D<A, int> d) { return d.get(); } // OK, lookup finds A::operator int
```

— end example]

[Example 2:

```
void f(char);

template<class T> void g(T t) {
 f(1); // f(char)
 f(T(1)); // dependent
 f(t); // dependent
 dd++; // not dependent; error: declaration for dd not found
}

enum E { e };
void f(E);

double dd;
void h() {
 g(e); // will cause one call of f(char) followed by two calls of f(E)
 g('a'); // will cause three calls of f(char)
}
```

— end example]

[Example 3:

```
struct A {
 struct B { /* ... */ };
};
```

```

 int a;
 int Y;
};

int a;

template<class T> struct Y : T {
 struct B { /* ... */ };
 B b; // The B defined in Y
 void f(int i) { a = i; } // ::a
 Y* p; // Y<T>
};

Y<A> ya;

```

The members `A::B`, `A::a`, and `A::Y` of the template argument `A` do not affect the binding of names in `Y<A>`. — *end example*

- <sup>2</sup> If the validity or meaning of the program would be changed by considering a default argument or default template argument introduced in a declaration that is reachable from the point of instantiation of a specialization (13.8.4.1) but is not found by lookup for the specialization, the program is ill-formed, no diagnostic required (G.10.8).

*typename-specifier:*

*typename nested-name-specifier identifier*

*typename nested-name-specifier template<sub>opt</sub> simple-template-id*

- <sup>3</sup> The component names of a *typename-specifier* are its *identifier* (if any) and those of its *nested-name-specifier* and *simple-template-id* (if any). A *typename-specifier* denotes the type or class template denoted by the *simple-type-specifier* (9.2.9.3) formed by omitting the keyword `typename`.

[Note 2: The usual qualified name lookup (6.5.5) applies even in the presence of `typename`. — *end note*]

[Example 4:

```

struct A {
 struct X { };
 int X;
};
struct B {
 struct X { };
};
template<class T> void f(T t) {
 typename T::X x;
}
void foo() {
 A a;
 B b;
 f(b); // OK, T::X refers to B::X
 f(a); // error: T::X refers to the data member A::X not the struct A::X
}

```

— *end example*

- <sup>4</sup> A *type-only context* is defined as follows: A qualified or unqualified name is said to be in a type-only context if it is the terminal name of

- (4.1) — a *typename-specifier*, *type-requirement*, *nested-name-specifier*, *elaborated-type-specifier*, *class-or-decltype*, *using-enum-declarator*, or
- (4.2) — a *simple-type-specifier* of a *friend-type-specifier*, or
- (4.3) — a *type-specifier* of a
  - (4.3.1) — *new-type-id*,
  - (4.3.2) — *defining-type-id*,
  - (4.3.3) — *conversion-type-id*,
  - (4.3.4) — *trailing-return-type*,

- (4.3.5) — default argument of a *type-parameter*, or
- (4.3.6) — *type-id* of a `static_cast`, `const_cast`, `reinterpret_cast`, or `dynamic_cast`, or
- (4.4) — a *decl-specifier* of the *decl-specifier-seq* of a
- (4.4.1) — *simple-declaration* or *function-definition* in namespace scope,
- (4.4.2) — *member-declaration*,
- (4.4.3) — *parameter-declaration* in a *member-declaration*,<sup>109</sup> unless that *parameter-declaration* appears in a default argument,
- (4.4.4) — *parameter-declaration* in a *declarator* of a function or function template declaration whose *declarator-id* is qualified, unless that *parameter-declaration* appears in a default argument,
- (4.4.5) — *parameter-declaration* in a *lambda-declarator* or *requirement-parameter-list*, unless that *parameter-declaration* appears in a default argument, or
- (4.4.6) — *parameter-declaration* of a *template-parameter* (which necessarily declares a constant template parameter).

A *splice-specifier* or *splice-specialization-specifier* (6.6) is said to be in a type-only context if a hypothetical qualified name appearing in the same position would be in a type-only context.

[Example 5:

```
template<class T> T::R f(); // OK, return type of a function declaration at global scope
template<class T> void f(T::R); // ill-formed, no diagnostic required: attempt to declare
 // a void variable template

enum class Enum { A, B, C };
template<class T> struct S {
 using Ptr = PtrTraits<T>::Ptr; // OK, in a defining-type-id
 using Alias = [::~int:]; // OK, in a defining-type-id
 T::R f(T::P p) { // OK, class scope
 return static_cast<T::R>(p); // OK, type-id of a static_cast
 }
 auto g() -> S<T*>::Ptr; // OK, trailing-return-type
 auto h() -> [::~S:]<T*>; // OK, trailing-return-type
 using enum [::~Enum:]; // OK, using-enum-declarator
};
template<typename T> void f() {
 void (*pf)(T::X); // variable pf of type void* initialized with T::X
 void g(T::X); // error: T::X at block scope does not denote a type
 // (attempt to declare a void variable)
}
```

— end example]

- <sup>5</sup> A *qualified-id* whose terminal name is dependent and that is in a type-only context is considered to denote a type. A name that refers to a *using-declarator* whose terminal name is dependent is interpreted as a *typedef-name* if the *using-declarator* uses the keyword `typename`.

[Example 6:

```
template <class T> void f(int i) {
 T::x * i; // expression, not the declaration of a variable i
}

struct Foo {
 typedef int x;
};

struct Bar {
 static int const x = 5;
};

int main() {
 f<Bar>(1); // OK
```

<sup>109</sup>) This includes friend function declarations.

```

 f<Foo>(1); // error: Foo::x is a type
}
— end example]

```

- 6 The validity of a templated entity may be checked prior to any instantiation.

[*Note 3*: Knowing which names are type names allows the syntax of every template to be checked in this way. — *end note*]

The program is ill-formed, no diagnostic required, if

- (6.1) — no valid specialization, ignoring *static\_assert-declarations* that fail (9.1), can be generated for a templated entity or a substatement of a *constexpr* if statement (8.5.2) within a templated entity and the innermost enclosing template is not instantiated, or
- (6.2) — no valid specialization, ignoring *static\_assert-declarations* that fail, can be generated for the *compound-statement* of an *expansion-statement* and there is no instantiation of it, or
- (6.3) — no valid specialization, ignoring *static\_assert-declarations* that fail, can be generated for a default *template-argument* and the default *template-argument* is not used in any instantiation, or
- (6.4) — no specialization of an alias template (13.7.8) is valid and no specialization of the alias template is named in the program, or
- (6.5) — any *constraint-expression* in the program, introduced or otherwise, has (in its normal form) an atomic constraint *A* where no satisfaction check of *A* could be well-formed and no satisfaction check of *A* is performed, or
- (6.6) — every valid specialization of a variadic template requires an empty template parameter pack, or
- (6.7) — a hypothetical instantiation of a templated entity immediately following its definition would be ill-formed due to a construct (other than a *static\_assert-declaration* that fails) that does not depend on a template parameter, or
- (6.8) — the interpretation of such a construct in the hypothetical instantiation is different from the interpretation of the corresponding construct in any actual instantiation of the templated entity.

[*Note 4*: This can happen in situations including the following:

- (6.9) — a type used in a non-dependent name is incomplete at the point at which a template is defined but is complete at the point at which an instantiation is performed, or
- (6.10) — lookup for a name in the template definition found a *using-declaration*, but the lookup in the corresponding scope in the instantiation does not find any declarations because the *using-declaration* was a pack expansion and the corresponding pack is empty, or
- (6.11) — an instantiation uses a default argument or default template argument that had not been defined at the point at which the template was defined, or
- (6.12) — constant expression evaluation (7.7.2) within the template instantiation uses
  - (6.12.1) — the value of a *const* object of integral or unscoped enumeration type or
  - (6.12.2) — the value of a *constexpr* object or
  - (6.12.3) — the value of a reference or
  - (6.12.4) — the definition of a *constexpr* function,
 and that entity was not defined when the template was defined, or
- (6.13) — a class template specialization or variable template specialization that is specified by a non-dependent *simple-template-id* is used by the template, and either it is instantiated from a partial specialization that was not defined when the template was defined or it names an explicit specialization that was not declared when the template was defined.

— *end note*]

[*Note 5*: If a template is instantiated, errors will be diagnosed according to the other rules in this document. Exactly when these errors are diagnosed is a quality of implementation issue. — *end note*]

[*Example 7*:

```

int j;
template<class T> class X {
 void f(T t, int i, char* p) {
 t = i; // diagnosed if X::f is instantiated, and the assignment to t is an error
 }
}

```

```

 p = i; // may be diagnosed even if X::f is not instantiated
 p = j; // may be diagnosed even if X::f is not instantiated
 X<T>::g(t); // OK
 X<T>::h(); // may be diagnosed even if X::f is not instantiated
}
void g(T t) {
 +; // may be diagnosed even if X::g is not instantiated
}
};

template<class... T> struct A {
 void operator++(int, T... t); // error: too many parameters
};
template<class... T> union X : T... { }; // error: union with base class
template<class... T> struct A : T..., T... { }; // error: duplicate base class

```

— end example]

- <sup>7</sup> [Note 6: For purposes of name lookup, default arguments and *noexcept-specifiers* of function templates and default arguments and *noexcept-specifiers* of member functions of class templates are considered definitions (13.7). — end note]

### 13.8.2 Locally declared names

[temp.local]

- <sup>1</sup> Like normal (non-template) classes, class templates have an injected-class-name (11.1). The injected-class-name can be used as a *template-name* or a *type-name*. When it is used with a *template-argument-list*, as a *template-argument* for a type template template parameter, or as the final identifier in the *elaborated-type-specifier* of a friend class template declaration, it is a *template-name* that refers to the class template itself. Otherwise, it is a *type-name* equivalent to the *template-name* followed by the template argument list (13.7.1, 13.4.1) of the class template enclosed in <>.
- <sup>2</sup> When the injected-class-name of a class template specialization or partial specialization is used as a *type-name*, it is equivalent to the *template-name* followed by the *template-arguments* of the class template specialization or partial specialization enclosed in <>.

[Example 1:

```

template<template<class> class T> class A { };
template<class T> class Y;
template<> class Y<int> {
 Y* p; // meaning Y<int>
 Y<char>* q; // meaning Y<char>
 A<Y>* a; // meaning A<::Y>
 class B {
 template<class> friend class Y; // meaning ::Y
 };
};

```

— end example]

- <sup>3</sup> The injected-class-name of a class template or class template specialization can be used as either a *template-name* or a *type-name* wherever it is named.

[Example 2:

```

template <class T> struct Base {
 Base* p;
};

template <class T> struct Derived: public Base<T> {
 typename Derived::Base* p; // meaning Derived::Base<T>
};

template<class T, template<class> class U = T::Base> struct Third { };
Third<Derived<int> > t; // OK, default argument uses injected-class-name as a template

```

— end example]

- <sup>4</sup> A lookup that finds an injected-class-name (6.5.2) can result in an ambiguity in certain cases (for example, if it is found in more than one base class). If all of the injected-class-names that are found refer to specializations

of the same class template, and if the name is used as a *template-name*, the reference refers to the class template itself and not a specialization thereof, and is not ambiguous.

[Example 3:

```
template <class T> struct Base { };
template <class T> struct Derived: Base<int>, Base<char> {
 typename Derived::Base b; // error: ambiguous
 typename Derived::Base<double> d; // OK
};
```

— end example]

- <sup>5</sup> When the normal name of the template (i.e., the name from the enclosing scope, not the injected-class-name) is used, it always refers to the class template itself and not a specialization of the template.

[Example 4:

```
template<class T> class X {
 X* p; // meaning X<T>
 X<T>* p2;
 X<int>* p3;
 ::X* p4; // error: missing template argument list
 // ::X does not refer to the injected-class-name
};
```

— end example]

- <sup>6</sup> The name of a template parameter shall not be bound to any following declaration whose locus is contained by the scope to which the template parameter belongs.

[Example 5:

```
template<class T, int i> class Y {
 int T; // error: template parameter hidden
 void f() {
 char T; // error: template parameter hidden
 }
 friend void T(); // OK, no name bound
};

template<class X> class X; // error: hidden by template parameter
```

— end example]

- <sup>7</sup> Unqualified name lookup considers the template parameter scope of a *template-declaration* immediately after the outermost scope associated with the template declared (even if its parent scope does not contain the *template-parameter-list*).

[Note 1: The scope of a class template, including its non-dependent base classes (13.8.3.2, 6.5.2), is searched before its template parameter scope. — end note]

[Example 6:

```
struct B { };
namespace N {
 typedef void V;
 template<class T> struct A : B {
 typedef void C;
 void f();
 template<class U> void g(U);
 };
}

template<class V> void N::A<V>::f() { // N::V not considered here
 V v; // V is still the template parameter, not N::V
}

template<class B> template<class C> void N::A::g(C) {
 B b; // B is the base class, not the template parameter
 C c; // C is the template parameter, not A's C
}
```



— *end example*]

### 13.8.3 Dependent names

[temp.dep]

#### 13.8.3.1 General

[temp.dep.general]

- <sup>1</sup> Inside a template, some constructs have semantics which may differ from one instantiation to another. Such a construct *depends* on the template parameters. In particular, types and expressions may depend on the type and/or value of template parameters (as determined by the template arguments) and this determines the context for name lookup for certain names. An expression may be *type-dependent* (that is, its type may depend on a template parameter) or *value-dependent* (that is, its value when evaluated as a constant expression (7.7.3) may depend on a template parameter) as described below.
- <sup>2</sup> A *dependent call* is an expression, possibly formed as a non-member candidate for an operator (12.2.2.3), of the form:

*postfix-expression* ( *expression-list*<sub>opt</sub> )

where the *postfix-expression* is an *unqualified-id* and

- (2.1) — any of the expressions in the *expression-list* is a pack expansion (13.7.4), or
- (2.2) — any of the expressions or *braced-init-lists* in the *expression-list* is type-dependent (13.8.3.3), or
- (2.3) — the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameter.

The component name of an *unqualified-id* (7.5.5.2) is dependent if

- (2.4) — it is a *conversion-function-id* whose *conversion-type-id* is dependent, or
- (2.5) — it is **operator=** and the current class is a templated entity, or
- (2.6) — the *unqualified-id* is the *postfix-expression* in a dependent call.

[Note 1: Such names are looked up only at the point of the template instantiation (13.8.4.1) in both the context of the template definition and the context of the point of instantiation (13.8.4.2). — *end note*]

- <sup>3</sup> [Example 1:

```
template<class T> struct X : B<T> {
 typename T::A* pa;
 void f(B<T>* pb) {
 static int i = B<T>::i;
 pb->j++;
 }
};
```

The base class name B<T>, the type name T::A, the names B<T>::i and pb->j explicitly depend on the *template-parameter*. — *end example*]

#### 13.8.3.2 Dependent types

[temp.dep.type]

- <sup>1</sup> A name or *template-id* refers to the *current instantiation* if it is
  - (1.1) — in the definition of a class template, a nested class of a class template, a member of a class template, or a member of a nested class of a class template, the injected-class-name (11.1) of the class template or nested class,
  - (1.2) — in the definition of a primary class template or a member of a primary class template, the name of the class template followed by the template argument list of its *template-head* (13.4) enclosed in <> (or an equivalent template alias specialization),
  - (1.3) — in the definition of a nested class of a class template, the name of the nested class referenced as a member of the current instantiation,
  - (1.4) — in the definition of a class template partial specialization or a member of a class template partial specialization, the name of the class template followed by a template argument list equivalent to that of the partial specialization (13.7.6) enclosed in <> (or an equivalent template alias specialization), or
  - (1.5) — in the definition of a templated function, the name of a local class (11.6).
- <sup>2</sup> A template argument that is equivalent to a template parameter can be used in place of that template parameter in a reference to the current instantiation. A template argument is equivalent to a type template parameter if it denotes the same type. A template argument is equivalent to a constant template parameter if

it is an *identifier* that names a variable that is equivalent to the template parameter. A variable is equivalent to a template parameter if

- (2.1) — it has the same type as the template parameter (ignoring cv-qualification) and
- (2.2) — its initializer consists of a single *identifier* that names the template parameter or, recursively, such a variable.

[Note 1: Using a parenthesized variable name breaks the equivalence. — end note]

[Example 1:

```
template <class T> class A {
 A* p1; // A is the current instantiation
 A<T>* p2; // A<T> is the current instantiation
 A<T*> p3; // A<T*> is not the current instantiation
 ::A<T>* p4; // ::A<T> is the current instantiation
 class B {
 B* p1; // B is the current instantiation
 A<T>::B* p2; // A<T>::B is the current instantiation
 typename A<T*>::B* p3; // A<T*>::B is not the current instantiation
 };
};

template <class T> class A<T*> {
 A<T*>* p1; // A<T*> is the current instantiation
 A<T>* p2; // A<T> is not the current instantiation
};

template <class T1, class T2, int I> struct B {
 B<T1, T2, I>* b1; // refers to the current instantiation
 B<T2, T1, I>* b2; // not the current instantiation
 typedef T1 my_T1;
 static const int my_I = I;
 static const int my_I2 = I+0;
 static const int my_I3 = my_I;
 static const long my_I4 = I;
 static const int my_I5 = (I);
 B<my_T1, T2, my_I>* b3; // refers to the current instantiation
 B<my_T1, T2, my_I2>* b4; // not the current instantiation
 B<my_T1, T2, my_I3>* b5; // refers to the current instantiation
 B<my_T1, T2, my_I4>* b6; // not the current instantiation
 B<my_T1, T2, my_I5>* b7; // not the current instantiation
};
```

— end example]

- <sup>3</sup> A *dependent base class* is a base class that is a dependent type and is not the current instantiation.

[Note 2: A base class can be the current instantiation in the case of a nested class naming an enclosing class as a base.

[Example 2:

```
template<class T> struct A {
 typedef int M;
 struct B {
 typedef void M;
 struct C;
 };
};

template<class T> struct A<T>::B::C : A<T> {
 M m; // OK, A<T>::M
};
```

— end example]

— end note]

- <sup>4</sup> A qualified (6.5.5) or unqualified name is a *member of the current instantiation* if

- (4.1) — its lookup context, if it is a qualified name, is the current instantiation, and

- (4.2) — lookup for it finds any member of a class that is the current instantiation.

[Example 3:

```
template <class T> class A {
 static const int i = 5;
 int n1[i]; // i refers to a member of the current instantiation
 int n2[A::i]; // A::i refers to a member of the current instantiation
 int n3[A<T>::i]; // A<T>::i refers to a member of the current instantiation
 int f();
};

template <class T> int A<T>::f() {
 return i; // i refers to a member of the current instantiation
}
```

— end example]

A qualified or unqualified name names a *dependent member of the current instantiation* if it is a member of the current instantiation that, when looked up, refers to at least one member declaration (including a *using-declarator* whose terminal name is dependent) of a class that is the current instantiation.

- <sup>5</sup> A qualified name (6.5.5) is dependent if

- (5.1) — it is a *conversion-function-id* whose *conversion-type-id* is dependent, or
- (5.2) — its lookup context is dependent and is not the current instantiation, or
- (5.3) — its lookup context is the current instantiation and it is `operator=`,<sup>110</sup> or
- (5.4) — its lookup context is the current instantiation and has at least one dependent base class, and qualified name lookup for the name finds nothing (6.5.5).

[Example 4:

```
struct A {
 using B = int;
 A f();
};
struct C : A {};
template<class T>
void g(T t) {
 decltype(t.A::f())::B i; // error: typename needed to interpret B as a type
}
template void g(C); // ... even though A is ::A here
```

— end example]

- <sup>6</sup> If, for a given set of template arguments, a specialization of a template is instantiated that refers to a member of the current instantiation with a qualified name, the name is looked up in the template instantiation context. If the result of this lookup differs from the result of name lookup in the template definition context, name lookup is ambiguous.

[Example 5:

```
struct A {
 int m;
};

struct B {
 int m;
};

template<typename T>
struct C : A, T {
 int f() { return this->m; } // finds A::m in the template definition context
 int g() { return m; } // finds A::m in the template definition context
};
```

<sup>110)</sup> Every instantiation of a class template declares a different set of assignment operators.

```

template int C::f(); // error: finds both A::m and B::m
template int C::g(); // OK, transformation to class member access syntax
 // does not occur in the template definition context; see 7.5.5.1

```

— end example]

7 An initializer is dependent if any constituent expression (6.10.1) of the initializer is type-dependent. A placeholder type (9.2.9.7.1) is dependent if it designates a type deduced from a dependent initializer.

8 A placeholder for a deduced class type (9.2.9.8) is dependent if

- (8.1) — it has a dependent initializer,
- (8.2) — it refers to a type template template parameter, or
- (8.3) — it refers to an alias template that is a member of the current instantiation and whose *defining-type-id* is dependent after class template argument deduction (12.2.2.9) and substitution (13.7.8).

9 [Example 6:

```

template<class T, class V>
struct S { S(T); };

template<class U>
struct A {
 template<class T> using X = S<T, U>;
 template<class T> using Y = S<T, int>;
 void f() {
 new X(1); // dependent
 new Y(1); // not dependent
 }
};

```

— end example]

10 A type is dependent if it is

- (10.1) — a template parameter,
- (10.2) — denoted by a dependent (qualified) name,
- (10.3) — a nested class or enumeration that is a direct member of a class that is the current instantiation,
- (10.4) — a cv-qualified type where the cv-unqualified type is dependent,
- (10.5) — a compound type constructed from any dependent type,
- (10.6) — an array type whose element type is dependent or whose bound (if any) is value-dependent,
- (10.7) — a function type whose parameters include one or more function parameter packs,
- (10.8) — a function type whose exception specification is value-dependent,
- (10.9) — denoted by a dependent placeholder type,
- (10.10) — denoted by a dependent placeholder for a deduced class type,
- (10.11) — denoted by a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is dependent (13.8.3.7),<sup>111</sup>
- (10.12) — a *pack-index-specifier*,
- (10.13) — denoted by `decltype(expression)`, where *expression* is type-dependent (13.8.3.3), or
- (10.14) — denoted by a *splice-type-specifier* in which either the *splice-specifier* or *splice-specialization-specifier* is dependent (13.8.3.5).

11 [Note 3: Because typedefs do not introduce new types, but instead simply refer to other types, a name that refers to a typedef that is a member of the current instantiation is dependent only if the type referred to is dependent. — end note]

### 13.8.3.3 Type-dependent expressions

[temp.dep.expr]

- 1 Except as described below, an expression is type-dependent if any subexpression is type-dependent.
- 2 **this** is type-dependent if the current class (7.5.3) is dependent (13.8.3.2).

<sup>111)</sup> This includes an injected-class-name (11.1) of a class template used without a *template-argument-list*.

<sup>3</sup> An *id-expression* is type-dependent if it is a *template-id* that is not a concept-id and is dependent; or if its terminal name is

- (3.1) — associated by name lookup with one or more declarations declared with a dependent type,
- (3.2) — associated by name lookup with a constant template parameter declared with a type that contains a placeholder type (9.2.9.7),
- (3.3) — associated by name lookup with a variable declared with a type that contains a placeholder type (9.2.9.7) where the initializer is type-dependent,
- (3.4) — associated by name lookup with one or more declarations of member functions of a class that is the current instantiation declared with a return type that contains a placeholder type,
- (3.5) — associated by name lookup with a structured binding declaration (9.7) whose *brace-or-equal-initializer* is type-dependent,
- (3.6) — associated by name lookup with a structured binding pack,

[Example 1:

```

struct C { };

void g(...); // #1

template <typename T>
void f() {
 C arr[1];
 auto [...e] = arr;
 g(e...); // calls #2
}

void g(C); // #2

int main() {
 f<int>();
}

```

— end example]

- (3.7) — associated by name lookup with an entity captured by copy (7.5.6.3) in a *lambda-expression* that has an explicit object parameter whose type is dependent (9.3.4.6),
- (3.8) — the *identifier* `__func__` (9.6.1), where any enclosing function is a template, a member of a class template, or a generic lambda,
- (3.9) — associated by name lookup with a result binding (9.4.2) of a function whose return type is dependent,
- (3.10) — a *conversion-function-id* that specifies a dependent type,
- (3.11) — a name *N* introduced by the *for-range-declaration* of an *expansion-statement* *S* if the type specified for *N* contains a placeholder type and either
  - (3.11.1) — *S* is not an iterating expansion statement or
  - (3.11.2) — the *expansion-initializer* of *S* is type-dependent, or
- (3.12) — dependent

or if it names a dependent member of the current instantiation that is a static data member of type “array of unknown bound of T” for some T (13.7.2.5). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *nofun-type-id*, *simple-type-specifier*, *typename-specifier*, or *new-type-id* is dependent, even if any subexpression is type-dependent:

```

simple-type-specifier (expression-listopt)
simple-type-specifier braced-init-list
typename-specifier (expression-listopt)
typename-specifier braced-init-list
::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt (nofun-type-id) new-initializeropt
dynamic_cast < type-id > (expression)
static_cast < type-id > (expression)
const_cast < type-id > (expression)
reinterpret_cast < type-id > (expression)
(nofun-type-id) cast-expression

```

- <sup>4</sup> Expressions of the following forms are never type-dependent (because the type of the expression cannot be dependent):

```

literal
sizeof unary-expression
sizeof (nofun-type-id)
sizeof ... (identifier)
alignof (nofun-type-id)
typeid (expression)
typeid (type-id)
::opt delete cast-expression
::opt delete [] cast-expression
throw assignment-expressionopt
noexcept (expression)
requires-expression
reflect-expression

```

[Note 1: For the standard library macro `offsetof`, see 17.2. — end note]

- <sup>5</sup> A class member access expression (7.6.1.5) is type-dependent if
- (5.1) — the terminal name of its *id-expression*, if any, is dependent,
  - (5.2) — its *splice-expression*, if any, is type-dependent, or
  - (5.3) — the expression refers to a member of the current instantiation and the type of the referenced member is dependent.

[Note 2: In an expression of the form `x.y` or `xp->y` the type of the expression is usually the type of the member `y` of the class of `x` (or the class pointed to by `xp`). However, if `x` or `xp` refers to a dependent type that is not the current instantiation, the type of `y` is always dependent. — end note]

- <sup>6</sup> A *braced-init-list* is type-dependent if any element is type-dependent or is a pack expansion.
- <sup>7</sup> A *fold-expression* is type-dependent.
- <sup>8</sup> A *pack-index-expression* is type-dependent if its *id-expression* is type-dependent.
- <sup>9</sup> A *splice-expression* is type-dependent if its *splice-specifier* or *splice-specialization-specifier* is dependent (13.8.3.5).

#### 13.8.3.4 Value-dependent expressions

[temp.dep.constexpr]

- <sup>1</sup> Except as described below, an expression used in a context where a constant expression is required is value-dependent if any subexpression is value-dependent.
- <sup>2</sup> An *id-expression* is value-dependent if
  - (2.1) — it is a concept-id and its *concept-name* is dependent or any of its arguments are dependent (13.8.3.7),
  - (2.2) — it is type-dependent,
  - (2.3) — it is the name of a constant template parameter,
  - (2.4) — it is a name introduced by the *for-range-declaration* of an *expansion-statement* (8.7),
  - (2.5) — it names a static data member that is a dependent member of the current instantiation and is not initialized in a *member-declarator*,
  - (2.6) — it names a static member function that is a dependent member of the current instantiation, or
  - (2.7) — it names a potentially-constant variable (7.7.4) that is initialized with an expression that is value-dependent.

Expressions of the following form are value-dependent if the *unary-expression* or *expression* is type-dependent or the *type-id* or *nofun-type-id* is dependent:

```
sizeof unary-expression
sizeof (nofun-type-id)
typeid (expression)
typeid (type-id)
alignof (nofun-type-id)
```

[Note 1: For the standard library macro `offsetof`, see 17.2. — end note]

- 3 Expressions of the following form are value-dependent if either the *type-id*, *nofun-type-id*, *simple-type-specifier*, or *typename-specifier* is dependent or the *expression* or *cast-expression* is value-dependent or any *expression* in the *expression-list* is value-dependent or any *assignment-expression* in the *braced-init-list* is value-dependent:

```
simple-type-specifier (expression-listopt)
typename-specifier (expression-listopt)
simple-type-specifier braced-init-list
typename-specifier braced-init-list
static_cast < type-id > (expression)
const_cast < type-id > (expression)
reinterpret_cast < type-id > (expression)
dynamic_cast < type-id > (expression)
(nofun-type-id) cast-expression
```

- 4 Expressions of the following form are value-dependent:

```
sizeof ... (identifier)
fold-expression
```

- 5 A *noexcept-expression* (7.6.2.7) is value-dependent if its *expression* involves a template parameter.

- 6 An expression of the form *&qualified-id* where the *qualified-id* names a dependent member of the current instantiation is value-dependent. An expression of the form *&cast-expression* is also value-dependent if evaluating *cast-expression* as a core constant expression (7.7.2) succeeds and the result of the evaluation refers to a templated entity that is an object with static or thread storage duration or a member function.

- 7 A *reflect-expression* is value-dependent if

(7.1) — it is of the form *^^reflection-name* and either lookup for the *reflection-name* finds a declaration that inhabits a scope corresponding to a templated entity or the *reflection-name* is

(7.1.1) — a dependent qualified name,

(7.1.2) — a dependent *namespace-name*, or

(7.1.3) — the name of a template parameter,

(7.2) — it is of the form *^^type-id* and the *type-id* denotes a dependent type, or

(7.3) — it is of the form *^^id-expression* and the *id-expression* is value-dependent.

- 8 A *splice-expression* is value-dependent if its *splice-specifier* or *splice-specialization-specifier* is dependent (13.8.3.5).

### 13.8.3.5 Dependent splice specifiers

[temp.dep.splice]

- 1 A *splice-specifier* is dependent if its converted *constant-expression* is value-dependent. A *splice-specialization-specifier* is dependent if its *splice-specifier* is dependent or if any of its template arguments are dependent. A *splice-scope-specifier* is dependent if its *splice-specifier* or *splice-specialization-specifier* is dependent.

- 2 [Example 1:

```
template<auto T, auto NS>
void fn() {
 using a = [:T:]<1>; // [:T:]<1> is dependent because [:T:] is dependent
 static_assert([:NS:]::template TCls<1>::v == a::v); // [:NS:] is dependent
}

namespace N {
 template <auto V> struct TCls { static constexpr int v = V; };
}
```

```
int main() {
 fn<N::Tcls, N>();
}
```

— end example]

3 [Example 2:

```
template<template<class> class X>
struct S {
 [N::X:]<int, float> m;
};

template<class> struct V1 {};
template<class, class = int> struct V2 {};
```

```
S<V1> s1; // error: V1<int, float> has too many template arguments
S<V2> s2; // OK
```

— end example]

### 13.8.3.6 Dependent namespaces

[temp.dep.namespace]

1 A namespace alias is dependent if it is introduced by a *namespace-alias-definition* whose *qualified-namespace-specifier* (if any) is a dependent qualified name or whose *splice-specifier* (if any) is dependent. A *namespace-name* is dependent if it names a dependent namespace alias.

2 [Example 1:

```
template<std::meta::info R>
int fn() {
 namespace Alias = [R:]; // [R:] is dependent
 return typename Alias::T{}; // Alias is dependent
}
```

```
namespace NS {
 using T = int;
}
```

```
int a = fn<NS>();
```

— end example]

### 13.8.3.7 Dependent template arguments

[temp.dep.temp]

1 A type *template-argument* is dependent if the type it specifies is dependent.

2 A constant *template-argument* is dependent if its type is dependent or the constant expression it specifies is value-dependent.

3 Furthermore, a constant *template-argument* is dependent if the corresponding constant template parameter is of reference or pointer type and the *template-argument* designates or points to a member of the current instantiation or a member of a dependent type.

4 A template argument is also dependent if it is a pack expansion.

5 A template template argument is dependent if it names a template parameter or its terminal name is dependent.

## 13.8.4 Dependent name resolution

[temp.dep.res]

### 13.8.4.1 Point of instantiation

[temp.point]

1 For a function template specialization, a member function template specialization, or a specialization for a member function or static data member of a class template, the point of instantiation of the specialization is the point of instantiation of the enclosing specialization if

- (1.1) — the specialization is implicitly instantiated because it is referenced from within another template specialization and
- (1.2) — the context from which it is referenced depends on a template parameter.



Otherwise, the point of instantiation for such a specialization immediately follows the namespace scope declaration or definition that refers to the specialization.

- 2 If a function template or member function of a class template is called in a way which uses the definition of a default argument of that function template or member function, the point of instantiation of the default argument is the point of instantiation of the function template or member function specialization.
- 3 For a *noexcept-specifier* of a function template specialization or specialization of a member function of a class template, the point of instantiation of the *noexcept-specifier* is the point of instantiation of the specialization that requires it if
  - (3.1) — the *noexcept-specifier* is implicitly instantiated because it is needed by another template specialization and
  - (3.2) — the context that requires it depends on a template parameter.

Otherwise, the point of instantiation for such a *noexcept-specifier* immediately follows the namespace scope declaration or definition that requires the *noexcept-specifier*.

- 4 For a class template specialization, a class member template specialization, or a specialization for a class member of a class template, the point of instantiation is immediately before the point of instantiation of the enclosing template if
  - (4.1) — the specialization is implicitly instantiated because it is referenced from within another template specialization,
  - (4.2) — the context from which the specialization is referenced depends on a template parameter, and
  - (4.3) — the specialization is not instantiated previous to the instantiation of the enclosing template.

Otherwise, the point of instantiation for such a specialization immediately precedes the namespace scope declaration or definition that refers to the specialization.

- 5 If a virtual function is implicitly instantiated, its point of instantiation is immediately following the point of instantiation of its enclosing class template specialization.
- 6 An explicit instantiation definition is an instantiation point for the specialization or specializations specified by the explicit instantiation.
- 7 A specialization for a function template, a member function template, or of a member function or static data member of a class template may have multiple points of instantiations within a translation unit, and in addition to the points of instantiation described above,
  - (7.1) — for any such specialization that has a point of instantiation within the *declaration-seq* of the *translation-unit*, prior to the *private-module-fragment* (if any), the point after the *declaration-seq* of the *translation-unit* is also considered a point of instantiation, and
  - (7.2) — for any such specialization that has a point of instantiation within the *private-module-fragment*, the end of the translation unit is also considered a point of instantiation.

A specialization for a class template has at most one point of instantiation within a translation unit. A specialization for any template may have points of instantiation in multiple translation units. If two different points of instantiation give a template specialization different meanings according to the one-definition rule (6.3), the program is ill-formed, no diagnostic required (G.10.9).

- 8 For the *compound-statement* of an *expansion-statement* (8.7), the point of instantiation is the point of instantiation of its enclosing templated entity, if any. Otherwise, it immediately follows the namespace-scope declaration or definition that contains the expansion statement.

#### 13.8.4.2 Candidate functions

[temp.dep.candidate]

- 1 If a dependent call (13.8.3) would be ill-formed or would find a better match had the lookup for its dependent name considered all the function declarations with external linkage introduced in the associated namespaces in all translation units, not just considering those declarations found in the template definition and template instantiation contexts (6.5.4), then the program is ill-formed, no diagnostic required (G.10.10).
- 2 [Example 1:

Source file "X.h":

```
namespace Q {
 struct X { };
}
```

Source file "G.h":

```
namespace Q {
 void g_impl(X, X);
}
```

Module interface unit of M1:

```
module;
#include "X.h"
#include "G.h"
export module M1;
export template<typename T>
void g(T t) {
 g_impl(t, Q::X{ }); // ADL in definition context finds Q::g_impl, g_impl not discarded
}
```

Module interface unit of M2:

```
module;
#include "X.h"
export module M2;
import M1;
void h(Q::X x) {
 g(x); // OK
}
```

— end example]

<sup>3</sup> [Example 2:

Module interface unit of Std:

```
export module Std;
export template<typename Iter>
void indirect_swap(Iter lhs, Iter rhs)
{
 swap(*lhs, *rhs); // swap not found by unqualified lookup, can be found only via ADL
}
```

Module interface unit of M:

```
export module M;
import Std;

struct S { /* ... */ };
void swap(S&, S&); // #1

void f(S* p, S* q)
{
 indirect_swap(p, q); // finds #1 via ADL in instantiation context
}
```

— end example]

<sup>4</sup> [Example 3:

Source file "X.h":

```
struct X { /* ... */ };
X operator+(X, X);
```

Module interface unit of F:

```
export module F;
export template<typename T>
void f(T t) {
 t + t;
}
```

Module interface unit of M:

```
module;
```

```

#include "X.h"
export module M;
import F;
void g(X x) {
 f(x); // OK, instantiates f from F,
 // operator+ is visible in instantiation context
}

```

— end example]

<sup>5</sup> [Example 4:

Module interface unit of A:

```

export module A;
export template<typename T>
void f(T t) {
 cat(t, t); // #1
 dog(t, t); // #2
}

```

Module interface unit of B:

```

export module B;
import A;
export template<typename T, typename U>
void g(T t, U u) {
 f(t);
}

```

Source file "foo.h", not an importable header:

```

struct foo {
 friend int cat(foo, foo);
};
int dog(foo, foo);

```

Module interface unit of C1:

```

module;
#include "foo.h" // dog not referenced, discarded
export module C1;
import B;
export template<typename T>
void h(T t) {
 g(foo{ }, t);
}

```

Translation unit:

```

import C1;
void i() {
 h(0); // error: dog not found at #2
}

```

Importable header "bar.h":

```

struct bar {
 friend int cat(bar, bar);
};
int dog(bar, bar);

```

Module interface unit of C2:

```

module;
#include "bar.h" // imports header unit "bar.h"
export module C2;
import B;
export template<typename T>
void j(T t) {
 g(bar{ }, t);
}

```

Translation unit:

```
import C2;
void k() {
 j(0); // OK, dog found in instantiation context:
 // visible at end of module interface unit of C2
}
— end example]
```

## 13.9 Template instantiation and specialization

[temp.spec]

### 13.9.1 General

[temp.spec.general]

- <sup>1</sup> The act of instantiating a function, a variable, a class, a member of a class template, or a member template is referred to as *template instantiation*.
- <sup>2</sup> A function instantiated from a function template is called an instantiated function. A class instantiated from a class template is called an instantiated class. A member function, a member class, a member enumeration, or a static data member of a class template instantiated from the member definition of the class template is called, respectively, an instantiated member function, member class, member enumeration, or static data member. A member function instantiated from a member function template is called an instantiated member function. A member class instantiated from a member class template is called an instantiated member class. A variable instantiated from a variable template is called an instantiated variable. A static data member instantiated from a static data member template is called an instantiated static data member.
- <sup>3</sup> An explicit specialization may be declared for a function template, a variable template, a class template, a member of a class template, or a member template. An explicit specialization declaration is introduced by `template<>`. In an explicit specialization declaration for a variable template, a class template, a member of a class template, or a class member template, the variable or class that is explicitly specialized shall be specified with a *simple-template-id*. In the explicit specialization declaration for a function template, the function explicitly specialized may be specified using a *template-id*.

[Example 1:

```
template<class T = int> struct A {
 static int x;
};
template<class U> void g(U) { }

template<> struct A<double> { }; // specialize for T == double
template<> struct A<> { }; // specialize for T == int
template<> void g(char) { } // specialize for U == char
 // U is deduced from the parameter type
template<> void g<int>(int) { } // specialize for U == int
template<> int A<char>::x = 0; // specialize for T == char

template<class T = int> struct B {
 static int x;
};
template<> int B<>::x = 1; // specialize for T == int
```

— end example]

- <sup>4</sup> An instantiated template specialization can be either implicitly instantiated (13.9.2) for a given argument list or be explicitly instantiated (13.9.3). A *specialization* is a class, variable, function, or class member that is either instantiated (13.9.2) from a templated entity or is an explicit specialization (13.9.4) of a templated entity.
- <sup>5</sup> For a given template and a given set of *template-arguments*,
  - (5.1) — an explicit instantiation definition shall appear at most once in a program,
  - (5.2) — an explicit specialization shall be defined at most once in a program, as specified in 6.3, and
  - (5.3) — both an explicit instantiation and a declaration of an explicit specialization shall not appear in a program unless the explicit specialization is reachable from the explicit instantiation.

An implementation is not required to diagnose a violation of this rule if neither declaration is reachable from the other.

- <sup>6</sup> The usual access checking rules do not apply to names in a declaration of an explicit instantiation or explicit specialization, with the exception of names appearing in a function body, default argument, *base-clause*, *member-specification*, *enumerator-list*, or static data member or variable template initializer.

[*Note 1*: In particular, the template arguments and names used in the function declarator (including parameter types, return types and exception specifications) can be private types or objects that would normally not be accessible. — *end note*]

- <sup>7</sup> Each class template specialization instantiated from a template has its own copy of any static members.

[*Example 2*:

```
template<class T> class X {
 static T s;
};
template<class T> T X<T>::s = 0;
X<int> aa;
X<char*> bb;
```

X<int> has a static member *s* of type *int* and X<char\*> has a static member *s* of type *char\**. — *end example*]

- <sup>8</sup> If a function declaration acquired its function type through a dependent type (13.8.3.2) without using the syntactic form of a function declarator, the program is ill-formed.

[*Example 3*:

```
template<class T> struct A {
 static T t;
};
typedef int function();
A<function> a; // error: would declare A<function>::t as a static member function
```

— *end example*]

### 13.9.2 Implicit instantiation

[temp.inst]

- <sup>1</sup> A template specialization *E* is a *declared specialization* if there is a reachable explicit instantiation definition (13.9.3) or explicit specialization declaration (13.9.4) for *E*, or if there is a reachable explicit instantiation declaration for *E* and *E* is not

- (1.1) — an inline function,
- (1.2) — declared with a type deduced from its initializer or return value (9.2.9.7),
- (1.3) — a potentially-constant variable (7.7.4), or
- (1.4) — a specialization of a templated class.

[*Note 1*: An implicit instantiation in an importing translation unit cannot use names with internal linkage from an imported translation unit (6.7). — *end note*]

- <sup>2</sup> Unless a class template specialization is a declared specialization, the class template specialization is implicitly instantiated when the specialization is referenced in a context that requires a completely-defined object type or when the completeness of the class type affects the semantics of the program.

[*Note 2*: In particular, if the semantics of an expression depend on the member or base class lists of a class template specialization, the class template specialization is implicitly generated. For instance, deleting a pointer to class type depends on whether or not the class declares a destructor, and a conversion between pointers to class type depends on the inheritance relationship between the two classes involved. — *end note*]

[*Example 1*:

```
template<class T> class B { /* ... */ };
template<class T> class D : public B<T> { /* ... */ };

void f(void*);
void f(B<int>*);

void g(D<int>* p, D<char>* pp, D<double>* ppp) {
 f(p); // instantiation of D<int> required: call f(B<int>*)
 B<char>* q = pp; // instantiation of D<char> required: convert D<char>* to B<char>*
 delete ppp; // instantiation of D<double> required
}
```

— *end example*]

If the template selected for the specialization (13.7.6.2) has been declared, but not defined, at the point of instantiation (13.8.4.1), the instantiation yields an incomplete class type (6.9.1).

[Example 2:

```
template<class T> class X;
X<char> ch; // error: incomplete type X<char>
```

— end example]

3 The implicit instantiation of a class template specialization causes

- (3.1) — the implicit instantiation of the declarations, but not of the definitions, of the non-deleted class member functions, member classes, scoped member enumerations, static data members, member templates, and friends; and
- (3.2) — the implicit instantiation of the definitions of deleted member functions, unscoped member enumerations, and member anonymous unions.

[Example 3:

```
template<class T>
struct C {
 void f() { T x; }
 void g() = delete;
};
C<void> c; // OK, definition of C<void>::f is not instantiated at this point
template<> void C<int>::g() { } // error: redefinition of C<int>::g
```

— end example]

However, for the purpose of determining whether an instantiated redeclaration is valid according to 6.3 and 11.4, an instantiated declaration that corresponds to a definition in the template is considered to be a definition.

[Example 4:

```
template<class T, class U>
struct Outer {
 template<class X, class Y> struct Inner;
 template<class Y> struct Inner<T, Y>; // #1a
 template<class Y> struct Inner<T, Y> { }; // #1b; OK, valid redeclaration of #1a
 template<class Y> struct Inner<U, Y> { }; // #2
};
```

```
Outer<int, int> outer; // error at #2
```

Outer<int, int>::Inner<int, Y> is redeclared at #1b. (It is not defined but noted as being associated with a definition in Outer<T, U>.) #2 is also a redeclaration of #1a. It is noted as associated with a definition, so it is an invalid redeclaration of the same partial specialization.

```
template<typename T> struct Friendly {
 template<typename U> friend int f(U) { return sizeof(T); }
};
Friendly<char> fc;
Friendly<float> ff; // error: produces second definition of f(U)
```

— end example]

- 4 Unless a member of a templated class is a declared specialization, the specialization of the member is implicitly instantiated when the specialization is referenced in a context that requires the member definition to exist or if the existence of the definition of the member affects the semantics of the program; in particular, the initialization (and any associated side effects) of a static data member does not occur unless the static data member is itself used in a way that requires the definition of the static data member to exist.
- 5 Unless a function template specialization is a declared specialization, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist or if the existence of the definition affects the semantics of the program. A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist or if the existence of the definition affects the semantics of the program.

[*Note 3:* An inline function that is the subject of an explicit instantiation declaration is not a declared specialization; the intent is that it still be implicitly instantiated when odr-used (6.3) so that the body can be considered for inlining, but that no out-of-line copy of it be generated in the translation unit. — *end note*]

6 [Example 5:

```
template<class T> struct Z {
 void f();
 void g();
};

void h() {
 Z<int> a; // instantiation of class Z<int> required
 Z<char>* p; // instantiation of class Z<char> not required
 Z<double>* q; // instantiation of class Z<double> not required

 a.f(); // instantiation of Z<int>::f() required
 p->g(); // instantiation of class Z<char> required, and
 // instantiation of Z<char>::g() required
}
```

Nothing in this example requires class Z<double>, Z<int>::g(), or Z<char>::f() to be implicitly instantiated. — *end example*

7 Unless a variable template specialization is a declared specialization, the variable template specialization is implicitly instantiated when it is referenced in a context that requires a variable definition to exist or if the existence of the definition affects the semantics of the program. A default template argument for a variable template is implicitly instantiated when the variable template is referenced in a context that requires the value of the default argument.

8 The existence of a definition of a variable or function is considered to affect the semantics of the program if the variable or function is needed for constant evaluation by an expression (7.7.7), even if constant evaluation of the expression is not required or if constant expression evaluation does not use the definition.

[Example 6:

```
template<typename T> constexpr int f() { return T::value; }
template<bool B, typename T> void g(decltype(B ? f<T>() : 0));
template<bool B, typename T> void g(...);
template<bool B, typename T> void h(decltype(int{B ? f<T>() : 0}));
template<bool B, typename T> void h(...);
void x() {
 g<false, int>(0); // OK, B ? f<T>() : 0 is not potentially constant evaluated
 h<false, int>(0); // error, instantiates f<int> even though B evaluates to false and
 // list-initialization of int from int cannot be narrowing
}
```

— *end example*

9 Similarly, the existence of a definition of a variable or function is considered to affect the semantics of the program if the instantiation is necessary to determine the type of an expression in the program, even if the variable or function is not odr-used (6.3).

[Example 7:

```
template<typename T> struct X {
 static inline int arr[] = {1, 2, T::error};
};
decltype(+X<int>::arr) r; // error: definition of X<int>::arr is instantiated to complete its type

template<typename T> struct X2 {
 static inline int arr[3] = {1, 2, T::error};
};
decltype(+X2<int>::arr) r2; // OK, type of arr is complete and arr is not odr-used
```

— *end example*

10 If the function selected by overload resolution (12.2) can be determined without instantiating a class template definition, it is unspecified whether that instantiation actually takes place.

[Example 8:

```
template <class T> struct S {
 operator int();
};

void f(int);
void f(S<int>&);
void f(S<float>);

void g(S<int>& sr) {
 f(sr); // instantiation of S<int> allowed but not required
 // instantiation of S<float> allowed but not required
};
```

— end example]

11 If a function template specialization is used in a way that involves overload resolution, a declaration of the specialization is implicitly instantiated (13.10.4).

12 An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, a member class or static data member of a templated class, or a substatement of a constexpr if statement (8.5.2), unless such instantiation is required.

[Note 4: The instantiation of a generic lambda does not require instantiation of substatements of a constexpr if statement within its *compound-statement* unless the call operator template is instantiated. — end note]

It is unspecified whether or not an implementation implicitly instantiates a virtual member function of a class template if the virtual member function would not otherwise be instantiated. The use of a template specialization in a default argument or default member initializer shall not cause the template to be implicitly instantiated except where needed to determine the correctness of the default argument or default member initializer. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated. Similarly, the use of a default member initializer in a constructor definition or an aggregate initialization causes specializations in the default member initializer to be instantiated.

13 The separately instantiated constructs (13.7.1) of a templated function *F* that is either

(13.1) — a member of a local class or

(13.2) — the function call operator of the closure type of a non-generic *lambda-expression*

are instantiated when the declaration of *F* is instantiated.

[Note 5: For the purposes of instantiation, these constructs are still considered separately from the function to which they belong. — end note]

14 Other than as specified above, when a templated function *f* is called in a way that requires a default argument to be used, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the default argument is done as if the default argument had been an initializer used in a function template specialization with the same scope, the same template parameters and the same access as that of the function template *f* used at that point, except that the scope in which a closure type is declared (7.5.6.2) — and therefore its associated namespaces — remain as determined from the context of the definition for the default argument. This analysis is called *default argument instantiation*. The instantiated default argument is then used as the argument of *f*.

15 [Note 6: Each default argument is instantiated independently. — end note]

[Example 9:

```
template<class T> void f(T x, T y = ydef(T()), T z = zdef(T()));

class A { };

A zdef(A);

void g(A a, A b, A c) {
 f(a, b, c); // no default argument instantiation
 f(a, b); // default argument z = zdef(T()) instantiated
 f(a); // error: ydef is not declared
}
```



— *end example*]

- 16 Other than as specified above, the separately instantiated constructs of a specialization of a templated function are instantiated only when needed (14.5, 9.4.1). When such a construct is instantiated, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the construct is done as if it were being done as part of instantiating the declaration of the specialization at that point.

- 17 [Note 7: 13.8.4.1 defines the point of instantiation of a template specialization. — *end note*]

- 18 There is an implementation-defined quantity that specifies the limit on the total depth of recursive instantiations (Annex B), which could involve more than one template. The result of an infinite recursion in instantiation is undefined (F.7.1).

[Example 10:

```
template<class T> class X {
 X<T*>* p; // OK
 X<T*>* a; // implicit generation of X<T> requires
 // the implicit instantiation of X<T*> which requires
 // the implicit instantiation of X<T**> which ...
};
```

— *end example*]

- 19 The *type-constraints* and *requires-clause* of a template specialization or member function are not instantiated along with the specialization or function itself, even for a member function of a local class; substitution into the atomic constraints formed from them is instead performed as specified in 13.5.3 and 13.5.2.3 when determining whether the constraints are satisfied or as specified in 13.5.3 when comparing declarations.

[Note 8: The satisfaction of constraints is determined during template argument deduction (13.10.3) and overload resolution (12.2). — *end note*]

[Example 11:

```
template<typename T> concept C = sizeof(T) > 2;
template<typename T> concept D = C<T> && sizeof(T) > 4;

template<typename T> struct S {
 S() requires C<T> { } // #1
 S() requires D<T> { } // #2
};

S<char> s1; // error: no matching constructor
S<char[8]> s2; // OK, calls #2
```

When `S<char>` is instantiated, both constructors are part of the specialization. Their constraints are not satisfied, and they suppress the implicit declaration of a default constructor for `S<char>` (11.4.5.2), so there is no viable constructor for `s1`. — *end example*]

[Example 12:

```
template<typename T> struct S1 {
 template<typename U>
 requires false
 struct Inner1; // ill-formed, no diagnostic required
};

template<typename T> struct S2 {
 template<typename U>
 requires (sizeof(T[-(int)sizeof(T)]) > 1)
 struct Inner2; // ill-formed, no diagnostic required
};
```

The class `S1<T>::Inner1` is ill-formed, no diagnostic required, because it has no valid specializations. `S2` is ill-formed, no diagnostic required, since no substitution into the constraints of its `Inner2` template would result in a valid expression. — *end example*]

### 13.9.3 Explicit instantiation

[temp.explicit]

- 1 A class, function, variable, or member template specialization can be explicitly instantiated from its template. A member function, member class or static data member of a class template can be explicitly instantiated

from the member definition associated with its class template.

- <sup>2</sup> The syntax for explicit instantiation is:

```
explicit-instantiation:
 externopt template declaration
```

There are two forms of explicit instantiation: an explicit instantiation definition and an explicit instantiation declaration. An explicit instantiation declaration begins with the **extern** keyword.

- <sup>3</sup> An explicit instantiation shall not use a *storage-class-specifier* (9.2.2) other than **thread\_local**. An explicit instantiation of a function template, member function of a class template, or variable template shall not use the **inline**, **constexpr**, or **consteval** specifiers. No *attribute-specifier-seq* (9.13.1) shall appertain to an explicit instantiation.
- <sup>4</sup> If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id*; otherwise, the *declaration* shall be a *simple-declaration* whose *init-declarator-list* comprises a single *init-declarator* that does not have an *initializer*. If the explicit instantiation is for a variable template specialization, the *unqualified-id* in the *declarator* shall be a *simple-template-id*.

[Example 1:

```
template<class T> class Array { void mf(); };
template class Array<char>;
template void Array<int>::mf();

template<class T> void sort(Array<T>& v) { /* ... */ }
template void sort(Array<char>&); // argument is deduced here (13.10.2)

namespace N {
 template<class T> void f(T&) { }
}
template void N::f<int>(int&);
```

— end example]

- <sup>5</sup> An explicit instantiation does not introduce a name (6.4.1). A declaration of a function template, a variable template, a member function or static data member of a class template, or a member function template of a class or class template shall be reachable from any explicit instantiation of that entity. A definition of a class template, a member class of a class template, or a member class template of a class or class template shall be reachable from any explicit instantiation of that entity unless an explicit specialization of the entity with the same template arguments is reachable therefrom. If the *declaration* of the explicit instantiation names an implicitly-declared special member function (11.4.4), the program is ill-formed.
- <sup>6</sup> The *declaration* in an *explicit-instantiation* and the *declaration* produced by the corresponding substitution into the templated function, variable, or class are two declarations of the same entity.

[Note 1: These declarations need to have matching types as specified in 6.7, except as specified in 14.5.

[Example 2:

```
template<typename T> T var = {};
template float var<float>; // OK, instantiated variable has type float
template int var<int[16]>[]; // OK, absence of major array bound is permitted
template int *var<int>; // error: instantiated variable has type int

template<typename T> auto av = T();
template int av<int>; // OK, variable with type auto can be redeclared with type int

template<typename T> auto f() {}
template void f<int>(); // error: function with deduced return type
 // redeclared with non-deduced return type (9.2.9.7)
```

— end example]

— end note]

Despite its syntactic form, the *declaration* in an *explicit-instantiation* for a variable is not itself a definition and does not conflict with the definition instantiated by an explicit instantiation definition for that variable.

- <sup>7</sup> For a given set of template arguments, if an explicit instantiation of a template appears after a declaration of an explicit specialization for that template, the explicit instantiation has no effect. Otherwise, for an

explicit instantiation definition, the definition of a function template, a variable template, a member function template, or a member function or static data member of a class template shall be present in every translation unit in which it is explicitly instantiated.

- <sup>8</sup> [Note 2: An explicit instantiation of a constrained template needs to satisfy that template's associated constraints (13.5.3). The satisfaction of constraints is determined when forming the template name of an explicit instantiation in which all template arguments are specified (13.3), or, for explicit instantiations of function templates, during template argument deduction (13.10.3.7) when one or more trailing template arguments are left unspecified. — end note]
- <sup>9</sup> An explicit instantiation that names a class template specialization is also an explicit instantiation of the same kind (declaration or definition) of each of its direct non-template members that has not been previously explicitly specialized in the translation unit containing the explicit instantiation, provided that the associated constraints, if any, of that member are satisfied by the template arguments of the explicit instantiation (13.5.3, 13.5.2), except as described below.
- [Note 3: In addition, it will typically be an explicit instantiation of certain implementation-dependent data about the class. — end note]
- <sup>10</sup> An explicit instantiation definition that names a class template specialization explicitly instantiates the class template specialization and is an explicit instantiation definition of only those members that have been defined at the point of instantiation.
- <sup>11</sup> An explicit instantiation of a prospective destructor (11.4.7) shall correspond to the selected destructor of the class.
- <sup>12</sup> If an entity is the subject of both an explicit instantiation declaration and an explicit instantiation definition in the same translation unit, the definition shall follow the declaration. An entity that is the subject of an explicit instantiation declaration and that is also used in a way that would otherwise cause an implicit instantiation (13.9.2) in the translation unit shall be the subject of an explicit instantiation definition somewhere in the program; otherwise the program is ill-formed, no diagnostic required (G.10.11).

[Note 4: This rule does apply to inline functions even though an explicit instantiation declaration of such an entity has no other normative effect. This is needed to ensure that if the address of an inline function is taken in a translation unit in which the implementation chose to suppress the out-of-line body, another translation unit will supply the body. — end note]

An explicit instantiation declaration shall not name a specialization of a template with internal linkage.

- <sup>13</sup> An explicit instantiation does not constitute a use of a default argument, so default argument instantiation is not done.

[Example 3:

```
char* p = 0;
template<class T> T g(T x = &p) { return x; }
template int g<int>(int); // OK even though &p isn't an int.
```

— end example]

### 13.9.4 Explicit specialization

[temp.expl.spec]

- <sup>1</sup> An explicit specialization of any of the following:
- (1.1) — function template
  - (1.2) — class template
  - (1.3) — variable template
  - (1.4) — member function of a class template
  - (1.5) — static data member of a class template
  - (1.6) — member class of a class template
  - (1.7) — member enumeration of a class template
  - (1.8) — member class template of a class or class template
  - (1.9) — member function template of a class or class template

can be declared by a declaration introduced by `template<>`; that is:

*explicit-specialization:*  
`template <> declaration`

[Example 1:

```
template<class T> class stream;

template<> class stream<char> { /* ... */ }; // #1

template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

template<> void sort<int>(Array<int>&); // #2
template<> void sort(Array<char*>&); // #3 template argument is deduced (13.10.2)
```

Given these declarations, #1 will be used as the definition of streams of `chars`; other streams will be handled by class template specializations instantiated from the class template. Similarly, #2 will be used as the sort function for arguments of type `Array<int>` and #3 will be used for arguments of type `Array<char*>`; other `Array` types will be sorted by functions generated from the function template. — *end example*]

- 2 The *declaration* in an *explicit-specialization* shall not be an *export-declaration*. An explicit specialization shall not use a *storage-class-specifier* (9.2.2) other than `thread_local`.
- 3 An explicit specialization may be declared in any scope in which the corresponding primary template may be defined (9.3.4, 11.4, 13.7.3).
- 4 An explicit specialization does not introduce a name (6.4.1). A declaration of a function template, class template, or variable template being explicitly specialized shall be reachable from the declaration of the explicit specialization.

[Note 1: A declaration, but not a definition of the template is needed. — *end note*]

The definition of a class or class template shall be reachable from the declaration of an explicit specialization for a member template of the class or class template.

[Example 2:

```
template<> class X<int> { /* ... */ }; // error: X not a template

template<class T> class X;

template<> class X<char*> { /* ... */ }; // OK, X is a template
```

— *end example*]

- 5 A member function, a member function template, a member class, a member enumeration, a member class template, a static data member, or a static data member template of a class template may be explicitly specialized for a class specialization that is implicitly instantiated; in this case, the definition of the class template shall be reachable from the explicit specialization for the member of the class template. If such an explicit specialization for the member of a class template names an implicitly-declared special member function (11.4.4), the program is ill-formed.
- 6 A member of an explicitly specialized class is not implicitly instantiated from the member declaration of the class template; instead, the member of the class template specialization shall itself be explicitly defined if its definition is required. The definition of the class template explicit specialization shall be reachable from the definition of any member of it. The definition of an explicitly specialized class is unrelated to the definition of a generated specialization. That is, its members need not have the same names, types, etc. as the members of a generated specialization. Members of an explicitly specialized class template are defined in the same manner as members of normal classes, and not using the `template<>` syntax. The same is true when defining a member of an explicitly specialized member class. However, `template<>` is used in defining a member of an explicitly specialized member class template that is specialized as a class template.

[Example 3:

```
template<class T> struct A {
 struct B { };
 template<class U> struct C { };
};

template<> struct A<int> {
 void f(int);
};
```

```

void h() {
 A<int> a;
 a.f(16); // A<int>::f must be defined somewhere
}

// template<> not used for a member of an explicitly specialized class template
void A<int>::f(int) { /* ... */ }

template<> struct A<char>::B {
 void f();
};
// template<> also not used when defining a member of an explicitly specialized member class
void A<char>::B::f() { /* ... */ }

template<> template<class U> struct A<char>::C {
 void f();
};
// template<> is used when defining a member of an explicitly specialized member class template
// specialized as a class template
template<>
template<class U> void A<char>::C<U>::f() { /* ... */ }

template<> struct A<short>::B {
 void f();
};
template<> void A<short>::B::f() { /* ... */ } // error: template<> not permitted

template<> template<class U> struct A<short>::C {
 void f();
};
template<class U> void A<short>::C<U>::f() { /* ... */ } // error: template<> required
—end example]

```

- <sup>7</sup> If a template, a member template or a member of a class template is explicitly specialized, a declaration of that specialization shall be reachable from every use of that specialization that would cause an implicit instantiation to take place, in every translation unit in which such a use occurs; no diagnostic is required (G.10.12). If the program does not provide a definition for an explicit specialization and either the specialization is used in a way that would cause an implicit instantiation to take place or the member is a virtual member function, the program is ill-formed, no diagnostic required (G.10.13). An implicit instantiation is never generated for an explicit specialization that is declared but not defined.

[Example 4:

```

class String { };
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

void f(Array<String>& v) {
 sort(v); // use primary template sort(Array<T>&), T is String
}

template<> void sort<String>(Array<String>& v); // error: specialization after use of primary template
template<> void sort<>(Array<char*>& v); // OK, sort<char*> not yet used
template<class T> struct A {
 enum E : T;
 enum class S : T;
};
template<> enum A<int>::E : int { eint }; // OK
template<> enum class A<int>::S : int { sint }; // OK
template<class T> enum A<T>::E : T { eT };
template<class T> enum class A<T>::S : T { sT };
template<> enum A<char>::E : char { echar }; // error: A<char>::E was instantiated
 // when A<char> was instantiated
template<> enum class A<char>::S : char { schar }; // OK

```

— end example]

- 8 The placement of explicit specialization declarations for function templates, class templates, variable templates, member functions of class templates, static data members of class templates, member classes of class templates, member enumerations of class templates, member class templates of class templates, member function templates of class templates, static data member templates of class templates, member functions of member templates of class templates, member functions of member templates of non-template classes, static data member templates of non-template classes, member function templates of member classes of class templates, etc., and the placement of partial specialization declarations of class templates, variable templates, member class templates of non-template classes, static data member templates of non-template classes, member class templates of class templates, etc., can affect whether a program is well-formed according to the relative positioning of the explicit specialization declarations and their points of instantiation in the translation unit as specified above and below. When writing a specialization, be careful about its location; or to make it compile will be such a trial as to kindle its self-immolation.

- 9 [Note 2: A class template explicit specialization that has been declared but not defined can be used exactly like other incompletely-defined classes (6.9).

[Example 5:

```
template<class T> class X; // X is a class template
template<> class X<int>;

X<int>* p; // OK, pointer to declared class X<int>
X<int> x; // error: object of incomplete class X<int>
```

— end example]

— end note]

- 10 [Note 3: An explicit specialization of a constrained template needs to satisfy that template's associated constraints (13.5.3). The satisfaction of constraints is determined when forming the template name of an explicit specialization in which all template arguments are specified (13.3), or, for explicit specializations of function templates, during template argument deduction (13.10.3.7) when one or more trailing template arguments are left unspecified. — end note]
- 11 A function with the same name as a template and a type that exactly matches that of a template specialization is not an explicit specialization (13.7.7).
- 12 Whether an explicit specialization of a function or variable template is inline, constexpr, constexpr, or consteval is determined by the explicit specialization and is independent of those properties of the template. Similarly, attributes and *function-contract-specifiers* appearing in the declaration of a template have no effect on an explicit specialization of that template.

[Example 6:

```
template<class T> void f(T) { /* ... */ }
template<class T> inline T g(T) { /* ... */ }

template<> inline void f<>(int) { /* ... */ } // OK, inline
template<> int g<>(int) { /* ... */ } // OK, not inline

template<typename> [[noreturn]] void h([[maybe_unused]] int i);
template<> void h<int>(int i) {
 // Implementations are expected not to warn that the function returns
 // but can warn about the unused parameter.
}
```

— end example]

- 13 An explicit specialization of a static data member of a template or an explicit specialization of a static data member template is a definition if the declaration includes an initializer; otherwise, it is a declaration.

[Note 4: The definition of a static data member of a template for which default-initialization is desired can use functional cast notation (7.6.1.4):

```
template<> X Q<int>::x; // declaration
template<> X Q<int>::x (); // error: declares a function
template<> X Q<int>::x = X(); // definition
```

— end note]

- <sup>14</sup> A member or a member template of a class template may be explicitly specialized for a given implicit instantiation of the class template, even if the member or member template is defined in the class template definition. An explicit specialization of a member or member template is specified using the syntax for explicit specialization.

[Example 7:

```
template<class T> struct A {
 void f(T);
 template<class X1> void g1(T, X1);
 template<class X2> void g2(T, X2);
 void h(T) { }
};

// specialization
template<> void A<int>::f(int);

// out of class member template definition
template<class T> template<class X1> void A<T>::g1(T, X1) { }

// member template specialization
template<> template<class X1> void A<int>::g1(int, X1);

// member template specialization
template<> template<>
 void A<int>::g1(int, char); // X1 deduced as char
template<> template<>
 void A<int>::g2<char>(int, char); // X2 specified as char

// member specialization even if defined in class definition
template<> void A<int>::h(int) { }
```

— end example]

- <sup>15</sup> A member or a member template may be nested within many enclosing class templates. In an explicit specialization for such a member, the member declaration shall be preceded by a `template<>` for each enclosing class template that is explicitly specialized.

[Example 8:

```
template<class T1> class A {
 template<class T2> class B {
 void mf();
 };
};

template<> template<> class A<int>::B<double>;
template<> template<> void A<char>::B<char>::mf();
```

— end example]

- <sup>16</sup> In an explicit specialization declaration for a member of a class template or a member template that appears in namespace scope, the member template and some of its enclosing class templates may remain unspecialized, except that the declaration shall not explicitly specialize a class member template if its enclosing class templates are not explicitly specialized as well. In such an explicit specialization declaration, the keyword `template` followed by a *template-parameter-list* shall be provided instead of the `template<>` preceding the explicit specialization declaration of the member. The types of the *template-parameters* in the *template-parameter-list* shall be the same as those specified in the primary template definition.

[Example 9:

```
template <class T1> class A {
 template<class T2> class B {
 template<class T3> void mf1(T3);
 void mf2();
 };
};

template <> template <class X>
 class A<int>::B {
 template <class T> void mf1(T);
```

```

};
template <> template <> template<class T>
void A<int>::B<double>::mf1(T t) { }
template <class Y> template <>
void A<Y>::B<double>::mf2() { } // error: B<double> is specialized but
 // its enclosing class template A is not

```

— end example]

- 17 A specialization of a member function template, member class template, or static data member template of a non-specialized class template is itself a template.
- 18 An explicit specialization declaration shall not be a friend declaration.
- 19 Default function arguments shall not be specified in a declaration or a definition for one of the following explicit specializations:
- (19.1) — the explicit specialization of a function template;
  - (19.2) — the explicit specialization of a member function template;
  - (19.3) — the explicit specialization of a member function of a class template where the class template specialization to which the member function specialization belongs is implicitly instantiated.
- [Note 5: Default function arguments can be specified in the declaration or definition of a member function of a class template specialization that is explicitly specialized. — end note]

## 13.10 Function template specializations

[temp.fct.spec]

### 13.10.1 General

[temp.fct.spec.general]

- 1 A function instantiated from a function template is called a function template specialization; so is an explicit specialization of a function template. Template arguments can be explicitly specified when naming the function template specialization, deduced from the context (e.g., deduced from the function arguments in a call to the function template specialization, see 13.10.3), or obtained from default template arguments.
- 2 Each function template specialization instantiated from a template has its own copy of any static variable.

[Example 1:

```

template<class T> void f(T* p) {
 static T s;
};

void g(int a, char* b) {
 f(&a); // calls f<int>(int*)
 f(&b); // calls f<char*>(char**)
}

```

Here `f<int>(int*)` has a static variable `s` of type `int` and `f<char*>(char**)` has a static variable `s` of type `char*`. — end example]

### 13.10.2 Explicit template argument specification

[temp.arg.explicit]

- 1 Template arguments can be specified when referring to a function template specialization that is not a specialization of a constructor template by qualifying the function template name with the list of *template-arguments* in the same way as *template-arguments* are specified in uses of a class template specialization.

[Example 1:

```

template<class T> void sort(Array<T>& v);
void f(Array<dcomplex>& cv, Array<int>& ci) {
 sort<dcomplex>(cv); // sort(Array<dcomplex>&)
 sort<int>(ci); // sort(Array<int>&)
}

```

and

```

template<class U, class V> U convert(V v);

void g(double d) {
 int i = convert<int,double>(d); // int convert(double)
 char c = convert<char,double>(d); // char convert(double)
}

```



— end example]

- 2 Template arguments shall not be specified when referring to a specialization of a constructor template (11.4.5, 6.5.5.2).
- 3 A template argument list may be specified when referring to a specialization of a function template
  - (3.1) — when a function is called,
  - (3.2) — when the address of a function is taken, when a function initializes a reference to function, or when a pointer to member function is formed,
  - (3.3) — in an explicit specialization,
  - (3.4) — in an explicit instantiation, or
  - (3.5) — in a friend declaration.

- 4 Trailing template arguments that can be deduced (13.10.3) or obtained from default *template-arguments* may be omitted from the list of explicit *template-arguments*.

[Note 1: A trailing template parameter pack (13.7.4) not otherwise deduced will be deduced as an empty sequence of template arguments. — end note]

If all of the template arguments can be deduced or obtained from default *template-arguments*, they may all be omitted; in this case, the empty template argument list <> itself may also be omitted.

[Example 2:

```
template<class X, class Y> X f(Y);
template<class X, class Y, class ... Z> X g(Y);
void h() {
 int i = f<int>(5.6); // Y deduced as double
 int j = f(5.6); // error: X cannot be deduced
 f<void>(f<int, bool>); // Y for outer f deduced as int (*)(bool)
 f<void>(f<int>); // error: f<int> does not denote a single function template specialization
 int k = g<int>(5.6); // Y deduced as double; Z deduced as an empty sequence
 f<void>(g<int, bool>); // Y for outer f deduced as int (*)(bool),
 // Z deduced as an empty sequence
}
```

— end example]

- 5 [Note 2: An empty template argument list can be used to indicate that a given use refers to a specialization of a function template even when a non-template function (9.3.4.6) is visible that would otherwise be used. For example:

```
template <class T> int f(T); // #1
int f(int); // #2
int k = f(1); // uses #2
int l = f<>(1); // uses #1
```

— end note]

- 6 Template arguments that are present shall be specified in the declaration order of their corresponding template parameters. The template argument list shall not specify more *template-arguments* than there are corresponding *template-parameters* unless one of the *template-parameters* declares a template parameter pack.

[Example 3:

```
template<class X, class Y, class Z> X f(Y,Z);
template<class ... Args> void f2();
void g() {
 f<int,const char*,double>("aa",3.0);
 f<int,const char*>("aa",3.0); // Z deduced as double
 f<int>("aa",3.0); // Y deduced as const char*; Z deduced as double
 f("aa",3.0); // error: X cannot be deduced
 f2<char, short, int, long>(); // OK
}
```

— end example]

- 7 Implicit conversions (7.3) will be performed on a function argument to convert it to the type of the corresponding function parameter if the parameter type contains no template parameters that participate in template argument deduction.

[*Note 3*: Template parameters do not participate in template argument deduction if they are explicitly specified. For example,

```
template<class T> void f(T);

class Complex {
 Complex(double);
};

void g() {
 f<Complex>(1); // OK, means f<Complex>(Complex(1))
}
```

— end note]

- <sup>8</sup> [*Note 4*: Because the explicit template argument list follows the function template name, and because constructor templates (11.4.5) are named without using a function name (6.5.5.2), there is no way to provide an explicit template argument list for these function templates. — end note]
- <sup>9</sup> Template argument deduction can extend the sequence of template arguments corresponding to a template parameter pack, even when the sequence contains explicitly specified template arguments.

[*Example 4*:

```
template<class ... Types> void f(Types ... values);

void g() {
 f<int*, float*>(0, 0, 0); // Types deduced as the sequence int*, float*, int
}
```

— end example]

### 13.10.3 Template argument deduction

[temp.deduct]

#### 13.10.3.1 General

[temp.deduct.general]

- <sup>1</sup> When a function template specialization is referenced, all of the template arguments shall have values. The values can be explicitly specified or, in some cases, be deduced from the use or obtained from default *template-arguments*.

[*Example 1*:

```
void f(Array<dcomplex>& cv, Array<int>& ci) {
 sort(cv); // calls sort(Array<dcomplex>&)
 sort(ci); // calls sort(Array<int>&)
}
```

and

```
void g(double d) {
 int i = convert<int>(d); // calls convert<int,double>(double)
 int c = convert<char>(d); // calls convert<char,double>(double)
}
```

— end example]

- <sup>2</sup> When an explicit template argument list is specified, if the given *template-id* or *splice-specialization-specifier* is not valid (13.3), type deduction fails. Otherwise, the specified template argument values are substituted for the corresponding template parameters as specified below.
- <sup>3</sup> After this substitution is performed, the function parameter type adjustments described in 9.3.4.6 are performed.

[*Example 2*: A parameter type of “void (const int, int[5])” becomes “void(\*) (int,int\*)”. — end example]

[*Note 1*: A top-level qualifier in a function parameter declaration does not affect the function type but still affects the type of the function parameter variable within the function. — end note]

[*Example 3*:

```
template <class T> void f(T t);
template <class X> void g(const X x);
template <class Z> void h(Z, Z*);
```

```

int main() {
 // #1: function type is f(int), t is non const
 f<int>(1);

 // #2: function type is f(int), t is const
 f<const int>(1);

 // #3: function type is g(int), x is const
 g<int>(1);

 // #4: function type is g(int), x is const
 g<const int>(1);

 // #5: function type is h(int, const int*)
 h<const int>(1,0);
}

```

— end example]

- 4 [Note 2: `f<int>(1)` and `f<const int>(1)` call distinct functions even though both of the functions called have the same function type. — end note]
- 5 The resulting substituted and adjusted function type is used as the type of the function template for template argument deduction. If a template argument has not been deduced and its corresponding template parameter has a default argument, the template argument is determined by substituting the template arguments determined for preceding template parameters into the default argument. If the substitution results in an invalid type, as described above, type deduction fails.

[Example 4:

```

template <class T, class U = double>
void f(T t = 0, U u = 0);

void g() {
 f(1, 'c'); // f<int,char>(1,'c')
 f(1); // f<int,double>(1,0)
 f(); // error: T cannot be deduced
 f<int>(); // f<int,double>(0,0)
 f<int,char>(); // f<int,char>(0,0)
}

```

— end example]

When all template arguments have been deduced or obtained from default template arguments, all uses of template parameters in the template parameter list of the template are replaced with the corresponding deduced or default argument values. If the substitution results in an invalid type, as described above, type deduction fails. If the function template has associated constraints (13.5.3), those constraints are checked for satisfaction (13.5.2). If the constraints are not satisfied, type deduction fails. In the context of a function call, if type deduction has not yet failed, then for those function parameters for which the function call has arguments, each function parameter with a type that was non-dependent before substitution of any explicitly-specified template arguments is checked against its corresponding argument; if the corresponding argument cannot be implicitly converted to the parameter type, type deduction fails.

[Note 3: Overload resolution will check the other parameters, including parameters with dependent types in which no template parameters participate in template argument deduction and parameters that became non-dependent due to substitution of explicitly-specified template arguments. — end note]

If type deduction has not yet failed, then all uses of template parameters in the function type are replaced with the corresponding deduced or default argument values. If the substitution results in an invalid type, as described above, type deduction fails.

[Example 5:

```

template <class T> struct Z {
 typedef typename T::x xx;
};

template <class T> concept C = requires { typename T::A; };

template <C T> typename Z<T>::xx f(void *, T); // #1
template <class T> void f(int, T); // #2

```

```

struct A {} a;
struct ZZ {
 template <class T, class = typename Z<T>::xx> operator T *();
 operator int();
};
int main() {
 ZZ zz;
 f(1, a); // OK, deduction fails for #1 because there is no conversion from int to void*
 f(zz, 42); // OK, deduction fails for #1 because C<int> is not satisfied
}
— end example]

```

- <sup>6</sup> At certain points in the template argument deduction process it is necessary to take a function type that makes use of template parameters and replace those template parameters with the corresponding template arguments. This is done at the beginning of template argument deduction when any explicitly specified template arguments are substituted into the function type, and again at the end of template argument deduction when any template arguments that were deduced or obtained from default arguments are substituted.

- <sup>7</sup> The *deduction substitution loci* are

- (7.1) — the function type outside of the *noexcept-specifier*,
- (7.2) — the *explicit-specifier*,
- (7.3) — the template parameter declarations, and
- (7.4) — the template argument list of a partial specialization (13.7.6.1).

The substitution occurs in all types and expressions that are used in the deduction substitution loci. The expressions include not only constant expressions such as those that appear in array bounds or as constant template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. If substitution into different declarations of the same function template would cause template instantiations to occur in a different order or not at all, the program is ill-formed; no diagnostic required (G.10.14).

[Note 4: The equivalent substitution in exception specifications (14.5) and function contract assertions (9.4.1) is done only when the *noexcept-specifier* or *function-contract-specifier*, respectively, is instantiated, at which point a program is ill-formed if the substitution results in an invalid type or expression. — end note]

[Example 6:

```

template <class T> struct A { using X = typename T::X; };
template <class T> typename T::X f(typename A<T>::X);
template <class T> void f(...) { }
template <class T> auto g(typename A<T>::X) -> typename T::X;
template <class T> void g(...) { }
template <class T> typename T::X h(typename A<T>::X);
template <class T> auto h(typename A<T>::X) -> typename T::X; // redeclaration
template <class T> void h(...) { }

void x() {
 f<int>(0); // OK, substituting return type causes deduction to fail
 g<int>(0); // error, substituting parameter type instantiates A<int>
 h<int>(0); // ill-formed, no diagnostic required
}

```

— end example]

- <sup>8</sup> If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written in the same context using the substituted arguments.

[Note 5: If no diagnostic is required, the program is still ill-formed. Access checking is done as part of the substitution process. — end note]

Invalid types and expressions can result in a deduction failure only in the immediate context of the deduction substitution loci. The *immediate context* of a deduction substitution locus is that deduction substitution locus, excluding bodies of *lambda-expressions* and separately instantiated constructs.

[Example 7:

```
template<typename T>
T* fun(T&& v); // #1: the deduction substitution locus is the function type
 // "function of (rvalue reference to T) returning pointer to T"

void fun(...); // #2

void test()
{
 int i;
 fun(i); // selects #2 (forming the type "pointer to reference to int" fails in #1)
}
```

— end example]

[Note 6: Separately instantiated constructs are excluded from the immediate contexts even if they are instantiated at the same time. — end note]

[Note 7: The substitution into types and expressions can result in effects such as the instantiation of template specializations, the generation of implicitly-defined functions, etc. Such effects are not in the immediate context and can result in the program being ill-formed. — end note]

- <sup>9</sup> [Note 8: No *statement* is ever in the immediate context of a deduction substitution locus.

[Example 8:

```
template <class T>
 auto f(T) -> decltype([]() { T::invalid; } ());
void f(...);
f(0); // error: invalid expression not part of the immediate context

template <class T, std::size_t = sizeof([]() { T::invalid; })>
 void g(T);
void g(...);
g(0); // error: invalid expression not part of the immediate context

template <class T>
 auto h(T) -> decltype([x = T::invalid]() { });
void h(...);
h(0); // OK, calls h(...)

template <class T>
 auto i(T) -> decltype([]() -> typename T::invalid { });
void i(...);
i(0); // OK, calls i(...)

template <class T>
 auto j(T t) -> decltype([](auto x) -> decltype(x.invalid) { } (t));
void j(...);
j(0); // OK, calls j(...)
```

— end example]

— end note]

- <sup>10</sup> [Example 9:

```
struct X { };
struct Y {
 Y(X) {}
};

template <class T> auto f(T t1, T t2) -> decltype(t1 + t2); // #1
X f(Y, Y); // #2

X x1, x2;
X x3 = f(x1, x2); // deduction fails on #1 (cannot add X+X), calls #2
```

— end example]

11 [Note 9: Type deduction can fail for the following reasons:

- (11.1) — Attempting to instantiate a pack expansion containing multiple packs of differing lengths.
- (11.2) — Attempting to create an array with an element type that is `void`, a function type, or a reference type, or attempting to create an array with a size that is zero or negative.

[Example 10:

```
template <class T> int f(T[5]);
int I = f<int>(0);
int j = f<void>(0); // invalid array
```

— end example]

- (11.3) — Attempting to use a type that is not a class or enumeration type in a qualified name.

[Example 11:

```
template <class T> int f(typename T::B*);
int i = f<int>(0);
```

— end example]

- (11.4) — Attempting to use a type in a *nested-name-specifier* of a *qualified-id* when that type does not contain the specified member, or
  - (11.4.1) — the specified member is not a type where a type is required, or
  - (11.4.2) — the specified member is not a template where a template is required, or
  - (11.4.3) — the specified member is not a non-type, non-template where a non-type, non-template is required.

[Example 12:

```
template <int I> struct X { };
template <template <class T> class> struct Z { };
template <class T> void f(typename T::Y*) {}
template <class T> void g(X<T::N>*) {}
template <class T> void h(Z<T::TT>*) {}
struct A {};
struct B { int Y; };
struct C {
 typedef int N;
};
struct D {
 typedef int TT;
};

int main() {
 // Deduction fails in each of these cases:
 f<A>(0); // A does not contain a member Y
 f(0); // The Y member of B is not a type
 g<C>(0); // The N member of C is not a non-type, non-template name
 h<D>(0); // The TT member of D is not a template
}
```

— end example]

- (11.5) — Attempting to create a pointer to reference type.
- (11.6) — Attempting to create a reference to `void`.
- (11.7) — Attempting to create “pointer to member of T” when T is not a class type.

[Example 13:

```
template <class T> int f(int T::*);
int i = f<int>(0);
```

— end example]

- (11.8) — Attempting to give an invalid type to a constant template parameter.

[Example 14:

```
template <class T, T> struct S {};
template <class T> int f(S<T, T{}>*); // #1
```

```

class X {
 int m;
};
int i0 = f<X>(0); // #1 uses a value of non-structural type X as a constant template argument
— end example]

```

- (11.9) — Attempting to perform an invalid conversion in either a template argument expression, or an expression used in the function declaration.

[Example 15:

```

template <class T, T*> int f(int);
int i2 = f<int,1>(0); // can't convert 1 to int*
— end example]

```

- (11.10) — Attempting to create a function type in which a parameter has a type of `void`, or in which the return type is a function type or array type.

- (11.11) — Attempting to give to an explicit object parameter of a lambda's function call operator a type not permitted for such (7.5.6.2).

— end note]

- 12 [Example 16: In the following example, assuming a `signed char` cannot represent the value 1000, a narrowing conversion (9.5.5) would be required to convert the *template-argument* of type `int` to `signed char`, therefore substitution fails for the second template (13.4.3).

```

template <int> int f(int);
template <signed char> int f(int);
int i1 = f<1000>(0); // OK
int i2 = f<1>(0); // ambiguous; not narrowing
— end example]

```

### 13.10.3.2 Deducing template arguments from a function call

[temp.deduct.call]

- <sup>1</sup> Template argument deduction is done by comparing each function template parameter type (call it *P*) that contains template parameters that participate in template argument deduction with the type of the corresponding argument of the call (call it *A*) as described below. If removing references and cv-qualifiers from *P* gives `std::initializer_list<P'>` or `P'[N]` for some *P'* and *N* and the argument is a non-empty initializer list (9.5.5), then deduction is instead performed as if each element of the initializer list were the argument for a separate parameter having type *P'*. In the `P'[N]` case, if *N* is a constant template parameter, *N* is deduced from the length of the initializer list. Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (13.10.3.6).

[Example 1:

```

template<class T> void f(std::initializer_list<T>);
f({1,2,3}); // T deduced as int
f({1,"asdf"}); // error: T deduced as both int and const char*

template<class T> void g(T);
g({1,2,3}); // error: no argument deduced for T

template<class T, int N> void h(T const(&)[N]);
h({1,2,3}); // T deduced as int; N deduced as 3

template<class T> void j(T const(&)[3]);
j({42}); // T deduced as int; array bound not considered

struct Aggr { int i; int j; };
template<int N> void k(Aggr const(&)[N]);
k({1,2,3}); // error: deduction fails, no conversion from int to Aggr
k({{1},{2},{3}}); // OK, N deduced as 3

template<int M, int N> void m(int const(&)[M][N]);
m({{1,2},{3,4}}); // M and N both deduced as 2

template<class T, int N> void n(T const(&)[N], T);
n({{1},{2},{3}},Aggr()); // OK, T is Aggr, N is 3

```

```

template<typename T, int N> void o(T (* const (&)[N])(T)) { }
int f1(int);
int f4(int);
char f4(char);
o({ &f1, &f4 }); // OK, T deduced as int from first element, nothing
// deduced from second element, N deduced as 2
o({ &f1, static_cast<char*>(char)>(&f4) }); // error: conflicting deductions for T
— end example]

```

For a function parameter pack that occurs at the end of the *parameter-declaration-list*, deduction is performed for each remaining argument of the call, taking the type *P* of the *declarator-id* of the function parameter pack as the corresponding function template parameter type. Each deduction deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. When a function parameter pack appears in a non-deduced context (13.10.3.6), the type of that pack is never deduced.

[Example 2:

```

template<class ... Types> void f(Types& ...);
template<class T1, class ... Types> void g(T1, Types ...);
template<class T1, class ... Types> void g1(Types ..., T1);

void h(int x, float& y) {
 const int z = x;
 f(x, y, z); // Types deduced as int, float, const int
 g(x, y, z); // T1 deduced as int; Types deduced as float, int
 g1(x, y, z); // error: Types is not deduced
 g1<int, int, int>(x, y, z); // OK, no deduction occurs
}

```

— end example]

2 If *P* is not a reference type:

- (2.1) — If *A* is an array type, the pointer type produced by the array-to-pointer standard conversion (7.3.3) is used in place of *A* for type deduction; otherwise,
- (2.2) — If *A* is a function type, the pointer type produced by the function-to-pointer standard conversion (7.3.4) is used in place of *A* for type deduction; otherwise,
- (2.3) — If *A* is a cv-qualified type, the top-level cv-qualifiers of *A*'s type are ignored for type deduction.

3 If *P* is a cv-qualified type, the top-level cv-qualifiers of *P*'s type are ignored for type deduction. If *P* is a reference type, the type referred to by *P* is used for type deduction.

[Example 3:

```

template<class T> int f(const T&);
int n1 = f(5); // calls f<int>(const int&)
const int i = 0;
int n2 = f(i); // calls f<int>(const int&)
template <class T> int g(volatile T&);
int n3 = g(i); // calls g<const int>(const volatile int&)

```

— end example]

A *forwarding reference* is an rvalue reference to a cv-unqualified template parameter that does not represent a template parameter of a class template (during class template argument deduction (12.2.2.9)). If *P* is a forwarding reference and the argument is an lvalue, the type “lvalue reference to *A*” is used in place of *A* for type deduction.

[Example 4:

```

template <class T> int f(T&& heisenreference);
template <class T> int g(const T&&);
int i;
int n1 = f(i); // calls f<int&>(int&)
int n2 = f(0); // calls f<int>(int&&)
int n3 = g(i); // error: would call g<int>(const int&&), which
// would bind an rvalue reference to an lvalue

```



```

template <class T> struct A {
 template <class U>
 A(T&&, U&&, int*); // #1: T&& is not a forwarding reference.
 // U&& is a forwarding reference.
 A(T&&, int*); // #2
};

template <class T> A(T&&, int*) -> A<T>; // #3: T&& is a forwarding reference.

int *ip;
A a{i, 0, ip}; // error: cannot deduce from #1
A a0{0, 0, ip}; // uses #1 to deduce A<int> and #1 to initialize
A a2{i, ip}; // uses #3 to deduce A<int&&> and #2 to initialize
— end example]

```

4 In general, the deduction process attempts to find template argument values that will make the deduced A identical to A (after the type A is transformed as described above). However, there are three cases that allow a difference:

- (4.1) — If the original P is a reference type, the deduced A (i.e., the type referred to by the reference) can be more cv-qualified than the transformed A.
- (4.2) — The transformed A can be another pointer or pointer-to-member type that can be converted to the deduced A via a function pointer conversion (7.3.14) and/or qualification conversion (7.3.6).
- (4.3) — If P is a class and P has the form *simple-template-id* or `typenameopt splice-specialization-specifier`, then the transformed A can be a derived class D of the deduced A. Likewise, if P is a pointer to a class of the form *simple-template-id* or `typenameopt splice-specialization-specifier`, the transformed A can be a pointer to a derived class D of the class pointed to by the deduced A. However, if there is a class C that is a (direct or indirect) base class of D and derived (directly or indirectly) from a class B and that would be a valid deduced A, the deduced A cannot be B or pointer to B, respectively.

[Example 5:

```

template <typename... T> struct X;
template <> struct X<> {};
template <typename T, typename... Ts>
 struct X<T, Ts...> : X<Ts...> {};
struct D : X<int> {};
struct E : X<>, X<int> {};

template <typename... T>
int f(const X<T...>&);
int x = f(D()); // calls f<int>, not f<>
 // B is X<>, C is X<int>
int z = f(E()); // calls f<int>, not f<>

```

— end example]

5 These alternatives are considered only if type deduction would otherwise fail. If they yield more than one possible deduced A, the type deduction fails.

[Note 1: If a template parameter is not used in any of the function parameters of a function template, or is used only in a non-deduced context, its corresponding *template-argument* cannot be deduced from a function call and the *template-argument* must be explicitly specified. — end note]

6 When P is a function type, function pointer type, or pointer-to-member-function type:

- (6.1) — If the argument is an overload set containing one or more function templates, the parameter is treated as a non-deduced context.
- (6.2) — If the argument is an overload set (not containing function templates), trial argument deduction is attempted using each of the members of the set whose associated constraints (13.5.2) are satisfied. If all successful deductions yield the same deduced A, that deduced A is the result of deduction; otherwise, the parameter is treated as a non-deduced context.

7 [Example 6:

```

// Only one function of an overload set matches the call so the function parameter is a deduced context.
template <class T> int f(T (*p)(T));

```

```
int g(int);
int g(char);
int i = f(g); // calls f(int (*)(int))
```

— end example]

8 [Example 7:

*// Ambiguous deduction causes the second function parameter to be a non-deduced context.*

```
template <class T> int f(T, T (*)(T));
int g(int);
char g(char);
int i = f(1, g); // calls f(int, int (*)(int))
```

— end example]

9 [Example 8:

*// The overload set contains a template, causing the second function parameter to be a non-deduced context.*

```
template <class T> int f(T, T (*)(T));
char g(char);
template <class T> T g(T);
int i = f(1, g); // calls f(int, int (*)(int))
```

— end example]

10 [Example 9:

*// All arguments for placeholder type deduction (9.2.9.7.2) yield the same deduced type.*

```
template<bool B> struct X {
 static void f(short) requires B; // #1
 static void f(short); // #2
};
void test() {
 auto x = &X<true>::f; // OK, deduces void (*)(short), selects #1
 auto y = &X<false>::f; // OK, deduces void (*)(short), selects #2
}
```

— end example]

### 13.10.3.3 Deducing template arguments taking the address of a function template [temp.deduct.funcaddr]

- 1 Template arguments can be deduced from the type specified when taking the address of an overload set (12.3). If there is a target, the function template's function type and the target type are used as the types of P and A, and the deduction is done as described in 13.10.3.6. Otherwise, deduction is performed with empty sets of types P and A.
- 2 A placeholder type (9.2.9.7) in the return type of a function template is a non-deduced context. If template argument deduction succeeds for such a function, the return type is determined from instantiation of the function body.

### 13.10.3.4 Deducing conversion function template arguments [temp.deduct.conv]

- 1 Template argument deduction is done by comparing the return type of the conversion function template (call it P) with the type specified by the *conversion-type-id* of the *conversion-function-id* being looked up (call it A), as described in 13.10.3.6. If the *conversion-function-id* is constructed during overload resolution (12.2.2), the rules in the remainder of this subclause apply.
- 2 If P is a reference type, the type referred to by P is used in place of P for type deduction and for any further references to or transformations of P in the remainder of this subclause.
- 3 If A is not a reference type:
  - (3.1) — If P is an array type, the pointer type produced by the array-to-pointer standard conversion (7.3.3) is used in place of P for type deduction; otherwise,
  - (3.2) — If P is a function type, the pointer type produced by the function-to-pointer standard conversion (7.3.4) is used in place of P for type deduction; otherwise,
  - (3.3) — If P is a cv-qualified type, the top-level cv-qualifiers of P's type are ignored for type deduction.
- 4 If A is a cv-qualified type, the top-level cv-qualifiers of A's type are ignored for type deduction. If A is a reference type, the type referred to by A is used for type deduction.

<sup>5</sup> In general, the deduction process attempts to find template argument values that will make the deduced **A** identical to **A**. However, certain attributes of **A** may be ignored:

- (5.1) — If the original **A** is a reference type, any cv-qualifiers of **A** (i.e., the type referred to by the reference).
- (5.2) — If the original **A** is a function pointer or pointer-to-member-function type with a potentially-throwing exception specification (14.5), the exception specification.
- (5.3) — Any cv-qualifiers in **A** that can be restored by a qualification conversion.

These attributes are ignored only if type deduction would otherwise fail. If ignoring them allows more than one possible deduced **A**, the type deduction fails.

### 13.10.3.5 Deducing template arguments during partial ordering [temp.deduct.partial]

<sup>1</sup> Template argument deduction is done by comparing certain types associated with the two function templates being compared.

<sup>2</sup> Two sets of types are used to determine the partial ordering. For each of the templates involved there is the original function type and the transformed function type.

[*Note 1*: The creation of the transformed type is described in 13.7.7.3. — *end note*]

The deduction process uses the transformed type as the argument template and the original type of the other template as the parameter template. This process is done twice for each type involved in the partial ordering comparison: once using the transformed template-1 as the argument template and template-2 as the parameter template and again using the transformed template-2 as the argument template and template-1 as the parameter template.

<sup>3</sup> The types used to determine the ordering depend on the context in which the partial ordering is done:

- (3.1) — In the context of a function call, the types used are those function parameter types for which the function call has arguments.<sup>112</sup>
- (3.2) — In the context of a call to a conversion function, the return types of the conversion function templates are used.
- (3.3) — In other contexts (13.7.7.3) the function template's function type is used.

<sup>4</sup> Each type nominated above from the parameter template and the corresponding type from the argument template are used as the types of **P** and **A**.

<sup>5</sup> Before the partial ordering is done, certain transformations are performed on the types used for partial ordering:

- (5.1) — If **P** is a reference type, **P** is replaced by the type referred to.
- (5.2) — If **A** is a reference type, **A** is replaced by the type referred to.

<sup>6</sup> If both **P** and **A** were reference types (before being replaced with the type referred to above), determine which of the two types (if any) is more cv-qualified than the other; otherwise the types are considered to be equally cv-qualified for partial ordering purposes. The result of this determination will be used below.

<sup>7</sup> Remove any top-level cv-qualifiers:

- (7.1) — If **P** is a cv-qualified type, **P** is replaced by the cv-unqualified version of **P**.
- (7.2) — If **A** is a cv-qualified type, **A** is replaced by the cv-unqualified version of **A**.

<sup>8</sup> If **A** was transformed from a function parameter pack and **P** is not a parameter pack, type deduction fails. Otherwise, using the resulting types **P** and **A**, the deduction is then done as described in 13.10.3.6. If **P** is a function parameter pack, the type **A** of each remaining parameter type of the argument template is compared with the type **P** of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. If deduction succeeds for a given type, the type from the argument template is considered to be at least as specialized as the type from the parameter template.

[*Example 1*:

```
template<class... Args> void f(Args... args); // #1
template<class T1, class... Args> void f(T1 a1, Args... args); // #2
template<class T1, class T2> void f(T1 a1, T2 a2); // #3
```

<sup>112</sup>) Default arguments are not considered to be arguments in this context; they only become arguments after a function has been selected.

```

f(); // calls #1
f(1, 2, 3); // calls #2
f(1, 2); // calls #3; non-variadic template #3 is more specialized
 // than the variadic templates #1 and #2

```

— end example]

- <sup>9</sup> If, for a given type, the types are identical after the transformations above and both **P** and **A** were reference types (before being replaced with the type referred to above):
- (9.1) — if the type from the argument template was an lvalue reference and the type from the parameter template was not, the parameter type is not considered to be at least as specialized as the argument type; otherwise,
- (9.2) — if the type from the argument template is more cv-qualified than the type from the parameter template (as described above), the parameter type is not considered to be at least as specialized as the argument type.
- <sup>10</sup> Function template **F** is *at least as specialized as* function template **G** if, for each pair of types used to determine the ordering, the type from **F** is at least as specialized as the type from **G**. **F** is *more specialized than* **G** if **F** is at least as specialized as **G** and **G** is not at least as specialized as **F**.
- <sup>11</sup> If, after considering the above, function template **F** is at least as specialized as function template **G** and vice-versa, and if **G** has a trailing function parameter pack for which **F** does not have a corresponding parameter, and if **F** does not have a trailing function parameter pack, then **F** is more specialized than **G**.
- <sup>12</sup> In most cases, deduction fails if not all template parameters have values, but for partial ordering purposes a template parameter may remain without a value provided it is not used in the types being used for partial ordering.

[Note 2: A template parameter used in a non-deduced context is considered used. — end note]

[Example 2:

```

template <class T> T f(int); // #1
template <class T, class U> T f(U); // #2
void g() {
 f<int>(1); // calls #1
}

```

— end example]

- <sup>13</sup> [Note 3: Partial ordering of function templates containing template parameter packs is independent of the number of deduced arguments for those template parameter packs. — end note]

[Example 3:

```

template<class ...> struct Tuple { };
template<class ... Types> void g(Tuple<Types ...>); // #1
template<class T1, class ... Types> void g(Tuple<T1, Types ...>); // #2
template<class T1, class ... Types> void g(Tuple<T1, Types& ...>); // #3

g(Tuple<>()); // calls #1
g(Tuple<int, float>()); // calls #2
g(Tuple<int, float&>()); // calls #3
g(Tuple<int>()); // calls #3

```

— end example]

### 13.10.3.6 Deducing template arguments from a type

[temp.deduct.type]

- <sup>1</sup> Template arguments can be deduced in several different contexts, but in each case a type that is specified in terms of template parameters (call it **P**) is compared with an actual type (call it **A**), and an attempt is made to find template argument values (a type for a type parameter, a value for a constant template parameter, or a template for a template template parameter) that will make **P**, after substitution of the deduced values (call it the deduced **A**), compatible with **A**.
- <sup>2</sup> In some cases, the deduction is done using a single set of types **P** and **A**, in other cases, there will be a set of corresponding types **P** and **A**. Type deduction is done independently for each **P/A** pair, and the deduced template argument values are then combined. If type deduction cannot be done for any **P/A** pair, or if for any pair the deduction leads to more than one possible set of deduced values, or if different pairs yield different deduced values, or if any template argument remains neither deduced nor explicitly specified, template

argument deduction fails. The type of a type parameter is only deduced from an array bound if it is not otherwise deduced.

- <sup>3</sup> A given type *P* can be composed from a number of other types, templates, and constant template argument values:

- (3.1) — A function type includes the types of each of the function parameters, the return type, and its exception specification.
- (3.2) — A pointer-to-member type includes the type of the class object pointed to and the type of the member pointed to.
- (3.3) — A type that is a specialization of a class template (e.g., `A<int>`) includes the types, templates, and constant template argument values referenced by the template argument list of the specialization.
- (3.4) — An array type includes the array element type and the value of the array bound.

- <sup>4</sup> In most cases, the types, templates, and constant template argument values that are used to compose *P* participate in template argument deduction. That is, they may be used to determine the value of a template argument, and template argument deduction fails if the value so determined is not consistent with the values determined elsewhere. In certain contexts, however, the value does not participate in type deduction, but instead uses the values of template arguments that were either deduced elsewhere or explicitly specified. If a template parameter is used only in non-deduced contexts and is not explicitly specified, template argument deduction fails.

[*Note 1:* Under 13.10.3.2, if *P* contains no template parameters that appear in deduced contexts, no deduction is done, so *P* and *A* need not have the same form. — *end note*]

- <sup>5</sup> The non-deduced contexts are:

- (5.1) — The *nested-name-specifier* of a type or template that was specified using a *qualified-id*.
- (5.2) — A *pack-index-specifier* or a *pack-index-expression*.
- (5.3) — A *type-constraint*.
- (5.4) — The *expression* of a *decltype-specifier*.
- (5.5) — The *constant-expression* of a *splice-specifier*.
- (5.6) — A constant template argument or an array bound in which a subexpression references a template parameter.
- (5.7) — A template parameter used in the parameter type of a function parameter that has a default argument that is being used in the call for which argument deduction is being done.
- (5.8) — A function parameter for which the associated argument is an overload set such that one or more of the following apply:
  - (5.8.1) — functions whose associated constraints are satisfied and that do not all have the same function type match the function parameter type (resulting in an ambiguous deduction), or
  - (5.8.2) — no function whose associated constraints are satisfied matches the function parameter type, or
  - (5.8.3) — the overload set supplied as an argument contains one or more function templates.

[*Note 2:* A particular function from the overload set is selected (12.3) after template argument deduction has succeeded (13.10.4). — *end note*]

- (5.9) — A function parameter for which the associated argument is an initializer list (9.5.5) but the parameter does not have a type for which deduction from an initializer list is specified (13.10.3.2).

[*Example 1:*

```
template<class T> void g(T);
g({1,2,3}); // error: no argument deduced for T
— end example]
```

- (5.10) — A function parameter pack that does not occur at the end of the *parameter-declaration-list*.

- <sup>6</sup> When a type name is specified in a way that includes a non-deduced context, all of the types that comprise that type name are also non-deduced. However, a compound type can include both deduced and non-deduced types.

[*Example 2:* If a type is specified as `A<T>::B<T2>`, both `T` and `T2` are non-deduced. Likewise, if a type is specified as `A<I+J>::X<T>`, `I`, `J`, and `T` are non-deduced. If a type is specified as `void f(typename A<T>::B, A<T>)`, the `T` in `A<T>::B` is non-deduced but the `T` in `A<T>` is deduced. — *end example*]

- 7 [*Example 3:* Here is an example in which different parameter/argument pairs produce inconsistent template argument deductions:

```
template<class T> void f(T x, T y) { /* ... */ }
struct A { /* ... */ };
struct B : A { /* ... */ };
void g(A a, B b) {
 f(a,b); // error: T deduced as both A and B
 f(b,a); // error: T deduced as both A and B
 f(a,a); // OK, T is A
 f(b,b); // OK, T is B
}
```

Here is an example where two template arguments are deduced from a single function parameter/argument pair. This can lead to conflicts that cause type deduction to fail:

```
template <class T, class U> void f(T (*)(T, U, U));

int g1(int, float, float);
char g2(int, float, float);
int g3(int, char, float);

void r() {
 f(g1); // OK, T is int and U is float
 f(g2); // error: T deduced as both char and int
 f(g3); // error: U deduced as both char and float
}
```

Here is an example where the exception specification of a function type is deduced:

```
template<bool E> void f1(void (*)() noexcept(E));
template<bool> struct A { };
template<bool B> void f2(void (*) (A) noexcept(B));

void g1();
void g2() noexcept;
void g3(A<true>);

void h() {
 f1(g1); // OK, E is false
 f1(g2); // OK, E is true
 f2(g3); // error: B deduced as both true and false
}
```

Here is an example where a qualification conversion applies between the argument type on the function call and the deduced template argument type:

```
template<class T> void f(const T*) { }
int* p;
void s() {
 f(p); // f(const int*)
}
```

Here is an example where the template argument is used to instantiate a derived class type of the corresponding function parameter type:

```
template <class T> struct B { };
template <class T> struct D : public B<T> {};
struct D2 : public B<int> {};
template <class T> void f(B<T>&) {}
void t() {
 D<int> d;
 D2 d2;
 f(d); // calls f(B<int>&)
 f(d2); // calls f(B<int>&)
}
```

— *end example*]

- <sup>8</sup> A type template argument *T*, a constant template argument *i*, a template template argument *TT* denoting a class template or an alias template, or a template template argument *VV* denoting a variable template or a concept can be deduced if *P* and *A* have one of the following forms:

```
cvopt T
T*
T&
T&&
Topt[iopt]
Topt(Topt) noexcept(iopt)
Topt Topt::*
TTopt<T>
TTopt<i>
TTopt<TT>
TTopt<VV>
TTopt<>
```

where

- (8.1) — *T<sub>opt</sub>* represents a type or parameter-type-list that either satisfies these rules recursively, is a non-deduced context in *P* or *A*, or is the same non-dependent type in *P* and *A*,
- (8.2) — *i<sub>opt</sub>* represents an expression that either is an *i*, is value-dependent in *P* or *A*, or has the same constant value in *P* and *A*,
- (8.3) — *TT<sub>opt</sub>* represents either a class template or a template template parameter, and
- (8.4) — *noexcept(i<sub>opt</sub>)* represents an exception specification (14.5) in which the (possibly-implicit, see 9.3.4.6) *noexcept-specifier*'s operand satisfies the rules for an *i<sub>opt</sub>* above.

[*Note 3*: If a type matches such a form but contains no *T*s, *i*s, or *TT*s, deduction is not possible. — *end note*]

Similarly, *<X>* represents template argument lists where at least one argument contains an *X*, where *X* is one of *T*, *i*, *TT*, or *VV*; and *<>* represents template argument lists where no argument contains a *T*, an *i*, a *TT*, or a *VV*.

- <sup>9</sup> If *P* has a form that contains *<T>*, *<i>*, *<TT>*, or *<VV>*, then each argument *P<sub>i</sub>* of the respective template argument list of *P* is compared with the corresponding argument *A<sub>i</sub>* of the corresponding template argument list of *A*. If the template argument list of *P* contains a pack expansion that is not the last template argument, the entire template argument list is a non-deduced context. If *P<sub>i</sub>* is a pack expansion, then the pattern of *P<sub>i</sub>* is compared with each remaining argument in the template argument list of *A*. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by *P<sub>i</sub>*. During partial ordering (13.10.3.5), if *A<sub>i</sub>* was originally a pack expansion:

- (9.1) — if *P* does not contain a template argument corresponding to *A<sub>i</sub>* then *A<sub>i</sub>* is ignored;
- (9.2) — otherwise, if *P<sub>i</sub>* is not a pack expansion, template argument deduction fails.

[*Example 4*:

```
template<class T1, class... Z> class S; // #1
template<class T1, class... Z> class S<T1, const Z&...> { }; // #2
template<class T1, class T2> class S<T1, const T2&> { }; // #3
S<int, const int&> s; // both #2 and #3 match; #3 is more specialized

template<class T, class... U> struct A { }; // #1
template<class T1, class T2, class... U> struct A<T1, T2*, U...> { }; // #2
template<class T1, class T2> struct A<T1, T2> { }; // #3
template struct A<int, int*> // selects #2
```

— *end example*]

- <sup>10</sup> Similarly, if *P* has a form that contains *(T)*, then each parameter type *P<sub>i</sub>* of the respective parameter-type-list (9.3.4.6) of *P* is compared with the corresponding parameter type *A<sub>i</sub>* of the corresponding parameter-type-list of *A*. If *P* and *A* are function types that originated from deduction when taking the address of a function template (13.10.3.3) or when deducing template arguments from a function declaration (13.10.3.7) and *P<sub>i</sub>* and *A<sub>i</sub>* are parameters of the top-level parameter-type-list of *P* and *A*, respectively, *P<sub>i</sub>* is adjusted if it is a forwarding reference (13.10.3.2) and *A<sub>i</sub>* is an lvalue reference, in which case the type of *P<sub>i</sub>* is changed to be the template parameter type (i.e., *T&&* is changed to simply *T*).

[Note 4: As a result, when  $P_i$  is  $T\&\&$  and  $A_i$  is  $X\&$ , the adjusted  $P_i$  will be  $T$ , causing  $T$  to be deduced as  $X\&$ . — end note]

[Example 5:

```
template <class T> void f(T&&);
template <> void f(int&) { } // #1
template <> void f(int&&) { } // #2
void g(int i) {
 f(i); // calls f<int&>(int&), i.e., #1
 f(0); // calls f<int>(int&&), i.e., #2
}
```

— end example]

If the *parameter-declaration* corresponding to  $P_i$  is a function parameter pack, then the type of its *declarator-id* is compared with each remaining parameter type in the parameter-type-list of  $A$ . Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. During partial ordering (13.10.3.5), if  $A_i$  was originally a function parameter pack:

- (10.1) — if  $P$  does not contain a function parameter type corresponding to  $A_i$  then  $A_i$  is ignored;
- (10.2) — otherwise, if  $P_i$  is not a function parameter pack, template argument deduction fails.

[Example 6:

```
template<class T, class... U> void f(T*, U...) { } // #1
template<class T> void f(T) { } // #2
template void f(int*); // selects #1
```

— end example]

- 11 These forms can be used in the same way as  $T$  is for further composition of types.

[Example 7:

```
X<int> (*) (char[6])
```

is of the form

```
template-name<T> (*) (type [i])
```

which is a variant of

```
type (*) (T)
```

where  $type$  is  $X<int>$  and  $T$  is  $char[6]$ . — end example]

- 12 Template arguments cannot be deduced from function arguments involving constructs other than the ones specified above.
- 13 When the value of the argument corresponding to a constant template parameter  $P$  that is declared with a dependent type is deduced from an expression, the template parameters in the type of  $P$  are deduced from the type of the value.

[Example 8:

```
template<long n> struct A { };

template<typename T> struct C;
template<typename T, T n> struct C<A<n>> {
 using Q = T;
};

using R = long;
using R = C<A<2>>::Q; // OK; T was deduced as long from the
// template argument value in the type A<2>
```

— end example]

- 14 The type of  $N$  in the type  $T[N]$  is `std::size_t`.

[Example 9:

```
template<typename T> struct S;
template<typename T, T n> struct S<int[n]> {
 using Q = T;
};
```



```
using V = decltype(sizeof 0);
using V = S<int[42]>::Q; // OK; T was deduced as std::size_t from the type int[42]
```

— end example]

- 15 The type of B in the *noexcept-specifier* `noexcept(B)` of a function type is `bool`.

[Example 10:

```
template<bool> struct A { };
template<auto> struct B;
template<auto X, void (*F)() noexcept(X)> struct B<F> {
 A<X> ax;
};
void f_nothrow() noexcept;
B<f_nothrow> bn; // OK, type of X deduced as bool
```

— end example]

- 16 [Example 11:

```
template<class T, T i> void f(int (&a)[i]);
int v[10];
void g() {
 f(v); // OK, T is std::size_t
}
```

— end example]

- 17 [Note 5: Except for reference and pointer types, a major array bound is not part of a function parameter type and cannot be deduced from an argument:

```
template<int i> void f1(int a[10][i]);
template<int i> void f2(int a[i][20]);
template<int i> void f3(int (&a)[i][20]);

void g() {
 int v[10][20];
 f1(v); // OK, i deduced as 20
 f1<20>(v); // OK
 f2(v); // error: cannot deduce template-argument i
 f2<10>(v); // OK
 f3(v); // OK, i deduced as 10
}
```

— end note]

- 18 [Note 6: If, in the declaration of a function template with a constant template parameter, the constant template parameter is used in a subexpression in the function parameter list, the expression is a non-deduced context as specified above.

[Example 12:

```
template <int i> class A { /* ... */ };
template <int i> void g(A<i+1>);
template <int i> void f(A<i>, A<i+1>);
void k() {
 A<1> a1;
 A<2> a2;
 g(a1); // error: deduction fails for expression i+1
 g<0>(a1); // OK
 f(a1, a2); // OK
}
```

— end example]

— end note]

- 19 [Note 7: Template parameters do not participate in template argument deduction if they are used only in non-deduced contexts. For example,

```
template<int i, typename T>
T deduce(typename A<T>::X x, // T is not deduced here
 T t, // but T is deduced here
 typename B<i>::Y y); // i is not deduced here
```

```

A<int> a;
B<77> b;

int x = deduce<77>(a.xm, 62, b.ym);
// T deduced as int; a.xm must be convertible to A<int>::X
// i is explicitly specified to be 77; b.ym must be convertible to B<77>::Y
— end note]

```

- <sup>20</sup> If P has a form that contains <i>, deduction fails unless the type of i is the same as that of the corresponding template parameter p in the specialization (from A) of the template named by the enclosing *simple-template-id* or *splice-specialization-specifier*; if the declared type of i contains a placeholder type, the corresponding template argument for the purposes of placeholder type deduction (9.2.9.7.2) is an *id-expression* for p. If P has a form that contains [i], and if the type of i is not an integral type, deduction fails.<sup>113</sup> If P has a form that includes noexcept(i) and the type of i is not bool, deduction fails.

[Example 13:

```

template<int i> class A { /* ... */ };
template<short s> void f(A<s>);
void k1() {
 A<1> a;
 f(a); // error: deduction fails for conversion from int to short
 f<1>(a); // OK
}

template<const short cs> class B { };
template<short s> void g(B<s>);
void k2() {
 B<1> b;
 g(b); // OK, cv-qualifiers are ignored on template parameter types
}

template<auto> struct C;
template<long long x> void f(C<x> *);
void g(C<0LL> *ap) {
 f(ap); // OK, deduces long long value from 0LL
}

template<int> struct D;
template<auto x> void f(D<x> *);
void g(D<0LL> *ap) {
 f(ap); // OK, deduces x as an int value
}

template<int &> struct E;
template<auto x> void f(E<x> *);
int v;
void g(E<v> *bp) {
 f(bp); // error: type int of x does not match the int & type
 // of the template parameter in the E<v> specialization of E
}

template<const int &> struct F;
template<decltype(auto) x> void f(F<x> *);
int i;
void g(F<i> *ap) {
 f(ap); // OK, deduces x as a constant template parameter of type const int &
}

template <decltype(auto) q> struct G;
template <auto x> long *f(G<x> *); // #1
template <decltype(auto) x> short *f(G<x> *); // #2

```

113) Although the *template-argument* corresponding to a template parameter of type bool can be deduced from an array bound, the resulting value will always be true because the array bound will be nonzero.

```
const int j = 0;
short *g(G<(j)> *ap) { // OK, q has type const int &
 return f(ap); // OK, only #2 matches
}
```

```
long *g(G<j> *ap) { // OK, q has type int
 return f(ap); // OK, #1 is more specialized
}
```

— end example]

- 21 A *template-argument* can be deduced from a function, pointer to function, or pointer-to-member-function type.

[Example 14:

```
template<class T> void f(void(*) (T,int));
template<class T> void foo(T,int);
void g(int,int);
void g(char,int);

void h(int,int,int);
void h(char,int);
int m() {
 f(&g); // error: ambiguous
 f(&h); // OK, void h(char,int) is a unique match
 f(&foo); // error: type deduction fails because foo is a template
}
```

— end example]

- 22 A *template type-parameter* cannot be deduced from the type of a function default argument.

[Example 15:

```
template <class T> void f(T = 5, T = 7);
void g() {
 f(1); // OK, calls f<int>(1,7)
 f(); // error: cannot deduce T
 f<int>(); // OK, calls f<int>(5,7)
}
```

— end example]

- 23 The *template-argument* corresponding to a template template parameter is deduced from the type of the *template-argument* of a class template specialization used in the argument list of a function call.

[Example 16:

```
template <template <class T> class X> struct A { };
template <template <class T> class X> void f(A<X>) { }
template<class T> struct B { };
A ab;
f(ab); // calls f(A)
```

— end example]

- 24 [Note 8: Template argument deduction involving parameter packs (13.7.4) can deduce zero or more arguments for each parameter pack. — end note]

[Example 17:

```
template<class> struct X { };
template<class R, class ... ArgTypes> struct X<R(int, ArgTypes ...)> { };
template<class ... Types> struct Y { };
template<class T, class ... Types> struct Y<T, Types& ...> { };

template<class ... Types> int f(void (*) (Types ...));
void g(int, float);

X<int> x1; // uses primary template
X<int(int, float, double)> x2; // uses partial specialization; ArgTypes contains float, double
X<int(float, int)> x3; // uses primary template
```

```

Y<> y1; // uses primary template; Types is empty
Y<int&, float&, double&> y2; // uses partial specialization; T is int&, Types contains float, double
Y<int, float, double> y3; // uses primary template; Types contains int, float, double
int fv = f(g); // OK; Types contains int, float
— end example]

```

### 13.10.3.7 Deducing template arguments from a function declaration [temp.deduct.decl]

- <sup>1</sup> In a declaration whose *declarator-id* refers to a specialization of a function template, template argument deduction is performed to identify the specialization to which the declaration refers. Specifically, this is done for explicit instantiations (13.9.3), explicit specializations (13.9.4), and certain friend declarations (13.7.5). This is also done to determine whether a deallocation function template specialization matches a placement allocation function (6.8.6.5.3, 7.6.2.8). In all these cases, P is the type of the function template being considered as a potential match and A is either the function type from the declaration or the type of the deallocation function that would match the placement allocation function as described in 7.6.2.8. The deduction is done as described in 13.10.3.6.
- <sup>2</sup> If, for the set of function templates so considered, there is either no match or more than one match after partial ordering has been considered (13.7.7.3), deduction fails and, in the declaration cases, the program is ill-formed.

### 13.10.4 Overload resolution [temp.over]

- <sup>1</sup> When a call of a function or function template is written (explicitly, or implicitly using the operator notation), template argument deduction (13.10.3) and checking of any explicit template arguments (13.4) are performed for each function template to find the template argument values (if any) that can be used with that function template to instantiate a function template specialization that can be invoked with the call arguments or, for conversion function templates, that can convert to the required type. For each function template:
  - (1.1) — If the argument deduction and checking succeeds, the *template-arguments* (deduced and/or explicit) are used to synthesize the declaration of a single function template specialization which is added to the candidate functions set to be used in overload resolution.
  - (1.2) — If the argument deduction fails or the synthesized function template specialization would be ill-formed, no such function is added to the set of candidate functions for that template.

The complete set of candidate functions includes all the synthesized declarations and all of the non-template functions found by name lookup. The synthesized declarations are treated like any other functions in the remainder of overload resolution, except as explicitly noted in 12.2.4.<sup>114</sup>

- <sup>2</sup> [Example 1:

```

template<class T> T max(T a, T b) { return a>b?a:b; }

void f(int a, int b, char c, char d) {
 int m1 = max(a,b); // max(int a, int b)
 char m2 = max(c,d); // max(char a, char b)
 int m3 = max(a,c); // error: cannot generate max(int,char)
}

```

Adding the non-template function

```
int max(int,int);
```

to the example above would resolve the third call, by providing a function that can be called for `max(a,c)` after using the standard conversion of `char` to `int` for `c`. — end example]

- <sup>3</sup> [Example 2: Here is an example involving conversions on a function argument involved in template argument deduction:

```

template<class T> struct B { /* ... */ };
template<class T> struct D : public B<T> { /* ... */ };
template<class T> void f(B<T>&);

```

<sup>114</sup>) The parameters of function template specializations contain no template parameter types. The set of conversions allowed on deduced arguments is limited, because the argument deduction process produces function templates with parameters that either match the call arguments exactly or differ only in ways that can be bridged by the allowed limited conversions. Non-deduced arguments allow the full range of conversions. Note also that 12.2.4 specifies that a non-template function will be given preference over a template specialization if the two functions are otherwise equally good candidates for an overload match.

```

void g(B<int>& bi, D<int>& di) {
 f(bi); // f(bi)
 f(di); // f((B<int>&)di)
}

```

— end example]

- <sup>4</sup> [Example 3: Here is an example involving conversions on a function argument not involved in template argument deduction:

```

template<class T> void f(T*,int); // #1
template<class T> void f(T,char); // #2

void h(int* pi, int i, char c) {
 f(pi,i); // #1: f<int>(pi,i)
 f(pi,c); // #2: f<int*>(pi,c)

 f(i,c); // #2: f<int>(i,c);
 f(i,i); // #2: f<int>(i,char(i))
}

```

— end example]

- <sup>5</sup> Only the signature of a function template specialization is needed to enter the specialization in a set of candidate functions. Therefore only the function template declaration is needed to resolve a call for which a template specialization is a candidate.

[Example 4:

```

template<class T> void f(T); // declaration

void g() {
 f("Annemarie"); // calls f<const char*>
}

```

The call to `f` is well-formed even if the template `f` is only declared and not defined at the point of the call. The program will be ill-formed unless a specialization for `f<const char*>` is explicitly instantiated in some translation unit (13.1). — end example]

# 14 Exception handling

[except]

## 14.1 Preamble

[except.pre]

- <sup>1</sup> Exception handling provides a way of transferring control and information from a point in the execution of a thread to an exception handler associated with a point previously passed by the execution. A handler will be invoked only by throwing an exception in code executed in the handler's try block or in functions called from the handler's try block.

```

try-block:
 try compound-statement handler-seq

function-try-block:
 try ctor-initializeropt compound-statement handler-seq

handler-seq:
 handler handler-seqopt

handler:
 catch (exception-declaration) compound-statement

exception-declaration:
 attribute-specifier-seqopt type-specifier-seq declarator
 attribute-specifier-seqopt type-specifier-seq abstract-declaratoropt
 ...

```

The optional *attribute-specifier-seq* in an *exception-declaration* appertains to the parameter of the catch clause (14.4).

- <sup>2</sup> A *try-block* is a *statement* (8.1).

[Note 1: Within this Clause “try block” is taken to mean both *try-block* and *function-try-block*. — end note]

- <sup>3</sup> The *compound-statement* of a try block or of a handler is a control-flow-limited statement (8.2).

[Example 1:

```

void f() {
 goto 11; // error
 goto 12; // error
 try {
 goto 11; // OK
 goto 12; // error
 11: ;
 } catch (...) {
 12: ;
 goto 11; // error
 goto 12; // OK
 }
}

```

— end example]

A **goto**, **break**, **return**, or **continue** statement can be used to transfer control out of a try block or handler. When this happens, each variable declared in the try block will be destroyed in the context that directly contains its declaration.

[Example 2:

```

lab: try {
 T1 t1;
 try {
 T2 t2;
 if (condition)
 goto lab;
 } catch(...) { /* handler 2 */ }
} catch(...) { /* handler 1 */ }

```

Here, executing `goto lab;` will destroy first `t2`, then `t1`, assuming the *condition* does not declare a variable. Any exception thrown while destroying `t2` will result in executing **handler 2**; any exception thrown while destroying `t1` will result in executing **handler 1**. — *end example*]

- <sup>4</sup> A *function-try-block* associates a *handler-seq* with the *ctor-initializer*, if present, and the *compound-statement*. An exception thrown during the execution of the *compound-statement* or, for constructors and destructors, during the initialization or destruction, respectively, of the class's subobjects, transfers control to a handler in a *function-try-block* in the same way as an exception thrown during the execution of a *try-block* transfers control to other handlers.

[*Example 3*:

```
int f(int);
class C {
 int i;
 double d;
public:
 C(int, double);
};

C::C(int ii, double id)
try : i(f(ii)), d(id) {
 // constructor statements
} catch (...) {
 // handles exceptions thrown from the ctor-initializer and from the constructor statements
}
```

— *end example*]

- <sup>5</sup> In this Clause, “before” and “after” refer to the “sequenced before” relation (6.10.1).

## 14.2 Throwing an exception

[**except.throw**]

- <sup>1</sup> Throwing an exception transfers control to a handler.

[*Note 1*: An exception can be thrown from one of the following contexts: *throw-expressions* (7.6.18), allocation functions (6.8.6.5.2), `dynamic_cast` (7.6.1.7), `typeid` (7.6.1.8), *new-expressions* (7.6.2.8), and standard library functions (16.3.2.4). — *end note*]

An object is passed and the type of that object determines which handlers can catch it.

[*Example 1*:

```
throw "Help!";
```

can be caught by a *handler* of `const char*` type:

```
try {
 // ...
} catch(const char* p) {
 // handle character string exceptions here
}
```

and

```
class Overflow {
public:
 Overflow(char,double,double);
};

void f(double x) {
 throw Overflow('+',x,3.45e107);
}
```

can be caught by a handler for exceptions of type `Overflow`:

```
try {
 f(1.2);
} catch(Overflow& oo) {
 // handle exceptions of type Overflow here
}
```

— *end example*]

- 2 When an exception is thrown, control is transferred to the nearest handler with a matching type (14.4); “nearest” means the handler for which the *compound-statement* or *ctor-initializer* following the **try** keyword was most recently entered by the thread of control and not yet exited.
- 3 Throwing an exception initializes an object with dynamic storage duration, called the *exception object*. If the type of the exception object would be an incomplete type (6.9.1), an abstract class type (11.7.4), or a pointer to an incomplete type other than *cv void* (6.9.5), the program is ill-formed.
- 4 The memory for the exception object is allocated in an unspecified way, except as noted in 6.8.6.5.2. If a handler exits by rethrowing, control is passed to another handler for the same exception object. The points of potential destruction for the exception object are:
  - (4.1) — when an active handler for the exception exits by any means other than rethrowing, immediately after the destruction of the object (if any) declared in the *exception-declaration* in the handler;
  - (4.2) — when an object of type `std::exception_ptr` (17.9.7) that refers to the exception object is destroyed, before the destructor of `std::exception_ptr` returns.

Among all points of potential destruction for the exception object, there is an unspecified last one where the exception object is destroyed. All other points happen before that last one (6.10.2.2).

[Note 2: No other thread synchronization is implied in exception handling. — end note]

The implementation may then deallocate the memory for the exception object; any such deallocation is done in an unspecified way.

[Note 3: A thrown exception does not propagate to other threads unless caught, stored, and rethrown using appropriate library functions; see 17.9.7 and 32.10. — end note]

- 5 Let *T* denote the type of the exception object. Copy-initialization of an object of type *T* from an lvalue of type `const T` in a context unrelated to *T* shall be well-formed. If *T* is a class type, the selected constructor is odr-used (6.3) and the destructor of *T* is potentially invoked (11.4.7).
- 6 An exception is considered *uncaught* after completing the initialization of the exception object until completing the activation of a handler for the exception (14.4).
 

[Note 4: As a consequence, an exception is considered uncaught during any stack unwinding resulting from it being thrown. — end note]
- 7 If an exception is rethrown (7.6.18, 17.9.7), it is considered uncaught from the point of rethrow until the rethrown exception is caught.
 

[Note 5: The function `std::uncaught_exceptions` (17.9.6) returns the number of uncaught exceptions in the current thread. — end note]
- 8 An exception is considered caught when a handler for that exception becomes active (14.4).
 

[Note 6: An exception can have active handlers and still be considered uncaught if it is rethrown. — end note]
- 9 If the exception handling mechanism handling an uncaught exception directly invokes a function that exits via an exception, the function `std::terminate` is invoked (14.6.2).

[Example 2:

```
struct C {
 C() { }
 C(const C&) {
 if (std::uncaught_exceptions()) {
 throw 0; // throw during copy to handler's exception-declaration object (14.4)
 }
 }
};
```

```
int main() {
 try {
 throw C(); // calls std::terminate if construction of the handler's
 // exception-declaration object is not elided (11.9.6)
 } catch(C) { }
}
```

— end example]

[Note 7: If a destructor directly invoked by stack unwinding exits via an exception, `std::terminate` is invoked. — end note]



### 14.3 Stack unwinding

[except.ctor]

- <sup>1</sup> As control passes from the point where an exception is thrown to a handler, objects are destroyed by a process, specified in this subclause, called *stack unwinding*.
- <sup>2</sup> Each object with automatic storage duration is destroyed if it has been constructed, but not yet destroyed, since the try block was entered. If an exception is thrown during the destruction of temporaries or local variables for a **return** statement (8.8.4), the destructor for the returned object (if any) is also invoked. The objects are destroyed in the reverse order of the completion of their construction.

[Example 1:

```

struct A { };

struct Y { ~Y() noexcept(false) { throw 0; } };

A f() {
 try {
 A a;
 Y y;
 A b;
 return {}; // #1
 } catch (...) {
 }
 return {}; // #2
}

```

At #1, the returned object of type **A** is constructed. Then, the local variable **b** is destroyed (8.8). Next, the local variable **y** is destroyed, causing stack unwinding, resulting in the destruction of the returned object, followed by the destruction of the local variable **a**. Finally, the returned object is constructed again at #2. — end example]

- <sup>3</sup> If the initialization of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's subobjects that were known to be initialized by the object's initialization and whose initialization has completed (9.5).

[Note 1: If such an object has a reference member that extends the lifetime of a temporary object, this ends the lifetime of the reference member, so the lifetime of the temporary object is effectively not extended. — end note]

A subobject is *known to be initialized* if it is not an anonymous union member and its initialization is specified

- (3.1) — in 11.9.3 for initialization by constructor,
- (3.2) — in 11.4.5.3 for initialization by defaulted copy/move constructor,
- (3.3) — in 11.9.4 for initialization by inherited constructor,
- (3.4) — in 9.5.2 for aggregate initialization,
- (3.5) — in 7.5.6.3 for the initialization of the closure object when evaluating a *lambda-expression*,
- (3.6) — in 9.5.1 for default-initialization, value-initialization, or direct-initialization of an array.

[Note 2: This includes virtual base class subobjects if the initialization is for a complete object, and can include variant members that were nominated explicitly by a *mem-initializer* or *designated-initializer-clause* or that have a default member initializer. — end note]

If the destructor of an object is terminated by an exception, each destructor invocation that would be performed after executing the body of the destructor (11.4.7) and that has not yet begun execution is performed.

[Note 3: This includes virtual base class subobjects if the destructor was invoked for a complete object. — end note]

The subobjects are destroyed in the reverse order of the completion of their construction. Such destruction is sequenced before entering a handler of the *function-try-block* of the constructor or destructor, if any.

- <sup>4</sup> If the *compound-statement* of the *function-body* of a delegating constructor for an object exits via an exception, the object's destructor is invoked. Such destruction is sequenced before entering a handler of the *function-try-block* of a delegating constructor for that object, if any.
- <sup>5</sup> [Note 4: If the object was allocated by a *new-expression* (7.6.2.8), the matching deallocation function (6.8.6.5.3), if any, is called to free the storage occupied by the object. — end note]

## 14.4 Handling an exception

[except.handle]

- <sup>1</sup> The *exception-declaration* in a *handler* describes the type(s) of exceptions that can cause that *handler* to be entered. The *exception-declaration* shall not denote an incomplete type, an abstract class type, or an rvalue reference type. The *exception-declaration* shall not denote a pointer or reference to an incomplete type, other than “pointer to *cv void*”.
- <sup>2</sup> A handler of type “array of *T*” or function type *T* is adjusted to be of type “pointer to *T*”.
- <sup>3</sup> A *handler* is a match for an exception object of type *E* if
  - (3.1) — The *handler* is of type *cv T* or *cv T&* and *E* and *T* are the same type (ignoring the top-level *cv-qualifiers*), or
  - (3.2) — the *handler* is of type *cv T* or *cv T&* and *T* is an unambiguous public base class of *E*, or
  - (3.3) — the *handler* is of type *cv T* or *const T&* where *T* is a pointer or pointer-to-member type and *E* is a pointer or pointer-to-member type that can be converted to *T* by one or more of
    - (3.3.1) — a standard pointer conversion (7.3.12) not involving conversions to pointers to private or protected or ambiguous classes,
    - (3.3.2) — a function pointer conversion (7.3.14),
    - (3.3.3) — a qualification conversion (7.3.6), or
  - (3.4) — the *handler* is of type *cv T* or *const T&* where *T* is a pointer or pointer-to-member type and *E* is `std::nullptr_t`.

[Note 1: A *throw-expression* whose operand is an integer literal with value zero does not match a handler of pointer or pointer-to-member type. A handler of reference to array or function type is never a match for any exception object (7.6.18). — end note]

[Example 1:

```
class Matherr { /* ... */ virtual void vf(); };
class Overflow: public Matherr { /* ... */ };
class Underflow: public Matherr { /* ... */ };
class Zerodivide: public Matherr { /* ... */ };

void f() {
 try {
 g();
 } catch (Overflow oo) {
 // ...
 } catch (Matherr mm) {
 // ...
 }
}
```

Here, the `Overflow` handler will catch exceptions of type `Overflow` and the `Matherr` handler will catch exceptions of type `Matherr` and of all types publicly derived from `Matherr` including exceptions of type `Underflow` and `Zerodivide`. — end example]

- <sup>4</sup> The handlers for a try block are tried in order of appearance.
 

[Note 2: This makes it possible to write handlers that can never be executed, for example by placing a handler for a final derived class after a handler for a corresponding unambiguous public base class. — end note]
- <sup>5</sup> A ... in a handler's *exception-declaration* specifies a match for any exception. If present, a ... handler shall be the last handler for its try block.
- <sup>6</sup> If no match is found among the handlers for a try block, the search for a matching handler continues in a dynamically surrounding try block of the same thread.
- <sup>7</sup> If the search for a handler exits the function body of a function with a non-throwing exception specification, the function `std::terminate` (14.6.2) is invoked.

[Note 3: An implementation is not permitted to reject an expression merely because, when executed, it throws or might throw an exception from a function with a non-throwing exception specification. — end note]

[Example 2:

```
extern void f(); // potentially-throwing
```

```

void g() noexcept {
 f(); // valid, even if f throws
 throw 42; // valid, effectively a call to std::terminate
}

```

The call to `f` is well-formed despite the possibility for it to throw an exception. — *end example*

- 8 If no matching handler is found, the function `std::terminate` is invoked; whether or not the stack is unwound before this invocation of `std::terminate` is implementation-defined (14.6.2).
- 9 A handler is considered *active* when initialization is complete for the parameter (if any) of the catch clause. [Note 4: The stack will have been unwound at that point. — *end note*  
Also, an implicit handler is considered active when the function `std::terminate` is entered due to a throw. A handler is no longer considered active when the catch clause exits.
- 10 The exception with the most recently activated handler that is still active is called the *currently handled exception*.
- 11 Referring to any non-static member or base class of an object in the handler for a *function-try-block* of a constructor or destructor for that object results in undefined behavior (F.8.1).
- 12 Exceptions thrown in destructors of objects with static storage duration or in constructors of objects associated with non-block variables with static storage duration are not caught by a *function-try-block* on the `main` function (6.10.3.1). Exceptions thrown in destructors of objects with thread storage duration or in constructors of objects associated with non-block variables with thread storage duration are not caught by a *function-try-block* on the initial function of the thread.
- 13 If a `return` statement (8.8.4) appears in a handler of the *function-try-block* of a constructor, the program is ill-formed.
- 14 The currently handled exception is rethrown if control reaches the end of a handler of the *function-try-block* of a constructor or destructor. Otherwise, flowing off the end of the *compound-statement* of a handler of a *function-try-block* is equivalent to flowing off the end of the *compound-statement* of that function (see 8.8.4).
- 15 The variable declared by the *exception-declaration*, of type *cv* T or *cv* T&, is initialized from the exception object, of type E, as follows:
  - (15.1) — if T is a base class of E, the variable is copy-initialized (9.5) from an lvalue of type T designating the corresponding base class subobject of the exception object;
  - (15.2) — otherwise, the variable is copy-initialized (9.5) from an lvalue of type E designating the exception object.
 The lifetime of the variable ends when the handler exits, after the destruction of any objects with automatic storage duration initialized within the handler.
- 16 [Note 5: When the handler declares an object, any changes to that object will not affect the exception object. When the handler declares a reference to an object, any changes to the referenced object are changes to the exception object. — *end note*

## 14.5 Exception specifications

[except.spec]

- 1 The predicate indicating whether a function cannot exit via an exception is called the *exception specification* of the function. If the predicate is false, the function has a *potentially-throwing exception specification*, otherwise it has a *non-throwing exception specification*. The exception specification is either defined implicitly, or defined explicitly by using a *noexcept-specifier* as a suffix of a function declarator (9.3.4.6).

```

noexcept-specifier:
 noexcept (constant-expression)
 noexcept

```

- 2 In a *noexcept-specifier*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool` (7.7.3). A `(` token that follows `noexcept` is part of the *noexcept-specifier* and does not commence an initializer (9.5). The *noexcept-specifier* `noexcept` without a *constant-expression* is equivalent to the *noexcept-specifier* `noexcept(true)`.

[Example 1:

```

void f() noexcept(sizeof(char[2])); // error: narrowing conversion of value 2 to type bool
void g() noexcept(sizeof(char)); // OK, conversion of value 1 to type bool is non-narrowing
— end example]

```

- <sup>3</sup> If the *noexcept-specifier* appears in a function type, the *constant-expression* is the type's exception specification.
- <sup>4</sup> If a declaration of a function does not have a *noexcept-specifier*, the declaration has a potentially throwing exception specification unless it is a destructor or a deallocation function or is defaulted on its first declaration, in which cases the exception specification is as specified below and no other declaration for that function shall have a *noexcept-specifier*. In an explicit instantiation (13.9.3) a *noexcept-specifier* may be specified, but is not required. If a *noexcept-specifier* is specified in an explicit instantiation, the exception specification shall be the same as the exception specification of all other declarations of that function. A diagnostic is required only if the exception specifications are not the same within a single translation unit.
- <sup>5</sup> If a virtual function has a non-throwing exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall have a non-throwing exception specification, unless the overriding function is defined as deleted.

[Example 2:

```
struct B {
 virtual void f() noexcept;
 virtual void g();
 virtual void h() noexcept = delete;
};

struct D: B {
 void f(); // error
 void g() noexcept; // OK
 void h() = delete; // OK
};
```

The declaration of `D::f` is ill-formed because it has a potentially-throwing exception specification, whereas `B::f` has a non-throwing exception specification. — end example]

- <sup>6</sup> An expression *E* is *potentially-throwing* if
- (6.1) — *E* is a function call (7.6.1.3) whose *postfix-expression* has a function type, or a pointer-to-function type, with a potentially-throwing exception specification, or
  - (6.2) — *E* implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, a constructor for a function argument, or a destructor) that has a potentially-throwing exception specification, or
  - (6.3) — *E* is a *throw-expression* (7.6.18), or
  - (6.4) — *E* is a `dynamic_cast` expression that casts to a reference type and requires a runtime check (7.6.1.7), or
  - (6.5) — *E* is a `typeid` expression applied to a (possibly parenthesized) built-in unary `*` operator applied to a pointer to a polymorphic class type (7.6.1.8), or
  - (6.6) — any of the immediate subexpressions (6.10.1) of *E* that is not an unevaluated operand is potentially-throwing.
- <sup>7</sup> An implicitly-declared constructor for a class *X*, or a constructor without a *noexcept-specifier* that is defaulted on its first declaration, has a potentially-throwing exception specification if and only if any of the following constructs is potentially-throwing:
- (7.1) — the invocation of a constructor selected by overload resolution in the implicit definition of the constructor for class *X* to initialize a potentially constructed subobject, or
  - (7.2) — a subexpression of such an initialization, such as a default argument expression, or,
  - (7.3) — for a default constructor, a default member initializer.

[Note 1: Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (14.3), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor would call the function `std::terminate` rather than escape the constructor (14.2, 14.6.2). — end note]

- <sup>8</sup> The exception specification for an implicitly-declared destructor, or a destructor without a *noexcept-specifier*, is potentially-throwing if and only if any of the destructors for any of its potentially constructed subobjects has a potentially-throwing exception specification or the destructor is virtual and the destructor of any virtual base class has a potentially-throwing exception specification.

<sup>9</sup> The exception specification for an implicitly-declared assignment operator, or an assignment-operator without a *noexcept-specifier* that is defaulted on its first declaration, is potentially-throwing if and only if the invocation of any assignment operator in the implicit definition is potentially-throwing.

<sup>10</sup> A deallocation function (6.8.6.5.3) with no explicit *noexcept-specifier* has a non-throwing exception specification.

<sup>11</sup> The exception specification for

(11.1) — a comparison operator function (12.4.3) or

(11.2) — a postfix increment or decrement operator function (12.4.7.1)

without a *noexcept-specifier* that is defaulted on its first declaration is potentially-throwing if and only if any expression in the implicit definition is potentially-throwing.

<sup>12</sup> [Example 3:

```
struct A {
 A(int = (A(5), 0)) noexcept;
 A(const A&) noexcept;
 A(A&&) noexcept;
 ~A();
};
struct B {
 B() noexcept;
 B(const B&) = default; // implicit exception specification is noexcept(true)
 B(B&&, int = (throw 42, 0)) noexcept;
 ~B() noexcept(false);
};
int n = 7;
struct D : public A, public B {
 int * p = new int[n];
 // D::D() potentially-throwing, as the new operator may throw bad_alloc or bad_array_new_length
 // D::D(const D&) non-throwing
 // D::D(D&&) potentially-throwing, as the default argument for B's constructor may throw
 // D::~~D() potentially-throwing
};
```

Furthermore, if `A::~~A()` were virtual, the program would be ill-formed since a function that overrides a virtual function from a base class shall not have a potentially-throwing exception specification if the base class function has a non-throwing exception specification. — end example]

<sup>13</sup> An exception specification is considered to be *needed* when:

(13.1) — in an expression, the function is selected by overload resolution (12.2, 12.3);

(13.2) — the function is odr-used (6.3);

(13.3) — the exception specification is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(13.4) — the function is defined;

(13.5) — the function is represented by a reflection; or

(13.6) — the exception specification is needed for a defaulted function that calls the function.

[Note 2: A defaulted declaration does not require the exception specification of a base member function to be evaluated until the implicit exception specification of the derived function is needed, but an explicit *noexcept-specifier* needs the implicit exception specification to compare against. — end note]

The exception specification of a defaulted function is evaluated as described above only when needed.

## 14.6 Special functions

[except.special]

### 14.6.1 General

[except.special.general]

<sup>1</sup> The function `std::terminate` (14.6.2) is used by the exception handling mechanism for coping with errors related to the exception handling mechanism itself. The function `std::uncaught_exceptions` (17.9.6) reports how many exceptions are uncaught in the current thread. The function `std::current_exception` (17.9.7) and the class `std::nested_exception` (17.9.8) can be used by a program to capture the currently handled exception.

### 14.6.2 The `std::terminate` function

[`except.terminate`]

- <sup>1</sup> Some errors in a program cannot be recovered from, such as when an exception is not handled or a `std::thread` object is destroyed while its thread function is still executing. In such cases, the function `std::terminate` (17.9.5) is invoked.

[*Note 1*: These situations are:

- (1.1) — when the exception handling mechanism, after completing the initialization of the exception object but before activation of a handler for the exception (14.2), calls a function that exits via an exception, or
- (1.2) — when the exception handling mechanism cannot find a handler for a thrown exception (14.4), or
- (1.3) — when the search for a handler (14.4) exits the function body of a function with a non-throwing exception specification (14.5), including when a contract-violation handler invoked from an evaluation of a function contract assertion (6.11.2) associated with the function exits via an exception, or
- (1.4) — when the destruction of an object during stack unwinding (14.3) terminates by throwing an exception, or
- (1.5) — when initialization of a non-block variable with static or thread storage duration (6.10.3.3) exits via an exception, or
- (1.6) — when destruction of an object with static or thread storage duration exits via an exception (6.10.3.4), or
- (1.7) — when execution of a function registered with `std::atexit` or `std::at_quick_exit` exits via an exception (17.5), or
- (1.8) — when a *throw-expression* (7.6.18) with no operand attempts to rethrow an exception and no exception is being handled (14.2), or
- (1.9) — when the function `std::nested_exception::rethrow_nested` is called for an object that has captured no exception (17.9.8), or
- (1.10) — when execution of the initial function of a thread exits via an exception (32.4.3.4), or
- (1.11) — for a parallel algorithm whose `ExecutionPolicy` specifies such behavior (26.3.6.3, 26.3.6.4, 26.3.6.5), when execution of an element access function (26.3.1) of the parallel algorithm exits via an exception (26.3.4), or
- (1.12) — when the destructor or the move assignment operator is invoked on an object of type `std::thread` that refers to a joinable thread (32.4.3.5, 32.4.3.6), or
- (1.13) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (32.7.4, 32.7.5) fails to meet a postcondition, or
- (1.14) — when a callback invocation exits via an exception when requesting stop on a `std::stop_source` or a `std::inplace_stop_source` (32.3.5.3, 32.3.9.3), or in the constructor of `std::stop_callback` or `std::inplace_stop_callback` (32.3.6.2, 32.3.10.2) when a callback invocation exits via an exception, or
- (1.15) — when a `run_loop` object is destroyed that is still in the `running` state (33.12.1), or
- (1.16) — when `unhandled_stopped` is called on a `with_awaitable_senders<T>` object (33.13.2) whose continuation is not a handle to a coroutine whose promise type has an `unhandled_stopped` member function, or
- (1.17) — when an object `scope` of type `std::execution::simple_counting_scope` or `std::execution::counting_scope` is destroyed and `scope.state` is not equal to *joined*, *unused*, or *unused-and-closed* (33.14.2.2.2), or
- (1.18) — when `std::execution::get_parallel_scheduler` is called and `std::execution::parallel_scheduler_replacement::query_parallel_scheduler_backend()` returns a null pointer value (33.15), or
- (1.19) — when an exception is thrown from a coroutine `std::execution::task` (33.13.6) which doesn't support a `std::execution::set_error_t(std::exception_ptr)` completion.

— *end note*]

- <sup>2</sup> In the situation where no matching handler is found, it is implementation-defined whether or not the stack is unwound before `std::terminate` is invoked. In the situation where the search for a handler (14.4) exits the function body of a function with a non-throwing exception specification (14.5), it is implementation-defined whether the stack is unwound, unwound partially, or not unwound at all before the function `std::terminate` is invoked. In all other situations, the stack shall not be unwound before the function `std::terminate` is invoked. An implementation is not permitted to finish stack unwinding prematurely based on a determination that the unwind process will eventually cause an invocation of the function `std::terminate`.

# 15 Preprocessing directives

[cpp]

## 15.1 Preamble

[cpp.pre]

```

preprocessing-file:
 groupopt
 module-file

module-file:
 line-directivesopt pp-global-module-fragmentopt pp-module groupopt pp-private-module-fragmentopt

pp-global-module-fragment:
 module ; new-line groupopt

pp-private-module-fragment:
 module : private ; new-line groupopt

group:
 group-part
 group group-part

group-part:
 control-line
 if-section
 text-line
 # conditionally-supported-directive

control-line:
 # include pp-tokens new-line
 pp-import
 # embed pp-tokens new-line
 # define identifier replacement-list new-line
 # define identifier lparen identifier-listopt) replacement-list new-line
 # define identifier lparen ...) replacement-list new-line
 # define identifier lparen identifier-list , ...) replacement-list new-line
 # undef identifier new-line
 line-directive
 # error pp-tokensopt new-line
 # warning pp-tokensopt new-line
 # pragma pp-tokensopt new-line
 # new-line

line-directives:
 line-directive line-directivesopt

if-section:
 if-group elif-groupsopt else-groupopt endif-line

if-group:
 # if constant-expression new-line groupopt
 # ifdef identifier new-line groupopt
 # ifndef identifier new-line groupopt

elif-groups:
 elif-group elif-groupsopt

elif-group:
 # elif constant-expression new-line groupopt
 # elifdef identifier new-line groupopt
 # elifndef identifier new-line groupopt

else-group:
 # else new-line groupopt

endif-line:
 # endif new-line

text-line:
 pp-tokensopt new-line

```

*conditionally-supported-directive*:  
*pp-tokens new-line*

*lparen*:  
 a ( character not immediately preceded by whitespace

*identifier-list*:  
*identifier*  
*identifier-list* , *identifier*

*replacement-list*:  
*pp-tokens<sub>opt</sub>*

*pp-tokens*:  
*preprocessing-token pp-tokens<sub>opt</sub>*

*embed-parameter-seq*:  
*embed-parameter embed-parameter-seq<sub>opt</sub>*

*embed-parameter*:  
*embed-standard-parameter*  
*embed-prefixed-parameter*

*embed-standard-parameter*:  
 limit ( *pp-balanced-token-seq* )  
 offset ( *pp-balanced-token-seq* )  
 prefix ( *pp-balanced-token-seq<sub>opt</sub>* )  
 suffix ( *pp-balanced-token-seq<sub>opt</sub>* )  
 if\_empty ( *pp-balanced-token-seq<sub>opt</sub>* )

*embed-prefixed-parameter*:  
*identifier :: identifier*  
*identifier :: identifier ( pp-balanced-token-seq<sub>opt</sub> )*

*pp-balanced-token-seq*:  
*pp-balanced-token pp-balanced-token-seq<sub>opt</sub>*

*pp-balanced-token*:  
 ( *pp-balanced-token-seq<sub>opt</sub>* )  
 [ *pp-balanced-token-seq<sub>opt</sub>* ]  
 { *pp-balanced-token-seq<sub>opt</sub>* }  
 any *pp-token* except:  
   parenthesis (U+0028 LEFT PARENTHESIS and U+0029 RIGHT PARENTHESIS),  
   bracket (U+005B LEFT SQUARE BRACKET and U+005D RIGHT SQUARE BRACKET), or  
   brace (U+007B LEFT CURLY BRACKET and U+007D RIGHT CURLY BRACKET).

*new-line*:  
 the new-line character

<sup>1</sup> The implementation can process and skip sections of source files conditionally, include other source files, import macros from header units, and replace macros. These capabilities are called *preprocessing*, because conceptually they occur before translation of the resulting translation unit.

<sup>2</sup> A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: At the start of translation phase 4, the first preprocessing token in the sequence, referred to as a *directive-introducing token*, begins with the first character in the source file (optionally after whitespace containing no new-line characters) or follows whitespace containing at least one new-line character, and is

- (2.1) — a **#** preprocessing token, or
- (2.2) — an **import** preprocessing token immediately followed on the same logical source line by a *header-name*, **<**, *identifier*, or **:** preprocessing token, or
- (2.3) — a **module** preprocessing token immediately followed on the same logical source line by an *identifier*, **:**, or **;** preprocessing token, or
- (2.4) — an **export** preprocessing token immediately followed on the same logical source line by one of the two preceding forms.

The last preprocessing token in the sequence is the first preprocessing token within the sequence that is immediately followed by whitespace containing a new-line character.

[*Note 1*: A new-line character ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro. — *end note*]



[Example 1:

```
// preprocessing directive
module ; // preprocessing directive
export module leftpad; // preprocessing directive
import <string>; // preprocessing directive
export import "squee"; // preprocessing directive
import rightpad; // preprocessing directive
import :part; // preprocessing directive

module // not a preprocessing directive
; // not a preprocessing directive

export // not a preprocessing directive
import // not a preprocessing directive
foo; // not a preprocessing directive

export // not a preprocessing directive
import foo; // preprocessing directive (ill-formed at phase 7)

import :: // not a preprocessing directive
import -> // not a preprocessing directive
```

— end example]

- <sup>3</sup> A sequence of preprocessing tokens is only a *text-line* if it does not begin with a directive-introducing token.

[Example 2:

```
using module = int;
module i; // not a text-line and not a control-line
int foo() {
 return i;
}
```

The example is not a valid *preprocessing-file*. — end example]

A sequence of preprocessing tokens is only a *conditionally-supported-directive* if it does not begin with any of the directive names appearing after a # in the syntax. A *conditionally-supported-directive* is conditionally-supported with implementation-defined semantics.

- <sup>4</sup> If one of the *pp-tokens* of a #*embed* directive (before macro replacement) is the *identifier* *limit*, *prefix*, *suffix*, or *if\_empty* and that *identifier* is defined as a macro (15.7.1), the program is ill-formed. Any *embed-prefixed-parameter* is conditionally-supported, with implementation-defined semantics.
- <sup>5</sup> At the start of phase 4 of translation, the *group* of a *pp-global-module-fragment* shall contain neither a *text-line* nor a *pp-import*.
- <sup>6</sup> When in a group that is skipped (15.2), the directive syntax is relaxed to allow any sequence of preprocessing tokens to occur between the directive name and the following new-line character.
- <sup>7</sup> The only whitespace characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the directive-introducing token through just before the terminating new-line character) are U+0020 SPACE and U+0009 CHARACTER TABULATION (including spaces that have replaced comments or possibly other whitespace characters in translation phase 3).
- <sup>8</sup> The preprocessing tokens within a preprocessing directive are not subject to macro expansion unless otherwise stated.

[Example 3: In:

```
#define EMPTY
EMPTY # include <file.h>
```

the sequence of preprocessing tokens on the second line is *not* a preprocessing directive, because it does not begin with a # at the start of translation phase 4, even though it will do so after the macro `EMPTY` has been replaced. — end example]

## 15.2 Conditional inclusion

[cpp.cond]

*defined-macro-expression:*

```
defined identifier
defined (identifier)
```

*h-preprocessing-token:*

```
any preprocessing-token other than >
```

*h-pp-tokens:*

```
h-preprocessing-token h-pp-tokensopt
```

*header-name-tokens:*

```
plain-string-literal
< h-pp-tokens >
```

*has-include-expression:*

```
__has_include (header-name)
__has_include (header-name-tokens)
```

*has-embed-expression:*

```
__has_embed (header-name pp-balanced-token-seqopt)
__has_embed (header-name-tokens pp-balanced-token-seqopt)
```

*has-attribute-expression:*

```
__has_cpp_attribute (pp-tokens)
```

- <sup>1</sup> The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below<sup>115</sup> and it may contain zero or more *defined-macro-expressions*, *has-include-expressions*, *has-attribute-expressions*, and/or *has-embed-expressions* as unary operator expressions. A *defined-macro-expression* shall not appear within a *has-include-expression* or *has-embed-expression*.
- <sup>2</sup> A *defined-macro-expression* evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has one or more active macro definitions (15.6), for example because it has been the subject of a **#define** preprocessing directive without an intervening **#undef** directive with the same subject identifier), 0 if it is not.
- <sup>3</sup> The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* of a **#include** directive, except that no further macro expansion is performed. If the preprocessing token sequence does not consist solely of a *header-name* or cannot be combined (15.3) into a single *header-name* preprocessing token, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.
- <sup>4</sup> The parenthesized preprocessing token sequence of each contained *has-embed-expression* is processed as if that preprocessing token sequence were the *pp-tokens* of a **#embed** directive (15.4), except that no further macro expansion is performed. The *has-embed-expression* evaluates to:
  - (4.1) — `__STDC_EMBED_FOUND__` if the search for the resource succeeds, all the given *embed-parameters* in the *embed-parameter-seq* are supported, and the resource is not empty.
  - (4.2) — Otherwise, `__STDC_EMBED_EMPTY__` if the search for the resource succeeds, all the given *embed-parameters* in the *embed-parameter-seq* are supported, and the resource is empty.
  - (4.3) — Otherwise, `__STDC_EMBED_NOT_FOUND__`.

[Note 1: An unrecognized *embed-parameter* in an *has-embed-expression* is not ill-formed and is instead treated as not supported. — end note]

- <sup>5</sup> Each *has-attribute-expression* is replaced by a non-zero *pp-number* matching the form of an *integer-literal* if the implementation supports an attribute with the name specified by interpreting the *pp-tokens*, after macro expansion, as an *attribute-token*, and by 0 otherwise. The program is ill-formed if the *pp-tokens* do not match the form of an *attribute-token*.
- <sup>6</sup> For an attribute specified in this document, it is implementation-defined whether the value of the *has-attribute-expression* is 0 or is given by Table 21. For other attributes recognized by the implementation, the value is implementation-defined.

[Note 2: It is expected that the availability of an attribute can be detected by any non-zero result. — end note]

Table 21 — `__has_cpp_attribute` values [tab:cpp.cond.ha]

| Attribute                      | Value   |
|--------------------------------|---------|
| <code>assume</code>            | 202207L |
| <code>deprecated</code>        | 201309L |
| <code>fallthrough</code>       | 201603L |
| <code>indeterminate</code>     | 202403L |
| <code>likely</code>            | 201803L |
| <code>maybe_unused</code>      | 201603L |
| <code>no_unique_address</code> | 201803L |
| <code>nodiscard</code>         | 201907L |
| <code>noreturn</code>          | 200809L |
| <code>unlikely</code>          | 201803L |

<sup>7</sup> The `#ifdef`, `#ifndef`, `#elifdef`, and `#elifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_include`, `__has_embed`, and `__has_cpp_attribute` as if they were the names of defined macros. The identifiers `__has_include`, `__has_embed`, and `__has_cpp_attribute` shall not appear in any context not mentioned in this subclause.

<sup>8</sup> Preprocessing directives of the forms

```
if constant-expression new-line groupopt
elif constant-expression new-line groupopt
```

check whether the controlling constant expression evaluates to nonzero.

<sup>9</sup> Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the `defined` unary operator), just as in normal text. If replacement of macros in the preprocessing tokens following the sequence `__has_embed` ( and before a matching ) (possibly produced by macro expansion) encounters a preprocessing token that is one of the *identifiers* `limit`, `prefix`, `suffix`, or `if_empty` and that *identifier* is defined as a macro (15.7.1), the program is ill-formed. If the preprocessing token `defined` is generated as a result of this replacement process (G.11.1) or use of the `defined` unary operator does not match one of the two specified forms prior to macro replacement, the program is ill-formed, no diagnostic required (G.11.2).

<sup>10</sup> After all replacements due to macro expansion and evaluations of *defined-macro-expressions*, *has-include-expressions*, *has-embed-expressions*, and *has-attribute-expressions* have been performed, all remaining identifiers (including those lexically identical to keywords), except for `true` and `false`, are replaced with the *pp-number* 0, and then each preprocessing token is converted into a token.

[Note 3: An alternative token (5.9) is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not subject to this replacement. — end note]

<sup>11</sup> The resulting tokens comprise the controlling constant expression which is evaluated according to the rules of 7.7.2 using arithmetic that has at least the ranges specified in 17.3. For the purposes of this token conversion and evaluation all signed and unsigned integer types act as if they have the same representation as, respectively, `intmax_t` or `uintmax_t` (17.4.1).

[Note 4: Thus on an implementation where `std::numeric_limits<int>::max()` is 0x7FFF and `std::numeric_limits<unsigned int>::max()` is 0xFFFF, the integer literal 0x8000 is signed and positive within a `#if` expression even though it is unsigned in translation phase 7 (5.2). — end note]

This includes interpreting *character-literals* according to the rules in 5.13.3.

[Note 5: The associated character encodings of literals are the same in `#if` and `#elif` directives and in any expression. — end note]

Each subexpression with type `bool` is subjected to integral promotion before processing continues.

<sup>12</sup> Preprocessing directives of the forms

115) Because the controlling constant expression is evaluated during translation phase 4, all identifiers either are or are not macro names — there simply are no keywords, enumeration constants, etc.

```
ifdef identifier new-line groupopt
ifndef identifier new-line groupopt
elifdef identifier new-line groupopt
elifndef identifier new-line groupopt
```

check whether the identifier is or is not currently defined as a macro name. Their conditions are equivalent to `#if defined identifier`, `#if !defined identifier`, `#elif defined identifier`, and `#elif !defined identifier`, respectively.

- <sup>13</sup> Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the directive name that determines the directive in order to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed; any following groups are skipped and their controlling directives are processed as if they were in a group that is skipped. If none of the conditions evaluates to true, and there is a `#else` directive, the group controlled by the `#else` is processed; lacking a `#else` directive, all the groups until the `#endif` are skipped.<sup>116</sup>

- <sup>14</sup> [Example 1: This demonstrates a way to include a library `optional` facility only if it is available:

```
#if __has_include(<optional>)
include <optional>
if __cpp_lib_optional >= 201603
define have_optional 1
endif
#elif __has_include(<experimental/optional>)
include <experimental/optional>
if __cpp_lib_experimental_optional >= 201411
define have_optional 1
define experimental_optional 1
endif
#endif
#ifdef have_optional
define have_optional 0
#endif
```

— end example]

- <sup>15</sup> [Example 2: This demonstrates a way to use the attribute `[[acme::deprecated]]` only if it is available.

```
#if __has_cpp_attribute(acme::deprecated)
define ATTR_DEPRECATED(msg) [[acme::deprecated(msg)]]
#else
define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#endif
ATTR_DEPRECATED("This function is deprecated") void anvil();
```

— end example]

### 15.3 Source file inclusion

[cpp.include]

- <sup>1</sup> A *header search* for a sequence of characters searches a sequence of places for a header identified uniquely by that sequence of characters. How the places are determined or the header identified is implementation-defined.
- <sup>2</sup> A *source file search* for a sequence of characters attempts to identify a source file that is named by the sequence of characters. The named source file is searched for in an implementation-defined manner. If the implementation does not support a source file search for that sequence of characters, or if the search fails, the result of the source file search is the result of a header search for the same sequence of characters.
- <sup>3</sup> A preprocessing directive of the form

```
include header-name new-line
```

causes the replacement of that directive by the entire contents of the header or source file identified by *header-name*.

- <sup>4</sup> If the *header-name* is of the form

```
< h-char-sequence >
```

<sup>116</sup>) As indicated by the syntax, a preprocessing token cannot follow a `#else` or `#endif` directive before the terminating new-line character. However, comments can appear anywhere in a source file, including within a preprocessing directive.

a header is identified by a header search for the sequence of characters of the *h-char-sequence*.

- 5 If the *header-name* is of the form

" *q-char-sequence* "

the source file or header is identified by a source file search for the sequence of characters of the *q-char-sequence*.

- 6 If a header search fails, or if a source file search or header search identifies a header or source file that cannot be processed by the implementation, the program is ill-formed.

[*Note 1*: If the header or source file cannot be processed, the program is ill-formed even when evaluating `__has_include`. — *end note*]

- 7 A preprocessing directive of the form

# include *pp-tokens new-line*

(that does not match the previous form) is permitted. The preprocessing tokens after `include` in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). The resulting sequence of preprocessing tokens shall be of the form

*header-name-tokens*

An attempt is then made to form a *header-name* preprocessing token (5.6) from the whitespace and the characters of the spellings of the *header-name-tokens*; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed *header-name* is processed as specified for the previous form. Otherwise, the program is ill-formed, no diagnostic required (G.11.3).

[*Note 2*: Adjacent *string-literals* are not concatenated into a single *string-literal* (see the translation phases in 5.2); thus, an expansion that results in two *string-literals* is an invalid directive. — *end note*]

- 8 The implementation shall provide unique mappings for sequences consisting of one or more *nondigits* or *digits* (5.11) followed by a period (.) and a single *nondigit*. The first character shall not be a *digit*. The implementation may ignore distinctions of alphabetical case.
- 9 A `#include` preprocessing directive may appear in a source file that has been read because of a `#include` directive in another file, up to an implementation-defined nesting limit.
- 10 If the header identified by the *header-name* denotes an importable header (10.3), it is implementation-defined whether the `#include` preprocessing directive is instead replaced by an `import` directive (15.6) of the form

import *header-name* ; *new-line*

- 11 [*Note 3*: An implementation can provide a mechanism for making arbitrary source files available to the `< >` search. However, using the `< >` form for headers provided with the implementation and the " " form for sources outside the control of the implementation achieves wider portability. For instance:

```
#include <stdio.h>
#include <unistd.h>
#include "usefullib.h"
#include "myprog.h"
```

— *end note*]

- 12 [*Example 1*: This illustrates macro-replaced `#include` directives:

```
#if VERSION == 1
 #define INCFILE "vers1.h"
#elif VERSION == 2
 #define INCFILE "vers2.h" // and so on
#else
 #define INCFILE "versN.h"
#endif
#include INCFILE
```

— *end example*]

## 15.4 Resource inclusion

[cpp.embed]

### 15.4.1 General

[cpp.embed.gen]

- 1 A *bracket resource search* for a sequence of characters searches a sequence of places for a resource identified uniquely by that sequence of characters. How the places are determined or the resource identified is

implementation-defined.

- <sup>2</sup> A *quote resource search* for a sequence of characters attempts to identify a resource that is named by the sequence of characters. The named resource is searched for in an implementation-defined manner. If the implementation does not support a quote resource search for that sequence of characters, or if the search fails, the result of the quote resource search is the result of a bracket resource search for the same sequence of characters.

- <sup>3</sup> A preprocessing directive of the form

```
embed header-name pp-tokensopt new-line
```

causes the replacement of that directive by preprocessing tokens derived from data in the resource identified by *header-name*, as specified below.

- <sup>4</sup> If the *header-name* is of the form

```
< h-char-sequence >
```

the resource is identified by a bracket resource search for the sequence of characters of the *h-char-sequence*.

- <sup>5</sup> If the *header-name* is of the form

```
" q-char-sequence "
```

the resource is identified by a quote resource search for the sequence of characters of the *q-char-sequence*.

- <sup>6</sup> If a bracket resource search fails, or if a quote or bracket resource search identifies a resource that cannot be processed by the implementation, the program is ill-formed.

[*Note 1*: If the resource cannot be processed, the program is ill-formed even when processing `#embed` with `limit(0)` (15.4.2.1) or evaluating `__has_embed`. — *end note*]

- <sup>7</sup> *Recommended practice*: A mechanism similar to, but distinct from, the implementation-defined search paths used for `#include` (15.3) is encouraged.

- <sup>8</sup> Either form of the `#embed` directive processes the *pp-tokens*, if present, just as in normal text. The *pp-tokens* shall then have the form *embed-parameter-seq*.

- <sup>9</sup> A resource is a source of data accessible from the translation environment. A resource has an *implementation-resource-width*, which is the implementation-defined size in bits of the resource. If the implementation-resource-width is not an integral multiple of `CHAR_BIT`, the program is ill-formed. Let *implementation-resource-count* be implementation-resource-width divided by `CHAR_BIT`. Every resource has a *resource-offset*, which is

- (9.1) — the value as computed from the optionally-provided *offset embed-parameter* (15.4.2.2), if present;  
 (9.2) — otherwise, 0.

Every resource also has a *resource-count*, which is

- (9.3) — the value  $\max(\min(\text{limit-value}, \text{implementation-resource-count} - \text{resource-offset}), 0)$  if the *limit embed-parameter* (15.4.2.1) is present, where the value computed from the *limit embed-parameter* is *limit-value*;  
 (9.4) — otherwise, the value  $\max(\text{implementation-resource-count} - \text{resource-offset}, 0)$ .

A resource is empty if the resource-count is zero.

- <sup>10</sup> [*Example 1*:

```
// ill-formed if the implementation-resource-width is 6 bits
#embed "6_bits.bin"
— end example]
```

- <sup>11</sup> The `#embed` directive is replaced by a comma-separated list of integer literals of type `int`, unless otherwise modified by embed parameters (15.4.2).

- <sup>12</sup> First, resource-offset calls to `std::fgetc` from the resource as a binary file have their result discarded and ignored. Next, resource-count consecutive calls to `std::fgetc` produce the elements of the comma-separated list of integer literals, in order. If any of the resource-count calls to `std::fgetc` returns `EOF`, the program is ill-formed.

- <sup>13</sup> *Recommended practice*: The value of each integer literal should closely represent the bit stream of the resource unmodified. This can require an implementation to consider potential differences between translation and execution environments, as well as any other applicable sources of mismatch.

[*Example 2*:

```

#include <cstring>
#include <cstdint>
#include <fstream>
#include <vector>
#include <cassert>

int main() {
 // If the file is the same as the resource in the translation environment, no assert in this program should fail.
 constexpr unsigned char d[] = {
#embed <data.dat>
 };
 const std::vector<unsigned char> vec_d = {
#embed <data.dat>
 };

 constexpr std::size_t expected_size = sizeof(d);

 // same file in execution environment as was embedded
 std::ifstream f_source("data.dat", std::ios::binary | std::ios::in);
 unsigned char runtime_d[expected_size];
 char* ifstream_ptr = reinterpret_cast<char*>(runtime_d);
 assert(!f_source.read(ifstream_ptr, expected_size));
 std::size_t ifstream_size = f_source.gcount();
 assert (ifstream_size == expected_size);
 int is_same = std::memcmp(&d[0], ifstream_ptr, ifstream_size);
 assert(is_same == 0);
 int is_same_vec = std::memcmp(vec_d.data(), ifstream_ptr, ifstream_size);
 assert(is_same_vec == 0);
}

```

— end example]

[Example 3:

```

int i = {
#embed "i.dat"
}; // well-formed if i.dat produces a single value
int i2 =
#embed "i.dat"
; // also well-formed if i.dat produces a single value
struct s {
 double a, b, c;
 struct { double e, f, g; } x;
 double h, i, j;
};
s x = {
// well-formed if the directive produces nine or fewer values
#embed "s.dat"
};

```

— end example]

- 14 A preprocessing directive of the form

```
embed pp-tokens new-line
```

(that does not match the previous form) is permitted. The preprocessing tokens after **embed** in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). Then, an attempt is made to form a *header-name* preprocessing token (5.6) from the whitespace and the characters of the spellings of the resulting sequence of preprocessing tokens immediately after **embed**; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed *header-name* is processed as specified for the previous form. Otherwise, the program is ill-formed.

[Note 2: Adjacent *string-literals* are not concatenated into a single *string-literal* (see the translation phases in 5.2); thus, an expansion that results in two *string-literals* is an invalid directive. — end note]

Any further processing as in normal text described for the previous form is not performed.



[*Note 3*: That is, processing as in normal text happens once and only once for the entire directive. — *end note*]

[*Example 4*: If the directive matches the second form, the whole directive is replaced. If the directive matches the first form, everything after the *header-name* is replaced.

```
#define EMPTY
#define X myfile
#define Y rsc
#define Z 42
#embed <myfile.rsc> prefix(Z)
#embed EMPTY <X.Y> prefix(Z)
```

is equivalent to:

```
#embed <myfile.rsc> prefix(42)
#embed <myfile.rsc> prefix(42)
```

— *end example*]

## 15.4.2 Embed parameters

[cpp.embed.param]

### 15.4.2.1 limit parameter

[cpp.embed.param.limit]

- <sup>1</sup> An *embed-parameter* of the form `limit ( pp-balanced-token-seq )` specifies the maximum possible number of elements in the comma-delimited list. It shall appear at most once in the *embed-parameter-seq*. The preprocessing token `defined` shall not appear in the *pp-balanced-token-seq*.
- <sup>2</sup> The *pp-balanced-token-seq* is evaluated as a *constant-expression* using the rules as described in conditional inclusion (15.2), but without being processed as in normal text an additional time.

[*Example 1*:

```
#undef DATA_LIMIT
#if __has_embed(<data.dat> limit(DATA_LIMIT))
#endif
```

is equivalent to:

```
#if __has_embed(<data.dat> limit(0))
#endif
```

— *end example*]

[*Example 2*:

```
#embed <data.dat> limit(__has_include("a.h"))

#if __has_embed(<data.dat> limit(__has_include("a.h")))
// ill-formed: __has_include (15.2) cannot appear here
#endif
```

— *end example*]

- <sup>3</sup> The *constant-expression* shall be an integral constant expression whose value is greater than or equal to zero.

[*Example 3*:

```
constexpr unsigned char sound_signature[] = {
 // a hypothetical resource capable of expanding to four or more elements
 #embed <sdk/jump.wav> limit(2+2)
};
```

```
static_assert(sizeof(sound_signature) == 4); // OK
```

— *end example*]

### 15.4.2.2 offset parameter

[cpp.embed.param.offset]

- <sup>1</sup> An *embed-parameter* of the form `offset ( pp-balanced-token-seq )` denotes the number of elements to be skipped from the resource. It shall appear at most once in the *embed-parameter-seq*.
- <sup>2</sup> The *pp-balanced-token-seq* is evaluated as a *constant-expression* using the rules as described in conditional inclusion (15.2), but without being processed as in normal text an additional time.
- <sup>3</sup> The *constant-expression* shall be an integral constant expression whose value is greater than or equal to zero.



<sup>4</sup> [Example 1:

```
constexpr const unsigned char arr[] = {
 // a hypothetical resource capable of expanding to four or more elements
 #embed <sdk/jump.wav>
};

constexpr const unsigned char offset_arr[] = {
 // the same hypothetical resource capable of expanding to four or more elements
 #embed <sdk/jump.wav> offset(2)
};

constexpr const unsigned char offset_limit_arr[] = {
 // the same hypothetical resource capable of expanding to four or more elements
 #embed <sdk/jump.wav> offset(1) limit(1)
};

static_assert(arr[2] == offset_arr[0]); // OK
static_assert(arr[3] == offset_arr[1]); // OK
static_assert(arr[1] == offset_limit_arr[0]); // OK

— end example]
```

#### 15.4.2.3 prefix parameter

[cpp.embed.param.prefix]

<sup>1</sup> An *embed-parameter* of the form

```
prefix (pp-balanced-token-seqopt)
```

shall appear at most once in the *embed-parameter-seq*.

<sup>2</sup> If the resource is empty, this *embed-parameter* is ignored. Otherwise, the *pp-balanced-token-seq* is placed immediately before the comma-delimited list of integral literals.

#### 15.4.2.4 suffix parameter

[cpp.embed.param.suffix]

<sup>1</sup> An *embed-parameter* of the form

```
suffix (pp-balanced-token-seqopt)
```

shall appear at most once in the *embed-parameter-seq*.

<sup>2</sup> If the resource is empty, this *embed-parameter* is ignored. Otherwise, the *pp-balanced-token-seq* is placed immediately after the comma-delimited list of the integral constant expressions.

[Example 1:

```
constexpr unsigned char whl[] = {
 #embed "ches.gls1" \
 prefix(0xEF, 0xBB, 0xBF,) /* a sequence of bytes */ \
 suffix(,)
 0
};
// always null-terminated, contains the sequence if not empty
constexpr bool is_empty = sizeof(whl) == 1 && whl[0] == '\0';
constexpr bool is_not_empty = sizeof(whl) >= 4
 && whl[sizeof(whl) - 1] == '\0'
 && whl[0] == '\xEF' && whl[1] == '\xBB' && whl[2] == '\xBF';
static_assert(is_empty || is_not_empty);

— end example]
```

#### 15.4.2.5 if\_empty parameter

[cpp.embed.param.if.empty]

<sup>1</sup> An *embed-parameter* of the form

```
if_empty (pp-balanced-token-seqopt)
```

shall appear at most once in the *embed-parameter-seq*.

<sup>2</sup> If the resource is not empty, this *embed-parameter* is ignored. Otherwise, the *#embed* directive is replaced by the *pp-balanced-token-seq*.

[*Example 1*: `limit(0)` affects when a resource is considered empty. Therefore, the following program:

```
#embed </owo/uwurandom> \
 if_empty(42203) limit(0)
```

expands to

```
42203
```

— *end example*]

[*Example 2*: This resource is considered empty due to the `limit(0)` *embed-parameter*, always, including in `__has_embed` clauses.

```
int infinity_zero () {
 #if __has_embed(</owo/uwurandom> limit(0) prefix(some tokens)) == __STDC_EMBED_EMPTY__
 // if </owo/uwurandom> exists, this conditional inclusion branch is taken and the function returns 0.
 return 0;
 #else
 // otherwise, the resource does not exist
 #error "The resource does not exist"
 #endif
}
```

— *end example*]

[*Example 3*: Given a resource `<single_byte>` that has an implementation-resource-count of 1, the following directives:

```
#embed <single_byte> offset(1) if_empty(42203)
```

are replaced with:

```
42203
```

— *end example*]

## 15.5 Module directive

[**cpp.module**]

*pp-module*:

```
exportopt module pp-tokensopt new-line
```

- <sup>1</sup> The *pp-tokens*, if any, of a *pp-module* shall be of the form:

```
pp-module-name pp-module-partitionopt pp-tokensopt
```

where the grammar non-terminals are defined as:

*pp-module-name*:

```
pp-module-name-qualifieropt identifier
```

*pp-module-partition*:

```
: pp-module-name-qualifieropt identifier
```

*pp-module-name-qualifier*:

```
identifier .
```

```
pp-module-name-qualifier identifier .
```

No *identifier* in the *pp-module-name* or *pp-module-partition* shall currently be defined as an object-like macro or followed by `(` as the next preprocessing token at the start of phase 4 of translation (5.2).

- <sup>2</sup> The `module` and `export` (if it exists) preprocessing tokens are replaced by the *module-keyword* and *export-keyword* preprocessing tokens respectively.

[*Note 1*: This makes the line no longer a directive so it is not removed at the end of phase 4. — *end note*]

After this replacement, the preprocessing tokens that constituted the directive are a *text-line* and are processed as normal text.

[*Note 2*: No macro expansion is possible for the *pp-module-name* and *pp-module-partition*. — *end note*]

After such processing, there shall be a `;` or `[` preprocessing token following the *pp-module-name* and optional *pp-module-partition*.

- <sup>3</sup> [*Example 1*:

Importable header `"common.h"`:

```
#define DOT_BAR .bar
#define MOD_ATTR [[vendor::shiny_module]]
```

Translation unit #1:

```
module;
#include "common.h"

export module foo DOT_BAR; // error: expansion of DOT_BAR; does not begin with ; or [
```

Translation unit #2:

```
module;
#include "common.h"

export module M MOD_ATTR ; // OK
— end example]
```

[Example 2:

```
export module a
.b; // error: preprocessing token after pp-module-name is not ; or [
— end example]
```

[Example 3:

```
export module M [[
attr1,
attr2]] ; // OK
— end example]
```

[Example 4:

```
export module M
[[attr1,
attr2]] ; // OK
— end example]
```

[Example 5:

```
export module M; int
n; // OK
— end example]
```

## 15.6 Header unit importation

[cpp.import]

*pp-import:*

```
exportopt import header-name pp-tokensopt ; new-line
exportopt import header-name-tokens pp-tokensopt ; new-line
exportopt import pp-tokens ; new-line
```

- <sup>1</sup> The preprocessing tokens after the **import** preprocessing token in the **import control-line** are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens).

[Note 1: An **import** directive matching the first two forms of a *pp-import* instructs the preprocessor to import macros from the header unit (10.3) denoted by the *header-name*, as described below. — end note]

The *point of macro import* for the first two forms of *pp-import* is immediately after the *new-line* terminating the *pp-import*. The last form of *pp-import* does not have a point of macro import, and is only considered if, after macro replacement, the first two forms did not match.

- <sup>2</sup> If a *pp-import* is produced by source file inclusion (including by the rewrite produced when a **#include** directive names an importable header) while processing the *group* of a *module-file*, the program is ill-formed.
- <sup>3</sup> In all three forms of *pp-import*, the **import** and **export** (if it exists) preprocessing tokens are replaced by the *import-keyword* and *export-keyword* preprocessing tokens respectively.

[Note 2: This makes the line no longer a directive so it is not removed at the end of phase 4. — end note]

Additionally, in the second form of *pp-import*, a *header-name* token is formed as if the *header-name-tokens* were the *pp-tokens* of a **#include** directive. The *header-name-tokens* are replaced by the *header-name* token.

[Note 3: This ensures that imports are treated consistently by the preprocessor and later phases of translation. — end note]

- <sup>4</sup> Each **#define** directive encountered when preprocessing each translation unit in a program results in a distinct *macro definition*.

[*Note 4*: A predefined macro name (15.12) is not introduced by a **#define** directive. Implementations providing mechanisms to predefine additional macros are encouraged to not treat them as being introduced by a **#define** directive. — *end note*]

Each macro definition has at most one point of definition in each translation unit and at most one point of undefinition, as follows:

- (4.1) — The *point of definition* of a macro definition within a translation unit *T* is
  - (4.1.1) — if the **#define** directive of the macro definition occurs within *T*, the point at which that directive occurs, or otherwise,
  - (4.1.2) — if the macro name is not lexically identical to a keyword (5.12) or to the *identifiers module* or **import**, the first point of macro import in *T* of a header unit containing a point of definition for the macro definition, if any.

In the latter case, the macro is said to be *imported* from the header unit.

- (4.2) — The *point of undefinition* of a macro definition within a translation unit is the first point at which a **#undef** directive naming the macro occurs after its point of definition, or the first point of macro import of a header unit containing a point of undefinition for the macro definition, whichever (if any) occurs first.
- <sup>5</sup> A macro definition is *active* at a source location if it has a point of definition in that translation unit preceding the location, and does not have a point of undefinition in that translation unit preceding the location.
- <sup>6</sup> If a macro would be replaced or redefined, and multiple macro definitions are active for that macro name, the active macro definitions shall all be valid redefinitions of the same macro (15.7).

[*Note 5*: The relative order of *pp-imports* has no bearing on whether a particular macro definition is active. — *end note*]

- <sup>7</sup> [*Example 1*:

Importable header "a.h":

```
#define X 123 // #1
#define Y 45 // #2
#define Z a // #3
#undef X // point of undefinition of #1 in "a.h"
```

Importable header "b.h":

```
import "a.h"; // point of definition of #1, #2, and #3, point of undefinition of #1 in "b.h"
#define X 456 // OK, #1 is not active
#define Y 6 // error: #2 is active
```

Importable header "c.h":

```
#define Y 45 // #4
#define Z c // #5
```

Importable header "d.h":

```
import "c.h"; // point of definition of #4 and #5 in "d.h"
```

Importable header "e.h":

```
import "a.h"; // point of definition of #1, #2, and #3, point of undefinition of #1 in "e.h"
import "d.h"; // point of definition of #4 and #5 in "e.h"
int a = Y; // OK, active macro definitions #2 and #4 are valid redefinitions
int c = Z; // error: active macro definitions #3 and #5 are not valid redefinitions of Z
```

Module unit f:

```
export module f;
export import "a.h";

int a = Y; // OK
```

Translation unit #1:

```
import f;
int x = Y; // error: Y is neither a defined macro nor a declared name
— end example]
```

## 15.7 Macro replacement

[cpp.replace]

### 15.7.1 General

[cpp.replace.general]

- 1 Two replacement lists are identical if and only if the preprocessing tokens in both have the same number, ordering, spelling, and whitespace separation, where all whitespace separations are considered identical.
- 2 An identifier currently defined as an object-like macro (see below) may be redefined by another **#define** preprocessing directive provided that the second definition is an object-like macro definition and the two replacement lists are identical, otherwise the program is ill-formed. Likewise, an identifier currently defined as a function-like macro (see below) may be redefined by another **#define** preprocessing directive provided that the second definition is a function-like macro definition that has the same number and spelling of parameters, and the two replacement lists are identical, otherwise the program is ill-formed.

- 3 [Example 1: The following sequence is valid:

```
#define OBJ_LIKE (1-1)
#define OBJ_LIKE /* whitespace */ (1-1) /* other */
#define FUNC_LIKE(a) (a)
#define FUNC_LIKE(a)(/* note the whitespace */ \
 a /* other stuff on this line
 */)
```

But the following redefinitions are invalid:

```
#define OBJ_LIKE (0) // different token sequence
#define OBJ_LIKE (1 - 1) // different whitespace
#define FUNC_LIKE(b) (a) // different parameter usage
#define FUNC_LIKE(b) (b) // different parameter spelling
```

— end example]

- 4 There shall be whitespace between the identifier and the replacement list in the definition of an object-like macro.
- 5 If the *identifier-list* in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be at least as many arguments in the invocation as there are parameters in the macro definition (excluding the ...). There shall exist a ) preprocessing token that terminates the invocation.
- 6 The identifiers `__VA_ARGS__` and `__VA_OPT__` shall occur only in the *replacement-list* of a function-like macro that uses the ellipsis notation in the parameters.
- 7 A parameter identifier in a function-like macro shall be uniquely declared within its scope.
- 8 The identifier immediately following the **define** is called the *macro name*. There is one name space for macro names. Any whitespace characters preceding or following the replacement list of preprocessing tokens are not considered part of the replacement list for either form of macro.
- 9 A translation unit shall not **#define** or **#undef** macro names lexically identical to keywords (5.12), to the identifiers listed in Table 4, or to the *attribute-tokens* described in 9.13, except that the macro names *likely* and *unlikely* may be defined as function-like macros and may be undefined.  
[Note 1: An alternative token (5.9) is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not possible to define a macro whose name is the same as that of an alternative token. — end note]
- 10 If a # preprocessing token, followed by an identifier, occurs lexically at the point at which a preprocessing directive can begin, the identifier is not subject to macro replacement.
- 11 A preprocessing directive of the form

```
define identifier replacement-list new-line
```

defines an *object-like macro* that causes each subsequent instance of the macro name<sup>117</sup> to be replaced by the replacement list of preprocessing tokens that constitute the remainder of the directive. The replacement list

<sup>117</sup> Since, by macro-replacement time, all *character-literals* and *string-literals* are preprocessing tokens, not sequences possibly containing identifier-like subsequences (see 5.2, translation phases), they are never scanned for macro names or parameters.

is then rescanned for more macro names as specified in 15.7.5.

- 12 [Example 2: The simplest use of this facility is to define a “manifest constant”, as in

```
#define TABSIZE 100
int table[TABSIZE];
```

— end example]

- 13 A preprocessing directive of the form

```
define identifier lparen identifier-listopt) replacement-list new-line
define identifier lparen ...) replacement-list new-line
define identifier lparen identifier-list , ...) replacement-list new-line
```

defines a *function-like macro* with parameters, whose use is similar syntactically to a function call. The parameters are specified by the optional list of identifiers. Each subsequent instance of the function-like macro name followed by a ( as the next preprocessing token introduces the sequence of preprocessing tokens that is replaced by the replacement list in the definition (an invocation of the macro). The replaced sequence of preprocessing tokens is terminated by the matching ) preprocessing token, skipping intervening matched pairs of left and right parenthesis preprocessing tokens. Within the sequence of preprocessing tokens making up an invocation of a function-like macro, new-line is considered a normal whitespace character.

- 14 The sequence of preprocessing tokens bounded by the outside-most matching parentheses forms the list of arguments for the function-like macro. The individual arguments within the list are separated by comma preprocessing tokens, but comma preprocessing tokens between matching inner parentheses do not separate arguments. If there are sequences of preprocessing tokens within the list of arguments that would otherwise act as preprocessing directives,<sup>118</sup> the program is ill-formed.

- 15 [Example 3: The following defines a function-like macro whose value is the maximum of its arguments. It has the disadvantages of evaluating one or the other of its arguments a second time (including side effects) and generating more code than a function if invoked several times. It also cannot have its address taken, as it has none.

```
#define max(a, b) ((a) > (b) ? (a) : (b))
```

The parentheses ensure that the arguments and the resulting expression are bound properly. — end example]

- 16 If there is a ... immediately preceding the ) in the function-like macro definition, then the trailing arguments (if any), including any separating comma preprocessing tokens, are merged to form a single item: the *variable arguments*. The number of arguments so combined is such that, following merger, the number of arguments is either equal to or one more than the number of parameters in the macro definition (excluding the ...).

## 15.7.2 Argument substitution

[cpp.subst]

*va-opt-replacement*:

```
__VA_OPT__ (pp-tokensopt)
```

- 1 After the arguments for the invocation of a function-like macro have been identified, argument substitution takes place. For each parameter in the replacement list that is neither preceded by a # or ## preprocessing token nor followed by a ## preprocessing token, the preprocessing tokens naming the parameter are replaced by a preprocessing token sequence determined as follows:
- (1.1) — If the parameter is of the form *va-opt-replacement*, the replacement preprocessing tokens are the preprocessing token sequence for the corresponding argument, as specified below.
  - (1.2) — Otherwise, the replacement preprocessing tokens are the preprocessing tokens of corresponding argument after all macros contained therein have been expanded. The argument’s preprocessing tokens are completely macro replaced before being substituted as if they formed the rest of the preprocessing translation unit with no other preprocessing tokens being available.

[Example 1:

```
#define LPAREN() (
#define G(Q) 42
#define F(R, X, ...) __VA_OPT__(G R X)
int x = F(LPAREN(), 0, <:-); // replaced by int x = 42;
```

— end example]

- 2 An identifier `__VA_ARGS__` that occurs in the replacement list shall be treated as if it were a parameter, and the variable arguments shall form the preprocessing tokens used to replace it.

<sup>118</sup>) A *conditionally-supported-directive* is a preprocessing directive regardless of whether the implementation supports it.

3 [Example 2:

```
#define debug(...) fprintf(stderr, __VA_ARGS__)
#define showlist(...) puts(#__VA_ARGS__)
#define report(test, ...) ((test) ? puts(#test) : printf(__VA_ARGS__))
debug("Flag");
debug("X = %d\n", x);
showlist(The first, second, and third items.);
report(x>y, "x is %d but y is %d", x, y);
```

results in

```
fprintf(stderr, "Flag");
fprintf(stderr, "X = %d\n", x);
puts("The first, second, and third items.");
((x>y) ? puts("x>y") : printf("x is %d but y is %d", x, y));
```

— end example]

- 4 The identifier `__VA_OPT__` shall always occur as part of the preprocessing token sequence *va-opt-replacement*; its closing `)` is determined by skipping intervening pairs of matching left and right parentheses in its *pp-tokens*. The *pp-tokens* of a *va-opt-replacement* shall not contain `__VA_OPT__`. If the *pp-tokens* would be ill-formed as the replacement list of the current function-like macro, the program is ill-formed. A *va-opt-replacement* is treated as if it were a parameter, and the preprocessing token sequence for the corresponding argument is defined as follows. If the substitution of `__VA_ARGS__` as neither an operand of `#` nor `##` consists of no preprocessing tokens, the argument consists of a single placemark preprocessing token (15.7.4, 15.7.5). Otherwise, the argument consists of the results of the expansion of the contained *pp-tokens* as the replacement list of the current function-like macro before removal of placemark tokens, rescanning, and further replacement.

[Note 1: The placemark tokens are removed before stringization (15.7.3), and can be removed by rescanning and further replacement (15.7.5). — end note]

[Example 3:

```
#define F(...) f(0 __VA_OPT__(,) __VA_ARGS__)
#define G(X, ...) f(0, X __VA_OPT__(,) __VA_ARGS__)
#define SDEF(sname, ...) S sname __VA_OPT__ (= { __VA_ARGS__ })
#define EMP

F(a, b, c) // replaced by f(0, a, b, c)
F() // replaced by f(0)
F(EMP) // replaced by f(0)

G(a, b, c) // replaced by f(0, a, b, c)
G(a,) // replaced by f(0, a)
G(a) // replaced by f(0, a)

SDEF(foo); // replaced by S foo;
SDEF(bar, 1, 2); // replaced by S bar = { 1, 2 };

#define H1(X, ...) X __VA_OPT__(##) __VA_ARGS__ // error: ## may not appear at
 // the beginning of a replacement list (15.7.4)

#define H2(X, Y, ...) __VA_OPT__(X ## Y,) __VA_ARGS__
H2(a, b, c, d) // replaced by ab, c, d

#define H3(X, ...) #__VA_OPT__(X##X X##X)
H3(, 0) // replaced by ""

#define H4(X, ...) __VA_OPT__(a X ## X) ## b
H4(, 1) // replaced by a b

#define H5A(...) __VA_OPT__()/**/__VA_OPT__()
#define H5B(X) a ## X ## b
#define H5C(X) H5B(X)
H5C(H5A()) // replaced by ab
```

— end example]

### 15.7.3 The # operator

[cpp.stringize]

- <sup>1</sup> Each # preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.
- <sup>2</sup> A *character string literal* is a *string-literal* with no prefix. If, in the replacement list, a parameter is immediately preceded by a # preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument (excluding placemaker tokens). Let the *stringizing argument* be the preprocessing token sequence for the corresponding argument with placemaker tokens removed. Each occurrence of whitespace between the stringizing argument's preprocessing tokens becomes a single space character in the character string literal. Whitespace before the first preprocessing token and after the last preprocessing token comprising the stringizing argument is deleted. Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of *header-names*, *character-literals*, and *string-literals* (including the delimiting U+0022 QUOTATION MARK (")) contained within the preprocessing token: a U+005C REVERSE SOLIDUS character (\) is inserted before each U+0022 QUOTATION MARK and U+005C REVERSE SOLIDUS character of a *header-name*, *character-literal*, or *string-literal*, and each new-line character is replaced by the two-character sequence \n. If the replacement that results is not a valid character string literal, the program is ill-formed. The character string literal corresponding to an empty stringizing argument is "". The order of evaluation of # and ## operators is unspecified.

### 15.7.4 The ## operator

[cpp.concat]

- <sup>1</sup> A ## preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.
- <sup>2</sup> If, in the replacement list of a function-like macro, a parameter is immediately preceded or followed by a ## preprocessing token, the parameter is replaced by the corresponding argument's preprocessing token sequence; however, if an argument consists of no preprocessing tokens, the parameter is replaced by a placemaker preprocessing token instead.<sup>119</sup>
- <sup>3</sup> For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a ## preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemaker preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemaker preprocessing token, and concatenation of a placemaker with a non-placemaker preprocessing token results in the non-placemaker preprocessing token.

[Note 1: Concatenation can form a *universal-character-name* (5.3.1). — end note]

If the result is not a valid preprocessing token, the program is ill-formed. The resulting preprocessing token is available for further macro replacement. The order of evaluation of ## operators is unspecified.

- <sup>4</sup> [Example 1: The sequence

```
#define str(s) # s
#define xstr(s) str(s)
#define debug(s, t) printf("x" # s " = %d, x" # t " = %s", \
 x ## s, x ## t)
#define INCFILE(n) vers ## n
#define glue(a, b) a ## b
#define xglue(a, b) glue(a, b)
#define HIGHLOW "hello"
#define LOW LOW ", world"

debug(1, 2);
fputs(str(strncmp("abc\0d", "abc", '\4') // this goes away
 == 0) str(: @\n), s);
#include xstr(INCFILE(2).h)
glue(HIGH, LOW);
xglue(HIGH, LOW)
```

<sup>119</sup>) Placemaker preprocessing tokens do not appear in the syntax because they are temporary entities that exist only within translation phase 4.



results in

```
printf("x" "1" "=" %d, x" "2" "=" %s", x1, x2);
fputs("strcmp(\"abc\\0d\", \"abc\", '\\4') == 0" ": @\\n", s);
#include "vers2.h" (after macro replacement, before file access)
"hello";
"hello" " ", world"
```

or, after concatenation of the character string literals,

```
printf("x1= %d, x2= %s", x1, x2);
fputs("strcmp(\"abc\\0d\", \"abc\", '\\4') == 0: @\\n", s);
#include "vers2.h" (after macro replacement, before file access)
"hello";
"hello, world"
```

Space around the # and ## preprocessing tokens in the macro definition is optional. — *end example*

<sup>5</sup> [Example 2: In the following fragment:

```
#define hash_hash # ## #
#define mkstr(a) # a
#define in_between(a) mkstr(a)
#define join(c, d) in_between(c hash_hash d)
char p[] = join(x, y); // equivalent to char p[] = "x ## y";
```

The expansion produces, at various stages:

```
join(x, y)
in_between(x hash_hash y)
in_between(x ## y)
mkstr(x ## y)
"x ## y"
```

In other words, expanding `hash_hash` produces a new preprocessing token, consisting of two adjacent sharp signs, but this new preprocessing token is not the `##` operator. — *end example*

<sup>6</sup> [Example 3: To illustrate the rules for placemaker preprocessing tokens, the sequence

```
#define t(x,y,z) x ## y ## z
int j[] = { t(1,2,3), t(,4,5), t(6,,7), t(8,9,),
 t(10,,), t(,11,), t(,,12), t(,,) };
```

results in

```
int j[] = { 123, 45, 67, 89,
 10, 11, 12, };
```

— *end example*

### 15.7.5 Rescanning and further replacement

[cpp.rescan]

<sup>1</sup> After all parameters in the replacement list have been substituted and # and ## processing has taken place, all placemaker preprocessing tokens are removed. Then the resulting preprocessing token sequence is rescanned, along with all subsequent preprocessing tokens of the source file, for more macro names to replace.

<sup>2</sup> [Example 1: The sequence

```
#define x 3
#define f(a) f(x * (a))
#undef x
#define x 2
#define g f
#define z z[0]
#define h g(~
#define m(a) a(w)
#define w 0,1
#define t(a) a
#define p() int
#define q(x) x
#define r(x,y) x ## y
#define str(x) # x
```

```

f(y+1) + f(f(z)) % t(t(g)(0) + t)(1);
g(x+(3,4)-w) | h 5) & m
(f)^m(m);
p() i[q()] = { q(1), r(2,3), r(4,), r(,5), r(,) };
char c[2][6] = { str(hello), str() };

```

results in

```

f(2 * (y+1)) + f(2 * (f(2 * (z[0])))) % f(2 * (0)) + t(1);
f(2 * (2+(3,4)-0,1)) | f(2 * (~ 5)) & f(2 * (0,1))^m(0,1);
int i[] = { 1, 23, 4, 5, };
char c[2][6] = { "hello", "" };

```

— *end example*

- 3 If the name of the macro being replaced is found during this scan of the replacement list (not including the rest of the source file's preprocessing tokens), it is not replaced. Furthermore, if any nested replacements encounter the name of the macro being replaced, it is not replaced. These nonreplaced macro name preprocessing tokens are no longer available for further replacement even if they are later (re)examined in contexts in which that macro name preprocessing token would otherwise have been replaced.
- 4 The resulting completely macro-replaced preprocessing token sequence is not processed as a preprocessing directive even if it resembles one, but all pragma unary operator expressions within it are then processed as specified in 15.13 below.

### 15.7.6 Scope of macro definitions

[cpp.scope]

- 1 A macro definition lasts (independent of block structure) until a corresponding **#undef** directive is encountered or (if none is encountered) until the end of the translation unit. Macro definitions have no significance after translation phase 4.
- 2 A preprocessing directive of the form

```
undef identifier new-line
```

causes the specified identifier no longer to be defined as a macro name. It is ignored if the specified identifier is not currently defined as a macro name.

### 15.8 Line control

[cpp.line]

*line-directive:*

```
line pp-tokens new-line
```

- 1 The *string-literal* of a **#line** directive, if present, shall be a character string literal.
- 2 The *line number* of the current source line is the line number of the current physical source line, i.e., it is one greater than the number of new-line characters read or introduced in translation phase 1 (5.2) while processing the source file to the current preprocessing token.
- 3 A preprocessing directive of the form

```
line digit-sequence new-line
```

causes the implementation to behave as if the following sequence of source lines begins with a source line that has a line number as specified by the digit sequence (interpreted as a decimal integer). Digit sequences representing a number outside of the range [1, 2147483647] are conditionally supported.

- 4 A preprocessing directive of the form

```
line digit-sequence " s-char-sequenceopt " new-line
```

sets the presumed line number similarly and changes the presumed name of the source file to be the contents of the character string literal.

- 5 A preprocessing directive of the form

```
line pp-tokens new-line
```

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **line** on the directive are processed just as in normal text (each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). If the directive resulting after all replacements does not match one of the two previous forms, the program is ill-formed; otherwise, the result is processed as appropriate.

**15.9 Diagnostic directives****[cpp.error]**

- <sup>1</sup> A preprocessing directive of the form

```
error pp-tokensopt new-line
```

renders the program ill-formed. A preprocessing directive of the form

```
warning pp-tokensopt new-line
```

requires the implementation to produce at least one diagnostic message for the preprocessing translation unit (4.1.1).

- <sup>2</sup> *Recommended practice:* Any diagnostic message caused by either of these directives should include the specified sequence of preprocessing tokens.

**15.10 Pragma directive****[cpp.pragma]**

- <sup>1</sup> A preprocessing directive of the form

```
pragma pp-tokensopt new-line
```

causes the implementation to behave in an implementation-defined manner. The behavior may cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any pragma that is not recognized by the implementation is ignored.

**15.11 Null directive****[cpp.null]**

- <sup>1</sup> A preprocessing directive of the form

```
new-line
```

has no effect.

**15.12 Predefined macro names****[cpp.predefined]**

- <sup>1</sup> The following macro names shall be defined by the implementation:

**\_\_cplusplus**

The integer literal 202603L.

[*Note 1:* Future revisions of this document will replace the value of this macro with a greater value. — *end note*]

The macro names listed in Table 22.

The macros defined in Table 22 shall be defined to the corresponding integer literal.

[*Note 2:* Future revisions of this document might replace the values of these macros with greater values. — *end note*]

**\_\_DATE\_\_**

The date of translation of the source file: a character string literal of the form "Mmm dd yyyy", where the names of the months are the same as those generated by the `asctime` function, and the first character of `dd` is a space character if the value is less than 10. If the date of translation is not available, an implementation-defined valid date shall be supplied.

**\_\_FILE\_\_**

A character string literal representing the presumed name of the current source file.

[*Note 3:* The presumed source file name can be changed by the `#line` directive (15.8). — *end note*]

**\_\_LINE\_\_**

The integer literal 0 or a decimal integer literal (5.13.2), with no digit separators and no *integer-suffix*, representing the presumed line number of the current source line within the current source file.

[*Note 4:* The presumed line number can be changed by the `#line` directive (15.8). — *end note*]

**\_\_STDC\_EMBED\_NOT\_FOUND\_\_**, **\_\_STDC\_EMBED\_FOUND\_\_**, and **\_\_STDC\_EMBED\_EMPTY\_\_**

The integer literals 0, 1, and 2, respectively.

[*Note 5:* These represent values replaced from *has-embed-expressions* (15.2). — *end note*]

**--STDC\_HOSTED--**

The integer literal 1 if the implementation is a hosted implementation or the integer literal 0 if it is a freestanding implementation (4.1).

**--STDCPP\_DEFAULT\_NEW\_ALIGNMENT--**

An integer literal of type `std::size_t` whose value is the alignment guaranteed by a call to `operator new(std::size_t)` or `operator new[](std::size_t)`.

[*Note 6*: Larger alignments will be passed to `operator new(std::size_t, std::align_val_t)`, etc. (7.6.2.8). — *end note*]

**--TIME--**

The time of translation of the source file: a character string literal of the form "`hh:mm:ss`" as in the time generated by the `asctime` function. If the time of translation is not available, an implementation-defined valid time shall be supplied.

**Table 22 — Feature-test macros** [tab:cpp.predefined.ft]

| Macro name                                       | Value   |
|--------------------------------------------------|---------|
| <code>--cpp_aggregate_bases</code>               | 201603L |
| <code>--cpp_aggregate_nsdmi</code>               | 201304L |
| <code>--cpp_aggregate_paren_init</code>          | 201902L |
| <code>--cpp_alias_templates</code>               | 200704L |
| <code>--cpp_aligned_new</code>                   | 201606L |
| <code>--cpp_attributes</code>                    | 200809L |
| <code>--cpp_auto_cast</code>                     | 202110L |
| <code>--cpp_binary_literals</code>               | 201304L |
| <code>--cpp_capture_star_this</code>             | 201603L |
| <code>--cpp_char8_t</code>                       | 202207L |
| <code>--cpp_concepts</code>                      | 202606L |
| <code>--cpp_conditional_explicit</code>          | 201806L |
| <code>--cpp_consteval</code>                     | 202606L |
| <code>--cpp_constexpr</code>                     | 202406L |
| <code>--cpp_constexpr_dynamic_alloc</code>       | 201907L |
| <code>--cpp_constexpr_exceptions</code>          | 202411L |
| <code>--cpp_constexpr_in_decltype</code>         | 201711L |
| <code>--cpp_constexpr_virtual_inheritance</code> | 202506L |
| <code>--cpp_constinit</code>                     | 201907L |
| <code>--cpp_contracts</code>                     | 202606L |
| <code>--cpp_decltype</code>                      | 200707L |
| <code>--cpp_decltype_auto</code>                 | 201304L |
| <code>--cpp_deduction_guides</code>              | 202207L |
| <code>--cpp_delegating_constructors</code>       | 200604L |
| <code>--cpp_deleted_function</code>              | 202403L |
| <code>--cpp_designated_initializers</code>       | 202606L |
| <code>--cpp_enumerator_attributes</code>         | 201411L |
| <code>--cpp_expansion_statements</code>          | 202506L |
| <code>--cpp_explicit_this_parameter</code>       | 202110L |
| <code>--cpp_fold_expressions</code>              | 201603L |
| <code>--cpp_generic_lambdas</code>               | 201707L |
| <code>--cpp_guaranteed_copy_elision</code>       | 201606L |
| <code>--cpp_hex_float</code>                     | 201603L |
| <code>--cpp_if_consteval</code>                  | 202106L |
| <code>--cpp_if_constexpr</code>                  | 201606L |
| <code>--cpp_impl_coroutine</code>                | 202606L |
| <code>--cpp_impl_destroying_delete</code>        | 201806L |
| <code>--cpp_impl_reflection</code>               | 202603L |
| <code>--cpp_impl_three_way_comparison</code>     | 201907L |

Table 22 — Feature-test macros (continued)

| Name                                               | Value   |
|----------------------------------------------------|---------|
| <code>__cpp_implicit_move</code>                   | 202207L |
| <code>__cpp_inheriting_constructors</code>         | 201511L |
| <code>__cpp_init_captures</code>                   | 201803L |
| <code>__cpp_initializer_lists</code>               | 200806L |
| <code>__cpp_inline_variables</code>                | 201606L |
| <code>__cpp_lambdas</code>                         | 200907L |
| <code>__cpp_modules</code>                         | 201907L |
| <code>__cpp_multidimensional_subscript</code>      | 202211L |
| <code>__cpp_named_character_escapes</code>         | 202606L |
| <code>__cpp_namespace_attributes</code>            | 201411L |
| <code>__cpp_noexcept_function_type</code>          | 201510L |
| <code>__cpp_nontype_template_args</code>           | 201911L |
| <code>__cpp_nontype_template_parameter_auto</code> | 201606L |
| <code>__cpp_nsdmi</code>                           | 200809L |
| <code>__cpp_pack_indexing</code>                   | 202606L |
| <code>__cpp_placeholder_variables</code>           | 202306L |
| <code>__cpp_pp_embed</code>                        | 202606L |
| <code>__cpp_range_based_for</code>                 | 202211L |
| <code>__cpp_raw_strings</code>                     | 200710L |
| <code>__cpp_ref_qualifiers</code>                  | 200710L |
| <code>__cpp_return_type_deduction</code>           | 201304L |
| <code>__cpp_rvalue_references</code>               | 200610L |
| <code>__cpp_size_t_suffix</code>                   | 202011L |
| <code>__cpp_sized_deallocation</code>              | 201309L |
| <code>__cpp_static_assert</code>                   | 202306L |
| <code>__cpp_static_call_operator</code>            | 202207L |
| <code>__cpp_structured_bindings</code>             | 202411L |
| <code>__cpp_template_parameters</code>             | 202502L |
| <code>__cpp_template_template_args</code>          | 201611L |
| <code>__cpp_threadsafe_static_init</code>          | 200806L |
| <code>__cpp_trivial_union</code>                   | 202603L |
| <code>__cpp_unicode_characters</code>              | 200704L |
| <code>__cpp_unicode_literals</code>                | 200710L |
| <code>__cpp_user_defined_literals</code>           | 200809L |
| <code>__cpp_using_enum</code>                      | 201907L |
| <code>__cpp_variable_templates</code>              | 201304L |
| <code>__cpp_variadic_friend</code>                 | 202403L |
| <code>__cpp_variadic_templates</code>              | 200704L |
| <code>__cpp_variadic_using</code>                  | 201611L |

<sup>2</sup> The following macro names are conditionally defined by the implementation:

`__STDCPP_FLOAT16_T__`

Defined, and has the value integer literal 1, if and only if the implementation supports the ISO/IEC 60559 floating-point interchange format binary16 as an extended floating-point type (6.9.4).

`__STDCPP_FLOAT32_T__`

Defined, and has the value integer literal 1, if and only if the implementation supports the ISO/IEC 60559 floating-point interchange format binary32 as an extended floating-point type.

`__STDCPP_FLOAT64_T__`

Defined, and has the value integer literal 1, if and only if the implementation supports the ISO/IEC 60559 floating-point interchange format binary64 as an extended floating-point type.

`__STDCPP_FLOAT128_T__`

Defined, and has the value integer literal 1, if and only if the implementation supports the ISO/IEC 60559 floating-point interchange format binary128 as an extended floating-point type.

`__STDCPP_BFLOAT16_T__`

Defined, and has the value integer literal 1, if and only if the implementation supports an extended floating-point type with the properties of the *typedef-name* `std::bfloat16_t` as described in 6.9.4.

`__STDC__`

Whether `__STDC__` is predefined and if so, what its value is, are implementation-defined.

`__STDC_MB_MIGHT_NEQ_WC__`

The integer literal 1, intended to indicate that, in the encoding for `wchar_t`, a member of the basic character set need not have a code value equal to its value when used as the lone character in an ordinary character literal.

`__STDC_VERSION__`

Whether `__STDC_VERSION__` is predefined and if so, what its value is, are implementation-defined.

`__STDC_ISO_10646__`

An integer literal of the form `yyyymmL` (for example, 199712L). Whether `__STDC_ISO_10646__` is predefined and if so, what its value is, are implementation-defined.

`__STDCPP_THREADS__`

Defined, and has the value integer literal 1, if and only if a program can have more than one thread of execution (6.10.2).

<sup>3</sup> The values of the predefined macros (except for `__FILE__` and `__LINE__`) remain constant throughout the translation unit.

<sup>4</sup> If any of the pre-defined macro names in this subclause, or the identifier `defined`, is the subject of a `#define` or a `#undef` preprocessing directive, the program is ill-formed. Any other predefined macro names shall begin with a leading underscore followed by an uppercase letter or a second underscore.

### 15.13 Pragma operator

[cpp.pragma.op]

<sup>1</sup> A unary operator expression of the form

```
_Pragma (string-literal)
```

is processed as follows: The *string-literal* is *destringized* by deleting any *encoding-prefix*, deleting the leading and trailing double-quotes, replacing each escape sequence `\` by a double-quote, and replacing each escape sequence `\\` by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a pragma directive. The original four preprocessing tokens in the unary operator expression are removed.

<sup>2</sup> [Example 1:

```
#pragma listing on "..\listing.dir"
```

can also be expressed as:

```
_Pragma ("listing on \"..\listing.dir\"")
```

The latter form is processed in the same way whether it appears literally as shown, or results from macro replacement, as in:

```
#define LISTING(x) PRAGMA(listing on #x)
#define PRAGMA(x) _Pragma(#x)
```

```
LISTING(..\listing.dir)
```

— end example]

# 16 Library introduction

[library]

## 16.1 General

[library.general]

- <sup>1</sup> This Clause describes the contents of the *C++ standard library*, how a well-formed C++ program makes use of the library, and how a conforming implementation may provide the entities in the library.
- <sup>2</sup> The following subclauses describe the method of description (16.3) and organization (16.4.2) of the library. 16.4, Clause 17 through Clause 33, and Annex D specify the contents of the library, as well as library requirements and constraints on both well-formed C++ programs and conforming implementations.
- <sup>3</sup> Detailed specifications for each of the components in the library are in Clause 17–Clause 33, as shown in Table 23.

**Table 23 — Library categories** [tab:library.categories]

| Clause    | Category                    |
|-----------|-----------------------------|
| Clause 17 | Language support library    |
| Clause 18 | Concepts library            |
| Clause 19 | Diagnostics library         |
| Clause 20 | Memory management library   |
| Clause 21 | Metaprogramming library     |
| Clause 22 | General utilities library   |
| Clause 23 | Containers library          |
| Clause 24 | Iterators library           |
| Clause 25 | Ranges library              |
| Clause 26 | Algorithms library          |
| Clause 27 | Strings library             |
| Clause 28 | Text processing library     |
| Clause 29 | Numerics library            |
| Clause 30 | Time library                |
| Clause 31 | Input/output library        |
| Clause 32 | Concurrency support library |
| Clause 33 | Execution control library   |

- <sup>4</sup> The operating system interface described in ISO/IEC/IEEE 9945:2009 is hereinafter called *POSIX*.
- <sup>5</sup> The language support library (Clause 17) provides components that are required by certain parts of the C++ language, such as memory allocation (7.6.2.8, 7.6.2.9) and exception processing (Clause 14).
- <sup>6</sup> The concepts library (Clause 18) describes library components that C++ programs may use to perform compile-time validation of template arguments and perform function dispatch based on properties of types.
- <sup>7</sup> The diagnostics library (Clause 19) provides a consistent framework for reporting errors in a C++ program, including predefined exception classes.
- <sup>8</sup> The memory management library (Clause 20) provides components for memory management, including smart pointers and scoped allocators.
- <sup>9</sup> The metaprogramming library (Clause 21) describes facilities for use in templates and during constant evaluation, including type traits, integer sequences, and rational arithmetic.
- <sup>10</sup> The general utilities library (Clause 22) includes components used by other library elements, such as a predefined storage allocator for dynamic storage management (6.8.6.5), and components used as infrastructure in C++ programs, such as tuples and function wrappers.
- <sup>11</sup> The containers (Clause 23), iterators (Clause 24), ranges (Clause 25), and algorithms (Clause 26) libraries provide a C++ program with access to a subset of the most widely used algorithms and data structures.

- <sup>12</sup> The strings library ([Clause 27](#)) provides support for manipulating sequences of type `char`, sequences of type `char8_t`, sequences of type `char16_t`, sequences of type `char32_t`, sequences of type `wchar_t`, and sequences of any other character-like type.
- <sup>13</sup> The text processing library ([Clause 28](#)) provides support for text processing, including formatting, internationalization support and regular expression matching and searching.
- <sup>14</sup> The numerics library ([Clause 29](#)) provides numeric algorithms and complex number components that extend support for numeric processing. The `valarray` component provides support for *n*-at-a-time processing, potentially implemented as parallel operations on platforms that support such processing. The random number component provides facilities for generating pseudo-random numbers.
- <sup>15</sup> The time library ([Clause 30](#)) provides generally useful time utilities.
- <sup>16</sup> The input/output library ([Clause 31](#)) provides the `iostream` components that are the primary mechanism for C++ program input and output. They can be used with other elements of the library, particularly strings, locales, and iterators.
- <sup>17</sup> The concurrency support library ([Clause 32](#)) provides components to create and manage threads, including atomic operations, mutual exclusion, and interthread communication.
- <sup>18</sup> The execution control library ([Clause 33](#)) provides components supporting execution of function objects.

## 16.2 The C standard library

[library.c]

- <sup>1</sup> The C++ standard library also makes available the facilities of the C standard library, suitably adjusted to ensure static type safety.
- <sup>2</sup> The descriptions of many library functions rely on the C standard library for the semantics of those functions. In some cases, the signatures specified in this document may be different from the signatures in the C standard library, and additional overloads may be declared in this document, but the behavior and the preconditions (including any preconditions implied by the use of a C `restrict` qualifier) are the same unless otherwise stated.
- <sup>3</sup> A call to a C standard library function is a non-constant library call ([3.35](#)) if it raises a floating-point exception other than `FE_INEXACT`. The semantics of a call to a C standard library function evaluated as a core constant expression are those specified in ISO/IEC 9899:2024, Annex F<sup>120</sup> to the extent applicable to the floating-point types ([6.9.3](#)) that are parameter types of the called function.

[Note 1: ISO/IEC 9899:2024, Annex F specifies the conditions under which floating-point exceptions are raised and the behavior when NaNs and/or infinities are passed as arguments. — end note]

[Note 2: Equivalently, a call to a C standard library function is a non-constant library call if `errno` is set when `math_errhandling & MATH_ERRNO` is `true`. — end note]

## 16.3 Method of description

[description]

### 16.3.1 General

[description.general]

- <sup>1</sup> Subclause [16.3](#) describes the conventions used to specify the C++ standard library. [16.3.2](#) describes the structure of [Clause 17](#) through [Clause 33](#) and [Annex D](#). [16.3.3](#) describes other editorial conventions.

### 16.3.2 Structure of each clause

[structure]

#### 16.3.2.1 Elements

[structure.elements]

- <sup>1</sup> Each library clause contains the following elements, as applicable:<sup>121</sup>
  - (1.1) — Summary
  - (1.2) — Requirements
  - (1.3) — Detailed specifications
  - (1.4) — References to the C standard library

<sup>120</sup>) See also ISO/IEC 9899:2024, 7.6.

<sup>121</sup>) To save space, items that do not apply to a Clause are omitted. For example, if a Clause does not specify any requirements, there will be no “Requirements” subclause.



**16.3.2.2 Summary****[structure.summary]**

- <sup>1</sup> The Summary provides a synopsis of the category, and introduces the first-level subclauses. Each subclause also provides a summary, listing the headers specified in the subclause and the library entities provided in each header.
- <sup>2</sup> The contents of the summary and the detailed specifications include:
  - (2.1) — macros
  - (2.2) — values
  - (2.3) — types and alias templates
  - (2.4) — classes and class templates
  - (2.5) — functions and function templates
  - (2.6) — objects and variable templates
  - (2.7) — concepts

**16.3.2.3 Requirements****[structure.requirements]**

- <sup>1</sup> Requirements describe constraints that shall be met by a C++ program that extends the standard library. Such extensions are generally one of the following:
  - (1.1) — Template arguments
  - (1.2) — Derived classes
  - (1.3) — Containers, iterators, and algorithms that meet an interface convention or model a concept
- <sup>2</sup> The string and iostream components use an explicit representation of operations required of template arguments. They use a class template `char_traits` to define these constraints.
- <sup>3</sup> Interface convention requirements are stated as generally as possible. Instead of stating “class *X* has to define a member function `operator++()`”, the interface requires “for any object *x* of class *X*, `++x` is defined”. That is, whether the operator is a member is unspecified.
- <sup>4</sup> Requirements are stated in terms of well-defined expressions that define valid terms of the types that meet the requirements. For every set of well-defined expression requirements there is either a named concept or a table that specifies an initial set of the valid expressions and their semantics. Any generic algorithm (Clause 26) that uses the well-defined expression requirements is described in terms of the valid expressions for its template type parameters.
- <sup>5</sup> The library specification uses a typographical convention for naming requirements. Names in *italic* type that begin with the prefix *Cpp17* refer to sets of well-defined expression requirements typically presented in tabular form, possibly with additional prose semantic requirements. For example, *Cpp17Destructible* (Table 35) is such a named requirement. Names in **constant width** type refer to library concepts which are presented as a concept definition (Clause 13), possibly with additional prose semantic requirements. For example, **destructible** (18.4.10) is such a named requirement.
- <sup>6</sup> Template argument requirements are sometimes referenced by name. See 16.3.3.3.
- <sup>7</sup> In some cases the semantic requirements are presented as C++ code. Such code is intended as a specification of equivalence of a construct to another construct, not necessarily as the way the construct must be implemented.<sup>122</sup>
- <sup>8</sup> Required operations of any concept defined in this document need not be total functions; that is, some arguments to a required operation may result in the required semantics failing to be met.  
 [Example 1: The required `<` operator of the **totally\_ordered** concept (18.5.5) does not meet the semantic requirements of that concept when operating on NaNs. — end example]  
 This does not affect whether a type models the concept.
- <sup>9</sup> A declaration may explicitly impose requirements through its associated constraints (13.5.3). When the associated constraints refer to a concept (13.7.9), the semantic constraints specified for that concept are additionally imposed on the use of the declaration.

---

<sup>122</sup>) Although in some cases the code given is unambiguously the optimum implementation.

## 16.3.2.4 Detailed specifications

[structure.specifications]

<sup>1</sup> The detailed specifications each contain the following elements:

- (1.1) — name and brief description
- (1.2) — synopsis (class definition or function declaration, as appropriate)
- (1.3) — restrictions on template arguments, if any
- (1.4) — description of class invariants
- (1.5) — description of function semantics

<sup>2</sup> Descriptions of class member functions follow the order (as appropriate):<sup>123</sup>

- (2.1) — constructor(s) and destructor
- (2.2) — copying, moving & assignment functions
- (2.3) — comparison operator functions
- (2.4) — modifier functions
- (2.5) — observer functions
- (2.6) — operators and other non-member functions

<sup>3</sup> Descriptions of function semantics contain the following elements (as appropriate):<sup>124</sup>

- (3.1) — *Constraints*: the conditions for the function's participation in overload resolution (12.2).  
 [Note 1: Failure to meet such a condition results in the function's silent non-viability. — end note]  
 [Example 1: An implementation can express such a condition via a *constraint-expression* (13.5.3). — end example]
- (3.2) — *Mandates*: the conditions that, if not met, render the program ill-formed.  
 [Example 2: An implementation can express such a condition via the *constant-expression* in a *static\_assert-declaration* (9.1). If the diagnostic is to be emitted only after the function has been selected by overload resolution, an implementation can express such a condition via a *constraint-expression* (13.5.3) and also define the function as deleted. — end example]
- (3.3) — *Constant When*: the conditions that are required for a call to the function to be a constant subexpression (3.15).
- (3.4) — *Preconditions*: conditions that the function assumes to hold whenever it is called; violation of any preconditions results in undefined behavior.  
 [Example 3: An implementation can express some such conditions via the use of a contract assertion, such as a precondition assertion (9.4.1). — end example]
- (3.5) — *Hardened preconditions*: conditions that the function assumes to hold whenever it is called.
  - (3.5.1) — When invoking the function in a hardened implementation, prior to any other observable side effects of the function, contract assertions whose predicates are as described in the hardened precondition are evaluated with a terminating semantic (6.11.2).
  - (3.5.2) — When invoking the function in a non-hardened implementation, if any hardened precondition is violated, the program has undefined behavior.
- (3.6) — *Effects*: the actions performed by the function.
- (3.7) — *Synchronization*: the synchronization operations (6.10.2) applicable to the function.
- (3.8) — *Postconditions*: the conditions (sometimes termed observable results) established by the function.  
 [Example 4: An implementation can express some such conditions via the use of a contract assertion, such as a postcondition assertion (9.4.1). — end example]
- (3.9) — *Result*: for a *typename-specifier*, a description of the named type; for an *expression*, a description of the type and value category of the expression; the expression is an lvalue if the type is an lvalue reference type, an xvalue if the type is an rvalue reference type, and a prvalue otherwise.

123) To save space, items that do not apply to a class are omitted. For example, if a class does not specify any comparison operator functions, there will be no “Comparison operator functions” subclause.

124) To save space, elements that do not apply to a function are omitted. For example, if a function specifies no preconditions, there will be no *Preconditions*: element.

- (3.10) — *Returns*: a description of the value(s) returned by the function.
  - (3.11) — *Throws*: any exceptions thrown by the function, and the conditions that would cause the exception.
  - (3.12) — *Complexity*: the time and/or space complexity of the function.
  - (3.13) — *Remarks*: additional semantic constraints on the function.
  - (3.14) — *Error conditions*: the error conditions for error codes reported by the function.
- 4 Whenever the *Effects* element specifies that the semantics of some function *F* are *Equivalent to* some code sequence, then the various elements are interpreted as follows. If *F*'s semantics specifies any *Constraints* or *Mandates* elements, then those requirements are logically imposed prior to the *equivalent-to* semantics. Next, the semantics of the code sequence are determined by the *Constraints*, *Mandates*, *Constant When*, *Preconditions*, *Hardened preconditions*, *Effects*, *Synchronization*, *Postconditions*, *Returns*, *Throws*, *Complexity*, *Remarks*, and *Error conditions* specified for the function invocations contained in the code sequence. The value returned from *F* is specified by *F*'s *Returns* element, or if *F* has no *Returns* element, a non-void return from *F* is specified by the `return` statements (8.8.4) in the code sequence. If *F*'s semantics contains a *Throws*, *Postconditions*, or *Complexity* element, then that supersedes any occurrences of that element in the code sequence.
  - 5 For non-reserved replacement and handler functions, [Clause 17](#) specifies two behaviors for the functions in question: their required and default behavior. The *default behavior* describes a function definition provided by the implementation. The *required behavior* describes the semantics of a function definition provided by either the implementation or a C++ program. Where no distinction is explicitly made in the description, the behavior described is the required behavior.
  - 6 If the formulation of a complexity requirement calls for a negative number of operations, the actual requirement is zero operations.<sup>125</sup>
  - 7 Complexity requirements specified in the library clauses are upper bounds, and implementations that provide better complexity guarantees meet the requirements.
  - 8 Error conditions specify conditions where a function may fail. The conditions are listed, together with a suitable explanation, as the `enum class errc` constants ([19.5](#)).

### 16.3.2.5 C library

[structure.see.also]

- 1 Paragraphs labeled “SEE ALSO” contain cross-references to the relevant portions of other standards ([Clause 2](#)).

## 16.3.3 Other conventions

[conventions]

### 16.3.3.1 General

[conventions.general]

- 1 Subclause [16.3.3](#) describes several editorial conventions used to describe the contents of the C++ standard library. These conventions are for describing implementation-defined types ([16.3.3.3](#)), and member functions ([16.3.3.5](#)).

### 16.3.3.2 Exposition-only entities, etc.

[expos.only.entity]

- 1 Several entities declared in [Clause 17](#) through [Clause 33](#) and [Annex D](#) are provided for exposition only. The declaration of such an entity is followed by a comment ending in *exposition only*.
- 2 The following are provided for exposition only to aid in the specification of the library:

```
namespace std {
 template<class T>
 requires convertible_to<T, decay_t<T>>
 constexpr decay_t<T> decay_copy(T&& v) // exposition only
 noexcept(is_nothrow_convertible_v<T, decay_t<T>>)
 { return std::forward<T>(v); }

 constexpr auto synth_three_way = // exposition only
 [<class T, class U>(const T& t, const U& u)
 requires requires {
 { t < u } -> boolean_testable;
 { u < t } -> boolean_testable;
 }
];
```

<sup>125</sup>) This simplifies the presentation of complexity requirements in some cases.

```

{
 if constexpr (three_way_comparable_with<T, U>) {
 return t <=> u;
 } else {
 if (t < u) return weak_ordering::less;
 if (u < t) return weak_ordering::greater;
 return weak_ordering::equivalent;
 }
};

template<class T, class U = T>
using synth_three_way_result = // exposition only
 decltype(synth_three_way(declval<T&>(), declval<U&>()));

template<class T>
 concept constexpr_wrapper_like = // exposition only
 convertible_to<T, decltype(T::value)> &&
 equality_comparable_with<T, decltype(T::value)> &&
 bool_constant<T() == T::value::value &&
 bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value>;
}

```

- <sup>3</sup> An object `dst` is said to be *decay-copied from* a subexpression `src` if the type of `dst` is `decay_t<decltype((src))>` and `dst` is copy-initialized from `src`.

### 16.3.3.3 Type descriptions

[type.descriptions]

#### 16.3.3.3.1 General

[type.descriptions.general]

- <sup>1</sup> The Requirements subclauses may describe names that are used to specify constraints on template arguments.<sup>126</sup> These names are used in library Clauses to describe the types that may be supplied as arguments by a C++ program when instantiating template components from the library.
- <sup>2</sup> Certain types shown in [Clause 31](#) are used to describe implementation-defined types. They are based on other types, but with added constraints.

#### 16.3.3.3.2 Enumerated types

[enumerated.types]

- <sup>1</sup> Several types specified in [Clause 31](#) and [28.6](#) are *enumerated types*. Each enumerated type may be implemented as an enumeration or as a synonym for an enumeration.<sup>127</sup>
- <sup>2</sup> The enumerated type *enumerated* can be written:

```

enum enumerated { V0, V1, V2, V3, ... };

inline const enumerated C0(V0);
inline const enumerated C1(V1);
inline const enumerated C2(V2);
inline const enumerated C3(V3);
:

```

- <sup>3</sup> Here, the names  $C_0$ ,  $C_1$ , etc. represent *enumerated elements* for this particular enumerated type. All such elements have distinct values.

#### 16.3.3.3.3 Bitmask types

[bitmask.types]

- <sup>1</sup> Several types specified in [Clause 17](#) through [Clause 33](#) and [Annex D](#) are *bitmask types*. Each bitmask type can be implemented as an enumerated type that overloads certain operators, as an integer type, or as a `bitset` ([22.9.2](#)).
- <sup>2</sup> The bitmask type *bitmask* can be written:

<sup>126</sup> Examples from [16.4.4](#) include: `Cpp17EqualityComparable`, `Cpp17LessThanComparable`, `Cpp17CopyConstructible`. Examples from [24.3](#) include: `Cpp17InputIterator`, `Cpp17ForwardIterator`.

<sup>127</sup> Such as an integer type, with constant integer values ([6.9.3](#)).

```

// For exposition only.
// int_type is an integral type capable of representing all values of the bitmask type.
enum bitmask : int_type {
 V0 = 1 << 0, V1 = 1 << 1, V2 = 1 << 2, V3 = 1 << 3, ...
};

inline constexpr bitmask C0(V0);
inline constexpr bitmask C1(V1);
inline constexpr bitmask C2(V2);
inline constexpr bitmask C3(V3);
:

constexpr bitmask operator&(bitmask X, bitmask Y) {
 return static_cast<bitmask>(
 static_cast<int_type>(X) & static_cast<int_type>(Y));
}
constexpr bitmask operator|(bitmask X, bitmask Y) {
 return static_cast<bitmask>(
 static_cast<int_type>(X) | static_cast<int_type>(Y));
}
constexpr bitmask operator^(bitmask X, bitmask Y) {
 return static_cast<bitmask>(
 static_cast<int_type>(X) ^ static_cast<int_type>(Y));
}
constexpr bitmask operator~(bitmask X) {
 return static_cast<bitmask>(~static_cast<int_type>(X));
}
bitmask& operator&=(bitmask& X, bitmask Y) {
 X = X & Y; return X;
}
bitmask& operator|=(bitmask& X, bitmask Y) {
 X = X | Y; return X;
}
bitmask& operator^=(bitmask& X, bitmask Y) {
 X = X ^ Y; return X;
}

```

<sup>3</sup> Here, the names  $C_0$ ,  $C_1$ , etc. represent *bitmask elements* for this particular bitmask type. All such elements have distinct, nonzero values such that, for any pair  $C_i$  and  $C_j$  where  $i \neq j$ ,  $C_i \& C_j$  is nonzero and  $C_i \& C_i$  is zero. Additionally, the value 0 is used to represent an *empty bitmask*, in which no bitmask elements are set.

<sup>4</sup> The following terms apply to objects and values of bitmask types:

- (4.1) — To *set* a value  $Y$  in an object  $X$  is to evaluate the expression  $X |= Y$ .
- (4.2) — To *clear* a value  $Y$  in an object  $X$  is to evaluate the expression  $X \&= ~Y$ .
- (4.3) — The value  $Y$  is *set* in the object  $X$  if the expression  $X \& Y$  is nonzero.

#### 16.3.3.3.4 Character sequences

[character.seq]

##### 16.3.3.3.4.1 General

[character.seq.general]

<sup>1</sup> The C standard library makes widespread use of characters and character sequences that follow a few uniform conventions:

- (1.1) — Properties specified as *locale-specific* may change during program execution by a call to `setlocale(int, const char*)` (28.3.5.1), or by a change to a `locale` object, as described in 28.3.3 and Clause 31.
- (1.2) — The *execution character set* and the *execution wide-character set* are supersets of the basic literal character set (5.3.1). The encodings of the execution character sets and the sets of additional elements (if any) are locale-specific. Each element of the execution wide-character set is encoded as a single code unit representable by a value of type `wchar_t`.  
 [Note 1: The encodings of the execution character sets can be unrelated to any literal encoding. — end note]
- (1.3) — A *letter* is any of the 26 lowercase or 26 uppercase letters in the basic character set.

- (1.4) — The *decimal-point character* is the locale-specific (single-byte) character used by functions that convert between a (single-byte) character sequence and a value of one of the floating-point types. It is used in the character sequence to denote the beginning of a fractional part. It is represented in [Clause 17](#) through [Clause 33](#) and [Annex D](#) by a period, `'.'`, which is also its value in the "C" locale.
- (1.5) — A *character sequence* is an array object ([9.3.4.5](#)) *A* that can be declared as *TA[N]*, where *T* is any of the types `char`, `unsigned char`, or `signed char` ([6.9.3](#)), optionally qualified by any combination of `const` or `volatile`. The initial elements of the array have defined contents up to and including an element determined by some predicate. A character sequence can be designated by a pointer value *S* that points to its first element.
- (1.6) — Let *STATICALLY-WIDEN*<charT>("...") be "..." if `charT` is `char` and `L"..."` if `charT` is `wchar_t`.

#### 16.3.3.3.4.2 Byte strings

[byte.strings]

- <sup>1</sup> A *null-terminated byte string*, or NTBS, is a character sequence whose highest-addressed element with defined content has the value zero (the *terminating null character*); no other element in the sequence has the value zero.<sup>128</sup>
- <sup>2</sup> The *length of an NTBS* is the number of elements that precede the terminating null character. An *empty NTBS* has a length of zero.
- <sup>3</sup> The *value of an NTBS* is the sequence of values of the elements up to and including the terminating null character.
- <sup>4</sup> A *static NTBS* is an NTBS with static storage duration.<sup>129</sup>

#### 16.3.3.3.4.3 Multibyte strings

[multibyte.strings]

- <sup>1</sup> A *multibyte character* is a sequence of one or more bytes representing the code unit sequence for an encoded character of the execution character set.
- <sup>2</sup> A *null-terminated multibyte string*, or NTMBS, is an NTBS that constitutes a sequence of valid multibyte characters, beginning and ending in the initial shift state.<sup>130</sup>
- <sup>3</sup> A *static NTMBS* is an NTMBS with static storage duration.

#### 16.3.3.3.5 Customization Point Object types

[customization.point.object]

- <sup>1</sup> A *customization point object* is a function object ([22.10](#)) with a literal class type that interacts with program-defined types while enforcing semantic requirements on that interaction.
- <sup>2</sup> The type of a customization point object, ignoring cv-qualifiers, shall model *semiregular* ([18.6](#)) and shall be a structural type ([13.2](#)) and a trivially copyable type ([11.2](#)). Every constructor of this type shall have a non-throwing exception specification ([14.5](#)).
- <sup>3</sup> All instances of a specific customization point object type shall be equal ([18.2](#)). The effects of invoking different instances of a specific customization point object type on the same arguments are equivalent.
- <sup>4</sup> The type *T* of a customization point object, ignoring *cv-qualifiers*, shall model *invocable*<*T*&, *Args...*>, *invocable*<*const T*&, *Args...*>, *invocable*<*T*, *Args...*>, and *invocable*<*const T*, *Args...*> ([18.7.2](#)) when the types in *Args...* meet the requirements specified in that customization point object's definition. When the types of *Args...* do not meet the customization point object's requirements, *T* shall not have a function call operator that participates in overload resolution.
- <sup>5</sup> For a given customization point object *o*, let *p* be a variable initialized as if by `auto p = o;`. Then for any sequence of arguments *args...*, the following expressions have effects equivalent to *o(args...)*:
  - (5.1) — `p(args...)`
  - (5.2) — `as_const(p)(args...)`
  - (5.3) — `std::move(p)(args...)`
  - (5.4) — `std::move(as_const(p))(args...)`

<sup>128</sup> Many of the objects manipulated by function signatures declared in *<cstring>* ([27.5.1](#)) are character sequences or NTBSs. The size of some of these character sequences is limited by a length value, maintained separately from the character sequence.

<sup>129</sup> A *string-literal*, such as `"abc"`, is a static NTBS.

<sup>130</sup> An NTBS that contains characters only from the basic literal character set is also an NTMBS. Each multibyte character then consists of a single byte.

**16.3.3.4 Algorithm function objects****[alg.func.obj]**

- <sup>1</sup> An *algorithm function object* is a customization point object (16.3.3.3.5) that is specified as one or more overloaded function templates. The name of these function templates designates the corresponding algorithm function object.
- <sup>2</sup> For an algorithm function object *o*, let *S* be the corresponding set of function templates. Then for any sequence of arguments *args...*, *o(args...)* is expression-equivalent to *s(args...)*, where the result of name lookup for *s* is the overload set *S*.

[*Note 1*: Algorithm function objects are not found by argument-dependent name lookup (6.5.4). When found by unqualified name lookup (6.5.3) for the *postfix-expression* in a function call (7.6.1.3), they inhibit argument-dependent name lookup.

[*Example 1*:

```
void foo() {
 using namespace std::ranges;
 std::vector<int> vec{1,2,3};
 find(begin(vec), end(vec), 2); // #1
}
```

The function call expression at #1 invokes `std::ranges::find`, not `std::find`. — *end example*]

— *end note*]

**16.3.3.5 Functions within classes****[functions.within.classes]**

- <sup>1</sup> For the sake of exposition, Clause 17 through Clause 33 and Annex D do not describe copy/move constructors, assignment operators, or (non-virtual) destructors with the same apparent semantics as those that can be generated by default (11.4.5.3, 11.4.6, 11.4.7). It is unspecified whether the implementation provides explicit definitions for such member function signatures, or for virtual destructors that can be generated by default.

**16.3.3.6 Private members****[objects.within.classes]**

- <sup>1</sup> Clause 17 through Clause 33 and Annex D do not specify the representation of classes, and intentionally omit specification of class members (11.4). An implementation may define static or non-static class members, or both, as needed to implement the semantics of the member functions specified in Clause 17 through Clause 33 and Annex D.
- <sup>2</sup> For the sake of exposition, some subclauses provide representative declarations, and semantic requirements, for private members of classes that meet the external specifications of the classes. The declarations for such members are followed by a comment that ends with *exposition only*, as in:

```
streambuf* sb; // exposition only
```

- <sup>3</sup> An implementation may use any technique that provides equivalent observable behavior.

**16.3.3.7 Freestanding items****[freestanding.item]**

- <sup>1</sup> A *freestanding item* is a declaration, entity, or macro that is required to be present in a freestanding implementation and a hosted implementation.
- <sup>2</sup> Unless otherwise specified, the requirements on freestanding items for a freestanding implementation are the same as the corresponding requirements for a hosted implementation, except that not all of the members of those items are required to be present.
- <sup>3</sup> Function declarations and function template declarations followed by a comment that include *freestanding-deleted* are *freestanding deleted functions*. On freestanding implementations, it is implementation-defined whether each entity introduced by a freestanding deleted function is a deleted function (9.6.3) or whether the requirements are the same as the corresponding requirements for a hosted implementation.

[*Note 1*: Deleted definitions reduce the chance of overload resolution silently changing when migrating from a freestanding implementation to a hosted implementation. — *end note*]

[*Example 1*:

```
double abs(double j); // freestanding-deleted
```

— *end example*]

- <sup>4</sup> A declaration in a synopsis is a freestanding item if

(4.1) — it is followed by a comment that includes *freestanding*,



- (4.2) — it is followed by a comment that includes *freestanding-deleted*, or
- (4.3) — the header synopsis begins with a comment that includes *freestanding* and the declaration is not followed by a comment that includes *hosted*.

[*Note 2*: Declarations followed by *hosted* in freestanding headers are not freestanding items. As a result, looking up the name of such functions can vary between hosted and freestanding implementations. — *end note*]

[*Example 2*:

```
// all freestanding
namespace std {
— end example]
```

- 5 An entity or deduction guide is a freestanding item if its introducing declaration is not followed by a comment that includes *hosted*, and is:
  - (5.1) — introduced by a declaration that is a freestanding item,
  - (5.2) — a member of a freestanding item other than a namespace,
  - (5.3) — an enumerator of a freestanding item,
  - (5.4) — a deduction guide of a freestanding item,
  - (5.5) — an enclosing namespace of a freestanding item,
  - (5.6) — a friend of a freestanding item,
  - (5.7) — denoted by a type alias that is a freestanding item, or
  - (5.8) — denoted by an alias template that is a freestanding item.
- 6 A macro is a freestanding item if it is defined in a header synopsis and
  - (6.1) — the definition is followed by a comment that includes *freestanding*, or
  - (6.2) — the header synopsis begins with a comment that includes *freestanding* and the definition is not followed by a comment that includes *hosted*.

[*Example 3*:

```
#define NULL see below // freestanding
— end example]
```

- 7 [*Note 3*: Freestanding annotations follow some additional exposition conventions that do not impose any additional normative requirements. Header synopses that begin with a comment containing "all freestanding" contain no hosted items and no freestanding deleted functions. Header synopses that begin with a comment containing "mostly freestanding" contain at least one hosted item or freestanding deleted function. Classes and class templates followed by a comment containing "partially freestanding" contain at least one hosted item or freestanding deleted function. — *end note*]

[*Example 4*:

```
template<class T, size_t N> struct array; // partially freestanding
template<class T, size_t N>
struct array {
 constexpr reference operator[] (size_type n);
 constexpr const_reference operator[] (size_type n) const;
 constexpr reference at(size_type n); // freestanding-deleted
 constexpr const_reference at(size_type n) const; // freestanding-deleted
};
— end example]
```

## 16.4 Library-wide requirements

[requirements]

### 16.4.1 General

[requirements.general]

- 1 Subclause 16.4 specifies requirements that apply to the entire C++ standard library. Clause 17 through Clause 33 and Annex D specify the requirements of individual entities within the library.
- 2 Requirements specified in terms of interactions between threads do not apply to programs having only a single thread of execution.
- 3 16.4.2 describes the library's contents and organization, 16.4.3 describes how well-formed C++ programs gain access to library entities, 16.4.4 describes constraints on types and functions used with the C++ standard



library, 16.4.5 describes constraints on well-formed C++ programs, and 16.4.6 describes constraints on conforming implementations.

## 16.4.2 Library contents and organization [organization]

### 16.4.2.1 General [organization.general]

- <sup>1</sup> 16.4.2.2 describes the entities and macros defined in the C++ standard library. 16.4.2.3 lists the standard library headers and some constraints on those headers. 16.4.2.5 lists requirements for a freestanding implementation of the C++ standard library.

### 16.4.2.2 Library contents [contents]

- <sup>1</sup> The C++ standard library provides definitions for the entities and macros described in the synopses of the C++ standard library headers (16.4.2.3), unless otherwise specified.
- <sup>2</sup> All library entities except `operator new` and `operator delete` are defined within the namespace `std` or namespaces nested within namespace `std`.<sup>131</sup> It is unspecified whether names declared in a specific namespace are declared directly in that namespace or in an inline namespace inside that namespace.<sup>132</sup>
- <sup>3</sup> Whenever an unqualified name other than `swap`, `make_error_code`, `make_error_condition`, `from_stream`, or `submdspan_mapping` is used in the specification of a declaration D in Clause 17 through Clause 33 or Annex D, its meaning is established as-if by performing unqualified name lookup (6.5.3) in the context of D.

[Note 1: Argument-dependent lookup is not performed. — end note]

Similarly, the meaning of a *qualified-id* is established as-if by performing qualified name lookup (6.5.5) in the context of D.

[Example 1: The reference to `is_array_v` in the specification of `std::to_array` (23.3.3.6) refers to `::std::is_array_v`. — end example]

[Note 2: Operators in expressions (12.2.2.3) are not so constrained; see 16.4.6.4. — end note]

The meaning of the unqualified name `swap` is established in an overload resolution context for swappable values (16.4.4.3). The meanings of the unqualified names `make_error_code`, `make_error_condition`, `from_stream`, and `submdspan_mapping` are established as-if by performing argument-dependent lookup (6.5.4).

### 16.4.2.3 Headers [headers]

- <sup>1</sup> Each element of the C++ standard library is declared or defined (as appropriate) in a *header*.<sup>133</sup>
- <sup>2</sup> The C++ standard library provides the *C++ library headers*, shown in Table 24.
- <sup>3</sup> The facilities of the C standard library are provided in the additional headers shown in Table 25.<sup>134</sup>
- <sup>4</sup> The headers listed in Table 24, or, for a freestanding implementation, the subset of such headers that are provided by the implementation, are collectively known as the *importable C++ library headers*.

[Note 1: Importable C++ library headers can be imported (10.3). — end note]

[Example 1:

```
import <vector>; // imports the <vector> header unit
std::vector<int> vi; // OK
```

— end example]

- <sup>5</sup> Except as noted in Clause 16 through Clause 33 and Annex D, the contents of each header *cname* is the same as that of the corresponding header *name.h* as specified in the C standard library (Clause 2). In the C++ standard library, however, the declarations (except for names which are defined as macros in C) are within namespace scope (6.4.6) of the namespace `std`. It is unspecified whether these names (including any overloads added in Clause 17 through Clause 33 and Annex D) are first declared within the global namespace scope and are then injected into namespace `std` by explicit *using-declarations* (9.10).
- <sup>6</sup> Names which are defined as macros in C shall be defined as macros in the C++ standard library, even if C grants license for implementation as functions.

<sup>131</sup>) The C standard library headers (17.15) also define names within the global namespace, while the C++ headers for C library facilities (16.4.2.3) can also define names within the global namespace.

<sup>132</sup>) This gives implementers freedom to use inline namespaces to support multiple configurations of the library.

<sup>133</sup>) A header is not necessarily a source file, nor are the sequences delimited by < and > in header names necessarily valid source file names (15.3).

<sup>134</sup>) It is intentional that there is no C++ header for any of these C headers: `<stdnoreturn.h>`, `<threads.h>`.

Table 24 — C++ library headers [tab:headers.cpp]

|                      |                    |                    |                 |
|----------------------|--------------------|--------------------|-----------------|
| <algorithm>          | <forward_list>     | <meta>             | <stack>         |
| <any>                | <fstream>          | <mutex>            | <stacktrace>    |
| <array>              | <functional>       | <new>              | <stdexcept>     |
| <atomic>             | <future>           | <numbers>          | <stdfloat>      |
| <barrier>            | <generator>        | <numeric>          | <stop_token>    |
| <bit>                | <hazard_pointer>   | <optional>         | <streambuf>     |
| <bitset>             | <hive>             | <ostream>          | <string>        |
| <charconv>           | <initializer_list> | <print>            | <string_view>   |
| <chrono>             | <inplace_vector>   | <queue>            | <syncstream>    |
| <compare>            | <iomanip>          | <random>           | <system_error>  |
| <complex>            | <ios>              | <ranges>           | <text_encoding> |
| <concepts>           | <iosfwd>           | <ratio>            | <thread>        |
| <condition_variable> | <iostream>         | <rcu>              | <tuple>         |
| <contracts>          | <istream>          | <regex>            | <type_traits>   |
| <coroutine>          | <iterator>         | <scoped_allocator> | <typeindex>     |
| <debugging>          | <latch>            | <semaphore>        | <typeinfo>      |
| <deque>              | <limits>           | <set>              | <unordered_map> |
| <exception>          | <linalg>           | <shared_mutex>     | <unordered_set> |
| <execution>          | <list>             | <simd>             | <utility>       |
| <expected>           | <locale>           | <source_location>  | <valarray>      |
| <filesystem>         | <map>              | <span>             | <variant>       |
| <flat_map>           | <mdspan>           | <spanstream>       | <vector>        |
| <flat_set>           | <memory>           | <sstream>          | <version>       |
| <format>             | <memory_resource>  |                    |                 |

Table 25 — C++ headers for C library facilities [tab:headers.cpp.c]

|           |             |           |           |           |           |           |
|-----------|-------------|-----------|-----------|-----------|-----------|-----------|
| <cassert> | <cfenv>     | <climits> | <setjmp>  | <cstddef> | <cstdlib> | <cuchar>  |
| <cctype>  | <cfloat>    | <locale>  | <csignal> | <cstdint> | <cstring> | <wchar>   |
| <cerrno>  | <cinttypes> | <cmath>   | <cstdarg> | <cstdio>  | <ctime>   | <cwctype> |

[Note 2: The names defined as macros in C include the following: `assert`, `offsetof`, `setjmp`, `va_arg`, `va_end`, and `va_start`. — end note]

- 7 Names that are defined as functions in C shall be defined as functions in the C++ standard library.<sup>135</sup>
- 8 Identifiers that are keywords or operators in C++ shall not be defined as macros in C++ standard library headers.<sup>136</sup>
- 9 Subclause 17.15 describes the effects of using the *name.h* (C header) form in a C++ program.<sup>137</sup>
- 10 ISO/IEC 9899:2024, Annex K describes a large number of functions, with associated types and macros, which “promote safer, more secure programming” than many of the traditional C library functions. The names of the functions have a suffix of `_s`; most of them provide the same service as the C library function with the unsuffixed name, but generally take an additional argument whose value is the size of the result array. If any C++ header is included, it is implementation-defined whether any of these names is declared in the global namespace. (None of them is declared in namespace `std`.)
- 11 Table 26 lists the Annex K names that may be declared in some header. These names are also subject to the restrictions of 16.4.5.3.3.

135) This disallows the practice, allowed in C, of providing a masking macro in addition to the function prototype. The only way to achieve equivalent inline behavior in C++ is to provide a definition as an extern inline function.

136) In particular, including the standard header `<iso646.h>` has no effect.

137) The “.h” headers dump all their names into the global namespace, whereas the newer forms keep their names in namespace `std`. Therefore, the newer forms are the preferred forms for all uses except for C++ programs which are intended to be strictly compatible with C.

Table 26 — Names from ISO/IEC 9899:2024, Annex K [tab:c.annex.k.names]

|                      |                          |              |             |
|----------------------|--------------------------|--------------|-------------|
| abort_handler_s      | mbstowcs_s               | strncat_s    | vswscanf_s  |
| asctime_s            | memcpy_s                 | strncpy_s    | vwprintf_s  |
| bsearch_s            | memmove_s                | strtok_s     | vscanf_s    |
| constraint_handler_t | memset_s                 | swprintf_s   | wcrtomb_s   |
| ctime_s              | printf_s                 | swscanf_s    | wscat_s     |
| errno_t              | qsort_s                  | tmpfile_s    | wscopy_s    |
| fopen_s              | RSIZE_MAX                | TMP_MAX_S    | wcsncat_s   |
| fprintf_s            | rsize_t                  | tmpnam_s     | wcsncpy_s   |
| freopen_s            | scanf_s                  | vfprintf_s   | wcsnlen_s   |
| fscanf_s             | set_constraint_handler_s | vfprintf_s   | wcsrtombs_s |
| fwprintf_s           | snprintf_s               | vfwprintf_s  | wcstok_s    |
| fwscanf_s            | snwprintf_s              | vfwscanf_s   | wcstombs_s  |
| getenv_s             | sprintf_s                | vprintf_s    | wctomb_s    |
| gets_s               | sscanf_s                 | vscanf_s     | wmemcpy_s   |
| gmtime_s             | strcat_s                 | vsnprintf_s  | wmemmove_s  |
| ignore_handler_s     | strcpy_s                 | vsnwprintf_s | wprintf_s   |
| localtime_s          | strerrorlen_s            | vsprintf_s   | wscanf_s    |
| L_tmpnam_s           | strerror_s               | vsscanf_s    |             |
| mbsrtowcs_s          | strlen_s                 | vswprintf_s  |             |

## 16.4.2.4 Modules

[std.modules]

- <sup>1</sup> The C++ standard library provides the following *C++ library modules*.
- <sup>2</sup> The named module `std` exports declarations in namespace `std` that are provided by the importable C++ library headers (Table 24 or the subset provided by a freestanding implementation) and the C++ headers for C library facilities (Table 25). It additionally exports declarations in the global namespace for the storage allocation and deallocation functions that are provided by `<new>` (17.6.2).
- <sup>3</sup> The named module `std.compat` exports the same declarations as the named module `std`, and additionally exports
  - (3.1) — declarations in the global namespace corresponding to the declarations in namespace `std` that are provided by the C++ headers for C library facilities (Table 25), except the explicitly excluded declarations described in 17.15.7 and
  - (3.2) — declarations provided by the headers `<stdbit.h>` (22.12) and `<stdckdint.h>` (29.11.1).
- <sup>4</sup> It is unspecified to which module a declaration in the standard library is attached.

[Note 1: Conforming implementations ensure that mixing `#include` and `import` does not result in conflicting attachments (6.7). — end note]

*Recommended practice:* Implementations should ensure such attachments do not preclude further evolution or decomposition of the standard library modules.

- <sup>5</sup> A declaration in the standard library denotes the same entity regardless of whether it was made reachable through including a header, importing a header unit, or importing a C++ library module.
- <sup>6</sup> *Recommended practice:* Implementations should avoid exporting any other declarations from the C++ library modules.

[Note 2: Like all named modules, the C++ library modules do not make macros visible (10.3), such as `assert` (19.3.2), `errno` (19.4.2), `offsetof` (17.2.1), and `va_arg` (17.14.2). — end note]

## 16.4.2.5 Freestanding implementations

[compliance]

- <sup>1</sup> Two kinds of implementations are defined: hosted and freestanding (4.1); the kind of the implementation is implementation-defined. For a hosted implementation, this document describes the set of available headers.
- <sup>2</sup> A freestanding implementation has an implementation-defined set of headers. This set shall include at least the headers shown in Table 27.
- <sup>3</sup> For each of the headers listed in Table 27, a freestanding implementation provides at least the freestanding items (16.3.3.7) declared in the header.

Table 27 — C++ headers for freestanding implementations [tab:headers.cpp.fs]

|           | Subclause                                       | Header                                   |
|-----------|-------------------------------------------------|------------------------------------------|
| 17.2      | Common definitions                              | <cstddef>                                |
| 17.2.2    | C standard library                              | <cstdlib>                                |
| 17.3      | Implementation properties                       | <cfloat>, <climits>, <limits>, <version> |
| 17.4.1    | Integer types                                   | <stdint>                                 |
| 17.6      | Dynamic memory management                       | <new>                                    |
| 17.7      | Type identification                             | <typeinfo>                               |
| 17.8      | Source location                                 | <source_location>                        |
| 17.9      | Exception handling                              | <exception>                              |
| 17.10     | Contract-violation handling                     | <contracts>                              |
| 17.11     | Initializer lists                               | <initializer_list>                       |
| 17.12     | Comparisons                                     | <compare>                                |
| 17.13     | Coroutines support                              | <coroutine>                              |
| 17.14     | Other runtime support                           | <cstdarg>                                |
| Clause 18 | Concepts library                                | <concepts>                               |
| 19.4      | Error numbers                                   | <cerrno>                                 |
| 19.5      | System error support                            | <system_error>                           |
| 19.7      | Debugging                                       | <debugging>                              |
| 20.2      | Memory                                          | <memory>                                 |
| 21.3      | Type traits                                     | <type_traits>                            |
| 21.5      | Compile-time rational arithmetic                | <ratio>                                  |
| 22.2      | Utility components                              | <utility>                                |
| 22.4      | Tuples                                          | <tuple>                                  |
| 22.5      | Optional objects                                | <optional>                               |
| 22.6      | Variants                                        | <variant>                                |
| 22.8      | Expected objects                                | <expected>                               |
| 22.10     | Function objects                                | <functional>                             |
| 22.11     | Bit manipulation                                | <bit>                                    |
| 22.12     | C-compatible bit manipulation                   | <stdbit.h>                               |
| 23.3.3    | Class template <code>array</code>               | <array>                                  |
| 23.3.16   | Class template <code>inplace_vector</code>      | <inplace_vector>                         |
| 23.7.2    | Contiguous access                               | <span>                                   |
| 23.7.3    | Multidimensional access                         | <mdspan>                                 |
| Clause 24 | Iterators library                               | <iterator>                               |
| Clause 25 | Ranges library                                  | <ranges>                                 |
| Clause 26 | Algorithms library                              | <algorithm>, <numeric>                   |
| 26.3.6    | Execution policies                              | <execution>                              |
| 27.3      | String view classes                             | <string_view>                            |
| 27.4      | String classes                                  | <string>                                 |
| 27.5      | Null-terminated sequence utilities              | <cstring>, <cwchar>                      |
| 28.2      | Primitive numeric conversions                   | <charconv>                               |
| 29.5      | Random number generation                        | <random>                                 |
| 29.7      | Mathematical functions for floating-point types | <cmath>                                  |
| 32.5      | Atomics                                         | <atomic>                                 |

<sup>4</sup> The *hosted library facilities* are the set of facilities described in this document that are required for hosted implementations, but not required for freestanding implementations. A freestanding implementation provides a (possibly empty) implementation-defined subset of the hosted library facilities. Unless otherwise specified, the requirements on each declaration, entity, and macro provided in this way are the same as the corresponding requirements for a hosted implementation, except that not all of the members of the namespaces are required to be present.

<sup>5</sup> A freestanding implementation provides deleted definitions (9.6.3) for a (possibly empty) implementation-defined subset of the namespace-scope functions and function templates from the hosted library facilities.

[Note 1: An implementation can provide a deleted definition so that the result of overload resolution does not silently change when migrating a program from a freestanding implementation to a hosted implementation. — end note]

### 16.4.3 Using the library [using]

#### 16.4.3.1 Overview [using.overview]

<sup>1</sup> Subclause 16.4.3 describes how a C++ program gains access to the facilities of the C++ standard library. 16.4.3.2 describes effects during translation phase 4, while 16.4.3.3 describes effects during phase 8 (5.2).

#### 16.4.3.2 Headers [using.headers]

<sup>1</sup> The entities in the C++ standard library are defined in headers, whose contents are made available to a translation unit when it contains the appropriate `#include` preprocessing directive (15.3) or the appropriate `import` declaration (10.3).

<sup>2</sup> A translation unit may include library headers in any order (5.1). Each may be included more than once, with no effect different from being included exactly once, except that the effect of including either `<cassert>` (19.3.2) or `<assert.h>` (17.15) depends each time on the lexically current definition of `NDEBUG`.<sup>138</sup>

<sup>3</sup> A translation unit shall include a header only outside of any declaration or definition and, in the case of a module unit, only in its *global-module-fragment*, and shall include the header or import the corresponding header unit lexically before the first reference in that translation unit to any of the entities declared in that header. No diagnostic is required.

#### 16.4.3.3 Linkage [using.linkage]

<sup>1</sup> Entities in the C++ standard library have external linkage (6.7). Unless otherwise specified, objects and functions have the default `extern "C++"` linkage (9.12).

<sup>2</sup> Whether a name from the C standard library declared with external linkage has `extern "C"` or `extern "C++"` linkage is implementation-defined. It is recommended that an implementation use `extern "C++"` linkage for this purpose.<sup>139</sup>

<sup>3</sup> Objects and functions defined in the library and required by a C++ program are included in the program prior to program startup.

<sup>4</sup> See also replacement functions (16.4.5.6), runtime changes (16.4.5.7).

### 16.4.4 Requirements on types and expressions [utility.requirements]

#### 16.4.4.1 General [utility.requirements.general]

<sup>1</sup> 16.4.4.2 describes requirements on types and expressions used to instantiate templates defined in the C++ standard library. 16.4.4.3 describes the requirements on swappable types and swappable expressions. 16.4.4.4 describes the requirements on pointer-like types that support null values. 16.4.4.5 describes the requirements on hash function objects. 16.4.4.6 describes the requirements on storage allocators.

#### 16.4.4.2 Template argument requirements [utility.arg.requirements]

<sup>1</sup> The template definitions in the C++ standard library refer to various named requirements whose details are set out in Tables 28–35. In these tables,

- (1.1) — `T` denotes an object or reference type to be supplied by a C++ program instantiating a template,
- (1.2) — `a`, `b`, and `c` denote values of type (possibly `const`) `T`,
- (1.3) — `s` and `t` denote modifiable lvalues of type `T`,

<sup>138</sup>) This is the same as the C standard library.

<sup>139</sup>) The only reliable way to declare an object or function signature from the C standard library is by including the header that declares it, notwithstanding the latitude granted in ISO/IEC 9899:2024, 7.1.4.

- (1.4) — **u** denotes an identifier,
- (1.5) — **rv** denotes an rvalue of type **T**, and
- (1.6) — **v** denotes an lvalue of type (possibly **const**) **T** or an rvalue of type **const T**.

**Table 28** — *Cpp17EqualityComparable* requirements [tab:cpp17.equalitycomparable]

| Expression    | Return type                                                  | Requirement                                                                                                                                                                                                                                                                                     |
|---------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>a == b</b> | <code>decltype(a == b)</code> models <i>boolean-testable</i> | <p><b>==</b> is an equivalence relation, that is, it has the following properties:</p> <ul style="list-style-type: none"> <li>— For all <b>a</b>, <b>a == a</b>.</li> <li>— If <b>a == b</b>, then <b>b == a</b>.</li> <li>— If <b>a == b</b> and <b>b == c</b>, then <b>a == c</b>.</li> </ul> |

**Table 29** — *Cpp17LessThanComparable* requirements [tab:cpp17.lessthancomparable]

| Expression      | Return type                                                    | Requirement                                           |
|-----------------|----------------------------------------------------------------|-------------------------------------------------------|
| <b>a &lt; b</b> | <code>decltype(a &lt; b)</code> models <i>boolean-testable</i> | <b>&lt;</b> is a strict weak ordering relation (26.8) |

**Table 30** — *Cpp17DefaultConstructible* requirements [tab:cpp17.defaultconstructible]

| Expression                | Post-condition                                                           |
|---------------------------|--------------------------------------------------------------------------|
| <b>T t;</b>               | object <b>t</b> is default-initialized                                   |
| <b>T u{ };</b>            | object <b>u</b> is value-initialized or aggregate-initialized            |
| <b>T()</b><br><b>T{ }</b> | an object of type <b>T</b> is value-initialized or aggregate-initialized |

**Table 31** — *Cpp17MoveConstructible* requirements [tab:cpp17.moveconstructible]

| Expression                                                                                                                                                                                                                                                | Post-condition                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>T u = rv;</b>                                                                                                                                                                                                                                          | <b>u</b> is equivalent to the value of <b>rv</b> before the construction     |
| <b>T(rv)</b>                                                                                                                                                                                                                                              | <b>T(rv)</b> is equivalent to the value of <b>rv</b> before the construction |
| <b>rv</b> 's state is unspecified<br><i>[Note 1: rv must still meet the requirements of the library component that is using it. The operations listed in those requirements must work as specified whether rv has been moved from or not. — end note]</i> |                                                                              |

**Table 32** — *Cpp17CopyConstructible* requirements (in addition to *Cpp17MoveConstructible*) [tab:cpp17.copyconstructible]

| Expression      | Post-condition                                                      |
|-----------------|---------------------------------------------------------------------|
| <b>T u = v;</b> | the value of <b>v</b> is unchanged and is equivalent to <b>u</b>    |
| <b>T(v)</b>     | the value of <b>v</b> is unchanged and is equivalent to <b>T(v)</b> |

Table 33 — *Cpp17MoveAssignable* requirements [tab:cpp17.moveassignable]

| Expression                                                                                                                                                                                                                                                                                    | Return type | Return value | Post-condition                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|--------------|-------------------------------------------------------------------------------------------------------|
| <b>t = rv</b>                                                                                                                                                                                                                                                                                 | T&          | t            | If t and rv do not refer to the same object, t is equivalent to the value of rv before the assignment |
| rv's state is unspecified.<br>[Note 2: rv must still meet the requirements of the library component that is using it, whether or not t and rv refer to the same object. The operations listed in those requirements must work as specified whether rv has been moved from or not. — end note] |             |              |                                                                                                       |

Table 34 — *Cpp17CopyAssignable* requirements (in addition to *Cpp17MoveAssignable*) [tab:cpp17.copyassignable]

| Expression   | Return type | Return value | Post-condition                                    |
|--------------|-------------|--------------|---------------------------------------------------|
| <b>t = v</b> | T&          | t            | t is equivalent to v, the value of v is unchanged |

Table 35 — *Cpp17Destructible* requirements [tab:cpp17.destructible]

| Expression                                                                               | Post-condition              |
|------------------------------------------------------------------------------------------|-----------------------------|
| <b>a.~T()</b>                                                                            | No exception is propagated. |
| [Note 3: Array types and non-object types are not <i>Cpp17Destructible</i> . — end note] |                             |

#### 16.4.4.3 Swappable requirements

[swappable.requirements]

- <sup>1</sup> This subclause provides definitions for swappable types and expressions. In these definitions, let **t** denote an expression of type T, and let **u** denote an expression of type U.
- <sup>2</sup> An object **t** is *swappable with* an object **u** if and only if
  - (2.1) — the expressions **swap(t, u)** and **swap(u, t)** are valid when evaluated in the context described below, and
  - (2.2) — these expressions have the following effects:
    - (2.2.1) — the object referred to by **t** has the value originally held by **u** and
    - (2.2.2) — the object referred to by **u** has the value originally held by **t**.
- <sup>3</sup> The context in which **swap(t, u)** and **swap(u, t)** are evaluated shall ensure that a binary non-member function named “swap” is selected via overload resolution (12.2) on a candidate set that includes:
  - (3.1) — the two **swap** function templates defined in <utility> (22.2.1) and
  - (3.2) — the lookup set produced by argument-dependent lookup (6.5.4).

[Note 1: If T and U are both fundamental types or arrays of fundamental types and the declarations from the header <utility> are in scope, the overall lookup set described above is equivalent to that of the qualified name lookup applied to the expression **std::swap(t, u)** or **std::swap(u, t)** as appropriate. — end note]

[Note 2: It is unspecified whether a library component that has a swappable requirement includes the header <utility> to ensure an appropriate evaluation context. — end note]
- <sup>4</sup> An rvalue or lvalue **t** is *swappable* if and only if **t** is swappable with any rvalue or lvalue, respectively, of type T.
- <sup>5</sup> A type X meets the *Cpp17Swappable* requirements if lvalues of type X are swappable.
- <sup>6</sup> A type X meeting any of the iterator requirements (24.3) meets the *Cpp17ValueSwappable* requirements if, for any dereferenceable object **x** of type X, **\*x** is swappable.
- <sup>7</sup> [Example 1: User code can ensure that the evaluation of **swap** calls is performed in an appropriate context under the various conditions as follows:



```

#include <cassert>
#include <utility>

// Preconditions: std::forward<T>(t) is swappable with std::forward<U>(u).
template<class T, class U>
void value_swap(T&& t, U&& u) {
 using std::swap;
 swap(std::forward<T>(t), std::forward<U>(u)); // OK, uses "swappable with" conditions
 // for rvalues and lvalues
}

// Preconditions: T meets the Cpp17Swappable requirements.
template<class T>
void lv_swap(T& t1, T& t2) {
 using std::swap;
 swap(t1, t2); // OK, uses swappable conditions for lvalues of type T
}

namespace N {
 struct A { int m; };
 struct Proxy { A* a; };
 Proxy proxy(A& a) { return Proxy{ &a }; }

 void swap(A& x, Proxy p) {
 std::swap(x.m, p.a->m); // OK, uses context equivalent to swappable
 // conditions for fundamental types
 }
 void swap(Proxy p, A& x) { swap(x, p); } // satisfy symmetry constraint
}

int main() {
 int i = 1, j = 2;
 lv_swap(i, j);
 assert(i == 2 && j == 1);

 N::A a1 = { 5 }, a2 = { -5 };
 value_swap(a1, proxy(a2));
 assert(a1.m == -5 && a2.m == 5);
}
— end example]

```

#### 16.4.4.4 Cpp17NullablePointer requirements [nullablepointer.requirements]

- <sup>1</sup> A Cpp17NullablePointer type is a pointer-like type that supports null values. A type P meets the Cpp17-NullablePointer requirements if
  - (1.1) — P meets the Cpp17EqualityComparable, Cpp17DefaultConstructible, Cpp17CopyConstructible, Cpp17-CopyAssignable, Cpp17Swappable, and Cpp17Destructible requirements,
  - (1.2) — the expressions shown in Table 36 are valid and have the indicated semantics, and
  - (1.3) — P meets all the other requirements of this subclause.
- <sup>2</sup> A value-initialized object of type P produces the null value of the type. The null value shall be equivalent only to itself. A default-initialized object of type P may have an indeterminate or erroneous value.
 

[Note 1: Operations involving indeterminate values can cause undefined behavior, and operations involving erroneous values can cause erroneous behavior (6.8.5). — end note]
- <sup>3</sup> An object p of type P can be contextually converted to bool (7.3). The effect shall be as if p != nullptr had been evaluated in place of p.
- <sup>4</sup> No operation which is part of the Cpp17NullablePointer requirements shall exit via an exception.
- <sup>5</sup> In Table 36, u denotes an identifier, t denotes a non-const lvalue of type P, a and b denote values of type (possibly const) P, and np denotes a value of type (possibly const) std::nullptr\_t.



Table 36 — *Cpp17NullablePointer* requirements [tab:cpp17.nullablepointer]

| Expression             | Return type                                                                      | Operational semantics                   |
|------------------------|----------------------------------------------------------------------------------|-----------------------------------------|
| P u(np);<br>P u = np;  |                                                                                  | <i>Postconditions:</i> u == nullptr     |
| P(np)                  |                                                                                  | <i>Postconditions:</i> P(np) == nullptr |
| t = np                 | P&                                                                               | <i>Postconditions:</i> t == nullptr     |
| a != b                 | decltype(a != b) models<br><i>boolean-testable</i>                               | !(a == b)                               |
| a == np<br><br>np == a | decltype(a == np) and<br>decltype(np == a) each model<br><i>boolean-testable</i> | a == P()                                |
| a != np<br><br>np != a | decltype(a != np) and<br>decltype(np != a) each model<br><i>boolean-testable</i> | !(a == np)                              |

16.4.4.5 *Cpp17Hash* requirements

[hash.requirements]

- <sup>1</sup> A type H meets the *Cpp17Hash* requirements if
- (1.1) — it is a function object type (22.10),
  - (1.2) — it meets the *Cpp17CopyConstructible* (Table 32) and *Cpp17Destructible* (Table 35) requirements, and
  - (1.3) — the expressions shown in Table 37 are valid and have the indicated semantics.
- <sup>2</sup> Given Key is an argument type for function objects of type H, in Table 37 h is a value of type (possibly const) H, u is an lvalue of type Key, and k is a value of a type convertible to (possibly const) Key.

Table 37 — *Cpp17Hash* requirements [tab:cpp17.hash]

| Expression | Return type | Requirement                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| h(k)       | size_t      | The value returned shall depend only on the argument k for the duration of the program.<br>[Note 1: Thus all evaluations of the expression h(k) with the same value for k yield the same result for a given execution of the program. — end note]<br>For two different values t1 and t2, the probability that h(t1) and h(t2) compare equal should be very small, approaching 1.0 / numeric_limits<size_t>::max(). |
| h(u)       | size_t      | Shall not modify u.                                                                                                                                                                                                                                                                                                                                                                                                |

16.4.4.6 *Cpp17Allocator* requirements

[allocator.requirements]

## 16.4.4.6.1 General

[allocator.requirements.general]

- <sup>1</sup> The library describes a standard set of requirements for *allocators*, which are class-type objects that encapsulate the information about an allocation model. This information includes the knowledge of pointer types, the type of their difference, the type of the size of objects in this allocation model, as well as the memory allocation and deallocation primitives for it. All of the string types (Clause 27), containers (Clause 23) (except `array` and `inplace_vector`), string buffers and string streams (Clause 31), and `match_results` (28.6) are parameterized in terms of allocators.
- <sup>2</sup> In 16.4.4.6,
- (2.1) — T, U, C denote any *cv*-unqualified object type (6.9.1),
  - (2.2) — X denotes an allocator class for type T,
  - (2.3) — Y denotes the corresponding allocator class for type U,

- (2.4) — `XX` denotes the type `allocator_traits<X>`,
- (2.5) — `YY` denotes the type `allocator_traits<Y>`,
- (2.6) — `a`, `a1`, `a2` denote lvalues of type `X`,
- (2.7) — `u` denotes the name of a variable being declared,
- (2.8) — `b` denotes a value of type `Y`,
- (2.9) — `c` denotes a pointer of type `C*` through which indirection is valid,
- (2.10) — `p` denotes a value of type `XX::pointer` obtained by calling `a1.allocate`, where `a1 == a`,
- (2.11) — `q` denotes a value of type `XX::const_pointer` obtained by conversion from a value `p`,
- (2.12) — `r` denotes a value of type `T&` obtained by the expression `*p`,
- (2.13) — `w` denotes a value of type `XX::void_pointer` obtained by conversion from a value `p`,
- (2.14) — `x` denotes a value of type `XX::const_void_pointer` obtained by conversion from a value `q` or a value `w`,
- (2.15) — `y` denotes a value of type `XX::const_void_pointer` obtained by conversion from a result value of `YY::allocate`, or else a value of type (possibly `const`) `std::nullptr_t`,
- (2.16) — `n` denotes a value of type `XX::size_type`,
- (2.17) — `Args` denotes a template parameter pack, and
- (2.18) — `args` denotes a function parameter pack with the pattern `Args&&`.

3 The class template `allocator_traits` (20.2.9) supplies a uniform interface to all allocator types. This subclause describes the requirements on allocator types and thus on types used to instantiate `allocator_traits`. A requirement is optional if a default for a given type or expression is specified. Within the standard library `allocator_traits` template, an optional requirement that is not supplied by an allocator is replaced by the specified default type or expression.

[Note 1: There are no program-defined specializations of `allocator_traits`. — end note]

typename `X::pointer`

4 *Remarks:* Default: `T*`

typename `X::const_pointer`

5 *Mandates:* `XX::pointer` is convertible to `XX::const_pointer`.

6 *Remarks:* Default: `pointer_traits<XX::pointer>::rebind<const T>`

typename `X::void_pointer`

typename `Y::void_pointer`

7 *Mandates:* `XX::pointer` is convertible to `XX::void_pointer`. `XX::void_pointer` and `YY::void_pointer` are the same type.

8 *Remarks:* Default: `pointer_traits<XX::pointer>::rebind<void>`

typename `X::const_void_pointer`

typename `Y::const_void_pointer`

9 *Mandates:* `XX::pointer`, `XX::const_pointer`, and `XX::void_pointer` are convertible to `XX::const_void_pointer`. `XX::const_void_pointer` and `YY::const_void_pointer` are the same type.

10 *Remarks:* Default: `pointer_traits<XX::pointer>::rebind<const void>`

typename `X::value_type`

11 *Result:* Identical to `T`.

typename `X::size_type`

12 *Result:* An unsigned integer type that can represent the size of the largest object in the allocation model.

13 *Remarks:* Default: `make_unsigned_t<XX::difference_type>`

`typename X::difference_type`

14     *Result:* A signed integer type that can represent the difference between any two pointers in the allocation model.

15     *Remarks:* Default: `pointer_traits<XX::pointer>::difference_type`

`typename X::rebind<U>::other`

16     *Result:* Y

17     *Postconditions:* For all U (including T), `YY::rebind_alloc<T>` is X.

18     *Remarks:* If `Allocator` is a class template specialization of the form `SomeAllocator<T, Args>`, where `Args` is zero or more type arguments, and `Allocator` does not supply a `rebind` member template, the standard `allocator_traits` template uses `SomeAllocator<U, Args>` in place of `Allocator::rebind<U>::other` by default. For allocator types that are not template specializations of the above form, no default is provided.

19     [*Note 2:* The member class template `rebind` of X is effectively a typedef template. In general, if the name `Allocator` is bound to `SomeAllocator<T>`, then `Allocator::rebind<U>::other` is the same type as `SomeAllocator<U>`, where `SomeAllocator<T>::value_type` is T and `SomeAllocator<U>::value_type` is U. — *end note*]

`*p`

20     *Result:* T&

`*q`

21     *Result:* const T&

22     *Postconditions:* `*q` refers to the same object as `*p`.

`p->m`

23     *Result:* Type of `T::m`.

24     *Preconditions:* `(*p).m` is well-defined.

25     *Effects:* Equivalent to `(*p).m`.

`q->m`

26     *Result:* Type of `T::m`.

27     *Preconditions:* `(*q).m` is well-defined.

28     *Effects:* Equivalent to `(*q).m`.

`static_cast<XX::pointer>(w)`

29     *Result:* `XX::pointer`

30     *Postconditions:* `static_cast<XX::pointer>(w) == p`.

`static_cast<XX::const_pointer>(x)`

31     *Result:* `XX::const_pointer`

32     *Postconditions:* `static_cast<XX::const_pointer>(x) == q`.

`pointer_traits<XX::pointer>::pointer_to(r)`

33     *Result:* `XX::pointer`

34     *Postconditions:* Same as p.

`a.allocate(n)`

35     *Result:* `XX::pointer`

36     *Effects:* Memory is allocated for an array of n T and such an object is created but array elements are not constructed.

[*Example 1:* When reusing storage denoted by some pointer value p, `launder(reinterpret_cast<T*>(new (p) byte[n * sizeof(T)]))` can be used to implicitly create a suitable array object and obtain a pointer to it. — *end example*]

37 *Throws:* `allocate` may throw an appropriate exception.

38 [Note 3: It is intended that `a.allocate` be an efficient means of allocating a single object of type `T`, even when `sizeof(T)` is small. That is, there is no need for a container to maintain its own free list. — end note]

39 *Remarks:* If `n == 0`, the return value is unspecified.

`a.allocate(n, y)`

40 *Result:* `XX::pointer`

41 *Effects:* Same as `a.allocate(n)`. The use of `y` is unspecified, but it is intended as an aid to locality.

42 *Remarks:* Default: `a.allocate(n)`

`a.allocate_at_least(n)`

43 *Result:* `allocation_result<XX::pointer, XX::size_type>`

44 *Returns:* `allocation_result<XX::pointer, XX::size_type>{ptr, count}` where `ptr` is memory allocated for an array of `count` `T` and such an object is created but array elements are not constructed, such that `count ≥ n`. If `n == 0`, the return value is unspecified.

45 *Throws:* `allocate_at_least` may throw an appropriate exception.

46 *Remarks:* Default: `{a.allocate(n), n}`.

`a.deallocate(p, n)`

47 *Result:* (not used)

48 *Preconditions:*

(48.1) — If `p` is memory that was obtained by a call to `a.allocate_at_least`, let `ret` be the value returned and `req` be the value passed as the first argument of that call. `p` is equal to `ret.ptr` and `n` is a value such that `req ≤ n ≤ ret.count`.

(48.2) — Otherwise, `p` is a pointer value obtained from `allocate`. `n` equals the value passed as the first argument to the invocation of `allocate` which returned `p`.

`p` has not been invalidated by an intervening call to `deallocate`.

49 *Throws:* Nothing.

`a.max_size()`

50 *Result:* `XX::size_type`

51 *Returns:* The largest value `n` that can meaningfully be passed to `a.allocate(n)`.

52 *Remarks:* Default: `numeric_limits<size_type>::max() / sizeof(value_type)`

`a1 == a2`

53 *Result:* `bool`

54 *Returns:* `true` only if storage allocated from each can be deallocated via the other.

55 *Throws:* Nothing.

56 *Remarks:* `operator==` shall be reflexive, symmetric, and transitive.

`a1 != a2`

57 *Result:* `bool`

58 *Returns:* `!(a1 == a2)`.

`a == b`

59 *Result:* `bool`

60 *Returns:* `a == YY::rebind_alloc<T>(b)`.

`a != b`

61 *Result:* `bool`

62 *Returns:* `!(a == b)`.

```

X u(a);
X u = a;
63 Postconditions: u == a
64 Throws: Nothing.

X u(b);
65 Postconditions: Y(u) == b and u == X(b).
66 Throws: Nothing.

X u(std::move(a));
X u = std::move(a);
67 Postconditions: The value of a is unchanged and is equal to u.
68 Throws: Nothing.

X u(std::move(b));
69 Postconditions: u is equal to the prior value of X(b).
70 Throws: Nothing.

a.construct(c, args...)
71 Result: (not used)
72 Effects: Constructs an object of type C at c.
73 Remarks: Default: construct_at(c, std::forward<Args>(args)...)

a.destroy(c)
74 Result: (not used)
75 Effects: Destroys the object at c.
76 Remarks: Default: destroy_at(c)

a.select_on_container_copy_construction()
77 Result: X
78 Returns: Typically returns either a or X().
79 Remarks: Default: return a;

typename X::propagate_on_container_copy_assignment
80 Result: Identical to or derived from true_type or false_type.
81 Returns: true_type only if an allocator of type X should be copied when the client container is
copy-assigned; if so, X shall meet the Cpp17CopyAssignable requirements (Table 34) and the copy
operation shall not throw exceptions.
82 Remarks: Default: false_type

typename X::propagate_on_container_move_assignment
83 Result: Identical to or derived from true_type or false_type.
84 Returns: true_type only if an allocator of type X should be moved when the client container is
move-assigned; if so, X shall meet the Cpp17MoveAssignable requirements (Table 33) and the move
operation shall not throw exceptions.
85 Remarks: Default: false_type

typename X::propagate_on_container_swap
86 Result: Identical to or derived from true_type or false_type.
87 Returns: true_type only if an allocator of type X should be swapped when the client container is
swapped; if so, X shall meet the Cpp17Swappable requirements (16.4.4.3) and the swap operation shall
not throw exceptions.
88 Remarks: Default: false_type

```

typename X::is\_always\_equal

89     *Result:* Identical to or derived from `true_type` or `false_type`.

90     *Returns:* `true_type` only if the expression `a1 == a2` is guaranteed to be `true` for any two (possibly `const`) values `a1`, `a2` of type `X`.

91     *Remarks:* Default: `is_empty<X>::type`

92     An allocator type `X` shall meet the *Cpp17CopyConstructible* requirements (Table 32). The `XX::pointer`, `XX::const_pointer`, `XX::void_pointer`, and `XX::const_void_pointer` types shall meet the *Cpp17NullablePointer* requirements (Table 36). No constructor, comparison operator function, copy operation, move operation, or swap operation on these pointer types shall exit via an exception. `XX::pointer` and `XX::const_pointer` shall also meet the requirements for a *Cpp17RandomAccessIterator* (24.3.5.7) and the additional requirement that, when `p` and `(p + n)` are dereferenceable pointer values for some integral value `n`,

`addressof(*(p + n)) == addressof(*p) + n`

is `true`.

93     Let `x1` and `x2` denote objects of (possibly different) types `XX::void_pointer`, `XX::const_void_pointer`, `XX::pointer`, or `XX::const_pointer`. Then, `x1` and `x2` are *equivalently-valued* pointer values, if and only if both `x1` and `x2` can be explicitly converted to the two corresponding objects `px1` and `px2` of type `XX::const_pointer`, using a sequence of `static_casts` using only these four types, and the expression `px1 == px2` evaluates to `true`.

94     Let `w1` and `w2` denote objects of type `XX::void_pointer`. Then for the expressions

`w1 == w2`

`w1 != w2`

either or both objects may be replaced by an equivalently-valued object of type `XX::const_void_pointer` with no change in semantics.

95     Let `p1` and `p2` denote objects of type `XX::pointer`. Then for the expressions

`p1 == p2`

`p1 != p2`

`p1 < p2`

`p1 <= p2`

`p1 >= p2`

`p1 > p2`

`p1 - p2`

either or both objects may be replaced by an equivalently-valued object of type `XX::const_pointer` with no change in semantics.

96     An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct` or `destroy` members may be called. If a type cannot be used with a particular allocator, the allocator class or the call to `construct` or `destroy` may fail to instantiate.

97     If the alignment associated with a specific over-aligned type is not supported by an allocator, instantiation of the allocator for that type may fail. The allocator also may silently ignore the requested alignment.

[Note 4: Additionally, the member function `allocate` for that type can fail by throwing an object of type `bad_alloc`. — end note]

98     [Example 2: The following is an allocator class template supporting the minimal interface that meets the requirements of 16.4.4.6.1:

```
template<class T>
struct SimpleAllocator {
 using value_type = T;
 SimpleAllocator(ctor args);

 template<class U> SimpleAllocator(const SimpleAllocator<U>& other);

 T* allocate(std::size_t n);
 void deallocate(T* p, std::size_t n);

 template<class U> bool operator==(const SimpleAllocator<U>& rhs) const;
};
```

— *end example*]

99 The following exposition-only concept defines the minimal requirements on an Allocator type.

```
namespace std {
 template<class Alloc>
 concept simple-allocator =
 requires(Alloc alloc, size_t n) {
 { *alloc.allocate(n) } -> same_as<typename Alloc::value_type&>;
 { alloc.deallocate(alloc.allocate(n), n) };
 } &&
 copy_constructible<Alloc> &&
 equality_comparable<Alloc>;
}
```

A type Alloc models *simple-allocator* if it meets the requirements of 16.4.4.6.1.

#### 16.4.4.6.2 Allocator completeness requirements [allocator.requirements.completeness]

<sup>1</sup> If X is an allocator class for type T, X additionally meets the allocator completeness requirements if, whether or not T is a complete type:

- (1.1) — X is a complete type, and
- (1.2) — all the member types of `allocator_traits<X>` (20.2.9) other than `value_type` are complete types.

### 16.4.5 Constraints on programs [constraints]

#### 16.4.5.1 Overview [constraints.overview]

<sup>1</sup> Subclause 16.4.5 describes restrictions on C++ programs that use the facilities of the C++ standard library. The following subclauses specify constraints on the program's use of namespaces (16.4.5.2.1), its use of various reserved names (16.4.5.3), its use of headers (16.4.5.4), its use of standard library classes as base classes (16.4.5.5), its definitions of replacement functions (16.4.5.6), and its installation of handler functions during execution (16.4.5.7).

#### 16.4.5.2 Namespace use [namespace.constraints]

##### 16.4.5.2.1 Namespace std [namespace.std]

<sup>1</sup> Unless otherwise specified, the behavior of a C++ program is undefined if it adds declarations or definitions to namespace `std` or to a namespace within namespace `std`.

<sup>2</sup> Unless explicitly prohibited, a program may add a template specialization for any standard library class template to namespace `std` provided that

- (2.1) — the added declaration depends on at least one program-defined type, and
- (2.2) — the specialization meets the standard library requirements for the original template.<sup>140</sup>

<sup>3</sup> The behavior of a C++ program is undefined if it declares an explicit or partial specialization of any standard library variable template, except where explicitly permitted by the specification of that variable template.

[Note 1: The requirements on an explicit or partial specialization are stated by each variable template that grants such permission. — *end note*]

<sup>4</sup> The behavior of a C++ program is undefined if it declares

- (4.1) — an explicit specialization of any member function of a standard library class template, or
- (4.2) — an explicit specialization of any member function template of a standard library class or class template, or
- (4.3) — an explicit or partial specialization of any member class template of a standard library class or class template, or
- (4.4) — a deduction guide for any standard library class template.

<sup>5</sup> A program may explicitly instantiate a class template defined in the standard library only if the declaration

- (5.1) — depends on the name of at least one program-defined type, and
- (5.2) — the instantiation meets the standard library requirements for the original template.

<sup>140</sup> Any library code that instantiates other library templates must be prepared to work adequately with any user-supplied specialization that meets the minimum requirements of this document.

- <sup>6</sup> Let *F* denote a standard library function (16.4.6.4), a standard library static member function, or a specialization of a standard library function template. Unless *F* is designated an *addressable function*, the behavior of a C++ program is unspecified (possibly ill-formed) if it explicitly or implicitly attempts to form a pointer to *F*.

[Note 2: Possible means of forming such pointers include application of the unary `&` operator (7.6.2.2), `addressof` (20.2.11), or a function-to-pointer standard conversion (7.3.4). — end note]

Moreover, the behavior of a C++ program is unspecified (possibly ill-formed) if it attempts to form a reference to *F* or if it attempts to form a pointer-to-member designating either a standard library non-static member function (16.4.6.5) or a specialization of a standard library member function template.

- <sup>7</sup> Let *F* denote a standard library function or function template. Unless *F* is designated an addressable function, it is unspecified if or how a reflection value designating the associated entity can be formed. For any value *p* of type `meta::info` that represents a reflection of a parameter of *F*, it is unspecified if `has_identifier(p)` returns `true` or `false`, and if `meta::has_identifier(p)` is `true`, then the NTMBS produced by `meta::identifier_of(p)` and `meta::u8identifier_of(p)` is unspecified.

[Note 3: For example, it is possible that `std::meta::members_of` will not return reflections of standard library functions that an implementation handles through an extra-linguistic mechanism. — end note]

- <sup>8</sup> Let *C* denote a standard library class or class template specialization. It is unspecified if or how a reflection value can be formed to any private member of *C*, or what the names of such members may be.
- <sup>9</sup> A translation unit shall not declare namespace `std` to be an inline namespace (9.9.2).

#### 16.4.5.2.2 Namespace `posix`

[namespace.posix]

- <sup>1</sup> The behavior of a C++ program is undefined if it adds declarations or definitions to namespace `posix` or to a namespace within namespace `posix` unless otherwise specified. The namespace `posix` is reserved for use by ISO/IEC/IEEE 9945 and other POSIX standards.

#### 16.4.5.2.3 Namespaces for future standardization

[namespace.future]

- <sup>1</sup> Top-level namespaces whose *namespace-name* consists of `std` followed by one or more *digits* (5.11) are reserved for future standardization. The behavior of a C++ program is undefined if it adds declarations or definitions to such a namespace.

[Example 1: The top-level namespace `std2` is reserved for use by future revisions of this International Standard. — end example]

#### 16.4.5.3 Reserved names

[reserved.names]

##### 16.4.5.3.1 General

[reserved.names.general]

- <sup>1</sup> The C++ standard library reserves the following kinds of names:
- (1.1) — macros
  - (1.2) — global names
  - (1.3) — names with external linkage
- <sup>2</sup> If a program declares or defines a name in a context where it is reserved, other than as explicitly allowed by Clause 16, its behavior is undefined.

##### 16.4.5.3.2 Zombie names

[zombie.names]

- <sup>1</sup> In namespace `std`, the names shown in Table 38 are reserved for previous standardization:
- <sup>2</sup> The names shown in Table 39 are reserved as members for previous standardization, and may not be used as a name for object-like macros in portable code:
- <sup>3</sup> The names shown in Table 40 are reserved as member functions for previous standardization, and may not be used as a name for function-like macros in portable code:
- <sup>4</sup> The header names shown in Table 41 are reserved for previous standardization:

##### 16.4.5.3.3 Macro names

[macro.names]

- <sup>1</sup> A translation unit that includes a standard library header shall not `#define` or `#undef` names declared in any standard library header.



Table 38 — Zombie names in namespace `std` [tab:zombie.names.std]

|                                   |                                   |                                         |
|-----------------------------------|-----------------------------------|-----------------------------------------|
| <code>auto_ptr</code>             | <code>generate_header</code>      | <code>pointer_to_binary_function</code> |
| <code>auto_ptr_ref</code>         | <code>get_pointer_safety</code>   | <code>pointer_to_unary_function</code>  |
| <code>binary_function</code>      | <code>get_temporary_buffer</code> | <code>ptr_fun</code>                    |
| <code>binary_negate</code>        | <code>get_unexpected</code>       | <code>random_shuffle</code>             |
| <code>bind1st</code>              | <code>gets</code>                 | <code>raw_storage_iterator</code>       |
| <code>bind2nd</code>              | <code>is_literal_type</code>      | <code>result_of</code>                  |
| <code>binder1st</code>            | <code>is_literal_type_v</code>    | <code>result_of_t</code>                |
| <code>binder2nd</code>            | <code>istrstream</code>           | <code>return_temporary_buffer</code>    |
| <code>codecvt_mode</code>         | <code>little_endian</code>        | <code>set_unexpected</code>             |
| <code>codecvt_utf16</code>        | <code>mem_fun1_ref_t</code>       | <code>strstream</code>                  |
| <code>codecvt_utf8</code>         | <code>mem_fun1_t</code>           | <code>strstreambuf</code>               |
| <code>codecvt_utf8_utf16</code>   | <code>mem_fun_ref_t</code>        | <code>unary_function</code>             |
| <code>const_mem_fun1_ref_t</code> | <code>mem_fun_ref</code>          | <code>unary_negate</code>               |
| <code>const_mem_fun1_t</code>     | <code>mem_fun_t</code>            | <code>uncaught_exception</code>         |
| <code>const_mem_fun_ref_t</code>  | <code>mem_fun</code>              | <code>undeclare_no_pointers</code>      |
| <code>const_mem_fun_t</code>      | <code>not1</code>                 | <code>undeclare_reachable</code>        |
| <code>consume_header</code>       | <code>not2</code>                 | <code>unexpected_handler</code>         |
| <code>declare_no_pointers</code>  | <code>ostrstream</code>           | <code>wbuffer_convert</code>            |
| <code>declare_reachable</code>    | <code>pointer_safety</code>       | <code>wstring_convert</code>            |

Table 39 — Zombie object-like macros [tab:zombie.names.objmacro]

|                                  |                        |                                   |
|----------------------------------|------------------------|-----------------------------------|
| <code>argument_type</code>       | <code>op</code>        | <code>second_argument_type</code> |
| <code>first_argument_type</code> | <code>open_mode</code> | <code>seek_dir</code>             |
| <code>io_state</code>            | <code>preferred</code> | <code>strict</code>               |

Table 40 — Zombie function-like macros [tab:zombie.names.fnmacro]

|                        |                     |                         |                     |                      |                       |
|------------------------|---------------------|-------------------------|---------------------|----------------------|-----------------------|
| <code>converted</code> | <code>freeze</code> | <code>from_bytes</code> | <code>pcount</code> | <code>stosscc</code> | <code>to_bytes</code> |
|------------------------|---------------------|-------------------------|---------------------|----------------------|-----------------------|

## 16.4.5.3.4 External linkage

[extern.names]

- <sup>1</sup> Each name declared as an object with external linkage in a header is reserved to the implementation to designate that library object with external linkage,<sup>141</sup> both in namespace `std` and in the global namespace.
- <sup>2</sup> Each global function signature declared with external linkage in a header is reserved to the implementation to designate that function signature with external linkage.<sup>142</sup>
- <sup>3</sup> Each name from the C standard library declared with external linkage is reserved to the implementation for use as a name with `extern "C"` linkage, both in namespace `std` and in the global namespace.
- <sup>4</sup> Each function signature from the C standard library declared with external linkage is reserved to the implementation for use as a function signature with both `extern "C"` and `extern "C++"` linkage,<sup>143</sup> or as a name of namespace scope in the global namespace.

<sup>141</sup> The list of such reserved names includes `errno`, declared or defined in `<cerrno>` (19.4.2).

<sup>142</sup> The list of such reserved function signatures with external linkage includes `setjmp(jmp_buf)`, declared or defined in `<csetjmp>` (17.14.3), and `va_end(va_list)`, declared or defined in `<cstdarg>` (17.14.2).

<sup>143</sup> The function signatures declared in `<cuchar>` (28.7.4), `<wchar>` (28.7.3), and `<cwctype>` (28.7.2) are always reserved, notwithstanding the restrictions imposed in subclause 4.5.1 of Amendment 1 to the C Standard for these headers.

Table 41 — Zombie headers [tab:zombie.names.header]

|                              |                                |                               |                              |                                |
|------------------------------|--------------------------------|-------------------------------|------------------------------|--------------------------------|
| <code>&lt;complex&gt;</code> | <code>&lt;codecvt&gt;</code>   | <code>&lt;cstdbool&gt;</code> | <code>&lt;ctgmath&gt;</code> | <code>&lt;strstream&gt;</code> |
| <code>&lt;ciso646&gt;</code> | <code>&lt;cstdalign&gt;</code> |                               |                              |                                |

**16.4.5.3.5 Types****[extern.types]**

- <sup>1</sup> For each type `T` from the C standard library, the types `::T` and `std::T` are reserved to the implementation and, when declared, `::T` shall be identical to `std::T`.

**16.4.5.3.6 User-defined literal suffixes****[usrlit.suffix]**

- <sup>1</sup> Literal suffix identifiers (12.6) that do not start with an underscore are reserved for future standardization. Literal suffix identifiers that contain a double underscore `__` are reserved for use by C++ implementations.

**16.4.5.4 Headers****[alt.headers]**

- <sup>1</sup> If a file with a name equivalent to the derived file name for one of the C++ standard library headers is not provided as part of the implementation, and a file with that name is placed in any of the standard places for a source file to be included (15.3), the behavior is undefined.

**16.4.5.5 Derived classes****[derived.classes]**

- <sup>1</sup> Virtual member function signatures defined for a base class in the C++ standard library may be overridden in a derived class defined in the program (11.7.3).

**16.4.5.6 Replacement functions****[replacement.functions]**

- <sup>1</sup> If a function defined in [Clause 17](#) through [Clause 33](#) and [Annex D](#) is specified as replaceable (9.6.5), the description of function semantics apply to both the default version defined by the C++ standard library and the replacement function defined by the program.

**16.4.5.7 Handler functions****[handler.functions]**

- <sup>1</sup> The C++ standard library provides a default version of the following handler function ([Clause 17](#)):
  - (1.1) — `terminate_handler`
  - <sup>2</sup> A C++ program may install different handler functions during execution, by supplying a pointer to a function defined in the program or the library as an argument to (respectively):
    - (2.1) — `set_new_handler`
    - (2.2) — `set_terminate`
 See also subclauses [17.6.4](#), Storage allocation errors, and [17.9](#), Exception handling.
  - <sup>3</sup> A C++ program can get a pointer to the current handler function by calling the following functions:
    - (3.1) — `get_new_handler`
    - (3.2) — `get_terminate`
  - <sup>4</sup> Calling the `set_*` and `get_*` functions shall not incur a data race ([6.10.2.2](#)). A call to any of the `set_*` functions synchronizes with subsequent calls to the same `set_*` function and to the corresponding `get_*` function.

**16.4.5.8 Other functions****[res.on.functions]**

- <sup>1</sup> In certain cases (replacement functions, handler functions, operations on types used to instantiate standard library template components), the C++ standard library depends on components supplied by a C++ program. If these components do not meet their requirements, this document places no requirements on the implementation.
- <sup>2</sup> In particular, the behavior is undefined in the following cases:
  - (2.1) — For replacement functions ([16.4.5.6](#)), if the installed replacement function does not implement the semantics of the applicable *Required behavior*: paragraph.
  - (2.2) — For handler functions ([17.6.4.3](#), [17.9.5.1](#)), if the installed handler function does not implement the semantics of the applicable *Required behavior*: paragraph.
  - (2.3) — For types used as template arguments when instantiating a template component, if the operations on the type do not implement the semantics of the applicable *Requirements* subclause ([16.4.4.6](#), [23.2](#), [24.3](#), [26.2](#), [29.2](#)). Operations on such types can report a failure by throwing an exception unless otherwise specified.
  - (2.4) — If any replacement function or handler function or destructor operation exits via an exception, unless specifically allowed in the applicable *Required behavior*: paragraph.

- (2.5) — If an incomplete type (6.9.1) is used as a template argument when instantiating a template component or evaluating a concept, unless specifically allowed for that component.

#### 16.4.5.9 Function arguments

[res.on.arguments]

- 1 Each of the following applies to all arguments to functions defined in the C++ standard library, unless explicitly stated otherwise.
  - (1.1) — If an argument to a function has an invalid value (such as a value outside the domain of the function or a pointer invalid for its intended use), the behavior is undefined.
  - (1.2) — If a function argument is described as being an array, the pointer actually passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are in fact valid.
  - (1.3) — If a function argument is bound to an rvalue reference parameter, the implementation may assume that this parameter is a unique reference to this argument, except that the argument passed to a move assignment operator may be a reference to `*this` (16.4.6.17).
 

[Note 1: If the type of a parameter is a forwarding reference (13.10.3.2) that is deduced to an lvalue reference type, then the argument is not bound to an rvalue reference. — end note]

[Note 2: If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g., by calling the function with the argument `std::move(x)`), the program is effectively asking that function to treat that lvalue as a temporary object. The implementation is free to optimize away aliasing checks which would possibly be needed if the argument was an lvalue. — end note]

#### 16.4.5.10 Library object access

[res.on.objects]

- 1 The behavior of a program is undefined if calls to standard library functions from different threads may introduce a data race. The conditions under which this may occur are specified in 16.4.6.10.
 

[Note 1: Modifying an object of a standard library type that is shared between threads risks undefined behavior unless objects of that type are explicitly specified as being shareable without data races or the user supplies a locking mechanism. — end note]
- 2 If an object of a standard library type is accessed, and the beginning of the object's lifetime (6.8.4) does not happen before the access, or the access does not happen before the end of the object's lifetime, the behavior is undefined unless otherwise specified.
 

[Note 2: This applies even to objects such as mutexes intended for thread synchronization. — end note]

#### 16.4.5.11 Semantic requirements

[res.on.requirements]

- 1 A sequence `Args` of template arguments is said to *model* a concept `C` if `Args` satisfies `C` (13.5.3) and meets all semantic requirements (if any) given in the specification of `C`.
- 2 If the validity or meaning of a program depends on whether a sequence of template arguments models a concept, and the concept is satisfied but not modeled, the program is ill-formed, no diagnostic required.
- 3 If the semantic requirements of a declaration's constraints (16.3.2.3) are not modeled at the point of use, the program is ill-formed, no diagnostic required.

### 16.4.6 Conforming implementations

[conforming]

#### 16.4.6.1 Overview

[conforming.overview]

- 1 Subclause 16.4.6 describes the constraints upon, and latitude of, implementations of the C++ standard library.
- 2 An implementation's use of
  - (2.1) — headers is discussed in 16.4.6.2,
  - (2.2) — macros in 16.4.6.3,
  - (2.3) — non-member functions in 16.4.6.4,
  - (2.4) — member functions in 16.4.6.5,
  - (2.5) — data race avoidance in 16.4.6.10,
  - (2.6) — access specifiers in 16.4.6.12,
  - (2.7) — class derivation in 16.4.6.13,
  - (2.8) — exceptions in 16.4.6.14, and

(2.9) — contract assertions in 16.4.6.15.

#### 16.4.6.2 Headers

[res.on.headers]

- <sup>1</sup> A C++ header may include other C++ headers. A C++ header shall provide the declarations and definitions that appear in its synopsis. A C++ header shown in its synopsis as including other C++ headers shall provide the declarations and definitions that appear in the synopses of those other headers.
- <sup>2</sup> Certain types and macros are defined in more than one header. Every such entity shall be defined such that any header that defines it may be included after any other header that also defines it (6.3).
- <sup>3</sup> The C standard library headers (17.15) shall include only their corresponding C++ standard library header, as described in 16.4.2.3.

#### 16.4.6.3 Restrictions on macro definitions

[res.on.macro.definitions]

- <sup>1</sup> The names and global function signatures described in 16.4.2.2 are reserved to the implementation.
- <sup>2</sup> All object-like macros defined by the C standard library and described in this Clause as expanding to integral constant expressions are also suitable for use in `#if` preprocessing directives, unless explicitly stated otherwise.

#### 16.4.6.4 Non-member functions

[global.functions]

- <sup>1</sup> It is unspecified whether any non-member functions in the C++ standard library are defined as inline (9.2.8).
- <sup>2</sup> A call to a non-member function signature described in Clause 17 through Clause 33 and Annex D shall behave as if the implementation declared no additional non-member function signatures.<sup>144</sup>
- <sup>3</sup> An implementation shall not declare a non-member function signature with additional default arguments.
- <sup>4</sup> Unless otherwise specified, calls made by functions in the standard library to non-operator, non-member functions do not use functions from another namespace which are found through argument-dependent name lookup (6.5.4).

[Note 1: The phrase “unless otherwise specified” applies to cases such as the swappable with requirements (16.4.4.3). The exception for overloaded operators allows argument-dependent lookup in cases like that of `ostream_iterator::operator=` (24.6.3.3) where lookup for the expression `*out_stream << value` includes the associated namespaces of `value`’s type. — end note]

#### 16.4.6.5 Member functions

[member.functions]

- <sup>1</sup> It is unspecified whether any member functions in the C++ standard library are defined as inline (9.2.8).
- <sup>2</sup> For a non-virtual member function described in the C++ standard library, an implementation may declare a different set of member function signatures, provided that any call to the member function that would select an overload from the set of declarations described in this document behaves as if that overload were selected.

[Note 1: For instance, an implementation can add parameters with default values, or replace a member function with default arguments with two or more member functions with equivalent behavior, or add additional signatures for a member function name. — end note]

#### 16.4.6.6 Friend functions

[hidden.friends]

- <sup>1</sup> Whenever this document specifies a friend declaration of a function or function template within a class or class template definition, that declaration shall be the only declaration of that function or function template provided by an implementation.

[Note 1: In particular, a conforming implementation does not provide any additional declarations of that function or function template at namespace scope. — end note]

[Note 2: Such a friend function or function template declaration is known as a hidden friend, as it is visible neither to ordinary unqualified lookup (6.5.3) nor to qualified lookup (6.5.5). — end note]

#### 16.4.6.7 Constexpr functions and constructors

[constexpr.functions]

- <sup>1</sup> This document explicitly requires that certain standard library functions are `constexpr` (9.2.6). An implementation shall not declare any standard library function signature as `constexpr` except for those where it is explicitly required. Within any header that provides any non-defining declarations of `constexpr` functions an implementation shall provide corresponding definitions.

<sup>144</sup>) A valid C++ program always calls the expected library non-member function. An implementation can also define additional non-member functions that would otherwise not be called by a valid C++ program.

#### 16.4.6.8 Requirements for stable algorithms [algorithm.stable]

- <sup>1</sup> When the requirements for an algorithm state that it is “stable” without further elaboration, it means:
- (1.1) — For the sort algorithms the relative order of equivalent elements is preserved.
  - (1.2) — For the remove and copy algorithms the relative order of the elements that are not removed is preserved.
  - (1.3) — For the merge algorithms, for equivalent elements in the original two ranges, the elements from the first range (preserving their original order) precede the elements from the second range (preserving their original order).

#### 16.4.6.9 Reentrancy [reentrancy]

- <sup>1</sup> Except where explicitly specified in this document, it is implementation-defined which functions in the C++ standard library may be recursively reentered.
- <sup>2</sup> During the execution of a standard library non-static member function  $F$  on an object, if that object is accessed through a glvalue that is not obtained, directly or indirectly, from the object parameter of  $F$ , in a manner that can conflict (6.10.2.2) with any access that  $F$  is permitted to perform (16.4.6.10), the behavior is undefined unless otherwise specified.

#### 16.4.6.10 Data race avoidance [res.on.data.races]

- <sup>1</sup> This subclause specifies requirements that implementations shall meet to prevent data races (6.10.2). Every standard library function shall meet each requirement unless otherwise specified. Implementations may prevent data races in cases other than those specified below.
- <sup>2</sup> A C++ standard library function shall not directly or indirectly access objects (6.10.2) accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function’s arguments, including `this`.
- <sup>3</sup> A C++ standard library function shall not directly or indirectly modify objects (6.10.2) accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function’s non-const arguments, including `this`.
- <sup>4</sup> [Note 1: This means, for example, that implementations can’t use an object with static storage duration for internal purposes without synchronization because doing so can cause a data race even in programs that do not explicitly share objects between threads. — end note]
- <sup>5</sup> A C++ standard library function shall not access objects indirectly accessible via its arguments or via elements of its container arguments except by invoking functions required by its specification on those container elements.
- <sup>6</sup> Operations on iterators obtained by calling a standard library container or string member function may access the underlying container, but shall not modify it.
- [Note 2: In particular, container operations that invalidate iterators conflict with operations on iterators associated with that container. — end note]
- <sup>7</sup> Implementations may share their own internal objects between threads if the objects are not visible to users and are protected against data races.
- <sup>8</sup> Unless otherwise specified, C++ standard library functions shall perform all operations solely within the current thread if those operations have effects that are visible (6.10.2) to users.
- <sup>9</sup> [Note 3: This allows implementations to parallelize operations if there are no visible side effects. — end note]

#### 16.4.6.11 Properties of library classes [library.class.props]

- <sup>1</sup> Unless explicitly stated otherwise, it is unspecified whether any class described in Clause 17 through Clause 33 and Annex D is a trivially copyable class, a standard-layout class, or an implicit-lifetime class (11.2).

#### 16.4.6.12 Protection within classes [protection.within.classes]

- <sup>1</sup> It is unspecified whether any function signature or class described in Clause 17 through Clause 33 and Annex D is a friend of another class in the C++ standard library.

#### 16.4.6.13 Derived classes [derivation]

- <sup>1</sup> An implementation may derive any class in the C++ standard library from a class with a name reserved to the implementation.

<sup>2</sup> Certain classes defined in the C++ standard library are required to be derived from other classes in the C++ standard library. An implementation may derive such a class directly from the required base or indirectly through a hierarchy of base classes with names reserved to the implementation.

<sup>3</sup> In any case:

- (3.1) — Every base class described as `virtual` shall be virtual;
- (3.2) — Every base class not specified as `virtual` shall not be virtual;
- (3.3) — Unless explicitly stated otherwise, types with distinct names shall be distinct types.

[*Note 1*: There is an implicit exception to this rule for types that are described as synonyms (9.2.4, 9.10), such as `size_t` (17.2) and `streamoff` (31.2.2). — *end note*]

<sup>4</sup> All types specified in the C++ standard library shall be non-`final` types unless otherwise specified.

#### 16.4.6.14 Restrictions on exception handling

[res.on.exception.handling]

- <sup>1</sup> Any of the functions defined in the C++ standard library can report a failure by throwing an exception of a type described in its *Throws*: paragraph, or of a type derived from a type named in the *Throws*: paragraph that would be caught by a *handler* (14.4) for the base type.
- <sup>2</sup> Functions from the C standard library shall not throw exceptions<sup>145</sup> except when such a function calls a program-supplied function that throws an exception.<sup>146</sup>
- <sup>3</sup> Destructor operations defined in the C++ standard library shall not throw exceptions. Every destructor in the C++ standard library shall behave as if it had a non-throwing exception specification (14.5).
- <sup>4</sup> Functions defined in the C++ standard library that do not have a *Throws*: paragraph but do have a potentially-throwing exception specification may throw implementation-defined exceptions.<sup>147</sup> Implementations should report errors by throwing exceptions of or derived from the standard exception classes (17.6.4.1, 17.9, 19.2).
- <sup>5</sup> An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing exception specification.

#### 16.4.6.15 Contract assertions

[res.contract.assertions]

- <sup>1</sup> Unless specified otherwise, an implementation may check the specified preconditions and postconditions of a function in the C++ standard library using contract assertions (6.11, 16.3.2.4).

#### 16.4.6.16 Value of error codes

[value.error.codes]

- <sup>1</sup> Certain functions in the C++ standard library report errors via an `error_code` (19.5.4.1) object. That object's `category()` member shall return `system_category()` for errors originating from the operating system, or a reference to an implementation-defined `error_category` object for errors originating elsewhere. The implementation shall define the possible values of `value()` for each of these error categories.

[*Example 1*: For operating systems that are based on POSIX, implementations should define the `std::system_category()` values as identical to the POSIX `errno` values, with additional values as defined by the operating system's documentation. Implementations for operating systems that are not based on POSIX should define values identical to the operating system's values. For errors that do not originate from the operating system, the implementation may provide enums for the associated values. — *end example*]

#### 16.4.6.17 Moved-from state of library types

[lib.types.movedfrom]

- <sup>1</sup> Objects of types defined in the C++ standard library may be moved from (11.4.5.3). Move operations may be explicitly specified or implicitly generated. Unless otherwise specified, such moved-from objects shall be placed in a valid but unspecified state (3.68).
- <sup>2</sup> An object of a type defined in the C++ standard library may be move-assigned (11.4.6) to itself. Unless otherwise specified, such an assignment places the object in a valid but unspecified state.

<sup>145</sup>) That is, the C standard library functions can all be treated as if they are marked `noexcept`. This allows implementations to make performance optimizations based on the absence of exceptions at runtime.

<sup>146</sup>) The functions `qsort()` and `bsearch()` (26.13) meet this condition.

<sup>147</sup>) In particular, they can report a failure to allocate storage by throwing an exception of type `bad_alloc`, or a class derived from `bad_alloc` (17.6.4.1).



# 17 Language support library [support]

## 17.1 General [support.general]

- <sup>1</sup> This Clause describes the function signatures that are called implicitly, and the types of objects generated implicitly, during the execution of some C++ programs. It also describes the headers that declare these function signatures and define any related types.
- <sup>2</sup> The following subclauses describe common type definitions used throughout the library, characteristics of the predefined types, functions supporting start and termination of a C++ program, support for dynamic memory management, support for dynamic type identification, support for contract-violation handling, support for exception processing, support for initializer lists, and other runtime support, as summarized in [Table 42](#).

**Table 42 — Language support library summary** [tab:support.summary]

| Subclause                                         | Header                                    |
|---------------------------------------------------|-------------------------------------------|
| <a href="#">17.2</a> Common definitions           | <cstddef>, <cstdlib>                      |
| <a href="#">17.3</a> Implementation properties    | <cfloat>, <climits>, <limits>, <version>  |
| <a href="#">17.4</a> Arithmetic types             | <cstdint>, <stdfloat>                     |
| <a href="#">17.5</a> Start and termination        | <cstdlib>                                 |
| <a href="#">17.6</a> Dynamic memory management    | <new>                                     |
| <a href="#">17.7</a> Type identification          | <typeinfo>, <typeid>                      |
| <a href="#">17.8</a> Source location              | <source_location>                         |
| <a href="#">17.9</a> Exception handling           | <exception>                               |
| <a href="#">17.10</a> Contract-violation handling | <contracts>                               |
| <a href="#">17.11</a> Initializer lists           | <initializer_list>                        |
| <a href="#">17.12</a> Comparisons                 | <compare>                                 |
| <a href="#">17.13</a> Coroutines                  | <coroutine>                               |
| <a href="#">17.14</a> Other runtime support       | <setjmp>, <csignal>, <cstdarg>, <cstdlib> |

## 17.2 Common definitions [support.types]

### 17.2.1 Header <cstddef> synopsis [cstddef.syn]

```
// all freestanding
namespace std {
 using ptrdiff_t = see below;
 using size_t = see below;
 using max_align_t = see below;
 using nullptr_t = decltype(nullptr);

 enum class byte : unsigned char {};

 // 17.2.5, byte type operations
 template<class IntType>
 constexpr byte& operator<=(byte& b, IntType shift) noexcept;
 template<class IntType>
 constexpr byte operator<<(byte b, IntType shift) noexcept;
 template<class IntType>
 constexpr byte& operator>=(byte& b, IntType shift) noexcept;
 template<class IntType>
 constexpr byte operator>>(byte b, IntType shift) noexcept;
 constexpr byte& operator|=(byte& l, byte r) noexcept;
 constexpr byte operator|(byte l, byte r) noexcept;
 constexpr byte& operator&=(byte& l, byte r) noexcept;
 constexpr byte operator&(byte l, byte r) noexcept;
 constexpr byte& operator^=(byte& l, byte r) noexcept;
 constexpr byte operator^(byte l, byte r) noexcept;
```

```
constexpr byte operator~(byte b) noexcept;
template<class IntType>
 constexpr IntType to_integer(byte b) noexcept;
}
```

```
#define NULL see below
#define offsetof(P, D) see below
```

- <sup>1</sup> The contents and meaning of the header `<cstddef>` are the same as the C standard library header `<stddef.h>`, except that it does not declare the type `wchar_t`, that it does not define the macro `unreachable`, that it also declares the type `byte` and its associated operations (17.2.5), and as noted in 17.2.3 and 17.2.4.

SEE ALSO: ISO/IEC 9899:2024, 7.21

## 17.2.2 Header `<cstdlib>` synopsis

[`cstdlib.syn`]

```
namespace std {
 using size_t = see below; // freestanding
 using div_t = see below; // freestanding
 using ldiv_t = see below; // freestanding
 using lldiv_t = see below; // freestanding
}

#define NULL see below // freestanding
#define EXIT_FAILURE see below // freestanding
#define EXIT_SUCCESS see below // freestanding
#define RAND_MAX see below
#define MB_CUR_MAX see below

namespace std {
 // Exposition-only function type aliases
 extern "C" using c-atexit-handler = void(); // exposition only
 extern "C++" using atexit-handler = void(); // exposition only
 extern "C" using c-compare-pred = int(const void*, const void*); // exposition only
 extern "C++" using compare-pred = int(const void*, const void*); // exposition only

 // 17.5, start and termination
 [[noreturn]] void abort() noexcept; // freestanding
 int atexit(c-atexit-handler* func) noexcept; // freestanding
 int atexit(atexit-handler* func) noexcept; // freestanding
 int at_quick_exit(c-atexit-handler* func) noexcept; // freestanding
 int at_quick_exit(atexit-handler* func) noexcept; // freestanding
 [[noreturn]] void exit(int status); // freestanding
 [[noreturn]] void _Exit(int status) noexcept; // freestanding
 [[noreturn]] void quick_exit(int status) noexcept; // freestanding

 char* getenv(const char* name);
 int system(const char* string);

 // 20.2.12, C library memory allocation
 void* aligned_alloc(size_t alignment, size_t size);
 void* calloc(size_t nmemb, size_t size);
 void free(void* ptr);
 void free_sized(void* ptr, size_t size);
 void free_aligned_sized(void* ptr, size_t alignment, size_t size);
 void* malloc(size_t size);
 void* realloc(void* ptr, size_t size);
 size_t memalignment(const void* p); // freestanding

 double atof(const char* nptr);
 int atoi(const char* nptr);
 long int atol(const char* nptr);
 long long int atoll(const char* nptr);
 double strtod(const char* nptr, char** endptr);
 int strfromd(char* s, size_t n, const char* format, double fp);
```



```

int strfromf(char* s, size_t n, const char* format, float fp);
int strfroml(char* s, size_t n, const char* format, long double fp);
float strtouf(const char* nptr, char** endptr);
long double strtold(const char* nptr, char** endptr);
long int strtol(const char* nptr, char** endptr, int base);
long long int strtoll(const char* nptr, char** endptr, int base);
unsigned long int strtoul(const char* nptr, char** endptr, int base);
unsigned long long int strtoull(const char* nptr, char** endptr, int base);

// 28.7.5, multibyte / wide string and character conversion functions
int mblen(const char* s, size_t n);
int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);

// 26.13, C standard library algorithms
void* bsearch(const void* key, void* base, size_t nmemb, size_t size, // freestanding
 c-compare-pred* compar);
void* bsearch(const void* key, void* base, size_t nmemb, size_t size, // freestanding
 compare-pred* compar);
const void* bsearch(const void* key, const void* base, size_t nmemb, // freestanding
 size_t size, c-compare-pred* compar);
const void* bsearch(const void* key, const void* base, size_t nmemb, // freestanding
 size_t size, compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar); // freestanding
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar); // freestanding

// 29.5.10, low-quality random number generation
int rand();
void srand(unsigned int seed);

// 29.7.2, absolute values
constexpr int abs(int j); // freestanding
constexpr long int abs(long int j); // freestanding
constexpr long long int abs(long long int j); // freestanding
constexpr floating-point-type abs(floating-point-type j); // freestanding-deleted

constexpr long int labs(long int j); // freestanding
constexpr long long int llabs(long long int j); // freestanding

constexpr div_t div(int numer, int denom); // freestanding
constexpr ldiv_t div(long int numer, long int denom); // freestanding; see 16.2
constexpr lldiv_t div(long long int numer, long long int denom); // freestanding; see 16.2
constexpr ldiv_t ldiv(long int numer, long int denom); // freestanding
constexpr lldiv_t lldiv(long long int numer, long long int denom); // freestanding
}

```

- <sup>1</sup> The contents and meaning of the header `<cstdlib>` are the same as the C standard library header `<stdlib.h>`, except that it does not declare the types `wchar_t` or `once_flag`, and does not declare the function `call_once`, and except as noted in 17.2.3, 17.2.4, 17.5, 20.2.12, 28.7.5, 26.13, 29.5.10, and 29.7.2.

[Note 1: Several functions have additional overloads in this document, but they have the same behavior as in the C standard library (16.2). — end note]

SEE ALSO: ISO/IEC 9899:2024, 7.24

### 17.2.3 Null pointers

[support.types.nullptr]

- <sup>1</sup> The type `nullptr_t` is a synonym for the type of a `nullptr` expression, and it has the characteristics described in 6.9.3 and 7.3.12.

[Note 1: Although `nullptr`'s address cannot be taken, the address of another `nullptr_t` object that is an lvalue can be taken. — end note]

- <sup>2</sup> The macro `NULL` is an implementation-defined null pointer constant that is a literal (5.13.2, 5.13.8).<sup>148</sup>  
SEE ALSO: ISO/IEC 9899:2024, 7.21

### 17.2.4 Sizes, alignments, and offsets

[support.types.layout]

- <sup>1</sup> The macro `offsetof(type, member-designator)` has the same semantics as the corresponding macro in the C standard library header `<stddef.h>`, but accepts a restricted set of *type* arguments in this document. Use of the `offsetof` macro with a *type* other than a standard-layout class (11.2) is conditionally-supported.<sup>149</sup> The expression `offsetof(type, member-designator)` is never type-dependent (13.8.3.3) and it is value-dependent (13.8.3.4) if and only if *type* is dependent. The result of applying the `offsetof` macro to a static data member or a function member is undefined. No operation invoked by the `offsetof` macro shall throw an exception and `noexcept(offsetof(type, member-designator))` shall be `true`.
- <sup>2</sup> The type `ptrdiff_t` is an implementation-defined signed integer type that can hold the difference of two subscripts in an array object, as described in 7.6.6.
- <sup>3</sup> The type `size_t` is an implementation-defined unsigned integer type that is large enough to contain the size in bytes of any object (7.6.2.5).
- <sup>4</sup> *Recommended practice:* An implementation should choose types for `ptrdiff_t` and `size_t` whose integer conversion ranks (6.9.7) are no greater than that of `signed long int` unless a larger size is necessary to contain all the possible values.
- <sup>5</sup> The type `max_align_t` is a trivially copyable standard-layout type whose alignment requirement is at least as great as that of every scalar type, and whose alignment requirement is supported in every context (6.8.3). `std::is_trivially_default_constructible_v<max_align_t>` is `true`.  
SEE ALSO: ISO/IEC 9899:2024, 7.21

### 17.2.5 byte type operations

[support.types.byteops]

- ```
template<class IntType>
constexpr byte& operator<<=(byte& b, IntType shift) noexcept;
```
- ¹ *Constraints:* `is_integral_v<IntType>` is `true`.
- ² *Effects:* Equivalent to: `return b = b << shift;`
- ```
template<class IntType>
constexpr byte operator<<(byte b, IntType shift) noexcept;
```
- <sup>3</sup> *Constraints:* `is_integral_v<IntType>` is `true`.
- <sup>4</sup> *Effects:* Equivalent to:
- ```
return static_cast<byte>(static_cast<unsigned int>(b) << shift);
```
- ```
template<class IntType>
constexpr byte& operator>>=(byte& b, IntType shift) noexcept;
```
- <sup>5</sup> *Constraints:* `is_integral_v<IntType>` is `true`.
- <sup>6</sup> *Effects:* Equivalent to: `return b = b >> shift;`
- ```
template<class IntType>
constexpr byte operator>>(byte b, IntType shift) noexcept;
```
- ⁷ *Constraints:* `is_integral_v<IntType>` is `true`.
- ⁸ *Effects:* Equivalent to:
- ```
return static_cast<byte>(static_cast<unsigned int>(b) >> shift);
```
- ```
constexpr byte& operator|=(byte& l, byte r) noexcept;
```
- ⁹ *Effects:* Equivalent to: `return l = l | r;`

¹⁴⁸) Possible definitions include `nullptr`, `0` and `0L`, but not `(void*)0`.

¹⁴⁹) Note that `offsetof` is required to work as specified even if unary `operator&` is overloaded for any of the types involved.

```
constexpr byte operator|(byte l, byte r) noexcept;
10   Effects: Equivalent to:
       return static_cast<byte>(static_cast<unsigned int>(l) | static_cast<unsigned int>(r));

constexpr byte& operator&=(byte& l, byte r) noexcept;
11   Effects: Equivalent to: return l = l & r;

constexpr byte operator&(byte l, byte r) noexcept;
12   Effects: Equivalent to:
       return static_cast<byte>(static_cast<unsigned int>(l) & static_cast<unsigned int>(r));

constexpr byte& operator^=(byte& l, byte r) noexcept;
13   Effects: Equivalent to: return l = l ^ r;

constexpr byte operator^(byte l, byte r) noexcept;
14   Effects: Equivalent to:
       return static_cast<byte>(static_cast<unsigned int>(l) ^ static_cast<unsigned int>(r));

constexpr byte operator~(byte b) noexcept;
15   Effects: Equivalent to:
       return static_cast<byte>(~static_cast<unsigned int>(b));

template<class IntType>
constexpr IntType to_integer(byte b) noexcept;
16   Constraints: is_integral_v<IntType> is true.
17   Effects: Equivalent to: return static_cast<IntType>(b);
```

17.3 Implementation properties

[support.limits]

17.3.1 General

[support.limits.general]

- 1 The headers <limits> (17.3.3), <climits> (17.3.6), and <cfloat> (17.3.7) supply characteristics of implementation-dependent arithmetic types (6.9.3).

17.3.2 Header <version> synopsis

[version.syn]

- 1 The header <version> supplies implementation-dependent information about the C++ standard library (e.g., version number and release date).
- 2 Each of the macros defined in <version> is also defined after inclusion of any member of the set of library headers indicated in the corresponding comment in this synopsis.

[Note 1: Future revisions of this document might replace the values of these macros with greater values. — end note]

```
#define __cpp_lib_adaptor_iterator_pair_constructor 202106L // also in <stack>, <queue>
#define __cpp_lib_addressof_constexpr 201603L // freestanding, also in <memory>
#define __cpp_lib_algorithm_default_value_type 202603L
// also in <algorithm>, <ranges>, <string>, <deque>, <list>, <forward_list>, <vector>, <memory>
#define __cpp_lib_algorithm_iterator_requirements 202207L
// also in <algorithm>, <numeric>, <memory>
#define __cpp_lib_aligned_accessor 202411L // freestanding, also in <mdspan>
#define __cpp_lib_allocate_at_least 202302L // also in <memory>
#define __cpp_lib_allocator_traits_is_always_equal 201411L
// freestanding, also in <memory>, <scoped_allocator>, <string>, <deque>, <forward_list>, <list>,
// <vector>, <map>, <set>, <unordered_map>, <unordered_set>
#define __cpp_lib_any 201606L // also in <any>
#define __cpp_lib_apply 202603L
// freestanding, also in <tuple>, <type_traits>, <meta>
#define __cpp_lib_array_constexpr 201811L
// freestanding, also in <iterator>, <array>
#define __cpp_lib_as_const 201510L // freestanding, also in <utility>
#define __cpp_lib_associative_heterogeneous_erasure 202110L
// also in <map>, <set>, <unordered_map>, <unordered_set>
```

```

#define __cpp_lib_associative_heterogeneous_insertion 202306L
    // also in <map>, <set>, <unordered_map>, <unordered_set>
#define __cpp_lib_assume_aligned 201811L // freestanding, also in <memory>
#define __cpp_lib_atomic_flag_test 201907L // freestanding, also in <atomic>
#define __cpp_lib_atomic_float 201711L // freestanding, also in <atomic>
#define __cpp_lib_atomic_is_always_lock_free 201603L // freestanding, also in <atomic>
#define __cpp_lib_atomic_lock_free_type_aliases 201907L // also in <atomic>
#define __cpp_lib_atomic_min_max 202506L // freestanding, also in <atomic>
#define __cpp_lib_atomic_reductions 202506L // freestanding, also in <atomic>
#define __cpp_lib_atomic_ref 202603L // freestanding, also in <atomic>
#define __cpp_lib_atomic_shared_ptr 201711L // also in <memory>
#define __cpp_lib_atomic_value_initialization 201911L // freestanding, also in <atomic>, <memory>
#define __cpp_lib_atomic_wait 201907L // freestanding, also in <atomic>
#define __cpp_lib_barrier 202302L // also in <barrier>
#define __cpp_lib_bind_back 202306L // freestanding, also in <functional>
#define __cpp_lib_bind_front 202306L // freestanding, also in <functional>
#define __cpp_lib_bit_cast 201806L // freestanding, also in <bit>
#define __cpp_lib_bitops 202607L // freestanding, also in <bit>
#define __cpp_lib_bitset 202306L // also in <bitset>
#define __cpp_lib_bool_constant 201505L // freestanding, also in <type_traits>
#define __cpp_lib_bounded_array_traits 201902L // freestanding, also in <type_traits>
#define __cpp_lib_boyer_moore_searcher 201603L // also in <functional>
#define __cpp_lib_byte 201603L // freestanding, also in <cstdint>
#define __cpp_lib_byteswap 202110L // freestanding, also in <bit>
#define __cpp_lib_char8_t 201907L
    // freestanding, also in <atomic>, <filesystem>, <iosfwd>, <istream>, <limits>, <locale>, <ostream>,
    // <string>, <string_view>
#define __cpp_lib_chrono 202306L // also in <chrono>
#define __cpp_lib_chrono_udls 201304L // also in <chrono>
#define __cpp_lib_clamp 201603L // freestanding, also in <algorithm>
#define __cpp_lib_common_reference 202302L // freestanding, also in <type_traits>
#define __cpp_lib_common_reference_wrapper 202302L // freestanding, also in <functional>
#define __cpp_lib_complex_udls 201309L // also in <complex>
#define __cpp_lib_concepts 202207L
    // freestanding, also in <concepts>, <compare>
#define __cpp_lib_constant_wrapper 202606L // freestanding, also in <utility>
#define __cpp_lib_constexpr_algorithms 202306L // also in <algorithm>, <utility>
#define __cpp_lib_constexpr_atomic 202411L // also in <atomic>
#define __cpp_lib_constexpr_bitset 202207L // also in <bitset>
#define __cpp_lib_constexpr_charconv 202207L // freestanding, also in <charconv>
#define __cpp_lib_constexpr_cmath 202306L // also in <cmath>, <cstdlib>
#define __cpp_lib_constexpr_complex 202306L // also in <complex>
#define __cpp_lib_constexpr_deque 202502L // also in <deque>
#define __cpp_lib_constexpr_dynamic_alloc 201907L // also in <memory>
#define __cpp_lib_constexpr_exceptions 202502L
    // also in <exception>, <stdexcept>, <expected>, <optional>, <variant>, and <format>
#define __cpp_lib_constexpr_flat_map 202502L // also in <flat_map>
#define __cpp_lib_constexpr_flat_set 202502L // also in <flat_set>
#define __cpp_lib_constexpr_format 202511L // also in <format>
#define __cpp_lib_constexpr_forward_list 202502L // also in <forward_list>
#define __cpp_lib_constexpr_functional 201907L // freestanding, also in <functional>
#define __cpp_lib_constexpr_inplace_vector 202502L // also in <inplace_vector>
#define __cpp_lib_constexpr_iterator 201811L // freestanding, also in <iterator>
#define __cpp_lib_constexpr_list 202502L // also in <list>
#define __cpp_lib_constexpr_map 202502L // also in <map>
#define __cpp_lib_constexpr_memory 202506L // freestanding, also in <memory>
#define __cpp_lib_constexpr_new 202406L // freestanding, also in <new>
#define __cpp_lib_constexpr_numeric 201911L // freestanding, also in <numeric>
#define __cpp_lib_constexpr_queue 202502L // also in <queue>
#define __cpp_lib_constexpr_set 202502L // also in <set>
#define __cpp_lib_constexpr_stack 202502L // also in <stack>
#define __cpp_lib_constexpr_string 202511L // also in <string>
#define __cpp_lib_constexpr_string_view 201811L // also in <string_view>
#define __cpp_lib_constexpr_tuple 201811L // freestanding, also in <tuple>

```

```

#define __cpp_lib_constexpr_typeinfo          202106L // freestanding, also in <typeinfo>
#define __cpp_lib_constexpr_unordered_map     202502L // also in <unordered_map>
#define __cpp_lib_constexpr_unordered_set     202502L // also in <unordered_set>
#define __cpp_lib_constexpr_utility           201811L // freestanding, also in <utility>
#define __cpp_lib_constexpr_vector            201907L // also in <vector>
#define __cpp_lib_constrained_equality         202411L // freestanding,
// also in <utility>, <tuple>, <optional>, <variant>, <expected>
#define __cpp_lib_containers_ranges            202202L
// also in <vector>, <list>, <forward_list>, <map>, <set>, <unordered_map>, <unordered_set>,
// <deque>, <queue>, <stack>, <string>
#define __cpp_lib_contracts                    202502L // freestanding, also in <contracts>
#define __cpp_lib_copyable_function           202306L // also in <functional>
#define __cpp_lib_coroutine                   201902L // freestanding, also in <coroutine>
#define __cpp_lib_counting_scope               202506L // also in <execution>
#define __cpp_lib_debugging                   202403L // freestanding, also in <debugging>
#define __cpp_lib_define_static                202506L // freestanding, also in <meta>
#define __cpp_lib_destroying_delete            201806L // freestanding, also in <new>
#define __cpp_lib_enable_shared_from_this     201603L // also in <memory>
#define __cpp_lib_endian                      201907L // freestanding, also in <bit>
#define __cpp_lib_erase_if                    202002L
// also in <string>, <deque>, <forward_list>, <list>, <vector>, <map>, <set>, <unordered_map>,
// <unordered_set>
#define __cpp_lib_exception_ptr_cast           202603L // also in <exception>
#define __cpp_lib_exchange_function            201304L // freestanding, also in <utility>
#define __cpp_lib_execution                   201902L // also in <execution>
#define __cpp_lib_expected                    202606L // also in <expected>
#define __cpp_lib_filesystem                  201703L // also in <filesystem>
#define __cpp_lib_flat_map                    202511L // also in <flat_map>
#define __cpp_lib_flat_set                    202511L // also in <flat_set>
#define __cpp_lib_format                      202603L // also in <format>
#define __cpp_lib_format_path                 202506L // also in <filesystem>
#define __cpp_lib_format_ranges               202207L // also in <format>
#define __cpp_lib_format_uchar                202311L // also in <format>
#define __cpp_lib_formatters                  202302L // also in <stacktrace>, <thread>
#define __cpp_lib_forward_like                 202207L // freestanding, also in <utility>
#define __cpp_lib_freestanding_algorithm       202502L // freestanding, also in <algorithm>
#define __cpp_lib_freestanding_array           202311L // freestanding, also in <array>
#define __cpp_lib_freestanding_char_traits     202306L // freestanding, also in <string>
#define __cpp_lib_freestanding_charconv        202306L // freestanding, also in <charconv>
#define __cpp_lib_freestanding_cstdlib         202306L // freestanding, also in <cstdlib>, <cmath>
#define __cpp_lib_freestanding_cstring         202311L // freestanding, also in <cstring>
#define __cpp_lib_freestanding_cwchar         202306L // freestanding, also in <cwchar>
#define __cpp_lib_freestanding_errc            202306L
// freestanding, also in <cerrno>, <system_error>
#define __cpp_lib_freestanding_execution       202502L // freestanding, also in <execution>
#define __cpp_lib_freestanding_expected        202311L // freestanding, also in <expected>
#define __cpp_lib_freestanding_feature_test_macros 202306L // freestanding
#define __cpp_lib_freestanding_functional      202306L // freestanding, also in <functional>
#define __cpp_lib_freestanding_iterator        202306L // freestanding, also in <iterator>
#define __cpp_lib_freestanding_mdspan          202311L // freestanding, also in <mdspan>
#define __cpp_lib_freestanding_memory          202502L // freestanding, also in <memory>
#define __cpp_lib_freestanding_numeric         202502L // freestanding, also in <numeric>
#define __cpp_lib_freestanding_operator_new     see below // freestanding, also in <new>
#define __cpp_lib_freestanding_optional        202506L // freestanding, also in <optional>
#define __cpp_lib_freestanding_random          202502L // freestanding, also in <random>
#define __cpp_lib_freestanding_ranges          202306L // freestanding, also in <ranges>
#define __cpp_lib_freestanding_ratio           202306L // freestanding, also in <ratio>
#define __cpp_lib_freestanding_string_view     202311L // freestanding, also in <string_view>
#define __cpp_lib_freestanding_tuple           202306L // freestanding, also in <tuple>
#define __cpp_lib_freestanding_utility         202306L // freestanding, also in <utility>
#define __cpp_lib_freestanding_variant         202311L // freestanding, also in <variant>
#define __cpp_lib_fstream_native_handle       202306L // also in <fstream>
#define __cpp_lib_function_ref                 202604L // freestanding, also in <functional>
#define __cpp_lib_gcd_lcm                     201606L // freestanding, also in <numeric>

```

```

#define __cpp_lib_generator 202207L // also in <generator>
#define __cpp_lib_generic_associative_lookup 201304L // also in <map>, <set>
#define __cpp_lib_generic_unordered_lookup 201811L
    // also in <unordered_map>, <unordered_set>
#define __cpp_lib_hardware_interference_size 201703L // freestanding, also in <new>
#define __cpp_lib_has_unique_object_representations 201606L // freestanding, also in <type_traits>
#define __cpp_lib_hazard_pointer 202606L // also in <hazard_pointer>
#define __cpp_lib_hive 202502L // also in <hive>
#define __cpp_lib_hypot 201603L // also in <cmath>
#define __cpp_lib_incomplete_container_elements 201505L
    // also in <forward_list>, <list>, <vector>
#define __cpp_lib_indirect 202502L // also in <memory>
#define __cpp_lib_initializer_list 202511L
    // freestanding, also in <initializer_list>
#define __cpp_lib_inplace_vector 202603L // also in <inplace_vector>
#define __cpp_lib_int_pow2 202002L // freestanding, also in <bit>
#define __cpp_lib_integer_comparison_functions 202002L // freestanding, also in <utility>
#define __cpp_lib_integer_sequence 202511L // freestanding, also in <utility>
#define __cpp_lib_integral_constant_callable 201304L // freestanding, also in <type_traits>
#define __cpp_lib_interpolate 201902L // also in <cmath>, <numeric>
#define __cpp_lib_invoke 201411L // freestanding, also in <functional>
#define __cpp_lib_invoke_r 202106L // freestanding, also in <functional>
#define __cpp_lib_ios_noreplace 202207L // also in <ios>
#define __cpp_lib_is_aggregate 201703L // freestanding, also in <type_traits>
#define __cpp_lib_is_constant_evaluated 201811L // freestanding, also in <type_traits>
#define __cpp_lib_is_final 201402L // freestanding, also in <type_traits>
#define __cpp_lib_is_implicit_lifetime 202302L // freestanding, also in <type_traits>
#define __cpp_lib_is_invocable 201703L // freestanding, also in <type_traits>
#define __cpp_lib_is_layout_compatible 201907L // freestanding, also in <type_traits>
#define __cpp_lib_is_nothrow_convertible 201806L // freestanding, also in <type_traits>
#define __cpp_lib_is_null_pointer 201309L // freestanding, also in <type_traits>
#define __cpp_lib_is_pointer_interconvertible 201907L // freestanding, also in <type_traits>
#define __cpp_lib_is_scoped_enum 202011L // freestanding, also in <type_traits>
#define __cpp_lib_is_structural 202603L
    // freestanding, also in <meta> and <type_traits>
#define __cpp_lib_is_sufficiently_aligned 202411L // freestanding, also in <memory>
#define __cpp_lib_is_swappable 201603L // freestanding, also in <type_traits>
#define __cpp_lib_is_virtual_base_of 202406L // freestanding, also in <type_traits>
#define __cpp_lib_is_within_lifetime 202603L // freestanding, also in <type_traits>
#define __cpp_lib_jthread 201911L // also in <stop_token>, <thread>
#define __cpp_lib_latch 201907L // also in <latch>
#define __cpp_lib_laundary 201606L // freestanding, also in <new>
#define __cpp_lib_linalg 202511L // also in <linalg>
#define __cpp_lib_list_remove_return_type 201806L // also in <forward_list>, <list>
#define __cpp_lib_logical_traits 201510L // freestanding, also in <type_traits>
#define __cpp_lib_make_from_tuple 201606L // freestanding, also in <tuple>
#define __cpp_lib_make_reverse_iterator 201402L // freestanding, also in <iterator>
#define __cpp_lib_make_unique 201304L // also in <memory>
#define __cpp_lib_map_lookup 202606L
    // also in <map>, <unordered_map>, <flat_map>
#define __cpp_lib_map_try_emplace 201411L // also in <map>
#define __cpp_lib_math_constants 201907L // also in <numbers>
#define __cpp_lib_math_special_functions 201603L // also in <cmath>
#define __cpp_lib_mdspan 202406L // freestanding, also in <mdspan>
#define __cpp_lib_mdspan_copy 202606L // also in <mdspan>
#define __cpp_lib_memory_resource 201603L // also in <memory_resource>
#define __cpp_lib_modules 202207L // freestanding
#define __cpp_lib_move_iterator_concept 202207L // freestanding, also in <iterator>
#define __cpp_lib_move_only_function 202110L // also in <functional>
#define __cpp_lib_node_extract 201606L
    // also in <map>, <set>, <unordered_map>, <unordered_set>
#define __cpp_lib_nonmember_container_access 201411L
    // freestanding, also in <array>, <deque>, <forward_list>, <iterator>, <list>, <map>, <regex>, <set>,
    // <string>, <unordered_map>, <unordered_set>, <vector>

```

```

#define __cpp_lib_not_fn                202306L // freestanding, also in <functional>
#define __cpp_lib_null_iterators        201304L // freestanding, also in <iterator>
#define __cpp_lib_observable_checkpoint  202506L // freestanding, also in <utility>
#define __cpp_lib_optional               202506L // also in <optional>
#define __cpp_lib_optional_range_support 202406L // freestanding, also in <optional>
#define __cpp_lib_out_ptr                 202311L // freestanding, also in <memory>
#define __cpp_lib_parallel_algorithm      202506L
// also in <algorithm>, <numeric>, <memory>
#define __cpp_lib_parallel_scheduler      202506L // also in <execution>
#define __cpp_lib_philox_engine           202406L // also in <random>
#define __cpp_lib_pointer_tag_pair        202606L // freestanding, also in <memory>
#define __cpp_lib_polymorphic             202502L // also in <memory>
#define __cpp_lib_polymorphic_allocator   201902L // also in <memory_resource>
#define __cpp_lib_print                    202406L // also in <print>, <ostream>
#define __cpp_lib_quoted_string_io        201304L // also in <iomanip>
#define __cpp_lib_ranges                  202406L
// also in <algorithm>, <functional>, <iterator>, <memory>, <ranges>
#define __cpp_lib_ranges_as_const         202311L // freestanding, also in <ranges>
#define __cpp_lib_ranges_as_input         202502L // freestanding, also in <ranges>
#define __cpp_lib_ranges_as_rvalue        202207L // freestanding, also in <ranges>
#define __cpp_lib_ranges_cache_latest     202411L // freestanding, also in <ranges>
#define __cpp_lib_ranges_cartesian_product 202207L // freestanding, also in <ranges>
#define __cpp_lib_ranges_chunk            202202L // freestanding, also in <ranges>
#define __cpp_lib_ranges_chunk_by         202202L // freestanding, also in <ranges>
#define __cpp_lib_ranges_concat           202403L // freestanding, also in <ranges>
#define __cpp_lib_ranges_contains         202207L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_enumerate        202302L // freestanding, also in <ranges>
#define __cpp_lib_ranges_filter           202603L // freestanding, also in <ranges>
#define __cpp_lib_ranges_find_last        202207L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_fold             202207L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_generate_random  202403L // also in <random>
#define __cpp_lib_ranges_indices          202506L // also in <ranges>
#define __cpp_lib_ranges_iota             202202L // freestanding, also in <numeric>
#define __cpp_lib_ranges_join_with        202202L // freestanding, also in <ranges>
#define __cpp_lib_ranges_repeat           202207L // freestanding, also in <ranges>
#define __cpp_lib_ranges_reserve_hint     202502L // also in <ranges>
#define __cpp_lib_ranges_slide            202202L // freestanding, also in <ranges>
#define __cpp_lib_ranges_starts_ends_with 202106L // freestanding, also in <algorithm>
#define __cpp_lib_ranges_stride            202207L // freestanding, also in <ranges>
#define __cpp_lib_ranges_to_container     202202L // freestanding, also in <ranges>
#define __cpp_lib_ranges_zip              202110L
// freestanding, also in <ranges>, <tuple>, <utility>
#define __cpp_lib_ratio                   202306L // freestanding, also in <ratio>
#define __cpp_lib_raw_memory_algorithms   202411L // also in <memory>
#define __cpp_lib_rcu                     202306L // also in <rcu>
#define __cpp_lib_reference_from_temporary 202202L // freestanding, also in <type_traits>
#define __cpp_lib_reference_wrapper       202403L // freestanding, also in <functional>
#define __cpp_lib_reflection              202603L // also in <meta>
#define __cpp_lib_remove_cvref            201711L // freestanding, also in <type_traits>
#define __cpp_lib_replaceable_contract_violation_handler see below
// freestanding, also in <contracts>
#define __cpp_lib_result_of_sfinae        201210L
// freestanding, also in <functional>, <type_traits>
#define __cpp_lib_robust_nonmodifying_seq_ops 201304L // freestanding, also in <algorithm>
#define __cpp_lib_sample                   201603L // freestanding, also in <algorithm>
#define __cpp_lib_saturation_arithmetic    202603L // freestanding, also in <numeric>
#define __cpp_lib_scoped_lock              201703L // also in <mutex>
#define __cpp_lib_semaphore                201907L // also in <semaphore>
#define __cpp_lib_senders                  202506L // also in <execution>
#define __cpp_lib_shared_mutex             201505L // also in <shared_mutex>
#define __cpp_lib_shared_ptr_arrays        201707L // also in <memory>
#define __cpp_lib_shared_ptr_weak_type     201606L // also in <memory>
#define __cpp_lib_shared_timed_mutex       201402L // also in <shared_mutex>
#define __cpp_lib_shift                    202202L // also in <algorithm>

```

```

#define __cpp_lib_simd                202606L // also in <simd>
#define __cpp_lib_simd_bitops         202607L // also in <simd>
#define __cpp_lib_simd_complex        202502L // also in <simd>
#define __cpp_lib_simd_permutations  202506L // also in <simd>
#define __cpp_lib_smart_ptr_for_overwrite 202002L // also in <memory>
#define __cpp_lib_smart_ptr_owner_equality 202306L // also in <memory>
#define __cpp_lib_source_location     201907L // freestanding, also in <source_location>
#define __cpp_lib_span                202311L // freestanding, also in <span>
#define __cpp_lib_spanstream          202106L // also in <iosfwd>, <spanstream>
#define __cpp_lib_ssize              201902L // freestanding, also in <iterator>
#define __cpp_lib_sstream_from_string_view 202306L // also in <sstream>
#define __cpp_lib_stacktrace          202011L // also in <stacktrace>
#define __cpp_lib_start_lifetime      202603L // freestanding, also in <memory>
#define __cpp_lib_start_lifetime_as   202207L // freestanding, also in <memory>
#define __cpp_lib_starts_ends_with    201711L // also in <string>, <string_view>
#define __cpp_lib_stdatomic_h         202011L // also in <stdatomic.h>
#define __cpp_lib_stdbit_h            202603L // also in <stdbit.h>
#define __cpp_lib_stdckdint_h         202603L // also in <stdckdint.h>
#define __cpp_lib_string_contains     202011L // also in <string>, <string_view>
#define __cpp_lib_string_resize_and_overwrite 202110L // also in <string>
#define __cpp_lib_string_subview      202506L // also in <string>, <string_view>
#define __cpp_lib_string_udls         201304L // also in <string>
#define __cpp_lib_string_view         202403L // also in <string>, <string_view>
#define __cpp_lib_submdspan           202603L // freestanding, also in <mdspan>
#define __cpp_lib_syncbuf             201803L // also in <iosfwd>, <syncstream>
#define __cpp_lib_task                202506L // also in <execution>
#define __cpp_lib_text_encoding        202306L // also in <text_encoding>
#define __cpp_lib_thread_attributes    202606L // also in <thread>
#define __cpp_lib_three_way_comparison 201907L // freestanding, also in <compare>
#define __cpp_lib_to_address           201711L // freestanding, also in <memory>
#define __cpp_lib_to_array            201907L // freestanding, also in <array>
#define __cpp_lib_to_chars            202606L // also in <charconv>
#define __cpp_lib_to_string           202306L // also in <string>
#define __cpp_lib_to_underlying        202102L // freestanding, also in <utility>
#define __cpp_lib_transformation_trait_aliases 201304L // freestanding, also in <type_traits>
#define __cpp_lib_transparent_operators 201510L
// freestanding, also in <memory>, <functional>
#define __cpp_lib_tuple_element_t      201402L // freestanding, also in <tuple>
#define __cpp_lib_tuple_like           202311L
// also in <utility>, <tuple>, <map>, <unordered_map>
#define __cpp_lib_tuples_by_type       201304L // freestanding, also in <utility>, <tuple>
#define __cpp_lib_type_identity        201806L // freestanding, also in <type_traits>
#define __cpp_lib_type_order          202506L // also in <compare>
#define __cpp_lib_type_trait_variable_templates 201510L // freestanding, also in <type_traits>
#define __cpp_lib_uncaught_exceptions 201411L // freestanding, also in <exception>
#define __cpp_lib_unordered_map_try_emplace 201411L // also in <unordered_map>
#define __cpp_lib_unreachable          202202L // freestanding, also in <utility>
#define __cpp_lib_unwrap_ref          201811L // freestanding, also in <type_traits>
#define __cpp_lib_valarray            202511L // also in <valarray>
#define __cpp_lib_variant             202306L // also in <variant>
#define __cpp_lib_view_interface       202606L // also in <ranges>
#define __cpp_lib_void_t              201411L // freestanding, also in <type_traits>

```

³ Additionally, each of the following macros is defined in a hardened implementation:

```

#define __cpp_lib_harden_array         202502L // also in <array>
#define __cpp_lib_harden_basic_stacktrace 202506L // also in <stacktrace>
#define __cpp_lib_harden_basic_string  202502L // also in <string>
#define __cpp_lib_harden_basic_string_view 202502L // also in <string_view>
#define __cpp_lib_harden_bitset        202502L // also in <bitset>
#define __cpp_lib_harden_common_iterator 202506L // also in <iterator>
#define __cpp_lib_harden_counted_iterator 202506L // also in <iterator>
#define __cpp_lib_harden_deque         202502L // also in <deque>
#define __cpp_lib_harden_expected      202502L // also in <expected>
#define __cpp_lib_harden_forward_list  202502L // also in <forward_list>

```



```

#define __cpp_lib_hardened_inplace_vector      202502L // also in <inplace_vector>
#define __cpp_lib_hardened_list                202502L // also in <list>
#define __cpp_lib_hardened_mdspan             202502L // also in <mdspan>
#define __cpp_lib_hardened_optional           202502L // also in <optional>
#define __cpp_lib_hardened_shared_ptr_array    202506L // also in <memory>
#define __cpp_lib_hardened_span               202502L // also in <span>
#define __cpp_lib_hardened_valarray           202502L // also in <valarray>
#define __cpp_lib_hardened_vector             202502L // also in <vector>
#define __cpp_lib_hardened_view_interface      202506L // also in <ranges>

```

- 4 The macro `__cpp_lib_freestanding_operator_new` is defined to the integer literal 202306L if all the default versions of the replaceable global allocation functions meet the requirements of a hosted implementation, and to the integer literal 0 otherwise (17.6.3).
- 5 The macro `__cpp_lib_replaceable_contract_violation_handler` is defined to the integer literal 202603L if the contract-violation handler is replaceable, and to the integer literal 0 otherwise (6.11.3).
- 6 *Recommended practice:* Freestanding implementations should only define a macro from `<version>` if the implementation provides the corresponding facility in its entirety.
- 7 *Recommended practice:* A non-hardened implementation should not define macros from `<version>` required for hardened implementations.

17.3.3 Header `<limits>` synopsis

[limits.syn]

```

// all freestanding
namespace std {
    // 17.3.4, enumeration float_round_style
    enum float_round_style;

    // 17.3.5, class template numeric_limits
    template<class T> class numeric_limits;

    template<class T> class numeric_limits<const T>;
    template<class T> class numeric_limits<volatile T>;
    template<class T> class numeric_limits<const volatile T>;

    template<> class numeric_limits<bool>;

    template<> class numeric_limits<char>;
    template<> class numeric_limits<signed char>;
    template<> class numeric_limits<unsigned char>;
    template<> class numeric_limits<char8_t>;
    template<> class numeric_limits<char16_t>;
    template<> class numeric_limits<char32_t>;
    template<> class numeric_limits<wchar_t>;

    template<> class numeric_limits<short>;
    template<> class numeric_limits<int>;
    template<> class numeric_limits<long>;
    template<> class numeric_limits<long long>;
    template<> class numeric_limits<unsigned short>;
    template<> class numeric_limits<unsigned int>;
    template<> class numeric_limits<unsigned long>;
    template<> class numeric_limits<unsigned long long>;

    template<> class numeric_limits<float>;
    template<> class numeric_limits<double>;
    template<> class numeric_limits<long double>;
}

```

17.3.4 Enum `float_round_style`

[round.style]

```

namespace std {
    enum float_round_style {
        round_indeterminate    = -1,
        round_toward_zero      = 0,

```

```

    round_to_nearest      = 1,
    round_toward_infinity = 2,
    round_toward_neg_infinity = 3
};
}

```

¹ The rounding mode for floating-point arithmetic is characterized by the values:

- (1.1) — `round_indeterminate` if the rounding style is indeterminable
- (1.2) — `round_toward_zero` if the rounding style is toward zero
- (1.3) — `round_to_nearest` if the rounding style is to the nearest representable value; if there are two equally near such values, the one with an even least significant digit is chosen
- (1.4) — `round_toward_infinity` if the rounding style is toward infinity
- (1.5) — `round_toward_neg_infinity` if the rounding style is toward negative infinity

17.3.5 Class template `numeric_limits`

[`numeric_limits`]

17.3.5.1 General

[`numeric_limits.general`]

¹ The `numeric_limits` class template provides a C++ program with information about various properties of the implementation's representation of the arithmetic types.

```

namespace std {
    template<class T> class numeric_limits {
    public:
        static constexpr bool is_specialized = false;
        static constexpr T min() noexcept { return T(); }
        static constexpr T max() noexcept { return T(); }
        static constexpr T lowest() noexcept { return T(); }

        static constexpr int digits = 0;
        static constexpr int digits10 = 0;
        static constexpr int max_digits10 = 0;
        static constexpr bool is_signed = false;
        static constexpr bool is_integer = false;
        static constexpr bool is_exact = false;
        static constexpr int radix = 0;
        static constexpr T epsilon() noexcept { return T(); }
        static constexpr T round_error() noexcept { return T(); }

        static constexpr int min_exponent = 0;
        static constexpr int min_exponent10 = 0;
        static constexpr int max_exponent = 0;
        static constexpr int max_exponent10 = 0;

        static constexpr bool has_infinity = false;
        static constexpr bool has_quiet_NaN = false;
        static constexpr bool has_signaling_NaN = false;
        static constexpr T infinity() noexcept { return T(); }
        static constexpr T quiet_NaN() noexcept { return T(); }
        static constexpr T signaling_NaN() noexcept { return T(); }
        static constexpr T denorm_min() noexcept { return T(); }

        static constexpr bool is_iec559 = false;
        static constexpr bool is_bounded = false;
        static constexpr bool is_modulo = false;

        static constexpr bool traps = false;
        static constexpr bool tinyness_before = false;
        static constexpr float_round_style round_style = round_toward_zero;
    };
}

```

² For all members declared `static constexpr` in the `numeric_limits` template, specializations shall define these values in such a way that they are usable as constant expressions.

- 3 For the `numeric_limits` primary template, all data members are value-initialized and all member functions return a value-initialized object.

[*Note 1*: This means all members have zero or `false` values unless `numeric_limits` is specialized for a type. — *end note*]

- 4 Specializations shall be provided for each arithmetic type, both floating-point and integer, including `bool`. The member `is_specialized` shall be `true` for all such specializations of `numeric_limits`.
- 5 The value of each member of a specialization of `numeric_limits` on a cv-qualified type `cv T` shall be equal to the value of the corresponding member of the specialization on the unqualified type `T`.
- 6 Non-arithmetic standard types, such as `complex<T>` (29.4.3), shall not have specializations.

17.3.5.2 `numeric_limits` members

[`numeric_limits.members`]

- 1 Each member function defined in this subclause is signal-safe (17.14.5).

```
static constexpr T min() noexcept;
```

- 2 Minimum finite value.¹⁵⁰

- 3 For floating-point types with subnormal numbers, returns the minimum positive normalized value.

- 4 Meaningful for all specializations in which `is_bounded != false`, or `is_bounded == false && is_signed == false`.

```
static constexpr T max() noexcept;
```

- 5 Maximum finite value.¹⁵¹

- 6 Meaningful for all specializations in which `is_bounded != false`.

```
static constexpr T lowest() noexcept;
```

- 7 A finite value `x` such that there is no other finite value `y` where `y < x`.¹⁵²

- 8 Meaningful for all specializations in which `is_bounded != false`.

```
static constexpr int digits;
```

- 9 Number of `radix` digits that can be represented without change.

- 10 For integer types, the number of non-sign bits in the representation.

- 11 For floating-point types, the number of `radix` digits in the significand.¹⁵³

```
static constexpr int digits10;
```

- 12 Number of base 10 digits that can be represented without change.¹⁵⁴

- 13 Meaningful for all specializations in which `is_bounded != false`.

```
static constexpr int max_digits10;
```

- 14 Number of base 10 digits required to ensure that values which differ are always differentiated.

- 15 Meaningful for all floating-point types.

```
static constexpr bool is_signed;
```

- 16 `true` if the type is signed.

- 17 Meaningful for all specializations.

```
static constexpr bool is_integer;
```

- 18 `true` if the type is integer.

- 19 Meaningful for all specializations.

150) Equivalent to `CHAR_MIN`, `SHRT_MIN`, `FLT_MIN`, `DBL_MIN`, etc.

151) Equivalent to `CHAR_MAX`, `SHRT_MAX`, `FLT_MAX`, `DBL_MAX`, etc.

152) `lowest()` is necessary because not all floating-point representations have a smallest (most negative) value that is the negative of the largest (most positive) finite value.

153) Equivalent to `FLT_MANT_DIG`, `DBL_MANT_DIG`, `LDBL_MANT_DIG`.

154) Equivalent to `FLT_DIG`, `DBL_DIG`, `LDBL_DIG`.

```

static constexpr bool is_exact;
20     true if the type uses an exact representation. All integer types are exact, but not all exact types are
        integer. For example, rational and fixed-exponent representations are exact but not integer.
21     Meaningful for all specializations.

static constexpr int radix;
22     For floating-point types, specifies the base or radix of the exponent representation (often 2).155
23     For integer types, specifies the base of the representation.156
24     Meaningful for all specializations.

static constexpr T epsilon() noexcept;
25     Machine epsilon: the difference between 1 and the least value greater than 1 that is representable.157
26     Meaningful for all floating-point types.

static constexpr T round_error() noexcept;
27     Measure of the maximum rounding error.

static constexpr int min_exponent;
28     Minimum negative integer such that radix raised to the power of one less than that integer is a
        normalized floating-point number.158
29     Meaningful for all floating-point types.

static constexpr int min_exponent10;
30     Minimum negative integer such that 10 raised to that power is in the range of normalized floating-point
        numbers.159
31     Meaningful for all floating-point types.

static constexpr int max_exponent;
32     Maximum positive integer such that radix raised to the power one less than that integer is a representable
        finite floating-point number.160
33     Meaningful for all floating-point types.

static constexpr int max_exponent10;
34     Maximum positive integer such that 10 raised to that power is in the range of representable finite
        floating-point numbers.161
35     Meaningful for all floating-point types.

static constexpr bool has_infinity;
36     true if the type has a representation for positive infinity.
37     Meaningful for all floating-point types.
38     Shall be true for all specializations in which is_iec559 != false.

static constexpr bool has_quiet_NaN;
39     true if the type has a representation for a quiet (non-signaling) “Not a Number”.
40     Meaningful for all floating-point types.
41     Shall be true for all specializations in which is_iec559 != false.

```

155) Equivalent to FLT_RADIX.

156) Distinguishes types with bases other than 2 (e.g., BCD).

157) Equivalent to FLT_EPSILON, DBL_EPSILON, LDBL_EPSILON.

158) Equivalent to FLT_MIN_EXP, DBL_MIN_EXP, LDBL_MIN_EXP.

159) Equivalent to FLT_MIN_10_EXP, DBL_MIN_10_EXP, LDBL_MIN_10_EXP.

160) Equivalent to FLT_MAX_EXP, DBL_MAX_EXP, LDBL_MAX_EXP.

161) Equivalent to FLT_MAX_10_EXP, DBL_MAX_10_EXP, LDBL_MAX_10_EXP.

```

static constexpr bool has_signaling_NaN;
42     true if the type has a representation for a signaling “Not a Number”.
43     Meaningful for all floating-point types.
44     Shall be true for all specializations in which is_iec559 != false.

static constexpr T infinity() noexcept;
45     Representation of positive infinity, if available.
46     Meaningful for all specializations for which has_infinity != false. Required in specializations for
        which is_iec559 != false.

static constexpr T quiet_NaN() noexcept;
47     Representation of a quiet “Not a Number”, if available.
48     Meaningful for all specializations for which has_quiet_NaN != false. Required in specializations for
        which is_iec559 != false.

static constexpr T signaling_NaN() noexcept;
49     Representation of a signaling “Not a Number”, if available.
50     Meaningful for all specializations for which has_signaling_NaN != false. Required in specializations
        for which is_iec559 != false.

static constexpr T denorm_min() noexcept;
51     Minimum positive subnormal value, if available. Otherwise, minimum positive normalized value.
52     Meaningful for all floating-point types.

static constexpr bool is_iec559;
53     true if and only if the type adheres to ISO/IEC 60559.162
        [Note 1: The value is true for any of the types float16_t, float32_t, float64_t, or float128_t, if present
        (6.9.4). — end note]
54     Meaningful for all floating-point types.

static constexpr bool is_bounded;
55     true if the set of values representable by the type is finite.
        [Note 2: All fundamental types (6.9.3) are bounded. This member would be false for arbitrary precision types.
        — end note]
56     Meaningful for all specializations.

static constexpr bool is_modulo;
57     true if the type is modulo. A type is modulo if, for any operation involving +, −, or * on values of that
        type whose result would fall outside the range [min(), max()], the value returned differs from the true
        value by an integer multiple of max() − min() + 1.
58     [Example 1: is_modulo is false for signed integer types (6.9.3) unless an implementation, as an extension to
        this document, defines signed integer overflow to wrap. — end example]
59     Meaningful for all specializations.

static constexpr bool traps;
60     true if, at the start of the program, there exists a value of the type that would cause an arithmetic
        operation using that value to trap.
61     Meaningful for all specializations.

static constexpr bool tinyness_before;
62     true if tinyness is detected before rounding.163

```

¹⁶²) ISO/IEC 60559:2020 is the same as IEEE 754-2019.

¹⁶³) Refer to ISO/IEC 60559.

63 Meaningful for all floating-point types.

```
static constexpr float_round_style round_style;
```

64 The rounding style for the type.¹⁶⁴

65 Meaningful for all floating-point types. Specializations for integer types shall return `round_toward_zero`.

17.3.5.3 numeric_limits specializations [numeric.special]

¹ All members shall be provided for all specializations. However, many values are only required to be meaningful under certain conditions (for example, `epsilon()` is only meaningful if `is_integer` is `false`). Any value that is not “meaningful” shall be set to 0 or `false`.

² [Example 1:

```
namespace std {
    template<> class numeric_limits<float> {
    public:
        static constexpr bool is_specialized = true;

        static constexpr float min() noexcept { return 1.17549435E-38F; }
        static constexpr float max() noexcept { return 3.40282347E+38F; }
        static constexpr float lowest() noexcept { return -3.40282347E+38F; }

        static constexpr int digits    = 24;
        static constexpr int digits10  =  6;
        static constexpr int max_digits10 =  9;

        static constexpr bool is_signed  = true;
        static constexpr bool is_integer = false;
        static constexpr bool is_exact   = false;

        static constexpr int radix = 2;
        static constexpr float epsilon() noexcept { return 1.19209290E-07F; }
        static constexpr float round_error() noexcept { return 0.5F; }

        static constexpr int min_exponent    = -125;
        static constexpr int min_exponent10 = - 37;
        static constexpr int max_exponent    = +128;
        static constexpr int max_exponent10 = + 38;

        static constexpr bool has_infinity      = true;
        static constexpr bool has_quiet_NaN     = true;
        static constexpr bool has_signaling_NaN = true;

        static constexpr float infinity()      noexcept { return value; }
        static constexpr float quiet_NaN()     noexcept { return value; }
        static constexpr float signaling_NaN() noexcept { return value; }
        static constexpr float denorm_min()    noexcept { return min(); }

        static constexpr bool is_iec559 = true;
        static constexpr bool is_bounded = true;
        static constexpr bool is_modulo  = false;
        static constexpr bool traps      = true;
        static constexpr bool tinyness_before = true;

        static constexpr float_round_style round_style = round_to_nearest;
    };
}
```

— end example]

164) Equivalent to FLT_ROUNDS.

- ³ The specialization for bool shall be provided as follows:

```
namespace std {
    template<> class numeric_limits<bool> {
    public:
        static constexpr bool is_specialized = true;
        static constexpr bool min() noexcept { return false; }
        static constexpr bool max() noexcept { return true; }
        static constexpr bool lowest() noexcept { return false; }

        static constexpr int  digits = 1;
        static constexpr int  digits10 = 0;
        static constexpr int  max_digits10 = 0;

        static constexpr bool is_signed = false;
        static constexpr bool is_integer = true;
        static constexpr bool is_exact = true;
        static constexpr int  radix = 2;
        static constexpr bool epsilon() noexcept { return 0; }
        static constexpr bool round_error() noexcept { return 0; }

        static constexpr int  min_exponent = 0;
        static constexpr int  min_exponent10 = 0;
        static constexpr int  max_exponent = 0;
        static constexpr int  max_exponent10 = 0;

        static constexpr bool has_infinity = false;
        static constexpr bool has_quiet_NaN = false;
        static constexpr bool has_signaling_NaN = false;
        static constexpr bool infinity() noexcept { return 0; }
        static constexpr bool quiet_NaN() noexcept { return 0; }
        static constexpr bool signaling_NaN() noexcept { return 0; }
        static constexpr bool denorm_min() noexcept { return 0; }

        static constexpr bool is_iec559 = false;
        static constexpr bool is_bounded = true;
        static constexpr bool is_modulo = false;

        static constexpr bool traps = false;
        static constexpr bool tinyness_before = false;
        static constexpr float_round_style round_style = round_toward_zero;
    };
}
```

17.3.6 Header <climits> synopsis

[climits.syn]

```
// all freestanding
#define BOOL_WIDTH see below
#define CHAR_BIT see below
#define CHAR_WIDTH see below
#define SCHAR_WIDTH see below
#define UCHAR_WIDTH see below
#define USHRT_WIDTH see below
#define SHRT_WIDTH see below
#define UINT_WIDTH see below
#define INT_WIDTH see below
#define ULONG_WIDTH see below
#define LONG_WIDTH see below
#define ULLONG_WIDTH see below
#define LLONG_WIDTH see below
#define SCHAR_MIN see below
#define SCHAR_MAX see below
#define UCHAR_MAX see below
#define CHAR_MIN see below
#define CHAR_MAX see below
```

```

#define MB_LEN_MAX see below
#define SHRT_MIN see below
#define SHRT_MAX see below
#define USHRT_MAX see below
#define INT_MIN see below
#define INT_MAX see below
#define UINT_MAX see below
#define LONG_MIN see below
#define LONG_MAX see below
#define ULONG_MAX see below
#define LLONG_MIN see below
#define LLONG_MAX see below
#define ULLONG_MAX see below

```

- ¹ The header <climits> defines all macros the same as the C standard library header <limits.h>, except that it does not define the macro BITINT_MAXWIDTH.

[*Note 1*: Except for the WIDTH macros, CHAR_BIT, and MB_LEN_MAX, a macro referring to an integer type T defines a constant whose type is the promoted type of T (7.3.7). — *end note*]

SEE ALSO: ISO/IEC 9899:2024, 5.3.5.3.2

17.3.7 Header <cfloat> synopsis

[cfloat.syn]

```

// all freestanding
#define __STDC_VERSION_FLOAT_H__ 202311L

#define FLT_ROUNDS see below
#define FLT_EVAL_METHOD see below
#define FLT_RADIX see below
#define INFINITY see below
#define NAN see below
#define FLT_SNAN see below
#define DBL_SNAN see below
#define LDBL_SNAN see below
#define FLT_MANT_DIG see below
#define DBL_MANT_DIG see below
#define LDBL_MANT_DIG see below
#define FLT_DECIMAL_DIG see below
#define DBL_DECIMAL_DIG see below
#define LDBL_DECIMAL_DIG see below
#define FLT_DIG see below
#define DBL_DIG see below
#define LDBL_DIG see below
#define FLT_MIN_EXP see below
#define DBL_MIN_EXP see below
#define LDBL_MIN_EXP see below
#define FLT_MIN_10_EXP see below
#define DBL_MIN_10_EXP see below
#define LDBL_MIN_10_EXP see below
#define FLT_MAX_EXP see below
#define DBL_MAX_EXP see below
#define LDBL_MAX_EXP see below
#define FLT_MAX_10_EXP see below
#define DBL_MAX_10_EXP see below
#define LDBL_MAX_10_EXP see below
#define FLT_MAX see below
#define DBL_MAX see below
#define LDBL_MAX see below
#define FLT_EPSILON see below
#define DBL_EPSILON see below
#define LDBL_EPSILON see below
#define FLT_MIN see below
#define DBL_MIN see below
#define LDBL_MIN see below
#define FLT_TRUE_MIN see below

```



```
#define DBL_TRUE_MIN see below
#define LDBL_TRUE_MIN see below
```

- ¹ The header <cfloating> defines all macros the same as the C standard library header <float.h>.

SEE ALSO: ISO/IEC 9899:2024, 5.3.5.3.3

17.4 Arithmetic types

[support.arith.types]

17.4.1 Header <cstdint> synopsis

[cstdint.syn]

- ¹ The header <cstdint> supplies integer types having specified widths, and macros that specify limits of integer types.

```
// all freestanding
#define __STDC_VERSION_STDINT_H__ 202311L

namespace std {
    using int8_t      = signed integer type;    // optional
    using int16_t     = signed integer type;    // optional
    using int32_t     = signed integer type;    // optional
    using int64_t     = signed integer type;    // optional
    using intN_t      = see below;              // optional

    using int_fast8_t  = signed integer type;
    using int_fast16_t = signed integer type;
    using int_fast32_t = signed integer type;
    using int_fast64_t = signed integer type;
    using int_fastN_t  = see below;              // optional

    using int_least8_t = signed integer type;
    using int_least16_t = signed integer type;
    using int_least32_t = signed integer type;
    using int_least64_t = signed integer type;
    using int_leastN_t = see below;              // optional

    using intmax_t     = signed integer type;
    using intptr_t     = signed integer type;

    using uint8_t      = unsigned integer type; // optional
    using uint16_t     = unsigned integer type; // optional
    using uint32_t     = unsigned integer type; // optional
    using uint64_t     = unsigned integer type; // optional
    using uintN_t      = see below;              // optional

    using uint_fast8_t = unsigned integer type;
    using uint_fast16_t = unsigned integer type;
    using uint_fast32_t = unsigned integer type;
    using uint_fast64_t = unsigned integer type;
    using uint_fastN_t  = see below;              // optional

    using uint_least8_t = unsigned integer type;
    using uint_least16_t = unsigned integer type;
    using uint_least32_t = unsigned integer type;
    using uint_least64_t = unsigned integer type;
    using uint_leastN_t = see below;              // optional

    using uintmax_t     = unsigned integer type;
    using uintptr_t     = unsigned integer type;
}

#define INTN_MIN      see below
#define INTN_MAX      see below
#define UINTN_MAX      see below
#define INTN_WIDTH     see below
#define UINTN_WIDTH     see below
```

```

#define INT_FASTN_MIN      see below
#define INT_FASTN_MAX      see below
#define UINT_FASTN_MAX     see below
#define INT_FASTN_WIDTH    see below
#define UINT_FASTN_WIDTH   see below

#define INT_LEASTN_MIN     see below
#define INT_LEASTN_MAX     see below
#define UINT_LEASTN_MAX    see below
#define INT_LEASTN_WIDTH   see below
#define UINT_LEASTN_WIDTH  see below

#define INTMAX_MIN         see below
#define INTMAX_MAX         see below
#define UINTMAX_MAX        see below
#define INTMAX_WIDTH       see below
#define UINTMAX_WIDTH      see below

#define INTPTR_MIN         see below
#define INTPTR_MAX         see below
#define UINTPTR_MAX        see below
#define INTPTR_WIDTH       see below
#define UINTPTR_WIDTH      see below

#define PTRDIFF_MIN        see below
#define PTRDIFF_MAX        see below
#define PTRDIFF_WIDTH      see below
#define SIZE_MAX           see below
#define SIZE_WIDTH         see below

#define SIG_ATOMIC_MIN     see below
#define SIG_ATOMIC_MAX     see below
#define SIG_ATOMIC_WIDTH   see below

#define WCHAR_MIN          see below
#define WCHAR_MAX          see below
#define WCHAR_WIDTH        see below

#define WINT_MIN           see below
#define WINT_MAX           see below
#define WINT_WIDTH         see below

#define INTN_C(value)      see below
#define UINTN_C(value)     see below
#define INTMAX_C(value)    see below
#define UINTMAX_C(value)   see below

```

- ² The header defines all types and macros the same as the C standard library header `<stdint.h>`, except that the types `intptr_t` and `uintptr_t` and the macros `INTPTR_MIN`, `INTPTR_MAX`, and `UINTPTR_MAX` are always defined. The types denoted by `intmax_t` and `uintmax_t` are not required to be able to represent all values of extended integer types wider than `long long` and `unsigned long long`, respectively.

SEE ALSO: ISO/IEC 9899:2024, 7.22

- ³ All types that use the placeholder *N* are optional when *N* is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for *N* = 8, 16, 32, and 64 are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it declares the corresponding *typedef-names*. Each of the macros listed in this subclause is defined if and only if the implementation declares the corresponding *typedef-name*.

[Note 1: The macros `INTN_C` and `UINTN_C` correspond to the *typedef-names* `int_leastN_t` and `uint_leastN_t`, respectively. — end note]

17.4.2 Header <stdfloat> synopsis

[stdfloat.syn]

- ¹ The header <stdfloat> declares type aliases for the optional extended floating-point types that are specified in 6.9.4.

```
namespace std {
    #if defined(__STDCPP_FLOAT16_T__)
        using float16_t = implementation-defined; // see 6.9.4
    #endif
    #if defined(__STDCPP_FLOAT32_T__)
        using float32_t = implementation-defined; // see 6.9.4
    #endif
    #if defined(__STDCPP_FLOAT64_T__)
        using float64_t = implementation-defined; // see 6.9.4
    #endif
    #if defined(__STDCPP_FLOAT128_T__)
        using float128_t = implementation-defined; // see 6.9.4
    #endif
    #if defined(__STDCPP_BFLOAT16_T__)
        using bfloat16_t = implementation-defined; // see 6.9.4
    #endif
}
```

17.5 Startup and termination

[support.start.term]

- ¹ [Note 1: The header <cstdlib> (17.2.2) declares the functions described in this subclause. — end note]

```
[[noreturn]] void _Exit(int status) noexcept;
```

- ² *Effects:* This function has the semantics specified in the C standard library.

- ³ *Remarks:* The program is terminated without executing destructors for objects with automatic, thread, or static storage duration and without calling functions passed to `atexit()` (6.10.3.4). The function `_Exit` is signal-safe (17.14.5).

```
[[noreturn]] void abort() noexcept;
```

- ⁴ *Effects:* This function has the semantics specified in the C standard library.

- ⁵ *Remarks:* The program is terminated without executing destructors for objects of automatic, thread, or static storage duration and without calling functions passed to `atexit()` (6.10.3.4). The function `abort` is signal-safe (17.14.5).

```
int atexit(c-atexit-handler* f) noexcept;
```

```
int atexit(atexit-handler* f) noexcept;
```

- ⁶ *Effects:* The `atexit()` functions register the function pointed to by `f` to be called without arguments at normal program termination. It is unspecified whether a call to `atexit()` that does not happen before (6.10.2) a call to `exit()` will succeed.

[Note 2: The `atexit()` functions do not introduce a data race (16.4.6.10). — end note]

- ⁷ *Implementation limits:* The implementation shall support the registration of at least 32 functions.

- ⁸ *Returns:* The `atexit()` function returns zero if the registration succeeds, nonzero if it fails.

```
[[noreturn]] void exit(int status);
```

- ⁹ *Effects:*

- (9.1) — First, objects with thread storage duration and associated with the current thread are destroyed. Next, objects with static storage duration are destroyed and functions registered by calling `atexit` are called.¹⁶⁵ See 6.10.3.4 for the order of destructions and calls. (Objects with automatic storage duration are not destroyed as a result of calling `exit()`.)¹⁶⁶

If a registered function invoked by `exit` exits via an exception, the function `std::terminate` is invoked (14.6.2).

¹⁶⁵) A function is called for every time it is registered.

¹⁶⁶) Objects with automatic storage duration are all destroyed in a program whose `main` function (6.10.3.1) contains no objects with automatic storage duration and executes the call to `exit()`. Control can be transferred directly to such a `main` function by throwing an exception that is caught in `main`.

- (9.2) — Next, all open C streams (as mediated by the function signatures declared in `<stdio>` (31.13.1)) with unwritten buffered data are flushed, all open C streams are closed, and all files created by calling `tmpfile()` are removed.
- (9.3) — Finally, control is returned to the host environment. If `status` is zero or `EXIT_SUCCESS`, an implementation-defined form of the status *successful termination* is returned. If `status` is `EXIT_FAILURE`, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.¹⁶⁷

```
int at_quick_exit(c-atexit-handler* f) noexcept;
int at_quick_exit(atexit-handler* f) noexcept;
```

- 10 *Effects:* The `at_quick_exit()` functions register the function pointed to by `f` to be called without arguments when `quick_exit` is called. It is unspecified whether a call to `at_quick_exit()` that does not happen before (6.10.2) all calls to `quick_exit` will succeed.

[Note 3: The `at_quick_exit()` functions do not introduce a data race (16.4.6.10). — end note]

[Note 4: The order of registration could be indeterminate if `at_quick_exit` was called from more than one thread. — end note]

[Note 5: The `at_quick_exit` registrations are distinct from the `atexit` registrations, and applications might need to call both registration functions with the same argument. — end note]

- 11 *Implementation limits:* The implementation shall support the registration of at least 32 functions.

- 12 *Returns:* Zero if the registration succeeds, nonzero if it fails.

```
[[noreturn]] void quick_exit(int status) noexcept;
```

- 13 *Effects:* Functions registered by calls to `at_quick_exit` are called in the reverse order of their registration, except that a function shall be called after any previously registered functions that had already been called at the time it was registered. Objects shall not be destroyed as a result of calling `quick_exit`. If a registered function invoked by `quick_exit` exits via an exception, the function `std::terminate` is invoked (14.6.2).

[Note 6: A function registered via `at_quick_exit` is invoked by the thread that calls `quick_exit`, which can be a different thread than the one that registered it, so registered functions cannot rely on the identity of objects with thread storage duration. — end note]

After calling registered functions, `quick_exit` shall call `_Exit(status)`.

- 14 *Remarks:* The function `quick_exit` is signal-safe (17.14.5) when the functions registered with `at_quick_exit` are.

SEE ALSO: ISO/IEC 9899:2024, 7.24.5

17.6 Dynamic memory management

[support.dynamic]

17.6.1 General

[support.dynamic.general]

- ¹ The header `<new>` defines several functions that manage the allocation of dynamic storage in a program. It also defines components for reporting storage management errors.

17.6.2 Header `<new>` synopsis

[new.syn]

```
// all freestanding
namespace std {
    // 17.6.4, storage allocation errors
    class bad_alloc;
    class bad_array_new_length;

    struct destroying_delete_t {
        explicit destroying_delete_t() = default;
    };
    inline constexpr destroying_delete_t destroying_delete{};

    // global operator new control
    enum class align_val_t : size_t {};
```

¹⁶⁷ The macros `EXIT_FAILURE` and `EXIT_SUCCESS` are defined in `<cstdlib>` (17.2.2).

```

struct nothrow_t { explicit nothrow_t() = default; };
extern const nothrow_t nothrow;

using new_handler = void (*);
new_handler get_new_handler() noexcept;
new_handler set_new_handler(new_handler new_p) noexcept;

// 17.6.5, pointer optimization barrier
template<class T> constexpr T* launder(T* p) noexcept;

// 17.6.6, hardware interference size
inline constexpr size_t hardware_destructive_interference_size = implementation-defined;
inline constexpr size_t hardware_constructive_interference_size = implementation-defined;
}

// 17.6.3, storage allocation and deallocation
void* operator new(std::size_t size);
void* operator new(std::size_t size, std::align_val_t alignment);
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
void* operator new(std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;

void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::size_t size, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;

void* operator new[](std::size_t size);
void* operator new[](std::size_t size, std::align_val_t alignment);
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void* operator new[](std::size_t size, std::align_val_t alignment,
                    const std::nothrow_t&) noexcept;

void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;

constexpr void* operator new (std::size_t size, void* ptr) noexcept;
constexpr void* operator new[](std::size_t size, void* ptr) noexcept;
constexpr void operator delete (void* ptr, void*) noexcept;
constexpr void operator delete[](void* ptr, void*) noexcept;

```

17.6.3 Storage allocation and deallocation

[new.delete]

17.6.3.1 General

[new.delete.general]

- ¹ Except where otherwise specified, the provisions of 6.8.6.5 apply to the library versions of `operator new` and `operator delete`. If the value of an alignment argument passed to any of these functions is not a valid alignment value, the behavior is undefined.
- ² On freestanding implementations, it is implementation-defined whether the default versions of the replaceable global allocation functions satisfy the required behaviors described in 17.6.3.2 and 17.6.3.3.

[Note 1: A freestanding implementation's default versions of the replaceable global allocation functions can cause undefined behavior when invoked. During constant evaluation, the behaviors of those default versions are irrelevant, as those calls are omitted (7.6.2.8). — end note]

Recommended practice: If any of the default versions of the replaceable global allocation functions meet the requirements of a hosted implementation, they all should.

17.6.3.2 Single-object forms

[new.delete.single]

```
void* operator new(std::size_t size);
```

```
void* operator new(std::size_t size, std::align_val_t alignment);
```

Effects: The allocation functions (6.8.6.5.2) called by a *new-expression* (7.6.2.8) to allocate **size** bytes of storage. The second form is called for a type with new-extended alignment, and the first form is called otherwise.

Required behavior: Return a non-null pointer to suitably aligned storage (6.8.6.5), or else throw a **bad_alloc** exception. This requirement is binding on any replacement versions of these functions.

Default behavior:

- (3.1) — Executes a loop: Within the loop, the function first attempts to allocate the requested storage. Whether the attempt involves a call to the C standard library functions **malloc** or **aligned_alloc** is unspecified.
- (3.2) — Returns a pointer to the allocated storage if the attempt is successful. Otherwise, if the current **new_handler** (17.6.4.5) is a null pointer value, throws **bad_alloc**.
- (3.3) — Otherwise, the function calls the current **new_handler** function (17.6.4.3). If the called function returns, the loop repeats.
- (3.4) — The loop terminates when an attempt to allocate the requested storage is successful or when a called **new_handler** function does not return.

Remarks: This function is replaceable (9.6.5).

```
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
```

```
void* operator new(std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

Effects: Same as above, except that these are called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a **bad_alloc** exception.

Required behavior: Return a non-null pointer to suitably aligned storage (6.8.6.5), or else return a null pointer. Each of these **nothrow** versions of **operator new** returns a pointer obtained as if acquired from the (possibly replaced) corresponding non-placement function. This requirement is binding on any replacement versions of these functions.

Default behavior: Calls **operator new(size)**, or **operator new(size, alignment)**, respectively. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

[*Example 1:*

```
T* p1 = new T;           // throws bad_alloc if it fails
T* p2 = new(nothrow) T;  // returns nullptr if it fails
— end example]
```

Remarks: This function is replaceable (9.6.5).

```
void operator delete(void* ptr) noexcept;
```

```
void operator delete(void* ptr, std::size_t size) noexcept;
```

```
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
```

```
void operator delete(void* ptr, std::size_t size, std::align_val_t alignment) noexcept;
```

Preconditions: **ptr** is a null pointer or its value represents the address of a block of memory allocated by an earlier call to a (possibly replaced) **operator new(std::size_t)** or **operator new(std::size_t, std::align_val_t)** which has not been invalidated by an intervening call to **operator delete**.

If the **alignment** parameter is not present, **ptr** was returned by an allocation function without an **alignment** parameter. If present, the **alignment** argument is equal to the **alignment** argument passed to the allocation function that returned **ptr**. If present, the **size** argument is equal to the **size** argument passed to the allocation function that returned **ptr**.

Effects: The deallocation functions (6.8.6.5.3) called by a *delete-expression* (7.6.2.9) to render the value of **ptr** invalid.

Required behavior: A call to an **operator delete** with a **size** parameter may be changed to a call to the corresponding **operator delete** without a **size** parameter, without affecting memory allocation.

[*Note 1:* A conforming implementation is for **operator delete(void* ptr, std::size_t size)** to simply call **operator delete(ptr)**. — end note]

Default behavior: The functions that have a **size** parameter forward their other parameters to the corresponding function without a **size** parameter.

[*Note 2*: See the note in the below *Remarks*: paragraph. — *end note*]

Default behavior: If `ptr` is null, does nothing. Otherwise, reclaims the storage allocated by the earlier call to `operator new`.

Remarks: It is unspecified under what conditions part or all of such reclaimed storage will be allocated by subsequent calls to `operator new` or any of `aligned_alloc`, `calloc`, `malloc`, or `realloc`, declared in `<cstdlib>` (17.2.2). This function is replaceable (9.6.5). If a replacement function without a `size` parameter is defined by the program, the program should also define the corresponding function with a `size` parameter. If a replacement function with a `size` parameter is defined by the program, the program shall also define the corresponding version without the `size` parameter.

[*Note 3*: The default behavior above might change in the future, which will require replacing both deallocation functions when replacing the allocation function. — *end note*]

```
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
void operator delete(void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

Preconditions: `ptr` is a null pointer or its value represents the address of a block of memory allocated by an earlier call to a (possibly replaced) `operator new(std::size_t)` or `operator new(std::size_t, std::align_val_t)` which has not been invalidated by an intervening call to `operator delete`.

If the `alignment` parameter is not present, `ptr` was returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument is equal to the `alignment` argument passed to the allocation function that returned `ptr`.

Effects: The deallocation functions (6.8.6.5.3) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the *new-expression* throws an exception.

Default behavior: Calls `operator delete(ptr)`, or `operator delete(ptr, alignment)`, respectively.

Remarks: This function is replaceable (9.6.5).

17.6.3.3 Array forms

[`new.delete.array`]

```
void* operator new[](std::size_t size);
void* operator new[](std::size_t size, std::align_val_t alignment);
```

Effects: The allocation functions (6.8.6.5.2) called by the array form of a *new-expression* (7.6.2.8) to allocate `size` bytes of storage. The second form is called for a type with new-extended alignment, and the first form is called otherwise.¹⁶⁸

Required behavior: Same as for the corresponding single-object forms. This requirement is binding on any replacement versions of these functions.

Default behavior: Returns `operator new(size)`, or `operator new(size, alignment)`, respectively.

Remarks: This function is replaceable (9.6.5).

```
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void* operator new[](std::size_t size, std::align_val_t alignment, const std::nothrow_t&) noexcept;
```

Effects: Same as above, except that these are called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a `bad_alloc` exception.

Required behavior: Return a non-null pointer to suitably aligned storage (6.8.6.5), or else return a null pointer. Each of these nothrow versions of `operator new[]` returns a pointer obtained as if acquired from the (possibly replaced) corresponding non-placement function. This requirement is binding on any replacement versions of these functions.

Default behavior: Calls `operator new[](size)`, or `operator new[](size, alignment)`, respectively. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

Remarks: This function is replaceable (9.6.5).

¹⁶⁸ It is not the direct responsibility of `operator new[]` or `operator delete[]` to note the repetition count or element size of the array. Those operations are performed elsewhere in the array `new` and `delete` expressions. The array `new` expression, can, however, increase the `size` argument to `operator new[]` to obtain space to store supplemental information.


```

void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t alignment) noexcept;

```

Preconditions: `ptr` is a null pointer or its value represents the address of a block of memory allocated by an earlier call to a (possibly replaced) `operator new[](std::size_t)` or `operator new[](std::size_t, std::align_val_t)` which has not been invalidated by an intervening call to `operator delete[]`.

If the `alignment` parameter is not present, `ptr` was returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument is equal to the `alignment` argument passed to the allocation function that returned `ptr`. If present, the `size` argument is equal to the `size` argument passed to the allocation function that returned `ptr`.

Effects: The deallocation functions (6.8.6.5.3) called by the array form of a *delete-expression* to render the value of `ptr` invalid.

Required behavior: A call to an `operator delete[]` with a `size` parameter may be changed to a call to the corresponding `operator delete[]` without a `size` parameter, without affecting memory allocation.

[Note 1: A conforming implementation is for `operator delete[](void* ptr, std::size_t size)` to simply call `operator delete[](ptr)`. — end note]

Default behavior: The functions that have a `size` parameter forward their other parameters to the corresponding function without a `size` parameter. The functions that do not have a `size` parameter forward their parameters to the corresponding `operator delete` (single-object) function.

Remarks: This function is replaceable (9.6.5). If a replacement function without a `size` parameter is defined by the program, the program should also define the corresponding function with a `size` parameter. If a replacement function with a `size` parameter is defined by the program, the program shall also define the corresponding version without the `size` parameter.

[Note 2: The default behavior above might change in the future, which will require replacing both deallocation functions when replacing the allocation function. — end note]

```

void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment, const std::nothrow_t&) noexcept;

```

Preconditions: `ptr` is a null pointer or its value represents the address of a block of memory allocated by an earlier call to a (possibly replaced) `operator new[](std::size_t)` or `operator new[](std::size_t, std::align_val_t)` which has not been invalidated by an intervening call to `operator delete[]`.

If the `alignment` parameter is not present, `ptr` was returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument is equal to the `alignment` argument passed to the allocation function that returned `ptr`.

Effects: The deallocation functions (6.8.6.5.3) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the array *new-expression* throws an exception.

Default behavior: Calls `operator delete[](ptr)`, or `operator delete[](ptr, alignment)`, respectively.

Remarks: This function is replaceable (9.6.5).

17.6.3.4 Non-allocating forms

[new.delete.placement]

These functions are reserved; a C++ program may not define functions that displace the versions in the C++ standard library (16.4.5). The provisions of 6.8.6.5 do not apply to these reserved placement forms of `operator new` and `operator delete`.

```
constexpr void* operator new(std::size_t size, void* ptr) noexcept;
```

Returns: `ptr`.

Remarks: Intentionally performs no other action.

4 [Example 1: This can be useful for constructing an object at a known address:

```
void* place = operator new(sizeof(Something));
Something* p = new (place) Something();
— end example]
```

```
constexpr void* operator new[](std::size_t size, void* ptr) noexcept;
```

5 *Returns:* `ptr`.

6 *Remarks:* Intentionally performs no other action.

```
constexpr void operator delete(void* ptr, void*) noexcept;
```

7 *Effects:* Intentionally performs no action.

8 *Remarks:* Default function called when any part of the initialization in a placement *new-expression* that invokes the library's non-array placement operator `new` terminates by throwing an exception (7.6.2.8).

```
constexpr void operator delete[](void* ptr, void*) noexcept;
```

9 *Effects:* Intentionally performs no action.

10 *Remarks:* Default function called when any part of the initialization in a placement *new-expression* that invokes the library's array placement operator `new` terminates by throwing an exception (7.6.2.8).

17.6.3.5 Data races

[new.delete.dataraces]

1 For purposes of determining the existence of data races, the library versions of `operator new`, user replacement versions of global `operator new`, the C standard library functions `aligned_alloc`, `calloc`, and `malloc`, the library versions of `operator delete`, user replacement versions of `operator delete`, the C standard library function `free`, and the C standard library function `realloc` shall not introduce a data race (16.4.6.10). Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before (6.10.2) the next allocation (if any) in this order.

17.6.4 Storage allocation errors

[alloc.errors]

17.6.4.1 Class `bad_alloc`

[bad.alloc]

```
namespace std {
    class bad_alloc : public exception {
    public:
        // see 17.9.3 for the specification of the special member functions
        constexpr const char* what() const noexcept override;
    };
}
```

1 The class `bad_alloc` defines the type of objects thrown as exceptions by the implementation to report a failure to allocate storage.

```
constexpr const char* what() const noexcept override;
```

2 *Returns:* An implementation-defined NTBS.

17.6.4.2 Class `bad_array_new_length`

[new.badlength]

```
namespace std {
    class bad_array_new_length : public bad_alloc {
    public:
        // see 17.9.3 for the specification of the special member functions
        constexpr const char* what() const noexcept override;
    };
}
```

1 The class `bad_array_new_length` defines the type of objects thrown as exceptions by the implementation to report an attempt to allocate an array of size less than zero or greater than an implementation-defined limit (7.6.2.8).

```
constexpr const char* what() const noexcept override;
```

2 *Returns:* An implementation-defined NTBS.

17.6.4.3 Type new_handler**[new.handler]**

```
using new_handler = void (*)();
```

1 The type of a *handler function* to be called by `operator new()` or `operator new[]()` (17.6.3) when they cannot satisfy a request for additional storage.

2 *Required behavior:* A `new_handler` shall perform one of the following:

- (2.1) — make more storage available for allocation and then return;
- (2.2) — throw an exception of type `bad_alloc` or a class derived from `bad_alloc`;
- (2.3) — terminate execution of the program without returning to the caller.

17.6.4.4 set_new_handler**[set.new.handler]**

```
new_handler set_new_handler(new_handler new_p) noexcept;
```

1 *Effects:* Establishes the function designated by `new_p` as the current `new_handler`.

2 *Returns:* The previous `new_handler`.

3 *Remarks:* The initial `new_handler` is a null pointer.

17.6.4.5 get_new_handler**[get.new.handler]**

```
new_handler get_new_handler() noexcept;
```

1 *Returns:* The current `new_handler`.

[Note 1: This can be a null pointer value. — end note]

17.6.5 Pointer optimization barrier**[ptr.laundry]**

```
template<class T> constexpr T* launder(T* p) noexcept;
```

1 *Mandates:* `!is_function_v<T> && !is_void_v<T>` is true.

2 *Preconditions:* `p` represents the address *A* of a byte in memory. An object *X* whose type is similar (7.3.6) to *T* is located at the address *A*, and is either within its lifetime (6.8.4) or is an array element subobject whose containing array object is within its lifetime. All bytes of storage that would be reachable through (6.9.5) the result are reachable through `p`.

3 *Returns:* A value of type *T** that points to *X*.

4 *Remarks:* An invocation of this function may be used in a core constant expression if and only if the (converted) value of its argument may be used in place of the function invocation.

5 [Note 1: If a new object is created in storage occupied by an existing object of the same type, a pointer to the original object can be used to refer to the new object unless its complete object is a `const` object or it is a base class subobject; in the latter cases, this function can be used to obtain a usable pointer to the new object. See 6.8.4. — end note]

6 [Example 1:

```
    struct X { int n; };
    const X *p = new const X{3};
    const int a = p->n;
    new (const_cast<X*>(p)) const X{5}; // p does not point to new object (6.8.4) because its type is const
    const int b = p->n;                 // undefined behavior
    const int c = std::launder(p)->n;   // OK
```

— end example]

17.6.6 Hardware interference size**[hardware.interference]**

```
inline constexpr size_t hardware_destructive_interference_size = implementation_defined;
```

1 This number is the minimum recommended offset between two concurrently-accessed objects to avoid additional performance degradation due to contention introduced by the implementation. It shall be at least `alignof(max_align_t)`.

[Example 1:

```
struct keep_apart {
    alignas(hardware_destructive_interference_size) atomic<int> cat;
    alignas(hardware_destructive_interference_size) atomic<int> dog;
};
```

— end example]

```
inline constexpr size_t hardware_constructive_interference_size = implementation-defined;
```

- ² This number is the maximum recommended size of contiguous memory occupied by two objects accessed with temporal locality by concurrent threads. It shall be at least `alignof(max_align_t)`.

[Example 2:

```
struct together {
    atomic<int> dog;
    int puppy;
};
struct kennel {
    // Other data members...
    alignas(sizeof(together)) together pack;
    // Other data members...
};
static_assert(sizeof(together) <= hardware_constructive_interference_size);
```

— end example]

17.7 Type identification

[support.rtti]

17.7.1 General

[support.rtti.general]

- ¹ The header `<typeinfo>` (17.7.2) defines a type associated with type information generated by the implementation. It also defines two types for reporting dynamic type identification errors. The header `<typeindex>` (17.7.6) defines a wrapper type for use as an index type in associative containers (23.4) and in unordered associative containers (23.5).

17.7.2 Header `<typeinfo>` synopsis

[typeinfo.syn]

```
// all freestanding
namespace std {
    class type_info;
    class bad_cast;
    class bad_typeid;
}
```

17.7.3 Class `type_info`

[type.info]

```
namespace std {
    class type_info {
    public:
        virtual ~type_info();
        constexpr bool operator==(const type_info& rhs) const noexcept;
        bool before(const type_info& rhs) const noexcept;
        size_t hash_code() const noexcept;
        const char* name() const noexcept;

        type_info(const type_info&) = delete;
        type_info& operator=(const type_info&) = delete;
    };
}
```

- ¹ The class `type_info` describes type information generated by the implementation (7.6.1.8). Objects of this class effectively store a pointer to a name for the type, and an encoded value suitable for comparing two types for equality or collating order. The names, encoding rule, and collating sequence for types are all unspecified and may differ between programs.

```
constexpr bool operator==(const type_info& rhs) const noexcept;
```

- ² *Effects:* Compares the current object with `rhs`.

3 *Returns:* `true` if the two values describe the same type.

```
bool before(const type_info& rhs) const noexcept;
```

4 *Effects:* Compares the current object with `rhs`.

5 *Returns:* `true` if `*this` precedes `rhs` in the implementation's collation order.

```
size_t hash_code() const noexcept;
```

6 *Returns:* An unspecified value, except that within a single execution of the program, it shall return the same value for any two `type_info` objects which compare equal.

7 *Remarks:* An implementation should return different values for two `type_info` objects which do not compare equal.

```
const char* name() const noexcept;
```

8 *Returns:* An implementation-defined NTBS.

9 *Remarks:* The message may be a null-terminated multibyte string (16.3.3.3.4.3), suitable for conversion and display as a `wstring` (27.4, 28.3.4.2.5).

17.7.4 Class `bad_cast`

[`bad.cast`]

```
namespace std {
    class bad_cast : public exception {
    public:
        // see 17.9.3 for the specification of the special member functions
        constexpr const char* what() const noexcept override;
    };
}
```

1 The class `bad_cast` defines the type of objects thrown as exceptions by the implementation to report the execution of an invalid `dynamic_cast` expression (7.6.1.7).

```
constexpr const char* what() const noexcept override;
```

2 *Returns:* An implementation-defined NTBS.

17.7.5 Class `bad_typeid`

[`bad.typeid`]

```
namespace std {
    class bad_typeid : public exception {
    public:
        // see 17.9.3 for the specification of the special member functions
        constexpr const char* what() const noexcept override;
    };
}
```

1 The class `bad_typeid` defines the type of objects thrown as exceptions by the implementation to report a null pointer in a `typeid` expression (7.6.1.8).

```
constexpr const char* what() const noexcept override;
```

2 *Returns:* An implementation-defined NTBS.

17.7.6 Header `<typeindex>` synopsis

[`type.index.synopsis`]

```
#include <compare>           // see 17.12.1
#include <typeinfo>          // see 17.7.2

namespace std {
    class type_index;
    template<class T> struct hash;
    template<> struct hash<type_index>;
}
```

17.7.7 Class `type_index`[`type.index`]

```

namespace std {
    class type_index {
    public:
        type_index(const type_info& rhs) noexcept;
        bool operator==(const type_index& rhs) const noexcept;
        bool operator< (const type_index& rhs) const noexcept;
        bool operator> (const type_index& rhs) const noexcept;
        bool operator<=(const type_index& rhs) const noexcept;
        bool operator>=(const type_index& rhs) const noexcept;
        strong_ordering operator<=>(const type_index& rhs) const noexcept;
        size_t hash_code() const noexcept;
        const char* name() const noexcept;

    private:
        const type_info* target;    // exposition only
        // Note that the use of a pointer here, rather than a reference,
        // means that the default copy/move constructor and assignment
        // operators will be provided and work as expected.
    };
}

```

- 1 The class `type_index` provides a simple wrapper for `type_info` which can be used as an index type in associative containers (23.4) and in unordered associative containers (23.5).

```
type_index(const type_info& rhs) noexcept;
```

- 2 *Effects:* Constructs a `type_index` object, the equivalent of `target = &rhs`.

```
bool operator==(const type_index& rhs) const noexcept;
```

- 3 *Returns:* `*target == *rhs.target`.

```
bool operator<(const type_index& rhs) const noexcept;
```

- 4 *Returns:* `target->before(*rhs.target)`.

```
bool operator>(const type_index& rhs) const noexcept;
```

- 5 *Returns:* `rhs.target->before(*target)`.

```
bool operator<=(const type_index& rhs) const noexcept;
```

- 6 *Returns:* `!rhs.target->before(*target)`.

```
bool operator>=(const type_index& rhs) const noexcept;
```

- 7 *Returns:* `!target->before(*rhs.target)`.

```
strong_ordering operator<=>(const type_index& rhs) const noexcept;
```

- 8 *Effects:* Equivalent to:

```

    if (*target == *rhs.target) return strong_ordering::equal;
    if (target->before(*rhs.target)) return strong_ordering::less;
    return strong_ordering::greater;

```

```
size_t hash_code() const noexcept;
```

- 9 *Returns:* `target->hash_code()`.

```
const char* name() const noexcept;
```

- 10 *Returns:* `target->name()`.

```
template<> struct hash<type_index>;
```

- 11 For an object `index` of type `type_index`, `hash<type_index>()(index)` shall evaluate to the same result as `index.hash_code()`.

17.8 Source location

[support.srcloc]

17.8.1 Header <source_location> synopsis

[source.location.syn]

- ¹ The header <source_location> defines the class `source_location` that provides a means to obtain source location information.

```
// all freestanding
namespace std {
    struct source_location;
}
```

17.8.2 Class `source_location`

[support.srcloc.class]

17.8.2.1 General

[support.srcloc.class.general]

```
namespace std {
    struct source_location {
        // source location construction
        static consteval source_location current() noexcept;
        constexpr source_location() noexcept;

        constexpr source_location(const source_location&) noexcept = default;
        constexpr source_location& operator=(const source_location&) noexcept = default;

        // source location field access
        constexpr uint_least32_t line() const noexcept;
        constexpr uint_least32_t column() const noexcept;
        constexpr const char* file_name() const noexcept;
        constexpr const char* function_name() const noexcept;

    private:
        uint_least32_t line_;           // exposition only
        uint_least32_t column_;         // exposition only
        const char* file_name_;         // exposition only
        const char* function_name_;     // exposition only
    };
}
```

- ¹ The type `source_location` models `semiregular`. `is_nothrow_swappable_v<source_location>` is `true`.
[Note 1: The intent of `source_location` is to have a small size and efficient copying. It is unspecified whether the copy/move constructors and the copy/move assignment operators are trivial. — end note]
- ² The data members `file_name_` and `function_name_` always each refer to an NTBS.
- ³ The copy/move constructors and the copy/move assignment operators of `source_location` meet the following postconditions: Given two objects `lhs` and `rhs` of type `source_location`, where `lhs` is a copy/move result of `rhs`, and where `rhs_p` is a value denoting the state of `rhs` before the corresponding copy/move operation, then each of the following conditions is `true`:

- (3.1) — `strcmp(lhs.file_name(), rhs_p.file_name()) == 0`
- (3.2) — `strcmp(lhs.function_name(), rhs_p.function_name()) == 0`
- (3.3) — `lhs.line() == rhs_p.line()`
- (3.4) — `lhs.column() == rhs_p.column()`

17.8.2.2 Creation

[support.srcloc.cons]

```
static consteval source_location current() noexcept;
```

- ¹ *Returns:*

- (1.1) — When invoked by a function call whose *postfix-expression* is a (possibly parenthesized) *id-expression* naming `current`, returns a `source_location` with an implementation-defined value. The value should be affected by `#line` (15.8) in the same manner as for `__LINE__` and `__FILE__`. The values of the exposition-only data members of the returned `source_location` object are indicated in Table 43.

Table 43 — Value of object returned by `current` [tab:support.srcloc.current]

Element	Value
<code>line_</code>	A presumed line number (15.12). Line numbers are presumed to be 1-indexed; however, an implementation is encouraged to use 0 when the line number is unknown.
<code>column_</code>	An implementation-defined value denoting some offset from the start of the line denoted by <code>line_</code> . Column numbers are presumed to be 1-indexed; however, an implementation is encouraged to use 0 when the column number is unknown.
<code>file_name_</code>	A presumed name of the current source file (15.12) as an NTBS.
<code>function_name_</code>	A name of the current function such as in <code>__func__</code> (9.6.1) if any, an empty string otherwise.

(1.2) — Otherwise, when invoked in some other way, returns a `source_location` whose data members are initialized with valid but unspecified values.

2 *Remarks:* Any call to `current` that appears as a default member initializer (11.4.1), or as a subexpression thereof, should correspond to the location of the constructor definition or aggregate initialization that uses the default member initializer. Any call to `current` that appears as a default argument (9.3.4.7), or as a subexpression thereof, should correspond to the location of the invocation of the function that uses the default argument (7.6.1.3).

3 [Example 1:

```
struct s {
    source_location member = source_location::current();
    int other_member;
    s(source_location loc = source_location::current())
        : member(loc)           // values of member refer to the location of the calling function (9.3.4.7)
    {}
    s(int blather) :             // values of member refer to this location
        other_member(blather)
    {}
    s(double)           // values of member refer to this location
    {}
};

void f(source_location a = source_location::current()) {
    source_location b = source_location::current();    // values in b refer to this line
}

void g() {
    f();           // f's first argument corresponds to this line of code

    source_location c = source_location::current();
    f(c);           // f's first argument gets the same values as c, above
}
```

— end example]

```
constexpr source_location() noexcept;
```

4 *Effects:* The data members are initialized with valid but unspecified values.

17.8.2.3 Observers

[support.srcloc.obs]

```
constexpr uint_least32_t line() const noexcept;
```

1 *Returns:* `line_`.

```
constexpr uint_least32_t column() const noexcept;
```

2 *Returns:* column_.

```
constexpr const char* file_name() const noexcept;
```

3 *Returns:* file_name_.

```
constexpr const char* function_name() const noexcept;
```

4 *Returns:* function_name_.

17.9 Exception handling

[support.exception]

17.9.1 General

[support.exception.general]

- 1 The header `<exception>` defines several types and functions related to the handling of exceptions in a C++ program.

17.9.2 Header `<exception>` synopsis

[exception.syn]

```
// all freestanding
namespace std {
    class exception;
    class bad_exception;
    class nested_exception;

    using terminate_handler = void (*)(void);
    terminate_handler get_terminate() noexcept;
    terminate_handler set_terminate(terminate_handler f) noexcept;
    [[noreturn]] void terminate() noexcept;

    int uncaught_exceptions() noexcept;

    using exception_ptr = unspecified;

    constexpr exception_ptr current_exception() noexcept;    // exposition only
    exception_ptr current_exception() noexcept;
    [[noreturn]] constexpr void rethrow_exception(exception_ptr p);
    template<class E> constexpr exception_ptr make_exception_ptr(E e) noexcept;
    template<class E>
        constexpr optional<const E&> exception_ptr_cast(const exception_ptr& p) noexcept;
    template<class E> void exception_ptr_cast(const exception_ptr&&) = delete;

    template<class T> [[noreturn]] void throw_with_nested(T&& t);
    template<class E> void rethrow_if_nested(const E& e);
}
```

17.9.3 Class `exception`

[exception]

```
namespace std {
    class exception {
    public:
        constexpr exception() noexcept;
        constexpr exception(const exception&) noexcept;
        constexpr exception& operator=(const exception&) noexcept;
        constexpr virtual ~exception();
        constexpr virtual const char* what() const noexcept;
    };
}
```

- 1 The class `exception` defines the base class for the types of objects thrown as exceptions by C++ standard library components, and certain expressions, to report errors detected during program execution.
- 2 Except where explicitly specified otherwise, each standard library class `T` that derives from class `exception` has the following publicly accessible member functions, each of them having a non-throwing exception specification (14.5):
- (2.1) — default constructor (unless the class synopsis shows other constructors)

- (2.2) — copy constructor
 (2.3) — copy assignment operator

The copy constructor and the copy assignment operator meet the following postcondition: If two objects `lhs` and `rhs` both have dynamic type `T` and `lhs` is a copy of `rhs`, then `strcmp(lhs.what(), rhs.what())` is equal to 0. The `what()` member function of each such `T` satisfies the constraints specified for `exception::what()` (see below).

```
constexpr exception(const exception& rhs) noexcept;
constexpr exception& operator=(const exception& rhs) noexcept;
```

- 3 *Postconditions:* If `*this` and `rhs` both have dynamic type `exception` then the value of the expression `strcmp(what(), rhs.what())` shall equal 0.

```
constexpr virtual ~exception();
```

- 4 *Effects:* Destroys an object of class `exception`.

```
constexpr virtual const char* what() const noexcept;
```

- 5 *Returns:* An implementation-defined NTBS, which during constant evaluation is encoded with the ordinary literal encoding (5.13.3).

- 6 *Remarks:* The message may be a null-terminated multibyte string (16.3.3.3.4.3), suitable for conversion and display as a `wstring` (27.4, 28.3.4.2.5). The return value remains valid until the exception object from which it is obtained is destroyed or a non-`const` member function of the exception object is called.

17.9.4 Class `bad_exception`

[`bad.exception`]

```
namespace std {
  class bad_exception : public exception {
  public:
    // see 17.9.3 for the specification of the special member functions
    constexpr const char* what() const noexcept override;
  };
}
```

- 1 The class `bad_exception` defines the type of the object referenced by the `exception_ptr` returned from a call to `current_exception` (17.9.7) when the currently active exception object fails to copy.

```
constexpr const char* what() const noexcept override;
```

- 2 *Returns:* An implementation-defined NTBS.

17.9.5 Abnormal termination

[`exception.terminate`]

17.9.5.1 Type `terminate_handler`

[`terminate.handler`]

```
using terminate_handler = void (*)( );
```

- 1 The type of a *handler function* to be invoked by `terminate` when terminating exception processing.
- 2 *Required behavior:* A `terminate_handler` shall terminate execution of the program without returning to the caller.
- 3 *Default behavior:* The implementation's default `terminate_handler` calls `abort()`.

17.9.5.2 `set_terminate`

[`set.terminate`]

```
terminate_handler set_terminate(terminate_handler f) noexcept;
```

- 1 *Effects:* Establishes the function designated by `f` as the current handler function for terminating exception processing.
- 2 *Returns:* The previous `terminate_handler`.
- 3 *Remarks:* It is unspecified whether a null pointer value designates the default `terminate_handler`.

17.9.5.3 `get_terminate`

[`get.terminate`]

```
terminate_handler get_terminate() noexcept;
```

- 1 *Returns:* The current `terminate_handler`.

[*Note 1*: This can be a null pointer value. — *end note*]

17.9.5.4 `terminate`

[`terminate`]

[[noreturn]] void `terminate`() noexcept;

- 1 *Effects*: Calls a `terminate_handler` function. It is unspecified which `terminate_handler` function will be called if an exception is active during a call to `set_terminate`. Otherwise calls the current `terminate_handler` function.

[*Note 1*: A default `terminate_handler` is always considered a callable handler in this context. — *end note*]

- 2 *Remarks*: Called by the implementation when exception handling must be abandoned for any of several reasons (14.6.2). May also be called directly by the program.

17.9.6 `uncaught_exceptions`

[`uncaught.exceptions`]

int `uncaught_exceptions`() noexcept;

- 1 *Returns*: The number of uncaught exceptions (14.2) in the current thread.

- 2 *Remarks*: When `uncaught_exceptions`() > 0, throwing an exception can result in a call of the function `std::terminate` (14.6.2).

17.9.7 Exception propagation

[`propagation`]

using `exception_ptr` = *unspecified*;

- 1 The type `exception_ptr` can be used to refer to an exception object.

- 2 `exception_ptr` meets the requirements of *Cpp17NullablePointer* (Table 36).

- 3 Two non-null values of type `exception_ptr` are equivalent and compare equal if and only if they refer to the same exception.

- 4 The default constructor of `exception_ptr` produces the null value of the type.

- 5 `exception_ptr` shall not be implicitly convertible to any arithmetic, enumeration, or pointer type.

- 6 [*Note 1*: An implementation can use a reference-counted smart pointer as `exception_ptr`. — *end note*]

- 7 For purposes of determining the presence of a data race, operations on `exception_ptr` objects shall access and modify only the `exception_ptr` objects themselves and not the exceptions they refer to. Use of `rethrow_exception` or `exception_ptr_cast` on `exception_ptr` objects that refer to the same exception object shall not introduce a data race.

[*Note 2*: If `rethrow_exception` rethrows the same exception object (rather than a copy), concurrent access to that rethrown exception object can introduce a data race. Changes in the number of `exception_ptr` objects that refer to a particular exception do not introduce a data race. — *end note*]

- 8 All member functions are marked `constexpr`.

constexpr `exception_ptr current_exception`() noexcept;

`exception_ptr current_exception`() noexcept;

- 9 *Returns*: An `exception_ptr` object that refers to the currently handled exception (14.4) or a copy of the currently handled exception, or a null `exception_ptr` object if no exception is being handled. The referenced object shall remain valid at least as long as there is an `exception_ptr` object that refers to it. If the function needs to allocate memory and the attempt fails, it returns an `exception_ptr` object that refers to an instance of `bad_alloc`. It is unspecified whether the return values of two successive calls to `current_exception` refer to the same exception object.

[*Note 3*: That is, it is unspecified whether `current_exception` creates a new copy each time it is called. — *end note*]

If the attempt to copy the current exception object throws an exception, the function returns an `exception_ptr` object that refers to the thrown exception or, if this is not possible, to an instance of `bad_exception`.

[*Note 4*: The copy constructor of the thrown exception can also fail, so the implementation can substitute a `bad_exception` object to avoid infinite recursion. — *end note*]

```
[[noreturn]] constexpr void rethrow_exception(exception_ptr p);
```

10 *Preconditions:* `p` is not a null pointer.

11 *Effects:* Let *u* be the exception object to which `p` refers, or a copy of that exception object. It is unspecified whether a copy is made, and memory for the copy is allocated in an unspecified way.

(11.1) — If allocating memory to form *u* fails, throws an instance of `bad_alloc`;

(11.2) — otherwise, if copying the exception to which `p` refers to form *u* throws an exception, throws that exception;

(11.3) — otherwise, throws *u*.

```
template<class E> constexpr exception_ptr make_exception_ptr(E e) noexcept;
```

12 *Effects:* Creates an `exception_ptr` object that refers to a copy of `e`, as if:

```
    try {
        throw e;
    } catch(...) {
        return current_exception();
    }
```

```
template<class E>
```

```
    constexpr optional<const E&> exception_ptr_cast(const exception_ptr& p) noexcept;
```

13 *Mandates:* `E` is a cv-unqualified complete object type. `E` is not an array type. `E` is not a pointer or pointer-to-member type.

[*Note 5:* When `E` is a pointer or pointer-to-member type, a handler of type `const E&` can match without binding to the exception object itself. — *end note*]

14 *Returns:* An `optional` containing a reference to the exception object referred to by `p`, if `p` is not null and a handler of type `const E&` would be a match (14.4) for that exception object. Otherwise, `nullopt`.

17.9.8 nested_exception

[except.nested]

```
namespace std {
    class nested_exception {
    public:
        nested_exception() noexcept;
        nested_exception(const nested_exception&) noexcept = default;
        nested_exception& operator=(const nested_exception&) noexcept = default;
        virtual ~nested_exception() = default;

        // access functions
        [[noreturn]] void rethrow_nested() const;
        exception_ptr nested_ptr() const noexcept;
    };

    template<class T> [[noreturn]] void throw_with_nested(T&& t);
    template<class E> void rethrow_if_nested(const E& e);
}
```

1 The class `nested_exception` is designed for use as a mixin through multiple inheritance. It captures the currently handled exception and stores it for later use.

2 [*Note 1:* `nested_exception` has a virtual destructor to make it a polymorphic class. Its presence can be tested for with `dynamic_cast`. — *end note*]

```
nested_exception() noexcept;
```

3 *Effects:* The constructor calls `current_exception()` and stores the returned value.

```
[[noreturn]] void rethrow_nested() const;
```

4 *Effects:* If `nested_ptr()` returns a null pointer, the function calls the function `std::terminate`. Otherwise, it throws the stored exception captured by `*this`.

```
exception_ptr nested_ptr() const noexcept;
```

5 *Returns:* The stored exception captured by this `nested_exception` object.

```
template<class T> [[noreturn]] void throw_with_nested(T&& t);
```

6 Let U be `decay_t<T>`.

7 *Preconditions:* U meets the *Cpp17CopyConstructible* requirements.

8 *Throws:* If `is_class_v<U> && !is_final_v<U> && !is_base_of_v<nested_exception, U>` is true, an exception of unspecified type that is publicly derived from both U and `nested_exception` and constructed from `std::forward<T>(t)`, otherwise `std::forward<T>(t)`.

```
template<class E> void rethrow_if_nested(const E& e);
```

9 *Effects:* If E is not a polymorphic class type, or if `nested_exception` is an inaccessible or ambiguous base class of E, there is no effect. Otherwise, performs:

```
    if (auto p = dynamic_cast<const nested_exception*>(addressof(e)))
        p->rethrow_nested();
```

17.10 Contract-violation handling

[support.contract]

17.10.1 Header `<contracts>` synopsis

[contracts.syn]

1 The header `<contracts>` defines types for reporting information about contract violations (6.11.2).

// all freestanding

```
namespace std::contracts {
```

```
    enum class assertion_kind : unspecified {
        pre = 1,
        post = 2,
        assert = 3
    };
```

```
    enum class evaluation_semantic : unspecified {
        ignore = 1,
        observe = 2,
        enforce = 3,
        quick_enforce = 4
    };
```

```
    enum class detection_mode : unspecified {
        predicate_false = 1,
        evaluation_exception = 2
    };
```

```
    class contract_violation {
        // no user-accessible constructor
    public:
        contract_violation(const contract_violation&) = delete;
        contract_violation& operator=(const contract_violation&) = delete;

        see below ~contract_violation();

        const char* comment() const noexcept;
        contracts::detection_mode detection_mode() const noexcept;
        bool is_terminating() const noexcept;
        assertion_kind kind() const noexcept;
        source_location location() const noexcept;
        evaluation_semantic semantic() const noexcept;
    };
```

```
    void invoke_default_contract_violation_handler(const contract_violation&);
}
```

17.10.2 Enumerations

[support.contract.enum]

1 *Recommended practice:* For all enumerations in 17.10.2, if implementation-defined enumerators are provided, they should have a minimum value of 1000.

- ² The enumerators of `assertion_kind` correspond to the syntactic forms of a contract assertion (6.11.1), with meanings listed in Table 44.

Table 44 — Enum `assertion_kind` [tab:support.contract.enum.kind]

Name	Meaning
<code>pre</code>	A precondition assertion
<code>post</code>	A postcondition assertion
<code>assert</code>	An <i>assertion-statement</i>

- ³ The enumerators of `evaluation_semantic` correspond to the evaluation semantics with which a contract assertion may be evaluated (6.11.2), with meanings listed in Table 45.

Table 45 — Enum `evaluation_semantic` [tab:support.contract.enum.semantic]

Name	Meaning
<code>ignore</code>	Ignore evaluation semantic
<code>observe</code>	Observe evaluation semantic
<code>enforce</code>	Enforce evaluation semantic
<code>quick_enforce</code>	Quick-enforce evaluation semantic

- ⁴ The enumerators of `detection_mode` correspond to the manners in which a contract violation can be identified (6.11.2), with meanings listed in Table 46.

Table 46 — Enum `detection_mode` [tab:support.contract.enum.detection]

Name	Meaning
<code>predicate_false</code>	The predicate of the contract assertion evaluated to <code>false</code> or would have evaluated to <code>false</code> .
<code>evaluation_exception</code>	An uncaught exception occurred during evaluation of the contract assertion.

17.10.3 Class `contract_violation`

[support.contract.violation]

- ¹ The class `contract_violation` defines the type of objects used to represent a contract violation that has been detected during the evaluation of a contract assertion with a particular evaluation semantic (6.11.2). Objects of this type can be created only by the implementation. It is implementation-defined whether the destructor is virtual.

```
const char* comment() const noexcept;
```

- ² *Returns:* An implementation-defined NTMBS in the ordinary literal encoding (5.3.1).

- ³ *Recommended practice:* The string returned should contain a textual representation of the predicate of the violated contract assertion or an empty string if storing a textual representation is undesired.

[*Note 1:* The string can represent a truncated, reformatted, or summarized rendering of the predicate, before or after preprocessing. — *end note*]

```
contracts::detection_mode detection_mode() const noexcept;
```

- ⁴ *Returns:* The enumerator value corresponding to the manner in which the contract violation was identified.

```
bool is_terminating() const noexcept;
```

- ⁵ *Returns:* `true` if the evaluation semantic is a terminating semantic (6.11.2); otherwise, `false`.

```
assertion_kind kind() const noexcept;
```

- ⁶ *Returns:* The enumerator value corresponding to the syntactic form of the violated contract assertion.

```
source_location location() const noexcept;
```

7 *Returns:* A `source_location` object with implementation-defined value.

8 *Recommended practice:* The value returned should be a default constructed `source_location` object or a value identifying the violated contract assertion:

(8.1) — When possible, if the violated contract assertion was a precondition, the source location of the function invocation should be returned.

(8.2) — Otherwise, the source location of the contract assertion should be returned.

```
evaluation_semantic semantic() const noexcept;
```

9 *Returns:* The enumerator value corresponding to the evaluation semantic with which the violated contract assertion was evaluated.

17.10.4 Invoke default handler [support.contract.invoke]

```
void invoke_default_contract_violation_handler(const contract_violation& v);
```

1 *Effects:* Invokes the default contract-violation handler (6.11.3) with the argument `v`.

17.11 Initializer lists [support.initlist]

17.11.1 General [support.initlist.general]

1 The header `<initializer_list>` defines a class template and several support functions related to list-initialization (see 9.5.5). All functions specified in 17.11 are signal-safe (17.14.5).

17.11.2 Header `<initializer_list>` synopsis [initializer.list.syn]

```
// all freestanding
namespace std {
    template<class E> class initializer_list {
    public:
        using value_type      = E;
        using reference       = const E&;
        using const_reference = const E&;
        using size_type       = size_t;

        using iterator        = const E*;
        using const_iterator  = const E*;

        constexpr initializer_list() noexcept;

        constexpr const E* data() const noexcept;
        constexpr size_t size() const noexcept;
        constexpr bool empty() const noexcept;
        constexpr const E* begin() const noexcept;
        constexpr const E* end() const noexcept;
    };
}
```

1 An object of type `initializer_list<E>` provides access to an array of objects of type `const E`.

[Note 1: A pair of pointers or a pointer plus a length would be obvious representations for `initializer_list`. `initializer_list` is used to implement initializer lists as specified in 9.5.5. Copying an `initializer_list` does not copy the underlying elements. — end note]

2 If an explicit specialization or partial specialization of `initializer_list` is declared, the program is ill-formed.

17.11.3 Initializer list constructors [support.initlist.cons]

```
constexpr initializer_list() noexcept;
```

1 *Postconditions:* `size() == 0`.

17.11.4 Initializer list access**[support.initlist.access]**

```
constexpr const E* begin() const noexcept;
```

- 1 *Returns:* A pointer to the beginning of the array. If `size() == 0` the values of `begin()` and `end()` are unspecified but they shall be identical.

```
constexpr const E* end() const noexcept;
```

- 2 *Returns:* `begin() + size()`.

```
constexpr const E* data() const noexcept;
```

- 3 *Returns:* `begin()`.

```
constexpr size_t size() const noexcept;
```

- 4 *Returns:* The number of elements in the array.

- 5 *Complexity:* Constant.

```
constexpr bool empty() const noexcept;
```

- 6 *Returns:* `size() == 0`.

17.12 Comparisons**[cmp]****17.12.1 Header <compare> synopsis****[compare.syn]**

- 1 The header <compare> specifies types, objects, and functions for use primarily in connection with the three-way comparison operator (7.6.8).

```
// all freestanding
namespace std {
    // 17.12.2, comparison category types
    class partial_ordering;
    class weak_ordering;
    class strong_ordering;

    // named comparison functions
    constexpr bool is_eq (partial_ordering cmp) noexcept { return cmp == 0; }
    constexpr bool is_neq (partial_ordering cmp) noexcept { return cmp != 0; }
    constexpr bool is_lt (partial_ordering cmp) noexcept { return cmp < 0; }
    constexpr bool is_lteq (partial_ordering cmp) noexcept { return cmp <= 0; }
    constexpr bool is_gt (partial_ordering cmp) noexcept { return cmp > 0; }
    constexpr bool is_gteq (partial_ordering cmp) noexcept { return cmp >= 0; }

    // 17.12.3, common comparison category type
    template<class... Ts>
    struct common_comparison_category {
        using type = see below;
    };
    template<class... Ts>
        using common_comparison_category_t = common_comparison_category<Ts...>::type;

    // 17.12.4, concept three_way_comparable
    template<class T, class Cat = partial_ordering>
        concept three_way_comparable = see below;
    template<class T, class U, class Cat = partial_ordering>
        concept three_way_comparable_with = see below;

    // 17.12.5, result of three-way comparison
    template<class T, class U = T> struct compare_three_way_result;

    template<class T, class U = T>
        using compare_three_way_result_t = compare_three_way_result<T, U>::type;

    // 22.10.8.8, class compare_three_way
    struct compare_three_way;
```

```

// 17.12.6, comparison algorithms
inline namespace unspecified {
    inline constexpr unspecified strong_order = unspecified;
    inline constexpr unspecified weak_order = unspecified;
    inline constexpr unspecified partial_order = unspecified;
    inline constexpr unspecified compare_strong_order_fallback = unspecified;
    inline constexpr unspecified compare_weak_order_fallback = unspecified;
    inline constexpr unspecified compare_partial_order_fallback = unspecified;
}

// 17.12.7, type ordering
template<class T, class U>
    struct type_order;

template<class T, class U>
    constexpr strong_ordering type_order_v = type_order<T, U>::value;
}

```

17.12.2 Comparison category types

[cmp.categories]

17.12.2.1 Preamble

[cmp.categories.pre]

- ¹ The types `partial_ordering`, `weak_ordering`, and `strong_ordering` are collectively termed the *comparison category types*. Each is specified in terms of an exposition-only data member named `value` whose value typically corresponds to that of an enumerator from one of the following exposition-only enumerations:

```

enum class ord { equal = 0, equivalent = equal, less = -1, greater = 1 }; // exposition only
enum class ncmp { unordered = -127 }; // exposition only

```

- ² [Note 1: The type `strong_ordering` corresponds to the term total ordering in mathematics. — end note]
- ³ The relational and equality operators for the comparison category types are specified with an anonymous parameter of unspecified type. This type shall be selected by the implementation such that these parameters can accept literal 0 as a corresponding argument.

[Example 1: `nullptr_t` meets this requirement. — end example]

In this context, the behavior of a program that supplies an argument other than a literal 0 is undefined.

- ⁴ For the purposes of 17.12.2, *substitutability* is the property that `f(a) == f(b)` is `true` whenever `a == b` is `true`, where `f` denotes a function that reads only comparison-salient state that is accessible via the argument's public const members.

17.12.2.2 Class `partial_ordering`

[cmp.partialord]

- ¹ The `partial_ordering` type is typically used as the result type of a three-way comparison operator (7.6.8) for a type that admits all of the six two-way comparison operators (7.6.9, 7.6.10), for which equality need not imply substitutability, and that permits two values to be incomparable.¹⁶⁹

```

namespace std {
    class partial_ordering {
        int value; // exposition only
        bool is-ordered; // exposition only

        // exposition-only constructors
        constexpr explicit
            partial_ordering(ord v) noexcept : value(int(v)), is-ordered(true) {} // exposition only
        constexpr explicit
            partial_ordering(ncmp v) noexcept : value(int(v)), is-ordered(false) {} // exposition only

    public:
        // valid values
        static const partial_ordering less;
        static const partial_ordering equivalent;
        static const partial_ordering greater;
        static const partial_ordering unordered;
    };
}

```

¹⁶⁹) That is, `a < b`, `a == b`, and `a > b` might all be false.


```

// comparisons
friend constexpr bool operator==(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator==(partial_ordering v, partial_ordering w) noexcept = default;
friend constexpr bool operator< (partial_ordering v, unspecified) noexcept;
friend constexpr bool operator> (partial_ordering v, unspecified) noexcept;
friend constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator>=(partial_ordering v, unspecified) noexcept;
friend constexpr bool operator< (unspecified, partial_ordering v) noexcept;
friend constexpr bool operator> (unspecified, partial_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, partial_ordering v) noexcept;
friend constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;
friend constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;
};

// valid values' definitions
inline constexpr partial_ordering partial_ordering::less(ord::less);
inline constexpr partial_ordering partial_ordering::equivalent(ord::equivalent);
inline constexpr partial_ordering partial_ordering::greater(ord::greater);
inline constexpr partial_ordering partial_ordering::unordered(ncmp::unordered);
}

```

```

constexpr bool operator==(partial_ordering v, unspecified) noexcept;
constexpr bool operator< (partial_ordering v, unspecified) noexcept;
constexpr bool operator> (partial_ordering v, unspecified) noexcept;
constexpr bool operator<=(partial_ordering v, unspecified) noexcept;
constexpr bool operator>=(partial_ordering v, unspecified) noexcept;

```

2 Returns: For operator@, v.is-ordered && v.value @ 0.

```

constexpr bool operator< (unspecified, partial_ordering v) noexcept;
constexpr bool operator> (unspecified, partial_ordering v) noexcept;
constexpr bool operator<=(unspecified, partial_ordering v) noexcept;
constexpr bool operator>=(unspecified, partial_ordering v) noexcept;

```

3 Returns: For operator@, v.is-ordered && 0 @ v.value.

```

constexpr partial_ordering operator<=>(partial_ordering v, unspecified) noexcept;

```

4 Returns: v.

```

constexpr partial_ordering operator<=>(unspecified, partial_ordering v) noexcept;

```

5 Returns: v < 0 ? partial_ordering::greater : v > 0 ? partial_ordering::less : v.

17.12.2.3 Class weak_ordering

[cmp.weakord]

1 The weak_ordering type is typically used as the result type of a three-way comparison operator (7.6.8) for a type that admits all of the six two-way comparison operators (7.6.9, 7.6.10) and for which equality need not imply substitutability.

```

namespace std {
    class weak_ordering {
        int value; // exposition only

        // exposition-only constructors
        constexpr explicit weak_ordering(ord v) noexcept : value(int(v)) {} // exposition only

    public:
        // valid values
        static const weak_ordering less;
        static const weak_ordering equivalent;
        static const weak_ordering greater;

        // conversions
        constexpr operator partial_ordering() const noexcept;
    };
}

```

```

// comparisons
friend constexpr bool operator==(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator==(weak_ordering v, weak_ordering w) noexcept = default;
friend constexpr bool operator< (weak_ordering v, unspecified) noexcept;
friend constexpr bool operator> (weak_ordering v, unspecified) noexcept;
friend constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator>=(weak_ordering v, unspecified) noexcept;
friend constexpr bool operator< (unspecified, weak_ordering v) noexcept;
friend constexpr bool operator> (unspecified, weak_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, weak_ordering v) noexcept;
friend constexpr weak_ordering operator<=(weak_ordering v, unspecified) noexcept;
friend constexpr weak_ordering operator<=(unspecified, weak_ordering v) noexcept;
};

// valid values' definitions
inline constexpr weak_ordering weak_ordering::less(ord::less);
inline constexpr weak_ordering weak_ordering::equivalent(ord::equivalent);
inline constexpr weak_ordering weak_ordering::greater(ord::greater);
}

```

```
constexpr operator partial_ordering() const noexcept;
```

2 *Returns:*

```

    value == 0 ? partial_ordering::equivalent :
    value < 0 ? partial_ordering::less :
               partial_ordering::greater

```

```

constexpr bool operator==(weak_ordering v, unspecified) noexcept;
constexpr bool operator< (weak_ordering v, unspecified) noexcept;
constexpr bool operator> (weak_ordering v, unspecified) noexcept;
constexpr bool operator<=(weak_ordering v, unspecified) noexcept;
constexpr bool operator>=(weak_ordering v, unspecified) noexcept;

```

3 *Returns:* $v.value @ 0$ for `operator@`.

```

constexpr bool operator< (unspecified, weak_ordering v) noexcept;
constexpr bool operator> (unspecified, weak_ordering v) noexcept;
constexpr bool operator<=(unspecified, weak_ordering v) noexcept;
constexpr bool operator>=(unspecified, weak_ordering v) noexcept;

```

4 *Returns:* $0 @ v.value$ for `operator@`.

```
constexpr weak_ordering operator<=(weak_ordering v, unspecified) noexcept;
```

5 *Returns:* v .

```
constexpr weak_ordering operator<=(unspecified, weak_ordering v) noexcept;
```

6 *Returns:* $v < 0 ? weak_ordering::greater : v > 0 ? weak_ordering::less : v$.

17.12.2.4 Class `strong_ordering`

[cmp.strongord]

1 The `strong_ordering` type is typically used as the result type of a three-way comparison operator (7.6.8) for a type that admits all of the six two-way comparison operators (7.6.9, 7.6.10) and for which equality does imply substitutability.

```

namespace std {
    class strong_ordering {
        int value; // exposition only

        // exposition-only constructors
        constexpr explicit strong_ordering(ord v) noexcept : value(int(v)) {} // exposition only

    public:
        // valid values
        static const strong_ordering less;
        static const strong_ordering equal;

```

```

static const strong_ordering equivalent;
static const strong_ordering greater;

// conversions
constexpr operator partial_ordering() const noexcept;
constexpr operator weak_ordering() const noexcept;

// comparisons
friend constexpr bool operator==(strong_ordering v, unspecified) noexcept;
friend constexpr bool operator==(strong_ordering v, strong_ordering w) noexcept = default;
friend constexpr bool operator< (strong_ordering v, unspecified) noexcept;
friend constexpr bool operator> (strong_ordering v, unspecified) noexcept;
friend constexpr bool operator<=(strong_ordering v, unspecified) noexcept;
friend constexpr bool operator>=(strong_ordering v, unspecified) noexcept;
friend constexpr bool operator< (unspecified, strong_ordering v) noexcept;
friend constexpr bool operator> (unspecified, strong_ordering v) noexcept;
friend constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
friend constexpr bool operator>=(unspecified, strong_ordering v) noexcept;
friend constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;
friend constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;
};

// valid values' definitions
inline constexpr strong_ordering strong_ordering::less(ord::less);
inline constexpr strong_ordering strong_ordering::equal(ord::equal);
inline constexpr strong_ordering strong_ordering::equivalent(ord::equivalent);
inline constexpr strong_ordering strong_ordering::greater(ord::greater);
}

```

```
constexpr operator partial_ordering() const noexcept;
```

2 *Returns:*

```

    value == 0 ? partial_ordering::equivalent :
    value < 0  ? partial_ordering::less :
               partial_ordering::greater

```

```
constexpr operator weak_ordering() const noexcept;
```

3 *Returns:*

```

    value == 0 ? weak_ordering::equivalent :
    value < 0  ? weak_ordering::less :
               weak_ordering::greater

```

```

constexpr bool operator==(strong_ordering v, unspecified) noexcept;
constexpr bool operator< (strong_ordering v, unspecified) noexcept;
constexpr bool operator> (strong_ordering v, unspecified) noexcept;
constexpr bool operator<=(strong_ordering v, unspecified) noexcept;
constexpr bool operator>=(strong_ordering v, unspecified) noexcept;

```

4 *Returns:* $v.value @ 0$ for operator@.

```

constexpr bool operator< (unspecified, strong_ordering v) noexcept;
constexpr bool operator> (unspecified, strong_ordering v) noexcept;
constexpr bool operator<=(unspecified, strong_ordering v) noexcept;
constexpr bool operator>=(unspecified, strong_ordering v) noexcept;

```

5 *Returns:* $0 @ v.value$ for operator@.

```
constexpr strong_ordering operator<=>(strong_ordering v, unspecified) noexcept;
```

6 *Returns:* v .

```
constexpr strong_ordering operator<=>(unspecified, strong_ordering v) noexcept;
```

7 *Returns:* $v < 0 ? strong_ordering::greater : v > 0 ? strong_ordering::less : v$.

17.12.3 Class template `common_comparison_category`

[cmp.common]

- ¹ The type `common_comparison_category` provides an alias for the strongest comparison category to which all of the template arguments can be converted.

[Note 1: A comparison category type is stronger than another if they are distinct types and an instance of the former can be converted to an instance of the latter. — end note]

```
template<class... Ts>
struct common_comparison_category {
    using type = see below;
};
```

- ² *Remarks:* The member *typedef-name* `type` denotes the common comparison type (11.10.3) of `Ts...`, the expanded parameter pack, or `void` if any element of `Ts` is not a comparison category type.

[Note 2: This is `std::strong_ordering` if the expansion is empty. — end note]

17.12.4 Concept `three_way_comparable`

[cmp.concept]

```
template<class T, class Cat>
concept compares-as = // exposition only
    same_as<common_comparison_category_t<T, Cat>, Cat>;

template<class T, class U>
concept partially-ordered-with = // exposition only
    requires(const remove_reference_t<T>& t, const remove_reference_t<U>& u) {
        { t < u } -> boolean-testable;
        { t > u } -> boolean-testable;
        { t <= u } -> boolean-testable;
        { t >= u } -> boolean-testable;
        { u < t } -> boolean-testable;
        { u > t } -> boolean-testable;
        { u <= t } -> boolean-testable;
        { u >= t } -> boolean-testable;
    };
};
```

- ¹ Let `t` and `u` be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. `T` and `U` model *partially-ordered-with*`<T, U>` only if

- (1.1) — `t < u`, `t <= u`, `t > u`, `t >= u`, `u < t`, `u <= t`, `u > t`, and `u >= t` have the same domain,
- (1.2) — `bool(t < u) == bool(u > t)` is true,
- (1.3) — `bool(u < t) == bool(t > u)` is true,
- (1.4) — `bool(t <= u) == bool(u >= t)` is true, and
- (1.5) — `bool(u <= t) == bool(t >= u)` is true.

```
template<class T, class Cat = partial_ordering>
concept three_way_comparable =
    weakly-equality-comparable-with<T, T> &&
    partially-ordered-with<T, T> &&
    requires(const remove_reference_t<T>& a, const remove_reference_t<T>& b) {
        { a <=> b } -> compares-as<Cat>;
    };
};
```

- ² Let `a` and `b` be lvalues of type `const remove_reference_t<T>`. `T` and `Cat` model *three_way_comparable*`<T, Cat>` only if

- (2.1) — `(a <=> b == 0) == bool(a == b)` is true,
- (2.2) — `(a <=> b != 0) == bool(a != b)` is true,
- (2.3) — `((a <=> b) <=> 0) and (0 <=> (b <=> a))` are equal,
- (2.4) — `(a <=> b < 0) == bool(a < b)` is true,
- (2.5) — `(a <=> b > 0) == bool(a > b)` is true,
- (2.6) — `(a <=> b <= 0) == bool(a <= b)` is true,
- (2.7) — `(a <=> b >= 0) == bool(a >= b)` is true, and

- (2.8) — if `Cat` is convertible to `strong_ordering`, `T` models `totally_ordered` (18.5.5).

```
template<class T, class U, class Cat = partial_ordering>
concept three_way_comparable_with =
    three_way_comparable<T, Cat> &&
    three_way_comparable<U, Cat> &&
    comparison-common-type-with<T, U> &&
    three_way_comparable<
        common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>, Cat> &&
        weakly-equality-comparable-with<T, U> &&
        partially-ordered-with<T, U> &&
    requires(const remove_reference_t<T>& t, const remove_reference_t<U>& u) {
        { t <=> u } -> compares-as<Cat>;
        { u <=> t } -> compares-as<Cat>;
    };
```

- ³ Let `t` and `t2` be lvalues denoting distinct equal objects of types `const remove_reference_t<T>` and `remove_cvref_t<T>`, respectively, and let `u` and `u2` be lvalues denoting distinct equal objects of types `const remove_reference_t<U>` and `remove_cvref_t<U>`, respectively. Let `C` be `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>`. Let `CONVERT_TO_LVALUE<C>(E)` be defined as in 18.5.1. `T`, `U`, and `Cat` model `three_way_comparable_with<T, U, Cat>` only if

- (3.1) — `t <=> u` and `u <=> t` have the same domain,
- (3.2) — `((t <=> u) <=> 0)` and `(0 <=> (u <=> t))` are equal,
- (3.3) — `(t <=> u == 0) == bool(t == u)` is true,
- (3.4) — `(t <=> u != 0) == bool(t != u)` is true,
- (3.5) — `Cat(t <=> u) == Cat(CONVERT_TO_LVALUE<C>(t2) <=> CONVERT_TO_LVALUE<C>(u2))` is true,
- (3.6) — `(t <=> u < 0) == bool(t < u)` is true,
- (3.7) — `(t <=> u > 0) == bool(t > u)` is true,
- (3.8) — `(t <=> u <= 0) == bool(t <= u)` is true,
- (3.9) — `(t <=> u >= 0) == bool(t >= u)` is true, and
- (3.10) — if `Cat` is convertible to `strong_ordering`, `T` and `U` model `totally_ordered_with<T, U>` (18.5.5).

17.12.5 Result of three-way comparison

[cmp.result]

- ¹ The behavior of a program that adds specializations for the `compare_three_way_result` template defined in this subclause is undefined.
- ² For the `compare_three_way_result` type trait applied to the types `T` and `U`, let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`, respectively. If the expression `t <=> u` is well-formed when treated as an unevaluated operand (7.2.3), the member `typedef-name` type denotes the type `decltype(t <=> u)`. Otherwise, there is no member type.

17.12.6 Comparison algorithms

[cmp.alg]

- ¹ The name `strong_order` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `strong_order(E, F)` is expression-equivalent (3.22) to the following:

- (1.1) — If the decayed types of `E` and `F` differ, `strong_order(E, F)` is ill-formed.
- (1.2) — Otherwise, `strong_ordering(strong_order(E, F))` if it is a well-formed expression where the meaning of `strong_order` is established as-if by performing argument-dependent lookup only (6.5.4).
- (1.3) — Otherwise, if the decayed type `T` of `E` is a floating-point type, yields a value of type `strong_ordering` that is consistent with the ordering observed by `T`'s comparison operators, and if `numeric_limits<T>::is_iec559` is true, is additionally consistent with the `totalOrder` operation as specified in ISO/IEC 60559.
- (1.4) — Otherwise, `strong_ordering(compare_three_way()(E, F))` if it is a well-formed expression.
- (1.5) — Otherwise, `strong_order(E, F)` is ill-formed.

[Note 1: Ill-formed cases above result in substitution failure when `strong_order(E, F)` appears in the immediate context of a template instantiation. — end note]

² The name `weak_order` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `weak_order(E, F)` is expression-equivalent (3.22) to the following:

- (2.1) — If the decayed types of `E` and `F` differ, `weak_order(E, F)` is ill-formed.
- (2.2) — Otherwise, `weak_ordering(weak_order(E, F))` if it is a well-formed expression where the meaning of `weak_order` is established as-if by performing argument-dependent lookup only (6.5.4).
- (2.3) — Otherwise, if the decayed type `T` of `E` is a floating-point type, yields a value of type `weak_ordering` that is consistent with the ordering observed by `T`'s comparison operators and `strong_order`, and if `numeric_limits<T>::is_iec559` is `true`, is additionally consistent with the following equivalence classes, ordered from lesser to greater:
 - (2.3.1) — together, all negative NaN values;
 - (2.3.2) — negative infinity;
 - (2.3.3) — each normal negative value;
 - (2.3.4) — each subnormal negative value;
 - (2.3.5) — together, both zero values;
 - (2.3.6) — each subnormal positive value;
 - (2.3.7) — each normal positive value;
 - (2.3.8) — positive infinity;
 - (2.3.9) — together, all positive NaN values.
- (2.4) — Otherwise, `weak_ordering(compare_three_way()(E, F))` if it is a well-formed expression.
- (2.5) — Otherwise, `weak_ordering(strong_order(E, F))` if it is a well-formed expression.
- (2.6) — Otherwise, `weak_order(E, F)` is ill-formed.

[Note 2: Ill-formed cases above result in substitution failure when `weak_order(E, F)` appears in the immediate context of a template instantiation. — end note]

³ The name `partial_order` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `partial_order(E, F)` is expression-equivalent (3.22) to the following:

- (3.1) — If the decayed types of `E` and `F` differ, `partial_order(E, F)` is ill-formed.
- (3.2) — Otherwise, `partial_ordering(partial_order(E, F))` if it is a well-formed expression where the meaning of `partial_order` is established as-if by performing argument-dependent lookup only (6.5.4).
- (3.3) — Otherwise, `partial_ordering(compare_three_way()(E, F))` if it is a well-formed expression.
- (3.4) — Otherwise, `partial_ordering(weak_order(E, F))` if it is a well-formed expression.
- (3.5) — Otherwise, `partial_order(E, F)` is ill-formed.

[Note 3: Ill-formed cases above result in substitution failure when `partial_order(E, F)` appears in the immediate context of a template instantiation. — end note]

⁴ The name `compare_strong_order_fallback` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `compare_strong_order_fallback(E, F)` is expression-equivalent (3.22) to:

- (4.1) — If the decayed types of `E` and `F` differ, `compare_strong_order_fallback(E, F)` is ill-formed.
- (4.2) — Otherwise, `strong_order(E, F)` if it is a well-formed expression.
- (4.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and each of `decltype(E == F)` and `decltype(E < F)` models *boolean-testable*,


```

E == F ? strong_ordering::equal :
E < F  ? strong_ordering::less  :
        strong_ordering::greater

```

except that `E` and `F` are evaluated only once.
- (4.4) — Otherwise, `compare_strong_order_fallback(E, F)` is ill-formed.

[Note 4: Ill-formed cases above result in substitution failure when `compare_strong_order_fallback(E, F)` appears in the immediate context of a template instantiation. — end note]

⁵ The name `compare_weak_order_fallback` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `compare_weak_order_fallback(E, F)` is expression-equivalent (3.22) to:

- (5.1) — If the decayed types of `E` and `F` differ, `compare_weak_order_fallback(E, F)` is ill-formed.
- (5.2) — Otherwise, `weak_order(E, F)` if it is a well-formed expression.
- (5.3) — Otherwise, if the expressions `E == F` and `E < F` are both well-formed and each of `decltype(E == F)` and `decltype(E < F)` models *boolean-testable*,

`E == F ? weak_ordering::equivalent :`
`E < F ? weak_ordering::less :`
`weak_ordering::greater`

 except that `E` and `F` are evaluated only once.
- (5.4) — Otherwise, `compare_weak_order_fallback(E, F)` is ill-formed.

[Note 5: Ill-formed cases above result in substitution failure when `compare_weak_order_fallback(E, F)` appears in the immediate context of a template instantiation. — end note]

⁶ The name `compare_partial_order_fallback` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expression `compare_partial_order_fallback(E, F)` is expression-equivalent (3.22) to:

- (6.1) — If the decayed types of `E` and `F` differ, `compare_partial_order_fallback(E, F)` is ill-formed.
- (6.2) — Otherwise, `partial_order(E, F)` if it is a well-formed expression.
- (6.3) — Otherwise, if the expressions `E == F`, `E < F`, and `F < E` are all well-formed and each of `decltype(E == F)`, `decltype(E < F)`, and `decltype(F < E)` models *boolean-testable*,

`E == F ? partial_ordering::equivalent :`
`E < F ? partial_ordering::less :`
`F < E ? partial_ordering::greater :`
`partial_ordering::unordered`

 except that `E` and `F` are evaluated only once.
- (6.4) — Otherwise, `compare_partial_order_fallback(E, F)` is ill-formed.

[Note 6: Ill-formed cases above result in substitution failure when `compare_partial_order_fallback(E, F)` appears in the immediate context of a template instantiation. — end note]

17.12.7 Type Ordering

[compare.type]

¹ There is an implementation-defined total ordering of all types. For any (possibly incomplete) types `X` and `Y`, the expression `TYPE-ORDER(X, Y)` is a constant expression (7.7.3) of type `strong_ordering` (17.12.2.4). Its value is `strong_ordering::less` if `X` precedes `Y` in this implementation-defined total order, `strong_ordering::greater` if `Y` precedes `X`, and `strong_ordering::equal` if they are the same type.

[Note 1: `int`, `const int` and `int&` are different types. — end note]

[Note 2: This ordering need not be consistent with the one induced by `type_info::before`. — end note]

[Note 3: The ordering of TU-local types from different translation units is not observable, because the necessary specialization of `type_order` is impossible to name. — end note]

```
template<class T, class U> struct type_order {
    static constexpr strong_ordering value = TYPE-ORDER(T, U);

    using value_type = strong_ordering;

    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
};
```

- ² If an explicit specialization or partial specialization of `type_order` is declared, the program is ill-formed.
- ³ The templates `type_order` and `type_order_v` may be instantiated with incomplete types as arguments.
- ⁴ *Recommended practice:* The order should be lexicographical on parameter-type-lists and template argument lists.

17.13 Coroutines

[support.coroutine]

17.13.1 General

[support.coroutine.general]

- ¹ The header `<coroutine>` defines several types providing compile and run-time support for coroutines in a C++ program.

17.13.2 Header `<coroutine>` synopsis

[coroutine.syn]

```
// all freestanding
#include <compare>           // see 17.12.1

namespace std {
    // 17.13.3, coroutine traits
    template<class R, class... ArgTypes>
        struct coroutine_traits;

    // 17.13.4, coroutine handle
    template<class Promise = void>
        struct coroutine_handle;

    // 17.13.4.8, comparison operators
    constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
    constexpr strong_ordering operator<==(coroutine_handle<> x, coroutine_handle<> y) noexcept;

    // 17.13.4.9, hash support
    template<class T> struct hash;
    template<class P> struct hash<coroutine_handle<P>>;

    // 17.13.5, no-op coroutines
    struct noop_coroutine_promise;

    template<> struct coroutine_handle<noop_coroutine_promise>;
    using noop_coroutine_handle = coroutine_handle<noop_coroutine_promise>;

    noop_coroutine_handle noop_coroutine() noexcept;

    // 17.13.6, trivial awaitables
    struct suspend_never;
    struct suspend_always;
}
```

17.13.3 Coroutine traits

[coroutine.traits]

17.13.3.1 General

[coroutine.traits.general]

- ¹ Subclause 17.13.3 defines requirements on classes representing *coroutine traits*, and defines the class template `coroutine_traits` that meets those requirements.

17.13.3.2 Class template `coroutine_traits`

[coroutine.traits.primary]

- ¹ The header `<coroutine>` defines the primary template `coroutine_traits` such that if `ArgTypes` is a parameter pack of types and if the *qualified-id* `R::promise_type` is valid and denotes a type (13.10.3), then `coroutine_traits<R, ArgTypes...>` has the following publicly accessible member:

```
using promise_type = R::promise_type;
```

Otherwise, `coroutine_traits<R, ArgTypes...>` has no members.

- ² Program-defined specializations of this template shall define a publicly accessible nested type named `promise_type`.

17.13.4 Class template `coroutine_handle`

[coroutine.handle]

17.13.4.1 General

[coroutine.handle.general]

```
namespace std {
    template<>
        struct coroutine_handle<void> {
            // 17.13.4.2, construct/reset
        };
}
```



```

constexpr coroutine_handle() noexcept;
constexpr coroutine_handle(nullptr_t) noexcept;
coroutine_handle& operator=(nullptr_t) noexcept;

// 17.13.4.4, export/import
constexpr void* address() const noexcept;
static constexpr coroutine_handle from_address(void* addr);

// 17.13.4.5, observers
constexpr explicit operator bool() const noexcept;
bool done() const;

// 17.13.4.6, resumption
void operator()() const;
void resume() const;
void destroy() const;

private:
    void* ptr; // exposition only
};

template<class Promise>
struct coroutine_handle {
    // 17.13.4.2, construct/reset
    constexpr coroutine_handle() noexcept;
    constexpr coroutine_handle(nullptr_t) noexcept;
    static coroutine_handle from_promise(Promise&);
    coroutine_handle& operator=(nullptr_t) noexcept;

    // 17.13.4.4, export/import
    constexpr void* address() const noexcept;
    static constexpr coroutine_handle from_address(void* addr);

    // 17.13.4.3, conversion
    constexpr operator coroutine_handle<>() const noexcept;

    // 17.13.4.5, observers
    constexpr explicit operator bool() const noexcept;
    bool done() const;

    // 17.13.4.6, resumption
    void operator()() const;
    void resume() const;
    void destroy() const;

    // 17.13.4.7, promise access
    Promise& promise() const;

private:
    void* ptr; // exposition only
};
}

```

- ¹ An object of type `coroutine_handle<T>` is called a *coroutine handle* and can be used to refer to a suspended or executing coroutine. A `coroutine_handle` object whose member `address()` returns a null pointer value does not refer to any coroutine. Two `coroutine_handle` objects refer to the same coroutine if and only if their member `address()` returns the same non-null value.
- ² If a program declares an explicit or partial specialization of `coroutine_handle`, the behavior is undefined.

17.13.4.2 Construct/reset

[[coroutine.handle.con](#)]

```

constexpr coroutine_handle() noexcept;
constexpr coroutine_handle(nullptr_t) noexcept;

```

- ¹ *Postconditions:* `address() == nullptr`.

```
static coroutine_handle from_promise(Promise& p);
```

2 *Preconditions:* `p` is a reference to a promise object of a coroutine.

3 *Postconditions:* `addressof(h.promise()) == addressof(p)`.

4 *Returns:* A coroutine handle `h` referring to the coroutine.

```
coroutine_handle& operator=(nullptr_t) noexcept;
```

5 *Postconditions:* `address() == nullptr`.

6 *Returns:* `*this`.

17.13.4.3 Conversion

[coroutine.handle.conv]

```
constexpr operator coroutine_handle<>() const noexcept;
```

1 *Effects:* Equivalent to: `return coroutine_handle<>::from_address(address());`

17.13.4.4 Export/import

[coroutine.handle.export.import]

```
constexpr void* address() const noexcept;
```

1 *Returns:* `ptr`.

```
static constexpr coroutine_handle<> coroutine_handle<>::from_address(void* addr);
```

2 *Preconditions:* `addr` was obtained via a prior call to `address` on an object whose type is a specialization of `coroutine_handle`.

3 *Postconditions:* `from_address(address()) == *this`.

```
static constexpr coroutine_handle<Promise> coroutine_handle<Promise>::from_address(void* addr);
```

4 *Preconditions:* `addr` was obtained via a prior call to `address` on an object of type *cv* `coroutine_handle<Promise>`.

5 *Postconditions:* `from_address(address()) == *this`.

17.13.4.5 Observers

[coroutine.handle.observers]

```
constexpr explicit operator bool() const noexcept;
```

1 *Returns:* `address() != nullptr`.

```
bool done() const;
```

2 *Preconditions:* `*this` refers to a suspended coroutine.

3 *Returns:* `true` if the coroutine is suspended at its final suspend point, otherwise `false`.

17.13.4.6 Resumption

[coroutine.handle.resumption]

1 Resuming a coroutine via `resume`, `operator()`, or `destroy` on an execution agent other than the one on which it was suspended has implementation-defined behavior unless each execution agent either is an instance of `std::thread` or `std::jthread`, or is the thread that executes `main`.

[Note 1: A coroutine that is resumed on a different execution agent should avoid relying on consistent thread identity throughout, such as holding a mutex object across a suspend point. — end note]

[Note 2: A concurrent resumption of the coroutine can result in a data race. — end note]

```
void operator()() const;
```

```
void resume() const;
```

2 *Preconditions:* `*this` refers to a suspended coroutine. The coroutine is not suspended at its final suspend point.

3 *Effects:* Resumes the execution of the coroutine.

```
void destroy() const;
```

4 *Preconditions:* `*this` refers to a suspended coroutine.

5 *Effects:* Destroys the coroutine (9.6.4).

17.13.4.7 Promise access

[coroutine.handle.promise]

```
Promise& promise() const;
```

1 *Preconditions:* *this refers to a coroutine.

2 *Returns:* A reference to the promise of the coroutine.

17.13.4.8 Comparison operators

[coroutine.handle.compare]

```
constexpr bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

1 *Returns:* x.address() == y.address().

```
constexpr strong_ordering operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
```

2 *Returns:* compare_three_way()(x.address(), y.address()).

17.13.4.9 Hash support

[coroutine.handle.hash]

```
template<class P> struct hash<coroutine_handle<P>>;
```

1 The specialization is enabled (22.10.19).

17.13.5 No-op coroutines

[coroutine.noop]

17.13.5.1 Class noop_coroutine_promise

[coroutine.promise.noop]

```
struct noop_coroutine_promise {};
```

1 The class `noop_coroutine_promise` defines the promise type for the coroutine referred to by `noop_coroutine_handle` (17.13.2).

17.13.5.2 Class coroutine_handle<noop_coroutine_promise>

[coroutine.handle.noop]

17.13.5.2.1 General

[coroutine.handle.noop.general]

```
namespace std {
    template<>
    struct coroutine_handle<noop_coroutine_promise> {
        // 17.13.5.2.2, conversion
        constexpr operator coroutine_handle<>() const noexcept;

        // 17.13.5.2.3, observers
        constexpr explicit operator bool() const noexcept;
        constexpr bool done() const noexcept;

        // 17.13.5.2.4, resumption
        constexpr void operator()() const noexcept;
        constexpr void resume() const noexcept;
        constexpr void destroy() const noexcept;

        // 17.13.5.2.5, promise access
        noop_coroutine_promise& promise() const noexcept;

        // 17.13.5.2.6, address
        constexpr void* address() const noexcept;
    private:
        coroutine_handle(unspecified);
        void* ptr; // exposition only
    };
}
```

17.13.5.2.2 Conversion

[coroutine.handle.noop.conv]

```
constexpr operator coroutine_handle<>() const noexcept;
```

1 *Effects:* Equivalent to: `return coroutine_handle<>::from_address(address());`

17.13.5.2.3 Observers**[coroutine.handle.noop.observers]**

constexpr explicit operator bool() const noexcept;

¹ *Returns:* true.

constexpr bool done() const noexcept;

² *Returns:* false.**17.13.5.2.4 Resumption****[coroutine.handle.noop.resumption]**

constexpr void operator()() const noexcept;

constexpr void resume() const noexcept;

constexpr void destroy() const noexcept;

¹ *Effects:* None.² *Remarks:* If `noop_coroutine_handle` is converted to `coroutine_handle<>`, calls to `operator()`, `resume` and `destroy` on that handle will also have no observable effects.**17.13.5.2.5 Promise access****[coroutine.handle.noop.promise]**

noop_coroutine_promise& promise() const noexcept;

¹ *Returns:* A reference to the promise object associated with this coroutine handle.**17.13.5.2.6 Address****[coroutine.handle.noop.address]**

constexpr void* address() const noexcept;

¹ *Returns:* ptr.² *Remarks:* A `noop_coroutine_handle`'s ptr is always a non-null pointer value.**17.13.5.3 Function noop_coroutine****[coroutine.noop.coroutine]**

noop_coroutine_handle noop_coroutine() noexcept;

¹ *Returns:* A handle to a coroutine that has no observable effects when resumed or destroyed.² *Remarks:* A handle returned from `noop_coroutine` may or may not compare equal to a handle returned from another invocation of `noop_coroutine`.**17.13.6 Trivial awaitables****[coroutine.trivial.awaitables]**

```

namespace std {
    struct suspend_never {
        constexpr bool await_ready() const noexcept { return true; }
        constexpr void await_suspend(coroutine_handle<>) const noexcept {}
        constexpr void await_resume() const noexcept {}
    };
    struct suspend_always {
        constexpr bool await_ready() const noexcept { return false; }
        constexpr void await_suspend(coroutine_handle<>) const noexcept {}
        constexpr void await_resume() const noexcept {}
    };
}

```

¹ [Note 1: The types `suspend_never` and `suspend_always` can be used to indicate that an *await-expression* either never suspends or always suspends, and in either case does not produce a value. — end note]**17.14 Other runtime support****[support.runtime]****17.14.1 General****[support.runtime.general]**¹ Headers `<csetjmp>` (nonlocal jumps), `<csignal>` (signal handling), `<cstdarg>` (variable arguments), and `<cstdliblib>` (runtime environment `getenv`, `system`), provide further compatibility with C code.² Calls to the function `getenv` (17.2.2) shall not introduce a data race (16.4.6.10) provided that nothing modifies the environment.[Note 1: Calls to the POSIX functions `setenv` and `putenv` modify the environment. — end note]

- ³ A call to the `setlocale` function (28.3.5) may introduce a data race with other calls to the `setlocale` function or with calls to functions that are affected by the current C locale. The implementation shall behave as if no library function other than `locale::global` calls the `setlocale` function.

17.14.2 Header `<cstdarg>` synopsis

[cstdarg.syn]

```
// all freestanding
#define __STDC_VERSION_STDARG_H__ 202311L

namespace std {
    using va_list = see below;
}

#define va_arg(V, P) see below
#define va_copy(VDST, VSRC) see below
#define va_end(V) see below
#define va_start(V, ...) see below
```

- ¹ The contents of the header `<cstdarg>` are the same as the C standard library header `<stdarg.h>`, with the following changes:
- (1.1) — In lieu of the default argument promotions specified in ISO/IEC 9899:2024 6.5.3.3, the definition in 7.6.1.3 applies.
 - (1.2) — If more than one argument is present for `va_start` and any of the second or subsequent arguments expands to include unbalanced parentheses, or a preprocessing token that does not convert to a token, the program is ill-formed, no diagnostic required. The preprocessing tokens comprising the second and subsequent arguments to `va_start` (if any) are discarded.

[Note 1: `va_start` accepts a second argument for compatibility with prior revisions of C++. — end note]

SEE ALSO: ISO/IEC 9899:2024, 7.16

17.14.3 Header `<csetjmp>` synopsis

[csetjmp.syn]

```
#define __STDC_VERSION_SETJMP_H__ 202311L

namespace std {
    using jmp_buf = see below;
    [[noreturn]] void longjmp(jmp_buf env, int val);
}

#define setjmp(env) see below
```

- ¹ The contents of the header `<csetjmp>` are the same as the C standard library header `<setjmp.h>`.
- ² The function signature `longjmp(jmp_buf jbuf, int val)` has more restricted behavior in this document. A `setjmp/longjmp` call pair has undefined behavior if replacing the `setjmp` and `longjmp` by `catch` and `throw` would invoke any non-trivial destructors for any objects with automatic storage duration. A call to `setjmp` or `longjmp` has undefined behavior if invoked in a suspension context of a coroutine (7.6.2.4).

SEE ALSO: ISO/IEC 9899:2024, 7.13

17.14.4 Header `<csignal>` synopsis

[csignal.syn]

```
namespace std {
    using sig_atomic_t = see below;

    // 17.14.5, signal handlers
    extern "C" using signal_handler = void(int); // exposition only
    signal_handler* signal(int sig, signal_handler* func);

    int raise(int sig);
}
```

```

#define SIG_DFL see below
#define SIG_ERR see below
#define SIG_IGN see below
#define SIGABRT see below
#define SIGFPE see below
#define SIGILL see below
#define SIGINT see below
#define SIGSEGV see below
#define SIGTERM see below

```

- ¹ The contents of the header `<csignal>` are the same as the C standard library header `<signal.h>`.

17.14.5 Signal handlers

[support.signal]

- ¹ A call to the function `signal` synchronizes with any resulting invocation of the signal handler so installed.

- ² A *plain lock-free atomic operation* is an invocation of a function `f` from 32.5, such that:

- (2.1) — `f` is the function `atomic_is_lock_free()`, or
- (2.2) — `f` is the member function `is_lock_free()`, or
- (2.3) — `f` is a non-static member function of class `atomic_flag`, or
- (2.4) — `f` is a non-member function, and the first parameter of `f` has type *cv* `atomic_flag*`, or
- (2.5) — `f` is a non-static member function invoked on an object `A`, such that `A.is_lock_free()` yields `true`, or
- (2.6) — `f` is a non-member function, and for every pointer-to-atomic argument `A` passed to `f`, `atomic_is_lock_free(A)` yields `true`.

- ³ An evaluation is *signal-safe* unless it includes one of the following:

- (3.1) — a call to any standard library function, except for plain lock-free atomic operations and functions explicitly identified as signal-safe;
 [Note 1: This implicitly excludes the use of `new` and `delete` expressions that rely on a library-provided memory allocator. — end note]
- (3.2) — an access to an object with thread storage duration;
- (3.3) — a `dynamic_cast` expression;
- (3.4) — throwing of an exception;
- (3.5) — control entering a *try-block* or *function-try-block*;
- (3.6) — initialization of a variable with static storage duration requiring dynamic initialization (6.10.3.3, 8.10)¹⁷⁰;
 ; or
- (3.7) — waiting for the completion of the initialization of a variable with static storage duration (8.10).

A signal handler invocation has undefined behavior if it includes an evaluation that is not signal-safe.

- ⁴ The function `signal` is signal-safe if it is invoked with the first argument equal to the signal number corresponding to the signal that caused the invocation of the handler.

SEE ALSO: ISO/IEC 9899:2024, 7.14

17.15 C headers

[support.c.headers]

17.15.1 General

[support.c.headers.general]

- ¹ For compatibility with the C standard library, the C++ standard library provides the *C headers* shown in Table 47. The intended use of these headers is for interoperability only. It is possible that C++ source files need to include one of these headers in order to be valid C. Source files that are not intended to also be valid C should not use any of the C headers.

[Note 1: The C headers either have no effect, such as `<stdbool.h>` (17.15.5) and `<stdalign.h>` (17.15.4), or otherwise the corresponding header of the form `<cname>` provides the same facilities and assuredly defines them in namespace `std`. — end note]

¹⁷⁰ Such initialization can occur because it is the first odr-use (6.3) of that variable.

[*Example 1*: The following source file is both valid C++ and valid C. Viewed as C++, it declares a function with C language linkage; viewed as C it simply declares a function (and provides a prototype).

```
#include <uchar.h>      // for char8_t in C, not necessary in C++
#include <stddef.h>      // for size_t

#ifdef __cplusplus      // see 15.12
extern "C"              // see 9.12
#endif
void f(char8_t s[], size_t n);

— end example]
```

Table 47 — C headers [tab:c.headers]

<assert.h>	<inttypes.h>	<signal.h>	<stdckdint.h>	<tgmath.h>
<complex.h>	<iso646.h>	<stdalign.h>	<stddef.h>	<time.h>
<ctype.h>	<limits.h>	<stdarg.h>	<stdint.h>	<uchar.h>
<errno.h>	<locale.h>	<stdatomic.h>	<stdio.h>	<wchar.h>
<fenv.h>	<math.h>	<stdbit.h>	<stdlib.h>	<wctype.h>
<float.h>	<setjmp.h>	<stdbool.h>	<string.h>	

17.15.2 Header <complex.h> synopsis

[complex.h.syn]

```
#include <complex>
```

- The header <complex.h> behaves as if it simply includes the header <complex> (29.4.2).
- [*Note 1*: Names introduced by <complex> in namespace `std` are not placed into the global namespace scope by <complex.h>. — end note]

17.15.3 Header <iso646.h> synopsis

[iso646.h.syn]

- The C++ header <iso646.h> is empty.

[*Note 1*: `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in C++ (5.12). — end note]

17.15.4 Header <stdalign.h> synopsis

[stdalign.h.syn]

- The contents of the C++ header <stdalign.h> are the same as the C standard library header <stdalign.h>. SEE ALSO: ISO/IEC 9899:2024, 7.15

17.15.5 Header <stdbool.h> synopsis

[stdbool.h.syn]

- The contents of the C++ header <stdbool.h> are the same as the C standard library header <stdbool.h>. SEE ALSO: ISO/IEC 9899:2024, 7.19

17.15.6 Header <tgmath.h> synopsis

[tgmath.h.syn]

```
#include <cmath>
#include <complex>
```

- The header <tgmath.h> behaves as if it simply includes the headers <cmath> (29.7.1) and <complex> (29.4.2).
- [*Note 1*: The overloads provided in C by type-generic macros are already provided in <complex> and <cmath> by “sufficient” additional overloads. — end note]
- [*Note 2*: Names introduced by <cmath> or <complex> in namespace `std` are not placed into the global namespace scope by <tgmath.h>. — end note]

17.15.7 Other C headers

[support.c.headers.other]

- Every C header other than <complex.h> (17.15.2), <iso646.h> (17.15.3), <stdalign.h> (17.15.4), <stdatomic.h> (32.5.12), <stdbit.h> (22.12), <stdbool.h> (17.15.5), <stdckdint.h> (29.11.1), and <tgmath.h> (17.15.6), each of which has a name of the form <name.h>, behaves as if each name placed in the standard library namespace by the corresponding <cname> header is placed within the global namespace scope, except for the functions described in 29.7.6, the `std::lerp` function overloads (29.7.4), the declaration

of `std::byte` (17.2.1), and the functions and function templates described in 17.2.5. It is unspecified whether these names are first declared or defined within namespace scope (6.4.6) of the namespace `std` and are then injected into the global namespace scope by explicit *using-declarations* (9.10).

- ² [*Example 1*: The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in ISO/IEC 9899. It may also provide these names within the namespace `std`. — *end example*]

18 Concepts library

[concepts]

18.1 General

[concepts.general]

- ¹ This Clause describes library components that C++ programs may use to perform compile-time validation of template arguments and perform function dispatch based on properties of types. The purpose of these concepts is to establish a foundation for equational reasoning in programs.
- ² The following subclauses describe language-related concepts, comparison concepts, object concepts, and callable concepts as summarized in [Table 48](#).

Table 48 — Fundamental concepts library summary [tab:concepts.summary]

Subclause	Header
18.2 Equality preservation	
18.4 Language-related concepts	<concepts>
18.5 Comparison concepts	
18.6 Object concepts	
18.7 Callable concepts	

18.2 Equality preservation

[concepts.equality]

- ¹ An expression is *equality-preserving* if, given equal inputs, the expression results in equal outputs. The inputs to an expression are the set of the expression's operands. The output of an expression is the expression's result and all operands modified by the expression. For the purposes of this subclause, the operands of an expression are the largest subexpressions that include only:
 - (1.1) — an *id-expression* ([7.5.5](#)), and
 - (1.2) — invocations of the library function templates `std::move`, `std::forward`, and `std::declval` ([22.2.4](#), [22.2.6](#)).

[*Example 1*: The operands of the expression `a = std::move(b)` are `a` and `std::move(b)`. — *end example*]
- ² Not all input values need be valid for a given expression.

[*Example 2*: For integers `a` and `b`, the expression `a / b` is not well-defined when `b` is 0. This does not preclude the expression `a / b` being equality-preserving. — *end example*]

The *domain* of an expression is the set of input values for which the expression is required to be well-defined.
- ³ Expressions required to be equality-preserving are further required to be stable: two evaluations of such an expression with the same input objects are required to have equal outputs absent any explicit intervening modification of those input objects.

[*Note 1*: This requirement allows generic code to reason about the current values of objects based on knowledge of the prior values as observed via equality-preserving expressions. It effectively forbids spontaneous changes to an object, changes to an object from another thread of execution, changes to an object as side effects of non-modifying expressions, and changes to an object as side effects of modifying a distinct object if those changes could be observable to a library function via an equality-preserving expression that is required to be valid for that object. — *end note*]
- ⁴ Expressions declared in a *requires-expression* in the library clauses are required to be equality-preserving, except for those annotated with the comment “not required to be equality-preserving.” An expression so annotated may be equality-preserving, but is not required to be so.
- ⁵ An expression that may alter the value of one or more of its inputs in a manner observable to equality-preserving expressions is said to modify those inputs. The library clauses use a notational convention to specify which expressions declared in a *requires-expression* modify which inputs: except where otherwise specified, an expression operand that is a non-constant lvalue or rvalue may be modified. Operands that are constant lvalues or rvalues are required to not be modified. For the purposes of this subclause, the cv-qualification and value category of each operand are determined by assuming that each template type parameter denotes a cv-unqualified complete non-array object type.

- ⁶ Where a *requires-expression* declares an expression that is non-modifying for some constant lvalue operand, additional variations of that expression that accept a non-constant lvalue or (possibly constant) rvalue for the given operand are also required except where such an expression variation is explicitly required with differing semantics. These *implicit expression variations* are required to meet the semantic requirements of the declared expression. The extent to which an implementation validates the syntax of the variations is unspecified.

⁷ [Example 3:

```
template<class T> concept C = requires(T a, T b, const T c, const T d) {
    c == d;           // #1
    a = std::move(b); // #2
    a = c;            // #3
};
```

For the above example:

- (7.1) — Expression #1 does not modify either of its operands, #2 modifies both of its operands, and #3 modifies only its first operand a.
- (7.2) — Expression #1 implicitly requires additional expression variations that meet the requirements for `c == d` (including non-modification), as if the expressions

	<code>c == b;</code>
<code>c == std::move(d);</code>	<code>c == std::move(b);</code>
<code>std::move(c) == d;</code>	<code>std::move(c) == b;</code>
<code>std::move(c) == std::move(d);</code>	<code>std::move(c) == std::move(b);</code>
<code>a == d;</code>	<code>a == b;</code>
<code>a == std::move(d);</code>	<code>a == std::move(b);</code>
<code>std::move(a) == d;</code>	<code>std::move(a) == b;</code>
<code>std::move(a) == std::move(d);</code>	<code>std::move(a) == std::move(b);</code>

had been declared as well.

- (7.3) — Expression #3 implicitly requires additional expression variations that meet the requirements for `a = c` (including non-modification of the second operand), as if the expressions `a = b` and `a = std::move(c)` had been declared. Expression #3 does not implicitly require an expression variation with a non-constant rvalue second operand, since expression #2 already specifies exactly such an expression explicitly.

— end example]

- ⁸ [Example 4: The following type T meets the explicitly stated syntactic requirements of concept C above but does not meet the additional implicit requirements:

```
struct T {
    bool operator==(const T&) const { return true; }
    bool operator==(T&) = delete;
};
```

T fails to meet the implicit requirements of C, so T satisfies but does not model C. Since implementations are not required to validate the syntax of implicit requirements, it is unspecified whether an implementation diagnoses as ill-formed a program that requires `C<T>`. — end example]

18.3 Header <concepts> synopsis

[concepts.syn]

```
// all freestanding
namespace std {
    // 18.4, language-related concepts
    // 18.4.2, concept same_as
    template<class T, class U>
        concept same_as = see below;

    // 18.4.3, concept derived_from
    template<class Derived, class Base>
        concept derived_from = see below;

    // 18.4.4, concept convertible_to
    template<class From, class To>
        concept convertible_to = see below;
```

```

// 18.4.5, concept common_reference_with
template<class T, class U>
    concept common_reference_with = see below;

// 18.4.6, concept common_with
template<class T, class U>
    concept common_with = see below;

// 18.4.7, arithmetic concepts
template<class T>
    concept integral = see below;
template<class T>
    concept signed_integral = see below;
template<class T>
    concept unsigned_integral = see below;
template<class T>
    concept floating_point = see below;

// 18.4.8, concept assignable_from
template<class LHS, class RHS>
    concept assignable_from = see below;

// 18.4.9, concept swappable
namespace ranges {
    inline namespace unspecified {
        inline constexpr unspecified swap = unspecified;
    }
}
template<class T>
    concept swappable = see below;
template<class T, class U>
    concept swappable_with = see below;

// 18.4.10, concept destructible
template<class T>
    concept destructible = see below;

// 18.4.11, concept constructible_from
template<class T, class... Args>
    concept constructible_from = see below;

// 18.4.12, concept default_initializable
template<class T>
    concept default_initializable = see below;

// 18.4.13, concept move_constructible
template<class T>
    concept move_constructible = see below;

// 18.4.14, concept copy_constructible
template<class T>
    concept copy_constructible = see below;

// 18.5, comparison concepts
// 18.5.4, concept equality_comparable
template<class T>
    concept equality_comparable = see below;
template<class T, class U>
    concept equality_comparable_with = see below;

// 18.5.5, concept totally_ordered
template<class T>
    concept totally_ordered = see below;

```

```

template<class T, class U>
    concept totally_ordered_with = see below;

// 18.6, object concepts
template<class T>
    concept movable = see below;
template<class T>
    concept copyable = see below;
template<class T>
    concept semiregular = see below;
template<class T>
    concept regular = see below;

// 18.7, callable concepts
// 18.7.2, concept invocable
template<class F, class... Args>
    concept invocable = see below;

// 18.7.3, concept regular_invocable
template<class F, class... Args>
    concept regular_invocable = see below;

// 18.7.4, concept predicate
template<class F, class... Args>
    concept predicate = see below;

// 18.7.5, concept relation
template<class R, class T, class U>
    concept relation = see below;

// 18.7.6, concept equivalence_relation
template<class R, class T, class U>
    concept equivalence_relation = see below;

// 18.7.7, concept strict_weak_order
template<class R, class T, class U>
    concept strict_weak_order = see below;
}

```

18.4 Language-related concepts

[concepts.lang]

18.4.1 General

[concepts.lang.general]

- ¹ Subclause 18.4 contains the definition of concepts corresponding to language features. These concepts express relationships between types, type classifications, and fundamental type properties.

18.4.2 Concept same_as

[concept.same]

```

template<class T, class U>
    concept same_as_impl = is_same_v<T, U>;           // exposition only

template<class T, class U>
    concept same_as = same_as_impl<T, U> && same_as_impl<U, T>;

```

- ¹ [Note 1: same_as<T, U> subsumes same_as<U, T> and vice versa. — end note]

18.4.3 Concept derived_from

[concept.derived]

```

template<class Derived, class Base>
    concept derived_from =
        is_base_of_v<Base, Derived> &&
        is_convertible_v<const volatile Derived*, const volatile Base*>;

```

- ¹ [Note 1: derived_from<Derived, Base> is satisfied if and only if Derived is publicly and unambiguously derived from Base, or Derived and Base are the same class type ignoring cv-qualifiers. — end note]

18.4.4 Concept convertible_to

[concept.convertible]

- ¹ Given types `From` and `To` and an expression `E` whose type and value category are the same as those of `declval<From>()`, `convertible_to<From, To>` requires `E` to be both implicitly and explicitly convertible to type `To`. The implicit and explicit conversions are required to produce equal results.

```
template<class From, class To>
concept convertible_to =
    is_convertible_v<From, To> &&
    requires {
        static_cast<To>(declval<From>());
    };

```

- ² Let `FromR` be `add_rvalue_reference_t<From>` and `test` be the invented function:

```
To test(FromR (&f)()) {
    return f();
}

```

and let `f` be a function with no arguments and return type `FromR` such that `f()` is equality-preserving. Types `From` and `To` model `convertible_to<From, To>` only if:

- (2.1) — `To` is not an object or reference-to-object type, or `static_cast<To>(f())` is equal to `test(f)`.
- (2.2) — `FromR` is not a reference-to-object type, or
 - (2.2.1) — If `FromR` is an rvalue reference to a non const-qualified type, the resulting state of the object referenced by `f()` after either above expression is valid but unspecified (16.4.6.17).
 - (2.2.2) — Otherwise, the object referred to by `f()` is not modified by either above expression.

18.4.5 Concept common_reference_with

[concept.commonref]

- ¹ For two types `T` and `U`, if `common_reference_t<T, U>` is well-formed and denotes a type `C` such that both `convertible_to<T, C>` and `convertible_to<U, C>` are modeled, then `T` and `U` share a *common reference type*, `C`.

[Note 1: `C` can be the same as `T` or `U`, or can be a different type. `C` can be a reference type. — end note]

```
template<class T, class U>
concept common_reference_with =
    same_as<common_reference_t<T, U>, common_reference_t<U, T>> &&
    convertible_to<T, common_reference_t<T, U>> &&
    convertible_to<U, common_reference_t<T, U>>;

```

- ² Let `C` be `common_reference_t<T, U>`. Let `t1` and `t2` be equality-preserving expressions (18.2) such that `decltype((t1))` and `decltype((t2))` are each `T`, and let `u1` and `u2` be equality-preserving expressions such that `decltype((u1))` and `decltype((u2))` are each `U`. `T` and `U` model `common_reference_with<T, U>` only if

- (2.1) — `C(t1)` equals `C(t2)` if and only if `t1` equals `t2`, and
- (2.2) — `C(u1)` equals `C(u2)` if and only if `u1` equals `u2`.

- ³ [Note 2: Users can customize the behavior of `common_reference_with` by specializing the `basic_common_reference` class template (21.3.8.7). — end note]

18.4.6 Concept common_with

[concept.common]

- ¹ If `T` and `U` can both be explicitly converted to some third type, `C`, then `T` and `U` share a *common type*, `C`.

[Note 1: `C` can be the same as `T` or `U`, or can be a different type. `C` is not necessarily unique. — end note]

```
template<class T, class U>
concept common_with =
    same_as<common_type_t<T, U>, common_type_t<U, T>> &&
    requires {
        static_cast<common_type_t<T, U>>(declval<T>());
        static_cast<common_type_t<T, U>>(declval<U>());
    } &&

```

```

common_reference_with<
    add_lvalue_reference_t<const T>,
    add_lvalue_reference_t<const U>> &&
common_reference_with<
    add_lvalue_reference_t<common_type_t<T, U>>,
    common_reference_t<
        add_lvalue_reference_t<const T>,
        add_lvalue_reference_t<const U>>>;

```

2 Let `C` be `common_type_t<T, U>`. Let `t1` and `t2` be equality-preserving expressions (18.2) such that `decltype((t1))` and `decltype((t2))` are each `T`, and let `u1` and `u2` be equality-preserving expressions such that `decltype((u1))` and `decltype((u2))` are each `U`. `T` and `U` model `common_with<T, U>` only if

(2.1) — `C(t1)` equals `C(t2)` if and only if `t1` equals `t2`, and

(2.2) — `C(u1)` equals `C(u2)` if and only if `u1` equals `u2`.

3 [Note 2: Users can customize the behavior of `common_with` by specializing the `common_type` class template (21.3.8.7). — end note]

18.4.7 Arithmetic concepts

[concepts.arithmetic]

```

template<class T>
    concept integral = is_integral_v<T>;
template<class T>
    concept signed_integral = integral<T> && is_signed_v<T>;
template<class T>
    concept unsigned_integral = integral<T> && !signed_integral<T>;
template<class T>
    concept floating_point = is_floating_point_v<T>;

```

1 [Note 1: `signed_integral` can be modeled even by types that are not signed integer types (6.9.3); for example, `char`. — end note]

2 [Note 2: `unsigned_integral` can be modeled even by types that are not unsigned integer types (6.9.3); for example, `bool`. — end note]

18.4.8 Concept assignable_from

[concept.assignable]

```

template<class LHS, class RHS>
    concept assignable_from =
        is_lvalue_reference_v<LHS> &&
        common_reference_with<const remove_reference_t<LHS>&, const remove_reference_t<RHS>&> &&
        requires(LHS lhs, RHS&& rhs) {
            { lhs = std::forward<RHS>(rhs) } -> same_as<LHS>;
        };

```

1 Let:

(1.1) — `lhs` be an lvalue that refers to an object `lcopy` such that `decltype((lhs))` is `LHS`,

(1.2) — `rhs` be an expression such that `decltype((rhs))` is `RHS`, and

(1.3) — `rcopy` be a distinct object that is equal to `rhs`.

`LHS` and `RHS` model `assignable_from<LHS, RHS>` only if

(1.4) — `addressof(lhs = rhs) == addressof(lcopy)`.

(1.5) — After evaluating `lhs = rhs`:

(1.5.1) — `lhs` is equal to `rcopy`, unless `rhs` is a non-const xvalue that refers to `lcopy`.

(1.5.2) — If `rhs` is a non-const xvalue, the resulting state of the object to which it refers is valid but unspecified (16.4.6.17).

(1.5.3) — Otherwise, if `rhs` is a glvalue, the object to which it refers is not modified.

2 [Note 1: Assignment need not be a total function (16.3.2.3); in particular, if assignment to an object `x` can result in a modification of some other object `y`, then `x = y` is likely not in the domain of `=`. — end note]

18.4.9 Concept swappable

[concept.swappable]

- ¹ Let `t1` and `t2` be equality-preserving expressions that denote distinct equal objects of type `T`, and let `u1` and `u2` similarly denote distinct equal objects of type `U`.

[*Note 1: `t1` and `u1` can denote distinct objects, or the same object. — end note*]

An operation *exchanges the values* denoted by `t1` and `u1` if and only if the operation modifies neither `t2` nor `u2` and:

- (1.1) — If `T` and `U` are the same type, the result of the operation is that `t1` equals `u2` and `u1` equals `t2`.
- (1.2) — If `T` and `U` are different types and `common_reference_with<decltype((t1)), decltype((u1))>` is modeled, the result of the operation is that `C(t1)` equals `C(u2)` and `C(u1)` equals `C(t2)` where `C` is `common_reference_t<decltype((t1)), decltype((u1))>`.

- ² The name `ranges::swap` denotes a customization point object (16.3.3.3.5). The expression `ranges::swap(E1, E2)` for subexpressions `E1` and `E2` is expression-equivalent to an expression `S` determined as follows:

- (2.1) — `S` is `(void)swap(E1, E2)`¹⁷¹ if `E1` or `E2` has class or enumeration type (6.9.5) and that expression is valid, with overload resolution performed in a context that includes the declaration

```
template<class T>
    void swap(T&, T&) = delete;
```

and does not include a declaration of `ranges::swap`. If the function selected by overload resolution does not exchange the values denoted by `E1` and `E2`, the program is ill-formed, no diagnostic required.

[*Note 2: This precludes calling unconstrained program-defined overloads of `swap`. When the deleted overload is viable, program-defined overloads need to be more specialized (13.7.7.3) to be selected. — end note*]

- (2.2) — Otherwise, if `E1` and `E2` are lvalues of array types (6.9.5) with equal extent and `ranges::swap(*E1, *E2)` is a valid expression, `S` is `(void)ranges::swap_ranges(E1, E2)`, except that `noexcept(S)` is equal to `noexcept(ranges::swap(*E1, *E2))`.
- (2.3) — Otherwise, if `E1` and `E2` are lvalues of the same type `T` that models `move_constructible<T>` and `assignable_from<T&, T>`, `S` is an expression that exchanges the denoted values. `S` is a constant expression if
 - (2.3.1) — `T` is a literal type (6.9.1),
 - (2.3.2) — both `E1 = std::move(E2)` and `E2 = std::move(E1)` are constant subexpressions (3.15), and
 - (2.3.3) — the full-expressions of the initializers in the declarations

```
T t1(std::move(E1));
T t2(std::move(E2));
```

are constant subexpressions.

`noexcept(S)` is equal to `is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>`.

- (2.4) — Otherwise, `ranges::swap(E1, E2)` is ill-formed.

[*Note 3: This case can result in substitution failure when `ranges::swap(E1, E2)` appears in the immediate context of a template instantiation. — end note*]

- ³ [*Note 4: Whenever `ranges::swap(E1, E2)` is a valid expression, it exchanges the values denoted by `E1` and `E2` and has type `void`. — end note*]

```
template<class T>
    concept swappable = requires(T& a, T& b) { ranges::swap(a, b); };
```

```
template<class T, class U>
    concept swappable_with =
        common_reference_with<T, U> &&
        requires(T&& t, U&& u) {
            ranges::swap(std::forward<T>(t), std::forward<T>(t));
            ranges::swap(std::forward<U>(u), std::forward<U>(u));
            ranges::swap(std::forward<T>(t), std::forward<U>(u));
            ranges::swap(std::forward<U>(u), std::forward<T>(t));
        };
```

¹⁷¹) The name `swap` is used here unqualified.

⁴ [Note 5: The semantics of the `swappable` and `swappable_with` concepts are fully defined by the `ranges::swap` customization point object. — end note]

⁵ [Example 1: User code can ensure that the evaluation of `swap` calls is performed in an appropriate context under the various conditions as follows:

```
#include <cassert>
#include <concepts>
#include <utility>

namespace ranges = std::ranges;

template<class T, std::swappable_with<T> U>
void value_swap(T&& t, U&& u) {
    ranges::swap(std::forward<T>(t), std::forward<U>(u));
}

template<std::swappable T>
void lv_swap(T& t1, T& t2) {
    ranges::swap(t1, t2);
}

namespace N {
    struct A { int m; };
    struct Proxy {
        A* a;
        Proxy(A& a) : a{&a} {}
        friend void swap(Proxy x, Proxy y) {
            ranges::swap(*x.a, *y.a);
        }
    };
    Proxy proxy(A& a) { return Proxy{ a }; }
}

int main() {
    int i = 1, j = 2;
    lv_swap(i, j);
    assert(i == 2 && j == 1);

    N::A a1 = { 5 }, a2 = { -5 };
    value_swap(a1, proxy(a2));
    assert(a1.m == -5 && a2.m == 5);
}
```

— end example]

18.4.10 Concept `destructible`

[concept.destructible]

¹ The `destructible` concept specifies properties of all types, instances of which can be destroyed at the end of their lifetime, or reference types.

```
template<class T>
concept destructible = is_nothrow_destructible_v<T>;
```

² [Note 1: Unlike the *Cpp17Destructible* requirements (Table 35), this concept forbids destructors that are potentially throwing, even if a particular invocation of the destructor does not actually throw. — end note]

18.4.11 Concept `constructible_from`

[concept.constructible]

¹ The `constructible_from` concept constrains the initialization of a variable of a given type with a particular set of argument types.

```
template<class T, class... Args>
concept constructible_from = destructible<T> && is_constructible_v<T, Args...>;
```


18.4.12 Concept `default_initializable`**[concept.default.init]**

```
template<class T>
constexpr bool is-default-initializable = see below;           // exposition only
```

```
template<class T>
concept default_initializable = constructible_from<T> &&
                                requires { T{}; } &&
                                is-default-initializable<T>;
```

- ¹ For a type `T`, *is-default-initializable*<`T`> is true if and only if the variable definition
- ```
T t;
```
- is well-formed for some invented variable `t`; otherwise it is **false**. Access checking is performed as if in a context unrelated to `T`. Only the validity of the immediate context of the variable initialization is considered.

**18.4.13 Concept `move_constructible`****[concept.moveconstructible]**

```
template<class T>
concept move_constructible = constructible_from<T, T> && convertible_to<T, T>;
```

- <sup>1</sup> If `T` is an object type, then let `rv` be an rvalue of type `T` and `u2` a distinct object of type `T` equal to `rv`. `T` models *move\_constructible* only if
- (1.1) — After the definition `T u = rv;`, `u` is equal to `u2`.
  - (1.2) — `T(rv)` is equal to `u2`.
  - (1.3) — If `T` is not **const**, `rv`'s resulting state is valid but unspecified (16.4.6.17); otherwise, it is unchanged.

**18.4.14 Concept `copy_constructible`****[concept.copyconstructible]**

```
template<class T>
concept copy_constructible =
 move_constructible<T> &&
 constructible_from<T, T&> && convertible_to<T&, T> &&
 constructible_from<T, const T&> && convertible_to<const T&, T> &&
 constructible_from<T, const T> && convertible_to<const T, T>;
```

- <sup>1</sup> If `T` is an object type, then let `v` be an lvalue of type `T` or **const** `T` or an rvalue of type **const** `T`. `T` models *copy\_constructible* only if
- (1.1) — After the definition `T u = v;`, `u` is equal to `v` (18.2) and `v` is not modified.
  - (1.2) — `T(v)` is equal to `v` and does not modify `v`.

**18.5 Comparison concepts****[concepts.compare]****18.5.1 General****[concepts.compare.general]**

- <sup>1</sup> Subclause 18.5 describes concepts that establish relationships and orderings on values of possibly differing object types.
- <sup>2</sup> Given an expression `E` and a type `C`, let *CONVERT\_TO\_LVALUE*<`C`>(`E`) be:
- (2.1) — `static_cast<const C&>(as_const(E))` if that is a valid expression, and
  - (2.2) — `static_cast<const C&>(std::move(E))` otherwise.

**18.5.2 Boolean testability****[concept.booleantestable]**

- <sup>1</sup> The exposition-only *boolean-testable* concept specifies the requirements on expressions that are convertible to **bool** and for which the logical operators (7.6.14, 7.6.15, 7.6.2.2) have the conventional semantics.
- ```
template<class T>
concept boolean_testable_impl = convertible_to<T, bool>; // exposition only
```
- ² Let `e` be an expression such that `decltype((e))` is `T`. `T` models *boolean-testable-impl* only if
- (2.1) — either `remove_cvref_t<T>` is not a class type, or a search for the names `operator&&` and `operator||` in the scope of `remove_cvref_t<T>` finds nothing; and

- (2.2) — argument-dependent lookup (6.5.4) for the names `operator&&` and `operator||` with `T` as the only argument type finds no disqualifying declaration (defined below).

³ A *disqualifying parameter* is a function parameter whose declared type `P`

- (3.1) — is not dependent on a template parameter, and there exists an implicit conversion sequence (12.2.4.2) from `e` to `P`; or
 - (3.2) — is dependent on one or more template parameters, and either
 - (3.2.1) — `P` contains no template parameter that participates in template argument deduction (13.10.3.6), or
 - (3.2.2) — template argument deduction using the rules for deducing template arguments in a function call (13.10.3.2) and `e` as the argument succeeds.
- ⁴ A *key parameter* of a function template `D` is a function parameter of type *cv* `X` or reference thereto, where `X` names a specialization of a class template that has the same innermost enclosing non-inline namespace as `D`, and `X` contains at least one template parameter that participates in template argument deduction.

[Example 1: In

```
namespace Z {
    template<class> struct C {};
    template<class T>
        void operator&&(C<T> x, T y);
    template<class T>
        void operator|| (C<type_identity_t<T>> x, T y);
}
```

the declaration of `Z::operator&&` contains one key parameter, `C<T> x`, and the declaration of `Z::operator||` contains no key parameters. — end example]

⁵ A *disqualifying declaration* is

- (5.1) — a (non-template) function declaration that contains at least one disqualifying parameter; or
- (5.2) — a function template declaration that contains at least one disqualifying parameter, where
 - (5.2.1) — at least one disqualifying parameter is a key parameter; or
 - (5.2.2) — the declaration contains no key parameters; or
 - (5.2.3) — the declaration declares a function template to which no name is bound (9.3.4).

⁶ [Note 1: The intention is to ensure that given two types `T1` and `T2` that each model *boolean-testable-impl*, the `&&` and `||` operators within the expressions `declval<T1>() && declval<T2>()` and `declval<T1>() || declval<T2>()` resolve to the corresponding built-in operators. — end note]

```
template<class T>
    concept boolean-testable = // exposition only
        boolean-testable-impl<T> && requires(T&& t) {
            { !std::forward<T>(t) } -> boolean-testable-impl;
        };
};
```

⁷ Let `e` be an expression such that `decltype((e))` is `T`. `T` models *boolean-testable* only if `bool(e) == !bool(!e)`.

⁸ [Example 2: The types `bool`, `true_type` (21.3.3), `int*`, and `bitset<N>::reference` (22.9.2) model *boolean-testable*. — end example]

18.5.3 Comparison common types

[concept.comparisoncommontype]

```
template<class T, class U, class C = common_reference_t<const T&, const U&>>
    concept comparison-common-type-with-impl = // exposition only
        same_as<common_reference_t<const T&, const U&>,
            common_reference_t<const U&, const T&>>> &&
        requires {
            requires convertible_to<const T&, const C&> || convertible_to<T, const C&>;
            requires convertible_to<const U&, const C&> || convertible_to<U, const C&>;
        };
};
```

```
template<class T, class U>
concept comparison-common-type-with = // exposition only
    comparison-common-type-with-impl<remove_cvref_t<T>, remove_cvref_t<U>>;
```

- ¹ Let C be `common_reference_t<const T&, const U&>`. Let t1 and t2 be equality-preserving expressions that are lvalues of type `remove_cvref_t<T>`, and let u1 and u2 be equality-preserving expressions that are lvalues of type `remove_cvref_t<U>`. T and U model *comparison-common-type-with*<T, U> only if
- (1.1) — `CONVERT_TO_LVALUE<C>(t1)` equals `CONVERT_TO_LVALUE<C>(t2)` if and only if t1 equals t2, and
- (1.2) — `CONVERT_TO_LVALUE<C>(u1)` equals `CONVERT_TO_LVALUE<C>(u2)` if and only if u1 equals u2.

18.5.4 Concept `equality_comparable`

[`concept.equalitycomparable`]

```
template<class T, class U>
concept weakly-equality-comparable-with = // exposition only
    requires(const remove_reference_t<T>& t,
        const remove_reference_t<U>& u) {
        { t == u } -> boolean-testable;
        { t != u } -> boolean-testable;
        { u == t } -> boolean-testable;
        { u != t } -> boolean-testable;
    };
```

- ¹ Given types T and U, let t and u be lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>` respectively. T and U model *weakly-equality-comparable-with*<T, U> only if
- (1.1) — `t == u`, `u == t`, `t != u`, and `u != t` have the same domain.
- (1.2) — `bool(u == t) == bool(t == u)`.
- (1.3) — `bool(t != u) == !bool(t == u)`.
- (1.4) — `bool(u != t) == bool(t != u)`.

```
template<class T>
concept equality_comparable = weakly-equality-comparable-with<T, T>;
```

- ² Let a and b be objects of type T. T models `equality_comparable` only if `bool(a == b)` is true when a is equal to b (18.2), and false otherwise.
- ³ [Note 1: The requirement that the expression `a == b` is equality-preserving implies that `==` is transitive and symmetric. — end note]

```
template<class T, class U>
concept equality_comparable_with =
    equality_comparable<T> && equality_comparable<U> &&
    comparison-common-type-with<T, U> &&
    equality_comparable<
        common_reference_t<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&>> &&
    weakly-equality-comparable-with<T, U>;
```

- ⁴ Given types T and U, let t and t2 be lvalues denoting distinct equal objects of types `const remove_reference_t<T>` and `remove_cvref_t<T>`, respectively, let u and u2 be lvalues denoting distinct equal objects of types `const remove_reference_t<U>` and `remove_cvref_t<U>`, respectively, and let C be:
- `common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>`
- T and U model `equality_comparable_with<T, U>` only if
- `bool(t == u) == bool(CONVERT_TO_LVALUE<C>(t2) == CONVERT_TO_LVALUE<C>(u2))`

18.5.5 Concept `totally_ordered`

[`concept.totallyordered`]

```
template<class T>
concept totally_ordered =
    equality_comparable<T> && partially-ordered-with<T, T>;
```

1 Given a type `T`, let `a`, `b`, and `c` be lvalues of type `const remove_reference_t<T>`. `T` models `totally_`
`ordered` only if

- (1.1) — Exactly one of `bool(a < b)`, `bool(a > b)`, or `bool(a == b)` is true.
- (1.2) — If `bool(a < b)` and `bool(b < c)`, then `bool(a < c)`.
- (1.3) — `bool(a <= b) == !bool(b < a)`.
- (1.4) — `bool(a >= b) == !bool(a < b)`.

```
template<class T, class U>
concept totally_ordered_with =
    totally_ordered<T> && totally_ordered<U> &&
    equality_comparable_with<T, U> &&
    totally_ordered<
        common_reference_t<
            const remove_reference_t<T>&,
            const remove_reference_t<U>&>> &&
    partially_ordered_with<T, U>;
```

2 Given types `T` and `U`, let `t` and `t2` be lvalues denoting distinct equal objects of types `const remove_`
`reference_t<T>` and `remove_cvref_t<T>`, respectively, let `u` and `u2` be lvalues denoting distinct equal
objects of types `const remove_reference_t<U>` and `remove_cvref_t<U>`, respectively, and let `C` be:

```
common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>
```

`T` and `U` model `totally_ordered_with<T, U>` only if

- (2.1) — `bool(t < u) == bool(CONVERT_TO_LVALUE<C>(t2) < CONVERT_TO_LVALUE<C>(u2))`.
- (2.2) — `bool(t > u) == bool(CONVERT_TO_LVALUE<C>(t2) > CONVERT_TO_LVALUE<C>(u2))`.
- (2.3) — `bool(t <= u) == bool(CONVERT_TO_LVALUE<C>(t2) <= CONVERT_TO_LVALUE<C>(u2))`.
- (2.4) — `bool(t >= u) == bool(CONVERT_TO_LVALUE<C>(t2) >= CONVERT_TO_LVALUE<C>(u2))`.
- (2.5) — `bool(u < t) == bool(CONVERT_TO_LVALUE<C>(u2) < CONVERT_TO_LVALUE<C>(t2))`.
- (2.6) — `bool(u > t) == bool(CONVERT_TO_LVALUE<C>(u2) > CONVERT_TO_LVALUE<C>(t2))`.
- (2.7) — `bool(u <= t) == bool(CONVERT_TO_LVALUE<C>(u2) <= CONVERT_TO_LVALUE<C>(t2))`.
- (2.8) — `bool(u >= t) == bool(CONVERT_TO_LVALUE<C>(u2) >= CONVERT_TO_LVALUE<C>(t2))`.

18.6 Object concepts

[concepts.object]

1 This subclause describes concepts that specify the basis of the value-oriented programming style on which
the library is based.

```
template<class T>
concept movable = is_object_v<T> && move_constructible<T> &&
    assignable_from<T&, T> && swappable<T>;

template<class T>
concept copyable = copy_constructible<T> && movable<T> && assignable_from<T&, T&> &&
    assignable_from<T&, const T&> && assignable_from<T&, const T>;

template<class T>
concept semiregular = copyable<T> && default_initializable<T>;

template<class T>
concept regular = semiregular<T> && equality_comparable<T>;
```

2 [Note 1: The `semiregular` concept is modeled by types that behave similarly to fundamental types like `int`,
except that they need not be comparable with `==`. — end note]

3 [Note 2: The `regular` concept is modeled by types that behave similarly to fundamental types like `int` and
that are comparable with `==`. — end note]

18.7 Callable concepts

[concepts.callable]

18.7.1 General

[concepts.callable.general]

1 The concepts in 18.7 describe the requirements on callable types (22.10.3) and their arguments.

18.7.2 Concept invocable**[concept.invocable]**

- ¹ The **invocable** concept specifies a relationship between a callable type (22.10.3) **F** and a set of argument types **Args...** which can be evaluated by the library function **invoke** (22.10.5).

```
template<class F, class... Args>
concept invocable = requires(F&& f, Args&&... args) {
    invoke(std::forward<F>(f), std::forward<Args>(args)...); // not required to be equality-preserving
};
```

- ² [Example 1: A function that generates random numbers can model **invocable**, since the **invoke** function call expression is not required to be equality-preserving (18.2). — end example]

18.7.3 Concept regular_invocable**[concept.regularinvocable]**

```
template<class F, class... Args>
concept regular_invocable = invocable<F, Args...>;
```

- ¹ The **invoke** function call expression shall be equality-preserving (18.2) and shall not modify the function object or the arguments.

[Note 1: This requirement supersedes the “not required to be equality-preserving” comment in the definition of **invocable**. — end note]

- ² [Example 1: A random number generator does not model **regular_invocable**. — end example]

- ³ [Note 2: The distinction between **invocable** and **regular_invocable** is purely semantic. — end note]

18.7.4 Concept predicate**[concept.predicate]**

```
template<class F, class... Args>
concept predicate =
    regular_invocable<F, Args...> && boolean-testable<invoke_result_t<F, Args...>>;
```

18.7.5 Concept relation**[concept.relation]**

```
template<class R, class T, class U>
concept relation =
    predicate<R, T, T> && predicate<R, U, U> &&
    predicate<R, T, U> && predicate<R, U, T>;
```

18.7.6 Concept equivalence_relation**[concept.equiv]**

```
template<class R, class T, class U>
concept equivalence_relation = relation<R, T, U>;
```

- ¹ A relation models **equivalence_relation** only if it imposes an equivalence relation on its arguments.

18.7.7 Concept strict_weak_order**[concept.strictweakorder]**

```
template<class R, class T, class U>
concept strict_weak_order = relation<R, T, U>;
```

- ¹ A relation models **strict_weak_order** only if it imposes a *strict weak ordering* on its arguments.

- ² The term *strict* refers to the requirement of an irreflexive relation (**!comp(x, x)** for all **x**), and the term *weak* to requirements that are not as strong as those for a total ordering, but stronger than those for a partial ordering. If we define **equiv(a, b)** as **!comp(a, b) && !comp(b, a)**, then the requirements are that **comp** and **equiv** both be transitive relations:

(2.1) — **comp(a, b) && comp(b, c)** implies **comp(a, c)**

(2.2) — **equiv(a, b) && equiv(b, c)** implies **equiv(a, c)**

- ³ [Note 1: Under these conditions, it can be shown that

(3.1) — **equiv** is an equivalence relation,

(3.2) — **comp** induces a well-defined relation on the equivalence classes determined by **equiv**, and

(3.3) — the induced relation is a strict total ordering.

— end note]

19 Diagnostics library

[diagnostics]

19.1 General

[diagnostics.general]

- ¹ This Clause describes components that C++ programs may use to detect and report error conditions.
- ² The following subclauses describe components for reporting several kinds of exceptional conditions, documenting program assertions, obtaining stacktraces, and a global variable for error number codes, as summarized in Table 49.

Table 49 — Diagnostics library summary [tab:diagnostics.summary]

Subclause	Header
19.2 Exception classes	<stdexcept>
19.3 Assertions	<cassert>
19.4 Error numbers	<cerrno>
19.5 System error support	<system_error>
19.6 Stacktrace	<stacktrace>
19.7 Debugging	<debugging>

19.2 Exception classes

[std.exceptions]

19.2.1 General

[std.exceptions.general]

- ¹ The C++ standard library provides classes to be used to report certain errors ([16.4.6.14](#)) in C++ programs. In the error model reflected in these classes, errors are divided into two broad categories: *logic* errors and *runtime* errors.
- ² The distinguishing characteristic of logic errors is that they are due to errors in the internal logic of the program. In theory, they are preventable.
- ³ By contrast, runtime errors are due to events beyond the scope of the program. They cannot be easily predicted in advance. The header <stdexcept> defines several types of predefined exceptions for reporting errors in a C++ program. These exceptions are related by inheritance.

19.2.2 Header <stdexcept> synopsis

[stdexcept.syn]

```
namespace std {
    class logic_error;
    class domain_error;
    class invalid_argument;
    class length_error;
    class out_of_range;
    class runtime_error;
    class range_error;
    class overflow_error;
    class underflow_error;
}
```

19.2.3 Class logic_error

[logic.error]

```
namespace std {
    class logic_error : public exception {
    public:
        constexpr explicit logic_error(const string& what_arg);
        constexpr explicit logic_error(const char* what_arg);
    };
}
```

- ¹ The class `logic_error` defines the type of objects thrown as exceptions to report errors presumably detectable before the program executes, such as violations of logical preconditions or class invariants.

```
constexpr logic_error(const string& what_arg);
```

2 *Postconditions:* strcmp(what(), what_arg.c_str()) == 0.

```
constexpr logic_error(const char* what_arg);
```

3 *Postconditions:* strcmp(what(), what_arg) == 0.

19.2.4 Class domain_error

[domain.error]

```
namespace std {
```

```
    class domain_error : public logic_error {
```

```
    public:
```

```
        constexpr explicit domain_error(const string& what_arg);
```

```
        constexpr explicit domain_error(const char* what_arg);
```

```
    };
```

```
}
```

1 The class `domain_error` defines the type of objects thrown as exceptions by the implementation to report domain errors.

```
constexpr domain_error(const string& what_arg);
```

2 *Postconditions:* strcmp(what(), what_arg.c_str()) == 0.

```
constexpr domain_error(const char* what_arg);
```

3 *Postconditions:* strcmp(what(), what_arg) == 0.

19.2.5 Class invalid_argument

[invalid.argument]

```
namespace std {
```

```
    class invalid_argument : public logic_error {
```

```
    public:
```

```
        constexpr explicit invalid_argument(const string& what_arg);
```

```
        constexpr explicit invalid_argument(const char* what_arg);
```

```
    };
```

```
}
```

1 The class `invalid_argument` defines the type of objects thrown as exceptions to report an invalid argument.

```
constexpr invalid_argument(const string& what_arg);
```

2 *Postconditions:* strcmp(what(), what_arg.c_str()) == 0.

```
constexpr invalid_argument(const char* what_arg);
```

3 *Postconditions:* strcmp(what(), what_arg) == 0.

19.2.6 Class length_error

[length.error]

```
namespace std {
```

```
    class length_error : public logic_error {
```

```
    public:
```

```
        constexpr explicit length_error(const string& what_arg);
```

```
        constexpr explicit length_error(const char* what_arg);
```

```
    };
```

```
}
```

1 The class `length_error` defines the type of objects thrown as exceptions to report an attempt to produce an object whose length exceeds its maximum allowable size.

```
constexpr length_error(const string& what_arg);
```

2 *Postconditions:* strcmp(what(), what_arg.c_str()) == 0.

```
constexpr length_error(const char* what_arg);
```

3 *Postconditions:* strcmp(what(), what_arg) == 0.

19.2.7 Class out_of_range**[out.of.range]**

```
namespace std {
    class out_of_range : public logic_error {
    public:
        constexpr explicit out_of_range(const string& what_arg);
        constexpr explicit out_of_range(const char* what_arg);
    };
}
```

- ¹ The class `out_of_range` defines the type of objects thrown as exceptions to report an argument value not in its expected range.

```
constexpr out_of_range(const string& what_arg);
```

- ² *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr out_of_range(const char* what_arg);
```

- ³ *Postconditions:* `strcmp(what(), what_arg) == 0.`

19.2.8 Class runtime_error**[runtime.error]**

```
namespace std {
    class runtime_error : public exception {
    public:
        constexpr explicit runtime_error(const string& what_arg);
        constexpr explicit runtime_error(const char* what_arg);
    };
}
```

- ¹ The class `runtime_error` defines the type of objects thrown as exceptions to report errors presumably detectable only when the program executes.

```
constexpr runtime_error(const string& what_arg);
```

- ² *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr runtime_error(const char* what_arg);
```

- ³ *Postconditions:* `strcmp(what(), what_arg) == 0.`

19.2.9 Class range_error**[range.error]**

```
namespace std {
    class range_error : public runtime_error {
    public:
        constexpr explicit range_error(const string& what_arg);
        constexpr explicit range_error(const char* what_arg);
    };
}
```

- ¹ The class `range_error` defines the type of objects thrown as exceptions to report range errors in internal computations.

```
constexpr range_error(const string& what_arg);
```

- ² *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr range_error(const char* what_arg);
```

- ³ *Postconditions:* `strcmp(what(), what_arg) == 0.`

19.2.10 Class overflow_error**[overflow.error]**

```
namespace std {
    class overflow_error : public runtime_error {
    public:
        constexpr explicit overflow_error(const string& what_arg);
        constexpr explicit overflow_error(const char* what_arg);
    };
}
```


- ¹ The class `overflow_error` defines the type of objects thrown as exceptions to report an arithmetic overflow error.

```
constexpr overflow_error(const string& what_arg);
```

- ² *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr overflow_error(const char* what_arg);
```

- ³ *Postconditions:* `strcmp(what(), what_arg) == 0.`

19.2.11 Class `underflow_error`

[underflow.error]

```
namespace std {
```

```
    class underflow_error : public runtime_error {
```

```
    public:
```

```
        constexpr explicit underflow_error(const string& what_arg);
```

```
        constexpr explicit underflow_error(const char* what_arg);
```

```
    };
```

```
}
```

- ¹ The class `underflow_error` defines the type of objects thrown as exceptions to report an arithmetic underflow error.

```
constexpr underflow_error(const string& what_arg);
```

- ² *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr underflow_error(const char* what_arg);
```

- ³ *Postconditions:* `strcmp(what(), what_arg) == 0.`

19.3 Assertions

[assertions]

19.3.1 General

[assertions.general]

- ¹ The header `<cassert>` provides a macro for documenting C++ program assertions and a mechanism for disabling the assertion checks through defining the macro `NDEBUG`.

19.3.2 Header `<cassert>` synopsis

[cassert.syn]

```
#define assert(...) see below
```

19.3.3 The `assert` macro

[assertions.assert]

- ¹ If `NDEBUG` is defined as a macro name at the point in the source file where `<cassert>` is included, the `assert` macro is defined as

```
#define assert(...) ((void)0)
```

- ² Otherwise, the `assert` macro puts a diagnostic test into programs; it expands to an expression of type `void` which has the following effects:

- (2.1) — `__VA_ARGS__` is evaluated and contextually converted to `bool`.
- (2.2) — If the evaluation yields `true` there are no further effects.
- (2.3) — Otherwise, the `assert` macro's expression creates a diagnostic on the standard error stream (ISO/IEC 9899:2024, 7.23.3) in an implementation-defined format and calls `abort()`. The diagnostic contains `#__VA_ARGS__` and information on the name of the source file, the source line number, and the name of the enclosing function (such as provided by `source_location::current()`).
- ³ If `__VA_ARGS__` does not expand to a well-formed *assignment-expression*, the program is ill-formed. If such an *assignment-expression* is ill-formed when treated as an unevaluated operand (7.6.2.4, 7.6.17), the program is ill-formed, no diagnostic required.
- ⁴ The macro `assert` is redefined according to the current state of `NDEBUG` each time that `<cassert>` is included.
- ⁵ An expression `assert(E)` is a constant subexpression (3.15), if
 - (5.1) — `NDEBUG` is defined at the point where `assert` is last defined or redefined, or
 - (5.2) — `E` contextually converted to `bool` (7.3) is a constant subexpression that evaluates to the value `true`.

19.4 Error numbers

[errno]

19.4.1 General

[errno.general]

- ¹ The contents of the header `<cerrno>` are the same as the POSIX header `<errno.h>`, except that `errno` shall be defined as a macro.

[*Note 1*: The intent is to remain in close alignment with the POSIX standard. — *end note*]

A separate `errno` value is provided for each thread.

19.4.2 Header `<cerrno>` synopsis

[cerrno.syn]

```
#define errno see below

#define E2BIG see below // freestanding
#define EACCES see below // freestanding
#define EADDRINUSE see below // freestanding
#define EADDRNOTAVAIL see below // freestanding
#define EAFNOSUPPORT see below // freestanding
#define EAGAIN see below // freestanding
#define EALREADY see below // freestanding
#define EBADF see below // freestanding
#define EBADMSG see below // freestanding
#define EBUSY see below // freestanding
#define ECANCELED see below // freestanding
#define ECHILD see below // freestanding
#define ECONNABORTED see below // freestanding
#define ECONNREFUSED see below // freestanding
#define ECONNRESET see below // freestanding
#define EDEADLK see below // freestanding
#define EDESTADDRREQ see below // freestanding
#define EDOM see below // freestanding
#define EEXIST see below // freestanding
#define EFAULT see below // freestanding
#define EFBIG see below // freestanding
#define EHOSTUNREACH see below // freestanding
#define EIDRM see below // freestanding
#define EILSEQ see below // freestanding
#define EINPROGRESS see below // freestanding
#define EINTR see below // freestanding
#define EINVAL see below // freestanding
#define EIO see below // freestanding
#define EISCONN see below // freestanding
#define EISDIR see below // freestanding
#define ELOOP see below // freestanding
#define EMFILE see below // freestanding
#define EMLINK see below // freestanding
#define EMSGSIZE see below // freestanding
#define ENAMETOOLONG see below // freestanding
#define ENETDOWN see below // freestanding
#define ENETRESET see below // freestanding
#define ENETUNREACH see below // freestanding
#define ENFILE see below // freestanding
#define ENOBUFS see below // freestanding
#define ENODEV see below // freestanding
#define ENOENT see below // freestanding
#define ENOEXEC see below // freestanding
#define ENOLCK see below // freestanding
#define ENOLINK see below // freestanding
#define ENOMEM see below // freestanding
#define ENOMSG see below // freestanding
#define ENOPROTOOPT see below // freestanding
#define ENOSPC see below // freestanding
#define ENOSYS see below // freestanding
#define ENOTCONN see below // freestanding
#define ENOTDIR see below // freestanding
```

```

#define ENOTEMPTY see below // freestanding
#define ENOTRECOVERABLE see below // freestanding
#define ENOTSOCK see below // freestanding
#define ENOTSUP see below // freestanding
#define ENOTTY see below // freestanding
#define ENXIO see below // freestanding
#define EOPNOTSUPP see below // freestanding
#define EOVERFLOW see below // freestanding
#define EOWNERDEAD see below // freestanding
#define EPERM see below // freestanding
#define EPIPE see below // freestanding
#define EPROTO see below // freestanding
#define EPROTONOSUPPORT see below // freestanding
#define EPROTOTYPE see below // freestanding
#define ERANGE see below // freestanding
#define EROFS see below // freestanding
#define ESPIPE see below // freestanding
#define ESRCH see below // freestanding
#define ETIMEDOUT see below // freestanding
#define ETXTBSY see below // freestanding
#define EWOULDBLOCK see below // freestanding
#define EXDEV see below // freestanding

```

¹ The meaning of the macros in this header is defined by the POSIX standard.

SEE ALSO: ISO/IEC 9899:2024, 7.5

19.5 System error support

[syserr]

19.5.1 General

[syserr.general]

¹ Subclause 19.5 describes components that the standard library and C++ programs may use to report error conditions originating from the operating system or other low-level application program interfaces.

² Components described in 19.5 do not change the value of `errno` (19.4).

Recommended practice: Implementations should leave the error states provided by other libraries unchanged.

19.5.2 Header <system_error> synopsis

[system.error.syn]

```

#include <compare> // see 17.12.1

namespace std {
    class error_category;
    const error_category& generic_category() noexcept;
    const error_category& system_category() noexcept;

    class error_code;
    class error_condition;
    class system_error;

    template<class T>
        struct is_error_code_enum : public false_type {};

    template<class T>
        struct is_error_condition_enum : public false_type {};

    enum class errc {
        address_family_not_supported, // EAFNOSUPPORT
        address_in_use, // EADDRINUSE
        address_not_available, // EADDRNOTAVAIL
        already_connected, // EISCONN
        argument_list_too_long, // E2BIG
        argument_out_of_domain, // EDOM
        bad_address, // EFAULT
        bad_file_descriptor, // EBADF
        bad_message, // EBADMSG
        broken_pipe, // EPIPE
    } // freestanding

```

connection_aborted,	// ECONNABORTED
connection_already_in_progress,	// EALREADY
connection_refused,	// ECONNREFUSED
connection_reset,	// ECONNRESET
cross_device_link,	// EXDEV
destination_address_required,	// EDESTADDRREQ
device_or_resource_busy,	// EBUSY
directory_not_empty,	// ENOTEMPTY
executable_format_error,	// ENOEXEC
file_exists,	// EEXIST
file_too_large,	// EFBIG
filename_too_long,	// ENAMETOOLONG
function_not_supported,	// ENOSYS
host_unreachable,	// EHOSTUNREACH
identifier_removed,	// EIDRM
illegal_byte_sequence,	// EILSEQ
inappropriate_io_control_operation,	// ENOTTY
interrupted,	// EINTR
invalid_argument,	// EINVAL
invalid_seek,	// ESPIPE
io_error,	// EIO
is_a_directory,	// EISDIR
message_size,	// EMSGSIZE
network_down,	// ENETDOWN
network_reset,	// ENETRESET
network_unreachable,	// ENETUNREACH
no_buffer_space,	// ENOBUFS
no_child_process,	// ECHILD
no_link,	// ENOLINK
no_lock_available,	// ENOLCK
no_message,	// ENOMSG
no_protocol_option,	// ENOPROTOOPT
no_space_on_device,	// ENOSPC
no_such_device_or_address,	// ENXIO
no_such_device,	// ENODEV
no_such_file_or_directory,	// ENOENT
no_such_process,	// ESRCH
not_a_directory,	// ENOTDIR
not_a_socket,	// ENOTSOCK
not_connected,	// ENOTCONN
not_enough_memory,	// ENOMEM
not_supported,	// ENOTSUP
operation_canceled,	// ECANCELED
operation_in_progress,	// EINPROGRESS
operation_not_permitted,	// EPERM
operation_not_supported,	// EOPNOTSUPP
operation_would_block,	// EWOULDBLOCK
owner_dead,	// EOWNERDEAD
permission_denied,	// EACCES
protocol_error,	// EPROTO
protocol_not_supported,	// EPROTONOSUPPORT
read_only_file_system,	// EROFS
resource_deadlock_would_occur,	// EDEADLK
resource_unavailable_try_again,	// EAGAIN
result_out_of_range,	// ERANGE
state_not_recoverable,	// ENOTRECOVERABLE
text_file_busy,	// ETXTBSY
timed_out,	// ETIMEDOUT
too_many_files_open_in_system,	// ENFILE
too_many_files_open,	// EMFILE
too_many_links,	// EMLINK
too_many_symbolic_link_levels,	// ELOOP
value_too_large,	// EOVERFLOW
wrong_protocol_type,	// EPROTOTYPE

```

};

template<> struct is_error_condition_enum<errc> : true_type {};

// 19.5.4.5, non-member functions
error_code make_error_code(errc e) noexcept;

template<class charT, class traits>
    basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const error_code& ec);

// 19.5.5.5, non-member functions
error_condition make_error_condition(errc e) noexcept;

// 19.5.6, comparison operator functions
bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
strong_ordering operator<=>(const error_code& lhs, const error_code& rhs) noexcept;
strong_ordering operator<=>(const error_condition& lhs, const error_condition& rhs) noexcept;

// 19.5.7, hash support
template<class T> struct hash;
template<> struct hash<error_code>;
template<> struct hash<error_condition>;

// 19.5, system error support
template<class T>
    constexpr bool is_error_code_enum_v = is_error_code_enum<T>::value;
template<class T>
    constexpr bool is_error_condition_enum_v = is_error_condition_enum<T>::value;

// 19.5.4.6, formatter
template<class T> struct formatter<error_code, charT>;
}

```

- ¹ The value of each enum `errc` enumerator is the same as the value of the `<cerrno>` macro shown in the above synopsis. Whether or not the `<system_error>` implementation exposes the `<cerrno>` macros is unspecified.
- ² The `is_error_code_enum` and `is_error_condition_enum` templates may be specialized for program-defined types to indicate that such types are eligible for `class error_code` and `class error_condition` implicit conversions, respectively.

19.5.3 Class `error_category`

[syserr.errcat]

19.5.3.1 Overview

[syserr.errcat.overview]

- ¹ The class `error_category` serves as a base class for types used to identify the source and encoding of a particular category of error code. Classes may be derived from `error_category` to support categories of errors in addition to those defined in this document. Such classes shall behave as specified in subclause 19.5.3.

[Note 1: `error_category` objects are passed by reference, and two such objects are equal if they have the same address. If there is more than a single object of a custom `error_category` type, such equality comparisons can evaluate to `false` even for objects holding the same value. — end note]

```

namespace std {
    class error_category {
    public:
        constexpr error_category() noexcept;
        virtual ~error_category();
        error_category(const error_category&) = delete;
        error_category& operator=(const error_category&) = delete;
        virtual const char* name() const noexcept = 0;
        virtual error_condition default_error_condition(int ev) const noexcept;
        virtual bool equivalent(int code, const error_condition& condition) const noexcept;
        virtual bool equivalent(const error_code& code, int condition) const noexcept;
        virtual string message(int ev) const = 0;
    };
}

```

```

    bool operator==(const error_category& rhs) const noexcept;
    strong_ordering operator<=>(const error_category& rhs) const noexcept;
};

const error_category& generic_category() noexcept;
const error_category& system_category() noexcept;
}

```

19.5.3.2 Virtual members

[syserr.errcat.virtuals]

```
virtual const char* name() const noexcept = 0;
```

1 *Returns:* A string in the ordinary literal encoding naming the error category.

```
virtual error_condition default_error_condition(int ev) const noexcept;
```

2 *Returns:* error_condition(ev, *this).

```
virtual bool equivalent(int code, const error_condition& condition) const noexcept;
```

3 *Returns:* default_error_condition(code) == condition.

```
virtual bool equivalent(const error_code& code, int condition) const noexcept;
```

4 *Returns:* *this == code.category() && code.value() == condition.

```
virtual string message(int ev) const = 0;
```

5 *Returns:* A string of multibyte characters in the execution character set that describes the error condition denoted by ev.

19.5.3.3 Non-virtual members

[syserr.errcat.nonvirtuals]

```
bool operator==(const error_category& rhs) const noexcept;
```

1 *Returns:* this == &rhs.

```
strong_ordering operator<=>(const error_category& rhs) const noexcept;
```

2 *Returns:* compare_three_way()(this, &rhs).

[Note 1: compare_three_way (22.10.8.8) provides a total ordering for pointers. — end note]

19.5.3.4 Program-defined classes derived from error_category

[syserr.errcat.derived]

```
virtual const char* name() const noexcept = 0;
```

1 *Returns:* A string naming the error category.

```
virtual error_condition default_error_condition(int ev) const noexcept;
```

2 *Returns:* An object of type error_condition that corresponds to ev.

```
virtual bool equivalent(int code, const error_condition& condition) const noexcept;
```

3 *Returns:* true if, for the category of error represented by *this, code is considered equivalent to condition; otherwise, false.

```
virtual bool equivalent(const error_code& code, int condition) const noexcept;
```

4 *Returns:* true if, for the category of error represented by *this, code is considered equivalent to condition; otherwise, false.

19.5.3.5 Error category objects

[syserr.errcat.objects]

```
const error_category& generic_category() noexcept;
```

1 *Returns:* A reference to an object of a type derived from class error_category. All calls to this function shall return references to the same object.

2 *Remarks:* The object's default_error_condition and equivalent virtual functions shall behave as specified for the class error_category. The object's name virtual function shall return a pointer to the string "generic".

```
const error_category& system_category() noexcept;
```

³ *Returns:* A reference to an object of a type derived from class `error_category`. All calls to this function shall return references to the same object.

⁴ *Remarks:* The object's equivalent virtual functions shall behave as specified for class `error_category`. The object's `name` virtual function shall return a pointer to the string "system". The object's `default_error_condition` virtual function shall behave as follows:

If the argument `ev` is equal to 0, the function returns `error_condition(0, generic_category())`. Otherwise, if `ev` corresponds to a POSIX `errno` value `pxv`, the function returns `error_condition(pxv, generic_category())`. Otherwise, the function returns `error_condition(ev, system_category())`. What constitutes correspondence for any given operating system is unspecified.

[*Note 1:* The number of potential system error codes is large and unbounded, and some might not correspond to any POSIX `errno` value. Thus implementations are given latitude in determining correspondence. — end note]

19.5.4 Class `error_code`

[syserr.errcode]

19.5.4.1 Overview

[syserr.errcode.overview]

¹ The class `error_code` describes an object used to hold error code values, such as those originating from the operating system or other low-level application program interfaces.

[*Note 1:* Class `error_code` is an adjunct to error reporting by exception. — end note]

```
namespace std {
    class error_code {
    public:
        // 19.5.4.2, constructors
        error_code() noexcept;
        error_code(int val, const error_category& cat) noexcept;
        template<class ErrorCodeEnum>
            error_code(ErrorCodeEnum e) noexcept;

        // 19.5.4.3, modifiers
        void assign(int val, const error_category& cat) noexcept;
        template<class ErrorCodeEnum>
            error_code& operator=(ErrorCodeEnum e) noexcept;
        void clear() noexcept;

        // 19.5.4.4, observers
        int value() const noexcept;
        const error_category& category() const noexcept;
        error_condition default_error_condition() const noexcept;
        string message() const;
        explicit operator bool() const noexcept;

    private:
        int val_; // exposition only
        const error_category* cat_; // exposition only
    };

    // 19.5.4.5, non-member functions
    error_code make_error_code(errc e) noexcept;

    template<class charT, class traits>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const error_code& ec);
}
```

19.5.4.2 Constructors

[syserr.errcode.constructors]

```
error_code() noexcept;
```

¹ *Effects:* Initializes `val_` with 0 and `cat_` with `&system_category()`.

```
error_code(int val, const error_category& cat) noexcept;
```

² *Effects:* Initializes `val_` with `val` and `cat_` with `&cat`.

```

template<class ErrorCodeEnum>
    error_code(ErrorCodeEnum e) noexcept;
3     Constraints: is_error_code_enum_v<ErrorCodeEnum> is true.
4     Effects: Equivalent to:
        error_code ec = make_error_code(e);
        assign(ec.value(), ec.category());

```

19.5.4.3 Modifiers

[syserr.errcode.modifiers]

```

void assign(int val, const error_category& cat) noexcept;
1     Postconditions: val_ == val and cat_ == &cat.

template<class ErrorCodeEnum>
    error_code& operator=(ErrorCodeEnum e) noexcept;
2     Constraints: is_error_code_enum_v<ErrorCodeEnum> is true.
3     Effects: Equivalent to:
        error_code ec = make_error_code(e);
        assign(ec.value(), ec.category());
4     Returns: *this.

void clear() noexcept;
5     Postconditions: value() == 0 and category() == system_category().

```

19.5.4.4 Observers

[syserr.errcode.observers]

```

int value() const noexcept;
1     Returns: val_.

const error_category& category() const noexcept;
2     Returns: *cat_.

error_condition default_error_condition() const noexcept;
3     Returns: category().default_error_condition(value()).

string message() const;
4     Returns: category().message(value()).

explicit operator bool() const noexcept;
5     Returns: value() != 0.

```

19.5.4.5 Non-member functions

[syserr.errcode.nonmembers]

```

error_code make_error_code(errc e) noexcept;
1     Returns: error_code(static_cast<int>(e), generic_category()).

template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& os, const error_code& ec);
2     Effects: Equivalent to: return os << ec.category().name() << ':' << ec.value();

```

19.5.4.6 Formatting

[syserr.fmt]

```

1     template<class charT>
        struct formatter<error_code, charT> {
            constexpr void set_debug_format();

            constexpr typename basic_format_parse_context<charT>::iterator
                parse(basic_format_parse_context<charT>& ctx);

```



```
template<class Out>
    typename basic_format_context<Out, charT>::iterator
        format(const error_code& ec, basic_format_context<Out, charT>& ctx) const;
};
```

```
constexpr void set_debug_format();
```

- ² *Effects:* Modifies the state of the formatter to be as if the *error-code-format-spec* parsed by the last call to `parse` contained the `?` option.

```
constexpr typename basic_format_parse_context<charT>::iterator
    parse(basic_format_parse_context<charT>& ctx);
```

- ³ *Effects:* Parses the format specifier as an *error-code-format-spec* and stores the parsed specifiers in `*this`. The syntax of format specifications is

error-code-format-spec:
*fill-and-align*_{opt} *width*_{opt} *?opt* *s*_{opt}

where the productions *fill-and-align* and *width* are described in 28.5.2.

- ⁴ *Returns:* An iterator past the end of the *error-code-format-spec*.

```
template<class Out>
    typename basic_format_context<Out, charT>::iterator
        format(const error_code& ec, basic_format_context<Out, charT>& ctx) const;
```

- ⁵ *Effects:* If the `s` option is used, then:

- (5.1) — If `charT` is `char` and the ordinary literal encoding is UTF-8, then let `msg` be `ec.message()` transcoded to UTF-8 with maximal subparts of ill-formed subsequences substituted with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

- (5.2) — Otherwise, let `msg` be `ec.message()` transcoded to an implementation-defined encoding.

Otherwise, let `msg` be `format("{}:{}", ec.category().name(), ec.value())`. If the `?` option is used then `msg` is formatted as an escaped string (28.5.6.5). Writes `msg` into `ctx.out()`, adjusted according to the *error-code-format-spec*.

- ⁶ *Returns:* An iterator past the end of the output range.

19.5.5 Class `error_condition`

[`syserr.errcondition`]

19.5.5.1 Overview

[`syserr.errcondition.overview`]

- ¹ The class `error_condition` describes an object used to hold values identifying error conditions.

[*Note 1: error_condition values are portable abstractions, while error_code values (19.5.4) are implementation specific. — end note*]

```
namespace std {
    class error_condition {
    public:
        // 19.5.5.2, constructors
        error_condition() noexcept;
        error_condition(int val, const error_category& cat) noexcept;
        template<class ErrorConditionEnum>
            error_condition(ErrorConditionEnum e) noexcept;

        // 19.5.5.3, modifiers
        void assign(int val, const error_category& cat) noexcept;
        template<class ErrorConditionEnum>
            error_condition& operator=(ErrorConditionEnum e) noexcept;
        void clear() noexcept;

        // 19.5.5.4, observers
        int value() const noexcept;
        const error_category& category() const noexcept;
        string message() const;
        explicit operator bool() const noexcept;
```

```

private:
    int val_; // exposition only
    const error_category* cat_; // exposition only
};
}

```

19.5.5.2 Constructors

[syserr.errcondition.constructors]

```
error_condition() noexcept;
```

1 *Effects:* Initializes *val_* with 0 and *cat_* with *&generic_category()*.

```
error_condition(int val, const error_category& cat) noexcept;
```

2 *Effects:* Initializes *val_* with *val* and *cat_* with *&cat*.

```
template<class ErrorConditionEnum>
error_condition(ErrorConditionEnum e) noexcept;
```

3 *Constraints:* *is_error_condition_enum_v<ErrorConditionEnum>* is true.

4 *Effects:* Equivalent to:

```

    error_condition ec = make_error_condition(e);
    assign(ec.value(), ec.category());

```

19.5.5.3 Modifiers

[syserr.errcondition.modifiers]

```
void assign(int val, const error_category& cat) noexcept;
```

1 *Postconditions:* *val_ == val* and *cat_ == &cat*.

```
template<class ErrorConditionEnum>
error_condition& operator=(ErrorConditionEnum e) noexcept;
```

2 *Constraints:* *is_error_condition_enum_v<ErrorConditionEnum>* is true.

3 *Effects:* Equivalent to:

```

    error_condition ec = make_error_condition(e);
    assign(ec.value(), ec.category());

```

4 *Returns:* **this*.

```
void clear() noexcept;
```

5 *Postconditions:* *value() == 0* and *category() == generic_category()*.

19.5.5.4 Observers

[syserr.errcondition.observers]

```
int value() const noexcept;
```

1 *Returns:* *val_*.

```
const error_category& category() const noexcept;
```

2 *Returns:* **cat_*.

```
string message() const;
```

3 *Returns:* *category().message(value())*.

```
explicit operator bool() const noexcept;
```

4 *Returns:* *value() != 0*.

19.5.5.5 Non-member functions

[syserr.errcondition.nonmembers]

```
error_condition make_error_condition(errc e) noexcept;
```

1 *Returns:* *error_condition(static_cast<int>(e), generic_category())*.

19.5.6 Comparison operator functions

[syserr.compare]

```
bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
```

1 *Returns:*

```
lhs.category() == rhs.category() && lhs.value() == rhs.value()
```

```
bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
```

2 *Returns:*

```
lhs.category().equivalent(lhs.value(), rhs) || rhs.category().equivalent(lhs, rhs.value())
```

```
bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
```

3 *Returns:*

```
lhs.category() == rhs.category() && lhs.value() == rhs.value()
```

```
strong_ordering operator<=>(const error_code& lhs, const error_code& rhs) noexcept;
```

4 *Effects:* Equivalent to:

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

```
strong_ordering operator<=>(const error_condition& lhs, const error_condition& rhs) noexcept;
```

5 *Returns:*

```
if (auto c = lhs.category() <=> rhs.category(); c != 0) return c;
return lhs.value() <=> rhs.value();
```

19.5.7 System error hash support

[syserr.hash]

```
template<> struct hash<error_code>;
```

```
template<> struct hash<error_condition>;
```

1 The specializations are enabled (22.10.19).

19.5.8 Class `system_error`

[syserr.syserr]

19.5.8.1 Overview

[syserr.syserr.overview]

1 The class `system_error` describes an exception object used to report error conditions that have an associated error code. Such error conditions typically originate from the operating system or other low-level application program interfaces.

2 [Note 1: If an error represents an out-of-memory condition, implementations are encouraged to throw an exception object of type `bad_alloc` (17.6.4.1) rather than `system_error`. — end note]

```
namespace std {
    class system_error : public runtime_error {
    public:
        system_error(error_code ec, const string& what_arg);
        system_error(error_code ec, const char* what_arg);
        system_error(error_code ec);
        system_error(int ev, const error_category&ecat, const string& what_arg);
        system_error(int ev, const error_category&ecat, const char* what_arg);
        system_error(int ev, const error_category&ecat);
        const error_code& code() const noexcept;
        const char* what() const noexcept override;
    };
}
```

19.5.8.2 Members

[syserr.syserr.members]

```
system_error(error_code ec, const string& what_arg);
```

1 *Postconditions:* `code() == ec` and

```
string_view(what()).find(what_arg.c_str()) != string_view::npos.
```

```
system_error(error_code ec, const char* what_arg);
```

2 *Postconditions:* `code() == ec` and `string_view(what()).find(what_arg) != string_view::npos.`

```

system_error(error_code ec);
3     Postconditions: code() == ec.

system_error(int ev, const error_category& ecat, const string& what_arg);
4     Postconditions: code() == error_code(ev, ecat) and
        string_view(what()).find(what_arg.c_str()) != string_view::npos.

system_error(int ev, const error_category& ecat, const char* what_arg);
5     Postconditions: code() == error_code(ev, ecat) and
        string_view(what()).find(what_arg) != string_view::npos.

system_error(int ev, const error_category& ecat);
6     Postconditions: code() == error_code(ev, ecat).

const error_code& code() const noexcept;
7     Returns: ec or error_code(ev, ecat), from the constructor, as appropriate.

const char* what() const noexcept override;
8     Returns: An NTBS incorporating the arguments supplied in the constructor.
        [Note 1: The returned NTBS might be the contents of what_arg + ": " + code.message(). — end note]

```

19.6 Stacktrace

[stacktrace]

19.6.1 General

[stacktrace.general]

- 1 Subclause 19.6 describes components that C++ programs may use to store the stacktrace of the current thread of execution and query information about the stored stacktrace at runtime.
- 2 The *invocation sequence* of the current evaluation x_0 in the current thread of execution is a sequence $(x_0, \dots, x_n)$ of evaluations such that, for $i \geq 0$, x_i is within the function invocation x_{i+1} (6.10.1).
- 3 A *stacktrace* is an approximate representation of an invocation sequence and consists of stacktrace entries. A *stacktrace entry* represents an evaluation in a stacktrace.

19.6.2 Header <stacktrace> synopsis

[stacktrace.syn]

```

#include <compare>                                // see 17.12.1

namespace std {
    // 19.6.3, class stacktrace_entry
    class stacktrace_entry;

    // 19.6.4, class template basic_stacktrace
    template<class Allocator>
        class basic_stacktrace;

    // basic_stacktrace typedef-names
    using stacktrace = basic_stacktrace<allocator<stacktrace_entry>>;

    // 19.6.4.6, non-member functions
    template<class Allocator>
        void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b)
            noexcept(noexcept(a.swap(b)));

    string to_string(const stacktrace_entry& f);

    template<class Allocator>
        string to_string(const basic_stacktrace<Allocator>& st);

    ostream& operator<<(ostream& os, const stacktrace_entry& f);
    template<class Allocator>
        ostream& operator<<(ostream& os, const basic_stacktrace<Allocator>& st);

```

```

// 19.6.5, formatting support
template<> struct formatter<stacktrace_entry>;
template<class Allocator> struct formatter<basic_stacktrace<Allocator>>;

namespace pmr {
    using stacktrace = basic_stacktrace<polymorphic_allocator<stacktrace_entry>>;
}

// 19.6.6, hash support
template<class T> struct hash;
template<> struct hash<stacktrace_entry>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
}

```

19.6.3 Class `stacktrace_entry`

[stacktrace.entry]

19.6.3.1 Overview

[stacktrace.entry.overview]

```

namespace std {
    class stacktrace_entry {
    public:
        using native_handle_type = implementation-defined;

        // 19.6.3.2, constructors
        constexpr stacktrace_entry() noexcept;
        constexpr stacktrace_entry(const stacktrace_entry& other) noexcept;
        constexpr stacktrace_entry& operator=(const stacktrace_entry& other) noexcept;

        ~stacktrace_entry();

        // 19.6.3.3, observers
        constexpr native_handle_type native_handle() const noexcept;
        constexpr explicit operator bool() const noexcept;

        // 19.6.3.4, query
        string description() const;
        string source_file() const;
        uint_least32_t source_line() const;

        // 19.6.3.5, comparison
        friend constexpr bool operator==(const stacktrace_entry& x,
                                         const stacktrace_entry& y) noexcept;
        friend constexpr strong_ordering operator<=>(const stacktrace_entry& x,
                                                     const stacktrace_entry& y) noexcept;
    };
}

```

- ¹ An object of type `stacktrace_entry` is either empty, or represents a stacktrace entry and provides operations for querying information about it. The class `stacktrace_entry` models `regular` (18.6) and `three_way_comparable<strong_ordering>` (17.12.4).

19.6.3.2 Constructors

[stacktrace.entry.cons]

```
constexpr stacktrace_entry() noexcept;
```

- ¹ *Postconditions:* `*this` is empty.

19.6.3.3 Observers

[stacktrace.entry.obs]

```
constexpr native_handle_type native_handle() const noexcept;
```

- ¹ The semantics of this function are implementation-defined.

- ² *Remarks:* Successive invocations of the `native_handle` function for an unchanged `stacktrace_entry` object return identical values.

```
constexpr explicit operator bool() const noexcept;
```

- ³ *Returns:* `false` if and only if `*this` is empty.

19.6.3.4 Query**[stacktrace.entry.query]**

- ¹ [Note 1: All the `stacktrace_entry` query functions treat errors other than memory allocation errors as “no information available” and do not throw in that case. — end note]

```
string description() const;
```

- ² *Returns:* A description of the evaluation represented by `*this`, or an empty string.

- ³ *Throws:* `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
string source_file() const;
```

- ⁴ *Returns:* The presumed or actual name of the source file (15.12) that lexically contains the expression or statement whose evaluation is represented by `*this`, or an empty string.

- ⁵ *Throws:* `bad_alloc` if memory for the internal data structures or the resulting string cannot be allocated.

```
uint_least32_t source_line() const;
```

- ⁶ *Returns:* 0, or a 1-based line number that lexically relates to the evaluation represented by `*this`. If `source_file` returns the presumed name of the source file, returns the presumed line number; if `source_file` returns the actual name of the source file, returns the actual line number.

- ⁷ *Throws:* `bad_alloc` if memory for the internal data structures cannot be allocated.

19.6.3.5 Comparison**[stacktrace.entry.cmp]**

```
friend constexpr bool operator==(const stacktrace_entry& x, const stacktrace_entry& y) noexcept;
```

- ¹ *Returns:* true if and only if x and y represent the same stacktrace entry or both x and y are empty.

19.6.4 Class template basic_stacktrace**[stacktrace.basic]****19.6.4.1 Overview****[stacktrace.basic.overview]**

```
namespace std {
    template<class Allocator>
    class basic_stacktrace {
    public:
        using value_type = stacktrace_entry;
        using const_reference = const value_type&;
        using reference = value_type&;
        using const_iterator = implementation-defined; // see 19.6.4.3
        using iterator = const_iterator;
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;
        using difference_type = implementation-defined;
        using size_type = implementation-defined;
        using allocator_type = Allocator;

        // 19.6.4.2, creation and assignment
        static basic_stacktrace current(const allocator_type& alloc = allocator_type()) noexcept;
        static basic_stacktrace current(size_type skip,
                                        const allocator_type& alloc = allocator_type()) noexcept;
        static basic_stacktrace current(size_type skip, size_type max_depth,
                                        const allocator_type& alloc = allocator_type()) noexcept;

        basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
        explicit basic_stacktrace(const allocator_type& alloc) noexcept;

        basic_stacktrace(const basic_stacktrace& other);
        basic_stacktrace(basic_stacktrace&& other) noexcept;
        basic_stacktrace(const basic_stacktrace& other, const allocator_type& alloc);
        basic_stacktrace(basic_stacktrace&& other, const allocator_type& alloc);
        basic_stacktrace& operator=(const basic_stacktrace& other);
```

```

basic_stacktrace& operator=(basic_stacktrace&& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
        allocator_traits<Allocator>::is_always_equal::value);

~basic_stacktrace();

// 19.6.4.3, observers
allocator_type get_allocator() const noexcept;

const_iterator begin() const noexcept;
const_iterator end() const noexcept;
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

const_reference operator[](size_type) const;
const_reference at(size_type) const;

// 19.6.4.4, comparisons
template<class Allocator2>
friend bool operator==(const basic_stacktrace& x,
    const basic_stacktrace<Allocator2>& y) noexcept;
template<class Allocator2>
friend strong_ordering operator<=>(const basic_stacktrace& x,
    const basic_stacktrace<Allocator2>& y) noexcept;

// 19.6.4.5, modifiers
void swap(basic_stacktrace& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
        allocator_traits<Allocator>::is_always_equal::value);

private:
    vector<value_type, allocator_type> frames_;           // exposition only
};

```

- ¹ The class template `basic_stacktrace` satisfies the requirements of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of a sequence container (23.2.4), except that
- (1.1) — only move, assignment, swap, and operations defined for const-qualified sequence containers are supported and,
 - (1.2) — the semantics of comparison functions are different from those required for a container.

19.6.4.2 Creation and assignment

[stacktrace.basic.cons]

```
static basic_stacktrace current(const allocator_type& alloc = allocator_type()) noexcept;
```

- ¹ *Returns:* A `basic_stacktrace` object with `frames_` storing the stacktrace of the current evaluation in the current thread of execution, or an empty `basic_stacktrace` object if the initialization of `frames_` failed. `alloc` is passed to the constructor of the `frames_` object.

[Note 1: If the stacktrace was successfully obtained, then `frames_.front()` is the `stacktrace_entry` representing approximately the current evaluation, and `frames_.back()` is the `stacktrace_entry` representing approximately the initial function of the current thread of execution. — end note]

```
static basic_stacktrace current(size_type skip,
                               const allocator_type& alloc = allocator_type()) noexcept;
```

2 Let *t* be a stacktrace as-if obtained via `basic_stacktrace::current(alloc)`. Let *n* be *t.size()*.

3 *Returns:* A `basic_stacktrace` object where *frames_* is direct-non-list-initialized from arguments *t.begin()* + *min(n, skip)*, *t.end()*, and *alloc*, or an empty `basic_stacktrace` object if the initialization of *frames_* failed.

```
static basic_stacktrace current(size_type skip, size_type max_depth,
                               const allocator_type& alloc = allocator_type()) noexcept;
```

4 Let *t* be a stacktrace as-if obtained via `basic_stacktrace::current(alloc)`. Let *n* be *t.size()*.

5 *Hardened preconditions:* *skip* <= *skip* + *max_depth* is true.

6 *Returns:* A `basic_stacktrace` object where *frames_* is direct-non-list-initialized from arguments *t.begin()* + *min(n, skip)*, *t.begin()* + *min(n, skip + max_depth)*, and *alloc*, or an empty `basic_stacktrace` object if the initialization of *frames_* failed.

```
basic_stacktrace() noexcept(is_nothrow_default_constructible_v<allocator_type>);
```

7 *Postconditions:* `empty()` is true.

```
explicit basic_stacktrace(const allocator_type& alloc) noexcept;
```

8 *Effects:* *alloc* is passed to the *frames_* constructor.

9 *Postconditions:* `empty()` is true.

```
basic_stacktrace(const basic_stacktrace& other);
basic_stacktrace(const basic_stacktrace& other, const allocator_type& alloc);
basic_stacktrace(basic_stacktrace&& other, const allocator_type& alloc);
basic_stacktrace& operator=(const basic_stacktrace& other);
basic_stacktrace& operator=(basic_stacktrace&& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
```

10 *Remarks:* Implementations may strengthen the exception specification for these functions (16.4.6.14) by ensuring that `empty()` is true on failed allocation.

19.6.4.3 Observers

[stacktrace.basic.obs]

```
using const_iterator = implementation-defined;
```

1 The type models `random_access_iterator` (24.3.4.13) and meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7).

```
allocator_type get_allocator() const noexcept;
```

2 *Returns:* *frames_.get_allocator()*.

```
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
```

3 *Returns:* An iterator referring to the first element in *frames_*. If `empty()` is true, then it returns the same value as `end()`.

```
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

4 *Returns:* The end iterator.

```
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator crbegin() const noexcept;
```

5 *Returns:* `reverse_iterator(cend())`.

```
const_reverse_iterator rend() const noexcept;
const_reverse_iterator crend() const noexcept;
```

6 *Returns:* `reverse_iterator(cbegin())`.


```

bool empty() const noexcept;
7     Returns: frames_.empty().

size_type size() const noexcept;
8     Returns: frames_.size().

size_type max_size() const noexcept;
9     Returns: frames_.max_size().

const_reference operator[](size_type frame_no) const;
10    Hardened preconditions: frame_no < size() is true.
11    Returns: frames_[frame_no].
12    Throws: Nothing.

const_reference at(size_type frame_no) const;
13    Returns: frames_[frame_no].
14    Throws: out_of_range if frame_no >= size().

```

19.6.4.4 Comparisons

[stacktrace.basic.cmp]

```

template<class Allocator2>
friend bool operator==(const basic_stacktrace& x, const basic_stacktrace<Allocator2>& y) noexcept;
1     Returns: equal(x.begin(), x.end(), y.begin(), y.end()).

template<class Allocator2>
friend strong_ordering
operator<=>(const basic_stacktrace& x, const basic_stacktrace<Allocator2>& y) noexcept;
2     Returns: x.size() <=> y.size() if x.size() != y.size(); lexicographical_compare_three-
            way(x.begin(), x.end(), y.begin(), y.end()) otherwise.

```

19.6.4.5 Modifiers

[stacktrace.basic.mod]

```

void swap(basic_stacktrace& other)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
1     Effects: Exchanges the contents of *this and other.

```

19.6.4.6 Non-member functions

[stacktrace.basic.nonmem]

```

template<class Allocator>
void swap(basic_stacktrace<Allocator>& a, basic_stacktrace<Allocator>& b)
noexcept(noexcept(a.swap(b)));
1     Effects: Equivalent to a.swap(b).

string to_string(const stacktrace_entry& f);
2     Returns: A string with a description of f.
3     Recommended practice: The description should provide information about the contained evaluation,
            including information from f.source_file() and f.source_line().

template<class Allocator>
string to_string(const basic_stacktrace<Allocator>& st);
4     Returns: A string with a description of st.
            [Note 1: The number of lines is not guaranteed to be equal to st.size(). — end note]

ostream& operator<<(ostream& os, const stacktrace_entry& f);
5     Effects: Equivalent to: return os << to_string(f);

```

```
template<class Allocator>
ostream& operator<<(ostream& os, const basic_stacktrace<Allocator>& st);
```

6 *Effects:* Equivalent to: `return os << to_string(st);`

19.6.5 Formatting support

[stacktrace.format]

```
template<> struct formatter<stacktrace_entry>;
```

1 `formatter<stacktrace_entry>` interprets *format-spec* as a *stacktrace-entry-format-spec*. The syntax of format specifications is as follows:

stacktrace-entry-format-spec:
fill-and-align_{opt} width_{opt}

[*Note 1:* The productions *fill-and-align* and *width* are described in 28.5.2.2. — *end note*]

2 A `stacktrace_entry` object *se* is formatted as if by copying `to_string(se)` through the output iterator of the context with additional padding and adjustments as specified by the format specifiers.

```
template<class Allocator> struct formatter<basic_stacktrace<Allocator>>;
```

3 For `formatter<basic_stacktrace<Allocator>>`, *format-spec* is empty.

4 A `basic_stacktrace<Allocator>` object *s* is formatted as if by copying `to_string(s)` through the output iterator of the context.

19.6.6 Hash support

[stacktrace.basic.hash]

```
template<> struct hash<stacktrace_entry>;
template<class Allocator> struct hash<basic_stacktrace<Allocator>>;
```

1 The specializations are enabled (22.10.19).

19.7 Debugging

[debugging]

19.7.1 General

[debugging.general]

1 Subclause 19.7 describes functionality to introspect and interact with the execution of the program.

[*Note 1:* The facilities provided by the debugging functionality interact with a program that could be tracing the execution of a C++ program, such as a debugger. — *end note*]

19.7.2 Header <debugging> synopsis

[debugging.syn]

```
// all freestanding
namespace std {
    // 19.7.3, utility
    void breakpoint() noexcept;
    void breakpoint_if_debugging() noexcept;
    bool is_debugger_present() noexcept;
}
```

19.7.3 Utility

[debugging.utility]

```
void breakpoint() noexcept;
```

1 The semantics of this function are implementation-defined.

[*Note 1:* It is intended that, when invoked with a debugger present, the execution of the program temporarily halts and execution is handed to the debugger until the program is either terminated by the debugger or the debugger resumes execution of the program as if the function was not invoked. In particular, there is no intent for a call to this function to accommodate resumption of the program in a different manner. If there is no debugger present, execution of the program can end abnormally. — *end note*]

```
void breakpoint_if_debugging() noexcept;
```

2 *Effects:* Equivalent to:

```
if (is_debugger_present()) breakpoint();
```

`bool is_debugger_present() noexcept;`

3 *Required behavior:* This function has no preconditions.

4 *Default behavior:* implementation-defined.

[*Note 2:* It is intended that, using an immediate (uncached) query to determine if the program is being traced by a debugger, an implementation returns **true** only when tracing the execution of the program with a debugger. On Windows or equivalent systems, this can be achieved by calling the `::IsDebuggerPresent()` Win32 function. For systems compatible with ISO/IEC 23360:2021, this can be achieved by checking for a tracing process, with a best-effort determination that such a tracing process is a debugger. — *end note*]

5 *Remarks:* This function is replaceable (9.6.5).

20 Memory management library [mem]

20.1 General [mem.general]

- ¹ This Clause describes components for memory management.
- ² The following subclauses describe general memory management facilities, smart pointers, pointer tagging, memory resources, and scoped allocators, as summarized in [Table 50](#).

Table 50 — Memory management library summary [tab:mem.summary]

Subclause	Header
20.2 Memory	<cstdlib>, <memory>
20.4 Smart pointers	<memory>
20.3 Pointer tagging	<memory>
20.5 Types for composite class design	<memory>
20.6 Memory resources	<memory_resource>
20.7 Scoped allocators	<scoped_allocator>

20.2 Memory [memory]

20.2.1 General [memory.general]

- ¹ Subclause [20.2](#) describes the contents of the header <memory> ([20.2.2](#)) and some of the contents of the header <cstdlib> ([17.2.2](#)).

20.2.2 Header <memory> synopsis [memory.syn]

- ¹ The header <memory> defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct objects in uninitialized memory buffers ([20.2.3–20.2.11](#) and [26.11](#)). The header also defines class template `pointer_tag_pair` to support storing additional information in unused bits of pointers. Selection of the bits is implementation-defined, and the size of the pointer wrapping object is the same as the size of the original pointer. The header also defines the templates `unique_ptr`, `shared_ptr`, `weak_ptr`, `out_ptr_t`, `inout_ptr_t`, and various function templates that operate on objects of these types ([20.4](#)).
- ² Let $POINTER_OF(T)$ denote a type that is
- (2.1) — `T::pointer` if the *qualified-id* `T::pointer` is valid and denotes a type,
- (2.2) — otherwise, `T::element_type*` if the *qualified-id* `T::element_type` is valid and denotes a type,
- (2.3) — otherwise, `pointer_traits<T>::element_type*`.
- ³ Let $POINTER_OF_OR(T, U)$ denote a type that is:
- (3.1) — $POINTER_OF(T)$ if $POINTER_OF(T)$ is valid and denotes a type,
- (3.2) — otherwise, `U`.

```
#include <compare>                                // see 17.12.1

namespace std {
    // 20.2.3, pointer traits
    template<class Ptr> struct pointer_traits;           // freestanding
    template<class T> struct pointer_traits<T*>;        // freestanding

    // 20.2.4, pointer conversion
    template<class T>
        constexpr T* to_address(T* p) noexcept;       // freestanding
    template<class Ptr>
        constexpr auto to_address(const Ptr& p) noexcept; // freestanding
```

```

// 20.2.5, pointer alignment
void* align(size_t alignment, size_t size, void*& ptr, size_t& space); // freestanding
template<size_t N, class T>
    constexpr T* assume_aligned(T* ptr); // freestanding
template<size_t Alignment, class T>
    bool is_sufficiently_aligned(T* ptr); // freestanding

// 20.2.6, explicit lifetime management
template<class T>
    constexpr void start_lifetime(T& r) noexcept; // freestanding
template<class T>
    T* start_lifetime_as(void* p) noexcept; // freestanding
template<class T>
    const T* start_lifetime_as(const void* p) noexcept; // freestanding
template<class T>
    volatile T* start_lifetime_as(volatile void* p) noexcept; // freestanding
template<class T>
    const volatile T* start_lifetime_as(const volatile void* p) noexcept; // freestanding
template<class T>
    T* start_lifetime_as_array(void* p, size_t n) noexcept; // freestanding
template<class T>
    const T* start_lifetime_as_array(const void* p, size_t n) noexcept; // freestanding
template<class T>
    volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept; // freestanding
template<class T>
    const volatile T* start_lifetime_as_array(const volatile void* p, // freestanding
                                              size_t n) noexcept;

// 20.2.7, allocator argument tag
struct allocator_arg_t { explicit allocator_arg_t() = default; }; // freestanding
inline constexpr allocator_arg_t allocator_arg{}; // freestanding

// 20.2.8, uses_allocator
template<class T, class Alloc> struct uses_allocator; // freestanding

// 20.2.8.1, uses_allocator
template<class T, class Alloc>
    constexpr bool uses_allocator_v = uses_allocator<T, Alloc>::value; // freestanding

// 20.2.8.2, uses-allocator construction
template<class T, class Alloc, class... Args>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    Args&&... args) noexcept;
template<class T, class Alloc, class Tuple1, class Tuple2>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    piecewise_construct_t,
                                                    Tuple1&& x, Tuple2&& y) noexcept;
template<class T, class Alloc>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept; // freestanding
template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    U&& u, V&& v) noexcept;
template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    pair<U, V>& pr) noexcept;
template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    const pair<U, V>& pr) noexcept;
template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    pair<U, V>&& pr) noexcept;
template<class T, class Alloc, class U, class V>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc, // freestanding
                                                    const pair<U, V>&& pr) noexcept;

```

```

template<class T, class Alloc, pair-like P>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,           // freestanding
                                                    P&& p) noexcept;

template<class T, class Alloc, class U>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,           // freestanding
                                                    U&& u) noexcept;

template<class T, class Alloc, class... Args>
    constexpr T make_obj_using_allocator(const Alloc& alloc, Args&&... args);     // freestanding
template<class T, class Alloc, class... Args>
    constexpr T* uninitialized_construct_using_allocator(T* p,                   // freestanding
                                                         const Alloc& alloc, Args&&... args);

// 20.2.9, allocator traits
template<class Alloc> struct allocator_traits;                                   // freestanding

template<class Pointer, class SizeType = size_t>
struct allocation_result {                                                       // freestanding
    Pointer ptr;
    SizeType count;
};

// 20.2.10, the default allocator
template<class T> class allocator;
template<class T, class U>
    constexpr bool operator==(const allocator<T>&, const allocator<U>&) noexcept;

// 20.2.11, addressof
template<class T>
    constexpr T* addressof(T& r) noexcept;                                       // freestanding
template<class T>
    const T* addressof(const T&&) = delete;                                       // freestanding

// 26.11, specialized algorithms
// 26.11.2, special memory concepts
template<class I>
    concept nothrow-input-iterator = see below;                               // exposition only
template<class I>
    concept nothrow-forward-iterator = see below;                             // exposition only
template<class I>
    concept nothrow-bidirectional-iterator = see below;                       // exposition only
template<class I>
    concept nothrow-random-access-iterator = see below;                       // exposition only
template<class S, class I>
    concept nothrow-sentinel-for = see below;                                   // exposition only
template<class S, class I>
    concept nothrow-sized-sentinel-for = see below;                           // exposition only
template<class R>
    concept nothrow-input-range = see below;                                   // exposition only
template<class R>
    concept nothrow-forward-range = see below;                                 // exposition only
template<class R>
    concept nothrow-bidirectional-range = see below;                           // exposition only
template<class R>
    concept nothrow-random-access-range = see below;                           // exposition only
template<class R>
    concept nothrow-sized-random-access-range = see below;                     // exposition only

template<class NoThrowForwardIterator>
    constexpr void uninitialized_default_construct(NoThrowForwardIterator first, // freestanding
                                                    NoThrowForwardIterator last);

template<class ExecutionPolicy, class NoThrowForwardIterator>
    void uninitialized_default_construct(ExecutionPolicy&& exec,                 // freestanding-deleted,
                                         NoThrowForwardIterator first,           // see 26.3.5
                                         NoThrowForwardIterator last);

```

```

template<class NoThrowForwardIterator, class Size>
constexpr NoThrowForwardIterator
    uninitialized_default_construct_n(NoThrowForwardIterator first, Size n);    // freestanding
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
NoThrowForwardIterator
    uninitialized_default_construct_n(ExecutionPolicy&& exec,                    // freestanding-deleted,
                                     NoThrowForwardIterator first,              // see 26.3.5
                                     Size n);

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_default_construct(I first, S last);            // freestanding
    template<nothrow-forward-range R>
        requires default_initializable<range_value_t<R>>
        constexpr borrowed_iterator_t<R> uninitialized_default_construct(R&& r); // freestanding

    template<nothrow-forward-iterator I>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_default_construct_n(I first,                    // freestanding
                                                      iter_difference_t<I> n);

    template<execution-policy Ep, nothrow-random-access-iterator I,
            nothrow-sized-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        I uninitialized_default_construct(Ep&& exec, I first, S last);            // freestanding-deleted,
                                                                                   // see 26.3.5
    template<execution-policy Ep, nothrow-sized-random-access-range R>
        requires default_initializable<range_value_t<R>>
        borrowed_iterator_t<R> uninitialized_default_construct(Ep&& exec,        // freestanding-deleted,
                                                                R&& r);            // see 26.3.5

    template<execution-policy Ep, nothrow-random-access-iterator I>
        requires default_initializable<iter_value_t<I>>
        I uninitialized_default_construct_n(Ep&& exec, I first,                    // freestanding-deleted,
                                             iter_difference_t<I> n);            // see 26.3.5
}

template<class NoThrowForwardIterator>
constexpr void uninitialized_value_construct(NoThrowForwardIterator first,      // freestanding
                                             NoThrowForwardIterator last);

template<class ExecutionPolicy, class NoThrowForwardIterator>
void uninitialized_value_construct(ExecutionPolicy&& exec,                        // freestanding-deleted,
                                  NoThrowForwardIterator first,                  // see 26.3.5
                                  NoThrowForwardIterator last);

template<class NoThrowForwardIterator, class Size>
constexpr NoThrowForwardIterator
    uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);        // freestanding
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
NoThrowForwardIterator
    uninitialized_value_construct_n(ExecutionPolicy&& exec,                        // freestanding-deleted,
                                    NoThrowForwardIterator first,                  // see 26.3.5
                                    Size n);

namespace ranges {
    template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
        requires default_initializable<iter_value_t<I>>
        constexpr I uninitialized_value_construct(I first, S last);              // freestanding
    template<nothrow-forward-range R>
        requires default_initializable<range_value_t<R>>
        constexpr borrowed_iterator_t<R> uninitialized_value_construct(R&& r);    // freestanding
}

```

```

template<nothrow-forward-iterator I>
    requires default_initializable<iter_value_t<I>>
    constexpr I uninitialized_value_construct_n(I first,
                                                iter_difference_t<I> n); // freestanding

template<execution-policy Ep, nothrow-random-access-iterator I,
        nothrow-sized-sentinel-for<I> S>
    requires default_initializable<iter_value_t<I>>
    I uninitialized_value_construct(Ep&& exec, I first, S last); // freestanding-deleted,
                                                                // see 26.3.5

template<execution-policy Ep, nothrow-sized-random-access-range R>
    requires default_initializable<range_value_t<R>>
    borrowed_iterator_t<R> uninitialized_value_construct(Ep&& exec,
                                                         R&& r); // freestanding-deleted,
                                                                // see 26.3.5

template<execution-policy Ep, nothrow-random-access-iterator I>
    requires default_initializable<iter_value_t<I>>
    I uninitialized_value_construct_n(Ep&& exec, I first,
                                      iter_difference_t<I> n); // freestanding-deleted,
                                                                // see 26.3.5
}

template<class InputIterator, class NoThrowForwardIterator>
    constexpr NoThrowForwardIterator uninitialized_copy(InputIterator first,
                                                         InputIterator last,
                                                         NoThrowForwardIterator result); // freestanding

template<class ExecutionPolicy, class ForwardIterator, class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy(ExecutionPolicy&& exec,
                                              ForwardIterator first,
                                              ForwardIterator last,
                                              NoThrowForwardIterator result); // freestanding-deleted,
                                                                // see 26.3.5

template<class InputIterator, class Size, class NoThrowForwardIterator>
    constexpr NoThrowForwardIterator uninitialized_copy_n(InputIterator first,
                                                         Size n,
                                                         NoThrowForwardIterator result); // freestanding

template<class ExecutionPolicy, class ForwardIterator, class Size,
        class NoThrowForwardIterator>
    NoThrowForwardIterator uninitialized_copy_n(ExecutionPolicy&& exec,
                                              ForwardIterator first,
                                              Size n,
                                              NoThrowForwardIterator result); // freestanding-deleted,
                                                                // see 26.3.5

namespace ranges {
    template<class I, class O>
        using uninitialized_copy_result = in_out_result<I, O>; // freestanding
    template<input_iterator I, sentinel_for<I> S1,
            nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
        requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
        constexpr uninitialized_copy_result<I, O>
            uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast); // freestanding
    template<input_range IR, nothrow-forward-range OR>
        requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
        constexpr uninitialized_copy_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
            uninitialized_copy(IR&& in_range, OR&& out_range); // freestanding

    template<class I, class O>
        using uninitialized_copy_n_result = in_out_result<I, O>; // freestanding
    template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
        requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
        constexpr uninitialized_copy_n_result<I, O>
            uninitialized_copy_n(I ifirst, iter_difference_t<I> n,
                                O ofirst, S olast); // freestanding

```



```

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S1,
        nothrow_random_access_iterator O, nothrow_sized_sentinel_for<O> S2>
requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
uninitialized_copy_result<I, O>
uninitialized_copy(Ep&& exec, I ifirst, S1 ilast,                                // freestanding-deleted,
                  O ofirst, S2 olast);                                         // see 26.3.5
template<execution_policy Ep, sized_random_access_range IR,
        nothrow_sized_random_access_range OR>
requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
uninitialized_copy_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
uninitialized_copy(Ep&& exec, IR&& in_range, OR&& out_range); // freestanding-deleted,
                                                             // see 26.3.5
template<execution_policy Ep, random_access_iterator I, nothrow_random_access_iterator O,
        nothrow_sized_sentinel_for<O> S>
requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
uninitialized_copy_n_result<I, O>
uninitialized_copy_n(Ep&& exec, I ifirst, iter_difference_t<I> n, // freestanding-deleted,
                    O ofirst, S olast);                             // see 26.3.5
}

template<class InputIterator, class NoThrowForwardIterator>
constexpr NoThrowForwardIterator uninitialized_move(InputIterator first,           // freestanding
                                                    InputIterator last,
                                                    NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class NoThrowForwardIterator>
NoThrowForwardIterator uninitialized_move(ExecutionPolicy&& exec, // freestanding-deleted,
                                          ForwardIterator first, // see 26.3.5
                                          ForwardIterator last,
                                          NoThrowForwardIterator result);
template<class InputIterator, class Size, class NoThrowForwardIterator>
constexpr pair<InputIterator, NoThrowForwardIterator>
uninitialized_move_n(InputIterator first, Size n, // freestanding
                    NoThrowForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class NoThrowForwardIterator>
pair<ForwardIterator, NoThrowForwardIterator>
uninitialized_move_n(ExecutionPolicy&& exec, // freestanding-deleted,
                    ForwardIterator first, Size n, // see 26.3.5
                    NoThrowForwardIterator result);

namespace ranges {
template<class I, class O>
using uninitialized_move_result = in_out_result<I, O>; // freestanding
template<input_iterator I, sentinel_for<I> S1,
        nothrow_forward_iterator O, nothrow_sentinel_for<O> S2>
requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
constexpr uninitialized_move_result<I, O>
uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast); // freestanding
template<input_range IR, nothrow_forward_range OR>
requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
constexpr uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
uninitialized_move(IR&& in_range, OR&& out_range); // freestanding

template<class I, class O>
using uninitialized_move_n_result = in_out_result<I, O>; // freestanding
template<input_iterator I,
        nothrow_forward_iterator O, nothrow_sentinel_for<O> S>
requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
constexpr uninitialized_move_n_result<I, O>
uninitialized_move_n(I ifirst, iter_difference_t<I> n, // freestanding
                    O ofirst, S olast);

```

```

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S1,
        nothrow_random_access_iterator O, nothrow_sized_sentinel_for<O> S2>
requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
uninitialized_move_result<I, O>
    uninitialized_move(Ep&& exec, I ifirst, S1 ilast,                // freestanding-deleted,
                      O ofirst, S2 olast);                          // see 26.3.5
template<execution_policy Ep, sized_random_access_range IR,
        nothrow_sized_random_access_range OR>
requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
    uninitialized_move(Ep&& exec, IR&& in_range, OR&& out_range);    // freestanding-deleted,
                                                                    // see 26.3.5

template<execution_policy Ep, random_access_iterator I,
        nothrow_random_access_iterator O, nothrow_sized_sentinel_for<O> S>
requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
uninitialized_move_n_result<I, O>
    uninitialized_move_n(Ep&& exec, I ifirst, iter_difference_t<I> n, // freestanding-deleted,
                        O ofirst, S olast);                          // see 26.3.5
}

template<class NoThrowForwardIterator,
        class T = iterator_traits<NoThrowForwardIterator>::value_type>
constexpr void uninitialized_fill(NoThrowForwardIterator first,    // freestanding
                                NoThrowForwardIterator last, const T& x);
template<class ExecutionPolicy, class NoThrowForwardIterator,
        class T = iterator_traits<NoThrowForwardIterator>::value_type>
void uninitialized_fill(ExecutionPolicy&& exec,                    // freestanding-deleted,
                        NoThrowForwardIterator first,              // see 26.3.5
                        NoThrowForwardIterator last,
                        const T& x);
template<class NoThrowForwardIterator, class Size,
        class T = iterator_traits<NoThrowForwardIterator>::value_type>
constexpr NoThrowForwardIterator
    uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x); // freestanding
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size,
        class T = iterator_traits<NoThrowForwardIterator>::value_type>
NoThrowForwardIterator
    uninitialized_fill_n(ExecutionPolicy&& exec,                    // freestanding-deleted,
                        NoThrowForwardIterator first,              // see 26.3.5
                        Size n, const T& x);

namespace ranges {
    template<nothrow_forward_iterator I, nothrow_sentinel_for<I> S, class T = iter_value_t<I>>
    requires constructible_from<iter_value_t<I>, const T&>
    constexpr I uninitialized_fill(I first, S last, const T& x);    // freestanding
    template<nothrow_forward_range R, class T = range_value_t<R>>
    requires constructible_from<range_value_t<R>, const T&>
    constexpr borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x); // freestanding

    template<nothrow_forward_iterator I, class T = iter_value_t<I>>
    requires constructible_from<iter_value_t<I>, const T&>
    constexpr I uninitialized_fill_n(I first,                        // freestanding
                                    iter_difference_t<I> n, const T& x);

    template<execution_policy Ep, nothrow_random_access_iterator I,
            nothrow_sized_sentinel_for<I> S, class T = iter_value_t<I>>
    requires constructible_from<iter_value_t<I>, const T&>
    I uninitialized_fill(Ep&& exec, I first, S last, const T& x);    // freestanding-deleted,
                                                                    // see 26.3.5

    template<execution_policy Ep, nothrow_sized_random_access_range R,
            class T = range_value_t<R>>
    requires constructible_from<range_value_t<R>, const T&>
    borrowed_iterator_t<R> uninitialized_fill(Ep&& exec, R&& r,      // freestanding-deleted,

```

```

        const T& x); // see 26.3.5

template<execution-policy Ep, nothrow-random-access-iterator I, class T = iter_value_t<I>>
    requires constructible_from<iter_value_t<I>, const T&>
    I uninitialized_fill_n(Ep&& exec, I first, // freestanding-deleted,
        iter_difference_t<I> n, const T& x); // see 26.3.5
}

// 26.11.8, construct_at
template<class T, class... Args>
    constexpr T* construct_at(T* location, Args&&... args); // freestanding

namespace ranges {
    template<class T, class... Args>
        constexpr T* construct_at(T* location, Args&&... args); // freestanding
}

// 26.11.9, destroy
template<class T>
    constexpr void destroy_at(T* location); // freestanding
template<class NoThrowForwardIterator>
    constexpr void destroy(NoThrowForwardIterator first, // freestanding
        NoThrowForwardIterator last);
template<class ExecutionPolicy, class NoThrowForwardIterator>
    void destroy(ExecutionPolicy&& exec, // freestanding-deleted,
        NoThrowForwardIterator first, // see 26.3.5
        NoThrowForwardIterator last);
template<class NoThrowForwardIterator, class Size>
    constexpr NoThrowForwardIterator destroy_n(NoThrowForwardIterator first, // freestanding
        Size n);
template<class ExecutionPolicy, class NoThrowForwardIterator, class Size>
    NoThrowForwardIterator destroy_n(ExecutionPolicy&& exec, // freestanding-deleted,
        NoThrowForwardIterator first, Size n); // see 26.3.5

namespace ranges {
    template<destructible T>
        constexpr void destroy_at(T* location) noexcept; // freestanding

    template<nothrow-input-iterator I, nothrow-sentinel-for<I> S>
        requires destructible<iter_value_t<I>>
        constexpr I destroy(I first, S last) noexcept; // freestanding
    template<nothrow-input-range R>
        requires destructible<range_value_t<R>>
        constexpr borrowed_iterator_t<R> destroy(R&& r) noexcept; // freestanding

    template<nothrow-input-iterator I>
        requires destructible<iter_value_t<I>>
        constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept; // freestanding

    template<execution-policy Ep, nothrow-random-access-iterator I,
        nothrow-sized-sentinel-for<I> S>
        requires destructible<iter_value_t<I>>
        I destroy(Ep&& exec, I first, S last); // freestanding-deleted,
        // see 26.3.5

    template<execution-policy Ep, nothrow-sized-random-access-range R>
        requires destructible<range_value_t<R>>
        borrowed_iterator_t<R> destroy(Ep&& exec, R&& r); // freestanding-deleted,
        // see 26.3.5

    template<execution-policy Ep, nothrow-random-access-iterator I>
        requires destructible<iter_value_t<I>>
        I destroy_n(Ep&& exec, I first, iter_difference_t<I> n); // freestanding-deleted,
        // see 26.3.5
}

```

```

// 20.3, pointer tagging
inline constexpr unsigned max_pointer_bits_available = see below;           // freestanding
constexpr unsigned pointer_bits_available(size_t alignment);                 // freestanding

template<class T>
    constexpr unsigned bits-available = pointer_bits_available(alignedof(T)); // exposition only

template<class U> using element-of = pointer_traits<U>::element_type;         // exposition only

template<class Ptr, unsigned BitsRequested = bits-available<element-of<Ptr>>,
        class TagT = unsigned>
    class pointer_tag_pair;                                                  // freestanding

template<class Ptr, unsigned BitsRequested, class TagT>
    struct tuple_size<pointer_tag_pair<Ptr, BitsRequested, TagT>>           // freestanding
        : integral_constant<size_t, 2> { };

template<class Ptr, unsigned BitsRequested, class TagT>
    struct tuple_element<0, pointer_tag_pair<Ptr, BitsRequested, TagT>> {   // freestanding
        using type = Ptr;
    };

template<class Ptr, unsigned BitsRequested, class TagT>
    struct tuple_element<1, pointer_tag_pair<Ptr, BitsRequested, TagT>> {   // freestanding
        using type = TagT;
    };

template<class Ptr, unsigned BitsRequested, class TagT>
    struct tuple_element<0, const pointer_tag_pair<Ptr, BitsRequested, TagT>> { // freestanding
        using type = Ptr;
    };

template<class Ptr, unsigned BitsRequested, class TagT>
    struct tuple_element<1, const pointer_tag_pair<Ptr, BitsRequested, TagT>> { // freestanding
        using type = TagT;
    };

template<class Ptr, unsigned BitsRequested, class TagT>
    constexpr tuple_element_t<I, pointer_tag_pair<Ptr, BitsRequested, TagT>> // freestanding
        get(pointer_tag_pair<Ptr, BitsRequested, TagT> p) noexcept;

// 20.4.1, class template unique_ptr
template<class T> struct default_delete;                                   // freestanding
template<class T> struct default_delete<T[]>;                             // freestanding
template<class T, class D = default_delete<T>> class unique_ptr;           // freestanding
template<class T, class D> class unique_ptr<T[], D>;                       // freestanding

template<class T, class... Args>
    constexpr unique_ptr<T> make_unique(Args&&... args);                   // T is not array
template<class T>
    constexpr unique_ptr<T> make_unique(size_t n);                         // T is U[]
template<class T, class... Args>
    unspecified make_unique(Args&&...) = delete;                           // T is U[N]

template<class T>
    constexpr unique_ptr<T> make_unique_for_overwrite();                   // T is not array
template<class T>
    constexpr unique_ptr<T> make_unique_for_overwrite(size_t n);           // T is U[]
template<class T, class... Args>
    unspecified make_unique_for_overwrite(Args&&...) = delete;             // T is U[N]

template<class T, class D>
    constexpr void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;   // freestanding

```

```

template<class T1, class D1, class T2, class D2>
    constexpr bool operator==(const unique_ptr<T1, D1>& x,
                              const unique_ptr<T2, D2>& y); // freestanding
template<class T1, class D1, class T2, class D2>
    constexpr bool operator<(const unique_ptr<T1, D1>& x,
                             const unique_ptr<T2, D2>& y); // freestanding
template<class T1, class D1, class T2, class D2>
    constexpr bool operator>(const unique_ptr<T1, D1>& x,
                             const unique_ptr<T2, D2>& y); // freestanding
template<class T1, class D1, class T2, class D2>
    constexpr bool operator<=(const unique_ptr<T1, D1>& x,
                              const unique_ptr<T2, D2>& y); // freestanding
template<class T1, class D1, class T2, class D2>
    constexpr bool operator>=(const unique_ptr<T1, D1>& x,
                              const unique_ptr<T2, D2>& y); // freestanding
template<class T1, class D1, class T2, class D2>
    requires three_way_comparable_with<typename unique_ptr<T1, D1>::pointer,
                                       typename unique_ptr<T2, D2>::pointer>
    constexpr compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer,
                                       typename unique_ptr<T2, D2>::pointer>
    operator<=>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y); // freestanding

template<class T, class D>
    constexpr bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept; // freestanding
template<class T, class D>
    constexpr bool operator<(const unique_ptr<T, D>& x, nullptr_t); // freestanding
template<class T, class D>
    constexpr bool operator<(nullptr_t, const unique_ptr<T, D>& y); // freestanding
template<class T, class D>
    constexpr bool operator>(const unique_ptr<T, D>& x, nullptr_t); // freestanding
template<class T, class D>
    constexpr bool operator>(nullptr_t, const unique_ptr<T, D>& y); // freestanding
template<class T, class D>
    constexpr bool operator<=(const unique_ptr<T, D>& x, nullptr_t); // freestanding
template<class T, class D>
    constexpr bool operator<=(nullptr_t, const unique_ptr<T, D>& y); // freestanding
template<class T, class D>
    constexpr bool operator>=(const unique_ptr<T, D>& x, nullptr_t); // freestanding
template<class T, class D>
    constexpr bool operator>=(nullptr_t, const unique_ptr<T, D>& y); // freestanding
template<class T, class D>
    requires three_way_comparable<typename unique_ptr<T, D>::pointer>
    constexpr compare_three_way_result_t<typename unique_ptr<T, D>::pointer>
    operator<=>(const unique_ptr<T, D>& x, nullptr_t); // freestanding

template<class E, class T, class Y, class D>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);

// 20.4.2.1, class bad_weak_ptr
class bad_weak_ptr;

// 20.4.2.2, class template shared_ptr
template<class T> class shared_ptr;

// 20.4.2.2.7, shared_ptr creation
template<class T, class... Args>
    constexpr shared_ptr<T> make_shared(Args&&... args); // T is not array
template<class T, class A, class... Args>
    constexpr shared_ptr<T> allocate_shared(const A& a, Args&&... args); // T is not array

template<class T>
    constexpr shared_ptr<T> make_shared(size_t N); // T is U[]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a, size_t N); // T is U[]

```

```

template<class T>
    constexpr shared_ptr<T> make_shared(); // T is U[N]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a); // T is U[N]

template<class T>
    constexpr shared_ptr<T> make_shared(size_t N, const remove_extent_t<T>& u); // T is U[]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a, size_t N,
                                             const remove_extent_t<T>& u); // T is U[]

template<class T>
    constexpr shared_ptr<T> make_shared(const remove_extent_t<T>& u); // T is U[N]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a,
                                             const remove_extent_t<T>& u); // T is U[N]

template<class T>
    constexpr shared_ptr<T> make_shared_for_overwrite(); // T is not U[]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a); // T is not U[]

template<class T>
    constexpr shared_ptr<T> make_shared_for_overwrite(size_t N); // T is U[]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a, size_t N); // T is U[]

// 20.4.2.2.8, shared_ptr comparisons
template<class T, class U>
    constexpr bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
    constexpr strong_ordering operator<=>(const shared_ptr<T>& a,
                                           const shared_ptr<U>& b) noexcept;

template<class T>
    constexpr bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template<class T>
    constexpr strong_ordering operator<=>(const shared_ptr<T>& x, nullptr_t) noexcept;

// 20.4.2.2.9, shared_ptr specialized algorithms
template<class T>
    constexpr void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

// 20.4.2.2.10, shared_ptr casts
template<class T, class U>
    constexpr shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    constexpr shared_ptr<T> static_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    constexpr shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    constexpr shared_ptr<T> dynamic_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    constexpr shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    constexpr shared_ptr<T> const_pointer_cast(shared_ptr<U>&& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
    shared_ptr<T> reinterpret_pointer_cast(shared_ptr<U>&& r) noexcept;

// 20.4.2.2.11, shared_ptr get_deleter
template<class D, class T>
    constexpr D* get_deleter(const shared_ptr<T>& p) noexcept;

```

```

// 20.4.2.2.12, shared_ptr I/O
template<class E, class T, class Y>
    basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const shared_ptr<Y>& p);

// 20.4.2.3, class template weak_ptr
template<class T> class weak_ptr;

// 20.4.2.3.7, weak_ptr specialized algorithms
template<class T> constexpr void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;

// 20.4.2.4, class template owner_less
template<class T = void> struct owner_less;

// 20.4.2.5, struct owner_hash
struct owner_hash;

// 20.4.2.6, struct owner_equal
struct owner_equal;

// 20.4.2.7, class template enable_shared_from_this
template<class T> class enable_shared_from_this;

// 20.4.3, hash support
template<class T> struct hash; // freestanding
template<class T, class D> struct hash<unique_ptr<T, D>>; // freestanding
template<class T> struct hash<shared_ptr<T>>;

// 32.5.8.7, atomic smart pointers
template<class T> struct atomic; // freestanding
template<class T> struct atomic<shared_ptr<T>>;
template<class T> struct atomic<weak_ptr<T>>;

// 20.4.4.1, class template out_ptr_t
template<class Smart, class Pointer, class... Args>
    class out_ptr_t; // freestanding

// 20.4.4.2, function template out_ptr
template<class Pointer = void, class Smart, class... Args>
    constexpr auto out_ptr(Smart& s, Args&&... args); // freestanding

// 20.4.4.3, class template inout_ptr_t
template<class Smart, class Pointer, class... Args>
    class inout_ptr_t; // freestanding

// 20.4.4.4, function template inout_ptr
template<class Pointer = void, class Smart, class... Args>
    constexpr auto inout_ptr(Smart& s, Args&&... args); // freestanding

// 20.5.1, class template indirect
template<class T, class Allocator = allocator<T>>
    class indirect;

// 20.5.1.10, hash support
template<class T, class Alloc> struct hash<indirect<T, Alloc>>;

// 20.5.2, class template polymorphic
template<class T, class Allocator = allocator<T>>
    class polymorphic;

namespace pmr {
    template<class T> using indirect      = indirect<T, polymorphic_allocator<T>>;
    template<class T> using polymorphic = polymorphic<T, polymorphic_allocator<T>>;
}
}

```


20.2.3 Pointer traits**[pointer.traits]****20.2.3.1 General****[pointer.traits.general]**

- ¹ The class template `pointer_traits` supplies a uniform interface to certain attributes of pointer-like types.

```
namespace std {
    template<class Ptr> struct pointer_traits {
        see below;
    };

    template<class T> struct pointer_traits<T*> {
        using pointer          = T*;
        using element_type     = T;
        using difference_type  = ptrdiff_t;

        template<class U> using rebind = U*;

        static constexpr pointer pointer_to(see below r) noexcept;
    };
}
```

20.2.3.2 Member types**[pointer.traits.types]**

- ¹ The definitions in this subclause make use of the following exposition-only class template and concept:

```
template<class T>
struct ptr-traits-elem           // exposition only
{ };

template<class T> requires requires { typename T::element_type; }
struct ptr-traits-elem<T>
{ using type = T::element_type; };

template<template<class...> class SomePointer, class T, class... Args>
    requires (!requires { typename SomePointer<T, Args...>::element_type; })
struct ptr-traits-elem<SomePointer<T, Args...>>
{ using type = T; };

template<class Ptr>
concept has-elm-type =           // exposition only
    requires { typename ptr-traits-elem<Ptr>::type; }
```

- ² If `Ptr` satisfies *has-elm-type*, a specialization `pointer_traits<Ptr>` generated from the `pointer_traits` primary template has the following members as well as those described in 20.2.3.3; otherwise, such a specialization has no members by any of those names.

```
using pointer = see below;
```

- ³ *Type*: `Ptr`.

```
using element_type = see below;
```

- ⁴ *Type*: `typename ptr-traits-elem<Ptr>::type`.

```
using difference_type = see below;
```

- ⁵ *Type*: `Ptr::difference_type` if the *qualified-id* `Ptr::difference_type` is valid and denotes a type (13.10.3); otherwise, `ptrdiff_t`.

```
template<class U> using rebind = see below;
```

- ⁶ *Alias template*: `Ptr::rebind<U>` if the *qualified-id* `Ptr::rebind<U>` is valid and denotes a type (13.10.3); otherwise, `SomePointer<U, Args>` if `Ptr` is a class template specialization of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the instantiation of `rebind` is ill-formed.

20.2.3.3 Member functions**[pointer.traits.functions]**

```
static constexpr pointer pointer_traits::pointer_to(see below r);
```



```
static constexpr pointer pointer_traits<T*>::pointer_to(see below r) noexcept;
```

1 *Mandates:* For the first member function, `Ptr::pointer_to(r)` is well-formed.

2 *Preconditions:* For the first member function, `Ptr::pointer_to(r)` returns a pointer to `r` through which indirection is valid.

3 *Returns:* The first member function returns `Ptr::pointer_to(r)`. The second member function returns `addressof(r)`.

4 *Remarks:* If `element_type` is *cv void*, the type of `r` is unspecified; otherwise, it is `element_type&`.

20.2.3.4 Optional members

[pointer.traits.optmem]

1 Specializations of `pointer_traits` may define the member declared in this subclause to customize the behavior of the standard library. A specialization generated from the `pointer_traits` primary template has no member by this name.

```
static element_type* to_address(pointer p) noexcept;
```

2 *Returns:* A pointer of type `element_type*` that references the same location as the argument `p`.

3 [Note 1: This function is intended to be the inverse of `pointer_to`. If defined, it customizes the behavior of the non-member function `to_address` (20.2.4). — end note]

20.2.4 Pointer conversion

[pointer.conversion]

```
template<class T> constexpr T* to_address(T* p) noexcept;
```

1 *Mandates:* `T` is not a function type.

2 *Returns:* `p`.

```
template<class Ptr> constexpr auto to_address(const Ptr& p) noexcept;
```

3 *Returns:* `pointer_traits<Ptr>::to_address(p)` if that expression is well-formed (see 20.2.3.4), otherwise `to_address(p.operator->())`.

20.2.5 Pointer alignment

[ptr.align]

```
void* align(size_t alignment, size_t size, void*& ptr, size_t& space);
```

1 *Preconditions:*

(1.1) — `alignment` is a power of two

(1.2) — `ptr` represents the address of contiguous storage of at least `space` bytes

2 *Effects:* If it is possible to fit `size` bytes of storage aligned by `alignment` into the buffer pointed to by `ptr` with length `space`, the function updates `ptr` to represent the first possible address of such storage and decreases `space` by the number of bytes used for alignment. Otherwise, the function does nothing.

3 *Returns:* A null pointer if the requested aligned buffer would not fit into the available space, otherwise the adjusted value of `ptr`.

4 [Note 1: The function updates its `ptr` and `space` arguments so that it can be called repeatedly with possibly different `alignment` and `size` arguments for the same buffer. — end note]

```
template<size_t N, class T>
constexpr T* assume_aligned(T* ptr);
```

5 *Mandates:* `N` is a power of two.

6 *Preconditions:* `ptr` points to an object `X` of a type similar (7.3.6) to `T`, where `X` has alignment `N` (6.8.3).

7 *Returns:* `ptr`.

8 *Throws:* Nothing.

9 [Note 2: The alignment assumption on an object `X` expressed by a call to `assume_aligned` might result in generation of more efficient code. It is up to the program to ensure that the assumption actually holds. The call does not cause the implementation to verify or enforce this. An implementation might only make the assumption for those operations on `X` that access `X` through the pointer returned by `assume_aligned`. — end note]

```
template<size_t Alignment, class T>
bool is_sufficiently_aligned(T* ptr);
```

10 *Mandates:* Alignment is a power of two.

11 *Preconditions:* p points to an object X of a type similar (7.3.6) to T.

12 *Returns:* true if X has alignment at least Alignment, otherwise false.

13 *Throws:* Nothing.

20.2.6 Explicit lifetime management

[obj.lifetime]

```
template<class T>
constexpr void start_lifetime(T& r) noexcept;
```

1 *Mandates:* T is a complete type and an implicit-lifetime (6.9) aggregate (9.5.2) type.

2 *Effects:* If the object referenced by r is already within its lifetime (6.8.4), no effects. Otherwise, begins the lifetime of the object referenced by r.

[Note 1: No initialization is performed and no subobject has its lifetime started. If r denotes a member of a union U, it becomes the active member of U (11.5). — end note]

```
template<class T>
T* start_lifetime_as(void* p) noexcept;
template<class T>
const T* start_lifetime_as(const void* p) noexcept;
template<class T>
volatile T* start_lifetime_as(volatile void* p) noexcept;
template<class T>
const volatile T* start_lifetime_as(const volatile void* p) noexcept;
```

3 *Mandates:* T is an implicit-lifetime type (6.9.1) and not an incomplete type (6.9.1).

4 *Preconditions:* [p, (char*)p + sizeof(T)) denotes a region of allocated storage that is a subset of the region of storage reachable through (6.9.5) p and suitably aligned for the type T.

5 *Effects:* Implicitly creates objects (6.8.2) within the denoted region consisting of an object a of type T whose address is p, and objects nested within a, as follows: The object representation of a is the contents of the storage prior to the call to start_lifetime_as. The value of each created object o of trivially copyable type (6.9.1) U is determined in the same manner as for a call to bit_cast<U>(E) (22.11.3), where E is an lvalue of type U denoting o, except that the storage is not accessed. The value of any other created object is unspecified.

[Note 2: The unspecified value can be indeterminate. — end note]

6 *Returns:* A pointer to the a defined in the Effects paragraph.

```
template<class T>
T* start_lifetime_as_array(void* p, size_t n) noexcept;
template<class T>
const T* start_lifetime_as_array(const void* p, size_t n) noexcept;
template<class T>
volatile T* start_lifetime_as_array(volatile void* p, size_t n) noexcept;
template<class T>
const volatile T* start_lifetime_as_array(const volatile void* p, size_t n) noexcept;
```

7 *Mandates:* T is a complete type.

8 *Preconditions:* p is suitably aligned for an array of T or is null. n <= size_t(-1) / sizeof(T) is true. If n > 0 is true, [(char*)p, (char*)p + (n * sizeof(T))] denotes a region of allocated storage that is a subset of the region of storage reachable through (6.9.5) p.

9 *Effects:* If n > 0 is true, equivalent to start_lifetime_as<U>(p) where U is the type “array of n T”. Otherwise, there are no effects.

10 *Returns:* A pointer to the first element of the created array, if any; otherwise, a pointer that compares equal to p (7.6.10).

20.2.7 Allocator argument tag

[allocator.tag]

```
namespace std {
    struct allocator_arg_t { explicit allocator_arg_t() = default; };
    inline constexpr allocator_arg_t allocator_arg{};
}
```

- ¹ The `allocator_arg_t` struct is an empty class type used as a unique type to disambiguate constructor and function overloading. Specifically, several types (see [tuple 22.4](#)) have constructors with `allocator_arg_t` as the first argument, immediately followed by an argument of a type that meets the *Cpp17Allocator* requirements ([16.4.4.6.1](#)).

20.2.8 uses_allocator

[allocator.uses]

20.2.8.1 uses_allocator trait

[allocator.uses.trait]

```
template<class T, class Alloc> struct uses_allocator;
```

- ¹ *Remarks:* Automatically detects whether `T` has a nested `allocator_type` that is convertible from `Alloc`. Meets the *Cpp17BinaryTypeTrait* requirements ([21.3.2](#)). The implementation shall provide a definition that is derived from `true_type` if the *qualified-id* `T::allocator_type` is valid and denotes a type ([13.10.3](#)) and `is_convertible_v<Alloc, T::allocator_type> != false`, otherwise it shall be derived from `false_type`. A program may specialize this template to derive from `true_type` for a program-defined type `T` that does not have a nested `allocator_type` but nonetheless can be constructed with an allocator where either:
- (1.1) — the first argument of a constructor has type `allocator_arg_t` and the second argument has type `Alloc` or
 - (1.2) — the last argument of a constructor has type `Alloc`.

20.2.8.2 Uses-allocator construction

[allocator.uses.construction]

- ¹ *Uses-allocator construction* with allocator `alloc` and constructor arguments `args...` refers to the construction of an object of type `T` such that `alloc` is passed to the constructor of `T` if `T` uses an allocator type compatible with `alloc`. When applied to the construction of an object of type `T`, it is equivalent to initializing it with the value of the expression `make_obj_using_allocator<T>(alloc, args...)`, described below.
- ² The following utility functions support three conventions for passing `alloc` to a constructor:
- (2.1) — If `T` does not use an allocator compatible with `alloc`, then `alloc` is ignored.
 - (2.2) — Otherwise, if `T` has a constructor invocable as `T(allocator_arg_t{}, alloc, args...)` (leading-allocator convention), then uses-allocator construction chooses this constructor form.
 - (2.3) — Otherwise, if `T` has a constructor invocable as `T(args..., alloc)` (trailing-allocator convention), then uses-allocator construction chooses this constructor form.
- ³ The `uses_allocator_construction_args` function template takes an allocator and argument list and produces (as a tuple) a new argument list matching one of the above conventions. Additionally, overloads are provided that treat specializations of `pair` such that uses-allocator construction is applied individually to the `first` and `second` data members. The `make_obj_using_allocator` and `uninitialized_construct_using_allocator` function templates apply the modified constructor arguments to construct an object of type `T` as a return value or in-place, respectively.

[Note 1: For `uses_allocator_construction_args` and `make_obj_using_allocator`, type `T` is not deduced and must therefore be specified explicitly by the caller. — end note]

```
template<class T, class Alloc, class... Args>
    constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                    Args&&... args) noexcept;
```

- ⁴ *Constraints:* `remove_cv_t<T>` is not a specialization of `pair`.
- ⁵ *Returns:* A tuple value determined as follows:
- (5.1) — If `uses_allocator_v<remove_cv_t<T>, Alloc>` is `false` and `is_constructible_v<T, Args...>` is `true`, return `forward_as_tuple(std::forward<Args>(args)...)...`.
 - (5.2) — Otherwise, if `uses_allocator_v<remove_cv_t<T>, Alloc>` is `true` and `is_constructible_v<T, allocator_arg_t, const Alloc&, Args...>` is `true`, return

```
tuple<allocator_arg_t, const Alloc&, Args&&...>(
    allocator_arg, alloc, std::forward<Args>(args)...)

```

- (5.3) — Otherwise, if `uses_allocator_v<remove_cv_t<T>, Alloc>` is true and `is_constructible_v<T, Args..., const Alloc&>` is true, return `forward_as_tuple(std::forward<Args>(args)... , alloc)`.

- (5.4) — Otherwise, the program is ill-formed.

[*Note 2:* This definition prevents a silent failure to pass the allocator to a constructor of a type for which `uses_allocator_v<T, Alloc>` is true. — *end note*]

```
template<class T, class Alloc, class Tuple1, class Tuple2>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, piecewise_construct_t,
    Tuple1&& x, Tuple2&& y) noexcept;

```

6 Let T1 be `T::first_type`. Let T2 be `T::second_type`.

7 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`.

8 *Effects:* Equivalent to:

```
return make_tuple(
    piecewise_construct,
    apply([&alloc](auto&&... args1) {
        return uses_allocator_construction_args<T1>(
            alloc, std::forward<decltype(args1)>(args1)...);
    }, std::forward<Tuple1>(x)),
    apply([&alloc](auto&&... args2) {
        return uses_allocator_construction_args<T2>(
            alloc, std::forward<decltype(args2)>(args2)...);
    }, std::forward<Tuple2>(y)));

```

```
template<class T, class Alloc>
constexpr auto uses_allocator_construction_args(const Alloc& alloc) noexcept;

```

9 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`.

10 *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
    tuple<>{}, tuple<>{});

```

```
template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
    U&& u, V&& v) noexcept;

```

11 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`.

12 *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
    forward_as_tuple(std::forward<U>(u)),
    forward_as_tuple(std::forward<V>(v)));

```

```
template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
    pair<U, V>& pr) noexcept;

```

```
template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
    const pair<U, V>& pr) noexcept;

```

13 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`.

14 *Effects:* Equivalent to:

```
return uses_allocator_construction_args<T>(alloc, piecewise_construct,
    forward_as_tuple(pr.first),
    forward_as_tuple(pr.second));

```

```
template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
    pair<U, V>&& pr) noexcept;

```

```
template<class T, class Alloc, class U, class V>
constexpr auto uses_allocator_construction_args(const Alloc& alloc,
                                                const pair<U, V>&& pr) noexcept;
```

15 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`.

16 *Effects:* Equivalent to:

```
    return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                                forward_as_tuple(get<0>(std::move(pr))),
                                                forward_as_tuple(get<1>(std::move(pr))));
```

```
template<class T, class Alloc, pair-like P>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, P&& p) noexcept;
```

17 *Constraints:* `remove_cv_t<T>` is a specialization of `pair` and `remove_cvref_t<P>` is not a specialization of `ranges::subrange`.

18 *Effects:* Equivalent to:

```
    return uses_allocator_construction_args<T>(alloc, piecewise_construct,
                                                forward_as_tuple(get<0>(std::forward<P>(p))),
                                                forward_as_tuple(get<1>(std::forward<P>(p))));
```

```
template<class T, class Alloc, class U>
constexpr auto uses_allocator_construction_args(const Alloc& alloc, U&& u) noexcept;
```

19 Let *FUN* be the function template:

```
    template<class A, class B>
    void FUN(const pair<A, B>&);
```

20 *Constraints:* `remove_cv_t<T>` is a specialization of `pair`, and either:

(20.1) — `remove_cvref_t<U>` is a specialization of `ranges::subrange`, or

(20.2) — `U` does not satisfy *pair-like* and the expression *FUN*(`u`) is not well-formed when considered as an unevaluated operand.

21 Let *pair-constructor* be an exposition-only class defined as follows:

```
class pair-constructor {
    using pair-type = remove_cv_t<T>;                                // exposition only

    constexpr auto do-construct(const pair-type& p) const {          // exposition only
        return make_obj_using_allocator<pair-type>(alloc_, p);
    }
    constexpr auto do-construct(pair-type&& p) const {               // exposition only
        return make_obj_using_allocator<pair-type>(alloc_, std::move(p));
    }

    const Alloc& alloc_;      // exposition only
    U& u_;                    // exposition only

public:
    constexpr operator pair-type() const {
        return do-construct(std::forward<U>(u_));
    }
};
```

22 *Returns:* `make_tuple(pc)`, where `pc` is a *pair-constructor* object whose `alloc_` member is initialized with `alloc` and whose `u_` member is initialized with `u`.

```
template<class T, class Alloc, class... Args>
constexpr T make_obj_using_allocator(const Alloc& alloc, Args&&... args);
```

23 *Effects:* Equivalent to:

```
    return make_from_tuple<T>(uses_allocator_construction_args<T>(
        alloc, std::forward<Args>(args)...));
```

```
template<class T, class Alloc, class... Args>
constexpr T* uninitialized_construct_using_allocator(T* p, const Alloc& alloc, Args&&... args);
```

24 *Effects:* Equivalent to:

```
return apply([&<class... U>(U&&... xs) {
    return construct_at(p, std::forward<U>(xs)...);
}, uses_allocator_construction_args<T>(alloc, std::forward<Args>(args)...));
```

20.2.9 Allocator traits

[allocator.traits]

20.2.9.1 General

[allocator.traits.general]

- ¹ The class template `allocator_traits` supplies a uniform interface to all allocator types. An allocator cannot be a non-class type, however, even if `allocator_traits` supplies the entire required interface.

[*Note 1:* Thus, it is always possible to create a derived class from an allocator. — *end note*]

If a program declares an explicit or partial specialization of `allocator_traits`, the program is ill-formed, no diagnostic required.

```
namespace std {
    template<class Alloc> struct allocator_traits {
        using allocator_type      = Alloc;

        using value_type          = Alloc::value_type;

        using pointer              = see below;
        using const_pointer        = see below;
        using void_pointer         = see below;
        using const_void_pointer   = see below;

        using difference_type      = see below;
        using size_type            = see below;

        using propagate_on_container_copy_assignment = see below;
        using propagate_on_container_move_assignment = see below;
        using propagate_on_container_swap            = see below;
        using is_always_equal                        = see below;

        template<class T> using rebind_alloc = see below;
        template<class T> using rebind_traits = allocator_traits<rebind_alloc<T>>;

        static constexpr pointer allocate(Alloc& a, size_type n);
        static constexpr pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
        static constexpr allocation_result<pointer, size_type>
            allocate_at_least(Alloc& a, size_type n);

        static constexpr void deallocate(Alloc& a, pointer p, size_type n);

        template<class T, class... Args>
            static constexpr void construct(Alloc& a, T* p, Args&&... args);

        template<class T>
            static constexpr void destroy(Alloc& a, T* p);

        static constexpr size_type max_size(const Alloc& a) noexcept;

        static constexpr Alloc select_on_container_copy_construction(const Alloc& rhs);
    };
}
```

20.2.9.2 Member types

[allocator.traits.types]

using `pointer` = *see below*;

- ¹ *Type:* `Alloc::pointer` if the *qualified-id* `Alloc::pointer` is valid and denotes a type (13.10.3); otherwise, `value_type*`.

```

using const_pointer = see below;
2   Type: Alloc::const_pointer if the qualified-id Alloc::const_pointer is valid and denotes a type
    (13.10.3); otherwise, pointer_traits<pointer>::rebind<const value_type>.

using void_pointer = see below;
3   Type: Alloc::void_pointer if the qualified-id Alloc::void_pointer is valid and denotes a type
    (13.10.3); otherwise, pointer_traits<pointer>::rebind<void>.

using const_void_pointer = see below;
4   Type: Alloc::const_void_pointer if the qualified-id Alloc::const_void_pointer is valid and de-
    notes a type (13.10.3); otherwise, pointer_traits<pointer>::rebind<const void>.

using difference_type = see below;
5   Type: Alloc::difference_type if the qualified-id Alloc::difference_type is valid and denotes a
    type (13.10.3); otherwise, pointer_traits<pointer>::difference_type.

using size_type = see below;
6   Type: Alloc::size_type if the qualified-id Alloc::size_type is valid and denotes a type (13.10.3);
    otherwise, make_unsigned_t<difference_type>.

using propagate_on_container_copy_assignment = see below;
7   Type: Alloc::propagate_on_container_copy_assignment if the qualified-id Alloc::propagate_-
    on_container_copy_assignment is valid and denotes a type (13.10.3); otherwise false_type.

using propagate_on_container_move_assignment = see below;
8   Type: Alloc::propagate_on_container_move_assignment if the qualified-id Alloc::propagate_-
    on_container_move_assignment is valid and denotes a type (13.10.3); otherwise false_type.

using propagate_on_container_swap = see below;
9   Type: Alloc::propagate_on_container_swap if the qualified-id Alloc::propagate_on_container_-
    swap is valid and denotes a type (13.10.3); otherwise false_type.

using is_always_equal = see below;
10  Type: Alloc::is_always_equal if the qualified-id Alloc::is_always_equal is valid and denotes a
    type (13.10.3); otherwise is_empty<Alloc>::type.

template<class T> using rebind_alloc = see below;
11  Alias template: Alloc::rebind<T>::other if the qualified-id Alloc::rebind<T>::other is valid and
    denotes a type (13.10.3); otherwise, Alloc<T, Args> if Alloc is a class template specialization of the
    form Alloc<U, Args>, where Args is zero or more type arguments; otherwise, the instantiation of
    rebind_alloc is ill-formed.

```

20.2.9.3 Static member functions

[allocator.traits.members]

```

static constexpr pointer allocate(Alloc& a, size_type n);
1   Returns: a.allocate(n).

static constexpr pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
2   Returns: a.allocate(n, hint) if that expression is well-formed; otherwise, a.allocate(n).

static constexpr allocation_result<pointer, size_type> allocate_at_least(Alloc& a, size_type n);
3   Returns: a.allocate_at_least(n) if that expression is well-formed; otherwise, {a.allocate(n), n}.

static constexpr void deallocate(Alloc& a, pointer p, size_type n);
4   Effects: Calls a.deallocate(p, n).
5   Throws: Nothing.

```

```

template<class T, class... Args>
    static constexpr void construct(Alloc& a, T* p, Args&&... args);
6     Effects: Calls a.construct(p, std::forward<Args>(args)...) if that call is well-formed; otherwise,
        invokes construct_at(p, std::forward<Args>(args)...).

template<class T>
    static constexpr void destroy(Alloc& a, T* p);
7     Effects: Calls a.destroy(p) if that call is well-formed; otherwise, invokes destroy_at(p).

static constexpr size_type max_size(const Alloc& a) noexcept;
8     Returns: a.max_size() if that expression is well-formed; otherwise, numeric_limits<size_type>::
        max() / sizeof(value_type).

static constexpr Alloc select_on_container_copy_construction(const Alloc& rhs);
9     Returns: rhs.select_on_container_copy_construction() if that expression is well-formed; other-
        wise, rhs.

```

20.2.9.4 Other

[allocator.traits.other]

- 1 The class template `allocation_result` has the template parameters, data members, and special members specified above. It has no base classes or members other than those specified.

20.2.10 The default allocator

[default.allocator]

20.2.10.1 General

[default.allocator.general]

- 1 All specializations of the default allocator meet the allocator completeness requirements (16.4.4.6.2).

```

namespace std {
    template<class T> class allocator {
    public:
        using value_type           = T;
        using size_type            = size_t;
        using difference_type      = ptrdiff_t;
        using propagate_on_container_move_assignment = true_type;

        constexpr allocator() noexcept;
        constexpr allocator(const allocator&) noexcept;
        template<class U> constexpr allocator(const allocator<U>&) noexcept;
        constexpr ~allocator();
        constexpr allocator& operator=(const allocator&) = default;

        constexpr T* allocate(size_t n);
        constexpr allocation_result<T*> allocate_at_least(size_t n);
        constexpr void deallocate(T* p, size_t n);
    };
}

```

- 2 `allocator_traits<allocator<T>>::is_always_equal::value` is true for any T.

20.2.10.2 Members

[allocator.members]

- 1 Except for the destructor, member functions of the default allocator shall not introduce data races (6.10.2) as a result of concurrent calls to those member functions from different threads. Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before the next allocation (if any) in this order.

```
constexpr T* allocate(size_t n);
```

- 2 *Mandates:* T is not an incomplete type (6.9.1).
- 3 *Returns:* A pointer to the initial element of an array of n T.
- 4 *Throws:* `bad_array_new_length` if `numeric_limits<size_t>::max() / sizeof(T) < n`, or `bad_alloc` if the storage cannot be obtained.

5 *Remarks:* The storage for the array is obtained by calling `::operator new` (17.6.3), but it is unspecified when or how often this function is called. This function starts the lifetime of the array object, but not that of any of the array elements.

```
constexpr allocation_result<T*> allocate_at_least(size_t n);
```

6 *Mandates:* T is not an incomplete type (6.9.1).

7 *Returns:* `allocation_result<T*>{ptr, count}`, where `ptr` is a pointer to the initial element of an array of count T and `count` $\geq$ n.

8 *Throws:* `bad_array_new_length` if `numeric_limits<size_t>::max() / sizeof(T) < n`, or `bad_alloc` if the storage cannot be obtained.

9 *Remarks:* The storage for the array is obtained by calling `::operator new`, but it is unspecified when or how often this function is called. This function starts the lifetime of the array object, but not that of any of the array elements.

```
constexpr void deallocate(T* p, size_t n);
```

10 *Preconditions:*

(10.1) — If `p` is memory that was obtained by a call to `allocate_at_least`, let `ret` be the value returned and `req` be the value passed as the first argument to that call. `p` is equal to `ret.ptr` and `n` is a value such that `req` $\leq$ `n` $\leq$ `ret.count`.

(10.2) — Otherwise, `p` is a pointer value obtained from `allocate`. `n` equals the value passed as the first argument to the invocation of `allocate` which returned `p`.

11 *Effects:* Deallocates the storage referenced by `p`.

12 *Remarks:* Uses `::operator delete` (17.6.3), but it is unspecified when this function is called.

20.2.10.3 Operators

[allocator.globals]

```
template<class T, class U>
```

```
constexpr bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
```

1 *Returns:* `true`.

20.2.11 addressof

[specialized.addressof]

```
template<class T> constexpr T* addressof(T& r) noexcept;
```

1 *Returns:* The actual address of the object or function referenced by `r`, even in the presence of an overloaded `operator&`.

2 *Remarks:* An expression `addressof(E)` is a constant subexpression (3.15) if `E` is an lvalue constant subexpression.

20.2.12 C library memory allocation

[c.malloc]

1 [Note 1: The header `<cstdlib>` (17.2.2) declares the functions described in this subclause. — end note]

```
void* aligned_alloc(size_t alignment, size_t size);
```

```
void* calloc(size_t nmemb, size_t size);
```

```
void* malloc(size_t size);
```

2 *Effects:* These functions have the semantics specified in the C standard library.

3 *Remarks:* These functions do not attempt to allocate storage by calling `::operator new()` (17.6.3).

4 These functions implicitly create objects (6.8.2) in the returned region of storage and return a pointer to a suitable created object. In the case of `calloc`, the objects are created before the storage is zeroed.

```
void* realloc(void* ptr, size_t size);
```

5 *Preconditions:* `free(ptr)` has well-defined behavior.

6 *Effects:* If `ptr` is not null and `size` is zero, the behavior is erroneous and the effects are implementation-defined. Otherwise, this function has the semantics specified in the C standard library.

7 *Remarks:* This function does not attempt to allocate storage by calling `::operator new()` (17.6.3). When a non-null pointer is returned, this function implicitly creates objects (6.8.2) in the returned

region of storage and returns a pointer to a suitable created object. The objects are created before the storage is copied.

```
void free(void* ptr);
```

8 *Effects:* This function has the semantics specified in the C standard library.

9 *Remarks:* This function does not attempt to deallocate storage by calling `::operator delete()`.

SEE ALSO: ISO/IEC 9899:2024, 7.24.4

20.3 Pointer tagging

[ptrtag]

20.3.1 Pointer tagging limits

[ptrtag.bits]

```
inline constexpr unsigned max_pointer_bits_available = see below;
```

1 The implementation-defined limit of usable bits for pointer tagging.

2 *Remarks:* A non-zero value of `max_pointer_bits_available` indicates that non-arithmetic operations on invalid pointers with implementation-defined behavior (6.9.5) preserve sufficient information to guarantee that for an object `tp` of a type `TP` that is a specialization of `tagged_pointer_pair` and a pointer value `q`, that is a result of applying a sequence of such operations to the result of `tp.tagged()`, `tp` is equal to `TP::from_tagged(q)`.

```
constexpr unsigned pointer_bits_available(size_t alignment);
```

3 *Constant When:* Precondition is met.

4 *Preconditions:* `alignment` is a power of two.

5 *Returns:* The implementation-defined number of unused bits in a pointer pointing to a hypothetical object with alignment `alignment`.

6 *Recommended practice:* The result is non-zero for `alignment` values larger than 1.

7 [Note 1: On most platforms, the value is the minimum of `countr_zero(alignment)` and `max_pointer_bits_available`. — end note]

20.3.2 Class template `pointer_tag_pair`

[ptrtag.pair]

20.3.2.1 General

[ptrtag.pair.general]

1 The class template `pointer_tag_pair` provides a type to store an object pointer together with a tag value.

```
namespace std {
    template<class U, class PtrT, unsigned BitsRequested, size_t Alignment = alignof(U)>
        concept tagging-compatible-pointee = // exposition only
            convertible_to<U*, PtrT> &&
            pointer_bits_available(Alignment) >= BitsRequested &&
            (is_void_v<element-of<PtrT>> ||
             is_scalar_v<element-of<PtrT>> ||
             is_union_v<element-of<PtrT>> ||
             is_pointer_interconvertible_base_of_v<element-of<PtrT>, U>);

    template<class TagT>
        constexpr unsigned tag-bit-width(TagT value) noexcept; // exposition only

    template<class Ptr, unsigned BitsRequested = bits-available<element-of<Ptr>>,
            class TagT = unsigned>
        class pointer_tag_pair { // freestanding
        public:
            using pointer_type      = Ptr;
            using element_type      = pointer_traits<Ptr>::element_type;
            using tagged_pointer_type = see below;
            using tag_type          = TagT;
            static constexpr unsigned bits_requested = BitsRequested;

            // 20.3.2.2, constructors
            constexpr pointer_tag_pair() noexcept;
```

```

template<tagging-compatible-pointee<pointer_type, bits_requested> U>
    constexpr pointer_tag_pair(U* p, tag_type t);
constexpr pointer_tag_pair(nullptr_t p, tag_type t);

// 20.3.2.3, support for overaligned pointers
template<size_t PromisedAlignment,
    tagging-compatible-pointee<pointer_type, bits_requested, PromisedAlignment> U>
    static constexpr pointer_tag_pair from_overaligned(U* p, tag_type t);

// 20.3.2.4, tagged pointer operations
tagged_pointer_type tagged_pointer() const noexcept;
static pointer_tag_pair from_tagged(tagged_pointer_type p) noexcept;

// 20.3.2.5, accessors
constexpr pointer_type pointer() const noexcept;
constexpr tag_type tag() const noexcept;

// 20.3.2.6, swap
constexpr void swap(pointer_tag_pair& o) noexcept;

// 20.3.2.7, comparisons
friend constexpr see_below operator<=>(pointer_tag_pair lhs, pointer_tag_pair rhs)
    noexcept requires three_way_comparable<tag_type>;
friend constexpr bool operator==(pointer_tag_pair, pointer_tag_pair)
    noexcept requires equality_comparable<tag_type>;
};

template<class Ptr>
    pointer_tag_pair(Ptr*)
        -> pointer_tag_pair<Ptr*>;
template<class Ptr, class TagT>
    pointer_tag_pair(Ptr*, TagT tag)
        -> pointer_tag_pair<Ptr*, bits-available<element-of<Ptr>>, TagT>;
}

```

² An object of class `pointer_tag_pair<Ptr, BitsRequested, TagT>` represents a pair of pointer value of type `Ptr` and tag value of type `TagT`.

³ Each specialization `PT` of `pointer_tag_pair` is a trivially copyable type that models `copyable` such that `sizeof(PT)` is equal to `sizeof(Ptr)` and `alignof(PT)` is equal to `alignof(Ptr)`.

⁴ *Mandates:*

- (4.1) — `is_same_v<remove_cvref_t<Ptr>, Ptr>` is true.
- (4.2) — `is_same_v<remove_cvref_t<TagT>, TagT>` is true.
- (4.3) — `is_pointer_v<Ptr> && !is_function_v<remove_pointer_t<Ptr>>` is true.
- (4.4) — `is_unsigned_v<UT>` is true, where `UT` is `underlying_type_t<TagT>` if `TagT` is an enumeration type, and `TagT` otherwise.
- (4.5) — `sizeof(TagT) <= sizeof(void*)` is true.
- (4.6) — `BitsRequested <= max_pointer_bits_available` is true.

```
template<class T> constexpr unsigned tag-bit-width(T v) noexcept;
```

⁵ *Returns:* `static_cast<unsigned>(bit_width(to_underlying(v)))` if `is_enum_v<T>` is true, otherwise `static_cast<unsigned>(bit_width(v))`.

20.3.2.2 Constructors

[ptrtag.pair.cons]

```
constexpr pointer_tag_pair() noexcept;
```

¹ *Postconditions:* `pointer()` is equal to `nullptr` and `tag()` is equal to `TagT()`.

```
template<tagging-compatible-pointee<pointer_type, bits_requested> U>
    constexpr pointer_tag_pair(U* p, tag_type t);
```

```
constexpr pointer_tag_pair(nullptr_t p, tag_type t);
```

2 *Constant When:* Preconditions are met.

3 *Preconditions:*

(3.1) — *p* is not a pointer past the end of an object (6.9.5).

(3.2) — *tag-bit-width*(*t*) ≤ *bits_requested* is true.

4 *Postconditions:* *pointer*() is equal to *p* and *tag*() is equal to *t*.

5 *Throws:* Nothing.

20.3.2.3 Support for overaligned pointers

[ptrtag.pair.overalign]

```
template<size_t PromisedAlignment,
```

```
        tagging-compatible-pointee<pointer_type, bits_requested, PromisedAlignment> U>
```

```
static constexpr pointer_tag_pair from_overaligned(U* p, tag_type t);
```

1 *Constant When:* Preconditions are met.

2 *Preconditions:*

(2.1) — *p* is not a pointer past the end of an object (6.9.5).

(2.2) — *p* == nullptr || *is_sufficiently_aligned*<PromisedAlignment>(p) is true.

(2.3) — *tag-bit-width*(*t*) ≤ *bits_requested* is true.

3 *Returns:* An object *ptp* of type *pointer_tag_pair* such that both *ptp.pointer*() is equal to *p* and *ptp.tag*() is equal to *t*.

4 *Throws:* Nothing.

20.3.2.4 Tagged pointer operations

[ptrtag.pair.tagops]

```
using tagged_pointer_type = see below;
```

1 *tagged_pointer_type* denotes *cv* void*, for *cv* such that *pointer* denotes *cv* U* for some type U.

```
tagged_pointer_type tagged_pointer() const noexcept;
```

2 *Returns:* An unspecified value *tp* of *tagged_pointer_type* type such that for any specialization DP of *pointer_tag_pair* and alignment value A for which

(2.1) — *pointer_bits_available*(A) ≥ DP::bits_requested is true,

(2.2) — *is_sufficiently_aligned*<A>(ptr) is true, and

(2.3) — *tag-bit-width*(tag) ≤ DP::bits_requested is true,

DP::from_tagged(*tp*) produces an object *dp* such that *reinterpret_cast*<pointer_type>(dp.*pointer*()) is equal to *ptr* and *static_cast*<TagT>(dp.*tag*()) is equal to *tag*.

[Note 1: This function returns an invalid pointer value (6.9.5) with implementation-defined behavior. — end note]

```
static pointer_tag_pair from_tagged(tagged_pointer_type p) noexcept;
```

3 *Returns:* An object *dp* of type *pointer_tag_type*, such that

(3.1) — *reinterpret_cast*<pointer_type>(sp.*pointer*()) is equal to *dp.pointer*() and *static_cast*<TagT>(sp.*tag*()) is equal to *dp.tag*(), if *p* is equal to *sp.tagged_pointer*() for some object *sp* of a type that is a specialization of *pointer_tag_type*, such that for some alignment value A

(3.1.1) — *pointer_bits_available*(A) ≥ *bits_requested* is true,

(3.1.2) — *is_sufficiently_aligned*<A>(sp.*pointer*()) is true, and

(3.1.3) — *tag-bit-width*(sp.*tag*()) ≤ *bits_requested* is true,

(3.2) — otherwise, the values of *dp.pointer*() and *dp.tag*() are unspecified.

20.3.2.5 Accessors**[ptrtag.pair.accessors]**

```
constexpr pointer_type pointer() const noexcept;
```

¹ *Returns:* The pointer value represented by **this*.

```
constexpr tag_type tag() const noexcept;
```

² *Returns:* The tag value represented by **this*.

20.3.2.6 Swap**[ptrtag.pair.swap]**

```
constexpr void swap(pointer_tag_pair& o) noexcept;
```

¹ *Effects:* Exchanges the values of **this* and *o*.

20.3.2.7 Comparisons**[ptrtag.pair.comp]**

```
friend constexpr auto operator<=>(pointer_tag_pair lhs, pointer_tag_pair rhs)
noexcept requires three_way_comparable<tag_type>;
```

¹ *Effects:* Equivalent to:

```
return pair(lhs.pointer(), lhs.tag()) <=> pair(rhs.pointer(), rhs.tag());
```

² *Recommended practice:* Directly compare the internal bit representations if the expression *lhs.tag() <=> rhs.tag()* results in a call to a built-in operator *<=>* comparing values of type *tag_type*.

[Note 1: The result of comparing unrelated pointers is unspecified. — end note]

```
friend constexpr bool operator==(pointer_tag_pair lhs, pointer_tag_pair rhs)
noexcept requires equality_comparable<tag_type>;
```

³ *Effects:* Equivalent to:

```
return pair(lhs.pointer(), lhs.tag()) == pair(rhs.pointer(), rhs.tag());
```

⁴ *Recommended practice:* Directly compare the internal bit representations if the expression *lhs.tag() == rhs.tag()* results in a call to a built-in operator *==* comparing values of type *tag_type*.

20.3.2.8 Tuple interface**[ptrtag.pair.get]**

```
template<class Ptr, unsigned BitsRequested, class TagT>
constexpr tuple_element_t<I, pointer_tag_pair<Ptr, BitsRequested, TagT>>
get(pointer_tag_pair<Ptr, BitsRequested, TagT> p) noexcept;
```

¹ *Mandates:* *I < 2*.

² *Returns:*

(2.1) — *p.pointer()* if *I* is equal to zero,

(2.2) — *p.tag()* otherwise.

20.4 Smart pointers**[smartptr]****20.4.1 Unique-ownership pointers****[unique.ptr]****20.4.1.1 General****[unique.ptr.general]**

¹ A *unique pointer* is an object that owns another object and manages that other object through a pointer. More precisely, a unique pointer is an object *u* that stores a pointer to a second object *p* and will dispose of *p* when *u* is itself destroyed (e.g., when leaving block scope (8.10)). In this context, *u* is said to *own p*.

² The mechanism by which *u* disposes of *p* is known as *p*'s associated *deleter*, a function object whose correct invocation results in *p*'s appropriate disposition (typically its deletion).

³ Let the notation *u.p* denote the pointer stored by *u*, and let *u.d* denote the associated deleter. Upon request, *u* can *reset* (replace) *u.p* and *u.d* with another pointer and deleter, but properly disposes of its owned object via the associated deleter before such replacement is considered completed.

⁴ Each object of a type *U* instantiated from the `unique_ptr` template specified in 20.4.1 has the strict ownership semantics, specified above, of a unique pointer. In partial satisfaction of these semantics, each such *U* is *Cpp17MoveConstructible* and *Cpp17MoveAssignable*, but is not *Cpp17CopyConstructible* nor *Cpp17CopyAssignable*. The template parameter *T* of `unique_ptr` may be an incomplete type.

- ⁵ [Note 1: The uses of `unique_ptr` include providing exception safety for dynamically allocated memory, passing ownership of dynamically allocated memory to a function, and returning dynamically allocated memory from a function. — end note]

20.4.1.2 Default deleters

[unique.ptr.dltr]

20.4.1.2.1 General

[unique.ptr.dltr.general]

- ¹ The class template `default_delete` serves as the default deleter (destruction policy) for the class template `unique_ptr`.
- ² The template parameter `T` of `default_delete` may be an incomplete type.

20.4.1.2.2 `default_delete`

[unique.ptr.dltr.dflt]

```
namespace std {
    template<class T> struct default_delete {
        constexpr default_delete() noexcept = default;
        template<class U> constexpr default_delete(const default_delete<U>&) noexcept;
        constexpr void operator()(T*) const;
    };
}
```

```
template<class U> constexpr default_delete(const default_delete<U>& other) noexcept;
```

- ¹ *Constraints:* `U*` is implicitly convertible to `T*`.
- ² *Effects:* Constructs a `default_delete` object from another `default_delete<U>` object.

```
constexpr void operator()(T* ptr) const;
```

- ³ *Mandates:* `T` is a complete type.
- ⁴ *Effects:* Calls `delete` on `ptr`.

20.4.1.2.3 `default_delete<T[]>`

[unique.ptr.dltr.dflt1]

```
namespace std {
    template<class T> struct default_delete<T[]> {
        constexpr default_delete() noexcept = default;
        template<class U> constexpr default_delete(const default_delete<U[]>&) noexcept;
        template<class U> constexpr void operator()(U* ptr) const;
    };
}
```

```
template<class U> constexpr default_delete(const default_delete<U[]>& other) noexcept;
```

- ¹ *Constraints:* `U(*) []` is convertible to `T(*) []`.
- ² *Effects:* Constructs a `default_delete` object from another `default_delete<U[]>` object.

```
template<class U> constexpr void operator()(U* ptr) const;
```

- ³ *Constraints:* `U(*) []` is convertible to `T(*) []`.
- ⁴ *Mandates:* `U` is a complete type.
- ⁵ *Effects:* Calls `delete[]` on `ptr`.

20.4.1.3 `unique_ptr` for single objects

[unique.ptr.single]

20.4.1.3.1 General

[unique.ptr.single.general]

```
namespace std {
    template<class T, class D = default_delete<T>> class unique_ptr {
    public:
        using pointer      = see below;
        using element_type = T;
        using deleter_type = D;

        // 20.4.1.3.2, constructors
        constexpr unique_ptr() noexcept;
        constexpr explicit unique_ptr(type_identity_t<pointer> p) noexcept;
        constexpr unique_ptr(type_identity_t<pointer> p, see below d1) noexcept;
```

```

constexpr unique_ptr(type_identity_t<pointer> p, see below d2) noexcept;
constexpr unique_ptr(unique_ptr&& u) noexcept;
constexpr unique_ptr(nullptr_t) noexcept;
template<class U, class E>
    constexpr unique_ptr(unique_ptr<U, E>&& u) noexcept;

// 20.4.1.3.3, destructor
constexpr ~unique_ptr();

// 20.4.1.3.4, assignment
constexpr unique_ptr& operator=(unique_ptr&& u) noexcept;
template<class U, class E>
    constexpr unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
constexpr unique_ptr& operator=(nullptr_t) noexcept;

// 20.4.1.3.5, observers
constexpr add_lvalue_reference_t<T> operator*() const noexcept(see below);
constexpr pointer operator->() const noexcept;
constexpr pointer get() const noexcept;
constexpr deleter_type& get_deleter() noexcept;
constexpr const deleter_type& get_deleter() const noexcept;
constexpr explicit operator bool() const noexcept;

// 20.4.1.3.6, modifiers
constexpr pointer release() noexcept;
constexpr void reset(pointer p = pointer()) noexcept;
constexpr void swap(unique_ptr& u) noexcept;

// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};
}

```

- ¹ A program that instantiates the definition of `unique_ptr<T, D>` is ill-formed if `T*` is an invalid type.
[Note 1: This prevents the instantiation of specializations such as `unique_ptr<T&, D>` and `unique_ptr<int() const, D>`. — end note]
- ² The default type for the template parameter `D` is `default_delete`. A client-supplied template argument `D` shall be a function object type (22.10), lvalue reference to function, or lvalue reference to function object type for which, given a value `d` of type `D` and a value `ptr` of type `unique_ptr<T, D>::pointer`, the expression `d(ptr)` is valid and has the effect of disposing of the pointer as appropriate for that deleter.
- ³ If the deleter's type `D` is not a reference type, `D` shall meet the *Cpp17Destructible* requirements (Table 35).
- ⁴ If the *qualified-id* `remove_reference_t<D>::pointer` is valid and denotes a type (13.10.3), then `unique_ptr<T, D>::pointer` shall be a synonym for `remove_reference_t<D>::pointer`. Otherwise `unique_ptr<T, D>::pointer` shall be a synonym for `element_type*`. The type `unique_ptr<T, D>::pointer` shall meet the *Cpp17NullablePointer* requirements (Table 36).
- ⁵ [Example 1: Given an allocator type `X` (16.4.4.6.1) and letting `A` be a synonym for `allocator_traits<X>`, the types `A::pointer`, `A::const_pointer`, `A::void_pointer`, and `A::const_void_pointer` may be used as `unique_ptr<T, D>::pointer`. — end example]

20.4.1.3.2 Constructors

[unique_ptr.single.ctor]

```

constexpr unique_ptr() noexcept;
constexpr unique_ptr(nullptr_t) noexcept;

```

- ¹ *Constraints:* `is_pointer_v<deleter_type>` is false and `is_default_constructible_v<deleter_type>` is true.
- ² *Preconditions:* `D` meets the *Cpp17DefaultConstructible* requirements (Table 30), and that construction does not throw an exception.
- ³ *Effects:* Constructs a `unique_ptr` object that owns nothing, value-initializing the stored pointer and the stored deleter.

4 *Postconditions:* `get() == nullptr`. `get_deleter()` returns a reference to the stored deleter.

```
constexpr explicit unique_ptr(type_identity_t<pointer> p) noexcept;
```

5 *Constraints:* `is_pointer_v<deleter_type>` is false and `is_default_constructible_v<deleter_type>` is true.

6 *Preconditions:* D meets the *Cpp17DefaultConstructible* requirements (Table 30), and that construction does not throw an exception.

7 *Effects:* Constructs a `unique_ptr` which owns p, initializing the stored pointer with p and value-initializing the stored deleter.

8 *Postconditions:* `get() == p`. `get_deleter()` returns a reference to the stored deleter.

```
constexpr unique_ptr(type_identity_t<pointer> p, const D& d) noexcept;
```

```
constexpr unique_ptr(type_identity_t<pointer> p, remove_reference_t<D>&& d) noexcept;
```

9 *Constraints:* `is_constructible_v<D, decltype(d)>` is true.

10 *Preconditions:* For the first constructor, if D is not a reference type, D meets the *Cpp17CopyConstructible* requirements and such construction does not exit via an exception. For the second constructor, if D is not a reference type, D meets the *Cpp17MoveConstructible* requirements and such construction does not exit via an exception.

11 *Effects:* Constructs a `unique_ptr` object which owns p, initializing the stored pointer with p and initializing the deleter from `std::forward<decltype(d)>(d)`.

12 *Postconditions:* `get() == p`. `get_deleter()` returns a reference to the stored deleter. If D is a reference type then `get_deleter()` returns a reference to the lvalue d.

13 *Remarks:* If D is a reference type, the second constructor is defined as deleted.

14 [Example 1:

```
D d;
unique_ptr<int, D> p1(new int, D());           // D must be Cpp17MoveConstructible
unique_ptr<int, D> p2(new int, d);             // D must be Cpp17CopyConstructible
unique_ptr<int, D&> p3(new int, d);            // p3 holds a reference to d
unique_ptr<int, const D&> p4(new int, D());     // error: rvalue deleter object combined
                                              // with reference deleter type
```

— end example]

```
constexpr unique_ptr(unique_ptr&& u) noexcept;
```

15 *Constraints:* `is_move_constructible_v<D>` is true.

16 *Preconditions:* If D is not a reference type, D meets the *Cpp17MoveConstructible* requirements (Table 31). Construction of the deleter from an rvalue of type D does not throw an exception.

17 *Effects:* Constructs a `unique_ptr` from u. If D is a reference type, this deleter is copy constructed from u's deleter; otherwise, this deleter is move constructed from u's deleter.

[Note 1: The construction of the deleter can be implemented with `std::forward<D>`. — end note]

18 *Postconditions:* `get()` yields the value `u.get()` yielded before the construction. `u.get() == nullptr`. `get_deleter()` returns a reference to the stored deleter that was constructed from `u.get_deleter()`. If D is a reference type then `get_deleter()` and `u.get_deleter()` both reference the same lvalue deleter.

```
template<class U, class E> constexpr unique_ptr(unique_ptr<U, E>&& u) noexcept;
```

19 *Constraints:*

(19.1) — `unique_ptr<U, E>::pointer` is implicitly convertible to `pointer`,

(19.2) — U is not an array type, and

(19.3) — either D is a reference type and E is the same type as D, or D is not a reference type and E is implicitly convertible to D.

20 *Preconditions:* If E is not a reference type, construction of the deleter from an rvalue of type E is well-formed and does not throw an exception. Otherwise, E is a reference type and construction of the deleter from an lvalue of type E is well-formed and does not throw an exception.

21 *Effects:* Constructs a `unique_ptr` from `u`. If `E` is a reference type, this deleter is copy constructed from `u`'s deleter; otherwise, this deleter is move constructed from `u`'s deleter.

[*Note 2:* The deleter constructor can be implemented with `std::forward<E>`. — *end note*]

22 *Postconditions:* `get()` yields the value `u.get()` yielded before the construction. `u.get() == nullptr`. `get_deleter()` returns a reference to the stored deleter that was constructed from `u.get_deleter()`.

20.4.1.3.3 Destructor

[`unique_ptr.single.dtor`]

`constexpr ~unique_ptr();`

1 *Effects:* Equivalent to: `if (get()) get_deleter()(get());`

[*Note 1:* The use of `default_delete` requires `T` to be a complete type. — *end note*]

2 *Remarks:* The behavior is undefined if the evaluation of `get_deleter()(get())` throws an exception.

20.4.1.3.4 Assignment

[`unique_ptr.single.asgn`]

`constexpr unique_ptr& operator=(unique_ptr&& u) noexcept;`

1 *Constraints:* `is_move_assignable_v<D>` is true.

2 *Preconditions:* If `D` is not a reference type, `D` meets the *Cpp17MoveAssignable* requirements (Table 33) and assignment of the deleter from an rvalue of type `D` does not throw an exception. Otherwise, `D` is a reference type; `remove_reference_t<D>` meets the *Cpp17CopyAssignable* requirements and assignment of the deleter from an lvalue of type `D` does not throw an exception.

3 *Effects:* Calls `reset(u.release())` followed by `get_deleter() = std::forward<D>(u.get_deleter())`.

4 *Postconditions:* If `this != addressof(u)`, `u.get() == nullptr`, otherwise `u.get()` is unchanged.

5 *Returns:* `*this`.

`template<class U, class E> constexpr unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;`

6 *Constraints:*

(6.1) — `unique_ptr<U, E>::pointer` is implicitly convertible to `pointer`, and

(6.2) — `U` is not an array type, and

(6.3) — `is_assignable_v<D&, E&&>` is true.

7 *Preconditions:* If `E` is not a reference type, assignment of the deleter from an rvalue of type `E` is well-formed and does not throw an exception. Otherwise, `E` is a reference type and assignment of the deleter from an lvalue of type `E` is well-formed and does not throw an exception.

8 *Effects:* Calls `reset(u.release())` followed by `get_deleter() = std::forward<E>(u.get_deleter())`.

9 *Postconditions:* `u.get() == nullptr`.

10 *Returns:* `*this`.

`constexpr unique_ptr& operator=(nullptr_t) noexcept;`

11 *Effects:* As if by `reset()`.

12 *Postconditions:* `get() == nullptr`.

13 *Returns:* `*this`.

20.4.1.3.5 Observers

[`unique_ptr.single.observers`]

`constexpr add_lvalue_reference_t<T> operator*() const noexcept(noexcept(*declval<pointer>()));`

1 *Constraints:* `*declval<pointer>()` is a well-formed expression.

2 *Mandates:* `reference_converts_from_temporary_v<add_lvalue_reference_t<T>, decltype(*declval<pointer>())>` is false.

3 *Preconditions:* `get() != nullptr` is true.

4 *Returns:* `*get()`.

```
constexpr pointer operator->() const noexcept;
```

5 *Preconditions:* `get() != nullptr`.

6 *Returns:* `get()`.

7 [*Note 1:* The use of this function typically requires that `T` be a complete type. — *end note*]

```
constexpr pointer get() const noexcept;
```

8 *Returns:* The stored pointer.

```
constexpr deleter_type& get_deleter() noexcept;
```

```
constexpr const deleter_type& get_deleter() const noexcept;
```

9 *Returns:* A reference to the stored deleter.

```
constexpr explicit operator bool() const noexcept;
```

10 *Returns:* `get() != nullptr`.

20.4.1.3.6 Modifiers

[unique.ptr.single.modifiers]

```
constexpr pointer release() noexcept;
```

1 *Postconditions:* `get() == nullptr`.

2 *Returns:* The value `get()` had at the start of the call to `release`.

```
constexpr void reset(pointer p = pointer()) noexcept;
```

3 *Effects:* Assigns `p` to the stored pointer, and then, with the old value of the stored pointer, `old_p`, evaluates `if (old_p) get_deleter()(old_p)`;

[*Note 1:* The order of these operations is significant because the call to `get_deleter()` might destroy `*this`. — *end note*]

4 *Postconditions:* `get() == p`.

[*Note 2:* The postcondition does not hold if the call to `get_deleter()` destroys `*this` since `this->get()` is no longer a valid expression. — *end note*]

5 *Remarks:* The behavior is undefined if the evaluation of `get_deleter()(old_p)` throws an exception.

```
constexpr void swap(unique_ptr& u) noexcept;
```

6 *Preconditions:* `get_deleter()` is swappable (16.4.4.3) and does not throw an exception under `swap`.

7 *Effects:* Invokes `swap` on the stored pointers and on the stored deleters of `*this` and `u`.

20.4.1.4 unique_ptr for array objects with a runtime length

[unique.ptr.runtime]

20.4.1.4.1 General

[unique.ptr.runtime.general]

```
namespace std {
    template<class T, class D> class unique_ptr<T[], D> {
    public:
        using pointer          = see below;
        using element_type     = T;
        using deleter_type     = D;

        // 20.4.1.4.2, constructors
        constexpr unique_ptr() noexcept;
        template<class U> constexpr explicit unique_ptr(U p) noexcept;
        template<class U> constexpr unique_ptr(U p, see below d) noexcept;
        template<class U> constexpr unique_ptr(U p, see below d) noexcept;
        constexpr unique_ptr(unique_ptr&& u) noexcept;
        template<class U, class E>
            constexpr unique_ptr(unique_ptr<U, E>&& u) noexcept;
        constexpr unique_ptr(nullptr_t) noexcept;

        // destructor
        constexpr ~unique_ptr();
```

```

// assignment
constexpr unique_ptr& operator=(unique_ptr&& u) noexcept;
template<class U, class E>
    constexpr unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
constexpr unique_ptr& operator=(nullptr_t) noexcept;

// 20.4.1.4.4, observers
constexpr T& operator[](size_t i) const;
constexpr pointer get() const noexcept;
constexpr deleter_type& get_deleter() noexcept;
constexpr const deleter_type& get_deleter() const noexcept;
constexpr explicit operator bool() const noexcept;

// 20.4.1.4.5, modifiers
constexpr pointer release() noexcept;
template<class U> constexpr void reset(U p) noexcept;
constexpr void reset(nullptr_t = nullptr) noexcept;
constexpr void swap(unique_ptr& u) noexcept;

// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};
}

```

¹ A specialization for array types is provided with a slightly altered interface.

- (1.1) — Conversions between different types of `unique_ptr<T[], D>` that would be disallowed for the corresponding pointer-to-array types, and conversions to or from the non-array forms of `unique_ptr`, produce an ill-formed program.
- (1.2) — Pointers to types derived from `T` are rejected by the constructors, and by `reset`.
- (1.3) — The observers `operator*` and `operator->` are not provided.
- (1.4) — The indexing observer `operator[]` is provided.
- (1.5) — The default deleter will call `delete[]`.

² Descriptions are provided below only for members that differ from the primary template.

³ The template argument `T` shall be a complete type.

20.4.1.4.2 Constructors

[unique_ptr.runtime.ctor]

```
template<class U> constexpr explicit unique_ptr(U p) noexcept;
```

¹ This constructor behaves the same as the constructor in the primary template that takes a single parameter of type `pointer`.

² *Constraints:*

- (2.1) — `U` is the same type as `pointer`, or
- (2.2) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, and `V(*) []` is convertible to `element_type(*) []`.

```
template<class U> constexpr unique_ptr(U p, see below d) noexcept;
template<class U> constexpr unique_ptr(U p, see below d) noexcept;
```

³ These constructors behave the same as the constructors in the primary template that take a parameter of type `pointer` and a second parameter.

⁴ *Constraints:*

- (4.1) — `U` is the same type as `pointer`,
- (4.2) — `U` is `nullptr_t`, or
- (4.3) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, and `V(*) []` is convertible to `element_type(*) []`.

```
template<class U, class E> constexpr unique_ptr(unique_ptr<U, E>&& u) noexcept;
```

5 This constructor behaves the same as in the primary template.

6 *Constraints:* Where UP is `unique_ptr<U, E>`:

- (6.1) — U is an array type, and
- (6.2) — `pointer` is the same type as `element_type*`, and
- (6.3) — `UP::pointer` is the same type as `UP::element_type*`, and
- (6.4) — `UP::element_type(*) []` is convertible to `element_type(*) []`, and
- (6.5) — either D is a reference type and E is the same type as D, or D is not a reference type and E is implicitly convertible to D.

[Note 1: This replaces the *Constraints:* specification of the primary template. — end note]

20.4.1.4.3 Assignment

[unique_ptr.runtime.asgn]

```
template<class U, class E> constexpr unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
```

1 This operator behaves the same as in the primary template.

2 *Constraints:* Where UP is `unique_ptr<U, E>`:

- (2.1) — U is an array type, and
- (2.2) — `pointer` is the same type as `element_type*`, and
- (2.3) — `UP::pointer` is the same type as `UP::element_type*`, and
- (2.4) — `UP::element_type(*) []` is convertible to `element_type(*) []`, and
- (2.5) — `is_assignable_v<D&, E&&>` is true.

[Note 1: This replaces the *Constraints:* specification of the primary template. — end note]

20.4.1.4.4 Observers

[unique_ptr.runtime.observers]

```
constexpr T& operator[](size_t i) const;
```

1 *Preconditions:* `i` < the number of elements in the array to which the stored pointer points.

2 *Returns:* `get()[i]`.

20.4.1.4.5 Modifiers

[unique_ptr.runtime.modifiers]

```
constexpr void reset(nullptr_t p = nullptr) noexcept;
```

1 *Effects:* Equivalent to `reset(pointer())`.

```
template<class U> constexpr void reset(U p) noexcept;
```

2 This function behaves the same as the `reset` member of the primary template.

3 *Constraints:*

- (3.1) — U is the same type as `pointer`, or
- (3.2) — `pointer` is the same type as `element_type*`, U is a pointer type `V*`, and `V(*) []` is convertible to `element_type(*) []`.

20.4.1.5 Creation

[unique_ptr.create]

```
template<class T, class... Args> constexpr unique_ptr<T> make_unique(Args&&... args);
```

1 *Constraints:* T is not an array type.

2 *Returns:* `unique_ptr<T>(new T(std::forward<Args>(args)...))`.

```
template<class T> constexpr unique_ptr<T> make_unique(size_t n);
```

3 *Constraints:* T is an array of unknown bound.

4 *Returns:* `unique_ptr<T>(new remove_extent_t<T>[n]())`.

```
template<class T, class... Args> unspecified make_unique(Args&&...) = delete;
```

5 *Constraints:* T is an array of known bound.

```
template<class T> constexpr unique_ptr<T> make_unique_for_overwrite();
```

6 *Constraints:* T is not an array type.

7 *Returns:* unique_ptr<T>(new T).

```
template<class T> constexpr unique_ptr<T> make_unique_for_overwrite(size_t n);
```

8 *Constraints:* T is an array of unknown bound.

9 *Returns:* unique_ptr<T>(new remove_extent_t<T>[n]).

```
template<class T, class... Args> unspecified make_unique_for_overwrite(Args&&...) = delete;
```

10 *Constraints:* T is an array of known bound.

20.4.1.6 Specialized algorithms

[unique_ptr.special]

```
template<class T, class D> constexpr void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;
```

1 *Constraints:* is_swappable_v<D> is true.

2 *Effects:* Calls x.swap(y).

```
template<class T1, class D1, class T2, class D2>
```

```
constexpr bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

3 *Returns:* x.get() == y.get().

```
template<class T1, class D1, class T2, class D2>
```

```
constexpr bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

4 Let CT denote

```
common_type_t<typename unique_ptr<T1, D1>::pointer,
               typename unique_ptr<T2, D2>::pointer>
```

5 *Mandates:*

(5.1) — unique_ptr<T1, D1>::pointer is implicitly convertible to CT and

(5.2) — unique_ptr<T2, D2>::pointer is implicitly convertible to CT.

6 *Preconditions:* The specialization less<CT> is a function object type (22.10) that induces a strict weak ordering (26.8) on the pointer values.

7 *Returns:* less<CT>()(x.get(), y.get()).

```
template<class T1, class D1, class T2, class D2>
```

```
constexpr bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

8 *Returns:* y < x.

```
template<class T1, class D1, class T2, class D2>
```

```
constexpr bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

9 *Returns:* !(y < x).

```
template<class T1, class D1, class T2, class D2>
```

```
constexpr bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

10 *Returns:* !(x < y).

```
template<class T1, class D1, class T2, class D2>
```

```
requires three_way_comparable_with<typename unique_ptr<T1, D1>::pointer,
                                   typename unique_ptr<T2, D2>::pointer>
constexpr compare_three_way_result_t<typename unique_ptr<T1, D1>::pointer,
                                     typename unique_ptr<T2, D2>::pointer>
operator<=>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

11 *Returns:* compare_three_way()(x.get(), y.get()).

```
template<class T, class D>
```

```
constexpr bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
```

12 *Returns:* !x.

```
template<class T, class D>
constexpr bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
constexpr bool operator<(nullptr_t, const unique_ptr<T, D>& x);
```

13 *Preconditions:* The specialization `less<unique_ptr<T, D>::pointer>` is a function object type (22.10) that induces a strict weak ordering (26.8) on the pointer values.

14 *Returns:* The first function template returns

```
less<unique_ptr<T, D>::pointer>()(x.get(), nullptr)
```

The second function template returns

```
less<unique_ptr<T, D>::pointer>()(nullptr, x.get())
```

```
template<class T, class D>
constexpr bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
constexpr bool operator>(nullptr_t, const unique_ptr<T, D>& x);
```

15 *Returns:* The first function template returns `nullptr < x`. The second function template returns `x < nullptr`.

```
template<class T, class D>
constexpr bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
constexpr bool operator<=(nullptr_t, const unique_ptr<T, D>& x);
```

16 *Returns:* The first function template returns `!(nullptr < x)`. The second function template returns `!(x < nullptr)`.

```
template<class T, class D>
constexpr bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template<class T, class D>
constexpr bool operator>=(nullptr_t, const unique_ptr<T, D>& x);
```

17 *Returns:* The first function template returns `!(x < nullptr)`. The second function template returns `!(nullptr < x)`.

```
template<class T, class D>
requires three_way_comparable<typename unique_ptr<T, D>::pointer>
constexpr compare_three_way_result_t<typename unique_ptr<T, D>::pointer>
operator<=>(const unique_ptr<T, D>& x, nullptr_t);
```

18 *Returns:*

```
compare_three_way()(x.get(), static_cast<typename unique_ptr<T, D>::pointer>(nullptr)).
```

20.4.1.7 I/O

[unique.ptr.io]

```
template<class E, class T, class Y, class D>
basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const unique_ptr<Y, D>& p);
```

1 *Constraints:* `os << p.get()` is a valid expression.

2 *Effects:* Equivalent to: `os << p.get();`

3 *Returns:* `os`.

20.4.2 Shared-ownership pointers

[util.sharedptr]

20.4.2.1 Class `bad_weak_ptr`

[util.smartptr.weak.bad]

```
namespace std {
class bad_weak_ptr : public exception {
public:
// see 17.9.3 for the specification of the special member functions
constexpr const char* what() const noexcept override;
};
}
```

1 An exception of type `bad_weak_ptr` is thrown by the `shared_ptr` constructor taking a `weak_ptr`.

```
constexpr const char* what() const noexcept override;
```

² *Returns:* An implementation-defined NTBS.

20.4.2.2 Class template `shared_ptr`

[util.smartptr.shared]

20.4.2.2.1 General

[util.smartptr.shared.general]

- ¹ The `shared_ptr` class template stores a pointer, usually obtained via `new`. `shared_ptr` implements semantics of shared ownership; the last remaining owner of the pointer is responsible for destroying the object, or otherwise releasing the resources associated with the stored pointer. A `shared_ptr` is said to be empty if it does not own a pointer.

```
namespace std {
    template<class T> class shared_ptr {
    public:
        using element_type = remove_extent_t<T>;
        using weak_type     = weak_ptr<T>;

        // 20.4.2.2.2, constructors
        constexpr shared_ptr() noexcept;
        constexpr shared_ptr(nullptr_t) noexcept : shared_ptr() { }
        template<class Y>
            constexpr explicit shared_ptr(Y* p);
        template<class Y, class D>
            constexpr shared_ptr(Y* p, D d);
        template<class Y, class D, class A>
            constexpr shared_ptr(Y* p, D d, A a);
        template<class D>
            constexpr shared_ptr(nullptr_t p, D d);
        template<class D, class A>
            constexpr shared_ptr(nullptr_t p, D d, A a);
        template<class Y>
            constexpr shared_ptr(const shared_ptr<Y>& r, element_type* p) noexcept;
        template<class Y>
            constexpr shared_ptr(shared_ptr<Y>&& r, element_type* p) noexcept;
        constexpr shared_ptr(const shared_ptr& r) noexcept;
        template<class Y>
            constexpr shared_ptr(const shared_ptr<Y>& r) noexcept;
        constexpr shared_ptr(shared_ptr&& r) noexcept;
        template<class Y>
            constexpr shared_ptr(shared_ptr<Y>&& r) noexcept;
        template<class Y>
            constexpr explicit shared_ptr(const weak_ptr<Y>& r);
        template<class Y, class D>
            constexpr shared_ptr(unique_ptr<Y, D>&& r);

        // 20.4.2.2.3, destructor
        constexpr ~shared_ptr();

        // 20.4.2.2.4, assignment
        constexpr shared_ptr& operator=(const shared_ptr& r) noexcept;
        template<class Y>
            constexpr shared_ptr& operator=(const shared_ptr<Y>& r) noexcept;
        constexpr shared_ptr& operator=(shared_ptr&& r) noexcept;
        template<class Y>
            constexpr shared_ptr& operator=(shared_ptr<Y>&& r) noexcept;
        template<class Y, class D>
            constexpr shared_ptr& operator=(unique_ptr<Y, D>&& r);

        // 20.4.2.2.5, modifiers
        constexpr void swap(shared_ptr& r) noexcept;
        constexpr void reset() noexcept;
        template<class Y>
            constexpr void reset(Y* p);
```

```

template<class Y, class D>
    constexpr void reset(Y* p, D d);
template<class Y, class D, class A>
    constexpr void reset(Y* p, D d, A a);

// 20.4.2.2.6, observers
constexpr element_type* get() const noexcept;
constexpr T& operator*() const noexcept;
constexpr T* operator->() const noexcept;
constexpr element_type& operator[](ptrdiff_t i) const;
constexpr long use_count() const noexcept;
constexpr explicit operator bool() const noexcept;
template<class U>
    bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_before(const weak_ptr<U>& b) const noexcept;
size_t owner_hash() const noexcept;
template<class U>
    constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
};

template<class T>
    shared_ptr<weak_ptr<T>> -> shared_ptr<T>;
template<class T, class D>
    shared_ptr<unique_ptr<T, D>> -> shared_ptr<T>;
}

```

- 2 Specializations of `shared_ptr` shall be *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, and *Cpp17LessThanComparable*, allowing their use in standard containers. Specializations of `shared_ptr` shall be contextually convertible to `bool`, allowing their use in boolean expressions and declarations in conditions.

- 3 The template parameter `T` of `shared_ptr` may be an incomplete type.

[Note 1: `T` can be a function type. — end note]

- 4 [Example 1:

```

    if (shared_ptr<X> px = dynamic_pointer_cast<X>(py)) {
        // do something with px
    }

```

— end example]

- 5 For purposes of determining the presence of a data race, member functions shall access and modify only the `shared_ptr` and `weak_ptr` objects themselves and not objects they refer to. Changes in `use_count()` do not reflect modifications that can introduce data races.
- 6 For the purposes of 20.4, a pointer type `Y*` is said to be *compatible with* a pointer type `T*` when either `Y*` is convertible to `T*` or `Y` is `U[N]` and `T` is `cv U[]`.

20.4.2.2.2 Constructors

[util.smartptr.shared.const]

- 1 In the constructor definitions below, enables `shared_from_this` with `p`, for a pointer `p` of type `Y*`, means that if `Y` has an unambiguous and accessible base class that is a specialization of `enable_shared_from_this` (20.4.2.7), then `remove_cv_t<Y>*` shall be implicitly convertible to `T*` and the constructor evaluates the statement:

```

    if (p != nullptr && p->weak-this.expired())
        p->weak-this = shared_ptr<remove_cv_t<Y>>(*this, const_cast<remove_cv_t<Y>*>(p));

```

The assignment to the `weak-this` member is not atomic and conflicts with any potentially concurrent access to the same object (6.10.2).

```
constexpr shared_ptr() noexcept;
```

- 2 *Postconditions:* `use_count() == 0 && get() == nullptr`.


```
template<class Y> constexpr explicit shared_ptr(Y* p);
```

3 *Constraints:* When T is an array type, the expression `delete[] p` is well-formed and either T is U[N] and Y(*) [N] is convertible to T*, or T is U[] and Y(*) [] is convertible to T*. When T is not an array type, the expression `delete p` is well-formed and Y* is convertible to T*.

4 *Mandates:* Y is a complete type.

5 *Preconditions:* The expression `delete[] p`, when T is an array type, or `delete p`, when T is not an array type, has well-defined behavior, and does not throw exceptions.

6 *Effects:* When T is not an array type, constructs a `shared_ptr` object that owns the pointer p. Otherwise, constructs a `shared_ptr` that owns p and a deleter of an unspecified type that calls `delete[] p`. When T is not an array type, enables `shared_from_this` with p. If an exception is thrown, `delete p` is called when T is not an array type, `delete[] p` otherwise.

7 *Postconditions:* `use_count() == 1` && `get() == p`.

8 *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory cannot be obtained.

```
template<class Y, class D> constexpr shared_ptr(Y* p, D d);
template<class Y, class D, class A> constexpr shared_ptr(Y* p, D d, A a);
template<class D> constexpr shared_ptr(nullptr_t p, D d);
template<class D, class A> constexpr shared_ptr(nullptr_t p, D d, A a);
```

9 *Constraints:* `is_move_constructible_v<D>` is true, and `d(p)` is a well-formed expression. For the first two overloads:

(9.1) — If T is an array type, then either T is U[N] and Y(*) [N] is convertible to T*, or T is U[] and Y(*) [] is convertible to T*.

(9.2) — If T is not an array type, then Y* is convertible to T*.

10 *Preconditions:* Construction of d and a deleter of type D initialized with `std::move(d)` do not throw exceptions. The expression `d(p)` has well-defined behavior and does not throw exceptions. A meets the *Cpp17Allocator* requirements (16.4.4.6.1).

11 *Effects:* Constructs a `shared_ptr` object that owns the object p and the deleter d. When T is not an array type, the first and second constructors enable `shared_from_this` with p. The second and fourth constructors shall use a copy of a to allocate memory for internal use. If an exception is thrown, `d(p)` is called.

12 *Postconditions:* `use_count() == 1` && `get() == p`.

13 *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory cannot be obtained.

```
template<class Y> constexpr shared_ptr(const shared_ptr<Y>& r, element_type* p) noexcept;
template<class Y> constexpr shared_ptr(shared_ptr<Y>&& r, element_type* p) noexcept;
```

14 *Effects:* Constructs a `shared_ptr` instance that stores p and shares ownership with the initial value of r.

15 *Postconditions:* `get() == p`. For the second overload, r is empty and `r.get() == nullptr`.

16 [Note 1: Use of this constructor leads to a dangling pointer unless p remains valid at least until the ownership group of r is destroyed. — end note]

17 [Note 2: This constructor allows creation of an empty `shared_ptr` instance with a non-null stored pointer. — end note]

```
constexpr shared_ptr(const shared_ptr& r) noexcept;
template<class Y> constexpr shared_ptr(const shared_ptr<Y>& r) noexcept;
```

18 *Constraints:* For the second constructor, Y* is compatible with T*.

19 *Effects:* If r is empty, constructs an empty `shared_ptr` object; otherwise, constructs a `shared_ptr` object that shares ownership with r.

20 *Postconditions:* `get() == r.get()` && `use_count() == r.use_count()`.

```
constexpr shared_ptr(shared_ptr&& r) noexcept;
```

```
template<class Y> constexpr shared_ptr(shared_ptr<Y>&& r) noexcept;
```

21 *Constraints:* For the second constructor, `Y*` is compatible with `T*`.

22 *Effects:* Move constructs a `shared_ptr` instance from `r`.

23 *Postconditions:* `*this` contains the old value of `r`. `r` is empty, and `r.get() == nullptr`.

```
template<class Y> constexpr explicit shared_ptr(const weak_ptr<Y>& r);
```

24 *Constraints:* `Y*` is compatible with `T*`.

25 *Effects:* Constructs a `shared_ptr` object that shares ownership with `r` and stores a copy of the pointer stored in `r`. If an exception is thrown, the constructor has no effect.

26 *Postconditions:* `use_count() == r.use_count()`.

27 *Throws:* `bad_weak_ptr` when `r.expired()`.

```
template<class Y, class D> constexpr shared_ptr(unique_ptr<Y, D>&& r);
```

28 *Constraints:* `Y*` is compatible with `T*` and `unique_ptr<Y, D>::pointer` is convertible to `element_type*`.

29 *Effects:* If `r.get() == nullptr`, equivalent to `shared_ptr()`. Otherwise, if `D` is not a reference type, equivalent to `shared_ptr(r.release(), std::move(r.get_deleter()))`. Otherwise, equivalent to `shared_ptr(r.release(), ref(r.get_deleter()))`. If an exception is thrown, the constructor has no effect.

20.4.2.2.3 Destructor

[util.smartptr.shared.dest]

```
constexpr ~shared_ptr();
```

1 *Effects:*

- (1.1) — If `*this` is empty or shares ownership with another `shared_ptr` instance (`use_count() > 1`), there are no side effects.
- (1.2) — Otherwise, if `*this` owns an object `p` and a deleter `d`, `d(p)` is called.
- (1.3) — Otherwise, `*this` owns a pointer `p`, and `delete p` is called.

2 [Note 1: Since the destruction of `*this` decreases the number of instances that share ownership with `*this` by one, after `*this` has been destroyed all `shared_ptr` instances that shared ownership with `*this` will report a `use_count()` that is one less than its previous value. — end note]

20.4.2.2.4 Assignment

[util.smartptr.shared.assign]

```
constexpr shared_ptr& operator=(const shared_ptr& r) noexcept;
```

```
template<class Y> constexpr shared_ptr& operator=(const shared_ptr<Y>& r) noexcept;
```

1 *Effects:* Equivalent to `shared_ptr(r).swap(*this)`.

2 *Returns:* `*this`.

3 [Note 1: The use count updates caused by the temporary object construction and destruction are not observable side effects, so the implementation can meet the effects (and the implied guarantees) via different means, without creating a temporary. In particular, in the example:

```
shared_ptr<int> p(new int);
shared_ptr<void> q(p);
p = p;
q = p;
```

both assignments can be no-ops. — end note]

```
constexpr shared_ptr& operator=(shared_ptr&& r) noexcept;
```

```
template<class Y> constexpr shared_ptr& operator=(shared_ptr<Y>&& r) noexcept;
```

4 *Effects:* Equivalent to `shared_ptr(std::move(r)).swap(*this)`.

5 *Returns:* `*this`.

```
template<class Y, class D> constexpr shared_ptr& operator=(unique_ptr<Y, D>&& r);
```

6 *Effects:* Equivalent to `shared_ptr(std::move(r)).swap(*this)`.

7 *Returns: *this.*

20.4.2.2.5 Modifiers

[util.smartptr.shared.mod]

constexpr void swap(shared_ptr& r) noexcept;

1 *Effects:* Exchanges the contents of **this* and *r*.

constexpr void reset() noexcept;

2 *Effects:* Equivalent to `shared_ptr().swap(*this)`.

template<class Y> constexpr void reset(Y* p);

3 *Effects:* Equivalent to `shared_ptr(p).swap(*this)`.

template<class Y, class D> constexpr void reset(Y* p, D d);

4 *Effects:* Equivalent to `shared_ptr(p, d).swap(*this)`.

template<class Y, class D, class A> constexpr void reset(Y* p, D d, A a);

5 *Effects:* Equivalent to `shared_ptr(p, d, a).swap(*this)`.

20.4.2.2.6 Observers

[util.smartptr.shared.obs]

constexpr element_type* get() const noexcept;

1 *Returns:* The stored pointer.

constexpr T& operator*() const noexcept;

2 *Preconditions:* `get() != nullptr`.

3 *Returns:* `*get()`.

4 *Remarks:* When *T* is an array type or *cv void*, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well-formed.

constexpr T* operator->() const noexcept;

5 *Preconditions:* `get() != nullptr`.

6 *Returns:* `get()`.

7 *Remarks:* When *T* is an array type, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well-formed.

constexpr element_type& operator[](ptrdiff_t i) const;

8 *Preconditions:* `get() != nullptr` is true.

9 *Hardened preconditions:* $i \geq 0$. If *T* is *U[N]*, $i < N$.

10 *Returns:* `get()[i]`.

11 *Throws:* Nothing.

12 *Remarks:* When *T* is not an array type, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well-formed.

constexpr long use_count() const noexcept;

13 *Synchronization:* None.

14 *Returns:* The number of `shared_ptr` objects, **this* included, that share ownership with **this*, or 0 when **this* is empty.

15 [Note 1: `get() == nullptr` does not imply a specific return value of `use_count()`. — end note]

16 [Note 2: `weak_ptr<T>::lock()` can affect the return value of `use_count()`. — end note]

17 [Note 3: When multiple threads might affect the return value of `use_count()`, the result is approximate. In particular, `use_count() == 1` does not imply that accesses through a previously destroyed `shared_ptr` have in any sense completed. — end note]

```
constexpr explicit operator bool() const noexcept;
```

18 *Returns:* `get() != nullptr`.

```
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
```

```
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

19 *Returns:* An unspecified value such that

(19.1) — `owner_before(b)` defines a strict weak ordering as defined in 26.8;

(19.2) — `!owner_before(b) && !b.owner_before(*this)` is true if and only if `owner_equal(b)` is true.

```
size_t owner_hash() const noexcept;
```

20 *Returns:* An unspecified value such that, for any object `x` where `owner_equal(x)` is true, `owner_hash() == x.owner_hash()` is true.

```
template<class U>
```

```
constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
```

```
template<class U>
```

```
constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

21 *Returns:* true if and only if `*this` and `b` share ownership or are both empty. Otherwise returns false.

22 *Remarks:* `owner_equal` is an equivalence relation.

20.4.2.2.7 Creation

[util.smartptr.shared.create]

1 The common requirements that apply to all `make_shared`, `allocate_shared`, `make_shared_for_overwrite`, and `allocate_shared_for_overwrite` overloads, unless specified otherwise, are described below.

```
template<class T, ...>
```

```
constexpr shared_ptr<T> make_shared(args);
```

```
template<class T, class A, ...>
```

```
constexpr shared_ptr<T> allocate_shared(const A& a, args);
```

```
template<class T, ...>
```

```
constexpr shared_ptr<T> make_shared_for_overwrite(args);
```

```
template<class T, class A, ...>
```

```
constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a, args);
```

2 *Preconditions:* `A` meets the *Cpp17Allocator* requirements (16.4.4.6.1).

3 *Effects:* Allocates memory for an object of type `T` (or `U[N]` when `T` is `U[]`, where `N` is determined from `args` as specified by the concrete overload). The object is initialized from `args` as specified by the concrete overload. The `allocate_shared` and `allocate_shared_for_overwrite` templates use a copy of `a` (rebound for an unspecified `value_type`) to allocate memory. If an exception is thrown, the functions have no effect.

4 *Postconditions:* `r.get() != nullptr && r.use_count() == 1`, where `r` is the return value.

5 *Returns:* A `shared_ptr` instance that stores and owns the address of the newly constructed object.

6 *Throws:* `bad_alloc`, or an exception thrown from `allocate` or from the initialization of the object.

7 *Remarks:*

(7.1) — Implementations should perform no more than one memory allocation.

[Note 1: This provides efficiency equivalent to an intrusive smart pointer. — end note]

(7.2) — When an object of an array type `U` is specified to have an initial value of `u` (of the same type), this shall be interpreted to mean that each array element of the object has as its initial value the corresponding element from `u`.

(7.3) — When an object of an array type is specified to have a default initial value, this shall be interpreted to mean that each array element of the object has a default initial value.

(7.4) — When a (sub)object of a non-array type `U` is specified to have an initial value of `v`, or `U(1...)`, where `1...` is a list of constructor arguments, `make_shared` shall initialize this (sub)object via the expression `::new(pv) U(v)` or `::new(pv) U(1...)` respectively, where `pv` has type `void*` and points to storage suitable to hold an object of type `U`.

- (7.5) — When a (sub)object of a non-array type `U` is specified to have an initial value of `v`, or `U(1...)`, where `1...` is a list of constructor arguments, `allocate_shared` shall initialize this (sub)object via the expression
- (7.5.1) — `allocator_traits<A2>::construct(a2, pu, v)` or
- (7.5.2) — `allocator_traits<A2>::construct(a2, pu, 1...)`
- respectively, where `pu` is a pointer of type `remove_cv_t<U>*` pointing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared`.
- (7.6) — When a (sub)object of non-array type `U` is specified to have a default initial value, `make_shared` shall initialize this (sub)object via the expression `::new(pv) U()`, where `pv` has type `void*` and points to storage suitable to hold an object of type `U`.
- (7.7) — When a (sub)object of non-array type `U` is specified to have a default initial value, `allocate_shared` initializes this (sub)object via the expression `allocator_traits<A2>::construct(a2, pu)`, where `pu` is a pointer of type `remove_cv_t<U>*` pointing to storage suitable to hold an object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared`.
- (7.8) — When a (sub)object of non-array type `U` is initialized by `make_shared_for_overwrite` or `allocate_shared_for_overwrite`, it is initialized via the expression `::new(pv) U`, where `pv` has type `void*` and points to storage suitable to hold an object of type `U`.
- (7.9) — Array elements are initialized in ascending order of their addresses.
- (7.10) — When the lifetime of the object managed by the return value ends, or when the initialization of an array element throws an exception, the initialized elements are destroyed in the reverse order of their original construction.
- (7.11) — When a (sub)object of non-array type `U` that was initialized by `make_shared`, `make_shared_for_overwrite`, or `allocate_shared_for_overwrite` is to be destroyed, it is destroyed via the expression `pu->~U()` where `pu` points to that object of type `U`.
- (7.12) — When a (sub)object of non-array type `U` that was initialized by `allocate_shared` is to be destroyed, it is destroyed via the expression `allocator_traits<A2>::destroy(a2, pu)` where `pu` is a pointer of type `remove_cv_t<U>*` pointing to that object of type `remove_cv_t<U>` and `a2` of type `A2` is a potentially rebound copy of the allocator `a` passed to `allocate_shared`.

[Note 2: These functions will typically allocate more memory than `sizeof(T)` to allow for internal bookkeeping structures such as reference counts. — end note]

```
template<class T, class... Args>
constexpr shared_ptr<T> make_shared(Args&&... args);           // T is not array
template<class T, class A, class... Args>
constexpr shared_ptr<T> allocate_shared(const A& a, Args&&... args); // T is not array
```

8 *Constraints:* `T` is not an array type.

9 *Returns:* A `shared_ptr` to an object of type `T` with an initial value `T(std::forward<Args>(args)...) .`

10 *Remarks:* The `shared_ptr` constructors called by these functions enable `shared_from_this` with the address of the newly constructed object of type `T`.

11 [Example 1:

```
shared_ptr<int> p = make_shared<int>(); // shared_ptr to int()
shared_ptr<vector<int>> q = make_shared<vector<int>>(16, 1);
// shared_ptr to vector of 16 elements with value 1
```

— end example]

```
template<class T>
constexpr shared_ptr<T> make_shared(size_t N);               // T is U[]
template<class T, class A>
constexpr shared_ptr<T> allocate_shared(const A& a, size_t N); // T is U[]
```

12 *Constraints:* `T` is an array of unknown bound.

13 *Returns:* A `shared_ptr` to an array of N elements of type `remove_extent_t<T>` with a default initial value.

14 *[Example 2:*

```
    shared_ptr<double[]> p = make_shared<double[]>(1024);
    // shared_ptr to a value-initialized double[1024]
    shared_ptr<double[][2][2]> q = make_shared<double[][2][2]>(6);
    // shared_ptr to a value-initialized double[6][2][2]
```

— end example]

```
template<class T>
    constexpr shared_ptr<T> make_shared(); // T is U[N]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a); // T is U[N]
```

15 *Constraints:* T is an array of known bound.

16 *Returns:* A `shared_ptr` to an object of type T with a default initial value.

17 *[Example 3:*

```
    shared_ptr<double[1024]> p = make_shared<double[1024]>();
    // shared_ptr to a value-initialized double[1024]
    shared_ptr<double[6][2][2]> q = make_shared<double[6][2][2]>();
    // shared_ptr to a value-initialized double[6][2][2]
```

— end example]

```
template<class T>
    constexpr shared_ptr<T> make_shared(size_t N,
                                        const remove_extent_t<T>& u); // T is U[]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a, size_t N,
                                        const remove_extent_t<T>& u); // T is U[]
```

18 *Constraints:* T is an array of unknown bound.

19 *Returns:* A `shared_ptr` to an array of N elements of type `remove_extent_t<T>` where each array element has an initial value of u.

20 *[Example 4:*

```
    shared_ptr<double[]> p = make_shared<double[]>(1024, 1.0);
    // shared_ptr to a double[1024], where each element is 1.0
    shared_ptr<double[][2]> q = make_shared<double[][2]>(6, {1.0, 0.0});
    // shared_ptr to a double[6][2], where each double[2] element is {1.0, 0.0}
    shared_ptr<vector<int>[]> r = make_shared<vector<int>[]>(4, {1, 2});
    // shared_ptr to a vector<int>[4], where each vector has contents {1, 2}
```

— end example]

```
template<class T>
    constexpr shared_ptr<T> make_shared(const remove_extent_t<T>& u); // T is U[N]
template<class T, class A>
    constexpr shared_ptr<T> allocate_shared(const A& a,
                                        const remove_extent_t<T>& u); // T is U[N]
```

21 *Constraints:* T is an array of known bound.

22 *Returns:* A `shared_ptr` to an object of type T, where each array element of type `remove_extent_t<T>` has an initial value of u.

23 *[Example 5:*

```
    shared_ptr<double[1024]> p = make_shared<double[1024]>(1.0);
    // shared_ptr to a double[1024], where each element is 1.0
    shared_ptr<double[6][2]> q = make_shared<double[6][2]>({1.0, 0.0});
    // shared_ptr to a double[6][2], where each double[2] element is {1.0, 0.0}
    shared_ptr<vector<int>[4]> r = make_shared<vector<int>[4]>({1, 2});
    // shared_ptr to a vector<int>[4], where each vector has contents {1, 2}
```

— end example]

```

template<class T>
constexpr shared_ptr<T> make_shared_for_overwrite();
template<class T, class A>
constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a);

```

24 *Constraints:* T is not an array of unknown bound.

25 *Returns:* A shared_ptr to an object of type T.

26 *[Example 6:*

```

    struct X { double data[1024]; };
    shared_ptr<X> p = make_shared_for_overwrite<X>();
    // shared_ptr to a default-initialized X, where each element in X::data has an indeterminate value

    shared_ptr<double[1024]> q = make_shared_for_overwrite<double[1024]>();
    // shared_ptr to a default-initialized double[1024], where each element has an indeterminate value
— end example]

```

```

template<class T>
constexpr shared_ptr<T> make_shared_for_overwrite(size_t N);
template<class T, class A>
constexpr shared_ptr<T> allocate_shared_for_overwrite(const A& a, size_t N);

```

27 *Constraints:* T is an array of unknown bound.

28 *Returns:* A shared_ptr to an array of N elements of type remove_extent_t<T>.

29 *[Example 7:*

```

    shared_ptr<double[]> p = make_shared_for_overwrite<double[]>(1024);
    // shared_ptr to a default-initialized double[1024], where each element has an indeterminate value
— end example]

```

20.4.2.2.8 Comparison

[util.smartptr.shared.cmp]

```

template<class T, class U>
constexpr bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

```

1 *Returns:* a.get() == b.get().

```

template<class T>
constexpr bool operator==(const shared_ptr<T>& a, nullptr_t) noexcept;

```

2 *Returns:* !a.

```

template<class T, class U>
constexpr strong_ordering operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

```

3 *Returns:* compare_three_way()(a.get(), b.get()).

4 *[Note 1:* Defining a comparison operator function allows shared_ptr objects to be used as keys in associative containers. — end note]

```

template<class T>
constexpr strong_ordering operator<=(const shared_ptr<T>& a, nullptr_t) noexcept;

```

5 *Returns:*

```

    compare_three_way()(a.get(), static_cast<typename shared_ptr<T>::element_type*>(nullptr))

```

20.4.2.2.9 Specialized algorithms

[util.smartptr.shared.spec]

```

template<class T>
constexpr void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

```

1 *Effects:* Equivalent to a.swap(b).

20.4.2.2.10 Casts

[util.smartptr.shared.cast]

```

template<class T, class U>
constexpr shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;

```

```
template<class T, class U>
constexpr shared_ptr<T> static_pointer_cast(shared_ptr<U>&& r) noexcept;
```

1 *Mandates:* The expression `static_cast<T*>((U*)nullptr)` is well-formed.

2 *Returns:*

`shared_ptr<T>(R, static_cast<typename shared_ptr<T>::element_type*>(r.get()))`

where *R* is *r* for the first overload, and `std::move(r)` for the second.

3 [Note 1: The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. — end note]

```
template<class T, class U>
constexpr shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
constexpr shared_ptr<T> dynamic_pointer_cast(shared_ptr<U>&& r) noexcept;
```

4 *Mandates:* The expression `dynamic_cast<T*>((U*)nullptr)` is well-formed. The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` is well-formed.

5 *Preconditions:* The expression `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` has well-defined behavior.

6 *Returns:*

(6.1) — When `dynamic_cast<typename shared_ptr<T>::element_type*>(r.get())` returns a non-null value *p*, `shared_ptr<T>(R, p)`, where *R* is *r* for the first overload, and `std::move(r)` for the second.

(6.2) — Otherwise, `shared_ptr<T>()`.

7 [Note 2: The seemingly equivalent expression `shared_ptr<T>(dynamic_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. — end note]

```
template<class T, class U>
constexpr shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
constexpr shared_ptr<T> const_pointer_cast(shared_ptr<U>&& r) noexcept;
```

8 *Mandates:* The expression `const_cast<T*>((U*)nullptr)` is well-formed.

9 *Returns:*

`shared_ptr<T>(R, const_cast<typename shared_ptr<T>::element_type*>(r.get()))`

where *R* is *r* for the first overload, and `std::move(r)` for the second.

10 [Note 3: The seemingly equivalent expression `shared_ptr<T>(const_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. — end note]

```
template<class T, class U>
shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
template<class T, class U>
shared_ptr<T> reinterpret_pointer_cast(shared_ptr<U>&& r) noexcept;
```

11 *Mandates:* The expression `reinterpret_cast<T*>((U*)nullptr)` is well-formed.

12 *Returns:*

`shared_ptr<T>(R, reinterpret_cast<typename shared_ptr<T>::element_type*>(r.get()))`

where *R* is *r* for the first overload, and `std::move(r)` for the second.

13 [Note 4: The seemingly equivalent expression `shared_ptr<T>(reinterpret_cast<T*>(r.get()))` can result in undefined behavior, attempting to delete the same object twice. — end note]

20.4.2.2.11 get_deleter

[util.smartptr.getdeleter]

```
template<class D, class T>
constexpr D* get_deleter(const shared_ptr<T>& p) noexcept;
```

1 *Returns:* If *p* owns a deleter *d* of type cv-unqualified *D*, returns `addressof(d)`; otherwise returns `nullptr`. The returned pointer remains valid as long as there exists a `shared_ptr` instance that owns *d*.

[*Note 1*: It is unspecified whether the pointer remains valid longer than that. This can happen if the implementation doesn't destroy the deleter until all `weak_ptr` instances that share ownership with `p` have been destroyed. — *end note*]

20.4.2.2.12 I/O

[util.smartptr.shared.io]

```
template<class E, class T, class Y>
basic_ostream<E, T>& operator<<(basic_ostream<E, T>& os, const shared_ptr<Y>& p);
```

¹ *Effects*: As if by: `os << p.get();`

² *Returns*: `os`.

20.4.2.3 Class template `weak_ptr`

[util.smartptr.weak]

20.4.2.3.1 General

[util.smartptr.weak.general]

- ¹ The `weak_ptr` class template stores a weak reference to an object that is already managed by a `shared_ptr`. To access the object, a `weak_ptr` can be converted to a `shared_ptr` using the member function `lock`.

```
namespace std {
    template<class T> class weak_ptr {
    public:
        using element_type = remove_extent_t<T>;

        // 20.4.2.3.2, constructors
        constexpr weak_ptr() noexcept;
        template<class Y>
            constexpr weak_ptr(const shared_ptr<Y>& r) noexcept;
        constexpr weak_ptr(const weak_ptr& r) noexcept;
        template<class Y>
            constexpr weak_ptr(const weak_ptr<Y>& r) noexcept;
        constexpr weak_ptr(weak_ptr&& r) noexcept;
        template<class Y>
            constexpr weak_ptr(weak_ptr<Y>&& r) noexcept;

        // 20.4.2.3.3, destructor
        constexpr ~weak_ptr();

        // 20.4.2.3.4, assignment
        constexpr weak_ptr& operator=(const weak_ptr& r) noexcept;
        template<class Y>
            constexpr weak_ptr& operator=(const weak_ptr<Y>& r) noexcept;
        template<class Y>
            constexpr weak_ptr& operator=(const shared_ptr<Y>& r) noexcept;
        constexpr weak_ptr& operator=(weak_ptr&& r) noexcept;
        template<class Y>
            constexpr weak_ptr& operator=(weak_ptr<Y>&& r) noexcept;

        // 20.4.2.3.5, modifiers
        constexpr void swap(weak_ptr& r) noexcept;
        constexpr void reset() noexcept;

        // 20.4.2.3.6, observers
        constexpr long use_count() const noexcept;
        constexpr bool expired() const noexcept;
        constexpr shared_ptr<T> lock() const noexcept;
        template<class U>
            bool owner_before(const shared_ptr<U>& b) const noexcept;
        template<class U>
            bool owner_before(const weak_ptr<U>& b) const noexcept;
        size_t owner_hash() const noexcept;
        template<class U>
            constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
        template<class U>
            constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
    };
```

```

    template<class T>
        weak_ptr(shared_ptr<T>) -> weak_ptr<T>;
}

```

- 2 Specializations of `weak_ptr` shall be *Cpp17CopyConstructible* and *Cpp17CopyAssignable*, allowing their use in standard containers. The template parameter `T` of `weak_ptr` may be an incomplete type.

20.4.2.3.2 Constructors

[util.smartptr.weak.const]

```
constexpr weak_ptr() noexcept;
```

- 1 *Effects*: Constructs an empty `weak_ptr` object that stores a null pointer value.

- 2 *Postconditions*: `use_count() == 0`.

```

constexpr weak_ptr(const weak_ptr& r) noexcept;
template<class Y> constexpr weak_ptr(const weak_ptr<Y>& r) noexcept;
template<class Y> constexpr weak_ptr(const shared_ptr<Y>& r) noexcept;

```

- 3 *Constraints*: For the second and third constructors, `Y*` is compatible with `T*`.

- 4 *Effects*: If `r` is empty, constructs an empty `weak_ptr` object that stores a null pointer value; otherwise, constructs a `weak_ptr` object that shares ownership with `r` and stores a copy of the pointer stored in `r`.

- 5 *Postconditions*: `use_count() == r.use_count()`.

```

constexpr weak_ptr(weak_ptr&& r) noexcept;
template<class Y> constexpr weak_ptr(weak_ptr<Y>&& r) noexcept;

```

- 6 *Constraints*: For the second constructor, `Y*` is compatible with `T*`.

- 7 *Effects*: Move constructs a `weak_ptr` instance from `r`.

- 8 *Postconditions*: `*this` contains the old value of `r`. `r` is empty, stores a null pointer value, and `r.use_count() == 0`.

20.4.2.3.3 Destructor

[util.smartptr.weak.dest]

```
constexpr ~weak_ptr();
```

- 1 *Effects*: Destroys this `weak_ptr` object but has no effect on the object its stored pointer points to.

20.4.2.3.4 Assignment

[util.smartptr.weak.assign]

```

constexpr weak_ptr& operator=(const weak_ptr& r) noexcept;
template<class Y> constexpr weak_ptr& operator=(const weak_ptr<Y>& r) noexcept;
template<class Y> constexpr weak_ptr& operator=(const shared_ptr<Y>& r) noexcept;

```

- 1 *Effects*: Equivalent to `weak_ptr(r).swap(*this)`.

- 2 *Returns*: `*this`.

- 3 *Remarks*: The implementation may meet the effects (and the implied guarantees) via different means, without creating a temporary object.

```

constexpr weak_ptr& operator=(weak_ptr&& r) noexcept;
template<class Y> constexpr weak_ptr& operator=(weak_ptr<Y>&& r) noexcept;

```

- 4 *Effects*: Equivalent to `weak_ptr(std::move(r)).swap(*this)`.

- 5 *Returns*: `*this`.

20.4.2.3.5 Modifiers

[util.smartptr.weak.mod]

```
constexpr void swap(weak_ptr& r) noexcept;
```

- 1 *Effects*: Exchanges the contents of `*this` and `r`.

```
constexpr void reset() noexcept;
```

- 2 *Effects*: Equivalent to `weak_ptr().swap(*this)`.

20.4.2.3.6 Observers

[util.smartptr.weak.obs]

```
constexpr long use_count() const noexcept;
```

- 1 *Returns:* 0 if **this* is empty; otherwise, the number of `shared_ptr` instances that share ownership with **this*.

```
constexpr bool expired() const noexcept;
```

- 2 *Returns:* `use_count() == 0`.

```
constexpr shared_ptr<T> lock() const noexcept;
```

- 3 *Returns:* `expired() ? shared_ptr<T>() : shared_ptr<T>(*this)`, executed atomically.

```
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
```

```
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

- 4 *Returns:* An unspecified value such that

(4.1) — `owner_before(b)` defines a strict weak ordering as defined in 26.8;

(4.2) — `!owner_before(b) && !b.owner_before(*this)` is true if and only if `owner_equal(b)` is true.

```
size_t owner_hash() const noexcept;
```

- 5 *Returns:* An unspecified value such that, for any object *x* where `owner_equal(x)` is true, `owner_hash() == x.owner_hash()` is true.

```
template<class U>
```

```
constexpr bool owner_equal(const shared_ptr<U>& b) const noexcept;
```

```
template<class U>
```

```
constexpr bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

- 6 *Returns:* true if and only if **this* and *b* share ownership or are both empty. Otherwise returns false.

- 7 *Remarks:* `owner_equal` is an equivalence relation.

20.4.2.3.7 Specialized algorithms

[util.smartptr.weak.spec]

```
template<class T>
```

```
constexpr void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;
```

- 1 *Effects:* Equivalent to `a.swap(b)`.

20.4.2.4 Class template `owner_less`

[util.smartptr.ownerless]

- 1 The class template `owner_less` allows ownership-based mixed comparisons of shared and weak pointers.

```
namespace std {
```

```
template<class T = void> struct owner_less;
```

```
template<class T> struct owner_less<shared_ptr<T>> {
```

```
bool operator()(const shared_ptr<T>&, const shared_ptr<T>&) const noexcept;
```

```
bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
```

```
bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
```

```
};
```

```
template<class T> struct owner_less<weak_ptr<T>> {
```

```
bool operator()(const weak_ptr<T>&, const weak_ptr<T>&) const noexcept;
```

```
bool operator()(const shared_ptr<T>&, const weak_ptr<T>&) const noexcept;
```

```
bool operator()(const weak_ptr<T>&, const shared_ptr<T>&) const noexcept;
```

```
};
```

```
template<> struct owner_less<void> {
```

```
template<class T, class U>
```

```
bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;
```

```
template<class T, class U>
```

```
bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;
```

```
template<class T, class U>
```

```
bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;
```

```

    template<class T, class U>
        bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;

    using is_transparent = unspecified;
};
}

```

² `operator()(x, y)` returns `x.owner_before(y)`.

[Note 1: Note that

- (2.1) — `operator()` defines a strict weak ordering as defined in 26.8;
 - (2.2) — `!operator()(a, b) && !operator()(b, a)` is true if and only if `a.owner_equal(b)` is true.
- end note]

20.4.2.5 Struct `owner_hash`

[`util.smartptr.owner.hash`]

¹ The class `owner_hash` provides ownership-based hashing.

```

namespace std {
    struct owner_hash {
        template<class T>
            size_t operator()(const shared_ptr<T>&) const noexcept;

        template<class T>
            size_t operator()(const weak_ptr<T>&) const noexcept;

        using is_transparent = unspecified;
    };
}

template<class T>
    size_t operator()(const shared_ptr<T>& x) const noexcept;
template<class T>
    size_t operator()(const weak_ptr<T>& x) const noexcept;

```

² Returns: `x.owner_hash()`.

³ [Note 1: For any object `y` where `x.owner_equal(y)` is true, `x.owner_hash() == y.owner_hash()` is true.
— end note]

20.4.2.6 Struct `owner_equal`

[`util.smartptr.owner.equal`]

¹ The class `owner_equal` provides ownership-based mixed equality comparisons of shared and weak pointers.

```

namespace std {
    struct owner_equal {
        template<class T, class U>
            constexpr bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;
        template<class T, class U>
            constexpr bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;
        template<class T, class U>
            constexpr bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;
        template<class T, class U>
            constexpr bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;

        using is_transparent = unspecified;
    };
}

template<class T, class U>
    constexpr bool operator()(const shared_ptr<T>& x, const shared_ptr<U>& y) const noexcept;
template<class T, class U>
    constexpr bool operator()(const shared_ptr<T>& x, const weak_ptr<U>& y) const noexcept;
template<class T, class U>
    constexpr bool operator()(const weak_ptr<T>& x, const shared_ptr<U>& y) const noexcept;
template<class T, class U>
    constexpr bool operator()(const weak_ptr<T>& x, const weak_ptr<U>& y) const noexcept;

```

² Returns: `x.owner_equal(y)`.

3 [Note 1: `x.owner_equal(y)` is true if and only if `x` and `y` share ownership or are both empty. — end note]

20.4.2.7 Class template `enable_shared_from_this` [util.smartptr.enab]

1 A class `T` can inherit from `enable_shared_from_this<T>` to inherit the `shared_from_this` member functions that obtain a `shared_ptr` instance pointing to `*this`.

2 [Example 1:

```
struct X: public enable_shared_from_this<X> { };

int main() {
    shared_ptr<X> p(new X);
    shared_ptr<X> q = p->shared_from_this();
    assert(p == q);
    assert(p.owner_equal(q));           // p and q share ownership
}
```

— end example]

```
namespace std {
    template<class T> class enable_shared_from_this {
    protected:
        constexpr enable_shared_from_this() noexcept;
        constexpr enable_shared_from_this(const enable_shared_from_this&) noexcept;
        constexpr enable_shared_from_this& operator=(const enable_shared_from_this&) noexcept;
        constexpr ~enable_shared_from_this();

    public:
        constexpr shared_ptr<T> shared_from_this();
        constexpr shared_ptr<T const> shared_from_this() const;
        constexpr weak_ptr<T> weak_from_this() noexcept;
        constexpr weak_ptr<T const> weak_from_this() const noexcept;

    private:
        mutable weak_ptr<T> weak_this; // exposition only
    };
}
```

3 The template parameter `T` of `enable_shared_from_this` may be an incomplete type.

```
constexpr enable_shared_from_this() noexcept;
constexpr enable_shared_from_this(const enable_shared_from_this<T>&) noexcept;
```

4 *Effects:* Value-initializes *weak_this*.

```
constexpr enable_shared_from_this<T>& operator=(const enable_shared_from_this<T>&) noexcept;
```

5 *Returns:* `*this`.

6 [Note 1: *weak_this* is not changed. — end note]

```
constexpr shared_ptr<T> shared_from_this();
constexpr shared_ptr<T const> shared_from_this() const;
```

7 *Returns:* `shared_ptr<T>(weak_this)`.

```
constexpr weak_ptr<T> weak_from_this() noexcept;
constexpr weak_ptr<T const> weak_from_this() const noexcept;
```

8 *Returns:* *weak_this*.

20.4.3 Smart pointer hash support [util.smartptr.hash]

```
template<class T, class D> struct hash<unique_ptr<T, D>>;
```

1 Letting `UP` be `unique_ptr<T, D>`, the specialization `hash<UP>` is enabled (22.10.19) if and only if `hash<typename UP::pointer>` is enabled. When enabled, for an object `p` of type `UP`, `hash<UP>()(p)` evaluates to the same value as `hash<typename UP::pointer>()(p.get())`. The member functions are not guaranteed to be `noexcept`.

```
template<class T> struct hash<shared_ptr<T>>;
```

- 2 For an object `p` of type `shared_ptr<T>`, `hash<shared_ptr<T>>()(p)` evaluates to the same value as `hash<typename shared_ptr<T>::element_type*>()(p.get())`.

20.4.4 Smart pointer adaptors

[smartptr.adapt]

20.4.4.1 Class template `out_ptr_t`

[out.ptr.t]

- 1 `out_ptr_t` is a class template used to adapt types such as smart pointers (20.4) for functions that use output pointer parameters.

- 2 [Example 1:

```
#include <memory>
#include <cstdio>

int fopen_s(std::FILE** f, const char* name, const char* mode);

struct fclose_deleter {
    void operator()(std::FILE* f) const noexcept {
        std::fclose(f);
    }
};

int main(int, char*[]) {
    constexpr const char* file_name = "ow.o";
    std::unique_ptr<std::FILE, fclose_deleter> file_ptr;
    int err = fopen_s(std::out_ptr<std::FILE*>(file_ptr), file_name, "r+b");
    if (err != 0)
        return 1;
    // *file_ptr is valid
    return 0;
}
```

`unique_ptr` can be used with `out_ptr` to be passed into an output pointer-style function, without needing to hold onto an intermediate pointer value and manually delete it on error or failure. — end example]

```
namespace std {
    template<class Smart, class Pointer, class... Args>
    class out_ptr_t {
    public:
        constexpr explicit out_ptr_t(Smart&, Args...);
        out_ptr_t(const out_ptr_t&) = delete;

        constexpr ~out_ptr_t();

        constexpr operator Pointer*() const noexcept;
        operator void**() const noexcept;

    private:
        Smart& s;                // exposition only
        tuple<Args...> a;        // exposition only
        Pointer p;               // exposition only
    };
}
```

- 3 `Pointer` shall meet the *Cpp17NullablePointer* requirements. If `Smart` is a specialization of `shared_ptr` and `sizeof...(Args) == 0`, the program is ill-formed.

[Note 1: It is typically a user error to reset a `shared_ptr` without specifying a deleter, as `shared_ptr` will replace a custom deleter upon usage of `reset`, as specified in 20.4.2.2.5. — end note]

- 4 Program-defined specializations of `out_ptr_t` that depend on at least one program-defined type need not meet the requirements for the primary template.
- 5 Evaluations of the conversion functions on the same object may conflict (6.10.2.2).

```
constexpr explicit out_ptr_t(Smart& smart, Args... args);
```

6 *Effects:* Initializes `s` with `smart`, `a` with `std::forward<Args>(args)...`, and value-initializes `p`. Then, equivalent to:

(6.1) — `s.reset()`;
if the expression `s.reset()` is well-formed;

(6.2) — otherwise,
`s = Smart()`;
if `is_constructible_v<Smart>` is true;

(6.3) — otherwise, the program is ill-formed.

7 [Note 2: The constructor is not `noexcept` to allow for a variety of non-terminating and safe implementation strategies. For example, an implementation can allocate a `shared_ptr`'s internal node in the constructor and let implementation-defined exceptions escape safely. The destructor can then move the allocated control block in directly and avoid any other exceptions. — end note]

```
constexpr ~out_ptr_t();
```

8 Let `SP` be `POINTER_OF_OR(Smart, Pointer)` (20.2.1).

9 *Effects:* Equivalent to:

(9.1) — if `(p)` {
 `apply([&](auto&&... args) {`
 `s.reset(static_cast<SP>(p), std::forward<Args>(args)...); }`, `std::move(a)`);
 }
if the expression `s.reset(static_cast<SP>(p), std::forward<Args>(args)...)` is well-formed;

(9.2) — otherwise,
 if `(p)` {
 `apply([&](auto&&... args) {`
 `s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }`, `std::move(a)`);
 }
 if `is_constructible_v<Smart, SP, Args...>` is true;

(9.3) — otherwise, the program is ill-formed.

```
constexpr operator Pointer*() const noexcept;
```

10 *Preconditions:* `operator void**()` has not been called on `*this`.

11 *Returns:* `addressof(const_cast<Pointer&>(p))`.

```
operator void**() const noexcept;
```

12 *Constraints:* `is_same_v<Pointer, void*>` is false.

13 *Mandates:* `is_pointer_v<Pointer>` is true.

14 *Preconditions:* `operator Pointer*()` has not been called on `*this`.

15 *Returns:* A pointer value `v` such that:

(15.1) — the initial value `*v` is equivalent to `static_cast<void*>(p)` and

(15.2) — any modification of `*v` that is not followed by a subsequent modification of `*this` affects the value of `p` during the destruction of `*this`, such that `static_cast<void*>(p) == *v`.

16 *Remarks:* Accessing `*v` outside the lifetime of `*this` has undefined behavior.

17 [Note 3: `reinterpret_cast<void**>(static_cast<Pointer*>(*this))` can be a viable implementation strategy for some implementations. — end note]

20.4.4.2 Function template `out_ptr`

[out_ptr]

```
template<class Pointer = void, class Smart, class... Args>
constexpr auto out_ptr(Smart& s, Args&&... args);
```

1 Let `P` be `Pointer` if `is_void_v<Pointer>` is false, otherwise `POINTER_OF(Smart)`.

² *Returns:* `out_ptr_t<Smart, P, Args&&...>(s, std::forward<Args>(args)...) .`

20.4.4.3 Class template `inout_ptr_t`

[`inout_ptr.t`]

¹ `inout_ptr_t` is a class template used to adapt types such as smart pointers (20.4) for functions that use output pointer parameters whose dereferenced values may first be deleted before being set to another allocated value.

² [Example 1:

```
#include <memory>

struct star_fish* star_fish_alloc();
int star_fish_populate(struct star_fish** ps, const char* description);

struct star_fish_deleter {
    void operator() (struct star_fish* c) const noexcept;
};

using star_fish_ptr = std::unique_ptr<star_fish, star_fish_deleter>;

int main(int, char*[]) {
    star_fish_ptr peach(star_fish_alloc());
    // ...
    // used, need to re-make
    int err = star_fish_populate(std::inout_ptr(peach), "caring clown-fish liker");
    return err;
}
```

A `unique_ptr` can be used with `inout_ptr` to be passed into an output pointer-style function. The original value will be properly deleted according to the function it is used with and a new value reset in its place. — *end example*]

```
namespace std {
    template<class Smart, class Pointer, class... Args>
    class inout_ptr_t {
    public:
        constexpr explicit inout_ptr_t(Smart&, Args...);
        inout_ptr_t(const inout_ptr_t&) = delete;

        constexpr ~inout_ptr_t();

        constexpr operator Pointer*() const noexcept;
        operator void**() const noexcept;

    private:
        Smart& s;                // exposition only
        tuple<Args...> a;         // exposition only
        Pointer p;                // exposition only
    };
}
```

³ `Pointer` shall meet the *Cpp17NullablePointer* requirements. If `Smart` is a specialization of `shared_ptr`, the program is ill-formed.

[*Note 1:* It is impossible to properly acquire unique ownership of the managed resource from a `shared_ptr` given its shared ownership model. — *end note*]

⁴ Program-defined specializations of `inout_ptr_t` that depend on at least one program-defined type need not meet the requirements for the primary template.

⁵ Evaluations of the conversion functions on the same object may conflict (6.10.2.2).

```
constexpr explicit inout_ptr_t(Smart& smart, Args... args);
```

⁶ *Effects:* Initializes `s` with `smart`, `a` with `std::forward<Args>(args)...`, and `p` to either

(6.1) — `smart` if `is_pointer_v<Smart>` is true,

(6.2) — otherwise, `smart.get()`.

7 *Remarks:* An implementation can call `s.release()`.

8 [Note 2: The constructor is not `noexcept` to allow for a variety of non-terminating and safe implementation strategies. For example, an intrusive pointer implementation with a control block can allocate in the constructor and safely fail with an exception. — end note]

```
constexpr ~inout_ptr_t();
```

9 Let SP be *POINTER_OF_OR*(Smart, Pointer) (20.2.1).

10 Let *release-statement* be `s.release()`; if an implementation does not call `s.release()` in the constructor. Otherwise, it is empty.

11 *Effects:* Equivalent to:

- (11.1) — `apply([&](auto&&... args) {
 s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));`
 if `is_pointer_v<Smart>` is true;
- (11.2) — otherwise,
 release-statement;
 if (p) {
 `apply([&](auto&&... args) {
 s.reset(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));`
 }
 if the expression `s.reset(static_cast<SP>(p), std::forward<Args>(args)...)` is well-formed;
- (11.3) — otherwise,
 release-statement;
 if (p) {
 `apply([&](auto&&... args) {
 s = Smart(static_cast<SP>(p), std::forward<Args>(args)...); }, std::move(a));`
 }
 if `is_constructible_v<Smart, SP, Args...>` is true;
- (11.4) — otherwise, the program is ill-formed.

```
constexpr operator Pointer*() const noexcept;
```

12 *Preconditions:* `operator void**()` has not been called on `*this`.

13 *Returns:* `addressof(const_cast<Pointer&>(p))`.

```
operator void**() const noexcept;
```

14 *Constraints:* `is_same_v<Pointer, void*>` is false.

15 *Mandates:* `is_pointer_v<Pointer>` is true.

16 *Preconditions:* `operator Pointer*()` has not been called on `*this`.

17 *Returns:* A pointer value `v` such that:

- (17.1) — the initial value `*v` is equivalent to `static_cast<void*>(p)` and
- (17.2) — any modification of `*v` that is not followed by subsequent modification of `*this` affects the value of `p` during the destruction of `*this`, such that `static_cast<void*>(p) == *v`.

18 *Remarks:* Accessing `*v` outside the lifetime of `*this` has undefined behavior.

19 [Note 3: `reinterpret_cast<void**>(static_cast<Pointer*>(*this))` can be a viable implementation strategy for some implementations. — end note]

20.4.4.4 Function template `inout_ptr`

[inout_ptr]

```
template<class Pointer = void, class Smart, class... Args>  
constexpr auto inout_ptr(Smart& s, Args&&... args);
```

1 Let P be Pointer if `is_void_v<Pointer>` is false, otherwise *POINTER_OF*(Smart).

2 *Returns:* `inout_ptr_t<Smart, P, Args&&...>(s, std::forward<Args>(args)...)`.

20.5 Types for composite class design

[mem.composite.types]

20.5.1 Class template indirect

[indirect]

20.5.1.1 General

[indirect.general]

- ¹ An indirect object manages the lifetime of an owned object. An indirect object is *valueless* if it has no owned object. An indirect object may become valueless only after it has been moved from.
- ² In every specialization `indirect<T, Allocator>`, if the type `allocator_traits<Allocator>::value_type` is not the same type as `T`, the program is ill-formed. Every object of type `indirect<T, Allocator>` uses an object of type `Allocator` to allocate and free storage for the owned object as needed.
- ³ Constructing an owned object with `args...` using the allocator `a` means calling `allocator_traits<Allocator>::construct(a, p, args...)` where `args` is an expression pack, `a` is an allocator, and `p` is a pointer obtained by calling `allocator_traits<Allocator>::allocate`.
- ⁴ The member `alloc` is used for any memory allocation and element construction performed by member functions during the lifetime of each indirect object. The allocator `alloc` may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if (23.2.2.2):
 - (4.1) — `allocator_traits<Allocator>::propagate_on_container_copy_assignment::value`, or
 - (4.2) — `allocator_traits<Allocator>::propagate_on_container_move_assignment::value`, or
 - (4.3) — `allocator_traits<Allocator>::propagate_on_container_swap::value`
 is `true` within the implementation of the corresponding `indirect` operation.
- ⁵ A program that instantiates the definition of the template `indirect<T, Allocator>` with a type for the `T` parameter that is a non-object type, an array type, `in_place_t`, a specialization of `in_place_type_t`, or a cv-qualified type is ill-formed.
- ⁶ The template parameter `T` of `indirect` may be an incomplete type.
- ⁷ The template parameter `Allocator` of `indirect` shall meet the *Cpp17Allocator* requirements.
- ⁸ If a program declares an explicit or partial specialization of `indirect`, the behavior is undefined.

20.5.1.2 Synopsis

[indirect.syn]

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class indirect {
    public:
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = allocator_traits<Allocator>::pointer;
        using const_pointer = allocator_traits<Allocator>::const_pointer;

        // 20.5.1.3, constructors
        constexpr explicit indirect();
        constexpr explicit indirect(allocator_arg_t, const Allocator& a);
        constexpr indirect(const indirect& other);
        constexpr indirect(allocator_arg_t, const Allocator& a, const indirect& other);
        constexpr indirect(indirect&& other) noexcept;
        constexpr indirect(allocator_arg_t, const Allocator& a, indirect&& other)
            noexcept(see below);
        template<class U = T>
            constexpr explicit indirect(U&& u);
        template<class U = T>
            constexpr explicit indirect(allocator_arg_t, const Allocator& a, U&& u);
        template<class... Us>
            constexpr explicit indirect(in_place_t, Us&&... us);
        template<class... Us>
            constexpr explicit indirect(allocator_arg_t, const Allocator& a,
                                      in_place_t, Us&&... us);
        template<class I, class... Us>
            constexpr explicit indirect(in_place_t, initializer_list<I> ilist, Us&&... us);
    };

```

```

template<class I, class... Us>
    constexpr explicit indirect(allocator_arg_t, const Allocator& a,
                                in_place_t, initializer_list<I> ilist, Us&&... us);

// 20.5.1.4, destructor
constexpr ~indirect();

// 20.5.1.5, assignment
constexpr indirect& operator=(const indirect& other);
constexpr indirect& operator=(indirect&& other) noexcept(see below);
template<class U = T>
    constexpr indirect& operator=(U&& u);

// 20.5.1.6, observers
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
constexpr const T&& operator*() const && noexcept;
constexpr T&& operator*() && noexcept;
constexpr const_pointer operator->() const noexcept;
constexpr pointer operator->() noexcept;
constexpr bool valueless_after_move() const noexcept;
constexpr allocator_type get_allocator() const noexcept;

// 20.5.1.7, swap
constexpr void swap(indirect& other) noexcept(see below);
friend constexpr void swap(indirect& lhs, indirect& rhs) noexcept(see below);

// 20.5.1.8, relational operators
template<class U, class AA>
    friend constexpr bool operator==(const indirect& lhs, const indirect<U, AA>& rhs)
        noexcept(see below);
template<class U, class AA>
    friend constexpr auto operator<=>(const indirect& lhs, const indirect<U, AA>& rhs)
        -> synth-three-way-result<T, U>;

// 20.5.1.9, comparison with T
template<class U>
    friend constexpr bool operator==(const indirect& lhs, const U& rhs) noexcept(see below);
template<class U>
    friend constexpr auto operator<=>(const indirect& lhs, const U& rhs)
        -> synth-three-way-result<T, U>;

private:
    pointer p; // exposition only
    Allocator alloc = Allocator(); // exposition only
};
template<class Value>
    indirect(Value) -> indirect<Value>;
template<class Allocator, class Value>
    indirect(allocator_arg_t, Allocator, Value)
        -> indirect<Value, typename allocator_traits<Allocator>::template rebind_alloc<Value>>;
}

```

20.5.1.3 Constructors

[indirect.ctor]

1 The following element applies to all functions in 20.5.1.3:

2 *Throws:* Nothing unless `allocator_traits<Allocator>::allocate` or `allocator_traits<Allocator>::construct` throws.

```
constexpr explicit indirect();
```

3 *Constraints:* `is_default_constructible_v<Allocator>` is true.

4 *Mandates:* `is_default_constructible_v<T>` is true.

5 *Effects:* Constructs an owned object of type `T` with an empty argument list, using the allocator `alloc`.

```
constexpr explicit indirect(allocator_arg_t, const Allocator& a);
```

6 *Mandates:* `is_default_constructible_v<T>` is true.

7 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `T` with an empty argument list, using the allocator `alloc`.

```
constexpr indirect(const indirect& other);
```

8 *Mandates:* `is_copy_constructible_v<T>` is true.

9 *Effects:* `alloc` is direct-non-list-initialized with `allocator_traits<Allocator>::select_on_container_copy_construction(other.alloc)`. If `other` is valueless, `*this` is valueless. Otherwise, constructs an owned object of type `T` with `*other`, using the allocator `alloc`.

```
constexpr indirect(allocator_arg_t, const Allocator& a, const indirect& other);
```

10 *Mandates:* `is_copy_constructible_v<T>` is true.

11 *Effects:* `alloc` is direct-non-list-initialized with `a`. If `other` is valueless, `*this` is valueless. Otherwise, constructs an owned object of type `T` with `*other`, using the allocator `alloc`.

```
constexpr indirect(indirect&& other) noexcept;
```

12 *Effects:* `alloc` is direct-non-list-initialized from `std::move(other.alloc)`. If `other` is valueless, `*this` is valueless. Otherwise `*this` takes ownership of the owned object of `other`.

13 *Postconditions:* `other` is valueless.

```
constexpr indirect(allocator_arg_t, const Allocator& a, indirect&& other)
noexcept(allocator_traits<Allocator>::is_always_equal::value);
```

14 *Mandates:* If `allocator_traits<Allocator>::is_always_equal::value` is false then `T` is a complete type.

15 *Effects:* `alloc` is direct-non-list-initialized with `a`. If `other` is valueless, `*this` is valueless. Otherwise, if `alloc == other.alloc` is true, constructs an object of type `indirect` that takes ownership of the owned object of `other`. Otherwise, constructs an owned object of type `T` with `*std::move(other)`, using the allocator `alloc`.

16 *Postconditions:* `other` is valueless.

```
template<class U = T>
constexpr explicit indirect(U&& u);
```

17 *Constraints:*

(17.1) — `is_same_v<remove_cvref_t<U>, indirect>` is false,

(17.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false,

(17.3) — `is_constructible_v<T, U>` is true, and

(17.4) — `is_default_constructible_v<Allocator>` is true.

18 *Effects:* Constructs an owned object of type `T` with `std::forward<U>(u)`, using the allocator `alloc`.

```
template<class U = T>
constexpr explicit indirect(allocator_arg_t, const Allocator& a, U&& u);
```

19 *Constraints:*

(19.1) — `is_same_v<remove_cvref_t<U>, indirect>` is false,

(19.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false, and

(19.3) — `is_constructible_v<T, U>` is true.

20 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `T` with `std::forward<U>(u)`, using the allocator `alloc`.

```
template<class... Us>
constexpr explicit indirect(in_place_t, Us&&... us);
```

21 *Constraints:*

(21.1) — `is_constructible_v<T, Us...>` is true, and

(21.2) — `is_default_constructible_v<Allocator>` is true.

22 *Effects:* Constructs an owned object of type `T` with `std::forward<Us>(us)...`, using the allocator `alloc`.

```
template<class... Us>
constexpr explicit indirect(allocator_arg_t, const Allocator& a,
                           in_place_t, Us&& ...us);
```

23 *Constraints:* `is_constructible_v<T, Us...>` is true.

24 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `T` with `std::forward<Us>(us)...`, using the allocator `alloc`.

```
template<class I, class... Us>
constexpr explicit indirect(in_place_t, initializer_list<I> ilist, Us&&... us);
```

25 *Constraints:*

(25.1) — `is_constructible_v<T, initializer_list<I>&, Us...>` is true, and

(25.2) — `is_default_constructible_v<Allocator>` is true.

26 *Effects:* Constructs an owned object of type `T` with the arguments `ilist`, `std::forward<Us>(us)...`, using the allocator `alloc`.

```
template<class I, class... Us>
constexpr explicit indirect(allocator_arg_t, const Allocator& a,
                           in_place_t, initializer_list<I> ilist, Us&&... us);
```

27 *Constraints:* `is_constructible_v<T, initializer_list<I>&, Us...>` is true.

28 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `T` with the arguments `ilist`, `std::forward<Us>(us)...`, using the allocator `alloc`.

20.5.1.4 Destructor

[indirect.dtor]

```
constexpr ~indirect();
```

1 *Mandates:* `T` is a complete type.

2 *Effects:* If `*this` is not valueless, destroys the owned object using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.

20.5.1.5 Assignment

[indirect.assign]

```
constexpr indirect& operator=(const indirect& other);
```

1 *Mandates:*

(1.1) — `is_copy_assignable_v<T>` is true, and

(1.2) — `is_copy_constructible_v<T>` is true.

2 *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

(2.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_copy_assignment::value` is true.

(2.2) — If `other` is valueless, `*this` becomes valueless and the owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.

(2.3) — Otherwise, if `alloc == other.alloc` is true and `*this` is not valueless, equivalent to `**this = *other`.

(2.4) — Otherwise a new owned object is constructed in `*this` using `allocator_traits<Allocator>::construct` with the owned object from `other` as the argument, using either the allocator in `*this` or the allocator in `other` if the allocator needs updating.

(2.5) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.

(2.6) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

3 *Returns:* A reference to `*this`.

4 *Remarks:* If any exception is thrown, the result of the expression `this->valueless_after_move()` remains unchanged. If an exception is thrown during the call to `T`'s selected copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its owned object is as defined by the exception safety guarantee of `T`'s copy assignment.

```
constexpr indirect& operator=(indirect&& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
```

5 *Mandates:* If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is false and `allocator_traits<Allocator>::is_always_equal::value` is false, `is_move_constructible_v<T>` is true.

6 *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

- (6.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is true.
- (6.2) — If `other` is valueless, `*this` becomes valueless.
- (6.3) — Otherwise, if the allocator needs updating or if `alloc == other.alloc` is true, `*this` takes ownership of the owned object of `other`.
- (6.4) — Otherwise, constructs a new owned object with the owned object of `other` as the argument as an rvalue, using the allocator in `*this`.
- (6.5) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
- (6.6) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

7 *Postconditions:* `other` is valueless.

8 *Returns:* A reference to `*this`.

9 *Remarks:* If any exception is thrown, there are no effects on `*this` or `other`.

```
template<class U = T>
    constexpr indirect& operator=(U&& u);
```

10 *Constraints:*

- (10.1) — `is_same_v<remove_cvref_t<U>, indirect>` is false,
- (10.2) — `is_constructible_v<T, U>` is true, and
- (10.3) — `is_assignable_v<T&, U>` is true.

11 *Effects:* If `*this` is valueless then constructs an owned object of type `T` with `std::forward<U>(u)` using the allocator `alloc`. Otherwise, equivalent to `**this = std::forward<U>(u)`.

12 *Returns:* A reference to `*this`.

20.5.1.6 Observers

[indirect.obs]

```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

1 *Preconditions:* `*this` is not valueless.

2 *Returns:* `*p`.

```
constexpr const T&& operator*() const && noexcept;
constexpr T&& operator*() && noexcept;
```

3 *Preconditions:* `*this` is not valueless.

4 *Returns:* `std::move(*p)`.

```
constexpr const_pointer operator->() const noexcept;
constexpr pointer operator->() noexcept;
```

5 *Preconditions:* `*this` is not valueless.

6 *Returns:* *p*.

```
constexpr bool valueless_after_move() const noexcept;
```

7 *Returns:* true if **this* is valueless, otherwise false.

```
constexpr allocator_type get_allocator() const noexcept;
```

8 *Returns:* *alloc*.

20.5.1.7 Swap

[indirect.swap]

```
constexpr void swap(indirect& other)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

1 *Preconditions:* If `allocator_traits<Allocator>::propagate_on_container_swap::value` is true, then `Allocator` meets the *Cpp17Swappable* requirements. Otherwise `get_allocator() == other.get_allocator()` is true.

2 *Effects:* Swaps the states of **this* and *other*, exchanging owned objects or valueless states. If `allocator_traits<Allocator>::propagate_on_container_swap::value` is true, then the allocators of **this* and *other* are exchanged by calling `swap` as described in 16.4.4.3. Otherwise, the allocators are not swapped.

[Note 1: Does not call `swap` on the owned objects directly. — end note]

```
constexpr void swap(indirect& lhs, indirect& rhs) noexcept(noexcept(lhs.swap(rhs)));
```

3 *Effects:* Equivalent to `lhs.swap(rhs)`.

20.5.1.8 Relational operators

[indirect.relops]

```
template<class U, class AA>
constexpr bool operator==(const indirect& lhs, const indirect<U, AA>& rhs)
noexcept(noexcept(bool(*lhs == *rhs)));
```

1 *Mandates:* The expression `*lhs == *rhs` is well-formed and its result is convertible to `bool`.

2 *Returns:* If *lhs* is valueless or *rhs* is valueless, `lhs.valueless_after_move() == rhs.valueless_after_move()`; otherwise `*lhs == *rhs`.

```
template<class U, class AA>
constexpr synth-three-way-result<T, U>
operator<=>(const indirect& lhs, const indirect<U, AA>& rhs);
```

3 *Returns:* If *lhs* is valueless or *rhs* is valueless, `!lhs.valueless_after_move() <=> !rhs.valueless_after_move()`; otherwise `synth-three-way(*lhs, *rhs)`.

20.5.1.9 Comparison with T

[indirect.comp.with.t]

```
template<class U>
constexpr bool operator==(const indirect& lhs, const U& rhs)
noexcept(noexcept(bool(*lhs == rhs)));
```

1 *Mandates:* The expression `*lhs == rhs` is well-formed and its result is convertible to `bool`.

2 *Returns:* If *lhs* is valueless, false; otherwise `*lhs == rhs`.

```
template<class U>
constexpr synth-three-way-result<T, U>
operator<=>(const indirect& lhs, const U& rhs);
```

3 *Returns:* If *lhs* is valueless, `strong_ordering::less`; otherwise `synth-three-way(*lhs, rhs)`.

20.5.1.10 Hash support

[indirect.hash]

```
template<class T, class Allocator>
struct hash<indirect<T, Allocator>>;
```

1 The specialization `hash<indirect<T, Allocator>>` is enabled (22.10.19) if and only if `hash<T>` is enabled. When enabled for an object *i* of type `indirect<T, Allocator>`, `hash<indirect<T,`

`Allocator>>()` (*i*) evaluates to either the same value as `hash<T>()(*i)`, if *i* is not valueless; otherwise to an implementation-defined value. The member functions are not guaranteed to be `noexcept`.

20.5.2 Class template `polymorphic`

[polymorphic]

20.5.2.1 General

[polymorphic.general]

- ¹ A polymorphic object manages the lifetime of an owned object. A polymorphic object may own objects of different types at different points in its lifetime. A polymorphic object is *valueless* if it has no owned object. A polymorphic object may become valueless only after it has been moved from.
- ² In every specialization `polymorphic<T, Allocator>`, if the type `allocator_traits<Allocator>::value_type` is not the same type as `T`, the program is ill-formed. Every object of type `polymorphic<T, Allocator>` uses an object of type `Allocator` to allocate and free storage for the owned object as needed.
- ³ Constructing an owned object of type `U` with `args...` using the allocator `a` means calling `allocator_traits<Allocator>::construct(a, p, args...)` where `args` is an expression pack, `a` is an allocator, and `p` points to storage suitable for an owned object of type `U`.
- ⁴ The member `alloc` is used for any memory allocation and element construction performed by member functions during the lifetime of each polymorphic value object, or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if (23.2.2.2):
 - (4.1) — `allocator_traits<Allocator>::propagate_on_container_copy_assignment::value`, or
 - (4.2) — `allocator_traits<Allocator>::propagate_on_container_move_assignment::value`, or
 - (4.3) — `allocator_traits<Allocator>::propagate_on_container_swap::value`
 is `true` within the implementation of the corresponding polymorphic operation.
- ⁵ A program that instantiates the definition of `polymorphic` for a non-object type, an array type, `in_place_t`, a specialization of `in_place_type_t`, or a cv-qualified type is ill-formed.
- ⁶ The template parameter `T` of `polymorphic` may be an incomplete type.
- ⁷ The template parameter `Allocator` of `polymorphic` shall meet the requirements of *Cpp17Allocator*.
- ⁸ If a program declares an explicit or partial specialization of `polymorphic`, the behavior is undefined.

20.5.2.2 Synopsis

[polymorphic.syn]

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class polymorphic {
    public:
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = allocator_traits<Allocator>::pointer;
        using const_pointer = allocator_traits<Allocator>::const_pointer;

        // 20.5.2.3, constructors
        constexpr explicit polymorphic();
        constexpr explicit polymorphic(allocator_arg_t, const Allocator& a);
        constexpr polymorphic(const polymorphic& other);
        constexpr polymorphic(allocator_arg_t, const Allocator& a, const polymorphic& other);
        constexpr polymorphic(polymorphic&& other) noexcept;
        constexpr polymorphic(allocator_arg_t, const Allocator& a, polymorphic&& other)
            noexcept(see below);
        template<class U = T>
            constexpr explicit polymorphic(U&& u);
        template<class U = T>
            constexpr explicit polymorphic(allocator_arg_t, const Allocator& a, U&& u);
        template<class U, class... Ts>
            constexpr explicit polymorphic(in_place_type_t<U>, Ts&&... ts);
        template<class U, class... Ts>
            constexpr explicit polymorphic(allocator_arg_t, const Allocator& a,
                                           in_place_type_t<U>, Ts&&... ts);
        template<class U, class I, class... Us>
            constexpr explicit polymorphic(in_place_type_t<U>, initializer_list<I> ilist, Us&&... us);
```



```

template<class U, class I, class... Us>
    constexpr explicit polymorphic(allocator_arg_t, const Allocator& a,
                                    in_place_type_t<U>, initializer_list<I> ilist, Us&&... us);

// 20.5.2.4, destructor
constexpr ~polymorphic();

// 20.5.2.5, assignment
constexpr polymorphic& operator=(const polymorphic& other);
constexpr polymorphic& operator=(polymorphic&& other) noexcept(see below);

// 20.5.2.6, observers
constexpr const T& operator*() const noexcept;
constexpr T& operator*() noexcept;
constexpr const_pointer operator->() const noexcept;
constexpr pointer operator->() noexcept;
constexpr bool valueless_after_move() const noexcept;
constexpr allocator_type get_allocator() const noexcept;

// 20.5.2.7, swap
constexpr void swap(polymorphic& other) noexcept(see below);
friend constexpr void swap(polymorphic& lhs, polymorphic& rhs) noexcept(see below);

private:
    Allocator alloc = Allocator(); // exposition only
};
}

```

20.5.2.3 Constructors

[polymorphic.ctor]

1 The following element applies to all functions in 20.5.2.3:

2 *Throws:* Nothing unless `allocator_traits<Allocator>::allocate` or `allocator_traits<Allocator>::construct` throws.

```
constexpr explicit polymorphic();
```

3 *Constraints:* `is_default_constructible_v<Allocator>` is true.

4 *Mandates:*

(4.1) — `is_default_constructible_v<T>` is true, and

(4.2) — `is_copy_constructible_v<T>` is true.

5 *Effects:* Constructs an owned object of type `T` with an empty argument list using the allocator `alloc`.

```
constexpr explicit polymorphic(allocator_arg_t, const Allocator& a);
```

6 *Mandates:*

(6.1) — `is_default_constructible_v<T>` is true, and

(6.2) — `is_copy_constructible_v<T>` is true.

7 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `T` with an empty argument list using the allocator `alloc`.

```
constexpr polymorphic(const polymorphic& other);
```

8 *Effects:* `alloc` is direct-non-list-initialized with `allocator_traits<Allocator>::select_on_container_copy_construction(other.alloc)`. If `other` is valueless, `*this` is valueless. Otherwise, constructs an owned object of type `U`, where `U` is the type of the owned object in `other`, with the owned object in `other` using the allocator `alloc`.

```
constexpr polymorphic(allocator_arg_t, const Allocator& a, const polymorphic& other);
```

9 *Effects:* `alloc` is direct-non-list-initialized with `a`. If `other` is valueless, `*this` is valueless. Otherwise, constructs an owned object of type `U`, where `U` is the type of the owned object in `other`, with the owned object in `other` using the allocator `alloc`.

```
constexpr polymorphic(polymorphic&& other) noexcept;
```

- 10 *Effects:* *alloc* is direct-non-list-initialized with `std::move(other.alloc)`. If *other* is valueless, **this* is valueless. Otherwise, either **this* takes ownership of the owned object of *other* or, owns an object of the same type constructed from the owned object of *other* considering that owned object as an rvalue, using the allocator *alloc*.

```
constexpr polymorphic(allocator_arg_t, const Allocator& a, polymorphic&& other)
noexcept(allocator_traits<Allocator>::is_always_equal::value);
```

- 11 *Effects:* *alloc* is direct-non-list-initialized with *a*. If *other* is valueless, **this* is valueless. Otherwise, if *alloc == other.alloc* is true, either constructs an object of type *polymorphic* that owns the owned object of *other*, making *other* valueless; or, owns an object of the same type constructed from the owned object of *other* considering that owned object as an rvalue. Otherwise, if *alloc != other.alloc* is true, constructs an owned object of type *U*, where *U* is the type of the owned object in *other*, with the owned object in *other* as an rvalue, using the allocator *alloc*.

```
template<class U = T>
constexpr explicit polymorphic(U&& u);
```

- 12 *Constraints:* Where *UU* is `remove_cvref_t<U>`,
- (12.1) — `is_same_v<UU, polymorphic>` is false,
- (12.2) — `derived_from<UU, T>` is true,
- (12.3) — `is_constructible_v<UU, U>` is true,
- (12.4) — `is_copy_constructible_v<UU>` is true,
- (12.5) — *UU* is not a specialization of `in_place_type_t`, and
- (12.6) — `is_default_constructible_v<Allocator>` is true.

- 13 *Effects:* Constructs an owned object of type *UU* with `std::forward<U>(u)` using the allocator *alloc*.

```
template<class U = T>
constexpr explicit polymorphic(allocator_arg_t, const Allocator& a, U&& u);
```

- 14 *Constraints:* Where *UU* is `remove_cvref_t<U>`,
- (14.1) — `is_same_v<UU, polymorphic>` is false,
- (14.2) — `derived_from<UU, T>` is true,
- (14.3) — `is_constructible_v<UU, U>` is true,
- (14.4) — `is_copy_constructible_v<UU>` is true, and
- (14.5) — *UU* is not a specialization of `in_place_type_t`.

- 15 *Effects:* *alloc* is direct-non-list-initialized with *a*. Constructs an owned object of type *UU* with `std::forward<U>(u)` using the allocator *alloc*.

```
template<class U, class... Ts>
constexpr explicit polymorphic(in_place_type_t<U>, Ts&&... ts);
```

- 16 *Constraints:*
- (16.1) — `is_same_v<remove_cvref_t<U>, U>` is true,
- (16.2) — `derived_from<U, T>` is true,
- (16.3) — `is_constructible_v<U, Ts...>` is true,
- (16.4) — `is_copy_constructible_v<U>` is true, and
- (16.5) — `is_default_constructible_v<Allocator>` is true.

- 17 *Effects:* Constructs an owned object of type *U* with `std::forward<Ts>(ts)...` using the allocator *alloc*.

```
template<class U, class... Ts>
constexpr explicit polymorphic(allocator_arg_t, const Allocator& a,
in_place_type_t<U>, Ts&&... ts);
```

- 18 *Constraints:*

- (18.1) — `is_same_v<remove_cvref_t<U>, U>` is true,
- (18.2) — `derived_from<U, T>` is true,
- (18.3) — `is_constructible_v<U, Ts...>` is true, and
- (18.4) — `is_copy_constructible_v<U>` is true.

19 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `U` with `std::forward<Ts>(ts)...` using the allocator `alloc`.

```
template<class U, class I, class... Us>
constexpr explicit polymorphic(in_place_type_t<U>, initializer_list<I> ilist, Us&&... us);
```

20 *Constraints:*

- (20.1) — `is_same_v<remove_cvref_t<U>, U>` is true,
- (20.2) — `derived_from<U, T>` is true,
- (20.3) — `is_constructible_v<U, initializer_list<I>&, Us...>` is true,
- (20.4) — `is_copy_constructible_v<U>` is true, and
- (20.5) — `is_default_constructible_v<Allocator>` is true.

21 *Effects:* Constructs an owned object of type `U` with the arguments `ilist`, `std::forward<Us>(us)...` using the allocator `alloc`.

```
template<class U, class I, class... Us>
constexpr explicit polymorphic(allocator_arg_t, const Allocator& a,
                               in_place_type_t<U>, initializer_list<I> ilist, Us&&... us);
```

22 *Constraints:*

- (22.1) — `is_same_v<remove_cvref_t<U>, U>` is true,
- (22.2) — `derived_from<U, T>` is true,
- (22.3) — `is_constructible_v<U, initializer_list<I>&, Us...>` is true, and
- (22.4) — `is_copy_constructible_v<U>` is true.

23 *Effects:* `alloc` is direct-non-list-initialized with `a`. Constructs an owned object of type `U` with the arguments `ilist`, `std::forward<Us>(us)...` using the allocator `alloc`.

20.5.2.4 Destructor

[polymorphic.dtor]

```
constexpr ~polymorphic();
```

1 *Mandates:* `T` is a complete type.

2 *Effects:* If `*this` is not valueless, calls `allocator_traits<Allocator>::destroy(p)`, where `p` is a pointer of type `U*` to the owned object and `U` is the type of the owned object; then the storage is deallocated.

20.5.2.5 Assignment

[polymorphic.assign]

```
constexpr polymorphic& operator=(const polymorphic& other);
```

1 *Mandates:* `T` is a complete type.

2 *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

- (2.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_copy_assignment::value` is true.
- (2.2) — If `other` is not valueless, a new owned object of type `U`, where `U` is the type of the owned object in `other`, is constructed in `*this` using `allocator_traits<Allocator>::construct` with the owned object from `other` as the argument, using either the allocator in `*this` or the allocator in `other` if the allocator needs updating.
- (2.3) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
- (2.4) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

3 *Returns:* A reference to `*this`.

4 *Remarks:* If any exception is thrown, there are no effects on `*this`.

```
constexpr polymorphic& operator=(polymorphic&& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
```

5 *Mandates:* If `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is false and `allocator_traits<Allocator>::is_always_equal::value` is false, `T` is a complete type.

6 *Effects:* If `addressof(other) == this` is true, there are no effects. Otherwise:

- (6.1) — The allocator needs updating if `allocator_traits<Allocator>::propagate_on_container_move_assignment::value` is true.
- (6.2) — If `other` is valueless, `*this` becomes valueless.
- (6.3) — Otherwise, if the allocator needs updating or `alloc == other.alloc` is true, `*this` takes ownership of the owned object of `other`.
- (6.4) — Otherwise, constructs a new owned object of type `U`, where `U` is the type of the owned object in `other`, with the owned object of `other` as the argument as an rvalue, using the allocator in `*this`.
- (6.5) — The previously owned object in `*this`, if any, is destroyed using `allocator_traits<Allocator>::destroy` and then the storage is deallocated.
- (6.6) — If the allocator needs updating, the allocator in `*this` is replaced with a copy of the allocator in `other`.

7 *Returns:* A reference to `*this`.

8 *Remarks:* If any exception is thrown, there are no effects on `*this` or `other`.

20.5.2.6 Observers

[polymorphic.obs]

```
constexpr const T& operator*() const noexcept;
constexpr T& operator*() noexcept;
```

1 *Preconditions:* `*this` is not valueless.

2 *Returns:* A reference to the owned object.

```
constexpr const_pointer operator->() const noexcept;
constexpr pointer operator->() noexcept;
```

3 *Preconditions:* `*this` is not valueless.

4 *Returns:* A pointer to the owned object.

```
constexpr bool valueless_after_move() const noexcept;
```

5 *Returns:* true if `*this` is valueless, otherwise false.

```
constexpr allocator_type get_allocator() const noexcept;
```

6 *Returns:* `alloc`.

20.5.2.7 Swap

[polymorphic.swap]

```
constexpr void swap(polymorphic& other)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
             allocator_traits<Allocator>::is_always_equal::value);
```

1 *Preconditions:* If `allocator_traits<Allocator>::propagate_on_container_swap::value` is true, then `Allocator` meets the *Cpp17Swappable* requirements. Otherwise `get_allocator() == other.get_allocator()` is true.

2 *Effects:* Swaps the states of `*this` and `other`, exchanging owned objects or valueless states. If `allocator_traits<Allocator>::propagate_on_container_swap::value` is true, then the allocators of `*this` and `other` are exchanged by calling `swap` as described in 16.4.4.3. Otherwise, the allocators are not swapped.

[Note 1: Does not call `swap` on the owned objects directly. — end note]

```
constexpr void swap(polymorphic& lhs, polymorphic& rhs) noexcept(noexcept(lhs.swap(rhs)));
```

³ *Effects:* Equivalent to `lhs.swap(rhs)`.

20.6 Memory resources

[mem.res]

20.6.1 Header <memory_resource> synopsis

[mem.res.syn]

```
namespace std::pmr {
    // 20.6.2, class memory_resource
    class memory_resource;

    bool operator==(const memory_resource& a, const memory_resource& b) noexcept;

    // 20.6.3, class template polymorphic_allocator
    template<class Tp = byte> class polymorphic_allocator;

    template<class T1, class T2>
        bool operator==(const polymorphic_allocator<T1>& a,
                        const polymorphic_allocator<T2>& b) noexcept;

    // 20.6.4, global memory resources
    memory_resource* new_delete_resource() noexcept;
    memory_resource* null_memory_resource() noexcept;
    memory_resource* set_default_resource(memory_resource* r) noexcept;
    memory_resource* get_default_resource() noexcept;

    // 20.6.5, pool resource classes
    struct pool_options;
    class synchronized_pool_resource;
    class unsynchronized_pool_resource;
    class monotonic_buffer_resource;
}
```

20.6.2 Class memory_resource

[mem.res.class]

20.6.2.1 General

[mem.res.class.general]

¹ The `memory_resource` class is an abstract interface to an unbounded set of classes encapsulating memory resources.

```
namespace std::pmr {
    class memory_resource {
        static constexpr size_t max_align = alignof(max_align_t);    // exposition only

    public:
        memory_resource() = default;
        memory_resource(const memory_resource&) = default;
        virtual ~memory_resource();

        memory_resource& operator=(const memory_resource&) = default;

        void* allocate(size_t bytes, size_t alignment = max_align);
        void deallocate(void* p, size_t bytes, size_t alignment = max_align);

        bool is_equal(const memory_resource& other) const noexcept;

    private:
        virtual void* do_allocate(size_t bytes, size_t alignment) = 0;
        virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;

        virtual bool do_is_equal(const memory_resource& other) const noexcept = 0;
    };
}
```

20.6.2.2 Public member functions**[mem.res.public]**`~memory_resource();`

1 *Effects:* Destroys this `memory_resource`.

`void* allocate(size_t bytes, size_t alignment = max_align);`

2 *Effects:* Allocates storage by calling `do_allocate(bytes, alignment)` and implicitly creates objects within the allocated region of storage.

3 *Returns:* A pointer to a suitable created object (6.8.2) in the allocated region of storage.

4 *Throws:* What and when the call to `do_allocate` throws.

`void deallocate(void* p, size_t bytes, size_t alignment = max_align);`

5 *Effects:* Equivalent to `do_deallocate(p, bytes, alignment)`.

`bool is_equal(const memory_resource& other) const noexcept;`

6 *Effects:* Equivalent to: `return do_is_equal(other);`

20.6.2.3 Private virtual member functions**[mem.res.private]**`virtual void* do_allocate(size_t bytes, size_t alignment) = 0;`

1 *Preconditions:* `alignment` is a power of two.

2 *Returns:* A derived class shall implement this function to return a pointer to allocated storage (6.8.6.5.2) with a size of at least `bytes`, aligned to the specified `alignment`.

3 *Throws:* A derived class implementation shall throw an appropriate exception if it is unable to allocate memory with the requested size and alignment.

`virtual void do_deallocate(void* p, size_t bytes, size_t alignment) = 0;`

4 *Preconditions:* `p` was returned from a prior call to `allocate(bytes, alignment)` on a memory resource equal to `*this`, and the storage at `p` has not yet been deallocated.

5 *Effects:* A derived class shall implement this function to dispose of allocated storage.

6 *Throws:* Nothing.

`virtual bool do_is_equal(const memory_resource& other) const noexcept = 0;`

7 *Returns:* A derived class shall implement this function to return `true` if memory allocated from `*this` can be deallocated from `other` and vice-versa, otherwise `false`.

[Note 1: It is possible that the most-derived type of `other` does not match the type of `*this`. For a derived class `D`, an implementation of this function can immediately return `false` if `dynamic_cast<const D*>(&other) == nullptr`. — end note]

20.6.2.4 Equality**[mem.res.eq]**`bool operator==(const memory_resource& a, const memory_resource& b) noexcept;`

1 *Returns:* `&a == &b || a.is_equal(b)`.

20.6.3 Class template `polymorphic_allocator`**[mem.poly.allocator.class]****20.6.3.1 General****[mem.poly.allocator.class.general]**

1 A specialization of class template `pmr::polymorphic_allocator` meets the *Cpp17Allocator* requirements (16.4.4.6.1) if its template argument is a *cv*-unqualified object type. Constructed with different memory resources, different instances of the same specialization of `pmr::polymorphic_allocator` can exhibit entirely different allocation behavior. This runtime polymorphism allows objects that use `polymorphic_allocator` to behave as if they used different allocator types at run time even though they use the same static allocator type.

2 A specialization of class template `pmr::polymorphic_allocator` meets the allocator completeness requirements (16.4.4.6.2) if its template argument is a *cv*-unqualified object type.

```
namespace std::pmr {
    template<class Tp = byte> class polymorphic_allocator {
        memory_resource* memory_rsrc;          // exposition only
    };
}
```

```

public:
    using value_type = Tp;

    // 20.6.3.2, constructors
    polymorphic_allocator() noexcept;
    polymorphic_allocator(memory_resource* r);

    polymorphic_allocator(const polymorphic_allocator& other) = default;

    template<class U>
        polymorphic_allocator(const polymorphic_allocator<U>& other) noexcept;

    polymorphic_allocator& operator=(const polymorphic_allocator&) = delete;

    // 20.6.3.3, member functions
    Tp* allocate(size_t n);
    void deallocate(Tp* p, size_t n);

    void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
    void deallocate_bytes(void* p, size_t nbytes, size_t alignment = alignof(max_align_t));
    template<class T> T* allocate_object(size_t n = 1);
    template<class T> void deallocate_object(T* p, size_t n = 1);
    template<class T, class... CtorArgs> T* new_object(CtorArgs&&... ctor_args);
    template<class T> void delete_object(T* p);

    template<class T, class... Args>
        void construct(T* p, Args&&... args);

    template<class T>
        void destroy(T* p);

    polymorphic_allocator select_on_container_copy_construction() const;

    memory_resource* resource() const;

    // friends
    friend bool operator==(const polymorphic_allocator& a,
                           const polymorphic_allocator& b) noexcept {
        return *a.resource() == *b.resource();
    }
};

```

20.6.3.2 Constructors

[mem.poly.allocator.ctor]

```
polymorphic_allocator() noexcept;
```

1 *Effects:* Sets `memory_rsrc` to `get_default_resource()`.

```
polymorphic_allocator(memory_resource* r);
```

2 *Preconditions:* `r` is non-null.

3 *Effects:* Sets `memory_rsrc` to `r`.

4 *Throws:* Nothing.

5 [Note 1: This constructor provides an implicit conversion from `memory_resource*`. — end note]

```
template<class U> polymorphic_allocator(const polymorphic_allocator<U>& other) noexcept;
```

6 *Effects:* Sets `memory_rsrc` to `other.resource()`.

20.6.3.3 Member functions

[mem.poly.allocator.mem]

```
Tp* allocate(size_t n);
```

1 *Effects:* If `numeric_limits<size_t>::max() / sizeof(Tp) < n`, throws `bad_array_new_length`.
 Otherwise equivalent to:

```

    return static_cast<Tp*>(memory_rsrc->allocate(n * sizeof(Tp), alignof(Tp)));

void deallocate(Tp* p, size_t n);
2    Preconditions: p was allocated from a memory resource x, equal to *memory_rsrc, using x.allocate(n
    * sizeof(Tp), alignof(Tp)).
3    Effects: Equivalent to memory_rsrc->deallocate(p, n * sizeof(Tp), alignof(Tp)).
4    Throws: Nothing.

void* allocate_bytes(size_t nbytes, size_t alignment = alignof(max_align_t));
5    Effects: Equivalent to: return memory_rsrc->allocate(nbytes, alignment);
6    [Note 1: The return type is void* (rather than, e.g., byte*) to support conversion to an arbitrary pointer type
    U* by static_cast<U*>, thus facilitating construction of a U object in the allocated memory. — end note]

void deallocate_bytes(void* p, size_t nbytes, size_t alignment = alignof(max_align_t));
7    Effects: Equivalent to memory_rsrc->deallocate(p, nbytes, alignment).

template<class T>
    T* allocate_object(size_t n = 1);
8    Effects: Allocates memory suitable for holding an array of n objects of type T, as follows:
(8.1) — if numeric_limits<size_t>::max() / sizeof(T) < n, throws bad_array_new_length,
(8.2) — otherwise equivalent to:
        return static_cast<T*>(allocate_bytes(n*sizeof(T), alignof(T)));
9    [Note 2: T is not deduced and must therefore be provided as a template argument. — end note]

template<class T>
    void deallocate_object(T* p, size_t n = 1);
10    Effects: Equivalent to deallocate_bytes(p, n*sizeof(T), alignof(T)).

template<class T, class... CtorArgs>
    T* new_object(CtorArgs&&... ctor_args);
11    Effects: Allocates and constructs an object of type T, as follows.
    Equivalent to:
        T* p = allocate_object<T>();
        try {
            construct(p, std::forward<CtorArgs>(ctor_args)...);
        } catch (...) {
            deallocate_object(p);
            throw;
        }
        return p;
12    [Note 3: T is not deduced and must therefore be provided as a template argument. — end note]

template<class T>
    void delete_object(T* p);
13    Effects: Equivalent to:
        destroy(p);
        deallocate_object(p);

template<class T, class... Args>
    void construct(T* p, Args&&... args);
14    Mandates: Uses-allocator construction of T with allocator *this (see 20.2.8.2) and constructor arguments
    std::forward<Args>(args)... is well-formed.
15    Effects: Constructs a T object in the storage whose address is represented by p by uses-allocator
    construction with allocator *this and constructor arguments std::forward<Args>(args)...
16    Throws: Nothing unless the constructor for T throws.

```



```
template<class T>
void destroy(T* p);
```

17 *Effects:* Equivalent to `p->~T()`.

```
polymorphic_allocator select_on_container_copy_construction() const;
```

18 *Returns:* `polymorphic_allocator()`.

19 [Note 4: The memory resource is not propagated. — end note]

```
memory_resource* resource() const;
```

20 *Returns:* `memory_rsrc`.

20.6.3.4 Equality

[mem.poly.allocator.eq]

```
template<class T1, class T2>
bool operator==(const polymorphic_allocator<T1>& a,
                const polymorphic_allocator<T2>& b) noexcept;
```

1 *Returns:* `*a.resource() == *b.resource()`.

20.6.4 Access to program-wide memory_resource objects

[mem.res.global]

```
memory_resource* new_delete_resource() noexcept;
```

1 *Returns:* A pointer to a static-duration object of a type derived from `memory_resource` that can serve as a resource for allocating memory using `::operator new` and `::operator delete`. The same value is returned every time this function is called. For a return value `p` and a memory resource `r`, `p->is_equal(r)` returns `&r == p`.

```
memory_resource* null_memory_resource() noexcept;
```

2 *Returns:* A pointer to a static-duration object of a type derived from `memory_resource` for which `allocate()` always throws `bad_alloc` and for which `deallocate()` has no effect. The same value is returned every time this function is called. For a return value `p` and a memory resource `r`, `p->is_equal(r)` returns `&r == p`.

3 The *default memory resource pointer* is a pointer to a memory resource that is used by certain facilities when an explicit memory resource is not supplied through the interface. Its initial value is the return value of `new_delete_resource()`.

```
memory_resource* set_default_resource(memory_resource* r) noexcept;
```

4 *Effects:* If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

5 *Returns:* The previous value of the default memory resource pointer.

6 *Remarks:* Calling the `set_default_resource` and `get_default_resource` functions shall not incur a data race (6.10.2.2). A call to the `set_default_resource` function synchronizes with subsequent calls to the `set_default_resource` and `get_default_resource` functions.

```
memory_resource* get_default_resource() noexcept;
```

7 *Returns:* The current value of the default memory resource pointer.

20.6.5 Pool resource classes

[mem.res.pool]

20.6.5.1 Classes `synchronized_pool_resource` and `unsynchronized_pool_resource`

[mem.res.pool.overview]

1 The `synchronized_pool_resource` and `unsynchronized_pool_resource` classes (collectively called *pool resource classes*) are general-purpose memory resources having the following qualities:

- (1.1) — Each resource frees its allocated memory on destruction, even if `deallocate` has not been called for some of the allocated blocks.
- (1.2) — A pool resource consists of a collection of *pools*, serving requests for different block sizes. Each individual pool manages a collection of *chunks* that are in turn divided into blocks of uniform size, returned via calls to `do_allocate`. Each call to `do_allocate(size, alignment)` is dispatched to the pool serving the smallest blocks accommodating at least `size` bytes.

- (1.3) — When a particular pool is exhausted, allocating a block from that pool results in the allocation of an additional chunk of memory from the *upstream allocator* (supplied at construction), thus replenishing the pool. With each successive replenishment, the chunk size obtained increases geometrically.
 [Note 1: By allocating memory in chunks, the pooling strategy increases the chance that consecutive allocations will be close together in memory. — end note]
 - (1.4) — Allocation requests that exceed the largest block size of any pool are fulfilled directly from the upstream allocator.
 - (1.5) — A `pool_options` struct may be passed to the pool resource constructors to tune the largest block size and the maximum chunk size.
- 2 A `synchronized_pool_resource` may be accessed from multiple threads without external synchronization and may have thread-specific pools to reduce synchronization costs. An `unsynchronized_pool_resource` class may not be accessed from multiple threads simultaneously and thus avoids the cost of synchronization entirely in single-threaded applications.

```

namespace std::pmr {
    struct pool_options {
        size_t max_blocks_per_chunk = 0;
        size_t largest_required_pool_block = 0;
    };

    class synchronized_pool_resource : public memory_resource {
    public:
        synchronized_pool_resource(const pool_options& opts, memory_resource* upstream);

        synchronized_pool_resource()
            : synchronized_pool_resource(pool_options(), get_default_resource()) {}
        explicit synchronized_pool_resource(memory_resource* upstream)
            : synchronized_pool_resource(pool_options(), upstream) {}
        explicit synchronized_pool_resource(const pool_options& opts)
            : synchronized_pool_resource(opts, get_default_resource()) {}

        synchronized_pool_resource(const synchronized_pool_resource&) = delete;
        virtual ~synchronized_pool_resource();

        synchronized_pool_resource& operator=(const synchronized_pool_resource&) = delete;

        void release();
        memory_resource* upstream_resource() const;
        pool_options options() const;

    protected:
        void* do_allocate(size_t bytes, size_t alignment) override;
        void do_deallocate(void* p, size_t bytes, size_t alignment) override;

        bool do_is_equal(const memory_resource& other) const noexcept override;
    };

    class unsynchronized_pool_resource : public memory_resource {
    public:
        unsynchronized_pool_resource(const pool_options& opts, memory_resource* upstream);

        unsynchronized_pool_resource()
            : unsynchronized_pool_resource(pool_options(), get_default_resource()) {}
        explicit unsynchronized_pool_resource(memory_resource* upstream)
            : unsynchronized_pool_resource(pool_options(), upstream) {}
        explicit unsynchronized_pool_resource(const pool_options& opts)
            : unsynchronized_pool_resource(opts, get_default_resource()) {}

        unsynchronized_pool_resource(const unsynchronized_pool_resource&) = delete;
        virtual ~unsynchronized_pool_resource();

        unsynchronized_pool_resource& operator=(const unsynchronized_pool_resource&) = delete;

```

```

    void release();
    memory_resource* upstream_resource() const;
    pool_options options() const;

protected:
    void* do_allocate(size_t bytes, size_t alignment) override;
    void do_deallocate(void* p, size_t bytes, size_t alignment) override;

    bool do_is_equal(const memory_resource& other) const noexcept override;
};
}

```

20.6.5.2 pool_options data members

[mem.res.pool.options]

- ¹ The members of `pool_options` comprise a set of constructor options for pool resources. The effect of each option on the pool resource behavior is described below:

```
size_t max_blocks_per_chunk;
```

- ² The maximum number of blocks that will be allocated at once from the upstream memory resource (20.6.6) to replenish a pool. If the value of `max_blocks_per_chunk` is zero or is greater than an implementation-defined limit, that limit is used instead. The implementation may choose to use a smaller value than is specified in this member and may use different values for different pools.

```
size_t largest_required_pool_block;
```

- ³ The largest allocation size that is required to be fulfilled using the pooling mechanism. Attempts to allocate a single block larger than this threshold will be allocated directly from the upstream memory resource. If `largest_required_pool_block` is zero or is greater than an implementation-defined limit, that limit is used instead. The implementation may choose a pass-through threshold larger than specified in this member.

20.6.5.3 Constructors and destructors

[mem.res.pool.ctor]

```

synchronized_pool_resource(const pool_options& opts, memory_resource* upstream);
unsynchronized_pool_resource(const pool_options& opts, memory_resource* upstream);

```

- ¹ *Preconditions:* `upstream` is the address of a valid memory resource.
- ² *Effects:* Constructs a pool resource object that will obtain memory from `upstream` whenever the pool resource is unable to satisfy a memory request from its own internal data structures. The resulting object will hold a copy of `upstream`, but will not own the resource to which `upstream` points.
- [Note 1: The intention is that in most cases there will be substantially fewer calls to `upstream->allocate()` than to `this->allocate()`. — end note]
- The behavior of the pooling mechanism is tuned according to the value of the `opts` argument.
- ³ *Throws:* Nothing unless `upstream->allocate()` throws. It is unspecified if, or under what conditions, this constructor calls `upstream->allocate()`.

```

virtual ~synchronized_pool_resource();
virtual ~unsynchronized_pool_resource();

```

- ⁴ *Effects:* Calls `release()`.

20.6.5.4 Members

[mem.res.pool.mem]

```
void release();
```

- ¹ *Effects:* Calls `upstream_resource()->deallocate()` as necessary to release all allocated memory.
- [Note 1: The memory is released back to `upstream_resource()` even if `deallocate` has not been called for some of the allocated blocks. — end note]

```
memory_resource* upstream_resource() const;
```

- ² *Returns:* The value of the `upstream` argument provided to the constructor of this object.

```
pool_options options() const;
```

- 3 *Returns:* The options that control the pooling behavior of this resource. The values in the returned struct may differ from those supplied to the pool resource constructor in that values of zero will be replaced with implementation-defined defaults, and sizes may be rounded to unspecified granularity.

```
void* do_allocate(size_t bytes, size_t alignment) override;
```

- 4 *Effects:* If the pool selected for a block of size `bytes` is unable to satisfy the memory request from its own internal data structures, it will call `upstream_resource()->allocate()` to obtain more memory. If `bytes` is larger than that which the largest pool can handle, then memory will be allocated using `upstream_resource()->allocate()`.

- 5 *Returns:* A pointer to allocated storage (6.8.6.5.2) with a size of at least `bytes`. The size and alignment of the allocated memory shall meet the requirements for a class derived from `memory_resource` (20.6.2).

- 6 *Throws:* Nothing unless `upstream_resource()->allocate()` throws.

```
void do_deallocate(void* p, size_t bytes, size_t alignment) override;
```

- 7 *Effects:* Returns the memory at `p` to the pool. It is unspecified if, or under what circumstances, this operation will result in a call to `upstream_resource()->deallocate()`.

- 8 *Throws:* Nothing.

```
bool do_is_equal(const memory_resource& other) const noexcept override;
```

- 9 *Returns:* `this == &other`.

20.6.6 Class `monotonic_buffer_resource`

[mem.res.monotonic.buffer]

20.6.6.1 General

[mem.res.monotonic.buffer.general]

- 1 A `monotonic_buffer_resource` is a special-purpose memory resource intended for very fast memory allocations in situations where memory is used to build up a few objects and then is released all at once when the memory resource object is destroyed.

```
namespace std::pmr {
    class monotonic_buffer_resource : public memory_resource {
        memory_resource* upstream_rsrc;    // exposition only
        void* current_buffer;              // exposition only
        size_t next_buffer_size;           // exposition only

    public:
        explicit monotonic_buffer_resource(memory_resource* upstream);
        monotonic_buffer_resource(size_t initial_size, memory_resource* upstream);
        monotonic_buffer_resource(void* buffer, size_t buffer_size, memory_resource* upstream);

        monotonic_buffer_resource()
            : monotonic_buffer_resource(get_default_resource()) {}
        explicit monotonic_buffer_resource(size_t initial_size)
            : monotonic_buffer_resource(initial_size, get_default_resource()) {}
        monotonic_buffer_resource(void* buffer, size_t buffer_size)
            : monotonic_buffer_resource(buffer, buffer_size, get_default_resource()) {}

        monotonic_buffer_resource(const monotonic_buffer_resource&) = delete;

        virtual ~monotonic_buffer_resource();

        monotonic_buffer_resource& operator=(const monotonic_buffer_resource&) = delete;

        void release();
        memory_resource* upstream_resource() const;

    protected:
        void* do_allocate(size_t bytes, size_t alignment) override;
        void do_deallocate(void* p, size_t bytes, size_t alignment) override;
```

```

    bool do_is_equal(const memory_resource& other) const noexcept override;
};

```

20.6.6.2 Constructors and destructor

[mem.res.monotonic.buffer.ctor]

```

explicit monotonic_buffer_resource(memory_resource* upstream);
monotonic_buffer_resource(size_t initial_size, memory_resource* upstream);

```

1 *Preconditions:* `upstream` is the address of a valid memory resource. `initial_size`, if specified, is greater than zero.

2 *Effects:* Sets `upstream_rsrc` to `upstream` and `current_buffer` to `nullptr`. If `initial_size` is specified, sets `next_buffer_size` to at least `initial_size`; otherwise sets `next_buffer_size` to an implementation-defined size.

```

monotonic_buffer_resource(void* buffer, size_t buffer_size, memory_resource* upstream);

```

3 *Preconditions:* `upstream` is the address of a valid memory resource. `buffer_size` is no larger than the number of bytes in `buffer`.

4 *Effects:* Sets `upstream_rsrc` to `upstream`, `current_buffer` to `buffer`, and `next_buffer_size` to `buffer_size` (but not less than 1), then increases `next_buffer_size` by an implementation-defined growth factor (which need not be integral).

```

~monotonic_buffer_resource();

```

5 *Effects:* Calls `release()`.

20.6.6.3 Members

[mem.res.monotonic.buffer.mem]

```

void release();

```

1 *Effects:* Calls `upstream_rsrc->deallocate()` as necessary to release all allocated memory. Resets `current_buffer` and `next_buffer_size` to their initial values at construction.

2 [Note 1: The memory is released back to `upstream_rsrc` even if some blocks that were allocated from `*this` have not been deallocated from `*this`. — end note]

```

memory_resource* upstream_resource() const;

```

3 *Returns:* The value of `upstream_rsrc`.

```

void* do_allocate(size_t bytes, size_t alignment) override;

```

4 *Effects:* If the unused space in `current_buffer` can fit a block with the specified `bytes` and `alignment`, then allocate the return block from `current_buffer`; otherwise set `current_buffer` to `upstream_rsrc->allocate(n, m)`, where `n` is not less than `max(bytes, next_buffer_size)` and `m` is not less than `alignment`, and increase `next_buffer_size` by an implementation-defined growth factor (which need not be integral), then allocate the return block from the newly-allocated `current_buffer`.

5 *Returns:* A pointer to allocated storage (6.8.6.5.2) with a size of at least `bytes`. The size and alignment of the allocated memory shall meet the requirements for a class derived from `memory_resource` (20.6.2).

6 *Throws:* Nothing unless `upstream_rsrc->allocate()` throws.

```

void do_deallocate(void* p, size_t bytes, size_t alignment) override;

```

7 *Effects:* None.

8 *Throws:* Nothing.

9 *Remarks:* Memory used by this resource increases monotonically until its destruction.

```

bool do_is_equal(const memory_resource& other) const noexcept override;

```

10 *Returns:* `this == &other`.

20.7 Class template `scoped_allocator_adaptor`

[allocator.adaptor]

20.7.1 Header `<scoped_allocator>` synopsis

[allocator.adaptor.syn]

```

namespace std {
    // class template scoped_allocator_adaptor

```

```

template<class OuterAlloc, class... InnerAlloc>
    class scoped_allocator_adaptor;

// 20.7.5, scoped allocator operators
template<class OuterA1, class OuterA2, class... InnerAllocs>
    bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                    const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
}

```

- ¹ The class template `scoped_allocator_adaptor` is an allocator template that specifies an allocator resource (the outer allocator) to be used by a container (as any other allocator does) and also specifies an inner allocator resource to be passed to the constructor of every element within the container. This adaptor is instantiated with one outer and zero or more inner allocator types. If instantiated with only one allocator type, the inner allocator becomes the `scoped_allocator_adaptor` itself, thus using the same allocator resource for the container and every element within the container and, if the elements themselves are containers, each of their elements recursively. If instantiated with more than one allocator, the first allocator is the outer allocator for use by the container, the second allocator is passed to the constructors of the container's elements, and, if the elements themselves are containers, the third allocator is passed to the elements' elements, and so on. If containers are nested to a depth greater than the number of allocators, the last allocator is used repeatedly, as in the single-allocator case, for any remaining recursions.

[*Note 1*: The `scoped_allocator_adaptor` is derived from the outer allocator type so it can be substituted for the outer allocator type in most expressions. — *end note*]

```

namespace std {
    template<class OuterAlloc, class... InnerAllocs>
    class scoped_allocator_adaptor : public OuterAlloc {
    private:
        using OuterTraits = allocator_traits<OuterAlloc>; // exposition only
        scoped_allocator_adaptor<InnerAllocs...> inner; // exposition only

    public:
        using outer_allocator_type = OuterAlloc;
        using inner_allocator_type = see below;

        using value_type          = OuterTraits::value_type;
        using size_type           = OuterTraits::size_type;
        using difference_type      = OuterTraits::difference_type;
        using pointer              = OuterTraits::pointer;
        using const_pointer        = OuterTraits::const_pointer;
        using void_pointer         = OuterTraits::void_pointer;
        using const_void_pointer   = OuterTraits::const_void_pointer;

        using propagate_on_container_copy_assignment = see below;
        using propagate_on_container_move_assignment = see below;
        using propagate_on_container_swap            = see below;
        using is_always_equal                        = see below;

        template<class Tp> struct rebind {
            using other = scoped_allocator_adaptor<
                OuterTraits::template rebind_alloc<Tp>, InnerAllocs...>;
        };

        scoped_allocator_adaptor();
        template<class OuterA2>
            scoped_allocator_adaptor(OuterA2&& outerAlloc,
                                    const InnerAllocs&... innerAllocs) noexcept;

        scoped_allocator_adaptor(const scoped_allocator_adaptor& other) noexcept;
        scoped_allocator_adaptor(scoped_allocator_adaptor&& other) noexcept;

        template<class OuterA2>
            scoped_allocator_adaptor(
                const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& other) noexcept;
    };
}

```

```

template<class OuterA2>
    scoped_allocator_adaptor(
        scoped_allocator_adaptor<OuterA2, InnerAllocs...>&& other) noexcept;

scoped_allocator_adaptor& operator=(const scoped_allocator_adaptor&) = default;
scoped_allocator_adaptor& operator=(scoped_allocator_adaptor&&) = default;

~scoped_allocator_adaptor();

inner_allocator_type& inner_allocator() noexcept;
const inner_allocator_type& inner_allocator() const noexcept;
outer_allocator_type& outer_allocator() noexcept;
const outer_allocator_type& outer_allocator() const noexcept;

pointer allocate(size_type n);
pointer allocate(size_type n, const_void_pointer hint);
void deallocate(pointer p, size_type n);
size_type max_size() const;

template<class T, class... Args>
    void construct(T* p, Args&&... args);

template<class T>
    void destroy(T* p);

scoped_allocator_adaptor select_on_container_copy_construction() const;
};

template<class OuterAlloc, class... InnerAllocs>
    scoped_allocator_adaptor(OuterAlloc, InnerAllocs...)
        -> scoped_allocator_adaptor<OuterAlloc, InnerAllocs...>;
}

```

20.7.2 Member types

[allocator.adaptor.types]

using inner_allocator_type = *see below*;

- 1 *Type:* `scoped_allocator_adaptor<OuterAlloc>` if `sizeof...(InnerAllocs)` is zero; otherwise, `scoped_allocator_adaptor<InnerAllocs...>`.

using propagate_on_container_copy_assignment = *see below*;

- 2 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_copy_assignment::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

using propagate_on_container_move_assignment = *see below*;

- 3 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_move_assignment::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

using propagate_on_container_swap = *see below*;

- 4 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_swap::value` is true for any A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

using is_always_equal = *see below*;

- 5 *Type:* `true_type` if `allocator_traits<A>::is_always_equal::value` is true for every A in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

20.7.3 Constructors

[allocator.adaptor.cnstr]

`scoped_allocator_adaptor()`;

- 1 *Effects:* Value-initializes the `OuterAlloc` base class and the `inner` allocator object.

```

template<class OuterA2>
    scoped_allocator_adaptor(OuterA2&& outerAlloc, const InnerAllocs&... innerAllocs) noexcept;
2     Constraints: is_constructible_v<OuterAlloc, OuterA2> is true.
3     Effects: Initializes the OuterAlloc base class with std::forward<OuterA2>(outerAlloc) and inner
    with innerAllocs... (hence recursively initializing each allocator within the adaptor with the corre-
    sponding allocator from the argument list).

    scoped_allocator_adaptor(const scoped_allocator_adaptor& other) noexcept;
4     Effects: Initializes each allocator within the adaptor with the corresponding allocator from other.

    scoped_allocator_adaptor(scoped_allocator_adaptor&& other) noexcept;
5     Effects: Move constructs each allocator within the adaptor with the corresponding allocator from
    other.

template<class OuterA2>
    scoped_allocator_adaptor(
        const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& other) noexcept;
6     Constraints: is_constructible_v<OuterAlloc, const OuterA2&> is true.
7     Effects: Initializes each allocator within the adaptor with the corresponding allocator from other.

template<class OuterA2>
    scoped_allocator_adaptor(scoped_allocator_adaptor<OuterA2, InnerAllocs...>&& other) noexcept;
8     Constraints: is_constructible_v<OuterAlloc, OuterA2> is true.
9     Effects: Initializes each allocator within the adaptor with the corresponding allocator rvalue from
    other.

```

20.7.4 Members

[allocator.adaptor.members]

- 1 In the construct member functions, *OUTERMOST*(x) is *OUTERMOST*(x.outer_allocator()) if the expression x.outer_allocator() is valid (13.10.3) and x otherwise; *OUTERMOST_ALLOC_TRAITS*(x) is allocator_traits<remove_reference_t<decltype(*OUTERMOST*(x))>>.

[Note 1: *OUTERMOST*(x) and *OUTERMOST_ALLOC_TRAITS*(x) are recursive operations. It is incumbent upon the definition of outer_allocator() to ensure that the recursion terminates. It will terminate for all instantiations of scoped_allocator_adaptor. — end note]

```

inner_allocator_type& inner_allocator() noexcept;
const inner_allocator_type& inner_allocator() const noexcept;
2     Returns: *this if sizeof...(InnerAllocs) is zero; otherwise, inner.

outer_allocator_type& outer_allocator() noexcept;
3     Returns: static_cast<OuterAlloc&>(*this).

const outer_allocator_type& outer_allocator() const noexcept;
4     Returns: static_cast<const OuterAlloc&>(*this).

pointer allocate(size_type n);
5     Returns: allocator_traits<OuterAlloc>::allocate(outer_allocator(), n).

pointer allocate(size_type n, const_void_pointer hint);
6     Returns: allocator_traits<OuterAlloc>::allocate(outer_allocator(), n, hint).

void deallocate(pointer p, size_type n) noexcept;
7     Effects: As if by: allocator_traits<OuterAlloc>::deallocate(outer_allocator(), p, n);

size_type max_size() const;
8     Returns: allocator_traits<OuterAlloc>::max_size(outer_allocator()).

```



```
template<class T, class... Args>
void construct(T* p, Args&&... args);
```

9 *Effects:* Equivalent to:

```
    apply([p, this](auto&&... newargs) {
        OUTERMOST_ALLOC_TRAITS(*this)::construct(
            OUTERMOST(*this), p,
            std::forward<decltype(newargs)>(newargs)...);
    },
    uses_allocator_construction_args<T>(inner_allocator(),
        std::forward<Args>(args)...));
```

```
template<class T>
void destroy(T* p);
```

10 *Effects:* Calls `OUTERMOST_ALLOC_TRAITS(*this)::destroy(OUTERMOST(*this), p)`.

```
scoped_allocator_adaptor select_on_container_copy_construction() const;
```

11 *Returns:* A new `scoped_allocator_adaptor` object where each allocator `a1` within the adaptor is initialized with `allocator_traits<A1>::select_on_container_copy_construction(a2)`, where `A1` is the type of `a1` and `a2` is the corresponding allocator in `*this`.

20.7.5 Operators

[scoped.adaptor.operators]

```
template<class OuterA1, class OuterA2, class... InnerAllocs>
bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
    const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

1 *Returns:* If `sizeof...(InnerAllocs)` is zero,

```
    a.outer_allocator() == b.outer_allocator()
```

otherwise

```
    a.outer_allocator() == b.outer_allocator() && a.inner_allocator() == b.inner_allocator()
```

21 Metaprogramming library [meta]

21.1 General [meta.general]

¹ This Clause describes metaprogramming facilities. These facilities are summarized in [Table 51](#).

Table 51 — Metaprogramming library summary [tab:meta.summary]

	Subclause	Header
21.2	Integer sequences	<utility>
21.3	Type traits	<type_traits>
21.4	Reflection	<meta>
21.5	Rational arithmetic	<ratio>

21.2 Compile-time integer sequences [intseq]

21.2.1 General [intseq.general]

¹ The library provides a class template that can represent an integer sequence. When used as an argument to a function template the template parameter pack defining the sequence can be deduced and used in a pack expansion.

[*Note 1*: The `index_sequence` alias template is provided for the common case of an integer sequence of type `size_t`; see also [22.4.6](#). — *end note*]

21.2.2 Class template `integer_sequence` [intseq.intseq]

```
namespace std {
    template<class T, T... I> struct integer_sequence {
        using value_type = T;
        static constexpr size_t size() noexcept { return sizeof...(I); }
    };
}
```

¹ *Mandates*: T is an integer type.

21.2.3 Alias template `make_integer_sequence` [intseq.make]

```
template<class T, T N>
    using make_integer_sequence = integer_sequence<T, see below>;
```

¹ *Mandates*: $N \geq 0$.

² The alias template `make_integer_sequence` denotes a specialization of `integer_sequence` with N constant template arguments. The type `make_integer_sequence<T, N>` is an alias for the type `integer_sequence<T, 0, 1, ..., N - 1>`.

[*Note 1*: `make_integer_sequence<int, 0>` is an alias for the type `integer_sequence<int>`. — *end note*]

21.2.4 Structured binding support [intseq.binding]

```
template<class T, T... Values>
    struct tuple_size<integer_sequence<T, Values...>>
        : integral_constant<size_t, sizeof...(Values)> { };

template<size_t I, class T, T... Values>
    struct tuple_element<I, integer_sequence<T, Values...>> {
        using type = T;
    };

template<size_t I, class T, T... Values>
    struct tuple_element<I, const integer_sequence<T, Values...>> {
        using type = T;
    };
```

```

};
1      Mandates: I < sizeof...(Values).

template<size_t I, class T, T... Values>
    constexpr T get(integer_sequence<T, Values...>) noexcept;
2      Mandates: I < sizeof...(Values).
3      Returns: Values...[I].

```

21.3 Metaprogramming and type traits

[type.traits]

21.3.1 General

[type.traits.general]

- 1 Subclause 21.3 describes components used by C++ programs, particularly in templates, to support the widest possible range of types, optimize template code usage, detect type related user errors, and perform type inference and transformation at compile time. It includes type classification traits, type property inspection traits, and type transformations. The type classification traits describe a complete taxonomy of all possible C++ types, and state where in that taxonomy a given type belongs. The type property inspection traits allow important characteristics of types or of combinations of types to be inspected. The type transformations allow certain properties of types to be manipulated.
- 2 All functions specified in 21.3 are signal-safe (17.14.5).

21.3.2 Requirements

[meta.rqmts]

- 1 A *Cpp17UnaryTypeTrait* describes a property of a type. It shall be a class template that takes one template type argument and, optionally, additional arguments that help define the property being described. It shall be *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, and publicly and unambiguously derived, directly or indirectly, from its *base characteristic*, which is a specialization of the template `integral_constant` (21.3.4), with the arguments to the template `integral_constant` determined by the requirements for the particular property being described. The member names of the base characteristic shall not be hidden and shall be unambiguously available in the *Cpp17UnaryTypeTrait*.
- 2 A *Cpp17BinaryTypeTrait* describes a relationship between two types. It shall be a class template that takes two template type arguments and, optionally, additional arguments that help define the relationship being described. It shall be *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, and publicly and unambiguously derived, directly or indirectly, from its *base characteristic*, which is a specialization of the template `integral_constant` (21.3.4), with the arguments to the template `integral_constant` determined by the requirements for the particular relationship being described. The member names of the base characteristic shall not be hidden and shall be unambiguously available in the *Cpp17BinaryTypeTrait*.
- 3 A *Cpp17TransformationTrait* modifies a property of a type. It shall be a class template that takes one template type argument and, optionally, additional arguments that help define the modification. It shall declare a publicly accessible nested type named `type`, which shall be a synonym for the modified type.
- 4 Unless otherwise specified, the behavior of a program that adds specializations for any of the templates specified in 21.3 is undefined.
- 5 Unless otherwise specified, an incomplete type may be used to instantiate a template specified in 21.3. The behavior of a program is undefined if
 - (5.1) — an instantiation of a template specified in 21.3 directly or indirectly depends on an incompletely-defined object type T, and
 - (5.2) — that instantiation could yield a different result were T hypothetically completed.

21.3.3 Header `<type_traits>` synopsis

[meta.type.synop]

```

// all freestanding
namespace std {
    // 21.3.4, helper class
    template<class T, T v> struct integral_constant;

    template<bool B>
        using bool_constant = integral_constant<bool, B>;
    using true_type  = bool_constant<true>;
    using false_type = bool_constant<false>;

```

```

// 21.3.5.2, primary type categories
template<class T> struct is_void;
template<class T> struct is_null_pointer;
template<class T> struct is_integral;
template<class T> struct is_floating_point;
template<class T> struct is_array;
template<class T> struct is_pointer;
template<class T> struct is_lvalue_reference;
template<class T> struct is_rvalue_reference;
template<class T> struct is_member_object_pointer;
template<class T> struct is_member_function_pointer;
template<class T> struct is_enum;
template<class T> struct is_union;
template<class T> struct is_class;
template<class T> struct is_function;
template<class T> struct is_reflection;

// 21.3.5.3, composite type categories
template<class T> struct is_reference;
template<class T> struct is_arithmetic;
template<class T> struct is_fundamental;
template<class T> struct is_object;
template<class T> struct is_scalar;
template<class T> struct is_compound;
template<class T> struct is_member_pointer;

// 21.3.5.4, type properties
template<class T> struct is_const;
template<class T> struct is_volatile;
template<class T> struct is_trivially_copyable;
template<class T> struct is_standard_layout;
template<class T> struct is_empty;
template<class T> struct is_polymorphic;
template<class T> struct is_abstract;
template<class T> struct is_final;
template<class T> struct is_aggregate;

template<class T> struct is_structural;
template<class T> struct is_signed;
template<class T> struct is_unsigned;
template<class T> struct is_bounded_array;
template<class T> struct is_unbounded_array;
template<class T> struct is_scoped_enum;

template<class T, class... Args> struct is_constructible;
template<class T> struct is_default_constructible;
template<class T> struct is_copy_constructible;
template<class T> struct is_move_constructible;

template<class T, class U> struct is_assignable;
template<class T> struct is_copy_assignable;
template<class T> struct is_move_assignable;

template<class T, class U> struct is_swappable_with;
template<class T> struct is_swappable;

template<class T> struct is_destructible;

template<class T, class... Args> struct is_trivially_constructible;
template<class T> struct is_trivially_default_constructible;
template<class T> struct is_trivially_copy_constructible;
template<class T> struct is_trivially_move_constructible;

```

```

template<class T, class U> struct is_trivially_assignable;
template<class T> struct is_trivially_copy_assignable;
template<class T> struct is_trivially_move_assignable;
template<class T> struct is_trivially_destructible;

template<class T, class... Args> struct is_nothrow_constructible;
template<class T> struct is_nothrow_default_constructible;
template<class T> struct is_nothrow_copy_constructible;
template<class T> struct is_nothrow_move_constructible;

template<class T, class U> struct is_nothrow_assignable;
template<class T> struct is_nothrow_copy_assignable;
template<class T> struct is_nothrow_move_assignable;

template<class T, class U> struct is_nothrow_swappable_with;
template<class T> struct is_nothrow_swappable;

template<class T> struct is_nothrow_destructible;

template<class T> struct is_implicit_lifetime;

template<class T> struct has_virtual_destructor;

template<class T> struct has_unique_object_representations;

template<class T, class U> struct reference_constructs_from_temporary;
template<class T, class U> struct reference_converts_from_temporary;

// 21.3.6, type property queries
template<class T> struct alignment_of;
template<class T> struct rank;
template<class T, unsigned I = 0> struct extent;

// 21.3.7, type relations
template<class T, class U> struct is_same;
template<class Base, class Derived> struct is_base_of;
template<class Base, class Derived> struct is_virtual_base_of;
template<class From, class To> struct is_convertible;
template<class From, class To> struct is_nothrow_convertible;
template<class T, class U> struct is_layout_compatible;
template<class Base, class Derived> struct is_pointer_interconvertible_base_of;

template<class Fn, class... ArgTypes> struct is_invocable;
template<class R, class Fn, class... ArgTypes> struct is_invocable_r;

template<class Fn, class... ArgTypes> struct is_nothrow_invocable;
template<class R, class Fn, class... ArgTypes> struct is_nothrow_invocable_r;

template<class Fn, class Tuple> struct is_applicable;
template<class Fn, class Tuple> struct is_nothrow_applicable;

// 21.3.8.2, const-volatile modifications
template<class T> struct remove_const;
template<class T> struct remove_volatile;
template<class T> struct remove_cv;
template<class T> struct add_const;
template<class T> struct add_volatile;
template<class T> struct add_cv;

template<class T>
    using remove_const_t = remove_const<T>::type;
template<class T>
    using remove_volatile_t = remove_volatile<T>::type;

```

```

template<class T>
    using remove_cv_t      = remove_cv<T>::type;
template<class T>
    using add_const_t      = add_const<T>::type;
template<class T>
    using add_volatile_t   = add_volatile<T>::type;
template<class T>
    using add_cv_t         = add_cv<T>::type;

// 21.3.8.3, reference modifications
template<class T> struct remove_reference;
template<class T> struct add_lvalue_reference;
template<class T> struct add_rvalue_reference;

template<class T>
    using remove_reference_t    = remove_reference<T>::type;
template<class T>
    using add_lvalue_reference_t = add_lvalue_reference<T>::type;
template<class T>
    using add_rvalue_reference_t = add_rvalue_reference<T>::type;

// 21.3.8.4, sign modifications
template<class T> struct make_signed;
template<class T> struct make_unsigned;

template<class T>
    using make_signed_t    = make_signed<T>::type;
template<class T>
    using make_unsigned_t  = make_unsigned<T>::type;

// 21.3.8.5, array modifications
template<class T> struct remove_extent;
template<class T> struct remove_all_extents;

template<class T>
    using remove_extent_t      = remove_extent<T>::type;
template<class T>
    using remove_all_extents_t = remove_all_extents<T>::type;

// 21.3.8.6, pointer modifications
template<class T> struct remove_pointer;
template<class T> struct add_pointer;

template<class T>
    using remove_pointer_t = remove_pointer<T>::type;
template<class T>
    using add_pointer_t    = add_pointer<T>::type;

// 21.3.8.7, other transformations
template<class T> struct type_identity;
template<class T> struct remove_cvref;
template<class T> struct decay;
template<bool, class T = void> struct enable_if;
template<bool, class T, class F> struct conditional;
template<class... T> struct common_type;
template<class T, class U, template<class> class TQual, template<class> class UQual>
    struct basic_common_reference { };
template<class... T> struct common_reference;
template<class T> struct underlying_type;
template<class Fn, class... ArgTypes> struct invoke_result;
template<class Fn, class Tuple> struct apply_result;
template<class T> struct unwrap_reference;
template<class T> struct unwrap_ref_decay;

```

```

template<class T>
    using type_identity_t      = type_identity<T>::type;
template<class T>
    using remove_cvref_t      = remove_cvref<T>::type;
template<class T>
    using decay_t              = decay<T>::type;
template<bool B, class T = void>
    using enable_if_t          = enable_if<B, T>::type;
template<bool B, class T, class F>
    using conditional_t        = conditional<B, T, F>::type;
template<class... T>
    using common_type_t        = common_type<T...>::type;
template<class... T>
    using common_reference_t   = common_reference<T...>::type;
template<class T>
    using underlying_type_t    = underlying_type<T>::type;
template<class Fn, class... ArgTypes>
    using invoke_result_t      = invoke_result<Fn, ArgTypes...>::type;
template<class Fn, class Tuple>
    using apply_result_t       = apply_result<Fn, Tuple>::type;
template<class T>
    using unwrap_reference_t    = unwrap_reference<T>::type;
template<class T>
    using unwrap_ref_decay_t    = unwrap_ref_decay<T>::type;
template<class...>
    using void_t                = void;

// 21.3.9, logical operator traits
template<class... B> struct conjunction;
template<class... B> struct disjunction;
template<class B> struct negation;

// 21.3.5.2, primary type categories
template<class T>
    constexpr bool is_void_v = is_void<T>::value;
template<class T>
    constexpr bool is_null_pointer_v = is_null_pointer<T>::value;
template<class T>
    constexpr bool is_integral_v = is_integral<T>::value;
template<class T>
    constexpr bool is_floating_point_v = is_floating_point<T>::value;
template<class T>
    constexpr bool is_array_v = is_array<T>::value;
template<class T>
    constexpr bool is_pointer_v = is_pointer<T>::value;
template<class T>
    constexpr bool is_lvalue_reference_v = is_lvalue_reference<T>::value;
template<class T>
    constexpr bool is_rvalue_reference_v = is_rvalue_reference<T>::value;
template<class T>
    constexpr bool is_member_object_pointer_v = is_member_object_pointer<T>::value;
template<class T>
    constexpr bool is_member_function_pointer_v = is_member_function_pointer<T>::value;
template<class T>
    constexpr bool is_enum_v = is_enum<T>::value;
template<class T>
    constexpr bool is_union_v = is_union<T>::value;
template<class T>
    constexpr bool is_class_v = is_class<T>::value;
template<class T>
    constexpr bool is_function_v = is_function<T>::value;
template<class T>
    constexpr bool is_reflection_v = is_reflection<T>::value;

```

// 21.3.5.3, composite type categories

```
template<class T>
    constexpr bool is_reference_v = is_reference<T>::value;
template<class T>
    constexpr bool is_arithmetic_v = is_arithmetic<T>::value;
template<class T>
    constexpr bool is_fundamental_v = is_fundamental<T>::value;
template<class T>
    constexpr bool is_object_v = is_object<T>::value;
template<class T>
    constexpr bool is_scalar_v = is_scalar<T>::value;
template<class T>
    constexpr bool is_compound_v = is_compound<T>::value;
template<class T>
    constexpr bool is_member_pointer_v = is_member_pointer<T>::value;
```

// 21.3.5.4, type properties

```
template<class T>
    constexpr bool is_const_v = is_const<T>::value;
template<class T>
    constexpr bool is_volatile_v = is_volatile<T>::value;
template<class T>
    constexpr bool is_trivially_copyable_v = is_trivially_copyable<T>::value;
template<class T>
    constexpr bool is_standard_layout_v = is_standard_layout<T>::value;
template<class T>
    constexpr bool is_empty_v = is_empty<T>::value;
template<class T>
    constexpr bool is_polymorphic_v = is_polymorphic<T>::value;
template<class T>
    constexpr bool is_abstract_v = is_abstract<T>::value;
template<class T>
    constexpr bool is_final_v = is_final<T>::value;
template<class T>
    constexpr bool is_aggregate_v = is_aggregate<T>::value;
template<class T>
    constexpr bool is_structural_v = is_structural<T>::value;
template<class T>
    constexpr bool is_signed_v = is_signed<T>::value;
template<class T>
    constexpr bool is_unsigned_v = is_unsigned<T>::value;
template<class T>
    constexpr bool is_bounded_array_v = is_bounded_array<T>::value;
template<class T>
    constexpr bool is_unbounded_array_v = is_unbounded_array<T>::value;
template<class T>
    constexpr bool is_scoped_enum_v = is_scoped_enum<T>::value;
template<class T, class... Args>
    constexpr bool is_constructible_v = is_constructible<T, Args...>::value;
template<class T>
    constexpr bool is_default_constructible_v = is_default_constructible<T>::value;
template<class T>
    constexpr bool is_copy_constructible_v = is_copy_constructible<T>::value;
template<class T>
    constexpr bool is_move_constructible_v = is_move_constructible<T>::value;
template<class T, class U>
    constexpr bool is_assignable_v = is_assignable<T, U>::value;
template<class T>
    constexpr bool is_copy_assignable_v = is_copy_assignable<T>::value;
template<class T>
    constexpr bool is_move_assignable_v = is_move_assignable<T>::value;
template<class T, class U>
    constexpr bool is_swappable_with_v = is_swappable_with<T, U>::value;
```



```

template<class T>
    constexpr bool is_swappable_v = is_swappable<T>::value;
template<class T>
    constexpr bool is_destructible_v = is_destructible<T>::value;
template<class T, class... Args>
    constexpr bool is_trivially_constructible_v = is_trivially_constructible<T, Args...>::value;
template<class T>
    constexpr bool is_trivially_default_constructible_v
        = is_trivially_default_constructible<T>::value;
template<class T>
    constexpr bool is_trivially_copy_constructible_v = is_trivially_copy_constructible<T>::value;
template<class T>
    constexpr bool is_trivially_move_constructible_v = is_trivially_move_constructible<T>::value;
template<class T, class U>
    constexpr bool is_trivially_assignable_v = is_trivially_assignable<T, U>::value;
template<class T>
    constexpr bool is_trivially_copy_assignable_v = is_trivially_copy_assignable<T>::value;
template<class T>
    constexpr bool is_trivially_move_assignable_v = is_trivially_move_assignable<T>::value;
template<class T>
    constexpr bool is_trivially_destructible_v = is_trivially_destructible<T>::value;
template<class T, class... Args>
    constexpr bool is_nothrow_constructible_v = is_nothrow_constructible<T, Args...>::value;
template<class T>
    constexpr bool is_nothrow_default_constructible_v
        = is_nothrow_default_constructible<T>::value;
template<class T>
    constexpr bool is_nothrow_copy_constructible_v = is_nothrow_copy_constructible<T>::value;
template<class T>
    constexpr bool is_nothrow_move_constructible_v = is_nothrow_move_constructible<T>::value;
template<class T, class U>
    constexpr bool is_nothrow_assignable_v = is_nothrow_assignable<T, U>::value;
template<class T>
    constexpr bool is_nothrow_copy_assignable_v = is_nothrow_copy_assignable<T>::value;
template<class T>
    constexpr bool is_nothrow_move_assignable_v = is_nothrow_move_assignable<T>::value;
template<class T, class U>
    constexpr bool is_nothrow_swappable_with_v = is_nothrow_swappable_with<T, U>::value;
template<class T>
    constexpr bool is_nothrow_swappable_v = is_nothrow_swappable<T>::value;
template<class T>
    constexpr bool is_nothrow_destructible_v = is_nothrow_destructible<T>::value;
template<class T>
    constexpr bool is_implicit_lifetime_v = is_implicit_lifetime<T>::value;
template<class T>
    constexpr bool has_virtual_destructor_v = has_virtual_destructor<T>::value;
template<class T>
    constexpr bool has_unique_object_representations_v
        = has_unique_object_representations<T>::value;
template<class T, class U>
    constexpr bool reference_constructs_from_temporary_v
        = reference_constructs_from_temporary<T, U>::value;
template<class T, class U>
    constexpr bool reference_converts_from_temporary_v
        = reference_converts_from_temporary<T, U>::value;

// 21.3.6, type property queries
template<class T>
    constexpr size_t alignment_of_v = alignment_of<T>::value;
template<class T>
    constexpr size_t rank_v = rank<T>::value;
template<class T, unsigned I = 0>
    constexpr size_t extent_v = extent<T, I>::value;

```

```

// 21.3.7, type relations
template<class T, class U>
    constexpr bool is_same_v = is_same<T, U>::value;
template<class Base, class Derived>
    constexpr bool is_base_of_v = is_base_of<Base, Derived>::value;
template<class Base, class Derived>
    constexpr bool is_virtual_base_of_v = is_virtual_base_of<Base, Derived>::value;
template<class From, class To>
    constexpr bool is_convertible_v = is_convertible<From, To>::value;
template<class From, class To>
    constexpr bool is_nothrow_convertible_v = is_nothrow_convertible<From, To>::value;
template<class T, class U>
    constexpr bool is_layout_compatible_v = is_layout_compatible<T, U>::value;
template<class Base, class Derived>
    constexpr bool is_pointer_interconvertible_base_of_v
        = is_pointer_interconvertible_base_of<Base, Derived>::value;
template<class Fn, class... ArgTypes>
    constexpr bool is_invocable_v = is_invocable<Fn, ArgTypes...>::value;
template<class R, class Fn, class... ArgTypes>
    constexpr bool is_invocable_r_v = is_invocable_r<R, Fn, ArgTypes...>::value;
template<class Fn, class... ArgTypes>
    constexpr bool is_nothrow_invocable_v = is_nothrow_invocable<Fn, ArgTypes...>::value;
template<class R, class Fn, class... ArgTypes>
    constexpr bool is_nothrow_invocable_r_v = is_nothrow_invocable_r<R, Fn, ArgTypes...>::value;
template<class Fn, class Tuple>
    constexpr bool is_applicable_v = is_applicable<Fn, Tuple>::value;
template<class Fn, class Tuple>
    constexpr bool is_nothrow_applicable_v = is_nothrow_applicable<Fn, Tuple>::value;

// 21.3.9, logical operator traits
template<class... B>
    constexpr bool conjunction_v = conjunction<B...>::value;
template<class... B>
    constexpr bool disjunction_v = disjunction<B...>::value;
template<class B>
    constexpr bool negation_v = negation<B>::value;

// 21.3.10, member relationships
template<class S, class M>
    constexpr bool is_pointer_interconvertible_with_class(M S::*m) noexcept;
template<class S1, class S2, class M1, class M2>
    constexpr bool is_corresponding_member(M1 S1::*m1, M2 S2::*m2) noexcept;

// 21.3.11, constant evaluation context
constexpr bool is_constant_evaluated() noexcept;
template<class U = void, class T>
    constexpr bool is_within_lifetime(const T*) noexcept;
}

```

21.3.4 Helper classes

[meta.help]

```

namespace std {
    template<class T, T v> struct integral_constant {
        static constexpr T value = v;

        using value_type = T;
        using type = integral_constant<T, v>;

        constexpr operator value_type() const noexcept { return value; }
        constexpr value_type operator()() const noexcept { return value; }
    };
}

```

¹ The class template `integral_constant`, alias template `bool_constant`, and its associated *typedef-names* `true_type` and `false_type` are used as base classes to define the interface for various type traits.

21.3.5 Unary type traits**[meta.unary]****21.3.5.1 General****[meta.unary.general]**

- ¹ Subclause 21.3.5 contains templates that may be used to query the properties of a type at compile time.
- ² Each of these templates shall be a *Cpp17UnaryTypeTrait* (21.3.2) with a base characteristic of `true_type` if the corresponding condition is `true`, otherwise `false_type`.

21.3.5.2 Primary type categories**[meta.unary.cat]**

- ¹ The primary type categories specified in Table 52 correspond to the descriptions given in subclause 6.9 of the C++ standard.
- ² For any given type T, the result of applying one of these templates to T and to *cv* T shall yield the same result.
- ³ [Note 1: For any given type T, exactly one of the primary type categories has a `value` member that evaluates to `true`. — end note]

Table 52 — Primary type category predicates **[tab:meta.unary.cat]**

Template	Condition	Comments
<code>template<class T> struct is_void;</code>	T is void	
<code>template<class T> struct is_null_pointer;</code>	T is <code>nullptr_t</code> (6.9.3)	
<code>template<class T> struct is_integral;</code>	T is an integral type (6.9.3)	
<code>template<class T> struct is_floating_point;</code>	T is a floating-point type (6.9.3)	
<code>template<class T> struct is_array;</code>	T is an array type (6.9.5) of known or unknown extent	Class template <code>array</code> (23.3.3) is not an array type.
<code>template<class T> struct is_pointer;</code>	T is a pointer type (6.9.5)	Includes pointers to functions but not pointers to non-static members.
<code>template<class T> struct is_lvalue_reference;</code>	T is an lvalue reference type (9.3.4.3)	
<code>template<class T> struct is_rvalue_reference;</code>	T is an rvalue reference type (9.3.4.3)	
<code>template<class T> struct is_member_object_pointer;</code>	T is a pointer to data member	
<code>template<class T> struct is_member_function_pointer;</code>	T is a pointer to member function	
<code>template<class T> struct is_enum;</code>	T is an enumeration type (6.9.5)	
<code>template<class T> struct is_union;</code>	T is a union type (6.9.5)	
<code>template<class T> struct is_class;</code>	T is a non-union class type (6.9.5)	
<code>template<class T> struct is_function;</code>	T is a function type (6.9.5)	

Table 52 — Primary type category predicates (continued)

Template	Condition	Comments
<pre>template<class T> struct is_reflection;</pre>	T is std::meta::info (6.9.3)	

21.3.5.3 Composite type traits

[meta.unary.comp]

- ¹ The templates specified in [Table 53](#) provide convenient compositions of the primary type categories, corresponding to the descriptions given in subclause [6.9](#).
- ² For any given type T, the result of applying one of these templates to T and to *cv* T shall yield the same result.

Table 53 — Composite type category predicates [tab:meta.unary.comp]

Template	Condition	Comments
<pre>template<class T> struct is_reference;</pre>	T is an lvalue reference or an rvalue reference	
<pre>template<class T> struct is_arithmetic;</pre>	T is an arithmetic type (6.9.3)	
<pre>template<class T> struct is_fundamental;</pre>	T is a fundamental type (6.9.3)	
<pre>template<class T> struct is_object;</pre>	T is an object type (6.9.1)	
<pre>template<class T> struct is_scalar;</pre>	T is a scalar type (6.9.1)	
<pre>template<class T> struct is_compound;</pre>	T is a compound type (6.9.5)	
<pre>template<class T> struct is_member_pointer;</pre>	T is a pointer-to-member type (6.9.5)	

21.3.5.4 Type properties

[meta.unary.prop]

- ¹ The templates specified in [Table 54](#) provide access to some of the more important properties of types.
- ² It is unspecified whether the library defines any full or partial specializations of any of these templates.
- ³ For all of the class templates **X** declared in this subclause, instantiating that template with a template-argument that is a class template specialization may result in the implicit instantiation of the template argument if and only if the semantics of **X** require that the argument is a complete type.
- ⁴ For the purpose of defining the templates in this subclause, a function call expression `declval<T>()` for any type T is considered to be a trivial ([D.13](#), [11.4.4](#)) function call that is not an odr-use ([6.3](#)) of `declval` in the context of the corresponding definition notwithstanding the restrictions of [22.2.6](#).
- ⁵ For the purpose of defining the templates in this subclause, let `VAL<T>` for some type T be an expression defined as follows:
 - (5.1) — If T is a reference or function type, `VAL<T>` is an expression with the same type and value category as `declval<T>()`.
 - (5.2) — Otherwise, `VAL<T>` is a prvalue that initially has type T.
 [Note 1: If T is cv-qualified, the cv-qualification is subject to adjustment ([7.2.2](#)). — end note]

Table 54 — Type property predicates [tab:meta.unary.prop]

Template	Condition	Preconditions
<code>template<class T> struct is_const;</code>	T is const-qualified (6.9.6)	
<code>template<class T> struct is_volatile;</code>	T is volatile-qualified (6.9.6)	
<code>template<class T> struct is_trivially_copyable;</code>	T is a trivially copyable type (6.9.1)	<code>remove_all_extents_t<T></code> shall be a complete type or <i>cv</i> void.
<code>template<class T> struct is_standard_layout;</code>	T is a standard-layout type (6.9.1)	<code>remove_all_extents_t<T></code> shall be a complete type or <i>cv</i> void.
<code>template<class T> struct is_empty;</code>	T is a class type, but not a union type, with no non-static data members other than subobjects of zero size, no virtual member functions, no virtual base classes, and no base class B for which <code>is_empty_v</code> is false.	If T is a non-union class type, T shall be a complete type.
<code>template<class T> struct is_polymorphic;</code>	T is a polymorphic class (11.7.3)	If T is a non-union class type, T shall be a complete type.
<code>template<class T> struct is_abstract;</code>	T is an abstract class (11.7.4)	If T is a non-union class type, T shall be a complete type.
<code>template<class T> struct is_final;</code>	T is a class type marked with the <i>class-property-specifier</i> final (11.1). [Note 2: A union is a class type that can be marked with final . — end note]	If T is a class type, T shall be a complete type.
<code>template<class T> struct is_aggregate;</code>	T is an aggregate type (9.5.2)	T shall be an array type, a complete type, or <i>cv</i> void.
<code>template<class T> struct is_structural;</code>	T is a structural type (13.2)	<code>remove_all_extents_t<T></code> shall be a complete type or <i>cv</i> void.
<code>template<class T> struct is_signed;</code>	If <code>is_arithmetic_v<T></code> is true, the same result as <code>T(-1) < T(0)</code> ; otherwise, false	
<code>template<class T> struct is_unsigned;</code>	If <code>is_arithmetic_v<T></code> is true, the same result as <code>T(0) < T(-1)</code> ; otherwise, false	
<code>template<class T> struct is_bounded_array;</code>	T is an array type of known bound (9.3.4.5)	
<code>template<class T> struct is_unbounded_array;</code>	T is an array type of unknown bound (9.3.4.5)	
<code>template<class T> struct is_scoped_enum;</code>	T is a scoped enumeration (9.8.1)	

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
<code>template<class T, class... Args> struct is_constructible;</code>	For a function type T or for a <i>cv</i> void type T, <code>is_constructible_v<T, Args...></code> is false, otherwise <i>see below</i>	T and all types in the template parameter pack Args shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<code>template<class T> struct is_default_constructible;</code>	<code>is_constructible_v<T></code> is true.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<code>template<class T> struct is_copy_constructible;</code>	For a referenceable type T (3.46), the same result as <code>is_constructible_v<T, const T&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<code>template<class T> struct is_move_constructible;</code>	For a referenceable type T, the same result as <code>is_constructible_v<T, T&&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<code>template<class T, class U> struct is_assignable;</code>	The expression <code>declval<T>() = declval<U>()</code> is well-formed when treated as an unevaluated operand (7.2.3). Access checking is performed as if in a context unrelated to T and U. Only the validity of the immediate context of the assignment expression is considered. [Note 3: The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — end note]	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<code>template<class T> struct is_copy_assignable;</code>	For a referenceable type T, the same result as <code>is_assignable_v<T&, const T&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<code>template<class T> struct is_move_assignable;</code>	For a referenceable type T, the same result as <code>is_assignable_v<T&, T&&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
<pre>template<class T, class U> struct is_swappable_with;</pre>	The expressions <code>swap(declval<T>())</code>, <code>declval<U>()</code> and <code>swap(declval<U>())</code>, <code>declval<T>()</code> are each well-formed when treated as an unevaluated operand (7.2.3) in an overload-resolution context for swappable values (16.4.4.3). Access checking is performed as if in a context unrelated to T and U. Only the validity of the immediate context of the <code>swap</code> expressions is considered. [Note 4: The compilation of the expressions can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — end note]	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<pre>template<class T> struct is_swappable;</pre>	For a referenceable type T, the same result as <code>is_swappable_with_v<T&, T&></code> , otherwise false .	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_destructible;</pre>	Either T is a reference type, or T is a complete object type for which the expression <code>declval<U&>().~U()</code> is well-formed when treated as an unevaluated operand (7.2.3), where U is <code>remove_all_extents_t<T></code> .	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T, class... Args> struct is_trivially_constructible;</pre>	<code>is_constructible_v<T, Args...></code> is true and the variable definition for <code>is_constructible</code> , as defined below, is known to call no operation that is not trivial (D.13, 11.4.4).	T and all types in the template parameter pack Args shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<pre>template<class T> struct is_trivially_default_constructible;</pre>	<code>is_trivially_constructible_v<T></code> is true .	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
<pre>template<class T> struct is_trivially_copy_constructible;</pre>	For a referenceable type T, the same result as <code>is_trivially_constructible_v<T, const T&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_trivially_move_constructible;</pre>	For a referenceable type T, the same result as <code>is_trivially_constructible_v<T, T&&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T, class U> struct is_trivially_assignable;</pre>	<code>is_assignable_v<T, U></code> is true and the assignment, as defined by <code>is_assignable</code> , is known to call no operation that is not trivial (D.13, 11.4.4).	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<pre>template<class T> struct is_trivially_copy_assignable;</pre>	For a referenceable type T, the same result as <code>is_trivially_assignable_v<T&, const T&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_trivially_move_assignable;</pre>	For a referenceable type T, the same result as <code>is_trivially_assignable_v<T&, T&&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_trivially_destructible;</pre>	<code>is_destructible_v<T></code> is true and <code>remove_all_extents_t<T></code> is either a non-class type or a class type with a trivial destructor.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T, class... Args> struct is_nothrow_constructible;</pre>	<code>is_constructible_v<T, Args...></code> is true and the variable definition for <code>is_constructible</code> , as defined below, is known not to throw any exceptions (7.6.2.7).	T and all types in the template parameter pack Args shall be complete types, <i>cv</i> void, or arrays of unknown bound.
<pre>template<class T> struct is_nothrow_default_constructible;</pre>	<code>is_nothrow_constructible_v<T></code> is true.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_nothrow_copy_constructible;</pre>	For a referenceable type T, the same result as <code>is_nothrow_constructible_v<T, const T&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
<pre>template<class T> struct is_nothrow_move_constructible;</pre>	For a referenceable type T, the same result as <code>is_nothrow_constructible_v<T, T&&></code> , otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
template<class T, class U> struct is_nothrow_assignable;	is_assignable_v<T, U> is true and the assignment is known not to throw any exceptions (7.6.2.7).	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.
template<class T> struct is_nothrow_copy_assignable;	For a referenceable type T, the same result as is_nothrow_ assignable_v<T&, const T&>, otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
template<class T> struct is_nothrow_move_assignable;	For a referenceable type T, the same result as is_nothrow_ assignable_v<T&, T&&>, otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
template<class T, class U> struct is_nothrow_swappable_with;	is_swappable_with_v<T, U> is true and each swap expression of the definition of is_swappable_with<T, U> is known not to throw any exceptions (7.6.2.7).	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.
template<class T> struct is_nothrow_swappable;	For a referenceable type T, the same result as is_nothrow_swappable_ with_v<T&, T&>, otherwise false.	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
template<class T> struct is_nothrow_destructible;	is_destructible_v<T> is true and the indicated destructor is known not to throw any exceptions (7.6.2.7).	T shall be a complete type, <i>cv</i> void, or an array of unknown bound.
template<class T> struct is_implicit_lifetime;	T is an implicit-lifetime type (6.9.1).	T shall be an array type, a complete type, or <i>cv</i> void.
template<class T> struct has_virtual_destructor;	T has a virtual destructor (11.4.7)	If T is a non-union class type, T shall be a complete type.
template<class T> struct has_unique_object_representations;	For an array type T, the same result as has_unique_object_ representations_ v<remove_all_extents_ t<T>>, otherwise <i>see</i> <i>below</i> .	remove_all_extents_ t<T> shall be a complete type or <i>cv</i> void.

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
<pre>template<class T, class U> struct reference_constructs_from_ temporary;</pre>	T is a reference type, and the initialization T t(VAL<U>); is well-formed and binds t to a temporary object whose lifetime is extended (6.8.7). The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3). Access checking is performed as if in a context unrelated to T and U. Only the validity of the immediate context of the variable initialization is considered. [Note 5: The initialization can result in effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such effects are not in the “immediate context” and can result in the program being ill-formed. — end note]	T and U shall be complete types, cv void, or arrays of unknown bound.

Table 54 — Type property predicates (continued)

Template	Condition	Preconditions
<pre>template<class T, class U> struct reference_converts_from_temporary;</pre>	T is a reference type, and the initialization <code>T t = VAL<U>;</code> is well-formed and binds <code>t</code> to a temporary object whose lifetime is extended (6.8.7). The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3). Access checking is performed as if in a context unrelated to T and U. Only the validity of the immediate context of the variable initialization is considered. [Note 6: The initialization can result in effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such effects are not in the “immediate context” and can result in the program being ill-formed. — end note]	T and U shall be complete types, <i>cv</i> void, or arrays of unknown bound.

6 [Example 1:

```
is_const_v<const volatile int>    // true
is_const_v<const int*>           // false
is_const_v<const int&>           // false
is_const_v<int[3]>                // false
is_const_v<const int[3]>         // true
```

— end example]

7 [Example 2:

```
remove_const_t<const volatile int> // volatile int
remove_const_t<const int* const>   // const int*
remove_const_t<const int&>         // const int&
remove_const_t<const int[3]>       // int[3]
```

— end example]

8 [Example 3:

```
// Given:
struct P final { };
union U1 { };
union U2 final { };

// the following assertions hold:
static_assert(!is_final_v<int>);
static_assert(is_final_v<P>);
static_assert(!is_final_v<U1>);
static_assert(is_final_v<U2>);
```

— end example]

- ⁹ The predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:

```
T t(declval<Args>()...);
```

[*Note 7*: These tokens are never interpreted as a function declaration. — *end note*]

The full-expression of the variable initialization is treated as an unevaluated operand (7.2.3). Access checking is performed as if in a context unrelated to `T` and any of the `Args`. Only the validity of the immediate context of the variable initialization is considered.

[*Note 8*: The evaluation of the initialization can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — *end note*]

- ¹⁰ The predicate condition for a template specialization `has_unique_object_representations<T>` shall be satisfied if and only if

- (10.1) — `T` is trivially copyable, and
- (10.2) — any two objects of type `T` with the same value have the same object representation, where
 - (10.2.1) — two objects of array or non-union class type are considered to have the same value if their respective sequences of direct subobjects have the same values, and
 - (10.2.2) — two objects of union type are considered to have the same value if they have the same active member and the corresponding members have the same value.

The set of scalar types for which this condition holds is implementation-defined.

[*Note 9*: If a type has padding bits, the condition does not hold; otherwise, the condition holds true for integral types. — *end note*]

21.3.6 Type property queries

[`meta.unary.prop.query`]

- ¹ The templates specified in Table 55 may be used to query properties of types at compile time.

Table 55 — Type property queries [tab:meta.unary.prop.query]

Template	Value
<pre>template<class T> struct alignof_of;</pre>	<code>alignof(T)</code>. <i>Mandates</i>: <code>alignof(T)</code> is a valid expression (7.6.2.6)
<pre>template<class T> struct rank;</pre>	If <code>T</code> is an array type, an integer value representing the number of dimensions of <code>T</code>; otherwise, 0.
<pre>template<class T, unsigned I = 0> struct extent;</pre>	If <code>T</code> is not an array type, or if it has rank less than or equal to <code>I</code>, or if <code>I</code> is 0 and <code>T</code> has type “array of unknown bound of <code>U</code>”, then 0; otherwise, the bound (9.3.4.5) of the I^{th} dimension of <code>T</code>, where indexing of <code>I</code> is zero-based

- ² Each of these templates shall be a *Cpp17UnaryTypeTrait* (21.3.2) with a base characteristic of `integral_constant<size_t, Value>`.

- ³ [*Example 1*:

```
// the following assertions hold:
static_assert(rank_v<int> == 0);
static_assert(rank_v<int[2]> == 1);
static_assert(rank_v<int[] [4]> == 2);
```

— *end example*]

- ⁴ [*Example 2*:

```
// the following assertions hold:
static_assert(extent_v<int> == 0);
static_assert(extent_v<int[2]> == 2);
static_assert(extent_v<int[2] [4]> == 2);
static_assert(extent_v<int[] [4]> == 0);
static_assert(extent_v<int, 1> == 0);
static_assert(extent_v<int[2], 1> == 0);
```

```
static_assert(extent_v<int[2][4], 1> == 4);
static_assert(extent_v<int[][4], 1> == 4);
```

— end example]

21.3.7 Relationships between types [meta.rel]

- ¹ The templates specified in [Table 56](#) may be used to query relationships between types at compile time.
- ² Each of these templates shall be a *Cpp17BinaryTypeTrait* ([21.3.2](#)) with a base characteristic of `true_type` if the corresponding condition is true, otherwise `false_type`.
- ³ Let *ELEMS-OF*(T) be the parameter pack `get<N>(declval<T>())`, where *N* is the pack of `size_t` template arguments of the specialization of `index_sequence` denoted by `make_index_sequence<tuple_size_v<remove_reference_t<T>>>`.

Table 56 — Type relationship predicates [tab:meta.rel]

Template	Condition	Comments
<code>template<class T, class U> struct is_same;</code>	T and U name the same type with the same cv-qualifications	
<code>template<class Base, class Derived> struct is_base_of;</code>	Base is a base class of Derived (11.7) without regard to cv-qualifiers or Base and Derived are not unions and name the same class type without regard to cv-qualifiers	If Base and Derived are non-union class types and are not (possibly cv-qualified versions of) the same type, Derived shall be a complete type. [Note 1: Base classes that are private, protected, or ambiguous are, nonetheless, base classes. — end note]
<code>template<class Base, class Derived> struct is_virtual_base_of;</code>	Base is a virtual base class of Derived (11.7.2) without regard to cv-qualifiers.	If Base and Derived are non-union class types, Derived shall be a complete type. [Note 2: Virtual base classes that are private, protected, or ambiguous are, nonetheless, virtual base classes. — end note] [Note 3: A class is never a virtual base class of itself. — end note]
<code>template<class From, class To> struct is_convertible;</code>	<i>see below</i>	From and To shall be complete types, cv void, or arrays of unknown bound.
<code>template<class From, class To> struct is_nothrow_convertible;</code>	<code>is_convertible_v<From, To></code> is true and the conversion, as defined by <code>is_convertible</code> , is known not to throw any exceptions (7.6.2.7)	From and To shall be complete types, cv void, or arrays of unknown bound.
<code>template<class T, class U> struct is_layout_compatible;</code>	T and U are layout-compatible (6.9.1)	T and U shall be complete types, cv void, or arrays of unknown bound.

Table 56 — Type relationship predicates (continued)

Template	Condition	Comments
<pre>template<class Base, class Derived> struct is_pointer_ interconvertible_base_of;</pre>	Derived is unambiguously derived from Base without regard to cv-qualifiers, and each object of type Derived is pointer-interconvertible (6.9.5) with its Base subobject, or Base and Derived are not unions and name the same class type without regard to cv-qualifiers.	If Base and Derived are non-union class types and are not (possibly cv-qualified versions of) the same type, Derived shall be a complete type.
<pre>template<class Fn, class... ArgTypes> struct is_invocable;</pre>	The expression <i>INVOKE</i> (declval<Fn>(), declval<ArgTypes>()...) (22.10.4) is well-formed when treated as an unevaluated operand (7.2.3)	Fn and all types in the template parameter pack ArgTypes shall be complete types, cv void, or arrays of unknown bound.
<pre>template<class R, class Fn, class... ArgTypes> struct is_invocable_r;</pre>	The expression <i>INVOKE</i> <R>(declval<Fn>(), declval<ArgTypes>()...) is well-formed when treated as an unevaluated operand	Fn, R, and all types in the template parameter pack ArgTypes shall be complete types, cv void, or arrays of unknown bound.
<pre>template<class Fn, class... ArgTypes> struct is_nothrow_invocable;</pre>	<i>is_invocable_v</i> <Fn, ArgTypes...> is true and the expression <i>INVOKE</i> (declval<Fn>(), declval<ArgTypes>()...) is known not to throw any exceptions (7.6.2.7)	Fn and all types in the template parameter pack ArgTypes shall be complete types, cv void, or arrays of unknown bound.
<pre>template<class R, class Fn, class... ArgTypes> struct is_nothrow_invocable_r;</pre>	<i>is_invocable_r_v</i> <R, Fn, ArgTypes...> is true and the expression <i>INVOKE</i> <R>(declval<Fn>(), declval<ArgTypes>()...) is known not to throw any exceptions (7.6.2.7)	Fn, R, and all types in the template parameter pack ArgTypes shall be complete types, cv void, or arrays of unknown bound.
<pre>template<class Fn, class Tuple> struct is_applicable;</pre>	<i>tuple-like</i> <Tuple> is true and the expression <i>INVOKE</i> (declval<Fn>(), <i>ELEMS-OF</i> (Tuple)...) is well-formed when treated as an unevaluated operand.	Fn and Tuple shall be complete types, cv void, or arrays of unknown bound.
<pre>template<class Fn, class Tuple> struct is_nothrow_applicable;</pre>	<i>is_applicable_v</i> <Fn, Tuple> is true and the expression <i>INVOKE</i> (declval<Fn>(), <i>ELEMS-OF</i> (Tuple)...) is known not to throw any exceptions (7.6.2.7).	Fn and Tuple shall be complete types, cv void, or arrays of unknown bound.

⁴ For the purpose of defining the templates in this subclause, a function call expression *declval*<T>() for any type T is considered to be a trivial (D.13, 11.4.4) function call that is not an odr-use (6.3) of *declval* in the context of the corresponding definition notwithstanding the restrictions of 22.2.6.

⁵ [Example 1:

```
struct B {};
struct B1 : B {};
```

```

struct B2 : B {};
struct D : private B1, private B2 {};

is_base_of_v<B, D>           // true
is_base_of_v<const B, D>     // true
is_base_of_v<B, const D>     // true
is_base_of_v<B, const B>     // true
is_base_of_v<D, B>           // false
is_base_of_v<B&, D&>         // false
is_base_of_v<B[3], D[3]>     // false
is_base_of_v<int, int>       // false

```

— end example]

- ⁶ The predicate condition for a template specialization `is_convertible<From, To>` shall be satisfied if and only if the return statement (8.8.4) in the following code would be well-formed, including any implicit conversions to the return type of the function:

```

To test() {
    return declval<From>();
}

```

[Note 4: This requirement gives well-defined results for reference types, array types, function types, and *cv* void. — end note]

Access checking is performed in a context unrelated to `To` and `From`. The operand of the `return` statement (including initialization of the returned object or reference, if any) is treated as an unevaluated operand (7.2.3), and only the validity of its immediate context is considered.

[Note 5: The initialization can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — end note]

21.3.8 Transformations between types

[meta.trans]

21.3.8.1 General

[meta.trans.general]

- ¹ Subclause 21.3.8 contains templates that may be used to transform one type to another following some predefined rule.
- ² Each of the templates in 21.3.8 shall be a *Cpp17TransformationTrait* (21.3.2).

21.3.8.2 Const-volatile modifications

[meta.trans.cv]

- ¹ The templates specified in Table 57 add or remove cv-qualifications (6.9.6).

Table 57 — Const-volatile modifications [tab:meta.trans.cv]

Template	Comments
<pre>template<class T> struct remove_const;</pre>	The member typedef <code>type</code> denotes the type formed by removing any top-level const-qualifier from <code>T</code>. [Example 1: <code>remove_const_t<const volatile int></code> evaluates to <code>volatile int</code>, whereas <code>remove_const_t<const int*></code> evaluates to <code>const int*</code>. — end example]
<pre>template<class T> struct remove_volatile;</pre>	The member typedef <code>type</code> denotes the type formed by removing any top-level volatile-qualifier from <code>T</code>. [Example 2: <code>remove_volatile_t<const volatile int></code> evaluates to <code>const int</code>, whereas <code>remove_volatile_t<volatile int*></code> evaluates to <code>volatile int*</code>. — end example]
<pre>template<class T> struct remove_cv;</pre>	The member typedef <code>type</code> denotes the type formed by removing any top-level cv-qualifiers from <code>T</code>. [Example 3: <code>remove_cv_t<const volatile int></code> evaluates to <code>int</code>, whereas <code>remove_cv_t<const volatile int*></code> evaluates to <code>const volatile int*</code>. — end example]
<pre>template<class T> struct add_const;</pre>	The member typedef <code>type</code> denotes <code>const T</code>. [Note 1: <code>const</code> has no effect when <code>T</code> is a reference, function, or top-level const-qualified type. — end note]

Table 57 — Const-volatile modifications (continued)

Template	Comments
<pre>template<class T> struct add_volatile;</pre>	The member typedef type denotes volatile T . [Note 2: volatile has no effect when T is a reference, function, or top-level volatile-qualified type. — end note]
<pre>template<class T> struct add_cv;</pre>	The member typedef type denotes add_const_t<add_volatile_t<T>> .

21.3.8.3 Reference modifications

[meta.trans.ref]

- ¹ The templates specified in Table 58 add or remove references.

Table 58 — Reference modifications [tab:meta.trans.ref]

Template	Comments
<pre>template<class T> struct remove_reference;</pre>	If T has type “reference to T1 ” then the member typedef type denotes T1 ; otherwise, type denotes T .
<pre>template<class T> struct add_lvalue_reference;</pre>	If T is a referenceable type (3.46) then the member typedef type denotes T& ; otherwise, type denotes T . [Note 1: This rule reflects the semantics of reference collapsing (9.3.4.3). — end note]
<pre>template<class T> struct add_rvalue_reference;</pre>	If T is a referenceable type then the member typedef type denotes T&& ; otherwise, type denotes T . [Note 2: This rule reflects the semantics of reference collapsing (9.3.4.3). For example, when a type T is a reference type T1& , the type add_rvalue_reference_t<T> is not an rvalue reference. — end note]

21.3.8.4 Sign modifications

[meta.trans.sign]

- ¹ The templates specified in Table 59 convert an integer type to its corresponding signed or unsigned type.

Table 59 — Sign modifications [tab:meta.trans.sign]

Template	Comments
<pre>template<class T> struct make_signed;</pre>	If T is a (possibly cv-qualified) signed integer type (6.9.3) then the member typedef type denotes T ; otherwise, if T is a (possibly cv-qualified) unsigned integer type then type denotes the corresponding signed integer type, with the same cv-qualifiers as T ; otherwise, type denotes the signed integer type with smallest rank (6.9.7) for which sizeof(T) == sizeof(type) , with the same cv-qualifiers as T . <i>Mandates:</i> T is an integral or enumeration type other than <i>cv bool</i> .
<pre>template<class T> struct make_unsigned;</pre>	If T is a (possibly cv-qualified) unsigned integer type (6.9.3) then the member typedef type denotes T ; otherwise, if T is a (possibly cv-qualified) signed integer type then type denotes the corresponding unsigned integer type, with the same cv-qualifiers as T ; otherwise, type denotes the unsigned integer type with smallest rank (6.9.7) for which sizeof(T) == sizeof(type) , with the same cv-qualifiers as T . <i>Mandates:</i> T is an integral or enumeration type other than <i>cv bool</i> .

21.3.8.5 Array modifications

[meta.trans.arr]

- ¹ The templates specified in Table 60 modify array types.

Table 60 — Array modifications [tab:meta.trans.arr]

Template	Comments
<pre>template<class T> struct remove_extent;</pre>	If T is a type “array of U”, the member typedef <code>type</code> denotes U, otherwise T. [Note 1: For multidimensional arrays, only the first array dimension is removed. For a type “array of <code>const U</code>”, the resulting type is <code>const U</code>. — end note]
<pre>template<class T> struct remove_all_extents;</pre>	If T is “multidimensional array of U”, the resulting member typedef <code>type</code> denotes U, otherwise T.

² [Example 1:

// the following assertions hold:

```
static_assert(is_same_v<remove_extent_t<int>, int>);
static_assert(is_same_v<remove_extent_t<int[2]>, int>);
static_assert(is_same_v<remove_extent_t<int[2][3]>, int[3]>);
static_assert(is_same_v<remove_extent_t<int[][3]>, int[3]>);
```

— end example]

³ [Example 2:

// the following assertions hold:

```
static_assert(is_same_v<remove_all_extents_t<int>, int>);
static_assert(is_same_v<remove_all_extents_t<int[2]>, int>);
static_assert(is_same_v<remove_all_extents_t<int[2][3]>, int>);
static_assert(is_same_v<remove_all_extents_t<int[][3]>, int>);
```

— end example]

21.3.8.6 Pointer modifications

[meta.trans.ptr]

¹ The templates specified in Table 61 add or remove pointers.

Table 61 — Pointer modifications [tab:meta.trans.ptr]

Template	Comments
<pre>template<class T> struct remove_pointer;</pre>	If T has type “(possibly cv-qualified) pointer to T1” then the member typedef <code>type</code> denotes T1; otherwise, it denotes T.
<pre>template<class T> struct add_pointer;</pre>	If T is a referenceable type (3.46) or a <i>cv</i> void type then the member typedef <code>type</code> denotes <code>remove_reference_t<T>*</code>; otherwise, <code>type</code> denotes T.

21.3.8.7 Other transformations

[meta.trans.other]

¹ The templates specified in Table 62 perform other modifications of a type.

Table 62 — Other transformations [tab:meta.trans.other]

Template	Comments
<pre>template<class T> struct type_identity;</pre>	The member typedef <code>type</code> denotes T.
<pre>template<class T> struct remove_cvref;</pre>	The member typedef <code>type</code> denotes <code>remove_cv_t<remove_reference_t<T>></code>.

Table 62 — Other transformations (continued)

Template	Comments
<pre>template<class T> struct decay;</pre>	Let <code>U</code> be <code>remove_reference_t<T></code>. If <code>is_array_v<U></code> is true, the member typedef <code>type</code> denotes <code>remove_extent_t<U>*</code>. If <code>is_function_v<U></code> is true, the member typedef <code>type</code> denotes <code>add_pointer_t<U></code>. Otherwise the member typedef <code>type</code> denotes <code>remove_cv_t<U></code>. [<i>Note 1</i>: This behavior is similar to the lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) conversions applied when an lvalue is used as an rvalue, but also strips cv-qualifiers from class types in order to more closely model by-value argument passing. — <i>end note</i>]
<pre>template<bool B, class T = void> struct enable_if;</pre>	If <code>B</code> is true, the member typedef <code>type</code> denotes <code>T</code>; otherwise, there shall be no member <code>type</code>.
<pre>template<bool B, class T, class F> struct conditional;</pre>	If <code>B</code> is true, the member typedef <code>type</code> denotes <code>T</code>. If <code>B</code> is false, the member typedef <code>type</code> denotes <code>F</code>.
<pre>template<class... T> struct common_type;</pre>	Unless this trait is specialized, the member <code>type</code> is declared or omitted as specified below. If it is omitted, there shall be no member <code>type</code>. Each type in the template parameter pack <code>T</code> shall be complete, <i>cv</i> void, or an array of unknown bound.
<pre>template<class, class, template<class> class, template<class> class> struct basic_common_reference;</pre>	Unless this trait is specialized, there shall be no member <code>type</code>.
<pre>template<class... T> struct common_reference;</pre>	The member <i>typedef-name</i> <code>type</code> is declared or omitted as specified below. Each type in the parameter pack <code>T</code> shall be complete or <i>cv</i> void.
<pre>template<class T> struct underlying_type;</pre>	If <code>T</code> is an enumeration type, the member typedef <code>type</code> denotes the underlying type of <code>T</code> (9.8.1); otherwise, there is no member <code>type</code>. <i>Mandates</i>: <code>T</code> is not an incomplete enumeration type.
<pre>template<class Fn, class... ArgTypes> struct invoke_result;</pre>	If the expression <code>INVOKE(declval<Fn>(), declval<ArgTypes>()...)</code> (22.10.4) is well-formed when treated as an unevaluated operand (7.2.3), the member typedef <code>type</code> denotes the type <code>decltype(INVOKE(declval<Fn>(), declval<ArgTypes>()...))</code>; otherwise, there shall be no member <code>type</code>. Access checking is performed as if in a context unrelated to <code>Fn</code> and <code>ArgTypes</code>. Only the validity of the immediate context of the expression is considered. [<i>Note 2</i>: The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — <i>end note</i>] <i>Preconditions</i>: <code>Fn</code> and all types in the template parameter pack <code>ArgTypes</code> are complete types, <i>cv</i> void, or arrays of unknown bound.

Table 62 — Other transformations (continued)

Template	Comments
<pre>template<class Fn, class Tuple> struct apply_result;</pre>	If <i>tuple-like</i><Tuple> is true and the expression <i>INVOKE</i>(declval<Fn>(), <i>ELEMS-OF</i>(Tuple)...) (22.10.4) is well-formed when treated as an unevaluated operand (7.2.3), the member typedef <i>type</i> denotes the type <i>decltype</i>(<i>INVOKE</i>(declval<Fn>(), <i>ELEMS-OF</i>(Tuple)...)); otherwise, there shall be no member <i>type</i>. Access checking is performed as if in a context unrelated to <i>Fn</i> and <i>Tuple</i>. Only the validity of the immediate context of the expression is considered. [Note 3: The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — end note] <i>Preconditions</i>: <i>Fn</i> and <i>Tuple</i> are complete types, <i>cv</i> void, or arrays of unknown bound.
<pre>template<class T> struct unwrap_reference;</pre>	If <i>T</i> is a specialization <i>reference_wrapper</i><<i>X</i>> for some type <i>X</i>, the member typedef <i>type</i> of <i>unwrap_reference</i><<i>T</i>> denotes <i>X</i>&, otherwise <i>type</i> denotes <i>T</i>.
<pre>template<class T> unwrap_ref_decay;</pre>	The member typedef <i>type</i> of <i>unwrap_ref_decay</i><<i>T</i>> denotes the type <i>unwrap_reference_t</i><<i>decay_t</i><<i>T</i>>>.

² In addition to being available via inclusion of the <type_traits> header, the templates *unwrap_reference*, *unwrap_ref_decay*, *unwrap_reference_t*, and *unwrap_ref_decay_t* are available when the header <functional> (22.10.2) is included.

³ Let:

- (3.1) — *CREF*(*A*) be *add_lvalue_reference_t*<const *remove_reference_t*<*A*>>,
- (3.2) — *XREF*(*A*) denote a unary alias template *T* such that *T*<*U*> denotes the same type as *U* with the addition of *A*’s cv and reference qualifiers, for a non-reference cv-unqualified type *U*,
- (3.3) — *COPYCV*(*FROM*, *TO*) be an alias for type *TO* with the addition of *FROM*’s top-level cv-qualifiers,
[Example 1: *COPYCV*(const int, volatile short) is an alias for const volatile short. — end example]
- (3.4) — *COND-RES*(*X*, *Y*) be *decltype*(false ? declval<*X*(&())>() : declval<*Y*(&())>()).

Given types *A* and *B*, let *X* be *remove_reference_t*<*A*>, let *Y* be *remove_reference_t*<*B*>, and let *COMMON-REF*(*A*, *B*) be:

- (3.5) — If *A* and *B* are both lvalue reference types, *COMMON-REF*(*A*, *B*) is *COND-RES*(*COPYCV*(*X*, *Y*) &, *COPYCV*(*Y*, *X*) &) if that type exists and is a reference type.
- (3.6) — Otherwise, let *C* be *remove_reference_t*<*COMMON-REF*(*X*&, *Y*&)>&&. If *A* and *B* are both rvalue reference types, *C* is well-formed, and *is_convertible_v*<*A*, *C*> && *is_convertible_v*<*B*, *C*> is true, then *COMMON-REF*(*A*, *B*) is *C*.
- (3.7) — Otherwise, let *D* be *COMMON-REF*(const *X*&, *Y*&). If *A* is an rvalue reference and *B* is an lvalue reference and *D* is well-formed and *is_convertible_v*<*A*, *D*> is true, then *COMMON-REF*(*A*, *B*) is *D*.
- (3.8) — Otherwise, if *A* is an lvalue reference and *B* is an rvalue reference, then *COMMON-REF*(*A*, *B*) is *COMMON-REF*(*B*, *A*).
- (3.9) — Otherwise, *COMMON-REF*(*A*, *B*) is ill-formed.

If any of the types computed above is ill-formed, then *COMMON-REF*(*A*, *B*) is ill-formed.

⁴ For the *common_type* trait applied to a template parameter pack *T* of types, the member *type* shall be either declared or not present as follows:

- (4.1) — If *sizeof... (T)* is zero, there shall be no member *type*.
- (4.2) — If *sizeof... (T)* is one, let *T0* denote the sole type constituting the pack *T*. The member *typedef-name type* shall denote the same type, if any, as *common_type_t*<*T0*, *T0*>; otherwise there shall be no member *type*.

- (4.3) — If `sizeof...(T)` is two, let the first and second types constituting `T` be denoted by `T1` and `T2`, respectively, and let `D1` and `D2` denote the same types as `decay_t<T1>` and `decay_t<T2>`, respectively.
- (4.3.1) — If `is_same_v<T1, D1>` is false or `is_same_v<T2, D2>` is false, let `C` denote the same type, if any, as `common_type_t<D1, D2>`.
- (4.3.2) — [Note 4: None of the following will apply if there is a specialization `common_type<D1, D2>`. — end note]
- (4.3.3) — Otherwise, if

`decay_t<decltype(false ? declval<D1>() : declval<D2>())>`
 denotes a valid type, let `C` denote that type.
- (4.3.4) — Otherwise, if `COND-RES(CREF(D1), CREF(D2))` denotes a type, let `C` denote the type `decay_t<COND-RES(CREF(D1), CREF(D2))>`.

In either case, the member *typedef-name* `type` shall denote the same type, if any, as `C`. Otherwise, there shall be no member `type`.

- (4.4) — If `sizeof...(T)` is greater than two, let `T1`, `T2`, and `R`, respectively, denote the first, second, and (pack of) remaining types constituting `T`. Let `C` denote the same type, if any, as `common_type_t<T1, T2>`. If there is such a type `C`, the member *typedef-name* `type` shall denote the same type, if any, as `common_type_t<C, R...>`. Otherwise, there shall be no member `type`.

- 5 Notwithstanding the provisions of 21.3.2, and pursuant to 16.4.5.2.1, a program may specialize `common_type<T1, T2>` for types `T1` and `T2` such that `is_same_v<T1, decay_t<T1>>` and `is_same_v<T2, decay_t<T2>>` are each true.

[Note 5: Such specializations are needed when only explicit conversions are desired between the template arguments. — end note]

Such a specialization need not have a member named `type`, but if it does, the *qualified-id* `common_type<T1, T2>::type` shall denote a cv-unqualified non-reference type to which each of the types `T1` and `T2` is explicitly convertible. Moreover, `common_type_t<T1, T2>` shall denote the same type, if any, as does `common_type_t<T2, T1>`. No diagnostic is required for a violation of this Note's rules.

- 6 For the `common_reference` trait applied to a parameter pack `T` of types, the member `type` shall be either declared or not present as follows:

- (6.1) — If `sizeof...(T)` is zero, there shall be no member `type`.
- (6.2) — Otherwise, if `sizeof...(T)` is one, let `T0` denote the sole type in the pack `T`. The member *typedef* `type` shall denote the same type as `T0`.
- (6.3) — Otherwise, if `sizeof...(T)` is two, let `T1` and `T2` denote the two types in the pack `T`. Then
 - (6.3.1) — Let `R` be `COMMON-REF(T1, T2)`. If `T1` and `T2` are reference types, `R` is well-formed, and `is_convertible_v<add_pointer_t<T1>, add_pointer_t<R>> && is_convertible_v<add_pointer_t<T2>, add_pointer_t<R>>` is true, then the member *typedef* `type` denotes `R`.
 - (6.3.2) — Otherwise, if `basic_common_reference<remove_cvref_t<T1>, remove_cvref_t<T2>, XREF(T1), XREF(T2)>::type` is well-formed, then the member *typedef* `type` denotes that type.
 - (6.3.3) — Otherwise, if `COND-RES(T1, T2)` is well-formed, then the member *typedef* `type` denotes that type.
 - (6.3.4) — Otherwise, if `common_type_t<T1, T2>` is well-formed, then the member *typedef* `type` denotes that type.
 - (6.3.5) — Otherwise, there shall be no member `type`.
- (6.4) — Otherwise, if `sizeof...(T)` is greater than two, let `T1`, `T2`, and `Rest`, respectively, denote the first, second, and (pack of) remaining types comprising `T`. Let `C` be the type `common_reference_t<T1, T2>`. Then:
 - (6.4.1) — If there is such a type `C`, the member *typedef* `type` shall denote the same type, if any, as `common_reference_t<C, Rest...>`.
 - (6.4.2) — Otherwise, there shall be no member `type`.

- 7 Notwithstanding the provisions of 21.3.2, and pursuant to 16.4.5.2.1, a program may partially specialize `basic_common_reference<T, U, TQual, UQual>` for types `T` and `U` such that `is_same_v<T, decay_t<T>>` and `is_same_v<U, decay_t<U>>` are each true.

[*Note 6:* Such specializations can be used to influence the result of `common_reference`, and are needed when only explicit conversions are desired between the template arguments. — *end note*]

Such a specialization need not have a member named `type`, but if it does, the *qualified-id* `basic_common_reference<T, U, TQual, UQual>::type` shall denote a type to which each of the types `TQual<T>` and `UQual<U>` is convertible. Moreover, `basic_common_reference<T, U, TQual, UQual>::type` shall denote the same type, if any, as does `basic_common_reference<U, T, UQual, TQual>::type`. No diagnostic is required for a violation of these rules.

8 [*Example 2:* Given these declarations:

```
using PF1 = bool (& )();
using PF2 = short (*) (long);

struct S {
    operator PF2() const;
    double operator()(char, int&);
    void fn(long) const;
    char data;
};

using PMF = void (S::*)(long) const;
using PMD = char S::*;
```

the following assertions will hold:

```
static_assert(is_same_v<invoke_result_t<S, int>, short>);
static_assert(is_same_v<invoke_result_t<S&, unsigned char, int&>, double>);
static_assert(is_same_v<invoke_result_t<PF1>, bool>);
static_assert(is_same_v<invoke_result_t<PMF, unique_ptr<S>, int>, void>);
static_assert(is_same_v<invoke_result_t<PMD, S>, char&&>);
static_assert(is_same_v<invoke_result_t<PMD, const S*>, const char&>);
```

— *end example*]

21.3.9 Logical operator traits

[**meta.logical**]

1 This subclause describes type traits for applying logical operators to other type traits.

```
template<class... B> struct conjunction : see below { };
```

2 The class template `conjunction` forms the logical conjunction of its template type arguments.

3 For a specialization `conjunction<B1, ..., BN>`, if there is a template type argument `Bi` for which `bool(Bi::value)` is `false`, then instantiating `conjunction<B1, ..., BN>::value` does not require the instantiation of `Bj::value` for $j > i$.

[*Note 1:* This is analogous to the short-circuiting behavior of the built-in operator `&&`. — *end note*]

4 Every template type argument for which `Bi::value` is instantiated shall be usable as a base class and shall have a member `value` which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

5 The specialization `conjunction<B1, ..., BN>` has a public and unambiguous base that is either

- (5.1) — the first type `Bi` in the list `true_type`, `B1, ..., BN` for which `bool(Bi::value)` is `false`, or
- (5.2) — if there is no such `Bi`, the last type in the list.

[*Note 2:* This means a specialization of `conjunction` does not necessarily inherit from either `true_type` or `false_type`. — *end note*]

6 The member names of the base class, other than `conjunction` and `operator=`, shall not be hidden and shall be unambiguously available in `conjunction`.

```
template<class... B> struct disjunction : see below { };
```

7 The class template `disjunction` forms the logical disjunction of its template type arguments.

8 For a specialization `disjunction<B1, ..., BN>`, if there is a template type argument `Bi` for which `bool(Bi::value)` is `true`, then instantiating `disjunction<B1, ..., BN>::value` does not require the instantiation of `Bj::value` for $j > i$.

[*Note 3:* This is analogous to the short-circuiting behavior of the built-in operator `||`. — *end note*]

9 Every template type argument for which `Bi::value` is instantiated shall be usable as a base class and shall have a member `value` which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

- 10 The specialization `disjunction<B1, ..., BN>` has a public and unambiguous base that is either
- (10.1) — the first type `Bi` in the list `false_type`, `B1`, ..., `BN` for which `bool(Bi::value)` is `true`, or
 - (10.2) — if there is no such `Bi`, the last type in the list.

[*Note 4*: This means a specialization of `disjunction` does not necessarily inherit from either `true_type` or `false_type`. — *end note*]

11 The member names of the base class, other than `disjunction` and `operator=`, shall not be hidden and shall be unambiguously available in `disjunction`.

```
template<class B> struct negation : see below { };
```

12 The class template `negation` forms the logical negation of its template type argument. The type `negation<B>` is a *Cpp17UnaryTypeTrait* with a base characteristic of `bool_constant<!bool(B::value)>`.

21.3.10 Member relationships

[**meta.member**]

```
template<class S, class M>
constexpr bool is_pointer_interconvertible_with_class(M S::*m) noexcept;
```

1 *Mandates*: `S` is a complete type.

2 *Returns*: `true` if and only if `S` is a standard-layout type, `M` is an object type, `m` is not null, and each object `s` of type `S` is pointer-interconvertible (6.9.5) with its subobject `s.*m`.

```
template<class S1, class S2, class M1, class M2>
constexpr bool is_corresponding_member(M1 S1::*m1, M2 S2::*m2) noexcept;
```

3 *Mandates*: `S1` and `S2` are complete types.

4 *Returns*: `true` if and only if `S1` and `S2` are standard-layout struct (11.2) types, `M1` and `M2` are object types, `m1` and `m2` are not null, and `m1` and `m2` point to corresponding members of the common initial sequence (11.4) of `S1` and `S2`.

5 [*Note 1*: The type of a pointer-to-member expression `&C::b` is not always a pointer to member of `C`, leading to potentially surprising results when using these functions in conjunction with inheritance.

[*Example 1*:

```
struct A { int a; };           // a standard-layout class
struct B { int b; };           // a standard-layout class
struct C: public A, public B { }; // not a standard-layout class

static_assert( is_pointer_interconvertible_with_class( &C::b ) );
// Succeeds because, despite its appearance, &C::b has type
// "pointer to member of B of type int".
static_assert( !is_pointer_interconvertible_with_class<C, int>( &C::b ) );
// Forces the use of class C, and the result is false.

static_assert( is_corresponding_member( &C::a, &C::b ) );
// Succeeds because, despite its appearance, &C::a and &C::b have types
// "pointer to member of A of type int" and
// "pointer to member of B of type int", respectively.
static_assert( !is_corresponding_member<C, C, int>( &C::a, &C::b ) );
// Forces the use of class C, and the result is false.
```

— *end example*]

— *end note*]

21.3.11 Constant evaluation context

[**meta.const.eval**]

```
constexpr bool is_constant_evaluated() noexcept;
```

1 *Effects*: Equivalent to:

```

    if consteval {
        return true;
    } else {
        return false;
    }

```

2 [Example 1:

```

    constexpr void f(unsigned char *p, int n) {
        if (std::is_constant_evaluated()) {           // should not be a constexpr if statement
            for (int k = 0; k < n; ++k) p[k] = 0;
        } else {
            memset(p, 0, n);                          // not a core constant expression
        }
    }

```

— end example]

```

template<class U = void, class T>
constexpr bool is_within_lifetime(const T* p) noexcept;

```

3 *Mandates:* `static_cast<const volatile U*>(p)` is well-formed.

4 *Returns:* true if `p` is a pointer to an object that is within its lifetime (6.8.4) and `static_cast<const volatile U*>(p)` is a constant subexpression; otherwise, false.

5 *Remarks:* During the evaluation of an expression `E` as a core constant expression, a call to this function is ill-formed unless `p` points to an object that is usable in constant expressions or whose complete object's lifetime began within `E`.

6 [Example 2:

```

    struct OptBool {
        union { bool b; char c; };

        // note: this assumes common implementation properties for bool and char:
        // * sizeof(bool) == sizeof(char), and
        // * the value representations for true and false are distinct
        // from the value representation for 2
        constexpr OptBool() : c(2) { }
        constexpr OptBool(bool b) : b(b) { }

        constexpr auto has_value() const -> bool {
            if consteval {
                return std::is_within_lifetime(&b);    // during constant evaluation, cannot read from c
            } else {
                return c != 2;                          // during runtime, must read from c
            }
        }

        constexpr auto operator*() const -> const bool& {
            return b;
        }
    };

    constexpr OptBool disengaged;
    constexpr OptBool engaged(true);
    static_assert(!disengaged.has_value());
    static_assert(engaged.has_value());
    static_assert(*engaged);

```

— end example]

21.4 Reflection

[meta.reflection]

21.4.1 Header <meta> synopsis

[meta.syn]

```

#include <compare>           // see 17.12.1
#include <initializer_list>  // see 17.11.2

```

```

namespace std {
    // 21.4.2, checking string literals
    consteval bool is_string_literal(const char* p);
    consteval bool is_string_literal(const wchar_t* p);
    consteval bool is_string_literal(const char8_t* p);
    consteval bool is_string_literal(const char16_t* p);
    consteval bool is_string_literal(const char32_t* p);

    // 21.4.3, promoting to static storage
    namespace meta {
        template<ranges::input_range R>
            consteval info reflect_constant_string(R&& r);
        template<ranges::input_range R>
            consteval info reflect_constant_array(R&& r);
    }
    template<ranges::input_range R>
        consteval const ranges::range_value_t<R>* define_static_string(R&& r);
    template<ranges::input_range R>
        consteval span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
    template<class T>
        consteval const remove_cvref_t<T>* define_static_object(T&& r);
}

namespace std::meta {
    using info = decltype(^::);

    // 21.4.4, class exception
    class exception;

    // 21.4.5, operator representations
    enum class operators {
        see below;
    };
    using enum operators;
    consteval operators operator_of(info r);
    consteval string_view symbol_of(operators op);
    consteval u8string_view u8symbol_of(operators op);

    // 21.4.6, reflection names and locations
    consteval bool has_identifier(info r);

    consteval string_view identifier_of(info r);
    consteval u8string_view u8identifier_of(info r);

    consteval string_view display_string_of(info r);
    consteval u8string_view u8display_string_of(info r);

    consteval source_location source_location_of(info r);

    // 21.4.7, reflection queries
    consteval info type_of(info r);
    consteval info object_of(info r);
    consteval info constant_of(info r);

    consteval bool is_public(info r);
    consteval bool is_protected(info r);
    consteval bool is_private(info r);

    consteval bool is_virtual(info r);
    consteval bool is_pure_virtual(info r);
    consteval bool is_override(info r);
    consteval bool is_final(info r);

```



```

consteval bool is_deleted(info r);
consteval bool is_defaulted(info r);
consteval bool is_user_provided(info r);
consteval bool is_user_declared(info r);
consteval bool is_explicit(info r);
consteval bool is_noexcept(info r);

consteval bool is_bit_field(info r);
consteval bool is_enumerator(info r);
consteval bool is_annotation(info r);

consteval bool is_const(info r);
consteval bool is_volatile(info r);
consteval bool is_mutable_member(info r);
consteval bool is_lvalue_reference_qualified(info r);
consteval bool is_rvalue_reference_qualified(info r);

consteval bool has_static_storage_duration(info r);
consteval bool has_thread_storage_duration(info r);
consteval bool has_automatic_storage_duration(info r);

consteval bool has_internal_linkage(info r);
consteval bool has_module_linkage(info r);
consteval bool has_external_linkage(info r);
consteval bool has_c_language_linkage(info r);
consteval bool has_linkage(info r);

consteval bool is_complete_type(info r);
consteval bool is_enumerable_type(info r);

consteval bool is_variable(info r);
consteval bool is_type(info r);
consteval bool is_namespace(info r);
consteval bool is_type_alias(info r);
consteval bool is_namespace_alias(info r);

consteval bool is_function(info r);
consteval bool is_conversion_function(info r);
consteval bool is_operator_function(info r);
consteval bool is_literal_operator(info r);
consteval bool is_special_member_function(info r);
consteval bool is_constructor(info r);
consteval bool is_default_constructor(info r);
consteval bool is_copy_constructor(info r);
consteval bool is_move_constructor(info r);
consteval bool is_assignment(info r);
consteval bool is_copy_assignment(info r);
consteval bool is_move_assignment(info r);
consteval bool is_destructor(info r);

consteval bool is_function_parameter(info r);
consteval bool is_explicit_object_parameter(info r);
consteval bool has_default_argument(info r);
consteval bool is_vararg_function(info r);

consteval bool is_template(info r);
consteval bool is_function_template(info r);
consteval bool is_variable_template(info r);
consteval bool is_class_template(info r);
consteval bool is_alias_template(info r);
consteval bool is_conversion_function_template(info r);
consteval bool is_operator_function_template(info r);
consteval bool is_literal_operator_template(info r);
consteval bool is_constructor_template(info r);

```

```

constexpr bool is_concept(info r);

constexpr bool is_value(info r);
constexpr bool is_object(info r);

constexpr bool is_structured_binding(info r);

constexpr bool is_class_member(info r);
constexpr bool is_namespace_member(info r);
constexpr bool is_nonstatic_data_member(info r);
constexpr bool is_static_member(info r);
constexpr bool is_base(info r);

constexpr bool has_default_member_initializer(info r);

constexpr bool has_parent(info r);
constexpr info parent_of(info r);

constexpr info dealias(info r);

constexpr bool has_template_arguments(info r);
constexpr info template_of(info r);
constexpr vector<info> template_arguments_of(info r);
constexpr vector<info> parameters_of(info r);
constexpr info variable_of(info r);
constexpr info return_type_of(info r);

// 21.4.9, access control context
struct access_context;

// 21.4.10, member accessibility queries
constexpr bool is_accessible(info r, access_context ctx);
constexpr bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
constexpr bool has_inaccessible_bases(info r, access_context ctx);
constexpr bool has_inaccessible_subobjects(info r, access_context ctx);

// 21.4.8, scope identification
constexpr info current_function();
constexpr info current_class();
constexpr info current_namespace();

// 21.4.11, reflection member queries
constexpr vector<info> members_of(info r, access_context ctx);
constexpr vector<info> bases_of(info type, access_context ctx);
constexpr vector<info> static_data_members_of(info type, access_context ctx);
constexpr vector<info> nonstatic_data_members_of(info type, access_context ctx);
constexpr vector<info> subobjects_of(info type, access_context ctx);
constexpr vector<info> enumerators_of(info type_enum);

// 21.4.12, reflection layout queries
struct member_offset;
constexpr member_offset offset_of(info r);
constexpr size_t size_of(info r);
constexpr size_t alignment_of(info r);
constexpr size_t bit_size_of(info r);

// 21.4.13, annotation reflection
constexpr vector<info> annotations_of(info item);
constexpr vector<info> annotations_of_with_type(info item, info type);

// 21.4.14, value extraction
template<class T>
constexpr T extract(info r);

```

```

// 21.4.15, reflection substitution
template<class R>
    concept reflection_range = see below;

template<reflection_range R = initializer_list<info>>
    consteval bool can_substitute(info templ, R&& arguments);
template<reflection_range R = initializer_list<info>>
    consteval info substitute(info templ, R&& arguments);

// 21.4.16, expression result reflection
template<class T>
    consteval info reflect_constant(T expr);
template<class T>
    consteval info reflect_object(T& expr);
template<class T>
    consteval info reflect_function(T& fn);

// 21.4.17, class definition generation
struct data_member_options;
consteval info data_member_spec(info type, data_member_options options);
consteval bool is_data_member_spec(info r);
template<reflection_range R = initializer_list<info>>
    consteval info define_aggregate(info type_class, R&& mdescrs);

// associated with 21.3.5.2, primary type categories
consteval bool is_void_type(info type);
consteval bool is_null_pointer_type(info type);
consteval bool is_integral_type(info type);
consteval bool is_floating_point_type(info type);
consteval bool is_array_type(info type);
consteval bool is_pointer_type(info type);
consteval bool is_lvalue_reference_type(info type);
consteval bool is_rvalue_reference_type(info type);
consteval bool is_member_object_pointer_type(info type);
consteval bool is_member_function_pointer_type(info type);
consteval bool is_enum_type(info type);
consteval bool is_union_type(info type);
consteval bool is_class_type(info type);
consteval bool is_function_type(info type);
consteval bool is_reflection_type(info type);

// associated with 21.3.5.3, composite type categories
consteval bool is_reference_type(info type);
consteval bool is_arithmetic_type(info type);
consteval bool is_fundamental_type(info type);
consteval bool is_object_type(info type);
consteval bool is_scalar_type(info type);
consteval bool is_compound_type(info type);
consteval bool is_member_pointer_type(info type);

// associated with 21.3.5.4, type properties
consteval bool is_const_type(info type);
consteval bool is_volatile_type(info type);
consteval bool is_trivially_copyable_type(info type);
consteval bool is_standard_layout_type(info type);
consteval bool is_empty_type(info type);
consteval bool is_polymorphic_type(info type);
consteval bool is_abstract_type(info type);
consteval bool is_final_type(info type);
consteval bool is_aggregate_type(info type);
consteval bool is_structural_type(info type);
consteval bool is_signed_type(info type);
consteval bool is_unsigned_type(info type);
consteval bool is_bounded_array_type(info type);

```

```

constexpr bool is_unbounded_array_type(info type);
constexpr bool is_scoped_enum_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_constructible_type(info type, R&& type_args);
constexpr bool is_default_constructible_type(info type);
constexpr bool is_copy_constructible_type(info type);
constexpr bool is_move_constructible_type(info type);

constexpr bool is_assignable_type(info type_dst, info type_src);
constexpr bool is_copy_assignable_type(info type);
constexpr bool is_move_assignable_type(info type);

constexpr bool is_swappable_with_type(info type1, info type2);
constexpr bool is_swappable_type(info type);

constexpr bool is_destructible_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_trivially_constructible_type(info type, R&& type_args);
constexpr bool is_trivially_default_constructible_type(info type);
constexpr bool is_trivially_copy_constructible_type(info type);
constexpr bool is_trivially_move_constructible_type(info type);

constexpr bool is_trivially_assignable_type(info type_dst, info type_src);
constexpr bool is_trivially_copy_assignable_type(info type);
constexpr bool is_trivially_move_assignable_type(info type);
constexpr bool is_trivially_destructible_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_constructible_type(info type, R&& type_args);
constexpr bool is_nothrow_default_constructible_type(info type);
constexpr bool is_nothrow_copy_constructible_type(info type);
constexpr bool is_nothrow_move_constructible_type(info type);

constexpr bool is_nothrow_assignable_type(info type_dst, info type_src);
constexpr bool is_nothrow_copy_assignable_type(info type);
constexpr bool is_nothrow_move_assignable_type(info type);

constexpr bool is_nothrow_swappable_with_type(info type1, info type2);
constexpr bool is_nothrow_swappable_type(info type);

constexpr bool is_nothrow_destructible_type(info type);

constexpr bool is_implicit_lifetime_type(info type);

constexpr bool has_virtual_destructor(info type);

constexpr bool has_unique_object_representations(info type);

constexpr bool reference_constructs_from_temporary(info type_dst, info type_src);
constexpr bool reference_converts_from_temporary(info type_dst, info type_src);

// associated with 21.3.6, type property queries
constexpr size_t rank(info type);
constexpr size_t extent(info type, unsigned i = 0);

// associated with 21.3.7, type relations
constexpr bool is_same_type(info type1, info type2);
constexpr bool is_base_of_type(info type_base, info type_derived);
constexpr bool is_virtual_base_of_type(info type_base, info type_derived);
constexpr bool is_convertible_type(info type_src, info type_dst);
constexpr bool is_nothrow_convertible_type(info type_src, info type_dst);
constexpr bool is_layout_compatible_type(info type1, info type2);

```

```

constexpr bool is_pointer_interconvertible_base_of_type(info type_base, info type_derived);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>
    constexpr bool is_invocable_r_type(info type_result, info type, R&& type_args);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_invocable_r_type(info type_result, info type, R&& type_args);

// associated with 21.3.8.2, const-volatile modifications
constexpr info remove_const(info type);
constexpr info remove_volatile(info type);
constexpr info remove_cv(info type);
constexpr info add_const(info type);
constexpr info add_volatile(info type);
constexpr info add_cv(info type);

// associated with 21.3.8.3, reference modifications
constexpr info remove_reference(info type);
constexpr info add_lvalue_reference(info type);
constexpr info add_rvalue_reference(info type);

// associated with 21.3.8.4, sign modifications
constexpr info make_signed(info type);
constexpr info make_unsigned(info type);

// associated with 21.3.8.5, array modifications
constexpr info remove_extent(info type);
constexpr info remove_all_extents(info type);

// associated with 21.3.8.6, pointer modifications
constexpr info remove_pointer(info type);
constexpr info add_pointer(info type);

// associated with 21.3.8.7, other transformations
constexpr info remove_cvref(info type);
constexpr info decay(info type);
template<reflection_range R = initializer_list<info>>
    constexpr info common_type(R&& type_args);
template<reflection_range R = initializer_list<info>>
    constexpr info common_reference(R&& type_args);
constexpr info underlying_type(info type);
template<reflection_range R = initializer_list<info>>
    constexpr info invoke_result(info type, R&& type_args);
constexpr info unwrap_reference(info type);
constexpr info unwrap_ref_decay(info type);

constexpr size_t tuple_size(info type);
constexpr info tuple_element(size_t index, info type);
constexpr bool is_applicable_type(info fn, info tuple);
constexpr bool is_nothrow_applicable_type(info fn, info tuple);
constexpr info apply_result(info fn, info tuple);

constexpr size_t variant_size(info type);
constexpr info variant_alternative(size_t index, info type);

constexpr strong_ordering type_order(info type_a, info type_b);
}

```

¹ Unless otherwise specified, each function, and each specialization of any function template, specified in this header is a designated addressable function (16.4.5.2.1).

- ² When a function or function template specialization F specified in this header throws a `meta::exception` E , $E.from()$ is a reflection representing F and $E.where()$ is a `source_location` representing from where the call to F originated.
- ³ The behavior of any function specified in namespace `std::meta` is implementation-defined when a reflection of a construct not otherwise specified by this document is provided as an argument.

[*Note 1*: Values of type `std::meta::info` can represent implementation-specific constructs (6.9.3). — *end note*]

[*Note 2*: Many of the functions specified in namespace `std::meta` have semantics that can be affected by the completeness of class types represented by reflection values. For such functions, for any reflection r such that `dealias(r)` represents a specialization of a templated class with a reachable definition, the specialization is implicitly instantiated (13.9.2).

[*Example 1*:

```
template<class T>
struct X {
    T mem;
};
```

```
static_assert(sizeof(^X<int>) == sizeof(int));    // instantiates X<int>
```

— *end example*]

— *end note*]

- ⁴ Any function in namespace `std::meta` whose return type is `string_view` or `u8string_view` returns an object V such that $V.data()[V.size()]$ equals `'\0'`. Each element of the range $V.data() + [0, V.size())$ is a potentially non-unique object with static storage duration that is usable in constant expressions (6.8.2, 7.7); a pointer to such an element is not suitable for use as a template argument for a constant template parameter of pointer type (13.4.3).

[*Example 2*:

```
struct C { };

constexpr string_view sv = identifier_of(^C);
static_assert(sv == "C");
static_assert(sv.data()[0] == 'C');
static_assert(sv.data()[1] == '\0');
```

— *end example*]

- ⁵ For the purpose of exposition, throughout this clause E is used to indicate a reflection representing source construct E .

21.4.2 Checking string literals

[`meta.string.literal`]

```
constexpr bool is_string_literal(const char* p);
constexpr bool is_string_literal(const wchar_t* p);
constexpr bool is_string_literal(const char8_t* p);
constexpr bool is_string_literal(const char16_t* p);
constexpr bool is_string_literal(const char32_t* p);
```

¹ *Returns:*

- (1.1) — If p points to an unspecified object (7.7.2), `false`.
- (1.2) — Otherwise, if p points to a subobject of a string literal object (5.13.5), `true`.
- (1.3) — Otherwise, `false`.

21.4.3 Promoting to static storage

[`meta.define.static`]

- ¹ The functions in this subclause promote compile-time storage into static storage.

```
template<ranges::input_range R>
constexpr info reflect_constant_string(R&& r);
```

² Let `CharT` be `ranges::range_value_t<R>`.

³ *Mandates:* `CharT` is a character type (6.9.3).

4 Let V be the pack of values of type `CharT` whose elements are the corresponding elements of r , except that if r is a reference to a string literal object, then V does not include the terminating `U+0000 NULL` character of r .

5 Let P be the template parameter object (13.2) of type `const CharT[sizeof...(V) + 1]` initialized with $\{V\dots, \text{CharT}()\}$.

6 *Returns:* $\sim P$.

7 [Note 1: P is a potentially non-unique object (6.8.2). — end note]

```
template<ranges::input_range R>
constexpr info reflect_constant_array(R&& r);
```

8 Let U be `ranges::range_value_t<R>` and T be `remove_all_extents_t<U>`.

9 *Mandates:*

(9.1) — T is a structural type (13.2),

(9.2) — T satisfies `copy_constructible`, and

(9.3) — if U is not an array type, then `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`.

10 Let V be the pack of values of type `info` of the same size as r , where the i^{th} element is

(10.1) — `reflect_constant_array(*iti)` if U is an array type,

(10.2) — `reflect_constant(static_cast<T>(*iti))` otherwise,

and it_i is an iterator to the i^{th} element of r .

11 Let P be

(11.1) — the template parameter object (13.2) of type `const T[sizeof...(V)]`, such that $P[I]$ is template-argument-equivalent (13.6) to the object represented by $V\dots[I]$ for all I in the range $[0, \text{sizeof}...(V))$ if `sizeof...(V) > 0` is `true`, otherwise

(11.2) — the template parameter object of type `const array<T, 0>` initialized with $\{\}$.

12 *Returns:* $\sim P$.

13 *Throws:* Any of

(13.1) — an exception thrown by any operation on r or on iterators and sentinels referring to r ,

(13.2) — an exception thrown by the evaluation of any argument of `reflect_constant` or by any evaluation of `reflect_constant_array`, or

(13.3) — `meta::exception` if any invocation of `reflect_constant` would exit via an exception.

14 [Note 2: When r is not empty, P is a potentially non-unique object (6.8.2). — end note]

```
template<ranges::input_range R>
constexpr const ranges::range_value_t<R>* define_static_string(R&& r);
```

15 *Effects:* Equivalent to:

```
return meta::extract<const ranges::range_value_t<R>*>(meta::reflect_constant_string(r));
```

```
template<ranges::input_range R>
constexpr span<const ranges::range_value_t<R>, see below> define_static_array(R&& r);
```

16 *Effects:* Equivalent to:

```
using T = ranges::range_value_t<R>;
meta::info array = meta::reflect_constant_array(r);
if (meta::is_array_type(meta::type_of(array))) {
    return span<const T, see below>(meta::extract<const T*>(array),
                                   meta::extent(meta::type_of(array)));
} else {
    return span<const T, see below>(static_cast<const T*>(nullptr), 0);
}
```

17 *Remarks:* If `ranges::size(r)` is a constant expression, the second template argument of the returned `span` type is `static_cast<size_t>(ranges::size(r))`; otherwise, it is `dynamic_extent`.

```
template<class T>
constexpr const remove_cvref_t<T>* define_static_object(T&& t);
```

18 *Effects:* Equivalent to:

```
using U = remove_cvref_t<T>;
if constexpr (meta::is_class_type(^U) || meta::is_union_type(^U)) {
    return addressof(meta::extract<const U&>(
        meta::reflect_constant(std::forward<T>(t))));
} else if constexpr (meta::is_array_type(^U)) {
    return addressof(meta::extract<const U&>(
        meta::reflect_constant_array(std::forward<T>(t))));
} else {
    return define_static_array(span(addressof(t), 1)).data();
}
```

19 [Note 3: For class types, `define_static_object` provides the address of the template parameter object (13.2) that is template-argument equivalent to `t`. — end note]

21.4.4 Class exception

[meta.reflection.exception]

```
namespace std::meta {
    class exception : public std::exception {
    private:
        optional<string> what_;           // exposition only
        u8string u8what_;                 // exposition only
        info from_;                       // exposition only
        source_location where_;           // exposition only

    public:
        constexpr exception(u8string_view what, info from,
                           source_location where = source_location::current()) noexcept;

        constexpr exception(string_view what, info from,
                           source_location where = source_location::current()) noexcept;

        exception(const exception&) = default;
        exception(exception&&) = default;

        exception& operator=(const exception&) = default;
        exception& operator=(exception&&) = default;

        constexpr const char* what() const noexcept override;
        constexpr u8string_view u8what() const noexcept;
        constexpr info from() const noexcept;
        constexpr source_location where() const noexcept;
    };
}
```

1 Reflection functions throw exceptions of type `meta::exception` to signal an error.

```
constexpr exception(u8string_view what, info from,
                   source_location where = source_location::current()) noexcept;
```

2 *Effects:* Initializes `u8what_` with `what`, `from_` with `from`, and `where_` with `where`. If `what` can be represented in the ordinary literal encoding, initializes `what_` with `what`, transcoded from UTF-8 to the ordinary literal encoding. Otherwise, `what_` is value-initialized.

```
constexpr exception(string_view what, info from,
                   source_location where = source_location::current()) noexcept;
```

3 *Constant When:* `what` designates a sequence of characters that can be encoded in UTF-8.

4 *Effects:* Initializes `what_` with `what`, `u8what_` with `what` transcoded from the ordinary literal encoding to UTF-8, `from_` with `from` and `where_` with `where`.

```
constexpr const char* what() const noexcept override;
```

5 *Constant When:* `what_.has_value()` is true.

6 *Returns:* `what_->c_str()`.

```
constexpr u8string_view u8what() const noexcept;
```

7 *Returns:* `u8what_`.

```
constexpr info from() const noexcept;
```

8 *Returns:* `from_`.

```
constexpr source_location where() const noexcept;
```

9 *Returns:* `where_`.

21.4.5 Operator representations

[meta.reflection.operators]

```
enum class operators {  
    see below;  
};  
using enum operators;
```

1 The enumeration type `operators` specifies constants used to identify operators that can be overloaded, with the meanings listed in Table 63. The values of the constants are distinct.

```
constexpr operators operator_of(info r);
```

2 *Returns:* The value of the enumerator from `operators` whose corresponding *operator-function-id* is the unqualified name of the entity represented by `r`.

3 *Throws:* `meta::exception` unless `r` represents an operator function or operator function template.

```
constexpr string_view symbol_of(operators op);  
constexpr u8string_view u8symbol_of(operators op);
```

4 *Returns:* A `string_view` or `u8string_view` containing the characters of the operator symbol name corresponding to `op`, respectively encoded with the ordinary literal encoding or with UTF-8.

5 *Throws:* `meta::exception` unless the value of `op` corresponds to one of the enumerators in `operators`.

21.4.6 Reflection names and locations

[meta.reflection.names]

```
constexpr bool has_identifier(info r);
```

1 *Returns:*

- (1.1) — If `r` represents an entity that has a typedef name for linkage purposes (9.2.4), then **true**.
- (1.2) — Otherwise, if `r` represents an unnamed entity, then **false**.
- (1.3) — Otherwise, if `r` represents a type alias, then `!has_template_arguments(r)`.
- (1.4) — Otherwise, if `r` represents a type, then **true** if
 - (1.4.1) — `r` represents a cv-unqualified class type and `has_template_arguments(r)` is **false**, or
 - (1.4.2) — `r` represents a cv-unqualified enumeration type.
 Otherwise, **false**.
- (1.5) — Otherwise, if `r` represents a function, then **true** if `has_template_arguments(r)` is **false** and the function is not a constructor, destructor, operator function, or conversion function. Otherwise, **false**.
- (1.6) — Otherwise, if `r` represents a template, then **true** if `r` does not represent a constructor template, operator function template, or conversion function template. Otherwise, **false**.
- (1.7) — Otherwise, if `r` represents the i^{th} parameter of a function F that is an (implicit or explicit) specialization of a templated function T and the i^{th} parameter of the instantiated declaration of T whose template arguments are those of F would be instantiated from a pack, then **false**.
- (1.8) — Otherwise, if `r` represents the parameter P of a function F , then let S be the set of declarations, ignoring any explicit instantiations, that are reachable from a point in the evaluation context and that declare either F or a templated function of which F is a specialization; **true** if

Table 63 — Enum class operators [tab:meta.reflection.operators]

Constant	Corresponding <i>operator-function-id</i>	Operator symbol name
op_new	operator new	new
op_delete	operator delete	delete
op_array_new	operator new[]	new[]
op_array_delete	operator delete[]	delete[]
op_co_await	operator co_await	co_await
op_parentheses	operator()	()
op_square_brackets	operator[]	[]
op_arrow	operator->	->
op_arrow_star	operator->*	->*
op_tilde	operator~	~
op_exclamation	operator!	!
op_plus	operator+	+
op_minus	operator-	-
op_star	operator*	*
op_slash	operator/	/
op_percent	operator%	%
op_caret	operator^	^
op_ampersand	operator&	&
op_equals	operator=	=
op_pipe	operator	
op_plus_equals	operator+=	+=
op_minus_equals	operator-=	-=
op_star_equals	operator*=	*=
op_slash_equals	operator/=	/=
op_percent_equals	operator%=	%=
op_caret_equals	operator^=	^=
op_ampersand_equals	operator&=	&=
op_pipe_equals	operator =	=
op_equals_equals	operator==	==
op_exclamation_equals	operator!=	!=
op_less	operator<	<
op_greater	operator>	>
op_less_equals	operator<=	<=
op_greater_equals	operator>=	>=
op_spaceship	operator<=>	<=>
op_ampersand_ampersand	operator&&	&&
op_pipe_pipe	operator	
op_less_less	operator<<	<<
op_greater_greater	operator>>	>>
op_less_less_equals	operator<<=	<<=
op_greater_greater_equals	operator>>=	>>=
op_plus_plus	operator++	++
op_minus_minus	operator--	--
op_comma	operator,	,

- (1.8.1) — there is a declaration D in S that introduces a name N for either P or the parameter corresponding to P in the templated function that D declares and
- (1.8.2) — no declaration in S does so using any name other than N .

Otherwise, **false**.

[*Example 1:*

```
void fun(int);
constexpr std::meta::info r = parameters_of(^fun)[0];
static_assert(!has_identifier(r));

void fun(int x);
static_assert(has_identifier(r));

void fun(int x);
static_assert(has_identifier(r));

void poison() {
    void fun(int y);
}
static_assert(!has_identifier(r));
```

— *end example*]

- (1.9) — Otherwise, if r represents a variable, then **false** if the declaration of that variable was instantiated from a function parameter pack. Otherwise, **!has_template_arguments(r)**.
- (1.10) — Otherwise, if r represents a structured binding, then **false** if the declaration of that structured binding was instantiated from a structured binding pack. Otherwise, **true**.
- (1.11) — Otherwise, if r represents an enumerator, non-static-data member, namespace, or namespace alias, then **true**.
- (1.12) — Otherwise, if r represents a direct base class relationship, then **has_identifier(type_of(r))**.
- (1.13) — Otherwise, if r represents a data member description (T, N, A, W, NUA, ANN) (11.4.1); **true** if N is not $\perp$. Otherwise, **false**.
- (1.14) — Otherwise, **false**.

```
constexpr string_view identifier_of(info r);
constexpr u8string_view u8identifier_of(info r);
```

2 Let E be UTF-8 for **u8identifier_of**, and otherwise the ordinary literal encoding.

3 *Returns:* An NTMBS, encoded with E , determined as follows:

- (3.1) — If r represents an entity with a typedef name for linkage purposes, then that name.
- (3.2) — Otherwise, if r represents a literal operator or literal operator template, then the *ud-suffix* of the operator or operator template.
- (3.3) — Otherwise, if r represents the parameter P of a function F , then let S be the set of declarations, ignoring any explicit instantiations, that are reachable from a point in the evaluation context and that declare either F or a templated function of which F is a specialization; the name that was introduced by a declaration in S for the parameter corresponding to P .
- (3.4) — Otherwise, if r represents an entity, then the identifier introduced by the declaration of that entity.
- (3.5) — Otherwise, if r represents a direct base class relationship, then **identifier_of(type_of(r))** or **u8identifier_of(type_of(r))**, respectively.
- (3.6) — Otherwise, r represents a data member description (T, N, A, W, NUA, ANN) (11.4.1); a **string_view** or **u8string_view**, respectively, containing the identifier N .

4 *Throws:* **meta::exception** unless **has_identifier(r)** is **true** and the identifier that would be returned (see above) is representable by E .

```
constexpr string_view display_string_of(info r);
constexpr u8string_view u8display_string_of(info r);
```

5 *Returns:* An implementation-defined **string_view** or **u8string_view**, respectively.

6 *Recommended practice:* Where possible, implementations should return a string suitable for identifying the represented construct.

```
constexpr source_location source_location_of(info r);
```

7 *Returns:* If **r** represents a value, a type other than a class type or an enumeration type, the global namespace, or a data member description, then `source_location{}`. Otherwise, an implementation-defined `source_location` value.

8 *Recommended practice:* If **r** represents an entity with a definition that is reachable from the evaluation context, a value corresponding to a definition should be returned.

21.4.7 Reflection queries

[meta.reflection.queries]

```
constexpr bool has_type(info r); // exposition only
```

1 *Returns:* `true` if **r** represents a value, annotation, object, variable, function whose type does not contain an undeduced placeholder type and that is not a constructor or destructor, enumerator, non-static data member, unnamed bit-field, direct base class relationship, data member description, or function parameter. Otherwise, `false`.

```
constexpr info type_of(info r);
```

2 *Returns:*

- (2.1) — If **r** represents the i^{th} parameter of a function F , then the i^{th} type in the parameter-type-list of F (9.3.4.6).
- (2.2) — Otherwise, if **r** represents a value, object, variable, function, non-static data member, or unnamed bit-field, then the type of what is represented by **r**.
- (2.3) — Otherwise, if **r** represents an annotation, then `type_of(constant_of(r))`.
- (2.4) — Otherwise, if **r** represents an enumerator N of an enumeration E , then:
 - (2.4.1) — If E is defined by a declaration D that is reachable from a point P in the evaluation context and P does not occur within an *enum-specifier* of D , then a reflection of E .
 - (2.4.2) — Otherwise, a reflection of the type of N prior to the closing brace of the *enum-specifier* as specified in 9.8.1.
- (2.5) — Otherwise, if **r** represents a direct base class relationship (D, B) , then a reflection of B .
- (2.6) — Otherwise, for a data member description (T, N, A, W, NUA, ANN) (11.4.1), a reflection of the type T .

3 *Throws:* `meta::exception` unless `has_type(r)` is `true`.

```
constexpr info object_of(info r);
```

4 *Returns:*

- (4.1) — If **r** represents an object, then **r**.
- (4.2) — Otherwise, if **r** represents a reference, then a reflection of the object referred to by that reference.
- (4.3) — Otherwise, **r** represents a variable; a reflection of the object declared by that variable.

5 *Throws:* `meta::exception` unless **r** is a reflection representing either

- (5.1) — an object with static storage duration (6.8.6.1), or
- (5.2) — a variable that either declares or refers to such an object, and if that variable is a reference R , then either
 - (5.2.1) — R is usable in constant expressions (7.7.4), or
 - (5.2.2) — the lifetime of R began within the core constant expression currently under evaluation.

6 *[Example 1:*

```
int x;
int& y = x;
```

```
static_assert(&x != &y);
```

// OK, **x** and **y** are different variables, so their

```

static_assert(object_of(^x) == object_of(^y)); // reflections compare different
// OK, because y is a reference
// to x, their underlying objects are the same
— end example]

```

```

constexpr info constant_of(info r);

```

7 Let R be a constant expression of type `info` such that $R == r$ is `true`. If r represents an annotation, then let C be its underlying constant.

8 *Effects*: Equivalent to:

```

if constexpr (is_annotation(R)) {
    return C;
} else if constexpr (is_array_type(type_of(R))) {
    return reflect_constant_array([: R :]);
} else if constexpr (is_function_type(type_of(R))) {
    return reflect_function([: R :]);
} else {
    return reflect_constant([: R :]);
}

```

9 *Throws*: `meta::exception` unless either r represents an annotation or `[: R :]` is a valid *splice-expression* (7.5.9).

10 [Example 2:

```

constexpr int x = 0;
constexpr int y = 0;

static_assert(^x != ^y); // OK, x and y are different variables,
// so their reflections compare different

static_assert(constant_of(^x) ==
              constant_of(^y)); // OK, both constant_of(^x) and
// constant_of(^y) represent the value 0

static_assert(constant_of(^x) ==
              reflect_constant(0)); // OK, likewise

struct S { int m; };
constexpr S s {42};

static_assert(is_object(constant_of(^s)) &&
              is_object(reflect_object(s)));

static_assert(constant_of(^s) !=
              reflect_object(s)); // OK, template parameter object that is
// template-argument-equivalent to s
// is a different object than s

static_assert(constant_of(^s) ==
              constant_of(reflect_object(s)));

constexpr info fn() {
    constexpr int x = 42;
    return ^x;
}
constexpr info r = constant_of(fn()); // error: x is outside its lifetime
— end example]

```

```

constexpr bool is_public(info r);
constexpr bool is_protected(info r);
constexpr bool is_private(info r);

```

11 *Returns*: `true` if r represents either

(11.1) — a class member or unnamed bit-field that is public, protected, or private, respectively, or

- (11.2) — a direct base class relationship (D, B) for which B is, respectively, a public, protected, or private base class of D .

Otherwise, **false**.

```
constexpr bool is_virtual(info r);
```

- 12 *Returns:* **true** if r represents either a virtual member function or a direct base class relationship (D, B) for which B is a virtual base class of D . Otherwise, **false**.

```
constexpr bool is_pure_virtual(info r);
```

```
constexpr bool is_override(info r);
```

- 13 *Returns:* **true** if r represents a member function that is pure virtual or overrides another member function, respectively. Otherwise, **false**.

```
constexpr bool is_final(info r);
```

- 14 *Returns:* **true** if r represents a final class or a final member function. Otherwise, **false**.

```
constexpr bool is_deleted(info r);
```

```
constexpr bool is_defaulted(info r);
```

- 15 *Returns:* **true** if r represents a function that is a deleted function (9.6.3) or defaulted function (9.6.2), respectively. Otherwise, **false**.

```
constexpr bool is_user_provided(info r);
```

```
constexpr bool is_user_declared(info r);
```

- 16 *Returns:* **true** if r represents a function that is user-provided or user-declared (9.6.2), respectively. Otherwise, **false**.

```
constexpr bool is_explicit(info r);
```

- 17 *Returns:* **true** if r represents a member function that is declared explicit. Otherwise, **false**.

[Note 1: If r represents a member function template that is declared explicit, `is_explicit(r)` is still **false** because in general, such queries for templates cannot be answered. — end note]

```
constexpr bool is_noexcept(info r);
```

- 18 *Returns:* **true** if r represents a **noexcept** function type or a function with a non-throwing exception specification (14.5). Otherwise, **false**.

[Note 2: If r represents a function template that is declared **noexcept**, `is_noexcept(r)` is still **false** because in general, such queries for templates cannot be answered. — end note]

```
constexpr bool is_bit_field(info r);
```

- 19 *Returns:* **true** if r represents a bit-field, or if r represents a data member description (T, N, A, W, NUA, ANN) (11.4.1) for which W is not $\perp$. Otherwise, **false**.

```
constexpr bool is_enumerator(info r);
```

```
constexpr bool is_annotation(info r);
```

- 20 *Returns:* **true** if r represents an enumerator or annotation, respectively. Otherwise, **false**.

```
constexpr bool is_const(info r);
```

```
constexpr bool is_volatile(info r);
```

- 21 Let T be `type_of(r)` if *has-type*(r) is **true**. Otherwise, let T be `dealias(r)`.

- 22 *Returns:* **true** if T represents a const or volatile type, respectively, or a const- or volatile-qualified function type, respectively. Otherwise, **false**.

```
constexpr bool is_mutable_member(info r);
```

- 23 *Returns:* **true** if r represents a mutable non-static data member. Otherwise, **false**.

```
constexpr bool is_lvalue_reference_qualified(info r);
```

```
constexpr bool is_rvalue_reference_qualified(info r);
```

- 24 Let T be `type_of(r)` if *has-type*(r) is **true**. Otherwise, let T be `dealias(r)`.

25 *Returns:* `true` if *T* represents an lvalue- or rvalue-qualified function type, respectively. Otherwise, `false`.

```
constexpr bool has_static_storage_duration(info r);
constexpr bool has_thread_storage_duration(info r);
constexpr bool has_automatic_storage_duration(info r);
```

26 *Returns:* `true` if *r* represents an object or variable that has static, thread, or automatic storage duration, respectively (6.8.6). Otherwise, `false`.

[*Note 3:* It is not possible to have a reflection representing an object or variable having dynamic storage duration. — *end note*]

```
constexpr bool has_internal_linkage(info r);
constexpr bool has_module_linkage(info r);
constexpr bool has_external_linkage(info r);
constexpr bool has_linkage(info r);
```

27 *Returns:* `true` if *r* represents a variable, function, type, template, or namespace whose name has internal linkage, module linkage, external linkage, or any linkage, respectively (6.7). Otherwise, `false`.

```
constexpr bool has_c_language_linkage(info r);
```

28 *Returns:* `true` if *r* represents a variable, function, or function type with C language linkage. Otherwise, `false`.

```
constexpr bool is_complete_type(info r);
```

29 *Returns:* `true` if `is_type(r)` is `true` and there is some point in the evaluation context from which the type represented by `dealias(r)` is not an incomplete type (6.9). Otherwise, `false`.

```
constexpr bool is_enumerable_type(info r);
```

30 A type *T* is *enumerable* from a point *P* if either

(30.1) — *T* is a class type complete at point *P* or

(30.2) — *T* is an enumeration type defined by a declaration *D* such that *D* is reachable from *P* but *P* does not occur within an *enum-specifier* of *D* (9.8.1).

31 *Returns:* `true` if `dealias(r)` represents a type that is enumerable from some point in the evaluation context. Otherwise, `false`.

32 [*Example 3:*

```
class S;
enum class E;
static_assert(!is_enumerable_type(^S));
static_assert(!is_enumerable_type(^E));

class S {
    void mfn() {
        static_assert(is_enumerable_type(^S));
    }
    static_assert(!is_enumerable_type(^S));
};
static_assert(is_enumerable_type(^S));

enum class E {
    A = is_enumerable_type(^E) ? 1 : 2
};
static_assert(is_enumerable_type(^E));
static_assert(static_cast<int>(E::A) == 2);
```

— *end example*]

```
constexpr bool is_variable(info r);
```

33 *Returns:* `true` if *r* represents a variable. Otherwise, `false`.

```
constexpr bool is_type(info r);
```

```
constexpr bool is_namespace(info r);
```

34 *Returns:* **true** if *r* represents an entity whose underlying entity is a type or namespace, respectively. Otherwise, **false**.

```
constexpr bool is_type_alias(info r);
constexpr bool is_namespace_alias(info r);
```

35 *Returns:* **true** if *r* represents a type alias or namespace alias, respectively. Otherwise, **false**.

[*Note 4:* A specialization of an alias template is a type alias. — *end note*]

```
constexpr bool is_function(info r);
```

36 *Returns:* **true** if *r* represents a function. Otherwise, **false**.

```
constexpr bool is_conversion_function(info r);
constexpr bool is_operator_function(info r);
constexpr bool is_literal_operator(info r);
```

37 *Returns:* **true** if *r* represents a function that is a conversion function (11.4.8.3), operator function (12.4), or literal operator (12.6), respectively. Otherwise, **false**.

```
constexpr bool is_special_member_function(info r);
constexpr bool is_constructor(info r);
constexpr bool is_default_constructor(info r);
constexpr bool is_copy_constructor(info r);
constexpr bool is_move_constructor(info r);
constexpr bool is_assignment(info r);
constexpr bool is_copy_assignment(info r);
constexpr bool is_move_assignment(info r);
constexpr bool is_destructor(info r);
```

38 *Returns:* **true** if *r* represents a function that is a special member function (11.4.4), a constructor, a default constructor, a copy constructor, a move constructor, an assignment operator, a copy assignment operator, a move assignment operator, or a destructor, respectively. Otherwise, **false**.

```
constexpr bool is_function_parameter(info r);
```

39 *Returns:* **true** if *r* represents a function parameter. Otherwise, **false**.

```
constexpr bool is_explicit_object_parameter(info r);
```

40 *Returns:* **true** if *r* represents a function parameter that is an explicit object parameter (9.3.4.6). Otherwise, **false**.

```
constexpr bool has_default_argument(info r);
```

41 *Returns:* If *r* represents a parameter *P* of a function *F*, then:

- (41.1) — If *F* is a specialization of a templated function *T*, then **true** if there exists a declaration *D* of *T* that is reachable from a point in the evaluation context and *D* specifies a default argument for the parameter of *T* corresponding to *P*. Otherwise, **false**.
- (41.2) — Otherwise, if there exists a declaration *D* of *F* that is reachable from a point in the evaluation context and *D* specifies a default argument for *P*, then **true**.

Otherwise, **false**.

```
constexpr bool is_vararg_function(info r);
```

42 *Returns:* **true** if *r* represents a function or function type that is a vararg function (9.3.4.6). Otherwise, **false**.

```
constexpr bool is_template(info r);
```

43 *Returns:* **true** if *r* represents a function template, class template, variable template, alias template, or concept. Otherwise, **false**.

44 [*Note 5:* A template specialization is not a template. For example, `is_template(decltype(std::vector))` is **true** but `is_template(decltype(std::vector<int>))` is **false**. — *end note*]


```

constexpr bool is_function_template(info r);
constexpr bool is_variable_template(info r);
constexpr bool is_class_template(info r);
constexpr bool is_alias_template(info r);
constexpr bool is_conversion_function_template(info r);
constexpr bool is_operator_function_template(info r);
constexpr bool is_literal_operator_template(info r);
constexpr bool is_constructor_template(info r);
constexpr bool is_concept(info r);

```

45 *Returns:* **true** if **r** represents a function template, variable template, class template, alias template, conversion function template, operator function template, literal operator template, constructor template, or concept, respectively. Otherwise, **false**.

```

constexpr bool is_value(info r);
constexpr bool is_object(info r);

```

46 *Returns:* **true** if **r** represents a value or object, respectively. Otherwise, **false**.

```

constexpr bool is_structured_binding(info r);

```

47 *Returns:* **true** if **r** represents a structured binding. Otherwise, **false**.

```

constexpr bool is_class_member(info r);
constexpr bool is_namespace_member(info r);
constexpr bool is_nonstatic_data_member(info r);
constexpr bool is_static_member(info r);
constexpr bool is_base(info r);

```

48 *Returns:* **true** if **r** represents a class member, namespace member, non-static data member, static member, or direct base class relationship, respectively. Otherwise, **false**.

```

constexpr bool has_default_member_initializer(info r);

```

49 *Returns:* **true** if **r** represents a non-static data member that has a default member initializer. Otherwise, **false**.

```

constexpr bool has_parent(info r);

```

50 *Returns:*

- (50.1) — If **r** represents the global namespace, then **false**.
- (50.2) — Otherwise, if **r** represents an entity that has C language linkage (9.12), then **false**.
- (50.3) — Otherwise, if **r** represents an entity that has a language linkage other than C++ language linkage, then an implementation-defined value.
- (50.4) — Otherwise, if **r** represents a type that is neither a class nor enumeration type, then **false**.
- (50.5) — Otherwise, if **r** represents an entity or direct base class relationship, then **true**.
- (50.6) — Otherwise, **false**.

```

constexpr info parent_of(info r);

```

51 *Returns:*

- (51.1) — If **r** represents a non-static data member that is a direct member of an anonymous union, or an unnamed bit-field declared within the *member-specification* of such a union, then a reflection representing the innermost enclosing anonymous union.
- (51.2) — Otherwise, if **r** represents an enumerator, then a reflection representing the corresponding enumeration type.
- (51.3) — Otherwise, if **r** represents a direct base class relationship (D, B), then a reflection representing D .
- (51.4) — Otherwise, let E be a class, function, or namespace whose class scope, function parameter scope, or namespace scope, respectively, is the innermost such scope that either is, or encloses, the target scope of a declaration of what is represented by **r**.
 - (51.4.1) — If E is the function call operator of a closure type for a *constexpr-block-declaration* (9.1), then `parent_of(parent_of(~~E))`.

[*Note 6:* In this case, the first `parent_of` will be the closure type, so the second `parent_of` is necessary to give the parent of that closure type. — *end note*]

(51.4.2) — Otherwise, `^^E`.

52 *Throws:* `meta::exception` unless `has_parent(r)` is `true`.

53 [*Example 4:*

```
struct I { };

struct F : I {
    union {
        int o;
    };

    enum N {
        A
    };
};

constexpr auto ctx = std::meta::access_context::current();

static_assert(parent_of(^F) == ^^:.);
static_assert(parent_of(bases_of(^F, ctx)[0]) == ^F);
static_assert(is_union_type(parent_of(^F::o)));
static_assert(parent_of(^F::N) == ^F);
static_assert(parent_of(^F::A) == ^F::N);
— end example]
```

```
constexpr info dealias(info r);
```

54 *Returns:* If `r` represents an entity, then a reflection representing the underlying entity of what `r` represents. Otherwise, `r`.

55 [*Example 5:*

```
using X = int;
using Y = X;
static_assert(dealias(^int) == ^^int);
static_assert(dealias(^X) == ^^int);
static_assert(dealias(^Y) == ^^int);
— end example]
```

```
constexpr bool has_template_arguments(info r);
```

56 *Returns:* `true` if `r` represents a specialization of a function template, variable template, class template, or an alias template. Otherwise, `false`.

```
constexpr info template_of(info r);
```

57 *Returns:* A reflection of the template of the specialization represented by `r`.

58 *Throws:* `meta::exception` unless `has_template_arguments(r)` is `true`.

```
constexpr vector<info> template_arguments_of(info r);
```

59 *Returns:* A `vector` containing reflections of the template arguments of the template specialization represented by `r`, in the order in which they appear in the corresponding template argument list. For a given template argument `A`, its corresponding reflection `R` is determined as follows:

- (59.1) — If `A` denotes a type or type alias, then `R` is a reflection representing the underlying entity of `A`.
[*Note 7:* `R` always represents a type, never a type alias. — *end note*]
- (59.2) — Otherwise, if `A` denotes a class template, variable template, concept, or alias template, then `R` is a reflection representing `A`.
- (59.3) — Otherwise, `A` is a constant template argument (13.4.3). Let `P` be the corresponding template parameter.

- (59.3.1) — If P has reference type, then R is a reflection representing the object or function referred to by A .
- (59.3.2) — Otherwise, if P has class type, then R represents the corresponding template parameter object.
- (59.3.3) — Otherwise, R is a reflection representing the value of A .

60 *Throws:* `meta::exception` unless `has_template_arguments(r)` is `true`.

61 [*Example 6:*

```
template<class T, class U = T> struct Pair { };
template<class T> struct Pair<char, T> { };
template<class T> using PairPtr = Pair<T*>;

static_assert(template_of(^Pair<int>) == ^Pair);
static_assert(template_of(^Pair<char, char>) == ^Pair);
static_assert(template_arguments_of(^Pair<int>).size() == 2);
static_assert(template_arguments_of(^Pair<int>)[0] == ^int);

static_assert(template_of(^PairPtr<int>) == ^PairPtr);
static_assert(template_arguments_of(^PairPtr<int>).size() == 1);

struct S { };
int i;
template<int, int&, S, template<class> class>
    struct X { };
constexpr auto T = ^X<1, i, S{}, PairPtr>;
static_assert(is_value(template_arguments_of(T)[0]));
static_assert(is_object(template_arguments_of(T)[1]));
static_assert(is_object(template_arguments_of(T)[2]));
static_assert(template_arguments_of(T)[3] == ^PairPtr);
```

— end example]

`constexpr vector<info> parameters_of(info r);`

62 *Returns:*

- (62.1) — If r represents a function F , then a `vector` containing reflections of the parameters of F , in the order in which they appear in a declaration of F .
- (62.2) — Otherwise, r represents a function type T ; a `vector` containing reflections of the types in parameter-type-list (9.3.4.6) of T , in the order in which they appear in the parameter-type-list.

63 *Throws:* `meta::exception` unless r represents a function or a function type.

`constexpr info variable_of(info r);`

64 *Returns:* The reflection of the parameter variable corresponding to r .

65 *Throws:* `meta::exception` unless

- (65.1) — r represents a parameter of a function F and
- (65.2) — there is a point P in the evaluation context for which the innermost non-block scope enclosing P is the function parameter scope (6.4.4) associated with F .

`constexpr info return_type_of(info r);`

66 *Returns:* The reflection of the return type of the function or function type represented by r .

67 *Throws:* `meta::exception` unless either r represents a function and *has-type* (r) is `true` or r represents a function type.

21.4.8 Scope identification

[`meta.reflection.scope`]

- 1 The functions in this subclause retrieve information about where in the program they are invoked.
 - 2 None of the functions in this subclause is an addressable function (16.4.5.2.1).
 - 3 Given a program point P , let *eval-point* (P) be the following program point:
- (3.1) — If a potentially-evaluated subexpression (6.10.1) of a default member initializer I for a member of class C (11.4.1) appears at P , then a point determined as follows:

- (3.1.1) — If an aggregate initialization is using I , *eval-point*(Q), where Q is the point at which that aggregate initialization appears.
 - (3.1.2) — Otherwise, if an initialization by an inherited constructor (11.9.4) is using I , a point whose immediate scope is the class scope corresponding to C .
 - (3.1.3) — Otherwise, a point whose immediate scope is the function parameter scope corresponding to the constructor definition that is using I .
 - (3.2) — Otherwise, if a potentially-evaluated subexpression of a default argument (9.3.4.7) appears at P , *eval-point*(Q), where Q is the point at which the invocation of the function (7.6.1.3) using that default argument appears.
 - (3.3) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a declaration D , and P appears either before the locus of D or within the trailing *requires-clause* of D , a point whose immediate scope is the innermost scope enclosing the locus of D that is not a template parameter scope.
 - (3.4) — Otherwise, if the immediate scope of P is a function parameter scope introduced by a *lambda-expression* L whose *lambda-introducer* appears at point Q , and P appears either within the *trailing-return-type* or the trailing *requires-clause* of L , *eval-point*(Q).
 - (3.5) — Otherwise, if the innermost non-block scope enclosing P is the function parameter scope introduced by a *consteval-block-declaration* (9.1), a point whose immediate scope is that inhabited by the outermost *consteval-block-declaration* D containing P such that each scope (if any) that intervenes between P and the function parameter scope introduced by D is either
 - (3.5.1) — a block scope or
 - (3.5.2) — a function parameter scope or lambda scope introduced by a *consteval-block-declaration*.
 - (3.6) — Otherwise, P .
- 4 Given a scope S , let *ctx-scope*(S) be the following scope:
- (4.1) — If S is a class scope or namespace scope, S .
 - (4.2) — Otherwise, if S is a function parameter scope introduced by the declaration of a function, S .
 - (4.3) — Otherwise, if S is a lambda scope introduced by a *lambda-expression* L , the function parameter scope corresponding to the call operator of the closure type of L .
 - (4.4) — Otherwise, *ctx-scope*(S'), where S' is the parent scope of S .
- 5 Let *CURRENT-SCOPE*(P) for a point P be a reflection representing the function, class, or namespace whose corresponding function parameter scope, class scope, or namespace scope, respectively, is *ctx-scope*(S), where S is the immediate scope of *eval-point*(P).

```
consteval info current_function();
```

- 6 An invocation of `current_function` that appears at a program point P is value-dependent (13.8.3.4) if *eval-point*(P) is enclosed by a scope corresponding to a templated entity.

7 Let S be *CURRENT-SCOPE*(P), where P is the point at which the invocation of `current_function` lexically appears.

8 *Returns:* S .

9 *Throws:* `meta::exception` unless S represents a function.

```
consteval info current_class();
```

- 10 An invocation of `current_class` that appears at a program point P is value-dependent (13.8.3.4) if *eval-point*(P) is enclosed by a scope corresponding to a templated entity.

11 Let S be *CURRENT-SCOPE*(P) where P is the point at which the invocation of `current_class` lexically appears.

12 *Returns:* S if S represents a class. Otherwise, *parent_of*(S).

13 *Throws:* `meta::exception` unless S represents either a class or a member function.

```
constexpr info current_namespace();
```

14 An invocation of `current_namespace` that appears at a program point P is value-dependent (13.8.3.4) if *eval-point* (P) is enclosed by a scope corresponding to a templated entity.

15 Let S be *CURRENT-SCOPE* (P) where P is the point at which the invocation of `current_namespace` lexically appears.

16 *Returns:* S if S represents a namespace. Otherwise, a reflection representing the nearest enclosing namespace of the entity represented by S .

21.4.9 Access control context

[meta.reflection.access.context]

1 The class `access_context` represents a namespace, class, or function from which queries pertaining to access rules may be performed, as well as the designating class (11.8.3), if any.

2 An `access_context` has an associated scope and designating class.

```
namespace std::meta {
    struct access_context {
        access_context() = delete;

        constexpr info scope() const;
        constexpr info designating_class() const;

        static constexpr access_context current() noexcept;
        static constexpr access_context unprivileged() noexcept;
        static constexpr access_context unchecked() noexcept;
        constexpr access_context via(info cls) const;
    };
}
```

3 The type `access_context` is a structural, non-aggregate type. A value `ac` of type `access_context` is consteval-only if either `ac.scope()` or `ac.designating_class()` is consteval-only. Two values `ac1` and `ac2` of type `access_context` are template-argument-equivalent (13.6) if `ac1.scope()` and `ac2.scope()` are template-argument-equivalent and `ac1.designating_class()` and `ac2.designating_class()` are template-argument-equivalent.

```
constexpr info scope() const;
constexpr info designating_class() const;
```

4 *Returns:* The `access_context`'s associated scope and designating class, respectively.

```
static constexpr access_context current() noexcept;
```

5 *Returns:* An `access_context` whose designating class is the null reflection and whose scope is *CURRENT-SCOPE* (P), where P is the point at which the invocation of `current` lexically appears.

6 *Remarks:* `current` is not an addressable function (16.4.5.2.1). An invocation of `current` that appears at a program point P is value-dependent (13.8.3.4) if *eval-point* (P) is enclosed by a scope corresponding to a templated entity.

7 [Example 1:

```
    struct A {
        int a = 0;
        constexpr A(int p) : a(p) {}
    };
    struct B : A {
        using A::A;
        constexpr B(int p, int q) : A(p * q) {}
        info s = access_context::current().scope();
    };
    struct C : B { using B::B; };

    struct Agg {
        constexpr bool eq(info rhs = access_context::current().scope()) {
            return s == rhs;
        }
    }
```

```

    info s = access_context::current().scope();
};

namespace NS {
    static_assert(Agg{}.s == access_context::current().scope());
    static_assert(Agg{}.eq());
    static_assert(B(1).s == ^^B);
    static_assert(is_constructor(B{1, 2}.s) && parent_of(B{1, 2}.s) == ^^B);
    static_assert(is_constructor(C{1, 2}.s) && parent_of(C{1, 2}.s) == ^^B);

    auto fn() -> [:is_namespace(access_context::current().scope()) ? ^^int : ^^bool:];
    static_assert(type_of(^^fn) == ^^auto()->int);

    template<auto R>
    struct TCls {
        consteval bool fn()
            requires (is_type(access_context::current().scope())) {
            return true; // OK, scope is TCls<R>.
        }
    };
    static_assert(TCls<0>{}.fn());
}
— end example]

```

```
static consteval access_context unprivileged() noexcept;
```

8 *Returns:* An `access_context` whose designating class is the null reflection and whose scope is the global namespace.

```
static consteval access_context unchecked() noexcept;
```

9 *Returns:* An `access_context` whose designating class and scope are both the null reflection.

```
consteval access_context via(info cls) const;
```

10 *Returns:* An `access_context` whose scope is `this->scope()` and whose designating class is `cls`.

11 *Throws:* `meta::exception` unless `cls` is either the null reflection or a reflection of a complete class type.

21.4.10 Member accessibility queries

[`meta.reflection.access.queries`]

```
consteval bool is_accessible(info r, access_context ctx);
```

1 Let *PARENT-CLS*(*r*) be:

- (1.1) — If `parent_of(r)` represents a class *C*, then *C*.
- (1.2) — Otherwise, *PARENT-CLS*(`parent_of(r)`).

2 Let *DESIGNATING-CLS*(*r*, *ctx*) be:

- (2.1) — If `ctx.designating_class()` represents a class *C*, then *C*.
- (2.2) — Otherwise, *PARENT-CLS*(*r*).

3 *Returns:*

- (3.1) — If *r* represents an unnamed bit-field *F*, then `is_accessible(rH, ctx)`, where *r_H* represents a hypothetical non-static data member of the class represented by *PARENT-CLS*(*r*) with the same access as *F*.
 [Note 1: Unnamed bit-fields are treated as class members for the purpose of `is_accessible`. — end note]
- (3.2) — Otherwise, if *r* does not represent a class member or a direct base class relationship, then `true`.
- (3.3) — Otherwise, if *r* represents
 - (3.3.1) — a class member that is not a (possibly indirect or variant) member of *DESIGNATING-CLS*(*r*, *ctx*) or
 - (3.3.2) — a direct base class relationship such that `parent_of(r)` does not represent *DESIGNATING-CLS*(*r*, *ctx*) or a (direct or indirect) base class thereof,

then `false`.

(3.4) — Otherwise, if `ctx.scope()` is the null reflection, then `true`.

(3.5) — Otherwise, letting P be a program point whose immediate scope is the function parameter scope, class scope, or namespace scope corresponding to the function, class, or namespace represented by `ctx.scope()`:

(3.5.1) — If r represents a direct base class relationship (D, B) , then `true` if base class B of *DESIGNATING-CLS* (r , `ctx`) is accessible at P (11.8.3); otherwise `false`.

(3.5.2) — Otherwise, r represents a class member M ; `true` if M would be accessible at P with the designating class (11.8.3) as *DESIGNATING-CLS* (r , `ctx`) if the effect of any *using-declarations* (9.10) were ignored. Otherwise, `false`.

[Note 2: The definitions of when a class member or base class is accessible from a point P do not consider whether a declaration of that entity is reachable from P . — end note]

4 *Throws*: `meta::exception` if r represents a class member for which *PARENT-CLS* (r) is an incomplete class.

5 [Example 1:

```
consteval access_context fn() {
    return access_context::current();
}

class Cls {
    int mem;
    friend consteval access_context fn();
public:
    static constexpr auto r = ^~mem;
};

static_assert(is_accessible(Cls::r, fn()));
static_assert(!is_accessible(Cls::r, access_context::current()));
static_assert(is_accessible(Cls::r, access_context::unchecked()));
— end example]
```

```
consteval bool has_inaccessible_nonstatic_data_members(info r, access_context ctx);
```

6 *Returns*: `true` if `is_accessible( $R$ , ctx)` is `false` for any R in `nonstatic_data_members_of(r, access_context::unchecked())`. Otherwise, `false`.

7 *Throws*: `meta::exception` if

(7.1) — the evaluation of `nonstatic_data_members_of(r, access_context::unchecked())` would exit via an exception or

(7.2) — r represents a closure type.

```
consteval bool has_inaccessible_bases(info r, access_context ctx);
```

8 *Returns*: `true` if `is_accessible( $R$ , ctx)` is `false` for any R in `bases_of(r, access_context::unchecked())`. Otherwise, `false`.

9 *Throws*: `meta::exception` if the evaluation of `bases_of(r, access_context::unchecked())` would exit via an exception.

```
consteval bool has_inaccessible_subobjects(info r, access_context ctx);
```

10 *Effects*: Equivalent to:

```
return has_inaccessible_bases(r, ctx) || has_inaccessible_nonstatic_data_members(r, ctx);
```

21.4.11 Reflection member queries

[meta.reflection.member.queries]

```
consteval vector<info> members_of(info r, access_context ctx);
```

1 A declaration D *members-of-precedes* a point P if D precedes either P or the point immediately following the *class-specifier* of the outermost class for which P is in a complete-class context.

2 A declaration D of a member M of a class or namespace Q is *Q -members-of-eligible* if

- (2.1) — the host scope of D (6.4.1) is the class scope or namespace scope associated with Q ,
- (2.2) — D is not a friend declaration,
- (2.3) — M is not a closure type (7.5.6.2),
- (2.4) — M is not a specialization of a template (13.1),
- (2.5) — if Q is a class that is not a closure type, then M is a direct member of Q (11.4.1) that is not a variant member of a nested anonymous union of Q (11.5.2), and
- (2.6) — if Q is a closure type, then M is a function call operator or function call operator template.

It is implementation-defined whether declarations of other members of a closure type Q are Q -members-of-eligible.

3 A member M of a class or namespace Q is *Q -members-of-representable* from a point P if a Q -members-of-eligible declaration of M members-of-precedes P , and M is

- (3.1) — a class or enumeration type,
- (3.2) — a type alias,
- (3.3) — a class template, function template, variable template, alias template, or concept,
- (3.4) — a variable or reference V for which the type of V does not contain an undeduced placeholder type,
- (3.5) — a function F for which
 - (3.5.1) — the type of F does not contain an undeduced placeholder type,
 - (3.5.2) — the constraints (if any) of F are satisfied, and
 - (3.5.3) — if F is a prospective destructor, F is the selected destructor (11.4.7),
- (3.6) — a non-static data member,
- (3.7) — a namespace, or
- (3.8) — a namespace alias.

[Note 1: Examples of direct members that are not Q -members-of-representable for any entity Q include: unscoped enumerators (9.8), partial specializations of templates (13.7.6), and closure types (7.5.6.2). — end note]

4 *Returns:* A **vector** containing reflections of all members M of the entity Q represented by `dealias(r)` for which

- (4.1) — M is Q -members-of-representable from some point in the evaluation context and
- (4.2) — `is_accessible(^M, ctx)` is `true`.

If `dealias(r)` represents a class C , then the **vector** also contains reflections representing all unnamed bit-fields B whose declarations inhabit the class scope corresponding to C for which `is_accessible(^B, ctx)` is `true`. Reflections of class members and unnamed bit-fields that are declared appear in the order in which they are declared.

[Note 2: Base classes are not members. Implicitly-declared special members appear after any user-declared members (11.4.4). — end note]

5 *Throws:* `meta::exception` unless `dealias(r)` is a reflection representing either a class type that is complete from some point in the evaluation context or a namespace.

6 [Example 1:

```
// TU1
export module M;
namespace NS {
    export int m;
    static int l;
}
static_assert(members_of(^NS, access_context::current()).size() == 2);

// TU2
import M;
```



```

static_assert(
    members_of(^^NS, access_context::current()).size() == 1);    // NS::1 does not precede
                                                                    // the constant-expression (6.5)

class B {};

struct S : B {
private:
    class I;
public:
    int m;
};

static_assert(
    members_of(^^S, access_context::current()).size() == 7);    // 6 special members,
                                                                    // 1 public member,
                                                                    // does not include base

static_assert(
    members_of(^^S, access_context::unchecked()).size() == 8);  // all of the above,
                                                                    // as well as a reflection
                                                                    // representing S::I

— end example]

```

```
constexpr vector<info> bases_of(info type, access_context ctx);
```

7 *Returns:* Let C be the class represented by `dealias(type)`. A vector containing the reflections of all the direct base class relationships B , if any, of C such that `is_accessible(^^B, ctx)` is true. The direct base class relationships appear in the order in which the corresponding base classes appear in the *base-specifier-list* of C .

8 *Throws:* `meta::exception` unless `dealias(type)` represents a class type that is complete from some point in the evaluation context.

```
constexpr vector<info> static_data_members_of(info type, access_context ctx);
```

9 *Returns:* A vector containing each element e of `members_of(type, ctx)` such that `is_variable(e)` is true, preserving their order.

10 *Throws:* `meta::exception` unless `dealias(type)` represents a class type that is complete from some point in the evaluation context.

```
constexpr vector<info> nonstatic_data_members_of(info type, access_context ctx);
```

11 *Returns:* A vector containing each element e of `members_of(type, ctx)` such that `is_nonstatic_data_member(e)` is true, preserving their order.

12 *Throws:* `meta::exception` unless `dealias(type)` represents a class type that is complete from some point in the evaluation context.

```
constexpr vector<info> subobjects_of(info type, access_context ctx);
```

13 *Returns:* A vector containing each element of `bases_of(type, ctx)` followed by each element of `nonstatic_data_members_of(type, ctx)`, preserving their order.

14 *Throws:* `meta::exception` unless `dealias(type)` represents a class type that is complete from some point in the evaluation context.

```
constexpr vector<info> enumerators_of(info type_enum);
```

15 *Returns:* A vector containing the reflections of each enumerator of the enumeration represented by `dealias(type_enum)`, in the order in which they are declared.

16 *Throws:* `meta::exception` unless `dealias(type_enum)` represents an enumeration type, and `is_enumerable_type(type_enum)` is true.

21.4.12 Reflection layout queries

[meta.reflection.layout]

```

struct member_offset {
    ptrdiff_t bytes;
    ptrdiff_t bits;
    constexpr ptrdiff_t total_bits() const;

```

```
    auto operator<=>(const member_offset&) const = default;
};
```

```
constexpr ptrdiff_t member_offset::total_bits() const;
```

1 *Returns:* bytes * CHAR_BIT + bits.

```
constexpr member_offset offset_of(info r);
```

2 Let V be the offset in bits from the beginning of a complete object of the type represented by `parent_of(r)` to the subobject associated with the construct represented by r .

3 *Returns:* $\{V / \text{CHAR_BIT}, V \% \text{CHAR_BIT}\}$.

4 *Throws:* `meta::exception` unless r represents a non-static data member, unnamed bit-field, or direct base class relationship (D, B) for which either B is not a virtual base class or D is not an abstract class.

```
constexpr size_t size_of(info r);
```

5 *Returns:* If

(5.1) — r represents a non-static data member of type T or a data member description (T, N, A, W, NUA, ANN) or

(5.2) — `dealias(r)` represents a type T ,

then `sizeof(T)` if T is not a reference type and `size_of(add_pointer(~~T))` otherwise. Otherwise, `size_of(type_of(r))`.

[*Note 1:* It is possible that while `sizeof(char) == size_of(~~char)` is true, that `sizeof(char&) == size_of(~~char&)` is false. If b represents a direct base class relationship of an empty base class, then `size_of(b) > 0` is true. — end note]

6 *Throws:* `meta::exception` unless all of the following conditions are met:

(6.1) — `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description (T, N, A, W, NUA, ANN) (11.4.1) where W is $\perp$.

(6.2) — If `dealias(r)` represents a type, then `is_complete_type(r)` is true.

```
constexpr size_t alignment_of(info r);
```

7 *Returns:*

(7.1) — If `dealias(r)` represents a type T , then `alignment_of(add_pointer(r))` if T is a reference type and the alignment requirement of T otherwise.

(7.2) — Otherwise, if `dealias(r)` represents a variable or object, then the alignment requirement of the variable or object.

(7.3) — Otherwise, if r represents a direct base class relationship, then `alignment_of(type_of(r))`.

(7.4) — Otherwise, if r represents a non-static data member M of a class C , then the alignment of the direct member subobject corresponding to M of a complete object of type C .

(7.5) — Otherwise, r represents a data member description (T, N, A, W, NUA, ANN) (11.4.1). If A is not $\perp$, then the value A . Otherwise, `alignment_of(~~T)`.

8 *Throws:* `meta::exception` unless all of the following conditions are met:

(8.1) — `dealias(r)` is a reflection of a type, object, variable of non-reference type, non-static data member that is not a bit-field, direct base class relationship, or data member description (T, N, A, W, NUA, ANN) (11.4.1) where W is $\perp$.

(8.2) — If `dealias(r)` represents a type, then `is_complete_type(r)` is true.

```
constexpr size_t bit_size_of(info r);
```

9 *Returns:*

(9.1) — If r represents an unnamed bit-field or a non-static data member that is a bit-field with width W , then W .

(9.2) — Otherwise, if r represents a data member description (T, N, A, W, NUA, ANN) (11.4.1) and W is not $\perp$, then W .

(9.3) — Otherwise, `CHAR_BIT * sizeof(r)`.

10 *Throws:* `meta::exception` unless all of the following conditions are met:

(10.1) — `dealias(r)` is a reflection of a type, object, value, variable of non-reference type, non-static data member, unnamed bit-field, direct base class relationship, or data member description.

(10.2) — If `dealias(r)` represents a type, then `is_complete_type(r)` is `true`.

21.4.13 Annotation reflection

[`meta.reflection.annotation`]

`constexpr vector<info> annotations_of(info item);`

1 For a function F , let $S(F)$ be the set of declarations, ignoring any explicit instantiations, that declare either F or a templated function of which F is a specialization.

2 *Returns:* A `vector` containing all of the reflections R representing each annotation applying to:

(2.1) — if `item` represents a function parameter P of a function F , then the declaration of P in each declaration of F in $S(F)$,

(2.2) — otherwise, if `item` represents a function F , then each declaration of F in $S(F)$,

(2.3) — otherwise, if `item` represents a direct base class relationship (D, B) , then the corresponding *base-specifier* in the definition of D ,

(2.4) — otherwise, each declaration of the entity represented by `item`,

such that each specified declaration is reachable from either some point in the evaluation context (7.7.6) or a point immediately following the *class-specifier* of the outermost class for which such a point is in a complete-class context. For any two reflections R_1 and R_2 in the returned `vector`, if the annotation represented by R_1 precedes the annotation represented by R_2 , then R_1 appears before R_2 . If R_1 and R_2 represent annotations from the same translation unit T , any element in the returned `vector` between R_1 and R_2 represents an annotation from T .

[*Note 1:* The order in which two annotations appear is otherwise unspecified. — *end note*]

3 *Throws:* `meta::exception` unless `item` represents a type, type alias, variable, function, function parameter, namespace, enumerator, direct base class relationship, or non-static data member.

4 [*Example 1:*

```
[[=1]] void f();
[[=2, =3]] void g();
void g [[=4]] ();

static_assert(annotations_of(f).size() == 1);
static_assert(annotations_of(g).size() == 3);
static_assert([: constant_of(annotations_of(g)[0]) :] == 2);
static_assert(extract<int>(annotations_of(g)[1]) == 3);
static_assert(extract<int>(annotations_of(g)[2]) == 4);

struct Option { bool value; };

struct C {
    [[=Option{true}]] int a;
    [[=Option{false}]] int b;
};

static_assert(extract<Option>(annotations_of(C::a)[0]).value);
static_assert(!extract<Option>(annotations_of(C::b)[0]).value);

template<class T>
    struct [[=42]] D { };

constexpr std::meta::info a1 = annotations_of(D<int>)[0];
constexpr std::meta::info a2 = annotations_of(D<char>)[0];
static_assert(a1 != a2);
static_assert(constant_of(a1) == constant_of(a2));
```

```
[[=1]] int x, y;
static_assert(annotations_of(^x)[0] == annotations_of(^y)[0]);
— end example]
```

```
constexpr vector<info> annotations_of_with_type(info item, info type);
```

5 *Returns:* A vector containing each element *e* of `annotations_of(item)` where
`remove_const(type_of(e)) == remove_const(type)`
is `true`, preserving their order.

6 *Throws:* `meta::exception` unless

(6.1) — the evaluation of `annotations_of(item)` would not exit via an exception and

(6.2) — `dealias(type)` represents a type and `is_complete_type(type)` is `true`.

21.4.14 Value extraction

[`meta.reflection.extract`]

1 The `extract` function template may be used to extract a value out of a reflection when its type is known.

2 The following are defined for exposition only to aid in the specification of `extract`.

```
template<class T>
constexpr T extract-ref(info r);          // exposition only
```

3 [Note 1: *T* is a reference type. — end note]

4 *Returns:* If *r* represents an object *O*, then a reference to *O*. Otherwise, a reference to the object declared, or referred to, by the variable represented by *r*.

5 *Throws:* `meta::exception` unless

(5.1) — *r* represents a variable or object of type *U*,

(5.2) — `is_convertible_v<remove_reference_t<U>(*), remove_reference_t<T>(*)>` is `true`,
and

[Note 2: The intent is to allow only qualification conversion from *U* to *T*. — end note]

(5.3) — If *r* represents a variable, then either that variable is usable in constant expressions or its lifetime began within the core constant expression currently under evaluation.

```
template<class T>
constexpr T extract-member-or-function(info r);          // exposition only
```

6 *Returns:*

(6.1) — If *T* is a pointer type, then a pointer value pointing to the function represented by *r*.

(6.2) — Otherwise, a pointer-to-member value designating the non-static data member or function represented by *r*.

7 *Throws:* `meta::exception` unless

(7.1) — *r* represents a non-static data member with type *X*, that is not a bit-field, that is a direct member of class *C*, *T* and `X C::*` are similar types (7.3.6), and `is_convertible_v<X C::*, T>` is `true`;

(7.2) — *r* represents an implicit object member function with type *F* or `F noexcept` that is a direct member of a class *C*, and *T* is `F C::*`; or

(7.3) — *r* represents a non-member function, static member function, or explicit object member function of function type *F* or `F noexcept`, and *T* is `F*`.

```
template<class T>
constexpr T extract-value(info r);          // exposition only
```

8 Let *U* be the type of the value or object that *r* represents.

9 *Returns:* `static_cast<T>([:R:])`, where *R* is a constant expression of type *info* such that *R* == *r* is `true`.

10 *Throws:* `meta::exception` unless

(10.1) — *U* is a pointer type, *T* and *U* are either similar (7.3.6) or both function pointer types, and `is_convertible_v<U, T>` is `true`,

- (10.2) — U is not a pointer type and the cv-unqualified types of T and U are the same,
- (10.3) — U is an array type, T is a pointer type, `remove_extent_t<U>*` and T are similar types, and the value `r` represents is convertible to T, or
- (10.4) — U is a closure type, T is a function pointer type, and the value that `r` represents is convertible to T.

```
template<class T>
constexpr T extract(info r);
11   Let U be remove_cv_t<T>.
12   Effects: Equivalent to:
      if constexpr (is_reference_type(^T)) {
        return extract-ref<T>(r);
      } else if (is_nonstatic_data_member(r) || is_function(r)) {
        return extract-member-or-function<U>(r);
      } else {
        return extract-value<U>(constant_of(r));
      }
```

21.4.15 Reflection substitution

[`meta.reflection.substitute`]

1 Let *TARG-SPLICE*(`x`) be:

- (1.1) — `template [: x :]` if `is_template(x)` is true, otherwise
- (1.2) — `typename [: x :]` if `is_type(x)` is true, otherwise
- (1.3) — `([: x :])`.

```
template<class R>
concept reflection_range =
    ranges::input_range<R> &&
    same_as<ranges::range_value_t<R>, info> &&
    same_as<remove_cvref_t<ranges::range_reference_t<R>>, info>;
```

```
template<reflection_range R = initializer_list<info>>
constexpr bool can_substitute(info templ, R&& arguments);
```

2 Let Z be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.

3 *Returns:* true if `Z<TARG-SPLICE(Args)...>` is a valid *template-id* (13.3) that does not name a function whose type contains an undeduced placeholder type. Otherwise, false.

4 *Throws:* `meta::exception` unless `templ` represents a template, and every reflection in `arguments` represents a construct usable as a template argument (13.4).

5 [Note 1: If forming `Z<TARG-SPLICE(Args)...>` leads to a failure outside of the immediate context, the program is ill-formed. — end note]

```
template<reflection_range R = initializer_list<info>>
constexpr info substitute(info templ, R&& arguments);
```

6 Let Z be the template represented by `templ` and let `Args...` be a sequence of prvalue constant expressions that compute the reflections held by the elements of `arguments`, in order.

7 *Returns:* `^^Z<TARG-SPLICE(Args)...>`.

8 *Throws:* `meta::exception` unless `can_substitute(templ, arguments)` is true.

9 [Note 2: If forming `Z<TARG-SPLICE(Args)...>` leads to a failure outside of the immediate context, the program is ill-formed. — end note]

10 [Example 1:

```
template<class T>
auto fn1();

static_assert(!can_substitute(^fn1, {^int}));
constexpr info r1 = substitute(^fn1, {^int});
```

// error: `fn1<int>` contains an undeduced
// placeholder type for its return type

```

template<class T>
auto fn2() {
    static_assert(^T != ^int);           // static assertion failed during
                                         // instantiation of fn2<int>

    return 0;
}

constexpr bool r2 = can_substitute(^fn2, {^int}); // error: instantiation of body of fn2<int>
                                                    // is needed to deduce return type

```

— end example]

11 [Example 2:

```

constexpr info to_integral_constant(unsigned i) {
    return substitute(^integral_constant, {^unsigned, reflect_constant(i)});
}

constexpr info r = to_integral_constant(2); // OK, r represents the type
                                              // integral_constant<unsigned, 2>

```

— end example]

21.4.16 Expression result reflection

[meta.reflection.result]

```

template<class T>
constexpr info reflect_constant(T expr);

```

1 *Mandates:* `is_copy_constructible_v<T>` is true and `T` is a cv-unqualified structural type (13.2) that is not a reference type.

2 *Let* `V` be:

(2.1) — if `T` is a class type, then an object that is template-argument-equivalent to the value of `expr`;

(2.2) — otherwise, the value of `expr`.

3 *Let* `TCLs` be the invented template:

```
template<T P> struct TCLs;
```

4 *Returns:* `template_arguments_of(^TCLs<V>)[0]`.

[Note 1: This is a reflection of an object for class types, and a reflection of a value otherwise. — end note]

5 *Throws:* `meta::exception` unless the *template-id* `TCLs<V>` would be valid.

6 [Example 1:

```

template<auto D>
struct A { };

struct N { int x; };
struct K { char const* p; };

constexpr info r1 = reflect_constant(42);
static_assert(is_value(r1));
static_assert(r1 == template_arguments_of(^A<42>)[0]);

constexpr info r2 = reflect_constant(N{42});
static_assert(is_object(r2));
static_assert(r2 == template_arguments_of(^A<N{42}>)[0]);

constexpr info r3 = reflect_constant(K{nullptr});
constexpr info r4 = reflect_constant(K{"ebab"});

```

// error: constituent pointer
// points to string literal

— end example]

```

template<class T>
constexpr info reflect_object(T& expr);

```

7 *Mandates:* `T` is an object type.

8 *Returns:* A reflection of the object designated by `expr`.

- 9 *Throws:* `meta::exception` if `E` is not suitable for use as a constant template argument for a constant template parameter of type `T&` (13.4.3), where `E` is an lvalue constant expression that computes the object that `expr` refers to.

```
template<class T>
constexpr info reflect_function(T& fn);
```

- 10 *Mandates:* `T` is a function type.

- 11 *Returns:* A reflection of the function designated by `fn`.

- 12 *Throws:* `meta::exception` if `F` is not suitable for use as a constant template argument for a constant template parameter of type `T&` (13.4.3), where `F` is an lvalue constant expression that computes the function that `fn` refers to.

21.4.17 Reflection class definition generation [meta.reflection.define.aggregate]

```
namespace std::meta {
    struct data_member_options {
        struct name-type {                                // exposition only
            template<class T>
                requires constructible_from<u8string, T>
                constexpr name-type (T&&);

            template<class T>
                requires constructible_from<string, T>
                constexpr name-type (T&&);

        private:
            variant<u8string, string> contents;            // exposition only
        };

        optional<name-type> name;
        optional<int> alignment;
        optional<int> bit_width;
        bool no_unique_address = false;
        vector<info> annotations;
    };
}
```

- 1 The classes `data_member_options` and `data_member_options::name-type` are not structural types (13.2). Values of both types are `constexpr`-only values.

```
template<class T>
requires constructible_from<u8string, T>
constexpr name-type (T&& value);
```

- 2 *Effects:* Initializes `contents` with `u8string(std::forward<T>(value))`.

```
template<class T>
requires constructible_from<string, T>
constexpr name-type (T&& value);
```

- 3 *Effects:* Initializes `contents` with `string(std::forward<T>(value))`.

[Note 1: The class `name-type` allows the function `data_member_spec` to accept an ordinary string literal (or `string_view`, `string`, etc.) or a UTF-8 string literal (or `u8string_view`, `u8string`, etc.) equally well.

[Example 1:

```
constexpr void fn() {
    data_member_options o1 = {.name = "ordinary_literal_encoding"};
    data_member_options o2 = {.name = u8"utf8_encoding"};
}
```

— end example]

— end note]

```
constexpr info data_member_spec(info type, data_member_options options);
```

- 4 *Returns:* A reflection of a data member description (`T`, `N`, `A`, `W`, `NUA`, `ANN`) (11.4.1) where

- (4.1) — T is the type represented by `dealias(type)`,
- (4.2) — N is either the identifier encoded by `options.name` or $\perp$ if `options.name` does not contain a value,
- (4.3) — A is either the alignment value held by `options.alignment` or $\perp$ if `options.alignment` does not contain a value,
- (4.4) — W is either the value held by `options.bit_width` or $\perp$ if `options.bit_width` does not contain a value,
- (4.5) — NUA is the value held by `options.no_unique_address`, and
- (4.6) — ANN is the sequence of values `constant_of(r)` for each r in `options.annotations`.

[*Note 2:* The returned reflection value is primarily useful in conjunction with `define_aggregate`; it can also be queried by certain other functions in `std::meta` (e.g., `type_of`, `identifier_of`). — *end note*]

5 *Throws:* `meta::exception` unless the following conditions are met:

- (5.1) — `dealias(type)` represents either an object type or a reference type;
- (5.2) — if `options.name` contains a value, then:
 - (5.2.1) — `holds_alternative<u8string>(options.name->contents)` is true and `get<u8string>(options.name->contents)` contains the spelling of a valid *token* that is an *identifier* (5.11) when interpreted with UTF-8, or
 - (5.2.2) — `holds_alternative<string>(options.name->contents)` is true and `get<string>(options.name->contents)` contains the spelling of a valid *token* that is an *identifier* (5.11) when interpreted with the ordinary literal encoding;
- [*Note 3:* Lexical constructs like *universal-character-names* (5.3.2) are not processed. For example, `R"(\u03B1)"` is an invalid identifier and is not interpreted as " α ". — *end note*]
- (5.3) — if `options.name` does not contain a value, then `options.bit_width` contains a value and `options.annotations` is empty;
- (5.4) — if `options.bit_width` contains a value V , then
 - (5.4.1) — `is_integral_type(type) || is_enum_type(type)` is true,
 - (5.4.2) — `options.alignment` does not contain a value,
 - (5.4.3) — `options.no_unique_address` is false,
 - (5.4.4) — V is not negative, and
 - (5.4.5) — if V equals 0, then `options.name` does not contain a value; and
 - (5.4.6) — if `options.name` does not contain a value, then `is_const(type) || is_volatile(type)` is false; and
- (5.5) — if `options.alignment` contains a value, it is an alignment value (6.8.3) not less than `alignment_of(type)`; and
- (5.6) — for every reflection r in `options.annotations`, `has_type(r)` is true, `type_of(r)` represents a non-array object type, and evaluation of `constant_of(r)` does not exit via an exception.

```
constexpr bool is_data_member_spec(info r);
```

6 *Returns:* true if r represents a data member description. Otherwise, false.

```
template<reflection_range R = initializer_list<info>>
constexpr info define_aggregate(info class_type, R&& mdescrs);
```

7 Let C be the type represented by `class_type` and r_K be the K^{th} reflection value in `mdescrs`. For every r_K in `mdescrs`, let $(T_K, N_K, A_K, W_K, NUA_K, ANN_K)$ be the corresponding data member description represented by r_K .

8 *Constant When:*

- (8.1) — `class_type` represents a cv-qualified class type;
- (8.2) — C is incomplete from every point in the evaluation context;

[*Note 4:* C can be a class template specialization for which there is a reachable definition of the class template. In this case, the injected declaration is an explicit specialization. — *end note*]

- (8.3) — `is_data_member_spec( $r_K$ )` is `true` for every r_K ;
 - (8.4) — `is_complete_type( $T_K$ )` is `true` for every r_K ; and
 - (8.5) — for every pair (r_K, r_L) where $K < L$, if N_K is not $\perp$ and N_L is not $\perp$, then either:
 - (8.5.1) — N_K is not the same identifier as N_L or
 - (8.5.2) — N_K is the identifier `_` (U+005F low line).
- [Note 5: Every provided identifier is unique or `"_"`. — end note]
- 9 *Effects:* Produces an injected declaration D (7.7.6) that defines C and has properties as follows:
- (9.1) — The target scope of D is the scope to which C belongs (6.4.1).
 - (9.2) — The locus of D follows immediately after the core constant expression currently under evaluation.
 - (9.3) — The characteristic sequence of D (7.7.6) is the sequence of reflection values r_K .
 - (9.4) — If C is a specialization of a templated class T , and C is not a local class, then D is an explicit specialization of T .
 - (9.5) — For each r_K , there is a corresponding entity M_K with public access belonging to the class scope of D with the following properties:
 - (9.5.1) — If N_K is $\perp$, M_K is an unnamed bit-field. Otherwise, M_K is a non-static data member whose name is the identifier N_K .
 - (9.5.2) — The type of M_K is T_K .
 - (9.5.3) — M_K is declared with the attribute `[[no_unique_address]]` if and only if NUA_K is `true`.
 - (9.5.4) — If W_K is not $\perp$, M_K is a bit-field whose width is that value. Otherwise, M_K is not a bit-field.
 - (9.5.5) — If A_K is not $\perp$, M_K has the *alignment-specifier* `alignas( $A_K$ )`. Otherwise, M_K has no *alignment-specifier*.
 - (9.5.6) — For every reflection $\mathbf{r}$ in ANN_K , M_K has an annotation whose underlying constant (9.13.12) is $\mathbf{r}$.
 - (9.6) — For every r_L in `mdescrs` such that $K < L$, the declaration corresponding to r_K precedes the declaration corresponding to r_L .
- 10 *Returns:* `class_type`.
- 11 *Remarks:* If C is a specialization of a templated class and it has not been instantiated, C is treated as an explicit specialization.

21.4.18 Reflection type traits

[meta.reflection.traits]

- 1 This subclause specifies `constexpr` functions to query the properties of types (21.3.5), query the relationships between types (21.3.7), or transform types (21.3.8), during program translation. Each `constexpr` function declared in this class has an associated class template declared elsewhere in this document.
- 2 Every function and function template declared in this subclause throws an exception of type `meta::exception` unless the following conditions are met:

- (2.1) — For every parameter $\mathbf{p}$ of type `info`, `is_type( $\mathbf{p}$ )` is `true`.
- (2.2) — For every parameter $\mathbf{r}$ whose type is constrained on `reflection_range`, `ranges::all_of( $\mathbf{r}$ , is_type)` is `true`.

// associated with 21.3.5.2, primary type categories

```
constexpr bool is_void_type(info type);
constexpr bool is_null_pointer_type(info type);
constexpr bool is_integral_type(info type);
constexpr bool is_floating_point_type(info type);
constexpr bool is_array_type(info type);
constexpr bool is_pointer_type(info type);
constexpr bool is_lvalue_reference_type(info type);
constexpr bool is_rvalue_reference_type(info type);
constexpr bool is_member_object_pointer_type(info type);
constexpr bool is_member_function_pointer_type(info type);
constexpr bool is_enum_type(info type);
constexpr bool is_union_type(info type);
```

```

constexpr bool is_class_type(info type);
constexpr bool is_function_type(info type);
constexpr bool is_reflection_type(info type);

// associated with 21.3.5.3, composite type categories
constexpr bool is_reference_type(info type);
constexpr bool is_arithmetic_type(info type);
constexpr bool is_fundamental_type(info type);
constexpr bool is_object_type(info type);
constexpr bool is_scalar_type(info type);
constexpr bool is_compound_type(info type);
constexpr bool is_member_pointer_type(info type);

// associated with 21.3.5.4, type properties
constexpr bool is_const_type(info type);
constexpr bool is_volatile_type(info type);
constexpr bool is_trivially_copyable_type(info type);
constexpr bool is_standard_layout_type(info type);
constexpr bool is_empty_type(info type);
constexpr bool is_polymorphic_type(info type);
constexpr bool is_abstract_type(info type);
constexpr bool is_final_type(info type);
constexpr bool is_aggregate_type(info type);
constexpr bool is_structural_type(info type);
constexpr bool is_signed_type(info type);
constexpr bool is_unsigned_type(info type);
constexpr bool is_bounded_array_type(info type);
constexpr bool is_unbounded_array_type(info type);
constexpr bool is_scoped_enum_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_constructible_type(info type, R&& type_args);
constexpr bool is_default_constructible_type(info type);
constexpr bool is_copy_constructible_type(info type);
constexpr bool is_move_constructible_type(info type);

constexpr bool is_assignable_type(info type_dst, info type_src);
constexpr bool is_copy_assignable_type(info type);
constexpr bool is_move_assignable_type(info type);

constexpr bool is_swappable_with_type(info type1, info type2);
constexpr bool is_swappable_type(info type);

constexpr bool is_destructible_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_trivially_constructible_type(info type, R&& type_args);
constexpr bool is_trivially_default_constructible_type(info type);
constexpr bool is_trivially_copy_constructible_type(info type);
constexpr bool is_trivially_move_constructible_type(info type);

constexpr bool is_trivially_assignable_type(info type_dst, info type_src);
constexpr bool is_trivially_copy_assignable_type(info type);
constexpr bool is_trivially_move_assignable_type(info type);
constexpr bool is_trivially_destructible_type(info type);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_constructible_type(info type, R&& type_args);
constexpr bool is_nothrow_default_constructible_type(info type);
constexpr bool is_nothrow_copy_constructible_type(info type);
constexpr bool is_nothrow_move_constructible_type(info type);

constexpr bool is_nothrow_assignable_type(info type_dst, info type_src);
constexpr bool is_nothrow_copy_assignable_type(info type);

```

```

constexpr bool is_nothrow_move_assignable_type(info type);

constexpr bool is_nothrow_swappable_with_type(info type1, info type2);
constexpr bool is_nothrow_swappable_type(info type);

constexpr bool is_nothrow_destructible_type(info type);

constexpr bool is_implicit_lifetime_type(info type);

constexpr bool has_virtual_destructor(info type);

constexpr bool has_unique_object_representations(info type);

constexpr bool reference_constructs_from_temporary(info type_dst, info type_src);
constexpr bool reference_converts_from_temporary(info type_dst, info type_src);

// associated with 21.3.7, type relations
constexpr bool is_same_type(info type1, info type2);
constexpr bool is_base_of_type(info type_base, info type_derived);
constexpr bool is_virtual_base_of_type(info type_base, info type_derived);
constexpr bool is_convertible_type(info type_src, info type_dst);
constexpr bool is_nothrow_convertible_type(info type_src, info type_dst);
constexpr bool is_layout_compatible_type(info type1, info type2);
constexpr bool is_pointer_interconvertible_base_of_type(info type_base, info type_derived);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>
    constexpr bool is_invocable_r_type(info type_result, info type, R&& type_args);

template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_invocable_type(info type, R&& type_args);
template<reflection_range R = initializer_list<info>>
    constexpr bool is_nothrow_invocable_r_type(info type_result, info type, R&& type_args);

// associated with 21.3.8.2, const-volatile modifications
constexpr info remove_const(info type);
constexpr info remove_volatile(info type);
constexpr info remove_cv(info type);
constexpr info add_const(info type);
constexpr info add_volatile(info type);
constexpr info add_cv(info type);

// associated with 21.3.8.3, reference modifications
constexpr info remove_reference(info type);
constexpr info add_lvalue_reference(info type);
constexpr info add_rvalue_reference(info type);

// associated with 21.3.8.4, sign modifications
constexpr info make_signed(info type);
constexpr info make_unsigned(info type);

// associated with 21.3.8.5, array modifications
constexpr info remove_extent(info type);
constexpr info remove_all_extents(info type);

// associated with 21.3.8.6, pointer modifications
constexpr info remove_pointer(info type);
constexpr info add_pointer(info type);

// associated with 21.3.8.7, other transformations
constexpr info remove_cvref(info type);
constexpr info decay(info type);

```

```

template<reflection_range R = initializer_list<info>>
    consteval info common_type(R&& type_args);
template<reflection_range R = initializer_list<info>>
    consteval info common_reference(R&& type_args);
consteval info underlying_type(info type);
template<reflection_range R = initializer_list<info>>
    consteval info invoke_result(info type, R&& type_args);
consteval info unwrap_reference(info type);
consteval info unwrap_ref_decay(info type);

```

- ³ For a function or function template F defined in this subclause, let C be its associated class template. For the evaluation of a call to F , let S be the specialization of C in terms of which the call is specified.

(3.1) — If

- (3.1.1) — the template arguments of S violate a condition specified in a *Mandates* element in the specification of C ;
- (3.1.2) — the call is specified to produce a reflection of a type, but S would have no member named `type`; or
- (3.1.3) — the call is specified to return $S::\text{value}$, but that expression would not be a valid converted constant expression of type R , where R is the return type of F ;

then an exception of type `meta::exception` is thrown.

[Note 1: For the first case, S is not instantiated. — end note]

- (3.2) — Otherwise, if the instantiation of S would result in undefined behavior due to dependence on an incomplete type (21.3.2), then the call is not a constant subexpression.
- (3.3) — Otherwise, if the template arguments of S do not meet the preconditions of C , then it is unspecified whether the call is a constant subexpression. If it is, the call produces the result that would be produced if C had no preconditions.

- ⁴ Each function or function template declared above has the following behavior based on the signature and return type of that function or function template.

[Note 2: The associated class template need not be instantiated. — end note]

Table 64 — Reflection type traits [tab:meta.reflection.traits]

Signature and Return Type	Returns
<pre> bool meta::UNARY(info type); bool meta::UNARY_type(info type); </pre>	<pre> std::UNARY<T>::value, where T is the type or type alias represented by <code>type</code> </pre>
<pre> bool meta::BINARY(info t1, info t2); bool meta::BINARY_type(info t1, info t2); </pre>	<pre> std::BINARY<T_1, T_2>::value, where T_1 and T_2 are the types or type aliases represented by <code>t1</code> and <code>t2</code>, respectively </pre>
<pre> template<reflection_range R> bool meta::VARIADIC_type(info type, R&& args); </pre>	<pre> std::VARIADIC<T, $U...$>::value, where T is the type or type alias represented by <code>type</code> and $U...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of <code>args</code> </pre>
<pre> template<reflection_range R> bool meta::VARIADIC_type(info t1, info t2, R&& args); </pre>	<pre> std::VARIADIC<T_1, T_2, $U...$>::value, where T_1 and T_2 are the types or type aliases represented by <code>t1</code> and <code>t2</code>, respectively, and $U...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of <code>args</code> </pre>
<pre> info meta::UNARY(info type); </pre>	A reflection representing the type denoted by <pre> std::UNARY<T>::type, where T is the type or type alias represented by <code>type</code> </pre>
<pre> template<reflection_range R> info meta::VARIADIC(R&& args); </pre>	A reflection representing the type denoted by <pre> std::VARIADIC<$T...$>::type, where $T...$ is the pack of types or type aliases whose elements are represented by the corresponding elements of <code>args</code> </pre>

Table 64 — Reflection type traits (continued)

Signature and Return Type	Returns
<pre>template<reflection_range R> info meta::VARIADIC(info type, R&& args);</pre>	A reflection representing the type denoted by <code>std::VARIADIC<T, U...>::type</code> , where <i>T</i> is the type or type alias represented by <code>type</code> and <i>U...</i> is the pack of types or type aliases whose elements are represented by the corresponding elements of <code>args</code>

⁵ [Note 3: For those functions or function templates which return a reflection, that reflection always represents a type and never a type alias. — end note]

⁶ [Note 4: If *t* is a reflection of the type `int` and *u* is a reflection of an alias to the type `int`, then `t == u` is `false` but `is_same_type(t, u)` is `true`. Also, `t == dealias(u)` is `true`. — end note]

```
constexpr size_t rank(info type);
```

⁷ Returns: `std::rank<T>::value`, where *T* is the type represented by `dealias(type)`.

```
constexpr size_t extent(info type, unsigned i = 0);
```

⁸ Returns: `std::extent<T, I>::value`, where *T* is the type represented by `dealias(type)` and *I* is a constant equal to *i*.

```
constexpr size_t tuple_size(info type);
```

⁹ Returns: `std::tuple_size<T>::value`, where *T* is the type represented by `dealias(type)`.

```
constexpr info tuple_element(size_t index, info type);
```

¹⁰ Returns: A reflection representing the type denoted by `std::tuple_element<I, T>::type`, where *T* is the type represented by `dealias(type)` and *I* is a constant equal to *index*.

```
constexpr size_t variant_size(info type);
```

¹¹ Returns: `std::variant_size<T>::value`, where *T* is the type represented by `dealias(type)`.

```
constexpr info variant_alternative(size_t index, info type);
```

¹² Returns: A reflection representing the type denoted by `std::variant_alternative<I, T>::type`, where *T* is the type represented by `dealias(type)` and *I* is a constant equal to *index*.

```
constexpr strong_ordering type_order(info t1, info t2);
```

¹³ Returns: `std::type_order<T1, T2>::value`, where *T₁* and *T₂* are the types represented by `dealias(t1)` and `dealias(t2)`, respectively.

21.5 Compile-time rational arithmetic

[ratio]

21.5.1 General

[ratio.general]

¹ Subclause 21.5 describes the ratio library. It provides a class template `ratio` which exactly represents any finite rational number with a numerator and denominator representable by compile-time constants of type `intmax_t`.

² Throughout subclause 21.5, the names of template parameters are used to express type requirements. If a template parameter is named *R1* or *R2*, and the template argument is not a specialization of the `ratio` template, the program is ill-formed.

21.5.2 Header <ratio> synopsis

[ratio.syn]

```
// all freestanding
namespace std {
    // 21.5.3, class template ratio
    template<intmax_t N, intmax_t D = 1> class ratio;

    // 21.5.4, ratio arithmetic
    template<class R1, class R2> using ratio_add = see below;
    template<class R1, class R2> using ratio_subtract = see below;
    template<class R1, class R2> using ratio_multiply = see below;
    template<class R1, class R2> using ratio_divide = see below;
```

```

// 21.5.5, ratio comparison
template<class R1, class R2> struct ratio_equal;
template<class R1, class R2> struct ratio_not_equal;
template<class R1, class R2> struct ratio_less;
template<class R1, class R2> struct ratio_less_equal;
template<class R1, class R2> struct ratio_greater;
template<class R1, class R2> struct ratio_greater_equal;

template<class R1, class R2>
    constexpr bool ratio_equal_v = ratio_equal<R1, R2>::value;
template<class R1, class R2>
    constexpr bool ratio_not_equal_v = ratio_not_equal<R1, R2>::value;
template<class R1, class R2>
    constexpr bool ratio_less_v = ratio_less<R1, R2>::value;
template<class R1, class R2>
    constexpr bool ratio_less_equal_v = ratio_less_equal<R1, R2>::value;
template<class R1, class R2>
    constexpr bool ratio_greater_v = ratio_greater<R1, R2>::value;
template<class R1, class R2>
    constexpr bool ratio_greater_equal_v = ratio_greater_equal<R1, R2>::value;

// 21.5.6, convenience SI typedefs
using quecto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>; // see below
using ronto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>; // see below
using yocto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>; // see below
using zepto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>; // see below
using atto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using femto = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using pico = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using nano = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using micro = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using milli = ratio<1, 1'000'000'000'000'000'000'000'000'000'000'000>;
using centi = ratio<1, 100>;
using deci = ratio<1, 10>;
using deca = ratio<10, 1>;
using hecto = ratio<100, 1>;
using kilo = ratio<1'000, 1>;
using mega = ratio<1'000'000, 1>;
using giga = ratio<1'000'000'000, 1>;
using tera = ratio<1'000'000'000'000, 1>;
using peta = ratio<1'000'000'000'000'000, 1>;
using exa = ratio<1'000'000'000'000'000'000, 1>;
using zetta = ratio<1'000'000'000'000'000'000'000, 1>; // see below
using yotta = ratio<1'000'000'000'000'000'000'000'000, 1>; // see below
using ronna = ratio<1'000'000'000'000'000'000'000'000'000, 1>; // see below
using quetta = ratio<1'000'000'000'000'000'000'000'000'000'000, 1>; // see below
}

```

21.5.3 Class template ratio

[ratio.ratio]

```

namespace std {
    template<intmax_t N, intmax_t D = 1> class ratio {
    public:
        static constexpr intmax_t num;
        static constexpr intmax_t den;
        using type = ratio<num, den>;
    };
}

```

¹ If the template argument D is zero or the absolute values of either of the template arguments N and D is not representable by type `intmax_t`, the program is ill-formed.

[Note 1: These rules ensure that infinite ratios are avoided and that for any negative input, there exists a representable value of its absolute value which is positive. This excludes the most negative value. — end note]

- ² The static data members `num` and `den` shall have the following values, where `gcd` represents the greatest common divisor of the absolute values of `N` and `D`:

- (2.1) — `num` shall have the value `sgn(N) * sgn(D) * abs(N) / gcd`.
 (2.2) — `den` shall have the value `abs(D) / gcd`.

21.5.4 Arithmetic on ratios

[ratio.arithmetic]

- ¹ Each of the alias templates `ratio_add`, `ratio_subtract`, `ratio_multiply`, and `ratio_divide` denotes the result of an arithmetic computation on two ratios `R1` and `R2`. With `X` and `Y` computed (in the absence of arithmetic overflow) as specified by Table 65, each alias denotes a `ratio<U, V>` such that `U` is the same as `ratio<X, Y>::num` and `V` is the same as `ratio<X, Y>::den`.
- ² If it is not possible to represent `U` or `V` with `intmax_t`, the program is ill-formed. Otherwise, an implementation should yield correct values of `U` and `V`. If it is not possible to represent `X` or `Y` with `intmax_t`, the program is ill-formed unless the implementation yields correct values of `U` and `V`.

Table 65 — Expressions used to perform ratio arithmetic [tab:ratio.arithmetic]

Type	Value of X	Value of Y
<code>ratio_add<R1, R2></code>	<code>R1::num * R2::den + R2::num * R1::den</code>	<code>R1::den * R2::den</code>
<code>ratio_subtract<R1, R2></code>	<code>R1::num * R2::den - R2::num * R1::den</code>	<code>R1::den * R2::den</code>
<code>ratio_multiply<R1, R2></code>	<code>R1::num * R2::num</code>	<code>R1::den * R2::den</code>
<code>ratio_divide<R1, R2></code>	<code>R1::num * R2::den</code>	<code>R1::den * R2::num</code>

- ³ [Example 1:

```
static_assert(ratio_add<ratio<1, 3>, ratio<1, 6>>::num == 1, "1/3+1/6 == 1/2");
static_assert(ratio_add<ratio<1, 3>, ratio<1, 6>>::den == 2, "1/3+1/6 == 1/2");
static_assert(ratio_multiply<ratio<1, 3>, ratio<3, 2>>::num == 1, "1/3*3/2 == 1/2");
static_assert(ratio_multiply<ratio<1, 3>, ratio<3, 2>>::den == 2, "1/3*3/2 == 1/2");
```

// The following cases may cause the program to be ill-formed under some implementations

```
static_assert(ratio_add<ratio<1, INT_MAX>, ratio<1, INT_MAX>>::num == 2,
  "1/MAX+1/MAX == 2/MAX");
static_assert(ratio_add<ratio<1, INT_MAX>, ratio<1, INT_MAX>>::den == INT_MAX,
  "1/MAX+1/MAX == 2/MAX");
static_assert(ratio_multiply<ratio<1, INT_MAX>, ratio<INT_MAX, 2>>::num == 1,
  "1/MAX * MAX/2 == 1/2");
static_assert(ratio_multiply<ratio<1, INT_MAX>, ratio<INT_MAX, 2>>::den == 2,
  "1/MAX * MAX/2 == 1/2");
```

— end example]

21.5.5 Comparison of ratios

[ratio.comparison]

```
template<class R1, class R2>
struct ratio_equal : bool_constant<R1::num == R2::num && R1::den == R2::den> { };

template<class R1, class R2>
struct ratio_not_equal : bool_constant<!ratio_equal_v<R1, R2>> { };

template<class R1, class R2>
struct ratio_less : bool_constant<see below> { };
```

- ¹ If `R1::num × R2::den` is less than `R2::num × R1::den`, `ratio_less<R1, R2>` shall be derived from `bool_constant<true>`; otherwise it shall be derived from `bool_constant<false>`. Implementations may use other algorithms to compute this relationship to avoid overflow. If overflow occurs, the program is ill-formed.

```

template<class R1, class R2>
    struct ratio_less_equal : bool_constant<!ratio_less_v<R2, R1>> { };

template<class R1, class R2>
    struct ratio_greater : bool_constant<ratio_less_v<R2, R1>> { };

template<class R1, class R2>
    struct ratio_greater_equal : bool_constant<!ratio_less_v<R1, R2>> { };

```

21.5.6 SI types for ratio

[ratio.si]

- ¹ For each of the *typedef-names* *quecto*, *ronto*, *yocto*, *zepto*, *zetta*, *yotta*, *ronna*, and *quetta*, if both of the constants used in its specification are representable by `intmax_t`, the typedef is declared; if either of the constants is not representable by `intmax_t`, the typedef is not declared.

22 General utilities library

[utilities]

22.1 General

[utilities.general]

- ¹ This Clause describes utilities that are generally useful in C++ programs; some of these utilities are used by other elements of the C++ standard library. These utilities are summarized in [Table 66](#).

Table 66 — General utilities library summary [tab:utilities.summary]

	Subclause	Header
22.2	Utility components	<utility>
22.3	Pairs	
22.4	Tuples	<tuple>
22.5	Optional objects	<optional>
22.6	Variants	<variant>
22.7	Storage for any type	<any>
22.8	Expected objects	<expected>
22.9	Fixed-size sequences of bits	<bitset>
22.10	Function objects	<functional>
22.11	Bit manipulation	<bit>

22.2 Utility components

[utility]

22.2.1 Header <utility> synopsis

[utility.syn]

- ¹ The header <utility> contains some basic function and class templates that are used throughout the rest of the library.

```
// all freestanding
#include <compare>           // see 17.12.1
#include <initializer_list>  // see 17.11.2

namespace std {
    // 22.2.2, swap
    template<class T>
        constexpr void swap(T& a, T& b) noexcept(see below);
    template<class T, size_t N>
        constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);

    // 22.2.3, exchange
    template<class T, class U = T>
        constexpr T exchange(T& obj, U&& new_val) noexcept(see below);

    // 22.2.4, forward/move
    template<class T>
        constexpr T&& forward(remove_reference_t<T>& t) noexcept;
    template<class T>
        constexpr T&& forward(remove_reference_t<T>&& t) noexcept;
    template<class T, class U>
        constexpr auto forward_like(U&& x) noexcept -> see below;
    template<class T>
        constexpr remove_reference_t<T>&& move(T&&) noexcept;
    template<class T>
        constexpr conditional_t<
            !is_nothrow_move_constructible_v<T> && is_copy_constructible_v<T>, const T&, T&&>
            move_if_noexcept(T& x) noexcept;
```

```

// 22.2.5, as_const
template<class T>
constexpr add_const_t<T>& as_const(T& t) noexcept;
template<class T>
void as_const(const T&&) = delete;

// 22.2.6, declval
template<class T>
add_rvalue_reference_t<T> declval() noexcept;    // as unevaluated operand

// 22.2.7, integer comparison functions
template<class T, class U>
constexpr bool cmp_equal(T t, U u) noexcept;
template<class T, class U>
constexpr bool cmp_not_equal(T t, U u) noexcept;

template<class T, class U>
constexpr bool cmp_less(T t, U u) noexcept;
template<class T, class U>
constexpr bool cmp_greater(T t, U u) noexcept;
template<class T, class U>
constexpr bool cmp_less_equal(T t, U u) noexcept;
template<class T, class U>
constexpr bool cmp_greater_equal(T t, U u) noexcept;

template<class R, class T>
constexpr bool in_range(T t) noexcept;

// 22.2.8, to_underlying
template<class T>
constexpr underlying_type_t<T> to_underlying(T value) noexcept;

// 22.2.9, undefined behavior
[[noreturn]] void unreachable();
void observable_checkpoint() noexcept;

// 22.2.10, class template constant_wrapper
template<auto X, class = decltype(X)>
struct constant_wrapper;

template<class T>
concept constexpr_param =           // exposition only
    requires { typename constant_wrapper<T::value>; };

struct cw_operators;                 // exposition only

template<auto X>
constexpr auto cw = constant_wrapper<X>{};

// 21.2, compile-time integer sequences
template<class T, T...>
struct integer_sequence;
template<size_t... I>
using index_sequence = integer_sequence<size_t, I...>;

template<class T, T N>
using make_integer_sequence = integer_sequence<T, see below>;
template<size_t N>
using make_index_sequence = make_integer_sequence<size_t, N>;

template<class... T>
using index_sequence_for = make_index_sequence<sizeof...(T)>;

```

```

// 21.2.4, structured binding support
template<class T> struct tuple_size;
template<size_t I, class T> struct tuple_element;

template<class T, T... Values>
    struct tuple_size<integer_sequence<T, Values...>>;

template<size_t I, class T, T... Values>
    struct tuple_element<I, integer_sequence<T, Values...>>;
template<size_t I, class T, T... Values>
    struct tuple_element<I, const integer_sequence<T, Values...>>;

template<size_t I, class T, T... Values>
    constexpr T get(integer_sequence<T, Values...>) noexcept;

// 22.3, class template pair
template<class T1, class T2>
    struct pair;

template<class T1, class T2, class U1, class U2,
        template<class> class TQual, template<class> class UQual>
    requires requires { typename pair<common_reference_t<TQual<T1>, UQual<U1>>,
        common_reference_t<TQual<T2>, UQual<U2>>>; }
struct basic_common_reference<pair<T1, T2>, pair<U1, U2>, TQual, UQual> {
    using type = pair<common_reference_t<TQual<T1>, UQual<U1>>,
        common_reference_t<TQual<T2>, UQual<U2>>>;
};

template<class T1, class T2, class U1, class U2>
    requires requires { typename pair<common_type_t<T1, U1>, common_type_t<T2, U2>>; }
struct common_type<pair<T1, T2>, pair<U1, U2>> {
    using type = pair<common_type_t<T1, U1>, common_type_t<T2, U2>>;
};

// 22.3.3, pair specialized algorithms
template<class T1, class T2, class U1, class U2>
    constexpr bool operator==(const pair<T1, T2>&, const pair<U1, U2>&);
template<class T1, class T2, class U1, class U2>
    constexpr common_comparison_category_t<synth-three-way-result<T1, U1>,
        synth-three-way-result<T2, U2>>
    operator<=>(const pair<T1, T2>&, const pair<U1, U2>&);

template<class T1, class T2>
    constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
template<class T1, class T2>
    constexpr void swap(const pair<T1, T2>& x, const pair<T1, T2>& y)
        noexcept(noexcept(x.swap(y)));

template<class T1, class T2>
    constexpr see below make_pair(T1&&, T2&&);

// 22.3.4, tuple-like access to pair
template<class T1, class T2> struct tuple_size<pair<T1, T2>>;
template<size_t I, class T1, class T2> struct tuple_element<I, pair<T1, T2>>;

template<size_t I, class T1, class T2>
    constexpr tuple_element_t<I, pair<T1, T2>>& get(pair<T1, T2>&) noexcept;
template<size_t I, class T1, class T2>
    constexpr tuple_element_t<I, pair<T1, T2>>&& get(pair<T1, T2>&&) noexcept;
template<size_t I, class T1, class T2>
    constexpr const tuple_element_t<I, pair<T1, T2>>& get(const pair<T1, T2>&) noexcept;
template<size_t I, class T1, class T2>
    constexpr const tuple_element_t<I, pair<T1, T2>>&& get(const pair<T1, T2>&&) noexcept;

```

```

template<class T1, class T2>
    constexpr T1& get(pair<T1, T2>& p) noexcept;
template<class T1, class T2>
    constexpr const T1& get(const pair<T1, T2>& p) noexcept;
template<class T1, class T2>
    constexpr T1&& get(pair<T1, T2>&& p) noexcept;
template<class T1, class T2>
    constexpr const T1&& get(const pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
    constexpr T2& get(pair<T1, T2>& p) noexcept;
template<class T2, class T1>
    constexpr const T2& get(const pair<T1, T2>& p) noexcept;
template<class T2, class T1>
    constexpr T2&& get(pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
    constexpr const T2&& get(const pair<T1, T2>&& p) noexcept;

// 22.3.5, pair piecewise construction
struct piecewise_construct_t {
    explicit piecewise_construct_t() = default;
};
inline constexpr piecewise_construct_t piecewise_construct{};
template<class... Types> class tuple; // defined in <tuple> (22.4.2)

// in-place construction
struct in_place_t {
    explicit in_place_t() = default;
};
inline constexpr in_place_t in_place{};

template<class T>
    struct in_place_type_t {
        explicit in_place_type_t() = default;
    };
template<class T> constexpr in_place_type_t<T> in_place_type{};

template<size_t I>
    struct in_place_index_t {
        explicit in_place_index_t() = default;
    };
template<size_t I> constexpr in_place_index_t<I> in_place_index{};

// 22.6.8, class monostate
struct monostate;

// 22.6.9, monostate relational operators
constexpr bool operator==(monostate, monostate) noexcept;
constexpr strong_ordering operator<=>(monostate, monostate) noexcept;

// 22.6.12, hash support
template<class T> struct hash;
template<> struct hash<monostate>;
}

```

22.2.2 swap

[utility.swap]

```

template<class T>
    constexpr void swap(T& a, T& b) noexcept(see below);

```

- 1 *Constraints:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.
- 2 *Preconditions:* Type `T` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Exchanges values stored in two locations.

4 *Remarks:* The exception specification is equivalent to:

```
is_nothrow_move_constructible_v<T> && is_nothrow_move_assignable_v<T>
```

```
template<class T, size_t N>
```

```
constexpr void swap(T (&a)[N], T (&b)[N]) noexcept(is_nothrow_swappable_v<T>);
```

5 *Constraints:* `is_swappable_v<T>` is true.

6 *Preconditions:* `a[i]` is swappable with (16.4.4.3) `b[i]` for all `i` in the range `[0, N)`.

7 *Effects:* As if by `swap_ranges(a, a + N, b)`.

22.2.3 exchange

[utility.exchange]

```
template<class T, class U = T>
```

```
constexpr T exchange(T& obj, U&& new_val) noexcept(see below);
```

1 *Effects:* Equivalent to:

```
T old_val = std::move(obj);
obj = std::forward<U>(new_val);
return old_val;
```

2 *Remarks:* The exception specification is equivalent to:

```
is_nothrow_move_constructible_v<T> && is_nothrow_assignable_v<T&, U>
```

22.2.4 Forward/move helpers

[forward]

1 The library provides templated helper functions to simplify applying move semantics to an lvalue and to simplify the implementation of forwarding functions. All functions specified in this subclause are signal-safe (17.14.5).

```
template<class T> constexpr T&& forward(remove_reference_t<T>& t) noexcept;
```

```
template<class T> constexpr T&& forward(remove_reference_t<T>&& t) noexcept;
```

2 *Mandates:* For the second overload, `is_lvalue_reference_v<T>` is false.

3 *Returns:* `static_cast<T&&>(t)`.

4 [Example 1:

```
template<class T, class A1, class A2>
shared_ptr<T> factory(A1&& a1, A2&& a2) {
    return shared_ptr<T>(new T(std::forward<A1>(a1), std::forward<A2>(a2)));
}
```

```
struct A {
    A(int&, const double&);
};
```

```
void g() {
    shared_ptr<A> sp1 = factory<A>(2, 1.414); // error: 2 will not bind to int&
    int i = 2;
    shared_ptr<A> sp2 = factory<A>(i, 1.414); // OK
}
```

In the first call to `factory`, `A1` is deduced as `int`, so `2` is forwarded to `A`'s constructor as an rvalue. In the second call to `factory`, `A1` is deduced as `int&`, so `i` is forwarded to `A`'s constructor as an lvalue. In both cases, `A2` is deduced as `double`, so `1.414` is forwarded to `A`'s constructor as an rvalue. — end example]

```
template<class T, class U>
```

```
constexpr auto forward_like(U&& x) noexcept -> see below;
```

5 *Mandates:* `T` is a referenceable type (3.46).

(5.1) — Let `COPY_CONST(A, B)` be `const B` if `A` is a `const` type, otherwise `B`.

(5.2) — Let `OVERRIDE_REF(A, B)` be `remove_reference_t<B>&&` if `A` is an rvalue reference type, otherwise `B&`.

(5.3) — Let `V` be

```
OVERRIDE_REF(T&&, COPY_CONST(remove_reference_t<T>, remove_reference_t<U>))
```

6 *Returns:* `static_cast<V>(x)`.

7 *Remarks:* The return type is V.

8 [Example 2:

```
    struct accessor {
        vector<string>* container;
        decltype(auto) operator[] (this auto&& self, size_t i) {
            return std::forward_like<decltype(self)>((*container)[i]);
        }
    };
    void g() {
        vector v{"a"s, "b"s};
        accessor a{&v};
        string& x = a[0];                // OK, binds to lvalue reference
        string&& y = std::move(a)[0];    // OK, is rvalue reference
        string const&& z = std::move(as_const(a))[1]; // OK, is const&&
        string& w = as_const(a)[1];      // error: will not bind to non-const
    }
    — end example]
```

```
template<class T> constexpr remove_reference_t<T>&& move(T&& t) noexcept;
```

9 *Returns:* `static_cast<remove_reference_t<T>&&>(t)`.

10 [Example 3:

```
    template<class T, class A1>
    shared_ptr<T> factory(A1&& a1) {
        return shared_ptr<T>(new T(std::forward<A1>(a1)));
    }

    struct A {
        A();
        A(const A&);           // copies from lvalues
        A(A&&);               // moves from rvalues
    };

    void g() {
        A a;
        shared_ptr<A> sp1 = factory<A>(a);           // “a” binds to A(const A&)
        shared_ptr<A> sp2 = factory<A>(std::move(a)); // “a” binds to A(A&&)
    }
```

In the first call to `factory`, `A1` is deduced as `A&`, so `a` is forwarded as a non-const lvalue. This binds to the constructor `A(const A&)`, which copies the value from `a`. In the second call to `factory`, because of the call `std::move(a)`, `A1` is deduced as `A`, so `a` is forwarded as an rvalue. This binds to the constructor `A(A&&)`, which moves the value from `a`. — end example]

```
template<class T> constexpr conditional_t<
    !is_nothrow_move_constructible_v<T> && is_copy_constructible_v<T>, const T&, T&&>
    move_if_noexcept(T& x) noexcept;
```

11 *Returns:* `std::move(x)`.

22.2.5 Function template `as_const`

[utility.as.const]

```
template<class T> constexpr add_const_t<T>& as_const(T& t) noexcept;
```

1 *Returns:* `t`.

22.2.6 Function template `declval`

[declval]

1 The library provides the function template `declval` to simplify the definition of expressions which occur as unevaluated operands (7.2.3).

```
template<class T> add_rvalue_reference_t<T> declval() noexcept;    // as unevaluated operand
```

2 *Mandates:* This function is not odr-used (6.3).

3 *Remarks:* The template parameter `T` of `declval` may be an incomplete type.

4 [*Example 1:*

```
    template<class To, class From> decltype(static_cast<To>(declval<From>())) convert(From&&);
```

declares a function template `convert` which only participates in overload resolution if the type `From` can be explicitly converted to type `To`. For another example see class template `common_type` (21.3.8.7). — *end example*]

22.2.7 Integer comparison functions

[utility.intcmp]

```
template<class T, class U>
constexpr bool cmp_equal(T t, U u) noexcept;
```

1 *Mandates:* Each of `T` and `U` is a signed or unsigned integer type (6.9.3).

2 *Effects:* Equivalent to:

```
    using UT = make_unsigned_t<T>;
    using UU = make_unsigned_t<U>;
    if constexpr (is_signed_v<T> == is_signed_v<U>)
        return t == u;
    else if constexpr (is_signed_v<T>)
        return t < 0 ? false : UT(t) == u;
    else
        return u < 0 ? false : t == UU(u);
```

```
template<class T, class U>
constexpr bool cmp_not_equal(T t, U u) noexcept;
```

3 *Effects:* Equivalent to: `return !cmp_equal(t, u);`

```
template<class T, class U>
constexpr bool cmp_less(T t, U u) noexcept;
```

4 *Mandates:* Each of `T` and `U` is a signed or unsigned integer type (6.9.3).

5 *Effects:* Equivalent to:

```
    using UT = make_unsigned_t<T>;
    using UU = make_unsigned_t<U>;
    if constexpr (is_signed_v<T> == is_signed_v<U>)
        return t < u;
    else if constexpr (is_signed_v<T>)
        return t < 0 ? true : UT(t) < u;
    else
        return u < 0 ? false : t < UU(u);
```

```
template<class T, class U>
constexpr bool cmp_greater(T t, U u) noexcept;
```

6 *Effects:* Equivalent to: `return cmp_less(u, t);`

```
template<class T, class U>
constexpr bool cmp_less_equal(T t, U u) noexcept;
```

7 *Effects:* Equivalent to: `return !cmp_greater(t, u);`

```
template<class T, class U>
constexpr bool cmp_greater_equal(T t, U u) noexcept;
```

8 *Effects:* Equivalent to: `return !cmp_less(t, u);`

```
template<class R, class T>
constexpr bool in_range(T t) noexcept;
```

9 *Mandates:* Each of `T` and `R` is a signed or unsigned integer type (6.9.3).

10 *Effects:* Equivalent to:

```
    return cmp_greater_equal(t, numeric_limits<R>::min()) &&
           cmp_less_equal(t, numeric_limits<R>::max());
```

11 [*Note 1:* These function templates cannot be used to compare `byte`, `char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and `bool`. — *end note*]

22.2.8 Function template to_underlying**[utility.underlying]**

```
template<class T>
constexpr underlying_type_t<T> to_underlying(T value) noexcept;
1   Returns: static_cast<underlying_type_t<T>>(value).
```

22.2.9 Undefined behavior**[utility.undefined]**

```
[[noreturn]] void unreachable();
```

1 *Preconditions:* false is true.

[Note 1: This precondition cannot be satisfied, thus the behavior of calling `unreachable` is undefined. — end note]

2 [Example 1:

```
    int f(int x) {
        switch (x) {
            case 0:
            case 1:
                return x;
            default:
                std::unreachable();
        }
    }
    int a = f(1);           // OK, a has value 1
    int b = f(3);           // undefined behavior
    — end example]
```

```
void observable_checkpoint() noexcept;
```

3 *Effects:* Establishes an observable checkpoint (4.1.2).

22.2.10 Class template constant_wrapper**[const.wrap.class]**

```
namespace std {
    struct cw_operators {                                // exposition only
        // unary operators
        template<constexpr-param T>
            friend constexpr auto operator+(T) noexcept -> constant_wrapper<(+T::value)>
            { return {}; }
        template<constexpr-param T>
            friend constexpr auto operator-(T) noexcept -> constant_wrapper<(-T::value)>
            { return {}; }
        template<constexpr-param T>
            friend constexpr auto operator~(T) noexcept -> constant_wrapper<(~T::value)>
            { return {}; }
        template<constexpr-param T>
            friend constexpr auto operator!(T) noexcept -> constant_wrapper<(!T::value)>
            { return {}; }
        template<constexpr-param T>
            friend constexpr auto operator&(T) noexcept -> constant_wrapper<(&T::value)>
            { return {}; }
        template<constexpr-param T>
            friend constexpr auto operator*(T) noexcept -> constant_wrapper<(*T::value)>
            { return {}; }

        // binary operators
        template<constexpr-param L, constexpr-param R>
            friend constexpr auto operator+(L, R) noexcept -> constant_wrapper<(L::value + R::value)>
            { return {}; }
        template<constexpr-param L, constexpr-param R>
            friend constexpr auto operator-(L, R) noexcept -> constant_wrapper<(L::value - R::value)>
            { return {}; }
    };
}
```



```

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator*(L, R) noexcept -> constant_wrapper<(L::value * R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator/(L, R) noexcept -> constant_wrapper<(L::value / R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator%(L, R) noexcept -> constant_wrapper<(L::value % R::value)>
    { return {}; }

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<<(L, R) noexcept -> constant_wrapper<(L::value << R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>>(L, R) noexcept -> constant_wrapper<(L::value >> R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator&(L, R) noexcept -> constant_wrapper<(L::value & R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator|(L, R) noexcept -> constant_wrapper<(L::value | R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator^(L, R) noexcept -> constant_wrapper<(L::value ^ R::value)>
    { return {}; }

template<constexpr-param L, constexpr-param R>
    requires (!is_constructible_v<bool, decltype(L::value)> ||
              !is_constructible_v<bool, decltype(R::value)>)
    friend constexpr auto operator&&(L, R) noexcept
        -> constant_wrapper<(L::value && R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    requires (!is_constructible_v<bool, decltype(L::value)> ||
              !is_constructible_v<bool, decltype(R::value)>)
    friend constexpr auto operator||(L, R) noexcept
        -> constant_wrapper<(L::value || R::value)>
    { return {}; }

// comparisons
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<=(L, R) noexcept
        -> constant_wrapper<(L::value <= R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<(L, R) noexcept -> constant_wrapper<(L::value < R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator<=(L, R) noexcept -> constant_wrapper<(L::value <= R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator==(L, R) noexcept -> constant_wrapper<(L::value == R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator!=(L, R) noexcept -> constant_wrapper<(L::value != R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>(L, R) noexcept -> constant_wrapper<(L::value > R::value)>
    { return {}; }
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator>=(L, R) noexcept -> constant_wrapper<(L::value >= R::value)>
    { return {}; }

```

```

template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator,(L, R) noexcept = delete;
template<constexpr-param L, constexpr-param R>
    friend constexpr auto operator->*(L, R) noexcept -> constant_wrapper<L::value->*(R::value)>
        { return {}; }

// pseudo-mutators
template<constexpr-param T>
    constexpr auto operator++(this T) noexcept
        -> constant_wrapper<(+T::value)> { return {}; }
template<constexpr-param T>
    constexpr auto operator++(this T, int) noexcept
        -> constant_wrapper<(T::value++)> { return {}; }
template<constexpr-param T>
    constexpr auto operator--(this T) noexcept
        -> constant_wrapper<--T::value> { return {}; }
template<constexpr-param T>
    constexpr auto operator--(this T, int) noexcept
        -> constant_wrapper<(T::value--)> { return {}; }

template<constexpr-param T, constexpr-param R>
    constexpr auto operator+=(this T, R) noexcept
        -> constant_wrapper<(T::value += R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator-=(this T, R) noexcept
        -> constant_wrapper<(T::value -= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator*=(this T, R) noexcept
        -> constant_wrapper<(T::value *= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator/=(this T, R) noexcept
        -> constant_wrapper<(T::value /= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator%=(this T, R) noexcept
        -> constant_wrapper<(T::value %= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator&=(this T, R) noexcept
        -> constant_wrapper<(T::value &= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator|=(this T, R) noexcept
        -> constant_wrapper<(T::value |= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator^=(this T, R) noexcept
        -> constant_wrapper<(T::value ^= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator<=(this T, R) noexcept
        -> constant_wrapper<(T::value <= R::value)> { return {}; }
template<constexpr-param T, constexpr-param R>
    constexpr auto operator>=(this T, R) noexcept
        -> constant_wrapper<(T::value >= R::value)> { return {}; }
};

template<auto X, class T = decltype(X)>
struct constant_wrapper : cw-operators {
    static constexpr decltype(auto) value = (X);
    using type = constant_wrapper;
    using value_type = decltype(X);

    template<constexpr-param R>
        constexpr auto operator=(R) const noexcept
            -> constant_wrapper<(value = R::value)> { return {}; }

    constexpr operator decltype(value)() const noexcept { return value; }
};

```

```

    template<class... Args>
        static constexpr decltype(auto) operator()(Args&&... args) noexcept(see below);
    template<class... Args>
        static constexpr decltype(auto) operator[](Args&&... args) noexcept(see below);
};
}

```

- 1 The class template `constant_wrapper` aids in metaprogramming by ensuring that the evaluation of expressions comprised entirely of `constant_wrapper` are core constant expressions (7.7.2), regardless of the context in which they appear. In particular, this enables use of `constant_wrapper` values that are passed as arguments to constexpr functions to be used in constant expressions.
- 2 If a specialization of `constant_wrapper` is instantiated with a type `T` such that `is_same_v<T, value_type>` is `false`, the program is ill-formed.

[Note 1: The second template parameter to `constant_wrapper` is present to aid argument-dependent lookup (6.5.4) in finding overloads for which `constant_wrapper`'s wrapped value is a suitable argument, but for which the `constant_wrapper` itself is not. — end note]

- 3 [Example 1:

```

    constexpr auto initial_phase(auto quantity_1, auto quantity_2) {
        return quantity_1 + quantity_2;
    }

    constexpr auto middle_phase(auto tbd) {
        return tbd;
    }

    void final_phase(auto gathered, auto available) {
        if constexpr (gathered == available)
            std::cout << "Profit!\n";
    }

    void impeccable_underground_planning() {
        auto gathered_quantity = middle_phase(initial_phase(std::cw<42>, std::cw<13>));
        static_assert(gathered_quantity == 55);
        auto all_available = std::cw<55>;
        final_phase(gathered_quantity, all_available);
    }

    void deeply_flawed_underground_planning() {
        constexpr auto gathered_quantity = middle_phase(initial_phase(42, 13));
        constexpr auto all_available = 55;
        final_phase(gathered_quantity, all_available); // error: gathered == available
                                                         // is not a constant expression
    }

```

— end example]

```

template<class... Args>
    static constexpr decltype(auto) operator()(Args&&... args) noexcept(see below);

```

- 4 Let *call-expr* be `constant_wrapper<INVOKE(value, remove_cvref_t<Args>::value...)>{}` if all types in `remove_cvref_t<Args>...` satisfy *constexpr-param* and `constant_wrapper<INVOKE(value, remove_cvref_t<Args>::value...)>` is a valid type, otherwise let *call-expr* be `INVOKE(value, std::forward<Args>(args)...)...`.
- 5 *Constraints:* *call-expr* is a valid expression.
- 6 *Effects:* Equivalent to: `return call-expr;`
- 7 *Remarks:* The exception specification is equivalent to `noexcept(call-expr)`.

```

template<class... Args>
    static constexpr decltype(auto) operator[](Args&&... args) noexcept(see below);

```

- 8 Let *subscr-expr* be `constant_wrapper<value[remove_cvref_t<Args>::value...]>{}` if all types in `remove_cvref_t<Args>...` satisfy *constexpr-param* and `constant_wrapper<value[remove_cv-`

`ref_t<Args>::value...]` is a valid type, otherwise let *subscr-expr* be `value[std::forward<Args>(args)...]`.

9 *Constraints:* *subscr-expr* is a valid expression.

10 *Effects:* Equivalent to: `return subscr-expr;`

11 *Remarks:* The exception specification is equivalent to `noexcept(subscr-expr)`.

22.3 Pairs

[pairs]

22.3.1 General

[pairs.general]

1 The library provides a template for heterogeneous pairs of values. The library also provides a matching function template to simplify their construction and several templates that provide access to `pair` objects as if they were tuple objects (see 22.4.7 and 22.4.8).

22.3.2 Class template `pair`

[pairs.pair]

```
namespace std {
    template<class T1, class T2>
    struct pair {
        using first_type = T1;
        using second_type = T2;

        T1 first;
        T2 second;

        pair(const pair&) = default;
        pair(pair&&) = default;
        constexpr explicit(see below) pair();
        constexpr explicit(see below) pair(const T1& x, const T2& y);
        template<class U1 = T1, class U2 = T2>
            constexpr explicit(see below) pair(U1&& x, U2&& y);
        template<class U1, class U2>
            constexpr explicit(see below) pair(pair<U1, U2>& p);
        template<class U1, class U2>
            constexpr explicit(see below) pair(const pair<U1, U2>& p);
        template<class U1, class U2>
            constexpr explicit(see below) pair(pair<U1, U2>&& p);
        template<class U1, class U2>
            constexpr explicit(see below) pair(const pair<U1, U2>&& p);
        template<pair-like P>
            constexpr explicit(see below) pair(P&& p);
        template<class... Args1, class... Args2>
            constexpr pair(piecewise_construct_t,
                tuple<Args1...> first_args, tuple<Args2...> second_args);

        constexpr pair& operator=(const pair& p);
        constexpr const pair& operator=(const pair& p) const;
        template<class U1, class U2>
            constexpr pair& operator=(const pair<U1, U2>& p);
        template<class U1, class U2>
            constexpr const pair& operator=(const pair<U1, U2>& p) const;
        constexpr pair& operator=(pair&& p) noexcept(see below);
        constexpr const pair& operator=(pair&& p) const;
        template<class U1, class U2>
            constexpr pair& operator=(pair<U1, U2>&& p);
        template<class U1, class U2>
            constexpr const pair& operator=(pair<U1, U2>&& p) const;
        template<pair-like P>
            constexpr pair& operator=(P&& p);
        template<pair-like P>
            constexpr const pair& operator=(P&& p) const;
    };
}
```

```
constexpr void swap(pair& p) noexcept(see below);
constexpr void swap(const pair& p) const noexcept(see below);
};
```

```
template<class T1, class T2>
    pair(T1, T2) -> pair<T1, T2>;
}
```

- 1 Member functions of `pair` do not throw exceptions unless one of the element-wise operations specified to be called for that operation throws an exception.
- 2 The defaulted move and copy constructor, respectively, of `pair` is a constexpr function if and only if all required element-wise initializations for move and copy, respectively, would be constexpr-suitable (9.2.6).
- 3 If (`is_trivially_destructible_v<T1>` && `is_trivially_destructible_v<T2>`) is true, then the destructor of `pair` is trivial.
- 4 `pair<T, U>` is a structural type (13.2) if `T` and `U` are both structural types. Two values `p1` and `p2` of type `pair<T, U>` are template-argument-equivalent (13.6) if and only if `p1.first` and `p2.first` are template-argument-equivalent and `p1.second` and `p2.second` are template-argument-equivalent.

```
constexpr explicit(see below) pair();
```

5 *Constraints:*

- (5.1) — `is_default_constructible_v<T1>` is true and
- (5.2) — `is_default_constructible_v<T2>` is true.

6 *Effects:* Value-initializes `first` and `second`.

7 *Remarks:* The expression inside `explicit` evaluates to true if and only if either `T1` or `T2` is not implicitly default-constructible.

[Note 1: This behavior can be implemented with a trait that checks whether a `const T1&` or a `const T2&` can be initialized with {}. — end note]

```
constexpr explicit(see below) pair(const T1& x, const T2& y);
```

8 *Constraints:*

- (8.1) — `is_copy_constructible_v<T1>` is true and
- (8.2) — `is_copy_constructible_v<T2>` is true.

9 *Effects:* Initializes `first` with `x` and `second` with `y`.

10 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const T1&, T1> || !is_convertible_v<const T2&, T2>
```

```
template<class U1 = T1, class U2 = T2> constexpr explicit(see below) pair(U1&& x, U2&& y);
```

11 *Constraints:*

- (11.1) — `is_constructible_v<T1, U1>` is true and
- (11.2) — `is_constructible_v<T2, U2>` is true.

12 *Effects:* Initializes `first` with `std::forward<U1>(x)` and `second` with `std::forward<U2>(y)`.

13 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U1, T1> || !is_convertible_v<U2, T2>
```

This constructor is defined as deleted if `reference_constructs_from_temporary_v<first_type, U1&&>` is true or `reference_constructs_from_temporary_v<second_type, U2&&>` is true.

```
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>& p);
template<class U1, class U2> constexpr explicit(see below) pair(pair<U1, U2>&& p);
template<class U1, class U2> constexpr explicit(see below) pair(const pair<U1, U2>&& p);
template<pair-like P> constexpr explicit(see below) pair(P&& p);
```

14 Let `FWD(u)` be `static_cast<decltype(u)>(u)`.

15 *Constraints:*

- (15.1) — For the last overload, `remove_cvref_t<P>` is not a specialization of `ranges::subrange`,
- (15.2) — `is_constructible_v<T1, decltype(get<0>(FWD(p)))>` is true, and
- (15.3) — `is_constructible_v<T2, decltype(get<1>(FWD(p)))>` is true.

16 *Effects:* Initializes `first` with `get<0>(FWD(p))` and `second` with `get<1>(FWD(p))`.

17 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<decltype(get<0>(FWD(p))), T1> ||
!is_convertible_v<decltype(get<1>(FWD(p))), T2>
```

The constructor is defined as deleted if

```
reference_constructs_from_temporary_v<first_type, decltype(get<0>(FWD(p)))> ||
reference_constructs_from_temporary_v<second_type, decltype(get<1>(FWD(p)))>
```

is true.

```
template<class... Args1, class... Args2>
constexpr pair(piecewise_construct_t,
               tuple<Args1...> first_args, tuple<Args2...> second_args);
```

18 *Mandates:*

- (18.1) — `is_constructible_v<T1, Args1...>` is true and
- (18.2) — `is_constructible_v<T2, Args2...>` is true.

19 *Effects:* Initializes `first` with arguments of types `Args1...` obtained by forwarding the elements of `first_args` and initializes `second` with arguments of types `Args2...` obtained by forwarding the elements of `second_args`. (Here, forwarding an element `x` of type `U` within a `tuple` object means calling `std::forward<U>(x)`.) This form of construction, whereby constructor arguments for `first` and `second` are each provided in a separate `tuple` object, is called *piecewise construction*.

[Note 2: If a data member of `pair` is of reference type and its initialization binds it to a temporary object, the program is ill-formed (11.9.3). — end note]

```
constexpr pair& operator=(const pair& p);
```

20 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

21 *Returns:* `*this`.

22 *Remarks:* This operator is defined as deleted unless `is_copy_assignable_v<T1>` is true and `is_copy_assignable_v<T2>` is true.

```
constexpr const pair& operator=(const pair& p) const;
```

23 *Constraints:*

- (23.1) — `is_copy_assignable_v<const T1>` is true and
- (23.2) — `is_copy_assignable_v<const T2>` is true.

24 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

25 *Returns:* `*this`.

```
template<class U1, class U2> constexpr pair& operator=(const pair<U1, U2>& p);
```

26 *Constraints:*

- (26.1) — `is_assignable_v<T1&, const U1&>` is true and
- (26.2) — `is_assignable_v<T2&, const U2&>` is true.

27 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

28 *Returns:* `*this`.

```
template<class U1, class U2> constexpr const pair& operator=(const pair<U1, U2>& p) const;
```

29 *Constraints:*

- (29.1) — `is_assignable_v<const T1&, const U1&>` is true, and
- (29.2) — `is_assignable_v<const T2&, const U2&>` is true.

30 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

31 *Returns:* `*this`.

```
constexpr pair& operator=(pair&& p) noexcept(see below);
```

32 *Constraints:*

(32.1) — `is_move_assignable_v<T1>` is true and

(32.2) — `is_move_assignable_v<T2>` is true.

33 *Effects:* Assigns `std::forward<T1>(p.first)` to `first` and `std::forward<T2>(p.second)` to `second`.

34 *Returns:* `*this`.

35 *Remarks:* The exception specification is equivalent to:

```
is_nothrow_move_assignable_v<T1> && is_nothrow_move_assignable_v<T2>
```

```
constexpr const pair& operator=(pair&& p) const;
```

36 *Constraints:*

(36.1) — `is_assignable_v<const T1&, T1>` is true and

(36.2) — `is_assignable_v<const T2&, T2>` is true.

37 *Effects:* Assigns `std::forward<T1>(p.first)` to `first` and `std::forward<T2>(p.second)` to `second`.

38 *Returns:* `*this`.

```
template<class U1, class U2> constexpr pair& operator=(pair<U1, U2>&& p);
```

39 *Constraints:*

(39.1) — `is_assignable_v<T1&, U1>` is true and

(39.2) — `is_assignable_v<T2&, U2>` is true.

40 *Effects:* Assigns `std::forward<U1>(p.first)` to `first` and `std::forward<U2>(p.second)` to `second`.

41 *Returns:* `*this`.

```
template<pair-like P> constexpr pair& operator=(P&& p);
```

42 *Constraints:*

(42.1) — *different-from*<P, pair> (25.5.2) is true,

(42.2) — `remove_cvref_t<P>` is not a specialization of `ranges::subrange`,

(42.3) — `is_assignable_v<T1&, decltype(get<0>(std::forward<P>(p)))>` is true, and

(42.4) — `is_assignable_v<T2&, decltype(get<1>(std::forward<P>(p)))>` is true.

43 *Effects:* Assigns `get<0>(std::forward<P>(p))` to `first` and `get<1>(std::forward<P>(p))` to `second`.

44 *Returns:* `*this`.

```
template<pair-like P> constexpr const pair& operator=(P&& p) const;
```

45 *Constraints:*

(45.1) — *different-from*<P, pair> (25.5.2) is true,

(45.2) — `remove_cvref_t<P>` is not a specialization of `ranges::subrange`,

(45.3) — `is_assignable_v<const T1&, decltype(get<0>(std::forward<P>(p)))>` is true, and

(45.4) — `is_assignable_v<const T2&, decltype(get<1>(std::forward<P>(p)))>` is true.

46 *Effects:* Assigns `get<0>(std::forward<P>(p))` to `first` and `get<1>(std::forward<P>(p))` to `second`.

47 *Returns:* `*this`.

```
template<class U1, class U2> constexpr const pair& operator=(pair<U1, U2>&& p) const;
```

48 *Constraints:*

(48.1) — `is_assignable_v<const T1&, U1>` is true, and

- (48.2) — `is_assignable_v<const T2&, U2>` is true.
- 49 *Effects:* Assigns `std::forward<U1>(p.first)` to `first` and `std::forward<U2>(u.second)` to `second`.
- 50 *Returns:* `*this`.
- ```
constexpr void swap(pair& p) noexcept(see below);
constexpr void swap(const pair& p) const noexcept(see below);
```
- 51 *Mandates:*
- (51.1) — For the first overload, `is_swappable_v<T1>` is true and `is_swappable_v<T2>` is true.
- (51.2) — For the second overload, `is_swappable_v<const T1>` is true and `is_swappable_v<const T2>` is true.
- 52 *Preconditions:* `first` is swappable with (16.4.4.3) `p.first` and `second` is swappable with `p.second`.
- 53 *Effects:* Swaps `first` with `p.first` and `second` with `p.second`.
- 54 *Remarks:* The exception specification is equivalent to:
- (54.1) — `is_nothrow_swappable_v<T1> && is_nothrow_swappable_v<T2>` for the first overload, and
- (54.2) — `is_nothrow_swappable_v<const T1> && is_nothrow_swappable_v<const T2>` for the second overload.

### 22.3.3 Specialized algorithms

[pairs.spec]

- ```
template<class T1, class T2, class U1, class U2>
constexpr bool operator==(const pair<T1, T2>& x, const pair<U1, U2>& y);
```
- 1 *Constraints:* `x.first == y.first` and `x.second == y.second` are valid expressions and each of `decltype(x.first == y.first)` and `decltype(x.second == y.second)` models *boolean-testable*.
- 2 *Returns:* `x.first == y.first && x.second == y.second`.
- ```
template<class T1, class T2, class U1, class U2>
constexpr common_comparison_category_t<synth-three-way-result<T1, U1>,
synth-three-way-result<T2, U2>>
operator<=>(const pair<T1, T2>& x, const pair<U1, U2>& y);
```
- 3 *Effects:* Equivalent to:
- ```
if (auto c = synth-three-way(x.first, y.first); c != 0) return c;
return synth-three-way(x.second, y.second);
```
- ```
template<class T1, class T2>
constexpr void swap(pair<T1, T2>& x, pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
template<class T1, class T2>
constexpr void swap(const pair<T1, T2>& x, const pair<T1, T2>& y) noexcept(noexcept(x.swap(y)));
```
- 4 *Constraints:*
- (4.1) — For the first overload, `is_swappable_v<T1>` is true and `is_swappable_v<T2>` is true.
- (4.2) — For the second overload, `is_swappable_v<const T1>` is true and `is_swappable_v<const T2>` is true.
- 5 *Effects:* Equivalent to `x.swap(y)`.
- ```
template<class T1, class T2>
constexpr pair<unwrap_ref_decay_t<T1>, unwrap_ref_decay_t<T2>> make_pair(T1&& x, T2&& y);
```
- 6 *Returns:*
- ```
pair<unwrap_ref_decay_t<T1>,
unwrap_ref_decay_t<T2>>(std::forward<T1>(x), std::forward<T2>(y))
```
- 7 [Example 1: In place of:
- ```
return pair<int, double>(5, 3.1415926); // explicit types
```
- a C++ program may contain:
- ```
return make_pair(5, 3.1415926); // types are deduced
```



— *end example*]

### 22.3.4 Tuple-like access to pair

[pair.astuple]

```
template<class T1, class T2>
 struct tuple_size<pair<T1, T2>> : integral_constant<size_t, 2> { };
```

```
template<size_t I, class T1, class T2>
 struct tuple_element<I, pair<T1, T2>> {
 using type = see below ;
 };
```

1     *Mandates:*  $I < 2$ .

2     *Result:* The type T1 if I is 0, otherwise the type T2.

```
template<size_t I, class T1, class T2>
 constexpr tuple_element_t<I, pair<T1, T2>>& get(pair<T1, T2>& p) noexcept;
template<size_t I, class T1, class T2>
 constexpr const tuple_element_t<I, pair<T1, T2>>& get(const pair<T1, T2>& p) noexcept;
template<size_t I, class T1, class T2>
 constexpr tuple_element_t<I, pair<T1, T2>>&& get(pair<T1, T2>&& p) noexcept;
template<size_t I, class T1, class T2>
 constexpr const tuple_element_t<I, pair<T1, T2>>&& get(const pair<T1, T2>&& p) noexcept;
```

3     *Mandates:*  $I < 2$ .

4     *Returns:*

(4.1)     — If I is 0, returns a reference to p.first.

(4.2)     — If I is 1, returns a reference to p.second.

```
template<class T1, class T2>
 constexpr T1& get(pair<T1, T2>& p) noexcept;
template<class T1, class T2>
 constexpr const T1& get(const pair<T1, T2>& p) noexcept;
template<class T1, class T2>
 constexpr T1&& get(pair<T1, T2>&& p) noexcept;
template<class T1, class T2>
 constexpr const T1&& get(const pair<T1, T2>&& p) noexcept;
```

5     *Mandates:* T1 and T2 are distinct types.

6     *Returns:* A reference to p.first.

```
template<class T2, class T1>
 constexpr T2& get(pair<T1, T2>& p) noexcept;
template<class T2, class T1>
 constexpr const T2& get(const pair<T1, T2>& p) noexcept;
template<class T2, class T1>
 constexpr T2&& get(pair<T1, T2>&& p) noexcept;
template<class T2, class T1>
 constexpr const T2&& get(const pair<T1, T2>&& p) noexcept;
```

7     *Mandates:* T1 and T2 are distinct types.

8     *Returns:* A reference to p.second.

### 22.3.5 Piecewise construction

[pair.piecewise]

```
struct piecewise_construct_t {
 explicit piecewise_construct_t() = default;
};
inline constexpr piecewise_construct_t piecewise_construct{};
```

1 The struct `piecewise_construct_t` is an empty class type used as a unique type to disambiguate constructor and function overloading. Specifically, `pair` has a constructor with `piecewise_construct_t` as the first argument, immediately followed by two `tuple` (22.4) arguments used for piecewise construction of the elements of the `pair` object.

## 22.4 Tuples

[tuple]

### 22.4.1 General

[tuple.general]

- <sup>1</sup> Subclause 22.4 describes the tuple library that provides a tuple type as the class template `tuple` that can be instantiated with any number of arguments. Each template argument specifies the type of an element in the tuple. Consequently, tuples are heterogeneous, fixed-size collections of values. A specialization of `tuple` with two arguments behaves similarly to a specialization of `pair` with the same two arguments. See 22.3.
- <sup>2</sup> In addition to being available via inclusion of the `<tuple>` header, `ignore` (22.4.2) is available when `<utility>` (22.2) is included.

### 22.4.2 Header `<tuple>` synopsis

[tuple.syn]

```

// all freestanding
#include <compare> // see 17.12.1

namespace std {
 // 22.4.4, class template tuple
 template<class... Types>
 class tuple;

 // 22.4.3, concept tuple-like
 template<class T>
 concept tuple-like = see below; // exposition only
 template<class T>
 concept pair-like = // exposition only
 tuple-like<T> && tuple_size_v<remove_cvref_t<T>> == 2;

 // 22.4.10, common reference related specializations
 template<tuple-like TTuple, tuple-like UTuple,
 template<class> class TQual, template<class> class UQual>
 struct basic_common_reference<TTuple, UTuple, TQual, UQual>;
 template<tuple-like TTuple, tuple-like UTuple>
 struct common_type<TTuple, UTuple>;

 // ignore
 struct ignore_type { // exposition only
 constexpr const ignore_type&
 operator=(const auto &) const noexcept { return *this; }
 };
 inline constexpr ignore_type ignore;

 // 22.4.5, tuple creation functions
 template<class... TTypes>
 constexpr tuple<unwrap_ref_decay_t<TTypes>...> make_tuple(TTypes&&...);

 template<class... TTypes>
 constexpr tuple<TTypes&&...> forward_as_tuple(TTypes&&...) noexcept;

 template<class... TTypes>
 constexpr tuple<TTypes&&...> tie(TTypes&&...) noexcept;

 template<tuple-like... Tuples>
 constexpr tuple<CTypes...> tuple_cat(Tuples&&...);

 // 22.4.6, calling a function with a tuple of arguments
 template<class F, tuple-like Tuple>
 constexpr apply_result_t<F, Tuple> apply(F&& f, Tuple&& t)
 noexcept(is_nothrow_applicable_v<F, Tuple>);

 template<class T, tuple-like Tuple>
 constexpr T make_from_tuple(Tuple&& t);

 // 22.4.7, tuple helper classes
 template<class T> struct tuple_size; // not defined

```

```

template<class T> struct tuple_size<const T>;

template<class... Types> struct tuple_size<tuple<Types...>>;

template<size_t I, class T> struct tuple_element; // not defined
template<size_t I, class T> struct tuple_element<I, const T>;

template<size_t I, class... Types>
 struct tuple_element<I, tuple<Types...>>;

template<size_t I, class T>
 using tuple_element_t = tuple_element<I, T>::type;

// 22.4.8, element access
template<size_t I, class... Types>
 constexpr tuple_element_t<I, tuple<Types...>>& get(tuple<Types...>&) noexcept;
template<size_t I, class... Types>
 constexpr tuple_element_t<I, tuple<Types...>>&& get(tuple<Types...>&&) noexcept;
template<size_t I, class... Types>
 constexpr const tuple_element_t<I, tuple<Types...>>& get(const tuple<Types...>&) noexcept;
template<size_t I, class... Types>
 constexpr const tuple_element_t<I, tuple<Types...>>&& get(const tuple<Types...>&&) noexcept;
template<class T, class... Types>
 constexpr T& get(tuple<Types...>& t) noexcept;
template<class T, class... Types>
 constexpr T&& get(tuple<Types...>&& t) noexcept;
template<class T, class... Types>
 constexpr const T& get(const tuple<Types...>& t) noexcept;
template<class T, class... Types>
 constexpr const T&& get(const tuple<Types...>&& t) noexcept;

// 22.4.9, relational operators
template<class... TTypes, class... UTypes>
 constexpr bool operator==(const tuple<TTypes...>&, const tuple<UTypes...>&);
template<class... TTypes, tuple-like UTuple>
 constexpr bool operator==(const tuple<TTypes...>&, const UTuple&);
template<class... TTypes, class... UTypes>
 constexpr common_comparison_category_t<synth-three-way-result<TTypes, UTypes>...>
 operator<=(const tuple<TTypes...>&, const tuple<UTypes...>&);
template<class... TTypes, tuple-like UTuple>
 constexpr see below operator<=(const tuple<TTypes...>&, const UTuple&);

// 22.4.11, allocator-related traits
template<class... Types, class Alloc>
 struct uses_allocator<tuple<Types...>, Alloc>;

// 22.4.12, specialized algorithms
template<class... Types>
 constexpr void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
template<class... Types>
 constexpr void swap(const tuple<Types...>& x, const tuple<Types...>& y) noexcept(see below);

// 22.4.7, tuple helper classes
template<class T>
 constexpr size_t tuple_size_v = tuple_size<T>::value;
}

```

### 22.4.3 Concept *tuple-like*

[*tuple.like*]

```

template<class T>
 concept tuple-like = see below; // exposition only

```

- <sup>1</sup> A type *T* models and satisfies the exposition-only concept *tuple-like* if `remove_cvref_t<T>` is a specialization of `array`, `complex`, `pair`, `tuple`, or `ranges::subrange`.

## 22.4.4 Class template tuple

[tuple(tuple)]

## 22.4.4.1 General

[tuple(tuple.general)]

```

namespace std {
 template<class... Types>
 class tuple {
 public:
 // 22.4.4.2, tuple construction
 constexpr explicit(see below) tuple();
 constexpr explicit(see below) tuple(const Types&...) // only if sizeof...(Types) >= 1
 noexcept(see below);
 template<class... UTypes>
 constexpr explicit(see below) tuple(UTypes&&...) // only if sizeof...(Types) >= 1
 noexcept(see below);

 tuple(const tuple&) = default;
 tuple(tuple&&) = default;

 template<class... UTypes>
 constexpr explicit(see below) tuple(tuple<UTypes...>&);
 template<class... UTypes>
 constexpr explicit(see below) tuple(const tuple<UTypes...>&);
 template<class... UTypes>
 constexpr explicit(see below) tuple(tuple<UTypes...>&&);
 template<class... UTypes>
 constexpr explicit(see below) tuple(const tuple<UTypes...>&&);

 template<class U1, class U2>
 constexpr explicit(see below) tuple(pair<U1, U2>&); // only if sizeof...(Types) == 2
 template<class U1, class U2>
 constexpr explicit(see below) tuple(const pair<U1, U2>&); // only if sizeof...(Types) == 2
 template<class U1, class U2>
 constexpr explicit(see below) tuple(pair<U1, U2>&&); // only if sizeof...(Types) == 2
 template<class U1, class U2>
 constexpr explicit(see below) tuple(const pair<U1, U2>&&); // only if sizeof...(Types) == 2

 template<tuple-like UTuple>
 constexpr explicit(see below) tuple(UTuple&&);

 // allocator-extended constructors
 template<class Alloc>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a);
 template<class Alloc>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, const Types&...);
 template<class Alloc, class... UTypes>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
 template<class Alloc>
 constexpr tuple(allocator_arg_t, const Alloc& a, const tuple&);
 template<class Alloc>
 constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
 template<class Alloc, class... UTypes>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&);
 template<class Alloc, class... UTypes>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
 template<class Alloc, class... UTypes>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
 };
}

```

```

template<class Alloc, class... UTypes>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&&);
template<class Alloc, class U1, class U2>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&);
template<class Alloc, class U1, class U2>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&&);
template<class Alloc, class U1, class U2>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
template<class Alloc, class U1, class U2>
 constexpr explicit(see below)
 tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&&);

template<class Alloc, tuple-like UTuple>
 constexpr explicit(see below) tuple(allocator_arg_t, const Alloc& a, UTuple&&);

// 22.4.4.3, tuple assignment
constexpr tuple& operator=(const tuple&);
constexpr const tuple& operator=(const tuple&) const;
constexpr tuple& operator=(tuple&&) noexcept(see below);
constexpr const tuple& operator=(tuple&&) const;

template<class... UTypes>
 constexpr tuple& operator=(const tuple<UTypes...>&);
template<class... UTypes>
 constexpr const tuple& operator=(const tuple<UTypes...>&) const;
template<class... UTypes>
 constexpr tuple& operator=(tuple<UTypes...>&&);
template<class... UTypes>
 constexpr const tuple& operator=(tuple<UTypes...>&&) const;

template<class U1, class U2>
 constexpr tuple& operator=(const pair<U1, U2>&); // only if sizeof...(Types) == 2
template<class U1, class U2>
 constexpr const tuple& operator=(const pair<U1, U2>&) const; // only if sizeof...(Types) == 2
template<class U1, class U2>
 constexpr tuple& operator=(pair<U1, U2>&&); // only if sizeof...(Types) == 2
template<class U1, class U2>
 constexpr const tuple& operator=(pair<U1, U2>&&) const; // only if sizeof...(Types) == 2

template<tuple-like UTuple>
 constexpr tuple& operator=(UTuple&&);
template<tuple-like UTuple>
 constexpr const tuple& operator=(UTuple&&) const;

// 22.4.4.4, tuple swap
constexpr void swap(tuple&) noexcept(see below);
constexpr void swap(const tuple&) const noexcept(see below);
};

template<class... UTypes>
 tuple(UTypes...) -> tuple<UTypes...>;
template<class T1, class T2>
 tuple(pair<T1, T2>) -> tuple<T1, T2>;
template<class Alloc, class... UTypes>
 tuple(allocator_arg_t, Alloc, UTypes...) -> tuple<UTypes...>;
template<class Alloc, class T1, class T2>
 tuple(allocator_arg_t, Alloc, pair<T1, T2>) -> tuple<T1, T2>;

```

```

template<class Alloc, class... UTypes>
 tuple(allocator_arg_t, Alloc, tuple<UTypes...>) -> tuple<UTypes...>;
}

```

- 1 If a program declares an explicit or partial specialization of `tuple`, the program is ill-formed, no diagnostic required.

#### 22.4.4.2 Construction

[`tuple.cnstr`]

- 1 In the descriptions that follow, let  $i$  be in the range  $[0, \text{sizeof} \dots (\text{Types}))$  in order,  $T_i$  be the  $i^{\text{th}}$  type in `Types`, and  $U_i$  be the  $i^{\text{th}}$  type in a template parameter pack named `UTypes`, where indexing is zero-based.
- 2 For each `tuple` constructor, an exception is thrown only if the construction of one of the types in `Types` throws an exception.
- 3 The defaulted move and copy constructor, respectively, of `tuple` is a constexpr function if and only if all required element-wise initializations for move and copy, respectively, would be constexpr-suitable (9.2.6). The defaulted move and copy constructor of `tuple<>` are constexpr functions.
- 4 If `is_trivially_destructible_v<Ti>` is true for all  $T_i$ , then the destructor of `tuple` is trivial.
- 5 The default constructor of `tuple<>` is trivial.

```
constexpr explicit(see below) tuple();
```

- 6 *Constraints:* `is_default_constructible_v<Ti>` is true for all  $i$ .

7 *Effects:* Value-initializes each element.

- 8 *Remarks:* The expression inside `explicit` evaluates to true if and only if  $T_i$  is not copy-list-initializable from an empty list for at least one  $i$ .

[Note 1: This behavior can be implemented with a trait that checks whether a `const Ti&` can be initialized with `{}`. — end note]

```
constexpr explicit(see below) tuple(const Types&...)
noexcept((is_nothrow_copy_constructible_v<Types> && ...));
```

- 9 *Constraints:* `sizeof... (Types) ≥ 1` and `is_copy_constructible_v<Ti>` is true for all  $i$ .

10 *Effects:* Initializes each element with the value of the corresponding parameter.

- 11 *Remarks:* The expression inside `explicit` is equivalent to:

```
!conjunction_v<is_convertible<const Types&, Types>...>
```

```
template<class... UTypes> constexpr explicit(see below) tuple(UTypes&&... u)
noexcept((is_nothrow_constructible_v<Types, UTypes> && ...));
```

- 12 Let *disambiguating-constraint* be:

- (12.1) — `negation<is_same<remove_cvref_t<U0>, tuple>>` if `sizeof... (Types)` is 1;
- (12.2) — otherwise, `bool_constant<!is_same_v<remove_cvref_t<U0>, allocator_arg_t> || is_same_v<remove_cvref_t<T0>, allocator_arg_t>>` if `sizeof... (Types)` is 2 or 3;
- (12.3) — otherwise, `true_type`.

- 13 *Constraints:*

- (13.1) — `sizeof... (Types)` equals `sizeof... (UTypes)`,
- (13.2) — `sizeof... (Types) ≥ 1`, and
- (13.3) — `conjunction_v<disambiguating-constraint, is_constructible<Types, UTypes>...>` is true.

- 14 *Effects:* Initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

- 15 *Remarks:* The expression inside `explicit` is equivalent to:

```
!conjunction_v<is_convertible<UTypes, Types>...>
```

This constructor is defined as deleted if

```
(reference_constructs_from_temporary_v<Types, UTypes&&> || ...)
```

is true.

```
tuple(const tuple& u) = default;
```

16 *Mandates:* `is_copy_constructible_v<Ti>` is true for all *i*.

17 *Effects:* Initializes each element of `*this` with the corresponding element of `u`.

```
tuple(tuple&& u) = default;
```

18 *Constraints:* `is_move_constructible_v<Ti>` is true for all *i*.

19 *Effects:* For all *i*, initializes the *i*<sup>th</sup> element of `*this` with `std::forward<Ti>(get<i>(u))`.

```
template<class... UTypes> constexpr explicit(see below) tuple(tuple<UTypes...>& u);
template<class... UTypes> constexpr explicit(see below) tuple(const tuple<UTypes...>& u);
template<class... UTypes> constexpr explicit(see below) tuple(tuple<UTypes...>&& u);
template<class... UTypes> constexpr explicit(see below) tuple(const tuple<UTypes...>&& u);
```

20 Let *I* be the pack 0, 1, ..., (sizeof...(Types) - 1).

Let *FWD*(*u*) be `static_cast<decltype(u)>(u)`.

21 *Constraints:*

(21.1) — `sizeof...(Types)` equals `sizeof...(UTypes)`, and

(21.2) — `(is_constructible_v<Types, decltype(get<I>(FWD(u)))> && ...)` is true, and

(21.3) — either `sizeof...(Types)` is not 1, or (when `Types...` expands to `T` and `UTypes...` expands to `U`) `is_convertible_v<decltype(u), T>`, `is_constructible_v<T, decltype(u)>`, and `is_same_v<T, U>` are all false.

22 *Effects:* For all *i*, initializes the *i*<sup>th</sup> element of `*this` with `get<i>(FWD(u))`.

23 *Remarks:* The expression inside `explicit` is equivalent to:

```
!(is_convertible_v<decltype(get<I>(FWD(u))), Types> && ...)
```

The constructor is defined as deleted if

```
(reference_constructs_from_temporary_v<Types, decltype(get<I>(FWD(u)))> || ...)
```

is true.

```
template<class U1, class U2> constexpr explicit(see below) tuple(pair<U1, U2>& u);
template<class U1, class U2> constexpr explicit(see below) tuple(const pair<U1, U2>& u);
template<class U1, class U2> constexpr explicit(see below) tuple(pair<U1, U2>&& u);
template<class U1, class U2> constexpr explicit(see below) tuple(const pair<U1, U2>&& u);
```

24 Let *FWD*(*u*) be `static_cast<decltype(u)>(u)`.

25 *Constraints:*

(25.1) — `sizeof...(Types)` is 2,

(25.2) — `is_constructible_v<T0, decltype(get<0>(FWD(u)))>` is true, and

(25.3) — `is_constructible_v<T1, decltype(get<1>(FWD(u)))>` is true.

26 *Effects:* Initializes the first element with `get<0>(FWD(u))` and the second element with `get<1>(FWD(u))`.

27 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<decltype(get<0>(FWD(u))), T0> ||
!is_convertible_v<decltype(get<1>(FWD(u))), T1>
```

The constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T0, decltype(get<0>(FWD(u)))> ||
reference_constructs_from_temporary_v<T1, decltype(get<1>(FWD(u)))>
```

is true.

```
template<tuple-like UTuple>
constexpr explicit(see below) tuple(UTuple&& u);
```

28 Let *I* be the pack 0, 1, ..., (sizeof...(Types) - 1).

29 *Constraints:*

(29.1) — `different-from<UTuple, tuple>` (25.5.2) is true,



- (29.2) — `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- (29.3) — `sizeof...(Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`,
- (29.4) — `(is_constructible_v<Types, decltype(get<I>(std::forward<UTuple>(u)))> && ...)` is true, and
- (29.5) — either `sizeof...(Types)` is not 1, or (when `Types...` expands to `T`) `is_convertible_v<UTuple, T>` and `is_constructible_v<T, UTuple>` are both false.

*Effects:* For all  $i$ , initializes the  $i^{\text{th}}$  element of `*this` with `get<i>(std::forward<UTuple>(u))`.

*Remarks:* The expression inside `explicit` is equivalent to:

```
!(is_convertible_v<decltype(get<I>(std::forward<UTuple>(u))), Types> && ...)
```

The constructor is defined as deleted if

```
(reference_constructs_from_temporary_v<Types, decltype(get<I>(std::forward<UTuple>(u)))>
|| ...)
```

is true.

```
template<class Alloc>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a);
template<class Alloc>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, const Types&...);
template<class Alloc, class... UTypes>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
template<class Alloc>
constexpr tuple(allocator_arg_t, const Alloc& a, const tuple&);
template<class Alloc>
constexpr tuple(allocator_arg_t, const Alloc& a, tuple&&);
template<class Alloc, class... UTypes>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&);
template<class Alloc, class... UTypes>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template<class Alloc, class... UTypes>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template<class Alloc, class... UTypes>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&&);
template<class Alloc, class U1, class U2>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&);
template<class Alloc, class U1, class U2>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template<class Alloc, class U1, class U2>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
template<class Alloc, class U1, class U2>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&&);
template<class Alloc, tuple-like UTuple>
constexpr explicit(see below)
tuple(allocator_arg_t, const Alloc& a, UTuple&&);
```

*Preconditions:* `Alloc` meets the *Cpp17Allocator* requirements (16.4.4.6.1).

*Effects:* Equivalent to the preceding constructors except that each element of non-reference type is constructed with `uses_allocator` construction (20.2.8.2).



## 22.4.4.3 Assignment

[tuple.assign]

- 1 For each `tuple` assignment operator, an exception is thrown only if the assignment of one of the types in `Types` throws an exception. In the function descriptions that follow, let  $i$  be in the range  $[0, \text{sizeof} \dots (\text{Types})]$  in order,  $T_i$  be the  $i^{\text{th}}$  type in `Types`, and  $U_i$  be the  $i^{\text{th}}$  type in a template parameter pack named `UTypes`, where indexing is zero-based.

```
constexpr tuple& operator=(const tuple& u);
```

- 2 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

- 3 *Returns:* `*this`.

- 4 *Remarks:* This operator is defined as deleted unless `is_copy_assignable_v<Ti>` is true for all  $i$ .

```
constexpr const tuple& operator=(const tuple& u) const;
```

- 5 *Constraints:* `(is_copy_assignable_v<const Types> && ...)` is true.

- 6 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

- 7 *Returns:* `*this`.

```
constexpr tuple& operator=(tuple&& u) noexcept(see below);
```

- 8 *Constraints:* `is_move_assignable_v<Ti>` is true for all  $i$ .

- 9 *Effects:* For all  $i$ , assigns `std::forward<Ti>(get< $i$ >(u))` to `get< $i$ >(*this)`.

- 10 *Returns:* `*this`.

- 11 *Remarks:* The exception specification is equivalent to the logical AND of the following expressions:

```
is_nothrow_move_assignable_v<Ti>
```

where  $T_i$  is the  $i^{\text{th}}$  type in `Types`.

```
constexpr const tuple& operator=(tuple&& u) const;
```

- 12 *Constraints:* `(is_assignable_v<const Types&, Types> && ...)` is true.

- 13 *Effects:* For all  $i$ , assigns `std::forward<Ti>(get< $i$ >(u))` to `get< $i$ >(*this)`.

- 14 *Returns:* `*this`.

```
template<class... UTypes> constexpr tuple& operator=(const tuple<UTypes...>& u);
```

- 15 *Constraints:*

- (15.1) — `sizeof...(Types)` equals `sizeof...(UTypes)` and

- (15.2) — `is_assignable_v<Ti&, const Ui>` is true for all  $i$ .

- 16 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

- 17 *Returns:* `*this`.

```
template<class... UTypes> constexpr const tuple& operator=(const tuple<UTypes...>& u) const;
```

- 18 *Constraints:*

- (18.1) — `sizeof...(Types)` equals `sizeof...(UTypes)` and

- (18.2) — `(is_assignable_v<const Types&, const UTypes&> && ...)` is true.

- 19 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

- 20 *Returns:* `*this`.

```
template<class... UTypes> constexpr tuple& operator=(tuple<UTypes...>&& u);
```

- 21 *Constraints:*

- (21.1) — `sizeof...(Types)` equals `sizeof...(UTypes)` and

- (21.2) — `is_assignable_v<Ti&, Ui>` is true for all  $i$ .

- 22 *Effects:* For all  $i$ , assigns `std::forward<Ui>(get< $i$ >(u))` to `get< $i$ >(*this)`.

- 23 *Returns:* `*this`.

```
template<class... UTypes> constexpr const tuple& operator=(tuple<UTypes...>&& u) const;
```

24     *Constraints:*

- (24.1)     — `sizeof...(Types)` equals `sizeof...(UTypes)` and
- (24.2)     — `(is_assignable_v<const Types&, UTypes> && ...)` is true.

25     *Effects:* For all  $i$ , assigns `std::forward<Ui>(get<i to get<i.`

26     *Returns:* `*this`.

```
template<class U1, class U2> constexpr tuple& operator=(const pair<U1, U2>& u);
```

27     *Constraints:*

- (27.1)     — `sizeof...(Types)` is 2 and
- (27.2)     — `is_assignable_v<T0&, const U1&>` is true, and
- (27.3)     — `is_assignable_v<T1&, const U2&>` is true.

28     *Effects:* Assigns `u.first` to the first element of `*this` and `u.second` to the second element of `*this`.

29     *Returns:* `*this`.

```
template<class U1, class U2> constexpr const tuple& operator=(const pair<U1, U2>& u) const;
```

30     *Constraints:*

- (30.1)     — `sizeof...(Types)` is 2,
- (30.2)     — `is_assignable_v<const T0&, const U1&>` is true, and
- (30.3)     — `is_assignable_v<const T1&, const U2&>` is true.

31     *Effects:* Assigns `u.first` to the first element and `u.second` to the second element.

32     *Returns:* `*this`.

```
template<class U1, class U2> constexpr tuple& operator=(pair<U1, U2>&& u);
```

33     *Constraints:*

- (33.1)     — `sizeof...(Types)` is 2 and
- (33.2)     — `is_assignable_v<T0&, U1>` is true, and
- (33.3)     — `is_assignable_v<T1&, U2>` is true.

34     *Effects:* Assigns `std::forward<U1>(u.first)` to the first element of `*this` and `std::forward<U2>(u.second)` to the second element of `*this`.

35     *Returns:* `*this`.

```
template<class U1, class U2> constexpr const tuple& operator=(pair<U1, U2>&& u) const;
```

36     *Constraints:*

- (36.1)     — `sizeof...(Types)` is 2,
- (36.2)     — `is_assignable_v<const T0&, U1>` is true, and
- (36.3)     — `is_assignable_v<const T1&, U2>` is true.

37     *Effects:* Assigns `std::forward<U1>(u.first)` to the first element and `std::forward<U2>(u.second)` to the second element.

38     *Returns:* `*this`.

```
template<tuple-like UTuple>
constexpr tuple& operator=(UTuple&& u);
```

39     *Constraints:*

- (39.1)     — *different-from*<UTuple, tuple> (25.5.2) is true,
- (39.2)     — `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,
- (39.3)     — `sizeof...(Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`, and
- (39.4)     — `is_assignable_v<Ti&, decltype(get<i is true for all  $i$ .`

40 *Effects:* For all  $i$ , assigns `get<i>(std::forward<UTuple>(u))` to `get<i>(*this)`.

41 *Returns:* `*this`.

```
template<tuple-like UTuple>
constexpr const tuple& operator=(UTuple&& u) const;
```

42 *Constraints:*

(42.1) — *different-from*<UTuple, tuple> (25.5.2) is true,

(42.2) — `remove_cvref_t<UTuple>` is not a specialization of `ranges::subrange`,

(42.3) — `sizeof...(Types)` equals `tuple_size_v<remove_cvref_t<UTuple>>`, and

(42.4) — `is_assignable_v<const Ti&, decltype(get<i>(std::forward<UTuple>(u)))>` is true for all  $i$ .

43 *Effects:* For all  $i$ , assigns `get<i>(std::forward<UTuple>(u))` to `get<i>(*this)`.

44 *Returns:* `*this`.

#### 22.4.4.4 swap

[tuple.swap]

```
constexpr void swap(tuple& rhs) noexcept(see below);
constexpr void swap(const tuple& rhs) const noexcept(see below);
```

1 Let  $i$  be in the range `[0, sizeof...(Types))` in order.

2 *Mandates:*

(2.1) — For the first overload, `(is_swappable_v<Types> && ...)` is true.

(2.2) — For the second overload, `(is_swappable_v<const Types> && ...)` is true.

3 *Preconditions:* For all  $i$ , `get<i>(*this)` is swappable with (16.4.4.3) `get<i>(rhs)`.

4 *Effects:* For each  $i$ , calls `swap` for `get<i>(*this)` and `get<i>(rhs)`.

5 *Throws:* Nothing unless one of the element-wise `swap` calls throws an exception.

6 *Remarks:* The exception specification is equivalent to

(6.1) — `(is_nothrow_swappable_v<Types> && ...)` for the first overload and

(6.2) — `(is_nothrow_swappable_v<const Types> && ...)` for the second overload.

#### 22.4.5 Tuple creation functions

[tuple.creation]

```
template<class... TTypes>
constexpr tuple<unwrap_ref_decay_t<TTypes>...> make_tuple(TTypes&&... t);
```

1 *Returns:* `tuple<unwrap_ref_decay_t<TTypes>...>(std::forward<TTypes>(t)...) .`

2 [Example 1:

```
 int i; float j;
 make_tuple(1, ref(i), cref(j));
```

creates a tuple of type `tuple<int, int&, const float&>`. — end example]

```
template<class... TTypes>
constexpr tuple<TTypes&&...> forward_as_tuple(TTypes&&... t) noexcept;
```

3 *Effects:* Constructs a tuple of references to the arguments in `t` suitable for forwarding as arguments to a function. Because the result may contain references to temporary objects, a program shall ensure that the return value of this function does not outlive any of its arguments (e.g., the program should typically not store the result in a named variable).

4 *Returns:* `tuple<TTypes&&...>(std::forward<TTypes>(t)...) .`

```
template<class... TTypes>
constexpr tuple<TTypes&...> tie(TTypes&... t) noexcept;
```

5 *Returns:* `tuple<TTypes&...>(t...) .`

6 [Example 2: `tie` functions allow one to create tuples that unpack tuples into variables. `ignore` can be used for elements that are not needed:

```

 int i; std::string s;
 tie(i, ignore, s) = make_tuple(42, 3.14, "C++");
 // i == 42, s == "C++"
— end example]

```

```

template<tuple-like... Tuples>
constexpr tuple<CTypes...> tuple_cat(Tuples&&... tpls);

```

- 7 Let  $n$  be `sizeof...(Tuples)`. For every integer  $0 \leq i < n$ :
- (7.1) — Let  $T_i$  be the  $i^{\text{th}}$  type in `Tuples`.
  - (7.2) — Let  $U_i$  be `remove_cvref_t<T_i>`.
  - (7.3) — Let  $tp_i$  be the  $i^{\text{th}}$  element in the function parameter pack `tpls`.
  - (7.4) — Let  $S_i$  be `tuple_size_v<U_i>`.
  - (7.5) — Let  $E_i^k$  be `tuple_element_t<k, U_i>`.
  - (7.6) — Let  $e_i^k$  be `get<k>(std::forward<T_i>(tp_i))`.
  - (7.7) — Let  $Elms_i$  be a pack of the types  $E_i^0, \dots, E_i^{S_i-1}$ .
  - (7.8) — Let  $elems_i$  be a pack of the expressions  $e_i^0, \dots, e_i^{S_i-1}$ .

The types in `CTypes` are equal to the ordered sequence of the expanded packs of types  $Elms_0, \dots, Elms_1, \dots, Elms_{n-1}, \dots$ . Let `celems` be the ordered sequence of the expanded packs of expressions  $elems_0, \dots, elems_{n-1}, \dots$ .

8 *Mandates:* `(is_constructible_v<CTypes, decltype(celems)> && ...)` is true.

9 *Returns:* `tuple<CTypes...>(celems...)`.

## 22.4.6 Calling a function with a tuple of arguments

[tuple.apply]

```

template<class F, tuple-like Tuple>
constexpr apply_result_t<F, Tuple> apply(F&& f, Tuple&& t)
noexcept(is_nothrow_applicable_v<F, Tuple>);

```

1 *Effects:* Given the exposition-only function template:

```

namespace std {
 template<class F, tuple-like Tuple, size_t... I>
 constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, index_sequence<I...>) {
 return INVOKE(std::forward<F>(f), get<I>(std::forward<Tuple>(t))...); // exposition only
 }
}

```

Equivalent to:

```

return apply_impl(std::forward<F>(f), std::forward<Tuple>(t),
 make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>>{});

```

```

template<class T, tuple-like Tuple>
constexpr T make_from_tuple(Tuple&& t);

```

2 *Mandates:* If `tuple_size_v<remove_reference_t<Tuple>>` is 1, then `reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))>` is false.

3 *Effects:* Given the exposition-only function template:

```

namespace std {
 template<class T, tuple-like Tuple, size_t... I>
 requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
 constexpr T make_from_tuple_impl(Tuple&& t, index_sequence<I...>) { // exposition only
 return T(get<I>(std::forward<Tuple>(t))...);
 }
}

```

Equivalent to:

```

 return make-from-tuple-impl<T>(
 std::forward<Tuple>(t),
 make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>>({});

```

[Note 1: The type of T must be supplied as an explicit template parameter, as it cannot be deduced from the argument list. — end note]

## 22.4.7 Tuple helper classes

[tuple.helper]

```
template<class T> struct tuple_size;
```

- 1 Except where specified otherwise, all specializations of `tuple_size` meet the *Cpp17UnaryTypeTrait* requirements (21.3.2) with a base characteristic of `integral_constant<size_t, N>` for some N.

```
template<class... Types>
 struct tuple_size<tuple<Types...>> : integral_constant<size_t, sizeof...(Types)> { };
```

```
template<size_t I, class... Types>
 struct tuple_element<I, tuple<Types...>> {
 using type = TI;
 };

```

- 2 *Mandates:* `I < sizeof...(Types)`.

- 3 *Result:* TI is the type of the  $I^{\text{th}}$  element of `Types`, where indexing is zero-based.

```
template<class T> struct tuple_size<const T>;
```

- 4 Let TS denote `tuple_size<T>` of the cv-unqualified type T. If the expression `TS::value` is well-formed when treated as an unevaluated operand (7.2.3), then each specialization of the template meets the *Cpp17UnaryTypeTrait* requirements (21.3.2) with a base characteristic of

```
integral_constant<size_t, TS::value>
```

Otherwise, it has no member `value`.

- 5 Access checking is performed as if in a context unrelated to TS and T. Only the validity of the immediate context of the expression is considered.

[Note 1: The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — end note]

- 6 In addition to being available via inclusion of the `<tuple>` header, the template is available when any of the headers `<array>` (23.3.2), `<complex>` (29.4.2), `<ranges>` (25.2), or `<utility>` (22.2.1) are included.

```
template<size_t I, class T> struct tuple_element<I, const T>;
```

- 7 Let TE denote `tuple_element_t<I, T>` of the cv-unqualified type T. Then each specialization of the template meets the *Cpp17TransformationTrait* requirements (21.3.2) with a member typedef `type` that names the type `const TE`.

- 8 In addition to being available via inclusion of the `<tuple>` header, the template is available when any of the headers `<array>` (23.3.2), `<complex>` (29.4.2), `<ranges>` (25.2), or `<utility>` (22.2.1) are included.

## 22.4.8 Element access

[tuple.elem]

```
template<size_t I, class... Types>
 constexpr tuple_element_t<I, tuple<Types...>>&
 get(tuple<Types...>& t) noexcept;
template<size_t I, class... Types>
 constexpr tuple_element_t<I, tuple<Types...>>&&
 get(tuple<Types...>&& t) noexcept; // #1
template<size_t I, class... Types>
 constexpr const tuple_element_t<I, tuple<Types...>>&
 get(const tuple<Types...>& t) noexcept; // #2

```

```
template<size_t I, class... Types>
constexpr const tuple_element_t<I, tuple<Types...>&& t> get(const tuple<Types...>&& t) noexcept;
```

1 *Mandates:*  $I < \text{sizeof}...(Types)$ .

2 *Returns:* A reference to the  $I^{\text{th}}$  element of  $t$ , where indexing is zero-based.

3 [Note 1: For the overload marked #1, if a type  $T$  in  $Types$  is some reference type  $X\&$ , the return type is  $X\&$ , not  $X\&\&$ . However, if the element type is a non-reference type  $T$ , the return type is  $T\&$ . — end note]

4 [Note 2: Constness is shallow. For the overload marked #2, if a type  $T$  in  $Types$  is some reference type  $X\&$ , the return type is  $X\&$ , not `const  $X\&$` . However, if the element type is a non-reference type  $T$ , the return type is `const  $T\&$` . This is consistent with how constness is defined to work for non-static data members of reference type. — end note]

```
template<class T, class... Types>
constexpr T& get(tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr T&& get(tuple<Types...>&& t) noexcept;
template<class T, class... Types>
constexpr const T& get(const tuple<Types...>& t) noexcept;
template<class T, class... Types>
constexpr const T&& get(const tuple<Types...>&& t) noexcept;
```

5 *Mandates:* The type  $T$  occurs exactly once in  $Types$ .

6 *Returns:* A reference to the element of  $t$  corresponding to the type  $T$  in  $Types$ .

7 [Example 1:

```
 const tuple<int, const int, double, double> t(1, 2, 3.4, 5.6);
 const int& i1 = get<int>(t); // OK, i1 has value 1
 const int& i2 = get<const int>(t); // OK, i2 has value 2
 const double& d = get<double>(t); // error: type double is not unique within t
```

— end example]

8 [Note 3: The reason `get` is a non-member function is that if this functionality had been provided as a member function, code where the type depended on a template parameter would have required using the `template` keyword. — end note]

## 22.4.9 Relational operators

[tuple.rel]

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr bool operator==(const tuple<TTypes...>& t, const UTuple& u);
```

1 For the first overload let  $UTuple$  be  $tuple<UTypes...>$ .

2 *Constraints:* For all  $i$ , where  $0 \leq i < \text{sizeof}...(TTypes)$ , `get< $i$ >(t) == get< $i$ >(u)` is a valid expression and `decltype(get< $i$ >(t) == get< $i$ >(u))` models *boolean-testable*. `sizeof...(TTypes)` equals `tuple_size_v<UTuple>`.

3 *Returns:* `true` if `get< $i$ >(t) == get< $i$ >(u)` for all  $i$ , otherwise `false`.

[Note 1: If `sizeof...(TTypes)` equals zero, returns `true`. — end note]

4 *Remarks:*

(4.1) — The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to `false`.

(4.2) — The second overload is to be found via argument-dependent lookup (6.5.4) only.

```
template<class... TTypes, class... UTypes>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, UTypes>...>
 operator<=>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, Elms>...>
 operator<=>(const tuple<TTypes...>& t, const UTuple& u);
```

5 For the second overload,  $Elms$  denotes the pack of types `tuple_element_t<0, UTuple>`, `tuple_element_t<1, UTuple>`, ..., `tuple_element_t<tuple_size_v<UTuple> - 1, UTuple>`.

6 *Effects:* Performs a lexicographical comparison between `t` and `u`. If `sizeof...(TTypes)` equals zero, returns `strong_ordering::equal`. Otherwise, equivalent to:

```
if (auto c = synth-three-way(get<0>(t), get<0>(u)); c != 0) return c;
return ttail <=> utail;
```

where `rtail` for some `r` is a tuple containing all but the first element of `r`.

7 *Remarks:* The second overload is to be found via argument-dependent lookup (6.5.4) only.

8 [Note 2: The above definition does not require `ttail` (or `utail`) to be constructed. It might not even be possible, as `t` and `u` are not required to be copy constructible. Also, all comparison operator functions are short circuited; they do not perform element accesses beyond what is needed to determine the result of the comparison. — end note]

## 22.4.10 common\_reference related specializations

[tuple.common.ref]

1 In the descriptions that follow:

(1.1) — Let `TTypes` be a pack formed by the sequence of `tuple_element_t<i, TTuple>` for every integer  $0 \leq i < \text{tuple\_size\_v}<\text{TTuple}>$ .

(1.2) — Let `UTypes` be a pack formed by the sequence of `tuple_element_t<i, UTuple>` for every integer  $0 \leq i < \text{tuple\_size\_v}<\text{UTuple}>$ .

```
template<tuple-like TTuple, tuple-like UTuple,
 template<class> class TQual, template<class> class UQual>
struct basic_common_reference<TTuple, UTuple, TQual, UQual> {
 using type = see below;
};
```

2 *Constraints:*

(2.1) — `TTuple` is a specialization of `tuple` or `UTuple` is a specialization of `tuple`.

(2.2) — `is_same_v<TTuple, decay_t<TTuple>>` is true.

(2.3) — `is_same_v<UTuple, decay_t<UTuple>>` is true.

(2.4) — `tuple_size_v<TTuple>` equals `tuple_size_v<UTuple>`.

(2.5) — `tuple<common_reference_t<TQual<TTypes>, UQual<UTypes>>...>` denotes a type.

3 *Result:* The member *typedef-name* type denotes the type `tuple<common_reference_t<TQual<TTypes>, UQual<UTypes>>...>`.

```
template<tuple-like TTuple, tuple-like UTuple>
struct common_type<TTuple, UTuple> {
 using type = see below;
};
```

4 *Constraints:*

(4.1) — `TTuple` is a specialization of `tuple` or `UTuple` is a specialization of `tuple`.

(4.2) — `is_same_v<TTuple, decay_t<TTuple>>` is true.

(4.3) — `is_same_v<UTuple, decay_t<UTuple>>` is true.

(4.4) — `tuple_size_v<TTuple>` equals `tuple_size_v<UTuple>`.

(4.5) — `tuple<common_type_t<TTypes, UTypes>...>` denotes a type.

5 *Result:* The member *typedef-name* type denotes the type `tuple<common_type_t<TTypes, UTypes>...>`.

## 22.4.11 Tuple traits

[tuple.traits]

```
template<class... Types, class Alloc>
struct uses_allocator<tuple<Types...>, Alloc> : true_type { };
```

1 *Preconditions:* `Alloc` meets the *Cpp17Allocator* requirements (16.4.4.6.1).

2 [Note 1: Specialization of this trait informs other library components that `tuple` can be constructed with an allocator, even though it does not have a nested `allocator_type`. — end note]

**22.4.12 Tuple specialized algorithms****[tuple.special]**

```
template<class... Types>
constexpr void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
template<class... Types>
constexpr void swap(const tuple<Types...>& x, const tuple<Types...>& y) noexcept(see below);
```

*Constraints:*

- (1.1) — For the first overload, (`is_swappable_v<Types> && ...`) is true.
- (1.2) — For the second overload, (`is_swappable_v<const Types> && ...`) is true.

*Effects:* As if by `x.swap(y)`.

*Remarks:* The exception specification is equivalent to:

```
noexcept(x.swap(y))
```

**22.5 Optional objects****[optional]****22.5.1 General****[optional.general]**

- 1 Subclause 22.5 describes class template `optional` that represents optional objects. An *optional object* is an object that contains the storage for another object and manages the lifetime of this contained object, if any. The contained object may be initialized after the optional object has been initialized, and may be destroyed before the optional object has been destroyed. The initialization state of the contained object is tracked by the optional object.

**22.5.2 Header <optional> synopsis****[optional.syn]**

```
// mostly freestanding
#include <compare> // see 17.12.1

namespace std {
 // 22.5.3, class template optional
 template<class T>
 class optional; // partially freestanding

 // 22.5.4, partial specialization of optional for lvalue reference types
 template<class T>
 class optional<T&>; // partially freestanding

 template<class T>
 constexpr bool ranges::enable_view<optional<T>> = true;
 template<class T>
 constexpr auto format_kind<optional<T>> = range_format::disabled;
 template<class T>
 constexpr bool ranges::enable_borrowed_range<optional<T&>> = true;

 template<class T>
 concept is-derived-from-optional = requires(const T& t) { // exposition only
 [<class U>(const optional<U>&){ }(t);
 };

 // 22.5.5, no-value state indicator
 struct nullopt_t{see below};
 inline constexpr nullopt_t nullopt(unspecified);

 // 22.5.6, class bad_optional_access
 class bad_optional_access;

 // 22.5.7, relational operators
 template<class T, class U>
 constexpr bool operator==(const optional<T>&, const optional<U>&);
 template<class T, class U>
 constexpr bool operator!=(const optional<T>&, const optional<U>&);
 template<class T, class U>
 constexpr bool operator<(const optional<T>&, const optional<U>&);
```



```

template<class T, class U>
 constexpr bool operator>(const optional<T>&, const optional<U>&);
template<class T, class U>
 constexpr bool operator<=(const optional<T>&, const optional<U>&);
template<class T, class U>
 constexpr bool operator>=(const optional<T>&, const optional<U>&);
template<class T, three_way_comparable_with<T> U>
 constexpr compare_three_way_result_t<T, U>
 operator<=>(const optional<T>&, const optional<U>&);

// 22.5.8, comparison with nullopt
template<class T> constexpr bool operator==(const optional<T>&, nullopt_t) noexcept;
template<class T>
 constexpr strong_ordering operator<=>(const optional<T>&, nullopt_t) noexcept;

// 22.5.9, comparison with T
template<class T, class U> constexpr bool operator==(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator==(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator!=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator!=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator<=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator<=(const T&, const optional<U>&);
template<class T, class U> constexpr bool operator>=(const optional<T>&, const U&);
template<class T, class U> constexpr bool operator>=(const T&, const optional<U>&);
template<class T, class U>
 requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
 constexpr compare_three_way_result_t<T, U>
 operator<=>(const optional<T>&, const U&);

// 22.5.10, specialized algorithms
template<class T>
 constexpr void swap(optional<T>&, optional<T>&) noexcept(see below);

template<class T>
 constexpr optional<decay_t<T>> make_optional(T&&);
template<class T, class... Args>
 constexpr optional<T> make_optional(Args&&... args);
template<class T, class U, class... Args>
 constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);

// 22.5.11, hash support
template<class T> struct hash;
template<class T> struct hash<optional<T>>;
}

```

## 22.5.3 Class template optional

[optional.optional]

### 22.5.3.1 General

[optional.optional.general]

```

namespace std {
 template<class T>
 class optional {
 public:
 using value_type = T;
 using iterator = implementation-defined; // see 22.5.3.6
 using const_iterator = implementation-defined; // see 22.5.3.6

 // 22.5.3.2, constructors
 constexpr optional() noexcept;
 constexpr optional(nullopt_t) noexcept;
 constexpr optional(const optional&);

```

```

constexpr optional(optional&&) noexcept(see below);
template<class... Args>
 constexpr explicit optional(in_place_t, Args&&...);
template<class U, class... Args>
 constexpr explicit optional(in_place_t, initializer_list<U>, Args&&...);
template<class U = remove_cv_t<T>>
 constexpr explicit(see below) optional(U&&);
template<class U>
 constexpr explicit(see below) optional(const optional<U>&);
template<class U>
 constexpr explicit(see below) optional(optional<U>&&);

// 22.5.3.3, destructor
constexpr ~optional();

// 22.5.3.4, assignment
constexpr optional& operator=(nullopt_t) noexcept;
constexpr optional& operator=(const optional&);
constexpr optional& operator=(optional&&) noexcept(see below);
template<class U = remove_cv_t<T>> constexpr optional& operator=(U&&);
template<class U> constexpr optional& operator=(const optional<U>&);
template<class U> constexpr optional& operator=(optional<U>&&);
template<class... Args> constexpr T& emplace(Args&&...);
template<class U, class... Args> constexpr T& emplace(initializer_list<U>, Args&&...);

// 22.5.3.5, swap
constexpr void swap(optional&) noexcept(see below);

// 22.5.3.6, iterator support
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

// 22.5.3.7, observers
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
constexpr const T& value() const &;
constexpr T& value() &; // freestanding-deleted
constexpr T&& value() &&; // freestanding-deleted
constexpr const T&& value() const &&; // freestanding-deleted
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;

// 22.5.3.8, monadic operations
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &;
template<class F> constexpr auto and_then(F&& f) const &&;
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &;
template<class F> constexpr auto transform(F&& f) const &&;
template<class F> constexpr optional or_else(F&& f) &&;
template<class F> constexpr optional or_else(F&& f) const &;

// 22.5.3.9, modifiers
constexpr void reset() noexcept;

```

```

private:
 union {
 remove_cv_t<T> val; // exposition only
 };
};

template<class T>
 optional(T) -> optional<T>;
}

```

- <sup>1</sup> An instance of `optional<T>` is said to *contain a value* when and only when its member `val` is active (11.5.1); `val` is referred to as its *contained value*. An optional object's contained value is nested within (6.8.2) the optional object.
- <sup>2</sup> A type `X` is a *valid contained type* for `optional` if `X` is an lvalue reference type or a complete non-array object type, and `remove_cvref_t<X>` is a type other than `in_place_t` or `nullopt_t`. If a specialization of `optional` is instantiated with a type `T` that is not a valid contained type for `optional`, the program is ill-formed. If `T` is an object type, `T` shall meet the *Cpp17Destructible* requirements (Table 35).

### 22.5.3.2 Constructors

[optional.ctor]

- <sup>1</sup> The exposition-only variable template *converts-from-any-cvref* is used by some constructors for `optional`.

```

template<class T, class W>
constexpr bool converts-from-any-cvref = // exposition only
 disjunction_v<is_constructible<T, W&>, is_convertible<W&, T>,
 is_constructible<T, W>, is_convertible<W, T>,
 is_constructible<T, const W&>, is_convertible<const W&, T>,
 is_constructible<T, const W>, is_convertible<const W, T>>;

```

```

constexpr optional() noexcept;
constexpr optional(nullopt_t) noexcept;

```

- <sup>2</sup> *Postconditions:* `*this` does not contain a value.
- <sup>3</sup> *Remarks:* No contained value is initialized. For every object type `T` these constructors are constexpr constructors (9.2.6).

```
constexpr optional(const optional& rhs);
```

- <sup>4</sup> *Effects:* If `rhs` contains a value, direct-non-list-initializes `val` with `rhs.val`.
- <sup>5</sup> *Postconditions:* `rhs.has_value() == this->has_value()`.
- <sup>6</sup> *Throws:* Any exception thrown by the selected constructor of `T`.
- <sup>7</sup> *Remarks:* This constructor is defined as deleted unless `is_copy_constructible_v<T>` is true. If `is_trivially_copy_constructible_v<T>` is true, this constructor is trivial.

```
constexpr optional(optional&& rhs) noexcept(see below);
```

- <sup>8</sup> *Constraints:* `is_move_constructible_v<T>` is true.
- <sup>9</sup> *Effects:* If `rhs` contains a value, direct-non-list-initializes `val` with `std::move(rhs.val)`. `rhs.has_value()` is unchanged.
- <sup>10</sup> *Postconditions:* `rhs.has_value() == this->has_value()`.
- <sup>11</sup> *Throws:* Any exception thrown by the selected constructor of `T`.
- <sup>12</sup> *Remarks:* The exception specification is equivalent to `is_nothrow_move_constructible_v<T>`. If `is_trivially_move_constructible_v<T>` is true, this constructor is trivial.

```
template<class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

- <sup>13</sup> *Constraints:* `is_constructible_v<T, Args...>` is true.
- <sup>14</sup> *Effects:* Direct-non-list-initializes `val` with `std::forward<Args>(args)...`.
- <sup>15</sup> *Postconditions:* `*this` contains a value.
- <sup>16</sup> *Throws:* Any exception thrown by the selected constructor of `T`.

17 *Remarks:* If T's constructor selected for the initialization is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class U, class... Args>
constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

18 *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

19 *Effects:* Direct-non-list-initializes *val* with `il`, `std::forward<Args>(args)...`

20 *Postconditions:* `*this` contains a value.

21 *Throws:* Any exception thrown by the selected constructor of T.

22 *Remarks:* If T's constructor selected for the initialization is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class U = remove_cv_t<T>> constexpr explicit(see below) optional(U&& v);
```

23 *Constraints:*

(23.1) — `is_constructible_v<T, U>` is true,

(23.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false,

(23.3) — `is_same_v<remove_cvref_t<U>, optional>` is false, and

(23.4) — if T is *cv bool*, `remove_cvref_t<U>` is not a specialization of `optional`.

24 *Effects:* Direct-non-list-initializes *val* with `std::forward<U>(v)`.

25 *Postconditions:* `*this` contains a value.

26 *Throws:* Any exception thrown by the selected constructor of T.

27 *Remarks:* If T's selected constructor is a constexpr constructor, this constructor is a constexpr constructor. The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

```
template<class U> constexpr explicit(see below) optional(const optional<U>& rhs);
```

28 *Constraints:*

(28.1) — `is_constructible_v<T, const U&>` is true, and

(28.2) — if T is not *cv bool*, `converts-from-any-cvref<T, optional<U>>` is false.

29 *Effects:* If `rhs` contains a value, direct-non-list-initializes *val* with `rhs.operator*()`.

30 *Postconditions:* `rhs.has_value() == this->has_value()`.

31 *Throws:* Any exception thrown by the selected constructor of T.

32 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<const U&, T>
```

```
template<class U> constexpr explicit(see below) optional(optional<U>&& rhs);
```

33 *Constraints:*

(33.1) — `is_constructible_v<T, U>` is true, and

(33.2) — if T is not *cv bool*, `converts-from-any-cvref<T, optional<U>>` is false.

34 *Effects:* If `rhs` contains a value, direct-non-list-initializes *val* with `std::move(rhs).operator*()`. `rhs.has_value()` is unchanged.

35 *Postconditions:* `rhs.has_value() == this->has_value()`.

36 *Throws:* Any exception thrown by the selected constructor of T.

37 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<U, T>
```

### 22.5.3.3 Destructor

[optional.dtor]

```
constexpr ~optional();
```

1 *Effects:* If `is_trivially_destructible_v<T>` is false and `*this` contains a value, calls `val.T::~~T()`.

2 *Remarks:* If `is_trivially_destructible_v<T>` is true, then this destructor is trivial.

### 22.5.3.4 Assignment

[optional.assign]

constexpr optional<T>& operator=(nullopt\_t) noexcept;

1 *Effects:* If `*this` contains a value, calls `val.T::~~T()` to destroy the contained value; otherwise no effect.

2 *Postconditions:* `*this` does not contain a value.

3 *Returns:* `*this`.

constexpr optional<T>& operator=(const optional& rhs);

4 *Effects:* See Table 67.

Table 67 — `optional::operator=(const optional&)` effects [tab:optional.assign.copy]

|                                     | <b>*this contains a value</b>                                     | <b>*this does not contain a value</b>                                  |
|-------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>rhs contains a value</b>         | assigns <code>rhs.val</code> to <code>val</code>                  | direct-non-list-initializes <code>val</code> with <code>rhs.val</code> |
| <b>rhs does not contain a value</b> | destroys the contained value by calling <code>val.T::~~T()</code> | no effect                                                              |

5 *Postconditions:* `rhs.has_value() == this->has_value()`.

6 *Returns:* `*this`.

7 *Remarks:* If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator is defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true. If `is_trivially_copy_constructible_v<T>` && `is_trivially_copy_assignable_v<T>` && `is_trivially_destructible_v<T>` is true, this assignment operator is trivial.

constexpr optional& operator=(optional&& rhs) noexcept(see below);

8 *Constraints:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.

9 *Effects:* See Table 68. The result of the expression `rhs.has_value()` remains unchanged.

Table 68 — `optional::operator=(optional&&)` effects [tab:optional.assign.move]

|                                     | <b>*this contains a value</b>                                     | <b>*this does not contain a value</b>                                             |
|-------------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>rhs contains a value</b>         | assigns <code>std::move(rhs.val)</code> to <code>val</code>       | direct-non-list-initializes <code>val</code> with <code>std::move(rhs.val)</code> |
| <b>rhs does not contain a value</b> | destroys the contained value by calling <code>val.T::~~T()</code> | no effect                                                                         |

10 *Postconditions:* `rhs.has_value() == this->has_value()`.

11 *Returns:* `*this`.

12 *Remarks:* The exception specification is equivalent to:

`is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>`

13 If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the states of `val` and `rhs.val` are determined by the exception safety guarantee of `T`'s move

assignment. If `is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T> && is_trivially_destructible_v<T>` is true, this assignment operator is trivial.

```
template<class U = remove_cv_t<T>> constexpr optional& operator=(U&& v);
```

14 *Constraints:*

- (14.1) — `is_same_v<remove_cvref_t<U>, optional>` is false,
- (14.2) — `conjunction_v<is_scalar<T>, is_same<T, decay_t<U>>>` is false,
- (14.3) — `is_constructible_v<T, U>` is true, and
- (14.4) — `is_assignable_v<T&, U>` is true.

15 *Effects:* If `*this` contains a value, assigns `std::forward<U>(v)` to `val`; otherwise direct-non-list-initializes `val` with `std::forward<U>(v)`.

16 *Postconditions:* `*this` contains a value.

17 *Returns:* `*this`.

18 *Remarks:* If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states of `val` and `v` are determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(const optional<U>& rhs);
```

19 *Constraints:*

- (19.1) — `is_constructible_v<T, const U&>` is true,
- (19.2) — `is_assignable_v<T&, const U&>` is true,
- (19.3) — `converts-from-any-cvref<T, optional<U>>` is false,
- (19.4) — `is_assignable_v<T&, optional<U>&>` is false,
- (19.5) — `is_assignable_v<T&, optional<U>&&>` is false,
- (19.6) — `is_assignable_v<T&, const optional<U>&>` is false, and
- (19.7) — `is_assignable_v<T&, const optional<U>&&>` is false.

20 *Effects:* See Table 69.

Table 69 — `optional::operator=(const optional<U>&)` effects [tab:optional.assign.copy.templ]

|                                           | <code>*this</code> contains a value                               | <code>*this</code> does not contain a value                                    |
|-------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>rhs</code> contains a value         | assigns <code>rhs.operator*()</code> to <code>val</code>          | direct-non-list-initializes <code>val</code> with <code>rhs.operator*()</code> |
| <code>rhs</code> does not contain a value | destroys the contained value by calling <code>val.T::~~T()</code> | no effect                                                                      |

21 *Postconditions:* `rhs.has_value() == this->has_value()`.

22 *Returns:* `*this`.

23 *Remarks:* If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the states of `val` and `rhs.val` are determined by the exception safety guarantee of `T`'s assignment.

```
template<class U> constexpr optional<T>& operator=(optional<U>&& rhs);
```

24 *Constraints:*

- (24.1) — `is_constructible_v<T, U>` is true,
- (24.2) — `is_assignable_v<T&, U>` is true,

- (24.3) — *converts-from-any-cvref*<T, optional<U>> is false,  
 (24.4) — *is\_assignable\_v*<T&, optional<U>&> is false,  
 (24.5) — *is\_assignable\_v*<T&, optional<U>&&> is false,  
 (24.6) — *is\_assignable\_v*<T&, const optional<U>&> is false, and  
 (24.7) — *is\_assignable\_v*<T&, const optional<U>&&> is false.

25 *Effects:* See Table 70. The result of the expression `rhs.has_value()` remains unchanged.

Table 70 — `optional::operator=(optional<U>&&)` effects [tab:optional.assign.move.templ]

|                                     | <b>*this contains a value</b>                                     | <b>*this does not contain a value</b>                                               |
|-------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>rhs contains a value</b>         | assigns <code>std::move(rhs).operator*()</code> to <i>val</i>     | direct-non-list-initializes <i>val</i> with <code>std::move(rhs).operator*()</code> |
| <b>rhs does not contain a value</b> | destroys the contained value by calling <code>val.T::~~T()</code> | no effect                                                                           |

26 *Postconditions:* `rhs.has_value() == this->has_value()`.

27 *Returns:* `*this`.

28 *Remarks:* If any exception is thrown, the result of the expression `this->has_value()` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the states of *val* and `rhs.val` are determined by the exception safety guarantee of T's assignment.

template<class... Args> constexpr T& emplace(Args&&... args);

29 *Constraints:* *is\_constructible\_v*<T, Args...> is true.

30 *Effects:* Calls `*this = nullopt`. Then direct-non-list-initializes *val* with `std::forward<Args>(args)...`

31 *Postconditions:* `*this` contains a value.

32 *Returns:* *val*.

33 *Throws:* Any exception thrown by the selected constructor of T.

34 *Remarks:* If an exception is thrown during the call to T's constructor, `*this` does not contain a value, and the previous *val* (if any) has been destroyed.

template<class U, class... Args> constexpr T& emplace(initializer\_list<U> il, Args&&... args);

35 *Constraints:* *is\_constructible\_v*<T, initializer\_list<U>&, Args...> is true.

36 *Effects:* Calls `*this = nullopt`. Then direct-non-list-initializes *val* with `il, std::forward<Args>(args)...`

37 *Postconditions:* `*this` contains a value.

38 *Returns:* *val*.

39 *Throws:* Any exception thrown by the selected constructor of T.

40 *Remarks:* If an exception is thrown during the call to T's constructor, `*this` does not contain a value, and the previous *val* (if any) has been destroyed.

### 22.5.3.5 Swap

[optional.swap]

constexpr void swap(optional& rhs) noexcept(see below);

1 *Mandates:* *is\_move\_constructible\_v*<T> is true.

2 *Preconditions:* T meets the *Cpp17Swappable* requirements (16.4.4.3).

3 *Effects:* See Table 71.

Table 71 — `optional::swap(optional&)` effects [tab:optional.swap]

|                                     | <b>*this contains a value</b>                                                                                                                                                                                                       | <b>*this does not contain a value</b>                                                                                                                                                                                                   |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>rhs contains a value</b>         | calls <code>swap(val, rhs.val)</code>                                                                                                                                                                                               | direct-non-list-initializes <code>val</code> with <code>std::move(rhs.val)</code> , followed by <code>rhs.val.T::~~T()</code> ; postcondition is that <code>*this</code> contains a value and <code>rhs</code> does not contain a value |
| <b>rhs does not contain a value</b> | direct-non-list-initializes <code>rhs.val</code> with <code>std::move(val)</code> , followed by <code>val.T::~~T()</code> ; postcondition is that <code>*this</code> does not contain a value and <code>rhs</code> contains a value | no effect                                                                                                                                                                                                                               |

*Throws:* Any exceptions thrown by the operations in the relevant part of Table 71.

*Remarks:* The exception specification is equivalent to:

```
is_nothrow_move_constructible_v<T> && is_nothrow_swappable_v<T>
```

If any exception is thrown, the results of the expressions `this->has_value()` and `rhs.has_value()` remain unchanged. If an exception is thrown during the call to function `swap`, the states of `val` and `rhs.val` are determined by the exception safety guarantee of `swap` for lvalues of `T`. If an exception is thrown during the call to `T`'s move constructor, the states of `val` and `rhs.val` are determined by the exception safety guarantee of `T`'s move constructor.

### 22.5.3.6 Iterator support

[optional.iterators]

```
using iterator = implementation-defined;
using const_iterator = implementation-defined;
```

These types model `contiguous_iterator` (24.3.4.14), meet the *Cpp17RandomAccessIterator* requirements (24.3.5.7), and meet the requirements for constexpr iterators (24.3.1), with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator` and `const T&` for `const_iterator`.

All requirements on container iterators (23.2.2.2) apply to `optional::iterator` and `optional::const_iterator` as well.

Any operation that initializes or destroys the contained value of an optional object invalidates all iterators into that object.

```
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
```

*Returns:* If `has_value()` is true, an iterator referring to the contained value. Otherwise, a past-the-end iterator value.

```
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
```

*Returns:* `begin() + has_value()`.

### 22.5.3.7 Observers

[optional.observe]

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

*Hardened preconditions:* `has_value()` is true.

*Returns:* `addressof(val)`.

*Remarks:* These functions are constexpr functions.



```
constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
```

4     *Hardened preconditions:* `has_value()` is true.

5     *Returns:* *val*.

6     *Remarks:* These functions are constexpr functions.

```
constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
```

7     *Hardened preconditions:* `has_value()` is true.

8     *Effects:* Equivalent to: `return std::move(val);`

```
constexpr explicit operator bool() const noexcept;
```

9     *Returns:* true if and only if `*this` contains a value.

10    *Remarks:* This function is a constexpr function.

```
constexpr bool has_value() const noexcept;
```

11    *Returns:* true if and only if `*this` contains a value.

12    *Remarks:* This function is a constexpr function.

```
constexpr const T& value() const &;
constexpr T& value() &;
```

13    *Effects:* Equivalent to:

```
return has_value() ? val : throw bad_optional_access();
```

```
constexpr T&& value() &&;
constexpr const T&& value() const &&;
```

14    *Effects:* Equivalent to:

```
return has_value() ? std::move(val) : throw bad_optional_access();
```

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) const &;
```

15    *Mandates:* `is_copy_constructible_v<T>` && `is_convertible_v<U&&, T>` is true.

16    *Effects:* Equivalent to:

```
return has_value() ? val : static_cast<T>(std::forward<U>(v));
```

```
template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) &&;
```

17    *Mandates:* `is_move_constructible_v<T>` && `is_convertible_v<U&&, T>` is true.

18    *Effects:* Equivalent to:

```
return has_value() ? std::move(val) : static_cast<T>(std::forward<U>(v));
```

### 22.5.3.8 Monadic operations

[optional.monadic]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

1     Let `U` be `invoke_result_t<F, decltype((val))>`.

2     *Mandates:* `remove_cvref_t<U>` is a specialization of `optional`.

3     *Effects:* Equivalent to:

```
if (*this) {
 return invoke(std::forward<F>(f), val);
} else {
 return remove_cvref_t<U>();
}
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

4     Let `U` be `invoke_result_t<F, decltype(std::move(val))>`.

5 *Mandates:* `remove_cvref_t<U>` is a specialization of `optional`.

6 *Effects:* Equivalent to:

```
 if (*this) {
 return invoke(std::forward<F>(f), std::move(val));
 } else {
 return remove_cvref_t<U>();
 }
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

7 Let `U` be `remove_cv_t<invoke_result_t<F, decltype((val))>>`.

8 *Mandates:* `U` is a valid contained type for `optional`. The declaration

```
 U u(invoke(std::forward<F>(f), val));
```

is well-formed for some invented variable `u`.

[*Note 1:* There is no requirement that `U` is movable (9.5.1). — *end note*]

9 *Returns:* If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), val)`; otherwise, `optional<U>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

10 Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(val))>>`.

11 *Mandates:* `U` is a valid contained type for `optional`. The declaration

```
 U u(invoke(std::forward<F>(f), std::move(val)));
```

is well-formed for some invented variable `u`.

[*Note 2:* There is no requirement that `U` is movable (9.5.1). — *end note*]

12 *Returns:* If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(val))`; otherwise, `optional<U>()`.

```
template<class F> constexpr optional or_else(F&& f) const &;
```

13 *Constraints:* `F` models invocable and `T` models copy\_constructible.

14 *Mandates:* `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is true.

15 *Effects:* Equivalent to:

```
 if (*this) {
 return *this;
 } else {
 return std::forward<F>(f)();
 }
```

```
template<class F> constexpr optional or_else(F&& f) &&;
```

16 *Constraints:* `F` models invocable and `T` models move\_constructible.

17 *Mandates:* `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is true.

18 *Effects:* Equivalent to:

```
 if (*this) {
 return std::move(*this);
 } else {
 return std::forward<F>(f)();
 }
```

### 22.5.3.9 Modifiers

[optional.mod]

```
constexpr void reset() noexcept;
```

1 *Effects:* If `*this` contains a value, calls `val.T::~~T()` to destroy the contained value; otherwise no effect.

2 *Postconditions:* `*this` does not contain a value.

## 22.5.4 Partial specialization of optional for reference types [optional.optional.ref]

### 22.5.4.1 General [optional.optional.ref.general]

```

namespace std {
 template<class T>
 class optional<T&&> {
 public:
 using value_type = T;
 using iterator = implementation-defined; // present only if T is an object type other than an array
 // of unknown bound; see 22.5.4.5

 // 22.5.4.2, constructors
 constexpr optional() noexcept = default;
 constexpr optional(nullopt_t) noexcept : optional() {}
 constexpr optional(const optional& rhs) noexcept = default;

 template<class Arg>
 constexpr explicit optional(in_place_t, Arg&& arg);
 template<class U>
 constexpr explicit(see below) optional(U&& u) noexcept(see below);
 template<class U>
 constexpr explicit(see below) optional(optional<U>& rhs) noexcept(see below);
 template<class U>
 constexpr explicit(see below) optional(const optional<U>& rhs) noexcept(see below);
 template<class U>
 constexpr explicit(see below) optional(optional<U>&& rhs) noexcept(see below);
 template<class U>
 constexpr explicit(see below) optional(const optional<U>&& rhs) noexcept(see below);

 constexpr ~optional() = default;

 // 22.5.4.3, assignment
 constexpr optional& operator=(nullopt_t) noexcept;
 constexpr optional& operator=(const optional& rhs) noexcept = default;

 template<class U> constexpr T& emplace(U&& u) noexcept(see below);

 // 22.5.4.4, swap
 constexpr void swap(optional& rhs) noexcept;

 // 22.5.4.5, iterator support
 constexpr auto begin() const noexcept;
 constexpr auto end() const noexcept;

 // 22.5.4.6, observers
 constexpr T* operator->() const noexcept;
 constexpr T& operator*() const noexcept;
 constexpr explicit operator bool() const noexcept;
 constexpr bool has_value() const noexcept;
 constexpr T& value() const; // freestanding-deleted
 template<class U = remove_cv_t<T>>
 constexpr remove_cv_t<T> value_or(U&& u) const;

 // 22.5.4.7, monadic operations
 template<class F> constexpr auto and_then(F&& f) const;
 template<class F> constexpr optional<invoke_result_t<F, T&>> transform(F&& f) const;
 template<class F> constexpr optional or_else(F&& f) const;

 // 22.5.4.8, modifiers
 constexpr void reset() noexcept;

 private:
 T* val = nullptr; // exposition only
 }

```

```

// 22.5.4.9, exposition only helper functions
template<class U>
 constexpr void convert-ref-init-val(U&& u); // exposition only
};

```

- <sup>1</sup> An object of type `optional<T&>` contains a value if and only if `val != nullptr` is true. When an `optional<T&>` contains a value, the contained value is a reference to `*val`.
- <sup>2</sup> Each type `optional<T&>` is a trivially copyable class (11.2).

#### 22.5.4.2 Constructors

[optional.ref.ctor]

```

template<class Arg>
 constexpr explicit optional(in_place_t, Arg&& arg);

```

<sup>1</sup> Constraints:

- (1.1) — `is_constructible_v<T&, Arg>` is true, and
- (1.2) — `reference_constructs_from_temporary_v<T&, Arg>` is false.

<sup>2</sup> Effects: Equivalent to: `convert-ref-init-val (std::forward<Arg>(arg))`.

<sup>3</sup> Postconditions: `*this` contains a value.

```

template<class U>
 constexpr explicit(!is_convertible_v<U, T&>)
 optional(U&& u) noexcept(is_nothrow_constructible_v<T&, U>);

```

<sup>4</sup> Constraints:

- (4.1) — `is_same_v<remove_cvref_t<U>, optional>` is false,
- (4.2) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false, and
- (4.3) — `is_constructible_v<T&, U>` is true.

<sup>5</sup> Effects: Equivalent to: `convert-ref-init-val (std::forward<U>(u))`.

<sup>6</sup> Postconditions: `*this` contains a value.

<sup>7</sup> Remarks: This constructor is defined as deleted if

```

 reference_constructs_from_temporary_v<T&, U>
is true.

```

```

template<class U>
 constexpr explicit(!is_convertible_v<U&, T&>)
 optional(optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, U&>);

```

<sup>8</sup> Constraints:

- (8.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is false,
- (8.2) — `is_same_v<T&, U>` is false, and
- (8.3) — `is_constructible_v<T&, U&>` is true.

<sup>9</sup> Effects: Equivalent to:

```

 if (rhs.has_value()) convert-ref-init-val (rhs.operator*());

```

<sup>10</sup> Remarks: This constructor is defined as deleted if

```

 reference_constructs_from_temporary_v<T&, U&>
is true.

```

```

template<class U>
 constexpr explicit(!is_convertible_v<const U&, T&>)
 optional(const optional<U>& rhs) noexcept(is_nothrow_constructible_v<T&, const U&>);

```

<sup>11</sup> Constraints:

- (11.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is false,
- (11.2) — `is_same_v<T&, U>` is false, and

(11.3) — `is_constructible_v<T&, const U&>` is true.

12 *Effects:* Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(rhs.operator*());
```

13 *Remarks:* This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, const U&>
```

is true.

```
template<class U>
constexpr explicit(!is_convertible_v<U, T&>)
optional(optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, U>);
```

14 *Constraints:*

(14.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is false,

(14.2) — `is_same_v<T&, U>` is false, and

(14.3) — `is_constructible_v<T&, U>` is true.

15 *Effects:* Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(std::move(rhs).operator*());
```

16 *Remarks:* This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, U>
```

is true.

```
template<class U>
constexpr explicit(!is_convertible_v<const U, T&>)
optional(const optional<U>&& rhs) noexcept(is_nothrow_constructible_v<T&, const U>);
```

17 *Constraints:*

(17.1) — `is_same_v<remove_cv_t<T>, optional<U>>` is false,

(17.2) — `is_same_v<T&, U>` is false, and

(17.3) — `is_constructible_v<T&, const U>` is true.

18 *Effects:* Equivalent to:

```
if (rhs.has_value()) convert-ref-init-val(std::move(rhs).operator*());
```

19 *Remarks:* This constructor is defined as deleted if

```
reference_constructs_from_temporary_v<T&, const U>
```

is true.

#### 22.5.4.3 Assignment

[optional.ref.assign]

```
constexpr optional& operator=(nullopt_t) noexcept;
```

1 *Effects:* Assigns `nullptr` to *val*.

2 *Postconditions:* `*this` does not contain a value.

3 *Returns:* `*this`.

```
template<class U>
constexpr T& emplace(U&& u) noexcept(is_nothrow_constructible_v<T&, U>);
```

4 *Constraints:*

(4.1) — `is_constructible_v<T&, U>` is true, and

(4.2) — `reference_constructs_from_temporary_v<T&, U>` is false.

5 *Effects:* Equivalent to: `convert-ref-init-val(std::forward<U>(u))`.

6 *Returns:* `*val`.

**22.5.4.4 Swap****[optional.ref.swap]**

```
constexpr void swap(optional& rhs) noexcept;
```

1     *Effects:* Equivalent to: `std::swap(val, rhs.val)`.

**22.5.4.5 Iterator support****[optional.ref.iterators]**

```
using iterator = implementation-defined; // present only if T is an object type other than an array
 // of unknown bound
```

1     This type models `contiguous_iterator` (24.3.4.14), meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7), and meets the requirements for `constexpr` iterators (24.3.1), with value type `remove_cv_t<T>`. The reference type is `T&` for `iterator`.

2     All requirements on container iterators (23.2.2.2) apply to `optional::iterator`.

```
constexpr auto begin() const noexcept;
```

3     *Constraints:* T is an object type other than an array of unknown bound.

4     *Returns:* An object `i` of type `iterator`, such that `i` is an iterator referring to `*val` if `has_value()` is `true`, and a past-the-end iterator value otherwise.

```
constexpr auto end() const noexcept;
```

5     *Constraints:* T is an object type other than an array of unknown bound.

6     *Returns:* `begin() + has_value()`.

**22.5.4.6 Observers****[optional.ref.observe]**

```
constexpr T* operator->() const noexcept;
```

1     *Hardened preconditions:* `has_value()` is `true`.

2     *Returns:* `val`.

```
constexpr T& operator*() const noexcept;
```

3     *Hardened preconditions:* `has_value()` is `true`.

4     *Returns:* `*val`.

```
constexpr explicit operator bool() const noexcept;
```

5     *Returns:* `val != nullptr`.

```
constexpr bool has_value() const noexcept;
```

6     *Returns:* `val != nullptr`.

```
constexpr T& value() const;
```

7     *Effects:* Equivalent to:

```
return has_value() ? *val : throw bad_optional_access();
```

```
template<class U = remove_cv_t<T>> constexpr remove_cv_t<T> value_or(U&& u) const;
```

8     *Constraints:* T is a non-array object type.

9     Let X be `remove_cv_t<T>`.

10    *Mandates:* `is_constructible_v<X, T&> && is_convertible_v<U, X>` is `true`.

11    *Effects:* Equivalent to:

```
return has_value() ? *val : static_cast<X>(std::forward<U>(u));
```

12    *Remarks:* The return type is unspecified if T is an array type or a non-object type.

[Note 1: This is to avoid the declaration being ill-formed. — end note]

**22.5.4.7 Monadic operations****[optional.ref.monadic]**

```
template<class F> constexpr auto and_then(F&& f) const;
```

1     Let U be `invoke_result_t<F, T&>`.

2 *Mandates:* `remove_cvref_t<U>` is a specialization of `optional`.

3 *Effects:* Equivalent to:

```
 if (has_value()) {
 return invoke(std::forward<F>(f), *val);
 } else {
 return remove_cvref_t<U>();
 }
```

```
template<class F>
constexpr optional<remove_cv_t<invoke_result_t<F, T>>> transform(F&& f) const;
```

4 Let `U` be `remove_cv_t<invoke_result_t<F, T>>>`.

5 *Mandates:* `U` is a valid contained type for `optional`. The declaration

```
 U u(invoke(std::forward<F>(f), *val));
```

is well-formed for some invented variable `u`.

[*Note 1:* There is no requirement that `U` is movable (9.5.1). — end note]

6 *Returns:* If `*this` contains a value, an `optional<U>` object whose contained value is direct-non-list-initialized with `invoke(std::forward<F>(f), *val)`; otherwise, `optional<U>()`.

```
template<class F> constexpr optional or_else(F&& f) const;
```

7 *Constraints:* `F` models invocable.

8 *Mandates:* `is_same_v<remove_cvref_t<invoke_result_t<F>>, optional>` is true.

9 *Effects:* Equivalent to:

```
 if (has_value()) {
 return *val;
 } else {
 return std::forward<F>(f)();
 }
```

#### 22.5.4.8 Modifiers

[optional.ref.mod]

```
constexpr void reset() noexcept;
```

1 *Effects:* Assigns `nullptr` to `val`.

2 *Postconditions:* `*this` does not contain a value.

#### 22.5.4.9 Exposition only helper functions

[optional.ref.expos]

```
template<class U>
constexpr void convert_ref_init_val(U&& u); // exposition only
```

1 *Effects:* Creates a variable `r` as if by `T& r(std::forward<U>(u))`; and then initializes `val` with `addressof(r)`.

#### 22.5.5 No-value state indicator

[optional.nullopt]

```
struct nullopt_t{see below};
inline constexpr nullopt_t nullopt(unspecified);
```

1 The struct `nullopt_t` is an empty class type used as a unique type to indicate the state of not containing a value for `optional` objects. In particular, `optional<T>` has a constructor with `nullopt_t` as a single argument; this indicates that an optional object not containing a value shall be constructed.

2 Type `nullopt_t` does not have a default constructor or an initializer-list constructor, and is not an aggregate. `nullopt_t` models copyable and `three_way_comparable<strong_ordering>`.

#### 22.5.6 Class `bad_optional_access`

[optional.bad.access]

```
namespace std {
 class bad_optional_access : public exception {
 public:
 // see 17.9.3 for the specification of the special member functions
 constexpr const char* what() const noexcept override;
```

```
};
}
```

- 1 The class `bad_optional_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of an optional object that does not contain a value.

```
constexpr const char* what() const noexcept override;
```

- 2 *Returns:* An implementation-defined NTBS, which during constant evaluation is encoded with the ordinary literal encoding (5.13.3).

## 22.5.7 Relational operators

[optional.relops]

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const optional<U>& y);
```

- 1 *Constraints:* The expression `*x == *y` is well-formed and its result is convertible to `bool`.

[Note 1: T need not be *Cpp17EqualityComparable*. — end note]

- 2 *Returns:* If `x.has_value() != y.has_value()`, `false`; otherwise if `x.has_value() == false`, `true`; otherwise `*x == *y`.

- 3 *Remarks:* Specializations of this function template for which `*x == *y` is a core constant expression are `constexpr` functions.

```
template<class T, class U> constexpr bool operator!=(const optional<T>& x, const optional<U>& y);
```

- 4 *Constraints:* The expression `*x != *y` is well-formed and its result is convertible to `bool`.

- 5 *Returns:* If `x.has_value() != y.has_value()`, `true`; otherwise, if `x.has_value() == false`, `false`; otherwise `*x != *y`.

- 6 *Remarks:* Specializations of this function template for which `*x != *y` is a core constant expression are `constexpr` functions.

```
template<class T, class U> constexpr bool operator<(const optional<T>& x, const optional<U>& y);
```

- 7 *Constraints:* `*x < *y` is well-formed and its result is convertible to `bool`.

- 8 *Returns:* If `!y`, `false`; otherwise, if `!x`, `true`; otherwise `*x < *y`.

- 9 *Remarks:* Specializations of this function template for which `*x < *y` is a core constant expression are `constexpr` functions.

```
template<class T, class U> constexpr bool operator>(const optional<T>& x, const optional<U>& y);
```

- 10 *Constraints:* The expression `*x > *y` is well-formed and its result is convertible to `bool`.

- 11 *Returns:* If `!x`, `false`; otherwise, if `!y`, `true`; otherwise `*x > *y`.

- 12 *Remarks:* Specializations of this function template for which `*x > *y` is a core constant expression are `constexpr` functions.

```
template<class T, class U> constexpr bool operator<=(const optional<T>& x, const optional<U>& y);
```

- 13 *Constraints:* The expression `*x <= *y` is well-formed and its result is convertible to `bool`.

- 14 *Returns:* If `!x`, `true`; otherwise, if `!y`, `false`; otherwise `*x <= *y`.

- 15 *Remarks:* Specializations of this function template for which `*x <= *y` is a core constant expression are `constexpr` functions.

```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const optional<U>& y);
```

- 16 *Constraints:* The expression `*x >= *y` is well-formed and its result is convertible to `bool`.

- 17 *Returns:* If `!y`, `true`; otherwise, if `!x`, `false`; otherwise `*x >= *y`.

- 18 *Remarks:* Specializations of this function template for which `*x >= *y` is a core constant expression are `constexpr` functions.

```
template<class T, three_way_comparable_with<T> U>
constexpr compare_three_way_result_t<T, U>
operator<=>(const optional<T>& x, const optional<U>& y);
```

- 19 *Returns:* If `x && y`, `*x <=> *y`; otherwise `x.has_value() <=> y.has_value()`.



20 *Remarks:* Specializations of this function template for which  $*x \leq *y$  is a core constant expression are constexpr functions.

## 22.5.8 Comparison with nullopt

[optional.nullopts]

```
template<class T> constexpr bool operator==(const optional<T>& x, nullopt_t) noexcept;
```

1 *Returns:* !x.

```
template<class T> constexpr strong_ordering operator<=(const optional<T>& x, nullopt_t) noexcept;
```

2 *Returns:* x.has\_value() <=> false.

## 22.5.9 Comparison with T

[optional.comp.with.t]

```
template<class T, class U> constexpr bool operator==(const optional<T>& x, const U& v);
```

1 *Constraints:* U is not a specialization of optional. The expression  $*x == v$  is well-formed and its result is convertible to bool.

[Note 1: T need not be *Cpp17EqualityComparable*. — end note]

2 *Effects:* Equivalent to:

```
if (x.has_value())
 return *x == v;
return false;
```

```
template<class T, class U> constexpr bool operator==(const T& v, const optional<U>& x);
```

3 *Constraints:* T is not a specialization of optional. The expression  $v == *x$  is well-formed and its result is convertible to bool.

4 *Effects:* Equivalent to:

```
if (x.has_value())
 return v == *x;
return false;
```

```
template<class T, class U> constexpr bool operator!=(const optional<T>& x, const U& v);
```

5 *Constraints:* U is not a specialization of optional. The expression  $*x != v$  is well-formed and its result is convertible to bool.

6 *Effects:* Equivalent to:

```
if (x.has_value())
 return *x != v;
return true;
```

```
template<class T, class U> constexpr bool operator!=(const T& v, const optional<U>& x);
```

7 *Constraints:* T is not a specialization of optional. The expression  $v != *x$  is well-formed and its result is convertible to bool.

8 *Effects:* Equivalent to:

```
if (x.has_value())
 return v != *x;
return true;
```

```
template<class T, class U> constexpr bool operator<(const optional<T>& x, const U& v);
```

9 *Constraints:* U is not a specialization of optional. The expression  $*x < v$  is well-formed and its result is convertible to bool.

10 *Effects:* Equivalent to:

```
if (x.has_value())
 return *x < v;
return true;
```

```
template<class T, class U> constexpr bool operator<(const T& v, const optional<U>& x);
```

11 *Constraints:* T is not a specialization of optional. The expression  $v < *x$  is well-formed and its result is convertible to bool.

12 *Effects:* Equivalent to:

```
 if (x.has_value())
 return v < *x;
 return false;
```

```
template<class T, class U> constexpr bool operator>(const optional<T>& x, const U& v);
```

13 *Constraints:* U is not a specialization of `optional`. The expression `*x > v` is well-formed and its result is convertible to `bool`.

14 *Effects:* Equivalent to:

```
 if (x.has_value())
 return *x > v;
 return false;
```

```
template<class T, class U> constexpr bool operator>(const T& v, const optional<U>& x);
```

15 *Constraints:* T is not a specialization of `optional`. The expression `v > *x` is well-formed and its result is convertible to `bool`.

16 *Effects:* Equivalent to:

```
 if (x.has_value())
 return v > *x;
 return true;
```

```
template<class T, class U> constexpr bool operator<=(const optional<T>& x, const U& v);
```

17 *Constraints:* U is not a specialization of `optional`. The expression `*x <= v` is well-formed and its result is convertible to `bool`.

18 *Effects:* Equivalent to:

```
 if (x.has_value())
 return *x <= v;
 return true;
```

```
template<class T, class U> constexpr bool operator<=(const T& v, const optional<U>& x);
```

19 *Constraints:* T is not a specialization of `optional`. The expression `v <= *x` is well-formed and its result is convertible to `bool`.

20 *Effects:* Equivalent to:

```
 if (x.has_value())
 return v <= *x;
 return false;
```

```
template<class T, class U> constexpr bool operator>=(const optional<T>& x, const U& v);
```

21 *Constraints:* U is not a specialization of `optional`. The expression `*x >= v` is well-formed and its result is convertible to `bool`.

22 *Effects:* Equivalent to:

```
 if (x.has_value())
 return *x >= v;
 return false;
```

```
template<class T, class U> constexpr bool operator>=(const T& v, const optional<U>& x);
```

23 *Constraints:* T is not a specialization of `optional`. The expression `v >= *x` is well-formed and its result is convertible to `bool`.

24 *Effects:* Equivalent to:

```
 if (x.has_value())
 return v >= *x;
 return true;
```

```
template<class T, class U>
requires (!is-derived-from-optional<U>) && three_way_comparable_with<T, U>
constexpr compare_three_way_result_t<T, U>
operator<=>(const optional<T>& x, const U& v);
```

25 *Effects:* Equivalent to: `return x.has_value() ? *x <=> v : strong_ordering::less;`

## 22.5.10 Specialized algorithms [optional.specalg]

```
template<class T>
constexpr void swap(optional<T>& x, optional<T>& y) noexcept(noexcept(x.swap(y)));
```

1 *Constraints:*

```
is_reference_v<T> || (is_move_constructible_v<T> && is_swappable_v<T>)
is true.
```

2 *Effects:* Calls `x.swap(y)`.

```
template<class T> constexpr optional<decay_t<T>> make_optional(T&& v);
```

3 *Constraints:* The call to `make_optional` does not use an explicit *template-argument-list* that begins with a type *template-argument*.

4 *Effects:* Equivalent to: `return optional<decay_t<T>>(std::forward<T>(v));`

```
template<class T, class... Args>
constexpr optional<T> make_optional(Args&&... args);
```

5 *Effects:* Equivalent to: `return optional<T>(in_place, std::forward<Args>(args)...);`

```
template<class T, class U, class... Args>
constexpr optional<T> make_optional(initializer_list<U> il, Args&&... args);
```

6 *Effects:* Equivalent to: `return optional<T>(in_place, il, std::forward<Args>(args)...);`

## 22.5.11 Hash support [optional.hash]

```
template<class T> struct hash<optional<T>>;
```

1 The specialization `hash<optional<T>>` is enabled (22.10.19) if and only if `hash<remove_const_t<T>>` is enabled. When enabled, for an object `o` of type `optional<T>`, if `o.has_value() == true`, then `hash<optional<T>>()(o)` evaluates to the same value as `hash<remove_const_t<T>>()(*o)`; otherwise it evaluates to an unspecified value. The member functions are not guaranteed to be `noexcept`.

## 22.6 Variants [variant]

### 22.6.1 General [variant.general]

1 A variant object holds and manages the lifetime of a value. If the `variant` holds a value, that value's type has to be one of the template argument types given to `variant`. These template arguments are called alternatives.

2 In 22.6, *GET* denotes a set of exposition-only function templates (22.6.5).

### 22.6.2 Header <variant> synopsis [variant.syn]

```
// mostly freestanding
#include <compare> // see 17.12.1

namespace std {
// 22.6.3, class template variant
template<class... Types>
class variant;

// 22.6.4, variant helper classes
template<class T> struct variant_size; // not defined
template<class T> struct variant_size<const T>;
template<class T>
constexpr size_t variant_size_v = variant_size<T>::value;
```

```

template<class... Types>
 struct variant_size<variant<Types...>>;

template<size_t I, class T> struct variant_alternative; // not defined
template<size_t I, class T> struct variant_alternative<I, const T>;
template<size_t I, class T>
 using variant_alternative_t = variant_alternative<I, T>::type;

template<size_t I, class... Types>
 struct variant_alternative<I, variant<Types...>>;

inline constexpr size_t variant_npos = -1;

// 22.6.5, value access
template<class T, class... Types>
 constexpr bool holds_alternative(const variant<Types...>&) noexcept;

template<size_t I, class... Types>
 constexpr variant_alternative_t<I, variant<Types...>>&
 get(variant<Types...>&); // freestanding-deleted
template<size_t I, class... Types>
 constexpr variant_alternative_t<I, variant<Types...>>&&
 get(variant<Types...>&&); // freestanding-deleted
template<size_t I, class... Types>
 constexpr const variant_alternative_t<I, variant<Types...>>&
 get(const variant<Types...>&); // freestanding-deleted
template<size_t I, class... Types>
 constexpr const variant_alternative_t<I, variant<Types...>>&&
 get(const variant<Types...>&&); // freestanding-deleted

template<class T, class... Types>
 constexpr T& get(variant<Types...>&); // freestanding-deleted
template<class T, class... Types>
 constexpr T&& get(variant<Types...>&&); // freestanding-deleted
template<class T, class... Types>
 constexpr const T& get(const variant<Types...>&); // freestanding-deleted
template<class T, class... Types>
 constexpr const T&& get(const variant<Types...>&&); // freestanding-deleted

template<size_t I, class... Types>
 constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>>
 get_if(variant<Types...>*) noexcept;
template<size_t I, class... Types>
 constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>>
 get_if(const variant<Types...>*) noexcept;

template<class T, class... Types>
 constexpr add_pointer_t<T>
 get_if(variant<Types...>*) noexcept;
template<class T, class... Types>
 constexpr add_pointer_t<const T>
 get_if(const variant<Types...>*) noexcept;

// 22.6.6, relational operators
template<class... Types>
 constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
 constexpr bool operator!=(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
 constexpr bool operator<(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
 constexpr bool operator>(const variant<Types...>&, const variant<Types...>&);
template<class... Types>
 constexpr bool operator<=(const variant<Types...>&, const variant<Types...>&);

```

```

template<class... Types>
 constexpr bool operator==(const variant<Types...>&, const variant<Types...>&);
template<class... Types> requires (three_way_comparable<Types> && ...)
 constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
 operator<=>(const variant<Types...>&, const variant<Types...>&);

// 22.6.7, visitation
template<class Visitor, class... Variants>
 constexpr see below visit(Visitor&&, Variants&&...);
template<class R, class Visitor, class... Variants>
 constexpr R visit(Visitor&&, Variants&&...);

// 22.6.8, class monostate
struct monostate;

// 22.6.9, monostate relational operators
constexpr bool operator==(monostate, monostate) noexcept;
constexpr strong_ordering operator<=>(monostate, monostate) noexcept;

// 22.6.10, specialized algorithms
template<class... Types>
 constexpr void swap(variant<Types...>&, variant<Types...>&) noexcept(see below);

// 22.6.11, class bad_variant_access
class bad_variant_access;

// 22.6.12, hash support
template<class T> struct hash;
template<class... Types> struct hash<variant<Types...>>;
template<> struct hash<monostate>;
}

```

## 22.6.3 Class template variant

[variant.variant]

### 22.6.3.1 General

[variant.variant.general]

```

namespace std {
 template<class... Types>
 class variant {
 public:
 // 22.6.3.2, constructors
 constexpr variant() noexcept(see below);
 constexpr variant(const variant&) noexcept(see below);
 constexpr variant(variant&&) noexcept(see below);

 template<class T>
 constexpr variant(T&&) noexcept(see below);

 template<class T, class... Args>
 constexpr explicit variant(in_place_type_t<T>, Args&&...);
 template<class T, class U, class... Args>
 constexpr explicit variant(in_place_type_t<T>, initializer_list<U>, Args&&...);

 template<size_t I, class... Args>
 constexpr explicit variant(in_place_index_t<I>, Args&&...);
 template<size_t I, class U, class... Args>
 constexpr explicit variant(in_place_index_t<I>, initializer_list<U>, Args&&...);

 // 22.6.3.3, destructor
 constexpr ~variant();

 // 22.6.3.4, assignment
 constexpr variant& operator=(const variant&);
 constexpr variant& operator=(variant&&) noexcept(see below);
 };
}

```

```

template<class T> constexpr variant& operator=(T&&) noexcept(see below);

// 22.6.3.5, modifiers
template<class T, class... Args>
 constexpr T& emplace(Args&&...);
template<class T, class U, class... Args>
 constexpr T& emplace(initializer_list<U>, Args&&...);
template<size_t I, class... Args>
 constexpr variant_alternative_t<I, variant<Types...>& emplace(Args&&...);
template<size_t I, class U, class... Args>
 constexpr variant_alternative_t<I, variant<Types...>&
 emplace(initializer_list<U>, Args&&...);

// 22.6.3.6, value status
constexpr bool valueless_by_exception() const noexcept;
constexpr size_t index() const noexcept;

// 22.6.3.7, swap
constexpr void swap(variant&) noexcept(see below);

// 22.6.7, visitation
template<class Self, class Visitor>
 constexpr decltype(auto) visit(this Self&&, Visitor&&);
template<class R, class Self, class Visitor>
 constexpr R visit(this Self&&, Visitor&&);
};
}

```

- 1 Any instance of **variant** at any given time either holds a value of one of its alternative types or holds no value. When an instance of **variant** holds a value of alternative type *T*, it means that a value of type *T*, referred to as the **variant** object's *contained value*, is nested within (6.8.2) the **variant** object.
- 2 All types in **Types** shall meet the *Cpp17Destructible* requirements (Table 35).
- 3 A program that instantiates the definition of **variant** with no template arguments is ill-formed.
- 4 If a program declares an explicit or partial specialization of **variant**, the program is ill-formed, no diagnostic required.

### 22.6.3.2 Constructors

[variant.ctor]

- 1 In the descriptions that follow, let *i* be in the range  $[0, \text{sizeof} \dots (\text{Types}))$ , and  $T_i$  be the  $i^{\text{th}}$  type in **Types**.

```
constexpr variant() noexcept(see below);
```

- 2 *Constraints:* `is_default_constructible_v<T0>` is true.
- 3 *Effects:* Constructs a **variant** holding a value-initialized value of type  $T_0$ .
- 4 *Postconditions:* `valueless_by_exception()` is false and `index()` is 0.
- 5 *Throws:* Any exception thrown by the value-initialization of  $T_0$ .
- 6 *Remarks:* This function is **constexpr** if and only if the value-initialization of the alternative type  $T_0$  would be **constexpr-suitable** (9.2.6). The exception specification is equivalent to `is_nothrow_default_constructible_v<T0>`.

[Note 1: See also class **monostate**. — end note]

```
constexpr variant(const variant& w) noexcept(see below);
```

- 7 *Effects:* If *w* holds a value, initializes the **variant** to hold the same alternative as *w* and direct-initializes the contained value with `GET<j>(w)`, where *j* is `w.index()`. Otherwise, initializes the **variant** to not hold a value.
- 8 *Throws:* Any exception thrown by the initialization of the contained value.
- 9 *Remarks:* This constructor is defined as deleted unless `is_copy_constructible_v<Ti>` is true for all *i*. If `is_trivially_copy_constructible_v<Ti>` is true for all *i*, this constructor is trivial. The exception specification is equivalent to the logical AND of `is_nothrow_copy_constructible_v<Ti>` for all *i*.

```
constexpr variant(variant&& w) noexcept(see below);
```

10     *Constraints:* `is_move_constructible_v<Ti>` is true for all *i*.

11     *Effects:* If *w* holds a value, initializes the `variant` to hold the same alternative as *w* and direct-initializes the contained value with `GET<j>(std::move(w))`, where *j* is `w.index()`. Otherwise, initializes the `variant` to not hold a value.

12     *Throws:* Any exception thrown by the initialization of the contained value.

13     *Remarks:* The exception specification is equivalent to the logical AND of `is_nothrow_move_constructible_v<Ti>` for all *i*. If `is_trivially_move_constructible_v<Ti>` is true for all *i*, this constructor is trivial.

```
template<class T> constexpr variant(T&& t) noexcept(see below);
```

14     Let *T<sub>j</sub>* be a type that is determined as follows: build an imaginary function *FUN*(*T<sub>i</sub>*) for each alternative type *T<sub>i</sub>* for which *T<sub>i</sub>* *x*[] = {`std::forward<T>(t)`}; is well-formed for some invented variable *x*. The overload *FUN*(*T<sub>j</sub>*) selected by overload resolution for the expression *FUN*(`std::forward<T>(t)`) defines the alternative *T<sub>j</sub>* which is the type of the contained value after construction.

15     *Constraints:*

(15.1)     — `sizeof...(Types)` is nonzero,

(15.2)     — `is_same_v<remove_cvref_t<T>, variant>` is false,

(15.3)     — `remove_cvref_t<T>` is neither a specialization of `in_place_type_t` nor a specialization of `in_place_index_t`,

(15.4)     — `is_constructible_v<Tj, T>` is true, and

(15.5)     — the expression *FUN*(`std::forward<T>(t)`) (with *FUN* being the above-mentioned set of imaginary functions) is well-formed.

[Note 2:

```
 variant<string, string> v("abc");
```

is ill-formed, as both alternative types have an equally viable constructor for the argument. — end note]

16     *Effects:* Initializes `*this` to hold the alternative type *T<sub>j</sub>* and direct-non-list-initializes the contained value with `std::forward<T>(t)`.

17     *Postconditions:* `holds_alternative<Tj>(*this)` is true.

18     *Throws:* Any exception thrown by the initialization of the contained value.

19     *Remarks:* The exception specification is equivalent to `is_nothrow_constructible_v<Tj, T>`. If *T<sub>j</sub>*'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class T, class... Args> constexpr explicit variant(in_place_type_t<T>, Args&&... args);
```

20     *Constraints:*

(20.1)     — There is exactly one occurrence of *T* in *Types...* and

(20.2)     — `is_constructible_v<T, Args...>` is true.

21     *Effects:* Direct-non-list-initializes the contained value of type *T* with `std::forward<Args>(args)...`

22     *Postconditions:* `holds_alternative<T>(*this)` is true.

23     *Throws:* Any exception thrown by the initialization of the contained value.

24     *Remarks:* If *T*'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

```
template<class T, class U, class... Args>
constexpr explicit variant(in_place_type_t<T>, initializer_list<U> il, Args&&... args);
```

25     *Constraints:*

(25.1)     — There is exactly one occurrence of *T* in *Types...* and

(25.2)     — `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

26 *Effects:* Direct-non-list-initializes the contained value of type `T` with `il`, `std::forward<Args>(args)`....

27 *Postconditions:* `holds_alternative<T>(*this)` is true.

28 *Throws:* Any exception thrown by the initialization of the contained value.

29 *Remarks:* If `T`'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

```
template<size_t I, class... Args> constexpr explicit variant(in_place_index_t<I>, Args&&... args);
```

30 *Constraints:*

(30.1) — `I` is less than `sizeof...(Types)` and

(30.2) — `is_constructible_v<TI, Args...>` is true.

31 *Effects:* Direct-non-list-initializes the contained value of type `TI` with `std::forward<Args>(args)`....

32 *Postconditions:* `index()` is `I`.

33 *Throws:* Any exception thrown by the initialization of the contained value.

34 *Remarks:* If `TI`'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

```
template<size_t I, class U, class... Args>
constexpr explicit variant(in_place_index_t<I>, initializer_list<U> il, Args&&... args);
```

35 *Constraints:*

(35.1) — `I` is less than `sizeof...(Types)` and

(35.2) — `is_constructible_v<TI, initializer_list<U>&, Args...>` is true.

36 *Effects:* Direct-non-list-initializes the contained value of type `TI` with `il`, `std::forward<Args>(args)`....

37 *Postconditions:* `index()` is `I`.

38 *Throws:* Any exception thrown by the initialization of the contained value.

39 *Remarks:* If `TI`'s selected constructor is a constexpr constructor, this constructor is a constexpr constructor.

### 22.6.3.3 Destructor

[variant.dtor]

```
constexpr ~variant();
```

1 *Effects:* If `valueless_by_exception()` is false, destroys the currently contained value.

2 *Remarks:* If `is_trivially_destructible_v<Ti>` is true for all `Ti`, then this destructor is trivial.

### 22.6.3.4 Assignment

[variant.assign]

```
constexpr variant& operator=(const variant& rhs);
```

1 Let `j` be `rhs.index()`.

2 *Effects:*

(2.1) — If neither `*this` nor `rhs` holds a value, there is no effect.

(2.2) — Otherwise, if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value.

(2.3) — Otherwise, if `index() == j`, assigns the value contained in `rhs` to the value contained in `*this`.

(2.4) — Otherwise, if either `is_nothrow_copy_constructible_v<Tj>` is true or `is_nothrow_move_constructible_v<Tj>` is false, equivalent to `emplace<j>(GET<j>(rhs))`.

(2.5) — Otherwise, equivalent to `operator=(variant(rhs))`.

3 *Postconditions:* `index() == rhs.index()`.

4 *Returns:* `*this`.



5 *Remarks:* This operator is defined as deleted unless `is_copy_constructible_v<Ti> && is_copy_assignable_v<Ti>` is true for all  $i$ . If `is_trivially_copy_constructible_v<Ti> && is_trivially_copy_assignable_v<Ti> && is_trivially_destructible_v<Ti>` is true for all  $i$ , this assignment operator is trivial.

```
constexpr variant& operator=(variant&& rhs) noexcept(see below);
```

6 Let  $j$  be `rhs.index()`.

7 *Constraints:* `is_move_constructible_v<Ti> && is_move_assignable_v<Ti>` is true for all  $i$ .

8 *Effects:*

- (8.1) — If neither `*this` nor `rhs` holds a value, there is no effect.
- (8.2) — Otherwise, if `*this` holds a value but `rhs` does not, destroys the value contained in `*this` and sets `*this` to not hold a value.
- (8.3) — Otherwise, if `index() == j`, assigns `GET<j>(std::move(rhs))` to the value contained in `*this`.
- (8.4) — Otherwise, equivalent to `emplace<j>(GET<j>(std::move(rhs)))`.

9 *Returns:* `*this`.

10 *Remarks:* If `is_trivially_move_constructible_v<Ti> && is_trivially_move_assignable_v<Ti> && is_trivially_destructible_v<Ti>` is true for all  $i$ , this assignment operator is trivial. The exception specification is equivalent to `is_nothrow_move_constructible_v<Ti> && is_nothrow_move_assignable_v<Ti>` for all  $i$ .

- (10.1) — If an exception is thrown during the call to  $T_j$ 's move construction (with  $j$  being `rhs.index()`), the `variant` will hold no value.
- (10.2) — If an exception is thrown during the call to  $T_j$ 's move assignment, the state of the contained value is as defined by the exception safety guarantee of  $T_j$ 's move assignment; `index()` will be  $j$ .

```
template<class T> constexpr variant& operator=(T&& t) noexcept(see below);
```

11 Let  $T_j$  be a type that is determined as follows: build an imaginary function  $FUN(T_i)$  for each alternative type  $T_i$  for which  $T_i \ x[] = \{\text{std::forward}<T>(t)\}$ ; is well-formed for some invented variable  $x$ . The overload  $FUN(T_j)$  selected by overload resolution for the expression  $FUN(\text{std::forward}<T>(t))$  defines the alternative  $T_j$  which is the type of the contained value after assignment.

12 *Constraints:*

- (12.1) — `is_same_v<remove_cvref_t<T>, variant>` is false,
- (12.2) — `is_assignable_v<Tj&, T> && is_constructible_v<Tj, T>` is true, and
- (12.3) — the expression  $FUN(\text{std::forward}<T>(t))$  (with  $FUN$  being the above-mentioned set of imaginary functions) is well-formed.

[Note 1:

```
variant<string, string> v;
v = "abc";
```

is ill-formed, as both alternative types have an equally viable constructor for the argument. — end note]

13 *Effects:*

- (13.1) — If `*this` holds a  $T_j$ , assigns `std::forward<T>(t)` to the value contained in `*this`.
- (13.2) — Otherwise, if `is_nothrow_constructible_v<Tj, T> || !is_nothrow_move_constructible_v<Tj>` is true, equivalent to `emplace<j>(std::forward<T>(t))`.
- (13.3) — Otherwise, equivalent to `emplace<j>(Tj(std::forward<T>(t)))`.

14 *Postconditions:* `holds_alternative<Tj is true, with  $T_j$  selected by the imaginary function overload resolution described above.`

15 *Returns:* `*this`.

16 *Remarks:* The exception specification is equivalent to:

```
is_nothrow_assignable_v<Tj&, T> && is_nothrow_constructible_v<Tj, T>
```

- (16.1) — If an exception is thrown during the assignment of `std::forward<T>(t)` to the value contained in `*this`, the state of the contained value and `t` are as defined by the exception safety guarantee of the assignment expression; `valueless_by_exception()` will be `false`.
- (16.2) — If an exception is thrown during the initialization of the contained value, the `variant` object is permitted to not hold a value.

### 22.6.3.5 Modifiers

[variant.mod]

```
template<class T, class... Args> constexpr T& emplace(Args&&... args);
```

1 *Constraints:* `is_constructible_v<T, Args...>` is true, and `T` occurs exactly once in `Types`.

2 *Effects:* Equivalent to:

```
return emplace<I>(std::forward<Args>(args)...);
```

where `I` is the zero-based index of `T` in `Types`.

```
template<class T, class U, class... Args>
```

```
constexpr T& emplace(initializer_list<U> il, Args&&... args);
```

3 *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is true, and `T` occurs exactly once in `Types`.

4 *Effects:* Equivalent to:

```
return emplace<I>(il, std::forward<Args>(args)...);
```

where `I` is the zero-based index of `T` in `Types`.

```
template<size_t I, class... Args>
```

```
constexpr variant_alternative_t<I, variant<Types...>& emplace(Args&&... args);
```

5 *Mandates:* `I < sizeof...(Types)`.

6 *Constraints:* `is_constructible_v<TI, Args...>` is true.

7 *Effects:* Destroys the currently contained value if `valueless_by_exception()` is false. Then direct-non-list-initializes the contained value of type `TI` with the arguments `std::forward<Args>(args)...`

8 *Postconditions:* `index()` is `I`.

9 *Returns:* A reference to the new contained value.

10 *Throws:* Any exception thrown during the initialization of the contained value.

11 *Remarks:* If an exception is thrown during the initialization of the contained value, the `variant` is permitted to not hold a value.

```
template<size_t I, class U, class... Args>
```

```
constexpr variant_alternative_t<I, variant<Types...>& emplace(initializer_list<U> il, Args&&... args);
```

```
emplace(initializer_list<U> il, Args&&... args);
```

12 *Mandates:* `I < sizeof...(Types)`.

13 *Constraints:* `is_constructible_v<TI, initializer_list<U>&, Args...>` is true.

14 *Effects:* Destroys the currently contained value if `valueless_by_exception()` is false. Then direct-non-list-initializes the contained value of type `TI` with `il, std::forward<Args>(args)...`

15 *Postconditions:* `index()` is `I`.

16 *Returns:* A reference to the new contained value.

17 *Throws:* Any exception thrown during the initialization of the contained value.

18 *Remarks:* If an exception is thrown during the initialization of the contained value, the `variant` is permitted to not hold a value.

### 22.6.3.6 Value status

[variant.status]

```
constexpr bool valueless_by_exception() const noexcept;
```

1 *Effects:* Returns `false` if and only if the `variant` holds a value.

- 2 [Note 1: It is possible for a **variant** to hold no value if an exception is thrown during a type-changing assignment or emplacement. The latter means that even a **variant<float, int>** can become **valueless\_by\_exception()**, for instance by

```
 struct S { operator int() { throw 42; } };
 variant<float, int> v{12.f};
 v.emplace<1>(S());
```

— end note]

```
constexpr size_t index() const noexcept;
```

- 3 *Effects:* If **valueless\_by\_exception()** is true, returns **variant\_npos**. Otherwise, returns the zero-based index of the alternative of the contained value.

### 22.6.3.7 Swap

[variant.swap]

```
constexpr void swap(variant& rhs) noexcept(see below);
```

- 1 *Mandates:* **is\_move\_constructible\_v<T<sub>i</sub>>** is true for all *i*.

- 2 *Preconditions:* Each **T<sub>i</sub>** meets the *Cpp17Swappable* requirements (16.4.4.3).

- 3 *Effects:*

- (3.1) — If **valueless\_by\_exception()** && **rhs.valueless\_by\_exception()** no effect.
- (3.2) — Otherwise, if **index() == rhs.index()**, calls **swap(GET<i>(\*this), GET<i>(rhs))** where *i* is **index()**.
- (3.3) — Otherwise, exchanges values of **rhs** and **\*this**.

- 4 *Throws:* If **index() == rhs.index()**, any exception thrown by **swap(GET<i>(\*this), GET<i>(rhs))** with *i* being **index()**. Otherwise, any exception thrown by the move constructor of **T<sub>i</sub>** or **T<sub>j</sub>** with *i* being **index()** and *j* being **rhs.index()**.

- 5 *Remarks:* If an exception is thrown during the call to function **swap(GET<i>(\*this), GET<i>(rhs))**, the states of the contained values of **\*this** and of **rhs** are determined by the exception safety guarantee of **swap** for lvalues of **T<sub>i</sub>** with *i* being **index()**. If an exception is thrown during the exchange of the values of **\*this** and **rhs**, the states of the values of **\*this** and of **rhs** are determined by the exception safety guarantee of **variant**'s move constructor. The exception specification is equivalent to the logical AND of **is\_nothrow\_move\_constructible\_v<T<sub>i</sub>>** && **is\_nothrow\_swappable\_v<T<sub>i</sub>>** for all *i*.

### 22.6.4 variant helper classes

[variant.helper]

```
template<class T> struct variant_size;
```

- 1 All specializations of **variant\_size** meet the *Cpp17UnaryTypeTrait* requirements (21.3.2) with a base characteristic of **integral\_constant<size\_t, N>** for some *N*.

```
template<class T> struct variant_size<const T>;
```

- 2 Let **VS** denote **variant\_size<T>** of the cv-unqualified type **T**. Then each specialization of the template meets the *Cpp17UnaryTypeTrait* requirements (21.3.2) with a base characteristic of **integral\_constant<size\_t, VS::value>**.

```
template<class... Types>
 struct variant_size<variant<Types...>> : integral_constant<size_t, sizeof...(Types)> { };
```

```
template<size_t I, class T> struct variant_alternative<I, const T>;
```

- 3 Let **VA** denote **variant\_alternative<I, T>** of the cv-unqualified type **T**. Then each specialization of the template meets the *Cpp17TransformationTrait* requirements (21.3.2) with a member typedef **type** that names the type **const VA::type**.

```
variant_alternative<I, variant<Types...>>::type
```

- 4 *Mandates:* **I < sizeof...(Types)**.

- 5 *Result:* The type **T<sub>I</sub>**.

## 22.6.5 Value access

[variant.get]

```
template<class T, class... Types>
constexpr bool holds_alternative(const variant<Types...>& v) noexcept;
```

1     *Mandates:* The type T occurs exactly once in Types.

2     *Returns:* true if index() is equal to the zero-based index of T in Types.

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&
 GET(variant<Types...>& v); // exposition only
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&&
 GET(variant<Types...>&& v); // exposition only
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&
 GET(const variant<Types...>& v); // exposition only
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&&
 GET(const variant<Types...>&& v); // exposition only
```

3     *Mandates:* I < sizeof...(Types).

4     *Preconditions:* v.index() is I.

5     *Returns:* A reference to the object stored in the variant.

```
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>& get(variant<Types...>& v);
template<size_t I, class... Types>
constexpr variant_alternative_t<I, variant<Types...>>&& get(variant<Types...>&& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>& get(const variant<Types...>& v);
template<size_t I, class... Types>
constexpr const variant_alternative_t<I, variant<Types...>>&& get(const variant<Types...>&& v);
```

6     *Mandates:* I < sizeof...(Types).

7     *Effects:* If v.index() is I, returns a reference to the object stored in the variant. Otherwise, throws an exception of type bad\_variant\_access.

```
template<class T, class... Types> constexpr T& get(variant<Types...>& v);
template<class T, class... Types> constexpr T&& get(variant<Types...>&& v);
template<class T, class... Types> constexpr const T& get(const variant<Types...>& v);
template<class T, class... Types> constexpr const T&& get(const variant<Types...>&& v);
```

8     *Mandates:* The type T occurs exactly once in Types.

9     *Effects:* If v holds a value of type T, returns a reference to that value. Otherwise, throws an exception of type bad\_variant\_access.

```
template<size_t I, class... Types>
constexpr add_pointer_t<variant_alternative_t<I, variant<Types...>>>
 get_if(variant<Types...>* v) noexcept;
template<size_t I, class... Types>
constexpr add_pointer_t<const variant_alternative_t<I, variant<Types...>>>
 get_if(const variant<Types...>* v) noexcept;
```

10     *Mandates:* I < sizeof...(Types).

11     *Returns:* A pointer to the value stored in the variant, if v != nullptr and v->index() == I. Otherwise, returns nullptr.

```
template<class T, class... Types>
constexpr add_pointer_t<T>
 get_if(variant<Types...>* v) noexcept;
```

```
template<class T, class... Types>
constexpr add_pointer_t<const T>
get_if(const variant<Types...>* v) noexcept;
```

12 *Mandates:* The type T occurs exactly once in Types.

13 *Effects:* Equivalent to: return `get_if<i>(v)`; with *i* being the zero-based index of T in Types.

## 22.6.6 Relational operators

[variant.relops]

```
template<class... Types>
constexpr bool operator==(const variant<Types...>& v, const variant<Types...>& w);
```

1 *Constraints:* `GET<i>(v) == GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

2 *Returns:* If `v.index() != w.index()`, false; otherwise if `v.valueless_by_exception()`, true; otherwise `GET<i>(v) == GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types>
constexpr bool operator!=(const variant<Types...>& v, const variant<Types...>& w);
```

3 *Constraints:* `GET<i>(v) != GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

4 *Returns:* If `v.index() != w.index()`, true; otherwise if `v.valueless_by_exception()`, false; otherwise `GET<i>(v) != GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types>
constexpr bool operator<(const variant<Types...>& v, const variant<Types...>& w);
```

5 *Constraints:* `GET<i>(v) < GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

6 *Returns:* If `w.valueless_by_exception()`, false; otherwise if `v.valueless_by_exception()`, true; otherwise, if `v.index() < w.index()`, true; otherwise if `v.index() > w.index()`, false; otherwise `GET<i>(v) < GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types>
constexpr bool operator>(const variant<Types...>& v, const variant<Types...>& w);
```

7 *Constraints:* `GET<i>(v) > GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

8 *Returns:* If `v.valueless_by_exception()`, false; otherwise if `w.valueless_by_exception()`, true; otherwise, if `v.index() > w.index()`, true; otherwise if `v.index() < w.index()`, false; otherwise `GET<i>(v) > GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types>
constexpr bool operator<=(const variant<Types...>& v, const variant<Types...>& w);
```

9 *Constraints:* `GET<i>(v) <= GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

10 *Returns:* If `v.valueless_by_exception()`, true; otherwise if `w.valueless_by_exception()`, false; otherwise, if `v.index() < w.index()`, true; otherwise if `v.index() > w.index()`, false; otherwise `GET<i>(v) <= GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types>
constexpr bool operator>=(const variant<Types...>& v, const variant<Types...>& w);
```

11 *Constraints:* `GET<i>(v) >= GET<i>(w)` is a valid expression that is convertible to bool, for all *i*.

12 *Returns:* If `w.valueless_by_exception()`, true; otherwise if `v.valueless_by_exception()`, false; otherwise, if `v.index() > w.index()`, true; otherwise if `v.index() < w.index()`, false; otherwise `GET<i>(v) >= GET<i>(w)` with *i* being `v.index()`.

```
template<class... Types> requires (three_way_comparable<Types> && ...)
constexpr common_comparison_category_t<compare_three_way_result_t<Types>...>
operator<=>(const variant<Types...>& v, const variant<Types...>& w);
```

13 *Effects:* Equivalent to:

```
if (v.valueless_by_exception() && w.valueless_by_exception())
 return strong_ordering::equal;
if (v.valueless_by_exception()) return strong_ordering::less;
if (w.valueless_by_exception()) return strong_ordering::greater;
if (auto c = v.index() <=> w.index(); c != 0) return c;
```

return  $GET\langle i \rangle(v) \Leftrightarrow GET\langle i \rangle(w)$ ;  
 with  $i$  being  $v.index()$ .

### 22.6.7 Visitation

[variant.visit]

```
template<class Visitor, class... Variants>
constexpr see below visit(Visitor&& vis, Variants&&... vars);
template<class R, class Visitor, class... Variants>
constexpr R visit(Visitor&& vis, Variants&&... vars);
```

1 Let *as-variant* denote the following exposition-only function templates:

```
template<class... Ts>
constexpr auto&& as-variant(variant<Ts...>& var) { return var; }
template<class... Ts>
constexpr auto&& as-variant(const variant<Ts...>& var) { return var; }
template<class... Ts>
constexpr auto&& as-variant(variant<Ts...>&& var) { return std::move(var); }
template<class... Ts>
constexpr auto&& as-variant(const variant<Ts...>&& var) { return std::move(var); }
```

Let  $n$  be  $\text{sizeof}...(Variants)$ . For each  $0 \leq i < n$ , let  $V_i$  denote the type  $\text{decltype}(as-variant(std::forward<Variants_i>(vars_i)))$ .

2 *Constraints*:  $V_i$  is a valid type for all  $0 \leq i < n$ .

3 Let  $V$  denote the pack of types  $V_i$ .

4 Let  $m$  be a pack of  $n$  values of type `size_t`. Such a pack is valid if  $0 \leq m_i < \text{variant\_size\_v}<\text{remove\_reference\_t}<V_i>>$  for all  $0 \leq i < n$ . For each valid pack  $m$ , let  $e(m)$  denote the expression:

`INVOKE(std::forward<Visitor>(vis), GET<m>(std::forward<V>(vars))...)` // see 22.10.4

for the first form and

`INVOKE<R>(std::forward<Visitor>(vis), GET<m>(std::forward<V>(vars))...)` // see 22.10.4

for the second form.

5 *Mandates*: For each valid pack  $m$ ,  $e(m)$  is a valid expression. All such expressions are of the same type and value category.

6 *Returns*:  $e(m)$ , where  $m$  is the pack for which  $m_i$  is  $as-variant(vars_i).index()$  for all  $0 \leq i < n$ . The return type is  $\text{decltype}(e(m))$  for the first form.

7 *Throws*: `bad_variant_access` if  $(as-variant(vars).valueless\_by\_exception() \parallel \dots)$  is true.

8 *Complexity*: For  $n \leq 1$ , the invocation of the callable object is implemented in constant time, i.e., for  $n = 1$ , it does not depend on the number of alternative types of  $V_0$ . For  $n > 1$ , the invocation of the callable object has no complexity requirements.

```
template<class Self, class Visitor>
constexpr decltype(auto) visit(this Self&& self, Visitor&& vis);
```

9 Let  $V$  be `OVERLOAD_REF(Self&&, COPY_CONST(remove_reference_t<Self>, variant))` (22.2.4).

10 *Constraints*: The call to `visit` does not use an explicit *template-argument-list* that begins with a type *template-argument*.

11 *Effects*: Equivalent to: `return std::visit(std::forward<Visitor>(vis), (V)self);`

```
template<class R, class Self, class Visitor>
constexpr R visit(this Self&& self, Visitor&& vis);
```

12 Let  $V$  be `OVERLOAD_REF(Self&&, COPY_CONST(remove_reference_t<Self>, variant))` (22.2.4).

13 *Effects*: Equivalent to: `return std::visit<R>(std::forward<Visitor>(vis), (V)self);`

**22.6.8 Class monostate****[variant.monostate]**

```
struct monostate{};
```

- <sup>1</sup> The class `monostate` can serve as a first alternative type for a `variant` to make the `variant` type default constructible.

**22.6.9 monostate relational operators****[variant.monostate.relops]**

```
constexpr bool operator==(monostate, monostate) noexcept { return true; }
constexpr strong_ordering operator<=(monostate, monostate) noexcept
{ return strong_ordering::equal; }
```

- <sup>1</sup> [Note 1: `monostate` objects have only a single state; they thus always compare equal. — end note]

**22.6.10 Specialized algorithms****[variant.specalg]**

```
template<class... Types>
constexpr void swap(variant<Types...>& v, variant<Types...>& w) noexcept(see below);
```

- <sup>1</sup> *Constraints:* `is_move_constructible_v<Ti>` && `is_swappable_v<Ti>` is true for all *i*.  
<sup>2</sup> *Effects:* Equivalent to `v.swap(w)`.  
<sup>3</sup> *Remarks:* The exception specification is equivalent to `noexcept(v.swap(w))`.

**22.6.11 Class bad\_variant\_access****[variant.bad.access]**

```
namespace std {
 class bad_variant_access : public exception {
 public:
 // see 17.9.3 for the specification of the special member functions
 constexpr const char* what() const noexcept override;
 };
}
```

- <sup>1</sup> Objects of type `bad_variant_access` are thrown to report invalid accesses to the value of a `variant` object.
- ```
constexpr const char* what() const noexcept override;
```
- ² *Returns:* An implementation-defined NTBS, which during constant evaluation is encoded with the ordinary literal encoding (5.13.3).

22.6.12 Hash support**[variant.hash]**

```
template<class... Types> struct hash<variant<Types...>>;
```

- ¹ The specialization `hash<variant<Types...>>` is enabled (22.10.19) if and only if every specialization in `hash<remove_const_t<Types>>...` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<monostate>;
```

- ² The specialization is enabled (22.10.19).

22.7 Storage for any type**[any]****22.7.1 General****[any.general]**

- ¹ Subclause 22.7 describes components that C++ programs may use to perform operations on objects of a discriminated type.
- ² [Note 1: The discriminated type can contain values of different types but does not attempt conversion between them, i.e., 5 is held strictly as an `int` and is not implicitly convertible either to "5" or to 5.0. This indifference to interpretation but awareness of type effectively allows safe, generic containers of single values, with no scope for surprises from ambiguous conversions. — end note]

22.7.2 Header <any> synopsis**[any.synop]**

```
#include <initializer_list> // see 17.11.2
#include <typeinfo>         // see 17.7.2
```

```

namespace std {
    // 22.7.3, class bad_any_cast
    class bad_any_cast;

    // 22.7.4, class any
    class any;

    // 22.7.5, non-member functions
    void swap(any& x, any& y) noexcept;

    template<class T, class... Args>
        any make_any(Args&&... args);
    template<class T, class U, class... Args>
        any make_any(initializer_list<U> il, Args&&... args);

    template<class T>
        T any_cast(const any& operand);
    template<class T>
        T any_cast(any& operand);
    template<class T>
        T any_cast(any&& operand);

    template<class T>
        const T* any_cast(const any* operand) noexcept;
    template<class T>
        T* any_cast(any* operand) noexcept;
}

```

22.7.3 Class bad_any_cast

[any.bad.any.cast]

```

namespace std {
    class bad_any_cast : public bad_cast {
    public:
        // see 17.9.3 for the specification of the special member functions
        const char* what() const noexcept override;
    };
}

```

- ¹ Objects of type bad_any_cast are thrown by a failed any_cast (22.7.5).

```
const char* what() const noexcept override;
```

- ² Returns: An implementation-defined NTBS.

22.7.4 Class any

[any.class]

22.7.4.1 General

[any.class.general]

```

namespace std {
    class any {
    public:
        // 22.7.4.2, construction and destruction
        constexpr any() noexcept;

        any(const any& other);
        any(any&& other) noexcept;

        template<class T>
            any(T&& value);

        template<class T, class... Args>
            explicit any(in_place_type_t<T>, Args&&...);
        template<class T, class U, class... Args>
            explicit any(in_place_type_t<T>, initializer_list<U>, Args&&...);

        ~any();
    };
}

```



```

// 22.7.4.3, assignments
any& operator=(const any& rhs);
any& operator=(any&& rhs) noexcept;

template<class T>
    any& operator=(T&& rhs);

// 22.7.4.4, modifiers
template<class T, class... Args>
    decay_t<T>& emplace(Args&&...);
template<class T, class U, class... Args>
    decay_t<T>& emplace(initializer_list<U>, Args&&...);
void reset() noexcept;
void swap(any& rhs) noexcept;

// 22.7.4.5, observers
bool has_value() const noexcept;
const type_info& type() const noexcept;
};
}

```

- 1 An object of class `any` stores an instance of any type that meets the constructor requirements or it has no value, and this is referred to as the *state* of the class `any` object. The stored instance is called the *contained value*. Two states are equivalent if either they both have no value, or they both have a value and the contained values are equivalent.
- 2 The non-member `any_cast` functions provide type-safe access to the contained value.
- 3 Implementations should avoid the use of dynamically allocated memory for a small contained value. However, any such small-object optimization shall only be applied to types `T` for which `is_nothrow_move_constructible_v<T>` is true.

[Example 1: A contained value of type `int` could be stored in an internal buffer, not in separately-allocated memory.
— end example]

22.7.4.2 Construction and destruction

[any.cons]

```
constexpr any() noexcept;
```

- 1 *Postconditions:* `has_value()` is false.

```
any(const any& other);
```

- 2 *Effects:* If `other.has_value()` is false, constructs an object that has no value. Otherwise, equivalent to `any(in_place_type<T>, any_cast<const T&>(other))` where `T` is the type of the contained value.
- 3 *Throws:* Any exceptions arising from calling the selected constructor for the contained value.

```
any(any&& other) noexcept;
```

- 4 *Effects:* If `other.has_value()` is false, constructs an object that has no value. Otherwise, constructs an object of type `any` that contains either the contained value of `other`, or contains an object of the same type constructed from the contained value of `other` considering that contained value as an rvalue.

```
template<class T>
    any(T&& value);
```

- 5 Let `VT` be `decay_t<T>`.
- 6 *Constraints:* `VT` is not the same type as `any`, `VT` is not a specialization of `in_place_type_t`, and `is_copy_constructible_v<VT>` is true.
- 7 *Preconditions:* `VT` meets the *Cpp17CopyConstructible* requirements.
- 8 *Effects:* Constructs an object of type `any` that contains an object of type `VT` direct-initialized with `std::forward<T>(value)`.
- 9 *Throws:* Any exception thrown by the selected constructor of `VT`.

```

template<class T, class... Args>
  explicit any(in_place_type_t<T>, Args&&... args);
10   Let VT be decay_t<T>.
11   Constraints: is_copy_constructible_v<VT> is true and is_constructible_v<VT, Args...> is
      true.
12   Preconditions: VT meets the Cpp17CopyConstructible requirements.
13   Effects: Direct-non-list-initializes the contained value of type VT with std::forward<Args>(args)....
14   Postconditions: *this contains a value of type VT.
15   Throws: Any exception thrown by the selected constructor of VT.

template<class T, class U, class... Args>
  explicit any(in_place_type_t<T>, initializer_list<U> il, Args&&... args);
16   Let VT be decay_t<T>.
17   Constraints: is_copy_constructible_v<VT> is true and is_constructible_v<VT, initializer_
      list<U>&, Args...> is true.
18   Preconditions: VT meets the Cpp17CopyConstructible requirements.
19   Effects: Direct-non-list-initializes the contained value of type VT with il, std::forward<Args>(
      args)....
20   Postconditions: *this contains a value.
21   Throws: Any exception thrown by the selected constructor of VT.

~any();
22   Effects: As if by reset().

```

22.7.4.3 Assignment

[any.assign]

```

any& operator=(const any& rhs);
1   Effects: As if by any(rhs).swap(*this). No effects if an exception is thrown.
2   Returns: *this.
3   Throws: Any exceptions arising from the copy constructor for the contained value.

any& operator=(any&& rhs) noexcept;
4   Effects: As if by any(std::move(rhs)).swap(*this).
5   Postconditions: The state of *this is equivalent to the original state of rhs.
6   Returns: *this.

template<class T>
  any& operator=(T&& rhs);
7   Let VT be decay_t<T>.
8   Constraints: VT is not the same type as any and is_copy_constructible_v<VT> is true.
9   Preconditions: VT meets the Cpp17CopyConstructible requirements.
10  Effects: Constructs an object tmp of type any that contains an object of type VT direct-initialized with
      std::forward<T>(rhs), and tmp.swap(*this). No effects if an exception is thrown.
11  Returns: *this.
12  Throws: Any exception thrown by the selected constructor of VT.

```

22.7.4.4 Modifiers

[any.modifiers]

```

template<class T, class... Args>
  decay_t<T>& emplace(Args&&... args);
1   Let VT be decay_t<T>.

```

2 *Constraints:* `is_copy_constructible_v<VT>` is true and `is_constructible_v<VT, Args...>` is true.

3 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

4 *Effects:* Calls `reset()`. Then direct-non-list-initializes the contained value of type VT with `std::forward<Args>(args)...`

5 *Postconditions:* `*this` contains a value.

6 *Returns:* A reference to the new contained value.

7 *Throws:* Any exception thrown by the selected constructor of VT.

8 *Remarks:* If an exception is thrown during the call to VT's constructor, `*this` does not contain a value, and any previously contained value has been destroyed.

```
template<class T, class U, class... Args>
decay_t<T>& emplace(initializer_list<U> il, Args&&... args);
```

9 Let VT be `decay_t<T>`.

10 *Constraints:* `is_copy_constructible_v<VT>` is true and `is_constructible_v<VT, initializer_list<U>&, Args...>` is true.

11 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

12 *Effects:* Calls `reset()`. Then direct-non-list-initializes the contained value of type VT with `il, std::forward<Args>(args)...`

13 *Postconditions:* `*this` contains a value.

14 *Returns:* A reference to the new contained value.

15 *Throws:* Any exception thrown by the selected constructor of VT.

16 *Remarks:* If an exception is thrown during the call to VT's constructor, `*this` does not contain a value, and any previously contained value has been destroyed.

```
void reset() noexcept;
```

17 *Effects:* If `has_value()` is true, destroys the contained value.

18 *Postconditions:* `has_value()` is false.

```
void swap(any& rhs) noexcept;
```

19 *Effects:* Exchanges the states of `*this` and `rhs`.

22.7.4.5 Observers

[any.observers]

```
bool has_value() const noexcept;
```

1 *Returns:* true if `*this` contains an object, otherwise false.

```
const type_info& type() const noexcept;
```

2 *Returns:* `typeid(T)` if `*this` has a contained value of type T, otherwise `typeid(void)`.

3 [Note 1: Useful for querying against types known either at compile time or only at runtime. — end note]

22.7.5 Non-member functions

[any.nonmembers]

```
void swap(any& x, any& y) noexcept;
```

1 *Effects:* Equivalent to `x.swap(y)`.

```
template<class T, class... Args>
any make_any(Args&&... args);
```

2 *Effects:* Equivalent to: `return any(in_place_type<T>, std::forward<Args>(args)...) ;`

```
template<class T, class U, class... Args>
any make_any(initializer_list<U> il, Args&&... args);
```

3 *Effects:* Equivalent to: `return any(in_place_type<T>, il, std::forward<Args>(args)...) ;`

```

template<class T>
    T any_cast(const any& operand);
template<class T>
    T any_cast(any& operand);
template<class T>
    T any_cast(any&& operand);

```

4 Let U be the type `remove_cvref_t<T>`.

5 *Mandates:* For the first overload, `is_constructible_v<T, const U&>` is true. For the second overload, `is_constructible_v<T, U&>` is true. For the third overload, `is_constructible_v<T, U>` is true.

6 *Returns:* For the first and second overload, `static_cast<T>(*any_cast<U>(&operand))`. For the third overload, `static_cast<T>(std::move(*any_cast<U>(&operand)))`.

7 *Throws:* `bad_any_cast` if `operand.type() != typeid(remove_reference_t<T>)`.

8 [*Example 1:*

```

    any x(5);                                // x holds int
    assert(any_cast<int>(x) == 5);            // cast to value
    any_cast<int&&>(x) = 10;                  // cast to reference
    assert(any_cast<int>(x) == 10);

    x = "Meow";                              // x holds const char*
    assert(strcmp(any_cast<const char*>(x), "Meow") == 0);
    any_cast<const char*&>(x) = "Harry";
    assert(strcmp(any_cast<const char*>(x), "Harry") == 0);

    x = string("Meow");                      // x holds string
    string s, s2("Jane");
    s = move(any_cast<string&>(x));            // move from any
    assert(s == "Meow");
    any_cast<string&>(x) = move(s2);           // move to any
    assert(any_cast<const string&>(x) == "Jane");

    string cat("Meow");
    const any y(cat);                        // const y holds string
    assert(any_cast<const string&>(y) == cat);

    any_cast<string&>(y);                     // error: cannot any_cast away const
— end example]

```

```

template<class T>
    const T* any_cast(const any* operand) noexcept;
template<class T>
    T* any_cast(any* operand) noexcept;

```

9 *Mandates:* `is_void_v<T>` is false.

10 *Returns:* If `operand != nullptr` && `operand->type() == typeid(T)` is true, a pointer to the object contained by `operand`; otherwise, `nullptr`.

11 [*Example 2:*

```

    bool is_string(const any& operand) {
        return any_cast<string>(&operand) != nullptr;
    }

```

— end example]

22.8 Expected objects

[expected]

22.8.1 General

[expected.general]

1 Subclause 22.8 describes the class template `expected` that represents expected objects. An `expected<T, E>` object holds an object of type T or an object of type E and manages the lifetime of the contained objects.

22.8.2 Header <expected> synopsis**[expected.syn]**

```

// mostly freestanding
namespace std {
    // 22.8.3, class template unexpected
    template<class E> class unexpected;

    // 22.8.4, class template bad_expected_access
    template<class E> class bad_expected_access;

    // 22.8.5, specialization for void
    template<> class bad_expected_access<void>;

    // in-place construction of unexpected values
    struct unexpect_t {
        explicit unexpect_t() = default;
    };
    inline constexpr unexpect_t unexpect{};

    // 22.8.6, class template expected
    template<class T, class E> class expected;

    // 22.8.7, partial specialization of expected for void types
    template<class T, class E> requires is_void_v<T> class expected<T, E>;
}

```

// partially freestanding

// partially freestanding

22.8.3 Class template unexpected**[expected.unexpected]****22.8.3.1 General****[expected.un.general]**

- ¹ Subclause 22.8.3 describes the class template `unexpected` that represents unexpected objects stored in expected objects.

```

namespace std {
    template<class E>
    class unexpected {
    public:
        // 22.8.3.2, constructors
        constexpr unexpected(const unexpected&) = default;
        constexpr unexpected(unexpected&&) = default;
        template<class Err = E>
            constexpr explicit unexpected(Err&&);
        template<class... Args>
            constexpr explicit unexpected(in_place_t, Args&&...);
        template<class U, class... Args>
            constexpr explicit unexpected(in_place_t, initializer_list<U>, Args&&...);

        constexpr unexpected& operator=(const unexpected&) = default;
        constexpr unexpected& operator=(unexpected&&) = default;

        constexpr const E& error() const & noexcept;
        constexpr E& error() & noexcept;
        constexpr const E&& error() const && noexcept;
        constexpr E&& error() && noexcept;

        constexpr void swap(unexpected& other) noexcept(see below);

        template<class E2>
            friend constexpr bool operator==(const unexpected&, const unexpected<E2>&);

        friend constexpr void swap(unexpected& x, unexpected& y) noexcept(noexcept(x.swap(y)));

    private:
        E unex; // exposition only
    };
}

```

```

    template<class E> unexpected(E) -> unexpected<E>;
}

```

- 2 A program that instantiates the definition of `unexpected` for a non-object type, an array type, a specialization of `unexpected`, or a cv-qualified type is ill-formed.

22.8.3.2 Constructors

[expected.un.cons]

```

template<class Err = E>
constexpr explicit unexpected(Err&& e);

```

- 1 *Constraints:*
- (1.1) — `is_same_v<remove_cvref_t<Err>, unexpected>` is false; and
 - (1.2) — `is_same_v<remove_cvref_t<Err>, in_place_t>` is false; and
 - (1.3) — `is_constructible_v<E, Err>` is true.

2 *Effects:* Direct-non-list-initializes `unex` with `std::forward<Err>(e)`.

3 *Throws:* Any exception thrown by the initialization of `unex`.

```

template<class... Args>
constexpr explicit unexpected(in_place_t, Args&&... args);

```

4 *Constraints:* `is_constructible_v<E, Args...>` is true.

5 *Effects:* Direct-non-list-initializes `unex` with `std::forward<Args>(args)...`

6 *Throws:* Any exception thrown by the initialization of `unex`.

```

template<class U, class... Args>
constexpr explicit unexpected(in_place_t, initializer_list<U> il, Args&&... args);

```

7 *Constraints:* `is_constructible_v<E, initializer_list<U>&, Args...>` is true.

8 *Effects:* Direct-non-list-initializes `unex` with `il, std::forward<Args>(args)...`

9 *Throws:* Any exception thrown by the initialization of `unex`.

22.8.3.3 Observers

[expected.un.obs]

```

constexpr const E& error() const & noexcept;
constexpr E& error() & noexcept;

```

1 *Returns:* `unex`.

```

constexpr E&& error() && noexcept;
constexpr const E&& error() const && noexcept;

```

2 *Returns:* `std::move(unex)`.

22.8.3.4 Swap

[expected.un.swap]

```

constexpr void swap(unexpected& other) noexcept(is_nothrow_swappable_v<E>);

```

1 *Mandates:* `is_swappable_v<E>` is true.

2 *Effects:* Equivalent to: `using std::swap; swap(unex, other.unex);`

```

friend constexpr void swap(unexpected& x, unexpected& y) noexcept(noexcept(x.swap(y)));

```

3 *Constraints:* `is_swappable_v<E>` is true.

4 *Effects:* Equivalent to `x.swap(y)`.

22.8.3.5 Equality operator

[expected.un.eq]

```

template<class E2>
friend constexpr bool operator==(const unexpected& x, const unexpected<E2>& y);

```

1 *Mandates:* The expression `x.error() == y.error()` is well-formed and its result is convertible to `bool`.

2 *Returns:* `x.error() == y.error()`.

22.8.4 Class template `bad_expected_access`**[expected.bad]**

```

namespace std {
    template<class E>
    class bad_expected_access : public bad_expected_access<void> {
    public:
        constexpr explicit bad_expected_access(E);
        constexpr const char* what() const noexcept override;
        constexpr E& error() & noexcept;
        constexpr const E& error() const & noexcept;
        constexpr E&& error() && noexcept;
        constexpr const E&& error() const && noexcept;

    private:
        E unex;           // exposition only
    };
}

```

- ¹ The class template `bad_expected_access` defines the type of objects thrown as exceptions to report the situation where an attempt is made to access the value of an `expected<T, E>` object for which `has_value()` is false.

```
constexpr explicit bad_expected_access(E e);
```

- ² *Effects:* Initializes *unex* with `std::move(e)`.

```
constexpr const E& error() const & noexcept;
constexpr E& error() & noexcept;
```

- ³ *Returns:* *unex*.

```
constexpr E&& error() && noexcept;
constexpr const E&& error() const && noexcept;
```

- ⁴ *Returns:* `std::move(unex)`.

```
constexpr const char* what() const noexcept override;
```

- ⁵ *Returns:* An implementation-defined NTBS, which during constant evaluation is encoded with the ordinary literal encoding (5.13.3).

22.8.5 Class template specialization `bad_expected_access<void>` **[expected.bad.void]**

```

namespace std {
    template<>
    class bad_expected_access<void> : public exception {
    protected:
        constexpr bad_expected_access() noexcept;
        constexpr bad_expected_access(const bad_expected_access&) noexcept;
        constexpr bad_expected_access(bad_expected_access&&) noexcept;
        constexpr bad_expected_access& operator=(const bad_expected_access&) noexcept;
        constexpr bad_expected_access& operator=(bad_expected_access&&) noexcept;
        constexpr ~bad_expected_access();

    public:
        constexpr const char* what() const noexcept override;
    };
}

```

```
constexpr const char* what() const noexcept override;
```

- ¹ *Returns:* An implementation-defined NTBS, which during constant evaluation is encoded with the ordinary literal encoding (5.13.3).

22.8.6 Class template `expected`**[expected.expected]****22.8.6.1 General****[expected.object.general]**

```

namespace std {
    template<class T, class E>

```

```

class expected {
public:
    using value_type = T;
    using error_type = E;
    using unexpected_type = unexpected<E>;

    template<class U>
    using rebind = expected<U, error_type>;

    // 22.8.6.2, constructors
    constexpr expected();
    constexpr expected(const expected&);
    constexpr expected(expected&&) noexcept(see below);
    template<class U, class G>
        constexpr explicit(see below) expected(const expected<U, G>&);
    template<class U, class G>
        constexpr explicit(see below) expected(expected<U, G>&&);

    template<class U = remove_cv_t<T>>
        constexpr explicit(see below) expected(U&& v);

    template<class G>
        constexpr explicit(see below) expected(const unexpected<G>&);
    template<class G>
        constexpr explicit(see below) expected(unexpected<G>&&);

    template<class... Args>
        constexpr explicit expected(in_place_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(in_place_t, initializer_list<U>, Args&&...);
    template<class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);

    // 22.8.6.3, destructor
    constexpr ~expected();

    // 22.8.6.4, assignment
    constexpr expected& operator=(const expected&);
    constexpr expected& operator=(expected&&) noexcept(see below);
    template<class U = remove_cv_t<T>> constexpr expected& operator=(U&&);
    template<class G>
        constexpr expected& operator=(const unexpected<G>&);
    template<class G>
        constexpr expected& operator=(unexpected<G>&&);

    template<class... Args>
        constexpr T& emplace(Args&&...) noexcept;
    template<class U, class... Args>
        constexpr T& emplace(initializer_list<U>, Args&&...) noexcept;

    // 22.8.6.5, swap
    constexpr void swap(expected&) noexcept(see below);
    friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));

    // 22.8.6.6, observers
    constexpr const T* operator->() const noexcept;
    constexpr T* operator->() noexcept;
    constexpr const T& operator*() const & noexcept;
    constexpr T& operator*() & noexcept;
    constexpr const T&& operator*() const && noexcept;
    constexpr T&& operator*() && noexcept;
    constexpr explicit operator bool() const noexcept;

```



```

constexpr bool has_value() const noexcept;
constexpr bool has_error() const noexcept;
constexpr const T& value() const &; // freestanding-deleted
constexpr T& value() &; // freestanding-deleted
constexpr const T&& value() const &&; // freestanding-deleted
constexpr T&& value() &&; // freestanding-deleted
constexpr const E& error() const & noexcept;
constexpr E& error() & noexcept;
constexpr const E&& error() const && noexcept;
constexpr E&& error() && noexcept;
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) const &;
template<class U = remove_cv_t<T>> constexpr T value_or(U&&) &&;
template<class G = E> constexpr E error_or(G&&) const &;
template<class G = E> constexpr E error_or(G&&) &&;

// 22.8.6.7, monadic operations
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &;
template<class F> constexpr auto and_then(F&& f) const &&;
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &;
template<class F> constexpr auto or_else(F&& f) const &&;
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &;
template<class F> constexpr auto transform(F&& f) const &&;
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &;
template<class F> constexpr auto transform_error(F&& f) const &&;

// 22.8.6.8, equality operators
template<class T2, class E2> requires (!is_void_v<T2>)
    friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
template<class T2>
    friend constexpr bool operator==(const expected&, const T2&);
template<class E2>
    friend constexpr bool operator==(const expected&, const unexpected<E2>&);

private:
    bool has_val; // exposition only
    union {
        remove_cv_t<T> val; // exposition only
        E unex; // exposition only
    };
};

```

- ¹ Any object of type `expected<T, E>` either contains a value of type `T` or a value of type `E` nested within (6.8.2) it. Member `has_val` indicates whether the `expected<T, E>` object contains an object of type `T`.
- ² A type `T` is a *valid value type for `expected`*, if `remove_cv_t<T>` is `void` or a complete non-array object type that is not `in_place_t`, `unexpected_t`, or a specialization of `unexpected`. A program which instantiates class template `expected<T, E>` with an argument `T` that is not a valid value type for `expected` is ill-formed. A program that instantiates the definition of the template `expected<T, E>` with a type for the `E` parameter that is not a valid template argument for `unexpected` is ill-formed.
- ³ When `T` is not *cv void*, it shall meet the *Cpp17Destructible* requirements (Table 35). `E` shall meet the *Cpp17Destructible* requirements.

22.8.6.2 Constructors

[expected.object.cons]

1 The exposition-only variable template *converts-from-any-cvref* defined in 22.5.3.2 is used by some constructors for *expected*.

```
constexpr expected();
```

2 *Constraints*: *is_default_constructible_v*<T> is true.

3 *Effects*: Value-initializes *val*.

4 *Postconditions*: *has_value()* is true.

5 *Throws*: Any exception thrown by the initialization of *val*.

```
constexpr expected(const expected& rhs);
```

6 *Effects*: If *rhs.has_value()* is true, direct-non-list-initializes *val* with **rhs*. Otherwise, direct-non-list-initializes *unex* with *rhs.error()*.

7 *Postconditions*: *rhs.has_value() == this->has_value()*.

8 *Throws*: Any exception thrown by the initialization of *val* or *unex*.

9 *Remarks*: This constructor is defined as deleted unless

(9.1) — *is_copy_constructible_v*<T> is true and

(9.2) — *is_copy_constructible_v*<E> is true.

10 This constructor is trivial if

(10.1) — *is_trivially_copy_constructible_v*<T> is true and

(10.2) — *is_trivially_copy_constructible_v*<E> is true.

```
constexpr expected(expected&& rhs) noexcept(see below);
```

11 *Constraints*:

(11.1) — *is_move_constructible_v*<T> is true and

(11.2) — *is_move_constructible_v*<E> is true.

12 *Effects*: If *rhs.has_value()* is true, direct-non-list-initializes *val* with *std::move(*rhs)*. Otherwise, direct-non-list-initializes *unex* with *std::move(rhs.error())*.

13 *Postconditions*: *rhs.has_value()* is unchanged; *rhs.has_value() == this->has_value()* is true.

14 *Throws*: Any exception thrown by the initialization of *val* or *unex*.

15 *Remarks*: The exception specification is equivalent to *is_nothrow_move_constructible_v*<T> && *is_nothrow_move_constructible_v*<E>.

16 This constructor is trivial if

(16.1) — *is_trivially_move_constructible_v*<T> is true and

(16.2) — *is_trivially_move_constructible_v*<E> is true.

```
template<class U, class G>
constexpr explicit(see below) expected(const expected<U, G>& rhs);
template<class U, class G>
constexpr explicit(see below) expected(expected<U, G>&& rhs);
```

17 Let:

(17.1) — UF be *const U&* for the first overload and *U* for the second overload.

(17.2) — GF be *const G&* for the first overload and *G* for the second overload.

18 *Constraints*:

(18.1) — *is_constructible_v*<T, UF> is true; and

(18.2) — *is_constructible_v*<E, GF> is true; and

(18.3) — if T is not *cv bool*, *converts-from-any-cvref*<T, expected<U, G>> is false; and

(18.4) — *is_constructible_v*<unexpected<E>, expected<U, G>&> is false; and

- (18.5) — `is_constructible_v<unexpected<E>, expected<U, G>>` is false; and
- (18.6) — `is_constructible_v<unexpected<E>, const expected<U, G>&>` is false; and
- (18.7) — `is_constructible_v<unexpected<E>, const expected<U, G>>` is false.
- 19 *Effects:* If `rhs.has_value()`, `direct-non-list-initializes` `val` with `std::forward<UF>(*rhs)`. Otherwise, `direct-non-list-initializes` `unex` with `std::forward<GF>(rhs.error())`.
- 20 *Postconditions:* `rhs.has_value()` is unchanged; `rhs.has_value() == this->has_value()` is true.
- 21 *Throws:* Any exception thrown by the initialization of `val` or `unex`.
- 22 *Remarks:* The expression inside `explicit` is equivalent to `!is_convertible_v<UF, T> || !is_convertible_v<GF, E>`.

```
template<class U = remove_cv_t<T>>
constexpr explicit(!is_convertible_v<U, T>) expected(U&& v);
```

23 *Constraints:*

- (23.1) — `is_same_v<remove_cvref_t<U>, in_place_t>` is false; and
- (23.2) — `is_same_v<remove_cvref_t<U>, expected>` is false; and
- (23.3) — `is_same_v<remove_cvref_t<U>, unexpect_t>` is false; and
- (23.4) — `remove_cvref_t<U>` is not a specialization of `unexpected`; and
- (23.5) — `is_constructible_v<T, U>` is true; and
- (23.6) — if `T` is `cv bool`, `remove_cvref_t<U>` is not a specialization of `expected`.

24 *Effects:* `Direct-non-list-initializes` `val` with `std::forward<U>(v)`.

25 *Postconditions:* `has_value()` is true.

26 *Throws:* Any exception thrown by the initialization of `val`.

```
template<class G>
constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);
template<class G>
constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

27 Let `GF` be `const G&` for the first overload and `G` for the second overload.

28 *Constraints:* `is_constructible_v<E, GF>` is true.

29 *Effects:* `Direct-non-list-initializes` `unex` with `std::forward<GF>(e.error())`.

30 *Postconditions:* `has_value()` is false.

31 *Throws:* Any exception thrown by the initialization of `unex`.

```
template<class... Args>
constexpr explicit expected(in_place_t, Args&&... args);
```

32 *Constraints:* `is_constructible_v<T, Args...>` is true.

33 *Effects:* `Direct-non-list-initializes` `val` with `std::forward<Args>(args)...`

34 *Postconditions:* `has_value()` is true.

35 *Throws:* Any exception thrown by the initialization of `val`.

```
template<class U, class... Args>
constexpr explicit expected(in_place_t, initializer_list<U> il, Args&&... args);
```

36 *Constraints:* `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

37 *Effects:* `Direct-non-list-initializes` `val` with `il, std::forward<Args>(args)...`

38 *Postconditions:* `has_value()` is true.

39 *Throws:* Any exception thrown by the initialization of `val`.

```
template<class... Args>
constexpr explicit expected(unexpect_t, Args&&... args);
```

40 *Constraints:* `is_constructible_v<E, Args...>` is true.

41 *Effects:* Direct-non-list-initializes *unex* with `std::forward<Args>(args)...`

42 *Postconditions:* `has_value()` is false.

43 *Throws:* Any exception thrown by the initialization of *unex*.

```
template<class U, class... Args>
constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

44 *Constraints:* `is_constructible_v<E, initializer_list<U>&, Args...>` is true.

45 *Effects:* Direct-non-list-initializes *unex* with `il, std::forward<Args>(args)...`

46 *Postconditions:* `has_value()` is false.

47 *Throws:* Any exception thrown by the initialization of *unex*.

22.8.6.3 Destructor

[expected.object.dtor]

```
constexpr ~expected();
```

1 *Effects:* If `has_value()` is true, destroys *val*, otherwise destroys *unex*.

2 *Remarks:* If `is_trivially_destructible_v<T>` is true, and `is_trivially_destructible_v<E>` is true, then this destructor is a trivial destructor.

22.8.6.4 Assignment

[expected.object.assign]

1 This subclause makes use of the following exposition-only function template:

```
template<class T, class U, class... Args>
constexpr void reinit-expected(T& newval, U& oldval, Args&&... args) { // exposition only
    if constexpr (is_nothrow_constructible_v<T, Args...>) {
        destroy_at(addressof(oldval));
        construct_at(addressof(newval), std::forward<Args>(args)...);
    } else if constexpr (is_nothrow_move_constructible_v<T>) {
        T tmp(std::forward<Args>(args)...);
        destroy_at(addressof(oldval));
        construct_at(addressof(newval), std::move(tmp));
    } else {
        U tmp(std::move(oldval));
        destroy_at(addressof(oldval));
        try {
            construct_at(addressof(newval), std::forward<Args>(args)...);
        } catch (...) {
            construct_at(addressof(oldval), std::move(tmp));
            throw;
        }
    }
}
```

```
constexpr expected& operator=(const expected& rhs);
```

2 *Effects:*

(2.1) — If `this->has_value() && rhs.has_value()` is true, equivalent to `val = *rhs`.

(2.2) — Otherwise, if `this->has_value()` is true, equivalent to:

`reinit-expected(unex, val, rhs.error())`

(2.3) — Otherwise, if `rhs.has_value()` is true, equivalent to:

`reinit-expected(val, unex, *rhs)`

(2.4) — Otherwise, equivalent to `unex = rhs.error()`.

Then, if no exception was thrown, equivalent to: `has_val = rhs.has_value(); return *this;`

3 *Returns:* `*this`.

4 *Remarks:* This operator is defined as deleted unless:

(4.1) — `is_copy_assignable_v<T>` is true and

(4.2) — `is_copy_constructible_v<T>` is true and

- (4.3) — `is_copy_assignable_v<E>` is true and
- (4.4) — `is_copy_constructible_v<E>` is true and
- (4.5) — `is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is true.

5 This operator is trivial if:

- (5.1) — `is_trivially_copy_constructible_v<T>` is true, and
- (5.2) — `is_trivially_copy_assignable_v<T>` is true, and
- (5.3) — `is_trivially_destructible_v<T>` is true, and
- (5.4) — `is_trivially_copy_constructible_v<E>` is true, and
- (5.5) — `is_trivially_copy_assignable_v<E>` is true, and
- (5.6) — `is_trivially_destructible_v<E>` is true.

`constexpr expected& operator=(expected&& rhs) noexcept(see below);`

6 *Constraints:*

- (6.1) — `is_move_constructible_v<T>` is true and
- (6.2) — `is_move_assignable_v<T>` is true and
- (6.3) — `is_move_constructible_v<E>` is true and
- (6.4) — `is_move_assignable_v<E>` is true and
- (6.5) — `is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is true.

7 *Effects:*

- (7.1) — If `this->has_value() && rhs.has_value()` is true, equivalent to `val = std::move(*rhs)`.
- (7.2) — Otherwise, if `this->has_value()` is true, equivalent to:
`reinit-expected(unex, val, std::move(rhs.error()))`
- (7.3) — Otherwise, if `rhs.has_value()` is true, equivalent to:
`reinit-expected(val, unex, std::move(*rhs))`
- (7.4) — Otherwise, equivalent to `unex = std::move(rhs.error())`.

Then, if no exception was thrown, equivalent to: `has_val = rhs.has_value(); return *this;`

8 *Returns: *this.*

9 *Remarks:* The exception specification is equivalent to:

`is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T> &&
is_nothrow_move_assignable_v<E> && is_nothrow_move_constructible_v<E>`

10 This operator is trivial if:

- (10.1) — `is_trivially_move_constructible_v<T>` is true, and
- (10.2) — `is_trivially_move_assignable_v<T>` is true, and
- (10.3) — `is_trivially_destructible_v<T>` is true, and
- (10.4) — `is_trivially_move_constructible_v<E>` is true, and
- (10.5) — `is_trivially_move_assignable_v<E>` is true, and
- (10.6) — `is_trivially_destructible_v<E>` is true.

`template<class U = remove_cv_t<T>>
constexpr expected& operator=(U&& v);`

11 *Constraints:*

- (11.1) — `is_same_v<expected, remove_cvref_t<U>>` is false; and
- (11.2) — `remove_cvref_t<U>` is not a specialization of `unexpected`; and
- (11.3) — `is_constructible_v<T, U>` is true; and
- (11.4) — `is_assignable_v<T&, U>` is true; and

- (11.5) — `is_nothrow_constructible_v<T, U> || is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is true.

12 *Effects:*

- (12.1) — If `has_value()` is true, equivalent to: `val = std::forward<U>(v);`

- (12.2) — Otherwise, equivalent to:

```
reinit-expected(val, unex, std::forward<U>(v));
has_val = true;
```

13 *Returns:* `*this`.

```
template<class G>
constexpr expected& operator=(const unexpected<G>& e);
template<class G>
constexpr expected& operator=(unexpected<G>&& e);
```

14 Let GF be `const G&` for the first overload and `G` for the second overload.

15 *Constraints:*

- (15.1) — `is_constructible_v<E, GF>` is true; and
- (15.2) — `is_assignable_v<E&, GF>` is true; and
- (15.3) — `is_nothrow_constructible_v<E, GF> || is_nothrow_move_constructible_v<T> || is_nothrow_move_constructible_v<E>` is true.

16 *Effects:*

- (16.1) — If `has_value()` is true, equivalent to:

```
reinit-expected(unex, val, std::forward<GF>(e.error()));
has_val = false;
```

- (16.2) — Otherwise, equivalent to: `unex = std::forward<GF>(e.error());`

17 *Returns:* `*this`.

```
template<class... Args>
constexpr T& emplace(Args&&... args) noexcept;
```

18 *Constraints:* `is_nothrow_constructible_v<T, Args...>` is true.

19 *Effects:* Equivalent to:

```
if (has_value()) {
    destroy_at(addressof(val));
} else {
    destroy_at(addressof(unex));
    has_val = true;
}
return *construct_at(addressof(val), std::forward<Args>(args)...);
```

```
template<class U, class... Args>
constexpr T& emplace(initializer_list<U> il, Args&&... args) noexcept;
```

20 *Constraints:* `is_nothrow_constructible_v<T, initializer_list<U>&, Args...>` is true.

21 *Effects:* Equivalent to:

```
if (has_value()) {
    destroy_at(addressof(val));
} else {
    destroy_at(addressof(unex));
    has_val = true;
}
return *construct_at(addressof(val), il, std::forward<Args>(args)...);
```

22.8.6.5 Swap

[expected.object.swap]

`constexpr void swap(expected& rhs) noexcept(see below);`

1 *Constraints:*

- (1.1) — `is_swappable_v<T>` is true and
- (1.2) — `is_swappable_v<E>` is true and
- (1.3) — `is_move_constructible_v<T>` && `is_move_constructible_v<E>` is true, and
- (1.4) — `is_nothrow_move_constructible_v<T>` || `is_nothrow_move_constructible_v<E>` is true.

2 *Effects:* See [Table 72](#).

Table 72 — `swap(expected&)` effects [tab:expected.object.swap]

	<code>this->has_value()</code>	<code>!this->has_value()</code>
<code>rhs.has_value()</code>	equivalent to: using <code>std::swap</code> ; <code>swap(val, rhs.val);</code>	calls <code>rhs.swap(*this)</code>
<code>!rhs.has_value()</code>	<i>see below</i>	equivalent to: using <code>std::swap</code> ; <code>swap(unex, rhs.unex);</code>

For the case where `rhs.has_value()` is false and `this->has_value()` is true, equivalent to:

```

if constexpr (is_nothrow_move_constructible_v<E>) {
    E tmp(std::move(rhs.unex));
    destroy_at(addressof(rhs.unex));
    try {
        construct_at(addressof(rhs.val), std::move(val));
        destroy_at(addressof(val));
        construct_at(addressof(unex), std::move(tmp));
    } catch (...) {
        construct_at(addressof(rhs.unex), std::move(tmp));
        throw;
    }
} else {
    remove_cv_t<T> tmp(std::move(val));
    destroy_at(addressof(val));
    try {
        construct_at(addressof(unex), std::move(rhs.unex));
        destroy_at(addressof(rhs.unex));
        construct_at(addressof(rhs.val), std::move(tmp));
    } catch (...) {
        construct_at(addressof(val), std::move(tmp));
        throw;
    }
}
has_val = false;
rhs.has_val = true;

```

3 *Throws:* Any exception thrown by the expressions in the *Effects*.

4 *Remarks:* The exception specification is equivalent to:

```

is_nothrow_move_constructible_v<T> && is_nothrow_swappable_v<T> &&
is_nothrow_move_constructible_v<E> && is_nothrow_swappable_v<E>

```

```
friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));
```

5 *Effects:* Equivalent to `x.swap(y)`.

22.8.6.6 Observers

[expected.object.obs]

```
constexpr const T* operator->() const noexcept;
constexpr T* operator->() noexcept;
```

1 *Hardened preconditions:* `has_value()` is true.

2 *Returns:* `addressof(val)`.

```

constexpr const T& operator*() const & noexcept;
constexpr T& operator*() & noexcept;
3     Hardened preconditions: has_value() is true.
4     Returns: val.

constexpr T&& operator*() && noexcept;
constexpr const T&& operator*() const && noexcept;
5     Hardened preconditions: has_value() is true.
6     Returns: std::move(val).

constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
7     Returns: has_val.

constexpr bool has_error() const noexcept;
8     Effects: Equivalent to: return !has_value();

constexpr const T& value() const &;
constexpr T& value() &;
9     Mandates: is_copy_constructible_v<E> is true.
10    Returns: val, if has_value() is true.
11    Throws: bad_expected_access(as_const(error())) if has_value() is false.

constexpr T&& value() &&;
constexpr const T&& value() const &&;
12    Mandates: is_copy_constructible_v<E> is true and is_constructible_v<E, decltype(std::
move(error()))> is true.
13    Returns: std::move(val), if has_value() is true.
14    Throws: bad_expected_access(std::move(error())) if has_value() is false.

constexpr const E& error() const & noexcept;
constexpr E& error() & noexcept;
15    Hardened preconditions: has_value() is false.
16    Returns: unex.

constexpr E&& error() && noexcept;
constexpr const E&& error() const && noexcept;
17    Hardened preconditions: has_value() is false.
18    Returns: std::move(unex).

template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) const &;
19    Mandates: is_copy_constructible_v<T> is true and is_convertible_v<U, T> is true.
20    Returns: has_value() ? *this : static_cast<T>(std::forward<U>(v)).

template<class U = remove_cv_t<T>> constexpr T value_or(U&& v) &&;
21    Mandates: is_move_constructible_v<T> is true and is_convertible_v<U, T> is true.
22    Returns: has_value() ? std::move(*this) : static_cast<T>(std::forward<U>(v)).

template<class G = E> constexpr E error_or(G&& e) const &;
23    Mandates: is_copy_constructible_v<E> is true and is_convertible_v<G, E> is true.
24    Returns: std::forward<G>(e) if has_value() is true, error() otherwise.

template<class G = E> constexpr E error_or(G&& e) &&;
25    Mandates: is_move_constructible_v<E> is true and is_convertible_v<G, E> is true.

```


26 *Returns:* `std::forward<G>(e)` if `has_value()` is true, `std::move(error())` otherwise.

22.8.6.7 Monadic operations

[expected.object.monadic]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

1 Let U be `remove_cvref_t<invoke_result_t<F, decltype((val))>>`.

2 *Constraints:* `is_constructible_v<E, decltype(error())>` is true.

3 *Mandates:* U is a specialization of `expected` and `is_same_v<typename U::error_type, E>` is true.

4 *Effects:* Equivalent to:

```
    if (has_value())
        return invoke(std::forward<F>(f), val);
    else
        return U(unexpect, error());
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

5 Let U be `remove_cvref_t<invoke_result_t<F, decltype(std::move(val))>>`.

6 *Constraints:* `is_constructible_v<E, decltype(std::move(error()))>` is true.

7 *Mandates:* U is a specialization of `expected` and `is_same_v<typename U::error_type, E>` is true.

8 *Effects:* Equivalent to:

```
    if (has_value())
        return invoke(std::forward<F>(f), std::move(val));
    else
        return U(unexpect, std::move(error()));
```

```
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) const &;
```

9 Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

10 *Constraints:* `is_constructible_v<T, decltype((val))>` is true.

11 *Mandates:* G is a specialization of `expected` and `is_same_v<typename G::value_type, T>` is true.

12 *Effects:* Equivalent to:

```
    if (has_value())
        return G(in_place, val);
    else
        return invoke(std::forward<F>(f), error());
```

```
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &&;
```

13 Let G be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

14 *Constraints:* `is_constructible_v<T, decltype(std::move(val))>` is true.

15 *Mandates:* G is a specialization of `expected` and `is_same_v<typename G::value_type, T>` is true.

16 *Effects:* Equivalent to:

```
    if (has_value())
        return G(in_place, std::move(val));
    else
        return invoke(std::forward<F>(f), std::move(error()));
```

```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

17 Let U be `remove_cv_t<invoke_result_t<F, decltype((val))>>`.

18 *Constraints:* `is_constructible_v<E, decltype(error())>` is true.

19 *Mandates:* U is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
    U u(invoke(std::forward<F>(f), val));
```

is well-formed.

20 *Effects:*

- (20.1) — If `has_value()` is false, returns `expected<U, E>(unexpected, error())`.
- (20.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose *has_val* member is true and *val* member is direct-non-list-initialized with `invoke(std::forward<F>(f), val)`.
- (20.3) — Otherwise, evaluates `invoke(std::forward<F>(f), val)` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

21 Let `U` be `remove_cv_t<invoke_result_t<F, decltype(std::move(val))>>`.

22 *Constraints:* `is_constructible_v<E, decltype(std::move(error()))>` is true.

23 *Mandates:* `U` is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f), std::move(val)));
```

is well-formed.

24 *Effects:*

- (24.1) — If `has_value()` is false, returns `expected<U, E>(unexpected, std::move(error()))`.
- (24.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose *has_val* member is true and *val* member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(val))`.
- (24.3) — Otherwise, evaluates `invoke(std::forward<F>(f), std::move(val))` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;
```

25 Let `G` be `remove_cv_t<invoke_result_t<F, decltype(error())>>`.

26 *Constraints:* `is_constructible_v<T, decltype((val))>` is true.

27 *Mandates:* `G` is a valid template argument for `unexpected` (22.8.3.1) and the declaration

```
G g(invoke(std::forward<F>(f), error()));
```

is well-formed.

28 *Returns:* If `has_value()` is true, `expected<T, G>(in_place, val)`; otherwise, an `expected<T, G>` object whose *has_val* member is false and *unex* member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

29 Let `G` be `remove_cv_t<invoke_result_t<F, decltype(std::move(error()))>>`.

30 *Constraints:* `is_constructible_v<T, decltype(std::move(val))>` is true.

31 *Mandates:* `G` is a valid template argument for `unexpected` (22.8.3.1) and the declaration

```
G g(invoke(std::forward<F>(f), std::move(error())));
```

is well-formed.

32 *Returns:* If `has_value()` is true, `expected<T, G>(in_place, std::move(val))`; otherwise, an `expected<T, G>` object whose *has_val* member is false and *unex* member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

22.8.6.8 Equality operators

[`expected.object.eq`]

```
template<class T2, class E2> requires (!is_void_v<T2>)
friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

- 1 *Constraints:* The expressions `*x == *y` and `x.error() == y.error()` are well-formed and their results are convertible to `bool`.

2 *Returns:* If `x.has_value()` does not equal `y.has_value()`, false; otherwise if `x.has_value()` is true, `*x == *y`; otherwise `x.error() == y.error()`.

```
template<class T2> friend constexpr bool operator==(const expected& x, const T2& v);
```

3 *Constraints:* T2 is not a specialization of `expected`. The expression `*x == v` is well-formed and its result is convertible to `bool`.

[Note 1: T need not be *Cpp17EqualityComparable*. — end note]

4 *Returns:* If `x.has_value()` is true, `*x == v`; otherwise false.

```
template<class E2> friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

5 *Constraints:* The expression `x.error() == e.error()` is well-formed and its result is convertible to `bool`.

6 *Returns:* If `!x.has_value()` is true, `x.error() == e.error()`; otherwise false.

22.8.7 Partial specialization of `expected` for void types

[`expected.void`]

22.8.7.1 General

[`expected.void.general`]

```
template<class T, class E> requires is_void_v<T>
class expected<T, E> {
public:
    using value_type = T;
    using error_type = E;
    using unexpected_type = unexpected<E>;

    template<class U>
    using rebind = expected<U, error_type>;

    // 22.8.7.2, constructors
    constexpr expected() noexcept;
    constexpr expected(const expected&);
    constexpr expected(expected&&) noexcept(see below);
    template<class U, class G>
        constexpr explicit(see below) expected(const expected<U, G>&);
    template<class U, class G>
        constexpr explicit(see below) expected(expected<U, G>&&);

    template<class G>
        constexpr explicit(see below) expected(const unexpected<G>&);
    template<class G>
        constexpr explicit(see below) expected(unexpected<G>&&);

    constexpr explicit expected(in_place_t) noexcept;
    template<class... Args>
        constexpr explicit expected(unexpect_t, Args&&...);
    template<class U, class... Args>
        constexpr explicit expected(unexpect_t, initializer_list<U>, Args&&...);

    // 22.8.7.3, destructor
    constexpr ~expected();

    // 22.8.7.4, assignment
    constexpr expected& operator=(const expected&);
    constexpr expected& operator=(expected&&) noexcept(see below);
    template<class G>
        constexpr expected& operator=(const unexpected<G>&);
    template<class G>
        constexpr expected& operator=(unexpected<G>&&);
    constexpr void emplace() noexcept;

    // 22.8.7.5, swap
    constexpr void swap(expected&) noexcept(see below);
    friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));
```

```

// 22.8.7.6, observers
constexpr explicit operator bool() const noexcept;
constexpr bool has_value() const noexcept;
constexpr bool has_error() const noexcept;
constexpr void operator*() const noexcept;
constexpr void value() const &; // freestanding-deleted
constexpr void value() &&; // freestanding-deleted
constexpr const E& error() const & noexcept;
constexpr E& error() & noexcept;
constexpr const E&& error() const && noexcept;
constexpr E&& error() && noexcept;
template<class G = E> constexpr E error_or(G&&) const &;
template<class G = E> constexpr E error_or(G&&) &&;

// 22.8.7.7, monadic operations
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &;
template<class F> constexpr auto and_then(F&& f) const &&;
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &;
template<class F> constexpr auto or_else(F&& f) const &&;
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &;
template<class F> constexpr auto transform(F&& f) const &&;
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &;
template<class F> constexpr auto transform_error(F&& f) const &&;

// 22.8.7.8, equality operators
template<class T2, class E2> requires is_void_v<T2>
    friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
template<class E2>
    friend constexpr bool operator==(const expected&, const unexpected<E2>&);

private:
    bool has_val; // exposition only
    union {
        E unex; // exposition only
    };
};

```

- ¹ Any object of type `expected<T, E>` either represents a value of type `T`, or contains a value of type `E` nested within (6.8.2) it. Member `has_val` indicates whether the `expected<T, E>` object represents a value of type `T`.
- ² A program that instantiates the definition of the template `expected<T, E>` with a type for the `E` parameter that is not a valid template argument for `unexpected` is ill-formed.
- ³ `E` shall meet the requirements of *Cpp17Destructible* (Table 35).

22.8.7.2 Constructors

[expected.void.cons]

```
constexpr expected() noexcept;
```

- ¹ *Postconditions:* `has_value()` is true.

```
constexpr expected(const expected& rhs);
```

- ² *Effects:* If `rhs.has_value()` is false, direct-non-list-initializes `unex` with `rhs.error()`.
- ³ *Postconditions:* `rhs.has_value() == this->has_value()`.
- ⁴ *Throws:* Any exception thrown by the initialization of `unex`.

5 *Remarks:* This constructor is defined as deleted unless `is_copy_constructible_v<E>` is true.
 6 This constructor is trivial if `is_trivially_copy_constructible_v<E>` is true.

```
constexpr expected(expected&& rhs) noexcept(is_nothrow_move_constructible_v<E>);
```

7 *Constraints:* `is_move_constructible_v<E>` is true.

8 *Effects:* If `rhs.has_value()` is false, direct-non-list-initializes *unex* with `std::move(rhs.error())`.

9 *Postconditions:* `rhs.has_value()` is unchanged; `rhs.has_value() == this->has_value()` is true.

10 *Throws:* Any exception thrown by the initialization of *unex*.

11 *Remarks:* This constructor is trivial if `is_trivially_move_constructible_v<E>` is true.

```
template<class U, class G>
constexpr explicit(!is_convertible_v<const G&, E>) expected(const expected<U, G>& rhs);
```

```
template<class U, class G>
constexpr explicit(!is_convertible_v<G, E>) expected(expected<U, G>&& rhs);
```

12 Let GF be `const G&` for the first overload and `G` for the second overload.

13 *Constraints:*

(13.1) — `is_void_v<U>` is true; and

(13.2) — `is_constructible_v<E, GF>` is true; and

(13.3) — `is_constructible_v<unexpected<E>, expected<U, G>&>` is false; and

(13.4) — `is_constructible_v<unexpected<E>, expected<U, G>>` is false; and

(13.5) — `is_constructible_v<unexpected<E>, const expected<U, G>&>` is false; and

(13.6) — `is_constructible_v<unexpected<E>, const expected<U, G>>` is false.

14 *Effects:* If `rhs.has_value()` is false, direct-non-list-initializes *unex* with `std::forward<GF>(rhs.error())`.

15 *Postconditions:* `rhs.has_value()` is unchanged; `rhs.has_value() == this->has_value()` is true.

16 *Throws:* Any exception thrown by the initialization of *unex*.

```
template<class G>
constexpr explicit(!is_convertible_v<const G&, E>) expected(const unexpected<G>& e);
```

```
template<class G>
constexpr explicit(!is_convertible_v<G, E>) expected(unexpected<G>&& e);
```

17 Let GF be `const G&` for the first overload and `G` for the second overload.

18 *Constraints:* `is_constructible_v<E, GF>` is true.

19 *Effects:* Direct-non-list-initializes *unex* with `std::forward<GF>(e.error())`.

20 *Postconditions:* `has_value()` is false.

21 *Throws:* Any exception thrown by the initialization of *unex*.

```
constexpr explicit expected(in_place_t) noexcept;
```

22 *Postconditions:* `has_value()` is true.

```
template<class... Args>
constexpr explicit expected(unexpect_t, Args&&... args);
```

23 *Constraints:* `is_constructible_v<E, Args...>` is true.

24 *Effects:* Direct-non-list-initializes *unex* with `std::forward<Args>(args)...`

25 *Postconditions:* `has_value()` is false.

26 *Throws:* Any exception thrown by the initialization of *unex*.

```
template<class U, class... Args>
constexpr explicit expected(unexpect_t, initializer_list<U> il, Args&&... args);
```

27 *Constraints:* `is_constructible_v<E, initializer_list<U>&, Args...>` is true.

28 *Effects:* Direct-non-list-initializes *unex* with `il, std::forward<Args>(args)...`

29 *Postconditions:* `has_value()` is false.

30 *Throws:* Any exception thrown by the initialization of *unex*.

22.8.7.3 Destructor

[expected.void.dtor]

```
constexpr ~expected();
```

1 *Effects:* If `has_value()` is false, destroys *unex*.

2 *Remarks:* If `is_trivially_destructible_v<E>` is true, then this destructor is a trivial destructor.

22.8.7.4 Assignment

[expected.void.assign]

```
constexpr expected& operator=(const expected& rhs);
```

1 *Effects:*

(1.1) — If `this->has_value() && rhs.has_value()` is true, no effects.

(1.2) — Otherwise, if `this->has_value()` is true, equivalent to: `construct_at(addressof(unex), rhs.unex); has_val = false;`

(1.3) — Otherwise, if `rhs.has_value()` is true, destroys *unex* and sets *has_val* to true.

(1.4) — Otherwise, equivalent to `unex = rhs.error()`.

2 *Returns:* **this*.

3 *Remarks:* This operator is defined as deleted unless `is_copy_assignable_v<E>` is true and `is_copy_constructible_v<E>` is true.

4 This operator is trivial if `is_trivially_copy_constructible_v<E>`, `is_trivially_copy_assignable_v<E>`, and `is_trivially_destructible_v<E>` are all true.

```
constexpr expected& operator=(expected&& rhs) noexcept(see below);
```

5 *Constraints:* `is_move_constructible_v<E>` is true and `is_move_assignable_v<E>` is true.

6 *Effects:*

(6.1) — If `this->has_value() && rhs.has_value()` is true, no effects.

(6.2) — Otherwise, if `this->has_value()` is true, equivalent to:

```
construct_at(addressof(unex), std::move(rhs.unex));
has_val = false;
```

(6.3) — Otherwise, if `rhs.has_value()` is true, destroys *unex* and sets *has_val* to true.

(6.4) — Otherwise, equivalent to `unex = std::move(rhs.error())`.

7 *Returns:* **this*.

8 *Remarks:* The exception specification is equivalent to `is_nothrow_move_constructible_v<E> && is_nothrow_move_assignable_v<E>`.

9 This operator is trivial if `is_trivially_move_constructible_v<E>`, `is_trivially_move_assignable_v<E>`, and `is_trivially_destructible_v<E>` are all true.

```
template<class G>
constexpr expected& operator=(const unexpected<G>& e);
template<class G>
constexpr expected& operator=(unexpected<G>&& e);
```

10 Let GF be `const G&` for the first overload and `G` for the second overload.

11 *Constraints:* `is_constructible_v<E, GF>` is true and `is_assignable_v<E&, GF>` is true.

12 *Effects:*

(12.1) — If `has_value()` is true, equivalent to:

```
construct_at(addressof(unex), std::forward<GF>(e.error()));
has_val = false;
```

(12.2) — Otherwise, equivalent to: `unex = std::forward<GF>(e.error())`;

13 *Returns:* **this*.

```
constexpr void emplace() noexcept;
```

14 *Effects:* If `has_value()` is false, destroys *unex* and sets *has_val* to true.

22.8.7.5 Swap

[expected.void.swap]

```
constexpr void swap(expected& rhs) noexcept(see below);
```

1 *Constraints:* `is_swappable_v<E>` is true and `is_move_constructible_v<E>` is true.

2 *Effects:* See Table 73.

Table 73 — `swap(expected&)` effects [tab:expected.void.swap]

<code>this->has_value()</code>		<code>!this->has_value()</code>
<code>rhs.has_value()</code>	no effects	calls <code>rhs.swap(*this)</code>
<code>!rhs.has_value()</code>	see below	equivalent to: using <code>std::swap</code> ; <code>swap(unex, rhs.unex);</code>

For the case where `rhs.has_value()` is false and `this->has_value()` is true, equivalent to:

```
construct_at(addressof(unex), std::move(rhs.unex));
destroy_at(addressof(rhs.unex));
has_val = false;
rhs.has_val = true;
```

3 *Throws:* Any exception thrown by the expressions in the *Effects*.

4 *Remarks:* The exception specification is equivalent to `is_nothrow_move_constructible_v<E> && is_nothrow_swappable_v<E>`.

```
friend constexpr void swap(expected& x, expected& y) noexcept(noexcept(x.swap(y)));
```

5 *Effects:* Equivalent to `x.swap(y)`.

22.8.7.6 Observers

[expected.void.obs]

```
constexpr explicit operator bool() const noexcept;
```

```
constexpr bool has_value() const noexcept;
```

1 *Returns:* *has_val*.

```
constexpr bool has_error() const noexcept;
```

2 *Effects:* Equivalent to: `return !has_value();`

```
constexpr void operator*() const noexcept;
```

3 *Hardened preconditions:* `has_value()` is true.

```
constexpr void value() const &;
```

4 *Mandates:* `is_copy_constructible_v<E>` is true.

5 *Throws:* `bad_expected_access(error())` if `has_value()` is false.

```
constexpr void value() &&;
```

6 *Mandates:* `is_copy_constructible_v<E>` is true and `is_move_constructible_v<E>` is true.

7 *Throws:* `bad_expected_access(std::move(error()))` if `has_value()` is false.

```
constexpr const E& error() const & noexcept;
```

```
constexpr E& error() & noexcept;
```

8 *Hardened preconditions:* `has_value()` is false.

9 *Returns:* *unex*.

```
constexpr E&& error() && noexcept;
```

```
constexpr const E&& error() const && noexcept;
```

10 *Hardened preconditions:* `has_value()` is false.

11 *Returns:* `std::move(unevaluated).`

```
template<class G = E> constexpr E error_or(G&& e) const &;
```

12 *Mandates:* `is_copy_constructible_v<E>` is true and `is_convertible_v<G, E>` is true.

13 *Returns:* `std::forward<G>(e)` if `has_value()` is true, `error()` otherwise.

```
template<class G = E> constexpr E error_or(G&& e) &&;
```

14 *Mandates:* `is_move_constructible_v<E>` is true and `is_convertible_v<G, E>` is true.

15 *Returns:* `std::forward<G>(e)` if `has_value()` is true, `std::move(error())` otherwise.

22.8.7.7 Monadic operations

[expected.void.monadic]

```
template<class F> constexpr auto and_then(F&& f) &;
template<class F> constexpr auto and_then(F&& f) const &;
```

1 Let U be `remove_cvref_t<invoke_result_t<F>>`.

2 *Constraints:* `is_constructible_v<E, decltype(error())>>` is true.

3 *Mandates:* U is a specialization of `expected` and `is_same_v<typename U::error_type, E>` is true.

4 *Effects:* Equivalent to:

```
    if (has_value())
        return invoke(std::forward<F>(f));
    else
        return U(unexpect, error());
```

```
template<class F> constexpr auto and_then(F&& f) &&;
template<class F> constexpr auto and_then(F&& f) const &&;
```

5 Let U be `remove_cvref_t<invoke_result_t<F>>`.

6 *Constraints:* `is_constructible_v<E, decltype(std::move(error()))>` is true.

7 *Mandates:* U is a specialization of `expected` and `is_same_v<typename U::error_type, E>` is true.

8 *Effects:* Equivalent to:

```
    if (has_value())
        return invoke(std::forward<F>(f));
    else
        return U(unexpect, std::move(error()));
```

```
template<class F> constexpr auto or_else(F&& f) &;
template<class F> constexpr auto or_else(F&& f) const &;
```

9 Let G be `remove_cvref_t<invoke_result_t<F, decltype(error())>>`.

10 *Mandates:* G is a specialization of `expected` and `is_same_v<typename G::value_type, T>` is true.

11 *Effects:* Equivalent to:

```
    if (has_value())
        return G();
    else
        return invoke(std::forward<F>(f), error());
```

```
template<class F> constexpr auto or_else(F&& f) &&;
template<class F> constexpr auto or_else(F&& f) const &&;
```

12 Let G be `remove_cvref_t<invoke_result_t<F, decltype(std::move(error()))>>`.

13 *Mandates:* G is a specialization of `expected` and `is_same_v<typename G::value_type, T>` is true.

14 *Effects:* Equivalent to:

```
    if (has_value())
        return G();
    else
        return invoke(std::forward<F>(f), std::move(error()));
```



```
template<class F> constexpr auto transform(F&& f) &;
template<class F> constexpr auto transform(F&& f) const &;
```

15 Let U be `remove_cv_t<invoke_result_t<F>>`.

16 *Constraints:* `is_constructible_v<E, decltype(error())>` is true.

17 *Mandates:* U is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f)));
```

is well-formed.

18 *Effects:*

(18.1) — If `has_value()` is false, returns `expected<U, E>(unexpected, error())`.

(18.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose *has_val* member is true and *val* member is direct-non-list-initialized with `invoke(std::forward<F>(f))`.

(18.3) — Otherwise, evaluates `invoke(std::forward<F>(f))` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform(F&& f) &&;
template<class F> constexpr auto transform(F&& f) const &&;
```

19 Let U be `remove_cv_t<invoke_result_t<F>>`.

20 *Constraints:* `is_constructible_v<E, decltype(std::move(error()))>` is true.

21 *Mandates:* U is a valid value type for `expected`. If `is_void_v<U>` is false, the declaration

```
U u(invoke(std::forward<F>(f)));
```

is well-formed.

22 *Effects:*

(22.1) — If `has_value()` is false, returns `expected<U, E>(unexpected, std::move(error()))`.

(22.2) — Otherwise, if `is_void_v<U>` is false, returns an `expected<U, E>` object whose *has_val* member is true and *val* member is direct-non-list-initialized with `invoke(std::forward<F>(f))`.

(22.3) — Otherwise, evaluates `invoke(std::forward<F>(f))` and then returns `expected<U, E>()`.

```
template<class F> constexpr auto transform_error(F&& f) &;
template<class F> constexpr auto transform_error(F&& f) const &;
```

23 Let G be `remove_cv_t<invoke_result_t<F, decltype(error())>>`.

24 *Mandates:* G is a valid template argument for `unexpected` (22.8.3.1) and the declaration

```
G g(invoke(std::forward<F>(f), error()));
```

is well-formed.

25 *Returns:* If `has_value()` is true, `expected<T, G>()`; otherwise, an `expected<T, G>` object whose *has_val* member is false and *unex* member is direct-non-list-initialized with `invoke(std::forward<F>(f), error())`.

```
template<class F> constexpr auto transform_error(F&& f) &&;
template<class F> constexpr auto transform_error(F&& f) const &&;
```

26 Let G be `remove_cv_t<invoke_result_t<F, decltype(std::move(error()))>>`.

27 *Mandates:* G is a valid template argument for `unexpected` (22.8.3.1) and the declaration

```
G g(invoke(std::forward<F>(f), std::move(error())));
```

is well-formed.

28 *Returns:* If `has_value()` is true, `expected<T, G>()`; otherwise, an `expected<T, G>` object whose *has_val* member is false and *unex* member is direct-non-list-initialized with `invoke(std::forward<F>(f), std::move(error()))`.

22.8.7.8 Equality operators

[expected.void.eq]

```
template<class T2, class E2> requires is_void_v<T2>
friend constexpr bool operator==(const expected& x, const expected<T2, E2>& y);
```

1 *Constraints:* The expression `x.error() == y.error()` is well-formed and its result is convertible to

bool.

- 2 *Returns:* If `x.has_value()` does not equal `y.has_value()`, false; otherwise if `x.has_value()` is true, true; otherwise `x.error() == y.error()`.

```
template<class E2>
```

```
    friend constexpr bool operator==(const expected& x, const unexpected<E2>& e);
```

- 3 *Constraints:* The expression `x.error() == e.error()` is well-formed and its result is convertible to bool.

- 4 *Returns:* If `!x.has_value()` is true, `x.error() == e.error()`; otherwise false.

22.9 Bitsets

[bitset]

22.9.1 Header <bitset> synopsis

[bitset.syn]

- 1 The header <bitset> defines a class template and several related functions for representing and manipulating fixed-size sequences of bits.

```
#include <string>    // see 27.4.2
#include <iosfwd>    // for istream (31.7.1), ostream (31.7.2), see 31.3.1

namespace std {
    template<size_t N> class bitset;

    // 22.9.4, bitset operators
    template<size_t N>
        constexpr bitset<N> operator&(const bitset<N>&, const bitset<N>&) noexcept;
    template<size_t N>
        constexpr bitset<N> operator|(const bitset<N>&, const bitset<N>&) noexcept;
    template<size_t N>
        constexpr bitset<N> operator^(const bitset<N>&, const bitset<N>&) noexcept;
    template<class charT, class traits, size_t N>
        basic_istream<charT, traits>&
            operator>>(basic_istream<charT, traits>& is, bitset<N>& x);
    template<class charT, class traits, size_t N>
        basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);
}
```

22.9.2 Class template bitset

[template.bitset]

22.9.2.1 General

[template.bitset.general]

```
namespace std {
    template<size_t N> class bitset {
    public:
        // bit reference
        class reference {
        public:
            constexpr reference(const reference& x) noexcept;
            constexpr ~reference();
            constexpr reference& operator=(bool x) noexcept;           // for b[i] = x;
            constexpr reference& operator=(const reference& x) noexcept; // for b[i] = b[j];
            constexpr const reference& operator=(bool x) const noexcept;
            constexpr operator bool() const noexcept;                 // for x = b[i];
            constexpr bool operator~() const noexcept;
            constexpr reference& flip() noexcept;                       // for b[i].flip();

            friend constexpr void swap(reference x, reference y) noexcept;
            friend constexpr void swap(reference x, bool& y) noexcept;
            friend constexpr void swap(bool& x, reference y) noexcept;
        };

        // 22.9.2.2, constructors
        constexpr bitset() noexcept;
        constexpr bitset(unsigned long long val) noexcept;
```

```

template<class charT, class traits, class Allocator>
constexpr explicit bitset(
    const basic_string<charT, traits, Allocator>& str,
    typename basic_string<charT, traits, Allocator>::size_type pos = 0,
    typename basic_string<charT, traits, Allocator>::size_type n
    = basic_string<charT, traits, Allocator>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
template<class charT, class traits>
constexpr explicit bitset(
    basic_string_view<charT, traits> str,
    typename basic_string_view<charT, traits>::size_type pos = 0,
    typename basic_string_view<charT, traits>::size_type n
    = basic_string_view<charT, traits>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
template<class charT>
constexpr explicit bitset(
    const charT* str,
    typename basic_string_view<charT>::size_type n = basic_string_view<charT>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));

// 22.9.2.3, bitset operations
constexpr bitset& operator&=(const bitset& rhs) noexcept;
constexpr bitset& operator|=(const bitset& rhs) noexcept;
constexpr bitset& operator^=(const bitset& rhs) noexcept;
constexpr bitset& operator<=(size_t pos) noexcept;
constexpr bitset& operator>=(size_t pos) noexcept;
constexpr bitset& operator<<(size_t pos) const noexcept;
constexpr bitset& operator>>(size_t pos) const noexcept;
constexpr bitset& set() noexcept;
constexpr bitset& set(size_t pos, bool val = true);
constexpr bitset& reset() noexcept;
constexpr bitset& reset(size_t pos);
constexpr bitset& operator~() const noexcept;
constexpr bitset& flip() noexcept;
constexpr bitset& flip(size_t pos);

// element access
constexpr bool operator[](size_t pos) const;
constexpr reference operator[](size_t pos);

constexpr unsigned long to_ulong() const;
constexpr unsigned long long to_ullong() const;
template<class charT = char,
        class traits = char_traits<charT>,
        class Allocator = allocator<charT>>
constexpr basic_string<charT, traits, Allocator>
    to_string(charT zero = charT('0'), charT one = charT('1')) const;

// observers
constexpr size_t count() const noexcept;
constexpr size_t size() const noexcept;
constexpr bool operator==(const bitset& rhs) const noexcept;
constexpr bool test(size_t pos) const;
constexpr bool all() const noexcept;
constexpr bool any() const noexcept;
constexpr bool none() const noexcept;
};

// 22.9.3, hash support
template<class T> struct hash;

```

```
template<size_t N> struct hash<bitset<N>>;
}
```

- 1 The class template `bitset<N>` describes an object that can store a sequence consisting of a fixed number of bits, `N`.
- 2 Each bit represents either the value zero (reset) or one (set). To *toggle* a bit is to change the value zero to one, or the value one to zero. Each bit has a non-negative position `pos`. When converting between an object of type `bitset<N>` and a value of some integral type, bit position `pos` corresponds to the *bit value* `1 << pos`. The integral value corresponding to two or more bits is the sum of their bit values.
- 3 **reference** is a class that simulates a reference to a single bit in the sequence.

```
constexpr reference::reference(const reference& x) noexcept;
```

- 4 *Effects*: Initializes `*this` to refer to the same bit as `x`.

```
constexpr reference::~~reference();
```

- 5 *Effects*: None.

```
constexpr reference& reference::operator=(bool x) noexcept;
constexpr reference& reference::operator=(const reference& x) noexcept;
constexpr const reference& reference::operator=(bool x) const noexcept;
```

- 6 *Effects*: Sets the bit referred to by `*this` if `bool(x)` is true, and clears it otherwise.

- 7 *Returns*: `*this`.

```
constexpr reference::operator bool() const noexcept;
```

- 8 *Returns*: true if the value of the bit referred to by `*this` is one, false otherwise.

```
constexpr bool reference::operator~() const noexcept;
```

- 9 *Returns*: `!bool(*this)`.

```
constexpr void swap(reference x, reference y) noexcept;
constexpr void swap(reference x, bool& y) noexcept;
constexpr void swap(bool& x, reference y) noexcept;
```

- 10 *Effects*: Exchanges the values denoted by `x` and `y` as if by:

```
bool b = x;
x = y;
y = b;
```

```
constexpr reference& reference::flip() noexcept;
```

- 11 *Effects*: Equivalent to `*this = !*this`.

- 12 *Returns*: `*this`.

- 13 The functions described in 22.9.2 can report three kinds of errors, each associated with a distinct exception:

- (13.1) — an *invalid-argument* error is associated with exceptions of type `invalid_argument` (19.2.5);
- (13.2) — an *out-of-range* error is associated with exceptions of type `out_of_range` (19.2.7);
- (13.3) — an *overflow* error is associated with exceptions of type `overflow_error` (19.2.10).

22.9.2.2 Constructors

[bitset.cons]

```
constexpr bitset() noexcept;
```

- 1 *Effects*: Initializes all bits in `*this` to zero.

```
constexpr bitset(unsigned long long val) noexcept;
```

- 2 *Effects*: Initializes the first `M` bit positions to the corresponding bit values in `val`. `M` is the smaller of `N` and the number of bits in the value representation (6.9.1) of `unsigned long long`. If `M < N`, the remaining bit positions are initialized to zero.

```
template<class charT, class traits, class Allocator>
constexpr explicit bitset(
    const basic_string<charT, traits, Allocator>& str,
    typename basic_string<charT, traits, Allocator>::size_type pos = 0,
    typename basic_string<charT, traits, Allocator>::size_type n
        = basic_string<charT, traits, Allocator>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
```

```
template<class charT, class traits>
constexpr explicit bitset(
    basic_string_view<charT, traits> str,
    typename basic_string_view<charT, traits>::size_type pos = 0,
    typename basic_string_view<charT, traits>::size_type n
        = basic_string_view<charT, traits>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
```

3 *Effects:* Determines the effective length `rlen` of the initializing string as the smaller of `n` and `str.size()` - `pos`. Initializes the first `M` bit positions to values determined from the corresponding characters in the string `str`. `M` is the smaller of `N` and `rlen`.

4 An element of the constructed object has value zero if the corresponding character in `str`, beginning at position `pos`, is `zero`. Otherwise, the element has the value one. Character position `pos + M - 1` corresponds to bit position zero. Subsequent decreasing character positions correspond to increasing bit positions.

5 If `M < N`, remaining bit positions are initialized to zero.

6 The function uses `traits::eq` to compare the character values.

7 *Throws:* `out_of_range` if `pos > str.size()` or `invalid_argument` if any of the `rlen` characters in `str` beginning at position `pos` is other than `zero` or `one`.

```
template<class charT>
constexpr explicit bitset(
    const charT* str,
    typename basic_string_view<charT>::size_type n = basic_string_view<charT>::npos,
    charT zero = charT('0'),
    charT one = charT('1'));
```

8 *Constraints:*

- (8.1) — `is_array_v<charT>` is false,
- (8.2) — `is_trivially_copyable_v<charT>` is true,
- (8.3) — `is_standard_layout_v<charT>` is true, and
- (8.4) — `is_trivially_default_constructible_v<charT>` is true.

9 *Effects:* As if by:

```
    bitset(n == basic_string_view<charT>::npos
        ? basic_string_view<charT>(str)
        : basic_string_view<charT>(str, n),
        0, n, zero, one)
```

22.9.2.3 Members

[bitset.members]

```
constexpr bitset& operator&=(const bitset& rhs) noexcept;
```

1 *Effects:* Clears each bit in `*this` for which the corresponding bit in `rhs` is clear, and leaves all other bits unchanged.

2 *Returns:* `*this`.

```
constexpr bitset& operator|=(const bitset& rhs) noexcept;
```

3 *Effects:* Sets each bit in `*this` for which the corresponding bit in `rhs` is set, and leaves all other bits unchanged.

4 *Returns:* `*this`.

```

constexpr bitset& operator^=(const bitset& rhs) noexcept;
5      Effects: Toggles each bit in *this for which the corresponding bit in rhs is set, and leaves all other
      bits unchanged.
6      Returns: *this.

constexpr bitset& operator<<=(size_t pos) noexcept;
7      Effects: Replaces each bit at position I in *this with a value determined as follows:
(7.1)    — If I < pos, the new value is zero;
(7.2)    — If I >= pos, the new value is the previous value of the bit at position I - pos.
8      Returns: *this.

constexpr bitset& operator>>=(size_t pos) noexcept;
9      Effects: Replaces each bit at position I in *this with a value determined as follows:
(9.1)    — If pos >= N - I, the new value is zero;
(9.2)    — If pos < N - I, the new value is the previous value of the bit at position I + pos.
10     Returns: *this.

constexpr bitset operator<<(size_t pos) const noexcept;
11     Returns: bitset(*this) <<= pos.

constexpr bitset operator>>(size_t pos) const noexcept;
12     Returns: bitset(*this) >>= pos.

constexpr bitset& set() noexcept;
13     Effects: Sets all bits in *this.
14     Returns: *this.

constexpr bitset& set(size_t pos, bool val = true);
15     Effects: Stores a new value in the bit at position pos in *this. If val is true, the stored value is one,
      otherwise it is zero.
16     Returns: *this.
17     Throws: out_of_range if pos does not correspond to a valid bit position.

constexpr bitset& reset() noexcept;
18     Effects: Resets all bits in *this.
19     Returns: *this.

constexpr bitset& reset(size_t pos);
20     Effects: Resets the bit at position pos in *this.
21     Returns: *this.
22     Throws: out_of_range if pos does not correspond to a valid bit position.

constexpr bitset operator~() const noexcept;
23     Effects: Constructs an object x of class bitset and initializes it with *this.
24     Returns: x.flip().

constexpr bitset& flip() noexcept;
25     Effects: Toggles all bits in *this.
26     Returns: *this.

constexpr bitset& flip(size_t pos);
27     Effects: Toggles the bit at position pos in *this.

```

28 *Returns:* **this*.

29 *Throws:* *out_of_range* if *pos* does not correspond to a valid bit position.

```
constexpr bool operator[](size_t pos) const;
```

30 *Hardened preconditions:* *pos < size()* is true.

31 *Returns:* true if the bit at position *pos* in **this* has the value one, otherwise false.

32 *Throws:* Nothing.

```
constexpr bitset::reference operator[](size_t pos);
```

33 *Hardened preconditions:* *pos < size()* is true.

34 *Returns:* An object of type *bitset::reference* such that *(*this)[pos] == this->test(pos)*, and such that *(*this)[pos] = val* is equivalent to *this->set(pos, val)*.

35 *Throws:* Nothing.

36 *Remarks:* For the purpose of determining the presence of a data race (6.10.2), any access or update through the resulting reference potentially accesses or modifies, respectively, the entire underlying *bitset*.

```
constexpr unsigned long to_ulong() const;
```

37 *Returns:* *x*.

38 *Throws:* *overflow_error* if the integral value *x* corresponding to the bits in **this* cannot be represented as type *unsigned long*.

```
constexpr unsigned long long to_ullong() const;
```

39 *Returns:* *x*.

40 *Throws:* *overflow_error* if the integral value *x* corresponding to the bits in **this* cannot be represented as type *unsigned long long*.

```
template<class charT = char,
        class traits = char_traits<charT>,
        class Allocator = allocator<charT>>
constexpr basic_string<charT, traits, Allocator>
to_string(charT zero = charT('0'), charT one = charT('1')) const;
```

41 *Effects:* Constructs a string object of the appropriate type and initializes it to a string of length *N* characters. Each character is determined by the value of its corresponding bit position in **this*. Character position *N - 1* corresponds to bit position zero. Subsequent decreasing character positions correspond to increasing bit positions. Bit value zero becomes the character *zero*, bit value one becomes the character *one*.

42 *Returns:* The created object.

```
constexpr size_t count() const noexcept;
```

43 *Returns:* A count of the number of bits set in **this*.

```
constexpr size_t size() const noexcept;
```

44 *Returns:* *N*.

```
constexpr bool operator==(const bitset& rhs) const noexcept;
```

45 *Returns:* true if the value of each bit in **this* equals the value of the corresponding bit in *rhs*.

```
constexpr bool test(size_t pos) const;
```

46 *Returns:* true if the bit at position *pos* in **this* has the value one.

47 *Throws:* *out_of_range* if *pos* does not correspond to a valid bit position.

```
constexpr bool all() const noexcept;
```

48 *Returns:* *count() == size()*.

```
constexpr bool any() const noexcept;
```

49 *Returns:* `count() != 0`.

```
constexpr bool none() const noexcept;
```

50 *Returns:* `count() == 0`.

22.9.3 bitset hash support

[bitset.hash]

```
template<size_t N> struct hash<bitset<N>>;
```

1 The specialization is enabled (22.10.19).

22.9.4 bitset operators

[bitset.operators]

```
template<size_t N>
```

```
constexpr bitset<N> operator&(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

1 *Returns:* `bitset<N>(lhs) &= rhs`.

```
template<size_t N>
```

```
constexpr bitset<N> operator|(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

2 *Returns:* `bitset<N>(lhs) |= rhs`.

```
template<size_t N>
```

```
constexpr bitset<N> operator^(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

3 *Returns:* `bitset<N>(lhs) ^= rhs`.

```
template<class charT, class traits, size_t N>
```

```
basic_istream<charT, traits>&
```

```
operator>>(basic_istream<charT, traits>& is, bitset<N>& x);
```

4 A formatted input function (31.7.5.3).

5 *Effects:* Extracts up to `N` characters from `is`. Stores these characters in a temporary object `str` of type `basic_string<charT, traits>`, then evaluates the expression `x = bitset<N>(str)`. Characters are extracted and stored until any of the following occurs:

(5.1) — `N` characters have been extracted and stored;

(5.2) — end-of-file occurs on the input sequence;

(5.3) — the next input character is neither `is.widen('0')` nor `is.widen('1')` (in which case the input character is not extracted).

6 If `N > 0` and no characters are stored in `str`, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

7 *Returns:* `is`.

```
template<class charT, class traits, size_t N>
```

```
basic_ostream<charT, traits>&
```

```
operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);
```

8 *Returns:*

```
os << x.template to_string<charT, traits, allocator<charT>>>(
    use_facet<ctype<charT>>(os.getloc()).widen('0'),
    use_facet<ctype<charT>>(os.getloc()).widen('1'))
```

(see 31.7.6.3).

22.10 Function objects

[function.objects]

22.10.1 General

[function.objects.general]

1 A *function object type* is an object type (6.9.1) that can be the type of the *postfix-expression* in a function call (7.6.1.3, 12.2.2.2).¹⁷² A *function object* is an object of a function object type. In the places where one would expect to pass a pointer to a function to an algorithmic template (Clause 26), the interface is specified

¹⁷² Such a type is a function pointer or a class type which has a member `operator()` or a class type which has a conversion to a pointer to function.

to accept a function object. This not only makes algorithmic templates work with pointers to functions, but also enables them to work with arbitrary function objects.

22.10.2 Header <functional> synopsis

[functional.syn]

```
#include <initializer_list>    // see 17.11.2
#include <typeinfo>            // see 17.7.2

namespace std {
    // 22.10.5, invoke
    template<class F, class... Args>
        constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)    // freestanding
            noexcept(is_nothrow_invocable_v<F, Args...>);

    template<class R, class F, class... Args>
        constexpr R invoke_r(F&& f, Args&&... args)    // freestanding
            noexcept(is_nothrow_invocable_r_v<R, F, Args...>);

    // 22.10.6, reference_wrapper
    template<class T> class reference_wrapper;    // freestanding

    template<class T> constexpr reference_wrapper<T> ref(T&) noexcept;    // freestanding
    template<class T> constexpr reference_wrapper<const T> cref(const T&) noexcept;    // freestanding
    template<class T> void ref(const T&&) = delete;    // freestanding
    template<class T> void cref(const T&&) = delete;    // freestanding

    template<class T>
        constexpr reference_wrapper<T> ref(reference_wrapper<T>) noexcept;    // freestanding
    template<class T>
        constexpr reference_wrapper<const T> cref(reference_wrapper<T>) noexcept;    // freestanding

    // 22.10.6.8, common_reference related specializations
    template<class R, class T, template<class> class RQual, template<class> class TQual>
        requires see below
    struct basic_common_reference<R, T, RQual, TQual>;

    template<class T, class R, template<class> class TQual, template<class> class RQual>
        requires see below
    struct basic_common_reference<T, R, TQual, RQual>;

    // 22.10.7, arithmetic operations
    template<class T = void> struct plus;    // freestanding
    template<class T = void> struct minus;    // freestanding
    template<class T = void> struct multiplies;    // freestanding
    template<class T = void> struct divides;    // freestanding
    template<class T = void> struct modulus;    // freestanding
    template<class T = void> struct negate;    // freestanding
    template<> struct plus<void>;    // freestanding
    template<> struct minus<void>;    // freestanding
    template<> struct multiplies<void>;    // freestanding
    template<> struct divides<void>;    // freestanding
    template<> struct modulus<void>;    // freestanding
    template<> struct negate<void>;    // freestanding

    // 22.10.8, comparisons
    template<class T = void> struct equal_to;    // freestanding
    template<class T = void> struct not_equal_to;    // freestanding
    template<class T = void> struct greater;    // freestanding
    template<class T = void> struct less;    // freestanding
    template<class T = void> struct greater_equal;    // freestanding
    template<class T = void> struct less_equal;    // freestanding
    template<> struct equal_to<void>;    // freestanding
    template<> struct not_equal_to<void>;    // freestanding
    template<> struct greater<void>;    // freestanding
```

```

template<> struct less<void>; // freestanding
template<> struct greater_equal<void>; // freestanding
template<> struct less_equal<void>; // freestanding

// 22.10.8.8, class compare_three_way
struct compare_three_way; // freestanding

// 22.10.10, logical operations
template<class T = void> struct logical_and; // freestanding
template<class T = void> struct logical_or; // freestanding
template<class T = void> struct logical_not; // freestanding
template<> struct logical_and<void>; // freestanding
template<> struct logical_or<void>; // freestanding
template<> struct logical_not<void>; // freestanding

// 22.10.11, bitwise operations
template<class T = void> struct bit_and; // freestanding
template<class T = void> struct bit_or; // freestanding
template<class T = void> struct bit_xor; // freestanding
template<class T = void> struct bit_not; // freestanding
template<> struct bit_and<void>; // freestanding
template<> struct bit_or<void>; // freestanding
template<> struct bit_xor<void>; // freestanding
template<> struct bit_not<void>; // freestanding

// 22.10.12, identity
struct identity; // freestanding

// 22.10.13, function template not_fn
template<class F> constexpr unspecified not_fn(F&& f); // freestanding
template<auto f> constexpr unspecified not_fn() noexcept; // freestanding

// 22.10.14, function templates bind_front and bind_back
template<class F, class... Args>
    constexpr unspecified bind_front(F&&, Args&&...); // freestanding
template<auto f, class... Args>
    constexpr unspecified bind_front(Args&&...); // freestanding
template<class F, class... Args>
    constexpr unspecified bind_back(F&&, Args&&...); // freestanding
template<auto f, class... Args>
    constexpr unspecified bind_back(Args&&...); // freestanding

// 22.10.15, bind
template<class T> struct is_bind_expression; // freestanding
template<class T>
    constexpr bool is_bind_expression_v = // freestanding
        is_bind_expression<T>::value;
template<class T> struct is_placeholder; // freestanding
template<class T>
    constexpr int is_placeholder_v = // freestanding
        is_placeholder<T>::value;

template<class F, class... BoundArgs>
    constexpr unspecified bind(F&&, BoundArgs&&...); // freestanding
template<class R, class F, class... BoundArgs>
    constexpr unspecified bind(F&&, BoundArgs&&...); // freestanding

namespace placeholders {
    // M is the implementation-defined number of placeholders
    see below _1; // freestanding
    see below _2; // freestanding
    :
    see below _M; // freestanding

```

```

}

// 22.10.16, member function adaptors
template<class R, class T>
    constexpr unspecified mem_fn(R T::*) noexcept; // freestanding

// 22.10.17, polymorphic function wrappers
// 22.10.17.2, class bad_function_call
class bad_function_call;

// 22.10.17.3, class template function
template<class> class function; // not defined
template<class R, class... ArgTypes> class function<R(ArgTypes...)>;

// 22.10.17.3.8, function specialized algorithms
template<class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;

// 22.10.17.3.7, function null pointer comparison operator functions
template<class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

// 22.10.17.4, move-only wrapper
template<class... S> class move_only_function; // not defined
template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

// 22.10.17.5, copyable wrapper
template<class... S> class copyable_function; // not defined
template<class R, class... ArgTypes>
    class copyable_function<R(ArgTypes...) cv ref noexcept(noex)>; // see below

// 22.10.17.6, non-owning wrapper
template<class... S> class function_ref; // freestanding, not defined
template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)>; // freestanding, see below

// 22.10.18, searchers
template<class ForwardIterator1, class BinaryPredicate = equal_to<>>
    class default_searcher; // freestanding

template<class RandomAccessIterator,
    class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
    class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher;

template<class RandomAccessIterator,
    class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
    class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher;

// 22.10.19, class template hash
template<class T>
    struct hash; // freestanding

namespace ranges {
    // 22.10.9, concept-constrained comparisons
    struct equal_to; // freestanding
    struct not_equal_to; // freestanding
    struct greater; // freestanding
    struct less; // freestanding
    struct greater_equal; // freestanding
    struct less_equal; // freestanding
}

```

```

template<class Fn, class... Args>
concept callable = // exposition only
    requires (Fn&& fn, Args&&... args) {
        std::forward<Fn>(fn)(std::forward<Args>(args)...);
    };

template<class Fn, class... Args>
concept nothrow-callable = // exposition only
    callable<Fn, Args...> &&
    requires (Fn&& fn, Args&&... args) {
        { std::forward<Fn>(fn)(std::forward<Args>(args)... ) } noexcept;
    };

template<class Fn, class... Args>
using call-result-t = decltype(declval<Fn>()(declval<Args>()...)); // exposition only

template<const auto& T>
using decayed-typeof = decltype(auto(T)); // exposition only
}

```

¹ [Example 1: If a C++ program wants to have a by-element addition of two vectors **a** and **b** containing **double** and put the result into **a**, it can do:

```
transform(a.begin(), a.end(), b.begin(), a.begin(), plus<double>());
```

— end example]

² [Example 2: To negate every element of **a**:

```
transform(a.begin(), a.end(), a.begin(), negate<double>());
```

— end example]

22.10.3 Definitions

[func.def]

¹ The following definitions apply to this Clause:

- ² A *call signature* is the name of a return type followed by a parenthesized comma-separated list of zero or more argument types.
- ³ A *callable type* is a function object type (22.10) or a pointer to member.
- ⁴ A *callable object* is an object of a callable type.
- ⁵ A *call wrapper type* is a type that holds a *target object*, which is either a callable object or an object representing a callable object, and supports a call operation that forwards to that callable object.
- ⁶ A *call wrapper* is an object of a call wrapper type.
- ⁷ A call wrapper type may additionally hold a sequence of objects and references that may be passed as arguments to the callable object. These entities are collectively referred to as *bound argument entities*.
- ⁸ The target object and bound argument entities of the call wrapper are collectively referred to as *state entities*.

22.10.4 Requirements

[func.require]

¹ Define *INVOKE*(*f*, *t*₁, *t*₂, ..., *t*_{*N*}) as follows:

- (1.1) — (*t*₁.**f*)(*t*₂, ..., *t*_{*N*}) when *f* is a pointer to a member function of a class *T* and *is_same_v*<*T*, *remove_cvref_t*<*decltype*(*t*₁)>> || *is_base_of_v*<*T*, *remove_cvref_t*<*decltype*(*t*₁)>> is true;
- (1.2) — (*t*₁.*get*().**f*)(*t*₂, ..., *t*_{*N*}) when *f* is a pointer to a member function of a class *T* and *remove_cvref_t*<*decltype*(*t*₁)> is a specialization of *reference_wrapper*;
- (1.3) — ((**t*₁).**f*)(*t*₂, ..., *t*_{*N*}) when *f* is a pointer to a member function of a class *T* and *t*₁ does not satisfy the previous two items;
- (1.4) — *t*₁.**f* when *N* = 1 and *f* is a pointer to data member of a class *T* and *is_same_v*<*T*, *remove_cvref_t*<*decltype*(*t*₁)>> || *is_base_of_v*<*T*, *remove_cvref_t*<*decltype*(*t*₁)>> is true;
- (1.5) — *t*₁.*get*().**f* when *N* = 1 and *f* is a pointer to data member of a class *T* and *remove_cvref_t*<*decltype*(*t*₁)> is a specialization of *reference_wrapper*;
- (1.6) — ((**t*₁).**f* when *N* = 1 and *f* is a pointer to data member of a class *T* and *t*₁ does not satisfy the previous two items;

(1.7) — $f(t_1, t_2, \dots, t_N)$ in all other cases.

² Define `INVOKE<R>(f, t1, t2, ..., tN)` as `static_cast<void>(INVOKE(f, t1, t2, ..., tN))` if `R` is `cv void`, otherwise `INVOKE(f, t1, t2, ..., tN)` implicitly converted to `R`. If `reference_converts_from_temporary_v<R, decltype(INVOKE(f, t1, t2, ..., tN))>` is true, `INVOKE<R>(f, t1, t2, ..., tN)` is ill-formed.

³ Every call wrapper (22.10.3) meets the *Cpp17MoveConstructible* and *Cpp17Destructible* requirements. An *argument forwarding call wrapper* is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the target object as references. This forwarding step delivers rvalue arguments as rvalue references and lvalue arguments as lvalue references.

[Note 1: In a typical implementation, argument forwarding call wrappers have an overloaded function call operator of the form

```
template<class... UnBoundArgs>
constexpr R operator()(UnBoundArgs&&... unbound_args) cv-qual;
```

— end note]

⁴ A *perfect forwarding call wrapper* is an argument forwarding call wrapper that forwards its state entities to the underlying call expression. This forwarding step delivers a state entity of type `T` as `cv T&` when the call is performed on an lvalue of the call wrapper type and as `cv T&&` otherwise, where `cv` represents the cv-qualifiers of the call wrapper and where `cv` shall be neither `volatile` nor `const volatile`.

⁵ A *call pattern* defines the semantics of invoking a perfect forwarding call wrapper. A postfix call performed on a perfect forwarding call wrapper is expression-equivalent (3.22) to an expression `e` determined from its call pattern `cp` by replacing all occurrences of the arguments of the call wrapper and its state entities with references as described in the corresponding forwarding steps.

⁶ A *simple call wrapper* is a perfect forwarding call wrapper that meets the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements and whose copy constructor, move constructor, and assignment operators are `constexpr` functions that do not throw exceptions.

⁷ The copy/move constructor of an argument forwarding call wrapper has the same apparent semantics as if memberwise copy/move of its state entities were performed (11.4.5.3).

[Note 2: This implies that each of the copy/move constructors has the same exception-specification as the corresponding implicit definition and is declared as `constexpr` if the corresponding implicit definition would be considered to be `constexpr`. — end note]

⁸ Argument forwarding call wrappers returned by a given standard library function template have the same type if the types of their corresponding state entities are the same.

22.10.5 invoke functions

[func.invoke]

```
template<class F, class... Args>
constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
noexcept(is_nothrow_invocable_v<F, Args...>);
```

¹ *Constraints:* `is_invocable_v<F, Args...>` is true.

² *Returns:* `INVOKE(std::forward<F>(f), std::forward<Args>(args)...) (22.10.4).`

```
template<class R, class F, class... Args>
constexpr R invoke_r(F&& f, Args&&... args)
noexcept(is_nothrow_invocable_r_v<R, F, Args...>);
```

³ *Constraints:* `is_invocable_r_v<R, F, Args...>` is true.

⁴ *Returns:* `INVOKE<R>(std::forward<F>(f), std::forward<Args>(args)...) (22.10.4).`

22.10.6 Class template reference_wrapper

[refwrap]

22.10.6.1 General

[refwrap.general]

```
namespace std {
template<class T> class reference_wrapper {
public:
    // types
    using type = T;
```

```

// 22.10.6.2, constructors
template<class U>
    constexpr reference_wrapper(U&&) noexcept(see below);
constexpr reference_wrapper(const reference_wrapper& x) noexcept;

// 22.10.6.3, assignment
constexpr reference_wrapper& operator=(const reference_wrapper& x) noexcept;

// 22.10.6.4, access
constexpr operator T& () const noexcept;
constexpr T& get() const noexcept;

// 22.10.6.5, invocation
template<class... ArgTypes>
    constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...) const
        noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);

// 22.10.6.6, comparisons
friend constexpr bool operator==(reference_wrapper, reference_wrapper);
friend constexpr bool operator==(reference_wrapper, const T&);
friend constexpr bool operator==(reference_wrapper, reference_wrapper<const T>);

friend constexpr auto operator<=>(reference_wrapper, reference_wrapper);
friend constexpr auto operator<=>(reference_wrapper, const T&);
friend constexpr auto operator<=>(reference_wrapper, reference_wrapper<const T>);
};

template<class T>
    reference_wrapper(T&) -> reference_wrapper<T>;
}

```

- 1 `reference_wrapper<T>` is a *Cpp17CopyConstructible* and *Cpp17CopyAssignable* wrapper around a reference to an object or function of type `T`.
- 2 `reference_wrapper<T>` is a trivially copyable type (6.9.1).
- 3 The template parameter `T` of `reference_wrapper` may be an incomplete type.
 [Note 1: Using the comparison operators described in 22.10.6.6 with `T` being an incomplete type can lead to an ill-formed program with no diagnostic required (13.8.4.1, 13.5.2.3). — end note]

22.10.6.2 Constructors

[refwrap.const]

```

template<class U>
    constexpr reference_wrapper(U&& u) noexcept(see below);

```

- 1 Let *FUN* denote the exposition-only functions


```

void FUN(T&) noexcept;
void FUN(T&&) = delete;

```
- 2 *Constraints:* The expression `FUN(declval<U>())` is well-formed and `is_same_v<remove_cvref_t<U>, reference_wrapper>` is false.
- 3 *Effects:* Creates a variable `r` as if by `T& r = std::forward<U>(u)`, then constructs a `reference_wrapper` object that stores a reference to `r`.
- 4 *Remarks:* The exception specification is equivalent to `noexcept(FUN(declval<U>()))`.

```

constexpr reference_wrapper(const reference_wrapper& x) noexcept;

```

- 5 *Effects:* Constructs a `reference_wrapper` object that stores a reference to `x.get()`.

22.10.6.3 Assignment

[refwrap.assign]

```

constexpr reference_wrapper& operator=(const reference_wrapper& x) noexcept;

```

- 1 *Postconditions:* `*this` stores a reference to `x.get()`.

22.10.6.4 Access

[refwrap.access]

```
constexpr operator T& () const noexcept;
```

1 *Returns:* The stored reference.

```
constexpr T& get() const noexcept;
```

2 *Returns:* The stored reference.

22.10.6.5 Invocation

[refwrap.invoke]

```
template<class... ArgTypes>
```

```
constexpr invoke_result_t<T&, ArgTypes...>
```

```
operator()(ArgTypes&&... args) const noexcept(is_nothrow_invocable_v<T&, ArgTypes...>);
```

1 *Mandates:* T is a complete type.

2 *Returns:* *INVOKE*(get(), std::forward<ArgTypes>(args)...) (22.10.4).

22.10.6.6 Comparisons

[refwrap.comparisons]

```
friend constexpr bool operator==(reference_wrapper x, reference_wrapper y);
```

1 *Constraints:* The expression `x.get() == y.get()` is well-formed and its result is convertible to `bool`.

2 *Returns:* `x.get() == y.get()`.

```
friend constexpr bool operator==(reference_wrapper x, const T& y);
```

3 *Constraints:* The expression `x.get() == y` is well-formed and its result is convertible to `bool`.

4 *Returns:* `x.get() == y`.

```
friend constexpr bool operator==(reference_wrapper x, reference_wrapper<const T> y);
```

5 *Constraints:* `is_const_v<T>` is false and the expression `x.get() == y.get()` is well-formed and its result is convertible to `bool`.

6 *Returns:* `x.get() == y.get()`.

```
friend constexpr auto operator<=>(reference_wrapper x, reference_wrapper y);
```

7 *Constraints:* The expression *synth-three-way*(x.get(), y.get()) is well-formed.

8 *Returns:* *synth-three-way*(x.get(), y.get()).

```
friend constexpr auto operator<=>(reference_wrapper x, const T& y);
```

9 *Constraints:* The expression *synth-three-way*(x.get(), y) is well-formed.

10 *Returns:* *synth-three-way*(x.get(), y).

```
friend constexpr auto operator<=>(reference_wrapper x, reference_wrapper<const T> y);
```

11 *Constraints:* `is_const_v<T>` is false. The expression *synth-three-way*(x.get(), y.get()) is well-formed.

12 *Returns:* *synth-three-way*(x.get(), y.get()).

22.10.6.7 Helper functions

[refwrap.helpers]

1 The template parameter T of the following `ref` and `cref` function templates may be an incomplete type.

```
template<class T> constexpr reference_wrapper<T> ref(T& t) noexcept;
```

2 *Returns:* `reference_wrapper<T>(t)`.

```
template<class T> constexpr reference_wrapper<T> ref(reference_wrapper<T> t) noexcept;
```

3 *Returns:* t.

```
template<class T> constexpr reference_wrapper<const T> cref(const T& t) noexcept;
```

4 *Returns:* `reference_wrapper<const T>(t)`.


```
template<class T> constexpr reference_wrapper<const T> cref(reference_wrapper<T> t) noexcept;
```

5 *Returns:* t.

22.10.6.8 common_reference related specializations

[refwrap.common.ref]

```
namespace std {
    template<class T>
        constexpr bool is-ref-wrapper = false; // exposition only

    template<class T>
        constexpr bool is-ref-wrapper<reference_wrapper<T>> = true;

    template<class R, class T, class RQ, class TQ>
        concept ref-wrap-common-reference-exists-with = // exposition only
            is-ref-wrapper<R> &&
            requires { typename common_reference_t<typename R::type&, TQ>; } &&
            convertible_to<RQ, common_reference_t<typename R::type&, TQ>>;

    template<class R, class T, template<class> class RQual, template<class> class TQual>
        requires (ref-wrap-common-reference-exists-with<R, T, RQual<R>, TQual<T>> &&
            !ref-wrap-common-reference-exists-with<T, R, TQual<T>, RQual<R>>)
    struct basic_common_reference<R, T, RQual, TQual> {
        using type = common_reference_t<typename R::type&, TQual<T>>;
    };

    template<class T, class R, template<class> class TQual, template<class> class RQual>
        requires (ref-wrap-common-reference-exists-with<R, T, RQual<R>, TQual<T>> &&
            !ref-wrap-common-reference-exists-with<T, R, TQual<T>, RQual<R>>)
    struct basic_common_reference<T, R, TQual, RQual> {
        using type = common_reference_t<typename R::type&, TQual<T>>;
    };
}
```

22.10.7 Arithmetic operations

[arithmetic.operations]

22.10.7.1 General

[arithmetic.operations.general]

¹ The library provides basic function object classes for all of the arithmetic operators in the language (7.6.5, 7.6.6).

22.10.7.2 Class template plus

[arithmetic.operations.plus]

```
template<class T = void> struct plus {
    constexpr T operator()(const T& x, const T& y) const;
};
```

```
constexpr T operator()(const T& x, const T& y) const;
```

¹ *Returns:* x + y.

```
template<> struct plus<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) + std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) + std::forward<U>(u));
```

² *Returns:* std::forward<T>(t) + std::forward<U>(u).

22.10.7.3 Class template minus

[arithmetic.operations.minus]

```
template<class T = void> struct minus {
    constexpr T operator()(const T& x, const T& y) const;
};
```



```
constexpr T operator()(const T& x, const T& y) const;
```

1 *Returns:* $x - y$.

```
template<> struct minus<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) - std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) - std::forward<U>(u));
```

2 *Returns:* $\text{std::forward<T>(t)} - \text{std::forward<U>(u)}$.

22.10.7.4 Class template multiplies

[arithmetic.operations.multiplies]

```
template<class T = void> struct multiplies {
    constexpr T operator()(const T& x, const T& y) const;
};
```

```
constexpr T operator()(const T& x, const T& y) const;
```

1 *Returns:* $x * y$.

```
template<> struct multiplies<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) * std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) * std::forward<U>(u));
```

2 *Returns:* $\text{std::forward<T>(t)} * \text{std::forward<U>(u)}$.

22.10.7.5 Class template divides

[arithmetic.operations.divides]

```
template<class T = void> struct divides {
    constexpr T operator()(const T& x, const T& y) const;
};
```

```
constexpr T operator()(const T& x, const T& y) const;
```

1 *Returns:* x / y .

```
template<> struct divides<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) / std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) / std::forward<U>(u));
```

2 *Returns:* $\text{std::forward<T>(t)} / \text{std::forward<U>(u)}$.

22.10.7.6 Class template modulus

[arithmetic.operations.modulus]

```
template<class T = void> struct modulus {
    constexpr T operator()(const T& x, const T& y) const;
};
```

```
constexpr T operator()(const T& x, const T& y) const;
```

1 *Returns:* $x \% y$.

```
template<> struct modulus<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) % std::forward<U>(u));

    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) % std::forward<U>(u));
```

² *Returns:* `std::forward<T>(t) % std::forward<U>(u)`.

22.10.7.7 Class template `negate`

[arithmetic.operations.negate]

```
template<class T = void> struct negate {
    constexpr T operator()(const T& x) const;
};
```

```
constexpr T operator()(const T& x) const;
```

¹ *Returns:* `-x`.

```
template<> struct negate<void> {
    template<class T> constexpr auto operator()(T&& t) const
        -> decltype(-std::forward<T>(t));

    using is_transparent = unspecified;
};
```

```
template<class T> constexpr auto operator()(T&& t) const
    -> decltype(-std::forward<T>(t));
```

² *Returns:* `-std::forward<T>(t)`.

22.10.8 Comparisons

[comparisons]

22.10.8.1 General

[comparisons.general]

¹ The library provides basic function object classes for all of the comparison operators in the language (7.6.9, 7.6.10).

² For templates `less`, `greater`, `less_equal`, and `greater_equal`, the specializations for any pointer type yield a result consistent with the implementation-defined strict total order over pointers (3.28).

[Note 1: If `a < b` is well-defined for pointers `a` and `b` of type `P`, then `(a < b) == less<P>()(a, b)`, `(a > b) == greater<P>()(a, b)`, and so forth. — end note]

For template specializations `less<void>`, `greater<void>`, `less_equal<void>`, and `greater_equal<void>`, if the call operator calls a built-in operator comparing pointers, the call operator yields a result consistent with the implementation-defined strict total order over pointers.

22.10.8.2 Class template `equal_to`

[comparisons.equal.to]

```
template<class T = void> struct equal_to {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

¹ *Returns:* `x == y`.

```
template<> struct equal_to<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) == std::forward<U>(u));

    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) == std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) == std::forward<U>(u).

22.10.8.3 Class template not_equal_to

[comparisons.not.equal.to]

```
template<class T = void> struct not_equal_to {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

1 *Returns:* x != y.

```
template<> struct not_equal_to<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) != std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) != std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) != std::forward<U>(u).

22.10.8.4 Class template greater

[comparisons.greater]

```
template<class T = void> struct greater {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

1 *Returns:* x > y.

```
template<> struct greater<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) > std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) > std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) > std::forward<U>(u).

22.10.8.5 Class template less

[comparisons.less]

```
template<class T = void> struct less {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

1 *Returns:* x < y.

```
template<> struct less<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) < std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) < std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) < std::forward<U>(u).

22.10.8.6 Class template greater_equal**[comparisons.greater.equal]**

```
template<class T = void> struct greater_equal {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

¹ *Returns:* `x >= y`.

```
template<> struct greater_equal<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) >= std::forward<U>(u));

    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) >= std::forward<U>(u));
```

² *Returns:* `std::forward<T>(t) >= std::forward<U>(u)`.

22.10.8.7 Class template less_equal**[comparisons.less.equal]**

```
template<class T = void> struct less_equal {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

¹ *Returns:* `x <= y`.

```
template<> struct less_equal<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) <= std::forward<U>(u));

    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) <= std::forward<U>(u));
```

² *Returns:* `std::forward<T>(t) <= std::forward<U>(u)`.

22.10.8.8 Class compare_three_way**[comparisons.three.way]**

```
namespace std {
    struct compare_three_way {
        template<class T, class U>
            constexpr auto operator()(T&& t, U&& u) const;

        using is_transparent = unspecified;
    };
}
```

```
template<class T, class U>
    constexpr auto operator()(T&& t, U&& u) const;
```

¹ *Constraints:* T and U satisfy `three_way_comparable_with`.

² *Preconditions:* If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type P, the conversion sequences from both T and U to P are equality-preserving (18.2); otherwise, T and U model `three_way_comparable_with`.

³ *Effects:*

(3.1) — If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type P, returns `strong_ordering::less` if (the converted value of) t precedes u in the implementation-defined strict total order over pointers (3.28), `strong_ordering::greater` if u precedes t, and otherwise `strong_ordering::equal`.

(3.2) — Otherwise, equivalent to: `return std::forward<T>(t) <=> std::forward<U>(u);`

22.10.9 Concept-constrained comparisons

[range.cmp]

```
struct ranges::equal_to {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

1 *Constraints:* T and U satisfy `equality_comparable_with`.

2 *Preconditions:* If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers of type P, the conversion sequences from both T and U to P are equality-preserving (18.2); otherwise, T and U model `equality_comparable_with`.

3 *Effects:*

(3.1) — If the expression `std::forward<T>(t) == std::forward<U>(u)` results in a call to a built-in operator `==` comparing pointers: returns `false` if either (the converted value of) `t` precedes `u` or `u` precedes `t` in the implementation-defined strict total order over pointers (3.28) and otherwise `true`.

(3.2) — Otherwise, equivalent to: `return std::forward<T>(t) == std::forward<U>(u);`

```
struct ranges::not_equal_to {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

4 *Constraints:* T and U satisfy `equality_comparable_with`.

5 *Effects:* Equivalent to:

```
    return !ranges::equal_to{}(std::forward<T>(t), std::forward<U>(u));
```

```
struct ranges::greater {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

6 *Constraints:* T and U satisfy `totally_ordered_with`.

7 *Effects:* Equivalent to:

```
    return ranges::less{}(std::forward<U>(u), std::forward<T>(t));
```

```
struct ranges::less {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

8 *Constraints:* T and U satisfy `totally_ordered_with`.

Preconditions: If the expression `std::forward<T>(t) < std::forward<U>(u)` results in a call to a built-in operator `<` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` are equality-preserving (18.2); otherwise, `T` and `U` model `totally_ordered_with`. For any expressions `ET` and `EU` such that `decltype((ET))` is `T` and `decltype((EU))` is `U`, exactly one of `ranges::less{}(ET, EU)`, `ranges::less{}(EU, ET)`, or `ranges::equal_to{}(ET, EU)` is true.

Effects:

— If the expression `std::forward<T>(t) < std::forward<U>(u)` results in a call to a built-in operator `<` comparing pointers: returns `true` if (the converted value of) `t` precedes `u` in the implementation-defined strict total order over pointers (3.28) and otherwise `false`.

— Otherwise, equivalent to: `return std::forward<T>(t) < std::forward<U>(u);`

```
struct ranges::greater_equal {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

Constraints: `T` and `U` satisfy `totally_ordered_with`.

Effects: Equivalent to:

```
return !ranges::less{}(std::forward<T>(t), std::forward<U>(u));
```

```
struct ranges::less_equal {
    template<class T, class U>
        constexpr bool operator()(T&& t, U&& u) const;
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U>
    constexpr bool operator()(T&& t, U&& u) const;
```

Constraints: `T` and `U` satisfy `totally_ordered_with`.

Effects: Equivalent to:

```
return !ranges::less{}(std::forward<U>(u), std::forward<T>(t));
```

22.10.10 Logical operations

[logical.operations]

22.10.10.1 General

[logical.operations.general]

1 The library provides basic function object classes for all of the logical operators in the language (7.6.14, 7.6.15, 7.6.2.2).

22.10.10.2 Class template `logical_and`

[logical.operations.and]

```
template<class T = void> struct logical_and {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

Returns: `x && y`.

```
template<> struct logical_and<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) && std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) && std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) && std::forward<U>(u).

22.10.10.3 Class template logical_or

[logical.operations.or]

```
template<class T = void> struct logical_or {
    constexpr bool operator()(const T& x, const T& y) const;
};
```

```
constexpr bool operator()(const T& x, const T& y) const;
```

1 *Returns:* x || y.

```
template<> struct logical_or<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) || std::forward<U>(u));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) || std::forward<U>(u));
```

2 *Returns:* std::forward<T>(t) || std::forward<U>(u).

22.10.10.4 Class template logical_not

[logical.operations.not]

```
template<class T = void> struct logical_not {
    constexpr bool operator()(const T& x) const;
};
```

```
constexpr bool operator()(const T& x) const;
```

1 *Returns:* !x.

```
template<> struct logical_not<void> {
    template<class T> constexpr auto operator()(T&& t) const
    -> decltype(!std::forward<T>(t));
```

```
    using is_transparent = unspecified;
};
```

```
template<class T> constexpr auto operator()(T&& t) const
-> decltype(!std::forward<T>(t));
```

2 *Returns:* !std::forward<T>(t).

22.10.11 Bitwise operations

[bitwise.operations]

22.10.11.1 General

[bitwise.operations.general]

1 The library provides basic function object classes for all of the bitwise operators in the language (7.6.11, 7.6.13, 7.6.12, 7.6.2.2).

22.10.11.2 Class template bit_and

[bitwise.operations.and]

```
template<class T = void> struct bit_and {
    constexpr T operator()(const T& x, const T& y) const;
};
```

```
constexpr T operator()(const T& x, const T& y) const;
```

1 *Returns:* x & y.

```
template<> struct bit_and<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) & std::forward<U>(u));
```

```

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) & std::forward<U>(u));

```

² *Returns:* $\text{std::forward<T>(t) \& std::forward<U>(u)}$.

22.10.11.3 Class template `bit_or`

[bitwise.operations.or]

```

template<class T = void> struct bit_or {
    constexpr T operator()(const T& x, const T& y) const;
};

```

```
constexpr T operator()(const T& x, const T& y) const;
```

¹ *Returns:* $x \mid y$.

```

template<> struct bit_or<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) | std::forward<U>(u));

```

```

    using is_transparent = unspecified;
};

```

```

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) | std::forward<U>(u));

```

² *Returns:* $\text{std::forward<T>(t) | std::forward<U>(u)}$.

22.10.11.4 Class template `bit_xor`

[bitwise.operations.xor]

```

template<class T = void> struct bit_xor {
    constexpr T operator()(const T& x, const T& y) const;
};

```

```
constexpr T operator()(const T& x, const T& y) const;
```

¹ *Returns:* $x \wedge y$.

```

template<> struct bit_xor<void> {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) ^ std::forward<U>(u));

```

```

    using is_transparent = unspecified;
};

```

```

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) ^ std::forward<U>(u));

```

² *Returns:* $\text{std::forward<T>(t) ^ std::forward<U>(u)}$.

22.10.11.5 Class template `bit_not`

[bitwise.operations.not]

```

template<class T = void> struct bit_not {
    constexpr T operator()(const T& x) const;
};

```

```
constexpr T operator()(const T& x) const;
```

¹ *Returns:* $\sim x$.

```

template<> struct bit_not<void> {
    template<class T> constexpr auto operator()(T&& t) const
    -> decltype(~std::forward<T>(t));

```

```

    using is_transparent = unspecified;
};

```



```
template<class T> constexpr auto operator()(T&& t) const
-> decltype(~std::forward<T>(t));
```

2 *Returns:* `~std::forward<T>(t)`.

22.10.12 Class identity

[func.identity]

```
struct identity {
    template<class T>
        constexpr T&& operator()(T&& t) const noexcept;

    using is_transparent = unspecified;
};
```

```
template<class T>
    constexpr T&& operator()(T&& t) const noexcept;
```

1 *Effects:* Equivalent to: `return std::forward<T>(t)`;

22.10.13 Function template not_fn

[func.not.fn]

```
template<class F> constexpr unspecified not_fn(F&& f);
```

1 In the text that follows:

- (1.1) — `g` is a value of the result of a `not_fn` invocation,
 - (1.2) — `FD` is the type `decay_t<F>`,
 - (1.3) — `fd` is the target object of `g` (22.10.3) of type `FD`, direct-non-list-initialized with `std::forward<F>(f)`,
 - (1.4) — `call_args` is an argument pack used in a function call expression (7.6.1.3) of `g`.
- 2 *Mandates:* `is_constructible_v<FD, F> && is_move_constructible_v<FD>` is true.
- 3 *Preconditions:* `FD` meets the *Cpp17MoveConstructible* requirements.
- 4 *Returns:* A perfect forwarding call wrapper (22.10.4) `g` with call pattern `!invoke(fd, call_args...)`.
- 5 *Throws:* Any exception thrown by the initialization of `fd`.

```
template<auto f> constexpr unspecified not_fn() noexcept;
```

6 In the text that follows:

- (6.1) — `F` is the type of `f`,
 - (6.2) — `g` is a value of the result of a `not_fn` invocation,
 - (6.3) — `call_args` is an argument pack used in a function call expression (7.6.1.3) of `g`.
- 7 *Mandates:* If `is_pointer_v<F> || is_member_pointer_v<F>` is true, then `f != nullptr` is true.
- 8 *Returns:* A perfect forwarding call wrapper (22.10.4) `g` whose target object is a copy of `fw<f>`, and whose call pattern is `!invoke(f, call_args...)`.

22.10.14 Function templates bind_front and bind_back

[func.bind.partial]

```
template<class F, class... Args>
    constexpr unspecified bind_front(F&& f, Args&&... args);
template<class F, class... Args>
    constexpr unspecified bind_back(F&& f, Args&&... args);
```

1 Within this subclause:

- (1.1) — `g` is a value of the result of a `bind_front` or `bind_back` invocation,
- (1.2) — `FD` is the type `decay_t<F>`,
- (1.3) — `fd` is the target object of `g` (22.10.3) of type `FD`, direct-non-list-initialized with `std::forward<F>(f)`,
- (1.4) — `BoundArgs` is a pack that denotes `decay_t<Args>...`,
- (1.5) — `bound_args` is a pack of bound argument entities of `g` (22.10.3) of types `BoundArgs...`, direct-non-list-initialized with `std::forward<Args>(args)...`, respectively, and

(1.6) — `call_args` is an argument pack used in a function call expression (7.6.1.3) of `g`.

2 *Mandates:*

```
is_constructible_v<FD, F> &&
is_move_constructible_v<FD> &&
(is_constructible_v<BoundArgs, Args> && ...) &&
(is_move_constructible_v<BoundArgs> && ...)
```

is true.

3 *Preconditions:* `FD` meets the *Cpp17MoveConstructible* requirements. For each T_i in `BoundArgs`, if T_i is an object type, T_i meets the *Cpp17MoveConstructible* requirements.

4 *Returns:* A perfect forwarding call wrapper (22.10.4) `g` with call pattern:

(4.1) — `invoke(fd, bound_args..., call_args...)` for a `bind_front` invocation, or

(4.2) — `invoke(fd, call_args..., bound_args...)` for a `bind_back` invocation.

5 *Throws:* Any exception thrown by the initialization of the state entities of `g` (22.10.3).

```
template<auto f, class... Args>
constexpr unspecified bind_front(Args&&... args);
template<auto f, class... Args>
constexpr unspecified bind_back(Args&&... args);
```

6 Within this subclause:

(6.1) — `F` is the type of `f`,

(6.2) — `g` is a value of the result of a `bind_front` or `bind_back` invocation,

(6.3) — `BoundArgs` is a pack that denotes `decay_t<Args>...`,

(6.4) — `bound_args` is a pack of bound argument entities of `g` (22.10.3) of types `BoundArgs...`, direct-non-list-initialized with `std::forward<Args>(args)...`, respectively, and

(6.5) — `call_args` is an argument pack used in a function call expression (7.6.1.3) of `g`.

7 *Mandates:*

(7.1) — `(is_constructible_v<BoundArgs, Args> && ...)` is true, and

(7.2) — `(is_move_constructible_v<BoundArgs> && ...)` is true, and

(7.3) — if `is_pointer_v<F> || is_member_pointer_v<F>` is true, then `f != nullptr` is true.

8 *Preconditions:* For each T_i in `BoundArgs`, T_i meets the *Cpp17MoveConstructible* requirements.

9 *Returns:* A perfect forwarding call wrapper (22.10.4) `g` whose target object is a copy of `cw<f>`, and whose call pattern is:

(9.1) — `invoke(f, bound_args..., call_args...)` for a `bind_front` invocation, or

(9.2) — `invoke(f, call_args..., bound_args...)` for a `bind_back` invocation.

10 *Throws:* Any exception thrown by the initialization of `bound_args`.

22.10.15 Function object binders

[func.bind]

22.10.15.1 General

[func.bind.general]

1 Subclause 22.10.15 describes a uniform mechanism for binding arguments of callable objects.

22.10.15.2 Class template `is_bind_expression`

[func.bind.isbind]

```
namespace std {
    template<class T> struct is_bind_expression; // see below
}
```

1 The class template `is_bind_expression` can be used to detect function objects generated by `bind`. The function template `bind` uses `is_bind_expression` to detect subexpressions.

2 Specializations of the `is_bind_expression` template shall meet the *Cpp17UnaryTypeTrait* requirements (21.3.2). The implementation provides a definition that has a base characteristic of `true_type` if `T` is a type returned from `bind`, otherwise it has a base characteristic of `false_type`. A program may specialize this template for a program-defined type `T` to have a base characteristic of `true_type` to indicate that `T` should be treated as a subexpression in a `bind` call.

22.10.15.3 Class template `is_placeholder`**[func.bind.isplace]**

```
namespace std {
    template<class T> struct is_placeholder;    // see below
}
```

- ¹ The class template `is_placeholder` can be used to detect the standard placeholders `_1`, `_2`, and so on (22.10.15.5). The function template `bind` uses `is_placeholder` to detect placeholders.
- ² Specializations of the `is_placeholder` template shall meet the *Cpp17UnaryTypeTrait* requirements (21.3.2). The implementation provides a definition that has the base characteristic of `integral_constant<int, J>` if `T` is the type of `std::placeholders::_J`, otherwise it has a base characteristic of `integral_constant<int, 0>`. A program may specialize this template for a program-defined type `T` to have a base characteristic of `integral_constant<int, N>` with `N > 0` to indicate that `T` should be treated as a placeholder type.

22.10.15.4 Function template `bind`**[func.bind.bind]**

- ¹ In the text that follows:

- (1.1) — `g` is a value of the result of a `bind` invocation,
- (1.2) — `FD` is the type `decay_t<F>`,
- (1.3) — `fd` is an lvalue that is a target object of `g` (22.10.3) of type `FD` direct-non-list-initialized with `std::forward<F>(f)`,
- (1.4) — `Ti` is the i^{th} type in the template parameter pack `BoundArgs`,
- (1.5) — `TDi` is the type `decay_t<Ti>`,
- (1.6) — `ti` is the i^{th} argument in the function parameter pack `bound_args`,
- (1.7) — `tdi` is a bound argument entity of `g` (22.10.3) of type `TDi` direct-non-list-initialized with `std::forward<Ti>(ti)`,
- (1.8) — `Uj` is the j^{th} deduced type of the `UnBoundArgs&&...` parameter of the argument forwarding call wrapper, and
- (1.9) — `uj` is the j^{th} argument associated with `Uj`.

```
template<class F, class... BoundArgs>
constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
template<class R, class F, class... BoundArgs>
constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
```

- ² *Mandates:* `is_constructible_v<FD, F>` is true. For each `Ti` in `BoundArgs`, `is_constructible_v<TDi, Ti>` is true.
- ³ *Preconditions:* `FD` and each `TDi` meet the *Cpp17MoveConstructible* and *Cpp17Destructible* requirements. `INVOKE(fd, w1, w2, ..., wN)` (22.10.4) is a valid expression for some values `w1, w2, ..., wN`, where `N` has the value `sizeof...(bound_args)`.

- ⁴ *Returns:* An argument forwarding call wrapper `g` (22.10.4). A program that attempts to invoke a volatile-qualified `g` is ill-formed. When `g` is not volatile-qualified, invocation of `g(u1, u2, ..., uM)` is expression-equivalent (3.22) to

```
INVOKE(static_cast<Vfd>(vfd),
        static_cast<V1>(v1), static_cast<V2>(v2), ..., static_cast<VN>(vN))
```

for the first overload, and

```
INVOKE<R>(static_cast<Vfd>(vfd),
          static_cast<V1>(v1), static_cast<V2>(v2), ..., static_cast<VN>(vN))
```

for the second overload, where the values and types of the target argument `vfd` and of the bound arguments `v1, v2, ..., vN` are determined as specified below.

- ⁵ *Throws:* Any exception thrown by the initialization of the state entities of `g`.
- ⁶ [Note 1: If all of `FD` and `TDi` meet the requirements of *Cpp17CopyConstructible*, then the return type meets the requirements of *Cpp17CopyConstructible*. — end note]

- ⁷ The values of the *bound arguments* `v1, v2, ..., vN` and their corresponding types `V1, V2, ..., VN` depend on the types `TDi` derived from the call to `bind` and the cv-qualifiers *cv* of the call wrapper `g` as follows:

- (7.1) — if `TDi` is `reference_wrapper<T>`, the argument is `tdi.get()` and its type `Vi` is `T&`;

- (7.2) — if the value of `is_bind_expression_v<TDi>` is true, the argument is
`static_cast<cv TDi&>(tdi)(std::forward<Uj>(uj)...)`
and its type V_i is `invoke_result_t<cv TDi&, Uj...&&`;
- (7.3) — if the value `j` of `is_placeholder_v<TDi>` is not zero, the argument is `std::forward<Uj>(uj)` and its type V_i is `Uj&&`;
- (7.4) — otherwise, the value is `tdi` and its type V_i is `cv TDi&`.
- 8 The value of the target argument `vfd` is `fd` and its corresponding type V_{fd} is `cv FD&`.

22.10.15.5 Placeholders

[func.bind.place]

```
namespace std::placeholders {
    // M is the number of placeholders
    see below _1;
    see below _2;
    :
    see below _M;
}
```

- 1 The number M of placeholders is implementation-defined.
- 2 All placeholder types meet the *Cpp17DefaultConstructible* and *Cpp17CopyConstructible* requirements, and their default constructors and copy/move constructors are constexpr functions that do not throw exceptions. It is implementation-defined whether placeholder types meet the *Cpp17CopyAssignable* requirements, but if so, their copy assignment operators are constexpr functions that do not throw exceptions.
- 3 Placeholders should be defined as:

```
inline constexpr unspecified _1{};
```

If they are not, they are declared as:

```
extern unspecified _1;
```

- 4 Placeholders are freestanding items (16.3.3.7).

22.10.16 Function template `mem_fn`

[func.memfn]

```
template<class R, class T> constexpr unspecified mem_fn(R T::* pm) noexcept;
```

- 1 *Returns:* A simple call wrapper (22.10.4) `fn` with call pattern `invoke(pmd, call_args...)`, where `pmd` is the target object of `fn` of type `R T::*` direct-non-list-initialized with `pm`, and `call_args` is an argument pack used in a function call expression (7.6.1.3) of `fn`.

22.10.17 Polymorphic function wrappers

[func.wrap]

22.10.17.1 General

[func.wrap.general]

- 1 Subclause 22.10.17 describes polymorphic wrapper classes that encapsulate arbitrary callable objects.
- 2 Let `t` be an object of a type that is a specialization of `function`, `copyable_function`, `move_only_function`, or `function_ref`, such that the target object `x` of `t` has a type that is a specialization of `function`, `copyable_function`, `move_only_function`, or `function_ref`. Each argument of the invocation of `x` evaluated as part of the invocation of `t` may alias an argument in the same position in the invocation of `t` that has the same type, even if the corresponding parameter is not of reference type.

[Example 1:

```
move_only_function<void(T)> f{copyable_function<void(T)>{[] (T) {}}};
T t;
f(t);                                     // it is unspecified how many copies of T are made
```

— end example]

- 3 *Recommended practice:* Implementations should avoid double wrapping when constructing polymorphic wrappers from one another.

22.10.17.2 Class `bad_function_call`

[func.wrap.badcall]

- 1 An exception of type `bad_function_call` is thrown by `function::operator()` (22.10.17.3.5) when the function wrapper object has no target.

```

namespace std {
    class bad_function_call : public exception {
    public:
        // see 17.9.3 for the specification of the special member functions
        const char* what() const noexcept override;
    };
}

```

```
const char* what() const noexcept override;
```

² Returns: An implementation-defined NTBS.

22.10.17.3 Class template function

[func.wrap.func]

22.10.17.3.1 General

[func.wrap.func.general]

```

namespace std {
    template<class R, class... ArgTypes>
    class function<R(ArgTypes...)> {
    public:
        using result_type = R;

        // 22.10.17.3.2, construct/copy/destroy
        function() noexcept;
        function(nullptr_t) noexcept;
        function(const function&);
        function(function&&) noexcept;
        template<class F> function(F&&);

        function& operator=(const function&);
        function& operator=(function&&);
        function& operator=(nullptr_t) noexcept;
        template<class F> function& operator=(F&&);
        template<class F> function& operator=(reference_wrapper<F>) noexcept;

        ~function();

        // 22.10.17.3.3, function modifiers
        void swap(function&) noexcept;

        // 22.10.17.3.4, function capacity
        explicit operator bool() const noexcept;

        // 22.10.17.3.5, function invocation
        R operator()(ArgTypes...) const;

        // 22.10.17.3.6, function target access
        const type_info& target_type() const noexcept;
        template<class T> T* target() noexcept;
        template<class T> const T* target() const noexcept;
    };

    template<class R, class... ArgTypes>
    function(R*)(ArgTypes...) -> function<R(ArgTypes...)>;

    template<class F> function(F) -> function<see below>;
}

```

¹ The `function` class template provides polymorphic wrappers that generalize the notion of a function pointer. Wrappers can store, copy, and call arbitrary callable objects (22.10.3), given a call signature (22.10.3).

² The `function` class template is a call wrapper (22.10.3) whose call signature (22.10.3) is `R(ArgTypes...)`.

³ [Note 1: The types deduced by the deduction guides for `function` might change in future revisions of C++. — end note]

22.10.17.3.2 Constructors and destructor

[func.wrap.func.con]

```

function() noexcept;
1      Postconditions: !*this.

function(nullptr_t) noexcept;
2      Postconditions: !*this.

function(const function& f);
3      Postconditions: !*this if !f; otherwise, the target object of *this is a copy of the target object of f.
4      Throws: Nothing if f's target is a specialization of reference_wrapper or a function pointer. Otherwise,
      may throw bad_alloc or any exception thrown by the copy constructor of the stored callable object.
5      Recommended practice: Implementations should avoid the use of dynamically allocated memory for
      small callable objects, for example, where f's target is an object holding only a pointer or reference to
      an object and a member function pointer.

function(function&& f) noexcept;
6      Postconditions: If !f, *this has no target; otherwise, the target of *this is equivalent to the target of
      f before the construction, and f is in a valid state with an unspecified value.
7      Recommended practice: Implementations should avoid the use of dynamically allocated memory for
      small callable objects, for example, where f's target is an object holding only a pointer or reference to
      an object and a member function pointer.

template<class F> function(F&& f);
8      Let FD be decay_t<F>.
9      Constraints:
(9.1)      — is_same_v<remove_cvref_t<F>, function> is false, and
(9.2)      — is_invocable_r_v<R, FD&, ArgTypes...> is true.
10     Mandates:
(10.1)     — is_copy_constructible_v<FD> is true, and
(10.2)     — is_constructible_v<FD, F> is true.
11     Preconditions: FD meets the Cpp17CopyConstructible requirements.
12     Postconditions: !*this is true if any of the following hold:
(12.1)     — f is a null function pointer value.
(12.2)     — f is a null member pointer value.
(12.3)     — remove_cvref_t<F> is a specialization of the function class template, and !f is true.
13     Otherwise, *this has a target object of type FD direct-non-list-initialized with std::forward<F>(f).
14     Throws: Nothing if FD is a specialization of reference_wrapper or a function pointer type. Otherwise,
      may throw bad_alloc or any exception thrown by the initialization of the target object.
15     Recommended practice: Implementations should avoid the use of dynamically allocated memory for
      small callable objects, for example, where f refers to an object holding only a pointer or reference to an
      object and a member function pointer.

template<class F> function(F) -> function<see below>;
16     Constraints: &F::operator() is well-formed when treated as an unevaluated operand and either
(16.1)     — F::operator() is a non-static member function and decltype(&F::operator()) is either of the
      form R(G::*)(A...) cv &opt noexceptopt or of the form R(*) (G, A...) noexceptopt for a type
      G, or
(16.2)     — F::operator() is a static member function and decltype(&F::operator()) is of the form
      R(*) (A...) noexceptopt.
17     Remarks: The deduced type is function<R(A...)>.

```

```

18      [Example 1:
        void f() {
            int i{5};
            function g = [&](double) { return i; };           // deduces function<int(double)>
        }
        —end example]

function& operator=(const function& f);
19      Effects: As if by function(f).swap(*this);
20      Returns: *this.

function& operator=(function&& f);
21      Effects: Replaces the target of *this with the target of f.
22      Returns: *this.

function& operator=(nullptr_t) noexcept;
23      Effects: If *this != nullptr, destroys the target of this.
24      Postconditions: !(*this).
25      Returns: *this.

template<class F> function& operator=(F&& f);
26      Constraints: is_invocable_r_v<R, decay_t<F>&, ArgTypes...> is true.
27      Effects: As if by: function(std::forward<F>(f)).swap(*this);
28      Returns: *this.

template<class F> function& operator=(reference_wrapper<F> f) noexcept;
29      Effects: As if by: function(f).swap(*this);
30      Returns: *this.

~function();
31      Effects: If *this != nullptr, destroys the target of this.

```

22.10.17.3.3 Modifiers

[func.wrap.func.mod]

```
void swap(function& other) noexcept;
```

```
1      Effects: Interchanges the target objects of *this and other.
```

22.10.17.3.4 Capacity

[func.wrap.func.cap]

```
explicit operator bool() const noexcept;
```

```
1      Returns: true if *this has a target, otherwise false.
```

22.10.17.3.5 Invocation

[func.wrap.func.inv]

```
R operator()(ArgTypes... args) const;
```

```
1      Returns: INVOKE<R>(f, std::forward<ArgTypes>(args)...) (22.10.4), where f is the target object
(22.10.3) of *this.
```

```
2      Throws: bad_function_call if !*this; otherwise, any exception thrown by the target object.
```

22.10.17.3.6 Target access

[func.wrap.func.targ]

```
const type_info& target_type() const noexcept;
```

```
1      Returns: If *this has a target of type T, typeid(T); otherwise, typeid(void).
```

```
template<class T>      T* target() noexcept;
template<class T> const T* target() const noexcept;
```

- 2 *Returns:* If `target_type() == typeid(T)` a pointer to the stored function target; otherwise a null pointer.

22.10.17.3.7 Null pointer comparison operator functions [func.wrap.func.nullptr]

```
template<class R, class... ArgTypes>
bool operator==(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
```

- 1 *Returns:* !f.

22.10.17.3.8 Specialized algorithms [func.wrap.func.alg]

```
template<class R, class... ArgTypes>
void swap(function<R(ArgTypes...)>& f1, function<R(ArgTypes...)>& f2) noexcept;
```

- 1 *Effects:* As if by: `f1.swap(f2);`

22.10.17.4 Move-only wrapper [func.wrap.move]

22.10.17.4.1 General [func.wrap.move.general]

- 1 The header provides partial specializations of `move_only_function` for each combination of the possible replacements of the placeholders *cv*, *ref*, and *noex* where

- (1.1) — *cv* is either `const` or empty,
- (1.2) — *ref* is either `&`, `&&`, or empty, and
- (1.3) — *noex* is either `true` or `false`.

- 2 For each of the possible combinations of the placeholders mentioned above, there is a placeholder *inv-quals* defined as follows:

- (2.1) — If *ref* is empty, let *inv-quals* be *cv&*,
- (2.2) — otherwise, let *inv-quals* be *cv ref*.

22.10.17.4.2 Class template `move_only_function` [func.wrap.move.class]

```
namespace std {
    template<class R, class... ArgTypes>
    class move_only_function<R(ArgTypes...) cv ref noexcept(noex)> {
    public:
        using result_type = R;

        // 22.10.17.4.3, constructors, assignments, and destructor
        move_only_function() noexcept;
        move_only_function(nullptr_t) noexcept;
        move_only_function(move_only_function&&) noexcept;
        template<class F> move_only_function(F&&);
        template<class T, class... Args>
            explicit move_only_function(in_place_type_t<T>, Args&&...);
        template<class T, class U, class... Args>
            explicit move_only_function(in_place_type_t<T>, initializer_list<U>, Args&&...);

        move_only_function& operator=(move_only_function&&);
        move_only_function& operator=(nullptr_t) noexcept;
        template<class F> move_only_function& operator=(F&&);

        ~move_only_function();

        // 22.10.17.4.4, invocation
        explicit operator bool() const noexcept;
        R operator()(ArgTypes...) cv ref noexcept(noex);

        // 22.10.17.4.5, utility
        void swap(move_only_function&) noexcept;
        friend void swap(move_only_function&, move_only_function&) noexcept;
        friend bool operator==(const move_only_function&, nullptr_t) noexcept;
```



```

private:
    template<class VT>
        static constexpr bool is-callable-from = see below;           // exposition only
};

```

¹ The `move_only_function` class template provides polymorphic wrappers that generalize the notion of a callable object (22.10.3). These wrappers can store, move, and call arbitrary callable objects, given a call signature.

² *Recommended practice*: Implementations should avoid the use of dynamically allocated memory for a small contained value.

[*Note 1*: Such small-object optimization can only be applied to a type `T` for which `is_nothrow_move_constructible_v<T>` is `true`. — end note]

22.10.17.4.3 Constructors, assignments, and destructor

[`func.wrap.move.ctor`]

```

template<class VT>
    static constexpr bool is-callable-from = see below;

```

¹ If `noex` is `true`, `is-callable-from<VT>` is equal to:

```

    is_nothrow_invocable_r_v<R, VT cv ref, ArgTypes...> &&
    is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes...>

```

Otherwise, `is-callable-from<VT>` is equal to:

```

    is_invocable_r_v<R, VT cv ref, ArgTypes...> &&
    is_invocable_r_v<R, VT inv-quals, ArgTypes...>

```

```

move_only_function() noexcept;
move_only_function(nullptr_t) noexcept;

```

² *Postconditions*: `*this` has no target object.

```

move_only_function(move_only_function&& f) noexcept;

```

³ *Postconditions*: The target object of `*this` is the target object `f` had before construction, and `f` is in a valid state with an unspecified value.

```

template<class F> move_only_function(F&& f);

```

⁴ Let `VT` be `decay_t<F>`.

⁵ *Constraints*:

- (5.1) — `remove_cvref_t<F>` is not the same type as `move_only_function`, and
- (5.2) — `remove_cvref_t<F>` is not a specialization of `in_place_type_t`, and
- (5.3) — `is-callable-from<VT>` is `true`.

⁶ *Mandates*: `is_constructible_v<VT, F>` is `true`.

⁷ *Preconditions*: `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

⁸ *Postconditions*: `*this` has no target object if any of the following hold:

- (8.1) — `f` is a null function pointer value,
- (8.2) — `f` is a null member pointer value, or
- (8.3) — `remove_cvref_t<F>` is a specialization of the `move_only_function` or `copyable_function` class template, and `f` has no target object.

Otherwise, `*this` has a target object of type `VT` direct-non-list-initialized with `std::forward<F>(f)`.

⁹ *Throws*: Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a function pointer or a specialization of `reference_wrapper`.

```

template<class T, class... Args>
    explicit move_only_function(in_place_type_t<T>, Args&&... args);

```

¹⁰ Let `VT` be `decay_t<T>`.

11 *Constraints:*

(11.1) — `is_constructible_v<VT, Args...>` is true, and

(11.2) — *is-callable-from*<VT> is true.

12 *Mandates:* VT is the same type as T.

13 *Preconditions:* VT meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is true, VT meets the *Cpp17MoveConstructible* requirements.

14 *Postconditions:* `*this` has a target object of type VT direct-non-list-initialized with `std::forward<Args>(args)...`

15 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless VT is a function pointer or a specialization of `reference_wrapper`.

`template<class T, class U, class... Args>`
`explicit move_only_function(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);`

16 Let VT be `decay_t<T>`.

17 *Constraints:*

(17.1) — `is_constructible_v<VT, initializer_list<U>&, Args...>` is true, and

(17.2) — *is-callable-from*<VT> is true.

18 *Mandates:* VT is the same type as T.

19 *Preconditions:* VT meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is true, VT meets the *Cpp17MoveConstructible* requirements.

20 *Postconditions:* `*this` has a target object of type VT direct-non-list-initialized with `ilist, std::forward<Args>(args)...`

21 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless VT is a function pointer or a specialization of `reference_wrapper`.

`move_only_function& operator=(move_only_function&& f);`

22 *Effects:* Equivalent to: `move_only_function(std::move(f)).swap(*this);`

23 *Returns:* `*this`.

`move_only_function& operator=(nullptr_t) noexcept;`

24 *Effects:* Destroys the target object of `*this`, if any.

25 *Returns:* `*this`.

`template<class F> move_only_function& operator=(F&& f);`

26 *Effects:* Equivalent to: `move_only_function(std::forward<F>(f)).swap(*this);`

27 *Returns:* `*this`.

`~move_only_function();`

28 *Effects:* Destroys the target object of `*this`, if any.

22.10.17.4.4 Invocation

[func.wrap.move.inv]

`explicit operator bool() const noexcept;`

1 *Returns:* true if `*this` has a target object, otherwise false.

`R operator()(ArgTypes... args) cv ref noexcept(noex);`

2 *Preconditions:* `*this` has a target object.

3 *Effects:* Equivalent to:

`return INVOKE<R>(static_cast<F inv-quals>(f), std::forward<ArgTypes>(args)...);`

where `f` is an lvalue designating the target object of `*this` and `F` is the type of `f`.

22.10.17.4.5 Utility

[func.wrap.move.util]

```
void swap(move_only_function& other) noexcept;
```

¹ *Effects:* Exchanges the target objects of **this* and *other*.

```
friend void swap(move_only_function& f1, move_only_function& f2) noexcept;
```

² *Effects:* Equivalent to *f1.swap(f2)*.

```
friend bool operator==(const move_only_function& f, nullptr_t) noexcept;
```

³ *Returns:* *true* if *f* has no target object, otherwise *false*.

22.10.17.5 Copyable wrapper

[func.wrap.copy]

22.10.17.5.1 General

[func.wrap.copy.general]

¹ The header provides partial specializations of *copyable_function* for each combination of the possible replacements of the placeholders *cv*, *ref*, and *noex* where

(1.1) — *cv* is either *const* or *empty*,

(1.2) — *ref* is either *&*, *&&*, or *empty*, and

(1.3) — *noex* is either *true* or *false*.

² For each of the possible combinations of the placeholders mentioned above, there is a placeholder *inv-quals* defined as follows:

(2.1) — If *ref* is *empty*, let *inv-quals* be *cv&*,

(2.2) — otherwise, let *inv-quals* be *cv ref*.

22.10.17.5.2 Class template copyable_function

[func.wrap.copy.class]

```
namespace std {
    template<class R, class... ArgTypes>
    class copyable_function<R(ArgTypes...) cv ref noexcept(noex)> {
    public:
        using result_type = R;

        // 22.10.17.5.3, constructors, assignments, and destructor
        copyable_function() noexcept;
        copyable_function(nullptr_t) noexcept;
        copyable_function(const copyable_function&);
        copyable_function(copyable_function&&) noexcept;
        template<class F> copyable_function(F&&);
        template<class T, class... Args>
            explicit copyable_function(in_place_type_t<T>, Args&&...);
        template<class T, class U, class... Args>
            explicit copyable_function(in_place_type_t<T>, initializer_list<U>, Args&&...);

        copyable_function& operator=(const copyable_function&);
        copyable_function& operator=(copyable_function&&);
        copyable_function& operator=(nullptr_t) noexcept;
        template<class F> copyable_function& operator=(F&&);

        ~copyable_function();

        // 22.10.17.5.4, invocation
        explicit operator bool() const noexcept;
        R operator()(ArgTypes...) cv ref noexcept(noex);

        // 22.10.17.5.5, utility
        void swap(copyable_function&) noexcept;
        friend void swap(copyable_function&, copyable_function&) noexcept;
        friend bool operator==(const copyable_function&, nullptr_t) noexcept;
```

```

private:
    template<class VT>
        static constexpr bool is-callable-from = see below;           // exposition only
};

```

¹ The `copyable_function` class template provides polymorphic wrappers that generalize the notion of a callable object (22.10.3). These wrappers can store, copy, move, and call arbitrary callable objects, given a call signature.

² *Recommended practice*: Implementations should avoid the use of dynamically allocated memory for a small contained value.

[*Note 1*: Such small-object optimization can only be applied to a type `T` for which `is_nothrow_move_constructible_v<T>` is `true`. — end note]

22.10.17.5.3 Constructors, assignments, and destructor

[`func.wrap.copy.ctor`]

```

template<class VT>
    static constexpr bool is-callable-from = see below;

```

¹ If `noex` is `true`, `is-callable-from<VT>` is equal to:

```

    is_nothrow_invocable_r_v<R, VT cv ref, ArgTypes...> &&
    is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes...>

```

Otherwise, `is-callable-from<VT>` is equal to:

```

    is_invocable_r_v<R, VT cv ref, ArgTypes...> &&
    is_invocable_r_v<R, VT inv-quals, ArgTypes...>

```

```

copyable_function() noexcept;
copyable_function(nullptr_t) noexcept;

```

² *Postconditions*: `*this` has no target object.

```

copyable_function(const copyable_function& f);

```

³ *Postconditions*: `*this` has no target object if `f` had no target object. Otherwise, the target object of `*this` is a copy of the target object of `f`.

⁴ *Throws*: Any exception thrown by the initialization of the target object. May throw `bad_alloc`.

```

copyable_function(copyable_function&& f) noexcept;

```

⁵ *Postconditions*: The target object of `*this` is the target object `f` had before construction, and `f` is in a valid state with an unspecified value.

```

template<class F> copyable_function(F&& f);

```

⁶ Let `VT` be `decay_t<F>`.

⁷ *Constraints*:

- (7.1) — `remove_cvref_t<F>` is not the same type as `copyable_function`, and
- (7.2) — `remove_cvref_t<F>` is not a specialization of `in_place_type_t`, and
- (7.3) — `is-callable-from<VT>` is `true`.

⁸ *Mandates*:

- (8.1) — `is_constructible_v<VT, F>` is `true`, and
- (8.2) — `is_copy_constructible_v<VT>` is `true`.

⁹ *Preconditions*: `VT` meets the *Cpp17Destructible* and *Cpp17CopyConstructible* requirements.

¹⁰ *Postconditions*: `*this` has no target object if any of the following hold:

- (10.1) — `f` is a null function pointer value, or
- (10.2) — `f` is a null member pointer value, or
- (10.3) — `remove_cvref_t<F>` is a specialization of the `copyable_function` class template, and `f` has no target object.

Otherwise, `*this` has a target object of type VT direct-non-list-initialized with `std::forward<F>(f)`.

11 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless VT is a function pointer or a specialization of `reference_wrapper`.

```
template<class T, class... Args>
  explicit copyable_function(in_place_type_t<T>, Args&&... args);
```

12 Let VT be `decay_t<T>`.

13 *Constraints:*

(13.1) — `is_constructible_v<VT, Args...>` is true, and

(13.2) — *is-callable-from*<VT> is true.

14 *Mandates:*

(14.1) — VT is the same type as T, and

(14.2) — `is_copy_constructible_v<VT>` is true.

15 *Preconditions:* VT meets the *Cpp17Destructible* and *Cpp17CopyConstructible* requirements.

16 *Postconditions:* `*this` has a target object of type VT direct-non-list-initialized with `std::forward<Args>(args)...`

17 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless VT is a pointer or a specialization of `reference_wrapper`.

```
template<class T, class U, class... Args>
  explicit copyable_function(in_place_type_t<T>, initializer_list<U> ilist, Args&&... args);
```

18 Let VT be `decay_t<T>`.

19 *Constraints:*

(19.1) — `is_constructible_v<VT, initializer_list<U>&, Args...>` is true, and

(19.2) — *is-callable-from*<VT> is true.

20 *Mandates:*

(20.1) — VT is the same type as T, and

(20.2) — `is_copy_constructible_v<VT>` is true.

21 *Preconditions:* VT meets the *Cpp17Destructible* and *Cpp17CopyConstructible* requirements.

22 *Postconditions:* `*this` has a target object of type VT direct-non-list-initialized with `ilist, std::forward<Args>(args)...`

23 *Throws:* Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless VT is a pointer or a specialization of `reference_wrapper`.

```
copyable_function& operator=(const copyable_function& f);
```

24 *Effects:* Equivalent to: `copyable_function(f).swap(*this);`

25 *Returns:* `*this`.

```
copyable_function& operator=(copyable_function&& f);
```

26 *Effects:* Equivalent to: `copyable_function(std::move(f)).swap(*this);`

27 *Returns:* `*this`.

```
copyable_function& operator=(nullptr_t) noexcept;
```

28 *Effects:* Destroys the target object of `*this`, if any.

29 *Returns:* `*this`.

```
template<class F> copyable_function& operator=(F&& f);
```

30 *Effects:* Equivalent to: `copyable_function(std::forward<F>(f)).swap(*this);`

31 *Returns:* `*this`.

~copyable_function();

32 *Effects:* Destroys the target object of *this, if any.

22.10.17.5.4 Invocation

[func.wrap.copy.inv]

explicit operator bool() const noexcept;

1 *Returns:* true if *this has a target object, otherwise false.

R operator()(ArgTypes... args) cv ref noexcept(*noex*);

2 *Preconditions:* *this has a target object.

3 *Effects:* Equivalent to:

```
return INVOKE<R>(static_cast<F inv-quals>(f), std::forward<ArgTypes>(args)...);
```

where f is an lvalue designating the target object of *this and F is the type of f.

22.10.17.5.5 Utility

[func.wrap.copy.util]

void swap(copyable_function& other) noexcept;

1 *Effects:* Exchanges the target objects of *this and other.

friend void swap(copyable_function& f1, copyable_function& f2) noexcept;

2 *Effects:* Equivalent to f1.swap(f2).

friend bool operator==(const copyable_function& f, nullptr_t) noexcept;

3 *Returns:* true if f has no target object, otherwise false.

22.10.17.6 Non-owning wrapper

[func.wrap.ref]

22.10.17.6.1 General

[func.wrap.ref.general]

1 The header provides partial specializations of `function_ref` for each combination of the possible replacements of the placeholders *cv* and *noex* where:

(1.1) — *cv* is either `const` or empty, and

(1.2) — *noex* is either `true` or `false`.

22.10.17.6.2 Class template `function_ref`

[func.wrap.ref.class]

```
namespace std {
    template<class R, class... ArgTypes>
    class function_ref<R(ArgTypes...) cv noexcept(noex)> {
    public:
        // 22.10.17.6.3, constructors and assignment operators
        template<class F> function_ref(F*) noexcept;
        template<class F> constexpr function_ref(F&&) noexcept;
        template<auto c, class F> constexpr function_ref(constant_wrapper<c, F>) noexcept;
        template<auto c, class F, class U>
            constexpr function_ref(constant_wrapper<c, F>, U&&) noexcept;
        template<auto c, class F, class T>
            constexpr function_ref(constant_wrapper<c, F>, cv T*) noexcept;

        constexpr function_ref(const function_ref&) noexcept = default;
        constexpr function_ref& operator=(const function_ref&) noexcept = default;
        template<class T> function_ref& operator=(T) = delete;

        // 22.10.17.6.4, invocation
        R operator()(ArgTypes...) const noexcept(noex);

    private:
        template<class... T>
            static constexpr bool is_invocable_using = see below; // exposition only
    };
}
```

```

    R (*thunk_ptr)(BoundEntityType, ArgTypes&&...) noexcept(noex);           // exposition only
    BoundEntityType bound-entity;                                           // exposition only
};

```

```

// 22.10.17.6.5, deduction guides

```

```

template<class F>
    function_ref(F*) -> function_ref<F>;
template<auto c, class F0>
    function_ref(constant_wrapper<c, F0>) -> function_ref<see below>;
template<auto c, class F, class T>
    function_ref(constant_wrapper<c, F>, T&&) -> function_ref<see below>;
}

```

¹ An object of class `function_ref<R(ArgTypes...) cv noexcept(noex)>` stores a pointer to function *thunk_ptr* and an object *bound-entity*. The object *bound-entity* has an unspecified trivially copyable type *BoundEntityType*, that models copyable and is capable of storing a pointer to object value or a pointer to function value. The type of *thunk_ptr* is `R(*) (BoundEntityType, ArgTypes&&...) noexcept(noex)`.

² Each specialization of `function_ref` is a trivially copyable type (6.9.1) that models copyable.

³ Within subclause 22.10.17.6, *call-args* is an argument pack with elements such that

```
decltype((call-args))...
```

denote `ArgTypes&&...` respectively.

22.10.17.6.3 Constructors and assignment operators

[func.wrap.ref.ctor]

```

template<class... T>
    static constexpr bool is-invocable-using = see below;

```

¹ If *noex* is true, *is-invocable-using*<T...> is equal to:

```
is_nothrow_invocable_r_v<R, T..., ArgTypes...>
```

Otherwise, *is-invocable-using*<T...> is equal to:

```
is_invocable_r_v<R, T..., ArgTypes...>
```

```

template<class F>
    static constexpr bool is-convertible-from-specialization = see below;

```

² If *F* denotes a specialization `function_ref<R(ArgTypes...) cv2 noexcept(noex2)>` for some placeholders *cv2* and *noex2*, *is-convertible-from-specialization*<*F*> is equal to:

```
is_convertible_v<R(&)(ArgTypes...) noexcept(noex2), R(&)(ArgTypes...) noexcept(noex)> &&
is_convertible_v<int cv&, int cv2&>.
```

Otherwise, *is-convertible-from-specialization*<*F*> is false.

```

template<class F> function_ref(F* f) noexcept;

```

³ *Constraints:*

(3.1) — *is_function_v*<*F*> is true, and

(3.2) — *is-invocable-using*<*F*> is true.

⁴ *Preconditions:* *f* is not a null pointer.

⁵ *Effects:* Initializes *bound-entity* with *f*, and *thunk_ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent (3.22) to *invoke_r*<*R*>(f, *call-args*...).

```

template<class F> constexpr function_ref(F&& f) noexcept;

```

⁶ Let *T* be `remove_reference_t<F>`.

⁷ *Constraints:*

(7.1) — *remove_cvref_t*<*F*> is not the same type as `function_ref`,

(7.2) — *is_member_pointer_v*<*T*> is false, and

(7.3) — *is-invocable-using*<*cv T*&> is true.

8 *Effects:* If *is-convertible-from-specialization*<remove_cv_t<T>> is false, initializes *bound-entity* with *addressof*(f), and *thunk_ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent (3.22) to *invoke_r*<R>(static_cast<cv T>(f), *call-args*...). Otherwise, initializes *bound-entity* with the value of f.*bound-entity* and *thunk_ptr* with the value of f.*thunk_ptr*.

9 *Remarks:* If *remove_cvref_t*<F> is a specialization of *function_ref*, an implementation may initialize *bound-entity* with the value of f.*bound-entity* and *thunk_ptr* with the value of f.*thunk_ptr*.

[Example 1:

```
void f1(std::string);
void f2(std::string);

function_ref<void(std::string)> r1(&f1);
function_ref<void(std::string&&)> r2(r1);
r2(""); // f1 is invoked
r1 = &f2;
r2(""); // it is unspecified if f1 or f2 is invoked
```

— end example]

```
template<auto c, class F> constexpr function_ref(constant_wrapper<c, F> f) noexcept;
```

10 *Constraints:* *is-invokable-using*<const F&> is true.

11 *Mandates:*

(11.1) — If *is_pointer_v*<F> || *is_member_pointer_v*<F> is true, then f.value != nullptr is true, and

(11.2) — if ArgTypes is not an empty pack and all types in *remove_cvref_t*<ArgTypes>... satisfy *constexpr-param* then constant_wrapper<INVOKE(f.value, *remove_cvref_t*<ArgTypes>::value...)> is not a valid type.

12 *Effects:* Initializes *bound-entity* with a pointer to an unspecified object or null pointer value, and *thunk_ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent (3.22) to *invoke_r*<R>(f.value, *call-args*...).

```
template<auto c, class F, class U>
constexpr function_ref(constant_wrapper<c, F> f, U&& obj) noexcept;
```

13 Let T be *remove_reference_t*<U>.

14 *Constraints:*

(14.1) — *is_rvalue_reference_v*<U&&> is false, and

(14.2) — *is-invokable-using*<const F&, cv T&> is true.

15 *Mandates:* If *is_pointer_v*<F> || *is_member_pointer_v*<F> is true, then f.value != nullptr is true.

16 *Effects:* Initializes *bound-entity* with *addressof*(obj), and *thunk_ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent (3.22) to *invoke_r*<R>(f.value, static_cast<cv T>(obj), *call-args*...).

```
template<auto c, class F, class T>
constexpr function_ref(constant_wrapper<c, F> f, cv T* obj) noexcept;
```

17 *Constraints:* *is-invokable-using*<const F&, cv T*> is true.

18 *Mandates:* If *is_pointer_v*<F> || *is_member_pointer_v*<F> is true, then f.value != nullptr is true.

19 *Preconditions:* If *is_member_pointer_v*<F> is true, obj is not a null pointer.

20 *Effects:* Initializes *bound-entity* with obj, and *thunk_ptr* with the address of a function *thunk* such that *thunk*(*bound-entity*, *call-args*...) is expression-equivalent (3.22) to *invoke_r*<R>(f.value, obj, *call-args*...).

```
template<class T> function_ref& operator=(T) = delete;
```

21 *Constraints:*

- (21.1) — *is-convertible-from-specialization*<T> is false,
- (21.2) — *is_pointer_v*<T> is false, and
- (21.3) — T is not a specialization of *constant_wrapper*.

22.10.17.6.4 Invocation

[func.wrap.ref.inv]

R operator() (ArgTypes... args) const noexcept(*noex*);

- 1 *Effects*: Equivalent to: return *thunk_ptr*(*bound-entity*, std::forward<ArgTypes>(args)...);

22.10.17.6.5 Deduction guides

[func.wrap.ref.deduct]

```
template<class F>
function_ref(F*) -> function_ref<F>;
```

- 1 *Constraints*: *is_function_v*<F> is true.

```
template<auto c, class F0>
function_ref(constant_wrapper<c, F0>) -> function_ref<see below>;
```

- 2 Let F be *remove_pointer_t*<F0>.

- 3 *Constraints*: *is_function_v*<F> is true.

- 4 *Remarks*: The deduced type is *function_ref*<F>.

```
template<auto c, class F, class T>
function_ref(constant_wrapper<c, F>, T&&) -> function_ref<see below>;
```

- 5 *Constraints*:

- (5.1) — F is of the form *R*(*G*::*)(*A*...) *cv* &*opt* noexcept(*E*) for a type *G*, or
- (5.2) — F is of the form *M* *G*::* for a type *G* and an object type *M*, in which case let *R* be *invoke_result_t*<F, *T*&>, *A*... be an empty pack, and *E* be true, or
- (5.3) — F is of the form *R*(*)(*G*, *A*...) noexcept(*E*) for a type *G*.

- 6 *Remarks*: The deduced type is *function_ref*<*R*(*A*...) noexcept(*E*)>.

22.10.18 Searchers

[func.search]

22.10.18.1 General

[func.search.general]

- 1 Subclause 22.10.18 provides function object types (22.10) for operations that search for a sequence [*pat_first*, *pat_last*) in another sequence [*first*, *last*) that is provided to the object's function call operator. The first sequence (the pattern to be searched for) is provided to the object's constructor, and the second (the sequence to be searched) is provided to the function call operator.
- 2 Each specialization of a class template specified in 22.10.18 shall meet the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements. Template parameters named

- (2.1) — *ForwardIterator*,
- (2.2) — *ForwardIterator1*,
- (2.3) — *ForwardIterator2*,
- (2.4) — *RandomAccessIterator*,
- (2.5) — *RandomAccessIterator1*,
- (2.6) — *RandomAccessIterator2*, and
- (2.7) — *BinaryPredicate*

of templates specified in 22.10.18 shall meet the same requirements and semantics as specified in 26.1. Template parameters named *Hash* shall meet the *Cpp17Hash* requirements (Table 37).

- 3 The Boyer-Moore searcher implements the Boyer-Moore search algorithm. The Boyer-Moore-Horspool searcher implements the Boyer-Moore-Horspool search algorithm. In general, the Boyer-Moore searcher will use more memory and give better runtime performance than Boyer-Moore-Horspool.

22.10.18.2 Class template default_searcher

[func.search.default]

```

namespace std {
    template<class ForwardIterator1, class BinaryPredicate = equal_to<>>
    class default_searcher {
    public:
        constexpr default_searcher(ForwardIterator1 pat_first, ForwardIterator1 pat_last,
                                   BinaryPredicate pred = BinaryPredicate());

        template<class ForwardIterator2>
        constexpr pair<ForwardIterator2, ForwardIterator2>
        operator()(ForwardIterator2 first, ForwardIterator2 last) const;

    private:
        ForwardIterator1 pat_first_;           // exposition only
        ForwardIterator1 pat_last_;            // exposition only
        BinaryPredicate pred_;                 // exposition only
    };
}

```

```

constexpr default_searcher(ForwardIterator1 pat_first, ForwardIterator1 pat_last,
                           BinaryPredicate pred = BinaryPredicate());

```

1 *Effects:* Constructs a default_searcher object, initializing pat_first_ with pat_first, pat_last_ with pat_last, and pred_ with pred.

2 *Throws:* Any exception thrown by the copy constructor of BinaryPredicate or ForwardIterator1.

```

template<class ForwardIterator2>
constexpr pair<ForwardIterator2, ForwardIterator2>
operator()(ForwardIterator2 first, ForwardIterator2 last) const;

```

3 *Effects:* Returns a pair of iterators i and j such that

(3.1) — i == search(first, last, pat_first_, pat_last_, pred_), and

(3.2) — if i == last, then j == last, otherwise j == next(i, distance(pat_first_, pat_last_)).

22.10.18.3 Class template boyer_moore_searcher

[func.search.bm]

```

namespace std {
    template<class RandomAccessIterator1,
             class Hash = hash<typename iterator_traits<RandomAccessIterator1>::value_type>,
             class BinaryPredicate = equal_to<>>
    class boyer_moore_searcher {
    public:
        boyer_moore_searcher(RandomAccessIterator1 pat_first,
                             RandomAccessIterator1 pat_last,
                             Hash hf = Hash(),
                             BinaryPredicate pred = BinaryPredicate());

        template<class RandomAccessIterator2>
        pair<RandomAccessIterator2, RandomAccessIterator2>
        operator()(RandomAccessIterator2 first, RandomAccessIterator2 last) const;

    private:
        RandomAccessIterator1 pat_first_;     // exposition only
        RandomAccessIterator1 pat_last_;      // exposition only
        Hash hash_;                           // exposition only
        BinaryPredicate pred_;                 // exposition only
    };
}

```

```

boyer_moore_searcher(RandomAccessIterator1 pat_first,
                      RandomAccessIterator1 pat_last,
                      Hash hf = Hash(),
                      BinaryPredicate pred = BinaryPredicate());

```

1 *Preconditions:* The value type of RandomAccessIterator1 meets the Cpp17DefaultConstructible,

Cpp17CopyConstructible, and *Cpp17CopyAssignable* requirements.

Let V be `iterator_traits<RandomAccessIterator1>::value_type`. For any two values A and B of type V , if `pred(A, B) == true`, then `hf(A) == hf(B)` is true.

Effects: Initializes `pat_first_` with `pat_first`, `pat_last_` with `pat_last`, `hash_` with `hf`, and `pred_` with `pred`.

Throws: Any exception thrown by the copy constructor of `RandomAccessIterator1`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `RandomAccessIterator1`, or the copy constructor or `operator()` of `BinaryPredicate` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<class RandomAccessIterator2>
pair<RandomAccessIterator2, RandomAccessIterator2>
operator()(RandomAccessIterator2 first, RandomAccessIterator2 last) const;
```

Mandates: `RandomAccessIterator1` and `RandomAccessIterator2` have the same value type.

Effects: Finds a subsequence of equal values in a sequence.

Returns: A pair of iterators i and j such that

— i is the first iterator in the range $[first, last - (pat_last_ - pat_first_))$ such that for every non-negative integer n less than `pat_last_ - pat_first_` the following condition holds: `pred(*(i + n), *(pat_first_ + n)) != false`, and

— $j == next(i, distance(pat_first_, pat_last_))$.

Returns `make_pair(first, first)` if $[pat_first_, pat_last_)$ is empty, otherwise returns `make_pair(last, last)` if no such iterator is found.

Complexity: At most $(last - first) * (pat_last_ - pat_first_)$ applications of the predicate.

22.10.18.4 Class template `boyer_moore_horspool_searcher` [func.search.bmh]

```
namespace std {
    template<class RandomAccessIterator1,
             class Hash = hash<typename iterator_traits<RandomAccessIterator1>::value_type>,
             class BinaryPredicate = equal_to<>>
    class boyer_moore_horspool_searcher {
    public:
        boyer_moore_horspool_searcher(RandomAccessIterator1 pat_first,
                                       RandomAccessIterator1 pat_last,
                                       Hash hf = Hash(),
                                       BinaryPredicate pred = BinaryPredicate());

        template<class RandomAccessIterator2>
        pair<RandomAccessIterator2, RandomAccessIterator2>
        operator()(RandomAccessIterator2 first, RandomAccessIterator2 last) const;

    private:
        RandomAccessIterator1 pat_first_; // exposition only
        RandomAccessIterator1 pat_last_; // exposition only
        Hash hash_; // exposition only
        BinaryPredicate pred_; // exposition only
    };
}
```

```
boyer_moore_horspool_searcher(RandomAccessIterator1 pat_first,
                               RandomAccessIterator1 pat_last,
                               Hash hf = Hash(),
                               BinaryPredicate pred = BinaryPredicate());
```

Preconditions: The value type of `RandomAccessIterator1` meets the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, and *Cpp17CopyAssignable* requirements.

Let V be `iterator_traits<RandomAccessIterator1>::value_type`. For any two values A and B of type V , if `pred(A, B) == true`, then `hf(A) == hf(B)` is true.

3 *Effects:* Initializes `pat_first_` with `pat_first`, `pat_last_` with `pat_last`, `hash_` with `hf`, and `pred_` with `pred`.

4 *Throws:* Any exception thrown by the copy constructor of `RandomAccessIterator1`, or by the default constructor, copy constructor, or the copy assignment operator of the value type of `RandomAccessIterator1`, or the copy constructor or `operator()` of `BinaryPredicate` or `Hash`. May throw `bad_alloc` if additional memory needed for internal data structures cannot be allocated.

```
template<class RandomAccessIterator2>
pair<RandomAccessIterator2, RandomAccessIterator2>
operator()(RandomAccessIterator2 first, RandomAccessIterator2 last) const;
```

5 *Mandates:* `RandomAccessIterator1` and `RandomAccessIterator2` have the same value type.

6 *Effects:* Finds a subsequence of equal values in a sequence.

7 *Returns:* A pair of iterators `i` and `j` such that

(7.1) — `i` is the first iterator in the range `[first, last - (pat_last_ - pat_first_))` such that for every non-negative integer `n` less than `pat_last_ - pat_first_` the following condition holds: `pred(*(i + n), *(pat_first_ + n)) != false`, and

(7.2) — `j == next(i, distance(pat_first_, pat_last_))`.

Returns `make_pair(first, first)` if `[pat_first_, pat_last_)` is empty, otherwise returns `make_pair(last, last)` if no such iterator is found.

8 *Complexity:* At most `(last - first) * (pat_last_ - pat_first_)` applications of the predicate.

22.10.19 Class template `hash`

[unord.hash]

1 The unordered associative containers defined in 23.5 use specializations of the class template `hash` (22.10.2) as the default hash function.

2 Each specialization of `hash` is either enabled or disabled, as described below.

[Note 1: Enabled specializations meet the *Cpp17Hash* requirements, and disabled specializations do not. — end note]

Each header that declares the template `hash` provides enabled specializations of `hash` for `nullptr_t` and all cv-unqualified arithmetic, enumeration, and pointer types. For any type `Key` for which neither the library nor the user provides an explicit or partial specialization of the class template `hash`, `hash<Key>` is disabled.

3 If the library provides an explicit or partial specialization of `hash<Key>`, that specialization is enabled except as noted otherwise, and its member functions are `noexcept` except as noted otherwise.

4 If `H` is a disabled specialization of `hash`, these values are `false`: `is_default_constructible_v<H>`, `is_copy_constructible_v<H>`, `is_move_constructible_v<H>`, `is_copy_assignable_v<H>`, and `is_move_assignable_v<H>`. Disabled specializations of `hash` are not function object types (22.10).

[Note 2: This means that the specialization of `hash` exists, but any attempts to use it as a *Cpp17Hash* will be ill-formed. — end note]

5 An enabled specialization `hash<Key>` will:

(5.1) — meet the *Cpp17Hash* requirements (Table 37), with `Key` as the function call argument type, the *Cpp17DefaultConstructible* requirements (Table 30), the *Cpp17CopyAssignable* requirements (Table 34), the *Cpp17Swappable* requirements (16.4.4.3),

(5.2) — meet the requirement that if `k1 == k2` is `true`, `h(k1) == h(k2)` is also `true`, where `h` is an object of type `hash<Key>` and `k1` and `k2` are objects of type `Key`;

(5.3) — meet the requirement that the expression `h(k)`, where `h` is an object of type `hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `hash<Key>` is a program-defined specialization.

22.11 Bit manipulation

[bit]

22.11.1 General

[bit.general]

1 The header `<bit>` provides components to access, manipulate and process both individual bits and bit sequences.

22.11.2 Header <bit> synopsis

[bit.syn]

```

// all freestanding
namespace std {
    // 22.11.3, bit_cast
    template<class To, class From>
        constexpr To bit_cast(const From& from) noexcept;

    // 22.11.4, byteswap
    template<class T>
        constexpr T byteswap(T value) noexcept;

    // 22.11.5, integral powers of 2
    template<class T>
        constexpr bool has_single_bit(T x) noexcept;
    template<class T>
        constexpr T bit_ceil(T x);
    template<class T>
        constexpr T bit_floor(T x) noexcept;
    template<class T>
        constexpr int bit_width(T x) noexcept;

    // 22.11.6, shifting
    template<class T, class S>
        constexpr T shl(T x, S s) noexcept;
    template<class T, class S>
        constexpr T shr(T x, S s) noexcept;

    // 22.11.7, rotating
    template<class T>
        constexpr T rotl(T x, int s) noexcept;
    template<class T>
        constexpr T rotr(T x, int s) noexcept;

    // 22.11.8, counting
    template<class T>
        constexpr int countl_zero(T x) noexcept;
    template<class T>
        constexpr int countl_one(T x) noexcept;
    template<class T>
        constexpr int countr_zero(T x) noexcept;
    template<class T>
        constexpr int countr_one(T x) noexcept;
    template<class T>
        constexpr int popcount(T x) noexcept;

    // 22.11.9, permutation
    template<class T>
        constexpr T bit_reverse(T x) noexcept;
    template<class T>
        constexpr T bit_repeat(T x, int l);
    template<class T>
        constexpr T bit_compress(T x, T m) noexcept;
    template<class T>
        constexpr T bit_expand(T x, T m) noexcept;

    // 22.11.10, endian
    enum class endian {
        little = see below,
        big    = see below,
        native = see below
    };
}

```

22.11.3 Function template `bit_cast`**[`bit.cast`]**

```
template<class To, class From>
constexpr To bit_cast(const From& from) noexcept;
```

1 *Constraints:*

- (1.1) — `sizeof(To) == sizeof(From)` is true;
- (1.2) — `is_trivially_copyable_v<To>` is true; and
- (1.3) — `is_trivially_copyable_v<From>` is true.

2 *Constant When:* Neither `To` nor `From` has constexpr-unknown representation (7.7.2).

3 *Returns:* An object of type `To`. Implicitly creates objects nested within the result (6.8.2). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. Every trivially copyable object among the result and each object created within it acquires the value representation produced; if any such object does not receive a value, the behavior is undefined. A bit in the value representation of the result is indeterminate if it does not correspond to a bit in the value representation of `from` or corresponds to a bit for which the smallest enclosing object is not within its lifetime or has an indeterminate value (6.8.5). A bit in the value representation of the result is erroneous if it corresponds to a bit for which the smallest enclosing object has an erroneous value. For each bit *b* in the value representation of the result that is indeterminate or erroneous, let *u* be the smallest object containing that bit enclosing *b*:

- (3.1) — If *u* is of unsigned ordinary character type or `std::byte` type, *u* has an indeterminate value if any of the bits in its value representation are indeterminate, or otherwise has an erroneous value.
- (3.2) — Otherwise, if *b* is indeterminate, the behavior is undefined.
- (3.3) — Otherwise, the behavior is erroneous, and the result is as specified above.

The result does not otherwise contain any indeterminate or erroneous values.

22.11.4 `byteswap`**[`bit.byteswap`]**

```
template<class T>
constexpr T byteswap(T value) noexcept;
```

1 *Constraints:* `T` models `integral`.

2 *Mandates:* `T` does not have padding bits (6.9.1).

3 Let the sequence *R* comprise the bytes of the object representation of `value` in reverse order.

4 *Returns:* An object *v* of type `T` such that each byte in the object representation of *v* is equal to the byte in the corresponding position in *R*.

22.11.5 Integral powers of 2**[`bit.pow.two`]**

```
template<class T>
constexpr bool has_single_bit(T x) noexcept;
```

1 *Constraints:* `T` is an unsigned integer type (6.9.3).

2 *Returns:* `true` if `x` is an integral power of two; `false` otherwise.

```
template<class T>
constexpr T bit_ceil(T x);
```

3 Let *N* be the smallest power of 2 greater than or equal to `x`.

4 *Constraints:* `T` is an unsigned integer type (6.9.3).

5 *Preconditions:* *N* is representable as a value of type `T`.

6 *Returns:* *N*.

7 *Throws:* Nothing.

8 *Remarks:* A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression (7.7.2).

```
template<class T>
constexpr T bit_floor(T x) noexcept;
```

9 *Constraints:* T is an unsigned integer type (6.9.3).

10 *Returns:* If $x == 0$, 0; otherwise the maximal value y such that `has_single_bit(y)` is true and $y \leq x$.

```
template<class T>
constexpr int bit_width(T x) noexcept;
```

11 *Constraints:* T is an unsigned integer type (6.9.3).

12 *Returns:* If $x == 0$, 0; otherwise one plus the base-2 logarithm of x , with any fractional part discarded.

22.11.6 Shifting

[bit.shift]

1 Within this subclause, the result r of an arithmetic operation is first represented in a hypothetical integer type with sufficient range, then converted to T as if by `static_cast<T>(r)`.

```
template<class T, class S>
constexpr T shl(T x, S s) noexcept;
```

2 *Constraints:* Each of T and S is a signed or unsigned integer type (6.9.3).

3 *Returns:* $x \times 2^s$ rounded towards negative infinity.

```
template<class T, class S>
constexpr T shr(T x, S s) noexcept;
```

4 *Constraints:* Each of T and S is a signed or unsigned integer type (6.9.3).

5 *Returns:* $x \times 2^{-s}$ rounded towards negative infinity.

22.11.7 Rotating

[bit.rotate]

1 In the following descriptions, let N denote `numeric_limits<T>::digits`.

```
template<class T>
constexpr T rotl(T x, int s) noexcept;
```

2 *Constraints:* T is an unsigned integer type (6.9.3).

3 Let r be $s \% N$.

4 *Returns:* If r is 0, x ; if r is positive, $(x \ll r) \mid (x \gg (N - r))$; if r is negative, `rotr(x, -r)`.

```
template<class T>
constexpr T rotr(T x, int s) noexcept;
```

5 *Constraints:* T is an unsigned integer type (6.9.3).

6 Let r be $s \% N$.

7 *Returns:* If r is 0, x ; if r is positive, $(x \gg r) \mid (x \ll (N - r))$; if r is negative, `rotr(x, -r)`.

22.11.8 Counting

[bit.count]

1 In the following descriptions, let N denote `numeric_limits<T>::digits`.

```
template<class T>
constexpr int countl_zero(T x) noexcept;
```

2 *Constraints:* T is an unsigned integer type (6.9.3).

3 *Returns:* The number of consecutive 0 bits in the value of x , starting from the most significant bit.
[Note 1: Returns N if $x == 0$. — end note]

```
template<class T>
constexpr int countl_one(T x) noexcept;
```

4 *Constraints:* T is an unsigned integer type (6.9.3).

5 *Returns:* The number of consecutive 1 bits in the value of x , starting from the most significant bit.
[Note 2: Returns N if $x == \text{numeric_limits<T>::max}()$. — end note]

```
template<class T>
constexpr int countr_zero(T x) noexcept;
```

6 *Constraints:* T is an unsigned integer type (6.9.3).

7 *Returns:* The number of consecutive 0 bits in the value of x, starting from the least significant bit.

[Note 3: Returns N if x == 0. — end note]

```
template<class T>
constexpr int countr_one(T x) noexcept;
```

8 *Constraints:* T is an unsigned integer type (6.9.3).

9 *Returns:* The number of consecutive 1 bits in the value of x, starting from the least significant bit.

[Note 4: Returns N if x == numeric_limits<T>::max(). — end note]

```
template<class T>
constexpr int popcount(T x) noexcept;
```

10 *Constraints:* T is an unsigned integer type (6.9.3).

11 *Returns:* The number of 1 bits in the value of x.

22.11.9 Permutation

[bit.permute]

1 In the following descriptions,

(1.1) — let N denote the value of numeric_limits<T>::digits,

(1.2) — let α_n denote the value of the n^{th} least significant bit in the base-2 representation of an integer α , so that α equals $\sum_{n=0}^{N-1} \alpha_n 2^n$, and

(1.3) — let $\sigma(\alpha, n) = \sum_{k=0}^{n-1} \alpha_k$ be the count of one-bits among the lowest n bits of α .

```
template<class T>
constexpr T bit_reverse(T x) noexcept;
```

2 *Constraints:* T is an unsigned integer type (6.9.3).

3 *Returns:* reverse(x), where reverse is given by Formula 22.1, and x is x.

$$\text{reverse}(x) = \sum_{n=0}^{N-1} x_n 2^{N-n-1} \quad (22.1)$$

4 [Note 1: bit_reverse(bit_reverse(x)) equals x. — end note]

```
template<class T>
constexpr T bit_repeat(T x, int l);
```

5 *Constraints:* T is an unsigned integer type (6.9.3).

6 *Preconditions:* l is greater than zero.

7 *Returns:* repeat(x, l), where repeat is given by Formula 22.2, x is x, and l is l.

$$\text{repeat}(x, l) = \sum_{n=0}^{N-1} x_{(n \bmod l)} 2^n \quad (22.2)$$

8 *Throws:* Nothing.

9 *Remarks:* A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression (7.7.2).

10 [Example 1: bit_repeat(uint32_t{0xc}, 4) equals 0xcccccccc. — end example]

```
template<class T>
constexpr T bit_compress(T x, T m) noexcept;
```

11 *Constraints:* T is an unsigned integer type (6.9.3).

12 *Returns:* `compress(x, m)`, where `compress` is given by Formula 22.3, x is $\mathbf{x}$, and m is $\mathbf{m}$.

$$\text{compress}(x, m) = \sum_{n=0}^{N-1} m_n x_n 2^{\sigma(m,n)} \quad (22.3)$$

13 [Example 2: `bit_compress(0bABCD, 0b0101)` equals `0b00BD` for any bits A , B , C , and D . — end example]

```
template<class T>
constexpr T bit_expand(T x, T m) noexcept;
```

14 *Constraints:* T is an unsigned integer type (6.9.3).

15 *Returns:* `expand(x, m)`, where `expand` is given by Formula 22.4, x is $\mathbf{x}$, and m is $\mathbf{m}$.

$$\text{expand}(x, m) = \sum_{n=0}^{N-1} m_n x_{\sigma(m,n)} 2^n \quad (22.4)$$

16 [Example 3: `bit_expand(0bABCD, 0b0101)` equals `0b0C0D` for any bits A , B , C , and D . — end example]

22.11.10 Endian

[bit.endian]

1 Two common methods of byte ordering in multibyte scalar types are big-endian and little-endian in the execution environment. Big-endian is a format for storage of binary data in which the most significant byte is placed first, with the rest in descending order. Little-endian is a format for storage of binary data in which the least significant byte is placed first, with the rest in ascending order. This subclause describes the endianness of the scalar types of the execution environment.

```
enum class endian {
    little = see below,
    big    = see below,
    native = see below
};
```

2 If all scalar types have size 1 byte, then all of `endian::little`, `endian::big`, and `endian::native` have the same value. Otherwise, `endian::little` is not equal to `endian::big`. If all scalar types are big-endian, `endian::native` is equal to `endian::big`. If all scalar types are little-endian, `endian::native` is equal to `endian::little`. Otherwise, `endian::native` is not equal to either `endian::big` or `endian::little`.

22.12 Header <stdbit.h> synopsis

[stdbit.h.syn]

// all freestanding

```
#define __STDC_VERSION_STDBIT_H__ 202311L
```

```
#define __STDC_ENDIAN_BIG__      see below
```

```
#define __STDC_ENDIAN_LITTLE__  see below
```

```
#define __STDC_ENDIAN_NATIVE__  see below
```

```
unsigned int stdc_leading_zeros_uc(unsigned char value);
unsigned int stdc_leading_zeros_us(unsigned short value);
unsigned int stdc_leading_zeros_ui(unsigned int value);
unsigned int stdc_leading_zeros_ul(unsigned long int value);
unsigned int stdc_leading_zeros_ull(unsigned long long int value);
template<class T> see below stdc_leading_zeros(T value);
```

```
unsigned int stdc_leading_ones_uc(unsigned char value);
unsigned int stdc_leading_ones_us(unsigned short value);
unsigned int stdc_leading_ones_ui(unsigned int value);
unsigned int stdc_leading_ones_ul(unsigned long int value);
unsigned int stdc_leading_ones_ull(unsigned long long int value);
template<class T> see below stdc_leading_ones(T value);
```

```

unsigned int stdc_trailing_zeros_uc(unsigned char value);
unsigned int stdc_trailing_zeros_us(unsigned short value);
unsigned int stdc_trailing_zeros_ui(unsigned int value);
unsigned int stdc_trailing_zeros_ul(unsigned long int value);
unsigned int stdc_trailing_zeros_ull(unsigned long long int value);
template<class T> see below stdc_trailing_zeros(T value);

unsigned int stdc_trailing_ones_uc(unsigned char value);
unsigned int stdc_trailing_ones_us(unsigned short value);
unsigned int stdc_trailing_ones_ui(unsigned int value);
unsigned int stdc_trailing_ones_ul(unsigned long int value);
unsigned int stdc_trailing_ones_ull(unsigned long long int value);
template<class T> see below stdc_trailing_ones(T value);

unsigned int stdc_first_leading_zero_uc(unsigned char value);
unsigned int stdc_first_leading_zero_us(unsigned short value);
unsigned int stdc_first_leading_zero_ui(unsigned int value);
unsigned int stdc_first_leading_zero_ul(unsigned long int value);
unsigned int stdc_first_leading_zero_ull(unsigned long long int value);
template<class T> see below stdc_first_leading_zero(T value);

unsigned int stdc_first_leading_one_uc(unsigned char value);
unsigned int stdc_first_leading_one_us(unsigned short value);
unsigned int stdc_first_leading_one_ui(unsigned int value);
unsigned int stdc_first_leading_one_ul(unsigned long int value);
unsigned int stdc_first_leading_one_ull(unsigned long long int value);
template<class T> see below stdc_first_leading_one(T value);

unsigned int stdc_first_trailing_zero_uc(unsigned char value);
unsigned int stdc_first_trailing_zero_us(unsigned short value);
unsigned int stdc_first_trailing_zero_ui(unsigned int value);
unsigned int stdc_first_trailing_zero_ul(unsigned long int value);
unsigned int stdc_first_trailing_zero_ull(unsigned long long int value);
template<class T> see below stdc_first_trailing_zero(T value);

unsigned int stdc_first_trailing_one_uc(unsigned char value);
unsigned int stdc_first_trailing_one_us(unsigned short value);
unsigned int stdc_first_trailing_one_ui(unsigned int value);
unsigned int stdc_first_trailing_one_ul(unsigned long int value);
unsigned int stdc_first_trailing_one_ull(unsigned long long int value);
template<class T> see below stdc_first_trailing_one(T value);

unsigned int stdc_count_zeros_uc(unsigned char value);
unsigned int stdc_count_zeros_us(unsigned short value);
unsigned int stdc_count_zeros_ui(unsigned int value);
unsigned int stdc_count_zeros_ul(unsigned long int value);
unsigned int stdc_count_zeros_ull(unsigned long long int value);
template<class T> see below stdc_count_zeros(T value);

unsigned int stdc_count_ones_uc(unsigned char value);
unsigned int stdc_count_ones_us(unsigned short value);
unsigned int stdc_count_ones_ui(unsigned int value);
unsigned int stdc_count_ones_ul(unsigned long int value);
unsigned int stdc_count_ones_ull(unsigned long long int value);
template<class T> see below stdc_count_ones(T value);

bool stdc_has_single_bit_uc(unsigned char value);
bool stdc_has_single_bit_us(unsigned short value);
bool stdc_has_single_bit_ui(unsigned int value);
bool stdc_has_single_bit_ul(unsigned long int value);
bool stdc_has_single_bit_ull(unsigned long long int value);
template<class T> bool stdc_has_single_bit(T value);

```

```

unsigned int stdc_bit_width_uc(unsigned char value);
unsigned int stdc_bit_width_us(unsigned short value);
unsigned int stdc_bit_width_ui(unsigned int value);
unsigned int stdc_bit_width_ul(unsigned long int value);
unsigned int stdc_bit_width_ull(unsigned long long int value);
template<class T> see below stdc_bit_width(T value);

unsigned char stdc_bit_floor_uc(unsigned char value);
unsigned short stdc_bit_floor_us(unsigned short value);
unsigned int stdc_bit_floor_ui(unsigned int value);
unsigned long int stdc_bit_floor_ul(unsigned long int value);
unsigned long long int stdc_bit_floor_ull(unsigned long long int value);
template<class T> T stdc_bit_floor(T value);

unsigned char stdc_bit_ceil_uc(unsigned char value);
unsigned short stdc_bit_ceil_us(unsigned short value);
unsigned int stdc_bit_ceil_ui(unsigned int value);
unsigned long int stdc_bit_ceil_ul(unsigned long int value);
unsigned long long int stdc_bit_ceil_ull(unsigned long long int value);
template<class T> T stdc_bit_ceil(T value);

```

- ¹ For a function template whose return type is not specified above, the return type is an implementation-defined unsigned integer type large enough to represent all possible result values. Each function template has the same semantics as the corresponding type-generic function with the same name specified in SEE ALSO: ISO/IEC 9899:2024, 7.18.
- ² *Mandates:* T is an unsigned integer type.
- ³ Otherwise, the contents and meaning of the header <stdbit.h> are the same as the C standard library header <stdbit.h>.

SEE ALSO: ISO/IEC 9899:2024, 7.18

23 Containers library

[containers]

23.1 General

[containers.general]

- ¹ This Clause describes components that C++ programs may use to organize collections of information.
- ² The following subclauses describe container requirements, and components for sequence containers and associative containers, as summarized in [Table 74](#).

Table 74 — Containers library summary [tab:containers.summary]

Subclause	Header
23.2 Requirements	
23.3 Sequence containers	<code><array></code> , <code><deque></code> , <code><forward_list></code> , <code><hive></code> , <code><inplace_vector></code> , <code><list></code> , <code><vector></code>
23.4 Associative containers	<code><map></code> , <code><set></code>
23.5 Unordered associative containers	<code><unordered_map></code> , <code><unordered_set></code>
23.6 Container adaptors	<code><queue></code> , <code><stack></code> , <code><flat_map></code> , <code><flat_set></code>
23.7 Views	<code></code> , <code><mdspan></code>

23.2 Requirements

[container.requirements]

23.2.1 Preamble

[container.requirements.pre]

- ¹ Containers are objects that store other objects. They control allocation and deallocation of these objects through constructors, destructors, insert and erase operations.
- ² All of the complexity requirements in this Clause are stated solely in terms of the number of operations on the contained objects.

[*Example 1*: The copy constructor of type `vector<vector<int>>` has linear complexity, even though the complexity of copying each contained `vector<int>` is itself linear. — *end example*]

- ³ Allocator-aware containers ([23.2.2.5](#)) other than `basic_string` construct elements using the function `allocator_traits<allocator_type>::rebind_traits<U>::construct` and destroy elements using the function `allocator_traits<allocator_type>::rebind_traits<U>::destroy` ([20.2.9.3](#)), where `U` is either `allocator_type::value_type` or an internal type used by the container. These functions are called only for the container's element type, not for internal types used by the container.

[*Note 1*: This means, for example, that a node-based container would need to construct nodes containing aligned buffers and call `construct` to place the element into the buffer. — *end note*]

23.2.2 General containers

[container.requirements.general]

23.2.2.1 Introduction

[container.intro.reqmts]

- ¹ In [23.2.2](#),

- (1.1) — `X` denotes a container class containing objects of type `T`,
- (1.2) — `a` denotes a value of type `X`,
- (1.3) — `b` and `c` denote values of type (possibly `const`) `X`,
- (1.4) — `i` and `j` denote values of type (possibly `const`) `X::iterator`,
- (1.5) — `u` denotes an identifier,
- (1.6) — `v` denotes an lvalue of type (possibly `const`) `X` or an rvalue of type `const X`,
- (1.7) — `s` and `t` denote non-`const` lvalues of type `X`, and
- (1.8) — `rv` denotes a non-`const` rvalue of type `X`.

- ² The following exposition-only concept is used in the definition of containers:

```
template<class R, class T>
concept container-compatible-range = // exposition only
    ranges::input_range<R> && convertible_to<ranges::range_reference_t<R>, T>;
```

23.2.2.2 Container requirements

[container.reqmts]

- 1 A type *X* meets the *container* requirements if the following types, statements, and expressions are well-formed and have the specified semantics.

```
typename X::value_type
```

- 2 *Result:* T

- 3 *Preconditions:* T is *Cpp17Erasable* from X (see 23.2.2.5, below).

```
typename X::reference
```

- 4 *Result:* T&

```
typename X::const_reference
```

- 5 *Result:* const T&

```
typename X::iterator
```

- 6 *Result:* A type that meets the forward iterator requirements (24.3.5.5) with value type T. The type *X::iterator* is convertible to *X::const_iterator*.

```
typename X::const_iterator
```

- 7 *Result:* A type that meets the requirements of a constant iterator and those of a forward iterator with value type T.

```
typename X::difference_type
```

- 8 *Result:* A signed integer type, identical to the difference type of *X::iterator* and *X::const_iterator*.

```
typename X::size_type
```

- 9 *Result:* An unsigned integer type that can represent any non-negative value of *X::difference_type*.

```
X u;
X u = X();
```

- 10 *Postconditions:* u.empty()

- 11 *Complexity:* Constant.

```
X u(v);
X u = v;
```

- 12 *Preconditions:* T is *Cpp17CopyInsertable* into X (see below).

- 13 *Postconditions:* u == v.

- 14 *Complexity:* Linear.

```
X u(rv);
X u = rv;
```

- 15 *Postconditions:* u is equal to the value that rv had before this construction.

- 16 *Complexity:* Linear for *array* and *inplace_vector* and constant for all other standard containers.

```
t = v
```

- 17 *Result:* X&.

- 18 *Postconditions:* t == v.

- 19 *Complexity:* Linear.

```
t = rv
```

- 20 *Result:* X&.

- 21 *Effects:* All existing elements of t are either move assigned to or destroyed.

22 *Postconditions:* If `t` and `rv` do not refer to the same object, `t` is equal to the value that `rv` had before this assignment.

23 *Complexity:* Linear.

`a.~X()`

24 *Result:* `void`.

25 *Effects:* Destroys every element of `a`; any memory obtained is deallocated.

26 *Complexity:* Linear.

`b.begin()`

27 *Result:* `const_iterator` if `b` is of type `const X`; `iterator` otherwise.

28 *Returns:* An iterator referring to the first element in the container.

29 *Complexity:* Constant.

`b.end()`

30 *Result:* `const_iterator` if `b` is of type `const X`; `iterator` otherwise.

31 *Returns:* An iterator which is the past-the-end value for the container.

32 *Complexity:* Constant.

`b.cbegin()`

33 *Result:* `const_iterator`.

34 *Returns:* `const_cast<X const&>(b).begin()`

35 *Complexity:* Constant.

`b.cend()`

36 *Result:* `const_iterator`.

37 *Returns:* `const_cast<X const&>(b).end()`

38 *Complexity:* Constant.

`i <=> j`

39 *Result:* `strong_ordering`.

40 *Constraints:* `X::iterator` meets the random access iterator requirements.

41 *Complexity:* Constant.

`c == b`

42 *Preconditions:* `T` meets the *Cpp17EqualityComparable* requirements.

43 *Result:* `bool`.

44 *Returns:* `equal(c.begin(), c.end(), b.begin(), b.end())`
 [*Note 1:* The algorithm `equal` is defined in [26.6.13](#). — *end note*]

45 *Complexity:* Constant if `c.size() != b.size()`, linear otherwise.

46 *Remarks:* `==` is an equivalence relation.

`c != b`

47 *Effects:* Equivalent to `!(c == b)`.

`t.swap(s)`

48 *Result:* `void`.

49 *Effects:* Exchanges the contents of `t` and `s`.

50 *Complexity:* Linear for `array` and `inplace_vector`, and constant for all other standard containers.

`swap(t, s)`

51 *Effects:* Equivalent to `t.swap(s)`.

`c.size()`

52 *Result:* `size_type`.

53 *Returns:* `distance(c.begin(), c.end())`, i.e., the number of elements in the container.

54 *Complexity:* Constant.

55 *Remarks:* The number of elements is defined by the rules of constructors, inserts, and erases.

`c.max_size()`

56 *Result:* `size_type`.

57 *Returns:* `distance(begin(), end())` for the largest possible container.

58 *Complexity:* Constant.

`c.empty()`

59 *Result:* `bool`.

60 *Returns:* `c.begin() == c.end()`

61 *Complexity:* Constant.

62 *Remarks:* If the container is empty, then `c.empty()` is `true`.

63 In the expressions

```
i == j
i != j
i < j
i <= j
i >= j
i > j
i <=> j
i - j
```

where `i` and `j` denote objects of a container's `iterator` type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

64 Unless otherwise specified, all containers defined in this Clause obtain memory using an allocator (see 16.4.4.6).

[Note 2: In particular, containers and iterators do not store references to allocated elements other than through the allocator's pointer type, i.e., as objects of type `P` or `pointer_traits<P>::template rebind<unspecified>`, where `P` is `allocator_traits<allocator_type>::pointer`. — end note]

Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. Such move construction of the allocator shall not exit via an exception. All other constructors for these container types take a `const allocator_type&` argument.

[Note 3: If an invocation of a constructor uses the default value of an optional allocator argument, then the allocator type must support value-initialization. — end note]

A copy of this allocator is used for any memory allocation and element construction performed, by these constructors and by all other member functions, during the lifetime of each container object or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if

(64.1) — `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value`,

(64.2) — `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value`, or

(64.3) — `allocator_traits<allocator_type>::propagate_on_container_swap::value`

is `true` within the implementation of the corresponding container operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement.

- 65 The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than `array` and `inplace_vector`, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Any `Compare`, `Pred`, or `Hash` types belonging to `a` and `b` shall meet the *Cpp17Swappable* requirements and shall be exchanged by calling `swap` as described in 16.4.4.3. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is `true`, then `allocator_type` shall meet the *Cpp17Swappable* requirements and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 16.4.4.3. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.
- 66 Unless otherwise specified (see 23.2.7.2, 23.2.8.2, 23.3.5.4, 23.3.16.5, and 23.3.13.5) all container types defined in this Clause meet the following additional requirements:
- (66.1) — If an exception is thrown by an `insert()` or `emplace()` function while inserting a single element, that function has no effects.
 - (66.2) — If an exception is thrown by a `push_back()`, `push_front()`, `emplace_back()`, or `emplace_front()` function, that function has no effects.
 - (66.3) — No `erase()`, `clear()`, `pop_back()` or `pop_front()` function throws an exception.
 - (66.4) — No copy constructor or assignment operator of a returned iterator throws an exception.
 - (66.5) — No `swap()` function throws an exception.
 - (66.6) — No `swap()` function invalidates any references, pointers, or iterators referring to the elements of the containers being swapped.
- [Note 4: The `end()` iterator does not refer to any element, so it can be invalidated. — end note]
- 67 Unless otherwise specified (either explicitly or by defining a function in terms of other functions), invoking a container member function or passing a container as an argument to a library function shall not invalidate iterators to, or change the values of, objects within that container.
- 68 A *contiguous container* is a container whose member types `iterator` and `const_iterator` meet the *Cpp17RandomAccessIterator* requirements (24.3.5.7) and model *contiguous_iterator* (24.3.4.14).
- 69 The behavior of certain container member functions and deduction guides depends on whether types qualify as input iterators or allocators. The extent to which an implementation determines that a type cannot be an input iterator is unspecified, except that as a minimum integral types shall not qualify as input iterators. Likewise, the extent to which an implementation determines that a type cannot be an allocator is unspecified, except that as a minimum a type `A` shall not qualify as an allocator unless it meets both of the following conditions:
- (69.1) — The *qualified-id* `A::value_type` is valid and denotes a type (13.10.3).
 - (69.2) — The expression `declval<A&>().allocate(size_t{})` is well-formed when treated as an unevaluated operand.

23.2.2.3 Reversible container requirements

[container.rev.reqmts]

- 1 A type `X` meets the *reversible container* requirements if `X` meets the container requirements, the iterator type of `X` belongs to the bidirectional or random access iterator categories (24.3), and the following types and expressions are well-formed and have the specified semantics.

typename `X::reverse_iterator`

- 2 *Result:* The type `reverse_iterator<X::iterator>`, an iterator type whose value type is `T`.

typename `X::const_reverse_iterator`

- 3 *Result:* The type `reverse_iterator<X::const_iterator>`, a constant iterator type whose value type is `T`.

`a.rbegin()`

- 4 *Result:* `const_reverse_iterator` if `a` is of type `const X`; `reverse_iterator` otherwise.

- 5 *Returns:* `reverse_iterator(end())`

- 6 *Complexity:* Constant.

`a.rend()`

7 *Result:* `const_reverse_iterator` if `a` is of type `const X`; `reverse_iterator` otherwise.

8 *Returns:* `reverse_iterator(begin())`

9 *Complexity:* Constant.

`a.crbegin()`

10 *Result:* `const_reverse_iterator`.

11 *Returns:* `const_cast<X const&>(a).rbegin()`

12 *Complexity:* Constant.

`a.crend()`

13 *Result:* `const_reverse_iterator`.

14 *Returns:* `const_cast<X const&>(a).rend()`

15 *Complexity:* Constant.

23.2.2.4 Optional container requirements

[`container.opt.reqmts`]

- 1 The following operations are provided for some types of containers but not others. Those containers for which the listed operations are provided shall implement the semantics as described unless otherwise stated. If the iterators passed to `lexicographical_compare_three_way` meet the `constexpr` iterator requirements (24.3.1) then the operations described below are implemented by `constexpr` functions.

`a <=> b`

2 *Result:* `synth-three-way-result<X::value_type>`.

3 *Preconditions:* Either `T` models `three_way_comparable`, or `<` is defined for values of type (possibly `const`) `T` and `<` is a total ordering relationship.

4 *Returns:* `lexicographical_compare_three_way(a.begin(), a.end(), b.begin(), b.end(), synth-three-way)`

[*Note 1:* The algorithm `lexicographical_compare_three_way` is defined in Clause 26. — end note]

5 *Complexity:* Linear.

23.2.2.5 Allocator-aware containers

[`container.alloc.reqmts`]

- 1 Except for `array` and `inplace_vector`, all of the containers defined in Clause 23, 19.6.4, 27.4.3, and 28.6.9 meet the additional requirements of an *allocator-aware container*, as described below.

- 2 Given an allocator type `A` and given a container type `X` having a `value_type` identical to `T` and an `allocator_type` identical to `allocator_traits<A>::rebind_alloc<T>` and given an lvalue `m` of type `A`, a pointer `p` of type `T*`, an expression `v` that denotes an lvalue of type `T` or `const T` or an rvalue of type `const T`, and an rvalue `rv` of type `T`, the following terms are defined. If `X` is not allocator-aware or is a specialization of `basic_string`, the terms below are defined as if `A` were `allocator<T>` — no allocator object needs to be created and user specializations of `allocator<T>` are not instantiated:

- (2.1) — `T` is *Cpp17DefaultInsertable into X* means that the following expression is well-formed:

`allocator_traits<A>::construct(m, p)`

- (2.2) — An element of `X` is *default-inserted* if it is initialized by evaluation of the expression

`allocator_traits<A>::construct(m, p)`

where `p` is the address of the uninitialized storage for the element allocated within `X`.

- (2.3) — `T` is *Cpp17MoveInsertable into X* means that the following expression is well-formed:

`allocator_traits<A>::construct(m, p, rv)`

and its evaluation causes the following postcondition to hold: The value of `*p` is equivalent to the value of `rv` before the evaluation.

[*Note 1:* `rv` remains a valid object. Its state is unspecified. — end note]

- (2.4) — *T* is *Cpp17CopyInsertable* into *X* means that, in addition to *T* being *Cpp17MoveInsertable* into *X*, the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, v)
```

and its evaluation causes the following postcondition to hold: The value of *v* is unchanged and is equivalent to **p*.

- (2.5) — *T* is *Cpp17EmplaceConstructible* into *X* from *args*, for zero or more arguments *args*, means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, args)
```

- (2.6) — *T* is *Cpp17Erased* from *X* means that the following expression is well-formed:

```
allocator_traits<A>::destroy(m, p)
```

[Note 2: A container calls `allocator_traits<A>::construct(m, p, args)` to construct an element at *p* using *args*, with *m* == `get_allocator()`. The default `construct` in `allocator` will call `::new((void*)p) T(args)`, but specialized allocators can choose a different definition. — end note]

³ In this subclause,

- (3.1) — *X* denotes an allocator-aware container class with a `value_type` of *T* using an allocator of type *A*,
- (3.2) — *u* denotes a variable,
- (3.3) — *a* and *b* denote non-const lvalues of type *X*,
- (3.4) — *c* denotes an lvalue of type `const X`,
- (3.5) — *t* denotes an lvalue or a const rvalue of type *X*,
- (3.6) — *rv* denotes a non-const rvalue of type *X*, and
- (3.7) — *m* is a value of type *A*.

A type *X* meets the allocator-aware container requirements if *X* meets the container requirements and the following types, statements, and expressions are well-formed and have the specified semantics.

```
typename X::allocator_type
```

⁴ *Result:* *A*

⁵ *Mandates:* `allocator_type::value_type` is the same as `X::value_type`.

```
c.get_allocator()
```

⁶ *Result:* *A*

⁷ *Complexity:* Constant.

```
X u;
X u = X();
```

⁸ *Preconditions:* *A* meets the *Cpp17DefaultConstructible* requirements.

⁹ *Postconditions:* `u.empty()` returns `true`, `u.get_allocator() == A()`.

¹⁰ *Complexity:* Constant.

```
X u(m);
```

¹¹ *Postconditions:* `u.empty()` returns `true`, `u.get_allocator() == m`.

¹² *Complexity:* Constant.

```
X u(t, m);
```

¹³ *Preconditions:* *T* is *Cpp17CopyInsertable* into *X*.

¹⁴ *Postconditions:* `u == t`, `u.get_allocator() == m`.

¹⁵ *Complexity:* Linear.

```
X u(rv);
```

¹⁶ *Postconditions:* *u* has the same elements as *rv* had before this construction; the value of `u.get_allocator()` is the same as the value of `rv.get_allocator()` before this construction.

¹⁷ *Complexity:* Constant.

X u(rv, m);

18 *Preconditions:* T is *Cpp17MoveInsertable* into X.

19 *Postconditions:* u has the same elements, or copies of the elements, that rv had before this construction, u.get_allocator() == m.

20 *Complexity:* Constant if m == rv.get_allocator(), otherwise linear.

a = t

21 *Result:* X&.

22 *Preconditions:* T is *Cpp17CopyInsertable* into X and *Cpp17CopyAssignable*.

23 *Postconditions:* a == t is true.

24 *Complexity:* Linear.

a = rv

25 *Result:* X&.

26 *Preconditions:* If allocator_traits<allocator_type>::propagate_on_container_move_assignment::value is false, T is *Cpp17MoveInsertable* into X and *Cpp17MoveAssignable*.

27 *Effects:* All existing elements of a are either move assigned to or destroyed.

28 *Postconditions:* If a and rv do not refer to the same object, a is equal to the value that rv had before this assignment.

29 *Complexity:* Linear.

a.swap(b)

30 *Result:* void

31 *Effects:* Exchanges the contents of a and b.

32 *Complexity:* Constant.

23.2.3 Container data races

[container.requirements.dataraces]

- 1 For purposes of avoiding data races (16.4.6.10), implementations shall consider the following functions to be **const**: **begin**, **end**, **rbegin**, **rend**, **front**, **back**, **data**, **find**, **lower_bound**, **upper_bound**, **equal_range**, **at** and, except in associative or unordered associative containers, **operator[]**.
- 2 Notwithstanding 16.4.6.10, implementations are required to avoid data races when the contents of the contained object in different elements in the same container, excepting **vector<bool>**, are modified concurrently.
- 3 [Note 1: For a **vector<int>** x with a size greater than one, x[1] = 5 and *x.begin() = 10 can be executed concurrently without a data race, but x[0] = 5 and *x.begin() = 10 executed concurrently can result in a data race. As an exception to the general rule, for a **vector<bool>** y, y[0] = true can race with y[1] = true. — end note]

23.2.4 Sequence containers

[sequence.reqmts]

- 1 A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides the following basic kinds of sequence containers: **vector**, **inplace_vector**, **forward_list**, **list**, and **deque**. In addition, **array** and **hive** are provided as sequence containers which provide limited sequence operations, in **array**'s case because it has a fixed number of elements, and in **hive**'s case because insertion order is unspecified. The library also provides container adaptors that make it easy to construct abstract data types, such as **stacks**, **queues**, **flat_maps**, **flat_multimaps**, **flat_sets**, or **flat_multisets**, out of the basic sequence container kinds (or out of other program-defined sequence containers).
- 2 In this subclause,
 - (2.1) — X denotes a sequence container class,
 - (2.2) — a denotes a value of type X containing elements of type T,
 - (2.3) — u denotes the name of a variable being declared,
 - (2.4) — A denotes X::allocator_type if the *qualified-id* X::allocator_type is valid and denotes a type (13.10.3) and allocator<T> if it doesn't,

- (2.5) — *i* and *j* denote iterators that meet the *Cpp17InputIterator* requirements and refer to elements implicitly convertible to `value_type`,
- (2.6) — [*i*, *j*) denotes a valid range,
- (2.7) — *rg* denotes a value of a type *R* that models *container-compatible-range*<*T*>,
- (2.8) — *il* designates an object of type `initializer_list<value_type>`,
- (2.9) — *n* denotes a value of type `X::size_type`,
- (2.10) — *p* denotes a valid constant iterator to *a*,
- (2.11) — *q* denotes a valid dereferenceable constant iterator to *a*,
- (2.12) — [*q1*, *q2*) denotes a valid range of constant iterators in *a*,
- (2.13) — *t* denotes an lvalue or a const rvalue of `X::value_type`, and
- (2.14) — *rv* denotes a non-const rvalue of `X::value_type`.
- (2.15) — *Args* denotes a template parameter pack;
- (2.16) — *args* denotes a function parameter pack with the pattern *Args*&&.

3 The complexities of the expressions are sequence dependent.

4 A type *X* meets the *sequence container* requirements if *X* meets the container requirements and the following statements and expressions are well-formed and have the specified semantics.

`X u(n, t);`

5 *Preconditions:* *T* is *Cpp17CopyInsertable* into *X*.

6 *Effects:* Constructs a sequence container with *n* copies of *t*.

7 *Postconditions:* `distance(u.begin(), u.end()) == n` is true.

`X u(i, j);`

8 *Preconditions:* *T* is *Cpp17EmplaceConstructible* into *X* from **i*. For *vector*, if the iterator does not meet the *Cpp17ForwardIterator* requirements (24.3.5.5), *T* is also *Cpp17MoveInsertable* into *X*.

9 *Effects:* Constructs a sequence container equal to the range [*i*, *j*). Each iterator in the range [*i*, *j*) is dereferenced exactly once.

10 *Postconditions:* `distance(u.begin(), u.end()) == distance(i, j)` is true.

`X(from_range, rg)`

11 *Preconditions:* *T* is *Cpp17EmplaceConstructible* into *X* from `*ranges::begin(rg)`. For *vector*, if *R* models `ranges::approximately_sized_range` but not `ranges::sized_range` or models `ranges::input_range` but not `ranges::forward_range`, *T* is also *Cpp17MoveInsertable* into *X*.

12 *Effects:* Constructs a sequence container equal to the range *rg*. Each iterator in the range *rg* is dereferenced exactly once.

13 *Recommended practice:* If *R* models `ranges::approximately_sized_range` and `ranges::distance(rg) <= ranges::reserve_hint(rg)` is true, an implementation should not perform more than a single reallocation.

14 *Postconditions:* `distance(begin(), end()) == ranges::distance(rg)` is true.

`X(il)`

15 *Effects:* Equivalent to `X(il.begin(), il.end())`.

`a = il`

16 *Result:* *X*&.

17 *Preconditions:* *T* is *Cpp17CopyInsertable* into *X* and *Cpp17CopyAssignable*.

18 *Effects:* Assigns the range [*il*.begin(), *il*.end()) into *a*. All existing elements of *a* are either assigned to or destroyed.

19 *Returns:* **this*.

`a.emplace(p, args)`

20 *Result:* iterator.

21 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from args. For `vector`, `inplace_vector`, and `deque`, T is also *Cpp17MoveInsertable* into X and *Cpp17MoveAssignable*.

22 *Effects:* Inserts an object of type T constructed with `std::forward<Args>(args)...` before p.

[*Note 1:* args can directly or indirectly refer to a value in a. — end note]

23 *Returns:* An iterator that points to the new element.

`a.insert(p, t)`

24 *Result:* iterator.

25 *Preconditions:* T is *Cpp17CopyInsertable* into X. For `vector`, `inplace_vector`, and `deque`, T is also *Cpp17CopyAssignable*.

26 *Effects:* Inserts a copy of t before p.

27 *Returns:* An iterator that points to the copy of t inserted into a.

`a.insert(p, rv)`

28 *Result:* iterator.

29 *Preconditions:* T is *Cpp17MoveInsertable* into X. For `vector`, `inplace_vector`, and `deque`, T is also *Cpp17MoveAssignable*.

30 *Effects:* Inserts a copy of rv before p.

31 *Returns:* An iterator that points to the copy of rv inserted into a.

`a.insert(p, n, t)`

32 *Result:* iterator.

33 *Preconditions:* T is *Cpp17CopyInsertable* into X and *Cpp17CopyAssignable*.

34 *Effects:* Inserts n copies of t before p.

35 *Returns:* An iterator that points to the copy of the first element inserted into a, or p if n == 0.

`a.insert(p, i, j)`

36 *Result:* iterator.

37 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from *i. For `vector`, `inplace_vector`, and `deque`, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3) requirements. Neither i nor j are iterators into a.

38 *Effects:* Inserts copies of elements in [i, j) before p. Each iterator in the range [i, j) shall be dereferenced exactly once.

39 *Returns:* An iterator that points to the copy of the first element inserted into a, or p if i == j.

`a.insert_range(p, rg)`

40 *Result:* iterator.

41 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from *ranges::begin(rg). For `vector`, `inplace_vector`, and `deque`, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3) requirements. rg and a do not overlap.

42 *Effects:* Inserts copies of elements in rg before p. Each iterator in the range rg is dereferenced exactly once.

43 *Returns:* An iterator that points to the copy of the first element inserted into a, or p if rg is empty.

`a.insert(p, il)`

44 *Effects:* Equivalent to `a.insert(p, il.begin(), il.end())`.

a.erase(q)

45 *Result:* iterator.

46 *Preconditions:* For `vector`, `inplace_vector`, and `deque`, `T` is *Cpp17MoveAssignable*.

47 *Effects:* Erases the element pointed to by `q`.

48 *Returns:* An iterator that points to the element immediately following `q` prior to the element being erased. If no such element exists, `a.end()` is returned.

a.erase(q1, q2)

49 *Result:* iterator.

50 *Preconditions:* For `vector`, `inplace_vector`, and `deque`, `T` is *Cpp17MoveAssignable*.

51 *Effects:* Erases the elements in the range `[q1, q2)`.

52 *Returns:* An iterator that points to the element pointed to by `q2` prior to any elements being erased. If no such element exists, `a.end()` is returned.

a.clear()

53 *Result:* void

54 *Effects:* Destroys all elements in `a`. Invalidates all references, pointers, and iterators referring to the elements of `a` and may invalidate the past-the-end iterator.

55 *Postconditions:* `a.empty()` is true.

56 *Complexity:* Linear.

a.assign(i, j)

57 *Result:* void

58 *Preconditions:* `T` is *Cpp17EmplaceConstructible* into `X` from `*i` and assignable from `*i`. For `vector`, if the iterator does not meet the forward iterator requirements (24.3.5.5), `T` is also *Cpp17MoveInsertable* into `X`. Neither `i` nor `j` are iterators into `a`.

59 *Effects:* Replaces elements in `a` with a copy of `[i, j)`. Invalidates all references, pointers and iterators referring to the elements of `a`. For `vector` and `deque`, also invalidates the past-the-end iterator. Each iterator in the range `[i, j)` is dereferenced exactly once.

a.assign_range(rg)

60 *Result:* void

61 *Mandates:* `assignable_from<T&, ranges::range_reference_t<R>>` is modeled. For `inplace_vector`, if `ranges::size(rg)` is a constant expression, then `ranges::size(rg) ≤ a.max_size()`.

62 *Preconditions:* `T` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`. For `vector`, if `R` models `ranges::approximately_sized_range` but not `ranges::sized_range` or models `ranges::input_range` but not `ranges::forward_range`, `T` is also *Cpp17MoveInsertable* into `X`. `rg` and `a` do not overlap.

63 *Effects:* Replaces elements in `a` with a copy of each element in `rg`. Invalidates all references, pointers, and iterators referring to the elements of `a`. For `vector` and `deque`, also invalidates the past-the-end iterator. Each iterator in the range `rg` is dereferenced exactly once.

64 *Recommended practice:* If `R` models `ranges::approximately_sized_range` and `ranges::distance(rg) ≤ ranges::reserve_hint(rg)` is true, an implementation should not perform any reallocation.

a.assign(il)

65 *Effects:* Equivalent to `a.assign(il.begin(), il.end())`.

a.assign(n, t)

66 *Result:* void

67 *Preconditions:* `T` is *Cpp17CopyInsertable* into `X` and *Cpp17CopyAssignable*. `t` is not a reference into `a`.

68 *Effects:* Replaces elements in `a` with `n` copies of `t`. Invalidates all references, pointers and iterators referring to the elements of `a`. For `vector` and `deque`, also invalidates the past-the-end iterator.

69 For every sequence container defined in this Clause and in [Clause 27](#):

(69.1) — If the constructor

```
template<class InputIterator>
  X(InputIterator first, InputIterator last,
    const allocator_type& alloc = allocator_type());
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the constructor shall not participate in overload resolution.

(69.2) — If the member functions of the forms:

```
template<class InputIterator>
  return-type F(const_iterator p,
                InputIterator first, InputIterator last);    // such as insert
```

```
template<class InputIterator>
  return-type F(InputIterator first, InputIterator last);    // such as append, assign
```

```
template<class InputIterator>
  return-type F(const_iterator i1, const_iterator i2,
                InputIterator first, InputIterator last);    // such as replace
```

are called with a type `InputIterator` that does not qualify as an input iterator, then these functions shall not participate in overload resolution.

(69.3) — A deduction guide for a sequence container shall not participate in overload resolution if it has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter, or if it has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.

70 The following operations are provided for some types of sequence containers but not others. Operations other than `prepend_range` and `append_range` are implemented so as to take amortized constant time.

`a.front()`

71 *Result:* `const_reference` if `a` is of type `const X`; `reference` otherwise.

72 *Hardened preconditions:* `a.empty()` is false.

73 *Returns:* `*a.begin()`

74 *Remarks:* Required for `basic_string`, `array`, `deque`, `forward_list`, `inplace_vector`, `list`, and `vector`.

`a.back()`

75 *Result:* `const_reference` if `a` is of type `const X`; `reference` otherwise.

76 *Hardened preconditions:* `a.empty()` is false.

77 *Effects:* Equivalent to:

```
auto tmp = a.end();
--tmp;
return *tmp;
```

78 *Remarks:* Required for `basic_string`, `array`, `deque`, `inplace_vector`, `list`, and `vector`.

`a.emplace_front(args)`

79 *Result:* `reference`

80 *Preconditions:* `T` is `Cpp17EmplaceConstructible` into `X` from `args`.

81 *Effects:* Prepends an object of type `T` constructed with `std::forward<Args>(args)...`

82 *Returns:* `a.front()`.

83 *Remarks:* Required for `deque`, `forward_list`, and `list`.

`a.emplace_back(args)`

84 *Result:* `reference`

85 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from `args`. For `vector`, T is also *Cpp17MoveInsertable* into X.

86 *Effects:* Appends an object of type T constructed with `std::forward<Args>(args)...`

87 *Returns:* `a.back()`.

88 *Remarks:* Required for `deque`, `inplace_vector`, `list`, and `vector`.

a.push_front(t)

89 *Result:* `void`

90 *Preconditions:* T is *Cpp17CopyInsertable* into X.

91 *Effects:* Prepends a copy of `t`.

92 *Remarks:* Required for `deque`, `forward_list`, and `list`.

a.push_front(rv)

93 *Result:* `void`

94 *Preconditions:* T is *Cpp17MoveInsertable* into X.

95 *Effects:* Prepends a copy of `rv`.

96 *Remarks:* Required for `deque`, `forward_list`, and `list`.

a.prepend_range(rg)

97 *Result:* `void`

98 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from `*ranges::begin(rg)`. For `deque`, T is also *Cpp17MoveInsertable* into X, and T meets the *Cpp17MoveConstructible*, *Cpp17MoveAssignable*, and *Cpp17Swappable* (16.4.4.3) requirements.

99 *Effects:* Inserts copies of elements in `rg` before `begin()`. Each iterator in the range `rg` is dereferenced exactly once.

 [Note 2: The order of elements in `rg` is not reversed. — end note]

100 *Remarks:* Required for `deque`, `forward_list`, and `list`.

a.push_back(t)

101 *Result:* `void`

102 *Preconditions:* T is *Cpp17CopyInsertable* into X.

103 *Effects:* Appends a copy of `t`.

104 *Remarks:* Required for `basic_string`, `deque`, `inplace_vector`, `list`, and `vector`.

a.push_back(rv)

105 *Result:* `void`

106 *Preconditions:* T is *Cpp17MoveInsertable* into X.

107 *Effects:* Appends a copy of `rv`.

108 *Remarks:* Required for `basic_string`, `deque`, `inplace_vector`, `list`, and `vector`.

a.append_range(rg)

109 *Result:* `void`

110 *Preconditions:* T is *Cpp17EmplaceConstructible* into X from `*ranges::begin(rg)`. For `vector`, T is also *Cpp17MoveInsertable* into X.

111 *Effects:* Inserts copies of elements in `rg` before `end()`. Each iterator in the range `rg` is dereferenced exactly once.

112 *Remarks:* Required for `deque`, `inplace_vector`, `list`, and `vector`.

a.pop_front()

113 *Result:* `void`

114 *Hardened preconditions:* `a.empty()` is false.

115 *Effects:* Destroys the first element.

116 *Remarks:* Required for `deque`, `forward_list`, and `list`.

`a.pop_back()`

117 *Result:* `void`

118 *Hardened preconditions:* `a.empty()` is false.

119 *Effects:* Destroys the last element.

120 *Remarks:* Required for `basic_string`, `deque`, `inplace_vector`, `list`, and `vector`.

`a[n]`

121 *Result:* `const_reference` if `a` is of type `const X`; reference otherwise.

122 *Hardened preconditions:* `n < a.size()` is true.

123 *Effects:* Equivalent to: `return *(a.begin() + n);`

124 *Remarks:* Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

`a.at(n)`

125 *Result:* `const_reference` if `a` is of type `const X`; reference otherwise.

126 *Returns:* `*(a.begin() + n)`

127 *Throws:* `out_of_range` if `n >= a.size()`.

128 *Remarks:* Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

23.2.5 Node handles

[container.node]

23.2.5.1 Overview

[container.node.overview]

- ¹ A *node handle* is an object that accepts ownership of a single element from an associative container (23.2.7) or an unordered associative container (23.2.8). It may be used to transfer that ownership to another container with compatible nodes. Containers with compatible nodes have the same node handle type. Elements may be transferred in either direction between container types in the same row of Table 75.

Table 75 — Container types with compatible nodes [tab:container.node.compat]

<code>map<K, T, C1, A></code>	<code>map<K, T, C2, A></code>
<code>map<K, T, C1, A></code>	<code>multimap<K, T, C2, A></code>
<code>set<K, C1, A></code>	<code>set<K, C2, A></code>
<code>set<K, C1, A></code>	<code>multiset<K, C2, A></code>
<code>unordered_map<K, T, H1, E1, A></code>	<code>unordered_map<K, T, H2, E2, A></code>
<code>unordered_map<K, T, H1, E1, A></code>	<code>unordered_multimap<K, T, H2, E2, A></code>
<code>unordered_set<K, H1, E1, A></code>	<code>unordered_set<K, H2, E2, A></code>
<code>unordered_set<K, H1, E1, A></code>	<code>unordered_multiset<K, H2, E2, A></code>

- ² If a node handle is not empty, then it contains an allocator that is equal to the allocator of the container when the element was extracted. If a node handle is empty, it contains no allocator.
- ³ Class *node-handle* is for exposition only.
- ⁴ If a user-defined specialization of `pair` exists for `pair<const Key, T>` or `pair<Key, T>`, where `Key` is the container's `key_type` and `T` is the container's `mapped_type`, the behavior of operations involving node handles is undefined.

```
template<unspecified>
class node-handle {
public:
    // These type declarations are described in 23.2.7 and 23.2.8.
    using value_type    = see below;    // not present for map containers
    using key_type      = see below;    // not present for set containers
    using mapped_type   = see below;    // not present for set containers
```

```

using allocator_type = see below;

private:
    using container-node-type = unspecified;           // exposition only
    using ator-traits = allocator_traits<allocator_type>; // exposition only

    typename ator-traits::template
        rebind_traits<container-node-type>::pointer ptr_; // exposition only
    optional<allocator_type> alloc_; // exposition only

public:
    // 23.2.5.2, constructors, copy, and assignment
    constexpr node-handle() noexcept : ptr_(), alloc_() {}
    constexpr node-handle(node-handle&&) noexcept;
    constexpr node-handle& operator=(node-handle&&);

    // 23.2.5.3, destructor
    constexpr ~node-handle();

    // 23.2.5.4, observers
    constexpr value_type& value() const;           // not present for map containers
    key_type& key() const; // not present for set containers
    constexpr mapped_type& mapped() const; // not present for set containers

    constexpr allocator_type get_allocator() const;
    constexpr explicit operator bool() const noexcept;
    constexpr bool empty() const noexcept;

    // 23.2.5.5, modifiers
    constexpr void swap(node-handle&)
        noexcept(ator-traits::propagate_on_container_swap::value ||
            ator-traits::is_always_equal::value);

    friend constexpr void swap(node-handle& x, node-handle& y) noexcept(noexcept(x.swap(y))) {
        x.swap(y);
    }
};

```

23.2.5.2 Constructors, copy, and assignment

[container.node.cons]

```
constexpr node-handle(node-handle&& nh) noexcept;
```

- 1 *Effects:* Constructs a *node-handle* object initializing *ptr_* with *nh.ptr_*. Move constructs *alloc_* with *nh.alloc_*. Assigns nullptr to *nh.ptr_* and assigns nullopt to *nh.alloc_*.

```
constexpr node-handle& operator=(node-handle&& nh);
```

- 2 *Preconditions:* Either *!alloc_* is true, or *ator-traits::propagate_on_container_move_assignment::value* is true, or *alloc_ == nh.alloc_* is true.

- 3 *Effects:*

- (3.1) — If *ptr_ != nullptr* is true, destroys the *value_type* subobject in the *container-node-type* object pointed to by *ptr_* by calling *ator-traits::destroy*, then deallocates *ptr_* by calling *ator-traits::template rebind_traits<container-node-type>::deallocate*.
- (3.2) — Assigns *nh.ptr_* to *ptr_*.
- (3.3) — If *!alloc_* is true or *ator-traits::propagate_on_container_move_assignment::value* is true, move assigns *nh.alloc_* to *alloc_*.
- (3.4) — Assigns nullptr to *nh.ptr_* and assigns nullopt to *nh.alloc_*.

- 4 *Returns:* **this*.

- 5 *Throws:* Nothing.

23.2.5.3 Destructor

[container.node.dtor]

constexpr ~node-handle();

- 1 *Effects:* If *ptr_* != nullptr is true, destroys the *value_type* subobject in the *container-node-type* object pointed to by *ptr_* by calling *ator-traits::destroy*, then deallocates *ptr_* by calling *ator-traits::template rebind_traits<container-node-type>::deallocate*.

23.2.5.4 Observers

[container.node.observers]

constexpr value_type& value() const;

- 1 *Preconditions:* *empty()* == false.
- 2 *Returns:* A reference to the *value_type* subobject in the *container-node-type* object pointed to by *ptr_*.
- 3 *Throws:* Nothing.

key_type& key() const;

- 4 *Preconditions:* *empty()* == false.
- 5 *Returns:* A non-const reference to the *key_type* member of the *value_type* subobject in the *container-node-type* object pointed to by *ptr_*.
- 6 *Throws:* Nothing.
- 7 *Remarks:* Modifying the key through the returned reference is permitted.

constexpr mapped_type& mapped() const;

- 8 *Preconditions:* *empty()* == false.
- 9 *Returns:* A reference to the *mapped_type* member of the *value_type* subobject in the *container-node-type* object pointed to by *ptr_*.
- 10 *Throws:* Nothing.

constexpr allocator_type get_allocator() const;

- 11 *Preconditions:* *empty()* == false.
- 12 *Returns:* **alloc_*.
- 13 *Throws:* Nothing.

constexpr explicit operator bool() const noexcept;

- 14 *Returns:* *ptr_* != nullptr.

constexpr bool empty() const noexcept;

- 15 *Returns:* *ptr_* == nullptr.

23.2.5.5 Modifiers

[container.node.modifiers]

```
constexpr void swap(node-handle& nh)
    noexcept(ator-traits::propagate_on_container_swap::value ||
             ator-traits::is_always_equal::value);
```

- 1 *Preconditions:* *!alloc_* is true, or *!nh.alloc_*, or *ator-traits::propagate_on_container_swap::value* is true, or *alloc_* == *nh.alloc_* is true.
- 2 *Effects:* Calls *swap(ptr_, nh.ptr_)*. If *!alloc_* is true, or *!nh.alloc_* is true, or *ator-traits::propagate_on_container_swap::value* is true calls *swap(alloc_, nh.alloc_)*.

23.2.6 Insert return type

[container.insert.return]

- 1 The associative containers with unique keys and the unordered containers with unique keys have a member function *insert* that returns a nested type *insert_return_type*. That return type is a specialization of the template specified in this subclause.

```
template<class Iterator, class NodeType>
struct insert-return-type
{
    Iterator position;
    bool      inserted;
    NodeType node;
};
```

- ² The name *insert-return-type* is for exposition only. *insert-return-type* has the template parameters, data members, and special members specified above. It has no base classes or members other than those specified.

23.2.7 Associative containers

[associative.reqmts]

23.2.7.1 General

[associative.reqmts.general]

- ¹ Associative containers provide fast retrieval of data based on keys. The library provides four basic kinds of associative containers: `set`, `multiset`, `map` and `multimap`. The library also provides container adaptors that make it easy to construct abstract data types, such as `flat_maps`, `flat_multimaps`, `flat_sets`, or `flat_multisets`, out of the basic sequence container kinds (or out of other program-defined sequence containers).
- ² Each associative container is parameterized on `Key` and an ordering relation `Compare` that induces a strict weak ordering (26.8) on elements of `Key`. In addition, `map` and `multimap` associate an arbitrary *mapped type* `T` with the `Key`. The object of type `Compare` is called the *comparison object* of a container.
- ³ The phrase “equivalence of keys” means the equivalence relation imposed by the comparison object. That is, two keys `k1` and `k2` are considered to be equivalent if for the comparison object `comp`, `comp(k1, k2) == false && comp(k2, k1) == false`.

[Note 1: This is not necessarily the same as the result of `k1 == k2`. — end note]

For any two keys `k1` and `k2` in the same container, calling `comp(k1, k2)` shall always return the same value.

- ⁴ An associative container supports *unique keys* if it may contain at most one element for each key. Otherwise, it supports *equivalent keys*. The `set` and `map` classes support unique keys; the `multiset` and `multimap` classes support equivalent keys. For `multiset` and `multimap`, `insert`, `emplace`, and `erase` preserve the relative ordering of equivalent elements.
- ⁵ For `set` and `multiset` the value type is the same as the key type. For `map` and `multimap` it is equal to `pair<const Key, T>`.
- ⁶ `iterator` of an associative container is of the bidirectional iterator category. For associative containers where the value type is the same as the key type, both `iterator` and `const_iterator` are constant iterators. It is unspecified whether or not `iterator` and `const_iterator` are the same type.

[Note 2: `iterator` and `const_iterator` have identical semantics in this case, and `iterator` is convertible to `const_iterator`. Users can avoid violating the one-definition rule by always using `const_iterator` in their function parameter lists. — end note]

- ⁷ In this subclause,

- (7.1) — `X` denotes an associative container class,
- (7.2) — `a` denotes a value of type `X`,
- (7.3) — `a2` denotes a value of a type with nodes compatible with type `X` (Table 75),
- (7.4) — `b` denotes a value of type `X` or `const X`,
- (7.5) — `u` denotes the name of a variable being declared,
- (7.6) — `a_uniq` denotes a value of type `X` when `X` supports unique keys,
- (7.7) — `a_eq` denotes a value of type `X` when `X` supports equivalent keys,
- (7.8) — `a_tran` denotes a value of type `X` or `const X` when the *qualified-id* `X::key_compare::is_transparent` is valid and denotes a type (13.10.3),
- (7.9) — `i` and `j` meet the *Cpp17InputIterator* requirements and refer to elements implicitly convertible to `value_type`,
- (7.10) — `[i, j)` denotes a valid range,

- (7.11) — **rg** denotes a value of a type **R** that models *container-compatible-range*<value_type>,
- (7.12) — **p** denotes a valid constant iterator to **a**,
- (7.13) — **q** denotes a valid dereferenceable constant iterator to **a**,
- (7.14) — **r** denotes a valid dereferenceable iterator to **a**,
- (7.15) — [**q1**, **q2**) denotes a valid range of constant iterators in **a**,
- (7.16) — **il** designates an object of type `initializer_list<value_type>`,
- (7.17) — **t** denotes a value of type `X::value_type`,
- (7.18) — **k** denotes a value of type `X::key_type`, and
- (7.19) — **c** denotes a value of type `X::key_compare` or `const X::key_compare`;
- (7.20) — **k1** is a value such that **a** is partitioned (26.8) with respect to `c(x, k1)`, with **x** the key value of **e** and **e** in **a**;
- (7.21) — **ku** is a value such that **a** is partitioned with respect to `!c(ku, x)`, with **x** the key value of **e** and **e** in **a**;
- (7.22) — **ke** is a value such that **a** is partitioned with respect to `c(x, ke)` and `!c(ke, x)`, with `c(x, ke)` implying `!c(ke, x)` and with **x** the key value of **e** and **e** in **a**;
- (7.23) — **kx** is a value such that
 - (7.23.1) — **a** is partitioned with respect to `c(x, kx)` and `!c(kx, x)`, with `c(x, kx)` implying `!c(kx, x)` and with **x** the key value of **e** and **e** in **a**, and
 - (7.23.2) — **kx** is not convertible to either `iterator` or `const_iterator`; and
- (7.24) — **A** denotes the storage allocator used by **X**, if any, or `allocator<X::value_type>` otherwise,
- (7.25) — **m** denotes an allocator of a type convertible to **A**, and **nh** denotes a non-const rvalue of type `X::node_type`.

⁸ A type **X** meets the *associative container* requirements if **X** meets all the requirements of an allocator-aware container (23.2.2.5) and the following types, statements, and expressions are well-formed and have the specified semantics, except that for `map` and `multimap`, the requirements placed on `value_type` in 23.2.2.2 apply instead to `key_type` and `mapped_type`.

[Note 3: For example, in some cases `key_type` and `mapped_type` need to be *Cpp17CopyAssignable* even though the associated `value_type`, `pair<const key_type, mapped_type>`, is not *Cpp17CopyAssignable*. — end note]

`typename X::key_type`

⁹ *Result:* Key.

`typename X::mapped_type`

¹⁰ *Result:* T.

¹¹ *Remarks:* For `map` and `multimap` only.

`typename X::value_type`

¹² *Result:* Key for `set` and `multiset` only; `pair<const Key, T>` for `map` and `multimap` only.

¹³ *Preconditions:* `X::value_type` is *Cpp17Erasable* from **X**.

`typename X::key_compare`

¹⁴ *Result:* Compare.

¹⁵ *Preconditions:* `key_compare` is *Cpp17CopyConstructible*.

`typename X::value_compare`

¹⁶ *Result:* A binary predicate type. It is the same as `key_compare` for `set` and `multiset`; is an ordering relation on pairs induced by the first component (i.e., Key) for `map` and `multimap`.

`typename X::node_type`

¹⁷ *Result:* A specialization of the *node-handle* class template (23.2.5), such that the public nested types are the same types as the corresponding types in **X**.

X(c)

18 *Effects:* Constructs an empty container. Uses a copy of **c** as a comparison object.

19 *Complexity:* Constant.

X u = X();

X u;

20 *Preconditions:* **key_compare** meets the *Cpp17DefaultConstructible* requirements.

21 *Effects:* Constructs an empty container. Uses **Compare()** as a comparison object.

22 *Complexity:* Constant.

X(i, j, c)

23 *Preconditions:* **value_type** is *Cpp17EmplaceConstructible* into **X** from ***i**.

24 *Effects:* Constructs an empty container and inserts elements from the range **[i, j)** into it; uses **c** as a comparison object.

25 *Complexity:* $N \log N$ in general, where N has the value **distance(i, j)**; linear if **[i, j)** is sorted with respect to **value_comp()**.

X(i, j)

26 *Preconditions:* **key_compare** meets the *Cpp17DefaultConstructible* requirements. **value_type** is *Cpp17EmplaceConstructible* into **X** from ***i**.

27 *Effects:* Constructs an empty container and inserts elements from the range **[i, j)** into it; uses **Compare()** as a comparison object.

28 *Complexity:* $N \log N$ in general, where N has the value **distance(i, j)**; linear if **[i, j)** is sorted with respect to **value_comp()**.

X(from_range, rg, c)

29 *Preconditions:* **value_type** is *Cpp17EmplaceConstructible* into **X** from ***ranges::begin(rg)**.

30 *Effects:* Constructs an empty container and inserts each element from **rg** into it. Uses **c** as the comparison object.

31 *Complexity:* $N \log N$ in general, where N has the value **ranges::distance(rg)**; linear if **rg** is sorted with respect to **value_comp()**.

X(from_range, rg)

32 *Preconditions:* **key_compare** meets the *Cpp17DefaultConstructible* requirements. **value_type** is *Cpp17EmplaceConstructible* into **X** from ***ranges::begin(rg)**.

33 *Effects:* Constructs an empty container and inserts each element from **rg** into it. Uses **Compare()** as the comparison object.

34 *Complexity:* Same as **X(from_range, rg, c)**.

X(il, c)

35 *Effects:* Equivalent to **X(il.begin(), il.end(), c)**.

X(il)

36 *Effects:* Equivalent to **X(il.begin(), il.end())**.

a = il

37 *Result:* **X&**

38 *Preconditions:* **value_type** is *Cpp17CopyInsertable* into **X** and *Cpp17CopyAssignable*.

39 *Effects:* Assigns the range **[il.begin(), il.end())** into **a**. All existing elements of **a** are either assigned to or destroyed.

40 *Complexity:* $N \log N$ in general, where N has the value **il.size() + a.size()**; linear if **[il.begin(), il.end())** is sorted with respect to **value_comp()**.

b.key_comp()

41 *Result:* `X::key_compare`

42 *Returns:* The comparison object out of which **b** was constructed.

43 *Complexity:* Constant.

b.value_comp()

44 *Result:* `X::value_compare`

45 *Returns:* An object of `value_compare` constructed out of the comparison object.

46 *Complexity:* Constant.

a_uniq.emplace(args)

47 *Result:* `pair<iterator, bool>`

48 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into **X** from `args`.

49 *Effects:* Inserts a `value_type` object **t** constructed with `std::forward<Args>(args)...` if and only if there is no element in the container with key equivalent to the key of **t**.

50 *Returns:* The `bool` component of the returned pair is `true` if and only if the insertion takes place, and the iterator component of the pair points to the element with key equivalent to the key of **t**.

51 *Complexity:* Logarithmic.

a_eq.emplace(args)

52 *Result:* `iterator`

53 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into **X** from `args`.

54 *Effects:* Inserts a `value_type` object **t** constructed with `std::forward<Args>(args)...`. If a range containing elements equivalent to **t** exists in **a_eq**, **t** is inserted at the end of that range.

55 *Returns:* An iterator pointing to the newly inserted element.

56 *Complexity:* Logarithmic.

a.emplace_hint(p, args)

57 *Result:* `iterator`

58 *Effects:* Equivalent to `a.emplace(std::forward<Args>(args)...)...`, except that the element is inserted as close as possible to the position just prior to **p**.

59 *Returns:* The iterator returned by `emplace`.

60 *Complexity:* Logarithmic in general, but amortized constant if the element is inserted right before **p**.

a_uniq.insert(t)

61 *Result:* `pair<iterator, bool>`

62 *Preconditions:* If **t** is a non-const rvalue, `value_type` is *Cpp17MoveInsertable* into **X**; otherwise, `value_type` is *Cpp17CopyInsertable* into **X**.

63 *Effects:* Inserts **t** if and only if there is no element in the container with key equivalent to the key of **t**.

64 *Returns:* The `bool` component of the returned pair is `true` if and only if the insertion takes place, and the iterator component of the pair points to the element with key equivalent to the key of **t**.

65 *Complexity:* Logarithmic.

a_eq.insert(t)

66 *Result:* `iterator`

67 *Preconditions:* If **t** is a non-const rvalue, `value_type` is *Cpp17MoveInsertable* into **X**; otherwise, `value_type` is *Cpp17CopyInsertable* into **X**.

68 *Effects:* Inserts **t** and returns the iterator pointing to the newly inserted element. If a range containing elements equivalent to **t** exists in **a_eq**, **t** is inserted at the end of that range.

69 *Complexity:* Logarithmic.

a.insert(p, t)

70 *Result:* iterator

71 *Preconditions:* If **t** is a non-const rvalue, **value_type** is *Cpp17MoveInsertable* into **X**; otherwise, **value_type** is *Cpp17CopyInsertable* into **X**.

72 *Effects:* Inserts **t** if and only if there is no element with key equivalent to the key of **t** in containers with unique keys; always inserts **t** in containers with equivalent keys. **t** is inserted as close as possible to the position just prior to **p**.

73 *Returns:* An iterator pointing to the element with key equivalent to the key of **t**.

74 *Complexity:* Logarithmic in general, but amortized constant if **t** is inserted right before **p**.

a.insert(i, j)

75 *Result:* void

76 *Preconditions:* **value_type** is *Cpp17EmplaceConstructible* into **X** from ***i**. Neither **i** nor **j** are iterators into **a**.

77 *Effects:* Inserts each element from the range **[i, j)** if and only if there is no element with key equivalent to the key of that element in containers with unique keys; always inserts that element in containers with equivalent keys.

78 *Complexity:* $N \log(a.size() + N)$, where N has the value **distance(i, j)**.

a.insert_range(rg)

79 *Result:* void

80 *Preconditions:* **value_type** is *Cpp17EmplaceConstructible* into **X** from ***ranges::begin(rg)**. **rg** and **a** do not overlap.

81 *Effects:* Inserts each element from **rg** if and only if there is no element with key equivalent to the key of that element in containers with unique keys; always inserts that element in containers with equivalent keys.

82 *Complexity:* $N \log(a.size() + N)$, where N has the value **ranges::distance(rg)**.

a.insert(il)

83 *Effects:* Equivalent to **a.insert(il.begin(), il.end())**.

a_uniq.insert(nh)

84 *Result:* insert_return_type

85 *Preconditions:* **nh** is empty or **a_uniq.get_allocator() == nh.get_allocator()** is true.

86 *Effects:* If **nh** is empty, has no effect. Otherwise, inserts the element owned by **nh** if and only if there is no element in the container with a key equivalent to **nh.key()**.

87 *Returns:* If **nh** is empty, **inserted** is false, **position** is **end()**, and **node** is empty. Otherwise if the insertion took place, **inserted** is true, **position** points to the inserted element, and **node** is empty; if the insertion failed, **inserted** is false, **node** has the previous value of **nh**, and **position** points to an element with a key equivalent to **nh.key()**.

88 *Complexity:* Logarithmic.

a_eq.insert(nh)

89 *Result:* iterator

90 *Preconditions:* **nh** is empty or **a_eq.get_allocator() == nh.get_allocator()** is true.

91 *Effects:* If **nh** is empty, has no effect and returns **a_eq.end()**. Otherwise, inserts the element owned by **nh** and returns an iterator pointing to the newly inserted element. If a range containing elements with keys equivalent to **nh.key()** exists in **a_eq**, the element is inserted at the end of that range.

92 *Postconditions:* **nh** is empty.

93 *Complexity:* Logarithmic.

a.insert(p, nh)

94 *Result:* iterator

95 *Preconditions:* nh is empty or a.get_allocator() == nh.get_allocator() is true.

96 *Effects:* If nh is empty, has no effect and returns a.end(). Otherwise, inserts the element owned by nh if and only if there is no element with key equivalent to nh.key() in containers with unique keys; always inserts the element owned by nh in containers with equivalent keys. The element is inserted as close as possible to the position just prior to p.

97 *Postconditions:* nh is empty if insertion succeeds, unchanged if insertion fails.

98 *Returns:* An iterator pointing to the element with key equivalent to nh.key().

99 *Complexity:* Logarithmic in general, but amortized constant if the element is inserted right before p.

a.extract(k)

100 *Result:* node_type

101 *Effects:* Removes the first element in the container with key equivalent to k.

102 *Returns:* A node_type owning the element if found, otherwise an empty node_type.

103 *Complexity:* log(a.size())

a_tran.extract(kx)

104 *Result:* node_type

105 *Effects:* Removes the first element in the container with key r such that !c(r, kx) && !c(kx, r) is true.

106 *Returns:* A node_type owning the element if found, otherwise an empty node_type.

107 *Complexity:* log(a_tran.size())

a.extract(q)

108 *Result:* node_type

109 *Effects:* Removes the element pointed to by q.

110 *Returns:* A node_type owning that element.

111 *Complexity:* Amortized constant.

a.merge(a2)

112 *Result:* void

113 *Preconditions:* a.get_allocator() == a2.get_allocator() is true.

114 *Effects:* Attempts to extract each element in a2 and insert it into a using the comparison object of a. In containers with unique keys, if there is an element in a with key equivalent to the key of an element from a2, then that element is not extracted from a2.

115 *Postconditions:* Pointers and references to the transferred elements of a2 refer to those same elements but as members of a. If a.begin() and a2.begin() have the same type, iterators referring to the transferred elements will continue to refer to their elements, but they now behave as iterators into a, not into a2.

116 *Throws:* Nothing unless the comparison object throws.

117 *Complexity:* $N \log(a.size() + N)$, where N has the value a2.size().

a.erase(k)

118 *Result:* size_type

119 *Effects:* Erases all elements in the container with key equivalent to k.

120 *Returns:* The number of erased elements.

121 *Complexity:* log(a.size()) + a.count(k)

a_tran.erase(kx)

122 *Result:* **size_type**

123 *Effects:* Erases all elements in the container with key **r** such that **!c(r, kx) && !c(kx, r)** is true.

124 *Returns:* The number of erased elements.

125 *Complexity:* $\log(a_tran.size()) + a_tran.count(kx)$

a.erase(q)

126 *Result:* **iterator**

127 *Effects:* Erases the element pointed to by **q**.

128 *Returns:* An iterator pointing to the element immediately following **q** prior to the element being erased. If no such element exists, returns **a.end()**.

129 *Complexity:* Amortized constant.

a.erase(r)

130 *Result:* **iterator**

131 *Effects:* Erases the element pointed to by **r**.

132 *Returns:* An iterator pointing to the element immediately following **r** prior to the element being erased. If no such element exists, returns **a.end()**.

133 *Complexity:* Amortized constant.

a.erase(q1, q2)

134 *Result:* **iterator**

135 *Effects:* Erases all the elements in the range **[q1, q2)**.

136 *Returns:* An iterator pointing to the element pointed to by **q2** prior to any elements being erased. If no such element exists, **a.end()** is returned.

137 *Complexity:* $\log(a.size()) + N$, where **N** has the value **distance(q1, q2)**.

a.clear()

138 *Effects:* Equivalent to **a.erase(a.begin(), a.end())**.

139 *Postconditions:* **a.empty()** is true.

140 *Complexity:* Linear in **a.size()**.

b.find(k)

141 *Result:* **const_iterator** if **b** is of type **const X**; **iterator** otherwise.

142 *Returns:* An iterator pointing to an element with the key equivalent to **k**, or **b.end()** if such an element is not found.

143 *Complexity:* Logarithmic.

a_tran.find(ke)

144 *Result:* **const_iterator** if **a_tran** is of type **const X**; **iterator** otherwise.

145 *Returns:* An iterator pointing to an element with key **r** such that **!c(r, ke) && !c(ke, r)** is true, or **a_tran.end()** if such an element is not found.

146 *Complexity:* Logarithmic.

b.count(k)

147 *Result:* **size_type**

148 *Returns:* The number of elements with key equivalent to **k**.

149 *Complexity:* $\log(b.size()) + b.count(k)$

a_tran.count(ke)

150 *Result:* **size_type**

151 *Returns:* The number of elements with key `r` such that `!c(r, ke) && !c(ke, r)`.

152 *Complexity:* `log(a_tran.size()) + a_tran.count(ke)`

`b.contains(k)`

153 *Result:* `bool`

154 *Effects:* Equivalent to: `return b.find(k) != b.end();`

`a_tran.contains(ke)`

155 *Result:* `bool`

156 *Effects:* Equivalent to: `return a_tran.find(ke) != a_tran.end();`

`b.lower_bound(k)`

157 *Result:* `const_iterator` if `b` is of type `const X`; `iterator` otherwise.

158 *Returns:* An iterator pointing to the first element with key not less than `k`, or `b.end()` if such an element is not found.

159 *Complexity:* Logarithmic.

`a_tran.lower_bound(kl)`

160 *Result:* `const_iterator` if `a_tran` is of type `const X`; `iterator` otherwise.

161 *Returns:* An iterator pointing to the first element with key `r` such that `!c(r, kl)`, or `a_tran.end()` if such an element is not found.

162 *Complexity:* Logarithmic.

`b.upper_bound(k)`

163 *Result:* `const_iterator` if `b` is of type `const X`; `iterator` otherwise.

164 *Returns:* An iterator pointing to the first element with key greater than `k`, or `b.end()` if such an element is not found.

165 *Complexity:* Logarithmic.

`a_tran.upper_bound(ku)`

166 *Result:* `const_iterator` if `a_tran` is of type `const X`; `iterator` otherwise.

167 *Returns:* An iterator pointing to the first element with key `r` such that `c(ku, r)`, or `a_tran.end()` if such an element is not found.

168 *Complexity:* Logarithmic.

`b.equal_range(k)`

169 *Result:* `pair<const_iterator, const_iterator>` if `b` is of type `const X`; `pair<iterator, iterator>` otherwise.

170 *Effects:* Equivalent to: `return make_pair(b.lower_bound(k), b.upper_bound(k));`

171 *Complexity:* Logarithmic.

`a_tran.equal_range(ke)`

172 *Result:* `pair<const_iterator, const_iterator>` if `a_tran` is of type `const X`; `pair<iterator, iterator>` otherwise.

173 *Effects:* Equivalent to: `return make_pair(a_tran.lower_bound(ke), a_tran.upper_bound(ke));`

174 *Complexity:* Logarithmic.

175 The `insert`, `insert_range`, and `emplace` members shall not affect the validity of iterators and references to the container, and the `erase` members shall invalidate only iterators and references to the erased elements.

176 The `extract` members invalidate only iterators to the removed element; pointers and references to the removed element remain valid. However, accessing the element through such pointers and references while the element is owned by a `node_type` is undefined behavior. References and pointers to an element obtained while it is owned by a `node_type` are invalidated if the element is successfully inserted.

- 177 The fundamental property of iterators of associative containers is that they iterate through the containers in the non-descending order of keys where non-descending is defined by the comparison that was used to construct them. For any two dereferenceable iterators `i` and `j` such that distance from `i` to `j` is positive, the following condition holds:

```
value_comp(*j, *i) == false
```

- 178 For associative containers with unique keys the stronger condition holds:

```
value_comp(*i, *j) != false
```

- 179 When an associative container is constructed by passing a comparison object the container shall not store a pointer or reference to the passed object, even if that object is passed by reference. When an associative container is copied, through either a copy constructor or an assignment operator, the target container shall then use the comparison object from the container being copied, as if that comparison object had been passed to the target container in its constructor.

- 180 The member function templates `find`, `count`, `contains`, `lower_bound`, `upper_bound`, `equal_range`, `erase`, and `extract` shall not participate in overload resolution unless the *qualified-id* `Compare::is_transparent` is valid and denotes a type (13.10.3). Additionally, the member function templates `extract` and `erase` shall not participate in overload resolution if `is_convertible_v<K&&, iterator> || is_convertible_v<K&&, const_iterator>` is `true`, where `K` is the type substituted as the first template argument.

- 181 A deduction guide for an associative container shall not participate in overload resolution if any of the following are true:

- (181.1) — It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- (181.2) — It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- (181.3) — It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.

23.2.7.2 Exception safety guarantees

[associative.reqmts.except]

- 1 For associative containers, no `clear()` function throws an exception. `erase(k)` does not throw an exception unless that exception is thrown by the container's `Compare` object (if any).
- 2 For associative containers, if an exception is thrown by any operation from within an `insert` or `emplace` function inserting a single element, the insertion has no effect.
- 3 For associative containers, no `swap` function throws an exception unless that exception is thrown by the swap of the container's `Compare` object (if any).

23.2.8 Unordered associative containers

[unord.req]

23.2.8.1 General

[unord.req.general]

- 1 Unordered associative containers provide an ability for fast retrieval of data based on keys. The worst-case complexity for most operations is linear, but the average case is much faster. The library provides four unordered associative containers: `unordered_set`, `unordered_map`, `unordered_multiset`, and `unordered_multimap`.
- 2 Unordered associative containers conform to the requirements for Containers (23.2), except that the expressions `a == b` and `a != b` have different semantics than for the other container types.
- 3 Each unordered associative container is parameterized by `Key`, by a function object type `Hash` that meets the *Cpp17Hash* requirements (16.4.4.5) and acts as a hash function for argument values of type `Key`, and by a binary predicate `Pred` that induces an equivalence relation on values of type `Key`. Additionally, `unordered_map` and `unordered_multimap` associate an arbitrary *mapped type* `T` with the `Key`.
- 4 The container's object of type `Hash` — denoted by `hash` — is called the *hash function* of the container. The container's object of type `Pred` — denoted by `pred` — is called the *key equality predicate* of the container.
- 5 Two values `k1` and `k2` are considered equivalent if the container's key equality predicate `pred(k1, k2)` is valid and returns `true` when passed those values. If `k1` and `k2` are equivalent, the container's hash function shall return the same value for both.

[Note 1: Thus, when an unordered associative container is instantiated with a non-default `Pred` parameter it usually needs a non-default `Hash` parameter as well. — end note]

For any two keys `k1` and `k2` in the same container, calling `pred(k1, k2)` shall always return the same value. For any key `k` in a container, calling `hash(k)` shall always return the same value.

- 6 An unordered associative container supports *unique keys* if it may contain at most one element for each key. Otherwise, it supports *equivalent keys*. `unordered_set` and `unordered_map` support unique keys. `unordered_multiset` and `unordered_multimap` support equivalent keys. In containers that support equivalent keys, elements with equivalent keys are adjacent to each other in the iteration order of the container. Thus, although the absolute order of elements in an unordered container is not specified, its elements are grouped into *equivalent-key groups* such that all elements of each group have equivalent keys. Mutating operations on unordered containers shall preserve the relative order of elements within each equivalent-key group unless otherwise specified.
- 7 For `unordered_set` and `unordered_multiset` the value type is the same as the key type. For `unordered_map` and `unordered_multimap` it is `pair<const Key, T>`.
- 8 For unordered containers where the value type is the same as the key type, both `iterator` and `const_iterator` are constant iterators. It is unspecified whether or not `iterator` and `const_iterator` are the same type.

[Note 2: `iterator` and `const_iterator` have identical semantics in this case, and `iterator` is convertible to `const_iterator`. Users can avoid violating the one-definition rule by always using `const_iterator` in their function parameter lists. — end note]

- 9 The elements of an unordered associative container are organized into *buckets*. Keys with the same hash code appear in the same bucket. The number of buckets is automatically increased as elements are added to an unordered associative container, so that the average number of elements per bucket is kept below a bound. Rehashing invalidates iterators, changes ordering between elements, and changes which buckets elements appear in, but does not invalidate pointers or references to elements. For `unordered_multiset` and `unordered_multimap`, rehashing preserves the relative ordering of equivalent elements.

10 In this subclause,

- (10.1) — `X` denotes an unordered associative container class,
- (10.2) — `a` denotes a value of type `X`,
- (10.3) — `a2` denotes a value of a type with nodes compatible with type `X` (Table 75),
- (10.4) — `b` denotes a value of type `X` or `const X`,
- (10.5) — `a_uniq` denotes a value of type `X` when `X` supports unique keys,
- (10.6) — `a_eq` denotes a value of type `X` when `X` supports equivalent keys,
- (10.7) — `a_tran` denotes a value of type `X` or `const X` when the *qualified-ids* `X::key_equal::is_transparent` and `X::hasher::is_transparent` are both valid and denote types (13.10.3),
- (10.8) — `i` and `j` denote input iterators that refer to `value_type`,
- (10.9) — `[i, j)` denotes a valid range,
- (10.10) — `rg` denotes a value of a type `R` that models *container-compatible-range*<`value_type`>,
- (10.11) — `p` and `q2` denote valid constant iterators to `a`,
- (10.12) — `q` and `q1` denote valid dereferenceable constant iterators to `a`,
- (10.13) — `r` denotes a valid dereferenceable iterator to `a`,
- (10.14) — `[q1, q2)` denotes a valid range in `a`,
- (10.15) — `il` denotes a value of type `initializer_list<value_type>`,
- (10.16) — `t` denotes a value of type `X::value_type`,
- (10.17) — `k` denotes a value of type `key_type`,
- (10.18) — `hf` denotes a value of type `hasher` or `const hasher`,
- (10.19) — `eq` denotes a value of type `key_equal` or `const key_equal`,
- (10.20) — `ke` is a value such that
 - (10.20.1) — `eq(r1, ke) == eq(ke, r1)`,
 - (10.20.2) — `hf(r1) == hf(ke)` if `eq(r1, ke)` is true, and

- (10.20.3) — if any two of `eq(r1, ke)`, `eq(r2, ke)`, and `eq(r1, r2)` are `true`, then all three are `true`, where `r1` and `r2` are keys of elements in `a_tran`,
- (10.21) — `kx` is a value such that
- (10.21.1) — `eq(r1, kx) == eq(kx, r1)`,
- (10.21.2) — `hf(r1) == hf(kx)` if `eq(r1, kx)` is `true`,
- (10.21.3) — if any two of `eq(r1, kx)`, `eq(r2, kx)`, and `eq(r1, r2)` are `true`, then all three are `true`, and
- (10.21.4) — `kx` is not convertible to either `iterator` or `const_iterator`, where `r1` and `r2` are keys of elements in `a_tran`,
- (10.22) — `n` denotes a value of type `size_type`,
- (10.23) — `z` denotes a value of type `float`, and
- (10.24) — `nh` denotes an rvalue of type `X::node_type`.

- 11 A type `X` meets the *unordered associative container* requirements if `X` meets all the requirements of an allocator-aware container (23.2.2.5) and the following types, statements, and expressions are well-formed and have the specified semantics, except that for `unordered_map` and `unordered_multimap`, the requirements placed on `value_type` in 23.2.2.2 apply instead to `key_type` and `mapped_type`.

[Note 3: For example, `key_type` and `mapped_type` sometimes need to be *Cpp17CopyAssignable* even though the associated `value_type`, `pair<const key_type, mapped_type>`, is not *Cpp17CopyAssignable*. — end note]

`typename X::key_type`

- 12 *Result:* `Key`.

`typename X::mapped_type`

- 13 *Result:* `T`.

- 14 *Remarks:* For `unordered_map` and `unordered_multimap` only.

`typename X::value_type`

- 15 *Result:* `Key` for `unordered_set` and `unordered_multiset` only; `pair<const Key, T>` for `unordered_map` and `unordered_multimap` only.

- 16 *Preconditions:* `value_type` is *Cpp17Erasable* from `X`.

`typename X::hasher`

- 17 *Result:* `Hash`.

- 18 *Preconditions:* `Hash` is a unary function object type such that the expression `hf(k)` has type `size_t`.

`typename X::key_equal`

- 19 *Result:* `Pred`.

- 20 *Preconditions:* `Pred` meets the *Cpp17CopyConstructible* requirements. `Pred` is a binary predicate that takes two arguments of type `Key`. `Pred` is an equivalence relation.

`typename X::local_iterator`

- 21 *Result:* An iterator type whose category, value type, difference type, and pointer and reference types are the same as `X::iterator`'s.

[Note 4: A `local_iterator` object can be used to iterate through a single bucket, but cannot be used to iterate across buckets. — end note]

`typename X::const_local_iterator`

- 22 *Result:* An iterator type whose category, value type, difference type, and pointer and reference types are the same as `X::const_iterator`'s.

[Note 5: A `const_local_iterator` object can be used to iterate through a single bucket, but cannot be used to iterate across buckets. — end note]

typename X::node_type

23 *Result:* A specialization of a *node-handle* class template (23.2.5), such that the public nested types are the same types as the corresponding types in X.

X(n, hf, eq)

24 *Effects:* Constructs an empty container with at least n buckets, using hf as the hash function and eq as the key equality predicate.

25 *Complexity:* $\mathcal{O}(n)$

X(n, hf)

26 *Preconditions:* key_equal meets the Cpp17DefaultConstructible requirements.

27 *Effects:* Constructs an empty container with at least n buckets, using hf as the hash function and key_equal() as the key equality predicate.

28 *Complexity:* $\mathcal{O}(n)$

X(n)

29 *Preconditions:* hasher and key_equal meet the Cpp17DefaultConstructible requirements.

30 *Effects:* Constructs an empty container with at least n buckets, using hasher() as the hash function and key_equal() as the key equality predicate.

31 *Complexity:* $\mathcal{O}(n)$

X a = X();

X a;

32 *Preconditions:* hasher and key_equal meet the Cpp17DefaultConstructible requirements.

33 *Effects:* Constructs an empty container with an unspecified number of buckets, using hasher() as the hash function and key_equal() as the key equality predicate.

34 *Complexity:* Constant.

X(i, j, n, hf, eq)

35 *Preconditions:* value_type is Cpp17EmplaceConstructible into X from *i.

36 *Effects:* Constructs an empty container with at least n buckets, using hf as the hash function and eq as the key equality predicate, and inserts elements from [i, j) into it.

37 *Complexity:* Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$.

X(i, j, n, hf)

38 *Preconditions:* key_equal meets the Cpp17DefaultConstructible requirements. value_type is Cpp17EmplaceConstructible into X from *i.

39 *Effects:* Constructs an empty container with at least n buckets, using hf as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.

40 *Complexity:* Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$.

X(i, j, n)

41 *Preconditions:* hasher and key_equal meet the Cpp17DefaultConstructible requirements. value_type is Cpp17EmplaceConstructible into X from *i.

42 *Effects:* Constructs an empty container with at least n buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.

43 *Complexity:* Average case $\mathcal{O}(N)$ (N is distance(i, j)), worst case $\mathcal{O}(N^2)$.

X(i, j)

44 *Preconditions:* hasher and key_equal meet the Cpp17DefaultConstructible requirements. value_type is Cpp17EmplaceConstructible into X from *i.

45 *Effects:* Constructs an empty container with an unspecified number of buckets, using hasher() as the hash function and key_equal() as the key equality predicate, and inserts elements from [i, j) into it.

46 *Complexity:* Average case $\mathcal{O}(N)$ (N is `distance(i, j)`), worst case $\mathcal{O}(N^2)$.

`X(from_range, rg, n, hf, eq)`

47 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`.

48 *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `eq` as the key equality predicate, and inserts elements from `rg` into it.

49 *Complexity:* Average case $\mathcal{O}(N)$ (N is `ranges::distance(rg)`), worst case $\mathcal{O}(N^2)$.

`X(from_range, rg, n, hf)`

50 *Preconditions:* `key_equal` meets the *Cpp17DefaultConstructible* requirements. `value_type` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`.

51 *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `rg` into it.

52 *Complexity:* Average case $\mathcal{O}(N)$ (N is `ranges::distance(rg)`), worst case $\mathcal{O}(N^2)$.

`X(from_range, rg, n)`

53 *Preconditions:* `hasher` and `key_equal` meet the *Cpp17DefaultConstructible* requirements. `value_type` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`.

54 *Effects:* Constructs an empty container with at least `n` buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `rg` into it.

55 *Complexity:* Average case $\mathcal{O}(N)$ (N is `ranges::distance(rg)`), worst case $\mathcal{O}(N^2)$.

`X(from_range, rg)`

56 *Preconditions:* `hasher` and `key_equal` meet the *Cpp17DefaultConstructible* requirements. `value_type` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`.

57 *Effects:* Constructs an empty container with an unspecified number of buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `rg` into it.

58 *Complexity:* Average case $\mathcal{O}(N)$ (N is `ranges::distance(rg)`), worst case $\mathcal{O}(N^2)$.

`X(il)`

59 *Effects:* Equivalent to `X(il.begin(), il.end())`.

`X(il, n)`

60 *Effects:* Equivalent to `X(il.begin(), il.end(), n)`.

`X(il, n, hf)`

61 *Effects:* Equivalent to `X(il.begin(), il.end(), n, hf)`.

`X(il, n, hf, eq)`

62 *Effects:* Equivalent to `X(il.begin(), il.end(), n, hf, eq)`.

`X(b)`

63 *Effects:* In addition to the container requirements (23.2.2.2), copies the hash function, predicate, and maximum load factor.

64 *Complexity:* Average case linear in `b.size()`, worst case quadratic.

`a = b`

65 *Result:* `X&`

66 *Effects:* In addition to the container requirements, copies the hash function, predicate, and maximum load factor.

67 *Complexity:* Average case linear in `b.size()`, worst case quadratic.

`a = il`

68 *Result:* `X&`

69 *Preconditions:* `value_type` is *Cpp17CopyInsertable* into `X` and *Cpp17CopyAssignable*.

70 *Effects:* Assigns the range `[il.begin(), il.end())` into `a`. All existing elements of `a` are either assigned to or destroyed.

71 *Complexity:* Average case linear in `il.size()`, worst case quadratic.

`b.hash_function()`

72 *Result:* `hasher`

73 *Returns:* `b`'s hash function.

74 *Complexity:* Constant.

`b.key_eq()`

75 *Result:* `key_equal`

76 *Returns:* `b`'s key equality predicate.

77 *Complexity:* Constant.

`a_uniq.emplace(args)`

78 *Result:* `pair<iterator, bool>`

79 *Preconditions:* `value_type` is *Cpp17EplaceConstructible* into `X` from `args`.

80 *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...` if and only if there is no element in the container with key equivalent to the key of `t`.

81 *Returns:* The `bool` component of the returned pair is `true` if and only if the insertion takes place, and the iterator component of the pair points to the element with key equivalent to the key of `t`.

82 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_uniq.size())$.

`a_eq.emplace(args)`

83 *Result:* `iterator`

84 *Preconditions:* `value_type` is *Cpp17EplaceConstructible* into `X` from `args`.

85 *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...`

86 *Returns:* An iterator pointing to the newly inserted element.

87 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_eq.size())$.

`a.emplace_hint(p, args)`

88 *Result:* `iterator`

89 *Effects:* Equivalent to `a.emplace(std::forward<Args>(args)...)...`, except that the `const_iterator` `p` is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.

90 *Returns:* The iterator returned by `emplace`.

`a_uniq.insert(t)`

91 *Result:* `pair<iterator, bool>`

92 *Preconditions:* If `t` is a non-const rvalue, `value_type` is *Cpp17MoveInsertable* into `X`; otherwise, `value_type` is *Cpp17CopyInsertable* into `X`.

93 *Effects:* Inserts `t` if and only if there is no element in the container with key equivalent to the key of `t`.

94 *Returns:* The `bool` component of the returned pair indicates whether the insertion takes place, and the iterator component points to the element with key equivalent to the key of `t`.

95 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_uniq.size())$.

`a_eq.insert(t)`

96 *Result:* `iterator`

97 *Preconditions:* If `t` is a non-const rvalue, `value_type` is *Cpp17MoveInsertable* into `X`; otherwise, `value_type` is *Cpp17CopyInsertable* into `X`.

98 *Effects:* Inserts `t`.

99 *Returns:* An iterator pointing to the newly inserted element.

100 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\text{a_eq.size}())$.

`a.insert(p, t)`

101 *Result:* iterator

102 *Preconditions:* If `t` is a non-const rvalue, `value_type` is *Cpp17MoveInsertable* into `X`; otherwise, `value_type` is *Cpp17CopyInsertable* into `X`.

103 *Effects:* Equivalent to `a.insert(t)`. The iterator `p` is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.

104 *Returns:* An iterator pointing to the element with the key equivalent to that of `t`.

105 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\text{a.size}())$.

`a.insert(i, j)`

106 *Result:* void

107 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `X` from `*i`. Neither `i` nor `j` are iterators into `a`.

108 *Effects:* Equivalent to `a.insert(t)` for each element in `[i, j)`.

109 *Complexity:* Average case $\mathcal{O}(N)$, where N is `distance(i, j)`, worst case $\mathcal{O}(N(\text{a.size}() + 1))$.

`a.insert_range(rg)`

110 *Result:* void

111 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `X` from `*ranges::begin(rg)`. `rg` and `a` do not overlap.

112 *Effects:* Equivalent to `a.insert(t)` for each element `t` in `rg`.

113 *Complexity:* Average case $\mathcal{O}(N)$, where N is `ranges::distance(rg)`, worst case $\mathcal{O}(N(\text{a.size}() + 1))$.

`a.insert(il)`

114 *Effects:* Equivalent to `a.insert(il.begin(), il.end())`.

`a_uniq.insert(nh)`

115 *Result:* `insert_return_type`

116 *Preconditions:* `nh` is empty or `a_uniq.get_allocator() == nh.get_allocator()` is true.

117 *Effects:* If `nh` is empty, has no effect. Otherwise, inserts the element owned by `nh` if and only if there is no element in the container with a key equivalent to `nh.key()`.

118 *Postconditions:* If `nh` is empty, `inserted` is false, `position` is `end()`, and `node` is empty. Otherwise if the insertion took place, `inserted` is true, `position` points to the inserted element, and `node` is empty; if the insertion failed, `inserted` is false, `node` has the previous value of `nh`, and `position` points to an element with a key equivalent to `nh.key()`.

119 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\text{a_uniq.size}())$.

`a_eq.insert(nh)`

120 *Result:* iterator

121 *Preconditions:* `nh` is empty or `a_eq.get_allocator() == nh.get_allocator()` is true.

122 *Effects:* If `nh` is empty, has no effect and returns `a_eq.end()`. Otherwise, inserts the element owned by `nh` and returns an iterator pointing to the newly inserted element.

123 *Postconditions:* `nh` is empty.

124 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\text{a_eq.size}())$.

`a.insert(q, nh)`

125 *Result:* iterator

126 *Preconditions:* `nh` is empty or `a.get_allocator() == nh.get_allocator()` is true.

127 *Effects:* If `nh` is empty, has no effect and returns `a.end()`. Otherwise, inserts the element owned by `nh` if and only if there is no element with key equivalent to `nh.key()` in containers with unique keys; always inserts the element owned by `nh` in containers with equivalent keys. The iterator `q` is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.

128 *Postconditions:* `nh` is empty if insertion succeeds, unchanged if insertion fails.

129 *Returns:* An iterator pointing to the element with key equivalent to `nh.key()`.

130 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a.size())$.

a.extract(k)

131 *Result:* `node_type`

132 *Effects:* Removes an element in the container with key equivalent to `k`.

133 *Returns:* A `node_type` owning the element if found, otherwise an empty `node_type`.

134 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a.size())$.

a_tran.extract(kx)

135 *Result:* `node_type`

136 *Effects:* Removes an element in the container with key equivalent to `kx`.

137 *Returns:* A `node_type` owning the element if found, otherwise an empty `node_type`.

138 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a_tran.size())$.

a.extract(q)

139 *Result:* `node_type`

140 *Effects:* Removes the element pointed to by `q`.

141 *Returns:* A `node_type` owning that element.

142 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(a.size())$.

a.merge(a2)

143 *Result:* `void`

144 *Preconditions:* `a.get_allocator() == a2.get_allocator()`.

145 *Effects:* Attempts to extract each element in `a2` and insert it into `a` using the hash function and key equality predicate of `a`. In containers with unique keys, if there is an element in `a` with key equivalent to the key of an element from `a2`, then that element is not extracted from `a2`.

146 *Postconditions:* Pointers and references to the transferred elements of `a2` refer to those same elements but as members of `a`. Iterators referring to the transferred elements and all iterators referring to `a` will be invalidated, but iterators to elements remaining in `a2` will remain valid.

147 *Complexity:* Average case $\mathcal{O}(N)$, where N is `a2.size()`, worst case $\mathcal{O}(N*a.size() + N)$.

a.erase(k)

148 *Result:* `size_type`

149 *Effects:* Erases all elements with key equivalent to `k`.

150 *Returns:* The number of elements erased.

151 *Complexity:* Average case $\mathcal{O}(a.count(k))$, worst case $\mathcal{O}(a.size())$.

a_tran.erase(kx)

152 *Result:* `size_type`

153 *Effects:* Erases all elements with key equivalent to `kx`.

154 *Returns:* The number of elements erased.

155 *Complexity:* Average case $\mathcal{O}(a_tran.count(kx))$, worst case $\mathcal{O}(a_tran.size())$.

a.erase(q)

156 *Result:* `iterator`

157 *Effects:* Erases the element pointed to by `q`.

158 *Returns:* The iterator immediately following `q` prior to the erasure.

159 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\mathbf{a.size}())$.

`a.erase(r)`

160 *Result:* `iterator`

161 *Effects:* Erases the element pointed to by `r`.

162 *Returns:* The iterator immediately following `r` prior to the erasure.

163 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\mathbf{a.size}())$.

`a.erase(q1, q2)`

164 *Result:* `iterator`

165 *Effects:* Erases all elements in the range `[q1, q2)`.

166 *Returns:* The iterator immediately following the erased elements prior to the erasure.

167 *Complexity:* Average case linear in `distance(q1, q2)`, worst case $\mathcal{O}(\mathbf{a.size}())$.

`a.clear()`

168 *Result:* `void`

169 *Effects:* Erases all elements in the container.

170 *Postconditions:* `a.empty()` is `true`.

171 *Complexity:* Linear in `a.size()`.

`b.find(k)`

172 *Result:* `const_iterator` if `b` is of type `const X`; `iterator` otherwise.

173 *Returns:* An iterator pointing to an element with key equivalent to `k`, or `b.end()` if no such element exists.

174 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\mathbf{b.size}())$.

`a_tran.find(ke)`

175 *Result:* `const_iterator` if `a_tran` is of type `const X`; `iterator` otherwise.

176 *Returns:* An iterator pointing to an element with key equivalent to `ke`, or `a_tran.end()` if no such element exists.

177 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}(\mathbf{a_tran.size}())$.

`b.count(k)`

178 *Result:* `size_type`

179 *Returns:* The number of elements with key equivalent to `k`.

180 *Complexity:* Average case $\mathcal{O}(\mathbf{b.count(k)})$, worst case $\mathcal{O}(\mathbf{b.size}())$.

`a_tran.count(ke)`

181 *Result:* `size_type`

182 *Returns:* The number of elements with key equivalent to `ke`.

183 *Complexity:* Average case $\mathcal{O}(\mathbf{a_tran.count(ke)})$, worst case $\mathcal{O}(\mathbf{a_tran.size}())$.

`b.contains(k)`

184 *Effects:* Equivalent to `b.find(k) != b.end()`.

`a_tran.contains(ke)`

185 *Effects:* Equivalent to `a_tran.find(ke) != a_tran.end()`.

b.equal_range(k)

186 *Result:* `pair<const_iterator, const_iterator>` if `b` is of type `const X`; `pair<iterator, iterator>` otherwise.

187 *Returns:* A range containing all elements with keys equivalent to `k`. Returns `make_pair(b.end(), b.end())` if no such elements exist.

188 *Complexity:* Average case $\mathcal{O}(b.count(k))$, worst case $\mathcal{O}(b.size())$.

a_tran.equal_range(ke)

189 *Result:* `pair<const_iterator, const_iterator>` if `a_tran` is of type `const X`; `pair<iterator, iterator>` otherwise.

190 *Returns:* A range containing all elements with keys equivalent to `ke`. Returns `make_pair(a_tran.end(), a_tran.end())` if no such elements exist.

191 *Complexity:* Average case $\mathcal{O}(a_tran.count(ke))$, worst case $\mathcal{O}(a_tran.size())$.

b.bucket_count()

192 *Result:* `size_type`

193 *Returns:* The number of buckets that `b` contains.

194 *Complexity:* Constant.

b.max_bucket_count()

195 *Result:* `size_type`

196 *Returns:* An upper bound on the number of buckets that `b` can ever contain.

197 *Complexity:* Constant.

b.bucket(k)

198 *Result:* `size_type`

199 *Preconditions:* `b.bucket_count() > 0`.

200 *Returns:* The index of the bucket in which elements with keys equivalent to `k` would be found, if any such element existed. The return value is in the range `[0, b.bucket_count())`.

201 *Complexity:* Constant.

a_tran.bucket(ke)

202 *Result:* `size_type`

203 *Preconditions:* `a_tran.bucket_count() > 0`.

204 *Postconditions:* The return value is in the range `[0, a_tran.bucket_count())`.

205 *Returns:* The index of the bucket in which elements with keys equivalent to `ke` would be found, if any such element existed.

206 *Complexity:* Constant.

b.bucket_size(n)

207 *Result:* `size_type`

208 *Preconditions:* `n` shall be in the range `[0, b.bucket_count())`.

209 *Returns:* The number of elements in the n^{th} bucket.

210 *Complexity:* $\mathcal{O}(b.bucket_size(n))$

b.begin(n)

211 *Result:* `const_local_iterator` if `b` is of type `const X`; `local_iterator` otherwise.

212 *Preconditions:* `n` is in the range `[0, b.bucket_count())`.

213 *Returns:* An iterator referring to the first element in the bucket. If the bucket is empty, then `b.begin(n) == b.end(n)`.

214 *Complexity:* Constant.

b.end(n)

215 *Result:* `const_local_iterator` if `b` is of type `const X`; `local_iterator` otherwise.

216 *Preconditions:* `n` is in the range `[0, b.bucket_count())`.

217 *Returns:* An iterator which is the past-the-end value for the bucket.

218 *Complexity:* Constant.

b.cbegin(n)

219 *Result:* `const_local_iterator`

220 *Preconditions:* `n` shall be in the range `[0, b.bucket_count())`.

221 *Returns:* An iterator referring to the first element in the bucket. If the bucket is empty, then `b.cbegin(n) == b.cend(n)`.

222 *Complexity:* Constant.

b.cend(n)

223 *Result:* `const_local_iterator`

224 *Preconditions:* `n` is in the range `[0, b.bucket_count())`.

225 *Returns:* An iterator which is the past-the-end value for the bucket.

226 *Complexity:* Constant.

b.load_factor()

227 *Result:* `float`

228 *Returns:* The average number of elements per bucket.

229 *Complexity:* Constant.

b.max_load_factor()

230 *Result:* `float`

231 *Returns:* A positive number that the container attempts to keep the load factor less than or equal to. The container automatically increases the number of buckets as necessary to keep the load factor below this number.

232 *Complexity:* Constant.

a.max_load_factor(z)

233 *Result:* `void`

234 *Preconditions:* `z` is positive. May change the container's maximum load factor, using `z` as a hint.

235 *Complexity:* Constant.

a.rehash(n)

236 *Result:* `void`

237 *Postconditions:* `a.bucket_count() >= a.size() / a.max_load_factor()` and `a.bucket_count() >= n`.

238 *Complexity:* Average case linear in `a.size()`, worst case quadratic.

a.reserve(n)

239 *Effects:* Equivalent to `a.rehash(ceil(n / a.max_load_factor()))`.

240 Two unordered containers `a` and `b` compare equal if `a.size() == b.size()` and, for every equivalent-key group `[Ea1, Ea2]` obtained from `a.equal_range(Ea1)`, there exists an equivalent-key group `[Eb1, Eb2]` obtained from `b.equal_range(Ea1)`, such that `is_permutation(Ea1, Ea2, Eb1, Eb2)` returns `true`. For `unordered_set` and `unordered_map`, the complexity of `operator==` (i.e., the number of calls to the `==` operator of the `value_type`, to the predicate returned by `key_eq()`, and to the hasher returned by `hash_function()`) is proportional to N in the average case and to N^2 in the worst case, where N is `a.size()`. For `unordered_multiset` and `unordered_multimap`, the complexity of `operator==` is proportional to $\sum E_i^2$ in the average case and to N^2 in the worst case, where N is `a.size()`, and E_i is the size of the i^{th} equivalent-key

group in **a**. However, if the respective elements of each corresponding pair of equivalent-key groups Ea_i and Eb_i are arranged in the same order (as is commonly the case, e.g., if **a** and **b** are unmodified copies of the same container), then the average-case complexity for `unordered_multiset` and `unordered_multimap` becomes proportional to N (but worst-case complexity remains $\mathcal{O}(N^2)$, e.g., for a pathologically bad hash function). The behavior of a program that uses `operator==` or `operator!=` on unordered containers is undefined unless the `Pred` function object has the same behavior for both containers and the equality comparison function for `Key` is a refinement¹⁷³ of the partition into equivalent-key groups produced by `Pred`.

- 241 The iterator types `iterator` and `const_iterator` of an unordered associative container are of at least the forward iterator category. For unordered associative containers where the key type and value type are the same, both `iterator` and `const_iterator` are constant iterators.
- 242 The `insert`, `insert_range`, and `emplace` members shall not affect the validity of references to container elements, but may invalidate all iterators to the container. The `erase` members shall invalidate only iterators and references to the erased elements, and preserve the relative order of the elements that are not erased.
- 243 The `insert`, `insert_range`, and `emplace` members shall not affect the validity of iterators if $(N + n) \leq z * B$, where N is the number of elements in the container prior to the insert operation, n is the number of elements inserted, B is the container's bucket count, and z is the container's maximum load factor.
- 244 The `extract` members invalidate only iterators to the removed element, and preserve the relative order of the elements that are not erased; pointers and references to the removed element remain valid. However, accessing the element through such pointers and references while the element is owned by a `node_type` is undefined behavior. References and pointers to an element obtained while it is owned by a `node_type` are invalidated if the element is successfully inserted.
- 245 The member function templates `find`, `count`, `equal_range`, `contains`, `extract`, `erase`, and `bucket` shall not participate in overload resolution unless the *qualified-ids* `Pred::is_transparent` and `Hash::is_transparent` are both valid and denote types (13.10.3). Additionally, the member function templates `extract` and `erase` shall not participate in overload resolution if `is_convertible_v<K&&, iterator> || is_convertible_v<K&&, const_iterator>` is `true`, where K is the type substituted as the first template argument.
- 246 A deduction guide for an unordered associative container shall not participate in overload resolution if any of the following are true:
- (246.1) — It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
 - (246.2) — It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
 - (246.3) — It has a `Hash` template parameter and an integral type or a type that qualifies as an allocator is deduced for that parameter.
 - (246.4) — It has a `Pred` template parameter and a type that qualifies as an allocator is deduced for that parameter.

23.2.8.2 Exception safety guarantees

[unord.req.except]

- 1 For unordered associative containers, no `clear()` function throws an exception. `erase(k)` does not throw an exception unless that exception is thrown by the container's `Hash` or `Pred` object (if any).
- 2 For unordered associative containers, if an exception is thrown by any operation other than the container's hash function from within an `insert` or `emplace` function inserting a single element, the insertion has no effect.
- 3 For unordered associative containers, no `swap` function throws an exception unless that exception is thrown by the swap of the container's `Hash` or `Pred` object (if any).
- 4 For unordered associative containers, if an exception is thrown from within a `rehash()` function other than by the container's hash function or comparison function, the `rehash()` function has no effect.

23.3 Sequence containers

[sequences]

23.3.1 General

[sequences.general]

- 1 The headers `<array>` (23.3.2), `<deque>` (23.3.4), `<forward_list>` (23.3.6), `<hive>` (23.3.8), `<inplace_vector>` (23.3.15), `<list>` (23.3.10), and `<vector>` (23.3.12) define class templates that meet the requirements for sequence containers.

¹⁷³) Equality comparison is a refinement of partitioning if no two objects that compare equal fall into different partitions.

- ² The following exposition-only alias template may appear in deduction guides for sequence containers:

```
template<class InputIterator>
    using iter-value-type = iterator_traits<InputIterator>::value_type; // exposition only
```

23.3.2 Header <array> synopsis

[array.syn]

```
// mostly freestanding
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
    // 23.3.3, class template array
    template<class T, size_t N> struct array; // partially freestanding

    template<class T, size_t N>
        constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);
    template<class T, size_t N>
        constexpr synth-three-way-result<T>
            operator<=>(const array<T, N>& x, const array<T, N>& y);

    // 23.3.3.4, specialized algorithms
    template<class T, size_t N>
        constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));

    // 23.3.3.6, array creation functions
    template<class T, size_t N>
        constexpr array<remove_cv_t<T>, N> to_array(T (&a)[N]);
    template<class T, size_t N>
        constexpr array<remove_cv_t<T>, N> to_array(T (&&a)[N]);

    // 23.3.3.7, tuple interface
    template<class T> struct tuple_size;
    template<size_t I, class T> struct tuple_element;
    template<class T, size_t N>
        struct tuple_size<array<T, N>>;
    template<size_t I, class T, size_t N>
        struct tuple_element<I, array<T, N>>;
    template<size_t I, class T, size_t N>
        constexpr T& get(array<T, N>&) noexcept;
    template<size_t I, class T, size_t N>
        constexpr T&& get(array<T, N>&&) noexcept;
    template<size_t I, class T, size_t N>
        constexpr const T& get(const array<T, N>&) noexcept;
    template<size_t I, class T, size_t N>
        constexpr const T&& get(const array<T, N>&&) noexcept;
}
```

23.3.3 Class template array

[array]

23.3.3.1 Overview

[array.overview]

- ¹ The header <array> defines a class template for storing fixed-size sequences of objects. An **array** is a contiguous container (23.2.2.2). An instance of **array<T, N>** stores *N* elements of type *T*, so that **size() == N** is an invariant.
- ² An **array** is an aggregate (9.5.2) that can be list-initialized with up to *N* elements whose types are convertible to *T*.
- ³ An **array** meets all of the requirements of a container (23.2.2.2) and of a reversible container (23.2.2.3), except that a default constructed **array** object is not empty if *N* > 0. An **array** meets some of the requirements of a sequence container (23.2.4). Descriptions are provided here only for operations on **array** that are not described in one of these tables and for operations where there is additional semantic information.
- ⁴ **array<T, N>** is a structural type (13.2) if *T* is a structural type. Two values **a1** and **a2** of type **array<T, N>** are template-argument-equivalent (13.6) if and only if each pair of corresponding elements in **a1** and **a2** are template-argument-equivalent.

- ⁵ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
    template<class T, size_t N>
    struct array {
        // types
        using value_type      = T;
        using pointer         = T*;
        using const_pointer   = const T*;
        using reference       = T&;
        using const_reference = const T&;
        using size_type       = size_t;
        using difference_type  = ptrdiff_t;
        using iterator         = implementation-defined; // see 23.2
        using const_iterator   = implementation-defined; // see 23.2
        using reverse_iterator = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // no explicit construct/copy/destroy for aggregate type

        constexpr void fill(const T& u);
        constexpr void swap(array&) noexcept(is_nothrow_swappable_v<T>);

        // iterators
        constexpr iterator      begin() noexcept;
        constexpr const_iterator begin() const noexcept;
        constexpr iterator      end() noexcept;
        constexpr const_iterator end() const noexcept;

        constexpr reverse_iterator rbegin() noexcept;
        constexpr const_reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator rend() noexcept;
        constexpr const_reverse_iterator rend() const noexcept;

        constexpr const_iterator cbegin() const noexcept;
        constexpr const_iterator cend() const noexcept;
        constexpr const_reverse_iterator crbegin() const noexcept;
        constexpr const_reverse_iterator crend() const noexcept;

        // capacity
        constexpr bool empty() const noexcept;
        constexpr size_type size() const noexcept;
        constexpr size_type max_size() const noexcept;

        // element access
        constexpr reference      operator[](size_type n);
        constexpr const_reference operator[](size_type n) const;
        constexpr reference      at(size_type n); // freestanding-deleted
        constexpr const_reference at(size_type n) const; // freestanding-deleted
        constexpr reference      front();
        constexpr const_reference front() const;
        constexpr reference      back();
        constexpr const_reference back() const;

        constexpr T*      data() noexcept;
        constexpr const T* data() const noexcept;
    };

    template<class T, class... U>
    array(T, U...) -> array<T, 1 + sizeof...(U)>;
}

```

23.3.3.2 Constructors, copy, and assignment

[array.cons]

- ¹ An `array` relies on the implicitly-declared special member functions (11.4.5.2, 11.4.7, 11.4.5.3) to conform to the container requirements table in 23.2. In addition to the requirements specified in the container requirements table, the implicitly-declared move constructor and move assignment operator for `array` require that `T` be `Cpp17MoveConstructible` or `Cpp17MoveAssignable`, respectively.

```
template<class T, class... U>
    array(T, U...) -> array<T, 1 + sizeof...(U)>;
```

- ² *Mandates:* (`is_same_v<T, U> && ...`) is true.

23.3.3.3 Member functions

[array.members]

```
constexpr size_type size() const noexcept;
```

- ¹ *Returns:* `N`.

```
constexpr T* data() noexcept;
constexpr const T* data() const noexcept;
```

- ² *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty array, `data() == addressof(front())` is true.

```
constexpr void fill(const T& u);
```

- ³ *Effects:* As if by `fill_n(begin(), N, u)`.

```
constexpr void swap(array& y) noexcept(is_nothrow_swappable_v<T>);
```

- ⁴ *Effects:* Equivalent to `swap_ranges(begin(), end(), y.begin())`.

- ⁵ [Note 1: Unlike the `swap` function for other containers, `array::swap` takes linear time, can exit via an exception, and does not cause iterators to become associated with the other container. — end note]

23.3.3.4 Specialized algorithms

[array.special]

```
template<class T, size_t N>
    constexpr void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
```

- ¹ *Constraints:* `N == 0` or `is_swappable_v<T>` is true.

- ² *Effects:* As if by `x.swap(y)`.

- ³ *Complexity:* Linear in `N`.

23.3.3.5 Zero-sized arrays

[array.zero]

- ¹ `array` shall provide support for the special case `N == 0`.
- ² In the case that `N == 0`, `begin() == end() == unique value`. The return value of `data()` is unspecified.
- ³ Member function `swap()` shall have a non-throwing exception specification.

23.3.3.6 Array creation functions

[array.creation]

```
template<class T, size_t N>
    constexpr array<remove_cv_t<T>, N> to_array(T (&a)[N]);
```

- ¹ *Mandates:* `is_array_v<T>` is false and `is_constructible_v<remove_cv_t<T>, T&>` is true.

- ² *Preconditions:* `T` meets the `Cpp17CopyConstructible` requirements.

- ³ *Returns:* `{{ a[0], ..., a[N - 1] }}`.

```
template<class T, size_t N>
    constexpr array<remove_cv_t<T>, N> to_array(T (&&a)[N]);
```

- ⁴ *Mandates:* `is_array_v<T>` is false and `is_constructible_v<remove_cv_t<T>, T>` is true.

- ⁵ *Preconditions:* `T` meets the `Cpp17MoveConstructible` requirements.

- ⁶ *Returns:* `{{ std::move(a[0]), ..., std::move(a[N - 1]) }}`.

23.3.3.7 Tuple interface

[array.tuple]

```
template<class T, size_t N>
    struct tuple_size<array<T, N>> : integral_constant<size_t, N> { };

template<size_t I, class T, size_t N>
    struct tuple_element<I, array<T, N>> {
        using type = T;
    };
```

¹ *Mandates:* $I < N$ is true.

```
template<size_t I, class T, size_t N>
    constexpr T& get(array<T, N>& a) noexcept;
template<size_t I, class T, size_t N>
    constexpr T&& get(array<T, N>&& a) noexcept;
template<size_t I, class T, size_t N>
    constexpr const T& get(const array<T, N>& a) noexcept;
template<size_t I, class T, size_t N>
    constexpr const T&& get(const array<T, N>&& a) noexcept;
```

² *Mandates:* $I < N$ is true.

³ *Returns:* A reference to the I^{th} element of *a*, where indexing is zero-based.

23.3.4 Header <deque> synopsis

[deque.syn]

```
#include <compare>           // see 17.12.1
#include <initializer_list>   // see 17.11.2

namespace std {
    // 23.3.5, class template deque
    template<class T, class Allocator = allocator<T>> class deque;

    template<class T, class Allocator>
        constexpr bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
    template<class T, class Allocator>
        constexpr synth-three-way-result<T>
            operator<=>(const deque<T, Allocator>& x, const deque<T, Allocator>& y);

    template<class T, class Allocator>
        constexpr void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    // 23.3.5.5, erasure
    template<class T, class Allocator, class U = T>
        constexpr typename deque<T, Allocator>::size_type
            erase(deque<T, Allocator>& c, const U& value);
    template<class T, class Allocator, class Predicate>
        constexpr typename deque<T, Allocator>::size_type
            erase_if(deque<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template<class T>
            using deque = std::deque<T, polymorphic_allocator<T>>;
    }
}
```

23.3.5 Class template deque

[deque]

23.3.5.1 Overview

[deque.overview]

¹ A *deque* is a sequence container that supports random access iterators (24.3.5.7). In addition, it supports constant time insert and erase operations at the beginning or the end; insert and erase in the middle take linear time. That is, a *deque* is especially optimized for pushing and popping elements at the beginning and end. Storage management is handled automatically.

- ² A `deque` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of a sequence container, including the optional sequence container requirements (23.2.4). Descriptions are provided here only for operations on `deque` that are not described in one of these tables or for operations where there is additional semantic information.
- ³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```
namespace std {
    template<class T, class Allocator = allocator<T>>
    class deque {
    public:
        // types
        using value_type           = T;
        using allocator_type       = Allocator;
        using pointer              = allocator_traits<Allocator>::pointer;
        using const_pointer        = allocator_traits<Allocator>::const_pointer;
        using reference            = value_type&;
        using const_reference      = const value_type&;
        using size_type            = implementation-defined; // see 23.2
        using difference_type      = implementation-defined; // see 23.2
        using iterator             = implementation-defined; // see 23.2
        using const_iterator       = implementation-defined; // see 23.2
        using reverse_iterator     = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 23.3.5.2, construct/copy/destroy
        constexpr deque() : deque(Allocator()) { }
        constexpr explicit deque(const Allocator&);
        constexpr explicit deque(size_type n, const Allocator& = Allocator());
        constexpr deque(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
            constexpr deque(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
            constexpr deque(from_range_t, R&& rg, const Allocator& = Allocator());
        constexpr deque(const deque& x);
        constexpr deque(deque&&);
        constexpr deque(const deque&, const type_identity_t<Allocator>&);
        constexpr deque(deque&&, const type_identity_t<Allocator>&);
        constexpr deque(initializer_list<T>, const Allocator& = Allocator());

        constexpr ~deque();
        constexpr deque& operator=(const deque& x);
        constexpr deque& operator=(deque&& x)
            noexcept(allocator_traits<Allocator>::is_always_equal::value);
        constexpr deque& operator=(initializer_list<T>);
        template<class InputIterator>
            constexpr void assign(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
            constexpr void assign_range(R&& rg);
        constexpr void assign(size_type n, const T& t);
        constexpr void assign(initializer_list<T>);
        constexpr allocator_type get_allocator() const noexcept;

        // iterators
        constexpr iterator          begin() noexcept;
        constexpr const_iterator    begin() const noexcept;
        constexpr iterator          end() noexcept;
        constexpr const_iterator    end() const noexcept;
        constexpr reverse_iterator  rbegin() noexcept;
        constexpr const_reverse_iterator rbegin() const noexcept;
        constexpr reverse_iterator  rend() noexcept;
        constexpr const_reverse_iterator rend() const noexcept;

        constexpr const_iterator    cbegin() const noexcept;
        constexpr const_iterator    cend() const noexcept;
    };
}
```

```

constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 23.3.5.3, capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr void      resize(size_type sz);
constexpr void      resize(size_type sz, const T& c);
constexpr void      shrink_to_fit();

// element access
constexpr reference      operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference      at(size_type n);
constexpr const_reference at(size_type n) const;
constexpr reference      front();
constexpr const_reference front() const;
constexpr reference      back();
constexpr const_reference back() const;

// 23.3.5.4, modifiers
template<class... Args> constexpr reference emplace_front(Args&&... args);
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);

constexpr void push_front(const T& x);
constexpr void push_front(T&& x);
template<container-compatible-range<T> R>
    constexpr void prepend_range(R&& rg);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);

constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position,
                             InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);

constexpr void pop_front();
constexpr void pop_back();

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void      swap(deque&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
constexpr void      clear() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
    deque(InputIterator, InputIterator, Allocator = Allocator())
    -> deque<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
    deque(from_range_t, R&&, Allocator = Allocator())
    -> deque<ranges::range_value_t<R>, Allocator>;
}

```

23.3.5.2 Constructors, copy, and assignment

[deque.cons]

```
constexpr explicit deque(const Allocator&);
```

1 *Effects:* Constructs an empty `deque`, using the specified allocator.

2 *Complexity:* Constant.

```
constexpr explicit deque(size_type n, const Allocator& = Allocator());
```

3 *Preconditions:* `T` is *Cpp17DefaultInsertable* into `deque`.

4 *Effects:* Constructs a `deque` with `n` default-inserted elements using the specified allocator.

5 *Complexity:* Linear in `n`.

```
constexpr deque(size_type n, const T& value, const Allocator& = Allocator());
```

6 *Preconditions:* `T` is *Cpp17CopyInsertable* into `deque`.

7 *Effects:* Constructs a `deque` with `n` copies of `value`, using the specified allocator.

8 *Complexity:* Linear in `n`.

```
template<class InputIterator>
```

```
    constexpr deque(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

9 *Effects:* Constructs a `deque` equal to the range `[first,last)`, using the specified allocator.

10 *Complexity:* Linear in `distance(first, last)`.

```
template<container-compatible-range<T> R>
```

```
    constexpr deque(from_range_t, R&& rg, const Allocator& = Allocator());
```

11 *Effects:* Constructs a `deque` with the elements of the range `rg`, using the specified allocator.

12 *Complexity:* Linear in `ranges::distance(rg)`.

23.3.5.3 Capacity

[deque.capacity]

```
constexpr void resize(size_type sz);
```

1 *Preconditions:* `T` is *Cpp17MoveInsertable* and *Cpp17DefaultInsertable* into `deque`.

2 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

```
constexpr void resize(size_type sz, const T& c);
```

3 *Preconditions:* `T` is *Cpp17CopyInsertable* into `deque`.

4 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

```
constexpr void shrink_to_fit();
```

5 *Preconditions:* `T` is *Cpp17MoveInsertable* into `deque`.

6 *Effects:* `shrink_to_fit` is a non-binding request to reduce memory use but does not change the size of the sequence.

[Note 1: The request is non-binding to allow latitude for implementation-specific optimizations. — end note]

If the size is equal to the old capacity, or if an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* `T`, then there are no effects.

7 *Complexity:* If the size is not equal to the old capacity, linear in the size of the sequence; otherwise constant.

8 *Remarks:* If the size is not equal to the old capacity, then invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator.

23.3.5.4 Modifiers

[deque.modifiers]

```
constexpr iterator insert(const_iterator position, const T& x);
```

```
constexpr iterator insert(const_iterator position, T&& x);
```

```
constexpr iterator insert(const_iterator position, size_type n, const T& x);
```

```

template<class InputIterator>
    constexpr iterator insert(const_iterator position,
                             InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);

template<class... Args> constexpr reference emplace_front(Args&&... args);
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr void push_front(const T& x);
constexpr void push_front(T&& x);
template<container-compatible-range<T> R>
    constexpr void prepend_range(R&& rg);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);

```

1 *Effects:* An insertion in the middle of the deque invalidates all the iterators and references to elements of the deque. An insertion at either end of the deque invalidates all the iterators to the deque, but has no effect on the validity of references to elements of the deque.

2 *Complexity:* The complexity is linear in the number of elements inserted plus the lesser of the distances to the beginning and end of the deque. Inserting a single element at either the beginning or end of a deque always takes constant time and causes a single call to a constructor of T.

3 *Remarks:* If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T, there are no effects. If an exception is thrown while inserting a single element at either end, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-*Cpp17CopyInsertable* T, the effects are unspecified.

```

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_front();
constexpr void pop_back();

```

4 *Effects:* An erase operation that erases the last element of a deque invalidates only the past-the-end iterator and all iterators and references to the erased elements. An erase operation that erases the first element of a deque but not the last element invalidates only iterators and references to the erased elements. An erase operation that erases neither the first element nor the last element of a deque invalidates the past-the-end iterator and all iterators and references to all the elements of the deque.

[Note 1: `pop_front` and `pop_back` are erase operations. — end note]

5 *Throws:* Nothing unless an exception is thrown by the assignment operator of T.

6 *Complexity:* The number of calls to the destructor of T is the same as the number of elements erased, but the number of calls to the assignment operator of T is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

23.3.5.5 Erasure

[deque.erasure]

```

template<class T, class Allocator, class U = T>
    constexpr typename deque<T, Allocator>::size_type
    erase(deque<T, Allocator>& c, const U& value);

```

1 *Effects:* Equivalent to:

```

    auto it = remove(c.begin(), c.end(), value);
    auto r = distance(it, c.end());
    c.erase(it, c.end());
    return r;

```

```

template<class T, class Allocator, class Predicate>
    constexpr typename deque<T, Allocator>::size_type
    erase_if(deque<T, Allocator>& c, Predicate pred);

```

2 *Effects:* Equivalent to:

```

    auto it = remove_if(c.begin(), c.end(), pred);
    auto r = distance(it, c.end());
    c.erase(it, c.end());
    return r;

```

23.3.6 Header <forward_list> synopsis

[forward.list.syn]

```

#include <compare>           // see 17.12.1
#include <initializer_list>   // see 17.11.2

namespace std {
    // 23.3.7, class template forward_list
    template<class T, class Allocator = allocator<T>> class forward_list;

    template<class T, class Allocator>
        constexpr bool operator==(const forward_list<T, Allocator>& x,
                                   const forward_list<T, Allocator>& y);
    template<class T, class Allocator>
        constexpr synth-three-way-result<T>
            operator<=>(const forward_list<T, Allocator>& x,
                       const forward_list<T, Allocator>& y);

    template<class T, class Allocator>
        constexpr void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));

    // 23.3.7.7, erasure
    template<class T, class Allocator, class U = T>
        constexpr typename forward_list<T, Allocator>::size_type
            erase(forward_list<T, Allocator>& c, const U& value);
    template<class T, class Allocator, class Predicate>
        constexpr typename forward_list<T, Allocator>::size_type
            erase_if(forward_list<T, Allocator>& c, Predicate pred);

    namespace pmr {
        template<class T>
            using forward_list = std::forward_list<T, polymorphic_allocator<T>>;
    }
}

```

23.3.7 Class template forward_list

[forward.list]

23.3.7.1 Overview

[forward.list.overview]

- ¹ A `forward_list` is a container that supports forward iterators and allows constant time insert and erase operations anywhere within the sequence, with storage management handled automatically. Fast random access to list elements is not supported.

[Note 1: It is intended that `forward_list` have zero space or time overhead relative to a hand-written C-style singly linked list. Features that would conflict with that goal have been omitted. — end note]

- ² A `forward_list` meets all of the requirements of a container (23.2.2.2), except that the `size()` member function is not provided and `operator==` has linear complexity. A `forward_list` also meets all of the requirements for an allocator-aware container (23.2.2.5). In addition, a `forward_list` provides the `assign` member functions and several of the optional sequence container requirements (23.2.4). Descriptions are provided here only for operations on `forward_list` that are not described in that table or for operations where there is additional semantic information.
- ³ [Note 2: Modifying any list requires access to the element preceding the first element of interest, but in a `forward_list` there is no constant-time way to access a preceding element. For this reason, `erase_after` and `splice_after` take fully-open ranges, not semi-open ranges. — end note]
- ⁴ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
        class forward_list {
        public:

```



```

// types
using value_type      = T;
using allocator_type  = Allocator;
using pointer         = allocator_traits<Allocator>::pointer;
using const_pointer   = allocator_traits<Allocator>::const_pointer;
using reference       = value_type&;
using const_reference = const value_type&;
using size_type       = implementation-defined; // see 23.2
using difference_type = implementation-defined; // see 23.2
using iterator        = implementation-defined; // see 23.2
using const_iterator  = implementation-defined; // see 23.2

// 23.3.7.2, construct/copy/destroy
constexpr forward_list() : forward_list(Allocator()) { }
constexpr explicit forward_list(const Allocator&);
constexpr explicit forward_list(size_type n, const Allocator& = Allocator());
constexpr forward_list(size_type n, const T& value, const Allocator& = Allocator());
template<class InputIterator>
    constexpr forward_list(InputIterator first, InputIterator last,
                           const Allocator& = Allocator());
template<container-compatible-range<T> R>
    constexpr forward_list(from_range_t, R&& rg, const Allocator& = Allocator());
constexpr forward_list(const forward_list& x);
constexpr forward_list(forward_list&& x);
constexpr forward_list(const forward_list& x, const type_identity_t<Allocator>&);
constexpr forward_list(forward_list&& x, const type_identity_t<Allocator>&);
constexpr forward_list(initializer_list<T>, const Allocator& = Allocator());
constexpr ~forward_list();
constexpr forward_list& operator=(const forward_list& x);
constexpr forward_list& operator=(forward_list&& x)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
constexpr forward_list& operator=(initializer_list<T>);
template<class InputIterator>
    constexpr void assign(InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr void assign_range(R&& rg);
constexpr void assign(size_type n, const T& t);
constexpr void assign(initializer_list<T>);
constexpr allocator_type get_allocator() const noexcept;

// 23.3.7.3, iterators
constexpr iterator before_begin() noexcept;
constexpr const_iterator before_begin() const noexcept;
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cbefore_begin() const noexcept;
constexpr const_iterator cend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.3.7.4, element access
constexpr reference front();
constexpr const_reference front() const;

// 23.3.7.5, modifiers
template<class... Args> constexpr reference emplace_front(Args&&... args);
constexpr void push_front(const T& x);
constexpr void push_front(T&& x);

```

```

template<container-compatible-range<T> R>
    constexpr void prepend_range(R&& rg);
constexpr void pop_front();

template<class... Args>
    constexpr iterator emplace_after(const_iterator position, Args&&... args);
constexpr iterator insert_after(const_iterator position, const T& x);
constexpr iterator insert_after(const_iterator position, T&& x);

constexpr iterator insert_after(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    constexpr iterator insert_after(const_iterator position,
                                   InputIterator first, InputIterator last);
constexpr iterator insert_after(const_iterator position, initializer_list<T> il);
template<container-compatible-range<T> R>
    constexpr iterator insert_range_after(const_iterator position, R&& rg);

constexpr iterator erase_after(const_iterator position);
constexpr iterator erase_after(const_iterator position, const_iterator last);
constexpr void swap(forward_list&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);

constexpr void resize(size_type sz);
constexpr void resize(size_type sz, const value_type& c);
constexpr void clear() noexcept;

// 23.3.7.6, forward_list operations
constexpr void splice_after(const_iterator position, forward_list& x);
constexpr void splice_after(const_iterator position, forward_list&& x);
constexpr void splice_after(const_iterator position, forward_list& x, const_iterator i);
constexpr void splice_after(const_iterator position, forward_list&& x, const_iterator i);
constexpr void splice_after(const_iterator position, forward_list& x,
                            const_iterator first, const_iterator last);
constexpr void splice_after(const_iterator position, forward_list&& x,
                            const_iterator first, const_iterator last);

constexpr size_type remove(const T& value);
template<class Predicate> constexpr size_type remove_if(Predicate pred);

constexpr size_type unique();
template<class BinaryPredicate> constexpr size_type unique(BinaryPredicate binary_pred);

constexpr void merge(forward_list& x);
constexpr void merge(forward_list&& x);
template<class Compare> constexpr void merge(forward_list& x, Compare comp);
template<class Compare> constexpr void merge(forward_list&& x, Compare comp);

constexpr void sort();
template<class Compare> constexpr void sort(Compare comp);

constexpr void reverse() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
    forward_list(InputIterator, InputIterator, Allocator = Allocator())
        -> forward_list<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
    forward_list(from_range_t, R&&, Allocator = Allocator())
        -> forward_list<ranges::range_value_t<R>, Allocator>;
}

```

- ⁵ An incomplete type `T` may be used when instantiating `forward_list` if the allocator meets the allocator completeness requirements (16.4.4.6.2). `T` shall be complete before any member of the resulting specialization of `forward_list` is referenced.

23.3.7.2 Constructors, copy, and assignment

[forward.list.cons]

```
constexpr explicit forward_list(const Allocator&);
```

- ¹ *Effects:* Constructs an empty `forward_list` object using the specified allocator.

- ² *Complexity:* Constant.

```
constexpr explicit forward_list(size_type n, const Allocator& = Allocator());
```

- ³ *Preconditions:* `T` is *Cpp17DefaultInsertable* into `forward_list`.

- ⁴ *Effects:* Constructs a `forward_list` object with `n` default-inserted elements using the specified allocator.

- ⁵ *Complexity:* Linear in `n`.

```
constexpr forward_list(size_type n, const T& value, const Allocator& = Allocator());
```

- ⁶ *Preconditions:* `T` is *Cpp17CopyInsertable* into `forward_list`.

- ⁷ *Effects:* Constructs a `forward_list` object with `n` copies of `value` using the specified allocator.

- ⁸ *Complexity:* Linear in `n`.

```
template<class InputIterator>
```

```
constexpr forward_list(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

- ⁹ *Effects:* Constructs a `forward_list` object equal to the range `[first, last)`.

- ¹⁰ *Complexity:* Linear in `distance(first, last)`.

```
template<container-compatible-range<T> R>
```

```
constexpr forward_list(from_range_t, R&& rg, const Allocator& = Allocator());
```

- ¹¹ *Effects:* Constructs a `forward_list` object with the elements of the range `rg`.

- ¹² *Complexity:* Linear in `ranges::distance(rg)`.

23.3.7.3 Iterators

[forward.list.iter]

```
constexpr iterator before_begin() noexcept;
```

```
constexpr const_iterator before_begin() const noexcept;
```

```
constexpr const_iterator cbefore_begin() const noexcept;
```

- ¹ *Effects:* `cbefore_begin()` is equivalent to `const_cast<forward_list const&>(*this).before_begin()`.

- ² *Returns:* A non-dereferenceable iterator that, when incremented, is equal to the iterator returned by `begin()`.

- ³ *Remarks:* `before_begin() == end()` shall equal `false`.

23.3.7.4 Element access

[forward.list.access]

```
constexpr reference front();
```

```
constexpr const_reference front() const;
```

- ¹ *Returns:* `*begin()`

23.3.7.5 Modifiers

[forward.list.modifiers]

- ¹ The member functions in this subclause do not affect the validity of iterators and references when inserting elements, and when erasing elements invalidate iterators and references to the erased elements only. If an exception is thrown by any of these member functions there is no effect on the container. Inserting `n` elements into a `forward_list` is linear in `n`, and the number of calls to the copy or move constructor of `T` is exactly equal to `n`. Erasing `n` elements from a `forward_list` is linear in `n` and the number of calls to the destructor of type `T` is exactly equal to `n`.

```

template<class... Args> constexpr reference emplace_front(Args&&... args);
2     Effects: Inserts an object of type value_type constructed with value_type(std::forward<Args>(args)...) at the beginning of the list.

constexpr void push_front(const T& x);
constexpr void push_front(T&& x);
3     Effects: Inserts a copy of x at the beginning of the list.

template<container-compatible-range<T> R>
    constexpr void prepend_range(R&& rg);
4     Effects: Inserts a copy of each element of rg at the beginning of the list.
    [Note 1: The order of elements is not reversed. — end note]

constexpr void pop_front();
5     Effects: As if by erase_after(before_begin()).

constexpr iterator insert_after(const_iterator position, const T& x);
6     Preconditions: T is Cpp17CopyInsertable into forward_list. position is before_begin() or is a
    dereferenceable iterator in the range [begin(), end()).
7     Effects: Inserts a copy of x after position.
8     Returns: An iterator pointing to the copy of x.

constexpr iterator insert_after(const_iterator position, T&& x);
9     Preconditions: T is Cpp17MoveInsertable into forward_list. position is before_begin() or is a
    dereferenceable iterator in the range [begin(), end()).
10    Effects: Inserts a copy of x after position.
11    Returns: An iterator pointing to the copy of x.

constexpr iterator insert_after(const_iterator position, size_type n, const T& x);
12    Preconditions: T is Cpp17CopyInsertable into forward_list. position is before_begin() or is a
    dereferenceable iterator in the range [begin(), end()).
13    Effects: Inserts n copies of x after position.
14    Returns: An iterator pointing to the last inserted copy of x, or position if n == 0 is true.

template<class InputIterator>
    constexpr iterator insert_after(const_iterator position,
                                   InputIterator first, InputIterator last);
15    Preconditions: T is Cpp17EmplaceConstructible into forward_list from *first. position is before_
    begin() or is a dereferenceable iterator in the range [begin(), end()). Neither first nor last are
    iterators in *this.
16    Effects: Inserts copies of elements in [first, last) after position.
17    Returns: An iterator pointing to the last inserted element, or position if first == last is true.

template<container-compatible-range<T> R>
    constexpr iterator insert_range_after(const_iterator position, R&& rg);
18    Preconditions: T is Cpp17EmplaceConstructible into forward_list from *ranges::begin(rg). posi-
    tion is before_begin() or is a dereferenceable iterator in the range [begin(), end()). rg and *this
    do not overlap.
19    Effects: Inserts copies of elements in the range rg after position.
20    Returns: An iterator pointing to the last inserted element, or position if rg is empty.

constexpr iterator insert_after(const_iterator position, initializer_list<T> il);
21    Effects: Equivalent to: return insert_after(position, il.begin(), il.end());

```

```

template<class... Args>
constexpr iterator emplace_after(const_iterator position, Args&&... args);
22     Preconditions: T is Cpp17EmplaceConstructible into forward_list from std::forward<Args>(args)... position is before_begin() or is a dereferenceable iterator in the range [begin(), end()).
23     Effects: Inserts an object of type value_type direct-non-list-initialized with std::forward<Args>(args)... after position.
24     Returns: An iterator pointing to the new object.

constexpr iterator erase_after(const_iterator position);
25     Preconditions: The iterator following position is dereferenceable.
26     Effects: Erases the element pointed to by the iterator following position.
27     Returns: An iterator pointing to the element following the one that was erased, or end() if no such element exists.
28     Throws: Nothing.

constexpr iterator erase_after(const_iterator position, const_iterator last);
29     Preconditions: All iterators in the range (position, last) are dereferenceable.
30     Effects: Erases the elements in the range (position, last).
31     Returns: last.
32     Throws: Nothing.

constexpr void resize(size_type sz);
33     Preconditions: T is Cpp17DefaultInsertable into forward_list.
34     Effects: If sz < distance(begin(), end()), erases the last distance(begin(), end()) - sz elements from the list. Otherwise, inserts sz - distance(begin(), end()) default-inserted elements at the end of the list.

constexpr void resize(size_type sz, const value_type& c);
35     Preconditions: T is Cpp17CopyInsertable into forward_list.
36     Effects: If sz < distance(begin(), end()), erases the last distance(begin(), end()) - sz elements from the list. Otherwise, inserts sz - distance(begin(), end()) copies of c at the end of the list.

constexpr void clear() noexcept;
37     Effects: Erases all elements in the range [begin(), end()).
38     Remarks: Does not invalidate past-the-end iterators.

```

23.3.7.6 Operations

[forward.list.ops]

- 1 In this subclause, arguments for a template parameter named *Predicate* or *BinaryPredicate* shall meet the corresponding requirements in 26.2. The semantics of *i + n*, where *i* is an iterator into the list and *n* is an integer, are the same as those of *next(i, n)*. The expression *i - n*, where *i* is an iterator into the list and *n* is an integer, means an iterator *j* such that *j + n == i* is true. For *merge* and *sort*, the definitions and requirements in 26.8 apply.

```

constexpr void splice_after(const_iterator position, forward_list& x);
constexpr void splice_after(const_iterator position, forward_list&& x);

```

- 2 *Preconditions:* *position* is *before_begin()* or is a dereferenceable iterator in the range [*begin()*, *end()*). *get_allocator() == x.get_allocator()* is true. *addressof(x) != this* is true.
- 3 *Effects:* Inserts the contents of *x* after *position*, and *x* becomes empty. Pointers and references to the moved elements of *x* now refer to those same elements but as members of **this*. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into **this*, not into *x*.
- 4 *Throws:* Nothing.

5 *Complexity:* $\mathcal{O}(\text{distance}(\text{x.begin()}, \text{x.end()}))$

```
constexpr void splice_after(const_iterator position, forward_list& x, const_iterator i);
constexpr void splice_after(const_iterator position, forward_list&& x, const_iterator i);
```

6 *Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. The iterator following `i` is a dereferenceable iterator in `x`. `get_allocator() == x.get_allocator()` is true.

7 *Effects:* Inserts the element following `i` into `*this`, following `position`, and removes it from `x`. The result is unchanged if `position == i` or `position == ++i`. Pointers and references to `*++i` continue to refer to the same element but as a member of `*this`. Iterators to `*++i` continue to refer to the same element, but now behave as iterators into `*this`, not into `x`.

8 *Throws:* Nothing.

9 *Complexity:* $\mathcal{O}(1)$

```
constexpr void splice_after(const_iterator position, forward_list& x,
                           const_iterator first, const_iterator last);
constexpr void splice_after(const_iterator position, forward_list&& x,
                           const_iterator first, const_iterator last);
```

10 *Preconditions:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(), end())`. `(first, last)` is a valid range in `x`, and all iterators in the range `(first, last)` are dereferenceable. `position` is not an iterator in the range `(first, last)`. `get_allocator() == x.get_allocator()` is true.

11 *Effects:* Inserts elements in the range `(first, last)` after `position` and removes the elements from `x`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

12 *Complexity:* $\mathcal{O}(\text{distance}(\text{first}, \text{last}))$

```
constexpr size_type remove(const T& value);
template<class Predicate> constexpr size_type remove_if(Predicate pred);
```

13 *Effects:* Erases all the elements in the list referred to by a list iterator `i` for which the following conditions hold: `*i == value` (for `remove()`), `pred(*i)` is true (for `remove_if()`). Invalidates only the iterators and references to the erased elements.

14 *Returns:* The number of elements erased.

15 *Throws:* Nothing unless an exception is thrown by the equality comparison or the predicate.

16 *Complexity:* Exactly `distance(begin(), end())` applications of the corresponding predicate.

17 *Remarks:* Stable (16.4.6.8).

```
constexpr size_type unique();
template<class BinaryPredicate> constexpr size_type unique(BinaryPredicate binary_pred);
```

18 Let `binary_pred` be `equal_to<>{}` for the first overload.

19 *Preconditions:* `binary_pred` is an equivalence relation.

20 *Effects:* Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty list, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is true. Invalidates only the iterators and references to the erased elements.

21 *Returns:* The number of elements erased.

22 *Throws:* Nothing unless an exception is thrown by the predicate.

23 *Complexity:* If `empty()` is false, exactly `distance(begin(), end()) - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
constexpr void merge(forward_list& x);
constexpr void merge(forward_list&& x);
template<class Compare> constexpr void merge(forward_list& x, Compare comp);
```

```
template<class Compare> constexpr void merge(forward_list&& x, Compare comp);
```

24 Let `comp` be `less<>` for the first two overloads.

25 *Preconditions:* `*this` and `x` are both sorted with respect to the comparator `comp`, and `get_allocator() == x.get_allocator()` is true.

26 *Effects:* If `addressof(x) == this`, there are no effects. Otherwise, merges the two sorted ranges `[begin(), end())` and `[x.begin(), x.end())`. The result is a range that is sorted with respect to the comparator `comp`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

27 *Complexity:* At most `distance(begin(), end()) + distance(x.begin(), x.end()) - 1` comparisons if `addressof(x) != this`; otherwise, no comparisons are performed.

28 *Remarks:* Stable (16.4.6.8). If `addressof(x) != this`, `x` is empty after the merge. No elements are copied by this operation. If an exception is thrown other than by a comparison, there are no effects.

```
constexpr void sort();
```

```
template<class Compare> constexpr void sort(Compare comp);
```

29 *Effects:* Sorts the list according to the `operator<` or the `comp` function object. If an exception is thrown, the order of the elements in `*this` is unspecified. Does not affect the validity of iterators and references.

30 *Complexity:* Approximately $N \log N$ comparisons, where N is `distance(begin(), end())`.

31 *Remarks:* Stable (16.4.6.8).

```
constexpr void reverse() noexcept;
```

32 *Effects:* Reverses the order of the elements in the list. Does not affect the validity of iterators and references.

33 *Complexity:* Linear time.

23.3.7.7 Erasure

[forward.list.erase]

```
template<class T, class Allocator, class U = T>
```

```
constexpr typename forward_list<T, Allocator>::size_type
erase(forward_list<T, Allocator>& c, const U& value);
```

1 *Effects:* Equivalent to:

```
return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });
```

```
template<class T, class Allocator, class Predicate>
```

```
constexpr typename forward_list<T, Allocator>::size_type
erase_if(forward_list<T, Allocator>& c, Predicate pred);
```

2 *Effects:* Equivalent to: `return c.remove_if(pred);`

23.3.8 Header <hive> synopsis

[hive.syn]

```
#include <initializer_list>    // see 17.11.2
#include <compare>              // see 17.12.1
```

```
namespace std {
    struct hive_limits {
        size_t min;
        size_t max;
        constexpr hive_limits(size_t minimum, size_t maximum) noexcept
            : min(minimum), max(maximum) {}
    };
```

```
// 23.3.9, class template hive
```

```
template<class T, class Allocator = allocator<T>> class hive;
```



```

template<class T, class Allocator>
void swap(hive<T, Allocator>& x, hive<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template<class T, class Allocator, class U = T>
typename hive<T, Allocator>::size_type
erase(hive<T, Allocator>& c, const U& value);

template<class T, class Allocator, class Predicate>
typename hive<T, Allocator>::size_type
erase_if(hive<T, Allocator>& c, Predicate pred);

namespace pmr {
    template<class T>
        using hive = std::hive<T, polymorphic_allocator<T>>;
}

```

23.3.9 Class template `hive`

[hive]

23.3.9.1 Overview

[hive.overview]

- ¹ A `hive` is a type of sequence container that provides constant-time insertion and erasure operations. Storage is automatically managed in multiple memory blocks, referred to as *element blocks*. Insertion (23.3.9.4) position is determined by the container, and insertion may re-use the memory locations of erased elements. [Note 1: Construction and assignment are not considered to involve insertion operations. — end note]
- ² Element blocks which contain elements are referred to as *active blocks*, those which do not are referred to as *reserved blocks*. Active blocks which become empty of elements are either deallocated or become reserved blocks. Reserved blocks become active blocks when they are used to store elements. A user can create additional reserved blocks by calling `reserve`.
- ³ Erasures use unspecified techniques of constant time complexity to identify the memory locations of erased elements, which are subsequently skipped during iteration, as opposed to relocating subsequent elements during erasure.
- ⁴ Active block capacities have an implementation-defined growth factor (which need not be integral), for example a new active block's capacity could be equal to the summed capacities of the pre-existing active blocks.
- ⁵ Limits can be placed on both the minimum and maximum element capacities of element blocks, both by users and implementations.
 - (5.1) — The minimum limit shall be no larger than the maximum limit.
 - (5.2) — When limits are not specified by a user during construction, the implementation's default limits are used.
 - (5.3) — The default limits of an implementation are not guaranteed to be the same as the minimum and maximum possible capacities for an implementation's element blocks.

[Note 2: To allow latitude for both implementation-specific and user-directed optimization. — end note]

The latter are defined as hard limits. The maximum hard limit shall be no larger than `std::allocator_traits<Allocator>::max_size()`.
 - (5.4) — If user-specified limits passed to a `hive` constructor or `reshape` are not within hard limits, or if the specified minimum limit is greater than the specified maximum limit, the behavior is erroneous and the effects are implementation-defined.

[Note 3: This condition can be checked using `is_within_hard_limits`. — end note]
 - (5.5) — An element block is said to be *within the bounds* of a pair of minimum/maximum limits when its capacity is greater-or-equal-to the minimum limit and less-than-or-equal-to the maximum limit.
- ⁶ A `hive` conforms to the requirements for containers (23.2.2.2), with the exception of operators `==` and `!=`. A `hive` also meets the requirements of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and some of the requirements of a sequence container (23.2.4). Descriptions are provided here only for operations on `hive` that are not described in that table or for operations where there is additional semantic information.

- ⁷ The iterators of `hive` meet the *Cpp17BidirectionalIterator* requirements but also model `three_way_comparable<strong_ordering>`.

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class hive {
    public:
        // types
        using value_type = T;
        using allocator_type = Allocator;
        using pointer = allocator_traits<Allocator>::pointer;
        using const_pointer = allocator_traits<Allocator>::const_pointer;
        using reference = value_type&;
        using const_reference = const value_type&;
        using size_type = implementation-defined; // see 23.2
        using difference_type = implementation-defined; // see 23.2
        using iterator = implementation-defined; // see 23.2
        using const_iterator = implementation-defined; // see 23.2
        using reverse_iterator = std::reverse_iterator<iterator>; // see 23.2
        using const_reverse_iterator = std::reverse_iterator<const_iterator>; // see 23.2

        // 23.3.9.2, construct/copy/destroy
        constexpr hive() noexcept(noexcept(Allocator())) : hive(Allocator()) {}
        constexpr explicit hive(const Allocator&) noexcept;
        constexpr explicit hive(hive_limits block_limits) : hive(block_limits, Allocator()) {}
        constexpr hive(hive_limits block_limits, const Allocator&);
        explicit hive(size_type n, const Allocator& = Allocator());
        hive(size_type n, hive_limits block_limits, const Allocator& = Allocator());
        hive(size_type n, const T& value, const Allocator& = Allocator());
        hive(size_type n, const T& value, hive_limits block_limits, const Allocator& = Allocator());
        template<class InputIterator>
            hive(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<class InputIterator>
            hive(InputIterator first, InputIterator last, hive_limits block_limits,
                const Allocator& = Allocator());
        template<container-compatible-range<T> R>
            hive(from_range_t, R&& rg, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
            hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& = Allocator());
        hive(const hive& x);
        hive(hive&&) noexcept;
        hive(const hive& x, const type_identity_t<Allocator>& alloc);
        hive(hive&&, const type_identity_t<Allocator>& alloc);
        hive(initializer_list<T> il, const Allocator& = Allocator());
        hive(initializer_list<T> il, hive_limits block_limits, const Allocator& = Allocator());
        ~hive();

        hive& operator=(const hive& x);
        hive& operator=(hive&& x) noexcept(see below);
        hive& operator=(initializer_list<T>);
        template<class InputIterator>
            void assign(InputIterator first, InputIterator last);
        template<container-compatible-range<T> R>
            void assign_range(R&& rg);
        void assign(size_type n, const T& t);
        void assign(initializer_list<T>);
        allocator_type get_allocator() const noexcept;

        // iterators
        iterator          begin() noexcept;
        const_iterator    begin() const noexcept;
        iterator          end() noexcept;
        const_iterator    end() const noexcept;
        reverse_iterator  rbegin() noexcept;

```

```

const_reverse_iterator rbegin() const noexcept;
reverse_iterator      rend() noexcept;
const_reverse_iterator rend() const noexcept;
const_iterator        cbegin() const noexcept;
const_iterator        cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// 23.3.9.3, capacity
bool empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
size_type capacity() const noexcept;
void reserve(size_type n);
void shrink_to_fit();
void trim_capacity() noexcept;
void trim_capacity(size_type n) noexcept;
constexpr hive_limits block_capacity_limits() const noexcept;
static constexpr hive_limits block_capacity_default_limits() noexcept;
static constexpr hive_limits block_capacity_hard_limits() noexcept;
static constexpr bool is_within_hard_limits(hive_limits) noexcept;
void reshape(hive_limits block_limits);

// 23.3.9.4, modifiers
template<class... Args> iterator emplace(Args&&... args);
template<class... Args> iterator emplace_hint(const_iterator hint, Args&&... args);
iterator insert(const T& x);
iterator insert(T&& x);
iterator insert(const_iterator hint, const T& x);
iterator insert(const_iterator hint, T&& x);
void insert(initializer_list<T> il);
template<container-compatible-range<T> R>
    void insert_range(R&& rg);
template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
void insert(size_type n, const T& x);

iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void swap(hive&) noexcept(see below);
void clear() noexcept;

// 23.3.9.5, hive operations
void splice(hive& x);
void splice(hive&& x);
template<class BinaryPredicate = equal_to<T>>
    size_type unique(BinaryPredicate binary_pred = BinaryPredicate());

template<class Compare = less<T>>
    void sort(Compare comp = Compare());

iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;

private:
    hive_limits current_limits = implementation-defined;    // exposition only
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
    hive(InputIterator, InputIterator, Allocator = Allocator())
    -> hive<iter-value-type<InputIterator>, Allocator>;

```

```

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
    hive(InputIterator, InputIterator, hive_limits, Allocator = Allocator())
    -> hive<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, Allocator = Allocator())
    -> hive<ranges::range_value_t<R>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
    hive(from_range_t, R&&, hive_limits, Allocator = Allocator())
    -> hive<ranges::range_value_t<R>, Allocator>;
}

```

23.3.9.2 Constructors, copy, and assignment

[hive.cons]

```
constexpr explicit hive(const Allocator&) noexcept;
```

1 *Effects:* Constructs an empty hive, using the specified allocator.

2 *Complexity:* Constant.

```
constexpr hive(hive_limits block_limits, const Allocator&);
```

3 *Effects:* Constructs an empty hive, using the specified allocator. Initializes *current-limits* with *block_limits*.

4 *Complexity:* Constant.

```
explicit hive(size_type n, const Allocator& = Allocator());
```

```
hive(size_type n, hive_limits block_limits, const Allocator& = Allocator());
```

5 *Preconditions:* T is *Cpp17DefaultInsertable* into hive.

6 *Effects:* Constructs a hive with n default-inserted elements, using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_limits*.

7 *Complexity:* Linear in n.

```
hive(size_type n, const T& value, const Allocator& = Allocator());
```

```
hive(size_type n, const T& value, hive_limits block_limits, const Allocator& = Allocator());
```

8 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

9 *Effects:* Constructs a hive with n copies of value, using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_limits*.

10 *Complexity:* Linear in n.

```
template<class InputIterator>
```

```
    hive(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

```
template<class InputIterator>
```

```
    hive(InputIterator first, InputIterator last, hive_limits block_limits,
        const Allocator& = Allocator());
```

11 *Effects:* Constructs a hive equal to the range [first, last), using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_limits*.

12 *Complexity:* Linear in distance(first, last).

```
template<container-compatible-range<T> R>
```

```
    hive(from_range_t, R&& rg, const Allocator& = Allocator());
```

```
template<container-compatible-range<T> R>
```

```
    hive(from_range_t, R&& rg, hive_limits block_limits, const Allocator& = Allocator());
```

13 *Effects:* Constructs a hive object equal to the range rg, using the specified allocator. If the second overload is called, also initializes *current-limits* with *block_limits*.

14 *Complexity:* Linear in ranges::distance(rg).

```
hive(const hive& x);
```

```
hive(const hive& x, const type_identity_t<Allocator>& alloc);
```

15 *Preconditions:* T is *Cpp17CopyInsertable* into hive.

16 *Effects:* Constructs a `hive` object equal to `x`. If the second overload is called, uses `alloc`. Initializes *current-limits* with `x.current-limits`.

17 *Complexity:* Linear in `x.size()`.

```
hive(hive&& x) noexcept;
hive(hive&& x, const type_identity_t<Allocator>& alloc);
```

18 *Preconditions:* For the second overload, when `allocator_traits<Allocator>::is_always_equal::value` is `false`, `T` meets the *Cpp17MoveInsertable* requirements.

19 *Effects:* When the first overload is called, or the second overload is called and `alloc == x.get_allocator()` is `true`, *current-limits* is set to `x.current-limits` and each element block is moved from `x` into `*this`. Pointers and references to the elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the elements of `x` will continue to refer to their elements, but they now behave as iterators into `*this`.

If the second overload is called and `alloc == x.get_allocator()` is `false`, each element in `x` is moved into `*this`. References, pointers and iterators referring to the elements of `x`, as well as the past-the-end iterator of `x`, are invalidated.

20 *Postconditions:* `x.empty()` is `true`. The relative order of the elements of `*this` is the same as that of the elements of `x` prior to the call.

21 *Complexity:* If the second overload is called and `alloc == x.get_allocator()` is `false`, linear in `x.size()`. Otherwise constant.

```
hive(initializer_list<T> il, const Allocator& = Allocator());
hive(initializer_list<T> il, hive_limits block_limits, const Allocator& = Allocator());
```

22 *Preconditions:* `T` is *Cpp17CopyInsertable* into `hive`.

23 *Effects:* Constructs a `hive` object equal to `il`, using the specified allocator. If the second overload is called, also initializes *current-limits* with `block_limits`.

24 *Complexity:* Linear in `il.size()`.

```
hive& operator=(const hive& x);
```

25 *Preconditions:* `T` is *Cpp17CopyInsertable* into `hive` and *Cpp17CopyAssignable*.

26 *Effects:* All elements in `*this` are either copy-assigned to, or destroyed. All elements in `x` are copied into `*this`, maintaining their relative order.

[Note 1: *current-limits* is unchanged. — end note]

27 *Complexity:* Linear in `size() + x.size()`.

```
hive& operator=(hive&& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
         allocator_traits<Allocator>::is_always_equal::value);
```

28 *Preconditions:* When

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 allocator_traits<Allocator>::is_always_equal::value)
```

is `false`, `T` is *Cpp17MoveInsertable* into `hive` and *Cpp17MoveAssignable*.

29 *Effects:* Each element in `*this` is either move-assigned to, or destroyed. When

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 get_allocator() == x.get_allocator())
```

is `true`, *current-limits* is set to `x.current-limits` and each element block is moved from `x` into `*this`. Pointers and references to the elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the elements of `x` will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

When

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 get_allocator() == x.get_allocator())
```

is **false**, each element in **x** is moved into ***this**. References, pointers and iterators referring to the elements of **x**, as well as the past-the-end iterator of **x**, are invalidated.

30 *Postconditions:* **x.empty()** is **true**. The relative order of the elements of ***this** is the same as that of the elements of **x** prior to this call.

31 *Complexity:* Linear in **size()**. If

```
(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
  get_allocator() == x.get_allocator())
```

is **false**, also linear in **x.size()**.

23.3.9.3 Capacity

[hive.capacity]

```
size_type capacity() const noexcept;
```

1 *Returns:* The total number of elements that ***this** can hold without requiring allocation of more element blocks.

2 *Complexity:* Constant.

```
void reserve(size_type n);
```

3 *Effects:* If **n <= capacity()** is **true**, there are no effects. Otherwise increases **capacity()** by allocating reserved blocks.

4 *Postconditions:* **capacity() >= n** is **true**.

5 *Throws:* **length_error** if satisfying the postcondition would cause **capacity()** to exceed **max_size()**, as well as any exceptions thrown by the allocator.

6 *Complexity:* Linear in the number of reserved blocks allocated.

7 *Remarks:* All references, pointers, and iterators referring to elements in ***this**, as well as the past-the-end iterator, remain valid.

```
void shrink_to_fit();
```

8 *Preconditions:* **T** is *Cpp17MoveInsertable* into **hive**.

9 *Effects:* **shrink_to_fit** is a non-binding request to reduce **capacity()** to be closer to **size()**.

[Note 1: The request is non-binding to allow latitude for implementation-specific optimizations. — end note]

It does not increase **capacity()**, but may reduce **capacity()**. It may reallocate elements. If **capacity()** is already equal to **size()**, there are no effects. If an exception is thrown during allocation of a new element block, **capacity()** may be reduced and reallocation may occur. Otherwise if an exception is thrown, the effects are unspecified.

10 *Complexity:* If reallocation happens, linear in the size of the sequence.

11 *Remarks:* If reallocation happens, the order of the elements in ***this** may change and all references, pointers, and iterators referring to the elements in ***this**, as well as the past-the-end iterator, are invalidated.

```
void trim_capacity() noexcept;
```

```
void trim_capacity(size_type n) noexcept;
```

12 *Effects:* For the first overload, all reserved blocks are deallocated, and **capacity()** is reduced accordingly. For the second overload, if **n >= capacity()** is **true**, there are no effects; otherwise, **capacity()** is reduced to no less than **n**.

13 *Complexity:* Linear in the number of reserved blocks deallocated.

14 *Remarks:* All references, pointers, and iterators referring to elements in ***this**, as well as the past-the-end iterator, remain valid.

```
constexpr hive_limits block_capacity_limits() const noexcept;
```

15 *Returns:* *current-limits*.

16 *Complexity:* Constant.

```
static constexpr hive_limits block_capacity_default_limits() noexcept;
```

17 *Returns:* A `hive_limits` struct with the `min` and `max` members set to the implementation's default limits.

18 *Complexity:* Constant.

```
static constexpr hive_limits block_capacity_hard_limits() noexcept;
```

19 *Returns:* A `hive_limits` struct with the `min` and `max` members set to the implementation's hard limits.

20 *Complexity:* Constant.

```
static constexpr bool is_within_hard_limits(hive_limits lim) noexcept;
```

21 Let `hl` be `block_capacity_hard_limits()`.

22 *Returns:* `hl.min <= lim.min && lim.min <= lim.max && lim.max <= hl.max`.

```
void reshape(hive_limits block_limits);
```

23 *Preconditions:* `T` is *Cpp17MoveInsertable* into `hive`.

24 *Effects:* For any active blocks not within the bounds of `block_limits`, the elements within those active blocks are reallocated to new or existing element blocks which are within the bounds. Any element blocks not within the bounds of `block_limits` are deallocated. If an exception is thrown during allocation of a new element block, `capacity()` may be reduced, reallocation may occur, and *current-limits* may be assigned a value other than `block_limits`. Otherwise `block_limits` is assigned to *current-limits*. If any other exception is thrown the effects are unspecified.

25 *Postconditions:* `size()` is unchanged.

26 *Complexity:* Linear in the number of element blocks in `*this`. If reallocation happens, also linear in the number of elements reallocated.

27 *Remarks:* This operation may change `capacity()`. If reallocation happens, the order of the elements in `*this` may change. Reallocation invalidates all references, pointers, and iterators referring to the elements in `*this`, as well as the past-the-end iterator.

[Note 2: If no reallocation happens, they remain valid. — end note]

23.3.9.4 Modifiers

[hive.modifiers]

```
template<class... Args> iterator emplace(Args&&... args);
```

```
template<class... Args> iterator emplace_hint(const_iterator hint, Args&&... args);
```

1 *Preconditions:* `T` is *Cpp17EmplaceConstructible* into `hive` from `args`.

2 *Effects:* Inserts an object of type `T` constructed with `std::forward<Args>(args)...`. The `hint` parameter is ignored. If an exception is thrown, there are no effects.

[Note 1: `args` can directly or indirectly refer to a value in `*this`. — end note]

3 *Returns:* An iterator that points to the new element.

4 *Complexity:* Constant. Exactly one object of type `T` is constructed.

5 *Remarks:* Invalidates the past-the-end iterator.

```
iterator insert(const T& x);
```

```
iterator insert(const_iterator hint, const T& x);
```

```
iterator insert(T&& x);
```

```
iterator insert(const_iterator hint, T&& x);
```

6 *Effects:* Equivalent to: `return emplace(std::forward<decltype(x)>(x));`

[Note 2: The `hint` parameter is ignored. — end note]

```
void insert(initializer_list<T> rg);
```

```
template<container-compatible-range<T> R>
```

```
void insert_range(R&& rg);
```

7 *Preconditions:* `T` is *Cpp17EmplaceInsertable* into `hive` from `*ranges::begin(rg)`. `rg` and `*this` do not overlap.

8 *Effects:* Inserts copies of elements in `rg`. Each iterator in the range `rg` is dereferenced exactly once.

9 *Complexity:* Linear in the number of elements inserted. Exactly one object of type `T` is constructed for each element inserted.

10 *Remarks:* If an element is inserted, invalidates the past-the-end iterator.

```
void insert(size_type n, const T& x);
```

11 *Preconditions:* `T` is *Cpp17CopyInsertable* into `hive`.

12 *Effects:* Inserts `n` copies of `x`.

13 *Complexity:* Linear in `n`. Exactly one object of type `T` is constructed for each element inserted.

14 *Remarks:* If an element is inserted, invalidates the past-the-end iterator.

```
template<class InputIterator>
void insert(InputIterator first, InputIterator last);
```

15 *Effects:* Equivalent to `insert_range(ranges::subrange(first, last))`.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
```

16 *Complexity:* Linear in the number of elements erased. Additionally, if any active blocks become empty of elements as a result of the function call, at worst linear in the number of element blocks.

17 *Remarks:* Invalidates references, pointers and iterators referring to the erased elements. An erase operation that erases the last element in `*this` also invalidates the past-the-end iterator.

```
void swap(hive& x)
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

18 *Effects:* Exchanges the contents, `capacity()`, and *current-limits* of `*this` with that of `x`.

19 *Complexity:* Constant.

23.3.9.5 Operations

[hive.operations]

1 In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in 26.2. The semantics of `i + n` and `i - n`, where `i` is an iterator into the `hive` and `n` is an integer, are the same as those of `next(i, n)` and `prev(i, n)`, respectively. For `sort`, the definitions and requirements in 26.8 apply.

```
void splice(hive& x);
void splice(hive&& x);
```

2 *Preconditions:* `get_allocator() == x.get_allocator()` is true.

3 *Effects:* If `addressof(x) == this` is true, the behavior is erroneous and there are no effects. If an exception is thrown, there are no effects. Otherwise, inserts the contents of `x` into `*this` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

4 *Throws:* `length_error` if any of `x`'s active blocks are not within the bounds of *current-limits*, as well as any exceptions thrown by the allocator.

5 *Complexity:* Linear in the sum of all element blocks in `x` plus all element blocks in `*this`.

6 *Remarks:* Reserved blocks in `x` are not transferred into `*this`. If `addressof(x) == this` is false, invalidates the past-the-end iterator for both `x` and `*this`.

```
template<class BinaryPredicate = equal_to<T>>
size_type unique(BinaryPredicate binary_pred = BinaryPredicate());
```

7 *Preconditions:* `binary_pred` is an equivalence relation.

8 *Effects:* Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty `hive`, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is true.

9 *Returns:* The number of elements erased.

- 10 *Throws:* Nothing unless an exception is thrown by the predicate.
- 11 *Complexity:* If `empty()` is false, exactly `size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.
- 12 *Remarks:* Invalidates references, pointers, and iterators referring to the erased elements. If the last element in `*this` is erased, also invalidates the past-the-end iterator.

```
template<class Compare = less<T>>
void sort(Compare comp = Compare());
```

- 13 *Preconditions:* `T` is *Cpp17MoveInsertable* into *hive*, *Cpp17MoveAssignable*, and *Cpp17Swappable*.
- 14 *Effects:* Sorts `*this` according to the `comp` function object. If an exception is thrown, the order of the elements in `*this` is unspecified.
- 15 *Complexity:* $\mathcal{O}(N \log N)$ comparisons, where N is `size()`.
- 16 *Remarks:* May allocate. References, pointers, and iterators referring to elements in `*this`, as well as the past-the-end iterator, may be invalidated.
- [Note 1: Not required to be stable (16.4.6.8). — end note]

```
iterator get_iterator(const_pointer p) noexcept;
const_iterator get_iterator(const_pointer p) const noexcept;
```

- 17 *Preconditions:* `p` points to an element in `*this`.
- 18 *Returns:* An iterator or `const_iterator` pointing to the same element as `p`.
- 19 *Complexity:* Linear in the number of active blocks in `*this`.

23.3.9.6 Erasure

[hive.erase]

```
template<class T, class Allocator, class U = T>
typename hive<T, Allocator>::size_type
erase(hive<T, Allocator>& c, const U& value);
```

- 1 *Effects:* Equivalent to:
- ```
return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });
```

```
template<class T, class Allocator, class Predicate>
typename hive<T, Allocator>::size_type
erase_if(hive<T, Allocator>& c, Predicate pred);
```

- 2 *Effects:* Equivalent to:
- ```
auto original_size = c.size();
for (auto i = c.begin(); i != c.end(); ) {
    if (pred(*i)) {
        i = c.erase(i);
    } else {
        ++i;
    }
}
return original_size - c.size();
```

23.3.10 Header <list> synopsis

[list.syn]

```
#include <compare>           // see 17.12.1
#include <initializer_list>   // see 17.11.2
```

```
namespace std {
    // 23.3.11, class template list
    template<class T, class Allocator = allocator<T>> class list;

    template<class T, class Allocator>
        constexpr bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
    template<class T, class Allocator>
        constexpr synth-three-way-result<T>
            operator<=>(const list<T, Allocator>& x, const list<T, Allocator>& y);
```



```

template<class T, class Allocator>
constexpr void swap(list<T, Allocator>& x, list<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

// 23.3.11.6, erasure
template<class T, class Allocator, class U = T>
constexpr typename list<T, Allocator>::size_type
    erase(list<T, Allocator>& c, const U& value);
template<class T, class Allocator, class Predicate>
constexpr typename list<T, Allocator>::size_type
    erase_if(list<T, Allocator>& c, Predicate pred);

namespace pmr {
    template<class T>
        using list = std::list<T, polymorphic_allocator<T>>;
}
}

```

23.3.11 Class template list

[list]

23.3.11.1 Overview

[list.overview]

- ¹ A `list` is a sequence container that supports bidirectional iterators and allows constant time insert and erase operations anywhere within the sequence, with storage management handled automatically. Unlike vectors (23.3.13) and deques (23.3.5), fast random access to list elements is not supported, but many algorithms only need sequential access anyway.
- ² A `list` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of a sequence container, including most of the optional sequence container requirements (23.2.4). The exceptions are the `operator[]` and `at` member functions, which are not provided.¹⁷⁴ Descriptions are provided here only for operations on `list` that are not described in one of these tables or for operations where there is additional semantic information.
- ³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class list {
    public:
        // types
        using value_type          = T;
        using allocator_type      = Allocator;
        using pointer             = allocator_traits<Allocator>::pointer;
        using const_pointer       = allocator_traits<Allocator>::const_pointer;
        using reference           = value_type&;
        using const_reference     = const value_type&;
        using size_type           = implementation-defined; // see 23.2
        using difference_type     = implementation-defined; // see 23.2
        using iterator            = implementation-defined; // see 23.2
        using const_iterator      = implementation-defined; // see 23.2
        using reverse_iterator    = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 23.3.11.2, construct/copy/destroy
        constexpr list() : list(Allocator()) { }
        constexpr explicit list(const Allocator&);
        constexpr explicit list(size_type n, const Allocator& = Allocator());
        constexpr list(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
            constexpr list(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<T> R>
            constexpr list(from_range_t, R&& rg, const Allocator& = Allocator());
        constexpr list(const list& x);
        constexpr list(list&& x);
    };
}

```

¹⁷⁴ These member functions are only provided by containers whose iterators are random access iterators.

```

constexpr list(const list&, const type_identity_t<Allocator>&);
constexpr list(list&&, const type_identity_t<Allocator>&);
constexpr list(initializer_list<T>, const Allocator& = Allocator());
constexpr ~list();
constexpr list& operator=(const list& x);
constexpr list& operator=(list&& x)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
constexpr list& operator=(initializer_list<T>);
template<class InputIterator>
    constexpr void assign(InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr void assign_range(R&& rg);
constexpr void assign(size_type n, const T& t);
constexpr void assign(initializer_list<T>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator          begin() noexcept;
constexpr const_iterator    begin() const noexcept;
constexpr iterator          end() noexcept;
constexpr const_iterator    end() const noexcept;
constexpr reverse_iterator  rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator  rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator    cbegin() const noexcept;
constexpr const_iterator    cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 23.3.11.3, capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr void      resize(size_type sz);
constexpr void      resize(size_type sz, const T& c);

// element access
constexpr reference      front();
constexpr const_reference front() const;
constexpr reference      back();
constexpr const_reference back() const;

// 23.3.11.4, modifiers
template<class... Args> constexpr reference emplace_front(Args&&... args);
template<class... Args> constexpr reference emplace_back(Args&&... args);
constexpr void push_front(const T& x);
constexpr void push_front(T&& x);
template<container-compatible-range<T> R>
    constexpr void prepend_range(R&& rg);
constexpr void pop_front();
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);
constexpr void pop_back();

template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);

```

```

template<class InputIterator>
    constexpr iterator insert(const_iterator position,
                             InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T> il);

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator position, const_iterator last);
constexpr void      swap(list&) noexcept(allocator_traits<Allocator>::is_always_equal::value);
constexpr void      clear() noexcept;

// 23.3.11.5, list operations
constexpr void splice(const_iterator position, list& x);
constexpr void splice(const_iterator position, list&& x);
constexpr void splice(const_iterator position, list& x, const_iterator i);
constexpr void splice(const_iterator position, list&& x, const_iterator i);
constexpr void splice(const_iterator position, list& x,
                     const_iterator first, const_iterator last);
constexpr void splice(const_iterator position, list&& x,
                     const_iterator first, const_iterator last);

constexpr size_type remove(const T& value);
template<class Predicate> constexpr size_type remove_if(Predicate pred);

constexpr size_type unique();
template<class BinaryPredicate>
    constexpr size_type unique(BinaryPredicate binary_pred);

constexpr void merge(list& x);
constexpr void merge(list&& x);
template<class Compare> constexpr void merge(list& x, Compare comp);
template<class Compare> constexpr void merge(list&& x, Compare comp);

constexpr void sort();
template<class Compare> constexpr void sort(Compare comp);

constexpr void reverse() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
    list(InputIterator, InputIterator, Allocator = Allocator())
    -> list<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
    list(from_range_t, R&&, Allocator = Allocator())
    -> list<ranges::range_value_t<R>, Allocator>;
}

```

- ⁴ An incomplete type T may be used when instantiating `list` if the allocator meets the allocator completeness requirements (16.4.4.6.2). T shall be complete before any member of the resulting specialization of `list` is referenced.

23.3.11.2 Constructors, copy, and assignment

[list.cons]

```
constexpr explicit list(const Allocator&);
```

- ¹ *Effects:* Constructs an empty list, using the specified allocator.

- ² *Complexity:* Constant.

```
constexpr explicit list(size_type n, const Allocator& = Allocator());
```

- ³ *Preconditions:* T is `Cpp17DefaultInsertable` into list.

- ⁴ *Effects:* Constructs a list with n default-inserted elements using the specified allocator.

5 *Complexity:* Linear in *n*.

```
constexpr list(size_type n, const T& value, const Allocator& = Allocator());
```

6 *Preconditions:* *T* is *Cpp17CopyInsertable* into *list*.

7 *Effects:* Constructs a *list* with *n* copies of *value*, using the specified allocator.

8 *Complexity:* Linear in *n*.

```
template<class InputIterator>
constexpr list(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

9 *Effects:* Constructs a *list* equal to the range *[first, last)*.

10 *Complexity:* Linear in *distance(first, last)*.

```
template<container-compatible-range<T> R>
constexpr list(from_range_t, R&& rg, const Allocator& = Allocator());
```

11 *Effects:* Constructs a *list* object with the elements of the range *rg*.

12 *Complexity:* Linear in *ranges::distance(rg)*.

23.3.11.3 Capacity

[list.capacity]

```
constexpr void resize(size_type sz);
```

1 *Preconditions:* *T* is *Cpp17DefaultInsertable* into *list*.

2 *Effects:* If *size() < sz*, appends *sz - size()* default-inserted elements to the sequence. If *sz <= size()*, equivalent to:

```
list<T>::iterator it = begin();
advance(it, sz);
erase(it, end());
```

```
constexpr void resize(size_type sz, const T& c);
```

3 *Preconditions:* *T* is *Cpp17CopyInsertable* into *list*.

4 *Effects:* As if by:

```
if (sz > size())
    insert(end(), sz-size(), c);
else if (sz < size()) {
    iterator i = begin();
    advance(i, sz);
    erase(i, end());
}
else
    ; // do nothing
```

23.3.11.4 Modifiers

[list.modifiers]

```
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
```

```
template<class InputIterator>
constexpr iterator insert(const_iterator position,
                        InputIterator first, InputIterator last);
```

```
template<container-compatible-range<T> R>
constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);
```

```
template<class... Args> constexpr reference emplace_front(Args&&... args);
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr void push_front(const T& x);
constexpr void push_front(T&& x);
template<container-compatible-range<T> R>
constexpr void prepend_range(R&& rg);
```

```
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
constexpr void append_range(R&& rg);
```

1 *Complexity:* Insertion of a single element into a list takes constant time and exactly one call to a constructor of T. Insertion of multiple elements into a list is linear in the number of elements inserted, and the number of calls to the copy constructor or move constructor of T is exactly equal to the number of elements inserted.

2 *Remarks:* Does not affect the validity of iterators and references. If an exception is thrown, there are no effects.

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_front();
constexpr void pop_back();
constexpr void clear() noexcept;
```

3 *Effects:* Invalidates only the iterators and references to the erased elements.

4 *Throws:* Nothing.

5 *Complexity:* Erasing a single element is a constant time operation with a single call to the destructor of T. Erasing a range in a list is linear time in the size of the range and the number of calls to the destructor of type T is exactly equal to the size of the range.

23.3.11.5 Operations

[list.ops]

1 Since lists allow fast insertion and erasing from the middle of a list, certain operations are provided specifically for them.¹⁷⁵ In this subclause, arguments for a template parameter named `Predicate` or `BinaryPredicate` shall meet the corresponding requirements in 26.2. The semantics of `i + n` and `i - n`, where `i` is an iterator into the list and `n` is an integer, are the same as those of `next(i, n)` and `prev(i, n)`, respectively. For `merge` and `sort`, the definitions and requirements in 26.8 apply.

2 `list` provides three splice operations that destructively move elements from one list to another. The behavior of splice operations is undefined if `get_allocator() != x.get_allocator()`.

```
constexpr void splice(const_iterator position, list& x);
constexpr void splice(const_iterator position, list&& x);
```

3 *Preconditions:* `addressof(x) != this` is true.

4 *Effects:* Inserts the contents of `x` before `position` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

5 *Throws:* Nothing.

6 *Complexity:* Constant time.

```
constexpr void splice(const_iterator position, list& x, const_iterator i);
constexpr void splice(const_iterator position, list&& x, const_iterator i);
```

7 *Preconditions:* `i` is a valid dereferenceable iterator of `x`.

8 *Effects:* Inserts an element pointed to by `i` from list `x` before `position` and removes the element from `x`. The result is unchanged if `position == i` or `position == ++i`. Pointers and references to `*i` continue to refer to this same element but as a member of `*this`. Iterators to `*i` (including `i` itself) continue to refer to the same element, but now behave as iterators into `*this`, not into `x`.

9 *Throws:* Nothing.

10 *Complexity:* Constant time.

```
constexpr void splice(const_iterator position, list& x,
                     const_iterator first, const_iterator last);
```

¹⁷⁵) As specified in 16.4.4.6, the requirements in this Clause apply only to lists whose allocators compare equal.

```
constexpr void splice(const_iterator position, list&& x,
                     const_iterator first, const_iterator last);
```

11 *Preconditions:* `[first, last)` is a valid range in `x`. `position` is not an iterator in the range `[first, last)`.

12 *Effects:* Inserts elements in the range `[first, last)` before `position` and removes the elements from `x`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

13 *Throws:* Nothing.

14 *Complexity:* Constant time if `addressof(x) == this`; otherwise, linear time.

```
constexpr size_type remove(const T& value);
template<class Predicate> constexpr size_type remove_if(Predicate pred);
```

15 *Effects:* Erases all the elements in the list referred to by a list iterator `i` for which the following conditions hold: `*i == value`, `pred(*i) != false`. Invalidates only the iterators and references to the erased elements.

16 *Returns:* The number of elements erased.

17 *Throws:* Nothing unless an exception is thrown by `*i == value` or `pred(*i) != false`.

18 *Complexity:* Exactly `size()` applications of the corresponding predicate.

19 *Remarks:* Stable (16.4.6.8).

```
constexpr size_type unique();
template<class BinaryPredicate> constexpr size_type unique(BinaryPredicate binary_pred);
```

20 Let `binary_pred` be `equal_to<>{}` for the first overload.

21 *Preconditions:* `binary_pred` is an equivalence relation.

22 *Effects:* Erases all but the first element from every consecutive group of equivalent elements. That is, for a nonempty list, erases all elements referred to by the iterator `i` in the range `[begin() + 1, end())` for which `binary_pred(*i, *(i - 1))` is true. Invalidates only the iterators and references to the erased elements.

23 *Returns:* The number of elements erased.

24 *Throws:* Nothing unless an exception is thrown by the predicate.

25 *Complexity:* If `empty()` is false, exactly `size() - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
constexpr void merge(list& x);
constexpr void merge(list&& x);
template<class Compare> constexpr void merge(list& x, Compare comp);
template<class Compare> constexpr void merge(list&& x, Compare comp);
```

26 Let `comp` be `less<>` for the first two overloads.

27 *Preconditions:* `*this` and `x` are both sorted with respect to the comparator `comp`, and `get_allocator() == x.get_allocator()` is true.

28 *Effects:* If `addressof(x) == this`, there are no effects. Otherwise, merges the two sorted ranges `[begin(), end())` and `[x.begin(), x.end())`. The result is a range that is sorted with respect to the comparator `comp`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

29 *Complexity:* At most `size() + x.size() - 1` comparisons if `addressof(x) != this`; otherwise, no comparisons are performed.

30 *Remarks:* Stable (16.4.6.8). If `addressof(x) != this`, `x` is empty after the merge. No elements are copied by this operation. If an exception is thrown other than by a comparison, there are no effects.

```
constexpr void reverse() noexcept;
```

31 *Effects:* Reverses the order of the elements in the list. Does not affect the validity of iterators and references.

32 *Complexity*: Linear time.

```
void sort();
template<class Compare> void sort(Compare comp);
```

33 *Effects*: Sorts the list according to the operator< or a Compare function object. If an exception is thrown, the order of the elements in *this is unspecified. Does not affect the validity of iterators and references.

34 *Complexity*: Approximately $N \log N$ comparisons, where N is size().

35 *Remarks*: Stable (16.4.6.8).

23.3.11.6 Erasure

[list.erase]

```
template<class T, class Allocator, class U = T>
typename list<T, Allocator>::size_type
constexpr erase(list<T, Allocator>& c, const U& value);
```

1 *Effects*: Equivalent to:

```
return erase_if(c, [&](const auto& elem) -> bool { return elem == value; });
```

```
template<class T, class Allocator, class Predicate>
typename list<T, Allocator>::size_type
constexpr erase_if(list<T, Allocator>& c, Predicate pred);
```

2 *Effects*: Equivalent to: return c.remove_if(pred);

23.3.12 Header <vector> synopsis

[vector.syn]

```
#include <compare>           // see 17.12.1
#include <initializer_list>   // see 17.11.2
```

```
namespace std {
    // 23.3.13, class template vector
    template<class T, class Allocator = allocator<T>> class vector;

    template<class T, class Allocator>
        constexpr bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
    template<class T, class Allocator>
        constexpr synth-three-way-result<T>
            operator<=>(const vector<T, Allocator>& x, const vector<T, Allocator>& y);

    template<class T, class Allocator>
        constexpr void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
            noexcept(noexcept(x.swap(y)));
```

// 23.3.13.6, erasure

```
template<class T, class Allocator, class U = T>
    constexpr typename vector<T, Allocator>::size_type
        erase(vector<T, Allocator>& c, const U& value);
template<class T, class Allocator, class Predicate>
    constexpr typename vector<T, Allocator>::size_type
        erase_if(vector<T, Allocator>& c, Predicate pred);
```

```
namespace pmr {
    template<class T>
        using vector = std::vector<T, polymorphic_allocator<T>>;
}
```

// 23.3.14, specialization of vector for bool

// 23.3.14.1, partial class template specialization vector<bool, Allocator>

```
template<class Allocator>
    class vector<bool, Allocator>;
```

```
template<class T>
    constexpr bool is-vector-bool-reference = see below;           // exposition only
```

```

// hash support
template<class T> struct hash;
template<class Allocator> struct hash<vector<bool, Allocator>>;

// 23.3.14.2, formatter specialization for vector<bool>
template<class T, class charT> requires is_vector_bool_reference<T>
    struct formatter<T, charT>;
}

```

23.3.13 Class template vector

[vector]

23.3.13.1 Overview

[vector.overview]

- ¹ A **vector** is a sequence container that supports (amortized) constant time insert and erase operations at the end; insert and erase in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.
- ² A **vector** meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), of a sequence container, including most of the optional sequence container requirements (23.2.4), and, for an element type other than **bool**, of a contiguous container (23.2.2.2). The exceptions are the **push_front**, **prepend_range**, **pop_front**, and **emplace_front** member functions, which are not provided. Descriptions are provided here only for operations on **vector** that are not described in one of these tables or for operations where there is additional semantic information.
- ³ The types **iterator** and **const_iterator** meet the constexpr iterator requirements (24.3.1).

```

namespace std {
    template<class T, class Allocator = allocator<T>>
    class vector {
    public:
        // types
        using value_type          = T;
        using allocator_type      = Allocator;
        using pointer             = allocator_traits<Allocator>::pointer;
        using const_pointer       = allocator_traits<Allocator>::const_pointer;
        using reference            = value_type&;
        using const_reference      = const value_type&;
        using size_type            = implementation-defined; // see 23.2
        using difference_type      = implementation-defined; // see 23.2
        using iterator             = implementation-defined; // see 23.2
        using const_iterator       = implementation-defined; // see 23.2
        using reverse_iterator     = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 23.3.13.2, construct/copy/destroy
        constexpr vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }
        constexpr explicit vector(const Allocator&) noexcept;
        constexpr explicit vector(size_type n, const Allocator& = Allocator());
        constexpr vector(size_type n, const T& value, const Allocator& = Allocator());
        template<class InputIterator>
            constexpr vector(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container_compatible_range<T> R>
            constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
        constexpr vector(const vector& x);
        constexpr vector(vector&&) noexcept;
        constexpr vector(const vector&, const type_identity_t<Allocator>&);
        constexpr vector(vector&&, const type_identity_t<Allocator>&);
        constexpr vector(initializer_list<T>, const Allocator& = Allocator());
        constexpr ~vector();
        constexpr vector& operator=(const vector& x);
        constexpr vector& operator=(vector&& x)
            noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
                allocator_traits<Allocator>::is_always_equal::value);
        constexpr vector& operator=(initializer_list<T>);
        template<class InputIterator>
            constexpr void assign(InputIterator first, InputIterator last);
    };
}

```



```

template<container-compatible-range<T> R>
    constexpr void assign_range(R&& rg);
constexpr void assign(size_type n, const T& u);
constexpr void assign(initializer_list<T>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator          begin() noexcept;
constexpr const_iterator    begin() const noexcept;
constexpr iterator          end() noexcept;
constexpr const_iterator    end() const noexcept;
constexpr reverse_iterator  rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator  rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator    cbegin() const noexcept;
constexpr const_iterator    cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 23.3.13.3, capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;
constexpr void      resize(size_type sz);
constexpr void      resize(size_type sz, const T& c);
constexpr void      reserve(size_type n);
constexpr void      shrink_to_fit();

// element access
constexpr reference    operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference    at(size_type n);
constexpr const_reference at(size_type n) const;
constexpr reference    front();
constexpr const_reference front() const;
constexpr reference    back();
constexpr const_reference back() const;

// 23.3.13.4, data access
constexpr T*          data() noexcept;
constexpr const T*    data() const noexcept;

// 23.3.13.5, modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);
constexpr void pop_back();

template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position,
                              InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T> il);
constexpr iterator erase(const_iterator position);

```

```

constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void      swap(vector&)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
             allocator_traits<Allocator>::is_always_equal::value);
constexpr void      clear() noexcept;
};

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>>
vector(InputIterator, InputIterator, Allocator = Allocator())
    -> vector<iter-value-type<InputIterator>, Allocator>;

template<ranges::input_range R, class Allocator = allocator<ranges::range_value_t<R>>>
vector(from_range_t, R&&, Allocator = Allocator())
    -> vector<ranges::range_value_t<R>, Allocator>;
}

```

- 4 An incomplete type *T* may be used when instantiating *vector* if the allocator meets the allocator completeness requirements (16.4.4.6.2). *T* shall be complete before any member of the resulting specialization of *vector* is referenced.

23.3.13.2 Constructors

[vector.cons]

```
constexpr explicit vector(const Allocator&) noexcept;
```

- 1 *Effects*: Constructs an empty *vector*, using the specified allocator.

- 2 *Complexity*: Constant.

```
constexpr explicit vector(size_type n, const Allocator& = Allocator());
```

- 3 *Preconditions*: *T* is *Cpp17DefaultInsertable* into *vector*.

- 4 *Effects*: Constructs a *vector* with *n* default-inserted elements using the specified allocator.

- 5 *Complexity*: Linear in *n*.

```
constexpr vector(size_type n, const T& value,
                 const Allocator& = Allocator());
```

- 6 *Preconditions*: *T* is *Cpp17CopyInsertable* into *vector*.

- 7 *Effects*: Constructs a *vector* with *n* copies of *value*, using the specified allocator.

- 8 *Complexity*: Linear in *n*.

```
template<class InputIterator>
constexpr vector(InputIterator first, InputIterator last,
                 const Allocator& = Allocator());
```

- 9 *Effects*: Constructs a *vector* equal to the range *[first, last)*, using the specified allocator.

- 10 *Complexity*: Makes only *N* calls to the copy constructor of *T* (where *N* is the distance between *first* and *last*) and no reallocations if *InputIterator* meets the *Cpp17ForwardIterator* requirements. It makes order *N* calls to the copy constructor of *T* and order $\log N$ reallocations if they are just input iterators.

```
template<container-compatible-range<T> R>
constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
```

- 11 *Effects*: Constructs a *vector* object with the elements of the range *rg*, using the specified allocator.

- 12 *Complexity*: Initializes exactly *N* elements from the results of dereferencing successive iterators of *rg*, where *N* is *ranges::distance(rg)*.

- 13 Performs no reallocations if:

- (13.1) — *R* models *ranges::approximately_sized_range*, and *ranges::distance(rg) <= ranges::reserve_hint(rg)* is true, or

- (13.2) — *R* models *ranges::forward_range* and *R* does not model *ranges::approximately_sized_range*. Otherwise, performs order $\log N$ reallocations and order *N* calls to the copy or move constructor of *T*.

23.3.13.3 Capacity

[vector.capacity]

```
constexpr size_type capacity() const noexcept;
```

1 *Returns:* The total number of elements that the vector can hold without requiring reallocation.

2 *Complexity:* Constant time.

```
constexpr void reserve(size_type n);
```

3 *Preconditions:* T is *Cpp17MoveInsertable* into **vector**.

4 *Effects:* After **reserve()**, **capacity()** is greater or equal to the argument of **reserve** if reallocation happens; and equal to the previous value of **capacity()** otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of **reserve()**. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* type, there are no effects.

5 *Throws:* **length_error** if $n > \text{max_size}()$.¹⁷⁶

6 *Complexity:* Linear in the size of the sequence.

7 *Remarks:* The size of the sequence is not changed. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator.

[*Note 1:* If no reallocation happens, they remain valid. — *end note*]

No reallocation shall take place during insertions that happen after a call to **reserve()** until an insertion would make the size of the vector greater than the value of **capacity()**.

```
constexpr void shrink_to_fit();
```

8 *Preconditions:* T is *Cpp17MoveInsertable* into **vector**.

9 *Effects:* **shrink_to_fit** is a non-binding request to reduce **capacity()** to **size()**.

[*Note 2:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*]

It does not increase **capacity()**, but may reduce **capacity()** by causing reallocation. If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* T, there are no effects.

10 *Complexity:* If reallocation happens, linear in the size of the sequence.

11 *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence as well as the past-the-end iterator.

[*Note 3:* If no reallocation happens, they remain valid. — *end note*]

```
constexpr void swap(vector& x)
```

```
noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
         allocator_traits<Allocator>::is_always_equal::value);
```

12 *Effects:* Exchanges the contents and **capacity()** of ***this** with that of **x**.

13 *Complexity:* Constant time.

```
constexpr void resize(size_type sz);
```

14 *Preconditions:* T is *Cpp17MoveInsertable* and *Cpp17DefaultInsertable* into **vector**.

15 *Effects:* If $sz < \text{size}()$, erases the last $\text{size}() - sz$ elements from the sequence. Otherwise, appends $sz - \text{size}()$ default-inserted elements to the sequence.

16 *Remarks:* If an exception is thrown other than by the move constructor of a non-*Cpp17CopyInsertable* T, there are no effects.

```
constexpr void resize(size_type sz, const T& c);
```

17 *Preconditions:* T is *Cpp17CopyInsertable* into **vector**.

18 *Effects:* If $sz < \text{size}()$, erases the last $\text{size}() - sz$ elements from the sequence. Otherwise, appends $sz - \text{size}()$ copies of **c** to the sequence.

19 *Remarks:* If an exception is thrown, there are no effects.

¹⁷⁶ **reserve()** uses **Allocator::allocate()** which can throw an appropriate exception.

23.3.13.4 Data**[vector.data]**

```
constexpr T*      data() noexcept;
constexpr const T* data() const noexcept;
```

1 *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty vector, `data() == addressof(front())` is true.

2 *Complexity:* Constant time.

23.3.13.5 Modifiers**[vector.modifiers]**

```
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T>);
```

```
template<class... Args> constexpr reference emplace_back(Args&&... args);
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg);
```

1 *Complexity:* If reallocation happens, linear in the number of elements of the resulting vector; otherwise, linear in the number of elements inserted plus the distance to the end of the vector.

2 *Remarks:* Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. If no reallocation happens, then references, pointers, and iterators before the insertion point remain valid but those at or after the insertion point, including the past-the-end iterator, are invalidated. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `T` or by any `InputIterator` operation, there are no effects. If an exception is thrown while inserting a single element at the end and `T` is `Cpp17CopyInsertable` or `is_nothrow_move_constructible_v<T>` is true, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`Cpp17CopyInsertable` `T`, the effects are unspecified.

3 For the declarations taking a range `R`, performs at most one reallocation if:

(3.1) — `R` models `ranges::approximately_sized_range` and `ranges::distance(rg) <= ranges::reserve_hint(rg)` is true, or

(3.2) — `R` models `ranges::forward_range` and `R` does not model `ranges::approximately_sized_range`.

For the declarations taking a pair of `InputIterator`, performs at most one reallocation if `InputIterator` meets the `Cpp17ForwardIterator` requirements.

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();
```

4 *Effects:* Invalidates iterators and references at or after the point of the erase.

5 *Throws:* Nothing unless an exception is thrown by the assignment operator or move assignment operator of `T`.

6 *Complexity:* The destructor of `T` is called the number of times equal to the number of the elements erased, but the assignment operator of `T` is called the number of times equal to the number of elements in the vector after the erased elements.

23.3.13.6 Erasure**[vector.erase]**

```
template<class T, class Allocator, class U = T>
    constexpr typename vector<T, Allocator>::size_type
        erase(vector<T, Allocator>& c, const U& value);
```

1 *Effects:* Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class T, class Allocator, class Predicate>
constexpr typename vector<T, Allocator>::size_type
erase_if(vector<T, Allocator>& c, Predicate pred);
```

2 *Effects:* Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

23.3.14 Specialization of vector for bool

[vector.bool]

23.3.14.1 Partial class template specialization vector<bool, Allocator>

[vector.bool.pspc]

1 To optimize space allocation, a partial specialization of vector for bool elements is provided:

```
namespace std {
    template<class Allocator>
    class vector<bool, Allocator> {
    public:
        // types
        using value_type          = bool;
        using allocator_type      = Allocator;
        using pointer              = implementation-defined;
        using const_pointer       = implementation-defined;
        using const_reference     = bool;
        using size_type           = implementation-defined; // see 23.2
        using difference_type     = implementation-defined; // see 23.2
        using iterator            = implementation-defined; // see 23.2
        using const_iterator      = implementation-defined; // see 23.2
        using reverse_iterator    = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // bit reference
        class reference {
        public:
            constexpr reference(const reference& x) noexcept;
            constexpr ~reference();
            constexpr reference& operator=(bool x) noexcept;
            constexpr reference& operator=(const reference& x) noexcept;
            constexpr const reference& operator=(bool x) const noexcept;
            constexpr operator bool() const noexcept;
            constexpr void flip() noexcept; // flips the bit

            friend constexpr void swap(reference x, reference y) noexcept;
            friend constexpr void swap(reference x, bool& y) noexcept;
            friend constexpr void swap(bool& x, reference y) noexcept;
        };

        // construct/copy/destroy
        constexpr vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }
        constexpr explicit vector(const Allocator&) noexcept;
        constexpr explicit vector(size_type n, const Allocator& = Allocator());
        constexpr vector(size_type n, const bool& value, const Allocator& = Allocator());
        template<class InputIterator>
            constexpr vector(InputIterator first, InputIterator last, const Allocator& = Allocator());
        template<container-compatible-range<bool> R>
            constexpr vector(from_range_t, R&& rg, const Allocator& = Allocator());
        constexpr vector(const vector& x);
        constexpr vector(vector&& x) noexcept;
```

```

constexpr vector(const vector&, const type_identity_t<Allocator>&);
constexpr vector(vector&&, const type_identity_t<Allocator>&);
constexpr vector(initializer_list<bool>, const Allocator& = Allocator());
constexpr ~vector();
constexpr vector& operator=(const vector& x);
constexpr vector& operator=(vector&& x)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
constexpr vector& operator=(initializer_list<bool>);
template<class InputIterator>
    constexpr void assign(InputIterator first, InputIterator last);
template<container-compatible-range<bool> R>
    constexpr void assign_range(R&& rg);
constexpr void assign(size_type n, const bool& t);
constexpr void assign(initializer_list<bool>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator          begin() noexcept;
constexpr const_iterator    begin() const noexcept;
constexpr iterator          end() noexcept;
constexpr const_iterator    end() const noexcept;
constexpr reverse_iterator  rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator  rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator    cbegin() const noexcept;
constexpr const_iterator    cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr size_type capacity() const noexcept;
constexpr void      resize(size_type sz, bool c = false);
constexpr void      reserve(size_type n);
constexpr void      shrink_to_fit();

// element access
constexpr reference      operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr reference      at(size_type n);
constexpr const_reference at(size_type n) const;
constexpr reference      front();
constexpr const_reference front() const;
constexpr reference      back();
constexpr const_reference back() const;

// modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const bool& x);
template<container-compatible-range<bool> R>
    constexpr void append_range(R&& rg);
constexpr void pop_back();
template<class... Args> constexpr iterator emplace(const_iterator position, Args&&... args);
constexpr iterator insert(const_iterator position, const bool& x);
constexpr iterator insert(const_iterator position, size_type n, const bool& x);
template<class InputIterator>
    constexpr iterator insert(const_iterator position,
                             InputIterator first, InputIterator last);

```

```

template<container-compatible-range<bool> R>
    constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<bool> il);

constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(vector&)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
              allocator_traits<Allocator>::is_always_equal::value);
constexpr void flip() noexcept;           // flips all bits
constexpr void clear() noexcept;
};
}

```

2 Unless described below, all operations have the same requirements and semantics as the `vector` primary template, except that operations dealing with the `bool` value type map to bit values in the container storage and `allocator_traits::construct` (20.2.9.3) is not used to construct these values.

3 There is no requirement that the data be stored as a contiguous allocation of `bool` values. A space-optimized representation of bits is recommended instead.

4 `reference` is a class that simulates a reference to a single bit in the sequence.

```
constexpr reference::reference(const reference& x) noexcept;
```

5 *Effects:* Initializes `*this` to refer to the same bit as `x`.

```
constexpr reference::~reference();
```

6 *Effects:* None.

```
constexpr reference& reference::operator=(bool x) noexcept;
constexpr reference& reference::operator=(const reference& x) noexcept;
constexpr const reference& reference::operator=(bool x) const noexcept;
```

7 *Effects:* Sets the bit referred to by `*this` when `bool(x)` is `true`, and clears it otherwise.

8 *Returns:* `*this`.

```
constexpr reference::operator bool() const noexcept;
```

9 *Returns:* `true` if the value of the bit referred to by `*this` is one, `false` otherwise.

```
constexpr void reference::flip() noexcept;
```

10 *Effects:* Equivalent to `*this = !*this`.

```
constexpr void swap(reference x, reference y) noexcept;
constexpr void swap(reference x, bool& y) noexcept;
constexpr void swap(bool& x, reference y) noexcept;
```

11 *Effects:* Exchanges the values denoted by `x` and `y` as if by:

```

    bool b = x;
    x = y;
    y = b;

```

```
constexpr void flip() noexcept;
```

12 *Effects:* Replaces each element in the container with its complement.

```
template<class Allocator> struct hash<vector<bool, Allocator>>;
```

13 The specialization is enabled (22.10.19).

```

template<class T>
    constexpr bool is-vector-bool-reference = see below;

```

14 The expression `is-vector-bool-reference<T>` is `true` if `T` denotes the type `vector<bool, Alloc>::reference` for some type `Alloc` and `vector<bool, Alloc>` is not a program-defined specialization.

23.3.14.2 Formatter specialization for vector<bool>**[vector.bool.fmt]**

```

namespace std {
    template<class T, class charT>
        requires is-vector-bool-reference<T>
    struct formatter<T, charT> {
    private:
        formatter<bool, charT> underlying_;           // exposition only

    public:
        template<class ParseContext>
            constexpr typename ParseContext::iterator
                parse(ParseContext& ctx);

        template<class FormatContext>
            constexpr typename FormatContext::iterator
                format(const T& ref, FormatContext& ctx) const;
    };
}

```

```

template<class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);

```

¹ Equivalent to: return *underlying_*.parse(ctx);

```

template<class FormatContext>
constexpr typename FormatContext::iterator
    format(const T& ref, FormatContext& ctx) const;

```

² Equivalent to: return *underlying_*.format(ref, ctx);

23.3.15 Header <inplace_vector> synopsis**[inplace.vector.syn]**

```

// mostly freestanding
#include <compare>           // see 17.12.1
#include <initializer_list>  // see 17.11.2

namespace std {
    // 23.3.16, class template inplace_vector
    template<class T, size_t N> class inplace_vector;           // partially freestanding

    // 23.3.16.6, erasure
    template<class T, size_t N, class U = T>
        constexpr typename inplace_vector<T, N>::size_type
            erase(inplace_vector<T, N>& c, const U& value);
    template<class T, size_t N, class Predicate>
        constexpr typename inplace_vector<T, N>::size_type
            erase_if(inplace_vector<T, N>& c, Predicate pred);
}

```

23.3.16 Class template inplace_vector**[inplace.vector]****23.3.16.1 Overview****[inplace.vector.overview]**

- ¹ An `inplace_vector` is a contiguous container. Its capacity is fixed and its elements are stored within the `inplace_vector` object itself.
- ² An `inplace_vector` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of a contiguous container, and of a sequence container, including most of the optional sequence container requirements (23.2.4). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `inplace_vector` that are not described in one of these tables or for operations where there is additional semantic information.
- ³ For any N, `inplace_vector<T, N>::iterator` and `inplace_vector<T, N>::const_iterator` meet the `constexpr` iterator requirements.

- ⁴ Any member function of `inplace_vector<T, N>` that would cause the size to exceed `N` throws an exception of type `bad_alloc`.
- ⁵ Let `IV` denote a specialization of `inplace_vector<T, N>`. If `N` is zero, then `IV` is trivially copyable and empty, and `std::is_trivially_default_constructible_v<IV>` is true. Otherwise:

(5.1) — If `is_trivially_copy_constructible_v<T>` is true, then `IV` has a trivial copy constructor.

(5.2) — If `is_trivially_move_constructible_v<T>` is true, then `IV` has a trivial move constructor.

(5.3) — If `is_trivially_destructible_v<T>` is true, then:

(5.3.1) — `IV` has a trivial destructor.

(5.3.2) — If

`is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T>`
is true, then `IV` has a trivial copy assignment operator.

(5.3.3) — If

`is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T>`
is true, then `IV` has a trivial move assignment operator.

```
namespace std {
    template<class T, size_t N>
    class inplace_vector {
    public:
        // types:
        using value_type          = T;
        using pointer              = T*;
        using const_pointer        = const T*;
        using reference            = value_type&;
        using const_reference      = const value_type&;
        using size_type            = size_t;
        using difference_type      = ptrdiff_t;
        using iterator             = implementation-defined; // see 23.2
        using const_iterator       = implementation-defined; // see 23.2
        using reverse_iterator     = std::reverse_iterator<iterator>;
        using const_reverse_iterator = std::reverse_iterator<const_iterator>;

        // 23.3.16.2, construct/copy/destroy
        constexpr inplace_vector() noexcept;
        constexpr explicit inplace_vector(size_type n); // freestanding-deleted
        constexpr inplace_vector(size_type n, const T& value); // freestanding-deleted
        template<class InputIterator>
            constexpr inplace_vector(InputIterator first, InputIterator last); // freestanding-deleted
        template<container-compatible-range<T> R>
            constexpr inplace_vector(from_range_t, R&& rg); // freestanding-deleted
        constexpr inplace_vector(const inplace_vector&);
        constexpr inplace_vector(inplace_vector&&)
            noexcept(N == 0 || is_nothrow_move_constructible_v<T>);
        constexpr inplace_vector(initializer_list<T> il); // freestanding-deleted
        constexpr ~inplace_vector();
        constexpr inplace_vector& operator=(const inplace_vector& other);
        constexpr inplace_vector& operator=(inplace_vector&& other)
            noexcept(N == 0 || (is_nothrow_move_assignable_v<T> &&
                               is_nothrow_move_constructible_v<T>));
        constexpr inplace_vector& operator=(initializer_list<T>); // freestanding-deleted
        template<class InputIterator>
            constexpr void assign(InputIterator first, InputIterator last); // freestanding-deleted
        template<container-compatible-range<T> R>
            constexpr void assign_range(R&& rg); // freestanding-deleted
        constexpr void assign(size_type n, const T& u); // freestanding-deleted
        constexpr void assign(initializer_list<T> il); // freestanding-deleted

        // iterators
        constexpr iterator          begin()          noexcept;
```

```

constexpr const_iterator      begin()    const noexcept;
constexpr iterator           end()      noexcept;
constexpr const_iterator      end()      const noexcept;
constexpr reverse_iterator    rbegin()   noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator    rend()     noexcept;
constexpr const_reverse_iterator rend()   const noexcept;

constexpr const_iterator      cbegin()   const noexcept;
constexpr const_iterator      cend()     const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend()  const noexcept;

// 23.3.16.3, capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
static constexpr size_type max_size() noexcept;
static constexpr size_type capacity() noexcept;
constexpr void resize(size_type sz); // freestanding-deleted
constexpr void resize(size_type sz, const T& c); // freestanding-deleted
static constexpr void reserve(size_type n); // freestanding-deleted
static constexpr void shrink_to_fit() noexcept;

// element access
constexpr reference          operator[](size_type n);
constexpr const_reference    operator[](size_type n) const;
constexpr reference          at(size_type n); // freestanding-deleted
constexpr const_reference    at(size_type n) const; // freestanding-deleted
constexpr reference          front();
constexpr const_reference    front() const;
constexpr reference          back();
constexpr const_reference    back() const;

// 23.3.16.4, data access
constexpr T* data()          noexcept;
constexpr const T* data()    const noexcept;

// 23.3.16.5, modifiers
template<class... Args>
    constexpr reference emplace_back(Args&&... args); // freestanding-deleted
constexpr reference push_back(const T& x); // freestanding-deleted
constexpr reference push_back(T&& x); // freestanding-deleted
template<container-compatible-range<T> R>
    constexpr void append_range(R&& rg); // freestanding-deleted
constexpr void pop_back();

template<class... Args>
    constexpr optional<reference> try_emplace_back(Args&&... args);
constexpr optional<reference> try_push_back(const T& x);
constexpr optional<reference> try_push_back(T&& x);

template<class... Args>
    constexpr reference unchecked_emplace_back(Args&&... args);
constexpr reference unchecked_push_back(const T& x);
constexpr reference unchecked_push_back(T&& x);

template<class... Args>
    constexpr iterator emplace(const_iterator position, Args&&... args); // freestanding-deleted
constexpr iterator insert(const_iterator position, const T& x); // freestanding-deleted
constexpr iterator insert(const_iterator position, T&& x); // freestanding-deleted
constexpr iterator insert(const_iterator position, size_type n, // freestanding-deleted
    const T& x);

```

```

template<class InputIterator>
    constexpr iterator insert(const_iterator position,                // freestanding-deleted
                             InputIterator first, InputIterator last);
template<container-compatible-range<T> R>
    constexpr iterator insert_range(const_iterator position, R&& rg); // freestanding-deleted
constexpr iterator insert(const_iterator position,                  // freestanding-deleted
                          initializer_list<T> il);
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(inplace_vector& x)
    noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
                        is_nothrow_move_constructible_v<T>));
constexpr void clear() noexcept;

friend constexpr bool operator==(const inplace_vector& x,
                                  const inplace_vector& y);

friend constexpr auto
    operator<=>(const inplace_vector& x, const inplace_vector& y)
    requires requires (const T t) { synth-three-way(t, t); }
{
    return lexicographical_compare_three_way(x.begin(), x.end(), y.begin(), y.end(),
                                              synth-three-way);
}

friend constexpr void swap(inplace_vector& x, inplace_vector& y)
    noexcept(N == 0 || (is_nothrow_swappable_v<T> &&
                        is_nothrow_move_constructible_v<T>))
{ x.swap(y); }
};
}

```

23.3.16.2 Constructors

[inplace.vector.cons]

```
constexpr explicit inplace_vector(size_type n);
```

- 1 *Preconditions:* T is Cpp17DefaultInsertable into inplace_vector.
2 *Effects:* Constructs an inplace_vector with n default-inserted elements.
3 *Complexity:* Linear in n.

```
constexpr inplace_vector(size_type n, const T& value);
```

- 4 *Preconditions:* T is Cpp17CopyInsertable into inplace_vector.
5 *Effects:* Constructs an inplace_vector with n copies of value.
6 *Complexity:* Linear in n.

```
template<class InputIterator>
    constexpr inplace_vector(InputIterator first, InputIterator last);
```

- 7 *Effects:* Constructs an inplace_vector equal to the range [first,last).
8 *Complexity:* Linear in distance(first, last).

```
template<container-compatible-range<T> R>
    constexpr inplace_vector(from_range_t, R&& rg);
```

- 9 *Mandates:* If ranges::size(rg) is a constant expression, then ranges::size(rg) ≤ N.
10 *Effects:* Constructs an inplace_vector with the elements of the range rg.
11 *Complexity:* Linear in ranges::distance(rg).

23.3.16.3 Capacity

[inplace.vector.capacity]

```
static constexpr size_type capacity() noexcept;
static constexpr size_type max_size() noexcept;
```

- 1 *Returns:* N.

```
constexpr void resize(size_type sz);
```

2 *Preconditions:* T is *Cpp17DefaultInsertable* into `inplace_vector`.

3 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

4 *Remarks:* If an exception is thrown, there are no effects on `*this`.

```
constexpr void resize(size_type sz, const T& c);
```

5 *Preconditions:* T is *Cpp17CopyInsertable* into `inplace_vector`.

6 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

7 *Remarks:* If an exception is thrown, there are no effects on `*this`.

```
static constexpr void reserve(size_type n);
```

8 *Effects:* None.

9 *Throws:* `bad_alloc` if `n > capacity()` is true.

```
static constexpr void shrink_to_fit() noexcept;
```

10 *Effects:* None.

23.3.16.4 Data

[`inplace_vector.data`]

```
constexpr T* data() noexcept;
```

```
constexpr const T* data() const noexcept;
```

1 *Returns:* A pointer such that `[data(), data() + size())` is a valid range. For a non-empty `inplace_vector`, `data() == addressof(front())` is true.

2 *Complexity:* Constant time.

23.3.16.5 Modifiers

[`inplace_vector.modifiers`]

```
constexpr iterator insert(const_iterator position, const T& x);
```

```
constexpr iterator insert(const_iterator position, T&& x);
```

```
constexpr iterator insert(const_iterator position, size_type n, const T& x);
```

```
template<class InputIterator>
```

```
    constexpr iterator insert(const_iterator position, InputIterator first, InputIterator last);
```

```
template<container-compatible-range<T> R>
```

```
    constexpr iterator insert_range(const_iterator position, R&& rg);
```

```
constexpr iterator insert(const_iterator position, initializer_list<T> il);
```

```
template<class... Args>
```

```
    constexpr iterator emplace(const_iterator position, Args&&... args);
```

```
template<container-compatible-range<T> R>
```

```
    constexpr void append_range(R&& rg);
```

1 Let n be the value of `size()` before this call for the `append_range` overload, and `distance(begin, position)` otherwise.

2 *Complexity:* Linear in the number of elements inserted plus the distance to the end of the vector.

3 *Remarks:* If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T or by any `InputIterator` operation, there are no effects. Otherwise, if an exception is thrown, then `size() ≥ n` and elements in the range `begin() + [0, n)` are not modified.

```
constexpr reference push_back(const T& x);
```

```
constexpr reference push_back(T&& x);
```

```
template<class... Args>
```

```
    constexpr reference emplace_back(Args&&... args);
```

4 *Returns:* `back()`.

5 *Throws:* `bad_alloc` or any exception thrown by the initialization of the inserted element.

6 *Complexity:* Constant.

7 *Remarks:* If an exception is thrown, there are no effects on **this*.

```
template<class... Args>
constexpr optional<reference> try_emplace_back(Args&&... args);
constexpr optional<reference> try_push_back(const T& x);
constexpr optional<reference> try_push_back(T&& x);
```

8 Let *vals* denote a pack:

(8.1) — `std::forward<Args>(args)...` for the first overload,

(8.2) — `x` for the second overload,

(8.3) — `std::move(x)` for the third overload.

9 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `inplace_vector` from *vals*....

10 *Effects:* If `size() < capacity()` is true, appends an object of type *T* direct-non-list-initialized with *vals*.... Otherwise, there are no effects.

11 *Returns:* nullopt if `size() == capacity()` is true, otherwise `optional<reference>(in_place, back())`.

12 *Throws:* Nothing unless an exception is thrown by the initialization of the inserted element.

13 *Complexity:* Constant.

14 *Remarks:* If an exception is thrown, there are no effects on **this*.

```
template<class... Args>
constexpr reference unchecked_emplace_back(Args&&... args);
```

15 *Preconditions:* `size() < capacity()` is true.

16 *Effects:* Equivalent to: `return *try_emplace_back(std::forward<Args>(args)...) ;`

```
constexpr reference unchecked_push_back(const T& x);
constexpr reference unchecked_push_back(T&& x);
```

17 *Preconditions:* `size() < capacity()` is true.

18 *Effects:* Equivalent to: `return *try_push_back(std::forward<decltype(x)>(x)) ;`

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();
```

19 *Effects:* Invalidates iterators and references at or after the point of the erase.

20 *Throws:* Nothing unless an exception is thrown by the assignment operator or move assignment operator of *T*.

21 *Complexity:* The destructor of *T* is called the number of times equal to the number of the elements erased, but the assignment operator of *T* is called the number of times equal to the number of elements after the erased elements.

```
constexpr void swap(inplace_vector& x) noexcept(
    N == 0 || (is_nothrow_swappable_v<T> && is_nothrow_move_constructible_v<T>));
```

22 *Preconditions:* *T* meets the *Cpp17MoveConstructible* requirements. Let *M* be `min(size(), x.size())`. For each non-negative integer *n* < *M*, `(*this)[n]` is swappable with `x[n]` (16.4.4.3).

23 *Effects:* Exchanges the contents of **this* and *x*.

23.3.16.6 Erasure

[inplace.vector.erase]

```
template<class T, size_t N, class U = T>
constexpr size_t erase(inplace_vector<T, N>& c, const U& value);
```

1 *Effects:* Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class T, size_t N, class Predicate>
constexpr size_t erase_if(inplace_vector<T, N>& c, Predicate pred);
```

2 *Effects:* Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

23.4 Associative containers

[associative]

23.4.1 General

[associative.general]

- 1 The header `<map>` (23.4.2) defines the class templates `map` and `multimap`; the header `<set>` (23.4.5) defines the class templates `set` and `multiset`.
- 2 The following exposition-only alias templates may appear in deduction guides for associative containers:

```
template<class InputIterator>
    using iter-value-type = iterator_traits<InputIterator>::value_type;    // exposition only
template<class InputIterator>
    using iter-key-type = remove_cvref_t<
        tuple_element_t<0, iter-value-type<InputIterator>>>;                // exposition only
template<class InputIterator>
    using iter-mapped-type = remove_cvref_t<
        tuple_element_t<1, iter-value-type<InputIterator>>>;                // exposition only
template<class InputIterator>
    using iter-to-alloc-type = pair<
        const iter-key-type<InputIterator>,
        iter-mapped-type<InputIterator>>;                                // exposition only
template<ranges::input_range Range>
    using range-key-type =
        remove_cvref_t<tuple_element_t<0, ranges::range_value_t<Range>>>;    // exposition only
template<ranges::input_range Range>
    using range-mapped-type =
        remove_cvref_t<tuple_element_t<1, ranges::range_value_t<Range>>>;    // exposition only
template<ranges::input_range Range>
    using range-to-alloc-type =
        pair<const range-key-type<Range>,
            range-mapped-type<Range>>;                                    // exposition only
```

23.4.2 Header `<map>` synopsis

[associative.map.syn]

```
#include <compare>                // see 17.12.1
#include <initializer_list>        // see 17.11.2

namespace std {
    // 23.4.3, class template map
    template<class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
        class map;

    template<class Key, class T, class Compare, class Allocator>
        constexpr bool operator==(const map<Key, T, Compare, Allocator>& x,
                                   const map<Key, T, Compare, Allocator>& y);
    template<class Key, class T, class Compare, class Allocator>
        constexpr synth-three-way-result<pair<const Key, T>>
            operator<=>(const map<Key, T, Compare, Allocator>& x,
                       const map<Key, T, Compare, Allocator>& y);

    template<class Key, class T, class Compare, class Allocator>
        constexpr void swap(map<Key, T, Compare, Allocator>& x,
                             map<Key, T, Compare, Allocator>& y)
            noexcept(noexcept(x.swap(y)));
```

```

// 23.4.3.5, erasure for map
template<class Key, class T, class Compare, class Allocator, class Predicate>
constexpr typename map<Key, T, Compare, Allocator>::size_type
erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);

// 23.4.4, class template multimap
template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
class multimap;

template<class Key, class T, class Compare, class Allocator>
constexpr bool operator==(const multimap<Key, T, Compare, Allocator>& x,
                          const multimap<Key, T, Compare, Allocator>& y);
template<class Key, class T, class Compare, class Allocator>
constexpr synth-three-way-result<pair<const Key, T>>
operator<=>(const multimap<Key, T, Compare, Allocator>& x,
           const multimap<Key, T, Compare, Allocator>& y);

template<class Key, class T, class Compare, class Allocator>
constexpr void swap(multimap<Key, T, Compare, Allocator>& x,
                   multimap<Key, T, Compare, Allocator>& y)
noexcept(noexcept(x.swap(y)));

// 23.4.4.4, erasure for multimap
template<class Key, class T, class Compare, class Allocator, class Predicate>
constexpr typename multimap<Key, T, Compare, Allocator>::size_type
erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);

namespace pmr {
    template<class Key, class T, class Compare = less<Key>>
    using map = std::map<Key, T, Compare,
                      polymorphic_allocator<pair<const Key, T>>>;

    template<class Key, class T, class Compare = less<Key>>
    using multimap = std::multimap<Key, T, Compare,
                                   polymorphic_allocator<pair<const Key, T>>>;
}
}

```

23.4.3 Class template map

[map]

23.4.3.1 Overview

[map.overview]

- ¹ A `map` is an associative container that supports unique keys (i.e., contains at most one of each key value) and provides for fast retrieval of values of another type `T` based on the keys. The `map` class supports bidirectional iterators.
- ² A `map` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of an associative container (23.2.7). A `map` also provides most operations described in 23.2.7 for unique keys. This means that a `map` supports the `a_uniq` operations in 23.2.7 but not the `a_eq` operations. For a `map<Key,T>` the `key_type` is `Key` and the `value_type` is `pair<const Key,T>`. Descriptions are provided here only for operations on `map` that are not described in one of those tables or for operations where there is additional semantic information.
- ³ The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
    template<class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T>>>
    class map {
    public:
        // types
        using key_type           = Key;
        using mapped_type       = T;
        using value_type        = pair<const Key, T>;
        using key_compare       = Compare;

```

```

using allocator_type      = Allocator;
using pointer             = allocator_traits<Allocator>::pointer;
using const_pointer       = allocator_traits<Allocator>::const_pointer;
using reference           = value_type&;
using const_reference     = const value_type&;
using size_type           = implementation-defined; // see 23.2
using difference_type     = implementation-defined; // see 23.2
using iterator            = implementation-defined; // see 23.2
using const_iterator      = implementation-defined; // see 23.2
using reverse_iterator    = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
using node_type           = unspecified;
using insert_return_type  = insert-return-type<iterator, node_type>;

class value_compare {
protected:
    Compare comp;
    constexpr value_compare(Compare c) : comp(c) {}

public:
    constexpr bool operator()(const value_type& x, const value_type& y) const {
        return comp(x.first, y.first);
    }
};

// 23.4.3.2, construct/copy/destroy
constexpr map() : map(Compare()) { }
constexpr explicit map(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
    constexpr map(InputIterator first, InputIterator last,
        const Compare& comp = Compare(), const Allocator& = Allocator());
template<container-compatible-range<value_type> R>
    constexpr map(from_range_t, R&& rg, const Compare& comp = Compare(),
        const Allocator& = Allocator());
constexpr map(const map& x);
constexpr map(map&& x);
constexpr explicit map(const Allocator&);
constexpr map(const map&, const type_identity_t<Allocator>&);
constexpr map(map&&, const type_identity_t<Allocator>&);
constexpr map(initializer_list<value_type>, const Compare& = Compare(),
    const Allocator& = Allocator());
template<class InputIterator>
    constexpr map(InputIterator first, InputIterator last, const Allocator& a)
        : map(first, last, Compare(), a) { }
template<container-compatible-range<value_type> R>
    constexpr map(from_range_t, R&& rg, const Allocator& a)
        : map(from_range, std::forward<R>(rg), Compare(), a) { }
constexpr map(initializer_list<value_type> il, const Allocator& a)
    : map(il, Compare(), a) { }
constexpr ~map();
constexpr map& operator=(const map& x);
constexpr map& operator=(map&& x)
    noexcept(allocator_traits<Allocator>::is_always_equal::value &&
        is_nothrow_move_assignable_v<Compare>);
constexpr map& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator      begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator      end() noexcept;
constexpr const_iterator end() const noexcept;

```



```

constexpr reverse_iterator      rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator      rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator        cbegin() const noexcept;
constexpr const_iterator        cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.4.3.3, element access
constexpr mapped_type& operator[](const key_type& x);
constexpr mapped_type& operator[](key_type&& x);
template<class K> constexpr mapped_type& operator[](K&& x);
constexpr mapped_type&          at(const key_type& x);
constexpr const mapped_type& at(const key_type& x) const;
template<class K> constexpr mapped_type&          at(const K& x);
template<class K> constexpr const mapped_type& at(const K& x) const;
constexpr optional<mapped_type&>          lookup(const key_type& x);
constexpr optional<const mapped_type&> lookup(const key_type& x) const;
template<class K> constexpr optional<mapped_type&>          lookup(const K& x);
template<class K> constexpr optional<const mapped_type&> lookup(const K& x) const;

// 23.4.3.4, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
    constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr pair<iterator, bool> insert(const value_type& x);
constexpr pair<iterator, bool> insert(value_type&& x);
template<class P> constexpr pair<iterator, bool> insert(P&& x);
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
template<class P>
    constexpr iterator insert(const_iterator position, P&&);
template<class InputIterator>
    constexpr void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
    constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr insert_return_type insert(node_type&& nh);
constexpr iterator          insert(const_iterator hint, node_type&& nh);

template<class... Args>
    constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
    constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template<class K, class... Args>
    constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class... Args>
    constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
    constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template<class K, class... Args>
    constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);

```

```

template<class M>
    constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
    constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template<class K, class M>
    constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
template<class M>
    constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
    constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
template<class K, class M>
    constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(map&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value &&
              is_nothrow_swappable_v<Compare>);
constexpr void clear() noexcept;

template<class C2>
    constexpr void merge(map<Key, T, C2, Allocator>& source);
template<class C2>
    constexpr void merge(map<Key, T, C2, Allocator>&& source);
template<class C2>
    constexpr void merge(multimap<Key, T, C2, Allocator>& source);
template<class C2>
    constexpr void merge(multimap<Key, T, C2, Allocator>&& source);

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// map operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
    constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
    constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;

```

```

};

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>,
        class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
map(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
-> map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare, Allocator>;

template<ranges::input_range R, class Compare = less<range-key-type<R>>,
        class Allocator = allocator<range-to-alloc-type<R>>>
map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> map<range-key-type<R>, range-mapped-type<R>, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
map(initializer_list<pair<Key, T>>, Compare = Compare(), Allocator = Allocator())
-> map<Key, T, Compare, Allocator>;

template<class InputIterator, class Allocator>
map(InputIterator, InputIterator, Allocator)
-> map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
    less<iter-key-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
map(from_range_t, R&&, Allocator)
-> map<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
map(initializer_list<pair<Key, T>>, Allocator) -> map<Key, T, less<Key>, Allocator>;
}

```

23.4.3.2 Constructors, copy, and assignment

[map.cons]

```
constexpr explicit map(const Compare& comp, const Allocator& = Allocator());
```

1 *Effects:* Constructs an empty map using the specified comparison object and allocator.

2 *Complexity:* Constant.

```

template<class InputIterator>
constexpr map(InputIterator first, InputIterator last,
              const Compare& comp = Compare(), const Allocator& = Allocator());

```

3 *Effects:* Constructs an empty map using the specified comparison object and allocator, and inserts elements from the range [first,last).

4 *Complexity:* Linear in N if the range [first,last) is already sorted with respect to comp and otherwise $N \log N$, where N is last - first.

```

template<container-compatible-range<value_type> R>
constexpr map(from_range_t, R&& rg, const Compare& comp = Compare(),
              const Allocator& = Allocator());

```

5 *Effects:* Constructs an empty map using the specified comparison object and allocator, and inserts elements from the range rg.

6 *Complexity:* Linear in N if rg is already sorted with respect to comp and otherwise $N \log N$, where N is ranges::distance(rg).

23.4.3.3 Element access

[map.access]

```
constexpr mapped_type& operator[](const key_type& x);
```

1 *Effects:* Equivalent to: return try_emplace(x).first->second;

```
constexpr mapped_type& operator[](key_type&& x);
```

2 *Effects:* Equivalent to: return try_emplace(std::move(x)).first->second;

```
template<class K> constexpr mapped_type& operator[] (K&& x);
```

3 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

4 *Effects:* Equivalent to: `return try_emplace(std::forward<K>(x)).first->second;`

```
constexpr mapped_type&          at(const key_type& x);
constexpr const mapped_type& at(const key_type& x) const;
```

5 *Returns:* A reference to the `mapped_type` corresponding to `x` in `*this`.

6 *Throws:* An exception object of type `out_of_range` if no such element is present.

7 *Complexity:* Logarithmic.

```
template<class K> constexpr mapped_type&          at(const K& x);
template<class K> constexpr const mapped_type& at(const K& x) const;
```

8 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

9 *Preconditions:* The expression `find(x)` is well-formed and has well-defined behavior.

10 *Returns:* A reference to `find(x)->second`.

11 *Throws:* An exception object of type `out_of_range` if `find(x) == end()` is true.

12 *Complexity:* Logarithmic.

```
constexpr optional<mapped_type&>          lookup(const key_type& x);
constexpr optional<const mapped_type&> lookup(const key_type& x) const;
template<class K> constexpr optional<mapped_type&>          lookup(const K& x);
template<class K> constexpr optional<const mapped_type&> lookup(const K& x) const;
```

13 *Constraints:* For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

14 *Preconditions:* The expression `find(x)` is well-formed and has well-defined behavior.

15 *Returns:* `find(x)->second` if `contains(x)` is true, otherwise `nullopt`.

16 *Complexity:* Logarithmic.

23.4.3.4 Modifiers

[map.modifiers]

```
template<class P>
constexpr pair<iterator, bool> insert(P&& x);
template<class P>
constexpr iterator insert(const_iterator position, P&& x);
```

1 *Constraints:* `is_constructible_v<value_type, P&&>` is true.

2 *Effects:* The first form is equivalent to `return emplace(std::forward<P>(x))`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x))`.

```
template<class... Args>
constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
```

3 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `map` from `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(std::forward<Args>(args)...)...`.

4 *Effects:* If the `map` already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(std::forward<Args>(args)...)...`.

5 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the `map` element whose key is equivalent to `k`.

6 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class... Args>
constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
```

```

template<class... Args>
constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
7   Preconditions: value_type is Cpp17EmplaceConstructible into map from piecewise_construct, forward_as_tuple(std::move(k)), forward_as_tuple(std::forward<Args>(args)...).
8   Effects: If the map already contains an element whose key is equivalent to k, there is no effect. Otherwise inserts an object of type value_type constructed with piecewise_construct, forward_as_tuple(std::move(k)), forward_as_tuple(std::forward<Args>(args)...).
9   Returns: In the first overload, the bool component of the returned pair is true if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to k.
10  Complexity: The same as emplace and emplace_hint, respectively.

template<class K, class... Args>
constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class K, class... Args>
constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
11  Constraints: The qualified-id Compare::is_transparent is valid and denotes a type. For the first overload, is_convertible_v<K&&, const_iterator> and is_convertible_v<K&&, iterator> are both false.
12  Preconditions: value_type is Cpp17EmplaceConstructible into map from piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...).
13  Effects: If the map already contains an element whose key is equivalent to k, there is no effect. Otherwise, let r be equal_range(k). Constructs an object u of type value_type with piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...). If equal_range(u.first) == r is false, the behavior is undefined. Inserts u into *this.
14  Returns: For the first overload, the bool component of the returned pair is true if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to k.
15  Complexity: The same as emplace and emplace_hint, respectively.

template<class M>
constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
16  Mandates: is_assignable_v<mapped_type&, M&&> is true.
17  Preconditions: value_type is Cpp17EmplaceConstructible into map from k, std::forward<M>(obj).
18  Effects: If the map already contains an element e whose key is equivalent to k, assigns std::forward<M>(obj) to e.second. Otherwise inserts an object of type value_type constructed with k, std::forward<M>(obj).
19  Returns: In the first overload, the bool component of the returned pair is true if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to k.
20  Complexity: The same as emplace and emplace_hint, respectively.

template<class M>
constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template<class M>
constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
21  Mandates: is_assignable_v<mapped_type&, M&&> is true.
22  Preconditions: value_type is Cpp17EmplaceConstructible into map from std::move(k), std::forward<M>(obj).
23  Effects: If the map already contains an element e whose key is equivalent to k, assigns std::forward<M>(obj) to e.second. Otherwise inserts an object of type value_type constructed with std::move(k), std::forward<M>(obj).
24  Returns: In the first overload, the bool component of the returned pair is true if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to k.
25  Complexity: The same as emplace and emplace_hint, respectively.

```

```

template<class K, class M>
constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
template<class K, class M>
constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);

```

- 26 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type.
- 27 *Mandates:* `is_assignable_v<mapped_type&, M&&>` is true.
- 28 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `map` from `std::forward<K>(k)`, `std::forward<M>(obj)`.
- 29 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, let `r` be `equal_range(k)`. Constructs an object `u` of type `value_type` with `std::forward<K>(k)`, `std::forward<M>(obj)`. If `equal_range(u.first) == r` is false, the behavior is undefined. Inserts `u` into `*this`.
- 30 *Returns:* For the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.
- 31 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

23.4.3.5 Erasure

[map.erasure]

```

template<class Key, class T, class Compare, class Allocator, class Predicate>
typename map<Key, T, Compare, Allocator>::size_type
constexpr erase_if(map<Key, T, Compare, Allocator>& c, Predicate pred);

```

- 1 *Effects:* Equivalent to:
- ```

auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
}
return original_size - c.size();

```

## 23.4.4 Class template multimap

[multimap]

### 23.4.4.1 Overview

[multimap.overview]

- 1 A `multimap` is an associative container that supports equivalent keys (i.e., possibly containing multiple copies of the same key value) and provides for fast retrieval of values of another type `T` based on the keys. The `multimap` class supports bidirectional iterators.
- 2 A `multimap` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of an associative container (23.2.7). A `multimap` also provides most operations described in 23.2.7 for equal keys. This means that a `multimap` supports the `a_eq` operations in 23.2.7 but not the `a_uniq` operations. For a `multimap<Key, T>` the `key_type` is `Key` and the `value_type` is `pair<const Key, T>`. Descriptions are provided here only for operations on `multimap` that are not described in one of those tables or for operations where there is additional semantic information.
- 3 The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
 template<class Key, class T, class Compare = less<Key>,
 class Allocator = allocator<pair<const Key, T>>>
 class multimap {
 public:
 // types
 using key_type = Key;
 using mapped_type = T;
 using value_type = pair<const Key, T>;
 using key_compare = Compare;
 using allocator_type = Allocator;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;

```

```

using reference = value_type&;
using const_reference = const value_type&;
using size_type = implementation-defined; // see 23.2
using difference_type = implementation-defined; // see 23.2
using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
using node_type = unspecified;

class value_compare {
protected:
 Compare comp;
 constexpr value_compare(Compare c) : comp(c) { }

public:
 constexpr bool operator()(const value_type& x, const value_type& y) const {
 return comp(x.first, y.first);
 }
};

// 23.4.4.2, construct/copy/destroy
constexpr multimap() : multimap(Compare()) { }
constexpr explicit multimap(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
 constexpr multimap(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());
template<container-compatible-range<value_type> R>
 constexpr multimap(from_range_t, R&& rg,
 const Compare& comp = Compare(), const Allocator& = Allocator());
constexpr multimap(const multimap& x);
constexpr multimap(multimap&& x);
constexpr explicit multimap(const Allocator&);
constexpr multimap(const multimap&, const type_identity_t<Allocator>&);
constexpr multimap(multimap&&, const type_identity_t<Allocator>&);
constexpr multimap(initializer_list<value_type>,
 const Compare& = Compare(), const Allocator& = Allocator());
template<class InputIterator>
 constexpr multimap(InputIterator first, InputIterator last, const Allocator& a)
 : multimap(first, last, Compare(), a) { }
template<container-compatible-range<value_type> R>
 constexpr multimap(from_range_t, R&& rg, const Allocator& a)
 : multimap(from_range, std::forward<R>(rg), Compare(), a) { }
constexpr multimap(initializer_list<value_type> il, const Allocator& a)
 : multimap(il, Compare(), a) { }
constexpr ~multimap();
constexpr multimap& operator=(const multimap& x);
constexpr multimap& operator=(multimap&& x)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Compare>);
constexpr multimap& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

```

```

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.4.4.3, modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr iterator insert(const value_type& x);
constexpr iterator insert(value_type&& x);
template<class P> constexpr iterator insert(P&& x);
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
template<class P> constexpr iterator insert(const_iterator position, P&& x);
template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> node_type extract(K&& x);
constexpr iterator insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(multimap&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Compare>);
constexpr void clear() noexcept;

template<class C2>
 constexpr void merge(multimap<Key, T, C2, Allocator>& source);
template<class C2>
 constexpr void merge(multimap<Key, T, C2, Allocator>&& source);
template<class C2>
 constexpr void merge(map<Key, T, C2, Allocator>& source);
template<class C2>
 constexpr void merge(map<Key, T, C2, Allocator>&& source);

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// map operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

```



```

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>,
 class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
 multimap(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
 -> multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 Compare, Allocator>;

template<ranges::input_range R, class Compare = less<range-key-type<R>>,
 class Allocator = allocator<range-to-alloc-type<R>>>
 multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> multimap<range-key-type<R>, range-mapped-type<R>, Compare, Allocator>;

template<class Key, class T, class Compare = less<Key>,
 class Allocator = allocator<pair<const Key, T>>>
 multimap(initializer_list<pair<Key, T>>, Compare = Compare(), Allocator = Allocator())
 -> multimap<Key, T, Compare, Allocator>;

template<class InputIterator, class Allocator>
 multimap(InputIterator, InputIterator, Allocator)
 -> multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 less<iter-key-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
 multimap(from_range_t, R&&, Allocator)
 -> multimap<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
 multimap(initializer_list<pair<Key, T>>, Allocator)
 -> multimap<Key, T, less<Key>, Allocator>;
}

```

#### 23.4.4.2 Constructors

[multimap.cons]

```
constexpr explicit multimap(const Compare& comp, const Allocator& = Allocator());
```

<sup>1</sup> *Effects:* Constructs an empty multimap using the specified comparison object and allocator.

<sup>2</sup> *Complexity:* Constant.

```
template<class InputIterator>
 constexpr multimap(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());
```

<sup>3</sup> *Effects:* Constructs an empty multimap using the specified comparison object and allocator, and inserts elements from the range [first, last).

- 4 *Complexity:* Linear in  $N$  if the range `[first, last)` is already sorted with respect to `comp` and otherwise  $N \log N$ , where  $N$  is `last - first`.

```
template<container-compatible-range<value_type> R>
constexpr multimap(from_range_t, R&& rg,
 const Compare& comp = Compare(), const Allocator& = Allocator());
```

- 5 *Effects:* Constructs an empty multimap using the specified comparison object and allocator, and inserts elements from the range `rg`.

- 6 *Complexity:* Linear in  $N$  if `rg` is already sorted with respect to `comp` and otherwise  $N \log N$ , where  $N$  is `ranges::distance(rg)`.

#### 23.4.4.3 Modifiers

[multimap.modifiers]

```
template<class P> constexpr iterator insert(P&& x);
template<class P> constexpr iterator insert(const_iterator position, P&& x);
```

- 1 *Constraints:* `is_constructible_v<value_type, P&&>` is true.

- 2 *Effects:* The first form is equivalent to `return.emplace(std::forward<P>(x))`. The second form is equivalent to `return.emplace_hint(position, std::forward<P>(x))`.

#### 23.4.4.4 Erasure

[multimap.erase]

```
template<class Key, class T, class Compare, class Allocator, class Predicate>
typename multimap<Key, T, Compare, Allocator>::size_type
constexpr erase_if(multimap<Key, T, Compare, Allocator>& c, Predicate pred);
```

- 1 *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
}
return original_size - c.size();
```

#### 23.4.5 Header <set> synopsis

[associative.set.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2
```

```
namespace std {
 // 23.4.6, class template set
 template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
 class set;
```

```
 template<class Key, class Compare, class Allocator>
 constexpr bool operator==(const set<Key, Compare, Allocator>& x,
 const set<Key, Compare, Allocator>& y);
 template<class Key, class Compare, class Allocator>
 constexpr synth-three-way-result<Key>
 operator<=>(const set<Key, Compare, Allocator>& x,
 const set<Key, Compare, Allocator>& y);
```

```
 template<class Key, class Compare, class Allocator>
 constexpr void swap(set<Key, Compare, Allocator>& x,
 set<Key, Compare, Allocator>& y)
 noexcept(noexcept(x.swap(y)));
```

// 23.4.6.3, erasure for set

```
 template<class Key, class Compare, class Allocator, class Predicate>
 constexpr typename set<Key, Compare, Allocator>::size_type
 erase_if(set<Key, Compare, Allocator>& c, Predicate pred);
```

```

// 23.4.7, class template multiset
template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
 class multiset;

template<class Key, class Compare, class Allocator>
 constexpr bool operator==(const multiset<Key, Compare, Allocator>& x,
 const multiset<Key, Compare, Allocator>& y);
template<class Key, class Compare, class Allocator>
 constexpr synth-three-way-result<Key>
 operator<=>(const multiset<Key, Compare, Allocator>& x,
 const multiset<Key, Compare, Allocator>& y);

template<class Key, class Compare, class Allocator>
 constexpr void swap(multiset<Key, Compare, Allocator>& x,
 multiset<Key, Compare, Allocator>& y)
 noexcept(noexcept(x.swap(y)));

// 23.4.7.3, erasure for multiset
template<class Key, class Compare, class Allocator, class Predicate>
 constexpr typename multiset<Key, Compare, Allocator>::size_type
 erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);

namespace pmr {
 template<class Key, class Compare = less<Key>>
 using set = std::set<Key, Compare, polymorphic_allocator<Key>>;

 template<class Key, class Compare = less<Key>>
 using multiset = std::multiset<Key, Compare, polymorphic_allocator<Key>>;
}

```

## 23.4.6 Class template set

[set]

### 23.4.6.1 Overview

[set.overview]

- <sup>1</sup> A **set** is an associative container that supports unique keys (i.e., contains at most one of each key value) and provides for fast retrieval of the keys themselves. The **set** class supports bidirectional iterators.
- <sup>2</sup> A **set** meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of an associative container (23.2.7). A **set** also provides most operations described in 23.2.7 for unique keys. This means that a **set** supports the **a\_uniq** operations in 23.2.7 but not the **a\_eq** operations. For a **set<Key>** both the **key\_type** and **value\_type** are **Key**. Descriptions are provided here only for operations on **set** that are not described in one of these tables and for operations where there is additional semantic information.
- <sup>3</sup> The types **iterator** and **const\_iterator** meet the **constexpr** iterator requirements (24.3.1).

```

namespace std {
 template<class Key, class Compare = less<Key>,
 class Allocator = allocator<Key>>
 class set {
 public:
 // types
 using key_type = Key;
 using key_compare = Compare;
 using value_type = Key;
 using value_compare = Compare;
 using allocator_type = Allocator;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;
 using reference = value_type&;
 using const_reference = const value_type&;
 using size_type = implementation-defined; // see 23.2
 using difference_type = implementation-defined; // see 23.2
 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 };
}

```

```

using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
using node_type = unspecified;
using insert_return_type = insert-return-type<iterator, node_type>;

// 23.4.6.2, construct/copy/destroy
constexpr set() : set(Compare()) { }
constexpr explicit set(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
 constexpr set(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());
template<container-compatible-range<value_type> R>
 constexpr set(from_range_t, R&& rg,
 const Compare& comp = Compare(), const Allocator& = Allocator());
constexpr set(const set& x);
constexpr set(set&& x);
constexpr explicit set(const Allocator&);
constexpr set(const set&, const type_identity_t<Allocator>&);
constexpr set(set&&, const type_identity_t<Allocator>&);
constexpr set(initializer_list<value_type>,
 const Compare& = Compare(), const Allocator& = Allocator());
template<class InputIterator>
 constexpr set(InputIterator first, InputIterator last, const Allocator& a)
 : set(first, last, Compare(), a) { }
template<container-compatible-range<value_type> R>
 constexpr set(from_range_t, R&& rg, const Allocator& a)
 : set(from_range, std::forward<R>(rg), Compare(), a) { }
constexpr set(initializer_list<value_type> il, const Allocator& a)
 : set(il, Compare(), a) { }
constexpr ~set();
constexpr set& operator=(const set& x);
constexpr set& operator=(set&& x)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Compare>);
constexpr set& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.4.6.4, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr pair<iterator, bool> insert(const value_type& x);
constexpr pair<iterator, bool> insert(value_type&& x);

```

```

template<class K> constexpr pair<iterator, bool> insert(K&& x);
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
template<class K> constexpr iterator insert(const_iterator position, K&& x);
template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr insert_return_type insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position)
 requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(set&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Compare>);
constexpr void clear() noexcept;

template<class C2>
 constexpr void merge(set<Key, C2, Allocator>& source);
template<class C2>
 constexpr void merge(set<Key, C2, Allocator>&& source);
template<class C2>
 constexpr void merge(multiset<Key, C2, Allocator>& source);
template<class C2>
 constexpr void merge(multiset<Key, C2, Allocator>&& source);

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// set operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;

```

```

template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class InputIterator,
 class Compare = less<iter-value-type<InputIterator>>,
 class Allocator = allocator<iter-value-type<InputIterator>>>
set(InputIterator, InputIterator,
 Compare = Compare(), Allocator = Allocator())
 -> set<iter-value-type<InputIterator>, Compare, Allocator>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
set(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> set<ranges::range_value_t<R>, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
set(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
 -> set<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
set(InputIterator, InputIterator, Allocator)
 -> set<iter-value-type<InputIterator>,
 less<iter-value-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
set(from_range_t, R&&, Allocator)
 -> set<ranges::range_value_t<R>, less<ranges::range_value_t<R>>, Allocator>;

template<class Key, class Allocator>
set(initializer_list<Key>, Allocator) -> set<Key, less<Key>, Allocator>;
}

```

#### 23.4.6.2 Constructors, copy, and assignment

[set.cons]

```
constexpr explicit set(const Compare& comp, const Allocator& = Allocator());
```

1     *Effects:* Constructs an empty set using the specified comparison object and allocator.

2     *Complexity:* Constant.

```

template<class InputIterator>
constexpr set(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());

```

3     *Effects:* Constructs an empty set using the specified comparison object and allocator, and inserts elements from the range [first, last).

4     *Complexity:* Linear in  $N$  if the range [first, last) is already sorted with respect to comp and otherwise  $N \log N$ , where  $N$  is last - first.

```

template<container-compatible-range<value_type> R>
constexpr set(from_range_t, R&& rg, const Compare& comp = Compare(),
 const Allocator& = Allocator());

```

5     *Effects:* Constructs an empty set using the specified comparison object and allocator, and inserts elements from the range rg.

6     *Complexity:* Linear in  $N$  if rg is already sorted with respect to comp and otherwise  $N \log N$ , where  $N$  is ranges::distance(rg).

## 23.4.6.3 Erasure

[set.erasure]

```
template<class Key, class Compare, class Allocator, class Predicate>
constexpr typename set<Key, Compare, Allocator>::size_type
erase_if(set<Key, Compare, Allocator>& c, Predicate pred);
```

1 *Effects:* Equivalent to:

```
 auto original_size = c.size();
 for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
 }
 return original_size - c.size();
```

## 23.4.6.4 Modifiers

[set.modifiers]

```
template<class K> constexpr pair<iterator, bool> insert(K&& x);
template<class K> constexpr iterator insert(const_iterator hint, K&& x);
```

1 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type. For the second overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both false.

2 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `set` from `std::forward<K>(x)`.

3 *Effects:* If the set already contains an element that is equivalent to `x`, there is no effect. Otherwise, let `r` be `equal_range(x)`. Constructs an object `u` of type `value_type` with `std::forward<K>(x)`. If `equal_range(u) == r` is false, the behavior is undefined. Inserts `u` into `*this`.

4 *Returns:* For the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the set element that is equivalent to `x`.

5 *Complexity:* Logarithmic.

23.4.7 Class template `multiset`

[multiset]

## 23.4.7.1 Overview

[multiset.overview]

1 A `multiset` is an associative container that supports equivalent keys (i.e., possibly contains multiple copies of the same key value) and provides for fast retrieval of the keys themselves. The `multiset` class supports bidirectional iterators.

2 A `multiset` meets all of the requirements of a container (23.2.2.2), of a reversible container (23.2.2.3), of an allocator-aware container (23.2.2.5), and of an associative container (23.2.7). `multiset` also provides most operations described in 23.2.7 for duplicate keys. This means that a `multiset` supports the `a_eq` operations in 23.2.7 but not the `a_uniq` operations. For a `multiset<Key>` both the `key_type` and `value_type` are `Key`. Descriptions are provided here only for operations on `multiset` that are not described in one of these tables and for operations where there is additional semantic information.

3 The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

```
namespace std {
 template<class Key, class Compare = less<Key>,
 class Allocator = allocator<Key>>
 class multiset {
 public:
 // types
 using key_type = Key;
 using key_compare = Compare;
 using value_type = Key;
 using value_compare = Compare;
 using allocator_type = Allocator;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;
 using reference = value_type&;
 using const_reference = const value_type&;
```

```

using size_type = implementation-defined; // see 23.2
using difference_type = implementation-defined; // see 23.2
using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
using node_type = unspecified;

// 23.4.7.2, construct/copy/destroy
constexpr multiset() : multiset(Compare()) { }
constexpr explicit multiset(const Compare& comp, const Allocator& = Allocator());
template<class InputIterator>
 constexpr multiset(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());
template<container-compatible-range<value_type> R>
 constexpr multiset(from_range_t, R&& rg,
 const Compare& comp = Compare(), const Allocator& = Allocator());
constexpr multiset(const multiset& x);
constexpr multiset(multiset&& x);
constexpr explicit multiset(const Allocator&);
constexpr multiset(const multiset&, const type_identity_t<Allocator>&);
constexpr multiset(multiset&&, const type_identity_t<Allocator>&);
constexpr multiset(initializer_list<value_type>, const Compare& = Compare(),
 const Allocator& = Allocator());
template<class InputIterator>
 constexpr multiset(InputIterator first, InputIterator last, const Allocator& a)
 : multiset(first, last, Compare(), a) { }
template<container-compatible-range<value_type> R>
 constexpr multiset(from_range_t, R&& rg, const Allocator& a)
 : multiset(from_range, std::forward<R>(rg), Compare(), a) { }
constexpr multiset(initializer_list<value_type> il, const Allocator& a)
 : multiset(il, Compare(), a) { }
constexpr ~multiset();
constexpr multiset& operator=(const multiset& x);
constexpr multiset& operator=(multiset&& x)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Compare>);
constexpr multiset& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);

```



```

template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr iterator insert(const value_type& x);
constexpr iterator insert(value_type&& x);
constexpr iterator insert(const_iterator position, const value_type& x);
constexpr iterator insert(const_iterator position, value_type&& x);
template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr iterator insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position)
 requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(multiset&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Compare>);
constexpr void clear() noexcept;

template<class C2>
 constexpr void merge(multiset<Key, C2, Allocator>& source);
template<class C2>
 constexpr void merge(multiset<Key, C2, Allocator>&& source);
template<class C2>
 constexpr void merge(set<Key, C2, Allocator>& source);
template<class C2>
 constexpr void merge(set<Key, C2, Allocator>&& source);

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// set operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

```

```

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class InputIterator,
 class Compare = less<iter-value-type<InputIterator>>,
 class Allocator = allocator<iter-value-type<InputIterator>>>
 multiset(InputIterator, InputIterator,
 Compare = Compare(), Allocator = Allocator())
 -> multiset<iter-value-type<InputIterator>, Compare, Allocator>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
 multiset(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> multiset<ranges::range_value_t<R>, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
 multiset(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
 -> multiset<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
 multiset(InputIterator, InputIterator, Allocator)
 -> multiset<iter-value-type<InputIterator>,
 less<iter-value-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
 multiset(from_range_t, R&&, Allocator)
 -> multiset<ranges::range_value_t<R>, less<ranges::range_value_t<R>>, Allocator>;

template<class Key, class Allocator>
 multiset(initializer_list<Key>, Allocator) -> multiset<Key, less<Key>, Allocator>;
}

```

#### 23.4.7.2 Constructors

[multiset.cons]

```
constexpr explicit multiset(const Compare& comp, const Allocator& = Allocator());
```

1     *Effects:* Constructs an empty `multiset` using the specified comparison object and allocator.

2     *Complexity:* Constant.

```

template<class InputIterator>
 constexpr multiset(InputIterator first, InputIterator last,
 const Compare& comp = Compare(), const Allocator& = Allocator());

```

3     *Effects:* Constructs an empty `multiset` using the specified comparison object and allocator, and inserts elements from the range `[first, last)`.

4     *Complexity:* Linear in  $N$  if the range `[first, last)` is already sorted with respect to `comp` and otherwise  $N \log N$ , where  $N$  is `last - first`.

```

template<container-compatible-range<value_type> R>
 constexpr multiset(from_range_t, R&& rg, const Compare& comp = Compare(),
 const Allocator& = Allocator());

```

5     *Effects:* Constructs an empty `multiset` using the specified comparison object and allocator, and inserts elements from the range `rg`.

6     *Complexity:* Linear in  $N$  if `rg` is already sorted with respect to `comp` and otherwise  $N \log N$ , where  $N$  is `ranges::distance(rg)`.

## 23.4.7.3 Erasure

[multiset.erase]

```
template<class Key, class Compare, class Allocator, class Predicate>
constexpr typename multiset<Key, Compare, Allocator>::size_type
erase_if(multiset<Key, Compare, Allocator>& c, Predicate pred);
```

<sup>1</sup> *Effects:* Equivalent to:

```
auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
}
return original_size - c.size();
```

## 23.5 Unordered associative containers

[unord]

## 23.5.1 General

[unord.general]

- <sup>1</sup> The header `<unordered_map>` (23.5.2) defines the class templates `unordered_map` and `unordered_multimap`; the header `<unordered_set>` (23.5.5) defines the class templates `unordered_set` and `unordered_multiset`.
- <sup>2</sup> The exposition-only alias templates *iter-value-type*, *iter-key-type*, *iter-mapped-type*, *iter-to-alloc-type*, *range-key-type*, *range-mapped-type*, and *range-to-alloc-type* defined in 23.4.1 may appear in deduction guides for unordered containers.

23.5.2 Header `<unordered_map>` synopsis

[unord.map.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.5.3, class template unordered_map
 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Alloc = allocator<pair<const Key, T>>>
 class unordered_map;

 // 23.5.4, class template unordered_multimap
 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Alloc = allocator<pair<const Key, T>>>
 class unordered_multimap;

 template<class Key, class T, class Hash, class Pred, class Alloc>
 constexpr bool operator==(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
 const unordered_map<Key, T, Hash, Pred, Alloc>& b);

 template<class Key, class T, class Hash, class Pred, class Alloc>
 constexpr bool operator==(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
 const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);

 template<class Key, class T, class Hash, class Pred, class Alloc>
 constexpr void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,
 unordered_map<Key, T, Hash, Pred, Alloc>& y)
 noexcept(noexcept(x.swap(y)));
```

```

template<class Key, class T, class Hash, class Pred, class Alloc>
constexpr void swap(unordered_multimap<Key, T, Hash, Pred, Alloc>& x,
 unordered_multimap<Key, T, Hash, Pred, Alloc>& y)
 noexcept(noexcept(x.swap(y)));

// 23.5.3.5, erasure for unordered_map
template<class K, class T, class H, class P, class A, class Predicate>
constexpr typename unordered_map<K, T, H, P, A>::size_type
 erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);

// 23.5.4.4, erasure for unordered_multimap
template<class K, class T, class H, class P, class A, class Predicate>
constexpr typename unordered_multimap<K, T, H, P, A>::size_type
 erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);

namespace pmr {
 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>>
 using unordered_map =
 std::unordered_map<Key, T, Hash, Pred,
 polymorphic_allocator<pair<const Key, T>>>;

 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>>
 using unordered_multimap =
 std::unordered_multimap<Key, T, Hash, Pred,
 polymorphic_allocator<pair<const Key, T>>>;

}
}

```

### 23.5.3 Class template unordered\_map

[unord.map]

#### 23.5.3.1 Overview

[unord.map.overview]

- <sup>1</sup> An `unordered_map` is an unordered associative container that supports unique keys (an `unordered_map` contains at most one of each key value) and that associates values of another type `mapped_type` with the keys. The `unordered_map` class supports forward iterators.
- <sup>2</sup> An `unordered_map` meets all of the requirements of a container (23.2.2.2), of an allocator-aware container (23.2.2.5), and of an unordered associative container (23.2.8). It provides the operations described in the preceding requirements table for unique keys; that is, an `unordered_map` supports the `a_uniq` operations in that table, not the `a_eq` operations. For an `unordered_map<Key, T>` the `key_type` is `Key`, the `mapped_type` is `T`, and the `value_type` is `pair<const Key, T>`.
- <sup>3</sup> Subclause 23.5.3 only describes operations on `unordered_map` that are not described in one of the requirement tables, or for which there is additional semantic information.
- <sup>4</sup> The types `iterator`, `const_iterator`, `local_iterator`, and `const_local_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Allocator = allocator<pair<const Key, T>>>
 class unordered_map {
 public:
 // types
 using key_type = Key;
 using mapped_type = T;
 using value_type = pair<const Key, T>;
 };
}

```

```

using hasher = Hash;
using key_equal = Pred;
using allocator_type = Allocator;
using pointer = allocator_traits<Allocator>::pointer;
using const_pointer = allocator_traits<Allocator>::const_pointer;
using reference = value_type&;
using const_reference = const value_type&;
using size_type = implementation-defined; // see 23.2
using difference_type = implementation-defined; // see 23.2

using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using local_iterator = implementation-defined; // see 23.2
using const_local_iterator = implementation-defined; // see 23.2
using node_type = unspecified;
using insert_return_type = insert-return-type<iterator, node_type>;

// 23.5.3.2, construct/copy/destroy
constexpr unordered_map();
constexpr explicit unordered_map(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<class InputIterator>
 constexpr unordered_map(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<container-compatible-range<value_type> R>
 constexpr unordered_map(from_range_t, R&& rg, size_type n = see below,
 const hasher& hf = hasher(), const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_map(const unordered_map&);
constexpr unordered_map(unordered_map&&);
constexpr explicit unordered_map(const Allocator&);
constexpr unordered_map(const unordered_map&, const type_identity_t<Allocator>&);
constexpr unordered_map(unordered_map&&, const type_identity_t<Allocator>&);
constexpr unordered_map(initializer_list<value_type> il, size_type n = see below,
 const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
constexpr unordered_map(size_type n, const allocator_type& a)
 : unordered_map(n, hasher(), key_equal(), a) { }
constexpr unordered_map(size_type n, const hasher& hf, const allocator_type& a)
 : unordered_map(n, hf, key_equal(), a) { }
template<class InputIterator>
 constexpr unordered_map(InputIterator f, InputIterator l, size_type n,
 const allocator_type& a)
 : unordered_map(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
 constexpr unordered_map(InputIterator f, InputIterator l, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_map(f, l, n, hf, key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_map(from_range_t, R&& rg, size_type n, const allocator_type& a)
 : unordered_map(from_range, std::forward<R>(rg), n, hasher(), key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_map(from_range_t, R&& rg, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_map(from_range, std::forward<R>(rg), n, hf, key_equal(), a) { }
constexpr unordered_map(initializer_list<value_type> il, size_type n,
 const allocator_type& a)
 : unordered_map(il, n, hasher(), key_equal(), a) { }

```

```

constexpr unordered_map(initializer_list<value_type> il, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_map(il, n, hf, key_equal(), a) { }
constexpr ~unordered_map();
constexpr unordered_map& operator=(const unordered_map&);
constexpr unordered_map& operator=(unordered_map&&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Hash> &&
 is_nothrow_move_assignable_v<Pred>);
constexpr unordered_map& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.5.3.4, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr pair<iterator, bool> insert(const value_type& obj);
constexpr pair<iterator, bool> insert(value_type&& obj);
template<class P> constexpr pair<iterator, bool> insert(P&& obj);
constexpr iterator insert(const_iterator hint, const value_type& obj);
constexpr iterator insert(const_iterator hint, value_type&& obj);
template<class P> constexpr iterator insert(const_iterator hint, P&& obj);
template<class InputIterator> constexpr void insert(InputIterator first, InputIterator last);
template<container_compatible_range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr insert_return_type insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

template<class... Args>
 constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
 constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template<class K, class... Args>
 constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class... Args>
 constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
 constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template<class K, class... Args>
 constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
template<class M>
 constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
 constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template<class K, class M>
 constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);

```

```

template<class M>
 constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
 constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
template<class K, class M>
 constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& k);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(unordered_map&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Hash> && is_nothrow_swappable_v<Pred>);
constexpr void clear() noexcept;

template<class H2, class P2>
 constexpr void merge(unordered_map<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_map<Key, T, H2, P2, Allocator>&& source);
template<class H2, class P2>
 constexpr void merge(unordered_multimap<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_multimap<Key, T, H2, P2, Allocator>&& source);

// observers
constexpr hasher hash_function() const;
constexpr key_equal key_eq() const;

// map operations
constexpr iterator find(const key_type& k);
constexpr const_iterator find(const key_type& k) const;
template<class K>
 constexpr iterator find(const K& k);
template<class K>
 constexpr const_iterator find(const K& k) const;
constexpr size_type count(const key_type& k) const;
template<class K>
 constexpr size_type count(const K& k) const;
constexpr bool contains(const key_type& k) const;
template<class K>
 constexpr bool contains(const K& k) const;
constexpr pair<iterator, iterator> equal_range(const key_type& k);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& k);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& k) const;

// 23.5.3.3, element access
constexpr mapped_type& operator[](const key_type& k);
constexpr mapped_type& operator[](key_type&& k);
template<class K> constexpr mapped_type& operator[](K&& k);
constexpr mapped_type& at(const key_type& k);
constexpr const mapped_type& at(const key_type& k) const;
template<class K> constexpr mapped_type& at(const K& k);
template<class K> constexpr const mapped_type& at(const K& k) const;
constexpr optional<mapped_type&> lookup(const key_type& k);
constexpr optional<const mapped_type&> lookup(const key_type& k) const;
template<class K> constexpr optional<mapped_type&> lookup(const K& k);
template<class K> constexpr optional<const mapped_type&> lookup(const K& k) const;

```

```

// bucket interface
constexpr size_type bucket_count() const noexcept;
constexpr size_type max_bucket_count() const noexcept;
constexpr size_type bucket_size(size_type n) const;
constexpr size_type bucket(const key_type& k) const;
template<class K> constexpr size_type bucket(const K& k) const;
constexpr local_iterator begin(size_type n);
constexpr const_local_iterator begin(size_type n) const;
constexpr local_iterator end(size_type n);
constexpr const_local_iterator end(size_type n) const;
constexpr const_local_iterator cbegin(size_type n) const;
constexpr const_local_iterator cend(size_type n) const;

// hash policy
constexpr float load_factor() const noexcept;
constexpr float max_load_factor() const noexcept;
constexpr void max_load_factor(float z);
constexpr void rehash(size_type n);
constexpr void reserve(size_type n);
};

template<class InputIterator,
 class Hash = hash<iter-key-type<InputIterator>>,
 class Pred = equal_to<iter-key-type<InputIterator>>,
 class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
unordered_map(InputIterator, InputIterator, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash, Pred,
 Allocator>;

template<ranges::input_range R, class Hash = hash<range-key-type<R>>,
 class Pred = equal_to<range-key-type<R>>,
 class Allocator = allocator<range-to-alloc-type<R>>>
unordered_map(from_range_t, R&&, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<range-key-type<R>, range-mapped-type<R>, Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
 class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_map(initializer_list<pair<Key, T>>,
 typename see below::size_type = see below, Hash = Hash(),
 Pred = Pred(), Allocator = Allocator())
-> unordered_map<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 hash<iter-key-type<InputIterator>>,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 hash<iter-key-type<InputIterator>>,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_type, Hash, Allocator)
-> unordered_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

```



```

template<ranges::input_range R, class Allocator>
 unordered_map(from_range_t, R&&, typename see below::size_type, Allocator)
 -> unordered_map<range-key-type<R>, range-mapped-type<R>, hash<range-key-type<R>>,
 equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
 unordered_map(from_range_t, R&&, Allocator)
 -> unordered_map<range-key-type<R>, range-mapped-type<R>, hash<range-key-type<R>>,
 equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
 unordered_map(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
 -> unordered_map<range-key-type<R>, range-mapped-type<R>, Hash,
 equal_to<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
 unordered_map(initializer_list<pair<Key, T>>, typename see below::size_type,
 Allocator)
 -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Allocator>
 unordered_map(initializer_list<pair<Key, T>>, Allocator)
 -> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
 unordered_map(initializer_list<pair<Key, T>>, typename see below::size_type, Hash,
 Allocator)
 -> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;
}

```

- 5 A `size_type` parameter type in an `unordered_map` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

### 23.5.3.2 Constructors

[unord.map.cnstr]

```

constexpr unordered_map() : unordered_map(size_type(see below)) { }
constexpr explicit unordered_map(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- 1 *Effects:* Constructs an empty `unordered_map` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.
- 2 *Complexity:* Constant.

```

template<class InputIterator>
 constexpr unordered_map(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<container-compatible-range<value_type> R>
 constexpr unordered_map(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_map(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- 3 *Effects:* Constructs an empty `unordered_map` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f, l)`, `rg`, or `il`, respectively. `max_load_factor()` returns 1.0.
- 4 *Complexity:* Average case linear, worst case quadratic.

**23.5.3.3 Element access****[unord.map.elem]**

```
constexpr mapped_type& operator[](const key_type& k);
```

1     *Effects:* Equivalent to: `return try_emplace(k).first->second;`

```
constexpr mapped_type& operator[](key_type&& k);
```

2     *Effects:* Equivalent to: `return try_emplace(std::move(k)).first->second;`

```
template<class K> constexpr mapped_type& operator[](K&& k);
```

3     *Constraints:* The *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

4     *Effects:* Equivalent to: `return try_emplace(std::forward<K>(k)).first->second;`

```
constexpr mapped_type& at(const key_type& k);
```

```
constexpr const mapped_type& at(const key_type& k) const;
```

5     *Returns:* A reference to `x.second`, where `x` is the (unique) element whose key is equivalent to `k`.

6     *Throws:* An exception object of type `out_of_range` if no such element is present.

```
template<class K> constexpr mapped_type& at(const K& k);
```

```
template<class K> constexpr const mapped_type& at(const K& k) const;
```

7     *Constraints:* The *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

8     *Preconditions:* The expression `find(k)` is well-formed and has well-defined behavior.

9     *Returns:* A reference to `find(k)->second`.

10    *Throws:* An exception object of type `out_of_range` if `find(k) == end()` is true.

```
constexpr optional<mapped_type&> lookup(const key_type& k);
```

```
constexpr optional<const mapped_type&> lookup(const key_type& k) const;
```

```
template<class K> constexpr optional<mapped_type&> lookup(const K& k);
```

```
template<class K> constexpr optional<const mapped_type&> lookup(const K& k) const;
```

11    *Constraints:* For the third and fourth overloads, the *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

12    *Preconditions:* The expression `find(k)` is well-formed and has well-defined behavior.

13    *Returns:* `find(k)->second` if `contains(k)` is true, otherwise `nullopt`.

14    *Complexity:* Average case constant, worst case linear in `size()`.

**23.5.3.4 Modifiers****[unord.map.modifiers]**

```
template<class P>
```

```
constexpr pair<iterator, bool> insert(P&& obj);
```

1     *Constraints:* `is_constructible_v<value_type, P&&>` is true.

2     *Effects:* Equivalent to: `return emplace(std::forward<P>(obj));`

```
template<class P>
```

```
constexpr iterator insert(const_iterator hint, P&& obj);
```

3     *Constraints:* `is_constructible_v<value_type, P&&>` is true.

4     *Effects:* Equivalent to: `return emplace_hint(hint, std::forward<P>(obj));`

```
template<class... Args>
```

```
constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
```

```
template<class... Args>
```

```
constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
```

5     *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(std::forward<Args>(args)...)...`.

6 *Effects:* If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_tuple(k), forward_as_tuple(std::forward<Args>(args)...) .`

7 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

8 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class... Args>
constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template<class... Args>
constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

9 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `piecewise_construct, forward_as_tuple(std::move(k)), forward_as_tuple(std::forward<Args>(args)...) .`

10 *Effects:* If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct, forward_as_tuple(std::move(k)), forward_as_tuple(std::forward<Args>(args)...) .`

11 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

12 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class K, class... Args>
constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class K, class... Args>
constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

13 *Constraints:* The *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types. For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both false.

14 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...) .`

15 *Effects:* If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise, let `h` be `hash_function()(k)`. Constructs an object `u` of type `value_type` with `piecewise_construct, forward_as_tuple(std::forward<K>(k)), forward_as_tuple(std::forward<Args>(args)...) .` If `hash_function()(u.first) != h || contains(u.first)` is true, the behavior is undefined. Inserts `u` into `*this`.

16 *Returns:* For the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

17 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class M>
constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
```

18 *Mandates:* `is_assignable_v<mapped_type&, M&&>` is true.

19 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `unordered_map` from `k, std::forward<M>(obj) .`

20 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `k, std::forward<M>(obj) .`

21 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

22 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class M>
constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
```

```

template<class M>
constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
23 Mandates: is_assignable_v<mapped_type&, M&&> is true.
24 Preconditions: value_type is Cpp17EmplaceConstructible into unordered_map from std::move(k),
 std::forward<M>(obj).
25 Effects: If the map already contains an element e whose key is equivalent to k, assigns std::for-
 ward<M>(obj) to e.second. Otherwise inserts an object of type value_type constructed with std::
 move(k), std::forward<M>(obj).
26 Returns: In the first overload, the bool component of the returned pair is true if and only if the
 insertion took place. The returned iterator points to the map element whose key is equivalent to k.
27 Complexity: The same as emplace and emplace_hint, respectively.

template<class K, class M>
constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
template<class K, class M>
constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
28 Constraints: The qualified-ids Hash::is_transparent and Pred::is_transparent are valid and
 denote types.
29 Mandates: is_assignable_v<mapped_type&, M&&> is true.
30 Preconditions: value_type is Cpp17EmplaceConstructible into unordered_map from std::forward<K>
 (k), std::forward<M>(obj).
31 Effects: If the map already contains an element e whose key is equivalent to k, assigns std::forward<M>
 (obj) to e.second. Otherwise, let h be hash_function()(k). Constructs an object u of type value_-
 type with std::forward<K>(k), std::forward<M>(obj). If hash_function()(u.first) != h ||
 contains(u.first) is true, the behavior is undefined. Inserts u into *this.
32 Returns: For the first overload, the bool component of the returned pair is true if and only if the
 insertion took place. The returned iterator points to the map element whose key is equivalent to k.
33 Complexity: The same as emplace and emplace_hint, respectively.

```

### 23.5.3.5 Erasure

[unord.map.erase]

```

template<class K, class T, class H, class P, class A, class Predicate>
constexpr typename unordered_map<K, T, H, P, A>::size_type
erase_if(unordered_map<K, T, H, P, A>& c, Predicate pred);
1 Effects: Equivalent to:
 auto original_size = c.size();
 for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
 }
 return original_size - c.size();

```

### 23.5.4 Class template unordered\_multimap

[unord.multimap]

#### 23.5.4.1 Overview

[unord.multimap.overview]

- 1 An `unordered_multimap` is an unordered associative container that supports equivalent keys (an instance of `unordered_multimap` may contain multiple copies of each key value) and that associates values of another type `mapped_type` with the keys. The `unordered_multimap` class supports forward iterators.
- 2 An `unordered_multimap` meets all of the requirements of a container (23.2.2.2), of an allocator-aware container (23.2.2.5), and of an unordered associative container (23.2.8). It provides the operations described in the preceding requirements table for equivalent keys; that is, an `unordered_multimap` supports the `a_eq` operations in that table, not the `a_uniq` operations. For an `unordered_multimap<Key, T>` the `key_type` is `Key`, the `mapped_type` is `T`, and the `value_type` is `pair<const Key, T>`.

- <sup>3</sup> Subclause 23.5.4 only describes operations on `unordered_multimap` that are not described in one of the requirement tables, or for which there is additional semantic information.
- <sup>4</sup> The types `iterator`, `const_iterator`, `local_iterator`, and `const_local_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
 template<class Key,
 class T,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Allocator = allocator<pair<const Key, T>>>
 class unordered_multimap {
 public:
 // types
 using key_type = Key;
 using mapped_type = T;
 using value_type = pair<const Key, T>;
 using hasher = Hash;
 using key_equal = Pred;
 using allocator_type = Allocator;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;
 using reference = value_type&;
 using const_reference = const value_type&;
 using size_type = implementation-defined; // see 23.2
 using difference_type = implementation-defined; // see 23.2

 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 using local_iterator = implementation-defined; // see 23.2
 using const_local_iterator = implementation-defined; // see 23.2
 using node_type = unspecified;

 // 23.5.4.2, construct/copy/destroy
 constexpr unordered_multimap();
 constexpr explicit unordered_multimap(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

 template<class InputIterator>
 constexpr unordered_multimap(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
 template<container-compatible-range<value_type> R>
 constexpr unordered_multimap(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

 constexpr unordered_multimap(const unordered_multimap&);
 constexpr unordered_multimap(unordered_multimap&&);
 constexpr explicit unordered_multimap(const Allocator&);
 constexpr unordered_multimap(const unordered_multimap&, const type_identity_t<Allocator>&);
 constexpr unordered_multimap(unordered_multimap&&, const type_identity_t<Allocator>&);
 constexpr unordered_multimap(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

 constexpr unordered_multimap(size_type n, const allocator_type& a)
 : unordered_multimap(n, hasher(), key_equal(), a) { }
 constexpr unordered_multimap(size_type n, const hasher& hf, const allocator_type& a)
 : unordered_multimap(n, hf, key_equal(), a) { }

 template<class InputIterator>
 constexpr unordered_multimap(InputIterator f, InputIterator l, size_type n,
 const allocator_type& a)

```

```

 : unordered_multimap(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
 constexpr unordered_multimap(InputIterator f, InputIterator l, size_type n,
 const hasher& hf, const allocator_type& a)
 : unordered_multimap(f, l, n, hf, key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_multimap(from_range_t, R&& rg, size_type n, const allocator_type& a)
 : unordered_multimap(from_range, std::forward<R>(rg),
 n, hasher(), key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_multimap(from_range_t, R&& rg, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_multimap(from_range, std::forward<R>(rg), n, hf, key_equal(), a) { }
constexpr unordered_multimap(initializer_list<value_type> il, size_type n,
 const allocator_type& a)
 : unordered_multimap(il, n, hasher(), key_equal(), a) { }
constexpr unordered_multimap(initializer_list<value_type> il, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_multimap(il, n, hf, key_equal(), a) { }
constexpr ~unordered_multimap();
constexpr unordered_multimap& operator=(const unordered_multimap&);
constexpr unordered_multimap& operator=(unordered_multimap&&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Hash> && is_nothrow_move_assignable_v<Pred>);
constexpr unordered_multimap& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.5.4.3, modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr iterator insert(const value_type& obj);
constexpr iterator insert(value_type&& obj);
template<class P> constexpr iterator insert(P&& obj);
constexpr iterator insert(const_iterator hint, const value_type& obj);
constexpr iterator insert(const_iterator hint, value_type&& obj);
template<class P> constexpr iterator insert(const_iterator hint, P&& obj);
template<class InputIterator> constexpr void insert(InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr iterator insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& k);

```

```

template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(unordered_multimap&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Hash> && is_nothrow_swappable_v<Pred>);
constexpr void clear() noexcept;

template<class H2, class P2>
 constexpr void merge(unordered_multimap<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_multimap<Key, T, H2, P2, Allocator>&& source);
template<class H2, class P2>
 constexpr void merge(unordered_map<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_map<Key, T, H2, P2, Allocator>&& source);

// observers
constexpr hasher hash_function() const;
constexpr key_equal key_eq() const;

// map operations
constexpr iterator find(const key_type& k);
constexpr const_iterator find(const key_type& k) const;
template<class K>
 constexpr iterator find(const K& k);
template<class K>
 constexpr const_iterator find(const K& k) const;
constexpr size_type count(const key_type& k) const;
template<class K>
 constexpr size_type count(const K& k) const;
constexpr bool contains(const key_type& k) const;
template<class K>
 constexpr bool contains(const K& k) const;
constexpr pair<iterator, iterator> equal_range(const key_type& k);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& k);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& k) const;

// bucket interface
constexpr size_type bucket_count() const noexcept;
constexpr size_type max_bucket_count() const noexcept;
constexpr size_type bucket_size(size_type n) const;
constexpr size_type bucket(const key_type& k) const;
template<class K> constexpr size_type bucket(const K& k) const;
constexpr local_iterator begin(size_type n);
constexpr const_local_iterator begin(size_type n) const;
constexpr local_iterator end(size_type n);
constexpr const_local_iterator end(size_type n) const;
constexpr const_local_iterator cbegin(size_type n) const;
constexpr const_local_iterator cend(size_type n) const;

// hash policy
constexpr float load_factor() const noexcept;
constexpr float max_load_factor() const noexcept;
constexpr void max_load_factor(float z);
constexpr void rehash(size_type n);
constexpr void reserve(size_type n);
};

```

```

template<class InputIterator,
 class Hash = hash<iter-key-type<InputIterator>>,
 class Pred = equal_to<iter-key-type<InputIterator>>,
 class Allocator = allocator<iter-to-alloc-type<InputIterator>>>
unordered_multimap(InputIterator, InputIterator,
 typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 Hash, Pred, Allocator>;

template<ranges::input_range R,
 class Hash = hash<range-key-type<R>>,
 class Pred = equal_to<range-key-type<R>>,
 class Allocator = allocator<range-to-alloc-type<R>>>
unordered_multimap(from_range_t, R&&, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<range-key-type<R>, range-mapped-type<R>, Hash, Pred, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
 class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_multimap(initializer_list<pair<Key, T>>,
 typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<Key, T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 hash<iter-key-type<InputIterator>>,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>,
 hash<iter-key-type<InputIterator>>,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see below::size_type, Hash,
 Allocator)
-> unordered_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Hash,
 equal_to<iter-key-type<InputIterator>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_multimap(from_range_t, R&&, typename see below::size_type, Allocator)
-> unordered_multimap<range-key-type<R>, range-mapped-type<R>, hash<range-key-type<R>>,
 equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
unordered_multimap(from_range_t, R&&, Allocator)
-> unordered_multimap<range-key-type<R>, range-mapped-type<R>, hash<range-key-type<R>>,
 equal_to<range-key-type<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
unordered_multimap(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
-> unordered_multimap<range-key-type<R>, range-mapped-type<R>, Hash,
 equal_to<range-key-type<R>>, Allocator>;

template<class Key, class T, class Allocator>
unordered_multimap(initializer_list<pair<Key, T>>, typename see below::size_type,
 Allocator)
-> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

```



```

template<class Key, class T, class Allocator>
 unordered_multimap(initializer_list<pair<Key, T>>, Allocator)
 -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
 unordered_multimap(initializer_list<pair<Key, T>>, typename see below::size_type,
 Hash, Allocator)
 -> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;
}

```

- <sup>5</sup> A `size_type` parameter type in an `unordered_multimap` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

#### 23.5.4.2 Constructors

[unord.multimap.cnstr]

```

constexpr unordered_multimap() : unordered_multimap(size_type(see below)) { }
constexpr explicit unordered_multimap(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- <sup>1</sup> *Effects:* Constructs an empty `unordered_multimap` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.

- <sup>2</sup> *Complexity:* Constant.

```

template<class InputIterator>
 constexpr unordered_multimap(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<container-compatible-range<value_type> R>
 constexpr unordered_multimap(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_multimap(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- <sup>3</sup> *Effects:* Constructs an empty `unordered_multimap` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)`, `rg`, or `il`, respectively. `max_load_factor()` returns 1.0.

- <sup>4</sup> *Complexity:* Average case linear, worst case quadratic.

#### 23.5.4.3 Modifiers

[unord.multimap.modifiers]

```

template<class P>
 constexpr iterator insert(P&& obj);

```

- <sup>1</sup> *Constraints:* `is_constructible_v<value_type, P&&>` is true.

- <sup>2</sup> *Effects:* Equivalent to: `return emplace(std::forward<P>(obj));`

```

template<class P>
 constexpr iterator insert(const_iterator hint, P&& obj);

```

- <sup>3</sup> *Constraints:* `is_constructible_v<value_type, P&&>` is true.

- <sup>4</sup> *Effects:* Equivalent to: `return emplace_hint(hint, std::forward<P>(obj));`

## 23.5.4.4 Erasure

[unord.multimap.erase]

```
template<class K, class T, class H, class P, class A, class Predicate>
constexpr typename unordered_multimap<K, T, H, P, A>::size_type
erase_if(unordered_multimap<K, T, H, P, A>& c, Predicate pred);
```

1 *Effects:* Equivalent to:

```
 auto original_size = c.size();
 for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
 }
 return original_size - c.size();
```

## 23.5.5 Header &lt;unordered\_set&gt; synopsis

[unord.set.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.5.6, class template unordered_set
 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Alloc = allocator<Key>>
 class unordered_set;

 // 23.5.7, class template unordered_multiset
 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Alloc = allocator<Key>>
 class unordered_multiset;

 template<class Key, class Hash, class Pred, class Alloc>
 constexpr bool operator==(const unordered_set<Key, Hash, Pred, Alloc>& a,
 const unordered_set<Key, Hash, Pred, Alloc>& b);

 template<class Key, class Hash, class Pred, class Alloc>
 constexpr bool operator==(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
 const unordered_multiset<Key, Hash, Pred, Alloc>& b);

 template<class Key, class Hash, class Pred, class Alloc>
 constexpr void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
 unordered_set<Key, Hash, Pred, Alloc>& y)
 noexcept(noexcept(x.swap(y)));

 template<class Key, class Hash, class Pred, class Alloc>
 constexpr void swap(unordered_multiset<Key, Hash, Pred, Alloc>& x,
 unordered_multiset<Key, Hash, Pred, Alloc>& y)
 noexcept(noexcept(x.swap(y)));

 // 23.5.6.3, erasure for unordered_set
 template<class K, class H, class P, class A, class Predicate>
 constexpr typename unordered_set<K, H, P, A>::size_type
 erase_if(unordered_set<K, H, P, A>& c, Predicate pred);

 // 23.5.7.3, erasure for unordered_multiset
 template<class K, class H, class P, class A, class Predicate>
 constexpr typename unordered_multiset<K, H, P, A>::size_type
 erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);
```

```

namespace pmr {
 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>>
 using unordered_set = std::unordered_set<Key, Hash, Pred,
 polymorphic_allocator<Key>>;

 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>>
 using unordered_multiset = std::unordered_multiset<Key, Hash, Pred,
 polymorphic_allocator<Key>>;
}

```

## 23.5.6 Class template unordered\_set

[unord.set]

### 23.5.6.1 Overview

[unord.set.overview]

- <sup>1</sup> An `unordered_set` is an unordered associative container that supports unique keys (an `unordered_set` contains at most one of each key value) and in which the elements' keys are the elements themselves. The `unordered_set` class supports forward iterators.
- <sup>2</sup> An `unordered_set` meets all of the requirements of a container (23.2.2.2), of an allocator-aware container (23.2.2.5), and of an unordered associative container (23.2.8). It provides the operations described in the preceding requirements table for unique keys; that is, an `unordered_set` supports the `a_uniq` operations in that table, not the `a_eq` operations. For an `unordered_set<Key>` the `key_type` and the `value_type` are both `Key`. The `iterator` and `const_iterator` types are both constant iterator types. It is unspecified whether they are the same type.
- <sup>3</sup> Subclause 23.5.6 only describes operations on `unordered_set` that are not described in one of the requirement tables, or for which there is additional semantic information.
- <sup>4</sup> The types `iterator`, `const_iterator`, `local_iterator`, and `const_local_iterator` meet the `constexpr` iterator requirements (24.3.1).

```

namespace std {
 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Allocator = allocator<Key>>
 class unordered_set {
 public:
 // types
 using key_type = Key;
 using value_type = Key;
 using hasher = Hash;
 using key_equal = Pred;
 using allocator_type = Allocator;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;
 using reference = value_type&;
 using const_reference = const value_type&;
 using size_type = implementation-defined; // see 23.2
 using difference_type = implementation-defined; // see 23.2

 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 using local_iterator = implementation-defined; // see 23.2
 using const_local_iterator = implementation-defined; // see 23.2
 using node_type = unspecified;
 using insert_return_type = insert_return_type<iterator, node_type>;

 // 23.5.6.2, construct/copy/destroy
 constexpr unordered_set();
 };
}

```

```

constexpr explicit unordered_set(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<class InputIterator>
 constexpr unordered_set(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
 constexpr unordered_set(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_set(const unordered_set&);
constexpr unordered_set(unordered_set&&);
constexpr explicit unordered_set(const Allocator&);
constexpr unordered_set(const unordered_set&, const type_identity_t<Allocator>&);
constexpr unordered_set(unordered_set&&, const type_identity_t<Allocator>&);
constexpr unordered_set(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_set(size_type n, const allocator_type& a)
 : unordered_set(n, hasher(), key_equal(), a) { }
constexpr unordered_set(size_type n, const hasher& hf, const allocator_type& a)
 : unordered_set(n, hf, key_equal(), a) { }

template<class InputIterator>
 constexpr unordered_set(InputIterator f, InputIterator l, size_type n,
 const allocator_type& a)
 : unordered_set(f, l, n, hasher(), key_equal(), a) { }

template<class InputIterator>
 constexpr unordered_set(InputIterator f, InputIterator l, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_set(f, l, n, hf, key_equal(), a) { }

constexpr unordered_set(initializer_list<value_type> il, size_type n,
 const allocator_type& a)
 : unordered_set(il, n, hasher(), key_equal(), a) { }

template<container-compatible-range<value_type> R>
 constexpr unordered_set(from_range_t, R&& rg, size_type n, const allocator_type& a)
 : unordered_set(from_range, std::forward<R>(rg), n, hasher(), key_equal(), a) { }

template<container-compatible-range<value_type> R>
 constexpr unordered_set(from_range_t, R&& rg, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_set(from_range, std::forward<R>(rg), n, hf, key_equal(), a) { }

constexpr unordered_set(initializer_list<value_type> il, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_set(il, n, hf, key_equal(), a) { }

constexpr ~unordered_set();
constexpr unordered_set& operator=(const unordered_set&);
constexpr unordered_set& operator=(unordered_set&&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Hash> && is_nothrow_move_assignable_v<Pred>);
constexpr unordered_set& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;

```

```

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.5.6.4, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr pair<iterator, bool> insert(const value_type& obj);
constexpr pair<iterator, bool> insert(value_type&& obj);
template<class K> constexpr pair<iterator, bool> insert(K&& obj);
constexpr iterator insert(const_iterator hint, const value_type& obj);
constexpr iterator insert(const_iterator hint, value_type&& obj);
template<class K> constexpr iterator insert(const_iterator hint, K&& obj);
template<class InputIterator> constexpr void insert(InputIterator first, InputIterator last);
template<container_compatible_range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr insert_return_type insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position)
 requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& k);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(unordered_set&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Hash> && is_nothrow_swappable_v<Pred>);
constexpr void clear() noexcept;

template<class H2, class P2>
 constexpr void merge(unordered_set<Key, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_set<Key, H2, P2, Allocator>&& source);
template<class H2, class P2>
 constexpr void merge(unordered_multiset<Key, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_multiset<Key, H2, P2, Allocator>&& source);

// observers
constexpr hasher hash_function() const;
constexpr key_equal key_eq() const;

// set operations
constexpr iterator find(const key_type& k);
constexpr const_iterator find(const key_type& k) const;
template<class K>
 constexpr iterator find(const K& k);
template<class K>
 constexpr const_iterator find(const K& k) const;
constexpr size_type count(const key_type& k) const;
template<class K>
 constexpr size_type count(const K& k) const;
constexpr bool contains(const key_type& k) const;
template<class K>
 constexpr bool contains(const K& k) const;
constexpr pair<iterator, iterator> equal_range(const key_type& k);

```

```

constexpr pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& k);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& k) const;

// bucket interface
constexpr size_type bucket_count() const noexcept;
constexpr size_type max_bucket_count() const noexcept;
constexpr size_type bucket_size(size_type n) const;
constexpr size_type bucket(const key_type& k) const;
template<class K> constexpr size_type bucket(const K& k) const;
constexpr local_iterator begin(size_type n);
constexpr const_local_iterator begin(size_type n) const;
constexpr local_iterator end(size_type n);
constexpr const_local_iterator end(size_type n) const;
constexpr const_local_iterator cbegin(size_type n) const;
constexpr const_local_iterator cend(size_type n) const;

// hash policy
constexpr float load_factor() const noexcept;
constexpr float max_load_factor() const noexcept;
constexpr void max_load_factor(float z);
constexpr void rehash(size_type n);
constexpr void reserve(size_type n);
};

template<class InputIterator,
 class Hash = hash<iter-value-type<InputIterator>>,
 class Pred = equal_to<iter-value-type<InputIterator>>,
 class Allocator = allocator<iter-value-type<InputIterator>>>
unordered_set(InputIterator, InputIterator, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<iter-value-type<InputIterator>,
 Hash, Pred, Allocator>;

template<ranges::input_range R,
 class Hash = hash<ranges::range_value_t<R>>,
 class Pred = equal_to<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
unordered_set(from_range_t, R&&, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<ranges::range_value_t<R>, Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
 class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_set(initializer_list<T>, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_set<iter-value-type<InputIterator>,
 hash<iter-value-type<InputIterator>>,
 equal_to<iter-value-type<InputIterator>>,
 Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type,
 Hash, Allocator)
-> unordered_set<iter-value-type<InputIterator>, Hash,
 equal_to<iter-value-type<InputIterator>>,
 Allocator>;

```

```

template<ranges::input_range R, class Allocator>
 unordered_set(from_range_t, R&&, typename see_below::size_type, Allocator)
 -> unordered_set<ranges::range_value_t<R>, hash<ranges::range_value_t<R>>,
 equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
 unordered_set(from_range_t, R&&, Allocator)
 -> unordered_set<ranges::range_value_t<R>, hash<ranges::range_value_t<R>>,
 equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
 unordered_set(from_range_t, R&&, typename see_below::size_type, Hash, Allocator)
 -> unordered_set<ranges::range_value_t<R>, Hash,
 equal_to<ranges::range_value_t<R>>, Allocator>;

template<class T, class Allocator>
 unordered_set(initializer_list<T>, typename see_below::size_type, Allocator)
 -> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
 unordered_set(initializer_list<T>, typename see_below::size_type, Hash, Allocator)
 -> unordered_set<T, Hash, equal_to<T>, Allocator>;
}

```

- <sup>5</sup> A `size_type` parameter type in an `unordered_set` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

### 23.5.6.2 Constructors

[unord.set.cnstr]

```

constexpr unordered_set() : unordered_set(size_type(see_below)) { }
constexpr explicit unordered_set(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- <sup>1</sup> *Effects:* Constructs an empty `unordered_set` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.
- <sup>2</sup> *Complexity:* Constant.

```

template<class InputIterator>
 constexpr unordered_set(InputIterator f, InputIterator l,
 size_type n = see_below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
 constexpr unordered_multiset(from_range_t, R&& rg,
 size_type n = see_below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
constexpr unordered_set(initializer_list<value_type> il,
 size_type n = see_below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- <sup>3</sup> *Effects:* Constructs an empty `unordered_set` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f, l)`, `rg`, or `il`, respectively. `max_load_factor()` returns 1.0.
- <sup>4</sup> *Complexity:* Average case linear, worst case quadratic.

**23.5.6.3 Erasure****[unord.set.erase]**

```
template<class K, class H, class P, class A, class Predicate>
constexpr typename unordered_set<K, H, P, A>::size_type
erase_if(unordered_set<K, H, P, A>& c, Predicate pred);
```

1 *Effects:* Equivalent to:

```
 auto original_size = c.size();
 for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
 }
 return original_size - c.size();
```

**23.5.6.4 Modifiers****[unord.set.modifiers]**

```
template<class K> constexpr pair<iterator, bool> insert(K&& obj);
template<class K> constexpr iterator insert(const_iterator hint, K&& obj);
```

1 *Constraints:* The *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types. For the second overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both false.

2 *Preconditions:* `value_type` is *Cpp17EmplaceConstructible* into `unordered_set` from `std::forward<K>(obj)`.

3 *Effects:* If the set already contains an element that is equivalent to `obj`, there is no effect. Otherwise, let `h` be `hash_function()(obj)`. Constructs an object `u` of type `value_type` with `std::forward<K>(obj)`. If `hash_function()(u) != h || contains(u) is true`, the behavior is undefined. Inserts `u` into `*this`.

4 *Returns:* For the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the set element that is equivalent to `obj`.

5 *Complexity:* Average case constant, worst case linear.

**23.5.7 Class template unordered\_multiset****[unord.multiset]****23.5.7.1 Overview****[unord.multiset.overview]**

1 An `unordered_multiset` is an unordered associative container that supports equivalent keys (an instance of `unordered_multiset` may contain multiple copies of the same key value) and in which each element's key is the element itself. The `unordered_multiset` class supports forward iterators.

2 An `unordered_multiset` meets all of the requirements of a container (23.2.2.2), of an allocator-aware container (23.2.2.5), and of an unordered associative container (23.2.8). It provides the operations described in the preceding requirements table for equivalent keys; that is, an `unordered_multiset` supports the `a_eq` operations in that table, not the `a_uniq` operations. For an `unordered_multiset<Key>` the `key_type` and the `value_type` are both `Key`. The `iterator` and `const_iterator` types are both constant iterator types. It is unspecified whether they are the same type.

3 Subclause 23.5.7 only describes operations on `unordered_multiset` that are not described in one of the requirement tables, or for which there is additional semantic information.

4 The types `iterator`, `const_iterator`, `local_iterator`, and `const_local_iterator` meet the `constexpr` iterator requirements (24.3.1).

```
namespace std {
 template<class Key,
 class Hash = hash<Key>,
 class Pred = equal_to<Key>,
 class Allocator = allocator<Key>>
 class unordered_multiset {
 public:
 // types
 using key_type = Key;
```



```

using value_type = Key;
using hasher = Hash;
using key_equal = Pred;
using allocator_type = Allocator;
using pointer = allocator_traits<Allocator>::pointer;
using const_pointer = allocator_traits<Allocator>::const_pointer;
using reference = value_type&;
using const_reference = const value_type&;
using size_type = implementation-defined; // see 23.2
using difference_type = implementation-defined; // see 23.2

using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using local_iterator = implementation-defined; // see 23.2
using const_local_iterator = implementation-defined; // see 23.2
using node_type = unspecified;

// 23.5.7.2, construct/copy/destroy
constexpr unordered_multiset();
constexpr explicit unordered_multiset(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

template<class InputIterator>
 constexpr unordered_multiset(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
template<container-compatible-range<value_type> R>
 constexpr unordered_multiset(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

constexpr unordered_multiset(const unordered_multiset&);
constexpr unordered_multiset(unordered_multiset&&);
constexpr explicit unordered_multiset(const Allocator&);
constexpr unordered_multiset(const unordered_multiset&, const type_identity_t<Allocator>&);
constexpr unordered_multiset(unordered_multiset&&, const type_identity_t<Allocator>&);
constexpr unordered_multiset(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
constexpr unordered_multiset(size_type n, const allocator_type& a)
 : unordered_multiset(n, hasher(), key_equal(), a) { }
constexpr unordered_multiset(size_type n, const hasher& hf, const allocator_type& a)
 : unordered_multiset(n, hf, key_equal(), a) { }
template<class InputIterator>
 constexpr unordered_multiset(InputIterator f, InputIterator l, size_type n,
 const allocator_type& a)
 : unordered_multiset(f, l, n, hasher(), key_equal(), a) { }
template<class InputIterator>
 constexpr unordered_multiset(InputIterator f, InputIterator l, size_type n,
 const hasher& hf, const allocator_type& a)
 : unordered_multiset(f, l, n, hf, key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_multiset(from_range_t, R&& rg, size_type n, const allocator_type& a)
 : unordered_multiset(from_range, std::forward<R>(rg),
 n, hasher(), key_equal(), a) { }
template<container-compatible-range<value_type> R>
 constexpr unordered_multiset(from_range_t, R&& rg, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_multiset(from_range, std::forward<R>(rg), n, hf, key_equal(), a) { }
constexpr unordered_multiset(initializer_list<value_type> il, size_type n,
 const allocator_type& a)
 : unordered_multiset(il, n, hasher(), key_equal(), a) { }

```

```

constexpr unordered_multiset(initializer_list<value_type> il, size_type n, const hasher& hf,
 const allocator_type& a)
 : unordered_multiset(il, n, hf, key_equal(), a) { }
constexpr ~unordered_multiset();
constexpr unordered_multiset& operator=(const unordered_multiset&);
constexpr unordered_multiset& operator=(unordered_multiset&&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_move_assignable_v<Hash> && is_nothrow_move_assignable_v<Pred>);
constexpr unordered_multiset& operator=(initializer_list<value_type>);
constexpr allocator_type get_allocator() const noexcept;

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);
constexpr iterator insert(const value_type& obj);
constexpr iterator insert(value_type&& obj);
constexpr iterator insert(const_iterator hint, const value_type& obj);
constexpr iterator insert(const_iterator hint, value_type&& obj);
template<class InputIterator> constexpr void insert(InputIterator first, InputIterator last);
template<container_compatible_range<value_type> R>
 constexpr void insert_range(R&& rg);
constexpr void insert(initializer_list<value_type>);

constexpr node_type extract(const_iterator position);
constexpr node_type extract(const key_type& x);
template<class K> constexpr node_type extract(K&& x);
constexpr iterator insert(node_type&& nh);
constexpr iterator insert(const_iterator hint, node_type&& nh);

constexpr iterator erase(iterator position)
 requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& k);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(unordered_multiset&)
 noexcept(allocator_traits<Allocator>::is_always_equal::value &&
 is_nothrow_swappable_v<Hash> && is_nothrow_swappable_v<Pred>);
constexpr void clear() noexcept;

template<class H2, class P2>
 constexpr void merge(unordered_multiset<Key, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_multiset<Key, H2, P2, Allocator>&& source);
template<class H2, class P2>
 constexpr void merge(unordered_set<Key, H2, P2, Allocator>& source);
template<class H2, class P2>
 constexpr void merge(unordered_set<Key, H2, P2, Allocator>&& source);

```

```

// observers
constexpr hasher hash_function() const;
constexpr key_equal key_eq() const;

// set operations
constexpr iterator find(const key_type& k);
constexpr const_iterator find(const key_type& k) const;
template<class K>
 constexpr iterator find(const K& k);
template<class K>
 constexpr const_iterator find(const K& k) const;
constexpr size_type count(const key_type& k) const;
template<class K>
 constexpr size_type count(const K& k) const;
constexpr bool contains(const key_type& k) const;
template<class K>
 constexpr bool contains(const K& k) const;
constexpr pair<iterator, iterator> equal_range(const key_type& k);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& k);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& k) const;

// bucket interface
constexpr size_type bucket_count() const noexcept;
constexpr size_type max_bucket_count() const noexcept;
constexpr size_type bucket_size(size_type n) const;
constexpr size_type bucket(const key_type& k) const;
template<class K> constexpr size_type bucket(const K& k) const;
constexpr local_iterator begin(size_type n);
constexpr const_local_iterator begin(size_type n) const;
constexpr local_iterator end(size_type n);
constexpr const_local_iterator end(size_type n) const;
constexpr const_local_iterator cbegin(size_type n) const;
constexpr const_local_iterator cend(size_type n) const;

// hash policy
constexpr float load_factor() const noexcept;
constexpr float max_load_factor() const noexcept;
constexpr void max_load_factor(float z);
constexpr void rehash(size_type n);
constexpr void reserve(size_type n);
};

template<class InputIterator,
 class Hash = hash<iter-value-type<InputIterator>>,
 class Pred = equal_to<iter-value-type<InputIterator>>,
 class Allocator = allocator<iter-value-type<InputIterator>>>
 unordered_multiset(InputIterator, InputIterator, typename see_below::size_type = see_below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
 -> unordered_multiset<iter-value-type<InputIterator>,
 Hash, Pred, Allocator>;

template<ranges::input_range R,
 class Hash = hash<ranges::range_value_t<R>>,
 class Pred = equal_to<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
 unordered_multiset(from_range_t, R&&, typename see_below::size_type = see_below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
 -> unordered_multiset<ranges::range_value_t<R>, Hash, Pred, Allocator>;

```

```

template<class T, class Hash = hash<T>,
 class Pred = equal_to<T>, class Allocator = allocator<T>>
 unordered_multiset(initializer_list<T>, typename see below::size_type = see below,
 Hash = Hash(), Pred = Pred(), Allocator = Allocator())
 -> unordered_multiset<T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
 unordered_multiset(InputIterator, InputIterator, typename see below::size_type, Allocator)
 -> unordered_multiset<iter-value-type<InputIterator>,
 hash<iter-value-type<InputIterator>>,
 equal_to<iter-value-type<InputIterator>>,
 Allocator>;

template<class InputIterator, class Hash, class Allocator>
 unordered_multiset(InputIterator, InputIterator, typename see below::size_type,
 Hash, Allocator)
 -> unordered_multiset<iter-value-type<InputIterator>, Hash,
 equal_to<iter-value-type<InputIterator>>,
 Allocator>;

template<ranges::input_range R, class Allocator>
 unordered_multiset(from_range_t, R&&, typename see below::size_type, Allocator)
 -> unordered_multiset<ranges::range_value_t<R>, hash<ranges::range_value_t<R>>,
 equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Allocator>
 unordered_multiset(from_range_t, R&&, Allocator)
 -> unordered_multiset<ranges::range_value_t<R>, hash<ranges::range_value_t<R>>,
 equal_to<ranges::range_value_t<R>>, Allocator>;

template<ranges::input_range R, class Hash, class Allocator>
 unordered_multiset(from_range_t, R&&, typename see below::size_type, Hash, Allocator)
 -> unordered_multiset<ranges::range_value_t<R>, Hash, equal_to<ranges::range_value_t<R>>,
 Allocator>;

template<class T, class Allocator>
 unordered_multiset(initializer_list<T>, typename see below::size_type, Allocator)
 -> unordered_multiset<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
 unordered_multiset(initializer_list<T>, typename see below::size_type, Hash, Allocator)
 -> unordered_multiset<T, Hash, equal_to<T>, Allocator>;
}

```

- <sup>5</sup> A `size_type` parameter type in an `unordered_multiset` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

### 23.5.7.2 Constructors

[unord.multiset.cnstr]

```

constexpr unordered_multiset() : unordered_multiset(size_type(see below)) { }
constexpr explicit unordered_multiset(size_type n, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

- <sup>1</sup> *Effects:* Constructs an empty `unordered_multiset` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.
- <sup>2</sup> *Complexity:* Constant.

```

template<class InputIterator>
 constexpr unordered_multiset(InputIterator f, InputIterator l,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

```

template<container-compatible-range<value_type> R>
constexpr unordered_multiset(from_range_t, R&& rg,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());
constexpr unordered_multiset(initializer_list<value_type> il,
 size_type n = see below, const hasher& hf = hasher(),
 const key_equal& eql = key_equal(),
 const allocator_type& a = allocator_type());

```

3 *Effects:* Constructs an empty `unordered_multiset` using the specified hash function, key equality predicate, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)`, `rg`, or `il`, respectively. `max_load_factor()` returns 1.0.

4 *Complexity:* Average case linear, worst case quadratic.

### 23.5.7.3 Erasure

[unord.multiset.erase]

```

template<class K, class H, class P, class A, class Predicate>
constexpr typename unordered_multiset<K, H, P, A>::size_type
erase_if(unordered_multiset<K, H, P, A>& c, Predicate pred);

```

1 *Effects:* Equivalent to:

```

auto original_size = c.size();
for (auto i = c.begin(), last = c.end(); i != last;) {
 if (pred(*i)) {
 i = c.erase(i);
 } else {
 ++i;
 }
}
return original_size - c.size();

```

## 23.6 Container adaptors

[container.adaptors]

### 23.6.1 General

[container.adaptors.general]

- 1 The headers `<queue>` (23.6.2), `<stack>` (23.6.5), `<flat_map>` (23.6.7), and `<flat_set>` (23.6.10) define the container adaptors `queue` and `priority_queue`, `stack`, `flat_map` and `flat_multimap`, and `flat_set` and `flat_multiset`, respectively.
- 2 Each container adaptor takes one or more template parameters named `Container`, `KeyContainer`, or `MappedContainer` that denote the types of containers that the container adaptor adapts. Each container adaptor has at least one constructor that takes a reference argument to one or more such template parameters. For each constructor reference argument to a container `C`, the constructor copies the container into the container adaptor. If `C` takes an allocator, then a compatible allocator may be passed in to the adaptor's constructor. Otherwise, normal copy or move construction is used for the container argument. For the container adaptors that take a single container template parameter `Container`, the first template parameter `T` of the container adaptor shall denote the same type as `Container::value_type`.
- 3 For container adaptors, no `swap` function throws an exception unless that exception is thrown by the swap of the adaptor's `Container`, `KeyContainer`, `MappedContainer`, or `Compare` object (if any).
- 4 A constructor template of a container adaptor shall not participate in overload resolution if it has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- 5 For container adaptors that have them, the `insert`, `emplace`, and `erase` members affect the validity of iterators, references, and pointers to the adaptor's container(s) in the same way that the containers' respective `insert`, `emplace`, and `erase` members do.  
[Example 1: A call to `flat_map<Key, T>::insert` can invalidate all iterators to the `flat_map`. — end example]
- 6 A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:
  - (6.1) — It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.

- (6.2) — It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.
  - (6.3) — It has a `Container`, `KeyContainer`, or `MappedContainer` template parameter and a type that qualifies as an allocator is deduced for that parameter.
  - (6.4) — It has no `Container`, `KeyContainer`, or `MappedContainer` template parameter, and it has an `Allocator` template parameter, and a type that does not qualify as an allocator is deduced for that parameter.
  - (6.5) — It has both `Container` and `Allocator` template parameters, and `uses_allocator_v<Container, Allocator>` is false.
  - (6.6) — It has both `KeyContainer` and `Allocator` template parameters, and `uses_allocator_v<KeyContainer, Allocator>` is false.
  - (6.7) — It has both `KeyContainer` and `Compare` template parameters, and
 

```
is_invocable_v<const Compare&,
 const typename KeyContainer::value_type&,
 const typename KeyContainer::value_type&>
```

 is not a valid expression or is false.
  - (6.8) — It has both `MappedContainer` and `Allocator` template parameters, and `uses_allocator_v<MappedContainer, Allocator>` is false.
- 7 The exposition-only alias template *iter-value-type* defined in 23.3.1 and the exposition-only alias templates *iter-key-type*, *iter-mapped-type*, *range-key-type*, and *range-mapped-type* defined in 23.4.1 may appear in deduction guides for container adaptors.
- 8 The following exposition-only alias template may appear in deduction guides for container adaptors:

```
template<class Allocator, class T>
using alloc_rebind = allocator_traits<Allocator>::template rebind_alloc<T>; // exposition only
```

### 23.6.2 Header <queue> synopsis

[queue.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.6.3, class template queue
 template<class T, class Container = deque<T>> class queue;

 template<class T, class Container>
 constexpr bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator<(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator>(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator>=(const queue<T, Container>& x, const queue<T, Container>& y);
 template<class T, three_way_comparable Container>
 constexpr compare_three_way_result_t<Container>
 operator<=>(const queue<T, Container>& x, const queue<T, Container>& y);

 template<class T, class Container>
 constexpr void swap(queue<T, Container>& x, queue<T, Container>& y)
 noexcept(noexcept(x.swap(y)));
 template<class T, class Container, class Alloc>
 struct uses_allocator<queue<T, Container>, Alloc>;

 // 23.6.13, formatter specialization for queue
 template<class charT, class T, formattable<charT> Container>
 struct formatter<queue<T, Container>, charT>;
```

```

template<class T, class Container>
 constexpr bool enable_nonlocking_formatter_optimization<queue<T, Container>> = false;

// 23.6.4, class template priority_queue
template<class T, class Container = vector<T>,
 class Compare = less<typename Container::value_type>>
 class priority_queue;

template<class T, class Container, class Compare>
 constexpr void swap(priority_queue<T, Container, Compare>& x,
 priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));
template<class T, class Container, class Compare, class Alloc>
 struct uses_allocator<priority_queue<T, Container, Compare>, Alloc>;

// 23.6.13, formatter specialization for priority_queue
template<class charT, class T, formattable<charT> Container, class Compare>
 struct formatter<priority_queue<T, Container, Compare>, charT>;

template<class T, class Container, class Compare>
 constexpr bool
 enable_nonlocking_formatter_optimization<priority_queue<T, Container, Compare>> = false;
}

```

### 23.6.3 Class template queue

[queue]

#### 23.6.3.1 Definition

[queue.defn]

- <sup>1</sup> Any sequence container supporting operations `front()`, `back()`, `push_back()` and `pop_front()` can be used to instantiate `queue`. In particular, `list` (23.3.11) and `deque` (23.3.5) can be used.

```

namespace std {
 template<class T, class Container = deque<T>>
 class queue {
 public:
 using value_type = Container::value_type;
 using reference = Container::reference;
 using const_reference = Container::const_reference;
 using size_type = Container::size_type;
 using container_type = Container;

 protected:
 Container c;

 public:
 constexpr queue() : queue(Container()) {}
 constexpr explicit queue(const Container&);
 constexpr explicit queue(Container&&);
 template<class InputIterator> constexpr queue(InputIterator first, InputIterator last);
 template<container-compatible-range<T> R> constexpr queue(from_range_t, R&& rg);
 template<class Alloc> constexpr explicit queue(const Alloc&);
 template<class Alloc> constexpr queue(const Container&, const Alloc&);
 template<class Alloc> constexpr queue(Container&&, const Alloc&);
 template<class Alloc> constexpr queue(const queue&, const Alloc&);
 template<class Alloc> constexpr queue(queue&&, const Alloc&);
 template<class InputIterator, class Alloc>
 constexpr queue(InputIterator first, InputIterator last, const Alloc&);
 template<container-compatible-range<T> R, class Alloc>
 constexpr queue(from_range_t, R&& rg, const Alloc&);

 constexpr bool empty() const { return c.empty(); }
 constexpr size_type size() const { return c.size(); }
 constexpr reference front() { return c.front(); }
 constexpr const_reference front() const { return c.front(); }
 constexpr reference back() { return c.back(); }
 constexpr const_reference back() const { return c.back(); }
 constexpr void push(const value_type& x) { c.push_back(x); }

```

```

constexpr void push(value_type&& x) { c.push_back(std::move(x)); }
template<container-compatible-range<T> R> constexpr void push_range(R&& rg);
template<class... Args>
 constexpr decltype(auto) emplace(Args&&... args)
 { return c.emplace_back(std::forward<Args>(args)...); }
constexpr void pop() { c.pop_front(); }
constexpr void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
 { using std::swap; swap(c, q.c); }
};

template<class Container>
 queue(Container) -> queue<typename Container::value_type, Container>;

template<class InputIterator>
 queue(InputIterator, InputIterator) -> queue<iter-value-type<InputIterator>>;

template<ranges::input_range R>
 queue(from_range_t, R&&) -> queue<ranges::range_value_t<R>>;

template<class Container, class Allocator>
 queue(Container, Allocator) -> queue<typename Container::value_type, Container>;

template<class InputIterator, class Allocator>
 queue(InputIterator, InputIterator, Allocator)
 -> queue<iter-value-type<InputIterator>, deque<iter-value-type<InputIterator>,
 Allocator>>;

template<ranges::input_range R, class Allocator>
 queue(from_range_t, R&&, Allocator)
 -> queue<ranges::range_value_t<R>, deque<ranges::range_value_t<R>, Allocator>>;

template<class T, class Container, class Alloc>
 struct uses_allocator<queue<T, Container>, Alloc>
 : uses_allocator<Container, Alloc>::type { };
}

```

### 23.6.3.2 Constructors

[queue.cons]

```
constexpr explicit queue(const Container& cont);
```

1 *Effects:* Initializes c with cont.

```
constexpr explicit queue(Container&& cont);
```

2 *Effects:* Initializes c with std::move(cont).

```
template<class InputIterator>
 constexpr queue(InputIterator first, InputIterator last);
```

3 *Effects:* Initializes c with first as the first argument and last as the second argument.

```
template<container-compatible-range<T> R>
 constexpr queue(from_range_t, R&& rg);
```

4 *Effects:* Initializes c with ranges::to<Container>(std::forward<R>(rg)).

### 23.6.3.3 Constructors with allocators

[queue.cons.alloc]

1 If uses\_allocator\_v<container\_type, Alloc> is false the constructors in this subclause shall not participate in overload resolution.

```
template<class Alloc> constexpr explicit queue(const Alloc& a);
```

2 *Effects:* Initializes c with a.

```
template<class Alloc> constexpr queue(const container_type& cont, const Alloc& a);
```

3 *Effects:* Initializes c with cont as the first argument and a as the second argument.



```
template<class Alloc> constexpr queue(container_type&& cont, const Alloc& a);
```

4     *Effects:* Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument.

```
template<class Alloc> constexpr queue(const queue& q, const Alloc& a);
```

5     *Effects:* Initializes `c` with `q.c` as the first argument and `a` as the second argument.

```
template<class Alloc> constexpr queue(queue&& q, const Alloc& a);
```

6     *Effects:* Initializes `c` with `std::move(q.c)` as the first argument and `a` as the second argument.

```
template<class InputIterator, class Alloc>
constexpr queue(InputIterator first, InputIterator last, const Alloc& alloc);
```

7     *Effects:* Initializes `c` with `first` as the first argument, `last` as the second argument, and `alloc` as the third argument.

```
template<container-compatible-range<T> R, class Alloc>
constexpr queue(from_range_t, R&& rg, const Alloc& a);
```

8     *Effects:* Initializes `c` with `ranges::to<Container>(std::forward<R>(rg), a)`.

### 23.6.3.4 Modifiers

[queue.mod]

```
template<container-compatible-range<T> R>
constexpr void push_range(R&& rg);
```

1     *Effects:* Equivalent to `c.append_range(std::forward<R>(rg))` if that is a valid expression, otherwise `ranges::copy(rg, back_inserter(c))`.

### 23.6.3.5 Operators

[queue.ops]

```
template<class T, class Container>
constexpr bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
```

1     *Returns:* `x.c == y.c`.

```
template<class T, class Container>
constexpr bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);
```

2     *Returns:* `x.c != y.c`.

```
template<class T, class Container>
constexpr bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);
```

3     *Returns:* `x.c < y.c`.

```
template<class T, class Container>
constexpr bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);
```

4     *Returns:* `x.c > y.c`.

```
template<class T, class Container>
constexpr bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);
```

5     *Returns:* `x.c <= y.c`.

```
template<class T, class Container>
constexpr bool operator>=(const queue<T, Container>& x, const queue<T, Container>& y);
```

6     *Returns:* `x.c >= y.c`.

```
template<class T, three_way_comparable Container>
constexpr compare_three_way_result_t<Container>
operator<=>(const queue<T, Container>& x, const queue<T, Container>& y);
```

7     *Returns:* `x.c <=> y.c`.

## 23.6.3.6 Specialized algorithms

[queue.special]

```
template<class T, class Container>
constexpr void swap(queue<T, Container>& x, queue<T, Container>& y)
 noexcept(noexcept(x.swap(y)));
```

<sup>1</sup> *Constraints:* `is_swappable_v<Container>` is true.

<sup>2</sup> *Effects:* As if by `x.swap(y)`.

23.6.4 Class template `priority_queue`

[priority.queue]

## 23.6.4.1 Overview

[priqueue.overview]

- <sup>1</sup> Any sequence container with random access iterator and supporting operations `front()`, `push_back()` and `pop_back()` can be used to instantiate `priority_queue`. In particular, `vector` (23.3.13) and `deque` (23.3.5) can be used. Instantiating `priority_queue` also involves supplying a function or function object for making priority comparisons; the library assumes that the function or function object defines a strict weak ordering (26.8).

```
namespace std {
 template<class T, class Container = vector<T>,
 class Compare = less<typename Container::value_type>>
 class priority_queue {
 public:
 using value_type = Container::value_type;
 using reference = Container::reference;
 using const_reference = Container::const_reference;
 using size_type = Container::size_type;
 using container_type = Container;
 using value_compare = Compare;

 protected:
 Container c;
 Compare comp;

 public:
 constexpr priority_queue() : priority_queue(Compare()) {}
 constexpr explicit priority_queue(const Compare& x) : priority_queue(x, Container()) {}
 constexpr priority_queue(const Compare& x, const Container&);
 constexpr priority_queue(const Compare& x, Container&&);
 template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last,
 const Compare& x = Compare());
 template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& x,
 const Container&);
 template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& x,
 Container&&);
 template<container-compatible-range<T> R>
 constexpr priority_queue(from_range_t, R&& rg, const Compare& x = Compare());
 template<class Alloc> constexpr explicit priority_queue(const Alloc&);
 template<class Alloc> constexpr priority_queue(const Compare&, const Alloc&);
 template<class Alloc>
 constexpr priority_queue(const Compare&, const Container&, const Alloc&);
 template<class Alloc> constexpr priority_queue(const Compare&, Container&&, const Alloc&);
 template<class Alloc> constexpr priority_queue(const priority_queue&, const Alloc&);
 template<class Alloc> constexpr priority_queue(priority_queue&&, const Alloc&);
 template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator, InputIterator, const Alloc&);
 template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator, InputIterator, const Compare&, const Alloc&);
 template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator, InputIterator, const Compare&, const Container&,
 const Alloc&);
```

```

template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator, InputIterator, const Compare&, Container&&,
 const Alloc&);
template<container-compatible-range<T> R, class Alloc>
 constexpr priority_queue(from_range_t, R&& rg, const Compare&, const Alloc&);
template<container-compatible-range<T> R, class Alloc>
 constexpr priority_queue(from_range_t, R&& rg, const Alloc&);

constexpr bool empty() const { return c.empty(); }
constexpr size_type size() const { return c.size(); }
constexpr const_reference top() const { return c.front(); }
constexpr void push(const value_type& x);
constexpr void push(value_type&& x);
template<container-compatible-range<T> R>
 constexpr void push_range(R&& rg);
template<class... Args> constexpr void emplace(Args&&... args);
constexpr void pop();
constexpr void swap(priority_queue& q)
 noexcept(is_nothrow_swappable_v<Container> && is_nothrow_swappable_v<Compare>)
 { using std::swap; swap(c, q.c); swap(comp, q.comp); }
};

template<class Compare, class Container>
 priority_queue(Compare, Container)
 -> priority_queue<typename Container::value_type, Container, Compare>;

template<class InputIterator,
 class Compare = less<iter-value-type<InputIterator>>,
 class Container = vector<iter-value-type<InputIterator>>>
 priority_queue(InputIterator, InputIterator, Compare = Compare(), Container = Container())
 -> priority_queue<iter-value-type<InputIterator>, Container, Compare>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>>
 priority_queue(from_range_t, R&&, Compare = Compare())
 -> priority_queue<ranges::range_value_t<R>, vector<ranges::range_value_t<R>>, Compare>;

template<class Compare, class Container, class Allocator>
 priority_queue(Compare, Container, Allocator)
 -> priority_queue<typename Container::value_type, Container, Compare>;

template<class InputIterator, class Allocator>
 priority_queue(InputIterator, InputIterator, Allocator)
 -> priority_queue<iter-value-type<InputIterator>,
 vector<iter-value-type<InputIterator>, Allocator>,
 less<iter-value-type<InputIterator>>>;

template<class InputIterator, class Compare, class Allocator>
 priority_queue(InputIterator, InputIterator, Compare, Allocator)
 -> priority_queue<iter-value-type<InputIterator>,
 vector<iter-value-type<InputIterator>, Allocator>, Compare>;

template<class InputIterator, class Compare, class Container, class Allocator>
 priority_queue(InputIterator, InputIterator, Compare, Container, Allocator)
 -> priority_queue<typename Container::value_type, Container, Compare>;

template<ranges::input_range R, class Compare, class Allocator>
 priority_queue(from_range_t, R&&, Compare, Allocator)
 -> priority_queue<ranges::range_value_t<R>, vector<ranges::range_value_t<R>, Allocator>,
 Compare>;

template<ranges::input_range R, class Allocator>
 priority_queue(from_range_t, R&&, Allocator)
 -> priority_queue<ranges::range_value_t<R>, vector<ranges::range_value_t<R>, Allocator>>;

```

// no equality is provided

```
template<class T, class Container, class Compare, class Alloc>
 struct uses_allocator<priority_queue<T, Container, Compare>, Alloc>
 : uses_allocator<Container, Alloc>::type { };
}
```

#### 23.6.4.2 Constructors

[priqueue.cons]

```
constexpr priority_queue(const Compare& x, const Container& y);
constexpr priority_queue(const Compare& x, Container&& y);
```

1     *Preconditions:* x defines a strict weak ordering (26.8).

2     *Effects:* Initializes comp with x and c with y (copy constructing or move constructing as appropriate); calls make\_heap(c.begin(), c.end(), comp).

```
template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& x = Compare());
```

3     *Preconditions:* x defines a strict weak ordering (26.8).

4     *Effects:* Initializes c with first as the first argument and last as the second argument, and initializes comp with x; then calls make\_heap(c.begin(), c.end(), comp).

```
template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& x,
 const Container& y);
```

```
template<class InputIterator>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& x,
 Container&& y);
```

5     *Preconditions:* x defines a strict weak ordering (26.8).

6     *Effects:* Initializes comp with x and c with y (copy constructing or move constructing as appropriate); calls c.insert(c.end(), first, last); and finally calls make\_heap(c.begin(), c.end(), comp).

```
template<container-compatible-range<T> R>
 constexpr priority_queue(from_range_t, R&& rg, const Compare& x = Compare());
```

7     *Preconditions:* x defines a strict weak ordering (26.8).

8     *Effects:* Initializes comp with x and c with ranges::to<Container>(std::forward<R>(rg)) and finally calls make\_heap(c.begin(), c.end(), comp).

#### 23.6.4.3 Constructors with allocators

[priqueue.cons.alloc]

1     If uses\_allocator\_v<container\_type, Alloc> is false the constructors in this subclause shall not participate in overload resolution.

```
template<class Alloc> constexpr explicit priority_queue(const Alloc& a);
```

2     *Effects:* Initializes c with a and value-initializes comp.

```
template<class Alloc> constexpr priority_queue(const Compare& compare, const Alloc& a);
```

3     *Effects:* Initializes c with a and initializes comp with compare.

```
template<class Alloc>
 constexpr priority_queue(const Compare& compare, const Container& cont, const Alloc& a);
```

4     *Effects:* Initializes c with cont as the first argument and a as the second argument, and initializes comp with compare; calls make\_heap(c.begin(), c.end(), comp).

```
template<class Alloc>
 constexpr priority_queue(const Compare& compare, Container&& cont, const Alloc& a);
```

5     *Effects:* Initializes c with std::move(cont) as the first argument and a as the second argument, and initializes comp with compare; calls make\_heap(c.begin(), c.end(), comp).

```

template<class Alloc> constexpr priority_queue(const priority_queue& q, const Alloc& a);
6 Effects: Initializes c with q.c as the first argument and a as the second argument, and initializes comp
 with q.comp.

template<class Alloc> constexpr priority_queue(priority_queue&& q, const Alloc& a);
7 Effects: Initializes c with std::move(q.c) as the first argument and a as the second argument, and
 initializes comp with std::move(q.comp).

template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator first, InputIterator last, const Alloc& a);
8 Effects: Initializes c with first as the first argument, last as the second argument, and a as the third
 argument, and value-initializes comp; calls make_heap(c.begin(), c.end(), comp).

template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator first, InputIterator last,
 const Compare& compare, const Alloc& a);
9 Effects: Initializes c with first as the first argument, last as the second argument, and a as the third
 argument, and initializes comp with compare; calls make_heap(c.begin(), c.end(), comp).

template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& compare,
 const Container& cont, const Alloc& a);
10 Effects: Initializes c with cont as the first argument and a as the second argument, and initializes
 comp with compare; calls c.insert(c.end(), first, last); and finally calls make_heap(c.begin(),
 c.end(), comp).

template<class InputIterator, class Alloc>
 constexpr priority_queue(InputIterator first, InputIterator last, const Compare& compare,
 Container&& cont, const Alloc& a);
11 Effects: Initializes c with std::move(cont) as the first argument and a as the second argument,
 and initializes comp with compare; calls c.insert(c.end(), first, last); and finally calls make_
 heap(c.begin(), c.end(), comp).

template<container-compatible-range<T> R, class Alloc>
 constexpr priority_queue(from_range_t, R&& rg, const Compare& compare, const Alloc& a);
12 Effects: Initializes comp with compare and c with ranges::to<Container>(std::forward<R>(rg),
 a); calls make_heap(c.begin(), c.end(), comp).

template<container-compatible-range<T> R, class Alloc>
 constexpr priority_queue(from_range_t, R&& rg, const Alloc& a);
13 Effects: Initializes c with ranges::to<Container>(std::forward<R>(rg), a) and value-initializes
 comp; calls make_heap(c.begin(), c.end(), comp).

```

#### 23.6.4.4 Members

[priqueue.members]

```

constexpr void push(const value_type& x);
1 Effects: As if by:
 c.push_back(x);
 push_heap(c.begin(), c.end(), comp);

constexpr void push(value_type&& x);
2 Effects: As if by:
 c.push_back(std::move(x));
 push_heap(c.begin(), c.end(), comp);

template<container-compatible-range<T> R>
 constexpr void push_range(R&& rg);
3 Effects: Inserts all elements of rg in c via c.append_range(std::forward<R>(rg)) if that is a valid
 expression, or ranges::copy(rg, back_inserter(c)) otherwise. Then restores the heap property as

```

if by `make_heap(c.begin(), c.end(), comp)`.

4 *Postconditions:* `is_heap(c.begin(), c.end(), comp)` is true.

```
template<class... Args> constexpr void emplace(Args&&... args);
```

5 *Effects:* As if by:

```
 c.emplace_back(std::forward<Args>(args)...);
 push_heap(c.begin(), c.end(), comp);
```

```
constexpr void pop();
```

6 *Effects:* As if by:

```
 pop_heap(c.begin(), c.end(), comp);
 c.pop_back();
```

#### 23.6.4.5 Specialized algorithms

[priority\_queue.special]

```
template<class T, class Container, class Compare>
constexpr void swap(priority_queue<T, Container, Compare>& x,
 priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));
```

1 *Constraints:* `is_swappable_v<Container>` is true and `is_swappable_v<Compare>` is true.

2 *Effects:* As if by `x.swap(y)`.

#### 23.6.5 Header <stack> synopsis

[stack.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.6.6, class template stack
 template<class T, class Container = deque<T>> class stack;

 template<class T, class Container>
 constexpr bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator<= (const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, class Container>
 constexpr bool operator>= (const stack<T, Container>& x, const stack<T, Container>& y);
 template<class T, three_way_comparable Container>
 constexpr compare_three_way_result_t<Container>
 operator<=>(const stack<T, Container>& x, const stack<T, Container>& y);

 template<class T, class Container>
 constexpr void swap(stack<T, Container>& x, stack<T, Container>& y)
 noexcept(noexcept(x.swap(y)));
 template<class T, class Container, class Alloc>
 struct uses_allocator<stack<T, Container>, Alloc>;

 // 23.6.13, formatter specialization for stack
 template<class charT, class T, formattable<charT> Container>
 struct formatter<stack<T, Container>, charT>;

 template<class T, class Container>
 constexpr bool enable_nonlocking_formatter_optimization<stack<T, Container>> = false;
}
```

**23.6.6 Class template stack****[stack]****23.6.6.1 General****[stack.general]**

- <sup>1</sup> Any sequence container supporting operations `back()`, `push_back()` and `pop_back()` can be used to instantiate `stack`. In particular, `vector` (23.3.13), `list` (23.3.11) and `deque` (23.3.5) can be used.

**23.6.6.2 Definition****[stack.defn]**

```

namespace std {
 template<class T, class Container = deque<T>>
 class stack {
 public:
 using value_type = Container::value_type;
 using reference = Container::reference;
 using const_reference = Container::const_reference;
 using size_type = Container::size_type;
 using container_type = Container;

 protected:
 Container c;

 public:
 constexpr stack() : stack(Container()) {}
 constexpr explicit stack(const Container&);
 constexpr explicit stack(Container&&);
 template<class InputIterator> constexpr stack(InputIterator first, InputIterator last);
 template<container-compatible-range<T> R>
 constexpr stack(from_range_t, R&& rg);
 template<class Alloc> constexpr explicit stack(const Alloc&);
 template<class Alloc> constexpr stack(const Container&, const Alloc&);
 template<class Alloc> constexpr stack(Container&&, const Alloc&);
 template<class Alloc> constexpr stack(const stack&, const Alloc&);
 template<class Alloc> constexpr stack(stack&&, const Alloc&);
 template<class InputIterator, class Alloc>
 constexpr stack(InputIterator first, InputIterator last, const Alloc&);
 template<container-compatible-range<T> R, class Alloc>
 constexpr stack(from_range_t, R&& rg, const Alloc&);

 constexpr bool empty() const { return c.empty(); }
 constexpr size_type size() const { return c.size(); }
 constexpr reference top() const { return c.back(); }
 constexpr const_reference top() const { return c.back(); }
 constexpr void push(const value_type& x) { c.push_back(x); }
 constexpr void push(value_type&& x) { c.push_back(std::move(x)); }
 template<container-compatible-range<T> R>
 constexpr void push_range(R&& rg);
 template<class... Args>
 constexpr decltype(auto) emplace(Args&&... args)
 { return c.emplace_back(std::forward<Args>(args)...); }
 constexpr void pop() { c.pop_back(); }
 constexpr void swap(stack& s) noexcept(is_nothrow_swappable_v<Container>)
 { using std::swap; swap(c, s.c); }
 };

 template<class Container>
 stack(Container) -> stack<typename Container::value_type, Container>;

 template<class InputIterator>
 stack(InputIterator, InputIterator) -> stack<iter-value-type<InputIterator>>;

 template<ranges::input_range R>
 stack(from_range_t, R&&) -> stack<ranges::range_value_t<R>>;

 template<class Container, class Allocator>
 stack(Container, Allocator) -> stack<typename Container::value_type, Container>;

```

```

template<class InputIterator, class Allocator>
stack(InputIterator, InputIterator, Allocator)
-> stack<iter-value-type<InputIterator>, deque<iter-value-type<InputIterator>,
Allocator>>;

template<ranges::input_range R, class Allocator>
stack(from_range_t, R&&, Allocator)
-> stack<ranges::range_value_t<R>, deque<ranges::range_value_t<R>, Allocator>>;

template<class T, class Container, class Alloc>
struct uses_allocator<stack<T, Container>, Alloc>
: uses_allocator<Container, Alloc>::type { };
}

```

### 23.6.6.3 Constructors

[stack.cons]

```
constexpr explicit stack(const Container& cont);
```

1 *Effects:* Initializes c with cont.

```
constexpr explicit stack(Container&& cont);
```

2 *Effects:* Initializes c with std::move(cont).

```
template<class InputIterator>
constexpr stack(InputIterator first, InputIterator last);
```

3 *Effects:* Initializes c with first as the first argument and last as the second argument.

```
template<container-compatible-range<T> R>
constexpr stack(from_range_t, R&& rg);
```

4 *Effects:* Initializes c with ranges::to<Container>(std::forward<R>(rg)).

### 23.6.6.4 Constructors with allocators

[stack.cons.alloc]

1 If uses\_allocator\_v<container\_type, Alloc> is false the constructors in this subclause shall not participate in overload resolution.

```
template<class Alloc> constexpr explicit stack(const Alloc& a);
```

2 *Effects:* Initializes c with a.

```
template<class Alloc> constexpr stack(const container_type& cont, const Alloc& a);
```

3 *Effects:* Initializes c with cont as the first argument and a as the second argument.

```
template<class Alloc> constexpr stack(container_type&& cont, const Alloc& a);
```

4 *Effects:* Initializes c with std::move(cont) as the first argument and a as the second argument.

```
template<class Alloc> constexpr stack(const stack& s, const Alloc& a);
```

5 *Effects:* Initializes c with s.c as the first argument and a as the second argument.

```
template<class Alloc> constexpr stack(stack&& s, const Alloc& a);
```

6 *Effects:* Initializes c with std::move(s.c) as the first argument and a as the second argument.

```
template<class InputIterator, class Alloc>
constexpr stack(InputIterator first, InputIterator last, const Alloc& alloc);
```

7 *Effects:* Initializes c with first as the first argument, last as the second argument, and alloc as the third argument.

```
template<container-compatible-range<T> R, class Alloc>
constexpr stack(from_range_t, R&& rg, const Alloc& a);
```

8 *Effects:* Initializes c with ranges::to<Container>(std::forward<R>(rg), a).



**23.6.6.5 Modifiers**

[stack.mod]

```
template<container-compatible-range<T> R>
constexpr void push_range(R&& rg);
```

- 1     *Effects:* Equivalent to `c.append_range(std::forward<R>(rg))` if that is a valid expression, otherwise `ranges::copy(rg, back_inserter(c))`.

**23.6.6.6 Operators**

[stack.ops]

```
template<class T, class Container>
constexpr bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
```

- 1     *Returns:* `x.c == y.c`.

```
template<class T, class Container>
constexpr bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
```

- 2     *Returns:* `x.c != y.c`.

```
template<class T, class Container>
constexpr bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
```

- 3     *Returns:* `x.c < y.c`.

```
template<class T, class Container>
constexpr bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
```

- 4     *Returns:* `x.c > y.c`.

```
template<class T, class Container>
constexpr bool operator<= (const stack<T, Container>& x, const stack<T, Container>& y);
```

- 5     *Returns:* `x.c <= y.c`.

```
template<class T, class Container>
constexpr bool operator>= (const stack<T, Container>& x, const stack<T, Container>& y);
```

- 6     *Returns:* `x.c >= y.c`.

```
template<class T, three_way_comparable Container>
constexpr compare_three_way_result_t<Container>
operator<=>(const stack<T, Container>& x, const stack<T, Container>& y);
```

- 7     *Returns:* `x.c <=> y.c`.

**23.6.6.7 Specialized algorithms**

[stack.special]

```
template<class T, class Container>
constexpr void swap(stack<T, Container>& x, stack<T, Container>& y)
noexcept(noexcept(x.swap(y)));
```

- 1     *Constraints:* `is_swappable_v<Container>` is true.

- 2     *Effects:* As if by `x.swap(y)`.

**23.6.7 Header <flat\_map> synopsis**

[flat.map.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.6.8, class template flat_map
 template<class Key, class T, class Compare = less<Key>,
 class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
 class flat_map;

 struct sorted_unique_t { explicit sorted_unique_t() = default; };
 inline constexpr sorted_unique_t sorted_unique{};
```

```

template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Allocator>
 struct uses_allocator<flat_map<Key, T, Compare, KeyContainer, MappedContainer>,
 Allocator>;

// 23.6.8.8, erasure for flat_map
template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Predicate>
 constexpr typename flat_map<Key, T, Compare, KeyContainer, MappedContainer>::size_type
 erase_if(flat_map<Key, T, Compare, KeyContainer, MappedContainer>& c, Predicate pred);

// 23.6.9, class template flat_multimap
template<class Key, class T, class Compare = less<Key>,
 class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
 class flat_multimap;

struct sorted_equivalent_t { explicit sorted_equivalent_t() = default; };
inline constexpr sorted_equivalent_t sorted_equivalent{};

template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Allocator>
 struct uses_allocator<flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>,
 Allocator>;

// 23.6.9.5, erasure for flat_multimap
template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Predicate>
 constexpr typename flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>::size_type
 erase_if(flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>& c, Predicate pred);
}

```

## 23.6.8 Class template flat\_map

[flat.map]

### 23.6.8.1 Overview

[flat.map.overview]

- <sup>1</sup> A **flat\_map** is a container adaptor that provides an associative container interface that supports unique keys (i.e., contains at most one of each key value) and provides for fast retrieval of values of another type *T* based on the keys. **flat\_map** supports iterators that meet the *Cpp17InputIterator* requirements and model the *random\_access\_iterator* concept (24.3.4.13).
- <sup>2</sup> A **flat\_map** meets all of the requirements of a container (23.2.2.2) and of a reversible container (23.2.2.3), plus the optional container requirements (23.2.2.4). **flat\_map** meets the requirements of an associative container (23.2.7), except that:
  - (2.1) — it does not meet the requirements related to node handles (23.2.5),
  - (2.2) — it does not meet the requirements related to iterator invalidation, and
  - (2.3) — the time complexity of the operations that insert or erase a single element from the map is linear, including the ones that take an insertion position iterator.

[Note 1: A **flat\_map** does not meet the additional requirements of an allocator-aware container (23.2.2.5). — end note]
- <sup>3</sup> A **flat\_map** also provides most operations described in 23.2.7 for unique keys. This means that a **flat\_map** supports the **a\_uniq** operations in 23.2.7 but not the **a\_eq** operations. For a **flat\_map**<Key, T> the **key\_type** is **Key** and the **value\_type** is **pair<Key, T>**.
- <sup>4</sup> Descriptions are provided here only for operations on **flat\_map** that are not described in one of those sets of requirements or for operations where there is additional semantic information.
- <sup>5</sup> A **flat\_map** maintains the following invariants:
  - (5.1) — it contains the same number of keys and values;
  - (5.2) — the keys are sorted with respect to the comparison object; and
  - (5.3) — the value at offset **off** within the value container is the value associated with the key at offset **off** within the key container.

- <sup>6</sup> If any member function in 23.6.8.2 exits via an exception the invariants of the object argument are restored. For the move constructor and move assignment operator, the invariants of both arguments are restored.  
 [Note 2: This can result in the `flat_map` being emptied. — end note]
- <sup>7</sup> Any type `C` that meets the sequence container requirements (23.2.4) can be used to instantiate `flat_map`, as long as `C::iterator` meets the *Cpp17RandomAccessIterator* requirements and invocations of member functions `C::size` and `C::max_size` do not exit via an exception. In particular, `vector` (23.3.13) and `deque` (23.3.5) can be used.  
 [Note 3: `vector<bool>` is not a sequence container. — end note]
- <sup>8</sup> The program is ill-formed if `Key` is not the same type as `KeyContainer::value_type` or `T` is not the same type as `MappedContainer::value_type`.
- <sup>9</sup> The effect of calling a constructor that takes both `key_container_type` and `mapped_container_type` arguments with containers of different sizes is undefined.
- <sup>10</sup> The effect of calling a member function that takes a `sorted_unique_t` argument with a container, containers, or range that is not sorted with respect to `key_comp()`, or that contains equal elements, is undefined.
- <sup>11</sup> The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

### 23.6.8.2 Definition

[flat.map.defn]

```
namespace std {
 template<class Key, class T, class Compare = less<Key>,
 class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
 class flat_map {
 public:
 // types
 using key_type = Key;
 using mapped_type = T;
 using value_type = pair<key_type, mapped_type>;
 using key_compare = Compare;
 using reference = pair<const key_type&, mapped_type&>;
 using const_reference = pair<const key_type&, const mapped_type&>;
 using size_type = size_t;
 using difference_type = ptrdiff_t;
 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 using reverse_iterator = std::reverse_iterator<iterator>;
 using const_reverse_iterator = std::reverse_iterator<const_iterator>;
 using key_container_type = KeyContainer;
 using mapped_container_type = MappedContainer;

 class value_compare {
 private:
 key_compare comp;
 constexpr value_compare(key_compare c) : comp(c) { } // exposition only
 public:
 constexpr bool operator()(const_reference x, const_reference y) const {
 return comp(x.first, y.first);
 }
 };

 struct containers {
 key_container_type keys;
 mapped_container_type values;
 };

 // 23.6.8.3, constructors
 constexpr flat_map() : flat_map(key_compare()) { }

 constexpr flat_map(const flat_map&);
 constexpr flat_map(flat_map&&);
 constexpr flat_map& operator=(const flat_map&);
 };
};
```

```

constexpr flat_map& operator=(flat_map&&);

constexpr explicit flat_map(const key_compare& comp)
 : c(), compare(comp) { }

constexpr flat_map(key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

constexpr flat_map(sorted_unique_t, key_container_type key_cont,
 mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

template<class InputIterator>
 constexpr flat_map(InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp) { insert(first, last); }

template<class InputIterator>
 constexpr flat_map(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp) { insert(sorted_unique, first, last); }

template<container-compatible-range<value_type> R>
 constexpr flat_map(from_range_t, R&& rg)
 : flat_map(from_range, std::forward<R>(rg), key_compare()) { }
template<container-compatible-range<value_type> R>
 constexpr flat_map(from_range_t, R&& rg, const key_compare& comp)
 : flat_map(comp) { insert_range(std::forward<R>(rg)); }

constexpr flat_map(initializer_list<value_type> il, const key_compare& comp = key_compare())
 : flat_map(il.begin(), il.end(), comp) { }

constexpr flat_map(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_map(sorted_unique, il.begin(), il.end(), comp) { }

// 23.6.8.4, constructors with allocators

template<class Alloc>
 constexpr explicit flat_map(const Alloc& a);
template<class Alloc>
 constexpr flat_map(const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_map(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont,
 const Alloc& a);
template<class Alloc>
 constexpr flat_map(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont,
 const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_map(sorted_unique_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);
template<class Alloc>
 constexpr flat_map(sorted_unique_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_map(const flat_map&, const Alloc& a);
template<class Alloc>
 constexpr flat_map(flat_map&&, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_map(InputIterator first, InputIterator last, const Alloc& a);

```

```

template<class InputIterator, class Alloc>
 constexpr flat_map(InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_map(sorted_unique_t, InputIterator first, InputIterator last,
 const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_map(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_map(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_map(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_map(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_map(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_map(sorted_unique_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_map(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

constexpr flat_map& operator=(initializer_list<value_type>);

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 23.6.8.5, capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.6.8.6, element access
constexpr mapped_type& operator[](const key_type& x);
constexpr mapped_type& operator[](key_type&& x);
template<class K> constexpr mapped_type& operator[](K&& x);
constexpr mapped_type& at(const key_type& x);
constexpr const mapped_type& at(const key_type& x) const;
template<class K> constexpr mapped_type& at(const K& x);
template<class K> constexpr const mapped_type& at(const K& x) const;
constexpr optional<mapped_type&> lookup(const key_type& x);
constexpr optional<const mapped_type&> lookup(const key_type& x) const;
template<class K> constexpr optional<mapped_type&> lookup(const K& x);
template<class K> constexpr optional<const mapped_type&> lookup(const K& x) const;

// 23.6.8.7, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);

```

```

constexpr pair<iterator, bool> insert(const value_type& x)
{ return emplace(x); }
constexpr pair<iterator, bool> insert(value_type&& x)
{ return emplace(std::move(x)); }
constexpr iterator insert(const_iterator position, const value_type& x)
{ return emplace_hint(position, x); }
constexpr iterator insert(const_iterator position, value_type&& x)
{ return emplace_hint(position, std::move(x)); }

template<class P> constexpr pair<iterator, bool> insert(P&& x);
template<class P>
 constexpr iterator insert(const_iterator position, P&&);
template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<class InputIterator>
 constexpr void insert(sorted_unique_t, InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(sorted_unique_t, R&& rg);

constexpr void insert(initializer_list<value_type> il)
{ insert(il.begin(), il.end()); }
constexpr void insert(sorted_unique_t, initializer_list<value_type> il)
{ insert(sorted_unique, il.begin(), il.end()); }

constexpr containers extract() &&;
constexpr void replace(key_container_type&& key_cont, mapped_container_type&& mapped_cont);

template<class... Args>
 constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
 constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template<class K, class... Args>
 constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class... Args>
 constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
 constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template<class K, class... Args>
 constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
template<class M>
 constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
 constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template<class K, class M>
 constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
template<class M>
 constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
 constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
template<class K, class M>
 constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void swap(flat_map& y) noexcept(see below);
constexpr void clear() noexcept;

```

```

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

constexpr const key_container_type& keys() const noexcept { return c.keys; }
constexpr const mapped_container_type& values() const noexcept { return c.values; }

// map operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K> constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;

friend constexpr bool operator==(const flat_map& x, const flat_map& y);

friend constexpr synth-three-way-result<value_type>
 operator<=>(const flat_map& x, const flat_map& y);

friend constexpr void swap(flat_map& x, flat_map& y) noexcept(noexcept(x.swap(y)))
 { x.swap(y); }

private:
 containers c; // exposition only
 key_compare compare; // exposition only

 struct key-equiv { // exposition only
 constexpr key-equiv(key_compare c) : comp(c) { }
 constexpr bool operator()(const_reference x, const_reference y) const {
 return !comp(x.first, y.first) && !comp(y.first, x.first);
 }
 key_compare comp;
 };
};

template<class KeyContainer, class MappedContainer,
 class Compare = less<typename KeyContainer::value_type>>
flat_map(KeyContainer, MappedContainer, Compare = Compare())
 -> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
 Compare, KeyContainer, MappedContainer>;

```

```

template<class KeyContainer, class MappedContainer, class Allocator>
flat_map(KeyContainer, MappedContainer, Allocator)
-> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
flat_map(KeyContainer, MappedContainer, Compare, Allocator)
-> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer,
class Compare = less<typename KeyContainer::value_type>>
flat_map(sorted_unique_t, KeyContainer, MappedContainer, Compare = Compare())
-> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Allocator>
flat_map(sorted_unique_t, KeyContainer, MappedContainer, Allocator)
-> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
flat_map(sorted_unique_t, KeyContainer, MappedContainer, Compare, Allocator)
-> flat_map<typename KeyContainer::value_type, typename MappedContainer::value_type,
Compare, KeyContainer, MappedContainer>;

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>>
flat_map(InputIterator, InputIterator, Compare = Compare())
-> flat_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare>;

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>>
flat_map(sorted_unique_t, InputIterator, InputIterator, Compare = Compare())
-> flat_map<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare>;

template<ranges::input_range R, class Compare = less<range-key-type<R>>,
class Allocator = allocator<byte>>
flat_map(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
-> flat_map<range-key-type<R>, range-mapped-type<R>, Compare,
vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<ranges::input_range R, class Allocator>
flat_map(from_range_t, R&&, Allocator)
-> flat_map<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
vector<range-key-type<R>, alloc-rebind<Allocator, range-key-type<R>>>,
vector<range-mapped-type<R>, alloc-rebind<Allocator, range-mapped-type<R>>>>;

template<class Key, class T, class Compare = less<Key>>
flat_map(initializer_list<pair<Key, T>>, Compare = Compare())
-> flat_map<Key, T, Compare>;

template<class Key, class T, class Compare = less<Key>>
flat_map(sorted_unique_t, initializer_list<pair<Key, T>>, Compare = Compare())
-> flat_map<Key, T, Compare>;

template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
class Allocator>
struct uses_allocator<flat_map<Key, T, Compare, KeyContainer, MappedContainer>, Allocator>
: bool_constant<uses_allocator_v<KeyContainer, Allocator> &&
uses_allocator_v<MappedContainer, Allocator>> { };
}

```

<sup>1</sup> The member type `containers` has the data members and special members specified above. It has no base classes or members other than those specified.



## 23.6.8.3 Constructors

[flat.map.cons]

```
constexpr flat_map(key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());
```

- 1 *Effects:* Initializes `c.keys` with `std::move(key_cont)`, `c.values` with `std::move(mapped_cont)`, and `compare` with `comp`; sorts the range `[begin(), end())` with respect to `value_comp()`; and finally erases the duplicate elements as if by:

```
 auto zv = views::zip(c.keys, c.values);
 auto it = ranges::unique(zv, key_equiv(compare)).begin();
 auto dist = distance(zv.begin(), it);
 c.keys.erase(c.keys.begin() + dist, c.keys.end());
 c.values.erase(c.values.begin() + dist, c.values.end());
```

- 2 *Complexity:* Linear in  $N$  if the container arguments are already sorted with respect to `value_comp()` and otherwise  $N \log N$ , where  $N$  is the value of `key_cont.size()` before this call.

```
constexpr flat_map(sorted_unique_t, key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());
```

- 3 *Effects:* Initializes `c.keys` with `std::move(key_cont)`, `c.values` with `std::move(mapped_cont)`, and `compare` with `comp`.

- 4 *Complexity:* Constant.

## 23.6.8.4 Constructors with allocators

[flat.map.cons.alloc]

- 1 The constructors in this subclause shall not participate in overload resolution unless `uses_allocator_v<key_container_type, Alloc>` is true and `uses_allocator_v<mapped_container_type, Alloc>` is true.

```
template<class Alloc>
constexpr flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
 const Alloc& a);
```

```
template<class Alloc>
constexpr flat_map(const key_container_type& key_cont, const mapped_container_type& mapped_cont,
 const key_compare& comp, const Alloc& a);
```

- 2 *Effects:* Equivalent to `flat_map(key_cont, mapped_cont)` and `flat_map(key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

- 3 *Complexity:* Same as `flat_map(key_cont, mapped_cont)` and `flat_map(key_cont, mapped_cont, comp)`, respectively.

```
template<class Alloc>
constexpr flat_map(sorted_unique_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);
```

```
template<class Alloc>
constexpr flat_map(sorted_unique_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const key_compare& comp,
 const Alloc& a);
```

- 4 *Effects:* Equivalent to `flat_map(sorted_unique, key_cont, mapped_cont)` and `flat_map(sorted_unique, key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

- 5 *Complexity:* Linear.

```
template<class Alloc>
constexpr explicit flat_map(const Alloc& a);
```

```
template<class Alloc>
constexpr flat_map(const key_compare& comp, const Alloc& a);
```

```
template<class Alloc>
constexpr flat_map(const flat_map&, const Alloc& a);
```

```
template<class Alloc>
constexpr flat_map(flat_map&&, const Alloc& a);
```

```
template<class InputIterator, class Alloc>
constexpr flat_map(InputIterator first, InputIterator last, const Alloc& a);
```

```

template<class InputIterator, class Alloc>
constexpr flat_map(InputIterator first, InputIterator last, const key_compare& comp,
 const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_map(sorted_unique_t, InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_map(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_map(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_map(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_map(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_map(initializer_list<value_type> il, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_map(sorted_unique_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_map(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

```

- 6 *Effects:* Equivalent to the corresponding non-allocator constructors except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

### 23.6.8.5 Capacity

[flat.map.capacity]

```
constexpr size_type size() const noexcept;
```

- 1 *Returns:* `c.keys.size()`.

```
constexpr size_type max_size() const noexcept;
```

- 2 *Returns:* `min<size_type>(c.keys.max_size(), c.values.max_size())`.

### 23.6.8.6 Access

[flat.map.access]

```
constexpr mapped_type& operator[](const key_type& x);
```

- 1 *Effects:* Equivalent to: `return try_emplace(x).first->second;`

```
constexpr mapped_type& operator[](key_type&& x);
```

- 2 *Effects:* Equivalent to: `return try_emplace(std::move(x)).first->second;`

```
template<class K> constexpr mapped_type& operator[](K&& x);
```

- 3 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

- 4 *Effects:* Equivalent to: `return try_emplace(std::forward<K>(x)).first->second;`

```
constexpr mapped_type& at(const key_type& x);
```

```
constexpr const mapped_type& at(const key_type& x) const;
```

- 5 *Returns:* A reference to the `mapped_type` corresponding to `x` in `*this`.

- 6 *Throws:* An exception object of type `out_of_range` if no such element is present.

- 7 *Complexity:* Logarithmic.

```
template<class K> constexpr mapped_type& at(const K& x);
```

```
template<class K> constexpr const mapped_type& at(const K& x) const;
```

- 8 *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

- 9 *Preconditions:* The expression `find(x)` is well-formed and has well-defined behavior.

- 10 *Returns:* A reference to the `mapped_type` corresponding to `x` in `*this`.

- 11 *Throws:* An exception object of type `out_of_range` if no such element is present.

- 12 *Complexity:* Logarithmic.

```
constexpr optional<mapped_type> lookup(const key_type& x);
constexpr optional<const mapped_type> lookup(const key_type& x) const;
template<class K> constexpr optional<mapped_type> lookup(const K& x);
template<class K> constexpr optional<const mapped_type> lookup(const K& x) const;
```

13 *Constraints:* For the third and fourth overloads, the *qualified-id* `Compare::is_transparent` is valid and denotes a type.

14 *Preconditions:* The expression `find(x)` is well-formed and has well-defined behavior.

15 *Returns:* `find(x)->second` if `contains(x)` is true, otherwise `nullopt`.

16 *Complexity:* Logarithmic.

### 23.6.8.7 Modifiers

[flat.map.modifiers]

```
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
```

1 *Constraints:* `is_constructible_v<pair<key_type, mapped_type>, Args...>` is true.

2 *Effects:* Initializes an object `t` of type `pair<key_type, mapped_type>` with `std::forward<Args>(args)...`; if the map already contains an element whose key is equivalent to `t.first`, `*this` is unchanged. Otherwise, equivalent to:

```
auto key_it = ranges::upper_bound(c.keys, t.first, compare);
auto value_it = c.values.begin() + distance(c.keys.begin(), key_it);
c.keys.insert(key_it, std::move(t.first));
c.values.insert(value_it, std::move(t.second));
```

3 *Returns:* The `bool` component of the returned pair is true if and only if the insertion took place, and the iterator component of the pair points to the element with key equivalent to `t.first`.

```
template<class P> constexpr pair<iterator, bool> insert(P&& x);
template<class P> constexpr iterator insert(const_iterator position, P&& x);
```

4 *Constraints:* `is_constructible_v<pair<key_type, mapped_type>, P>` is true.

5 *Effects:* The first form is equivalent to `return emplace(std::forward<P>(x));`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x));`.

```
template<class InputIterator>
constexpr void insert(InputIterator first, InputIterator last);
```

6 *Effects:* Adds elements to `c` as if by:

```
for (; first != last; ++first) {
 value_type value = *first;
 c.keys.insert(c.keys.end(), std::move(value.first));
 c.values.insert(c.values.end(), std::move(value.second));
}
```

Then, sorts the range of newly inserted elements with respect to `value_comp()`; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases the duplicate elements as if by:

```
auto zv = views::zip(c.keys, c.values);
auto it = ranges::unique(zv, key-equiv(compare)).begin();
auto dist = distance(zv.begin(), it);
c.keys.erase(c.keys.begin() + dist, c.keys.end());
c.values.erase(c.values.begin() + dist, c.values.end());
```

7 *Complexity:*  $N + M \log M$ , where  $N$  is `size()` before the operation and  $M$  is `distance(first, last)`.

8 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```
template<class InputIterator>
constexpr void insert(sorted_unique_t, InputIterator first, InputIterator last);
```

9 *Effects:* Equivalent to `insert(first, last)`.

10 *Complexity:* Linear in  $N$ , where  $N$  is `size()` after the operation.

```
template<container-compatible-range<value_type> R>
constexpr void insert_range(R&& rg);
```

11 *Effects:* Adds elements to *c* as if by:

```
 ranges::for_each(rg, [&](value_type e) {
 c.keys.insert(c.keys.end(), std::move(e.first));
 c.values.insert(c.values.end(), std::move(e.second));
 });
```

Then, sorts the range of newly inserted elements with respect to `value_comp()`; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases the duplicate elements as if by:

```
 auto zv = views::zip(c.keys, c.values);
 auto it = ranges::unique(zv, key-equiv(compare)).begin();
 auto dist = distance(zv.begin(), it);
 c.keys.erase(c.keys.begin() + dist, c.keys.end());
 c.values.erase(c.values.begin() + dist, c.values.end());
```

12 *Complexity:*  $N + M \log M$ , where  $N$  is `size()` before the operation and  $M$  is `ranges::distance(rg)`.

13 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```
template<container-compatible-range<value_type> R>
constexpr void insert_range(sorted_unique_t, R&& rg);
```

14 *Effects:* Equivalent to `insert_range(rg)`.

15 *Complexity:* Linear in  $N$ , where  $N$  is `size()` after the operation.

```
template<class... Args>
constexpr pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template<class... Args>
constexpr pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template<class... Args>
constexpr iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template<class... Args>
constexpr iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

16 *Constraints:* `is_constructible_v<mapped_type, Args...>` is true.

17 *Effects:* If the map already contains an element whose key is equivalent to *k*, *\*this* and *args...* are unchanged. Otherwise equivalent to:

```
 auto key_it = ranges::upper_bound(c.keys, k, compare);
 auto value_it = c.values.begin() + distance(c.keys.begin(), key_it);
 c.keys.insert(key_it, std::forward<decltype(k)>(k));
 c.values.emplace(value_it, std::forward<Args>(args)...);
```

18 *Returns:* In the first two overloads, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to *k*.

19 *Complexity:* The same as `emplace` for the first two overloads, and the same as `emplace_hint` for the last two overloads.

```
template<class K, class... Args>
constexpr pair<iterator, bool> try_emplace(K&& k, Args&&... args);
template<class K, class... Args>
constexpr iterator try_emplace(const_iterator hint, K&& k, Args&&... args);
```

20 *Constraints:*

(20.1) — The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

(20.2) — `is_constructible_v<key_type, K>` is true.

(20.3) — `is_constructible_v<mapped_type, Args...>` is true.

(20.4) — For the first overload, `is_convertible_v<K&&, const_iterator>` and `is_convertible_v<K&&, iterator>` are both false.

21 *Preconditions:* The conversion from *k* into `key_type` constructs an object *u*, for which `find(k) == find(u)` is true.

22 *Effects:* If the map already contains an element whose key is equivalent to `k`, `*this` and `args...` are unchanged. Otherwise equivalent to:

```
 auto key_it = upper_bound(c.keys.begin(), c.keys.end(), k, compare);
 auto value_it = c.values.begin() + distance(c.keys.begin(), key_it);
 c.keys.emplace(key_it, std::forward<K>(k));
 c.values.emplace(value_it, std::forward<Args>(args)...);
```

23 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

24 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template<class M>
constexpr pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template<class M>
constexpr pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template<class M>
constexpr iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template<class M>
constexpr iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

25 *Constraints:* `is_assignable_v<mapped_type&, M>` is `true` and `is_constructible_v<mapped_type, M>` is `true`.

26 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, equivalent to

```
 try_emplace(std::forward<decltype(k)>(k), std::forward<M>(obj))
```

for the first two overloads or

```
 try_emplace(hint, std::forward<decltype(k)>(k), std::forward<M>(obj))
```

for the last two overloads.

27 *Returns:* In the first two overloads, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

28 *Complexity:* The same as `emplace` for the first two overloads and the same as `emplace_hint` for the last two overloads.

```
template<class K, class M>
constexpr pair<iterator, bool> insert_or_assign(K&& k, M&& obj);
template<class K, class M>
constexpr iterator insert_or_assign(const_iterator hint, K&& k, M&& obj);
```

29 *Constraints:*

(29.1) — The *qualified-id* `Compare::is_transparent` is valid and denotes a type.

(29.2) — `is_constructible_v<key_type, K>` is `true`.

(29.3) — `is_assignable_v<mapped_type&, M>` is `true`.

(29.4) — `is_constructible_v<mapped_type, M>` is `true`.

30 *Preconditions:* The conversion from `k` into `key_type` constructs an object `u`, for which `find(k) == find(u)` is `true`.

31 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `std::forward<M>(obj)` to `e.second`. Otherwise, equivalent to

```
 try_emplace(std::forward<K>(k), std::forward<M>(obj))
```

for the first overload or

```
 try_emplace(hint, std::forward<K>(k), std::forward<M>(obj))
```

for the second overload.

32 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

33 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
constexpr void swap(flat_map& y)
 noexcept(is_nothrow_swappable_v<key_container_type> &&
 is_nothrow_swappable_v<mapped_container_type> &&
 is_nothrow_swappable_v<key_compare>);
```

34 *Effects:* Equivalent to:

```
 ranges::swap(compare, y.compare);
 ranges::swap(c.keys, y.c.keys);
 ranges::swap(c.values, y.c.values);
```

```
constexpr containers extract() &&;
```

35 *Postconditions:* *\*this* is emptied, even if the function exits via an exception.

36 *Returns:* `std::move(c)`.

```
constexpr void replace(key_container_type&& key_cont, mapped_container_type&& mapped_cont);
```

37 *Preconditions:* `key_cont.size() == mapped_cont.size()` is true, the elements of `key_cont` are sorted with respect to *compare*, and `key_cont` contains no equal elements.

38 *Effects:* Equivalent to:

```
 c.keys = std::move(key_cont);
 c.values = std::move(mapped_cont);
```

### 23.6.8.8 Erasure

[flat.map.erase]

```
template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Predicate>
constexpr typename flat_map<Key, T, Compare, KeyContainer, MappedContainer>::size_type
erase_if(flat_map<Key, T, Compare, KeyContainer, MappedContainer>& c, Predicate pred);
```

1 *Preconditions:* Key and T meet the *Cpp17MoveAssignable* requirements.

2 *Effects:* Let *E* be `bool(pred(pair<const Key&, const T&>(e)))`. Erases all elements *e* in *c* for which *E* holds.

3 *Returns:* The number of elements erased.

4 *Complexity:* Exactly `c.size()` applications of the predicate.

5 *Remarks:* Stable (16.4.6.8). If an invocation of `erase_if` exits via an exception, *c* is in a valid but unspecified state (3.68).

[Note 1: *c* still meets its invariants, but can be empty. — end note]

## 23.6.9 Class template flat\_multimap

[flat.multimap]

### 23.6.9.1 Overview

[flat.multimap.overview]

- 1 A `flat_multimap` is a container adaptor that provides an associative container interface that supports equivalent keys (i.e., possibly containing multiple copies of the same key value) and provides for fast retrieval of values of another type *T* based on the keys. `flat_multimap` supports iterators that meet the *Cpp17InputIterator* requirements and model the `random_access_iterator` concept (24.3.4.13).
- 2 A `flat_multimap` meets all of the requirements for a container (23.2.2.2) and for a reversible container (23.2.2.3), plus the optional container requirements (23.2.2.4). `flat_multimap` meets the requirements of an associative container (23.2.7), except that:
  - (2.1) — it does not meet the requirements related to node handles (23.2.5),
  - (2.2) — it does not meet the requirements related to iterator invalidation, and
  - (2.3) — the time complexity of the operations that insert or erase a single element from the map is linear, including the ones that take an insertion position iterator.

[Note 1: A `flat_multimap` does not meet the additional requirements of an allocator-aware container (23.2.2.5). — end note]

- 3 A `flat_multimap` also provides most operations described in 23.2.7 for equal keys. This means that a `flat_multimap` supports the `a_eq` operations in 23.2.7 but not the `a_uniq` operations. For a `flat_multimap<Key, T>` the `key_type` is *Key* and the `value_type` is `pair<Key, T>`.

- 4 Except as otherwise noted, operations on `flat_multimap` are equivalent to those of `flat_map`, except that `flat_multimap` operations do not remove or replace elements with equal keys.

[*Example 1: `flat_multimap` constructors and `emplace` do not erase non-unique elements after sorting them. — end example*]

- 5 A `flat_multimap` maintains the following invariants:

- (5.1) — it contains the same number of keys and values;
- (5.2) — the keys are sorted with respect to the comparison object; and
- (5.3) — the value at offset `off` within the value container is the value associated with the key at offset `off` within the key container.

- 6 If any member function in 23.6.9.2 exits via an exception, the invariants of the object argument are restored. For the move constructor and move assignment operator, the invariants of both arguments are restored.

[*Note 2: This can result in the `flat_multimap` being emptied. — end note*]

- 7 Any type `C` that meets the sequence container requirements (23.2.4) can be used to instantiate `flat_multimap`, as long as `C::iterator` meets the *Cpp17RandomAccessIterator* requirements and invocations of member functions `C::size` and `C::max_size` do not exit via an exception. In particular, `vector` (23.3.13) and `deque` (23.3.5) can be used.

[*Note 3: `vector<bool>` is not a sequence container. — end note*]

- 8 The program is ill-formed if `Key` is not the same type as `KeyContainer::value_type` or `T` is not the same type as `MappedContainer::value_type`.

- 9 The effect of calling a constructor that takes both `key_container_type` and `mapped_container_type` arguments with containers of different sizes is undefined.

- 10 The effect of calling a member function that takes a `sorted_equivalent_t` argument with a container, containers, or range that are not sorted with respect to `key_comp()` is undefined.

- 11 The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

### 23.6.9.2 Definition

[`flat.multimap.defn`]

```
namespace std {
 template<class Key, class T, class Compare = less<Key>,
 class KeyContainer = vector<Key>, class MappedContainer = vector<T>>
 class flat_multimap {
 public:
 // types
 using key_type = Key;
 using mapped_type = T;
 using value_type = pair<key_type, mapped_type>;
 using key_compare = Compare;
 using reference = pair<const key_type&, mapped_type&>;
 using const_reference = pair<const key_type&, const mapped_type&>;
 using size_type = size_t;
 using difference_type = ptrdiff_t;
 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 using reverse_iterator = std::reverse_iterator<iterator>;
 using const_reverse_iterator = std::reverse_iterator<const_iterator>;
 using key_container_type = KeyContainer;
 using mapped_container_type = MappedContainer;

 class value_compare {
 private:
 key_compare comp; // exposition only
 constexpr value_compare(key_compare c) : comp(c) { } // exposition only

 public:
 constexpr bool operator()(const_reference x, const_reference y) const {
 return comp(x.first, y.first);
 }
 };
 };
}
```

```

struct containers {
 key_container_type keys;
 mapped_container_type values;
};

// 23.6.9.3, constructors
constexpr flat_multimap() : flat_multimap(key_compare()) { }

constexpr flat_multimap(const flat_multimap&);
constexpr flat_multimap(flat_multimap&&);
constexpr flat_multimap& operator=(const flat_multimap&);
constexpr flat_multimap& operator=(flat_multimap&&);

constexpr explicit flat_multimap(const key_compare& comp)
 : c(), compare(comp) { }

constexpr flat_multimap(key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

constexpr flat_multimap(sorted_equivalent_t,
 key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

template<class InputIterator>
 constexpr flat_multimap(InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp)
 { insert(first, last); }

template<class InputIterator>
 constexpr flat_multimap(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp) { insert(sorted_equivalent, first, last); }

template<container-compatible-range<value_type> R>
 constexpr flat_multimap(from_range_t, R&& rg)
 : flat_multimap(from_range, std::forward<R>(rg), key_compare()) { }
template<container-compatible-range<value_type> R>
 constexpr flat_multimap(from_range_t, R&& rg, const key_compare& comp)
 : flat_multimap(comp) { insert_range(std::forward<R>(rg)); }

constexpr flat_multimap(initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_multimap(il.begin(), il.end(), comp) { }

constexpr flat_multimap(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_multimap(sorted_equivalent, il.begin(), il.end(), comp) { }

// 23.6.9.4, constructors with allocators

template<class Alloc>
 constexpr explicit flat_multimap(const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont,
 const key_compare& comp, const Alloc& a);

```



```

template<class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont,
 const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(const flat_multimap&, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(flat_multimap&&, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multimap(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multimap(InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, InputIterator first, InputIterator last,
 const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multimap(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multimap(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, initializer_list<value_type> il,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multimap(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

flat_multimap& operator=(initializer_list<value_type>);

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);

```

```

template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);

constexpr iterator insert(const value_type& x)
 { return emplace(x); }
constexpr iterator insert(value_type&& x)
 { return emplace(std::move(x)); }
constexpr iterator insert(const_iterator position, const value_type& x)
 { return emplace_hint(position, x); }
constexpr iterator insert(const_iterator position, value_type&& x)
 { return emplace_hint(position, std::move(x)); }

template<class P> constexpr iterator insert(P&& x);
template<class P>
 constexpr iterator insert(const_iterator position, P&&);
template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<class InputIterator>
 constexpr void insert(sorted_equivalent_t, InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(sorted_equivalent_t, R&& rg);

constexpr void insert(initializer_list<value_type> il)
 { insert(il.begin(), il.end()); }
constexpr void insert(sorted_equivalent_t, initializer_list<value_type> il)
 { insert(sorted_equivalent, il.begin(), il.end()); }

constexpr containers extract() &&;
constexpr void replace(key_container_type&& key_cont, mapped_container_type&& mapped_cont);

constexpr iterator erase(iterator position);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void swap(flat_multimap&)
 noexcept(is_nothrow_swappable_v<key_container_type> &&
 is_nothrow_swappable_v<mapped_container_type> &&
 is_nothrow_swappable_v<key_compare>);
constexpr void clear() noexcept;

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

constexpr const key_container_type& keys() const noexcept { return c.keys; }
constexpr const mapped_container_type& values() const noexcept { return c.values; }

// map operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

```

```

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;

friend constexpr bool operator==(const flat_multimap& x, const flat_multimap& y);

friend constexpr synth-three-way-result<value_type>
 operator<=>(const flat_multimap& x, const flat_multimap& y);

friend constexpr void swap(flat_multimap& x, flat_multimap& y)
 noexcept(noexcept(x.swap(y)))
 { x.swap(y); }

private:
 containers c; // exposition only
 key_compare compare; // exposition only
};

template<class KeyContainer, class MappedContainer,
 class Compare = less<typename KeyContainer::value_type>>
 flat_multimap(KeyContainer, MappedContainer, Compare = Compare())
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Allocator>
 flat_multimap(KeyContainer, MappedContainer, Allocator)
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
 flat_multimap(KeyContainer, MappedContainer, Compare, Allocator)
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer,
 class Compare = less<typename KeyContainer::value_type>>
 flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Compare = Compare())
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 Compare, KeyContainer, MappedContainer>;

template<class KeyContainer, class MappedContainer, class Allocator>
 flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Allocator)
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 less<typename KeyContainer::value_type>, KeyContainer, MappedContainer>;
template<class KeyContainer, class MappedContainer, class Compare, class Allocator>
 flat_multimap(sorted_equivalent_t, KeyContainer, MappedContainer, Compare, Allocator)
 -> flat_multimap<typename KeyContainer::value_type, typename MappedContainer::value_type,
 Compare, KeyContainer, MappedContainer>;

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>>
 flat_multimap(InputIterator, InputIterator, Compare = Compare())
 -> flat_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare>;

```

```

template<class InputIterator, class Compare = less<iter-key-type<InputIterator>>>
 flat_multimap(sorted_equivalent_t, InputIterator, InputIterator, Compare = Compare())
 -> flat_multimap<iter-key-type<InputIterator>, iter-mapped-type<InputIterator>, Compare>;

template<ranges::input_range R, class Compare = less<range-key-type<R>>,
 class Allocator = allocator<byte>>
 flat_multimap(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> flat_multimap<range-key-type<R>, range-mapped-type<R>, Compare,
 vector<range-key-type<R>,
 alloc-rebind<Allocator, range-key-type<R>>>,
 vector<range-mapped-type<R>,
 alloc-rebind<Allocator, range-mapped-type<R>>>>>;

template<ranges::input_range R, class Allocator>
 flat_multimap(from_range_t, R&&, Allocator)
 -> flat_multimap<range-key-type<R>, range-mapped-type<R>, less<range-key-type<R>>,
 vector<range-key-type<R>,
 alloc-rebind<Allocator, range-key-type<R>>>,
 vector<range-mapped-type<R>,
 alloc-rebind<Allocator, range-mapped-type<R>>>>>;

template<class Key, class T, class Compare = less<Key>>
 flat_multimap(initializer_list<pair<Key, T>>, Compare = Compare())
 -> flat_multimap<Key, T, Compare>;

template<class Key, class T, class Compare = less<Key>>
 flat_multimap(sorted_equivalent_t, initializer_list<pair<Key, T>>, Compare = Compare())
 -> flat_multimap<Key, T, Compare>;

template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Allocator>
 struct uses_allocator<flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>,
 Allocator>
 : bool_constant<uses_allocator_v<KeyContainer, Allocator> &&
 uses_allocator_v<MappedContainer, Allocator>> { };
}

```

- <sup>1</sup> The member type `containers` has the data members and special members specified above. It has no base classes or members other than those specified.

### 23.6.9.3 Constructors

[flat.multimap.cons]

```

constexpr flat_multimap(key_container_type key_cont, mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

```

- <sup>1</sup> *Effects:* Initializes `c.keys` with `std::move(key_cont)`, `c.values` with `std::move(mapped_cont)`, and `compare` with `comp`; sorts the range `[begin(), end())` with respect to `value_comp()`.
- <sup>2</sup> *Complexity:* Linear in  $N$  if the container arguments are already sorted with respect to `value_comp()` and otherwise  $N \log N$ , where  $N$  is the value of `key_cont.size()` before this call.

```

constexpr flat_multimap(sorted_equivalent_t, key_container_type key_cont,
 mapped_container_type mapped_cont,
 const key_compare& comp = key_compare());

```

- <sup>3</sup> *Effects:* Initializes `c.keys` with `std::move(key_cont)`, `c.values` with `std::move(mapped_cont)`, and `compare` with `comp`.
- <sup>4</sup> *Complexity:* Constant.

### 23.6.9.4 Constructors with allocators

[flat.multimap.cons.alloc]

- <sup>1</sup> The constructors in this subclause shall not participate in overload resolution unless `uses_allocator_v<key_container_type, Alloc>` is true and `uses_allocator_v<mapped_container_type, Alloc>` is true.

```

template<class Alloc>
 constexpr flat_multimap(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);

```

```
template<class Alloc>
constexpr flat_multimap(const key_container_type& key_cont,
 const mapped_container_type& mapped_cont,
 const key_compare& comp, const Alloc& a);
```

2     *Effects:* Equivalent to `flat_multimap(key_cont, mapped_cont)` and `flat_multimap(key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

3     *Complexity:* Same as `flat_multimap(key_cont, mapped_cont)` and `flat_multimap(key_cont, mapped_cont, comp)`, respectively.

```
template<class Alloc>
constexpr flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const Alloc& a);
```

```
template<class Alloc>
constexpr flat_multimap(sorted_equivalent_t, const key_container_type& key_cont,
 const mapped_container_type& mapped_cont, const key_compare& comp,
 const Alloc& a);
```

4     *Effects:* Equivalent to `flat_multimap(sorted_equivalent, key_cont, mapped_cont)` and `flat_multimap(sorted_equivalent, key_cont, mapped_cont, comp)`, respectively, except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

5     *Complexity:* Linear.

```
template<class Alloc>
constexpr explicit flat_multimap(const Alloc& a);
template<class Alloc>
constexpr flat_multimap(const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_multimap(const flat_multimap&, const Alloc& a);
template<class Alloc>
constexpr flat_multimap(flat_multimap&&, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_multimap(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_multimap(InputIterator first, InputIterator last, const key_compare& comp,
 const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_multimap(sorted_equivalent_t, InputIterator first, InputIterator last,
 const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_multimap(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_multimap(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_multimap(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_multimap(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_multimap(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
constexpr flat_multimap(sorted_equivalent_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_multimap(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);
```

6     *Effects:* Equivalent to the corresponding non-allocator constructors except that `c.keys` and `c.values` are constructed with uses-allocator construction (20.2.8.2).

## 23.6.9.5 Erasure

[flat.multimap.erase]

```
template<class Key, class T, class Compare, class KeyContainer, class MappedContainer,
 class Predicate>
constexpr typename flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>::size_type
erase_if(flat_multimap<Key, T, Compare, KeyContainer, MappedContainer>& c, Predicate pred);
```

1     *Preconditions:* Key and T meet the *Cpp17MoveAssignable* requirements.

2     *Effects:* Let  $E$  be `bool(pred(pair<const Key&, const T&>(e)))`. Erases all elements  $e$  in  $c$  for which  $E$  holds.

3     *Returns:* The number of elements erased.

4     *Complexity:* Exactly `c.size()` applications of the predicate.

5     *Remarks:* Stable (16.4.6.8). If an invocation of `erase_if` exits via an exception,  $c$  is in a valid but unspecified state (3.68).

[Note 1:  $c$  still meets its invariants, but can be empty. — end note]

## 23.6.10 Header &lt;flat\_set&gt; synopsis

[flat.set.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 23.6.11, class template flat_set
 template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
 class flat_set;

 struct sorted_unique_t { explicit sorted_unique_t() = default; };
 inline constexpr sorted_unique_t sorted_unique{};

 template<class Key, class Compare, class KeyContainer, class Allocator>
 struct uses_allocator<flat_set<Key, Compare, KeyContainer>, Allocator>;

 // 23.6.11.6, erasure for flat_set
 template<class Key, class Compare, class KeyContainer, class Predicate>
 constexpr typename flat_set<Key, Compare, KeyContainer>::size_type
 erase_if(flat_set<Key, Compare, KeyContainer>& c, Predicate pred);

 // 23.6.12, class template flat_multiset
 template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
 class flat_multiset;

 struct sorted_equivalent_t { explicit sorted_equivalent_t() = default; };
 inline constexpr sorted_equivalent_t sorted_equivalent{};

 template<class Key, class Compare, class KeyContainer, class Allocator>
 struct uses_allocator<flat_multiset<Key, Compare, KeyContainer>, Allocator>;

 // 23.6.12.6, erasure for flat_multiset
 template<class Key, class Compare, class KeyContainer, class Predicate>
 constexpr typename flat_multiset<Key, Compare, KeyContainer>::size_type
 erase_if(flat_multiset<Key, Compare, KeyContainer>& c, Predicate pred);
}
```

## 23.6.11 Class template flat\_set

[flat.set]

## 23.6.11.1 Overview

[flat.set.overview]

- 1 A `flat_set` is a container adaptor that provides an associative container interface that supports unique keys (i.e., contains at most one of each key value) and provides for fast retrieval of the keys themselves. `flat_set` supports iterators that model the `random_access_iterator` concept (24.3.4.13).
- 2 A `flat_set` meets all of the requirements for a container (23.2.2.2) and for a reversible container (23.2.2.3), plus the optional container requirements (23.2.2.4). `flat_set` meets the requirements of an associative container (23.2.7), except that:

- (2.1) — it does not meet the requirements related to node handles (23.2.5.1),
- (2.2) — it does not meet the requirements related to iterator invalidation, and
- (2.3) — the time complexity of the operations that insert or erase a single element from the set is linear, including the ones that take an insertion position iterator.

[Note 1: A `flat_set` does not meet the additional requirements of an allocator-aware container, as described in 23.2.2.5. — end note]

- 3 A `flat_set` also provides most operations described in 23.2.7 for unique keys. This means that a `flat_set` supports the `a_uniq` operations in 23.2.7 but not the `a_eq` operations. For a `flat_set<Key>`, both the `key_type` and `value_type` are `Key`.
  - 4 Descriptions are provided here only for operations on `flat_set` that are not described in one of those sets of requirements or for operations where there is additional semantic information.
  - 5 A `flat_set` maintains the invariant that the keys are sorted with respect to the comparison object.
  - 6 If any member function in 23.6.11.2 exits via an exception, the invariant of the object argument is restored. For the move constructor and move assignment operator, the invariants of both arguments are restored.
- [Note 2: This can result in the `flat_set`'s being emptied. — end note]
- 7 Any sequence container (23.2.4) supporting *Cpp17RandomAccessIterator* can be used to instantiate `flat_set`. In particular, `vector` (23.3.13) and `deque` (23.3.5) can be used.
- [Note 3: `vector<bool>` is not a sequence container. — end note]
- 8 The program is ill-formed if `Key` is not the same type as `KeyContainer::value_type`.
  - 9 The effect of calling a member function that takes a `sorted_unique_t` argument with a range that is not sorted with respect to `key_comp()`, or that contains equal elements, is undefined.
  - 10 The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

### 23.6.11.2 Definition

[flat.set.defn]

```
namespace std {
 template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
 class flat_set {
 public:
 // types
 using key_type = Key;
 using value_type = Key;
 using key_compare = Compare;
 using value_compare = Compare;
 using reference = value_type&;
 using const_reference = const value_type&;
 using size_type = KeyContainer::size_type;
 using difference_type = KeyContainer::difference_type;
 using iterator = implementation-defined; // see 23.2
 using const_iterator = implementation-defined; // see 23.2
 using reverse_iterator = std::reverse_iterator<iterator>;
 using const_reverse_iterator = std::reverse_iterator<const_iterator>;
 using container_type = KeyContainer;

 // 23.6.11.3, constructors
 constexpr flat_set() : flat_set(key_compare()) { }

 constexpr flat_set(const flat_set&);
 constexpr flat_set(flat_set&&);
 constexpr flat_set& operator=(const flat_set&);
 constexpr flat_set& operator=(flat_set&&);

 constexpr explicit flat_set(const key_compare& comp)
 : c(), compare(comp) { }

 constexpr explicit flat_set(container_type cont, const key_compare& comp = key_compare());
```

```

constexpr flat_set(sorted_unique_t, container_type cont,
 const key_compare& comp = key_compare())
 : c(std::move(cont)), compare(comp) { }

template<class InputIterator>
constexpr flat_set(InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp)
 { insert(first, last); }

template<class InputIterator>
constexpr flat_set(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(first, last), compare(comp) { }

template<container_compatible_range<value_type> R>
constexpr flat_set(from_range_t, R&& rg)
 : flat_set(from_range, std::forward<R>(rg), key_compare()) { }
template<container_compatible_range<value_type> R>
constexpr flat_set(from_range_t, R&& rg, const key_compare& comp)
 : flat_set(comp)
 { insert_range(std::forward<R>(rg)); }

constexpr flat_set(initializer_list<value_type> il, const key_compare& comp = key_compare())
 : flat_set(il.begin(), il.end(), comp) { }

constexpr flat_set(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_set(sorted_unique, il.begin(), il.end(), comp) { }

// 23.6.11.4, constructors with allocators

template<class Alloc>
constexpr explicit flat_set(const Alloc& a);
template<class Alloc>
constexpr flat_set(const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(const container_type& cont, const Alloc& a);
template<class Alloc>
constexpr flat_set(const container_type& cont, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(sorted_unique_t, const container_type& cont, const Alloc& a);
template<class Alloc>
constexpr flat_set(sorted_unique_t, const container_type& cont,
 const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(const flat_set&, const Alloc& a);
template<class Alloc>
constexpr flat_set(flat_set&&, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(sorted_unique_t, InputIterator first, InputIterator last,
 const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container_compatible_range<value_type> R, class Alloc>
constexpr flat_set(from_range_t, R&& rg, const Alloc& a);
template<container_compatible_range<value_type> R, class Alloc>
constexpr flat_set(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);

```



```

template<class Alloc>
 constexpr flat_set(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_set(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_set(sorted_unique_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_set(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

constexpr flat_set& operator=(initializer_list<value_type>);

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

// 23.6.11.5, modifiers
template<class... Args> constexpr pair<iterator, bool> emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);

constexpr pair<iterator, bool> insert(const value_type& x)
 { return emplace(x); }
constexpr pair<iterator, bool> insert(value_type&& x)
 { return emplace(std::move(x)); }
template<class K> constexpr pair<iterator, bool> insert(K&& x);
constexpr iterator insert(const_iterator position, const value_type& x)
 { return emplace_hint(position, x); }
constexpr iterator insert(const_iterator position, value_type&& x)
 { return emplace_hint(position, std::move(x)); }
template<class K> constexpr iterator insert(const_iterator hint, K&& x);

template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<class InputIterator>
 constexpr void insert(sorted_unique_t, InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(sorted_unique_t, R&& rg);

constexpr void insert(initializer_list<value_type> il)
 { insert(il.begin(), il.end()); }
constexpr void insert(sorted_unique_t, initializer_list<value_type> il)
 { insert(sorted_unique, il.begin(), il.end()); }

```

```

constexpr container_type extract() &&;
constexpr void replace(container_type&&);

constexpr iterator erase(iterator position) requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void swap(flat_set& y) noexcept(see below);
constexpr void clear() noexcept;

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// set operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);
template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;

friend constexpr bool operator==(const flat_set& x, const flat_set& y);

friend constexpr synth-three-way-result<value_type>
 operator<=>(const flat_set& x, const flat_set& y);

friend constexpr void swap(flat_set& x, flat_set& y) noexcept(noexcept(x.swap(y)))
 { x.swap(y); }

private:
 container_type c; // exposition only
 key_compare compare; // exposition only
};

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
 flat_set(KeyContainer, Compare = Compare())
 -> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
 flat_set(KeyContainer, Allocator)
 -> flat_set<typename KeyContainer::value_type,

```

```

 less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
 flat_set(KeyContainer, Compare, Allocator)
 -> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
 flat_set(sorted_unique_t, KeyContainer, Compare = Compare())
 -> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
 flat_set(sorted_unique_t, KeyContainer, Allocator)
 -> flat_set<typename KeyContainer::value_type,
 less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
 flat_set(sorted_unique_t, KeyContainer, Compare, Allocator)
 -> flat_set<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class InputIterator, class Compare = less<iter-value-type<InputIterator>>>
 flat_set(InputIterator, InputIterator, Compare = Compare())
 -> flat_set<iter-value-type<InputIterator>, Compare>;

template<class InputIterator, class Compare = less<iter-value-type<InputIterator>>>
 flat_set(sorted_unique_t, InputIterator, InputIterator, Compare = Compare())
 -> flat_set<iter-value-type<InputIterator>, Compare>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
 flat_set(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> flat_set<ranges::range_value_t<R>, Compare,
 vector<ranges::range_value_t<R>,
 alloc-rebind<Allocator, ranges::range_value_t<R>>>>>;

template<ranges::input_range R, class Allocator>
 flat_set(from_range_t, R&&, Allocator)
 -> flat_set<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
 vector<ranges::range_value_t<R>,
 alloc-rebind<Allocator, ranges::range_value_t<R>>>>>;

template<class Key, class Compare = less<Key>>
 flat_set(initializer_list<Key>, Compare = Compare())
 -> flat_set<Key, Compare>;

template<class Key, class Compare = less<Key>>
 flat_set(sorted_unique_t, initializer_list<Key>, Compare = Compare())
 -> flat_set<Key, Compare>;

template<class Key, class Compare, class KeyContainer, class Allocator>
 struct uses_allocator<flat_set<Key, Compare, KeyContainer>, Allocator>
 : bool_constant<uses_allocator_v<KeyContainer, Allocator>> { };
}

```

### 23.6.11.3 Constructors

[flat.set.cons]

constexpr explicit flat\_set(container\_type cont, const key\_compare& comp = key\_compare());

- <sup>1</sup> *Effects:* Initializes *c* with `std::move(cont)` and *compare* with *comp*, sorts the range `[begin(), end())` with respect to *compare*, and finally erases all but the first element from each group of consecutive equivalent elements.
- <sup>2</sup> *Complexity:* Linear in *N* if *cont* is already sorted with respect to *compare* and otherwise  $N \log N$ , where *N* is the value of *cont.size()* before this call.

### 23.6.11.4 Constructors with allocators

[flat.set.cons.alloc]

- <sup>1</sup> The constructors in this subclause shall not participate in overload resolution unless `uses_allocator_v<container_type, Alloc>` is true.

```

template<class Alloc>
constexpr flat_set(const container_type& cont, const Alloc& a);
template<class Alloc>
constexpr flat_set(const container_type& cont, const key_compare& comp, const Alloc& a);

```

2     *Effects:* Equivalent to `flat_set(cont)` and `flat_set(cont, comp)`, respectively, except that `c` is constructed with uses-allocator construction (20.2.8.2).

3     *Complexity:* Same as `flat_set(cont)` and `flat_set(cont, comp)`, respectively.

```

template<class Alloc>
constexpr flat_set(sorted_unique_t, const container_type& cont, const Alloc& a);
template<class Alloc>
constexpr flat_set(sorted_unique_t, const container_type& cont,
 const key_compare& comp, const Alloc& a);

```

4     *Effects:* Equivalent to `flat_set(sorted_unique, cont)` and `flat_set(sorted_unique, cont, comp)`, respectively, except that `c` is constructed with uses-allocator construction (20.2.8.2).

5     *Complexity:* Linear.

```

template<class Alloc>
constexpr explicit flat_set(const Alloc& a);
template<class Alloc>
constexpr flat_set(const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(const flat_set&, const Alloc& a);
template<class Alloc>
constexpr flat_set(flat_set&&, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(InputIterator first, InputIterator last, const key_compare& comp,
 const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(sorted_unique_t, InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
constexpr flat_set(sorted_unique_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_set(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
constexpr flat_set(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_set(initializer_list<value_type> il, const key_compare& comp, const Alloc& a);
template<class Alloc>
constexpr flat_set(sorted_unique_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
constexpr flat_set(sorted_unique_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

```

6     *Effects:* Equivalent to the corresponding non-allocator constructors except that `c` is constructed with uses-allocator construction (20.2.8.2).

### 23.6.11.5 Modifiers

[flat.set.modifiers]

```

template<class K> constexpr pair<iterator, bool> insert(K&& x);
template<class K> constexpr iterator insert(const_iterator hint, K&& x);

```

1     *Constraints:* The *qualified-id* `Compare::is_transparent` is valid and denotes a type. `is_constructible_v<value_type, K>` is true.

2     *Preconditions:* The conversion from `x` into `value_type` constructs an object `u`, for which `find(x) == find(u)` is true.

3 *Effects:* If the set already contains an element equivalent to `x`, `*this` and `x` are unchanged. Otherwise, inserts a new element as if by `emplace(std::forward<K>(x))`.

4 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the element whose key is equivalent to `x`.

```
template<class InputIterator>
constexpr void insert(InputIterator first, InputIterator last);
```

5 *Effects:* Adds elements to `c` as if by:

```
c.insert(c.end(), first, last);
```

Then, sorts the range of newly inserted elements with respect to `compare`; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases all but the first element from each group of consecutive equivalent elements.

6 *Complexity:*  $N + M \log M$ , where  $N$  is `size()` before the operation and  $M$  is `distance(first, last)`.

7 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```
template<class InputIterator>
constexpr void insert(sorted_unique_t, InputIterator first, InputIterator last);
```

8 *Effects:* Equivalent to `insert(first, last)`.

9 *Complexity:* Linear.

```
template<container-compatible-range<value_type> R>
constexpr void insert_range(R&& rg);
```

10 *Effects:* Adds elements to `c` as if by:

```
ranges::for_each(rg, [&](value_type e) {
 c.insert(c.end(), std::move(e));
});
```

Then, sorts the range of newly inserted elements with respect to `compare`; merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range; and finally erases all but the first element from each group of consecutive equivalent elements.

11 *Complexity:*  $N + M \log M$ , where  $N$  is `size()` before the operation and  $M$  is `ranges::distance(rg)`.

12 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```
template<container-compatible-range<value_type> R>
constexpr void insert_range(sorted_unique_t, R&& rg);
```

13 *Effects:* Equivalent to `insert_range(rg)`.

14 *Complexity:* Linear in  $N$ , where  $N$  is `size()` after the operation.

```
constexpr void swap(flat_set& y)
noexcept(is_nothrow_swappable_v<container_type> &&
is_nothrow_swappable_v<key_compare>);
```

15 *Effects:* Equivalent to:

```
ranges::swap(compare, y.compare);
ranges::swap(c, y.c);
```

```
constexpr container_type extract() &&;
```

16 *Postconditions:* `*this` is emptied, even if the function exits via an exception.

17 *Returns:* `std::move(c)`.

```
constexpr void replace(container_type&& cont);
```

18 *Preconditions:* The elements of `cont` are sorted with respect to `compare`, and `cont` contains no equal elements.

19 *Effects:* Equivalent to: `c = std::move(cont)`;

**23.6.11.6 Erasure****[flat.set.erase]**

```
template<class Key, class Compare, class KeyContainer, class Predicate>
constexpr typename flat_set<Key, Compare, KeyContainer>::size_type
erase_if(flat_set<Key, Compare, KeyContainer>& c, Predicate pred);
```

- 1     *Preconditions:* Key meets the *Cpp17MoveAssignable* requirements.
- 2     *Effects:* Let  $E$  be `bool(pred(as_const(e)))`. Erases all elements  $e$  in  $c$  for which  $E$  holds.
- 3     *Returns:* The number of elements erased.
- 4     *Complexity:* Exactly  $c.size()$  applications of the predicate.
- 5     *Remarks:* Stable (16.4.6.8). If an invocation of `erase_if` exits via an exception,  $c$  is in a valid but unspecified state (3.68).
- [Note 1:  $c$  still meets its invariants, but can be empty. — end note]

**23.6.12 Class template flat\_multiset****[flat.multiset]****23.6.12.1 Overview****[flat.multiset.overview]**

- 1 A `flat_multiset` is a container adaptor that provides an associative container interface that supports equivalent keys (i.e., possibly containing multiple copies of the same key value) and provides for fast retrieval of the keys themselves. `flat_multiset` supports iterators that model the `random_access_iterator` concept (24.3.4.13).
- 2 A `flat_multiset` meets all of the requirements for a container (23.2.2.2) and for a reversible container (23.2.2.3), plus the optional container requirements (23.2.2.4). `flat_multiset` meets the requirements of an associative container (23.2.7), except that:
- (2.1) — it does not meet the requirements related to node handles (23.2.5.1),
- (2.2) — it does not meet the requirements related to iterator invalidation, and
- (2.3) — the time complexity of the operations that insert or erase a single element from the set is linear, including the ones that take an insertion position iterator.
- [Note 1: A `flat_multiset` does not meet the additional requirements of an allocator-aware container, as described in 23.2.2.5. — end note]
- 3 A `flat_multiset` also provides most operations described in 23.2.7 for equal keys. This means that a `flat_multiset` supports the `a_eq` operations in 23.2.7 but not the `a_uniq` operations. For a `flat_multiset<Key>`, both the `key_type` and `value_type` are `Key`.
- 4 Descriptions are provided here only for operations on `flat_multiset` that are not described in one of the general sections or for operations where there is additional semantic information.
- 5 A `flat_multiset` maintains the invariant that the keys are sorted with respect to the comparison object.
- 6 If any member function in 23.6.12.2 exits via an exception, the invariant of the object argument is restored. For the move constructor and move assignment operator, the invariants of both arguments are restored.
- [Note 2: This can result in the `flat_multiset`'s being emptied. — end note]
- 7 Any sequence container (23.2.4) supporting *Cpp17RandomAccessIterator* can be used to instantiate `flat_multiset`. In particular, `vector` (23.3.13) and `deque` (23.3.5) can be used.
- [Note 3: `vector<bool>` is not a sequence container. — end note]
- 8 The program is ill-formed if `Key` is not the same type as `KeyContainer::value_type`.
- 9 The effect of calling a member function that takes a `sorted_equivalent_t` argument with a range that is not sorted with respect to `key_comp()` is undefined.
- 10 The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

**23.6.12.2 Definition****[flat.multiset.defn]**

```
namespace std {
 template<class Key, class Compare = less<Key>, class KeyContainer = vector<Key>>
 class flat_multiset {
 public:
 // types
 using key_type = Key;
```

```

using value_type = Key;
using key_compare = Compare;
using value_compare = Compare;
using reference = value_type&;
using const_reference = const value_type&;
using size_type = KeyContainer::size_type;
using difference_type = KeyContainer::difference_type;
using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
using container_type = KeyContainer;

// 23.6.12.3, constructors
constexpr flat_multiset() : flat_multiset(key_compare()) { }

constexpr flat_multiset(const flat_multiset&);
constexpr flat_multiset(flat_multiset&&);
constexpr flat_multiset& operator=(const flat_multiset&);
constexpr flat_multiset& operator=(flat_multiset&&);

constexpr explicit flat_multiset(const key_compare& comp)
 : c(), compare(comp) { }

constexpr explicit flat_multiset(container_type cont,
 const key_compare& comp = key_compare());

constexpr flat_multiset(sorted_equivalent_t, container_type cont,
 const key_compare& comp = key_compare())
 : c(std::move(cont)), compare(comp) { }

template<class InputIterator>
constexpr flat_multiset(InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(), compare(comp)
 { insert(first, last); }

template<class InputIterator>
constexpr flat_multiset(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp = key_compare())
 : c(first, last), compare(comp) { }

template<container-compatible-range<value_type> R>
constexpr flat_multiset(from_range_t, R&& rg)
 : flat_multiset(from_range, std::forward<R>(rg), key_compare()) { }
template<container-compatible-range<value_type> R>
constexpr flat_multiset(from_range_t, R&& rg, const key_compare& comp)
 : flat_multiset(comp)
 { insert_range(std::forward<R>(rg)); }

constexpr flat_multiset(initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_multiset(il.begin(), il.end(), comp) { }

constexpr flat_multiset(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp = key_compare())
 : flat_multiset(sorted_equivalent, il.begin(), il.end(), comp) { }

// 23.6.12.4, constructors with allocators

template<class Alloc>
constexpr explicit flat_multiset(const Alloc& a);
template<class Alloc>
constexpr flat_multiset(const key_compare& comp, const Alloc& a);

```

```

template<class Alloc>
 constexpr flat_multiset(const container_type& cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(const container_type& cont, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, const container_type& cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, const container_type& cont,
 const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(const flat_multiset&, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(flat_multiset&&, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, InputIterator first, InputIterator last,
 const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multiset(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multiset(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, initializer_list<value_type> il,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

constexpr flat_multiset& operator=(initializer_list<value_type>);

// iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
constexpr size_type max_size() const noexcept;

```



```

// 23.6.12.5, modifiers
template<class... Args> constexpr iterator emplace(Args&&... args);
template<class... Args>
 constexpr iterator emplace_hint(const_iterator position, Args&&... args);

constexpr iterator insert(const value_type& x)
 { return emplace(x); }
constexpr iterator insert(value_type&& x)
 { return emplace(std::move(x)); }
constexpr iterator insert(const_iterator position, const value_type& x)
 { return emplace_hint(position, x); }
constexpr iterator insert(const_iterator position, value_type&& x)
 { return emplace_hint(position, std::move(x)); }

template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);
template<class InputIterator>
 constexpr void insert(sorted_equivalent_t, InputIterator first, InputIterator last);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);
template<container-compatible-range<value_type> R>
 constexpr void insert_range(sorted_equivalent_t, R&& rg);

constexpr void insert(initializer_list<value_type> il)
 { insert(il.begin(), il.end()); }
constexpr void insert(sorted_equivalent_t, initializer_list<value_type> il)
 { insert(sorted_equivalent, il.begin(), il.end()); }

constexpr container_type extract() &&;
constexpr void replace(container_type&&);

constexpr iterator erase(iterator position) requires (!same_as<iterator, const_iterator>);
constexpr iterator erase(const_iterator position);
constexpr size_type erase(const key_type& x);
template<class K> constexpr size_type erase(K&& x);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void swap(flat_multiset& y) noexcept(see below);
constexpr void clear() noexcept;

// observers
constexpr key_compare key_comp() const;
constexpr value_compare value_comp() const;

// set operations
constexpr iterator find(const key_type& x);
constexpr const_iterator find(const key_type& x) const;
template<class K> constexpr iterator find(const K& x);
template<class K> constexpr const_iterator find(const K& x) const;

constexpr size_type count(const key_type& x) const;
template<class K> constexpr size_type count(const K& x) const;

constexpr bool contains(const key_type& x) const;
template<class K> constexpr bool contains(const K& x) const;

constexpr iterator lower_bound(const key_type& x);
constexpr const_iterator lower_bound(const key_type& x) const;
template<class K> constexpr iterator lower_bound(const K& x);
template<class K> constexpr const_iterator lower_bound(const K& x) const;

constexpr iterator upper_bound(const key_type& x);
constexpr const_iterator upper_bound(const key_type& x) const;
template<class K> constexpr iterator upper_bound(const K& x);

```

```

template<class K> constexpr const_iterator upper_bound(const K& x) const;

constexpr pair<iterator, iterator> equal_range(const key_type& x);
constexpr pair<const_iterator, const_iterator> equal_range(const key_type& x) const;
template<class K>
 constexpr pair<iterator, iterator> equal_range(const K& x);
template<class K>
 constexpr pair<const_iterator, const_iterator> equal_range(const K& x) const;

friend constexpr bool operator==(const flat_multiset& x, const flat_multiset& y);

friend constexpr synth-three-way-result<value_type>
 operator<=>(const flat_multiset& x, const flat_multiset& y);

friend constexpr void swap(flat_multiset& x, flat_multiset& y)
 noexcept(noexcept(x.swap(y)))
 { x.swap(y); }

private:
 container_type c; // exposition only
 key_compare compare; // exposition only
};

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
 flat_multiset(KeyContainer, Compare = Compare())
 -> flat_multiset<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
 flat_multiset(KeyContainer, Allocator)
 -> flat_multiset<typename KeyContainer::value_type,
 less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
 flat_multiset(KeyContainer, Compare, Allocator)
 -> flat_multiset<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class KeyContainer, class Compare = less<typename KeyContainer::value_type>>
 flat_multiset(sorted_equivalent_t, KeyContainer, Compare = Compare())
 -> flat_multiset<typename KeyContainer::value_type, Compare, KeyContainer>;
template<class KeyContainer, class Allocator>
 flat_multiset(sorted_equivalent_t, KeyContainer, Allocator)
 -> flat_multiset<typename KeyContainer::value_type,
 less<typename KeyContainer::value_type>, KeyContainer>;
template<class KeyContainer, class Compare, class Allocator>
 flat_multiset(sorted_equivalent_t, KeyContainer, Compare, Allocator)
 -> flat_multiset<typename KeyContainer::value_type, Compare, KeyContainer>;

template<class InputIterator, class Compare = less<iter-value-type<InputIterator>>>
 flat_multiset(InputIterator, InputIterator, Compare = Compare())
 -> flat_multiset<iter-value-type<InputIterator>, Compare>;

template<class InputIterator, class Compare = less<iter-value-type<InputIterator>>>
 flat_multiset(sorted_equivalent_t, InputIterator, InputIterator, Compare = Compare())
 -> flat_multiset<iter-value-type<InputIterator>, Compare>;

template<ranges::input_range R, class Compare = less<ranges::range_value_t<R>>,
 class Allocator = allocator<ranges::range_value_t<R>>>
 flat_multiset(from_range_t, R&&, Compare = Compare(), Allocator = Allocator())
 -> flat_multiset<ranges::range_value_t<R>, Compare,
 vector<ranges::range_value_t<R>,
 alloc_rebind<Allocator, ranges::range_value_t<R>>>>>;

template<ranges::input_range R, class Allocator>
 flat_multiset(from_range_t, R&&, Allocator)
 -> flat_multiset<ranges::range_value_t<R>, less<ranges::range_value_t<R>>,
 vector<ranges::range_value_t<R>,

```

```

 alloc-rebind<Allocator, ranges::range_value_t<R>>>;

template<class Key, class Compare = less<Key>>
 flat_multiset(initializer_list<Key>, Compare = Compare())
 -> flat_multiset<Key, Compare>;

template<class Key, class Compare = less<Key>>
 flat_multiset(sorted_equivalent_t, initializer_list<Key>, Compare = Compare())
 -> flat_multiset<Key, Compare>;

template<class Key, class Compare, class KeyContainer, class Allocator>
 struct uses_allocator<flat_multiset<Key, Compare, KeyContainer>, Allocator>
 : bool_constant<uses_allocator_v<KeyContainer, Allocator>> { };
}

```

### 23.6.12.3 Constructors

[flat.multiset.cons]

```
constexpr explicit flat_multiset(container_type cont, const key_compare& comp = key_compare());
```

- <sup>1</sup> *Effects:* Initializes *c* with `std::move(cont)` and *compare* with *comp*, and sorts the range `[begin(), end())` with respect to *compare*.
- <sup>2</sup> *Complexity:* Linear in *N* if *cont* is already sorted with respect to *compare* and otherwise  $N \log N$ , where *N* is the value of *cont.size()* before this call.

### 23.6.12.4 Constructors with allocators

[flat.multiset.cons.alloc]

- <sup>1</sup> The constructors in this subclause shall not participate in overload resolution unless `uses_allocator_v<container_type, Alloc>` is true.

```

template<class Alloc>
 constexpr flat_multiset(const container_type& cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(const container_type& cont, const key_compare& comp, const Alloc& a);

```

- <sup>2</sup> *Effects:* Equivalent to `flat_multiset(cont)` and `flat_multiset(cont, comp)`, respectively, except that *c* is constructed with uses-allocator construction (20.2.8.2).
- <sup>3</sup> *Complexity:* Same as `flat_multiset(cont)` and `flat_multiset(cont, comp)`, respectively.

```

template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, const container_type& cont, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, const container_type& cont,
 const key_compare& comp, const Alloc& a);

```

- <sup>4</sup> *Effects:* Equivalent to `flat_multiset(sorted_equivalent, cont)` and `flat_multiset(sorted_equivalent, cont, comp)`, respectively, except that *c* is constructed with uses-allocator construction (20.2.8.2).
- <sup>5</sup> *Complexity:* Linear.

```

template<class Alloc>
 constexpr explicit flat_multiset(const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(const flat_multiset&, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(flat_multiset&&, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(InputIterator first, InputIterator last, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<class InputIterator, class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, InputIterator first, InputIterator last,
 const Alloc& a);

```

```

template<class InputIterator, class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, InputIterator first, InputIterator last,
 const key_compare& comp, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multiset(from_range_t, R&& rg, const Alloc& a);
template<container-compatible-range<value_type> R, class Alloc>
 constexpr flat_multiset(from_range_t, R&& rg, const key_compare& comp, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(initializer_list<value_type> il, const key_compare& comp,
 const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, initializer_list<value_type> il, const Alloc& a);
template<class Alloc>
 constexpr flat_multiset(sorted_equivalent_t, initializer_list<value_type> il,
 const key_compare& comp, const Alloc& a);

```

- 6 *Effects:* Equivalent to the corresponding non-allocator constructors except that *c* is constructed with uses-allocator construction (20.2.8.2).

### 23.6.12.5 Modifiers

[flat.multiset.modifiers]

```
template<class... Args> constexpr iterator emplace(Args&&... args);
```

- 1 *Constraints:* `is_constructible_v<value_type, Args...>` is true.
- 2 *Effects:* First, initializes an object *t* of type `value_type` with `std::forward<Args>(args)...`, then inserts *t* as if by:

```

 auto it = ranges::upper_bound(c, t, compare);
 c.insert(it, std::move(t));

```

- 3 *Returns:* An iterator that points to the inserted element.

```

template<class InputIterator>
 constexpr void insert(InputIterator first, InputIterator last);

```

- 4 *Effects:* Adds elements to *c* as if by:

```
c.insert(c.end(), first, last);
```

Then, sorts the range of newly inserted elements with respect to *compare*, and merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range.

- 5 *Complexity:*  $N + M \log M$ , where *N* is `size()` before the operation and *M* is `distance(first, last)`.

- 6 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```

template<class InputIterator>
 constexpr void insert(sorted_equivalent_t, InputIterator first, InputIterator last);

```

- 7 *Effects:* Equivalent to `insert(first, last)`.

- 8 *Complexity:* Linear in *N*, where *N* is `size()` after the operation.

```

template<container-compatible-range<value_type> R>
 constexpr void insert_range(R&& rg);

```

- 9 *Effects:* Adds elements to *c* as if by:

```

 ranges::for_each(rg, [&](value_type e) {
 c.insert(c.end(), std::move(e));
 });

```

Then, sorts the range of newly inserted elements with respect to *compare*, and merges the resulting sorted range and the sorted range of pre-existing elements into a single sorted range.

- 10 *Complexity:*  $N + M \log M$ , where *N* is `size()` before the operation and *M* is `ranges::distance(rg)`.

- 11 *Remarks:* Since this operation performs an in-place merge, it may allocate memory.

```

template<container-compatible-range<value_type> R>
constexpr void insert_range(sorted_equivalent_t, R&& rg);
12 Effects: Equivalent to insert_range(rg).
13 Complexity: Linear in N , where N is size() after the operation.

constexpr void swap(flat_multiset& y)
 noexcept(is_nothrow_swappable_v<container_type> &&
 is_nothrow_swappable_v<key_compare>);
14 Effects: Equivalent to:
 ranges::swap(compare, y.compare);
 ranges::swap(c, y.c);

constexpr container_type extract() &&;
15 Postconditions: *this is emptied, even if the function exits via an exception.
16 Returns: std::move(c).

constexpr void replace(container_type&& cont);
17 Preconditions: The elements of cont are sorted with respect to compare.
18 Effects: Equivalent to: c = std::move(cont);

```

### 23.6.12.6 Erasure

[flat.multiset.erase]

```

template<class Key, class Compare, class KeyContainer, class Predicate>
constexpr typename flat_multiset<Key, Compare, KeyContainer>::size_type
erase_if(flat_multiset<Key, Compare, KeyContainer>& c, Predicate pred);
1 Preconditions: Key meets the Cpp17MoveAssignable requirements.
2 Effects: Let E be bool(pred(as_const(e))). Erases all elements e in c for which E holds.
3 Returns: The number of elements erased.
4 Complexity: Exactly $c.size()$ applications of the predicate.
5 Remarks: Stable (16.4.6.8). If an invocation of erase_if exits via an exception, c is in a valid but
 unspecified state (3.68).
 [Note 1: c still meets its invariants, but can be empty. — end note]

```

### 23.6.13 Container adaptors formatting

[container.adaptors.format]

- 1 For each of queue, priority\_queue, and stack, the library provides the following constexpr-enabled formatter specialization where *adaptor-type* is the name of the template:

```

namespace std {
 template<class charT, class T, formattable<charT> Container, class... U>
 struct formatter<adaptor-type<T, Container, U...>, charT> {
 private:
 using maybe-const-container = // exposition only
 fmt-maybe-const<Container, charT>;
 using maybe-const-adaptor = // exposition only
 maybe-const<is_const_v<maybe-const-container>,
 adaptor-type<T, Container, U...>>;
 formatter<ranges::ref_view<maybe-const-container>, charT> underlying_; // exposition only

 public:
 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<class FormatContext>
 constexpr typename FormatContext::iterator
 format(maybe-const-adaptor& r, FormatContext& ctx) const;
 };
}

```

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

<sup>2</sup> *Effects:* Equivalent to: return *underlying\_.parse*(ctx);

```
template<class FormatContext>
constexpr typename FormatContext::iterator
format(maybe-const-adaptor& r, FormatContext& ctx) const;
```

<sup>3</sup> *Effects:* Equivalent to: return *underlying\_.format*(r.c, ctx);

## 23.7 Views

[views]

### 23.7.1 General

[views.general]

- <sup>1</sup> The header `<span>` (23.7.2.1) defines the view `span`. The header `<mdspan>` (23.7.3.2) defines the class template `mdspan` and other facilities for interacting with these multidimensional views.
- <sup>2</sup> In addition to being available via inclusion of the `<span>` (23.7.2.1) header, `dynamic_extent` is available when the header `<mdspan>` (23.7.3.2) is included.

### 23.7.2 Contiguous access

[views.contiguous]

#### 23.7.2.1 Header `<span>` synopsis

[span.syn]

```
// mostly freestanding
namespace std {
 // constants
 inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();

 template<class T>
 concept integral-constant-like = // exposition only
 is_integral_v<remove_cvref_t<decltype(T::value)>> &&
 !is_same_v<bool, remove_cvref_t<decltype(T::value)>> &&
 constexpr-wrapper-like<T>;

 template<class T>
 constexpr size_t maybe-static-ext = dynamic_extent; // exposition only
 template<integral-constant-like T>
 constexpr size_t maybe-static-ext<T> = {T::value};

 // 23.7.2.2, class template span
 template<class ElementType, size_t Extent = dynamic_extent>
 class span; // partially freestanding

 template<class ElementType, size_t Extent>
 constexpr bool ranges::enable_view<span<ElementType, Extent>> = true;
 template<class ElementType, size_t Extent>
 constexpr bool ranges::enable_borrowed_range<span<ElementType, Extent>> = true;

 // 23.7.2.3, views of object representation
 template<class ElementType, size_t Extent>
 span<const byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
 as_bytes(span<ElementType, Extent> s) noexcept;

 template<class ElementType, size_t Extent>
 span<byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
 as_writable_bytes(span<ElementType, Extent> s) noexcept;
}
```

#### 23.7.2.2 Class template `span`

[views.span]

##### 23.7.2.2.1 Overview

[span.overview]

- <sup>1</sup> A `span` is a view over a contiguous sequence of objects, the storage of which is owned by some other object.
- <sup>2</sup> All member functions of `span` have constant time complexity.

```

namespace std {
 template<class ElementType, size_t Extent = dynamic_extent>
 class span {
 public:
 // constants and types
 using element_type = ElementType;
 using value_type = remove_cv_t<ElementType>;
 using size_type = size_t;
 using difference_type = ptrdiff_t;
 using pointer = element_type*;
 using const_pointer = const element_type*;
 using reference = element_type&;
 using const_reference = const element_type&;
 using iterator = implementation-defined; // see 23.7.2.2.7
 using const_iterator = std::const_iterator<iterator>;
 using reverse_iterator = std::reverse_iterator<iterator>;
 using const_reverse_iterator = std::const_iterator<reverse_iterator>;
 static constexpr size_type extent = Extent;

 // 23.7.2.2.2, constructors, copy, and assignment
 constexpr span() noexcept;
 template<class It>
 constexpr explicit(extent != dynamic_extent) span(It first, size_type count);
 template<class It, class End>
 constexpr explicit(extent != dynamic_extent) span(It first, End last);
 template<size_t N>
 constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
 template<class T, size_t N>
 constexpr span(array<T, N>& arr) noexcept;
 template<class T, size_t N>
 constexpr span(const array<T, N>& arr) noexcept;
 template<class R>
 constexpr explicit(extent != dynamic_extent) span(R&& r);
 constexpr span(const span& other) noexcept = default;
 template<class OtherElementType, size_t OtherExtent>
 constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;

 constexpr span& operator=(const span& other) noexcept = default;

 // 23.7.2.2.4, subviews
 template<size_t Count>
 constexpr span<element_type, Count> first() const;
 template<size_t Count>
 constexpr span<element_type, Count> last() const;
 template<size_t Offset, size_t Count = dynamic_extent>
 constexpr span<element_type, see below> subspan() const;

 constexpr span<element_type, dynamic_extent> first(size_type count) const;
 constexpr span<element_type, dynamic_extent> last(size_type count) const;
 constexpr span<element_type, dynamic_extent> subspan(
 size_type offset, size_type count = dynamic_extent) const;

 // 23.7.2.2.5, observers
 constexpr size_type size() const noexcept;
 constexpr size_type size_bytes() const noexcept;
 constexpr bool empty() const noexcept;

 // 23.7.2.2.6, element access
 constexpr reference operator[](size_type idx) const;
 constexpr reference at(size_type idx) const; // freestanding-deleted
 constexpr reference front() const;
 constexpr reference back() const;
 constexpr pointer data() const noexcept;
 };
}

```

```

// 23.7.2.2.7, iterator support
constexpr iterator begin() const noexcept;
constexpr iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept { return begin(); }
constexpr const_iterator cend() const noexcept { return end(); }
constexpr reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept { return rbegin(); }
constexpr const_reverse_iterator crend() const noexcept { return rend(); }

private:
 pointer data_; // exposition only
 size_type size_; // exposition only
};

template<class It, class EndOrSize>
 span(It, EndOrSize) -> span<remove_reference_t<iter_reference_t<It>>,
 maybe_static_ext<EndOrSize>>>;

template<class T, size_t N>
 span(T (&)[N]) -> span<T, N>;
template<class T, size_t N>
 span(array<T, N>&) -> span<T, N>;
template<class T, size_t N>
 span(const array<T, N>&) -> span<const T, N>;
template<class R>
 span(R&&) -> span<remove_reference_t<ranges::range_reference_t<R>>>>;
}

```

- 3 `span<ElementType, Extent>` is a trivially copyable type (6.9.1).
- 4 `ElementType` is required to be a complete object type that is not an abstract class type.
- 5 For a `span` `s`, any operation that invalidates a pointer in the range `[s.data(), s.data() + s.size())` invalidates pointers, iterators, and references to elements of `s`.

### 23.7.2.2.2 Constructors, copy, and assignment

[span.cons]

```
constexpr span() noexcept;
```

1 *Constraints:* `Extent == dynamic_extent || Extent == 0` is true.

2 *Postconditions:* `size() == 0 && data() == nullptr`.

```
template<class It>
 constexpr explicit(extent != dynamic_extent) span(It first, size_type count);
```

3 *Constraints:* Let `U` be `remove_reference_t<iter_reference_t<It>>`.

(3.1) — It satisfies `contiguous_iterator`.

(3.2) — `is_convertible_v<U(*)[], element_type(*)[]>` is true.

[Note 1: The intent is to allow only qualification conversions of the iterator reference type to `element_type`.  
— end note]

4 *Preconditions:*

(4.1) — `[first, first + count)` is a valid range.

(4.2) — It models `contiguous_iterator`.

5 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `count == extent` is true.

6 *Effects:* Initializes `data_` with `to_address(first)` and `size_` with `count`.

7 *Throws:* Nothing.

```
template<class It, class End>
 constexpr explicit(extent != dynamic_extent) span(It first, End last);
```

8 *Constraints:* Let `U` be `remove_reference_t<iter_reference_t<It>>`.

(8.1) — `is_convertible_v<U(*)[], element_type(*)[]>` is true.



[Note 2: The intent is to allow only qualification conversions of the iterator reference type to `element_type`.  
— end note]

(8.2) — It satisfies `contiguous_iterator`.

(8.3) — End satisfies `sized_sentinel_for<It>`.

(8.4) — `is_convertible_v<End, size_t>` is false.

9 *Preconditions:*

(9.1) — `[first, last)` is a valid range.

(9.2) — It models `contiguous_iterator`.

(9.3) — End models `sized_sentinel_for<It>`.

10 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `(last - first) == extent` is true.

11 *Effects:* Initializes `data_` with `to_address(first)` and `size_` with `last - first`.

12 *Throws:* When and what `last - first` throws.

```
template<size_t N> constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<class T, size_t N> constexpr span(array<T, N>& arr) noexcept;
template<class T, size_t N> constexpr span(const array<T, N>& arr) noexcept;
```

13 *Constraints:* Let U be `remove_pointer_t<decltype(std::data(arr))>`.

(13.1) — `extent == dynamic_extent || N == extent` is true, and

(13.2) — `is_convertible_v<U(*)[], element_type(*)[]>` is true.

[Note 3: The intent is to allow only qualification conversions of the array element type to `element_type`.  
— end note]

14 *Effects:* Constructs a `span` that is a view over the supplied array.

[Note 4: `type_identity_t` affects class template argument deduction. — end note]

15 *Postconditions:* `size() == N` && `data() == std::data(arr)` is true.

```
template<class R> constexpr explicit(extent != dynamic_extent) span(R&& r);
```

16 *Constraints:* Let U be `remove_reference_t<ranges::range_reference_t<R>>`.

(16.1) — R satisfies `ranges::contiguous_range` and `ranges::sized_range`.

(16.2) — Either R satisfies `ranges::borrowed_range` or `is_const_v<element_type>` is true.

(16.3) — `remove_cvref_t<R>` is not a specialization of `span`.

(16.4) — `remove_cvref_t<R>` is not a specialization of `array`.

(16.5) — `is_array_v<remove_cvref_t<R>>` is false.

(16.6) — `is_convertible_v<U(*)[], element_type(*)[]>` is true.

[Note 5: The intent is to allow only qualification conversions of the range reference type to `element_type`.  
— end note]

17 *Preconditions:*

(17.1) — R models `ranges::contiguous_range` and `ranges::sized_range`.

(17.2) — If `is_const_v<element_type>` is false, R models `ranges::borrowed_range`.

18 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `ranges::size(r) == extent` is true.

19 *Effects:* Initializes `data_` with `ranges::data(r)` and `size_` with `ranges::size(r)`.

20 *Throws:* What and when `ranges::data(r)` and `ranges::size(r)` throw.

```
constexpr span(const span& other) noexcept = default;
```

21 *Postconditions:* `other.size() == size()` && `other.data() == data()`.

```
template<class OtherElementType, size_t OtherExtent>
constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

22 *Constraints:*

(22.1) — `extent == dynamic_extent || OtherExtent == dynamic_extent || extent == OtherExtent` is true, and

(22.2) — `is_convertible_v<OtherElementType(*)[], element_type(*)[]>` is true.

[Note 6: The intent is to allow only qualification conversions of the `OtherElementType` to `element_type`.  
— end note]

23 *Hardened preconditions:* If `extent` is not equal to `dynamic_extent`, then `s.size() == extent` is true.

24 *Effects:* Constructs a `span` that is a view over the range `[s.data(), s.data() + s.size())`.

25 *Postconditions:* `size() == s.size() && data() == s.data()`.

26 *Remarks:* The expression inside `explicit` is equivalent to:

```
extent != dynamic_extent && OtherExtent == dynamic_extent
```

```
constexpr span& operator=(const span& other) noexcept = default;
```

27 *Postconditions:* `size() == other.size() && data() == other.data()`.

### 23.7.2.2.3 Deduction guides

[span.deduct]

```
template<class It, class EndOrSize>
span(It, EndOrSize) -> span<remove_reference_t<iter_reference_t<It>>,
 maybe-static-ext<EndOrSize>>;
```

1 *Constraints:* It satisfies `contiguous_iterator`.

```
template<class R>
span(R&&) -> span<remove_reference_t<ranges::range_reference_t<R>>>;
```

2 *Constraints:* R satisfies `ranges::contiguous_range`.

### 23.7.2.2.4 Subviews

[span.sub]

```
template<size_t Count> constexpr span<element_type, Count> first() const;
```

1 *Mandates:* `Count <= Extent` is true.

2 *Hardened preconditions:* `Count <= size()` is true.

3 *Effects:* Equivalent to: `return R(data(), Count);` where R is the return type.

```
template<size_t Count> constexpr span<element_type, Count> last() const;
```

4 *Mandates:* `Count <= Extent` is true.

5 *Hardened preconditions:* `Count <= size()` is true.

6 *Effects:* Equivalent to: `return R(data() + (size() - Count), Count);` where R is the return type.

```
template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;
```

7 *Mandates:*

```
Offset <= Extent && (Count == dynamic_extent || Count <= Extent - Offset)
is true.
```

8 *Hardened preconditions:*

```
Offset <= size() && (Count == dynamic_extent || Count <= size() - Offset)
is true.
```

9 *Effects:* Equivalent to:

```
return span<ElementType, see below>(
 data() + Offset, Count != dynamic_extent ? Count : size() - Offset);
```

10 *Remarks:* The second template argument of the returned `span` type is:

```

 Count != dynamic_extent ? Count
 : (Extent != dynamic_extent ? Extent - Offset
 : dynamic_extent)

```

```
constexpr span<element_type, dynamic_extent> first(size_type count) const;
```

11     *Hardened preconditions:* count <= size() is true.

12     *Effects:* Equivalent to: return R(data(), count); where R is the return type.

```
constexpr span<element_type, dynamic_extent> last(size_type count) const;
```

13     *Hardened preconditions:* count <= size() is true.

14     *Effects:* Equivalent to: return R(data() + (size() - count), count); where R is the return type.

```
constexpr span<element_type, dynamic_extent> subspan(
 size_type offset, size_type count = dynamic_extent) const;
```

15     *Hardened preconditions:*

```
 offset <= size() && (count == dynamic_extent || count <= size() - offset)
```

is true.

16     *Effects:* Equivalent to:

```
 return R(data() + offset, count == dynamic_extent ? size() - offset : count);
```

where R is the return type.

#### 23.7.2.2.5 Observers

[span.obs]

```
constexpr size_type size() const noexcept;
```

1     *Effects:* Equivalent to: return *size\_*;

```
constexpr size_type size_bytes() const noexcept;
```

2     *Effects:* Equivalent to: return size() \* sizeof(element\_type);

```
constexpr bool empty() const noexcept;
```

3     *Effects:* Equivalent to: return size() == 0;

#### 23.7.2.2.6 Element access

[span.elem]

```
constexpr reference operator[](size_type idx) const;
```

1     *Hardened preconditions:* idx < size() is true.

2     *Returns:* \*(data() + idx).

3     *Throws:* Nothing.

```
constexpr reference at(size_type idx) const;
```

4     *Returns:* \*(data() + idx).

5     *Throws:* out\_of\_range if idx >= size() is true.

```
constexpr reference front() const;
```

6     *Hardened preconditions:* empty() is false.

7     *Returns:* \*data().

8     *Throws:* Nothing.

```
constexpr reference back() const;
```

9     *Hardened preconditions:* empty() is false.

10     *Returns:* \*(data() + (size() - 1)).

11     *Throws:* Nothing.

```
constexpr pointer data() const noexcept;
```

12     *Returns:* *data\_*.

**23.7.2.2.7 Iterator support****[span.iterators]**

```
using iterator = implementation-defined;
```

1 The type models `contiguous_iterator` (24.3.4.14), meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7), and meets the requirements for `constexpr` iterators (24.3.1), whose value type is `value_type` and whose reference type is `reference`.

2 All requirements on container iterators (23.2.2.2) apply to `span::iterator` as well.

```
constexpr iterator begin() const noexcept;
```

3 *Returns:* An iterator referring to the first element in the span. If `empty()` is `true`, then it returns the same value as `end()`.

```
constexpr iterator end() const noexcept;
```

4 *Returns:* An iterator which is the past-the-end value.

```
constexpr reverse_iterator rbegin() const noexcept;
```

5 *Effects:* Equivalent to: `return reverse_iterator(end());`

```
constexpr reverse_iterator rend() const noexcept;
```

6 *Effects:* Equivalent to: `return reverse_iterator(begin());`

**23.7.2.3 Views of object representation****[span.objectrep]**

```
template<class ElementType, size_t Extent>
span<const byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
as_bytes(span<ElementType, Extent> s) noexcept;
```

1 *Constraints:* `is_volatile_v<ElementType>` is `false`.

2 *Effects:* Equivalent to: `return R{reinterpret_cast<const byte*>(s.data()), s.size_bytes()};` where `R` is the return type.

```
template<class ElementType, size_t Extent>
span<byte, Extent == dynamic_extent ? dynamic_extent : sizeof(ElementType) * Extent>
as_writable_bytes(span<ElementType, Extent> s) noexcept;
```

3 *Constraints:* `is_const_v<ElementType>` is `false` and `is_volatile_v<ElementType>` is `false`.

4 *Effects:* Equivalent to: `return R{reinterpret_cast<byte*>(s.data()), s.size_bytes()};` where `R` is the return type.

**23.7.3 Multidimensional access****[views.multidim]****23.7.3.1 Overview****[mdspan.overview]**

1 A *multidimensional index space* is a Cartesian product of integer intervals. Each interval can be represented by a half-open range  $[L_i, U_i)$ , where  $L_i$  and  $U_i$  are the lower and upper bounds of the  $i^{\text{th}}$  dimension. The *rank* of a multidimensional index space is the number of intervals it represents. The *size of a multidimensional index space* is the product of  $U_i - L_i$  for each dimension  $i$  if its rank is greater than 0, and 1 otherwise.

2 An integer  $r$  is a *rank index* of an index space  $S$  if  $r$  is in the range  $[0, \text{rank of } S)$ .

3 A pack of integers `idx` is a *multidimensional index* in a multidimensional index space  $S$  (or representation thereof) if both of the following are true:

- (3.1) — `sizeof...(idx)` is equal to the rank of  $S$ , and
- (3.2) — for every rank index  $i$  of  $S$ , the  $i^{\text{th}}$  value of `idx` is an integer in the interval  $[L_i, U_i)$  of  $S$ .

**23.7.3.2 Header `<mdspan>` synopsis****[mdspan.syn]**

```
// mostly freestanding
namespace std {
 // 23.7.3.3, class template extents
 template<class IndexType, size_t... Extents>
 class extents;
```

```

// 23.7.3.3.6, alias template dextents
template<class IndexType, size_t Rank>
 using dextents = see below;

// 23.7.3.3.7, alias template dims
template<size_t Rank, class IndexType = size_t>
 using dims = see below;

// 23.7.3.4, layout mapping
struct layout_left;
struct layout_right;
struct layout_stride;
template<size_t PaddingValue = dynamic_extent>
 struct layout_left_padded;
template<size_t PaddingValue = dynamic_extent>
 struct layout_right_padded;

// 23.7.3.5.3, class template default_accessor
template<class ElementType>
 class default_accessor;

// 23.7.3.5.4, class template aligned_accessor
template<class ElementType, size_t ByteAlignment>
 class aligned_accessor;

// 23.7.3.6, class template mdspan
template<class ElementType, class Extents, class LayoutPolicy = layout_right,
 class AccessorPolicy = default_accessor<ElementType>>
 class mdspan; // partially freestanding

// 23.7.3.7, submdspan creation
template<class OffsetType, class LengthType, class StrideType>
 struct extent_slice;
template<class FirstType, class LastType, class StrideType = constant_wrapper<1zu>>
 struct range_slice;

template<class LayoutMapping>
 struct submdspan_mapping_result;

struct full_extent_t { explicit full_extent_t() = default; };
inline constexpr full_extent_t full_extent{};

template<class IndexType, size_t... Extents, class... SliceSpecifiers>
 constexpr auto subextents(const extents<IndexType, Extents...>&, SliceSpecifiers...);

// 23.7.3.7.5, submdspan slice canonicalization
template<class IndexType, size_t... Extents, class... SliceSpecifiers>
 constexpr auto canonical_slices(const extents<IndexType, Extents...>& src,
 SliceSpecifiers... slices);

// 23.7.3.7.8, submdspan function template
template<class ElementType, class Extents, class LayoutPolicy,
 class AccessorPolicy, class... SliceSpecifiers>
 constexpr auto submdspan(
 const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& src,
 SliceSpecifiers... raw_slices) -> see below;

template<class T, class IndexType>
 concept index-pair-like = // exposition only
 pair-like<T> &&
 convertible_to<tuple_element_t<0, T>, IndexType> &&
 convertible_to<tuple_element_t<1, T>, IndexType>;

```

```
// 23.7.3.8, multidimensional copy algorithms
template<class Src, class Dst>
 constexpr void copy(const Src& src, const Dst& dst);
template<class ExecutionPolicy, class Src, class Dst>
 void copy(ExecutionPolicy&& policy, const Src& src, const Dst& dst);

template<class Dst, class T = Dst::value_type>
 constexpr void fill(const Dst& dst, const T& value);
template<class ExecutionPolicy, class Dst, class T = Dst::value_type>
 void fill(ExecutionPolicy&& policy, const Dst& dst, const T& value);
}
```

### 23.7.3.3 Class template extents

[mdspan.extents]

#### 23.7.3.3.1 Overview

[mdspan.extents.overview]

The class template `extents` represents a multidimensional index space of rank equal to `sizeof...(Extents)`. In 23.7, `extents` is used synonymously with multidimensional index space.

```
namespace std {
 template<class IndexType, size_t... Extents>
 class extents {
 public:
 using index_type = IndexType;
 using size_type = make_unsigned_t<index_type>;
 using rank_type = size_t;

 // 23.7.3.3.4, observers of the multidimensional index space
 static constexpr rank_type rank() noexcept { return sizeof...(Extents); }
 static constexpr rank_type rank_dynamic() noexcept { return dynamic-index(rank()); }
 static constexpr size_t static_extent(rank_type) noexcept;
 constexpr index_type extent(rank_type) const noexcept;

 // 23.7.3.3.3, constructors
 constexpr extents() noexcept = default;

 template<class OtherIndexType, size_t... OtherExtents>
 constexpr explicit(see below)
 extents(const extents<OtherIndexType, OtherExtents...>&) noexcept;
 template<class... OtherIndexTypes>
 constexpr explicit extents(OtherIndexTypes...) noexcept;
 template<class OtherIndexType, size_t N>
 constexpr explicit(N != rank_dynamic())
 extents(span<OtherIndexType, N>) noexcept;
 template<class OtherIndexType, size_t N>
 constexpr explicit(N != rank_dynamic())
 extents(const array<OtherIndexType, N>&) noexcept;

 // 23.7.3.3.5, comparison operators
 template<class OtherIndexType, size_t... OtherExtents>
 friend constexpr bool operator==(const extents&,
 const extents<OtherIndexType, OtherExtents...>&) noexcept;

 // 23.7.3.3.2, exposition-only helpers
 constexpr size_t fwd-prod-of-extents(rank_type) const noexcept; // exposition only
 constexpr size_t rev-prod-of-extents(rank_type) const noexcept; // exposition only
 template<class OtherIndexType>
 static constexpr auto index-cast(OtherIndexType&&) noexcept; // exposition only

 private:
 static constexpr rank_type dynamic-index(rank_type) noexcept; // exposition only
 static constexpr rank_type dynamic-index-inv(rank_type) noexcept; // exposition only
 array<index_type, rank_dynamic()> dynamic-extents{}; // exposition only
 };
}
```

```

template<class... Integrals>
 explicit extents(Integrals...)
 -> see below;
}

```

1 *Mandates:*

- (1.1) — `IndexType` is a signed or unsigned integer type, and
- (1.2) — each element of `Extents` is either equal to `dynamic_extent`, or is representable as a value of type `IndexType`.
- 2 Each specialization of `extents` models **regular** and is trivially copyable.
- 3 Let  $E_r$  be the  $r^{\text{th}}$  element of `Extents`.  $E_r$  is a *dynamic extent* if it is equal to `dynamic_extent`, otherwise  $E_r$  is a *static extent*. Let  $D_r$  be the value of `dynamic-extents[dynamic-index(r)]` if  $E_r$  is a dynamic extent, otherwise  $E_r$ .
- 4 The  $r^{\text{th}}$  interval of the multidimensional index space represented by an `extents` object is  $[0, D_r)$ .

### 23.7.3.3.2 Exposition-only helpers

[mdspan.extents.expo]

```
static constexpr rank_type dynamic-index(rank_type i) noexcept;
```

1 *Preconditions:* `i <= rank()` is true.

2 *Returns:* The number of  $E_r$  with  $r < i$  for which  $E_r$  is a dynamic extent.

```
static constexpr rank_type dynamic-index-inv(rank_type i) noexcept;
```

3 *Preconditions:* `i < rank_dynamic()` is true.

4 *Returns:* The minimum value of  $r$  such that `dynamic-index(r + 1) == i + 1` is true.

```
constexpr size_t fwd-prod-of-extents(rank_type i) const noexcept;
```

5 *Preconditions:* `i <= rank()` is true.

6 *Returns:* If `i > 0` is true, the product of `extent(k)` for all  $k$  in the range  $[0, i)$ , otherwise 1.

```
constexpr size_t rev-prod-of-extents(rank_type i) const noexcept;
```

7 *Preconditions:* `i < rank()` is true.

8 *Returns:* If `i + 1 < rank()` is true, the product of `extent(k)` for all  $k$  in the range  $[i + 1, \text{rank}())$ , otherwise 1.

```
template<class OtherIndexType>
```

```
 static constexpr auto index-cast(OtherIndexType&& i) noexcept;
```

9 *Effects:*

- (9.1) — If `remove_cvref_t<OtherIndexType>` is an integral type other than `bool`, then equivalent to `return i;`
- (9.2) — otherwise, equivalent to `return static_cast<index_type>(std::forward<OtherIndexType>(i));`

[Note 1: This function will always return an integral type other than `bool`. Since this function's call sites are constrained on convertibility of `OtherIndexType` to `index_type`, integer-class types can use the `static_cast` branch without loss of precision. — end note]

### 23.7.3.3.3 Constructors

[mdspan.extents.cons]

```
template<class OtherIndexType, size_t... OtherExtents>
```

```
 constexpr explicit(see below)
```

```
 extents(const extents<OtherIndexType, OtherExtents...>& other) noexcept;
```

1 *Constraints:*

- (1.1) — `sizeof...(OtherExtents) == rank()` is true.
- (1.2) — `((OtherExtents == dynamic_extent || Extents == dynamic_extent || OtherExtents == Extents) && ...)` is true.

2 *Preconditions:*

(2.1) — `other.extent(r)` equals  $E_r$  for each  $r$  for which  $E_r$  is a static extent, and

(2.2) — either

(2.2.1) — `sizeof...(OtherExtents)` is zero, or

(2.2.2) — `other.extent(r)` is representable as a value of type `index_type` for every rank index  $r$  of `other`.

3 *Postconditions:* `*this == other` is true.

4 *Remarks:* The expression inside `explicit` is equivalent to:

```
((Extents != dynamic_extent) && (OtherExtents == dynamic_extent)) || ...) ||
(numeric_limits<index_type>::max() < numeric_limits<OtherIndexType>::max())
```

```
template<class... OtherIndexTypes>
```

```
constexpr explicit extents(OtherIndexTypes... exts) noexcept;
```

5 Let  $N$  be `sizeof...(OtherIndexTypes)`, and let `exts_arr` be `array<index_type, N>{static_cast<index_type>(std::move(exts))...}`.

6 *Constraints:*

(6.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is true,

(6.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is true, and

(6.3) — `N == rank_dynamic() || N == rank()` is true.

[Note 1: One can construct `extents` from just dynamic extents, which are all the values getting stored, or from all the extents with a precondition. — end note]

7 *Preconditions:*

(7.1) — If `N != rank_dynamic()` is true, `exts_arr[r]` equals  $E_r$  for each  $r$  for which  $E_r$  is a static extent, and

(7.2) — either

(7.2.1) — `sizeof...(exts) == 0` is true, or

(7.2.2) — each element of `exts` is representable as a nonnegative value of type `index_type`.

8 *Postconditions:* `*this == extents(exts_arr)` is true.

```
template<class OtherIndexType, size_t N>
```

```
constexpr explicit(N != rank_dynamic())
```

```
extents(span<OtherIndexType, N> exts) noexcept;
```

```
template<class OtherIndexType, size_t N>
```

```
constexpr explicit(N != rank_dynamic())
```

```
extents(const array<OtherIndexType, N>& exts) noexcept;
```

9 *Constraints:*

(9.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true,

(9.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is true, and

(9.3) — `N == rank_dynamic() || N == rank()` is true.

10 *Preconditions:*

(10.1) — If `N != rank_dynamic()` is true, `exts[r]` equals  $E_r$  for each  $r$  for which  $E_r$  is a static extent, and

(10.2) — either

(10.2.1) —  $N$  is zero, or

(10.2.2) — `exts[r]` is representable as a nonnegative value of type `index_type` for every rank index  $r$ .

11 *Effects:*

(11.1) — If  $N$  equals `rank_dynamic()`, for all  $d$  in the range  $[0, \text{rank\_dynamic}())$ , direct-non-list-initializes `dynamic-extents[d]` with `as_const(exts[d])`.



- (11.2) — Otherwise, for all  $d$  in the range  $[0, \text{rank\_dynamic}())$ , `direct-non-list-initializes` *dynamic-extents*  $[d]$  with `as_const(extends[dynamic-index-inv(d)])`.

```
template<class... Integrals>
explicit extents(Integrals...) -> see below;
```

12 *Constraints:* `(is_convertible_v<Integrals, size_t> && ...)` is true.

13 *Remarks:* The deduced type is `extents<size_t, maybe-static-ext<Integrals>...>`.

#### 23.7.3.3.4 Observers of the multidimensional index space [mdspan.extents.obs]

```
static constexpr size_t static_extent(rank_type i) noexcept;
```

1 *Preconditions:*  $i < \text{rank}()$  is true.

2 *Returns:*  $E_i$ .

```
constexpr index_type extent(rank_type i) const noexcept;
```

3 *Preconditions:*  $i < \text{rank}()$  is true.

4 *Returns:*  $D_i$ .

#### 23.7.3.3.5 Comparison operators [mdspan.extents.cmp]

```
template<class OtherIndexType, size_t... OtherExtents>
friend constexpr bool operator==(const extents& lhs,
 const extents<OtherIndexType, OtherExtents...>& rhs) noexcept;
```

1 *Returns:* true if `lhs.rank()` equals `rhs.rank()` and if `lhs.extent(r)` equals `rhs.extent(r)` for every rank index  $r$  of `rhs`, otherwise false.

#### 23.7.3.3.6 Alias template dextents [mdspan.extents.dextents]

```
template<class IndexType, size_t Rank>
using dextents = see below;
```

1 *Result:* A type  $E$  that is a specialization of `extents` such that `E::rank() == Rank` && `E::rank() == E::rank_dynamic()` is true, and `E::index_type` denotes `IndexType`.

#### 23.7.3.3.7 Alias template dims [mdspan.extents.dims]

```
template<size_t Rank, class IndexType = size_t>
using dims = see below;
```

1 *Result:* A type  $E$  that is a specialization of `extents` such that `E::rank() == Rank` && `E::rank() == E::rank_dynamic()` is true, and `E::index_type` denotes `IndexType`.

#### 23.7.3.4 Layout mapping [mdspan.layout]

##### 23.7.3.4.1 General [mdspan.layout.general]

1 In 23.7.3.4.2 and 23.7.3.4.3:

- (1.1) —  $M$  denotes a layout mapping class.
- (1.2) —  $m$  denotes a (possibly const) value of type  $M$ .
- (1.3) —  $i$  and  $j$  are packs of (possibly const) integers that are multidimensional indices in `m.extents()` (23.7.3.1).  
[Note 1: The type of each element of the packs can be a different integer type. — end note]
- (1.4) —  $r$  is a (possibly const) rank index of `typename M::extents_type`.
- (1.5) —  $d_r$  is a pack of (possibly const) integers for which `sizeof...(dr) == M::extents_type::rank()` is true, the  $r^{\text{th}}$  element is equal to 1, and all other elements are equal to 0.

2 In 23.7.3.4.2 through 23.7.3.4.7:

- (2.1) — Let *is-mapping-of* be the exposition-only variable template defined as follows:

```
template<class Layout, class Mapping>
constexpr bool is-mapping-of = // exposition only
 is_same_v<typename Layout::template mapping<typename Mapping::extents_type>, Mapping>;
```

- (2.2) — Let *is-layout-left-padded-mapping-of* be the exposition-only variable template defined as follows:

```
template<class Mapping>
constexpr bool is-layout-left-padded-mapping-of = see below; // exposition only
```

where *is-layout-left-padded-mapping-of*<Mapping> is true if and only if Mapping denotes a specialization of `layout_left_padded<S>::mapping` for some value S of type `size_t`.

- (2.3) — Let *is-layout-right-padded-mapping-of* be the exposition-only variable template defined as follows:

```
template<class Mapping>
constexpr bool is-layout-right-padded-mapping-of = see below; // exposition only
```

where *is-layout-right-padded-mapping-of*<Mapping> is true if and only if Mapping denotes a specialization of `layout_right_padded<S>::mapping` for some value S of type `size_t`.

- (2.4) — For nonnegative integers  $x$  and  $y$ , let *LEAST-MULTIPLE-AT-LEAST*( $x, y$ ) denote

(2.4.1) —  $y$  if  $x$  is zero,

(2.4.2) — otherwise, the least multiple of  $x$  that is greater than or equal to  $y$ .

### 23.7.3.4.2 Requirements

[mdspan.layout.reqmts]

- <sup>1</sup> A type M meets the *layout mapping* requirements if

- (1.1) — M models copyable and equality\_comparable,
- (1.2) — `is_nothrow_move_constructible_v<M>` is true,
- (1.3) — `is_nothrow_move_assignable_v<M>` is true,
- (1.4) — `is_nothrow_swappable_v<M>` is true, and
- (1.5) — the following types and expressions are well-formed and have the specified semantics.

```
typename M::extents_type
```

- <sup>2</sup> *Result:* A type that is a specialization of `extents`.

```
typename M::index_type
```

- <sup>3</sup> *Result:* `typename M::extents_type::index_type`.

```
typename M::rank_type
```

- <sup>4</sup> *Result:* `typename M::extents_type::rank_type`.

```
typename M::layout_type
```

- <sup>5</sup> *Result:* A type MP that meets the layout mapping policy requirements (23.7.3.4.3) and for which *is-mapping-of*<MP, M> is true.

```
m.extents()
```

- <sup>6</sup> *Result:* `const typename M::extents_type&`

```
m(i...)
```

- <sup>7</sup> *Result:* `typename M::index_type`

- <sup>8</sup> *Returns:* A nonnegative integer less than `numeric_limits<typename M::index_type>::max()` and less than or equal to `numeric_limits<size_t>::max()`.

```
m(i...) == m(static_cast<typename M::index_type>(i)...)
```

- <sup>9</sup> *Result:* `bool`

- <sup>10</sup> *Returns:* `true`

```
m.required_span_size()
```

- <sup>11</sup> *Result:* `typename M::index_type`

- <sup>12</sup> *Returns:* If the size of the multidimensional index space `m.extents()` is 0, then 0, else 1 plus the maximum value of `m(i...)` for all  $i$ .

`m.is_unique()`

13     *Result:* `bool`

14     *Returns:* `true` only if for every `i` and `j` where `(i != j || ...)` is `true`, `m(i...)` `!=` `m(j...)` is `true`.

[*Note 1:* A mapping can return `false` even if the condition is met. For certain layouts, it is possibly not feasible to determine efficiently whether the layout is unique. — *end note*]

`m.is_exhaustive()`

15     *Result:* `bool`

16     *Returns:* `true` only if for all `k` in the range `[0, m.required_span_size())` there exists an `i` such that `m(i...)` equals `k`.

[*Note 2:* A mapping can return `false` even if the condition is met. For certain layouts, it is possibly not feasible to determine efficiently whether the layout is exhaustive. — *end note*]

`m.is_strided()`

17     *Result:* `bool`

18     *Returns:* `true` only if for every rank index `r` of `m.extents()` there exists an integer  $s_r$  such that, for all `i` where `(i +  $d_r$ )` is a multidimensional index in `m.extents()` (23.7.3.1), `m((i +  $d_r$ )...)` — `m(i...)` equals  $s_r$ .

[*Note 3:* This implies that for a strided layout  $m(i_0, \dots, i_k) = m(0, \dots, 0) + i_0 \times s_0 + \dots + i_k \times s_k$ . — *end note*]

[*Note 4:* A mapping can return `false` even if the condition is met. For certain layouts, it is possibly not feasible to determine efficiently whether the layout is strided. — *end note*]

`m.stride(r)`

19     *Preconditions:* `m.is_strided()` is `true`.

20     *Result:* `typename M::index_type`

21     *Returns:*  $s_r$  as defined in `m.is_strided()` above.

22     [*Note 5:* It is not required for `m.stride(r)` to be well-formed if `m.extents().rank()` is zero, even if `m.is_always_strided()` is `true`. — *end note*]

`M::is_always_unique()`

23     *Result:* A constant expression (7.7.3) of type `bool`.

24     *Returns:* `true` only if `m.is_unique()` is `true` for all possible objects `m` of type `M`.

[*Note 6:* A mapping can return `false` even if the above condition is met. For certain layout mappings, it is possibly not feasible to determine whether every instance is unique. — *end note*]

`M::is_always_exhaustive()`

25     *Result:* A constant expression (7.7.3) of type `bool`.

26     *Returns:* `true` only if `m.is_exhaustive()` is `true` for all possible objects `m` of type `M`.

[*Note 7:* A mapping can return `false` even if the above condition is met. For certain layout mappings, it is possibly not feasible to determine whether every instance is exhaustive. — *end note*]

`M::is_always_strided()`

27     *Result:* A constant expression (7.7.3) of type `bool`.

28     *Returns:* `true` only if `m.is_strided()` is `true` for all possible objects `m` of type `M`.

[*Note 8:* A mapping can return `false` even if the above condition is met. For certain layout mappings, it is possibly not feasible to determine whether every instance is strided. — *end note*]

#### 23.7.3.4.3 Layout mapping policy requirements

[`mdspan.layout.policy.reqmts`]

- <sup>1</sup> A type `MP` meets the *layout mapping policy* requirements if for a type `E` that is a specialization of `extents`, `MP::mapping<E>` is valid and denotes a type `X` that meets the layout mapping requirements (23.7.3.4.2), and for which the *qualified-id* `X::layout_type` is valid and denotes the type `MP` and the *qualified-id* `X::extents_type` denotes `E`.

## 23.7.3.4.4 Layout mapping policies

[mdspan.layout.policy.overview]

```

namespace std {
 struct layout_left {
 template<class Extents>
 class mapping;
 };
 struct layout_right {
 template<class Extents>
 class mapping;
 };
 struct layout_stride {
 template<class Extents>
 class mapping;
 };

 template<size_t PaddingValue>
 struct layout_left_padded {
 template<class Extents> class mapping;
 };
 template<size_t PaddingValue>
 struct layout_right_padded {
 template<class Extents> class mapping;
 };
}

```

- <sup>1</sup> Each of `layout_left`, `layout_right`, and `layout_stride`, as well as each specialization of `layout_left_padded` and `layout_right_padded`, meets the layout mapping policy requirements and is a trivially copyable type. Furthermore, `is_trivially_default_constructible_v<T>` is true for any such type `T`.

23.7.3.4.5 Class template `layout_left::mapping`

[mdspan.layout.left]

## 23.7.3.4.5.1 Overview

[mdspan.layout.left.overview]

- <sup>1</sup> `layout_left` provides a layout mapping where the leftmost extent has stride 1, and strides increase left-to-right as the product of extents.

```

namespace std {
 template<class Extents>
 class layout_left::mapping {
 public:
 using extents_type = Extents;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using layout_type = layout_left;

 // 23.7.3.4.5.2, constructors
 constexpr mapping() noexcept = default;
 constexpr mapping(const mapping&) noexcept = default;
 constexpr mapping(const extents_type&) noexcept;
 template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const mapping<OtherExtents>&) noexcept;
 template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_right::mapping<OtherExtents>&) noexcept;
 template<class LayoutLeftPaddedMapping>
 constexpr explicit(!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type,
 extents_type>)
 mapping(const LayoutLeftPaddedMapping&) noexcept;
 template<class OtherExtents>
 constexpr explicit(see below)
 mapping(const layout_stride::mapping<OtherExtents>&);

 constexpr mapping& operator=(const mapping&) noexcept = default;
 };
}

```

```

// 23.7.3.4.5.3, observers
constexpr const extents_type& extents() const noexcept { return extents_; }

constexpr index_type required_span_size() const noexcept;

template<class... Indices>
 constexpr index_type operator()(Indices...) const noexcept;

static constexpr bool is_always_unique() noexcept { return true; }
static constexpr bool is_always_exhaustive() noexcept { return true; }
static constexpr bool is_always_strided() noexcept { return true; }

static constexpr bool is_unique() noexcept { return true; }
static constexpr bool is_exhaustive() noexcept { return true; }
static constexpr bool is_strided() noexcept { return true; }

constexpr index_type stride(rank_type) const noexcept;

template<class OtherExtents>
 friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

private:
 extents_type extents_{}; // exposition only

// 23.7.3.7.7, submdspan mapping specialization
template<class... SliceSpecifiers>
 constexpr auto submdspan-mapping-impl(SliceSpecifiers...) const // exposition only
 -> see below;

template<class... SliceSpecifiers>
 friend constexpr auto submdspan_mapping(
 const mapping& src, SliceSpecifiers... slices) {
 return src.submdspan-mapping-impl(slices...);
 }
};
}

```

2 If Extents is not a specialization of extents, then the program is ill-formed.

3 layout\_left::mapping<E> is a trivially copyable type that models regular for each E.

4 Mandates: If Extents::rank\_dynamic() == 0 is true, then the size of the multidimensional index space Extents() is representable as a value of type typename Extents::index\_type.

#### 23.7.3.4.5.2 Constructors

[mdspan.layout.left.cons]

```
constexpr mapping(const extents_type& e) noexcept;
```

1 *Preconditions:* The size of the multidimensional index space e is representable as a value of type index\_type (6.9.3).

2 *Effects:* Direct-non-list-initializes *extents\_* with e.

```
template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const mapping<OtherExtents>& other) noexcept;
```

3 *Constraints:* is\_constructible\_v<extents\_type, OtherExtents> is true.

4 *Preconditions:* other.required\_span\_size() is representable as a value of type index\_type (6.9.3).

5 *Effects:* Direct-non-list-initializes *extents\_* with other.extents().

```
template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_right::mapping<OtherExtents>& other) noexcept;
```

6 *Constraints:*

(6.1) — extents\_type::rank() <= 1 is true, and

(6.2) — `is_constructible_v<extents_type, OtherExtents>` is true.

7 *Preconditions:* `other.required_span_size()` is representable as a value of type `index_type` (6.9.3).

8 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

```
template<class LayoutLeftPaddedMapping>
constexpr explicit(!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type,
 extents_type>)
mapping(const LayoutLeftPaddedMapping&) noexcept;
```

9 *Constraints:*

(9.1) — *is-layout-left-padded-mapping-of* <LayoutLeftPaddedMapping> is true.

(9.2) — `is_constructible_v<extents_type, typename LayoutLeftPaddedMapping::extents_type>` is true.

10 *Mandates:* If

(10.1) — `Extents::rank()` is greater than one,

(10.2) — `Extents::static_extent(0)` does not equal `dynamic_extent`, and

(10.3) — `LayoutLeftPaddedMapping::static-padding-stride` does not equal `dynamic_extent`,  
then `Extents::static_extent(0)` equals `LayoutLeftPaddedMapping::static-padding-stride`.

11 *Preconditions:*

(11.1) — If `extents_type::rank() > 1` is true, then `other.stride(1)` equals `other.extents().extent(0)`.

(11.2) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

```
template<class OtherExtents>
constexpr explicit(see below)
mapping(const layout_stride::mapping<OtherExtents>& other);
```

13 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

14 *Preconditions:*

(14.1) — If `extents_type::rank() > 0` is true, then for all  $r$  in the range  $[0, \text{extents\_type::rank}())$ , `other.stride(r)` equals `other.extents().fwd-prod-of-extents(r)`, and

(14.2) — `other.required_span_size()` is representable as a value of type `index_type` (6.9.3).

15 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

16 *Remarks:* The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

### 23.7.3.4.5.3 Observers

[mdspan.layout.left.obs]

```
constexpr index_type required_span_size() const noexcept;
```

1 *Returns:* `extents().fwd-prod-of-extents(extents_type::rank())`.

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;
```

2 *Constraints:*

(2.1) — `sizeof...(Indices) == extents_type::rank()` is true,

(2.2) — `(is_convertible_v<Indices, index_type> && ...)` is true, and

(2.3) — `(is_nothrow_constructible_v<index_type, Indices> && ...)` is true.

3 *Preconditions:* `extents_type::index-cast(i)` is a multidimensional index in `extents_` (23.7.3.1).

4 *Effects:* Let  $P$  be a parameter pack such that

```
is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>
```

is true. Equivalent to:

```

 return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);

constexpr index_type stride(rank_type i) const noexcept;
5 Constraints: extents_type::rank() > 0 is true.
6 Preconditions: i < extents_type::rank() is true.
7 Returns: extents().fwd-prod-of-extents(i).

template<class OtherExtents>
 friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
8 Constraints: extents_type::rank() == OtherExtents::rank() is true.
9 Effects: Equivalent to: return x.extents() == y.extents();

```

#### 23.7.3.4.6 Class template layout\_right::mapping

[mdspan.layout.right]

##### 23.7.3.4.6.1 Overview

[mdspan.layout.right.overview]

- 1 layout\_right provides a layout mapping where the rightmost extent is stride 1, and strides increase right-to-left as the product of extents.

```

namespace std {
 template<class Extents>
 class layout_right::mapping {
 public:
 using extents_type = Extents;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using layout_type = layout_right;

 // 23.7.3.4.6.2, constructors
 constexpr mapping() noexcept = default;
 constexpr mapping(const mapping&) noexcept = default;
 constexpr mapping(const extents_type&) noexcept;
 template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const mapping<OtherExtents>&) noexcept;
 template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_left::mapping<OtherExtents>&) noexcept;
 template<class LayoutRightPaddedMapping>
 constexpr explicit(!is_convertible_v<typename LayoutRightPaddedMapping::extents_type,
 extents_type>)
 mapping(const LayoutRightPaddedMapping&) noexcept;
 template<class OtherExtents>
 constexpr explicit(see below)
 mapping(const layout_stride::mapping<OtherExtents>&) noexcept;

 constexpr mapping& operator=(const mapping&) noexcept = default;

 // 23.7.3.4.6.3, observers
 constexpr const extents_type& extents() const noexcept { return extents_; }

 constexpr index_type required_span_size() const noexcept;

 template<class... Indices>
 constexpr index_type operator()(Indices...) const noexcept;

 static constexpr bool is_always_unique() noexcept { return true; }
 static constexpr bool is_always_exhaustive() noexcept { return true; }
 static constexpr bool is_always_strided() noexcept { return true; }

 static constexpr bool is_unique() noexcept { return true; }
 static constexpr bool is_exhaustive() noexcept { return true; }
 static constexpr bool is_strided() noexcept { return true; }

```



```

constexpr index_type stride(rank_type) const noexcept;

template<class OtherExtents>
 friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

private:
 extents_type extents_{}; // exposition only

 // 23.7.3.7.7, submdspan mapping specialization
 template<class... SliceSpecifiers>
 constexpr auto submdspan-mapping-impl (SliceSpecifiers...) const // exposition only
 -> see below;

 template<class... SliceSpecifiers>
 friend constexpr auto submdspan_mapping(
 const mapping& src, SliceSpecifiers... slices) {
 return src.submdspan-mapping-impl (slices...);
 }
};

```

2 If `Extents` is not a specialization of `extents`, then the program is ill-formed.

3 `layout_right::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

4 *Mandates:* If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

#### 23.7.3.4.6.2 Constructors

[mdspan.layout.right.cons]

```
constexpr mapping(const extents_type& e) noexcept;
```

1 *Preconditions:* The size of the multidimensional index space `e` is representable as a value of type `index_type` (6.9.3).

2 *Effects:* Direct-non-list-initializes `extents_` with `e`.

```

template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const mapping<OtherExtents>& other) noexcept;

```

3 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

4 *Preconditions:* `other.required_span_size()` is representable as a value of type `index_type` (6.9.3).

5 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

```

template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_left::mapping<OtherExtents>& other) noexcept;

```

6 *Constraints:*

(6.1) — `extents_type::rank() <= 1` is true, and

(6.2) — `is_constructible_v<extents_type, OtherExtents>` is true.

7 *Preconditions:* `other.required_span_size()` is representable as a value of type `index_type` (6.9.3).

8 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

```

template<class LayoutRightPaddedMapping>
 constexpr explicit(!is_convertible_v<typename LayoutRightPaddedMapping::extents_type,
 extents_type>)
 mapping(const LayoutRightPaddedMapping&) noexcept;

```

9 *Constraints:*

(9.1) — `is-layout-right-padded-mapping-of<LayoutRightPaddedMapping>` is true.

(9.2) — `is_constructible_v<extents_type, typename LayoutRightPaddedMapping::extents_type>` is true.



10 *Mandates:* If

(10.1) — `Extents::rank()` is greater than one,

(10.2) — `Extents::static_extent(Extents::rank() - 1)` does not equal `dynamic_extent`, and

(10.3) — `LayoutRightPaddedMapping::static-padding-stride` does not equal `dynamic_extent`,  
 then `Extents::static_extent(Extents::rank() - 1)` equals `LayoutRightPaddedMapping::static-padding-stride`.

11 *Preconditions:*

(11.1) — If `extents_type::rank() > 1` is true, then `other.stride(extents_type::rank() - 2)` equals `other.extents().extent(extents_type::rank() - 1)`.

(11.2) — `other.required_span_size()` is representable as a value of type `index_type`.

12 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

```
template<class OtherExtents>
constexpr explicit(see below)
mapping(const layout_stride::mapping<OtherExtents>& other) noexcept;
```

13 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

14 *Preconditions:*

(14.1) — If `extents_type::rank() > 0` is true, then for all  $r$  in the range  $[0, \text{extents\_type::rank}())$ , `other.stride(r)` equals `other.extents().rev-prod-of-extents(r)`.

(14.2) — `other.required_span_size()` is representable as a value of type `index_type` (6.9.3).

15 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

16 *Remarks:* The expression inside `explicit` is equivalent to:

```
!(extents_type::rank() == 0 && is_convertible_v<OtherExtents, extents_type>)
```

#### 23.7.3.4.6.3 Observers

[mdspan.layout.right.obs]

```
constexpr index_type required_span_size() const noexcept;
```

1 *Returns:* `extents().fwd-prod-of-extents(extents_type::rank())`.

```
template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;
```

2 *Constraints:*

- (2.1) — `sizeof...(Indices) == extents_type::rank()` is true,
- (2.2) — `(is_convertible_v<Indices, index_type> && ...)` is true, and
- (2.3) — `(is_nothrow_constructible_v<index_type, Indices> && ...)` is true.

3 *Preconditions:* `extents_type::index-cast(i)` is a multidimensional index in `extents_` (23.7.3.1).

4 *Effects:* Let  $P$  be a parameter pack such that

```
is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>
is true. Equivalent to:
return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);
```

```
constexpr index_type stride(rank_type i) const noexcept;
```

5 *Constraints:* `extents_type::rank() > 0` is true.

6 *Preconditions:* `i < extents_type::rank()` is true.

7 *Returns:* `extents().rev-prod-of-extents(i)`.

```
template<class OtherExtents>
friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

8 *Constraints:* `extents_type::rank() == OtherExtents::rank()` is true.

9 *Effects:* Equivalent to: `return x.extents() == y.extents();`

### 23.7.3.4.7 Class template `layout_stride::mapping` [mdspan.layout.stride]

#### 23.7.3.4.7.1 Overview [mdspan.layout.stride.overview]

<sup>1</sup> `layout_stride` provides a layout mapping where the strides are user-defined.

```
namespace std {
 template<class Extents>
 class layout_stride::mapping {
 public:
 using extents_type = Extents;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using layout_type = layout_stride;

 private:
 static constexpr rank_type rank_ = extents_type::rank(); // exposition only

 public:
 // 23.7.3.4.7.3, constructors
 constexpr mapping() noexcept;
 constexpr mapping(const mapping&) noexcept = default;
 template<class OtherIndexType>
 constexpr mapping(const extents_type&, span<OtherIndexType, rank_>) noexcept;
 template<class OtherIndexType>
 constexpr mapping(const extents_type&, const array<OtherIndexType, rank_>&) noexcept;

 template<class StridedLayoutMapping>
 constexpr explicit(see below) mapping(const StridedLayoutMapping&) noexcept;

 constexpr mapping& operator=(const mapping&) noexcept = default;

 // 23.7.3.4.7.4, observers
 constexpr const extents_type& extents() const noexcept { return extents_; }
 constexpr array<index_type, rank_> strides() const noexcept { return strides_; }

 constexpr index_type required_span_size() const noexcept;

 template<class... Indices>
 constexpr index_type operator()(Indices...) const noexcept;

 static constexpr bool is_always_unique() noexcept { return true; }
 static constexpr bool is_always_exhaustive() noexcept;
 static constexpr bool is_always_strided() noexcept { return true; }

 static constexpr bool is_unique() noexcept { return true; }
 constexpr bool is_exhaustive() const noexcept;
 static constexpr bool is_strided() noexcept { return true; }

 constexpr index_type stride(rank_type i) const noexcept { return strides_[i]; }

 template<class OtherMapping>
 friend constexpr bool operator==(const mapping&, const OtherMapping&) noexcept;

 private:
 extents_type extents_{}; // exposition only
 array<index_type, rank_> strides_{}; // exposition only

 // 23.7.3.7.7, submdspan mapping specialization
 template<class... SliceSpecifiers>
 constexpr auto submdspan-mapping-impl(SliceSpecifiers...) const // exposition only
 -> see below;
```

```

template<class... SliceSpecifiers>
 friend constexpr auto submdspan_mapping(
 const mapping& src, SliceSpecifiers... slices) {
 return src.submdspan-mapping-impl(slices...);
 }
};
}

```

- <sup>2</sup> If `Extents` is not a specialization of `extents`, then the program is ill-formed.
- <sup>3</sup> `layout_stride::mapping<E>` is a trivially copyable type that models `regular` for each `E`.
- <sup>4</sup> *Mandates*: If `Extents::rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `typename Extents::index_type`.

#### 23.7.3.4.7.2 Exposition-only helpers

[mdspan.layout.stride.expo]

- <sup>1</sup> Let *REQUIRED-SPAN-SIZE*(`e`, `strides`) be:

- (1.1) — 1, if `e.rank() == 0` is true,
- (1.2) — otherwise 0, if the size of the multidimensional index space `e` is 0,
- (1.3) — otherwise 1 plus the sum of products of `(e.extent(r) - 1)` and `extents_type::index-cast(strides[r])` for all *r* in the range `[0, e.rank())`.

- <sup>2</sup> Let *OFFSET*(`m`) be:

- (2.1) — `m()`, if `e.rank() == 0` is true,
- (2.2) — otherwise 0, if the size of the multidimensional index space `e` is 0,
- (2.3) — otherwise `m(z...)` for a pack of integers `z` that is a multidimensional index in `m.extents()` and each element of `z` equals 0.

- <sup>3</sup> Let *is-extents* be the exposition-only variable template defined as follows:

```

template<class T>
 constexpr bool is-extents = false; // exposition only
template<class IndexType, size_t... Args>
 constexpr bool is-extents<extents<IndexType, Args...>> = true; // exposition only

```

- <sup>4</sup> Let *layout-mapping-alike* be the exposition-only concept defined as follows:

```

template<class M>
 concept layout-mapping-alike = requires { // exposition only
 requires is-extents<typename M::extents_type>;
 { M::is_always_strided() } -> same_as<bool>;
 { M::is_always_exhaustive() } -> same_as<bool>;
 { M::is_always_unique() } -> same_as<bool>;
 bool_constant<M::is_always_strided()>::value;
 bool_constant<M::is_always_exhaustive()>::value;
 bool_constant<M::is_always_unique()>::value;
 };

```

[Note 1: This concept checks that the functions `M::is_always_strided()`, `M::is_always_exhaustive()`, and `M::is_always_unique()` exist, are constant expressions, and have a return type of `bool`. — end note]

#### 23.7.3.4.7.3 Constructors

[mdspan.layout.stride.cons]

constexpr mapping() noexcept;

- <sup>1</sup> *Preconditions*: `layout_right::mapping<extents_type>().required_span_size()` is representable as a value of type `index_type` (6.9.3).
- <sup>2</sup> *Effects*: Direct-non-list-initializes `extents_` with `extents_type()`, and for all *d* in the range `[0, rank_)`, direct-non-list-initializes `strides_[d]` with `layout_right::mapping<extents_type>().stride(d)`.

```

template<class OtherIndexType>
 constexpr mapping(const extents_type& e, span<OtherIndexType, rank_> s) noexcept;

```

```

template<class OtherIndexType>
constexpr mapping(const extents_type& e, const array<OtherIndexType, rank_>& s) noexcept;
3 Constraints:
(3.1) — is_convertible_v<const OtherIndexType&, index_type> is true, and
(3.2) — is_nothrow_constructible_v<index_type, const OtherIndexType&> is true.
4 Preconditions:
(4.1) — The result of converting $s[i]$ to index_type is greater than 0 for all i in the range $[0, rank_)$.
(4.2) — REQUIRED-SPAN-SIZE(e, s) is representable as a value of type index_type (6.9.3).
(4.3) — If rank_ is greater than 0, then there exists a permutation P of the integers in the range $[0, rank_)$,
 such that $s[p_i] \geq s[p_{i-1}] * e.extent(p_{i-1})$ is true for all i in the range $[1, rank_)$, where p_i
 is the i^{th} element of P .
 [Note 1: For layout_stride, this condition is necessary and sufficient for is_unique() to be true. — end
 note]
5 Effects: Direct-non-list-initializes extents_ with e, and for all d in the range $[0, rank_)$, direct-non-list-
 initializes strides_[d] with as_const(s[d]).

template<class StridedLayoutMapping>
constexpr explicit(see below)
mapping(const StridedLayoutMapping& other) noexcept;
6 Constraints:
(6.1) — layout-mapping-alike<StridedLayoutMapping> is satisfied.
(6.2) — is_constructible_v<extents_type, typename StridedLayoutMapping::extents_type> is
 true.
(6.3) — StridedLayoutMapping::is_always_unique() is true.
(6.4) — StridedLayoutMapping::is_always_strided() is true.
7 Preconditions:
(7.1) — StridedLayoutMapping meets the layout mapping requirements (23.7.3.4.2),
(7.2) — other.stride(r) > 0 is true for every rank index r of extents(),
(7.3) — other.required_span_size() is representable as a value of type index_type (6.9.3), and
(7.4) — OFFSET(other) == 0 is true.
8 Effects: Direct-non-list-initializes extents_ with other.extents(), and for all d in the range $[0, rank_)$,
 direct-non-list-initializes strides_[d] with other.stride(d).
9 Remarks: The expression inside explicit is equivalent to:
 !(is_convertible_v<typename StridedLayoutMapping::extents_type, extents_type> &&
 (is_mapping-of<layout_left, StridedLayoutMapping> ||
 is_mapping-of<layout_right, StridedLayoutMapping> ||
 is-layout-left-padded-mapping-of<StridedLayoutMapping> ||
 is-layout-right-padded-mapping-of<StridedLayoutMapping> ||
 is-mapping-of<layout_stride, StridedLayoutMapping>))

```

#### 23.7.3.4.7.4 Observers

[mdspan.layout.stride.obs]

```
constexpr index_type required_span_size() const noexcept;
```

1 Returns: *REQUIRED-SPAN-SIZE*(`extents()`, `strides_`).

```

template<class... Indices>
constexpr index_type operator()(Indices... i) const noexcept;

```

2 Constraints:

- (2.1) — `sizeof...(Indices) == rank_` is true,
- (2.2) — (`is_convertible_v<Indices, index_type> && ...`) is true, and
- (2.3) — (`is_nothrow_constructible_v<index_type, Indices> && ...`) is true.

3 *Preconditions:* `extents_type::index-cast(i)` is a multidimensional index in `extents_ (23.7.3.1)`.

4 *Effects:* Let `P` be a parameter pack such that

```
is_same_v<index_sequence_for<Indices...>, index_sequence<P...>>
```

is true. Equivalent to:

```
return ((static_cast<index_type>(std::move(i)) * stride(P)) + ... + 0);
```

```
static constexpr bool is_always_exhaustive() noexcept;
```

5 *Returns:* true if `rank_` is 0 or if there is a rank index `r` of `extents()` such that `extents_type::static_extent(r)` is 0, otherwise false.

```
constexpr bool is_exhaustive() const noexcept;
```

6 *Returns:*

(6.1) — true if `rank_` or the size of the multidimensional index space `m.extents()` is 0.

(6.2) — Otherwise, true if there is a permutation `P` of the integers in the range `[0, rank_)` such that `stride(p0)` equals 1, and `stride(pi)` equals `stride(pi-1) * extents().extent(pi-1)` for `i` in the range `[1, rank_)`, where `pi` is the  $i^{\text{th}}$  element of `P`.

(6.3) — Otherwise, false.

```
template<class OtherMapping>
friend constexpr bool operator==(const mapping& x, const OtherMapping& y) noexcept;
```

7 *Constraints:*

(7.1) — `layout-mapping-alike<OtherMapping>` is satisfied.

(7.2) — `rank_ == OtherMapping::extents_type::rank()` is true.

(7.3) — `OtherMapping::is_always_strided()` is true.

8 *Preconditions:* `OtherMapping` meets the layout mapping requirements (23.7.3.4.3).

9 *Returns:* true if `x.extents() == y.extents()` is true, `OFFSET(y) == 0` is true, and each of `x.stride(r) == y.stride(r)` is true for `r` in the range `[0, x.extents().rank())`. Otherwise, false.

#### 23.7.3.4.8 Class template `layout_left_padded::mapping` [mdspan.layout.leftpad]

##### 23.7.3.4.8.1 Overview [mdspan.layout.leftpad.overview]

1 `layout_left_padded` provides a layout mapping that behaves like `layout_left::mapping`, except that the padding stride `stride(1)` can be greater than or equal to `extent(0)`.

```
namespace std {
 template<size_t PaddingValue>
 template<class Extents>
 class layout_left_padded<PaddingValue>::mapping {
 public:
 static constexpr size_t padding_value = PaddingValue;

 using extents_type = Extents;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using layout_type = layout_left_padded<PaddingValue>;

 private:
 static constexpr size_t rank_ = extents_type::rank(); // exposition only
 static constexpr size_t first-static-extent = // exposition only
 extents_type::static_extent(0);

 // 23.7.3.4.8.2, exposition-only members
 static constexpr size_t static-padding-stride = see below; // exposition only

 public:
 // 23.7.3.4.8.3, constructors
 constexpr mapping() noexcept : mapping(extents_type{}) {}
 };
}
```

```

constexpr mapping(const mapping&) noexcept = default;
constexpr mapping(const extents_type&);
template<class OtherIndexType>
 constexpr mapping(const extents_type&, OtherIndexType);
template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_left::mapping<OtherExtents>&);
template<class OtherExtents>
 constexpr explicit(see below)
 mapping(const layout_stride::mapping<OtherExtents>&);
template<class LayoutLeftPaddedMapping>
 constexpr explicit(see below)
 mapping(const LayoutLeftPaddedMapping&);
template<class LayoutRightPaddedMapping>
 constexpr explicit(see below)
 mapping(const LayoutRightPaddedMapping&) noexcept;

constexpr mapping& operator=(const mapping&) noexcept = default;

// 23.7.3.4.8.4, observers
constexpr const extents_type& extents() const noexcept { return extents_; }
constexpr array<index_type, rank_> strides() const noexcept;

constexpr index_type required_span_size() const noexcept;
template<class... Indices>
 constexpr index_type operator()(Indices...) const noexcept;

static constexpr bool is_always_unique() noexcept { return true; }
static constexpr bool is_always_exhaustive() noexcept;
static constexpr bool is_always_strided() noexcept { return true; }

static constexpr bool is_unique() noexcept { return true; }
constexpr bool is_exhaustive() const noexcept;
static constexpr bool is_strided() noexcept { return true; }

constexpr index_type stride(rank_type) const noexcept;

template<class LayoutLeftPaddedMapping>
 friend constexpr bool operator==(const mapping&, const LayoutLeftPaddedMapping&) noexcept;

private:
 // 23.7.3.4.8.2, exposition-only members
 index_type stride-1 = static-padding-stride; // exposition only
 extents_type extents_{}; // exposition only
 // 23.7.3.7.7, submdspan mapping specialization
 template<class... SliceSpecifiers>
 constexpr auto submdspan-mapping-impl(SliceSpecifiers...) const // exposition only
 -> see below;

 template<class... SliceSpecifiers>
 friend constexpr auto submdspan_mapping(const mapping& src, SliceSpecifiers... slices) {
 return src.submdspan-mapping-impl(slices...);
 }
};

```

<sup>2</sup> If `Extents` is not a specialization of `extents`, then the program is ill-formed.

<sup>3</sup> `layout_left_padded::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

<sup>4</sup> Throughout 23.7.3.4.8, let `P_rank` be the following size `rank_` parameter pack of `size_t` values:

- (4.1) — the empty parameter pack, if `rank_` equals zero;
- (4.2) — otherwise, `0zu`, if `rank_` equals one;
- (4.3) — otherwise, the parameter pack `0zu, 1zu, ..., rank_- 1`.

5 *Mandates:*

- (5.1) — If `rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `index_type`.
- (5.2) — If `padding_value` is not equal to `dynamic_extent`, then `padding_value` is representable as a value of type `index_type`.
- (5.3) — If
  - (5.3.1) — `rank_` is greater than one,
  - (5.3.2) — `padding_value` does not equal `dynamic_extent`, and
  - (5.3.3) — `first-static-extent` does not equal `dynamic_extent`,
 then `LEAST-MULTIPLE-AT-LEAST(padding_value, first-static-extent)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.
- (5.4) — If
  - (5.4.1) — `rank_` is greater than one,
  - (5.4.2) — `padding_value` does not equal `dynamic_extent`, and
  - (5.4.3) — `extents_type::static_extent(k)` does not equal `dynamic_extent` for all  $k$  in the range  $[0, \text{extents\_type::rank}())$ ,
 then the product of `LEAST-MULTIPLE-AT-LEAST(padding_value, ext.static_extent(0))` and all values `ext.static_extent(k)` with  $k$  in the range of  $[1, \text{rank}_)$  is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

**23.7.3.4.8.2 Exposition-only members**

[mdspan.layout.leftpad.expo]

```
static constexpr size_t static-padding-stride = see below;
```

1 The value is

- (1.1) — 0, if `rank_` equals zero or one;
- (1.2) — otherwise, `dynamic_extent`, if `padding_value` or `first-static-extent` equals `dynamic_extent`;
- (1.3) — otherwise, the `size_t` value which is `LEAST-MULTIPLE-AT-LEAST(padding_value, first-static-extent)`.

```
index_type stride-1 = static-padding-stride;
```

2 *Recommended practice:* Implementations should not store this value if `static-padding-stride` is not `dynamic_extent`.

[Note 1: Using `extents<index_type, static-padding-stride>` instead of `index_type` as the type of `stride-1` would achieve this. — end note]

**23.7.3.4.8.3 Constructors**

[mdspan.layout.leftpad.cons]

```
constexpr mapping(const extents_type& ext);
```

1 *Preconditions:*

- (1.1) — The size of the multidimensional index space `ext` is representable as a value of type `index_type`.
- (1.2) — If `rank_` is greater than one and `padding_value` does not equal `dynamic_extent`, then `LEAST-MULTIPLE-AT-LEAST(padding_value, ext.extent(0))` is representable as a value of type `index_type`.
- (1.3) — If `rank_` is greater than one and `padding_value` does not equal `dynamic_extent`, then the product of `LEAST-MULTIPLE-AT-LEAST(padding_value, ext.extent(0))` and all values `ext.extent(k)` with  $k$  in the range of  $[1, \text{rank}_)$  is representable as a value of type `index_type`.

2 *Effects:*

- (2.1) — Direct-non-list-initializes `extents_` with `ext`; and
- (2.2) — if `rank_` is greater than one, direct-non-list-initializes `stride-1`
  - (2.2.1) — with `ext.extent(0)` if `padding_value` is `dynamic_extent`,
  - (2.2.2) — otherwise with `LEAST-MULTIPLE-AT-LEAST(padding_value, ext.extent(0))`.

```

template<class OtherIndexType>
constexpr mapping(const extents_type& ext, OtherIndexType padding);
3 Let pad be extents_type::index-cast(std::move(padding)).
4 Constraints:
(4.1) — is_convertible_v<OtherIndexType, index_type> is true.
(4.2) — is_nothrow_constructible_v<index_type, OtherIndexType> is true.
5 Preconditions:
(5.1) — pad is representable as a value of type index_type.
(5.2) — pad is greater than zero.
(5.3) — If rank_ is greater than one, then LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(0)) is representable as a value of type index_type.
(5.4) — If rank_ is greater than one, then the product of LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(0)) and all values ext.extent(k) with k in the range of [1, rank_) is representable as a value of type index_type.
(5.5) — If padding_value is not equal to dynamic_extent, padding_value equals pad.
6 Effects: Direct-non-list-initializes extents_ with ext, and if rank_ is greater than one, direct-non-list-initializes stride-1 with LEAST-MULTIPLE-AT-LEAST(pad, ext.extent(0)).

template<class OtherExtents>
constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
mapping(const layout_left::mapping<OtherExtents>& other);
7 Constraints: is_constructible_v<extents_type, OtherExtents> is true.
8 Mandates: If OtherExtents::rank() is greater than 1, then
 (static-padding-stride == dynamic_extent) ||
 (OtherExtents::static_extent(0) == dynamic_extent) ||
 (static-padding-stride == OtherExtents::static_extent(0))
 is true.
9 Preconditions:
(9.1) — If extents_type::rank() > 1 is true and padding_value == dynamic_extent is false, then other.stride(1) equals
 LEAST-MULTIPLE-AT-LEAST(padding_value,
 extents_type::index-cast(other.extents().extent(0)))
 and
(9.2) — other.required_span_size() is representable as a value of type index_type.
10 Effects: Equivalent to mapping(other.extents()).

template<class OtherExtents>
constexpr explicit(see below)
mapping(const layout_stride::mapping<OtherExtents>& other);
11 Constraints: is_constructible_v<extents_type, OtherExtents> is true.
12 Preconditions:
(12.1) — If rank_ is greater than 1 and padding_value does not equal dynamic_extent, then other.stride(1) equals
 LEAST-MULTIPLE-AT-LEAST(padding_value,
 extents_type::index-cast(other.extents().extent(0)))
(12.2) — If rank_ is greater than 0, then other.stride(0) equals 1.
(12.3) — If rank_ is greater than 2, then for all r in the range [2, rank_), other.stride(r) equals
 (other.extents().fwd-prod-of-extents(r) / other.extents().extent(0)) * other.stride(1)
(12.4) — other.required_span_size() is representable as a value of type index_type.

```



13 *Effects:*

(13.1) — Direct-non-list-initializes *extents\_* with *other.extents()* and

(13.2) — if *rank\_* is greater than one, direct-non-list-initializes *stride-1* with *other.stride(1)*.

14 *Remarks:* The expression inside *explicit* is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

```
template<class LayoutLeftPaddedMapping>
constexpr explicit(see below)
mapping(const LayoutLeftPaddedMapping& other);
```

15 *Constraints:*

(15.1) — *is-layout-left-padded-mapping-of*<LayoutLeftPaddedMapping> is true.

(15.2) — *is\_constructible\_v*<*extents\_type*, *typename LayoutLeftPaddedMapping::extents\_type*> is true.

16 *Mandates:* If *rank\_* is greater than 1, then

```
padding_value == dynamic_extent ||
LayoutLeftPaddedMapping::padding_value == dynamic_extent ||
padding_value == LayoutLeftPaddedMapping::padding_value
```

is true.

17 *Preconditions:*

(17.1) — If *rank\_* is greater than 1 and *padding\_value* does not equal *dynamic\_extent*, then *other.stride(1)* equals

```
LEAST-MULTIPLE-AT-LEAST(padding_value,
extents_type::index-cast(other.extent(0)))
```

(17.2) — *other.required\_span\_size()* is representable as a value of type *index\_type*.

18 *Effects:*

(18.1) — Direct-non-list-initializes *extents\_* with *other.extents()* and

(18.2) — if *rank\_* is greater than one, direct-non-list-initializes *stride-1* with *other.stride(1)*.

19 *Remarks:* The expression inside *explicit* is equivalent to:

```
!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type> ||
rank_ > 1 &&
(padding_value != dynamic_extent ||
LayoutLeftPaddedMapping::padding_value == dynamic_extent)
```

```
template<class LayoutRightPaddedMapping>
constexpr explicit(see below)
mapping(const LayoutRightPaddedMapping& other) noexcept;
```

20 *Constraints:*

(20.1) — *is-layout-right-padded-mapping-of*<LayoutRightPaddedMapping> is true or *is-mapping-of*<*layout\_right*, LayoutRightPaddedMapping> is true.

(20.2) — *rank\_* equals zero or one.

(20.3) — *is\_constructible\_v*<*extents\_type*, *typename LayoutRightPaddedMapping::extents\_type*> is true.

21 *Preconditions:* *other.required\_span\_size()* is representable as a value of type *index\_type*.

22 *Effects:* Direct-non-list-initializes *extents\_* with *other.extents()*.

23 *Remarks:* The expression inside *explicit* is equivalent to:

```
!is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type>
```

[*Note 1:* Neither the input mapping nor the mapping to be constructed uses the padding stride in the rank-0 or rank-1 case, so the padding stride does not affect either the constraints or the preconditions. — *end note*]

## 23.7.3.4.8.4 Observers

[mdspan.layout.leftpad.obs]

```
constexpr array<index_type, rank_> strides() const noexcept;
1 Returns: array<index_type, rank_>({stride(P_rank)...}).

constexpr index_type required_span_size() const noexcept;
2 Returns:
(2.1) — 0 if the multidimensional index space extents_ is empty,
(2.2) — otherwise, (*this)(extents_.extent(P_rank) - index_type(1)...) + 1.

template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;
3 Constraints:
(3.1) — sizeof...(Indices) == rank_ is true.
(3.2) — (is_convertible_v<Indices, index_type> && ...) is true.
(3.3) — (is_nothrow_constructible_v<index_type, Indices> && ...) is true.
4 Preconditions: extents_type::index-cast(idxs) is a multidimensional index in extents() (23.7.3.1).
5 Returns: ((static_cast<index_type>(std::move(idxs)) * stride(P_rank)) + ... + 0).

static constexpr bool is_always_exhaustive() noexcept;
6 Returns:
(6.1) — If rank_ equals zero or one, then true;
(6.2) — otherwise, if neither static-padding-stride nor first-static-extent equal dynamic_extent,
 then static-padding-stride == first-static-extent;
(6.3) — otherwise, false.

constexpr bool is_exhaustive() const noexcept;
7 Returns: true if rank_ equals zero or one; otherwise, extents_.extent(0) == stride(1).

constexpr index_type stride(rank_type r) const noexcept;
8 Preconditions: r is smaller than rank_.
9 Returns:
(9.1) — If r equals zero: 1;
(9.2) — otherwise, if r equals one: stride-1;
(9.3) — otherwise, the product of stride-1 and all values extents_.extent(k) with k in the range [1, r).

template<class LayoutLeftPaddedMapping>
friend constexpr bool operator==(const mapping& x, const LayoutLeftPaddedMapping& y) noexcept;
10 Constraints:
(10.1) — is-layout-left-padded-mapping-of<LayoutLeftPaddedMapping> is true.
(10.2) — LayoutLeftPaddedMapping::extents_type::rank() == rank_ is true.
11 Returns: true if x.extents() == y.extents() is true and rank_ < 2 || x.stride(1) == y.
 stride(1) is true. Otherwise, false.
```

## 23.7.3.4.9 Class template layout\_right\_padded::mapping

[mdspan.layout.rightpad]

## 23.7.3.4.9.1 Overview

[mdspan.layout.rightpad.overview]

1 layout\_right\_padded provides a layout mapping that behaves like layout\_right::mapping, except that the padding stride stride(extents\_type::rank() - 2) can be greater than or equal to extents\_type::extent(extents\_type::rank() - 1).

```
namespace std {
 template<size_t PaddingValue>
 template<class Extents>
```

```

class layout_right_padded<PaddingValue>::mapping {
public:
 static constexpr size_t padding_value = PaddingValue;

 using extents_type = Extents;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using layout_type = layout_right_padded<PaddingValue>;

private:
 static constexpr size_t rank_ = extents_type::rank(); // exposition only
 static constexpr size_t last-static-extent = // exposition only
 extents_type::static_extent(rank_ - 1);

 // 23.7.3.4.9.2, exposition-only members
 static constexpr size_t static-padding-stride = see below; // exposition only

public:
 // 23.7.3.4.9.3, constructors
 constexpr mapping() noexcept : mapping(extents_type{}) {}
 constexpr mapping(const mapping&) noexcept = default;
 constexpr mapping(const extents_type&);
 template<class OtherIndexType>
 constexpr mapping(const extents_type&, OtherIndexType);

 template<class OtherExtents>
 constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
 mapping(const layout_right::mapping<OtherExtents>&);
 template<class OtherExtents>
 constexpr explicit(see below)
 mapping(const layout_stride::mapping<OtherExtents>&);
 template<class LayoutRightPaddedMapping>
 constexpr explicit(see below)
 mapping(const LayoutRightPaddedMapping&);
 template<class LayoutLeftPaddedMapping>
 constexpr explicit(see below)
 mapping(const LayoutLeftPaddedMapping&) noexcept;

 constexpr mapping& operator=(const mapping&) noexcept = default;

 // 23.7.3.4.9.4, observers
 constexpr const extents_type& extents() const noexcept { return extents_; }
 constexpr array<index_type, rank_> strides() const noexcept;

 constexpr index_type required_span_size() const noexcept;

 template<class... Indices>
 constexpr index_type operator()(Indices...) const noexcept;

 static constexpr bool is_always_unique() noexcept { return true; }
 static constexpr bool is_always_exhaustive() noexcept;
 static constexpr bool is_always_strided() noexcept { return true; }

 static constexpr bool is_unique() noexcept { return true; }
 constexpr bool is_exhaustive() const noexcept;
 static constexpr bool is_strided() noexcept { return true; }

 constexpr index_type stride(rank_type) const noexcept;

 template<class LayoutRightPaddedMapping>
 friend constexpr bool operator==(const mapping&, const LayoutRightPaddedMapping&) noexcept;

```

```

private:
 // 23.7.3.4.9.2, exposition-only members
 index_type stride-rm2 = static-padding-stride; // exposition only
 extents_type extents_{}; // exposition only

 // 23.7.3.7.7, submdspan mapping specialization
 template<class... SliceSpecifiers>
 constexpr auto submdspan-mapping-impl (SliceSpecifiers...) const // exposition only
 -> see below;

 template<class... SliceSpecifiers>
 friend constexpr auto submdspan_mapping(const mapping& src, SliceSpecifiers... slices) {
 return src.submdspan-mapping-impl(slices...);
 }
};
}

```

<sup>2</sup> If `Extents` is not a specialization of `extents`, then the program is ill-formed.

<sup>3</sup> `layout_right_padded::mapping<E>` is a trivially copyable type that models `regular` for each `E`.

<sup>4</sup> Throughout 23.7.3.4.9, let `P_rank` be the following size `rank_` parameter pack of `size_t` values:

- (4.1) — the empty parameter pack, if `rank_` equals zero;
- (4.2) — otherwise, `0zu`, if `rank_` equals one;
- (4.3) — otherwise, the parameter pack `0zu, 1zu, ..., rank_ - 1`.

<sup>5</sup> *Mandates:*

- (5.1) — If `rank_dynamic() == 0` is true, then the size of the multidimensional index space `Extents()` is representable as a value of type `index_type`.
- (5.2) — If `padding_value` is not equal to `dynamic_extent`, then `padding_value` is representable as a value of type `index_type`.
- (5.3) — If
  - (5.3.1) — `rank_` is greater than one,
  - (5.3.2) — `padding_value` does not equal `dynamic_extent`, and
  - (5.3.3) — `last-static-extent` does not equal `dynamic_extent`,
 then `LEAST-MULTIPLE-AT-LEAST(padding_value, last-static-extent)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.
- (5.4) — If
  - (5.4.1) — `rank_` is greater than one,
  - (5.4.2) — `padding_value` does not equal `dynamic_extent`, and
  - (5.4.3) — `extents_type::static_extent(k)` does not equal `dynamic_extent` for all `k` in the range `[0, rank_)`,
 then the product of `LEAST-MULTIPLE-AT-LEAST(padding_value, ext.static_extent(rank_ - 1))` and all values `ext.static_extent(k)` with `k` in the range of `[0, rank_ - 1)` is representable as a value of type `size_t`, and is representable as a value of type `index_type`.

#### 23.7.3.4.9.2 Exposition-only members

[mdspan.layout.rightpad.expo]

```
static constexpr size_t static-padding-stride = see below;
```

<sup>1</sup> The value is

- (1.1) — 0, if `rank_` equals zero or one;
- (1.2) — otherwise, `dynamic_extent`, if `padding_value` or `last-static-extent` equals `dynamic_extent`;
- (1.3) — otherwise, the `size_t` value which is `LEAST-MULTIPLE-AT-LEAST(padding_value, last-static-extent)`.

```
index_type stride-rm2 = static-padding-stride;
```

- 2 *Recommended practice:* Implementations should not store this value if *static-padding-stride* is not *dynamic\_extent*.

[Note 1: Using `extents<index_type, static-padding-stride>` instead of `index_type` as the type of *stride-rm2* would achieve this. — end note]

### 23.7.3.4.9.3 Constructors

[mdspan.layout.rightpad.cons]

```
constexpr mapping(const extents_type& ext);
```

- 1 *Preconditions:*

- (1.1) — The size of the multidimensional index space *ext* is representable as a value of type *index\_type*.
- (1.2) — If *rank\_* is greater than one and *padding\_value* does not equal *dynamic\_extent*, then *LEAST-MULTIPLE-AT-LEAST*(*padding\_value*, *ext.extent(rank\_ - 1)*) is representable as a value of type *index\_type*.
- (1.3) — If *rank\_* is greater than one and *padding\_value* does not equal *dynamic\_extent*, then the product of *LEAST-MULTIPLE-AT-LEAST*(*padding\_value*, *ext.extent(rank\_ - 1)*) and all values *ext.extent(k)* with *k* in the range of  $[0, rank_ - 1)$  is representable as a value of type *index\_type*.

- 2 *Effects:*

- (2.1) — Direct-non-list-initializes *extents\_* with *ext*; and
- (2.2) — if *rank\_* is greater than one, direct-non-list-initializes *stride-rm2*
  - (2.2.1) — with *ext.extent(rank\_ - 1)* if *padding\_value* is *dynamic\_extent*,
  - (2.2.2) — otherwise with *LEAST-MULTIPLE-AT-LEAST*(*padding\_value*, *ext.extent(rank\_ - 1)*).

```
template<class OtherIndexType>
```

```
constexpr mapping(const extents_type& ext, OtherIndexType padding);
```

- 3 Let *pad* be *extents\_type::index-cast*(*std::move*(*padding*)).

- 4 *Constraints:*

- (4.1) — *is\_convertible\_v<OtherIndexType, index\_type>* is true.
- (4.2) — *is\_nothrow\_constructible\_v<index\_type, OtherIndexType>* is true.

- 5 *Preconditions:*

- (5.1) — *pad* is representable as a value of type *index\_type*.
- (5.2) — *pad* is greater than zero.
- (5.3) — If *rank\_* is greater than one, then *LEAST-MULTIPLE-AT-LEAST*(*pad*, *ext.extent(rank\_ - 1)*) is representable as a value of type *index\_type*.
- (5.4) — If *rank\_* is greater than one, then the product of *LEAST-MULTIPLE-AT-LEAST*(*pad*, *ext.extent(rank\_ - 1)*) and all values *ext.extent(k)* with *k* in the range of  $[0, rank_ - 1)$  is representable as a value of type *index\_type*.
- (5.5) — If *padding\_value* is not equal to *dynamic\_extent*, *padding\_value* equals *pad*.

- 6 *Effects:* Direct-non-list-initializes *extents\_* with *ext*, and if *rank\_* is greater than one, direct-non-list-initializes *stride-rm2* with *LEAST-MULTIPLE-AT-LEAST*(*pad*, *ext.extent(rank\_ - 1)*).

```
template<class OtherExtents>
```

```
constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
```

```
mapping(const layout_right::mapping<OtherExtents>& other);
```

- 7 *Constraints:* *is\_constructible\_v<extents\_type, OtherExtents>* is true.

- 8 *Mandates:* If *OtherExtents::rank()* is greater than 1, then

```
(static-padding-stride == dynamic_extent) ||
(OtherExtents::static_extent(rank_ - 1) == dynamic_extent) ||
(static-padding-stride == OtherExtents::static_extent(rank_ - 1))
```

is true.

9      *Preconditions:*

- (9.1) — If `rank_` > 1 is true and `padding_value == dynamic_extent` is false, then `other.strides(rank_ - 2)` equals

```
LEAST-MULTIPLE-AT-LEAST(padding_value,
 extents_type::index-cast(other.extents().extent(rank_ - 1)))
```

and

- (9.2) — `other.required_span_size()` is representable as a value of type `index_type`.

<sup>10</sup> *Effects*: Equivalent to `mapping(other.extents())`.

```
template<class OtherExtents>
 constexpr explicit(see below)
```

```
mapping(const layout_stride::mapping<OtherExtents>& other);
```

<sup>11</sup> *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

12      *Preconditions:*

- (12.1) — If `rank_` is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(rank_ - 2)` equals

```
LEAST-MULTIPLE-AT-LEAST(padding_value,
 extents_type::index-cast(other.extents().extent(rank - 1)))
```

- (12.2) — If *rank\_* is greater than 0, then `other.stride(rank_ - 1)` equals 1.

- (12.3) — If `rank_` is greater than 2, then for all `r` in the range `[0, rank_ - 2)`, `other.stride(r)` equals
- $$(\text{other.extents().rev-prod-of-extents}(r) / \text{other.extents().extent}(\text{rank\_} - 1)) * \text{other.stride}(\text{rank\_} - 2)$$

- (12.4) — `other.required_span_size()` is representable as a value of type `index_type`.

13      *Effects:*

- (13.1) — Direct-non-list-initializes *extents\_* with `other.extents()`; and

- (13.2) — if `rank_` is greater than one, `direct-non-list-initializes` `stride-rm2` with `other.stride(rank_ - 2)`.

<sup>14</sup> *Remarks:* The expression inside `explicit` is equivalent to:

```
!(rank_ == 0 && is_convertible_v<OtherExtents, extents_type>)
```

```
template<class LayoutRightPaddedMapping>
```

```
constexpr explicit(see below)
```

```
mapping(const LayoutRightPaddedMapping& other);
```

15 *Constraints:*

- (15.1) — *is-layout-right-padded-mapping-of* <LayoutRightPaddedMapping> is true.

- (15.2) — `is_constructible_v<extents_type, typename LayoutRightPaddedMapping::extents_type>` is true.

<sup>16</sup> *Mandates:* If *rank* is greater than 1, then

```
padding_value == dynamic_extent ||
LayoutRightPaddedMapping::padding_value == dynamic_extent ||
padding_value == LayoutRightPaddedMapping::padding_value
```

is true.

17      *Preconditions:*

- (17.1) — If `rank_` is greater than 1 and `padding_value` does not equal `dynamic_extent`, then `other.stride(rank_ - 2)` equals

```
LEAST-MULTIPLE-AT-LEAST(padding_value,
 extents_type::index-cast(other.extent(rank_ - 1)))
```

- (17.2) — `other.required_span_size()` is representable as a value of type `index_type`.

18 *Effects:*

- (18.1) — Direct-non-list-initializes *extents\_* with `other.extents()`; and

- (18.2) — if *rank\_* is greater than one, direct-non-list-initializes *stride-rm2* with `other.stride(rank_ - 2)`.

19 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutRightPaddedMapping::extents_type, extents_type> ||
rank_ > 1 &&
(padding_value != dynamic_extent ||
LayoutRightPaddedMapping::padding_value == dynamic_extent)
```

```
template<class LayoutLeftPaddedMapping>
constexpr explicit(see below)
mapping(const LayoutLeftPaddedMapping& other) noexcept;
```

20 *Constraints:*

- (20.1) — *is-layout-left-padded-mapping-of*<LayoutLeftPaddedMapping> is true or  
*is-mapping-of*<layout\_left, LayoutLeftPaddedMapping> is true.
- (20.2) — *rank\_* equals zero or one.
- (20.3) — *is\_constructible\_v*<extents\_type, typename LayoutLeftPaddedMapping::extents\_type>  
is true.

21 *Preconditions:* `other.required_span_size()` is representable as a value of type *index\_type*.

22 *Effects:* Direct-non-list-initializes *extents\_* with `other.extents()`.

23 *Remarks:* The expression inside `explicit` is equivalent to:

```
!is_convertible_v<typename LayoutLeftPaddedMapping::extents_type, extents_type>
```

[*Note 1:* Neither the input mapping nor the mapping to be constructed uses the padding stride in the rank-0 or rank-1 case, so the padding stride affects neither the constraints nor the preconditions. — *end note*]

#### 23.7.3.4.9.4 Observers

[`mdspan.layout.rightpad.obs`]

```
constexpr array<index_type, rank_> strides() const noexcept;
```

1 *Returns:* `array<index_type, rank_>(stride(P_rank)...) .`

```
constexpr index_type required_span_size() const noexcept;
```

2 *Returns:* 0 if the multidimensional index space *extents\_* is empty, otherwise `(*this)(extents_ .extent(P_rank) - index_type(1)...) + 1`.

```
template<class... Indices>
constexpr index_type operator()(Indices... idxs) const noexcept;
```

3 *Constraints:*

- (3.1) — `sizeof...(Indices) == rank_` is true.
- (3.2) — `(is_convertible_v<Indices, index_type> && ...)` is true.
- (3.3) — `(is_nothrow_constructible_v<index_type, Indices> && ...)` is true.

4 *Preconditions:* *extents\_type::index-cast*(*idxs*) is a multidimensional index in *extents()* (23.7.3.1).

5 *Returns:* `((static_cast<index_type>(std::move(idxs)) * stride(P_rank)) + ... + 0)`.

```
static constexpr bool is_always_exhaustive() noexcept;
```

6 *Returns:*

- (6.1) — If *rank\_* equals zero or one, then true;
- (6.2) — otherwise, if neither *static-padding-stride* nor *last-static-extent* equal *dynamic\_extent*, then *static-padding-stride* == *last-static-extent*;
- (6.3) — otherwise, false.

```
constexpr bool is_exhaustive() const noexcept;
```

7 *Returns:* true if *rank\_* equals zero or one; otherwise,

```
extents_.extent(rank_ - 1) == stride(rank_ - 2)
```

```
constexpr index_type stride(rank_type r) const noexcept;
```

8     *Preconditions:*  $r$  is smaller than  $rank\_$ .

9     *Returns:*

- (9.1)     — If  $r$  equals  $rank\_ - 1$ : 1;
- (9.2)     — otherwise, if  $r$  equals  $rank\_ - 2$ :  $stride\_rm2$ ;
- (9.3)     — otherwise, the product of  $stride\_rm2$  and all values  $extents\_extent(k)$  with  $k$  in the range of  $[r + 1, rank\_ - 1)$ .

```
template<class LayoutRightPaddedMapping>
```

```
friend constexpr bool operator==(const mapping& x, const LayoutRightPaddedMapping& y) noexcept;
```

10     *Constraints:*

- (10.1)     — *is-layout-right-padded-mapping-of*<LayoutRightPaddedMapping> is true.
- (10.2)     — `LayoutRightPaddedMapping::extents_type::rank() == rank_` is true.

11     *Returns:* true if `x.extents() == y.extents()` is true and  $rank\_ < 2$  || `x.stride(rank_ - 2) == y.stride(rank_ - 2)` is true. Otherwise, false.

### 23.7.3.5 Accessor policy

[mdspan.accessor]

#### 23.7.3.5.1 General

[mdspan.accessor.general]

- 1 An *accessor policy* defines types and operations by which a reference to a single object is created from an abstract data handle to a number of such objects and an index.
- 2 A range of indices  $[0, N)$  is an *accessible range* of a given data handle and an accessor if, for each  $i$  in the range, the accessor policy's *access* function produces a valid reference to an object.
- 3 In 23.7.3.5.2,
- (3.1)     —  $A$  denotes an accessor policy.
- (3.2)     —  $a$  denotes a value of type  $A$  or `const A`.
- (3.3)     —  $p$  denotes a value of type `A::data_handle_type` or `const A::data_handle_type`.  
[Note 1: The type `A::data_handle_type` need not be dereferenceable. — end note]
- (3.4)     —  $n$ ,  $i$ , and  $j$  each denote values of type `size_t`.

#### 23.7.3.5.2 Requirements

[mdspan.accessor.reqmts]

- 1 A type  $A$  meets the accessor policy requirements if
- (1.1)     —  $A$  models *copyable*,
- (1.2)     — `is_nothrow_move_constructible_v<A>` is true,
- (1.3)     — `is_nothrow_move_assignable_v<A>` is true,
- (1.4)     — `is_nothrow_swappable_v<A>` is true, and
- (1.5)     — the following types and expressions are well-formed and have the specified semantics.

```
typename A::element_type
```

2     *Result:* A complete object type that is not an abstract class type.

```
typename A::data_handle_type
```

3     *Result:* A type that models *copyable*, and for which `is_nothrow_move_constructible_v<A::data_handle_type>` is true, `is_nothrow_move_assignable_v<A::data_handle_type>` is true, and `is_nothrow_swappable_v<A::data_handle_type>` is true.

[Note 1: The type of `data_handle_type` need not be `element_type*`. — end note]

```
typename A::reference
```

4     *Result:* A type that models *common\_reference\_with*<`A::reference&&`, `A::element_type&`>.

[Note 2: The type of `reference` need not be `element_type&`. — end note]



typename A::offset\_policy

5     *Result:* A type OP such that:

- (5.1)     — OP meets the accessor policy requirements,
- (5.2)     — constructible\_from<OP, const A&> is modeled, and
- (5.3)     — is\_same\_v<typename OP::element\_type, typename A::element\_type> is true.

a.access(p, i)

6     *Result:* A::reference

7     *Remarks:* The expression is equality preserving.

8     [*Note 3:* Concrete accessor policies can impose preconditions for their `access` function. However, they might not. For example, an accessor where `p` is `span<A::element_type, dynamic_extent>` and `access(p, i)` returns `p[i % p.size()]` does not need to impose a precondition on `i`. — *end note*]

a.offset(p, i)

9     *Result:* A::offset\_policy::data\_handle\_type

10    *Returns:* `q` such that for `b` being `A::offset_policy(a)`, and any integer `n` for which `[0, n)` is an accessible range of `p` and `a`:

- (10.1)    — `[0, n - i)` is an accessible range of `q` and `b`; and
- (10.2)    — `b.access(q, j)` provides access to the same element as `a.access(p, i + j)`, for every `j` in the range `[0, n - i)`.

11    *Remarks:* The expression is equality-preserving.

### 23.7.3.5.3 Class template default\_accessor

[mdspan.accessor.default]

#### 23.7.3.5.3.1 Overview

[mdspan.accessor.default.overview]

```
namespace std {
 template<class ElementType>
 struct default_accessor {
 using offset_policy = default_accessor;
 using element_type = ElementType;
 using reference = ElementType&;
 using data_handle_type = ElementType*;

 constexpr default_accessor() noexcept = default;
 template<class OtherElementType>
 constexpr default_accessor(default_accessor<OtherElementType>) noexcept;
 constexpr reference access(data_handle_type p, size_t i) const noexcept;
 constexpr data_handle_type offset(data_handle_type p, size_t i) const noexcept;
 };
}
```

1    `default_accessor` meets the accessor policy requirements.

2    `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

3    Each specialization of `default_accessor` is a trivially copyable type that models `semiregular`.

4    `[0, n)` is an accessible range for an object `p` of type `data_handle_type` and an object of type `default_accessor` if and only if `[p, p + n)` is a valid range.

#### 23.7.3.5.3.2 Members

[mdspan.accessor.default.members]

```
template<class OtherElementType>
 constexpr default_accessor(default_accessor<OtherElementType>) noexcept {}
```

1    *Constraints:* `is_convertible_v<OtherElementType(*), element_type(*)>` is true.

```
constexpr reference access(data_handle_type p, size_t i) const noexcept;
```

2    *Effects:* Equivalent to: `return p[i];`

```
constexpr data_handle_type offset(data_handle_type p, size_t i) const noexcept;
```

3 *Effects:* Equivalent to: return `p + i`;

#### 23.7.3.5.4 Class template `aligned_accessor`

[mdspan.accessor.aligned]

##### 23.7.3.5.4.1 Overview

[mdspan.accessor.aligned.overview]

```
namespace std {
 template<class ElementType, size_t ByteAlignment>
 struct aligned_accessor {
 using offset_policy = default_accessor<ElementType>;
 using element_type = ElementType;
 using reference = ElementType&;
 using data_handle_type = ElementType*;

 static constexpr size_t byte_alignment = ByteAlignment;

 constexpr aligned_accessor() noexcept = default;
 template<class OtherElementType, size_t OtherByteAlignment>
 constexpr aligned_accessor(
 aligned_accessor<OtherElementType, OtherByteAlignment>) noexcept;
 template<class OtherElementType>
 constexpr explicit aligned_accessor(default_accessor<OtherElementType>) noexcept;

 template<class OtherElementType>
 constexpr operator default_accessor<OtherElementType>() const noexcept;

 constexpr reference access(data_handle_type p, size_t i) const noexcept;

 constexpr typename offset_policy::data_handle_type offset(
 data_handle_type p, size_t i) const noexcept;
 };
}
```

1 *Mandates:*

(1.1) — `byte_alignment` is a power of two, and

(1.2) — `byte_alignment >= alignof(ElementType)` is true.

2 `aligned_accessor` meets the accessor policy requirements.

3 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.

4 Each specialization of `aligned_accessor` is a trivially copyable type that models `semiregular`.

5 `[0, n)` is an accessible range for an object `p` of type `data_handle_type` and an object of type `aligned_accessor` if and only if

(5.1) — `[p, p + n)` is a valid range, and,

(5.2) — if `n` is greater than zero, then `is_sufficiently_aligned<byte_alignment>(p)` is true.

6 [Example 1: The following function `compute` uses `is_sufficiently_aligned` to check whether a given `mdspan` with `default_accessor` has a data handle with sufficient alignment to be used with `aligned_accessor<float, 4 * sizeof(float)>`. If so, the function dispatches to a function `compute_using_fourfold_overalignment` that requires fourfold over-alignment of arrays, but can therefore use hardware-specific instructions, such as four-wide SIMD (Single Instruction Multiple Data) instructions. Otherwise, `compute` dispatches to a possibly less optimized function `compute_without_requiring_overalignment` that has no over-alignment requirement.

```
void compute_using_fourfold_overalignment(
 mdspan<float, dims<1>, layout_right, aligned_accessor<float, 4 * alignof(float)>> x);

void compute_without_requiring_overalignment(
 mdspan<float, dims<1>, layout_right> x);

void compute(mdspan<float, dims<1>> x) {
 constexpr auto byte_alignment = 4 * sizeof(float);
 auto accessor = aligned_accessor<float, byte_alignment>{};
 auto x_handle = x.data_handle();
```

```

 if (is_sufficiently_aligned<byte_alignment>(x_handle)) {
 compute_using_fourfold_overalignment(mdspan{x_handle, x.mapping(), accessor});
 } else {
 compute_without_requiring_overalignment(x);
 }
}

```

— end example]

#### 23.7.3.5.4.2 Members

[mdspan.accessor.aligned.members]

```

template<class OtherElementType, size_t OtherByteAlignment>
constexpr aligned_accessor(aligned_accessor<OtherElementType, OtherByteAlignment>) noexcept;

```

1 *Constraints:*

(1.1) — is\_convertible\_v<OtherElementType(\*)[], element\_type(\*)[]> is true.

(1.2) — OtherByteAlignment >= byte\_alignment is true.

2 *Effects:* None.

```

template<class OtherElementType>
constexpr explicit aligned_accessor(default_accessor<OtherElementType>) noexcept;

```

3 *Constraints:* is\_convertible\_v<OtherElementType(\*)[], element\_type(\*)[]> is true.

4 *Effects:* None.

```

constexpr reference access(data_handle_type p, size_t i) const noexcept;

```

5 *Preconditions:* [0, i + 1) is an accessible range for p and \*this.

6 *Effects:* Equivalent to: return assume\_aligned<byte\_alignment>(p)[i];

```

template<class OtherElementType>
constexpr operator default_accessor<OtherElementType>() const noexcept;

```

7 *Constraints:* is\_convertible\_v<element\_type(\*)[], OtherElementType(\*)[]> is true.

8 *Effects:* Equivalent to: return {};

```

constexpr typename offset_policy::data_handle_type
offset(data_handle_type p, size_t i) const noexcept;

```

9 *Preconditions:* [0, i + 1) is an accessible range for p and \*this.

10 *Effects:* Equivalent to: return assume\_aligned<byte\_alignment>(p) + i;

#### 23.7.3.6 Class template mdspan

[mdspan.mdspan]

##### 23.7.3.6.1 Overview

[mdspan.mdspan.overview]

1 mdspan is a view of a multidimensional array of elements.

```

namespace std {
 template<class ElementType, class Extents, class LayoutPolicy = layout_right,
 class AccessorPolicy = default_accessor<ElementType>>
 class mdspan {
 public:
 using extents_type = Extents;
 using layout_type = LayoutPolicy;
 using accessor_type = AccessorPolicy;
 using mapping_type = layout_type::template mapping<extents_type>;
 using element_type = ElementType;
 using value_type = remove_cv_t<element_type>;
 using index_type = extents_type::index_type;
 using size_type = extents_type::size_type;
 using rank_type = extents_type::rank_type;
 using data_handle_type = accessor_type::data_handle_type;
 using reference = accessor_type::reference;
 };
}

```

```

static constexpr rank_type rank() noexcept { return extents_type::rank(); }
static constexpr rank_type rank_dynamic() noexcept { return extents_type::rank_dynamic(); }
static constexpr size_t static_extent(rank_type r) noexcept
{ return extents_type::static_extent(r); }
constexpr index_type extent(rank_type r) const noexcept { return extents().extent(r); }

// 23.7.3.6.2, constructors
constexpr mdspan();
constexpr mdspan(const mdspan& rhs) = default;
constexpr mdspan(mdspan&& rhs) = default;

template<class... OtherIndexTypes>
constexpr explicit mdspan(data_handle_type ptr, OtherIndexTypes... exts);
template<class OtherIndexType, size_t N>
constexpr explicit(N != rank_dynamic())
 mdspan(data_handle_type p, span<OtherIndexType, N> exts);
template<class OtherIndexType, size_t N>
constexpr explicit(N != rank_dynamic())
 mdspan(data_handle_type p, const array<OtherIndexType, N>& exts);
constexpr mdspan(data_handle_type p, const extents_type& ext);
constexpr mdspan(data_handle_type p, const mapping_type& m);
constexpr mdspan(data_handle_type p, const mapping_type& m, const accessor_type& a);

template<class OtherElementType, class OtherExtents,
 class OtherLayoutPolicy, class OtherAccessorPolicy>
constexpr explicit(see below)
 mdspan(const mdspan<OtherElementType, OtherExtents,
 OtherLayoutPolicy, OtherAccessorPolicy>& other);

constexpr mdspan& operator=(const mdspan& rhs) = default;
constexpr mdspan& operator=(mdspan&& rhs) = default;

// 23.7.3.6.3, members
template<class... OtherIndexTypes>
constexpr reference operator[](OtherIndexTypes... indices) const;
template<class OtherIndexType>
constexpr reference operator[](span<OtherIndexType, rank()> indices) const;
template<class OtherIndexType>
constexpr reference operator[](const array<OtherIndexType, rank()>& indices) const;

template<class... OtherIndexTypes>
constexpr reference
 at(OtherIndexTypes... indices) const; // freestanding-deleted
template<class OtherIndexType>
constexpr reference
 at(span<OtherIndexType, rank()> indices) const; // freestanding-deleted
template<class OtherIndexType>
constexpr reference
 at(const array<OtherIndexType, rank()>& indices) const; // freestanding-deleted

constexpr size_type size() const noexcept;
constexpr bool empty() const noexcept;

friend constexpr void swap(mdspan& x, mdspan& y) noexcept;

constexpr const extents_type& extents() const noexcept { return map_.extents(); }
constexpr const data_handle_type& data_handle() const noexcept { return ptr_; }
constexpr const mapping_type& mapping() const noexcept { return map_; }
constexpr const accessor_type& accessor() const noexcept { return acc_; }

static constexpr bool is_always_unique()
{ return mapping_type::is_always_unique(); }
static constexpr bool is_always_exhaustive()
{ return mapping_type::is_always_exhaustive(); }

```

```

static constexpr bool is_always_strided()
{ return mapping_type::is_always_strided(); }

constexpr bool is_unique() const
{ return map_.is_unique(); }
constexpr bool is_exhaustive() const
{ return map_.is_exhaustive(); }
constexpr bool is_strided() const
{ return map_.is_strided(); }
constexpr index_type stride(rank_type r) const
{ return map_.stride(r); }

private:
 accessor_type acc_; // exposition only
 mapping_type map_; // exposition only
 data_handle_type ptr_; // exposition only
};

template<class CArray>
 requires (is_array_v<CArray> && rank_v<CArray> == 1)
 mdspan(CArray&)
 -> mdspan<remove_all_extents_t<CArray>, extents<size_t, extent_v<CArray, 0>>>;

template<class Pointer>
 requires (is_pointer_v<remove_reference_t<Pointer>>)
 mdspan(Pointer&&)
 -> mdspan<remove_pointer_t<remove_reference_t<Pointer>>, extents<size_t>>;

template<class ElementType, class... Integrals>
 requires ((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integrals) > 0)
 explicit mdspan(ElementType*, Integrals...)
 -> mdspan<ElementType, extents<size_t, maybe-static-ext<Integrals>...>>;

template<class ElementType, class OtherIndexType, size_t N>
 mdspan(ElementType*, span<OtherIndexType, N>)
 -> mdspan<ElementType, dextents<size_t, N>>;

template<class ElementType, class OtherIndexType, size_t N>
 mdspan(ElementType*, const array<OtherIndexType, N>&)
 -> mdspan<ElementType, dextents<size_t, N>>;

template<class ElementType, class IndexType, size_t... ExtentsPack>
 mdspan(ElementType*, const extents<IndexType, ExtentsPack...>&)
 -> mdspan<ElementType, extents<IndexType, ExtentsPack...>>;

template<class ElementType, class MappingType>
 mdspan(ElementType*, const MappingType&)
 -> mdspan<ElementType, typename MappingType::extents_type,
 typename MappingType::layout_type>;

template<class MappingType, class AccessorType>
 mdspan(typename AccessorType::data_handle_type, const MappingType&,
 const AccessorType&)
 -> mdspan<typename AccessorType::element_type, typename MappingType::extents_type,
 typename MappingType::layout_type, AccessorType>;
}

```

## 2 Mandates:

- (2.1) — `ElementType` is a complete object type that is neither an abstract class type nor an array type,
- (2.2) — `Extents` is a specialization of `extents`, and
- (2.3) — `is_same_v<ElementType, typename AccessorPolicy::element_type>` is true.

<sup>3</sup> `LayoutPolicy` shall meet the layout mapping policy requirements (23.7.3.4.3), and `AccessorPolicy` shall meet the accessor policy requirements (23.7.3.5.2).

<sup>4</sup> Each specialization MDS of `mdspan` models copyable and

- (4.1) — `is_nothrow_move_constructible_v<MDS>` is true,
- (4.2) — `is_nothrow_move_assignable_v<MDS>` is true, and
- (4.3) — `is_nothrow_swappable_v<MDS>` is true.

<sup>5</sup> A specialization of `mdspan` is a trivially copyable type if its `accessor_type`, `mapping_type`, and `data_handle_type` are trivially copyable types.

### 23.7.3.6.2 Constructors

[`mdspan.mdspan.cons`]

```
constexpr mdspan();
```

<sup>1</sup> *Constraints:*

- (1.1) — `rank_dynamic() > 0` is true.
- (1.2) — `is_default_constructible_v<data_handle_type>` is true.
- (1.3) — `is_default_constructible_v<mapping_type>` is true.
- (1.4) — `is_default_constructible_v<accessor_type>` is true.

<sup>2</sup> *Preconditions:* `[0, map_.required_span_size())` is an accessible range of `ptr_` and `acc_` for the values of `map_` and `acc_` after the invocation of this constructor.

<sup>3</sup> *Effects:* Value-initializes `ptr_`, `map_`, and `acc_`.

```
template<class... OtherIndexTypes>
```

```
constexpr explicit mdspan(data_handle_type p, OtherIndexTypes... exts);
```

<sup>4</sup> Let `N` be `sizeof...(OtherIndexTypes)`.

<sup>5</sup> *Constraints:*

- (5.1) — `(is_convertible_v<OtherIndexTypes, index_type> && ...)` is true,
- (5.2) — `(is_nothrow_constructible_v<index_type, OtherIndexTypes> && ...)` is true,
- (5.3) — `N == rank() || N == rank_dynamic()` is true,
- (5.4) — `is_constructible_v<mapping_type, extents_type>` is true, and
- (5.5) — `is_default_constructible_v<accessor_type>` is true.

<sup>6</sup> *Preconditions:* `[0, map_.required_span_size())` is an accessible range of `p` and `acc_` for the values of `map_` and `acc_` after the invocation of this constructor.

<sup>7</sup> *Effects:*

- (7.1) — Direct-non-list-initializes `ptr_` with `std::move(p)`,
- (7.2) — direct-non-list-initializes `map_` with `extents_type(static_cast<index_type>(std::move(exts))...)`, and
- (7.3) — value-initializes `acc_`.

```
template<class OtherIndexType, size_t N>
```

```
constexpr explicit(N != rank_dynamic())
```

```
mdspan(data_handle_type p, span<OtherIndexType, N> exts);
```

```
template<class OtherIndexType, size_t N>
```

```
constexpr explicit(N != rank_dynamic())
```

```
mdspan(data_handle_type p, const array<OtherIndexType, N>& exts);
```

<sup>8</sup> *Constraints:*

- (8.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true,
- (8.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is true,
- (8.3) — `N == rank() || N == rank_dynamic()` is true,
- (8.4) — `is_constructible_v<mapping_type, extents_type>` is true, and
- (8.5) — `is_default_constructible_v<accessor_type>` is true.

9 *Preconditions:* `[0, map_.required_span_size())` is an accessible range of `p` and `acc_` for the values of `map_` and `acc_` after the invocation of this constructor.

10 *Effects:*

- (10.1) — Direct-non-list-initializes `ptr_` with `std::move(p)`,
- (10.2) — direct-non-list-initializes `map_` with `extents_type(exts)`, and
- (10.3) — value-initializes `acc_`.

```
constexpr mdspan(data_handle_type p, const extents_type& ext);
```

11 *Constraints:*

- (11.1) — `is_constructible_v<mapping_type, const extents_type&>` is true, and
- (11.2) — `is_default_constructible_v<accessor_type>` is true.

12 *Preconditions:* `[0, map_.required_span_size())` is an accessible range of `p` and `acc_` for the values of `map_` and `acc_` after the invocation of this constructor.

13 *Effects:*

- (13.1) — Direct-non-list-initializes `ptr_` with `std::move(p)`,
- (13.2) — direct-non-list-initializes `map_` with `ext`, and
- (13.3) — value-initializes `acc_`.

```
constexpr mdspan(data_handle_type p, const mapping_type& m);
```

14 *Constraints:* `is_default_constructible_v<accessor_type>` is true.

15 *Preconditions:* `[0, m.required_span_size())` is an accessible range of `p` and `acc_` for the value of `acc_` after the invocation of this constructor.

16 *Effects:*

- (16.1) — Direct-non-list-initializes `ptr_` with `std::move(p)`,
- (16.2) — direct-non-list-initializes `map_` with `m`, and
- (16.3) — value-initializes `acc_`.

```
constexpr mdspan(data_handle_type p, const mapping_type& m, const accessor_type& a);
```

17 *Preconditions:* `[0, m.required_span_size())` is an accessible range of `p` and `a`.

18 *Effects:*

- (18.1) — Direct-non-list-initializes `ptr_` with `std::move(p)`,
- (18.2) — direct-non-list-initializes `map_` with `m`, and
- (18.3) — direct-non-list-initializes `acc_` with `a`.

```
template<class OtherElementType, class OtherExtents,
 class OtherLayoutPolicy, class OtherAccessor>
constexpr explicit(see below)
mdspan(const mdspan<OtherElementType, OtherExtents,
 OtherLayoutPolicy, OtherAccessor>& other);
```

19 *Constraints:*

- (19.1) — `is_constructible_v<mapping_type, const OtherLayoutPolicy::template mapping<OtherExtents>&>` is true, and
- (19.2) — `is_constructible_v<accessor_type, const OtherAccessor&>` is true.

20 *Mandates:*

- (20.1) — `is_constructible_v<data_handle_type, const OtherAccessor::data_handle_type&>` is true, and
- (20.2) — `is_constructible_v<extents_type, OtherExtents>` is true.

21 *Preconditions:* `[0, map_.required_span_size())` is an accessible range of `ptr_` and `acc_` for values of `ptr_`, `map_`, and `acc_` after the invocation of this constructor.

22 *Hardened preconditions:* For each rank index *r* of *extents\_type*, *static\_extent(r) == dynamic\_extent || static\_extent(r) == other.extent(r)* is true.

23 *Effects:*

- (23.1) — Direct-non-list-initializes *ptr\_* with *other.ptr\_*,
- (23.2) — direct-non-list-initializes *map\_* with *other.map\_*, and
- (23.3) — direct-non-list-initializes *acc\_* with *other.acc\_*.

24 *Remarks:* The expression inside *explicit* is equivalent to:

```
!is_convertible_v<const OtherLayoutPolicy::template mapping<OtherExtents>&, mapping_type>
|| !is_convertible_v<const OtherAccessor&, accessor_type>
```

### 23.7.3.6.3 Members

[mdspan.mdspan.members]

```
template<class... OtherIndexTypes>
constexpr reference operator[](OtherIndexTypes... indices) const;
```

1 *Constraints:*

- (1.1) — (*is\_convertible\_v<OtherIndexTypes, index\_type> && ...*) is true,
- (1.2) — (*is\_nothrow\_constructible\_v<index\_type, OtherIndexTypes> && ...*) is true, and
- (1.3) — *sizeof...(OtherIndexTypes) == rank()* is true.

2 Let *I* be *extents\_type::index-cast* (*std::move*(*indices*)).

3 *Hardened preconditions:* *I* is a multidimensional index in *extents()*.

[*Note 1:* This implies that *map\_(I) < map\_.required\_span\_size()* is true. — *end note*]

4 *Effects:* Equivalent to:

```
return acc_.access(ptr_, map_(static_cast<index_type>(std::move(indices))...));
```

```
template<class OtherIndexType>
constexpr reference operator[](span<OtherIndexType, rank()> indices) const;
template<class OtherIndexType>
constexpr reference operator[](const array<OtherIndexType, rank()>& indices) const;
```

5 *Constraints:*

- (5.1) — *is\_convertible\_v<const OtherIndexType&, index\_type>* is true, and
- (5.2) — *is\_nothrow\_constructible\_v<index\_type, const OtherIndexType&>* is true.

6 *Effects:* Let *P* be a parameter pack such that

```
is_same_v<make_index_sequence<rank()>, index_sequence<P...>>
```

is true. Equivalent to:

```
return operator[](extents_type::index-cast(as_const(indices[P]))...);
```

```
template<class... OtherIndexTypes>
constexpr reference at(OtherIndexTypes... indices) const;
```

7 *Constraints:*

- (7.1) — (*is\_convertible\_v<OtherIndexTypes, index\_type> && ...*) is true,
- (7.2) — (*is\_nothrow\_constructible\_v<index\_type, OtherIndexTypes> && ...*) is true, and
- (7.3) — *sizeof...(OtherIndexTypes) == rank()* is true.

8 Let *I* be *extents\_type::index-cast* (*std::move*(*indices*)).

9 *Returns:* *(\*this)[I...]*.

10 *Throws:* *out\_of\_range* if *I* is not a multidimensional index in *extents()*.

```
template<class OtherIndexType>
constexpr reference at(span<OtherIndexType, rank()> indices) const;
template<class OtherIndexType>
constexpr reference at(const array<OtherIndexType, rank()>& indices) const;
```

11 *Constraints:*



(11.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true, and  
 (11.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is true.

12 *Effects:* Let *P* be a parameter pack such that  
       `is_same_v<make_index_sequence<rank()>, index_sequence<P...>>`  
 is true. Equivalent to:  
       `return at(extents_type::index-cast(as_const(indices[P]))...);`

`constexpr size_type size() const noexcept;`

13 *Preconditions:* The size of the multidimensional index space `extents()` is representable as a value of type `size_type` (6.9.3).

14 *Returns:* `extents().fwd-prod-of-extents(rank())`.

`constexpr bool empty() const noexcept;`

15 *Returns:* true if the size of the multidimensional index space `extents()` is 0, otherwise false.

`friend constexpr void swap(mdspan& x, mdspan& y) noexcept;`

16 *Effects:* Equivalent to:  
       `swap(x.ptr_, y.ptr_);`  
       `swap(x.map_, y.map_);`  
       `swap(x.acc_, y.acc_);`

### 23.7.3.7 submdspan

[mdspan.sub]

#### 23.7.3.7.1 Overview

[mdspan.sub.overview]

- 1 The `submdspan` facilities create a new `mdspan` viewing a subset of elements of an existing input `mdspan`. The subset viewed by the created `mdspan` is determined by the `SliceSpecifier` arguments.
- 2 Given a signed or unsigned integer type `IndexType`, a type *S* is a *submdspan slice type for IndexType* if at least one of the following holds:
  - (2.1) — `is_convertible_v<S, full_extent_t>` is true;
  - (2.2) — `is_convertible_v<S, IndexType>` is true;
  - (2.3) — *S* is a specialization of `extent_slice` and `is_convertible_v<X, IndexType>` is true for *X* denoting `S::offset_type`, `S::extent_type`, and `S::stride_type`;
  - (2.4) — *S* is a specialization of `range_slice` and `is_convertible_v<X, IndexType>` is true for *X* denoting type of `S::first`, `S::last`, and `S::stride` members; or
  - (2.5) — all of the following hold:
    - (2.5.1) — the declaration `auto [...ls] = std::move(s);` is well-formed for some object *s* of type *S*,
    - (2.5.2) — `sizeof...(ls)` is equal to 2, and
    - (2.5.3) — `(is_convertible_v<decltype(std::move(ls)), IndexType> && ...)` is true.
- 3 Given a signed or unsigned integer type `IndexType`, a type *S* is a *canonical submdspan index type for IndexType* if *S* is either `IndexType` or `constant_wrapper<v>` for some value *v* of type `IndexType`, such that *v* is greater than or equal to zero.
- 4 Given a signed or unsigned integer type `IndexType`, a type *S* is a *canonical submdspan slice type for IndexType* if exactly one of the following is true:
  - (4.1) — *S* is `full_extent_t`;
  - (4.2) — *S* is a canonical `submdspan` index type for `IndexType`; or
  - (4.3) — *S* is a specialization of `extent_slice` where all of the following hold:
    - (4.3.1) — `S::offset_type`, `S::extent_type`, and `S::stride_type` are all canonical `submdspan` index types for `IndexType`; and
    - (4.3.2) — if `S::stride_type` and `S::extent_type` are both specializations of `constant_wrapper`, then `S::stride_type::value` is greater than zero.
- 5 A type *S* is a *collapsing slice type* if it is neither `full_extent_t` nor a specialization of `extent_slice`.

[*Note 1:* Each collapsing slice type in `submdspan_mapping`'s parameter pack of slice specifier types reduces the rank of the result of `submdspan_mapping` by one. — *end note*]

<sup>6</sup> A type `S` is a *unit-stride slice type* if

- (6.1) — `S` is a specialization of `extent_slice` where `S::stride_type` is a specialization of `constant_wrapper` and `S::stride_type::value` is equal to 1, or
- (6.2) — `S` denotes `full_extent_t`.

<sup>7</sup> Given an object `e` of type `E` that is a specialization of `extents`, and an object `s` of type `S` that is a canonical `submdspan` slice type for `E::index_type`, the *submdspan slice range of `s` for the  $k^{\text{th}}$  extent of `e`* is:

- (7.1) — `[0, e.extent(k))`, if `S` is `full_extent_t`;
- (7.2) — `[E::index_type(s.offset), E::index_type(s.offset))`, if `S` is a specialization of `extent_slice` and `E::index_type(s.extent)` is zero; otherwise
- (7.3) — `[E::index_type(s.offset), E::index_type(s.offset + 1 + (s.extent - 1) * s.stride))`, if `S` is a specialization of `extent_slice`; otherwise
- (7.4) — `[E::index_type(s), E::index_type(s) + 1)`

<sup>8</sup> Given a type `E` that is a specialization of `extents`, a type `S` is a *valid submdspan slice type for the  $k^{\text{th}}$  extent of `E`* if `S` is a canonical slice type for `E::index_type`, and for  $x$  equal to `E::static_extent(k)`, either  $x$  is equal to `dynamic_extent`; or

- (8.1) — if `S` is a specialization of `extent_slice`, then
  - (8.1.1) —  $o$  is less than or equal to  $x$ ;
  - (8.1.2) —  $e$  is less than or equal to  $x$ ;
  - (8.1.3) — if  $e$  is greater than one, then  $t$  is greater than zero,
  - (8.1.4) — if  $e$  is greater than zero, then  $o + 1 + (e - 1) * t$  is less than or equal to  $x$ ,

where

- (8.1.5) —  $o$  is the value of `S::offset_type::value` if `S::offset_type` is a specialization of `constant_wrapper` and 0 otherwise,
- (8.1.6) —  $e$  is the value of `S::extent_type::value` if `S::extent_type` is a specialization of `constant_wrapper` and 0 otherwise,
- (8.1.7) —  $t$  is the value of `S::stride_type::value` if `S::stride_type` is a specialization of `constant_wrapper` and 1 otherwise; and
- (8.2) — if `S` is a specialization of `constant_wrapper`, then `S::value` is less than  $x$ .

<sup>9</sup> Given an object `e` of type `E` that is a specialization of `extents` and an object `s` of type `S`, `s` is a *valid submdspan slice for the  $k^{\text{th}}$  extent of `e`* if

- (9.1) — `S` is a valid `submdspan` slice type for the  $k^{\text{th}}$  extent of `E`;
- (9.2) — the  $k^{\text{th}}$  interval of `e` contains the `submdspan` slice range of `s` for the  $k^{\text{th}}$  extent of `e`; and
- (9.3) — if `S` is a specialization of `extent_slice`, then
  - (9.3.1) — `s.extent` is greater than or equal to zero, and
  - (9.3.2) — either `s.extent` is less than two or `s.stride` is greater than zero.

### 23.7.3.7.2 Range slices

[`mdspan.sub.range.slices`]

<sup>1</sup> `extent_slice` and `range_slice` represent a set of `extent` regularly spaced integer indices. The indices start at `offset` and `first`, respectively, and increase by increments of `stride`.

```
namespace std {
 template<class OffsetType, class ExtentType, class StrideType>
 struct extent_slice {
 using offset_type = OffsetType;
 using extent_type = ExtentType;
 using stride_type = StrideType;

 [[no_unique_address]] offset_type offset{};
 [[no_unique_address]] extent_type extent{};
 };
}
```

```
[[no_unique_address]] stride_type stride{};
};
```

```
template<class FirstType, class LastType, class StrideType = constant_wrapper<1zu>>
struct range_slice {
 [[no_unique_address]] FirstType first{};
 [[no_unique_address]] LastType last{};
 [[no_unique_address]] StrideType stride{};
};
}
```

<sup>2</sup> `extent_slice` and `range_slice` have the data members and special members specified above. They have no base classes or members other than those specified.

<sup>3</sup> *Mandates:* `OffsetType`, `ExtentType`, `FirstType`, `LastType`, and `StrideType` are signed or unsigned integer types, or model *integral-constant-like*.

[Note 1: Both `extent_slice{.offset = 1, .extent = 4, .stride = 3}` and `range_slice{.first = 1, .last = 11, .stride = 3}` indicate the indices 1, 4, 7, and 10. Indices are selected from the half-open interval  $[1, 1 + 10)$ . — end note]

### 23.7.3.7.3 submdspan\_mapping\_result

[mdspan.sub.map.result]

<sup>1</sup> Specializations of `submdspan_mapping_result` are returned by overloads of `submdspan_mapping`.

```
namespace std {
 template<class LayoutMapping>
 struct submdspan_mapping_result {
 [[no_unique_address]] LayoutMapping mapping = LayoutMapping();
 size_t offset{};
 };
}
```

<sup>2</sup> `submdspan_mapping_result` has the data members and special members specified above. It has no base classes or members other than those specified.

<sup>3</sup> `LayoutMapping` shall meet the layout mapping requirements (23.7.3.4.3).

### 23.7.3.7.4 Exposition-only helpers

[mdspan.sub.helpers]

<sup>1</sup> For a pack `p` and an integer `i`, let `MAP_RANK(p, i)` be the number of elements `p...[j]` for  $0 \leq j < i$  whose types are not collapsing slice types.

```
template<class T>
 concept is_extent_slice = see below;
```

<sup>2</sup> The concept `is_extent_slice<T>` is satisfied and modeled if and only if `T` is a specialization of `extent_slice`.

```
template<class T>
 concept is_range_slice = see below;
```

<sup>3</sup> The concept `is_range_slice<T>` is satisfied and modeled if and only if `T` is a specialization of `range_slice`.

```
template<class IndexType, class S>
 constexpr auto canonical_index(S s);
```

<sup>4</sup> *Mandates:* If `S` models *integral-constant-like*, then `extents<IndexType>::index-cast(S::value)` is representable as a value of type `IndexType`.

<sup>5</sup> *Preconditions:* `extents<IndexType>::index-cast(std::move(s))` is representable as a value of type `IndexType`.

<sup>6</sup> *Effects:* Equivalent to:

(6.1) — return `cw<IndexType>(S::value)`; if `S` models *integral-constant-like*;

(6.2) — return `IndexType(std::move(s))`; otherwise.

```
template<class IndexType, class OffsetType, class SpanType, class... StrideTypes>
constexpr auto canonical-range-slice(OffsetType offset, SpanType span, StrideTypes... strides);
```

7 Let

- (7.1) — StrideType denote constant\_wrapper<IndexType(1)> if StrideTypes is an empty pack or SpanType denotes constant\_wrapper<IndexType(0)>, otherwise StrideTypes...[0];
- (7.2) — stride be StrideType() if StrideType is a specialization of constant\_wrapper, otherwise IndexType(1) if span == 0 is true, otherwise strides...[0];
- (7.3) — extent-value be 1 + (span - 1) / stride if span != 0 is true, and 0 otherwise; and
- (7.4) — let extent be cw<IndexType(extent-value)> if both SpanType and StrideType are specializations of constant\_wrapper, and IndexType(extent-value) otherwise.

8 *Mandates:* sizeof...(StrideTypes) <= 1 is true, and if StrideType is a specialization of constant\_wrapper, then StrideType::value > 0 is true.

9 *Preconditions:* stride > 0 is true.

10 *Returns:* extent\_slice{.offset = offset, .extent = extent, .stride = stride}.

```
template<class IndexType, class S>
constexpr auto canonical-slice(S s);
```

11 *Mandates:* S is a submdspan slice type for IndexType.

12 *Effects:* Equivalent to:

```
if constexpr (is_convertible_v<S, full_extent_t>) {
 return static_cast<full_extent_t>(std::move(s));
} else if constexpr (is_convertible_v<S, IndexType>) {
 return canonical-index<IndexType>(std::move(s));
} else if constexpr (is_extent-slice<S>) {
 return extent_slice{
 .offset = canonical-index<IndexType>(std::move(s.offset)),
 .extent = canonical-index<IndexType>(std::move(s.extent)),
 .stride = canonical-index<IndexType>(std::move(s.stride))
 };
} else if constexpr (is-range-slice<S>) {
 auto c_first = canonical-index<IndexType>(std::move(s.first));
 auto c_last = canonical-index<IndexType>(std::move(s.last));
 return canonical-slice-range<IndexType>(
 c_first,
 canonical-index<IndexType>(c_last - c_first),
 canonical-index<IndexType>(std::move(s.stride)));
} else {
 auto [s_first, s_last] = std::move(s);
 auto c_first = canonical-index<IndexType>(std::move(s_first));
 auto c_last = canonical-index<IndexType>(std::move(s_last));
 return canonical-slice-range<IndexType>(
 c_first,
 canonical-index<IndexType>(c_last - c_first));
}
```

### 23.7.3.7.5 submdspan slice canonicalization

[mdspan.sub.canonical]

```
template<class IndexType, size_t... Extents, class... SliceSpecifiers>
constexpr auto canonical_slices(const extents<IndexType, Extents...>& src,
 SliceSpecifiers... slices);
```

1 *Constraints:* sizeof...(SliceSpecifiers) equals sizeof...(Extents).

2 *Mandates:* For each rank index  $k$  of src:

- (2.1) — SliceSpecifiers...[ $k$ ] is a submdspan slice type for IndexType, and
- (2.2) — decltype(canonical-slice<IndexType>(slices...[ $k$ ])) is a valid submdspan slice type for the  $k^{\text{th}}$  extent of extents<IndexType, Extents...>.

3 *Preconditions:* For each rank index  $k$  of `src`, `canonical-slice<IndexType>(slices...[k])` is a valid submdspan slice for the  $k^{\text{th}}$  extent of `src`.

4 *Returns:* `make_tuple(canonical-slice<IndexType>(slices)...) .`

### 23.7.3.7.6 subextents function

[mdspan.sub.extents]

```
template<class IndexType, size_t... Extents, class... SliceSpecifiers>
constexpr auto subextents(const extents<IndexType, Extents...>& src,
 SliceSpecifiers... raw_slices);
```

1 Let `slices` be the pack introduced by the following declaration:

```
auto [...slices] = canonical_slices(src, raw_slices...);
```

2 *Constraints:* `sizeof...(SliceSpecifiers)` equals `sizeof...(Extents)`.

3 *Mandates:* For each rank index  $k$  of `src`:

(3.1) — `SliceSpecifiers...[k]` is a submdspan slice type for `IndexType`, and

(3.2) — `decltype(slices...[k])` is a valid submdspan slice type for the  $k^{\text{th}}$  extent of `extents<IndexType, Extents...>`.

4 *Preconditions:* For each rank index  $k$  of `src`, `slices...[k]` is a valid submdspan slice for the  $k^{\text{th}}$  extent of `src`.

5 Let `SubExtents` be a specialization of `extents` such that:

(5.1) — `SubExtents::rank()` equals `MAP_RANK(slices, Extents::rank())`; and

(5.2) — for each rank index  $k$  of `extents<IndexType, Extents...>` such that the type of `slices...[k]` is not a collapsing slice type, `SubExtents::static_extent(MAP_RANK(slices, k))` equals the following, where  $\Sigma_k$  denotes the type of `slices...[k]`:

(5.2.1) — `Extents::static_extent(k)` if  $\Sigma_k$  denotes `full_extent_t`; otherwise

(5.2.2) —  $\Sigma_k::\text{extent\_type}::\text{value}$  if  $\Sigma_k$  is a specialization of `extent_slice` whose `extent_type` denotes a specialization of `constant_wrapper`;

(5.2.3) — otherwise, `dynamic_extent`.

6 *Returns:* A value `ext` of type `SubExtents` such that for each rank index  $k$  of `extents<IndexType, Extents...>`, where the type of `slices...[k]` is not a collapsing slice type, `ext.extent(MAP_RANK(slices, k))` equals the following, where  $\sigma_k$  denotes `slices...[k]`:

(6.1) —  $\sigma_k.\text{extent}$  if the type of  $\sigma_k$  is a specialization of `extent_slice`,

(6.2) — otherwise,  $U - L$ , where  $[L, U)$  is the submdspan slice range of  $\sigma_k$  for the  $k^{\text{th}}$  extent of `src`.

### 23.7.3.7.7 Specializations of submdspan\_mapping

[mdspan.sub.map]

#### 23.7.3.7.7.1 Sliceable layout mapping requirements

[mdspan.sub.map.sliceable]

1 Let:

(1.1) — `M` denote a layout mapping class;

(1.2) — `IT` denote `M::extents_type::index_type`;

(1.3) — `m` denote a value of type (possibly `const`) `M`;

(1.4) — `M_rank` be equal to `M::extents_type::rank()`;

(1.5) — `valid_slices` denote a pack of (possibly `const`) objects for which `sizeof...(valid_slices) == M_rank` is `true` and, for each rank index  $i$  of `m.extents()`, `valid_slices...[i]` is a valid submdspan slice for the  $i^{\text{th}}$  extent of `m.extents()`;

(1.6) — `invalid_slices` denote a pack of objects for which `sizeof...(invalid_slices) == M_rank` is `true` and there exists an integer  $k$  such that the cv-unqualified type of `invalid_slices...[k]` is none of the following:

(1.6.1) — `IT`,

(1.6.2) — `full_extent_t`,

(1.6.3) — a specialization of `constant_wrapper`, or

(1.6.4) — a specialization of `extent_slice`.

<sup>2</sup> For the purpose of this section, the meaning of `submdspan_mapping` is established as if by performing argument-dependent lookup only (6.5.4).

<sup>3</sup> A type `M` meets the *sliceable layout mapping requirements* if

- (3.1) — `M` meets the layout mapping requirements (23.7.3.4.3),
- (3.2) — the expression `submdspan_mapping(m, invalid_slices...)` is ill-formed, and
- (3.3) — the following expression is well-formed and has the specified semantics:

`submdspan_mapping(m, valid_slices...)`

<sup>4</sup> *Result:* A type `SMR` that is a specialization of type `submdspan_mapping_result<SM>` for some type `SM` such that

- (4.1) — `SM` meets the layout mapping requirements (23.7.3.4.3),
- (4.2) — `SM::extents_type` is a specialization of `extents`,
- (4.3) — `SM::extents_type::rank()` equals `MAP_RANK(valid_slices, M_rank)`, and
- (4.4) — `SM::extents_type::index_type` denotes `IT`.

<sup>5</sup> *Returns:* An object `smr` of type `SMR` such that

- (5.1) — `smr.mapping.extents() == subextents(m.extents(), valid_slices...)` is true; and
- (5.2) — for each integer pack `i` which is a multidimensional index in `smr.mapping.extents()`, `smr.mapping(i...) + smr.offset == m(j)` is true, where `j` is an integer pack such that
  - (5.2.1) — `sizeof...(j)` is equal to `M_rank`; and
  - (5.2.2) — for each rank index  $\rho$  of `m.extents()`, `j...[ $\rho$ ]` is equal to the sum of
    - (5.2.2.1) — the lower bound of the `submdspan` slice range of `valid_slices...[ $\rho$ ]` for extent  $\rho$  of `m.extents()`, and
    - (5.2.2.2) — zero if the type of `valid_slices...[ $\rho$ ]` is a collapsing slice type, `i...[MAP_RANK(valid_slices,  $\rho$ )]` otherwise.

`template<class LayoutMapping>`

`concept sliceable-mapping = see below;`

<sup>6</sup> Let `lm` be an object of type `LayoutMapping` and let `fe` denote a pack of objects of type `full_extent_t` for which `sizeof...(fe) == LayoutMapping::extents_type::rank()` is true. A type `LayoutMapping` satisfies *sliceable-mapping* if

- (6.1) — the expression `submdspan_mapping(lm, fe...)` is well-formed when treated as an unevaluated operand, and
- (6.2) — the type of that expression is a specialization of `submdspan_mapping_result`.

<sup>7</sup> A type `LayoutMapping` models *sliceable-mapping* if `LayoutMapping` meets the sliceable layout mapping requirements.

#### 23.7.3.7.7.2 Common

[mdspan.sub.map.common]

<sup>1</sup> The following elements apply to all functions in 23.7.3.7.7.

<sup>2</sup> *Constraints:* `sizeof...(SliceSpecifiers)` equals `extents_type::rank()`.

<sup>3</sup> *Mandates:* For each rank index  $k$  of `extents()`, `SliceSpecifiers...[ $k$ ]` is a valid `submdspan` slice type for the  $k^{\text{th}}$  extent of `Extents`.

<sup>4</sup> *Preconditions:* For each rank index  $k$  of `extents()`, `slices...[ $k$ ]` is a valid slice for the  $k^{\text{th}}$  extent of `extents()`.

<sup>5</sup> Let `sub_ext` be the result of `subextents(extents(), slices...)` and let `SubExtents` be `decltype(sub_ext)`.

<sup>6</sup> Let `sub_strides` be an `array<SubExtents::index_type, SubExtents::rank()>` such that for each rank index  $k$  of `extents()` for which the type of `slices...[ $k$ ]` is not a collapsing slice type, `sub_strides[MAP_RANK(slices,  $k$ )]` equals:

- (6.1) — `stride(k) * s.stride` if the type of `s` is a specialization of `extent_slice` and `s.extent > 1` is true, where `s` is `slices...[k]`;
- (6.2) — otherwise, `stride(k)`.
- 7 Let `ls` be a pack of values of `index_type`, where the  $\rho^{\text{th}}$  element equals the lower bound of the `submdspan` slice range of `slices...[ $\rho$ ]` for extent  $\rho$  of `extents()`.
- 8 If `ls...[k]` equals `extents().extent(k)` for any rank index  $k$  of `extents()`, then let `offset` be a value of type `size_t` equal to `required_span_size()`. Otherwise, let `offset` be a value of type `size_t` equal to `operator()(ls...)`.

### 23.7.3.7.7.3 layout\_left specialization of submdspan\_mapping

[mdspan.sub.map.left]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left::mapping<Extents>::submdspan-mapping-impl (
 SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank() == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if `SubExtents::rank() == 0` is true;
- (1.3) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if
- (1.3.1) — for each  $k$  in the range  $[0, \text{SubExtents::rank()} - 1]$ , `SliceSpecifiers...[k]` denotes `full_extent_t`; and
- (1.3.2) — for  $k$  equal to `SubExtents::rank() - 1`, `SliceSpecifiers...[k]` is a unit-stride slice type;
- [Note 1: If the above conditions are true, all `SliceSpecifiers...[k]` with  $k$  larger than `SubExtents::rank() - 1` are convertible to `index_type`. — end note]
- (1.4) — otherwise,
- `submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u + 1)), offset}`
- if for a value  $u$  for which  $u+1$  is the smallest value  $p$  larger than zero for which `SliceSpecifiers...[p]` is a unit-stride slice type, the following conditions are met:
- (1.4.1) — `SliceSpecifiers...[0]` is a unit-stride slice type; and
- (1.4.2) — for each  $k$  in the range  $[u + 1, u + \text{SubExtents::rank()} - 1]$ , `SliceSpecifiers...[k]` denotes `full_extent_t`; and
- (1.4.3) — for  $k$  equal to  $u + \text{SubExtents::rank()} - 1$ , `SliceSpecifiers...[k]` is a unit-stride slice type;
- and where `S_static` is:
- (1.4.4) — `dynamic_extent`, if `static_extent(k)` is `dynamic_extent` for any  $k$  in the range  $[0, u + 1]$ ,
- (1.4.5) — otherwise, the product of all values `static_extent(k)` for  $k$  in the range  $[0, u + 1]$ ;
- (1.5) — otherwise,
- `submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

### 23.7.3.7.7.4 layout\_right specialization of submdspan\_mapping

[mdspan.sub.map.right]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right::mapping<Extents>::submdspan-mapping-impl (
 SliceSpecifiers... slices) const -> see below;
```

1 *Returns:*

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank() == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if `SubExtents::rank() == 0` is true;

- (1.3) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if
- (1.3.1) — for each  $k$  in the range  $[rank\_ - SubExtents::rank() + 1, rank\_]$ , `SliceSpecifiers...[k]` denotes `full_extent_t`; and
- (1.3.2) — for  $k$  equal to  $rank\_ - SubExtents::rank()$ , `SliceSpecifiers...[k]` is a unit-stride slice type;  

[Note 1: If the above conditions are true, all `SliceSpecifiers...[k]` with  $k < rank\_ - SubExtents::rank()$  are convertible to `index_type`. — end note]
- (1.4) — otherwise,  

`submdspan_mapping_result{layout_right_padded<S_static>::mapping(sub_ext, stride(rank\_ - u - 2)), offset}`

if for a value  $u$  for which  $rank\_ - u - 2$  is the largest value  $p$  smaller than  $rank\_ - 1$  for which `SliceSpecifiers...[p]` is a unit-stride slice type, the following conditions are met:

  - (1.4.1) — for  $k$  equal to  $rank\_ - 1$ , `SliceSpecifiers...[k]` is a unit-stride slice type; and
  - (1.4.2) — for each  $k$  in the range  $[rank\_ - SubExtents::rank() - u + 1, rank\_ - u - 1]$ , `SliceSpecifiers...[p]` denotes `full_extent_t`; and
  - (1.4.3) — for  $k$  equal to  $rank\_ - SubExtents::rank() - u$ , `SliceSpecifiers...[k]` is a unit-stride slice type;

and where `S_static` is:

  - (1.4.4) — `dynamic_extent`, if `static_extent(k)` is `dynamic_extent` for any  $k$  in the range  $[rank\_ - u - 1, rank\_]$ ,
  - (1.4.5) — otherwise, the product of all values `static_extent(k)` for  $k$  in the range  $[rank\_ - u - 1, rank\_]$ ;
- (1.5) — otherwise,  

`submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

#### 23.7.3.7.7.5 layout\_stride specialization of submdspan\_mapping [mdspan.sub.map.stride]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_stride::mapping<Extents>::submdspan-mapping-impl(
 SliceSpecifiers... slices) const -> see below;
```

1 Returns:

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank() == 0` is true;
- (1.2) — otherwise,  

`submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

#### 23.7.3.7.7.6 layout\_left\_padded specialization of submdspan\_mapping[mdspan.sub.map.leftpad]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_left_padded::mapping<Extents>::submdspan-mapping-impl(
 SliceSpecifiers... slices) const -> see below;
```

1 Returns:

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `Extents::rank() == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if  $rank\_ == 1$  is true or `SubExtents::rank() == 0` is true;
- (1.3) — otherwise, `submdspan_mapping_result{layout_left::mapping(sub_ext), offset}`, if
  - (1.3.1) — `SubExtents::rank() == 1` is true and
  - (1.3.2) — `SliceSpecifiers...[0]` is a unit-stride slice type;
- (1.4) — otherwise,  

`submdspan_mapping_result{layout_left_padded<S_static>::mapping(sub_ext, stride(u + 1)), offset}`



if for a value  $u$  for which  $u + 1$  is the smallest value  $p$  larger than zero for which `SliceSpecifiers...[p]` is a unit-stride slice type, the following conditions are met:

- (1.4.1) — `SliceSpecifiers...[0]` is a unit-stride slice type; and
- (1.4.2) — for each  $k$  in the range  $[u + 1, u + \text{SubExtents}::\text{rank}() - 1]$ , `SliceSpecifiers...[k]` denotes `full_extent_t`; and
- (1.4.3) — for  $k$  equal to  $u + \text{SubExtents}::\text{rank}() - 1$ , `SliceSpecifiers...[k]` is a unit-stride slice type;

where `S_static` is:

- (1.4.4) — `dynamic_extent`, if `static-padding-stride` is `dynamic_extent` or `static_extent(k)` is `dynamic_extent` for any  $k$  in the range  $[1, u + 1]$ ,
- (1.4.5) — otherwise, the product of `static-padding-stride` and all values `static_extent(k)` for  $k$  in the range  $[1, u + 1]$ ;
- (1.5) — otherwise,

`submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

#### 23.7.3.7.7.7 layout\_right\_padded specialization of submdspan\_mapping [mdspan.sub.map.rightpad]

```
template<class Extents>
template<class... SliceSpecifiers>
constexpr auto layout_right_padded::mapping<Extents>::submdspan_mapping_impl(
 SliceSpecifiers... slices) const -> see below;
```

1 Returns:

- (1.1) — `submdspan_mapping_result{*this, 0}`, if `rank_ == 0` is true;
- (1.2) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if `rank_ == 1` is true or `SubExtents::rank() == 0` is true;
- (1.3) — otherwise, `submdspan_mapping_result{layout_right::mapping(sub_ext), offset}`, if
  - (1.3.1) — `SubExtents::rank() == 1` is true and
  - (1.3.2) — for  $k$  equal to `rank_ - 1`, `SliceSpecifiers...[k]` is a unit-stride slice type;
  - (1.4) — otherwise,

`submdspan_mapping_result{layout_right_padded<S_static>::mapping(sub_ext, stride(rank_ - u - 2)), offset}`

if for a value  $u$  for which `rank_ - u - 2` is the largest value  $p$  smaller than `rank_ - 1` for which `SliceSpecifiers...[p]` is a unit-stride slice type, the following conditions are met:

- (1.4.1) — for  $k$  equal to `rank_ - 1`, `SliceSpecifiers...[k]` is a unit-stride slice type; and
- (1.4.2) — for each  $k$  in the range  $[\text{rank\_} - \text{SubExtents}::\text{rank}() - u + 1, \text{rank\_} - u - 1]$ , `SliceSpecifiers...[k]` denotes `full_extent_t`; and
- (1.4.3) — for  $k$  equal to `rank_ - SubExtents::rank() - u`, `SliceSpecifiers...[k]` is a unit-stride slice type;

and where `S_static` is:

- (1.4.4) — `dynamic_extent` if `static-padding-stride` is `dynamic_extent` or for any  $k$  in the range  $[\text{rank\_} - u - 1, \text{rank\_} - 1]$  `static_extent(k)` is `dynamic_extent`,
- (1.4.5) — otherwise, the product of `static-padding-stride` and all values `static_extent(k)` with  $k$  in the range  $[\text{rank\_} - u - 1, \text{rank\_} - 1]$ ;
- (1.5) — otherwise,

`submdspan_mapping_result{layout_stride::mapping(sub_ext, sub_strides), offset}`

## 23.7.3.7.8 submdspan function template

[mdspan.sub.sub]

```
template<class ElementType, class Extents, class LayoutPolicy,
 class AccessorPolicy, class... SliceSpecifiers>
constexpr auto submdspan(
 const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& src,
 SliceSpecifiers... raw_slices) -> see below;
```

1 Let `index_type` be `typename Extents::index_type`.

2 Let `slices` be the pack introduced by the following declaration:

```
 auto [...slices] = canonical_slices(src.extents(), raw_slices...);
```

3 Let `sub_map_offset` be the result of `submdspan_mapping(src.mapping(), slices...)`.

[*Note 1*: This invocation of `submdspan_mapping` selects a function call via overload resolution on a candidate set that includes the lookup set found by argument-dependent lookup (6.5.4). — *end note*]

4 *Constraints*:

(4.1) — `sizeof...(slices)` equals `Extents::rank()`, and

(4.2) — `LayoutPolicy::mapping<Extents>` models *sliceable-mapping*.

5 *Mandates*: For each rank index  $k$  of `src`:

(5.1) — `SliceSpecifiers...[k]` is a `submdspan` slice type for `index_type`, and

(5.2) — `decltype(slices...[k])` is a valid `submdspan` slice type for the  $k^{\text{th}}$  extent of `Extents`.

6 *Preconditions*: For each rank index  $k$  of `src.extents()`, `slices...[k]` is a valid `submdspan` slice for the  $k^{\text{th}}$  extent of `src.extents()`.

7 *Effects*: Equivalent to:

```
 auto sub_map_result = submdspan_mapping(src.mapping(), slices...);
 return mdspan(src.accessor().offset(src.data_handle(), sub_map_result.offset),
 sub_map_result.mapping,
 typename AccessorPolicy::offset_policy(src.accessor()));
```

8 [*Example 1*: Given a rank-3 `mdspan` `grid3d` representing a three-dimensional grid of regularly spaced points in a rectangular prism, the function `zero_surface` sets all elements on the surface of the 3-dimensional shape to zero. It does so by reusing a function `zero_2d` that takes a rank-2 `mdspan`.

```
// zero out all elements in an mdspan
template<class T, class E, class L, class A>
void zero_2d(mdspan<T, E, L, A> a) {
 static_assert(a.rank() == 2);
 for (int i = 0; i < a.extent(0); i++)
 for (int j = 0; j < a.extent(1); j++)
 a[i, j] = 0;
}

// zero out just the surface
template<class T, class E, class L, class A>
void zero_surface(mdspan<T, E, L, A> grid3d) {
 static_assert(grid3d.rank() == 3);
 zero_2d(submdspan(grid3d, 0, full_extent, full_extent));
 zero_2d(submdspan(grid3d, full_extent, 0, full_extent));
 zero_2d(submdspan(grid3d, full_extent, full_extent, 0));
 zero_2d(submdspan(grid3d, grid3d.extent(0) - 1, full_extent, full_extent));
 zero_2d(submdspan(grid3d, full_extent, grid3d.extent(1) - 1, full_extent));
 zero_2d(submdspan(grid3d, full_extent, full_extent, grid3d.extent(2) - 1));
}
```

— *end example*]

## 23.7.3.8 Multidimensional copy algorithms

[mdspan.copy]

```
template<class Src, class Dst>
constexpr void copy(const Src& src, const Dst& dst);
```

```
template<class ExecutionPolicy, class Src, class Dst>
void copy(ExecutionPolicy&& policy, const Src& src, const Dst& dst);
```

1     *Constraints:*

- (1.1)     — `Src` and `Dst` are specializations of `mdspan`,
- (1.2)     — `is_assignable_v<typename Dst::reference, typename Src::reference>` is true, and
- (1.3)     — `is_constructible_v<typename Src::extents_type, typename Dst::extents_type>` is true.

[*Note 1:* This constraint effectively verifies that the ranks of the extents are the same and that each extent is either static and the same, or `dynamic_extent`. — *end note*]

2     *Preconditions:*

- (2.1)     — `dst.is_unique()` is true, and
- (2.2)     — for each multidimensional index `i` in `src.extents()`, there is no multidimensional index `j` in `dst.extents()` such that `src[i]` and `dst[j]` refer to the same element.

3     *Hardened preconditions:* `src.extents()` equals `dst.extents()`.

4     *Effects:* Assigns each element of `src` to the corresponding element of `dst`.

```
template<class Dst, class T = Dst::value_type>
constexpr void fill(const Dst& dst, const T& value);
```

```
template<class ExecutionPolicy, class Dst, class T = Dst::value_type>
void fill(ExecutionPolicy&& policy, const Dst& dst, const T& value);
```

5     *Constraints:*

- (5.1)     — `Dst` is a specialization of `mdspan`, and
- (5.2)     — `is_assignable_v<typename Dst::reference, const T&>` is true.

6     *Effects:* Assigns `value` to each element of `dst`.

## 24 Iterators library

[iterators]

### 24.1 General

[iterators.general]

- <sup>1</sup> This Clause describes components that C++ programs may use to perform iterations over containers ([Clause 23](#)), streams ([31.7](#)), stream buffers ([31.6](#)), and other ranges ([Clause 25](#)).
- <sup>2</sup> The following subclauses describe iterator requirements, and components for iterator primitives, predefined iterators, and stream iterators, as summarized in [Table 76](#).

**Table 76 — Iterators library summary** [tab:iterators.summary]

| Subclause                                  | Header     |
|--------------------------------------------|------------|
| <a href="#">24.3</a> Iterator requirements | <iterator> |
| <a href="#">24.4</a> Iterator primitives   |            |
| <a href="#">24.5</a> Iterator adaptors     |            |
| <a href="#">24.6</a> Stream iterators      |            |
| <a href="#">24.7</a> Range access          |            |

### 24.2 Header <iterator> synopsis

[iterator.synopsis]

```

#include <compare> // see 17.12.1
#include <concepts> // see 18.3
#include <initializer_list> // see 17.11.2

namespace std {
 template<class T> using with-reference = T&; // exposition only
 template<class T> concept can-reference // exposition only
 = requires { typename with-reference<T>; };
 template<class T> concept dereferenceable // exposition only
 = requires(T& t) {
 { *t } -> can-reference; // not required to be equality-preserving
 };

 // 24.3.2, associated types
 // 24.3.2.1, incrementable traits
 template<class> struct incrementable_traits; // freestanding
 template<class T>
 using iter_difference_t = see below; // freestanding

 // 24.3.2.2, indirectly readable traits
 template<class> struct indirectly_readable_traits; // freestanding
 template<class T>
 using iter_value_t = see below; // freestanding

 // 24.3.2.3, iterator traits
 template<class I> struct iterator_traits; // freestanding
 template<class T> requires is_object_v<T> struct iterator_traits<T*>; // freestanding

 template<dereferenceable T>
 using iter_reference_t = decltype(*declval<T&>()); // freestanding

 namespace ranges {
 // 24.3.3, customization point objects
 inline namespace unspecified {
 // 24.3.3.1, ranges::iter_move
 inline constexpr unspecified iter_move = unspecified; // freestanding
 }
 }
}

```

```

 // 24.3.3.2, ranges::iter_swap
 inline constexpr unspecified iter_swap = unspecified; // freestanding
}

template<dereferenceable T>
 requires requires(T& t) {
 { ranges::iter_move(t) } -> can-reference;
 }
using iter_rvalue_reference_t // freestanding
 = decltype(ranges::iter_move(declval<T&>()));

// 24.3.4, iterator concepts
// 24.3.4.2, concept indirectly_readable
template<class In>
 concept indirectly_readable = see below; // freestanding

// 24.3.6.2, indirect callable traits
template<indirectly_readable T>
 using indirect-value-t = see below; // exposition only

template<indirectly_readable T>
 using iter_common_reference_t = // freestanding
 common_reference_t<iter_reference_t<T>, indirect-value-t<T>>;

// 24.3.4.3, concept indirectly_writable
template<class Out, class T>
 concept indirectly_writable = see below; // freestanding

// 24.3.4.4, concept weakly_incrementable
template<class I>
 concept weakly_incrementable = see below; // freestanding

// 24.3.4.5, concept incrementable
template<class I>
 concept incrementable = see below; // freestanding

// 24.3.4.6, concept input_or_output_iterator
template<class I>
 concept input_or_output_iterator = see below; // freestanding

// 24.3.4.7, concept sentinel_for
template<class S, class I>
 concept sentinel_for = see below; // freestanding

// 24.3.4.8, concept sized_sentinel_for
template<class S, class I>
 constexpr bool disable_sized_sentinel_for = false; // freestanding

template<class S, class I>
 concept sized_sentinel_for = see below; // freestanding

// 24.3.4.9, concept input_iterator
template<class I>
 concept input_iterator = see below; // freestanding

// 24.3.4.10, concept output_iterator
template<class I, class T>
 concept output_iterator = see below; // freestanding

// 24.3.4.11, concept forward_iterator
template<class I>
 concept forward_iterator = see below; // freestanding

```

```

// 24.3.4.12, concept bidirectional_iterator
template<class I>
 concept bidirectional_iterator = see below; // freestanding

// 24.3.4.13, concept random_access_iterator
template<class I>
 concept random_access_iterator = see below; // freestanding

// 24.3.4.14, concept contiguous_iterator
template<class I>
 concept contiguous_iterator = see below; // freestanding

// 24.3.6, indirect callable requirements
// 24.3.6.3, indirect callables
template<class F, class I>
 concept indirectly_unary_invocable = see below; // freestanding

template<class F, class I>
 concept indirectly_regular_unary_invocable = see below; // freestanding

template<class F, class I>
 concept indirect_unary_predicate = see below; // freestanding

template<class F, class I1, class I2>
 concept indirect_binary_predicate = see below; // freestanding

template<class F, class I1, class I2 = I1>
 concept indirect_equivalence_relation = see below; // freestanding

template<class F, class I1, class I2 = I1>
 concept indirect_strict_weak_order = see below; // freestanding

template<class F, class... Is>
 requires (indirectly_readable<Is> && ...) && invocable<F, iter_reference_t<Is>...>
 using indirect_result_t = invoke_result_t<F, iter_reference_t<Is>...>; // freestanding

// 24.3.6.4, projected
template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
 using projected = see below; // freestanding

template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
using projected_value_t = // freestanding
 remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>&>>;

// 24.3.7, common algorithm requirements
// 24.3.7.2, concept indirectly_movable
template<class In, class Out>
 concept indirectly_movable = see below; // freestanding

template<class In, class Out>
 concept indirectly_movable_storable = see below; // freestanding

// 24.3.7.3, concept indirectly_copyable
template<class In, class Out>
 concept indirectly_copyable = see below; // freestanding

template<class In, class Out>
 concept indirectly_copyable_storable = see below; // freestanding

// 24.3.7.4, concept indirectly_swappable
template<class I1, class I2 = I1>
 concept indirectly_swappable = see below; // freestanding

```

```

// 24.3.7.5, concept indirectly_comparable
template<class I1, class I2, class R, class P1 = identity, class P2 = identity>
 concept indirectly_comparable = see below; // freestanding

// 24.3.7.6, concept permutable
template<class I>
 concept permutable = see below; // freestanding

// 24.3.7.7, concept mergeable
template<class I1, class I2, class Out,
 class R = ranges::less, class P1 = identity, class P2 = identity>
 concept mergeable = see below; // freestanding

// 24.3.7.8, concept sortable
template<class I, class R = ranges::less, class P = identity>
 concept sortable = see below; // freestanding

// 24.4, primitives
// 24.4.2, iterator tags
struct input_iterator_tag { }; // freestanding
struct output_iterator_tag { }; // freestanding
struct forward_iterator_tag: public input_iterator_tag { }; // freestanding
struct bidirectional_iterator_tag: public forward_iterator_tag { }; // freestanding
struct random_access_iterator_tag: public bidirectional_iterator_tag { }; // freestanding
struct contiguous_iterator_tag: public random_access_iterator_tag { }; // freestanding

// 24.4.3, iterator operations
template<class InputIterator, class Distance>
 constexpr void
 advance(InputIterator& i, Distance n); // freestanding
template<class InputIterator>
 constexpr typename iterator_traits<InputIterator>::difference_type
 distance(InputIterator first, InputIterator last); // freestanding
template<class InputIterator>
 constexpr InputIterator
 next(InputIterator x, // freestanding
 typename iterator_traits<InputIterator>::difference_type n = 1);
template<class BidirectionalIterator>
 constexpr BidirectionalIterator
 prev(BidirectionalIterator x, // freestanding
 typename iterator_traits<BidirectionalIterator>::difference_type n = 1);

// 24.4.4, range iterator operations
namespace ranges {
 // 24.4.4.2, ranges::advance
 template<input_or_output_iterator I>
 constexpr void advance(I& i, iter_difference_t<I> n); // freestanding
 template<input_or_output_iterator I, sentinel_for<I> S>
 constexpr void advance(I& i, S bound); // freestanding
 template<input_or_output_iterator I, sentinel_for<I> S>
 constexpr iter_difference_t<I> advance(I& i, iter_difference_t<I> n, // freestanding
 S bound);

 // 24.4.4.3, ranges::distance
 template<class I, sentinel_for<I> S>
 requires (!sized_sentinel_for<S, I>)
 constexpr iter_difference_t<I> distance(I first, S last); // freestanding
 template<class I, sized_sentinel_for<decay_t<I>> S>
 constexpr iter_difference_t<decay_t<I>> distance(I&& first, S last); // freestanding
 template<range R>
 constexpr range_difference_t<R> distance(R&& r); // freestanding

```

```

// 24.4.4.4, ranges::next
template<input_or_output_iterator I>
 constexpr I next(I x); // freestanding
template<input_or_output_iterator I>
 constexpr I next(I x, iter_difference_t<I> n); // freestanding
template<input_or_output_iterator I, sentinel_for<I> S>
 constexpr I next(I x, S bound); // freestanding
template<input_or_output_iterator I, sentinel_for<I> S>
 constexpr I next(I x, iter_difference_t<I> n, S bound); // freestanding

// 24.4.4.5, ranges::prev
template<bidirectional_iterator I>
 constexpr I prev(I x); // freestanding
template<bidirectional_iterator I>
 constexpr I prev(I x, iter_difference_t<I> n); // freestanding
template<bidirectional_iterator I>
 constexpr I prev(I x, iter_difference_t<I> n, I bound); // freestanding
}

// 24.5, predefined iterators and sentinels
// 24.5.1, reverse iterators
template<class Iterator> class reverse_iterator; // freestanding

template<class Iterator1, class Iterator2>
 constexpr bool operator==(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
 constexpr bool operator!=(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
 constexpr bool operator<(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
 constexpr bool operator>(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
 constexpr bool operator<=(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, class Iterator2>
 constexpr bool operator>=(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
 constexpr compare_three_way_result_t<Iterator1, Iterator2>
 operator<=>(const reverse_iterator<Iterator1>& x, // freestanding
 const reverse_iterator<Iterator2>& y);

template<class Iterator1, class Iterator2>
 constexpr auto operator-(// freestanding
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
template<class Iterator>
 constexpr reverse_iterator<Iterator> operator+(// freestanding
 iter_difference_t<Iterator> n,
 const reverse_iterator<Iterator>& x);

template<class Iterator>
 constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i); // freestanding

```



```

template<class Iterator1, class Iterator2>
 requires (!sized_sentinel_for<Iterator1, Iterator2>)
 constexpr bool disable_sized_sentinel_for<reverse_iterator<Iterator1>, // freestanding
 reverse_iterator<Iterator2>> = true;

// 24.5.2, insert iterators
template<class Container> class back_insert_iterator; // freestanding
template<class Container>
 constexpr back_insert_iterator<Container> back_inserter(Container& x); // freestanding

template<class Container> class front_insert_iterator; // freestanding
template<class Container>
 constexpr front_insert_iterator<Container> front_inserter(Container& x); // freestanding

template<class Container> class insert_iterator; // freestanding
template<class Container>
 constexpr insert_iterator<Container>
 inserter(Container& x, ranges::iterator_t<Container> i); // freestanding

// 24.5.3, constant iterators and sentinels
// 24.5.3.2, alias templates
template<indirectly_readable I>
 using iter_const_reference_t = see below; // freestanding
template<class Iterator>
 concept constant_iterator = see below; // exposition only
template<input_iterator I>
 using const_iterator = see below; // freestanding
template<semiregular S>
 using const_sentinel = see below; // freestanding

// 24.5.3.3, class template basic_const_iterator
template<input_iterator Iterator>
 class basic_const_iterator; // freestanding

template<class T, common_with<T> U>
 requires input_iterator<common_type_t<T, U>>
struct common_type<basic_const_iterator<T>, U> { // freestanding
 using type = basic_const_iterator<common_type_t<T, U>>;
};
template<class T, common_with<T> U>
 requires input_iterator<common_type_t<T, U>>
struct common_type<U, basic_const_iterator<T>> { // freestanding
 using type = basic_const_iterator<common_type_t<T, U>>;
};
template<class T, common_with<T> U>
 requires input_iterator<common_type_t<T, U>>
struct common_type<basic_const_iterator<T>, basic_const_iterator<U>> { // freestanding
 using type = basic_const_iterator<common_type_t<T, U>>;
};

template<input_iterator I>
 constexpr const_iterator<I> make_const_iterator(I it) { return it; } // freestanding

template<semiregular S>
 constexpr const_sentinel<S> make_const_sentinel(S s) { return s; } // freestanding

// 24.5.4, move iterators and sentinels
template<class Iterator> class move_iterator; // freestanding

template<class Iterator1, class Iterator2>
 constexpr bool operator==(// freestanding
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);

```

```

template<class Iterator1, class Iterator2>
 constexpr bool operator<(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
 constexpr bool operator>(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
 constexpr bool operator<=(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, class Iterator2>
 constexpr bool operator>=(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y); // freestanding
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
 constexpr compare_three_way_result_t<Iterator1, Iterator2>
 operator<=>(const move_iterator<Iterator1>& x, // freestanding
 const move_iterator<Iterator2>& y);

template<class Iterator1, class Iterator2>
 constexpr auto operator-(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y) // freestanding
 -> decltype(x.base() - y.base());
template<class Iterator>
 constexpr move_iterator<Iterator>
 operator+(iter_difference_t<Iterator> n, const move_iterator<Iterator>& x); // freestanding

template<class Iterator>
 constexpr move_iterator<Iterator> make_move_iterator(Iterator i); // freestanding

template<class Iterator1, class Iterator2>
 requires (!sized_sentinel_for<Iterator1, Iterator2>)
 constexpr bool disable_sized_sentinel_for<move_iterator<Iterator1>, // freestanding
 move_iterator<Iterator2>> = true;

template<semiregular S> class move_sentinel; // freestanding

// 24.5.5, common iterators
template<input_or_output_iterator I, sentinel_for<I> S>
 requires (!same_as<I, S> && copyable<I>)
 class common_iterator; // freestanding

template<class I, class S>
 struct incrementable_traits<common_iterator<I, S>>; // freestanding

template<input_iterator I, class S>
 struct iterator_traits<common_iterator<I, S>>; // freestanding

// 24.5.6, default sentinel
struct default_sentinel_t; // freestanding
inline constexpr default_sentinel_t default_sentinel{}; // freestanding

// 24.5.7, counted iterators
template<input_or_output_iterator I> class counted_iterator; // freestanding

template<input_iterator I>
 requires see_below
 struct iterator_traits<counted_iterator<I>>; // freestanding

// 24.5.8, unreachable sentinel
struct unreachable_sentinel_t; // freestanding
inline constexpr unreachable_sentinel_t unreachable_sentinel{}; // freestanding

```

// 24.6, stream iterators

```
template<class T, class charT = char, class traits = char_traits<charT>,
 class Distance = ptrdiff_t>
class istream_iterator;

template<class T, class charT, class traits, class Distance>
 bool operator==(const istream_iterator<T, charT, traits, Distance>& x,
 const istream_iterator<T, charT, traits, Distance>& y);

template<class T, class charT = char, class traits = char_traits<charT>>
 class ostream_iterator;

template<class charT, class traits = char_traits<charT>>
 class istreambuf_iterator;

template<class charT, class traits>
 bool operator==(const istreambuf_iterator<charT, traits>& a,
 const istreambuf_iterator<charT, traits>& b);

template<class charT, class traits = char_traits<charT>>
 class ostreambuf_iterator;
```

// 24.7, range access

```
template<class C> constexpr auto
 begin(C& c) noexcept(noexcept(c.begin())) -> decltype(c.begin()); // freestanding
template<class C> constexpr auto
 begin(const C& c) noexcept(noexcept(c.begin())) -> decltype(c.begin()); // freestanding
template<class C> constexpr auto
 end(C& c) noexcept(noexcept(c.end())) -> decltype(c.end()); // freestanding
template<class C> constexpr auto
 end(const C& c) noexcept(noexcept(c.end())) -> decltype(c.end()); // freestanding
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept; // freestanding
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept; // freestanding
template<class C> constexpr auto
 cbegin(const C& c) noexcept(noexcept(std::begin(c))) // freestanding
 -> decltype(std::begin(c));
template<class C> constexpr auto
 cend(const C& c) noexcept(noexcept(std::end(c))) -> decltype(std::end(c)); // freestanding
template<class C> constexpr auto
 rbegin(C& c) noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin()); // freestanding
template<class C> constexpr auto
 rbegin(const C& c) noexcept(noexcept(c.rbegin())) -> decltype(c.rbegin()); // freestanding
template<class C> constexpr auto
 rend(C& c) noexcept(noexcept(c.rend())) -> decltype(c.rend()); // freestanding
template<class C> constexpr auto
 rend(const C& c) noexcept(noexcept(c.rend())) -> decltype(c.rend()); // freestanding
template<class T, size_t N> constexpr reverse_iterator<T*>
 rbegin(T (&array)[N]) noexcept; // freestanding
template<class T, size_t N> constexpr reverse_iterator<T*>
 rend(T (&array)[N]) noexcept; // freestanding
template<class E> constexpr reverse_iterator<const E*>
 rbegin(initializer_list<E> il) noexcept; // freestanding
template<class E> constexpr reverse_iterator<const E*>
 rend(initializer_list<E> il) noexcept; // freestanding
template<class C> constexpr auto
 crbegin(const C& c) noexcept(noexcept(std::rbegin(c))) // freestanding
 -> decltype(std::rbegin(c));
template<class C> constexpr auto
 crend(const C& c) noexcept(noexcept(std::rend(c))) -> decltype(std::rend(c)); // freestanding

template<class C> constexpr auto
 size(const C& c) noexcept(noexcept(c.size())) -> decltype(c.size()); // freestanding
template<class T, size_t N> constexpr size_t
 size(const T (&array)[N]) noexcept; // freestanding
```

```

template<class C> constexpr auto
 ssize(const C& c) noexcept(noexcept(c.size()))
 -> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>; // freestanding
template<class T, ptrdiff_t N> constexpr ptrdiff_t
 ssize(const T (&array)[N]) noexcept; // freestanding

template<class C> constexpr auto
 empty(const C& c) noexcept(noexcept(c.empty())) -> decltype(c.empty()); // freestanding
template<class T, size_t N> constexpr bool
 empty(const T (&array)[N]) noexcept; // freestanding

template<class C> constexpr auto
 data(C& c) noexcept(noexcept(c.data())) -> decltype(c.data()); // freestanding
template<class C> constexpr auto
 data(const C& c) noexcept(noexcept(c.data())) -> decltype(c.data()); // freestanding
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept; // freestanding
}

```

## 24.3 Iterator requirements

[iterator.requirements]

### 24.3.1 General

[iterator.requirements.general]

- <sup>1</sup> Iterators are a generalization of pointers that allow a C++ program to work with different data structures (for example, containers and ranges) in a uniform manner. To be able to construct template algorithms that work correctly and efficiently on different types of data structures, the library formalizes not just the interfaces but also the semantics and complexity assumptions of iterators. An input iterator *i* supports the expression *\*i*, resulting in a value of some object type *T*, called the *value type* of the iterator. An output iterator *i* has a non-empty set of types that are *writable* to the iterator; for each such type *T*, the expression *\*i = o* is valid where *o* is a value of type *T*. For every iterator type *X*, there is a corresponding signed integer-like type (24.3.4.4) called the *difference type* of the iterator.
- <sup>2</sup> Since iterators are an abstraction of pointers, their semantics are a generalization of most of the semantics of pointers in C++. This ensures that every function template that takes iterators works as well with regular pointers. This document defines six categories of iterators, according to the operations defined on them: *input iterators*, *output iterators*, *forward iterators*, *bidirectional iterators*, *random access iterators*, and *contiguous iterators*, as shown in Table 77.

Table 77 — Relations among iterator categories [tab:iterators.relations]

|            |   |               |   |               |   |         |   |       |   |        |
|------------|---|---------------|---|---------------|---|---------|---|-------|---|--------|
| Contiguous | → | Random Access | → | Bidirectional | → | Forward | → | Input | → | Output |
|------------|---|---------------|---|---------------|---|---------|---|-------|---|--------|

- <sup>3</sup> The six categories of iterators correspond to the iterator concepts

- (3.1) — `input_iterator` (24.3.4.9),
- (3.2) — `output_iterator` (24.3.4.10),
- (3.3) — `forward_iterator` (24.3.4.11),
- (3.4) — `bidirectional_iterator` (24.3.4.12),
- (3.5) — `random_access_iterator` (24.3.4.13), and
- (3.6) — `contiguous_iterator` (24.3.4.14),

respectively. The generic term *iterator* refers to any type that models the `input_or_output_iterator` concept (24.3.4.6).

- <sup>4</sup> Forward iterators meet all the requirements of input iterators and can be used whenever an input iterator is specified; Bidirectional iterators also meet all the requirements of forward iterators and can be used whenever a forward iterator is specified; Random access iterators also meet all the requirements of bidirectional iterators and can be used whenever a bidirectional iterator is specified; Contiguous iterators also meet all the requirements of random access iterators and can be used whenever a random access iterator is specified.
- <sup>5</sup> Iterators that further meet the requirements of output iterators are called *mutable iterators*. Nonmutable iterators are referred to as *constant iterators*.

- <sup>6</sup> In addition to the requirements in this subclause, the nested *typedef-names* specified in 24.3.2.3 shall be provided for the iterator type.

[Note 1: Either the iterator type must provide the *typedef-names* directly (in which case `iterator_traits` pick them up automatically), or an `iterator_traits` specialization must provide them. — end note]

- <sup>7</sup> Just as a regular pointer to an array guarantees that there is a pointer value pointing past the last element of the array, so for any iterator type there is an iterator value that points past the last element of a corresponding sequence. Such a value is called a *past-the-end value*. Values of an iterator `i` for which the expression `*i` is defined are called *dereferenceable*. The library never assumes that past-the-end values are dereferenceable. Iterators can also have singular values that are not associated with any sequence. Results of most expressions are undefined for singular values; the only exceptions are destroying an iterator that holds a singular value, the assignment of a non-singular value to an iterator that holds a singular value, and, for iterators that meet the *Cpp17DefaultConstructible* requirements, using a value-initialized iterator as the source of a copy or move operation.

[Note 2: This guarantee is not offered for default-initialization, although the distinction only matters for types with trivial default constructors such as pointers or aggregates holding pointers. — end note]

In these cases the singular value is overwritten the same way as any other value. Dereferenceable values are always non-singular.

- <sup>8</sup> Most of the library’s algorithmic templates that operate on data structures have interfaces that use ranges. A *range* is an iterator and a *sentinel* that designate the beginning and end of the computation, or an iterator and a count that designate the beginning and the number of elements to which the computation is to be applied.<sup>177</sup>
- <sup>9</sup> An iterator and a sentinel denoting a range are comparable. A range `[i, s)` is empty if `i == s`; otherwise, `[i, s)` refers to the elements in the data structure starting with the element pointed to by `i` and up to but not including the element, if any, pointed to by the first iterator `j` such that `j == s`.
- <sup>10</sup> A sentinel `s` is called *reachable from* an iterator `i` if and only if there is a finite sequence of applications of the expression `++i` that makes `i == s`. If `s` is reachable from `i`, `[i, s)` denotes a *valid range*.
- <sup>11</sup> A *counted range* `i + [0, n)` is empty if `n == 0`; otherwise, `i + [0, n)` refers to the `n` elements in the data structure starting with the element pointed to by `i` and up to but not including the element, if any, pointed to by the result of `n` applications of `++i`. A counted range `i + [0, n)` is *valid* if and only if `n == 0`; or `n` is positive, `i` is dereferenceable, and `++i + [0, --n)` is valid.
- <sup>12</sup> The result of the application of library functions to invalid ranges is undefined.
- <sup>13</sup> For an iterator `i` of a type that models `contiguous_iterator` (24.3.4.14), library functions are permitted to replace `[i, s)` with `[to_address(i), to_address(i + ranges::distance(i, s)))`, and to replace `i + [0, n)` with `[to_address(i), to_address(i + n))`.
- [Note 3: This means a program cannot rely on any side effects of dereferencing a contiguous iterator `i`, because library functions might operate on pointers obtained by `to_address(i)` instead of operating on `i`. Similarly, a program cannot rely on any side effects of individual increments on a contiguous iterator `i`, because library functions might advance `i` only once. — end note]
- <sup>14</sup> All the categories of iterators require only those functions that are realizable for a given category in constant time (amortized). Therefore, requirement tables and concept definitions for the iterators do not specify complexity.
- <sup>15</sup> Destruction of an iterator may invalidate pointers and references previously obtained from that iterator if its type does not meet the *Cpp17ForwardIterator* requirements and does not model `forward_iterator`.
- <sup>16</sup> An *invalid iterator* is an iterator that may be singular.<sup>178</sup>
- <sup>17</sup> Iterators meet the *constexpr iterator* requirements if all operations provided to meet iterator category requirements are *constexpr* functions.

[Note 4: For example, the types “pointer to `int`” and `reverse_iterator<int*>` meet the *constexpr iterator* requirements. — end note]

<sup>177</sup>) The sentinel denoting the end of a range can have the same type as the iterator denoting the beginning of the range, or a different type.

<sup>178</sup>) This definition applies to pointers, since pointers are iterators. The effect of dereferencing an iterator that has been invalidated is undefined.

## 24.3.2 Associated types

[iterator.assoc.types]

### 24.3.2.1 Incrementable traits

[incrementable.traits]

- <sup>1</sup> To implement algorithms only in terms of incrementable types, it is often necessary to determine the difference type that corresponds to a particular incrementable type. Accordingly, it is required that if `WI` is the name of a type that models the `weakly_incrementable` concept (24.3.4.4), the type

```
iter_difference_t<WI>
```

be defined as the incrementable type's difference type.

```
namespace std {
 template<class> struct incrementable_traits { };

 template<class T>
 requires is_object_v<T>
 struct incrementable_traits<T*> {
 using difference_type = ptrdiff_t;
 };

 template<class I>
 struct incrementable_traits<const I>
 : incrementable_traits<I> { };

 template<class T>
 requires requires { typename T::difference_type; }
 struct incrementable_traits<T> {
 using difference_type = T::difference_type;
 };

 template<class T>
 requires (!requires { typename T::difference_type; } &&
 requires(const T& a, const T& b) { { a - b } -> integral; })
 struct incrementable_traits<T> {
 using difference_type = make_signed_t<decltype(declval<T>() - declval<T>())>;
 };

 template<class T>
 using iter_difference_t = see below;
}
```

- <sup>2</sup> Let  $R_I$  be `remove_cvref_t<I>`. The type `iter_difference_t<I>` denotes

- (2.1) — `incrementable_traits< $R_I$ >::difference_type` if `iterator_traits< $R_I$ >` names a specialization generated from the primary template, and
- (2.2) — `iterator_traits< $R_I$ >::difference_type` otherwise.

- <sup>3</sup> Users may specialize `incrementable_traits` on program-defined types.

### 24.3.2.2 Indirectly readable traits

[readable.traits]

- <sup>1</sup> To implement algorithms only in terms of indirectly readable types, it is often necessary to determine the value type that corresponds to a particular indirectly readable type. Accordingly, it is required that if `R` is the name of a type that models the `indirectly_readable` concept (24.3.4.2), the type

```
iter_value_t<R>
```

be defined as the indirectly readable type's value type.

```
template<class> struct cond-value-type { }; // exposition only
template<class T>
 requires is_object_v<T>
struct cond-value-type<T> {
 using value_type = remove_cv_t<T>;
};

template<class T>
 concept has-member-value-type = requires { typename T::value_type; }; // exposition only
```

```

template<class T>
 concept has-member-element-type = requires { typename T::element_type; }; // exposition only

template<class> struct indirectly_readable_traits { };

template<class T>
struct indirectly_readable_traits<T*>
 : cond-value-type<T> { };

template<class I>
 requires is_array_v<I>
struct indirectly_readable_traits<I> {
 using value_type = remove_cv_t<remove_extnt_t<I>>;
};

template<class I>
struct indirectly_readable_traits<const I>
 : indirectly_readable_traits<I> { };

template<has-member-value-type T>
struct indirectly_readable_traits<T>
 : cond-value-type<typename T::value_type> { };

template<has-member-element-type T>
struct indirectly_readable_traits<T>
 : cond-value-type<typename T::element_type> { };

template<has-member-value-type T>
 requires has-member-element-type<T>
struct indirectly_readable_traits<T> { };

template<has-member-value-type T>
 requires has-member-element-type<T> &&
 same_as<remove_cv_t<typename T::element_type>, remove_cv_t<typename T::value_type>>
struct indirectly_readable_traits<T>
 : cond-value-type<typename T::value_type> { };

template<class T> using iter_value_t = see below;

```

- 2 Let  $R_I$  be `remove_cvref_t<I>`. The type `iter_value_t<I>` denotes
- (2.1) — `indirectly_readable_traits< $R_I$ >::value_type` if `iterator_traits< $R_I$ >` names a specialization generated from the primary template, and
  - (2.2) — `iterator_traits< $R_I$ >::value_type` otherwise.
- 3 Class template `indirectly_readable_traits` may be specialized on program-defined types.
- 4 [Note 1: Some legacy output iterators declare a nested type named `value_type` that is an alias for `void`. These types are not `indirectly_readable` and have no associated value types. — end note]
- 5 [Note 2: Smart pointers like `shared_ptr<int>` are `indirectly_readable` and have an associated value type, but a smart pointer like `shared_ptr<void>` is not `indirectly_readable` and has no associated value type. — end note]

### 24.3.2.3 Iterator traits

[iterator.traits]

- 1 To implement algorithms only in terms of iterators, it is sometimes necessary to determine the iterator category that corresponds to a particular iterator type. Accordingly, it is required that if `I` is the type of an iterator, the type

```
iterator_traits<I>::iterator_category
```

be declared as the iterator's iterator category. In addition, the types

```
iterator_traits<I>::pointer
iterator_traits<I>::reference
```

shall be declared as the iterator's pointer and reference types; that is, for an iterator object `a` of class type, the same type as `decltype(a.operator->())` and `decltype(*a)`, respectively. The type `iterator_`

`traits<I>::pointer` shall be void for an iterator of class type `I` that does not support `operator->`. Additionally, in the case of an output iterator, the types

```
iterator_traits<I>::value_type
iterator_traits<I>::difference_type
iterator_traits<I>::reference
```

may be declared as void.

- 2 The definitions in this subclause make use of the following exposition-only concepts:

```
template<class I>
concept cpp17-iterator =
 requires(I i) {
 { *i } -> can-reference;
 { ++i } -> same_as<I&>;
 { *i++ } -> can-reference;
 } && copyable<I>;
```

```
template<class I>
concept cpp17-input-iterator =
 cpp17-iterator<I> && equality_comparable<I> && requires(I i) {
 typename incrementable_traits<I>::difference_type;
 typename indirectly_readable_traits<I>::value_type;
 typename common_reference_t<iter_reference_t<I>&&,
 typename indirectly_readable_traits<I>::value_type&&>;
 typename common_reference_t<decltype(*i++)&&,
 typename indirectly_readable_traits<I>::value_type&&>;
 requires signed_integral<typename incrementable_traits<I>::difference_type>;
 };
```

```
template<class I>
concept cpp17-forward-iterator =
 cpp17-input-iterator<I> && constructible_from<I> &&
 is_reference_v<iter_reference_t<I>>> &&
 same_as<remove_cvref_t<iter_reference_t<I>>>,
 typename indirectly_readable_traits<I>::value_type> &&
 requires(I i) {
 { i++ } -> convertible_to<const I&>;
 { *i++ } -> same_as<iter_reference_t<I>>;
 };
```

```
template<class I>
concept cpp17-bidirectional-iterator =
 cpp17-forward-iterator<I> && requires(I i) {
 { --i } -> same_as<I&>;
 { i-- } -> convertible_to<const I&>;
 { *i-- } -> same_as<iter_reference_t<I>>;
 };
```

```
template<class I>
concept cpp17-random-access-iterator =
 cpp17-bidirectional-iterator<I> && totally_ordered<I> &&
 requires(I i, typename incrementable_traits<I>::difference_type n) {
 { i += n } -> same_as<I&>;
 { i -= n } -> same_as<I&>;
 { i + n } -> same_as<I>;
 { n + i } -> same_as<I>;
 { i - n } -> same_as<I>;
 { i - i } -> same_as<decltype(n)>;
 { i[n] } -> convertible_to<iter_reference_t<I>>;
 };
```

- 3 The members of a specialization `iterator_traits<I>` generated from the `iterator_traits` primary template are computed as follows:



- (3.1) — If *I* has valid (13.10.3) member types `difference_type`, `value_type`, `reference`, and `iterator_category`, then `iterator_traits<I>` has the following publicly accessible members:

```
using iterator_category = I::iterator_category;
using value_type = I::value_type;
using difference_type = I::difference_type;
using pointer = see below;
using reference = I::reference;
```

If the *qualified-id* `I::pointer` is valid and denotes a type, then `iterator_traits<I>::pointer` names that type; otherwise, it names `void`.

- (3.2) — Otherwise, if *I* satisfies the exposition-only concept *cpp17-input-iterator*, `iterator_traits<I>` has the following publicly accessible members:

```
using iterator_category = see below;
using value_type = indirectly_readable_traits<I>::value_type;
using difference_type = incrementable_traits<I>::difference_type;
using pointer = see below;
using reference = see below;
```

- (3.2.1) — If the *qualified-id* `I::pointer` is valid and denotes a type, `pointer` names that type. Otherwise, if `decltype(declval<I&>().operator->())` is well-formed, then `pointer` names that type. Otherwise, `pointer` names `void`.

- (3.2.2) — If the *qualified-id* `I::reference` is valid and denotes a type, `reference` names that type. Otherwise, `reference` names `iter_reference_t<I>`.

- (3.2.3) — If the *qualified-id* `I::iterator_category` is valid and denotes a type, `iterator_category` names that type. Otherwise, `iterator_category` names:

- (3.2.3.1) — `random_access_iterator_tag` if *I* satisfies *cpp17-random-access-iterator*, or otherwise
- (3.2.3.2) — `bidirectional_iterator_tag` if *I* satisfies *cpp17-bidirectional-iterator*, or otherwise
- (3.2.3.3) — `forward_iterator_tag` if *I* satisfies *cpp17-forward-iterator*, or otherwise
- (3.2.3.4) — `input_iterator_tag`.

- (3.3) — Otherwise, if *I* satisfies the exposition-only concept *cpp17-iterator*, then `iterator_traits<I>` has the following publicly accessible members:

```
using iterator_category = output_iterator_tag;
using value_type = void;
using difference_type = see below;
using pointer = void;
using reference = void;
```

If the *qualified-id* `incrementable_traits<I>::difference_type` is valid and denotes a type, then `difference_type` names that type; otherwise, it names `void`.

- (3.4) — Otherwise, `iterator_traits<I>` has no members by any of the above names.

- <sup>4</sup> Explicit or partial specializations of `iterator_traits` may have a member type `iterator_concept` that is used to indicate conformance to the iterator concepts (24.3.4).

[*Example 1:* To indicate conformance to the `input_iterator` concept but a lack of conformance to the *Cpp17InputIterator* requirements (24.3.5.3), an `iterator_traits` specialization might have `iterator_concept` denote `input_iterator_tag` but not declare `iterator_category`. — *end example*]

- <sup>5</sup> `iterator_traits` is specialized for pointers as

```
namespace std {
 template<class T>
 requires is_object_v<T>
 struct iterator_traits<T*> {
 using iterator_concept = contiguous_iterator_tag;
 using iterator_category = random_access_iterator_tag;
 using value_type = remove_cv_t<T>;
 using difference_type = ptrdiff_t;
 using pointer = T*;
 using reference = T&;
 };
}
```

```
};
}
```

<sup>6</sup> [Example 2: To implement a generic `reverse` function, a C++ program can do the following:

```
template<class BI>
void reverse(BI first, BI last) {
 typename iterator_traits<BI>::difference_type n = distance(first, last);
 --n;
 while (n > 0) {
 typename iterator_traits<BI>::value_type tmp = *first;
 *first++ = *--last;
 *last = tmp;
 n -= 2;
 }
}
```

— end example]

### 24.3.3 Customization point objects

[iterator.cust]

#### 24.3.3.1 `ranges::iter_move`

[iterator.cust.move]

<sup>1</sup> The name `ranges::iter_move` denotes a customization point object (16.3.3.3.5). The expression `ranges::iter_move(E)` for a subexpression `E` is expression-equivalent to:

- (1.1) — `iter_move(E)`, if `E` has class or enumeration type and `iter_move(E)` is a well-formed expression when treated as an unevaluated operand, where the meaning of `iter_move` is established as-if by performing argument-dependent lookup only (6.5.4).
- (1.2) — Otherwise, if the expression `*E` is well-formed:
  - (1.2.1) — if `*E` is an lvalue, `std::move(*E)`;
  - (1.2.2) — otherwise, `*E`.
- (1.3) — Otherwise, `ranges::iter_move(E)` is ill-formed.

[Note 1: This case can result in substitution failure when `ranges::iter_move(E)` appears in the immediate context of a template instantiation. — end note]

<sup>2</sup> If `ranges::iter_move(E)` is not equal to `*E`, the program is ill-formed, no diagnostic required.

#### 24.3.3.2 `ranges::iter_swap`

[iterator.cust.swap]

<sup>1</sup> The name `ranges::iter_swap` denotes a customization point object (16.3.3.3.5) that exchanges the values (18.4.9) denoted by its arguments.

<sup>2</sup> Let *iter-exchange-move* be the exposition-only function template:

```
template<class X, class Y>
constexpr iter_value_t<X> iter-exchange-move(X&& x, Y&& y)
 noexcept(noexcept(iter_value_t<X>(iter_move(x))) &&
 noexcept(*x = iter_move(y)));
```

<sup>3</sup> *Effects:* Equivalent to:

```
iter_value_t<X> old_value(iter_move(x));
*x = iter_move(y);
return old_value;
```

<sup>4</sup> The expression `ranges::iter_swap(E1, E2)` for subexpressions `E1` and `E2` is expression-equivalent to:

- (4.1) — `(void)iter_swap(E1, E2)`, if either `E1` or `E2` has class or enumeration type and `iter_swap(E1, E2)` is a well-formed expression with overload resolution performed in a context that includes the declaration

```
template<class I1, class I2>
void iter_swap(I1, I2) = delete;
```

and does not include a declaration of `ranges::iter_swap`. If the function selected by overload resolution does not exchange the values denoted by `E1` and `E2`, the program is ill-formed, no diagnostic required.

[Note 1: This precludes calling unconstrained `std::iter_swap`. When the deleted overload is viable, program-defined overloads need to be more specialized (13.7.7.3) to be selected. — end note]

- (4.2) — Otherwise, if the types of E1 and E2 each model `indirectly_readable`, and if the reference types of E1 and E2 model `swappable_with` (18.4.9), then `ranges::swap(*E1, *E2)`.
- (4.3) — Otherwise, if the types T1 and T2 of E1 and E2 model `indirectly_movable_storable<T1, T2>` and `indirectly_movable_storable<T2, T1>`, then `(void)(*E1 = iter-exchange-move(E2, E1))`, except that E1 is evaluated only once.
- (4.4) — Otherwise, `ranges::iter_swap(E1, E2)` is ill-formed.

[Note 2: This case can result in substitution failure when `ranges::iter_swap(E1, E2)` appears in the immediate context of a template instantiation. — end note]

### 24.3.4 Iterator concepts

[iterator.concepts]

#### 24.3.4.1 General

[iterator.concepts.general]

- <sup>1</sup> For a type I, let `ITER_TRAITS(I)` denote the type I if `iterator_traits<I>` names a specialization generated from the primary template. Otherwise, `ITER_TRAITS(I)` denotes `iterator_traits<I>`.
- (1.1) — If the *qualified-id* `ITER_TRAITS(I)::iterator_concept` is valid and names a type, then `ITER_CONCEPT(I)` denotes that type.
- (1.2) — Otherwise, if the *qualified-id* `ITER_TRAITS(I)::iterator_category` is valid and names a type, then `ITER_CONCEPT(I)` denotes that type.
- (1.3) — Otherwise, if `iterator_traits<I>` names a specialization generated from the primary template, then `ITER_CONCEPT(I)` denotes `random_access_iterator_tag`.
- (1.4) — Otherwise, `ITER_CONCEPT(I)` does not denote a type.

<sup>2</sup> [Note 1: `ITER_TRAITS` enables independent syntactic determination of an iterator's category and concept. — end note]

[Example 1:

```
struct I {
 using value_type = int;
 using difference_type = int;

 int operator*() const;
 I& operator++();
 I operator++(int);
 I& operator--();
 I operator--(int);

 bool operator==(I) const;
};
```

`iterator_traits<I>::iterator_category` denotes `input_iterator_tag`, and `ITER_CONCEPT(I)` denotes `random_access_iterator_tag`. — end example]

#### 24.3.4.2 Concept `indirectly_readable`

[iterator.concept.readable]

- <sup>1</sup> Types that are indirectly readable by applying `operator*` model the `indirectly_readable` concept, including pointers, smart pointers, and iterators.

```
template<class In>
concept indirectly_readable_impl = // exposition only
 requires(const In in) {
 typename iter_value_t<In>;
 typename iter_reference_t<In>;
 typename iter_rvalue_reference_t<In>;
 { *in } -> same_as<iter_reference_t<In>>;
 { ranges::iter_move(in) } -> same_as<iter_rvalue_reference_t<In>>;
 } &&
 common_reference_with<iter_reference_t<In>&&, iter_value_t<In>&& &&
 common_reference_with<iter_reference_t<In>&&, iter_rvalue_reference_t<In>&& &&
 common_reference_with<iter_rvalue_reference_t<In>&&, const iter_value_t<In>&>;

template<class In>
concept indirectly_readable =
 indirectly_readable_impl<remove_cvref_t<In>>;
```

- <sup>2</sup> Given a value *i* of type *I*, *I* models `indirectly_readable` only if the expression `*i` is equality-preserving.

#### 24.3.4.3 Concept `indirectly_writable`

[`iterator.concept.writable`]

- <sup>1</sup> The `indirectly_writable` concept specifies the requirements for writing a value into an iterator's referenced object.

```
template<class Out, class T>
concept indirectly_writable =
 requires(Out&& o, T&& t) {
 *o = std::forward<T>(t); // not required to be equality-preserving
 *std::forward<Out>(o) = std::forward<T>(t); // not required to be equality-preserving
 const_cast<const iter_reference_t<Out>&&>(*o) =
 std::forward<T>(t); // not required to be equality-preserving
 const_cast<const iter_reference_t<Out>&&>(*std::forward<Out>(o)) =
 std::forward<T>(t); // not required to be equality-preserving
 };

```

- <sup>2</sup> Let *E* be an expression such that `decltype((E))` is *T*, and let *o* be a dereferenceable object of type *Out*. *Out* and *T* model `indirectly_writable<Out, T>` only if:
- (2.1) — If *Out* and *T* model `indirectly_readable<Out> && same_as<iter_value_t<Out>, decay_t<T>>`, then `*o` after any above assignment is equal to the value of *E* before the assignment.
- <sup>3</sup> After evaluating any above assignment expression, *o* is not required to be dereferenceable.
- <sup>4</sup> If *E* is an xvalue (7.2.1), the resulting state of the object it denotes is valid but unspecified (16.4.6.17).
- <sup>5</sup> [Note 1: The only valid use of an `operator*` is on the left side of the assignment statement. Assignment through the same value of the indirectly writable type happens only once. — end note]
- <sup>6</sup> [Note 2: `indirectly_writable` has the awkward `const_cast` expressions to reject iterators with prvalue non-proxy reference types that permit rvalue assignment but do not also permit `const` rvalue assignment. Consequently, an iterator type *I* that returns `std::string` by value does not model `indirectly_writable<I, std::string>`. — end note]

#### 24.3.4.4 Concept `weakly_incrementable`

[`iterator.concept.winc`]

- <sup>1</sup> The `weakly_incrementable` concept specifies the requirements on types that can be incremented with the pre- and post-increment operators. The increment operations are not required to be equality-preserving, nor is the type required to be `equality_comparable`.

```
template<class T>
constexpr bool is-integer-like = see below; // exposition only

template<class T>
constexpr bool is-signed-integer-like = see below; // exposition only

template<class I>
concept weakly_incrementable =
 movable<I> &&
 requires(I i) {
 typename iter_difference_t<I>;
 requires is-signed-integer-like<iter_difference_t<I>>;
 { ++i } -> same_as<I&>; // not required to be equality-preserving
 i++; // not required to be equality-preserving
 };

```

- <sup>2</sup> A type *I* is an *integer-class type* if it is in a set of implementation-defined types that behave as integer types do, as defined below.
- [Note 1: An integer-class type is not necessarily a class type. — end note]
- <sup>3</sup> The range of representable values of an integer-class type is the continuous set of values over which it is defined. For any integer-class type, its range of representable values is either  $-2^{N-1}$  to  $2^{N-1} - 1$  (inclusive) for some integer *N*, in which case it is a *signed-integer-class type*, or 0 to  $2^N - 1$  (inclusive) for some integer *N*, in which case it is an *unsigned-integer-class type*. In both cases, *N* is called the *width* of the integer-class type. The width of an integer-class type is greater than that of every integral type of the same signedness.
- <sup>4</sup> A type *I* other than `cv bool` is *integer-like* if it models `integral<I>` or if it is an integer-class type. An integer-like type *I* is *signed-integer-like* if it models `signed_integral<I>` or if it is a signed-integer-class

type. An integer-like type *I* is *unsigned-integer-like* if it models `unsigned_integral<I>` or if it is an unsigned-integer-class type.

- 5 For every integer-class type *I*, let *B(I)* be a unique hypothetical extended integer type of the same signedness with the same width (6.9.3) as *I*.

[Note 2: The corresponding hypothetical specialization `numeric_limits<B(I)>` meets the requirements on `numeric_limits` specializations for integral types (17.3.5). — end note]

For every integral type *J*, let *B(J)* be the same type as *J*.

- 6 Expressions of integer-class type are explicitly convertible to any integer-like type, and implicitly convertible to any integer-class type of equal or greater width and the same signedness. Expressions of integral type are both implicitly and explicitly convertible to any integer-class type. Conversions between integral and integer-class types and between two integer-class types do not exit via an exception. The result of such a conversion is the unique value of the destination type that is congruent to the source modulo  $2^N$ , where *N* is the width of the destination type.
- 7 Let *a* be an object of integer-class type *I*, let *b* be an object of integer-like type *I2* such that the expression *b* is implicitly convertible to *I*, let *x* and *y* be, respectively, objects of type *B(I)* and *B(I2)* as described above that represent the same values as *a* and *b*, and let *c* be an lvalue of any integral type.
- (7.1) — The expressions *a++* and *a--* shall be prvalues of type *I* whose values are equal to that of *a* prior to the evaluation of the expressions. The expression *a++* shall modify the value of *a* by adding 1 to it. The expression *a--* shall modify the value of *a* by subtracting 1 from it.
  - (7.2) — The expressions *++a*, *--a*, and *&a* shall be expression-equivalent to *a += 1*, *a -= 1*, and *addressof(a)*, respectively.
  - (7.3) — For every *unary-operator* *@* other than *&* for which the expression *@x* is well-formed, *@a* shall also be well-formed and have the same value, effects, and value category as *@x*. If *@x* has type *bool*, so too does *@a*; if *@x* has type *B(I)*, then *@a* has type *I*.
  - (7.4) — For every assignment operator *@=* for which *c @= x* is well-formed, *c @= a* shall also be well-formed and shall have the same value and effects as *c @= x*. The expression *c @= a* shall be an lvalue referring to *c*.
  - (7.5) — For every assignment operator *@=* for which *x @= y* is well-formed, *a @= b* shall also be well-formed and shall have the same effects as *x @= y*, except that the value that would be stored into *x* is stored into *a*. The expression *a @= b* shall be an lvalue referring to *a*.
  - (7.6) — For every non-assignment binary operator *@* for which *x @ y* and *y @ x* are well-formed, *a @ b* and *b @ a* shall also be well-formed and shall have the same value, effects, and value category as *x @ y* and *y @ x*, respectively. If *x @ y* or *y @ x* has type *B(I)*, then *a @ b* or *b @ a*, respectively, has type *I*; if *x @ y* or *y @ x* has type *B(I2)*, then *a @ b* or *b @ a*, respectively, has type *I2*; if *x @ y* or *y @ x* has any other type, then *a @ b* or *b @ a*, respectively, has that type.
- 8 An expression *E* of integer-class type *I* is contextually convertible to *bool* as if by `bool(E != I(0))`.
- 9 All integer-class types model `regular` (18.6) and `three_way_comparable<strong_ordering>` (17.12.4).
- 10 A value-initialized object of integer-class type has value 0.
- 11 For every (possibly cv-qualified) integer-class type *I*, `numeric_limits<I>` is specialized such that each static data member *m* has the same value as `numeric_limits<B(I)>::m`, and each static member function *f* returns `I(numeric_limits<B(I)>::f())`.
- 12 For any two integer-like types *I1* and *I2*, at least one of which is an integer-class type, `common_type_t<I1, I2>` denotes an integer-class type whose width is not less than that of *I1* or *I2*. If both *I1* and *I2* are signed-integer-like types, then `common_type_t<I1, I2>` is also a signed-integer-like type.
- 13 *is-integer-like<I>* is `true` if and only if *I* is an integer-like type. *is-signed-integer-like<I>* is `true` if and only if *I* is a signed-integer-like type.
- 14 Let *i* be an object of type *I*. When *i* is in the domain of both pre- and post-increment, *i* is said to be *incrementable*. *I* models `weakly_incrementable<I>` only if:
- (14.1) — The expressions *++i* and *i++* have the same domain.
  - (14.2) — If *i* is incrementable, then both *++i* and *i++* advance *i* to the next element.
  - (14.3) — If *i* is incrementable, then `addressof(++i)` is equal to `addressof(i)`.

- <sup>15</sup> *Recommended practice:* The implementation of an algorithm on a weakly incrementable type should never attempt to pass through the same incrementable value twice; such an algorithm should be a single-pass algorithm.

[*Note 3:* For `weakly_incrementable` types, `a` equals `b` does not imply that `++a` equals `++b`. (Equality does not guarantee the substitution property or referential transparency.) Such algorithms can be used with `istream_iterator` as the source of the input data through the `istream_iterator` class template. — *end note*]

#### 24.3.4.5 Concept `incrementable`

[`iterator.concept.inc`]

- <sup>1</sup> The `incrementable` concept specifies requirements on types that can be incremented with the pre- and post-increment operators. The increment operations are required to be equality-preserving, and the type is required to be `equality_comparable`.

[*Note 1:* This supersedes the “not required to be equality-preserving” comments on the increment expressions in the definition of `weakly_incrementable`. — *end note*]

```
template<class I>
concept incrementable =
 regular<I> &&
 weakly_incrementable<I> &&
 requires(I i) {
 { i++ } -> same_as<I>;
 };

```

- <sup>2</sup> Let `a` and `b` be incrementable objects of type `I`. `I` models `incrementable` only if:

- (2.1) — If `bool(a == b)` then `bool(a++ == b)`.  
 (2.2) — If `bool(a == b)` then `bool(((void)a++, a) == ++b)`.

- <sup>3</sup> [*Note 2:* The requirement that `a` equals `b` implies `++a` equals `++b` (which is not true for weakly incrementable types) allows the use of multi-pass one-directional algorithms with types that model `incrementable`. — *end note*]

#### 24.3.4.6 Concept `input_or_output_iterator`

[`iterator.concept.iterator`]

- <sup>1</sup> The `input_or_output_iterator` concept forms the basis of the iterator concept taxonomy; every iterator models `input_or_output_iterator`. This concept specifies operations for dereferencing and incrementing an iterator. Most algorithms will require additional operations to compare iterators with sentinels (24.3.4.7), to read (24.3.4.9) or write (24.3.4.10) values, or to provide a richer set of iterator movements (24.3.4.11, 24.3.4.12, 24.3.4.13).

```
template<class I>
concept input_or_output_iterator =
 requires(I i) {
 { *i } -> can-reference;
 } &&
 weakly_incrementable<I>;

```

- <sup>2</sup> [*Note 1:* Unlike the `Cpp17Iterator` requirements, the `input_or_output_iterator` concept does not require copyability. — *end note*]

#### 24.3.4.7 Concept `sentinel_for`

[`iterator.concept.sentinel`]

- <sup>1</sup> The `sentinel_for` concept specifies the relationship between an `input_or_output_iterator` type and a semiregular type whose values denote a range.

```
template<class S, class I>
concept sentinel_for =
 semiregular<S> &&
 !is-integer-like<S> &&
 input_or_output_iterator<I> &&
 weakly-equality-comparable-with<S, I>; // see 18.5.4

```

- <sup>2</sup> Let `s` and `i` be values of type `S` and `I` such that `[i, s)` denotes a range. Types `S` and `I` model `sentinel_for<S, I>` only if:

- (2.1) — `i == s` is well-defined.  
 (2.2) — If `bool(i != s)` then `i` is dereferenceable and `[++i, s)` denotes a range.  
 (2.3) — `assignable_from<I&, S>` is either modeled or not satisfied.



(2.4) — If `I` and `S` are the same type, then `i == i` is `true`.

- <sup>3</sup> The domain of `==` is not static. Given an iterator `i` and sentinel `s` such that `[i, s)` denotes a range and `i != s`, `i` and `s` are not required to continue to denote a range after incrementing any other iterator equal to `i`. Consequently, `i == s` is no longer required to be well-defined.

#### 24.3.4.8 Concept `sized_sentinel_for`

[iterator.concept.sizedsentinel]

- <sup>1</sup> The `sized_sentinel_for` concept specifies requirements on an `input_or_output_iterator` type `I` and a corresponding `sentinel_for<I>` that allow the use of the `-` operator to compute the distance between them in constant time.

```
template<class S, class I>
concept sized_sentinel_for =
 sentinel_for<S, I> &&
 !disable_sized_sentinel_for<remove_cv_t<S>, remove_cv_t<I>>> &&
 requires(const I& i, const S& s) {
 { s - i } -> same_as<iter_difference_t<I>>;
 { i - s } -> same_as<iter_difference_t<I>>;
 };
```

- <sup>2</sup> Let `i` be an iterator of type `I`, and `s` a sentinel of type `S` such that `[i, s)` denotes a range. Let `N` be the smallest number of applications of `++i` necessary to make `bool(i == s)` be `true`. `S` and `I` model `sized_sentinel_for<S, I>` only if:

(2.1) — If `N` is representable by `iter_difference_t<I>`, then `s - i` is well-defined and equals `N`.

(2.2) — If `-N` is representable by `iter_difference_t<I>`, then `i - s` is well-defined and equals `-N`.

```
template<class S, class I>
constexpr bool disable_sized_sentinel_for = false;
```

- <sup>3</sup> *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `disable_sized_sentinel_for` for cv-unqualified non-array object types `S` and `I` if `S` and/or `I` is a program-defined type. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.

- <sup>4</sup> [Note 1: `disable_sized_sentinel_for` allows use of sentinels and iterators with the library that satisfy but do not in fact model `sized_sentinel_for`. — end note]

- <sup>5</sup> [Example 1: The `sized_sentinel_for` concept is modeled by pairs of `random_access_iterators` (24.3.4.13) and by counted iterators and their sentinels (24.5.7.1). — end example]

#### 24.3.4.9 Concept `input_iterator`

[iterator.concept.input]

- <sup>1</sup> The `input_iterator` concept defines requirements for a type whose referenced values can be read (from the requirement for `indirectly_readable` (24.3.4.2)) and which can be both pre- and post-incremented.

[Note 1: Unlike the `Cpp17InputIterator` requirements (24.3.5.3), the `input_iterator` concept does not need equality comparison since iterators are typically compared to sentinels. — end note]

```
template<class I>
concept input_iterator =
 input_or_output_iterator<I> &&
 indirectly_readable<I> &&
 requires { typename ITER_CONCEPT(I); } &&
 derived_from<ITER_CONCEPT(I), input_iterator_tag>;
```

#### 24.3.4.10 Concept `output_iterator`

[iterator.concept.output]

- <sup>1</sup> The `output_iterator` concept defines requirements for a type that can be used to write values (from the requirement for `indirectly_writable` (24.3.4.3)) and which can be both pre- and post-incremented.

[Note 1: Output iterators are not required to model `equality_comparable`. — end note]

```
template<class I, class T>
concept output_iterator =
 input_or_output_iterator<I> &&
 indirectly_writable<I, T> &&
 requires(I i, T&& t) {
 *i++ = std::forward<T>(t); // not required to be equality-preserving
 };
```

- <sup>2</sup> Let *E* be an expression such that `decltype(E)` is *T*, and let *i* be a dereferenceable object of type *I*. *I* and *T* model `output_iterator<I, T>` only if `*i++ = E`; has effects equivalent to:

```
*i = E;
++i;
```

- <sup>3</sup> *Recommended practice*: The implementation of an algorithm on output iterators should never attempt to pass through the same iterator twice; such an algorithm should be a single-pass algorithm.

#### 24.3.4.11 Concept `forward_iterator`

[`iterator.concept.forward`]

- <sup>1</sup> The `forward_iterator` concept adds copyability, equality comparison, and the multi-pass guarantee, specified below.

```
template<class I>
concept forward_iterator =
 input_iterator<I> &&
 derived_from<ITER_CONCEPT(I), forward_iterator_tag> &&
 incrementable<I> &&
 sentinel_for<I, I>;
```

- <sup>2</sup> The domain of `==` for forward iterators is that of iterators over the same underlying sequence. However, value-initialized iterators of the same type may be compared and shall compare equal to other value-initialized iterators of the same type.

[*Note 1*: Value-initialized iterators behave as if they refer past the end of the same empty sequence. — *end note*]

- <sup>3</sup> Pointers and references obtained from a forward iterator into a range [*i*, *s*) shall remain valid while [*i*, *s*) continues to denote a range.

- <sup>4</sup> Two dereferenceable iterators *a* and *b* of type *X* offer the *multi-pass guarantee* if

(4.1) — *a* == *b* implies `++a == ++b` and

(4.2) — the expression `((void) [] (X x){++x;})(a), *a` is equivalent to the expression `*a`.

- <sup>5</sup> [*Note 2*: The requirement that *a* == *b* implies `++a == ++b` and the removal of the restrictions on the number of assignments through a mutable iterator (which applies to output iterators) allow the use of multi-pass one-directional algorithms with forward iterators. — *end note*]

#### 24.3.4.12 Concept `bidirectional_iterator`

[`iterator.concept.bidir`]

- <sup>1</sup> The `bidirectional_iterator` concept adds the ability to move an iterator backward as well as forward.

```
template<class I>
concept bidirectional_iterator =
 forward_iterator<I> &&
 derived_from<ITER_CONCEPT(I), bidirectional_iterator_tag> &&
 requires(I i) {
 { --i } -> same_as<I&>;
 { i-- } -> same_as<I>;
 };
};
```

- <sup>2</sup> A bidirectional iterator *r* is decrementable if and only if there exists some *q* such that `++q == r`. Decrementable iterators *r* shall be in the domain of the expressions `--r` and `r--`.

- <sup>3</sup> Let *a* and *b* be equal objects of type *I*. *I* models `bidirectional_iterator` only if:

(3.1) — If *a* and *b* are decrementable, then all of the following are `true`:

(3.1.1) — `addressof(--a) == addressof(a)`

(3.1.2) — `bool(a-- == b)`

(3.1.3) — after evaluating both `a--` and `--b`, `bool(a == b)` is still `true`

(3.1.4) — `bool(++(--a) == b)`

(3.2) — If *a* and *b* are incrementable, then `bool(--(++a) == b)`.

#### 24.3.4.13 Concept `random_access_iterator`

[`iterator.concept.random.access`]

- <sup>1</sup> The `random_access_iterator` concept adds support for constant-time advancement with `+=`, `+`, `-=`, and `-`, as well as the computation of distance in constant time with `-`. Random access iterators also support array notation via subscripting.



```

template<class I>
concept random_access_iterator =
 bidirectional_iterator<I> &&
 derived_from<ITER_CONCEPT(I), random_access_iterator_tag> &&
 totally_ordered<I> &&
 sized_sentinel_for<I, I> &&
 requires(I i, const I j, const iter_difference_t<I> n) {
 { i += n } -> same_as<I&>;
 { j + n } -> same_as<I>;
 { n + j } -> same_as<I>;
 { i -= n } -> same_as<I&>;
 { j - n } -> same_as<I>;
 { j[n] } -> same_as<iter_reference_t<I>>;
 };

```

- <sup>2</sup> Let *a* and *b* be valid iterators of type *I* such that *b* is reachable from *a* after *n* applications of *++a*, let *D* be *iter\_difference\_t<I>*, and let *n* denote a value of type *D*. *I* models *random\_access\_iterator* only if:

- (2.1) — (*a* += *n*) is equal to *b*.
- (2.2) — *addressof*(*a* += *n*) is equal to *addressof*(*a*).
- (2.3) — (*a* + *n*) is equal to (*a* += *n*).
- (2.4) — For any two positive values *x* and *y* of type *D*, if (*a* + *D*(*x* + *y*)) is valid, then (*a* + *D*(*x* + *y*)) is equal to ((*a* + *x*) + *y*).
- (2.5) — (*a* + *D*(0)) is equal to *a*.
- (2.6) — If (*a* + *D*(*n* - 1)) is valid, then (*a* + *n*) is equal to [](*I* c){ return ++c; }(a + *D*(*n* - 1)).
- (2.7) — (*b* += *D*(-*n*)) is equal to *a*.
- (2.8) — (*b* -= *n*) is equal to *a*.
- (2.9) — *addressof*(*b* -= *n*) is equal to *addressof*(*b*).
- (2.10) — (*b* - *n*) is equal to (*b* -= *n*).
- (2.11) — If *b* is dereferenceable, then *a*[*n*] is valid and is equal to \**b*.
- (2.12) — *bool*(*a* <= *b*) is true.

#### 24.3.4.14 Concept *contiguous\_iterator*

[iterator.concept.contiguous]

- <sup>1</sup> The *contiguous\_iterator* concept provides a guarantee that the denoted elements are stored contiguously in memory.

```

template<class I>
concept contiguous_iterator =
 random_access_iterator<I> &&
 derived_from<ITER_CONCEPT(I), contiguous_iterator_tag> &&
 is_lvalue_reference_v<iter_reference_t<I>> &&
 same_as<iter_value_t<I>, remove_cvref_t<iter_reference_t<I>>> &&
 requires(const I& i) {
 { to_address(i) } -> same_as<add_pointer_t<iter_reference_t<I>>>;
 };

```

- <sup>2</sup> Let *a* and *b* be dereferenceable iterators and *c* be a non-dereferenceable iterator of type *I* such that *b* is reachable from *a* and *c* is reachable from *b*, and let *D* be *iter\_difference\_t<I>*. The type *I* models *contiguous\_iterator* only if

- (2.1) — *to\_address*(*a*) == *addressof*(\**a*),
- (2.2) — *to\_address*(*b*) == *to\_address*(*a*) + *D*(*b* - *a*),
- (2.3) — *to\_address*(*c*) == *to\_address*(*a*) + *D*(*c* - *a*),
- (2.4) — *to\_address*(*I*{}) is well-defined,
- (2.5) — *ranges::iter\_move*(*a*) has the same type, value category, and effects as *std::move*(\**a*), and
- (2.6) — if *ranges::iter\_swap*(*a*, *b*) is well-formed, it has effects equivalent to *ranges::swap*(\**a*, \**b*).

**24.3.5 C++17 iterator requirements****[iterator.cpp17]****24.3.5.1 General****[iterator.cpp17.general]**

- <sup>1</sup> In the following sections, *a* and *b* denote values of type *X* or `const X`, `difference_type` and `reference` refer to the types `iterator_traits<X>::difference_type` and `iterator_traits<X>::reference`, respectively, *n* denotes a value of `difference_type`, *u*, *tmp*, and *m* denote identifiers, *r* denotes a value of *X*&, *t* denotes a value of value type *T*, *o* denotes a value of some type that is writable to the output iterator.

[*Note 1*: For an iterator type *X* there must be an instantiation of `iterator_traits<X>` (24.3.2.3). — *end note*]

**24.3.5.2 Cpp17Iterator****[iterator.iterators]**

- <sup>1</sup> The *Cpp17Iterator* requirements form the basis of the iterator taxonomy; every iterator meets the *Cpp17Iterator* requirements. This set of requirements specifies operations for dereferencing and incrementing an iterator. Most algorithms will require additional operations to read (24.3.5.3) or write (24.3.5.4) values, or to provide a richer set of iterator movements (24.3.5.5, 24.3.5.6, 24.3.5.7).
- <sup>2</sup> A type *X* meets the *Cpp17Iterator* requirements if
- (2.1) — *X* meets the *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, *Cpp17Swappable*, and *Cpp17Destructible* requirements (16.4.4.2, 16.4.4.3), and
  - (2.2) — `iterator_traits<X>::difference_type` is a signed integer type or `void`, and
  - (2.3) — the expressions in Table 78 are valid and have the indicated semantics.

**Table 78 — Cpp17Iterator requirements** [tab:iterator]

| Expression       | Return type | Operational semantics | Assertion/note pre-/post-condition                  |
|------------------|-------------|-----------------------|-----------------------------------------------------|
| <code>*r</code>  | unspecified |                       | <i>Preconditions</i> : <i>r</i> is dereferenceable. |
| <code>++r</code> | <i>X</i> &  |                       |                                                     |

**24.3.5.3 Input iterators****[input.iterators]**

- <sup>1</sup> A class or pointer type *X* meets the requirements of an input iterator for the value type *T* if *X* meets the *Cpp17Iterator* (24.3.5.2) and *Cpp17EqualityComparable* (Table 28) requirements and the expressions in Table 79 are valid and have the indicated semantics.
- <sup>2</sup> In Table 79, the term *the domain of ==* is used in the ordinary mathematical sense to denote the set of values over which `==` is (required to be) defined. This set can change over time. Each algorithm places additional requirements on the domain of `==` for the iterator values it uses. These requirements can be inferred from the uses that algorithm makes of `==` and `!=`.

[*Example 1*: The call `find(a,b,x)` is defined only if the value of *a* has the property *p* defined as follows: *b* has property *p* and a value *i* has property *p* if `(*i==x)` or if `(*i!=x` and `++i` has property *p*). — *end example*]

**Table 79 — Cpp17InputIterator requirements (in addition to Cpp17Iterator)** [tab:inputiterator]

| Expression          | Return type                                                  | Operational semantics  | Assertion/note pre-/post-condition                                                   |
|---------------------|--------------------------------------------------------------|------------------------|--------------------------------------------------------------------------------------|
| <code>a != b</code> | <code>decltype(a != b)</code> models <i>boolean-testable</i> | <code>!(a == b)</code> | <i>Preconditions</i> : ( <i>a</i> , <i>b</i> ) is in the domain of <code>==</code> . |

Table 79 — *Cpp17InputIterator* requirements (in addition to *Cpp17Iterator*) (continued)

| Expression        | Return type                    | Operational semantics                                    | Assertion/note pre-/post-condition                                                                                                                                                                                                                                                     |
|-------------------|--------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>*a</b>         | reference,<br>convertible to T |                                                          | <i>Preconditions:</i> <b>a</b> is<br>dereferenceable.<br>The expression<br>(void)* <b>a</b> , * <b>a</b> is equivalent<br>to * <b>a</b> .<br>If <b>a</b> == <b>b</b> and ( <b>a</b> , <b>b</b> ) is in the<br>domain of == then * <b>a</b> is<br>equivalent to * <b>b</b> .            |
| <b>a-&gt;m</b>    |                                | ( <b>*a</b> ).m                                          | <i>Preconditions:</i> <b>a</b> is<br>dereferenceable.                                                                                                                                                                                                                                  |
| <b>++r</b>        | <b>X&amp;</b>                  |                                                          | <i>Preconditions:</i> <b>r</b> is<br>dereferenceable.<br><i>Postconditions:</i> <b>r</b> is<br>dereferenceable or <b>r</b> is<br>past-the-end;<br>any copies of the previous<br>value of <b>r</b> are no longer<br>required to be<br>dereferenceable nor to be in<br>the domain of ==. |
| (void) <b>r++</b> |                                |                                                          | equivalent to (void) <b>++r</b>                                                                                                                                                                                                                                                        |
| <b>*r++</b>       | convertible to T               | { T tmp = * <b>r</b> ;<br>++ <b>r</b> ;<br>return tmp; } |                                                                                                                                                                                                                                                                                        |

- <sup>3</sup> *Recommended practice:* The implementation of an algorithm on input iterators should never attempt to pass through the same iterator twice; such an algorithm should be a single pass algorithm.

[*Note 1:* For input iterators, **a** == **b** does not imply ++**a** == ++**b**. (Equality does not guarantee the substitution property or referential transparency.) Value type T is not required to be a *Cpp17CopyAssignable* type (Table 34). Such an algorithm can be used with istreams as the source of the input data through the `istream_iterator` class template. — end note]

#### 24.3.5.4 Output iterators

[output.iterators]

- <sup>1</sup> A class or pointer type **X** meets the requirements of an output iterator if **X** meets the *Cpp17Iterator* requirements (24.3.5.2) and the expressions in Table 80 are valid and have the indicated semantics.
- <sup>2</sup> *Recommended practice:* The implementation of an algorithm on output iterators should never attempt to pass through the same iterator twice; such an algorithm should be a single-pass algorithm.

[*Note 1:* The only valid use of an `operator*` is on the left side of the assignment statement. Assignment through the same value of the iterator happens only once. Equality and inequality are not necessarily defined. — end note]

#### 24.3.5.5 Forward iterators

[forward.iterators]

- <sup>1</sup> A class or pointer type **X** meets the *Cpp17ForwardIterator* requirements if
- (1.1) — **X** meets the *Cpp17InputIterator* requirements (24.3.5.3),
  - (1.2) — **X** meets the *Cpp17DefaultConstructible* requirements (16.4.4.2),
  - (1.3) — if **X** is a mutable iterator, **reference** is a reference to T; if **X** is a constant iterator, **reference** is a reference to `const T`,
  - (1.4) — the expressions in Table 81 are valid and have the indicated semantics, and
  - (1.5) — objects of type **X** offer the multi-pass guarantee, described below.
- <sup>2</sup> The domain of == for forward iterators is that of iterators over the same underlying sequence. However, value-initialized iterators may be compared and shall compare equal to other value-initialized iterators of the same type.

Table 80 — *Cpp17OutputIterator* requirements (in addition to *Cpp17Iterator*)  
[tab:outputiterator]

| Expression            | Return type                              | Operational semantics                                | Assertion/note pre-/post-condition                                                                                                                                                                     |
|-----------------------|------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>*r = o</code>   | result is not used                       |                                                      | <i>Remarks:</i> After this operation <code>r</code> is not required to be dereferenceable.<br><i>Postconditions:</i> <code>r</code> is incrementable.                                                  |
| <code>++r</code>      | <code>X&amp;</code>                      |                                                      | <code>addressof(r) == addressof(++r)</code> .<br><i>Remarks:</i> After this operation <code>r</code> is not required to be dereferenceable.<br><i>Postconditions:</i> <code>r</code> is incrementable. |
| <code>r++</code>      | convertible to <code>const X&amp;</code> | <code>{ X tmp = r;<br/>++r;<br/>return tmp; }</code> | <i>Remarks:</i> After this operation <code>r</code> is not required to be dereferenceable.<br><i>Postconditions:</i> <code>r</code> is incrementable.                                                  |
| <code>*r++ = o</code> | result is not used                       |                                                      | <i>Remarks:</i> After this operation <code>r</code> is not required to be dereferenceable.<br><i>Postconditions:</i> <code>r</code> is incrementable.                                                  |

[Note 1: Value-initialized iterators behave as if they refer past the end of the same empty sequence. — end note]

<sup>3</sup> Two dereferenceable iterators `a` and `b` of type `X` offer the *multi-pass guarantee* if

(3.1) — `a == b` implies `++a == ++b` and

(3.2) — `X` is a pointer type or the expression `(void)++X(a)`, `*a` is equivalent to the expression `*a`.

<sup>4</sup> [Note 2: The requirement that `a == b` implies `++a == ++b` (which is not true for input and output iterators) and the removal of the restrictions on the number of the assignments through a mutable iterator (which applies to output iterators) allows the use of multi-pass one-directional algorithms with forward iterators. — end note]

Table 81 — *Cpp17ForwardIterator* requirements (in addition to *Cpp17InputIterator*)  
[tab:forwarditerator]

| Expression        | Return type                              | Operational semantics                                | Assertion/note pre-/post-condition |
|-------------------|------------------------------------------|------------------------------------------------------|------------------------------------|
| <code>r++</code>  | convertible to <code>const X&amp;</code> | <code>{ X tmp = r;<br/>++r;<br/>return tmp; }</code> |                                    |
| <code>*r++</code> | reference                                |                                                      |                                    |

<sup>5</sup> If `a` and `b` are equal, then either `a` and `b` are both dereferenceable or else neither is dereferenceable.

<sup>6</sup> If `a` and `b` are both dereferenceable, then `a == b` if and only if `*a` and `*b` are bound to the same object.

#### 24.3.5.6 Bidirectional iterators

[bidirectional.iterators]

<sup>1</sup> A class or pointer type `X` meets the requirements of a bidirectional iterator if, in addition to meeting the *Cpp17ForwardIterator* requirements, the following expressions are valid as shown in Table 82.

<sup>2</sup> [Note 1: Bidirectional iterators allow algorithms to move iterators backward as well as forward. — end note]

Table 82 — *Cpp17BidirectionalIterator* requirements (in addition to *Cpp17ForwardIterator*)  
[tab:bidirectionaliterator]

| Expression | Return type                | Operational semantics                 | Assertion/note pre-/post-condition                                                                                                                                                                                                              |
|------------|----------------------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| --r        | X&                         |                                       | <i>Preconditions:</i> there exists <b>s</b> such that <b>r == ++s</b> .<br><i>Postconditions:</i> <b>r</b> is dereferenceable.<br><b>--(++r) == r</b> .<br><b>--r == --s</b> implies <b>r == s</b> .<br><b>addressof(r) == addressof(--r)</b> . |
| r--        | convertible to<br>const X& | { X tmp = r;<br>--r;<br>return tmp; } |                                                                                                                                                                                                                                                 |
| *r--       | reference                  |                                       |                                                                                                                                                                                                                                                 |

#### 24.3.5.7 Random access iterators

[random.access.iterators]

- <sup>1</sup> A class or pointer type **X** meets the requirements of a random access iterator if, in addition to meeting the *Cpp17BidirectionalIterator* requirements, the following expressions are valid as shown in Table 83.

Table 83 — *Cpp17RandomAccessIterator* requirements (in addition to *Cpp17BidirectionalIterator*) [tab:randomaccessiterator]

| Expression     | Return type                                       | Operational semantics                                                                                        | Assertion/note pre-/post-condition                                                                                                            |
|----------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| r += n         | X&                                                | { difference_type m = n;<br>if (m >= 0)<br>while (m--)<br>++r;<br>else<br>while (m++)<br>--r;<br>return r; } |                                                                                                                                               |
| a + n<br>n + a | X                                                 | { X tmp = a;<br>return tmp += n; }                                                                           | a + n == n + a.                                                                                                                               |
| r -= n         | X&                                                | return r += -n;                                                                                              | <i>Preconditions:</i> the absolute value of <b>n</b> is in the range of representable values of <b>difference_type</b> .                      |
| a - n          | X                                                 | { X tmp = a;<br>return tmp -= n; }                                                                           |                                                                                                                                               |
| b - a          | difference_type                                   | return n;                                                                                                    | <i>Preconditions:</i> there exists a value <b>n</b> of type <b>difference_type</b> such that <b>a + n == b</b> .<br><b>b == a + (b - a)</b> . |
| a[n]           | convertible to<br>reference                       | *(a + n)                                                                                                     |                                                                                                                                               |
| a < b          | decltype(a < b) models<br><i>boolean-testable</i> | <i>Effects:</i> Equivalent to: return b - a > 0;                                                             | < is a total ordering relation                                                                                                                |

Table 83 — *Cpp17RandomAccessIterator* requirements (in addition to *Cpp17BidirectionalIterator*) (continued)

| Expression             | Return type                                                     | Operational semantics    | Assertion/note pre-/post-condition                                             |
|------------------------|-----------------------------------------------------------------|--------------------------|--------------------------------------------------------------------------------|
| <code>a &gt; b</code>  | <code>decltype(a &gt; b)</code> models <i>boolean-testable</i>  | <code>b &lt; a</code>    | <code>&gt;</code> is a total ordering relation opposite to <code>&lt;</code> . |
| <code>a &gt;= b</code> | <code>decltype(a &gt;= b)</code> models <i>boolean-testable</i> | <code>!(a &lt; b)</code> |                                                                                |
| <code>a &lt;= b</code> | <code>decltype(a &lt;= b)</code> models <i>boolean-testable</i> | <code>!(a &gt; b)</code> |                                                                                |

**24.3.6 Indirect callable requirements**

[indirectcallable]

**24.3.6.1 General**

[indirectcallable.general]

- <sup>1</sup> There are several concepts that group requirements of algorithms that take callable objects (22.10.3) as arguments.

**24.3.6.2 Indirect callable traits**

[indirectcallable.traits]

- <sup>1</sup> To implement algorithms taking projections, it is necessary to determine the projected type of an iterator's value type. For the exposition-only alias template *indirect-value-t*, *indirect-value-t*<T> denotes

- (1.1) — `invoke_result_t<Proj&, indirect-value-t<I>>` if T names `projected<I, Proj>`, and  
 (1.2) — `iter_value_t<T>&` otherwise.

**24.3.6.3 Indirect callables**

[indirectcallable.indirectinvocable]

- <sup>1</sup> The indirect callable concepts are used to constrain those algorithms that accept callable objects (22.10.3) as arguments.

```

namespace std {
 template<class F, class I>
 concept indirectly_unary_invocable =
 indirectly_readable<I> &&
 copy_constructible<F> &&
 invocable<F&, indirect-value-t<I>> &&
 invocable<F&, iter_reference_t<I>> &&
 common_reference_with<
 invoke_result_t<F&, indirect-value-t<I>>,
 invoke_result_t<F&, iter_reference_t<I>>>;

 template<class F, class I>
 concept indirectly_regular_unary_invocable =
 indirectly_readable<I> &&
 copy_constructible<F> &&
 regular_invocable<F&, indirect-value-t<I>> &&
 regular_invocable<F&, iter_reference_t<I>> &&
 common_reference_with<
 invoke_result_t<F&, indirect-value-t<I>>,
 invoke_result_t<F&, iter_reference_t<I>>>;

 template<class F, class I>
 concept indirect_unary_predicate =
 indirectly_readable<I> &&
 copy_constructible<F> &&
 predicate<F&, indirect-value-t<I>> &&
 predicate<F&, iter_reference_t<I>>;

```

```

template<class F, class I1, class I2>
concept indirectly_binary_predicate =
 indirectly_readable<I1> && indirectly_readable<I2> &&
 copy_constructible<F> &&
 predicate<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
 predicate<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
 predicate<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
 predicate<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

template<class F, class I1, class I2 = I1>
concept indirectly_equivalence_relation =
 indirectly_readable<I1> && indirectly_readable<I2> &&
 copy_constructible<F> &&
 equivalence_relation<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
 equivalence_relation<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
 equivalence_relation<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
 equivalence_relation<F&, iter_reference_t<I1>, iter_reference_t<I2>>;

template<class F, class I1, class I2 = I1>
concept indirect_strict_weak_order =
 indirectly_readable<I1> && indirectly_readable<I2> &&
 copy_constructible<F> &&
 strict_weak_order<F&, indirect_value_t<I1>, indirect_value_t<I2>> &&
 strict_weak_order<F&, indirect_value_t<I1>, iter_reference_t<I2>> &&
 strict_weak_order<F&, iter_reference_t<I1>, indirect_value_t<I2>> &&
 strict_weak_order<F&, iter_reference_t<I1>, iter_reference_t<I2>>;
}

```

#### 24.3.6.4 Alias template projected

[projected]

- <sup>1</sup> Alias template `projected` is used to constrain algorithms that accept callable objects and projections (3.44). It combines an `indirectly_readable` type `I` and a callable object type `Proj` into a new `indirectly_readable` type whose reference type is the result of applying `Proj` to the `iter_reference_t` of `I`.

```

namespace std {
 template<class I, class Proj>
 struct projected_impl {
 struct type {
 using value_type = remove_cvref_t<indirect_result_t<Proj&, I>>;
 using difference_type = iter_difference_t<I>;
 indirect_result_t<Proj&, I> operator*() const;
 };
 };

 template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
 using projected = projected_impl<I, Proj>::type;
}

```

### 24.3.7 Common algorithm requirements

[alg.req]

#### 24.3.7.1 General

[alg.req.general]

- <sup>1</sup> There are several additional iterator concepts that are commonly applied to families of algorithms. These group together iterator requirements of algorithm families. There are three relational concepts that specify how element values are transferred between `indirectly_readable` and `indirectly_writable` types: `indirectly_movable`, `indirectly_copyable`, and `indirectly_swappable`. There are three relational concepts for rearrangements: `permutable`, `mergeable`, and `sortable`. There is one relational concept for comparing values from different sequences: `indirectly_comparable`.
- <sup>2</sup> [Note 1: The `ranges::less` function object type used in the concepts below imposes constraints on the concepts' arguments in addition to those that appear in the concepts' bodies (22.10.9). — end note]

#### 24.3.7.2 Concept `indirectly_movable`

[alg.req.ind.move]

- <sup>1</sup> The `indirectly_movable` concept specifies the relationship between an `indirectly_readable` type and an `indirectly_writable` type between which values may be moved.

```
template<class In, class Out>
concept indirectly_movable =
 indirectly_readable<In> &&
 indirectly_writable<Out, iter_rvalue_reference_t<In>>;
```

- <sup>2</sup> The `indirectly_movable_storable` concept augments `indirectly_movable` with additional requirements enabling the transfer to be performed through an intermediate object of the `indirectly_readable` type's value type.

```
template<class In, class Out>
concept indirectly_movable_storable =
 indirectly_movable<In, Out> &&
 indirectly_writable<Out, iter_value_t<In>> &&
 movable<iter_value_t<In>> &&
 constructible_from<iter_value_t<In>, iter_rvalue_reference_t<In>> &&
 assignable_from<iter_value_t<In>&, iter_rvalue_reference_t<In>>;
```

- <sup>3</sup> Let `i` be a dereferenceable value of type `In`. `In` and `Out` model `indirectly_movable_storable<In, Out>` only if after the initialization of the object `obj` in

```
iter_value_t<In> obj(ranges::iter_move(i));
```

`obj` is equal to the value previously denoted by `*i`. If `iter_rvalue_reference_t<In>` is an rvalue reference type, the resulting state of the value denoted by `*i` is valid but unspecified (16.4.6.17).

### 24.3.7.3 Concept `indirectly_copyable`

[alg.req.ind.copy]

- <sup>1</sup> The `indirectly_copyable` concept specifies the relationship between an `indirectly_readable` type and an `indirectly_writable` type between which values may be copied.

```
template<class In, class Out>
concept indirectly_copyable =
 indirectly_readable<In> &&
 indirectly_writable<Out, iter_reference_t<In>>;
```

- <sup>2</sup> The `indirectly_copyable_storable` concept augments `indirectly_copyable` with additional requirements enabling the transfer to be performed through an intermediate object of the `indirectly_readable` type's value type. It also requires the capability to make copies of values.

```
template<class In, class Out>
concept indirectly_copyable_storable =
 indirectly_copyable<In, Out> &&
 indirectly_writable<Out, iter_value_t<In>&> &&
 indirectly_writable<Out, const iter_value_t<In>&> &&
 indirectly_writable<Out, iter_value_t<In>&&> &&
 indirectly_writable<Out, const iter_value_t<In>&&> &&
 copyable<iter_value_t<In>> &&
 constructible_from<iter_value_t<In>, iter_reference_t<In>> &&
 assignable_from<iter_value_t<In>&, iter_reference_t<In>>;
```

- <sup>3</sup> Let `i` be a dereferenceable value of type `In`. `In` and `Out` model `indirectly_copyable_storable<In, Out>` only if after the initialization of the object `obj` in

```
iter_value_t<In> obj(*i);
```

`obj` is equal to the value previously denoted by `*i`. If `iter_reference_t<In>` is an rvalue reference type, the resulting state of the value denoted by `*i` is valid but unspecified (16.4.6.17).

### 24.3.7.4 Concept `indirectly_swappable`

[alg.req.ind.swap]

- <sup>1</sup> The `indirectly_swappable` concept specifies a swappable relationship between the values referenced by two `indirectly_readable` types.

```
template<class I1, class I2 = I1>
concept indirectly_swappable =
 indirectly_readable<I1> && indirectly_readable<I2> &&
 requires(const I1 i1, const I2 i2) {
 ranges::iter_swap(i1, i1);
 ranges::iter_swap(i2, i2);
 ranges::iter_swap(i1, i2);
 ranges::iter_swap(i2, i1);
 }
```



};

**24.3.7.5 Concept indirectly\_comparable****[alg.req.ind.cmp]**

- <sup>1</sup> The `indirectly_comparable` concept specifies the common requirements of algorithms that compare values from two different sequences.

```
template<class I1, class I2, class R, class P1 = identity,
 class P2 = identity>
concept indirectly_comparable =
 indirect_binary_predicate<R, projected<I1, P1>, projected<I2, P2>>;
```

**24.3.7.6 Concept permutable****[alg.req.permutable]**

- <sup>1</sup> The `permutable` concept specifies the common requirements of algorithms that reorder elements in place by moving or swapping them.

```
template<class I>
concept permutable =
 forward_iterator<I> &&
 indirectly_movable_storable<I, I> &&
 indirectly_swappable<I, I>;
```

**24.3.7.7 Concept mergeable****[alg.req.mergeable]**

- <sup>1</sup> The `mergeable` concept specifies the requirements of algorithms that merge sorted sequences into an output sequence by copying elements.

```
template<class I1, class I2, class Out, class R = ranges::less,
 class P1 = identity, class P2 = identity>
concept mergeable =
 input_iterator<I1> &&
 input_iterator<I2> &&
 weakly_incrementable<Out> &&
 indirectly_copyable<I1, Out> &&
 indirectly_copyable<I2, Out> &&
 indirect_strict_weak_order<R, projected<I1, P1>, projected<I2, P2>>;
```

**24.3.7.8 Concept sortable****[alg.req.sortable]**

- <sup>1</sup> The `sortable` concept specifies the common requirements of algorithms that permute sequences into ordered sequences (e.g., `sort`).

```
template<class I, class R = ranges::less, class P = identity>
concept sortable =
 permutable<I> &&
 indirect_strict_weak_order<R, projected<I, P>>;
```

**24.4 Iterator primitives****[iterator.primitives]****24.4.1 General****[iterator.primitives.general]**

- <sup>1</sup> To simplify the use of iterators, the library provides several classes and functions.

**24.4.2 Standard iterator tags****[std.iterator.tags]**

- <sup>1</sup> It is often desirable for a function template specialization to find out what is the most specific category of its iterator argument, so that the function can select the most efficient algorithm at compile time. To facilitate this, the library introduces *category tag* classes which are used as compile time tags for algorithm selection. They are: `output_iterator_tag`, `input_iterator_tag`, `forward_iterator_tag`, `bidirectional_iterator_tag`, `random_access_iterator_tag`, and `contiguous_iterator_tag`. For every iterator of type `I`, `iterator_traits<I>::iterator_category` shall be declared to be a category tag that describes the iterator's behavior. Additionally, `iterator_traits<I>::iterator_concept` may be used to indicate conformance to the iterator concepts (24.3.4).

```
namespace std {
 struct output_iterator_tag { };
 struct input_iterator_tag { };
 struct forward_iterator_tag: public input_iterator_tag { };
 struct bidirectional_iterator_tag: public forward_iterator_tag { };
}
```

```

 struct random_access_iterator_tag: public bidirectional_iterator_tag { };
 struct contiguous_iterator_tag: public random_access_iterator_tag { };
}

```

- <sup>2</sup> [Example 1: A program-defined iterator `BinaryTreeIterator` can be included into the bidirectional iterator category by specializing the `iterator_traits` template:

```

template<class T> struct iterator_traits<BinaryTreeIterator<T>> {
 using iterator_category = bidirectional_iterator_tag;
 using difference_type = ptrdiff_t;
 using value_type = T;
 using pointer = T*;
 using reference = T&;
};

```

— end example]

- <sup>3</sup> [Example 2: If `evolve()` is well-defined for bidirectional iterators, but can be implemented more efficiently for random access iterators, then the implementation is as follows:

```

template<class BidirectionalIterator>
inline void
evolve(BidirectionalIterator first, BidirectionalIterator last) {
 evolve(first, last,
 typename iterator_traits<BidirectionalIterator>::iterator_category());
}

template<class BidirectionalIterator>
void evolve(BidirectionalIterator first, BidirectionalIterator last,
 bidirectional_iterator_tag) {
 // more generic, but less efficient algorithm
}

template<class RandomAccessIterator>
void evolve(RandomAccessIterator first, RandomAccessIterator last,
 random_access_iterator_tag) {
 // more efficient, but less generic algorithm
}

```

— end example]

### 24.4.3 Iterator operations

[iterator.operations]

- <sup>1</sup> Since only random access iterators provide `+` and `-` operators, the library provides two function templates `advance` and `distance`. These function templates use `+` and `-` for random access iterators (and are, therefore, constant time for them); for input, forward and bidirectional iterators they use `++` to provide linear time implementations.

```

template<class InputIterator, class Distance>
constexpr void advance(InputIterator& i, Distance n);

```

- <sup>2</sup>     *Preconditions:* `n` is negative only for bidirectional iterators.
- <sup>3</sup>     *Effects:* Increments `i` by `n` if `n` is non-negative, and decrements `i` by `-n` otherwise.

```

template<class InputIterator>
constexpr typename iterator_traits<InputIterator>::difference_type
distance(InputIterator first, InputIterator last);

```

- <sup>4</sup>     *Preconditions:* `last` is reachable from `first`, or `InputIterator` meets the *Cpp17RandomAccessIterator* requirements and `first` is reachable from `last`.
- <sup>5</sup>     *Effects:* If `InputIterator` meets the *Cpp17RandomAccessIterator* requirements, returns `(last - first)`; otherwise, increments `first` until `last` is reached and returns the number of increments.

```

template<class InputIterator>
constexpr InputIterator next(InputIterator x,
 typename iterator_traits<InputIterator>::difference_type n = 1);

```

- <sup>6</sup>     *Effects:* Equivalent to: `advance(x, n); return x;`

```
template<class BidirectionalIterator>
constexpr BidirectionalIterator prev(BidirectionalIterator x,
 typename iterator_traits<BidirectionalIterator>::difference_type n = 1);
```

7 *Effects:* Equivalent to: `advance(x, -n); return x;`

## 24.4.4 Range iterator operations

[range.iter.ops]

### 24.4.4.1 General

[range.iter.ops.general]

1 The library includes the function templates `ranges::advance`, `ranges::distance`, `ranges::next`, and `ranges::prev` to manipulate iterators. These operations adapt to the set of operators provided by each iterator category to provide the most efficient implementation possible for a concrete iterator type.

[*Example 1:* `ranges::advance` uses the `+` operator to move a `random_access_iterator` forward `n` steps in constant time. For an iterator type that does not model `random_access_iterator`, `ranges::advance` instead performs `n` individual increments with the `++` operator. — *end example*]

2 The entities defined in 24.4.4 are algorithm function objects (16.3.3.4).

### 24.4.4.2 `ranges::advance`

[range.iter.op.advance]

```
template<input_or_output_iterator I>
constexpr void ranges::advance(I& i, iter_difference_t<I> n);
```

1 *Preconditions:* If `I` does not model `bidirectional_iterator`, `n` is not negative.

2 *Effects:*

(2.1) — If `I` models `random_access_iterator`, equivalent to `i += n`.

(2.2) — Otherwise, if `n` is non-negative, increments `i` by `n`.

(2.3) — Otherwise, decrements `i` by `-n`.

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr void ranges::advance(I& i, S bound);
```

3 *Preconditions:* Either `assignable_from<I&, S> || sized_sentinel_for<S, I>` is modeled, or `[i, bound)` denotes a range.

4 *Effects:*

(4.1) — If `I` and `S` model `assignable_from<I&, S>`, equivalent to `i = std::move(bound)`.

(4.2) — Otherwise, if `S` and `I` model `sized_sentinel_for<S, I>`, equivalent to `ranges::advance(i, bound - i)`.

(4.3) — Otherwise, while `bool(i != bound)` is true, increments `i`.

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr iter_difference_t<I> ranges::advance(I& i, iter_difference_t<I> n, S bound);
```

5 *Preconditions:* If `n > 0`, `[i, bound)` denotes a range. If `n == 0`, `[i, bound)` or `[bound, i)` denotes a range. If `n < 0`, `[bound, i)` denotes a range, `I` models `bidirectional_iterator`, and `I` and `S` model `same_as<I, S>`.

6 *Effects:*

(6.1) — If `S` and `I` model `sized_sentinel_for<S, I>`:

(6.1.1) — If  $|n| \geq |\text{bound} - i|$ , equivalent to `ranges::advance(i, bound)`.

(6.1.2) — Otherwise, equivalent to `ranges::advance(i, n)`.

(6.2) — Otherwise,

(6.2.1) — if `n` is non-negative, while `bool(i != bound)` is true, increments `i` but at most `n` times.

(6.2.2) — Otherwise, while `bool(i != bound)` is true, decrements `i` but at most `-n` times.

7 *Returns:* `n - M`, where `M` is the difference between the ending and starting positions of `i`.

**24.4.4.3 ranges::distance****[range.iter.op.distance]**

```
template<class I, sentinel_for<I> S>
requires (!sized_sentinel_for<S, I>)
constexpr iter_difference_t<I> ranges::distance(I first, S last);
```

1     *Preconditions:* [first,last) denotes a range.

2     *Effects:* Increments first until last is reached and returns the number of increments.

```
template<class I, sized_sentinel_for<decay_t<I>> S>
constexpr iter_difference_t<decay_t<I>> ranges::distance(I&& first, S last);
```

3     *Effects:* Equivalent to:

```
 if constexpr (!is_array_v<remove_reference_t<I>>)
 return last - first;
 else
 return last - static_cast<decay_t<I>>(first);
```

```
template<range R>
constexpr range_difference_t<R> ranges::distance(R&& r);
```

4     *Effects:* If R models sized\_range, equivalent to:

```
 return static_cast<range_difference_t<R>>(ranges::size(r)); // 25.3.10
```

Otherwise, equivalent to:

```
 return ranges::distance(ranges::begin(r), ranges::end(r)); // 25.3
```

**24.4.4.4 ranges::next****[range.iter.op.next]**

```
template<input_or_output_iterator I>
constexpr I ranges::next(I x);
```

1     *Effects:* Equivalent to: ++x; return x;

```
template<input_or_output_iterator I>
constexpr I ranges::next(I x, iter_difference_t<I> n);
```

2     *Effects:* Equivalent to: ranges::advance(x, n); return x;

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr I ranges::next(I x, S bound);
```

3     *Effects:* Equivalent to: ranges::advance(x, bound); return x;

```
template<input_or_output_iterator I, sentinel_for<I> S>
constexpr I ranges::next(I x, iter_difference_t<I> n, S bound);
```

4     *Effects:* Equivalent to: ranges::advance(x, n, bound); return x;

**24.4.4.5 ranges::prev****[range.iter.op.prev]**

```
template<bidirectional_iterator I>
constexpr I ranges::prev(I x);
```

1     *Effects:* Equivalent to: --x; return x;

```
template<bidirectional_iterator I>
constexpr I ranges::prev(I x, iter_difference_t<I> n);
```

2     *Effects:* Equivalent to: ranges::advance(x, -n); return x;

```
template<bidirectional_iterator I>
constexpr I ranges::prev(I x, iter_difference_t<I> n, I bound);
```

3     *Effects:* Equivalent to: ranges::advance(x, -n, bound); return x;

**24.5 Iterator adaptors**

[predef.iterators]

**24.5.1 Reverse iterators**

[reverse.iterators]

**24.5.1.1 General**

[reverse.iterators.general]

- <sup>1</sup> Class template `reverse_iterator` is an iterator adaptor that iterates from the end of the sequence defined by its underlying iterator to the beginning of that sequence.

**24.5.1.2 Class template `reverse_iterator`**

[reverse.iterator]

```

namespace std {
 template<class Iterator>
 class reverse_iterator {
 public:
 using iterator_type = Iterator;
 using iterator_concept = see below;
 using iterator_category = see below;
 using value_type = iter_value_t<Iterator>;
 using difference_type = iter_difference_t<Iterator>;
 using pointer = iterator_traits<Iterator>::pointer;
 using reference = iter_reference_t<Iterator>;

 constexpr reverse_iterator();
 constexpr explicit reverse_iterator(Iterator x);
 template<class U> constexpr reverse_iterator(const reverse_iterator<U>& u);
 template<class U> constexpr reverse_iterator& operator=(const reverse_iterator<U>& u);

 constexpr Iterator base() const;
 constexpr reference operator*() const;
 constexpr pointer operator->() const requires see below;

 constexpr reverse_iterator& operator++();
 constexpr reverse_iterator operator++(int);
 constexpr reverse_iterator& operator--();
 constexpr reverse_iterator operator--(int);

 constexpr reverse_iterator operator+ (difference_type n) const;
 constexpr reverse_iterator& operator+=(difference_type n);
 constexpr reverse_iterator operator- (difference_type n) const;
 constexpr reverse_iterator& operator-=(difference_type n);
 constexpr unspecified operator[] (difference_type n) const;

 friend constexpr iter_rvalue_reference_t<Iterator>
 iter_move(const reverse_iterator& i) noexcept(see below);
 template<indirectly_swappable<Iterator> Iterator2>
 friend constexpr void
 iter_swap(const reverse_iterator& x,
 const reverse_iterator<Iterator2>& y) noexcept(see below);

 protected:
 Iterator current;
 };
}

```

- <sup>1</sup> The member *typedef-name* `iterator_concept` denotes

- (1.1) — `random_access_iterator_tag` if `Iterator` models `random_access_iterator`, and
- (1.2) — `bidirectional_iterator_tag` otherwise.

- <sup>2</sup> The member *typedef-name* `iterator_category` denotes

- (2.1) — `random_access_iterator_tag` if the type `iterator_traits<Iterator>::iterator_category` models `derived_from<random_access_iterator_tag>`, and
- (2.2) — `iterator_traits<Iterator>::iterator_category` otherwise.

**24.5.1.3 Requirements****[reverse.iter.requirements]**

- 1 The template parameter `Iterator` shall either meet the requirements of a *Cpp17BidirectionalIterator* (24.3.5.6) or model `bidirectional_iterator` (24.3.4.12).
- 2 Additionally, `Iterator` shall either meet the requirements of a *Cpp17RandomAccessIterator* (24.3.5.7) or model `random_access_iterator` (24.3.4.13) if the definitions of any of the members
- (2.1) — `operator+`, `operator-`, `operator+=`, `operator-=` (24.5.1.7), or
- (2.2) — `operator[]` (24.5.1.6),
- or the non-member operators (24.5.1.8)
- (2.3) — `operator<`, `operator>`, `operator<=`, `operator>=`, `operator-`, or `operator+` (24.5.1.9)
- are instantiated (13.9.2).

**24.5.1.4 Construction and assignment****[reverse.iter.cons]**

```
constexpr reverse_iterator();
```

1     *Effects:* Value-initializes `current`.

```
constexpr explicit reverse_iterator(Iterator x);
```

2     *Effects:* Initializes `current` with `x`.

```
template<class U> constexpr reverse_iterator(const reverse_iterator<U>& u);
```

3     *Constraints:* `is_same_v<U, Iterator>` is false and `const U&` models `convertible_to<Iterator>`.

4     *Effects:* Initializes `current` with `u.current`.

```
template<class U>
constexpr reverse_iterator&
operator=(const reverse_iterator<U>& u);
```

5     *Constraints:* `is_same_v<U, Iterator>` is false, `const U&` models `convertible_to<Iterator>`, and `assignable_from<Iterator&, const U&>` is modeled.

6     *Effects:* Assigns `u.current` to `current`.

7     *Returns:* `*this`.

**24.5.1.5 Conversion****[reverse.iter.conv]**

```
constexpr Iterator base() const;
```

1     *Returns:* `current`.

**24.5.1.6 Element access****[reverse.iter.elem]**

```
constexpr reference operator*() const;
```

1     *Effects:* As if by:

```
 Iterator tmp = current;
 return *--tmp;
```

```
constexpr pointer operator->() const
requires (is_pointer_v<Iterator> ||
requires(const Iterator i) { i.operator->(); });
```

2     *Effects:*

(2.1) — If `Iterator` is a pointer type, equivalent to: `return prev(current);`

(2.2) — Otherwise, equivalent to: `return prev(current).operator->();`

```
constexpr unspecified operator[](difference_type n) const;
```

3     *Returns:* `current[-n - 1]`.

**24.5.1.7 Navigation****[reverse.iter.nav]**

```

constexpr reverse_iterator operator+(difference_type n) const;
1 Returns: reverse_iterator(current - n).

constexpr reverse_iterator operator-(difference_type n) const;
2 Returns: reverse_iterator(current + n).

constexpr reverse_iterator& operator++();
3 Effects: As if by: --current;
4 Returns: *this.

constexpr reverse_iterator operator++(int);
5 Effects: As if by:
 reverse_iterator tmp = *this;
 --current;
 return tmp;

constexpr reverse_iterator& operator--();
6 Effects: As if by ++current.
7 Returns: *this.

constexpr reverse_iterator operator--(int);
8 Effects: As if by:
 reverse_iterator tmp = *this;
 ++current;
 return tmp;

constexpr reverse_iterator& operator+=(difference_type n);
9 Effects: As if by: current -= n;
10 Returns: *this.

constexpr reverse_iterator& operator-=(difference_type n);
11 Effects: As if by: current += n;
12 Returns: *this.

```

**24.5.1.8 Comparisons****[reverse.iter.cmp]**

```

template<class Iterator1, class Iterator2>
constexpr bool operator==(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
1 Constraints: x.base() == y.base() is well-formed and convertible to bool.
2 Returns: x.base() == y.base().

template<class Iterator1, class Iterator2>
constexpr bool operator!=(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
3 Constraints: x.base() != y.base() is well-formed and convertible to bool.
4 Returns: x.base() != y.base().

template<class Iterator1, class Iterator2>
constexpr bool operator<(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
5 Constraints: x.base() > y.base() is well-formed and convertible to bool.

```

6 *Returns:* `x.base() > y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
```

7 *Constraints:* `x.base() < y.base()` is well-formed and convertible to `bool`.

8 *Returns:* `x.base() < y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator<=(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
```

9 *Constraints:* `x.base() >= y.base()` is well-formed and convertible to `bool`.

10 *Returns:* `x.base() >= y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>=(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
```

11 *Constraints:* `x.base() <= y.base()` is well-formed and convertible to `bool`.

12 *Returns:* `x.base() <= y.base()`.

```
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
constexpr compare_three_way_result_t<Iterator1, Iterator2>
operator<=>(const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y);
```

13 *Returns:* `y.base() <=> x.base()`.

14 [Note 1: The argument order in the *Returns:* element is reversed because this is a reverse iterator. — end note]

#### 24.5.1.9 Non-member functions

[reverse.iter.nonmember]

```
template<class Iterator1, class Iterator2>
constexpr auto operator-(
 const reverse_iterator<Iterator1>& x,
 const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
```

1 *Returns:* `y.base() - x.base()`.

```
template<class Iterator>
constexpr reverse_iterator<Iterator> operator+(
 iter_difference_t<Iterator> n,
 const reverse_iterator<Iterator>& x);
```

2 *Returns:* `reverse_iterator<Iterator>(x.base() - n)`.

```
friend constexpr iter_rvalue_reference_t<Iterator>
iter_move(const reverse_iterator& i) noexcept(see below);
```

3 *Effects:* Equivalent to:

```
auto tmp = i.base();
return ranges::iter_move(--tmp);
```

4 *Remarks:* The exception specification is equivalent to:

```
is_nothrow_copy_constructible_v<Iterator> &&
noexcept(ranges::iter_move(--declval<Iterator&>()))
```

```
template<indirectly_swappable<Iterator> Iterator2>
friend constexpr void
iter_swap(const reverse_iterator& x,
 const reverse_iterator<Iterator2>& y) noexcept(see below);
```

5 *Effects:* Equivalent to:



```

 auto xtmp = x.base();
 auto ytmp = y.base();
 ranges::iter_swap(--xtmp, --ytmp);

```

6 *Remarks:* The exception specification is equivalent to:

```

 is_nothrow_copy_constructible_v<Iterator> &&
 is_nothrow_copy_constructible_v<Iterator2> &&
 noexcept(ranges::iter_swap(--declval<Iterator&>(), --declval<Iterator2&>()))

```

```

template<class Iterator>
constexpr reverse_iterator<Iterator> make_reverse_iterator(Iterator i);

```

7 *Returns:* reverse\_iterator<Iterator>(i).

## 24.5.2 Insert iterators

[insert.iterators]

### 24.5.2.1 General

[insert.iterators.general]

<sup>1</sup> To make it possible to deal with insertion in the same way as writing into an array, a special kind of iterator adaptors, called *insert iterators*, are provided in the library. With regular iterator classes,

```
while (first != last) *result++ = *first++;
```

causes a range [first, last) to be copied into a range starting with result. The same code with result being an insert iterator will insert corresponding elements into the container. This device allows all of the copying algorithms in the library to work in the *insert mode* instead of the *regular overwrite mode*.

<sup>2</sup> An insert iterator is constructed from a container and possibly one of its iterators pointing to where insertion takes place if it is neither at the beginning nor at the end of the container. Insert iterators meet the requirements of output iterators. `operator*` returns the insert iterator itself. The assignment `operator=(const T& x)` is defined on insert iterators to allow writing into them, it inserts `x` right before where the insert iterator is pointing. In other words, an insert iterator is like a cursor pointing into the container where the insertion takes place. `back_insert_iterator` inserts elements at the end of a container, `front_insert_iterator` inserts elements at the beginning of a container, and `insert_iterator` inserts elements where the iterator points to in a container. `back_inserter`, `front_inserter`, and `inserter` are three functions making the insert iterators out of a container.

### 24.5.2.2 Class template back\_insert\_iterator

[back.insert.iterator]

#### 24.5.2.2.1 General

[back.insert.iter.general]

```

namespace std {
 template<class Container>
 class back_insert_iterator {
 protected:
 Container* container;

 public:
 using iterator_category = output_iterator_tag;
 using value_type = void;
 using difference_type = ptrdiff_t;
 using pointer = void;
 using reference = void;
 using container_type = Container;

 constexpr explicit back_insert_iterator(Container& x);
 constexpr back_insert_iterator& operator=(const typename Container::value_type& value);
 constexpr back_insert_iterator& operator=(typename Container::value_type&& value);

 constexpr back_insert_iterator& operator*();
 constexpr back_insert_iterator& operator++();
 constexpr back_insert_iterator operator++(int);
 };
}

```

**24.5.2.2.2 Operations****[back.insert.iter.ops]**

```
constexpr explicit back_insert_iterator(Container& x);
```

1     *Effects:* Initializes container with `addressof(x)`.

```
constexpr back_insert_iterator& operator=(const typename Container::value_type& value);
```

2     *Effects:* As if by: `container->push_back(value)`;

3     *Returns:* `*this`.

```
constexpr back_insert_iterator& operator=(typename Container::value_type&& value);
```

4     *Effects:* As if by: `container->push_back(std::move(value))`;

5     *Returns:* `*this`.

```
constexpr back_insert_iterator& operator*();
```

6     *Returns:* `*this`.

```
constexpr back_insert_iterator& operator++();
```

```
constexpr back_insert_iterator operator++(int);
```

7     *Returns:* `*this`.

**24.5.2.2.3 back\_inserter****[back.inserter]**

```
template<class Container>
```

```
constexpr back_insert_iterator<Container> back_inserter(Container& x);
```

1     *Returns:* `back_insert_iterator<Container>(x)`.

**24.5.2.3 Class template front\_insert\_iterator****[front.insert.iterator]****24.5.2.3.1 General****[front.insert.iter.general]**

```
namespace std {
```

```
template<class Container>
```

```
class front_insert_iterator {
```

```
protected:
```

```
Container* container;
```

```
public:
```

```
using iterator_category = output_iterator_tag;
```

```
using value_type = void;
```

```
using difference_type = ptrdiff_t;
```

```
using pointer = void;
```

```
using reference = void;
```

```
using container_type = Container;
```

```
constexpr explicit front_insert_iterator(Container& x);
```

```
constexpr front_insert_iterator& operator=(const typename Container::value_type& value);
```

```
constexpr front_insert_iterator& operator=(typename Container::value_type&& value);
```

```
constexpr front_insert_iterator& operator*();
```

```
constexpr front_insert_iterator& operator++();
```

```
constexpr front_insert_iterator operator++(int);
```

```
};
```

```
}
```

**24.5.2.3.2 Operations****[front.insert.iter.ops]**

```
constexpr explicit front_insert_iterator(Container& x);
```

1     *Effects:* Initializes container with `addressof(x)`.

```
constexpr front_insert_iterator& operator=(const typename Container::value_type& value);
```

2     *Effects:* As if by: `container->push_front(value)`;

3     *Returns:* `*this`.

```

constexpr front_insert_iterator& operator=(typename Container::value_type&& value);
4 Effects: As if by: container->push_front(std::move(value));
5 Returns: *this.

constexpr front_insert_iterator& operator*();
6 Returns: *this.

constexpr front_insert_iterator& operator++();
constexpr front_insert_iterator operator++(int);
7 Returns: *this.

```

#### 24.5.2.3.3 front\_inserter

[front.inserter]

```

template<class Container>
constexpr front_insert_iterator<Container> front_inserter(Container& x);
1 Returns: front_insert_iterator<Container>(x).

```

#### 24.5.2.4 Class template insert\_iterator

[insert.iterator]

##### 24.5.2.4.1 General

[insert.iter.general]

```

namespace std {
 template<class Container>
 class insert_iterator {
 protected:
 Container* container;
 ranges::iterator_t<Container> iter;

 public:
 using iterator_category = output_iterator_tag;
 using value_type = void;
 using difference_type = ptrdiff_t;
 using pointer = void;
 using reference = void;
 using container_type = Container;

 constexpr insert_iterator(Container& x, ranges::iterator_t<Container> i);
 constexpr insert_iterator& operator=(const typename Container::value_type& value);
 constexpr insert_iterator& operator=(typename Container::value_type&& value);

 constexpr insert_iterator& operator*();
 constexpr insert_iterator& operator++();
 constexpr insert_iterator& operator++(int);
 };
}

```

##### 24.5.2.4.2 Operations

[insert.iter.ops]

```

constexpr insert_iterator(Container& x, ranges::iterator_t<Container> i);
1 Effects: Initializes container with addressof(x) and iter with i.

constexpr insert_iterator& operator=(const typename Container::value_type& value);
2 Effects: As if by:
 iter = container->insert(iter, value);
 ++iter;
3 Returns: *this.

constexpr insert_iterator& operator=(typename Container::value_type&& value);
4 Effects: As if by:
 iter = container->insert(iter, std::move(value));
 ++iter;
5 Returns: *this.

```

```
constexpr insert_iterator& operator*();
```

6     *Returns:* *\*this*.

```
constexpr insert_iterator& operator++();
constexpr insert_iterator& operator++(int);
```

7     *Returns:* *\*this*.

#### 24.5.2.4.3 inserter

[inserter]

```
template<class Container>
constexpr insert_iterator<Container>
inserter(Container& x, ranges::iterator_t<Container> i);
```

1     *Returns:* *insert\_iterator<Container>(x, i)*.

### 24.5.3 Constant iterators and sentinels

[const.iterators]

#### 24.5.3.1 General

[const.iterators.general]

1 Class template `basic_const_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that its indirection operator implicitly converts the value returned by the underlying iterator's indirection operator to a type such that the adapted iterator is a constant iterator (24.3). Some generic algorithms can be called with constant iterators to avoid mutation.

2 Specializations of `basic_const_iterator` are constant iterators.

#### 24.5.3.2 Alias templates

[const.iterators.alias]

```
template<indirectly_readable It>
using iter_const_reference_t =
common_reference_t<const iter_value_t<It>&&, iter_reference_t<It>>;
```

```
template<class It>
concept constant_iterator = // exposition only
input_iterator<It> && same_as<iter_const_reference_t<It>, iter_reference_t<It>>;
```

```
template<input_iterator I>
using const_iterator = see below;
```

1     *Result:* If *I* models *constant\_iterator*, *I*. Otherwise, `basic_const_iterator<I>`.

```
template<semiregular S>
using const_sentinel = see below;
```

2     *Result:* If *S* models `input_iterator`, `const_iterator<S>`. Otherwise, *S*.

#### 24.5.3.3 Class template `basic_const_iterator`

[const.iterators.iterator]

```
namespace std {
template<class I>
concept not-a-const-iterator = see below; // exposition only

template<indirectly_readable I>
using iter-const-rvalue-reference-t = // exposition only
common_reference_t<const iter_value_t<I>&&, iter_rvalue_reference_t<I>>;

template<input_iterator Iterator>
class basic_const_iterator {
 Iterator current_ = Iterator(); // exposition only
 using reference = iter_const_reference_t<Iterator>; // exposition only
 using rvalue-reference = // exposition only
iter-const-rvalue-reference-t<Iterator>;

public:
 using iterator_type = Iterator;
 using iterator_concept = see below;
 using iterator_category = see below; // not always present
 using value_type = iter_value_t<Iterator>;
```

```

using difference_type = iter_difference_t<Iterator>;

basic_const_iterator() requires default_initializable<Iterator> = default;
constexpr basic_const_iterator(Iterator current);
template<convertible_to<Iterator> U>
 constexpr basic_const_iterator(basic_const_iterator<U> current);
template<different-from<basic_const_iterator> T>
 requires convertible_to<T, Iterator>
 constexpr basic_const_iterator(T&& current);

constexpr const Iterator& base() const & noexcept;
constexpr Iterator base() &&;

constexpr reference operator*() const;
constexpr const auto* operator->() const
 requires is_lvalue_reference_v<iter_reference_t<Iterator>> &&
 same_as<remove_cvref_t<iter_reference_t<Iterator>>, value_type>;

constexpr basic_const_iterator& operator++();
constexpr void operator++(int);
basic_const_iterator operator++(int) requires forward_iterator<Iterator> = default;

constexpr basic_const_iterator& operator--() requires bidirectional_iterator<Iterator>;
basic_const_iterator operator--(int) = default;

constexpr basic_const_iterator& operator+=(difference_type n)
 requires random_access_iterator<Iterator>;
constexpr basic_const_iterator& operator-=(difference_type n)
 requires random_access_iterator<Iterator>;

constexpr reference operator[](difference_type n) const
 requires random_access_iterator<Iterator>;

template<sentinel_for<Iterator> S>
 constexpr bool operator==(const S& s) const;

template<not-a-const-iterator CI>
 requires constant_iterator<CI> && convertible_to<Iterator const&, CI>
 constexpr operator CI() const &;
template<not-a-const-iterator CI>
 requires constant_iterator<CI> && convertible_to<Iterator, CI>
 constexpr operator CI() &&;

constexpr bool operator<(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator>(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator<=(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator>=(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr auto operator<=>(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator> && three_way_comparable<Iterator>;

template<different-from<basic_const_iterator> I>
 constexpr bool operator<(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr bool operator>(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr bool operator<=(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;

```

```

template<different-from<basic_const_iterator> I>
constexpr bool operator==(const I& x, const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
constexpr auto operator<=(const I& x, const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I> &&
 three_way_comparable_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator<(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator>(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator<=(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator>=(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;

friend constexpr basic_const_iterator operator+(const basic_const_iterator& i,
 difference_type n)
 requires random_access_iterator<Iterator>;
friend constexpr basic_const_iterator operator+(difference_type n,
 const basic_const_iterator& i)
 requires random_access_iterator<Iterator>;
friend constexpr basic_const_iterator operator-(const basic_const_iterator& i,
 difference_type n)
 requires random_access_iterator<Iterator>;
template<sized_sentinel_for<Iterator> S>
constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
template<not-a-const-iterator S>
 requires sized_sentinel_for<S, Iterator>
 friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
friend constexpr rvalue-reference iter_move(const basic_const_iterator& i)
 noexcept(noexcept(static_cast<rvalue-reference>(ranges::iter_move(i.current_))))
{
 return static_cast<rvalue-reference>(ranges::iter_move(i.current_));
}
};
}

```

- <sup>1</sup> Given some type *I*, the concept *not-a-const-iterator* is defined as *false* if *I* is a specialization of *basic\_const\_iterator* and *true* otherwise.

#### 24.5.3.4 Member types

[const.iterators.types]

- <sup>1</sup> *basic\_const\_iterator*<*Iterator*>::*iterator\_concept* is defined as follows:

- (1.1) — If *Iterator* models *contiguous\_iterator*, then *iterator\_concept* denotes *contiguous\_iterator\_tag*.
- (1.2) — Otherwise, if *Iterator* models *random\_access\_iterator*, then *iterator\_concept* denotes *random\_access\_iterator\_tag*.
- (1.3) — Otherwise, if *Iterator* models *bidirectional\_iterator*, then *iterator\_concept* denotes *bidirectional\_iterator\_tag*.
- (1.4) — Otherwise, if *Iterator* models *forward\_iterator*, then *iterator\_concept* denotes *forward\_iterator\_tag*.
- (1.5) — Otherwise, *iterator\_concept* denotes *input\_iterator\_tag*.

- <sup>2</sup> The member *typedef-name* *iterator\_category* is declared if and only if *Iterator* models *forward\_iterator*. In that case, *basic\_const\_iterator*<*Iterator*>::*iterator\_category* denotes the type *iterator\_traits*<*Iterator*>::*iterator\_category*.

## 24.5.3.5 Operations

[const.iterators.ops]

```

constexpr basic_const_iterator(Iterator current);
1 Effects: Initializes current_ with std::move(current).

template<convertible_to<Iterator> U>
constexpr basic_const_iterator(basic_const_iterator<U> current);
2 Effects: Initializes current_ with std::move(current.current_).

template<different-from<basic_const_iterator> T>
requires convertible_to<T, Iterator>
constexpr basic_const_iterator(T&& current);
3 Effects: Initializes current_ with std::forward<T>(current).

constexpr const Iterator& base() const & noexcept;
4 Effects: Equivalent to: return current_;

constexpr Iterator base() &&;
5 Effects: Equivalent to: return std::move(current_);

constexpr reference operator*() const;
6 Effects: Equivalent to: return static_cast<reference>(*current_);

constexpr const auto* operator->() const
requires is_lvalue_reference_v<iter_reference_t<Iterator>> &&
 same_as<remove_cvref_t<iter_reference_t<Iterator>>, value_type>;
7 Returns: If Iterator models contiguous_iterator, to_address(current_); otherwise, address-
of(*current_).

constexpr basic_const_iterator& operator++();
8 Effects: Equivalent to:
 ++current_;
 return *this;

constexpr void operator++(int);
9 Effects: Equivalent to: ++current_;

constexpr basic_const_iterator& operator--() requires bidirectional_iterator<Iterator>;
10 Effects: Equivalent to:
 --current_;
 return *this;

constexpr basic_const_iterator& operator+=(difference_type n)
requires random_access_iterator<Iterator>;
constexpr basic_const_iterator& operator-=(difference_type n)
requires random_access_iterator<Iterator>;
11 Let op be the operator.
12 Effects: Equivalent to:
 current_ op n;
 return *this;

constexpr reference operator[](difference_type n) const requires random_access_iterator<Iterator>
13 Effects: Equivalent to: return static_cast<reference>(current_[n]);

template<sentinel_for<Iterator> S>
constexpr bool operator==(const S& s) const;
14 Effects: Equivalent to: return current_ == s;

```

```
template<not-a-const-iterator CI>
 requires constant-iterator<CI> && convertible_to<Iterator const&, CI>
constexpr operator CI() const &;
```

15     *Returns:* *current\_*.

```
template<not-a-const-iterator CI>
 requires constant-iterator<CI> && convertible_to<Iterator, CI>
constexpr operator CI() &&;
```

16     *Returns:* *std::move(current\_)*.

```
constexpr bool operator<(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator>(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator<=(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr bool operator>=(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator>;
constexpr auto operator<=>(const basic_const_iterator& y) const
 requires random_access_iterator<Iterator> && three_way_comparable<Iterator>;
```

17     Let *op* be the operator.

18     *Effects:* Equivalent to: return *current\_ op y.current\_*;

```
template<different-from<basic_const_iterator> I>
 constexpr bool operator<(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr bool operator>(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr bool operator<=(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr bool operator>=(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<different-from<basic_const_iterator> I>
 constexpr auto operator<=>(const I& y) const
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I> &&
 three_way_comparable_with<Iterator, I>;
```

19     Let *op* be the operator.

20     *Effects:* Equivalent to: return *current\_ op y*;

```
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator<(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator>(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator<=(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
template<not-a-const-iterator I, same_as<Iterator> J>
 friend constexpr bool operator>=(const I& x, const basic_const_iterator<J>& y)
 requires random_access_iterator<Iterator> && totally_ordered_with<Iterator, I>;
```

21     Let *op* be the operator.

22     *Effects:* Equivalent to: return *x op y.current\_*;

```
friend constexpr basic_const_iterator operator+(const basic_const_iterator& i, difference_type n)
 requires random_access_iterator<Iterator>;
```



```

friend constexpr basic_const_iterator operator+(difference_type n, const basic_const_iterator& i)
 requires random_access_iterator<Iterator>;
23 Effects: Equivalent to: return basic_const_iterator(i.current_ + n);

friend constexpr basic_const_iterator operator-(const basic_const_iterator& i, difference_type n)
 requires random_access_iterator<Iterator>;
24 Effects: Equivalent to: return basic_const_iterator(i.current_ - n);

template< sized_sentinel_for<Iterator> S>
 constexpr difference_type operator-(const S& y) const;
25 Effects: Equivalent to: return current_ - y;

template< not-a-const-iterator S>
 requires sized_sentinel_for<S, Iterator>
 friend constexpr difference_type operator-(const S& x, const basic_const_iterator& y);
26 Effects: Equivalent to: return x - y.current_;

```

## 24.5.4 Move iterators and sentinels

[move.iterators]

### 24.5.4.1 General

[move.iterators.general]

- <sup>1</sup> Class template `move_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that its indirection operator implicitly converts the value returned by the underlying iterator's indirection operator to an rvalue. Some generic algorithms can be called with move iterators to replace copying with moving.

- <sup>2</sup> [Example 1:

```

list<string> s;
// populate the list s
vector<string> v1(s.begin(), s.end()); // copies strings into v1
vector<string> v2(make_move_iterator(s.begin()),
 make_move_iterator(s.end())); // moves strings into v2

```

— end example]

### 24.5.4.2 Class template `move_iterator`

[move.iterator]

```

namespace std {
 template<class Iterator>
 class move_iterator {
 public:
 using iterator_type = Iterator;
 using iterator_concept = see below;
 using iterator_category = see below; // not always present
 using value_type = iter_value_t<Iterator>;
 using difference_type = iter_difference_t<Iterator>;
 using pointer = Iterator;
 using reference = iter_rvalue_reference_t<Iterator>;

 constexpr move_iterator() requires default_initializable<Iterator> = default;
 constexpr explicit move_iterator(Iterator i);
 template<class U> constexpr move_iterator(const move_iterator<U>& u);
 template<class U> constexpr move_iterator& operator=(const move_iterator<U>& u);

 constexpr const Iterator& base() const & noexcept;
 constexpr Iterator base() &&;
 constexpr reference operator*() const;

 constexpr move_iterator& operator++();
 constexpr void operator++(int);
 move_iterator operator++(int) requires forward_iterator<Iterator> = default;
 constexpr move_iterator& operator--();
 move_iterator operator--(int) = default;

```

```

constexpr move_iterator operator+(difference_type n) const;
constexpr move_iterator& operator+=(difference_type n);
constexpr move_iterator operator-(difference_type n) const;
constexpr move_iterator& operator-=(difference_type n);
constexpr reference operator[](difference_type n) const;

template<sentinel_for<Iterator> S>
 friend constexpr bool
 operator==(const move_iterator& x, const move_sentinel<S>& y);
template<sized_sentinel_for<Iterator> S>
 friend constexpr iter_difference_t<Iterator>
 operator-(const move_sentinel<S>& x, const move_iterator& y);
template<sized_sentinel_for<Iterator> S>
 friend constexpr iter_difference_t<Iterator>
 operator-(const move_iterator& x, const move_sentinel<S>& y);
friend constexpr iter_rvalue_reference_t<Iterator>
 iter_move(const move_iterator& i)
 noexcept(noexcept(ranges::iter_move(i.current)));
template<indirectly_swappable<Iterator> Iterator2>
 friend constexpr void
 iter_swap(const move_iterator& x, const move_iterator<Iterator2>& y)
 noexcept(noexcept(ranges::iter_swap(x.current, y.current)));

private:
 Iterator current = Iterator(); // exposition only
};
}

```

<sup>1</sup> The member *typedef-name* `iterator_concept` is defined as follows:

- (1.1) — If `Iterator` models `random_access_iterator`, then `iterator_concept` denotes `random_access_iterator_tag`.
- (1.2) — Otherwise, if `Iterator` models `bidirectional_iterator`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (1.3) — Otherwise, if `Iterator` models `forward_iterator`, then `iterator_concept` denotes `forward_iterator_tag`.
- (1.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

<sup>2</sup> The member *typedef-name* `iterator_category` is declared if and only if the *qualified-id* `iterator_traits<Iterator>::iterator_category` is valid and denotes a type. In that case, `iterator_category` denotes

- (2.1) — `random_access_iterator_tag` if the type `iterator_traits<Iterator>::iterator_category` models `derived_from<random_access_iterator_tag>`, and
- (2.2) — `iterator_traits<Iterator>::iterator_category` otherwise.

#### 24.5.4.3 Requirements

[move.iter.requirements]

- <sup>1</sup> The template parameter `Iterator` shall either meet the *Cpp17InputIterator* requirements (24.3.5.3) or model `input_iterator` (24.3.4.9). Additionally, if any of the bidirectional traversal functions are instantiated, the template parameter shall either meet the *Cpp17BidirectionalIterator* requirements (24.3.5.6) or model `bidirectional_iterator` (24.3.4.12). If any of the random access traversal functions are instantiated, the template parameter shall either meet the *Cpp17RandomAccessIterator* requirements (24.3.5.7) or model `random_access_iterator` (24.3.4.13).

#### 24.5.4.4 Construction and assignment

[move.iter.cons]

```
constexpr explicit move_iterator(Iterator i);
```

- <sup>1</sup> *Effects:* Initializes `current` with `std::move(i)`.

```
template<class U> constexpr move_iterator(const move_iterator<U>& u);
```

- <sup>2</sup> *Constraints:* `is_same_v<U, Iterator>` is false and `const U&` models `convertible_to<Iterator>`.

- <sup>3</sup> *Effects:* Initializes `current` with `u.current`.

```
template<class U> constexpr move_iterator& operator=(const move_iterator<U>& u);
```

4     *Constraints:* `is_same_v<U, Iterator>` is false, `const U&` models `convertible_to<Iterator>`, and `assignable_from<Iterator&, const U&>` is modeled.

5     *Effects:* Assigns `u.current` to `current`.

6     *Returns:* `*this`.

#### 24.5.4.5 Conversion

[move.iter.op.conv]

```
constexpr const Iterator& base() const & noexcept;
```

1     *Returns:* `current`.

```
constexpr Iterator base() &&;
```

2     *Returns:* `std::move(current)`.

#### 24.5.4.6 Element access

[move.iter.elem]

```
constexpr reference operator*() const;
```

1     *Effects:* Equivalent to: `return ranges::iter_move(current);`

```
constexpr reference operator[](difference_type n) const;
```

2     *Effects:* Equivalent to: `return ranges::iter_move(current + n);`

#### 24.5.4.7 Navigation

[move.iter.nav]

```
constexpr move_iterator& operator++();
```

1     *Effects:* As if by `++current`.

2     *Returns:* `*this`.

```
constexpr void operator++(int);
```

3     *Effects:* Equivalent to `++current`.

```
constexpr move_iterator& operator--();
```

4     *Effects:* As if by `--current`.

5     *Returns:* `*this`.

```
constexpr move_iterator operator+(difference_type n) const;
```

6     *Returns:* `move_iterator(current + n)`.

```
constexpr move_iterator& operator+=(difference_type n);
```

7     *Effects:* As if by: `current += n;`

8     *Returns:* `*this`.

```
constexpr move_iterator operator-(difference_type n) const;
```

9     *Returns:* `move_iterator(current - n)`.

```
constexpr move_iterator& operator-=(difference_type n);
```

10     *Effects:* As if by: `current -= n;`

11     *Returns:* `*this`.

#### 24.5.4.8 Comparisons

[move.iter.op.comp]

```
template<class Iterator1, class Iterator2>
```

```
constexpr bool operator==(const move_iterator<Iterator1>& x,
 const move_iterator<Iterator2>& y);
```

```
template<sentinel_for<Iterator> S>
```

```
friend constexpr bool operator==(const move_iterator& x,
 const move_sentinel<S>& y);
```

1     *Constraints:* `x.base() == y.base()` is well-formed and convertible to `bool`.

2       *Returns:* `x.base() == y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator<(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

3       *Constraints:* `x.base() < y.base()` is well-formed and convertible to `bool`.

4       *Returns:* `x.base() < y.base()`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

5       *Constraints:* `y.base() < x.base()` is well-formed and convertible to `bool`.

6       *Returns:* `y < x`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator<=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

7       *Constraints:* `y.base() < x.base()` is well-formed and convertible to `bool`.

8       *Returns:* `!(y < x)`.

```
template<class Iterator1, class Iterator2>
constexpr bool operator>=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

9       *Constraints:* `x.base() < y.base()` is well-formed and convertible to `bool`.

10       *Returns:* `!(x < y)`.

```
template<class Iterator1, three_way_comparable_with<Iterator1> Iterator2>
constexpr compare_three_way_result_t<Iterator1, Iterator2>
operator<=>(const move_iterator<Iterator1>& x,
 const move_iterator<Iterator2>& y);
```

11       *Returns:* `x.base() <=> y.base()`.

#### 24.5.4.9 Non-member functions

[move.iter.nonmember]

```
template<class Iterator1, class Iterator2>
constexpr auto operator-(
 const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y)
 -> decltype(x.base() - y.base());
```

```
template< sized_sentinel_for<Iterator> S>
friend constexpr iter_difference_t<Iterator>
operator-(const move_sentinel<S>& x, const move_iterator& y);
```

```
template< sized_sentinel_for<Iterator> S>
friend constexpr iter_difference_t<Iterator>
operator-(const move_iterator& x, const move_sentinel<S>& y);
```

1       *Returns:* `x.base() - y.base()`.

```
template<class Iterator>
constexpr move_iterator<Iterator>
operator+(iter_difference_t<Iterator> n, const move_iterator<Iterator>& x);
```

2       *Constraints:* `x.base() + n` is well-formed and has type `Iterator`.

3       *Returns:* `x + n`.

```
friend constexpr iter_rvalue_reference_t<Iterator>
iter_move(const move_iterator& i)
 noexcept(noexcept(ranges::iter_move(i.current)));
```

4       *Effects:* Equivalent to: `return ranges::iter_move(i.current);`

```
template<indirectly_swappable<Iterator> Iterator2>
friend constexpr void
iter_swap(const move_iterator& x, const move_iterator<Iterator2>& y)
 noexcept(noexcept(ranges::iter_swap(x.current, y.current)));
```

5       *Effects:* Equivalent to: `ranges::iter_swap(x.current, y.current)`.

```
template<class Iterator>
constexpr move_iterator<Iterator> make_move_iterator(Iterator i);
```

6     *Returns:* move\_iterator<Iterator>(std::move(i)).

#### 24.5.4.10 Class template move\_sentinel [move.sentinel]

1 Class template move\_sentinel is a sentinel adaptor useful for denoting ranges together with move\_iterator. When an input iterator type I and sentinel type S model sentinel\_for<S, I>, move\_sentinel<S> and move\_iterator<I> model sentinel\_for<move\_sentinel<S>, move\_iterator<I>> as well.

2 [Example 1: A move\_if algorithm is easily implemented with copy\_if using move\_iterator and move\_sentinel:

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
 indirect_unary_predicate<I> Pred>
 requires indirectly_movable<I, O>
void move_if(I first, S last, O out, Pred pred) {
 ranges::copy_if(move_iterator<I>{std::move(first)}, move_sentinel<S>{last},
 std::move(out), pred);
}
```

— end example]

```
namespace std {
 template<semiregular S>
 class move_sentinel {
 public:
 constexpr move_sentinel();
 constexpr explicit move_sentinel(S s);
 template<class S2>
 requires convertible_to<const S2&, S>
 constexpr move_sentinel(const move_sentinel<S2>& s);
 template<class S2>
 requires assignable_from<S&, const S2&>
 constexpr move_sentinel& operator=(const move_sentinel<S2>& s);

 constexpr S base() const;

 private:
 S last; // exposition only
 };
}
```

#### 24.5.4.11 Operations [move.sent.ops]

```
constexpr move_sentinel();
```

1     *Effects:* Value-initializes last. If is\_trivially\_default\_constructible\_v<S> is true, then this constructor is a constexpr constructor.

```
constexpr explicit move_sentinel(S s);
```

2     *Effects:* Initializes last with std::move(s).

```
template<class S2>
 requires convertible_to<const S2&, S>
 constexpr move_sentinel(const move_sentinel<S2>& s);
```

3     *Effects:* Initializes last with s.last.

```
template<class S2>
 requires assignable_from<S&, const S2&>
 constexpr move_sentinel& operator=(const move_sentinel<S2>& s);
```

4     *Effects:* Equivalent to: last = s.last; return \*this;

```
constexpr S base() const;
```

5     *Returns:* last.

## 24.5.5 Common iterators

[iterators.common]

### 24.5.5.1 Class template `common_iterator`

[common.iterator]

<sup>1</sup> Class template `common_iterator` is an iterator/sentinel adaptor that is capable of representing a non-common range of elements (where the types of the iterator and sentinel differ) as a common range (where they are the same). It does this by holding either an iterator or a sentinel, and implementing the equality comparison operators appropriately.

<sup>2</sup> [Note 1: The `common_iterator` type is useful for interfacing with legacy code that expects the begin and end of a range to have the same type. — end note]

<sup>3</sup> [Example 1:

```
template<class ForwardIterator>
void fun(ForwardIterator begin, ForwardIterator end);

list<int> s;
// populate the list s
using CI = common_iterator<counted_iterator<list<int>::iterator>, default_sentinel_t>;
// call fun on a range of 10 ints
fun(CI(counted_iterator(s.begin(), 10)), CI(default_sentinel));
```

— end example]

```
namespace std {
 template<input_or_output_iterator I, sentinel_for<I> S>
 requires (!same_as<I, S> && copyable<I>)>
 class common_iterator {
 public:
 constexpr common_iterator() requires default_initializable<I> = default;
 constexpr common_iterator(I i);
 constexpr common_iterator(S s);
 template<class I2, class S2>
 requires convertible_to<const I2&, I> && convertible_to<const S2&, S>
 constexpr common_iterator(const common_iterator<I2, S2>& x);

 template<class I2, class S2>
 requires convertible_to<const I2&, I> && convertible_to<const S2&, S> &&
 assignable_from<I&, const I2&> && assignable_from<S&, const S2&>
 constexpr common_iterator& operator=(const common_iterator<I2, S2>& x);

 constexpr decltype(auto) operator*();
 constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;
 constexpr auto operator->() const
 requires see below;

 constexpr common_iterator& operator++();
 constexpr decltype(auto) operator++(int);
 common_iterator operator++(int) requires forward_iterator<I> = default;

 template<class I2, sentinel_for<I> S2>
 requires sentinel_for<S, I2>
 friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);
 template<class I2, sentinel_for<I> S2>
 requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
 friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);

 template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
 requires sized_sentinel_for<S, I2>
 friend constexpr iter_difference_t<I2> operator-(
 const common_iterator& x, const common_iterator<I2, S2>& y);
```

```

friend constexpr decltype(auto) iter_move(const common_iterator& i)
 noexcept(noexcept(ranges::iter_move(declval<const I&>())))
 requires input_iterator<I>;
template<indirectly_swappable<I> I2, class S2>
 friend constexpr void iter_swap(const common_iterator& x, const common_iterator<I2, S2>& y)
 noexcept(noexcept(ranges::iter_swap(declval<const I&>(), declval<const I2&>())));

private:
 variant<I, S> v_; // exposition only
};

template<class I, class S>
struct incrementable_traits<common_iterator<I, S>> {
 using difference_type = iter_difference_t<I>;
};

template<input_iterator I, class S>
struct iterator_traits<common_iterator<I, S>> {
 using iterator_concept = see below;
 using iterator_category = see below; // not always present
 using value_type = iter_value_t<I>;
 using difference_type = iter_difference_t<I>;
 using pointer = see below;
 using reference = iter_reference_t<I>;
};
}

```

#### 24.5.5.2 Associated types

[common.iter.types]

- 1 The nested *typedef-name* `iterator_category` of the specialization of `iterator_traits` for `common_iterator<I, S>` is declared if and only if `iter_difference_t<I>` is an integral type. In that case, `iterator_category` denotes `forward_iterator_tag` if the *qualified-id* `iterator_traits<I>::iterator_category` is valid and denotes a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.
- 2 The remaining nested *typedef-names* of the specialization of `iterator_traits` for `common_iterator<I, S>` are defined as follows:
  - (2.1) — `iterator_concept` denotes `forward_iterator_tag` if `I` models `forward_iterator`; otherwise it denotes `input_iterator_tag`.
  - (2.2) — Let `a` denote an lvalue of type `const common_iterator<I, S>`. If the expression `a.operator->()` is well-formed, then `pointer` denotes `decltype(a.operator->())`. Otherwise, `pointer` denotes `void`.

#### 24.5.5.3 Constructors and conversions

[common.iter.const]

- ```
constexpr common_iterator(I i);
```
- 1 *Effects:* Initializes `v_` as if by `v_{in_place_type<I>, std::move(i)}`.
- ```
constexpr common_iterator(S s);
```
- 2 *Effects:* Initializes `v_` as if by `v_{in_place_type<S>, std::move(s)}`.
- ```
template<class I2, class S2>
    requires convertible_to<const I2&, I> && convertible_to<const S2&, S>
    constexpr common_iterator(const common_iterator<I2, S2>& x);
```
- 3 *Hardened preconditions:* `x.v_.valueless_by_exception()` is false.
 - 4 *Effects:* Initializes `v_` as if by `v_{in_place_index<i>, get<i>(x.v_)}`, where `i` is `x.v_.index()`.
- ```
template<class I2, class S2>
 requires convertible_to<const I2&, I> && convertible_to<const S2&, S> &&
 assignable_from<I&, const I2&> && assignable_from<S&, const S2&>
 constexpr common_iterator& operator=(const common_iterator<I2, S2>& x);
```
- 5 *Hardened preconditions:* `x.v_.valueless_by_exception()` is false.
  - 6 *Effects:* Equivalent to:

- (6.1) — If `v_.index() == x.v_.index()`, then `get<i>(v_) = get<i>(x.v_)`.  
 (6.2) — Otherwise, `v_.emplace<i>(get<i>(x.v_))`.  
 where `i` is `x.v_.index()`.

7 *Returns: \*this.*

#### 24.5.5.4 Accessors

[common.iter.access]

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;
```

1 *Hardened preconditions:* `holds_alternative<I>(v_)` is true.

2 *Effects:* Equivalent to: `return *get<I>(v_);`

```
constexpr auto operator->() const
 requires see below;
```

3 The expression in the *requires-clause* is equivalent to:

```
indirectly_readable<const I> &&
 (requires(const I& i) { i.operator->(); } ||
 is_reference_v<iter_reference_t<I>> ||
 constructible_from<iter_value_t<I>, iter_reference_t<I>>>)
```

4 *Hardened preconditions:* `holds_alternative<I>(v_)` is true.

5 *Effects:*

(5.1) — If `I` is a pointer type or if the expression `get<I>(v_).operator->()` is well-formed, equivalent to: `return get<I>(v_);`

(5.2) — Otherwise, if `iter_reference_t<I>` is a reference type, equivalent to:

```
auto&& tmp = *get<I>(v_);
return addressof(tmp);
```

(5.3) — Otherwise, equivalent to: `return proxy(*get<I>(v_))`; where *proxy* is the exposition-only class:

```
class proxy {
 iter_value_t<I> keep_;
 constexpr proxy(iter_reference_t<I>&& x)
 : keep_(std::move(x)) {}
public:
 constexpr const iter_value_t<I>* operator->() const noexcept {
 return addressof(keep_);
 }
};
```

#### 24.5.5.5 Navigation

[common.iter.nav]

```
constexpr common_iterator& operator++();
```

1 *Hardened preconditions:* `holds_alternative<I>(v_)` is true.

2 *Effects:* Equivalent to `++get<I>(v_)`.

3 *Returns: \*this.*

```
constexpr decltype(auto) operator++(int);
```

4 *Hardened preconditions:* `holds_alternative<I>(v_)` is true.

5 *Effects:* If `requires(I& i) { { *i++ } -> can-reference; }` is true or

```
indirectly_readable<I> && constructible_from<iter_value_t<I>, iter_reference_t<I>> &&
move_constructible<iter_value_t<I>>
```

is false, equivalent to:

```
return get<I>(v_)+;
```

Otherwise, equivalent to:



```

 postfix-proxy p(**this);
 ++*this;
 return p;

```

where *postfix-proxy* is the exposition-only class:

```

class postfix-proxy {
 iter_value_t<I> keep_;
 constexpr postfix-proxy(iter_reference_t<I>&& x)
 : keep_(std::forward<iter_reference_t<I>>(x)) {}
public:
 constexpr const iter_value_t<I>& operator*() const noexcept {
 return keep_;
 }
};

```

#### 24.5.5.6 Comparisons

[common.iter.cmp]

```

template<class I2, sentinel_for<I> S2>
 requires sentinel_for<S, I2>
friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);

```

- 1 *Hardened preconditions:* `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each false.
- 2 *Returns:* true if  $i == j$ , and otherwise `get<i>(x.v_) == get<j>(y.v_)`, where  $i$  is `x.v_.index()` and  $j$  is `y.v_.index()`.

```

template<class I2, sentinel_for<I> S2>
 requires sentinel_for<S, I2> && equality_comparable_with<I, I2>
friend constexpr bool operator==(
 const common_iterator& x, const common_iterator<I2, S2>& y);

```

- 3 *Hardened preconditions:* `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each false.
- 4 *Returns:* true if  $i$  and  $j$  are each 1, and otherwise `get<i>(x.v_) == get<j>(y.v_)`, where  $i$  is `x.v_.index()` and  $j$  is `y.v_.index()`.

```

template<sized_sentinel_for<I> I2, sized_sentinel_for<I> S2>
 requires sized_sentinel_for<S, I2>
friend constexpr iter_difference_t<I2> operator-(
 const common_iterator& x, const common_iterator<I2, S2>& y);

```

- 5 *Hardened preconditions:* `x.v_.valueless_by_exception()` and `y.v_.valueless_by_exception()` are each false.
- 6 *Returns:* 0 if  $i$  and  $j$  are each 1, and otherwise `get<i>(x.v_) - get<j>(y.v_)`, where  $i$  is `x.v_.index()` and  $j$  is `y.v_.index()`.

#### 24.5.5.7 Customizations

[common.iter.cust]

```

friend constexpr decltype(auto) iter_move(const common_iterator& i)
 noexcept(noexcept(ranges::iter_move(declval<const I&>())))
 requires input_iterator<I>;

```

- 1 *Hardened preconditions:* `holds_alternative<I>(i.v_)` is true.
- 2 *Effects:* Equivalent to: `return ranges::iter_move(get<I>(i.v_));`

```

template<indirectly_swappable<I> I2, class S2>
 friend constexpr void iter_swap(const common_iterator& x, const common_iterator<I2, S2>& y)
 noexcept(noexcept(ranges::iter_swap(declval<const I&>(), declval<const I2&>())));

```

- 3 *Hardened preconditions:* `holds_alternative<I>(x.v_)` and `holds_alternative<I2>(y.v_)` are each true.
- 4 *Effects:* Equivalent to `ranges::iter_swap(get<I>(x.v_), get<I2>(y.v_))`.

## 24.5.6 Default sentinel

[default.sentinel]

```
namespace std {
 struct default_sentinel_t { };
}
```

- <sup>1</sup> Class `default_sentinel_t` is an empty type used to denote the end of a range. It can be used together with iterator types that know the bound of their range (e.g., `counted_iterator` (24.5.7.1)).

## 24.5.7 Counted iterators

[iterators.counted]

### 24.5.7.1 Class template `counted_iterator`

[counted.iterator]

- <sup>1</sup> Class template `counted_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that it keeps track of the distance to the end of its range. It can be used together with `default_sentinel` in calls to generic algorithms to operate on a range of  $N$  elements starting at a given position without needing to know the end position a priori.

- <sup>2</sup> [Example 1:

```
list<string> s;
// populate the list s with at least 10 strings
vector<string> v;
// copies 10 strings into v:
ranges::copy(counted_iterator(s.begin(), 10), default_sentinel, back_inserter(v));
```

— end example]

- <sup>3</sup> Two values `i1` and `i2` of types `counted_iterator<I1>` and `counted_iterator<I2>` refer to elements of the same sequence if and only if there exists some integer  $n$  such that `next(i1.base(), i1.count() + n)` and `next(i2.base(), i2.count() + n)` refer to the same (possibly past-the-end) element.

```
namespace std {
 template<input_or_output_iterator I>
 class counted_iterator {
 public:
 using iterator_type = I;
 using value_type = iter_value_t<I>; // present only
 // if I models indirectly_readable
 using difference_type = iter_difference_t<I>;
 using iterator_concept = I::iterator_concept; // present only
 // if the qualified-id I::iterator_concept is valid and denotes a type
 using iterator_category = I::iterator_category; // present only
 // if the qualified-id I::iterator_category is valid and denotes a type
 constexpr counted_iterator() requires default_initializable<I> = default;
 constexpr counted_iterator(I x, iter_difference_t<I> n);
 template<class I2>
 requires convertible_to<const I2&, I>
 constexpr counted_iterator(const counted_iterator<I2>& x);

 template<class I2>
 requires assignable_from<I&, const I2&>
 constexpr counted_iterator& operator=(const counted_iterator<I2>& x);

 constexpr const I& base() const & noexcept;
 constexpr I base() &&;
 constexpr iter_difference_t<I> count() const noexcept;
 constexpr decltype(auto) operator*();
 constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;

 constexpr auto operator->() const noexcept
 requires contiguous_iterator<I>;

 constexpr counted_iterator& operator++();
 constexpr decltype(auto) operator++(int);
 counted_iterator operator++(int)
 requires forward_iterator<I> = default;
 };
}
```

```

constexpr counted_iterator& operator--()
 requires bidirectional_iterator<I>;
counted_iterator operator--(int) = default;

constexpr counted_iterator operator+(iter_difference_t<I> n) const
 requires random_access_iterator<I>;
friend constexpr counted_iterator operator+(
 iter_difference_t<I> n, const counted_iterator& x)
 requires random_access_iterator<I>;
constexpr counted_iterator& operator+=(iter_difference_t<I> n)
 requires random_access_iterator<I>;

constexpr counted_iterator operator-(iter_difference_t<I> n) const
 requires random_access_iterator<I>;
template<common_with<I> I2>
 friend constexpr iter_difference_t<I2> operator-(
 const counted_iterator& x, const counted_iterator<I2>& y);
friend constexpr iter_difference_t<I> operator-(
 const counted_iterator& x, default_sentinel_t) noexcept;
friend constexpr iter_difference_t<I> operator-(
 default_sentinel_t, const counted_iterator& y) noexcept;
constexpr counted_iterator& operator--=(iter_difference_t<I> n)
 requires random_access_iterator<I>;

constexpr decltype(auto) operator[](iter_difference_t<I> n) const
 requires random_access_iterator<I>;

template<common_with<I> I2>
 friend constexpr bool operator==(
 const counted_iterator& x, const counted_iterator<I2>& y);
friend constexpr bool operator==(
 const counted_iterator& x, default_sentinel_t) noexcept;

template<common_with<I> I2>
 friend constexpr strong_ordering operator<=>(
 const counted_iterator& x, const counted_iterator<I2>& y);

friend constexpr decltype(auto) iter_move(const counted_iterator& i)
 noexcept(noexcept(ranges::iter_move(i.current)))
 requires input_iterator<I>;
template<indirectly_swappable<I> I2>
 friend constexpr void iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
 noexcept(noexcept(ranges::iter_swap(x.current, y.current)));

private:
 I current = I(); // exposition only
 iter_difference_t<I> length = 0; // exposition only
};

template<input_iterator I>
 requires same_as<ITER_TRAITS(I), iterator_traits<I>> // see 24.3.4.1
struct iterator_traits<counted_iterator<I>> : iterator_traits<I> {
 using pointer = conditional_t<contiguous_iterator<I>,
 add_pointer_t<iter_reference_t<I>>, void>;
};
}

```

#### 24.5.7.2 Constructors and conversions

[counted.iter.const]

```
constexpr counted_iterator(I i, iter_difference_t<I> n);
```

<sup>1</sup> *Hardened preconditions:* `n >= 0` is true.

<sup>2</sup> *Effects:* Initializes `current` with `std::move(i)` and `length` with `n`.

```

template<class I2>
 requires convertible_to<const I2&, I>
 constexpr counted_iterator(const counted_iterator<I2>& x);
3 Effects: Initializes current with x.current and length with x.length.

template<class I2>
 requires assignable_from<I&, const I2&>
 constexpr counted_iterator& operator=(const counted_iterator<I2>& x);
4 Effects: Assigns x.current to current and x.length to length.
5 Returns: *this.

```

### 24.5.7.3 Accessors

[counted.iter.access]

```

constexpr const I& base() const & noexcept;
1 Effects: Equivalent to: return current;

constexpr I base() &&;
2 Returns: std::move(current).

constexpr iter_difference_t<I> count() const noexcept;
3 Effects: Equivalent to: return length;

```

### 24.5.7.4 Element access

[counted.iter.elem]

```

constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
 requires dereferenceable<const I>;
1 Hardened preconditions: length > 0 is true.
2 Effects: Equivalent to: return *current;

constexpr auto operator->() const noexcept
 requires contiguous_iterator<I>;
3 Effects: Equivalent to: return to_address(current);

constexpr decltype(auto) operator[](iter_difference_t<I> n) const
 requires random_access_iterator<I>;
4 Hardened preconditions: n < length is true.
5 Effects: Equivalent to: return current[n];

```

### 24.5.7.5 Navigation

[counted.iter.nav]

```

constexpr counted_iterator& operator++();
1 Hardened preconditions: length > 0 is true.
2 Effects: Equivalent to:
 ++current;
 --length;
 return *this;

constexpr decltype(auto) operator++(int);
3 Hardened preconditions: length > 0 is true.
4 Effects: Equivalent to:
 --length;
 try { return current++; }
 catch(...) { ++length; throw; }

constexpr counted_iterator& operator--()
 requires bidirectional_iterator<I>;
5 Effects: Equivalent to:

```

```

--current;
++length;
return *this;

```

```

constexpr counted_iterator operator+(iter_difference_t<I> n) const
requires random_access_iterator<I>;

```

6 *Effects:* Equivalent to: return counted\_iterator(current + n, length - n);

```

friend constexpr counted_iterator operator+(
iter_difference_t<I> n, const counted_iterator& x)
requires random_access_iterator<I>;

```

7 *Effects:* Equivalent to: return x + n;

```

constexpr counted_iterator& operator+=(iter_difference_t<I> n)
requires random_access_iterator<I>;

```

8 *Hardened preconditions:* n <= length is true.

9 *Effects:* Equivalent to:

```

current += n;
length -= n;
return *this;

```

```

constexpr counted_iterator operator-(iter_difference_t<I> n) const
requires random_access_iterator<I>;

```

10 *Effects:* Equivalent to: return counted\_iterator(current - n, length + n);

```

template<common_with<I> I2>
friend constexpr iter_difference_t<I2> operator-(
const counted_iterator& x, const counted_iterator<I2>& y);

```

11 *Preconditions:* x and y refer to elements of the same sequence (24.5.7.1).

12 *Effects:* Equivalent to: return y.length - x.length;

```

friend constexpr iter_difference_t<I> operator-(
const counted_iterator& x, default_sentinel_t) noexcept;

```

13 *Effects:* Equivalent to: return -x.length;

```

friend constexpr iter_difference_t<I> operator-(
default_sentinel_t, const counted_iterator& y) noexcept;

```

14 *Effects:* Equivalent to: return y.length;

```

constexpr counted_iterator& operator-=(iter_difference_t<I> n)
requires random_access_iterator<I>;

```

15 *Hardened preconditions:* -n <= length is true.

16 *Effects:* Equivalent to:

```

current -= n;
length += n;
return *this;

```

#### 24.5.7.6 Comparisons

[counted.iter.cmp]

```

template<common_with<I> I2>
friend constexpr bool operator==(
const counted_iterator& x, const counted_iterator<I2>& y);

```

1 *Preconditions:* x and y refer to elements of the same sequence (24.5.7.1).

2 *Effects:* Equivalent to: return x.length == y.length;

```

friend constexpr bool operator==(
const counted_iterator& x, default_sentinel_t) noexcept;

```

3 *Effects:* Equivalent to: return x.length == 0;

```
template<common_with<I> I2>
 friend constexpr strong_ordering operator<=(
 const counted_iterator& x, const counted_iterator<I2>& y);
4 Preconditions: x and y refer to elements of the same sequence (24.5.7.1).
5 Effects: Equivalent to: return y.length <=> x.length;
6 [Note 1: The argument order in the Effects: element is reversed because length counts down, not up. — end note]
```

#### 24.5.7.7 Customizations

[counted.iter.cust]

```
friend constexpr decltype(auto)
 iter_move(const counted_iterator& i)
 noexcept(noexcept(ranges::iter_move(i.current)))
 requires input_iterator<I>;
1 Hardened preconditions: i.length > 0 is true.
2 Effects: Equivalent to: return ranges::iter_move(i.current);

template<indirectly_swappable<I> I2>
 friend constexpr void
 iter_swap(const counted_iterator& x, const counted_iterator<I2>& y)
 noexcept(noexcept(ranges::iter_swap(x.current, y.current)));
3 Hardened preconditions: Both x.length > 0 and y.length > 0 are true.
4 Effects: Equivalent to ranges::iter_swap(x.current, y.current).
```

#### 24.5.8 Unreachable sentinel

[unreachable.sentinel]

1 Class `unreachable_sentinel_t` can be used with any `weakly_incrementable` type to denote the “upper bound” of an unbounded interval.

2 [*Example 1:*

```
char* p;
// set p to point to a character buffer containing newlines
char* nl = find(p, unreachable_sentinel, '\n');
```

Provided a newline character really exists in the buffer, the use of `unreachable_sentinel` above potentially makes the call to `find` more efficient since the loop test against the sentinel does not require a conditional branch. — *end example*

```
namespace std {
 struct unreachable_sentinel_t {
 template<weakly_incrementable I>
 friend constexpr bool operator==(unreachable_sentinel_t, const I&) noexcept
 { return false; }
 };
}
```

### 24.6 Stream iterators

[stream.iterators]

#### 24.6.1 General

[stream.iterators.general]

1 To make it possible for algorithmic templates to work directly with input/output streams, appropriate iterator-like class templates are provided.

[*Example 1:*

```
partial_sum(istream_iterator<double, char>(cin),
 istream_iterator<double, char>(),
 ostream_iterator<double, char>(cout, "\n"));
```

reads a file containing floating-point numbers from `cin`, and prints the partial sums onto `cout`. — *end example*

#### 24.6.2 Class template `istream_iterator`

[istream.iterator]

##### 24.6.2.1 General

[istream.iterator.general]

1 The class template `istream_iterator` is an input iterator (24.3.5.3) that reads successive elements from the input stream for which it was constructed.

```

namespace std {
 template<class T, class charT = char, class traits = char_traits<charT>,
 class Distance = ptrdiff_t>
 class istream_iterator {
 public:
 using iterator_category = input_iterator_tag;
 using value_type = T;
 using difference_type = Distance;
 using pointer = const T*;
 using reference = const T&;
 using char_type = charT;
 using traits_type = traits;
 using istream_type = basic_istream<charT,traits>;

 constexpr istream_iterator();
 constexpr istream_iterator(default_sentinel_t);
 istream_iterator(istream_type& s);
 constexpr istream_iterator(const istream_iterator& x) noexcept(see below);
 ~istream_iterator() = default;
 istream_iterator& operator=(const istream_iterator&) = default;

 const T& operator*() const;
 const T* operator->() const;
 istream_iterator& operator++();
 istream_iterator operator++(int) = default;

 friend bool operator==(const istream_iterator& i, default_sentinel_t);

 private:
 basic_istream<charT,traits>* in_stream; // exposition only
 T value; // exposition only
 };
}

```

- 2 The type T shall meet the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, and *Cpp17CopyAssignable* requirements.

#### 24.6.2.2 Constructors and destructor

[istream.iterator.cons]

```

constexpr istream_iterator();
constexpr istream_iterator(default_sentinel_t);

```

- 1 *Effects:* Constructs the end-of-stream iterator, value-initializing value.

- 2 *Postconditions:* `in_stream == nullptr` is true.

- 3 *Remarks:* If the initializer `T()` in the declaration `auto x = T();` is a constant initializer (7.7.4), then these constructors are `constexpr` constructors.

```
istream_iterator(istream_type& s);
```

- 4 *Effects:* Initializes `in_stream` with `addressof(s)`, value-initializes `value`, and then calls `operator++()`.

```
constexpr istream_iterator(const istream_iterator& x) noexcept(see below);
```

- 5 *Effects:* Initializes `in_stream` with `x.in_stream` and initializes `value` with `x.value`.

- 6 *Remarks:* An invocation of this constructor may be used in a core constant expression if and only if the initialization of `value` from `x.value` is a constant subexpression (3.15). The exception specification is equivalent to `is_nothrow_copy_constructible_v<T>`.

```
~istream_iterator() = default;
```

- 7 *Remarks:* If `is_trivially_destructible_v<T>` is true, then this destructor is trivial.

#### 24.6.2.3 Operations

[istream.iterator.ops]

```
const T& operator*() const;
```

- 1 *Preconditions:* `in_stream != nullptr` is true.

2     *Returns:* value.

```
const T* operator->() const;
```

3     *Preconditions:* `in_stream != nullptr` is true.

4     *Returns:* `addressof(value)`.

```
istream_iterator& operator++();
```

5     *Preconditions:* `in_stream != nullptr` is true.

6     *Effects:* Equivalent to:

```
 if (!(*in_stream >> value))
 in_stream = nullptr;
```

7     *Returns:* `*this`.

```
template<class T, class charT, class traits, class Distance>
bool operator==(const istream_iterator<T,charT,traits,Distance>& x,
 const istream_iterator<T,charT,traits,Distance>& y);
```

8     *Returns:* `x.in_stream == y.in_stream`.

```
friend bool operator==(const istream_iterator& i, default_sentinel_t);
```

9     *Returns:* `!i.in_stream`.

### 24.6.3 Class template `ostream_iterator`

[ostream.iterator]

#### 24.6.3.1 General

[ostream.iterator.general]

1 `ostream_iterator` writes (using `operator<<`) successive elements onto the output stream from which it was constructed. If it was constructed with `charT*` as a constructor argument, this string, called a *delimiter string*, is written to the stream after every `T` is written.

```
namespace std {
 template<class T, class charT = char, class traits = char_traits<charT>>
 class ostream_iterator {
 public:
 using iterator_category = output_iterator_tag;
 using value_type = void;
 using difference_type = ptrdiff_t;
 using pointer = void;
 using reference = void;
 using char_type = charT;
 using traits_type = traits;
 using ostream_type = basic_ostream<charT,traits>;

 ostream_iterator(ostream_type& s);
 ostream_iterator(ostream_type& s, const charT* delimiter);
 ostream_iterator(const ostream_iterator& x);
 ~ostream_iterator();
 ostream_iterator& operator=(const ostream_iterator&) = default;
 ostream_iterator& operator=(const T& value);

 ostream_iterator& operator*();
 ostream_iterator& operator++();
 ostream_iterator& operator++(int);

 private:
 basic_ostream<charT,traits>* out_stream; // exposition only
 const charT* delim; // exposition only
 };
}
```

#### 24.6.3.2 Constructors and destructor

[ostream.iterator.cons.des]

```
ostream_iterator(ostream_type& s);
```

1     *Effects:* Initializes `out_stream` with `addressof(s)` and `delim` with `nullptr`.



```
ostream_iterator(ostream_type& s, const charT* delimiter);
```

2 *Effects:* Initializes `out_stream` with `addressof(s)` and `delim` with `delimiter`.

### 24.6.3.3 Operations

[ostream.iterator.ops]

```
ostream_iterator& operator=(const T& value);
```

1 *Effects:* As if by:

```
*out_stream << value;
if (delim)
 *out_stream << delim;
return *this;
```

```
ostream_iterator& operator*();
```

2 *Returns:* `*this`.

```
ostream_iterator& operator++();
ostream_iterator& operator++(int);
```

3 *Returns:* `*this`.

## 24.6.4 Class template `istreambuf_iterator`

[istreambuf.iterator]

### 24.6.4.1 General

[istreambuf.iterator.general]

- 1 The class template `istreambuf_iterator` defines an input iterator (24.3.5.3) that reads successive *characters* from the streambuf for which it was constructed. `operator*` provides access to the current input character, if any. Each time `operator++` is evaluated, the iterator advances to the next input character. If the end of stream is reached (`streambuf_type::sgetc()` returns `traits::eof()`), the iterator becomes equal to the *end-of-stream* iterator value. The default constructor `istreambuf_iterator()` and the constructor `istreambuf_iterator(nullptr)` both construct an end-of-stream iterator object suitable for use as an end-of-range. All specializations of `istreambuf_iterator` shall have a trivial copy constructor, a `constexpr` default constructor, and a trivial destructor.
- 2 The result of `operator*()` on an end-of-stream iterator is undefined. For any other iterator value a `char_type` value is returned. It is impossible to assign a character via an input iterator.

```
namespace std {
 template<class charT, class traits = char_traits<charT>>
 class istreambuf_iterator {
 public:
 using iterator_category = input_iterator_tag;
 using value_type = charT;
 using difference_type = traits::off_type;
 using pointer = unspecified;
 using reference = charT;
 using char_type = charT;
 using traits_type = traits;
 using int_type = traits::int_type;
 using streambuf_type = basic_streambuf<charT,traits>;
 using istream_type = basic_istream<charT,traits>;

 // 24.6.4.2, class istreambuf_iterator::proxy
 class proxy; // exposition only

 constexpr istreambuf_iterator() noexcept;
 constexpr istreambuf_iterator(default_sentinel_t) noexcept;
 istreambuf_iterator(const istreambuf_iterator&) noexcept = default;
 ~istreambuf_iterator() = default;
 istreambuf_iterator(istream_type& s) noexcept;
 istreambuf_iterator(streambuf_type* s) noexcept;
 istreambuf_iterator(const proxy& p) noexcept;
 istreambuf_iterator& operator=(const istreambuf_iterator&) noexcept = default;
 charT operator*() const;
 istreambuf_iterator& operator++();
 proxy operator++(int);
 };
}
```

```

 bool equal(const istreambuf_iterator& b) const;

 friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s);

private:
 streambuf_type* sbuf_; // exposition only
};
}

```

#### 24.6.4.2 Class `istreambuf_iterator::proxy` [istreambuf.iterator.proxy]

- <sup>1</sup> Class `istreambuf_iterator<charT, traits>::proxy` is for exposition only. An implementation is permitted to provide equivalent functionality without providing a class with this name. Class `istreambuf_iterator<charT, traits>::proxy` provides a temporary placeholder as the return value of the post-increment operator (`operator++`). It keeps the character pointed to by the previous value of the iterator for some possible future access to get the character.

```

namespace std {
 template<class charT, class traits>
 class istreambuf_iterator<charT, traits>::proxy { // exposition only
 charT keep_;
 basic_streambuf<charT, traits>* sbuf_;
 proxy(charT c, basic_streambuf<charT, traits>* sbuf)
 : keep_(c), sbuf_(sbuf) { }
 public:
 charT operator*() { return keep_; }
 };
}

```

#### 24.6.4.3 Constructors [istreambuf.iterator.cons]

- <sup>1</sup> For each `istreambuf_iterator` constructor in this subclause, an end-of-stream iterator is constructed if and only if the exposition-only member `sbuf_` is initialized with a null pointer value.

```

constexpr istreambuf_iterator() noexcept;
constexpr istreambuf_iterator(default_sentinel_t) noexcept;

```

- <sup>2</sup> *Effects:* Initializes `sbuf_` with `nullptr`.

```
istreambuf_iterator(istream_type& s) noexcept;
```

- <sup>3</sup> *Effects:* Initializes `sbuf_` with `s.rdbuf()`.

```
istreambuf_iterator(streambuf_type* s) noexcept;
```

- <sup>4</sup> *Effects:* Initializes `sbuf_` with `s`.

```
istreambuf_iterator(const proxy& p) noexcept;
```

- <sup>5</sup> *Effects:* Initializes `sbuf_` with `p.sbuf_`.

#### 24.6.4.4 Operations [istreambuf.iterator.ops]

```
charT operator*() const;
```

- <sup>1</sup> *Returns:* The character obtained via the `streambuf` member `sbuf_>sgetc()`.

```
istreambuf_iterator& operator++();
```

- <sup>2</sup> *Effects:* As if by `sbuf_>sputc()`.

- <sup>3</sup> *Returns:* `*this`.

```
proxy operator++(int);
```

- <sup>4</sup> *Returns:* `proxy(sbuf_>sputc(), sbuf_)`.

```
bool equal(const istreambuf_iterator& b) const;
```

- <sup>5</sup> *Returns:* `true` if and only if both iterators are at end-of-stream, or neither is at end-of-stream, regardless of what `streambuf` object they use.

```
template<class charT, class traits>
 bool operator==(const istreambuf_iterator<charT,traits>& a,
 const istreambuf_iterator<charT,traits>& b);
```

6     *Returns:* a.equal(b).

```
friend bool operator==(const istreambuf_iterator& i, default_sentinel_t s);
```

7     *Returns:* i.equal(s).

### 24.6.5 Class template ostreambuf\_iterator

[ostreambuf.iterator]

#### 24.6.5.1 General

[ostreambuf.iterator.general]

1 The class template ostreambuf\_iterator writes successive *characters* onto the output stream from which it was constructed.

```
namespace std {
 template<class charT, class traits = char_traits<charT>>
 class ostreambuf_iterator {
 public:
 using iterator_category = output_iterator_tag;
 using value_type = void;
 using difference_type = ptrdiff_t;
 using pointer = void;
 using reference = void;
 using char_type = charT;
 using traits_type = traits;
 using streambuf_type = basic_streambuf<charT,traits>;
 using ostream_type = basic_ostream<charT,traits>;

 ostreambuf_iterator(ostream_type& s) noexcept;
 ostreambuf_iterator(streambuf_type* s) noexcept;
 ostreambuf_iterator& operator=(charT c);

 ostreambuf_iterator& operator*();
 ostreambuf_iterator& operator++();
 ostreambuf_iterator& operator++(int);
 bool failed() const noexcept;

 private:
 streambuf_type* sbuf_; // exposition only
 };
}
```

#### 24.6.5.2 Constructors

[ostreambuf.iter.cons]

```
ostreambuf_iterator(ostream_type& s) noexcept;
```

1     *Preconditions:* s.rdbuf() is not a null pointer.

2     *Effects:* Initializes sbuf\_ with s.rdbuf().

```
ostreambuf_iterator(streambuf_type* s) noexcept;
```

3     *Preconditions:* s is not a null pointer.

4     *Effects:* Initializes sbuf\_ with s.

#### 24.6.5.3 Operations

[ostreambuf.iter.ops]

```
ostreambuf_iterator& operator=(charT c);
```

1     *Effects:* If failed() yields false, calls sbuf\_->sputc(c); otherwise has no effect.

2     *Returns:* \*this.

```
ostreambuf_iterator& operator*();
```

3     *Returns:* \*this.

```
ostreambuf_iterator& operator++();
ostreambuf_iterator& operator++(int);
```

4 *Returns:* `*this`.

```
bool failed() const noexcept;
```

5 *Returns:* `true` if in any prior use of member `operator=`, the call to `sbuf_->sputc()` returned `traits::eof()`; or `false` otherwise.

## 24.7 Range access

[iterator.range]

<sup>1</sup> In addition to being available via inclusion of the `<iterator>` header, the function templates in 24.7 are available when any of the following headers are included:

- (1.1) — `<array>` (23.3.2)
- (1.2) — `<deque>` (23.3.4)
- (1.3) — `<flat_map>` (23.6.7)
- (1.4) — `<flat_set>` (23.6.10)
- (1.5) — `<forward_list>` (23.3.6)
- (1.6) — `<hive>` (23.3.8)
- (1.7) — `<inplace_vector>` (23.3.15)
- (1.8) — `<list>` (23.3.10)
- (1.9) — `<map>` (23.4.2)
- (1.10) — `<optional>` (22.5.2)
- (1.11) — `<regex>` (28.6.3)
- (1.12) — `<set>` (23.4.5)
- (1.13) — `<simd>` (29.10.3)
- (1.14) — `<span>` (23.7.2.1)
- (1.15) — `<stacktrace>` (19.6.2)
- (1.16) — `<string>` (27.4.2)
- (1.17) — `<string_view>` (27.3.2)
- (1.18) — `<unordered_map>` (23.5.2)
- (1.19) — `<unordered_set>` (23.5.5)
- (1.20) — `<valarray>` (29.6.1)
- (1.21) — `<vector>` (23.3.12)

```
template<class C> constexpr auto begin(C& c) noexcept(noexcept(c.begin()))
-> decltype(c.begin());
template<class C> constexpr auto begin(const C& c) noexcept(noexcept(c.begin()))
-> decltype(c.begin());
```

2 *Returns:* `c.begin()`.

```
template<class C> constexpr auto end(C& c) noexcept(noexcept(c.end())) -> decltype(c.end());
template<class C> constexpr auto end(const C& c) noexcept(noexcept(c.end())) -> decltype(c.end());
```

3 *Returns:* `c.end()`.

```
template<class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
```

4 *Returns:* `array`.

```
template<class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
```

5 *Returns:* `array + N`.

```

template<class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
-> decltype(std::begin(c));
6 Returns: std::begin(c).

template<class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
-> decltype(std::end(c));
7 Returns: std::end(c).

template<class C> constexpr auto rbegin(C& c) noexcept(noexcept(c.rbegin()))
-> decltype(c.rbegin());
template<class C> constexpr auto rbegin(const C& c) noexcept(noexcept(c.rbegin()))
-> decltype(c.rbegin());
8 Returns: c.rbegin().

template<class C> constexpr auto rend(C& c) noexcept(noexcept(c.rend())) -> decltype(c.rend());
template<class C> constexpr auto rend(const C& c) noexcept(noexcept(c.rend()))
-> decltype(c.rend());
9 Returns: c.rend().

template<class T, size_t N> constexpr reverse_iterator<T*> rbegin(T (&array)[N]) noexcept;
10 Returns: reverse_iterator<T*>(array + N).

template<class T, size_t N> constexpr reverse_iterator<T*> rend(T (&array)[N]) noexcept;
11 Returns: reverse_iterator<T*>(array).

template<class E> constexpr reverse_iterator<const E*> rbegin(initializer_list<E> il) noexcept;
12 Returns: reverse_iterator<const E*>(il.end()).

template<class E> constexpr reverse_iterator<const E*> rend(initializer_list<E> il) noexcept;
13 Returns: reverse_iterator<const E*>(il.begin()).

template<class C> constexpr auto crbegin(const C& c) noexcept(noexcept(std::rbegin(c)))
-> decltype(std::rbegin(c));
14 Returns: std::rbegin(c).

template<class C> constexpr auto crend(const C& c) noexcept(noexcept(c.crend()))
-> decltype(std::rend(c));
15 Returns: std::rend(c).

template<class C> constexpr auto size(const C& c) noexcept(noexcept(c.size()))
-> decltype(c.size());
16 Returns: c.size().

template<class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept;
17 Returns: N.

template<class C> constexpr auto ssize(const C& c) noexcept(noexcept(c.size()))
-> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;
18 Effects: Equivalent to:
 return static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size());

template<class T, ptrdiff_t N> constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
19 Returns: N.

template<class C> constexpr auto empty(const C& c) noexcept(noexcept(c.empty()))
-> decltype(c.empty());
20 Returns: c.empty().

```

```
template<class T, size_t N> constexpr bool empty(const T (&array)[N]) noexcept;
```

21     *Returns:* false.

```
template<class C> constexpr auto data(C& c) noexcept(noexcept(c.data())) -> decltype(c.data());
template<class C> constexpr auto data(const C& c) noexcept(noexcept(c.data()))
 -> decltype(c.data());
```

22     *Returns:* c.data().

```
template<class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
```

23     *Returns:* array.

## 25 Ranges library

[ranges]

### 25.1 General

[ranges.general]

- <sup>1</sup> This Clause describes components for dealing with ranges of elements.
- <sup>2</sup> The following subclauses describe range and view requirements, and components for range primitives and range generators as summarized in [Table 84](#).

Table 84 — Ranges library summary [tab:range.summary]

|                      | Subclause        | Header      |
|----------------------|------------------|-------------|
| <a href="#">25.3</a> | Range access     | <ranges>    |
| <a href="#">25.4</a> | Requirements     |             |
| <a href="#">25.5</a> | Range utilities  |             |
| <a href="#">25.6</a> | Range factories  |             |
| <a href="#">25.7</a> | Range adaptors   |             |
| <a href="#">25.8</a> | Range generators | <generator> |

### 25.2 Header <ranges> synopsis

[ranges.syn]

```
// mostly freestanding
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2
#include <iterator> // see 24.2

namespace std::ranges {
 inline namespace unspecified {
 // 25.3, range access
 inline constexpr unspecified begin = unspecified;
 inline constexpr unspecified end = unspecified;
 inline constexpr unspecified cbegin = unspecified;
 inline constexpr unspecified cend = unspecified;
 inline constexpr unspecified rbegin = unspecified;
 inline constexpr unspecified rend = unspecified;
 inline constexpr unspecified crbegin = unspecified;
 inline constexpr unspecified crend = unspecified;

 inline constexpr unspecified size = unspecified;
 inline constexpr unspecified reserve_hint = unspecified;
 inline constexpr unspecified ssize = unspecified;
 inline constexpr unspecified empty = unspecified;
 inline constexpr unspecified data = unspecified;
 inline constexpr unspecified cdata = unspecified;
 }

 // 25.4.2, ranges
 template<class T>
 concept range = see below;

 template<class T>
 constexpr bool enable_borrowed_range = false;

 template<class T>
 concept borrowed_range = see below;

 template<class T>
 using iterator_t = decltype(ranges::begin(declval<T&>()));
```

```

template<range R>
 using sentinel_t = decltype(ranges::end(declval<R&>()));
template<range R>
 using const_iterator_t = decltype(ranges::cbegin(declval<R&>()));
template<range R>
 using const_sentinel_t = decltype(ranges::cend(declval<R&>()));
template<range R>
 using range_difference_t = iter_difference_t<iterator_t<R>>;
template<sized_range R>
 using range_size_t = decltype(ranges::size(declval<R&>()));
template<range R>
 using range_value_t = iter_value_t<iterator_t<R>>;
template<range R>
 using range_reference_t = iter_reference_t<iterator_t<R>>;
template<range R>
 using range_const_reference_t = iter_const_reference_t<iterator_t<R>>;
template<range R>
 using range_rvalue_reference_t = iter_rvalue_reference_t<iterator_t<R>>;
template<range R>
 using range_common_reference_t = iter_common_reference_t<iterator_t<R>>;

// 25.4.4, sized ranges
template<class>
 constexpr bool disable_sized_range = false;

template<class T>
 concept approximately_sized_range = see below;

template<class T>
 concept sized_range = see below;

// 25.4.5, views
template<class T>
 constexpr bool enable_view = see below;

struct view_base {};

template<class T>
 concept view = see below;

// 25.4.6, other range refinements
template<class R, class T>
 concept output_range = see below;

template<class T>
 concept input_range = see below;

template<class T>
 concept forward_range = see below;

template<class T>
 concept bidirectional_range = see below;

template<class T>
 concept random_access_range = see below;

template<class T>
 concept contiguous_range = see below;

template<class T>
 concept common_range = see below;

template<class T>
 concept viewable_range = see below;

```



```

template<class T>
 concept constant_range = see below;

template<class T>
 concept sized-random-access-range = see below; // exposition only

// 25.5.3, class template view_interface
template<class D>
 requires is_class_v<D> && same_as<D, remove_cv_t<D>>
class view_interface; // partially freestanding

// 25.5.4, sub-ranges
enum class subrange_kind : bool { unsized, sized };

template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K = see below>
 requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
class subrange;

template<class I, class S, subrange_kind K>
 constexpr bool enable_borrowed_range<subrange<I, S, K>> = true;

template<size_t N, class I, class S, subrange_kind K>
 requires ((N == 0 && copyable<I>) || N == 1)
 constexpr auto get(const subrange<I, S, K>& r);

template<size_t N, class I, class S, subrange_kind K>
 requires (N < 2)
 constexpr auto get(subrange<I, S, K>&& r);
}

namespace std {
 using ranges::get;
}

namespace std::ranges {
 // 25.5.5, dangling iterator handling
 struct dangling;

 // 25.5.6, class template elements_of
 template<range R, class Allocator = allocator<byte>>
 struct elements_of; // hosted

 template<range R>
 using borrowed_iterator_t = see below;

 template<range R>
 using borrowed_subrange_t = see below;

 // 25.5.7, range conversions
 template<class C, input_range R, class... Args> requires (!view<C>)
 constexpr C to(R&& r, Args&&... args);
 template<template<class...> class C, input_range R, class... Args>
 constexpr auto to(R&& r, Args&&... args);
 template<class C, class... Args> requires (!view<C>)
 constexpr auto to(Args&&... args);
 template<template<class...> class C, class... Args>
 constexpr auto to(Args&&... args);

 // 25.6.2, empty view
 template<class T>
 requires is_object_v<T>
 class empty_view;

```

```

template<class T>
 constexpr bool enable_borrowed_range<empty_view<T>> = true;

namespace views {
 template<class T>
 constexpr empty_view<T> empty{};
}

// 25.6.3, single view
template<move_constructible T>
 requires is_object_v<T>
class single_view;

namespace views { inline constexpr unspecified single = unspecified; }

template<bool Const, class T>
 using maybe-const = conditional_t<Const, const T, T>; // exposition only

// 25.6.4, iota view
template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>
 requires weakly-equality-comparable-with<W, Bound> && copyable<W>
class iota_view;

template<class W, class Bound>
 constexpr bool enable_borrowed_range<iota_view<W, Bound>> = true;

namespace views {
 inline constexpr unspecified iota = unspecified;
 inline constexpr unspecified indices = unspecified;
}

// 25.6.5, repeat view
template<move_constructible T, semiregular Bound = unreachable_sentinel_t>
 requires see below
class repeat_view;

namespace views { inline constexpr unspecified repeat = unspecified; }

// 25.6.6, istream view
template<movable Val, class CharT, class Traits = char_traits<CharT>>
 requires see below
class basic_istream_view; // hosted
template<class Val>
 using istream_view = basic_istream_view<Val, char>; // hosted
template<class Val>
 using wistream_view = basic_istream_view<Val, wchar_t>; // hosted

namespace views {
 template<class T> constexpr unspecified istream = unspecified; // hosted
}

// 25.7.2, range adaptor objects
template<class D>
 requires is_class_v<D> && same_as<D, remove_cv_t<D>>
class range_adaptor_closure { };

// 25.7.6, all view
namespace views {
 inline constexpr unspecified all = unspecified;

 template<viewable_range R>
 using all_t = decltype(all(declval<R>()));
}

```

```

// 25.7.6.2, ref view
template<range R>
 requires is_object_v<R>
class ref_view;

template<class T>
 constexpr bool enable_borrowed_range<ref_view<T>> = true;

// 25.7.6.3, owning view
template<range R>
 requires see below
class owning_view;

template<class T>
 constexpr bool enable_borrowed_range<owning_view<T>> =
 enable_borrowed_range<T>;

// 25.7.7, as rvalue view
template<view V>
 requires input_range<V>
class as_rvalue_view;

template<class T>
 constexpr bool enable_borrowed_range<as_rvalue_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified as_rvalue = unspecified; }

// 25.7.8, filter view
template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
 requires view<V> && is_object_v<Pred>
class filter_view;

namespace views { inline constexpr unspecified filter = unspecified; }

// 25.7.9, transform view
template<input_range V, move_constructible F>
 requires view<V> && is_object_v<F> &&
 regular_invocable<F&, range_reference_t<V>> &&
 can_reference<invoke_result_t<F&, range_reference_t<V>>>
class transform_view;

namespace views { inline constexpr unspecified transform = unspecified; }

// 25.7.10, take view
template<view> class take_view;

template<class T>
 constexpr bool enable_borrowed_range<take_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified take = unspecified; }

// 25.7.11, take while view
template<view V, class Pred>
 requires input_range<V> && is_object_v<Pred> &&
 indirect_unary_predicate<const Pred, iterator_t<V>>
class take_while_view;

namespace views { inline constexpr unspecified take_while = unspecified; }

// 25.7.12, drop view
template<view V>
class drop_view;

```

```

template<class T>
 constexpr bool enable_borrowed_range<drop_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified drop = unspecified; }

// 25.7.13, drop while view
template<view V, class Pred>
 requires input_range<V> && is_object_v<Pred> &&
 indirect_unary_predicate<const Pred, iterator_t<V>>
 class drop_while_view;

template<class T, class Pred>
 constexpr bool enable_borrowed_range<drop_while_view<T, Pred>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified drop_while = unspecified; }

// 25.7.14, join view
template<input_range V>
 requires view<V> && input_range<range_reference_t<V>>
 class join_view;

namespace views { inline constexpr unspecified join = unspecified; }

// 25.7.15, join with view
template<input_range V, forward_range Pattern>
 requires see below
 class join_with_view;

namespace views { inline constexpr unspecified join_with = unspecified; }

// 25.7.16, lazy split view
template<class R>
 concept tiny-range = see below; // exposition only

template<input_range V, forward_range Pattern>
 requires view<V> && view<Pattern> &&
 indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
 (forward_range<V> || tiny-range<Pattern>)
 class lazy_split_view;

// 25.7.17, split view
template<forward_range V, forward_range Pattern>
 requires view<V> && view<Pattern> &&
 indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
 class split_view;

namespace views {
 inline constexpr unspecified lazy_split = unspecified;
 inline constexpr unspecified split = unspecified;
}

// 25.7.18, concat view
template<input_range... Views>
 requires see below
 class concat_view;

namespace views { inline constexpr unspecified concat = unspecified; }

// 25.7.19, counted view
namespace views { inline constexpr unspecified counted = unspecified; }

```

```

// 25.7.20, common view
template<view V>
 requires (!common_range<V> && copyable<iterator_t<V>>())
class common_view;

template<class T>
 constexpr bool enable_borrowed_range<common_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified common = unspecified; }

// 25.7.21, reverse view
template<view V>
 requires bidirectional_range<V>
class reverse_view;

template<class T>
 constexpr bool enable_borrowed_range<reverse_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified reverse = unspecified; }

// 25.7.22, as const view
template<input_range R>
 constexpr auto& possibly-const-range(R& r) noexcept { // exposition only
 if constexpr (input_range<const R>) {
 return const_cast<const R&>(r);
 } else {
 return r;
 }
 }

template<view V>
 requires input_range<V>
class as_const_view;

template<class T>
 constexpr bool enable_borrowed_range<as_const_view<T>> =
 enable_borrowed_range<T>;

namespace views { inline constexpr unspecified as_const = unspecified; }

// 25.7.23, elements view
template<input_range V, size_t N>
 requires see below
class elements_view;

template<class T, size_t N>
 constexpr bool enable_borrowed_range<elements_view<T, N>> =
 enable_borrowed_range<T>;

template<class R>
 using keys_view = elements_view<R, 0>;
template<class R>
 using values_view = elements_view<R, 1>;

namespace views {
 template<size_t N>
 constexpr unspecified elements = unspecified;
 inline constexpr auto keys = elements<0>;
 inline constexpr auto values = elements<1>;
}

```

```

// 25.7.24, enumerate view
template<view V>
 requires see below
class enumerate_view;

template<class View>
 constexpr bool enable_borrowed_range<enumerate_view<View>> =
 enable_borrowed_range<View>;

namespace views { inline constexpr unspecified enumerate = unspecified; }

// 25.7.25, zip view
template<input_range... Views>
 requires (view<Views> && ...) && (sizeof...(Views) > 0)
class zip_view;

template<class... Views>
 constexpr bool enable_borrowed_range<zip_view<Views...>> =
 (enable_borrowed_range<Views> && ...);

namespace views { inline constexpr unspecified zip = unspecified; }

// 25.7.26, zip transform view
template<move_constructible F, input_range... Views>
 requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
 regular_invocable<F&, range_reference_t<Views>...> &&
 can_reference<invoke_result_t<F&, range_reference_t<Views>...>>
class zip_transform_view;

namespace views { inline constexpr unspecified zip_transform = unspecified; }

// 25.7.27, adjacent view
template<forward_range V, size_t N>
 requires view<V> && (N > 0)
class adjacent_view;

template<class V, size_t N>
 constexpr bool enable_borrowed_range<adjacent_view<V, N>> =
 enable_borrowed_range<V>;

namespace views {
 template<size_t N>
 constexpr unspecified adjacent = unspecified;
 inline constexpr auto pairwise = adjacent<2>;
}

// 25.7.28, adjacent transform view
template<forward_range V, move_constructible F, size_t N>
 requires see below
class adjacent_transform_view;

namespace views {
 template<size_t N>
 constexpr unspecified adjacent_transform = unspecified;
 inline constexpr auto pairwise_transform = adjacent_transform<2>;
}

// 25.7.29, chunk view
template<view V>
 requires input_range<V>
class chunk_view;

```

```

template<view V>
 requires forward_range<V>
class chunk_view<V>;

template<class V>
 constexpr bool enable_borrowed_range<chunk_view<V>> =
 forward_range<V> && enable_borrowed_range<V>;

namespace views { inline constexpr unspecified chunk = unspecified; }

// 25.7.30, slide view
template<forward_range V>
 requires view<V>
class slide_view;

template<class V>
 constexpr bool enable_borrowed_range<slide_view<V>> =
 enable_borrowed_range<V>;

namespace views { inline constexpr unspecified slide = unspecified; }

// 25.7.31, chunk by view
template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>> Pred>
 requires view<V> && is_object_v<Pred>
class chunk_by_view;

namespace views { inline constexpr unspecified chunk_by = unspecified; }

// 25.7.32, stride view
template<input_range V>
 requires view<V>
class stride_view;

template<class V>
 constexpr bool enable_borrowed_range<stride_view<V>> =
 enable_borrowed_range<V>;

namespace views { inline constexpr unspecified stride = unspecified; }

// 25.7.33, cartesian product view
template<input_range First, forward_range... Vs>
 requires (view<First> && ... && view<Vs>)
class cartesian_product_view;

namespace views { inline constexpr unspecified cartesian_product = unspecified; }

// 25.7.34, cache latest view
template<input_range V>
 requires view<V>
class cache_latest_view;

namespace views { inline constexpr unspecified cache_latest = unspecified; }

// 25.7.35, as input view
template<input_range V>
 requires view<V>
class as_input_view;

template<class V>
 constexpr bool enable_borrowed_range<as_input_view<V>> =
 enable_borrowed_range<V>;

namespace views { inline constexpr unspecified as_input = unspecified; }
}

```

```

namespace std {
 namespace views = ranges::views;

 template<class T> struct tuple_size;
 template<size_t I, class T> struct tuple_element;

 template<class I, class S, ranges::subrange_kind K>
 struct tuple_size<ranges::subrange<I, S, K>>
 : integral_constant<size_t, 2> {};
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<0, ranges::subrange<I, S, K>> {
 using type = I;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<1, ranges::subrange<I, S, K>> {
 using type = S;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<0, const ranges::subrange<I, S, K>> {
 using type = I;
 };
 template<class I, class S, ranges::subrange_kind K>
 struct tuple_element<1, const ranges::subrange<I, S, K>> {
 using type = S;
 };

 struct from_range_t { explicit from_range_t() = default; };
 inline constexpr from_range_t from_range{};
}

```

- <sup>1</sup> Within this Clause, for an integer-like type *X* (24.3.4.4), *make-unsigned-like-t<X>* denotes `make_unsigned_t<X>` if *X* is an integer type; otherwise, it denotes a corresponding unspecified unsigned-integer-like type of the same width as *X*. For an expression *x* of type *X*, *to-unsigned-like(x)* is *x* explicitly converted to *make-unsigned-like-t<X>*.
- <sup>2</sup> Also within this Clause, *make-signed-like-t<X>* for an integer-like type *X* denotes `make_signed_t<X>` if *X* is an integer type; otherwise, it denotes a corresponding unspecified signed-integer-like type of the same width as *X*.

## 25.3 Range access

[range.access]

### 25.3.1 General

[range.access.general]

- <sup>1</sup> In addition to being available via inclusion of the `<ranges>` header, the customization point objects in 25.3 are available when the header `<iterator>` (24.2) is included.
- <sup>2</sup> Within 25.3, the *reified object* of a subexpression *E* denotes
  - (2.1) — the same object as *E* if *E* is a glvalue, or
  - (2.2) — the result of applying the temporary materialization conversion (7.3.5) to *E* otherwise.

### 25.3.2 ranges::begin

[range.access.begin]

- <sup>1</sup> The name `ranges::begin` denotes a customization point object (16.3.3.5).
- <sup>2</sup> Given a subexpression *E* with type *T*, let *t* be an lvalue that denotes the reified object for *E*. Then:
  - (2.1) — If *E* is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::begin(E)` is ill-formed.
  - (2.2) — Otherwise, if *T* is an array type (9.3.4.5) and `remove_all_extents_t<T>` is an incomplete type, `ranges::begin(E)` is ill-formed with no diagnostic required.
  - (2.3) — Otherwise, if *T* is an array type, `ranges::begin(E)` is expression-equivalent to `t + 0`.
  - (2.4) — Otherwise, if `auto(t.begin())` is a valid expression whose type models `input_or_output_iterator`, `ranges::begin(E)` is expression-equivalent to `auto(t.begin())`.



- (2.5) — Otherwise, if `T` is a class or enumeration type and `auto(begin(t))` is a valid expression whose type models `input_or_output_iterator` where the meaning of `begin` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::begin(E)` is expression-equivalent to that expression.
- (2.6) — Otherwise, `ranges::begin(E)` is ill-formed.
- <sup>3</sup> [Note 1: Diagnosable ill-formed cases above result in substitution failure when `ranges::begin(E)` appears in the immediate context of a template instantiation. — end note]
- <sup>4</sup> [Note 2: Whenever `ranges::begin(E)` is a valid expression, its type models `input_or_output_iterator`. — end note]

### 25.3.3 `ranges::end`

[range.access.end]

- <sup>1</sup> The name `ranges::end` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (2.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is false, `ranges::end(E)` is ill-formed.
  - (2.2) — Otherwise, if `T` is an array type (9.3.4.5) and `remove_all_extents_t<T>` is an incomplete type, `ranges::end(E)` is ill-formed with no diagnostic required.
  - (2.3) — Otherwise, if `T` is an array of unknown bound, `ranges::end(E)` is ill-formed.
  - (2.4) — Otherwise, if `T` is an array, `ranges::end(E)` is expression-equivalent to `t + extent_v<T>`.
  - (2.5) — Otherwise, if `auto(t.end())` is a valid expression whose type models `sentinel_for<iterator_t<T>>` then `ranges::end(E)` is expression-equivalent to `auto(t.end())`.
  - (2.6) — Otherwise, if `T` is a class or enumeration type and `auto(end(t))` is a valid expression whose type models `sentinel_for<iterator_t<T>>` where the meaning of `end` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::end(E)` is expression-equivalent to that expression.
  - (2.7) — Otherwise, `ranges::end(E)` is ill-formed.
- <sup>3</sup> [Note 1: Diagnosable ill-formed cases above result in substitution failure when `ranges::end(E)` appears in the immediate context of a template instantiation. — end note]
- <sup>4</sup> [Note 2: Whenever `ranges::end(E)` is a valid expression, the types `S` and `I` of `ranges::end(E)` and `ranges::begin(E)` model `sentinel_for<S, I>`. — end note]

### 25.3.4 `ranges::cbegin`

[range.access.cbegin]

- <sup>1</sup> The name `ranges::cbegin` denotes a customization point object (16.3.3.3.5). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is false, `ranges::cbegin(E)` is ill-formed.
  - (1.2) — Otherwise, let `U` be `ranges::begin(possibly-const-range(t))`. `ranges::cbegin(E)` is expression-equivalent to `const_iterator<decltype(U)>(U)`.
- <sup>2</sup> [Note 1: Whenever `ranges::cbegin(E)` is a valid expression, its type models `input_or_output_iterator` and `constant_iterator`. — end note]

### 25.3.5 `ranges::cend`

[range.access.cend]

- <sup>1</sup> The name `ranges::cend` denotes a customization point object (16.3.3.3.5). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is false, `ranges::cend(E)` is ill-formed.
  - (1.2) — Otherwise, let `U` be `ranges::end(possibly-const-range(t))`. `ranges::cend(E)` is expression-equivalent to `const_sentinel<decltype(U)>(U)`.
- <sup>2</sup> [Note 1: Whenever `ranges::cend(E)` is a valid expression, the types `S` and `I` of the expressions `ranges::cend(E)` and `ranges::cbegin(E)` model `sentinel_for<S, I>`. If `S` models `input_iterator`, then `S` also models `constant_iterator`. — end note]

**25.3.6 ranges::rbegin****[range.access.rbegin]**

- <sup>1</sup> The name `ranges::rbegin` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (2.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::rbegin(E)` is ill-formed.
  - (2.2) — Otherwise, if `T` is an array type (9.3.4.5) and `remove_all_extents_t<T>` is an incomplete type, `ranges::rbegin(E)` is ill-formed with no diagnostic required.
  - (2.3) — Otherwise, if `auto(t.rbegin())` is a valid expression whose type models `input_or_output_iterator`, `ranges::rbegin(E)` is expression-equivalent to `auto(t.rbegin())`.
  - (2.4) — Otherwise, if `T` is a class or enumeration type and `auto(rbegin(t))` is a valid expression whose type models `input_or_output_iterator` where the meaning of `rbegin` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::rbegin(E)` is expression-equivalent to that expression.
  - (2.5) — Otherwise, if both `ranges::begin(t)` and `ranges::end(t)` are valid expressions of the same type which models `bidirectional_iterator` (24.3.4.12), `ranges::rbegin(E)` is expression-equivalent to `make_reverse_iterator(ranges::end(t))`.
  - (2.6) — Otherwise, `ranges::rbegin(E)` is ill-formed.
- <sup>3</sup> [Note 1: Diagnosable ill-formed cases above result in substitution failure when `ranges::rbegin(E)` appears in the immediate context of a template instantiation. — end note]
- <sup>4</sup> [Note 2: Whenever `ranges::rbegin(E)` is a valid expression, its type models `input_or_output_iterator`. — end note]

**25.3.7 ranges::rend****[range.access.rend]**

- <sup>1</sup> The name `ranges::rend` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (2.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::rend(E)` is ill-formed.
  - (2.2) — Otherwise, if `T` is an array type (9.3.4.5) and `remove_all_extents_t<T>` is an incomplete type, `ranges::rend(E)` is ill-formed with no diagnostic required.
  - (2.3) — Otherwise, if `auto(t.rend())` is a valid expression whose type models `sentinel_for<decltype(ranges::rbegin(E))>` then `ranges::rend(E)` is expression-equivalent to `auto(t.rend())`.
  - (2.4) — Otherwise, if `T` is a class or enumeration type and `auto(rend(t))` is a valid expression whose type models `sentinel_for<decltype(ranges::rbegin(E))>` where the meaning of `rend` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::rend(E)` is expression-equivalent to that expression.
  - (2.5) — Otherwise, if both `ranges::begin(t)` and `ranges::end(t)` are valid expressions of the same type which models `bidirectional_iterator` (24.3.4.12), then `ranges::rend(E)` is expression-equivalent to `make_reverse_iterator(ranges::begin(t))`.
  - (2.6) — Otherwise, `ranges::rend(E)` is ill-formed.
- <sup>3</sup> [Note 1: Diagnosable ill-formed cases above result in substitution failure when `ranges::rend(E)` appears in the immediate context of a template instantiation. — end note]
- <sup>4</sup> [Note 2: Whenever `ranges::rend(E)` is a valid expression, the types `S` and `I` of the expressions `ranges::rend(E)` and `ranges::rbegin(E)` model `sentinel_for<S, I>`. — end note]

**25.3.8 ranges::crbegin****[range.access.crbegin]**

- <sup>1</sup> The name `ranges::crbegin` denotes a customization point object (16.3.3.3.5). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::crbegin(E)` is ill-formed.
  - (1.2) — Otherwise, let `U` be `ranges::rbegin(possibly-const-range(t))`. `ranges::crbegin(E)` is expression-equivalent to `const_iterator<decltype(U)>(U)`.

- <sup>2</sup> [Note 1: Whenever `ranges::crbegin(E)` is a valid expression, its type models `input_or_output_iterator` and `constant_iterator`. — end note]

### 25.3.9 `ranges::crend`

[range.access.crend]

- <sup>1</sup> The name `ranges::crend` denotes a customization point object (16.3.3.3.5). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
- (1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::crend(E)` is ill-formed.
  - (1.2) — Otherwise, let `U` be `ranges::rend(possibly-const-range(t))`. `ranges::crend(E)` is expression-equivalent to `const_sentinel<decltype(U)>(U)`.
- <sup>2</sup> [Note 1: Whenever `ranges::crend(E)` is a valid expression, the types `S` and `I` of the expressions `ranges::crend(E)` and `ranges::crbegin(E)` model `sentinel_for<S, I>`. If `S` models `input_iterator`, then `S` also models `constant_iterator`. — end note]

### 25.3.10 `ranges::size`

[range.prim.size]

- <sup>1</sup> The name `ranges::size` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
- (2.1) — If `T` is an array of unknown bound (9.3.4.5), `ranges::size(E)` is ill-formed.
  - (2.2) — Otherwise, if `T` is an array type, `ranges::size(E)` is expression-equivalent to `auto(extent_v<T>)`.
  - (2.3) — Otherwise, if `disable_sized_range<remove_cv_t<T>>` (25.4.4) is `false` and `auto(t.size())` is a valid expression of integer-like type (24.3.4.4), `ranges::size(E)` is expression-equivalent to `auto(t.size())`.
  - (2.4) — Otherwise, if `T` is a class or enumeration type, `disable_sized_range<remove_cv_t<T>>` is `false` and `auto(size(t))` is a valid expression of integer-like type where the meaning of `size` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::size(E)` is expression-equivalent to that expression.
  - (2.5) — Otherwise, if `to-unsigned-like(ranges::end(t) - ranges::begin(t))` (25.2) is a valid expression and the types `I` and `S` of `ranges::begin(t)` and `ranges::end(t)` (respectively) model both `sized_sentinel_for<S, I>` (24.3.4.8) and `forward_iterator<I>`, then `ranges::size(E)` is expression-equivalent to `to-unsigned-like(ranges::end(t) - ranges::begin(t))`.
  - (2.6) — Otherwise, `ranges::size(E)` is ill-formed.
- <sup>3</sup> [Note 1: Diagnosable ill-formed cases above result in substitution failure when `ranges::size(E)` appears in the immediate context of a template instantiation. — end note]
- <sup>4</sup> [Note 2: Whenever `ranges::size(E)` is a valid expression, its type is integer-like. — end note]

### 25.3.11 `ranges::ssize`

[range.prim.ssize]

- <sup>1</sup> The name `ranges::ssize` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. If `ranges::size(t)` is ill-formed, `ranges::ssize(E)` is ill-formed. Otherwise let `D` be `make_signed-like-t<decltype(ranges::size(t))>`, or `ptrdiff_t` if it is wider than that type; `ranges::ssize(E)` is expression-equivalent to `static_cast<D>(ranges::size(t))`.

### 25.3.12 `ranges::reserve_hint`

[range.prim.size.hint]

- <sup>1</sup> The name `ranges::reserve_hint` denotes a customization point object (16.3.3.3.5).
- <sup>2</sup> Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
- (2.1) — If `ranges::size(E)` is a valid expression, `ranges::reserve_hint(E)` is expression-equivalent to `ranges::size(E)`.
  - (2.2) — Otherwise, if `auto(t.reserve_hint())` is a valid expression of integer-like type (24.3.4.4), `ranges::reserve_hint(E)` is expression-equivalent to `auto(t.reserve_hint())`.
  - (2.3) — Otherwise, if `T` is a class or enumeration type and `auto(reserve_hint(t))` is a valid expression of integer-like type where the meaning of `reserve_hint` is established as-if by performing argument-dependent lookup only (6.5.4), then `ranges::reserve_hint(E)` is expression-equivalent to that expression.

- (2.4) — Otherwise, `ranges::reserve_hint(E)` is ill-formed.

[*Note 1:* Diagnosable ill-formed cases above result in substitution failure when `ranges::reserve_hint(E)` appears in the immediate context of a template instantiation. — *end note*]

[*Note 2:* Whenever `ranges::reserve_hint(E)` is a valid expression, its type is integer-like. — *end note*]

### 25.3.13 `ranges::empty`

[`range.prim.empty`]

- 1 The name `ranges::empty` denotes a customization point object (16.3.3.3.5).
- 2 Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (2.1) — If `T` is an array of unknown bound (9.3.4.5), `ranges::empty(E)` is ill-formed.
  - (2.2) — Otherwise, if `bool(t.empty())` is a valid expression, `ranges::empty(E)` is expression-equivalent to `bool(t.empty())`.
  - (2.3) — Otherwise, if `(ranges::size(t) == 0)` is a valid expression, `ranges::empty(E)` is expression-equivalent to `(ranges::size(t) == 0)`.
  - (2.4) — Otherwise, if `bool(ranges::begin(t) == ranges::end(t))` is a valid expression and the type of `ranges::begin(t)` models `forward_iterator`, `ranges::empty(E)` is expression-equivalent to `bool(ranges::begin(t) == ranges::end(t))`.
  - (2.5) — Otherwise, `ranges::empty(E)` is ill-formed.
- 3 [*Note 1:* Diagnosable ill-formed cases above result in substitution failure when `ranges::empty(E)` appears in the immediate context of a template instantiation. — *end note*]
- 4 [*Note 2:* Whenever `ranges::empty(E)` is a valid expression, it has type `bool`. — *end note*]

### 25.3.14 `ranges::data`

[`range.prim.data`]

- 1 The name `ranges::data` denotes a customization point object (16.3.3.3.5).
- 2 Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (2.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::data(E)` is ill-formed.
  - (2.2) — Otherwise, if `T` is an array type (9.3.4.5) and `remove_all_extents_t<T>` is an incomplete type, `ranges::data(E)` is ill-formed with no diagnostic required.
  - (2.3) — Otherwise, if `auto(t.data())` is a valid expression of pointer to object type, `ranges::data(E)` is expression-equivalent to `auto(t.data())`.
  - (2.4) — Otherwise, if `ranges::begin(t)` is a valid expression whose type models `contiguous_iterator`, `ranges::data(E)` is expression-equivalent to `to_address(ranges::begin(t))`.
  - (2.5) — Otherwise, `ranges::data(E)` is ill-formed.
- 3 [*Note 1:* Diagnosable ill-formed cases above result in substitution failure when `ranges::data(E)` appears in the immediate context of a template instantiation. — *end note*]
- 4 [*Note 2:* Whenever `ranges::data(E)` is a valid expression, it has pointer to object type. — *end note*]

### 25.3.15 `ranges::cdata`

[`range.prim.cdata`]

```
template<class T>
constexpr auto as-const-pointer(const T* p) noexcept { return p; } // exposition only
```

- 1 The name `ranges::cdata` denotes a customization point object (16.3.3.3.5). Given a subexpression `E` with type `T`, let `t` be an lvalue that denotes the reified object for `E`. Then:
  - (1.1) — If `E` is an rvalue and `enable_borrowed_range<remove_cv_t<T>>` is `false`, `ranges::cdata(E)` is ill-formed.
  - (1.2) — Otherwise, `ranges::cdata(E)` is expression-equivalent to `as-const-pointer(ranges::data(possibly-const-range(t)))`.
- 2 [*Note 1:* Whenever `ranges::cdata(E)` is a valid expression, it has pointer to constant object type. — *end note*]

## 25.4 Range requirements

[range.req]

### 25.4.1 General

[range.req.general]

- 1 Ranges are an abstraction that allows a C++ program to operate on elements of data structures uniformly. Calling `ranges::begin` on a range returns an object whose type models `input_or_output_iterator` (24.3.4.6). Calling `ranges::end` on a range returns an object whose type `S`, together with the type `I` of the object returned by `ranges::begin`, models `sentinel_for<S, I>`. The library formalizes the interfaces, semantics, and complexity of ranges to enable algorithms and range adaptors that work efficiently on different types of sequences.
- 2 The `range` concept requires that `ranges::begin` and `ranges::end` return an iterator and a sentinel, respectively. The `sized_range` concept refines `range` with the requirement that `ranges::size` be amortized  $\mathcal{O}(1)$ . The `view` concept specifies requirements on a `range` type to provide operations with predictable complexity.
- 3 Several refinements of `range` group requirements that arise frequently in concepts and algorithms. Common ranges are ranges for which `ranges::begin` and `ranges::end` return objects of the same type. Random access ranges are ranges for which `ranges::begin` returns a type that models `random_access_iterator` (24.3.4.13). (Contiguous, bidirectional, forward, input, and output ranges are defined similarly.) Viewable ranges can be converted to views.

### 25.4.2 Ranges

[range.range]

- 1 The `range` concept defines the requirements of a type that allows iteration over its elements by providing an iterator and sentinel that denote the elements of the range.

```
template<class T>
concept range =
 requires(T& t) {
 ranges::begin(t); // sometimes equality-preserving (see below)
 ranges::end(t);
 };

```

- 2 Given an expression `t` such that `decltype(t)` is `T&`, `T` models `range` only if
  - (2.1) — `[ranges::begin(t), ranges::end(t))` denotes a range (24.3.1),
  - (2.2) — both `ranges::begin(t)` and `ranges::end(t)` are amortized constant time and non-modifying, and
  - (2.3) — if the type of `ranges::begin(t)` models `forward_iterator`, `ranges::begin(t)` is equality-preserving.
- 3 [Note 1: Equality preservation of both `ranges::begin` and `ranges::end` enables passing a range whose iterator type models `forward_iterator` to multiple algorithms and making multiple passes over the range by repeated calls to `ranges::begin` and `ranges::end`. Since `ranges::begin` is not required to be equality-preserving when the return type does not model `forward_iterator`, it is possible for repeated calls to not return equal values or to not be well-defined. — end note]

```
template<class T>
concept borrowed_range =
 range<T> && (is_lvalue_reference_v<T> || enable_borrowed_range<remove_cvref_t<T>>);

```

- 4 Let `U` be `remove_reference_t<T>` if `T` is an rvalue reference type, and `T` otherwise. Given a variable `u` of type `U`, `T` models `borrowed_range` only if the validity of iterators obtained from `u` is not tied to the lifetime of that variable.
- 5 [Note 2: Since the validity of iterators is not tied to the lifetime of a variable whose type models `borrowed_range`, a function with a parameter of such a type can return iterators obtained from it without danger of dangling. — end note]

```
template<class>
constexpr bool enable_borrowed_range = false;

```

- 6 *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `enable_borrowed_range` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.
- 7 [Example 1: Each specialization `S` of class template `subrange` (25.5.4) models `borrowed_range` because
  - (7.1) — `enable_borrowed_range<S>` is specialized to have the value `true`, and

- (7.2) — S's iterators do not have validity tied to the lifetime of an S object because they are “borrowed” from some other range.  
 — *end example*]

### 25.4.3 Approximately sized ranges

[range.approximately.sized]

- <sup>1</sup> The `approximately_sized_range` concept refines `range` with the requirement that an approximation of the number of elements in the range can be determined in amortized constant time using `ranges::reserve_hint`.

```
template<class T>
concept approximately_sized_range =
 range<T> && requires(T& t) { ranges::reserve_hint(t); };
```

- <sup>2</sup> Given an lvalue `t` of type `remove_reference_t<T>`, `T` models `approximately_sized_range` only if
- (2.1) — `ranges::reserve_hint(t)` is amortized  $\mathcal{O}(1)$ , does not modify `t`, and has a value that is not negative and is representable in `range_difference_t<T>`, and
- (2.2) — if `iterator_t<T>` models `forward_iterator`, `ranges::reserve_hint(t)` is well-defined regardless of the evaluation of `ranges::begin(t)`.  
 [Note 1: `ranges::reserve_hint(t)` is otherwise not required to be well-defined after evaluating `ranges::begin(t)`. For example, it is possible for `ranges::reserve_hint(t)` to be well-defined for an `approximately_sized_range` whose iterator type does not model `forward_iterator` only if evaluated before the first call to `ranges::begin(t)`. — end note]

### 25.4.4 Sized ranges

[range.sized]

- <sup>1</sup> The `sized_range` concept refines `approximately_sized_range` with the requirement that the number of elements in the range can be determined in amortized constant time using `ranges::size`.

```
template<class T>
concept sized_range =
 approximately_sized_range<T> && requires(T& t) { ranges::size(t); };
```

- <sup>2</sup> Given an lvalue `t` of type `remove_reference_t<T>`, `T` models `sized_range` only if
- (2.1) — `ranges::size(t)` is amortized  $\mathcal{O}(1)$ , does not modify `t`, and is equal to `ranges::distance(ranges::begin(t), ranges::end(t))`, and
- (2.2) — if `iterator_t<T>` models `forward_iterator`, `ranges::size(t)` is well-defined regardless of the evaluation of `ranges::begin(t)`.  
 [Note 1: `ranges::size(t)` is otherwise not required to be well-defined after evaluating `ranges::begin(t)`. For example, it is possible for `ranges::size(t)` to be well-defined for a `sized_range` whose iterator type does not model `forward_iterator` only if evaluated before the first call to `ranges::begin(t)`. — end note]

```
template<class>
constexpr bool disable_sized_range = false;
```

- <sup>3</sup> *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `disable_sized_range` for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.
- <sup>4</sup> [Note 2: `disable_sized_range` allows use of `range` types with the library that satisfy but do not in fact model `sized_range`. — end note]

### 25.4.5 Views

[range.view]

- <sup>1</sup> The `view` concept specifies the requirements of a `range` type that has the semantic properties below, which make it suitable for use in constructing range adaptor pipelines (25.7).

```
template<class T>
concept view =
 range<T> && movable<T> && enable_view<T>;
```

- <sup>2</sup> `T` models `view` only if
- (2.1) — `T` has  $\mathcal{O}(1)$  move construction; and



- (2.2) — move assignment of an object of type `T` is no more complex than destruction followed by move construction; and
- (2.3) — if  $N$  copies and/or moves are made from an object of type `T` that contained  $M$  elements, then those  $N$  objects have  $\mathcal{O}(N + M)$  destruction; and
- (2.4) — `copy_constructible<T>` is `false`, or `T` has  $\mathcal{O}(1)$  copy construction; and
- (2.5) — `copyable<T>` is `false`, or copy assignment of an object of type `T` is no more complex than destruction followed by copy construction.

3 [Note 1: The constraints on copying and moving imply that a moved-from object of type `T` has  $\mathcal{O}(1)$  destruction. — end note]

4 [Example 1: Examples of views are:

- (4.1) — A `range` type that wraps a pair of iterators.
- (4.2) — A `range` type that holds its elements by `shared_ptr` and shares ownership with all its copies.
- (4.3) — A `range` type that generates its elements on demand.

A container such as `vector<string>` does not meet the semantic requirements of `view` since copying the container copies all of the elements, which cannot be done in constant time. — end example]

- 5 Since the difference between `range` and `view` is largely semantic, the two are differentiated with the help of `enable_view`.

```
template<class T>
constexpr bool is_derived_from_view_interface = see below; // exposition only
template<class T>
constexpr bool enable_view =
 derived_from<T, view_base> || is_derived_from_view_interface<T>;
```

- 6 For a type `T`, `is_derived_from_view_interface<T>` is `true` if and only if `T` has exactly one public base class `view_interface<U>` for some type `U` and `T` has no base classes of type `view_interface<V>` for any other type `V`.
- 7 *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `enable_view` to `true` for cv-unqualified program-defined types that model `view`, and `false` for types that do not. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.

## 25.4.6 Other range refinements

[range.refinements]

- 1 The `output_range` concept specifies requirements of a `range` type for which `ranges::begin` returns a model of `output_iterator` (24.3.4.10). `input_range`, `forward_range`, `bidirectional_range`, and `random_access_range` are defined similarly.

```
template<class R, class T>
concept output_range =
 range<R> && output_iterator<iterator_t<R>, T>;

template<class T>
concept input_range =
 range<T> && input_iterator<iterator_t<T>>;

template<class T>
concept forward_range =
 input_range<T> && forward_iterator<iterator_t<T>>;

template<class T>
concept bidirectional_range =
 forward_range<T> && bidirectional_iterator<iterator_t<T>>;

template<class T>
concept random_access_range =
 bidirectional_range<T> && random_access_iterator<iterator_t<T>>;
```

- 2 `contiguous_range` additionally requires that the `ranges::data` customization point object (25.3.14) is usable with the range.

```
template<class T>
concept contiguous_range =
 random_access_range<T> && contiguous_iterator<iterator_t<T>>> &&
 requires(T& t) {
 { ranges::data(t) } -> same_as<add_pointer_t<range_reference_t<T>>>;
 };

```

<sup>3</sup> Given an expression `t` such that `decltype((t))` is `T&`, `T` models `contiguous_range` only if `to_address(ranges::begin(t)) == ranges::data(t)` is true.

<sup>4</sup> The `common_range` concept specifies requirements of a `range` type for which `ranges::begin` and `ranges::end` return objects of the same type.

[Example 1: The standard containers (Clause 23) model `common_range`. — end example]

```
template<class T>
concept common_range =
 range<T> && same_as<iterator_t<T>, sentinel_t<T>>;

```

```
template<class R>
constexpr bool is-initializer-list = see below; // exposition only

```

<sup>5</sup> For a type `R`, `is-initializer-list<R>` is true if and only if `remove_cvref_t<R>` is a specialization of `initializer_list`.

<sup>6</sup> The `viewable_range` concept specifies the requirements of a `range` type that can be converted to a view safely.

```
template<class T>
concept viewable_range =
 range<T> &&
 ((view<remove_cvref_t<T>> && constructible_from<remove_cvref_t<T>, T>) ||
 (!view<remove_cvref_t<T>> &&
 (is_lvalue_reference_v<T> || (movable<remove_reference_t<T>> && !is-initializer-list<T>))));

```

<sup>7</sup> The `constant_range` concept specifies the requirements of a `range` type whose elements are not modifiable.

```
template<class T>
concept constant_range =
 input_range<T> && constant_iterator<iterator_t<T>>;

```

<sup>8</sup> The exposition-only concept `sized-random-access-range` specifies the requirements of a `range` type that is sized and allows random access to its elements.

```
template<class T>
concept sized-random-access-range = // exposition only
 random_access_range<T> && sized_range<T>;

```

[Note 1: This concept constrains some parallel algorithm overloads; see Clause 26. — end note]

## 25.5 Range utilities

[range.utility]

### 25.5.1 General

[range.utility.general]

<sup>1</sup> The components in 25.5 are general utilities for representing and manipulating ranges.

### 25.5.2 Helper concepts

[range.utility.helpers]

<sup>1</sup> Many of the types in 25.5 are specified in terms of the following exposition-only concepts:

```
template<class R>
concept simple-view = // exposition only
 view<R> && range<const R> &&
 same_as<iterator_t<R>, iterator_t<const R>> &&
 same_as<sentinel_t<R>, sentinel_t<const R>>;

template<class I>
concept has-arrow = // exposition only
 input_iterator<I> && (is_pointer_v<I> || requires(const I i) { i.operator->(); });

```



```

template<class T, class U>
concept different-from = // exposition only
 !same_as<remove_cvref_t<T>, remove_cvref_t<U>>;

template<class R>
concept range-with-movable-references = // exposition only
 input_range<R> && move_constructible<range_reference_t<R>> &&
 move_constructible<range_rvalue_reference_t<R>>;

```

### 25.5.3 View interface

[view.interface]

#### 25.5.3.1 General

[view.interface.general]

- <sup>1</sup> The class template `view_interface` is a helper for defining view-like types that offer a container-like interface. It is parameterized with the type that is derived from it.

```

namespace std::ranges {
 template<class D>
 requires is_class_v<D> && same_as<D, remove_cv_t<D>>
 class view_interface {
 private:
 constexpr D& derived() noexcept { // exposition only
 return static_cast<D&>(*this);
 }
 constexpr const D& derived() const noexcept { // exposition only
 return static_cast<const D&>(*this);
 }
 public:
 constexpr bool empty() requires sized_range<D> || forward_range<D> {
 if constexpr (sized_range<D>)
 return ranges::size(derived()) == 0;
 else
 return ranges::begin(derived()) == ranges::end(derived());
 }
 constexpr bool empty() const requires sized_range<const D> || forward_range<const D> {
 if constexpr (sized_range<const D>)
 return ranges::size(derived()) == 0;
 else
 return ranges::begin(derived()) == ranges::end(derived());
 }

 constexpr auto cbegin() requires input_range<D> {
 return ranges::cbegin(derived());
 }
 constexpr auto cbegin() const requires input_range<const D> {
 return ranges::cbegin(derived());
 }
 constexpr auto cend() requires input_range<D> {
 return ranges::cend(derived());
 }
 constexpr auto cend() const requires input_range<const D> {
 return ranges::cend(derived());
 }

 constexpr explicit operator bool()
 requires requires { ranges::empty(derived()); } {
 return !ranges::empty(derived());
 }
 constexpr explicit operator bool() const
 requires requires { ranges::empty(derived()); } {
 return !ranges::empty(derived());
 }
 }
 }

```

```

constexpr auto data() requires contiguous_iterator<iterator_t<D>> {
 return to_address(ranges::begin(derived()));
}
constexpr auto data() const
 requires range<const D> && contiguous_iterator<iterator_t<const D>> {
 return to_address(ranges::begin(derived()));
}

constexpr auto size() requires forward_range<D> &&
 sized_sentinel_for<sentinel_t<D>, iterator_t<D>> {
 return to_unsigned_like(ranges::end(derived()) - ranges::begin(derived()));
}
constexpr auto size() const requires forward_range<const D> &&
 sized_sentinel_for<sentinel_t<const D>, iterator_t<const D>> {
 return to_unsigned_like(ranges::end(derived()) - ranges::begin(derived()));
}

constexpr decltype(auto) front() requires forward_range<D>;
constexpr decltype(auto) front() const requires forward_range<const D>;

constexpr decltype(auto) back() requires bidirectional_range<D> && common_range<D>;
constexpr decltype(auto) back() const
 requires bidirectional_range<const D> && common_range<const D>;

template<random_access_range R = D>
 constexpr decltype(auto) operator[](range_difference_t<R> n) {
 return ranges::begin(derived())[n];
 }
template<random_access_range R = const D>
 constexpr decltype(auto) operator[](range_difference_t<R> n) const {
 return ranges::begin(derived())[n];
 }

template<random_access_range R = D> requires sized_range<R> // freestanding-deleted
 constexpr decltype(auto) at(range_difference_t<R> n);
template<random_access_range R = const D> requires sized_range<R> // freestanding-deleted
 constexpr decltype(auto) at(range_difference_t<R> n) const;
};
}

```

- <sup>2</sup> The template parameter `D` for `view_interface` may be an incomplete type. Before any member of the resulting specialization of `view_interface` other than special member functions is referenced, `D` shall be complete, and model both `derived_from<view_interface<D>>` and `view`.

### 25.5.3.2 Members

[view.interface.members]

```

constexpr decltype(auto) front() requires forward_range<D>;
constexpr decltype(auto) front() const requires forward_range<const D>;

```

- <sup>1</sup> *Hardened preconditions:* `!empty()` is true.

- <sup>2</sup> *Effects:* Equivalent to: `return *ranges::begin(derived());`

```

constexpr decltype(auto) back() requires bidirectional_range<D> && common_range<D>;
constexpr decltype(auto) back() const
 requires bidirectional_range<const D> && common_range<const D>;

```

- <sup>3</sup> *Hardened preconditions:* `!empty()` is true.

- <sup>4</sup> *Effects:* Equivalent to: `return *ranges::prev(ranges::end(derived()));`

```

template<random_access_range R = D> requires sized_range<R>
 constexpr decltype(auto) at(range_difference_t<R> n);
template<random_access_range R = const D> requires sized_range<R>
 constexpr decltype(auto) at(range_difference_t<R> n) const;

```

- <sup>5</sup> *Returns:* `(*this)[n]`.

<sup>6</sup> *Throws*: `out_of_range` if `n < 0` is true or `n >= ranges::distance(derived())` is true.

## 25.5.4 Sub-ranges

[range.subrange]

### 25.5.4.1 General

[range.subrange.general]

- <sup>1</sup> The `subrange` class template combines together an iterator and a sentinel into a single object that models the `view_concept`. Additionally, it models the `sized_range` concept when the final template parameter is `subrange_kind::sized`.

```
namespace std::ranges {
 template<class From, class To>
 concept uses-nonqualification-pointer-conversion = // exposition only
 is_pointer_v<From> && is_pointer_v<To> &&
 !convertible_to<remove_pointer_t<From>(*), remove_pointer_t<To>(*)[]>;

 template<class From, class To>
 concept convertible-to-non-slicing = // exposition only
 convertible_to<From, To> &&
 !uses-nonqualification-pointer-conversion<decay_t<From>, decay_t<To>>;

 template<class T, class U, class V>
 concept pair-like-convertible-from = // exposition only
 !range<T> && !is_reference_v<T> && pair-like<T> &&
 constructible_from<T, U, V> &&
 convertible-to-non-slicing<U, tuple_element_t<0, T>> &&
 convertible_to<V, tuple_element_t<1, T>>;

 template<input_or_output_iterator I, sentinel_for<I> S = I, subrange_kind K =
 sized_sentinel_for<S, I> ? subrange_kind::sized : subrange_kind::unsized>
 requires (K == subrange_kind::sized || !sized_sentinel_for<S, I>)
 class subrange : public view_interface<subrange<I, S, K>> {
 private:
 static constexpr bool StoreSize = // exposition only
 K == subrange_kind::sized && !sized_sentinel_for<S, I>;
 I begin_ = I(); // exposition only
 S end_ = S(); // exposition only
 make_unsigned_like_t<iter_difference_t<I>> size_ = 0; // exposition only; present only
 // if StoreSize is true
 public:
 subrange() requires default_initializable<I> = default;

 constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);

 constexpr subrange(convertible-to-non-slicing<I> auto i, S s,
 make_unsigned_like_t<iter_difference_t<I>> n)
 requires (K == subrange_kind::sized);

 template<different-from<subrange> R>
 requires borrowed_range<R> &&
 convertible-to-non-slicing<iterator_t<R>, I> &&
 convertible_to<sentinel_t<R>, S>
 constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);

 template<borrowed_range R>
 requires convertible-to-non-slicing<iterator_t<R>, I> &&
 convertible_to<sentinel_t<R>, S>
 constexpr subrange(R&& r, make_unsigned_like_t<iter_difference_t<I>> n)
 requires (K == subrange_kind::sized)
 : subrange{ranges::begin(r), ranges::end(r), n} {};

 template<different-from<subrange> PairLike>
 requires pair-like-convertible-from<PairLike, const I&, const S&>
 constexpr operator PairLike() const;
 };
}
```

```

constexpr I begin() const requires copyable<I>;
constexpr I begin() requires (!copyable<I>);
constexpr S end() const;

constexpr bool empty() const;
constexpr make-unsigned-like-t<iter_difference_t<I>> size() const
 requires (K == subrange_kind::sized);

constexpr subrange next(iter_difference_t<I> n = 1) const &
 requires forward_iterator<I>;
constexpr subrange next(iter_difference_t<I> n = 1) &&;
constexpr subrange prev(iter_difference_t<I> n = 1) const
 requires bidirectional_iterator<I>;
constexpr subrange&& advance(iter_difference_t<I> n);
};

template<input_or_output_iterator I, sentinel_for<I> S>
 subrange(I, S) -> subrange<I, S>;

template<input_or_output_iterator I, sentinel_for<I> S>
 subrange(I, S, make-unsigned-like-t<iter_difference_t<I>>) ->
 subrange<I, S, subrange_kind::sized>;

template<borrowed_range R>
 subrange(R&&) ->
 subrange<iterator_t<R>, sentinel_t<R>,
 (sized_range<R> || sized_sentinel_for<sentinel_t<R>, iterator_t<R>>)
 ? subrange_kind::sized : subrange_kind::unsized>;

template<borrowed_range R>
 subrange(R&&, make-unsigned-like-t<range_difference_t<R>>) ->
 subrange<iterator_t<R>, sentinel_t<R>, subrange_kind::sized>;
}

```

#### 25.5.4.2 Constructors and conversions

[range.subrange.ctor]

```
constexpr subrange(convertible-to-non-slicing<I> auto i, S s) requires (!StoreSize);
```

1     *Preconditions:* [i, s) is a valid range.

2     *Effects:* Initializes *begin\_* with `std::move(i)` and *end\_* with s.

```
constexpr subrange(convertible-to-non-slicing<I> auto i, S s,
 make-unsigned-like-t<iter_difference_t<I>> n)
 requires (K == subrange_kind::sized);
```

3     *Preconditions:* [i, s) is a valid range, and `n == to-unsigned-like(ranges::distance(i, s))` is true.

4     *Effects:* Initializes *begin\_* with `std::move(i)` and *end\_* with s. If *StoreSize* is true, initializes *size\_* with n.

5     [Note 1: Accepting the length of the range and storing it to later return from `size()` enables `subrange` to model `sized_range` even when it stores an iterator and sentinel that do not model `sized_sentinel_for`. — end note]

```
template<different-from<subrange> R>
 requires borrowed_range<R> &&
 convertible-to-non-slicing<iterator_t<R>, I> &&
 convertible_to<sentinel_t<R>, S>
constexpr subrange(R&& r) requires (!StoreSize || sized_range<R>);
```

6     *Effects:* Equivalent to:

(6.1)     — If *StoreSize* is true, `subrange(r, static_cast<decltype(size_)>(ranges::size(r)))`.

(6.2)     — Otherwise, `subrange(ranges::begin(r), ranges::end(r))`.

```
template<different-from<subrange> PairLike>
requires pair-like-convertible-from<PairLike, const I&, const S&>
constexpr operator PairLike() const;
```

7 *Effects:* Equivalent to: return PairLike(begin\_, end\_);

### 25.5.4.3 Accessors

[range.subrange.access]

```
constexpr I begin() const requires copyable<I>;
```

1 *Effects:* Equivalent to: return begin\_;

```
constexpr I begin() requires (!copyable<I>;
```

2 *Effects:* Equivalent to: return std::move(begin\_);

```
constexpr S end() const;
```

3 *Effects:* Equivalent to: return end\_;

```
constexpr bool empty() const;
```

4 *Effects:* Equivalent to: return begin\_ == end\_;

```
constexpr make_unsigned_like_t<iter_difference_t<I>> size() const
requires (K == subrange_kind::sized);
```

5 *Effects:*

(5.1) — If *StoreSize* is true, equivalent to: return size\_;

(5.2) — Otherwise, equivalent to: return to\_unsigned\_like(end\_ - begin\_);

```
constexpr subrange next(iter_difference_t<I> n = 1) const &
requires forward_iterator<I>;
```

6 *Effects:* Equivalent to:

```
auto tmp = *this;
tmp.advance(n);
return tmp;
```

```
constexpr subrange next(iter_difference_t<I> n = 1) &&;
```

7 *Effects:* Equivalent to:

```
advance(n);
return std::move(*this);
```

```
constexpr subrange prev(iter_difference_t<I> n = 1) const
requires bidirectional_iterator<I>;
```

8 *Effects:* Equivalent to:

```
auto tmp = *this;
tmp.advance(-n);
return tmp;
```

```
constexpr subrange& advance(iter_difference_t<I> n);
```

9 *Effects:* Equivalent to:

```
if constexpr (bidirectional_iterator<I>) {
 if (n < 0) {
 ranges::advance(begin_, n);
 if constexpr (StoreSize)
 size_ += to_unsigned_like(-n);
 return *this;
 }
}
```

```
auto d = n - ranges::advance(begin_, n, end_);
if constexpr (StoreSize)
 size_ -= to_unsigned_like(d);
return *this;
```

```
template<size_t N, class I, class S, subrange_kind K>
requires ((N == 0 && copyable<I>) || N == 1)
constexpr auto get(const subrange<I, S, K>& r);
template<size_t N, class I, class S, subrange_kind K>
requires (N < 2)
constexpr auto get(subrange<I, S, K>&& r);
```

10 *Effects:* Equivalent to:

```
if constexpr (N == 0)
 return r.begin();
else
 return r.end();
```

### 25.5.5 Dangling iterator handling

[range.dangling]

- <sup>1</sup> The type **dangling** is used together with the template aliases **borrowed\_iterator\_t** and **borrowed\_subrange\_t**. When an algorithm that typically returns an iterator into, or a subrange of, a range argument is called with an rvalue range argument that does not model **borrowed\_range** (25.4.2), the return value possibly refers to a range whose lifetime has ended. In such cases, the type **dangling** is returned instead of an iterator or subrange.

```
namespace std::ranges {
 struct dangling {
 constexpr dangling() noexcept = default;
 constexpr dangling(auto&&...) noexcept {}
 };
}
```

<sup>2</sup> [Example 1:

```
vector<int> f();
auto result1 = ranges::find(f(), 42); // #1
static_assert(same_as<decltype(result1), ranges::dangling>);
auto vec = f();
auto result2 = ranges::find(vec, 42); // #2
static_assert(same_as<decltype(result2), vector<int>::iterator>);
auto result3 = ranges::find(ranges::subrange{vec}, 42); // #3
static_assert(same_as<decltype(result3), vector<int>::iterator>);
```

The call to `ranges::find` at #1 returns `ranges::dangling` since `f()` is an rvalue `vector`; it is possible for the `vector` to be destroyed before a returned iterator is dereferenced. However, the calls at #2 and #3 both return iterators since the lvalue `vec` and specializations of `subrange` model **borrowed\_range**. — end example]

<sup>3</sup> For a type `R` that models **range**:

- (3.1) — if `R` models **borrowed\_range**, then `borrowed_iterator_t<R>` denotes `iterator_t<R>`, and `borrowed_subrange_t<R>` denotes `subrange<iterator_t<R>>`;
- (3.2) — otherwise, both `borrowed_iterator_t<R>` and `borrowed_subrange_t<R>` denote **dangling**.

### 25.5.6 Class template `elements_of`

[range.elementsof]

Specializations of `elements_of` encapsulate a range and act as a tag in overload sets to disambiguate when a range should be treated as a sequence rather than a single value.

[Example 1:

```
template<bool YieldElements>
generator<any> f(ranges::input_range auto&& r) {
 if constexpr (YieldElements)
 co_yield ranges::elements_of(r); // yield each element of r
 else
 co_yield r; // yield r as a single value
}
```

— end example]

```
namespace std::ranges {
 template<range R, class Allocator = allocator<byte>>
 struct elements_of {
 [[no_unique_address]] R range;
```

```

[[no_unique_address]] Allocator allocator = Allocator();
};

template<class R, class Allocator = allocator<byte>>
 elements_of(R&&, Allocator = Allocator()) -> elements_of<R&&, Allocator>;
}

```

## 25.5.7 Range conversions

[range.utility.conv]

### 25.5.7.1 General

[range.utility.conv.general]

- <sup>1</sup> The range conversion functions construct an object (usually a container) from a range, by using a constructor taking a range, a `from_range_t` tagged constructor, or a constructor taking a pair of iterators, or by inserting each element of the range into the default-constructed object.
- <sup>2</sup> `ranges::to` is applied recursively, allowing the conversion of a range of ranges.

[Example 1:

```

string_view str = "the quick brown fox";
auto words = views::split(str, ' ') | to<vector<string>>();
// words is vector<string>{"the", "quick", "brown", "fox"}

```

— end example]

- <sup>3</sup> Let *reservable-container* be defined as follows:

```

template<class Container>
constexpr bool reservable-container = // exposition only
 sized_range<Container> &&
 requires(Container& c, range_size_t<Container> n) {
 c.reserve(n);
 { c.capacity() } -> same_as<decltype(n)>;
 { c.max_size() } -> same_as<decltype(n)>;
 };

```

- <sup>4</sup> Let *container-appendable* be defined as follows:

```

template<class Container, class Ref>
constexpr bool container-appendable = // exposition only
 requires(Container& c, Ref&& ref) {
 requires (requires { c.emplace_back(std::forward<Ref>(ref)); } ||
 requires { c.push_back(std::forward<Ref>(ref)); } ||
 requires { c.emplace_hint(c.end(), std::forward<Ref>(ref)); } ||
 requires { c.insert(c.end(), std::forward<Ref>(ref)); });
 };

```

- <sup>5</sup> Let *container-append* be defined as follows:

```

template<class Container>
constexpr auto container-append(Container& c) { // exposition only
 return [&c]<class Ref>(Ref&& ref) {
 if constexpr (requires { c.emplace_back(declval<Ref>()); })
 c.emplace_back(std::forward<Ref>(ref));
 else if constexpr (requires { c.push_back(declval<Ref>()); })
 c.push_back(std::forward<Ref>(ref));
 else if constexpr (requires { c.emplace_hint(c.end(), declval<Ref>()); })
 c.emplace_hint(c.end(), std::forward<Ref>(ref));
 else
 c.insert(c.end(), std::forward<Ref>(ref));
 };
}

```

### 25.5.7.2 ranges::to

[range.utility.conv.to]

```

template<class C, input_range R, class... Args> requires (!view<C>)
constexpr C to(R&& r, Args&&... args);

```

- <sup>1</sup> *Mandates:* C is a cv-unqualified class type.
- <sup>2</sup> *Returns:* An object of type C constructed from the elements of r in the following manner:

- (2.1) — If `C` does not satisfy `input_range` or `convertible_to<range_reference_t<R>, range_value_t<C>>` is true:
  - (2.1.1) — If `constructible_from<C, R, Args...>` is true:
    - `C(std::forward<R>(r), std::forward<Args>(args)...)()`
  - (2.1.2) — Otherwise, if `constructible_from<C, from_range_t, R, Args...>` is true:
    - `C(from_range, std::forward<R>(r), std::forward<Args>(args)...)()`
  - (2.1.3) — Otherwise, if
    - (2.1.3.1) — `common_range<R>` is true,
    - (2.1.3.2) — the *qualified-id* `iterator_traits<iterator_t<R>>::iterator_category` is valid and denotes a type that models `derived_from<input_iterator_tag>`, and
    - (2.1.3.3) — `constructible_from<C, iterator_t<R>, sentinel_t<R>, Args...>` is true:
      - `C(ranges::begin(r), ranges::end(r), std::forward<Args>(args)...)()`
  - (2.1.4) — Otherwise, if
    - (2.1.4.1) — `constructible_from<C, Args...>` is true, and
    - (2.1.4.2) — `container-appendable<C, range_reference_t<R>>` is true:
      - `C c(std::forward<Args>(args)...)();`
      - if constexpr (approximately\_sized\_range<R> && reservable-container<C>)
      - `c.reserve(static_cast<range_size_t<C>>(ranges::reserve_hint(r)));`
      - `ranges::for_each(r, container-append(c));`
  - (2.1.5) — Otherwise, the program is ill-formed.
- (2.2) — Otherwise, if `input_range<range_reference_t<R>>` is true:
  - `to<C>(ref_view(r) | views::transform([](auto&& elem) {`
  - `return to<range_value_t<C>>(std::forward<decltype(elem)>(elem));`
  - `}), std::forward<Args>(args)...)();`
- (2.3) — Otherwise, the program is ill-formed.

```
template<template<class...> class C, input_range R, class... Args>
constexpr auto to(R&& r, Args&&... args);
```

3 Let *input-iterator* be an exposition-only type:

```
struct input-iterator { // exposition only
 using iterator_category = input_iterator_tag;
 using value_type = range_value_t<R>;
 using difference_type = ptrdiff_t;
 using pointer = add_pointer_t<range_reference_t<R>>;
 using reference = range_reference_t<R>;
 reference operator*() const;
 pointer operator->() const;
 input-iterator& operator++();
 input-iterator operator++(int);
 bool operator==(const input-iterator&) const;
};
```

[Note 1: *input-iterator* meets the syntactic requirements of *Cpp17InputIterator*. — end note]

4 Let *DEDUCE\_EXPR* be defined as follows:

- (4.1) — `C(declval<R>(), declval<Args>()...)` if that is a valid expression,
- (4.2) — otherwise, `C(from_range, declval<R>(), declval<Args>()...)` if that is a valid expression,
- (4.3) — otherwise,
  - `C(declval<input-iterator>(), declval<input-iterator>(), declval<Args>()...)`
  - if that is a valid expression,
- (4.4) — otherwise, the program is ill-formed.

5 *Returns:* `to<decltype(DEDUCE_EXPR)>(std::forward<R>(r), std::forward<Args>(args)...)()`.



**25.5.7.3 ranges::to\_adaptors**

[range.utility.conv.adaptors]

```
template<class C, class... Args> requires (!view<C>)
constexpr auto to(Args&&... args);
template<template<class...> class C, class... Args>
constexpr auto to(Args&&... args);
```

- <sup>1</sup> *Mandates:* For the first overload, *C* is a cv-unqualified class type.
- <sup>2</sup> *Returns:* A range adaptor closure object (25.7.2) *f* that is a perfect forwarding call wrapper (22.10.4) with the following properties:
- (2.1) — It has no target object.
  - (2.2) — Its bound argument entities `bound_args` consist of objects of types `decay_t<Args>...` direct-non-list-initialized with `std::forward<Args>(args)...`, respectively.
  - (2.3) — Its call pattern is `to<C>(r, bound_args...)`, where *r* is the argument used in a function call expression of *f*.

**25.6 Range factories**

[range.factories]

**25.6.1 General**

[range.factories.general]

- <sup>1</sup> Subclause 25.6 defines *range factories*, which are utilities to create a view.
- <sup>2</sup> Range factories are declared in namespace `std::ranges::views`.

**25.6.2 Empty view**

[range.empty]

**25.6.2.1 Overview**

[range.empty.overview]

- <sup>1</sup> `empty_view` produces a view of no elements of a particular type.
- <sup>2</sup> [Example 1:

```
auto e = views::empty<int>;
static_assert(ranges::empty(e));
static_assert(0 == e.size());
```

— end example]

**25.6.2.2 Class template empty\_view**

[range.empty.view]

```
namespace std::ranges {
 template<class T>
 requires is_object_v<T>
 class empty_view : public view_interface<empty_view<T>> {
 public:
 static constexpr T* begin() noexcept { return nullptr; }
 static constexpr T* end() noexcept { return nullptr; }
 static constexpr T* data() noexcept { return nullptr; }
 static constexpr size_t size() noexcept { return 0; }
 static constexpr bool empty() noexcept { return true; }
 };
}
```

**25.6.3 Single view**

[range.single]

**25.6.3.1 Overview**

[range.single.overview]

- <sup>1</sup> `single_view` produces a view that contains exactly one element of a specified value.
- <sup>2</sup> The name `views::single` denotes a customization point object (16.3.3.3.5). Given a subexpression *E*, the expression `views::single(E)` is expression-equivalent to `single_view<decay_t<decltype(E)>>(E)`.
- <sup>3</sup> [Example 1:
- ```
for (int i : views::single(4))
    cout << i;           // prints 4
```
- end example]

25.6.3.2 Class template `single_view`

[range.single.view]

```

namespace std::ranges {
    template<move_constructible T>
        requires is_object_v<T>
        class single_view : public view_interface<single_view<T>> {
        private:
            movable_box<T> value_;                // exposition only (see 25.7.3)

        public:
            single_view() requires default_initializable<T> = default;
            constexpr explicit single_view(const T& t) requires copy_constructible<T>;
            constexpr explicit single_view(T&& t);
            template<class... Args>
                requires constructible_from<T, Args...>
            constexpr explicit single_view(in_place_t, Args&&... args);

            constexpr T* begin() noexcept;
            constexpr const T* begin() const noexcept;
            constexpr T* end() noexcept;
            constexpr const T* end() const noexcept;
            static constexpr bool empty() noexcept;
            static constexpr size_t size() noexcept;
            constexpr T* data() noexcept;
            constexpr const T* data() const noexcept;
        };

        template<class T>
            single_view(T) -> single_view<T>;
    }

```

```
constexpr explicit single_view(const T& t) requires copy_constructible<T>;
```

1 *Effects:* Initializes *value_* with *t*.

```
constexpr explicit single_view(T&& t);
```

2 *Effects:* Initializes *value_* with `std::move(t)`.

```

template<class... Args>
    requires constructible_from<T, Args...>
constexpr explicit single_view(in_place_t, Args&&... args);

```

3 *Effects:* Initializes *value_* as if by `value_{in_place, std::forward<Args>(args)...}`.

```

constexpr T* begin() noexcept;
constexpr const T* begin() const noexcept;

```

4 *Effects:* Equivalent to: `return data();`

```

constexpr T* end() noexcept;
constexpr const T* end() const noexcept;

```

5 *Effects:* Equivalent to: `return data() + 1;`

```
static constexpr bool empty() noexcept;
```

6 *Effects:* Equivalent to: `return false;`

```
static constexpr size_t size() noexcept;
```

7 *Effects:* Equivalent to: `return 1;`

```

constexpr T* data() noexcept;
constexpr const T* data() const noexcept;

```

8 *Effects:* Equivalent to: `return value_.operator->();`

25.6.4 Iota view

[range.iota]

25.6.4.1 Overview

[range.iota.overview]

- ¹ `iota_view` generates a sequence of elements by repeatedly incrementing an initial value.
- ² The name `views::iota` denotes a customization point object (16.3.3.3.5). Given subexpressions `E` and `F`, the expressions `views::iota(E)` and `views::iota(E, F)` are expression-equivalent to `iota_view<decay_t<decltype((E))>>(E)` and `iota_view(E, F)`, respectively.

³ [Example 1:

```
for (int i : views::iota(1, 10))
    cout << i << ' '; // prints 1 2 3 4 5 6 7 8 9
```

— end example]

- ⁴ The name `views::indices` denotes a customization point object (16.3.3.3.5). Given subexpression `E`, let `T` be `remove_cvref_t<decltype((E))>`. `views::indices(E)` is expression-equivalent to `views::iota(T(0), E)` if *is-integer-like*`<T>` is true, and ill-formed otherwise.

25.6.4.2 Class template `iota_view`

[range.iota.view]

```
namespace std::ranges {
    template<class I>
        concept decrementable = see below; // exposition only

    template<class I>
        concept advanceable = see below; // exposition only

    template<weakly_incrementable W, semiregular Bound = unreachable_sentinel_t>
        requires weakly-equality-comparable-with<W, Bound> && copyable<W>
        class iota_view : public view_interface<iota_view<W, Bound>> {
        private:
            // 25.6.4.3, class iota_view::iterator
            struct iterator; // exposition only

            // 25.6.4.4, class iota_view::sentinel
            struct sentinel; // exposition only

            W value_ = W(); // exposition only
            Bound bound_ = Bound(); // exposition only

        public:
            iota_view() requires default_initializable<W> = default;
            constexpr explicit iota_view(W value);
            constexpr explicit iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
            constexpr explicit iota_view(iterator first, see below last);

            constexpr iterator begin() const;
            constexpr auto end() const;
            constexpr iterator end() const requires same_as<W, Bound>;

            constexpr bool empty() const;
            constexpr auto size() const requires see below;
        };

    template<class W, class Bound>
        requires (!is-integer-like<W> || !is-integer-like<Bound> ||
            (is-signed-integer-like<W> == is-signed-integer-like<Bound>))
        iota_view(W, Bound) -> iota_view<W, Bound>;
}
```

¹ Let *IOTA-DIFF-T*(`W`) be defined as follows:

- (1.1) — If `W` is not an integral type, or if it is an integral type and `sizeof(iter_difference_t<W>)` is greater than `sizeof(W)`, then *IOTA-DIFF-T*(`W`) denotes `iter_difference_t<W>`.
- (1.2) — Otherwise, *IOTA-DIFF-T*(`W`) is a signed integer type of width greater than the width of `W` if such a type exists.

- (1.3) — Otherwise, *IOTA-DIFF-T(W)* is an unspecified signed-integer-like type (24.3.4.4) of width not less than the width of *W*.

[Note 1: It is unspecified whether this type satisfies `weakly_incrementable`. — end note]

- 2 The exposition-only *decrementable* concept is equivalent to:

```
template<class I>
concept decrementable =           // exposition only
    incrementable<I> && requires(I i) {
        { --i } -> same_as<I&&>;
        { i-- } -> same_as<I>;
    };

```

- 3 When an object is in the domain of both pre- and post-decrement, the object is said to be *decrementable*.

- 4 Let *a* and *b* be equal objects of type *I*. *I* models *decrementable* only if

- (4.1) — If *a* and *b* are decrementable, then the following are all true:

(4.1.1) — `addressof(--a) == addressof(a)`

(4.1.2) — `bool(a-- == b)`

(4.1.3) — `bool(((void)a--, a) == --b)`

(4.1.4) — `bool(++(--a) == b)`.

- (4.2) — If *a* and *b* are incrementable, then `bool(--(++a) == b)`.

- 5 The exposition-only *advanceable* concept is equivalent to:

```
template<class I>
concept advanceable =           // exposition only
    decrementable<I> && totally_ordered<I> &&
    requires(I i, const I j, const IOTA-DIFF-T(I) n) {
        { i += n } -> same_as<I&&>;
        { i -= n } -> same_as<I&&>;
        I(j + n);
        I(n + j);
        I(j - n);
        { j - j } -> convertible_to<IOTA-DIFF-T(I)>;
    };

```

Let *D* be *IOTA-DIFF-T(I)*. Let *a* and *b* be objects of type *I* such that *b* is reachable from *a* after *n* applications of `++a`, for some value *n* of type *D*. *I* models *advanceable* only if

- (5.1) — `(a += n)` is equal to *b*.

- (5.2) — `addressof(a += n)` is equal to `addressof(a)`.

- (5.3) — `I(a + n)` is equal to `(a += n)`.

- (5.4) — For any two positive values *x* and *y* of type *D*, if `I(a + D(x + y))` is well-defined, then `I(a + D(x + y))` is equal to `I(I(a + x) + y)`.

- (5.5) — `I(a + D(0))` is equal to *a*.

- (5.6) — If `I(a + D(n - 1))` is well-defined, then `I(a + n)` is equal to `[] (I c) { return ++c; }(I(a + D(n - 1)))`.

- (5.7) — `(b += -n)` is equal to *a*.

- (5.8) — `(b -= n)` is equal to *a*.

- (5.9) — `addressof(b -= n)` is equal to `addressof(b)`.

- (5.10) — `I(b - n)` is equal to `(b -= n)`.

- (5.11) — `D(b - a)` is equal to *n*.

- (5.12) — `D(a - b)` is equal to `D(-n)`.

- (5.13) — `bool(a <= b)` is true.

```
constexpr explicit iota_view(W value);
```

6 *Preconditions:* Bound denotes `unreachable_sentinel_t` or `Bound()` is reachable from `value`. When `W` and `Bound` model `totally_ordered_with`, then `bool(value <= Bound())` is true.

7 *Effects:* Initializes `value_` with `value`.

```
constexpr explicit iota_view(type_identity_t<W> value, type_identity_t<Bound> bound);
```

8 *Preconditions:* Bound denotes `unreachable_sentinel_t` or `bound` is reachable from `value`. When `W` and `Bound` model `totally_ordered_with`, then `bool(value <= bound)` is true.

9 *Effects:* Initializes `value_` with `value` and `bound_` with `bound`.

```
constexpr explicit iota_view(iterator first, see below last);
```

10 *Effects:* Equivalent to:

- (10.1) — If `same_as<W, Bound>` is true, `iota_view(first.value_, last.value_)`.
- (10.2) — Otherwise, if `Bound` denotes `unreachable_sentinel_t`, `iota_view(first.value_, last)`.
- (10.3) — Otherwise, `iota_view(first.value_, last.bound_)`.

11 *Remarks:* The type of `last` is:

- (11.1) — If `same_as<W, Bound>` is true, `iterator`.
- (11.2) — Otherwise, if `Bound` denotes `unreachable_sentinel_t`, `Bound`.
- (11.3) — Otherwise, `sentinel`.

```
constexpr iterator begin() const;
```

12 *Effects:* Equivalent to: `return iterator{value_};`

```
constexpr auto end() const;
```

13 *Effects:* Equivalent to:

```
if constexpr (same_as<Bound, unreachable_sentinel_t>)
    return unreachable_sentinel;
else
    return sentinel{bound_};
```

```
constexpr iterator end() const requires same_as<W, Bound>;
```

14 *Effects:* Equivalent to: `return iterator{bound_};`

```
constexpr bool empty() const;
```

15 *Effects:* Equivalent to: `return value_ == bound_;`

```
constexpr auto size() const requires see below;
```

16 *Effects:* Equivalent to:

```
if constexpr (is-integer-like<W> && is-integer-like<Bound>)
    return (value_ < 0)
        ? ((bound_ < 0)
            ? to-unsigned-like(-value_) - to-unsigned-like(-bound_)
            : to-unsigned-like(bound_) + to-unsigned-like(-value_))
        : to-unsigned-like(bound_) - to-unsigned-like(value_);
else
    return to-unsigned-like(bound_ - value_);
```

17 *Remarks:* The expression in the *requires-clause* is equivalent to:

```
(same_as<W, Bound> && advanceable<W>) || (is-integer-like<W> && is-integer-like<Bound>) ||
sized_sentinel_for<Bound, W>
```

25.6.4.3 Class `iota_view::iterator`

[range.iota.iterator]

```
namespace std::ranges {
    template<weakly_incrementable W, semiregular Bound>
        requires weakly-equality-comparable-with<W, Bound> && copyable<W>
        struct iota_view<W, Bound>::iterator {
```

```

private:
    W value_ = W();                                // exposition only
    constexpr explicit iterator(W value);           // exposition only

public:
    using iterator_concept = see below;
    using iterator_category = input_iterator_tag;   // present only if W models incrementable and
                                                    // IOTA-DIFF-T(W) is an integral type

    using value_type = W;
    using difference_type = IOTA-DIFF-T(W);

    iterator() requires default_initializable<W> = default;

    constexpr W operator*() const noexcept(is_nothrow_copy_constructible_v<W>);

    constexpr iterator& operator++();
    constexpr void operator++(int);
    iterator operator++(int) requires incrementable<W> = default;

    constexpr iterator& operator--() requires decrementable<W>;
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type n)
        requires advanceable<W>;
    constexpr iterator& operator-=(difference_type n)
        requires advanceable<W>;
    constexpr W operator[](difference_type n) const
        requires advanceable<W>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<W>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires totally_ordered<W>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires totally_ordered<W>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires totally_ordered<W>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires totally_ordered<W>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires totally_ordered<W> && three_way_comparable<W>;

    friend constexpr iterator operator+(iterator i, difference_type n)
        requires advanceable<W>;
    friend constexpr iterator operator+(difference_type n, iterator i)
        requires advanceable<W>;

    friend constexpr iterator operator-(iterator i, difference_type n)
        requires advanceable<W>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
        requires advanceable<W>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *W* models *advanceable*, then *iterator_concept* is *random_access_iterator_tag*.
- (1.2) — Otherwise, if *W* models *decrementable*, then *iterator_concept* is *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *W* models *incrementable*, then *iterator_concept* is *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* is *input_iterator_tag*.

² [Note 1: Overloads for *iter_move* and *iter_swap* are omitted intentionally. — end note]

```

constexpr explicit iterator(W value);
3     Effects: Initializes value_ with value.

constexpr W operator*() const noexcept(is_nothrow_copy_constructible_v<W>);
4     Effects: Equivalent to: return value_;
5     [Note 2: The noexcept clause is needed by the default iter_move implementation. — end note]

constexpr iterator& operator++();
6     Effects: Equivalent to:
        ++value_;
        return *this;

constexpr void operator++(int);
7     Effects: Equivalent to ++*this.

constexpr iterator& operator--() requires decrementable<W>;
8     Effects: Equivalent to:
        --value_;
        return *this;

constexpr iterator& operator+=(difference_type n)
requires advanceable<W>;
9     Effects: Equivalent to:
        if constexpr (is-integer-like<W> && !is-signed-integer-like<W>) {
            if (n >= difference_type(0))
                value_ += static_cast<W>(n);
            else
                value_ -= static_cast<W>(-n);
        } else {
            value_ += n;
        }
        return *this;

constexpr iterator& operator-=(difference_type n)
requires advanceable<W>;
10    Effects: Equivalent to:
        if constexpr (is-integer-like<W> && !is-signed-integer-like<W>) {
            if (n >= difference_type(0))
                value_ -= static_cast<W>(n);
            else
                value_ += static_cast<W>(-n);
        } else {
            value_ -= n;
        }
        return *this;

constexpr W operator[](difference_type n) const
requires advanceable<W>;
11    Effects: Equivalent to: return W(value_ + n);

friend constexpr bool operator==(const iterator& x, const iterator& y)
requires equality_comparable<W>;
12    Effects: Equivalent to: return x.value_ == y.value_;

friend constexpr bool operator<(const iterator& x, const iterator& y)
requires totally_ordered<W>;
13    Effects: Equivalent to: return x.value_ < y.value_;

```

```

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires totally_ordered<W>;
14     Effects: Equivalent to: return y < x;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires totally_ordered<W>;
15     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires totally_ordered<W>;
16     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires totally_ordered<W> && three_way_comparable<W>;
17     Effects: Equivalent to: return x.value_ <=> y.value_;

friend constexpr iterator operator+(iterator i, difference_type n)
    requires advanceable<W>;
18     Effects: Equivalent to:
        i += n;
        return i;

friend constexpr iterator operator+(difference_type n, iterator i)
    requires advanceable<W>;
19     Effects: Equivalent to: return i + n;

friend constexpr iterator operator-(iterator i, difference_type n)
    requires advanceable<W>;
20     Effects: Equivalent to:
        i -= n;
        return i;

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires advanceable<W>;
21     Effects: Equivalent to:
        using D = difference_type;
        if constexpr (is-integer-like<W>) {
            if constexpr (is-signed-integer-like<W>)
                return D(D(x.value_) - D(y.value_));
            else
                return (y.value_ > x.value_)
                    ? D(-D(y.value_ - x.value_))
                    : D(x.value_ - y.value_);
        } else {
            return x.value_ - y.value_;
        }

```

25.6.4.4 Class `iota_view::sentinel`

[range.iota.sentinel]

```

namespace std::ranges {
    template<weakly_incrementable W, semiregular Bound>
        requires weakly_equality_comparable_with<W, Bound> && copyable<W>
        struct iota_view<W, Bound>::sentinel {
        private:
            Bound bound_ = Bound(); // exposition only
            constexpr explicit sentinel(Bound bound); // exposition only

        public:
            sentinel() = default;

            friend constexpr bool operator==(const iterator& x, const sentinel& y);

```



```

    friend constexpr iter_difference_t<W> operator-(const iterator& x, const sentinel& y)
        requires sized_sentinel_for<Bound, W>;
    friend constexpr iter_difference_t<W> operator-(const sentinel& x, const iterator& y)
        requires sized_sentinel_for<Bound, W>;
};
}

```

```
constexpr explicit sentinel(Bound bound);
```

¹ *Effects:* Initializes *bound_* with *bound*.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

² *Effects:* Equivalent to: return *x.value_* == *y.bound_*;

```
friend constexpr iter_difference_t<W> operator-(const iterator& x, const sentinel& y)
    requires sized_sentinel_for<Bound, W>;
```

³ *Effects:* Equivalent to: return *x.value_* - *y.bound_*;

```
friend constexpr iter_difference_t<W> operator-(const sentinel& x, const iterator& y)
    requires sized_sentinel_for<Bound, W>;
```

⁴ *Effects:* Equivalent to: return -(*y* - *x*);

25.6.5 Repeat view

[range.repeat]

25.6.5.1 Overview

[range.repeat.overview]

¹ `repeat_view` generates a sequence of elements by repeatedly producing the same value.

² The name `views::repeat` denotes a customization point object (16.3.3.3.5). Given subexpressions *E* and *F*, the expressions `views::repeat(E)` and `views::repeat(E, F)` are expression-equivalent to `repeat_view<decay_t<decltype((E))>>(E)` and `repeat_view(E, F)`, respectively.

³ [Example 1:

```

for (int i : views::repeat(17, 4))
    cout << i << ' ';
// prints 17 17 17 17

```

— end example]

25.6.5.2 Class template `repeat_view`

[range.repeat.view]

```

namespace std::ranges {
    template<class T>
        concept integer-like-with-usable-difference-type = // exposition only
            is-signed-integer-like<T> || (is-integer-like<T> && weakly_incrementable<T>);

    template<move_constructible T, semiregular Bound = unreachable_sentinel_t>
        requires (is_object_v<T> && same_as<T, remove_cv_t<T>> &&
            (integer-like-with-usable-difference-type<Bound> ||
            same_as<Bound, unreachable_sentinel_t>))
        class repeat_view : public view_interface<repeat_view<T, Bound>> {
        private:
            // 25.6.5.3, class repeat_view::iterator
            struct iterator; // exposition only

            movable_box<T> value_; // exposition only, see 25.7.3
            Bound bound_ = Bound(); // exposition only

        public:
            repeat_view() requires default_initializable<T> = default;

            constexpr explicit repeat_view(const T& value, Bound bound = Bound())
                requires copy_constructible<T>;
            constexpr explicit repeat_view(T&& value, Bound bound = Bound());

```

```

template<class... TArgs, class... BoundArgs>
    requires constructible_from<T, TArgs...> &&
        constructible_from<Bound, BoundArgs...>
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<TArgs...> value_args, tuple<BoundArgs...> bound_args = tuple<>{});

constexpr iterator begin() const;
constexpr iterator end() const requires (!same_as<Bound, unreachable_sentinel_t>);
constexpr unreachable_sentinel_t end() const noexcept;

constexpr auto size() const requires (!same_as<Bound, unreachable_sentinel_t>);
};

template<class T, class Bound = unreachable_sentinel_t>
    repeat_view(T, Bound = Bound()) -> repeat_view<T, Bound>;
}

```

```

constexpr explicit repeat_view(const T& value, Bound bound = Bound())
    requires copy_constructible<T>;

```

1 *Preconditions:* If `Bound` is not `unreachable_sentinel_t`, `bound` ≥ 0 .

2 *Effects:* Initializes `value_` with `value` and `bound_` with `bound`.

```

constexpr explicit repeat_view(T&& value, Bound bound = Bound());

```

3 *Preconditions:* If `Bound` is not `unreachable_sentinel_t`, `bound` ≥ 0 .

4 *Effects:* Initializes `value_` with `std::move(value)` and `bound_` with `bound`.

```

template<class... TArgs, class... BoundArgs>
    requires constructible_from<T, TArgs...> &&
        constructible_from<Bound, BoundArgs...>
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<TArgs...> value_args, tuple<BoundArgs...> bound_args = tuple<>{});

```

5 *Effects:* Initializes `value_` with `make_from_tuple<T>(std::move(value_args))` and initializes `bound_` with `make_from_tuple<Bound>(std::move(bound_args))`. The behavior is undefined if `Bound` is not `unreachable_sentinel_t` and `bound_` is negative.

```

constexpr iterator begin() const;

```

6 *Effects:* Equivalent to: `return iterator(addressof(*value_));`

```

constexpr iterator end() const requires (!same_as<Bound, unreachable_sentinel_t>);

```

7 *Effects:* Equivalent to: `return iterator(addressof(*value_), bound_);`

```

constexpr unreachable_sentinel_t end() const noexcept;

```

8 *Effects:* Equivalent to: `return unreachable_sentinel;`

```

constexpr auto size() const requires (!same_as<Bound, unreachable_sentinel_t>);

```

9 *Effects:* Equivalent to: `return to_unsigned_like(bound_);`

25.6.5.3 Class `repeat_view::iterator`

[range.repeat.iterator]

```

namespace std::ranges {
    template<move_constructible T, semiregular Bound>
        requires (is_object_v<T> && same_as<T, remove_cv_t<T>> &&
            (integer_like_with_usable_difference_type<Bound> ||
                same_as<Bound, unreachable_sentinel_t>))
        class repeat_view<T, Bound>::iterator {
        private:
            using index_type =                // exposition only
                conditional_t<same_as<Bound, unreachable_sentinel_t>, ptrdiff_t, Bound>;
            const T* value_ = nullptr;        // exposition only
            index_type current_ = index_type(); // exposition only

            constexpr explicit iterator(const T* value, index_type b = index_type()); // exposition only

```

```

public:
    using iterator_concept = random_access_iterator_tag;
    using iterator_category = random_access_iterator_tag;
    using value_type = T;
    using difference_type = see below;

    iterator() = default;

    constexpr const T& operator*() const noexcept;

    constexpr iterator& operator++();
    iterator operator++(int) = default;

    constexpr iterator& operator--();
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type n);
    constexpr iterator& operator-=(difference_type n);
    constexpr const T& operator[](difference_type n) const noexcept;

    friend constexpr bool operator==(const iterator& x, const iterator& y);
    friend constexpr auto operator<=>(const iterator& x, const iterator& y);

    friend constexpr iterator operator+(iterator i, difference_type n);
    friend constexpr iterator operator+(difference_type n, iterator i);

    friend constexpr iterator operator-(iterator i, difference_type n);
    friend constexpr difference_type operator-(const iterator& x, const iterator& y);
};
}

```

- ¹ If *is-signed-integer-like*<*index-type*> is true, the member *typedef-name* *difference_type* denotes *index-type*. Otherwise, it denotes *IOTA-DIFF-T(index-type)* (25.6.4.2).

```
constexpr explicit iterator(const T* value, index-type b = index-type());
```

- ² *Preconditions*: If Bound is not *unreachable_sentinel_t*, $b \geq 0$.

- ³ *Effects*: Initializes *value_* with value and *current_* with b.

```
constexpr const T& operator*() const noexcept;
```

- ⁴ *Effects*: Equivalent to: `return *value_;`

```
constexpr iterator& operator++();
```

- ⁵ *Effects*: Equivalent to:

```
++current_;
return *this;
```

```
constexpr iterator& operator--();
```

- ⁶ *Preconditions*: If Bound is not *unreachable_sentinel_t*, $current_ > 0$.

- ⁷ *Effects*: Equivalent to:

```
--current_;
return *this;
```

```
constexpr iterator& operator+=(difference_type n);
```

- ⁸ *Preconditions*: If Bound is not *unreachable_sentinel_t*, $current_ + n \geq 0$.

- ⁹ *Effects*: Equivalent to:

```
current_ += n;
return *this;
```

```
constexpr iterator& operator-=(difference_type n);
```

- ¹⁰ *Preconditions*: If Bound is not *unreachable_sentinel_t*, $current_ - n \geq 0$.

11 *Effects:* Equivalent to:

```

    current_ -= n;
    return *this;

constexpr const T& operator[](difference_type n) const noexcept;
12 Effects: Equivalent to: return *(*this + n);

friend constexpr bool operator==(const iterator& x, const iterator& y);
13 Effects: Equivalent to: return x.current_ == y.current_;

friend constexpr auto operator<=>(const iterator& x, const iterator& y);
14 Effects: Equivalent to: return x.current_ <=> y.current_;

friend constexpr iterator operator+(iterator i, difference_type n);
friend constexpr iterator operator+(difference_type n, iterator i);

15 Effects: Equivalent to:

    i += n;
    return i;

friend constexpr iterator operator-(iterator i, difference_type n);
16 Effects: Equivalent to:

    i -= n;
    return i;

friend constexpr difference_type operator-(const iterator& x, const iterator& y);
17 Effects: Equivalent to:

    return static_cast<difference_type>(x.current_) - static_cast<difference_type>(y.current_);

```

25.6.6 Istream view

[range.istream]

25.6.6.1 Overview

[range.istream.overview]

- 1 `basic_istream_view` models `input_range` and reads (using `operator>>`) successive elements from its corresponding input stream.
- 2 The name `views::istream<T>` denotes a customization point object (16.3.3.3.5). Given a type `T` and a subexpression `E` of type `U`, if `U` models `derived_from<basic_istream<typename U::char_type, typename U::traits_type>>`, then the expression `views::istream<T>(E)` is expression-equivalent to `basic_istream_view<T, typename U::char_type, typename U::traits_type>(E)`; otherwise, `views::istream<T>(E)` is ill-formed.

3 [Example 1:

```

auto ints = istringstream{"0 1 2 3 4"};
ranges::copy(views::istream<int>(ints), ostream_iterator<int>{cout, "-"});
// prints 0-1-2-3-4-
— end example]

```

25.6.6.2 Class template `basic_istream_view`

[range.istream.view]

```

namespace std::ranges {
    template<class Val, class CharT, class Traits>
        concept stream-extractable = // exposition only
            requires(basic_istream<CharT, Traits>& is, Val& t) {
                is >> t;
            };

    template<movable Val, class CharT, class Traits = char_traits<CharT>>
        requires default_initializable<Val> &&
            stream-extractable<Val, CharT, Traits>
        class basic_istream_view : public view_interface<basic_istream_view<Val, CharT, Traits>> {
        public:
            constexpr explicit basic_istream_view(basic_istream<CharT, Traits>& stream);

```

```
constexpr auto begin() {
    *stream_ >> value_;
    return iterator{*this};
}
```

```
constexpr default_sentinel_t end() const noexcept;
```

```
private:
```

```
// 25.6.6.3, class basic_istream_view::iterator
struct iterator; // exposition only
basic_istream<CharT, Traits>* stream_; // exposition only
Val value_ = Val(); // exposition only
};
```

```
constexpr explicit basic_istream_view(basic_istream<CharT, Traits>& stream);
```

1 *Effects:* Initializes *stream_* with `addressof(stream)`.

```
constexpr default_sentinel_t end() const noexcept;
```

2 *Effects:* Equivalent to: `return default_sentinel;`

25.6.6.3 Class `basic_istream_view::iterator`

[range.istream.iterator]

```
namespace std::ranges {
    template<movable Val, class CharT, class Traits>
        requires default_initializable<Val> &&
            stream_extractable<Val, CharT, Traits>
    class basic_istream_view<Val, CharT, Traits>::iterator {
    public:
        using iterator_concept = input_iterator_tag;
        using difference_type = ptrdiff_t;
        using value_type = Val;

        iterator(const iterator&) = delete;
        iterator(iterator&&) = default;

        iterator& operator=(const iterator&) = delete;
        iterator& operator=(iterator&&) = default;

        iterator& operator++();
        void operator++(int);

        Val& operator*() const;

        friend bool operator==(const iterator& x, default_sentinel_t);

    private:
        constexpr explicit iterator(basic_istream_view& parent) noexcept;
        basic_istream_view* parent_;
    };
}
```

```
constexpr explicit iterator(basic_istream_view& parent) noexcept;
```

1 *Effects:* Initializes *parent_* with `addressof(parent)`.

```
iterator& operator++();
```

2 *Effects:* Equivalent to:

```
*parent_->stream_ >> parent_->value_;
return *this;
```

```
void operator++(int);
```

3 *Effects:* Equivalent to `++*this`.

```

Val& operator*() const;
4     Effects: Equivalent to: return parent_->value_;

friend bool operator==(const iterator& x, default_sentinel_t);
5     Effects: Equivalent to: return !*x.parent_->stream_;

```

25.7 Range adaptors

[range.adaptors]

25.7.1 General

[range.adaptors.general]

- 1 Subclause 25.7 defines *range adaptors*, which are utilities that transform a range into a view with custom behaviors. These adaptors can be chained to create pipelines of range transformations that evaluate lazily as the resulting view is iterated.
- 2 Range adaptors are declared in namespace `std::ranges::views`.
- 3 The bitwise OR operator is overloaded for the purpose of creating adaptor chain pipelines. The adaptors also support function call syntax with equivalent semantics.
- 4 [Example 1:

```

vector<int> ints{0,1,2,3,4,5};
auto even = [](int i) { return 0 == i % 2; };
auto square = [](int i) { return i * i; };
for (int i : ints | views::filter(even) | views::transform(square)) {
    cout << i << ' '; // prints 0 4 16
}
assert(ranges::equal(ints | views::filter(even), views::filter(ints, even)));
— end example]

```

25.7.2 Range adaptor objects

[range.adaptor.object]

- 1 A *range adaptor closure object* is a unary function object that accepts a range argument. For a range adaptor closure object `C` and an expression `R` such that `decltype((R))` models **range**, the following expressions are equivalent:

```

C(R)
R | C

```

Given an additional range adaptor closure object `D`, the expression `C | D` produces another range adaptor closure object `E`. `E` is a perfect forwarding call wrapper (22.10.4) with the following properties:

- (1.1) — Its target object is an object `d` of type `decay_t<decltype((D))>` direct-non-list-initialized with `D`.
- (1.2) — It has one bound argument entity, an object `c` of type `decay_t<decltype((C))>` direct-non-list-initialized with `C`.
- (1.3) — Its call pattern is `d(c(arg))`, where `arg` is the argument used in a function call expression of `E`.

The expression `C | D` is well-formed if and only if the initializations of the state entities of `E` are all well-formed.

- 2 Given an object `t` of type `T`, where

- (2.1) — `t` is a unary function object that accepts a range argument,
- (2.2) — `T` models `derived_from<range_adaptor_closure<T>>`,
- (2.3) — `T` has no other base classes of type `range_adaptor_closure<U>` for any other type `U`, and
- (2.4) — `T` does not model **range**

then the implementation ensures that `t` is a range adaptor closure object.

- 3 The template parameter `D` for `range_adaptor_closure` may be an incomplete type. If an expression of type `cv D` is used as an operand to the `|` operator, `D` shall be complete and model `derived_from<range_adaptor_closure<D>>`. The behavior of an expression involving an object of type `cv D` as an operand to the `|` operator is undefined if overload resolution selects a program-defined `operator|` function.
- 4 If an expression of type `cv U` is used as an operand to the `|` operator, where `U` has a base class of type `range_adaptor_closure<T>` for some type `T` other than `U`, the behavior is undefined.
- 5 The behavior of a program that adds a specialization for `range_adaptor_closure` is undefined.

- ⁶ A *range adaptor object* is a customization point object (16.3.3.3.5) that accepts a `viewable_range` as its first argument and returns a view.
- ⁷ If a range adaptor object accepts only one argument, then it is a range adaptor closure object.
- ⁸ If a range adaptor object `adaptor` accepts more than one argument, then let `range` be an expression such that `decltype((range))` models `viewable_range`, let `args...` be arguments such that `adaptor(range, args...)` is a well-formed expression as specified in the rest of subclause 25.7, and let `BoundArgs` be a pack that denotes `decay_t<decltype((args))>...`. The expression `adaptor(args...)` produces a range adaptor closure object `f` that is a perfect forwarding call wrapper (22.10.4) with the following properties:
- (8.1) — Its target object is a copy of `adaptor`.
 - (8.2) — Its bound argument entities `bound_args` consist of objects of types `BoundArgs...` direct-non-list-initialized with `std::forward<decltype((args))>(args)...`, respectively.
 - (8.3) — Its call pattern is `adaptor(r, bound_args...)`, where `r` is the argument used in a function call expression of `f`.

The expression `adaptor(args...)` is well-formed if and only if the initialization of the bound argument entities of the result, as specified above, are all well-formed.

25.7.3 Movable wrapper

[range.move.wrap]

- ¹ Many types in this subclause are specified in terms of an exposition-only class template *movable-box*. *movable-box*<T> behaves exactly like *optional*<T> with the following differences:
- (1.1) — *movable-box*<T> constrains its type parameter T with `move_constructible<T> && is_object_v<T>`.
 - (1.2) — The default constructor of *movable-box*<T> is equivalent to:


```
constexpr movable-box() noexcept(is_nothrow_default_constructible_v<T>)
    requires default_initializable<T>
    : movable-box{in_place} {}
```
 - (1.3) — If `copyable<T>` is not modeled, the copy assignment operator is equivalent to:


```
constexpr movable-box& operator=(const movable-box& that)
    noexcept(is_nothrow_copy_constructible_v<T>)
    requires copy_constructible<T> {
    if (this != addressof(that)) {
        if (that) emplace(*that);
        else reset();
    }
    return *this;
}
```
 - (1.4) — If `movable<T>` is not modeled, the move assignment operator is equivalent to:


```
constexpr movable-box& operator=(movable-box&& that)
    noexcept(is_nothrow_move_constructible_v<T>) {
    if (this != addressof(that)) {
        if (that) emplace(std::move(*that));
        else reset();
    }
    return *this;
}
```

² Recommended practice:

- (2.1) — If `copy_constructible<T>` is true, *movable-box*<T> should store only a T if either T models copyable, or `is_nothrow_move_constructible_v<T> && is_nothrow_copy_constructible_v<T>` is true.
- (2.2) — Otherwise, *movable-box*<T> should store only a T if either T models movable or `is_nothrow_move_constructible_v<T>` is true.

25.7.4 Non-propagating cache

[range.nonprop.cache]

- ¹ Some types in 25.7 are specified in terms of an exposition-only class template *non-propagating-cache*. *non-propagating-cache*<T> behaves exactly like *optional*<T> with the following differences:
- (1.1) — *non-propagating-cache*<T> constrains its type parameter T with `is_object_v<T>`.

- (1.2) — The copy constructor is equivalent to:

```
constexpr non-propagating-cache(const non-propagating-cache&) noexcept {}
```

- (1.3) — The move constructor is equivalent to:

```
constexpr non-propagating-cache(non-propagating-cache&& other) noexcept {
    other.reset();
}
```

- (1.4) — The copy assignment operator is equivalent to:

```
constexpr non-propagating-cache& operator=(const non-propagating-cache& other) noexcept {
    if (addressof(other) != this)
        reset();
    return *this;
}
```

- (1.5) — The move assignment operator is equivalent to:

```
constexpr non-propagating-cache& operator=(non-propagating-cache&& other) noexcept {
    reset();
    other.reset();
    return *this;
}
```

- (1.6) — *non-propagating-cache*<T> has an additional member function template specified as follows:

```
template<class I>
constexpr T& emplace-deref(const I& i);    // exposition only
```

Mandates: The declaration `T t(*i);` is well-formed for some invented variable `t`.

[*Note 1:* If `*i` is a prvalue of type *cv* T, there is no requirement that it is movable (9.5.1). — *end note*]

Effects: Calls `reset()`. Then direct-non-list-initializes the contained value with `*i`.

Postconditions: `*this` contains a value.

Returns: A reference to the new contained value.

Throws: Any exception thrown by the initialization of the contained value.

Remarks: If an exception is thrown during the initialization of T, `*this` does not contain a value, and the previous value (if any) has been destroyed.

- ² [*Note 2:* *non-propagating-cache* enables an input view to temporarily cache values as it is iterated over. — *end note*]

25.7.5 Range adaptor helpers

[range.adaptor.helpers]

```
namespace std::ranges {
    template<class F, class Tuple>
    constexpr auto tuple-transform(F&& f, Tuple&& t) { // exposition only
        return apply([&<class... Ts>(Ts&&... elements) {
            return tuple<invoke_result_t<F&, Ts>...>(invoke(f, std::forward<Ts>(elements))...);
        }, std::forward<Tuple>(t));
    }

    template<class F, class Tuple>
    constexpr void tuple-for-each(F&& f, Tuple&& t) { // exposition only
        apply([&<class... Ts>(Ts&&... elements) {
            (static_cast<void>(invoke(f, std::forward<Ts>(elements))), ...);
        }, std::forward<Tuple>(t));
    }

    template<class T>
    constexpr T& as-lvalue(T&& t) { // exposition only
        return static_cast<T&>(t);
    }
}
```



```

template<bool Const, class... Views>
    concept all-random-access = // exposition only
        (random_access_range<maybe-const<Const, Views>> && ...);
template<bool Const, class... Views>
    concept all-bidirectional = // exposition only
        (bidirectional_range<maybe-const<Const, Views>> && ...);
template<bool Const, class... Views>
    concept all-forward = // exposition only
        (forward_range<maybe-const<Const, Views>> && ...);
}

```

25.7.6 All view

[range.all]

25.7.6.1 General

[range.all.general]

- ¹ `views::all` returns a view that includes all elements of its range argument.
- ² The name `views::all` denotes a range adaptor object (25.7.2). Given a subexpression E, the expression `views::all(E)` is expression-equivalent to:
- (2.1) — *decay-copy*(E) if the decayed type of E models view.
 - (2.2) — Otherwise, `ref_view{E}` if that expression is well-formed.
 - (2.3) — Otherwise, `owning_view{E}`.

25.7.6.2 Class template `ref_view`

[range.ref.view]

- ¹ `ref_view` is a view of the elements of some other range.

```

namespace std::ranges {
    template<range R>
        requires is_object_v<R>
        class ref_view : public view_interface<ref_view<R>> {
        private:
            R* r_; // exposition only

        public:
            template<different-from<ref_view> T>
                requires see below
                constexpr ref_view(T&& t);

            constexpr R& base() const { return *r_; }

            constexpr iterator_t<R> begin() const { return ranges::begin(*r_); }
            constexpr sentinel_t<R> end() const { return ranges::end(*r_); }

            constexpr bool empty() const
                requires requires { ranges::empty(*r_); }
            { return ranges::empty(*r_); }

            constexpr auto size() const requires sized_range<R>
            { return ranges::size(*r_); }

            constexpr auto reserve_hint() const requires approximately_sized_range<R>
            { return ranges::reserve_hint(*r_); }

            constexpr auto data() const requires contiguous_range<R>
            { return ranges::data(*r_); }
        };

    template<class R>
        ref_view(R&) -> ref_view<R>;
}

```

```
template<different-from<ref_view> T>
requires see below
constexpr ref_view(T&& t);
```

² *Effects:* Initializes r_* with `addressof(static_cast<R&>(std::forward<T>(t)))`.

³ *Remarks:* Let *FUN* denote the exposition-only functions

```
void FUN(R&);
void FUN(R&&) = delete;
```

The expression in the *requires-clause* is equivalent to:

```
convertible_to<T, R&> && requires { FUN(declval<T>()); }
```

25.7.6.3 Class template `owning_view`

[range.owning.view]

¹ `owning_view` is a move-only view of the elements of some other range.

```
namespace std::ranges {
template<range R>
requires movable<R> && (!is-initializer-list<R>) // see 25.4.6
class owning_view : public view_interface<owning_view<R>> {
private:
    R r_ = R();          // exposition only

public:
    owning_view() requires default_initializable<R> = default;
    constexpr owning_view(R&& t);

    owning_view(owning_view&&) = default;
    owning_view& operator=(owning_view&&) = default;

    constexpr R& base() & noexcept { return r_; }
    constexpr const R& base() const & noexcept { return r_; }
    constexpr R&& base() && noexcept { return std::move(r_); }
    constexpr const R&& base() const && noexcept { return std::move(r_); }

    constexpr iterator_t<R> begin() { return ranges::begin(r_); }
    constexpr sentinel_t<R> end() { return ranges::end(r_); }

    constexpr auto begin() const requires range<const R>
    { return ranges::begin(r_); }
    constexpr auto end() const requires range<const R>
    { return ranges::end(r_); }

    constexpr bool empty() requires requires { ranges::empty(r_); }
    { return ranges::empty(r_); }
    constexpr bool empty() const requires requires { ranges::empty(r_); }
    { return ranges::empty(r_); }

    constexpr auto size() requires sized_range<R>
    { return ranges::size(r_); }
    constexpr auto size() const requires sized_range<const R>
    { return ranges::size(r_); }

    constexpr auto reserve_hint() requires approximately_sized_range<R>
    { return ranges::reserve_hint(r_); }
    constexpr auto reserve_hint() const requires approximately_sized_range<const R>
    { return ranges::reserve_hint(r_); }

    constexpr auto data() requires contiguous_range<R>
    { return ranges::data(r_); }
    constexpr auto data() const requires contiguous_range<const R>
    { return ranges::data(r_); }
};
}
```

```
constexpr owning_view(R&& t);
```

² *Effects:* Initializes r_+ with `std::move(t)`.

25.7.7 As rvalue view

[range.as.rvalue]

25.7.7.1 Overview

[range.as.rvalue.overview]

¹ `as_rvalue_view` presents a view of an underlying sequence with the same behavior as the underlying sequence except that its elements are rvalues. Some generic algorithms can be called with an `as_rvalue_view` to replace copying with moving.

² The name `views::as_rvalue` denotes a range adaptor object (25.7.2). Let E be an expression and let T be `decltype((E))`. The expression `views::as_rvalue(E)` is expression-equivalent to:

(2.1) — `views::all(E)` if T models `input_range` and `same_as<range_rvalue_reference_t<T>, range_reference_t<T>>` is true.

(2.2) — Otherwise, `as_rvalue_view(E)`.

³ [Example 1:

```
vector<string> words = {"the", "quick", "brown", "fox", "ate", "a", "pterodactyl"};
vector<string> new_words;
ranges::copy(words | views::as_rvalue, back_inserter(new_words));
// moves each string from words into new_words
```

— end example]

25.7.7.2 Class template `as_rvalue_view`

[range.as.rvalue.view]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class as_rvalue_view : public view_interface<as_rvalue_view<V>> {
            V base_ = V(); // exposition only

        public:
            as_rvalue_view() requires default_initializable<V> = default;
            constexpr explicit as_rvalue_view(V base);

            constexpr V base() const & requires copy_constructible<V> { return base_; }
            constexpr V base() && { return std::move(base_); }

            constexpr auto begin() requires (!simple_view<V>)
            { return move_iterator(ranges::begin(base_)); }
            constexpr auto begin() const requires range<const V>
            { return move_iterator(ranges::begin(base_)); }

            constexpr auto end() requires (!simple_view<V>) {
                if constexpr (common_range<V>) {
                    return move_iterator(ranges::end(base_));
                } else {
                    return move_sentinel(ranges::end(base_));
                }
            }
            constexpr auto end() const requires range<const V> {
                if constexpr (common_range<const V>) {
                    return move_iterator(ranges::end(base_));
                } else {
                    return move_sentinel(ranges::end(base_));
                }
            }

            constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
            constexpr auto size() const requires sized_range<const V> { return ranges::size(base_); }

            constexpr auto reserve_hint() requires approximately_sized_range<V>
            { return ranges::reserve_hint(base_); }
        }
}
```

```
constexpr auto reserve_hint() const requires approximately_sized_range<const V>
{ return ranges::reserve_hint(base_); }
};
```

```
template<class R>
    as_rvalue_view(R&&) -> as_rvalue_view<views::all_t<R>>;
}
```

```
constexpr explicit as_rvalue_view(V base);
```

¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.8 Filter view

[range.filter]

25.7.8.1 Overview

[range.filter.overview]

¹ `filter_view` presents a view of the elements of an underlying sequence that satisfy a predicate.

² The name `views::filter` denotes a range adaptor object (25.7.2). Given subexpressions *E* and *P*, the expression `views::filter(E, P)` is expression-equivalent to `filter_view(E, P)`.

³ [Example 1:

```
vector<int> is{ 0, 1, 2, 3, 4, 5, 6 };
auto evens = views::filter(is, [](int i) { return 0 == i % 2; });
for (int i : evens)
    cout << i << ' '; // prints 0 2 4 6
```

— end example]

25.7.8.2 Class template `filter_view`

[range.filter.view]

```
namespace std::ranges {
    template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
        class filter_view : public view_interface<filter_view<V, Pred>> {
    private:
        V base_ = V(); // exposition only
        movable_box<Pred> pred_; // exposition only

        // 25.7.8.3, class template filter_view::iterator
        template<bool> class iterator; // exposition only

        // 25.7.8.4, class template filter_view::sentinel
        template<bool> class sentinel; // exposition only

    public:
        filter_view() requires default_initializable<V> && default_initializable<Pred> = default;
        constexpr explicit filter_view(V base, Pred pred);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr const Pred& pred() const;

        constexpr iterator<false> begin();
        constexpr iterator<true> begin() const
            requires (input_range<const V> && !forward_range<const V> &&
                indirect_unary_predicate<const Pred, iterator_t<const V>>);

        constexpr auto end() {
            if constexpr (common_range<V>)
                return iterator<false>{*this, ranges::end(base_)};
            else
                return sentinel<false>{*this};
        }
    }
```

```
constexpr sentinel<true> end() const
    requires (input_range<const V> && !forward_range<const V> &&
        indirect_unary_predicate<const Pred, iterator_t<const V>>) {
    return sentinel<true>{*this};
}
};
```

```
template<class R, class Pred>
    filter_view(R&&, Pred) -> filter_view<views::all_t<R>, Pred>;
}
```

```
constexpr explicit filter_view(V base, Pred pred);
```

1 *Effects:* Initializes *base_* with `std::move(base)` and initializes *pred_* with `std::move(pred)`.

```
constexpr const Pred& pred() const;
```

2 *Effects:* Equivalent to: `return *pred_;`

```
constexpr iterator<false> begin();
```

3 *Preconditions:* *pred_.has_value()* is true.

4 *Returns:* `{*this, ranges::find_if(base_, ref(*pred_))}`.

5 *Remarks:* In order to provide the amortized constant time complexity required by the `range` concept when `filter_view` models `forward_range`, this function caches the result within the `filter_view` for use on subsequent calls.

```
constexpr iterator<true> begin() const
    requires (input_range<const V> && !forward_range<const V> &&
        indirect_unary_predicate<const Pred, iterator_t<const V>>);
```

6 *Preconditions:* *pred_.has_value()* is true.

7 *Returns:* `{*this, ranges::find_if(base_, ref(*pred_))}`.

[*Note 1:* This function does not cache the result within the `filter_view`. — *end note*]

25.7.8.3 Class template `filter_view::iterator`

[`range.filter.iterator`]

```
namespace std::ranges {
    template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
    template<bool Const>
    class filter_view<V, Pred>::iterator {
    private:
        using Parent = maybe_const<Const, filter_view>;
        using Base = maybe_const<Const, V>;
        iterator_t<Base> current_ = iterator_t<Base>();
        Parent* parent_ = nullptr;
        constexpr iterator(Parent& parent, iterator_t<Base> current);

    public:
        using iterator_concept = see below;
        using iterator_category = see below;
        using value_type = range_value_t<Base>;
        using difference_type = range_difference_t<Base>;

        iterator() requires default_initializable<iterator_t<Base>> = default;
        constexpr iterator(iterator<!Const> i)
            requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

        constexpr const iterator_t<Base>& base() const & noexcept;
        constexpr iterator_t<Base> base() &&;
        constexpr range_reference_t<Base> operator*() const;
        constexpr iterator_t<Base> operator->() const
            requires has_arrow<iterator_t<Base>> && copyable<iterator_t<Base>>;
    };
};
```

```

constexpr iterator& operator++();
constexpr void operator++(int);
iterator operator++(int) requires forward_range<Base> = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
iterator operator--(int) = default;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;

friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
};
}

```

¹ [Note 1: Modification of the element a `filter_view::iterator` denotes can result in undefined behavior if the underlying range is a `forward_range` and the resulting value does not satisfy the filter predicate when the predicate is next evaluated for that element (18.2). — end note]

² `iterator::iterator_concept` is defined as follows:

- (2.1) — If `Const` is true, then `iterator_concept` denotes `input_iterator_tag`.
- (2.2) — Otherwise, if `V` models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (2.3) — Otherwise, if `V` models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- (2.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

³ The member *typedef-name* `iterator_category` is declared if and only if `Base` models `forward_range`. In that case, `iterator::iterator_category` is defined as follows:

- (3.1) — Let `C` denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.
- (3.2) — If `C` models `derived_from<bidirectional_iterator_tag>`, then `iterator_category` denotes `bidirectional_iterator_tag`.
- (3.3) — Otherwise, if `C` models `derived_from<forward_iterator_tag>`, then `iterator_category` denotes `forward_iterator_tag`.
- (3.4) — Otherwise, `iterator_category` denotes `C`.

```
constexpr iterator(Parent& parent, iterator_t<Base> current);
```

⁴ *Effects:* Initializes `current_` with `std::move(current)` and `parent_` with `addressof(parent)`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

⁵ *Effects:* Initializes `parent_` with `i.parent_` and `current_` with `std::move(i.current_)`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

⁶ *Effects:* Equivalent to: return `current_`;

```
constexpr iterator_t<Base> base() &&;
```

⁷ *Effects:* Equivalent to: return `std::move(current_)`;

```
constexpr range_reference_t<Base> operator*() const;
```

⁸ *Effects:* Equivalent to: return `*current_`;

```
constexpr iterator_t<Base> operator->() const
    requires has_arrow<iterator_t<Base>> && copyable<iterator_t<Base>>;
```

⁹ *Effects:* Equivalent to: return `current_`;

```

constexpr iterator& operator++();
10    Effects: Equivalent to:
        current_ = ranges::find_if(std::move(++current_), ranges::end(parent->base_),
                                   ref(*parent->pred_));
        return *this;

constexpr void operator++(int);
11    Effects: Equivalent to ++*this.

constexpr iterator& operator--() requires bidirectional_range<Base>;
12    Effects: Equivalent to:
        do
            --current_;
        while (!invoke(*parent->pred_, *current_));
        return *this;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;
13    Effects: Equivalent to: return x.current_ == y.current_;

friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
14    Effects: Equivalent to: return ranges::iter_move(i.current_);

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
15    Effects: Equivalent to ranges::iter_swap(x.current_, y.current_).

25.7.8.4 Class template filter_view::sentinel [range.filter.sentinel]
namespace std::ranges {
    template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
    template<bool Const>
    class filter_view<V, Pred>::sentinel {
    private:
        using Base = maybe_const<Const, V>;                                // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();                        // exposition only
        constexpr explicit sentinel(Parent& parent);                       // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel<!Const> other)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;

        template<bool OtherConst>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
            friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
    };
}

constexpr sentinel(sentinel<!Const> other)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
1    Effects: Initializes end_ with std::move(other.end_).

constexpr explicit sentinel(Parent& parent);
2    Effects: Initializes end_ with ranges::end(parent.base_).

```

```
constexpr sentinel_t<Base> base() const;
```

3 *Effects:* Equivalent to: return *end_*;

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const <OtherConst, V>>>
    friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel_t& y);
```

4 *Effects:* Equivalent to: return *x.current_ == y.end_*;

25.7.9 Transform view

[range.transform]

25.7.9.1 Overview

[range.transform.overview]

1 transform_view presents a view of an underlying sequence after applying a transformation function to each element.

2 The name `views::transform` denotes a range adaptor object (25.7.2). Given subexpressions *E* and *F*, the expression `views::transform(E, F)` is expression-equivalent to `transform_view(E, F)`.

3 [Example 1:

```
vector<int> is{ 0, 1, 2, 3, 4 };
auto squares = views::transform(is, [](int i) { return i * i; });
for (int i : squares)
    cout << i << ' '; // prints 0 1 4 9 16
```

— end example]

25.7.9.2 Class template transform_view

[range.transform.view]

```
namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
            can_reference<invoke_result_t<F&, range_reference_t<V>>>
    class transform_view : public view_interface<transform_view<V, F>> {
    private:
        // 25.7.9.3, class template transform_view::iterator
        template<bool> struct iterator; // exposition only

        // 25.7.9.4, class template transform_view::sentinel
        template<bool> struct sentinel; // exposition only

        V base_ = V(); // exposition only
        movable_box<F> fun_; // exposition only

    public:
        transform_view() requires default_initializable<V> && default_initializable<F> = default;
        constexpr explicit transform_view(V base, F fun);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr iterator<false> begin();
        constexpr iterator<true> begin() const
            requires range<const V> &&
            regular_invocable<const F&, range_reference_t<const V>>;

        constexpr sentinel<false> end();
        constexpr iterator<false> end() requires common_range<V>;
        constexpr sentinel<true> end() const
            requires range<const V> &&
            regular_invocable<const F&, range_reference_t<const V>>;
        constexpr iterator<true> end() const
            requires common_range<const V> &&
            regular_invocable<const F&, range_reference_t<const V>>;
```



```

constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
constexpr auto size() const requires sized_range<const V>
{ return ranges::size(base_); }

constexpr auto reserve_hint() requires approximately_sized_range<V>
{ return ranges::reserve_hint(base_); }
constexpr auto reserve_hint() const requires approximately_sized_range<const V>
{ return ranges::reserve_hint(base_); }
};

template<class R, class F>
    transform_view(R&&, F) -> transform_view<views::all_t<R>, F>;
}

```

```
constexpr explicit transform_view(V base, F fun);
```

1 *Effects:* Initializes *base_* with `std::move(base)` and *fun_* with `std::move(fun)`.

```
constexpr iterator<false> begin();
```

2 *Effects:* Equivalent to:

```
    return iterator<false>{*this, ranges::begin(base_)};
```

```
constexpr iterator<true> begin() const
    requires range<const V> &&
    regular_invocable<const F&, range_reference_t<const V>>;
```

3 *Effects:* Equivalent to:

```
    return iterator<true>{*this, ranges::begin(base_)};
```

```
constexpr sentinel<false> end();
```

4 *Effects:* Equivalent to:

```
    return sentinel<false>{ranges::end(base_)};
```

```
constexpr iterator<false> end() requires common_range<V>;
```

5 *Effects:* Equivalent to:

```
    return iterator<false>{*this, ranges::end(base_)};
```

```
constexpr sentinel<true> end() const
    requires range<const V> &&
    regular_invocable<const F&, range_reference_t<const V>>;
```

6 *Effects:* Equivalent to:

```
    return sentinel<true>{ranges::end(base_)};
```

```
constexpr iterator<true> end() const
    requires common_range<const V> &&
    regular_invocable<const F&, range_reference_t<const V>>;
```

7 *Effects:* Equivalent to:

```
    return iterator<true>{*this, ranges::end(base_)};
```

25.7.9.3 Class template `transform_view::iterator`

[range.transform.iterator]

```

namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> &&
        regular_invocable<F&, range_reference_t<V>> &&
        can_reference_invoke_result_t<F&, range_reference_t<V>>>
    template<bool Const>
    class transform_view<V, F>::iterator {
    private:
        using Parent = maybe_const<Const, transform_view>;
        using Base = maybe_const<Const, V>;
        iterator_t<Base> current_ = iterator_t<Base>();

```

// exposition only
// exposition only
// exposition only

```

Parent* parent_ = nullptr; // exposition only
constexpr iterator(Parent& parent, iterator_t<Base> current); // exposition only

public:
    using iterator_concept = see below;
    using iterator_category = see below; // not always present
    using value_type =
        remove_cvref_t<invoke_result_t<maybe-const<Const, F>&, range_reference_t<Base>>>;
    using difference_type = range_difference_t<Base>;

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr iterator(iterator_t<Const> i)
        requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr const iterator_t<Base>& base() const & noexcept;
    constexpr iterator_t<Base> base() &&;

    constexpr decltype(auto) operator*() const
        noexcept(noexcept(invoke(*parent_ -> fun_, *current_))) {
        return invoke(*parent_ -> fun_, *current_);
    }

    constexpr iterator& operator++();
    constexpr void operator++(int);
    iterator operator++(int) requires forward_range<Base> = default;

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type n)
        requires random_access_range<Base>;
    constexpr iterator& operator-=(difference_type n)
        requires random_access_range<Base>;

    constexpr decltype(auto) operator[](difference_type n) const
        requires random_access_range<Base> {
        return invoke(*parent_ -> fun_, current_[n]);
    }

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<iterator_t<Base>>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;

    friend constexpr iterator operator+(iterator i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, iterator i)
        requires random_access_range<Base>;

    friend constexpr iterator operator-(iterator i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
        requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *Base* models *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² The member *typedef-name* *iterator_category* is declared if and only if *Base* models *forward_range*. In that case, *iterator::iterator_category* is defined as follows: Let *C* denote the type *iterator_traits<iterator_t<Base>>::iterator_category*.

- (2.1) — If *is_reference_v<invoke_result_t<maybe_const<Const, F>&, range_reference_t<Base>>>* is true, then
 - (2.1.1) — if *C* models *derived_from<contiguous_iterator_tag>*, *iterator_category* denotes *random_access_iterator_tag*;
 - (2.1.2) — otherwise, *iterator_category* denotes *C*.
- (2.2) — Otherwise, *iterator_category* denotes *input_iterator_tag*.

```
constexpr iterator (Parent& parent, iterator_t<Base> current);
```

³ *Effects*: Initializes *current_* with `std::move(current)` and *parent_* with `addressof(parent)`.

```
constexpr iterator (iterator<!Const> i)
  requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

⁴ *Effects*: Initializes *current_* with `std::move(i.current_)` and *parent_* with `i.parent_`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

⁵ *Effects*: Equivalent to: `return current_;`

```
constexpr iterator_t<Base> base() &&;
```

⁶ *Effects*: Equivalent to: `return std::move(current_);`

```
constexpr iterator& operator++();
```

⁷ *Effects*: Equivalent to:

```
++current_;
return *this;
```

```
constexpr void operator++(int);
```

⁸ *Effects*: Equivalent to `++current_`.

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

⁹ *Effects*: Equivalent to:

```
--current_;
return *this;
```

```
constexpr iterator& operator+=(difference_type n)
  requires random_access_range<Base>;
```

¹⁰ *Effects*: Equivalent to:

```
current_ += n;
return *this;
```

```
constexpr iterator& operator-=(difference_type n)
  requires random_access_range<Base>;
```

¹¹ *Effects*: Equivalent to:

```
current_ -= n;
return *this;
```

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;
12     Effects: Equivalent to: return x.current_ == y.current_;

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
13     Effects: Equivalent to: return x.current_ < y.current_;

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
14     Effects: Equivalent to: return y < x;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
15     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
16     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;
17     Effects: Equivalent to: return x.current_ <=> y.current_;

friend constexpr iterator operator+(iterator i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, iterator i)
    requires random_access_range<Base>;
18     Effects: Equivalent to: return iterator{*i.parent_, i.current_ + n};

friend constexpr iterator operator-(iterator i, difference_type n)
    requires random_access_range<Base>;
19     Effects: Equivalent to: return iterator{*i.parent_, i.current_ - n};

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
20     Effects: Equivalent to: return x.current_ - y.current_;

```

25.7.9.4 Class template transform_view::sentinel

[range.transform.sentinel]

```

namespace std::ranges {
    template<input_range V, move_constructible F>
        requires view<V> && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<V>> &&
            can_reference<invoke_result_t<F&, range_reference_t<V>>>
    template<bool Const>
    class transform_view<V, F>::sentinel {
    private:
        using Parent = maybe_const<Const, transform_view>; // exposition only
        using Base = maybe_const<Const, V>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only
        constexpr explicit sentinel(sentinel_t<Base> end); // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel_t<Const> i)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;

```

```

template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const iterator<OtherConst>& x, const sentinel& y);

template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const sentinel& y, const iterator<OtherConst>& x);
};
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects:* Initializes *end_* with *end*.

```
constexpr sentinel(sentinel_t<!Const> i)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes *end_* with `std::move(i.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

3 *Effects:* Equivalent to: `return end_;`

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

4 *Effects:* Equivalent to: `return x.current_ == y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const iterator<OtherConst>& x, const sentinel& y);
```

5 *Effects:* Equivalent to: `return x.current_ - y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const sentinel& y, const iterator<OtherConst>& x);
```

6 *Effects:* Equivalent to: `return y.end_ - x.current_;`

25.7.10 Take view

[range.take]

25.7.10.1 Overview

[range.take.overview]

- 1 `take_view` produces a view of the first *N* elements from another view, or all the elements if the adapted view contains fewer than *N*.
- 2 The name `views::take` denotes a range adaptor object (25.7.2). Let *E* and *F* be expressions, let *T* be `remove_cvref_t<decltype((E))>`, and let *D* be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:
 - (2.1) — If *T* is a specialization of `empty_view` (25.6.2.2), then `((void)F, decay-copy(E))`, except that the evaluations of *E* and *F* are indeterminately sequenced.
 - (2.2) — Otherwise, if *T* is a specialization of `optional` and *T* models view, then `(static_cast<D>(F) == D() ? ((void)E, T()) : decay-copy(E))`.
 - (2.3) — Otherwise, if *T* models `random_access_range` and `sized_range` and is a specialization of `span` (23.7.2.2), `basic_string_view` (27.3), or `subrange` (25.5.4), then `U(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distance(E), F))`, except that *E* is evaluated only once, where *U* is a type determined as follows:

- (2.3.1) — if T is a specialization of `span`, then U is `span<typename T::element_type>`;
- (2.3.2) — otherwise, if T is a specialization of `basic_string_view`, then U is T;
- (2.3.3) — otherwise, T is a specialization of `subrange`, and U is `subrange<iterator_t<T>>`;
- (2.4) — otherwise, if T is a specialization of `iota_view` (25.6.4.2) that models `random_access_range` and `sized_range`, then `iota_view(*ranges::begin(E), *(ranges::begin(E) + std::min<D>(ranges::distance(E), F)))`, except that E is evaluated only once.
- (2.5) — Otherwise, if T is a specialization of `repeat_view` (25.6.5.2):
- (2.5.1) — if T models `sized_range`, then
 - `views::repeat(*E.value_, std::min<D>(ranges::distance(E), F))`
 - except that E is evaluated only once;
- (2.5.2) — otherwise, `views::repeat(*E.value_, static_cast<D>(F))`.
- (2.6) — Otherwise, `take_view(E, F)`.

³ [Example 1:

```
vector<int> is{0,1,2,3,4,5,6,7,8,9};
for (int i : is | views::take(5))
  cout << i << ' '; // prints 0 1 2 3 4
```

— end example]

25.7.10.2 Class template `take_view`

[range.take.view]

```
namespace std::ranges {
  template<view V>
  class take_view : public view_interface<take_view<V>> {
  private:
    V base_ = V(); // exposition only
    range_difference_t<V> count_ = 0; // exposition only

    // 25.7.10.3, class template take_view::sentinel
    template<bool> class sentinel; // exposition only

  public:
    take_view() requires default_initializable<V> = default;
    constexpr explicit take_view(V base, range_difference_t<V> count);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() requires (!simple_view<V>) {
      if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>) {
          return ranges::begin(base_);
        } else {
          auto sz = range_difference_t<V>(size());
          return counted_iterator(ranges::begin(base_), sz);
        }
      } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>) {
        auto it = ranges::begin(base_);
        auto sz = std::min(count_, ranges::end(base_) - it);
        return counted_iterator(std::move(it), sz);
      } else {
        return counted_iterator(ranges::begin(base_), count_);
      }
    }

    constexpr auto begin() const requires range<const V> {
      if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>) {
          return ranges::begin(base_);
        } else {

```

```

        auto sz = range_difference_t<const V>(size());
        return counted_iterator(ranges::begin(base_), sz);
    }
} else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>) {
    auto it = ranges::begin(base_);
    auto sz = std::min(count_, ranges::end(base_) - it);
    return counted_iterator(std::move(it), sz);
} else {
    return counted_iterator(ranges::begin(base_), count_);
}
}

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
            return ranges::begin(base_) + range_difference_t<V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>) {
        return default_sentinel;
    } else {
        return sentinel<false>{ranges::end(base_)};
    }
}

constexpr auto end() const requires range<const V> {
    if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>)
            return ranges::begin(base_) + range_difference_t<const V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>) {
        return default_sentinel;
    } else {
        return sentinel<true>{ranges::end(base_)};
    }
}

constexpr auto size() requires sized_range<V> {
    auto n = ranges::size(base_);
    return ranges::min(n, static_cast<decltype(n)>(count_));
}

constexpr auto size() const requires sized_range<const V> {
    auto n = ranges::size(base_);
    return ranges::min(n, static_cast<decltype(n)>(count_));
}

constexpr auto reserve_hint() {
    if constexpr (approximately_sized_range<V>) {
        auto n = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
        return to_unsigned-like(ranges::min(n, count_));
    }
    return to_unsigned-like(count_);
}

constexpr auto reserve_hint() const {
    if constexpr (approximately_sized_range<const V>) {
        auto n = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
        return to_unsigned-like(ranges::min(n, count_));
    }
    return to_unsigned-like(count_);
}
};

```

```

template<class R>
    take_view(R&&, range_difference_t<R>)
        -> take_view<views::all_t<R>>;
}

```

```
constexpr explicit take_view(V base, range_difference_t<V> count);
```

1 *Preconditions:* `count >= 0` is true.

2 *Effects:* Initializes `base_` with `std::move(base)` and `count_` with `count`.

25.7.10.3 Class template `take_view::sentinel`

[range.take.sentinel]

```

namespace std::ranges {
    template<view V>
    template<bool Const>
    class take_view<V>::sentinel {
    private:
        using Base = maybe_const<Const, V>;                                     // exposition only
        template<bool OtherConst>
        using CI = counted_iterator<iterator_t<maybe_const<OtherConst, V>>>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();                          // exposition only
        constexpr explicit sentinel(sentinel_t<Base> end);                  // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel_t<Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;

        friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

        template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);
    };
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects:* Initializes `end_` with `end`.

```

constexpr sentinel(sentinel_t<Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

2 *Effects:* Initializes `end_` with `std::move(s.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

3 *Effects:* Equivalent to: `return end_;`

```
friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);
```

```

template<bool OtherConst = !Const>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
    friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x);

```

4 *Effects:* Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

25.7.11 Take while view

[range.take.while]

25.7.11.1 Overview

[range.take.while.overview]

1 Given a unary predicate `pred` and a view `r`, `take_while_view` produces a view of the range `[ranges::begin(r), ranges::find_if_not(r, pred))`.

2 The name `views::take_while` denotes a range adaptor object (25.7.2). Given subexpressions `E` and `F`, the expression `views::take_while(E, F)` is expression-equivalent to `take_while_view(E, F)`.

³ [Example 1:

```
auto input = istream{"0 1 2 3 4 5 6 7 8 9"};
auto small = [] (const auto x) noexcept { return x < 5; };
auto small_ints = views::istream<int>(input) | views::take_while(small);
for (const auto i : small_ints) {
    cout << i << ' ';
    // prints 0 1 2 3 4
}
auto i = 0;
input >> i;
cout << i;
// prints 6
```

— end example]

25.7.11.2 Class template `take_while_view`

[range.take.while.view]

```
namespace std::ranges {
    template<view V, class Pred>
        requires input_range<V> && is_object_v<Pred> &&
            indirect_unary_predicate<const Pred, iterator_t<V>>
    class take_while_view : public view_interface<take_while_view<V, Pred>> {
        // 25.7.11.3, class template take_while_view::sentinel
        template<bool> class sentinel;
        // exposition only

        V base_ = V();
        // exposition only
        movable_box<Pred> pred_;
        // exposition only

    public:
        take_while_view() requires default_initializable<V> && default_initializable<Pred> = default;
        constexpr explicit take_while_view(V base, Pred pred);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr const Pred& pred() const;

        constexpr auto begin() requires (!simple_view<V>)
        { return ranges::begin(base_); }

        constexpr auto begin() const
            requires range<const V> &&
            indirect_unary_predicate<const Pred, iterator_t<const V>>
        { return ranges::begin(base_); }

        constexpr auto end() requires (!simple_view<V>)
        { return sentinel<false>(ranges::end(base_), addressof(*pred_)); }

        constexpr auto end() const
            requires range<const V> &&
            indirect_unary_predicate<const Pred, iterator_t<const V>>
        { return sentinel<true>(ranges::end(base_), addressof(*pred_)); }
    };

    template<class R, class Pred>
        take_while_view(R&&, Pred) -> take_while_view<views::all_t<R>, Pred>;
}

constexpr explicit take_while_view(V base, Pred pred);

1     Effects: Initializes base_ with std::move(base) and pred_ with std::move(pred).

constexpr const Pred& pred() const;

2     Effects: Equivalent to: return *pred_;
```

25.7.11.3 Class template `take_while_view::sentinel`

[range.take.while.sentinel]

```

namespace std::ranges {
    template<view V, class Pred>
        requires input_range<V> && is_object_v<Pred> &&
            indirect_unary_predicate<const Pred, iterator_t<V>>
    template<bool Const>
    class take_while_view<V, Pred>::sentinel {
        using Base = maybe_const<Const, V>;                                     // exposition only

        sentinel_t<Base> end_ = sentinel_t<Base>();                           // exposition only
        const Pred* pred_ = nullptr;                                           // exposition only
        constexpr explicit sentinel(sentinel_t<Base> end, const Pred* pred);   // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel_t<!Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const { return end_; }

        friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);

        template<bool OtherConst = !Const>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator_t<maybe_const<OtherConst, V>>& x,
                                         const sentinel& y);
    };
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end, const Pred* pred);
```

¹ *Effects:* Initializes `end_` with `end` and `pred_` with `pred`.

```
constexpr sentinel(sentinel_t<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

² *Effects:* Initializes `end_` with `std::move(s.end_)` and `pred_` with `s.pred_`.

```
friend constexpr bool operator==(const iterator_t<Base>& x, const sentinel& y);
```

```
template<bool OtherConst = !Const>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator_t<maybe_const<OtherConst, V>>& x,
                                const sentinel& y);
```

³ *Effects:* Equivalent to: `return y.end_ == x || !invoke(*y.pred_, *x);`

25.7.12 Drop view

[range.drop]

25.7.12.1 Overview

[range.drop.overview]

¹ `drop_view` produces a view excluding the first N elements from another view, or an empty range if the adapted view contains fewer than N elements.

² The name `views::drop` denotes a range adaptor object (25.7.2). Let E and F be expressions, let T be `remove_cvref_t<decltype((E))>`, and let D be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

(2.1) — If T is a specialization of `empty_view` (25.6.2.2), then `((void)F, decay-copy(E))`, except that the evaluations of E and F are indeterminately sequenced.

(2.2) — Otherwise, if T is a specialization of `optional` and T models view, then `(static_cast<D>(F) == D() ? decay-copy(E) : ((void)E, T()))`.

(2.3) — Otherwise, if T models `random_access_range` and `sized_range` and is

(2.3.1) — a specialization of `span` (23.7.2.2),

(2.3.2) — a specialization of `basic_string_view` (27.3),

- (2.3.3) — a specialization of `iota_view` (25.6.4.2), or
- (2.3.4) — a specialization of `subrange` (25.5.4) where `T::StoreSize` is false,
then `U(ranges::begin(E) + std::min<D>(ranges::distance(E), F), ranges::end(E))`, except
that `E` is evaluated only once, where `U` is `span<typename T::element_type>` if `T` is a specialization of
`span` and `T` otherwise.
- (2.4) — Otherwise, if `T` is a specialization of `subrange` that models `random_access_range` and `sized_range`,
then `T(ranges::begin(E) + std::min<D>(ranges::distance(E), F), ranges::
end(E), to-unsigned-like(ranges::distance(E) - std::min<D>(ranges::distance(E), F)))`,
except that `E` and `F` are each evaluated only once.
- (2.5) — Otherwise, if `T` is a specialization of `repeat_view` (25.6.5.2):
- (2.5.1) — if `T` models `sized_range`, then
`views::repeat(*E.value_, ranges::distance(E) - std::min<D>(ranges::distance(E), F))`
except that `E` is evaluated only once;
- (2.5.2) — otherwise, `((void)F, decay-copy(E))`, except that the evaluations of `E` and `F` are indeterminately
sequenced.
- (2.6) — Otherwise, `drop_view(E, F)`.

³ [Example 1:

```
auto ints = views::iota(0) | views::take(10);
for (auto i : ints | views::drop(5)) {
    cout << i << ' ';
} // prints 5 6 7 8 9
```

— end example]

25.7.12.2 Class template `drop_view`

[range.drop.view]

```
namespace std::ranges {
    template<view V>
    class drop_view : public view_interface<drop_view<V>> {
    public:
        drop_view() requires default_initializable<V> = default;
        constexpr explicit drop_view(V base, range_difference_t<V> count);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin()
            requires (!simple-view<V> &&
                    random_access_range<const V> && sized_range<const V>);
        constexpr auto begin() const
            requires random_access_range<const V> && sized_range<const V>;

        constexpr auto end() requires (!simple-view<V>)
        { return ranges::end(base_); }

        constexpr auto end() const requires range<const V>
        { return ranges::end(base_); }

        constexpr auto size() requires sized_range<V> {
            const auto s = ranges::size(base_);
            const auto c = static_cast<decltype(s)>(count_);
            return s < c ? 0 : s - c;
        }

        constexpr auto size() const requires sized_range<const V> {
            const auto s = ranges::size(base_);
            const auto c = static_cast<decltype(s)>(count_);
            return s < c ? 0 : s - c;
        }
    }
}
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V> {
    const auto s = static_cast<range_difference_t<V>>(ranges::reserve_hint(base_));
    return to_unsigned-like(s < count_ ? 0 : s - count_);
}

constexpr auto reserve_hint() const requires approximately_sized_range<const V> {
    const auto s = static_cast<range_difference_t<const V>>(ranges::reserve_hint(base_));
    return to_unsigned-like(s < count_ ? 0 : s - count_);
}
```

```
private:
    V base_ = V(); // exposition only
    range_difference_t<V> count_ = 0; // exposition only
};
```

```
template<class R>
    drop_view(R&&, range_difference_t<R>) -> drop_view<views::all_t<R>>;
}
```

```
constexpr explicit drop_view(V base, range_difference_t<V> count);
```

1 *Preconditions:* `count >= 0` is true.

2 *Effects:* Initializes `base_` with `std::move(base)` and `count_` with `count`.

```
constexpr auto begin()
    requires (! (simple-view<V> &&
        random_access_range<const V> && sized_range<const V>));
constexpr auto begin() const
    requires random_access_range<const V> && sized_range<const V>;
```

3 *Returns:* `ranges::next(ranges::begin(base_), count_, ranges::end(base_))`.

4 *Remarks:* In order to provide the amortized constant-time complexity required by the `range` concept when `drop_view` models `forward_range`, the first overload caches the result within the `drop_view` for use on subsequent calls.

[*Note 1:* Without this, applying a `reverse_view` over a `drop_view` would have quadratic iteration complexity. — end note]

25.7.13 Drop while view

[range.drop.while]

25.7.13.1 Overview

[range.drop.while.overview]

1 Given a unary predicate `pred` and a view `r`, `drop_while_view` produces a view of the range `[ranges::find_if_not(r, pred), ranges::end(r))`.

2 The name `views::drop_while` denotes a range adaptor object (25.7.2). Given subexpressions `E` and `F`, the expression `views::drop_while(E, F)` is expression-equivalent to `drop_while_view(E, F)`.

3 [*Example 1:*

```
constexpr auto source = " \t \t \t hello there"sv;
auto is_invisible = [](const auto x) { return x == ' ' || x == '\t'; };
auto skip_ws = views::drop_while(source, is_invisible);
for (auto c : skip_ws) {
    cout << c; // prints hello there with no leading space
}
```

— end example]

25.7.13.2 Class template `drop_while_view`

[range.drop.while.view]

```
namespace std::ranges {
    template<view V, class Pred>
        requires input_range<V> && is_object_v<Pred> &&
            indirect_unary_predicate<const Pred, iterator_t<V>>
        class drop_while_view : public view_interface<drop_while_view<V, Pred>> {
        public:
            drop_while_view() requires default_initializable<V> && default_initializable<Pred> = default;
            constexpr explicit drop_while_view(V base, Pred pred);
```

```

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }

constexpr const Pred& pred() const;

constexpr auto begin();

constexpr auto end() { return ranges::end(base_); }

private:
    V base_ = V(); // exposition only
    movable_box<Pred> pred_; // exposition only
};

template<class R, class Pred>
    drop_while_view(R&&, Pred) -> drop_while_view<views::all_t<R>, Pred>;
}

```

```
constexpr explicit drop_while_view(V base, Pred pred);
```

1 *Effects:* Initializes *base_* with `std::move(base)` and *pred_* with `std::move(pred)`.

```
constexpr const Pred& pred() const;
```

2 *Effects:* Equivalent to: `return *pred_;`

```
constexpr auto begin();
```

3 *Preconditions:* *pred_.has_value()* is true.

4 *Returns:* `ranges::find_if_not(base_, cref(*pred_))`.

5 *Remarks:* In order to provide the amortized constant-time complexity required by the `range` concept when `drop_while_view` models `forward_range`, the first call caches the result within the `drop_while_view` for use on subsequent calls.

[*Note 1:* Without this, applying a `reverse_view` over a `drop_while_view` would have quadratic iteration complexity. — end note]

25.7.14 Join view

[range.join]

25.7.14.1 Overview

[range.join.overview]

1 `join_view` flattens a view of ranges into a view.

2 The name `views::join` denotes a range adaptor object (25.7.2). Given a subexpression *E*, the expression `views::join(E)` is expression-equivalent to `join_view<views::all_t<decltype(E)>>>{E}`.

3 [*Example 1:*

```

vector<string> ss{"hello", " ", "world", "!"};
for (char ch : ss | views::join)
    cout << ch; // prints hello world!
— end example]

```

25.7.14.2 Class template `join_view`

[range.join.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>>
        class join_view : public view_interface<join_view<V>> {
        private:
            using InnerRng = range_reference_t<V>; // exposition only

            // 25.7.14.3, class template join_view::iterator
            template<bool Const>
                struct iterator; // exposition only

            // 25.7.14.4, class template join_view::sentinel
            template<bool Const>
                struct sentinel; // exposition only
        };
}

```

```

V base_ = V(); // exposition only

non-propagating-cache<iterator_t<V>> outer_; // exposition only, present only
// if forward_range<V> is false
non-propagating-cache<remove_cv_t<InnerRng>> inner_; // exposition only, present only
// if is_reference_v<InnerRng> is false

public:
    join_view() requires default_initializable<V> = default;
    constexpr explicit join_view(V base);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() {
        if constexpr (forward_range<V>) {
            constexpr bool use_const = simple-view<V> &&
                                     is_reference_v<InnerRng>;
            return iterator<use_const>{*this, ranges::begin(base_)};
        } else {
            outer_ = ranges::begin(base_);
            return iterator<false>{*this};
        }
    }

    constexpr auto begin() const
        requires forward_range<const V> &&
                 is_reference_v<range_reference_t<const V>> &&
                 input_range<range_reference_t<const V>>
    { return iterator<true>{*this, ranges::begin(base_)}; }

    constexpr auto end() {
        if constexpr (forward_range<V> &&
                     is_reference_v<InnerRng> && forward_range<InnerRng> &&
                     common_range<V> && common_range<InnerRng>)
            return iterator<simple-view<V>>{*this, ranges::end(base_)};
        else
            return sentinel<simple-view<V>>{*this};
    }

    constexpr auto end() const
        requires forward_range<const V> &&
                 is_reference_v<range_reference_t<const V>> &&
                 input_range<range_reference_t<const V>> {
        if constexpr (forward_range<range_reference_t<const V>> &&
                     common_range<const V> &&
                     common_range<range_reference_t<const V>>)
            return iterator<true>{*this, ranges::end(base_)};
        else
            return sentinel<true>{*this};
        }
    };

    template<class R>
        explicit join_view(R&&) -> join_view<views::all_t<R>>;
}

```

```
constexpr explicit join_view(V base);
```

¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.14.3 Class template `join_view::iterator`

[range.join.iterator]

```

namespace std::ranges {
    template<input_range V>

```

```

    requires view<V> && input_range<range_reference_t<V>>
template<bool Const>
struct join_view<V>::iterator {
private:
    using Parent      = maybe_const<Const, join_view>;           // exposition only
    using Base        = maybe_const<Const, V>;                 // exposition only
    using OuterIter   = iterator_t<Base>;                       // exposition only
    using InnerIter    = iterator_t<range_reference_t<Base>>;    // exposition only

    static constexpr bool ref-is-glvalue =                     // exposition only
        is_reference_v<range_reference_t<Base>>;

    OuterIter outer_ = OuterIter();                             // exposition only, present only
                                                                // if Base models forward_range
    optional<InnerIter> inner_;                                  // exposition only
    Parent* parent_ = nullptr;                                   // exposition only

    constexpr void satisfy();                                    // exposition only

    constexpr OuterIter& outer();                               // exposition only
    constexpr const OuterIter& outer() const;                  // exposition only

    constexpr iterator(Parent& parent, OuterIter outer)
        requires forward_range<Base>;                           // exposition only
    constexpr explicit iterator(Parent& parent)
        requires (!forward_range<Base>);                         // exposition only

public:
    using iterator_concept = see below;
    using iterator_category = see below;                         // not always present
    using value_type        = range_value_t<range_reference_t<Base>>;
    using difference_type    = see below;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
        requires Const &&
            convertible_to<iterator_t<V>, OuterIter> &&
            convertible_to<iterator_t<InnerRng>, InnerIter>;

    constexpr decltype(auto) operator*() const { return **inner_; }

    constexpr InnerIter operator->() const
        requires has_arrow<InnerIter> && copyable<InnerIter>;

    constexpr iterator& operator++();
    constexpr void operator++(int);
    iterator operator++(int)
        requires ref-is-glvalue && forward_range<Base> &&
            forward_range<range_reference_t<Base>> = default;

    constexpr iterator& operator--()
        requires ref-is-glvalue && bidirectional_range<Base> &&
            bidirectional_range<range_reference_t<Base>> &&
            common_range<range_reference_t<Base>>;

    iterator operator--(int) = default;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires ref-is-glvalue && forward_range<Base> &&
            equality_comparable<iterator_t<range_reference_t<Base>>>;

    friend constexpr decltype(auto) iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(*i.inner_))) {
        return ranges::iter_move(*i.inner_);
    }

```

```

    }

    friend constexpr void iter_swap(const iterator& x, const iterator& y)
        noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
        requires indirectly_swappable<InnerIter>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *ref-is-glvalue* is true, *Base* models *bidirectional_range*, and *range_reference_t<Base>* models both *bidirectional_range* and *common_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.2) — Otherwise, if *ref-is-glvalue* is true and *Base* and *range_reference_t<Base>* each model *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² The member *typedef-name* *iterator_category* is declared if and only if *ref-is-glvalue* is true, *Base* models *forward_range*, and *range_reference_t<Base>* models *forward_range*. In that case, *iterator::iterator_category* is defined as follows:

- (2.1) — Let *OUTERC* denote *iterator_traits<iterator_t<Base>>::iterator_category*, and let *INNERC* denote *iterator_traits<iterator_t<range_reference_t<Base>>>::iterator_category*.
- (2.2) — If *OUTERC* and *INNERC* each model *derived_from<bidirectional_iterator_tag>* and *range_reference_t<Base>* models *common_range*, *iterator_category* denotes *bidirectional_iterator_tag*.
- (2.3) — Otherwise, if *OUTERC* and *INNERC* each model *derived_from<forward_iterator_tag>*, *iterator_category* denotes *forward_iterator_tag*.
- (2.4) — Otherwise, *iterator_category* denotes *input_iterator_tag*.

³ *iterator::difference_type* denotes the type:

```

common_type_t<
    range_difference_t<Base>,
    range_difference_t<range_reference_t<Base>>>

```

⁴ *join_view* iterators use the *satisfy* function to skip over empty inner ranges.

```

constexpr OuterIter& outer();
constexpr const OuterIter& outer() const;

```

⁵ *Returns:* *outer_* if *Base* models *forward_range*; otherwise, **parent_->outer_*.

```
constexpr void satisfy();
```

⁶ *Effects:* Equivalent to:

```

auto update_inner = [this](const iterator_t<Base>& x) -> auto&& {
    if constexpr (ref-is-glvalue)    // *x is a reference
        return *x;
    else
        return parent_->inner_.emplace-deref(x);
};

for (; outer() != ranges::end(parent_->base_); ++outer()) {
    auto&& inner = update_inner(outer());
    inner_ = ranges::begin(inner);
    if (*inner_ != ranges::end(inner))
        return;
}
if constexpr (ref-is-glvalue)
    inner_.reset();

```



```

constexpr iterator(Parent& parent, OuterIter outer)
    requires forward_range<Base>;

7     Effects: Initializes outer_ with std::move(outer) and parent_ with addressof(parent); then calls
        satisfy().

constexpr explicit iterator(Parent& parent)
    requires (!forward_range<Base>);

8     Effects: Initializes parent_ with addressof(parent); then calls satisfy().

constexpr iterator(iterator<!Const> i)
    requires Const &&
        convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter>;

9     Effects: Initializes outer_ with std::move(i.outer_), inner_ with std::move(i.inner_), and
        parent_ with i.parent_.

10    [Note 1: Const can only be true when Base models forward_range. — end note]

constexpr InnerIter operator->() const
    requires has_arrow<InnerIter> && copyable<InnerIter>;

11    Effects: Equivalent to: return *inner_;

constexpr iterator& operator++();

12    Let inner-range be:
(12.1)    — If ref-is-glvalue is true, *outer().
(12.2)    — Otherwise, *parent_ -> inner_.

13    Effects: Equivalent to:
        if (++*inner_ == ranges::end(as-lvalue(inner-range))) {
            ++outer();
            satisfy();
        }
        return *this;

constexpr void operator++(int);

14    Effects: Equivalent to: ++*this.

constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional_range<range_reference_t<Base>> &&
        common_range<range_reference_t<Base>>;

15    Effects: Equivalent to:
        if (outer_ == ranges::end(parent_ -> base_))
            inner_ = ranges::end(as-lvalue(*--outer_));
        while (*inner_ == ranges::begin(as-lvalue(*outer_)))
            *inner_ = ranges::end(as-lvalue(*--outer_));
        --*inner_;
        return *this;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base> &&
        equality_comparable<iterator_t<range_reference_t<Base>>>;

16    Effects: Equivalent to: return x.outer_ == y.outer_ && x.inner_ == y.inner_;

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(*x.inner_, *y.inner_)))
    requires indirectly_swappable<InnerIter>;

17    Effects: Equivalent to: ranges::iter_swap(*x.inner_, *y.inner_);

```

25.7.14.4 Class template `join_view::sentinel`

[range.join.sentinel]

```

namespace std::ranges {
    template<input_range V>
        requires view<V> && input_range<range_reference_t<V>>
    template<bool Const>
    struct join_view<V>::sentinel {
    private:
        using Parent = maybe_const<Const, join_view>;           // exposition only
        using Base = maybe_const<Const, V>;                     // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();              // exposition only
        constexpr explicit sentinel(Parent& parent);             // exposition only

    public:
        sentinel() = default;

        constexpr sentinel(sentinel<!Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        template<bool OtherConst>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
    };
}

```

```
constexpr explicit sentinel(Parent& parent);
```

1 *Effects:* Initializes `end_` with `ranges::end(parent.base_)`.

```
constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes `end_` with `std::move(s.end_)`.

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects:* Equivalent to: `return x.outer() == y.end_;`

25.7.15 Join with view

[range.join.with]

25.7.15.1 Overview

[range.join.with.overview]

1 `join_with_view` takes a view and a delimiter, and flattens the view, inserting every element of the delimiter in between elements of the view. The delimiter can be a single element or a view of elements.

2 The name `views::join_with` denotes a range adaptor object (25.7.2). Given subexpressions `E` and `F`, the expression `views::join_with(E, F)` is expression-equivalent to `join_with_view(E, F)`.

3 [Example 1:

```

vector<string> vs = {"the", "quick", "brown", "fox"};
for (char c : vs | views::join_with('-')) {
    cout << c;
}

```

// The above prints the-quick-brown-fox

— end example]

25.7.15.2 Class template `join_with_view`

[range.join.with.view]

```

namespace std::ranges {
    template<class R>
    concept bidirectional_common = bidirectional_range<R> && common_range<R>;           // exposition only

    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
            && view<Pattern>
            && concatible<range_reference_t<V>, Pattern>
    class join_with_view : public view_interface<join_with_view<V, Pattern>> {

```

```

using InnerRng = range_reference_t<V>;                                // exposition only

V base_ = V();                                                       // exposition only
non-propagating-cache<iterator_t<V>> outer_it_;                     // exposition only, present only
                                                                    // if forward_range<V> is false
non-propagating-cache<remove_cv_t<InnerRng>> inner_;                 // exposition only, present only
                                                                    // if is_reference_v<InnerRng> is false
Pattern pattern_ = Pattern();                                        // exposition only

// 25.7.15.3, class template join_with_view::iterator
template<bool Const> struct iterator;                                // exposition only

// 25.7.15.4, class template join_with_view::sentinel
template<bool Const> struct sentinel;                                // exposition only

public:
    join_with_view()
        requires default_initializable<V> && default_initializable<Pattern> = default;

    constexpr explicit join_with_view(V base, Pattern pattern);

    template<input_range R>
        requires constructible_from<V, views::all_t<R>> &&
             constructible_from<Pattern, single_view<range_value_t<InnerRng>>>
    constexpr explicit join_with_view(R&& r, range_value_t<InnerRng> e);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin() {
        if constexpr (forward_range<V>) {
            constexpr bool use_const =
                simple-view<V> && is_reference_v<InnerRng> && simple-view<Pattern>;
            return iterator<use_const>{*this, ranges::begin(base_)};
        }
        else {
            outer_it_ = ranges::begin(base_);
            return iterator<false>{*this};
        }
    }

    constexpr auto begin() const
        requires forward_range<const V> &&
             forward_range<const Pattern> &&
             is_reference_v<range_reference_t<const V>> &&
             input_range<range_reference_t<const V>> &&
             concatable<range_reference_t<const V>, const Pattern> {
        return iterator<true>{*this, ranges::begin(base_)};
    }

    constexpr auto end() {
        if constexpr (forward_range<V> &&
             is_reference_v<InnerRng> && forward_range<InnerRng> &&
             common_range<V> && common_range<InnerRng>)
            return iterator<simple-view<V> && simple-view<Pattern>>{*this, ranges::end(base_)};
        else
            return sentinel<simple-view<V> && simple-view<Pattern>>{*this};
    }

    constexpr auto end() const
        requires forward_range<const V> && forward_range<const Pattern> &&
             is_reference_v<range_reference_t<const V>> &&
             input_range<range_reference_t<const V>> &&
             concatable<range_reference_t<const V>, const Pattern> {
        using InnerConstRng = range_reference_t<const V>;

```

```

        if constexpr (forward_range<InnerConstRng> &&
                       common_range<const V> && common_range<InnerConstRng>)
            return iterator<true>{*this, ranges::end(base_)};
        else
            return sentinel<true>{*this};
    }
};

template<class R, class P>
    join_with_view(R&&, P&&) -> join_with_view<views::all_t<R>, views::all_t<P>>;

template<input_range R>
    join_with_view(R&&, range_value_t<range_reference_t<R>>)
        -> join_with_view<views::all_t<R>, single_view<range_value_t<range_reference_t<R>>>>;
}

```

```
constexpr explicit join_with_view(V base, Pattern pattern);
```

¹ *Effects:* Initializes *base_* with `std::move(base)` and *pattern_* with `std::move(pattern)`.

```

template<input_range R>
    requires constructible_from<V, views::all_t<R>> &&
             constructible_from<Pattern, single_view<range_value_t<InnerRng>>>
constexpr explicit join_with_view(R&& r, range_value_t<InnerRng> e);

```

² *Effects:* Initializes *base_* with `views::all(std::forward<R>(r))` and *pattern_* with `views::single(std::move(e))`.

25.7.15.3 Class template `join_with_view::iterator`

[range.join.with.iterator]

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
                 && view<Pattern> && concatable<range_reference_t<V>, Pattern>
    template<bool Const>
    class join_with_view<V, Pattern>::iterator {
        using Parent = maybe_const<Const, join_with_view>;
        using Base = maybe_const<Const, V>;
        using InnerBase = range_reference_t<Base>;
        using PatternBase = maybe_const<Const, Pattern>;

        using OuterIter = iterator_t<Base>;
        using InnerIter = iterator_t<InnerBase>;
        using PatternIter = iterator_t<PatternBase>;

        static constexpr bool ref-is-glvalue = is_reference_v<InnerBase>;

        Parent* parent_ = nullptr;
        OuterIter outer_it_ = OuterIter();
        variant<PatternIter, InnerIter> inner_it_;

        constexpr iterator(Parent& parent, OuterIter outer)
            requires forward_range<Base>;
        constexpr explicit iterator(Parent& parent)
            requires (!forward_range<Base>);
        constexpr OuterIter& outer();
        constexpr const OuterIter& outer() const;
        constexpr auto& update-inner();
        constexpr auto& get-inner();
        constexpr void satisfy();

    public:
        using iterator_concept = see below;
        using iterator_category = see below;
        using value_type = see below;

```

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only, present only

// if Base models forward_range

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// exposition only

// not always present

```

using difference_type = see below;

iterator() = default;
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter> &&
        convertible_to<iterator_t<Pattern>, PatternIter>;

constexpr decltype(auto) operator*() const;

constexpr iterator& operator++();
constexpr void operator++(int);
iterator operator++(int)
    requires ref-is-glvalue && forward_iterator<OuterIter> &&
        forward_iterator<InnerIter> = default;

constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional-common<InnerBase> && bidirectional-common<PatternBase>;
iterator operator--(int) = default;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base> &&
        equality_comparable<InnerIter>;

friend constexpr decltype(auto) iter_move(const iterator& x) {
    using rvalue_reference = common_reference_t<
        iter_rvalue_reference_t<InnerIter>,
        iter_rvalue_reference_t<PatternIter>>;
    return visit<rvalue_reference>(ranges::iter_move, x.inner_it_);
}

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    requires indirectly_swappable<InnerIter, PatternIter> {
    visit(ranges::iter_swap, x.inner_it_, y.inner_it_);
}
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *ref-is-glvalue* is true, *Base* models *bidirectional_range*, and *InnerBase* and *PatternBase* each model *bidirectional-common*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.2) — Otherwise, if *ref-is-glvalue* is true and *Base* and *InnerBase* each model *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² The member *typedef-name* *iterator_category* is declared if and only if *ref-is-glvalue* is true, and *Base* and *InnerBase* each model *forward_range*. In that case, *iterator::iterator_category* is defined as follows:

- (2.1) — Let *OUTERC* denote *iterator_traits<OuterIter>::iterator_category*, let *INNERC* denote *iterator_traits<InnerIter>::iterator_category*, and let *PATTERNC* denote *iterator_traits<PatternIter>::iterator_category*.
- (2.2) — If

is_reference_v<common_reference_t<iter_reference_t<InnerIter>,
 iter_reference_t<PatternIter>>>

 is false, *iterator_category* denotes *input_iterator_tag*.
- (2.3) — Otherwise, if *OUTERC*, *INNERC*, and *PATTERNC* each model *derived_from<bidirectional_iterator_tag>* and *InnerBase* and *PatternBase* each model *common_range*, *iterator_category* denotes *bidirectional_iterator_tag*.

- (2.4) — Otherwise, if *OUTERC*, *INNERC*, and *PATTERNC* each model `derived_from<forward_iterator_tag>`, `iterator_category` denotes `forward_iterator_tag`.
- (2.5) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

3 *iterator::value_type* denotes the type:

```
common_type_t<iter_value_t<InnerIter>, iter_value_t<PatternIter>>
```

4 *iterator::difference_type* denotes the type:

```
common_type_t<
    iter_difference_t<OuterIter>,
    iter_difference_t<InnerIter>,
    iter_difference_t<PatternIter>>
```

```
constexpr OuterIter& outer();
constexpr const OuterIter& outer() const;
```

5 *Returns:* *outer_it_* if *Base* models `forward_range`; otherwise, **parent_*→*outer_it_*.

```
constexpr auto& update-inner();
```

6 *Effects:* Equivalent to:

```
if constexpr (ref-is-glvalue)
    return as-lvalue(*outer());
else
    return parent_→inner_.emplace-deref(outer());
```

```
constexpr auto& get-inner();
```

7 *Effects:* Equivalent to:

```
if constexpr (ref-is-glvalue)
    return as-lvalue(*outer());
else
    return *parent_→inner_;
```

```
constexpr void satisfy();
```

8 *Effects:* Equivalent to:

```
while (true) {
    if (inner_it_.index() == 0) {
        if (std::get<0>(inner_it_) != ranges::end(parent_→pattern_))
            break;
        inner_it_.template emplace<1>(ranges::begin(update-inner()));
    } else {
        if (std::get<1>(inner_it_) != ranges::end(get-inner()))
            break;
        if (++outer() == ranges::end(parent_→base_)) {
            if constexpr (ref-is-glvalue)
                inner_it_.template emplace<0>();
            break;
        }
        inner_it_.template emplace<0>(ranges::begin(parent_→pattern_));
    }
}
```

[Note 1: `join_with_view` iterators use the *satisfy* function to skip over empty inner ranges. — end note]

```
constexpr iterator(Parent& parent, OuterIter outer)
```

```
requires forward_range<Base>;
```

```
constexpr explicit iterator(Parent& parent)
```

```
requires (!forward_range<Base>);
```

9 *Effects:* Initializes *parent_* with `addressof(parent)`. For the first overload, also initializes *outer_it_* with `std::move(outer)`. Then, equivalent to:

```
if (outer() != ranges::end(parent_→base_)) {
    inner_it_.template emplace<1>(ranges::begin(update-inner()));
```

```

        satisfy();
    }

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, OuterIter> &&
        convertible_to<iterator_t<InnerRng>, InnerIter> &&
        convertible_to<iterator_t<Pattern>, PatternIter>;
10    Effects: Initializes outer_it_ with std::move(i.outer_it_) and parent_ with i.parent_. Then,
    equivalent to:
        if (i.inner_it_.index() == 0)
            inner_it_.template emplace<0>(std::get<0>(std::move(i.inner_it_)));
        else
            inner_it_.template emplace<1>(std::get<1>(std::move(i.inner_it_)));
11    [Note 2: Const can only be true when Base models forward_range. — end note]

constexpr decltype(auto) operator*() const;
12    Effects: Equivalent to:
        using reference =
            common_reference_t<iter_reference_t<InnerIter>, iter_reference_t<PatternIter>>;
        return visit([](auto& it) -> reference { return *it; }, inner_it_);

constexpr iterator& operator++();
13    Effects: Equivalent to:
        visit([](auto& it){ ++it; }, inner_it_);
        satisfy();
        return *this;

constexpr void operator++(int);
14    Effects: Equivalent to ++*this.

constexpr iterator& operator--()
    requires ref-is-glvalue && bidirectional_range<Base> &&
        bidirectional-common<InnerBase> && bidirectional-common<PatternBase>;
15    Effects: Equivalent to:
        if (outer_it_ == ranges::end(parent_->base_)) {
            auto&& inner = *--outer_it_;
            inner_it_.template emplace<1>(ranges::end(inner));
        }

        while (true) {
            if (inner_it_.index() == 0) {
                auto& it = std::get<0>(inner_it_);
                if (it == ranges::begin(parent_->pattern_)) {
                    auto&& inner = *--outer_it_;
                    inner_it_.template emplace<1>(ranges::end(inner));
                } else {
                    break;
                }
            } else {
                auto& it = std::get<1>(inner_it_);
                auto&& inner = *outer_it_;
                if (it == ranges::begin(inner)) {
                    inner_it_.template emplace<0>(ranges::end(parent_->pattern_));
                } else {
                    break;
                }
            }
        }
        visit([](auto& it){ --it; }, inner_it_);
        return *this;

```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires ref-is-glvalue && forward_range<Base> &&
        equality_comparable<InnerIter>;
```

16 *Effects:* Equivalent to:

```
return x.outer_it_ == y.outer_it_ && x.inner_it_ == y.inner_it_;
```

25.7.15.4 Class template `join_with_view::sentinel` [range.join.with.sentinel]

```
namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && input_range<range_reference_t<V>>
            && view<Pattern> && concatable<range_reference_t<V>, Pattern>
    template<bool Const>
    class join_with_view<V, Pattern>::sentinel {
        using Parent = maybe-const<Const, join_with_view>; // exposition only
        using Base = maybe-const<Const, V>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only

        constexpr explicit sentinel(Parent& parent); // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel<!Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        template<bool OtherConst>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
    };
}
```

```
constexpr explicit sentinel(Parent& parent);
```

1 *Effects:* Initializes `end_` with `ranges::end(parent.base_)`.

```
constexpr sentinel(sentinel<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes `end_` with `std::move(s.end_)`.

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects:* Equivalent to: `return x.outer() == y.end_;`

25.7.16 Lazy split view [range.lazy.split]

25.7.16.1 Overview [range.lazy.split.overview]

1 `lazy_split_view` takes a view and a delimiter, and splits the view into subranges on the delimiter. The delimiter can be a single element or a view of elements.

2 The name `views::lazy_split` denotes a range adaptor object (25.7.2). Given subexpressions `E` and `F`, the expression `views::lazy_split(E, F)` is expression-equivalent to `lazy_split_view(E, F)`.

3 [Example 1:

```
string str{"the quick brown fox"};
for (auto word : str | views::lazy_split(' ')) {
    for (char ch : word)
        cout << ch;
    cout << '*';
}
```

```
// The above prints the*quick*brown*fox*
```

— end example]

25.7.16.2 Class template `lazy_split_view`

[range.lazy.split.view]

```

namespace std::ranges {
    template<auto> struct require-constant; // exposition only

    template<class R>
    concept tiny-range = // exposition only
        sized_range<R> &&
        requires { typename require-constant<remove_reference_t<R>::size()>; } &&
        (remove_reference_t<R>::size() <= 1);

    template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
        indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
        (forward_range<V> || tiny-range<Pattern>)
    class lazy_split_view : public view_interface<lazy_split_view<V, Pattern>> {
    private:
        V base_ = V(); // exposition only
        Pattern pattern_ = Pattern(); // exposition only

        non-propagating-cache<iterator_t<V>> current_; // exposition only, present only
        // if forward_range<V> is false

        // 25.7.16.3, class template lazy_split_view::outer-iterator
        template<bool> struct outer-iterator; // exposition only

        // 25.7.16.5, class template lazy_split_view::inner-iterator
        template<bool> struct inner-iterator; // exposition only

    public:
        lazy_split_view()
            requires default_initializable<V> && default_initializable<Pattern> = default;
        constexpr explicit lazy_split_view(V base, Pattern pattern);

        template<input_range R>
        requires constructible_from<V, views::all_t<R>> &&
            constructible_from<Pattern, single_view<range_value_t<R>>>
        constexpr explicit lazy_split_view(R&& r, range_value_t<R> e);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() {
            if constexpr (forward_range<V>) {
                return outer-iterator<simple-view<V> && simple-view<Pattern>>
                    {*this, ranges::begin(base_)};
            } else {
                current_ = ranges::begin(base_);
                return outer-iterator<false>{*this};
            }
        }

        constexpr auto begin() const requires forward_range<V> && forward_range<const V> &&
            forward_range<const Pattern> {
            return outer-iterator<true>{*this, ranges::begin(base_)};
        }

        constexpr auto end() requires forward_range<V> && common_range<V> {
            return outer-iterator<simple-view<V> && simple-view<Pattern>>
                {*this, ranges::end(base_)};
        }
    }
}

```

```
constexpr auto end() const {
    if constexpr (forward_range<V> && forward_range<const V> && common_range<const V> &&
        forward_range<const Pattern>)
        return outer-iterator<true>{*this, ranges::end(base_)};
    else
        return default_sentinel;
}
};
```

```
template<class R, class P>
    lazy_split_view(R&&, P&&) -> lazy_split_view<views::all_t<R>, views::all_t<P>>;

template<input_range R>
    lazy_split_view(R&&, range_value_t<R>)
        -> lazy_split_view<views::all_t<R>, single_view<range_value_t<R>>>;
}
```

```
constexpr explicit lazy_split_view(V base, Pattern pattern);
```

¹ *Effects:* Initializes *base_* with `std::move(base)`, and *pattern_* with `std::move(pattern)`.

```
template<input_range R>
    requires constructible_from<V, views::all_t<R>> &&
        constructible_from<Pattern, single_view<range_value_t<R>>>
constexpr explicit lazy_split_view(R&& r, range_value_t<R> e);
```

² *Effects:* Initializes *base_* with `views::all(std::forward<R>(r))`, and *pattern_* with `views::single(std::move(e))`.

25.7.16.3 Class template `lazy_split_view::outer-iterator` [range.lazy.split.outer]

```
namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
                (forward_range<V> || tiny_range<Pattern>)
    template<bool Const>
    struct lazy_split_view<V, Pattern>::outer-iterator {
    private:
        using Parent = maybe-const<Const, lazy_split_view>;           // exposition only
        using Base = maybe-const<Const, V>;                           // exposition only
        Parent* parent_ = nullptr;                                     // exposition only

        iterator_t<Base> current_ = iterator_t<Base>();               // exposition only, present only
                                                                    // if V models forward_range

        bool trailing_empty_ = false;                                 // exposition only
        constexpr explicit outer-iterator(Parent& parent)             // exposition only
            requires (!forward_range<Base>);
        constexpr outer-iterator(Parent& parent, iterator_t<Base> current) // exposition only
            requires forward_range<Base>;

    public:
        using iterator_concept =
            conditional_t<forward_range<Base>, forward_iterator_tag, input_iterator_tag>;

        using iterator_category = input_iterator_tag;                 // present only if Base
                                                                    // models forward_range

        // 25.7.16.4, class lazy_split_view::outer-iterator::value_type
        struct value_type;
        using difference_type = range_difference_t<Base>;

        outer-iterator() = default;
        constexpr outer-iterator(outer-iterator<!Const> i)
            requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

```

constexpr value_type operator*() const;

constexpr outer-iterator& operator++();
constexpr void operator++(int) {
    ++*this;
}
outer-iterator operator++(int) requires forward_range<Base> = default;

friend constexpr bool operator==(const outer-iterator& x, const outer-iterator& y)
    requires forward_range<Base>;

friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);
};
}

```

- ¹ Many of the specifications in 25.7.16 refer to the notional member *current* of *outer-iterator*. *current* is equivalent to *current_* if *V* models *forward_range*, and **parent_->current_* otherwise.

```

constexpr explicit outer-iterator(Parent& parent)
    requires (!forward_range<Base>);

```

- ² *Effects*: Initializes *parent_* with *addressof*(*parent*).

```

constexpr outer-iterator(Parent& parent, iterator_t<Base> current)
    requires forward_range<Base>;

```

- ³ *Effects*: Initializes *parent_* with *addressof*(*parent*) and *current_* with *std::move*(*current*).

```

constexpr outer-iterator(outer-iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

```

- ⁴ *Effects*: Initializes *parent_* with *i.parent_*, *current_* with *std::move*(*i.current_*), and *trailing_empty_* with *i.trailing_empty_*.

```

constexpr value_type operator*() const;

```

- ⁵ *Effects*: Equivalent to: *return value_type{*this};*

```

constexpr outer-iterator& operator++();

```

- ⁶ *Effects*: Equivalent to:

```

    const auto end = ranges::end(parent_->base_);
    if (current == end) {
        trailing_empty_ = false;
        return *this;
    }
    const auto [pbegin, pend] = subrange{parent_->pattern_};
    if (pbegin == pend) ++current;
    else if constexpr (tiny-range<Pattern>) {
        current = ranges::find(std::move(current), end, *pbegin);
        if (current != end) {
            ++current;
            if (current == end)
                trailing_empty_ = true;
        }
        else if constexpr (!forward_range<V>)
            trailing_empty_ = true;
    }
}
else {
    do {
        auto [b, p] = ranges::mismatch(current, end, pbegin, pend);
        if (p == pend) {
            current = b;
            if (current == end)
                trailing_empty_ = true;
            break; // The pattern matched; skip it
        }
    }
}

```

```

        } while (++current != end);
    }
    return *this;

friend constexpr bool operator==(const outer-iterator& x, const outer-iterator& y)
    requires forward_range<Base>;

```

7 *Effects:* Equivalent to:

```
        return x.current_ == y.current_ && x.trailing_empty_ == y.trailing_empty_;
```

```
friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);
```

8 *Effects:* Equivalent to:

```
        return x.current == ranges::end(x.parent_->base_) && !x.trailing_empty_;
```

25.7.16.4 Class lazy_split_view::outer-iterator::value_type [range.lazy.split.outer.value]

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
                (forward_range<V> || tiny_range<Pattern>)
    template<bool Const>
    struct lazy_split_view<V, Pattern>::outer-iterator<Const>::value_type
        : view_interface<value_type> {
    private:
        outer-iterator i_ = outer-iterator(); // exposition only

        constexpr explicit value_type(outer-iterator i); // exposition only

    public:
        constexpr inner-iterator<Const> begin() const;
        constexpr default_sentinel_t end() const noexcept;
    };
}

```

```
constexpr explicit value_type(outer-iterator i);
```

1 *Effects:* Initializes *i_* with `std::move(i)`.

```
constexpr inner-iterator<Const> begin() const;
```

2 *Effects:* Equivalent to: `return inner-iterator<Const>{i_};`

```
constexpr default_sentinel_t end() const noexcept;
```

3 *Effects:* Equivalent to: `return default_sentinel;`

25.7.16.5 Class template lazy_split_view::inner-iterator [range.lazy.split.inner]

```

namespace std::ranges {
    template<input_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to> &&
                (forward_range<V> || tiny_range<Pattern>)
    template<bool Const>
    struct lazy_split_view<V, Pattern>::inner-iterator {
    private:
        using Base = maybe-const<Const, V>; // exposition only
        outer-iterator<Const> i_ = outer-iterator<Const>(); // exposition only
        bool incremented_ = false; // exposition only
        constexpr explicit inner-iterator(outer-iterator<Const> i); // exposition only

    public:
        using iterator_concept = outer-iterator<Const>::iterator_concept;

        using iterator_category = see below; // present only if Base
    };
}

```

```

// models forward_range
using value_type      = range_value_t<Base>;
using difference_type = range_difference_t<Base>;

inner-iterator() = default;

constexpr const iterator_t<Base>& base() const & noexcept;
constexpr iterator_t<Base> base() && requires forward_range<V>;

constexpr decltype(auto) operator*() const { return *i_.current; }

constexpr inner-iterator& operator++();
constexpr void operator++(int) {
    ++*this;
}
inner-iterator operator++(int) requires forward_range<Base> = default;

friend constexpr bool operator==(const inner-iterator& x, const inner-iterator& y)
    requires forward_range<Base>;

friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);

friend constexpr decltype(auto) iter_move(const inner-iterator& i)
    noexcept(noexcept(ranges::iter_move(i.i_.current))) {
    return ranges::iter_move(i.i_.current);
}

friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.i_.current, y.i_.current)))
    requires indirectly_swappable<iterator_t<Base>>;
};
}

```

¹ If *Base* does not model `forward_range` there is no member `iterator_category`. Otherwise, the *typedef*-name `iterator_category` denotes:

- (1.1) — `forward_iterator_tag` if `iterator_traits<iterator_t<Base>>::iterator_category` models `derived_from<forward_iterator_tag>`;
- (1.2) — otherwise, `iterator_traits<iterator_t<Base>>::iterator_category`.

```
constexpr explicit inner-iterator(outer-iterator<Const> i);
```

² *Effects*: Initializes `i_` with `std::move(i)`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

³ *Effects*: Equivalent to: `return i_.current;`

```
constexpr iterator_t<Base> base() && requires forward_range<V>;
```

⁴ *Effects*: Equivalent to: `return std::move(i_.current);`

```
constexpr inner-iterator& operator++();
```

⁵ *Effects*: Equivalent to:

```

    incremented_ = true;
    if constexpr (!forward_range<Base>) {
        if constexpr (Pattern::size() == 0) {
            return *this;
        }
    }
    ++i_.current;
    return *this;

```

```
friend constexpr bool operator==(const inner-iterator& x, const inner-iterator& y)
    requires forward_range<Base>;
```

6 *Effects:* Equivalent to: `return x.i_.current == y.i_.current;`

```
friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);
```

7 *Effects:* Equivalent to:

```
    auto [pcur, pend] = subrange{x.i_.parent_->pattern_};
    auto end = ranges::end(x.i_.parent_->base_);
    if constexpr (tiny-range<Pattern>) {
        const auto & cur = x.i_.current;
        if (cur == end) return true;
        if (pcur == pend) return x.incremented_;
        return *cur == *pcur;
    } else {
        auto cur = x.i_.current;
        if (cur == end) return true;
        if (pcur == pend) return x.incremented_;
        do {
            if (*cur != *pcur) return false;
            if (++pcur == pend) return true;
        } while (++cur != end);
        return false;
    }
```

```
friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.i_.current, y.i_.current)))
    requires indirectly_swappable<iterator_t<Base>>;
```

8 *Effects:* Equivalent to `ranges::iter_swap(x.i_.current, y.i_.current).`

25.7.17 Split view

[range.split]

25.7.17.1 Overview

[range.split.overview]

1 `split_view` takes a view and a delimiter, and splits the view into `subranges` on the delimiter. The delimiter can be a single element or a view of elements.

2 The name `views::split` denotes a range adaptor object (25.7.2). Given subexpressions `E` and `F`, the expression `views::split(E, F)` is expression-equivalent to `split_view(E, F)`.

3 [Example 1:

```
    string str{"the quick brown fox"};
    for (auto word : views::split(str, ' ')) {
        cout << string_view(word) << '*';
    }
```

*// The above prints the*quick*brown*fox**

—end example]

25.7.17.2 Class template `split_view`

[range.split.view]

```
namespace std::ranges {
    template<forward_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&
            indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
        class split_view : public view_interface<split_view<V, Pattern>> {
        private:
            V base_ = V(); // exposition only
            Pattern pattern_ = Pattern(); // exposition only

            // 25.7.17.3, class split_view::iterator
            struct iterator; // exposition only

            // 25.7.17.4, class split_view::sentinel
            struct sentinel; // exposition only
        };
```

```

public:
    split_view()
        requires default_initializable<V> && default_initializable<Pattern> = default;
    constexpr explicit split_view(V base, Pattern pattern);

    template<forward_range R>
        requires constructible_from<V, views::all_t<R>> &&
            constructible_from<Pattern, single_view<range_value_t<R>>>
    constexpr explicit split_view(R&& r, range_value_t<R> e);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr iterator begin();

    constexpr auto end() {
        if constexpr (common_range<V>) {
            return iterator{*this, ranges::end(base_), {}};
        } else {
            return sentinel{*this};
        }
    }

    constexpr subrange<iterator_t<V>> find_next(iterator_t<V>); // exposition only
};

template<class R, class P>
    split_view(R&&, P&&) -> split_view<views::all_t<R>, views::all_t<P>>;

template<forward_range R>
    split_view(R&&, range_value_t<R>)
        -> split_view<views::all_t<R>, single_view<range_value_t<R>>>;
}

```

```
constexpr explicit split_view(V base, Pattern pattern);
```

1 *Effects:* Initializes *base_* with `std::move(base)`, and *pattern_* with `std::move(pattern)`.

```

template<forward_range R>
    requires constructible_from<V, views::all_t<R>> &&
        constructible_from<Pattern, single_view<range_value_t<R>>>
constexpr explicit split_view(R&& r, range_value_t<R> e);

```

2 *Effects:* Initializes *base_* with `views::all(std::forward<R>(r))`, and *pattern_* with `views::single(std::move(e))`.

```
constexpr iterator begin();
```

3 *Returns:* `{*this, ranges::begin(base_), find_next(ranges::begin(base_))}`.

4 *Remarks:* In order to provide the amortized constant time complexity required by the `range` concept, this function caches the result within the `split_view` for use on subsequent calls.

```
constexpr subrange<iterator_t<V>> find_next(iterator_t<V> it);
```

5 *Effects:* Equivalent to:

```

    auto [b, e] = ranges::search(subrange(it, ranges::end(base_)), pattern_);
    if (b != ranges::end(base_) && ranges::empty(pattern_)) {
        ++b;
        ++e;
    }
    return {b, e};

```

25.7.17.3 Class `split_view::iterator`

[range.split.iterator]

```

namespace std::ranges {
    template<forward_range V, forward_range Pattern>
        requires view<V> && view<Pattern> &&

```

```

        indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
class split_view<V, Pattern>::iterator {
private:
    split_view* parent_ = nullptr;           // exposition only
    iterator_t<V> cur_ = iterator_t<V>();    // exposition only
    subrange<iterator_t<V>> next_ = subrange<iterator_t<V>>(); // exposition only
    bool trailing_empty_ = false;           // exposition only
    constexpr iterator(split_view& parent, iterator_t<V> current, // exposition only
                        subrange<iterator_t<V>> next);

public:
    using iterator_concept = forward_iterator_tag;
    using iterator_category = input_iterator_tag;
    using value_type = subrange<iterator_t<V>>;
    using difference_type = range_difference_t<V>;

    iterator() = default;

    constexpr iterator_t<V> base() const;
    constexpr value_type operator*() const;

    constexpr iterator& operator++();
    iterator operator++(int) = default;

    friend constexpr bool operator==(const iterator& x, const iterator& y);
};

constexpr iterator(split_view& parent, iterator_t<V> current, subrange<iterator_t<V>> next);
1      Effects: Initializes parent_ with addressof(parent), cur_ with std::move(current), and next_
      with std::move(next).

constexpr iterator_t<V> base() const;
2      Effects: Equivalent to: return cur_;

constexpr value_type operator*() const;
3      Effects: Equivalent to: return {cur_, next_.begin()};

constexpr iterator& operator++();
4      Effects: Equivalent to:
        cur_ = next_.begin();
        if (cur_ != ranges::end(parent_->base_)) {
            cur_ = next_.end();
            if (cur_ == ranges::end(parent_->base_)) {
                trailing_empty_ = true;
                next_ = {cur_, cur_};
            } else {
                next_ = parent_->find-next(cur_);
            }
        } else {
            trailing_empty_ = false;
        }
        return *this;

friend constexpr bool operator==(const iterator& x, const iterator& y);
5      Effects: Equivalent to:
        return x.cur_ == y.cur_ && x.trailing_empty_ == y.trailing_empty_;

```

25.7.17.4 Class `split_view::sentinel`

[range.split.sentinel]

```

namespace std::ranges {
    template<forward_range V, forward_range Pattern>

```



```

    requires view<V> && view<Pattern> &&
        indirectly_comparable<iterator_t<V>, iterator_t<Pattern>, ranges::equal_to>
    struct split_view<V, Pattern>::sentinel {
    private:
        sentinel_t<V> end_ = sentinel_t<V>();           // exposition only
        constexpr explicit sentinel(split_view& parent); // exposition only

    public:
        sentinel() = default;

        friend constexpr bool operator==(const iterator& x, const sentinel& y);
    };
}

```

```
constexpr explicit sentinel(split_view& parent);
```

¹ *Effects:* Initializes *end_* with *ranges::end(parent.base_)*.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

² *Effects:* Equivalent to: *return x.cur_ == y.end_ && !x.trailing_empty_;*

25.7.18 Concat view

[range.concat]

25.7.18.1 Overview

[range.concat.overview]

¹ *concat_view* presents a view that concatenates all the underlying ranges.

² The name *views::concat* denotes a customization point object (16.3.3.3.5). Given a pack of subexpressions *Es...*, the expression *views::concat(Es...)* is expression-equivalent to

(2.1) — *views::all(Es...)* if *Es* is a pack with only one element whose type models *input_range*,

(2.2) — otherwise, *concat_view(Es...)*.

[Example 1:

```

vector<int> v1{1, 2, 3}, v2{4, 5}, v3{};
array a{6, 7, 8};
auto s = views::single(9);
for (auto&& i : views::concat(v1, v2, v3, a, s)) {
    print("{} ", i);           // prints 1 2 3 4 5 6 7 8 9
}

```

— end example]

25.7.18.2 Class template *concat_view*

[range.concat.view]

```

namespace std::ranges {
    template<class... Rs>
        using concat-reference-t = common_reference_t<range_reference_t<Rs>...>;           // exposition only
    template<class... Rs>
        using concat-value-t = common_type_t<range_value_t<Rs>...>;                     // exposition only
    template<class... Rs>
        using concat-rvalue-reference-t =
            common_reference_t<range_rvalue_reference_t<Rs>...>;                         // exposition only

    template<class... Rs>
        concept concat-indirectly-readable = see below;                               // exposition only
    template<class... Rs>
        concept concatable = see below;                                                // exposition only
    template<bool Const, class... Rs>
        concept concat-is-random-access = see below;                                   // exposition only
    template<bool Const, class... Rs>
        concept concat-is-bidirectional = see below;                                  // exposition only

    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
            concatable<Views...>
    class concat_view : public view_interface<concat_view<Views...>> {

```

```

tuple<Views...> views_; // exposition only

// 25.7.18.3, class template concat_view::iterator
template<bool> class iterator; // exposition only

public:
constexpr concat_view() = default;
constexpr explicit concat_view(Views... views);

constexpr iterator<false> begin() requires (!simple-view<Views> && ...);
constexpr iterator<true> begin() const
    requires (range<const Views> && ...) && concatable<const Views...>;

constexpr auto end() requires (!simple-view<Views> && ...);
constexpr auto end() const
    requires (range<const Views> && ...) && concatable<const Views...>;

constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
constexpr auto reserve_hint() requires (approximately_sized_range<Views> && ...);
constexpr auto reserve_hint() const requires (approximately_sized_range<const Views> && ...);
};

template<class... R>
    concat_view(R&&...) -> concat_view<views::all_t<R>...>;
}

```

```

template<class... Rs>
    concept concat-indirectly-readable = see below; // exposition only

```

1 The exposition-only *concat-indirectly-readable* concept is equivalent to:

```

template<class Ref, class RRef, class It>
    concept concat-indirectly-readable-impl = // exposition only
        requires (const It it) {
            { *it } -> convertible_to<Ref>;
            { ranges::iter_move(it) } -> convertible_to<RRef>;
        };

```

```

template<class... Rs>
    concept concat-indirectly-readable = // exposition only
        common_reference_with<concat-reference-t<Rs...>&&,
                               concat-value-t<Rs...>&& &&
        common_reference_with<concat-reference-t<Rs...>&&,
                               concat-rvalue-reference-t<Rs...>&& &&
        common_reference_with<concat-rvalue-reference-t<Rs...>&&,
                               concat-value-t<Rs...> const&& &&
        (concat-indirectly-readable-impl<concat-reference-t<Rs...>,
          concat-rvalue-reference-t<Rs...>,
          iterator_t<Rs>> && ...);

```

```

template<class... Rs>
    concept concatable = see below; // exposition only

```

2 The exposition-only *concatable* concept is equivalent to:

```

template<class... Rs>
    concept concatable = requires { // exposition only
        typename concat-reference-t<Rs...>;
        typename concat-value-t<Rs...>;
        typename concat-rvalue-reference-t<Rs...>;
    } && concat-indirectly-readable<Rs...>;

```

```

template<bool Const, class... Rs>
    concept concat-is-random-access = see below;                                // exposition only
3    Let Fs be the pack that consists of all elements of Rs except the last element, then concat-is-random-access is equivalent to:

        template<bool Const, class... Rs>
            concept concat-is-random-access =                                // exposition only
                all-random-access<Const, Rs...> &&
                (common_range<maybe-const<Const, Fs>> && ...);

template<bool Const, class... Rs>
    concept concat-is-bidirectional = see below;                            // exposition only
4    Let Fs be the pack that consists of all elements of Rs except the last element, then concat-is-bidirectional is equivalent to:

        template<bool Const, class... Rs>
            concept concat-is-bidirectional =                                // exposition only
                all-bidirectional<Const, Rs...> &&
                (common_range<maybe-const<Const, Fs>> && ...);

constexpr explicit concat_view(Views... views);

5    Effects: Initializes views_ with std::move(views)...

constexpr iterator<false> begin() requires (!(simple-view<Views> && ...));
constexpr iterator<true> begin() const
    requires (range<const Views> && ...) && concatable<const Views...>;

6    Effects: Let is-const be true for the const-qualified overload, and false otherwise. Equivalent to:

        iterator<is-const> it(this, in_place_index<0>, ranges::begin(std::get<0>(views_)));
        it.template satisfy<0>();
        return it;

constexpr auto end() requires (!(simple-view<Views> && ...));
constexpr auto end() const
    requires (range<const Views> && ...) && concatable<const Views...>;

7    Effects: Let is-const be true for the const-qualified overload, and false otherwise. Equivalent to:

        constexpr auto N = sizeof...(Views);
        if constexpr (all-forward<is-const, Views...> &&
            common_range<maybe-const<is-const, Views...[N - 1]>>) {
            return iterator<is-const>(this, in_place_index<N - 1>,
                ranges::end(std::get<N - 1>(views_)));
        } else {
            return default_sentinel;
        }

constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);

8    Effects: Equivalent to:

        return apply(
            [](auto... sizes) {
                using CT = make-unsigned-like-t<common_type_t<decltype(sizes)...>>;
                return (CT(sizes) + ...);
            },
            tuple_transform(ranges::size, views_));

constexpr auto reserve_hint() requires (approximately_sized_range<Views> && ...);
constexpr auto reserve_hint() const requires (approximately_sized_range<const Views> && ...);

9    Effects: Equivalent to:

        return apply(
            [](auto... sizes) {
                using CT = make-unsigned-like-t<common_type_t<decltype(sizes)...>>;
                return (CT(sizes) + ...);
            },
            tuple_transform(ranges::size, views_));

```

```

},
tuple-transform(ranges::reserve_hint, views_));

```

25.7.18.3 Class concat_view::iterator

[range.concat.iterator]

```

namespace std::ranges {
    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) &&
            concatable<Views...>
    template<bool Const>
    class concat_view<Views...>::iterator {
    public:
        using iterator_category = see below; // not always present
        using iterator_concept = see below;
        using value_type = concat-value-t<maybe-const<Const, Views>...>;
        using difference_type = common_type_t<range-difference-t<maybe-const<Const, Views>>...>;

    private:
        using base-iter = // exposition only
            variant<iterator_t<maybe-const<Const, Views>>...>;

        maybe-const<Const, concat_view>* parent_ = nullptr; // exposition only
        base-iter it_; // exposition only

        template<size_t N>
            constexpr void satisfy(); // exposition only
        template<size_t N>
            constexpr void prev(); // exposition only

        template<size_t N>
            constexpr void advance-fwd(difference_type offset, // exposition only
                                       difference_type steps);

        template<size_t N>
            constexpr void advance-bwd(difference_type offset, // exposition only
                                       difference_type steps);

        template<class... Args>
            constexpr explicit iterator(maybe-const<Const, concat_view>* parent, // exposition only
                                       Args&&... args)
                requires constructible_from<base-iter, Args&&...>;

    public:
        iterator() = default;

        constexpr iterator(iterator<!Const> i)
            requires Const && (convertible_to<iterator_t<Views>, iterator_t<const Views>> && ...);

        constexpr decltype(auto) operator*() const;
        constexpr iterator& operator++();
        constexpr void operator++(int);
        iterator operator++(int)
            requires all-forward<Const, Views...> = default;
        constexpr iterator& operator--()
            requires concat-is-bidirectional<Const, Views...>;
        iterator operator--(int) = default;
        constexpr iterator& operator+=(difference_type n)
            requires concat-is-random-access<Const, Views...>;
        constexpr iterator& operator-=(difference_type n)
            requires concat-is-random-access<Const, Views...>;
        iterator operator--(int) = default;

        friend constexpr bool operator==(const iterator& x, const iterator& y)
            requires (equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...);
        friend constexpr bool operator==(const iterator& it, default_sentinel_t);

```

```

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires (all-random-access<Const, Views...> &&
        (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...));
friend constexpr iterator operator+(const iterator& it, difference_type n)
    requires concat-is-random-access<Const, Views...>;
friend constexpr iterator operator+(difference_type n, const iterator& it)
    requires concat-is-random-access<Const, Views...>;
friend constexpr iterator operator-(const iterator& it, difference_type n)
    requires concat-is-random-access<Const, Views...>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires concat-is-random-access<Const, Views...>;
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
    requires see below;
friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
    requires see below;
friend constexpr decltype(auto) iter_move(const iterator& it) noexcept(see below);
friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see below)
    requires see below;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *concat-is-random-access*<Const, Views...> is modeled, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *concat-is-bidirectional*<Const, Views...> is modeled, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *all-forward*<Const, Views...> is modeled, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² The member *typedef-name* *iterator_category* is declared if and only if *all-forward*<Const, Views...> is modeled. In that case, *iterator::iterator_category* is defined as follows:

- (2.1) — If *is_reference_v*<*concat-reference-t*<maybe-const<Const, Views>...>> is false, then *iterator_category* denotes *input_iterator_tag*.
- (2.2) — Otherwise, let Cs denote the pack of types *iterator_traits*<iterator_t<maybe-const<Const, Views>>>::*iterator_category*....
 - (2.2.1) — If (derived_from<Cs, *random_access_iterator_tag*> && ...) && *concat-is-random-access*<Const, Views...> is true, *iterator_category* denotes *random_access_iterator_tag*.
 - (2.2.2) — Otherwise, if (derived_from<Cs, *bidirectional_iterator_tag*> && ...) && *concat-is-bidirectional*<Const, Views...> is true, *iterator_category* denotes *bidirectional_iterator_tag*.
 - (2.2.3) — Otherwise, if (derived_from<Cs, *forward_iterator_tag*> && ...) is true, *iterator_category* denotes *forward_iterator_tag*.
 - (2.2.4) — Otherwise, *iterator_category* denotes *input_iterator_tag*.

```

template<size_t N>
constexpr void satisfy();

```

³ *Effects*: Equivalent to:

```

if constexpr (N < (sizeof...(Views) - 1)) {
    if (std::get<N>(it_) == ranges::end(std::get<N>(parent_->views_))) {

```

```

        it_.template emplace<N + 1>(ranges::begin(std::get<N + 1>(parent_>views_)));
        satisfy<N + 1>();
    }
}

template<size_t N>
constexpr void prev();
4    Effects: Equivalent to:
        if constexpr (N == 0) {
            --std::get<0>(it_);
        } else {
            if (std::get<N>(it_) == ranges::begin(std::get<N>(parent_>views_))) {
                it_.template emplace<N - 1>(ranges::end(std::get<N - 1>(parent_>views_)));
                prev<N - 1>();
            } else {
                --std::get<N>(it_);
            }
        }

template<size_t N>
constexpr void advance-fwd(difference_type offset, difference_type steps);
5    Effects: Equivalent to:
        using underlying_diff_type = iter_difference_t<variant_alternative_t<N, base-iter>>;
        if constexpr (N == sizeof...(Views) - 1) {
            std::get<N>(it_) += static_cast<underlying_diff_type>(steps);
        } else {
            auto n_size = ranges::distance(std::get<N>(parent_>views_));
            if (offset + steps < n_size) {
                std::get<N>(it_) += static_cast<underlying_diff_type>(steps);
            } else {
                it_.template emplace<N + 1>(ranges::begin(std::get<N + 1>(parent_>views_)));
                advance-fwd<N + 1>(0, offset + steps - n_size);
            }
        }

template<size_t N>
constexpr void advance-bwd(difference_type offset, difference_type steps);
6    Effects: Equivalent to:
        using underlying_diff_type = iter_difference_t<variant_alternative_t<N, base-iter>>;
        if constexpr (N == 0) {
            std::get<N>(it_) -= static_cast<underlying_diff_type>(steps);
        } else {
            if (offset >= steps) {
                std::get<N>(it_) -= static_cast<underlying_diff_type>(steps);
            } else {
                auto prev_size = ranges::distance(std::get<N - 1>(parent_>views_));
                it_.template emplace<N - 1>(ranges::end(std::get<N - 1>(parent_>views_)));
                advance-bwd<N - 1>(prev_size, steps - offset);
            }
        }

template<class... Args>
constexpr explicit iterator(maybe-const<Const, concat_view>* parent,
                           Args&&... args)
    requires constructible_from<base-iter, Args&&...>;
7    Effects: Initializes parent_ with parent, and initializes it_ with std::forward<Args>(args)....

constexpr iterator(iterator<!Const> it)
    requires Const &&
        (convertible_to<iterator_t<Views>, iterator_t<const Views>> && ...);
8    Preconditions: it.it_.valueless_by_exception() is false.

```

9 *Effects:* Initializes *parent_* with *it.parent_*, and let *i* be *it.it_.index()*, initializes *it_* with *base-iter*(*in_place_index*<*i*>, *std::get*<*i*>(*std::move*(*it.it_*))).

```
constexpr decltype(auto) operator*() const;
```

10 *Preconditions:* *it_.valueless_by_exception()* is false.

11 *Effects:* Equivalent to:

```
using reference = concat-reference-t<maybe-const<Const, Views>...>;
return std::visit([](auto&& it) -> reference { return *it; },
                 it_);
```

```
constexpr iterator& operator++();
```

12 *Preconditions:* *it_.valueless_by_exception()* is false.

13 *Effects:* Let *i* be *it_.index()*. Equivalent to:

```
++std::get<i>(it_);
satisfy<i>();
return *this;
```

```
constexpr void operator++(int);
```

14 *Effects:* Equivalent to:

```
+++this;
```

```
constexpr iterator& operator--()
requires concat-is-bidirectional<Const, Views...>;
```

15 *Preconditions:* *it_.valueless_by_exception()* is false.

16 *Effects:* Let *i* be *it_.index()*. Equivalent to:

```
prev<i>();
return *this;
```

```
constexpr iterator& operator+=(difference_type n)
requires concat-is-random-access<Const, Views...>;
```

17 *Preconditions:* *it_.valueless_by_exception()* is false.

18 *Effects:* Let *i* be *it_.index()*. Equivalent to:

```
if (n > 0) {
    advance-fwd<i>(std::get<i>(it_) - ranges::begin(std::get<i>(parent_->views_)), n);
} else if (n < 0) {
    advance-bwd<i>(std::get<i>(it_) - ranges::begin(std::get<i>(parent_->views_)), -n);
}
return *this;
```

```
constexpr iterator& operator-=(difference_type n)
requires concat-is-random-access<Const, Views...>;
```

19 *Effects:* Equivalent to:

```
*this += -n;
return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
requires concat-is-random-access<Const, Views...>;
```

20 *Effects:* Equivalent to:

```
return *((*this) + n);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
requires equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...>;
```

21 *Preconditions:* *x.it_.valueless_by_exception()* and *y.it_.valueless_by_exception()* are each false.

22 *Effects:* Equivalent to:

```
return x.it_ == y.it_;
```

```
friend constexpr bool operator==(const iterator& it, default_sentinel_t);
```

23 *Preconditions:* `it.it_.valueless_by_exception()` is false.

24 *Effects:* Equivalent to:

```
    constexpr auto last_idx = sizeof...(Views) - 1;
    return it.it_.index() == last_idx &&
        std::get<last_idx>(it.it_) == ranges::end(std::get<last_idx>(it.parent_->views_));
```

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
```

```
    requires all-random-access<Const, Views...>;
```

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
```

```
    requires all-random-access<Const, Views...>;
```

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
```

```
    requires all-random-access<Const, Views...>;
```

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
```

```
    requires all-random-access<Const, Views...>;
```

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
```

```
    requires (all-random-access<Const, Views...> &&
        (three_way_comparable<iterator_t<maybe-const<Const, Views>>> && ...));
```

25 *Preconditions:* `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each false.

26 Let *op* be the operator.

27 *Effects:* Equivalent to:

```
    return x.it_ op y.it_;
```

```
friend constexpr iterator operator+(const iterator& it, difference_type n)
```

```
    requires concat-is-random-access<Const, Views...>;
```

28 *Effects:* Equivalent to:

```
    auto temp = it;
    temp += n;
    return temp;
```

```
friend constexpr iterator operator+(difference_type n, const iterator& it)
```

```
    requires concat-is-random-access<Const, Views...>;
```

29 *Effects:* Equivalent to:

```
    return it + n;
```

```
friend constexpr iterator operator-(const iterator& it, difference_type n)
```

```
    requires concat-is-random-access<Const, Views...>;
```

30 *Effects:* Equivalent to:

```
    auto temp = it;
    temp -= n;
    return temp;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
```

```
    requires concat-is-random-access<Const, Views...>;
```

31 *Preconditions:* `x.it_.valueless_by_exception()` and `y.it_.valueless_by_exception()` are each false.

32 *Effects:* Let i_x denote `x.it_.index()` and i_y denote `y.it_.index()`.

(32.1) — If $i_x > i_y$, let d_y be `ranges::distance(std::get< $i_yi_y,  $d_x$  be ranges::distance(ranges::begin(std::get< $i_xi_x. Let  $s$  denote the sum of the sizes of all the ranges std::get< $i for every integer  $i$  in the range  $[i_y + 1, i_x)$  if there is any, and 0 otherwise, of type difference_type, equivalent to:$$$`

```
    return  $d_y + s + d_x$ ;
```

(32.2) — otherwise, if $i_x < i_y$ is true, equivalent to:

```
    return -(y - x);
```


(32.3) — otherwise, equivalent to:

```

    return std::get<ix>(x.it_) - std::get<iy>(y.it_);

```

friend constexpr difference_type operator-(const iterator& x, default_sentinel_t)
requires see below;

33 *Preconditions:* x.it_.valueless_by_exception() is false.

34 *Effects:* Let i_x denote x.it_.index(), d_x be ranges::distance(std::get<i_x>(x.it_), ranges::end(std::get<i_x>(x.parent_->views_))). Let s denote the sum of the sizes of all the ranges std::get<i>(x.parent_->views_) for every integer i in the range [$i_x + 1$, sizeof...(Views)) if there is any, and 0 otherwise, of type difference_type, equivalent to:

```

    return -(dx + s);

```

35 *Remarks:* Let Fs be the pack that consists of all elements of Views except the first element, the expression in the *requires-clause* is equivalent to:

```

    (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
      iterator_t<maybe-const<Const, Views>>> && ...) &&
    (sized_range<maybe-const<Const, Fs>> && ...)

```

friend constexpr difference_type operator-(default_sentinel_t, const iterator& x)
requires see below;

36 *Effects:* Equivalent to:

```

    return -(x - default_sentinel);

```

37 *Remarks:* Let Fs be the pack that consists of all elements of Views except the first element, the expression in the *requires-clause* is equivalent to:

```

    (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
      iterator_t<maybe-const<Const, Views>>> && ...) &&
    (sized_range<maybe-const<Const, Fs>> && ...)

```

friend constexpr decltype(auto) iter_move(const iterator& it) noexcept(see below);

38 *Preconditions:* it.it_.valueless_by_exception() is false.

39 *Effects:* Equivalent to:

```

    return std::visit([](const auto& i)
      -> concat-rvalue-reference-t<maybe-const<Const, Views>...> {
        return ranges::iter_move(i);
      },
      it.it_);

```

40 *Remarks:* The exception specification is equivalent to:

```

    ((is_nothrow_invocable_v<decltype(ranges::iter_move),
      const iterator_t<maybe-const<Const, Views>>> &&
      is_nothrow_convertible_v<range_rvalue_reference_t<maybe-const<Const, Views>>,
      concat-rvalue-reference-t<maybe-const<Const, Views>...>>) &&
      ...)

```

friend constexpr void iter_swap(const iterator& x, const iterator& y) noexcept(see below)
requires see below;

41 *Preconditions:* x.it_.valueless_by_exception() and y.it_.valueless_by_exception() are each false.

42 *Effects:* Equivalent to:

```

    std::visit([&](const auto& it1, const auto& it2) {
      if constexpr (is_same_v<decltype(it1), decltype(it2)>) {
        ranges::iter_swap(it1, it2);
      } else {
        ranges::swap(*x, *y);
      }
    },
    x.it_, y.it_);

```

43 *Remarks:* The exception specification is equivalent to

```
(noexcept(ranges::swap(*x, *y)) && ... && noexcept(ranges::iter_swap(its, its)))
```

where `its` is a pack of lvalues of type `const iterator_t<maybe-const<Const, Views>>` respectively.

The expression in the *requires-clause* is equivalent to

```
swappable_with<iter_reference_t<iterator>, iter_reference_t<iterator>> &&
(... && indirectly_swappable<iterator_t<maybe-const<Const, Views>>>)
```

25.7.19 Counted view

[range.counted]

- ¹ A counted view presents a view of the elements of the counted range (24.3.1) `i + [0, n)` for an iterator `i` and non-negative integer `n`.
- ² The name `views::counted` denotes a customization point object (16.3.3.3.5). Let `E` and `F` be expressions, let `T` be `decay_t<decltype((E))>`, and let `D` be `iter_difference_t<T>`. If `decltype((F))` does not model `convertible_to<D>`, `views::counted(E, F)` is ill-formed.

[Note 1: This case can result in substitution failure when `views::counted(E, F)` appears in the immediate context of a template instantiation. — end note]

Otherwise, `views::counted(E, F)` is expression-equivalent to:

- (2.1) — If `T` models `contiguous_iterator`, then `span(to_address(E), static_cast<size_t>(static_cast<D>(F)))`.
- (2.2) — Otherwise, if `T` models `random_access_iterator`, then `subrange(E, E + static_cast<D>(F))`, except that `E` is evaluated only once.
- (2.3) — Otherwise, `subrange(counted_iterator(E, F), default_sentinel)`.

25.7.20 Common view

[range.common]

25.7.20.1 Overview

[range.common.overview]

- ¹ `common_view` takes a view which has different types for its iterator and sentinel and turns it into a view of the same elements with an iterator and sentinel of the same type.
- ² [Note 1: `common_view` is useful for calling legacy algorithms that expect a range's iterator and sentinel types to be the same. — end note]
- ³ The name `views::common` denotes a range adaptor object (25.7.2). Given a subexpression `E`, the expression `views::common(E)` is expression-equivalent to:
 - (3.1) — `views::all(E)`, if `decltype((E))` models `common_range` and `views::all(E)` is a well-formed expression.
 - (3.2) — Otherwise, `common_view{E}`.

⁴ [Example 1:

```
// Legacy algorithm:
template<class ForwardIterator>
size_t count(ForwardIterator first, ForwardIterator last);

template<forward_range R>
void my_algo(R&& r) {
    auto&& common = views::common(r);
    auto cnt = count(common.begin(), common.end());
    // ...
}
```

— end example]

25.7.20.2 Class template `common_view`

[range.common.view]

```
namespace std::ranges {
    template<view V>
        requires (!common_range<V> && copyable<iterator_t<V>>)
        class common_view : public view_interface<common_view<V>> {
        private:
            V base_ = V(); // exposition only

        public:
            common_view() requires default_initializable<V> = default;
```

```

constexpr explicit common_view(V r);

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }

constexpr auto begin() requires (!simple-view<V>) {
    if constexpr (random_access_range<V> && sized_range<V>)
        return ranges::begin(base_);
    else
        return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::begin(base_));
}

constexpr auto begin() const requires range<const V> {
    if constexpr (random_access_range<const V> && sized_range<const V>)
        return ranges::begin(base_);
    else
        return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::begin(base_));
}

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (random_access_range<V> && sized_range<V>)
        return ranges::begin(base_) + ranges::distance(base_);
    else
        return common_iterator<iterator_t<V>, sentinel_t<V>>(ranges::end(base_));
}

constexpr auto end() const requires range<const V> {
    if constexpr (random_access_range<const V> && sized_range<const V>)
        return ranges::begin(base_) + ranges::distance(base_);
    else
        return common_iterator<iterator_t<const V>, sentinel_t<const V>>(ranges::end(base_));
}

constexpr auto size() requires sized_range<V> {
    return ranges::size(base_);
}

constexpr auto size() const requires sized_range<const V> {
    return ranges::size(base_);
}

constexpr auto reserve_hint() requires approximately_sized_range<V> {
    return ranges::reserve_hint(base_);
}

constexpr auto reserve_hint() const requires approximately_sized_range<const V> {
    return ranges::reserve_hint(base_);
}
};

template<class R>
    common_view(R&&) -> common_view<views::all_t<R>>;
}

```

constexpr explicit common_view(V base);

¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.21 Reverse view

[range.reverse]

25.7.21.1 Overview

[range.reverse.overview]

¹ `reverse_view` takes a bidirectional view and produces another view that iterates the same elements in reverse order.

² The name `views::reverse` denotes a range adaptor object (25.7.2). Given a subexpression *E*, the expression `views::reverse(E)` is expression-equivalent to:

(2.1) — If the type of *E* is a (possibly cv-qualified) specialization of `reverse_view`, then `E.base()`.

- (2.2) — Otherwise, if E is a specialization of `optional` and E models `view`, then *decay-copy*(E).
- (2.3) — Otherwise, if the type of E is `cv subrange<reverse_iterator<I>, reverse_iterator<I>, K>` for some iterator type I and value K of type `subrange_kind`,
 - (2.3.1) — if K is `subrange_kind::sized`, then `subrange<I, I, K>(E.end().base(), E.begin().base(), E.size());`
 - (2.3.2) — otherwise, `subrange<I, I, K>(E.end().base(), E.begin().base());`

However, in either case E is evaluated only once.

- (2.4) — Otherwise, `reverse_view{E}`.

³ [Example 1:

```
vector<int> is {0,1,2,3,4};
for (int i : is | views::reverse)
  cout << i << ' '; // prints 4 3 2 1 0
```

— end example]

25.7.21.2 Class template `reverse_view`

[range.reverse.view]

```
namespace std::ranges {
  template<view V>
    requires bidirectional_range<V>
    class reverse_view : public view_interface<reverse_view<V>> {
    private:
      V base_ = V(); // exposition only

    public:
      reverse_view() requires default_initializable<V> = default;

      constexpr explicit reverse_view(V r);

      constexpr V base() const & requires copy_constructible<V> { return base_; }
      constexpr V base() && { return std::move(base_); }

      constexpr reverse_iterator<iterator_t<V>> begin();
      constexpr reverse_iterator<iterator_t<V>> begin() requires common_range<V>;
      constexpr auto begin() const requires common_range<const V>;

      constexpr reverse_iterator<iterator_t<V>> end();
      constexpr auto end() const requires common_range<const V>;

      constexpr auto size() requires sized_range<V> {
        return ranges::size(base_);
      }

      constexpr auto size() const requires sized_range<const V> {
        return ranges::size(base_);
      }

      constexpr auto reserve_hint() requires approximately_sized_range<V> {
        return ranges::reserve_hint(base_);
      }
      constexpr auto reserve_hint() const requires approximately_sized_range<const V> {
        return ranges::reserve_hint(base_);
      }
    };

  template<class R>
    reverse_view(R&&) -> reverse_view<views::all_t<R>>;
}
```

constexpr explicit reverse_view(V base);

¹ *Effects:* Initializes `base_` with `std::move(base)`.

```
constexpr reverse_iterator<iterator_t<V>> begin();
```

2 *Returns:*

```
make_reverse_iterator(ranges::next(ranges::begin(base_), ranges::end(base_)))
```

3 *Remarks:* In order to provide the amortized constant time complexity required by the `range` concept, this function caches the result within the `reverse_view` for use on subsequent calls.

```
constexpr reverse_iterator<iterator_t<V>> begin() requires common_range<V>;
constexpr auto begin() const requires common_range<const V>;
```

4 *Effects:* Equivalent to: `return make_reverse_iterator(ranges::end(base_));`

```
constexpr reverse_iterator<iterator_t<V>> end();
constexpr auto end() const requires common_range<const V>;
```

5 *Effects:* Equivalent to: `return make_reverse_iterator(ranges::begin(base_));`

25.7.22 As const view [range.as.const]

25.7.22.1 Overview [range.as.const.overview]

1 `as_const_view` presents a view of an underlying sequence as constant. That is, the elements of an `as_const_view` cannot be modified.

2 The name `views::as_const` denotes a range adaptor object (25.7.2). Let `E` be an expression, let `T` be `decltype((E))`, and let `U` be `remove_cvref_t<T>`. The expression `views::as_const(E)` is expression-equivalent to:

- (2.1) — If `views::all_t<T>` models `constant_range`, then `views::all(E)`.
- (2.2) — Otherwise, if `U` denotes `empty_view<X>` for some type `X`, then `auto(views::empty<const X>)`.
- (2.3) — Otherwise, if `U` denotes `optional<X&>` for some type `X`, then `optional<const X&>(E)`.
- (2.4) — Otherwise, if `U` denotes `span<X, Extent>` for some type `X` and some extent `Extent`, then `span<const X, Extent>(E)`.
- (2.5) — Otherwise, if `U` denotes `ref_view<X>` for some type `X` and `const X` models `constant_range`, then `ref_view(static_cast<const X&>(E.base()))`.
- (2.6) — Otherwise, if `E` is an lvalue, `const U` models `constant_range`, and `U` does not model `view`, then `ref_view(static_cast<const U&>(E))`.
- (2.7) — Otherwise, `as_const_view(E)`.

3 [Example 1:

```
template<constant_range R>
void cant_touch_this(R&&);
```

```
vector<char> hammer = {'m', 'c'};
span<char> beat = hammer;
cant_touch_this(views::as_const(beat));           // will not modify the elements of hammer
```

— end example]

25.7.22.2 Class template `as_const_view` [range.as.const.view]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class as_const_view : public view_interface<as_const_view<V>> {
            V base_ = V();           // exposition only

        public:
            as_const_view() requires default_initializable<V> = default;
            constexpr explicit as_const_view(V base);

            constexpr V base() const & requires copy_constructible<V> { return base_; }
            constexpr V base() && { return std::move(base_); }
```

```
constexpr auto begin() requires (!simple-view<V>) { return ranges::cbegin(base_); }
constexpr auto begin() const requires range<const V> { return ranges::cbegin(base_); }

constexpr auto end() requires (!simple-view<V>) { return ranges::cend(base_); }
constexpr auto end() const requires range<const V> { return ranges::cend(base_); }

constexpr auto size() requires sized_range<V> { return ranges::size(base_); }
constexpr auto size() const requires sized_range<const V> { return ranges::size(base_); }

constexpr auto reserve_hint() requires approximately_sized_range<V>
{ return ranges::reserve_hint(base_); }
constexpr auto reserve_hint() const requires approximately_sized_range<const V>
{ return ranges::reserve_hint(base_); }
};

template<class R>
    as_const_view(R&&) -> as_const_view<views::all_t<R>>;
}
```

```
constexpr explicit as_const_view(V base);
```

- ¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.23 Elements view

[range.elements]

25.7.23.1 Overview

[range.elements.overview]

- ¹ `elements_view` takes a view of tuple-like values and a `size_t`, and produces a view with a value-type of the N^{th} element of the adapted view's value-type.
- ² The name `views::elements<N>` denotes a range adaptor object (25.7.2). Given a subexpression *E* and constant expression *N*, the expression `views::elements<N>(E)` is expression-equivalent to `elements_view<views::all_t<decltype((E))>, N>{E}`.

[Example 1:

```
auto historical_figures = map{
    pair{"Lovelace"sv, 1815},
    {"Turing"sv, 1912},
    {"Babbage"sv, 1791},
    {"Hamilton"sv, 1936}
};

auto names = historical_figures | views::elements<0>;
for (auto&& name : names) {
    cout << name << ' ';           // prints Babbage Hamilton Lovelace Turing
}

auto birth_years = historical_figures | views::elements<1>;
for (auto&& born : birth_years) {
    cout << born << ' ';           // prints 1791 1936 1815 1912
}
```

— end example]

- ³ `keys_view` is an alias for `elements_view<R, 0>`, and is useful for extracting keys from associative containers.

[Example 2:

```
auto names = historical_figures | views::keys;
for (auto&& name : names) {
    cout << name << ' ';           // prints Babbage Hamilton Lovelace Turing
}
```

— end example]

- ⁴ `values_view` is an alias for `elements_view<R, 1>`, and is useful for extracting values from associative containers.

[Example 3:

```
auto is_even = [](const auto x) { return x % 2 == 0; };
```

```
cout << ranges::count_if(historical_figures | views::values, is_even); // prints 2
— end example]
```

25.7.23.2 Class template `elements_view`

[range.elements.view]

```
namespace std::ranges {
    template<class T, size_t N>
    concept has_tuple_element = // exposition only
        tuple_like<T> && N < tuple_size_v<T> &&
        requires(T t) {
            { std::get<N>(t) } -> convertible_to<const tuple_element_t<N, T>&>;
        };

    template<class T, size_t N>
    concept returnable_element = // exposition only
        is_reference_v<T> || move_constructible<tuple_element_t<N, T>>;

    template<input_range V, size_t N>
    requires view<V> && has_tuple_element<range_value_t<V>, N> &&
        has_tuple_element<remove_reference_t<range_reference_t<V>>, N> &&
        returnable_element<range_reference_t<V>, N>
    class elements_view : public view_interface<elements_view<V, N>> {
    public:
        elements_view() requires default_initializable<V> = default;
        constexpr explicit elements_view(V base_);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple_view<V>)
        { return iterator<false>(ranges::begin(base_)); }

        constexpr auto begin() const requires range<const V>
        { return iterator<true>(ranges::begin(base_)); }

        constexpr auto end() requires (!simple_view<V> && !common_range<V>)
        { return sentinel<false>(ranges::end(base_)); }

        constexpr auto end() requires (!simple_view<V> && common_range<V>)
        { return iterator<false>(ranges::end(base_)); }

        constexpr auto end() const requires range<const V>
        { return sentinel<true>(ranges::end(base_)); }

        constexpr auto end() const requires common_range<const V>
        { return iterator<true>(ranges::end(base_)); }

        constexpr auto size() requires sized_range<V>
        { return ranges::size(base_); }

        constexpr auto size() const requires sized_range<const V>
        { return ranges::size(base_); }

        constexpr auto reserve_hint() requires approximately_sized_range<V>
        { return ranges::reserve_hint(base_); }

        constexpr auto reserve_hint() const requires approximately_sized_range<const V>
        { return ranges::reserve_hint(base_); }

    private:
        // 25.7.23.3, class template elements_view::iterator
        template<bool> class iterator; // exposition only
    };
}
```

```

// 25.7.23.4, class template elements_view::sentinel
template<bool> class sentinel; // exposition only

V base_ = V(); // exposition only
};
}

```

```
constexpr explicit elements_view(V base);
```

¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.23.3 Class template `elements_view::iterator` [range.elements.iterator]

```

namespace std::ranges {
template<input_range V, size_t N>
requires view<V> && has-tuple-element<range_value_t<V>, N> &&
    has-tuple-element<remove_reference_t<range_reference_t<V>>, N> &&
    returnable-element<range_reference_t<V>, N>
template<bool Const>
class elements_view<V, N>::iterator {
    using Base = maybe-const<Const, V>; // exposition only

    iterator_t<Base> current_ = iterator_t<Base>(); // exposition only

    static constexpr decltype(auto) get-element(const iterator_t<Base>& i); // exposition only
    constexpr explicit iterator(iterator_t<Base> current); // exposition only

public:
    using iterator_concept = see below;
    using iterator_category = see below; // not always present
    using value_type = remove_cvref_t<tuple_element_t<N, range_value_t<Base>>>;
    using difference_type = range_difference_t<Base>;

    iterator() requires default_initializable<iterator_t<Base>> = default;
    constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr const iterator_t<Base>& base() const & noexcept;
    constexpr iterator_t<Base> base() &&;

    constexpr decltype(auto) operator*() const
    { return get-element(current_); }

    constexpr iterator& operator++();
    constexpr void operator++(int);
    iterator operator++(int) requires forward_range<Base> = default;

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type x)
        requires random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x)
        requires random_access_range<Base>;

    constexpr decltype(auto) operator[](difference_type n) const
        requires random_access_range<Base>
    { return get-element(current_ + n); }

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<iterator_t<Base>>;

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
}

```



```

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
};
}

```

¹ The member *typedef-name* `iterator_concept` is defined as follows:

- (1.1) — If *Base* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.
- (1.2) — Otherwise, if *Base* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (1.3) — Otherwise, if *Base* models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- (1.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

² The member *typedef-name* `iterator_category` is declared if and only if *Base* models `forward_range`. In that case, `iterator_category` is defined as follows: Let *C* denote the type `iterator_traits<iterator_t<Base>>::iterator_category`.

- (2.1) — If `std::get<N>(*current_)` is an rvalue, `iterator_category` denotes `input_iterator_tag`.
- (2.2) — Otherwise, if *C* models `derived_from<random_access_iterator_tag>`, `iterator_category` denotes `random_access_iterator_tag`.
- (2.3) — Otherwise, `iterator_category` denotes *C*.

```
static constexpr decltype(auto) get_element(const iterator_t<Base>& i);
```

³ *Effects*: Equivalent to:

```

if constexpr (is_reference_v<range_reference_t<Base>>) {
    return std::get<N>(*i);
} else {
    using E = remove_cv_t<tuple_element_t<N, range_reference_t<Base>>>;
    return static_cast<E>(std::get<N>(*i));
}

```

```
constexpr explicit iterator(iterator_t<Base> current);
```

⁴ *Effects*: Initializes `current_` with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

⁵ *Effects*: Initializes `current_` with `std::move(i.current_)`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

⁶ *Effects*: Equivalent to: return `current_`;

```
constexpr iterator_t<Base> base() &&;
```

⁷ *Effects*: Equivalent to: return `std::move(current_)`;

```

constexpr iterator& operator++();
8     Effects: Equivalent to:
        ++current_;
        return *this;

constexpr void operator++(int);
9     Effects: Equivalent to: ++current_.

constexpr iterator& operator--() requires bidirectional_range<Base>;
10    Effects: Equivalent to:
        --current_;
        return *this;

constexpr iterator& operator+=(difference_type n)
    requires random_access_range<Base>;
11    Effects: Equivalent to:
        current_ += n;
        return *this;

constexpr iterator& operator-=(difference_type n)
    requires random_access_range<Base>;
12    Effects: Equivalent to:
        current_ -= n;
        return *this;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<Base>;
13    Effects: Equivalent to: return x.current_ == y.current_;

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
14    Effects: Equivalent to: return x.current_ < y.current_;

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
15    Effects: Equivalent to: return y < x;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
16    Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
17    Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;
18    Effects: Equivalent to: return x.current_ <=> y.current_;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires random_access_range<Base>;
19    Effects: Equivalent to: return iterator{x} += y;

friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires random_access_range<Base>;
20    Effects: Equivalent to: return y + x;

```

```
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires random_access_range<Base>;
```

21 *Effects:* Equivalent to: return `iterator{x} -= y`;

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
```

22 *Effects:* Equivalent to: return `x.current_ - y.current_`;

25.7.23.4 Class template `elements_view::sentinel` [range.elements.sentinel]

```
namespace std::ranges {
    template<input_range V, size_t N>
        requires view<V> && has_tuple_element<range_value_t<V>, N> &&
            has_tuple_element<remove_reference_t<range_reference_t<V>>, N> &&
            returnable_element<range_reference_t<V>, N>
    template<bool Const>
    class elements_view<V, N>::sentinel {
    private:
        using Base = maybe_const<Const, V>; // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only
        constexpr explicit sentinel(sentinel_t<Base> end); // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel<!Const> other)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;

        template<bool OtherConst>
            requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr range_difference_t<maybe_const<OtherConst, V>>
            operator-(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
        friend constexpr range_difference_t<maybe_const<OtherConst, V>>
            operator-(const sentinel& x, const iterator<OtherConst>& y);
    };
}
```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects:* Initializes `end_` with `end`.

```
constexpr sentinel(sentinel<!Const> other)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes `end_` with `std::move(other.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

3 *Effects:* Equivalent to: return `end_`;

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe_const<OtherConst, V>>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

4 *Effects:* Equivalent to: return `x.current_ == y.end_`;

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const iterator_t<OtherConst>& x, const sentinel_t& y);
```

5 *Effects:* Equivalent to: return `x.current_ - y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const sentinel_t& x, const iterator_t<OtherConst>& y);
```

6 *Effects:* Equivalent to: return `x.end_ - y.current_;`

25.7.24 Enumerate view

[range.enumerate]

25.7.24.1 Overview

[range.enumerate.overview]

- 1 `enumerate_view` is a view whose elements represent both the position and value from a sequence of elements.
- 2 The name `views::enumerate` denotes a range adaptor object (25.7.2). Given a subexpression `E`, the expression `views::enumerate(E)` is expression-equivalent to `enumerate_view<views::all_t<decltype(E)>>>(E)`.

[Example 1:

```
vector<int> vec{ 1, 2, 3 };
for (auto [index, value] : views::enumerate(vec))
    cout << index << ":" << value << ' ';           // prints 0:1 1:2 2:3
```

— end example]

25.7.24.2 Class template `enumerate_view`

[range.enumerate.view]

```
namespace std::ranges {
    template<view V>
        requires range-with-movable-references<V>
    class enumerate_view : public view_interface<enumerate_view<V>> {
        V base_ = V();                                     // exposition only

        // 25.7.24.3, class template enumerate_view::iterator
        template<bool Const>
            class iterator;                                // exposition only

        // 25.7.24.4, class template enumerate_view::sentinel
        template<bool Const>
            class sentinel;                                // exposition only

    public:
        constexpr enumerate_view() requires default_initializable<V> = default;
        constexpr explicit enumerate_view(V base);

        constexpr auto begin() requires (!simple-view<V>)
        { return iterator<false>(ranges::begin(base_), 0); }
        constexpr auto begin() const requires range-with-movable-references<const V>
        { return iterator<true>(ranges::begin(base_), 0); }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (forward_range<V> && common_range<V> && sized_range<V>)
                return iterator<false>(ranges::end(base_), ranges::distance(base_));
            else
                return sentinel<false>(ranges::end(base_));
        }
        constexpr auto end() const requires range-with-movable-references<const V> {
            if constexpr (forward_range<const V> && common_range<const V> && sized_range<const V>)
                return iterator<true>(ranges::end(base_), ranges::distance(base_));
            else
                return sentinel<true>(ranges::end(base_));
        }
    }
}
```

```

constexpr auto size() requires sized_range<V>
{ return ranges::size(base_); }
constexpr auto size() const requires sized_range<const V>
{ return ranges::size(base_); }

constexpr auto reserve_hint() requires approximately_sized_range<V>
{ return ranges::reserve_hint(base_); }
constexpr auto reserve_hint() const requires approximately_sized_range<const V>
{ return ranges::reserve_hint(base_); }

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }
};

template<class R>
    enumerate_view(R&&) -> enumerate_view<views::all_t<R>>;
}

```

```
constexpr explicit enumerate_view(V base);
```

¹ *Effects:* Initializes *base_* with `std::move(base)`.

25.7.24.3 Class template `enumerate_view::iterator`

[range.enumerate.iterator]

```

namespace std::ranges {
    template<view V>
        requires range-with-movable-references<V>
        template<bool Const>
        class enumerate_view<V>::iterator {
            using Base = maybe-const<Const, V>;                                     // exposition only

        public:
            using iterator_category = input_iterator_tag;
            using iterator_concept = see below;
            using difference_type = range_difference_t<Base>;
            using value_type = tuple<difference_type, range_value_t<Base>>;

        private:
            using reference-type =                                           // exposition only
                tuple<difference_type, range_reference_t<Base>>;
            iterator_t<Base> current_ = iterator_t<Base>();                 // exposition only
            difference_type pos_ = 0;                                       // exposition only

            constexpr explicit
                iterator(iterator_t<Base> current, difference_type pos);    // exposition only

        public:
            iterator() requires default_initializable<iterator_t<Base>> = default;
            constexpr iterator(iterator<!Const> i)
                requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

            constexpr const iterator_t<Base>& base() const & noexcept;
            constexpr iterator_t<Base> base() &&;

            constexpr difference_type index() const noexcept;

            constexpr auto operator*() const {
                return reference-type(pos_, *current_);
            }

            constexpr iterator& operator++();
            constexpr void operator++(int);
            iterator operator++(int) requires forward_range<Base> = default;

```

```

constexpr iterator& operator--() requires bidirectional_range<Base>;
iterator operator--(int) = default;

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;

constexpr auto operator[](difference_type n) const
    requires random_access_range<Base>
{ return reference_type(pos_ + n, current_[n]); }

friend constexpr bool operator==(const iterator& x, const iterator& y) noexcept;
friend constexpr strong_ordering operator<=>(const iterator& x, const iterator& y) noexcept;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y) noexcept;

friend constexpr auto iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)) &&
        is_nothrow_move_constructible_v<range_rvalue_reference_t<Base>> ) {
    return tuple<difference_type,
        range_rvalue_reference_t<Base>>(i.pos_, ranges::iter_move(i.current_));
}
};
}

```

¹ The member *typedef-name* `iterator::iterator_concept` is defined as follows:

- (1.1) — If *Base* models `random_access_range`, then `iterator_concept` denotes `random_access_iterator_tag`.
- (1.2) — Otherwise, if *Base* models `bidirectional_range`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- (1.3) — Otherwise, if *Base* models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- (1.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

```
constexpr explicit iterator(iterator_t<Base> current, difference_type pos);
```

- ² *Effects*: Initializes `current_` with `std::move(current)` and `pos_` with `pos`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
```

- ³ *Effects*: Initializes `current_` with `std::move(i.current_)` and `pos_` with `i.pos_`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

- ⁴ *Effects*: Equivalent to: return `current_`;

```
constexpr iterator_t<Base> base() &&;
```

- ⁵ *Effects*: Equivalent to: return `std::move(current_)`;

```
constexpr difference_type index() const noexcept;
```

- ⁶ *Effects*: Equivalent to: return `pos_`;

```
constexpr iterator& operator++();
```

- ⁷ *Effects*: Equivalent to:

```
++current_;
++pos_;
```

```

        return *this;

constexpr void operator++(int);
8      Effects: Equivalent to ++*this.

constexpr iterator& operator--() requires bidirectional_range<Base>;
9      Effects: Equivalent to:
        --current_;
        --pos_;
        return *this;

constexpr iterator& operator+=(difference_type n)
    requires random_access_range<Base>;
10     Effects: Equivalent to:
        current_ += n;
        pos_ += n;
        return *this;

constexpr iterator& operator-=(difference_type n)
    requires random_access_range<Base>;
11     Effects: Equivalent to:
        current_ -= n;
        pos_ -= n;
        return *this;

friend constexpr bool operator==(const iterator& x, const iterator& y) noexcept;
12     Effects: Equivalent to: return x.pos_ == y.pos_;

friend constexpr strong_ordering operator<=>(const iterator& x, const iterator& y) noexcept;
13     Effects: Equivalent to: return x.pos_ <=> y.pos_;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires random_access_range<Base>;
14     Effects: Equivalent to:
        auto temp = x;
        temp += y;
        return temp;

friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires random_access_range<Base>;
15     Effects: Equivalent to: return y + x;

friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires random_access_range<Base>;
16     Effects: Equivalent to:
        auto temp = x;
        temp -= y;
        return temp;

friend constexpr difference_type operator-(const iterator& x, const iterator& y) noexcept;
17     Effects: Equivalent to: return x.pos_ - y.pos_;

```

25.7.24.4 Class template `enumerate_view::sentinel`

[range.enumerate.sentinel]

```

namespace std::ranges {
    template<view V>
        requires range-with-movable-references<V>
        template<bool Const>
        class enumerate_view<V>::sentinel {
            using Base = maybe-const<Const, V>;
            // exposition only

```

```

    sentinel_t<Base> end_ = sentinel_t<Base>();           // exposition only
    constexpr explicit sentinel(sentinel_t<Base> end);    // exposition only

public:
    sentinel() = default;
    constexpr sentinel(sentinel_t<!Const> other)
        requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

    constexpr sentinel_t<Base> base() const;

    template<bool OtherConst>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel_t<y>);

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
        operator-(const iterator_t<OtherConst>& x, const sentinel_t<y>);

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
        operator-(const sentinel_t<x>, const iterator_t<OtherConst>& y);
};
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects:* Initializes *end_* with `std::move(end)`.

```
constexpr sentinel(sentinel_t<!Const> other)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes *end_* with `std::move(other.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

3 *Effects:* Equivalent to: `return end_;`

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel_t<y>);
```

4 *Effects:* Equivalent to: `return x.current_ == y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const iterator_t<OtherConst>& x, const sentinel_t<y>);
```

5 *Effects:* Equivalent to: `return x.current_ - y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const sentinel_t<x>, const iterator_t<OtherConst>& y);
```

6 *Effects:* Equivalent to: `return x.end_ - y.current_;`

25.7.25 Zip view

[range.zip]

25.7.25.1 Overview

[range.zip.overview]

1 `zip_view` takes any number of views and produces a view of tuples of references to the corresponding elements of the constituent views.

2 The name `views::zip` denotes a customization point object (16.3.3.3.5). Given a pack of subexpressions `Es...`, the expression `views::zip(Es...)` is expression-equivalent to

(2.1) — `auto(views::empty<tuple<>>)` if `Es` is an empty pack,

(2.2) — otherwise, `zip_view<views::all_t<decltype((Es))>...>(Es...)`.

[Example 1:

```
vector v = {1, 2};
list l = {'a', 'b', 'c'};

auto z = views::zip(v, l);
range_reference_t<decltype(z)> f = z.front();    // f is a tuple<int&, char&>
                                                // that refers to the first element of v and l

for (auto&& [x, y] : z) {
    cout << '(' << x << ", " << y << " ";      // prints (1, a) (2, b)
}
```

— end example]

25.7.25.2 Class template `zip_view`

[range.zip.view]

```
namespace std::ranges {
    template<class... Rs>
    concept zip-is-common =                // exposition only
        (sizeof...(Rs) == 1 && (common_range<Rs> && ...)) ||
        (!bidirectional_range<Rs> && ... && (common_range<Rs> && ...)) ||
        ((random_access_range<Rs> && ... && (sized_range<Rs> && ...)));

    template<input_range... Views>
    requires (view<Views> && ...) && (sizeof...(Views) > 0)
    class zip_view : public view_interface<zip_view<Views...>> {
        tuple<Views...> views_;            // exposition only

        // 25.7.25.3, class template zip_view::iterator
        template<bool> class iterator;      // exposition only

        // 25.7.25.4, class template zip_view::sentinel
        template<bool> class sentinel;      // exposition only

    public:
        zip_view() = default;
        constexpr explicit zip_view(Views... views);

        constexpr auto begin() requires (!(simple-view<Views> && ...)) {
            return iterator<false>(tuple-transform(ranges::begin, views_));
        }
        constexpr auto begin() const requires (range<const Views> && ...) {
            return iterator<true>(tuple-transform(ranges::begin, views_));
        }

        constexpr auto end() requires (!(simple-view<Views> && ...)) {
            if constexpr (!zip-is-common<Views...>) {
                return sentinel<false>(tuple-transform(ranges::end, views_));
            } else if constexpr ((random_access_range<Views> && ...)) {
                return begin() + iter_difference_t<iterator<false>>(size());
            } else {
                return iterator<false>(tuple-transform(ranges::end, views_));
            }
        }

        constexpr auto end() const requires (range<const Views> && ...) {
            if constexpr (!zip-is-common<const Views...>) {
                return sentinel<true>(tuple-transform(ranges::end, views_));
            } else if constexpr ((random_access_range<const Views> && ...)) {
                return begin() + iter_difference_t<iterator<true>>(size());
            } else {
                return iterator<true>(tuple-transform(ranges::end, views_));
            }
        }
    };
}
```

```

}

constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
};

template<class... Rs>
    zip_view(Rs&&...) -> zip_view<views::all_t<Rs>...>;
}

```

- ¹ Two `zip_view` objects have the same underlying sequence if and only if the corresponding elements of `views_` are equal (18.2) and have the same underlying sequence.

[Note 1: In particular, comparison of iterators obtained from `zip_view` objects that do not have the same underlying sequence is not required to produce meaningful results (24.3.4.11). — end note]

```
constexpr explicit zip_view(Views... views);
```

- ² *Effects:* Initializes `views_` with `std::move(views)....`

```
constexpr auto size() requires (sized_range<Views> && ...);
constexpr auto size() const requires (sized_range<const Views> && ...);
```

- ³ *Effects:* Equivalent to:

```

return apply([](auto... sizes) {
    using CT = make_unsigned_like_t<common_type_t<decltype(sizes)...>>;
    return ranges::min({CT(sizes)...});
}, tuple_transform(ranges::size, views_));

```

25.7.25.3 Class template `zip_view::iterator`

[range.zip.iterator]

```

namespace std::ranges {
    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0)
        template<bool Const>
        class zip_view<Views...>::iterator {
            tuple<iterator_t<maybe_const<Const, Views>>...> current_;           // exposition only
            constexpr explicit iterator(tuple<iterator_t<maybe_const<Const, Views>>...>); // exposition only

        public:
            using iterator_category = input_iterator_tag;                       // not always present
            using iterator_concept = see below;
            using value_type = tuple<range_value_t<maybe_const<Const, Views>>...>;
            using difference_type = common_type_t<range_difference_t<maybe_const<Const, Views>>...>;

            iterator() = default;
            constexpr iterator(iterator<!Const> i)
                requires Const && (convertible_to<iterator_t<Views>, iterator_t<const Views>> && ...);

            constexpr auto operator*() const;
            constexpr iterator& operator++();
            constexpr void operator++(int);
            iterator operator++(int) requires all_forward<Const, Views...> = default;

            constexpr iterator& operator--() requires all_bidirectional<Const, Views...>;
            iterator operator--(int) = default;

            constexpr iterator& operator+=(difference_type x)
                requires all_random_access<Const, Views...>;
            constexpr iterator& operator-=(difference_type x)
                requires all_random_access<Const, Views...>;

            constexpr auto operator[](difference_type n) const
                requires all_random_access<Const, Views...>;

            friend constexpr bool operator==(const iterator& x, const iterator& y)
                requires (equality_comparable<iterator_t<maybe_const<Const, Views>>> && ...);

```

```

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires all-random-access<Const, Views...>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires all-random-access<Const, Views...>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires all-random-access<Const, Views...>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires (sized_sentinel_for<iterator_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<Const, Views>>>> && ...);

friend constexpr auto iter_move(const iterator& i) noexcept(see below);

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, Views>>>> && ...);
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *all-random-access*<Const, Views...> is modeled, then *iterator_concept* denotes *random-access_iterator_tag*.
- (1.2) — Otherwise, if *all-bidirectional*<Const, Views...> is modeled, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *all-forward*<Const, Views...> is modeled, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² *iterator::iterator_category* is present if and only if *all-forward*<Const, Views...> is modeled.

³ If the invocation of any non-const member function of *iterator* exits via an exception, the iterator acquires a singular value.

```
constexpr explicit iterator(tuple<iterator_t<maybe-const<Const, Views>>...> current);
```

⁴ *Effects*: Initializes *current_* with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i)
    requires Const && (convertible_to<iterator_t<Views>, iterator_t<const Views>>> && ...);
```

⁵ *Effects*: Initializes *current_* with `std::move(i.current_)`.

```
constexpr auto operator*() const;
```

⁶ *Effects*: Equivalent to:

```
return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);
```

```
constexpr iterator& operator++();
```

⁷ *Effects*: Equivalent to:

```
tuple-for-each([](auto& i) { ++i; }, current_);
return *this;
```

```
constexpr void operator++(int);
```

⁸ *Effects*: Equivalent to `++*this`.

```
constexpr iterator& operator--() requires all-bidirectional<Const, Views...>;
```

⁹ *Effects*: Equivalent to:

```
tuple-for-each([](auto& i) { --i; }, current_);
return *this;
```

```
constexpr iterator& operator+=(difference_type x)
    requires all-random-access<Const, Views...>;
```

10 *Effects*: Equivalent to:

```
tuple-for-each([&<class I>(I& i) { i += iter_difference_t<I>(x); }, current_);
return *this;
```

```
constexpr iterator& operator-=(difference_type x)
    requires all-random-access<Const, Views...>;
```

11 *Effects*: Equivalent to:

```
tuple-for-each([&<class I>(I& i) { i -= iter_difference_t<I>(x); }, current_);
return *this;
```

```
constexpr auto operator[](difference_type n) const
    requires all-random-access<Const, Views...>;
```

12 *Effects*: Equivalent to:

```
return tuple-transform([&<class I>(I& i) -> decltype(auto) {
    return i[iter_difference_t<I>(n)];
}, current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires (equality_comparable<iterator_t<maybe-const<Const, Views>>> && ...);
```

13 *Returns*:

- (13.1) — $x.current_ == y.current_$ if *all-bidirectional*<Const, Views...> is true.
- (13.2) — Otherwise, true if there exists an integer $0 \leq i < \text{sizeof...}(\text{Views})$ such that `bool(std::get<i>(x.current_) == std::get<i>(y.current_))` is true.
[Note 1: This allows `zip_view` to model `common_range` when all constituent views model `common_range`.
— end note]
- (13.3) — Otherwise, false.

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, Views...>;
```

14 *Returns*: $x.current_ <= y.current_$.

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires all-random-access<Const, Views...>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires all-random-access<Const, Views...>;
```

15 *Effects*: Equivalent to:

```
auto r = i;
r += n;
return r;
```

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires all-random-access<Const, Views...>;
```

16 *Effects*: Equivalent to:

```
auto r = i;
r -= n;
return r;
```

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires (sized_sentinel_for<iterator_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<Const, Views>>> && ...);
```

17 Let $DIST(i)$ be $\text{difference_type}(\text{std::get}<i>(x.current_) - \text{std::get}<i>(y.current_))$.

18 *Returns*: The value with the smallest absolute value among $DIST(n)$ for all integers $0 \leq n < \text{sizeof...}(\text{Views})$.

```
friend constexpr auto iter_move(const iterator& i) noexcept(see below);
```

19 *Effects:* Equivalent to:

```
return tuple-transform(ranges::iter_move, i.current_);
```

20 *Remarks:* The exception specification is equivalent to:

```
(noexcept(ranges::iter_move(declval<const iterator_t<maybe-const<Const,
Views>>>())) && ...) &&
(is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-const<Const,
Views>>> && ...))
```

```
friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
requires (indirectly_swappable<iterator_t<maybe-const<Const, Views>>> && ...);
```

21 *Effects:* For every integer $0 \leq i < \text{sizeof} \dots (\text{Views})$, performs:

```
ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_))
```

22 *Remarks:* The exception specification is equivalent to the logical AND of the following expressions:

```
noexcept(ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_)))
```

for every integer $0 \leq i < \text{sizeof} \dots (\text{Views})$.

25.7.25.4 Class template `zip_view::sentinel` [range.zip.sentinel]

```
namespace std::ranges {
    template<input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0)
    template<bool Const>
    class zip_view<Views...>::sentinel {
        tuple<sentinel_t<maybe-const<Const, Views>>>...> end_; // exposition only
        constexpr explicit sentinel(tuple<sentinel_t<maybe-const<Const, Views>>>...> end); // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel<!Const> i)
            requires Const && (convertible_to<sentinel_t<Views>, sentinel_t<const Views>> && ...);

        template<bool OtherConst>
            requires (sentinel_for<sentinel_t<maybe-const<Const, Views>>,
                iterator_t<maybe-const<OtherConst, Views>>> && ...)
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
                iterator_t<maybe-const<OtherConst, Views>>> && ...)
        friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>>...>
            operator-(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
                iterator_t<maybe-const<OtherConst, Views>>> && ...)
        friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>>...>
            operator-(const sentinel& y, const iterator<OtherConst>& x);
    };
}
```

```
constexpr explicit sentinel(tuple<sentinel_t<maybe-const<Const, Views>>>...> end);
```

1 *Effects:* Initializes `end_` with `end`.

```
constexpr sentinel(sentinel<!Const> i)
requires Const && (convertible_to<sentinel_t<Views>, sentinel_t<const Views>> && ...);
```

2 *Effects:* Initializes `end_` with `std::move(i.end_)`.

```

template<bool OtherConst>
    requires (sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
3     Returns: true if there exists an integer  $0 \leq i < \text{sizeof}...(Views)$  such that bool(std::get<i>(x.current_) == std::get<i>(y.end_)) is true. Otherwise, false.

template<bool OtherConst>
    requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>...>
    operator-(const iterator<OtherConst>& x, const sentinel& y);
4     Let D be the return type. Let  $DIST(i)$  be  $D(\text{std::get<i>(x.current_) - std::get<i>(y.end_)})$ .
5     Returns: The value with the smallest absolute value among  $DIST(n)$  for all integers  $0 \leq n < \text{sizeof}...(Views)$ .

template<bool OtherConst>
    requires (sized_sentinel_for<sentinel_t<maybe-const<Const, Views>>,
        iterator_t<maybe-const<OtherConst, Views>>> && ...)
friend constexpr common_type_t<range_difference_t<maybe-const<OtherConst, Views>>...>
    operator-(const sentinel& y, const iterator<OtherConst>& x);
6     Effects: Equivalent to: return -(x - y);

```

25.7.26 Zip transform view

[range.zip.transform]

25.7.26.1 Overview

[range.zip.transform.overview]

- 1 `zip_transform_view` takes an invocable object and any number of views and produces a view whose M^{th} element is the result of applying the invocable object to the M^{th} elements of all views.
- 2 The name `views::zip_transform` denotes a customization point object (16.3.3.3.5). Let F be a subexpression, and let `Es...` be a pack of subexpressions.
 - (2.1) — If `Es` is an empty pack, let FD be `decay_t<decltype((F))>`.
 - (2.1.1) — If `move_constructible<FD> && regular_invocable<FD&>` is false, or if `decay_t<invoke_result_t<FD&>>` is not an object type, `views::zip_transform(F, Es...)` is ill-formed.
 - (2.1.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to `((void)F, auto(v1 : views::empty<decay_t<invoke_result_t<FD&>>>))`
 - (2.2) — Otherwise, the expression `views::zip_transform(F, Es...)` is expression-equivalent to `zip_transform_view(F, Es...)`.

3 [Example 1:

```

vector v1 = {1, 2};
vector v2 = {4, 5, 6};

for (auto i : views::zip_transform(plus(), v1, v2)) {
    cout << i << ' ';    // prints 5 7
}

```

— end example]

25.7.26.2 Class template zip_transform_view

[range.zip.transform.view]

```

namespace std::ranges {
    template<move_constructible F, input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<Views>...> &&
            can_reference<invoke_result_t<F&, range_reference_t<Views>...>>
    class zip_transform_view : public view_interface<zip_transform_view<F, Views...>> {
        movable_box<F> fun_;    // exposition only
        zip_view<Views...> zip_;    // exposition only
    };
}

```

```

using InnerView = zip_view<Views...>; // exposition only
template<bool Const>
    using ziperator = iterator_t<maybe-const<Const, InnerView>>; // exposition only
template<bool Const>
    using zentinel = sentinel_t<maybe-const<Const, InnerView>>; // exposition only

// 25.7.26.3, class template zip_transform_view::iterator
template<bool> class iterator; // exposition only

// 25.7.26.4, class template zip_transform_view::sentinel
template<bool> class sentinel; // exposition only

public:
    zip_transform_view() = default;

    constexpr explicit zip_transform_view(F fun, Views... views);

    constexpr auto begin() { return iterator<false>(*this, zip_.begin()); }

    constexpr auto begin() const
        requires range<const InnerView> &&
            regular_invocable<const F&, range_reference_t<const Views>...> {
        return iterator<true>(*this, zip_.begin());
    }

    constexpr auto end() {
        if constexpr (common_range<InnerView>) {
            return iterator<false>(*this, zip_.end());
        } else {
            return sentinel<false>(zip_.end());
        }
    }

    constexpr auto end() const
        requires range<const InnerView> &&
            regular_invocable<const F&, range_reference_t<const Views>...> {
        if constexpr (common_range<const InnerView>) {
            return iterator<true>(*this, zip_.end());
        } else {
            return sentinel<true>(zip_.end());
        }
    }

    constexpr auto size() requires sized_range<InnerView> {
        return zip_.size();
    }

    constexpr auto size() const requires sized_range<const InnerView> {
        return zip_.size();
    }
};

template<class F, class... Rs>
    zip_transform_view(F, Rs&&...) -> zip_transform_view<F, views::all_t<Rs>...>;
}

constexpr explicit zip_transform_view(F fun, Views... views);

```

¹ *Effects:* Initializes *fun_* with `std::move(fun)` and *zip_* with `std::move(views)....`

25.7.26.3 Class template `zip_transform_view::iterator` [range.zip.transform.iterator]

```

namespace std::ranges {
    template<move_constructible F, input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&

```

```

        regular_invocable<F&, range_reference_t<Views>...> &&
        can-reference<invoke_result_t<F&, range_reference_t<Views>...>>
template<bool Const>
class zip_transform_view<F, Views...>::iterator {
    using Parent = maybe-const<Const, zip_transform_view>;           // exposition only
    using Base = maybe-const<Const, InnerView>;                     // exposition only
    Parent* parent_ = nullptr;                                       // exposition only
    ziperator<Const> inner_;                                         // exposition only

    constexpr iterator(Parent& parent, ziperator<Const> inner);      // exposition only

public:
    using iterator_category = see below;                             // not always present
    using iterator_concept = ziperator<Const>::iterator_concept;
    using value_type =
        remove_cvref_t<invoke_result_t<maybe-const<Const, F>&,
                        range_reference_t<maybe-const<Const, Views>>...>>;
    using difference_type = range_difference_t<Base>;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<ziperator<false>, ziperator<Const>>;

    constexpr decltype(auto) operator*() const noexcept(see below);
    constexpr iterator& operator++();
    constexpr void operator++(int);
    iterator operator++(int) requires forward_range<Base> = default;

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type x) requires random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x) requires random_access_range<Base>;

    constexpr decltype(auto) operator[](difference_type n) const
        requires random_access_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<ziperator<Const>>;

    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires random_access_range<Base>;

    friend constexpr iterator operator+(const iterator& i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, const iterator& i)
        requires random_access_range<Base>;
    friend constexpr iterator operator-(const iterator& i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
        requires sized_sentinel_for<ziperator<Const>, ziperator<Const>>;
};
}

```

¹ The member *typedef-name* `iterator::iterator_category` is declared if and only if *Base* models `forward_range`. In that case, `iterator::iterator_category` is defined as follows:

(1.1) — If

`invoke_result_t<maybe-const<Const, F>&, range_reference_t<maybe-const<Const, Views>>...>`
is not a reference, `iterator_category` denotes `input_iterator_tag`.

(1.2) — Otherwise, let *Cs* denote the pack of types `iterator_traits<iterator_t<maybe-const<Const, Views>>>::iterator_category...`

- (1.2.1) — If (derived_from<Cs, random_access_iterator_tag> && ...) is true, iterator_category denotes random_access_iterator_tag.
- (1.2.2) — Otherwise, if (derived_from<Cs, bidirectional_iterator_tag> && ...) is true, iterator_category denotes bidirectional_iterator_tag.
- (1.2.3) — Otherwise, if (derived_from<Cs, forward_iterator_tag> && ...) is true, iterator_category denotes forward_iterator_tag.
- (1.2.4) — Otherwise, iterator_category denotes input_iterator_tag.

```
constexpr iterator (Parent& parent, ziperator<Const> inner);
```

- 2 *Effects:* Initializes *parent_* with addressof(parent) and *inner_* with std::move(inner).

```
constexpr iterator (iterator<!Const> i)
    requires Const && convertible_to<ziperator<false>, ziperator<Const>>;
```

- 3 *Effects:* Initializes *parent_* with i.parent_ and *inner_* with std::move(i.inner_).

```
constexpr decltype(auto) operator*() const noexcept(see below);
```

- 4 *Effects:* Equivalent to:

```
    return apply([&](const auto&... iters) -> decltype(auto) {
        return invoke(*parent_->fun_, *iters...);
    }, inner_.current_);
```

- 5 *Remarks:* Let Is be the pack 0, 1, ..., (sizeof...(Views) - 1). The exception specification is equivalent to: noexcept(invoke(*parent_->fun_, *std::get<Is>(inner_.current_)...)).

```
constexpr iterator& operator++();
```

- 6 *Effects:* Equivalent to:

```
    ++inner_;
    return *this;
```

```
constexpr void operator++(int);
```

- 7 *Effects:* Equivalent to: ++*this.

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

- 8 *Effects:* Equivalent to:

```
    --inner_;
    return *this;
```

```
constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
```

- 9 *Effects:* Equivalent to:

```
    inner_ += x;
    return *this;
```

```
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;
```

- 10 *Effects:* Equivalent to:

```
    inner_ -= x;
    return *this;
```

```
constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_range<Base>;
```

- 11 *Effects:* Equivalent to:

```
    return apply([&]<class... Is>(const Is&... iters) -> decltype(auto) {
        return invoke(*parent_->fun_, iters[iter_difference_t<Is>(n)]...);
    }, inner_.current_);
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<ziperator<Const>>;
```

```

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
12     Let op be the operator.
13     Effects: Equivalent to: return x.inner_ op y.inner_;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
14     Effects: Equivalent to: return iterator(*i.parent_, i.inner_ + n);

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
15     Effects: Equivalent to: return iterator(*i.parent_, i.inner_ - n);

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<ziperator<Const>, ziperator<Const>>;
16     Effects: Equivalent to: return x.inner_ - y.inner_;

25.7.26.4 Class template zip_transform_view::sentinel [range.zip.transform.sentinel]
namespace std::ranges {
    template<move_constructible F, input_range... Views>
        requires (view<Views> && ...) && (sizeof...(Views) > 0) && is_object_v<F> &&
            regular_invocable<F&, range_reference_t<Views>...> &&
            can_reference_invoke_result_t<F&, range_reference_t<Views>...>
    template<bool Const>
    class zip_transform_view<F, Views...>::sentinel {
        zentinel<Const> inner_; // exposition only
        constexpr explicit sentinel(zentinel<Const> inner); // exposition only

    public:
        sentinel() = default;
        constexpr sentinel(sentinel<!Const> i)
            requires Const && convertible_to<zentinel<false>, zentinel<Const>>;

        template<bool OtherConst>
            requires sentinel_for<zentinel<Const>, ziperator<OtherConst>>
        friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
        friend constexpr range_difference_t<maybe_const<OtherConst, InnerView>>
            operator-(const iterator<OtherConst>& x, const sentinel& y);

        template<bool OtherConst>
            requires sized_sentinel_for<zentinel<Const>, ziperator<OtherConst>>
        friend constexpr range_difference_t<maybe_const<OtherConst, InnerView>>
            operator-(const sentinel& x, const iterator<OtherConst>& y);
    };
}

constexpr explicit sentinel(zentinel<Const> inner);
1     Effects: Initializes inner_ with inner.

constexpr sentinel(sentinel<!Const> i)
    requires Const && convertible_to<zentinel<false>, zentinel<Const>>;
2     Effects: Initializes inner_ with std::move(i.inner_).

```

```
template<bool OtherConst>
    requires sentinel_for<z Sentinel<Const>, ziperator<OtherConst>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const Sentinel& y);
```

3 *Effects:* Equivalent to: `return x.inner_ == y.inner_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<z Sentinel<Const>, ziperator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
    operator-(const iterator<OtherConst>& x, const Sentinel& y);
template<bool OtherConst>
    requires sized_sentinel_for<z Sentinel<Const>, ziperator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
    operator-(const Sentinel& x, const iterator<OtherConst>& y);
```

4 *Effects:* Equivalent to: `return x.inner_ - y.inner_;`

25.7.27 Adjacent view

[range.adjacent]

25.7.27.1 Overview

[range.adjacent.overview]

1 `adjacent_view` takes a view and produces a view whose M^{th} element is a tuple of references to the M^{th} through $(M + N - 1)^{\text{th}}$ elements of the original view. If the original view has fewer than N elements, the resulting view is empty.

2 The name `views::adjacent<N>` denotes a range adaptor object (25.7.2). Given a subexpression E and a constant expression N , the expression `views::adjacent<N>(E)` is expression-equivalent to

(2.1) — `((void)E, auto{views::empty<tuple<>>})` if N is equal to 0 and `decltype((E))` models `forward_range`,

(2.2) — otherwise, `adjacent_view<views::all_t<decltype((E))>, N>(E)`.

[Example 1:

```
vector v = {1, 2, 3, 4};
```

```
for (auto i : v | views::adjacent<2>) {
    cout << "(" << std::get<0>(i) << ", " << std::get<1>(i) << ") "; // prints (1, 2) (2, 3) (3, 4)
}
```

— end example]

3 Define *REPEAT*(T , N) as a pack of N types, each of which denotes the same type as T .

25.7.27.2 Class template `adjacent_view`

[range.adjacent.view]

```
namespace std::ranges {
    template<forward_range V, size_t N>
        requires view<V> && (N > 0)
    class adjacent_view : public view_interface<adjacent_view<V, N>> {
        V base_ = V(); // exposition only

        // 25.7.27.3, class template adjacent_view::iterator
        template<bool> class iterator; // exposition only

        // 25.7.27.4, class template adjacent_view::sentinel
        template<bool> class sentinel; // exposition only

        struct as-sentinel{}; // exposition only

    public:
        adjacent_view() requires default_initializable<V> = default;
        constexpr explicit adjacent_view(V base_);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>) {
            return iterator<false>(ranges::begin(base_), ranges::end(base_));
        }
    }
```

```
constexpr auto begin() const requires range<const V> {
    return iterator<true>(ranges::begin(base_), ranges::end(base_));
}

constexpr auto end() requires (!simple-view<V>) {
    if constexpr (common_range<V>) {
        return iterator<false>(as-sentinel{}, ranges::begin(base_), ranges::end(base_));
    } else {
        return sentinel<false>(ranges::end(base_));
    }
}

constexpr auto end() const requires range<const V> {
    if constexpr (common_range<const V>) {
        return iterator<true>(as-sentinel{}, ranges::begin(base_), ranges::end(base_));
    } else {
        return sentinel<true>(ranges::end(base_));
    }
}

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;

constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};
}
```

```
constexpr explicit adjacent_view(V base);
```

1 *Effects:* Initializes *base_* with `std::move(base)`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

2 *Effects:* Equivalent to:

```
using ST = decltype(ranges::size(base_));
using CT = common_type_t<ST, size_t>;
auto sz = static_cast<CT>(ranges::size(base_));
sz -= std::min<CT>(sz, N - 1);
return static_cast<ST>(sz);
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

3 *Effects:* Equivalent to:

```
using DT = range_difference_t<decltype((base_))>;
using CT = common_type_t<DT, size_t>;
auto sz = static_cast<CT>(ranges::reserve_hint(base_));
sz -= std::min<CT>(sz, N - 1);
return to-unsigned-like(sz);
```

25.7.27.3 Class template `adjacent_view::iterator`

[range.adjacent.iterator]

```
namespace std::ranges {
    template<forward_range V, size_t N>
        requires view<V> && (N > 0)
        template<bool Const>
        class adjacent_view<V, N>::iterator {
            using Base = maybe_const<Const, V>;                                // exposition only
            array<iterator_t<Base>, N> current_ = array<iterator_t<Base>, N>();    // exposition only
            constexpr iterator(iterator_t<Base> first, sentinel_t<Base> last);    // exposition only
            constexpr iterator(as-sentinel, iterator_t<Base> first, iterator_t<Base> last);    // exposition only

        public:
            using iterator_category = input_iterator_tag;
```

```

using iterator_concept = see below;
using value_type = tuple<REPEAT(range_value_t<Base>, N)...>;
using difference_type = range_difference_t<Base>;

iterator() = default;
constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

constexpr auto operator*() const;
constexpr iterator& operator++();
iterator operator++(int) = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
iterator operator--(int) = default;

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;

constexpr auto operator[](difference_type n) const
    requires random_access_range<Base>;

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

friend constexpr auto iter_move(const iterator& i) noexcept(see below);
friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires indirectly_swappable<iterator_t<Base>>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *forward_iterator_tag*.

² If the invocation of any non-const member function of *iterator* exits via an exception, the *iterator* acquires a singular value.

```

constexpr iterator(iterator_t<Base> first, sentinel_t<Base> last);
3   Postconditions: current_[0] == first is true, and for every integer  $1 \leq i < N$ , current_[i] ==
   ranges::next(current_[i-1], 1, last) is true.

constexpr iterator(as-sentinel, iterator_t<Base> first, iterator_t<Base> last);
4   Postconditions: If Base does not model bidirectional_range, each element of current_ is equal to
   last. Otherwise, current_[N-1] == last is true, and for every integer  $0 \leq i < (N-1)$ , current_[i]
   == ranges::prev(current_[i+1], 1, first) is true.

constexpr iterator(iterator<!Const> i)
   requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
5   Effects: Initializes each element of current_ with the corresponding element of i.current_ as an
   xvalue.

constexpr auto operator*() const;
6   Effects: Equivalent to:
   return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);

constexpr iterator& operator++();
7   Preconditions: current_.back() is incrementable.
8   Postconditions: Each element of current_ is equal to ranges::next(i), where i is the value of that
   element before the call.
9   Returns: *this.

constexpr iterator& operator--() requires bidirectional_range<Base>;
10  Preconditions: current_.front() is decrementable.
11  Postconditions: Each element of current_ is equal to ranges::prev(i), where i is the value of that
   element before the call.
12  Returns: *this.

constexpr iterator& operator+=(difference_type x)
   requires random_access_range<Base>;
13  Preconditions: current_.back() + x has well-defined behavior.
14  Postconditions: Each element of current_ is equal to i + x, where i is the value of that element
   before the call.
15  Returns: *this.

constexpr iterator& operator-=(difference_type x)
   requires random_access_range<Base>;
16  Preconditions: current_.front() - x has well-defined behavior.
17  Postconditions: Each element of current_ is equal to i - x, where i is the value of that element
   before the call.
18  Returns: *this.

constexpr auto operator[](difference_type n) const
   requires random_access_range<Base>;
19  Effects: Equivalent to:
   return tuple-transform([&](auto& i) -> decltype(auto) { return i[n]; }, current_);

friend constexpr bool operator==(const iterator& x, const iterator& y);
20  Returns: x.current_.back() == y.current_.back().

friend constexpr bool operator<(const iterator& x, const iterator& y)
   requires random_access_range<Base>;
21  Returns: x.current_.back() < y.current_.back().

```

```

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
22     Effects: Equivalent to: return y < x;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
23     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
24     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;
25     Returns: x.current_.back() <=> y.current_.back().

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
26     Effects: Equivalent to:
        auto r = i;
        r += n;
        return r;

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
27     Effects: Equivalent to:
        auto r = i;
        r -= n;
        return r;

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
28     Effects: Equivalent to: return x.current_.back() - y.current_.back();

friend constexpr auto iter_move(const iterator& i) noexcept(see below);
29     Effects: Equivalent to: return tuple-transform(ranges::iter_move, i.current_);
30     Remarks: The exception specification is equivalent to:
        noexcept(ranges::iter_move(declval<const iterator_t<Base>&>())) &&
        is_nothrow_move_constructible_v<range_rvalue_reference_t<Base>>

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires indirectly_swappable<iterator_t<Base>>;
31     Preconditions: None of the iterators in l.current_ is equal to an iterator in r.current_.
32     Effects: For every integer  $0 \leq i < N$ , performs ranges::iter_swap(l.current_[i], r.current_[i]).
33     Remarks: The exception specification is equivalent to:
        noexcept(ranges::iter_swap(declval<iterator_t<Base>>(), declval<iterator_t<Base>>()))

```

25.7.27.4 Class template adjacent_view::sentinel

[range.adjacent.sentinel]

```

namespace std::ranges {
    template<forward_range V, size_t N>
        requires view<V> && (N > 0)
        template<bool Const>
        class adjacent_view<V, N>::sentinel {
            using Base = maybe_const<Const, V>;
            sentinel_t<Base> end_ = sentinel_t<Base>();
        };
}
// exposition only
// exposition only

```

```
constexpr explicit sentinel(sentinel_t<Base> end);           // exposition only

public:
    sentinel() = default;
    constexpr sentinel(sentinel_t<!Const> i)
        requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

    template<bool OtherConst>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
        operator-(const iterator_t<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
    friend constexpr range_difference_t<maybe-const<OtherConst, V>>
        operator-(const sentinel& y, const iterator_t<OtherConst>& x);
};
}
```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

1 *Effects:* Initializes *end_* with *end*.

```
constexpr sentinel(sentinel_t<!Const> i)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
```

2 *Effects:* Initializes *end_* with `std::move(i.end_)`.

```
template<bool OtherConst>
    requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr bool operator==(const iterator_t<OtherConst>& x, const sentinel& y);
```

3 *Effects:* Equivalent to: `return x.current_.back() == y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const iterator_t<OtherConst>& x, const sentinel& y);
```

4 *Effects:* Equivalent to: `return x.current_.back() - y.end_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
friend constexpr range_difference_t<maybe-const<OtherConst, V>>
    operator-(const sentinel& y, const iterator_t<OtherConst>& x);
```

5 *Effects:* Equivalent to: `return y.end_ - x.current_.back();`

25.7.28 Adjacent transform view

[range.adjacent.transform]

25.7.28.1 Overview

[range.adjacent.transform.overview]

- 1 `adjacent_transform_view` takes an invocable object and a view and produces a view whose M^{th} element is the result of applying the invocable object to the M^{th} through $(M + N - 1)^{\text{th}}$ elements of the original view. If the original view has fewer than N elements, the resulting view is empty.
- 2 The name `views::adjacent_transform<N>` denotes a range adaptor object (25.7.2). Given subexpressions *E* and *F* and a constant expression *N*:
 - (2.1) — If *N* is equal to 0 and `decltype((E))` models `forward_range`, `views::adjacent_transform<N>(E, F)` is expression-equivalent to `((void)E, views::zip_transform(F))`, except that the evaluations of *E* and *F* are indeterminately sequenced.
 - (2.2) — Otherwise, the expression `views::adjacent_transform<N>(E, F)` is expression-equivalent to `adjacent_transform_view<views::all_t<decltype((E))>, decay_t<decltype((F))>, N>(E, F)`.

³ [Example 1:

```
vector v = {1, 2, 3, 4};

for (auto i : v | views::adjacent_transform<2>(std::multiplies())) {
    cout << i << ' ';    // prints 2 6 12
}
```

— end example]

25.7.28.2 Class template `adjacent_transform_view` [range.adjacent.transform.view]

```
namespace std::ranges {
    template<forward_range V, move_constructible F, size_t N>
        requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can_reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
    class adjacent_transform_view : public view_interface<adjacent_transform_view<V, F, N>> {
        movable_box<F> fun_;                                // exposition only
        adjacent_view<V, N> inner_;                         // exposition only

        using InnerView = adjacent_view<V, N>;             // exposition only
        template<bool Const>
            using inner_iterator = iterator_t<maybe_const<Const, InnerView>>; // exposition only
        template<bool Const>
            using inner_sentinel = sentinel_t<maybe_const<Const, InnerView>>; // exposition only

        // 25.7.28.3, class template adjacent_transform_view::iterator
        template<bool> class iterator;                      // exposition only

        // 25.7.28.4, class template adjacent_transform_view::sentinel
        template<bool> class sentinel;                     // exposition only

    public:
        adjacent_transform_view() = default;
        constexpr explicit adjacent_transform_view(V base, F fun);

        constexpr V base() const & requires copy_constructible<V> { return inner_.base(); }
        constexpr V base() && { return std::move(inner_).base(); }

        constexpr auto begin() {
            return iterator<false>(*this, inner_.begin());
        }

        constexpr auto begin() const
            requires range<const InnerView> &&
            regular_invocable<const F&, REPEAT(range_reference_t<const V>, N)...> {
            return iterator<true>(*this, inner_.begin());
        }

        constexpr auto end() {
            if constexpr (common_range<InnerView>) {
                return iterator<false>(*this, inner_.end());
            } else {
                return sentinel<false>(inner_.end());
            }
        }

        constexpr auto end() const
            requires range<const InnerView> &&
            regular_invocable<const F&, REPEAT(range_reference_t<const V>, N)...> {
            if constexpr (common_range<const InnerView>) {
                return iterator<true>(*this, inner_.end());
            } else {
                return sentinel<true>(inner_.end());
            }
        }
    };
}
```

```

}

constexpr auto size() requires sized_range<InnerView> {
    return inner_.size();
}

constexpr auto size() const requires sized_range<const InnerView> {
    return inner_.size();
}

constexpr auto reserve_hint() requires approximately_sized_range<InnerView> {
    return inner_.reserve_hint();
}

constexpr auto reserve_hint() const requires approximately_sized_range<const InnerView> {
    return inner_.reserve_hint();
}
};
}

```

```
constexpr explicit adjacent_transform_view(V base, F fun);
```

¹ *Effects:* Initializes *fun_* with `std::move(fun)` and *inner_* with `std::move(base)`.

25.7.28.3 Class template `adjacent_transform_view::iterator` [range.adjacent.transform.iterator]

```

namespace std::ranges {
    template<forward_range V, move_constructible F, size_t N>
        requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can_reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
    template<bool Const>
    class adjacent_transform_view<V, F, N>::iterator {
        using Parent = maybe_const<Const, adjacent_transform_view>;           // exposition only
        using Base = maybe_const<Const, V>;                                   // exposition only
        Parent* parent_ = nullptr;                                           // exposition only
        inner_iterator<Const> inner_;                                         // exposition only

        constexpr iterator(Parent& parent, inner_iterator<Const> inner);      // exposition only

    public:
        using iterator_category = see below;
        using iterator_concept = inner_iterator<Const>::iterator_concept;
        using value_type =
            remove_cvref_t<invoke_result_t<maybe_const<Const, F>&,
                REPEAT(range_reference_t<Base>, N)...>>;
        using difference_type = range_difference_t<Base>;

        iterator() = default;
        constexpr iterator(iterator<!Const> i)
            requires Const && convertible_to<inner_iterator<false>, inner_iterator<Const>>;

        constexpr decltype(auto) operator*() const noexcept(see below);
        constexpr iterator& operator++();
        iterator operator++(int) = default;
        constexpr iterator& operator--() requires bidirectional_range<Base>;
        iterator operator--(int) = default;
        constexpr iterator& operator+=(difference_type x) requires random_access_range<Base>;
        constexpr iterator& operator-=(difference_type x) requires random_access_range<Base>;

        constexpr decltype(auto) operator[](difference_type n) const
            requires random_access_range<Base>;
    }
}

```

```

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<inner-iterator<Const>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<inner-iterator<Const>, inner-iterator<Const>>;
};
}

```

¹ The member *typedef-name* *iterator::iterator_category* is defined as follows:

- (1.1) — If *invoke_result_t<maybe-const<Const, F>&, REPEAT(range_reference_t<Base>, N)...>* is not a reference, *iterator_category* denotes *input_iterator_tag*.
- (1.2) — Otherwise, let *C* denote the type *iterator_traits<iterator_t<Base>>::iterator_category*.
 - (1.2.1) — If *derived_from<C, random_access_iterator_tag>* is true, *iterator_category* denotes *random_access_iterator_tag*.
 - (1.2.2) — Otherwise, if *derived_from<C, bidirectional_iterator_tag>* is true, *iterator_category* denotes *bidirectional_iterator_tag*.
 - (1.2.3) — Otherwise, if *derived_from<C, forward_iterator_tag>* is true, *iterator_category* denotes *forward_iterator_tag*.
 - (1.2.4) — Otherwise, *iterator_category* denotes *input_iterator_tag*.

```
constexpr iterator (Parent& parent, inner-iterator<Const> inner);
```

² *Effects*: Initializes *parent_* with *addressof*(parent) and *inner_* with *std::move*(inner).

```
constexpr iterator (iterator<!Const> i)
    requires Const && convertible_to<inner-iterator<false>, inner-iterator<Const>>;
```

³ *Effects*: Initializes *parent_* with *i.parent_* and *inner_* with *std::move*(*i.inner_*).

```
constexpr decltype(auto) operator*() const noexcept(see below);
```

⁴ *Effects*: Equivalent to:

```

return apply([&](const auto&... iters) -> decltype(auto) {
    return invoke(*parent_->fun_, *iters...);
}, inner_.current_);

```

⁵ *Remarks*: Let *Is* be the pack 0, 1, ..., (N - 1). The exception specification is equivalent to:

```
noexcept(invoke(*parent_->fun_, *std::get<Is>(inner_.current_)...))
```

```
constexpr iterator& operator++();
```

⁶ *Effects*: Equivalent to:

```

++inner_;
return *this;

```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

⁷ *Effects*: Equivalent to:

```
--inner_;
```

```

        return *this;

constexpr iterator& operator+=(difference_type x) requires random_access_range<Base>;
8     Effects: Equivalent to:
        inner_ += x;
        return *this;

constexpr iterator& operator-=(difference_type x) requires random_access_range<Base>;
9     Effects: Equivalent to:
        inner_ -= x;
        return *this;

constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_range<Base>;
10    Effects: Equivalent to:
        return apply([&](const auto&... iters) -> decltype(auto) {
            return invoke(*parent_->fun_, iters[n]...);
        }, inner_.current_);

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<inner-iterator<Const>>;

11    Let op be the operator.
12    Effects: Equivalent to: return x.inner_ op y.inner_;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
13    Effects: Equivalent to: return iterator(*i.parent_, i.inner_ + n);

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
14    Effects: Equivalent to: return iterator(*i.parent_, i.inner_ - n);

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<inner-iterator<Const>, inner-iterator<Const>>;
15    Effects: Equivalent to: return x.inner_ - y.inner_;

```

25.7.28.4 Class template adjacent_transform_view::sentinel [range.adjacent.transform.sentinel]

```

namespace std::ranges {
    template<forward_range V, move_constructible F, size_t N>
        requires view<V> && (N > 0) && is_object_v<F> &&
            regular_invocable<F&, REPEAT(range_reference_t<V>, N)...> &&
            can_reference<invoke_result_t<F&, REPEAT(range_reference_t<V>, N)...>>
        template<bool Const>
        class adjacent_transform_view<V, F, N>::sentinel {
            inner-sentinel<Const> inner_; // exposition only
            constexpr explicit sentinel(inner-sentinel<Const> inner); // exposition only

```

```

public:
    sentinel() = default;
    constexpr sentinel(sentinel<!Const> i)
        requires Const && convertible_to<inner-sentinel<false>, inner-sentinel<Const>>;

    template<bool OtherConst>
        requires sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
        requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
        operator-(const iterator<OtherConst>& x, const sentinel& y);

    template<bool OtherConst>
        requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
    friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
        operator-(const sentinel& x, const iterator<OtherConst>& y);
};
}

```

```
constexpr explicit sentinel(inner-sentinel<Const> inner);
```

1 *Effects:* Initializes *inner_* with *inner*.

```
constexpr sentinel(sentinel<!Const> i)
    requires Const && convertible_to<inner-sentinel<false>, inner-sentinel<Const>>;
```

2 *Effects:* Initializes *inner_* with `std::move(i.inner_)`.

```
template<bool OtherConst>
    requires sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y);
```

3 *Effects:* Equivalent to: `return x.inner_ == y.inner_;`

```
template<bool OtherConst>
    requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
    operator-(const iterator<OtherConst>& x, const sentinel& y);
```

```
template<bool OtherConst>
    requires sized_sentinel_for<inner-sentinel<Const>, inner-iterator<OtherConst>>
friend constexpr range_difference_t<maybe-const<OtherConst, InnerView>>
    operator-(const sentinel& x, const iterator<OtherConst>& y);
```

4 *Effects:* Equivalent to: `return x.inner_ - y.inner_;`

25.7.29 Chunk view

[range.chunk]

25.7.29.1 Overview

[range.chunk.overview]

- 1 `chunk_view` takes a view and a number *N* and produces a range of views that are *N*-sized non-overlapping successive chunks of the elements of the original view, in order. The last view in the range can have fewer than *N* elements.
- 2 The name `views::chunk` denotes a range adaptor object (25.7.2). Given subexpressions *E* and *N*, the expression `views::chunk(E, N)` is expression-equivalent to `chunk_view(E, N)`.

[Example 1:

```

vector v = {1, 2, 3, 4, 5};

for (auto r : v | views::chunk(2)) {
    cout << '[';
    auto sep = "";
    for (auto i : r) {
        cout << sep << i;
        sep = ", ";
    }
}

```

```

    }
    cout << "]" ";
}
// The above prints [1, 2] [3, 4] [5]
—end example]

```

25.7.29.2 Class template `chunk_view` for input ranges

[range.chunk.view.input]

```

namespace std::ranges {
    template<class I>
    constexpr I div-ceil(I num, I denom) { // exposition only
        I r = num / denom;
        if (num % denom)
            ++r;
        return r;
    }

    template<view V>
    requires input_range<V>
    class chunk_view : public view_interface<chunk_view<V>> {
        V base_; // exposition only
        range_difference_t<V> n_; // exposition only
        range_difference_t<V> remainder_ = 0; // exposition only

        non-propagating_cache<iterator_t<V>> current_; // exposition only

        // 25.7.29.3, class chunk_view::outer-iterator
        class outer-iterator; // exposition only

        // 25.7.29.5, class chunk_view::inner-iterator
        class inner-iterator; // exposition only

    public:
        constexpr explicit chunk_view(V base, range_difference_t<V> n);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr outer-iterator begin();
        constexpr default_sentinel_t end() const noexcept;

        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;

        constexpr auto reserve_hint() requires approximately_sized_range<V>;
        constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
    };

    template<class R>
    chunk_view(R&&, range_difference_t<R>) -> chunk_view<views::all_t<R>>;
}

```

```
constexpr explicit chunk_view(V base, range_difference_t<V> n);
```

1 *Preconditions:* `n > 0` is true.

2 *Effects:* Initializes `base_` with `std::move(base)` and `n_` with `n`.

```
constexpr outer-iterator begin();
```

3 *Effects:* Equivalent to:

```

    current_ = ranges::begin(base_);
    remainder_ = n_;
    return outer-iterator(*this);

```

```
constexpr default_sentinel_t end() const noexcept;
```

4 *Returns:* default_sentinel.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

5 *Effects:* Equivalent to:

```
return to-unsigned-like(div-ceil(ranges::distance(base_), n_));
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

6 *Effects:* Equivalent to:

```
auto s = static_cast<range_difference_t<decltype((base_))>>(ranges::reserve_hint(base_));
return to-unsigned-like(div-ceil(s, n_));
```

25.7.29.3 Class chunk_view::outer-iterator

[range.chunk.outer.iter]

```
namespace std::ranges {
    template<view V>
        requires input_range<V>
        class chunk_view<V>::outer-iterator {
            chunk_view* parent_; // exposition only

            constexpr explicit outer-iterator(chunk_view& parent); // exposition only

        public:
            using iterator_concept = input_iterator_tag;
            using difference_type = range_difference_t<V>;

            // 25.7.29.4, class chunk_view::outer-iterator::value_type
            struct value_type;

            outer-iterator(outer-iterator&&) = default;
            outer-iterator& operator=(outer-iterator&&) = default;

            constexpr value_type operator*() const;
            constexpr outer-iterator& operator++();
            constexpr void operator++(int);

            friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);

            friend constexpr difference_type operator-(default_sentinel_t y, const outer-iterator& x)
                requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
            friend constexpr difference_type operator-(const outer-iterator& x, default_sentinel_t y)
                requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
        };
}
```

```
constexpr explicit outer-iterator(chunk_view& parent);
```

1 *Effects:* Initializes *parent_* with addressof(*parent*).

```
constexpr value_type operator*() const;
```

2 *Preconditions:* *this == default_sentinel is false.

3 *Returns:* value_type(*parent_).

```
constexpr outer-iterator& operator++();
```

4 *Preconditions:* *this == default_sentinel is false.

5 *Effects:* Equivalent to:

```
ranges::advance(*parent_->current_, parent_->remainder_, ranges::end(parent_->base_));
parent_->remainder_ = parent_->n_;
return *this;
```

```

constexpr void operator++(int);
6   Effects: Equivalent to ++*this.

friend constexpr bool operator==(const outer-iterator& x, default_sentinel_t);
7   Effects: Equivalent to:
    return *x.parent_->current_ == ranges::end(x.parent_->base_) && x.parent_->remainder_ != 0;

friend constexpr difference_type operator-(default_sentinel_t y, const outer-iterator& x)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
8   Effects: Equivalent to:
    const auto dist = ranges::end(x.parent_->base_) - *x.parent_->current_;
    if (dist < x.parent_->remainder_) {
        return dist == 0 ? 0 : 1;
    }
    return div-ceil(dist - x.parent_->remainder_, x.parent_->n_) + 1;

friend constexpr difference_type operator-(const outer-iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
9   Effects: Equivalent to: return -(y - x);

25.7.29.4 Class chunk_view::outer-iterator::value_type [range.chunk.outer.value]

namespace std::ranges {
    template<view V>
        requires input_range<V>
        struct chunk_view<V>::outer-iterator::value_type : view_interface<value_type> {
            private:
                chunk_view* parent_; // exposition only

                constexpr explicit value_type(chunk_view& parent); // exposition only

            public:
                constexpr inner-iterator begin() const noexcept;
                constexpr default_sentinel_t end() const noexcept;

                constexpr auto size() const
                    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

                constexpr auto reserve_hint() const noexcept;
        };
}

constexpr explicit value_type(chunk_view& parent);
1   Effects: Initializes parent_ with addressof(parent).

constexpr inner-iterator begin() const noexcept;
2   Returns: inner-iterator(*parent_).

constexpr default_sentinel_t end() const noexcept;
3   Returns: default_sentinel.

constexpr auto size() const
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
4   Effects: Equivalent to:
    return to-unsigned-like(ranges::min(parent_->remainder_,
                                         ranges::end(parent_->base_) - *parent_->current_));

constexpr auto reserve_hint() const noexcept;
5   Effects: Equivalent to:
    return to-unsigned-like(parent_->remainder_);

```


25.7.29.5 Class `chunk_view::inner-iterator`

[range.chunk.inner.iter]

```

namespace std::ranges {
    template<view V>
        requires input_range<V>
    class chunk_view<V>::inner-iterator {
        chunk_view* parent_;                                // exposition only

        constexpr explicit inner-iterator(chunk_view& parent) noexcept;    // exposition only

    public:
        using iterator_concept = input_iterator_tag;
        using difference_type = range_difference_t<V>;
        using value_type = range_value_t<V>;

        inner-iterator(inner-iterator&&) = default;
        inner-iterator& operator=(inner-iterator&&) = default;

        constexpr const iterator_t<V>& base() const &;

        constexpr range_reference_t<V> operator*() const;
        constexpr inner-iterator& operator++();
        constexpr void operator++(int);

        friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);

        friend constexpr difference_type operator-(default_sentinel_t y, const inner-iterator& x)
            requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
        friend constexpr difference_type operator-(const inner-iterator& x, default_sentinel_t y)
            requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

        friend constexpr range_rvalue_reference_t<V> iter_move(const inner-iterator& i)
            noexcept(noexcept(ranges::iter_move(*i.parent_->current_)));

        friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
            noexcept(noexcept(ranges::iter_swap(*x.parent_->current_, *y.parent_->current_)))
            requires indirectly_swappable<iterator_t<V>>;
    };
}

```

```
constexpr explicit inner-iterator(chunk_view& parent) noexcept;
```

1 *Effects:* Initializes *parent_* with `addressof(parent)`.

```
constexpr const iterator_t<V>& base() const &;
```

2 *Effects:* Equivalent to: `return *parent_->current_;`

```
constexpr range_reference_t<V> operator*() const;
```

3 *Preconditions:* `*this == default_sentinel` is false.

4 *Effects:* Equivalent to: `return **parent_->current_;`

```
constexpr inner-iterator& operator++();
```

5 *Preconditions:* `*this == default_sentinel` is false.

6 *Effects:* Equivalent to:

```

    ++*parent_->current_;
    if (*parent_->current_ == ranges::end(parent_->base_))
        parent_->remainder_ = 0;
    else
        --parent_->remainder_;
    return *this;

```

```
constexpr void operator++(int);
```

7 *Effects:* Equivalent to `+++this`.

```
friend constexpr bool operator==(const inner-iterator& x, default_sentinel_t);
```

8 *Returns:* `x.parent_->remainder_ == 0`.

```
friend constexpr difference_type operator-(default_sentinel_t y, const inner-iterator& x)
requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

9 *Effects:* Equivalent to:

```
return ranges::min(x.parent_->remainder_,
                  ranges::end(x.parent_->base_) - *x.parent_->current_);
```

```
friend constexpr difference_type operator-(const inner-iterator& x, default_sentinel_t y)
requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

10 *Effects:* Equivalent to: `return -(y - x);`

```
friend constexpr range_rvalue_reference_t<V> iter_move(const inner-iterator& i)
noexcept(noexcept(ranges::iter_move(*i.parent_->current_)));
```

11 *Effects:* Equivalent to: `return ranges::iter_move(*i.parent_->current_);`

```
friend constexpr void iter_swap(const inner-iterator& x, const inner-iterator& y)
noexcept(noexcept(ranges::iter_swap(*x.parent_->current_, *y.parent_->current_)))
requires indirectly_swappable<iterator_t<V>>;
```

12 *Effects:* Equivalent to: `ranges::iter_swap(*x.parent_->current_, *y.parent_->current_);`

25.7.29.6 Class template `chunk_view` for forward ranges [range.chunk.view.fwd]

```
namespace std::ranges {
    template<view V>
    requires forward_range<V>
    class chunk_view<V> : public view_interface<chunk_view<V>> {
        V base_; // exposition only
        range_difference_t<V> n_; // exposition only

        // 25.7.29.7, class template chunk_view::iterator
        template<bool> class iterator; // exposition only

    public:
        constexpr explicit chunk_view(V base, range_difference_t<V> n);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>) {
            return iterator<false>(this, ranges::begin(base_));
        }

        constexpr auto begin() const requires forward_range<const V> {
            return iterator<true>(this, ranges::begin(base_));
        }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (common_range<V> && sized_range<V>) {
                auto missing = (n_ - ranges::distance(base_) % n_) % n_;
                return iterator<false>(this, ranges::end(base_), missing);
            } else if constexpr (common_range<V> && !bidirectional_range<V>) {
                return iterator<false>(this, ranges::end(base_));
            } else {
                return default_sentinel;
            }
        }

        constexpr auto end() const requires forward_range<const V> {
            if constexpr (common_range<const V> && sized_range<const V>) {
                auto missing = (n_ - ranges::distance(base_) % n_) % n_;
                return iterator<true>(this, ranges::end(base_), missing);
            }
        }
    };
}
```

```

    } else if constexpr (common_range<const V> && !bidirectional_range<const V>) {
        return iterator<true>(this, ranges::end(base_));
    } else {
        return default_sentinel;
    }
}

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;

constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};
}

```

```
constexpr explicit chunk_view(V base, range_difference_t<V> n);
```

1 *Preconditions:* $n > 0$ is true.

2 *Effects:* Initializes *base_* with `std::move(base)` and *n_* with *n*.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

3 *Effects:* Equivalent to:

```
    return to-unsigned-like(div-ceil(ranges::distance(base_), n_));
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

4 *Effects:* Equivalent to:

```
    auto s = static_cast<range_difference_t<decltype((base_))>>(ranges::reserve_hint(base_));
    return to-unsigned-like(div-ceil(s, n_));
```

25.7.29.7 Class template `chunk_view::iterator` for forward ranges [range.chunk.fwd.iter]

```

namespace std::ranges {
    template<view V>
        requires forward_range<V>
    template<bool Const>
    class chunk_view<V>::iterator {
        using Parent = maybe_const<Const, chunk_view>;
        using Base = maybe_const<Const, V>;

        iterator_t<Base> current_ = iterator_t<Base>();
        sentinel_t<Base> end_ = sentinel_t<Base>();
        range_difference_t<Base> n_ = 0;
        range_difference_t<Base> missing_ = 0;

        constexpr iterator(Parent* parent, iterator_t<Base> current,
                           range_difference_t<Base> missing = 0);

    public:
        using iterator_category = input_iterator_tag;
        using iterator_concept = see_below;
        using value_type = decltype(views::take(subrange(current_, end_), n_));
        using difference_type = range_difference_t<Base>;

        iterator() = default;
        constexpr iterator(iterator<!Const> i)
            requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
                && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr iterator_t<Base> base() const;

        constexpr value_type operator*() const;
        constexpr iterator& operator++();
    };
}

```

```

    iterator operator++(int) = default;

constexpr iterator& operator--() requires bidirectional_range<Base>;
iterator operator--(int) = default;

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;

constexpr value_type operator[](difference_type n) const
    requires random_access_range<Base>;

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *forward_iterator_tag*.

```

constexpr iterator (Parent* parent, iterator_t<Base> current,
                    range_difference_t<Base> missing = 0);

```

- ² *Effects*: Initializes *current_* with *current*, *end_* with *ranges::end(parent->base_)*, *n_* with *parent->n_*, and *missing_* with *missing*.

```

constexpr iterator (iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
        && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

- ³ *Effects*: Initializes *current_* with *std::move(i.current_)*, *end_* with *std::move(i.end_)*, *n_* with *i.n_*, and *missing_* with *i.missing_*.

```

constexpr iterator_t<Base> base() const;
4   Returns: current_.

constexpr value_type operator*() const;
5   Preconditions: current_ != end_ is true.
6   Returns: views::take(subrange(current_, end_), n_).

constexpr iterator& operator++();
7   Preconditions: current_ != end_ is true.
8   Effects: Equivalent to:
    missing_ = ranges::advance(current_, n_, end_);
    return *this;

constexpr iterator& operator--() requires bidirectional_range<Base>;
9   Effects: Equivalent to:
    ranges::advance(current_, missing_ - n_);
    missing_ = 0;
    return *this;

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
10  Preconditions: If x is positive, ranges::distance(current_, end_) > n_ * (x - 1) is true.
    [Note 1: If x is negative, the Effects paragraph implies a precondition. — end note]
11  Effects: Equivalent to:
    if (x > 0) {
        ranges::advance(current_, n_ * (x - 1));
        missing_ = ranges::advance(current_, n_, end_);
    } else if (x < 0) {
        ranges::advance(current_, n_ * x + missing_);
        missing_ = 0;
    }
    return *this;

constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;
12  Effects: Equivalent to: return *this += -x;

constexpr value_type operator[](difference_type n) const
    requires random_access_range<Base>;
13  Returns: *(this + n).

friend constexpr bool operator==(const iterator& x, const iterator& y);
14  Returns: x.current_ == y.current_.

friend constexpr bool operator==(const iterator& x, default_sentinel_t);
15  Returns: x.current_ == x.end_.

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
16  Returns: x.current_ < y.current_.

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
17  Effects: Equivalent to: return y < x;

```

```

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
18     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
19     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;
20     Returns: x.current_ <=> y.current_.

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
21     Effects: Equivalent to:
        auto r = i;
        r += n;
        return r;

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
22     Effects: Equivalent to:
        auto r = i;
        r -= n;
        return r;

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
23     Returns: (x.current_ - y.current_ + x.missing_ - y.missing_) / x.n_.

friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
24     Returns: div-ceil(x.end_ - x.current_, x.n_).

friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
25     Effects: Equivalent to: return -(y - x);

```

25.7.30 Slide view

[range.slide]

25.7.30.1 Overview

[range.slide.overview]

- 1 `slide_view` takes a view and a number N and produces a view whose M^{th} element is a view over the M^{th} through $(M + N - 1)^{\text{th}}$ elements of the original view. If the original view has fewer than N elements, the resulting view is empty.
- 2 The name `views::slide` denotes a range adaptor object (25.7.2). Given subexpressions E and N , the expression `views::slide(E, N)` is expression-equivalent to `slide_view(E, N)`.

[Example 1:

```

vector v = {1, 2, 3, 4};

for (auto i : v | views::slide(2)) {
    cout << '[' << i[0] << ", " << i[1] << "]" << " ";           // prints [1, 2] [2, 3] [3, 4]
}

```

— end example]

25.7.30.2 Class template `slide_view`

[range.slide.view]

```

namespace std::ranges {
    template<class V>
    concept slide-caches-nothing = random_access_range<V> && sized_range<V>;           // exposition only

    template<class V>
    concept slide-caches-last =                                                         // exposition only
        !slide-caches-nothing<V> && bidirectional_range<V> && common_range<V>;

    template<class V>
    concept slide-caches-first =                                                         // exposition only
        !slide-caches-nothing<V> && !slide-caches-last<V>;

    template<forward_range V>
    requires view<V>
    class slide_view : public view_interface<slide_view<V>> {
        V base_;                               // exposition only
        range_difference_t<V> n_;               // exposition only

        // 25.7.30.3, class template slide_view::iterator
        template<bool> class iterator;           // exposition only

        // 25.7.30.4, class slide_view::sentinel
        class sentinel;                           // exposition only

    public:
        constexpr explicit slide_view(V base, range_difference_t<V> n);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin()
            requires (!(simple-view<V> && slide-caches-nothing<const V>));
        constexpr auto begin() const requires slide-caches-nothing<const V>;

        constexpr auto end()
            requires (!(simple-view<V> && slide-caches-nothing<const V>));
        constexpr auto end() const requires slide-caches-nothing<const V>;

        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;

        constexpr auto reserve_hint() requires approximately_sized_range<V>;
        constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
    };

    template<class R>
    slide_view(R&&, range_difference_t<R>) -> slide_view<views::all_t<R>>;
}

```

```
constexpr explicit slide_view(V base, range_difference_t<V> n);
```

1 *Preconditions:* $n > 0$ is true.

2 *Effects:* Initializes `base_` with `std::move(base)` and `n_` with `n`.

```
constexpr auto begin()
    requires (!(simple-view<V> && slide-caches-nothing<const V>));
```

3 *Returns:*

- (3.1) — If `V` models *slide-caches-first*,
- `iterator<false>(ranges::begin(base_),`
`ranges::next(ranges::begin(base_), n_ - 1, ranges::end(base_)), n_)`
- (3.2) — Otherwise, `iterator<false>(ranges::begin(base_), n_)`.

4 *Remarks:* In order to provide the amortized constant-time complexity required by the `range` concept, this function caches the result within the `slide_view` for use on subsequent calls when `V` models *slide-caches-first*.

```
constexpr auto begin() const requires slide-caches-nothing<const V>;
```

5 *Returns:* `iterator<true>(ranges::begin(base_), n_)`.

```
constexpr auto end()
requires (!simple-view<V> && slide-caches-nothing<const V>);
```

6 *Returns:*

- (6.1) — If `V` models *slide-caches-nothing*,
`iterator<false>(ranges::begin(base_) + range_difference_t<V>(size()), n_)`
- (6.2) — Otherwise, if `V` models *slide-caches-last*,
`iterator<false>(ranges::prev(ranges::end(base_), n_ - 1, ranges::begin(base_)), n_)`
- (6.3) — Otherwise, if `V` models *common_range*,
`iterator<false>(ranges::end(base_), ranges::end(base_), n_)`
- (6.4) — Otherwise, *sentinel* (`ranges::end(base_)`).

7 *Remarks:* In order to provide the amortized constant-time complexity required by the `range` concept, this function caches the result within the `slide_view` for use on subsequent calls when `V` models *slide-caches-last*.

```
constexpr auto end() const requires slide-caches-nothing<const V>;
```

8 *Returns:* `begin() + range_difference_t<const V>(size())`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

9 *Effects:* Equivalent to:

```
auto sz = ranges::distance(base_) - n_ + 1;
if (sz < 0) sz = 0;
return to-unsigned-like(sz);
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

10 *Effects:* Equivalent to:

```
auto sz = static_cast<range_difference_t<decltype((base_))>>(ranges::reserve_hint(base_)) -
n_ + 1;
if (sz < 0) sz = 0;
return to-unsigned-like(sz);
```

25.7.30.3 Class template `slide_view::iterator`

[range.slide.iterator]

```
namespace std::ranges {
template<forward_range V>
requires view<V>
template<bool Const>
class slide_view<V>::iterator {
using Base = maybe-const<Const, V>; // exposition only
iterator_t<Base> current_ = iterator_t<Base>(); // exposition only
iterator_t<Base> last_ele_ = iterator_t<Base>(); // exposition only,
// present only if Base models slide-caches-first
range_difference_t<Base> n_ = 0; // exposition only

constexpr iterator(iterator_t<Base> current, range_difference_t<Base> n) // exposition only
requires (!slide-caches-first<Base>);

constexpr iterator(iterator_t<Base> current, iterator_t<Base> last_ele, // exposition only
range_difference_t<Base> n)
requires slide-caches-first<Base>;
```



```

public:
    using iterator_category = input_iterator_tag;
    using iterator_concept = see below;
    using value_type = decltype(views::counted(current_, n_));
    using difference_type = range_difference_t<Base>;

    iterator() = default;
    constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

    constexpr auto operator*() const;
    constexpr iterator& operator++();
    iterator operator++(int) = default;

    constexpr iterator& operator--() requires bidirectional_range<Base>;
    iterator operator--(int) = default;

    constexpr iterator& operator+=(difference_type x)
        requires random_access_range<Base>;
    constexpr iterator& operator-=(difference_type x)
        requires random_access_range<Base>;

    constexpr auto operator[](difference_type n) const
        requires random_access_range<Base>;

    friend constexpr bool operator==(const iterator& x, const iterator& y);

    friend constexpr bool operator<(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator<=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr bool operator>=(const iterator& x, const iterator& y)
        requires random_access_range<Base>;
    friend constexpr auto operator<=>(const iterator& x, const iterator& y)
        requires random_access_range<Base> &&
            three_way_comparable<iterator_t<Base>>;

    friend constexpr iterator operator+(const iterator& i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr iterator operator+(difference_type n, const iterator& i)
        requires random_access_range<Base>;
    friend constexpr iterator operator-(const iterator& i, difference_type n)
        requires random_access_range<Base>;
    friend constexpr difference_type operator-(const iterator& x, const iterator& y)
        requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, *iterator_concept* denotes *forward_iterator_tag*.

² If the invocation of any non-const member function of *iterator* exits via an exception, the *iterator* acquires a singular value.

```

constexpr iterator(iterator_t<Base> current, range_difference_t<Base> n)
    requires (!slide-caches-first<Base>);

```

³ *Effects*: Initializes *current_* with *current* and *n_* with *n*.

```

constexpr iterator(iterator_t<Base> current, iterator_t<Base> last_ele,
                    range_difference_t<Base> n)
    requires slide-caches-first<Base>;
4     Effects: Initializes current_ with current, last_ele_ with last_ele, and n_ with n.

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
5     Effects: Initializes current_ with std::move(i.current_) and n_ with i.n_.
    [Note 1: iterator<true> can only be formed when Base models slide-caches-nothing, in which case
    last_ele_ is not present. — end note]

constexpr auto operator*() const;
6     Returns: views::counted(current_, n_).

constexpr iterator& operator++();
7     Preconditions: current_ and last_ele_ (if present) are incrementable.
8     Postconditions: current_ and last_ele_ (if present) are each equal to ranges::next(i), where i is
    the value of that data member before the call.
9     Returns: *this.

constexpr iterator& operator--() requires bidirectional_range<Base>;
10    Preconditions: current_ and last_ele_ (if present) are decrementable.
11    Postconditions: current_ and last_ele_ (if present) are each equal to ranges::prev(i), where i is
    the value of that data member before the call.
12    Returns: *this.

constexpr iterator& operator+=(difference_type x)
    requires random_access_range<Base>;
13    Preconditions: current_ + x and last_ele_ + x (if last_ele_ is present) have well-defined behavior.
14    Postconditions: current_ and last_ele_ (if present) are each equal to i + x, where i is the value of
    that data member before the call.
15    Returns: *this.

constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;
16    Preconditions: current_ - x and last_ele_ - x (if last_ele_ is present) have well-defined behavior.
17    Postconditions: current_ and last_ele_ (if present) are each equal to i - x, where i is the value of
    that data member before the call.
18    Returns: *this.

constexpr auto operator[](difference_type n) const
    requires random_access_range<Base>;
19    Effects: Equivalent to: return views::counted(current_ + n, n_);

friend constexpr bool operator==(const iterator& x, const iterator& y);
20    Returns: If last_ele_ is present, x.last_ele_ == y.last_ele_; otherwise, x.current_ == y.current_.

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
21    Returns: x.current_ < y.current_.

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
22    Effects: Equivalent to: return y < x;

```

```

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
23     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
24     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;
25     Returns: x.current_ <=> y.current_.

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
26     Effects: Equivalent to:
        auto r = i;
        r += n;
        return r;

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
27     Effects: Equivalent to:
        auto r = i;
        r -= n;
        return r;

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
28     Returns: If last_ele_ is present, x.last_ele_ - y.last_ele_; otherwise, x.current_ - y.current_.

```

25.7.30.4 Class slide_view::sentinel

[range.slide.sentinel]

```

namespace std::ranges {
    template<forward_range V>
        requires view<V>
        class slide_view<V>::sentinel {
            sentinel_t<V> end_ = sentinel_t<V>();           // exposition only
            constexpr explicit sentinel(sentinel_t<V> end);   // exposition only

        public:
            sentinel() = default;

            friend constexpr bool operator==(const iterator<false>& x, const sentinel& y);

            friend constexpr range_difference_t<V>
                operator-(const iterator<false>& x, const sentinel& y)
                    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;

            friend constexpr range_difference_t<V>
                operator-(const sentinel& y, const iterator<false>& x)
                    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
        };
}

```

¹ [Note 1: *sentinel* is used only when *slide-caches-first*<V> is true. — end note]

```
constexpr explicit sentinel(sentinel_t<V> end);
```

² *Effects:* Initializes *end_* with end.

```

friend constexpr bool operator==(const iterator<false>& x, const sentinel& y);
3     Returns: x.last_ele_ == y.end_.

friend constexpr range_difference_t<V>
operator-(const iterator<false>& x, const sentinel& y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
4     Returns: x.last_ele_ - y.end_.

friend constexpr range_difference_t<V>
operator-(const sentinel& y, const iterator<false>& x)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
5     Returns: y.end_ - x.last_ele_.

```

25.7.31 Chunk by view

[range.chunk.by]

25.7.31.1 Overview

[range.chunk.by.overview]

- 1 chunk_by_view takes a view and a predicate, and splits the view into subranges between each pair of adjacent elements for which the predicate returns false.
- 2 The name views::chunk_by denotes a range adaptor object (25.7.2). Given subexpressions E and F, the expression views::chunk_by(E, F) is expression-equivalent to chunk_by_view(E, F).

[Example 1:

```

vector v = {1, 2, 2, 3, 0, 4, 5, 2};

for (auto r : v | views::chunk_by(ranges::less_equal{})) {
    cout << '[';
    auto sep = "";
    for (auto i : r) {
        cout << sep << i;
        sep = ", ";
    }
    cout << "]" ";
}
// The above prints [1, 2, 2, 3] [0, 4, 5] [2]

```

— end example]

25.7.31.2 Class template chunk_by_view

[range.chunk.by.view]

```

namespace std::ranges {
    template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
    class chunk_by_view : public view_interface<chunk_by_view<V, Pred>> {
        V base_ = V(); // exposition only
        movable_box<Pred> pred_; // exposition only

        // 25.7.31.3, class chunk_by_view::iterator
        class iterator; // exposition only

    public:
        chunk_by_view() requires default_initializable<V> && default_initializable<Pred> = default;
        constexpr explicit chunk_by_view(V base, Pred pred);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr const Pred& pred() const;

        constexpr iterator begin();
        constexpr auto end();

        constexpr iterator_t<V> find_next(iterator_t<V>); // exposition only
        constexpr iterator_t<V> find_prev(iterator_t<V>); // exposition only
        requires bidirectional_range<V>;
    };
}

```

```

};

template<class R, class Pred>
    chunk_by_view(R&&, Pred) -> chunk_by_view<views::all_t<R>, Pred>;
}

constexpr explicit chunk_by_view(V base, Pred pred);
1     Effects: Initializes base_ with std::move(base) and pred_ with std::move(pred).

constexpr const Pred& pred() const;
2     Effects: Equivalent to: return *pred_;

constexpr iterator begin();
3     Preconditions: pred_.has_value() is true.
4     Returns: iterator(*this, ranges::begin(base_), find-next(ranges::begin(base_))).
5     Remarks: In order to provide the amortized constant-time complexity required by the range concept,
    this function caches the result within the chunk_by_view for use on subsequent calls.

constexpr auto end();
6     Effects: Equivalent to:
        if constexpr (common_range<V>) {
            return iterator(*this, ranges::end(base_), ranges::end(base_));
        } else {
            return default_sentinel;
        }

constexpr iterator_t<V> find-next(iterator_t<V> current);
7     Preconditions: pred_.has_value() is true.
8     Returns:
        ranges::next(ranges::adjacent_find(current, ranges::end(base_), not_fn(ref(*pred_))),
                    1, ranges::end(base_))

constexpr iterator_t<V> find-prev(iterator_t<V> current) requires bidirectional_range<V>;
9     Preconditions:
(9.1) — current is not equal to ranges::begin(base_).
(9.2) — pred_.has_value() is true.
10    Returns: An iterator i in the range [ranges::begin(base_), current) such that:
(10.1) — ranges::adjacent_find(i, current, not_fn(ref(*pred_))) is equal to current; and
(10.2) — if i is not equal to ranges::begin(base_), then bool(invoke(*pred_, *ranges::prev(i), *i)) is false.

```

25.7.31.3 Class `chunk_by_view::iterator`

[range.chunk.by.iter]

```

namespace std::ranges {
    template<forward_range V, indirect_binary_predicate<iterator_t<V>, iterator_t<V>> Pred>
        requires view<V> && is_object_v<Pred>
    class chunk_by_view<V, Pred>::iterator {
        chunk_by_view* parent_ = nullptr;           // exposition only
        iterator_t<V> current_ = iterator_t<V>();   // exposition only
        iterator_t<V> next_ = iterator_t<V>();      // exposition only

        constexpr iterator(chunk_by_view& parent, iterator_t<V> current, // exposition only
                           iterator_t<V> next);

    public:
        using value_type = subrange<iterator_t<V>>;
        using difference_type = range_difference_t<V>;
        using iterator_category = input_iterator_tag;

```

```

using iterator_concept = see below;

iterator() = default;

constexpr value_type operator*() const;
constexpr iterator& operator++();
iterator operator++(int) = default;

constexpr iterator& operator--() requires bidirectional_range<V>;
iterator operator--(int) = default;

friend constexpr bool operator==(const iterator& x, const iterator& y);
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
};
}

```

¹ *iterator*::*iterator_concept* is defined as follows:

- (1.1) — If *V* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.2) — Otherwise, *iterator_concept* denotes *forward_iterator_tag*.

```
constexpr iterator(chunk_by_view& parent, iterator_t<V> current, iterator_t<V> next);
```

² *Effects*: Initializes *parent_* with *addressof*(*parent*), *current_* with *current*, and *next_* with *next*.

```
constexpr value_type operator*() const;
```

³ *Preconditions*: *current_* is not equal to *next_*.

⁴ *Returns*: *subrange(current_, next_)*.

```
constexpr iterator& operator++();
```

⁵ *Preconditions*: *current_* is not equal to *next_*.

⁶ *Effects*: Equivalent to:

```

    current_ = next_;
    next_ = parent_->find-next(current_);
    return *this;

```

```
constexpr iterator& operator--() requires bidirectional_range<V>;
```

⁷ *Effects*: Equivalent to:

```

    next_ = current_;
    current_ = parent_->find-prev(next_);
    return *this;

```

```
friend constexpr bool operator==(const iterator& x, const iterator& y);
```

⁸ *Returns*: *x.current_ == y.current_*.

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

⁹ *Returns*: *x.current_ == x.next_*.

25.7.32 Stride view

[range.stride]

25.7.32.1 Overview

[range.stride.overview]

- ¹ *stride_view* presents a view of an underlying sequence, advancing over *n* elements at a time, as opposed to the usual single-step succession.
- ² The name *views::stride* denotes a range adaptor object (25.7.2). Given subexpressions *E* and *N*, the expression *views::stride*(*E*, *N*) is expression-equivalent to *stride_view*(*E*, *N*).
- ³ [Example 1:

```

auto input = views::iota(0, 12) | views::stride(3);
ranges::copy(input, ostream_iterator<int>(cout, " ")); // prints 0 3 6 9
ranges::copy(input | views::reverse, ostream_iterator<int>(cout, " ")); // prints 9 6 3 0
— end example]

```

25.7.32.2 Class template `stride_view`

[range.stride.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
    class stride_view : public view_interface<stride_view<V>> {
        V base_; // exposition only
        range_difference_t<V> stride_; // exposition only
        // 25.7.32.3, class template stride_view::iterator
        template<bool> class iterator; // exposition only
    public:
        constexpr explicit stride_view(V base, range_difference_t<V> stride);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr range_difference_t<V> stride() const noexcept;

        constexpr auto begin() requires (!simple-view<V>) {
            return iterator<false>(this, ranges::begin(base_));
        }

        constexpr auto begin() const requires range<const V> {
            return iterator<true>(this, ranges::begin(base_));
        }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (common_range<V> && sized_range<V> && forward_range<V>) {
                auto missing = (stride_ - ranges::distance(base_) % stride_) % stride_;
                return iterator<false>(this, ranges::end(base_), missing);
            } else if constexpr (common_range<V> && !bidirectional_range<V>) {
                return iterator<false>(this, ranges::end(base_));
            } else {
                return default_sentinel;
            }
        }

        constexpr auto end() const requires range<const V> {
            if constexpr (common_range<const V> && sized_range<const V> && forward_range<const V>) {
                auto missing = (stride_ - ranges::distance(base_) % stride_) % stride_;
                return iterator<true>(this, ranges::end(base_), missing);
            } else if constexpr (common_range<const V> && !bidirectional_range<const V>) {
                return iterator<true>(this, ranges::end(base_));
            } else {
                return default_sentinel;
            }
        }

        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;

        constexpr auto reserve_hint() requires approximately_sized_range<V>;
        constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
    };

    template<class R>
        stride_view(R&&, range_difference_t<R>) -> stride_view<views::all_t<R>>;
}

```

```
constexpr stride_view(V base, range_difference_t<V> stride);
```

1 *Preconditions:* `stride > 0` is true.

2 *Effects:* Initializes `base_` with `std::move(base)` and `stride_` with `stride`.

```
constexpr range_difference_t<V> stride() const noexcept;
```

3 *Returns:* *stride_*.

```
constexpr auto size() requires sized_range<V>;
```

```
constexpr auto size() const requires sized_range<const V>;
```

4 *Effects:* Equivalent to:

```
return to-unsigned-like(div-ceil(ranges::distance(base_), stride_));
```

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
```

```
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

5 *Effects:* Equivalent to:

```
auto s = static_cast<range_difference_t<decltype((base_))>>(ranges::reserve_hint(base_));
```

```
return to-unsigned-like(div-ceil(s, stride_));
```

25.7.32.3 Class template `stride_view::iterator`

[range.stride.iterator]

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
    template<bool Const>
    class stride_view<V>::iterator {
        using Parent = maybe_const<Const, stride_view>;           // exposition only
        using Base = maybe_const<Const, V>;                       // exposition only

        iterator_t<Base> current_ = iterator_t<Base>();           // exposition only
        sentinel_t<Base> end_ = sentinel_t<Base>();               // exposition only
        range_difference_t<Base> stride_ = 0;                     // exposition only
        range_difference_t<Base> missing_ = 0;                    // exposition only

        constexpr iterator(Parent* parent, iterator_t<Base> current, // exposition only
                           range_difference_t<Base> missing = 0);

    public:
        using difference_type = range_difference_t<Base>;
        using value_type = range_value_t<Base>;
        using iterator_concept = see below;
        using iterator_category = see below;    // not always present

        iterator() requires default_initializable<iterator_t<Base>> = default;

        constexpr iterator(iterator<!Const> other)
            requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
                && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr iterator_t<Base> base() &&;
        constexpr const iterator_t<Base>& base() const & noexcept;

        constexpr decltype(auto) operator*() const { return *current_; }

        constexpr iterator& operator++();
        constexpr void operator++(int);
        iterator operator++(int) requires forward_range<Base> = default;

        constexpr iterator& operator--() requires bidirectional_range<Base>;
        iterator operator--(int) = default;

        constexpr iterator& operator+=(difference_type n) requires random_access_range<Base>;
        constexpr iterator& operator-=(difference_type n) requires random_access_range<Base>;

        constexpr decltype(auto) operator[](difference_type n) const
            requires random_access_range<Base>
        { return *(*this + n); }

        friend constexpr bool operator==(const iterator& x, default_sentinel_t);
    };
}
```



```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;

friend constexpr iterator operator+(const iterator& x, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& x)
    requires random_access_range<Base>;
friend constexpr iterator operator-(const iterator& x, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
};
}

```

¹ *iterator::iterator_concept* is defined as follows:

- (1.1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *Base* models *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² The member *typedef-name* *iterator_category* is declared if and only if *Base* models *forward_range*. In that case, *iterator::iterator_category* is defined as follows:

- (2.1) — Let *C* denote the type *iterator_traits<iterator_t<Base>>::iterator_category*.
- (2.2) — If *C* models *derived_from<random_access_iterator_tag>*, then *iterator_category* denotes *random_access_iterator_tag*.
- (2.3) — Otherwise, *iterator_category* denotes *C*.

```

constexpr iterator (Parent* parent, iterator_t<Base> current,
                    range_difference_t<Base> missing = 0);

```

³ *Effects*: Initializes *current_* with `std::move(current)`, *end_* with `ranges::end(parent->base_)`, *stride_* with `parent->stride_`, and *missing_* with *missing*.

```

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
        && convertible_to<sentinel_t<V>, sentinel_t<Base>>;
4     Effects: Initializes current_ with std::move(i.current_), end_ with std::move(i.end_), stride_
    with i.stride_, and missing_ with i.missing_.

constexpr iterator_t<Base> base() &&;
5     Returns: std::move(current_).

constexpr const iterator_t<Base>& base() const & noexcept;
6     Returns: current_.

constexpr iterator& operator++();
7     Preconditions: current_ != end_ is true.
8     Effects: Equivalent to:
        missing_ = ranges::advance(current_, stride_, end_);
        return *this;

constexpr void operator++(int);
9     Effects: Equivalent to ++*this;

constexpr iterator& operator--() requires bidirectional_range<Base>;
10    Effects: Equivalent to:
        ranges::advance(current_, missing_ - stride_);
        missing_ = 0;
        return *this;

constexpr iterator& operator+=(difference_type n) requires random_access_range<Base>;
11    Preconditions: If n is positive, ranges::distance(current_, end_) > stride_ * (n - 1) is true.
    [Note 1: If n is negative, the Effects paragraph implies a precondition. — end note]
12    Effects: Equivalent to:
        if (n > 0) {
            ranges::advance(current_, stride_ * (n - 1));
            missing_ = ranges::advance(current_, stride_, end_);
        } else if (n < 0) {
            ranges::advance(current_, stride_ * n + missing_);
            missing_ = 0;
        }
        return *this;

constexpr iterator& operator-=(difference_type x)
    requires random_access_range<Base>;
13    Effects: Equivalent to: return *this += -x;

friend constexpr bool operator==(const iterator& x, default_sentinel_t);
14    Returns: x.current_ == x.end_.

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;
15    Returns: x.current_ == y.current_.

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
16    Returns: x.current_ < y.current_.

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
17    Effects: Equivalent to: return y < x;

```

```

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
18     Effects: Equivalent to: return !(y < x);

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
19     Effects: Equivalent to: return !(x < y);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;
20     Returns: x.current_ <=> y.current_.

friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
21     Effects: Equivalent to:
        auto r = i;
        r += n;
        return r;

friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
22     Effects: Equivalent to:
        auto r = i;
        r -= n;
        return r;

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;
23     Returns: Let N be (x.current_ - y.current_).
(23.1) — If Base models forward_range, (N + x.missing_ - y.missing_) / x.stride_.
(23.2) — Otherwise, if N is negative, -div-ceil(-N, x.stride_).
(23.3) — Otherwise, div-ceil(N, x.stride_).

friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
24     Returns: div-ceil(x.end_ - x.current_, x.stride_).

friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
25     Effects: Equivalent to: return -(y - x);

friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
26     Effects: Equivalent to: return ranges::iter_move(i.current_);

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
27     Effects: Equivalent to: ranges::iter_swap(x.current_, y.current_);

```

25.7.33 Cartesian product view

[range.cartesian]

25.7.33.1 Overview

[range.cartesian.overview]

- ¹ cartesian_product_view takes any non-zero number of ranges *n* and produces a view of tuples calculated by the *n*-ary cartesian product of the provided ranges.

² The name `views::cartesian_product` denotes a customization point object (16.3.3.3.5). Given a pack of subexpressions `Es`, the expression `views::cartesian_product(Es...)` is expression-equivalent to

- (2.1) — `views::single(tuple())` if `Es` is an empty pack,
- (2.2) — otherwise, `cartesian_product_view<views::all_t<decltype((Es))>...>(Es...)`.

³ [Example 1:

```
vector<int> v { 0, 1, 2 };
for (auto&& [a, b, c] : views::cartesian_product(v, v, v)) {
    cout << a << ' ' << b << ' ' << c << '\n';
}
// The above prints
// 0 0 0
// 0 0 1
// 0 0 2
// 0 1 0
// 0 1 1
// ...
```

— end example]

25.7.33.2 Class template `cartesian_product_view`

[range.cartesian.view]

```
namespace std::ranges {
    template<bool Const, class First, class... Vs>
    concept cartesian_product_is_random_access =           // exposition only
        (random_access_range<maybe-const<Const, First>> && ... &&
         (random_access_range<maybe-const<Const, Vs>>
          && sized_range<maybe-const<Const, Vs>>));

    template<class R>
    concept cartesian_product_common_arg =                 // exposition only
        common_range<R> || (sized_range<R> && random_access_range<R>);

    template<bool Const, class First, class... Vs>
    concept cartesian_product_is_bidirectional =           // exposition only
        (bidirectional_range<maybe-const<Const, First>> && ... &&
         (bidirectional_range<maybe-const<Const, Vs>>
          && cartesian_product_common_arg<maybe-const<Const, Vs>>));

    template<class First, class...>
    concept cartesian_product_is_common =                  // exposition only
        cartesian_product_common_arg<First>;

    template<class... Vs>
    concept cartesian_product_is_sized =                    // exposition only
        (sized_range<Vs> && ...);

    template<bool Const, template<class> class FirstSent, class First, class... Vs>
    concept cartesian_is_sized_sentinel =                  // exposition only
        (sized_sentinel_for<FirstSent<maybe-const<Const, First>>,
         iterator_t<maybe-const<Const, First>>> && ...
         && (sized_range<maybe-const<Const, Vs>>
          && sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>,
          iterator_t<maybe-const<Const, Vs>>>));

    template<cartesian_product_common_arg R>
    constexpr auto cartesian_common_arg_end(R& r) {        // exposition only
        if constexpr (common_range<R>) {
            return ranges::end(r);
        } else {
            return ranges::begin(r) + ranges::distance(r);
        }
    }
}
```

```

template<input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
class cartesian_product_view : public view_interface<cartesian_product_view<First, Vs...>> {
private:
    tuple<First, Vs...> bases_; // exposition only
    // 25.7.33.3, class template cartesian_product_view::iterator
    template<bool Const> class iterator; // exposition only

public:
    constexpr cartesian_product_view() = default;
    constexpr explicit cartesian_product_view(First first_base, Vs... bases);

    constexpr iterator<false> begin()
        requires (!simple-view<First> || ... || !simple-view<Vs>);
    constexpr iterator<true> begin() const
        requires (range<const First> && ... && range<const Vs>);

    constexpr iterator<false> end()
        requires ((!simple-view<First> || ... || !simple-view<Vs>) &&
            cartesian-product-is-common<First, Vs...>);
    constexpr iterator<true> end() const
        requires cartesian-product-is-common<const First, const Vs...>;
    constexpr default_sentinel_t end() const noexcept;

    constexpr see below size()
        requires cartesian-product-is-sized<First, Vs...>;
    constexpr see below size() const
        requires cartesian-product-is-sized<const First, const Vs...>;
};

template<class... Vs>
    cartesian_product_view(Vs&&...) -> cartesian_product_view<views::all_t<Vs>...>;
}

```

```
constexpr explicit cartesian_product_view(First first_base, Vs... bases);
```

1 *Effects:* Initializes *bases_* with `std::move(first_base)`, `std::move(bases)`....

```
constexpr iterator<false> begin()
    requires (!simple-view<First> || ... || !simple-view<Vs>);
```

2 *Effects:* Equivalent to:

```
    return iterator<false>(*this, tuple-transform(ranges::begin, bases_));
```

```
constexpr iterator<true> begin() const
    requires (range<const First> && ... && range<const Vs>);
```

3 *Effects:* Equivalent to:

```
    return iterator<true>(*this, tuple-transform(ranges::begin, bases_));
```

```
constexpr iterator<false> end()
    requires ((!simple-view<First> || ... || !simple-view<Vs>)
        && cartesian-product-is-common<First, Vs...>);
constexpr iterator<true> end() const
    requires cartesian-product-is-common<const First, const Vs...>;
```

4 *Let:*

- (4.1) — *is-const* be true for the const-qualified overload, and false otherwise;
- (4.2) — *is-empty* be true if the expression `ranges::empty(rng)` is true for any *rng* among the underlying ranges except the first one and false otherwise; and
- (4.3) — *begin-or-first-end(rng)* be expression-equivalent to `is-empty ? ranges::begin(rng) : cartesian-common-arg-end(rng)` if *rng* is the first underlying range and `ranges::begin(rng)` otherwise.

5 *Effects:* Equivalent to:

```
    iterator<is-const> it(*this, tuple-transform(
        [](auto& rng){ return begin-or-first-end(rng); }, bases_));
    return it;
```

```
constexpr default_sentinel_t end() const noexcept;
```

6 *Returns:* default_sentinel.

```
constexpr see below size()
    requires cartesian-product-is-sized<First, Vs...>;
constexpr see below size() const
    requires cartesian-product-is-sized<const First, const Vs...>;
```

7 The return type is an implementation-defined unsigned-integer-like type.

8 *Recommended practice:* The return type should be the smallest unsigned-integer-like type that is sufficiently wide to store the product of the maximum sizes of all the underlying ranges, if such a type exists.

9 Let p be the product of the sizes of all the ranges in *bases_*.

10 *Preconditions:* p can be represented by the return type.

11 *Returns:* p .

25.7.33.3 Class template cartesian_product_view::iterator [range.cartesian.iterator]

```
namespace std::ranges {
    template<input_range First, forward_range... Vs>
        requires (view<First> && ... && view<Vs>)
        template<bool Const>
        class cartesian_product_view<First, Vs...>::iterator {
        public:
            using iterator_category = input_iterator_tag;
            using iterator_concept = see below;
            using value_type = tuple<range_value_t<maybe-const<Const, First>>,
                range_value_t<maybe-const<Const, Vs>>...>;
            using reference = tuple<range_reference_t<maybe-const<Const, First>>,
                range_reference_t<maybe-const<Const, Vs>>...>;
            using difference_type = see below;

            iterator() = default;

            constexpr iterator(iterator<!Const> i) requires Const &&
                (convertible_to<iterator_t<First>, iterator_t<const First>> &&
                 ... && convertible_to<iterator_t<Vs>, iterator_t<const Vs>>);

            constexpr auto operator*() const;
            constexpr iterator& operator++();
            constexpr void operator++(int);
            iterator operator++(int) requires forward_range<maybe-const<Const, First>> = default;

            constexpr iterator& operator--()
                requires cartesian-product-is-bidirectional<Const, First, Vs...>;
            iterator operator--(int) = default;

            constexpr iterator& operator+=(difference_type x)
                requires cartesian-product-is-random-access<Const, First, Vs...>;
            constexpr iterator& operator-=(difference_type x)
                requires cartesian-product-is-random-access<Const, First, Vs...>;

            constexpr reference operator[](difference_type n) const
                requires cartesian-product-is-random-access<Const, First, Vs...>;

            friend constexpr bool operator==(const iterator& x, const iterator& y)
                requires equality_comparable<iterator_t<maybe-const<Const, First>>>;
```

```

friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, First, Vs...>;

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires cartesian-is-sized-sentinel<Const, iterator_t, First, Vs...>;

friend constexpr difference_type operator-(const iterator& i, default_sentinel_t)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
friend constexpr difference_type operator-(default_sentinel_t, const iterator& i)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;

friend constexpr auto iter_move(const iterator& i) noexcept(see below);

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>> && ... &&
        indirectly_swappable<iterator_t<maybe-const<Const, Vs>>>);

private:
    using Parent = maybe-const<Const, cartesian_product_view>;           // exposition only
    Parent* parent_ = nullptr;                                           // exposition only
    tuple<iterator_t<maybe-const<Const, First>>,
        iterator_t<maybe-const<Const, Vs>>...> current_;               // exposition only

    template<size_t N = sizeof...(Vs)>
        constexpr void next();                                           // exposition only

    template<size_t N = sizeof...(Vs)>
        constexpr void prev();                                           // exposition only

    template<class Tuple>
        constexpr difference_type distance-from(const Tuple& t) const;   // exposition only

    constexpr iterator(Parent& parent, tuple<iterator_t<maybe-const<Const, First>>,
        iterator_t<maybe-const<Const, Vs>>...> current);               // exposition only
};
}

```

¹ *iterator*::iterator_concept is defined as follows:

- (1.1) — If *cartesian-product-is-random-access*<Const, First, Vs...> is modeled, then *iterator_concept* denotes *random_access_iterator_tag*.
- (1.2) — Otherwise, if *cartesian-product-is-bidirectional*<Const, First, Vs...> is modeled, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- (1.3) — Otherwise, if *maybe-const*<Const, First> models *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- (1.4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

² *iterator*::difference_type is an implementation-defined signed-integer-like type.

³ *Recommended practice*: *iterator*::difference_type should be the smallest signed-integer-like type that is sufficiently wide to store the product of the maximum sizes of all underlying ranges if such a type exists.

```

template<size_t N = sizeof...(Vs)>
    constexpr void next();

```

⁴ *Effects*: Equivalent to:

```

    auto& it = std::get<N>(current_);
    ++it;
    if constexpr (N > 0) {
        if (it == ranges::end(std::get<N>(parent_>bases_))) {
            it = ranges::begin(std::get<N>(parent_>bases_));
            next<N - 1>();
        }
    }
}

```

```

template<size_t N = sizeof...(Vs)>
constexpr void prev();

```

5 *Effects:* Equivalent to:

```

    auto& it = std::get<N>(current_);
    if constexpr (N > 0) {
        if (it == ranges::begin(std::get<N>(parent_>bases_))) {
            it = cartesian-common-arg-end(std::get<N>(parent_>bases_));
            prev<N - 1>();
        }
    }
    --it;

```

```

template<class Tuple>
constexpr difference_type distance-from(const Tuple& t) const;

```

6 *Let:*

- (6.1) — *scaled-size*(N) be the product of `static_cast<difference_type>(ranges::size(std::get<N>(parent_>bases_)))` and *scaled-size*($N + 1$) if $N \leq \text{sizeof}...(Vs)$, otherwise `static_cast<difference_type>(1)`;
- (6.2) — *scaled-distance*(N) be the product of `static_cast<difference_type>(std::get<N>(current_) - std::get<N>(t))` and *scaled-size*($N + 1$); and
- (6.3) — *scaled-sum* be the sum of *scaled-distance*(N) for every integer $0 \leq N \leq \text{sizeof}...(Vs)$.

7 *Preconditions:* *scaled-sum* can be represented by `difference_type`.

8 *Returns:* *scaled-sum*.

```

constexpr iterator (Parent& parent, tuple<iterator_t<maybe-const<Const, First>>,
    iterator_t<maybe-const<Const, Vs>>...> current);

```

9 *Effects:* Initializes `parent_` with `addressof(parent)` and `current_` with `std::move(current)`.

```

constexpr iterator (iterator<!Const> i) requires Const &&
    (convertible_to<iterator_t<First>, iterator_t<const First>> &&
    ... && convertible_to<iterator_t<Vs>, iterator_t<const Vs>>);

```

10 *Effects:* Initializes `parent_` with `i.parent_` and `current_` with `std::move(i.current_)`.

```

constexpr auto operator*() const;

```

11 *Effects:* Equivalent to:

```

    return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);

```

```

constexpr iterator& operator++();

```

12 *Effects:* Equivalent to:

```

    next();
    return *this;

```

```

constexpr void operator++(int);

```

13 *Effects:* Equivalent to `++*this`.

```

constexpr iterator& operator--()
    requires cartesian-product-is-bidirectional<Const, First, Vs...>;

```

14 *Effects:* Equivalent to:


```

    prev();
    return *this;

```

```

constexpr iterator& operator+=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

15 Let *orig* be the value of **this* before the call.

Let *ret* be:

(15.1) — If $x > 0$, the value of **this* had *next* been called x times.

(15.2) — Otherwise, if $x < 0$, the value of **this* had *prev* been called $-x$ times.

(15.3) — Otherwise, *orig*.

16 *Preconditions*: x is in the range $[\text{ranges::distance}(*this, \text{ranges::begin}(*parent_)), \text{ranges::distance}(*this, \text{ranges::end}(*parent_))]$.

17 *Effects*: Sets the value of **this* to *ret*.

18 *Returns*: **this*.

19 *Complexity*: Constant.

```

constexpr iterator& operator-=(difference_type x)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

20 *Effects*: Equivalent to:

```

    *this += -x;
    return *this;

```

```

constexpr reference operator[](difference_type n) const
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

21 *Effects*: Equivalent to: `return *((*this) + n);`

```

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<maybe-const<Const, First>>>;

```

22 *Effects*: Equivalent to: `return x.current_ == y.current_;`

```

friend constexpr bool operator==(const iterator& x, default_sentinel_t);

```

23 *Returns*: true if `std::get<i>(x.current_) == ranges::end(std::get<i>(x.parent_->bases_))` is true for any integer $0 \leq i \leq \text{sizeof} \dots (\text{Vs})$; otherwise, false.

```

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires all-random-access<Const, First, Vs...>;

```

24 *Effects*: Equivalent to: `return x.current_ <=> y.current_;`

```

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

25 *Effects*: Equivalent to: `return iterator(x) += y;`

```

friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

26 *Effects*: Equivalent to: `return y + x;`

```

friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires cartesian-product-is-random-access<Const, First, Vs...>;

```

27 *Effects*: Equivalent to: `return iterator(x) -= y;`

```

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires cartesian-is-sized-sentinel<Const, iterator_t, First, Vs...>;

```

28 *Effects*: Equivalent to: `return x.distance-from(y.current_);`

```

friend constexpr difference_type operator-(const iterator& i, default_sentinel_t)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
29     Let end-tuple be an object of a type that is a specialization of tuple, such that:
(29.1)     — std::get<0>(end-tuple) has the same value as ranges::end(std::get<0>(i.parent_->bases_));
(29.2)     — std::get<N>(end-tuple) has the same value as ranges::begin(std::get<N>(i.parent_->bases_)) for every integer  $1 \leq N \leq \text{sizeof...}(Vs)$ .
30     Effects: Equivalent to: return i.distance-from(end-tuple);

friend constexpr difference_type operator-(default_sentinel_t s, const iterator& i)
    requires cartesian-is-sized-sentinel<Const, sentinel_t, First, Vs...>;
31     Effects: Equivalent to: return -(i - s);

friend constexpr auto iter_move(const iterator& i) noexcept(see below);
32     Effects: Equivalent to: return tuple-transform(ranges::iter_move, i.current_);
33     Remarks: The exception specification is equivalent to the logical AND of the following expressions:
(33.1)     — noexcept(ranges::iter_move(std::get<N>(i.current_))) for every integer  $0 \leq N \leq \text{sizeof...}(Vs)$ ,
(33.2)     — is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-const<Const, T>>> for every type T in First, Vs....

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>> && ... &&
        indirectly_swappable<iterator_t<maybe-const<Const, Vs>>>);
34     Effects: For every integer  $0 \leq i \leq \text{sizeof...}(Vs)$ , performs:
        ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_))
35     Remarks: The exception specification is equivalent to the logical AND of the following expressions:
(35.1)     — noexcept(ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_))) for every integer  $0 \leq i \leq \text{sizeof...}(Vs)$ .

```

25.7.34 Cache latest view

[range.cache.latest]

25.7.34.1 Overview

[range.cache.latest.overview]

- ¹ `cache_latest_view` caches the last-accessed element of its underlying sequence so that the element does not have to be recomputed on repeated access.

[Note 1: This is useful if computation of the element to produce is expensive. — end note]

- ² The name `views::cache_latest` denotes a range adaptor object (25.7.2). Let E be an expression. The expression `views::cache_latest(E)` is expression-equivalent to `cache_latest_view(E)`.

25.7.34.2 Class template `cache_latest_view`

[range.cache.latest.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
    class cache_latest_view : public view_interface<cache_latest_view<V>> {
        V base_ = V(); // exposition only
        using cache_t = conditional_t<is_reference_v<range_reference_t<V>>, // exposition only
            add_pointer_t<range_reference_t<V>>,
            range_reference_t<V>>;

        non-propagating-cache<cache_t> cache_; // exposition only

        // 25.7.34.3, class cache_latest_view::iterator
        class iterator; // exposition only
        // 25.7.34.4, class cache_latest_view::sentinel
        class sentinel; // exposition only
    };
}

```

```

public:
    cache_latest_view() requires default_initializable<V> = default;
    constexpr explicit cache_latest_view(V base);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin();
    constexpr auto end();

    constexpr auto size() requires sized_range<V>;
    constexpr auto size() const requires sized_range<const V>;

    constexpr auto reserve_hint() requires approximately_sized_range<V>;
    constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};

template<class R>
    cache_latest_view(R&&) -> cache_latest_view<views::all_t<R>>;
}

constexpr explicit cache_latest_view(V base);
1     Effects: Initializes base_ with std::move(base).

constexpr auto begin();
2     Effects: Equivalent to: return iterator(*this);

constexpr auto end();
3     Effects: Equivalent to: return sentinel(*this);

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
4     Effects: Equivalent to: return ranges::size(base_);

constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
5     Effects: Equivalent to: return ranges::reserve_hint(base_);

```

25.7.34.3 Class `cache_latest_view::iterator`

[range.cache.latest.iterator]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
        class cache_latest_view<V>::iterator {
            cache_latest_view* parent_;           // exposition only
            iterator_t<V> current_;               // exposition only

            constexpr explicit iterator(cache_latest_view& parent); // exposition only

        public:
            using difference_type = range_difference_t<V>;
            using value_type = range_value_t<V>;
            using iterator_concept = input_iterator_tag;

            iterator(iterator&&) = default;
            iterator& operator=(iterator&&) = default;

            constexpr iterator_t<V> base() &&;
            constexpr const iterator_t<V>& base() const & noexcept;

            constexpr range_reference_t<V>& operator*() const;

```

```

constexpr iterator& operator++();
constexpr void operator++(int);

friend constexpr range_rvalue_reference_t<V> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<V>>;
};
}

constexpr explicit iterator(cache_latest_view& parent);
1    Effects: Initializes current_ with ranges::begin(parent.base_) and parent_ with addressof(par-
    ent).

constexpr iterator_t<V> base() &&;
2    Returns: std::move(current_).

constexpr const iterator_t<V>& base() const & noexcept;
3    Returns: current_.

constexpr iterator& operator++();
4    Effects: Equivalent to:
        parent_→cache_.reset();
        ++current_;
        return *this;

constexpr void operator++(int);
5    Effects: Equivalent to: ++*this.

constexpr range_reference_t<V>& operator*() const;
6    Effects: Equivalent to:
        if constexpr (is_reference_v<range_reference_t<V>>) {
            if (!parent_→cache_) {
                parent_→cache_ = addressof(as-lvalue(*current_));
            }
            return **parent_→cache_;
        } else {
            if (!parent_→cache_) {
                parent_→cache_.emplace_deref(current_);
            }
            return *parent_→cache_;
        }

[Note 1: Evaluations of operator* on the same iterator object can conflict (6.10.2.2). — end note]

friend constexpr range_rvalue_reference_t<V> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
7    Effects: Equivalent to: return ranges::iter_move(i.current_);

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<V>>;
8    Effects: Equivalent to ranges::iter_swap(x.current_, y.current_).

```

25.7.34.4 Class `cache_latest_view::sentinel`

[range.cache.latest.sentinel]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
        class cache_latest_view<V>::sentinel {

```

```

    sentinel_t<V> end_ = sentinel_t<V>(); // exposition only

    constexpr explicit sentinel(cache_latest_view& parent); // exposition only

public:
    sentinel() = default;

    constexpr sentinel_t<V> base() const;

    friend constexpr bool operator==(const iterator& x, const sentinel& y);

    friend constexpr range_difference_t<V> operator-(const iterator& x, const sentinel& y)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
    friend constexpr range_difference_t<V> operator-(const sentinel& x, const iterator& y)
        requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
};
}

```

```
constexpr explicit sentinel(cache_latest_view& parent);
```

1 *Effects:* Initializes *end_* with `ranges::end(parent.base_)`.

```
constexpr sentinel_t<V> base() const;
```

2 *Returns:* *end_*.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

3 *Returns:* `x.current_ == y.end_`.

```
friend constexpr range_difference_t<V> operator-(const iterator& x, const sentinel& y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

4 *Returns:* `x.current_ - y.end_`.

```
friend constexpr range_difference_t<V> operator-(const sentinel& x, const iterator& y)
    requires sized_sentinel_for<sentinel_t<V>, iterator_t<V>>;
```

5 *Returns:* `x.end_ - y.current_`.

25.7.35 As input view

[range.as.input]

25.7.35.1 Overview

[range.as.input.overview]

1 `as_input_view` presents a view of an underlying sequence as an input-only non-common range.

[Note 1: This is useful to avoid overhead that can be necessary to provide support for the operations needed for greater iterator strength. — end note]

2 The name `views::as_input` denotes a range adaptor object (25.7.2). Let *E* be an expression and let *T* be `decltype((E))`. The expression `views::as_input(E)` is expression-equivalent to:

(2.1) — `views::all(E)` if *T* models `input_range`, does not satisfy `common_range`, and does not satisfy `forward_range`.

(2.2) — Otherwise, `as_input_view(E)`.

25.7.35.2 Class template `as_input_view`

[range.as.input.view]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
    class as_input_view : public view_interface<as_input_view<V>> {
        V base_ = V(); // exposition only

        // 25.7.35.3, class template as_input_view::iterator
        template<bool Const> class iterator; // exposition only

public:
    as_input_view() requires default_initializable<V> = default;
    constexpr explicit as_input_view(V base);
}

```

```

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }

constexpr auto begin() requires (!simple-view<V>);
constexpr auto begin() const requires range<const V>;

constexpr auto end() requires (!simple-view<V>);
constexpr auto end() const requires range<const V>;

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;

constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};

template<class R>
    as_input_view(R&&) -> as_input_view<views::all_t<R>>;
}

constexpr explicit as_input_view(V base);
1     Effects: Initializes base_ with std::move(base).

constexpr auto begin() requires (!simple-view<V>);
2     Effects: Equivalent to: return iterator<false>(ranges::begin(base_));

constexpr auto begin() const requires range<const V>;
3     Effects: Equivalent to: return iterator<true>(ranges::begin(base_));

constexpr auto end() requires (!simple-view<V>);
constexpr auto end() const requires range<const V>;
4     Effects: Equivalent to: return ranges::end(base_);

constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
5     Effects: Equivalent to: return ranges::size(base_);

constexpr auto reserve_hint() requires approximately_sized_range<V>;
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
6     Effects: Equivalent to: return ranges::reserve_hint(base_);

```

25.7.35.3 Class template `as_input_view::iterator`

[range.as.input.iterator]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
        template<bool Const>
        class as_input_view<V>::iterator {
            using Base = maybe_const<Const, V>;                                // exposition only

            iterator_t<Base> current_ = iterator_t<Base>();                    // exposition only

            constexpr explicit iterator(iterator_t<Base> current);              // exposition only

        public:
            using difference_type = range_difference_t<Base>;
            using value_type = range_value_t<Base>;
            using iterator_concept = input_iterator_tag;

            iterator() requires default_initializable<iterator_t<Base>> = default;

            iterator(iterator&&) = default;
            iterator& operator=(iterator&&) = default;

```

```

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;

constexpr iterator_t<Base> base() &&;
constexpr const iterator_t<Base>& base() const & noexcept;

constexpr decltype(auto) operator*() const { return *current_; }

constexpr iterator& operator++();
constexpr void operator++(int);

friend constexpr bool operator==(const iterator& x, const sentinel_t<Base>& y);

friend constexpr difference_type operator-(const sentinel_t<Base>& y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
friend constexpr difference_type operator-(const iterator& x, const sentinel_t<Base>& y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

friend constexpr range_value_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
};
}

constexpr explicit iterator(iterator_t<Base> current);
1     Effects: Initializes current_ with std::move(current).

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>;
2     Effects: Initializes current_ with std::move(i.current_).

constexpr iterator_t<Base> base() &&;
3     Returns: std::move(current_).

constexpr const iterator_t<Base>& base() const & noexcept;
4     Returns: current_.

constexpr iterator& operator++();
5     Effects: Equivalent to:
        ++current_;
        return *this;

constexpr void operator++(int);
6     Effects: Equivalent to: ++*this;

friend constexpr bool operator==(const iterator& x, const sentinel_t<Base>& y);
7     Returns: x.current_ == y.

friend constexpr difference_type operator-(const sentinel_t<Base>& y, const iterator& x)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
8     Returns: y - x.current_.

friend constexpr difference_type operator-(const iterator& x, const sentinel_t<Base>& y)
    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
9     Returns: x.current_ - y.

```

```
friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));
```

10 *Effects:* Equivalent to: `return ranges::iter_move(i.current_);`

```
friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<Base>>;
```

11 *Effects:* Equivalent to: `ranges::iter_swap(x.current_, y.current_);`

25.8 Range generators

[coro.generator]

25.8.1 Overview

[coroutine.generator.overview]

1 Class template `generator` presents a view of the elements yielded by the evaluation of a coroutine.

2 A `generator` generates a sequence of elements by repeatedly resuming the coroutine from which it was returned. Elements of the sequence are produced by the coroutine each time a `co_yield` statement is evaluated. When the `co_yield` statement is of the form `co_yield elements_of(r)`, each element of the range `r` is successively produced as an element of the sequence.

[Example 1:

```
generator<int> ints(int start = 0) {
    while (true)
        co_yield start++;
}

void f() {
    for (auto i : ints() | views::take(3))
        cout << i << ' ';    // prints 0 1 2
}
```

— end example]

25.8.2 Header <generator> synopsis

[generator.syn]

```
namespace std {
    // 25.8.3, class template generator
    template<class Ref, class Val = void, class Allocator = void>
        class generator;

    namespace pmr {
        template<class Ref, class Val = void>
            using generator = std::generator<Ref, Val, polymorphic_allocator<>>;
    }
}
```

25.8.3 Class template generator

[coro.generator.class]

```
namespace std {
    template<class Ref, class Val = void, class Allocator = void>
    class generator : public ranges::view_interface<generator<Ref, Val, Allocator>> {
    private:
        using value = conditional_t<is_void_v<Val>, remove_cvref_t<Ref>, Val>;    // exposition only
        using reference = conditional_t<is_void_v<Val>, Ref&&, Ref>;    // exposition only

        // 25.8.6, class generator::iterator    // exposition only
        class iterator;

    public:
        using yielded =
            conditional_t<is_reference_v<reference>, reference, const reference&>;

        // 25.8.5, class generator::promise_type
        class promise_type;

        generator(const generator&) = delete;
        generator(generator&& other) noexcept;
```



```

~generator();

generator& operator=(generator other) noexcept;

iterator begin();
default_sentinel_t end() const noexcept;

private:
    coroutine_handle<promise_type> coroutine_ = nullptr; // exposition only
    unique_ptr<stack<coroutine_handle<>>> active_; // exposition only
};
}

```

1 *Mandates:*

- (1.1) — If `Allocator` is not `void`, `allocator_traits<Allocator>::pointer` is a pointer type.
- (1.2) — *value* is a cv-unqualified object type.
- (1.3) — *reference* is either a reference type, or a cv-unqualified object type that models `copy_constructible`.
- (1.4) — Let `RRef` denote `remove_reference_t<reference>&&` if *reference* is a reference type, and *reference* otherwise. Each of:
 - (1.4.1) — `common_reference_with<reference&&, value&>`,
 - (1.4.2) — `common_reference_with<reference&&, RRef&&>`, and
 - (1.4.3) — `common_reference_with<RRef&&, const value&>`

is modeled.

[*Note 1:* These requirements ensure the exposition-only *iterator* type can model `indirectly_readable` and thus `input_iterator`. — *end note*]

2 If `Allocator` is not `void`, it shall meet the *Cpp17Allocator* requirements.

3 Specializations of `generator` model view and `input_range`.

4 The behavior of a program that adds a specialization for `generator` is undefined.

25.8.4 Members

[`coro.generator.members`]

```
generator(generator&& other) noexcept;
```

1 *Effects:* Initializes *coroutine_* with `exchange(other.coroutine_, {})` and *active_* with `exchange(other.active_, nullptr)`.

2 [*Note 1:* Iterators previously obtained from *other* are not invalidated; they become iterators into **this*. — *end note*]

```
~generator();
```

3 *Effects:* Equivalent to:

```

if (coroutine_) {
    coroutine_.destroy();
}

```

4 [*Note 2:* Ownership of recursively yielded generators is held in awaitable objects in the coroutine frame of the yielding generator, so destroying the root generator effectively destroys the entire stack of yielded generators. — *end note*]

```
generator& operator=(generator other) noexcept;
```

5 *Effects:* Equivalent to:

```

swap(coroutine_, other.coroutine_);
swap(active_, other.active_);

```

6 *Returns:* **this*.

7 [*Note 3:* Iterators previously obtained from *other* are not invalidated; they become iterators into **this*. — *end note*]

iterator begin();

8 *Preconditions:* *coroutine_* refers to a coroutine suspended at its initial suspend point (9.6.4).

9 *Effects:* Pushes *coroutine_* into **active_*, then evaluates *coroutine_.resume()*.

10 *Returns:* An *iterator* object whose member *coroutine_* refers to the same coroutine as does *coroutine_*.

11 [*Note 4:* A program that calls *begin* more than once on the same generator has undefined behavior. — *end note*]

default_sentinel_t end() const noexcept;

12 *Returns:* default_sentinel.

25.8.5 Class generator::promise_type

[coro.generator.promise]

```
namespace std {
    template<class Ref, class Val, class Allocator>
    class generator<Ref, Val, Allocator>::promise_type {
    public:
        generator get_return_object() noexcept;

        suspend_always initial_suspend() const noexcept { return {}; }
        auto final_suspend() noexcept;

        suspend_always yield_value(yielded val) noexcept;

        auto yield_value(const remove_reference_t<yielded>& lval)
            requires is_rvalue_reference_v<yielded> &&
            constructible_from<remove_cvref_t<yielded>, const remove_reference_t<yielded>&>;

        template<class R2, class V2, class Alloc2, class Unused>
            requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
            auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&&, Unused> g) noexcept;
        template<class R2, class V2, class Alloc2, class Unused>
            requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
            auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&, Unused> g) noexcept;

        template<ranges::input_range R, class Alloc>
            requires convertible_to<ranges::range_reference_t<R>, yielded>
            auto yield_value(ranges::elements_of<R, Alloc> r);

        void await_transform() = delete;

        void return_void() const noexcept {}
        void unhandled_exception();

        void* operator new(size_t size)
            requires same_as<Allocator, void> || default_initializable<Allocator>;

        template<class Alloc, class... Args>
            void* operator new(size_t size, allocator_arg_t, const Alloc& alloc, const Args&...);

        template<class This, class Alloc, class... Args>
            void* operator new(size_t size, const This&, allocator_arg_t, const Alloc& alloc,
                               const Args&...);

        void operator delete(void* pointer, size_t size) noexcept;

    private:
        add_pointer_t<yielded> value_ = nullptr;    // exposition only
        exception_ptr except_;                      // exposition only
    };
}
```

```
generator get_return_object() noexcept;
```

- 1 *Returns:* A generator object whose member *coroutine_* is *coroutine_handle<promise_type>::from_promise(*this)*, and whose member *active_* points to an empty stack.

```
auto final_suspend() noexcept;
```

- 2 *Preconditions:* A handle referring to the coroutine whose promise object is **this* is at the top of **active_* of some generator object *x*. This function is called by that coroutine upon reaching its final suspend point (9.6.4).

- 3 *Returns:* An awaitable object of unspecified type (7.6.2.4) whose member functions arrange for the calling coroutine to be suspended, pop the coroutine handle from the top of **x.active_*, and resume execution of the coroutine referred to by *x.active_>top()* if **x.active_* is not empty. If it is empty, control flow returns to the current coroutine caller or resumer (9.6.4).

```
suspend_always yield_value(yielded val) noexcept;
```

- 4 *Effects:* Equivalent to *value_ = addressof(val)*.

- 5 *Returns:* {}.

```
auto yield_value(const remove_reference_t<yielded>& lval)
requires is_rvalue_reference_v<yielded> &&
constructible_from<remove_cvref_t<yielded>, const remove_reference_t<yielded>&>;
```

- 6 *Preconditions:* A handle referring to the coroutine whose promise object is **this* is at the top of **active_* of some generator object.

- 7 *Returns:* An awaitable object of an unspecified type (7.6.2.4) that stores an object of type *remove_cvref_t<yielded>* direct-non-list-initialized with *lval*, whose member functions arrange for *value_* to point to that stored object and then suspend the coroutine.

- 8 *Throws:* Any exception thrown by the initialization of the stored object.

- 9 *Remarks:* A *yield-expression* that calls this function has type *void* (7.6.17).

```
template<class R2, class V2, class Alloc2, class Unused>
requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&&, Unused> g) noexcept;
template<class R2, class V2, class Alloc2, class Unused>
requires same_as<typename generator<R2, V2, Alloc2>::yielded, yielded>
auto yield_value(ranges::elements_of<generator<R2, V2, Alloc2>&, Unused> g) noexcept;
```

- 10 *Preconditions:* A handle referring to the coroutine whose promise object is **this* is at the top of **active_* of some generator object *x*. The coroutine referred to by *g.range.coroutine_* is suspended at its initial suspend point.

- 11 *Returns:* An awaitable object of an unspecified type (7.6.2.4) into which *g.range* is moved, whose member *await_ready* returns *false*, whose member *await_suspend* pushes *g.range.coroutine_* into **x.active_* and resumes execution of the coroutine referred to by *g.range.coroutine_*, and whose member *await_resume* evaluates *rethrow_exception(exception_)* if *bool(exception_)* is *true*. If *bool(exception_)* is *false*, the *await_resume* member has no effects.

- 12 *Remarks:* A *yield-expression* that calls one of these functions has type *void* (7.6.17).

```
template<ranges::input_range R, class Alloc>
requires convertible_to<ranges::range_reference_t<R>, yielded>
auto yield_value(ranges::elements_of<R, Alloc> r);
```

- 13 *Effects:* Equivalent to:

```
auto nested = [] (allocator_arg_t, Alloc, ranges::iterator_t<R> i, ranges::sentinel_t<R> s)
-> generator<yielded, void, Alloc> {
    for (; i != s; ++i) {
        co_yield static_cast<yielded>(*i);
    }
};
return yield_value(ranges::elements_of(nested(
    allocator_arg, r.allocator, ranges::begin(r.range), ranges::end(r.range))));
```

14 [Note 1: A *yield-expression* that calls this function has type `void` (7.6.17). — end note]

```
void unhandled_exception();
```

15 *Preconditions:* A handle referring to the coroutine whose promise object is `*this` is at the top of `*active_` of some generator object `x`.

16 *Effects:* If the handle referring to the coroutine whose promise object is `*this` is the sole element of `*x.active_`, equivalent to `throw`, otherwise, assigns `current_exception()` to `except_`.

```
void* operator new(size_t size)
    requires same_as<Allocator, void> || default_initializable<Allocator>;
```

```
template<class Alloc, class... Args>
    void* operator new(size_t size, allocator_arg_t, const Alloc& alloc, const Args&...);
```

```
template<class This, class Alloc, class... Args>
    void* operator new(size_t size, const This&, allocator_arg_t, const Alloc& alloc,
        const Args&...);
```

17 Let `A` be

(17.1) — `Allocator`, if it is not `void`,

(17.2) — `Alloc` for the overloads with a template parameter `Alloc`, or

(17.3) — `allocator<void>` otherwise.

Let `B` be `allocator_traits<A>::template rebind_alloc<U>` where `U` is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

18 *Mandates:* `allocator_traits<B>::pointer` is a pointer type. For the overloads with a template parameter `Alloc`, `same_as<Allocator, void> || convertible_to<const Alloc&, Allocator>` is modeled.

19 *Effects:* Initializes an allocator `b` of type `B` with `A(alloc)`, for the overloads with a function parameter `alloc`, and with `A()` otherwise. Uses `b` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `b`.

20 *Returns:* A pointer to the allocated storage.

```
void operator delete(void* pointer, size_t size) noexcept;
```

21 *Preconditions:* `pointer` was returned from an invocation of one of the above overloads of `operator new` with a `size` argument equal to `size`.

22 *Effects:* Deallocates the storage pointed to by `pointer` using an allocator equivalent to that used to allocate it.

25.8.6 Class `generator::iterator`

[`coro.generator.iterator`]

```
namespace std {
    template<class Ref, class Val, class Allocator>
    class generator<Ref, Val, Allocator>::iterator {
    public:
        using value_type = Val;
        using difference_type = ptrdiff_t;

        iterator(iterator&& other) noexcept;
        iterator& operator=(iterator&& other) noexcept;

        reference operator*() const noexcept(is_nothrow_copy_constructible_v<reference>);
        iterator& operator++();
        void operator++(int);

        friend bool operator==(const iterator& i, default_sentinel_t);

    private:
        coroutine_handle<promise_type> coroutine_; // exposition only
    }
```

```

    };
}

iterator(iterator&& other) noexcept;
1      Effects: Initializes coroutine_ with exchange(other.coroutine_, {}).

iterator& operator=(iterator&& other) noexcept;
2      Effects: Equivalent to coroutine_ = exchange(other.coroutine_, {}).
3      Returns: *this.

reference operator*() const noexcept(is_nothrow_copy_constructible_v<reference>);
4      Preconditions: For some generator object x, coroutine_ is in *x.active_ and x.active_→top()
    refers to a suspended coroutine with promise object p.
5      Effects: Equivalent to: return static_cast<reference>(*p.value_);

iterator& operator++();
6      Preconditions: For some generator object x, coroutine_ is in *x.active_.
7      Effects: Equivalent to x.active_→top().resume().
8      Returns: *this.

void operator++(int);
9      Effects: Equivalent to +++this.

friend bool operator==(const iterator& i, default_sentinel_t);
10     Effects: Equivalent to: return i.coroutine_.done();

```

26 Algorithms library

[algorithms]

26.1 General

[algorithms.general]

- ¹ This Clause describes components that C++ programs may use to perform algorithmic operations on containers (Clause 23) and other sequences.
- ² The following subclauses describe components for non-modifying sequence operations, mutating sequence operations, sorting and related operations, and algorithms from the C library, as summarized in Table 85.

Table 85 — Algorithms library summary [tab:algorithms.summary]

Subclause	Header
26.2 Algorithms requirements	
26.3 Parallel algorithms	<execution>
26.5 Algorithm result types	<algorithm>
26.6 Non-modifying sequence operations	
26.7 Mutating sequence operations	
26.8 Sorting and related operations	
26.10 Generalized numeric operations	<numeric>
26.11 Specialized <memory> algorithms	<memory>
26.12 Specialized <random> algorithms	<random>
26.13 C library algorithms	<cstdlib>

26.2 Algorithms requirements

[algorithms.requirements]

- ¹ All of the algorithms are separated from the particular implementations of data structures and are parameterized by iterator types. Because of this, they can work with program-defined data structures, as long as these data structures have iterator types satisfying the assumptions on the algorithms.
 - ² The entities defined in the `std::ranges` namespace in this Clause and specified as function templates are algorithm function objects (16.3.3.4).
 - ³ For purposes of determining the existence of data races, algorithms shall not modify objects referenced through an iterator argument unless the specification requires such modification.
 - ⁴ Throughout this Clause, where the template parameters are not constrained, the names of template parameters are used to express type requirements.
- (4.1) — If an algorithm's *Effects* element specifies that a value pointed to by any iterator passed as an argument is modified, then the type of that argument shall meet the requirements of a mutable iterator (24.3).
 - (4.2) — If an algorithm's template parameter is named `InputIterator`, `InputIterator1`, or `InputIterator2`, the template argument shall meet the *Cpp17InputIterator* requirements (24.3.5.3).
 - (4.3) — If an algorithm's template parameter is named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, the template argument shall meet the *Cpp17OutputIterator* requirements (24.3.5.4).
 - (4.4) — If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, `ForwardIterator2`, or `NoThrowForwardIterator`, the template argument shall meet the *Cpp17ForwardIterator* requirements (24.3.5.5) if it is required to be a mutable iterator, or model `forward_iterator` (24.3.4.11) otherwise.
 - (4.5) — If an algorithm's template parameter is named `NoThrowForwardIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.
 - (4.6) — If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, or `BidirectionalIterator2`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements (24.3.5.6) if it is required to be a mutable iterator, or model `bidirectional_iterator` (24.3.4.12) otherwise.

- (4.7) — If an algorithm's template parameter is named `RandomAccessIterator`, `RandomAccessIterator1`, or `RandomAccessIterator2`, the template argument shall meet the *Cpp17RandomAccessIterator* requirements (24.3.5.7) if it is required to be a mutable iterator, or model `random_access_iterator` (24.3.4.13) otherwise.

[Note 1: These requirements do not affect iterator arguments that are constrained, for which iterator category and mutability requirements are expressed explicitly. — end note]

- 5 Both in-place and copying versions are provided for certain algorithms.¹⁷⁹ When such a version is provided for *algorithm* it is called *algorithm_copy*. Algorithms that take predicates end with the suffix `_if` (which follows the suffix `_copy`).
- 6 When not otherwise constrained, the `Predicate` parameter is used whenever an algorithm expects a function object (22.10) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as `true`. If an algorithm takes `Predicate pred` as its argument and `first` as its iterator argument with value type `T`, the expression `pred(*first)` shall be well-formed and the type `decltype(pred(*first))` shall model *boolean-testable* (18.5.2). The function object `pred` shall not apply any non-constant function through its argument. Given a glvalue `u` of type (possibly `const`) `T` that designates the same object as `*first`, `pred(u)` shall be a valid expression that is equal to `pred(*first)`.
- 7 When not otherwise constrained, the `BinaryPredicate` parameter is used whenever an algorithm expects a function object that, when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type `T` when `T` is part of the signature, returns a value testable as `true`. If an algorithm takes `BinaryPredicate binary_pred` as its argument and `first1` and `first2` as its iterator arguments with respective value types `T1` and `T2`, the expression `binary_pred(*first1, *first2)` shall be well-formed and the type `decltype(binary_pred(*first1, *first2))` shall model *boolean-testable*. Unless otherwise specified, `BinaryPredicate` always takes the first iterator's `value_type` as its first argument, that is, in those cases when `T` value is part of the signature, the expression `binary_pred(*first1, value)` shall be well-formed and the type `decltype(binary_pred(*first1, value))` shall model *boolean-testable*. `binary_pred` shall not apply any non-constant function through any of its arguments. Given a glvalue `u` of type (possibly `const`) `T1` that designates the same object as `*first1`, and a glvalue `v` of type (possibly `const`) `T2` that designates the same object as `*first2`, `binary_pred(u, *first2)`, `binary_pred(*first1, v)`, and `binary_pred(u, v)` shall each be a valid expression that is equal to `binary_pred(*first1, *first2)`, and `binary_pred(u, value)` shall be a valid expression that is equal to `binary_pred(*first1, value)`.
- 8 The parameters `UnaryOperation`, `BinaryOperation`, `BinaryOperation1`, and `BinaryOperation2` are used whenever an algorithm expects a function object (22.10).
- 9 [Note 2: Unless otherwise specified, algorithms that take function objects as arguments can copy those function objects freely. If object identity is important, a wrapper class that points to a non-copied implementation object such as `reference_wrapper<T>` (22.10.6), or some equivalent solution, can be used. — end note]
- 10 When the description of an algorithm gives an expression such as `*first == value` for a condition, the expression shall evaluate to either `true` or `false` in boolean contexts.
- 11 In the description of the algorithms, operator `+` is used for some of the iterator categories for which it does not have to be defined. In these cases the semantics of `a + n` are the same as those of

```
auto tmp = a;
for (; n < 0; ++n) --tmp;
for (; n > 0; --n) ++tmp;
return tmp;
```

Similarly, operator `-` is used for some combinations of iterators and sentinel types for which it does not have to be defined. If `[a, b)` denotes a range, the semantics of `b - a` in these cases are the same as those of

```
iter_difference_t<decltype(a)> n = 0;
for (auto tmp = a; tmp != b; ++tmp) ++n;
return n;
```

and if `[b, a)` denotes a range, the same as those of

```
iter_difference_t<decltype(b)> n = 0;
for (auto tmp = b; tmp != a; ++tmp) --n;
return n;
```

¹⁷⁹) The decision whether to include a copying version was usually based on complexity considerations. When the cost of doing the operation dominates the cost of copy, the copying version is not included. For example, `sort_copy` is not included because the cost of sorting is much more significant, and users can invoke `copy` followed by `sort`.

For each iterator *i* and sentinel *s* produced from a range *r*, the semantics of *s* - *i* are the same as those of an expression that has the same type, value, and value category as `ranges::distance(i, s)`.

[*Note 3*: The implementation can use `ranges::distance(r)` when that produces the same value as `ranges::distance(i, s)`. This can be more efficient for sized ranges. — *end note*]

- 12 In the description of the algorithms, given an iterator *a* whose difference type is *D*, and an expression *n* of integer-like type other than *cv D*, the semantics of *a* + *n* and *a* - *n* are, respectively, those of *a* + *D*(*n*) and *a* - *D*(*n*).
- 13 In the description of algorithm return values, a sentinel value *s* denoting the end of a range [*i*, *s*) is sometimes returned where an iterator is expected. In these cases, the semantics are as if the sentinel is converted into an iterator using `ranges::next(i, s)`.
- 14 Overloads of algorithms that take **range** arguments (25.4.2) behave as if they are implemented by dispatching to the overload in namespace **ranges** that takes separate iterator and sentinel arguments, where for each range argument *r*
 - (14.1) — a corresponding iterator argument is initialized with `ranges::begin(r)` and
 - (14.2) — a corresponding sentinel argument is initialized with `ranges::end(r)`, or `ranges::begin(r) + N` where *N* is equal to `ranges::distance(r)`.
- 15 The well-formedness and behavior of a call to an algorithm with an explicitly-specified template argument list is unspecified, except where explicitly stated otherwise.

[*Note 4*: Consequently, an implementation can declare an algorithm with different template parameters than those presented. — *end note*]

26.3 Parallel algorithms

[**algorithms.parallel**]

26.3.1 Preamble

[**algorithms.parallel.defns**]

- 1 Subclause 26.3 describes components that C++ programs may use to perform operations on containers and other sequences in parallel.
 - 2 A *parallel algorithm* is a function template listed in this document with a template parameter named **ExecutionPolicy** or constrained by the following exposition-only concept:

```
template<class Ep>
concept execution_policy = is_execution_policy_v<remove_cvref_t<Ep>>;    // exposition only
```

Such a template parameter is termed an *execution policy template parameter*.
 - 3 A parallel algorithm accesses objects indirectly accessible via its arguments by invoking the following functions:
 - (3.1) — All operations of the categories of the iterators, sentinels, or **mdspan** types that the algorithm is instantiated with.
 - (3.2) — Operations on those sequence elements that are required by its specification.
 - (3.3) — User-provided invocable objects to be applied during the execution of the algorithm, if required by the specification.
 - (3.4) — Operations on those invocable objects required by the specification.[*Note 1*: See 26.2. — *end note*]
- These functions are herein called *element access functions*.
- 4 [*Example 1*: The **sort** function may invoke the following element access functions:
 - (4.1) — Operations of the random-access iterator of the actual template argument (as per 24.3.5.7), as implied by the name of the template parameter **RandomAccessIterator**.
 - (4.2) — The **swap** function on the elements of the sequence (as per the preconditions specified in 26.8.2.1).
 - (4.3) — The user-provided **Compare** function object.— *end example*]
 - 5 A standard library function is *vectorization-unsafe* if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function or lock-free atomic modify-write operation (32.5.4).

[*Note 2:* Implementations must ensure that internal synchronization inside standard library functions does not prevent forward progress when those functions are executed by threads of execution with weakly parallel forward progress guarantees. — *end note*]

[*Example 2:*

```
int x = 0;
std::mutex m;
void f() {
    int a[] = {1,2};
    std::for_each(std::execution::par_unseq, std::begin(a), std::end(a), [&](int) {
        std::lock_guard<mutex> guard(m);           // incorrect: lock_guard constructor calls m.lock()
        ++x;
    });
}
```

The above program may result in two consecutive calls to `m.lock()` on the same thread of execution (which may deadlock), because the applications of the function object are not guaranteed to run on different threads of execution. — *end example*]

26.3.2 Requirements on user-provided function objects [algorithms.parallel.user]

- ¹ Unless otherwise specified, invocable objects passed into parallel algorithms as objects of a type denoted by a template parameter named `Predicate`, `BinaryPredicate`, `Compare`, `UnaryOperation`, `BinaryOperation`, `BinaryOperation1`, `BinaryOperation2`, `BinaryDivideOp`, or constrained by a concept whose semantic requirements include that the type models `regular_invocable` and the operators used by the analogous overloads to these parallel algorithms that are formed by an invocation with the specified default predicate or operation (where applicable) shall not directly or indirectly modify objects via their arguments, nor shall they rely on the identity of the provided objects.

26.3.3 Effect of execution policies on algorithm execution [algorithms.parallel.exec]

- ¹ An execution policy template parameter describes the manner in which the execution of a parallel algorithm may be parallelized and the manner in which it applies the element access functions.
- ² If an object is modified by an element access function, the algorithm will perform no other unsynchronized accesses to that object. The modifying element access functions are those which are specified as modifying the object.

[*Note 1:* For example, `swap`, `++`, `--`, `@=`, and assignments modify the object. For the assignment and `@=` operators, only the left argument is modified. — *end note*]

- ³ Unless otherwise stated, implementations may make arbitrary copies of elements (with type `T`) from sequences where `is_trivially_copy_constructible_v<T>` and `is_trivially_destructible_v<T>` are `true`.

[*Note 2:* This implies that user-supplied function objects cannot rely on object identity of arguments for such input sequences. If object identity of the arguments to these function objects is important, a wrapping iterator that returns a non-copied implementation object such as `reference_wrapper<T>` (22.10.6), or some equivalent solution, can be used. — *end note*]

- ⁴ The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::sequenced_policy` all occur in the calling thread of execution.

[*Note 3:* The invocations are not interleaved; see 6.10.1. — *end note*]

- ⁵ The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::unsequenced_policy` are permitted to execute in an unordered fashion in the calling thread of execution, unsequenced with respect to one another in the calling thread of execution.

[*Note 4:* This means that multiple function object invocations can be interleaved on a single thread of execution, which overrides the usual guarantee from 6.10.1 that function executions do not overlap with one another. — *end note*]

The behavior of a program is undefined if it invokes a vectorization-unsafe standard library function from user code called from an `execution::unsequenced_policy` algorithm.

[*Note 5:* Because `execution::unsequenced_policy` allows the execution of element access functions to be interleaved on a single thread of execution, blocking synchronization, including the use of mutexes, risks deadlock. — *end note*]

- ⁶ The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::parallel_policy` are permitted to execute either in the invoking thread of execution or in a thread of execution implicitly created by the library to support parallel algorithm execution. If the

threads of execution created by `thread` (32.4.3) or `jthread` (32.4.4) provide concurrent forward progress guarantees (6.10.2.3), then a thread of execution implicitly created by the library will provide parallel forward progress guarantees; otherwise, the provided forward progress guarantee is implementation-defined. Any such invocations executing in the same thread of execution are indeterminately sequenced with respect to each other.

[Note 6: It is the caller's responsibility to ensure that the invocation does not introduce data races or deadlocks. — end note]

[Example 1:

```
int a[] = {0,1};
std::vector<int> v;
std::for_each(std::execution::par, std::begin(a), std::end(a), [&](int i) {
    v.push_back(i*2+1);           // incorrect: data race
});
```

The program above has a data race because of the unsynchronized access to the container `v`. — end example]

[Example 2:

```
std::atomic<int> x{0};
int a[] = {1,2};
std::for_each(std::execution::par, std::begin(a), std::end(a), [&](int) {
    x.fetch_add(1, std::memory_order::relaxed);
    // spin wait for another iteration to change the value of x
    while (x.load(std::memory_order::relaxed) == 1) { }           // incorrect: assumes execution order
});
```

The above example depends on the order of execution of the iterations, and will not terminate if both iterations are executed sequentially on the same thread of execution. — end example]

[Example 3:

```
int x = 0;
std::mutex m;
int a[] = {1,2};
std::for_each(std::execution::par, std::begin(a), std::end(a), [&](int) {
    std::lock_guard<mutex> guard(m);
    ++x;
});
```

The above example synchronizes access to object `x` ensuring that it is incremented correctly. — end example]

- 7 The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::parallel_unsequenced_policy` are permitted to execute in an unordered fashion in unspecified threads of execution, and unsequenced with respect to one another within each thread of execution. These threads of execution are either the invoking thread of execution or threads of execution implicitly created by the library; the latter will provide weakly parallel forward progress guarantees.

[Note 7: This means that multiple function object invocations can be interleaved on a single thread of execution, which overrides the usual guarantee from 6.10.1 that function executions do not overlap with one another. — end note]

The behavior of a program is undefined if it invokes a vectorization-unsafe standard library function from user code called from an `execution::parallel_unsequenced_policy` algorithm.

[Note 8: Because `execution::parallel_unsequenced_policy` allows the execution of element access functions to be interleaved on a single thread of execution, blocking synchronization, including the use of mutexes, risks deadlock. — end note]

- 8 [Note 9: The semantics of invocation with `execution::unsequenced_policy`, `execution::parallel_policy`, or `execution::parallel_unsequenced_policy` allow the implementation to fall back to sequential execution if the system cannot parallelize an algorithm invocation, e.g., due to lack of resources. — end note]

- 9 If an invocation of a parallel algorithm uses threads of execution implicitly created by the library, then the invoking thread of execution will either

- (9.1) — temporarily block with forward progress guarantee delegation (6.10.2.3) on the completion of these library-managed threads of execution, or
- (9.2) — eventually execute an element access function;

the thread of execution will continue to do so until the algorithm is finished.

[*Note 10*: In blocking with forward progress guarantee delegation in this context, a thread of execution created by the library is considered to have finished execution as soon as it has finished the execution of the particular element access function that the invoking thread of execution logically depends on. — *end note*]

- ¹⁰ The semantics of parallel algorithms invoked with an execution policy object of implementation-defined type are implementation-defined.

26.3.4 Parallel algorithm exceptions

[algorithms.parallel.exceptions]

- ¹ During the execution of a parallel algorithm, if temporary memory resources are required for parallelization and none are available, the algorithm throws a `bad_alloc` exception.
- ² During the execution of a parallel algorithm, if the invocation of an element access function exits via an uncaught exception, the behavior is determined by the execution policy.

26.3.5 Parallel algorithm overloads

[algorithms.parallel.overloads]

- ¹ Parallel algorithms are algorithm overloads. Each parallel algorithm overload has an additional function parameter *P* of type `T&&` as the first function parameter, where *T* is the execution policy template parameter. [*Note 1*: Not all algorithms have parallel algorithm overloads. — *end note*]
- ² Unless otherwise specified, the semantics of calling a parallel algorithm overload are identical to calling the corresponding algorithm overload without the parameter *P*, using all but the first argument.
- ³ Unless otherwise specified, the complexity requirements of a parallel algorithm overload are relaxed from the complexity requirements of the corresponding overload without the parameter *P* as follows: when the guarantee says “at most *expr*” or “exactly *expr*” and does not specify the number of assignments or swaps, and *expr* is not already expressed with $\mathcal{O}()$ notation, the complexity of the algorithm shall be $\mathcal{O}(\textit{expr})$.
- ⁴ A parallel algorithm with a template parameter named `ExecutionPolicy` shall not participate in overload resolution unless that template parameter satisfies *execution-policy*.

26.3.6 Execution policies

[execpol]

26.3.6.1 General

[execpol.general]

- ¹ Subclause 26.3.6 describes classes that are *execution policy* types. An object of an execution policy type indicates the kinds of parallelism allowed in the execution of an algorithm and expresses the consequent requirements on the element access functions. Execution policy types are declared in header `<execution>` (33.4).

[*Example 1*:

```
using namespace std;
vector<int> v = /* ... */;

// standard sequential sort
sort(v.begin(), v.end());

// explicitly sequential sort
sort(execution::seq, v.begin(), v.end());

// permitting parallel execution
sort(execution::par, v.begin(), v.end());

// permitting vectorization as well
sort(execution::par_unseq, v.begin(), v.end());
```

— *end example*]

[*Note 1*: Implementations can provide additional execution policies to those described in this document as extensions to address parallel architectures that require idiosyncratic parameters for efficient execution. — *end note*]

26.3.6.2 Execution policy type trait

[execpol.type]

```
template<class T> struct is_execution_policy;
```

- ¹ `is_execution_policy` can be used to detect execution policies for the purpose of excluding function signatures from otherwise ambiguous overload resolution participation.

2 `is_execution_policy<T>` is a *Cpp17UnaryTypeTrait* with a base characteristic of `true_type` if `T` is the type of a standard or implementation-defined execution policy, otherwise `false_type`.
 [Note 1: This provision reserves the privilege of creating non-standard execution policies to the library implementation. — end note]

3 The behavior of a program that adds specializations for `is_execution_policy` is undefined.

26.3.6.3 Sequenced execution policy [execpol.seq]

```
class execution::sequenced_policy { unspecified };
```

1 The class `execution::sequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and require that a parallel algorithm's execution may not be parallelized.

2 During the execution of a parallel algorithm with the `execution::sequenced_policy` policy, if the invocation of an element access function exits via an exception, `terminate` is invoked (14.6.2).

26.3.6.4 Parallel execution policy [execpol.par]

```
class execution::parallel_policy { unspecified };
```

1 The class `execution::parallel_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized.

2 During the execution of a parallel algorithm with the `execution::parallel_policy` policy, if the invocation of an element access function exits via an exception, `terminate` is invoked (14.6.2).

26.3.6.5 Parallel and unsequenced execution policy [execpol.parunseq]

```
class execution::parallel_unsequenced_policy { unspecified };
```

1 The class `execution::parallel_unsequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be parallelized and vectorized.

2 During the execution of a parallel algorithm with the `execution::parallel_unsequenced_policy` policy, if the invocation of an element access function exits via an exception, `terminate` is invoked (14.6.2).

26.3.6.6 Unsequenced execution policy [execpol.unseq]

```
class execution::unsequenced_policy { unspecified };
```

1 The class `unsequenced_policy` is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be vectorized, e.g., executed on a single thread using instructions that operate on multiple data items.

2 During the execution of a parallel algorithm with the `execution::unsequenced_policy` policy, if the invocation of an element access function exits via an exception, `terminate` is invoked (14.6.2).

26.3.6.7 Execution policy objects [execpol.objects]

```
inline constexpr execution::sequenced_policy      execution::seq{ unspecified };
inline constexpr execution::parallel_policy        execution::par{ unspecified };
inline constexpr execution::parallel_unsequenced_policy execution::par_unseq{ unspecified };
inline constexpr execution::unsequenced_policy     execution::unseq{ unspecified };
```

1 The header `<execution>` (33.4) declares global objects associated with each type of execution policy.

26.4 Header `<algorithm>` synopsis [algorithm.syn]

```
// mostly freestanding
#include <initializer_list>    // see 17.11.2
```

```
namespace std {
  namespace ranges {
    // 26.5, algorithm result types
    template<class I, class F>
      struct in_fun_result;
```

```

template<class I1, class I2>
    struct in_in_result;

template<class I, class O>
    struct in_out_result;

template<class I1, class I2, class O>
    struct in_in_out_result;

template<class I, class O1, class O2>
    struct in_out_out_result;

template<class T>
    struct min_max_result;

template<class I>
    struct in_found_result;

template<class I, class T>
    struct in_value_result;

template<class O, class T>
    struct out_value_result;
}

// 26.6, non-modifying sequence operations
// 26.6.1, all of
template<class InputIterator, class Predicate>
    constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool all_of(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        constexpr bool all_of(I first, S last, Pred pred, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        constexpr bool all_of(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        bool all_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        bool all_of(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.2, any of
template<class InputIterator, class Predicate>
    constexpr bool any_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool any_of(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
               ForwardIterator first, ForwardIterator last, Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        constexpr bool any_of(I first, S last, Pred pred, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        constexpr bool any_of(R&& r, Pred pred, Proj proj = {});
}

```

```

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    bool any_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    bool any_of(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.3, none of
template<class InputIterator, class Predicate>
    constexpr bool none_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool none_of(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        constexpr bool none_of(I first, S last, Pred pred, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        constexpr bool none_of(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        bool none_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        bool none_of(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.4, contains
namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        constexpr bool contains(I first, S last, const T& value, Proj proj = {});
    template<input_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires
            indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
        constexpr bool contains(R&& r, const T& value, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        bool contains(Ep&& exec, I first, S last, const T& value,
            Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires
            indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
        bool contains(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted

    template<forward_iterator I1, sentinel_for<I1> S1,
            forward_iterator I2, sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        constexpr bool contains_subrange(I1 first1, S1 last1, I2 first2, S2 last2,
            Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<forward_range R1, forward_range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        constexpr bool contains_subrange(R1&& r1, R2&& r2,

```



```

        Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool contains_subrange(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted

template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool contains_subrange(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {}, Proj1 proj1 = {},
        Proj2 proj2 = {}); // freestanding-deleted
}

// 26.6.5, for each
template<class InputIterator, class Function>
constexpr Function for_each(InputIterator first, InputIterator last, Function f);
template<class ExecutionPolicy, class ForwardIterator, class Function>
void for_each(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, Function f);

namespace ranges {
    template<class I, class F>
    using for_each_result = in_fun_result<I, F>;

    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirectly_unary_invocable<projected<I, Proj>> Fun>
    constexpr for_each_result<I, Fun>
        for_each(I first, S last, Fun f, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    constexpr for_each_result<borrowed_iterator_t<R>, Fun>
        for_each(R&& r, Fun f, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity,
            indirectly_unary_invocable<projected<I, Proj>> Fun>
    I for_each(Ep&& exec, I first, S last, Fun f, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    borrowed_iterator_t<R>
        for_each(Ep&& exec, R&& r, Fun f, Proj proj = {}); // freestanding-deleted
}

template<class InputIterator, class Size, class Function>
constexpr InputIterator for_each_n(InputIterator first, Size n, Function f);
template<class ExecutionPolicy, class ForwardIterator, class Size, class Function>
ForwardIterator for_each_n(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, Size n, Function f);

namespace ranges {
    template<class I, class F>
    using for_each_n_result = in_fun_result<I, F>;

    template<input_iterator I, class Proj = identity,
            indirectly_unary_invocable<projected<I, Proj>> Fun>
    constexpr for_each_n_result<I, Fun>
        for_each_n(I first, iter_difference_t<I> n, Fun f, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, class Proj = identity,
            indirectly_unary_invocable<projected<I, Proj>> Fun>
    I for_each_n(Ep&& exec, I first, iter_difference_t<I> n, Fun f,
        Proj proj = {}); // freestanding-deleted
}

```

```

}

// 26.6.6, find
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
    constexpr InputIterator find(InputIterator first, InputIterator last,
                                const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
    ForwardIterator find(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                        ForwardIterator first, ForwardIterator last,
                        const T& value);
template<class InputIterator, class Predicate>
    constexpr InputIterator find_if(InputIterator first, InputIterator last,
                                    Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator find_if(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                           ForwardIterator first, ForwardIterator last,
                           Predicate pred);
template<class InputIterator, class Predicate>
    constexpr InputIterator find_if_not(InputIterator first, InputIterator last,
                                        Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator find_if_not(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                               ForwardIterator first, ForwardIterator last,
                               Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        constexpr I find(I first, S last, const T& value, Proj proj = {});
    template<input_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires indirect_binary_predicate<ranges::equal_to,
                                           projected<iterator_t<R>, Proj>, const T*>
        constexpr borrowed_iterator_t<R>
            find(R&& r, const T& value, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        I find(Ep&& exec, I first, S last, const T& value, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires indirect_binary_predicate<ranges::equal_to,
                                           projected<iterator_t<R>, Proj>, const T*>
        borrowed_iterator_t<R>
            find(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted

    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        constexpr I find_if(I first, S last, Pred pred, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        constexpr borrowed_iterator_t<R>
            find_if(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        I find_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        borrowed_iterator_t<R>
            find_if(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted

```



```

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr I find_if_not(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr borrowed_iterator_t<R>
    find_if_not(R&& r, Pred pred, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
I find_if_not(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_iterator_t<R>
    find_if_not(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.7, find last
namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr subrange<I> find_last(I first, S last, const T& value, Proj proj = {});
    template<forward_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
    requires
    indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_subrange_t<R> find_last(R&& r, const T& value, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    subrange<I>
    find_last(Ep&& exec, I first, S last, const T& value,
        Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
    requires
    indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
    borrowed_subrange_t<R>
    find_last(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted

    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr subrange<I> find_last_if(I first, S last, Pred pred, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_subrange_t<R> find_last_if(R&& r, Pred pred, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    subrange<I>
    find_last_if(Ep&& exec, I first, S last, Pred pred,
        Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R,
            class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_subrange_t<R>
    find_last_if(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted

    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr subrange<I> find_last_if_not(I first, S last, Pred pred, Proj proj = {});

```

```

template<forward_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr borrowed_subrange_t<R> find_last_if_not(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
subrange<I>
    find_last_if_not(Ep&& exec, I first, S last, Pred pred,
                    Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
borrowed_subrange_t<R>
    find_last_if_not(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.8, find end
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
    find_end(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1,
        class ForwardIterator2, class BinaryPredicate>
ForwardIterator1
    find_end(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);

namespace ranges {
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr subrange<I1>
    find_end(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr borrowed_subrange_t<R1>
    find_end(R1&& r1, R2&& r2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
subrange<I1>
    find_end(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
            Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
borrowed_subrange_t<R1>

```

```

        find_end(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                Proj1 proj1 = {}, Proj2 proj2 = {});           // freestanding-deleted
    }

// 26.6.9, find first of
template<class InputIterator, class ForwardIterator>
constexpr InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2);
template<class InputIterator, class ForwardIterator, class BinaryPredicate>
constexpr InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2,
                  BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
    find_first_of(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1,
         class ForwardIterator2, class BinaryPredicate>
ForwardIterator1
    find_first_of(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  BinaryPredicate pred);

namespace ranges {
    template<input_iterator I1, sentinel_for<I1> S1, forward_iterator I2, sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        constexpr I1 find_first_of(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                                    Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, forward_range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        constexpr borrowed_iterator_t<R1>
            find_first_of(R1&& r1, R2&& r2, Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        I1 find_first_of(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});           // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        borrowed_iterator_t<R1>
            find_first_of(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});           // freestanding-deleted
}

// 26.6.10, adjacent find
template<class ForwardIterator>
constexpr ForwardIterator
    adjacent_find(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator
    adjacent_find(ForwardIterator first, ForwardIterator last,
                  BinaryPredicate pred);

```

```

template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator
        adjacent_find(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
    ForwardIterator
        adjacent_find(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last,
                      BinaryPredicate pred);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_binary_predicate<projected<I, Proj>,
                                     projected<I, Proj>> Pred = ranges::equal_to>
        constexpr I adjacent_find(I first, S last, Pred pred = {},
                                   Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_binary_predicate<projected<iterator_t<R>, Proj>,
                                     projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
        constexpr borrowed_iterator_t<R>
            adjacent_find(R&& r, Pred pred = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity,
            indirect_binary_predicate<projected<I, Proj>,
                                     projected<I, Proj>> Pred = ranges::equal_to>
        I adjacent_find(Ep&& exec, I first, S last, Pred pred = {},
                        Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
            indirect_binary_predicate<projected<iterator_t<R>, Proj>,
                                     projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
        borrowed_iterator_t<R>
            adjacent_find(Ep&& exec, R&& r, Pred pred = {}, Proj proj = {}); // freestanding-deleted
}

// 26.6.11, count
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
    constexpr typename iterator_traits<InputIterator>::difference_type
        count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
    typename iterator_traits<ForwardIterator>::difference_type
        count(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
              ForwardIterator first, ForwardIterator last, const T& value);
template<class InputIterator, class Predicate>
    constexpr typename iterator_traits<InputIterator>::difference_type
        count_if(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    typename iterator_traits<ForwardIterator>::difference_type
        count_if(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                 ForwardIterator first, ForwardIterator last, Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        constexpr iter_difference_t<I>
            count(I first, S last, const T& value, Proj proj = {});
    template<input_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires indirect_binary_predicate<ranges::equal_to,
                                           projected<iterator_t<R>, Proj>, const T*>
        constexpr range_difference_t<R>
            count(R&& r, const T& value, Proj proj = {});
}

```

```

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
iter_difference_t<I>
    count(Ep&& exec, I first, S last, const T& value, Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
requires indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T*>
range_difference_t<R>
    count(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr iter_difference_t<I>
    count_if(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr range_difference_t<R>
    count_if(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
iter_difference_t<I>
    count_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
range_difference_t<R>
    count_if(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

// 26.6.12, mismatch
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2);
template<class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, BinaryPredicate pred);
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, InputIterator2 last2,
            BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
    mismatch(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
pair<ForwardIterator1, ForwardIterator2>
    mismatch(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
    mismatch(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,

```

```

        ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
pair<ForwardIterator1, ForwardIterator2>
    mismatch(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2,
        BinaryPredicate pred);

namespace ranges {
    template<class I1, class I2>
        using mismatch_result = in_in_result<I1, I2>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        constexpr mismatch_result<I1, I2>
            mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        constexpr mismatch_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            mismatch(R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        mismatch_result<I1, I2>
            mismatch(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        mismatch_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            mismatch(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

// 26.6.13, equal
template<class InputIterator1, class InputIterator2>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2);
template<class InputIterator1, class InputIterator2, class BinaryPredicate>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, BinaryPredicate pred);
template<class InputIterator1, class InputIterator2>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class BinaryPredicate>
    constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2,
        BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    bool equal(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
    bool equal(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, BinaryPredicate pred);

```



```

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    bool equal(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
               ForwardIterator1 first1, ForwardIterator1 last1,
               ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
    bool equal(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
               ForwardIterator1 first1, ForwardIterator1 last1,
               ForwardIterator2 first2, ForwardIterator2 last2,
               BinaryPredicate pred);

namespace ranges {
    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        constexpr bool equal(I1 first1, S1 last1, I2 first2, S2 last2,
                             Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, class Pred = ranges::equal_to,
            class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        constexpr bool equal(R1&& r1, R2&& r2, Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
        bool equal(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                  Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
        bool equal(Ep&& exec, R1&& r1, R2&& r2,
                  Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

// 26.6.14, is permutation
template<class ForwardIterator1, class ForwardIterator2>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, BinaryPredicate pred);
template<class ForwardIterator1, class ForwardIterator2>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, ForwardIterator2 last2,
                                   BinaryPredicate pred);

namespace ranges {
    template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
            sentinel_for<I2> S2, class Proj1 = identity, class Proj2 = identity,
            indirect_equivalence_relation<projected<I1, Proj1>,
                                           projected<I2, Proj2>> Pred = ranges::equal_to>
        constexpr bool is_permutation(I1 first1, S1 last1, I2 first2, S2 last2,
                                       Pred pred = {},
                                       Proj1 proj1 = {}, Proj2 proj2 = {});
    template<forward_range R1, forward_range R2,
            class Proj1 = identity, class Proj2 = identity,
            indirect_equivalence_relation<projected<iterator_t<R1>, Proj1>,
                                           projected<iterator_t<R2>, Proj2>>

```

```

        Pred = ranges::equal_to>
    constexpr bool is_permutation(R1&& r1, R2&& r2, Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
}

// 26.6.15, search
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1
    search(ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2, class BinaryPredicate>
constexpr ForwardIterator1
    search(ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2,
           BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
    search(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
           ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
ForwardIterator1
    search(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
           ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2,
           BinaryPredicate pred);

namespace ranges {
    template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
            sentinel_for<I2> S2, class Pred = ranges::equal_to,
            class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr subrange<I1>
        search(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});
    template<forward_range R1, forward_range R2, class Pred = ranges::equal_to,
            class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
    constexpr borrowed_subrange_t<R1>
        search(R1&& r1, R2&& r2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    subrange<I1>
        search(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
    borrowed_subrange_t<R1>
        search(Ep&& exec, R1&& r1, R2&& r2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

template<class ForwardIterator, class Size,
         class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
             Size count, const T& value);

```



```

template<class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type, class BinaryPredicate>
constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
            Size count, const T& value, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator
    search_n(ExecutionPolicy&& exec,                                // freestanding-deleted, see 26.3.5
            ForwardIterator first, ForwardIterator last,
            Size count, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type, class BinaryPredicate>
ForwardIterator
    search_n(ExecutionPolicy&& exec,                                // freestanding-deleted, see 26.3.5
            ForwardIterator first, ForwardIterator last,
            Size count, const T& value,
            BinaryPredicate pred);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S,
            class Pred = ranges::equal_to, class Proj = identity,
            class T = projected_value_t<I, Proj>>
    requires indirectly_comparable<I, const T*, Pred, Proj>
    constexpr subrange<I>
        search_n(I first, S last, iter_difference_t<I> count,
                const T& value, Pred pred = {}, Proj proj = {});
    template<forward_range R, class Pred = ranges::equal_to,
            class Proj = identity, class T = projected_value_t<I, Proj>>
    requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
    constexpr borrowed_subrange_t<R>
        search_n(R&& r, range_difference_t<R> count,
                const T& value, Pred pred = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Pred = ranges::equal_to, class Proj = identity,
            class T = projected_value_t<I, Proj>>
    requires indirectly_comparable<I, const T*, Pred, Proj>
    subrange<I>
        search_n(Ep&& exec, I first, S last, iter_difference_t<I> count,
                const T& value, Pred pred = {}, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Pred = ranges::equal_to,
            class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
    borrowed_subrange_t<R>
        search_n(Ep&& exec, R&& r, range_difference_t<R> count,
                const T& value, Pred pred = {}, Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator, class Searcher>
constexpr ForwardIterator
    search(ForwardIterator first, ForwardIterator last, const Searcher& searcher);

namespace ranges {
    // 26.6.16, starts with
    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool starts_with(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, class Pred = ranges::equal_to,
            class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
    constexpr bool starts_with(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

```

Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool starts_with(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                 Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted

template<execution_policy Ep, sized_random_access_range R1,
        sized_random_access_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool starts_with(Ep&& exec, R1&& r1, R2&& r2,
                 Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted

// 26.6.17, ends with
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires (forward_iterator<I1> || sized_sentinel_for<S1, I1>) &&
        (forward_iterator<I2> || sized_sentinel_for<S2, I2>) &&
        indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ends_with(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                         Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires (forward_range<R1> || sized_range<R1>) &&
        (forward_range<R2> || sized_range<R2>) &&
        indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ends_with(R1&& r1, R2&& r2, Pred pred = {},
                         Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool ends_with(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted

template<execution_policy Ep, sized_random_access_range R1,
        sized_random_access_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool ends_with(Ep&& exec, R1&& r1, R2&& r2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted

// 26.6.18, fold
template<class F>
class flipped { // exposition only
    F f; // exposition only

public:
    template<class T, class U> requires invocable<F&, U, T>
    invoke_result_t<F&, U, T> operator()(T&&, U&&);
};

template<class F, class T, class I, class U>
concept indirectly_binary_left_foldable_impl = // exposition only
    movable<T> && movable<U> &&
    convertible_to<T, U> && invocable<F&, U, iter_reference_t<I>> &&
    assignable_from<U&, invoke_result_t<F&, U, iter_reference_t<I>>>;

template<class F, class T, class I>
concept indirectly_binary_left_foldable = // exposition only
    copy_constructible<F> && indirectly_readable<I> &&
    invocable<F&, T, iter_reference_t<I>> &&

```

```

    convertible_to<invoke_result_t<F&, T, iter_reference_t<I>>,
        decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>> &&
    indirectly-binary-left-foldable-impl<F, T, I,
        decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>>;

template<class F, class T, class I>
    concept indirectly-binary-right-foldable = // exposition only
        indirectly-binary-left-foldable<flipped<F>, T, I>;

template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
    indirectly-binary-left-foldable<T, I> F>
    constexpr auto fold_left(I first, S last, T init, F f);

template<input_range R, class T = range_value_t<R>,
    indirectly-binary-left-foldable<T, iterator_t<R>> F>
    constexpr auto fold_left(R&& r, T init, F f);

template<input_iterator I, sentinel_for<I> S,
    indirectly-binary-left-foldable<iter_value_t<I>, I> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr auto fold_left_first(I first, S last, F f);

template<input_range R, indirectly-binary-left-foldable<range_value_t<R>, iterator_t<R>> F>
    requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr auto fold_left_first(R&& r, F f);

template<bidirectional_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
    indirectly-binary-right-foldable<T, I> F>
    constexpr auto fold_right(I first, S last, T init, F f);

template<bidirectional_range R, class T = range_value_t<R>,
    indirectly-binary-right-foldable<T, iterator_t<R>> F>
    constexpr auto fold_right(R&& r, T init, F f);

template<bidirectional_iterator I, sentinel_for<I> S,
    indirectly-binary-right-foldable<iter_value_t<I>, I> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr auto fold_right_last(I first, S last, F f);

template<bidirectional_range R,
    indirectly-binary-right-foldable<range_value_t<R>, iterator_t<R>> F>
    requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr auto fold_right_last(R&& r, F f);

template<class I, class T>
    using fold_left_with_iter_result = in_value_result<I, T>;
template<class I, class T>
    using fold_left_first_with_iter_result = in_value_result<I, T>;

template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
    indirectly-binary-left-foldable<T, I> F>
    constexpr see below fold_left_with_iter(I first, S last, T init, F f);

template<input_range R, class T = range_value_t<R>,
    indirectly-binary-left-foldable<T, iterator_t<R>> F>
    constexpr see below fold_left_with_iter(R&& r, T init, F f);

template<input_iterator I, sentinel_for<I> S,
    indirectly-binary-left-foldable<iter_value_t<I>, I> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr see below fold_left_first_with_iter(I first, S last, F f);

```

```

template<input_range R,
        indirectly_binary_left_foldable<range_value_t<R>, iterator_t<R>> F>
    requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr see_below fold_left_first_with_iter(R&& r, F f);
}

// 26.7, mutating sequence operations
// 26.7.1, copy
template<class InputIterator, class OutputIterator>
    constexpr OutputIterator copy(InputIterator first, InputIterator last,
                                  OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator2 copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                          ForwardIterator1 first, ForwardIterator1 last,
                          ForwardIterator2 result);

namespace ranges {
    template<class I, class O>
        using copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremtable O>
        requires indirectly_copyable<I, O>
        constexpr copy_result<I, O>
            copy(I first, S last, O result);
    template<input_range R, weakly_incremtable O>
        requires indirectly_copyable<iterator_t<R>, O>
        constexpr copy_result<borrowed_iterator_t<R>, O>
            copy(R&& r, O result);

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS>
        requires indirectly_copyable<I, O>
        copy_result<I, O>
            copy(Ep&& exec, I first, S last, O result, OutS result_last); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR>
        requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
        copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            copy(Ep&& exec, R&& r, OutR&& result_r); // freestanding-deleted
}

template<class InputIterator, class Size, class OutputIterator>
    constexpr OutputIterator copy_n(InputIterator first, Size n,
                                    OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class Size,
        class ForwardIterator2>
    ForwardIterator2 copy_n(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                            ForwardIterator1 first, Size n,
                            ForwardIterator2 result);

namespace ranges {
    template<class I, class O>
        using copy_n_result = in_out_result<I, O>;

    template<input_iterator I, weakly_incremtable O>
        requires indirectly_copyable<I, O>
        constexpr copy_n_result<I, O>
            copy_n(I first, iter_difference_t<I> n, O result);

    template<execution_policy Ep, random_access_iterator I, random_access_iterator O,
            sized_sentinel_for<O> OutS>
        requires indirectly_copyable<I, O>
        copy_n_result<I, O>
            copy_n(Ep&& exec, I first, iter_difference_t<I> n, O result,
                  OutS result_last); // freestanding-deleted
}

```

```

}

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator copy_if(InputIterator first, InputIterator last,
                                OutputIterator result, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate>
ForwardIterator2 copy_if(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                        ForwardIterator1 first, ForwardIterator1 last,
                        ForwardIterator2 result, Predicate pred);

namespace ranges {
    template<class I, class O>
        using copy_if_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremmentable O, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        requires indirectly_copyable<I, O>
        constexpr copy_if_result<I, O>
            copy_if(I first, S last, O result, Pred pred, Proj proj = {});
    template<input_range R, weakly_incremmentable O, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires indirectly_copyable<iterator_t<R>, O>
        constexpr copy_if_result<borrowed_iterator_t<R>, O>
            copy_if(R&& r, O result, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires indirectly_copyable<I, O>
        copy_if_result<I, O>
            copy_if(Ep&& exec, I first, S last, O result, OutS result_last,
                    Pred pred, Proj proj = {});           // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, sized-random-access-range OutR,
            class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
        copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred,
                    Proj proj = {});                       // freestanding-deleted
}

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2
    copy_backward(BidirectionalIterator1 first, BidirectionalIterator1 last,
                 BidirectionalIterator2 result);

namespace ranges {
    template<class I1, class I2>
        using copy_backward_result = in_out_result<I1, I2>;

    template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator I2>
        requires indirectly_copyable<I1, I2>
        constexpr copy_backward_result<I1, I2>
            copy_backward(I1 first, S1 last, I2 result);
    template<bidirectional_range R, bidirectional_iterator I>
        requires indirectly_copyable<iterator_t<R>, I>
        constexpr copy_backward_result<borrowed_iterator_t<R>, I>
            copy_backward(R&& r, I result);
}

```

```

// 26.7.2, move
template<class InputIterator, class OutputIterator>
    constexpr OutputIterator move(InputIterator first, InputIterator last,
                                   OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1,
         class ForwardIterator2>
    ForwardIterator2 move(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                          ForwardIterator1 first, ForwardIterator1 last,
                          ForwardIterator2 result);

namespace ranges {
    template<class I, class O>
        using move_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremtable O>
        requires indirectly_movable<I, O>
        constexpr move_result<I, O>
            move(I first, S last, O result);
    template<input_range R, weakly_incremtable O>
        requires indirectly_movable<iterator_t<R>, O>
        constexpr move_result<borrowed_iterator_t<R>, O>
            move(R&& r, O result);

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS>
        requires indirectly_movable<I, O>
        move_result<I, O>
            move(Ep&& exec, I first, S last, O result, OutS result_last);           // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
        requires indirectly_movable<iterator_t<R>, iterator_t<OutR>>
        move_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            move(Ep&& exec, R&& r, OutR&& result_r);           // freestanding-deleted
}

template<class BidirectionalIterator1, class BidirectionalIterator2>
    constexpr BidirectionalIterator2
        move_backward(BidirectionalIterator1 first, BidirectionalIterator1 last,
                      BidirectionalIterator2 result);

namespace ranges {
    template<class I1, class I2>
        using move_backward_result = in_out_result<I1, I2>;

    template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator I2>
        requires indirectly_movable<I1, I2>
        constexpr move_backward_result<I1, I2>
            move_backward(I1 first, S1 last, I2 result);
    template<bidirectional_range R, bidirectional_iterator I>
        requires indirectly_movable<iterator_t<R>, I>
        constexpr move_backward_result<borrowed_iterator_t<R>, I>
            move_backward(R&& r, I result);
}

// 26.7.3, swap
template<class ForwardIterator1, class ForwardIterator2>
    constexpr ForwardIterator2 swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
                                           ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    ForwardIterator2 swap_ranges(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                                ForwardIterator1 first1, ForwardIterator1 last1,
                                ForwardIterator2 first2);

```

```

namespace ranges {
    template<class I1, class I2>
        using swap_ranges_result = in_in_result<I1, I2>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2>
        requires indirectly_swappable<I1, I2>
        constexpr swap_ranges_result<I1, I2>
            swap_ranges(I1 first1, S1 last1, I2 first2, S2 last2);
    template<input_range R1, input_range R2>
        requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
        constexpr swap_ranges_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            swap_ranges(R1&& r1, R2&& r2);

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2>
        requires indirectly_swappable<I1, I2>
        swap_ranges_result<I1, I2>
            swap_ranges(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2>
        requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
        swap_ranges_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            swap_ranges(Ep&& exec, R1&& r1, R2&& r2); // freestanding-deleted
}

template<class ForwardIterator1, class ForwardIterator2>
    constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

// 26.7.4, transform
template<class InputIterator, class OutputIterator, class UnaryOperation>
    constexpr OutputIterator
        transform(InputIterator first1, InputIterator last1,
            OutputIterator result, UnaryOperation op);
template<class InputIterator1, class InputIterator2, class OutputIterator,
    class BinaryOperation>
    constexpr OutputIterator
        transform(InputIterator1 first1, InputIterator1 last1,
            InputIterator2 first2, OutputIterator result,
            BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
    class UnaryOperation>
    ForwardIterator2
        transform(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 result, UnaryOperation op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
    class ForwardIterator, class BinaryOperation>
    ForwardIterator
        transform(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2, ForwardIterator result,
            BinaryOperation binary_op);

namespace ranges {
    template<class I, class O>
        using unary_transform_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremtable O,
        copy_constructible F, class Proj = identity>
        requires indirectly_writable<O, indirect_result_t<F&, projected<I, Proj>>>
        constexpr unary_transform_result<I, O>
            transform(I first1, S last1, O result, F op, Proj proj = {});
    template<input_range R, weakly_incremtable O, copy_constructible F,
        class Proj = identity>
        requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R>, Proj>>>

```

```

constexpr unary_transform_result<borrowed_iterator_t<R>, 0>
    transform(R&& r, 0 result, F op, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        copy_constructible F, class Proj = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I, Proj>>>
unary_transform_result<I, 0>
    transform(Ep&& exec, I first1, S last1, O result, OutS result_last,
        F op, Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        copy_constructible F, class Proj = identity>
requires indirectly_writable<iterator_t<OutR>,
        indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
unary_transform_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    transform(Ep&& exec, R&& r, OutR&& result_r, F op, Proj proj = {}); // freestanding-deleted

template<class I1, class I2, class O>
using binary_transform_result = in_in_out_result<I1, I2, O>;

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, copy_constructible F, class Proj1 = identity,
        class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
constexpr binary_transform_result<I1, I2, O>
    transform(I1 first1, S1 last1, I2 first2, S2 last2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
constexpr binary_transform_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
    transform(R1&& r1, R2&& r2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
binary_transform_result<I1, I2, O>
    transform(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
        O result, OutS result_last,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, copy_constructible F,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<iterator_t<OutR>,
        indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
binary_transform_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
        borrowed_iterator_t<OutR>>
    transform(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

// 26.7.5, replace
template<class ForwardIterator, class T>
constexpr void replace(ForwardIterator first, ForwardIterator last,
        const T& old_value, const T& new_value);

```



```

template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
void replace(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator first, ForwardIterator last,
            const T& old_value, const T& new_value);
template<class ForwardIterator, class Predicate,
        class T = iterator_traits<ForwardIterator>::value_type>
constexpr void replace_if(ForwardIterator first, ForwardIterator last,
                        Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class Predicate,
        class T = iterator_traits<ForwardIterator>::value_type>
void replace_if(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
               ForwardIterator first, ForwardIterator last,
               Predicate pred, const T& new_value);

namespace ranges {
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<I>>
requires indirectly_writable<I, const T2&> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
constexpr I
    replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
template<input_range R, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = range_value_t<R>>
requires indirectly_writable<iterator_t<R>, const T2&> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T1*>
constexpr borrowed_iterator_t<R>
    replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<I>>
requires indirectly_writable<I, const T2&> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
I replace(Ep&& exec, I first, S last,
        const T1& old_value, const T2& new_value, Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = range_value_t<R>>
requires indirectly_writable<iterator_t<R>, const T2&> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T1*>
borrowed_iterator_t<R>
    replace(Ep&& exec, R&& r, const T1& old_value, const T2& new_value,
            Proj proj = {}); // freestanding-deleted

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = iter_value_t<I>,
        indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_writable<I, const T&>
constexpr I replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});
template<input_range R, class Proj = identity, class T = range_value_t<R>,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_writable<iterator_t<R>, const T&>
constexpr borrowed_iterator_t<R>
    replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = iter_value_t<I>,
        indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_writable<I, const T&>
I replace_if(Ep&& exec, I first, S last, Pred pred,
            const T& new_value, Proj proj = {}); // freestanding-deleted

```

```

template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        class T = range_value_t<R>,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_writable<iterator_t<R>, const T&>
borrowed_iterator_t<R>
    replace_if(Ep&& exec, R&& r, Pred pred, const T& new_value,
               Proj proj = {}); // freestanding-deleted
}

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator replace_copy(InputIterator first, InputIterator last,
                                     OutputIterator result,
                                     const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2 replace_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result,
                             const T& old_value, const T& new_value);
template<class InputIterator, class OutputIterator, class Predicate,
        class T = iterator_traits<OutputIterator>::value_type>
constexpr OutputIterator replace_copy_if(InputIterator first, InputIterator last,
                                       OutputIterator result,
                                       Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate, class T = iterator_traits<ForwardIterator2>::value_type>
ForwardIterator2 replace_copy_if(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                                ForwardIterator1 first, ForwardIterator1 last,
                                ForwardIterator2 result,
                                Predicate pred, const T& new_value);

namespace ranges {
    template<class I, class O>
        using replace_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, class O,
            class Proj = identity,
            class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<O>>
    requires indirectly_copyable<I, O> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
        output_iterator<O, const T2&>
    constexpr replace_copy_result<I, O>
        replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
                     Proj proj = {});

    template<input_range R, class O, class Proj = identity,
            class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = iter_value_t<O>>
    requires indirectly_copyable<iterator_t<R>, O> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T1*> &&
        output_iterator<O, const T2&>
    constexpr replace_copy_result<borrowed_iterator_t<R>, O>
        replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                     Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS,
            class Proj = identity,
            class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<O>>
    requires indirectly_copyable<I, O> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
        indirectly_writable<O, const T2&>
    replace_copy_result<I, O>
        replace_copy(Ep&& exec, I first, S last, O result, OutS result_last, const T1& old_value,
                     const T2& new_value, Proj proj = {}); // freestanding-deleted
}

```

```

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class Proj = identity, class T1 = projected_value_t<iterator_t<R>, Proj>,
        class T2 = range_value_t<OutR>>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T1*> &&
        indirectly_writable<iterator_t<OutR>, const T2>
replace_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
replace_copy(Ep&& exec, R&& r, OutR&& result_r, const T1& old_value, const T2& new_value,
            Proj proj = {}); // freestanding-deleted

template<class I, class O>
using replace_copy_if_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class O, class T = iter_value_t<O>,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O> && output_iterator<O, const T>
constexpr replace_copy_if_result<I, O>
replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                Proj proj = {});

template<input_range R, class O, class T = iter_value_t<O>, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O> && output_iterator<O, const T>
constexpr replace_copy_if_result<borrowed_iterator_t<R>, O>
replace_copy_if(R&& r, O result, Pred pred, const T& new_value,
                Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS, class T = iter_value_t<O>,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O> && indirectly_writable<O, const T>
replace_copy_if_result<I, O>
replace_copy_if(Ep&& exec, I first, S last, O result, OutS result_last,
                Pred pred, const T& new_value, Proj proj = {}); // freestanding-deleted

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class T = range_value_t<OutR>, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
        indirectly_writable<iterator_t<OutR>, const T>
replace_copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
replace_copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred, const T& new_value,
                Proj proj = {}); // freestanding-deleted
}

// 26.7.6, fill
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
void fill(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, const T& value);
template<class OutputIterator, class Size,
        class T = iterator_traits<OutputIterator>::value_type>
constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class Size, class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator fill_n(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, Size n, const T& value);

namespace ranges {
template<class O, sentinel_for<O> S, class T = iter_value_t<O>>
requires output_iterator<O, const T>
constexpr O fill(O first, S last, const T& value);

```

```

template<class R, class T = range_value_t<R>>
    requires output_range<R, const T&>
    constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
template<class O, class T = iter_value_t<O>>
    requires output_iterator<O, const T&>
    constexpr O fill_n(O first, iter_difference_t<O> n, const T& value);

template<execution_policy Ep, random_access_iterator O, sized_sentinel_for<O> S,
        class T = iter_value_t<O>>
    requires indirectly_writable<O, const T&>
    O fill(Ep&& exec, O first, S last, const T& value); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class T = range_value_t<R>>
    requires indirectly_writable<iterator_t<R>, const T&>
    borrowed_iterator_t<R> fill(Ep&& exec, R&& r, const T& value); // freestanding-deleted
template<execution_policy Ep, random_access_iterator O, class T = iter_value_t<O>>
    requires indirectly_writable<O, const T&>
    O fill_n(Ep&& exec, O first, iter_difference_t<O> n, const T& value); // freestanding-deleted
}

// 26.7.7, generate
template<class ForwardIterator, class Generator>
    constexpr void generate(ForwardIterator first, ForwardIterator last,
                           Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Generator>
    void generate(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                 ForwardIterator first, ForwardIterator last,
                 Generator gen);
template<class OutputIterator, class Size, class Generator>
    constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Size, class Generator>
    ForwardIterator generate_n(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                              ForwardIterator first, Size n, Generator gen);

namespace ranges {
    template<input_or_output_iterator O, sentinel_for<O> S, copy_constructible F>
        requires invocable<F&& && indirectly_writable<O, invoke_result_t<F&>>
        constexpr O generate(O first, S last, F gen);
    template<class R, copy_constructible F>
        requires invocable<F&& && output_range<R, invoke_result_t<F&>>
        constexpr borrowed_iterator_t<R> generate(R&& r, F gen);
    template<input_or_output_iterator O, copy_constructible F>
        requires invocable<F&& && indirectly_writable<O, invoke_result_t<F&>>
        constexpr O generate_n(O first, iter_difference_t<O> n, F gen);
}

// 26.7.8, remove
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
    constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                                     const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
    ForwardIterator remove(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                          ForwardIterator first, ForwardIterator last,
                          const T& value);
template<class ForwardIterator, class Predicate>
    constexpr ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                                       Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator remove_if(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                              ForwardIterator first, ForwardIterator last,
                              Predicate pred);

```

```

namespace ranges {
    template<permutable I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>>
        requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        constexpr subrange<I> remove(I first, S last, const T& value, Proj proj = {});
    template<forward_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires permutable<iterator_t<R>> &&
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T*>
        constexpr borrowed_subrange_t<R>
            remove(R&& r, const T& value, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, class T = projected_value_t<I, Proj>>
        requires permutable<I> &&
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        subrange<I> remove(Ep&& exec, I first, S last, const T& value,
                           Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires permutable<iterator_t<R>> &&
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T*>
        borrowed_subrange_t<R>
            remove(Ep&& exec, R&& r, const T& value, Proj proj = {}); // freestanding-deleted

    template<permutable I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        constexpr subrange<I> remove_if(I first, S last, Pred pred, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires permutable<iterator_t<R>>
        constexpr borrowed_subrange_t<R>
            remove_if(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires permutable<I>
        subrange<I>
            remove_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires permutable<iterator_t<R>>
        borrowed_subrange_t<R>
            remove_if(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

template<class InputIterator, class OutputIterator,
        class T = iterator_traits<InputIterator>::value_type>
constexpr OutputIterator
    remove_copy(InputIterator first, InputIterator last,
                OutputIterator result, const T& value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class T = iterator_traits<ForwardIterator1>::value_type>
ForwardIterator2
    remove_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, const T& value);
template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator
    remove_copy_if(InputIterator first, InputIterator last,
                   OutputIterator result, Predicate pred);

```

```

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate>
ForwardIterator2
remove_copy_if(ExecutionPolicy&& exec,                                // freestanding-deleted, see 26.3.5
               ForwardIterator1 first, ForwardIterator1 last,
               ForwardIterator2 result, Predicate pred);

namespace ranges {
    template<class I, class O>
        using remove_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremtable O,
            class Proj = identity, class T = projected_value_t<I, Proj>>
        requires indirectly_copyable<I, O> &&
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        constexpr remove_copy_result<I, O>
            remove_copy(I first, S last, O result, const T& value, Proj proj = {});
    template<input_range R, weakly_incremtable O, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>>
        requires indirectly_copyable<iterator_t<R>, O> &&
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T*>
        constexpr remove_copy_result<borrowed_iterator_t<R>, O>
            remove_copy(R&& r, O result, const T& value, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS,
            class Proj = identity, class T = projected_value_t<I, Proj>>
        requires indirectly_copyable<I, O> &&
            indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
        remove_copy_result<I, O>
            remove_copy(Ep&& exec, I first, S last, O result, OutS result_last, const T& value,
                        Proj proj = {});                                // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
            class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
        requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
            indirect_binary_predicate<ranges::equal_to,
                                    projected<iterator_t<R>, Proj>, const T*>
        remove_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            remove_copy(Ep&& exec, R&& r, OutR&& result_r, const T& value,
                        Proj proj = {});                                // freestanding-deleted

    template<class I, class O>
        using remove_copy_if_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremtable O,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires indirectly_copyable<I, O>
        constexpr remove_copy_if_result<I, O>
            remove_copy_if(I first, S last, O result, Pred pred, Proj proj = {});
    template<input_range R, weakly_incremtable O, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires indirectly_copyable<iterator_t<R>, O>
        constexpr remove_copy_if_result<borrowed_iterator_t<R>, O>
            remove_copy_if(R&& r, O result, Pred pred, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires indirectly_copyable<I, O>
        remove_copy_if_result<I, O>
            remove_copy_if(Ep&& exec, I first, S last, O result, OutS result_last, Pred pred,
                        Proj proj = {});                                // freestanding-deleted

```



```

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
remove_copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
remove_copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred,
               Proj proj = {}); // freestanding-deleted
}

// 26.7.9, unique
template<class ForwardIterator>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                                BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator unique(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
ForwardIterator unique(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last,
                      BinaryPredicate pred);

namespace ranges {
template<permutable I, sentinel_for<I> S, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
constexpr subrange<I> unique(I first, S last, C comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
unique(R&& r, C comp = {}, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires permutable<I>
subrange<I> unique(Ep&& exec, I first, S last, C comp = {},
                  Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R>
unique(Ep&& exec, R&& r, C comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class InputIterator, class OutputIterator>
constexpr OutputIterator
unique_copy(InputIterator first, InputIterator last,
            OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryPredicate>
constexpr OutputIterator
unique_copy(InputIterator first, InputIterator last,
            OutputIterator result, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
unique_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first, ForwardIterator1 last,
            ForwardIterator2 result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
ForwardIterator2
unique_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator1 first, ForwardIterator1 last,

```

```

        ForwardIterator2 result, BinaryPredicate pred);

namespace ranges {
    template<class I, class O>
        using unique_copy_result = in_out_result<I, O>;

    template<input_iterator I, sentinel_for<I> S, weakly_incremmentable O, class Proj = identity,
            indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
        requires indirectly_copyable<I, O> &&
            (forward_iterator<I> ||
             (input_iterator<O> && same_as<iter_value_t<I>, iter_value_t<O>>) ||
             indirectly_copyable_storable<I, O>)
        constexpr unique_copy_result<I, O>
            unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});
    template<input_range R, weakly_incremmentable O, class Proj = identity,
            indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
        requires indirectly_copyable<iterator_t<R>, O> &&
            (forward_iterator<iterator_t<R>> ||
             (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>) ||
             indirectly_copyable_storable<iterator_t<R>, O>)
        constexpr unique_copy_result<borrowed_iterator_t<R>, O>
            unique_copy(R&& r, O result, C comp = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Proj = identity,
            indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
        requires indirectly_copyable<I, O>
        unique_copy_result<I, O>
            unique_copy(Ep&& exec, I first, S last, O result, OutS result_last, C comp = {},
                        Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, sized-random-access-range OutR,
            class Proj = identity,
            indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
        requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
        unique_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            unique_copy(Ep&& exec, R&& r, OutR&& result_r, C comp = {},
                        Proj proj = {}); // freestanding-deleted
}

// 26.7.10, reverse
template<class BidirectionalIterator>
    constexpr void reverse(BidirectionalIterator first, BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator>
    void reverse(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                BidirectionalIterator first, BidirectionalIterator last);

namespace ranges {
    template<bidirectional_iterator I, sentinel_for<I> S>
        requires permutable<I>
        constexpr I reverse(I first, S last);
    template<bidirectional_range R>
        requires permutable<iterator_t<R>>
        constexpr borrowed_iterator_t<R> reverse(R&& r);

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
        requires permutable<I>
        I reverse(Ep&& exec, I first, S last); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R>
        requires permutable<iterator_t<R>>
        borrowed_iterator_t<R> reverse(Ep&& exec, R&& r); // freestanding-deleted
}

```



```

template<class BidirectionalIterator, class OutputIterator>
constexpr OutputIterator
reverse_copy(BidirectionalIterator first, BidirectionalIterator last,
             OutputIterator result);
template<class ExecutionPolicy, class BidirectionalIterator, class ForwardIterator>
ForwardIterator
reverse_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
             BidirectionalIterator first, BidirectionalIterator last,
             ForwardIterator result);

namespace ranges {
template<class I, class O>
using reverse_copy_result = in_out_result<I, O>;
template<class I, class O>
using reverse_copy_truncated_result = in_in_out_result<I, I, O>;

template<bidirectional_iterator I, sentinel_for<I> S, weakly_increamentable O>
requires indirectly_copyable<I, O>
constexpr reverse_copy_result<I, O>
reverse_copy(I first, S last, O result);
template<bidirectional_range R, weakly_increamentable O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr reverse_copy_result<borrowed_iterator_t<R>, O>
reverse_copy(R&& r, O result);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
reverse_copy_truncated_result<I, O>
reverse_copy(Ep&& exec, I first, S last, O result,
             OutS result_last); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
reverse_copy(Ep&& exec, R&& r, OutR&& result_r); // freestanding-deleted
}

// 26.7.11, rotate
template<class ForwardIterator>
constexpr ForwardIterator rotate(ForwardIterator first,
                                ForwardIterator middle,
                                ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator rotate(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                     ForwardIterator first,
                     ForwardIterator middle,
                     ForwardIterator last);

namespace ranges {
template<permutable I, sentinel_for<I> S>
constexpr subrange<I> rotate(I first, I middle, S last);
template<forward_range R>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R> rotate(R&& r, iterator_t<R> middle);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
requires permutable<I>
subrange<I>
rotate(Ep&& exec, I first, I middle, S last); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R>
rotate(Ep&& exec, R&& r, iterator_t<R> middle); // freestanding-deleted
}

```

```

template<class ForwardIterator, class OutputIterator>
constexpr OutputIterator
    rotate_copy(ForwardIterator first, ForwardIterator middle,
                ForwardIterator last, OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    rotate_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                ForwardIterator1 first, ForwardIterator1 middle,
                ForwardIterator1 last, ForwardIterator2 result);

namespace ranges {
    template<class I, class O>
        using rotate_copy_result = in_out_result<I, O>;
    template<class I, class O>
        using rotate_copy_truncated_result = in_in_out_result<I, I, O>;

    template<forward_iterator I, sentinel_for<I> S, weakly_incrementsable O>
        requires indirectly_copyable<I, O>
        constexpr rotate_copy_result<I, O>
            rotate_copy(I first, I middle, S last, O result);
    template<forward_range R, weakly_incrementsable O>
        requires indirectly_copyable<iterator_t<R>, O>
        constexpr rotate_copy_result<borrowed_iterator_t<R>, O>
            rotate_copy(R&& r, iterator_t<R> middle, O result);

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            random_access_iterator O, sized_sentinel_for<O> OutS>
        requires indirectly_copyable<I, O>
        rotate_copy_truncated_result<I, O>
            rotate_copy(Ep&& exec, I first, I middle, S last, O result, // freestanding-deleted
                        OutS result_last);
    template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR>
        requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
        rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
            rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, // freestanding-deleted
                        OutR&& result_r);
}

// 26.7.12, sample
template<class PopulationIterator, class SampleIterator,
        class Distance, class UniformRandomBitGenerator>
SampleIterator sample(PopulationIterator first, PopulationIterator last,
                    SampleIterator out, Distance n,
                    UniformRandomBitGenerator&& g);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S,
            weakly_incrementsable O, class Gen>
        requires (forward_iterator<I> || random_access_iterator<O>) &&
            indirectly_copyable<I, O> &&
            uniform_random_bit_generator<remove_reference_t<Gen>>
        O sample(I first, S last, O out, iter_difference_t<I> n, Gen&& g);
    template<input_range R, weakly_incrementsable O, class Gen>
        requires (forward_range<R> || random_access_iterator<O>) &&
            indirectly_copyable<iterator_t<R>, O> &&
            uniform_random_bit_generator<remove_reference_t<Gen>>
        O sample(R&& r, O out, range_difference_t<R> n, Gen&& g);
}

// 26.7.13, shuffle
template<class RandomAccessIterator, class UniformRandomBitGenerator>
void shuffle(RandomAccessIterator first,
            RandomAccessIterator last,
            UniformRandomBitGenerator&& g);

```

```

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Gen>
        requires permutable<I> &&
            uniform_random_bit_generator<remove_reference_t<Gen>>
        I shuffle(I first, S last, Gen&& g);
    template<random_access_range R, class Gen>
        requires permutable<iterator_t<R>> &&
            uniform_random_bit_generator<remove_reference_t<Gen>>
        borrowed_iterator_t<R> shuffle(R&& r, Gen&& g);
}

// 26.7.14, shift
template<class ForwardIterator>
    constexpr ForwardIterator
        shift_left(ForwardIterator first, ForwardIterator last,
                    typename iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator
        shift_left(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                    ForwardIterator first, ForwardIterator last,
                    typename iterator_traits<ForwardIterator>::difference_type n);

namespace ranges {
    template<permutable I, sentinel_for<I> S>
        constexpr subrange<I> shift_left(I first, S last, iter_difference_t<I> n);
    template<forward_range R>
        requires permutable<iterator_t<R>>
        constexpr borrowed_subrange_t<R> shift_left(R&& r, range_difference_t<R> n);

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
        requires permutable<I>
        subrange<I>
            shift_left(Ep&& exec, I first, S last, iter_difference_t<I> n); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R>
        requires permutable<iterator_t<R>>
        borrowed_subrange_t<R>
            shift_left(Ep&& exec, R&& r, range_difference_t<R> n); // freestanding-deleted
}

template<class ForwardIterator>
    constexpr ForwardIterator
        shift_right(ForwardIterator first, ForwardIterator last,
                    typename iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator
        shift_right(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                    ForwardIterator first, ForwardIterator last,
                    typename iterator_traits<ForwardIterator>::difference_type n);

namespace ranges {
    template<permutable I, sentinel_for<I> S>
        constexpr subrange<I> shift_right(I first, S last, iter_difference_t<I> n);
    template<forward_range R>
        requires permutable<iterator_t<R>>
        constexpr borrowed_subrange_t<R> shift_right(R&& r, range_difference_t<R> n);

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
        requires permutable<I>
        subrange<I>
            shift_right(Ep&& exec, I first, S last, iter_difference_t<I> n); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R>
        requires permutable<iterator_t<R>>
        borrowed_subrange_t<R>
            shift_right(Ep&& exec, R&& r, range_difference_t<R> n); // freestanding-deleted
}

```

```

}

// 26.8, sorting and related operations
// 26.8.2, sorting
template<class RandomAccessIterator>
    constexpr void sort(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void sort(RandomAccessIterator first, RandomAccessIterator last,
                        Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    void sort(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void sort(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              RandomAccessIterator first, RandomAccessIterator last,
              Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            sort(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            sort(R&& r, Comp comp = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Comp = ranges::less, class Proj = identity>
        requires sortable<I, Comp, Proj>
        I sort(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        borrowed_iterator_t<R>
            sort(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class RandomAccessIterator>
    constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last); // hosted
template<class RandomAccessIterator, class Compare>
    constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last, // hosted
                              Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    void stable_sort(ExecutionPolicy&& exec, // hosted, see 26.3.5
                    RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void stable_sort(ExecutionPolicy&& exec, // hosted, see 26.3.5
                    RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I stable_sort(I first, S last, Comp comp = {}, Proj proj = {}); // hosted
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            stable_sort(R&& r, Comp comp = {}, Proj proj = {}); // hosted
}

```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I stable_sort(Ep&& exec, I first, S last, Comp comp = {},
              Proj proj = {}); // hosted
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
    stable_sort(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // hosted
}

template<class RandomAccessIterator>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle,
                           RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
constexpr void partial_sort(RandomAccessIterator first, RandomAccessIterator middle,
                           RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                 RandomAccessIterator first, RandomAccessIterator middle,
                 RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                 RandomAccessIterator first, RandomAccessIterator middle,
                 RandomAccessIterator last, Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
    requires sortable<I, Comp, Proj>
    constexpr I
        partial_sort(I first, I middle, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    constexpr borrowed_iterator_t<R>
        partial_sort(R&& r, iterator_t<R> middle, Comp comp = {},
                    Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Comp = ranges::less, class Proj = identity>
    requires sortable<I, Comp, Proj>
    I partial_sort(Ep&& exec, I first, I middle, S last, Comp comp = {},
                  Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R,
            class Comp = ranges::less, class Proj = identity>
    requires sortable<iterator_t<R>, Comp, Proj>
    borrowed_iterator_t<R>
        partial_sort(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
                    Proj proj = {}); // freestanding-deleted
}

template<class InputIterator, class RandomAccessIterator>
constexpr RandomAccessIterator
    partial_sort_copy(InputIterator first, InputIterator last,
                    RandomAccessIterator result_first,
                    RandomAccessIterator result_last);
template<class InputIterator, class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator
    partial_sort_copy(InputIterator first, InputIterator last,
                    RandomAccessIterator result_first,
                    RandomAccessIterator result_last,
                    Compare comp);

```

```

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator>
RandomAccessIterator
    partial_sort_copy(ExecutionPolicy&& exec,                // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last,
                      RandomAccessIterator result_first,
                      RandomAccessIterator result_last);
template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator,
        class Compare>
RandomAccessIterator
    partial_sort_copy(ExecutionPolicy&& exec,                // freestanding-deleted, see 26.3.5
                      ForwardIterator first, ForwardIterator last,
                      RandomAccessIterator result_first,
                      RandomAccessIterator result_last,
                      Compare comp);

namespace ranges {
    template<class I, class O>
        using partial_sort_copy_result = in_out_result<I, O>;

    template<input_iterator I1, sentinel_for<I1> S1,
            random_access_iterator I2, sentinel_for<I2> S2,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
        constexpr partial_sort_copy_result<I1, I2>
            partial_sort_copy(I1 first, S1 last, I2 result_first, S2 result_last,
                             Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, random_access_range R2, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires indirectly_copyable<iterator_t<R1>, iterator_t<R2>> &&
            sortable<iterator_t<R2>, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<iterator_t<R1>, Proj1>,
                                     projected<iterator_t<R2>, Proj2>>
        constexpr partial_sort_copy_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            partial_sort_copy(R1&& r, R2&& result_r, Comp comp = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
        partial_sort_copy_result<I1, I2>
            partial_sort_copy(Ep&& exec, I1 first, S1 last, I2 result_first, S2 result_last,
                             Comp comp = {}, Proj1 proj1 = {},
                             Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires indirectly_copyable<iterator_t<R1>, iterator_t<R2>> &&
            sortable<iterator_t<R2>, Comp, Proj2> &&
            indirect_strict_weak_order<Comp, projected<iterator_t<R1>, Proj1>,
                                     projected<iterator_t<R2>, Proj2>>
        partial_sort_copy_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
            partial_sort_copy(Ep&& exec, R1&& r, R2&& result_r, Comp comp = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

template<class ForwardIterator>
    constexpr bool is_sorted(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr bool is_sorted(ForwardIterator first, ForwardIterator last,
                             Compare comp);

```

```

template<class ExecutionPolicy, class ForwardIterator>
    bool is_sorted(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                   ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    bool is_sorted(ExecutionPolicy&& exec,                      // freestanding-deleted, see 26.3.5
                   ForwardIterator first, ForwardIterator last,
                   Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
             indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        constexpr bool is_sorted(I first, S last, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
             indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        constexpr bool is_sorted(R&& r, Comp comp = {}, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
             class Proj = identity,
             indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        bool is_sorted(Ep&& exec, I first, S last, Comp comp = {},
                       Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
             indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        bool is_sorted(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator>
    constexpr ForwardIterator
        is_sorted_until(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr ForwardIterator
        is_sorted_until(ForwardIterator first, ForwardIterator last,
                       Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator
        is_sorted_until(ExecutionPolicy&& exec,                // freestanding-deleted, see 26.3.5
                       ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator
        is_sorted_until(ExecutionPolicy&& exec,                // freestanding-deleted, see 26.3.5
                       ForwardIterator first, ForwardIterator last,
                       Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
             indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        constexpr I is_sorted_until(I first, S last, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
             indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        constexpr borrowed_iterator_t<R>
            is_sorted_until(R&& r, Comp comp = {}, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
             class Proj = identity,
             indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        I is_sorted_until(Ep&& exec, I first, S last, Comp comp = {},
                          Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
             indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        borrowed_iterator_t<R>
            is_sorted_until(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

```



```

// 26.8.3, Nth element
template<class RandomAccessIterator>
    constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                               RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                               RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    void nth_element(ExecutionPolicy&& exec,                               // freestanding-deleted, see 26.3.5
                     RandomAccessIterator first, RandomAccessIterator nth,
                     RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    void nth_element(ExecutionPolicy&& exec,                               // freestanding-deleted, see 26.3.5
                     RandomAccessIterator first, RandomAccessIterator nth,
                     RandomAccessIterator last, Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
             class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            nth_element(I first, I nth, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            nth_element(R&& r, iterator_t<R> nth, Comp comp = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
             class Comp = ranges::less, class Proj = identity>
        requires sortable<I, Comp, Proj>
        I nth_element(Ep&& exec, I first, I nth, S last, Comp comp = {},
                      Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, class Comp = ranges::less,
             class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        borrowed_iterator_t<R>
            nth_element(Ep&& exec, R&& r, iterator_t<R> nth, Comp comp = {},
                      Proj proj = {}); // freestanding-deleted
}

// 26.8.4, binary search
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
    constexpr ForwardIterator
        lower_bound(ForwardIterator first, ForwardIterator last,
                    const T& value);
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
         class Compare>
    constexpr ForwardIterator
        lower_bound(ForwardIterator first, ForwardIterator last,
                    const T& value, Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
             class T = projected_value_t<I, Proj>,
             indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
        constexpr I lower_bound(I first, S last, const T& value, Comp comp = {},
                                Proj proj = {});
    template<forward_range R, class Proj = identity,
             class T = projected_value_t<iterator_t<R>, Proj>,
             indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
                ranges::less>
        constexpr borrowed_iterator_t<R>
            lower_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}

```



```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>
            indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr I upper_bound(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>,
            indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
                ranges::less>
    constexpr borrowed_iterator_t<R>
        upper_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>,
            indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr subrange<I>
        equal_range(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            class T = projected_value_t<iterator_t<R>, Proj>,
            indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
                ranges::less>
    constexpr borrowed_subrange_t<R>
        equal_range(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value);
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value, Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            class T = projected_value_t<I, Proj>,
            indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr bool binary_search(I first, S last, const T& value, Comp comp = {},
                                Proj proj = {});
}

```

```

template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>,
        indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
constexpr bool binary_search(R&& r, const T& value, Comp comp = {},
                             Proj proj = {});
}

// 26.8.5, partitions
template<class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
bool is_partitioned(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                   ForwardIterator first, ForwardIterator last, Predicate pred);

namespace ranges {
    template<input_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr bool is_partitioned(I first, S last, Pred pred, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr bool is_partitioned(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    bool is_partitioned(Ep&& exec, I first, S last, Pred pred,
                       Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    bool is_partitioned(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator partition(ForwardIterator first,
                                   ForwardIterator last,
                                   Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator partition(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                         ForwardIterator first,
                         ForwardIterator last,
                         Predicate pred);

namespace ranges {
    template<permutable I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr subrange<I>
        partition(I first, S last, Pred pred, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    constexpr borrowed_subrange_t<R>
        partition(R&& r, Pred pred, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
        partition(Ep&& exec, I first, S last, Pred pred, Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
        partition(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // freestanding-deleted
}

```

```

template<class BidirectionalIterator, class Predicate>
    constexpr BidirectionalIterator stable_partition(BidirectionalIterator first, // hosted
                                                    BidirectionalIterator last,
                                                    Predicate pred);
template<class ExecutionPolicy, class BidirectionalIterator, class Predicate>
    BidirectionalIterator stable_partition(ExecutionPolicy&& exec, // hosted,
                                                    BidirectionalIterator first, // see 26.3.5
                                                    BidirectionalIterator last,
                                                    Predicate pred);

namespace ranges {
    template<bidirectional_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
        requires permutable<I>
        constexpr subrange<I> stable_partition(I first, S last, Pred pred, // hosted
                                                Proj proj = {});
    template<bidirectional_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires permutable<iterator_t<R>>
        constexpr borrowed_subrange_t<R> stable_partition(R&& r, Pred pred, // hosted
                                                            Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires permutable<I>
        subrange<I>
            stable_partition(Ep&& exec, I first, S last, Pred pred,
                            Proj proj = {}); // hosted
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires permutable<iterator_t<R>>
        borrowed_subrange_t<R>
            stable_partition(Ep&& exec, R&& r, Pred pred, Proj proj = {}); // hosted
}

template<class InputIterator, class OutputIterator1,
        class OutputIterator2, class Predicate>
    constexpr pair<OutputIterator1, OutputIterator2>
        partition_copy(InputIterator first, InputIterator last,
                        OutputIterator1 out_true, OutputIterator2 out_false,
                        Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class ForwardIterator1,
        class ForwardIterator2, class Predicate>
    pair<ForwardIterator1, ForwardIterator2>
        partition_copy(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                        ForwardIterator first, ForwardIterator last,
                        ForwardIterator1 out_true, ForwardIterator2 out_false,
                        Predicate pred);

namespace ranges {
    template<class I, class O1, class O2>
        using partition_copy_result = in_out_out_result<I, O1, O2>;

    template<input_iterator I, sentinel_for<I> S,
            weakly_incrementable O1, weakly_incrementable O2,
            class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
        requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
        constexpr partition_copy_result<I, O1, O2>
            partition_copy(I first, S last, O1 out_true, O2 out_false, Pred pred,
                            Proj proj = {});
    template<input_range R, weakly_incrementable O1, weakly_incrementable O2,
            class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
        requires indirectly_copyable<iterator_t<R>, O1> &&

```

```

        indirectly_copyable<iterator_t<R>, 02>
constexpr partition_copy_result<borrowed_iterator_t<R>, 01, 02>
    partition_copy(R&& r, 01 out_true, 02 out_false, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator 01, sized_sentinel_for<01> OutS1,
        random_access_iterator 02, sized_sentinel_for<02> OutS2,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, 01> && indirectly_copyable<I, 02>
partition_copy_result<I, 01, 02>
    partition_copy(Ep&& exec, I first, S last, 01 out_true, OutS1 last_true,
        02 out_false, OutS2 last_false, Pred pred,
        Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R,
        sized-random-access-range OutR1, sized-random-access-range OutR2,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR1>> &&
        indirectly_copyable<iterator_t<R>, iterator_t<OutR2>>
partition_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR1>,
        borrowed_iterator_t<OutR2>>
    partition_copy(Ep&& exec, R&& r, OutR1&& out_true_r, OutR2&& out_false_r, Pred pred,
        Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator
    partition_point(ForwardIterator first, ForwardIterator last,
        Predicate pred);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr I partition_point(I first, S last, Pred pred, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_iterator_t<R>
        partition_point(R&& r, Pred pred, Proj proj = {});
}

// 26.8.6, merge
template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    merge(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2,
        OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
        class Compare>
constexpr OutputIterator
    merge(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2,
        OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    merge(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2,
        ForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    merge(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,

```

```

        ForwardIterator2 first2, ForwardIterator2 last2,
        ForwardIterator result, Compare comp);

namespace ranges {
    template<class I1, class I2, class O>
        using merge_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            weakly_incrementable O, class Comp = ranges::less, class Proj1 = identity,
            class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr merge_result<I1, I2, O>
            merge(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incrementable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
            merge(R1&& r1, R2&& r2, O result,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        merge_result<I1, I2, O>
            merge(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, O result, OutS result_last,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            sized-random-access-range OutR, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, borrowed_iterator_t<OutR>>
            merge(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
                Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

template<class BidirectionalIterator>
    constexpr void inplace_merge(BidirectionalIterator first, // hosted
                                BidirectionalIterator middle,
                                BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
    constexpr void inplace_merge(BidirectionalIterator first, // hosted
                                BidirectionalIterator middle,
                                BidirectionalIterator last, Compare comp);
template<class ExecutionPolicy, class BidirectionalIterator>
    void inplace_merge(ExecutionPolicy&& exec, // hosted, see 26.3.5
                       BidirectionalIterator first,
                       BidirectionalIterator middle,
                       BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator, class Compare>
    void inplace_merge(ExecutionPolicy&& exec, // hosted, see 26.3.5
                       BidirectionalIterator first,
                       BidirectionalIterator middle,
                       BidirectionalIterator last, Compare comp);

namespace ranges {
    template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            inplace_merge(I first, I middle, S last, Comp comp = {}, Proj proj = {}); // hosted
}

```

```

template<bidirectional_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    inplace_merge(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj = {});    // hosted

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I inplace_merge(Ep&& exec, I first, I middle, S last, Comp comp = {},
                Proj proj = {});    // hosted
template<execution_policy Ep, sized-random-access-range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
    inplace_merge(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
                Proj proj = {});    // hosted
}

// 26.8.7, set operations
template<class InputIterator1, class InputIterator2>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool includes(ExecutionPolicy&& exec,    // freestanding-deleted, see 26.3.5
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Compare>
bool includes(ExecutionPolicy&& exec,    // freestanding-deleted, see 26.3.5
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              Compare comp);

namespace ranges {
    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            class Proj1 = identity, class Proj2 = identity,
            indirect_strict_weak_order<projected<I1, Proj1>, projected<I2, Proj2>> Comp =
                ranges::less>
        constexpr bool includes(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, class Proj1 = identity,
            class Proj2 = identity,
            indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                    projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
        constexpr bool includes(R1&& r1, R2&& r2, Comp comp = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            class Proj1 = identity, class Proj2 = identity,
            indirect_strict_weak_order<projected<I1, Proj1>, projected<I2, Proj2>> Comp =
                ranges::less>
        bool includes(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});    // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            class Proj1 = identity, class Proj2 = identity,
            indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                    projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
        bool includes(Ep&& exec, R1&& r1, R2&& r2,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});    // freestanding-deleted
}

```

```

}

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_union(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_union(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result, Compare comp);

namespace ranges {
    template<class I1, class I2, class O>
        using set_union_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            weakly_incrementable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_union_result<I1, I2, O>
            set_union(I1 first1, S1 last1, I2 first2, S2 last2, O result, Comp comp = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incrementable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
            set_union(R1&& r1, R2&& r2, O result, Comp comp = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        set_union_result<I1, I2, O>
            set_union(Ep&& exec, I1 first1, S1 last1,
                    I2 first2, S2 last2, O result, OutS result_last,
                    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            sized-random-access-range OutR, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
            borrowed_iterator_t<OutR>>
            set_union(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

```



```

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2,
                    OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2,
                    OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_intersection(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                    ForwardIterator1 first1, ForwardIterator1 last1,
                    ForwardIterator2 first2, ForwardIterator2 last2,
                    ForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_intersection(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                    ForwardIterator1 first1, ForwardIterator1 last1,
                    ForwardIterator2 first2, ForwardIterator2 last2,
                    ForwardIterator result, Compare comp);

namespace ranges {
    template<class I1, class I2, class O>
        using set_intersection_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            weakly_incremtable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_intersection_result<I1, I2, O>
            set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incremtable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
            set_intersection(R1&& r1, R2&& r2, O result,
                            Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        set_intersection_result<I1, I2, O>
            set_intersection(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                            O result, OutS result_last, Comp comp = {}, Proj1 proj1 = {},
                            Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
            sized_random_access_range OutR, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
                                borrowed_iterator_t<OutR>>
            set_intersection(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                            Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

```



```

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_difference(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_difference(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result, Compare comp);

namespace ranges {
    template<class I, class O>
        using set_difference_result = in_out_result<I, O>;
    template<class I1, class I2, class O>
        using set_difference_truncated_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            weakly_incrementable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_difference_result<I1, O>
            set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
    template<input_range R1, input_range R2, weakly_incrementable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_difference_result<borrowed_iterator_t<R1>, O>
            set_difference(R1&& r1, R2&& r2, O result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        set_difference_truncated_result<I1, I2, O>
            set_difference(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                          O result, OutS result_last, Comp comp = {}, Proj1 proj1 = {},
                          Proj2 proj2 = {}); // freestanding-deleted
    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            sized-random-access-range OutR, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        set_difference_truncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
            borrowed_iterator_t<OutR>>
            set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

```

```

template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2, InputIterator2 last2,
                             OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2, InputIterator2 last2,
                             OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_symmetric_difference(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                             ForwardIterator1 first1, ForwardIterator1 last1,
                             ForwardIterator2 first2, ForwardIterator2 last2,
                             ForwardIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_symmetric_difference(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                             ForwardIterator1 first1, ForwardIterator1 last1,
                             ForwardIterator2 first2, ForwardIterator2 last2,
                             ForwardIterator result, Compare comp);

namespace ranges {
    template<class I1, class I2, class O>
        using set_symmetric_difference_result = in_in_out_result<I1, I2, O>;

    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            weakly_incremtable O, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        constexpr set_symmetric_difference_result<I1, I2, O>
            set_symmetric_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                                     Comp comp = {}, Proj1 proj1 = {},
                                     Proj2 proj2 = {});

    template<input_range R1, input_range R2, weakly_incremtable O,
            class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
        constexpr set_symmetric_difference_result<borrowed_iterator_t<R1>,
            borrowed_iterator_t<R2>, O>
            set_symmetric_difference(R1&& r1, R2&& r2, O result, Comp comp = {},
                                     Proj1 proj1 = {}, Proj2 proj2 = {});

    template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
            random_access_iterator I2, sized_sentinel_for<I2> S2,
            random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
        set_symmetric_difference_result<I1, I2, O>
            set_symmetric_difference(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                                     O result, OutS result_last, Comp comp = {},
                                     Proj1 proj1 = {}, Proj2 proj2 = {});           // freestanding-deleted

    template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
            sized-random-access-range OutR, class Comp = ranges::less,
            class Proj1 = identity, class Proj2 = identity>
        requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
        set_symmetric_difference_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
            borrowed_iterator_t<OutR>>
            set_symmetric_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                                     Proj1 proj1 = {}, Proj2 proj2 = {});           // freestanding-deleted
}

```

// 26.8.8, heap operations

```

template<class RandomAccessIterator>
    constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                             Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            push_heap(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            push_heap(R&& r, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
    constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last,
                             Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            pop_heap(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            pop_heap(R&& r, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
    constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                             Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            make_heap(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            make_heap(R&& r, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
    constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
                             Compare comp);

```

```

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr I
            sort_heap(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Comp = ranges::less, class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr borrowed_iterator_t<R>
            sort_heap(R&& r, Comp comp = {}, Proj proj = {});
}

template<class RandomAccessIterator>
    constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last,
                           Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    bool is_heap(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                 RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    bool is_heap(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                 RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        constexpr bool is_heap(I first, S last, Comp comp = {}, Proj proj = {});
    template<random_access_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        constexpr bool is_heap(R&& r, Comp comp = {}, Proj proj = {});

    template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        bool is_heap(Ep&& exec, I first, S last, Comp comp = {},
                     Proj proj = {}); // freestanding-deleted
    template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
        bool is_heap(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class RandomAccessIterator>
    constexpr RandomAccessIterator
        is_heap_until(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
    constexpr RandomAccessIterator
        is_heap_until(RandomAccessIterator first, RandomAccessIterator last,
                     Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator>
    RandomAccessIterator
        is_heap_until(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                     RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
    RandomAccessIterator
        is_heap_until(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                     RandomAccessIterator first, RandomAccessIterator last,
                     Compare comp);

namespace ranges {
    template<random_access_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
        constexpr I is_heap_until(I first, S last, Comp comp = {}, Proj proj = {});
}

```

```

template<random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr borrowed_iterator_t<R>
    is_heap_until(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
I is_heap_until(Ep&& exec, I first, S last, Comp comp = {},
               Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
borrowed_iterator_t<R>
    is_heap_until(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

// 26.8.9, minimum and maximum
template<class T> constexpr const T& min(const T& a, const T& b);
template<class T, class Compare>
    constexpr const T& min(const T& a, const T& b, Compare comp);
template<class T>
    constexpr T min(initializer_list<T> t);
template<class T, class Compare>
    constexpr T min(initializer_list<T> t, Compare comp);

namespace ranges {
    template<class T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr const T& min(const T& a, const T& b, Comp comp = {}, Proj proj = {});
    template<copyable T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr T min(initializer_list<T> r, Comp comp = {}, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    constexpr range_value_t<R>
        min(R&& r, Comp comp = {}, Proj proj = {});
    template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    range_value_t<R>
        min(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class T> constexpr const T& max(const T& a, const T& b);
template<class T, class Compare>
    constexpr const T& max(const T& a, const T& b, Compare comp);
template<class T>
    constexpr T max(initializer_list<T> t);
template<class T, class Compare>
    constexpr T max(initializer_list<T> t, Compare comp);

namespace ranges {
    template<class T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr const T& max(const T& a, const T& b, Comp comp = {}, Proj proj = {});
    template<copyable T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr T max(initializer_list<T> r, Comp comp = {}, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    constexpr range_value_t<R>
        max(R&& r, Comp comp = {}, Proj proj = {});
}

```

```

template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
range_value_t<R>
    max(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});           // freestanding-deleted
}

template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);
template<class T, class Compare>
    constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);
template<class T>
    constexpr pair<T, T> minmax(initializer_list<T> t);
template<class T, class Compare>
    constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

namespace ranges {
    template<class T>
        using minmax_result = min_max_result<T>;

    template<class T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr minmax_result<const T&>
        minmax(const T& a, const T& b, Comp comp = {}, Proj proj = {});
    template<copyable T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr minmax_result<T>
        minmax(initializer_list<T> r, Comp comp = {}, Proj proj = {});
    template<input_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    constexpr minmax_result<range_value_t<R>>
        minmax(R&& r, Comp comp = {}, Proj proj = {});
    template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
    minmax_result<range_value_t<R>>
        minmax(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});           // freestanding-deleted
}

template<class ForwardIterator>
    constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
    constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last,
                                         Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator min_element(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                               ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator min_element(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                               ForwardIterator first, ForwardIterator last,
                               Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    constexpr I min_element(I first, S last, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    constexpr borrowed_iterator_t<R>
        min_element(R&& r, Comp comp = {}, Proj proj = {});
}

```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
I min_element(Ep&& exec, I first, S last, Comp comp = {},
              Proj proj = {}); // freestanding-deleted
template<execution-policy Ep, sized-random-access-range R,
        class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
borrowed_iterator_t<R>
min_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last,
                                      Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator max_element(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                          ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
ForwardIterator max_element(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                          ForwardIterator first, ForwardIterator last,
                          Compare comp);

namespace ranges {
    template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    constexpr I max_element(I first, S last, Comp comp = {}, Proj proj = {});
    template<forward_range R, class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    constexpr borrowed_iterator_t<R>
    max_element(R&& r, Comp comp = {}, Proj proj = {});

    template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
            class Proj = identity,
            indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    I max_element(Ep&& exec, I first, S last, Comp comp = {},
                  Proj proj = {}); // freestanding-deleted
    template<execution-policy Ep, sized-random-access-range R,
            class Proj = identity,
            indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    borrowed_iterator_t<R>
    max_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

template<class ForwardIterator>
constexpr pair<ForwardIterator, ForwardIterator>
minmax_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);
template<class ExecutionPolicy, class ForwardIterator>
pair<ForwardIterator, ForwardIterator>
minmax_element(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
pair<ForwardIterator, ForwardIterator>
minmax_element(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
              ForwardIterator first, ForwardIterator last, Compare comp);

namespace ranges {
    template<class I>
    using minmax_element_result = min_max_result<I>;

```



```

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr minmax_element_result<I>
    minmax_element(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr minmax_element_result<borrowed_iterator_t<R>>
    minmax_element(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
minmax_element_result<I>
    minmax_element(Ep&& exec, I first, S last, Comp comp = {},
                    Proj proj = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
minmax_element_result<borrowed_iterator_t<R>>
    minmax_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {}); // freestanding-deleted
}

// 26.8.10, bounded value
template<class T>
    constexpr const T& clamp(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
    constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);

namespace ranges {
    template<class T, class Proj = identity,
            indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
    constexpr const T&
        clamp(const T& v, const T& lo, const T& hi, Comp comp = {}, Proj proj = {});
}

// 26.8.11, lexicographical comparison
template<class InputIterator1, class InputIterator2>
    constexpr bool
        lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                                InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
    constexpr bool
        lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                                InputIterator2 first2, InputIterator2 last2,
                                Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
    bool
        lexicographical_compare(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                                ForwardIterator1 first1, ForwardIterator1 last1,
                                ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Compare>
    bool
        lexicographical_compare(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                                ForwardIterator1 first1, ForwardIterator1 last1,
                                ForwardIterator2 first2, ForwardIterator2 last2,
                                Compare comp);

namespace ranges {
    template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
            class Proj1 = identity, class Proj2 = identity,
            indirect_strict_weak_order<projected<I1, Proj1>, projected<I2, Proj2>> Comp =
                ranges::less>
    constexpr bool
        lexicographical_compare(I1 first1, S1 last1, I2 first2, S2 last2,

```



```

        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Proj1 = identity,
        class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
constexpr bool
    lexicographical_compare(R1&& r1, R2&& r2, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<I1, Proj1>,
        projected<I2, Proj2>> Comp = ranges::less>
bool lexicographical_compare(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
        Comp comp = {}, Proj1 proj1 = {},
        Proj2 proj2 = {}); // freestanding-deleted
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
bool lexicographical_compare(Ep&& exec, R1&& r1, R2&& r2, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {}); // freestanding-deleted
}

// 26.8.12, three-way comparison algorithms
template<class InputIterator1, class InputIterator2, class Cmp>
constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
        InputIterator2 b2, InputIterator2 e2,
        Cmp comp)
    -> decltype(comp(*b1, *b2));
template<class InputIterator1, class InputIterator2>
constexpr auto
    lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
        InputIterator2 b2, InputIterator2 e2);

// 26.8.13, permutations
template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first,
        BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first,
        BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
        using next_permutation_result = in_found_result<I>;

    template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr next_permutation_result<I>
            next_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<bidirectional_range R, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr next_permutation_result<borrowed_iterator_t<R>>
            next_permutation(R&& r, Comp comp = {}, Proj proj = {});
}

template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
        BidirectionalIterator last);

```

```

template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first,
                                BidirectionalIterator last, Compare comp);

namespace ranges {
    template<class I>
        using prev_permutation_result = in_found_result<I>;

    template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<I, Comp, Proj>
        constexpr prev_permutation_result<I>
            prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
    template<bidirectional_range R, class Comp = ranges::less,
            class Proj = identity>
        requires sortable<iterator_t<R>, Comp, Proj>
        constexpr prev_permutation_result<borrowed_iterator_t<R>>
            prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
}
}

```

26.5 Algorithm result types

[algorithms.results]

- ¹ Each of the class templates specified in this subclause has the template parameters, data members, and special members specified below, and has no base classes or members other than those specified.

```

namespace std::ranges {
    template<class I, class F>
    struct in_fun_result {
        [[no_unique_address]] I in;
        [[no_unique_address]] F fun;

        template<class I2, class F2>
            requires convertible_to<const I&, I2> && convertible_to<const F&, F2>
            constexpr operator in_fun_result<I2, F2>() const & {
                return {in, fun};
            }

        template<class I2, class F2>
            requires convertible_to<I, I2> && convertible_to<F, F2>
            constexpr operator in_fun_result<I2, F2>() && {
                return {std::move(in), std::move(fun)};
            }
    };

    template<class I1, class I2>
    struct in_in_result {
        [[no_unique_address]] I1 in1;
        [[no_unique_address]] I2 in2;

        template<class II1, class II2>
            requires convertible_to<const I1&, II1> && convertible_to<const I2&, II2>
            constexpr operator in_in_result<II1, II2>() const & {
                return {in1, in2};
            }

        template<class II1, class II2>
            requires convertible_to<I1, II1> && convertible_to<I2, II2>
            constexpr operator in_in_result<II1, II2>() && {
                return {std::move(in1), std::move(in2)};
            }
    };
}

```

```

template<class I, class O>
struct in_out_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] O out;

    template<class I2, class O2>
        requires convertible_to<const I&, I2> && convertible_to<const O&, O2>
    constexpr operator in_out_result<I2, O2>() const & {
        return {in, out};
    }

    template<class I2, class O2>
        requires convertible_to<I, I2> && convertible_to<O, O2>
    constexpr operator in_out_result<I2, O2>() && {
        return {std::move(in), std::move(out)};
    }
};

template<class I1, class I2, class O>
struct in_in_out_result {
    [[no_unique_address]] I1 in1;
    [[no_unique_address]] I2 in2;
    [[no_unique_address]] O out;

    template<class II1, class II2, class OO>
        requires convertible_to<const I1&, II1> &&
            convertible_to<const I2&, II2> &&
            convertible_to<const O&, OO>
    constexpr operator in_in_out_result<II1, II2, OO>() const & {
        return {in1, in2, out};
    }

    template<class II1, class II2, class OO>
        requires convertible_to<I1, II1> &&
            convertible_to<I2, II2> &&
            convertible_to<O, OO>
    constexpr operator in_in_out_result<II1, II2, OO>() && {
        return {std::move(in1), std::move(in2), std::move(out)};
    }
};

template<class I, class O1, class O2>
struct in_out_out_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] O1 out1;
    [[no_unique_address]] O2 out2;

    template<class II, class OO1, class OO2>
        requires convertible_to<const I&, II> &&
            convertible_to<const O1&, OO1> &&
            convertible_to<const O2&, OO2>
    constexpr operator in_out_out_result<II, OO1, OO2>() const & {
        return {in, out1, out2};
    }

    template<class II, class OO1, class OO2>
        requires convertible_to<I, II> &&
            convertible_to<O1, OO1> &&
            convertible_to<O2, OO2>
    constexpr operator in_out_out_result<II, OO1, OO2>() && {
        return {std::move(in), std::move(out1), std::move(out2)};
    }
};

```

```

template<class T>
struct min_max_result {
    [[no_unique_address]] T min;
    [[no_unique_address]] T max;

    template<class T2>
        requires convertible_to<const T&, T2>
    constexpr operator min_max_result<T2>() const & {
        return {min, max};
    }

    template<class T2>
        requires convertible_to<T, T2>
    constexpr operator min_max_result<T2>() && {
        return {std::move(min), std::move(max)};
    }
};

template<class I>
struct in_found_result {
    [[no_unique_address]] I in;
    bool found;

    template<class I2>
        requires convertible_to<const I&, I2>
    constexpr operator in_found_result<I2>() const & {
        return {in, found};
    }

    template<class I2>
        requires convertible_to<I, I2>
    constexpr operator in_found_result<I2>() && {
        return {std::move(in), found};
    }
};

template<class I, class T>
struct in_value_result {
    [[no_unique_address]] I in;
    [[no_unique_address]] T value;

    template<class I2, class T2>
        requires convertible_to<const I&, I2> && convertible_to<const T&, T2>
    constexpr operator in_value_result<I2, T2>() const & {
        return {in, value};
    }

    template<class I2, class T2>
        requires convertible_to<I, I2> && convertible_to<T, T2>
    constexpr operator in_value_result<I2, T2>() && {
        return {std::move(in), std::move(value)};
    }
};

template<class O, class T>
struct out_value_result {
    [[no_unique_address]] O out;
    [[no_unique_address]] T value;

    template<class O2, class T2>
        requires convertible_to<const O&, O2> && convertible_to<const T&, T2>
    constexpr operator out_value_result<O2, T2>() const & {
        return {out, value};
    }
};

```

```

template<class O2, class T2>
    requires convertible_to<O, O2> && convertible_to<T, T2>
constexpr operator out_value_result<O2, T2>() && {
    return {std::move(out), std::move(value)};
}
};
}

```

26.6 Non-modifying sequence operations

[alg.nonmodifying]

26.6.1 All of

[alg.all.of]

```

template<class InputIterator, class Predicate>
    constexpr bool all_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool all_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr bool ranges::all_of(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr bool ranges::all_of(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    bool ranges::all_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    bool ranges::all_of(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

- 1 Let *E* be:
 - (1.1) — `pred(*i)` for the overloads in namespace `std`;
 - (1.2) — `invoke(pred, invoke(proj, *i))` for the overloads in namespace `ranges`.
- 2 *Returns:* `false` if *E* is `false` for some iterator *i* in the range `[first, last)`, and `true` otherwise.
- 3 *Complexity:* At most `last - first` applications of the predicate and any projection.

26.6.2 Any of

[alg.any.of]

```

template<class InputIterator, class Predicate>
    constexpr bool any_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    bool any_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr bool ranges::any_of(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr bool ranges::any_of(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    bool ranges::any_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    bool ranges::any_of(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

- 1 Let *E* be:
 - (1.1) — `pred(*i)` for the overloads in namespace `std`;
 - (1.2) — `invoke(pred, invoke(proj, *i))` for the overloads in namespace `ranges`.

2 *Returns:* true if E is true for some iterator i in the range $[first, last)$, and false otherwise.

3 *Complexity:* At most $last - first$ applications of the predicate and any projection.

26.6.3 None of

[alg.none.of]

```
template<class InputIterator, class Predicate>
constexpr bool none_of(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
bool none_of(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
             Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr bool ranges::none_of(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr bool ranges::none_of(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
bool ranges::none_of(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
bool ranges::none_of(Ep&& exec, R&& r, Pred pred, Proj proj = {});
```

1 Let E be:

(1.1) — $\text{pred}(*i)$ for the overloads in namespace `std`;

(1.2) — $\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i))$ for the overloads in namespace `ranges`.

2 *Returns:* false if E is true for some iterator i in the range $[first, last)$, and true otherwise.

3 *Complexity:* At most $last - first$ applications of the predicate and any projection.

26.6.4 Contains

[alg.contains]

```
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
constexpr bool ranges::contains(I first, S last, const T& value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr bool ranges::contains(R&& r, const T& value, Proj proj = {});
```

1 *Returns:* $\text{ranges::find}(\text{std::move}(\text{first}), \text{last}, \text{value}, \text{proj}) \neq \text{last}$.

```
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
bool ranges::contains(Ep&& exec, I first, S last, const T& value, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         class T = projected_value_t<iterator_t<R>, Proj>>
requires
indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
bool ranges::contains(Ep&& exec, R&& r, const T& value, Proj proj = {});
```

2 *Returns:* $\text{ranges::find}(\text{std::forward}<Ep>(\text{exec}), \text{first}, \text{last}, \text{value}, \text{proj}) \neq \text{last}$.

```
template<forward_iterator I1, sentinel_for<I1> S1,
         forward_iterator I2, sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::contains_subrange(I1 first1, S1 last1, I2 first2, S2 last2,
                                         Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

```
template<forward_range R1, forward_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::contains_subrange(R1&& r1, R2&& r2, Pred pred = {},
                                         Proj1 proj1 = {}, Proj2 proj2 = {});
```

3 *Returns:*

```
first2 == last2 || !ranges::search(first1, last1, first2, last2,
                                   pred, proj1, proj2).empty()
```

```
template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool ranges::contains_subrange(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool ranges::contains_subrange(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                               Proj1 proj1 = {}, Proj2 proj2 = {});
```

4 *Returns:*

```
first2 == last2 || !ranges::search(std::forward<Ep>(exec), first1, last1,
                                   first2, last2, pred, proj1, proj2).empty()
```

26.6.5 For each

[alg.foreach]

```
template<class InputIterator, class Function>
constexpr Function for_each(InputIterator first, InputIterator last, Function f);
```

1 *Preconditions:* Function meets the *Cpp17MoveConstructible* requirements (Table 31).

[Note 1: Function need not meet the requirements of *Cpp17CopyConstructible* (Table 32). — end note]

2 *Effects:* Applies *f* to the result of dereferencing every iterator in the range *[first, last)*, starting from *first* and proceeding to *last - 1*.

[Note 2: If the type of *first* meets the requirements of a mutable iterator, *f* can apply non-constant functions through the dereferenced iterator. — end note]

3 *Returns:* *f*.

4 *Complexity:* Applies *f* exactly *last - first* times.

5 *Remarks:* If *f* returns a result, the result is ignored.

```
template<class ExecutionPolicy, class ForwardIterator, class Function>
void for_each(ExecutionPolicy&& exec,
              ForwardIterator first, ForwardIterator last,
              Function f);
```

6 *Preconditions:* Function meets the *Cpp17CopyConstructible* requirements.

7 *Effects:* Applies *f* to the result of dereferencing every iterator in the range *[first, last)*.

[Note 3: If the type of *first* meets the requirements of a mutable iterator, *f* can apply non-constant functions through the dereferenced iterator. — end note]

8 *Complexity:* Applies *f* exactly *last - first* times.

9 *Remarks:* If *f* returns a result, the result is ignored. Implementations do not have the freedom granted under 26.3.3 to make arbitrary copies of elements from the input sequence.

10 [Note 4: Does not return a copy of its Function parameter, since parallelization often does not permit efficient state accumulation. — end note]

```
template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
constexpr ranges::for_each_result<I, Fun>
ranges::for_each(I first, S last, Fun f, Proj proj = {});
```

```
template<input_range R, class Proj = identity,
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
ranges::for_each(R&& r, Fun f, Proj proj = {});
```

11 *Effects:* Calls `invoke(f, invoke(proj, *i))` for every iterator `i` in the range `[first, last)`, starting from `first` and proceeding to `last - 1`.

[Note 5: If the result of `invoke(proj, *i)` is a mutable reference, `f` can apply non-constant functions. — end note]

12 *Returns:* `{last, std::move(f)}`.

13 *Complexity:* Applies `f` and `proj` exactly `last - first` times.

14 *Remarks:* If `f` returns a result, the result is ignored.

15 [Note 6: The overloads in namespace `ranges` require `Fun` to model `copy_constructible`. — end note]

```
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
I ranges::for_each(Ep&& exec, I first, S last, Fun f, Proj proj = {});
template<execution_policy Ep, sized-random-access-range R, class Proj = identity,
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
borrowed_iterator_t<R>
ranges::for_each(Ep&& exec, R&& r, Fun f, Proj proj = {});
```

16 *Effects:* Calls `invoke(f, invoke(proj, *i))` for every iterator `i` in the range `[first, last)`.

[Note 7: If the result of `invoke(proj, *i)` is a mutable reference, `f` can apply non-constant functions. — end note]

17 *Returns:* `last`.

18 *Complexity:* Applies `f` and `proj` exactly `last - first` times.

19 *Remarks:*

(19.1) — If `f` returns a result, the result is ignored.

(19.2) — Implementations do not have the freedom granted under 26.3.3 to make arbitrary copies of elements from the input sequence.

(19.3) — `f` may modify objects via its arguments (26.3.2).

[Note 8: Does not return a copy of its `Fun` parameter, since parallelization often does not permit efficient state accumulation. — end note]

```
template<class InputIterator, class Size, class Function>
constexpr InputIterator for_each_n(InputIterator first, Size n, Function f);
```

20 *Mandates:* The type `Size` is convertible to an integral type (7.3.9, 11.4.8).

21 *Preconditions:* `n >= 0` is true. `Function` meets the *Cpp17MoveConstructible* requirements.

[Note 9: `Function` need not meet the requirements of *Cpp17CopyConstructible*. — end note]

22 *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first, first + n)` in order.

[Note 10: If the type of `first` meets the requirements of a mutable iterator, `f` can apply non-constant functions through the dereferenced iterator. — end note]

23 *Returns:* `first + n`.

24 *Remarks:* If `f` returns a result, the result is ignored.

```
template<class ExecutionPolicy, class ForwardIterator, class Size, class Function>
ForwardIterator for_each_n(ExecutionPolicy&& exec, ForwardIterator first, Size n,
                          Function f);
```

25 *Mandates:* The type `Size` is convertible to an integral type (7.3.9, 11.4.8).

26 *Preconditions:* `n >= 0` is true. `Function` meets the *Cpp17CopyConstructible* requirements.

27 *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first, first + n)`.

[Note 11: If the type of `first` meets the requirements of a mutable iterator, `f` can apply non-constant functions through the dereferenced iterator. — end note]

28 *Returns:* `first + n`.

29 *Remarks:* If `f` returns a result, the result is ignored. Implementations do not have the freedom granted under 26.3.3 to make arbitrary copies of elements from the input sequence.

```
template<input_iterator I, class Proj = identity,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
constexpr ranges::for_each_n_result<I, Fun>
ranges::for_each_n(I first, iter_difference_t<I> n, Fun f, Proj proj = {});
```

30 *Preconditions:* `n >= 0` is true.

31 *Effects:* Calls `invoke(f, invoke(proj, *i))` for every iterator `i` in the range `[first, first + n)` in order.

[Note 12: If the result of `invoke(proj, *i)` is a mutable reference, `f` can apply non-constant functions. — end note]

32 *Returns:* `{first + n, std::move(f)}`.

33 *Remarks:* If `f` returns a result, the result is ignored.

34 [Note 13: The overload in namespace `ranges` requires `Fun` to model `copy_constructible`. — end note]

```
template<execution_policy Ep, random_access_iterator I, class Proj = identity,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
I ranges::for_each_n(Ep&& exec, I first, iter_difference_t<I> n, Fun f, Proj proj = {});
```

35 *Preconditions:* `n >= 0` is true.

36 *Effects:* Calls `invoke(f, invoke(proj, *i))` for every iterator `i` in the range `[first, first + n)`.

[Note 14: If the result of `invoke(proj, *i)` is a mutable reference, `f` can apply non-constant functions. — end note]

37 *Returns:* `first + n`.

38 *Remarks:*

(38.1) — If `f` returns a result, the result is ignored.

(38.2) — Implementations do not have the freedom granted under 26.3.3 to make arbitrary copies of elements from the input sequence.

(38.3) — `f` may modify objects via its arguments (26.3.2).

[Note 15: Does not return a copy of its `Fun` parameter, since parallelization often does not permit efficient state accumulation. — end note]

26.6.6 Find

[alg.find]

```
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
constexpr InputIterator find(InputIterator first, InputIterator last,
                             const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator find(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                     const T& value);

template<class InputIterator, class Predicate>
constexpr InputIterator find_if(InputIterator first, InputIterator last,
                                Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator find_if(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                        Predicate pred);

template<class InputIterator, class Predicate>
constexpr InputIterator find_if_not(InputIterator first, InputIterator last,
                                    Predicate pred);
```

```

template<class ExecutionPolicy, class ForwardIterator, class Predicate>
    ForwardIterator find_if_not(ExecutionPolicy&& exec,
                              ForwardIterator first, ForwardIterator last,
                              Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr I ranges::find(I first, S last, const T& value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_iterator_t<R>
        ranges::find(R&& r, const T& value, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    I ranges::find(Ep&& exec, I first, S last, const T& value, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    borrowed_iterator_t<R> ranges::find(Ep&& exec, R&& r, const T& value, Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr I ranges::find_if(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_iterator_t<R>
        ranges::find_if(R&& r, Pred pred, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    I ranges::find_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_iterator_t<R> ranges::find_if(Ep&& exec, R&& r, Pred pred, Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr I ranges::find_if_not(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_iterator_t<R>
        ranges::find_if_not(R&& r, Pred pred, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    I ranges::find_if_not(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_iterator_t<R> ranges::find_if_not(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

1 Let E be:

- (1.1) — $*i == \text{value}$ for `find`;
- (1.2) — $\text{pred}(*i) != \text{false}$ for `find_if`;
- (1.3) — $\text{pred}(*i) == \text{false}$ for `find_if_not`;
- (1.4) — $\text{bool}(\text{invoke}(\text{proj}, *i) == \text{value})$ for `ranges::find`;
- (1.5) — $\text{bool}(\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i)))$ for `ranges::find_if`;
- (1.6) — $\text{bool}(!\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i)))$ for `ranges::find_if_not`.

2 *Returns:* The first iterator *i* in the range `[first,last)` for which *E* is true. Returns `last` if no such iterator is found.

3 *Complexity:* At most `last - first` applications of the corresponding predicate and any projection.

26.6.7 Find last

[alg.find.last]

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr subrange<I> ranges::find_last(I first, S last, const T& value, Proj proj = {});
template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_subrange_t<R> ranges::find_last(R&& r, const T& value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    subrange<I> ranges::find_last(Ep&& exec, I first, S last, const T& value, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
    requires
        indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
        borrowed_subrange_t<R> ranges::find_last(Ep&& exec, R&& r, const T& value, Proj proj = {});

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr subrange<I> ranges::find_last_if(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_subrange_t<R> ranges::find_last_if(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    subrange<I> ranges::find_last_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_subrange_t<R> ranges::find_last_if(Ep&& exec, R&& r, Pred pred, Proj proj = {});

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    constexpr subrange<I> ranges::find_last_if_not(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    constexpr borrowed_subrange_t<R> ranges::find_last_if_not(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    subrange<I> ranges::find_last_if_not(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    borrowed_subrange_t<R> ranges::find_last_if_not(Ep&& exec, R&& r, Pred pred, Proj proj = {});
```

1 Let *E* be:

- (1.1) — `bool(invoker(proj, *i) == value)` for `ranges::find_last`;
- (1.2) — `bool(invoker(pred, invoker(proj, *i)))` for `ranges::find_last_if`;
- (1.3) — `bool(!invoker(pred, invoker(proj, *i)))` for `ranges::find_last_if_not`.

2 *Returns:* Let *i* be the last iterator in the range `[first,last)` for which *E* is true. Returns `{i, last}`, or `{last, last}` if no such iterator is found.

3 *Complexity:* At most `last - first` applications of the corresponding predicate and projection.

26.6.8 Find end

[alg.find.end]

```

template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
    find_end(ExecutionPolicy&& exec,
             ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
constexpr ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2,
             BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
ForwardIterator1
    find_end(ExecutionPolicy&& exec,
             ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2,
             BinaryPredicate pred);

template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2, sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr subrange<I1>
    ranges::find_end(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr borrowed_subrange_t<R1>
    ranges::find_end(R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
         random_access_iterator I2, sized_sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
subrange<I1>
    ranges::find_end(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
borrowed_subrange_t<R1>
    ranges::find_end(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — `pred` be `equal_to{}` for the overloads with no parameter `pred`;
- (1.2) — `E` be:
 - (1.2.1) — `pred(*(i + n), *(first2 + n))` for the overloads in namespace `std`;
 - (1.2.2) — `invoke(pred, invoke(proj1, *(i + n)), invoke(proj2, *(first2 + n)))` for the overloads in namespace `ranges`;
- (1.3) — `i` be `last1` if `[first2, last2)` is empty, or if `(last2 - first2) > (last1 - first1)` is true, or if there is no iterator in the range `[first1, last1 - (last2 - first2))` such that for every

non-negative integer $n < (\text{last2} - \text{first2})$, E is true. Otherwise i is the last such iterator in $[\text{first1}, \text{last1} - (\text{last2} - \text{first2})]$.

2 *Returns:*

(2.1) — i for the overloads in namespace `std`.

(2.2) — $\{i, i + (i == \text{last1} ? 0 : \text{last2} - \text{first2})\}$ for the overloads in namespace `ranges`.

3 *Complexity:* At most $(\text{last2} - \text{first2}) * (\text{last1} - \text{first1} - (\text{last2} - \text{first2}) + 1)$ applications of the corresponding predicate and any projections.

26.6.9 Find first of

[alg.find.first.of]

```
template<class InputIterator, class ForwardIterator>
constexpr InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
    find_first_of(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator, class ForwardIterator,
          class BinaryPredicate>
constexpr InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2,
                  BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
          class BinaryPredicate>
ForwardIterator1
    find_first_of(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  BinaryPredicate pred);

template<input_iterator I1, sentinel_for<I1> S1, forward_iterator I2, sentinel_for<I2> S2,
          class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr I1 ranges::find_first_of(I1 first1, S1 last1, I2 first2, S2 last2,
                                   Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, forward_range R2,
          class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr borrowed_iterator_t<R1>
    ranges::find_first_of(R1&& r1, R2&& r2,
                          Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
          random_access_iterator I2, sized_sentinel_for<I2> S2,
          class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
I1 ranges::find_first_of(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
          class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
borrowed_iterator_t<R1>
    ranges::find_first_of(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Let E be:

- (1.1) — $*i == *j$ for the overloads with no parameter `pred`;
 - (1.2) — `pred(*i, *j) != false` for the overloads with a parameter `pred` and no parameter `proj1`;
 - (1.3) — `bool(invoker(pred, invoker(proj1, *i), invoker(proj2, *j)))` for the overloads with parameters `pred` and `proj1`.
- 2 *Effects*: Finds an element that matches one of a set of values.
- 3 *Returns*: The first iterator `i` in the range `[first1, last1)` such that for some iterator `j` in the range `[first2, last2)` *E* holds. Returns `last1` if `[first2, last2)` is empty or if no such iterator is found.
- 4 *Complexity*: At most $(last1 - first1) * (last2 - first2)$ applications of the corresponding predicate and any projections.

26.6.10 Adjacent find

[alg.adjacent.find]

```
template<class ForwardIterator>
constexpr ForwardIterator
    adjacent_find(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    adjacent_find(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator
    adjacent_find(ForwardIterator first, ForwardIterator last,
                  BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
ForwardIterator
    adjacent_find(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last,
                  BinaryPredicate pred);

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_binary_predicate<projected<I, Proj>,
                                   projected<I, Proj>> Pred = ranges::equal_to>
constexpr I ranges::adjacent_find(I first, S last, Pred pred = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_binary_predicate<projected<iterator_t<R>, Proj>,
                                   projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
constexpr borrowed_iterator_t<R> ranges::adjacent_find(R&& r, Pred pred = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_binary_predicate<projected<I, Proj>,
                                   projected<I, Proj>> Pred = ranges::equal_to>
I ranges::adjacent_find(Ep&& exec, I first, S last, Pred pred = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_binary_predicate<projected<iterator_t<R>, Proj>,
                                   projected<iterator_t<R>, Proj>> Pred = ranges::equal_to>
borrowed_iterator_t<R>
    ranges::adjacent_find(Ep&& exec, R&& r, Pred pred = {}, Proj proj = {});
```

- 1 Let *E* be:
- (1.1) — $*i == *(i + 1)$ for the overloads with no parameter `pred`;
 - (1.2) — `pred(*i, *(i + 1)) != false` for the overloads with a parameter `pred` and no parameter `proj`;
 - (1.3) — `bool(invoker(pred, invoker(proj, *i), invoker(proj, *(i + 1))))` for the overloads with both parameters `pred` and `proj`.
- 2 *Returns*: The first iterator `i` such that both `i` and `i + 1` are in the range `[first, last)` for which *E* holds. Returns `last` if no such iterator is found.
- 3 *Complexity*: For the non-parallel algorithm overloads, exactly

$$\min((i - first) + 1, (last - first) - 1)$$

applications of the corresponding predicate, where `i` is `adjacent_find`'s return value. For the parallel algorithm overloads, $\mathcal{O}(\text{last} - \text{first})$ applications of the corresponding predicate. No more than twice as many applications of any projection.

26.6.11 Count

[alg.count]

```
template<class InputIterator, class T = iterator_traits<InputIterator>::value_type>
constexpr typename iterator_traits<InputIterator>::difference_type
    count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
typename iterator_traits<ForwardIterator>::difference_type
    count(ExecutionPolicy&& exec,
        ForwardIterator first, ForwardIterator last, const T& value);

template<class InputIterator, class Predicate>
constexpr typename iterator_traits<InputIterator>::difference_type
    count_if(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
typename iterator_traits<ForwardIterator>::difference_type
    count_if(ExecutionPolicy&& exec,
        ForwardIterator first, ForwardIterator last, Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
constexpr iter_difference_t<I>
    ranges::count(I first, S last, const T& value, Proj proj = {});
template<input_range R, class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr range_difference_t<R>
    ranges::count(R&& r, const T& value, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
iter_difference_t<I>
    ranges::count(Ep&& exec, I first, S last, const T& value, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
requires indirect_binary_predicate<ranges::equal_to,
    projected<iterator_t<R>, Proj>, const T*>
range_difference_t<R> ranges::count(Ep&& exec, R&& r, const T& value, Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr iter_difference_t<I>
    ranges::count_if(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr range_difference_t<R>
    ranges::count_if(R&& r, Pred pred, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
iter_difference_t<I>
    ranges::count_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
range_difference_t<R>
    ranges::count_if(Ep&& exec, R&& r, Pred pred, Proj proj = {});
```

1 Let E be:

(1.1) — $*i == \text{value}$ for the overloads with no parameter `pred` or `proj`;

- (1.2) — `pred(*i) != false` for the overloads with a parameter `pred` but no parameter `proj`;
 - (1.3) — `invoke(proj, *i) == value` for the overloads with a parameter `proj` but no parameter `pred`;
 - (1.4) — `bool(invoke(pred, invoke(proj, *i)))` for the overloads with both parameters `proj` and `pred`.
- 2 *Effects*: Returns the number of iterators `i` in the range `[first, last)` for which *E* holds.
- 3 *Complexity*: Exactly `last - first` applications of the corresponding predicate and any projection.

26.6.12 Mismatch

[alg.mismatch]

```
template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
constexpr pair<InputIterator1, InputIterator2>
mismatch(InputIterator1 first1, InputIterator1 last1,
         InputIterator2 first2, InputIterator2 last2,
         BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
pair<ForwardIterator1, ForwardIterator2>
mismatch(ExecutionPolicy&& exec,
         ForwardIterator1 first1, ForwardIterator1 last1,
         ForwardIterator2 first2, ForwardIterator2 last2,
         BinaryPredicate pred);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr ranges::mismatch_result<I1, I2>
ranges::mismatch(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                 Proj1 proj1 = {}, Proj2 proj2 = {});
```



```

template<input_range R1, input_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr ranges::mismatch_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
    ranges::mismatch(R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
ranges::mismatch_result<I1, I2>
    ranges::mismatch(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
ranges::mismatch_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
    ranges::mismatch(Ep&& exec, R1&& r1, R2&& r2, Pred pred = {},
                    Proj1 proj1 = {}, Proj2 proj2 = {});

```

- 1 Let $last2$ be $first2 + (last1 - first1)$ for the overloads in namespace `std` with no parameter $last2$.
- 2 Let E be:
 - (2.1) — $!(*(first1 + n) == *(first2 + n))$ for the overloads with no parameter $pred$;
 - (2.2) — $pred(*(first1 + n), *(first2 + n)) == false$ for the overloads with a parameter $pred$ and no parameter $proj1$;
 - (2.3) — $!invoke(pred, invoke(proj1, *(first1 + n)), invoke(proj2, *(first2 + n)))$ for the overloads with both parameters $pred$ and $proj1$.
- 3 Let N be $\min(last1 - first1, last2 - first2)$.
- 4 *Returns:* $\{ first1 + n, first2 + n \}$, where n is the smallest integer in $[0, N)$ such that E holds, or N if no such integer exists.
- 5 *Complexity:* At most N applications of the corresponding predicate and any projections.

26.6.13 Equal

[alg.equal]

```

template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool equal(ExecutionPolicy&& exec,
           ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2);

template<class InputIterator1, class InputIterator2,
        class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, BinaryPredicate pred);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
bool equal(ExecutionPolicy&& exec,
           ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool equal(ExecutionPolicy&& exec,
           ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2);

```

```

template<class InputIterator1, class InputIterator2,
        class BinaryPredicate>
constexpr bool equal(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, InputIterator2 last2,
                    BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
bool equal(ExecutionPolicy&& exec,
          ForwardIterator1 first1, ForwardIterator1 last1,
          ForwardIterator2 first2, ForwardIterator2 last2,
          BinaryPredicate pred);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::equal(I1 first1, S1 last1, I2 first2, S2 last2,
                             Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::equal(R1&& r1, R2&& r2, Pred pred = {},
                             Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool ranges::equal(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                  Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool ranges::equal(Ep&& exec, R1&& r1, R2&& r2,
                  Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — last2 be first2 + (last1 - first1) for the overloads in namespace std with no parameter last2;
- (1.2) — pred be equal_to{} for the overloads with no parameter pred;
- (1.3) — *E* be:
 - (1.3.1) — pred(*i, *(first2 + (i - first1))) for the overloads with no parameter proj1;
 - (1.3.2) — invoke(pred, invoke(proj1, *i), invoke(proj2, *(first2 + (i - first1)))) for the overloads with parameter proj1.

2 *Returns:* If last1 - first1 != last2 - first2, return false. Otherwise return true if *E* holds for every iterator *i* in the range [first1, last1). Otherwise, returns false.

3 *Complexity:* If

- (3.1) — the types of first1, last1, first2, and last2 meet the *Cpp17RandomAccessIterator* requirements (24.3.5.7) and last1 - first1 != last2 - first2 for the overloads in namespace std;
- (3.2) — the types of first1, last1, first2, and last2 pairwise model sized_sentinel_for (24.3.4.8) and last1 - first1 != last2 - first2 for the first and third overloads in namespace ranges, or
- (3.3) — R1 and R2 each model sized_range and ranges::distance(r1) != ranges::distance(r2) for the second and fourth overloads in namespace ranges,

then no applications of the corresponding predicate and each projection; otherwise, at most

$$\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$$

applications of the corresponding predicate and any projections.

26.6.14 Is permutation

[alg.is.permutation]

```

template<class ForwardIterator1, class ForwardIterator2>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, BinaryPredicate pred);
template<class ForwardIterator1, class ForwardIterator2>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
    constexpr bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                                   ForwardIterator2 first2, ForwardIterator2 last2,
                                   BinaryPredicate pred);

```

1 Let `last2` be `first2 + (last1 - first1)` for the overloads with no parameter named `last2`, and let `pred` be `equal_to{}` for the overloads with no parameter `pred`.

2 *Mandates:* `ForwardIterator1` and `ForwardIterator2` have the same value type.

3 *Preconditions:* The comparison function is an equivalence relation.

4 *Returns:* If `last1 - first1 != last2 - first2`, return `false`. Otherwise return `true` if there exists a permutation of the elements in the range `[first2, last2)`, beginning with `ForwardIterator2 begin`, such that `equal(first1, last1, begin, pred)` returns `true`; otherwise, returns `false`.

5 *Complexity:* No applications of the corresponding predicate if `ForwardIterator1` and `ForwardIterator2` meet the requirements of random access iterators and `last1 - first1 != last2 - first2`. Otherwise, exactly `last1 - first1` applications of the corresponding predicate if `equal(first1, last1, first2, last2, pred)` would return `true`; otherwise, at worst $\mathcal{O}(N^2)$, where N has the value `last1 - first1`.

```

template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
        sentinel_for<I2> S2, class Proj1 = identity, class Proj2 = identity,
        indirect_equivalence_relation<projected<I1, Proj1>,
        projected<I2, Proj2>> Pred = ranges::equal_to>
    constexpr bool ranges::is_permutation(I1 first1, S1 last1, I2 first2, S2 last2,
                                           Pred pred = {},
                                           Proj1 proj1 = {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_equivalence_relation<projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>> Pred = ranges::equal_to>
    constexpr bool ranges::is_permutation(R1&& r1, R2&& r2, Pred pred = {},
                                           Proj1 proj1 = {}, Proj2 proj2 = {});

```

6 *Returns:* If `last1 - first1 != last2 - first2`, return `false`. Otherwise return `true` if there exists a permutation of the elements in the range `[first2, last2)`, bounded by `[pfirst, plast)`, such that `ranges::equal(first1, last1, pfirst, plast, pred, proj1, proj2)` returns `true`; otherwise, returns `false`.

7 *Complexity:* No applications of the corresponding predicate and projections if

- (7.1) — for the first overload,
- (7.1.1) — `S1` and `I1` model `sized_sentinel_for<S1, I1>`,
- (7.1.2) — `S2` and `I2` model `sized_sentinel_for<S2, I2>`, and
- (7.1.3) — `last1 - first1 != last2 - first2`;
- (7.2) — for the second overload, `R1` and `R2` each model `sized_range`, and `ranges::distance(r1) != ranges::distance(r2)`.

Otherwise, exactly `last1 - first1` applications of the corresponding predicate and projections if `ranges::equal(first1, last1, first2, last2, pred, proj1, proj2)` would return `true`; otherwise, at worst $\mathcal{O}(N^2)$, where N has the value `last1 - first1`.

26.6.15 Search

[alg.search]

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1
search(ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator1
search(ExecutionPolicy&& exec,
       ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2);
```

```
template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
constexpr ForwardIterator1
search(ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2,
       BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
ForwardIterator1
search(ExecutionPolicy&& exec,
       ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2,
       BinaryPredicate pred);
```

¹ *Returns:* The first iterator *i* in the range $[first1, last1 - (last2 - first2)]$ such that for every non-negative integer *n* less than $last2 - first2$ the following corresponding conditions hold: $*(i + n) == *(first2 + n)$, $pred(*(i + n), *(first2 + n)) != false$. Returns *first1* if $[first2, last2)$ is empty, otherwise returns *last1* if no such iterator is found.

² *Complexity:* At most $(last1 - first1) * (last2 - first2)$ applications of the corresponding predicate.

```
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
         sentinel_for<I2> S2, class Pred = ranges::equal_to,
         class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr subrange<I1>
ranges::search(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});
template<forward_range R1, forward_range R2, class Pred = ranges::equal_to,
         class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr borrowed_subrange_t<R1>
ranges::search(R1&& r1, R2&& r2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});
```

```
template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
         random_access_iterator I2, sized_sentinel_for<I2> S2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
subrange<I1>
ranges::search(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
         class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
borrowed_subrange_t<R1>
ranges::search(Ep&& exec, R1&& r1, R2&& r2,
               Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

³ *Returns:*

(3.1) — $\{i, i + (last2 - first2)\}$, where *i* is the first iterator in the range $[first1, last1 - (last2 - first2)]$ such that for every non-negative integer *n* less than $last2 - first2$ the condition

```
bool(invoker(pred, invoker(proj1, *(i + n)), invoker(proj2, *(first2 + n))))
is true.
```

(3.2) — Returns {last1, last1} if no such iterator exists.

4 *Complexity:* At most (last1 - first1) * (last2 - first2) applications of the corresponding predicate and projections.

```
template<class ForwardIterator, class Size, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
search_n(ForwardIterator first, ForwardIterator last,
        Size count, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator
search_n(ExecutionPolicy&& exec,
        ForwardIterator first, ForwardIterator last,
        Size count, const T& value);
```

```
template<class ForwardIterator, class Size, class T = iterator_traits<ForwardIterator>::value_type,
        class BinaryPredicate>
constexpr ForwardIterator
search_n(ForwardIterator first, ForwardIterator last,
        Size count, const T& value,
        BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type,
        class BinaryPredicate>
ForwardIterator
search_n(ExecutionPolicy&& exec,
        ForwardIterator first, ForwardIterator last,
        Size count, const T& value,
        BinaryPredicate pred);
```

5 *Mandates:* The type Size is convertible to an integral type (7.3.9, 11.4.8).

6 Let E be $\text{pred}(*(i + n), \text{value}) \neq \text{false}$ for the overloads with a parameter pred , and $*(i + n) == \text{value}$ otherwise.

7 *Returns:* The first iterator i in the range $[\text{first}, \text{last} - \text{count}]$ such that for every non-negative integer n less than count the condition E is true. Returns last if no such iterator is found.

8 *Complexity:* At most $\text{last} - \text{first}$ applications of the corresponding predicate.

```
template<forward_iterator I, sentinel_for<I> S,
        class Pred = ranges::equal_to, class Proj = identity,
        class T = projected_value_t<I, Proj>>
requires indirectly_comparable<I, const T*, Pred, Proj>
constexpr subrange<I>
ranges::search_n(I first, S last, iter_difference_t<I> count,
        const T& value, Pred pred = {}, Proj proj = {});
template<forward_range R, class Pred = ranges::equal_to,
        class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
constexpr borrowed_subrange_t<R>
ranges::search_n(R&& r, range_difference_t<R> count,
        const T& value, Pred pred = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Pred = ranges::equal_to, class Proj = identity,
        class T = projected_value_t<I, Proj>>
requires indirectly_comparable<I, const T*, Pred, Proj>
subrange<I>
ranges::search_n(Ep&& exec, I first, S last, iter_difference_t<I> count,
        const T& value, Pred pred = {}, Proj proj = {});
```

```
template<execution-policy Ep, sized-random-access-range R, class Pred = ranges::equal_to,
        class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
borrowed_subrange_t<R>
ranges::search_n(Ep&& exec, R&& r, range_difference_t<R> count,
                 const T& value, Pred pred = {}, Proj proj = {});
```

Returns: {i, i + count} where i is the first iterator in the range [first, last - count] such that for every non-negative integer n less than count, the following condition holds: invoke(pred, invoke(proj, *(i + n)), value). Returns {last, last} if no such iterator is found.

Complexity: At most last - first applications of the corresponding predicate and projection.

```
template<class ForwardIterator, class Searcher>
constexpr ForwardIterator
search(ForwardIterator first, ForwardIterator last, const Searcher& searcher);
```

Effects: Equivalent to: return searcher(first, last).first;

Remarks: Searcher need not meet the *Cpp17CopyConstructible* requirements.

26.6.16 Starts with

[alg.starts.with]

```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::starts_with(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, class Pred = ranges::equal_to, class Proj1 = identity,
        class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::starts_with(R1&& r1, R2&& r2, Pred pred = {},
                                   Proj1 proj1 = {}, Proj2 proj2 = {});
```

Returns:

```
ranges::mismatch(std::move(first1), last1, std::move(first2), last2,
                 pred, proj1, proj2).in2 == last2
```

```
template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool ranges::starts_with(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                         Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution-policy Ep, sized-random-access-range R1,
        sized-random-access-range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool ranges::starts_with(Ep&& exec, R1&& r1, R2&& r2,
                         Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

Returns:

```
ranges::mismatch(std::forward<Ep>(exec), std::move(first1), last1, std::move(first2),
                 last2, pred, proj1, proj2).in2 == last2
```

26.6.17 Ends with

[alg.ends.with]

```
template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires (forward_iterator<I1> || sized_sentinel_for<S1, I1>) &&
        (forward_iterator<I2> || sized_sentinel_for<S2, I2>) &&
        indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
                                  Proj1 proj1 = {}, Proj2 proj2 = {});
```

Let N1 be last1 - first1 and N2 be last2 - first2.

Returns: false if N1 < N2, otherwise:

```

        ranges::equal(std::move(first1) + (N1 - N2), last1, std::move(first2), last2,
                      pred, proj1, proj2)

template<execution-policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Pred = ranges::equal_to, class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
bool ranges::ends_with(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                      Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
3   Let N1 be last1 - first1 and N2 be last2 - first2.
4   Returns: false if N1 < N2, otherwise:
        ranges::equal(std::forward<Ep>(exec), std::move(first1) + (N1 - N2), last1,
                      std::move(first2), last2, pred, proj1, proj2)

template<input_range R1, input_range R2, class Pred = ranges::equal_to, class Proj1 = identity,
        class Proj2 = identity>
requires (forward_range<R1> || sized_range<R1>) &&
        (forward_range<R2> || sized_range<R2>) &&
        indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr bool ranges::ends_with(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});
5   Let N1 be ranges::distance(r1) and N2 be ranges::distance(r2).
6   Returns: false if N1 < N2, otherwise:
        ranges::equal(views::drop(ranges::ref_view(r1), N1 - static_cast<decltype(N1)>(N2)),
                      r2, pred, proj1, proj2)

template<execution-policy Ep, sized-random-access-range R1,
        sized-random-access-range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
bool ranges::ends_with(Ep&& exec, R1&& r1, R2&& r2,
                      Pred pred = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
7   Let N1 be ranges::distance(r1) and N2 be ranges::distance(r2).
8   Returns: false if N1 < N2, otherwise:
        ranges::equal(std::forward<Ep>(exec),
                      views::drop(ranges::ref_view(r1), N1 - static_cast<decltype(N1)>(N2)),
                      r2, pred, proj1, proj2)

```

26.6.18 Fold

[alg.fold]

```

template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
        indirectly_binary_left_foldable<T, I> F>
constexpr auto ranges::fold_left(I first, S last, T init, F f);
template<input_range R, class T = range_value_t<R>,
        indirectly_binary_left_foldable<T, iterator_t<R>> F>
constexpr auto ranges::fold_left(R&& r, T init, F f);
1   Returns:
        ranges::fold_left_with_iter(std::move(first), last, std::move(init), f).value

template<input_iterator I, sentinel_for<I> S,
        indirectly_binary_left_foldable<iter_value_t<I>, I> F>
requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
constexpr auto ranges::fold_left_first(I first, S last, F f);
template<input_range R, indirectly_binary_left_foldable<range_value_t<R>, iterator_t<R>> F>
requires constructible_from<range_value_t<R>, range_reference_t<R>>
constexpr auto ranges::fold_left_first(R&& r, F f);
2   Returns:
        ranges::fold_left_first_with_iter(std::move(first), last, f).value

```

```
template<bidirectional_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
        indirectly_binary_right_foldable<T, I> F>
    constexpr auto ranges::fold_right(I first, S last, T init, F f);
template<bidirectional_range R, class T = range_value_t<R>,
        indirectly_binary_right_foldable<T, iterator_t<R>> F>
    constexpr auto ranges::fold_right(R&& r, T init, F f);
```

3 *Effects:* Equivalent to:

```
    using U = decay_t<invoke_result_t<F&, iter_reference_t<I>, T>>;
    if (first == last)
        return U(std::move(init));
    I tail = ranges::next(first, last);
    U accum = invoke(f, *--tail, std::move(init));
    while (first != tail)
        accum = invoke(f, *--tail, std::move(accum));
    return accum;
```

```
template<bidirectional_iterator I, sentinel_for<I> S,
        indirectly_binary_right_foldable<iter_value_t<I>, I> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr auto ranges::fold_right_last(I first, S last, F f);
template<bidirectional_range R,
        indirectly_binary_right_foldable<range_value_t<R>, iterator_t<R>> F>
    requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr auto ranges::fold_right_last(R&& r, F f);
```

4 Let U be decltype(ranges::fold_right(first, last, iter_value_t<I>(*first), f)).

5 *Effects:* Equivalent to:

```
    if (first == last)
        return optional<U>();
    I tail = ranges::prev(ranges::next(first, std::move(last)));
    return optional<U>(in_place,
        ranges::fold_right(std::move(first), tail, iter_value_t<I>(*tail), std::move(f)));
```

```
template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
        indirectly_binary_left_foldable<T, I> F>
    constexpr see below ranges::fold_left_with_iter(I first, S last, T init, F f);
template<input_range R, class T = range_value_t<R>,
        indirectly_binary_left_foldable<T, iterator_t<R>> F>
    constexpr see below ranges::fold_left_with_iter(R&& r, T init, F f);
```

6 Let U be decay_t<invoke_result_t<F&, T, iter_reference_t<I>>>.

7 *Effects:* Equivalent to:

```
    if (first == last)
        return {std::move(first), U(std::move(init))};
    U accum = invoke(f, std::move(init), *first);
    for (++first; first != last; ++first)
        accum = invoke(f, std::move(accum), *first);
    return {std::move(first), std::move(accum)};
```

8 *Remarks:* The return type is fold_left_with_iter_result<I, U> for the first overload and fold_left_with_iter_result<borrowed_iterator_t<R>, U> for the second overload.

```
template<input_iterator I, sentinel_for<I> S,
        indirectly_binary_left_foldable<iter_value_t<I>, I> F>
    requires constructible_from<iter_value_t<I>, iter_reference_t<I>>
    constexpr see below ranges::fold_left_first_with_iter(I first, S last, F f);
template<input_range R, indirectly_binary_left_foldable<range_value_t<R>, iterator_t<R>> F>
    requires constructible_from<range_value_t<R>, range_reference_t<R>>
    constexpr see below ranges::fold_left_first_with_iter(R&& r, F f);
```

9 Let U be

```
    decltype(ranges::fold_left(std::move(first), last, iter_value_t<I>(*first), f))
```

10 *Effects:* Equivalent to:


```

    if (first == last)
        return {std::move(first), optional<U>{}};
    optional<U> init(in_place, *first);
    for (++first; first != last; ++first)
        *init = invoke(f, std::move(*init), *first);
    return {std::move(first), std::move(init)};

```

11 *Remarks:* The return type is `fold_left_first_with_iter_result<I, optional<U>>` for the first overload and `fold_left_first_with_iter_result<borrowed_iterator_t<R>, optional<U>>` for the second overload.

26.7 Mutating sequence operations

[alg.modifying.operations]

26.7.1 Copy

[alg.copy]

```

template<class InputIterator, class OutputIterator>
constexpr OutputIterator copy(InputIterator first, InputIterator last,
                             OutputIterator result);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
requires indirectly_copyable<I, O>
constexpr ranges::copy_result<I, O> ranges::copy(I first, S last, O result);
template<input_range R, weakly_incrementable O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::copy_result<borrowed_iterator_t<R>, O> ranges::copy(R&& r, O result);

```

1 Let N be `last - first`.

2 *Preconditions:* `result` is not in the range `[first, last)`.

3 *Effects:* Copies elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer $n < N$, performs `*(result + n) = *(first + n)`.

4 *Returns:*

(4.1) — `result + N` for the overload in namespace `std`.

(4.2) — `{last, result + N}` for the overloads in namespace `ranges`.

5 *Complexity:* Exactly N assignments.

```

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 copy(ExecutionPolicy&& exec,
                     ForwardIterator1 first, ForwardIterator1 last,
                     ForwardIterator2 result);

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
ranges::copy_result<I, O>
ranges::copy(Ep&& exec, I first, S last, O result, OutS result_last);
template<execution_policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
ranges::copy(Ep&& exec, R&& r, OutR&& result_r);

```

6 Let `result_last` be `result + (last - first)` for the overload in namespace `std`.

7 Let N be `min(last - first, result_last - result)`.

8 *Preconditions:* The ranges `[first, last)` and `[result, result + N)` do not overlap.

9 *Effects:* Copies elements in the range `[first, first + N)` into the range `[result, result + N)`. For each non-negative integer $n < N$, performs `*(result + n) = *(first + n)`.

10 *Returns:*

(10.1) — `result + N` for the overload in namespace `std`.

(10.2) — `{first + N, result + N}` for the overloads in namespace `ranges`.

11 *Complexity:* Exactly N assignments.

```

template<class InputIterator, class Size, class OutputIterator>
constexpr OutputIterator copy_n(InputIterator first, Size n,
                                OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class Size, class ForwardIterator2>
ForwardIterator2 copy_n(ExecutionPolicy&& exec,
                        ForwardIterator1 first, Size n,
                        ForwardIterator2 result);

template<input_iterator I, weakly_incrementable O>
requires indirectly_copyable<I, O>
constexpr ranges::copy_n_result<I, O>
    ranges::copy_n(I first, iter_difference_t<I> n, O result);

template<execution-policy Ep, random_access_iterator I, random_access_iterator O,
        sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
ranges::copy_n_result<I, O>
    ranges::copy_n(Ep&& exec, I first, iter_difference_t<I> n, O result, OutS result_last);
12     Let  $M$  be  $\max(0, n)$ .
13     Let  $\text{result\_last}$  be  $\text{result} + M$  for the overloads with no parameter  $\text{result\_last}$ .
14     Let  $N$  be  $\min(\text{result\_last} - \text{result}, M)$ .
15     Mandates: The type  $\text{Size}$  is convertible to an integral type (7.3.9, 11.4.8).
16     Effects: For each non-negative integer  $i < N$ , performs  $\text{*(result + i) = *(first + i)}$ .
17     Returns:
(17.1) —  $\text{result} + N$  for the overloads in namespace std.
(17.2) —  $\{\text{first} + N, \text{result} + N\}$  for the overload in namespace ranges.
18     Complexity: Exactly  $N$  assignments.

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator copy_if(InputIterator first, InputIterator last,
                                OutputIterator result, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate>
ForwardIterator2 copy_if(ExecutionPolicy&& exec,
                        ForwardIterator1 first, ForwardIterator1 last,
                        ForwardIterator2 result, Predicate pred);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O>
constexpr ranges::copy_if_result<I, O>
    ranges::copy_if(I first, S last, O result, Pred pred, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::copy_if_result<borrowed_iterator_t<R>, O>
    ranges::copy_if(R&& r, O result, Pred pred, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O>
ranges::copy_if_result<I, O>
    ranges::copy_if(Ep&& exec, I first, S last, O result, OutS result_last,
                    Pred pred, Proj proj = {});

```

```

template<execution_policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred, Proj proj = {});
19     Let  $E(i)$  be:
(19.1)     — bool(pred(*i)) for the overloads in namespace std;
(19.2)     — bool(invoker(pred, invoker(proj, *i))) for the overloads in namespace ranges.
20     Let:
(20.1)     —  $M$  be the number of iterators  $i$  in the range  $[first, last)$  for which the condition  $E(i)$  holds;
(20.2)     — result_last be result + M for the overloads with no parameter result_last or result_r;
(20.3)     —  $N$  be  $\min(M, result\_last - result)$ .
21     Preconditions: The ranges  $[first, last)$  and  $[result, result + N)$  do not overlap.
    [Note 1: For the parallel algorithm overload in namespace std, there can be a performance cost if iterator_traits<ForwardIterator1>::value_type does not meet the Cpp17MoveConstructible (Table 31) requirements. For the parallel algorithm overloads in namespace ranges, there can be a performance cost if iter_value_t<I> does not model move_constructible. — end note]
22     Effects: Copies the first  $N$  elements referred to by the iterator  $i$  in the range  $[first, last)$  for which  $E(i)$  is true into the range  $[result, result + N)$ .
23     Returns:
(23.1)     — result + N for the overloads in namespace std.
(23.2)     —  $\{last, result + N\}$  for the overloads in namespace ranges, if  $N$  is equal to  $M$ .
(23.3)     — Otherwise,  $\{j, result\_last\}$  for the overloads in namespace ranges, where  $j$  is the iterator in  $[first, last)$  for which  $E(j)$  holds and there are exactly  $N$  iterators  $i$  in  $[first, j)$  for which  $E(i)$  holds.
24     Complexity: At most  $last - first$  applications of the corresponding predicate and any projection.
25     Remarks: Stable (16.4.6.8).

template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2
    copy_backward(BidirectionalIterator1 first,
                 BidirectionalIterator1 last,
                 BidirectionalIterator2 result);

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator I2>
requires indirectly_copyable<I1, I2>
constexpr ranges::copy_backward_result<I1, I2>
    ranges::copy_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
requires indirectly_copyable<iterator_t<R>, I>
constexpr ranges::copy_backward_result<borrowed_iterator_t<R>, I>
    ranges::copy_backward(R&& r, I result);
26     Let  $N$  be  $last - first$ .
27     Preconditions: result is not in the range  $(first, last]$ .
28     Effects: Copies elements in the range  $[first, last)$  into the range  $[result - N, result)$  starting from  $last - 1$  and proceeding to  $first$ .180 For each positive integer  $n \leq N$ , performs  $*(result - n) = *(last - n)$ .
29     Returns:
(29.1)     — result - N for the overload in namespace std.
(29.2)     —  $\{last, result - N\}$  for the overloads in namespace ranges.

```

¹⁸⁰ `copy_backward` can be used instead of `copy` when `last` is in the range $[result - N, result)$.

30 *Complexity*: Exactly N assignments.

26.7.2 Move

[alg.move]

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator move(InputIterator first, InputIterator last,
                             OutputIterator result);
```

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O>
requires indirectly_movable<I, O>
constexpr ranges::move_result<I, O>
    ranges::move(I first, S last, O result);
template<input_range R, weakly_incrementable O>
requires indirectly_movable<iterator_t<R>, O>
constexpr ranges::move_result<borrowed_iterator_t<R>, O>
    ranges::move(R&& r, O result);
```

- 1 Let $E(n)$ be
- (1.1) — `std::move(*(first + n))` for the overload in namespace `std`;
- (1.2) — `ranges::iter_move(first + n)` for the overloads in namespace `ranges`.

Let N be `last - first`.

- 2 *Preconditions*: `result` is not in the range `[first, last)`.
- 3 *Effects*: Moves elements in the range `[first, last)` into the range `[result, result + N)` starting from `first` and proceeding to `last`. For each non-negative integer $n < N$, performs $*(result + n) = E(n)$.

- 4 *Returns*:
- (4.1) — `result + N` for the overload in namespace `std`.
- (4.2) — `{last, result + N}` for the overloads in namespace `ranges`.

5 *Complexity*: Exactly N assignments.

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2 move(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last,
                    ForwardIterator2 result);
```

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_movable<I, O>
ranges::move_result<I, O>
    ranges::move(Ep&& exec, I first, S last, O result, OutS result_last);
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_movable<iterator_t<R>, iterator_t<OutR>>
ranges::move_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::move(Ep&& exec, R&& r, OutR&& result_r);
```

- 6 Let $E(n)$ be:
- (6.1) — `std::move(*(first + n))` for the overload in namespace `std`;
- (6.2) — `ranges::iter_move(first + n)` for the overloads in namespace `ranges`.

7 Let `result_last` be `result + (last - first)` for the overloads in namespace `std`.

8 Let N be `min(last - first, result_last - result)`.

9 *Preconditions*: The ranges `[first, last)` and `[result, result + N)` do not overlap.

- 10 *Effects*: Moves elements in the range `[first, first + N)` into the range `[result, result + N)`. For each non-negative integer $n < N$, performs $*(result + n) = E(n)$.

- 11 *Returns*:
- (11.1) — `result + N` for the overload in namespace `std`.
- (11.2) — `{first + N, result + N}` for the overloads in namespace `ranges`.

12 *Complexity*: Exactly N assignments.

```
template<class BidirectionalIterator1, class BidirectionalIterator2>
constexpr BidirectionalIterator2
move_backward(BidirectionalIterator1 first, BidirectionalIterator1 last,
              BidirectionalIterator2 result);

template<bidirectional_iterator I1, sentinel_for<I1> S1, bidirectional_iterator I2>
requires indirectly_movable<I1, I2>
constexpr ranges::move_backward_result<I1, I2>
ranges::move_backward(I1 first, S1 last, I2 result);
template<bidirectional_range R, bidirectional_iterator I>
requires indirectly_movable<iterator_t<R>, I>
constexpr ranges::move_backward_result<borrowed_iterator_t<R>, I>
ranges::move_backward(R&& r, I result);
```

13 Let $E(n)$ be

(13.1) — `std::move(*(last - n))` for the overload in namespace `std`;

(13.2) — `ranges::iter_move(last - n)` for the overloads in namespace `ranges`.

Let N be `last - first`.

14 *Preconditions*: `result` is not in the range $[first, last]$.

15 *Effects*: Moves elements in the range $[first, last)$ into the range $[result - N, result)$ starting from `last - 1` and proceeding to `first`.¹⁸¹ For each positive integer $n \leq N$, performs $*(result - n) = E(n)$.

16 *Returns*:

(16.1) — `result - N` for the overload in namespace `std`.

(16.2) — $\{last, result - N\}$ for the overloads in namespace `ranges`.

17 *Complexity*: Exactly N assignments.

26.7.3 Swap

[alg.swap]

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator2
swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
swap_ranges(ExecutionPolicy&& exec,
            ForwardIterator1 first1, ForwardIterator1 last1,
            ForwardIterator2 first2);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2>
requires indirectly_swappable<I1, I2>
constexpr ranges::swap_ranges_result<I1, I2>
ranges::swap_ranges(I1 first1, S1 last1, I2 first2, S2 last2);
template<input_range R1, input_range R2>
requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
constexpr ranges::swap_ranges_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
ranges::swap_ranges(R1&& r1, R2&& r2);

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2>
requires indirectly_swappable<I1, I2>
ranges::swap_ranges_result<I1, I2>
ranges::swap_ranges(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2);
```

¹⁸¹) `move_backward` can be used instead of `move` when `last` is in the range $[result - N, result)$.

```
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2>
requires indirectly_swappable<iterator_t<R1>, iterator_t<R2>>
ranges::swap_ranges_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
ranges::swap_ranges(Ep&& exec, R1&& r1, R2&& r2);
```

- 1 Let:
- (1.1) — last2 be first2 + (last1 - first1) for the overloads in namespace `std` with no parameter named last2;
- (1.2) — M be $\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$.
- 2 *Preconditions:* The two ranges $[\text{first1}, \text{last1})$ and $[\text{first2}, \text{last2})$ do not overlap. For the overloads in namespace `std`, $\text{*(first1 + } n\text{)}$ is swappable with (16.4.4.3) $\text{*(first2 + } n\text{)}$.
- 3 *Effects:* For each non-negative integer $n < M$ performs:
- (3.1) — `swap(*(first1 + n), *(first2 + n))` for the overloads in namespace `std`;
- (3.2) — `ranges::iter_swap(first1 + n, first2 + n)` for the overloads in namespace `ranges`.
- 4 *Returns:*
- (4.1) — last2 for the overloads in namespace `std`.
- (4.2) — $\{\text{first1} + M, \text{first2} + M\}$ for the overloads in namespace `ranges`.
- 5 *Complexity:* Exactly M swaps.

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr void iter_swap(ForwardIterator1 a, ForwardIterator2 b);
```

6 *Preconditions:* `a` and `b` are dereferenceable. *a is swappable with (16.4.4.3) *b .

7 *Effects:* As if by `swap(*a, *b)`.

26.7.4 Transform

[alg.transform]

```
template<class InputIterator, class OutputIterator,
class UnaryOperation>
constexpr OutputIterator
transform(InputIterator first1, InputIterator last1,
OutputIterator result, UnaryOperation op);
```

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
class UnaryOperation>
ForwardIterator2
transform(ExecutionPolicy&& exec,
ForwardIterator1 first1, ForwardIterator1 last1,
ForwardIterator2 result, UnaryOperation op);
```

```
template<class InputIterator1, class InputIterator2,
class OutputIterator, class BinaryOperation>
constexpr OutputIterator
transform(InputIterator1 first1, InputIterator1 last1,
InputIterator2 first2, OutputIterator result,
BinaryOperation binary_op);
```

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
class ForwardIterator, class BinaryOperation>
ForwardIterator
transform(ExecutionPolicy&& exec,
ForwardIterator1 first1, ForwardIterator1 last1,
ForwardIterator2 first2, ForwardIterator result,
BinaryOperation binary_op);
```

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
copy_constructible F, class Proj = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I, Proj>>>
constexpr ranges::unary_transform_result<I, O>
ranges::transform(I first1, S last1, O result, F op, Proj proj = {});
```

```

template<input_range R, weakly_incrementable O, copy_constructible F,
        class Proj = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
constexpr ranges::unary_transform_result<borrowed_iterator_t<R>, O>
ranges::transform(R&& r, O result, F op, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        copy_constructible F, class Proj = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I, Proj>>>
ranges::unary_transform_result<I, O>
ranges::transform(Ep&& exec, I first1, S last1, O result, OutS result_last,
        F op, Proj proj = {});

template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR,
        copy_constructible F, class Proj = identity>
requires indirectly_writable<iterator_t<OutR>,
        indirect_result_t<F&, projected<iterator_t<R>, Proj>>>
ranges::unary_transform_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
ranges::transform(Ep&& exec, R&& r, OutR&& result_r, F op, Proj proj = {});

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, copy_constructible F, class Proj1 = identity,
        class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
constexpr ranges::binary_transform_result<I1, I2, O>
ranges::transform(I1 first1, S1 last1, I2 first2, S2 last2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incrementable O,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
constexpr ranges::binary_transform_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
ranges::transform(R1&& r1, R2&& r2, O result,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        copy_constructible F, class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<O, indirect_result_t<F&, projected<I1, Proj1>,
        projected<I2, Proj2>>>
ranges::binary_transform_result<I1, I2, O>
ranges::transform(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
        O result, OutS result_last,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        sized_random_access_range OutR, copy_constructible F,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_writable<iterator_t<OutR>,
        indirect_result_t<F&, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>>
ranges::binary_transform_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
        borrowed_iterator_t<OutR>>
ranges::transform(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
        F binary_op, Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — $last2$ be $first2 + (last1 - first1)$ for the overloads in namespace `std` with parameter $first2$ but no parameter $last2$;
- (1.2) — M be $last1 - first1$ for unary transforms, or $\min(last1 - first1, last2 - first2)$ for binary transforms;
- (1.3) — $result_last$ be $result + M$ for the overloads with no parameter $result_last$ or $result_r$;

- (1.4) — N be $\min(M, \text{result_last} - \text{result})$;
- (1.5) — $E(i)$ be
 - (1.5.1) — $\text{op}(*(\text{first1} + (i - \text{result})))$ for unary transforms defined in namespace `std`;
 - (1.5.2) — $\text{binary_op}(*(\text{first1} + (i - \text{result})), *(\text{first2} + (i - \text{result})))$ for binary transforms defined in namespace `std`;
 - (1.5.3) — $\text{invoke}(\text{op}, \text{invoke}(\text{proj}, *(\text{first1} + (i - \text{result}))))$ for unary transforms defined in namespace `ranges`;
 - (1.5.4) — $\text{invoke}(\text{binary_op}, \text{invoke}(\text{proj1}, *(\text{first1} + (i - \text{result}))), \text{invoke}(\text{proj2}, *(\text{first2} + (i - \text{result}))))$ for binary transforms defined in namespace `ranges`.
- 2 *Preconditions:* For parallel algorithm overloads `op` and `binary_op` satisfy the requirements specified in 26.3.2. `op` and `binary_op` do not invalidate iterators or subranges, nor modify elements in the ranges
 - (2.1) — $[\text{first1}, \text{first1} + N]$,
 - (2.2) — $[\text{first2}, \text{first2} + N]$, and
 - (2.3) — $[\text{result}, \text{result} + N]$.¹⁸²
- 3 *Effects:* Assigns through every iterator i in the range $[\text{result}, \text{result} + N)$ a new corresponding value equal to $E(i)$.
- 4 *Returns:*
 - (4.1) — $\text{result} + N$ for the overloads defined in namespace `std`.
 - (4.2) — $\{\text{first1} + N, \text{result} + N\}$ for unary transforms defined in namespace `ranges`.
 - (4.3) — $\{\text{first1} + N, \text{first2} + N, \text{result} + N\}$ for binary transforms defined in namespace `ranges`.
- 5 *Complexity:* Exactly N applications of `op` or `binary_op`, and any projections. This requirement also applies to the parallel algorithm overloads.
- 6 *Remarks:* `result` may be equal to `first1` or `first2`.

26.7.5 Replace

[alg.replace]

```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr void replace(ForwardIterator first, ForwardIterator last,
                      const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator,
         class T = iterator_traits<ForwardIterator>::value_type>
void replace(ExecutionPolicy&& exec,
            ForwardIterator first, ForwardIterator last,
            const T& old_value, const T& new_value);

template<class ForwardIterator, class Predicate,
         class T = iterator_traits<ForwardIterator>::value_type>
constexpr void replace_if(ForwardIterator first, ForwardIterator last,
                        Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class Predicate,
         class T = iterator_traits<ForwardIterator>::value_type>
void replace_if(ExecutionPolicy&& exec,
                ForwardIterator first, ForwardIterator last,
                Predicate pred, const T& new_value);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
         class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<I>>
requires indirectly_writable<I, const T2&> &&
         indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
constexpr I
ranges::replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
```

¹⁸²⁾ The use of fully closed ranges is intentional.


```

template<input_range R, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = range_value_t<R>>
requires indirectly_writable<iterator_t<R>, const T2&> &&
        indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
constexpr borrowed_iterator_t<R>
        ranges::replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<I>>
requires indirectly_writable<I, const T2&> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
I ranges::replace(Ep&& exec, I first, S last,
        const T1& old_value, const T2& new_value, Proj proj = {});

template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = range_value_t<R>>
requires indirectly_writable<iterator_t<R>, const T2&> &&
        indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T1*>
        borrowed_iterator_t<R>
        ranges::replace(Ep&& exec, R&& r, const T1& old_value, const T2& new_value,
        Proj proj = {});

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = iter_value_t<I>,
        indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_writable<I, const T&>
constexpr I ranges::replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});

template<input_range R, class Proj = identity, class T = range_value_t<R>,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_writable<iterator_t<R>, const T&>
constexpr borrowed_iterator_t<R>
        ranges::replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, class T = iter_value_t<I>,
        indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_writable<I, const T&>
I ranges::replace_if(Ep&& exec, I first, S last, Pred pred,
        const T& new_value, Proj proj = {});

template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        class T = range_value_t<R>,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_writable<iterator_t<R>, const T&>
        borrowed_iterator_t<R>
        ranges::replace_if(Ep&& exec, R&& r, Pred pred, const T& new_value, Proj proj = {});

```

- 1 Let $E(i)$ be
 - (1.1) — $\text{bool}(*i == \text{old_value})$ for `replace`;
 - (1.2) — $\text{bool}(\text{pred}(*i))$ for `replace_if`;
 - (1.3) — $\text{bool}(\text{invoke}(\text{proj}, *i) == \text{old_value})$ for `ranges::replace`;
 - (1.4) — $\text{bool}(\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i)))$ for `ranges::replace_if`.
- 2 *Mandates:* `new_value` is writable (24.3.1) to `first`.
- 3 *Effects:* Substitutes elements referred by the iterator `i` in the range $[\text{first}, \text{last})$ with `new_value`, when $E(i)$ is true.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* Exactly `last - first` applications of the corresponding predicate and any projection.

```

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator
    replace_copy(InputIterator first, InputIterator last,
                  OutputIterator result,
                  const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2
    replace_copy(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result,
                  const T& old_value, const T& new_value);

template<class InputIterator, class OutputIterator, class Predicate, class T>
constexpr OutputIterator
    replace_copy_if(InputIterator first, InputIterator last,
                    OutputIterator result,
                    Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate, class T>
ForwardIterator2
    replace_copy_if(ExecutionPolicy&& exec,
                    ForwardIterator1 first, ForwardIterator1 last,
                    ForwardIterator2 result,
                    Predicate pred, const T& new_value);

template<input_iterator I, sentinel_for<I> S, class O,
        class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<O>>
requires indirectly_copyable<I, O> &&
    indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
    output_iterator<O, const T2&>
constexpr ranges::replace_copy_result<I, O>
    ranges::replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
                          Proj proj = {});
template<input_range R, class O, class Proj = identity,
        class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = iter_value_t<O>>
requires indirectly_copyable<iterator_t<R>, O> &&
    indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T1*>
    && output_iterator<O, const T2&>
constexpr ranges::replace_copy_result<borrowed_iterator_t<R>, O>
    ranges::replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                          Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        class Proj = identity,
        class T1 = projected_value_t<I, Proj>, class T2 = iter_value_t<O>>
requires indirectly_copyable<I, O> &&
    indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
    indirectly_writable<O, const T2&>
ranges::replace_copy_result<I, O>
    ranges::replace_copy(Ep&& exec, I first, S last, O result, OutS result_last,
                          const T1& old_value, const T2& new_value, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class Proj = identity, class T1 = projected_value_t<iterator_t<R>, Proj>,
        class T2 = range_value_t<OutR>>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
    indirect_binary_predicate<ranges::equal_to,
                             projected<iterator_t<R>, Proj>, const T1*> &&
    indirectly_writable<iterator_t<OutR>, const T2&>
ranges::replace_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::replace_copy(Ep&& exec, R&& r, OutR&& result_r, const T1& old_value,
                          const T2& new_value, Proj proj = {});

```

```

template<input_iterator I, sentinel_for<I> S, class O, class T = iter_value_t<O>,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O> && output_iterator<O, const T&>
constexpr ranges::replace_copy_if_result<I, O>
    ranges::replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                           Proj proj = {});
template<input_range R, class O, class T = iter_value_t<O>, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O> && output_iterator<O, const T&>
constexpr ranges::replace_copy_if_result<borrowed_iterator_t<R>, O>
    ranges::replace_copy_if(R&& r, O result, Pred pred, const T& new_value,
                           Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS, class T = iter_value_t<O>,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O> && indirectly_writable<O, const T&>
ranges::replace_copy_if_result<I, O>
    ranges::replace_copy_if(Ep&& exec, I first, S last, O result, OutS result_last,
                           Pred pred, const T& new_value, Proj proj = {});
template<execution_policy Ep, sized-random-access-range R, sized-random-access-range OutR,
        class T = range_value_t<OutR>, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
        indirectly_writable<OutR, const T&>
ranges::replace_copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::replace_copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred, const T& new_value,
                           Proj proj = {});

```

6 Let $E(i)$ be

- (6.1) — `bool(*(first + (i - result)) == old_value)` for `replace_copy`;
- (6.2) — `bool(pred(*(first + (i - result))))` for `replace_copy_if`;
- (6.3) — `bool(invoker(proj, *(first + (i - result))) == old_value)` for `ranges::replace_copy`;
- (6.4) — `bool(invoker(pred, invoker(proj, *(first + (i - result))))` for `ranges::replace_copy_if`.

7 Let:

- (7.1) — `result_last` be `result + (last - first)` for the overloads with no parameter `result_last` or `result_r`;
- (7.2) — `N` be `min(last - first, result_last - result)`.

8 *Mandates:* The results of the expressions `*first` and `new_value` are writable (24.3.1) to `result`.

9 *Preconditions:* The ranges `[first, last)` and `[result, result + N)` do not overlap.

10 *Effects:* Assigns through every iterator `i` in the range `[result, result + N)` a new corresponding value

- (10.1) — `new_value` if $E(i)$ is true or
- (10.2) — `*(first + (i - result))` otherwise.

11 *Returns:*

- (11.1) — `result + N` for the overloads in namespace `std`.
- (11.2) — `{first + N, result + N}` for the overloads in namespace `ranges`.

12 *Complexity:* Exactly N applications of the corresponding predicate and any projection.

26.7.6 Fill

[alg.fill]

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
        class T = iterator_traits<ForwardIterator>::value_type>
void fill(ExecutionPolicy&& exec,
         ForwardIterator first, ForwardIterator last, const T& value);

```

```

template<class OutputIterator, class Size, class T = iterator_traits<OutputIterator>::value_type>
constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator fill_n(ExecutionPolicy&& exec,
                      ForwardIterator first, Size n, const T& value);

template<class O, sentinel_for<O> S, class T = iter_value_t<O>>
requires output_iterator<O, const T&>
constexpr O ranges::fill(O first, S last, const T& value);
template<class R, class T = range_value_t<R>>
requires output_range<R, const T&>
constexpr borrowed_iterator_t<R> ranges::fill(R&& r, const T& value);
template<class O, class T = iter_value_t<O>>
requires output_iterator<O, const T&>
constexpr O ranges::fill_n(O first, iter_difference_t<O> n, const T& value);

template<execution_policy Ep, random_access_iterator O, sized_sentinel_for<O> S,
        class T = iter_value_t<O>>
requires indirectly_writable<O, const T&>
O ranges::fill(Ep&& exec, O first, S last, const T& value);
template<execution_policy Ep, sized_random_access_range R, class T = range_value_t<R>>
requires indirectly_writable<iterator_t<R>, const T&>
borrowed_iterator_t<R> fill(Ep&& exec, R&& r, const T& value);
template<execution_policy Ep, random_access_iterator O, class T = iter_value_t<O>>
requires indirectly_writable<O, const T&>
O ranges::fill_n(Ep&& exec, O first, iter_difference_t<O> n, const T& value);

```

- 1 Let N be $\max(0, n)$ for the `fill_n` algorithms, and `last - first` for the `fill` algorithms.
- 2 *Mandates:* The expression `value` is writable (24.3.1) to the output iterator. The type `Size` is convertible to an integral type (7.3.9, 11.4.8).
- 3 *Effects:* Assigns `value` through all the iterators in the range `[first, first + N)`.
- 4 *Returns:* `first + N`.
- 5 *Complexity:* Exactly N assignments.

26.7.7 Generate

[alg.generate]

```

template<class ForwardIterator, class Generator>
constexpr void generate(ForwardIterator first, ForwardIterator last,
                      Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Generator>
void generate(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             Generator gen);

template<class OutputIterator, class Size, class Generator>
constexpr OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
template<class ExecutionPolicy, class ForwardIterator, class Size, class Generator>
ForwardIterator generate_n(ExecutionPolicy&& exec,
                          ForwardIterator first, Size n, Generator gen);

template<input_or_output_iterator O, sentinel_for<O> S, copy_constructible F>
requires invocable<F&&> && indirectly_writable<O, invoke_result_t<F&&>>
constexpr O ranges::generate(O first, S last, F gen);
template<class R, copy_constructible F>
requires invocable<F&&> && output_range<R, invoke_result_t<F&&>>
constexpr borrowed_iterator_t<R> ranges::generate(R&& r, F gen);
template<input_or_output_iterator O, copy_constructible F>
requires invocable<F&&> && indirectly_writable<O, invoke_result_t<F&&>>
constexpr O ranges::generate_n(O first, iter_difference_t<O> n, F gen);

```

- 1 Let N be $\max(0, n)$ for the `generate_n` algorithms, and `last - first` for the `generate` algorithms.
- 2 *Mandates:* `Size` is convertible to an integral type (7.3.9, 11.4.8).

- 3 *Effects:* Assigns the result of successive evaluations of `gen()` through each iterator in the range `[first, first + N)`.
- 4 *Returns:* `first + N`.
- 5 *Complexity:* Exactly N evaluations of `gen()` and assignments.
- 6 *Remarks:* `gen` may modify objects via its arguments for parallel algorithm overloads (26.3.2).

26.7.8 Remove

[alg.remove]

```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                                const T& value);
template<class ExecutionPolicy, class ForwardIterator,
         class T = iterator_traits<ForwardIterator>::value_type>
ForwardIterator remove(ExecutionPolicy&& exec,
                      ForwardIterator first, ForwardIterator last,
                      const T& value);

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                                   Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator remove_if(ExecutionPolicy&& exec,
                        ForwardIterator first, ForwardIterator last,
                        Predicate pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
         class T = projected_value_t<I, Proj>>
requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
constexpr subrange<I> ranges::remove(I first, S last, const T& value, Proj proj = {});
template<forward_range R, class Proj = identity,
         class T = projected_value_t<iterator_t<R>, Proj>>
requires permutable<iterator_t<R>> &&
indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr borrowed_subrange_t<R>
ranges::remove(R&& r, const T& value, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity, class T = projected_value_t<I, Proj>>
requires permutable<I> &&
indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
subrange<I>
ranges::remove(Ep&& exec, I first, S last, const T& value, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         class T = projected_value_t<iterator_t<R>, Proj>>
requires permutable<iterator_t<R>> &&
indirect_binary_predicate<ranges::equal_to,
                        projected<iterator_t<R>, Proj>, const T*>
borrowed_subrange_t<R>
ranges::remove(Ep&& exec, R&& r, const T& value, Proj proj = {});

template<permutable I, sentinel_for<I> S, class Proj = identity,
         indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr subrange<I> ranges::remove_if(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
ranges::remove_if(R&& r, Pred pred, Proj proj = {});
```

```

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires permutable<I>
subrange<I>
    ranges::remove_if(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R>
    ranges::remove_if(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

1 Let $E(i)$ be

- (1.1) — `bool(*i == value)` for `remove`;
- (1.2) — `bool(pred(*i))` for `remove_if`;
- (1.3) — `bool(invoke(proj, *i) == value)` for `ranges::remove`;
- (1.4) — `bool(invoke(pred, invoke(proj, *i)))` for `ranges::remove_if`.

2 *Preconditions:* For the algorithms in namespace `std`, the type of `*first` meets the *Cpp17MoveAssignable* requirements (Table 33).

3 *Effects:* Eliminates all the elements referred to by iterator `i` in the range `[first, last)` for which $E(i)$ holds.

4 *Returns:* Let j be the end of the resulting range. Returns:

- (4.1) — j for the overloads in namespace `std`.
- (4.2) — `{j, last}` for the overloads in namespace `ranges`.

5 *Complexity:* Exactly `last - first` applications of the corresponding predicate and any projection.

6 *Remarks:* Stable (16.4.6.8).

7 [Note 1: Each element in the range `[ret, last)`, where `ret` is the returned value, has a valid but unspecified state, because the algorithms can eliminate elements by moving from elements that were originally in that range. — end note]

```

template<class InputIterator, class OutputIterator,
        class T = iterator_traits<InputIterator>::value_type>
constexpr OutputIterator
    remove_copy(InputIterator first, InputIterator last,
                OutputIterator result, const T& value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class T = iterator_traits<ForwardIterator1>::value_type>
ForwardIterator2
    remove_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate>
constexpr OutputIterator
    remove_copy_if(InputIterator first, InputIterator last,
                   OutputIterator result, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Predicate>
ForwardIterator2
    remove_copy_if(ExecutionPolicy&& exec,
                   ForwardIterator1 first, ForwardIterator1 last,
                   ForwardIterator2 result, Predicate pred);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
        class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirectly_copyable<I, O> &&
    indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
constexpr ranges::remove_copy_result<I, O>
    ranges::remove_copy(I first, S last, O result, const T& value, Proj proj = {});

```

```
template<input_range R, weakly_incrementable O, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_copyable<iterator_t<R>, O> &&
        indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, const T*>
constexpr ranges::remove_copy_result<borrowed_iterator_t<R>, O>
        ranges::remove_copy(R&& r, O result, const T& value, Proj proj = {});
```

```
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        class Proj = identity, class T = projected_value_t<I, Proj>>
requires indirectly_copyable<I, O> &&
        indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
ranges::remove_copy_result<I, O>
        ranges::remove_copy(Ep&& exec, I first, S last, O result, OutS result_last,
                            const T& value, Proj proj = {});
```

```
template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR,
        class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>> &&
        indirect_binary_predicate<ranges::equal_to,
                                projected<iterator_t<R>, Proj>, const T*>
ranges::remove_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
        ranges::remove_copy(Ep&& exec, R&& r, OutR&& result_r, const T& value, Proj proj = {});
```

```
template<input_iterator I, sentinel_for<I> S, weakly_incrementable O,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O>
constexpr ranges::remove_copy_if_result<I, O>
        ranges::remove_copy_if(I first, S last, O result, Pred pred, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::remove_copy_if_result<borrowed_iterator_t<R>, O>
        ranges::remove_copy_if(R&& r, O result, Pred pred, Proj proj = {});
```

```
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O>
ranges::remove_copy_if_result<I, O>
        ranges::remove_copy_if(Ep&& exec, I first, S last, O result, OutS result_last,
                                Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::remove_copy_if_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
        ranges::remove_copy_if(Ep&& exec, R&& r, OutR&& result_r, Pred pred, Proj proj = {});
```

8 Let $E(i)$ be

- (8.1) — $\text{bool}(*i == \text{value})$ for `remove_copy`;
- (8.2) — $\text{bool}(\text{pred}(*i))$ for `remove_copy_if`;
- (8.3) — $\text{bool}(\text{invoke}(\text{proj}, *i) == \text{value})$ for `ranges::remove_copy`;
- (8.4) — $\text{bool}(\text{invoke}(\text{pred}, \text{invoke}(\text{proj}, *i)))$ for `ranges::remove_copy_if`.

9 Let:

- (9.1) — M be the number of iterators i in $[\text{first}, \text{last})$ for which $E(i)$ is false;
- (9.2) — result_last be $\text{result} + M$ for the overloads with no parameter result_last or result_r ;
- (9.3) — N be $\min(M, \text{result_last} - \text{result})$.

10 *Mandates:* $*\text{first}$ is writable (24.3.1) to result .

11 *Preconditions:* The ranges $[\text{first}, \text{last})$ and $[\text{result}, \text{result} + N)$ do not overlap.

[Note 2: For the parallel algorithm overloads in namespace `std`, there can be a performance cost if `iterator_traits<ForwardIterator1>::value_type` does not meet the *Cpp17MoveConstructible* (Table 31) requirements. For the parallel algorithm overloads in namespace `ranges`, there can be a performance cost if `iter_value_t<I>` does not model `move_constructible`. — end note]

Effects: Copies the first N elements referred to by the iterator i in the range $[first, last)$ for which $E(i)$ is false into the range $[result, result + N)$.

Returns:

— `result + N`, for the algorithms in namespace `std`.

— $\{last, result + N\}$, for the algorithms in namespace `ranges`, if N is equal to M .

— Otherwise, $\{j, result_last\}$, for the algorithms in namespace `ranges`, where j is the iterator in $[first, last)$ for which $E(j)$ is false and there are exactly N iterators i in $[first, j)$ for which $E(i)$ is false.

Complexity: At most $last - first$ applications of the corresponding predicate and any projection.

Remarks: Stable (16.4.6.8).

26.7.9 Unique

[alg.unique]

```
template<class ForwardIterator>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator unique(ExecutionPolicy&& exec,
                      ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
constexpr ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                                BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class BinaryPredicate>
ForwardIterator unique(ExecutionPolicy&& exec,
                      ForwardIterator first, ForwardIterator last,
                      BinaryPredicate pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
constexpr subrange<I> ranges::unique(I first, S last, C comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
ranges::unique(R&& r, C comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires permutable<I>
subrange<I> ranges::unique(Ep&& exec, I first, S last, C comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R> ranges::unique(Ep&& exec, R&& r, C comp = {}, Proj proj = {});
```

Let `pred` be `equal_to{}` for the overloads with no parameter `pred`, and let $E(i)$ be

— false if i is equal to `first`; otherwise

— `bool(pred(*(i - 1), *i))` for the overloads in namespace `std`;

— `bool(invoker(comp, invoker(proj, *(i - 1)), invoker(proj, *i)))` for the overloads in namespace `ranges`.

Preconditions: For the overloads in namespace `std`, `pred` is an equivalence relation and the type of `*first` meets the *Cpp17MoveAssignable* requirements (Table 33).

3 *Effects:* Eliminates all elements referred to by the iterator *i* in the range `[first, last)` for which *E*(*i*) is `true`.

4 *Returns:* Let *j* be the end of the resulting range. Returns:

(4.1) — *j* for the overloads in namespace `std`.

(4.2) — `{j, last}` for the overloads in namespace `ranges`.

5 *Complexity:* For nonempty ranges, exactly `(last - first) - 1` applications of the corresponding predicate and no more than twice as many applications of any projection.

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    unique_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result);

template<class InputIterator, class OutputIterator,
         class BinaryPredicate>
constexpr OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
ForwardIterator2
    unique_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, BinaryPredicate pred);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires indirectly_copyable<I, O> &&
    (forward_iterator<I> ||
     (input_iterator<O> && same_as<iter_value_t<I>, iter_value_t<O>>) ||
     indirectly_copyable_storable<I, O>)
constexpr ranges::unique_copy_result<I, O>
    ranges::unique_copy(I first, S last, O result, C comp = {}, Proj proj = {});
template<input_range R, weakly_incrementable O, class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires indirectly_copyable<iterator_t<R>, O> &&
    (forward_iterator<iterator_t<R>> ||
     (input_iterator<O> && same_as<range_value_t<R>, iter_value_t<O>>) ||
     indirectly_copyable_storable<iterator_t<R>, O>)
constexpr ranges::unique_copy_result<borrowed_iterator_t<R>, O>
    ranges::unique_copy(R&& r, O result, C comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Proj = identity,
        indirect_equivalence_relation<projected<I, Proj>> C = ranges::equal_to>
requires indirectly_copyable<I, O>
ranges::unique_copy_result<I, O>
    ranges::unique_copy(Ep&& exec, I first, S last, O result, OutS result_last,
                        C comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, sized_random_access_range OutR,
        class Proj = identity,
        indirect_equivalence_relation<projected<iterator_t<R>, Proj>> C = ranges::equal_to>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::unique_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::unique_copy(Ep&& exec, R&& r, OutR&& result_r, C comp = {}, Proj proj = {});
```

6 Let `pred` be `equal_to{}` for the overloads in namespace `std` with no parameter `pred`, and let *E*(*i*) be

- (6.1) — `false` if `i` is equal to `first`; otherwise
- (6.2) — `bool(pred(*(i - 1), *i))` for the overloads in namespace `std`;
- (6.3) — `bool(invoker(comp, invoker(proj, *(i - 1)), invoker(proj, *i)))` for the overloads in namespace `ranges`.

7 Let:

- (7.1) — `M` be the number of iterators `i` in the range `[first, last)` for which `E(i)` is `false`;
- (7.2) — `result_last` be `result + M` for the overloads with no parameter `result_last` or `result_r`;
- (7.3) — `N` be `min(M, result_last - result)`.

8 *Mandates:* `*first` is writable (24.3.1) to `result`.

9 *Preconditions:*

- (9.1) — The ranges `[first, last)` and `[result, result + N)` do not overlap.
- (9.2) — For the overloads in namespace `std`:
 - (9.2.1) — The comparison function is an equivalence relation.
 - (9.2.2) — For the overloads with no `ExecutionPolicy`, let `T` be the value type of `InputIterator`. If `InputIterator` models `forward_iterator` (24.3.4.11), then there are no additional requirements for `T`. Otherwise, if `OutputIterator` meets the *Cpp17ForwardIterator* requirements and its value type is the same as `T`, then `T` meets the *Cpp17CopyAssignable* (Table 34) requirements. Otherwise, `T` meets both the *Cpp17CopyConstructible* (Table 32) and *Cpp17CopyAssignable* requirements.

[*Note 1:* For the parallel algorithm overloads in namespace `std`, there can be a performance cost if the value type of `ForwardIterator1` does not meet both the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements. For the parallel algorithm overloads in namespace `ranges`, there can be a performance cost if `iter_value_t<I>` does not model `copyable`. — *end note*]

10 *Effects:* Copies only the first `N` elements referred to by the iterator `i` in the range `[first, last)` for which `E(i)` is `false` into the range `[result, result + N)`.

11 *Returns:*

- (11.1) — `result + N` for the overloads in namespace `std`.
- (11.2) — `{last, result + N}` for the overloads in namespace `ranges`, if `N` is equal to `M`.
- (11.3) — Otherwise, `{j, result_last}` for the overloads in namespace `ranges`, where `j` is the iterator in `[first, last)` for which `E(j)` is `false` and there are exactly `N` iterators `i` in `[first, j)` for which `E(i)` is `false`.

12 *Complexity:* At most `last - first - 1` applications of the corresponding predicate and no more than twice as many applications of any projection.

26.7.10 Reverse

[`alg.reverse`]

```
template<class BidirectionalIterator>
constexpr void reverse(BidirectionalIterator first, BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator>
void reverse(ExecutionPolicy&& exec,
             BidirectionalIterator first, BidirectionalIterator last);

template<bidirectional_iterator I, sentinel_for<I> S>
requires permutable<I>
constexpr I ranges::reverse(I first, S last);
template<bidirectional_range R>
requires permutable<iterator_t<R>>
constexpr borrowed_iterator_t<R> ranges::reverse(R&& r);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
requires permutable<I>
I ranges::reverse(Ep&& exec, I first, S last);
```

```
template<execution-policy Ep, sized-random-access-range R>
requires permutable<iterator_t<R>>
borrowed_iterator_t<R> ranges::reverse(Ep&& exec, R&& r);
```

1 *Preconditions:* For the overloads in namespace `std`, `BidirectionalIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3).

2 *Effects:* For each non-negative integer $i < (\text{last} - \text{first}) / 2$, applies `std::iter_swap`, or `ranges::iter_swap` for the overloads in namespace `ranges`, to all pairs of iterators $\text{first} + i$, $(\text{last} - i) - 1$.

3 *Returns:* `last` for the overloads in namespace `ranges`.

4 *Complexity:* Exactly $(\text{last} - \text{first})/2$ swaps.

```
template<class BidirectionalIterator, class OutputIterator>
constexpr OutputIterator
reverse_copy(BidirectionalIterator first, BidirectionalIterator last,
            OutputIterator result);
template<class ExecutionPolicy, class BidirectionalIterator, class ForwardIterator>
ForwardIterator
reverse_copy(ExecutionPolicy&& exec,
            BidirectionalIterator first, BidirectionalIterator last,
            ForwardIterator result);
```

```
template<bidirectional_iterator I, sentinel_for<I> S, weakly_incrementable O>
requires indirectly_copyable<I, O>
constexpr ranges::reverse_copy_result<I, O>
ranges::reverse_copy(I first, S last, O result);
template<bidirectional_range R, weakly_incrementable O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::reverse_copy_result<borrowed_iterator_t<R>, O>
ranges::reverse_copy(R&& r, O result);
```

5 Let N be $\text{last} - \text{first}$.

6 *Preconditions:* The ranges $[\text{first}, \text{last})$ and $[\text{result}, \text{result} + N)$ do not overlap.

7 *Effects:* Copies the range $[\text{first}, \text{last})$ to the range $[\text{result}, \text{result} + N)$ such that for every non-negative integer $i < N$ the following assignment takes place: $*(\text{result} + N - 1 - i) = *(\text{first} + i)$.

8 *Returns:*

(8.1) — `result + N` for the overloads in namespace `std`.

(8.2) — $\{\text{last}, \text{result} + N\}$ for the overloads in namespace `ranges`.

9 *Complexity:* Exactly N assignments.

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
ranges::reverse_copy_truncated_result<I, O>
ranges::reverse_copy(Ep&& exec, I first, S last, O result,
                    OutS result_last);
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
ranges::reverse_copy(Ep&& exec, R&& r, OutR&& result_r);
```

10 Let N be $\min(\text{last} - \text{first}, \text{result_last} - \text{result})$, and let *NEW_FIRST* be $\text{first} + (\text{last} - \text{first}) - N$.

11 *Preconditions:* The ranges $[\text{first}, \text{last})$ and $[\text{result}, \text{result} + N)$ do not overlap.

12 *Effects:* Copies the range $[\text{NEW_FIRST}, \text{last})$ to the range $[\text{result}, \text{result} + N)$ such that for every non-negative integer $i < N$ the following assignment takes place: $*(\text{result} + N - 1 - i) = *(\text{NEW_FIRST} + i)$.

13 *Returns:* $\{\text{last}, \text{NEW_FIRST}, \text{result} + N\}$.

14 *Complexity*: Exactly N assignments.

26.7.11 Rotate

[alg.rotate]

```
template<class ForwardIterator>
constexpr ForwardIterator
    rotate(ForwardIterator first, ForwardIterator middle, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    rotate(ExecutionPolicy&& exec,
           ForwardIterator first, ForwardIterator middle, ForwardIterator last);
```

```
template<permutable I, sentinel_for<I> S>
constexpr subrange<I> ranges::rotate(I first, I middle, S last);
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
requires permutable<I>
subrange<I> ranges::rotate(Ep&& exec, I first, I middle, S last);
```

1 *Preconditions*: $[first, middle)$ and $[middle, last)$ are valid ranges. For the overloads in namespace `std`, `ForwardIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3), and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.

2 *Effects*: For each non-negative integer $i < (last - first)$, places the element from the position `first + i` into position `first + (i + (last - middle)) % (last - first)`.

[Note 1: This is a left rotate. — end note]

3 *Returns*:

(3.1) — `first + (last - middle)` for the overloads in namespace `std`.

(3.2) — $\{first + (last - middle), last\}$ for the overload in namespace `ranges`.

4 *Complexity*: At most $last - first$ swaps.

```
template<forward_range R>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R> ranges::rotate(R&& r, iterator_t<R> middle);
```

5 *Effects*: Equivalent to: `return ranges::rotate(ranges::begin(r), middle, ranges::end(r));`

```
template<execution-policy Ep, sized-random-access-range R>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R> ranges::rotate(Ep&& exec, R&& r, iterator_t<R> middle);
```

6 *Effects*: Equivalent to:

```
return ranges::rotate(std::forward<Ep>(exec), ranges::begin(r), middle,
                    ranges::begin(r) + ranges::distance(r));
```

```
template<class ForwardIterator, class OutputIterator>
constexpr OutputIterator
    rotate_copy(ForwardIterator first, ForwardIterator middle, ForwardIterator last,
               OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    rotate_copy(ExecutionPolicy&& exec,
               ForwardIterator1 first, ForwardIterator1 middle, ForwardIterator1 last,
               ForwardIterator2 result);
```

```
template<forward_iterator I, sentinel_for<I> S, weakly_incrementable O>
requires indirectly_copyable<I, O>
constexpr ranges::rotate_copy_result<I, O>
    ranges::rotate_copy(I first, I middle, S last, O result);
```

7 Let N be $last - first$.

8 *Preconditions*: $[first, middle)$ and $[middle, last)$ are valid ranges. The ranges $[first, last)$ and $[result, result + N)$ do not overlap.

9 *Effects:* Copies the range `[first,last)` to the range `[result,result + N)` such that for each non-negative integer $i < N$ the following assignment takes place: `*(result + i) = *(first + (i + (middle - first)) % N)`.

10 *Returns:*

(10.1) — `result + N` for the overloads in namespace `std`.

(10.2) — `{last, result + N}` for the overload in namespace `ranges`.

11 *Complexity:* Exactly N assignments.

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O, sized_sentinel_for<O> OutS>
requires indirectly_copyable<I, O>
ranges::rotate_copy_truncated_result<I, O>
ranges::rotate_copy(Ep&& exec, I first, I middle, S last, O result, OutS result_last);
```

12 Let M be `last - first` and N be `min(M, result_last - result)`.

13 *Preconditions:* `[first,middle)` and `[middle,last)` are valid ranges. The ranges `[first,last)` and `[result,result + N)` do not overlap.

14 *Effects:* Copies the range `[first,last)` to the range `[result,result + N)` such that for each non-negative integer $i < N$ the following assignment takes place: `*(result + i) = *(first + (i + (middle - first)) % M)`.

15 *Returns:*

(15.1) — `{middle + N, first, result + N}` if N is less than `last - middle`.

(15.2) — Otherwise, `{last, first + (N + (middle - first)) % M, result + N}`.

16 *Complexity:* Exactly N assignments.

```
template<forward_range R, weakly_incrementable O>
requires indirectly_copyable<iterator_t<R>, O>
constexpr ranges::rotate_copy_result<borrowed_iterator_t<R>, O>
ranges::rotate_copy(R&& r, iterator_t<R> middle, O result);
```

17 *Effects:* Equivalent to:

```
return ranges::rotate_copy(ranges::begin(r), middle, ranges::end(r), std::move(result));
```

```
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);
```

18 *Effects:* Equivalent to:

```
return ranges::rotate_copy(std::forward<Ep>(exec), ranges::begin(r), middle,
    ranges::begin(r) + ranges::distance(r),
    ranges::begin(result_r),
    ranges::begin(result_r) + ranges::distance(result_r));
```

26.7.12 Sample

[alg.random.sample]

```
template<class PopulationIterator, class SampleIterator,
        class Distance, class UniformRandomBitGenerator>
SampleIterator sample(PopulationIterator first, PopulationIterator last,
    SampleIterator out, Distance n,
    UniformRandomBitGenerator&& g);

template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class Gen>
requires (forward_iterator<I> || random_access_iterator<O>) &&
    indirectly_copyable<I, O> &&
    uniform_random_bit_generator<remove_reference_t<Gen>>
O ranges::sample(I first, S last, O out, iter_difference_t<I> n, Gen&& g);
```

```
template<input_range R, weakly_incrementable O, class Gen>
requires (forward_range<R> || random_access_iterator<O>) &&
    indirectly_copyable<iterator_t<R>, O> &&
    uniform_random_bit_generator<remove_reference_t<Gen>>
O ranges::sample(R&& r, O out, range_difference_t<R> n, Gen&& g);
```

1 *Mandates:* For the overload in namespace `std`, `Distance` is an integer type and `*first` is writable (24.3.1) to `out`.

2 *Preconditions:* `out` is not in the range `[first, last)`. For the overload in namespace `std`:

- (2.1) — `PopulationIterator` meets the *Cpp17InputIterator* requirements (24.3.5.3).
- (2.2) — `SampleIterator` meets the *Cpp17OutputIterator* requirements (24.3.5.4).
- (2.3) — `SampleIterator` meets the *Cpp17RandomAccessIterator* requirements (24.3.5.7) unless `PopulationIterator` models *forward_iterator* (24.3.4.11).
- (2.4) — `remove_reference_t<UniformRandomBitGenerator>` meets the requirements of a uniform random bit generator type (29.5.3.3).

3 *Effects:* Copies `min(last - first, n)` elements (the *sample*) from `[first, last)` (the *population*) to `out` such that each possible sample has equal probability of appearance.

[Note 1: Algorithms that obtain such effects include *selection sampling* and *reservoir sampling*. — end note]

4 *Returns:* The end of the resulting sample range.

5 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$.

6 *Remarks:*

- (6.1) — For the overload in namespace `std`, stable if and only if `PopulationIterator` models *forward_iterator*. For the first overload in namespace `ranges`, stable if and only if `I` models *forward_iterator*.
- (6.2) — To the extent that the implementation of this function makes use of random numbers, the object `g` serves as the implementation's source of randomness.

26.7.13 Shuffle

[alg.random.shuffle]

```
template<class RandomAccessIterator, class UniformRandomBitGenerator>
void shuffle(RandomAccessIterator first,
            RandomAccessIterator last,
            UniformRandomBitGenerator&& g);
```

```
template<random_access_iterator I, sentinel_for<I> S, class Gen>
requires permutable<I> &&
    uniform_random_bit_generator<remove_reference_t<Gen>>
I ranges::shuffle(I first, S last, Gen&& g);
```

```
template<random_access_range R, class Gen>
requires permutable<iterator_t<R>> &&
    uniform_random_bit_generator<remove_reference_t<Gen>>
borrowed_iterator_t<R> ranges::shuffle(R&& r, Gen&& g);
```

1 *Preconditions:* For the overload in namespace `std`:

- (1.1) — `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3).
- (1.2) — The type `remove_reference_t<UniformRandomBitGenerator>` meets the uniform random bit generator (29.5.3.3) requirements.

2 *Effects:* Permutes the elements in the range `[first, last)` such that each possible permutation of those elements has equal probability of appearance.

3 *Returns:* `last` for the overloads in namespace `ranges`.

4 *Complexity:* Exactly `(last - first) - 1` swaps.

5 *Remarks:* To the extent that the implementation of this function makes use of random numbers, the object referenced by `g` shall serve as the implementation's source of randomness.

26.7.14 Shift

[alg.shift]

```

template<class ForwardIterator>
constexpr ForwardIterator
    shift_left(ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_left(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
               typename iterator_traits<ForwardIterator>::difference_type n);

template<permutable I, sentinel_for<I> S>
constexpr subrange<I> ranges::shift_left(I first, S last, iter_difference_t<I> n);
template<forward_range R>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R> ranges::shift_left(R&& r, range_difference_t<R> n);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
requires permutable<I>
subrange<I>
    ranges::shift_left(Ep&& exec, I first, S last, iter_difference_t<I> n);
template<execution-policy Ep, sized-random-access-range R>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R>
    ranges::shift_left(Ep&& exec, R&& r, range_difference_t<R> n);

```

- 1 *Preconditions:* $n \geq 0$ is true. For the overloads in namespace `std`, the type of `*first` meets the *Cpp17MoveAssignable* requirements.
- 2 *Effects:* If $n == 0$ or $n \geq \text{last} - \text{first}$, does nothing. Otherwise, moves the element from position $\text{first} + n + i$ into position $\text{first} + i$ for each non-negative integer $i < (\text{last} - \text{first}) - n$. For the non-parallel algorithm overloads, does so in order starting from $i = 0$ and proceeding to $i = (\text{last} - \text{first}) - n - 1$.
- 3 *Returns:* Let *NEW_LAST* be $\text{first} + (\text{last} - \text{first} - n)$ if $n < \text{last} - \text{first}$, otherwise `first`. Returns:
 - (3.1) — *NEW_LAST* for the overloads in namespace `std`.
 - (3.2) — `{first, NEW_LAST}` for the overloads in namespace `ranges`.
- 4 *Complexity:* At most $(\text{last} - \text{first}) - n$ assignments.

```

template<class ForwardIterator>
constexpr ForwardIterator
    shift_right(ForwardIterator first, ForwardIterator last,
                typename iterator_traits<ForwardIterator>::difference_type n);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
    shift_right(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last,
                typename iterator_traits<ForwardIterator>::difference_type n);

template<permutable I, sentinel_for<I> S>
constexpr subrange<I> ranges::shift_right(I first, S last, iter_difference_t<I> n);
template<forward_range R>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R> ranges::shift_right(R&& r, range_difference_t<R> n);

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S>
requires permutable<I>
subrange<I>
    ranges::shift_right(Ep&& exec, I first, S last, iter_difference_t<I> n);
template<execution-policy Ep, sized-random-access-range R>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R>
    ranges::shift_right(Ep&& exec, R&& r, range_difference_t<R> n);

```

- 5 *Preconditions:* $n \geq 0$ is true. For the overloads in namespace `std`, the type of `*first` meets

the *Cpp17MoveAssignable* requirements, and *ForwardIterator* meets the *Cpp17BidirectionalIterator* requirements (24.3.5.6) or the *Cpp17ValueSwappable* requirements.

- 6 *Effects:* If `n == 0` or `n >= last - first`, does nothing. Otherwise, moves the element from position `first + i` into position `first + n + i` for each non-negative integer `i < (last - first) - n`. Does so in order starting from `i = (last - first) - n - 1` and proceeding to `i = 0` if
- (6.1) — for the non-parallel algorithm overload in namespace `std`, *ForwardIterator* meets the *Cpp17BidirectionalIterator* requirements,
- (6.2) — for the non-parallel algorithm overloads in namespace `ranges`, `I` models `bidirectional_iterator`.
- 7 *Returns:* Let `NEW_FIRST` be `first + n` if `n < last - first`, otherwise `last`. Returns:
- (7.1) — `NEW_FIRST` for the overloads in namespace `std`.
- (7.2) — `{NEW_FIRST, last}` for the overloads in namespace `ranges`.
- 8 *Complexity:* At most `(last - first) - n` assignments or swaps.

26.8 Sorting and related operations

[alg.sorting]

26.8.1 General

[alg.sorting.general]

- 1 The operations in 26.8 defined directly in namespace `std` have two versions: one that takes a function object of type `Compare` and one that uses an `operator<`.
- 2 `Compare` is a function object type (22.10) that meets the requirements for a template parameter named `BinaryPredicate` (26.2). The return value of the function call operation applied to an object of type `Compare`, when converted to `bool`, yields `true` if the first argument of the call is less than the second, and `false` otherwise. `Compare comp` is used throughout for algorithms assuming an ordering relation.
- 3 For all algorithms that take `Compare`, there is a version that uses `operator<` instead. That is, `comp(*i, *j) != false` defaults to `*i < *j != false`. For algorithms other than those described in 26.8.4, `comp` shall induce a strict weak ordering on the values.
- 4 The term *strict* refers to the requirement of an irreflexive relation (`!comp(x, x)` for all `x`), and the term *weak* to requirements that are not as strong as those for a total ordering, but stronger than those for a partial ordering. If we define `equiv(a, b)` as `!comp(a, b) && !comp(b, a)`, then the requirements are that `comp` and `equiv` both be transitive relations:
- (4.1) — `comp(a, b) && comp(b, c)` implies `comp(a, c)`
- (4.2) — `equiv(a, b) && equiv(b, c)` implies `equiv(a, c)`
- [Note 1: Under these conditions, it can be shown that
- (4.3) — `equiv` is an equivalence relation,
- (4.4) — `comp` induces a well-defined relation on the equivalence classes determined by `equiv`, and
- (4.5) — the induced relation is a strict total ordering.
- end note]
- 5 A sequence is *sorted with respect to a comp and proj* for a comparator and projection `comp` and `proj` if for every iterator `i` pointing to the sequence and every non-negative integer `n` such that `i + n` is a valid iterator pointing to an element of the sequence,
- `bool(invoker(comp, invoker(proj, *(i + n)), invoker(proj, *i)))`
- is `false`.
- 6 A sequence is *sorted with respect to a comparator comp* for a comparator `comp` if it is sorted with respect to `comp` and `identity{}` (the identity projection).
- 7 A sequence `[start, finish)` is *partitioned with respect to an expression f(e)* if there exists an integer `n` such that for all `0 <= i < (finish - start)`, `f(*(start + i))` is `true` if and only if `i < n`.
- 8 In the descriptions of the functions that deal with ordering relationships we frequently use a notion of equivalence to describe concepts such as stability. The equivalence to which we refer is not necessarily an `operator==`, but an equivalence relation induced by the strict weak ordering. That is, two elements `a` and `b` are considered equivalent if and only if `!(a < b) && !(b < a)`.

26.8.2 Sorting

[alg.sort]

26.8.2.1 sort

[sort]

```

template<class RandomAccessIterator>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator>
void sort(ExecutionPolicy&& exec,
          RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort(RandomAccessIterator first, RandomAccessIterator last,
                    Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void sort(ExecutionPolicy&& exec,
          RandomAccessIterator first, RandomAccessIterator last,
          Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::sort(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::sort(R&& r, Comp comp = {}, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I ranges::sort(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R> ranges::sort(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Sorts the elements in the range `[first,last)` with respect to `comp` and `proj`.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* Let N be `last - first`. $\mathcal{O}(N \log N)$ comparisons and projections.

26.8.2.2 stable_sort

[stable.sort]

```

template<class RandomAccessIterator>
constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator>
void stable_sort(ExecutionPolicy&& exec,
                RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void stable_sort(RandomAccessIterator first, RandomAccessIterator last,
                          Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void stable_sort(ExecutionPolicy&& exec,
                RandomAccessIterator first, RandomAccessIterator last,
                Compare comp);

```

```

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I ranges::stable_sort(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
ranges::stable_sort(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I ranges::stable_sort(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
ranges::stable_sort(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Sorts the elements in the range `[first, last)` with respect to `comp` and `proj`.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* Let N be `last - first`. If enough extra memory is available, $N \log(N)$ comparisons. Otherwise, at most $N \log^2(N)$ comparisons. In either case, twice as many projections as the number of comparisons.
- 6 *Remarks:* Stable (16.4.6.8).

26.8.2.3 partial_sort

[partial.sort]

```

template<class RandomAccessIterator>
constexpr void partial_sort(RandomAccessIterator first,
                           RandomAccessIterator middle,
                           RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator>
void partial_sort(ExecutionPolicy&& exec,
                  RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void partial_sort(RandomAccessIterator first,
                           RandomAccessIterator middle,
                           RandomAccessIterator last,
                           Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void partial_sort(ExecutionPolicy&& exec,
                  RandomAccessIterator first,
                  RandomAccessIterator middle,
                  RandomAccessIterator last,
                  Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
ranges::partial_sort(I first, I middle, S last, Comp comp = {}, Proj proj = {});

```

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I ranges::partial_sort(Ep&& exec, I first, I middle, S last, Comp comp = {}, Proj proj = {});
```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* `[first,middle)` and `[middle,last)` are valid ranges. For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Places the first `middle - first` elements from the range `[first,last)` as sorted with respect to `comp` and `proj` into the range `[first,middle)`. The rest of the elements in the range `[middle,last)` are placed in an unspecified order.
- 4 *Returns:* `last` for the overload in namespace `ranges`.
- 5 *Complexity:* Approximately $(last - first) * \log(middle - first)$ comparisons, and twice as many projections.

```
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
ranges::partial_sort(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj = {});
```

- 6 *Effects:* Equivalent to:
- ```
return ranges::partial_sort(ranges::begin(r), middle, ranges::end(r), comp, proj);
```

```
template<execution-policy Ep, sized-random-access-range R,
 class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
ranges::partial_sort(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
 Proj proj = {});
```

- 7 *Effects:* Equivalent to:
- ```
return ranges::partial_sort(std::forward<Ep>(exec), ranges::begin(r), middle,
                        ranges::begin(r) + ranges::distance(r),
                        comp, proj);
```

26.8.2.4 partial_sort_copy

[partial.sort.copy]

```
template<class InputIterator, class RandomAccessIterator>
constexpr RandomAccessIterator
partial_sort_copy(InputIterator first, InputIterator last,
                  RandomAccessIterator result_first,
                  RandomAccessIterator result_last);

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator>
RandomAccessIterator
partial_sort_copy(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last,
                  RandomAccessIterator result_first,
                  RandomAccessIterator result_last);

template<class InputIterator, class RandomAccessIterator,
        class Compare>
constexpr RandomAccessIterator
partial_sort_copy(InputIterator first, InputIterator last,
                  RandomAccessIterator result_first,
                  RandomAccessIterator result_last,
                  Compare comp);
```

```

template<class ExecutionPolicy, class ForwardIterator, class RandomAccessIterator,
        class Compare>
RandomAccessIterator
partial_sort_copy(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last,
                  RandomAccessIterator result_first,
                  RandomAccessIterator result_last,
                  Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, random_access_iterator I2, sentinel_for<I2> S2,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires indirectly_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
        indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
constexpr ranges::partial_sort_copy_result<I1, I2>
ranges::partial_sort_copy(I1 first, S1 last, I2 result_first, S2 result_last,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, random_access_range R2, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_copyable<iterator_t<R1>, iterator_t<R2>> &&
        sortable<iterator_t<R2>, Comp, Proj2> &&
        indirect_strict_weak_order<Comp, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>
constexpr ranges::partial_sort_copy_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
ranges::partial_sort_copy(R1&& r, R2&& result_r, Comp comp = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires indirectly_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
        indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I2, Proj2>>
ranges::partial_sort_copy_result<I1, I2>
ranges::partial_sort_copy(Ep&& exec, I1 first, S1 last, I2 result_first, S2 result_last,
                          Comp comp = {}, Proj1 proj1 = {},
                          Proj2 proj2 = {});

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires indirectly_copyable<iterator_t<R1>, iterator_t<R2>> &&
        sortable<iterator_t<R2>, Comp, Proj2> &&
        indirect_strict_weak_order<Comp, projected<iterator_t<R1>, Proj1>,
        projected<iterator_t<R2>, Proj2>>
ranges::partial_sort_copy_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>>
ranges::partial_sort_copy(Ep&& exec, R1&& r, R2&& result_r, Comp comp = {},
                          Proj1 proj1 = {}, Proj2 proj2 = {});

```

- 1 Let N be $\min(\text{last} - \text{first}, \text{result_last} - \text{result_first})$. Let comp be $\text{less}\{\}$, and proj1 and proj2 be $\text{identity}\{\}$ for the overloads with no parameters by those names.
- 2 *Mandates:* For the overloads in namespace `std`, the expression `*first` is writable (24.3.1) to `result_first`.
- 3 *Preconditions:* For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3), the type of `*result_first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 4 For iterators $a1$ and $b1$ in $[\text{first}, \text{last})$, and iterators $x2$ and $y2$ in $[\text{result_first}, \text{result_last})$, after evaluating the assignment `*y2 = *b1`, let E be the value of


```
bool(invoker(comp, invoker(proj1, *a1), invoker(proj2, *y2))).
```

 Then, after evaluating the assignment `*x2 = *a1`, E is equal to


```
bool(invoker(comp, invoker(proj2, *x2), invoker(proj2, *y2))).
```

[Note 1: Writing a value from the input range into the output range does not affect how it is ordered by `comp` and `proj1` or `proj2`. — end note]

5 *Effects:* Places the first N elements as sorted with respect to `comp` and `proj2` into the range `[result_`
`first, result_first + N)`.

6 *Returns:*

- (6.1) — `result_first + N` for the overloads in namespace `std`.
- (6.2) — `{last, result_first + N}` for the overloads in namespace `ranges`.

7 *Complexity:* Approximately $(\text{last} - \text{first}) * \log N$ comparisons, and twice as many projections.

26.8.2.5 `is_sorted`

[is.sorted]

```
template<class ForwardIterator>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last);
```

1 *Effects:* Equivalent to: `return is_sorted_until(first, last) == last;`

```
template<class ExecutionPolicy, class ForwardIterator>
bool is_sorted(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last);
```

2 *Effects:* Equivalent to:

```
return is_sorted_until(std::forward<ExecutionPolicy>(exec), first, last) == last;
```

```
template<class ForwardIterator, class Compare>
constexpr bool is_sorted(ForwardIterator first, ForwardIterator last,
                         Compare comp);
```

3 *Effects:* Equivalent to: `return is_sorted_until(first, last, comp) == last;`

```
template<class ExecutionPolicy, class ForwardIterator, class Compare>
bool is_sorted(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last,
               Compare comp);
```

4 *Effects:* Equivalent to:

```
return is_sorted_until(std::forward<ExecutionPolicy>(exec), first, last, comp) == last;
```

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
         indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr bool ranges::is_sorted(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
         indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr bool ranges::is_sorted(R&& r, Comp comp = {}, Proj proj = {});
```

5 *Effects:* Equivalent to: `return ranges::is_sorted_until(first, last, comp, proj) == last;`

```
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Proj = identity,
         indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
bool ranges::is_sorted(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
         indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
bool ranges::is_sorted(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});
```

6 *Effects:* Equivalent to:

```
return ranges::is_sorted_until(std::forward<Ep>(exec), first, last, comp, proj) == last;
```

```
template<class ForwardIterator>
constexpr ForwardIterator
is_sorted_until(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator
is_sorted_until(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last);
```

```

template<class ForwardIterator, class Compare>
constexpr ForwardIterator
    is_sorted_until(ForwardIterator first, ForwardIterator last,
                    Compare comp);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
ForwardIterator
    is_sorted_until(ExecutionPolicy&& exec,
                    ForwardIterator first, ForwardIterator last,
                    Compare comp);

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr I ranges::is_sorted_until(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr borrowed_iterator_t<R>
    ranges::is_sorted_until(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
I ranges::is_sorted_until(Ep&& exec, I first, S last, Comp comp = {},
                          Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
borrowed_iterator_t<R>
    ranges::is_sorted_until(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

- 7 Let comp be less{} and proj be identity{} for the overloads with no parameters by those names.
- 8 *Returns:* The last iterator i in [first,last] for which the range [first,i) is sorted with respect to comp and proj.
- 9 *Complexity:* Linear.

26.8.3 Nth element

[alg.nth.element]

```

template<class RandomAccessIterator>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                          RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator>
void nth_element(ExecutionPolicy&& exec,
                 RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                          RandomAccessIterator last, Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
void nth_element(ExecutionPolicy&& exec,
                 RandomAccessIterator first, RandomAccessIterator nth,
                 RandomAccessIterator last, Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::nth_element(I first, I nth, S last, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I ranges::nth_element(Ep&& exec, I first, I nth, S last, Comp comp = {}, Proj proj = {});

```

- 1 Let comp be less{} and proj be identity{} for the overloads with no parameters by those names.
- 2 *Preconditions:* [first,nth) and [nth,last) are valid ranges. For the overloads in namespace std, RandomAccessIterator meets the *Cpp17ValueSwappable* requirements (16.4.4.3), and the type of

***first** meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.

3 *Effects:* After `nth_element` the element in the position pointed to by `nth` is the element that would be in that position if the whole range were sorted with respect to `comp` and `proj`, unless `nth == last`. Also for every iterator `i` in the range `[first, nth)` and every iterator `j` in the range `[nth, last)` it holds that: `bool(invoker(comp, invoker(proj, *j), invoker(proj, *i)))` is false.

4 *Returns:* `last` for the overload in namespace `ranges`.

5 *Complexity:* For the non-parallel algorithm overloads, linear on average. For the parallel algorithm overloads, $\mathcal{O}(N)$ applications of the predicate, and $\mathcal{O}(N \log N)$ swaps, where $N = \text{last} - \text{first}$.

```
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
ranges::nth_element(R&& r, iterator_t<R> nth, Comp comp = {}, Proj proj = {});
```

6 *Effects:* Equivalent to:

```
return ranges::nth_element(ranges::begin(r), nth, ranges::end(r), comp, proj);
```

```
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
ranges::nth_element(Ep&& exec, R&& r, iterator_t<R> nth, Comp comp = {}, Proj proj = {});
```

7 *Effects:* Equivalent to:

```
return ranges::nth_element(std::forward<Ep>(exec), ranges::begin(r), nth,
ranges::begin(r) + ranges::distance(r),
comp, proj);
```

26.8.4 Binary search

[alg.binary.search]

26.8.4.1 General

[alg.binary.search.general]

1 All of the algorithms in 26.8.4 are versions of binary search and assume that the sequence being searched is partitioned with respect to an expression formed by binding the search key to an argument of the comparison function. They work on non-random access iterators minimizing the number of comparisons, which will be logarithmic for all types of iterators. They are especially appropriate for random access iterators, because these algorithms do a logarithmic number of steps through the data structure. For non-random access iterators they execute a linear number of steps.

26.8.4.2 lower_bound

[lower.bound]

```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
lower_bound(ForwardIterator first, ForwardIterator last,
const T& value);
```

```
template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
class Compare>
constexpr ForwardIterator
lower_bound(ForwardIterator first, ForwardIterator last,
const T& value, Compare comp);
```

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
class T = projected_value_t<I, Proj>,
indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
constexpr I ranges::lower_bound(I first, S last, const T& value, Comp comp = {},
Proj proj = {});
```

```

template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>,
        indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
constexpr borrowed_iterator_t<R>
    ranges::lower_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});

```

1 Let comp be less{} and proj be identity{} for overloads with no parameters by those names.

2 *Preconditions:* The elements e of [first,last) are partitioned with respect to the expression
bool(invoker(comp, invoker(proj, e), value)).

3 *Returns:* The furthestmost iterator i in the range [first,last] such that for every iterator j in the
range [first,i), bool(invoker(comp, invoker(proj, *j), value)) is true.

4 *Complexity:* At most $\log_2(\text{last} - \text{first}) + \mathcal{O}(1)$ comparisons and projections.

26.8.4.3 upper_bound

[upper.bound]

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value);

```

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);

```

```

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>,
        indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
constexpr I ranges::upper_bound(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>,
        indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
constexpr borrowed_iterator_t<R>
    ranges::upper_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});

```

1 Let comp be less{} and proj be identity{} for overloads with no parameters by those names.

2 *Preconditions:* The elements e of [first,last) are partitioned with respect to the expression
!bool(invoker(comp, value, invoker(proj, e))).

3 *Returns:* The furthestmost iterator i in the range [first,last] such that for every iterator j in the
range [first,i), !bool(invoker(comp, value, invoker(proj, *j))) is true.

4 *Complexity:* At most $\log_2(\text{last} - \text{first}) + \mathcal{O}(1)$ comparisons and projections.

26.8.4.4 equal_range

[equal.range]

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first,
                ForwardIterator last, const T& value);

```

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first,
                ForwardIterator last, const T& value,
                Compare comp);

```



```

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>,
        indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
constexpr subrange<I>
    ranges::equal_range(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>,
        indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
constexpr borrowed_subrange_t<R>
    ranges::equal_range(R&& r, const T& value, Comp comp = {}, Proj proj = {});

```

1 Let `comp` be `less{}` and `proj` be `identity{}` for overloads with no parameters by those names.

2 *Preconditions:* The elements `e` of `[first,last)` are partitioned with respect to the expressions `bool(invoker(comp, invoker(proj, e), value))` and `!bool(invoker(comp, value, invoker(proj, e)))`. Also, for all elements `e` of `[first,last)`, `bool(comp(e, value))` implies `!bool(comp(value, e))` for the overloads in namespace `std`.

3 *Returns:*

(3.1) — For the overloads in namespace `std`:

```

    {lower_bound(first, last, value, comp),
     upper_bound(first, last, value, comp)}

```

(3.2) — For the overloads in namespace `ranges`:

```

    {ranges::lower_bound(first, last, value, comp, proj),
     ranges::upper_bound(first, last, value, comp, proj)}

```

4 *Complexity:* At most $2 * \log_2(\text{last} - \text{first}) + \mathcal{O}(1)$ comparisons and projections.

26.8.4.5 binary_search

[binary.search]

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type>
constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value);

```

```

template<class ForwardIterator, class T = iterator_traits<ForwardIterator>::value_type,
        class Compare>
constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value, Compare comp);

```

```

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        class T = projected_value_t<I, Proj>,
        indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
constexpr bool ranges::binary_search(I first, S last, const T& value, Comp comp = {},
                                     Proj proj = {});

```

```

template<forward_range R, class Proj = identity,
        class T = projected_value_t<iterator_t<R>, Proj>,
        indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
            ranges::less>
constexpr bool ranges::binary_search(R&& r, const T& value, Comp comp = {},
                                     Proj proj = {});

```

1 Let `comp` be `less{}` and `proj` be `identity{}` for overloads with no parameters by those names.

2 *Preconditions:* The elements `e` of `[first,last)` are partitioned with respect to the expressions `bool(invoker(comp, invoker(proj, e), value))` and `!bool(invoker(comp, value, invoker(proj, e)))`. Also, for all elements `e` of `[first,last)`, `bool(comp(e, value))` implies `!bool(comp(value, e))` for the overloads in namespace `std`.

3 *Returns:* `true` if and only if for some iterator `i` in the range `[first,last)`, `!bool(invoker(comp, invoker(proj, *i), value)) && !bool(invoker(comp, value, invoker(proj, *i)))` is `true`.

4 *Complexity:* At most $\log_2(\text{last} - \text{first}) + \mathcal{O}(1)$ comparisons and projections.

26.8.5 Partitions

[alg.partitions]

```

template<class InputIterator, class Predicate>
constexpr bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
bool is_partitioned(ExecutionPolicy&& exec,
                   ForwardIterator first, ForwardIterator last, Predicate pred);

template<input_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr bool ranges::is_partitioned(I first, S last, Pred pred, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr bool ranges::is_partitioned(R&& r, Pred pred, Proj proj = {});

```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
bool ranges::is_partitioned(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
bool ranges::is_partitioned(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

- 1 Let `proj` be `identity{}` for the overloads with no parameter named `proj`.
- 2 *Returns:* `true` if and only if the elements `e` of `[first,last)` are partitioned with respect to the expression `bool(invoker(pred, invoker(proj, e)))`.
- 3 *Complexity:* Linear. At most `last - first` applications of `pred` and `proj`.

```

template<class ForwardIterator, class Predicate>
constexpr ForwardIterator
partition(ForwardIterator first, ForwardIterator last, Predicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Predicate>
ForwardIterator
partition(ExecutionPolicy&& exec, ForwardIterator first, ForwardIterator last, Predicate pred);

template<permutable I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr subrange<I>
ranges::partition(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires permutable<iterator_t<R>>
constexpr borrowed_subrange_t<R>
ranges::partition(R&& r, Pred pred, Proj proj = {});

```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires permutable<I>
subrange<I> ranges::partition(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires permutable<iterator_t<R>>
borrowed_subrange_t<R> ranges::partition(Ep&& exec, R&& r, Pred pred, Proj proj = {});

```

- 4 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool(invoker(pred, invoker(proj, x)))`.
- 5 *Preconditions:* For the overloads in namespace `std`, `ForwardIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3).
- 6 *Effects:* Places all the elements `e` in `[first,last)` that satisfy $E(e)$ before all the elements that do not.
- 7 *Returns:* Let `i` be an iterator such that $E(*j)$ is `true` for every iterator `j` in `[first,i)` and `false` for every iterator `j` in `[i,last)`. Returns:
 - (7.1) — `i` for the overloads in namespace `std`.
 - (7.2) — `{i, last}` for the overloads in namespace `ranges`.

8 *Complexity:* Let $N = \text{last} - \text{first}$:

- (8.1) — For the non-parallel algorithm overloads, exactly N applications of the predicate and projection. At most $N/2$ swaps if the type of `first` meets the *Cpp17BidirectionalIterator* requirements for the overloads in namespace `std` or models `bidirectional_iterator` for the overloads in namespace `ranges`, and at most N swaps otherwise.
- (8.2) — For the parallel algorithm overloads, $\mathcal{O}(N \log N)$ swaps and $\mathcal{O}(N)$ applications of the predicate.

```
template<class BidirectionalIterator, class Predicate>
    BidirectionalIterator
        constexpr stable_partition(BidirectionalIterator first, BidirectionalIterator last,
                                   Predicate pred);
template<class ExecutionPolicy, class BidirectionalIterator, class Predicate>
    BidirectionalIterator
        stable_partition(ExecutionPolicy&& exec,
                        BidirectionalIterator first, BidirectionalIterator last, Predicate pred);

template<bidirectional_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    constexpr subrange<I> ranges::stable_partition(I first, S last, Pred pred, Proj proj = {});
template<bidirectional_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    constexpr borrowed_subrange_t<R> ranges::stable_partition(R&& r, Pred pred, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
    requires permutable<I>
    subrange<I>
        ranges::stable_partition(Ep&& exec, I first, S last, Pred pred, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires permutable<iterator_t<R>>
    borrowed_subrange_t<R>
        ranges::stable_partition(Ep&& exec, R&& r, Pred pred, Proj proj = {});
```

9 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool(invoker(pred, invoker(proj, x)))`.

10 *Preconditions:* For the overloads in namespace `std`, `BidirectionalIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.

11 *Effects:* Places all the elements `e` in `[first, last)` that satisfy $E(e)$ before all the elements that do not. The relative order of the elements in both groups is preserved.

12 *Returns:* Let `i` be an iterator such that for every iterator `j` in `[first, i)`, $E(*j)$ is `true`, and for every iterator `j` in the range `[i, last)`, $E(*j)$ is `false`. Returns:

- (12.1) — `i` for the overloads in namespace `std`.
- (12.2) — `{i, last}` for the overloads in namespace `ranges`.

13 *Complexity:* Let $N = \text{last} - \text{first}$:

- (13.1) — For the non-parallel algorithm overloads, at most $N \log_2 N$ swaps, but only $\mathcal{O}(N)$ swaps if there is enough extra memory. Exactly N applications of the predicate and projection.
- (13.2) — For the parallel algorithm overloads, $\mathcal{O}(N \log N)$ swaps and $\mathcal{O}(N)$ applications of the predicate.

```
template<class InputIterator, class OutputIterator1, class OutputIterator2, class Predicate>
    constexpr pair<OutputIterator1, OutputIterator2>
        partition_copy(InputIterator first, InputIterator last,
                      OutputIterator1 out_true, OutputIterator2 out_false, Predicate pred);
```

```

template<class ExecutionPolicy, class ForwardIterator, class ForwardIterator1,
        class ForwardIterator2, class Predicate>
pair<ForwardIterator1, ForwardIterator2>
partition_copy(ExecutionPolicy&& exec,
              ForwardIterator first, ForwardIterator last,
              ForwardIterator1 out_true, ForwardIterator2 out_false, Predicate pred);

template<input_iterator I, sentinel_for<I> S, weakly_incremtable O1, weakly_incremtable O2,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
constexpr ranges::partition_copy_result<I, O1, O2>
ranges::partition_copy(I first, S last, O1 out_true, O2 out_false, Pred pred,
                      Proj proj = {});

template<input_range R, weakly_incremtable O1, weakly_incremtable O2,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, O1> &&
        indirectly_copyable<iterator_t<R>, O2>
constexpr ranges::partition_copy_result<borrowed_iterator_t<R>, O1, O2>
ranges::partition_copy(R&& r, O1 out_true, O2 out_false, Pred pred, Proj proj = {});

```

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        random_access_iterator O1, sized_sentinel_for<O1> OutS1,
        random_access_iterator O2, sized_sentinel_for<O2> OutS2,
        class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
requires indirectly_copyable<I, O1> && indirectly_copyable<I, O2>
ranges::partition_copy_result<I, O1, O2>
ranges::partition_copy(Ep&& exec, I first, S last, O1 out_true, OutS1 last_true,
                      O2 out_false, OutS2 last_false, Pred pred, Proj proj = {});

template<execution-policy Ep, sized-random-access-range R,
        sized-random-access-range OutR1, sized-random-access-range OutR2,
        class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
requires indirectly_copyable<iterator_t<R>, iterator_t<OutR1>> &&
        indirectly_copyable<iterator_t<R>, iterator_t<OutR2>>
ranges::partition_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR1>,
                             borrowed_iterator_t<OutR2>>
ranges::partition_copy(Ep&& exec, R&& r, OutR1&& out_true_r, OutR2&& out_false_r,
                      Pred pred, Proj proj = {});

```

- 14 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool(invoker(pred, invoker(proj, x))).`
- 15 For the overloads with no parameters `last_true`, `last_false`, `out_true_r`, or `out_false_r`, let
- (15.1) — M be the number of iterators i in $[first, last)$ for which $E(*i)$ is true, and K be `last - first -  $M$` ;
- (15.2) — `last_true` be `out_true +  $M$` , and `last_false` be `out_false +  $K$` .
- 16 For the overloads with parameters `last_true`, `last_false`, `out_true_r`, or `out_false_r`, let M be `last_true - out_true` and K be `last_false - out_false`.
- 17 Let:
- (17.1) — $i1$ be the iterator in $[first, last)$ for which $E(*i1)$ is true and there are exactly M iterators j in $[first, i1)$ for which $E(*j)$ is true, or `last` if no such iterator exists;
- (17.2) — $i2$ be the iterator in $[first, last)$ for which $E(*i2)$ is false and there are exactly K iterators j in $[first, i2)$ for which $E(*j)$ is false, or `last` if no such iterator exists;
- (17.3) — N be `min(i1 - first, i2 - first)`.
- 18 **Mandates:** For the overloads in namespace `std`, the expression `*first` is writable (24.3.1) to `out_true` and `out_false`.
- 19 **Preconditions:** The input range and output ranges do not overlap.
- [Note 1: For the parallel algorithm overload in namespace `std`, there can be a performance cost if `first`'s value type does not meet the *Cpp17CopyConstructible* requirements. For the parallel algorithm overloads in

namespace `ranges`, there can be a performance cost if `first`'s value type does not model `copy_constructible`.
— *end note*]

20 *Effects*: For each iterator `i` in `[first, first + N)`, copies `*i` to the output range `[out_true, last_true)` if $E(*i)$ is true, or to the output range `[out_false, last_false)` otherwise.

21 *Returns*: Let Q be the number of elements copied into the output range `[out_true, last_true)`, and V be the number of elements copied into the output range `[out_false, last_false)`. Returns:

(21.1) — `{out_true + Q, out_false + V}` for the overloads in namespace `std`.

(21.2) — `{first + N, out_true + Q, out_false + V}` for the overloads in namespace `ranges`.

22 *Complexity*: At most `last - first` applications of `pred` and `proj`.

```
template<class ForwardIterator, class Predicate>
constexpr ForwardIterator
partition_point(ForwardIterator first, ForwardIterator last, Predicate pred);
```

```
template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_unary_predicate<projected<I, Proj>> Pred>
constexpr I ranges::partition_point(I first, S last, Pred pred, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
constexpr borrowed_iterator_t<R>
ranges::partition_point(R&& r, Pred pred, Proj proj = {});
```

23 Let `proj` be `identity{}` for the overloads with no parameter named `proj` and let $E(x)$ be `bool(invoker(pred, invoker(proj, x)))`.

24 *Preconditions*: The elements `e` of `[first, last)` are partitioned with respect to $E(e)$.

25 *Returns*: An iterator `mid` such that $E(*i)$ is true for all iterators `i` in `[first, mid)`, and false for all iterators `i` in `[mid, last)`.

26 *Complexity*: $\mathcal{O}(\log(\text{last} - \text{first}))$ applications of `pred` and `proj`.

26.8.6 Merge

[alg.merge]

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
merge(InputIterator1 first1, InputIterator1 last1,
      InputIterator2 first2, InputIterator2 last2,
      OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
merge(ExecutionPolicy&& exec,
      ForwardIterator1 first1, ForwardIterator1 last1,
      ForwardIterator2 first2, ForwardIterator2 last2,
      ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
merge(InputIterator1 first1, InputIterator1 last1,
      InputIterator2 first2, InputIterator2 last2,
      OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
merge(ExecutionPolicy&& exec,
      ForwardIterator1 first1, ForwardIterator1 last1,
      ForwardIterator2 first2, ForwardIterator2 last2,
      ForwardIterator result, Compare comp);
```

```

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incremtable 0, class Comp = ranges::less, class Proj1 = identity,
        class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
constexpr ranges::merge_result<I1, I2, 0>
    ranges::merge(I1 first1, S1 last1, I2 first2, S2 last2, 0 result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incremtable 0, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, 0, Comp, Proj1, Proj2>
constexpr ranges::merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, 0>
    ranges::merge(R1&& r1, R2&& r2, 0 result,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator 0, sized_sentinel_for<0> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, 0, Comp, Proj1, Proj2>
ranges::merge_result<I1, I2, 0>
    ranges::merge(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2, 0 result, OutS result_last,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::merge_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, borrowed_iterator_t<OutR>>
    ranges::merge(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

```

Let:

- (1.1) — N be:
- (1.1.1) — $(\text{last1} - \text{first1}) + (\text{last2} - \text{first2})$ for the overloads with no parameter `result_last` or `result_r`;
- (1.1.2) — $\min((\text{last1} - \text{first1}) + (\text{last2} - \text{first2}), \text{result_last} - \text{result})$ for the overloads with parameters `result_last` or `result_r`;
- (1.2) — `comp` be `less{}`, `proj1` be `identity{}`, and `proj2` be `identity{}`, for the overloads with no parameters by those names;
- (1.3) — E be `bool(invoker(comp, invoker(proj2, e2), invoker(proj1, e1)))`;
- (1.4) — K be the smallest integer in $[0, \text{last1} - \text{first1})$ such that for the element e_1 in the position $\text{first1} + K$ there are at least $N - K$ elements e_2 in $[\text{first2}, \text{last2})$ for which E holds, and be equal to $\text{last1} - \text{first1}$ if no such integer exists.

[Note 1: $\text{first1} + K$ points to the position past the last element to be copied. — end note]

Preconditions: The ranges $[\text{first1}, \text{last1})$ and $[\text{first2}, \text{last2})$ are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

Effects: Copies the first K elements of the range $[\text{first1}, \text{last1})$ and the first $N - K$ elements of the range $[\text{first2}, \text{last2})$ into the range $[\text{result}, \text{result} + N)$. If an element a precedes b in an input range, a is copied into the output range before b . If e_1 is an element of $[\text{first1}, \text{last1})$ and e_2 of $[\text{first2}, \text{last2})$, e_2 is copied into the output range before e_1 if and only if E is true.

Returns:

- (4.1) — `result + N` for the overloads in namespace `std`.
- (4.2) — $\{\text{first1} + K, \text{first2} + N - K, \text{result} + N\}$ for the overloads in namespace `ranges`.

Complexity:

- (5.1) — For the non-parallel algorithm overloads, at most $N - 1$ comparisons and applications of each projection.
- (5.2) — For the parallel algorithm overloads, $\mathcal{O}(N)$ comparisons and applications of each projection.

6 *Remarks:* Stable (16.4.6.8).

```
template<class BidirectionalIterator>
constexpr void inplace_merge(BidirectionalIterator first,
                             BidirectionalIterator middle,
                             BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator>
void inplace_merge(ExecutionPolicy&& exec,
                   BidirectionalIterator first,
                   BidirectionalIterator middle,
                   BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr void inplace_merge(BidirectionalIterator first,
                             BidirectionalIterator middle,
                             BidirectionalIterator last, Compare comp);
template<class ExecutionPolicy, class BidirectionalIterator, class Compare>
void inplace_merge(ExecutionPolicy&& exec,
                   BidirectionalIterator first,
                   BidirectionalIterator middle,
                   BidirectionalIterator last, Compare comp);

template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
         class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I ranges::inplace_merge(I first, I middle, S last, Comp comp = {}, Proj proj = {});
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         class Comp = ranges::less, class Proj = identity>
requires sortable<I, Comp, Proj>
I ranges::inplace_merge(Ep&& exec, I first, I middle, S last, Comp comp = {}, Proj proj = {});
```

7 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.

8 *Preconditions:* `[first,middle)` and `[middle,last)` are valid ranges sorted with respect to `comp` and `proj`. For the overloads in namespace `std`, `BidirectionalIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.

9 *Effects:* Merges two sorted consecutive ranges `[first,middle)` and `[middle,last)`, putting the result of the merge into the range `[first,last)`. The resulting range is sorted with respect to `comp` and `proj`.

10 *Returns:* `last` for the overload in namespace `ranges`.

11 *Complexity:* Let $N = \text{last} - \text{first}$:

(11.1) — For the non-parallel algorithm overloads, and if enough additional memory is available, at most $N - 1$ comparisons.

(11.2) — Otherwise, $\mathcal{O}(N \log N)$ comparisons.

In either case, twice as many projections as comparisons.

12 *Remarks:* Stable (16.4.6.8).

```
template<bidirectional_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
ranges::inplace_merge(R&& r, iterator_t<R> middle, Comp comp = {}, Proj proj = {});
```

13 *Effects:* Equivalent to:

```
return ranges::inplace_merge(ranges::begin(r), middle, ranges::end(r), comp, proj);
```

```
template<execution-policy Ep, sized-random-access-range R, class Comp = ranges::less,
         class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
borrowed_iterator_t<R>
ranges::inplace_merge(Ep&& exec, R&& r, iterator_t<R> middle, Comp comp = {},
```

```
Proj proj = {});
```

14 *Effects:* Equivalent to:

```
return ranges::inplace_merge(std::forward<Ep>(exec), ranges::begin(r), middle,
                             ranges::begin(r) + ranges::distance(r),
                             comp, proj);
```

26.8.7 Set operations on sorted structures

[alg.set.operations]

26.8.7.1 General

[alg.set.operations.general]

- ¹ Subclause 26.8.7 defines all the basic set operations on sorted structures. They also work with *multisets* (23.4.7) containing multiple copies of equivalent elements. The semantics of the set operations are generalized to *multisets* in a standard way by defining `set_union` to contain the maximum number of occurrences of every element, `set_intersection` to contain the minimum, and so on.

26.8.7.2 includes

[includes]

```
template<class InputIterator1, class InputIterator2>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool includes(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool includes(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class Compare>
bool includes(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<I1, Proj1>,
                                   projected<I2, Proj2>> Comp = ranges::less>
constexpr bool ranges::includes(I1 first1, S1 last1, I2 first2, S2 last2, Comp comp = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, class Proj1 = identity,
        class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                   projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
constexpr bool ranges::includes(R1&& r1, R2&& r2, Comp comp = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<I1, Proj1>, projected<I2, Proj2>> Comp =
            ranges::less>
bool ranges::includes(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, sized_random_access_range R1, sized_random_access_range R2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                   projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
bool ranges::includes(Ep&& exec, R1&& r1, R2&& r2,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

- ¹ Let `comp` be `less{}`, `proj1` be `identity{}`, and `proj2` be `identity{}`, for the overloads with no parameters by those names.

- 2 *Preconditions:* The ranges $[first1, last1)$ and $[first2, last2)$ are sorted with respect to `comp` and `proj1` or `proj2`, respectively.
- 3 *Returns:* `true` if and only if $[first2, last2)$ is a subsequence of $[first1, last1)$.
 [*Note 1:* A sequence S is a subsequence of another sequence T if S can be obtained from T by removing some, all, or none of T 's elements and keeping the remaining elements in the same order. — *end note*]
- 4 *Complexity:* At most $2 * ((last1 - first1) + (last2 - first2)) - 1$ comparisons and applications of each projection.

26.8.7.3 set_union

[`set.union`]

```
template<class InputIterator1, class InputIterator2, class OutputIterator>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
    set_union(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result);

template<class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
constexpr OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result, Compare comp);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_union(ExecutionPolicy&& exec,
              ForwardIterator1 first1, ForwardIterator1 last1,
              ForwardIterator2 first2, ForwardIterator2 last2,
              ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incremtable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_union_result<I1, I2, O>
    ranges::set_union(I1 first1, S1 last1, I2 first2, S2 last2, O result, Comp comp = {},
                      Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incremtable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
    ranges::set_union(R1&& r1, R2&& r2, O result, Comp comp = {},
                      Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_union_result<I1, I2, O>
    ranges::set_union(Ep&& exec, I1 first1, S1 last1,
                      I2 first2, S2 last2, O result, OutS result_last,
                      Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

```
template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_union_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
                        borrowed_iterator_t<OutR>>
ranges::set_union(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                  Proj1 proj1 = {}, Proj2 proj2 = {});
```

1 Let:

- (1.1) — `comp` be `less{}`, and `proj1` and `proj2` be `identity{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted union (see below);
- (1.3) — `result_last` be `result + M` for the overloads with no parameter `result_last` or `result_r`;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted union of the elements from the two ranges; that is, the set of elements that are present in one or both of the ranges. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then all m elements from the first range are included in the union, in order, and then the final $\max(n - m, 0)$ elements from the second range are included in the union, in order. If, of those elements, k elements from the first range are copied to the output range, then the first $\min(k, n)$ elements from the second range are considered *skipped*. Copies the first N elements of the sorted union to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — `{last1, last2, result + N}` for the overloads in namespace `ranges`, if N is equal to M .
- (4.3) — Otherwise, `{first1 + A, first2 + B, result_last}` for the overloads in namespace `ranges`, where A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

5 *Complexity:* At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.

6 *Remarks:* Stable (16.4.6.8).

26.8.7.4 set_intersection

[set.intersection]

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
set_intersection(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2, InputIterator2 last2,
                 OutputIterator result);

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
set_intersection(ExecutionPolicy&& exec,
                 ForwardIterator1 first1, ForwardIterator1 last1,
                 ForwardIterator2 first2, ForwardIterator2 last2,
                 ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
set_intersection(InputIterator1 first1, InputIterator1 last1,
                 InputIterator2 first2, InputIterator2 last2,
                 OutputIterator result, Compare comp);
```

```

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
set_intersection(ExecutionPolicy&& exec,
                ForwardIterator1 first1, ForwardIterator1 last1,
                ForwardIterator2 first2, ForwardIterator2 last2,
                ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incremmentable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<I1, I2, O>
ranges::set_intersection(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incremmentable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>, O>
ranges::set_intersection(R1&& r1, R2&& r2, O result,
                        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_intersection_result<I1, I2, O>
ranges::set_intersection(Ep&& exec, I1 first1, S1 last1,
                        I2 first2, S2 last2, O result, OutS result_last,
                        Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_intersection_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
                                borrowed_iterator_t<OutR>>
ranges::set_intersection(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
                        Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — `comp` be `less{}`, and `proj1` and `proj2` be `identity{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted intersection (see below);
- (1.3) — `result_last` be `result` + M for the overloads with no parameter `result_last` or `result_r`;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1,last1)` and `[first2,last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted intersection of the elements from the two ranges; that is, the set of elements that are present in both of the ranges. If `[first1,last1)` contains m elements that are equivalent to each other and `[first2,last2)` contains n elements that are equivalent to them, the first $\min(m, n)$ elements from the first range are included in the sorted intersection. If, of those elements, k elements from the first range are copied to the output range, then the first k elements from the second range are considered *skipped*. A non-copied element is also considered skipped if it compares less than the $\min(M, N + 1)^{\text{th}}$ element of the sorted intersection. Copies the first N elements of the sorted intersection to the range `[result,result + N)`.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — `{last1, last2, result + N}` for the non-parallel algorithm overloads in namespace `ranges`.

- (4.3) — Otherwise, $\{\text{first1} + A, \text{first2} + B, \text{result} + N\}$ for the parallel algorithm overloads in namespace `ranges`, where A and B are the numbers of copied or skipped elements in $[\text{first1}, \text{last1})$ and $[\text{first2}, \text{last2})$, respectively.
- 5 *Complexity*: At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.
- 6 *Remarks*: Stable (16.4.6.8).

26.8.7.5 `set_difference`

[set.difference]

```
template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
    set_difference(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result);

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, InputIterator2 last2,
                  OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
    set_difference(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2, ForwardIterator2 last2,
                  ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<I1, O>
    ranges::set_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_difference_result<borrowed_iterator_t<R1>, O>
    ranges::set_difference(R1&& r1, R2&& r2, O result,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<I1, I2, O>
    ranges::set_difference(Ep&& exec, I1 first1, S1 last1,
                          I2 first2, S2 last2, O result, OutS result_last,
                          Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
```

```

template<execution-policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_difference_truncated_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
        borrowed_iterator_t<OutR>>
ranges::set_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r, Comp comp = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — `comp` be `less{}`, and `proj1` and `proj2` be `identity{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted difference (see below);
- (1.3) — `result_last` be `result + M` for the overloads with no parameter `result_last` or `result_r`;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted difference between the elements from the two ranges; that is, the set of elements that are present in the range `[first1, last1)` but not `[first2, last2)`. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, the last $\max(m - n, 0)$ elements from `[first1, last1)` are included in the sorted difference, in order. Of those equivalent elements, the first $\min(m, n)$ elements in both ranges are considered *skipped*. An element from the second range is also considered skipped if it compares less than the $\min(N + 1, M)^{\text{th}}$ element of the sorted difference. Copies the first N elements of the sorted difference to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — `{last1, result + N}` for the non-parallel overloads in namespace `ranges`.
- (4.3) — For the parallel algorithm overloads in namespace `ranges`:
 - (4.3.1) — `{last1, first2 + B, result + N}`, if N is equal to M , where B is the number of skipped elements in `[first2, last2)`.
 - (4.3.2) — Otherwise, `{first1 + A, first2 + B, result_last}`, where A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

5 *Complexity:* At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.

6 *Remarks:* Stable ([16.4.6.8](#)).

26.8.7.6 set_symmetric_difference

[set.symmetric.difference]

```

template<class InputIterator1, class InputIterator2,
        class OutputIterator>
constexpr OutputIterator
set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, InputIterator2 last2,
        OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator>
ForwardIterator
set_symmetric_difference(ExecutionPolicy&& exec,
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, ForwardIterator2 last2,
        ForwardIterator result);

```

```

template<class InputIterator1, class InputIterator2,
        class OutputIterator, class Compare>
constexpr OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, InputIterator2 last2,
                            OutputIterator result, Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class ForwardIterator, class Compare>
ForwardIterator
    set_symmetric_difference(ExecutionPolicy&& exec,
                            ForwardIterator1 first1, ForwardIterator1 last1,
                            ForwardIterator2 first2, ForwardIterator2 last2,
                            ForwardIterator result, Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        weakly_incrementable O, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
constexpr ranges::set_symmetric_difference_result<I1, I2, O>
    ranges::set_symmetric_difference(I1 first1, S1 last1, I2 first2, S2 last2, O result,
                                    Comp comp = {}, Proj1 proj1 = {},
                                    Proj2 proj2 = {});

template<input_range R1, input_range R2, weakly_incrementable O,
        class Comp = ranges::less, class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, O, Comp, Proj1, Proj2>
constexpr ranges::set_symmetric_difference_result<borrowed_iterator_t<R1>,
        borrowed_iterator_t<R2>, O>
    ranges::set_symmetric_difference(R1&& r1, R2&& r2, O result, Comp comp = {},
                                    Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        random_access_iterator O, sized_sentinel_for<O> OutS, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<I1, I2, O, Comp, Proj1, Proj2>
ranges::set_symmetric_difference_result<I1, I2, O>
    ranges::set_symmetric_difference(Ep&& exec, I1 first1, S1 last1,
                                    I2 first2, S2 last2, O result, OutS result_last,
                                    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        sized-random-access-range OutR, class Comp = ranges::less,
        class Proj1 = identity, class Proj2 = identity>
requires mergeable<iterator_t<R1>, iterator_t<R2>, iterator_t<OutR>, Comp, Proj1, Proj2>
ranges::set_symmetric_difference_result<borrowed_iterator_t<R1>, borrowed_iterator_t<R2>,
        borrowed_iterator_t<OutR>>
    ranges::set_symmetric_difference(Ep&& exec, R1&& r1, R2&& r2, OutR&& result_r,
                                    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});

```

1 Let:

- (1.1) — `comp` be `less{}`, and `proj1` and `proj2` be `identity{}` for the overloads with no parameters by those names;
- (1.2) — M be the number of elements in the sorted symmetric difference (see below);
- (1.3) — `result_last` be `result + M` for the overloads with no parameter `result_last` or `result_r`;
- (1.4) — N be $\min(M, \text{result_last} - \text{result})$.

2 *Preconditions:* The ranges `[first1, last1)` and `[first2, last2)` are sorted with respect to `comp` and `proj1` or `proj2`, respectively. The resulting range does not overlap with either of the original ranges.

3 *Effects:* Constructs a sorted symmetric difference of the elements from the two ranges; that is, the set of elements that are present in exactly one of `[first1, last1)` and `[first2, last2)`. If `[first1, last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then $|m - n|$ of those elements are included in the symmetric difference: the last $m - n$ of these elements from `[first1, last1)`, in order, if $m > n$, and the last $n - m$ of these elements

from `[first2, last2)`, in order, if $m < n$. If $N < M$, a non-copied element is considered *skipped* if it compares less than or equivalent to the $(N + 1)^{\text{th}}$ element of the sorted symmetric difference, unless it is from the same range as that element and does not precede it. Copies the first N elements of the sorted symmetric difference to the range `[result, result + N)`.

4 *Returns:*

- (4.1) — `result_last` for the overloads in namespace `std`.
- (4.2) — `{last1, last2, result + N}` for the overloads in namespace `ranges`, if N is equal to $M + K$.
- (4.3) — Otherwise, `{first1 + A, first2 + B, result_last}` for the overloads in namespace `ranges`, where A and B are the numbers of copied or skipped elements in `[first1, last1)` and `[first2, last2)`, respectively.

5 *Complexity:* At most $2 * ((\text{last1} - \text{first1}) + (\text{last2} - \text{first2})) - 1$ comparisons and applications of each projection.

6 *Remarks:* Stable (16.4.6.8).

26.8.8 Heap operations

[alg.heap.operations]

26.8.8.1 General

[alg.heap.operations.general]

1 A random access range `[a, b)` is a *heap* with respect to *comp* and *proj* for a comparator and projection *comp* and *proj* if its elements are organized such that:

- (1.1) — With $N = b - a$, for all i , $0 < i < N$, `bool(invoker(comp, invoker(proj, a[ $\lfloor \frac{i-1}{2} \rfloor$ )]), invoker(proj, a[i]))` is false.
- (1.2) — `*a` may be removed by `pop_heap`, or a new element added by `push_heap`, in $\mathcal{O}(\log N)$ time.

2 These properties make heaps useful as priority queues.

3 `make_heap` converts a range into a heap and `sort_heap` turns a heap into a sorted sequence.

26.8.8.2 push_heap

[push.heap]

```
template<class RandomAccessIterator>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                          Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
         class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::push_heap(I first, S last, Comp comp = {}, Proj proj = {});

template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::push_heap(R&& r, Comp comp = {}, Proj proj = {});
```

1 Let *comp* be `less{}` and *proj* be `identity{}` for the overloads with no parameters by those names.

2 *Preconditions:* The range `[first, last - 1)` is a valid heap with respect to *comp* and *proj*. For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* requirements (Table 31) and the *Cpp17MoveAssignable* requirements (Table 33).

3 *Effects:* Places the value in the location `last - 1` into the resulting heap `[first, last)`.

4 *Returns:* `last` for the overloads in namespace `ranges`.

5 *Complexity:* At most $\log(\text{last} - \text{first})$ comparisons and twice as many projections.

26.8.8.3 pop_heap

[pop.heap]

```
template<class RandomAccessIterator>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last);
```

```

template<class RandomAccessIterator, class Compare>
constexpr void pop_heap(RandomAccessIterator first, RandomAccessIterator last,
                        Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::pop_heap(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::pop_heap(R&& r, Comp comp = {}, Proj proj = {});

```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* The range `[first,last)` is a valid non-empty heap with respect to `comp` and `proj`. For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Swaps the value in the location `first` with the value in the location `last - 1` and makes `[first,last - 1)` into a heap with respect to `comp` and `proj`.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* At most $2 \log(\text{last} - \text{first})$ comparisons and twice as many projections.

26.8.8.4 make_heap

[make.heap]

```

template<class RandomAccessIterator>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                        Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::make_heap(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::make_heap(R&& r, Comp comp = {}, Proj proj = {});

```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Constructs a heap with respect to `comp` and `proj` out of the range `[first,last)`.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* At most $3(\text{last} - \text{first})$ comparisons and twice as many projections.

26.8.8.5 sort_heap

[sort.heap]

```

template<class RandomAccessIterator>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
                        Compare comp);

```



```

template<random_access_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I
    ranges::sort_heap(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Comp = ranges::less, class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr borrowed_iterator_t<R>
    ranges::sort_heap(R&& r, Comp comp = {}, Proj proj = {});

```

- 1 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 2 *Preconditions:* The range `[first, last)` is a valid heap with respect to `comp` and `proj`. For the overloads in namespace `std`, `RandomAccessIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3) and the type of `*first` meets the *Cpp17MoveConstructible* (Table 31) and *Cpp17MoveAssignable* (Table 33) requirements.
- 3 *Effects:* Sorts elements in the heap `[first, last)` with respect to `comp` and `proj`.
- 4 *Returns:* `last` for the overloads in namespace `ranges`.
- 5 *Complexity:* At most $2N \log N$ comparisons, where $N = \text{last} - \text{first}$, and twice as many projections.

26.8.8.6 `is_heap`

[is.heap]

```

template<class RandomAccessIterator>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last);

```

- 1 *Effects:* Equivalent to: `return is_heap_until(first, last) == last;`

```

template<class ExecutionPolicy, class RandomAccessIterator>
bool is_heap(ExecutionPolicy&& exec,
             RandomAccessIterator first, RandomAccessIterator last);

```

- 2 *Effects:* Equivalent to:
 `return is_heap_until(std::forward<ExecutionPolicy>(exec), first, last) == last;`

```

template<class RandomAccessIterator, class Compare>
constexpr bool is_heap(RandomAccessIterator first, RandomAccessIterator last,
                      Compare comp);

```

- 3 *Effects:* Equivalent to: `return is_heap_until(first, last, comp) == last;`

```

template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
bool is_heap(ExecutionPolicy&& exec,
             RandomAccessIterator first, RandomAccessIterator last,
             Compare comp);

```

- 4 *Effects:* Equivalent to:
 `return is_heap_until(std::forward<ExecutionPolicy>(exec), first, last, comp) == last;`

```

template<random_access_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr bool ranges::is_heap(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr bool ranges::is_heap(R&& r, Comp comp = {}, Proj proj = {});

```

- 5 *Effects:* Equivalent to: `return ranges::is_heap_until(first, last, comp, proj) == last;`

```

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
bool ranges::is_heap(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
bool ranges::is_heap(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

- 6 *Effects:* Equivalent to:
 `return ranges::is_heap_until(std::forward<Ep>(exec), first, last, comp, proj) == last;`

```

template<class RandomAccessIterator>
constexpr RandomAccessIterator
is_heap_until(RandomAccessIterator first, RandomAccessIterator last);
template<class ExecutionPolicy, class RandomAccessIterator>
RandomAccessIterator
is_heap_until(ExecutionPolicy&& exec,
              RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
constexpr RandomAccessIterator
is_heap_until(RandomAccessIterator first, RandomAccessIterator last,
              Compare comp);
template<class ExecutionPolicy, class RandomAccessIterator, class Compare>
RandomAccessIterator
is_heap_until(ExecutionPolicy&& exec,
              RandomAccessIterator first, RandomAccessIterator last,
              Compare comp);

template<random_access_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr I ranges::is_heap_until(I first, S last, Comp comp = {}, Proj proj = {});
template<random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr borrowed_iterator_t<R>
ranges::is_heap_until(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
I ranges::is_heap_until(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
borrowed_iterator_t<R>
ranges::is_heap_until(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

7 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.

8 *Returns:* The last iterator `i` in `[first, last]` for which the range `[first, i)` is a heap with respect to `comp` and `proj`.

9 *Complexity:* Linear.

26.8.9 Minimum and maximum

[alg.min.max]

```

template<class T>
constexpr const T& min(const T& a, const T& b);
template<class T, class Compare>
constexpr const T& min(const T& a, const T& b, Compare comp);

template<class T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr const T& ranges::min(const T& a, const T& b, Comp comp = {}, Proj proj = {});

```

1 *Preconditions:* For the first form, `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

2 *Returns:* The smaller value. Returns the first argument when the arguments are equivalent.

3 *Complexity:* Exactly one comparison and two applications of the projection, if any.

4 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```

template<class T>
constexpr T min(initializer_list<T> r);
template<class T, class Compare>
constexpr T min(initializer_list<T> r, Compare comp);

```

```

template<copyable T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr T ranges::min(initializer_list<T> r, Comp comp = {}, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
constexpr range_value_t<R>
    ranges::min(R&& r, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
range_value_t<R>
    ranges::min(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

5 *Preconditions:* `ranges::distance(r) > 0`. For the overloads in namespace `std`, `T` meets the *Cpp17-CopyConstructible* requirements. For the first form, `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

6 *Returns:* The smallest value in the input range. Returns a copy of the leftmost element when several elements are equivalent to the smallest.

7 *Complexity:* Exactly `ranges::distance(r) - 1` comparisons and twice as many applications of the projection, if any.

8 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```

template<class T>
constexpr const T& max(const T& a, const T& b);
template<class T, class Compare>
constexpr const T& max(const T& a, const T& b, Compare comp);

```

```

template<class T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr const T& ranges::max(const T& a, const T& b, Comp comp = {}, Proj proj = {});

```

9 *Preconditions:* For the first form, `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

10 *Returns:* The larger value. Returns the first argument when the arguments are equivalent.

11 *Complexity:* Exactly one comparison and two applications of the projection, if any.

12 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```

template<class T>
constexpr T max(initializer_list<T> r);
template<class T, class Compare>
constexpr T max(initializer_list<T> r, Compare comp);

```

```

template<copyable T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr T ranges::max(initializer_list<T> r, Comp comp = {}, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
constexpr range_value_t<R>
    ranges::max(R&& r, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
range_value_t<R>
    ranges::max(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

13 *Preconditions:* `ranges::distance(r) > 0`. For the overloads in namespace `std`, `T` meets the *Cpp17-CopyConstructible* requirements. For the first form, `T` meets the *Cpp17LessThanComparable* requirements (Table 29).

14 *Returns:* The largest value in the input range. Returns a copy of the leftmost element when several elements are equivalent to the largest.

15 *Complexity:* Exactly `ranges::distance(r) - 1` comparisons and twice as many applications of the projection, if any.

16 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```
template<class T>
constexpr pair<const T&, const T&> minmax(const T& a, const T& b);
template<class T, class Compare>
constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);

template<class T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr ranges::minmax_result<const T&>
ranges::minmax(const T& a, const T& b, Comp comp = {}, Proj proj = {});
```

17 *Preconditions:* For the first form, `T` meets the *Cpp17LessThanComparable* requirements ([Table 29](#)).

18 *Returns:* `{b, a}` if `b` is smaller than `a`, and `{a, b}` otherwise.

19 *Complexity:* Exactly one comparison and two applications of the projection, if any.

20 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```
template<class T>
constexpr pair<T, T> minmax(initializer_list<T> t);
template<class T, class Compare>
constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

template<copyable T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr ranges::minmax_result<T>
ranges::minmax(initializer_list<T> r, Comp comp = {}, Proj proj = {});
template<input_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
constexpr ranges::minmax_result<range_value_t<R>>
ranges::minmax(R&& r, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
requires indirectly_copyable_storable<iterator_t<R>, range_value_t<R>*>
ranges::minmax_result<range_value_t<R>>
ranges::minmax(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});
```

21 *Preconditions:* `ranges::distance(r) > 0`. For the overloads in namespace `std`, `T` meets the *Cpp17-CopyConstructible* requirements. For the first form, type `T` meets the *Cpp17LessThanComparable* requirements ([Table 29](#)).

22 *Returns:* Let `X` be the return type. Returns `X{x, y}`, where `x` is a copy of the leftmost element with the smallest value and `y` a copy of the rightmost element with the largest value in the input range.

23 *Complexity:* At most $(3/2)\text{ranges::distance}(r)$ applications of the corresponding predicate and twice as many applications of the projection, if any.

24 *Remarks:* An invocation may explicitly specify an argument for the template parameter `T` of the overloads in namespace `std`.

```
template<class ForwardIterator>
constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last);

template<class ExecutionPolicy, class ForwardIterator>
ForwardIterator min_element(ExecutionPolicy&& exec,
                           ForwardIterator first, ForwardIterator last);
```

```

template<class ForwardIterator, class Compare>
    constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last,
                                          Compare comp);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator min_element(ExecutionPolicy&& exec,
                               ForwardIterator first, ForwardIterator last, Compare comp);

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    constexpr I ranges::min_element(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    constexpr borrowed_iterator_t<R>
        ranges::min_element(R&& r, Comp comp = {}, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    I ranges::min_element(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R,
        class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    borrowed_iterator_t<R>
        ranges::min_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});
25     Let comp be less{} and proj be identity{} for the overloads with no parameters by those names.
26     Returns: The first iterator i in the range [first,last) such that for every iterator j in the range
        [first,last),
            bool(invoker(comp, invoker(proj, *j), invoker(proj, *i)))
        is false. Returns last if first == last.
27     Complexity: Exactly max(last - first - 1, 0) comparisons and twice as many projections.

template<class ForwardIterator>
    constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
    ForwardIterator max_element(ExecutionPolicy&& exec,
                               ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
    constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last,
                                          Compare comp);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
    ForwardIterator max_element(ExecutionPolicy&& exec,
                               ForwardIterator first, ForwardIterator last,
                               Compare comp);

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    constexpr I ranges::max_element(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    constexpr borrowed_iterator_t<R>
        ranges::max_element(R&& r, Comp comp = {}, Proj proj = {});

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
    I ranges::max_element(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution-policy Ep, sized-random-access-range R,
        class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
    borrowed_iterator_t<R>
        ranges::max_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});

```

- 28 Let `comp` be `less{}` and `proj` be `identity{}` for the overloads with no parameters by those names.
- 29 *Returns:* The first iterator `i` in the range `[first,last)` such that for every iterator `j` in the range `[first,last)`,
- ```
bool(invoker(comp, invoker(proj, *i), invoker(proj, *j)))
```
- is false. Returns `last` if `first == last`.
- 30 *Complexity:* Exactly  $\max(\text{last} - \text{first} - 1, 0)$  comparisons and twice as many projections.
- ```
template<class ForwardIterator>
constexpr pair<ForwardIterator, ForwardIterator>
minmax_element(ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator>
pair<ForwardIterator, ForwardIterator>
minmax_element(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
constexpr pair<ForwardIterator, ForwardIterator>
minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);
template<class ExecutionPolicy, class ForwardIterator, class Compare>
pair<ForwardIterator, ForwardIterator>
minmax_element(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last, Compare comp);

template<forward_iterator I, sentinel_for<I> S, class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
constexpr ranges::minmax_element_result<I>
ranges::minmax_element(I first, S last, Comp comp = {}, Proj proj = {});
template<forward_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
constexpr ranges::minmax_element_result<borrowed_iterator_t<R>>
ranges::minmax_element(R&& r, Comp comp = {}, Proj proj = {});

template<execution_policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
        class Proj = identity,
        indirect_strict_weak_order<projected<I, Proj>> Comp = ranges::less>
ranges::minmax_element_result<I>
ranges::minmax_element(Ep&& exec, I first, S last, Comp comp = {}, Proj proj = {});
template<execution_policy Ep, sized_random_access_range R, class Proj = identity,
        indirect_strict_weak_order<projected<iterator_t<R>, Proj>> Comp = ranges::less>
ranges::minmax_element_result<borrowed_iterator_t<R>>
ranges::minmax_element(Ep&& exec, R&& r, Comp comp = {}, Proj proj = {});
```
- 31 *Returns:* `{first, first}` if `[first,last)` is empty, otherwise `{m, M}`, where `m` is the first iterator in `[first,last)` such that no iterator in the range refers to a smaller element, and where `M` is the last iterator¹⁸³ in `[first,last)` such that no iterator in the range refers to a larger element.
- 32 *Complexity:* Let N be `last - first`. At most $\max(\lfloor \frac{3}{2}(N - 1) \rfloor, 0)$ comparisons and twice as many applications of the projection, if any.

26.8.10 Bounded value

[alg.clamp]

```
template<class T>
constexpr const T& clamp(const T& v, const T& lo, const T& hi);
template<class T, class Compare>
constexpr const T& clamp(const T& v, const T& lo, const T& hi, Compare comp);
template<class T, class Proj = identity,
        indirect_strict_weak_order<projected<const T*, Proj>> Comp = ranges::less>
constexpr const T&
ranges::clamp(const T& v, const T& lo, const T& hi, Comp comp = {}, Proj proj = {});
```

- 1 Let `comp` be `less{}` for the overloads with no parameter `comp`, and let `proj` be `identity{}` for the overloads with no parameter `proj`.

¹⁸³) This behavior intentionally differs from `max_element`.

- 2 *Preconditions:* `bool(invoker(comp, invoker(proj, hi), invoker(proj, lo)))` is false. For the first form, type `T` meets the *Cpp17LessThanComparable* requirements (Table 29).
- 3 *Returns:* `lo` if `bool(invoker(comp, invoker(proj, v), invoker(proj, lo)))` is true, `hi` if `bool(invoker(comp, invoker(proj, hi), invoker(proj, v)))` is true, otherwise `v`.
- 4 [Note 1: If NaN is avoided, `T` can be a floating-point type. — end note]
- 5 *Complexity:* At most two comparisons and three applications of the projection.

26.8.11 Lexicographical comparison

[alg.lex.comparison]

```
template<class InputIterator1, class InputIterator2>
constexpr bool
    lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
bool
    lexicographical_compare(ExecutionPolicy&& exec,
                           ForwardIterator1 first1, ForwardIterator1 last1,
                           ForwardIterator2 first2, ForwardIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
constexpr bool
    lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2,
                           Compare comp);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class Compare>
bool
    lexicographical_compare(ExecutionPolicy&& exec,
                           ForwardIterator1 first1, ForwardIterator1 last1,
                           ForwardIterator2 first2, ForwardIterator2 last2,
                           Compare comp);

template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<I1, Proj1>,
                                   projected<I2, Proj2>> Comp = ranges::less>
constexpr bool
    ranges::lexicographical_compare(I1 first1, S1 last1, I2 first2, S2 last2,
                                    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<input_range R1, input_range R2, class Proj1 = identity,
        class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                   projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
constexpr bool
    ranges::lexicographical_compare(R1&& r1, R2&& r2, Comp comp = {},
                                    Proj1 proj1 = {}, Proj2 proj2 = {});

template<execution_policy Ep, random_access_iterator I1, sized_sentinel_for<I1> S1,
        random_access_iterator I2, sized_sentinel_for<I2> S2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<I1, Proj1>,
                                   projected<I2, Proj2>> Comp = ranges::less>
bool ranges::lexicographical_compare(Ep&& exec, I1 first1, S1 last1, I2 first2, S2 last2,
                                    Comp comp = {}, Proj1 proj1 = {}, Proj2 proj2 = {});
template<execution_policy Ep, sized-random-access-range R1, sized-random-access-range R2,
        class Proj1 = identity, class Proj2 = identity,
        indirect_strict_weak_order<projected<iterator_t<R1>, Proj1>,
                                   projected<iterator_t<R2>, Proj2>> Comp = ranges::less>
bool ranges::lexicographical_compare(Ep&& exec, R1&& r1, R2&& r2, Comp comp = {},
                                    Proj1 proj1 = {}, Proj2 proj2 = {});
```

- 1 *Returns:* true if and only if the sequence of elements defined by the range `[first1,last1)` is lexicographically less than the sequence of elements defined by the range `[first2,last2)`.

- 2 *Complexity:* At most $2\min(\text{last1} - \text{first1}, \text{last2} - \text{first2})$ applications of the corresponding comparison and each projection, if any.
- 3 *Remarks:* If two sequences have the same number of elements and their corresponding elements (if any) are equivalent, then neither sequence is lexicographically less than the other. If one sequence is a proper prefix of the other, then the shorter sequence is lexicographically less than the longer sequence. Otherwise, the lexicographical comparison of the sequences yields the same result as the comparison of the first corresponding pair of elements that are not equivalent.
- 4 [Example 1: `ranges::lexicographical_compare(I1, S1, I2, S2, Comp, Proj1, Proj2)` can be implemented as:
- ```

 for (; first1 != last1 && first2 != last2; ++first1, (void)++first2) {
 if (invoke(comp, invoke(proj1, *first1), invoke(proj2, *first2))) return true;
 if (invoke(comp, invoke(proj2, *first2), invoke(proj1, *first1))) return false;
 }
 return first1 == last1 && first2 != last2;

```
- end example]
- 5 [Note 1: An empty sequence is lexicographically less than any non-empty sequence, but not less than any empty sequence. — end note]

### 26.8.12 Three-way comparison algorithms

[alg.three.way]

```

template<class InputIterator1, class InputIterator2, class Cmp>
constexpr auto
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
 InputIterator2 b2, InputIterator2 e2,
 Cmp comp)
 -> decltype(comp(*b1, *b2));

```

- 1 Let  $N$  be  $\min(e1 - b1, e2 - b2)$ . Let  $E(n)$  be `comp(*(b1 + n), *(b2 + n))`.
- 2 *Mandates:* `decltype(comp(*b1, *b2))` is a comparison category type.
- 3 *Returns:*  $E(i)$ , where  $i$  is the smallest integer in  $[0, N)$  such that  $E(i) \neq 0$  is true, or  $(e1 - b1) <= (e2 - b2)$  if no such integer exists.
- 4 *Complexity:* At most  $N$  applications of `comp`.

```

template<class InputIterator1, class InputIterator2>
constexpr auto
lexicographical_compare_three_way(InputIterator1 b1, InputIterator1 e1,
 InputIterator2 b2, InputIterator2 e2);

```

- 5 *Effects:* Equivalent to:
- ```

    return lexicographical_compare_three_way(b1, e1, b2, e2, compare_three_way());

```

26.8.13 Permutation generators

[alg.permutation.generators]

```

template<class BidirectionalIterator>
constexpr bool next_permutation(BidirectionalIterator first,
                                BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool next_permutation(BidirectionalIterator first,
                                BidirectionalIterator last, Compare comp);

template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr ranges::next_permutation_result<I>
ranges::next_permutation(I first, S last, Comp comp = {}, Proj proj = {});

```



```
template<bidirectional_range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr ranges::next_permutation_result<borrowed_iterator_t<R>>
ranges::next_permutation(R&& r, Comp comp = {}, Proj proj = {});
```

1 Let `comp` be `less{}` and `proj` be `identity{}` for overloads with no parameters by those names.

2 *Preconditions:* For the overloads in namespace `std`, `BidirectionalIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3).

3 *Effects:* Takes a sequence defined by the range `[first, last)` and transforms it into the next permutation. The next permutation is found by assuming that the set of all permutations is lexicographically sorted with respect to `comp` and `proj`. If no such permutation exists, transforms the sequence into the first permutation; that is, the ascendingly-sorted one.

4 *Returns:* Let `B` be `true` if a next permutation was found and otherwise `false`. Returns:

(4.1) — `B` for the overloads in namespace `std`.

(4.2) — `{ last, B }` for the overloads in namespace `ranges`.

5 *Complexity:* At most $(last - first) / 2$ swaps.

```
template<class BidirectionalIterator>
constexpr bool prev_permutation(BidirectionalIterator first,
                                BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
constexpr bool prev_permutation(BidirectionalIterator first,
                                BidirectionalIterator last, Compare comp);
```

```
template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr ranges::prev_permutation_result<I>
ranges::prev_permutation(I first, S last, Comp comp = {}, Proj proj = {});
template<bidirectional_range R, class Comp = ranges::less,
        class Proj = identity>
requires sortable<iterator_t<R>, Comp, Proj>
constexpr ranges::prev_permutation_result<borrowed_iterator_t<R>>
ranges::prev_permutation(R&& r, Comp comp = {}, Proj proj = {});
```

6 Let `comp` be `less{}` and `proj` be `identity{}` for overloads with no parameters by those names.

7 *Preconditions:* For the overloads in namespace `std`, `BidirectionalIterator` meets the *Cpp17ValueSwappable* requirements (16.4.4.3).

8 *Effects:* Takes a sequence defined by the range `[first, last)` and transforms it into the previous permutation. The previous permutation is found by assuming that the set of all permutations is lexicographically sorted with respect to `comp` and `proj`. If no such permutation exists, transforms the sequence into the last permutation; that is, the descendingly-sorted one.

9 *Returns:* Let `B` be `true` if a previous permutation was found and otherwise `false`. Returns:

(9.1) — `B` for the overloads in namespace `std`.

(9.2) — `{ last, B }` for the overloads in namespace `ranges`.

10 *Complexity:* At most $(last - first) / 2$ swaps.

26.9 Header `<numeric>` synopsis

[numeric.ops.overview]

```
// mostly freestanding
namespace std {
    // 26.10.3, accumulate
    template<class InputIterator, class T>
    constexpr T accumulate(InputIterator first, InputIterator last, T init);
    template<class InputIterator, class T, class BinaryOperation>
    constexpr T accumulate(InputIterator first, InputIterator last, T init,
                            BinaryOperation binary_op);
```

```

// 26.10.4, reduce
template<class InputIterator>
    constexpr typename iterator_traits<InputIterator>::value_type
        reduce(InputIterator first, InputIterator last);
template<class InputIterator, class T>
    constexpr T reduce(InputIterator first, InputIterator last, T init);
template<class InputIterator, class T, class BinaryOperation>
    constexpr T reduce(InputIterator first, InputIterator last, T init,
        BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator>
    typename iterator_traits<ForwardIterator>::value_type
        reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
            ForwardIterator first, ForwardIterator last);
template<class ExecutionPolicy, class ForwardIterator, class T>
    T reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, T init);
template<class ExecutionPolicy, class ForwardIterator, class T, class BinaryOperation>
    T reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, T init, BinaryOperation binary_op);

// 26.10.5, inner product
template<class InputIterator1, class InputIterator2, class T>
    constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, T init);
template<class InputIterator1, class InputIterator2, class T,
    class BinaryOperation1, class BinaryOperation2>
    constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, T init,
        BinaryOperation1 binary_op1, BinaryOperation2 binary_op2);

// 26.10.6, transform reduce
template<class InputIterator1, class InputIterator2, class T>
    constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, T init);
template<class InputIterator1, class InputIterator2, class T,
    class BinaryOperation1, class BinaryOperation2>
    constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
        InputIterator2 first2, T init,
        BinaryOperation1 binary_op1, BinaryOperation2 binary_op2);
template<class InputIterator, class T,
    class BinaryOperation, class UnaryOperation>
    constexpr T transform_reduce(InputIterator first, InputIterator last, T init,
        BinaryOperation binary_op, UnaryOperation unary_op);
template<class ExecutionPolicy,
    class ForwardIterator1, class ForwardIterator2, class T>
    T transform_reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, T init);
template<class ExecutionPolicy,
    class ForwardIterator1, class ForwardIterator2, class T,
    class BinaryOperation1, class BinaryOperation2>
    T transform_reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator1 first1, ForwardIterator1 last1,
        ForwardIterator2 first2, T init,
        BinaryOperation1 binary_op1, BinaryOperation2 binary_op2);
template<class ExecutionPolicy, class ForwardIterator, class T,
    class BinaryOperation, class UnaryOperation>
    T transform_reduce(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
        ForwardIterator first, ForwardIterator last, T init,
        BinaryOperation binary_op, UnaryOperation unary_op);

```

```

// 26.10.7, partial sum
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    partial_sum(InputIterator first, InputIterator last,
                OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    partial_sum(InputIterator first, InputIterator last,
                OutputIterator result, BinaryOperation binary_op);

// 26.10.8, exclusive scan
template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, T init);
template<class InputIterator, class OutputIterator, class T, class BinaryOperation>
constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, T init, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation>
ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, T init, BinaryOperation binary_op);

// 26.10.9, inclusive scan
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, BinaryOperation binary_op);
template<class InputIterator, class OutputIterator, class BinaryOperation, class T>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, BinaryOperation binary_op, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class T>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec, // freestanding-deleted, see 26.3.5
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, BinaryOperation binary_op, T init);

```

```

// 26.10.10, transform exclusive scan
template<class InputIterator, class OutputIterator, class T,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator
    transform_exclusive_scan(InputIterator first, InputIterator last,
                            OutputIterator result, T init,
                            BinaryOperation binary_op, UnaryOperation unary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2
    transform_exclusive_scan(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                            ForwardIterator1 first, ForwardIterator1 last,
                            ForwardIterator2 result, T init,
                            BinaryOperation binary_op, UnaryOperation unary_op);

// 26.10.11, transform inclusive scan
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op, UnaryOperation unary_op);
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation, class T>
constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                             OutputIterator result,
                             BinaryOperation binary_op, UnaryOperation unary_op, T init);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result, BinaryOperation binary_op,
                             UnaryOperation unary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation, class T>
ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                             ForwardIterator1 first, ForwardIterator1 last,
                             ForwardIterator2 result,
                             BinaryOperation binary_op, UnaryOperation unary_op, T init);

// 26.10.12, adjacent difference
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    adjacent_difference(InputIterator first, InputIterator last,
                        OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    adjacent_difference(InputIterator first, InputIterator last,
                        OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    adjacent_difference(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                        ForwardIterator1 first, ForwardIterator1 last,
                        ForwardIterator2 result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
ForwardIterator2
    adjacent_difference(ExecutionPolicy&& exec,           // freestanding-deleted, see 26.3.5
                        ForwardIterator1 first, ForwardIterator1 last,
                        ForwardIterator2 result, BinaryOperation binary_op);

```

```

// 26.10.13, iota
template<class ForwardIterator, class T>
constexpr void iota(ForwardIterator first, ForwardIterator last, T value);

namespace ranges {
    template<class O, class T>
        using iota_result = out_value_result<O, T>;

    template<input_or_output_iterator O, sentinel_for<O> S, weakly_incrementable T>
        requires indirectly_writable<O, const T&>
        constexpr iota_result<O, T> iota(O first, S last, T value);

    template<weakly_incrementable T, output_range<const T&> R>
        constexpr iota_result<borrowed_iterator_t<R>, T> iota(R&& r, T value);
}

// 26.10.14, greatest common divisor
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);

// 26.10.15, least common multiple
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);

// 26.10.16, midpoint
template<class T>
constexpr T midpoint(T a, T b) noexcept;
template<class T>
constexpr T* midpoint(T* a, T* b);

// 26.10.17, saturation arithmetic
template<class T>
constexpr T saturating_add(T x, T y) noexcept;
template<class T>
constexpr T saturating_sub(T x, T y) noexcept;
template<class T>
constexpr T saturating_mul(T x, T y) noexcept;
template<class T>
constexpr T saturating_div(T x, T y) noexcept;
template<class T, class U>
constexpr T saturating_cast(U x) noexcept;
}

```

26.10 Generalized numeric operations

[numeric.ops]

26.10.1 General

[numeric.ops.general]

- ¹ [Note 1: The use of closed ranges as well as semi-open ranges to specify requirements throughout 26.10 is intentional. — end note]

26.10.2 Definitions

[numerics.defns]

- ¹ Define *GENERALIZED_NONCOMMUTATIVE_SUM*(op, a1, ..., aN) as follows:
- (1.1) — a1 when N is 1, otherwise
 - (1.2) — op(*GENERALIZED_NONCOMMUTATIVE_SUM*(op, a1, ..., aK), *GENERALIZED_NONCOMMUTATIVE_SUM*(op, aM, ..., aN)) for any K where $1 < K + 1 = M \leq N$.
- ² Define *GENERALIZED_SUM*(op, a1, ..., aN) as *GENERALIZED_NONCOMMUTATIVE_SUM*(op, b1, ..., bN), where b1, ..., bN may be any permutation of a1, ..., aN.

26.10.3 Accumulate

[accumulate]

```

template<class InputIterator, class T>
constexpr T accumulate(InputIterator first, InputIterator last, T init);

```

```
template<class InputIterator, class T, class BinaryOperation>
constexpr T accumulate(InputIterator first, InputIterator last, T init,
    BinaryOperation binary_op);
```

1 *Preconditions:* T meets the *Cpp17CopyConstructible* (Table 32) and *Cpp17CopyAssignable* (Table 34) requirements. In the range [first, last], binary_op neither modifies elements nor invalidates iterators or subranges.¹⁸⁴

2 *Effects:* Computes its result by initializing the accumulator acc with the initial value init and then modifies it with acc = std::move(acc) + *i or acc = binary_op(std::move(acc), *i) for every iterator i in the range [first, last) in order.¹⁸⁵

26.10.4 Reduce

[reduce]

```
template<class InputIterator>
constexpr typename iterator_traits<InputIterator>::value_type
    reduce(InputIterator first, InputIterator last);
```

1 *Effects:* Equivalent to:

```
    return reduce(first, last,
        typename iterator_traits<InputIterator>::value_type{});
```

```
template<class ExecutionPolicy, class ForwardIterator>
typename iterator_traits<ForwardIterator>::value_type
    reduce(ExecutionPolicy&& exec,
        ForwardIterator first, ForwardIterator last);
```

2 *Effects:* Equivalent to:

```
    return reduce(std::forward<ExecutionPolicy>(exec), first, last,
        typename iterator_traits<ForwardIterator>::value_type{});
```

```
template<class InputIterator, class T>
constexpr T reduce(InputIterator first, InputIterator last, T init);
```

3 *Effects:* Equivalent to:

```
    return reduce(first, last, init, plus<>());
```

```
template<class ExecutionPolicy, class ForwardIterator, class T>
T reduce(ExecutionPolicy&& exec,
    ForwardIterator first, ForwardIterator last, T init);
```

4 *Effects:* Equivalent to:

```
    return reduce(std::forward<ExecutionPolicy>(exec), first, last, init, plus<>());
```

```
template<class InputIterator, class T, class BinaryOperation>
constexpr T reduce(InputIterator first, InputIterator last, T init,
    BinaryOperation binary_op);
```

```
template<class ExecutionPolicy, class ForwardIterator, class T, class BinaryOperation>
T reduce(ExecutionPolicy&& exec,
    ForwardIterator first, ForwardIterator last, T init,
    BinaryOperation binary_op);
```

5 *Mandates:* All of

- (5.1) — binary_op(init, *first),
- (5.2) — binary_op(*first, init),
- (5.3) — binary_op(init, init), and
- (5.4) — binary_op(*first, *first)

are convertible to T.

6 *Preconditions:*

- (6.1) — T meets the *Cpp17MoveConstructible* (Table 31) requirements.

¹⁸⁴) The use of fully closed ranges is intentional.

¹⁸⁵) **accumulate** is similar to the APL reduction operator and Common Lisp reduce function, but it avoids the difficulty of defining the result of reduction on an empty sequence by always requiring an initial value.

(6.2) — `binary_op` neither invalidates iterators or subranges, nor modifies elements in the range `[first, last]`.

7 *Returns:* `GENERALIZED_SUM(binary_op, init, *i, ...)` for every `i` in `[first, last)`.

8 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$ applications of `binary_op`.

9 [Note 1: The difference between `reduce` and `accumulate` is that `reduce` applies `binary_op` in an unspecified order, which yields a nondeterministic result for non-associative or non-commutative `binary_op` such as floating-point addition. — end note]

26.10.5 Inner product

[inner.product]

```
template<class InputIterator1, class InputIterator2, class T>
constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                          InputIterator2 first2, T init);
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
constexpr T inner_product(InputIterator1 first1, InputIterator1 last1,
                          InputIterator2 first2, T init,
                          BinaryOperation1 binary_op1,
                          BinaryOperation2 binary_op2);
```

1 *Preconditions:* `T` meets the *Cpp17CopyConstructible* (Table 32) and *Cpp17CopyAssignable* (Table 34) requirements. In the ranges `[first1, last1]` and `[first2, first2 + (last1 - first1)]` `binary_op1` and `binary_op2` neither modifies elements nor invalidates iterators or subranges.¹⁸⁶

2 *Effects:* Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifying it with `acc = std::move(acc) + (*i1) * (*i2)` or `acc = binary_op1(std::move(acc), binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1, last1)` and iterator `i2` in the range `[first2, first2 + (last1 - first1))` in order.

26.10.6 Transform reduce

[transform.reduce]

```
template<class InputIterator1, class InputIterator2, class T>
constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2,
                             T init);
```

1 *Effects:* Equivalent to:

```
return transform_reduce(first1, last1, first2, init, plus<>(), multiplies<>());
```

```
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T>
T transform_reduce(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2,
                  T init);
```

2 *Effects:* Equivalent to:

```
return transform_reduce(std::forward<ExecutionPolicy>(exec),
                        first1, last1, first2, init, plus<>(), multiplies<>());
```

```
template<class InputIterator1, class InputIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
constexpr T transform_reduce(InputIterator1 first1, InputIterator1 last1,
                              InputIterator2 first2,
                              T init,
                              BinaryOperation1 binary_op1,
                              BinaryOperation2 binary_op2);
```

¹⁸⁶) The use of fully closed ranges is intentional.


```

template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation1, class BinaryOperation2>
T transform_reduce(ExecutionPolicy&& exec,
                  ForwardIterator1 first1, ForwardIterator1 last1,
                  ForwardIterator2 first2,
                  T init,
                  BinaryOperation1 binary_op1,
                  BinaryOperation2 binary_op2);

```

3 *Mandates:* All of

- (3.1) — `binary_op1`(`init`, `init`),
- (3.2) — `binary_op1`(`init`, `binary_op2`(*`first1`, *`first2`)),
- (3.3) — `binary_op1`(`binary_op2`(*`first1`, *`first2`), `init`), and
- (3.4) — `binary_op1`(`binary_op2`(*`first1`, *`first2`), `binary_op2`(*`first1`, *`first2`))

are convertible to `T`.

4 *Preconditions:*

- (4.1) — `T` meets the *Cpp17MoveConstructible* (Table 31) requirements.
- (4.2) — Neither `binary_op1` nor `binary_op2` invalidates subranges, nor modifies elements in the ranges [`first1`,`last1`] and [`first2`,`first2` + (`last1` - `first1`)].

5 *Returns:*

`GENERALIZED_SUM`(`binary_op1`, `init`, `binary_op2`(**i*, *(`first2` + (*i* - `first1`))), ...) for every iterator *i* in [`first1`,`last1`].

6 *Complexity:* $\mathcal{O}(\text{last1} - \text{first1})$ applications each of `binary_op1` and `binary_op2`.

```

template<class InputIterator, class T,
        class BinaryOperation, class UnaryOperation>
constexpr T transform_reduce(InputIterator first, InputIterator last, T init,
                             BinaryOperation binary_op, UnaryOperation unary_op);
template<class ExecutionPolicy,
        class ForwardIterator, class T,
        class BinaryOperation, class UnaryOperation>
T transform_reduce(ExecutionPolicy&& exec,
                  ForwardIterator first, ForwardIterator last,
                  T init, BinaryOperation binary_op, UnaryOperation unary_op);

```

7 *Mandates:* All of

- (7.1) — `binary_op`(`init`, `init`),
- (7.2) — `binary_op`(`init`, `unary_op`(*`first`)),
- (7.3) — `binary_op`(`unary_op`(*`first`), `init`), and
- (7.4) — `binary_op`(`unary_op`(*`first`), `unary_op`(*`first`))

are convertible to `T`.

8 *Preconditions:*

- (8.1) — `T` meets the *Cpp17MoveConstructible* (Table 31) requirements.
- (8.2) — Neither `unary_op` nor `binary_op` invalidates subranges, nor modifies elements in the range [`first`,`last`].

9 *Returns:*

`GENERALIZED_SUM`(`binary_op`, `init`, `unary_op`(**i*), ...) for every iterator *i* in [`first`,`last`].

10 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$ applications each of `unary_op` and `binary_op`.

11 [Note 1: `transform_reduce` does not apply `unary_op` to `init`. — end note]

26.10.7 Partial sum

[partial.sum]

```

template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    partial_sum(InputIterator first, InputIterator last,
                OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    partial_sum(InputIterator first, InputIterator last,
                OutputIterator result, BinaryOperation binary_op);

```

- 1 *Mandates:* InputIterator's value type is constructible from *first. The result of the expression `std::move(acc) + *i` or `binary_op(std::move(acc), *i)` is implicitly convertible to InputIterator's value type. acc is writable (24.3.1) to result.
- 2 *Preconditions:* In the ranges [first,last] and [result,result + (last - first)] binary_op neither modifies elements nor invalidates iterators or subranges.¹⁸⁷
- 3 *Effects:* For a non-empty range, the function creates an accumulator acc whose type is InputIterator's value type, initializes it with *first, and assigns the result to *result. For every iterator i in [first + 1,last) in order, acc is then modified by `acc = std::move(acc) + *i` or `acc = binary_op(std::move(acc), *i)` and the result is assigned to `*(result + (i - first))`.
- 4 *Returns:* result + (last - first).
- 5 *Complexity:* Exactly (last - first) - 1 applications of the binary operation.
- 6 *Remarks:* result may be equal to first.

26.10.8 Exclusive scan

[exclusive.scan]

```

template<class InputIterator, class OutputIterator, class T>
constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, T init);
1
    Effects: Equivalent to:
        return exclusive_scan(first, last, result, init, plus<>());

template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2, class T>
ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, T init);
2
    Effects: Equivalent to:
        return exclusive_scan(std::forward<ExecutionPolicy>(exec),
                              first, last, result, init, plus<>());

template<class InputIterator, class OutputIterator, class T, class BinaryOperation>
constexpr OutputIterator
    exclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, T init, BinaryOperation binary_op);
template<class ExecutionPolicy,
          class ForwardIterator1, class ForwardIterator2, class T, class BinaryOperation>
ForwardIterator2
    exclusive_scan(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, T init, BinaryOperation binary_op);

```

- 3 *Mandates:* All of
- (3.1) — `binary_op(init, init)`,
 - (3.2) — `binary_op(init, *first)`, and
 - (3.3) — `binary_op(*first, *first)`

¹⁸⁷) The use of fully closed ranges is intentional.

are convertible to T.

Preconditions:

- (4.1) — T meets the *Cpp17MoveConstructible* (Table 31) requirements.
- (4.2) — `binary_op` neither invalidates iterators or subranges, nor modifies elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

Effects: For each integer K in `[0, last - first)` assigns through `result + K` the value of:

```
GENERALIZED_NONCOMMUTATIVE_SUM(
    binary_op, init, *(first + 0), *(first + 1), ..., *(first + K - 1))
```

Returns: The end of the resulting range beginning at `result`.

Complexity: $\mathcal{O}(\text{last} - \text{first})$ applications of `binary_op`.

Remarks: `result` may be equal to `first`.

[Note 1: The difference between `exclusive_scan` and `inclusive_scan` is that `exclusive_scan` excludes the i^{th} input element from the i^{th} sum. If `binary_op` is not mathematically associative, the behavior of `exclusive_scan` can be nondeterministic. — end note]

26.10.9 Inclusive scan

[inclusive.scan]

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result);
```

Effects: Equivalent to:

```
return inclusive_scan(first, last, result, plus<>());
```

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result);
```

Effects: Equivalent to:

```
return inclusive_scan(std::forward<ExecutionPolicy>(exec), first, last, result, plus<>());
```

```
template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, BinaryOperation binary_op);
```

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, BinaryOperation binary_op);
```

```
template<class InputIterator, class OutputIterator, class BinaryOperation, class T>
constexpr OutputIterator
    inclusive_scan(InputIterator first, InputIterator last,
                  OutputIterator result, BinaryOperation binary_op, T init);
```

```
template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class BinaryOperation, class T>
ForwardIterator2
    inclusive_scan(ExecutionPolicy&& exec,
                  ForwardIterator1 first, ForwardIterator1 last,
                  ForwardIterator2 result, BinaryOperation binary_op, T init);
```

Let U be the value type of `decltype(first)`.

Mandates: If `init` is provided, all of

- (4.1) — `binary_op(init, init)`,
- (4.2) — `binary_op(init, *first)`, and

(4.3) — `binary_op(*first, *first)`

are convertible to T; otherwise, `binary_op(*first, *first)` is convertible to U.

5 *Preconditions:*

(5.1) — If `init` is provided, T meets the *Cpp17MoveConstructible* (Table 31) requirements; otherwise, U meets the *Cpp17MoveConstructible* requirements.

(5.2) — `binary_op` neither invalidates iterators or subranges, nor modifies elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

6 *Effects:* For each integer K in `[0, last - first)` assigns through `result + K` the value of

(6.1) — `GENERALIZED_NONCOMMUTATIVE_SUM(`
 `binary_op, init, *(first + 0), *(first + 1), ..., *(first + K))`
 if `init` is provided, or

(6.2) — `GENERALIZED_NONCOMMUTATIVE_SUM(`
 `binary_op, *(first + 0), *(first + 1), ..., *(first + K))`
 otherwise.

7 *Returns:* The end of the resulting range beginning at `result`.

8 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$ applications of `binary_op`.

9 *Remarks:* `result` may be equal to `first`.

10 [Note 1: The difference between `exclusive_scan` and `inclusive_scan` is that `inclusive_scan` includes the i^{th} input element in the i^{th} sum. If `binary_op` is not mathematically associative, the behavior of `inclusive_scan` can be nondeterministic. — end note]

26.10.10 Transform exclusive scan

[transform.exclusive.scan]

```
template<class InputIterator, class OutputIterator, class T,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator
    transform_exclusive_scan(InputIterator first, InputIterator last,
                            OutputIterator result, T init,
                            BinaryOperation binary_op, UnaryOperation unary_op);

template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2, class T,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2
    transform_exclusive_scan(ExecutionPolicy&& exec,
                            ForwardIterator1 first, ForwardIterator1 last,
                            ForwardIterator2 result, T init,
                            BinaryOperation binary_op, UnaryOperation unary_op);
```

1 *Mandates:* All of

(1.1) — `binary_op(init, init)`,

(1.2) — `binary_op(init, unary_op(*first))`, and

(1.3) — `binary_op(unary_op(*first), unary_op(*first))`

are convertible to T.

2 *Preconditions:*

(2.1) — T meets the *Cpp17MoveConstructible* (Table 31) requirements.

(2.2) — Neither `unary_op` nor `binary_op` invalidates iterators or subranges, nor modifies elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

3 *Effects:* For each integer K in `[0, last - first)` assigns through `result + K` the value of:

```
GENERALIZED_NONCOMMUTATIVE_SUM(
    binary_op, init,
    unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K - 1)))
```

4 *Returns:* The end of the resulting range beginning at `result`.

5 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$ applications each of `unary_op` and `binary_op`.

6 *Remarks:* `result` may be equal to `first`.

7 [Note 1: The difference between `transform_exclusive_scan` and `transform_inclusive_scan` is that `transform_exclusive_scan` excludes the i^{th} input element from the i^{th} sum. If `binary_op` is not mathematically associative, the behavior of `transform_exclusive_scan` can be nondeterministic. `transform_exclusive_scan` does not apply `unary_op` to `init`. — end note]

26.10.11 Transform inclusive scan

[`transform.inclusive.scan`]

```
template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation>
constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                            OutputIterator result,
                            BinaryOperation binary_op, UnaryOperation unary_op);

template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation>
ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,
                            ForwardIterator1 first, ForwardIterator1 last,
                            ForwardIterator2 result,
                            BinaryOperation binary_op, UnaryOperation unary_op);

template<class InputIterator, class OutputIterator,
        class BinaryOperation, class UnaryOperation, class T>
constexpr OutputIterator
    transform_inclusive_scan(InputIterator first, InputIterator last,
                            OutputIterator result,
                            BinaryOperation binary_op, UnaryOperation unary_op,
                            T init);

template<class ExecutionPolicy,
        class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation, class UnaryOperation, class T>
ForwardIterator2
    transform_inclusive_scan(ExecutionPolicy&& exec,
                            ForwardIterator1 first, ForwardIterator1 last,
                            ForwardIterator2 result,
                            BinaryOperation binary_op, UnaryOperation unary_op,
                            T init);
```

1 Let `U` be the value type of `decltype(first)`.

2 *Mandates:* If `init` is provided, all of

- (2.1) — `binary_op(init, init)`,
- (2.2) — `binary_op(init, unary_op(*first))`, and
- (2.3) — `binary_op(unary_op(*first), unary_op(*first))`

are convertible to `T`; otherwise, `binary_op(unary_op(*first), unary_op(*first))` is convertible to `U`.

3 *Preconditions:*

- (3.1) — If `init` is provided, `T` meets the *Cpp17MoveConstructible* (Table 31) requirements; otherwise, `U` meets the *Cpp17MoveConstructible* requirements.
- (3.2) — Neither `unary_op` nor `binary_op` invalidates iterators or subranges, nor modifies elements in the ranges `[first, last]` or `[result, result + (last - first)]`.

4 *Effects:* For each integer `K` in `[0, last - first)` assigns through `result + K` the value of

- (4.1) — `GENERALIZED_NONCOMMUTATIVE_SUM(`
 `binary_op, init,`
 `unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K))`
 if `init` is provided, or
- (4.2) — `GENERALIZED_NONCOMMUTATIVE_SUM(`
 `binary_op,`

unary_op(*(first + 0)), unary_op(*(first + 1)), ..., unary_op(*(first + K)))
otherwise.

5 *Returns:* The end of the resulting range beginning at **result**.

6 *Complexity:* $\mathcal{O}(\text{last} - \text{first})$ applications each of unary_op and binary_op.

7 *Remarks:* **result** may be equal to **first**.

8 [Note 1: The difference between transform_exclusive_scan and transform_inclusive_scan is that transform_inclusive_scan includes the i^{th} input element in the i^{th} sum. If binary_op is not mathematically associative, the behavior of transform_inclusive_scan can be nondeterministic. transform_inclusive_scan does not apply unary_op to init. — end note]

26.10.12 Adjacent difference

[adjacent.difference]

```
template<class InputIterator, class OutputIterator>
constexpr OutputIterator
    adjacent_difference(InputIterator first, InputIterator last,
                       OutputIterator result);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2>
ForwardIterator2
    adjacent_difference(ExecutionPolicy&& exec,
                       ForwardIterator1 first, ForwardIterator1 last, ForwardIterator2 result);

template<class InputIterator, class OutputIterator, class BinaryOperation>
constexpr OutputIterator
    adjacent_difference(InputIterator first, InputIterator last,
                       OutputIterator result, BinaryOperation binary_op);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryOperation>
ForwardIterator2
    adjacent_difference(ExecutionPolicy&& exec,
                       ForwardIterator1 first, ForwardIterator1 last,
                       ForwardIterator2 result, BinaryOperation binary_op);
```

1 Let T be the value type of `decltype(first)`. For the overloads that do not take an argument **binary_op**, let **binary_op** be an lvalue that denotes an object of type `minus<>`.

2 *Mandates:*

- (2.1) — For the overloads with no **ExecutionPolicy**, T is constructible from ***first**. **acc** (defined below) is writable (24.3.1) to the **result** output iterator. The result of the expression `binary_op(val, std::move(acc))` is writable to **result**.
- (2.2) — For the overloads with an **ExecutionPolicy**, the result of the expressions `binary_op(*first, *first)` and ***first** are writable to **result**.

3 *Preconditions:*

- (3.1) — For the overloads with no **ExecutionPolicy**, T meets the *Cpp17MoveAssignable* (Table 33) requirements.
- (3.2) — For all overloads, in the ranges `[first, last]` and `[result, result + (last - first)]`, **binary_op** neither modifies elements nor invalidates iterators or subranges.¹⁸⁸

4 *Effects:* For the overloads with no **ExecutionPolicy** and a non-empty range, the function creates an accumulator **acc** of type T, initializes it with ***first**, and assigns the result to ***result**. For every iterator **i** in `[first + 1, last)` in order, creates an object **val** whose type is T, initializes it with ***i**, computes `binary_op(val, std::move(acc))`, assigns the result to `*(result + (i - first))`, and move assigns from **val** to **acc**.

5 For the overloads with an **ExecutionPolicy** and a non-empty range, performs `*result = *first`. Then, for every **d** in `[1, last - first - 1]`, performs `*(result + d) = binary_op(*(first + d), *(first + (d - 1)))`.

6 *Returns:* **result + (last - first)**.

7 *Complexity:* Exactly $(\text{last} - \text{first}) - 1$ applications of the binary operation.

¹⁸⁸ The use of fully closed ranges is intentional.

- 8 *Remarks:* For the overloads with no `ExecutionPolicy`, `result` may be equal to `first`. For the overloads with an `ExecutionPolicy`, the ranges `[first,last)` and `[result,result + (last - first))` shall not overlap.

26.10.13 Iota

[numeric.iota]

```
template<class ForwardIterator, class T>
constexpr void iota(ForwardIterator first, ForwardIterator last, T value);
```

- 1 *Mandates:* T is convertible to ForwardIterator's value type. The expression `++val`, where `val` has type T, is well-formed.
- 2 *Effects:* For each element referred to by the iterator `i` in the range `[first,last)`, assigns `*i = value` and increments `value` as if by `++value`.
- 3 *Complexity:* Exactly `last - first` increments and assignments.

```
template<input_or_output_iterator O, sentinel_for<O> S, weakly_incrementable T>
requires indirectly_writable<O, const T&>
constexpr ranges::iota_result<O, T> ranges::iota(O first, S last, T value);
template<weakly_incrementable T, output_range<const T&> R>
constexpr ranges::iota_result<borrowed_iterator_t<R>, T> ranges::iota(R&& r, T value);
```

- 4 *Effects:* Equivalent to:
- ```
while (first != last) {
 *first = as_const(value);
 ++first;
 ++value;
}
return {std::move(first), std::move(value)};
```

### 26.10.14 Greatest common divisor

[numeric.ops.gcd]

```
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);
```

- 1 *Mandates:* M and N both are integer types other than *cv bool*.
- 2 *Preconditions:* `|m|` and `|n|` are representable as a value of `common_type_t<M, N>`.  
[Note 1: These requirements ensure, for example, that `gcd(m, m) = |m|` is representable as a value of type M. — end note]
- 3 *Returns:* Zero when `m` and `n` are both zero. Otherwise, returns the greatest common divisor of `|m|` and `|n|`.
- 4 *Throws:* Nothing.

### 26.10.15 Least common multiple

[numeric.ops.lcm]

```
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);
```

- 1 *Mandates:* M and N both are integer types other than *cv bool*.
- 2 *Preconditions:* `|m|` and `|n|` are representable as a value of `common_type_t<M, N>`. The least common multiple of `|m|` and `|n|` is representable as a value of type `common_type_t<M, N>`.
- 3 *Returns:* Zero when either `m` or `n` is zero. Otherwise, returns the least common multiple of `|m|` and `|n|`.
- 4 *Throws:* Nothing.

### 26.10.16 Midpoint

[numeric.ops.midpoint]

```
template<class T>
constexpr T midpoint(T a, T b) noexcept;
```

- 1 *Constraints:* T is an arithmetic type other than *cv bool*.
- 2 *Returns:* Half the sum of `a` and `b`. If T is an integer type and the sum is odd, the result is rounded towards `a`.
- 3 *Remarks:* No overflow occurs. If T is a floating-point type, at most one inexact operation occurs.

```
template<class T>
constexpr T* midpoint(T* a, T* b);
```

4     *Constraints:* T is an object type.

5     *Mandates:* T is a complete type.

6     *Preconditions:* a and b point to, respectively, elements *i* and *j* of the same array object **x**.

[*Note 1:* As specified in 6.9.5, an object that is not an array element is considered to belong to a single-element array for this purpose and a pointer past the last element of an array of *n* elements is considered to be equivalent to a pointer to a hypothetical array element *n* for this purpose. — end note]

7     *Returns:* A pointer to array element  $i + \frac{j-i}{2}$  of **x**, where the result of the division is truncated towards zero.

## 26.10.17 Saturation arithmetic

[numeric.sat]

### 26.10.17.1 Arithmetic functions

[numeric.sat.func]

1 In the following descriptions, an arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type.

```
template<class T>
constexpr T saturating_add(T x, T y) noexcept;
```

2     *Constraints:* T is a signed or unsigned integer type (6.9.3).

3     *Returns:* If  $x + y$  is representable as a value of type T,  $x + y$ ; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of  $x + y$ .

```
template<class T>
constexpr T saturating_sub(T x, T y) noexcept;
```

4     *Constraints:* T is a signed or unsigned integer type (6.9.3).

5     *Returns:* If  $x - y$  is representable as a value of type T,  $x - y$ ; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of  $x - y$ .

```
template<class T>
constexpr T saturating_mul(T x, T y) noexcept;
```

6     *Constraints:* T is a signed or unsigned integer type (6.9.3).

7     *Returns:* If  $x \times y$  is representable as a value of type T,  $x \times y$ ; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of  $x \times y$ .

```
template<class T>
constexpr T saturating_div(T x, T y) noexcept;
```

8     *Constraints:* T is a signed or unsigned integer type (6.9.3).

9     *Preconditions:*  $y \neq 0$  is true.

10     *Returns:* If T is a signed integer type and  $x == \text{numeric\_limits}<T>::\text{min}() \ \&\& \ y == -1$  is true,  $\text{numeric\_limits}<T>::\text{max}()$ , otherwise,  $x / y$ .

11     *Remarks:* A function call expression that violates the precondition in the *Preconditions* element is not a core constant expression (7.7.2).

### 26.10.17.2 Casting

[numeric.sat.cast]

```
template<class R, class T>
constexpr R saturating_cast(T x) noexcept;
```

1     *Constraints:* R and T are signed or unsigned integer types (6.9.3).

2     *Returns:* If *x* is representable as a value of type R, *x*; otherwise, either the largest or smallest representable value of type R, whichever is closer to the value of *x*.

## 26.11 Specialized <memory> algorithms

[specialized.algorithms]

### 26.11.1 General

[specialized.algorithms.general]

1 The contents specified in 26.11 are declared in the header <memory> (20.2.2).

- <sup>2</sup> Unless otherwise specified, if an exception is thrown in the following algorithms, objects constructed by a placement *new-expression* (7.6.2.8) are destroyed in an unspecified order before allowing the exception to propagate.
- <sup>3</sup> [Note 1: When new objects are created by the algorithms specified in 26.11, the lifetime ends for any existing objects (including potentially-overlapping subobjects 6.8.2) in storage that is reused 6.8.4. — end note]
- <sup>4</sup> Some algorithms specified in 26.11 make use of the following exposition-only function templates:

```
template<class T>
constexpr void* voidify(T& obj) noexcept {
 return addressof(obj);
}

template<class I>
constexpr decltype(auto) deref-move(I& it) {
 if constexpr (is_lvalue_reference_v<decltype(*it)>)
 return std::move(*it);
 else
 return *it;
}
```

### 26.11.2 Special memory concepts

[special.mem.concepts]

- <sup>1</sup> Some algorithms in this subclause are constrained with the following exposition-only concepts:

```
template<class I>
concept nothrow-input-iterator = // exposition only
 input_iterator<I> &&
 is_lvalue_reference_v<iter_reference_t<I>> &&
 same_as<remove_cvref_t<iter_reference_t<I>>, iter_value_t<I>>;
```

- <sup>2</sup> A type *I* models *nothrow-input-iterator* only if no exceptions are thrown from increment, copy construction, move construction, copy assignment, move assignment, or indirection through valid iterators.

- <sup>3</sup> [Note 1: This concept allows some *input\_iterator* (24.3.4.9) operations to throw exceptions. — end note]

```
template<class S, class I>
concept nothrow-sentinel-for = sentinel_for<S, I>; // exposition only
```

- <sup>4</sup> Types *S* and *I* model *nothrow-sentinel-for* only if no exceptions are thrown from copy construction, move construction, copy assignment, move assignment, or comparisons between valid values of type *I* and *S*.

- <sup>5</sup> [Note 2: This concept allows some *sentinel\_for* (24.3.4.7) operations to throw exceptions. — end note]

```
template<class S, class I>
concept nothrow-sized-sentinel-for = // exposition only
 nothrow-sentinel-for<S, I> &&
 sized_sentinel_for<S, I>;
```

- <sup>6</sup> Types *S* and *I* model *nothrow-sized-sentinel-for* only if no exceptions are thrown from the *-* operator for valid values of type *I* and *S*.

- <sup>7</sup> [Note 3: This concept allows some *sized\_sentinel\_for* (24.3.4.8) operations to throw exceptions. — end note]

```
template<class R>
concept nothrow-input-range = // exposition only
 range<R> &&
 nothrow-input-iterator<iterator_t<R>> &&
 nothrow-sentinel-for<sentinel_t<R>, iterator_t<R>>;
```

- <sup>8</sup> A type *R* models *nothrow-input-range* only if no exceptions are thrown from calls to *ranges::begin* and *ranges::end* on an object of type *R*.



```
template<class I>
concept nothrow-forward-iterator = // exposition only
 nothrow-input-iterator<I> &&
 forward-iterator<I> &&
 nothrow-sentinel-for<I, I>;
```

9 [Note 4: This concept allows some *forward-iterator* (24.3.4.11) operations to throw exceptions. — end note]

```
template<class R>
concept nothrow-forward-range = // exposition only
 nothrow-input-range<R> &&
 nothrow-forward-iterator<iterator_t<R>>;
```

```
template<class I>
concept nothrow-bidirectional-iterator = // exposition only
 nothrow-forward-iterator<I> &&
 bidirectional-iterator<I>;
```

10 A type I models *nothrow-bidirectional-iterator* only if no exceptions are thrown from decrementing valid iterators.

[Note 5: This concept allows some *bidirectional-iterator* (24.3.4.12) operations to throw exceptions. — end note]

```
template<class R>
concept nothrow-bidirectional-range = // exposition only
 nothrow-forward-range<R> &&
 nothrow-bidirectional-iterator<iterator_t<R>>;
```

```
template<class I>
concept nothrow-random-access-iterator = // exposition only
 nothrow-bidirectional-iterator<I> &&
 random-access-iterator<I> &&
 nothrow-sized-sentinel-for<I, I>;
```

11 A type I models *nothrow-random-access-iterator* only if no exceptions are thrown from comparisons of valid iterators, or the -, +, -=, +=, [] operators on valid values of type I and *iter\_difference\_t*<I>.

[Note 6: This concept allows some *random-access-iterator* (24.3.4.13) operations to throw exceptions. — end note]

```
template<class R>
concept nothrow-random-access-range = // exposition only
 nothrow-bidirectional-range<R> &&
 nothrow-random-access-iterator<iterator_t<R>>;
```

```
template<class R>
concept nothrow-sized-random-access-range = // exposition only
 nothrow-random-access-range<R> && sized_range<R>;
```

12 A type R models *nothrow-sized-random-access-range* only if no exceptions are thrown from the call to *ranges::size* on an object of type R.

### 26.11.3 uninitialized\_default\_construct [uninitialized.construct.default]

```
template<class NoThrowForwardIterator>
constexpr void uninitialized_default_construct(NoThrowForwardIterator first,
 NoThrowForwardIterator last);
```

1 *Effects*: Equivalent to:

```
for (; first != last; ++first)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type;
```

```
namespace ranges {
 template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
 requires default_initializable<iter_value_t<I>>
 constexpr I uninitialized_default_construct(I first, S last);
```

```

template<nothrow-forward-range R>
 requires default_initializable<range_value_t<R>>
 constexpr borrowed_iterator_t<R> uninitialized_default_construct(R&& r);
}

```

2     *Effects:* Equivalent to:

```

 for (; first != last; ++first)
 ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>;
 return first;

```

```

template<class NoThrowForwardIterator, class Size>
 constexpr NoThrowForwardIterator
 uninitialized_default_construct_n(NoThrowForwardIterator first, Size n);

```

3     *Effects:* Equivalent to:

```

 for (; n > 0; (void)++first, --n)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type;
 return first;

```

```

namespace ranges {
 template<nothrow-forward-iterator I>
 requires default_initializable<iter_value_t<I>>
 constexpr I uninitialized_default_construct_n(I first, iter_difference_t<I> n);
}

```

4     *Effects:* Equivalent to:

```

 return uninitialized_default_construct(counted_iterator(first, n),
 default_sentinel).base();

```

#### 26.11.4 uninitialized\_value\_construct [uninitialized.construct.value]

```

template<class NoThrowForwardIterator>
 constexpr void uninitialized_value_construct(NoThrowForwardIterator first,
 NoThrowForwardIterator last);

```

1     *Effects:* Equivalent to:

```

 for (; first != last; ++first)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type();

```

```

namespace ranges {
 template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S>
 requires default_initializable<iter_value_t<I>>
 constexpr I uninitialized_value_construct(I first, S last);
 template<nothrow-forward-range R>
 requires default_initializable<range_value_t<R>>
 constexpr borrowed_iterator_t<R> uninitialized_value_construct(R&& r);
}

```

2     *Effects:* Equivalent to:

```

 for (; first != last; ++first)
 ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>();
 return first;

```

```

template<class NoThrowForwardIterator, class Size>
 constexpr NoThrowForwardIterator
 uninitialized_value_construct_n(NoThrowForwardIterator first, Size n);

```

3     *Effects:* Equivalent to:

```

 for (; n > 0; (void)++first, --n)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type();
 return first;

```

```

namespace ranges {
 template<nothrow-forward-iterator I>
 requires default_initializable<iter_value_t<I>>
 constexpr I uninitialized_value_construct_n(I first, iter_difference_t<I> n);
}

```

4     *Effects:* Equivalent to:

```

 return uninitialized_value_construct(counted_iterator(first, n),
 default_sentinel).base();

```

### 26.11.5 uninitialized\_copy

[uninitialized.copy]

```

template<class InputIterator, class NoThrowForwardIterator>
 constexpr NoThrowForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
 NoThrowForwardIterator result);

```

1     *Preconditions:* result + [0, (last - first)) does not overlap with [first, last).

2     *Effects:* Equivalent to:

```

 for (; first != last; ++result, (void)++first)
 ::new (voidify(*result)) iterator_traits<NoThrowForwardIterator>::value_type(*first);

```

3     *Returns:* result.

```

namespace ranges {
 template<input_iterator I, sentinel_for<I> S1,
 nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
 requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
 constexpr uninitialized_copy_result<I, O>
 uninitialized_copy(I ifirst, S1 ilast, O ofirst, S2 olast);
 template<input_range IR, nothrow-forward-range OR>
 requires constructible_from<range_value_t<OR>, range_reference_t<IR>>
 constexpr uninitialized_copy_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
 uninitialized_copy(IR&& in_range, OR&& out_range);
}

```

4     *Preconditions:* [ofirst, olast) does not overlap with [ifirst, ilast).

5     *Effects:* Equivalent to:

```

 for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst)
 ::new (voidify(*ofirst)) remove_reference_t<iter_reference_t<O>>(*ifirst);
 return {std::move(ifirst), ofirst};

```

```

template<class InputIterator, class Size, class NoThrowForwardIterator>
 constexpr NoThrowForwardIterator uninitialized_copy_n(InputIterator first, Size n,
 NoThrowForwardIterator result);

```

6     *Preconditions:* result + [0, n) does not overlap with first + [0, n).

7     *Effects:* Equivalent to:

```

 for (; n > 0; ++result, (void)++first, --n)
 ::new (voidify(*result)) iterator_traits<NoThrowForwardIterator>::value_type(*first);

```

8     *Returns:* result.

```

namespace ranges {
 template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
 requires constructible_from<iter_value_t<O>, iter_reference_t<I>>
 constexpr uninitialized_copy_n_result<I, O>
 uninitialized_copy_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}

```

9     *Preconditions:* [ofirst, olast) does not overlap with ifirst + [0, n).

10    *Effects:* Equivalent to:

```

 auto t = uninitialized_copy(counted_iterator(std::move(ifirst), n),
 default_sentinel, ofirst, olast);
 return {std::move(t.in).base(), t.out};

```

## 26.11.6 uninitialized\_move

[uninitialized.move]

```
template<class InputIterator, class NoThrowForwardIterator>
constexpr NoThrowForwardIterator uninitialized_move(InputIterator first, InputIterator last,
 NoThrowForwardIterator result);
```

1     *Preconditions:* result + [0, (last - first)) does not overlap with [first, last).

2     *Effects:* Equivalent to:

```
 for (; first != last; (void)++result, ++first)
 ::new (voidify(*result))
 iterator_traits<NoThrowForwardIterator>::value_type(deref-move(first));
 return result;
```

```
namespace ranges {
 template<input_iterator I, sentinel_for<I> S1,
 nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
 requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
 constexpr uninitialized_move_result<I, O>
 uninitialized_move(I ifirst, S1 ilast, O ofirst, S2 olast);
 template<input_range IR, nothrow-forward-range OR>
 requires constructible_from<range_value_t<OR>, range_rvalue_reference_t<IR>>
 constexpr uninitialized_move_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
 uninitialized_move(IR&& in_range, OR&& out_range);
}
```

3     *Preconditions:* [ofirst, olast) does not overlap with [ifirst, ilast).

4     *Effects:* Equivalent to:

```
 for (; ifirst != ilast && ofirst != olast; ++ofirst, (void)++ifirst)
 ::new (voidify(*ofirst))
 remove_reference_t<iter_reference_t<O>>(ranges::iter_move(ifirst));
 return {std::move(ifirst), ofirst};
```

5     [Note 1: If an exception is thrown, some objects in the range [ifirst, ilast) are left in a valid, but unspecified state. — end note]

```
template<class InputIterator, class Size, class NoThrowForwardIterator>
constexpr pair<InputIterator, NoThrowForwardIterator>
uninitialized_move_n(InputIterator first, Size n, NoThrowForwardIterator result);
```

6     *Preconditions:* result + [0, n) does not overlap with first + [0, n).

7     *Effects:* Equivalent to:

```
 for (; n > 0; ++result, (void)++first, --n)
 ::new (voidify(*result))
 iterator_traits<NoThrowForwardIterator>::value_type(deref-move(first));
 return {first, result};
```

```
namespace ranges {
 template<input_iterator I, nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
 requires constructible_from<iter_value_t<O>, iter_rvalue_reference_t<I>>
 constexpr uninitialized_move_n_result<I, O>
 uninitialized_move_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

8     *Preconditions:* [ofirst, olast) does not overlap with ifirst + [0, n).

9     *Effects:* Equivalent to:

```
 auto t = uninitialized_move(counted_iterator(std::move(ifirst), n),
 default_sentinel, ofirst, olast);
 return {std::move(t.in).base(), t.out};
```

10    [Note 2: If an exception is thrown, some objects in the range ifirst + [0, n) are left in a valid but unspecified state. — end note]

## 26.11.7 uninitialized\_fill

[uninitialized.fill]

```
template<class NoThrowForwardIterator,
 class T = iterator_traits<NoThrowForwardIterator>::value_type>
constexpr void uninitialized_fill(NoThrowForwardIterator first,
 NoThrowForwardIterator last, const T& x);
```

1     *Effects:* Equivalent to:

```
 for (; first != last; ++first)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type(x);
```

```
namespace ranges {
 template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S, class T = iter_value_t<I>>
 requires constructible_from<iter_value_t<I>, const T&>
 constexpr I uninitialized_fill(I first, S last, const T& x);
 template<nothrow-forward-range R, class T = range_value_t<R>>
 requires constructible_from<range_value_t<R>, const T&>
 constexpr borrowed_iterator_t<R> uninitialized_fill(R&& r, const T& x);
}
```

2     *Effects:* Equivalent to:

```
 for (; first != last; ++first)
 ::new (voidify(*first)) remove_reference_t<iter_reference_t<I>>(x);
 return first;
```

```
template<class NoThrowForwardIterator, class Size,
 class T = iterator_traits<NoThrowForwardIterator>::value_type>
constexpr NoThrowForwardIterator
uninitialized_fill_n(NoThrowForwardIterator first, Size n, const T& x);
```

3     *Effects:* Equivalent to:

```
 for (; n--; ++first)
 ::new (voidify(*first)) iterator_traits<NoThrowForwardIterator>::value_type(x);
 return first;
```

```
namespace ranges {
 template<nothrow-forward-iterator I, class T = iter_value_t<I>>
 requires constructible_from<iter_value_t<I>, const T&>
 constexpr I uninitialized_fill_n(I first, iter_difference_t<I> n, const T& x);
}
```

4     *Effects:* Equivalent to:

```
 return uninitialized_fill(counted_iterator(first, n), default_sentinel, x).base();
```

## 26.11.8 construct\_at

[specialized.construct]

```
template<class T, class... Args>
constexpr T* construct_at(T* location, Args&&... args);
```

```
namespace ranges {
 template<class T, class... Args>
 constexpr T* construct_at(T* location, Args&&... args);
}
```

1     *Constraints:* `is_unbounded_array_v<T>` is false. The expression `::new (declval<void*>()) T(declval<Args>()...)` is well-formed when treated as an unevaluated operand (7.2.3).

2     *Mandates:* If `is_array_v<T>` is true, `sizeof...(Args)` is zero.

3     *Effects:* Equivalent to:

```
 if constexpr (is_array_v<T>)
 return ::new (voidify(*location)) T[1]();
 else
 return ::new (voidify(*location)) T(std::forward<Args>(args)...);
```

**26.11.9 destroy****[specialized.destroy]**

```
template<class T>
constexpr void destroy_at(T* location);
namespace ranges {
 template<destructible T>
 constexpr void destroy_at(T* location) noexcept;
}
```

1 *Effects:*

- (1.1) — If T is an array type, equivalent to `destroy(begin(*location), end(*location))`.
- (1.2) — Otherwise, equivalent to `location->~T()`.

```
template<class NoThrowForwardIterator>
constexpr void destroy(NoThrowForwardIterator first, NoThrowForwardIterator last);
```

2 *Effects:* Equivalent to:

```
for (; first != last; ++first)
 destroy_at(addressof(*first));
```

```
namespace ranges {
 template<nothrow-input-iterator I, nothrow-sentinel-for<I> S>
 requires destructible<iter_value_t<I>>
 constexpr I destroy(I first, S last) noexcept;
 template<nothrow-input-range R>
 requires destructible<range_value_t<R>>
 constexpr borrowed_iterator_t<R> destroy(R&& r) noexcept;
}
```

3 *Effects:* Equivalent to:

```
for (; first != last; ++first)
 destroy_at(addressof(*first));
return first;
```

```
template<class NoThrowForwardIterator, class Size>
constexpr NoThrowForwardIterator destroy_n(NoThrowForwardIterator first, Size n);
```

4 *Effects:* Equivalent to:

```
for (; n > 0; (void)++first, --n)
 destroy_at(addressof(*first));
return first;
```

```
namespace ranges {
 template<nothrow-input-iterator I>
 requires destructible<iter_value_t<I>>
 constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept;
}
```

5 *Effects:* Equivalent to:

```
return destroy(counted_iterator(std::move(first), n), default_sentinel).base();
```

**26.12 Specialized <random> algorithms****[alg.rand]****26.12.1 General****[alg.rand.general]**

- 1 The contents specified in 26.12 are declared in the header `<random>` (29.5.2).

**26.12.2 generate\_random****[alg.rand.generate]**

```
template<class R, class G>
requires output_range<R, invoke_result_t<G&&>> && uniform_random_bit_generator<remove_cvref_t<G>>
constexpr borrowed_iterator_t<R> ranges::generate_random(R&& r, G&& g);
```

1 *Effects:*

- (1.1) — Calls `g.generate_random(std::forward<R>(r))` if this expression is well-formed.

- (1.2) — Otherwise, if `R` models `sized_range`, fills `r` with `ranges::size(r)` values of type `invoke_result_t<G&&>` by performing an unspecified number of invocations of the form `g()` or `g.generate_random(s)`, if such an expression is well-formed for a value `N` and an object `s` of type `span<invoke_result_t<G&&>, N>`.

[*Note 1*: Values of `N` can differ between invocations. — *end note*]

- (1.3) — Otherwise, calls `ranges::generate(std::forward<R>(r), ref(g))`.

2 *Returns*: `ranges::end(r)`.

3 *Remarks*: The effects of `generate_random(r, g)` shall be equivalent to `ranges::generate(std::forward<R>(r), ref(g))`.

[*Note 2*: This implies that `g.generate_random(a)` fills `a` with the same values as produced by invocation of `g()`. — *end note*]

```
template<class G, output_iterator<invoke_result_t<G&&>> O, sentinel_for<O> S>
requires uniform_random_bit_generator<remove_cvref_t<G>>
constexpr O ranges::generate_random(O first, S last, G&& g);
```

4 *Effects*: Equivalent to:

```
return generate_random(subrange<O, S>(std::move(first), last), g);
```

```
template<class R, class G, class D>
requires output_range<R, invoke_result_t<D&, G&>> && invocable<D&, G&> &&
uniform_random_bit_generator<remove_cvref_t<G>> &&
is_arithmetic_v<invoke_result_t<D&, G&>>
constexpr borrowed_iterator_t<R> ranges::generate_random(R&& r, G&& g, D&& d);
```

5 *Effects*:

- (5.1) — Calls `d.generate_random(std::forward<R>(r), g)` if this expression is well-formed.

- (5.2) — Otherwise, if `R` models `sized_range`, fills `r` with `ranges::size(r)` values of type `invoke_result_t<D&, G&>` by performing an unspecified number of invocations of the form `invoke(d, g)` or `d.generate_random(s, g)`, if such an expression is well-formed for a value `N` and an object `s` of type `span<invoke_result_t<D&, G&>, N>`.

[*Note 3*: Values of `N` can differ between invocations. — *end note*]

- (5.3) — Otherwise, calls

```
ranges::generate(std::forward<R>(r), [&d, &g] { return invoke(d, g); });
```

6 *Returns*: `ranges::end(r)`.

7 *Remarks*: The effects of `generate_random(r, g, d)` shall be equivalent to

```
ranges::generate(std::forward<R>(r), [&d, &g] { return invoke(d, g); })
```

[*Note 4*: This implies that `d.generate_random(a, g)` fills `a` with the values with the same random distribution as produced by invocation of `invoke(d, g)`. — *end note*]

```
template<class G, class D, output_iterator<invoke_result_t<D&, G&>> O, sentinel_for<O> S>
requires invocable<D&, G&> && uniform_random_bit_generator<remove_cvref_t<G>> &&
is_arithmetic_v<invoke_result_t<D&, G&>>
constexpr O ranges::generate_random(O first, S last, G&& g, D&& d);
```

8 *Effects*: Equivalent to:

```
return generate_random(subrange<O, S>(std::move(first), last), g, d);
```

## 26.13 C library algorithms

[alg.c.library]

- <sup>1</sup> [*Note 1*: The header `<cstdlib>` (17.2.2) declares the functions described in this subclause. — *end note*]

```
void* bsearch(const void* key, void* base, size_t nmemb, size_t size,
 c-compare-pred* compar);
void* bsearch(const void* key, void* base, size_t nmemb, size_t size,
 compare-pred* compar);
const void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
 c-compare-pred* compar);
```

```
const void* bsearch(const void* key, const void* base, size_t nmemb, size_t size,
 compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, c-compare-pred* compar);
void qsort(void* base, size_t nmemb, size_t size, compare-pred* compar);
```

<sup>2</sup>     *Preconditions:* For `qsort`, the objects in the array pointed to by `base` are of trivially copyable type.

<sup>3</sup>     *Effects:* These functions have the semantics specified in the C standard library.

<sup>4</sup>     *Throws:* Any exception thrown by `compar` ([16.4.6.14](#)).

SEE ALSO: ISO/IEC 9899:2024, 7.24.6



## 27 Strings library

[strings]

### 27.1 General

[strings.general]

- <sup>1</sup> This Clause describes components for manipulating sequences of any non-array trivially copyable standard-layout (6.9.1) type `T` where `is_trivially_default_constructible_v<T>` is `true`. Such types are called *char-like types*, and objects of char-like types are called *char-like objects* or simply *characters*.
- <sup>2</sup> The following subclauses describe a character traits class, string classes, and null-terminated sequence utilities, as summarized in Table 86.

Table 86 — Strings library summary [tab:strings.summary]

| Subclause                               | Header        |
|-----------------------------------------|---------------|
| 27.2 Character traits                   | <string>      |
| 27.3 String view classes                | <string_view> |
| 27.4 String classes                     | <string>      |
| 27.5 Null-terminated sequence utilities | <cstring>     |

### 27.2 Character traits

[char.traits]

#### 27.2.1 General

[char.traits.general]

- <sup>1</sup> Subclause 27.2 defines requirements on classes representing *character traits*, and defines a class template `char_traits<charT>`, along with five specializations, `char_traits<char>`, `char_traits<char8_t>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, that meet those requirements.
- <sup>2</sup> Most classes specified in 27.4, 27.3, and Clause 31 need a set of related types and functions to complete the definition of their semantics. These types and functions are provided as a set of member *typedef-names* and functions in the template parameter `traits` used by each such template. Subclause 27.2 defines the semantics of these members.
- <sup>3</sup> To specialize those templates to generate a string, string view, or iostream class to handle a particular character container type (3.10) `C`, that and its related character traits class `X` are passed as a pair of parameters to the string, string view, or iostream template as parameters `charT` and `traits`. If `X::char_type` is not the same type as `C`, the program is ill-formed.

#### 27.2.2 Character traits requirements

[char.traits.require]

- <sup>1</sup> In Table 87, `X` denotes a traits class defining types and functions for the character container type `C`; `c` and `d` denote values of type `C`; `p` and `q` denote values of type `const C*`; `s` denotes a value of type `C*`; `n`, `i` and `j` denote values of type `size_t`; `e` and `f` denote values of type `X::int_type`; `pos` denotes a value of type `X::pos_type`; and `r` denotes an lvalue of type `C`. No expression which is part of the character traits requirements specified in 27.2.2 shall exit via an exception.

Table 87 — Character traits requirements [tab:char.traits.req]

| Expression                 | Return type       | Assertion/note<br>pre-/post-condition                                          | Complexity |
|----------------------------|-------------------|--------------------------------------------------------------------------------|------------|
| <code>X::char_type</code>  | <code>C</code>    |                                                                                |            |
| <code>X::int_type</code>   |                   | (described in 27.2.3)                                                          |            |
| <code>X::off_type</code>   |                   | (described in 31.2.3 and 31.3)                                                 |            |
| <code>X::pos_type</code>   |                   | (described in 31.2.3 and 31.3)                                                 |            |
| <code>X::state_type</code> |                   | (described in 27.2.3)                                                          |            |
| <code>X::eq(c,d)</code>    | <code>bool</code> | Returns: whether <code>c</code> is to be treated as equal to <code>d</code> .  | constant   |
| <code>X::lt(c,d)</code>    | <code>bool</code> | Returns: whether <code>c</code> is to be treated as less than <code>d</code> . | constant   |

Table 87 — Character traits requirements (continued)

| Expression                       | Return type                      | Assertion/note<br>pre-/post-condition                                                                                                                                                                                                                                                                                                                                                                                                                                | Complexity |
|----------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <code>X::compare(p,q,n)</code>   | <code>int</code>                 | <i>Returns:</i> 0 if for each <code>i</code> in <code>[0,n)</code> , <code>X::eq(p[i],q[i])</code> is <code>true</code> ; else, a negative value if, for some <code>j</code> in <code>[0,n)</code> , <code>X::lt(p[j],q[j])</code> is <code>true</code> and for each <code>i</code> in <code>[0,j)</code> <code>X::eq(p[i],q[i])</code> is <code>true</code> ; else a positive value.                                                                                | linear     |
| <code>X::length(p)</code>        | <code>size_t</code>              | <i>Returns:</i> the smallest <code>i</code> such that <code>X::eq(p[i],charT())</code> is <code>true</code> .                                                                                                                                                                                                                                                                                                                                                        | linear     |
| <code>X::find(p,n,c)</code>      | <code>const X::char_type*</code> | <i>Returns:</i> the smallest <code>q</code> in <code>[p,p+n)</code> such that <code>X::eq(*q,c)</code> is <code>true</code> , <code>nullptr</code> otherwise.                                                                                                                                                                                                                                                                                                        | linear     |
| <code>X::move(s,p,n)</code>      | <code>X::char_type*</code>       | for each <code>i</code> in <code>[0,n)</code> , performs <code>X::assign(s[i],p[i])</code> . Copies correctly even where the ranges <code>[p,p+n)</code> and <code>[s,s+n)</code> overlap.<br><i>Returns:</i> <code>s</code> .                                                                                                                                                                                                                                       | linear     |
| <code>X::copy(s,p,n)</code>      | <code>X::char_type*</code>       | <i>Preconditions:</i> The ranges <code>[p,p+n)</code> and <code>[s,s+n)</code> do not overlap.<br><i>Returns:</i> <code>s</code> .<br>for each <code>i</code> in <code>[0,n)</code> , performs <code>X::assign(s[i],p[i])</code> .                                                                                                                                                                                                                                   | linear     |
| <code>X::assign(r,d)</code>      | (not used)                       | assigns <code>r=d</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                           | constant   |
| <code>X::assign(s,n,c)</code>    | <code>X::char_type*</code>       | for each <code>i</code> in <code>[0,n)</code> , performs <code>X::assign(s[i],c)</code> .<br><i>Returns:</i> <code>s</code> .                                                                                                                                                                                                                                                                                                                                        | linear     |
| <code>X::not_eof(e)</code>       | <code>int_type</code>            | <i>Returns:</i> <code>e</code> if <code>X::eq_int_type(e,X::eof())</code> is <code>false</code> , otherwise a value <code>f</code> such that <code>X::eq_int_type(f,X::eof())</code> is <code>false</code> .                                                                                                                                                                                                                                                         | constant   |
| <code>X::to_char_type(e)</code>  | <code>X::char_type</code>        | <i>Returns:</i> if for some <code>c</code> , <code>X::eq_int_type(e,X::to_int_type(c))</code> is <code>true</code> , <code>c</code> ; else some unspecified value.                                                                                                                                                                                                                                                                                                   | constant   |
| <code>X::to_int_type(c)</code>   | <code>X::int_type</code>         | <i>Returns:</i> some value <code>e</code> , constrained by the definitions of <code>to_char_type</code> and <code>eq_int_type</code> .                                                                                                                                                                                                                                                                                                                               | constant   |
| <code>X::eq_int_type(e,f)</code> | <code>bool</code>                | <i>Returns:</i> for all <code>c</code> and <code>d</code> , <code>X::eq(c,d)</code> is equal to <code>X::eq_int_type(X::to_int_type(c),X::to_int_type(d))</code> ; otherwise, yields <code>true</code> if <code>e</code> and <code>f</code> are both copies of <code>X::eof()</code> ; otherwise, yields <code>false</code> if one of <code>e</code> and <code>f</code> is a copy of <code>X::eof()</code> and the other is not; otherwise the value is unspecified. | constant   |

Table 87 — Character traits requirements (continued)

| Expression            | Return type              | Assertion/note<br>pre-/post-condition                                                                                                       | Complexity |
|-----------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <code>X::eof()</code> | <code>X::int_type</code> | <i>Returns:</i> a value <code>e</code> such that <code>X::eq_int_type(e, X::to_int_type(c))</code> is false for all values <code>c</code> . | constant   |

<sup>2</sup> The class template

```
template<class charT> struct char_traits;
```

is provided in the header `<string>` (27.4.2) as a basis for explicit specializations.

### 27.2.3 Traits typedefs

[char.traits.typedefs]

```
using int_type = see below;
```

<sup>1</sup> *Preconditions:* `int_type` shall be able to represent all of the valid characters converted from the corresponding `char_type` values, as well as an end-of-file value, `eof()`.<sup>189</sup>

```
using state_type = see below;
```

<sup>2</sup> *Preconditions:* `state_type` meets the *Cpp17Destructible* (Table 35), *Cpp17CopyAssignable* (Table 34), *Cpp17CopyConstructible* (Table 32), and *Cpp17DefaultConstructible* (Table 30) requirements.

### 27.2.4 char\_traits specializations

[char.traits.specializations]

#### 27.2.4.1 General

[char.traits.specializations.general]

```
namespace std {
 template<> struct char_traits<char>;
 template<> struct char_traits<char8_t>;
 template<> struct char_traits<char16_t>;
 template<> struct char_traits<char32_t>;
 template<> struct char_traits<wchar_t>;
}
```

<sup>1</sup> The header `<string>` defines five specializations of the class template `char_traits`: `char_traits<char>`, `char_traits<char8_t>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`.

#### 27.2.4.2 struct char\_traits<char>

[char.traits.specializations.char]

```
namespace std {
 template<> struct char_traits<char> {
 using char_type = char;
 using int_type = int;
 using off_type = streamoff;
 using pos_type = streampos;
 using state_type = mbstate_t;
 using comparison_category = strong_ordering;

 static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
 static constexpr bool eq(char_type c1, char_type c2) noexcept;
 static constexpr bool lt(char_type c1, char_type c2) noexcept;

 static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
 static constexpr size_t length(const char_type* s);
 static constexpr const char_type* find(const char_type* s, size_t n,
 const char_type& a);
 static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* assign(char_type* s, size_t n, char_type a);
 };
}
```

<sup>189</sup> If `eof()` can be held in `char_type` then some iostreams operations can give surprising results.

```

 static constexpr int_type not_eof(int_type c) noexcept;
 static constexpr char_type to_char_type(int_type c) noexcept;
 static constexpr int_type to_int_type(char_type c) noexcept;
 static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
 static constexpr int_type eof() noexcept;
};
}

```

- <sup>1</sup> The type `mbstate_t` is defined in `<cwchar>` (28.7.3) and can represent any of the conversion states that can occur in an implementation-defined set of supported multibyte character encoding rules.
- <sup>2</sup> The two-argument member `assign` is defined identically to the built-in operator `=`. The two-argument members `eq` and `lt` are defined identically to the built-in operators `==` and `<` for type `unsigned char`.
- <sup>3</sup> The member `eof()` returns EOF.

#### 27.2.4.3 struct `char_traits<char8_t>`

[char.traits.specializations.char8.t]

```

namespace std {
 template<> struct char_traits<char8_t> {
 using char_type = char8_t;
 using int_type = unsigned int;
 using off_type = streamoff;
 using pos_type = u8streampos;
 using state_type = mbstate_t;
 using comparison_category = strong_ordering;

 static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
 static constexpr bool eq(char_type c1, char_type c2) noexcept;
 static constexpr bool lt(char_type c1, char_type c2) noexcept;

 static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
 static constexpr size_t length(const char_type* s);
 static constexpr const char_type* find(const char_type* s, size_t n,
 const char_type& a);
 static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* assign(char_type* s, size_t n, char_type a);
 static constexpr int_type not_eof(int_type c) noexcept;
 static constexpr char_type to_char_type(int_type c) noexcept;
 static constexpr int_type to_int_type(char_type c) noexcept;
 static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
 static constexpr int_type eof() noexcept;
 };
}

```

- <sup>1</sup> The two-argument members `assign`, `eq`, and `lt` are defined identically to the built-in operators `=`, `==`, and `<` respectively.
- <sup>2</sup> The member `eof()` returns an implementation-defined constant that cannot appear as a valid UTF-8 code unit.

#### 27.2.4.4 struct `char_traits<char16_t>`

[char.traits.specializations.char16.t]

```

namespace std {
 template<> struct char_traits<char16_t> {
 using char_type = char16_t;
 using int_type = uint_least16_t;
 using off_type = streamoff;
 using pos_type = u16streampos;
 using state_type = mbstate_t;
 using comparison_category = strong_ordering;

 static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
 static constexpr bool eq(char_type c1, char_type c2) noexcept;
 static constexpr bool lt(char_type c1, char_type c2) noexcept;
 };
}

```

```

static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
static constexpr size_t length(const char_type* s);
static constexpr const char_type* find(const char_type* s, size_t n,
 const char_type& a);

static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
static constexpr char_type* assign(char_type* s, size_t n, char_type a);

static constexpr int_type not_eof(int_type c) noexcept;
static constexpr char_type to_char_type(int_type c) noexcept;
static constexpr int_type to_int_type(char_type c) noexcept;
static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
static constexpr int_type eof() noexcept;
};
}

```

- <sup>1</sup> The two-argument members `assign`, `eq`, and `lt` are defined identically to the built-in operators `=`, `==`, and `<`, respectively.
- <sup>2</sup> The member `eof()` returns an implementation-defined constant that cannot appear as a valid UTF-16 code unit.

#### 27.2.4.5 struct `char_traits<char32_t>`

[[char.traits.specializations.char32.t](#)]

```

namespace std {
 template<> struct char_traits<char32_t> {
 using char_type = char32_t;
 using int_type = uint_least32_t;
 using off_type = streamoff;
 using pos_type = u32streampos;
 using state_type = mbstate_t;
 using comparison_category = strong_ordering;

 static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
 static constexpr bool eq(char_type c1, char_type c2) noexcept;
 static constexpr bool lt(char_type c1, char_type c2) noexcept;

 static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
 static constexpr size_t length(const char_type* s);
 static constexpr const char_type* find(const char_type* s, size_t n,
 const char_type& a);

 static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
 static constexpr char_type* assign(char_type* s, size_t n, char_type a);

 static constexpr int_type not_eof(int_type c) noexcept;
 static constexpr char_type to_char_type(int_type c) noexcept;
 static constexpr int_type to_int_type(char_type c) noexcept;
 static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
 static constexpr int_type eof() noexcept;
 };
}

```

- <sup>1</sup> The two-argument members `assign`, `eq`, and `lt` are defined identically to the built-in operators `=`, `==`, and `<`, respectively.
- <sup>2</sup> The member `eof()` returns an implementation-defined constant that cannot appear as a Unicode code point.

#### 27.2.4.6 struct `char_traits<wchar_t>`

[[char.traits.specializations.wchar.t](#)]

```

namespace std {
 template<> struct char_traits<wchar_t> {
 using char_type = wchar_t;
 using int_type = wint_t;
 using off_type = streamoff;
 using pos_type = wstreampos;
 using state_type = mbstate_t;

```

```

using comparison_category = strong_ordering;

static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
static constexpr bool eq(char_type c1, char_type c2) noexcept;
static constexpr bool lt(char_type c1, char_type c2) noexcept;

static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
static constexpr size_t length(const char_type* s);
static constexpr const char_type* find(const char_type* s, size_t n,
 const char_type& a);
static constexpr char_type* move(char_type* s1, const char_type* s2, size_t n);
static constexpr char_type* copy(char_type* s1, const char_type* s2, size_t n);
static constexpr char_type* assign(char_type* s, size_t n, char_type a);

static constexpr int_type not_eof(int_type c) noexcept;
static constexpr char_type to_char_type(int_type c) noexcept;
static constexpr int_type to_int_type(char_type c) noexcept;
static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
static constexpr int_type eof() noexcept;
};
}

```

- <sup>1</sup> The two-argument members `assign`, `eq`, and `lt` are defined identically to the built-in operators `=`, `==`, and `<`, respectively.
- <sup>2</sup> The member `eof()` returns `WEOF`.

## 27.3 String view classes

[string.view]

### 27.3.1 General

[string.view.general]

- <sup>1</sup> The class template `basic_string_view` describes an object that can refer to a constant contiguous sequence of char-like (27.1) objects with the first element of the sequence at position zero. In the rest of 27.3, the type of the char-like objects held in a `basic_string_view` object is designated by `charT`.
- <sup>2</sup> [Note 1: The library provides implicit conversions from `const charT*` and `std::basic_string<charT, ...>` to `std::basic_string_view<charT, ...>` so that user code can accept just `std::basic_string_view<charT>` as a non-templated parameter wherever a sequence of characters is expected. User-defined types can define their own implicit conversions to `std::basic_string_view<charT>` in order to interoperate with these functions. — end note]

### 27.3.2 Header <string\_view> synopsis

[string.view.synop]

```

// mostly freestanding
#include <compare> // see 17.12.1

namespace std {
 // 27.3.3, class template basic_string_view
 template<class charT, class traits = char_traits<charT>>
 class basic_string_view; // partially freestanding

 template<class charT, class traits>
 constexpr bool ranges::enable_view<basic_string_view<charT, traits>> = true;
 template<class charT, class traits>
 constexpr bool ranges::enable_borrowed_range<basic_string_view<charT, traits>> = true;

 // 27.3.4, non-member comparison functions
 template<class charT, class traits>
 constexpr bool operator==(basic_string_view<charT, traits> x,
 type_identity_t<basic_string_view<charT, traits>> y) noexcept;
 template<class charT, class traits>
 constexpr see below operator<=>(basic_string_view<charT, traits> x,
 type_identity_t<basic_string_view<charT,
 traits>> y) noexcept;
}

```

```

// 27.3.5, inserters and extractors
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 basic_string_view<charT, traits> str); // hosted

// basic_string_view typedef-names
using string_view = basic_string_view<char>;
using u8string_view = basic_string_view<char8_t>;
using u16string_view = basic_string_view<char16_t>;
using u32string_view = basic_string_view<char32_t>;
using wstring_view = basic_string_view<wchar_t>;

// 27.3.6, hash support
template<class T> struct hash;
template<> struct hash<string_view>;
template<> struct hash<u8string_view>;
template<> struct hash<u16string_view>;
template<> struct hash<u32string_view>;
template<> struct hash<wstring_view>;

inline namespace literals {
 inline namespace string_view_literals {
 // 27.3.7, suffix for basic_string_view literals
 constexpr string_view operator""sv(const char* str, size_t len) noexcept;
 constexpr u8string_view operator""sv(const char8_t* str, size_t len) noexcept;
 constexpr u16string_view operator""sv(const char16_t* str, size_t len) noexcept;
 constexpr u32string_view operator""sv(const char32_t* str, size_t len) noexcept;
 constexpr wstring_view operator""sv(const wchar_t* str, size_t len) noexcept;
 }
}

```

- <sup>1</sup> The function templates defined in 22.2.2 and 24.7 are available when `<string_view>` is included.

### 27.3.3 Class template `basic_string_view`

[string.view.template]

#### 27.3.3.1 General

[string.view.template.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_string_view {
 public:
 // types
 using traits_type = traits;
 using value_type = charT;
 using pointer = value_type*;
 using const_pointer = const value_type*;
 using reference = value_type&;
 using const_reference = const value_type&;
 using const_iterator = implementation-defined; // see 27.3.3.4
 using iterator = const_iterator;190
 using const_reverse_iterator = reverse_iterator<const_iterator>;
 using reverse_iterator = const_reverse_iterator;
 using size_type = size_t;
 using difference_type = ptrdiff_t;
 static constexpr size_type npos = size_type(-1);

 // 27.3.3.2, construction and assignment
 constexpr basic_string_view() noexcept;
 constexpr basic_string_view(const basic_string_view&) noexcept = default;
 constexpr basic_string_view& operator=(const basic_string_view&) noexcept = default;
 constexpr basic_string_view(const charT* str);
 basic_string_view(nullptr_t) = delete;
 };
}

```

<sup>190</sup> Because `basic_string_view` refers to a constant sequence, `iterator` and `const_iterator` are the same type.

```

constexpr basic_string_view(const charT* str, size_type len);
template<class It, class End>
 constexpr basic_string_view(It begin, End end);
template<class R>
 constexpr explicit basic_string_view(R&& r);

// 27.3.3.4, iterator support
constexpr const_iterator begin() const noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

// 27.3.3.5, capacity
constexpr size_type size() const noexcept;
constexpr size_type length() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr bool empty() const noexcept;

// 27.3.3.6, element access
constexpr const_reference operator[](size_type pos) const;
constexpr const_reference at(size_type pos) const; // freestanding-deleted
constexpr const_reference front() const;
constexpr const_reference back() const;
constexpr const charT* data() const noexcept;

// 27.3.3.7, modifiers
constexpr void remove_prefix(size_type n);
constexpr void remove_suffix(size_type n);
constexpr void swap(basic_string_view& s) noexcept;

// 27.3.3.8, string operations
constexpr size_type copy(charT* s, size_type n,
 size_type pos = 0) const; // freestanding-deleted

constexpr basic_string_view substr(size_type pos = 0,
 size_type n = npos) const; // freestanding-deleted
constexpr basic_string_view subview(size_type pos = 0,
 size_type n = npos) const; // freestanding-deleted

constexpr int compare(basic_string_view s) const noexcept;
constexpr int compare(size_type pos1, size_type n1,
 basic_string_view s) const; // freestanding-deleted
constexpr int compare(size_type pos1, size_type n1, basic_string_view s,
 size_type pos2, size_type n2) const; // freestanding-deleted
constexpr int compare(const charT* s) const;
constexpr int compare(size_type pos1, size_type n1,
 const charT* s) const; // freestanding-deleted
constexpr int compare(size_type pos1, size_type n1, const charT* s,
 size_type n2) const; // freestanding-deleted

constexpr bool starts_with(basic_string_view x) const noexcept;
constexpr bool starts_with(charT x) const noexcept;
constexpr bool starts_with(const charT* x) const;
constexpr bool ends_with(basic_string_view x) const noexcept;
constexpr bool ends_with(charT x) const noexcept;
constexpr bool ends_with(const charT* x) const;

constexpr bool contains(basic_string_view x) const noexcept;
constexpr bool contains(charT x) const noexcept;
constexpr bool contains(const charT* x) const;

```



```

// 27.3.3.9, searching
constexpr size_type find(basic_string_view s, size_type pos = 0) const noexcept;
constexpr size_type find(charT c, size_type pos = 0) const noexcept;
constexpr size_type find(const charT* s, size_type pos, size_type n) const;
constexpr size_type find(const charT* s, size_type pos = 0) const;
constexpr size_type rfind(basic_string_view s, size_type pos = npos) const noexcept;
constexpr size_type rfind(charT c, size_type pos = npos) const noexcept;
constexpr size_type rfind(const charT* s, size_type pos, size_type n) const;
constexpr size_type rfind(const charT* s, size_type pos = npos) const;

constexpr size_type find_first_of(basic_string_view s, size_type pos = 0) const noexcept;
constexpr size_type find_first_of(charT c, size_type pos = 0) const noexcept;
constexpr size_type find_first_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_first_of(const charT* s, size_type pos = 0) const;
constexpr size_type find_last_of(basic_string_view s, size_type pos = npos) const noexcept;
constexpr size_type find_last_of(charT c, size_type pos = npos) const noexcept;
constexpr size_type find_last_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_last_of(const charT* s, size_type pos = npos) const;
constexpr size_type find_first_not_of(basic_string_view s, size_type pos = 0) const noexcept;
constexpr size_type find_first_not_of(charT c, size_type pos = 0) const noexcept;
constexpr size_type find_first_not_of(const charT* s, size_type pos,
 size_type n) const;
constexpr size_type find_first_not_of(const charT* s, size_type pos = 0) const;
constexpr size_type find_last_not_of(basic_string_view s,
 size_type pos = npos) const noexcept;
constexpr size_type find_last_not_of(charT c, size_type pos = npos) const noexcept;
constexpr size_type find_last_not_of(const charT* s, size_type pos,
 size_type n) const;
constexpr size_type find_last_not_of(const charT* s, size_type pos = npos) const;

private:
 const charT* data_; // exposition only
 size_type size_; // exposition only
};

// 27.3.3.3, deduction guides
template<class It, class End>
 basic_string_view(It, End) -> basic_string_view<iter_value_t<It>>;
template<class R>
 basic_string_view(R&&) -> basic_string_view<ranges::range_value_t<R>>;
}

```

- <sup>1</sup> In every specialization `basic_string_view<charT, traits>`, the type `traits` shall meet the character traits requirements (27.2).

[Note 1: The program is ill-formed if `traits::char_type` is not the same type as `charT`. — end note]

- <sup>2</sup> For a `basic_string_view str`, any operation that invalidates a pointer in the range

[`str.data()`, `str.data() + str.size()`)

invalidates pointers, iterators, and references to elements of `str`.

- <sup>3</sup> The complexity of `basic_string_view` member functions is  $\mathcal{O}(1)$  unless otherwise specified.
- <sup>4</sup> `basic_string_view<charT, traits>` is a trivially copyable type (6.9.1).

### 27.3.3.2 Construction and assignment

[string.view.cons]

```
constexpr basic_string_view() noexcept;
```

- <sup>1</sup> *Postconditions:* `size_ == 0` and `data_ == nullptr`.

```
constexpr basic_string_view(const charT* str);
```

- <sup>2</sup> *Preconditions:* [`str`, `str + traits::length(str)`) is a valid range.

- <sup>3</sup> *Effects:* Constructs a `basic_string_view`, initializing `data_` with `str` and initializing `size_` with `traits::length(str)`.

4     *Complexity:*  $\mathcal{O}(\text{traits}::\text{length}(\text{str}))$ .

```
constexpr basic_string_view(const charT* str, size_type len);
```

5     *Preconditions:* `[str, str + len)` is a valid range.

6     *Effects:* Constructs a `basic_string_view`, initializing *data\_* with `str` and initializing *size\_* with `len`.

```
template<class It, class End>
constexpr basic_string_view(It begin, End end);
```

7     *Constraints:*

- (7.1)     — It satisfies `contiguous_iterator`.
- (7.2)     — `End` satisfies `sized_sentinel_for<It>`.
- (7.3)     — `is_same_v<iter_value_t<It>, charT>` is true.
- (7.4)     — `is_convertible_v<End, size_type>` is false.

8     *Preconditions:*

- (8.1)     — `[begin, end)` is a valid range.
- (8.2)     — It models `contiguous_iterator`.
- (8.3)     — `End` models `sized_sentinel_for<It>`.

9     *Effects:* Initializes *data\_* with `to_address(begin)` and initializes *size\_* with `end - begin`.

10    *Throws:* When and what `end - begin` throws.

```
template<class R>
constexpr explicit basic_string_view(R&& r);
```

11    Let `d` be an lvalue of type `remove_cvref_t<R>`.

12    *Constraints:*

- (12.1)    — `remove_cvref_t<R>` is not the same type as `basic_string_view`,
- (12.2)    — `R` models `ranges::contiguous_range` and `ranges::sized_range`,
- (12.3)    — `is_same_v<ranges::range_value_t<R>, charT>` is true,
- (12.4)    — `is_convertible_v<R, const charT*>` is false, and
- (12.5)    — `d.operator::std::basic_string_view<charT, traits>()` is not a valid expression.

13    *Effects:* Initializes *data\_* with `ranges::data(r)` and *size\_* with `ranges::size(r)`.

14    *Throws:* Any exception thrown by `ranges::data(r)` and `ranges::size(r)`.

### 27.3.3.3 Deduction guides

[string.view.deduct]

```
template<class It, class End>
basic_string_view(It, End) -> basic_string_view<iter_value_t<It>>;
```

1     *Constraints:*

- (1.1)     — It satisfies `contiguous_iterator`.
- (1.2)     — `End` satisfies `sized_sentinel_for<It>`.

```
template<class R>
basic_string_view(R&&) -> basic_string_view<ranges::range_value_t<R>>;
```

2     *Constraints:* `R` satisfies `ranges::contiguous_range`.

### 27.3.3.4 Iterator support

[string.view.iterators]

```
using const_iterator = implementation-defined;
```

1     A type that meets the requirements of a constant *Cpp17RandomAccessIterator* (24.3.5.7), models `contiguous_iterator` (24.3.4.14), and meets the `constexpr` iterator requirements (24.3.1), whose *value\_type* is the template parameter `charT`.

2     All requirements on container iterators (23.2) apply to `basic_string_view::const_iterator` as well.

```
constexpr const_iterator begin() const noexcept;
constexpr const_iterator cbegin() const noexcept;
```

- 3       *Returns:* An iterator such that
- (3.1)       — if `!empty()`, `addressof(*begin()) == data_`,
- (3.2)       — otherwise, an unspecified value such that `[begin(), end())` is a valid range.

```
constexpr const_iterator end() const noexcept;
constexpr const_iterator cend() const noexcept;
```

- 4       *Returns:* `begin() + size()`.

```
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
```

- 5       *Returns:* `const_reverse_iterator(end())`.

```
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;
```

- 6       *Returns:* `const_reverse_iterator(begin())`.

### 27.3.3.5 Capacity

[string.view.capacity]

```
constexpr size_type size() const noexcept;
constexpr size_type length() const noexcept;
```

- 1       *Returns:* `size_`.

```
constexpr size_type max_size() const noexcept;
```

- 2       *Returns:* The largest possible number of char-like objects that can be referred to by a `basic_string_view`.

```
constexpr bool empty() const noexcept;
```

- 3       *Returns:* `size_ == 0`.

### 27.3.3.6 Element access

[string.view.access]

```
constexpr const_reference operator[](size_type pos) const;
```

- 1       *Hardened preconditions:* `pos < size()` is true.

[Note 1: This precondition is stronger than the one on `basic_string::operator[]`. — end note]

- 2       *Returns:* `data_[pos]`.

- 3       *Throws:* Nothing.

```
constexpr const_reference at(size_type pos) const;
```

- 4       *Returns:* `data_[pos]`.

- 5       *Throws:* `out_of_range` if `pos >= size()`.

```
constexpr const_reference front() const;
```

- 6       *Hardened preconditions:* `empty()` is false.

- 7       *Returns:* `data_[0]`.

- 8       *Throws:* Nothing.

```
constexpr const_reference back() const;
```

- 9       *Hardened preconditions:* `empty()` is false.

- 10       *Returns:* `data_[size() - 1]`.

- 11       *Throws:* Nothing.

```
constexpr const charT* data() const noexcept;
```

- 12       *Returns:* `data_`.

13 [Note 2: Unlike `basic_string::data()` and *string-literals*, `data()` can return a pointer to a buffer that is not null-terminated. Therefore it is typically a mistake to pass `data()` to a function that takes just a `const charT*` and expects a null-terminated string. — end note]

### 27.3.3.7 Modifiers

[string.view.modifiers]

```
constexpr void remove_prefix(size_type n);
```

1 *Hardened preconditions:* `n <= size()` is true.

2 *Effects:* Equivalent to: `data_ += n; size_ -= n;`

```
constexpr void remove_suffix(size_type n);
```

3 *Hardened preconditions:* `n <= size()` is true.

4 *Effects:* Equivalent to: `size_ -= n;`

```
constexpr void swap(basic_string_view& s) noexcept;
```

5 *Effects:* Exchanges the values of `*this` and `s`.

### 27.3.3.8 String operations

[string.view.ops]

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

1 Let `rlen` be the smaller of `n` and `size() - pos`.

2 *Preconditions:* `[s, s + rlen)` is a valid range.

3 *Effects:* Equivalent to `traits::copy(s, data() + pos, rlen)`.

4 *Returns:* `rlen`.

5 *Throws:* `out_of_range` if `pos > size()`.

6 *Complexity:*  $\mathcal{O}(\text{rlen})$ .

```
constexpr basic_string_view substr(size_type pos = 0, size_type n = npos) const;
```

```
constexpr basic_string_view subview(size_type pos = 0, size_type n = npos) const;
```

7 Let `rlen` be the smaller of `n` and `size() - pos`.

8 *Effects:* Determines `rlen`, the effective length of the string to reference.

9 *Returns:* `basic_string_view(data() + pos, rlen)`.

10 *Throws:* `out_of_range` if `pos > size()`.

```
constexpr int compare(basic_string_view str) const noexcept;
```

11 Let `rlen` be the smaller of `size()` and `str.size()`.

12 *Effects:* Determines `rlen`, the effective length of the strings to compare. The function then compares the two strings by calling `traits::compare(data(), str.data(), rlen)`.

13 *Returns:* The nonzero result if the result of the comparison is nonzero. Otherwise, returns a value as indicated in Table 88.

Table 88 — `compare()` results [tab:string.view.compare]

| Condition                           | Return Value |
|-------------------------------------|--------------|
| <code>size() &lt; str.size()</code> | < 0          |
| <code>size() == str.size()</code>   | 0            |
| <code>size() &gt; str.size()</code> | > 0          |

14 *Complexity:*  $\mathcal{O}(\text{rlen})$ .

```
constexpr int compare(size_type pos1, size_type n1, basic_string_view str) const;
```

15 *Effects:* Equivalent to: `return substr(pos1, n1).compare(str);`

```

constexpr int compare(size_type pos1, size_type n1, basic_string_view str,
 size_type pos2, size_type n2) const;
16 Effects: Equivalent to: return substr(pos1, n1).compare(str.substr(pos2, n2));

constexpr int compare(const charT* s) const;
17 Effects: Equivalent to: return compare(basic_string_view(s));

constexpr int compare(size_type pos1, size_type n1, const charT* s) const;
18 Effects: Equivalent to: return substr(pos1, n1).compare(basic_string_view(s));

constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const;
19 Effects: Equivalent to: return substr(pos1, n1).compare(basic_string_view(s, n2));

constexpr bool starts_with(basic_string_view x) const noexcept;
20 Let rlen be the smaller of size() and x.size().
21 Effects: Equivalent to: return basic_string_view(data(), rlen) == x;

constexpr bool starts_with(charT x) const noexcept;
22 Effects: Equivalent to: return !empty() && traits::eq(front(), x);

constexpr bool starts_with(const charT* x) const;
23 Effects: Equivalent to: return starts_with(basic_string_view(x));

constexpr bool ends_with(basic_string_view x) const noexcept;
24 Let rlen be the smaller of size() and x.size().
25 Effects: Equivalent to:
 return basic_string_view(data() + (size() - rlen), rlen) == x;

constexpr bool ends_with(charT x) const noexcept;
26 Effects: Equivalent to: return !empty() && traits::eq(back(), x);

constexpr bool ends_with(const charT* x) const;
27 Effects: Equivalent to: return ends_with(basic_string_view(x));

constexpr bool contains(basic_string_view x) const noexcept;
constexpr bool contains(charT x) const noexcept;
constexpr bool contains(const charT* x) const;
28 Effects: Equivalent to: return find(x) != npos;

```

### 27.3.3.9 Searching

[string.view.find]

- 1 Member functions in this subclause have complexity  $\mathcal{O}(\text{size()} * \text{str.size}())$  at worst, although implementations should do better.
- 2 Let  $F$  be one of `find`, `rfind`, `find_first_of`, `find_last_of`, `find_first_not_of`, and `find_last_not_of`.
  - (2.1) — Each member function of the form
 

```
constexpr return-type F(const charT* s, size_type pos) const;
```

 has effects equivalent to: `return F(basic_string_view(s), pos);`
  - (2.2) — Each member function of the form
 

```
constexpr return-type F(const charT* s, size_type pos, size_type n) const;
```

 has effects equivalent to: `return F(basic_string_view(s, n), pos);`
  - (2.3) — Each member function of the form
 

```
constexpr return-type F(charT c, size_type pos) const noexcept;
```

 has effects equivalent to: `return F(basic_string_view(addressof(c), 1), pos);`

```
constexpr size_type find(basic_string_view str, size_type pos = 0) const noexcept;
```

Let `xpos` be the lowest position, if possible, such that the following conditions hold:

- (3.1) — `pos <= xpos`
- (3.2) — `xpos + str.size() <= size()`
- (3.3) — `traits::eq(data_[xpos + I], str[I])` for all elements `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

```
constexpr size_type rfind(basic_string_view str, size_type pos = npos) const noexcept;
```

Let `xpos` be the highest position, if possible, such that the following conditions hold:

- (6.1) — `xpos <= pos`
- (6.2) — `xpos + str.size() <= size()`
- (6.3) — `traits::eq(data_[xpos + I], str[I])` for all elements `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

```
constexpr size_type find_first_of(basic_string_view str, size_type pos = 0) const noexcept;
```

Let `xpos` be the lowest position, if possible, such that the following conditions hold:

- (9.1) — `pos <= xpos`
- (9.2) — `xpos < size()`
- (9.3) — `traits::eq(data_[xpos], str[I])` for some element `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

```
constexpr size_type find_last_of(basic_string_view str, size_type pos = npos) const noexcept;
```

Let `xpos` be the highest position, if possible, such that the following conditions hold:

- (12.1) — `xpos <= pos`
- (12.2) — `xpos < size()`
- (12.3) — `traits::eq(data_[xpos], str[I])` for some element `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

```
constexpr size_type find_first_not_of(basic_string_view str, size_type pos = 0) const noexcept;
```

Let `xpos` be the lowest position, if possible, such that the following conditions hold:

- (15.1) — `pos <= xpos`
- (15.2) — `xpos < size()`
- (15.3) — `traits::eq(data_[xpos], str[I])` for no element `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

```
constexpr size_type find_last_not_of(basic_string_view str, size_type pos = npos) const noexcept;
```

Let `xpos` be the highest position, if possible, such that the following conditions hold:

- (18.1) — `xpos <= pos`
- (18.2) — `xpos < size()`
- (18.3) — `traits::eq(data_[xpos], str[I])` for no element `I` of the string referenced by `str`.

*Effects:* Determines `xpos`.

*Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

### 27.3.4 Non-member comparison functions

[string.view.comparison]

```
template<class charT, class traits>
constexpr bool operator==(basic_string_view<charT, traits> lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs) noexcept;
```

1 *Returns:* lhs.compare(rhs) == 0.

```
template<class charT, class traits>
constexpr see below operator<=>(basic_string_view<charT, traits> lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs) noexcept;
```

2 Let R denote the type traits::comparison\_category if that *qualified-id* is valid and denotes a type (13.10.3), otherwise R is weak\_ordering.

3 *Mandates:* R denotes a comparison category type (17.12.2).

4 *Returns:* static\_cast<R>(lhs.compare(rhs) <=> 0).

5 [Note 1: The usage of type\_identity\_t as parameter ensures that an object of type basic\_string\_view<charT, traits> can always be compared with an object of a type T with an implicit conversion to basic\_string\_view<charT, traits>, and vice versa, as per 12.2.2.3. — end note]

### 27.3.5 Inserters and extractors

[string.view.io]

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, basic_string_view<charT, traits> str);
```

1 *Effects:* Behaves as a formatted output function (31.7.6.3.1) of os. Forms a character sequence seq, initially consisting of the elements defined by the range [str.begin(), str.end()). Determines padding for seq as described in 31.7.6.3.1. Then inserts seq as if by calling os.rdbuf()->sputn(seq, n), where n is the larger of os.width() and str.size(); then calls os.width(0).

2 *Returns:* os.

### 27.3.6 Hash support

[string.view.hash]

```
template<> struct hash<string_view>;
template<> struct hash<u8string_view>;
template<> struct hash<u16string_view>;
template<> struct hash<u32string_view>;
template<> struct hash<wstring_view>;
```

1 The specialization is enabled (22.10.19).

[Note 1: The hash value of a string view object is equal to the hash value of the corresponding string object (27.4.6). — end note]

### 27.3.7 Suffix for basic\_string\_view literals

[string.view.literals]

```
constexpr string_view operator""sv(const char* str, size_t len) noexcept;
```

1 *Returns:* string\_view{str, len}.

```
constexpr u8string_view operator""sv(const char8_t* str, size_t len) noexcept;
```

2 *Returns:* u8string\_view{str, len}.

```
constexpr u16string_view operator""sv(const char16_t* str, size_t len) noexcept;
```

3 *Returns:* u16string\_view{str, len}.

```
constexpr u32string_view operator""sv(const char32_t* str, size_t len) noexcept;
```

4 *Returns:* u32string\_view{str, len}.

```
constexpr wstring_view operator""sv(const wchar_t* str, size_t len) noexcept;
```

5 *Returns:* wstring\_view{str, len}.

## 27.4 String classes

[string.classes]

### 27.4.1 General

[string.classes.general]

- <sup>1</sup> The header `<string>` defines the `basic_string` class template for manipulating varying-length sequences of char-like objects and five *typedef-names*, `string`, `u8string`, `u16string`, `u32string`, and `wstring`, that name the specializations `basic_string<char>`, `basic_string<char8_t>`, `basic_string<char16_t>`, `basic_string<char32_t>`, and `basic_string<wchar_t>`, respectively.

### 27.4.2 Header `<string>` synopsis

[string.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 27.2, character traits
 template<class charT> struct char_traits; // freestanding
 template<> struct char_traits<char>; // freestanding
 template<> struct char_traits<char8_t>; // freestanding
 template<> struct char_traits<char16_t>; // freestanding
 template<> struct char_traits<char32_t>; // freestanding
 template<> struct char_traits<wchar_t>; // freestanding

 // 27.4.3, basic_string
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_string;

 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(const basic_string<charT, traits, Allocator>& lhs,
 const basic_string<charT, traits, Allocator>& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(basic_string<charT, traits, Allocator>&& lhs,
 const basic_string<charT, traits, Allocator>& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(const basic_string<charT, traits, Allocator>& lhs,
 basic_string<charT, traits, Allocator>&& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(basic_string<charT, traits, Allocator>&& lhs,
 basic_string<charT, traits, Allocator>&& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(const charT* lhs,
 const basic_string<charT, traits, Allocator>& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(const charT* lhs,
 basic_string<charT, traits, Allocator>&& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(charT lhs,
 const basic_string<charT, traits, Allocator>& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(charT lhs,
 basic_string<charT, traits, Allocator>&& rhs);
 template<class charT, class traits, class Allocator>
 constexpr basic_string<charT, traits, Allocator>
 operator+(const basic_string<charT, traits, Allocator>& lhs,
 const charT* rhs);
```



```

template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 const charT* rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
 charT rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 charT rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
 const basic_string<charT, traits, Allocator>& rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
 basic_string<charT, traits, Allocator>&& rhs);

template<class charT, class traits, class Allocator>
constexpr bool
operator==(const basic_string<charT, traits, Allocator>& lhs,
 const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr bool operator==(const basic_string<charT, traits, Allocator>& lhs,
 const charT* rhs);

template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
 const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
 const charT* rhs);

// 27.4.4.3, swap
template<class charT, class traits, class Allocator>
constexpr void
swap(basic_string<charT, traits, Allocator>& lhs,
 basic_string<charT, traits, Allocator>& rhs)
noexcept(noexcept(lhs.swap(rhs)));

// 27.4.4.4, inserters and extractors
template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is,
 basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
 const basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
getline(basic_istream<charT, traits>& is,
 basic_string<charT, traits, Allocator>& str,

```

```

 charT delim);
template<class charT, class traits, class Allocator>
 basic_istream<charT, traits>&
 getline(basic_istream<charT, traits>&& is,
 basic_string<charT, traits, Allocator>& str,
 charT delim);
template<class charT, class traits, class Allocator>
 basic_istream<charT, traits>&
 getline(basic_istream<charT, traits>& is,
 basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
 basic_istream<charT, traits>&
 getline(basic_istream<charT, traits>&& is,
 basic_string<charT, traits, Allocator>& str);

// 27.4.4.5, erasure
template<class charT, class traits, class Allocator, class U = charT>
 constexpr typename basic_string<charT, traits, Allocator>::size_type
 erase(basic_string<charT, traits, Allocator>& c, const U& value);
template<class charT, class traits, class Allocator, class Predicate>
 constexpr typename basic_string<charT, traits, Allocator>::size_type
 erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);

// basic_string typedef-names
using string = basic_string<char>;
using u8string = basic_string<char8_t>;
using u16string = basic_string<char16_t>;
using u32string = basic_string<char32_t>;
using wstring = basic_string<wchar_t>;

// 27.4.5, numeric conversions
int stoi(const string& str, size_t* idx = nullptr, int base = 10);
long stol(const string& str, size_t* idx = nullptr, int base = 10);
unsigned long stoul(const string& str, size_t* idx = nullptr, int base = 10);
long long stoll(const string& str, size_t* idx = nullptr, int base = 10);
unsigned long long stoull(const string& str, size_t* idx = nullptr, int base = 10);
float stof(const string& str, size_t* idx = nullptr);
double stod(const string& str, size_t* idx = nullptr);
long double stold(const string& str, size_t* idx = nullptr);
constexpr string to_string(int val);
constexpr string to_string(unsigned val);
constexpr string to_string(long val);
constexpr string to_string(unsigned long val);
constexpr string to_string(long long val);
constexpr string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);

int stoi(const wstring& str, size_t* idx = nullptr, int base = 10);
long stol(const wstring& str, size_t* idx = nullptr, int base = 10);
unsigned long stoul(const wstring& str, size_t* idx = nullptr, int base = 10);
long long stoll(const wstring& str, size_t* idx = nullptr, int base = 10);
unsigned long long stoull(const wstring& str, size_t* idx = nullptr, int base = 10);
float stof(const wstring& str, size_t* idx = nullptr);
double stod(const wstring& str, size_t* idx = nullptr);
long double stold(const wstring& str, size_t* idx = nullptr);
constexpr wstring to_wstring(int val);
constexpr wstring to_wstring(unsigned val);
constexpr wstring to_wstring(long val);
constexpr wstring to_wstring(unsigned long val);
constexpr wstring to_wstring(long long val);
constexpr wstring to_wstring(unsigned long long val);
wstring to_wstring(float val);

```

```

wstring to_wstring(double val);
wstring to_wstring(long double val);

namespace pmr {
 template<class charT, class traits = char_traits<charT>>
 using basic_string = std::basic_string<charT, traits, polymorphic_allocator<charT>>;

 using string = basic_string<char>;
 using u8string = basic_string<char8_t>;
 using u16string = basic_string<char16_t>;
 using u32string = basic_string<char32_t>;
 using wstring = basic_string<wchar_t>;
}

// 27.4.6, hash support
template<class T> struct hash;
template<class A> struct hash<basic_string<char, char_traits<char>, A>>;
template<class A> struct hash<basic_string<char8_t, char_traits<char8_t>, A>>;
template<class A> struct hash<basic_string<char16_t, char_traits<char16_t>, A>>;
template<class A> struct hash<basic_string<char32_t, char_traits<char32_t>, A>>;
template<class A> struct hash<basic_string<wchar_t, char_traits<wchar_t>, A>>;

inline namespace literals {
 inline namespace string_literals {
 // 27.4.7, suffix for basic_string literals
 constexpr string operator""s(const char* str, size_t len);
 constexpr u8string operator""s(const char8_t* str, size_t len);
 constexpr u16string operator""s(const char16_t* str, size_t len);
 constexpr u32string operator""s(const char32_t* str, size_t len);
 constexpr wstring operator""s(const wchar_t* str, size_t len);
 }
}

```

## 27.4.3 Class template `basic_string`

[basic.string]

### 27.4.3.1 General

[basic.string.general]

- <sup>1</sup> The class template `basic_string` describes objects that can store a sequence consisting of a varying number of arbitrary char-like objects with the first element of the sequence at position zero. Such a sequence is also called a “string” if the type of the char-like objects that it holds is clear from context. In the rest of 27.4.3, the type of the char-like objects held in a `basic_string` object is designated by `charT`.
- <sup>2</sup> A specialization of `basic_string` is a contiguous container (23.2.2.2).
- <sup>3</sup> In all cases, `[data(), data() + size())` is a valid range, `data() + size()` points at an object with value `charT()` (a “null terminator”), and `size() <= capacity()` is true. The program has undefined behavior if the null terminator is modified to any value other than `charT()`.

```

namespace std {
 template<class charT, class traits = char_traits<charT>,
 class Allocator = allocator<charT>>
 class basic_string {
 public:
 // types
 using traits_type = traits;
 using value_type = charT;
 using allocator_type = Allocator;
 using size_type = allocator_traits<Allocator>::size_type;
 using difference_type = allocator_traits<Allocator>::difference_type;
 using pointer = allocator_traits<Allocator>::pointer;
 using const_pointer = allocator_traits<Allocator>::const_pointer;
 using reference = value_type&;
 using const_reference = const value_type&;
 };
}

```

```

using iterator = implementation-defined; // see 23.2
using const_iterator = implementation-defined; // see 23.2
using reverse_iterator = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;
static constexpr size_type npos = size_type(-1);

// 27.4.3.3, construct/copy/destroy
constexpr basic_string() noexcept(noexcept(Allocator())) : basic_string(Allocator()) { }
constexpr explicit basic_string(const Allocator& a) noexcept;
constexpr basic_string(const basic_string& str);
constexpr basic_string(basic_string&& str) noexcept;
constexpr basic_string(const basic_string& str, size_type pos,
 const Allocator& a = Allocator());
constexpr basic_string(const basic_string& str, size_type pos, size_type n,
 const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos,
 const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, size_type n,
 const Allocator& a = Allocator());
template<class T>
 constexpr basic_string(const T& t, size_type pos, size_type n,
 const Allocator& a = Allocator());
template<class T>
 constexpr explicit basic_string(const T& t, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
constexpr basic_string(const charT* s, const Allocator& a = Allocator());
basic_string(nullptr_t) = delete;
constexpr basic_string(size_type n, charT c, const Allocator& a = Allocator());
template<class InputIterator>
 constexpr basic_string(InputIterator begin, InputIterator end,
 const Allocator& a = Allocator());
template<container-compatible-range<charT> R>
 constexpr basic_string(from_range_t, R&& rg, const Allocator& a = Allocator());
constexpr basic_string(initializer_list<charT>, const Allocator& a = Allocator());
constexpr basic_string(const basic_string&, const Allocator&);
constexpr basic_string(basic_string&&, const Allocator&);
constexpr ~basic_string();

constexpr basic_string& operator=(const basic_string& str);
constexpr basic_string& operator=(basic_string&& str)
 noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 allocator_traits<Allocator>::is_always_equal::value);
template<class T>
 constexpr basic_string& operator=(const T& t);
constexpr basic_string& operator=(const charT* s);
basic_string& operator=(nullptr_t) = delete;
constexpr basic_string& operator=(charT c);
constexpr basic_string& operator=(initializer_list<charT>);

// 27.4.3.4, iterators
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;

constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;

constexpr const_iterator cbegin() const noexcept;
constexpr const_iterator cend() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;

```

```

// 27.4.3.5, capacity
constexpr size_type size() const noexcept;
constexpr size_type length() const noexcept;
constexpr size_type max_size() const noexcept;
constexpr void resize(size_type n, charT c);
constexpr void resize(size_type n);
template<class Operation> constexpr void resize_and_overwrite(size_type n, Operation op);
constexpr size_type capacity() const noexcept;
constexpr void reserve(size_type res_arg);
constexpr void shrink_to_fit();
constexpr void clear() noexcept;
constexpr bool empty() const noexcept;

// 27.4.3.6, element access
constexpr const_reference operator[](size_type pos) const;
constexpr reference operator[](size_type pos);
constexpr const_reference at(size_type n) const;
constexpr reference at(size_type n);

constexpr const_reference front() const;
constexpr reference front();
constexpr const_reference back() const;
constexpr reference back();

// 27.4.3.7, modifiers
constexpr basic_string& operator+=(const basic_string& str);
template<class T>
 constexpr basic_string& operator+=(const T& t);
constexpr basic_string& operator+=(const charT* s);
constexpr basic_string& operator+=(charT c);
constexpr basic_string& operator+=(initializer_list<charT>);
constexpr basic_string& append(const basic_string& str);
constexpr basic_string& append(const basic_string& str, size_type pos, size_type n = npos);
template<class T>
 constexpr basic_string& append(const T& t);
template<class T>
 constexpr basic_string& append(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& append(const charT* s, size_type n);
constexpr basic_string& append(const charT* s);
constexpr basic_string& append(const charT* s, size_type pos, size_type n);
constexpr basic_string& append(size_type n, charT c);
template<class InputIterator>
 constexpr basic_string& append(InputIterator first, InputIterator last);
template<container_compatible_range<charT> R>
 constexpr basic_string& append_range(R&& rg);
constexpr basic_string& append(initializer_list<charT>);

constexpr void push_back(charT c);

constexpr basic_string& assign(const basic_string& str);
constexpr basic_string& assign(basic_string&& str)
 noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 allocator_traits<Allocator>::is_always_equal::value);
constexpr basic_string& assign(const basic_string& str, size_type pos, size_type n = npos);
template<class T>
 constexpr basic_string& assign(const T& t);
template<class T>
 constexpr basic_string& assign(const T& t, size_type pos, size_type n = npos);
constexpr basic_string& assign(const charT* s, size_type n);
constexpr basic_string& assign(const charT* s);
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);
constexpr basic_string& assign(size_type n, charT c);
template<class InputIterator>
 constexpr basic_string& assign(InputIterator first, InputIterator last);

```

```

template<container-compatible-range<charT> R>
 constexpr basic_string& assign_range(R&& rg);
constexpr basic_string& assign(initializer_list<charT>);

constexpr basic_string& insert(size_type pos, const basic_string& str);
constexpr basic_string& insert(size_type pos1, const basic_string& str,
 size_type pos2, size_type n = npos);

template<class T>
 constexpr basic_string& insert(size_type pos, const T& t);
template<class T>
 constexpr basic_string& insert(size_type pos1, const T& t,
 size_type pos2, size_type n = npos);
constexpr basic_string& insert(size_type pos, const charT* s, size_type n);
constexpr basic_string& insert(size_type pos, const charT* s);
constexpr basic_string& insert(size_type pos, size_type n, charT c);
constexpr iterator insert(const_iterator p, charT c);
constexpr iterator insert(const_iterator p, size_type n, charT c);
template<class InputIterator>
 constexpr iterator insert(const_iterator p, InputIterator first, InputIterator last);
template<container-compatible-range<charT> R>
 constexpr iterator insert_range(const_iterator p, R&& rg);
constexpr iterator insert(const_iterator p, initializer_list<charT>);

constexpr basic_string& erase(size_type pos = 0, size_type n = npos);
constexpr iterator erase(const_iterator p);
constexpr iterator erase(const_iterator first, const_iterator last);

constexpr void pop_back();

constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str);
constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str,
 size_type pos2, size_type n2 = npos);

template<class T>
 constexpr basic_string& replace(size_type pos1, size_type n1, const T& t);
template<class T>
 constexpr basic_string& replace(size_type pos1, size_type n1, const T& t,
 size_type pos2, size_type n2 = npos);
constexpr basic_string& replace(size_type pos, size_type n1, const charT* s, size_type n2);
constexpr basic_string& replace(size_type pos, size_type n1, const charT* s);
constexpr basic_string& replace(size_type pos, size_type n1, size_type n2, charT c);
constexpr basic_string& replace(const_iterator i1, const_iterator i2,
 const basic_string& str);

template<class T>
 constexpr basic_string& replace(const_iterator i1, const_iterator i2, const T& t);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s,
 size_type n);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s);
constexpr basic_string& replace(const_iterator i1, const_iterator i2, size_type n, charT c);
template<class InputIterator>
 constexpr basic_string& replace(const_iterator i1, const_iterator i2,
 InputIterator j1, InputIterator j2);
template<container-compatible-range<charT> R>
 constexpr basic_string& replace_with_range(const_iterator i1, const_iterator i2, R&& rg);
constexpr basic_string& replace(const_iterator, const_iterator, initializer_list<charT>);

constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;

constexpr void swap(basic_string& str)
 noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
 allocator_traits<Allocator>::is_always_equal::value);

// 27.4.3.8, string operations
constexpr const charT* c_str() const noexcept;
constexpr const charT* data() const noexcept;

```

```

constexpr charT* data() noexcept;
constexpr operator basic_string_view<charT, traits>() const noexcept;
constexpr allocator_type get_allocator() const noexcept;

template<class T>
 constexpr size_type find(const T& t, size_type pos = 0) const noexcept(see below);
constexpr size_type find(const basic_string& str, size_type pos = 0) const noexcept;
constexpr size_type find(const charT* s, size_type pos, size_type n) const;
constexpr size_type find(const charT* s, size_type pos = 0) const;
constexpr size_type find(charT c, size_type pos = 0) const noexcept;
template<class T>
 constexpr size_type rfind(const T& t, size_type pos = npos) const noexcept(see below);
constexpr size_type rfind(const basic_string& str, size_type pos = npos) const noexcept;
constexpr size_type rfind(const charT* s, size_type pos, size_type n) const;
constexpr size_type rfind(const charT* s, size_type pos = npos) const;
constexpr size_type rfind(charT c, size_type pos = npos) const noexcept;

template<class T>
 constexpr size_type find_first_of(const T& t, size_type pos = 0) const noexcept(see below);
constexpr size_type find_first_of(const basic_string& str, size_type pos = 0) const noexcept;
constexpr size_type find_first_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_first_of(const charT* s, size_type pos = 0) const;
constexpr size_type find_first_of(charT c, size_type pos = 0) const noexcept;
template<class T>
 constexpr size_type find_last_of(const T& t,
 size_type pos = npos) const noexcept(see below);
constexpr size_type find_last_of(const basic_string& str,
 size_type pos = npos) const noexcept;
constexpr size_type find_last_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_last_of(const charT* s, size_type pos = npos) const;
constexpr size_type find_last_of(charT c, size_type pos = npos) const noexcept;

template<class T>
 constexpr size_type find_first_not_of(const T& t,
 size_type pos = 0) const noexcept(see below);
constexpr size_type find_first_not_of(const basic_string& str,
 size_type pos = 0) const noexcept;
constexpr size_type find_first_not_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_first_not_of(const charT* s, size_type pos = 0) const;
constexpr size_type find_first_not_of(charT c, size_type pos = 0) const noexcept;
template<class T>
 constexpr size_type find_last_not_of(const T& t,
 size_type pos = npos) const noexcept(see below);
constexpr size_type find_last_not_of(const basic_string& str,
 size_type pos = npos) const noexcept;
constexpr size_type find_last_not_of(const charT* s, size_type pos, size_type n) const;
constexpr size_type find_last_not_of(const charT* s, size_type pos = npos) const;
constexpr size_type find_last_not_of(charT c, size_type pos = npos) const noexcept;

constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &;
constexpr basic_string substr(size_type pos = 0, size_type n = npos) &&;
constexpr basic_string_view<charT, traits> subview(size_type pos = 0,
 size_type n = npos) const;

template<class T>
 constexpr int compare(const T& t) const noexcept(see below);
template<class T>
 constexpr int compare(size_type pos1, size_type n1, const T& t) const;
template<class T>
 constexpr int compare(size_type pos1, size_type n1, const T& t,
 size_type pos2, size_type n2 = npos) const;
constexpr int compare(const basic_string& str) const noexcept;
constexpr int compare(size_type pos1, size_type n1, const basic_string& str) const;

```



```

constexpr int compare(size_type pos1, size_type n1, const basic_string& str,
 size_type pos2, size_type n2 = npos) const;
constexpr int compare(const charT* s) const;
constexpr int compare(size_type pos1, size_type n1, const charT* s) const;
constexpr int compare(size_type pos1, size_type n1, const charT* s, size_type n2) const;

constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
constexpr bool starts_with(charT x) const noexcept;
constexpr bool starts_with(const charT* x) const;
constexpr bool ends_with(basic_string_view<charT, traits> x) const noexcept;
constexpr bool ends_with(charT x) const noexcept;
constexpr bool ends_with(const charT* x) const;

constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;
constexpr bool contains(charT x) const noexcept;
constexpr bool contains(const charT* x) const;
};

template<class InputIterator,
 class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
basic_string(InputIterator, InputIterator, Allocator = Allocator())
 -> basic_string<typename iterator_traits<InputIterator>::value_type,
 char_traits<typename iterator_traits<InputIterator>::value_type>,
 Allocator>;

template<ranges::input_range R,
 class Allocator = allocator<ranges::range_value_t<R>>>
basic_string(from_range_t, R&&, Allocator = Allocator())
 -> basic_string<ranges::range_value_t<R>, char_traits<ranges::range_value_t<R>>,
 Allocator>;

template<class charT,
 class traits,
 class Allocator = allocator<charT>>
explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator())
 -> basic_string<charT, traits, Allocator>;

template<class charT,
 class traits,
 class Allocator = allocator<charT>>
basic_string(basic_string_view<charT, traits>,
 typename see_below::size_type, typename see_below::size_type,
 const Allocator& = Allocator())
 -> basic_string<charT, traits, Allocator>;
}

```

<sup>4</sup> A `size_type` parameter type in a `basic_string` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

<sup>5</sup> The types `iterator` and `const_iterator` meet the `constexpr` iterator requirements (24.3.1).

### 27.4.3.2 General requirements

[string.require]

- <sup>1</sup> If any operation would cause `size()` to exceed `max_size()`, that operation throws an exception object of type `length_error`.
- <sup>2</sup> If any member function or operator of `basic_string` throws an exception, that function or operator has no other effect on the `basic_string` object.
- <sup>3</sup> Every object of type `basic_string<charT, traits, Allocator>` uses an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used is obtained as described in 23.2.2.2. In every specialization `basic_string<charT, traits, Allocator>`, the type `traits` shall meet the character traits requirements (27.2).

[Note 1: Every specialization `basic_string<charT, traits, Allocator>` is an allocator-aware container (23.2.2.5), but does not use the allocator's `construct` and `destroy` member functions (23.2.1). The program is ill-formed if `Allocator::value_type` is not the same type as `charT`. — end note]



[Note 2: The program is ill-formed if `traits::char_type` is not the same type as `charT`. — end note]

4 References, pointers, and iterators referring to the elements of a `basic_string` sequence may be invalidated by the following uses of that `basic_string` object:

- (4.1) — Passing as an argument to any standard library function taking a reference to non-const `basic_string` as an argument.<sup>191</sup>
- (4.2) — Calling non-const member functions, except `operator[]`, `at`, `data`, `front`, `back`, `begin`, `rbegin`, `end`, and `rend`.

### 27.4.3.3 Constructors and assignment operators

[string.cons]

```
constexpr explicit basic_string(const Allocator& a) noexcept;
```

1 *Postconditions:* `size()` is equal to 0.

```
constexpr basic_string(const basic_string& str);
constexpr basic_string(basic_string&& str) noexcept;
```

2 *Effects:* Constructs an object whose value is that of `str` prior to this call.

3 *Remarks:* In the second form, `str` is left in a valid but unspecified state.

```
constexpr basic_string(const basic_string& str, size_type pos,
 const Allocator& a = Allocator());
constexpr basic_string(const basic_string& str, size_type pos, size_type n,
 const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos,
 const Allocator& a = Allocator());
constexpr basic_string(basic_string&& str, size_type pos, size_type n,
 const Allocator& a = Allocator());
```

4 Let

(4.1) — `s` be the value of `str` prior to this call and

(4.2) — `rlen` be `pos + min(n, s.size() - pos)` for the overloads with parameter `n`, and `s.size()` otherwise.

5 *Effects:* Constructs an object whose initial value is the range `[s.data() + pos, s.data() + rlen)`.

6 *Throws:* `out_of_range` if `pos > s.size()`.

7 *Remarks:* For the overloads with a `basic_string&&` parameter, `str` is left in a valid but unspecified state.

8 *Recommended practice:* For the overloads with a `basic_string&&` parameter, implementations should avoid allocation if `s.get_allocator() == a` is true.

```
template<class T>
constexpr basic_string(const T& t, size_type pos, size_type n, const Allocator& a = Allocator());
```

9 *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

10 *Effects:* Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then behaves the same as:

```
basic_string(sv.substr(pos, n), a);
```

```
template<class T>
constexpr explicit basic_string(const T& t, const Allocator& a = Allocator());
```

11 *Constraints:*

(11.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(11.2) — `is_convertible_v<const T&, const charT*>` is false.

12 *Effects:* Creates a variable, `sv`, as if by `basic_string_view<charT, traits> sv = t;` and then behaves the same as `basic_string(sv.data(), sv.size(), a)`.

<sup>191</sup> For example, as an argument to non-member functions `swap()` (27.4.4.3), `operator>>()` (27.4.4.4), and `getline()` (27.4.4.4), or as an argument to `basic_string::swap()`.

```
constexpr basic_string(const charT* s, size_type n, const Allocator& a = Allocator());
```

13     *Preconditions:* `[s, s + n)` is a valid range.

14     *Effects:* Constructs an object whose initial value is the range `[s, s + n)`.

15     *Postconditions:* `size()` is equal to `n`, and `traits::compare(data(), s, n)` is equal to 0.

```
constexpr basic_string(const charT* s, const Allocator& a = Allocator());
```

16     *Constraints:* `Allocator` is a type that qualifies as an allocator (23.2.2.2).  
     [Note 1: This affects class template argument deduction. — end note]

17     *Effects:* Equivalent to: `basic_string(s, traits::length(s), a)`.

```
constexpr basic_string(size_type n, charT c, const Allocator& a = Allocator());
```

18     *Constraints:* `Allocator` is a type that qualifies as an allocator (23.2.2.2).  
     [Note 2: This affects class template argument deduction. — end note]

19     *Effects:* Constructs an object whose value consists of `n` copies of `c`.

```
template<class InputIterator>
constexpr basic_string(InputIterator begin, InputIterator end, const Allocator& a = Allocator());
```

20     *Constraints:* `InputIterator` is a type that qualifies as an input iterator (23.2.2.2).

21     *Effects:* Constructs a string from the values in the range `[begin, end)`, as specified in 23.2.4.

```
template<container-compatible-range<charT> R>
constexpr basic_string(from_range_t, R&& rg, const Allocator& = Allocator());
```

22     *Effects:* Constructs a string from the values in the range `rg`, as specified in 23.2.4.

```
constexpr basic_string(initializer_list<charT> il, const Allocator& a = Allocator());
```

23     *Effects:* Equivalent to `basic_string(il.begin(), il.end(), a)`.

```
constexpr basic_string(const basic_string& str, const Allocator& alloc);
constexpr basic_string(basic_string&& str, const Allocator& alloc);
```

24     *Effects:* Constructs an object whose value is that of `str` prior to this call. The stored allocator is constructed from `alloc`. In the second form, `str` is left in a valid but unspecified state.

25     *Throws:* The second form throws nothing if `alloc == str.get_allocator()`.

```
template<class InputIterator,
 class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
basic_string(InputIterator, InputIterator, Allocator = Allocator())
-> basic_string<typename iterator_traits<InputIterator>::value_type,
 char_traits<typename iterator_traits<InputIterator>::value_type>,
 Allocator>;
```

26     *Constraints:* `InputIterator` is a type that qualifies as an input iterator, and `Allocator` is a type that qualifies as an allocator (23.2.2.2).

```
template<class charT,
 class traits,
 class Allocator = allocator<charT>>
explicit basic_string(basic_string_view<charT, traits>, const Allocator& = Allocator())
-> basic_string<charT, traits, Allocator>;
```

```
template<class charT,
 class traits,
 class Allocator = allocator<charT>>
basic_string(basic_string_view<charT, traits>,
 typename see below::size_type, typename see below::size_type,
 const Allocator& = Allocator())
-> basic_string<charT, traits, Allocator>;
```

27     *Constraints:* `Allocator` is a type that qualifies as an allocator (23.2.2.2).

```
constexpr basic_string& operator=(const basic_string& str);
```

28 *Effects:* If *\*this* and *str* are the same object, has no effect. Otherwise, replaces the value of *\*this* with a copy of *str*.

29 *Returns:* *\*this*.

```
constexpr basic_string& operator=(basic_string&& str)
 noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
 allocator_traits<Allocator>::is_always_equal::value);
```

30 *Effects:* Move assigns as a sequence container (23.2.4), except that iterators, pointers and references may be invalidated.

31 *Returns:* *\*this*.

```
template<class T>
constexpr basic_string& operator=(const T& t);
```

32 *Constraints:*

(32.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(32.2) — `is_convertible_v<const T&, const charT*>` is false.

33 *Effects:* Equivalent to:

```
 basic_string_view<charT, traits> sv = t;
 return assign(sv);
```

```
constexpr basic_string& operator=(const charT* s);
```

34 *Effects:* Equivalent to: `return *this = basic_string_view<charT, traits>(s);`

```
constexpr basic_string& operator=(charT c);
```

35 *Effects:* Equivalent to:

```
 return *this = basic_string_view<charT, traits>(addressof(c), 1);
```

```
constexpr basic_string& operator=(initializer_list<charT> il);
```

36 *Effects:* Equivalent to:

```
 return *this = basic_string_view<charT, traits>(il.data(), il.size());
```

#### 27.4.3.4 Iterator support

[string.iterators]

```
constexpr iterator begin() noexcept;
constexpr const_iterator begin() const noexcept;
constexpr const_iterator cbegin() const noexcept;
```

1 *Returns:* An iterator referring to the first character in the string.

```
constexpr iterator end() noexcept;
constexpr const_iterator end() const noexcept;
constexpr const_iterator cend() const noexcept;
```

2 *Returns:* An iterator which is the past-the-end value.

```
constexpr reverse_iterator rbegin() noexcept;
constexpr const_reverse_iterator rbegin() const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
```

3 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(end())`.

```
constexpr reverse_iterator rend() noexcept;
constexpr const_reverse_iterator rend() const noexcept;
constexpr const_reverse_iterator crend() const noexcept;
```

4 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(begin())`.

#### 27.4.3.5 Capacity

[string.capacity]

```
constexpr size_type size() const noexcept;
```

```
constexpr size_type length() const noexcept;
```

1     *Returns:* A count of the number of char-like objects currently in the string.

2     *Complexity:* Constant time.

```
constexpr size_type max_size() const noexcept;
```

3     *Returns:* The largest possible number of char-like objects that can be stored in a `basic_string`.

4     *Complexity:* Constant time.

```
constexpr void resize(size_type n, charT c);
```

5     *Effects:* Alters the value of `*this` as follows:

(5.1)     — If `n <= size()`, erases the last `size() - n` elements.

(5.2)     — If `n > size()`, appends `n - size()` copies of `c`.

```
constexpr void resize(size_type n);
```

6     *Effects:* Equivalent to `resize(n, charT())`.

```
template<class Operation> constexpr void resize_and_overwrite(size_type n, Operation op);
```

7     Let

(7.1)     — `o = size()` before the call to `resize_and_overwrite`.

(7.2)     — `k` be `min(o, n)`.

(7.3)     — `p` be a value of type `charT*` or `charT* const`, such that the range `[p, p + n]` is valid and `this->compare(0, k, p, k) == 0` is true before the call. The values in the range `[p + k, p + n]` may be indeterminate (6.8.5).

(7.4)     — `m` be a value of type `size_type` or `const size_type` equal to `n`.

(7.5)     — `OP` be the expression `std::move(op)(p, m)`.

(7.6)     — `r = OP`.

8     *Mandates:* `OP` has an integer-like type (24.3.4.4).

9     *Preconditions:*

(9.1)     — `OP` does not throw an exception or modify `p` or `m`.

(9.2)     — `r ≥ 0`.

(9.3)     — `r ≤ m`.

(9.4)     — After evaluating `OP` there are no indeterminate values in the range `[p, p + r]`.

10     *Effects:* Evaluates `OP`, replaces the contents of `*this` with `[p, p + r]`, and invalidates all pointers and references to the range `[p, p + n]`.

11     *Recommended practice:* Implementations should avoid unnecessary copies and allocations by, for example, making `p` a pointer into internal storage and by restoring `*(p + r)` to `charT()` after evaluating `OP`.

```
constexpr size_type capacity() const noexcept;
```

12     *Returns:* The size of the allocated storage in the string.

13     *Complexity:* Constant time.

```
constexpr void reserve(size_type res_arg);
```

14     *Effects:* A directive that informs a `basic_string` of a planned change in size, so that the storage allocation can be managed accordingly. Following a call to `reserve`, `capacity()` is greater or equal to the argument of `reserve` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of `reserve`.

15     *Throws:* `length_error` if `res_arg > max_size()` or any exceptions thrown by `allocator_traits<Allocator>::allocate`.

```
constexpr void shrink_to_fit();
```

- 16 *Effects:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`.  
 [Note 1: The request is non-binding to allow latitude for implementation-specific optimizations. — end note]  
 It does not increase `capacity()`, but may reduce `capacity()` by causing reallocation.
- 17 *Complexity:* If the size is not equal to the old capacity, linear in the size of the sequence; otherwise constant.
- 18 *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator.  
 [Note 2: If no reallocation happens, they remain valid. — end note]

```
constexpr void clear() noexcept;
```

- 19 *Effects:* Equivalent to: `erase(begin(), end())`;

```
constexpr bool empty() const noexcept;
```

- 20 *Effects:* Equivalent to: `return size() == 0;`

#### 27.4.3.6 Element access

[string.access]

```
constexpr const_reference operator[](size_type pos) const;
constexpr reference operator[](size_type pos);
```

- 1 *Hardened preconditions:* `pos <= size()` is true.  
 2 *Returns:* `data()[pos]`.  
 3 *Throws:* Nothing.  
 4 *Complexity:* Constant time.

```
constexpr const_reference at(size_type pos) const;
constexpr reference at(size_type pos);
```

- 5 *Returns:* `operator[](pos)`.  
 6 *Throws:* `out_of_range` if `pos >= size()`.

```
constexpr const_reference front() const;
constexpr reference front();
```

- 7 *Hardened preconditions:* `empty()` is false.  
 8 *Effects:* Equivalent to: `return operator[](0);`

```
constexpr const_reference back() const;
constexpr reference back();
```

- 9 *Hardened preconditions:* `empty()` is false.  
 10 *Effects:* Equivalent to: `return operator[](size() - 1);`

#### 27.4.3.7 Modifiers

[string.modifiers]

##### 27.4.3.7.1 `basic_string::operator+=`

[string.op.append]

```
constexpr basic_string& operator+=(const basic_string& str);
```

- 1 *Effects:* Equivalent to: `return append(str);`

```
template<class T>
constexpr basic_string& operator+=(const T& t);
```

- 2 *Constraints:*
- (2.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and
  - (2.2) — `is_convertible_v<const T&, const charT*>` is false.
- 3 *Effects:* Equivalent to:
- ```
basic_string_view<charT, traits> sv = t;
return append(sv);
```

```
constexpr basic_string& operator+=(const charT* s);
```

4 *Effects:* Equivalent to: return append(s);

```
constexpr basic_string& operator+=(charT c);
```

5 *Effects:* Equivalent to: return append(size_type{1}, c);

```
constexpr basic_string& operator+=(initializer_list<charT> il);
```

6 *Effects:* Equivalent to: return append(il);

27.4.3.7.2 basic_string::append [string.append]

```
constexpr basic_string& append(const basic_string& str);
```

1 *Effects:* Equivalent to: return append(str.data(), str.size());

```
constexpr basic_string& append(const basic_string& str, size_type pos, size_type n = npos);
```

2 *Effects:* Equivalent to:

```
return append(basic_string_view<charT, traits>(str).substr(pos, n));
```

```
template<class T>
```

```
constexpr basic_string& append(const T& t);
```

3 *Constraints:*

(3.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(3.2) — is_convertible_v<const T&, const charT*> is false.

4 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return append(sv.data(), sv.size());
```

```
template<class T>
```

```
constexpr basic_string& append(const T& t, size_type pos, size_type n = npos);
```

5 *Constraints:*

(5.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(5.2) — is_convertible_v<const T&, const charT*> is false.

6 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return append(sv.substr(pos, n));
```

```
constexpr basic_string& append(const charT* s, size_type n);
```

7 *Preconditions:* [s, s + n) is a valid range.

8 *Effects:* Appends a copy of the range [s, s + n) to the string.

9 *Returns:* *this.

```
constexpr basic_string& append(const charT* s);
```

10 *Effects:* Equivalent to: return append(s, traits::length(s));

```
constexpr basic_string& append(const charT* s, size_type pos, size_type n);
```

11 *Effects:* Equivalent to: return append(basic_string_view<charT, traits>(s).substr(pos, n));

```
constexpr basic_string& append(size_type n, charT c);
```

12 *Effects:* Appends n copies of c to the string.

13 *Returns:* *this.

```
template<class InputIterator>
```

```
constexpr basic_string& append(InputIterator first, InputIterator last);
```

14 *Constraints:* InputIterator is a type that qualifies as an input iterator (23.2.2.2).

15 *Effects:* Equivalent to: return append(basic_string(first, last, get_allocator()));

```
template<container-compatible-range<charT> R>
constexpr basic_string& append_range(R&& rg);
```

16 *Effects:* Equivalent to: return append(basic_string(from_range, std::forward<R>(rg), get_allocator()));

```
constexpr basic_string& append(initializer_list<charT> il);
```

17 *Effects:* Equivalent to: return append(il.data(), il.size());

```
constexpr void push_back(charT c);
```

18 *Effects:* Equivalent to append(size_type{1}, c).

27.4.3.7.3 basic_string::assign

[string.assign]

```
constexpr basic_string& assign(const basic_string& str);
```

1 *Effects:* Equivalent to: return *this = str;

```
constexpr basic_string& assign(basic_string&& str)
noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
allocator_traits<Allocator>::is_always_equal::value);
```

2 *Effects:* Equivalent to: return *this = std::move(str);

```
constexpr basic_string& assign(const basic_string& str, size_type pos, size_type n = npos);
```

3 *Effects:* Equivalent to:

```
return assign(basic_string_view<charT, traits>(str).substr(pos, n));
```

```
template<class T>
```

```
constexpr basic_string& assign(const T& t);
```

4 *Constraints:*

- (4.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and
- (4.2) — is_convertible_v<const T&, const charT*> is false.

5 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return assign(sv.data(), sv.size());
```

```
template<class T>
```

```
constexpr basic_string& assign(const T& t, size_type pos, size_type n = npos);
```

6 *Constraints:*

- (6.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and
- (6.2) — is_convertible_v<const T&, const charT*> is false.

7 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return assign(sv.substr(pos, n));
```

```
constexpr basic_string& assign(const charT* s, size_type n);
```

8 *Preconditions:* [s, s + n) is a valid range.

9 *Effects:* Replaces the string controlled by *this with a copy of the range [s, s + n).

10 *Returns:* *this.

```
constexpr basic_string& assign(const charT* s);
```

11 *Effects:* Equivalent to: return assign(s, traits::length(s));

```
constexpr basic_string& assign(const charT* s, size_type pos, size_type n);
```

12 *Effects:* Equivalent to: return assign(basic_string_view<charT, traits>(s).substr(pos, n));

```
constexpr basic_string& assign(initializer_list<charT> il);
```

13 *Effects:* Equivalent to: return assign(il.data(), il.size());

```
constexpr basic_string& assign(size_type n, charT c);
```

14 *Effects:* Equivalent to:

```
clear();
resize(n, c);
return *this;
```

```
template<class InputIterator>
```

```
constexpr basic_string& assign(InputIterator first, InputIterator last);
```

15 *Constraints:* InputIterator is a type that qualifies as an input iterator (23.2.2.2).

16 *Effects:* Equivalent to: return assign(basic_string(first, last, get_allocator()));

```
template<container-compatible-range<charT> R>
```

```
constexpr basic_string& assign_range(R&& rg);
```

17 *Effects:* Equivalent to: return assign(basic_string(from_range, std::forward<R>(rg), get_allocator()));

27.4.3.7.4 basic_string::insert

[string.insert]

```
constexpr basic_string& insert(size_type pos, const basic_string& str);
```

1 *Effects:* Equivalent to: return insert(pos, str.data(), str.size());

```
constexpr basic_string& insert(size_type pos1, const basic_string& str,
                             size_type pos2, size_type n = npos);
```

2 *Effects:* Equivalent to:

```
return insert(pos1, basic_string_view<charT, traits>(str), pos2, n);
```

```
template<class T>
```

```
constexpr basic_string& insert(size_type pos, const T& t);
```

3 *Constraints:*

(3.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(3.2) — is_convertible_v<const T&, const charT*> is false.

4 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return insert(pos, sv.data(), sv.size());
```

```
template<class T>
```

```
constexpr basic_string& insert(size_type pos1, const T& t,
                             size_type pos2, size_type n = npos);
```

5 *Constraints:*

(5.1) — is_convertible_v<const T&, basic_string_view<charT, traits>> is true and

(5.2) — is_convertible_v<const T&, const charT*> is false.

6 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return insert(pos1, sv.substr(pos2, n));
```

```
constexpr basic_string& insert(size_type pos, const charT* s, size_type n);
```

7 *Preconditions:* [s, s + n) is a valid range.

8 *Effects:* Inserts a copy of the range [s, s + n) immediately before the character at position pos if pos < size(), or otherwise at the end of the string.

9 *Returns:* *this.

10 *Throws:*

(10.1) — out_of_range if pos > size(),

(10.2) — length_error if n > max_size() - size(), or

(10.3) — any exceptions thrown by allocator_traits<Allocator>::allocate.


```
constexpr basic_string& insert(size_type pos, const charT* s);
```

11 *Effects:* Equivalent to: `return insert(pos, s, traits::length(s));`

```
constexpr basic_string& insert(size_type pos, size_type n, charT c);
```

12 *Effects:* Inserts `n` copies of `c` before the character at position `pos` if `pos < size()`, or otherwise at the end of the string.

13 *Returns:* `*this`.

14 *Throws:*

(14.1) — `out_of_range` if `pos > size()`,

(14.2) — `length_error` if `n > max_size() - size()`, or

(14.3) — any exceptions thrown by `allocator_traits<Allocator>::allocate`.

```
constexpr iterator insert(const_iterator p, charT c);
```

15 *Preconditions:* `p` is a valid iterator on `*this`.

16 *Effects:* Inserts a copy of `c` at the position `p`.

17 *Returns:* An iterator which refers to the inserted character.

```
constexpr iterator insert(const_iterator p, size_type n, charT c);
```

18 *Preconditions:* `p` is a valid iterator on `*this`.

19 *Effects:* Inserts `n` copies of `c` at the position `p`.

20 *Returns:* An iterator which refers to the first inserted character, or `p` if `n == 0`.

```
template<class InputIterator>
```

```
constexpr iterator insert(const_iterator p, InputIterator first, InputIterator last);
```

21 *Constraints:* `InputIterator` is a type that qualifies as an input iterator (23.2.2.2).

22 *Preconditions:* `p` is a valid iterator on `*this`.

23 *Effects:* Equivalent to `insert(p - begin(), basic_string(first, last, get_allocator()))`.

24 *Returns:* An iterator which refers to the first inserted character, or `p` if `first == last`.

```
template<container-compatible-range<charT> R>
```

```
constexpr iterator insert_range(const_iterator p, R&& rg);
```

25 *Preconditions:* `p` is a valid iterator on `*this`.

26 *Effects:* Equivalent to `insert(p - begin(), basic_string(from_range, std::forward<R>(rg), get_allocator()))`.

27 *Returns:* An iterator which refers to the first inserted character, or `p` if `rg` is empty.

```
constexpr iterator insert(const_iterator p, initializer_list<charT> il);
```

28 *Effects:* Equivalent to: `return insert(p, il.begin(), il.end());`

27.4.3.7.5 basic_string::erase

[string.erase]

```
constexpr basic_string& erase(size_type pos = 0, size_type n = npos);
```

1 *Effects:* Determines the effective length `xlen` of the string to be removed as the smaller of `n` and `size() - pos`. Removes the characters in the range `[begin() + pos, begin() + pos + xlen)`.

2 *Returns:* `*this`.

3 *Throws:* `out_of_range` if `pos > size()`.

```
constexpr iterator erase(const_iterator p);
```

4 *Preconditions:* `p` is a valid dereferenceable iterator on `*this`.

5 *Effects:* Removes the character referred to by `p`.

6 *Returns:* An iterator which points to the element immediately following `p` prior to the element being erased. If no such element exists, `end()` is returned.

7 *Throws:* Nothing.

```
constexpr iterator erase(const_iterator first, const_iterator last);
```

8 *Preconditions:* `first` and `last` are valid iterators on `*this`. `[first, last)` is a valid range.

9 *Effects:* Removes the characters in the range `[first, last)`.

10 *Returns:* An iterator which points to the element pointed to by `last` prior to the other elements being erased. If no such element exists, `end()` is returned.

11 *Throws:* Nothing.

```
constexpr void pop_back();
```

12 *Hardened preconditions:* `empty()` is false.

13 *Effects:* Equivalent to `erase(end() - 1)`.

14 *Throws:* Nothing.

27.4.3.7.6 `basic_string::replace` [string.replace]

```
constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str);
```

1 *Effects:* Equivalent to: `return replace(pos1, n1, str.data(), str.size());`

```
constexpr basic_string& replace(size_type pos1, size_type n1, const basic_string& str,
                               size_type pos2, size_type n2 = npos);
```

2 *Effects:* Equivalent to:

```
return replace(pos1, n1, basic_string_view<charT, traits>(str).substr(pos2, n2));
```

```
template<class T>
```

```
constexpr basic_string& replace(size_type pos1, size_type n1, const T& t);
```

3 *Constraints:*

(3.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(3.2) — `is_convertible_v<const T&, const charT*>` is false.

4 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return replace(pos1, n1, sv.data(), sv.size());
```

```
template<class T>
```

```
constexpr basic_string& replace(size_type pos1, size_type n1, const T& t,
                               size_type pos2, size_type n2 = npos);
```

5 *Constraints:*

(5.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(5.2) — `is_convertible_v<const T&, const charT*>` is false.

6 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
return replace(pos1, n1, sv.substr(pos2, n2));
```

```
constexpr basic_string& replace(size_type pos1, size_type n1, const charT* s, size_type n2);
```

7 *Preconditions:* `[s, s + n2)` is a valid range.

8 *Effects:* Determines the effective length `xlen` of the string to be removed as the smaller of `n1` and `size() - pos1`. If `size() - xlen >= max_size() - n2` throws `length_error`. Otherwise, the function replaces the characters in the range `[begin() + pos1, begin() + pos1 + xlen)` with a copy of the range `[s, s + n2)`.

9 *Returns:* `*this`.

10 *Throws:*

(10.1) — `out_of_range` if `pos1 > size()`,

(10.2) — `length_error` if the length of the resulting string would exceed `max_size()`, or

(10.3) — any exceptions thrown by `allocator_traits<Allocator>::allocate`.

```
constexpr basic_string& replace(size_type pos, size_type n, const charT* s);
```

11 *Effects:* Equivalent to: `return replace(pos, n, s, traits::length(s));`

```
constexpr basic_string& replace(size_type pos1, size_type n1, size_type n2, charT c);
```

12 *Effects:* Determines the effective length `xlen` of the string to be removed as the smaller of `n1` and `size() - pos1`. If `size() - xlen >= max_size() - n2` throws `length_error`. Otherwise, the function replaces the characters in the range `[begin() + pos1, begin() + pos1 + xlen)` with `n2` copies of `c`.

13 *Returns:* `*this`.

14 *Throws:*

(14.1) — `out_of_range` if `pos1 > size()`,

(14.2) — `length_error` if the length of the resulting string would exceed `max_size()`, or

(14.3) — any exceptions thrown by `allocator_traits<Allocator>::allocate`.

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const basic_string& str);
```

15 *Effects:* Equivalent to: `return replace(i1, i2, basic_string_view<charT, traits>(str));`

```
template<class T>
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const T& t);
```

16 *Constraints:*

(16.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(16.2) — `is_convertible_v<const T&, const charT*>` is false.

17 *Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges.

18 *Effects:* Equivalent to:

```
    basic_string_view<charT, traits> sv = t;
    return replace(i1 - begin(), i2 - i1, sv.data(), sv.size());
```

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s, size_type n);
```

19 *Effects:* Equivalent to: `return replace(i1, i2, basic_string_view<charT, traits>(s, n));`

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, const charT* s);
```

20 *Effects:* Equivalent to: `return replace(i1, i2, basic_string_view<charT, traits>(s));`

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, size_type n, charT c);
```

21 *Preconditions:* `[begin(), i1)` and `[i1, i2)` are valid ranges.

22 *Effects:* Equivalent to: `return replace(i1 - begin(), i2 - i1, n, c);`

```
template<class InputIterator>
constexpr basic_string& replace(const_iterator i1, const_iterator i2,
                               InputIterator j1, InputIterator j2);
```

23 *Constraints:* `InputIterator` is a type that qualifies as an input iterator (23.2.2.2).

24 *Effects:* Equivalent to: `return replace(i1, i2, basic_string(j1, j2, get_allocator()));`

```
template<container-compatible-range<charT> R>
constexpr basic_string& replace_with_range(const_iterator i1, const_iterator i2, R&& rg);
```

25 *Effects:* Equivalent to:

```
    return replace(i1, i2, basic_string(from_range, std::forward<R>(rg), get_allocator()));
```

```
constexpr basic_string& replace(const_iterator i1, const_iterator i2, initializer_list<charT> il);
```

26 *Effects:* Equivalent to: `return replace(i1, i2, il.data(), il.size());`

27.4.3.7.7 basic_string::copy [string.copy]

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

- 1 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(*this).copy(s, n, pos);`
 [Note 1: This does not terminate `s` with a null object. — end note]

27.4.3.7.8 basic_string::swap [string.swap]

```
constexpr void swap(basic_string& s)
    noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
              allocator_traits<Allocator>::is_always_equal::value);
```

- 1 *Preconditions:* `allocator_traits<Allocator>::propagate_on_container_swap::value` is true or `get_allocator() == s.get_allocator()`.
 2 *Postconditions:* `*this` contains the same sequence of characters that was in `s`, `s` contains the same sequence of characters that was in `*this`.
 3 *Throws:* Nothing.
 4 *Complexity:* Constant time.

27.4.3.8 String operations [string.ops]**27.4.3.8.1 Accessors** [string.accessors]

```
constexpr const charT* c_str() const noexcept;
constexpr const charT* data() const noexcept;
constexpr charT* data() noexcept;
```

- 1 *Returns:* `to_address(begin())`.
 2 *Complexity:* Constant time.

```
constexpr operator basic_string_view<charT, traits>() const noexcept;
```

- 3 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(data(), size());`

```
constexpr allocator_type get_allocator() const noexcept;
```

- 4 *Returns:* A copy of the Allocator object used to construct the string or, if that allocator has been replaced, a copy of the most recent replacement.

27.4.3.8.2 Searching [string.find]

- 1 Let F be one of `find`, `rfind`, `find_first_of`, `find_last_of`, `find_first_not_of`, and `find_last_not_of`.
- (1.1) — Each member function of the form
- ```
constexpr size_type F(const basic_string& str, size_type pos) const noexcept;
```
- has effects equivalent to: `return F(basic_string_view<charT, traits>(str), pos);`
- (1.2) — Each member function of the form
- ```
constexpr size_type F(const charT* s, size_type pos) const;
```
- has effects equivalent to: `return F(basic_string_view<charT, traits>(s), pos);`
- (1.3) — Each member function of the form
- ```
constexpr size_type F(const charT* s, size_type pos, size_type n) const;
```
- has effects equivalent to: `return F(basic_string_view<charT, traits>(s, n), pos);`
- (1.4) — Each member function of the form
- ```
constexpr size_type F(charT c, size_type pos) const noexcept;
```
- has effects equivalent to:
- ```
return F(basic_string_view<charT, traits>(addressof(c), 1), pos);
```

```
template<class T>
constexpr size_type find(const T& t, size_type pos = 0) const noexcept(see below);
template<class T>
constexpr size_type rfind(const T& t, size_type pos = npos) const noexcept(see below);
```

```

template<class T>
constexpr size_type find_first_of(const T& t, size_type pos = 0) const noexcept(see below);
template<class T>
constexpr size_type find_last_of(const T& t, size_type pos = npos) const noexcept(see below);
template<class T>
constexpr size_type find_first_not_of(const T& t, size_type pos = 0) const noexcept(see below);
template<class T>
constexpr size_type find_last_not_of(const T& t, size_type pos = npos) const noexcept(see below);

```

2 *Constraints:*

- (2.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and
- (2.2) — `is_convertible_v<const T&, const charT*>` is false.

3 *Effects:* Let  $G$  be the name of the function. Equivalent to:

```

basic_string_view<charT, traits> s = *this, sv = t;
if (auto result = s.G(sv, pos); result == s.npos) {
 return npos;
} else {
 return result;
}

```

4 *Remarks:* The exception specification is equivalent to `is_nothrow_convertible_v<const T&, basic_string_view<charT, traits>>`.

#### 27.4.3.8.3 `basic_string::substr`

[string.substr]

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) const &;
```

1 *Effects:* Equivalent to: `return basic_string(*this, pos, n);`

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos) &&;
```

2 *Effects:* Equivalent to: `return basic_string(std::move(*this), pos, n);`

```
constexpr basic_string_view<charT, traits> subview(size_type pos = 0, size_type n = npos) const;
```

3 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(*this).subview(pos, n);`

#### 27.4.3.8.4 `basic_string::compare`

[string.compare]

```
template<class T>
constexpr int compare(const T& t) const noexcept(see below);
```

1 *Constraints:*

- (1.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and
- (1.2) — `is_convertible_v<const T&, const charT*>` is false.

2 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(*this).compare(t);`

3 *Remarks:* The exception specification is equivalent to `is_nothrow_convertible_v<const T&, basic_string_view<charT, traits>>`.

```
template<class T>
constexpr int compare(size_type pos1, size_type n1, const T& t) const;
```

4 *Constraints:*

- (4.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and
- (4.2) — `is_convertible_v<const T&, const charT*>` is false.

5 *Effects:* Equivalent to:

```
return basic_string_view<charT, traits>(*this).substr(pos1, n1).compare(t);
```

```
template<class T>
constexpr int compare(size_type pos1, size_type n1, const T& t,
 size_type pos2, size_type n2 = npos) const;
```

6 *Constraints:*

- (6.1) — `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true and

(6.2) — `is_convertible_v<const T&, const charT*>` is false.

7 *Effects:* Equivalent to:

```
 basic_string_view<charT, traits> s = *this, sv = t;
 return s.substr(pos1, n1).compare(sv.substr(pos2, n2));
```

```
constexpr int compare(const basic_string& str) const noexcept;
```

8 *Effects:* Equivalent to: `return compare(basic_string_view<charT, traits>(str));`

```
constexpr int compare(size_type pos1, size_type n1, const basic_string& str) const;
```

9 *Effects:* Equivalent to: `return compare(pos1, n1, basic_string_view<charT, traits>(str));`

```
constexpr int compare(size_type pos1, size_type n1, const basic_string& str,
 size_type pos2, size_type n2 = npos) const;
```

10 *Effects:* Equivalent to:

```
 return compare(pos1, n1, basic_string_view<charT, traits>(str), pos2, n2);
```

```
constexpr int compare(const charT* s) const;
```

11 *Effects:* Equivalent to: `return compare(basic_string_view<charT, traits>(s));`

```
constexpr int compare(size_type pos, size_type n1, const charT* s) const;
```

12 *Effects:* Equivalent to: `return compare(pos, n1, basic_string_view<charT, traits>(s));`

```
constexpr int compare(size_type pos, size_type n1, const charT* s, size_type n2) const;
```

13 *Effects:* Equivalent to: `return compare(pos, n1, basic_string_view<charT, traits>(s, n2));`

#### 27.4.3.8.5 `basic_string::starts_with` [string.starts.with]

```
constexpr bool starts_with(basic_string_view<charT, traits> x) const noexcept;
```

```
constexpr bool starts_with(charT x) const noexcept;
```

```
constexpr bool starts_with(const charT* x) const;
```

1 *Effects:* Equivalent to:

```
 return basic_string_view<charT, traits>(data(), size()).starts_with(x);
```

#### 27.4.3.8.6 `basic_string::ends_with` [string.ends.with]

```
constexpr bool ends_with(basic_string_view<charT, traits> x) const noexcept;
```

```
constexpr bool ends_with(charT x) const noexcept;
```

```
constexpr bool ends_with(const charT* x) const;
```

1 *Effects:* Equivalent to:

```
 return basic_string_view<charT, traits>(data(), size()).ends_with(x);
```

#### 27.4.3.8.7 `basic_string::contains` [string.contains]

```
constexpr bool contains(basic_string_view<charT, traits> x) const noexcept;
```

```
constexpr bool contains(charT x) const noexcept;
```

```
constexpr bool contains(const charT* x) const;
```

1 *Effects:* Equivalent to:

```
 return basic_string_view<charT, traits>(data(), size()).contains(x);
```

### 27.4.4 Non-member functions [string.nonmembers]

#### 27.4.4.1 `operator+` [string.op.plus]

```
template<class charT, class traits, class Allocator>
```

```
 constexpr basic_string<charT, traits, Allocator>
```

```
 operator+(const basic_string<charT, traits, Allocator>& lhs,
```

```
 const basic_string<charT, traits, Allocator>& rhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs, const charT* rhs);
```

1 *Effects:* Equivalent to:

```
 basic_string<charT, traits, Allocator> r = lhs;
 r.append(rhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 const basic_string<charT, traits, Allocator>& rhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs, const charT* rhs);
```

2 *Effects:* Equivalent to:

```
 lhs.append(rhs);
 return std::move(lhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 basic_string<charT, traits, Allocator>&& rhs);
```

3 *Effects:* Equivalent to:

```
 lhs.append(rhs);
 return std::move(lhs);
```

except that both lhs and rhs are left in valid but unspecified states.

[*Note 1:* If lhs and rhs have equal allocators, the implementation can move from either. — *end note*]

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
 basic_string<charT, traits, Allocator>&& rhs);
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const charT* lhs, basic_string<charT, traits, Allocator>&& rhs);
```

4 *Effects:* Equivalent to:

```
 rhs.insert(0, lhs);
 return std::move(rhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const charT* lhs, const basic_string<charT, traits, Allocator>& rhs);
```

5 *Effects:* Equivalent to:

```
 basic_string<charT, traits, Allocator> r = rhs;
 r.insert(0, lhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(charT lhs, const basic_string<charT, traits, Allocator>& rhs);
```

6 *Effects:* Equivalent to:

```
 basic_string<charT, traits, Allocator> r = rhs;
 r.insert(r.begin(), lhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(charT lhs, basic_string<charT, traits, Allocator>&& rhs);
```

7 *Effects:* Equivalent to:

```
 rhs.insert(rhs.begin(), lhs);
 return std::move(rhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs, charT rhs);
```

8 *Effects:* Equivalent to:

```
 basic_string<charT, traits, Allocator> r = lhs;
 r.push_back(rhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs, charT rhs);
```

9 *Effects:* Equivalent to:

```
 lhs.push_back(rhs);
 return std::move(lhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(const basic_string<charT, traits, Allocator>& lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs);
```

10 Equivalent to:

```
 basic_string<charT, traits, Allocator> r = lhs;
 r.append(rhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(basic_string<charT, traits, Allocator>&& lhs,
 type_identity_t<basic_string_view<charT, traits>> rhs);
```

11 Equivalent to:

```
 lhs.append(rhs);
 return std::move(lhs);
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
 const basic_string<charT, traits, Allocator>& rhs);
```

12 Equivalent to:

```
 basic_string<charT, traits, Allocator> r = rhs;
 r.insert(0, lhs);
 return r;
```

```
template<class charT, class traits, class Allocator>
constexpr basic_string<charT, traits, Allocator>
operator+(type_identity_t<basic_string_view<charT, traits>> lhs,
 basic_string<charT, traits, Allocator>&& rhs);
```

13 Equivalent to:

```
 rhs.insert(0, lhs);
 return std::move(rhs);
```

14 [Note 2: Using a specialization of `type_identity_t` as a parameter type ensures that an object of type `basic_string<charT, traits, Allocator>` can be concatenated with an object of a type `T` having an implicit conversion to `basic_string_view<charT, traits>` (12.2.2.3). — end note]



## 27.4.4.2 Non-member comparison operator functions

[string.cmp]

```

template<class charT, class traits, class Allocator>
constexpr bool
 operator==(const basic_string<charT, traits, Allocator>& lhs,
 const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr bool operator==(const basic_string<charT, traits, Allocator>& lhs,
 const charT* rhs);

template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
 const basic_string<charT, traits, Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
constexpr see below operator<=>(const basic_string<charT, traits, Allocator>& lhs,
 const charT* rhs);

```

1 *Effects:* Let *op* be the operator. Equivalent to:

```
return basic_string_view<charT, traits>(lhs) op basic_string_view<charT, traits>(rhs);
```

## 27.4.4.3 swap

[string.special]

```

template<class charT, class traits, class Allocator>
constexpr void
 swap(basic_string<charT, traits, Allocator>& lhs,
 basic_string<charT, traits, Allocator>& rhs)
 noexcept(noexcept(lhs.swap(rhs)));

```

1 *Effects:* Equivalent to `lhs.swap(rhs)`.

## 27.4.4.4 Inserters and extractors

[string.io]

```

template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
 operator>>(basic_istream<charT, traits>& is, basic_string<charT, traits, Allocator>& str);

```

1 *Effects:* Behaves as a formatted input function (31.7.5.3.1). After constructing a `sentry` object, if the `sentry` object returns `true` when converted to a value of type `bool`, calls `str.erase()` and then extracts characters from `is` and appends them to `str` as if by calling `str.append(1, c)`. If `is.width()` is greater than zero, the maximum number `n` of characters appended is `is.width()`; otherwise `n` is `str.max_size()`. Characters are extracted and appended until any of the following occurs:

- (1.1) — `n` characters are stored;
- (1.2) — end-of-file occurs on the input sequence;
- (1.3) — `isspace(c, is.getloc())` is `true` for the next available input character `c`.

2 After the last character (if any) is extracted, `is.width(0)` is called and the `sentry` object is destroyed.

3 If the function extracts no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

4 *Returns:* `is`.

```

template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 const basic_string<charT, traits, Allocator>& str);

```

5 *Effects:* Equivalent to: `return os << basic_string_view<charT, traits>(str);`

```

template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
 getline(basic_istream<charT, traits>& is,
 basic_string<charT, traits, Allocator>& str,
 charT delim);

```

```
template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
getline(basic_istream<charT, traits>&& is,
 basic_string<charT, traits, Allocator>& str,
 charT delim);
```

6 *Effects:* Behaves as an unformatted input function (31.7.5.4), except that it does not affect the value returned by subsequent calls to `basic_istream<>::gcount()`. After constructing a `sentry` object, if the `sentry` object returns `true` when converted to a value of type `bool`, calls `str.erase()` and then extracts characters from `is` and appends them to `str` as if by calling `str.append(1, c)` until any of the following occurs:

- (6.1) — end-of-file occurs on the input sequence;
- (6.2) — `traits::eq(c, delim)` for the next available input character *c* (in which case, *c* is extracted but not appended);
- (6.3) — `str.max_size()` characters are stored (in which case, `ios_base::failbit` is set in the input function's local error state).

7 The conditions are tested in the order shown. In any case, after the last character is extracted, the `sentry` object is destroyed.

8 If the function extracts no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

9 *Returns:* `is`.

```
template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
getline(basic_istream<charT, traits>& is,
 basic_string<charT, traits, Allocator>& str);
template<class charT, class traits, class Allocator>
basic_istream<charT, traits>&
getline(basic_istream<charT, traits>&& is,
 basic_string<charT, traits, Allocator>& str);
```

10 *Returns:* `getline(is, str, is.widen('\n'))`.

#### 27.4.4.5 Erasure

[string.erase]

```
template<class charT, class traits, class Allocator, class U = charT>
constexpr typename basic_string<charT, traits, Allocator>::size_type
erase(basic_string<charT, traits, Allocator>& c, const U& value);
```

1 *Effects:* Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class charT, class traits, class Allocator, class Predicate>
constexpr typename basic_string<charT, traits, Allocator>::size_type
erase_if(basic_string<charT, traits, Allocator>& c, Predicate pred);
```

2 *Effects:* Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

#### 27.4.5 Numeric conversions

[string.conversions]

```
int stoi(const string& str, size_t* idx = nullptr, int base = 10);
long stol(const string& str, size_t* idx = nullptr, int base = 10);
unsigned long stoul(const string& str, size_t* idx = nullptr, int base = 10);
long long stoll(const string& str, size_t* idx = nullptr, int base = 10);
```

```
unsigned long long stoull(const string& str, size_t* idx = nullptr, int base = 10);
```

1     *Effects:* The first two functions call `strtol(str.c_str(), ptr, base)`, and the last three functions call `strtoul(str.c_str(), ptr, base)`, `strtoll(str.c_str(), ptr, base)`, and `strtoull(str.c_str(), ptr, base)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != nullptr`, the function stores in `*idx` the index of the first unconverted element of `str`.

2     *Returns:* The converted result.

3     *Throws:* `invalid_argument` if `strtol`, `strtoul`, `strtoll`, or `strtoull` reports that no conversion can be performed. Throws `out_of_range` if `strtol`, `strtoul`, `strtoll` or `strtoull` sets `errno` to `ERANGE`, or if the converted value is outside the range of representable values for the return type.

```
float stof(const string& str, size_t* idx = nullptr);
double stod(const string& str, size_t* idx = nullptr);
long double stold(const string& str, size_t* idx = nullptr);
```

4     *Effects:* These functions call `strtof(str.c_str(), ptr)`, `strtod(str.c_str(), ptr)`, and `strtold(str.c_str(), ptr)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != nullptr`, the function stores in `*idx` the index of the first unconverted element of `str`.

5     *Returns:* The converted result.

6     *Throws:* `invalid_argument` if `strtof`, `strtod`, or `strtold` reports that no conversion can be performed. Throws `out_of_range` if `strtof`, `strtod`, or `strtold` sets `errno` to `ERANGE` or if the converted value is outside the range of representable values for the return type.

```
constexpr string to_string(int val);
constexpr string to_string(unsigned val);
constexpr string to_string(long val);
constexpr string to_string(unsigned long val);
constexpr string to_string(long long val);
constexpr string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
```

7     *Returns:* `format("{} ", val)`.

```
int stoi(const wstring& str, size_t* idx = nullptr, int base = 10);
long stol(const wstring& str, size_t* idx = nullptr, int base = 10);
unsigned long stoul(const wstring& str, size_t* idx = nullptr, int base = 10);
long long stoll(const wstring& str, size_t* idx = nullptr, int base = 10);
unsigned long long stoull(const wstring& str, size_t* idx = nullptr, int base = 10);
```

8     *Effects:* The first two functions call `wcstol(str.c_str(), ptr, base)`, and the last three functions call `wcstoul(str.c_str(), ptr, base)`, `wcstoll(str.c_str(), ptr, base)`, and `wcstoull(str.c_str(), ptr, base)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != nullptr`, the function stores in `*idx` the index of the first unconverted element of `str`.

9     *Returns:* The converted result.

10    *Throws:* `invalid_argument` if `wcstol`, `wcstoul`, `wcstoll`, or `wcstoull` reports that no conversion can be performed. Throws `out_of_range` if the converted value is outside the range of representable values for the return type.

```
float stof(const wstring& str, size_t* idx = nullptr);
double stod(const wstring& str, size_t* idx = nullptr);
long double stold(const wstring& str, size_t* idx = nullptr);
```

11    *Effects:* These functions call `wcstof(str.c_str(), ptr)`, `wcstod(str.c_str(), ptr)`, and `wcstold(str.c_str(), ptr)`, respectively. Each function returns the converted result, if any. The argument

`ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != nullptr`, the function stores in `*idx` the index of the first unconverted element of `str`.

12 *Returns:* The converted result.

13 *Throws:* `invalid_argument` if `wcstof`, `wcstod`, or `wcstold` reports that no conversion can be performed. Throws `out_of_range` if `wcstof`, `wcstod`, or `wcstold` sets `errno` to `ERANGE`.

```
constexpr wstring to_wstring(int val);
constexpr wstring to_wstring(unsigned val);
constexpr wstring to_wstring(long val);
constexpr wstring to_wstring(unsigned long val);
constexpr wstring to_wstring(long long val);
constexpr wstring to_wstring(unsigned long long val);
wstring to_wstring(float val);
wstring to_wstring(double val);
wstring to_wstring(long double val);
```

14 *Returns:* `format(L"{", val)`.

## 27.4.6 Hash support

[basic.string.hash]

```
template<class A> struct hash<basic_string<char, char_traits<char>, A>>;
template<class A> struct hash<basic_string<char8_t, char_traits<char8_t>, A>>;
template<class A> struct hash<basic_string<char16_t, char_traits<char16_t>, A>>;
template<class A> struct hash<basic_string<char32_t, char_traits<char32_t>, A>>;
template<class A> struct hash<basic_string<wchar_t, char_traits<wchar_t>, A>>;
```

1 If `S` is one of these string types, `SV` is the corresponding string view type, and `s` is an object of type `S`, then `hash<S>()(s) == hash<SV>()(SV(s))`.

## 27.4.7 Suffix for `basic_string` literals

[basic.string.literals]

```
constexpr string operator""s(const char* str, size_t len);
```

1 *Returns:* `string{str, len}`.

```
constexpr u8string operator""s(const char8_t* str, size_t len);
```

2 *Returns:* `u8string{str, len}`.

```
constexpr u16string operator""s(const char16_t* str, size_t len);
```

3 *Returns:* `u16string{str, len}`.

```
constexpr u32string operator""s(const char32_t* str, size_t len);
```

4 *Returns:* `u32string{str, len}`.

```
constexpr wstring operator""s(const wchar_t* str, size_t len);
```

5 *Returns:* `wstring{str, len}`.

6 [Note 1: The same suffix `s` is used for `chrono::duration` literals denoting seconds but there is no conflict, since duration suffixes apply to numbers and string literal suffixes apply to character array literals. — end note]

## 27.5 Null-terminated sequence utilities

[c.strings]

### 27.5.1 Header `<cstring>` synopsis

[cstring.syn]

```
#define __STDC_VERSION_STRING_H__ 202311L
```

```
namespace std {
```

```
 using size_t = see 17.2.4;
```

// freestanding

```
 void* memcpy(void* s1, const void* s2, size_t n);
```

// freestanding

```
 void* memccpy(void* s1, const void* s2, int c, size_t n);
```

// freestanding

```
 void* memmove(void* s1, const void* s2, size_t n);
```

// freestanding

```
 char* strcpy(char* s1, const char* s2);
```

// freestanding

```
 char* strncpy(char* s1, const char* s2, size_t n);
```

// freestanding

```
 char* strdup(const char* s);
```

```

char* strndup(const char* s, size_t size);
char* strcat(char* s1, const char* s2); // freestanding
char* strncat(char* s1, const char* s2, size_t n); // freestanding
int memcmp(const void* s1, const void* s2, size_t n); // freestanding
int strcmp(const char* s1, const char* s2); // freestanding
int strcoll(const char* s1, const char* s2);
int strncmp(const char* s1, const char* s2, size_t n); // freestanding
size_t strxfrm(char* s1, const char* s2, size_t n);
const void* memchr(const void* s, int c, size_t n); // freestanding; see 16.2
void* memchr(void* s, int c, size_t n); // freestanding; see 16.2
const char* strchr(const char* s, int c); // freestanding; see 16.2
char* strchr(char* s, int c); // freestanding; see 16.2
size_t strcspn(const char* s1, const char* s2); // freestanding
const char* strpbrk(const char* s1, const char* s2); // freestanding; see 16.2
char* strpbrk(char* s1, const char* s2); // freestanding; see 16.2
const char* strrchr(const char* s, int c); // freestanding; see 16.2
char* strrchr(char* s, int c); // freestanding; see 16.2
size_t strspn(const char* s1, const char* s2); // freestanding
const char* strstr(const char* s1, const char* s2); // freestanding; see 16.2
char* strstr(char* s1, const char* s2); // freestanding; see 16.2
char* strtok(char* s1, const char* s2);
void* memset(void* s, int c, size_t n); // freestanding
void* memset_explicit(void* s, int c, size_t n); // freestanding
char* strerror(int errnum);
size_t strlen(const char* s); // freestanding
}

#define NULL see 17.2.3 // freestanding

```

- <sup>1</sup> The contents and meaning of the header `<cstring>` are the same as the C standard library header `<string.h>`.
- <sup>2</sup> The functions `strerror` and `strtok` are not required to avoid data races (16.4.6.10).
- <sup>3</sup> The functions `memcpy` and `memmove` are signal-safe (17.14.5). Each of these functions implicitly creates objects (6.8.2) in the destination region of storage immediately prior to copying the sequence of characters to the destination. Each of these functions returns a pointer to a suitable created object, if any, otherwise the value of the first parameter.
- <sup>4</sup> [Note 1: The functions `strchr`, `strpbrk`, `strrchr`, `strstr`, and `memchr`, have different signatures in this document, but they have the same behavior as in the C standard library (16.2). — end note]

SEE ALSO: ISO/IEC 9899:2024, 7.26

## 28 Text processing library

[text]

### 28.1 General

[text.general]

This Clause describes components for dealing with text. These components are summarized in [Table 89](#).

Table 89 — Text library summary [tab:text.summary]

| Subclause                                               | Header                                           |
|---------------------------------------------------------|--------------------------------------------------|
| <a href="#">28.2</a> Primitive numeric conversions      | <charconv>                                       |
| <a href="#">28.3</a> Localization library               | <locale>, <clocale>                              |
| <a href="#">28.4</a> Text encodings identification      | <text_encoding>                                  |
| <a href="#">28.5</a> Formatting                         | <format>                                         |
| <a href="#">28.6</a> Regular expressions library        | <regex>                                          |
| <a href="#">28.7</a> Null-terminated sequence utilities | <cctype>, <cstdlib>, <cuchar>, <wchar>, <wctype> |

### 28.2 Primitive numeric conversions

[charconv]

#### 28.2.1 Header <charconv> synopsis

[charconv.syn]

- <sup>1</sup> When a function is specified with a type placeholder of *integer-type*, the implementation provides overloads for `char` and all signed and unsigned integer types in lieu of *integer-type*. When a function is specified with a type placeholder of *floating-point-type*, the implementation provides overloads for all cv-unqualified floating-point types ([6.9.3](#)) in lieu of *floating-point-type*.

```
namespace std {
 // floating-point format for primitive numerical conversion
 enum class chars_format {
 scientific = unspecified,
 fixed = unspecified,
 hex = unspecified,
 general = fixed | scientific
 };

 // 28.2.2, primitive numerical output conversion
 struct to_chars_result { // freestanding
 char* ptr;
 errc ec;
 friend bool operator==(const to_chars_result&, const to_chars_result&) = default;
 constexpr explicit operator bool() const noexcept { return ec == errc{}; }
 };

 constexpr to_chars_result to_chars(char* first, char* last, // freestanding
 integer-type value, int base = 10);
 to_chars_result to_chars(char* first, char* last, // freestanding
 bool value, int base = 10) = delete;

 to_chars_result to_chars(char* first, char* last, // freestanding-deleted
 floating-point-type value);
 to_chars_result to_chars(char* first, char* last, // freestanding-deleted
 floating-point-type value, chars_format fmt);
 to_chars_result to_chars(char* first, char* last, // freestanding-deleted
 floating-point-type value, chars_format fmt, int precision);

 // 28.2.3, primitive numerical input conversion
 struct from_chars_result { // freestanding
 const char* ptr;
 errc ec;
 friend bool operator==(const from_chars_result&, const from_chars_result&) = default;
```

```

constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};

constexpr from_chars_result from_chars(const char* first, const char* last, // freestanding
 integer-type& value, int base = 10);

from_chars_result from_chars(const char* first, const char* last, // freestanding-deleted
 floating-point-type& value,
 chars_format fmt = chars_format::general);
}

```

- <sup>2</sup> The type `chars_format` is a bitmask type (16.3.3.3.3) with elements `scientific`, `fixed`, and `hex`.
- <sup>3</sup> The types `to_chars_result` and `from_chars_result` have the data members and special members specified above. They have no base classes or members other than those specified.

## 28.2.2 Primitive numeric output conversion

[charconv.to.chars]

- <sup>1</sup> All functions named `to_chars` convert `value` into a character string by successively filling the range `[first, last)`, where `[first, last)` is required to be a valid range. If the member `ec` of the return value is such that the value is equal to the value of a value-initialized `errc`, the conversion was successful and the member `ptr` is the one-past-the-end pointer of the characters written. Otherwise, the member `ec` has the value `errc::value_too_large`, the member `ptr` has the value `last`, and the contents of the range `[first, last)` are unspecified.
- <sup>2</sup> The functions that take a floating-point `value` but not a `precision` parameter ensure that the string representation consists of the smallest number of characters such that there is at least one digit before the radix point (if present) and parsing the representation using the corresponding `from_chars` function recovers `value` exactly.

[Note 1: This guarantee applies only if `to_chars` and `from_chars` are executed on the same implementation. — end note]

If there are several such representations, the representation with the smallest difference from the floating-point argument value is chosen, resolving any remaining ties using rounding according to `round_to_nearest` (17.3.4).

- <sup>3</sup> The functions taking a `chars_format` parameter determine the conversion specifier for `printf` as follows: The conversion specifier is `f` if `fmt` is `chars_format::fixed`, `e` if `fmt` is `chars_format::scientific`, `a` (without leading "0x" in the result) if `fmt` is `chars_format::hex`, and `g` if `fmt` is `chars_format::general`.

```
constexpr to_chars_result to_chars(char* first, char* last, integer-type value, int base = 10);
```

- <sup>4</sup> *Preconditions:* `base` has a value between 2 and 36 (inclusive).

- <sup>5</sup> *Effects:* The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 10..35 (inclusive) are represented as lowercase characters `a..z`. If `value` is less than zero, the representation starts with `'-'`.

- <sup>6</sup> *Throws:* Nothing.

```
to_chars_result to_chars(char* first, char* last, floating-point-type value);
```

- <sup>7</sup> *Effects:* Let `T` be `floating-point-type`. `value` is converted to a string in the style of `printf` in the "C" locale. The conversion specifier is `f` if the absolute value of `value` is in the range `[l, u)`, where `l` is the smallest value larger than or equal to  $10^{-4}$  that is representable by `T` and `u` is the largest value smaller than or equal to `numeric_limits<T>::radixnumeric_limits<T>::digits + 1` rounded down to the nearest power of 10 representable by `T`, otherwise `e`.

- <sup>8</sup> *Throws:* Nothing.

```
to_chars_result to_chars(char* first, char* last, floating-point-type value, chars_format fmt);
```

- <sup>9</sup> *Preconditions:* `fmt` has the value of one of the enumerators of `chars_format`.

- <sup>10</sup> *Effects:* `value` is converted to a string in the style of `printf` in the "C" locale.

- <sup>11</sup> *Throws:* Nothing.

```
to_chars_result to_chars(char* first, char* last, floating-point-type value,
 chars_format fmt, int precision);
```

12     *Preconditions:* `fmt` has the value of one of the enumerators of `chars_format`.

13     *Effects:* `value` is converted to a string in the style of `printf` in the "C" locale with the given precision.

14     *Throws:* Nothing.

SEE ALSO: ISO/IEC 9899:2024, 7.23.6.2

### 28.2.3 Primitive numeric input conversion

[`charconv.from.chars`]

- 1 All functions named `from_chars` analyze the string `[first,last)` for a pattern, where `[first,last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`.

[*Note 1:* If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note*]

Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest` (17.3.4), and the member `ec` is value-initialized.

```
constexpr from_chars_result from_chars(const char* first, const char* last,
 integer-type& value, int base = 10);
```

2     *Preconditions:* `base` has a value between 2 and 36 (inclusive).

3     *Effects:* The pattern is the expected form of the subject sequence in the "C" locale for the given nonzero base, as described for `strtoul`, except that no "0b" or "0B" prefix shall appear if the value of `base` is 2, no "0x" or "0X" prefix shall appear if the value of `base` is 16, and except that '-' is the only sign that may appear, and only if `value` has a signed type.

4     *Throws:* Nothing.

```
from_chars_result from_chars(const char* first, const char* last, floating-point-type& value,
 chars_format fmt = chars_format::general);
```

5     *Preconditions:* `fmt` has the value of one of the enumerators of `chars_format`.

6     *Effects:* The pattern is the expected form of the subject sequence in the "C" locale, as described for `strtod`, except that

- (6.1) — the sign '+' may only appear in the exponent part;
- (6.2) — if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- (6.3) — if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- (6.4) — if `fmt` is `chars_format::hex`, the prefix "0x" or "0X" is assumed.

[*Example 1:* The string 0x123 is parsed to have the value 0 with remaining characters x123. — *end example*]

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

7     *Throws:* Nothing.

SEE ALSO: ISO/IEC 9899:2024, 7.24.2.6, 7.24.2.8

## 28.3 Localization library

[`localization`]

### 28.3.1 General

[`localization.general`]

- 1 Subclause 28.3 describes components that C++ programs may use to encapsulate (and therefore be more portable when confronting) cultural differences. The locale facility includes internationalization support for



character classification and string collation, numeric, monetary, and date/time formatting and parsing, and message retrieval.

- <sup>2</sup> The following subclauses describe components for locales themselves, the standard facets, and facilities from the C library, as summarized in [Table 90](#).

**Table 90 — Localization library summary** [tab:localization.summary]

|                        | Subclause                  | Header    |
|------------------------|----------------------------|-----------|
| <a href="#">28.3.3</a> | Locales                    | <locale>  |
| <a href="#">28.3.4</a> | Standard locale categories |           |
| <a href="#">28.3.5</a> | C library locales          | <clocale> |

### 28.3.2 Header <locale> synopsis

[locale.syn]

```
namespace std {
 // 28.3.3.1, locale
 class locale;
 template<class Facet> const Facet& use_facet(const locale&);
 template<class Facet> bool has_facet(const locale&) noexcept;

 // 28.3.3.3, convenience interfaces
 template<class charT> bool isspace (charT c, const locale& loc);
 template<class charT> bool isprint (charT c, const locale& loc);
 template<class charT> bool iscntrl (charT c, const locale& loc);
 template<class charT> bool isupper (charT c, const locale& loc);
 template<class charT> bool islower (charT c, const locale& loc);
 template<class charT> bool isalpha (charT c, const locale& loc);
 template<class charT> bool isdigit (charT c, const locale& loc);
 template<class charT> bool ispunct (charT c, const locale& loc);
 template<class charT> bool isxdigit(charT c, const locale& loc);
 template<class charT> bool isalnum (charT c, const locale& loc);
 template<class charT> bool isgraph (charT c, const locale& loc);
 template<class charT> bool isblank (charT c, const locale& loc);
 template<class charT> charT toupper(charT c, const locale& loc);
 template<class charT> charT tolower(charT c, const locale& loc);

 // 28.3.4.2, ctype
 class ctype_base;
 template<class charT> class ctype;
 template<> class ctype<char>; // specialization
 template<class charT> class ctype_byname;
 class codecvt_base;
 template<class internT, class externT, class stateT> class codecvt;
 template<class internT, class externT, class stateT> class codecvt_byname;

 // 28.3.4.3, numeric
 template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class num_get;
 template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class num_put;
 template<class charT>
 class numpunct;
 template<class charT>
 class numpunct_byname;

 // 28.3.4.5, collation
 template<class charT> class collate;
 template<class charT> class collate_byname;

 // 28.3.4.6, date and time
 class time_base;
```

```

template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class time_get;
template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class time_get_byname;
template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class time_put;
template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class time_put_byname;

// 28.3.4.7, money
class money_base;
template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class money_get;
template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class money_put;
template<class charT, bool Intl = false>
 class money_punct;
template<class charT, bool Intl = false>
 class money_punct_byname;

// 28.3.4.8, message retrieval
class messages_base;
template<class charT> class messages;
template<class charT> class messages_byname;
}

```

- <sup>1</sup> The header `<locale>` defines classes and declares functions that encapsulate and manipulate the information peculiar to a locale.<sup>192</sup>

### 28.3.3 Locales

[locales]

#### 28.3.3.1 Class locale

[locale]

##### 28.3.3.1.1 General

[locale.general]

```

namespace std {
 class locale {
 public:
 // 28.3.3.1.2, types
 // 28.3.3.1.2.2, class locale::facet
 class facet;
 // 28.3.3.1.2.3, class locale::id
 class id;
 // 28.3.3.1.2.1, type locale::category
 using category = int;
 static const category // values assigned here are for exposition only
 none = 0,
 collate = 0x010, ctype = 0x020,
 monetary = 0x040, numeric = 0x080,
 time = 0x100, messages = 0x200,
 all = collate | ctype | monetary | numeric | time | messages;

 // 28.3.3.1.3, construct/copy/destroy
 locale() noexcept;
 locale(const locale& other) noexcept;
 explicit locale(const char* std_name);
 explicit locale(const string& std_name);
 locale(const locale& other, const char* std_name, category);
 locale(const locale& other, const string& std_name, category);
 template<class Facet> locale(const locale& other, Facet* f);
 locale(const locale& other, const locale& one, category);
 ~locale(); // not virtual
 const locale& operator=(const locale& other) noexcept;
 };
}

```

<sup>192</sup> In this subclause, the type name `tm` is an incomplete type that is defined in `<ctime>` (30.15).

```

// 28.3.3.1.4, locale operations
template<class Facet> locale combine(const locale& other) const;
string name() const;
text_encoding encoding() const;

bool operator==(const locale& other) const;

template<class charT, class traits, class Allocator>
 bool operator()(const basic_string<charT, traits, Allocator>& s1,
 const basic_string<charT, traits, Allocator>& s2) const;

// 28.3.3.1.6, global locale objects
static locale global(const locale&);
static const locale& classic();
};
}

```

<sup>1</sup> Class `locale` implements a type-safe polymorphic set of facets, indexed by facet *type*. In other words, a facet has a dual role: in one sense, it's just a class interface; at the same time, it's an index into a locale's set of facets.

<sup>2</sup> Access to the facets of a `locale` is via two function templates, `use_facet<>` and `has_facet<>`.

<sup>3</sup> [Example 1: An `iostream` operator<< can be implemented as:<sup>193</sup>

```

template<class charT, class traits>
basic_ostream<charT, traits>&
operator<< (basic_ostream<charT, traits>& s, Date d) {
 typename basic_ostream<charT, traits>::sentry cerberos(s);
 if (cerberos) {
 tm tmbuf; d.extract(tmbuf);
 bool failed =
 use_facet<time_put<charT, ostreambuf_iterator<charT, traits>>>>(
 s.getloc()).put(s, s, s.fill(), &tmbuf, 'x').failed();
 if (failed)
 s.setstate(s.badbit); // can throw
 }
 return s;
}

```

— end example]

<sup>4</sup> In the call to `use_facet<Facet>(loc)`, the type argument chooses a facet, making available all members of the named type. If `Facet` is not present in a locale, it throws the standard exception `bad_cast`. A C++ program can check if a locale implements a particular facet with the function template `has_facet<Facet>()`. User-defined facets may be installed in a locale, and used identically as may standard facets.

<sup>5</sup> [Note 1: All locale semantics are accessed via `use_facet<>` and `has_facet<>`, except that:

(5.1) — A member operator template

```
operator()(const basic_string<C, T, A>&, const basic_string<C, T, A>&)
```

is provided so that a locale can be used as a predicate argument to the standard collections, to collate strings.

(5.2) — Convenient global interfaces are provided for traditional `ctype` functions such as `isdigit()` and `isspace()`, so that given a locale object `loc` a C++ program can call `isspace(c, loc)`. (This eases upgrading existing extractors (31.7.5.3).)

— end note]

<sup>6</sup> Once a facet reference is obtained from a locale object by calling `use_facet<>`, that reference remains usable, and the results from member functions of it may be cached and re-used, as long as some locale object refers to that facet.

<sup>7</sup> In successive calls to a locale facet member function on a facet object installed in the same locale, the returned result shall be identical.

<sup>193</sup>) Note that in the call to `put`, the stream is implicitly converted to an `ostreambuf_iterator<charT, traits>`.

- <sup>8</sup> A **locale** constructed from a name string (such as "POSIX"), or from parts of two named locales, has a name; all others do not. Named locales may be compared for equality; an unnamed locale is equal only to (copies of) itself. For an unnamed locale, `locale::name()` returns the string "\*".
- <sup>9</sup> Whether there is one global locale object for the entire program or one global locale object per thread is implementation-defined. Implementations should provide one global locale object per thread. If there is a single global locale object for the entire program, implementations are not required to avoid data races on it (16.4.6.10).

### 28.3.3.1.2 Types

[locale.types]

#### 28.3.3.1.2.1 Type `locale::category`

[locale.category]

```
using category = int;
```

- <sup>1</sup> *Valid category values* include the **locale** member bitmask elements `collate`, `ctype`, `monetary`, `numeric`, `time`, and `messages`, each of which represents a single locale category. In addition, **locale** member bitmask constant `none` is defined as zero and represents no category. And **locale** member bitmask constant `all` is defined such that the expression

```
(collate | ctype | monetary | numeric | time | messages | all) == all
```

is **true**, and represents the union of all categories. Further, the expression  $(X \mid Y)$ , where  $X$  and  $Y$  each represent a single category, represents the union of the two categories.

- <sup>2</sup> **locale** member functions expecting a `category` argument require one of the `category` values defined above, or the union of two or more such values. Such a `category` value identifies a set of locale categories. Each locale category, in turn, identifies a set of locale facets, including at least those shown in Table 91.

**Table 91 — Locale category facets** [tab:locale.category.facets]

| Category              | Includes facets                                                                                                                                                                                                                                                                                                                      |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>collate</code>  | <code>collate&lt;char&gt;</code> , <code>collate&lt;wchar_t&gt;</code>                                                                                                                                                                                                                                                               |
| <code>ctype</code>    | <code>ctype&lt;char&gt;</code> , <code>ctype&lt;wchar_t&gt;</code><br><code>codecvt&lt;char, char, mbstate_t&gt;</code><br><code>codecvt&lt;wchar_t, char, mbstate_t&gt;</code>                                                                                                                                                      |
| <code>monetary</code> | <code>moneypunct&lt;char&gt;</code> , <code>moneypunct&lt;wchar_t&gt;</code><br><code>moneypunct&lt;char, true&gt;</code> , <code>moneypunct&lt;wchar_t, true&gt;</code><br><code>money_get&lt;char&gt;</code> , <code>money_get&lt;wchar_t&gt;</code><br><code>money_put&lt;char&gt;</code> , <code>money_put&lt;wchar_t&gt;</code> |
| <code>numeric</code>  | <code>num_punct&lt;char&gt;</code> , <code>num_punct&lt;wchar_t&gt;</code><br><code>num_get&lt;char&gt;</code> , <code>num_get&lt;wchar_t&gt;</code><br><code>num_put&lt;char&gt;</code> , <code>num_put&lt;wchar_t&gt;</code>                                                                                                       |
| <code>time</code>     | <code>time_get&lt;char&gt;</code> , <code>time_get&lt;wchar_t&gt;</code><br><code>time_put&lt;char&gt;</code> , <code>time_put&lt;wchar_t&gt;</code>                                                                                                                                                                                 |
| <code>messages</code> | <code>messages&lt;char&gt;</code> , <code>messages&lt;wchar_t&gt;</code>                                                                                                                                                                                                                                                             |

- <sup>3</sup> For any locale `loc` either constructed, or returned by `locale::classic()`, and any facet `Facet` shown in Table 91, `has_facet<Facet>(loc)` is **true**. Each **locale** member function which takes a `locale::category` argument operates on the corresponding set of facets.
- <sup>4</sup> An implementation is required to provide those specializations for facet templates identified as members of a category, and for those shown in Table 92.
- <sup>5</sup> The provided implementation of members of facets `num_get<charT>` and `num_put<charT>` calls `use_facet<F>(1)` only for facet `F` of types `num_punct<charT>` and `ctype<charT>`, and for locale `l` the value obtained by calling member `getloc()` on the `ios_base&` argument to these functions.
- <sup>6</sup> In declarations of facets, a template parameter with name `InputIterator` or `OutputIterator` indicates the set of all possible specializations on parameters that meet the *Cpp17InputIterator* requirements or *Cpp17OutputIterator* requirements, respectively (24.3). A template parameter with name `C` represents the set of types containing `char`, `wchar_t`, and any other implementation-defined character container types (3.10) that meet the requirements for a character on which any of the iostream components can be instantiated. A

Table 92 — Required specializations [tab:locale.spec]

| Category | Includes facets                                                                                                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| collate  | collate_byname<char>, collate_byname<wchar_t>                                                                                                                                                                                                                                                                    |
| ctype    | ctype_byname<char>, ctype_byname<wchar_t><br>codecvt_byname<char, char, mbstate_t><br>codecvt_byname<wchar_t, char, mbstate_t>                                                                                                                                                                                   |
| monetary | moneypunct_byname<char, International><br>moneypunct_byname<wchar_t, International><br>money_get<C, InputIterator><br>money_put<C, OutputIterator>                                                                                                                                                               |
| numeric  | numpunct_byname<char>, numpunct_byname<wchar_t><br>num_get<C, InputIterator>, num_put<C, OutputIterator>                                                                                                                                                                                                         |
| time     | time_get<char, InputIterator><br>time_get_byname<char, InputIterator><br>time_get<wchar_t, InputIterator><br>time_get_byname<wchar_t, InputIterator><br>time_put<char, OutputIterator><br>time_put_byname<char, OutputIterator><br>time_put<wchar_t, OutputIterator><br>time_put_byname<wchar_t, OutputIterator> |
| messages | messages_byname<char>, messages_byname<wchar_t>                                                                                                                                                                                                                                                                  |

template parameter with name `International` represents the set of all possible specializations on a `bool` parameter.

#### 28.3.3.1.2.2 Class `locale::facet`

[locale.facet]

```

namespace std {
 class locale::facet {
 protected:
 explicit facet(size_t refs = 0);
 virtual ~facet();
 facet(const facet&) = delete;
 void operator=(const facet&) = delete;
 };
}

```

- <sup>1</sup> Class `facet` is the base class for locale feature sets. A class is a *facet* if it is publicly derived from another facet, or if it is a class derived from `locale::facet` and contains a publicly accessible declaration as follows:<sup>194</sup>

```
static ::std::locale::id id;
```

- <sup>2</sup> Template parameters in this Clause which are required to be facets are those named `Facet` in declarations. A program that passes a type that is *not* a facet, or a type that refers to a volatile-qualified facet, as an (explicit or deduced) template parameter to a locale function expecting a facet, is ill-formed. A const-qualified facet is a valid template argument to any locale function that expects a `Facet` template parameter.
- <sup>3</sup> The `refs` argument to the constructor is used for lifetime management. For `refs == 0`, the implementation performs `delete static_cast<locale::facet*>(f)` (where `f` is a pointer to the facet) when the last `locale` object containing the facet is destroyed; for `refs == 1`, the implementation never destroys the facet.
- <sup>4</sup> Constructors of all facets defined in this Clause take such an argument and pass it along to their `facet` base class constructor. All one-argument constructors defined in this Clause are explicit, preventing their participation in implicit conversions.
- <sup>5</sup> For some standard facets a standard “...\_byname” class, derived from it, implements the virtual function semantics equivalent to that facet of the locale constructed by `locale(const char*)` with the same name. Each such facet provides a constructor that takes a `const char*` argument, which names the locale, and a `refs` argument, which is passed to the base class constructor. Each such facet also provides a constructor that

<sup>194</sup> This is a complete list of requirements; there are no other requirements. Thus, a facet class need not have a public copy constructor, assignment, default constructor, destructor, etc.

takes a **string** argument **str** and a **refs** argument, which has the same effect as calling the first constructor with the two arguments **str.c\_str()** and **refs**. If there is no “**...\_byname**” version of a facet, the base class implements named locale semantics itself by reference to other facets.

#### 28.3.3.1.2.3 Class `locale::id`

[`locale.id`]

```
namespace std {
 class locale::id {
 public:
 id();
 void operator=(const id&) = delete;
 id(const id&) = delete;
 };
}
```

- 1 The class `locale::id` provides identification of a locale facet interface, used as an index for lookup and to encapsulate initialization.
- 2 [Note 1: Because facets are used by iostreams, potentially while static constructors are running, their initialization cannot depend on programmed static initialization. One initialization strategy is for `locale` to initialize each facet's `id` member the first time an instance of the facet is installed into a locale. This depends only on static storage being zero before constructors run (6.10.3.2). — end note]

#### 28.3.3.1.3 Constructors and destructor

[`locale.cons`]

```
locale() noexcept;
```

- 1 *Effects:* Constructs a copy of the argument last passed to `locale::global(locale&)`, if it has been called; else, the resulting facets have virtual function semantics identical to those of `locale::classic()`.  
[Note 1: This constructor yields a copy of the current global locale. It is commonly used as a default argument for function parameters of type `const locale&`. — end note]

```
explicit locale(const char* std_name);
```

- 2 *Effects:* Constructs a locale using standard C locale names, e.g., "POSIX". The resulting locale implements semantics defined to be associated with that name.
- 3 *Throws:* `runtime_error` if the argument is not valid, or is null.
- 4 *Remarks:* The set of valid string argument values is "C", "", and any implementation-defined values.

```
explicit locale(const string& std_name);
```

- 5 *Effects:* Equivalent to `locale(std_name.c_str())`.

```
locale(const locale& other, const char* std_name, category cats);
```

- 6 *Preconditions:* `cats` is a valid `category` value (28.3.3.1.2.1).
- 7 *Effects:* Constructs a locale as a copy of `other` except for the facets identified by the `category` argument, which instead implement the same semantics as `locale(std_name)`.
- 8 *Throws:* `runtime_error` if the second argument is not valid, or is null.
- 9 *Remarks:* The locale has a name if and only if `other` has a name.

```
locale(const locale& other, const string& std_name, category cats);
```

- 10 *Effects:* Equivalent to `locale(other, std_name.c_str(), cats)`.

```
template<class Facet> locale(const locale& other, Facet* f);
```

- 11 *Effects:* Constructs a locale incorporating all facets from the first argument except that of type `Facet`, and installs the second argument as the remaining facet. If `f` is null, the resulting object is a copy of `other`.
- 12 *Remarks:* If `f` is null, the resulting locale has the same name as `other`. Otherwise, the resulting locale has no name.

```
locale(const locale& other, const locale& one, category cats);
```

- 13 *Preconditions:* `cats` is a valid `category` value.

14 *Effects:* Constructs a locale incorporating all facets from the first argument except those that implement `cats`, which are instead incorporated from the second argument.

15 *Remarks:* If `cats` is equal to `locale::none`, the resulting locale has a name if and only if the first argument has a name. Otherwise, the resulting locale has a name if and only if the first two arguments both have names.

```
const locale& operator=(const locale& other) noexcept;
```

16 *Effects:* Creates a copy of `other`, replacing the current value.

17 *Returns:* `*this`.

#### 28.3.3.1.4 Members

[`locale.members`]

```
template<class Facet> locale combine(const locale& other) const;
```

1 *Effects:* Constructs a locale incorporating all facets from `*this` except for that one facet of `other` that is identified by `Facet`.

2 *Returns:* The newly created locale.

3 *Throws:* `runtime_error` if `has_facet<Facet>(other)` is `false`.

4 *Remarks:* The resulting locale has no name.

```
string name() const;
```

5 *Returns:* The name of `*this`, if it has one; otherwise, the string `"*"`.

```
text_encoding encoding() const;
```

6 *Mandates:* `CHAR_BIT == 8` is `true`.

7 *Returns:* A `text_encoding` object representing the implementation-defined encoding scheme associated with the locale `*this`.

#### 28.3.3.1.5 Operators

[`locale.operators`]

```
bool operator==(const locale& other) const;
```

1 *Returns:* `true` if both arguments are the same locale, or one is a copy of the other, or each has a name and the names are identical; `false` otherwise.

```
template<class charT, class traits, class Allocator>
bool operator()(const basic_string<charT, traits, Allocator>& s1,
 const basic_string<charT, traits, Allocator>& s2) const;
```

2 *Effects:* Compares two strings according to the `std::collate<charT>` facet.

3 *Returns:*

```
use_facet<std::collate<charT>>(*this).compare(s1.data(), s1.data() + s1.size(),
 s2.data(), s2.data() + s2.size()) < 0
```

4 *Remarks:* This member operator template (and therefore `locale` itself) meets the requirements for a comparator predicate template argument ([Clause 26](#)) applied to strings.

5 *[Example 1:* A vector of strings `v` can be collated according to collation rules in locale `loc` simply by ([26.8.2, 23.3.13](#)):

```
std::sort(v.begin(), v.end(), loc);
```

— end example]

#### 28.3.3.1.6 Static members

[`locale.statics`]

```
static locale global(const locale& loc);
```

1 *Effects:* Sets the global locale to its argument. Causes future calls to the constructor `locale()` to return a copy of the argument. If the argument has a name, does

```
setlocale(LC_ALL, loc.name().c_str());
```

otherwise, the effect on the C locale, if any, is implementation-defined.

2 *Returns:* The previous value of `locale()`.



3 *Remarks:* No library function other than `locale::global()` affects the value returned by `locale()`.  
 [Note 1: See 28.3.5 for data race considerations when `setlocale` is invoked. — end note]

```
static const locale& classic();
```

4 The "C" locale.

5 *Returns:* A locale that implements the classic "C" locale semantics, equivalent to the value `locale("C")`.

6 *Remarks:* This locale, its facets, and their member functions, do not change with time.

### 28.3.3.2 locale globals

[locale.global.templates]

```
template<class Facet> const Facet& use_facet(const locale& loc);
```

1 *Mandates:* Facet is a facet class whose definition contains the public static member `id` as defined in 28.3.3.1.2.2.

2 *Returns:* A reference to the corresponding facet of `loc`, if present.

3 *Throws:* `bad_cast` if `has_facet<Facet>(loc)` is false.

4 *Remarks:* The reference returned remains valid at least as long as any copy of `loc` exists.

```
template<class Facet> bool has_facet(const locale& loc) noexcept;
```

5 *Returns:* true if the facet requested is present in `loc`; otherwise false.

### 28.3.3.3 Convenience interfaces

[locale.convenience]

#### 28.3.3.3.1 Character classification

[classification]

```
template<class charT> bool isspace (charT c, const locale& loc);
template<class charT> bool isprint (charT c, const locale& loc);
template<class charT> bool iscntrl (charT c, const locale& loc);
template<class charT> bool isupper (charT c, const locale& loc);
template<class charT> bool islower (charT c, const locale& loc);
template<class charT> bool isalpha (charT c, const locale& loc);
template<class charT> bool isdigit (charT c, const locale& loc);
template<class charT> bool ispunct (charT c, const locale& loc);
template<class charT> bool isxdigit(charT c, const locale& loc);
template<class charT> bool isalnum (charT c, const locale& loc);
template<class charT> bool isgraph (charT c, const locale& loc);
template<class charT> bool isblank (charT c, const locale& loc);
```

1 Each of these functions `isF` returns the result of the expression:

```
use_facet<ctype<charT>>(loc).is(ctype_base::F, c)
```

where *F* is the `ctype_base::mask` value corresponding to that function (28.3.4.2).<sup>195</sup>

#### 28.3.3.3.2 Character conversions

[conversions.character]

```
template<class charT> charT toupper(charT c, const locale& loc);
```

1 *Returns:* `use_facet<ctype<charT>>(loc).toupper(c)`.

```
template<class charT> charT tolower(charT c, const locale& loc);
```

2 *Returns:* `use_facet<ctype<charT>>(loc).tolower(c)`.

### 28.3.4 Standard locale categories

[locale.categories]

#### 28.3.4.1 General

[locale.categories.general]

1 Each of the standard categories includes a family of facets. Some of these implement formatting or parsing of a datum, for use by standard or users' iostream operators `<<` and `>>`, as members `put()` and `get()`, respectively. Each such member function takes an `ios_base&` argument whose members `flags()`, `precision()`, and `width()`, specify the format of the corresponding datum (31.5.2). Those functions which need to use other facets call its member `getloc()` to retrieve the locale imbued there. Formatting facets use the character argument `fill` to fill out the specified width where necessary.

<sup>195</sup>) When used in a loop, it is faster to cache the `ctype<>` facet and use it directly, or use the vector form of `ctype<>::is`.



- <sup>2</sup> The `put()` members make no provision for error reporting. (Any failures of the `OutputIterator` argument can be extracted from the returned iterator.) The `get()` members take an `ios_base::iostate&` argument whose value they ignore, but set to `ios_base::failbit` in case of a parse error.
- <sup>3</sup> Within 28.3.4 it is unspecified whether one virtual function calls another virtual function.

#### 28.3.4.2 The ctype category

[category.ctype]

##### 28.3.4.2.1 General

[category.ctype.general]

```
namespace std {
 class ctype_base {
 public:
 using mask = see below;

 // numeric values are for exposition only.
 static constexpr mask space = 1 << 0;
 static constexpr mask print = 1 << 1;
 static constexpr mask cntrl = 1 << 2;
 static constexpr mask upper = 1 << 3;
 static constexpr mask lower = 1 << 4;
 static constexpr mask alpha = 1 << 5;
 static constexpr mask digit = 1 << 6;
 static constexpr mask punct = 1 << 7;
 static constexpr mask xdigit = 1 << 8;
 static constexpr mask blank = 1 << 9;
 static constexpr mask alnum = alpha | digit;
 static constexpr mask graph = alnum | punct;
 };
}
```

- <sup>1</sup> The type `mask` is a bitmask type (16.3.3.3.3).

#### 28.3.4.2.2 Class template ctype

[locale.ctype]

##### 28.3.4.2.2.1 General

[locale.ctype.general]

```
namespace std {
 template<class charT>
 class ctype : public locale::facet, public ctype_base {
 public:
 using char_type = charT;

 explicit ctype(size_t refs = 0);

 bool is(mask m, charT c) const;
 const charT* is(const charT* low, const charT* high, mask* vec) const;
 const charT* scan_is(mask m, const charT* low, const charT* high) const;
 const charT* scan_not(mask m, const charT* low, const charT* high) const;
 charT toupper(charT c) const;
 const charT* toupper(charT* low, const charT* high) const;
 charT tolower(charT c) const;
 const charT* tolower(charT* low, const charT* high) const;

 charT widen(char c) const;
 const char* widen(const char* low, const char* high, charT* to) const;
 char narrow(charT c, char dfault) const;
 const charT* narrow(const charT* low, const charT* high, char dfault, char* to) const;

 static locale::id id;

 protected:
 ~ctype();
 virtual bool do_is(mask m, charT c) const;
 virtual const charT* do_is(const charT* low, const charT* high, mask* vec) const;
 virtual const charT* do_scan_is(mask m, const charT* low, const charT* high) const;
 virtual const charT* do_scan_not(mask m, const charT* low, const charT* high) const;
 virtual charT do_toupper(charT) const;
```

```

virtual const charT* do_toupper(charT* low, const charT* high) const;
virtual charT do_tolower(charT) const;
virtual const charT* do_tolower(charT* low, const charT* high) const;
virtual charT do_widen(char) const;
virtual const char* do_widen(const char* low, const char* high, charT* dest) const;
virtual char do_narrow(charT, char default) const;
virtual const charT* do_narrow(const charT* low, const charT* high,
 char default, char* dest) const;
};
}

```

- <sup>1</sup> Class `ctype` encapsulates the C library `<cctype>` (28.7.1) features. `istream` members are required to use `ctype<>` for character classing during input parsing.
- <sup>2</sup> The specializations required in Table 91 (28.3.3.1.2.1), namely `ctype<char>` and `ctype<wchar_t>`, implement character classing appropriate to the implementation's native character set.

#### 28.3.4.2.2.2 `ctype` members

[locale.ctype.members]

```

bool is(mask m, charT c) const;
const charT* is(const charT* low, const charT* high, mask* vec) const;

```

- <sup>1</sup> *Returns:* `do_is(m, c)` or `do_is(low, high, vec)`.

```
const charT* scan_is(mask m, const charT* low, const charT* high) const;
```

- <sup>2</sup> *Returns:* `do_scan_is(m, low, high)`.

```
const charT* scan_not(mask m, const charT* low, const charT* high) const;
```

- <sup>3</sup> *Returns:* `do_scan_not(m, low, high)`.

```
charT toupper(charT c) const;
const charT* toupper(charT* low, const charT* high) const;

```

- <sup>4</sup> *Returns:* `do_toupper(c)` or `do_toupper(low, high)`.

```
charT tolower(charT c) const;
const charT* tolower(charT* low, const charT* high) const;

```

- <sup>5</sup> *Returns:* `do_tolower(c)` or `do_tolower(low, high)`.

```
charT widen(char c) const;
const char* widen(const char* low, const char* high, charT* to) const;

```

- <sup>6</sup> *Returns:* `do_widen(c)` or `do_widen(low, high, to)`.

```
char narrow(charT c, char default) const;
const charT* narrow(const charT* low, const charT* high, char default, char* to) const;

```

- <sup>7</sup> *Returns:* `do_narrow(c, default)` or `do_narrow(low, high, default, to)`.

#### 28.3.4.2.2.3 `ctype` virtual functions

[locale.ctype.virtuals]

```

bool do_is(mask m, charT c) const;
const charT* do_is(const charT* low, const charT* high, mask* vec) const;

```

- <sup>1</sup> *Effects:* Classifies a character or sequence of characters. For each argument character, identifies a value `M` of type `ctype_base::mask`. The second form identifies a value `M` of type `ctype_base::mask` for each `*p` where `(low <= p && p < high)`, and places it into `vec[p - low]`.

- <sup>2</sup> *Returns:* The first form returns the result of the expression `(M & m) != 0`; i.e., `true` if the character has the characteristics specified. The second form returns `high`.

```
const charT* do_scan_is(mask m, const charT* low, const charT* high) const;
```

- <sup>3</sup> *Effects:* Locates a character in a buffer that conforms to a classification `m`.

- <sup>4</sup> *Returns:* The smallest pointer `p` in the range `[low, high)` such that `is(m, *p)` would return `true`; otherwise, returns `high`.

```
const charT* do_scan_not(mask m, const charT* low, const charT* high) const;
```

5 *Effects:* Locates a character in a buffer that fails to conform to a classification *m*.

6 *Returns:* The smallest pointer *p*, if any, in the range [*low*,*high*) such that *is*(*m*, \**p*) would return *false*; otherwise, returns *high*.

```
charT do_toupper(charT c) const;
```

```
const charT* do_toupper(charT* low, const charT* high) const;
```

7 *Effects:* Converts a character or characters to upper case. The second form replaces each character \**p* in the range [*low*,*high*) for which a corresponding upper-case character exists, with that character.

8 *Returns:* The first form returns the corresponding upper-case character if it is known to exist, or its argument if not. The second form returns *high*.

```
charT do_tolower(charT c) const;
```

```
const charT* do_tolower(charT* low, const charT* high) const;
```

9 *Effects:* Converts a character or characters to lower case. The second form replaces each character \**p* in the range [*low*,*high*) and for which a corresponding lower-case character exists, with that character.

10 *Returns:* The first form returns the corresponding lower-case character if it is known to exist, or its argument if not. The second form returns *high*.

```
charT do_widen(char c) const;
```

```
const char* do_widen(const char* low, const char* high, charT* dest) const;
```

11 *Effects:* Applies the simplest reasonable transformation from a *char* value or sequence of *char* values to the corresponding *charT* value or values.<sup>196</sup> The only characters for which unique transformations are required are those in the basic character set (5.3.1).

For any named *ctype* category with a *ctype*<*char*> facet *ctc* and valid *ctype\_base::mask* value *M*, (*ctc.is*(*M*, *c*) || !*is*(*M*, *do\_widen*(*c*)) ) is true.<sup>197</sup>

The second form transforms each character \**p* in the range [*low*,*high*), placing the result in *dest*[*p* - *low*].

12 *Returns:* The first form returns the transformed value. The second form returns *high*.

```
char do_narrow(charT c, char ddefault) const;
```

```
const charT* do_narrow(const charT* low, const charT* high, char ddefault, char* dest) const;
```

13 *Effects:* Applies the simplest reasonable transformation from a *charT* value or sequence of *charT* values to the corresponding *char* value or values.

For any character *c* in the basic character set (5.3.1) the transformation is such that

```
do_widen(do_narrow(c, 0)) == c
```

For any named *ctype* category with a *ctype*<*char*> facet *ctc* however, and *ctype\_base::mask* value *M*,

```
(is(M, c) || !ctc.is(M, do_narrow(c, ddefault)))
```

is true (unless *do\_narrow* returns *ddefault*). In addition, for any digit character *c*, the expression (*do\_narrow*(*c*, *ddefault*) - '0') evaluates to the digit value of the character. The second form transforms each character \**p* in the range [*low*,*high*), placing the result (or *ddefault* if no simple transformation is readily available) in *dest*[*p* - *low*].

14 *Returns:* The first form returns the transformed value; or *ddefault* if no mapping is readily available. The second form returns *high*.

#### 28.3.4.2.3 Class template *ctype\_byname*

[*locale.ctype.byname*]

```
namespace std {
 template<class charT>
 class ctype_byname : public ctype<charT> {
 public:
 using mask = ctype<charT>::mask;
```

<sup>196</sup>) The parameter *c* of *do\_widen* is intended to accept values derived from *character-literals* for conversion to the locale's encoding.

<sup>197</sup>) In other words, the transformed character is not a member of any character classification that *c* is not also a member of.

```

explicit ctype_byname(const char*, size_t refs = 0);
explicit ctype_byname(const string&, size_t refs = 0);

protected:
 ~ctype_byname();
};
}

```

#### 28.3.4.2.4 ctype<char> specialization

[facet.ctype.special]

##### 28.3.4.2.4.1 General

[facet.ctype.special.general]

```

namespace std {
 template<>
 class ctype<char> : public locale::facet, public ctype_base {
 public:
 using char_type = char;

 explicit ctype(const mask* tab = nullptr, bool del = false, size_t refs = 0);

 bool is(mask m, char c) const;
 const char* is(const char* low, const char* high, mask* vec) const;
 const char* scan_is (mask m, const char* low, const char* high) const;
 const char* scan_not(mask m, const char* low, const char* high) const;

 char toupper(char c) const;
 const char* toupper(char* low, const char* high) const;
 char tolower(char c) const;
 const char* tolower(char* low, const char* high) const;

 char widen(char c) const;
 const char* widen(const char* low, const char* high, char* to) const;
 char narrow(char c, char dfault) const;
 const char* narrow(const char* low, const char* high, char dfault, char* to) const;

 static locale::id id;
 static const size_t table_size = implementation-defined;

 const mask* table() const noexcept;
 static const mask* classic_table() noexcept;

 protected:
 ~ctype();
 virtual char do_toupper(char c) const;
 virtual const char* do_toupper(char* low, const char* high) const;
 virtual char do_tolower(char c) const;
 virtual const char* do_tolower(char* low, const char* high) const;

 virtual char do_widen(char c) const;
 virtual const char* do_widen(const char* low, const char* high, char* to) const;
 virtual char do_narrow(char c, char dfault) const;
 virtual const char* do_narrow(const char* low, const char* high,
 char dfault, char* to) const;
 };
}

```

<sup>1</sup> A specialization `ctype<char>` is provided so that the member functions on type `char` can be implemented inline.<sup>198</sup> The implementation-defined value of member `table_size` is at least 256.

<sup>198</sup> Only the `char` (not `unsigned char` and `signed char`) form is provided. The specialization is specified in the standard, and not left as an implementation detail, because it affects the derivation interface for `ctype<char>`.

**28.3.4.2.4.2 Destructor**

[facet.ctype.char.dtor]

~ctype();

- 1 *Effects:* If the constructor's first argument was nonzero, and its second argument was `true`, does delete  
[] `table()`.

**28.3.4.2.4.3 Members**

[facet.ctype.char.members]

- 1 In the following member descriptions, for `unsigned char` values `v` where `v >= table_size`, `table()[v]` is assumed to have an implementation-specific value (possibly different for each such value `v`) without performing the array lookup.

```
explicit ctype(const mask* tbl = nullptr, bool del = false, size_t refs = 0);
```

- 2 *Preconditions:* Either `tbl == nullptr` is true or `[tbl, tbl + table_size)` is a valid range.

- 3 *Effects:* Passes its `refs` argument to its base class constructor.

```
bool is(mask m, char c) const;
const char* is(const char* low, const char* high, mask* vec) const;
```

- 4 *Effects:* The second form, for all `*p` in the range `[low, high)`, assigns into `vec[p - low]` the value `table()[ (unsigned char)*p ]`.

- 5 *Returns:* The first form returns `table()[ (unsigned char)c ] & m`; the second form returns `high`.

```
const char* scan_is(mask m, const char* low, const char* high) const;
```

- 6 *Returns:* The smallest `p` in the range `[low, high)` such that

```
table()[(unsigned char) *p] & m
is true.
```

```
const char* scan_not(mask m, const char* low, const char* high) const;
```

- 7 *Returns:* The smallest `p` in the range `[low, high)` such that

```
table()[(unsigned char) *p] & m
is false.
```

```
char toupper(char c) const;
const char* toupper(char* low, const char* high) const;
```

- 8 *Returns:* `do_toupper(c)` or `do_toupper(low, high)`, respectively.

```
char tolower(char c) const;
const char* tolower(char* low, const char* high) const;
```

- 9 *Returns:* `do_tolower(c)` or `do_tolower(low, high)`, respectively.

```
char widen(char c) const;
const char* widen(const char* low, const char* high, char* to) const;
```

- 10 *Returns:* `do_widen(c)` or `do_widen(low, high, to)`, respectively.

```
char narrow(char c, char dfault) const;
const char* narrow(const char* low, const char* high, char dfault, char* to) const;
```

- 11 *Returns:* `do_narrow(c, dfault)` or `do_narrow(low, high, dfault, to)`, respectively.

```
const mask* table() const noexcept;
```

- 12 *Returns:* The first constructor argument, if it was nonzero, otherwise `classic_table()`.

**28.3.4.2.4.4 Static members**

[facet.ctype.char.statics]

```
static const mask* classic_table() noexcept;
```

- 1 *Returns:* A pointer to the initial element of an array of size `table_size` which represents the classifications of characters in the "C" locale.

## 28.3.4.2.4.5 Virtual functions

[facet.ctype.char.virtuals]

```

char do_toupper(char) const;
const char* do_toupper(char* low, const char* high) const;
char do_tolower(char) const;
const char* do_tolower(char* low, const char* high) const;

virtual char do_widen(char c) const;
virtual const char* do_widen(const char* low, const char* high, char* to) const;
virtual char do_narrow(char c, char dfaul) const;
virtual const char* do_narrow(const char* low, const char* high,
 char dfaul, char* to) const;

```

- <sup>1</sup> These functions are described identically as those members of the same name in the `ctype` class template (28.3.4.2.2.2).

28.3.4.2.5 Class template `codecvt`

[locale.codecvt]

## 28.3.4.2.5.1 General

[locale.codecvt.general]

```

namespace std {
 class codecvt_base {
 public:
 enum result { ok, partial, error, noconv };
 };

 template<class internT, class externT, class stateT>
 class codecvt : public locale::facet, public codecvt_base {
 public:
 using intern_type = internT;
 using extern_type = externT;
 using state_type = stateT;

 explicit codecvt(size_t refs = 0);

 result out(
 stateT& state,
 const internT* from, const internT* from_end, const internT*& from_next,
 externT* to, externT* to_end, externT*& to_next) const;

 result unshift(
 stateT& state,
 externT* to, externT* to_end, externT*& to_next) const;

 result in(
 stateT& state,
 const externT* from, const externT* from_end, const externT*& from_next,
 internT* to, internT* to_end, internT*& to_next) const;

 int encoding() const noexcept;
 bool always_noconv() const noexcept;
 int length(stateT&, const externT* from, const externT* end, size_t max) const;
 int max_length() const noexcept;

 static locale::id id;

 protected:
 ~codecvt();
 virtual result do_out(
 stateT& state,
 const internT* from, const internT* from_end, const internT*& from_next,
 externT* to, externT* to_end, externT*& to_next) const;
 virtual result do_in(
 stateT& state,
 const externT* from, const externT* from_end, const externT*& from_next,
 internT* to, internT* to_end, internT*& to_next) const;
 };
}

```

```

 virtual result do_unshift(
 stateT& state,
 externT* to, externT* to_end, externT*& to_next) const;

 virtual int do_encoding() const noexcept;
 virtual bool do_always_noconv() const noexcept;
 virtual int do_length(stateT& state, const externT* from, const externT* end, size_t max) const;
 virtual int do_max_length() const noexcept;
};

```

- <sup>1</sup> The class `codecvt<internT, externT, stateT>` is for use when converting from one character encoding to another, such as from wide characters to multibyte characters or between wide character encodings such as UTF-32 and EUC.
- <sup>2</sup> The `stateT` argument selects the pair of character encodings being mapped between.
- <sup>3</sup> The specializations required in [Table 91 \(28.3.3.1.2.1\)](#) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. `codecvt<wchar_t, char, mbstate_t>` converts between the native character sets for ordinary and wide characters. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a program-defined `stateT` type. Objects of type `stateT` can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

#### 28.3.4.2.5.2 Members

[locale.codecvt.members]

```

result out(
 stateT& state,
 const internT* from, const internT* from_end, const internT*& from_next,
 externT* to, externT* to_end, externT*& to_next) const;

```

- <sup>1</sup> *Returns:* `do_out(state, from, from_end, from_next, to, to_end, to_next)`.

```

result unshift(stateT& state, externT* to, externT* to_end, externT*& to_next) const;

```

- <sup>2</sup> *Returns:* `do_unshift(state, to, to_end, to_next)`.

```

result in(
 stateT& state,
 const externT* from, const externT* from_end, const externT*& from_next,
 internT* to, internT* to_end, internT*& to_next) const;

```

- <sup>3</sup> *Returns:* `do_in(state, from, from_end, from_next, to, to_end, to_next)`.

```

int encoding() const noexcept;

```

- <sup>4</sup> *Returns:* `do_encoding()`.

```

bool always_noconv() const noexcept;

```

- <sup>5</sup> *Returns:* `do_always_noconv()`.

```

int length(stateT& state, const externT* from, const externT* from_end, size_t max) const;

```

- <sup>6</sup> *Returns:* `do_length(state, from, from_end, max)`.

```

int max_length() const noexcept;

```

- <sup>7</sup> *Returns:* `do_max_length()`.

#### 28.3.4.2.5.3 Virtual functions

[locale.codecvt.virtuals]

```

result do_out(
 stateT& state,
 const internT* from, const internT* from_end, const internT*& from_next,
 externT* to, externT* to_end, externT*& to_next) const;

```

```

result do_in(
 stateT& state,
 const externT* from, const externT* from_end, const externT*& from_next,
 internT* to, internT* to_end, internT*& to_next) const;

```

1     *Preconditions:* (`from <= from_end` && `to <= to_end`) is well-defined and **true**; **state** is initialized, if at the beginning of a sequence, or else is equal to the result of converting the preceding characters in the sequence.

2     *Effects:* Translates characters in the source range `[from,from_end)`, placing the results in sequential positions starting at destination `to`. Converts no more than `(from_end - from)` source elements, and stores no more than `(to_end - to)` destination elements.

3     Stops if it encounters a character it cannot convert. It always leaves the `from_next` and `to_next` pointers pointing one beyond the last element successfully converted. If it returns **noconv**, **internT** and **externT** are the same type, and the converted sequence is identical to the input sequence `[from,from_next)`, `to_next` is set equal to `to`, the value of **state** is unchanged, and there are no changes to the values in `[to,to_end)`.

4     A `codecvt` facet that is used by `basic_filebuf` (31.10) shall have the property that if

```
do_out(state, from, from_end, from_next, to, to_end, to_next)
```

would return **ok**, where `from != from_end`, then

```
do_out(state, from, from + 1, from_next, to, to_end, to_next)
```

shall also return **ok**, and that if

```
do_in(state, from, from_end, from_next, to, to_end, to_next)
```

would return **ok**, where `to != to_end`, then

```
do_in(state, from, from_end, from_next, to, to + 1, to_next)
```

shall also return **ok**.<sup>199</sup>

[*Note 1:* As a result of operations on **state**, it can return **ok** or **partial** and set `from_next == from` and `to_next != to`. — *end note*]

5     *Returns:* An enumeration value, as summarized in Table 93.

**Table 93 — do\_in/do\_out result values** [tab:locale.codecvt.inout]

| Value          | Meaning                                                                                                    |
|----------------|------------------------------------------------------------------------------------------------------------|
| <b>ok</b>      | completed the conversion                                                                                   |
| <b>partial</b> | not all source characters converted                                                                        |
| <b>error</b>   | encountered a character in <code>[from,from_end)</code> that cannot be converted                           |
| <b>noconv</b>  | <b>internT</b> and <b>externT</b> are the same type, and input sequence is identical to converted sequence |

A return value of **partial**, if (`from_next == from_end`), indicates that either the destination sequence has not absorbed all the available destination elements, or that additional source elements are needed before another destination element can be produced.

6     *Remarks:* Its operations on **state** are unspecified.

[*Note 2:* This argument can be used, for example, to maintain shift state, to specify conversion options (such as count only), or to identify a cache of seek offsets. — *end note*]

```

result do_unshift(stateT& state, externT* to, externT* to_end, externT*& to_next) const;

```

7     *Preconditions:* (`to <= to_end`) is well-defined and **true**; **state** is initialized, if at the beginning of a sequence, or else is equal to the result of converting the preceding characters in the sequence.

<sup>199</sup>) Informally, this means that `basic_filebuf` assumes that the mappings from internal to external characters is 1 to N: that a `codecvt` facet that is used by `basic_filebuf` can translate characters one internal character at a time.



- 8 *Effects:* Places characters starting at `to` that should be appended to terminate a sequence when the current `stateT` is given by `state`.<sup>200</sup> Stores no more than `(to_end - to)` destination elements, and leaves the `to_next` pointer pointing one beyond the last element successfully stored.
- 9 *Returns:* An enumeration value, as summarized in Table 94.

Table 94 — `do_unshift` result values [tab:locale.codecvt.unshift]

| Value                | Meaning                                                                                                                                    |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ok</code>      | completed the sequence                                                                                                                     |
| <code>partial</code> | space for more than <code>to_end - to</code> destination elements was needed to terminate a sequence given the value of <code>state</code> |
| <code>error</code>   | an unspecified error has occurred                                                                                                          |
| <code>noconv</code>  | no termination is needed for this <code>state_type</code>                                                                                  |

- ```
int do_encoding() const noexcept;
```
- 10 *Returns:* -1 if the encoding of the `externT` sequence is state-dependent; else the constant number of `externT` characters needed to produce an internal character; or 0 if this number is not a constant.²⁰¹
- ```
bool do_always_noconv() const noexcept;
```
- 11 *Returns:* true if `do_in()` and `do_out()` return `noconv` for all valid argument values. `codecvt<char, char, mbstate_t>` returns true.
- ```
int do_length(stateT& state, const externT* from, const externT* from_end, size_t max) const;
```
- 12 *Preconditions:* `(from <= from_end)` is well-defined and true; `state` is initialized, if at the beginning of a sequence, or else is equal to the result of converting the preceding characters in the sequence.
- 13 *Effects:* The effect on the `state` argument is as if it called `do_in(state, from, from_end, from, to, to + max, to)` for `to` pointing to a buffer of at least `max` elements.
- 14 *Returns:* `(from_next - from)` where `from_next` is the largest value in the range `[from, from_end)` such that the sequence of values in the range `[from, from_next)` represents `max` or fewer valid complete characters of type `internT`. The specialization `codecvt<char, char, mbstate_t>`, returns the lesser of `max` and `(from_end - from)`.
- ```
int do_max_length() const noexcept;
```
- 15 *Returns:* The maximum value that `do_length(state, from, from_end, 1)` can return for any valid range `[from, from_end)` and `stateT` value `state`. The specialization `codecvt<char, char, mbstate_t>::do_max_length()` returns 1.

#### 28.3.4.2.6 Class template `codecvt_byname`

[locale.codecvt.byname]

```
namespace std {
 template<class internT, class externT, class stateT>
 class codecvt_byname : public codecvt<internT, externT, stateT> {
 public:
 explicit codecvt_byname(const char*, size_t refs = 0);
 explicit codecvt_byname(const string&, size_t refs = 0);

 protected:
 ~codecvt_byname();
 };
}
```

200) Typically these will be characters to return the state to `stateT()`.

201) If `encoding()` yields -1, then more than `max_length()` `externT` elements can be consumed when producing a single `internT` character, and additional `externT` elements can appear at the end of a sequence after those that yield the final `internT` character.

**28.3.4.3 The numeric category**

[category.numeric]

**28.3.4.3.1 General**

[category.numeric.general]

- <sup>1</sup> The classes `num_get<>` and `num_put<>` handle numeric formatting and parsing. Virtual functions are provided for several numeric types. Implementations may (but are not required to) delegate extraction of smaller types to extractors for larger types.<sup>202</sup>
- <sup>2</sup> All specifications of member functions for `num_put` and `num_get` in the subclauses of 28.3.4.3 only apply to the specializations required in Tables 91 and 92 (28.3.3.1.2.1), namely `num_get<char>`, `num_get<wchar_t>`, `num_get<C, InputIterator>`, `num_put<char>`, `num_put<wchar_t>`, and `num_put<C, OutputIterator>`. These specializations refer to the `ios_base&` argument for formatting specifications (28.3.4), and to its imbued locale for the `num_punct<>` facet to identify all numeric punctuation preferences, and also for the `ctype<>` facet to perform character classification.
- <sup>3</sup> Extractor and inserter members of the standard iostreams use `num_get<>` and `num_put<>` member functions for formatting and parsing numeric values (31.7.5.3.1, 31.7.6.3.1).

**28.3.4.3.2 Class template num\_get**

[locale.num.get]

**28.3.4.3.2.1 General**

[locale.num.get.general]

```

namespace std {
 template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class num_get : public locale::facet {
 public:
 using char_type = charT;
 using iter_type = InputIterator;

 explicit num_get(size_t refs = 0);

 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, bool& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, long& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, long long& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, unsigned short& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, unsigned int& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, unsigned long& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, unsigned long long& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, float& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, double& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, long double& v) const;
 iter_type get(iter_type in, iter_type end, ios_base&,
 ios_base::iostate& err, void*& v) const;

 static locale::id id;

 protected:
 ~num_get();
 virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, bool& v) const;
 virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, long& v) const;
 virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, long long& v) const;

```

<sup>202</sup>) Parsing "-1" correctly into, e.g., an `unsigned short` requires that the corresponding member `get()` at least extract the sign before delegating.

```

virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, unsigned short& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, unsigned int& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, unsigned long& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, unsigned long long& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, float& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, double& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, long double& v) const;
virtual iter_type do_get(iter_type, iter_type, ios_base&,
 ios_base::iostate& err, void*& v) const;
};
}

```

<sup>1</sup> The facet `num_get` is used to parse numeric values from an input sequence such as an `istream`.

#### 28.3.4.3.2.2 Members

[facet.num.get.members]

```

iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, bool& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned short& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned int& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned long long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, float& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, double& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long double& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, void*& val) const;

```

<sup>1</sup> *Returns:* `do_get(in, end, str, err, val)`.

#### 28.3.4.3.2.3 Virtual functions

[facet.num.get.virtuals]

```

iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned short& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned int& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, unsigned long long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, float& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, double& val) const;

```

```

iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, long double& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
 ios_base::iostate& err, void*& val) const;

```

1 *Effects:* Reads characters from `in`, interpreting them according to `str.flags()`, `use_facet<ctype<charT>>(loc)`, and `use_facet<numput<charT>>(loc)`, where `loc` is `str.getloc()`.

2 The details of this operation occur in three stages:

(2.1) — Stage 1: Determine a conversion specifier.

(2.2) — Stage 2: Extract characters from `in` and determine a corresponding `char` value for the format expected by the conversion specification determined in stage 1.

(2.3) — Stage 3: Store results.

3 The details of the stages are presented below.

**Stage 1:** The function initializes local variables via

```

fmtflags flags = str.flags();
fmtflags basefield = (flags & ios_base::basefield);
fmtflags uppercase = (flags & ios_base::uppercase);
fmtflags boolalpha = (flags & ios_base::boolalpha);

```

For conversion to an integral type, the function determines the integral conversion specifier as indicated in [Table 95](#). The table is ordered. That is, the first line whose condition is true applies.

**Table 95 — Integer conversions** [tab:facet.num.get.int]

| State                         | stdio equivalent |
|-------------------------------|------------------|
| <code>basefield == oct</code> | <code>%o</code>  |
| <code>basefield == hex</code> | <code>%X</code>  |
| <code>basefield == 0</code>   | <code>%i</code>  |
| signed integral type          | <code>%d</code>  |
| unsigned integral type        | <code>%u</code>  |

For conversions to a floating-point type the specifier is `%g`.

For conversions to `void*` the specifier is `%p`.

A length modifier is added to the conversion specification, if needed, as indicated in [Table 96](#).

**Table 96 — Length modifier** [tab:facet.num.get.length]

| Type                            | Length modifier |
|---------------------------------|-----------------|
| <code>short</code>              | <code>h</code>  |
| <code>unsigned short</code>     | <code>h</code>  |
| <code>long</code>               | <code>l</code>  |
| <code>unsigned long</code>      | <code>l</code>  |
| <code>long long</code>          | <code>ll</code> |
| <code>unsigned long long</code> | <code>ll</code> |
| <code>double</code>             | <code>l</code>  |
| <code>long double</code>        | <code>L</code>  |

**Stage 2:** If `in == end` then stage 2 terminates. Otherwise a `charT` is taken from `in` and local variables are initialized as if by

```

char_type ct = *in;
char c = src[find(atoms, atoms + sizeof(src) - 1, ct) - atoms];
if (ct == use_facet<numput<charT>>(loc).decimal_point())
 c = '.';
bool discard =
 ct == use_facet<numput<charT>>(loc).thousands_sep()
 && use_facet<numput<charT>>(loc).grouping().length() != 0;

```

where the values `src` and `atoms` are defined as if by:

```
static const char src[] = "0123456789abcdefpxABCDEFPX+-";
char_type atoms[sizeof(src)];
use_facet<ctype<charT>>(loc).widen(src, src + sizeof(src), atoms);
```

for this value of `loc`.

If `discard` is `true`, then if `'.'` has not yet been accumulated, then the position of the character is remembered, but the character is otherwise ignored. Otherwise, if `'.'` has already been accumulated, the character is discarded and Stage 2 terminates. If it is not discarded, then a check is made to determine if `c` is allowed as the next character of an input field of the conversion specifier returned by Stage 1. If so, it is accumulated.

If the character is either discarded or accumulated then `in` is advanced by `++in` and processing returns to the beginning of stage 2.

[*Example 1:* Given an input sequence of `"0x1a.bp+07p"`,

- (3.1) — if the conversion specifier returned by Stage 1 is `%d`, `"0"` is accumulated;
- (3.2) — if the conversion specifier returned by Stage 1 is `%i`, `"0x1a"` are accumulated;
- (3.3) — if the conversion specifier returned by Stage 1 is `%g`, `"0x1a.bp+07"` are accumulated.

In all cases, the remainder is left in the input. — *end example*]

**Stage 3:** The sequence of `chars` accumulated in stage 2 (the field) is converted to a numeric value by the rules of one of the functions declared in the header `<cstdlib>` (17.2.2):

- (3.4) — For a signed integer value, the function `strtoll`.
- (3.5) — For an unsigned integer value, the function `strtoull`.
- (3.6) — For a `float` value, the function `strtof`.
- (3.7) — For a `double` value, the function `strtod`.
- (3.8) — For a `long double` value, the function `strtold`.

The numeric value to be stored can be one of:

- (3.9) — zero, if the conversion function does not convert the entire field.
- (3.10) — the most positive (or negative) representable value, if the field to be converted to a signed integer type represents a value too large positive (or negative) to be represented in `val`.
- (3.11) — the most positive representable value, if the field to be converted to an unsigned integer type represents a value that cannot be represented in `val`.
- (3.12) — the converted value, otherwise.

The resultant numeric value is stored in `val`. If the conversion function does not convert the entire field, or if the field represents a value outside the range of representable values, `ios_base::failbit` is assigned to `err`.

- 4 Digit grouping is checked. That is, the positions of discarded separators are examined for consistency with `use_facet<num_punct<charT>>(loc).grouping()`. If they are not consistent then `ios_base::failbit` is assigned to `err`.
- 5 In any case, if stage 2 processing was terminated by the test for `in == end` then `err |= ios_base::eofbit` is performed.

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
ios_base::iostate& err, bool& val) const;
```

- 6 *Effects:* If `(str.flags() & ios_base::boolalpha) == 0` then input proceeds as it would for a `long` except that if a value is being stored into `val`, the value is determined according to the following: If the value to be stored is 0 then `false` is stored. If the value is 1 then `true` is stored. Otherwise `true` is stored and `ios_base::failbit` is assigned to `err`.
- 7 Otherwise target sequences are determined “as if” by calling the members `false_name()` and `true_name()` of the facet obtained by `use_facet<num_punct<charT>>(str.getloc())`. Successive characters in the range `[in, end)` (see 23.2.4) are obtained and matched against corresponding positions in the target sequences only as necessary to identify a unique match. The input iterator `in` is compared to `end` only when necessary to obtain a character. If a target sequence is uniquely matched, `val` is set to the corresponding value. Otherwise `false` is stored and `ios_base::failbit` is assigned to `err`.
- 8 The `in` iterator is always left pointing one position beyond the last character successfully matched. If `val` is set, then `err` is set to `str.goodbit`; or to `str.eofbit` if, when seeking another character to match, it is found that `(in == end)`. If `val` is not set, then `err` is set to `str.failbit`; or to `(str.failbit | str.eofbit)` if the reason for the failure was that `(in == end)`.

[*Example 2:* For targets `true`: "a" and `false`: "abb", the input sequence "a" yields `val == true` and `err == str.eofbit`; the input sequence "abc" yields `err = str.failbit`, with in ending at the 'c' element. For targets `true`: "1" and `false`: "0", the input sequence "1" yields `val == true` and `err == str.goodbit`. For empty targets (""), any input sequence yields `err == str.failbit`. — *end example*]

9 *Returns:* in.

### 28.3.4.3.3 Class template `num_put`

[`locale.num.put`]

#### 28.3.4.3.3.1 General

[`locale.num.put.general`]

```
namespace std {
 template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class num_put : public locale::facet {
 public:
 using char_type = charT;
 using iter_type = OutputIterator;

 explicit num_put(size_t refs = 0);

 iter_type put(iter_type s, ios_base& f, char_type fill, bool v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, long v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, long long v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, unsigned long v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, unsigned long long v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, double v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, long double v) const;
 iter_type put(iter_type s, ios_base& f, char_type fill, const void* v) const;

 static locale::id id;

 protected:
 ~num_put();
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, bool v) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, long v) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, long long v) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, unsigned long) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, unsigned long long) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, double v) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, long double v) const;
 virtual iter_type do_put(iter_type, ios_base&, char_type fill, const void* v) const;
 };
}
```

<sup>1</sup> The facet `num_put` is used to format numeric values to a character sequence such as an ostream.

#### 28.3.4.3.3.2 Members

[`facet.num.put.members`]

```
iter_type put(iter_type out, ios_base& str, char_type fill, bool val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, long long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, unsigned long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, unsigned long long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, double val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, long double val) const;
iter_type put(iter_type out, ios_base& str, char_type fill, const void* val) const;
```

<sup>1</sup> *Returns:* `do_put(out, str, fill, val)`.

#### 28.3.4.3.3.3 Virtual functions

[`facet.num.put.virtuals`]

```
iter_type do_put(iter_type out, ios_base& str, char_type fill, long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill, long long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill, unsigned long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill, unsigned long long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill, double val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill, long double val) const;
```

```
iter_type do_put(iter_type out, ios_base& str, char_type fill, const void* val) const;
```

1 *Effects:* Writes characters to the sequence `out`, formatting `val` as desired. In the following description, `loc` names a local variable initialized as

```
locale loc = str.getloc();
```

2 The details of this operation occur in several stages:

(2.1) — Stage 1: Determine a printf conversion specifier `spec` and determine the characters that would be printed by `printf` (31.13) given this conversion specifier for

```
printf(spec, val)
```

assuming that the current locale is the "C" locale.

(2.2) — Stage 2: Adjust the representation by converting each `char` determined by stage 1 to a `charT` using a conversion and values returned by members of `use_facet<numput<charT>>(loc)`.

(2.3) — Stage 3: Determine where padding is required.

(2.4) — Stage 4: Insert the sequence into the `out`.

3 Detailed descriptions of each stage follow.

4 *Returns:* `out`.

**Stage 1:** The first action of stage 1 is to determine a conversion specifier. The tables that describe this determination use the following local variables

```
fmtflags flags = str.flags();
fmtflags basefield = (flags & (ios_base::basefield));
fmtflags uppercase = (flags & (ios_base::uppercase));
fmtflags floatfield = (flags & (ios_base::floatfield));
fmtflags showpos = (flags & (ios_base::showpos));
fmtflags showbase = (flags & (ios_base::showbase));
fmtflags showpoint = (flags & (ios_base::showpoint));
```

All tables used in describing stage 1 are ordered. That is, the first line whose condition is true applies. A line without a condition is the default behavior when none of the earlier lines apply.

For conversion from an integral type other than a character type, the function determines the integral conversion specifier as indicated in Table 97.

**Table 97 — Integer conversions** [tab:facet.num.put.int]

| State                                                           | stdio equivalent |
|-----------------------------------------------------------------|------------------|
| <code>basefield == ios_base::oct</code>                         | <code>%o</code>  |
| <code>(basefield == ios_base::hex) &amp;&amp; !uppercase</code> | <code>%x</code>  |
| <code>(basefield == ios_base::hex)</code>                       | <code>%X</code>  |
| for a signed integral type                                      | <code>%d</code>  |
| for an unsigned integral type                                   | <code>%u</code>  |

For conversion from a floating-point type, the function determines the floating-point conversion specifier as indicated in Table 98.

**Table 98 — Floating-point conversions** [tab:facet.num.put.fp]

| State                                                                                     | stdio equivalent |
|-------------------------------------------------------------------------------------------|------------------|
| <code>floatfield == ios_base::fixed &amp;&amp; !uppercase</code>                          | <code>%f</code>  |
| <code>floatfield == ios_base::fixed</code>                                                | <code>%F</code>  |
| <code>floatfield == ios_base::scientific &amp;&amp; !uppercase</code>                     | <code>%e</code>  |
| <code>floatfield == ios_base::scientific</code>                                           | <code>%E</code>  |
| <code>floatfield == (ios_base::fixed   ios_base::scientific) &amp;&amp; !uppercase</code> | <code>%a</code>  |
| <code>floatfield == (ios_base::fixed   ios_base::scientific)</code>                       | <code>%A</code>  |
| <code>!uppercase</code>                                                                   | <code>%g</code>  |
| <i>otherwise</i>                                                                          | <code>%G</code>  |

For conversions from an integral or floating-point type a length modifier is added to the conversion specifier as indicated in [Table 99](#).

**Table 99 — Length modifier** [tab:facet.num.put.length]

| Type               | Length modifier |
|--------------------|-----------------|
| long               | l               |
| long long          | ll              |
| unsigned long      | l               |
| unsigned long long | ll              |
| long double        | L               |
| <i>otherwise</i>   | <i>none</i>     |

The conversion specifier has the following optional additional qualifiers prepended as indicated in [Table 100](#).

**Table 100 — Numeric conversions** [tab:facet.num.put.conv]

| Type(s)               | State     | stdio equivalent |
|-----------------------|-----------|------------------|
| an integral type      | showpos   | +                |
|                       | showbase  | #                |
| a floating-point type | showpos   | +                |
|                       | showpoint | #                |

For conversion from a floating-point type, if `floatfield != (ios_base::fixed | ios_base::scientific)`, `str.precision()` is specified as precision in the conversion specification. Otherwise, no precision is specified.

For conversion from `void*` the specifier is `%p`.

The representations at the end of stage 1 consists of the `char`'s that would be printed by a call of `printf(s, val)` where `s` is the conversion specifier determined above.

**Stage 2:** Any character `c` other than a decimal point(`.`) is converted to a `charT` via

```
use_facet<ctype<charT>>(loc).widen(c)
```

A local variable `punct` is initialized via

```
const numpunct<charT>& punct = use_facet<numpunct<charT>>(loc);
```

For arithmetic types, `punct.thousands_sep()` characters are inserted into the sequence as determined by the value returned by `punct.do_grouping()` using the method described in [28.3.4.4.1.3](#).

Decimal point characters(`.`) are replaced by `punct.decimal_point()`.

**Stage 3:** A local variable is initialized as

```
fmtflags adjustfield = (flags & (ios_base::adjustfield));
```

The location of any padding<sup>203</sup> is determined according to [Table 101](#).

**Table 101 — Fill padding** [tab:facet.num.put.fill]

| State                                                                                                               | Location                                   |
|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| <code>adjustfield == ios_base::left</code>                                                                          | pad after                                  |
| <code>adjustfield == ios_base::right</code>                                                                         | pad before                                 |
| <code>adjustfield == internal</code> and a sign occurs in the representation                                        | pad after the sign                         |
| <code>adjustfield == internal</code> and representation after stage 1 began with <code>0x</code> or <code>0X</code> | pad after <code>x</code> or <code>X</code> |
| <i>otherwise</i>                                                                                                    | pad before                                 |

If `str.width()` is nonzero and the number of `charT`'s in the sequence after stage 2 is less than `str.width()`, then enough `fill` characters are added to the sequence at the position indicated for padding to bring the length of the sequence to `str.width()`.

<sup>203</sup>) The conversion specification `#o` generates a leading 0 which is *not* a padding character.



`str.width(0)` is called.

**Stage 4:** The sequence of `charT`'s at the end of stage 3 are output via

```
*out++ = c
```

```
iter_type do_put(iter_type out, ios_base& str, char_type fill, bool val) const;
```

5 *Returns:* If `(str.flags() & ios_base::boolalpha) == 0` returns `do_put(out, str, fill, (int)val)`, otherwise obtains a string `s` as if by

```
string_type s =
 val ? use_facet<numpunct<charT>>(loc).truename()
 : use_facet<numpunct<charT>>(loc).falsename();
```

and then inserts each character `c` of `s` into `out` via `*out++ = c` and returns `out`.

#### 28.3.4.4 The numeric punctuation facet

[facet.numpunct]

##### 28.3.4.4.1 Class template numpunct

[locale.numpunct]

##### 28.3.4.4.1.1 General

[locale.numpunct.general]

```
namespace std {
 template<class charT>
 class numpunct : public locale::facet {
 public:
 using char_type = charT;
 using string_type = basic_string<charT>;

 explicit numpunct(size_t refs = 0);

 char_type decimal_point() const;
 char_type thousands_sep() const;
 string grouping() const;
 string_type truename() const;
 string_type falsename() const;

 static locale::id id;

 protected:
 ~numpunct(); // virtual
 virtual char_type do_decimal_point() const;
 virtual char_type do_thousands_sep() const;
 virtual string do_grouping() const;
 virtual string_type do_truename() const; // for bool
 virtual string_type do_falsename() const; // for bool
 };
}
```

<sup>1</sup> `numpunct<>` specifies numeric punctuation. The specializations required in Table 91 (28.3.3.1.2.1), namely `numpunct<wchar_t>` and `numpunct<char>`, provide classic "C" numeric formats, i.e., they contain information equivalent to that contained in the "C" locale or their wide character counterparts as if obtained by a call to `widen`.

<sup>2</sup> The syntax for number formats is as follows, where *digit* represents the radix set specified by the `fmtflags` argument value, and *thousands-sep* and *decimal-point* are the results of corresponding `numpunct<charT>` members. Integer values have the format:

```
intval:
 signopt units

sign:
 +
 -

units:
 digits
 digits thousands-sep units

digits:
 digit digitsopt
```

and floating-point values have:

```
floatval:
 signopt units fractionalopt exponentopt
 signopt decimal-point digits exponentopt

fractional:
 decimal-point digitsopt

exponent:
 e signopt digits

e:
 e
 E
```

where the number of digits between *thousands-seps* is as specified by `do_grouping()`. For parsing, if the *digits* portion contains no thousands-separators, no grouping constraint is applied.

#### 28.3.4.4.1.2 Members [facet.numpunct.members]

```
char_type decimal_point() const;
1 Returns: do_decimal_point().

char_type thousands_sep() const;
2 Returns: do_thousands_sep().

string grouping() const;
3 Returns: do_grouping().

string_type truename() const;
string_type falsename() const;
4 Returns: do_truename() or do_falsename(), respectively.
```

#### 28.3.4.4.1.3 Virtual functions [facet.numpunct.virtuals]

```
char_type do_decimal_point() const;
1 Returns: A character for use as the decimal radix separator. The required specializations return '.' or
 L'.'.

char_type do_thousands_sep() const;
2 Returns: A character for use as the digit group separator. The required specializations return ',' or
 L','.

string do_grouping() const;
3 Returns: A string vec used as a vector of integer values, in which each element vec[i] represents
 the number of digits204 in the group at position i, starting with position 0 as the rightmost group. If
 vec.size() <= i, the number is the same as group (i - 1); if (i < 0 || vec[i] <= 0 || vec[i]
 == CHAR_MAX), the size of the digit group is unlimited.

4 The required specializations return the empty string, indicating no grouping.

string_type do_truename() const;
string_type do_falsename() const;
5 Returns: A string representing the name of the boolean value true or false, respectively.
6 In the base class implementation these names are "true" and "false", or L"true" and L"false".
```

#### 28.3.4.4.2 Class template `numpunct_byname` [locale.numpunct.byname]

```
namespace std {
 template<class charT>
 class numpunct_byname : public numpunct<charT> {
 // this class is specialized for char and wchar_t.
```

<sup>204</sup>) Thus, the string "\003" specifies groups of 3 digits each, and "3" probably indicates groups of 51 (!) digits each, because 51 is the ASCII value of "3".

```

public:
 using char_type = charT;
 using string_type = basic_string<charT>;

 explicit numpunct_byname(const char*, size_t refs = 0);
 explicit numpunct_byname(const string&, size_t refs = 0);

protected:
 ~numpunct_byname();
};
}

```

#### 28.3.4.5 The collate category

[category.collate]

##### 28.3.4.5.1 Class template collate

[locale.collate]

##### 28.3.4.5.1.1 General

[locale.collate.general]

```

namespace std {
 template<class charT>
 class collate : public locale::facet {
 public:
 using char_type = charT;
 using string_type = basic_string<charT>;

 explicit collate(size_t refs = 0);

 int compare(const charT* low1, const charT* high1,
 const charT* low2, const charT* high2) const;
 string_type transform(const charT* low, const charT* high) const;
 long hash(const charT* low, const charT* high) const;

 static locale::id id;

 protected:
 ~collate();
 virtual int do_compare(const charT* low1, const charT* high1,
 const charT* low2, const charT* high2) const;
 virtual string_type do_transform(const charT* low, const charT* high) const;
 virtual long do_hash (const charT* low, const charT* high) const;
 };
}

```

<sup>1</sup> The class `collate<charT>` provides features for use in the collation (comparison) and hashing of strings. A locale member function template, `operator()`, uses the collate facet to allow a locale to act directly as the predicate argument for standard algorithms ([Clause 26](#)) and containers operating on strings. The specializations required in [Table 91 \(28.3.3.1.2.1\)](#), namely `collate<char>` and `collate<wchar_t>`, apply lexicographical ordering ([26.8.11](#)).

<sup>2</sup> Each function compares a string of characters `*p` in the range `[low,high)`.

##### 28.3.4.5.1.2 Members

[locale.collate.members]

```

int compare(const charT* low1, const charT* high1,
 const charT* low2, const charT* high2) const;

```

<sup>1</sup> *Returns:* `do_compare(low1, high1, low2, high2)`.

```

string_type transform(const charT* low, const charT* high) const;

```

<sup>2</sup> *Returns:* `do_transform(low, high)`.

```

long hash(const charT* low, const charT* high) const;

```

<sup>3</sup> *Returns:* `do_hash(low, high)`.

**28.3.4.5.1.3 Virtual functions****[locale.collate.virtuals]**

```
int do_compare(const charT* low1, const charT* high1,
 const charT* low2, const charT* high2) const;
```

- 1 *Returns:* 1 if the first string is greater than the second, -1 if less, zero otherwise. The specializations required in [Table 91](#) ([28.3.3.1.2.1](#)), namely `collate<char>` and `collate<wchar_t>`, implement a lexicographical comparison ([26.8.11](#)).

```
string_type do_transform(const charT* low, const charT* high) const;
```

- 2 *Returns:* A `basic_string<charT>` value that, compared lexicographically with the result of calling `transform()` on another string, yields the same result as calling `do_compare()` on the same two strings.<sup>205</sup>

```
long do_hash(const charT* low, const charT* high) const;
```

- 3 *Returns:* An integer value equal to the result of calling `hash()` on any other string for which `do_compare()` returns 0 (equal) when passed the two strings.
- 4 *Recommended practice:* The probability that the result equals that for another string which does not compare equal should be very small, approaching  $(1.0/\text{numeric\_limits}<\text{unsigned long}>::\text{max}())$ .

**28.3.4.5.2 Class template `collate_byname`****[locale.collate.byname]**

```
namespace std {
 template<class charT>
 class collate_byname : public collate<charT> {
 public:
 using string_type = basic_string<charT>;

 explicit collate_byname(const char*, size_t refs = 0);
 explicit collate_byname(const string&, size_t refs = 0);

 protected:
 ~collate_byname();
 };
}
```

**28.3.4.6 The time category****[category.time]****28.3.4.6.1 General****[category.time.general]**

- 1 Templates `time_get<charT, InputIterator>` and `time_put<charT, OutputIterator>` provide date and time formatting and parsing. All specifications of member functions for `time_put` and `time_get` in the subclauses of [28.3.4.6](#) only apply to the specializations required in [Tables 91](#) and [92](#) ([28.3.3.1.2.1](#)). Their members use their `ios_base&`, `ios_base::iostate&`, and fill arguments as described in [28.3.4](#), and the `ctype<>` facet, to determine formatting details.

**28.3.4.6.2 Class template `time_get`****[locale.time.get]****28.3.4.6.2.1 General****[locale.time.get.general]**

```
namespace std {
 class time_base {
 public:
 enum dateorder { no_order, dmy, mdy, ymd, ydm };
 };

 template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class time_get : public locale::facet, public time_base {
 public:
 using char_type = charT;
 using iter_type = InputIterator;

 explicit time_get(size_t refs = 0);
```

<sup>205</sup>) This function is useful when one string is being compared to many other strings.

```

dateorder date_order() const { return do_date_order(); }
iter_type get_time(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t) const;
iter_type get_date(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t) const;
iter_type get_weekday(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t) const;
iter_type get_monthname(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t) const;
iter_type get_year(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t) const;
iter_type get(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t, char format, char modifier = 0) const;
iter_type get(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t, const char_type* fmt,
 const char_type* fmtend) const;

static locale::id id;

protected:
 ~time_get();
 virtual dateorder do_date_order() const;
 virtual iter_type do_get_time(iter_type s, iter_type end, ios_base&,
 ios_base::iostate& err, tm* t) const;
 virtual iter_type do_get_date(iter_type s, iter_type end, ios_base&,
 ios_base::iostate& err, tm* t) const;
 virtual iter_type do_get_weekday(iter_type s, iter_type end, ios_base&,
 ios_base::iostate& err, tm* t) const;
 virtual iter_type do_get_monthname(iter_type s, iter_type end, ios_base&,
 ios_base::iostate& err, tm* t) const;
 virtual iter_type do_get_year(iter_type s, iter_type end, ios_base&,
 ios_base::iostate& err, tm* t) const;
 virtual iter_type do_get(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t, char format, char modifier) const;
};
}

```

- <sup>1</sup> `time_get` is used to parse a character sequence, extracting components of a time or date into a `tm` object. Each `get` member parses a format as produced by a corresponding format specifier to `time_put<>::put`. If the sequence being parsed matches the correct format, the corresponding members of the `tm` argument are set to the values used to produce the sequence; otherwise either an error is reported or unspecified values are assigned.<sup>206</sup>
- <sup>2</sup> If the end iterator is reached during parsing by any of the `get()` member functions, the member sets `ios_base::eofbit` in `err`.

#### 28.3.4.6.2.2 Members

[`locale.time.get.members`]

```
dateorder date_order() const;
```

- <sup>1</sup> *Returns:* `do_date_order()`.

```
iter_type get_time(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

- <sup>2</sup> *Returns:* `do_get_time(s, end, str, err, t)`.

```
iter_type get_date(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

- <sup>3</sup> *Returns:* `do_get_date(s, end, str, err, t)`.

<sup>206</sup>) In other words, user confirmation is required for reliable parsing of user-entered dates and times, but machine-generated formats can be parsed reliably. This allows parsers to be aggressive about interpreting user variations on standard formats.

```

iter_type get_weekday(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
iter_type get_monthname(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;

```

4     *Returns:* `do_get_weekday(s, end, str, err, t)` or `do_get_monthname(s, end, str, err, t)`.

```

iter_type get_year(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;

```

5     *Returns:* `do_get_year(s, end, str, err, t)`.

```

iter_type get(iter_type s, iter_type end, ios_base& f, ios_base::iostate& err,
 tm* t, char format, char modifier = 0) const;

```

6     *Returns:* `do_get(s, end, f, err, t, format, modifier)`.

```

iter_type get(iter_type s, iter_type end, ios_base& f, ios_base::iostate& err,
 tm* t, const char_type* fmt, const char_type* fmtend) const;

```

7     *Preconditions:* `[fmt,fmtend)` is a valid range.

8     *Effects:* The function starts by evaluating `err = ios_base::goodbit`. It then enters a loop, reading zero or more characters from `s` at each iteration. Unless otherwise specified below, the loop terminates when the first of the following conditions holds:

- (8.1)     — The expression `fmt == fmtend` evaluates to `true`.
- (8.2)     — The expression `err == ios_base::goodbit` evaluates to `false`.
- (8.3)     — The expression `s == end` evaluates to `true`, in which case the function evaluates `err = ios_base::eofbit | ios_base::failbit`.
- (8.4)     — The next element of `fmt` is equal to `'%'`, optionally followed by a modifier character, followed by a conversion specifier character, `format`, together forming a conversion specification valid for the POSIX function `strptime`. If the number of elements in the range `[fmt,fmtend)` is not sufficient to unambiguously determine whether the conversion specification is complete and valid, the function evaluates `err = ios_base::failbit`. Otherwise, the function evaluates `s = do_get(s, end, f, err, t, format, modifier)`, where the value of `modifier` is `'\0'` when the optional modifier is absent from the conversion specification. If `err == ios_base::goodbit` holds after the evaluation of the expression, the function increments `fmt` to point just past the end of the conversion specification and continues looping.
- (8.5)     — The expression `isspace(*fmt, f.getloc())` evaluates to `true`, in which case the function first increments `fmt` until `fmt == fmtend || !isspace(*fmt, f.getloc())` evaluates to `true`, then advances `s` until `s == end || !isspace(*s, f.getloc())` is `true`, and finally resumes looping.
- (8.6)     — The next character read from `s` matches the element pointed to by `fmt` in a case-insensitive comparison, in which case the function evaluates `++fmt`, `++s` and continues looping. Otherwise, the function evaluates `err = ios_base::failbit`.

9     [*Note 1:* The function uses the `ctype<charT>` facet installed in `f`'s locale to determine valid whitespace characters. It is unspecified by what means the function performs case-insensitive comparison or whether multi-character sequences are considered while doing so. — *end note*]

10    *Returns:* `s`.

### 28.3.4.6.2.3 Virtual functions

[`locale.time.get.virtuals`]

```

dateorder do_date_order() const;

```

1     *Returns:* An enumeration value indicating the preferred order of components for those date formats that are composed of day, month, and year.<sup>207</sup> Returns `no_order` if the date format specified by `'x'` contains other variable components (e.g., Julian day, week number, week day).

<sup>207</sup>) This function is intended as a convenience only, for common formats, and can return `no_order` in valid locales.

```
iter_type do_get_time(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

2 *Effects:* Reads characters starting at `s` until it has extracted those `tm` members, and remaining format characters, used by `time_put<>::put` to produce the format specified by `"%H:%M:%S"`, or until it encounters an error or end of sequence.

3 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid time.

```
iter_type do_get_date(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

4 *Effects:* Reads characters starting at `s` until it has extracted those `tm` members and remaining format characters used by `time_put<>::put` to produce one of the following formats, or until it encounters an error. The format depends on the value returned by `date_order()` as shown in [Table 102](#).

Table 102 — `do_get_date` effects [tab:locale.time.get.dogetdate]

| date_order() | Format   |
|--------------|----------|
| no_order     | "%m%d%y" |
| dmy          | "%d%m%y" |
| mdy          | "%m%d%y" |
| ymd          | "%y%m%d" |
| ydm          | "%y%d%m" |

5 An implementation may also accept additional implementation-defined formats.

6 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid date.

```
iter_type do_get_weekday(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
iter_type do_get_monthname(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

7 *Effects:* Reads characters starting at `s` until it has extracted the (perhaps abbreviated) name of a weekday or month. If it finds an abbreviation that is followed by characters that can match a full name, it continues reading until it matches the full name or fails. It sets the appropriate `tm` member accordingly.

8 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid name.

```
iter_type do_get_year(iter_type s, iter_type end, ios_base& str,
 ios_base::iostate& err, tm* t) const;
```

9 *Effects:* Reads characters starting at `s` until it has extracted an unambiguous year identifier. It is implementation-defined whether two-digit year numbers are accepted, and (if so) what century they are assumed to lie in. Sets the `t->tm_year` member accordingly.

10 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid year identifier.

```
iter_type do_get(iter_type s, iter_type end, ios_base& f,
 ios_base::iostate& err, tm* t, char format, char modifier) const;
```

11 *Preconditions:* `t` points to an object.

12 *Effects:* The function starts by evaluating `err = ios_base::goodbit`. It then reads characters starting at `s` until it encounters an error, or until it has extracted and assigned those `tm` members, and any remaining format characters, corresponding to a conversion specification appropriate for the POSIX function `strptime`, formed by concatenating `'%'`, the `modifier` character, when non-NUL, and the `format` character. When the concatenation fails to yield a complete valid directive the function leaves the object pointed to by `t` unchanged and evaluates `err != ios_base::failbit`. When `s == end` evaluates to `true` after reading a character the function evaluates `err != ios_base::eofbit`.

13 For complex conversion specifications such as `%c`, `%x`, or `%X`, or conversion specifications that involve the optional modifiers `E` or `O`, when the function is unable to unambiguously determine some or all `tm` members from the input sequence `[s,end)`, it evaluates `err |= ios_base::eofbit`. In such cases the values of those `tm` members are unspecified and may be outside their valid range.

14 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid input sequence for the given `format` and `modifier`.

15 *Remarks:* It is unspecified whether multiple calls to `do_get()` with the address of the same `tm` object will update the current contents of the object or simply overwrite its members. Portable programs should zero out the object before invoking the function.

#### 28.3.4.6.3 Class template `time_get_byname`

[`locale.time.get.byname`]

```
namespace std {
 template<class charT, class InputIterator = istreambuf_iterator<charT>>
 class time_get_byname : public time_get<charT, InputIterator> {
 public:
 using dateorder = time_base::dateorder;
 using iter_type = InputIterator;

 explicit time_get_byname(const char*, size_t refs = 0);
 explicit time_get_byname(const string&, size_t refs = 0);

 protected:
 ~time_get_byname();
 };
}
```

#### 28.3.4.6.4 Class template `time_put`

[`locale.time.put`]

##### 28.3.4.6.4.1 General

[`locale.time.put.general`]

```
namespace std {
 template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class time_put : public locale::facet {
 public:
 using char_type = charT;
 using iter_type = OutputIterator;

 explicit time_put(size_t refs = 0);

 // the following is implemented in terms of other member functions.
 iter_type put(iter_type s, ios_base& f, char_type fill, const tm* tmb,
 const charT* pattern, const charT* pat_end) const;
 iter_type put(iter_type s, ios_base& f, char_type fill,
 const tm* tmb, char format, char modifier = 0) const;

 static locale::id id;

 protected:
 ~time_put();
 virtual iter_type do_put(iter_type s, ios_base&, char_type, const tm* t,
 char format, char modifier) const;
 };
}
```

##### 28.3.4.6.4.2 Members

[`locale.time.put.members`]

```
iter_type put(iter_type s, ios_base& str, char_type fill, const tm* t,
 const charT* pattern, const charT* pat_end) const;
iter_type put(iter_type s, ios_base& str, char_type fill, const tm* t,
 char format, char modifier = 0) const;
```

1 *Effects:* The first form steps through the sequence from `pattern` to `pat_end`, identifying characters that are part of a format sequence. Each character that is not part of a format sequence is written to `s` immediately, and each format sequence, as it is identified, results in a call to `do_put`; thus, format



elements and other characters are interleaved in the output in the order in which they appear in the pattern. Format sequences are identified by converting each character `c` to a `char` value as if by `ct.narrow(c, 0)`, where `ct` is a reference to `ctype<charT>` obtained from `str.getloc()`. The first character of each sequence is equal to `'%'`, followed by an optional modifier character `mod` and a format specifier character `spec` as defined for the function `strftime`. If no modifier character is present, `mod` is zero. For each valid format sequence identified, calls `do_put(s, str, fill, t, spec, mod)`.

The second form calls `do_put(s, str, fill, t, format, modifier)`.

[Note 1: The `fill` argument can be used in the implementation-defined formats or by derivations. A space character is a reasonable default for this argument. — end note]

Returns: An iterator pointing immediately after the last character produced.

#### 28.3.4.6.4.3 Virtual functions

[locale.time.put.virtuals]

```
iter_type do_put(iter_type s, ios_base&, char_type fill, const tm* t,
 char format, char modifier) const;
```

Effects: Formats the contents of the parameter `t` into characters placed on the output sequence `s`. Formatting is controlled by the parameters `format` and `modifier`, interpreted identically as the format specifiers in the string argument to the standard library function `strftime()`, except that the sequence of characters produced for those specifiers that are described as depending on the C locale are instead implementation-defined.

[Note 1: Interpretation of the `modifier` argument is implementation-defined. — end note]

Returns: An iterator pointing immediately after the last character produced.

[Note 2: The `fill` argument can be used in the implementation-defined formats or by derivations. A space character is a reasonable default for this argument. — end note]

Recommended practice: Interpretation of the `modifier` should follow POSIX conventions. Implementations should refer to other standards such as POSIX for a specification of the character sequences produced for those specifiers described as depending on the C locale.

#### 28.3.4.6.5 Class template `time_put_byname`

[locale.time.put.byname]

```
namespace std {
 template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class time_put_byname : public time_put<charT, OutputIterator> {
 public:
 using char_type = charT;
 using iter_type = OutputIterator;

 explicit time_put_byname(const char*, size_t refs = 0);
 explicit time_put_byname(const string&, size_t refs = 0);

 protected:
 ~time_put_byname();
 };
}
```

#### 28.3.4.7 The monetary category

[category.monetary]

##### 28.3.4.7.1 General

[category.monetary.general]

These templates handle monetary formats. A template parameter indicates whether local or international monetary formats are to be used.

All specifications of member functions for `money_put` and `money_get` in the subclauses of 28.3.4.7 only apply to the specializations required in Tables 91 and 92 (28.3.3.1.2.1). Their members use their `ios_base&`, `ios_base::iostate&`, and `fill` arguments as described in 28.3.4, and the `money_punct<>` and `ctype<>` facets, to determine formatting details.

##### 28.3.4.7.2 Class template `money_get`

[locale.money.get]

###### 28.3.4.7.2.1 General

[locale.money.get.general]

```
namespace std {
 template<class charT, class InputIterator = istreambuf_iterator<charT>>
```

```

class money_get : public locale::facet {
public:
 using char_type = charT;
 using iter_type = InputIterator;
 using string_type = basic_string<charT>;

 explicit money_get(size_t refs = 0);

 iter_type get(iter_type s, iter_type end, bool intl,
 ios_base& f, ios_base::iostate& err,
 long double& units) const;
 iter_type get(iter_type s, iter_type end, bool intl,
 ios_base& f, ios_base::iostate& err,
 string_type& digits) const;

 static locale::id id;

protected:
 ~money_get();
 virtual iter_type do_get(iter_type, iter_type, bool, ios_base&,
 ios_base::iostate& err, long double& units) const;
 virtual iter_type do_get(iter_type, iter_type, bool, ios_base&,
 ios_base::iostate& err, string_type& digits) const;
};

```

#### 28.3.4.7.2.2 Members

[locale.money.get.members]

```

iter_type get(iter_type s, iter_type end, bool intl, ios_base& f,
 ios_base::iostate& err, long double& quant) const;
iter_type get(iter_type s, iter_type end, bool intl, ios_base& f,
 ios_base::iostate& err, string_type& quant) const;

```

<sup>1</sup> *Returns:* do\_get(s, end, intl, f, err, quant).

#### 28.3.4.7.2.3 Virtual functions

[locale.money.get.virtuals]

```

iter_type do_get(iter_type s, iter_type end, bool intl, ios_base& str,
 ios_base::iostate& err, long double& units) const;
iter_type do_get(iter_type s, iter_type end, bool intl, ios_base& str,
 ios_base::iostate& err, string_type& digits) const;

```

<sup>1</sup> *Effects:* Reads characters from **s** to parse and construct a monetary value according to the format specified by a `money_punct<charT, Intl>` facet reference **mp** and the character mapping specified by a `ctype<charT>` facet reference **ct** obtained from the locale returned by **str.getloc()**, and **str.flags()**. If a valid sequence is recognized, does not change **err**; otherwise, sets **err** to **(err | str.failbit)**, or **(err | str.failbit | str.eofbit)** if no more characters are available, and does not change **units** or **digits**. Uses the pattern returned by **mp.neg\_format()** to parse all values. The result is returned as an integral value stored in **units** or as a sequence of digits possibly preceded by a minus sign (as produced by **ct.widen(c)** where **c** is '-' or in the range from '0' through '9' (inclusive)) stored in **digits**.

[*Example 1:* The sequence \$1,056.23 in a common United States locale would yield, for **units**, 105623, or, for **digits**, "105623". — end example]

If **mp.grouping()** indicates that no thousands separators are permitted, any such characters are not read, and parsing is terminated at the point where they first appear. Otherwise, thousands separators are optional; if present, they are checked for correct placement only after all format components have been read.

<sup>2</sup> Where **money\_base::space** or **money\_base::none** appears as the last element in the format pattern, no whitespace is consumed. Otherwise, where **money\_base::space** appears in any of the initial elements of the format pattern, at least one whitespace character is required. Where **money\_base::none** appears in any of the initial elements of the format pattern, whitespace is allowed but not required. If **(str.flags() & str.showbase)** is **false**, the currency symbol is optional and is consumed only if other characters are needed to complete the format; otherwise, the currency symbol is required.

- 3 If the first character (if any) in the string `pos` returned by `mp.positive_sign()` or the string `neg` returned by `mp.negative_sign()` is recognized in the position indicated by `sign` in the format pattern, it is consumed and any remaining characters in the string are required after all the other format components.

[*Example 2:* If `showbase` is off, then for a `neg` value of "(" and a currency symbol of "L", in "(100 L)" the "L" is consumed; but if `neg` is "-", the "L" in "-100 L" is not consumed. — *end example*]

If `pos` or `neg` is empty, the sign component is optional, and if no sign is detected, the result is given the sign that corresponds to the source of the empty string. Otherwise, the character in the indicated position must match the first character of `pos` or `neg`, and the result is given the corresponding sign. If the first character of `pos` is equal to the first character of `neg`, or if both strings are empty, the result is given a positive sign.

- 4 Digits in the numeric monetary component are extracted and placed in `digits`, or into a character buffer `buf1` for conversion to produce a value for `units`, in the order in which they appear, preceded by a minus sign if and only if the result is negative. The value `units` is produced as if by<sup>208</sup>

```
for (int i = 0; i < n; ++i)
 buf2[i] = src[find(atoms, atoms + sizeof(src), buf1[i]) - atoms];
buf2[n] = 0;
sscanf(buf2, "%Lf", &units);
```

where `n` is the number of characters placed in `buf1`, `buf2` is a character buffer, and the values `src` and `atoms` are defined as if by

```
static const char src[] = "0123456789-";
charT atoms[sizeof(src)];
ct.widen(src, src + sizeof(src) - 1, atoms);
```

- 5 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid monetary quantity.

### 28.3.4.7.3 Class template `money_put`

[`locale.money.put`]

#### 28.3.4.7.3.1 General

[`locale.money.put.general`]

```
namespace std {
 template<class charT, class OutputIterator = ostreambuf_iterator<charT>>
 class money_put : public locale::facet {
 public:
 using char_type = charT;
 using iter_type = OutputIterator;
 using string_type = basic_string<charT>;

 explicit money_put(size_t refs = 0);

 iter_type put(iter_type s, bool intl, ios_base& f,
 char_type fill, long double units) const;
 iter_type put(iter_type s, bool intl, ios_base& f,
 char_type fill, const string_type& digits) const;

 static locale::id id;

 protected:
 ~money_put();
 virtual iter_type do_put(iter_type, bool, ios_base&, char_type fill,
 long double units) const;
 virtual iter_type do_put(iter_type, bool, ios_base&, char_type fill,
 const string_type& digits) const;
 };
}
```

#### 28.3.4.7.3.2 Members

[`locale.money.put.members`]

```
iter_type put(iter_type s, bool intl, ios_base& f, char_type fill, long double quant) const;
```

<sup>208</sup>) The semantics here are different from `ct.narrow`.

```
iter_type put(iter_type s, bool intl, ios_base& f, char_type fill, const string_type& quant) const;
```

1 *Returns:* `do_put(s, intl, f, fill, quant)`.

### 28.3.4.7.3 Virtual functions

[`locale.money.put.virtuals`]

```
iter_type do_put(iter_type s, bool intl, ios_base& str,
 char_type fill, long double units) const;
```

```
iter_type do_put(iter_type s, bool intl, ios_base& str,
 char_type fill, const string_type& digits) const;
```

1 *Effects:* Writes characters to `s` according to the format specified by a `money_punct<charT, Intl>` facet reference `mp` and the character mapping specified by a `ctype<charT>` facet reference `ct` obtained from the locale returned by `str.getloc()`, and `str.flags()`. The argument `units` is transformed into a sequence of wide characters as if by

```
ct.widen(buf1, buf1 + sprintf(buf1, "%.0Lf", units), buf2)
```

for character buffers `buf1` and `buf2`. If the first character in `digits` or `buf2` is equal to `ct.widen('‑')`, then the pattern used for formatting is the result of `mp.neg_format()`; otherwise the pattern is the result of `mp.pos_format()`. Digit characters are written, interspersed with any thousands separators and decimal point specified by the format, in the order they appear (after the optional leading minus sign) in `digits` or `buf2`. In `digits`, only the optional leading minus sign and the immediately subsequent digit characters (as classified according to `ct`) are used; any trailing characters (including digits appearing after a non-digit character) are ignored. Calls `str.width(0)`.

2 *Returns:* An iterator pointing immediately after the last character produced.

3 *Remarks:* The currency symbol is generated if and only if `(str.flags() & str.showbase)` is nonzero. If the number of characters generated for the specified format is less than the value returned by `str.width()` on entry to the function, then copies of `fill` are inserted as necessary to pad to the specified width. For the value `af` equal to `(str.flags() & str.adjustfield)`, if `(af == str.internal)` is true, the fill characters are placed where `none` or `space` appears in the formatting pattern; otherwise if `(af == str.left)` is true, they are placed after the other characters; otherwise, they are placed before the other characters.

[*Note 1:* It is possible, with some combinations of format patterns and flag values, to produce output that cannot be parsed using `num_get<>::get`. — end note]

### 28.3.4.7.4 Class template `money_punct`

[`locale.money_punct`]

#### 28.3.4.7.4.1 General

[`locale.money_punct.general`]

```
namespace std {
 class money_base {
 public:
 enum part { none, space, symbol, sign, value };
 struct pattern { char field[4]; };
 };

 template<class charT, bool International = false>
 class money_punct : public locale::facet, public money_base {
 public:
 using char_type = charT;
 using string_type = basic_string<charT>;

 explicit money_punct(size_t refs = 0);

 charT decimal_point() const;
 charT thousands_sep() const;
 string grouping() const;
 string_type curr_symbol() const;
 string_type positive_sign() const;
 string_type negative_sign() const;
 int frac_digits() const;
 pattern pos_format() const;
 pattern neg_format() const;
 };
}
```

```

static locale::id id;
static const bool intl = International;

protected:
 ~moneypunct();
 virtual charT do_decimal_point() const;
 virtual charT do_thousands_sep() const;
 virtual string do_grouping() const;
 virtual string_type do_curr_symbol() const;
 virtual string_type do_positive_sign() const;
 virtual string_type do_negative_sign() const;
 virtual int do_frac_digits() const;
 virtual pattern do_pos_format() const;
 virtual pattern do_neg_format() const;
};
}

```

- <sup>1</sup> The `moneypunct<>` facet defines monetary formatting parameters used by `money_get<>` and `money_put<>`. A monetary format is a sequence of four components, specified by a `pattern` value `p`, such that the `part` value `static_cast<part>(p.field[i])` determines the  $i^{\text{th}}$  component of the format.<sup>209</sup> In the `field` member of a `pattern` object, each value `symbol`, `sign`, `value`, and either `space` or `none` appears exactly once. The value `none`, if present, is not first; the value `space`, if present, is neither first nor last.
- <sup>2</sup> Where `none` or `space` appears, whitespace is permitted in the format, except where `none` appears at the end, in which case no whitespace is permitted. The value `space` indicates that at least one space is required at that position. Where `symbol` appears, the sequence of characters returned by `curr_symbol()` is permitted, and can be required. Where `sign` appears, the first (if any) of the sequence of characters returned by `positive_sign()` or `negative_sign()` (respectively as the monetary value is non-negative or negative) is required. Any remaining characters of the sign sequence are required after all other format components. Where `value` appears, the absolute numeric monetary value is required.
- <sup>3</sup> The format of the numeric monetary value is a decimal number:

```

value:
 units fractionalopt
 decimal-point digits

fractional:
 decimal-point digitsopt

```

if `frac_digits()` returns a positive value, or

```

value:
 units

```

otherwise. The symbol *decimal-point* indicates the character returned by `decimal_point()`. The other symbols are defined as follows:

```

units:
 digits
 digits thousands-sep units

digits:
 adigit digitsopt

```

In the syntax specification, the symbol *adigit* is any of the values `ct.widen(c)` for `c` in the range '0' through '9' (inclusive) and `ct` is a reference of type `const ctype<charT>&` obtained as described in the definitions of `money_get<>` and `money_put<>`. The symbol *thousands-sep* is the character returned by `thousands_sep()`. The space character used is the value `ct.widen(' ')`. Whitespace characters are those characters `c` for which `ci.is(space, c)` returns `true`. The number of digits required after the decimal point (if any) is exactly the value returned by `frac_digits()`.

- <sup>4</sup> The placement of thousands-separator characters (if any) is determined by the value returned by `grouping()`, defined identically as the member `numput<>::do_grouping()`.

#### 28.3.4.7.4.2 Members

[`locale.moneypunct.members`]

```
charT decimal_point() const;
```

<sup>209</sup> An array of `char`, rather than an array of `part`, is specified for `pattern::field` purely for efficiency.

```

charT thousands_sep() const;
string grouping() const;
string_type curr_symbol() const;
string_type positive_sign() const;
string_type negative_sign() const;
int frac_digits() const;
pattern pos_format() const;
pattern neg_format() const;

```

<sup>1</sup> Each of these functions *F* returns the result of calling the corresponding virtual member function `do_F()`.

#### 28.3.4.7.4.3 Virtual functions

[`locale.money_punct.virtuals`]

```
charT do_decimal_point() const;
```

<sup>1</sup> *Returns:* The radix separator to use in case `do_frac_digits()` is greater than zero.<sup>210</sup>

```
charT do_thousands_sep() const;
```

<sup>2</sup> *Returns:* The digit group separator to use in case `do_grouping()` specifies a digit grouping pattern.<sup>211</sup>

```
string do_grouping() const;
```

<sup>3</sup> *Returns:* A pattern defined identically as, but not necessarily equal to, the result of `num_punct<charT>::do_grouping()`.<sup>212</sup>

```
string_type do_curr_symbol() const;
```

<sup>4</sup> *Returns:* A string to use as the currency identifier symbol.

[*Note 1:* For specializations where the second template parameter is `true`, this is typically four characters long: a three-letter code as specified by ISO 4217<sup>[1]</sup> followed by a space. — *end note*]

```
string_type do_positive_sign() const;
```

```
string_type do_negative_sign() const;
```

<sup>5</sup> *Returns:* `do_positive_sign()` returns the string to use to indicate a positive monetary value;<sup>213</sup> `do_negative_sign()` returns the string to use to indicate a negative value.

```
int do_frac_digits() const;
```

<sup>6</sup> *Returns:* The number of digits after the decimal radix separator, if any.<sup>214</sup>

```
pattern do_pos_format() const;
```

```
pattern do_neg_format() const;
```

<sup>7</sup> *Returns:* The specializations required in Table 92 (28.3.3.1.2.1), namely

(7.1) — `money_punct<char>`,

(7.2) — `money_punct<wchar_t>`,

(7.3) — `money_punct<char, true>`, and

(7.4) — `money_punct<wchar_t, true>`,

return an object of type `pattern` initialized to { `symbol`, `sign`, `none`, `value` }.<sup>215</sup>

#### 28.3.4.7.5 Class template `money_punct_byname`

[`locale.money_punct.byname`]

```

namespace std {
 template<class charT, bool Intl = false>
 class money_punct_byname : public money_punct<charT, Intl> {
 public:
 using pattern = money_base::pattern;
 using string_type = basic_string<charT>;

```

<sup>210</sup>) In common U.S. locales this is `'.'`.

<sup>211</sup>) In common U.S. locales this is `','`.

<sup>212</sup>) To specify grouping by 3s, the value is `"\003"` *not* `"3"`.

<sup>213</sup>) This is usually the empty string.

<sup>214</sup>) In common U.S. locales, this is 2.

<sup>215</sup>) Note that the international symbol returned by `do_curr_symbol()` usually contains a space, itself; for example, `"USD "`.

```

 explicit moneypunct_byname(const char*, size_t refs = 0);
 explicit moneypunct_byname(const string&, size_t refs = 0);

protected:
 ~moneypunct_byname();
};
}

```

#### 28.3.4.8 The message retrieval category

[category.messages]

##### 28.3.4.8.1 General

[category.messages.general]

- <sup>1</sup> Class `messages<charT>` implements retrieval of strings from message catalogs.

##### 28.3.4.8.2 Class template messages

[locale.messages]

##### 28.3.4.8.2.1 General

[locale.messages.general]

```

namespace std {
 class messages_base {
 public:
 using catalog = unspecified signed integer type;
 };

 template<class charT>
 class messages : public locale::facet, public messages_base {
 public:
 using char_type = charT;
 using string_type = basic_string<charT>;

 explicit messages(size_t refs = 0);

 catalog open(const string& fn, const locale&) const;
 string_type get(catalog c, int set, int msgid,
 const string_type& dfault) const;
 void close(catalog c) const;

 static locale::id id;

 protected:
 ~messages();
 virtual catalog do_open(const string&, const locale&) const;
 virtual string_type do_get(catalog, int set, int msgid,
 const string_type& dfault) const;
 virtual void do_close(catalog) const;
 };
}

```

- <sup>1</sup> Values of type `messages_base::catalog` usable as arguments to members `get` and `close` can be obtained only by calling member `open`.

##### 28.3.4.8.2.2 Members

[locale.messages.members]

```
catalog open(const string& name, const locale& loc) const;
```

- <sup>1</sup> *Returns:* `do_open(name, loc)`.

```
string_type get(catalog cat, int set, int msgid, const string_type& dfault) const;
```

- <sup>2</sup> *Returns:* `do_get(cat, set, msgid, dfault)`.

```
void close(catalog cat) const;
```

- <sup>3</sup> *Effects:* Calls `do_close(cat)`.

**28.3.4.8.2.3 Virtual functions****[locale.messages.virtuals]**

```
catalog do_open(const string& name, const locale& loc) const;
```

1 *Returns:* A value that may be passed to `get()` to retrieve a message from the message catalog identified by the string `name` according to an implementation-defined mapping. The result can be used until it is passed to `close()`.

2 Returns a value less than 0 if no such catalog can be opened.

3 *Remarks:* The locale argument `loc` is used for character set code conversion when retrieving messages, if needed.

```
string_type do_get(catalog cat, int set, int msgid, const string_type& default) const;
```

4 *Preconditions:* `cat` is a catalog obtained from `open()` and not yet closed.

5 *Returns:* A message identified by arguments `set`, `msgid`, and `default`, according to an implementation-defined mapping. If no such message can be found, returns `default`.

```
void do_close(catalog cat) const;
```

6 *Preconditions:* `cat` is a catalog obtained from `open()` and not yet closed.

7 *Effects:* Releases unspecified resources associated with `cat`.

8 *Remarks:* The limit on such resources, if any, is implementation-defined.

**28.3.4.8.3 Class template `messages_byname`****[locale.messages.byname]**

```
namespace std {
 template<class charT>
 class messages_byname : public messages<charT> {
 public:
 using catalog = messages_base::catalog;
 using string_type = basic_string<charT>;

 explicit messages_byname(const char*, size_t refs = 0);
 explicit messages_byname(const string&, size_t refs = 0);

 protected:
 ~messages_byname();
 };
}
```

**28.3.5 C library locales****[c.locales]****28.3.5.1 Header `<locale>` synopsis****[clocale.syn]**

```
namespace std {
 struct lconv;

 char* setlocale(int category, const char* locale);
 lconv* localeconv();
}

#define NULL see 17.2.3
#define LC_ALL see below
#define LC_COLLATE see below
#define LC_CTYPE see below
#define LC_MONETARY see below
#define LC_NUMERIC see below
#define LC_TIME see below
```

1 The contents and meaning of the header `<locale>` are the same as the C standard library header `<locale.h>`.

**28.3.5.2 Data races****[clocale.data.races]**

1 Calls to the function `setlocale` may introduce a data race (16.4.6.10) with other calls to `setlocale` or with calls to the functions listed in Table 103.

SEE ALSO: ISO/IEC 9899:2024, 7.11



Table 103 — Potential `setlocale` data races [tab:setlocale.data.races]

|                      |                       |                        |                         |                       |
|----------------------|-----------------------|------------------------|-------------------------|-----------------------|
| <code>fprintf</code> | <code>isprint</code>  | <code>iswdigit</code>  | <code>localeconv</code> | <code>tolower</code>  |
| <code>fscanf</code>  | <code>ispunct</code>  | <code>iswgraph</code>  | <code>mblen</code>      | <code>toupper</code>  |
| <code>isalnum</code> | <code>isspace</code>  | <code>iswlower</code>  | <code>mbstowcs</code>   | <code>towlower</code> |
| <code>isalpha</code> | <code>isupper</code>  | <code>iswprint</code>  | <code>mbtowc</code>     | <code>towupper</code> |
| <code>isblank</code> | <code>iswalnum</code> | <code>iswpunct</code>  | <code>setlocale</code>  | <code>wscoll</code>   |
| <code>iscntrl</code> | <code>iswalpha</code> | <code>iswspace</code>  | <code>strcoll</code>    | <code>wctod</code>    |
| <code>isdigit</code> | <code>iswblank</code> | <code>iswupper</code>  | <code>strerror</code>   | <code>wctombs</code>  |
| <code>isgraph</code> | <code>iswcntrl</code> | <code>iswxdigit</code> | <code>strtod</code>     | <code>wcsxfrm</code>  |
| <code>islower</code> | <code>iswctype</code> | <code>isxdigit</code>  | <code>strxfrm</code>    | <code>wctomb</code>   |

## 28.4 Text encodings identification

[text.encoding]

### 28.4.1 Header `<text_encoding>` synopsis

[text.encoding.syn]

```
namespace std {
 struct text_encoding;

 // 28.4.2.7, hash support
 template<class T> struct hash;
 template<> struct hash<text_encoding>;
}
```

### 28.4.2 Class `text_encoding`

[text.encoding.class]

#### 28.4.2.1 Overview

[text.encoding.overview]

- <sup>1</sup> The class `text_encoding` describes an interface for accessing the IANA Character Sets registry<sup>[9]</sup>.

```
namespace std {
 struct text_encoding {
 static constexpr size_t max_name_length = 63;

 // 28.4.2.6, enumeration text_encoding::id
 enum class id : int_least32_t {
 see below
 };
 using enum id;

 constexpr text_encoding() = default;
 constexpr explicit text_encoding(string_view enc) noexcept;
 constexpr text_encoding(id i) noexcept;

 constexpr id mib() const noexcept;
 constexpr const char* name() const noexcept;

 // 28.4.2.5, class text_encoding::aliases_view
 struct aliases_view;
 constexpr aliases_view aliases() const noexcept;

 friend constexpr bool operator==(const text_encoding& a,
 const text_encoding& b) noexcept;
 friend constexpr bool operator==(const text_encoding& encoding, id i) noexcept;

 static constexpr text_encoding literal() noexcept;
 static text_encoding environment();
 template<id i> static bool environment_is();

 private:
 id mib_ = id::unknown; // exposition only
 char name_[max_name_length + 1] = {0}; // exposition only
 static constexpr bool comp_name(string_view a, string_view b); // exposition only
 };
}
```

```
};
}
```

<sup>2</sup> Class `text_encoding` is a trivially copyable type (6.9.1).

#### 28.4.2.2 General

[text.encoding.general]

<sup>1</sup> A *registered character encoding* is a character encoding scheme in the IANA Character Sets registry.

[Note 1: The IANA Character Sets registry uses the term “character sets” to refer to character encodings. — end note]

The primary name of a registered character encoding is the name of that encoding specified in the IANA Character Sets registry.

<sup>2</sup> The set of known registered character encodings contains every registered character encoding specified in the IANA Character Sets registry except for the following:

(2.1) — NATS-DANO (33)

(2.2) — NATS-DANO-ADD (34)

<sup>3</sup> Each known registered character encoding is identified by an enumerator in `text_encoding::id`, and has a set of zero or more *aliases*.

<sup>4</sup> The set of aliases of a known registered character encoding is an implementation-defined superset of the aliases specified in the IANA Character Sets registry. The set of aliases for US-ASCII includes “ASCII”. No two aliases or primary names of distinct registered character encodings are equivalent when compared by `text_encoding::comp-name`.

<sup>5</sup> How a `text_encoding` object is determined to be representative of a character encoding scheme implemented in the translation or execution environment is implementation-defined.

<sup>6</sup> An object `e` of type `text_encoding` such that `e.mib() == text_encoding::id::unknown` is false and `e.mib() == text_encoding::id::other` is false maintains the following invariants:

(6.1) — `*e.name() == '\0'` is false, and

(6.2) — `e.mib() == text_encoding(e.name()).mib()` is true.

<sup>7</sup> *Recommended practice*:

(7.1) — Implementations should not consider registered encodings to be interchangeable.

[Example 1: Shift\_JIS and Windows-31J denote different encodings. — end example]

(7.2) — Implementations should not use the name of a registered encoding to describe another similar yet different non-registered encoding unless there is a precedent on that implementation.

[Example 2: Big5 — end example]

#### 28.4.2.3 Members

[text.encoding.members]

```
constexpr explicit text_encoding(string_view enc) noexcept;
```

<sup>1</sup> *Preconditions*:

(1.1) — `enc` represents a string in the ordinary literal encoding consisting only of elements of the basic character set (5.3.1).

(1.2) — `enc.size() <= max_name_length` is true.

(1.3) — `enc.contains('\0')` is false.

<sup>2</sup> *Postconditions*:

(2.1) — If there exists a primary name or alias `a` of a known registered character encoding such that `comp-name(a, enc)` is true, `mib_` has the value of the enumerator of `id` associated with that registered character encoding. Otherwise, `mib_ == id::other` is true.

(2.2) — `enc.compare(name_) == 0` is true.

```
constexpr text_encoding(id i) noexcept;
```

<sup>3</sup> *Preconditions*: `i` has the value of one of the enumerators of `id`.

<sup>4</sup> *Postconditions*:

- (4.1) — `mib_ == i` is true.
- (4.2) — If `(mib_ == id::unknown || mib_ == id::other)` is true, `strlen(name_) == 0` is true. Otherwise, `ranges::contains(aliases(), string_view(name_))` is true.

```
constexpr id mib() const noexcept;
```

5     *Returns:* `mib_`.

```
constexpr const char* name() const noexcept;
```

6     *Returns:* `name_`.

7     *Remarks:* `name()` is an NTBS and accessing elements of `name_` outside of the range `name() + [0, strlen(name()) + 1)` is undefined behavior.

```
constexpr aliases_view aliases() const noexcept;
```

Let `r` denote an instance of `aliases_view`. If `*this` represents a known registered character encoding, then:

- (7.1) — `r.front()` is the primary name of the registered character encoding,
- (7.2) — `r` contains the aliases of the registered character encoding, and
- (7.3) — `r` does not contain duplicate values when compared with `strcmp`.

Otherwise, `r` is an empty range.

8     Each element in `r` is a non-null, non-empty NTBS encoded in the literal character encoding and comprising only characters from the basic character set.

9     *Returns:* `r`.

10    [*Note 1:* The order of aliases in `r` is unspecified. — *end note*]

```
static consteval text_encoding literal() noexcept;
```

11    *Mandates:* `CHAR_BIT == 8` is true.

12    *Returns:* A `text_encoding` object representing the ordinary character literal encoding (5.3.1).

```
static text_encoding environment();
```

13    *Mandates:* `CHAR_BIT == 8` is true.

14    *Returns:* A `text_encoding` object representing the implementation-defined character encoding scheme of the environment. On a POSIX implementation, this is the encoding scheme associated with the POSIX locale denoted by the empty string "".

15    [*Note 2:* This function is not affected by calls to `setlocale`. — *end note*]

16    *Recommended practice:* Implementations should return a value that is not affected by calls to the POSIX function `setenv` and other functions which can modify the environment (17.14).

```
template<id i>
```

```
static bool environment_is();
```

17    *Mandates:* `CHAR_BIT == 8` is true.

18    *Returns:* `environment() == i`.

```
static constexpr bool comp_name(string_view a, string_view b);
```

19    *Returns:* true if the two strings `a` and `b` encoded in the ordinary literal encoding are equal, ignoring, from left-to-right,

- (19.1) — all elements that are not digits or letters (16.3.3.3.4.1),
- (19.2) — character case, and
- (19.3) — any sequence of one or more 0 characters not immediately preceded by a numeric prefix, where a numeric prefix is a sequence consisting of a digit in the range [1, 9] optionally followed by one or more elements which are not digits or letters,

and false otherwise.

[*Note 3:* This comparison is identical to the “Charset Alias Matching” algorithm described in the Unicode Technical Standard 22<sup>[11]</sup>. — *end note*]

[*Example 1:*

```
static_assert(comp_name("UTF-8", "utf8") == true);
static_assert(comp_name("u.t.f-008", "utf8") == true);
static_assert(comp_name("ut8", "utf8") == false);
static_assert(comp_name("utf-80", "utf8") == false);
```

— *end example*]

#### 28.4.2.4 Comparison functions

[text.encoding.cmp]

```
friend constexpr bool operator==(const text_encoding& a, const text_encoding& b) noexcept;
```

1 *Returns:* If `a.mib_ == id::other` && `b.mib_ == id::other` is true, then `comp_name(a.name_, b.name_)`. Otherwise, `a.mib_ == b.mib_`.

```
friend constexpr bool operator==(const text_encoding& encoding, id i) noexcept;
```

2 *Returns:* `encoding.mib_ == i`.

3 *Remarks:* This operator induces an equivalence relation on its arguments if and only if `i != id::other` is true.

#### 28.4.2.5 Class `text_encoding::aliases_view`

[text.encoding.aliases]

```
struct text_encoding::aliases_view : ranges::view_interface<text_encoding::aliases_view> {
 constexpr implementation_defined begin() const;
 constexpr implementation_defined end() const;
};
```

1 `text_encoding::aliases_view` models `copyable`, `ranges::view`, `ranges::random_access_range`, and `ranges::borrowed_range`.

[*Note 1:* `text_encoding::aliases_view` is not required to satisfy `ranges::common_range`, nor `default_initializable`. — *end note*]

2 Both `ranges::range_value_t<text_encoding::aliases_view>` and `ranges::range_reference_t<text_encoding::aliases_view>` denote `const char*`.

3 `ranges::iterator_t<text_encoding::aliases_view>` is a `constexpr` iterator (24.3.1).

#### 28.4.2.6 Enumeration `text_encoding::id`

[text.encoding.id]

```
namespace std {
 enum class text_encoding::id : int_least32_t {
 other = 1,
 unknown = 2,
 ASCII = 3,
 ISOLatin1 = 4,
 ISOLatin2 = 5,
 ISOLatin3 = 6,
 ISOLatin4 = 7,
 ISOLatinCyrillic = 8,
 ISOLatinArabic = 9,
 ISOLatinGreek = 10,
 ISOLatinHebrew = 11,
 ISOLatin5 = 12,
 ISOLatin6 = 13,
 ISOTextComm = 14,
 HalfWidthKatakana = 15,
 JISEncoding = 16,
 ShiftJIS = 17,
 EUCPkdFmtJapanese = 18,
 EUFixWidJapanese = 19,
 ISO4UnitedKingdom = 20,
 ISO11SwedishForNames = 21,
 ISO15Italian = 22,
 ISO17Spanish = 23,
```

ISO21German = 24,  
 ISO60DanishNorwegian = 25,  
 ISO69French = 26,  
 ISO10646UTF1 = 27,  
 ISO646basic1983 = 28,  
 INVARIANT = 29,  
 ISO2IntlRefVersion = 30,  
 NATSSEFI = 31,  
 NATSSEFIADD = 32,  
 ISO10Swedish = 35,  
 KSC56011987 = 36,  
 ISO2022KR = 37,  
 EUCKR = 38,  
 ISO2022JP = 39,  
 ISO2022JP2 = 40,  
 ISO13JISC6220jp = 41,  
 ISO14JISC6220ro = 42,  
 ISO16Portuguese = 43,  
 ISO18Greek70ld = 44,  
 ISO19LatinGreek = 45,  
 ISO25French = 46,  
 ISO27LatinGreek1 = 47,  
 ISO5427Cyrillic = 48,  
 ISO42JISC62261978 = 49,  
 ISO47BSViewdata = 50,  
 ISO49INIS = 51,  
 ISO50INIS8 = 52,  
 ISO51INISCyrillic = 53,  
 ISO54271981 = 54,  
 ISO5428Greek = 55,  
 ISO57GB1988 = 56,  
 ISO58GB231280 = 57,  
 ISO61Norwegian2 = 58,  
 ISO70VideotexSuppl = 59,  
 ISO84Portuguese2 = 60,  
 ISO85Spanish2 = 61,  
 ISO86Hungarian = 62,  
 ISO87JISX0208 = 63,  
 ISO88Greek7 = 64,  
 ISO89ASMO449 = 65,  
 ISO90 = 66,  
 ISO91JISC62291984a = 67,  
 ISO92JISC62991984b = 68,  
 ISO93JIS62291984badd = 69,  
 ISO94JIS62291984hand = 70,  
 ISO95JIS62291984handadd = 71,  
 ISO96JISC62291984kana = 72,  
 ISO2033 = 73,  
 ISO99NAPLPS = 74,  
 ISO102T617bit = 75,  
 ISO103T618bit = 76,  
 ISO111ECMACyrillic = 77,  
 ISO121Canadian1 = 78,  
 ISO122Canadian2 = 79,  
 ISO123CSAZ24341985gr = 80,  
 ISO88596E = 81,  
 ISO88596I = 82,  
 ISO128T101G2 = 83,  
 ISO88598E = 84,  
 ISO88598I = 85,  
 ISO139CSN369103 = 86,  
 ISO141JUSIB1002 = 87,  
 ISO143IECP271 = 88,  
 ISO146Serbian = 89,

ISO147Macedonian = 90,  
 ISO150 = 91,  
 ISO151Cuba = 92,  
 ISO6937Add = 93,  
 ISO153GOST1976874 = 94,  
 ISO8859Supp = 95,  
 ISO10367Box = 96,  
 ISO158Lap = 97,  
 ISO159JISX02121990 = 98,  
 ISO646Danish = 99,  
 USDK = 100,  
 DKUS = 101,  
 KSC5636 = 102,  
 Unicode11UTF7 = 103,  
 ISO2022CN = 104,  
 ISO2022CNEXT = 105,  
 UTF8 = 106,  
 ISO885913 = 109,  
 ISO885914 = 110,  
 ISO885915 = 111,  
 ISO885916 = 112,  
 GBK = 113,  
 GB18030 = 114,  
 OSDEBCDICDF0415 = 115,  
 OSDEBCDICDF03IRV = 116,  
 OSDEBCDICDF041 = 117,  
 ISO115481 = 118,  
 KZ1048 = 119,  
 UCS2 = 1000,  
 UCS4 = 1001,  
 UnicodeASCII = 1002,  
 UnicodeLatin1 = 1003,  
 UnicodeJapanese = 1004,  
 UnicodeIBM1261 = 1005,  
 UnicodeIBM1268 = 1006,  
 UnicodeIBM1276 = 1007,  
 UnicodeIBM1264 = 1008,  
 UnicodeIBM1265 = 1009,  
 Unicode11 = 1010,  
 SCSU = 1011,  
 UTF7 = 1012,  
 UTF16BE = 1013,  
 UTF16LE = 1014,  
 UTF16 = 1015,  
 CESU8 = 1016,  
 UTF32 = 1017,  
 UTF32BE = 1018,  
 UTF32LE = 1019,  
 BOCU1 = 1020,  
 UTF7IMAP = 1021,  
 Windows30Latin1 = 2000,  
 Windows31Latin1 = 2001,  
 Windows31Latin2 = 2002,  
 Windows31Latin5 = 2003,  
 HPRoman8 = 2004,  
 AdobeStandardEncoding = 2005,  
 VenturaUS = 2006,  
 VenturaInternational = 2007,  
 DECMCS = 2008,  
 PC850Multilingual = 2009,  
 PCp852 = 2010,  
 PC8CodePage437 = 2011,  
 PC8DanishNorwegian = 2012,  
 PC862LatinHebrew = 2013,

PC8Turkish = 2014,  
IBMSymbols = 2015,  
IBMThai = 2016,  
HPLegal = 2017,  
HPPiFont = 2018,  
HPMath8 = 2019,  
HPPSMath = 2020,  
HPDesktop = 2021,  
VenturaMath = 2022,  
MicrosoftPublishing = 2023,  
Windows31J = 2024,  
GB2312 = 2025,  
Big5 = 2026,  
Macintosh = 2027,  
IBM037 = 2028,  
IBM038 = 2029,  
IBM273 = 2030,  
IBM274 = 2031,  
IBM275 = 2032,  
IBM277 = 2033,  
IBM278 = 2034,  
IBM280 = 2035,  
IBM281 = 2036,  
IBM284 = 2037,  
IBM285 = 2038,  
IBM290 = 2039,  
IBM297 = 2040,  
IBM420 = 2041,  
IBM423 = 2042,  
IBM424 = 2043,  
IBM500 = 2044,  
IBM851 = 2045,  
IBM855 = 2046,  
IBM857 = 2047,  
IBM860 = 2048,  
IBM861 = 2049,  
IBM863 = 2050,  
IBM864 = 2051,  
IBM865 = 2052,  
IBM868 = 2053,  
IBM869 = 2054,  
IBM870 = 2055,  
IBM871 = 2056,  
IBM880 = 2057,  
IBM891 = 2058,  
IBM903 = 2059,  
IBM904 = 2060,  
IBM905 = 2061,  
IBM918 = 2062,  
IBM1026 = 2063,  
IBMEBCDICATDE = 2064,  
EBCDICATDEA = 2065,  
EBCDICCAFR = 2066,  
EBCDICDKNO = 2067,  
EBCDICDKNOA = 2068,  
EBCDICFISE = 2069,  
EBCDICFISEA = 2070,  
EBCDICFR = 2071,  
EBCDICIT = 2072,  
EBCDICPT = 2073,  
EBCDICES = 2074,  
EBCDICESA = 2075,  
EBCDICESSE = 2076,  
EBCDICUK = 2077,

```

EBCDICUS = 2078,
Unknown8BiT = 2079,
Mnemonic = 2080,
Mnem = 2081,
VISCII = 2082,
VIQR = 2083,
KOI8R = 2084,
HZGB2312 = 2085,
IBM866 = 2086,
PC775Baltic = 2087,
KOI8U = 2088,
IBM00858 = 2089,
IBM00924 = 2090,
IBM01140 = 2091,
IBM01141 = 2092,
IBM01142 = 2093,
IBM01143 = 2094,
IBM01144 = 2095,
IBM01145 = 2096,
IBM01146 = 2097,
IBM01147 = 2098,
IBM01148 = 2099,
IBM01149 = 2100,
Big5HKSCS = 2101,
IBM1047 = 2102,
PTCP154 = 2103,
Amiga1251 = 2104,
KOI7switched = 2105,
BRF = 2106,
TSCII = 2107,
CP51932 = 2108,
windows874 = 2109,
windows1250 = 2250,
windows1251 = 2251,
windows1252 = 2252,
windows1253 = 2253,
windows1254 = 2254,
windows1255 = 2255,
windows1256 = 2256,
windows1257 = 2257,
windows1258 = 2258,
TIS620 = 2259,
CP50220 = 2260
};
}

```

[Note 1: The `text_encoding::id` enumeration contains an enumerator for each known registered character encoding. For each encoding, the corresponding enumerator is derived from the alias beginning with “cs”, as follows:

- `csUnicode` is mapped to `text_encoding::id::UCS2`,
  - `csIBM904` is mapped to `text_encoding::id::IBM904`, and
  - the “cs” prefix is removed from other names.
- end note]

#### 28.4.2.7 Hash support

[text.encoding.hash]

```
template<> struct hash<text_encoding>;
```

<sup>1</sup> The specialization is enabled (22.10.19).

## 28.5 Formatting

[format]

### 28.5.1 Header <format> synopsis

[format.syn]

```

namespace std {
 // 28.5.6.7, class template basic_format_context

```



```

template<class Out, class charT> class basic_format_context;
using format_context = basic_format_context<unspecified, char>;
using wformat_context = basic_format_context<unspecified, wchar_t>;

// 28.5.8.3, class template basic_format_args
template<class Context> class basic_format_args;
using format_args = basic_format_args<format_context>;
using wformat_args = basic_format_args<wformat_context>;

// 28.5.4, class template basic_format_string
template<class charT, class... Args>
 struct basic_format_string;

template<class charT> struct dynamic-format-string { // exposition only
private:
 basic_string_view<charT> str; // exposition only
public:
 constexpr dynamic-format-string(basic_string_view<charT> s) noexcept : str(s) {}
 dynamic-format-string(const dynamic-format-string&) = delete;
 dynamic-format-string& operator=(const dynamic-format-string&) = delete;
};
constexpr dynamic-format-string<char>
 dynamic_format(string_view fmt) noexcept { return fmt; }
constexpr dynamic-format-string<wchar_t>
 dynamic_format(wstring_view fmt) noexcept { return fmt; }

template<class... Args>
 using format_string = basic_format_string<char, type_identity_t<Args>...>;
template<class... Args>
 using wformat_string = basic_format_string<wchar_t, type_identity_t<Args>...>;

// 28.5.5, formatting functions
template<class... Args>
 constexpr string format(format_string<Args...> fmt, Args&&... args);
template<class... Args>
 constexpr wstring format(wformat_string<Args...> fmt, Args&&... args);
template<class... Args>
 string format(const locale& loc, format_string<Args...> fmt, Args&&... args);
template<class... Args>
 wstring format(const locale& loc, wformat_string<Args...> fmt, Args&&... args);

constexpr string vformat(string_view fmt, format_args args);
constexpr wstring vformat(wstring_view fmt, wformat_args args);
string vformat(const locale& loc, string_view fmt, format_args args);
wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);

template<class Out, class... Args>
 constexpr Out format_to(Out out, format_string<Args...> fmt, Args&&... args);
template<class Out, class... Args>
 constexpr Out format_to(Out out, wformat_string<Args...> fmt, Args&&... args);
template<class Out, class... Args>
 Out format_to(Out out, const locale& loc, format_string<Args...> fmt, Args&&... args);
template<class Out, class... Args>
 Out format_to(Out out, const locale& loc, wformat_string<Args...> fmt, Args&&... args);

template<class Out>
 constexpr Out vformat_to(Out out, string_view fmt, format_args args);
template<class Out>
 constexpr Out vformat_to(Out out, wstring_view fmt, wformat_args args);
template<class Out>
 Out vformat_to(Out out, const locale& loc, string_view fmt, format_args args);
template<class Out>
 Out vformat_to(Out out, const locale& loc, wstring_view fmt, wformat_args args);

```

```

template<class Out> struct format_to_n_result {
 Out out;
 iter_difference_t<Out> size;
};
template<class Out, class... Args>
 constexpr format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 format_string<Args...> fmt,
 Args&&... args);
template<class Out, class... Args>
 constexpr format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 wformat_string<Args...> fmt,
 Args&&... args);
template<class Out, class... Args>
 format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 const locale& loc, format_string<Args...> fmt,
 Args&&... args);
template<class Out, class... Args>
 format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 const locale& loc, wformat_string<Args...> fmt,
 Args&&... args);

template<class... Args>
 constexpr size_t formatted_size(format_string<Args...> fmt, Args&&... args);
template<class... Args>
 constexpr size_t formatted_size(wformat_string<Args...> fmt, Args&&... args);
template<class... Args>
 size_t formatted_size(const locale& loc, format_string<Args...> fmt, Args&&... args);
template<class... Args>
 size_t formatted_size(const locale& loc, wformat_string<Args...> fmt, Args&&... args);

// 28.5.6, formatter
template<class T, class charT = char> struct formatter;

// 28.5.6.2, formatter locking
template<class T>
 constexpr bool enable_nonlocking_formatter_optimization = false;

// 28.5.6.3, concept formattable
template<class T, class charT>
 concept formattable = see below;

template<class R, class charT>
 concept const-formattable-range = // exposition only
 ranges::input_range<const R> &&
 formattable<ranges::range_reference_t<const R>, charT>;

template<class R, class charT>
 using fmt-maybe-const = // exposition only
 conditional_t<const-formattable-range<R, charT>, const R, R>;

// 28.5.6.6, class template basic_format_parse_context
template<class charT> class basic_format_parse_context;
using format_parse_context = basic_format_parse_context<char>;
using wformat_parse_context = basic_format_parse_context<wchar_t>;

// 28.5.7, formatting of ranges
// 28.5.7.1, variable template format_kind
enum class range_format {
 disabled,
 map,
 set,
 sequence,
 string,
 debug_string
};

```

```

};

template<class R>
 constexpr unspecified format_kind = unspecified;

template<ranges::input_range R>
 requires same_as<R, remove_cvref_t<R>>
 constexpr range_format format_kind<R> = see below;

// 28.5.7.2, class template range_formatter
template<class T, class charT = char>
 requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
 class range_formatter;

// 28.5.7.3, class template range-default-formatter
template<range_format K, ranges::input_range R, class charT>
 struct range-default-formatter; // exposition only

// 28.5.7.4, 28.5.7.5, 28.5.7.6, specializations for maps, sets, and strings
template<ranges::input_range R, class charT>
 requires (format_kind<R> != range_format::disabled) &&
 formattable<ranges::range_reference_t<R>, charT>
 struct formatter<R, charT> : range-default-formatter<format_kind<R>, R, charT> { };

template<ranges::input_range R>
 requires (format_kind<R> != range_format::disabled)
 constexpr bool enable_nonlocking_formatter_optimization<R> = false;

// 28.5.8, arguments
// 28.5.8.1, class template basic_format_arg
template<class Context> class basic_format_arg;

// 28.5.8.2, class template format-arg-store
template<class Context, class... Args> class format-arg-store; // exposition only

template<class Context = format_context, class... Args>
 constexpr format-arg-store<Context, Args...>
 make_format_args(Args&... fmt_args);
template<class... Args>
 constexpr format-arg-store<wformat_context, Args...>
 make_wformat_args(Args&... args);

// 28.5.10, class format_error
class format_error;
}

```

- <sup>1</sup> The class template `format_to_n_result` has the template parameters, data members, and special members specified above. It has no base classes or members other than those specified.

## 28.5.2 Format string

[format.string]

### 28.5.2.1 General

[format.string.general]

- <sup>1</sup> A *format string* for arguments `args` is a (possibly empty) sequence of *replacement fields*, *escape sequences*, and characters other than `{` and `}`. Let `charT` be the character type of the format string. Each character that is not part of a replacement field or an escape sequence is copied unchanged to the output. An escape sequence is one of `{ {` or `} }`. It is replaced with `{` or `}`, respectively, in the output. The syntax of replacement fields is as follows:

```

replacement-field:
 { arg-idopt format-specifieropt }

arg-id:
 0
 positive-integer

```

*positive-integer*:  
*nonzero-digit*  
*positive-integer digit*

*nonnegative-integer*:  
*digit*  
*nonnegative-integer digit*

*nonzero-digit*: one of  
 1 2 3 4 5 6 7 8 9

*digit*: one of  
 0 1 2 3 4 5 6 7 8 9

*format-specifier*:  
 : *format-spec*

*format-spec*:  
 as specified by the **formatter** specialization for the argument type; cannot start with }

- <sup>2</sup> The *arg-id* field specifies the index of the argument in **args** whose value is to be formatted and inserted into the output instead of the replacement field. If there is no argument with the index *arg-id* in **args**, the string is not a format string for **args**. The optional *format-specifier* field explicitly specifies a format for the replacement value.

<sup>3</sup> [Example 1:

```
string s = format("{0}-{", 8); // value of s is "8-{"
```

— end example]

- <sup>4</sup> If all *arg-ids* in a format string are omitted (including those in the *format-spec*, as interpreted by the corresponding **formatter** specialization), argument indices 0, 1, 2, ... will automatically be used in that order. If some *arg-ids* are omitted and some are present, the string is not a format string.

[Note 1: A format string cannot contain a mixture of automatic and manual indexing. — end note]

[Example 2:

```
string s0 = format("{} to {}", "a", "b"); // OK, automatic indexing
string s1 = format("{1} to {0}", "a", "b"); // OK, manual indexing
string s2 = format("{0} to {}", "a", "b"); // not a format string (mixing automatic and manual indexing),
 // ill-formed
string s3 = format("{} to {1}", "a", "b"); // not a format string (mixing automatic and manual indexing),
 // ill-formed
```

— end example]

- <sup>5</sup> The *format-spec* field contains *format specifications* that define how the value should be presented. Each type can define its own interpretation of the *format-spec* field. If *format-spec* does not conform to the format specifications for the argument type referred to by *arg-id*, the string is not a format string for **args**.

[Example 3:

- (5.1) — For arithmetic, pointer, and string types the *format-spec* is interpreted as a *std-format-spec* as described in 28.5.2.2.
- (5.2) — For chrono types the *format-spec* is interpreted as a *chrono-format-spec* as described in 30.12.
- (5.3) — For user-defined **formatter** specializations, the behavior of the **parse** member function determines how the *format-spec* is interpreted.

— end example]

### 28.5.2.2 Standard format specifiers

[format.string.std]

- <sup>1</sup> Each **formatter** specialization described in 28.5.6.4 for fundamental and string types interprets *format-spec* as a *std-format-spec*.

[Note 1: The format specification can be used to specify such details as minimum field width, alignment, padding, and decimal precision. Some of the formatting options are only supported for arithmetic types. — end note]

The syntax of format specifications is as follows:

*std-format-spec*:  
*fill-and-align<sub>opt</sub> sign<sub>opt</sub> #<sub>opt</sub> 0<sub>opt</sub> width<sub>opt</sub> precision<sub>opt</sub> L<sub>opt</sub> type<sub>opt</sub>*

*fill-and-align*:  
*fill<sub>opt</sub>* *align*

*fill*:  
 any character other than { or }

*align*: one of  
 < > ^

*sign*: one of  
 + - space

*width*:  
*positive-integer*  
 { *arg-id<sub>opt</sub>* }

*precision*:  
 . *nonnegative-integer*  
 . { *arg-id<sub>opt</sub>* }

*type*: one of  
 a A b B c d e E f F g G o p P s x X ?

- <sup>2</sup> Field widths are specified in *field width units*; the number of column positions required to display a sequence of characters in a terminal. The *minimum field width* is the number of field width units a replacement field minimally requires of the formatted sequence of characters produced for a format argument. The *estimated field width* is the number of field width units that are required for the formatted sequence of characters produced for a format argument independent of the effects of the *width* option. The *padding width* is the greater of 0 and the difference of the minimum field width and the estimated field width.

[Note 2: The POSIX `wcswidth` function is an example of a function that, given a string, returns the number of column positions required by a terminal to display the string. — end note]

- <sup>3</sup> The *fill character* is the character denoted by the *fill* option or, if the *fill* option is absent, the space character. For a format specification in UTF-8, UTF-16, or UTF-32, the fill character corresponds to a single Unicode scalar value.

[Note 3: The presence of a *fill* option is signaled by the character following it, which must be one of the alignment options. If the second character of *std-format-spec* is not a valid alignment option, then it is assumed that the *fill* and *align* options are both absent. — end note]

- <sup>4</sup> The *align* option applies to all argument types. The meaning of the various alignment options is as specified in Table 104.

[Example 1:

```
char c = 120;
string s0 = format("{:6}", 42); // value of s0 is " 42"
string s1 = format("{:6}", 'x'); // value of s1 is "x "
string s2 = format("{:*<6}", 'x'); // value of s2 is "x*****"
string s3 = format("{:>6}", 'x'); // value of s3 is "*****x"
string s4 = format("{:*^6}", 'x'); // value of s4 is "***x***"
string s5 = format("{:6d}", c); // value of s5 is " 120"
string s6 = format("{:6}", true); // value of s6 is "true "
string s7 = format("{:*<6.3}", "123456"); // value of s7 is "123***"
string s8 = format("{:02}", 1234); // value of s8 is "1234"
string s9 = format("{:<}", "12"); // value of s9 is "12"
string sA = format("{:*<6}", "12345678"); // value of sA is "12345678"
string sB = format("{:🤡^6}", "x"); // value of sB is "🤡🤡x🤡🤡🤡"
string sC = format("{:*^6}", "🤡🤡🤡"); // value of sC is "🤡🤡🤡"
```

— end example]

[Note 4: The *fill*, *align*, and 0 options have no effect when the minimum field width is not greater than the estimated field width because padding width is 0 in that case. Since fill characters are assumed to have a field width of 1, use of a character with a different field width can produce misaligned output. The 🤡 (U+1F921 CLOWN FACE) character has a field width of 2. The examples above that include that character illustrate the effect of the field width when that character is used as a fill character as opposed to when it is used as a formatting argument. — end note]

- <sup>5</sup> The *sign* option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. The meaning of the various options is as specified in Table 105.
- <sup>6</sup> The *sign* option applies to floating-point infinity and NaN.

Table 104 — Meaning of *align* options [tab:format.align]

| Option | Meaning                                                                                                                                                                                                                                                                                                                             |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <      | Forces the formatted argument to be aligned to the start of the field by inserting $n$ fill characters after the formatted argument where $n$ is the padding width. This is the default for non-arithmetic non-pointer types, <code>charT</code> , and <code>bool</code> , unless an integer presentation type is specified.        |
| >      | Forces the formatted argument to be aligned to the end of the field by inserting $n$ fill characters before the formatted argument where $n$ is the padding width. This is the default for arithmetic types other than <code>charT</code> and <code>bool</code> , pointer types, or when an integer presentation type is specified. |
| ~      | Forces the formatted argument to be centered within the field by inserting $\lfloor \frac{n}{2} \rfloor$ fill characters before and $\lceil \frac{n}{2} \rceil$ fill characters after the formatted argument, where $n$ is the padding width.                                                                                       |

Table 105 — Meaning of *sign* options [tab:format.sign]

| Option | Meaning                                                                                                                                                                                                                                                                                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| +      | Indicates that a sign should be used for both non-negative and negative numbers. The + sign is inserted before the output of <code>to_chars</code> for non-negative numbers other than negative zero.<br>[Note 5: For negative numbers and negative zero the output of <code>to_chars</code> will already contain the sign so no additional transformation is performed. — end note] |
| -      | Indicates that a sign should be used for negative numbers and negative zero only (this is the default behavior).                                                                                                                                                                                                                                                                     |
| space  | Indicates that a leading space should be used for non-negative numbers other than negative zero, and a minus sign for negative numbers and negative zero.                                                                                                                                                                                                                            |

[Example 2:

```
double inf = numeric_limits<double>::infinity();
double nan = numeric_limits<double>::quiet_NaN();
string s0 = format("{0:},{0:+},{0:-},{0: }", 1); // value of s0 is "1,+1,1, 1"
string s1 = format("{0:},{0:+},{0:-},{0: }", -1); // value of s1 is "-1,-1,-1,-1"
string s2 = format("{0:},{0:+},{0:-},{0: }", inf); // value of s2 is "inf,+inf,inf, inf"
string s3 = format("{0:},{0:+},{0:-},{0: }", nan); // value of s3 is "nan,+nan,nan, nan"
```

— end example]

- <sup>7</sup> The # option causes the *alternate form* to be used for the conversion. This option is valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified, and not otherwise. For integral types, the alternate form inserts the base prefix (if any) specified in Table 107 into the output after the sign character (possibly space) if there is one, or before the output of `to_chars` otherwise. For floating-point types, the alternate form causes the result of the conversion of finite values to always contain a decimal-point character, even if no digits follow it. Normally, a decimal-point character appears in the result of these conversions only if a digit follows it. In addition, for `g` and `G` conversions, trailing zeros are not removed from the result.
- <sup>8</sup> The 0 option is valid for arithmetic types other than `charT` and `bool`, pointer types, or when an integer presentation type is specified. For formatting arguments that have a value other than an infinity or a NaN, this option pads the formatted argument by inserting the 0 character  $n$  times following the sign or base prefix indicators (if any) where  $n$  is 0 if the *align* option is present and is the padding width otherwise.

[Example 3:

```
char c = 120;
string s1 = format("{: +06d}", c); // value of s1 is "+00120"
string s2 = format("{: #06x}", 0xa); // value of s2 is "0x000a"
string s3 = format("{: <06}", -42); // value of s3 is "-42 " (0 has no effect)
string s4 = format("{: 06}", inf); // value of s4 is " inf" (0 has no effect)
```

— *end example*]

- 9 The *width* option specifies the minimum field width. If the *width* option is absent, the minimum field width is 0.
- 10 If { *arg-id<sub>opt</sub>* } is used in a *width* or *precision* option, the value of the corresponding formatting argument is used as the value of the option. The option is valid only if the corresponding formatting argument is of standard signed or unsigned integer type. If its value is negative, an exception of type `format_error` is thrown.
- 11 If *positive-integer* is used in a *width* option, the value of the *positive-integer* is interpreted as a decimal integer and used as the value of the option.
- 12 For the purposes of width computation, a string is assumed to be in a locale-independent, implementation-defined encoding. Implementations should use either UTF-8, UTF-16, or UTF-32, on platforms capable of displaying Unicode text in a terminal.  
[Note 6: This is the case for Windows®-based<sup>216</sup> and many POSIX-based operating systems. — *end note*]
- 13 For a sequence of characters in UTF-8, UTF-16, or UTF-32, an implementation should use as its field width the sum of the field widths of the first code point of each extended grapheme cluster. Extended grapheme clusters are defined by UAX #29 of the Unicode Standard. The following code points have a field width of 2:
  - (13.1) — any code point with the `East_Asian_Width="W"` or `East_Asian_Width="F"` property as described by UAX #44 of the Unicode Standard
  - (13.2) — U+4DC0 – U+4DFF (Yijing Hexagram Symbols)
  - (13.3) — U+1F300 – U+1F5FF (Miscellaneous Symbols and Pictographs)
  - (13.4) — U+1F900 – U+1F9FF (Supplemental Symbols and Pictographs)

The field width of all other code points is 1.

- 14 For a sequence of characters in neither UTF-8, UTF-16, nor UTF-32, the field width is unspecified.
- 15 The *precision* option is valid for floating-point and string types. For floating-point types, the value of this option specifies the precision to be used for the floating-point presentation type. For string types, this option specifies the longest prefix of the formatted argument to be included in the replacement field such that the field width of the prefix is no greater than the value of this option.
- 16 If *nonnegative-integer* is used in a *precision* option, the value of the decimal integer is used as the value of the option.
- 17 When the `L` option is used, the form used for the conversion is called the *locale-specific form*. The `L` option is only valid for arithmetic types, and its effect depends upon the type. A call to `format` on a given formatter specialization is not a constant subexpression if the locale-specific form is specified.
  - (17.1) — For integral types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group separator characters as if obtained with `numpunct<charT>::grouping` and `numpunct<charT>::thousands_sep`.
  - (17.2) — For floating-point types, the locale-specific form causes the context's locale to be used to insert the appropriate digit group and radix separator characters as if obtained with `numpunct<charT>::grouping`, `numpunct<charT>::thousands_sep`, and `numpunct<charT>::decimal_point`.
  - (17.3) — For the textual representation of `bool`, the locale-specific form causes the context's locale to be used to insert the appropriate string as if obtained with `numpunct<charT>::truename` or `numpunct<charT>::falsename`.

If the string literal encoding is a Unicode encoding form and the locale is among an implementation-defined set of locales, each replacement that depends on the locale is performed as if the replacement character sequence is converted to the string literal encoding.

- 18 The *type* determines how the data should be presented.
- 19 The available string presentation types are specified in Table 106.
- 20 The meaning of some non-string presentation types is defined in terms of a call to `to_chars`. In such cases, let `[first, last)` be a range large enough to hold the `to_chars` output and `value` be the formatting argument

<sup>216</sup>) Windows® is a registered trademark of Microsoft Corporation. This information is given for the convenience of users of this document and does not constitute an endorsement by ISO or IEC of this product.



Table 106 — Meaning of *type* options for strings [tab:format.type.string]

| Type    | Meaning                                             |
|---------|-----------------------------------------------------|
| none, s | Copies the string to the output.                    |
| ?       | Copies the escaped string (28.5.6.5) to the output. |

value. Formatting is done as if by calling `to_chars` as specified, transcoding the `to_chars` output to the wide literal encoding if `charT` is `wchar_t`, and copying the output through the output iterator of the format context.

[Note 7: Additional padding and adjustments are performed prior to copying the output through the output iterator as specified by the format specifiers. — end note]

- 21 The available integer presentation types for integral types other than `bool` and `charT` are specified in Table 107.

[Example 4:

```
string s0 = format("{} ", 42); // value of s0 is "42"
string s1 = format("{0:b} {0:d} {0:o} {0:x}", 42); // value of s1 is "101010 42 52 2a"
string s2 = format("{0:#x} {0:#X}", 42); // value of s2 is "0x2a 0X2A"
string s3 = format("{:L}", 1234); // value of s3 can be "1,234"
 // (depending on the locale)
```

— end example]

Table 107 — Meaning of *type* options for integer types [tab:format.type.int]

| Type | Meaning                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| b    | <code>to_chars(first, last, value, 2)</code> ; the base prefix is <code>0b</code> .                                                                                                                          |
| B    | The same as <code>b</code> , except that the base prefix is <code>0B</code> .                                                                                                                                |
| c    | Copies the character <code>static_cast&lt;charT&gt;(value)</code> to the output. Throws <code>format_error</code> if <code>value</code> is not in the range of representable values for <code>charT</code> . |
| d    | <code>to_chars(first, last, value)</code> .                                                                                                                                                                  |
| o    | <code>to_chars(first, last, value, 8)</code> ; the base prefix is <code>0</code> if <code>value</code> is nonzero and is empty otherwise.                                                                    |
| x    | <code>to_chars(first, last, value, 16)</code> ; the base prefix is <code>0x</code> .                                                                                                                         |
| X    | The same as <code>x</code> , except that it uses uppercase letters for digits above 9 and the base prefix is <code>0X</code> .                                                                               |
| none | The same as <code>d</code> .<br>[Note 8: If the formatting argument type is <code>charT</code> or <code>bool</code> , the default is instead <code>c</code> or <code>s</code> , respectively. — end note]    |

- 22 The available `charT` presentation types are specified in Table 108.

Table 108 — Meaning of *type* options for `charT` [tab:format.type.char]

| Type             | Meaning                                                                                                     |
|------------------|-------------------------------------------------------------------------------------------------------------|
| none, c          | Copies the character to the output.                                                                         |
| b, B, d, o, x, X | As specified in Table 107 with <code>value</code> converted to the unsigned version of the underlying type. |
| ?                | Copies the escaped character (28.5.6.5) to the output.                                                      |

- 23 The available `bool` presentation types are specified in Table 109.

- 24 The available floating-point presentation types and their meanings for values other than infinity and NaN are specified in Table 110. For lower-case presentation types, infinity and NaN are formatted as `inf` and `nan`, respectively. For upper-case presentation types, infinity and NaN are formatted as `INF` and `NAN`, respectively.

[Note 9: In either case, a sign is included if indicated by the *sign* option. — end note]



Table 109 — Meaning of *type* options for `bool` [tab:format.type.bool]

| Type             | Meaning                                                                                        |
|------------------|------------------------------------------------------------------------------------------------|
| none, s          | Copies textual representation, either <code>true</code> or <code>false</code> , to the output. |
| b, B, d, o, x, X | As specified in Table 107 for the value <code>static_cast&lt;unsigned char&gt;(value)</code> . |

Table 110 — Meaning of *type* options for floating-point types [tab:format.type.float]

| Type | Meaning                                                                                                                                                                                                                                                                                 |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| a    | If <i>precision</i> is specified, equivalent to<br><code>to_chars(first, last, value, chars_format::hex, precision)</code><br>where <i>precision</i> is the specified formatting precision; equivalent to<br><code>to_chars(first, last, value, chars_format::hex)</code><br>otherwise. |
| A    | The same as <code>a</code> , except that it uses uppercase letters for digits above 9 and <code>P</code> to indicate the exponent.                                                                                                                                                      |
| e    | Equivalent to<br><code>to_chars(first, last, value, chars_format::scientific, precision)</code><br>where <i>precision</i> is the specified formatting precision, or 6 if <i>precision</i> is not specified.                                                                             |
| E    | The same as <code>e</code> , except that it uses <code>E</code> to indicate exponent.                                                                                                                                                                                                   |
| f, F | Equivalent to<br><code>to_chars(first, last, value, chars_format::fixed, precision)</code><br>where <i>precision</i> is the specified formatting precision, or 6 if <i>precision</i> is not specified.                                                                                  |
| g    | Equivalent to<br><code>to_chars(first, last, value, chars_format::general, precision)</code><br>where <i>precision</i> is the specified formatting precision, or 6 if <i>precision</i> is not specified.                                                                                |
| G    | The same as <code>g</code> , except that it uses <code>E</code> to indicate exponent.                                                                                                                                                                                                   |
| none | If <i>precision</i> is specified, equivalent to<br><code>to_chars(first, last, value, chars_format::general, precision)</code><br>where <i>precision</i> is the specified formatting precision; equivalent to<br><code>to_chars(first, last, value)</code><br>otherwise.                |

<sup>25</sup> The available pointer presentation types and their mapping to `to_chars` are specified in Table 111.

[Note 10: Pointer presentation types also apply to `nullptr_t`. — end note]

Table 111 — Meaning of *type* options for pointer types [tab:format.type.ptr]

| Type    | Meaning                                                                                                                                                                               |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| none, p | <code>to_chars(first, last, reinterpret_cast&lt;uintptr_t&gt;(value), 16)</code><br>with the prefix <code>0x</code> inserted immediately before the output of <code>to_chars</code> . |
| P       | The same as <code>p</code> , except that it uses uppercase letters for digits above 9 and the base prefix is <code>0X</code> .                                                        |

### 28.5.3 Error reporting

[format.err.report]

- <sup>1</sup> Formatting functions throw `format_error` if an argument `fmt` is passed that is not a format string for `args`. They propagate exceptions thrown by operations of `formatter` specializations and iterators. Failure to allocate storage is reported by throwing an exception as described in 16.4.6.14.

### 28.5.4 Class template `basic_format_string`

[format.fmt.string]

```
namespace std {
 template<class charT, class... Args>
 struct basic_format_string {
 private:
```

```

 basic_string_view<charT> str; // exposition only

public:
 template<class T> consteval basic_format_string(const T& s);
 constexpr basic_format_string(dynamic-format-string<charT> s) noexcept : str(s.str) {}

 constexpr basic_string_view<charT> get() const noexcept { return str; }
};

```

```

template<class T> consteval basic_format_string(const T& s);

```

1     *Constraints:* const T& models convertible\_to<basic\_string\_view<charT>>.

2     *Effects:* Direct-non-list-initializes *str* with s.

3     *Remarks:* A call to this function is not a core constant expression (7.7.2) unless there exist args of types Args such that *str* is a format string for args.

### 28.5.5 Formatting functions

[format.functions]

1 A call to any of the functions defined in this subclause is a constant subexpression only if each of the used formatter specializations is a constexpr-enabled specialization (28.5.6.4).

2 In the description of the functions, operator + is used for some of the iterator categories for which it does not have to be defined. In these cases the semantics of a + n are the same as in 26.2.

```

template<class... Args>
 constexpr string format(format_string<Args...> fmt, Args&&... args);

```

3     *Effects:* Equivalent to:

```

 return vformat(fmt.str, make_format_args(args...));

```

```

template<class... Args>
 constexpr wstring format(wformat_string<Args...> fmt, Args&&... args);

```

4     *Effects:* Equivalent to:

```

 return vformat(fmt.str, make_wformat_args(args...));

```

```

template<class... Args>
 string format(const locale& loc, format_string<Args...> fmt, Args&&... args);

```

5     *Effects:* Equivalent to:

```

 return vformat(loc, fmt.str, make_format_args(args...));

```

```

template<class... Args>
 wstring format(const locale& loc, wformat_string<Args...> fmt, Args&&... args);

```

6     *Effects:* Equivalent to:

```

 return vformat(loc, fmt.str, make_wformat_args(args...));

```

```

constexpr string vformat(string_view fmt, format_args args);
constexpr wstring vformat(wstring_view fmt, wformat_args args);
string vformat(const locale& loc, string_view fmt, format_args args);
wstring vformat(const locale& loc, wstring_view fmt, wformat_args args);

```

7     *Returns:* A string object holding the character representation of formatting arguments provided by args formatted according to specifications given in fmt. If present, loc is used for locale-specific formatting.

8     *Throws:* As specified in 28.5.3.

```

template<class Out, class... Args>
 constexpr Out format_to(Out out, format_string<Args...> fmt, Args&&... args);

```

9     *Effects:* Equivalent to:

```

 return vformat_to(std::move(out), fmt.str, make_format_args(args...));

```

```
template<class Out, class... Args>
constexpr Out format_to(Out out, wformat_string<Args...> fmt, Args&&... args);
```

10 *Effects:* Equivalent to:

```
return vformat_to(std::move(out), fmt.str, make_wformat_args(args...));
```

```
template<class Out, class... Args>
Out format_to(Out out, const locale& loc, format_string<Args...> fmt, Args&&... args);
```

11 *Effects:* Equivalent to:

```
return vformat_to(std::move(out), loc, fmt.str, make_format_args(args...));
```

```
template<class Out, class... Args>
Out format_to(Out out, const locale& loc, wformat_string<Args...> fmt, Args&&... args);
```

12 *Effects:* Equivalent to:

```
return vformat_to(std::move(out), loc, fmt.str, make_wformat_args(args...));
```

```
template<class Out>
constexpr Out vformat_to(Out out, string_view fmt, format_args args);
template<class Out>
constexpr Out vformat_to(Out out, wstring_view fmt, wformat_args args);
template<class Out>
Out vformat_to(Out out, const locale& loc, string_view fmt, format_args args);
template<class Out>
Out vformat_to(Out out, const locale& loc, wstring_view fmt, wformat_args args);
```

13 Let `charT` be `decltype(fmt)::value_type`.

14 *Constraints:* `Out` satisfies `output_iterator<const charT&>`.

15 *Preconditions:* `Out` models `output_iterator<const charT&>`.

16 *Effects:* Places the character representation of formatting the arguments provided by `args`, formatted according to the specifications given in `fmt`, into the range `[out, out + N)`, where `N` is the number of characters in that character representation. If present, `loc` is used for locale-specific formatting.

17 *Returns:* `out + N`.

18 *Throws:* As specified in [28.5.3](#).

```
template<class Out, class... Args>
constexpr format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 format_string<Args...> fmt,
 Args&&... args);
```

```
template<class Out, class... Args>
constexpr format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 wformat_string<Args...> fmt,
 Args&&... args);
```

```
template<class Out, class... Args>
format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 const locale& loc, format_string<Args...> fmt,
 Args&&... args);
```

```
template<class Out, class... Args>
format_to_n_result<Out> format_to_n(Out out, iter_difference_t<Out> n,
 const locale& loc, wformat_string<Args...> fmt,
 Args&&... args);
```

19 Let

(19.1) — `charT` be `decltype(fmt.str)::value_type`,

(19.2) — `N` be `formatted_size(fmt, args...)` for the functions without a `loc` parameter and `formatted_size(loc, fmt, args...)` for the functions with a `loc` parameter, and

(19.3) — `M` be `clamp(n, 0, N)`.

20 *Constraints:* `Out` satisfies `output_iterator<const charT&>`.

21 *Preconditions:* `Out` models `output_iterator<const charT&>`, and `formatter<remove_cvref_t<Ti>, charT>` meets the *BasicFormatter* requirements ([28.5.6.1](#)) for each `Ti` in `Args`.

22 *Effects:* Places the first *M* characters of the character representation of formatting the arguments provided by *args*, formatted according to the specifications given in *fmt*, into the range *[out, out + M)*. If present, *loc* is used for locale-specific formatting.

23 *Returns:* *{out + M, N}*.

24 *Throws:* As specified in 28.5.3.

```
template<class... Args>
 constexpr size_t formatted_size(format_string<Args...> fmt, Args&&... args);
template<class... Args>
 constexpr size_t formatted_size(wformat_string<Args...> fmt, Args&&... args);
template<class... Args>
 size_t formatted_size(const locale& loc, format_string<Args...> fmt, Args&&... args);
template<class... Args>
 size_t formatted_size(const locale& loc, wformat_string<Args...> fmt, Args&&... args);
```

25 Let *charT* be *decltype*(*fmt.str*)::*value\_type*.

26 *Preconditions:* *formatter<remove\_cvref\_t<T<sub>i</sub>>, charT>* meets the *BasicFormatter* requirements (28.5.6.1) for each *T<sub>i</sub>* in *Args*.

27 *Returns:* The number of characters in the character representation of formatting arguments *args* formatted according to specifications given in *fmt*. If present, *loc* is used for locale-specific formatting.

28 *Throws:* As specified in 28.5.3.

## 28.5.6 Formatter

[format.formatter]

### 28.5.6.1 Formatter requirements

[formatter.requirements]

1 A type *F* meets the *BasicFormatter* requirements if it meets the

- (1.1) — *Cpp17DefaultConstructible* (Table 30),
- (1.2) — *Cpp17CopyConstructible* (Table 32),
- (1.3) — *Cpp17CopyAssignable* (Table 34),
- (1.4) — *Cpp17Swappable* (16.4.4.3), and
- (1.5) — *Cpp17Destructible* (Table 35)

requirements, and the expressions shown in Table 112 are valid and have the indicated semantics.

2 A type *F* meets the *Formatter* requirements if it meets the *BasicFormatter* requirements and the expressions shown in Table 113 are valid and have the indicated semantics.

3 Given character type *charT*, output iterator type *Out*, and formatting argument type *T*, in Table 112 and Table 113:

- (3.1) — *f* is a value of type (possibly *const*) *F*,
- (3.2) — *g* is an lvalue of type *F*,
- (3.3) — *u* is an lvalue of type *T*,
- (3.4) — *t* is a value of a type convertible to (possibly *const*) *T*,
- (3.5) — *PC* is *basic\_format\_parse\_context<charT>*,
- (3.6) — *FC* is *basic\_format\_context<Out, charT>*,
- (3.7) — *pc* is an lvalue of type *PC*, and
- (3.8) — *fc* is an lvalue of type *FC*.

*pc.begin()* points to the beginning of the *format-spec* (28.5.2) of the replacement field being formatted in the format string. If *format-spec* is not present or empty then either *pc.begin() == pc.end()* or *\*pc.begin() == '}'*.

### 28.5.6.2 Formatter locking

[format.formatter.locking]

```
template<class T>
 constexpr bool enable_nonlocking_formatter_optimization = false;
```

1 *Remarks:* Pursuant to 16.4.5.2.1, users may specialize *enable\_nonlocking\_formatter\_optimization*

Table 112 — *BasicFormatter* requirements [tab:formatter.basic]

| Expression                   | Return type               | Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>g.parse(pc)</code>     | <code>PC::iterator</code> | Parses <i>format-spec</i> (28.5.2) for type <code>T</code> in the range <code>[pc.begin(), pc.end())</code> until the first unmatched character. Throws <code>format_error</code> unless the whole range is parsed or the unmatched character is <code>}</code> .<br>[Note 1: This allows formatters to emit meaningful error messages. — end note]<br>Stores the parsed format specifiers in <code>*this</code> and returns an iterator past the end of the parsed range. |
| <code>f.format(u, fc)</code> | <code>FC::iterator</code> | Formats <code>u</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> , and returns an iterator past the end of the output range. The output shall only depend on <code>u</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>f.parse(pc)</code> .                  |

Table 113 — *Formatter* requirements [tab:formatter]

| Expression                   | Return type               | Requirement                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>f.format(t, fc)</code> | <code>FC::iterator</code> | Formats <code>t</code> according to the specifiers stored in <code>*this</code> , writes the output to <code>fc.out()</code> , and returns an iterator past the end of the output range. The output shall only depend on <code>t</code> , <code>fc.locale()</code> , <code>fc.arg(n)</code> for any value <code>n</code> of type <code>size_t</code> , and the range <code>[pc.begin(), pc.end())</code> from the last call to <code>f.parse(pc)</code> . |
| <code>f.format(u, fc)</code> | <code>FC::iterator</code> | As above, but does not modify <code>u</code> .                                                                                                                                                                                                                                                                                                                                                                                                            |

for cv-unqualified program-defined types. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.

### 28.5.6.3 Concept *formattable*

[format.formattable]

- <sup>1</sup> Let *fmt-iter-for*<charT> be an unspecified type that models `output_iterator<const charT>` (24.3.4.10).

```
template<class T, class Context,
 class Formatter = typename Context::template formatter_type<remove_const_t<T>>>
concept formattable-with = // exposition only
 semiregular<Formatter> &&
 requires(Formatter& f, const Formatter& cf, T&& t, Context fc,
 basic_format_parse_context<typename Context::char_type> pc)
{
 { f.parse(pc) } -> same_as<typename decltype(pc)::iterator>;
 { cf.format(t, fc) } -> same_as<typename Context::iterator>;
};

template<class T, class charT>
concept formattable =
 formattable-with<remove_reference_t<T>, basic_format_context<fmt-iter-for<charT>, charT>>>;
```

- <sup>2</sup> A type `T` and a character type `charT` model *formattable* if `formatter<remove_cvref_t<T>, charT>` meets the *BasicFormatter* requirements (28.5.6.1) and, if `remove_reference_t<T>` is const-qualified, the *Formatter* requirements.

## 28.5.6.4 Formatter specializations

[format.formatter.spec]

- <sup>1</sup> The functions defined in 28.5.5 use specializations of the class template `formatter` to format individual arguments.
- <sup>2</sup> Let `charT` be either `char` or `wchar_t`. Each specialization of `formatter` is either enabled or disabled, as described below. A *debug-enabled* specialization of `formatter` additionally provides a public, `constexpr`, non-static member function `set_debug_format()` which modifies the state of the `formatter` to be as if the type of the *std-format-spec* parsed by the last call to `parse` were `?`. A *constexpr-enabled* specialization of `formatter` has its `format` member function declared `constexpr`. Each header that declares the template `formatter` provides the following enabled specializations:
  - (2.1) — The debug-enabled and `constexpr`-enabled specializations
 

```
template<> struct formatter<char, char>;
template<> struct formatter<char, wchar_t>;
template<> struct formatter<wchar_t, wchar_t>;
```
  - (2.2) — For each `charT`, the debug-enabled and `constexpr`-enabled string type specializations
 

```
template<> struct formatter<charT*, charT>;
template<> struct formatter<const charT*, charT>;
template<size_t N> struct formatter<charT[N], charT>;
template<class traits, class Allocator>
 struct formatter<basic_string<charT, traits, Allocator>, charT>;
template<class traits>
 struct formatter<basic_string_view<charT, traits>, charT>;
```
  - (2.3) — For each `charT`, for each `IntegerT` that is either a signed or unsigned integer type or `bool`, a `constexpr`-enabled specialization
 

```
template<> struct formatter<IntegerT, charT>;
```
  - (2.4) — For each `charT`, for each `FloatingT` that is a cv-unqualified floating-point type, a specialization
 

```
template<> struct formatter<FloatingT, charT>;
```
  - (2.5) — For each `charT`, the `constexpr`-enabled pointer type specialization
 

```
template<> struct formatter<nullptr_t, charT>;
```
  - (2.6) — For each `charT`, the pointer type specializations
 

```
template<> struct formatter<void*, charT>;
template<> struct formatter<const void*, charT>;
```

The `parse` member functions of these formatters interpret the format specification as a *std-format-spec* as described in 28.5.2.2.

- <sup>3</sup> Unless specified otherwise, for each type `T` for which a `formatter` specialization is provided by the library, each of the headers provides the following specialization:
 

```
template<> inline constexpr bool enable_nonlocking_formatter_optimization<T> = true;
```

[Note 1: Specializations such as `formatter<wchar_t, char>` that would require implicit multibyte / wide string or character conversion are disabled. — end note]
- <sup>4</sup> The header `<format>` provides the following disabled specializations:
  - (4.1) — The string type specializations
 

```
template<> struct formatter<char*, wchar_t>;
template<> struct formatter<const char*, wchar_t>;
template<size_t N> struct formatter<char[N], wchar_t>;
template<class traits, class Allocator>
 struct formatter<basic_string<char, traits, Allocator>, wchar_t>;
template<class traits>
 struct formatter<basic_string_view<char, traits>, wchar_t>;
```
- <sup>5</sup> For any types `T` and `charT` for which neither the library nor the user provides an explicit or partial specialization of the class template `formatter`, `formatter<T, charT>` is disabled.
- <sup>6</sup> If the library provides an explicit or partial specialization of `formatter<T, charT>`, that specialization is enabled and meets the *Formatter* requirements except as noted otherwise.
- <sup>7</sup> If `F` is a disabled specialization of `formatter`, these values are `false`:

- (7.1) — `is_default_constructible_v<F>`,
- (7.2) — `is_copy_constructible_v<F>`,
- (7.3) — `is_move_constructible_v<F>`,
- (7.4) — `is_copy_assignable_v<F>`, and
- (7.5) — `is_move_assignable_v<F>`.

<sup>8</sup> An enabled specialization `formatter<T, charT>` meets the *BasicFormatter* requirements (28.5.6.1).

[Example 1:

```
#include <format>
#include <string>

enum color { red, green, blue };
const char* color_names[] = { "red", "green", "blue" };

template<> struct std::formatter<color> : std::formatter<const char*> {
 auto format(color c, format_context& ctx) const {
 return formatter<const char*>::format(color_names[c], ctx);
 }
};

struct err {};

std::string s0 = std::format("{} ", 42); // OK, library-provided formatter
std::string s1 = std::format("{} ", L"foo"); // error: disabled formatter
std::string s2 = std::format("{} ", red); // OK, user-provided formatter
std::string s3 = std::format("{} ", err{}); // error: disabled formatter
```

— end example]

#### 28.5.6.5 Formatting escaped characters and strings [format.string.escaped]

- <sup>1</sup> A character or string can be formatted as *escaped* to make it more suitable for debugging or for logging.
- <sup>2</sup> The escaped string *E* representation of a string *S* is constructed by encoding a sequence of characters as follows. The associated character encoding *CE* for `charT` (Table 12) is used to both interpret *S* and construct *E*.

- (2.1) — U+0022 QUOTATION MARK (") is appended to *E*.
- (2.2) — For each code unit sequence *X* in *S* that either encodes a single character, is a shift sequence, or is a sequence of ill-formed code units, processing is in order as follows:
  - (2.2.1) — If *X* encodes a single character *C*, then:
    - (2.2.1.1) — If *C* is one of the characters in Table 114, then the two characters shown as the corresponding escape sequence are appended to *E*.
    - (2.2.1.2) — Otherwise, if *C* is not U+0020 SPACE and
      - (2.2.1.2) — *CE* is UTF-8, UTF-16, or UTF-32 and *C* corresponds to a Unicode scalar value whose Unicode property **General\_Category** has a value in the groups **Separator** (Z) or **Other** (C), as described by UAX #44 of the Unicode Standard, or
      - (2.2.1.2) — *CE* is UTF-8, UTF-16, or UTF-32 and *C* corresponds to a Unicode scalar value with the Unicode property **Grapheme\_Extend=Yes** as described by UAX #44 of the Unicode Standard and *C* is not immediately preceded in *S* by a character *P* appended to *E* without translation to an escape sequence, or
      - (2.2.1.2) — *CE* is neither UTF-8, UTF-16, nor UTF-32 and *C* is one of an implementation-defined set of separator or non-printable characters
  - then the sequence `\u{hex-digit-sequence}` is appended to *E*, where *hex-digit-sequence* is the shortest hexadecimal representation of *C* using lower-case hexadecimal digits.
  - (2.2.1.3) — Otherwise, *C* is appended to *E*.
  - (2.2.2) — Otherwise, if *X* is a shift sequence, the effect on *E* and further decoding of *S* is unspecified.



*Recommended practice:* A shift sequence should be represented in  $E$  such that the original code unit sequence of  $S$  can be reconstructed.

- (2.2.3) — Otherwise ( $X$  is a sequence of ill-formed code units), each code unit  $U$  is appended to  $E$  in order as the sequence `\x{hex-digit-sequence}`, where *hex-digit-sequence* is the shortest hexadecimal representation of  $U$  using lower-case hexadecimal digits.
- (2.3) — Finally, U+0022 QUOTATION MARK (") is appended to  $E$ .

**Table 114 — Mapping of characters to escape sequences** [tab:format.escape.sequences]

| Character                   | Escape sequence |
|-----------------------------|-----------------|
| U+0009 CHARACTER TABULATION | <code>\t</code> |
| U+000A LINE FEED            | <code>\n</code> |
| U+000D CARRIAGE RETURN      | <code>\r</code> |
| U+0022 QUOTATION MARK       | <code>\"</code> |
| U+005C REVERSE SOLIDUS      | <code>\\</code> |

<sup>3</sup> The escaped string representation of a character  $C$  is equivalent to the escaped string representation of a string of  $C$ , except that:

- (3.1) — the result starts and ends with U+0027 APOSTROPHE (') instead of U+0022 QUOTATION MARK ("), and
- (3.2) — if  $C$  is U+0027 APOSTROPHE, the two characters `\'` are appended to  $E$ , and
- (3.3) — if  $C$  is U+0022 QUOTATION MARK, then  $C$  is appended unchanged.

[Example 1:

```
string s0 = format("[{}]", "h\tllo"); // s0 has value: [h llo]
string s1 = format("[{:?}]", "h\tllo"); // s1 has value: ["h\tllo"]
string s2 = format("[{:?}]", "Спасибо, Виктор ♥!"); // s2 has value: ["Спасибо, Виктор ♥!"]
string s3 = format("[{:?}, {:?}]", '\'', "'"); // s3 has value: ['\'', '\']

// The following examples assume use of the UTF-8 encoding
string s4 = format("[{:?}]", string("\0 \n \t \x02 \x1b", 9));
// s4 has value: ["\u{0} \n \t \u{2} \u{1b}"]

string s5 = format("[{:?}]", "\xc3\x28"); // invalid UTF-8, s5 has value: ["\xc3"]
string s6 = format("[{:?}]", "👨"); // s6 has value: ["\u{200d}♂"]
string s7 = format("[{:?}]", "\u0301"); // s7 has value: ["\u{301}"]
string s8 = format("[{:?}]", "\\u0301"); // s8 has value: ["\\u{301}"]
string s9 = format("[{:?}]", "e\u0301\u0323"); // s9 has value: ["ê"]
```

— end example]

#### 28.5.6.6 Class template basic\_format\_parse\_context

[format.parse.ctx]

```
namespace std {
 template<class charT>
 class basic_format_parse_context {
 public:
 using char_type = charT;
 using const_iterator = basic_string_view<charT>::const_iterator;
 using iterator = const_iterator;

 private:
 iterator begin_; // exposition only
 iterator end_; // exposition only
 enum indexing { unknown, manual, automatic }; // exposition only
 indexing indexing_; // exposition only
 size_t next_arg_id_; // exposition only
 size_t num_args_; // exposition only

 public:
 constexpr explicit basic_format_parse_context(basic_string_view<charT> fmt) noexcept;
 basic_format_parse_context(const basic_format_parse_context&) = delete;
 basic_format_parse_context& operator=(const basic_format_parse_context&) = delete;
```



```

constexpr const_iterator begin() const noexcept;
constexpr const_iterator end() const noexcept;
constexpr void advance_to(const_iterator it);

constexpr size_t next_arg_id();
constexpr void check_arg_id(size_t id);

template<class... Ts>
 constexpr void check_dynamic_spec(size_t id) noexcept;
 constexpr void check_dynamic_spec_integral(size_t id) noexcept;
 constexpr void check_dynamic_spec_string(size_t id) noexcept;
};

```

- <sup>1</sup> An instance of `basic_format_parse_context` holds the format string parsing state, consisting of the format string range being parsed and the argument counter for automatic indexing.
- <sup>2</sup> If a program declares an explicit or partial specialization of `basic_format_parse_context`, the program is ill-formed, no diagnostic required.

```
constexpr explicit basic_format_parse_context(basic_string_view<charT> fmt) noexcept;
```

- <sup>3</sup> *Effects:* Initializes `begin_` with `fmt.begin()`, `end_` with `fmt.end()`, `indexing_` with `unknown`, `next_arg_id_` with 0, and `num_args_` with 0.

[*Note 1:* Any call to `next_arg_id`, `check_arg_id`, or `check_dynamic_spec` on an instance of `basic_format_parse_context` initialized using this constructor is not a core constant expression. — *end note*]

```
constexpr const_iterator begin() const noexcept;
```

- <sup>4</sup> *Returns:* `begin_`.

```
constexpr const_iterator end() const noexcept;
```

- <sup>5</sup> *Returns:* `end_`.

```
constexpr void advance_to(const_iterator it);
```

- <sup>6</sup> *Preconditions:* `end()` is reachable from `it`.

- <sup>7</sup> *Effects:* Equivalent to: `begin_ = it;`

```
constexpr size_t next_arg_id();
```

- <sup>8</sup> *Effects:* If `indexing_ != manual` is true, equivalent to:

```

 if (indexing_ == unknown)
 indexing_ = automatic;
 return next_arg_id++;

```

- <sup>9</sup> *Throws:* `format_error` if `indexing_ == manual` is true.

[*Note 2:* This indicates mixing of automatic and manual argument indexing. — *end note*]

- <sup>10</sup> *Remarks:* Let *cur-arg-id* be the value of `next_arg_id_` prior to this call. Call expressions where *cur-arg-id*  $\geq$  `num_args_` is true are not core constant expressions (7.7.2).

```
constexpr void check_arg_id(size_t id);
```

- <sup>11</sup> *Effects:* If `indexing_ != automatic` is true, equivalent to:

```

 if (indexing_ == unknown)
 indexing_ = manual;

```

- <sup>12</sup> *Throws:* `format_error` if `indexing_ == automatic` is true.

[*Note 3:* This indicates mixing of automatic and manual argument indexing. — *end note*]

- <sup>13</sup> *Remarks:* A call to this function is a core constant expression (7.7.2) only if `id < num_args_` is true.

```
template<class... Ts>
```

```
 constexpr void check_dynamic_spec(size_t id) noexcept;
```

- <sup>14</sup> *Mandates:* `sizeof...(Ts)  $\geq$  1`. The types in `Ts...` are unique. Each type in `Ts...` is one of `bool`, `char_type`, `int`, `unsigned int`, `long long int`, `unsigned long long int`, `float`, `double`, `long`

double, const char\_type\*, basic\_string\_view<char\_type>, or const void\*.

15 *Remarks:* A call to this function is a core constant expression only if

(15.1) — id < num\_args\_ is true and

(15.2) — the type of the corresponding format argument (after conversion to basic\_format\_arg<Context>) is one of the types in Ts....

```
constexpr void check_dynamic_spec_integral(size_t id) noexcept;
```

16 *Effects:* Equivalent to:

```
check_dynamic_spec<int, unsigned int, long long int, unsigned long long int>(id);
```

```
constexpr void check_dynamic_spec_string(size_t id) noexcept;
```

17 *Effects:* Equivalent to:

```
check_dynamic_spec<const char_type*, basic_string_view<char_type>>(id);
```

### 28.5.6.7 Class template basic\_format\_context

[format.context]

```
namespace std {
 template<class Out, class charT>
 class basic_format_context {
 basic_format_args<basic_format_context> args_; // exposition only
 Out out_; // exposition only

 basic_format_context(const basic_format_context&) = delete;
 basic_format_context& operator=(const basic_format_context&) = delete;

 public:
 using iterator = Out;
 using char_type = charT;
 template<class T> using formatter_type = formatter<T, charT>;

 constexpr basic_format_arg<basic_format_context> arg(size_t id) const noexcept;
 std::locale locale();

 constexpr iterator out();
 constexpr void advance_to(iterator it);
 };
}
```

1 An instance of `basic_format_context` holds formatting state consisting of the formatting arguments and the output iterator.

2 If a program declares an explicit or partial specialization of `basic_format_context`, the program is ill-formed, no diagnostic required.

3 `Out` shall model `output_iterator<const charT&>`.

4 `format_context` is an alias for a specialization of `basic_format_context` with an output iterator that appends to `string`, such as `back_insert_iterator<string>`. Similarly, `wformat_context` is an alias for a specialization of `basic_format_context` with an output iterator that appends to `wstring`.

5 *Recommended practice:* For a given type `charT`, implementations should provide a single specialization of `basic_format_context` for appending to `basic_string<charT>`, `vector<charT>`, or any other container with contiguous storage by wrapping those in temporary objects with a uniform interface (such as a `span<charT>`) and polymorphic reallocation.

```
constexpr basic_format_arg<basic_format_context> arg(size_t id) const noexcept;
```

6 *Returns:* `args_.get(id)`.

```
std::locale locale();
```

7 *Returns:* The locale passed to the formatting function if the latter takes one, and `std::locale()` otherwise.

```
constexpr iterator out();
```

8     *Effects:* Equivalent to: `return std::move(out_);`

```
constexpr void advance_to(iterator it);
```

9     *Effects:* Equivalent to: `out_ = std::move(it);`

[Example 1:

```
struct S { int value; };
```

```
template<> struct std::formatter<S> {
 size_t width_arg_id = 0;
```

```
 // Parses a width argument id in the format { digit }.
 constexpr auto parse(format_parse_context& ctx) {
```

```
 auto iter = ctx.begin();
 auto is_digit = [](auto c) { return c >= '0' && c <= '9'; };
 auto get_char = [&]() { return iter != ctx.end() ? *iter : 0; };
 if (get_char() != '{')
 return iter;
 ++iter;
 char c = get_char();
 if (!is_digit(c) || (++iter, get_char()) != '}')
 throw format_error("invalid format");
 width_arg_id = c - '0';
 ctx.check_arg_id(width_arg_id);
 return ++iter;
 }
```

```
 // Formats an S with width given by the argument width_arg_id.
```

```
 auto format(S s, format_context& ctx) const {
 int width = ctx.arg(width_arg_id).visit([](auto value) -> int {
 if constexpr (!is_integral_v<decltype(value)>))
 throw format_error("width is not integral");
 else if (value < 0 || value > numeric_limits<int>::max())
 throw format_error("invalid width");
 else
 return value;
 });
 return format_to(ctx.out(), "{0:x}>{1}}", s.value, width);
 }
};
```

```
std::string s = std::format("{0:{1}}", S{42}, 10); // value of s is "xxxxxxx42"
```

— end example]

## 28.5.7 Formatting of ranges

[format.range]

### 28.5.7.1 Variable template `format_kind`

[format.range.fmtkind]

```
template<ranges::input_range R>
requires same_as<R, remove_cvref_t<R>>
constexpr range_format format_kind<R> = see below;
```

1     A program that instantiates the primary template of `format_kind` is ill-formed.

2     For a type `R`, `format_kind<R>` is defined as follows:

- (2.1) — If `same_as<remove_cvref_t<ranges::range_reference_t<R>>, R>` is true, `format_kind<R>` is `range_format::disabled`.

[Note 1: This prevents constraint recursion for ranges whose reference type is the same range type. For example, `std::filesystem::path` is a range of `std::filesystem::path`. — end note]

- (2.2) — Otherwise, if the *qualified-id* `R::key_type` is valid and denotes a type:

- (2.2.1) — If the *qualified-id* `R::mapped_type` is valid and denotes a type, let `U` be `remove_cvref_t<ranges::range_reference_t<R>>`. If either `U` is a specialization of `pair` or `U` is a specialization of `tuple` and `tuple_size_v<U> == 2`, `format_kind<R>` is `range_format::map`.
- (2.2.2) — Otherwise, `format_kind<R>` is `range_format::set`.
- (2.3) — Otherwise, `format_kind<R>` is `range_format::sequence`.
- 3 *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `format_kind` for cv-unqualified program-defined types that model `ranges::input_range`. Such specializations shall be usable in constant expressions (7.7.4) and have type `const range_format`.

### 28.5.7.2 Class template `range_formatter`

[`format.range.formatter`]

```
namespace std {
 template<class T, class charT = char>
 requires same_as<remove_cvref_t<T>, T> && formattable<T, charT>
 class range_formatter {
 formatter<T, charT> underlying_; // exposition only
 basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", "); // exposition only
 basic_string_view<charT> opening-bracket_ = STATICALLY-WIDEN<charT>("["); // exposition only
 basic_string_view<charT> closing-bracket_ = STATICALLY-WIDEN<charT>("]"); // exposition only

 public:
 constexpr void set_separator(basic_string_view<charT> sep) noexcept;
 constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;
 constexpr formatter<T, charT>& underlying() noexcept { return underlying_; }
 constexpr const formatter<T, charT>& underlying() const noexcept { return underlying_; }

 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<ranges::input_range R, class FormatContext>
 requires formattable<ranges::range_reference_t<R>, charT> &&
 same_as<remove_cvref_t<ranges::range_reference_t<R>>, T>
 constexpr typename FormatContext::iterator
 format(R&& r, FormatContext& ctx) const;
 };
}
```

- 1 The class template `range_formatter` is a utility for implementing `formatter` specializations for range types.
- 2 `range_formatter` interprets *format-spec* as a *range-format-spec*. The syntax of format specifications is as follows:

```
range-format-spec:
 range-fill-and-alignopt widthopt nopt range-typeopt range-underlying-specopt

range-fill-and-align:
 range-fillopt align

range-fill:
 any character other than { or } or :

range-type:
 m
 s
 ?s

range-underlying-spec:
 : format-spec
```

- 3 For `range_formatter<T, charT>`, the *format-spec* in a *range-underlying-spec*, if any, is interpreted by `formatter<T, charT>`.
- 4 The *range-fill-and-align* is interpreted the same way as a *fill-and-align* (28.5.2.2). The productions *align* and *width* are described in 28.5.2.
- 5 The *n* option causes the range to be formatted without the opening and closing brackets.

[Note 1: This is equivalent to invoking `set_brackets({}, {})`. — end note]

- <sup>6</sup> The *range-type* specifier changes the way a range is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table 115.

Table 115 — Meaning of *range-type* options [tab:formatter.range.type]

| Option    | Requirements                                                                                                                                         | Meaning                                                                                                                                                                                                                                                                                                                                                                                         |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>m</b>  | T shall be either a specialization of <code>pair</code> or a specialization of <code>tuple</code> such that <code>tuple_size_v&lt;T&gt;</code> is 2. | Indicates that the opening bracket should be "{", the closing bracket should be "}", the separator should be ", ", and each range element should be formatted as if <code>m</code> were specified for its <i>tuple-type</i> .<br>[Note 2: If the <code>n</code> option is provided in addition to the <code>m</code> option, both the opening and closing brackets are still empty. — end note] |
| <b>s</b>  | T shall be <code>charT</code> .                                                                                                                      | Indicates that the range should be formatted as a <b>string</b> .                                                                                                                                                                                                                                                                                                                               |
| <b>?s</b> | T shall be <code>charT</code> .                                                                                                                      | Indicates that the range should be formatted as an escaped string (28.5.6.5).                                                                                                                                                                                                                                                                                                                   |

If the *range-type* is `s` or `?s`, then there shall be no `n` option and no *range-underlying-spec*.

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

- <sup>7</sup> *Effects*: Equivalent to: `separator_ = sep;`

```
constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;
```

- <sup>8</sup> *Effects*: Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;
```

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

- <sup>9</sup> *Effects*: Parses the format specifiers as a *range-format-spec* and stores the parsed specifiers in `*this`. Calls `underlying_.parse(ctx)` to parse *format-spec* in *range-format-spec* or, if the latter is not present, an empty *format-spec*. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *range-type* or the `n` option, if present. If:

- (9.1) — the *range-type* is neither `s` nor `?s`,
- (9.2) — `underlying_.set_debug_format()` is a valid expression, and
- (9.3) — there is no *range-underlying-spec*,

then calls `underlying_.set_debug_format()`.

- <sup>10</sup> *Returns*: An iterator past the end of the *range-format-spec*.

```
template<ranges::input_range R, class FormatContext>
requires formattable<ranges::range_reference_t<R>, charT> &&
 same_as<remove_cvref_t<ranges::range_reference_t<R>>, T>
constexpr typename FormatContext::iterator
format(R&& r, FormatContext& ctx) const;
```

- <sup>11</sup> *Effects*: Writes the following into `ctx.out()`, adjusted according to the *range-format-spec*:

- (11.1) — If the *range-type* was `s`, then as if by formatting `basic_string<charT>(from_range, r)`.
- (11.2) — Otherwise, if the *range-type* was `?s`, then as if by formatting `basic_string<charT>(from_range, r)` as an escaped string (28.5.6.5).
- (11.3) — Otherwise,

- (11.3.1) — *opening-bracket\_*,
- (11.3.2) — for each element *e* of the range *r*:
  - (11.3.2.1) — the result of writing *e* via *underlying\_* and
  - (11.3.2.2) — *separator\_*, unless *e* is the last element of *r*, and
  - (11.3.3) — *closing-bracket\_*.

12 *Returns*: An iterator past the end of the output range.

### 28.5.7.3 Class template *range-default-formatter*

[format.range.fmtdef]

```
namespace std {
 template<ranges::input_range R, class charT>
 struct range-default-formatter<range_format::sequence, R, charT> { // exposition only
 private:
 using maybe-const-r = fmt-maybe-const<R, charT>; // exposition only
 range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-r>>,
 charT> underlying_; // exposition only

 public:
 constexpr void set_separator(basic_string_view<charT> sep) noexcept;
 constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;

 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<class FormatContext>
 constexpr typename FormatContext::iterator
 format(maybe-const-r& elems, FormatContext& ctx) const;
 };
}
```

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

1 *Effects*: Equivalent to: *underlying\_.set\_separator(sep)*;

```
constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;
```

2 *Effects*: Equivalent to: *underlying\_.set\_brackets(opening, closing)*;

```
template<class ParseContext>
constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);
```

3 *Effects*: Equivalent to: return *underlying\_.parse(ctx)*;

```
template<class FormatContext>
constexpr typename FormatContext::iterator
 format(maybe-const-r& elems, FormatContext& ctx) const;
```

4 *Effects*: Equivalent to: return *underlying\_.format(elems, ctx)*;

### 28.5.7.4 Specialization of *range-default-formatter* for maps

[format.range.fmtmap]

```
namespace std {
 template<range::input_range R, class charT>
 struct range-default-formatter<range_format::map, R, charT> {
 private:
 using maybe-const-map = fmt-maybe-const<R, charT>; // exposition only
 using element-type = // exposition only
 remove_cvref_t<ranges::range_reference_t<maybe-const-map>>;
 range_formatter<element-type, charT> underlying_; // exposition only

 public:
 constexpr range-default-formatter();
 };
}
```

```

template<class ParseContext>
constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

template<class FormatContext>
constexpr typename FormatContext::iterator
 format(maybe-const-map& r, FormatContext& ctx) const;
};
}

```

```
constexpr range-default-formatter();
```

1 *Mandates:* Either:

- (1.1) — *element-type* is a specialization of `pair`, or
- (1.2) — *element-type* is a specialization of `tuple` and `tuple_size_v<element-type> == 2`.

2 *Effects:* Equivalent to:

```

 underlying_.set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-WIDEN<charT>("}"));
 underlying_.underlying().set_brackets({}, {});
 underlying_.underlying().set_separator(STATICALLY-WIDEN<charT>(": "));

```

```

template<class ParseContext>
constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

```

3 *Effects:* Equivalent to: return `underlying_.parse(ctx)`;

```

template<class FormatContext>
constexpr typename FormatContext::iterator
 format(maybe-const-map& r, FormatContext& ctx) const;

```

4 *Effects:* Equivalent to: return `underlying_.format(r, ctx)`;

#### 28.5.7.5 Specialization of `range-default-formatter` for sets

[format.range.fmtset]

```

namespace std {
 template<ranges::input_range R, class charT>
 struct range-default-formatter<range_format::set, R, charT> {
 private:
 using maybe-const-set = fmt-maybe-const<R, charT>; // exposition only
 range_formatter<remove_cvref_t<ranges::range_reference_t<maybe-const-set>>,
 charT> underlying_; // exposition only

 public:
 constexpr range-default-formatter();

 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<class FormatContext>
 constexpr typename FormatContext::iterator
 format(maybe-const-set& r, FormatContext& ctx) const;
 };
}

```

```
constexpr range-default-formatter();
```

1 *Effects:* Equivalent to:

```

 underlying_.set_brackets(STATICALLY-WIDEN<charT>("{"), STATICALLY-WIDEN<charT>("}"));

```

```

template<class ParseContext>
constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

```

2 *Effects:* Equivalent to: return `underlying_.parse(ctx)`;

```
template<class FormatContext>
constexpr typename FormatContext::iterator
format(maybe-const-set& r, FormatContext& ctx) const;
```

3 *Effects:* Equivalent to: return *underlying\_.format*(r, ctx);

### 28.5.7.6 Specialization of *range-default-formatter* for strings [format.range.fmtstr]

```
namespace std {
 template<range_format K, ranges::input_range R, class charT>
 requires (K == range_format::string || K == range_format::debug_string)
 struct range-default-formatter<K, R, charT> {
 private:
 formatter<basic_string<charT>, charT> underlying_; // exposition only

 public:
 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<class FormatContext>
 constexpr typename FormatContext::iterator
 format(see below& str, FormatContext& ctx) const;
 };
 }
```

1 *Mandates:* *same\_as*<*remove\_cvref\_t*<*range\_reference\_t*<R>>, charT> is true.

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

2 *Effects:* Equivalent to:

```
 auto i = underlying_.parse(ctx);
 if constexpr (K == range_format::debug_string) {
 underlying_.set_debug_format();
 }
 return i;
```

```
template<class FormatContext>
typename FormatContext::iterator
format(see below& r, FormatContext& ctx) const;
```

3 The type of r is const R& if *ranges::input\_range*<const R> is true and R& otherwise.

4 *Effects:* Let s be a *basic\_string*<charT> such that *ranges::equal*(s, r) is true. Equivalent to: return *underlying\_.format*(s, ctx);

## 28.5.8 Arguments [format.arguments]

### 28.5.8.1 Class template *basic\_format\_arg* [format.arg]

```
namespace std {
 template<class Context>
 class basic_format_arg {
 public:
 class handle;

 private:
 using char-type = Context::char_type; // exposition only

 variant<monostate, bool, char-type,
 int, unsigned int, long long int, unsigned long long int,
 float, double, long double,
 const char-type*, basic_string_view<char-type>,
 const void*, handle> value_; // exposition only

 template<class T> constexpr explicit basic_format_arg(T& v) noexcept; // exposition only
 }
```



```

public:
 constexpr basic_format_arg() noexcept;

 constexpr explicit operator bool() const noexcept;

 template<class Visitor>
 constexpr decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);
 template<class R, class Visitor>
 constexpr R visit(this basic_format_arg arg, Visitor&& vis);
};
}

```

<sup>1</sup> An instance of `basic_format_arg` provides access to a formatting argument for user-defined formatters.

<sup>2</sup> The behavior of a program that adds specializations of `basic_format_arg` is undefined.

```
constexpr basic_format_arg() noexcept;
```

<sup>3</sup> *Postconditions:* `!(*this)`.

```
template<class T> constexpr explicit basic_format_arg(T& v) noexcept;
```

<sup>4</sup> *Constraints:* `T` satisfies *formattable-with*<`Context`>.

<sup>5</sup> *Preconditions:* If `decay_t<T>` is *char-type\** or *const char-type\**, `static_cast<const char-type*>(v)` points to an NTCTS (3.36).

<sup>6</sup> *Effects:* Let `TD` be `remove_const_t<T>`.

- (6.1) — If `TD` is `bool` or *char-type*, initializes *value\_* with `v`;
- (6.2) — otherwise, if `TD` is `char` and *char-type* is `wchar_t`, initializes *value\_* with `static_cast<wchar_t>(static_cast<unsigned char>(v))`;
- (6.3) — otherwise, if `TD` is a signed integer type (6.9.3) and `sizeof(TD) <= sizeof(int)`, initializes *value\_* with `static_cast<int>(v)`;
- (6.4) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned int)`, initializes *value\_* with `static_cast<unsigned int>(v)`;
- (6.5) — otherwise, if `TD` is a signed integer type and `sizeof(TD) <= sizeof(long long int)`, initializes *value\_* with `static_cast<long long int>(v)`;
- (6.6) — otherwise, if `TD` is an unsigned integer type and `sizeof(TD) <= sizeof(unsigned long long int)`, initializes *value\_* with `static_cast<unsigned long long int>(v)`;
- (6.7) — otherwise, if `TD` is a standard floating-point type, initializes *value\_* with `v`;
- (6.8) — otherwise, if `TD` is a specialization of `basic_string_view` or `basic_string` and `TD::value_type` is *char-type*, initializes *value\_* with `basic_string_view<char-type>(v.data(), v.size())`;
- (6.9) — otherwise, if `decay_t<TD>` is *char-type\** or *const char-type\**, initializes *value\_* with `static_cast<const char-type*>(v)`;
- (6.10) — otherwise, if `is_void_v<remove_pointer_t<TD>>` is `true` or `is_null_pointer_v<TD>` is `true`, initializes *value\_* with `static_cast<const void*>(v)`;
- (6.11) — otherwise, initializes *value\_* with `handle(v)`.

[Note 1: Constructing `basic_format_arg` from a pointer to a member is ill-formed unless the user provides an enabled specialization of `formatter` for that pointer to member type. — end note]

```
constexpr explicit operator bool() const noexcept;
```

<sup>7</sup> *Returns:* `!holds_alternative<monostate>(value_)`.

```
template<class Visitor>
 constexpr decltype(auto) visit(this basic_format_arg arg, Visitor&& vis);
```

<sup>8</sup> *Effects:* Equivalent to: `return arg.value_.visit(std::forward<Visitor>(vis));`

```
template<class R, class Visitor>
 constexpr R visit(this basic_format_arg arg, Visitor&& vis);
```

<sup>9</sup> *Effects:* Equivalent to: `return arg.value_.visit<R>(std::forward<Visitor>(vis));`

10 The class `handle` allows formatting an object of a user-defined type.

```
namespace std {
 template<class Context>
 class basic_format_arg<Context>::handle {
 const void* ptr_; // exposition only
 void (*format_)(basic_format_parse_context<char-type>&,
 Context&, const void*); // exposition only

 template<class T> constexpr explicit handle(T& val) noexcept; // exposition only

 public:
 constexpr void format(basic_format_parse_context<char-type>&, Context& ctx) const;
 };
}
```

```
template<class T> constexpr explicit handle(T& val) noexcept;
```

11 Let

(11.1) — TD be `remove_const_t<T>`,

(11.2) — TQ be `const TD` if `const TD` satisfies *formattable-with*<Context> and TD otherwise.

12 *Mandates:* TQ satisfies *formattable-with*<Context>.

13 *Effects:* Initializes `ptr_` with `addressof(val)` and `format_` with

```
[] (basic_format_parse_context<char-type>& parse_ctx,
 Context& format_ctx, const void* ptr) {
 typename Context::template formatter_type<TD> f;
 parse_ctx.advance_to(f.parse(parse_ctx));
 format_ctx.advance_to(f.format(*const_cast<TQ*>(static_cast<const TD*>(ptr)),
 format_ctx));
}
```

```
constexpr void format(basic_format_parse_context<char-type>& parse_ctx, Context& format_ctx) const;
```

14 *Effects:* Equivalent to: `format_(parse_ctx, format_ctx, ptr_);`

### 28.5.8.2 Class template *format-arg-store*

[format.arg.store]

```
namespace std {
 template<class Context, class... Args>
 class format_arg_store { // exposition only
 array<basic_format_arg<Context>, sizeof...(Args)> args; // exposition only
 };
}
```

1 An instance of *format-arg-store* stores formatting arguments.

```
template<class Context = format_context, class... Args>
constexpr format_arg_store<Context, Args...>
make_format_args(Args&... fmt_args);
```

2 *Preconditions:* The type `typename Context::template formatter_type<remove_const_t<Ti>>` meets the *BasicFormatter* requirements (28.5.6.1) for each `Ti` in `Args`.

3 *Returns:* An object of type `format_arg_store<Context, Args...>` whose `args` data member is initialized with `{basic_format_arg<Context>(fmt_args)...}`.

```
template<class... Args>
constexpr format_arg_store<wformat_context, Args...>
make_wformat_args(Args&... args);
```

4 *Effects:* Equivalent to: `return make_format_args<wformat_context>(args...);`

### 28.5.8.3 Class template *basic\_format\_args*

[format.args]

```
namespace std {
 template<class Context>
 class basic_format_args {
 size_t size_; // exposition only
 };
}
```

```

 const basic_format_arg<Context>* data_; // exposition only

public:
 template<class... Args>
 constexpr basic_format_args(const format_arg_store<Context, Args...>& store) noexcept;

 constexpr basic_format_arg<Context> get(size_t i) const noexcept;
};

template<class Context, class... Args>
 basic_format_args(format_arg_store<Context, Args...>) -> basic_format_args<Context>;
}

```

<sup>1</sup> An instance of `basic_format_args` provides access to formatting arguments.

<sup>2</sup> *Recommended practice:* Implementations should optimize the representation of `basic_format_args` for a small number of formatting arguments.

[Note 1: For example, by storing indices of type alternatives separately from values and packing the former. — end note]

```

template<class... Args>
 constexpr basic_format_args(const format_arg_store<Context, Args...>& store) noexcept;

```

<sup>3</sup> *Effects:* Initializes `size_` with `sizeof...(Args)` and `data_` with `store.args.data()`.

```

constexpr basic_format_arg<Context> get(size_t i) const noexcept;

```

<sup>4</sup> *Returns:* `i < size_ ? data_[i] : basic_format_arg<Context>()`.

## 28.5.9 Tuple formatter

[format.tuple]

<sup>1</sup> For each of `pair` and `tuple`, the library provides the following formatter specialization where *pair-or-tuple* is the name of the template:

```

namespace std {
 template<class charT, formattable<charT>... Ts>
 struct formatter<pair-or-tuple<Ts...>, charT> {
 private:
 tuple<formatter<remove_cvref_t<Ts>, charT>...> underlying_; // exposition only
 basic_string_view<charT> separator_ = STATICALLY-WIDEN<charT>(", "); // exposition only
 basic_string_view<charT> opening_bracket_ = STATICALLY-WIDEN<charT>("("); // exposition only
 basic_string_view<charT> closing_bracket_ = STATICALLY-WIDEN<charT>(")"); // exposition only

 public:
 constexpr void set_separator(basic_string_view<charT> sep) noexcept;
 constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;

 template<class ParseContext>
 constexpr typename ParseContext::iterator
 parse(ParseContext& ctx);

 template<class FormatContext>
 constexpr typename FormatContext::iterator
 format(see below& elems, FormatContext& ctx) const;
 };

 template<class... Ts>
 constexpr bool enable_nonlocking_formatter_optimization<pair-or-tuple<Ts...>> =
 (enable_nonlocking_formatter_optimization<remove_cvref_t<Ts>> && ...);
}

```

<sup>2</sup> The `parse` member functions of these formatters interpret the format specification as a *tuple-format-spec* according to the following syntax:

```

tuple-format-spec:
 tuple-fill-and-alignopt widthopt tuple-typeopt

```

*tuple-fill-and-align*:

*tuple-fill*<sub>opt</sub> *align*

*tuple-fill*:

any character other than { or } or :

*tuple-type*:

*m*

*n*

- <sup>3</sup> The *tuple-fill-and-align* is interpreted the same way as a *fill-and-align* (28.5.2.2). The productions *align* and *width* are described in 28.5.2.
- <sup>4</sup> The *tuple-type* specifier changes the way a **pair** or **tuple** is formatted, with certain options only valid with certain argument types. The meaning of the various type options is as specified in Table 116.

**Table 116 — Meaning of *tuple-type* options** [tab:formatter.tuple.type]

| Option   | Requirements                    | Meaning                                                                                                                  |
|----------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>m</i> | <code>sizeof...(Ts) == 2</code> | Equivalent to:<br><code>set_separator(STATICALLY-WIDEN&lt;charT&gt;(": "));</code><br><code>set_brackets({}, {});</code> |
| <i>n</i> | none                            | Equivalent to: <code>set_brackets({}, {});</code>                                                                        |
| none     | none                            | No effects                                                                                                               |

```
constexpr void set_separator(basic_string_view<charT> sep) noexcept;
```

- <sup>5</sup> *Effects*: Equivalent to: `separator_ = sep;`

```
constexpr void set_brackets(basic_string_view<charT> opening,
 basic_string_view<charT> closing) noexcept;
```

- <sup>6</sup> *Effects*: Equivalent to:

```
opening-bracket_ = opening;
closing-bracket_ = closing;
```

```
template<class ParseContext>
constexpr typename ParseContext::iterator
parse(ParseContext& ctx);
```

- <sup>7</sup> *Effects*: Parses the format specifiers as a *tuple-format-spec* and stores the parsed specifiers in `*this`. The values of `opening-bracket_`, `closing-bracket_`, and `separator_` are modified if and only if required by the *tuple-type*, if present. For each element *e* in *underlying\_*, calls `e.parse(ctx)` to parse an empty *format-spec* and, if `e.set_debug_format()` is a valid expression, calls `e.set_debug_format()`.

- <sup>8</sup> *Returns*: An iterator past the end of the *tuple-format-spec*.

```
template<class FormatContext>
constexpr typename FormatContext::iterator
format(see below& elems, FormatContext& ctx) const;
```

- <sup>9</sup> The type of `elems` is:

(9.1) — If `(formattable<const Ts, charT> && ...)` is true, const *pair-or-tuple*<Ts...>&.

(9.2) — Otherwise *pair-or-tuple*<Ts...>&.

- <sup>10</sup> *Effects*: Writes the following into `ctx.out()`, adjusted according to the *tuple-format-spec*:

(10.1) — `opening-bracket_`,

(10.2) — for each index *I* in the `[0, sizeof...(Ts))`:

(10.2.1) — if `I != 0`, `separator_`,

(10.2.2) — the result of writing `get<I>(elems)` via `get<I>(underlying_)`, and

(10.3) — `closing-bracket_`.

- <sup>11</sup> *Returns*: An iterator past the end of the output range.

**28.5.10 Class `format_error`****[format.error]**

```

namespace std {
 class format_error : public runtime_error {
 public:
 constexpr explicit format_error(const string& what_arg);
 constexpr explicit format_error(const char* what_arg);
 };
}

```

- <sup>1</sup> The class `format_error` defines the type of objects thrown as exceptions to report errors from the formatting library.

```
constexpr format_error(const string& what_arg);
```

- <sup>2</sup> *Postconditions:* `strcmp(what(), what_arg.c_str()) == 0.`

```
constexpr format_error(const char* what_arg);
```

- <sup>3</sup> *Postconditions:* `strcmp(what(), what_arg) == 0.`

**28.6 Regular expressions library****[re]****28.6.1 General****[re.general]**

- <sup>1</sup> Subclause 28.6 describes components that C++ programs may use to perform operations involving regular expression matching and searching.
- <sup>2</sup> The following subclauses describe a basic regular expression class template and its traits that can handle `char`-like (27.1) template arguments, two specializations of this class template that handle sequences of `char` and `wchar_t`, a class template that holds the result of a regular expression match, a series of algorithms that allow a character sequence to be operated upon by a regular expression, and two iterator types for enumerating regular expression matches, as summarized in Table 117.

**Table 117 — Regular expressions library summary** **[tab:re.summary]**

|  | Subclause | Header                      |
|--|-----------|-----------------------------|
|  | 28.6.2    | Requirements                |
|  | 28.6.4    | Constants                   |
|  | 28.6.5    | Exception type              |
|  | 28.6.6    | Traits                      |
|  | 28.6.7    | Regular expression template |
|  | 28.6.8    | Submatches                  |
|  | 28.6.9    | Match results               |
|  | 28.6.10   | Algorithms                  |
|  | 28.6.11   | Iterators                   |
|  | 28.6.12   | Grammar                     |

- <sup>3</sup> The ECMAScript Language Specification described in Standard Ecma-262 is called *ECMA-262* in this Clause.

**28.6.2 Requirements****[re.req]**

- <sup>1</sup> This subclause defines requirements on classes representing regular expression traits.

[Note 1: The class template `regex_traits`, defined in 28.6.6, meets these requirements. — end note]

- <sup>2</sup> The class template `basic_regex`, defined in 28.6.7, needs a set of related types and functions to complete the definition of its semantics. These types and functions are provided as a set of member *typedef-names* and functions in the template parameter `traits` used by the `basic_regex` class template. This subclause defines the semantics of these members.

- <sup>3</sup> To specialize class template `basic_regex` for a character container `CharT` and its related regular expression traits class `Traits`, use `basic_regex<CharT, Traits>`.

- <sup>4</sup> In the following requirements,

(4.1) — **X** denotes a traits class defining types and functions for the character container type `charT`;

- (4.2) — `u` is an object of type `X`;
- (4.3) — `v` is an object of type `const X`;
- (4.4) — `p` is a value of type `const charT*`;
- (4.5) — `I1` and `I2` are input iterators (24.3.5.3);
- (4.6) — `F1` and `F2` are forward iterators (24.3.5.5);
- (4.7) — `c` is a value of type `const charT`;
- (4.8) — `s` is an object of type `X::string_type`;
- (4.9) — `cs` is an object of type `const X::string_type`;
- (4.10) — `b` is a value of type `bool`;
- (4.11) — `I` is a value of type `int`;
- (4.12) — `cl` is an object of type `X::char_class_type`; and
- (4.13) — `loc` is an object of type `X::locale_type`.

5 A traits class `X` meets the regular expression traits requirements if the following types and expressions are well-formed and have the specified semantics.

`typename X::char_type`

6 *Result:* `charT`, the character container type used in the implementation of class template `basic_regex`.

`typename X::string_type`

7 *Result:* `basic_string<charT>`

`typename X::locale_type`

8 *Result:* A copy constructible type that represents the locale used by the traits class.

`typename X::char_class_type`

9 *Result:* A bitmask type (16.3.3.3.3) representing a particular character classification.

`X::length(p)`

10 *Result:* `size_t`

11 *Returns:* The smallest `i` such that `p[i] == 0`.

12 *Complexity:* Linear in `i`.

`v.translate(c)`

13 *Result:* `X::char_type`

14 *Returns:* A character such that for any character `d` that is to be considered equivalent to `c` then `v.translate(c) == v.translate(d)`.

`v.translate_nocase(c)`

15 *Result:* `X::char_type`

16 *Returns:* For all characters `C` that are to be considered equivalent to `c` when comparisons are to be performed without regard to case, then `v.translate_nocase(c) == v.translate_nocase(C)`.

`v.transform(F1, F2)`

17 *Result:* `X::string_type`

18 *Returns:* A sort key for the character sequence designated by the iterator range `[F1,F2)` such that if the character sequence `[G1,G2)` sorts before the character sequence `[H1,H2)` then `v.transform(G1, G2) < v.transform(H1, H2)`.

`v.transform_primary(F1, F2)`

19 *Result:* `X::string_type`

20 *Returns:* A sort key for the character sequence designated by the iterator range `[F1, F2)` such that if the character sequence `[G1, G2)` sorts before the character sequence `[H1, H2)` when character case is not considered then `v.transform_primary(G1, G2) < v.transform_primary(H1, H2)`.

`v.lookup_collatename(F1, F2)`

21 *Result:* `X::string_type`

22 *Returns:* A sequence of characters that represents the collating element consisting of the character sequence designated by the iterator range `[F1, F2)`. Returns an empty string if the character sequence is not a valid collating element.

`v.lookup_classname(F1, F2, b)`

23 *Result:* `X::char_class_type`

24 *Returns:* Converts the character sequence designated by the iterator range `[F1, F2)` into a value of a bitmask type that can subsequently be passed to `isctype`. Values returned from `lookup_classname` can be bitwise OR'ed together; the resulting value represents membership in either of the corresponding character classes. If `b` is `true`, the returned bitmask is suitable for matching characters without regard to their case. Returns 0 if the character sequence is not the name of a character class recognized by `X`. The value returned shall be independent of the case of the characters in the sequence.

`v.isctype(c, cl)`

25 *Result:* `bool`

26 *Returns:* Returns `true` if character `c` is a member of one of the character classes designated by `cl`, `false` otherwise.

`v.value(c, I)`

27 *Result:* `int`

28 *Returns:* Returns the value represented by the digit `c` in base `I` if the character `c` is a valid digit in base `I`; otherwise returns `-1`.

[Note 2: The value of `I` will only be 8, 10, or 16. — end note]

`u.imbue(loc)`

29 *Result:* `X::locale_type`

30 *Effects:* Imbues `u` with the locale `loc` and returns the previous locale used by `u` if any.

`v.getloc()`

31 *Result:* `X::locale_type`

32 *Returns:* Returns the current locale used by `v`, if any.

33 [Note 3: Class template `regex_traits` meets the requirements for a regular expression traits class when it is specialized for `char` or `wchar_t`. This class template is described in the header `<regex>`, and is described in 28.6.6. — end note]

### 28.6.3 Header `<regex>` synopsis

[re.syn]

```
#include <compare> // see 17.12.1
#include <initializer_list> // see 17.11.2

namespace std {
 // 28.6.4, regex constants
 namespace regex_constants {
 using syntax_option_type = T1;
 using match_flag_type = T2;
 using error_type = T3;
 }

 // 28.6.5, class regex_error
 class regex_error;

 // 28.6.6, class template regex_traits
 template<class charT> struct regex_traits;
```

```

// 28.6.7, class template basic_regex
template<class charT, class traits = regex_traits<charT>> class basic_regex;

using regex = basic_regex<char>;
using wregex = basic_regex<wchar_t>;

// 28.6.7.6, basic_regex swap
template<class charT, class traits>
 void swap(basic_regex<charT, traits>& e1, basic_regex<charT, traits>& e2);

// 28.6.8, class template sub_match
template<class BidirectionalIterator>
 class sub_match;

using csub_match = sub_match<const char*>;
using wsub_match = sub_match<const wchar_t*>;
using ssub_match = sub_match<string::const_iterator>;
using wssub_match = sub_match<wstring::const_iterator>;

// 28.6.8.3, sub_match non-member operators
template<class BiIter>
 bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
template<class BiIter>
 auto operator<=>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);

template<class BiIter, class ST, class SA>
 bool operator==(
 const sub_match<BiIter>& lhs,
 const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template<class BiIter, class ST, class SA>
 auto operator<=>(
 const sub_match<BiIter>& lhs,
 const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

template<class BiIter>
 bool operator==(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type* rhs);
template<class BiIter>
 auto operator<=>(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type* rhs);

template<class BiIter>
 bool operator==(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type& rhs);
template<class BiIter>
 auto operator<=>(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type& rhs);

template<class charT, class ST, class BiIter>
 basic_ostream<charT, ST>&
 operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);

// 28.6.9, class template match_results
template<class BidirectionalIterator,
 class Allocator = allocator<sub_match<BidirectionalIterator>>>
 class match_results;

using cmatch = match_results<const char*>;
using wcmatch = match_results<const wchar_t*>;
using smatch = match_results<string::const_iterator>;
using wsmatch = match_results<wstring::const_iterator>;

```



```

// match_results comparisons
template<class BidirectionalIterator, class Allocator>
 bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
 const match_results<BidirectionalIterator, Allocator>& m2);

// 28.6.9.8, match_results swap
template<class BidirectionalIterator, class Allocator>
 void swap(match_results<BidirectionalIterator, Allocator>& m1,
 match_results<BidirectionalIterator, Allocator>& m2);

// 28.6.10.2, function template regex_match
template<class BidirectionalIterator, class Allocator, class charT, class traits>
 bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
 match_results<BidirectionalIterator, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
 bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class Allocator, class traits>
 bool regex_match(const charT* str, match_results<const charT*, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
 bool regex_match(const basic_string<charT, ST, SA>& s,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
 bool regex_match(const basic_string<charT, ST, SA>&&,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>&,
 const basic_regex<charT, traits>&,
 regex_constants::match_flag_type = regex_constants::match_default) = delete;
template<class charT, class traits>
 bool regex_match(const charT* str,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class charT, class traits>
 bool regex_match(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);

// 28.6.10.3, function template regex_search
template<class BidirectionalIterator, class Allocator, class charT, class traits>
 bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
 match_results<BidirectionalIterator, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class BidirectionalIterator, class charT, class traits>
 bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class Allocator, class traits>
 bool regex_search(const charT* str,
 match_results<const charT*, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class charT, class traits>
 bool regex_search(const charT* str,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);

```

```

template<class ST, class SA, class charT, class traits>
 bool regex_search(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
 bool regex_search(const basic_string<charT, ST, SA>& s,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class ST, class SA, class Allocator, class charT, class traits>
 bool regex_search(const basic_string<charT, ST, SA>&&,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>&,
 const basic_regex<charT, traits>&,
 regex_constants::match_flag_type
 = regex_constants::match_default) = delete;

// 28.6.10.4, function template regex_replace
template<class OutputIterator, class BidirectionalIterator,
 class traits, class charT, class ST, class SA>
 OutputIterator
 regex_replace(OutputIterator out,
 BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class OutputIterator, class BidirectionalIterator, class traits, class charT>
 OutputIterator
 regex_replace(OutputIterator out,
 BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA, class FST, class FSA>
 basic_string<charT, ST, SA>
 regex_replace(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, FST, FSA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA>
 basic_string<charT, ST, SA>
 regex_replace(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA>
 basic_string<charT>
 regex_replace(const charT* s,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT>
 basic_string<charT>
 regex_replace(const charT* s,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);

// 28.6.11.1, class template regex_iterator
template<class BidirectionalIterator,
 class charT = typename iterator_traits<BidirectionalIterator>::value_type,
 class traits = regex_traits<charT>>
 class regex_iterator;

```

```

using cregex_iterator = regex_iterator<const char*>;
using wcregex_iterator = regex_iterator<const wchar_t*>;
using sregex_iterator = regex_iterator<string::const_iterator>;
using wsregex_iterator = regex_iterator<wstring::const_iterator>;

// 28.6.11.2, class template regex_token_iterator
template<class BidirectionalIterator,
 class charT = typename iterator_traits<BidirectionalIterator>::value_type,
 class traits = regex_traits<charT>>
 class regex_token_iterator;

using cregex_token_iterator = regex_token_iterator<const char*>;
using wcregex_token_iterator = regex_token_iterator<const wchar_t*>;
using sregex_token_iterator = regex_token_iterator<string::const_iterator>;
using wsregex_token_iterator = regex_token_iterator<wstring::const_iterator>;

namespace pmr {
 template<class BidirectionalIterator>
 using match_results =
 std::match_results<BidirectionalIterator,
 polymorphic_allocator<sub_match<BidirectionalIterator>>>>;

 using cmatch = match_results<const char*>;
 using wcmatch = match_results<const wchar_t*>;
 using smatch = match_results<string::const_iterator>;
 using wsmatch = match_results<wstring::const_iterator>;
}
}

```

## 28.6.4 Namespace `std::regex_constants` [re.const]

### 28.6.4.1 General [re.const.general]

- <sup>1</sup> The namespace `std::regex_constants` holds symbolic constants used by the regular expression library. This namespace provides three types, `syntax_option_type`, `match_flag_type`, and `error_type`, along with several constants of these types.

### 28.6.4.2 Bitmask type `syntax_option_type` [re.synopt]

```

namespace std::regex_constants {
 using syntax_option_type = T1;
 inline constexpr syntax_option_type icafe = unspecified;
 inline constexpr syntax_option_type nosubs = unspecified;
 inline constexpr syntax_option_type optimize = unspecified;
 inline constexpr syntax_option_type collate = unspecified;
 inline constexpr syntax_option_type ECMAScript = unspecified;
 inline constexpr syntax_option_type basic = unspecified;
 inline constexpr syntax_option_type extended = unspecified;
 inline constexpr syntax_option_type awk = unspecified;
 inline constexpr syntax_option_type grep = unspecified;
 inline constexpr syntax_option_type egrep = unspecified;
 inline constexpr syntax_option_type multiline = unspecified;
}

```

- <sup>1</sup> The type `syntax_option_type` is an implementation-defined bitmask type (16.3.3.3.3). Setting its elements has the effects listed in Table 118. A valid value of type `syntax_option_type` shall have at most one of the grammar elements ECMAScript, basic, extended, awk, grep, egrep, set. If no grammar element is set, the default grammar is ECMAScript.

### 28.6.4.3 Bitmask type `match_flag_type` [re.matchflag]

```

namespace std::regex_constants {
 using match_flag_type = T2;
 inline constexpr match_flag_type match_default = {};
 inline constexpr match_flag_type match_not_bol = unspecified;
 inline constexpr match_flag_type match_not_eol = unspecified;
 inline constexpr match_flag_type match_not_bow = unspecified;
}

```

Table 118 — `syntax_option_type` effects [tab:re.synopt]

| Element                 | Effect(s) if set                                                                                                                                                                                                                                                          |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>icase</code>      | Specifies that matching of regular expressions against a character container sequence shall be performed without regard to case.                                                                                                                                          |
| <code>nosubs</code>     | Specifies that no sub-expressions shall be considered to be marked, so that when a regular expression is matched against a character container sequence, no sub-expression matches shall be stored in the supplied <code>match_results</code> object.                     |
| <code>optimize</code>   | Specifies that the regular expression engine should pay more attention to the speed with which regular expressions are matched, and less to the speed with which regular expression objects are constructed. Otherwise it has no detectable effect on the program output. |
| <code>collate</code>    | Specifies that character ranges of the form "[a-b]" shall be locale sensitive.                                                                                                                                                                                            |
| <code>ECMAScript</code> | Specifies that the grammar recognized by the regular expression engine shall be that used by ECMAScript in ECMA-262, as modified in 28.6.12.<br>SEE ALSO: ECMA-262 15.10                                                                                                  |
| <code>basic</code>      | Specifies that the grammar recognized by the regular expression engine shall be that used by basic regular expressions in POSIX.<br>SEE ALSO: POSIX, Base Definitions and Headers, Section 9.3                                                                            |
| <code>extended</code>   | Specifies that the grammar recognized by the regular expression engine shall be that used by extended regular expressions in POSIX.<br>SEE ALSO: POSIX, Base Definitions and Headers, Section 9.4                                                                         |
| <code>awk</code>        | Specifies that the grammar recognized by the regular expression engine shall be that used by the utility <code>awk</code> in POSIX.                                                                                                                                       |
| <code>grep</code>       | Specifies that the grammar recognized by the regular expression engine shall be that used by the utility <code>grep</code> in POSIX.                                                                                                                                      |
| <code>egrep</code>      | Specifies that the grammar recognized by the regular expression engine shall be that used by the utility <code>grep</code> when given the <code>-E</code> option in POSIX.                                                                                                |
| <code>multiline</code>  | Specifies that <code>^</code> shall match the beginning of a line and <code>\$</code> shall match the end of a line, if the <code>ECMAScript</code> engine is selected.                                                                                                   |

```

inline constexpr match_flag_type match_not_eow = unspecified;
inline constexpr match_flag_type match_any = unspecified;
inline constexpr match_flag_type match_not_null = unspecified;
inline constexpr match_flag_type match_continuous = unspecified;
inline constexpr match_flag_type match_prev_avail = unspecified;
inline constexpr match_flag_type format_default = {};
inline constexpr match_flag_type format_sed = unspecified;
inline constexpr match_flag_type format_no_copy = unspecified;
inline constexpr match_flag_type format_first_only = unspecified;
}

```

- <sup>1</sup> The type `match_flag_type` is an implementation-defined bitmask type (16.3.3.3.3). The constants of that type, except for `match_default` and `format_default`, are bitmask elements. The `match_default` and `format_default` constants are empty bitmasks. Matching a regular expression against a sequence of characters `[first, last)` proceeds according to the rules of the grammar specified for the regular expression object, modified according to the effects listed in Table 119 for any bitmask elements set.

Table 119 — `regex_constants::match_flag_type` effects [tab:re.matchflag]

| Element                    | Effect(s) if set                                                                                                                                                                                                                        |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>match_not_bol</code> | The first character in the sequence <code>[first, last)</code> shall be treated as though it is not at the beginning of a line, so the character <code>^</code> in the regular expression shall not match <code>[first, first)</code> . |
| <code>match_not_eol</code> | The last character in the sequence <code>[first, last)</code> shall be treated as though it is not at the end of a line, so the character <code>"\$"</code> in the regular expression shall not match <code>[last, last)</code> .       |

Table 119 — `regex_constants::match_flag_type` effects (continued)

| Element                        | Effect(s) if set                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>match_not_bow</code>     | The expression "\\b" shall not match the sub-sequence <code>[first, first)</code> .                                                                                                                                                                                                                                                                                                                                                                                         |
| <code>match_not_eow</code>     | The expression "\\b" shall not match the sub-sequence <code>[last, last)</code> .                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>match_any</code>         | If more than one match is possible then any match is an acceptable result.                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>match_not_null</code>    | The expression shall not match an empty sequence.                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>match_continuous</code>  | The expression shall only match a sub-sequence that begins at <code>first</code> .                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>match_prev_avail</code>  | <code>--first</code> is a valid iterator position. When this flag is set the flags <code>match_not_bol</code> and <code>match_not_bow</code> shall be ignored by the regular expression algorithms (28.6.10) and iterators (28.6.11).                                                                                                                                                                                                                                       |
| <code>format_default</code>    | When a regular expression match is to be replaced by a new string, the new string shall be constructed using the rules used by the ECMAScript replace function in ECMA-262, part 15.5.4.11 String.prototype.replace. In addition, during search and replace operations all non-overlapping occurrences of the regular expression shall be located and replaced, and sections of the input that did not match the expression shall be copied unchanged to the output string. |
| <code>format_sed</code>        | When a regular expression match is to be replaced by a new string, the new string shall be constructed using the rules used by the sed utility in POSIX.                                                                                                                                                                                                                                                                                                                    |
| <code>format_no_copy</code>    | During a search and replace operation, sections of the character container sequence being searched that do not match the regular expression shall not be copied to the output string.                                                                                                                                                                                                                                                                                       |
| <code>format_first_only</code> | When specified during a search and replace operation, only the first occurrence of the regular expression shall be replaced.                                                                                                                                                                                                                                                                                                                                                |

28.6.4.4 Implementation-defined `error_type`

[re.err]

```

namespace std::regex_constants {
 using error_type = T3;
 inline constexpr error_type error_collate = unspecified;
 inline constexpr error_type error_ctype = unspecified;
 inline constexpr error_type error_escape = unspecified;
 inline constexpr error_type error_backref = unspecified;
 inline constexpr error_type error_brack = unspecified;
 inline constexpr error_type error_paren = unspecified;
 inline constexpr error_type error_brace = unspecified;
 inline constexpr error_type error_badbrace = unspecified;
 inline constexpr error_type error_range = unspecified;
 inline constexpr error_type error_space = unspecified;
 inline constexpr error_type error_badrepeat = unspecified;
 inline constexpr error_type error_complexity = unspecified;
 inline constexpr error_type error_stack = unspecified;
}

```

- <sup>1</sup> The type `error_type` is an implementation-defined enumerated type (16.3.3.3.2). Values of type `error_type` represent the error conditions described in Table 120:

Table 120 — `error_type` values in the C locale [tab:re.err]

| Value                       | Error condition                                                             |
|-----------------------------|-----------------------------------------------------------------------------|
| <code>error_collate</code>  | The expression contains an invalid collating element name.                  |
| <code>error_ctype</code>    | The expression contains an invalid character class name.                    |
| <code>error_escape</code>   | The expression contains an invalid escaped character, or a trailing escape. |
| <code>error_backref</code>  | The expression contains an invalid back reference.                          |
| <code>error_brack</code>    | The expression contains mismatched <code>[</code> and <code>]</code> .      |
| <code>error_paren</code>    | The expression contains mismatched <code>(</code> and <code>)</code> .      |
| <code>error_brace</code>    | The expression contains mismatched <code>{</code> and <code>}</code> .      |
| <code>error_badbrace</code> | The expression contains an invalid range in a <code>{}</code> expression.   |

Table 120 — `error_type` values in the C locale (continued)

| Value                         | Error condition                                                                                                    |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>error_range</code>      | The expression contains an invalid character range, such as <code>[b-a]</code> in most encodings.                  |
| <code>error_space</code>      | There is insufficient memory to convert the expression into a finite state machine.                                |
| <code>error_badrepeat</code>  | One of <code>*?+{</code> is not preceded by a valid regular expression.                                            |
| <code>error_complexity</code> | The complexity of an attempted match against a regular expression exceeds a pre-set level.                         |
| <code>error_stack</code>      | There is insufficient memory to determine whether the regular expression matches the specified character sequence. |

**28.6.5 Class `regex_error`****[re.badexp]**

```
namespace std {
 class regex_error : public runtime_error {
 public:
 explicit regex_error(regex_constants::error_type ecode);
 regex_constants::error_type code() const;
 };
}
```

- <sup>1</sup> The class `regex_error` defines the type of objects thrown as exceptions to report errors from the regular expression library.

```
regex_error(regex_constants::error_type ecode);
```

- <sup>2</sup> *Postconditions:* `ecode == code()`.

```
regex_constants::error_type code() const;
```

- <sup>3</sup> *Returns:* The error code that was passed to the constructor.

**28.6.6 Class template `regex_traits`****[re.traits]**

```
namespace std {
 template<class charT>
 struct regex_traits {
 using char_type = charT;
 using string_type = basic_string<char_type>;
 using locale_type = locale;
 using char_class_type = bitmask_type;

 regex_traits();
 static size_t length(const char_type* p);
 charT translate(charT c) const;
 charT translate_nocase(charT c) const;
 template<class ForwardIterator>
 string_type transform(ForwardIterator first, ForwardIterator last) const;
 template<class ForwardIterator>
 string_type transform_primary(
 ForwardIterator first, ForwardIterator last) const;
 template<class ForwardIterator>
 string_type lookup_collatename(
 ForwardIterator first, ForwardIterator last) const;
 template<class ForwardIterator>
 char_class_type lookup_classname(
 ForwardIterator first, ForwardIterator last, bool icase = false) const;
 bool istype(charT c, char_class_type f) const;
 int value(charT ch, int radix) const;
 locale_type imbue(locale_type l);
 locale_type getloc() const;
 };
}
```

- <sup>1</sup> The specializations `regex_traits<char>` and `regex_traits<wchar_t>` meet the requirements for a regular expression traits class (28.6.2).

```
using char_class_type = bitmask_type;
```

- <sup>2</sup> The type `char_class_type` is used to represent a character classification and is capable of holding an implementation specific set returned by `lookup_classname`.

```
static size_t length(const char_type* p);
```

- <sup>3</sup> *Returns:* `char_traits<charT>::length(p)`.

```
charT translate(charT c) const;
```

- <sup>4</sup> *Returns:* `c`.

```
charT translate_nocase(charT c) const;
```

- <sup>5</sup> *Returns:* `use_facet<ctype<charT>>(getloc()).tolower(c)`.

```
template<class ForwardIterator>
```

```
string_type transform(ForwardIterator first, ForwardIterator last) const;
```

- <sup>6</sup> *Effects:* As if by:

```
string_type str(first, last);
return use_facet<collate<charT>>(
 getloc()).transform(str.data(), str.data() + str.length());
```

```
template<class ForwardIterator>
```

```
string_type transform_primary(ForwardIterator first, ForwardIterator last) const;
```

- <sup>7</sup> *Effects:* If

```
typeid(use_facet<collate<charT>>(getloc())) == typeid(collate_byname<charT>)
```

and the form of the sort key returned by `collate_byname<charT>::transform(first, last)` is known and can be converted into a primary sort key then returns that key, otherwise returns an empty string.

```
template<class ForwardIterator>
```

```
string_type lookup_collatename(ForwardIterator first, ForwardIterator last) const;
```

- <sup>8</sup> *Returns:* A sequence of one or more characters that represents the collating element consisting of the character sequence designated by the iterator range `[first,last)`. Returns an empty string if the character sequence is not a valid collating element.

```
template<class ForwardIterator>
```

```
char_class_type lookup_classname(
```

```
ForwardIterator first, ForwardIterator last, bool icalse = false) const;
```

- <sup>9</sup> *Returns:* An unspecified value that represents the character classification named by the character sequence designated by the iterator range `[first,last)`. If the parameter `icalse` is `true` then the returned mask identifies the character classification without regard to the case of the characters being matched, otherwise it does honor the case of the characters being matched.<sup>217</sup> The value returned shall be independent of the case of the characters in the character sequence. If the name is not recognized then returns `char_class_type()`.

- <sup>10</sup> *Remarks:* For `regex_traits<char>`, at least the narrow character names in Table 121 shall be recognized. For `regex_traits<wchar_t>`, at least the wide character names in Table 121 shall be recognized.

```
bool isctype(charT c, char_class_type f) const;
```

- <sup>11</sup> *Effects:* Determines if the character `c` is a member of the character classification represented by `f`.

- <sup>12</sup> *Returns:* Given the following function declaration:

```
// for exposition only
```

```
template<class C>
```

```
ctype_base::mask convert(typename regex_traits<C>::char_class_type f);
```

<sup>217</sup> For example, if the parameter `icalse` is `true` then `[[:lower:]]` is the same as `[[:alpha:]]`.



that returns a value in which each `ctype_base::mask` value corresponding to a value in `f` named in Table 121 is set, then the result is determined as if by:

```
ctype_base::mask m = convert<charT>(f);
const ctype<charT>& ct = use_facet<ctype<charT>>(getloc());
if (ct.is(m, c)) {
 return true;
} else if (c == ct.widen('_')) {
 charT w[1] = { ct.widen('w') };
 char_class_type x = lookup_classname(w, w+1);
 return (f&x) == x;
} else {
 return false;
}
```

[Example 1:

```
regex_traits<char> t;
string d("d");
string u("upper");
regex_traits<char>::char_class_type f;
f = t.lookup_classname(d.begin(), d.end());
f |= t.lookup_classname(u.begin(), u.end());
ctype_base::mask m = convert<char>(f); // m == ctype_base::digit | ctype_base::upper
```

— end example]

[Example 2:

```
regex_traits<char> t;
string w("w");
regex_traits<char>::char_class_type f;
f = t.lookup_classname(w.begin(), w.end());
t.isctype('A', f); // returns true
t.isctype('_', f); // returns true
t.isctype(' ', f); // returns false
```

— end example]

```
int value(charT ch, int radix) const;
```

13     *Preconditions:* The value of `radix` is 8, 10, or 16.

14     *Returns:* The value represented by the digit `ch` in base `radix` if the character `ch` is a valid digit in base `radix`; otherwise returns -1.

```
locale_type imbue(locale_type loc);
```

15     *Effects:* Imbues `*this` with a copy of the locale `loc`.

[Note 1: Calling `imbue` with a different locale than the one currently in use invalidates all cached data held by `*this`. — end note]

16     *Postconditions:* `getloc() == loc`.

17     *Returns:* If no locale has been previously imbued then a copy of the global locale in effect at the time of construction of `*this`, otherwise a copy of the last argument passed to `imbue`.

```
locale_type getloc() const;
```

18     *Returns:* If no locale has been imbued then a copy of the global locale in effect at the time of construction of `*this`, otherwise a copy of the last argument passed to `imbue`.

## 28.6.7 Class template `basic_regex`

[re.regex]

### 28.6.7.1 General

[re.regex.general]

- <sup>1</sup> For a char-like type `charT`, specializations of class template `basic_regex` represent regular expressions constructed from character sequences of `charT` characters. In the rest of 28.6.7, `charT` denotes a given char-like type. Storage for a regular expression is allocated and freed as necessary by the member functions of class `basic_regex`.



Table 121 — Character class names and corresponding ctype masks [tab:re.traits.classnames]

| Narrow character name | Wide character name | Corresponding ctype_base::mask value |
|-----------------------|---------------------|--------------------------------------|
| "alnum"               | L"alnum"            | ctype_base::alnum                    |
| "alpha"               | L"alpha"            | ctype_base::alpha                    |
| "blank"               | L"blank"            | ctype_base::blank                    |
| "cntrl"               | L"cntrl"            | ctype_base::cntrl                    |
| "digit"               | L"digit"            | ctype_base::digit                    |
| "d"                   | L"d"                | ctype_base::digit                    |
| "graph"               | L"graph"            | ctype_base::graph                    |
| "lower"               | L"lower"            | ctype_base::lower                    |
| "print"               | L"print"            | ctype_base::print                    |
| "punct"               | L"punct"            | ctype_base::punct                    |
| "space"               | L"space"            | ctype_base::space                    |
| "s"                   | L"s"                | ctype_base::space                    |
| "upper"               | L"upper"            | ctype_base::upper                    |
| "w"                   | L"w"                | ctype_base::alnum                    |
| "xdigit"              | L"xdigit"           | ctype_base::xdigit                   |

- <sup>2</sup> Objects of type specialization of `basic_regex` are responsible for converting the sequence of `charT` objects to an internal representation. It is not specified what form this representation takes, nor how it is accessed by algorithms that operate on regular expressions.

[Note 1: Implementations will typically declare some function templates as friends of `basic_regex` to achieve this. — end note]

- <sup>3</sup> The functions described in 28.6.7 report errors by throwing exceptions of type `regex_error`.

```
namespace std {
 template<class charT, class traits = regex_traits<charT>>
 class basic_regex {
 public:
 // types
 using value_type = charT;
 using traits_type = traits;
 using string_type = traits::string_type;
 using flag_type = regex_constants::syntax_option_type;
 using locale_type = traits::locale_type;

 // 28.6.4.2, constants
 static constexpr flag_type icase = regex_constants::icase;
 static constexpr flag_type nosubs = regex_constants::nosubs;
 static constexpr flag_type optimize = regex_constants::optimize;
 static constexpr flag_type collate = regex_constants::collate;
 static constexpr flag_type ECMAScript = regex_constants::ECMAScript;
 static constexpr flag_type basic = regex_constants::basic;
 static constexpr flag_type extended = regex_constants::extended;
 static constexpr flag_type awk = regex_constants::awk;
 static constexpr flag_type grep = regex_constants::grep;
 static constexpr flag_type egrep = regex_constants::egrep;
 static constexpr flag_type multiline = regex_constants::multiline;

 // 28.6.7.2, construct/copy/destroy
 basic_regex();
 explicit basic_regex(const charT* p, flag_type f = regex_constants::ECMAScript);
 basic_regex(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
 basic_regex(const basic_regex&);
 basic_regex(basic_regex&&) noexcept;
 template<class ST, class SA>
 explicit basic_regex(const basic_string<charT, ST, SA>& s,
 flag_type f = regex_constants::ECMAScript);
```

```

template<class ForwardIterator>
 basic_regex(ForwardIterator first, ForwardIterator last,
 flag_type f = regex_constants::ECMAScript);
basic_regex(initializer_list<charT> il, flag_type f = regex_constants::ECMAScript);

~basic_regex();

// 28.6.7.3, assign
basic_regex& operator=(const basic_regex& e);
basic_regex& operator=(basic_regex&& e) noexcept;
basic_regex& operator=(const charT* p);
basic_regex& operator=(initializer_list<charT> il);
template<class ST, class SA>
 basic_regex& operator=(const basic_string<charT, ST, SA>& s);

basic_regex& assign(const basic_regex& e);
basic_regex& assign(basic_regex&& e) noexcept;
basic_regex& assign(const charT* p, flag_type f = regex_constants::ECMAScript);
basic_regex& assign(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
template<class ST, class SA>
 basic_regex& assign(const basic_string<charT, ST, SA>& s,
 flag_type f = regex_constants::ECMAScript);
template<class InputIterator>
 basic_regex& assign(InputIterator first, InputIterator last,
 flag_type f = regex_constants::ECMAScript);
basic_regex& assign(initializer_list<charT>,
 flag_type f = regex_constants::ECMAScript);

// 28.6.7.4, const operations
unsigned mark_count() const;
flag_type flags() const;

// 28.6.7.5, locale
locale_type imbue(locale_type loc);
locale_type getloc() const;

// 28.6.7.6, swap
void swap(basic_regex&);
};

template<class ForwardIterator>
 basic_regex(ForwardIterator, ForwardIterator,
 regex_constants::syntax_option_type = regex_constants::ECMAScript)
 -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
}

```

### 28.6.7.2 Constructors

[re.regex.construct]

```
basic_regex();
```

1     *Postconditions:* *\*this* does not match any character sequence.

```
explicit basic_regex(const charT* p, flag_type f = regex_constants::ECMAScript);
```

2     *Preconditions:*  $[p, p + \text{char\_traits}<\text{charT}>::\text{length}(p))$  is a valid range.

3     *Effects:* The object's internal finite state machine is constructed from the regular expression contained in the sequence of characters  $[p, p + \text{char\_traits}<\text{charT}>::\text{length}(p))$ , and interpreted according to the flags *f*.

4     *Postconditions:* *flags()* returns *f*. *mark\_count()* returns the number of marked sub-expressions within the expression.

5     *Throws:* *regex\_error* if  $[p, p + \text{char\_traits}<\text{charT}>::\text{length}(p))$  is not a valid regular expression.

```
basic_regex(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
```

6     *Preconditions:*  $[p, p + \text{len})$  is a valid range.

7     *Effects:* The object's internal finite state machine is constructed from the regular expression contained in the sequence of characters `[p, p + len)`, and interpreted according the flags specified in `f`.

8     *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

9     *Throws:* `regex_error` if `[p, p + len)` is not a valid regular expression.

```
basic_regex(const basic_regex& e);
```

10    *Postconditions:* `flags()` and `mark_count()` return `e.flags()` and `e.mark_count()`, respectively.

```
basic_regex(basic_regex&& e) noexcept;
```

11    *Postconditions:* `flags()` and `mark_count()` return the values that `e.flags()` and `e.mark_count()`, respectively, had before construction.

```
template<class ST, class SA>
explicit basic_regex(const basic_string<charT, ST, SA>& s,
 flag_type f = regex_constants::ECMAScript);
```

12    *Effects:* The object's internal finite state machine is constructed from the regular expression contained in the string `s`, and interpreted according to the flags specified in `f`.

13    *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

14    *Throws:* `regex_error` if `s` is not a valid regular expression.

```
template<class ForwardIterator>
basic_regex(ForwardIterator first, ForwardIterator last,
 flag_type f = regex_constants::ECMAScript);
```

15    *Effects:* The object's internal finite state machine is constructed from the regular expression contained in the sequence of characters `[first, last)`, and interpreted according to the flags specified in `f`.

16    *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

17    *Throws:* `regex_error` if the sequence `[first, last)` is not a valid regular expression.

```
basic_regex(initializer_list<charT> il, flag_type f = regex_constants::ECMAScript);
```

18    *Effects:* Same as `basic_regex(il.begin(), il.end(), f)`.

### 28.6.7.3 Assignment

[re.regex.assign]

```
basic_regex& operator=(const basic_regex& e);
```

1    *Postconditions:* `flags()` and `mark_count()` return `e.flags()` and `e.mark_count()`, respectively.

```
basic_regex& operator=(basic_regex&& e) noexcept;
```

2    *Postconditions:* `flags()` and `mark_count()` return the values that `e.flags()` and `e.mark_count()`, respectively, had before assignment. `e` is in a valid state with unspecified value.

```
basic_regex& operator=(const charT* p);
```

3    *Effects:* Equivalent to: `return assign(p);`

```
basic_regex& operator=(initializer_list<charT> il);
```

4    *Effects:* Equivalent to: `return assign(il.begin(), il.end());`

```
template<class ST, class SA>
basic_regex& operator=(const basic_string<charT, ST, SA>& s);
```

5    *Effects:* Equivalent to: `return assign(s);`

```
basic_regex& assign(const basic_regex& e);
```

6    *Effects:* Equivalent to: `return *this = e;`

```
basic_regex& assign(basic_regex&& e) noexcept;
```

7     *Effects:* Equivalent to: `return *this = std::move(e);`

```
basic_regex& assign(const charT* p, flag_type f = regex_constants::ECMAScript);
```

8     *Effects:* Equivalent to: `return assign(string_type(p), f);`

```
basic_regex& assign(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
```

9     *Effects:* Equivalent to: `return assign(string_type(p, len), f);`

```
template<class ST, class SA>
```

```
 basic_regex& assign(const basic_string<charT, ST, SA>& s,
 flag_type f = regex_constants::ECMAScript);
```

10     *Effects:* Assigns the regular expression contained in the string `s`, interpreted according the flags specified in `f`. If an exception is thrown, `*this` is unchanged.

11     *Postconditions:* If no exception is thrown, `flags()` returns `f` and `mark_count()` returns the number of marked sub-expressions within the expression.

12     *Returns:* `*this`.

13     *Throws:* `regex_error` if `s` is not a valid regular expression.

```
template<class InputIterator>
```

```
 basic_regex& assign(InputIterator first, InputIterator last,
 flag_type f = regex_constants::ECMAScript);
```

14     *Effects:* Equivalent to: `return assign(string_type(first, last), f);`

```
basic_regex& assign(initializer_list<charT> il,
 flag_type f = regex_constants::ECMAScript);
```

15     *Effects:* Equivalent to: `return assign(il.begin(), il.end(), f);`

#### 28.6.7.4 Constant operations

[re.regex.operations]

```
unsigned mark_count() const;
```

1     *Effects:* Returns the number of marked sub-expressions within the regular expression.

```
flag_type flags() const;
```

2     *Effects:* Returns a copy of the regular expression syntax flags that were passed to the object's constructor or to the last call to `assign`.

#### 28.6.7.5 Locale

[re.regex.locale]

```
locale_type imbue(locale_type loc);
```

1     *Effects:* Returns the result of `traits_inst.imbue(loc)` where `traits_inst` is a (default-initialized) instance of the template type argument `traits` stored within the object. After a call to `imbue` the `basic_regex` object does not match any character sequence.

```
locale_type getloc() const;
```

2     *Effects:* Returns the result of `traits_inst.getloc()` where `traits_inst` is a (default-initialized) instance of the template parameter `traits` stored within the object.

#### 28.6.7.6 Swap

[re.regex.swap]

```
void swap(basic_regex& e);
```

1     *Effects:* Swaps the contents of the two regular expressions.

2     *Postconditions:* `*this` contains the regular expression that was in `e`, `e` contains the regular expression that was in `*this`.

3     *Complexity:* Constant time.

**28.6.7.7 Non-member functions**

[re.regex.nonmemb]

```
template<class charT, class traits>
void swap(basic_regex<charT, traits>& lhs, basic_regex<charT, traits>& rhs);
```

1 *Effects:* Calls `lhs.swap(rhs)`.

**28.6.8 Class template `sub_match`**

[re.submatch]

**28.6.8.1 General**

[re.submatch.general]

1 Class template `sub_match` denotes the sequence of characters matched by a particular marked sub-expression.

```
namespace std {
 template<class BidirectionalIterator>
 class sub_match : public pair<BidirectionalIterator, BidirectionalIterator> {
 public:
 using value_type = iterator_traits<BidirectionalIterator>::value_type;
 using difference_type = iterator_traits<BidirectionalIterator>::difference_type;
 using iterator = BidirectionalIterator;
 using string_type = basic_string<value_type>;

 bool matched;

 constexpr sub_match();

 difference_type length() const;
 operator string_type() const;
 string_type str() const;

 int compare(const sub_match& s) const;
 int compare(const string_type& s) const;
 int compare(const value_type* s) const;

 void swap(sub_match& s) noexcept(see below);
 };
}
```

**28.6.8.2 Members**

[re.submatch.members]

```
constexpr sub_match();
```

1 *Effects:* Value-initializes the pair base class subobject and the member `matched`.

```
difference_type length() const;
```

2 *Returns:* `matched ? distance(first, second) : 0`.

```
operator string_type() const;
```

3 *Returns:* `matched ? string_type(first, second) : string_type()`.

```
string_type str() const;
```

4 *Returns:* `matched ? string_type(first, second) : string_type()`.

```
int compare(const sub_match& s) const;
```

5 *Returns:* `str().compare(s.str())`.

```
int compare(const string_type& s) const;
```

6 *Returns:* `str().compare(s)`.

```
int compare(const value_type* s) const;
```

7 *Returns:* `str().compare(s)`.

```
void swap(sub_match& s) noexcept(see below);
```

8 *Preconditions:* `BidirectionalIterator` meets the *Cpp17Swappable* requirements ([16.4.4.3](#)).

9 *Effects:* Equivalent to:

```

 this->pair<BidirectionalIterator, BidirectionalIterator>::swap(s);
 std::swap(matched, s.matched);

```

10 *Remarks:* The exception specification is equivalent to `is_nothrow_swappable_v<BidirectionalIterator>`.

### 28.6.8.3 Non-member operators

[re.submatch.op]

1 Let *SM-CAT*(I) be

```

 compare_three_way_result_t<basic_string<typename iterator_traits<I>::value_type>>

```

```

template<class BiIter>

```

```

 bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);

```

2 *Returns:* `lhs.compare(rhs) == 0`.

```

template<class BiIter>

```

```

 auto operator<=>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);

```

3 *Returns:* `static_cast<SM-CAT(BiIter)>(lhs.compare(rhs) <=> 0)`.

```

template<class BiIter, class ST, class SA>

```

```

 bool operator==(
 const sub_match<BiIter>& lhs,
 const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

```

4 *Returns:*

```

 lhs.compare(typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())) == 0

```

```

template<class BiIter, class ST, class SA>

```

```

 auto operator<=>(
 const sub_match<BiIter>& lhs,
 const basic_string<typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

```

5 *Returns:*

```

 static_cast<SM-CAT(BiIter)>(lhs.compare(
 typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())
 <=> 0
))

```

```

template<class BiIter>

```

```

 bool operator==(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type* rhs);

```

6 *Returns:* `lhs.compare(rhs) == 0`.

```

template<class BiIter>

```

```

 auto operator<=>(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type* rhs);

```

7 *Returns:* `static_cast<SM-CAT(BiIter)>(lhs.compare(rhs) <=> 0)`.

```

template<class BiIter>

```

```

 bool operator==(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type& rhs);

```

8 *Returns:* `lhs.compare(typename sub_match<BiIter>::string_type(1, rhs)) == 0`.

```

template<class BiIter>

```

```

 auto operator<=>(const sub_match<BiIter>& lhs,
 const typename iterator_traits<BiIter>::value_type& rhs);

```

9 *Returns:*

```

 static_cast<SM-CAT(BiIter)>(lhs.compare(
 typename sub_match<BiIter>::string_type(1, rhs)
 <=> 0
))

```

```
template<class charT, class ST, class Bilter>
 basic_ostream<charT, ST>&
 operator<<(basic_ostream<charT, ST>& os, const sub_match<Bilter>& m);
```

10     *Returns:* os << m.str().

## 28.6.9 Class template `match_results`

[re.results]

### 28.6.9.1 General

[re.results.general]

- <sup>1</sup> Class template `match_results` denotes a collection of character sequences representing the result of a regular expression match. Storage for the collection is allocated and freed as necessary by the member functions of class template `match_results`.
- <sup>2</sup> The class template `match_results` meets the requirements of an allocator-aware container (23.2.2.5) and of a sequence container (23.2.2, 23.2.4) except that only copy assignment, move assignment, and operations defined for const-qualified sequence containers are supported and that the semantics of the comparison operator functions are different from those required for a container.
- <sup>3</sup> A default-constructed `match_results` object has no fully established result state. A match result is *ready* when, as a consequence of a completed regular expression match modifying such an object, its result state becomes fully established. The effects of calling most member functions from a `match_results` object that is not ready are undefined.
- <sup>4</sup> The `sub_match` object stored at index 0 represents sub-expression 0, i.e., the whole match. In this case the `sub_match` member `matched` is always `true`. The `sub_match` object stored at index `n` denotes what matched the marked sub-expression `n` within the matched expression. If the sub-expression `n` participated in a regular expression match then the `sub_match` member `matched` evaluates to `true`, and members `first` and `second` denote the range of characters [`first`, `second`) which formed that match. Otherwise `matched` is `false`, and members `first` and `second` point to the end of the sequence that was searched.

[Note 1: The `sub_match` objects representing different sub-expressions that did not participate in a regular expression match need not be distinct. — end note]

```
namespace std {
 template<class BidirectionalIterator,
 class Allocator = allocator<sub_match<BidirectionalIterator>>>
 class match_results {
 public:
 using value_type = sub_match<BidirectionalIterator>;
 using const_reference = const value_type&;
 using reference = value_type&;
 using const_iterator = implementation-defined;
 using iterator = const_iterator;
 using difference_type = iterator_traits<BidirectionalIterator>::difference_type;
 using size_type = allocator_traits<Allocator>::size_type;
 using allocator_type = Allocator;
 using char_type = iterator_traits<BidirectionalIterator>::value_type;
 using string_type = basic_string<char_type>;

 // 28.6.9.2, construct/copy/destroy
 match_results() : match_results(Allocator()) {}
 explicit match_results(const Allocator& a);
 match_results(const match_results& m);
 match_results(const match_results& m, const Allocator& a);
 match_results(match_results&& m) noexcept;
 match_results(match_results&& m, const Allocator& a);
 match_results& operator=(const match_results& m);
 match_results& operator=(match_results&& m);
 ~match_results();

 // 28.6.9.3, state
 bool ready() const;

 // 28.6.9.4, size
 size_type size() const;
 size_type max_size() const;
```

```

bool empty() const;

// 28.6.9.5, element access
difference_type length(size_type sub = 0) const;
difference_type position(size_type sub = 0) const;
string_type str(size_type sub = 0) const;
const_reference operator[](size_type n) const;

const_reference prefix() const;
const_reference suffix() const;
const_iterator begin() const;
const_iterator end() const;
const_iterator cbegin() const;
const_iterator cend() const;

// 28.6.9.6, format
template<class OutputIter>
 OutputIter
 format(OutputIter out,
 const char_type* fmt_first, const char_type* fmt_last,
 regex_constants::match_flag_type flags = regex_constants::format_default) const;
template<class OutputIter, class ST, class SA>
 OutputIter
 format(OutputIter out,
 const basic_string<char_type, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::format_default) const;
template<class ST, class SA>
 basic_string<char_type, ST, SA>
 format(const basic_string<char_type, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::format_default) const;
string_type
 format(const char_type* fmt,
 regex_constants::match_flag_type flags = regex_constants::format_default) const;

// 28.6.9.7, allocator
allocator_type get_allocator() const;

// 28.6.9.8, swap
void swap(match_results& that);
};
}

```

### 28.6.9.2 Constructors

[re.results.const]

- <sup>1</sup> Table 122 lists the postconditions of `match_results` copy/move constructors and copy/move assignment operators. For move operations, the results of the expressions depending on the parameter `m` denote the values they had before the respective function calls.

```
explicit match_results(const Allocator& a);
```

- <sup>2</sup> *Effects:* The stored `Allocator` value is constructed from `a`.  
<sup>3</sup> *Postconditions:* `ready()` returns `false`. `size()` returns 0.

```
match_results(const match_results& m);
match_results(const match_results& m, const Allocator& a);
```

- <sup>4</sup> *Effects:* For the first form, the stored `Allocator` value is obtained as specified in 23.2.2.2. For the second form, the stored `Allocator` value is constructed from `a`.  
<sup>5</sup> *Postconditions:* As specified in Table 122.

```
match_results(match_results&& m) noexcept;
match_results(match_results&& m, const Allocator& a);
```

- <sup>6</sup> *Effects:* For the first form, the stored `Allocator` value is move constructed from `m.get_allocator()`. For the second form, the stored `Allocator` value is constructed from `a`.



7 *Postconditions:* As specified in [Table 122](#).

8 *Throws:* The second form throws nothing if `a == m.get_allocator()` is true.

```
match_results& operator=(const match_results& m);
```

9 *Postconditions:* As specified in [Table 122](#).

```
match_results& operator=(match_results&& m);
```

10 *Postconditions:* As specified in [Table 122](#).

**Table 122** — `match_results` copy/move operation postconditions [tab:re.results.const]

| Element                  | Value                                                                                 |
|--------------------------|---------------------------------------------------------------------------------------|
| <code>ready()</code>     | <code>m.ready()</code>                                                                |
| <code>size()</code>      | <code>m.size()</code>                                                                 |
| <code>str(n)</code>      | <code>m.str(n)</code> for all non-negative integers <code>n &lt; m.size()</code>      |
| <code>prefix()</code>    | <code>m.prefix()</code>                                                               |
| <code>suffix()</code>    | <code>m.suffix()</code>                                                               |
| <code>(*this)[n]</code>  | <code>m[n]</code> for all non-negative integers <code>n &lt; m.size()</code>          |
| <code>length(n)</code>   | <code>m.length(n)</code> for all non-negative integers <code>n &lt; m.size()</code>   |
| <code>position(n)</code> | <code>m.position(n)</code> for all non-negative integers <code>n &lt; m.size()</code> |

### 28.6.9.3 State

[re.results.state]

```
bool ready() const;
```

1 *Returns:* true if `*this` has a fully established result state, otherwise false.

### 28.6.9.4 Size

[re.results.size]

```
size_type size() const;
```

1 *Returns:* One plus the number of marked sub-expressions in the regular expression that was matched if `*this` represents the result of a successful match. Otherwise returns 0.

[Note 1: The state of a `match_results` object can be modified only by passing that object to `regex_match` or `regex_search`. Subclauses [28.6.10.2](#) and [28.6.10.3](#) specify the effects of those algorithms on their `match_results` arguments. — end note]

```
size_type max_size() const;
```

2 *Returns:* The maximum number of `sub_match` elements that can be stored in `*this`.

```
bool empty() const;
```

3 *Returns:* `size() == 0`.

### 28.6.9.5 Element access

[re.results.acc]

```
difference_type length(size_type sub = 0) const;
```

1 *Preconditions:* `ready() == true`.

2 *Returns:* `(*this)[sub].length()`.

```
difference_type position(size_type sub = 0) const;
```

3 *Preconditions:* `ready() == true`.

4 *Returns:* The distance from the start of the target sequence to `(*this)[sub].first`.

```
string_type str(size_type sub = 0) const;
```

5 *Preconditions:* `ready() == true`.

6 *Returns:* `string_type((*this)[sub])`.

```
const_reference operator[](size_type n) const;
```

7     *Preconditions:* `ready() == true`.

8     *Returns:* A reference to the `sub_match` object representing the character sequence that matched marked sub-expression `n`. If `n == 0` then returns a reference to a `sub_match` object representing the character sequence that matched the whole regular expression. If `n >= size()` then returns a `sub_match` object representing an unmatched sub-expression.

```
const_reference prefix() const;
```

9     *Preconditions:* `ready() == true`.

10    *Returns:* A reference to the `sub_match` object representing the character sequence from the start of the string being matched/searched to the start of the match found.

```
const_reference suffix() const;
```

11    *Preconditions:* `ready() == true`.

12    *Returns:* A reference to the `sub_match` object representing the character sequence from the end of the match found to the end of the string being matched/searched.

```
const_iterator begin() const;
```

```
const_iterator cbegin() const;
```

13    *Returns:* A starting iterator that enumerates over all the sub-expressions stored in `*this`.

```
const_iterator end() const;
```

```
const_iterator cend() const;
```

14    *Returns:* A terminating iterator that enumerates over all the sub-expressions stored in `*this`.

#### 28.6.9.6 Formatting

[re.results.form]

```
template<class OutputIter>
```

```
OutputIter format(
```

```
OutputIter out,
```

```
const char_type* fmt_first, const char_type* fmt_last,
```

```
regex_constants::match_flag_type flags = regex_constants::format_default) const;
```

1     *Preconditions:* `ready() == true` and `OutputIter` meets the requirements for a *Cpp17OutputIterator* (24.3.5.4).

2     *Effects:* Copies the character sequence `[fmt_first, fmt_last)` to `OutputIter out`. Replaces each format specifier or escape sequence in the copied range with either the character(s) it represents or the sequence of characters within `*this` to which it refers. The bitmasks specified in `flags` determine which format specifiers and escape sequences are recognized.

3     *Returns:* `out`.

```
template<class OutputIter, class ST, class SA>
```

```
OutputIter format(
```

```
OutputIter out,
```

```
const basic_string<char_type, ST, SA>& fmt,
```

```
regex_constants::match_flag_type flags = regex_constants::format_default) const;
```

4     *Effects:* Equivalent to:

```
return format(out, fmt.data(), fmt.data() + fmt.size(), flags);
```

```
template<class ST, class SA>
```

```
basic_string<char_type, ST, SA> format(
```

```
const basic_string<char_type, ST, SA>& fmt,
```

```
regex_constants::match_flag_type flags = regex_constants::format_default) const;
```

5     *Preconditions:* `ready() == true`.

6     *Effects:* Constructs an empty string `result` of type `basic_string<char_type, ST, SA>` and calls:

```
format(back_inserter(result), fmt, flags);
```

7     *Returns:* `result`.

```
string_type format(
 const char_type* fmt,
 regex_constants::match_flag_type flags = regex_constants::format_default) const;
```

8     *Preconditions:* `ready() == true`.

9     *Effects:* Constructs an empty string `result` of type `string_type` and calls:

```
format(back_inserter(result), fmt, fmt + char_traits<char_type>::length(fmt), flags);
```

10    *Returns:* `result`.

### 28.6.9.7 Allocator

[re.results.all]

```
allocator_type get_allocator() const;
```

1     *Returns:* A copy of the Allocator that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement.

### 28.6.9.8 Swap

[re.results.swap]

```
void swap(match_results& that);
```

1     *Effects:* Swaps the contents of the two sequences.

2     *Postconditions:* `*this` contains the sequence of matched sub-expressions that were in `that`, `that` contains the sequence of matched sub-expressions that were in `*this`.

3     *Complexity:* Constant time.

```
template<class BidirectionalIterator, class Allocator>
void swap(match_results<BidirectionalIterator, Allocator>& m1,
 match_results<BidirectionalIterator, Allocator>& m2);
```

4     *Effects:* As if by `m1.swap(m2)`.

### 28.6.9.9 Non-member functions

[re.results.nonmember]

```
template<class BidirectionalIterator, class Allocator>
bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
 const match_results<BidirectionalIterator, Allocator>& m2);
```

1     *Returns:* `true` if neither match result is ready, `false` if one match result is ready and the other is not. If both match results are ready, returns `true` only if

(1.1)     — `m1.empty() && m2.empty()`, or

(1.2)     — `!m1.empty() && !m2.empty()`, and the following conditions are satisfied:

(1.2.1)     — `m1.prefix() == m2.prefix()`,

(1.2.2)     — `m1.size() == m2.size() && equal(m1.begin(), m1.end(), m2.begin())`, and

(1.2.3)     — `m1.suffix() == m2.suffix()`.

[Note 1: The algorithm `equal` is defined in [Clause 26](#). — end note]

## 28.6.10 Regular expression algorithms

[re.alg]

### 28.6.10.1 Exceptions

[re.exception]

1     The algorithms described in subclause [28.6.10](#) may throw an exception of type `regex_error`. If such an exception `e` is thrown, `e.code()` shall return either `regex_constants::error_complexity` or `regex_constants::error_stack`.

### 28.6.10.2 `regex_match`

[re.alg.match]

```
template<class BidirectionalIterator, class Allocator, class charT, class traits>
bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
 match_results<BidirectionalIterator, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

1     *Preconditions:* `BidirectionalIterator` models `bidirectional_iterator` ([24.3.4.12](#)).

- 2 *Effects:* Determines whether there is a match between the regular expression *e*, and all of the character sequence `[first,last)`. The parameter *flags* is used to control how the expression is matched against the character sequence. When determining if there is a match, only potential matches that match the entire character sequence are considered. Returns `true` if such a match exists, `false` otherwise.

[Example 1:

```
std::regex re("Get|GetValue");
std::cmatch m;
regex_search("GetValue", m, re); // returns true, and m[0] contains "Get"
regex_match ("GetValue", m, re); // returns true, and m[0] contains "GetValue"
regex_search("GetValues", m, re); // returns true, and m[0] contains "Get"
regex_match ("GetValues", m, re); // returns false
```

— end example]

- 3 *Postconditions:* `m.ready() == true` in all cases. If the function returns `false`, then the effect on parameter *m* is unspecified except that `m.size()` returns 0 and `m.empty()` returns `true`. Otherwise the effects on parameter *m* are given in Table 123.

Table 123 — Effects of `regex_match` algorithm [tab:re.alg.match]

| Element                         | Value                                                                                                                                                                                                       |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>m.size()</code>           | <code>1 + e.mark_count()</code>                                                                                                                                                                             |
| <code>m.empty()</code>          | <code>false</code>                                                                                                                                                                                          |
| <code>m.prefix().first</code>   | <code>first</code>                                                                                                                                                                                          |
| <code>m.prefix().second</code>  | <code>first</code>                                                                                                                                                                                          |
| <code>m.prefix().matched</code> | <code>false</code>                                                                                                                                                                                          |
| <code>m.suffix().first</code>   | <code>last</code>                                                                                                                                                                                           |
| <code>m.suffix().second</code>  | <code>last</code>                                                                                                                                                                                           |
| <code>m.suffix().matched</code> | <code>false</code>                                                                                                                                                                                          |
| <code>m[0].first</code>         | <code>first</code>                                                                                                                                                                                          |
| <code>m[0].second</code>        | <code>last</code>                                                                                                                                                                                           |
| <code>m[0].matched</code>       | <code>true</code>                                                                                                                                                                                           |
| <code>m[n].first</code>         | For all integers $0 < n < m.size()$ , the start of the sequence that matched sub-expression <i>n</i> . Alternatively, if sub-expression <i>n</i> did not participate in the match, then <code>last</code> . |
| <code>m[n].second</code>        | For all integers $0 < n < m.size()$ , the end of the sequence that matched sub-expression <i>n</i> . Alternatively, if sub-expression <i>n</i> did not participate in the match, then <code>last</code> .   |
| <code>m[n].matched</code>       | For all integers $0 < n < m.size()$ , <code>true</code> if sub-expression <i>n</i> participated in the match, <code>false</code> otherwise.                                                                 |

```
template<class BidirectionalIterator, class charT, class traits>
bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

- 4 *Effects:* Behaves “as if” by constructing an instance of `match_results<BidirectionalIterator>` *what*, and then returning the result of `regex_match(first, last, what, e, flags)`.

```
template<class charT, class Allocator, class traits>
bool regex_match(const charT* str,
 match_results<const charT*, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

- 5 *Returns:* `regex_match(str, str + char_traits<charT>::length(str), m, e, flags)`.

```
template<class ST, class SA, class Allocator, class charT, class traits>
bool regex_match(const basic_string<charT, ST, SA>& s,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

6     *Returns:* `regex_match(s.begin(), s.end(), m, e, flags)`.

```
template<class charT, class traits>
bool regex_match(const charT* str,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

7     *Returns:* `regex_match(str, str + char_traits<charT>::length(str), e, flags)`.

```
template<class ST, class SA, class charT, class traits>
bool regex_match(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

8     *Returns:* `regex_match(s.begin(), s.end(), e, flags)`.

### 28.6.10.3 regex\_search

[re.alg.search]

```
template<class BidirectionalIterator, class Allocator, class charT, class traits>
bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
 match_results<BidirectionalIterator, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

1     *Preconditions:* `BidirectionalIterator` models `bidirectional_iterator` (24.3.4.12).

2     *Effects:* Determines whether there is some sub-sequence within `[first, last)` that matches the regular expression `e`. The parameter `flags` is used to control how the expression is matched against the character sequence. Returns `true` if such a sequence exists, `false` otherwise.

3     *Postconditions:* `m.ready() == true` in all cases. If the function returns `false`, then the effect on parameter `m` is unspecified except that `m.size()` returns 0 and `m.empty()` returns `true`. Otherwise the effects on parameter `m` are given in Table 124.

Table 124 — Effects of `regex_search` algorithm [tab:re.alg.search]

| Element                         | Value                                                                                                                                                                                                                                    |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>m.size()</code>           | <code>1 + e.mark_count()</code>                                                                                                                                                                                                          |
| <code>m.empty()</code>          | <code>false</code>                                                                                                                                                                                                                       |
| <code>m.prefix().first</code>   | <code>first</code>                                                                                                                                                                                                                       |
| <code>m.prefix().second</code>  | <code>m[0].first</code>                                                                                                                                                                                                                  |
| <code>m.prefix().matched</code> | <code>m.prefix().first != m.prefix().second</code>                                                                                                                                                                                       |
| <code>m.suffix().first</code>   | <code>m[0].second</code>                                                                                                                                                                                                                 |
| <code>m.suffix().second</code>  | <code>last</code>                                                                                                                                                                                                                        |
| <code>m.suffix().matched</code> | <code>m.suffix().first != m.suffix().second</code>                                                                                                                                                                                       |
| <code>m[0].first</code>         | The start of the sequence of characters that matched the regular expression                                                                                                                                                              |
| <code>m[0].second</code>        | The end of the sequence of characters that matched the regular expression                                                                                                                                                                |
| <code>m[0].matched</code>       | <code>true</code>                                                                                                                                                                                                                        |
| <code>m[n].first</code>         | For all integers <code>0 &lt; n &lt; m.size()</code> , the start of the sequence that matched sub-expression <code>n</code> . Alternatively, if sub-expression <code>n</code> did not participate in the match, then <code>last</code> . |
| <code>m[n].second</code>        | For all integers <code>0 &lt; n &lt; m.size()</code> , the end of the sequence that matched sub-expression <code>n</code> . Alternatively, if sub-expression <code>n</code> did not participate in the match, then <code>last</code> .   |

Table 124 — Effects of `regex_search` algorithm (continued)

| Element                   | Value                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>m[n].matched</code> | For all integers $0 < n < m.size()$ , true if sub-expression <code>n</code> participated in the match, false otherwise. |

```
template<class charT, class Allocator, class traits>
bool regex_search(const charT* str, match_results<const charT*, Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

4     *Returns:* `regex_search(str, str + char_traits<charT>::length(str), m, e, flags)`.

```
template<class ST, class SA, class Allocator, class charT, class traits>
bool regex_search(const basic_string<charT, ST, SA>& s,
 match_results<typename basic_string<charT, ST, SA>::const_iterator,
 Allocator>& m,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

5     *Returns:* `regex_search(s.begin(), s.end(), m, e, flags)`.

```
template<class BidirectionalIterator, class charT, class traits>
bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

6     *Effects:* Behaves “as if” by constructing an object what of type `match_results<BidirectionalIterator>` and returning `regex_search(first, last, what, e, flags)`.

```
template<class charT, class traits>
bool regex_search(const charT* str,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

7     *Returns:* `regex_search(str, str + char_traits<charT>::length(str), e, flags)`.

```
template<class ST, class SA, class charT, class traits>
bool regex_search(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

8     *Returns:* `regex_search(s.begin(), s.end(), e, flags)`.

#### 28.6.10.4 `regex_replace`

[re.alg.replace]

```
template<class OutputIterator, class BidirectionalIterator,
 class traits, class charT, class ST, class SA>
OutputIterator
regex_replace(OutputIterator out,
 BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

```
template<class OutputIterator, class BidirectionalIterator, class traits, class charT>
OutputIterator
regex_replace(OutputIterator out,
 BidirectionalIterator first, BidirectionalIterator last,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

1     *Effects:* Constructs a `regex_iterator` object `i` as if by

```
regex_iterator<BidirectionalIterator, charT, traits> i(first, last, e, flags)
```

and uses `i` to enumerate through all of the matches `m` of type `match_results<BidirectionalIterator>` that occur within the sequence `[first, last)`. If no such matches are found and `!(flags & regex_constants::format_no_copy)`, then calls

```
out = copy(first, last, out)
```

If any matches are found then, for each such match:

(1.1) — If `!(flags & regex_constants::format_no_copy)`, calls

```
out = copy(m.prefix().first, m.prefix().second, out)
```

(1.2) — Then calls

```
out = m.format(out, fmt, flags)
```

for the first form of the function and

```
out = m.format(out, fmt, fmt + char_traits<charT>::length(fmt), flags)
```

for the second.

Finally, if such a match is found and `!(flags & regex_constants::format_no_copy)`, calls

```
out = copy(last_m.suffix().first, last_m.suffix().second, out)
```

where `last_m` is a copy of the last match found. If `flags & regex_constants::format_first_only` is nonzero, then only the first match found is replaced.

2 *Returns:* `out`.

```
template<class traits, class charT, class ST, class SA, class FST, class FSA>
basic_string<charT, ST, SA>
regex_replace(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, FST, FSA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT, class ST, class SA>
basic_string<charT, ST, SA>
regex_replace(const basic_string<charT, ST, SA>& s,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

3 *Effects:* Constructs an empty string result of type `basic_string<charT, ST, SA>` and calls:

```
regex_replace(back_inserter(result), s.begin(), s.end(), e, fmt, flags);
```

4 *Returns:* `result`.

```
template<class traits, class charT, class ST, class SA>
basic_string<charT>
regex_replace(const charT* s,
 const basic_regex<charT, traits>& e,
 const basic_string<charT, ST, SA>& fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
template<class traits, class charT>
basic_string<charT>
regex_replace(const charT* s,
 const basic_regex<charT, traits>& e,
 const charT* fmt,
 regex_constants::match_flag_type flags = regex_constants::match_default);
```

5 *Effects:* Constructs an empty string result of type `basic_string<charT>` and calls:

```
regex_replace(back_inserter(result), s, s + char_traits<charT>::length(s), e, fmt, flags);
```

6 *Returns:* `result`.

**28.6.11 Regular expression iterators****[re.iter]****28.6.11.1 Class template `regex_iterator`****[re.regiter]****28.6.11.1.1 General****[re.regiter.general]**

- <sup>1</sup> The class template `regex_iterator` is an iterator adaptor. It represents a new view of an existing iterator sequence, by enumerating all the occurrences of a regular expression within that sequence. A `regex_iterator` uses `regex_search` to find successive regular expression matches within the sequence from which it was constructed. After the iterator is constructed, and every time `operator++` is used, the iterator finds and stores a value of `match_results<BidirectionalIterator>`. If the end of the sequence is reached (`regex_search` returns `false`), the iterator becomes equal to the end-of-sequence iterator value. The default constructor constructs an end-of-sequence iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-sequence iterator is not defined. For any other iterator value a `const match_results<BidirectionalIterator>&` is returned. The result of `operator->` on an end-of-sequence iterator is not defined. For any other iterator value a `const match_results<BidirectionalIterator>*` is returned. It is impossible to store things into `regex_iterators`. Two end-of-sequence iterators are always equal. An end-of-sequence iterator is not equal to a non-end-of-sequence iterator. Two non-end-of-sequence iterators are equal when they are constructed from the same arguments.

```

namespace std {
 template<class BidirectionalIterator,
 class charT = typename iterator_traits<BidirectionalIterator>::value_type,
 class traits = regex_traits<charT>>
 class regex_iterator {
 public:
 using regex_type = basic_regex<charT, traits>;
 using iterator_category = forward_iterator_tag;
 using iterator_concept = input_iterator_tag;
 using value_type = match_results<BidirectionalIterator>;
 using difference_type = ptrdiff_t;
 using pointer = const value_type*;
 using reference = const value_type&;

 regex_iterator();
 regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 regex_constants::match_flag_type m = regex_constants::match_default);
 regex_iterator(BidirectionalIterator, BidirectionalIterator,
 const regex_type&&,
 regex_constants::match_flag_type = regex_constants::match_default) = delete;
 regex_iterator(const regex_iterator&);
 regex_iterator& operator=(const regex_iterator&);
 bool operator==(const regex_iterator&) const;
 bool operator==(default_sentinel_t) const { return *this == regex_iterator(); }
 const value_type& operator*() const;
 const value_type* operator->() const;
 regex_iterator& operator++();
 regex_iterator operator++(int) = default;

 private:
 BidirectionalIterator begin; // exposition only
 BidirectionalIterator end; // exposition only
 const regex_type* pregex; // exposition only
 regex_constants::match_flag_type flags; // exposition only
 match_results<BidirectionalIterator> match; // exposition only
 };
}

```

- <sup>2</sup> An object of type `regex_iterator` that is not an end-of-sequence iterator holds a *zero-length match* if `match[0].matched == true` and `match[0].first == match[0].second`.

[Note 1: For example, this can occur when the part of the regular expression that matched consists only of an assertion (such as `'^'`, `'$'`, `'\b'`, `'\B'`). — end note]



## 28.6.11.1.2 Constructors

[re.regiter.cnstr]

```
regex_iterator();
```

1 *Effects:* Constructs an end-of-sequence iterator.

```
regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 regex_constants::match_flag_type m = regex_constants::match_default);
```

2 *Effects:* Initializes `begin` and `end` to `a` and `b`, respectively, sets `pregex` to `addressof(re)`, sets `flags` to `m`, then calls `regex_search(begin, end, match, *pregex, flags)`. If this call returns `false` the constructor sets `*this` to the end-of-sequence iterator.

## 28.6.11.1.3 Comparisons

[re.regiter.comp]

```
bool operator==(const regex_iterator& right) const;
```

1 *Returns:* `true` if `*this` and `right` are both end-of-sequence iterators or if the following conditions all hold:

- (1.1) — `begin == right.begin`,
  - (1.2) — `end == right.end`,
  - (1.3) — `pregex == right.pregex`,
  - (1.4) — `flags == right.flags`, and
  - (1.5) — `match[0] == right.match[0]`;
- otherwise `false`.

## 28.6.11.1.4 Indirection

[re.regiter.deref]

```
const value_type& operator*() const;
```

1 *Returns:* `match`.

```
const value_type* operator->() const;
```

2 *Returns:* `addressof(match)`.

## 28.6.11.1.5 Increment

[re.regiter.incr]

```
regex_iterator& operator++();
```

1 *Effects:* Constructs a local variable `start` of type `BidirectionalIterator` and initializes it with the value of `match[0].second`.

2 If the iterator holds a zero-length match and `start == end` the operator sets `*this` to the end-of-sequence iterator and returns `*this`.

3 Otherwise, if the iterator holds a zero-length match, the operator calls:

```
regex_search(start, end, match, *pregex,
 flags | regex_constants::match_not_null | regex_constants::match_continuous)
```

If the call returns `true` the operator returns `*this`. Otherwise the operator increments `start` and continues as if the most recent match was not a zero-length match.

4 If the most recent match was not a zero-length match, the operator sets `flags` to `flags | regex_constants::match_prev_avail` and calls `regex_search(start, end, match, *pregex, flags)`. If the call returns `false` the iterator sets `*this` to the end-of-sequence iterator. The iterator then returns `*this`.

5 In all cases in which the call to `regex_search` returns `true`, `match.prefix().first` shall be equal to the previous value of `match[0].second`, and for each index `i` in the half-open range `[0, match.size())` for which `match[i].matched` is `true`, `match.position(i)` shall return `distance(begin, match[i].first)`.

6 [Note 1: This means that `match.position(i)` gives the offset from the beginning of the target sequence, which is often not the same as the offset from the sequence passed in the call to `regex_search`. — end note]

7 It is unspecified how the implementation makes these adjustments.

- 8 [Note 2: This means that an implementation can call an implementation-specific search function, in which case a program-defined specialization of `regex_search` will not be called. — end note]

### 28.6.11.2 Class template `regex_token_iterator`

[re.tokiter]

#### 28.6.11.2.1 General

[re.tokiter.general]

- 1 The class template `regex_token_iterator` is an iterator adaptor; that is to say it represents a new view of an existing iterator sequence, by enumerating all the occurrences of a regular expression within that sequence, and presenting one or more sub-expressions for each match found. Each position enumerated by the iterator is a `sub_match` class template instance that represents what matched a particular sub-expression within the regular expression.
- 2 When class `regex_token_iterator` is used to enumerate a single sub-expression with index `-1` the iterator performs field splitting: that is to say it enumerates one sub-expression for each section of the character container sequence that does not match the regular expression specified.
- 3 After it is constructed, the iterator finds and stores a value `regex_iterator<BidirectionalIterator>` `position` and sets the internal count `N` to zero. It also maintains a sequence `subs` which contains a list of the sub-expressions which will be enumerated. Every time `operator++` is used the count `N` is incremented; if `N` exceeds or equals `subs.size()`, then the iterator increments member `position` and sets count `N` to zero.
- 4 If the end of sequence is reached (`position` is equal to the end of sequence iterator), the iterator becomes equal to the end-of-sequence iterator value, unless the sub-expression being enumerated has index `-1`, in which case the iterator enumerates one last sub-expression that contains all the characters from the end of the last regular expression match to the end of the input sequence being enumerated, provided that this would not be an empty sub-expression.
- 5 The default constructor constructs an end-of-sequence iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-sequence iterator is not defined. For any other iterator value a `const sub_match<BidirectionalIterator>&` is returned. The result of `operator->` on an end-of-sequence iterator is not defined. For any other iterator value a `const sub_match<BidirectionalIterator>*` is returned.
- 6 It is impossible to store things into `regex_token_iterator`s. Two end-of-sequence iterators are always equal. An end-of-sequence iterator is not equal to a non-end-of-sequence iterator. Two non-end-of-sequence iterators are equal when they are constructed from the same arguments.

```
namespace std {
 template<class BidirectionalIterator,
 class charT = typename iterator_traits<BidirectionalIterator>::value_type,
 class traits = regex_traits<charT>>
 class regex_token_iterator {
 public:
 using regex_type = basic_regex<charT, traits>;
 using iterator_category = forward_iterator_tag;
 using iterator_concept = input_iterator_tag;
 using value_type = sub_match<BidirectionalIterator>;
 using difference_type = ptrdiff_t;
 using pointer = const value_type*;
 using reference = const value_type&;

 regex_token_iterator();
 regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 int submatch = 0,
 regex_constants::match_flag_type m =
 regex_constants::match_default);
 regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 const vector<int>& submatches,
 regex_constants::match_flag_type m =
 regex_constants::match_default);
 regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 initializer_list<int> submatches,
 regex_constants::match_flag_type m =
```

```

 regex_constants::match_default);
template<size_t N>
 regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 const int (&submatches)[N],
 regex_constants::match_flag_type m =
 regex_constants::match_default);
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type&& re,
 int submatch = 0,
 regex_constants::match_flag_type m =
 regex_constants::match_default) = delete;
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type&& re,
 const vector<int>& submatches,
 regex_constants::match_flag_type m =
 regex_constants::match_default) = delete;
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type&& re,
 initializer_list<int> submatches,
 regex_constants::match_flag_type m =
 regex_constants::match_default) = delete;

template<size_t N>
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type&& re,
 const int (&submatches)[N],
 regex_constants::match_flag_type m =
 regex_constants::match_default) = delete;
regex_token_iterator(const regex_token_iterator&);
regex_token_iterator& operator=(const regex_token_iterator&);
bool operator==(const regex_token_iterator&) const;
bool operator==(default_sentinel_t) const { return *this == regex_token_iterator(); }
const value_type& operator*() const;
const value_type* operator->() const;
regex_token_iterator& operator++();
regex_token_iterator operator++(int) = default;

private:
 using position_iterator =
 regex_iterator<BidirectionalIterator, charT, traits>; // exposition only
 position_iterator position; // exposition only
 const value_type* result; // exposition only
 value_type suffix; // exposition only
 size_t N; // exposition only
 vector<int> subs; // exposition only
};
}

```

<sup>7</sup> A *suffix iterator* is a `regex_token_iterator` object that points to a final sequence of characters at the end of the target sequence. In a suffix iterator the member `result` holds a pointer to the data member `suffix`, the value of the member `suffix.matched` is `true`, `suffix.first` points to the beginning of the final sequence, and `suffix.second` points to the end of the final sequence.

<sup>8</sup> [Note 1: For a suffix iterator, data member `suffix.first` is the same as the end of the last match found, and `suffix.second` is the same as the end of the target sequence. — end note]

<sup>9</sup> The *current match* is `(*position).prefix()` if `subs[N] == -1`, or `(*position)[subs[N]]` for any other value of `subs[N]`.

#### 28.6.11.2.2 Constructors

[re.tokiter.cnstr]

```
regex_token_iterator();
```

<sup>1</sup> *Effects:* Constructs the end-of-sequence iterator.

```

regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 int submatch = 0,
 regex_constants::match_flag_type m = regex_constants::match_default);

regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 const vector<int>& submatches,
 regex_constants::match_flag_type m = regex_constants::match_default);

regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 initializer_list<int> submatches,
 regex_constants::match_flag_type m = regex_constants::match_default);

template<size_t N>
 regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
 const regex_type& re,
 const int (&submatches)[N],
 regex_constants::match_flag_type m = regex_constants::match_default);

```

2     *Preconditions:* Each of the initialization values of `submatches` is  $\geq -1$ .

3     *Effects:* The first constructor initializes the member `subs` to hold the single value `submatch`. The second, third, and fourth constructors initialize the member `subs` to hold a copy of the sequence of integer values pointed to by the iterator range `[begin(submatches), end(submatches))`.

4     Each constructor then sets `N` to 0, and `position` to `position_iterator(a, b, re, m)`. If `position` is not an end-of-sequence iterator the constructor sets `result` to the address of the current match. Otherwise if any of the values stored in `subs` is equal to  $-1$  the constructor sets `*this` to a suffix iterator that points to the range `[a, b)`, otherwise the constructor sets `*this` to an end-of-sequence iterator.

### 28.6.11.2.3 Comparisons

[re.tokiter.comp]

```
bool operator==(const regex_token_iterator& right) const;
```

1     *Returns:* `true` if `*this` and `right` are both end-of-sequence iterators, or if `*this` and `right` are both suffix iterators and `suffix == right.suffix`; otherwise returns `false` if `*this` or `right` is an end-of-sequence iterator or a suffix iterator. Otherwise returns `true` if `position == right.position`, `N == right.N`, and `subs == right.subs`. Otherwise returns `false`.

### 28.6.11.2.4 Indirection

[re.tokiter.deref]

```
const value_type& operator*() const;
```

1     *Returns:* `*result`.

```
const value_type* operator->() const;
```

2     *Returns:* `result`.

### 28.6.11.2.5 Increment

[re.tokiter.incr]

```
regex_token_iterator& operator++();
```

1     *Effects:* Constructs a local variable `prev` of type `position_iterator`, initialized with the value of `position`.

2     If `*this` is a suffix iterator, sets `*this` to an end-of-sequence iterator.

3     Otherwise, if `N + 1 < subs.size()`, increments `N` and sets `result` to the address of the current match.

4     Otherwise, sets `N` to 0 and increments `position`. If `position` is not an end-of-sequence iterator the operator sets `result` to the address of the current match.

5     Otherwise, if any of the values stored in `subs` is equal to  $-1$  and `prev->suffix().length()` is not 0 the operator sets `*this` to a suffix iterator that points to the range `[prev->suffix().first, prev->suffix().second)`.

6 Otherwise, sets **\*this** to an end-of-sequence iterator.

7 *Returns:* **\*this**.

### 28.6.12 Modified ECMAScript regular expression grammar [re.grammar]

- 1 The regular expression grammar recognized by **basic\_regex** objects constructed with the ECMAScript flag is that specified by ECMA-262, except as specified below.
- 2 Objects of type specialization of **basic\_regex** store within themselves a default-constructed instance of their **traits** template parameter, henceforth referred to as **traits\_inst**. This **traits\_inst** object is used to support localization of the regular expression; **basic\_regex** member functions shall not call any locale dependent C or C++ API, including the formatted string input functions. Instead they shall call the appropriate traits member function to achieve the required effect.
- 3 The following productions within the ECMAScript grammar are modified as follows:

```

ClassAtom ::
 -
 ClassAtomNoDash
 ClassAtomExClass
 ClassAtomCollatingElement
 ClassAtomEquivalence

```

```

IdentityEscape ::
 SourceCharacter but not c

```

- 4 The following new productions are then added:

```

ClassAtomExClass ::
 [: ClassName :]

ClassAtomCollatingElement ::
 [. ClassName .]

ClassAtomEquivalence ::
 [= ClassName =]

ClassName ::
 ClassNameCharacter
 ClassNameCharacter ClassName

ClassNameCharacter ::
 SourceCharacter but not one of . or = or :

```

- 5 The productions *ClassAtomExClass*, *ClassAtomCollatingElement* and *ClassAtomEquivalence* provide functionality equivalent to that of the same features in regular expressions in POSIX.
- 6 The regular expression grammar may be modified by any **regex\_constants::syntax\_option\_type** flags specified when constructing an object of type specialization of **basic\_regex** according to the rules in Table 118.
- 7 A *ClassName* production, when used in *ClassAtomExClass*, is not valid if **traits\_inst.lookup\_classname** returns zero for that name. The names recognized as valid *ClassNames* are determined by the type of the traits class, but at least the following names shall be recognized: **alnum**, **alpha**, **blank**, **cntrl**, **digit**, **graph**, **lower**, **print**, **punct**, **space**, **upper**, **xdigit**, **d**, **s**, **w**. In addition the following expressions shall be equivalent:

```

\d and [[:digit:]]

\D and [^[:digit:]]

\s and [[:space:]]

\S and [^[:space:]]

\w and [_[:alnum:]]

\W and [^_[:alnum:]]

```

- 8 A *ClassName* production when used in a *ClassAtomCollatingElement* production is not valid if the value returned by **traits\_inst.lookup\_collatename** for that name is an empty string.

- <sup>9</sup> The results from multiple calls to `traits_inst.lookup_classname` can be bitwise OR'ed together and subsequently passed to `traits_inst.isctype`.
- <sup>10</sup> A *ClassName* production when used in a *ClassAtomEquivalence* production is not valid if the value returned by `traits_inst.lookup_collatename` for that name is an empty string or if the value returned by `traits_inst.transform_primary` for the result of the call to `traits_inst.lookup_collatename` is an empty string.
- <sup>11</sup> When the sequence of characters being transformed to a finite state machine contains an invalid class name the translator shall throw an exception object of type `regex_error`.
- <sup>12</sup> If the *CV* of a *UnicodeEscapeSequence* is greater than the largest value that can be held in an object of type `charT` the translator shall throw an exception object of type `regex_error`.  
[Note 1: This means that values of the form `"\uxxxx"` that do not fit in a character are invalid. — end note]
- <sup>13</sup> Where the regular expression grammar requires the conversion of a sequence of characters to an integral value, this is accomplished by calling `traits_inst.value`.
- <sup>14</sup> The behavior of the internal finite state machine representation when used to match a sequence of characters is as described in ECMA-262. The behavior is modified according to any `match_flag_type` flags (28.6.4.3) specified when using the regular expression object in one of the regular expression algorithms (28.6.10). The behavior is also localized by interaction with the traits class template parameter as follows:
- (14.1) — During matching of a regular expression finite state machine against a sequence of characters, two characters `c` and `d` are compared using the following rules:
    - (14.1.1) — if `(flags() & regex_constants::icase)` the two characters are equal if `traits_inst.translate_nocase(c) == traits_inst.translate_nocase(d)`;
    - (14.1.2) — otherwise, if `flags() & regex_constants::collate` the two characters are equal if `traits_inst.translate(c) == traits_inst.translate(d)`;
    - (14.1.3) — otherwise, the two characters are equal if `c == d`.
  - (14.2) — During matching of a regular expression finite state machine against a sequence of characters, comparison of a collating element range `c1-c2` against a character `c` is conducted as follows: if `flags() & regex_constants::collate` is `false` then the character `c` is matched if `c1 <= c && c <= c2`, otherwise `c` is matched in accordance with the following algorithm:
 

```
string_type str1 = string_type(1,
 flags() & icase ?
 traits_inst.translate_nocase(c1) : traits_inst.translate(c1));
string_type str2 = string_type(1,
 flags() & icase ?
 traits_inst.translate_nocase(c2) : traits_inst.translate(c2));
string_type str = string_type(1,
 flags() & icase ?
 traits_inst.translate_nocase(c) : traits_inst.translate(c));
return traits_inst.transform(str1.begin(), str1.end())
 <= traits_inst.transform(str.begin(), str.end())
 && traits_inst.transform(str.begin(), str.end())
 <= traits_inst.transform(str2.begin(), str2.end());
```
  - (14.3) — During matching of a regular expression finite state machine against a sequence of characters, testing whether a collating element is a member of a primary equivalence class is conducted by first converting the collating element and the equivalence class to sort keys using `traits::transform_primary`, and then comparing the sort keys for equality.
  - (14.4) — During matching of a regular expression finite state machine against a sequence of characters, a character `c` is a member of a character class designated by an iterator range `[first,last)` if `traits_inst.isctype(c, traits_inst.lookup_classname(first, last, flags() & icase))` is true.

SEE ALSO: ECMA-262 15.10

## 28.7 Null-terminated sequence utilities

[text.c.strings]

### 28.7.1 Header `<cctype>` synopsis

[cctype.syn]

```
namespace std {
 int isalnum(int c);
```

```

int isalpha(int c);
int isblank(int c);
int iscntrl(int c);
int isdigit(int c);
int isgraph(int c);
int islower(int c);
int isprint(int c);
int ispunct(int c);
int isspace(int c);
int isupper(int c);
int isxdigit(int c);
int tolower(int c);
int toupper(int c);
}

```

- <sup>1</sup> The contents and meaning of the header <cctype> are the same as the C standard library header <ctype.h>.

SEE ALSO: ISO/IEC 9899:2024, 7.4

### 28.7.2 Header <cwctype> synopsis

[cwctype.syn]

```

namespace std {
 using wint_t = see below;
 using wctrans_t = see below;
 using wctype_t = see below;

 int iswalnum(wint_t wc);
 int iswalpha(wint_t wc);
 int iswblank(wint_t wc);
 int iswcntrl(wint_t wc);
 int iswdigit(wint_t wc);
 int iswgraph(wint_t wc);
 int iswlower(wint_t wc);
 int iswprint(wint_t wc);
 int iswpunct(wint_t wc);
 int iswspace(wint_t wc);
 int iswupper(wint_t wc);
 int iswxdigit(wint_t wc);
 int iswctype(wint_t wc, wctype_t desc);
 wctype_t wctype(const char* property);
 wint_t tolower(wint_t wc);
 wint_t toupper(wint_t wc);
 wint_t towctrans(wint_t wc, wctrans_t desc);
 wctrans_t wctrans(const char* property);
}

```

#define WEOF *see below*

- <sup>1</sup> The contents and meaning of the header <cwctype> are the same as the C standard library header <wctype.h>.

SEE ALSO: ISO/IEC 9899:2024, 7.32

### 28.7.3 Header <wchar> synopsis

[wchar.syn]

```
#define __STDC_VERSION_WCHAR_H__ 202311L
```

```

namespace std {
 using size_t = see 17.2.4; // freestanding
 using mbstate_t = see below; // freestanding
 using wint_t = see below; // freestanding

 struct tm;

 int fwprintf(FILE* stream, const wchar_t* format, ...);
 int fwcscanf(FILE* stream, const wchar_t* format, ...);
 int swprintf(wchar_t* s, size_t n, const wchar_t* format, ...);
 int swscanf(const wchar_t* s, const wchar_t* format, ...);
}

```

```

int vfwprintf(FILE* stream, const wchar_t* format, va_list arg);
int vfwscanf(FILE* stream, const wchar_t* format, va_list arg);
int vswprintf(wchar_t* s, size_t n, const wchar_t* format, va_list arg);
int vswscanf(const wchar_t* s, const wchar_t* format, va_list arg);
int vwprintf(const wchar_t* format, va_list arg);
int vwscanf(const wchar_t* format, va_list arg);
int wprintf(const wchar_t* format, ...);
int wscanf(const wchar_t* format, ...);
wint_t fgetwc(FILE* stream);
wchar_t* fgetws(wchar_t* s, int n, FILE* stream);
wint_t fputwc(wchar_t c, FILE* stream);
int fputws(const wchar_t* s, FILE* stream);
int fwide(FILE* stream, int mode);
wint_t getwc(FILE* stream);
wint_t getwchar();
wint_t putwc(wchar_t c, FILE* stream);
wint_t putwchar(wchar_t c);
wint_t ungetwc(wint_t c, FILE* stream);
double wcstod(const wchar_t* nptr, wchar_t** endptr);
float wcstof(const wchar_t* nptr, wchar_t** endptr);
long double wcstold(const wchar_t* nptr, wchar_t** endptr);
long int wcstol(const wchar_t* nptr, wchar_t** endptr, int base);
long long int wcstoll(const wchar_t* nptr, wchar_t** endptr, int base);
unsigned long int wcstoul(const wchar_t* nptr, wchar_t** endptr, int base);
unsigned long long int wcstoull(const wchar_t* nptr, wchar_t** endptr, int base);
wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2); // freestanding
wchar_t* wcsncpy(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wmemcpy(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wmemmove(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
wchar_t* wscat(wchar_t* s1, const wchar_t* s2); // freestanding
wchar_t* wcsncat(wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
int wscmp(const wchar_t* s1, const wchar_t* s2); // freestanding
int wscoll(const wchar_t* s1, const wchar_t* s2);
int wcsncmp(const wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
size_t wcsxfrm(wchar_t* s1, const wchar_t* s2, size_t n);
int wmemcmp(const wchar_t* s1, const wchar_t* s2, size_t n); // freestanding
const wchar_t* wcschr(const wchar_t* s, wchar_t c); // freestanding; see 16.2
wchar_t* wcschr(wchar_t* s, wchar_t c); // freestanding; see 16.2
size_t wcslen(const wchar_t* s1, const wchar_t* s2); // freestanding
const wchar_t* wcsrchr(const wchar_t* s1, const wchar_t* s2); // freestanding; see 16.2
wchar_t* wcsrchr(wchar_t* s, wchar_t c); // freestanding; see 16.2
size_t wcsspncpy(const wchar_t* s1, const wchar_t* s2); // freestanding
const wchar_t* wcsstr(const wchar_t* s1, const wchar_t* s2); // freestanding; see 16.2
wchar_t* wcsstr(wchar_t* s1, const wchar_t* s2); // freestanding; see 16.2
wchar_t* wcstok(wchar_t* s1, const wchar_t* s2, wchar_t** ptr); // freestanding
const wchar_t* wmemchr(const wchar_t* s, wchar_t c, size_t n); // freestanding; see 16.2
wchar_t* wmemchr(wchar_t* s, wchar_t c, size_t n); // freestanding; see 16.2
size_t wcslen(const wchar_t* s); // freestanding
wchar_t* wmemset(wchar_t* s, wchar_t c, size_t n); // freestanding
size_t wcsftime(wchar_t* s, size_t maxsize, const wchar_t* format, const tm* timeptr);
wint_t btowc(int c);
int wctob(wint_t c);

// 28.7.5, multibyte / wide string and character conversion functions
int mbsinit(const mbstate_t* ps);
size_t mbrlen(const char* s, size_t n, mbstate_t* ps);
size_t mbrtowc(wchar_t* pwc, const char* s, size_t n, mbstate_t* ps);
size_t wctomb(char* s, wchar_t wc, mbstate_t* ps);
size_t mbsrtowcs(wchar_t* dst, const char** src, size_t len, mbstate_t* ps);
size_t wcsrtombs(char* dst, const wchar_t** src, size_t len, mbstate_t* ps);
}

```



```

#define NULL see 17.2.3 // freestanding
#define WCHAR_MAX see below // freestanding
#define WCHAR_MIN see below // freestanding
#define WEOF see below // freestanding
#define WCHAR_WIDTH see below // freestanding

```

- <sup>1</sup> The contents and meaning of the header `<wchar>` are the same as the C standard library header `<wchar.h>`, except that it does not declare a type `wchar_t`.
- <sup>2</sup> [Note 1: The functions `wcschr`, `wcspbrk`, `wcsrchr`, `wcsstr`, and `wmemchr` have different signatures in this document, but they have the same behavior as in the C standard library (16.2). — end note]

SEE ALSO: ISO/IEC 9899:2024, 7.31

#### 28.7.4 Header `<cuchar>` synopsis

[`cuchar.syn`]

```

#define __STDC_VERSION_UCHAR_H__ 202311L

namespace std {
 using mbstate_t = see below;
 using size_t = see 17.2.4;

 size_t mbrtoc8(char8_t* pc8, const char* s, size_t n, mbstate_t* ps);
 size_t c8rtomb(char* s, char8_t c8, mbstate_t* ps);
 size_t mbrtoc16(char16_t* pc16, const char* s, size_t n, mbstate_t* ps);
 size_t c16rtomb(char* s, char16_t c16, mbstate_t* ps);
 size_t mbrtoc32(char32_t* pc32, const char* s, size_t n, mbstate_t* ps);
 size_t c32rtomb(char* s, char32_t c32, mbstate_t* ps);
}

```

- <sup>1</sup> The contents and meaning of the header `<cuchar>` are the same as the C standard library header `<uchar.h>`, except that it does not declare types `char8_t`, `char16_t`, or `char32_t`.

SEE ALSO: ISO/IEC 9899:2024, 7.30

#### 28.7.5 Multibyte / wide string and character conversion functions

[`c.mb.wcs`]

- <sup>1</sup> [Note 1: The headers `<stdlib>` (17.2.2), `<cuchar>` (28.7.4), and `<wchar>` (28.7.3) declare the functions described in this subclause. — end note]

```

int mbsinit(const mbstate_t* ps);
int mblen(const char* s, size_t n);
size_t mbstowcs(wchar_t* pwcs, const char* s, size_t n);
size_t wcstombs(char* s, const wchar_t* pwcs, size_t n);

```

- <sup>2</sup> *Effects:* These functions have the semantics specified in the C standard library.

SEE ALSO: ISO/IEC 9899:2024, 7.24.8.2, 7.24.9, 7.31.6.3.1

```

int mbtowc(wchar_t* pwc, const char* s, size_t n);
int wctomb(char* s, wchar_t wchar);

```

- <sup>3</sup> *Effects:* These functions have the semantics specified in the C standard library.

- <sup>4</sup> *Remarks:* Calls to these functions may introduce a data race (16.4.6.10) with other calls to the same function.

SEE ALSO: ISO/IEC 9899:2024, 7.24.8

```

size_t mbrlen(const char* s, size_t n, mbstate_t* ps);
size_t mbrtowc(wchar_t* pwc, const char* s, size_t n, mbstate_t* ps);
size_t wcrctomb(char* s, wchar_t wc, mbstate_t* ps);
size_t mbrtoc8(char8_t* pc8, const char* s, size_t n, mbstate_t* ps);
size_t c8rtomb(char* s, char8_t c8, mbstate_t* ps);
size_t mbrtoc16(char16_t* pc16, const char* s, size_t n, mbstate_t* ps);
size_t c16rtomb(char* s, char16_t c16, mbstate_t* ps);
size_t mbrtoc32(char32_t* pc32, const char* s, size_t n, mbstate_t* ps);
size_t c32rtomb(char* s, char32_t c32, mbstate_t* ps);
size_t mbsrtowcs(wchar_t* dst, const char** src, size_t len, mbstate_t* ps);

```

```
size_t wcsrtombs(char* dst, const wchar_t** src, size_t len, mbstate_t* ps);
```

5     *Effects:* These functions have the semantics specified in the C standard library.

6     *Remarks:* Calling these functions with an `mbstate_t*` argument that is a null pointer value may introduce a data race ([16.4.6.10](#)) with other calls to the same function with an `mbstate_t*` argument that is a null pointer value.

SEE ALSO: ISO/IEC 9899:2024, 7.30.2, 7.31.6.4, 7.31.6.5

## 29 Numerics library

[numerics]

### 29.1 General

[numerics.general]

- <sup>1</sup> This Clause describes components that C++ programs may use to perform seminumerical operations.
- <sup>2</sup> The following subclauses describe components for complex number types, random number generation, numeric ( $n$ -at-a-time) arrays, generalized numeric algorithms, and mathematical constants and functions for floating-point types, as summarized in Table 125.

Table 125 — Numerics library summary [tab:numerics.summary]

| Subclause                                                            | Header             |
|----------------------------------------------------------------------|--------------------|
| <a href="#">29.2</a> Requirements                                    |                    |
| <a href="#">29.3</a> Floating-point environment                      | <cfenv>            |
| <a href="#">29.4</a> Complex numbers                                 | <complex>          |
| <a href="#">29.5</a> Random number generation                        | <random>           |
| <a href="#">29.6</a> Numeric arrays                                  | <valarray>         |
| <a href="#">29.7</a> Mathematical functions for floating-point types | <cmath>, <cstdlib> |
| <a href="#">29.8</a> Numbers                                         | <numbers>          |
| <a href="#">29.9</a> Linear algebra                                  | <linalg>           |
| <a href="#">29.10</a> Data-parallel types                            | <simd>             |

### 29.2 Numeric type requirements

[numeric.requirements]

- <sup>1</sup> The `complex` and `valarray` components are parameterized by the type of information they contain and manipulate. A C++ program shall instantiate these components only with a numeric type. A *numeric type* is a cv-unqualified object type `T` that meets the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, and *Cpp17Destructible* requirements (16.4.4.2).<sup>218</sup>
- <sup>2</sup> If any operation on `T` throws an exception the effects are undefined.
- <sup>3</sup> In addition, many member and related functions of `valarray<T>` can be successfully instantiated and will exhibit well-defined behavior if and only if `T` meets additional requirements specified for each such member or related function.
- <sup>4</sup> [Example 1: It is valid to instantiate `valarray<complex>`, but `operator<>()` will not be successfully instantiated for `valarray<complex>` operands, since `complex` does not have any ordering operators. — end example]

### 29.3 The floating-point environment

[cfenv]

#### 29.3.1 Header <cfenv> synopsis

[cfenv.syn]

```
#define FE_ALL_EXCEPT see below
#define FE_DIVBYZERO see below // optional
#define FE_INEXACT see below // optional
#define FE_INVALID see below // optional
#define FE_OVERFLOW see below // optional
#define FE_UNDERFLOW see below // optional

#define FE_DOWNWARD see below // optional
#define FE_TONEAREST see below // optional
#define FE_TOWARDZERO see below // optional
#define FE_UPWARD see below // optional

#define FE_DFL_ENV see below
```

<sup>218</sup> In other words, value types. These include arithmetic types, pointers, the library class `complex`, and specializations of `valarray` for value types.

```

namespace std {
 // types
 using fenv_t = object type;
 using fexcept_t = object type;

 // functions
 int feclearexcept(int except);
 int fegetexceptflag(fexcept_t* pflag, int except);
 int feraiseexcept(int except);
 int fesetexceptflag(const fexcept_t* pflag, int except);
 int fetestexcept(int except);

 int fegetround();
 int fesetround(int mode);

 int fegetenv(fenv_t* penv);
 int feholdexcept(fenv_t* penv);
 int fesetenv(const fenv_t* penv);
 int feupdateenv(const fenv_t* penv);
}

```

- <sup>1</sup> The contents and meaning of the header `<cfenv>` are a subset of the C standard library header `<fenv.h>` and only the declarations shown in the synopsis above are present.

[*Note 1*: This document does not require an implementation to support the `FENV_ACCESS` pragma; it is implementation-defined (15.10) whether the pragma is supported. As a consequence, it is implementation-defined whether these functions can be used to test floating-point status flags, set floating-point control modes, or run under non-default mode settings. If the pragma is used to enable control over the floating-point environment, this document does not specify the effect on floating-point evaluation in constant expressions. — *end note*]

SEE ALSO: ISO/IEC 9899:2024, 7.6

### 29.3.2 Threads

[`cfenv.thread`]

- <sup>1</sup> The floating-point environment has thread storage duration (6.8.6.3). The initial state for a thread's floating-point environment is the state of the floating-point environment of the thread that constructs the corresponding `thread` object (32.4.3) or `jthread` object (32.4.4) at the time it constructed the object.

[*Note 1*: That is, the child thread gets the floating-point state of the parent thread at the time of the child's creation. — *end note*]

- <sup>2</sup> A separate floating-point environment is maintained for each thread. Each function accesses the environment corresponding to its calling thread.

## 29.4 Complex numbers

[`complex.numbers`]

### 29.4.1 General

[`complex.numbers.general`]

- <sup>1</sup> The header `<complex>` defines a class template, and numerous functions for representing and manipulating complex numbers.
- <sup>2</sup> The effect of instantiating the primary template of `complex` for any type that is not a cv-unqualified floating-point type (6.9.3) is unspecified. Specializations of `complex` for cv-unqualified floating-point types are trivially copyable literal types (6.9.1).
- <sup>3</sup> If the result of a function is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.
- <sup>4</sup> If `z` is an lvalue of type `cv complex<T>` then:

- (4.1) — the expression `reinterpret_cast<cv T(&)[2]>(z)` is well-formed,
- (4.2) — `reinterpret_cast<cv T(&)[2]>(z)[0]` designates the real part of `z`, and
- (4.3) — `reinterpret_cast<cv T(&)[2]>(z)[1]` designates the imaginary part of `z`.

Moreover, if `a` is an expression of type `cv complex<T>*` and the expression `a[i]` is well-defined for an integer expression `i`, then:

- (4.4) — `reinterpret_cast<cv T*>(a)[2 * i]` designates the real part of `a[i]`, and
- (4.5) — `reinterpret_cast<cv T*>(a)[2 * i + 1]` designates the imaginary part of `a[i]`.

## 29.4.2 Header &lt;complex&gt; synopsis

[complex.syn]

```

namespace std {
 // 29.4.3, class template complex
 template<class T> class complex;

 // 29.4.6, operators
 template<class T> constexpr complex<T> operator+(const complex<T>&, const complex<T>&);
 template<class T> constexpr complex<T> operator+(const complex<T>&, const T&);
 template<class T> constexpr complex<T> operator+(const T&, const complex<T>&);

 template<class T> constexpr complex<T> operator-(const complex<T>&, const complex<T>&);
 template<class T> constexpr complex<T> operator-(const complex<T>&, const T&);
 template<class T> constexpr complex<T> operator-(const T&, const complex<T>&);

 template<class T> constexpr complex<T> operator*(const complex<T>&, const complex<T>&);
 template<class T> constexpr complex<T> operator*(const complex<T>&, const T&);
 template<class T> constexpr complex<T> operator*(const T&, const complex<T>&);

 template<class T> constexpr complex<T> operator/(const complex<T>&, const complex<T>&);
 template<class T> constexpr complex<T> operator/(const complex<T>&, const T&);
 template<class T> constexpr complex<T> operator/(const T&, const complex<T>&);

 template<class T> constexpr complex<T> operator+(const complex<T>&);
 template<class T> constexpr complex<T> operator-(const complex<T>&);

 template<class T> constexpr bool operator==(const complex<T>&, const complex<T>&);
 template<class T> constexpr bool operator==(const complex<T>&, const T&);

 template<class T, class charT, class traits>
 basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, complex<T>&);

 template<class T, class charT, class traits>
 basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const complex<T>&);

 // 29.4.7, values
 template<class T> constexpr T real(const complex<T>&);
 template<class T> constexpr T imag(const complex<T>&);

 template<class T> constexpr T abs(const complex<T>&);
 template<class T> constexpr T arg(const complex<T>&);
 template<class T> constexpr T norm(const complex<T>&);

 template<class T> constexpr complex<T> conj(const complex<T>&);
 template<class T> constexpr complex<T> proj(const complex<T>&);
 template<class T> constexpr complex<T> polar(const T&, const T& = T());

 // 29.4.8, transcendentals
 template<class T> constexpr complex<T> acos(const complex<T>&);
 template<class T> constexpr complex<T> asin(const complex<T>&);
 template<class T> constexpr complex<T> atan(const complex<T>&);

 template<class T> constexpr complex<T> acosh(const complex<T>&);
 template<class T> constexpr complex<T> asinh(const complex<T>&);
 template<class T> constexpr complex<T> atanh(const complex<T>&);

 template<class T> constexpr complex<T> cos (const complex<T>&);
 template<class T> constexpr complex<T> cosh (const complex<T>&);
 template<class T> constexpr complex<T> exp (const complex<T>&);
 template<class T> constexpr complex<T> log (const complex<T>&);
 template<class T> constexpr complex<T> log10(const complex<T>&);

 template<class T> constexpr complex<T> pow (const complex<T>&, const T&);
 template<class T> constexpr complex<T> pow (const complex<T>&, const complex<T>&);
 template<class T> constexpr complex<T> pow (const T&, const complex<T>&);

```

```

template<class T> constexpr complex<T> sin (const complex<T>&);
template<class T> constexpr complex<T> sinh (const complex<T>&);
template<class T> constexpr complex<T> sqrt (const complex<T>&);
template<class T> constexpr complex<T> tan (const complex<T>&);
template<class T> constexpr complex<T> tanh (const complex<T>&);

// 29.4.9, tuple interface
template<class T> struct tuple_size;
template<size_t I, class T> struct tuple_element;
template<class T> struct tuple_size<complex<T>>;
template<size_t I, class T> struct tuple_element<I, complex<T>>;
template<size_t I, class T>
 constexpr T& get(complex<T>&) noexcept;
template<size_t I, class T>
 constexpr T&& get(complex<T>&&) noexcept;
template<size_t I, class T>
 constexpr const T& get(const complex<T>&) noexcept;
template<size_t I, class T>
 constexpr const T&& get(const complex<T>&&) noexcept;

// 29.4.11, complex literals
inline namespace literals {
 inline namespace complex_literals {
 constexpr complex<long double> operator""il(long double);
 constexpr complex<long double> operator""il(unsigned long long);
 constexpr complex<double> operator""i(long double);
 constexpr complex<double> operator""i(unsigned long long);
 constexpr complex<float> operator""if(long double);
 constexpr complex<float> operator""if(unsigned long long);
 }
}

```

### 29.4.3 Class template complex

[complex]

```

namespace std {
 template<class T> class complex {
 public:
 using value_type = T;

 constexpr complex(const T& re = T(), const T& im = T());
 constexpr complex(const complex&) = default;
 template<class X> constexpr explicit(see below) complex(const complex<X>&);

 constexpr T real() const;
 constexpr void real(T);
 constexpr T imag() const;
 constexpr void imag(T);

 constexpr complex& operator= (const T&);
 constexpr complex& operator+=(const T&);
 constexpr complex& operator-=(const T&);
 constexpr complex& operator*=(const T&);
 constexpr complex& operator/=(const T&);

 constexpr complex& operator=(const complex&);
 template<class X> constexpr complex& operator= (const complex<X>&);
 template<class X> constexpr complex& operator+=(const complex<X>&);
 template<class X> constexpr complex& operator-=(const complex<X>&);
 template<class X> constexpr complex& operator*=(const complex<X>&);
 template<class X> constexpr complex& operator/=(const complex<X>&);
 };
}

```

- <sup>1</sup> The class `complex` describes an object that can store the Cartesian components, `real()` and `imag()`, of a complex number.

#### 29.4.4 Member functions

[`complex.members`]

```
constexpr complex(const T& re = T(), const T& im = T());
```

- <sup>1</sup> *Postconditions:* `real() == re` && `imag() == im` is true.

```
template<class X> constexpr explicit(see below) complex(const complex<X>& other);
```

- <sup>2</sup> *Effects:* Initializes the real part with `other.real()` and the imaginary part with `other.imag()`.

- <sup>3</sup> *Remarks:* The expression inside `explicit` evaluates to `false` if and only if the floating-point conversion rank of `T` is greater than or equal to the floating-point conversion rank of `X`.

```
constexpr T real() const;
```

- <sup>4</sup> *Returns:* The value of the real component.

```
constexpr void real(T val);
```

- <sup>5</sup> *Effects:* Assigns `val` to the real component.

```
constexpr T imag() const;
```

- <sup>6</sup> *Returns:* The value of the imaginary component.

```
constexpr void imag(T val);
```

- <sup>7</sup> *Effects:* Assigns `val` to the imaginary component.

#### 29.4.5 Member operators

[`complex.member.ops`]

```
constexpr complex& operator+=(const T& rhs);
```

- <sup>1</sup> *Effects:* Adds the scalar value `rhs` to the real part of the complex value `*this` and stores the result in the real part of `*this`, leaving the imaginary part unchanged.

- <sup>2</sup> *Returns:* `*this`.

```
constexpr complex& operator-=(const T& rhs);
```

- <sup>3</sup> *Effects:* Subtracts the scalar value `rhs` from the real part of the complex value `*this` and stores the result in the real part of `*this`, leaving the imaginary part unchanged.

- <sup>4</sup> *Returns:* `*this`.

```
constexpr complex& operator*=(const T& rhs);
```

- <sup>5</sup> *Effects:* Multiplies the scalar value `rhs` by the complex value `*this` and stores the result in `*this`.

- <sup>6</sup> *Returns:* `*this`.

```
constexpr complex& operator/=(const T& rhs);
```

- <sup>7</sup> *Effects:* Divides the scalar value `rhs` into the complex value `*this` and stores the result in `*this`.

- <sup>8</sup> *Returns:* `*this`.

```
template<class X> constexpr complex& operator=(const complex<X>& rhs);
```

- <sup>9</sup> *Effects:* Assigns the value `rhs.real()` to the real part and the value `rhs.imag()` to the imaginary part of the complex value `*this`.

- <sup>10</sup> *Returns:* `*this`.

```
template<class X> constexpr complex& operator+=(const complex<X>& rhs);
```

- <sup>11</sup> *Effects:* Adds the complex value `rhs` to the complex value `*this` and stores the sum in `*this`.

- <sup>12</sup> *Returns:* `*this`.

```
template<class X> constexpr complex& operator-=(const complex<X>& rhs);
```

- <sup>13</sup> *Effects:* Subtracts the complex value `rhs` from the complex value `*this` and stores the difference in `*this`.

14     *Returns:* `*this`.

```
template<class X> constexpr complex& operator*=(const complex<X>& rhs);
```

15     *Effects:* Multiplies the complex value `rhs` by the complex value `*this` and stores the product in `*this`.

16     *Returns:* `*this`.

```
template<class X> constexpr complex& operator/=(const complex<X>& rhs);
```

17     *Effects:* Divides the complex value `rhs` into the complex value `*this` and stores the quotient in `*this`.

18     *Returns:* `*this`.

## 29.4.6 Non-member operations

[complex.ops]

```
template<class T> constexpr complex<T> operator+(const complex<T>& lhs);
```

1     *Returns:* `complex<T>(lhs)`.

```
template<class T> constexpr complex<T> operator+(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr complex<T> operator+(const complex<T>& lhs, const T& rhs);
template<class T> constexpr complex<T> operator+(const T& lhs, const complex<T>& rhs);
```

2     *Returns:* `complex<T>(lhs) += rhs`.

```
template<class T> constexpr complex<T> operator-(const complex<T>& lhs);
```

3     *Returns:* `complex<T>(-lhs.real(), -lhs.imag())`.

```
template<class T> constexpr complex<T> operator-(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr complex<T> operator-(const complex<T>& lhs, const T& rhs);
template<class T> constexpr complex<T> operator-(const T& lhs, const complex<T>& rhs);
```

4     *Returns:* `complex<T>(lhs) -= rhs`.

```
template<class T> constexpr complex<T> operator*(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr complex<T> operator*(const complex<T>& lhs, const T& rhs);
template<class T> constexpr complex<T> operator*(const T& lhs, const complex<T>& rhs);
```

5     *Returns:* `complex<T>(lhs) *= rhs`.

```
template<class T> constexpr complex<T> operator/(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr complex<T> operator/(const complex<T>& lhs, const T& rhs);
template<class T> constexpr complex<T> operator/(const T& lhs, const complex<T>& rhs);
```

6     *Returns:* `complex<T>(lhs) /= rhs`.

```
template<class T> constexpr bool operator==(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator==(const complex<T>& lhs, const T& rhs);
```

7     *Returns:* `lhs.real() == rhs.real() && lhs.imag() == rhs.imag()`.

8     *Remarks:* The imaginary part is assumed to be `T()`, or `0.0`, for the `T` arguments.

```
template<class T, class charT, class traits>
basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>& is, complex<T>& x);
```

9     *Preconditions:* The input values are convertible to `T`.

10    *Effects:* Extracts a complex number `x` of the form: `u`, `(u)`, or `(u,v)`, where `u` is the real part and `v` is the imaginary part (31.7.5.3).

11    If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` (31.5.4.4)).

12    *Returns:* `is`.

13    *Remarks:* This extraction is performed as a series of simpler extractions. Therefore, the skipping of whitespace is specified to be the same for each of the simpler extractions.

```
template<class T, class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& o, const complex<T>& x);
```

14    *Effects:* Inserts the complex number `x` onto the stream `o` as if it were implemented as follows:



```

 basic_ostringstream<charT, traits> s;
 s.flags(o.flags());
 s.imbue(o.getloc());
 s.precision(o.precision());
 s << '(' << x.real() << ',' << x.imag() << ')';
 return o << s.str();

```

15 [Note 1: In a locale in which comma is used as a decimal point character, the use of comma as a field separator can be ambiguous. Inserting `showpoint` into the output stream forces all outputs to show an explicit decimal point character; as a result, all inserted sequences of complex numbers can be extracted unambiguously. — end note]

### 29.4.7 Value operations

[complex.value.ops]

```
template<class T> constexpr T real(const complex<T>& x);
```

1 *Returns:* `x.real()`.

```
template<class T> constexpr T imag(const complex<T>& x);
```

2 *Returns:* `x.imag()`.

```
template<class T> constexpr T abs(const complex<T>& x);
```

3 *Returns:* The magnitude of `x`.

```
template<class T> constexpr T arg(const complex<T>& x);
```

4 *Returns:* The phase angle of `x`, or `atan2(imag(x), real(x))`.

```
template<class T> constexpr T norm(const complex<T>& x);
```

5 *Returns:* The squared magnitude of `x`.

```
template<class T> constexpr complex<T> conj(const complex<T>& x);
```

6 *Returns:* The complex conjugate of `x`.

```
template<class T> constexpr complex<T> proj(const complex<T>& x);
```

7 *Returns:* The projection of `x` onto the Riemann sphere.

8 *Remarks:* Behaves the same as the C function `cproj`. SEE ALSO: ISO/IEC 9899:2024, 7.3.9.5

```
template<class T> constexpr complex<T> polar(const T& rho, const T& theta = T());
```

9 *Preconditions:* `rho` is non-negative and non-NaN. `theta` is finite.

10 *Returns:* The complex value corresponding to a complex number whose magnitude is `rho` and whose phase angle is `theta`.

### 29.4.8 Transcendentals

[complex.transcendentals]

```
template<class T> constexpr complex<T> acos(const complex<T>& x);
```

1 *Returns:* The complex arc cosine of `x`.

2 *Remarks:* Behaves the same as the C function `cacos`. SEE ALSO: ISO/IEC 9899:2024, 7.3.5.1

```
template<class T> constexpr complex<T> asin(const complex<T>& x);
```

3 *Returns:* The complex arc sine of `x`.

4 *Remarks:* Behaves the same as the C function `casin`. SEE ALSO: ISO/IEC 9899:2024, 7.3.5.2

```
template<class T> constexpr complex<T> atan(const complex<T>& x);
```

5 *Returns:* The complex arc tangent of `x`.

6 *Remarks:* Behaves the same as the C function `catan`. SEE ALSO: ISO/IEC 9899:2024, 7.3.5.3

```
template<class T> constexpr complex<T> acosh(const complex<T>& x);
```

7 *Returns:* The complex arc hyperbolic cosine of `x`.

8 *Remarks:* Behaves the same as the C function `cacosh`. SEE ALSO: ISO/IEC 9899:2024, 7.3.6.1

```
template<class T> constexpr complex<T> asinh(const complex<T>& x);
```

*Returns:* The complex arc hyperbolic sine of  $x$ .

*Remarks:* Behaves the same as the C function `casinh`. SEE ALSO: ISO/IEC 9899:2024, 7.3.6.2

```
template<class T> constexpr complex<T> atanh(const complex<T>& x);
```

*Returns:* The complex arc hyperbolic tangent of  $x$ .

*Remarks:* Behaves the same as the C function `catanh`. SEE ALSO: ISO/IEC 9899:2024, 7.3.6.3

```
template<class T> constexpr complex<T> cos(const complex<T>& x);
```

*Returns:* The complex cosine of  $x$ .

```
template<class T> constexpr complex<T> cosh(const complex<T>& x);
```

*Returns:* The complex hyperbolic cosine of  $x$ .

```
template<class T> constexpr complex<T> exp(const complex<T>& x);
```

*Returns:* The complex base- $e$  exponential of  $x$ .

```
template<class T> constexpr complex<T> log(const complex<T>& x);
```

*Returns:* The complex natural (base- $e$ ) logarithm of  $x$ . For all  $x$ , `imag(log(x))` lies in the interval  $[-\pi, \pi]$ .

[*Note 1:* The semantics of this function are intended to be the same in C++ as they are for `clog` in C. — end note]

*Remarks:* The branch cuts are along the negative real axis.

```
template<class T> constexpr complex<T> log10(const complex<T>& x);
```

*Returns:* The complex common (base-10) logarithm of  $x$ , defined as  $\log(x) / \log(10)$ .

*Remarks:* The branch cuts are along the negative real axis.

```
template<class T> constexpr complex<T> pow(const complex<T>& x, const complex<T>& y);
```

```
template<class T> constexpr complex<T> pow(const complex<T>& x, const T& y);
```

```
template<class T> constexpr complex<T> pow(const T& x, const complex<T>& y);
```

*Returns:* The complex power of base  $x$  raised to the  $y^{\text{th}}$  power, defined as  $\exp(y * \log(x))$ . The value returned for `pow(0, 0)` is implementation-defined.

*Remarks:* The branch cuts are along the negative real axis.

```
template<class T> constexpr complex<T> sin(const complex<T>& x);
```

*Returns:* The complex sine of  $x$ .

```
template<class T> constexpr complex<T> sinh(const complex<T>& x);
```

*Returns:* The complex hyperbolic sine of  $x$ .

```
template<class T> constexpr complex<T> sqrt(const complex<T>& x);
```

*Returns:* The complex square root of  $x$ , in the range of the right half-plane.

[*Note 2:* The semantics of this function are intended to be the same in C++ as they are for `csqrt` in C. — end note]

*Remarks:* The branch cuts are along the negative real axis.

```
template<class T> constexpr complex<T> tan(const complex<T>& x);
```

*Returns:* The complex tangent of  $x$ .

```
template<class T> constexpr complex<T> tanh(const complex<T>& x);
```

*Returns:* The complex hyperbolic tangent of  $x$ .

### 29.4.9 Tuple interface

[`complex.tuple`]

```
template<class T>
```

```
struct tuple_size<complex<T>> : integral_constant<size_t, 2> {};
```

```
template<size_t I, class T>
struct tuple_element<I, complex<T>> {
 using type = T;
};
```

1 *Mandates:*  $I < 2$  is true.

```
template<size_t I, class T>
constexpr T& get(complex<T>& z) noexcept;
template<size_t I, class T>
constexpr T&& get(complex<T>&& z) noexcept;
template<size_t I, class T>
constexpr const T& get(const complex<T>& z) noexcept;
template<size_t I, class T>
constexpr const T&& get(const complex<T>&& z) noexcept;
```

2 *Mandates:*  $I < 2$  is true.

3 *Returns:* A reference to the real part of  $z$  if  $I == 0$  is true, otherwise a reference to the imaginary part of  $z$ .

### 29.4.10 Additional overloads

[cmplx.over]

1 The following function templates have additional constexpr overloads:

|      |      |
|------|------|
| arg  | norm |
| conj | proj |
| imag | real |

2 The additional constexpr overloads are sufficient to ensure:

- (2.1) — If the argument has a floating-point type  $T$ , then it is effectively cast to `complex<T>`.
- (2.2) — Otherwise, if the argument has integer type, then it is effectively cast to `complex<double>`.

3 Function template `pow` has additional constexpr overloads sufficient to ensure, for a call with one argument of type `complex<T1>` and the other argument of type  $T2$  or `complex<T2>`, both arguments are effectively cast to `complex<common_type_t<T1, T3>>`, where  $T3$  is `double` if  $T2$  is an integer type and  $T2$  otherwise. If `common_type_t<T1, T3>` is not well-formed, then the program is ill-formed.

### 29.4.11 Suffixes for complex number literals

[complex.literals]

1 This subclause describes literal suffixes for constructing complex number literals. The suffixes `i`, `il`, and `if` create complex numbers of the types `complex<double>`, `complex<long double>`, and `complex<float>` respectively, with their imaginary part denoted by the given literal number and the real part being zero.

```
constexpr complex<long double> operator""il(long double d);
constexpr complex<long double> operator""il(unsigned long long d);
```

2 *Returns:* `complex<long double>{0.0L, static_cast<long double>(d)}`.

```
constexpr complex<double> operator""i(long double d);
constexpr complex<double> operator""i(unsigned long long d);
```

3 *Returns:* `complex<double>{0.0, static_cast<double>(d)}`.

```
constexpr complex<float> operator""if(long double d);
constexpr complex<float> operator""if(unsigned long long d);
```

4 *Returns:* `complex<float>{0.0f, static_cast<float>(d)}`.

## 29.5 Random number generation

[rand]

### 29.5.1 General

[rand.general]

1 Subclause 29.5 defines a facility for generating (pseudo-)random numbers.

2 In addition to a few utilities, four categories of entities are described: *uniform random bit generators*, *random number engines*, *random number engine adaptors*, and *random number distributions*. These categorizations are applicable to types that meet the corresponding requirements, to objects instantiated from such types, and to templates producing such types when instantiated.

[*Note 1*: These entities are specified in such a way as to permit the binding of any uniform random bit generator object *e* as the argument to any random number distribution object *d*, thus producing a zero-argument function object such as given by `bind(d,e)`. — *end note*]

- <sup>3</sup> Each of the entities specified in 29.5 has an associated arithmetic type (6.9.3) identified as `result_type`. With *T* as the `result_type` thus associated with such an entity, that entity is characterized:

- (3.1) — as *boolean* or equivalently as *boolean-valued*, if *T* is `bool`;
- (3.2) — otherwise as *integral* or equivalently as *integer-valued*, if `numeric_limits<T>::is_integer` is `true`;
- (3.3) — otherwise as *floating-point* or equivalently as *real-valued*.

If integer-valued, an entity may optionally be further characterized as *signed* or *unsigned*, according to `numeric_limits<T>::is_signed`.

- <sup>4</sup> Unless otherwise specified, all descriptions of calculations in 29.5 use mathematical real numbers.
- <sup>5</sup> Throughout 29.5, the operators `bitand`, `bitor`, and `xor` denote the respective conventional bitwise operations. Further:
- (5.1) — the operator `rshift` denotes a bitwise right shift with zero-valued bits appearing in the high bits of the result, and
  - (5.2) — the operator `lshiftw` denotes a bitwise left shift with zero-valued bits appearing in the low bits of the result, and whose result is always taken modulo  $2^w$ .

## 29.5.2 Header `<random>` synopsis

[rand.synopsis]

```
#include <initializer_list> // see 17.11.2

namespace std {
 // 29.5.3.3, uniform random bit generator requirements
 template<class G>
 concept uniform_random_bit_generator = see below; // freestanding

 // 29.5.4.2, class template linear_congruential_engine
 template<class UIntType, UIntType a, UIntType c, UIntType m>
 class linear_congruential_engine; // partially freestanding

 // 29.5.4.3, class template mersenne_twister_engine
 template<class UIntType, size_t w, size_t n, size_t m, size_t r,
 UIntType a, size_t u, UIntType d, size_t s,
 UIntType b, size_t t,
 UIntType c, size_t l, UIntType f>
 class mersenne_twister_engine;

 // 29.5.4.4, class template subtract_with_carry_engine
 template<class UIntType, size_t w, size_t s, size_t r>
 class subtract_with_carry_engine; // partially freestanding

 // 29.5.5.2, class template discard_block_engine
 template<class Engine, size_t p, size_t r>
 class discard_block_engine; // partially freestanding

 // 29.5.5.3, class template independent_bits_engine
 template<class Engine, size_t w, class UIntType>
 class independent_bits_engine; // partially freestanding

 // 29.5.5.4, class template shuffle_order_engine
 template<class Engine, size_t k>
 class shuffle_order_engine;

 // 29.5.4.5, class template philox_engine
 template<class UIntType, size_t w, size_t n, size_t r, UIntType... consts>
 class philox_engine; // partially freestanding

 // 29.5.6, engines and engine adaptors with predefined parameters
 using minstd_rand0 = see below; // freestanding
```

```

using minstd_rand = see below; // freestanding
using mt19937 = see below; // freestanding
using mt19937_64 = see below; // freestanding
using ranlux24_base = see below; // freestanding
using ranlux48_base = see below; // freestanding
using ranlux24 = see below; // freestanding
using ranlux48 = see below; // freestanding
using knuth_b = see below;
using philox4x32 = see below; // freestanding
using philox4x64 = see below; // freestanding

using default_random_engine = see below;

// 29.5.7, class random_device
class random_device;

// 29.5.8.1, class seed_seq
class seed_seq;

// 29.5.8.2, function template generate_canonical
template<class RealType, size_t digits, class URBG>
 RealType generate_canonical(URBG& g);

namespace ranges {
 // 26.12.2, generate_random
 template<class R, class G>
 requires output_range<R, invoke_result_t<G>>> &&
 uniform_random_bit_generator<remove_cvref_t<G>>>
 constexpr borrowed_iterator_t<R> generate_random(R&& r, G&& g);

 template<class G, output_iterator<invoke_result_t<G>>> O, sentinel_for<O> S>
 requires uniform_random_bit_generator<remove_cvref_t<G>>>
 constexpr O generate_random(O first, S last, G&& g);

 template<class R, class G, class D>
 requires output_range<R, invoke_result_t<D&, G>>> && invocable<D&, G> &&
 uniform_random_bit_generator<remove_cvref_t<G>>> &&
 is_arithmetic_v<invoke_result_t<D&, G>>>
 constexpr borrowed_iterator_t<R> generate_random(R&& r, G&& g, D&& d);

 template<class G, class D, output_iterator<invoke_result_t<D&, G>>> O, sentinel_for<O> S>
 requires invocable<D&, G> && uniform_random_bit_generator<remove_cvref_t<G>>> &&
 is_arithmetic_v<invoke_result_t<D&, G>>>
 constexpr O generate_random(O first, S last, G&& g, D&& d);
}

// 29.5.9.2.1, class template uniform_int_distribution
template<class IntType = int>
 class uniform_int_distribution; // partially freestanding

// 29.5.9.2.2, class template uniform_real_distribution
template<class RealType = double>
 class uniform_real_distribution;

// 29.5.9.3.1, class bernoulli_distribution
class bernoulli_distribution;

// 29.5.9.3.2, class template binomial_distribution
template<class IntType = int>
 class binomial_distribution;

// 29.5.9.3.3, class template geometric_distribution
template<class IntType = int>
 class geometric_distribution;

```

```

// 29.5.9.3.4, class template negative_binomial_distribution
template<class IntType = int>
 class negative_binomial_distribution;

// 29.5.9.4.1, class template poisson_distribution
template<class IntType = int>
 class poisson_distribution;

// 29.5.9.4.2, class template exponential_distribution
template<class RealType = double>
 class exponential_distribution;

// 29.5.9.4.3, class template gamma_distribution
template<class RealType = double>
 class gamma_distribution;

// 29.5.9.4.4, class template weibull_distribution
template<class RealType = double>
 class weibull_distribution;

// 29.5.9.4.5, class template extreme_value_distribution
template<class RealType = double>
 class extreme_value_distribution;

// 29.5.9.5.1, class template normal_distribution
template<class RealType = double>
 class normal_distribution;

// 29.5.9.5.2, class template lognormal_distribution
template<class RealType = double>
 class lognormal_distribution;

// 29.5.9.5.3, class template chi_squared_distribution
template<class RealType = double>
 class chi_squared_distribution;

// 29.5.9.5.4, class template cauchy_distribution
template<class RealType = double>
 class cauchy_distribution;

// 29.5.9.5.5, class template fisher_f_distribution
template<class RealType = double>
 class fisher_f_distribution;

// 29.5.9.5.6, class template student_t_distribution
template<class RealType = double>
 class student_t_distribution;

// 29.5.9.6.1, class template discrete_distribution
template<class IntType = int>
 class discrete_distribution;

// 29.5.9.6.2, class template piecewise_constant_distribution
template<class RealType = double>
 class piecewise_constant_distribution;

// 29.5.9.6.3, class template piecewise_linear_distribution
template<class RealType = double>
 class piecewise_linear_distribution;
}

```

**29.5.3 Requirements****[rand.req]****29.5.3.1 General requirements****[rand.req.genl]**

- <sup>1</sup> Throughout 29.5, where the template parameters are not constrained, the names of template parameters are used to express type requirements on an instantiated template *T*:
- (1.1) — If *T* has a template type parameter named **Sseq**, **URBG**, **Engine**, **RealType**, **IntType**, or **UIntType**, the program is ill-formed if the corresponding template argument is cv-qualified.
  - (1.2) — A template argument corresponding to a template parameter named **Sseq** shall meet the requirements of seed sequence (29.5.3.2).
  - (1.3) — A template argument corresponding to a template parameter named **URBG** shall meet the requirements of uniform random bit generator (29.5.3.3).
  - (1.4) — A template argument corresponding to a template parameter named **Engine** shall meet the requirements of random number engine (29.5.3.4).
  - (1.5) — If a template argument corresponding to a template parameter named **RealType** is neither a standard floating-point type (6.9.3) nor a member of an implementation-defined subset of extended floating-point types, the program is ill-formed.
  - (1.6) — If a template argument corresponding to a template parameter named **IntType** is neither a standard signed nor a standard unsigned integer type (6.9.3), nor an extended integer type whose width is greater or equal to that of **char** and less than or equal to that of **long long**, nor a member of an implementation-defined subset of integer types, the program is ill-formed.
  - (1.7) — If a template argument corresponding to a template parameter named **UIntType** is neither a standard or extended unsigned integer type whose width is greater or equal to that of **short** and less than or equal to that of **long long**, nor a member of an implementation-defined subset of unsigned integer types, the program is ill-formed.
- <sup>2</sup> Throughout 29.5, phrases of the form “*x* is an iterator of a specific kind” shall be interpreted as equivalent to the more formal requirement that “*x* is a value of a type meeting the requirements of the specified iterator type”.
- <sup>3</sup> Throughout 29.5, any constructor that can be called with a single argument and that meets a requirement specified in this subclause shall be declared **explicit**.

**29.5.3.2 Seed sequence requirements****[rand.req.seedseq]**

- <sup>1</sup> A *seed sequence* is an object that consumes a sequence of integer-valued data and produces a requested number of unsigned integer values *i*,  $0 \leq i < 2^{32}$ , based on the consumed data.
- [Note 1: Such an object provides a mechanism to avoid replication of streams of random variates. This can be useful, for example, in applications requiring large numbers of random number engines. — end note]
- <sup>2</sup> A class **S** meets the requirements of a seed sequence if the expressions shown in Table 126 are valid and have the indicated semantics, and if **S** also meets all other requirements of 29.5.3.2. In Table 126 and throughout this subclause:
- (2.1) — **T** is the type named by **S**’s associated **result\_type**;
  - (2.2) — **q** is a value of type **S** and **r** is a value of type **S** or **const S**;
  - (2.3) — **ib** and **ie** are input iterators with an unsigned integer **value\_type** of at least 32 bits;
  - (2.4) — **rb** and **re** are mutable random access iterators with an unsigned integer **value\_type** of at least 32 bits;
  - (2.5) — **ob** is an output iterator; and
  - (2.6) — **il** is a value of type **initializer\_list<T>**.

**Table 126 — Seed sequence requirements** [tab:rand.req.seedseq]

| Expression            | Return type | Pre/post-condition                                                | Complexity |
|-----------------------|-------------|-------------------------------------------------------------------|------------|
| <b>S::result_type</b> | <b>T</b>    | <b>T</b> is an unsigned integer type (6.9.3) of at least 32 bits. |            |

Table 126 — Seed sequence requirements (continued)

| Expression                     | Return type         | Pre/post-condition                                                                                                                                                                                                                                                           | Complexity                                   |
|--------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <code>S()</code>               |                     | Creates a seed sequence with the same initial state as all other default-constructed seed sequences of type <code>S</code> .                                                                                                                                                 | constant                                     |
| <code>S(ib,ie)</code>          |                     | Creates a seed sequence having internal state that depends on some or all of the bits of the supplied sequence <code>[ib,ie)</code> .                                                                                                                                        | $\mathcal{O}(\text{ie} - \text{ib})$         |
| <code>S(il)</code>             |                     | Same as <code>S(il.begin(), il.end())</code> .                                                                                                                                                                                                                               | same as <code>S(il.begin(), il.end())</code> |
| <code>q.generate(rb,re)</code> | <code>void</code>   | Does nothing if <code>rb == re</code> . Otherwise, fills the supplied sequence <code>[rb,re)</code> with 32-bit quantities that depend on the sequence supplied to the constructor and possibly also depend on the history of <code>generate</code> 's previous invocations. | $\mathcal{O}(\text{re} - \text{rb})$         |
| <code>r.size()</code>          | <code>size_t</code> | The number of 32-bit units that would be copied by a call to <code>r.param</code> .                                                                                                                                                                                          | constant                                     |
| <code>r.param(ob)</code>       | <code>void</code>   | Copies to the given destination a sequence of 32-bit units that can be provided to the constructor of a second object of type <code>S</code> , and that would reproduce in that second object a state indistinguishable from the state of the first object.                  | $\mathcal{O}(\text{r.size}())$               |

### 29.5.3.3 Uniform random bit generator requirements

[rand.req.urng]

- <sup>1</sup> A *uniform random bit generator* `g` of type `G` is a function object returning unsigned integer values such that each value in the range of possible results has (ideally) equal probability of being returned.

[Note 1: The degree to which `g`'s results approximate the ideal is often determined statistically. — end note]

```
template<class G>
concept uniform_random_bit_generator =
 invocable<G&& && unsigned_integral<invoke_result_t<G&&>> &&
 requires {
 { G::min() } -> same_as<invoke_result_t<G&&>>;
 { G::max() } -> same_as<invoke_result_t<G&&>>;
 requires bool_constant<(G::min() < G::max())>::value;
 };

```

- <sup>2</sup> Let `g` be an object of type `G`. `G` models `uniform_random_bit_generator` only if

(2.1) — `G::min() <= g()`,

(2.2) — `g() <= G::max()`, and

(2.3) — `g()` has amortized constant complexity.

- <sup>3</sup> A class `G` meets the *uniform random bit generator* requirements if `G` models `uniform_random_bit_generator`, `invoke_result_t<G&&>` is an unsigned integer type (6.9.3), and `G` provides a nested *typedef-name* `result_type` that denotes the same type as `invoke_result_t<G&&>`.



**29.5.3.4 Random number engine requirements**

[rand.req.eng]

- <sup>1</sup> A *random number engine* (commonly shortened to *engine*) **e** of type **E** is a uniform random bit generator that additionally meets the requirements (e.g., for seeding and for input/output) specified in this subclause.
- <sup>2</sup> At any given time, **e** has a state  $\mathbf{e}_i$  for some integer  $i \geq 0$ . Upon construction, **e** has an initial state  $\mathbf{e}_0$ . An engine's state may be established via a constructor, a **seed** function, assignment, or a suitable **operator>>**.
- <sup>3</sup> E's specification shall define:
  - (3.1) — the size of E's state in multiples of the size of **result\_type**, given as an integral constant expression;
  - (3.2) — the *transition algorithm* **TA** by which  $\mathbf{e}_i$  is advanced to its *successor state*  $\mathbf{e}_{i+1}$ ; and
  - (3.3) — the *generation algorithm* **GA** by which an engine's state is mapped to a value of type **result\_type**.
- <sup>4</sup> A class **E** that meets the requirements of a uniform random bit generator (29.5.3.3) also meets the requirements of a *random number engine* if the expressions shown in Table 127 are valid and have the indicated semantics, and if **E** also meets all other requirements of 29.5.3.4. In Table 127 and throughout this subclause:
  - (4.1) — **T** is the type named by E's associated **result\_type**;
  - (4.2) — **e** is a value of **E**, **v** is an lvalue of **E**, **x** and **y** are (possibly const) values of **E**;
  - (4.3) — **s** is a value of **T**;
  - (4.4) — **q** is an lvalue meeting the requirements of a seed sequence (29.5.3.2);
  - (4.5) — **z** is a value of type **unsigned long long**;
  - (4.6) — **os** is an lvalue of the type of some class template specialization **basic\_ostream<charT, traits>**; and
  - (4.7) — **is** is an lvalue of the type of some class template specialization **basic\_istream<charT, traits>**;

where **charT** and **traits** are constrained according to Clause 27 and Clause 31.

**Table 127 — Random number engine requirements** [tab:rand.req.eng]

| Expression                         | Return type | Pre/post-condition                                                                                                                       | Complexity                                                                                 |
|------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>E()</b>                         |             | Creates an engine with the same initial state as all other default-constructed engines of type <b>E</b> .                                | $\mathcal{O}(\text{size of state})$                                                        |
| <b>E(x)</b>                        |             | Creates an engine that compares equal to <b>x</b> .                                                                                      | $\mathcal{O}(\text{size of state})$                                                        |
| <b>E(s)</b>                        |             | Creates an engine with initial state determined by <b>s</b> .                                                                            | $\mathcal{O}(\text{size of state})$                                                        |
| <b>E(q)</b> <sup>219</sup>         |             | Creates an engine with an initial state that depends on a sequence produced by one call to <b>q.generate</b> .                           | same as complexity of <b>q.generate</b> called on a sequence whose length is size of state |
| <b>e.seed()</b>                    | <b>void</b> | <i>Postconditions:</i> <b>e</b> == <b>E()</b> .                                                                                          | same as <b>E()</b>                                                                         |
| <b>e.seed(s)</b>                   | <b>void</b> | <i>Postconditions:</i> <b>e</b> == <b>E(s)</b> .                                                                                         | same as <b>E(s)</b>                                                                        |
| <b>e.seed(q)</b>                   | <b>void</b> | <i>Postconditions:</i> <b>e</b> == <b>E(q)</b> .                                                                                         | same as <b>E(q)</b>                                                                        |
| <b>e()</b>                         | <b>T</b>    | Advances <b>e</b> 's state $\mathbf{e}_i$ to $\mathbf{e}_{i+1}$ = <b>TA</b> ( $\mathbf{e}_i$ ) and returns <b>GA</b> ( $\mathbf{e}_i$ ). | per 29.5.3.3                                                                               |
| <b>e.discard(z)</b> <sup>220</sup> | <b>void</b> | Advances <b>e</b> 's state $\mathbf{e}_i$ to $\mathbf{e}_{i+z}$ by any means equivalent to <b>z</b> consecutive calls <b>e()</b> .       | no worse than the complexity of <b>z</b> consecutive calls <b>e()</b>                      |

<sup>219</sup> This constructor (as well as the subsequent corresponding **seed()** function) can be particularly useful to applications requiring a large number of independent random sequences.

<sup>220</sup> This operation is common in user code, and can often be implemented in an engine-specific manner so as to provide significant performance improvements over an equivalent naive loop that makes **z** consecutive calls **e()**.

Table 127 — Random number engine requirements (continued)

| Expression          | Return type       | Pre/post-condition                                                                                                                                                                                                                                                                        | Complexity                          |
|---------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <code>x == y</code> | <code>bool</code> | This operator is an equivalence relation. With $S_x$ and $S_y$ as the infinite sequences of values that would be generated by repeated future calls to <code>x()</code> and <code>y()</code> , respectively, returns <code>true</code> if $S_x = S_y$ ; else returns <code>false</code> . | $\mathcal{O}(\text{size of state})$ |
| <code>x != y</code> | <code>bool</code> | <code>!(x == y)</code> .                                                                                                                                                                                                                                                                  | $\mathcal{O}(\text{size of state})$ |

<sup>5</sup> E shall meet the *Cpp17CopyConstructible* (Table 32) and *Cpp17CopyAssignable* (Table 34) requirements. These operations shall each be of complexity no worse than  $\mathcal{O}(\text{size of state})$ .

<sup>6</sup> On hosted implementations, the following expressions are well-formed and have the specified semantics.

`os << x`

<sup>7</sup> *Effects:* With `os.fmtflags` set to `ios_base::dec|ios_base::left` and the fill character set to the space character, writes to `os` the textual representation of `x`'s current state. In the output, adjacent numbers are separated by one or more space characters.

<sup>8</sup> *Postconditions:* The `os.fmtflags` and fill character are unchanged.

<sup>9</sup> *Result:* reference to the type of `os`.

<sup>10</sup> *Returns:* `os`.

<sup>11</sup> *Complexity:*  $\mathcal{O}(\text{size of state})$

`is >> v`

<sup>12</sup> *Preconditions:* `is` provides a textual representation that was previously written using an output stream whose imbued locale was the same as that of `is`, and whose type's template specialization arguments `charT` and `traits` were respectively the same as those of `is`.

<sup>13</sup> *Effects:* With `is.fmtflags` set to `ios_base::dec`, sets `v`'s state as determined by reading its textual representation from `is`. If bad input is encountered, ensures that `v`'s state is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` (31.5.4.4)). If a textual representation written via `os << x` was subsequently read via `is >> v`, then `x == v` provided that there have been no intervening invocations of `x` or of `v`.

<sup>14</sup> *Postconditions:* The `is.fmtflags` are unchanged.

<sup>15</sup> *Result:* reference to the type of `is`.

<sup>16</sup> *Returns:* `is`.

<sup>17</sup> *Complexity:*  $\mathcal{O}(\text{size of state})$

### 29.5.3.5 Random number engine adaptor requirements

[rand.req.adapt]

<sup>1</sup> A *random number engine adaptor* (commonly shortened to *adaptor*) `a` of type `A` is a random number engine that takes values produced by some other random number engine, and applies an algorithm to those values in order to deliver a sequence of values with different randomness properties. An engine `b` of type `B` adapted in this way is termed a *base engine* in this context. The expression `a.base()` shall be valid and shall return a `const` reference to `a`'s base engine.

<sup>2</sup> The requirements of a random number engine type shall be interpreted as follows with respect to a random number engine adaptor type.

`A::A();`

<sup>3</sup> *Effects:* The base engine is initialized as if by its default constructor.

`bool operator==(const A& a1, const A& a2);`

<sup>4</sup> *Returns:* `true` if `a1`'s base engine is equal to `a2`'s base engine. Otherwise returns `false`.

```
A::A(result_type s);
```

5       *Effects:* The base engine is initialized with **s**.

```
template<class Sseq> A::A(Sseq& q);
```

6       *Effects:* The base engine is initialized with **q**.

```
void seed();
```

7       *Effects:* With **b** as the base engine, invokes **b.seed()**.

```
void seed(result_type s);
```

8       *Effects:* With **b** as the base engine, invokes **b.seed(s)**.

```
template<class Sseq> void seed(Sseq& q);
```

9       *Effects:* With **b** as the base engine, invokes **b.seed(q)**.

10 **A** shall also meet the following additional requirements:

- (10.1) — The complexity of each function shall not exceed the complexity of the corresponding function applied to the base engine.
- (10.2) — The state of **A** shall include the state of its base engine. The size of **A**'s state shall be no less than the size of the base engine.
- (10.3) — Copying **A**'s state (e.g., during copy construction or copy assignment) shall include copying the state of the base engine of **A**.
- (10.4) — The textual representation of **A** shall include the textual representation of its base engine.

### 29.5.3.6 Random number distribution requirements

[rand.req.dist]

- 1 A *random number distribution* (commonly shortened to *distribution*) **d** of type **D** is a function object returning values that are distributed according to an associated mathematical *probability density function*  $p(z)$  or according to an associated *discrete probability function*  $P(z_i)$ . A distribution's specification identifies its associated probability function  $p(z)$  or  $P(z_i)$ .
- 2 An associated probability function is typically expressed using certain externally-supplied quantities known as the *parameters of the distribution*. Such distribution parameters are identified in this context by writing, for example,  $p(z | a, b)$  or  $P(z_i | a, b)$ , to name specific parameters, or by writing, for example,  $p(z | \{p\})$  or  $P(z_i | \{p\})$ , to denote a distribution's parameters **p** taken as a whole.
- 3 A class **D** meets the requirements of a *random number distribution* if the expressions shown in Table 128 are valid and have the indicated semantics, and if **D** and its associated types also meet all other requirements of 29.5.3.6. In Table 128 and throughout this subclause,
  - (3.1) — **T** is the type named by **D**'s associated **result\_type**;
  - (3.2) — **P** is the type named by **D**'s associated **param\_type**;
  - (3.3) — **d** is a value of **D**, and **x** and **y** are (possibly const) values of **D**;
  - (3.4) — **glb** and **lub** are values of **T** respectively corresponding to the greatest lower bound and the least upper bound on the values potentially returned by **d**'s **operator()**, as determined by the current values of **d**'s parameters;
  - (3.5) — **p** is a (possibly const) value of **P**;
  - (3.6) — **g**, **g1**, and **g2** are lvalues of a type meeting the requirements of a uniform random bit generator (29.5.3.3);
  - (3.7) — **os** is an lvalue of the type of some class template specialization **basic\_ostream<charT, traits>**; and
  - (3.8) — **is** is an lvalue of the type of some class template specialization **basic\_istream<charT, traits>**;

where **charT** and **traits** are constrained according to Clause 27 and Clause 31.

Table 128 — Random number distribution requirements [tab:rand.req.dist]

| Expression            | Return type | Pre/post-condition                      | Complexity |
|-----------------------|-------------|-----------------------------------------|------------|
| <b>D::result_type</b> | <b>T</b>    | <b>T</b> is an arithmetic type (6.9.3). |            |
| <b>D::param_type</b>  | <b>P</b>    |                                         |            |

Table 128 — Random number distribution requirements (continued)

| Expression              | Return type       | Pre/post-condition                                                                                                                                                                                                                                                                                                                                                           | Complexity                                                 |
|-------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <code>D()</code>        |                   | Creates a distribution whose behavior is indistinguishable from that of any other newly default-constructed distribution of type <code>D</code> .                                                                                                                                                                                                                            | constant                                                   |
| <code>D(p)</code>       |                   | Creates a distribution whose behavior is indistinguishable from that of a distribution newly constructed directly from the values used to construct <code>p</code> .                                                                                                                                                                                                         | same as <code>p</code> 's construction                     |
| <code>d.reset()</code>  | <code>void</code> | Subsequent uses of <code>d</code> do not depend on values produced by any engine prior to invoking <code>reset</code> .                                                                                                                                                                                                                                                      | constant                                                   |
| <code>x.param()</code>  | <code>P</code>    | Returns a value <code>p</code> such that <code>D(p).param() == p</code> .                                                                                                                                                                                                                                                                                                    | no worse than the complexity of <code>D(p)</code>          |
| <code>d.param(p)</code> | <code>void</code> | <i>Postconditions:</i> <code>d.param() == p</code> .                                                                                                                                                                                                                                                                                                                         | no worse than the complexity of <code>D(p)</code>          |
| <code>d(g)</code>       | <code>T</code>    | With <code>p = d.param()</code> , the sequence of numbers returned by successive invocations with the same object <code>g</code> is randomly distributed according to the associated $p(z   \{p\})$ or $P(z_i   \{p\})$ function.                                                                                                                                            | amortized constant number of invocations of <code>g</code> |
| <code>d(g,p)</code>     | <code>T</code>    | The sequence of numbers returned by successive invocations with the same objects <code>g</code> and <code>p</code> is randomly distributed according to the associated $p(z   \{p\})$ or $P(z_i   \{p\})$ function.                                                                                                                                                          | amortized constant number of invocations of <code>g</code> |
| <code>x.min()</code>    | <code>T</code>    | Returns <code>glb</code> .                                                                                                                                                                                                                                                                                                                                                   | constant                                                   |
| <code>x.max()</code>    | <code>T</code>    | Returns <code>lub</code> .                                                                                                                                                                                                                                                                                                                                                   | constant                                                   |
| <code>x == y</code>     | <code>bool</code> | This operator is an equivalence relation. Returns <code>true</code> if <code>x.param() == y.param()</code> and $S_1 = S_2$ , where $S_1$ and $S_2$ are the infinite sequences of values that would be generated, respectively, by repeated future calls to <code>x(g1)</code> and <code>y(g2)</code> whenever <code>g1 == g2</code> . Otherwise returns <code>false</code> . | constant                                                   |
| <code>x != y</code>     | <code>bool</code> | <code>!(x == y)</code> .                                                                                                                                                                                                                                                                                                                                                     | same as <code>x == y</code> .                              |

<sup>4</sup> `D` shall meet the *Cpp17CopyConstructible* (Table 32) and *Cpp17CopyAssignable* (Table 34) requirements.

<sup>5</sup> The sequence of numbers produced by repeated invocations of `d(g)` shall be independent of any invocation of `os << d` or of any `const` member function of `D` between any of the invocations of `d(g)`.

<sup>6</sup> If a textual representation is written using `os << x` and that representation is restored into the same or a different object `y` of the same type using `is >> y`, repeated invocations of `y(g)` shall produce the same sequence of numbers as would repeated invocations of `x(g)`.

- 7 It is unspecified whether `D::param_type` is declared as a (nested) `class` or via a `typedef`. In 29.5, declarations of `D::param_type` are in the form of `typedefs` for convenience of exposition only.
- 8 P shall meet the *Cpp17CopyConstructible* (Table 32), *Cpp17CopyAssignable* (Table 34), and *Cpp17EqualityComparable* (Table 28) requirements.
- 9 For each of the constructors of D taking arguments corresponding to parameters of the distribution, P shall have a corresponding constructor subject to the same requirements and taking arguments identical in number, type, and default values. Moreover, for each of the member functions of D that return values corresponding to parameters of the distribution, P shall have a corresponding member function with the identical name, type, and semantics.
- 10 P shall have a declaration of the form
- ```
using distribution_type = D;
```
- 11 On hosted implementations, the following expressions are well-formed and have the specified semantics.

```
os << x
```

Effects: Writes to `os` a textual representation for the parameters and the additional internal data of `x`.

Postconditions: The `os.fmtflags` and fill character are unchanged.

Result: reference to the type of `os`.

Returns: `os`.

```
is >> d
```

Preconditions: `is` provides a textual representation that was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`.

Effects: Restores from `is` the parameters and additional internal data of the lvalue `d`. If bad input is encountered, ensures that `d` is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` (31.5.4.4)).

Postconditions: The `is.fmtflags` are unchanged.

Result: reference to the type of `is`.

Returns: `is`.

29.5.4 Random number engine class templates

[rand.eng]

29.5.4.1 General

[rand.eng.general]

- 1 Each type instantiated from a class template specified in 29.5.4 meets the requirements of a random number engine (29.5.3.4) type.
- 2 Except where specified otherwise, the complexity of each function specified in 29.5.4 is constant.
- 3 Except where specified otherwise, no function described in 29.5.4 throws an exception.
- 4 Every function described in 29.5.4 that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `seed_seq` throws what and when the invocation of `q.generate` throws.
- 5 Descriptions are provided in 29.5.4 only for engine operations that are not described in 29.5.3.4 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.
- 6 Each template specified in 29.5.4 requires one or more relationships, involving the value(s) of its constant template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.
- 7 For every random number engine and for every random number engine adaptor X defined in 29.5.4 and in 29.5.5:
- (7.1) — if the constructor
- ```
template<class Sseq> explicit X(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

- (7.2) — if the member function

```
template<class Sseq> void seed(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution.

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

#### 29.5.4.2 Class template `linear_congruential_engine` [rand.eng.lcong]

- <sup>1</sup> A `linear_congruential_engine` random number engine produces unsigned integer random numbers. The state  $x_i$  of a `linear_congruential_engine` object `x` is of size 1 and consists of a single integer. The transition algorithm is a modular linear function of the form  $TA(x_i) = (a \cdot x_i + c) \bmod m$ ; the generation algorithm is  $GA(x_i) = x_{i+1}$ .

```
namespace std {
 template<class UIntType, UIntType a, UIntType c, UIntType m>
 class linear_congruential_engine {
 public:
 // types
 using result_type = UIntType;

 // engine characteristics
 static constexpr result_type multiplier = a;
 static constexpr result_type increment = c;
 static constexpr result_type modulus = m;
 static constexpr result_type min() { return c == 0u ? 1u : 0u; }
 static constexpr result_type max() { return m - 1u; }
 static constexpr result_type default_seed = 1u;

 // constructors and seeding functions
 linear_congruential_engine() : linear_congruential_engine(default_seed) {}
 explicit linear_congruential_engine(result_type s);
 template<class Sseq> explicit linear_congruential_engine(Sseq& q);
 void seed(result_type s = default_seed);
 template<class Sseq> void seed(Sseq& q);

 // equality operators
 friend bool operator==(const linear_congruential_engine& x,
 const linear_congruential_engine& y);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);

 // inserters and extractors
 template<class charT, class traits>
 friend basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 const linear_congruential_engine& x); // hosted
 template<class charT, class traits>
 friend basic_istream<charT, traits>&
 operator>>(basic_istream<charT, traits>& is,
 linear_congruential_engine& x); // hosted
 };
}
```

- <sup>2</sup> If the template parameter `m` is 0, the modulus  $m$  used throughout 29.5.4.2 is `numeric_limits<result_type>::max()` plus 1.

[Note 1:  $m$  need not be representable as a value of type `result_type`. — end note]

<sup>3</sup> If the template parameter `m` is not 0, the following relations shall hold: `a < m` and `c < m`.

<sup>4</sup> The textual representation consists of the value of `xi`.

```
explicit linear_congruential_engine(result_type s);
```

<sup>5</sup> *Effects:* If `c mod m` is 0 and `s mod m` is 0, sets the engine's state to 1, otherwise sets the engine's state to `s mod m`.

```
template<class Sseq> explicit linear_congruential_engine(Sseq& q);
```

<sup>6</sup> *Effects:* With  $k = \left\lceil \frac{\log_2 m}{32} \right\rceil$  and `a` an array (or equivalent) of length  $k + 3$ , invokes `q.generate(a + 0, a + k + 3)` and then computes  $S = \left( \sum_{j=0}^{k-1} a_{j+3} \cdot 2^{32j} \right) \bmod m$ . If `c mod m` is 0 and `S` is 0, sets the engine's state to 1, else sets the engine's state to `S`.

#### 29.5.4.3 Class template `mersenne_twister_engine`

[rand.eng.mers]

<sup>1</sup> A `mersenne_twister_engine` random number engine<sup>221</sup> produces unsigned integer random numbers in the closed interval  $[0, 2^w - 1]$ . The state `xi` of a `mersenne_twister_engine` object `x` is of size  $n$  and consists of a sequence  $X$  of  $n$  values of the type delivered by `x`; all subscripts applied to  $X$  are to be taken modulo  $n$ .

<sup>2</sup> The transition algorithm employs a twisted generalized feedback shift register defined by shift values  $n$  and  $m$ , a twist value  $r$ , and a conditional xor-mask  $a$ . To improve the uniformity of the result, the bits of the raw shift register are additionally *tempered* (i.e., scrambled) according to a bit-scrambling matrix defined by values  $u, d, s, b, t, c$ , and  $\ell$ .

The state transition is performed as follows:

(2.1) — Concatenate the upper  $w - r$  bits of  $X_{i-n}$  with the lower  $r$  bits of  $X_{i+1-n}$  to obtain an unsigned integer value  $Y$ .

(2.2) — With  $\alpha = a \cdot (Y \text{ bitand } 1)$ , set  $X_i$  to  $X_{i+m-n}$  xor  $(Y \text{ rshift } 1)$  xor  $\alpha$ .

The sequence  $X$  is initialized with the help of an initialization multiplier  $f$ .

<sup>3</sup> The generation algorithm determines the unsigned integer values  $z_1, z_2, z_3, z_4$  as follows, then delivers  $z_4$  as its result:

(3.1) — Let  $z_1 = X_i$  xor  $((X_i \text{ rshift } u) \text{ bitand } d)$ .

(3.2) — Let  $z_2 = z_1$  xor  $((z_1 \text{ lshift}_w s) \text{ bitand } b)$ .

(3.3) — Let  $z_3 = z_2$  xor  $((z_2 \text{ lshift}_w t) \text{ bitand } c)$ .

(3.4) — Let  $z_4 = z_3$  xor  $(z_3 \text{ rshift } \ell)$ .

```
namespace std {
 template<class UIntType, size_t w, size_t n, size_t m, size_t r,
 UIntType a, size_t u, UIntType d, size_t s,
 UIntType b, size_t t,
 UIntType c, size_t l, UIntType f>
 class mersenne_twister_engine {
 public:
 // types
 using result_type = UIntType;

 // engine characteristics
 static constexpr size_t word_size = w;
 static constexpr size_t state_size = n;
 static constexpr size_t shift_size = m;
 static constexpr size_t mask_bits = r;
 static constexpr UIntType xor_mask = a;
 static constexpr size_t tempering_u = u;
 static constexpr UIntType tempering_d = d;
 static constexpr size_t tempering_s = s;
 static constexpr UIntType tempering_b = b;
 static constexpr size_t tempering_t = t;
```

<sup>221</sup> The name of this engine refers, in part, to a property of its period: For properly-selected values of the parameters, the period is closely related to a large Mersenne prime number.

```

static constexpr UIntType tempering_c = c;
static constexpr size_t tempering_l = 1;
static constexpr UIntType initialization_multiplier = f;
static constexpr result_type min() { return 0; }
static constexpr result_type max() { return 2w - 1; }
static constexpr result_type default_seed = 5489u;

// constructors and seeding functions
mersenne_twister_engine() : mersenne_twister_engine(default_seed) {}
explicit mersenne_twister_engine(result_type value);
template<class Sseq> explicit mersenne_twister_engine(Sseq& q);
void seed(result_type value = default_seed);
template<class Sseq> void seed(Sseq& q);

// equality operators
friend bool operator==(const mersenne_twister_engine& x, const mersenne_twister_engine& y);

// generating functions
result_type operator()();
void discard(unsigned long long z);

// inserters and extractors
template<class charT, class traits>
 friend basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, // hosted
 const mersenne_twister_engine& x);
template<class charT, class traits>
 friend basic_istream<charT, traits>&
 operator>>(basic_istream<charT, traits>& is, // hosted
 mersenne_twister_engine& x);
};
}

```

- 4 The following relations shall hold:  $0 < m, m \leq n, 2u < w, r \leq w, u \leq w, s \leq w, t \leq w, l \leq w, w \leq \text{numeric\_limits}<\text{UIntType}>::\text{digits}$ ,  $a \leq (1u \ll w) - 1u$ ,  $b \leq (1u \ll w) - 1u$ ,  $c \leq (1u \ll w) - 1u$ ,  $d \leq (1u \ll w) - 1u$ , and  $f \leq (1u \ll w) - 1u$ .

- 5 The textual representation of  $x_i$  consists of the values of  $X_{i-n}, \dots, X_{i-1}$ , in that order.

```
explicit mersenne_twister_engine(result_type value);
```

- 6 *Effects:* Sets  $X_{-n}$  to  $\text{value} \bmod 2^w$ . Then, iteratively for  $i = 1 - n, \dots, -1$ , sets  $X_i$  to

$$[f \cdot (X_{i-1} \text{ xor } (X_{i-1} \text{ rshift } (w - 2))) + i \bmod n] \bmod 2^w.$$

- 7 *Complexity:*  $\mathcal{O}(n)$ .

```
template<class Sseq> explicit mersenne_twister_engine(Sseq& q);
```

- 8 *Effects:* With  $k = \lceil w/32 \rceil$  and  $a$  an array (or equivalent) of length  $n \cdot k$ , invokes  $q.\text{generate}(a + 0, a + n \cdot k)$  and then, iteratively for  $i = -n, \dots, -1$ , sets  $X_i$  to  $\left(\sum_{j=0}^{k-1} a_{k(i+n)+j} \cdot 2^{32j}\right) \bmod 2^w$ . Finally, if the most significant  $w - r$  bits of  $X_{-n}$  are zero, and if each of the other resulting  $X_i$  is 0, changes  $X_{-n}$  to  $2^{w-1}$ .

#### 29.5.4.4 Class template subtract\_with\_carry\_engine

[rand.eng.sub]

- 1 A `subtract_with_carry_engine` random number engine produces unsigned integer random numbers.
- 2 The state  $x_i$  of a `subtract_with_carry_engine` object  $x$  is of size  $\mathcal{O}(r)$ , and consists of a sequence  $X$  of  $r$  integer values  $0 \leq X_i < m = 2^w$ ; all subscripts applied to  $X$  are to be taken modulo  $r$ . The state  $x_i$  additionally consists of an integer  $c$  (known as the *carry*) whose value is either 0 or 1.
- 3 The state transition is performed as follows:
- (3.1) — Let  $Y = X_{i-s} - X_{i-r} - c$ .
- (3.2) — Set  $X_i$  to  $y = Y \bmod m$ . Set  $c$  to 1 if  $Y < 0$ , otherwise set  $c$  to 0.



[*Note 1:* This algorithm corresponds to a modular linear function of the form  $TA(\mathbf{x}_i) = (a \cdot \mathbf{x}_i) \bmod b$ , where  $b$  is of the form  $m^r - m^s + 1$  and  $a = b - (b - 1)/m$ . — *end note*]

- 4 The generation algorithm is given by  $GA(\mathbf{x}_i) = y$ , where  $y$  is the value produced as a result of advancing the engine's state as described above.

```
namespace std {
 template<class UIntType, size_t w, size_t s, size_t r>
 class subtract_with_carry_engine {
 public:
 // types
 using result_type = UIntType;

 // engine characteristics
 static constexpr size_t word_size = w;
 static constexpr size_t short_lag = s;
 static constexpr size_t long_lag = r;
 static constexpr result_type min() { return 0; }
 static constexpr result_type max() { return m - 1; }
 static constexpr uint_least32_t default_seed = 19780503u;

 // constructors and seeding functions
 subtract_with_carry_engine() : subtract_with_carry_engine(0u) {}
 explicit subtract_with_carry_engine(result_type value);
 template<class Sseq> explicit subtract_with_carry_engine(Sseq& q);
 void seed(result_type value = 0u);
 template<class Sseq> void seed(Sseq& q);

 // equality operators
 friend bool operator==(const subtract_with_carry_engine& x,
 const subtract_with_carry_engine& y);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);

 // inserters and extractors
 template<class charT, class traits>
 friend basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 const subtract_with_carry_engine& x); // hosted
 template<class charT, class traits>
 friend basic_istream<charT, traits>&
 operator>>(basic_istream<charT, traits>& is,
 subtract_with_carry_engine& x); // hosted
 };
}
```

- 5 The following relations shall hold:  $0u < s$ ,  $s < r$ ,  $0 < w$ , and  $w \leq \text{numeric\_limits}<\text{UIntType}>::\text{digits}$ .
- 6 The textual representation consists of the values of  $X_{i-r}, \dots, X_{i-1}$ , in that order, followed by  $c$ .

```
explicit subtract_with_carry_engine(result_type value);
```

- 7 *Effects:* Sets the values of  $X_{-r}, \dots, X_{-1}$ , in that order, as specified below. If  $X_{-1}$  is then 0, sets  $c$  to 1; otherwise sets  $c$  to 0.

To set the values  $X_k$ , first construct **e**, a `linear_congruential_engine` object, as if by the following definition:

```
linear_congruential_engine<uint_least32_t, 40014u, 0u, 2147483563u> e(
 value == 0u ? default_seed : static_cast<uint_least32_t>(value % 2147483563u));
```

Then, to set each  $X_k$ , obtain new values  $z_0, \dots, z_{n-1}$  from  $n = \lceil w/32 \rceil$  successive invocations of **e**. Set  $X_k$  to  $\left( \sum_{j=0}^{n-1} z_j \cdot 2^{32j} \right) \bmod m$ .

- 8 *Complexity:* Exactly  $n \cdot r$  invocations of **e**.

```
template<class Sseq> explicit subtract_with_carry_engine(Sseq& q);
```

- 9 *Effects:* With  $k = \lceil w/32 \rceil$  and  $a$  an array (or equivalent) of length  $r \cdot k$ , invokes `q.generate(a + 0, a + r · k)` and then, iteratively for  $i = -r, \dots, -1$ , sets  $X_i$  to  $\left(\sum_{j=0}^{k-1} a_{k(i+r)+j} \cdot 2^{32j}\right) \bmod m$ . If  $X_{-1}$  is then 0, sets  $c$  to 1; otherwise sets  $c$  to 0.

#### 29.5.4.5 Class template `philox_engine`

[rand.eng.philox]

- 1 A `philox_engine` random number engine produces unsigned integer random numbers in the interval  $[0, m)$ , where  $m = 2^w$  and the template parameter  $w$  defines the range of the produced numbers. The state of a `philox_engine` object consists of a sequence  $X$  of  $n$  unsigned integer values of width  $w$ , a sequence  $K$  of  $n/2$  values of `result_type`, a sequence  $Y$  of  $n$  values of `result_type`, and a scalar  $i$ , where
- (1.1) —  $X$  is the interpretation of the unsigned integer *counter* value  $Z := \sum_{j=0}^{n-1} X_j \cdot 2^{wj}$  of  $n \cdot w$  bits,
  - (1.2) —  $K$  are keys, which are generated once from the seed (see constructors below) and remain constant unless the `seed` function (29.5.3.4) is invoked,
  - (1.3) —  $Y$  stores a batch of output values, and
  - (1.4) —  $i$  is an index for an element of the sequence  $Y$ .
- 2 The generation algorithm returns  $Y_i$ , the value stored in the  $i^{\text{th}}$  element of  $Y$  after applying the transition algorithm.
- 3 The state transition is performed as if by the following algorithm:
- ```

i = i + 1
if (i == n) {
    Y = Philox(K, X) // see below
    Z = Z + 1
    i = 0
}

```
- 4 The `Philox` function maps the length- $n/2$ sequence K and the length- n sequence X into a length- n output sequence Y . `Philox` applies an r -round substitution-permutation network to the values in X . A single round of the generation algorithm performs the following steps:
- (4.1) — The output sequence X' of the previous round (X in case of the first round) is permuted to obtain the intermediate state V :
$$V_j = X'_{f_n(j)}$$
where $j = 0, \dots, n-1$ and $f_n(j)$ is defined in Table 129.

Table 129 — Values for the word permutation $f_n(j)$ [tab:rand.eng.philox.f]

$f_n(j)$		j			
		0	1	2	3
n	2	0	1		
	4	2	1	0	3

[Note 1: For $n = 2$ the sequence is not permuted. — end note]

- (4.2) — The following computations are applied to the elements of the V sequence:

$$X_{2k+0} = \text{mulhi}(V_{2k}, M_k, w) \text{ xor } \text{key}_k^q \text{ xor } V_{2k+1}$$

$$X_{2k+1} = \text{mullo}(V_{2k}, M_k, w)$$

where:

- (4.2.1) — $\text{mullo}(\mathbf{a}, \mathbf{b}, \mathbf{w})$ is the low half of the modular multiplication of $\mathbf{a}$ and $\mathbf{b}$: $(\mathbf{a} \cdot \mathbf{b}) \bmod 2^w$,
- (4.2.2) — $\text{mulhi}(\mathbf{a}, \mathbf{b}, \mathbf{w})$ is the high half of the modular multiplication of $\mathbf{a}$ and $\mathbf{b}$: $(\lfloor (\mathbf{a} \cdot \mathbf{b}) / 2^w \rfloor)$,
- (4.2.3) — $k = 0, \dots, n/2 - 1$ is the index in the sequences,
- (4.2.4) — $q = 0, \dots, r - 1$ is the index of the round,
- (4.2.5) — key_k^q is the k^{th} round key for round q , $\text{key}_k^q := (K_k + q \cdot C_k) \bmod 2^w$,
- (4.2.6) — K_k are the elements of the key sequence K ,

(4.2.7) — M_k is `multipliers[k]`, and

(4.2.8) — C_k is `round_consts[k]`.

5 After r applications of the single-round function, Philox returns the sequence $Y = X'$.

```
namespace std {
    template<class UIntType, size_t w, size_t n, size_t r, UIntType... consts>
    class philox_engine {
        static constexpr size_t array_size = n / 2;    // exposition only
    public:
        // types
        using result_type = UIntType;

        // engine characteristics
        static constexpr size_t word_size = w;
        static constexpr size_t word_count = n;
        static constexpr size_t round_count = r;
        static constexpr array<result_type, array_size> multipliers;
        static constexpr array<result_type, array_size> round_consts;
        static constexpr result_type min() { return 0; }
        static constexpr result_type max() { return m - 1; }
        static constexpr result_type default_seed = 20111115u;

        // constructors and seeding functions
        philox_engine() : philox_engine(default_seed) {}
        explicit philox_engine(result_type value);
        template<class Sseq> explicit philox_engine(Sseq& q);
        void seed(result_type value = default_seed);
        template<class Sseq> void seed(Sseq& q);

        void set_counter(const array<result_type, n>& counter);

        // equality operators
        friend bool operator==(const philox_engine& x, const philox_engine& y);

        // generating functions
        result_type operator()();
        void discard(unsigned long long z);

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const philox_engine& x);    // hosted
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, philox_engine& x);    // hosted
    };
}
```

6 *Mandates:*

(6.1) — `sizeof...(consts) == n` is true, and

(6.2) — `n == 2 || n == 4` is true, and

(6.3) — `0 < r` is true, and

(6.4) — `0 < w && w <= numeric_limits<UIntType>::digits` is true.

7 The template parameter pack `consts` represents the M_k and C_k constants which are grouped as follows:
 $[M_0, C_0, M_1, C_1, M_2, C_2, \dots, M_{n/2-1}, C_{n/2-1}]$.

8 The textual representation consists of the values of $K_0, \dots, K_{n/2-1}, X_0, \dots, X_{n-1}, i$, in that order.

[Note 2: The stream extraction operator can reconstruct Y from K and X , as needed. — end note]

```
explicit philox_engine(result_type value);
```

- 9 *Effects:* Sets the K_0 element of sequence K to `value` mod 2^w . All elements of sequences X and K (except K_0) are set to 0. The value of i is set to $n - 1$.

```
template<class Sseq> explicit philox_engine(Sseq& q);
```

- 10 *Effects:* With $p = \lceil w/32 \rceil$ and an array (or equivalent) `a` of length $(n/2) \cdot p$, invokes `q.generate(a + 0, a + n / 2 * p)` and then iteratively for $k = 0, \dots, n/2 - 1$, sets K_k to $\left(\sum_{j=0}^{p-1} a_{kp+j} \cdot 2^{32j} \right) \bmod 2^w$. All elements of sequence X are set to 0. The value of i is set to $n - 1$.

```
void set_counter(const array<result_type, n>& c);
```

- 11 *Effects:* For $j = 0, \dots, n - 1$ sets X_j to $C_{n-1-j} \bmod 2^w$. The value of i is set to $n - 1$.
[Note 3: The counter is the value Z introduced at the beginning of this subclause. — end note]

29.5.5 Random number engine adaptor class templates

[rand.adapt]

29.5.5.1 General

[rand.adapt.general]

- 1 Each type instantiated from a class template specified in 29.5.5 meets the requirements of a random number engine adaptor (29.5.3.5) type.
- 2 Except where specified otherwise, the complexity of each function specified in 29.5.5 is constant.
- 3 Except where specified otherwise, no function described in 29.5.5 throws an exception.
- 4 Every function described in 29.5.5 that has a function parameter `q` of type `Sseq&` for a template type parameter named `Sseq` that is different from type `seed_seq` throws what and when the invocation of `q.generate` throws.
- 5 Descriptions are provided in 29.5.5 only for adaptor operations that are not described in subclause 29.5.3.5 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.
- 6 Each template specified in 29.5.5 requires one or more relationships, involving the value(s) of its constant template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

29.5.5.2 Class template `discard_block_engine`

[rand.adapt.disc]

- 1 A `discard_block_engine` random number engine adaptor produces random numbers selected from those produced by some base engine e . The state $\mathbf{x}_i$ of a `discard_block_engine` engine adaptor object $\mathbf{x}$ consists of the state $\mathbf{e}_i$ of its base engine $\mathbf{e}$ and an additional integer n . The size of the state is the size of e 's state plus 1.
- 2 The transition algorithm discards all but $r > 0$ values from each block of $p \geq r$ values delivered by e . The state transition is performed as follows: If $n \geq r$, advance the state of $\mathbf{e}$ from $\mathbf{e}_i$ to $\mathbf{e}_{i+p-r}$ and set n to 0. In any case, then increment n and advance $\mathbf{e}$'s then-current state $\mathbf{e}_j$ to $\mathbf{e}_{j+1}$.
- 3 The generation algorithm yields the value returned by the last invocation of `e()` while advancing $\mathbf{e}$'s state as described above.

```
namespace std {
    template<class Engine, size_t p, size_t r>
    class discard_block_engine {
    public:
        // types
        using result_type = Engine::result_type;

        // engine characteristics
        static constexpr size_t block_size = p;
        static constexpr size_t used_block = r;
        static constexpr result_type min() { return Engine::min(); }
        static constexpr result_type max() { return Engine::max(); }

        // constructors and seeding functions
        discard_block_engine();
        explicit discard_block_engine(const Engine& e);
    };
}
```

```

explicit discard_block_engine(Engine&& e);
explicit discard_block_engine(result_type s);
template<class Sseq> explicit discard_block_engine(Sseq& q);
void seed();
void seed(result_type s);
template<class Sseq> void seed(Sseq& q);

// equality operators
friend bool operator==(const discard_block_engine& x, const discard_block_engine& y);

// generating functions
result_type operator()();
void discard(unsigned long long z);

// property functions
const Engine& base() const noexcept { return e; }

// inserters and extractors
template<class charT, class traits>
friend basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const discard_block_engine& x); // hosted
template<class charT, class traits>
friend basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, discard_block_engine& x); // hosted

private:
    Engine e; // exposition only
    size_t n; // exposition only
};
}

```

⁴ The following relations shall hold: $0 < r$ and $r \leq p$.

⁵ The textual representation consists of the textual representation of **e** followed by the value of **n**.

⁶ In addition to its behavior pursuant to subclause 29.5.3.5, each constructor that is not a copy constructor sets **n** to 0.

29.5.5.3 Class template independent_bits_engine

[rand.adapt.ibits]

¹ An **independent_bits_engine** random number engine adaptor combines random numbers that are produced by some base engine *e*, so as to produce random numbers with a specified number of bits *w*. The state $\mathbf{x}_i$ of an **independent_bits_engine** engine adaptor object **x** consists of the state $\mathbf{e}_i$ of its base engine **e**; the size of the state is the size of *e*'s state.

² The transition and generation algorithms are described in terms of the following integral constants:

(2.1) — Let $R = \mathbf{e}.\text{max}() - \mathbf{e}.\text{min}() + 1$ and $m = \lfloor \log_2 R \rfloor$.

(2.2) — With *n* as determined below, let $w_0 = \lfloor w/n \rfloor$, $n_0 = n - w \bmod n$, $y_0 = 2^{w_0} \lfloor R/2^{w_0} \rfloor$, and $y_1 = 2^{w_0+1} \lfloor R/2^{w_0+1} \rfloor$.

(2.3) — Let $n = \lceil w/m \rceil$ if and only if the relation $R - y_0 \leq \lfloor y_0/n \rfloor$ holds as a result. Otherwise let $n = 1 + \lceil w/m \rceil$.

[Note 1: The relation $w = n_0 w_0 + (n - n_0)(w_0 + 1)$ always holds. — end note]

³ The transition algorithm is carried out by invoking **e()** as often as needed to obtain n_0 values less than $y_0 + \mathbf{e}.\text{min}()$ and $n - n_0$ values less than $y_1 + \mathbf{e}.\text{min}()$.

⁴ The generation algorithm uses the values produced while advancing the state as described above to yield a quantity *S* obtained as if by the following algorithm:

```

S = 0;
for (k = 0; k ≠ n0; k += 1) {
    do u = e() - e.min(); while (u ≥ y0);
    S = 2w0 · S + u mod 2w0;
}
for (k = n0; k ≠ n; k += 1) {
    do u = e() - e.min(); while (u ≥ y1);

```

```

     $S = 2^{w_0+1} \cdot S + u \bmod 2^{w_0+1};$ 
}

namespace std {
    template<class Engine, size_t w, class UIntType>
    class independent_bits_engine {
    public:
        // types
        using result_type = UIntType;

        // engine characteristics
        static constexpr result_type min() { return 0; }
        static constexpr result_type max() { return  $2^w - 1$ ; }

        // constructors and seeding functions
        independent_bits_engine();
        explicit independent_bits_engine(const Engine& e);
        explicit independent_bits_engine(Engine&& e);
        explicit independent_bits_engine(result_type s);
        template<class Sseq> explicit independent_bits_engine(Sseq& q);
        void seed();
        void seed(result_type s);
        template<class Sseq> void seed(Sseq& q);

        // equality operators
        friend bool operator==(const independent_bits_engine& x, const independent_bits_engine& y);

        // generating functions
        result_type operator()();
        void discard(unsigned long long z);

        // property functions
        const Engine& base() const noexcept { return e; }

        // inserters and extractors
        template<class charT, class traits>
        friend basic_ostream<charT, traits>&
            operator<<(basic_ostream<charT, traits>& os, const independent_bits_engine& x); // hosted
        template<class charT, class traits>
        friend basic_istream<charT, traits>&
            operator>>(basic_istream<charT, traits>& is, independent_bits_engine& x); // hosted

    private:
        Engine e; // exposition only
    };
}

```

⁵ The following relations shall hold: $0 < w$ and $w \leq \text{numeric_limits}<\text{result_type}>::\text{digits}$.

⁶ The textual representation consists of the textual representation of e .

29.5.5.4 Class template `shuffle_order_engine` [rand.adapt.shuf]

- ¹ A `shuffle_order_engine` random number engine adaptor produces the same random numbers that are produced by some base engine e , but delivers them in a different sequence. The state $\mathbf{x}_i$ of a `shuffle_order_engine` engine adaptor object $\mathbf{x}$ consists of the state $\mathbf{e}_i$ of its base engine $\mathbf{e}$, an additional value Y of the type delivered by $\mathbf{e}$, and an additional sequence V of k values also of the type delivered by $\mathbf{e}$. The size of the state is the size of e 's state plus $k + 1$.
- ² The transition algorithm permutes the values produced by e . The state transition is performed as follows:
- (2.1) — Calculate an integer $j = \left\lfloor \frac{k \cdot (Y - e_{\min})}{e_{\max} - e_{\min} + 1} \right\rfloor$.
 - (2.2) — Set Y to V_j and then set V_j to $\mathbf{e}()$.
- ³ The generation algorithm yields the last value of Y produced while advancing $\mathbf{e}$'s state as described above.

```

namespace std {
    template<class Engine, size_t k>
    class shuffle_order_engine {
    public:
        // types
        using result_type = Engine::result_type;

        // engine characteristics
        static constexpr size_t table_size = k;
        static constexpr result_type min() { return Engine::min(); }
        static constexpr result_type max() { return Engine::max(); }

        // constructors and seeding functions
        shuffle_order_engine();
        explicit shuffle_order_engine(const Engine& e);
        explicit shuffle_order_engine(Engine&& e);
        explicit shuffle_order_engine(result_type s);
        template<class Sseq> explicit shuffle_order_engine(Sseq& q);
        void seed();
        void seed(result_type s);
        template<class Sseq> void seed(Sseq& q);

        // equality operators
        friend bool operator==(const shuffle_order_engine& x, const shuffle_order_engine& y);

        // generating functions
        result_type operator()();
        void discard(unsigned long long z);

        // property functions
        const Engine& base() const noexcept { return e; }

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const shuffle_order_engine& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, shuffle_order_engine& x);

    private:
        Engine e;           // exposition only
        result_type V[k];    // exposition only
        result_type Y;       // exposition only
    };
}

```

⁴ The following relation shall hold: $0 < k$.

⁵ The textual representation consists of the textual representation of **e**, followed by the **k** values of **V**, followed by the value of **Y**.

⁶ In addition to its behavior pursuant to subclause 29.5.3.5, each constructor that is not a copy constructor initializes $V[0], \dots, V[k - 1]$ and **Y**, in that order, with values returned by successive invocations of **e()**.

29.5.6 Engines and engine adaptors with predefined parameters [rand.predef]

```

using minstd_rand0 =
    linear_congruential_engine<uint_fast32_t, 16'807, 0, 2'147'483'647>;

```

¹ *Required behavior:* The 10000th consecutive invocation of a default-constructed object of type `minstd_rand0` produces the value 1043618065.

```

using minstd_rand =
    linear_congruential_engine<uint_fast32_t, 48'271, 0, 2'147'483'647>;
2    Required behavior: The 10000th consecutive invocation of a default-constructed object of type minstd_
    rand produces the value 399268537.

using mt19937 =
    mersenne_twister_engine<uint_fast32_t, 32, 624, 397, 31,
        0x9908'b0df, 11, 0xffff'ffff, 7, 0x9d2c'5680, 15, 0xefc6'0000, 18, 1'812'433'253>;
3    Required behavior: The 10000th consecutive invocation of a default-constructed object of type mt19937
    produces the value 4123659995.

using mt19937_64 =
    mersenne_twister_engine<uint_fast64_t, 64, 312, 156, 31,
        0xb502'6f5a'a966'19e9, 29, 0x5555'5555'5555'5555, 17,
        0x71d6'7fff'eda6'0000, 37, 0xffff'7'eee0'0000'0000, 43, 6'364'136'223'846'793'005>;
4    Required behavior: The 10000th consecutive invocation of a default-constructed object of type mt19937_
    64 produces the value 9981545732273789042.

using ranlux24_base =
    subtract_with_carry_engine<uint_fast32_t, 24, 10, 24>;
5    Required behavior: The 10000th consecutive invocation of a default-constructed object of type ran-
    lux24_base produces the value 7937952.

using ranlux48_base =
    subtract_with_carry_engine<uint_fast64_t, 48, 5, 12>;
6    Required behavior: The 10000th consecutive invocation of a default-constructed object of type ran-
    lux48_base produces the value 61839128582725.

using ranlux24 = discard_block_engine<ranlux24_base, 223, 23>;
7    Required behavior: The 10000th consecutive invocation of a default-constructed object of type ranlux24
    produces the value 9901578.

using ranlux48 = discard_block_engine<ranlux48_base, 389, 11>;
8    Required behavior: The 10000th consecutive invocation of a default-constructed object of type ranlux48
    produces the value 249142670248501.

using knuth_b = shuffle_order_engine<minstd_rand0, 256>;
9    Required behavior: The 10000th consecutive invocation of a default-constructed object of type knuth_b
    produces the value 1112339016.

using default_random_engine = implementation-defined;
10    Remarks: The choice of engine type named by this typedef is implementation-defined.
    [Note 1: The implementation can select this type on the basis of performance, size, quality, or any combination
    of such factors, so as to provide at least acceptable engine behavior for relatively casual, inexpert, and/or
    lightweight use. Because different implementations can select different underlying engine types, code that uses
    this typedef need not generate identical sequences across implementations. — end note]

using philox4x32 =
    philox_engine<uint_fast32_t, 32, 4, 10,
        0xCD9E8D57, 0x9E3779B9, 0xD2511F53, 0xBB67AE85>;
11    Required behavior: The 10000th consecutive invocation a default-constructed object of type philox4x32
    produces the value 1955073260.

using philox4x64 =
    philox_engine<uint_fast64_t, 64, 4, 10,
        0xCA5A826395121157, 0x9E3779B97F4A7C15, 0xD2E7470EE14C6C93, 0xBB67AE8584CAA73B>;
12    Required behavior: The 10000th consecutive invocation a default-constructed object of type philox4x64
    produces the value 3409172418970261260.

```


29.5.7 Class random_device**[rand.device]**

- ¹ A `random_device` uniform random bit generator produces nondeterministic random numbers.
- ² If implementation limitations prevent generating nondeterministic random numbers, the implementation may employ a random number engine.

```
namespace std {
    class random_device {
    public:
        // types
        using result_type = unsigned int;

        // generator characteristics
        static constexpr result_type min() { return numeric_limits<result_type>::min(); }
        static constexpr result_type max() { return numeric_limits<result_type>::max(); }

        // constructors
        random_device() : random_device(implementation-defined) {}
        explicit random_device(const string& token);

        // generating functions
        result_type operator()();

        // property functions
        double entropy() const noexcept;

        // no copy functions
        random_device(const random_device&) = delete;
        void operator=(const random_device&) = delete;
    };
}
```

```
explicit random_device(const string& token);
```

- ³ *Throws:* A value of an implementation-defined type derived from `exception` if the `random_device` cannot be initialized.
- ⁴ *Remarks:* The semantics of the `token` parameter and the token value used by the default constructor are implementation-defined.²²²

```
double entropy() const noexcept;
```

- ⁵ *Returns:* If the implementation employs a random number engine, returns 0.0. Otherwise, returns an entropy estimate²²³ for the random numbers returned by `operator()`, in the range `min()` to `log2(max() + 1)`.

```
result_type operator()();
```

- ⁶ *Returns:* A nondeterministic random value, uniformly distributed between `min()` and `max()` (inclusive). It is implementation-defined how these values are generated.
- ⁷ *Throws:* A value of an implementation-defined type derived from `exception` if a random number cannot be obtained.

29.5.8 Utilities**[rand.util]****29.5.8.1 Class seed_seq****[rand.util.seedseq]**

```
namespace std {
    class seed_seq {
    public:
        // types
        using result_type = uint_least32_t;
```

²²²) The parameter is intended to allow an implementation to differentiate between different sources of randomness.

²²³) If a device has n states whose respective probabilities are $P_0, \dots, P_{n-1}$, the device entropy S is defined as

$$S = - \sum_{i=0}^{n-1} P_i \cdot \log P_i.$$

```

// constructors
seed_seq() noexcept;
template<class T>
    seed_seq(initializer_list<T> il);
template<class InputIterator>
    seed_seq(InputIterator begin, InputIterator end);

// generating functions
template<class RandomAccessIterator>
    void generate(RandomAccessIterator begin, RandomAccessIterator end);

// property functions
size_t size() const noexcept;
template<class OutputIterator>
    void param(OutputIterator dest) const;

// no copy functions
seed_seq(const seed_seq&) = delete;
void operator=(const seed_seq&) = delete;

private:
    vector<result_type> v;      // exposition only
};
}

```

seed_seq() noexcept;

1 *Postconditions:* v.empty() is true.

```

template<class T>
    seed_seq(initializer_list<T> il);

```

2 *Constraints:* T is an integer type.

3 *Effects:* Same as seed_seq(il.begin(), il.end()).

```

template<class InputIterator>
    seed_seq(InputIterator begin, InputIterator end);

```

4 *Mandates:* iterator_traits<InputIterator>::value_type is an integer type.

5 *Preconditions:* InputIterator meets the Cpp17InputIterator requirements (24.3.5.3).

6 *Effects:* Initializes v by the following algorithm:

```

    for (InputIterator s = begin; s != end; ++s)
        v.push_back((*s) mod 232);

```

```

template<class RandomAccessIterator>
    void generate(RandomAccessIterator begin, RandomAccessIterator end);

```

7 *Mandates:* iterator_traits<RandomAccessIterator>::value_type is an unsigned integer type capable of accommodating 32-bit quantities.

8 *Preconditions:* RandomAccessIterator meets the Cpp17RandomAccessIterator requirements (24.3.5.7) and the requirements of a mutable iterator.

9 *Effects:* Does nothing if begin == end. Otherwise, with $s = v.size()$ and $n = end - begin$, fills the supplied range [begin, end) according to the following algorithm in which each operation is to be carried out modulo 2^{32} , each indexing operator applied to begin is to be taken modulo n , and $T(x)$ is defined as $x \text{ xor } (x \text{ rshift } 27)$:

(9.1) — By way of initialization, set each element of the range to the value 0x8b8b8b8b. Additionally, for use in subsequent steps, let $p = (n - t)/2$ and let $q = p + t$, where

$$t = (n \geq 623) ? 11 : (n \geq 68) ? 7 : (n \geq 39) ? 5 : (n \geq 7) ? 3 : (n - 1)/2;$$

- (9.2) — With m as the larger of $s + 1$ and n , transform the elements of the range: iteratively for $k = 0, \dots, m - 1$, calculate values

$$\begin{aligned} r_1 &= 1664525 \cdot T(\text{begin}[k] \text{ xor } \text{begin}[k + p] \text{ xor } \text{begin}[k - 1]) \\ r_2 &= r_1 + \begin{cases} s & , k = 0 \\ k \bmod n + v[k - 1] & , 0 < k \leq s \\ k \bmod n & , s < k \end{cases} \end{aligned}$$

and, in order, increment $\text{begin}[k + p]$ by r_1 , increment $\text{begin}[k + q]$ by r_2 , and set $\text{begin}[k]$ to r_2 .

- (9.3) — Transform the elements of the range again, beginning where the previous step ended: iteratively for $k = m, \dots, m + n - 1$, calculate values

$$\begin{aligned} r_3 &= 1566083941 \cdot T(\text{begin}[k] + \text{begin}[k + p] + \text{begin}[k - 1]) \\ r_4 &= r_3 - (k \bmod n) \end{aligned}$$

and, in order, update $\text{begin}[k + p]$ by xoring it with r_3 , update $\text{begin}[k + q]$ by xoring it with r_4 , and set $\text{begin}[k]$ to r_4 .

10 *Throws:* What and when `RandomAccessIterator` operations of `begin` and `end` throw.

`size_t size() const noexcept;`

11 *Returns:* The number of 32-bit units that would be returned by a call to `param()`.

12 *Complexity:* Constant time.

`template<class OutputIterator>`
`void param(OutputIterator dest) const;`

13 *Mandates:* Values of type `result_type` are writable (24.3.1) to `dest`.

14 *Preconditions:* `OutputIterator` meets the `Cpp17OutputIterator` requirements (24.3.5.4).

15 *Effects:* Copies the sequence of prepared 32-bit units to the given destination, as if by executing the following statement:

`copy(v.begin(), v.end(), dest);`

16 *Throws:* What and when `OutputIterator` operations of `dest` throw.

29.5.8.2 Function template `generate_canonical`

[rand.util.canonical]

`template<class RealType, size_t digits, class URBG>`
`RealType generate_canonical(URBG& g);`

1 Let

- (1.1) — r be `numeric_limits<RealType>::radix`,
- (1.2) — R be `g.max() - g.min() + 1`,
- (1.3) — d be the smaller of `digits` and `numeric_limits<RealType>::digits`,²²⁴
- (1.4) — k be the smallest integer such that $R^k \geq r^d$, and
- (1.5) — x be $\lfloor R^k / r^d \rfloor$.

An *attempt* is k invocations of `g()` to obtain values $g_0, \dots, g_{k-1}$, respectively, and the calculation of a quantity S given by Formula 29.1:

$$S = \sum_{i=0}^{k-1} (g_i - \text{g.min}()) \cdot R^i \quad (29.1)$$

2 *Effects:* Attempts are made until $S < xr^d$.

[Note 1: When R is a power of r , precisely one attempt is made. — end note]

3 *Returns:* $\lfloor S/x \rfloor / r^d$.

²²⁴ d is introduced to avoid any attempt to produce more bits of randomness than can be held in `RealType`.

[Note 2: The return value c satisfies $0 \leq c < 1$. — end note]

4 *Throws:* What and when **g** throws.

5 *Complexity:* Exactly k invocations of **g** per attempt.

6 [Note 3: If the values g_i produced by **g** are uniformly distributed, **generate_canonical**'s results are distributed as uniformly as possible. Obtaining a value in this way can be a useful step in the process of transforming a value generated by a uniform random bit generator into a value that can be delivered by a random number distribution. — end note]

7 [Note 4: When R is a power of r , an implementation can avoid using an arithmetic type that is wider than the output when computing S . — end note]

29.5.9 Random number distribution class templates

[rand.dist]

29.5.9.1 General

[rand.dist.general]

- 1 Each type instantiated from a class template specified in 29.5.9 meets the requirements of a random number distribution (29.5.3.6) type.
- 2 Descriptions are provided in 29.5.9 only for distribution operations that are not described in 29.5.3.6 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.
- 3 The algorithms for producing each of the specified distributions are implementation-defined.
- 4 The value of each probability density function $p(z)$ and of each discrete probability function $P(z_i)$ specified in this subclause is 0 everywhere outside its stated domain.

29.5.9.2 Uniform distributions

[rand.dist.uni]

29.5.9.2.1 Class template uniform_int_distribution

[rand.dist.uni.int]

- 1 A **uniform_int_distribution** random number distribution produces random integers i , $a \leq i \leq b$, distributed according to the constant discrete probability function in Formula 29.2.

$$P(i | a, b) = 1/(b - a + 1) \quad (29.2)$$

```
namespace std {
    template<class IntType = int>
    class uniform_int_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructors and reset functions
        uniform_int_distribution() : uniform_int_distribution(0) {}
        explicit uniform_int_distribution(IntType a, IntType b = numeric_limits<IntType>::max());
        explicit uniform_int_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const uniform_int_distribution& x, const uniform_int_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        result_type a() const;
        result_type b() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;
    };
}
```

```

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os,           // hosted
                    const uniform_int_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is,         // hosted
                    uniform_int_distribution& x);
};
}

```

```
explicit uniform_int_distribution(IntType a, IntType b = numeric_limits<IntType>::max());
```

2 *Preconditions:* $a \leq b$.

3 *Remarks:* a and b correspond to the respective parameters of the distribution.

```
result_type a() const;
```

4 *Returns:* The value of the a parameter with which the object was constructed.

```
result_type b() const;
```

5 *Returns:* The value of the b parameter with which the object was constructed.

29.5.9.2.2 Class template `uniform_real_distribution` [rand.dist.uni.real]

1 A `uniform_real_distribution` random number distribution produces random numbers x , $a \leq x < b$, distributed according to the constant probability density function in Formula 29.3.

$$p(x | a, b) = 1/(b - a) \quad (29.3)$$

[Note 1: This implies that $p(x | a, b)$ is undefined when $a == b$. — end note]

```

namespace std {
    template<class RealType = double>
        class uniform_real_distribution {
        public:
            // types
            using result_type = RealType;
            using param_type = unspecified;

            // constructors and reset functions
            uniform_real_distribution() : uniform_real_distribution(0.0) {}
            explicit uniform_real_distribution(RealType a, RealType b = 1.0);
            explicit uniform_real_distribution(const param_type& parm);
            void reset();

            // equality operators
            friend bool operator==(const uniform_real_distribution& x,
                                   const uniform_real_distribution& y);

            // generating functions
            template<class URBG>
                result_type operator()(URBG& g);
            template<class URBG>
                result_type operator()(URBG& g, const param_type& parm);

            // property functions
            result_type a() const;
            result_type b() const;
            param_type param() const;
            void param(const param_type& parm);
            result_type min() const;
            result_type max() const;

```

```

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const uniform_real_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, uniform_real_distribution& x);
};
}

```

```
explicit uniform_real_distribution(RealType a, RealType b = 1.0);
```

2 *Preconditions:* $a \leq b$ and $b - a \leq \text{numeric_limits}<\text{RealType}>::\text{max}()$.

3 *Remarks:* a and b correspond to the respective parameters of the distribution.

```
result_type a() const;
```

4 *Returns:* The value of the a parameter with which the object was constructed.

```
result_type b() const;
```

5 *Returns:* The value of the b parameter with which the object was constructed.

29.5.9.3 Bernoulli distributions

[rand.dist.bern]

29.5.9.3.1 Class `bernoulli_distribution`

[rand.dist.bern.bernoulli]

1 A `bernoulli_distribution` random number distribution produces `bool` values b distributed according to the discrete probability function in Formula 29.4.

$$P(b|p) = \begin{cases} p & \text{if } b = \text{true} \\ 1 - p & \text{if } b = \text{false} \end{cases} \quad (29.4)$$

```

namespace std {
    class bernoulli_distribution {
    public:
        // types
        using result_type = bool;
        using param_type = unspecified;

        // constructors and reset functions
        bernoulli_distribution() : bernoulli_distribution(0.5) {}
        explicit bernoulli_distribution(double p);
        explicit bernoulli_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const bernoulli_distribution& x, const bernoulli_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        double p() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const bernoulli_distribution& x);
    };
}

```

```

    template<class charT, class traits>
        friend basic_istream<charT, traits>&
            operator>>(basic_istream<charT, traits>& is, bernoulli_distribution& x);
};
}

```

```
explicit bernoulli_distribution(double p);
```

² *Preconditions:* $0 \leq p \leq 1$.

³ *Remarks:* p corresponds to the parameter of the distribution.

```
double p() const;
```

⁴ *Returns:* The value of the p parameter with which the object was constructed.

29.5.9.3.2 Class template `binomial_distribution` [rand.dist.bern.bin]

¹ A `binomial_distribution` random number distribution produces integer values $i \geq 0$ distributed according to the discrete probability function in Formula 29.5.

$$P(i | t, p) = \binom{t}{i} \cdot p^i \cdot (1 - p)^{t-i} \quad (29.5)$$

```

namespace std {
    template<class IntType = int>
    class binomial_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructors and reset functions
        binomial_distribution() : binomial_distribution(1) {}
        explicit binomial_distribution(IntType t, double p = 0.5);
        explicit binomial_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const binomial_distribution& x, const binomial_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        IntType t() const;
        double p() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const binomial_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, binomial_distribution& x);
    };
}

```

```
explicit binomial_distribution(IntType t, double p = 0.5);
```

2 *Preconditions:* $0 \leq p \leq 1$ and $0 \leq t$.

3 *Remarks:* t and p correspond to the respective parameters of the distribution.

```
IntType t() const;
```

4 *Returns:* The value of the t parameter with which the object was constructed.

```
double p() const;
```

5 *Returns:* The value of the p parameter with which the object was constructed.

29.5.9.3.3 Class template `geometric_distribution` [rand.dist.bern.geo]

1 A `geometric_distribution` random number distribution produces integer values $i \geq 0$ distributed according to the discrete probability function in Formula 29.6.

$$P(i|p) = p \cdot (1 - p)^i \quad (29.6)$$

```
namespace std {
    template<class IntType = int>
    class geometric_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructors and reset functions
        geometric_distribution() : geometric_distribution(0.5) {}
        explicit geometric_distribution(double p);
        explicit geometric_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const geometric_distribution& x, const geometric_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        double p() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const geometric_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, geometric_distribution& x);
    };
}
```

```
explicit geometric_distribution(double p);
```

2 *Preconditions:* $0 < p < 1$.

3 *Remarks:* p corresponds to the parameter of the distribution.

```
double p() const;
```

4 *Returns:* The value of the p parameter with which the object was constructed.

29.5.9.3.4 Class template negative_binomial_distribution**[rand.dist.bern.negbin]**

- ¹ A `negative_binomial_distribution` random number distribution produces random integers $i \geq 0$ distributed according to the discrete probability function in Formula 29.7.

$$P(i|k, p) = \binom{k+i-1}{i} \cdot p^k \cdot (1-p)^i \quad (29.7)$$

[Note 1: This implies that $P(i|k, p)$ is undefined when $p == 1$. — end note]

```
namespace std {
    template<class IntType = int>
    class negative_binomial_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructor and reset functions
        negative_binomial_distribution() : negative_binomial_distribution(1) {}
        explicit negative_binomial_distribution(IntType k, double p = 0.5);
        explicit negative_binomial_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const negative_binomial_distribution& x,
                               const negative_binomial_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        IntType k() const;
        double p() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const negative_binomial_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, negative_binomial_distribution& x);
    };
}
```

```
explicit negative_binomial_distribution(IntType k, double p = 0.5);
```

- ² *Preconditions:* $0 < p \leq 1$ and $0 < k$.

- ³ *Remarks:* k and p correspond to the respective parameters of the distribution.

```
IntType k() const;
```

- ⁴ *Returns:* The value of the k parameter with which the object was constructed.

```
double p() const;
```

- ⁵ *Returns:* The value of the p parameter with which the object was constructed.

29.5.9.4 Poisson distributions

[rand.dist.pois]

29.5.9.4.1 Class template poisson_distribution

[rand.dist.pois.poisson]

- ¹ A `poisson_distribution` random number distribution produces integer values $i \geq 0$ distributed according to the discrete probability function in Formula 29.8.

$$P(i|\mu) = \frac{e^{-\mu} \mu^i}{i!} \quad (29.8)$$

The distribution parameter μ is also known as this distribution's *mean*.

```
namespace std {
    template<class IntType = int>
    class poisson_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructors and reset functions
        poisson_distribution() : poisson_distribution(1.0) {}
        explicit poisson_distribution(double mean);
        explicit poisson_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const poisson_distribution& x, const poisson_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        double mean() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const poisson_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, poisson_distribution& x);
    };
}
```

```
explicit poisson_distribution(double mean);
```

- ² *Preconditions:* $0 < \text{mean}$.

- ³ *Remarks:* `mean` corresponds to the parameter of the distribution.

```
double mean() const;
```

- ⁴ *Returns:* The value of the `mean` parameter with which the object was constructed.

29.5.9.4.2 Class template exponential_distribution

[rand.dist.pois.exp]

- ¹ An `exponential_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function in Formula 29.9.

$$p(x|\lambda) = \lambda e^{-\lambda x} \quad (29.9)$$

```

namespace std {
    template<class RealType = double>
    class exponential_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructors and reset functions
        exponential_distribution() : exponential_distribution(1.0) {}
        explicit exponential_distribution(RealType lambda);
        explicit exponential_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const exponential_distribution& x, const exponential_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        RealType lambda() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const exponential_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, exponential_distribution& x);
    };
}

```

explicit exponential_distribution(RealType lambda);

² *Preconditions:* $0 < \text{lambda}$.

³ *Remarks:* lambda corresponds to the parameter of the distribution.

RealType lambda() const;

⁴ *Returns:* The value of the lambda parameter with which the object was constructed.

29.5.9.4.3 Class template gamma_distribution [rand.dist.pois.gamma]

¹ A gamma_distribution random number distribution produces random numbers $x > 0$ distributed according to the probability density function in Formula 29.10.

$$p(x | \alpha, \beta) = \frac{e^{-x/\beta}}{\beta^\alpha \cdot \Gamma(\alpha)} \cdot x^{\alpha-1} \quad (29.10)$$

```

namespace std {
    template<class RealType = double>
    class gamma_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

```

```

// constructors and reset functions
gamma_distribution() : gamma_distribution(1.0) {}
explicit gamma_distribution(RealType alpha, RealType beta = 1.0);
explicit gamma_distribution(const param_type& parm);
void reset();

// equality operators
friend bool operator==(const gamma_distribution& x, const gamma_distribution& y);

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
RealType alpha() const;
RealType beta() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const gamma_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, gamma_distribution& x);
};
}

```

```
explicit gamma_distribution(RealType alpha, RealType beta = 1.0);
```

2 *Preconditions:* $0 < \alpha$ and $0 < \beta$.

3 *Remarks:* α and β correspond to the parameters of the distribution.

```
RealType alpha() const;
```

4 *Returns:* The value of the α parameter with which the object was constructed.

```
RealType beta() const;
```

5 *Returns:* The value of the β parameter with which the object was constructed.

29.5.9.4.4 Class template `weibull_distribution`

[rand.dist.pois.weibull]

1 A `weibull_distribution` random number distribution produces random numbers $x \geq 0$ distributed according to the probability density function in Formula 29.11.

$$p(x | a, b) = \frac{a}{b} \cdot \left(\frac{x}{b}\right)^{a-1} \cdot \exp\left(-\left(\frac{x}{b}\right)^a\right) \quad (29.11)$$

```

namespace std {
    template<class RealType = double>
    class weibull_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        weibull_distribution() : weibull_distribution(1.0) {}
        explicit weibull_distribution(RealType a, RealType b = 1.0);
        explicit weibull_distribution(const param_type& parm);
        void reset();
    };
}

```

```

// equality operators
friend bool operator==(const weibull_distribution& x, const weibull_distribution& y);

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
RealType a() const;
RealType b() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const weibull_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, weibull_distribution& x);
};
}

```

```
explicit weibull_distribution(RealType a, RealType b = 1.0);
```

2 *Preconditions:* $0 < a$ and $0 < b$.

3 *Remarks:* a and b correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the a parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the b parameter with which the object was constructed.

29.5.9.4.5 Class template `extreme_value_distribution` [rand.dist.pois.extreme]

1 An `extreme_value_distribution` random number distribution produces random numbers x distributed according to the probability density function in Formula 29.12.²²⁵

$$p(x | a, b) = \frac{1}{b} \cdot \exp \left(\frac{a - x}{b} - \exp \left(\frac{a - x}{b} \right) \right) \quad (29.12)$$

```

namespace std {
    template<class RealType = double>
    class extreme_value_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        extreme_value_distribution() : extreme_value_distribution(0.0) {}
        explicit extreme_value_distribution(RealType a, RealType b = 1.0);
        explicit extreme_value_distribution(const param_type& parm);
        void reset();
    };
}

```

²²⁵) The distribution corresponding to this probability density function is also known (with a possible change of variable) as the Gumbel Type I, the log-Weibull, or the Fisher-Tippett Type I distribution.

```

// equality operators
friend bool operator==(const extreme_value_distribution& x,
                       const extreme_value_distribution& y);

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
RealType a() const;
RealType b() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const extreme_value_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, extreme_value_distribution& x);
};
}

```

```
explicit extreme_value_distribution(RealType a, RealType b = 1.0);
```

2 *Preconditions:* $0 < b$.

3 *Remarks:* a and b correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the a parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the b parameter with which the object was constructed.

29.5.9.5 Normal distributions

[rand.dist.norm]

29.5.9.5.1 Class template normal_distribution

[rand.dist.norm.normal]

1 A `normal_distribution` random number distribution produces random numbers x distributed according to the probability density function in Formula 29.13.

$$p(x | \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (29.13)$$

The distribution parameters μ and σ are also known as this distribution's *mean* and *standard deviation*.

```

namespace std {
    template<class RealType = double>
    class normal_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructors and reset functions
        normal_distribution() : normal_distribution(0.0) {}
        explicit normal_distribution(RealType mean, RealType stddev = 1.0);
        explicit normal_distribution(const param_type& parm);
        void reset();
    };
}

```

```

// equality operators
friend bool operator==(const normal_distribution& x, const normal_distribution& y);

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
RealType mean() const;
RealType stddev() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const normal_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, normal_distribution& x);
};
}

```

```
explicit normal_distribution(RealType mean, RealType stddev = 1.0);
```

2 *Preconditions:* $0 < \text{stddev}$.

3 *Remarks:* `mean` and `stddev` correspond to the respective parameters of the distribution.

```
RealType mean() const;
```

4 *Returns:* The value of the `mean` parameter with which the object was constructed.

```
RealType stddev() const;
```

5 *Returns:* The value of the `stddev` parameter with which the object was constructed.

29.5.9.5.2 Class template `lognormal_distribution` [rand.dist.norm.lognormal]

1 A `lognormal_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function in Formula 29.14.

$$p(x | m, s) = \frac{1}{sx\sqrt{2\pi}} \cdot \exp\left(-\frac{(\ln x - m)^2}{2s^2}\right) \quad (29.14)$$

```

namespace std {
    template<class RealType = double>
    class lognormal_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        lognormal_distribution() : lognormal_distribution(0.0) {}
        explicit lognormal_distribution(RealType m, RealType s = 1.0);
        explicit lognormal_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const lognormal_distribution& x, const lognormal_distribution& y);
    };
}

```

```

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
RealType m() const;
RealType s() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const lognormal_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, lognormal_distribution& x);
};
}

```

```
explicit lognormal_distribution(RealType m, RealType s = 1.0);
```

2 *Preconditions:* $0 < s$.

3 *Remarks:* m and s correspond to the respective parameters of the distribution.

```
RealType m() const;
```

4 *Returns:* The value of the m parameter with which the object was constructed.

```
RealType s() const;
```

5 *Returns:* The value of the s parameter with which the object was constructed.

29.5.9.5.3 Class template `chi_squared_distribution` [rand.dist.norm.chisq]

1 A `chi_squared_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function in Formula 29.15.

$$p(x | n) = \frac{x^{(n/2)-1} \cdot e^{-x/2}}{\Gamma(n/2) \cdot 2^{n/2}} \quad (29.15)$$

```

namespace std {
    template<class RealType = double>
    class chi_squared_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        chi_squared_distribution() : chi_squared_distribution(1.0) {}
        explicit chi_squared_distribution(RealType n);
        explicit chi_squared_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const chi_squared_distribution& x, const chi_squared_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);
    };
}

```



```

// property functions
RealType n() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const chi_squared_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, chi_squared_distribution& x);
};
}

explicit chi_squared_distribution(RealType n);

```

² *Preconditions:* $0 < n$.

³ *Remarks:* n corresponds to the parameter of the distribution.

RealType n() const;

⁴ *Returns:* The value of the n parameter with which the object was constructed.

29.5.9.5.4 Class template `cauchy_distribution`

[rand.dist.norm.cauchy]

¹ A `cauchy_distribution` random number distribution produces random numbers x distributed according to the probability density function in Formula 29.16.

$$p(x | a, b) = \left(\pi b \left(1 + \left(\frac{x - a}{b} \right)^2 \right) \right)^{-1} \quad (29.16)$$

```

namespace std {
    template<class RealType = double>
    class cauchy_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        cauchy_distribution() : cauchy_distribution(0.0) {}
        explicit cauchy_distribution(RealType a, RealType b = 1.0);
        explicit cauchy_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const cauchy_distribution& x, const cauchy_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        RealType a() const;
        RealType b() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;
    };
}

```

```

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const cauchy_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, cauchy_distribution& x);
};
}

```

```
explicit cauchy_distribution(RealType a, RealType b = 1.0);
```

2 *Preconditions:* $0 < b$.

3 *Remarks:* a and b correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the a parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the b parameter with which the object was constructed.

29.5.9.5.5 Class template `fisher_f_distribution`

[rand.dist.norm.f]

1 A `fisher_f_distribution` random number distribution produces random numbers $x \geq 0$ distributed according to the probability density function in Formula 29.17.

$$p(x|m,n) = \frac{\Gamma((m+n)/2)}{\Gamma(m/2)\Gamma(n/2)} \cdot \left(\frac{m}{n}\right)^{m/2} \cdot x^{(m/2)-1} \cdot \left(1 + \frac{mx}{n}\right)^{-(m+n)/2} \quad (29.17)$$

```

namespace std {
    template<class RealType = double>
        class fisher_f_distribution {
        public:
            // types
            using result_type = RealType;
            using param_type = unspecified;

            // constructor and reset functions
            fisher_f_distribution() : fisher_f_distribution(1.0) {}
            explicit fisher_f_distribution(RealType m, RealType n = 1.0);
            explicit fisher_f_distribution(const param_type& parm);
            void reset();

            // equality operators
            friend bool operator==(const fisher_f_distribution& x, const fisher_f_distribution& y);

            // generating functions
            template<class URBG>
                result_type operator()(URBG& g);
            template<class URBG>
                result_type operator()(URBG& g, const param_type& parm);

            // property functions
            RealType m() const;
            RealType n() const;
            param_type param() const;
            void param(const param_type& parm);
            result_type min() const;
            result_type max() const;

            // inserters and extractors
            template<class charT, class traits>
                friend basic_ostream<charT, traits>&
                    operator<<(basic_ostream<charT, traits>& os, const fisher_f_distribution& x);

```

```

    template<class charT, class traits>
        friend basic_istream<charT, traits>&
            operator>>(basic_istream<charT, traits>& is, fisher_f_distribution& x);
};
}

```

```
explicit fisher_f_distribution(RealType m, RealType n = 1);
```

2 *Preconditions:* $0 < m$ and $0 < n$.

3 *Remarks:* m and n correspond to the respective parameters of the distribution.

```
RealType m() const;
```

4 *Returns:* The value of the m parameter with which the object was constructed.

```
RealType n() const;
```

5 *Returns:* The value of the n parameter with which the object was constructed.

29.5.9.5.6 Class template `student_t_distribution` [rand.dist.norm.t]

1 A `student_t_distribution` random number distribution produces random numbers x distributed according to the probability density function in Formula 29.18.

$$p(x|n) = \frac{1}{\sqrt{n\pi}} \cdot \frac{\Gamma((n+1)/2)}{\Gamma(n/2)} \cdot \left(1 + \frac{x^2}{n}\right)^{-(n+1)/2} \quad (29.18)$$

```

namespace std {
    template<class RealType = double>
    class student_t_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        student_t_distribution() : student_t_distribution(1.0) {}
        explicit student_t_distribution(RealType n);
        explicit student_t_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const student_t_distribution& x, const student_t_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        RealType n() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const student_t_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, student_t_distribution& x);
    };
}

```

```
explicit student_t_distribution(RealType n);
```

2 *Preconditions:* $0 < n$.

3 *Remarks:* n corresponds to the parameter of the distribution.

```
RealType n() const;
```

4 *Returns:* The value of the n parameter with which the object was constructed.

29.5.9.6 Sampling distributions

[rand.dist.samp]

29.5.9.6.1 Class template `discrete_distribution`

[rand.dist.samp.discrete]

1 A `discrete_distribution` random number distribution produces random integers i , $0 \leq i < n$, distributed according to the discrete probability function in Formula 29.19.

$$P(i | p_0, \dots, p_{n-1}) = p_i \quad (29.19)$$

2 Unless specified otherwise, the distribution parameters are calculated as: $p_k = w_k / S$ for $k = 0, \dots, n-1$, in which the values w_k , commonly known as the *weights*, shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold: $0 < S = w_0 + \dots + w_{n-1}$.

```
namespace std {
    template<class IntType = int>
    class discrete_distribution {
    public:
        // types
        using result_type = IntType;
        using param_type = unspecified;

        // constructor and reset functions
        discrete_distribution();
        template<class InputIterator>
            discrete_distribution(InputIterator firstW, InputIterator lastW);
        discrete_distribution(initializer_list<double> wl);
        template<class UnaryOperation>
            discrete_distribution(size_t nw, double xmin, double xmax, UnaryOperation fw);
        explicit discrete_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const discrete_distribution& x, const discrete_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);

        // property functions
        vector<double> probabilities() const;
        param_type param() const;
        void param(const param_type& parm);
        result_type min() const;
        result_type max() const;

        // inserters and extractors
        template<class charT, class traits>
            friend basic_ostream<charT, traits>&
                operator<<(basic_ostream<charT, traits>& os, const discrete_distribution& x);
        template<class charT, class traits>
            friend basic_istream<charT, traits>&
                operator>>(basic_istream<charT, traits>& is, discrete_distribution& x);
    };
}
```

```
discrete_distribution();
```

3 *Effects:* Constructs a `discrete_distribution` object with $n = 1$ and $p_0 = 1$.

[*Note 1:* Such an object will always deliver the value 0. — *end note*]

```
template<class InputIterator>
discrete_distribution(InputIterator firstW, InputIterator lastW);
```

4 *Mandates:* `is_convertible_v<iterator_traits<InputIterator>::value_type, double>` is true.

5 *Preconditions:* `InputIterator` meets the *Cpp17InputIterator* requirements (24.3.5.3). If `firstW == lastW`, let $n = 1$ and $w_0 = 1$. Otherwise, `[firstW, lastW)` forms a sequence w of length $n > 0$.

6 *Effects:* Constructs a `discrete_distribution` object with probabilities given by the Formula 29.19.

```
discrete_distribution(initializer_list<double> wl);
```

7 *Effects:* Same as `discrete_distribution(wl.begin(), wl.end())`.

```
template<class UnaryOperation>
discrete_distribution(size_t nw, double xmin, double xmax, UnaryOperation fw);
```

8 *Mandates:* `is_invocable_r_v<double, UnaryOperation&, double>` is true.

9 *Preconditions:* If `nw = 0`, let $n = 1$, otherwise let $n = nw$. The relation $0 < \delta = (xmax - xmin)/n$ holds.

10 *Effects:* Constructs a `discrete_distribution` object with probabilities given by the formula above, using the following values: If `nw = 0`, let $w_0 = 1$. Otherwise, let $w_k = fw(xmin + k \cdot \delta + \delta/2)$ for $k = 0, \dots, n - 1$.

11 *Complexity:* The number of invocations of `fw` does not exceed n .

```
vector<double> probabilities() const;
```

12 *Returns:* A `vector<double>` whose `size` member returns n and whose `operator[]` member returns p_k when invoked with argument k for $k = 0, \dots, n - 1$.

29.5.9.6.2 Class template `piecewise_constant_distribution` [rand.dist.samp.pconst]

1 A `piecewise_constant_distribution` random number distribution produces random numbers x , $b_0 \leq x < b_n$, uniformly distributed over each subinterval $[b_i, b_{i+1})$ according to the probability density function in Formula 29.20.

$$p(x | b_0, \dots, b_n, \rho_0, \dots, \rho_{n-1}) = \rho_i, \text{ for } b_i \leq x < b_{i+1} \quad (29.20)$$

2 The $n + 1$ distribution parameters b_i , also known as this distribution's *interval boundaries*, shall satisfy the relation $b_i < b_{i+1}$ for $i = 0, \dots, n - 1$. Unless specified otherwise, the remaining n distribution parameters are calculated as:

$$\rho_k = \frac{w_k}{S \cdot (b_{k+1} - b_k)} \text{ for } k = 0, \dots, n - 1,$$

in which the values w_k , commonly known as the *weights*, shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold: $0 < S = w_0 + \dots + w_{n-1}$.

```
namespace std {
    template<class RealType = double>
    class piecewise_constant_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        piecewise_constant_distribution();
        template<class InputIteratorB, class InputIteratorW>
            piecewise_constant_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                           InputIteratorW firstW);

        template<class UnaryOperation>
            piecewise_constant_distribution(initializer_list<RealType> bl, UnaryOperation fw);
        template<class UnaryOperation>
            piecewise_constant_distribution(size_t nw, RealType xmin, RealType xmax,
                                           UnaryOperation fw);
    };
}
```

```

explicit piecewise_constant_distribution(const param_type& parm);
void reset();

// equality operators
friend bool operator==(const piecewise_constant_distribution& x,
                        const piecewise_constant_distribution& y);

// generating functions
template<class URBG>
    result_type operator()(URBG& g);
template<class URBG>
    result_type operator()(URBG& g, const param_type& parm);

// property functions
vector<result_type> intervals() const;
vector<result_type> densities() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const piecewise_constant_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, piecewise_constant_distribution& x);
};
}

```

```
piecewise_constant_distribution();
```

- 3 *Effects:* Constructs a `piecewise_constant_distribution` object with $n = 1$, $\rho_0 = 1$, $b_0 = 0$, and $b_1 = 1$.

```

template<class InputIteratorB, class InputIteratorW>
    piecewise_constant_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                   InputIteratorW firstW);

```

- 4 *Mandates:* Both of

(4.1) — `is_convertible_v<iterator_traits<InputIteratorB>::value_type, double>`

(4.2) — `is_convertible_v<iterator_traits<InputIteratorW>::value_type, double>`

are true.

- 5 *Preconditions:* `InputIteratorB` and `InputIteratorW` each meet the *Cpp17InputIterator* requirements (24.3.5.3). If `firstB == lastB` or `++firstB == lastB`, let $n = 1$, $w_0 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, $[firstB, lastB)$ forms a sequence b of length $n + 1$, the length of the sequence w starting from `firstW` is at least n , and any w_k for $k \geq n$ are ignored by the distribution.

- 6 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters as specified above.

```

template<class UnaryOperation>
    piecewise_constant_distribution(initializer_list<RealType> bl, UnaryOperation fw);

```

- 7 *Mandates:* `is_invocable_r_v<double, UnaryOperation&, double>` is true.

- 8 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters taken or calculated from the following values: If `bl.size() < 2`, let $n = 1$, $w_0 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, let $[bl.begin(), bl.end())$ form a sequence $b_0, \dots, b_n$, and let $w_k = fw((b_{k+1} + b_k)/2)$ for $k = 0, \dots, n-1$.

- 9 *Complexity:* The number of invocations of `fw` does not exceed n .

```

template<class UnaryOperation>
    piecewise_constant_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);

```

- 10 *Mandates:* `is_invocable_r_v<double, UnaryOperation&, double>` is true.

- 11 *Preconditions:* If `nw = 0`, let $n = 1$, otherwise let $n = \text{nw}$. The relation $0 < \delta = (\text{xmax} - \text{xmin})/n$ holds.
- 12 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters taken or calculated from the following values: Let $b_k = \text{xmin} + k \cdot \delta$ for $k = 0, \dots, n$, and $w_k = \text{fw}(b_k + \delta/2)$ for $k = 0, \dots, n-1$.
- 13 *Complexity:* The number of invocations of `fw` does not exceed n .

`vector<result_type> intervals() const;`

- 14 *Returns:* A `vector<result_type>` whose `size` member returns $n + 1$ and whose `operator[]` member returns b_k when invoked with argument k for $k = 0, \dots, n$.

`vector<result_type> densities() const;`

- 15 *Returns:* A `vector<result_type>` whose `size` member returns n and whose `operator[]` member returns ρ_k when invoked with argument k for $k = 0, \dots, n-1$.

29.5.9.6.3 Class template `piecewise_linear_distribution` [rand.dist.samp.plinear]

- ¹ A `piecewise_linear_distribution` random number distribution produces random numbers x , $b_0 \leq x < b_n$, distributed over each subinterval $[b_i, b_{i+1})$ according to the probability density function in Formula 29.21.

$$p(x | b_0, \dots, b_n, \rho_0, \dots, \rho_n) = \rho_i \cdot \frac{b_{i+1} - x}{b_{i+1} - b_i} + \rho_{i+1} \cdot \frac{x - b_i}{b_{i+1} - b_i}, \text{ for } b_i \leq x < b_{i+1}. \quad (29.21)$$

- ² The $n + 1$ distribution parameters b_i , also known as this distribution's *interval boundaries*, shall satisfy the relation $b_i < b_{i+1}$ for $i = 0, \dots, n-1$. Unless specified otherwise, the remaining $n + 1$ distribution parameters are calculated as $\rho_k = w_k/S$ for $k = 0, \dots, n$, in which the values w_k , commonly known as the *weights at boundaries*, shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold:

$$0 < S = \frac{1}{2} \cdot \sum_{k=0}^{n-1} (w_k + w_{k+1}) \cdot (b_{k+1} - b_k).$$

```
namespace std {
    template<class RealType = double>
    class piecewise_linear_distribution {
    public:
        // types
        using result_type = RealType;
        using param_type = unspecified;

        // constructor and reset functions
        piecewise_linear_distribution();
        template<class InputIteratorB, class InputIteratorW>
            piecewise_linear_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                         InputIteratorW firstW);
        template<class UnaryOperation>
            piecewise_linear_distribution(initializer_list<RealType> bl, UnaryOperation fw);
        template<class UnaryOperation>
            piecewise_linear_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);
        explicit piecewise_linear_distribution(const param_type& parm);
        void reset();

        // equality operators
        friend bool operator==(const piecewise_linear_distribution& x,
                              const piecewise_linear_distribution& y);

        // generating functions
        template<class URBG>
            result_type operator()(URBG& g);
        template<class URBG>
            result_type operator()(URBG& g, const param_type& parm);
    };
}
```

```

// property functions
vector<result_type> intervals() const;
vector<result_type> densities() const;
param_type param() const;
void param(const param_type& parm);
result_type min() const;
result_type max() const;

// inserters and extractors
template<class charT, class traits>
    friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const piecewise_linear_distribution& x);
template<class charT, class traits>
    friend basic_istream<charT, traits>&
        operator>>(basic_istream<charT, traits>& is, piecewise_linear_distribution& x);
};
}

```

```
piecewise_linear_distribution();
```

- 3 *Effects:* Constructs a `piecewise_linear_distribution` object with $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$.

```

template<class InputIteratorB, class InputIteratorW>
    piecewise_linear_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                InputIteratorW firstW);

```

- 4 *Mandates:* Both of

(4.1) — `is_convertible_v<iterator_traits<InputIteratorB>::value_type, double>`

(4.2) — `is_convertible_v<iterator_traits<InputIteratorW>::value_type, double>`

are true.

- 5 *Preconditions:* `InputIteratorB` and `InputIteratorW` each meet the *Cpp17InputIterator* requirements (24.3.5.3). If `firstB == lastB` or `++firstB == lastB`, let $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, $[firstB, lastB)$ forms a sequence b of length $n + 1$, the length of the sequence w starting from `firstW` is at least $n + 1$, and any w_k for $k \geq n + 1$ are ignored by the distribution.

- 6 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters as specified above.

```

template<class UnaryOperation>
    piecewise_linear_distribution(initializer_list<RealType> bl, UnaryOperation fw);

```

- 7 *Mandates:* `is_invocable_r_v<double, UnaryOperation&, double>` is true.

- 8 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters taken or calculated from the following values: If `bl.size() < 2`, let $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, let $[bl.begin(), bl.end())$ form a sequence $b_0, \dots, b_n$, and let $w_k = fw(b_k)$ for $k = 0, \dots, n$.

- 9 *Complexity:* The number of invocations of `fw` does not exceed $n + 1$.

```

template<class UnaryOperation>
    piecewise_linear_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);

```

- 10 *Mandates:* `is_invocable_r_v<double, UnaryOperation&, double>` is true.

- 11 *Preconditions:* If `nw = 0`, let $n = 1$, otherwise let $n = nw$. The relation $0 < \delta = (xmax - xmin)/n$ holds.

- 12 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters taken or calculated from the following values: Let $b_k = xmin + k \cdot \delta$ for $k = 0, \dots, n$, and $w_k = fw(b_k)$ for $k = 0, \dots, n$.

- 13 *Complexity:* The number of invocations of `fw` does not exceed $n + 1$.

```
vector<result_type> intervals() const;
```

- 14 *Returns:* A `vector<result_type>` whose `size` member returns $n + 1$ and whose `operator[]` member returns b_k when invoked with argument k for $k = 0, \dots, n$.


```
vector<result_type> densities() const;
```

- 15 *Returns:* A `vector<result_type>` whose `size` member returns n and whose `operator[]` member returns ρ_k when invoked with argument k for $k = 0, \dots, n$.

29.5.10 Low-quality random number generation

[c.math.rand]

- 1 [Note 1: The header `<cstdlib>` (17.2.2) declares the functions described in this subclause. — end note]

```
int rand();
void srand(unsigned int seed);
```

- 2 *Effects:* The `rand` and `srand` functions have the semantics specified in the C standard library.

- 3 *Remarks:* The implementation may specify that particular library functions may call `rand`. It is implementation-defined whether the `rand` function may introduce data races (16.4.6.10).

[Note 2: The other random number generation facilities in this document (29.5) are often preferable to `rand`, because `rand`'s underlying algorithm is unspecified. Use of `rand` therefore continues to be non-portable, with unpredictable and oft-questionable quality and performance. — end note]

SEE ALSO: ISO/IEC 9899:2024, 7.24.3

29.6 Numeric arrays

[numarray]

29.6.1 Header `<valarray>` synopsis

[valarray.syn]

```
#include <initializer_list> // see 17.11.2

namespace std {
    template<class T> class valarray; // An array of type T
    class slice; // a BLAS-like slice out of an array
    template<class T> class slice_array;
    class gslice; // a generalized slice out of an array
    template<class T> class gslice_array;
    template<class T> class mask_array; // a masked array
    template<class T> class indirect_array; // an indirected array

    template<class T> void swap(valarray<T>&, valarray<T>&) noexcept;

    template<class T> valarray<T> operator* (const valarray<T>&, const valarray<T>&);
    template<class T> valarray<T> operator* (const valarray<T>&,
        const typename valarray<T>::value_type&);
    template<class T> valarray<T> operator* (const typename valarray<T>::value_type&,
        const valarray<T>&);

    template<class T> valarray<T> operator/ (const valarray<T>&, const valarray<T>&);
    template<class T> valarray<T> operator/ (const valarray<T>&,
        const typename valarray<T>::value_type&);
    template<class T> valarray<T> operator/ (const typename valarray<T>::value_type&,
        const valarray<T>&);

    template<class T> valarray<T> operator% (const valarray<T>&, const valarray<T>&);
    template<class T> valarray<T> operator% (const valarray<T>&,
        const typename valarray<T>::value_type&);
    template<class T> valarray<T> operator% (const typename valarray<T>::value_type&,
        const valarray<T>&);

    template<class T> valarray<T> operator+ (const valarray<T>&, const valarray<T>&);
    template<class T> valarray<T> operator+ (const valarray<T>&,
        const typename valarray<T>::value_type&);
    template<class T> valarray<T> operator+ (const typename valarray<T>::value_type&,
        const valarray<T>&);

    template<class T> valarray<T> operator- (const valarray<T>&, const valarray<T>&);
    template<class T> valarray<T> operator- (const valarray<T>&,
        const typename valarray<T>::value_type&);
    template<class T> valarray<T> operator- (const typename valarray<T>::value_type&,
        const valarray<T>&);
```

[illegible]

```

template<class T> valarray<bool> operator<=(const valarray<T>&,
                                           const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator<=(const typename valarray<T>::value_type&,
                                           const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&,
                                           const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator>=(const typename valarray<T>::value_type&,
                                           const valarray<T>&);

template<class T> valarray<T> abs (const valarray<T>&);
template<class T> valarray<T> acos (const valarray<T>&);
template<class T> valarray<T> asin (const valarray<T>&);
template<class T> valarray<T> atan (const valarray<T>&);

template<class T> valarray<T> atan2(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&,
                                     const typename valarray<T>::value_type&);
template<class T> valarray<T> atan2(const typename valarray<T>::value_type&,
                                     const valarray<T>&);

template<class T> valarray<T> cos (const valarray<T>&);
template<class T> valarray<T> cosh (const valarray<T>&);
template<class T> valarray<T> exp (const valarray<T>&);
template<class T> valarray<T> log (const valarray<T>&);
template<class T> valarray<T> log10(const valarray<T>&);

template<class T> valarray<T> pow(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> pow(const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> pow(const typename valarray<T>::value_type&, const valarray<T>&);

template<class T> valarray<T> sin (const valarray<T>&);
template<class T> valarray<T> sinh (const valarray<T>&);
template<class T> valarray<T> sqrt (const valarray<T>&);
template<class T> valarray<T> tan (const valarray<T>&);
template<class T> valarray<T> tanh (const valarray<T>&);
}

```

- ¹ The header `<valarray>` defines five class templates (`valarray`, `slice_array`, `gslice_array`, `mask_array`, and `indirect_array`), two classes (`slice` and `gslice`), and a series of related function templates for representing and manipulating arrays of values.
- ² The `valarray` array classes are defined to be free of certain forms of aliasing, thus allowing operations on these classes to be optimized.
- ³ Any function returning a `valarray<T>` is permitted to return an object of another type, provided all the `const` member functions of `valarray<T>` other than `begin` and `end` are also applicable to this type. This return type shall not add more than two levels of template nesting over the most deeply nested argument type.²²⁶
- ⁴ Implementations introducing such replacement types shall provide additional functions and operators as follows:
 - (4.1) — for every function taking a `const valarray<T>&`, identical functions taking the replacement types shall be added;
 - (4.2) — for every function taking two `const valarray<T>&` arguments, identical functions taking every combination of `const valarray<T>&` and replacement types shall be added.
- ⁵ In particular, an implementation shall allow a `valarray<T>` to be constructed from such replacement types and shall allow assignments and compound assignments of such types to `valarray<T>`, `slice_array<T>`, `gslice_array<T>`, `mask_array<T>` and `indirect_array<T>` objects.

²²⁶) Annex B recommends a minimum number of recursively nested template instantiations. This requirement thus indirectly suggests a minimum allowable complexity for `valarray` expressions.

- ⁶ These library functions are permitted to throw a `bad_alloc` (17.6.4.1) exception if there are not sufficient resources available to carry out the operation. Note that the exception is not mandated.

29.6.2 Class template `valarray`

[template.valarray]

29.6.2.1 Overview

[template.valarray.overview]

```
namespace std {
    template<class T> class valarray {
    public:
        using value_type      = T;
        using iterator        = unspecified;
        using const_iterator  = unspecified;

        // 29.6.2.2, construct/destroy
        valarray();
        explicit valarray(size_t);
        valarray(const T&, size_t);
        valarray(const T*, size_t);
        valarray(const valarray&);
        valarray(valarray&&) noexcept;
        valarray(const slice_array<T>&);
        valarray(const gslice_array<T>&);
        valarray(const mask_array<T>&);
        valarray(const indirect_array<T>&);
        valarray(initializer_list<T>);
        ~valarray();

        // 29.6.2.3, assignment
        valarray& operator=(const valarray&);
        valarray& operator=(valarray&&) noexcept;
        valarray& operator=(initializer_list<T>);
        valarray& operator=(const T&);
        valarray& operator=(const slice_array<T>&);
        valarray& operator=(const gslice_array<T>&);
        valarray& operator=(const mask_array<T>&);
        valarray& operator=(const indirect_array<T>&);

        // 29.6.2.4, element access
        const T&      operator[] (size_t) const;
        T&            operator[] (size_t);

        // 29.6.2.5, subset operations
        valarray      operator[] (slice) const;
        slice_array<T> operator[] (slice);
        valarray      operator[] (const gslice&) const;
        gslice_array<T> operator[] (const gslice&);
        valarray      operator[] (const valarray<bool>&) const;
        mask_array<T>  operator[] (const valarray<bool>&);
        valarray      operator[] (const valarray<size_t>&) const;
        indirect_array<T> operator[] (const valarray<size_t>&);

        // 29.6.2.6, unary operators
        valarray operator+() const;
        valarray operator-() const;
        valarray operator~() const;
        valarray<bool> operator!() const;

        // 29.6.2.7, compound assignment
        valarray& operator*= (const T&);
        valarray& operator/= (const T&);
        valarray& operator%= (const T&);
        valarray& operator+= (const T&);
        valarray& operator-= (const T&);
        valarray& operator^= (const T&);
```

```

valarray& operator&= (const T&);
valarray& operator|= (const T&);
valarray& operator<=<= (const T&);
valarray& operator>=>= (const T&);

valarray& operator*= (const valarray&);
valarray& operator/= (const valarray&);
valarray& operator%= (const valarray&);
valarray& operator+= (const valarray&);
valarray& operator-= (const valarray&);
valarray& operator^= (const valarray&);
valarray& operator|= (const valarray&);
valarray& operator&= (const valarray&);
valarray& operator<=<= (const valarray&);
valarray& operator>=>= (const valarray&);

// 29.6.2.8, range access
iterator begin();
iterator end();
const_iterator begin() const;
const_iterator end() const;

// 29.6.2.9, member functions
void swap(valarray&) noexcept;

size_t size() const;

T sum() const;
T min() const;
T max() const;

valarray shift (int) const;
valarray cshift(int) const;
valarray apply(T func(T)) const;
valarray apply(T func(const T&)) const;
void resize(size_t sz, T c = T());
};

template<class T, size_t cnt> valarray(const T(&)[cnt], size_t) -> valarray<T>;
}

```

- ¹ The class template `valarray<T>` is a one-dimensional smart array, with elements numbered sequentially from zero. It is a representation of the mathematical concept of an ordered set of values. For convenience, an object of type `valarray<T>` is referred to as an “array” throughout the remainder of 29.6. The illusion of higher dimensionality may be produced by the familiar idiom of computed indices, together with the powerful subsetting capabilities provided by the generalized subscript operators.²²⁷

29.6.2.2 Constructors

[`valarray.cons`]

```
valarray();
```

- ¹ *Effects:* Constructs a `valarray` that has zero length.²²⁸

```
explicit valarray(size_t n);
```

- ² *Effects:* Constructs a `valarray` that has length `n`. Each element of the array is value-initialized (9.5).

```
valarray(const T& v, size_t n);
```

- ³ *Effects:* Constructs a `valarray` that has length `n`. Each element of the array is initialized with `v`.

²²⁷) The intent is to specify an array template that has the minimum functionality necessary to address aliasing ambiguities and the proliferation of temporary objects. Thus, the `valarray` template is neither a matrix class nor a field class. However, it is a very useful building block for designing such classes.

²²⁸) This default constructor is essential, since arrays of `valarray` can be useful. After initialization, the length of an empty array can be increased with the `resize` member function.

```
valarray(const T* p, size_t n);
```

4 *Preconditions:* `[p, p + n)` is a valid range.

5 *Effects:* Constructs a `valarray` that has length `n`. The values of the elements of the array are initialized with the first `n` values pointed to by the first argument.²²⁹

```
valarray(const valarray& v);
```

6 *Effects:* Constructs a `valarray` that has the same length as `v`. The elements are initialized with the values of the corresponding elements of `v`.²³⁰

```
valarray(valarray&& v) noexcept;
```

7 *Effects:* Constructs a `valarray` that has the same length as `v`. The elements are initialized with the values of the corresponding elements of `v`.

8 *Complexity:* Constant.

```
valarray(initializer_list<T> il);
```

9 *Effects:* Equivalent to `valarray(il.data(), il.size())`.

```
valarray(const slice_array<T>&);
```

```
valarray(const gslice_array<T>&);
```

```
valarray(const mask_array<T>&);
```

```
valarray(const indirect_array<T>&);
```

10 These conversion constructors convert one of the four reference templates to a `valarray`.

```
~valarray();
```

11 *Effects:* The destructor is applied to every element of `*this`; an implementation may return all allocated memory.

29.6.2.3 Assignment

[`valarray.assign`]

```
valarray& operator=(const valarray& v);
```

1 *Effects:* Each element of the `*this` array is assigned the value of the corresponding element of `v`. If the length of `v` is not equal to the length of `*this`, resizes `*this` to make the two arrays the same length, as if by calling `resize(v.size())`, before performing the assignment.

2 *Postconditions:* `size() == v.size()`.

3 *Returns:* `*this`.

```
valarray& operator=(valarray&& v) noexcept;
```

4 *Effects:* `*this` obtains the value of `v`. The value of `v` after the assignment is not specified.

5 *Returns:* `*this`.

6 *Complexity:* Linear.

```
valarray& operator=(initializer_list<T> il);
```

7 *Effects:* Equivalent to: `return *this = valarray(il);`

```
valarray& operator=(const T& v);
```

8 *Effects:* Assigns `v` to each element of `*this`.

9 *Returns:* `*this`.

```
valarray& operator=(const slice_array<T>&);
```

```
valarray& operator=(const gslice_array<T>&);
```

```
valarray& operator=(const mask_array<T>&);
```

229) This constructor is the preferred method for converting a C array to a `valarray` object.

230) This copy constructor creates a distinct array rather than an alias. Implementations in which arrays share storage are permitted, but they would need to implement a copy-on-reference mechanism to ensure that arrays are conceptually distinct.

```
valarray& operator=(const indirect_array<T>&);
```

10 *Preconditions:* The length of the array to which the argument refers equals `size()`. The value of an element in the left-hand side of a `valarray` assignment operator does not depend on the value of another element in that left-hand side.

11 These operators allow the results of a generalized subscripting operation to be assigned directly to a `valarray`.

29.6.2.4 Element access

[`valarray.access`]

```
const T& operator[](size_t n) const;
```

```
T& operator[](size_t n);
```

1 *Hardened preconditions:* `n < size()` is true.

2 *Returns:* A reference to the corresponding element of the array.

[*Note 1:* The expression `(a[i] = q, a[i]) == q` evaluates to `true` for any non-constant `valarray<T> a`, any `T q`, and for any `size_t i` such that the value of `i` is less than the length of `a`. — end note]

3 *Remarks:* The expression `addressof(a[i + j]) == addressof(a[i]) + j` evaluates to `true` for all `size_t i` and `size_t j` such that `i + j < a.size()`.

4 The expression `addressof(a[i]) != addressof(b[j])` evaluates to `true` for any two arrays `a` and `b` and for any `size_t i` and `size_t j` such that `i < a.size()` and `j < b.size()`.

[*Note 2:* This property indicates an absence of aliasing and can be used to advantage by optimizing compilers. Compilers can take advantage of inlining, constant propagation, loop fusion, tracking of pointers obtained from `operator new`, and other techniques to generate efficient `valarrays`. — end note]

5 The reference returned by the subscript operator for an array shall be valid until the member function `resize(size_t, T)` (29.6.2.9) is called for that array or until the lifetime of that array ends, whichever happens first.

29.6.2.5 Subset operations

[`valarray.sub`]

1 The member `operator[]` is overloaded to provide several ways to select sequences of elements from among those controlled by `*this`. Each of these operations returns a subset of the array. The `const`-qualified versions return this subset as a new `valarray` object. The non-`const` versions return a class template object which has reference semantics to the original array, working in conjunction with various overloads of `operator=` and other assigning operators to allow selective replacement (slicing) of the controlled sequence. In each case the selected element(s) shall exist.

```
valarray operator[](slice slicearr) const;
```

2 *Returns:* A `valarray` containing those elements of the controlled sequence designated by `slicearr`.

[*Example 1:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
// v0[slice(2, 5, 3)] returns valarray<char>("cfilo", 5)
```

— end example]

```
slice_array<T> operator[](slice slicearr);
```

3 *Returns:* An object that holds references to elements of the controlled sequence selected by `slicearr`.

[*Example 2:*

```
valarray<char> v0("abcdefghijklmnop", 16);
valarray<char> v1("ABCDE", 5);
v0[slice(2, 5, 3)] = v1;
// v0 == valarray<char>("abAdeBghCjkDmnEp", 16);
```

— end example]

```
valarray operator[](const gslice& gslicearr) const;
```

4 *Returns:* A `valarray` containing those elements of the controlled sequence designated by `gslicearr`.

[*Example 3:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
const size_t lv[] = { 2, 3 };
```

```

const size_t dv[] = { 7, 2 };
const valarray<size_t> len(lv, 2), str(dv, 2);
// v0[gslice(3, len, str)] returns
// valarray<char>("dfhkmo", 6)
— end example]

```

```
gslice_array<T> operator[](const gslice& gslicearr);
```

5 *Returns:* An object that holds references to elements of the controlled sequence selected by `gslicearr`.

[Example 4:

```

valarray<char> v0("abcdefghijklnop", 16);
valarray<char> v1("ABCDEF", 6);
const size_t lv[] = { 2, 3 };
const size_t dv[] = { 7, 2 };
const valarray<size_t> len(lv, 2), str(dv, 2);
v0[gslice(3, len, str)] = v1;
// v0 == valarray<char>("abcAeBgCijDlEnFp", 16)
— end example]

```

```
valarray operator[](const valarray<bool>& boolarr) const;
```

6 *Returns:* A `valarray` containing those elements of the controlled sequence designated by `boolarr`.

[Example 5:

```

const valarray<char> v0("abcdefghijklnop", 16);
const bool vb[] = { false, false, true, true, false, true };
// v0[valarray<bool>(vb, 6)] returns
// valarray<char>("cdf", 3)
— end example]

```

```
mask_array<T> operator[](const valarray<bool>& boolarr);
```

7 *Returns:* An object that holds references to elements of the controlled sequence selected by `boolarr`.

[Example 6:

```

valarray<char> v0("abcdefghijklnop", 16);
valarray<char> v1("ABC", 3);
const bool vb[] = { false, false, true, true, false, true };
v0[valarray<bool>(vb, 6)] = v1;
// v0 == valarray<char>("abABeCghijklnop", 16)
— end example]

```

```
valarray operator[](const valarray<size_t>& indarr) const;
```

8 *Returns:* A `valarray` containing those elements of the controlled sequence designated by `indarr`.

[Example 7:

```

const valarray<char> v0("abcdefghijklnop", 16);
const size_t vi[] = { 7, 5, 2, 3, 8 };
// v0[valarray<size_t>(vi, 5)] returns
// valarray<char>("hfcdi", 5)
— end example]

```

```
indirect_array<T> operator[](const valarray<size_t>& indarr);
```

9 *Returns:* An object that holds references to elements of the controlled sequence selected by `indarr`.

[Example 8:

```

valarray<char> v0("abcdefghijklnop", 16);
valarray<char> v1("ABCDE", 5);
const size_t vi[] = { 7, 5, 2, 3, 8 };
v0[valarray<size_t>(vi, 5)] = v1;
// v0 == valarray<char>("abCDeBgAEjklmnop", 16)
— end example]

```


29.6.2.6 Unary operators**[valarray.unary]**

```

valarray operator+() const;
valarray operator-() const;
valarray operator~() const;
valarray<bool> operator!() const;

```

- 1 *Mandates:* The indicated operator can be applied to operands of type T and returns a value of type T (bool for operator!) or which may be unambiguously implicitly converted to type T (bool for operator!).
- 2 *Returns:* A valarray whose length is `size()`. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array.

29.6.2.7 Compound assignment**[valarray.cassign]**

```

valarray& operator*= (const valarray& v);
valarray& operator/= (const valarray& v);
valarray& operator%= (const valarray& v);
valarray& operator+= (const valarray& v);
valarray& operator-= (const valarray& v);
valarray& operator^= (const valarray& v);
valarray& operator&= (const valarray& v);
valarray& operator|= (const valarray& v);
valarray& operator<<=(const valarray& v);
valarray& operator>>=(const valarray& v);

```

- 1 *Mandates:* The indicated operator can be applied to two operands of type T.
- 2 *Preconditions:* `size() == v.size()` is true.
- The value of an element in the left-hand side of a valarray compound assignment operator does not depend on the value of another element in that left hand side.
- 3 *Effects:* Each of these operators performs the indicated operation on each of the elements of `*this` and the corresponding element of `v`.
- 4 *Returns:* `*this`.
- 5 *Remarks:* The appearance of an array on the left-hand side of a compound assignment does not invalidate references or pointers.

```

valarray& operator*= (const T& v);
valarray& operator/= (const T& v);
valarray& operator%= (const T& v);
valarray& operator+= (const T& v);
valarray& operator-= (const T& v);
valarray& operator^= (const T& v);
valarray& operator&= (const T& v);
valarray& operator|= (const T& v);
valarray& operator<<=(const T& v);
valarray& operator>>=(const T& v);

```

- 6 *Mandates:* The indicated operator can be applied to two operands of type T.
- 7 *Effects:* Each of these operators applies the indicated operation to each element of `*this` and `v`.
- 8 *Returns:* `*this`.
- 9 *Remarks:* The appearance of an array on the left-hand side of a compound assignment does not invalidate references or pointers to the elements of the array.

29.6.2.8 Range access**[valarray.range]**

- 1 The `iterator` type is a type that meets the requirements of a mutable *Cpp17RandomAccessIterator* (24.3.5.7) and models `contiguous_iterator` (24.3.4.14). Its `value_type` is the template parameter T and its `reference` type is T&. The `const_iterator` type meets the requirements of a constant *Cpp17RandomAccessIterator* and models `contiguous_iterator`. Its `value_type` is the template parameter T and its `reference` type is `const T&`.

- 2 The iterators returned by **begin** and **end** for an array are guaranteed to be valid until the member function **resize(size_t, T)** (29.6.2.9) is called for that array or until the lifetime of that array ends, whichever happens first.

```
iterator begin();
const_iterator begin() const;
```

- 3 *Returns:* An iterator referencing the first value in the array.

```
iterator end();
const_iterator end() const;
```

- 4 *Returns:* An iterator referencing one past the last value in the array.

29.6.2.9 Member functions

[valarray.members]

```
void swap(valarray& v) noexcept;
```

- 1 *Effects:* ***this** obtains the value of **v**. **v** obtains the value of ***this**.

- 2 *Complexity:* Constant.

```
size_t size() const;
```

- 3 *Returns:* The number of elements in the array.

- 4 *Complexity:* Constant time.

```
T sum() const;
```

- 5 *Mandates:* **operator+=** can be applied to operands of type **T**.

- 6 *Preconditions:* **size() > 0** is true.

- 7 *Returns:* The sum of all the elements of the array. If the array has length 1, returns the value of element 0. Otherwise, the returned value is calculated by applying **operator+=** to a copy of an element of the array and all other elements of the array in an unspecified order.

```
T min() const;
```

- 8 *Preconditions:* **size() > 0** is true.

- 9 *Returns:* The minimum value contained in ***this**. For an array of length 1, the value of element 0 is returned. For all other array lengths, the determination is made using **operator<**.

```
T max() const;
```

- 10 *Preconditions:* **size() > 0** is true.

- 11 *Returns:* The maximum value contained in ***this**. For an array of length 1, the value of element 0 is returned. For all other array lengths, the determination is made using **operator<**.

```
valarray shift(int n) const;
```

- 12 *Returns:* A **valarray** of length **size()**, each of whose elements **I** is **(*this)[I + n]** if **I + n** is non-negative and less than **size()**, otherwise **T()**.

[Note 1: If element zero is taken as the leftmost element, a positive value of **n** shifts the elements left **n** places, with zero fill. — end note]

- 13 [Example 1: If the argument has the value **-2**, the first two elements of the result will be value-initialized (9.5); the third element of the result will be assigned the value of the first element of ***this**; etc. — end example]

```
valarray cshift(int n) const;
```

- 14 *Returns:* A **valarray** of length **size()** that is a circular shift of ***this**. If element zero is taken as the leftmost element, a non-negative value of **n** shifts the elements circularly left **n** places and a negative value of **n** shifts the elements circularly right **-n** places.

```
valarray apply(T func(T)) const;
valarray apply(T func(const T&)) const;
```

- 15 *Returns:* A **valarray** whose length is **size()**. Each element of the returned array is assigned the value returned by applying the argument function to the corresponding element of ***this**.

```
void resize(size_t sz, T c = T());
```

- 16 *Effects:* Changes the length of the `*this` array to `sz` and then assigns to each element the value of the second argument. Resizing invalidates all pointers and references to elements in the array.

29.6.3 valarray non-member operations

[valarray.nonmembers]

29.6.3.1 Binary operators

[valarray.binary]

```
template<class T> valarray<T> operator* (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator/ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator<< (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator>> (const valarray<T>&, const valarray<T>&);
```

- 1 *Mandates:* The indicated operator can be applied to operands of type `T` and returns a value of type `T` or which can be unambiguously implicitly converted to `T`.

- 2 *Preconditions:* The argument arrays have the same length.

- 3 *Returns:* A `valarray` whose length is equal to the lengths of the argument arrays. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding elements of the argument arrays.

```
template<class T> valarray<T> operator* (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator* (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator/ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator/ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator% (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator+ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator- (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator^ (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator& (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator| (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator<< (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<T> operator<< (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<T> operator>> (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
```

```
template<class T> valarray<T> operator>>(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
```

4 *Mandates:* The indicated operator can be applied to operands of type T and returns a value of type T or which can be unambiguously implicitly converted to T.

5 *Returns:* A `valarray` whose length is equal to the length of the array argument. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array argument and the non-array argument.

29.6.3.2 Logical operators

[`valarray.comparison`]

```
template<class T> valarray<bool> operator==(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator<=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator|| (const valarray<T>&, const valarray<T>&);
```

1 *Mandates:* The indicated operator can be applied to operands of type T and returns a value of type bool or which can be unambiguously implicitly converted to bool.

2 *Preconditions:* The two array arguments have the same length.

3 *Returns:* A `valarray<bool>` whose length is equal to the length of the array arguments. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding elements of the argument arrays.

```
template<class T> valarray<bool> operator==(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator==(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator!=(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator< (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator> (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator<=(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator<=(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator>=(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator&&(const typename valarray<T>::value_type&,
                                         const valarray<T>&);
template<class T> valarray<bool> operator|| (const valarray<T>&,
                                         const typename valarray<T>::value_type&);
template<class T> valarray<bool> operator|| (const typename valarray<T>::value_type&,
                                         const valarray<T>&);
```

4 *Mandates:* The indicated operator can be applied to operands of type T and returns a value of type bool or which can be unambiguously implicitly converted to bool.

- 5 *Returns:* A `valarray<bool>` whose length is equal to the length of the array argument. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array and the non-array argument.

29.6.3.3 Transcendentals

[`valarray.transcend`]

```
template<class T> valarray<T> abs (const valarray<T>&);
template<class T> valarray<T> acos (const valarray<T>&);
template<class T> valarray<T> asin (const valarray<T>&);
template<class T> valarray<T> atan (const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> atan2(const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<T> cos (const valarray<T>&);
template<class T> valarray<T> cosh (const valarray<T>&);
template<class T> valarray<T> exp (const valarray<T>&);
template<class T> valarray<T> log (const valarray<T>&);
template<class T> valarray<T> log10(const valarray<T>&);
template<class T> valarray<T> pow (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> pow (const valarray<T>&, const typename valarray<T>::value_type&);
template<class T> valarray<T> pow (const typename valarray<T>::value_type&, const valarray<T>&);
template<class T> valarray<T> sin (const valarray<T>&);
template<class T> valarray<T> sinh (const valarray<T>&);
template<class T> valarray<T> sqrt (const valarray<T>&);
template<class T> valarray<T> tan (const valarray<T>&);
template<class T> valarray<T> tanh (const valarray<T>&);
```

- 1 *Mandates:* A unique function with the indicated name can be applied (unqualified) to an operand of type T. This function returns a value of type T or which can be unambiguously implicitly converted to type T.

29.6.3.4 Specialized algorithms

[`valarray.special`]

```
template<class T> void swap(valarray<T>& x, valarray<T>& y) noexcept;
```

- 1 *Effects:* Equivalent to `x.swap(y)`.

29.6.4 Class `slice`

[`class.slice`]

29.6.4.1 Overview

[`class.slice.overview`]

```
namespace std {
    class slice {
    public:
        slice();
        slice(size_t, size_t, size_t);
        slice(const slice&);

        size_t start() const;
        size_t size() const;
        size_t stride() const;

        friend bool operator==(const slice& x, const slice& y);
    };
}
```

- 1 The `slice` class represents a BLAS-like slice from an array. Such a slice is specified by a starting index, a length, and a stride.²³¹

29.6.4.2 Constructors

[`cons.slice`]

```
slice();
```

²³¹ BLAS stands for *Basic Linear Algebra Subprograms*. C++ programs can instantiate this class. See, for example, Dongarra, Du Croz, Duff, and Hammerling: *A set of Level 3 Basic Linear Algebra Subprograms*; Technical Report MCS-P1-0888, Argonne National Laboratory (USA), Mathematics and Computer Science Division, August, 1988.

```
slice(size_t start, size_t length, size_t stride);
```

- ¹ The default constructor is equivalent to `slice(0, 0, 0)`. A default constructor is provided only to permit the declaration of arrays of slices. The constructor with arguments for a slice takes a start, length, and stride parameter.

- ² [*Example 1*: `slice(3, 8, 2)` constructs a slice which selects elements 3, 5, 7, ..., 17 from an array. — *end example*]

29.6.4.3 Access functions

[slice.access]

```
size_t start() const;
size_t size() const;
size_t stride() const;
```

- ¹ *Returns*: The start, length, or stride specified by a `slice` object.

- ² *Complexity*: Constant time.

29.6.4.4 Operators

[slice.ops]

```
friend bool operator==(const slice& x, const slice& y);
```

- ¹ *Effects*: Equivalent to:

```
return x.start() == y.start() && x.size() == y.size() && x.stride() == y.stride();
```

29.6.5 Class template `slice_array`

[template.slice.array]

29.6.5.1 Overview

[template.slice.array.overview]

```
namespace std {
    template<class T> class slice_array {
    public:
        using value_type = T;

        void operator= (const valarray<T>&) const;
        void operator*= (const valarray<T>&) const;
        void operator/= (const valarray<T>&) const;
        void operator%= (const valarray<T>&) const;
        void operator+= (const valarray<T>&) const;
        void operator-= (const valarray<T>&) const;
        void operator^= (const valarray<T>&) const;
        void operator&= (const valarray<T>&) const;
        void operator|= (const valarray<T>&) const;
        void operator<<= (const valarray<T>&) const;
        void operator>>= (const valarray<T>&) const;

        slice_array(const slice_array&);
        ~slice_array();
        const slice_array& operator=(const slice_array&) const;
        void operator=(const T&) const;

        slice_array() = delete;
    };
}
```

- ¹ This template is a helper template used by the `slice` subscript operator

```
slice_array<T> valarray<T>::operator[](slice);
```

- ² It has reference semantics to a subset of an array specified by a `slice` object.

[*Example 1*: The expression `a[slice(1, 5, 3)] = b;` has the effect of assigning the elements of `b` to a slice of the elements in `a`. For the slice shown, the elements selected from `a` are 1, 4, ..., 13. — *end example*]

29.6.5.2 Assignment

[slice.arr.assign]

```
void operator=(const valarray<T>&) const;
```

```
const slice_array& operator=(const slice_array&) const;
```

- ¹ These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which the `slice_array` object refers.

29.6.5.3 Compound assignment

[slice.arr.comp.assign]

```
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%= (const valarray<T>&) const;
void operator+= (const valarray<T>&) const;
void operator-= (const valarray<T>&) const;
void operator^= (const valarray<T>&) const;
void operator&= (const valarray<T>&) const;
void operator|= (const valarray<T>&) const;
void operator<<= (const valarray<T>&) const;
void operator>>= (const valarray<T>&) const;
```

- ¹ These compound assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `slice_array` object refers.

29.6.5.4 Fill function

[slice.arr.fill]

```
void operator=(const T&) const;
```

- ¹ This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `slice_array` object refers.

29.6.6 The `gslice` class

[class.gslice]

29.6.6.1 Overview

[class.gslice.overview]

```
namespace std {
    class gslice {
    public:
        gslice();
        gslice(size_t s, const valarray<size_t>& l, const valarray<size_t>& d);

        size_t      start() const;
        valarray<size_t> size() const;
        valarray<size_t> stride() const;
    };
}
```

- ¹ This class represents a generalized slice out of an array. A `gslice` is defined by a starting offset (s), a set of lengths (l_j), and a set of strides (d_j). The number of lengths shall equal the number of strides.
- ² A `gslice` represents a mapping from a set of indices (i_j), equal in number to the number of strides, to a single index k . It is useful for building multidimensional array classes using the `valarray` template, which is one-dimensional. The set of one-dimensional index values specified by a `gslice` are

$$k = s + \sum_j i_j d_j$$

where the multidimensional indices i_j range in value from 0 to $l_{ij} - 1$.

- ³ [Example 1: The `gslice` specification

```
start   = 3
length  = {2, 4, 3}
stride  = {19, 4, 1}
```

yields the sequence of one-dimensional indices

$$k = 3 + (0, 1) \times 19 + (0, 1, 2, 3) \times 4 + (0, 1, 2) \times 1$$

which are ordered as shown in the following table:

```

(i0, i1, i2, k) =
(0, 0, 0, 3),
(0, 0, 1, 4),
(0, 0, 2, 5),
(0, 1, 0, 7),
(0, 1, 1, 8),
(0, 1, 2, 9),
(0, 2, 0, 11),
(0, 2, 1, 12),
(0, 2, 2, 13),
(0, 3, 0, 15),
(0, 3, 1, 16),
(0, 3, 2, 17),
(1, 0, 0, 22),
(1, 0, 1, 23),
...
(1, 3, 2, 36)

```

That is, the highest-ordered index turns fastest. — *end example*]

⁴ It is possible to have degenerate generalized slices in which an address is repeated.

⁵ [*Example 2*: If the stride parameters in the previous example are changed to {1, 1, 1}, the first few elements of the resulting sequence of indices will be

```

(0, 0, 0, 3),
(0, 0, 1, 4),
(0, 0, 2, 5),
(0, 1, 0, 4),
(0, 1, 1, 5),
(0, 1, 2, 6),
...

```

— *end example*]

⁶ If a degenerate slice is used as the argument to the non-`const` version of `operator[]` (`const gslice&`), the behavior is undefined.

29.6.6.2 Constructors

[`gslice.cons`]

```

gslice();
gslice(size_t start, const valarray<size_t>& lengths,
       const valarray<size_t>& strides);

```

¹ The default constructor is equivalent to `gslice(0, valarray<size_t>(), valarray<size_t>())`. The constructor with arguments builds a `gslice` based on a specification of start, lengths, and strides, as explained in the previous subclause.

29.6.6.3 Access functions

[`gslice.access`]

```

size_t      start() const;
valarray<size_t> size() const;
valarray<size_t> stride() const;

```

¹ *Returns*: The representation of the start, lengths, or strides specified for the `gslice`.

² *Complexity*: `start()` is constant time. `size()` and `stride()` are linear in the number of strides.

29.6.7 Class template `gslice_array`

[`template.gslice.array`]

29.6.7.1 Overview

[`template.gslice.array.overview`]

```

namespace std {
    template<class T> class gslice_array {
    public:
        using value_type = T;

```



```

void operator= (const valarray<T>&) const;
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

gslice_array(const gslice_array&);
~gslice_array();
const gslice_array& operator=(const gslice_array&) const;
void operator=(const T&) const;

gslice_array() = delete;
};
}

```

- ¹ This template is a helper template used by the `gslice` subscript operator

```
gslice_array<T> valarray<T>::operator[](const gslice&);
```

- ² It has reference semantics to a subset of an array specified by a `gslice` object. Thus, the expression `a[gslice(1, length, stride)] = b` has the effect of assigning the elements of `b` to a generalized slice of the elements in `a`.

29.6.7.2 Assignment

[gslice.array.assign]

```

void operator=(const valarray<T>&) const;
const gslice_array& operator=(const gslice_array&) const;

```

- ¹ These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which the `gslice_array` refers.

29.6.7.3 Compound assignment

[gslice.array.comp.assign]

```

void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

```

- ¹ These compound assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `gslice_array` object refers.

29.6.7.4 Fill function

[gslice.array.fill]

```
void operator=(const T&) const;
```

- ¹ This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `gslice_array` object refers.

29.6.8 Class template `mask_array`

[template.mask.array]

29.6.8.1 Overview

[template.mask.array.overview]

```

namespace std {
    template<class T> class mask_array {
    public:
        using value_type = T;

```

```

void operator= (const valarray<T>&) const;
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

mask_array(const mask_array&);
~mask_array();
const mask_array& operator=(const mask_array&) const;
void operator=(const T&) const;

mask_array() = delete;
};
}

```

- ¹ This template is a helper template used by the mask subscript operator:

```
mask_array<T> valarray<T>::operator[] (const valarray<bool>&);
```

- ² It has reference semantics to a subset of an array specified by a boolean mask. Thus, the expression `a[mask] = b;` has the effect of assigning the elements of `b` to the masked elements in `a` (those for which the corresponding element in `mask` is true).

29.6.8.2 Assignment

[mask.array.assign]

```

void operator=(const valarray<T>&) const;
const mask_array& operator=(const mask_array&) const;

```

- ¹ These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which the `mask_array` object refers.

29.6.8.3 Compound assignment

[mask.array.comp.assign]

```

void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

```

- ¹ These compound assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `mask_array` object refers.

29.6.8.4 Fill function

[mask.array.fill]

```
void operator=(const T&) const;
```

- ¹ This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `mask_array` object refers.

29.6.9 Class template `indirect_array`

[template.indirect.array]

29.6.9.1 Overview

[template.indirect.array.overview]

```

namespace std {
    template<class T> class indirect_array {
    public:
        using value_type = T;

```

```

void operator= (const valarray<T>&) const;
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

indirect_array(const indirect_array&);
~indirect_array();
const indirect_array& operator=(const indirect_array&) const;
void operator=(const T&) const;

indirect_array() = delete;
};
}

```

- ¹ This template is a helper template used by the indirect subscript operator

```
indirect_array<T> valarray<T>::operator[](const valarray<size_t>&);
```

- ² It has reference semantics to a subset of an array specified by an `indirect_array`. Thus, the expression `a[indirect] = b;` has the effect of assigning the elements of `b` to the elements in `a` whose indices appear in `indirect`.

29.6.9.2 Assignment

[indirect.array.assign]

```

void operator=(const valarray<T>&) const;
const indirect_array& operator=(const indirect_array&) const;

```

- ¹ These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which it refers.
- ² If the `indirect_array` specifies an element in the `valarray<T>` object to which it refers more than once, the behavior is undefined.

- ³ [Example 1:

```

int addr[] = {2, 3, 1, 4, 4};
valarray<size_t> indirect(addr, 5);
valarray<double> a(0., 10), b(1., 5);
a[indirect] = b;

```

results in undefined behavior since element 4 is specified twice in the indirection. — end example]

29.6.9.3 Compound assignment

[indirect.array.comp.assign]

```

void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%=(const valarray<T>&) const;
void operator+=(const valarray<T>&) const;
void operator-=(const valarray<T>&) const;
void operator^=(const valarray<T>&) const;
void operator&=(const valarray<T>&) const;
void operator|=(const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;

```

- ¹ These compound assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `indirect_array` object refers.
- ² If the `indirect_array` specifies an element in the `valarray<T>` object to which it refers more than once, the behavior is undefined.

29.6.9.4 Fill function

[indirect.array.fill]

```
void operator=(const T&) const;
```

- ¹ This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `indirect_array` object refers.

29.7 Mathematical functions for floating-point types

[c.math]

29.7.1 Header <cmath> synopsis

[cmath.syn]

```
#define HUGE_VAL see below
#define HUGE_VALF see below
#define HUGE_VALL see below
#define INFINITY see below
#define NAN see below
#define FP_INFINITE see below
#define FP_NAN see below
#define FP_NORMAL see below
#define FP_SUBNORMAL see below
#define FP_ZERO see below
#define FP_FAST_FMA see below
#define FP_FAST_FMAF see below
#define FP_FAST_FMAL see below
#define FP_ILOGB0 see below
#define FP_ILOGBNAN see below
#define MATH_ERRNO see below
#define MATH_ERREXCEPT see below

#define math_errhandling see below

namespace std {
    using float_t = see below;
    using double_t = see below;

    constexpr floating-point-type acos(floating-point-type x);
    constexpr float acosf(float x);
    constexpr long double acosl(long double x);

    constexpr floating-point-type asin(floating-point-type x);
    constexpr float asinf(float x);
    constexpr long double asinl(long double x);

    constexpr floating-point-type atan(floating-point-type x);
    constexpr float atanf(float x);
    constexpr long double atanl(long double x);

    constexpr floating-point-type atan2(floating-point-type y, floating-point-type x);
    constexpr float atan2f(float y, float x);
    constexpr long double atan2l(long double y, long double x);

    constexpr floating-point-type cos(floating-point-type x);
    constexpr float cosf(float x);
    constexpr long double cosl(long double x);

    constexpr floating-point-type sin(floating-point-type x);
    constexpr float sinf(float x);
    constexpr long double sinl(long double x);

    constexpr floating-point-type tan(floating-point-type x);
    constexpr float tanf(float x);
    constexpr long double tanl(long double x);

    constexpr floating-point-type acosh(floating-point-type x);
    constexpr float acoshf(float x);
    constexpr long double acoshl(long double x);
```

```

constexpr floating-point-type asinh(floating-point-type x);
constexpr float               asinhf(float x);
constexpr long double         asinhl(long double x);

constexpr floating-point-type atanh(floating-point-type x);
constexpr float               atanhf(float x);
constexpr long double         atanh1(long double x);

constexpr floating-point-type cosh(floating-point-type x);
constexpr float               coshf(float x);
constexpr long double         coshl(long double x);

constexpr floating-point-type sinh(floating-point-type x);
constexpr float               sinhf(float x);
constexpr long double         sinhl(long double x);

constexpr floating-point-type tanh(floating-point-type x);
constexpr float               tanhf(float x);
constexpr long double         tanhl(long double x);

constexpr floating-point-type exp(floating-point-type x);
constexpr float               expf(float x);
constexpr long double         expl(long double x);

constexpr floating-point-type exp2(floating-point-type x);
constexpr float               exp2f(float x);
constexpr long double         exp2l(long double x);

constexpr floating-point-type expm1(floating-point-type x);
constexpr float               expm1f(float x);
constexpr long double         expm1l(long double x);

constexpr floating-point-type frexp(floating-point-type value, int* exp);
constexpr float               frexpf(float value, int* exp);
constexpr long double         frexpl(long double value, int* exp);

constexpr int ilogb(floating-point-type x);
constexpr int ilogbf(float x);
constexpr int ilogbl(long double x);

constexpr floating-point-type ldexp(floating-point-type x, int exp);
constexpr float               ldexpf(float x, int exp);
constexpr long double         ldexpl(long double x, int exp);

constexpr floating-point-type log(floating-point-type x);
constexpr float               logf(float x);
constexpr long double         logl(long double x);

constexpr floating-point-type log10(floating-point-type x);
constexpr float               log10f(float x);
constexpr long double         log10l(long double x);

constexpr floating-point-type log1p(floating-point-type x);
constexpr float               log1pf(float x);
constexpr long double         log1pl(long double x);

constexpr floating-point-type log2(floating-point-type x);
constexpr float               log2f(float x);
constexpr long double         log2l(long double x);

constexpr floating-point-type logb(floating-point-type x);
constexpr float               logbf(float x);
constexpr long double         logbl(long double x);

```

```

constexpr floating-point-type modf(floating-point-type value, floating-point-type* iptr);
constexpr float modff(float value, float* iptr);
constexpr long double modfl(long double value, long double* iptr);

constexpr floating-point-type scalbn(floating-point-type x, int n);
constexpr float scalbnf(float x, int n);
constexpr long double scalbnl(long double x, int n);

constexpr floating-point-type scalbln(floating-point-type x, long int n);
constexpr float scalblnf(float x, long int n);
constexpr long double scalblnl(long double x, long int n);

constexpr floating-point-type cbrt(floating-point-type x);
constexpr float cbrtf(float x);
constexpr long double cbrtl(long double x);

// 29.7.2, absolute values
constexpr int abs(int j); // freestanding
constexpr long int abs(long int j); // freestanding
constexpr long long int abs(long long int j); // freestanding
constexpr floating-point-type abs(floating-point-type j); // freestanding-deleted

constexpr floating-point-type fabs(floating-point-type x);
constexpr float fabsf(float x);
constexpr long double fabsl(long double x);

constexpr floating-point-type hypot(floating-point-type x, floating-point-type y);
constexpr float hypotf(float x, float y);
constexpr long double hypotl(long double x, long double y);

// 29.7.3, three-dimensional hypotenuse
constexpr floating-point-type hypot(floating-point-type x, floating-point-type y,
                                   floating-point-type z);

constexpr floating-point-type pow(floating-point-type x, floating-point-type y);
constexpr float powf(float x, float y);
constexpr long double powl(long double x, long double y);

constexpr floating-point-type sqrt(floating-point-type x);
constexpr float sqrtf(float x);
constexpr long double sqrtl(long double x);

constexpr floating-point-type erf(floating-point-type x);
constexpr float erff(float x);
constexpr long double erfl(long double x);

constexpr floating-point-type erfc(floating-point-type x);
constexpr float erfcf(float x);
constexpr long double erfcl(long double x);

constexpr floating-point-type lgamma(floating-point-type x);
constexpr float lgammaf(float x);
constexpr long double lgammal(long double x);

constexpr floating-point-type tgamma(floating-point-type x);
constexpr float tgammaf(float x);
constexpr long double tgammal(long double x);

constexpr floating-point-type ceil(floating-point-type x);
constexpr float ceilf(float x);
constexpr long double ceill(long double x);

constexpr floating-point-type floor(floating-point-type x);
constexpr float floorf(float x);

```

```

constexpr long double      floorl(long double x);

floating-point-type nearbyint(floating-point-type x);
float                    nearbyintf(float x);
long double              nearbyintl(long double x);

floating-point-type rint(floating-point-type x);
float                    rintf(float x);
long double              rintl(long double x);

long int lrint(floating-point-type x);
long int lrintf(float x);
long int lrintl(long double x);

long long int llrint(floating-point-type x);
long long int llrintf(float x);
long long int llrintl(long double x);

constexpr floating-point-type round(floating-point-type x);
constexpr float              roundf(float x);
constexpr long double        roundl(long double x);

constexpr long int lround(floating-point-type x);
constexpr long int lroundf(float x);
constexpr long int lroundl(long double x);

constexpr long long int llround(floating-point-type x);
constexpr long long int llroundf(float x);
constexpr long long int llroundl(long double x);

constexpr floating-point-type trunc(floating-point-type x);
constexpr float              truncf(float x);
constexpr long double        truncf(long double x);

constexpr floating-point-type fmod(floating-point-type x, floating-point-type y);
constexpr float              fmodf(float x, float y);
constexpr long double        fmodl(long double x, long double y);

constexpr floating-point-type remainder(floating-point-type x, floating-point-type y);
constexpr float              remainderf(float x, float y);
constexpr long double        remainderl(long double x, long double y);

constexpr floating-point-type remquo(floating-point-type x, floating-point-type y, int* quo);
constexpr float              remquof(float x, float y, int* quo);
constexpr long double        remquol(long double x, long double y, int* quo);

constexpr floating-point-type copysign(floating-point-type x, floating-point-type y);
constexpr float              copysignf(float x, float y);
constexpr long double        copysignl(long double x, long double y);

double      nan(const char* tagp);
float       nanf(const char* tagp);
long double nanl(const char* tagp);

constexpr floating-point-type nextafter(floating-point-type x, floating-point-type y);
constexpr float              nextafterf(float x, float y);
constexpr long double        nextafterl(long double x, long double y);

constexpr floating-point-type nexttoward(floating-point-type x, long double y);
constexpr float              nexttowardf(float x, long double y);
constexpr long double        nexttowardl(long double x, long double y);

constexpr floating-point-type nextup(floating-point-type x);
constexpr float              nextupf(float x);

```

```

constexpr long double      nextupl(long double x);

constexpr floating-point-type nextdown(floating-point-type x);
constexpr float           nextdownf(float x);
constexpr long double      nextdownl(long double x);

constexpr floating-point-type fdim(floating-point-type x, floating-point-type y);
constexpr float           fdimf(float x, float y);
constexpr long double      fdiml(long double x, long double y);

constexpr floating-point-type fmax(floating-point-type x, floating-point-type y);
constexpr float           fmaxf(float x, float y);
constexpr long double      fmaxl(long double x, long double y);

constexpr floating-point-type fmin(floating-point-type x, floating-point-type y);
constexpr float           fminf(float x, float y);
constexpr long double      fminl(long double x, long double y);

constexpr floating-point-type fmaximum(floating-point-type x, floating-point-type y);
constexpr floating-point-type fmaximum_num(floating-point-type x, floating-point-type y);
constexpr floating-point-type fminimum(floating-point-type x, floating-point-type y);
constexpr floating-point-type fminimum_num(floating-point-type x, floating-point-type y);

constexpr floating-point-type fma(floating-point-type x, floating-point-type y,
                                   floating-point-type z);
constexpr float           fmaf(float x, float y, float z);
constexpr long double      fmal(long double x, long double y, long double z);

// 29.7.4, linear interpolation
constexpr floating-point-type lerp(floating-point-type a, floating-point-type b,
                                   floating-point-type t) noexcept;

// 29.7.5, classification / comparison functions
constexpr int fpclassify(floating-point-type x);
constexpr bool isfinite(floating-point-type x);
constexpr bool isinf(floating-point-type x);
constexpr bool isnan(floating-point-type x);
constexpr bool isnormal(floating-point-type x);
constexpr bool signbit(floating-point-type x);
constexpr bool isgreater(floating-point-type x, floating-point-type y);
constexpr bool isgreaterequal(floating-point-type x, floating-point-type y);
constexpr bool isless(floating-point-type x, floating-point-type y);
constexpr bool islessequal(floating-point-type x, floating-point-type y);
constexpr bool islessgreater(floating-point-type x, floating-point-type y);
constexpr bool isunordered(floating-point-type x, floating-point-type y);

// 29.7.6, mathematical special functions

// 29.7.6.2, associated Laguerre polynomials
floating-point-type assoc_laguerre(unsigned n, unsigned m, floating-point-type x);
float               assoc_laguerref(unsigned n, unsigned m, float x);
long double         assoc_laguerrel(unsigned n, unsigned m, long double x);

// 29.7.6.3, associated Legendre functions
floating-point-type assoc_legendre(unsigned l, unsigned m, floating-point-type x);
float               assoc_legendref(unsigned l, unsigned m, float x);
long double         assoc_legendrel(unsigned l, unsigned m, long double x);

// 29.7.6.4, beta function
floating-point-type beta(floating-point-type x, floating-point-type y);
float               betaf(float x, float y);
long double         betal(long double x, long double y);

```



```

// 29.7.6.5, complete elliptic integral of the first kind
floating-point-type comp_ellint_1(floating-point-type k);
float               comp_ellint_1f(float k);
long double         comp_ellint_1l(long double k);

// 29.7.6.6, complete elliptic integral of the second kind
floating-point-type comp_ellint_2(floating-point-type k);
float               comp_ellint_2f(float k);
long double         comp_ellint_2l(long double k);

// 29.7.6.7, complete elliptic integral of the third kind
floating-point-type comp_ellint_3(floating-point-type k, floating-point-type nu);
float               comp_ellint_3f(float k, float nu);
long double         comp_ellint_3l(long double k, long double nu);

// 29.7.6.8, regular modified cylindrical Bessel functions
floating-point-type cyl_bessel_i(floating-point-type nu, floating-point-type x);
float               cyl_bessel_if(float nu, float x);
long double         cyl_bessel_il(long double nu, long double x);

// 29.7.6.9, cylindrical Bessel functions of the first kind
floating-point-type cyl_bessel_j(floating-point-type nu, floating-point-type x);
float               cyl_bessel_jf(float nu, float x);
long double         cyl_bessel_jl(long double nu, long double x);

// 29.7.6.10, irregular modified cylindrical Bessel functions
floating-point-type cyl_bessel_k(floating-point-type nu, floating-point-type x);
float               cyl_bessel_kf(float nu, float x);
long double         cyl_bessel_kl(long double nu, long double x);

// 29.7.6.11, cylindrical Neumann functions
// cylindrical Bessel functions of the second kind
floating-point-type cyl_neumann(floating-point-type nu, floating-point-type x);
float               cyl_neumannf(float nu, float x);
long double         cyl_neumannl(long double nu, long double x);

// 29.7.6.12, incomplete elliptic integral of the first kind
floating-point-type ellint_1(floating-point-type k, floating-point-type phi);
float               ellint_1f(float k, float phi);
long double         ellint_1l(long double k, long double phi);

// 29.7.6.13, incomplete elliptic integral of the second kind
floating-point-type ellint_2(floating-point-type k, floating-point-type phi);
float               ellint_2f(float k, float phi);
long double         ellint_2l(long double k, long double phi);

// 29.7.6.14, incomplete elliptic integral of the third kind
floating-point-type ellint_3(floating-point-type k, floating-point-type nu,
                             floating-point-type phi);
float               ellint_3f(float k, float nu, float phi);
long double         ellint_3l(long double k, long double nu, long double phi);

// 29.7.6.15, exponential integral
floating-point-type expint(floating-point-type x);
float               expintf(float x);
long double         expintl(long double x);

// 29.7.6.16, Hermite polynomials
floating-point-type hermite(unsigned n, floating-point-type x);
float               hermitef(unsigned n, float x);
long double         hermitel(unsigned n, long double x);

// 29.7.6.17, Laguerre polynomials
floating-point-type laguerre(unsigned n, floating-point-type x);

```

```

float          laguerref(unsigned n, float x);
long double    laguerrel(unsigned n, long double x);

// 29.7.6.18, Legendre polynomials
floating-point-type legendre(unsigned l, floating-point-type x);
float              legendref(unsigned l, float x);
long double        legendrel(unsigned l, long double x);

// 29.7.6.19, Riemann zeta function
floating-point-type riemann_zeta(floating-point-type x);
float              riemann_zetaf(float x);
long double        riemann_zetal(long double x);

// 29.7.6.20, spherical Bessel functions of the first kind
floating-point-type sph_bessel(unsigned n, floating-point-type x);
float              sph_besself(unsigned n, float x);
long double        sph_bessell(unsigned n, long double x);

// 29.7.6.21, spherical associated Legendre functions
floating-point-type sph_legendre(unsigned l, unsigned m, floating-point-type theta);
float              sph_legendref(unsigned l, unsigned m, float theta);
long double        sph_legendrel(unsigned l, unsigned m, long double theta);

// 29.7.6.22, spherical Neumann functions;
// spherical Bessel functions of the second kind
floating-point-type sph_neumann(unsigned n, floating-point-type x);
float              sph_neumannf(unsigned n, float x);
long double        sph_neumannl(unsigned n, long double x);
}

```

- ¹ The contents and meaning of the header `<cmath>` are a subset of the C standard library header `<math.h>` and only the declarations shown in the synopsis above are present, with the addition of a three-dimensional hypotenuse function (29.7.3), a linear interpolation function (29.7.4), and the mathematical special functions described in 29.7.6.

[Note 1: Several functions have additional overloads in this document, but they have the same behavior as in the C standard library (16.2). — end note]

- ² For each function with at least one parameter of type *floating-point-type*, the implementation provides an overload for each cv-unqualified floating-point type (6.9.3) where all uses of *floating-point-type* in the function signature are replaced with that floating-point type.
- ³ For each function with at least one parameter of type *floating-point-type* other than `abs`, the implementation also provides additional overloads sufficient to ensure that, if every argument corresponding to a *floating-point-type* parameter has arithmetic type, then every such argument is effectively cast to the floating-point type with the greatest floating-point conversion rank and greatest floating-point conversion subrank among the types of all such arguments, where arguments of integer type are considered to have the same floating-point conversion rank as `double`. If no such floating-point type with the greatest rank and subrank exists, then overload resolution does not result in a usable candidate (12.2.1) from the overloads provided by the implementation.
- ⁴ An invocation of `nexttoward` is ill-formed if the argument corresponding to the *floating-point-type* parameter has extended floating-point type.

SEE ALSO: ISO/IEC 9899:2024, 7.12

29.7.2 Absolute values

[c.math.abs]

- ¹ [Note 1: The headers `<cstdlib>` (17.2.2) and `<cmath>` (29.7.1) declare the functions described in this subclause. — end note]

```

constexpr int abs(int j);
constexpr long int abs(long int j);
constexpr long long int abs(long long int j);

```

- ² *Effects:* These functions have the semantics specified in the C standard library for the functions `abs`, `labs`, and `llabs`, respectively.

- 3 *Remarks:* If `abs` is called with an argument of type `X` for which `is_unsigned_v<X>` is `true` and if `X` cannot be converted to `int` by integral promotion (7.3.7), the program is ill-formed.

[Note 2: Allowing arguments that can be promoted to `int` provides compatibility with C. — end note]

```
constexpr floating-point-type abs(floating-point-type x);
```

- 4 *Returns:* The absolute value of `x`.

SEE ALSO: ISO/IEC 9899:2024, 7.12.8.3, 7.24.7.1

29.7.3 Three-dimensional hypotenuse [c.math.hypot3]

```
constexpr floating-point-type hypot(floating-point-type x, floating-point-type y,
                                   floating-point-type z);
```

- 1 *Returns:* $\sqrt{x^2 + y^2 + z^2}$.

29.7.4 Linear interpolation [c.math.lerp]

```
constexpr floating-point-type lerp(floating-point-type a, floating-point-type b,
                                   floating-point-type t) noexcept;
```

- 1 *Returns:* $a + t(b - a)$.

- 2 *Remarks:* Let `r` be the value returned. If `isfinite(a) && isfinite(b)`, then:

- (2.1) — If `t == 0`, then `r == a`.
- (2.2) — If `t == 1`, then `r == b`.
- (2.3) — If `t >= 0 && t <= 1`, then `isfinite(r)`.
- (2.4) — If `isfinite(t) && a == b`, then `r == a`.
- (2.5) — If `isfinite(t) || !isnan(t) && b - a != 0`, then `!isnan(r)`.

Let $CMP(x, y)$ be 1 if $x > y$, -1 if $x < y$, and 0 otherwise. For any `t1` and `t2`, the product of $CMP(lerp(a, b, t2), lerp(a, b, t1))$, $CMP(t2, t1)$, and $CMP(b, a)$ is non-negative.

29.7.5 Classification / comparison functions [c.math.fpclass]

- 1 The classification / comparison functions behave the same as the C macros with the corresponding names defined in the C standard library.

SEE ALSO: ISO/IEC 9899:2024, 7.12.4, 7.12.18

29.7.6 Mathematical special functions [sf.cmath]

29.7.6.1 General [sf.cmath.general]

- 1 If any argument value to any of the functions specified in 29.7.6 is a NaN (Not a Number), the function shall return a NaN but it shall not report a domain error. Otherwise, the function shall report a domain error for just those argument values for which:

- (1.1) — the function description's *Returns*: element explicitly specifies a domain and those argument values fall outside the specified domain, or
- (1.2) — the corresponding mathematical function value has a nonzero imaginary component, or
- (1.3) — the corresponding mathematical function is not mathematically defined.²³²

- 2 Unless otherwise specified, each function is defined for all finite values, for negative infinity, and for positive infinity.

29.7.6.2 Associated Laguerre polynomials [sf.cmath.assoc.laguerre]

```
floating-point-type assoc_laguerre(unsigned n, unsigned m, floating-point-type x);
float               assoc_laguerref(unsigned n, unsigned m, float x);
long double         assoc_laguerrel(unsigned n, unsigned m, long double x);
```

- 1 *Effects:* These functions compute the associated Laguerre polynomials of their respective arguments `n`, `m`, and `x`.

²³² A mathematical function is mathematically defined for a given set of argument values (a) if it is explicitly defined for that set of argument values, or (b) if its limiting value exists and does not depend on the direction of approach.

- 2 *Returns:* $L_n^m(x)$, where L_n^m is given by Formula 29.22, L_{n+m} is given by Formula 29.37, n is n , m is m , and x is x .

$$L_n^m(x) = (-1)^m \frac{d^m}{dx^m} L_{n+m}(x), \quad \text{for } x \geq 0 \quad (29.22)$$

- 3 *Remarks:* The effect of calling each of these functions is implementation-defined if $n \geq 128$ or if $m \geq 128$.

29.7.6.3 Associated Legendre functions

[sf.cmath.assoc.legendre]

```
floating-point-type assoc_legendre(unsigned l, unsigned m, floating-point-type x);
float assoc_legendref(unsigned l, unsigned m, float x);
long double assoc_legendrel(unsigned l, unsigned m, long double x);
```

- 1 *Effects:* These functions compute the associated Legendre functions of their respective arguments l , m , and x .

- 2 *Returns:* $P_\ell^m(x)$, where P_ℓ^m is given by Formula 29.23, P_ℓ is given by Formula 29.38, ℓ is l , m is m , and x is x .

$$P_\ell^m(x) = (1 - x^2)^{m/2} \frac{d^m}{dx^m} P_\ell(x), \quad \text{for } |x| \leq 1 \quad (29.23)$$

- 3 *Remarks:* The effect of calling each of these functions is implementation-defined if $l \geq 128$.

29.7.6.4 Beta function

[sf.cmath.beta]

```
floating-point-type beta(floating-point-type x, floating-point-type y);
float betaf(float x, float y);
long double betal(long double x, long double y);
```

- 1 *Effects:* These functions compute the beta function of their respective arguments x and y .

- 2 *Returns:* $B(x, y)$, where B is given by Formula 29.24, x is x and y is y .

$$B(x, y) = \frac{\Gamma(x)\Gamma(y)}{\Gamma(x+y)}, \quad \text{for } x > 0, y > 0 \quad (29.24)$$

29.7.6.5 Complete elliptic integral of the first kind

[sf.cmath.comp.ellint.1]

```
floating-point-type comp_ellint_1(floating-point-type k);
float comp_ellint_1f(float k);
long double comp_ellint_1l(long double k);
```

- 1 *Effects:* These functions compute the complete elliptic integral of the first kind of their respective arguments k .

- 2 *Returns:* $K(k)$, where K is given by Formula 29.25 and k is k .

$$K(k) = F(k, \pi/2), \quad \text{for } |k| \leq 1 \quad (29.25)$$

- 3 See also 29.7.6.12.

29.7.6.6 Complete elliptic integral of the second kind

[sf.cmath.comp.ellint.2]

```
floating-point-type comp_ellint_2(floating-point-type k);
float comp_ellint_2f(float k);
long double comp_ellint_2l(long double k);
```

- 1 *Effects:* These functions compute the complete elliptic integral of the second kind of their respective arguments k .

- 2 *Returns:* $E(k)$, where E is given by Formula 29.26 and k is k .

$$E(k) = E(k, \pi/2), \quad \text{for } |k| \leq 1 \quad (29.26)$$

- 3 See also 29.7.6.13.

29.7.6.7 Complete elliptic integral of the third kind

[sf.cmath.comp.ellint.3]

```
floating-point-type comp_ellint_3(floating-point-type k, floating-point-type nu);
float comp_ellint_3f(float k, float nu);
long double comp_ellint_3l(long double k, long double nu);
```

¹ *Effects:* These functions compute the complete elliptic integral of the third kind of their respective arguments k and ν .

² *Returns:* $\Pi(\nu, k)$, where Π is given by Formula 29.27, k is k , and ν is ν .

$$\Pi(\nu, k) = \Pi(\nu, k, \pi/2), \quad \text{for } |k| \leq 1 \quad (29.27)$$

³ See also 29.7.6.14.

29.7.6.8 Regular modified cylindrical Bessel functions

[sf.cmath.cyl.bessel.i]

```
floating-point-type cyl_bessel_i(floating-point-type nu, floating-point-type x);
float cyl_bessel_if(float nu, float x);
long double cyl_bessel_il(long double nu, long double x);
```

¹ *Effects:* These functions compute the regular modified cylindrical Bessel functions of their respective arguments ν and x .

² *Returns:* $I_\nu(x)$, where I_ν is given by Formula 29.28, ν is ν , and x is x .

$$I_\nu(x) = i^{-\nu} J_\nu(ix) = \sum_{k=0}^{\infty} \frac{(x/2)^{\nu+2k}}{k! \Gamma(\nu+k+1)}, \quad \text{for } x \geq 0 \quad (29.28)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $\nu \geq 128$.

⁴ See also 29.7.6.9.

29.7.6.9 Cylindrical Bessel functions of the first kind

[sf.cmath.cyl.bessel.j]

```
floating-point-type cyl_bessel_j(floating-point-type nu, floating-point-type x);
float cyl_bessel_jf(float nu, float x);
long double cyl_bessel_jl(long double nu, long double x);
```

¹ *Effects:* These functions compute the cylindrical Bessel functions of the first kind of their respective arguments ν and x .

² *Returns:* $J_\nu(x)$, where J_ν is given by Formula 29.29, ν is ν , and x is x .

$$J_\nu(x) = \sum_{k=0}^{\infty} \frac{(-1)^k (x/2)^{\nu+2k}}{k! \Gamma(\nu+k+1)}, \quad \text{for } x \geq 0 \quad (29.29)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $\nu \geq 128$.

29.7.6.10 Irregular modified cylindrical Bessel functions

[sf.cmath.cyl.bessel.k]

```
floating-point-type cyl_bessel_k(floating-point-type nu, floating-point-type x);
float cyl_bessel_kf(float nu, float x);
long double cyl_bessel_kl(long double nu, long double x);
```

¹ *Effects:* These functions compute the irregular modified cylindrical Bessel functions of their respective arguments ν and x .

² *Returns:* $K_\nu(x)$, where K_ν is given by Formula 29.30, ν is ν , and x is x .

$$K_\nu(x) = (\pi/2) i^{\nu+1} (J_\nu(ix) + i N_\nu(ix)) = \begin{cases} \frac{\pi}{2} \frac{I_{-\nu}(x) - I_\nu(x)}{\sin \nu\pi}, & \text{for } x \geq 0 \text{ and non-integral } \nu \\ \frac{\pi}{2} \lim_{\mu \rightarrow \nu} \frac{I_{-\mu}(x) - I_\mu(x)}{\sin \mu\pi}, & \text{for } x \geq 0 \text{ and integral } \nu \end{cases} \quad (29.30)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $\nu \geq 128$.

⁴ See also 29.7.6.8, 29.7.6.9, 29.7.6.11.

29.7.6.11 Cylindrical Neumann functions

[sf.cmath.cyl.neumann]

```
floating-point-type cyl_neumann(floating-point-type nu, floating-point-type x);
float          cyl_neumannf(float nu, float x);
long double    cyl_neumannl(long double nu, long double x);
```

¹ *Effects:* These functions compute the cylindrical Neumann functions, also known as the cylindrical Bessel functions of the second kind, of their respective arguments **nu** and **x**.

² *Returns:* $N_\nu(x)$, where N_ν is given by Formula 29.31, ν is **nu**, and x is **x**.

$$N_\nu(x) = \begin{cases} \frac{J_\nu(x) \cos \nu\pi - J_{-\nu}(x)}{\sin \nu\pi}, & \text{for } x \geq 0 \text{ and non-integral } \nu \\ \lim_{\mu \rightarrow \nu} \frac{J_\mu(x) \cos \mu\pi - J_{-\mu}(x)}{\sin \mu\pi}, & \text{for } x \geq 0 \text{ and integral } \nu \end{cases} \quad (29.31)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if **nu** $\geq$ 128.

⁴ See also 29.7.6.9.

29.7.6.12 Incomplete elliptic integral of the first kind

[sf.cmath.ellint.1]

```
floating-point-type ellint_1(floating-point-type k, floating-point-type phi);
float          ellint_1f(float k, float phi);
long double    ellint_1l(long double k, long double phi);
```

¹ *Effects:* These functions compute the incomplete elliptic integral of the first kind of their respective arguments **k** and **phi** (**phi** measured in radians).

² *Returns:* $F(k, \phi)$, where F is given by Formula 29.32, k is **k**, and ϕ is **phi**.

$$F(k, \phi) = \int_0^\phi \frac{d\theta}{\sqrt{1 - k^2 \sin^2 \theta}}, \quad \text{for } |k| \leq 1 \quad (29.32)$$

29.7.6.13 Incomplete elliptic integral of the second kind

[sf.cmath.ellint.2]

```
floating-point-type ellint_2(floating-point-type k, floating-point-type phi);
float          ellint_2f(float k, float phi);
long double    ellint_2l(long double k, long double phi);
```

¹ *Effects:* These functions compute the incomplete elliptic integral of the second kind of their respective arguments **k** and **phi** (**phi** measured in radians).

² *Returns:* $E(k, \phi)$, where E is given by Formula 29.33, k is **k**, and ϕ is **phi**.

$$E(k, \phi) = \int_0^\phi \sqrt{1 - k^2 \sin^2 \theta} d\theta, \quad \text{for } |k| \leq 1 \quad (29.33)$$

29.7.6.14 Incomplete elliptic integral of the third kind

[sf.cmath.ellint.3]

```
floating-point-type ellint_3(floating-point-type k, floating-point-type nu,
                             floating-point-type phi);
float          ellint_3f(float k, float nu, float phi);
long double    ellint_3l(long double k, long double nu, long double phi);
```

¹ *Effects:* These functions compute the incomplete elliptic integral of the third kind of their respective arguments **k**, **nu**, and **phi** (**phi** measured in radians).

² *Returns:* $\Pi(\nu, k, \phi)$, where Π is given by Formula 29.34, ν is **nu**, k is **k**, and ϕ is **phi**.

$$\Pi(\nu, k, \phi) = \int_0^\phi \frac{d\theta}{(1 - \nu \sin^2 \theta) \sqrt{1 - k^2 \sin^2 \theta}}, \quad \text{for } |k| \leq 1 \quad (29.34)$$

29.7.6.15 Exponential integral**[sf.cmath.expint]**

```
floating-point-type expint(floating-point-type x);
float               expintf(float x);
long double         expintl(long double x);
```

¹ *Effects:* These functions compute the exponential integral of their respective arguments x .

² *Returns:* $Ei(x)$, where Ei is given by Formula 29.35 and x is x .

$$Ei(x) = - \int_{-x}^{\infty} \frac{e^{-t}}{t} dt \quad (29.35)$$

29.7.6.16 Hermite polynomials**[sf.cmath.hermite]**

```
floating-point-type hermite(unsigned n, floating-point-type x);
float               hermitef(unsigned n, float x);
long double         hermitel(unsigned n, long double x);
```

¹ *Effects:* These functions compute the Hermite polynomials of their respective arguments n and x .

² *Returns:* $H_n(x)$, where H_n is given by Formula 29.36, n is n , and x is x .

$$H_n(x) = (-1)^n e^{x^2} \frac{d^n}{dx^n} e^{-x^2} \quad (29.36)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $n \geq 128$.

29.7.6.17 Laguerre polynomials**[sf.cmath.laguerre]**

```
floating-point-type laguerre(unsigned n, floating-point-type x);
float               laguerref(unsigned n, float x);
long double         laguerrel(unsigned n, long double x);
```

¹ *Effects:* These functions compute the Laguerre polynomials of their respective arguments n and x .

² *Returns:* $L_n(x)$, where L_n is given by Formula 29.37, n is n , and x is x .

$$L_n(x) = \frac{e^x}{n!} \frac{d^n}{dx^n} (x^n e^{-x}) , \quad \text{for } x \geq 0 \quad (29.37)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $n \geq 128$.

29.7.6.18 Legendre polynomials**[sf.cmath.legendre]**

```
floating-point-type legendre(unsigned l, floating-point-type x);
float               legendref(unsigned l, float x);
long double         legendrel(unsigned l, long double x);
```

¹ *Effects:* These functions compute the Legendre polynomials of their respective arguments l and x .

² *Returns:* $P_\ell(x)$, where P_ℓ is given by Formula 29.38, l is l , and x is x .

$$P_\ell(x) = \frac{1}{2^\ell \ell!} \frac{d^\ell}{dx^\ell} (x^2 - 1)^\ell , \quad \text{for } |x| \leq 1 \quad (29.38)$$

³ *Remarks:* The effect of calling each of these functions is implementation-defined if $l \geq 128$.

29.7.6.19 Riemann zeta function**[sf.cmath.riemann.zeta]**

```
floating-point-type riemann_zeta(floating-point-type x);
float               riemann_zetaf(float x);
long double         riemann_zetal(long double x);
```

¹ *Effects:* These functions compute the Riemann zeta function of their respective arguments x .

2 *Returns:* $\zeta(x)$, where ζ is given by Formula 29.39 and x is **x**.

$$\zeta(x) = \begin{cases} \sum_{k=1}^{\infty} k^{-x}, & \text{for } x > 1 \\ \frac{1}{1-2^{1-x}} \sum_{k=1}^{\infty} (-1)^{k-1} k^{-x}, & \text{for } 0 \leq x \leq 1 \\ 2^x \pi^{x-1} \sin\left(\frac{\pi x}{2}\right) \Gamma(1-x) \zeta(1-x), & \text{for } x < 0 \end{cases} \quad (29.39)$$

29.7.6.20 Spherical Bessel functions of the first kind

[sf.cmath.sph.bessel]

```
floating-point-type sph_bessel(unsigned n, floating-point-type x);
float sph_besself(unsigned n, float x);
long double sph_bessell(unsigned n, long double x);
```

1 *Effects:* These functions compute the spherical Bessel functions of the first kind of their respective arguments **n** and **x**.

2 *Returns:* $j_n(x)$, where j_n is given by Formula 29.40, n is **n**, and x is **x**.

$$j_n(x) = (\pi/2x)^{1/2} J_{n+1/2}(x), \quad \text{for } x \geq 0 \quad (29.40)$$

3 *Remarks:* The effect of calling each of these functions is implementation-defined if **n** $\geq$ 128.

4 See also 29.7.6.9.

29.7.6.21 Spherical associated Legendre functions

[sf.cmath.sph.legendre]

```
floating-point-type sph_legendre(unsigned l, unsigned m, floating-point-type theta);
float sph_legendref(unsigned l, unsigned m, float theta);
long double sph_legendrel(unsigned l, unsigned m, long double theta);
```

1 *Effects:* These functions compute the spherical associated Legendre functions of their respective arguments **l**, **m**, and **theta** (**theta** measured in radians).

2 *Returns:* $Y_\ell^m(\theta, 0)$, where Y_ℓ^m is given by Formula 29.41, l is **l**, m is **m**, and θ is **theta**.

$$Y_\ell^m(\theta, \phi) = (-1)^m \left[\frac{(2\ell+1)}{4\pi} \frac{(\ell-m)!}{(\ell+m)!} \right]^{1/2} P_\ell^m(\cos \theta) e^{im\phi}, \quad \text{for } |m| \leq \ell \quad (29.41)$$

3 *Remarks:* The effect of calling each of these functions is implementation-defined if **l** $\geq$ 128.

4 See also 29.7.6.3.

29.7.6.22 Spherical Neumann functions

[sf.cmath.sph.neumann]

```
floating-point-type sph_neumann(unsigned n, floating-point-type x);
float sph_neumannf(unsigned n, float x);
long double sph_neumannl(unsigned n, long double x);
```

1 *Effects:* These functions compute the spherical Neumann functions, also known as the spherical Bessel functions of the second kind, of their respective arguments **n** and **x**.

2 *Returns:* $n_n(x)$, where n_n is given by Formula 29.42, n is **n**, and x is **x**.

$$n_n(x) = (\pi/2x)^{1/2} N_{n+1/2}(x), \quad \text{for } x \geq 0 \quad (29.42)$$

3 *Remarks:* The effect of calling each of these functions is implementation-defined if **n** $\geq$ 128.

4 See also 29.7.6.11.

29.8 Numbers

[numbers]

29.8.1 Header <numbers> synopsis

[numbers.syn]

```

namespace std::numbers {
    template<class T> constexpr T e_v          = unspecified;
    template<class T> constexpr T log2e_v     = unspecified;
    template<class T> constexpr T log10e_v    = unspecified;
    template<class T> constexpr T pi_v       = unspecified;
    template<class T> constexpr T inv_pi_v    = unspecified;
    template<class T> constexpr T inv_sqrtpi_v = unspecified;
    template<class T> constexpr T ln2_v      = unspecified;
    template<class T> constexpr T ln10_v     = unspecified;
    template<class T> constexpr T sqrt2_v    = unspecified;
    template<class T> constexpr T sqrt3_v    = unspecified;
    template<class T> constexpr T inv_sqrt3_v = unspecified;
    template<class T> constexpr T egamma_v   = unspecified;
    template<class T> constexpr T phi_v      = unspecified;

    template<floating_point T> constexpr T e_v<T>          = see below;
    template<floating_point T> constexpr T log2e_v<T>      = see below;
    template<floating_point T> constexpr T log10e_v<T>     = see below;
    template<floating_point T> constexpr T pi_v<T>        = see below;
    template<floating_point T> constexpr T inv_pi_v<T>     = see below;
    template<floating_point T> constexpr T inv_sqrtpi_v<T> = see below;
    template<floating_point T> constexpr T ln2_v<T>       = see below;
    template<floating_point T> constexpr T ln10_v<T>      = see below;
    template<floating_point T> constexpr T sqrt2_v<T>     = see below;
    template<floating_point T> constexpr T sqrt3_v<T>     = see below;
    template<floating_point T> constexpr T inv_sqrt3_v<T>  = see below;
    template<floating_point T> constexpr T egamma_v<T>    = see below;
    template<floating_point T> constexpr T phi_v<T>       = see below;

    inline constexpr double e          = e_v<double>;
    inline constexpr double log2e     = log2e_v<double>;
    inline constexpr double log10e    = log10e_v<double>;
    inline constexpr double pi        = pi_v<double>;
    inline constexpr double inv_pi    = inv_pi_v<double>;
    inline constexpr double inv_sqrtpi = inv_sqrtpi_v<double>;
    inline constexpr double ln2       = ln2_v<double>;
    inline constexpr double ln10      = ln10_v<double>;
    inline constexpr double sqrt2     = sqrt2_v<double>;
    inline constexpr double sqrt3     = sqrt3_v<double>;
    inline constexpr double inv_sqrt3 = inv_sqrt3_v<double>;
    inline constexpr double egamma    = egamma_v<double>;
    inline constexpr double phi       = phi_v<double>;
}

```

29.8.2 Mathematical constants

[math.constants]

- ¹ The library-defined partial specializations of mathematical constant variable templates are initialized with the nearest representable values of e , $\log_2 e$, $\log_{10} e$, π , $\frac{1}{\pi}$, $\frac{1}{\sqrt{\pi}}$, $\ln 2$, $\ln 10$, $\sqrt{2}$, $\sqrt{3}$, $\frac{1}{\sqrt{3}}$, the Euler-Mascheroni γ constant, and the golden ratio ϕ constant $\frac{1+\sqrt{5}}{2}$, respectively.
- ² Pursuant to 16.4.5.2.1, a program may partially or explicitly specialize a mathematical constant variable template provided that the specialization depends on a program-defined type.
- ³ A program that instantiates a primary template of a mathematical constant variable template is ill-formed.

29.9 Basic linear algebra algorithms

[linalg]

29.9.1 Overview

[linalg.overview]

- ¹ Subclause 29.9 defines basic linear algebra algorithms. The algorithms that access the elements of arrays view those elements through `mdspan` (23.7.3).

29.9.2 Header <linalg> synopsis

[linalg.syn]

```

namespace std::linalg {
    // 29.9.5.1, storage order tags
    struct column_major_t;
    inline constexpr column_major_t column_major;
    struct row_major_t;
    inline constexpr row_major_t row_major;

    // 29.9.5.2, triangle tags
    struct upper_triangle_t;
    inline constexpr upper_triangle_t upper_triangle;
    struct lower_triangle_t;
    inline constexpr lower_triangle_t lower_triangle;

    // 29.9.5.3, diagonal tags
    struct implicit_unit_diagonal_t;
    inline constexpr implicit_unit_diagonal_t implicit_unit_diagonal;
    struct explicit_diagonal_t;
    inline constexpr explicit_diagonal_t explicit_diagonal;

    // 29.9.6, class template layout_blas_packed
    template<class Triangle, class StorageOrder>
        class layout_blas_packed;

    // 29.9.7, exposition-only helpers

    // 29.9.7.5, linear algebra argument concepts
    template<class T>
        constexpr bool is-mdspan = see below; // exposition only

    template<class T>
        concept in-vector = see below; // exposition only

    template<class T>
        concept out-vector = see below; // exposition only

    template<class T>
        concept inout-vector = see below; // exposition only

    template<class T>
        concept in-matrix = see below; // exposition only

    template<class T>
        concept out-matrix = see below; // exposition only

    template<class T>
        concept inout-matrix = see below; // exposition only

    template<class T>
        concept possibly-packed-out-matrix = see below; // exposition only

    template<class T>
        concept in-object = see below; // exposition only

    template<class T>
        concept out-object = see below; // exposition only

    template<class T>
        concept inout-object = see below; // exposition only

    template<class T>
        concept scalar = see below; // exposition only

    // 29.9.8, scaled in-place transformation

```

```

// 29.9.8.2, class template scaled_accessor
template<class ScalingFactor, class NestedAccessor>
    class scaled_accessor;

// 29.9.8.3, function template scaled
template<class ScalingFactor,
        class ElementType, class Extents, class Layout, class Accessor>
    constexpr auto scaled(ScalingFactor alpha, mdspan<ElementType, Extents, Layout, Accessor> x);

// 29.9.9, conjugated in-place transformation

// 29.9.9.2, class template conjugated_accessor
template<class NestedAccessor>
    class conjugated_accessor;

// 29.9.9.3, function template conjugated
template<class ElementType, class Extents, class Layout, class Accessor>
    constexpr auto conjugated(mdspan<ElementType, Extents, Layout, Accessor> a);

// 29.9.10, transpose in-place transformation

// 29.9.10.3, class template layout_transpose
template<class Layout>
    class layout_transpose;

// 29.9.10.4, function template transposed
template<class ElementType, class Extents, class Layout, class Accessor>
    constexpr auto transposed(mdspan<ElementType, Extents, Layout, Accessor> a);

// 29.9.11, conjugated transpose in-place transformation
template<class ElementType, class Extents, class Layout, class Accessor>
    constexpr auto conjugate_transposed(mdspan<ElementType, Extents, Layout, Accessor> a);

// 29.9.13, BLAS 1 algorithms

// 29.9.13.2, Givens rotations

// 29.9.13.2.1, compute Givens rotation

template<class Real>
    struct setup_givens_rotation_result {
        Real c;
        Real s;
        Real r;
    };

template<class Real>
    struct setup_givens_rotation_result<complex<Real>> {
        Real c;
        complex<Real> s;
        complex<Real> r;
    };

template<class Real>
    setup_givens_rotation_result<Real> setup_givens_rotation(Real a, Real b) noexcept;

template<class Real>
    setup_givens_rotation_result<complex<Real>>
        setup_givens_rotation(complex<Real> a, complex<Real> b) noexcept;

// 29.9.13.2.2, apply computed Givens rotation
template<inout_vector InOutVec1, inout_vector InOutVec2, class Real>
    void apply_givens_rotation(InOutVec1 x, InOutVec2 y, Real c, Real s);

```

```

template<class ExecutionPolicy, inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(ExecutionPolicy&& exec,
                              InOutVec1 x, InOutVec2 y, Real c, Real s);
template<inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(InOutVec1 x, InOutVec2 y, Real c, complex<Real> s);
template<class ExecutionPolicy, inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(ExecutionPolicy&& exec,
                              InOutVec1 x, InOutVec2 y, Real c, complex<Real> s);

// 29.9.13.3, swap elements
template<inout-object InOutObj1, inout-object InOutObj2>
    void swap_elements(InOutObj1 x, InOutObj2 y);
template<class ExecutionPolicy, inout-object InOutObj1, inout-object InOutObj2>
    void swap_elements(ExecutionPolicy&& exec, InOutObj1 x, InOutObj2 y);

// 29.9.13.4, multiply elements by scalar
template<scalar Scalar, inout-object InOutObj>
    void scale(Scalar alpha, InOutObj x);
template<class ExecutionPolicy, scalar Scalar, inout-object InOutObj>
    void scale(ExecutionPolicy&& exec, Scalar alpha, InOutObj x);

// 29.9.13.5, copy elements
template<in-object InObj, out-object OutObj>
    void copy(InObj x, OutObj y);
template<class ExecutionPolicy, in-object InObj, out-object OutObj>
    void copy(ExecutionPolicy&& exec, InObj x, OutObj y);

// 29.9.13.6, add elementwise
template<in-object InObj1, in-object InObj2, out-object OutObj>
    void add(InObj1 x, InObj2 y, OutObj z);
template<class ExecutionPolicy, in-object InObj1, in-object InObj2, out-object OutObj>
    void add(ExecutionPolicy&& exec, InObj1 x, InObj2 y, OutObj z);

// 29.9.13.7, dot product of two vectors
template<in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dot(InVec1 v1, InVec2 v2, Scalar init);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dot(ExecutionPolicy&& exec, InVec1 v1, InVec2 v2, Scalar init);
template<in-vector InVec1, in-vector InVec2>
    auto dot(InVec1 v1, InVec2 v2);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2>
    auto dot(ExecutionPolicy&& exec, InVec1 v1, InVec2 v2);

template<in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dotc(InVec1 v1, InVec2 v2, Scalar init);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dotc(ExecutionPolicy&& exec, InVec1 v1, InVec2 v2, Scalar init);
template<in-vector InVec1, in-vector InVec2>
    auto dotc(InVec1 v1, InVec2 v2);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2>
    auto dotc(ExecutionPolicy&& exec, InVec1 v1, InVec2 v2);

// 29.9.13.8, Euclidean norm of a vector
template<in-vector InVec, scalar Scalar>
    Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, scalar Scalar>
    Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
template<in-vector InVec>
    auto vector_two_norm(InVec v);
template<class ExecutionPolicy, in-vector InVec>
    auto vector_two_norm(ExecutionPolicy&& exec, InVec v);

```

```

// 29.9.13.9, sum of absolute values of vector elements
template<in-vector InVec, scalar Scalar>
    Scalar vector_abs_sum(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, scalar Scalar>
    Scalar vector_abs_sum(ExecutionPolicy&& exec, InVec v, Scalar init);
template<in-vector InVec>
    auto vector_abs_sum(InVec v);
template<class ExecutionPolicy, in-vector InVec>
    auto vector_abs_sum(ExecutionPolicy&& exec, InVec v);

// 29.9.13.10, index of maximum absolute value of vector elements
template<in-vector InVec>
    typename InVec::size_type vector_idx_abs_max(InVec v);
template<class ExecutionPolicy, in-vector InVec>
    typename InVec::size_type vector_idx_abs_max(ExecutionPolicy&& exec, InVec v);

// 29.9.13.11, Frobenius norm of a matrix
template<in-matrix InMat, scalar Scalar>
    Scalar matrix_frob_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
    Scalar matrix_frob_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
template<in-matrix InMat>
    auto matrix_frob_norm(InMat A);
template<class ExecutionPolicy, in-matrix InMat>
    auto matrix_frob_norm(ExecutionPolicy&& exec, InMat A);

// 29.9.13.12, one norm of a matrix
template<in-matrix InMat, scalar Scalar>
    Scalar matrix_one_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
    Scalar matrix_one_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
template<in-matrix InMat>
    auto matrix_one_norm(InMat A);
template<class ExecutionPolicy, in-matrix InMat>
    auto matrix_one_norm(ExecutionPolicy&& exec, InMat A);

// 29.9.13.13, infinity norm of a matrix
template<in-matrix InMat, scalar Scalar>
    Scalar matrix_inf_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
    Scalar matrix_inf_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
template<in-matrix InMat>
    auto matrix_inf_norm(InMat A);
template<class ExecutionPolicy, in-matrix InMat>
    auto matrix_inf_norm(ExecutionPolicy&& exec, InMat A);

// 29.9.14, BLAS 2 algorithms

// 29.9.14.1, general matrix-vector product
template<in-matrix InMat, in-vector InVec, out-vector OutVec>
    void matrix_vector_product(InMat A, InVec x, OutVec y);
template<class ExecutionPolicy, in-matrix InMat, in-vector InVec, out-vector OutVec>
    void matrix_vector_product(ExecutionPolicy&& exec, InMat A, InVec x, OutVec y);
template<in-matrix InMat, in-vector InVec1, in-vector InVec2, out-vector OutVec>
    void matrix_vector_product(InMat A, InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
    in-matrix InMat, in-vector InVec1, in-vector InVec2, out-vector OutVec>
    void matrix_vector_product(ExecutionPolicy&& exec, InMat A, InVec1 x, InVec2 y, OutVec z);

// 29.9.14.2, symmetric matrix-vector product
template<in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
    void symmetric_matrix_vector_product(InMat A, Triangle t, InVec x, OutVec y);

```

```

template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
    void symmetric_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, InVec x, OutVec y);
template<in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2,
        out-vector OutVec>
    void symmetric_matrix_vector_product(InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2,
        out-vector OutVec>
    void symmetric_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);

// 29.9.14.3, Hermitian matrix-vector product
template<in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
    void hermitian_matrix_vector_product(InMat A, Triangle t, InVec x, OutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
    void hermitian_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, InVec x, OutVec y);
template<in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2,
        out-vector OutVec>
    void hermitian_matrix_vector_product(InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2,
        out-vector OutVec>
    void hermitian_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);

// 29.9.14.4, triangular matrix-vector product

// Overwriting triangular matrix-vector product
template<in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
    void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d,
                                         InVec x, OutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
        out-vector OutVec>
    void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d,
                                         InVec x, OutVec y);

// In-place triangular matrix-vector product
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
    void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InOutVec y);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
    void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d, InOutVec y);

// Updating triangular matrix-vector product
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
    void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d,
                                         InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec1, in-vector InVec2, out-vector OutVec>
    void triangular_matrix_vector_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d,
                                         InVec1 x, InVec2 y, OutVec z);

// 29.9.14.5, solve a triangular linear system

```

```

// Solve a triangular linear system, not in place
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec, out-vector OutVec, class BinaryDivideOp>
    void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d,
                                        InVec b, OutVec x, BinaryDivideOp divide);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec, out-vector OutVec, class BinaryDivideOp>
    void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
                                        InMat A, Triangle t, DiagonalStorage d,
                                        InVec b, OutVec x, BinaryDivideOp divide);
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec, out-vector OutVec>
    void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d,
                                        InVec b, OutVec x);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
        in-vector InVec, out-vector OutVec>
    void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
                                        InMat A, Triangle t, DiagonalStorage d,
                                        InVec b, OutVec x);

// Solve a triangular linear system, in place
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-vector InOutVec, class BinaryDivideOp>
    void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d,
                                        InOutVec b, BinaryDivideOp divide);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-vector InOutVec, class BinaryDivideOp>
    void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
                                        InMat A, Triangle t, DiagonalStorage d,
                                        InOutVec b, BinaryDivideOp divide);
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
    void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d, InOutVec b);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
    void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
                                        InMat A, Triangle t, DiagonalStorage d, InOutVec b);

// 29.9.14.6, nonsymmetric rank-1 matrix update

// overwriting nonsymmetric rank-1 matrix update
template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
    void matrix_rank_1_update(InVec1 x, InVec2 y, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, out-matrix OutMat>
    void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x, InVec2 y, OutMat A);

template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
    void matrix_rank_1_update_c(InVec1 x, InVec2 y, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, out-matrix OutMat>
    void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x, InVec2 y, OutMat A);

// updating nonsymmetric rank-1 matrix update
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix OutMat>
    void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
        out-matrix OutMat>
    void matrix_rank_1_update(ExecutionPolicy&& exec,
                              InVec1 x, InVec2 y, InMat E, OutMat A);

template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix OutMat>
    void matrix_rank_1_update_c(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
        out-matrix OutMat>
    void matrix_rank_1_update_c(ExecutionPolicy&& exec,
                              InVec1 x, InVec2 y, InMat E, OutMat A);

```


// 29.9.14.7, symmetric or Hermitian rank-1 matrix update

// overwriting symmetric rank-1 matrix update

```
template<scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, OutMat A, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                        Scalar alpha, InVec x, OutMat A, Triangle t);
```

// overwriting Hermitian rank-1 matrix update

```
template<scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, OutMat A, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                        Scalar alpha, InVec x, OutMat A, Triangle t);
```

// updating symmetric rank-1 matrix update

```
template<scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
        class Triangle>
    void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
                                        Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
```

// updating Hermitian rank-1 matrix update

```
template<scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
        class Triangle>
    void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
        class Triangle>
    void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
                                        Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
```

// 29.9.14.8, symmetric and Hermitian rank-2 matrix updates

// overwriting symmetric rank-2 matrix update

```
template<in-vector InVec1, in-vector InVec2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                        InVec1 x, InVec2 y, OutMat A, Triangle t);
```

// overwriting Hermitian rank-2 matrix update

```
template<in-vector InVec1, in-vector InVec2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                        InVec1 x, InVec2 y, OutMat A, Triangle t);
```

// updating symmetric rank-2 matrix update

```
template<in-vector InVec1, in-vector InVec2,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
```



```

template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
                                   InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);

// updating Hermitian rank-2 matrix update
template<in-vector InVec1, in-vector InVec2,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
        in-matrix InMat,
        possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
                                   InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);

// 29.9.15, BLAS 3 algorithms

// 29.9.15.1, general matrix-matrix product
template<in-matrix InMat1, in-matrix InMat2, out-matrix OutMat>
void matrix_product(InMat1 A, InMat2 B, OutMat C);
template<class ExecutionPolicy, in-matrix InMat1, in-matrix InMat2, out-matrix OutMat>
void matrix_product(ExecutionPolicy&& exec,
                   InMat1 A, InMat2 B, OutMat C);
template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void matrix_product(InMat1 A, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void matrix_product(ExecutionPolicy&& exec,
                   InMat1 A, InMat2 B, InMat3 E, OutMat C);

// 29.9.15.2, symmetric, Hermitian, and triangular matrix-matrix product

template<in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, Triangle t, InMat2 B, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec,
                              InMat1 A, Triangle t, InMat2 B, OutMat C);

template<in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void hermitian_matrix_product(InMat1 A, Triangle t, InMat2 B, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void hermitian_matrix_product(ExecutionPolicy&& exec,
                              InMat1 A, Triangle t, InMat2 B, OutMat C);

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_product(InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_product(ExecutionPolicy&& exec,
                              InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, InMat2 B, Triangle t, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec,
                              InMat1 A, InMat2 B, Triangle t, OutMat C);

```

```

template<in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
    void hermitian_matrix_product(InMat1 A, InMat2 B, Triangle t, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
    void hermitian_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, InMat2 B, Triangle t, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
        out-matrix OutMat>
    void triangular_matrix_product(InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
        out-matrix OutMat>
    void triangular_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, OutMat C);

template<in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
        out-matrix OutMat>
    void symmetric_matrix_product(InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
        out-matrix OutMat>
    void symmetric_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);

template<in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
        out-matrix OutMat>
    void hermitian_matrix_product(InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
        out-matrix OutMat>
    void hermitian_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
    void triangular_matrix_product(InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, InMat3 E,
                                   OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
    void triangular_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, InMat3 E,
                                   OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
        out-matrix OutMat>
    void symmetric_matrix_product(InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
        out-matrix OutMat>
    void symmetric_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
        out-matrix OutMat>
    void hermitian_matrix_product(InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
        out-matrix OutMat>
    void hermitian_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);

```

```

template<in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
        in-matrix InMat3, out-matrix OutMat>
    void triangular_matrix_product(InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, InMat3 E,
                                   OutMat C);

template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
        in-matrix InMat3, out-matrix OutMat>
    void triangular_matrix_product(ExecutionPolicy&& exec,
                                   InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, InMat3 E,
                                   OutMat C);

```

// 29.9.15.3, in-place triangular matrix-matrix product

```

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_left_product(InMat A, Triangle t, DiagonalStorage d, InOutMat C);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_left_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d, InOutMat C);

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_right_product(InMat A, Triangle t, DiagonalStorage d, InOutMat C);
template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_right_product(ExecutionPolicy&& exec,
                                         InMat A, Triangle t, DiagonalStorage d, InOutMat C);

```

// 29.9.15.4, rank-k update of a symmetric or Hermitian matrix

// overwriting rank-k symmetric matrix update

```

template<scalar Scalar, in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_k_update(Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy, scalar Scalar,
        in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
                                         Scalar alpha, InMat A, OutMat C, Triangle t);

```

// overwriting rank-k Hermitian matrix update

```

template<scalar Scalar, in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_k_update(Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec,
                                         Scalar alpha, InMat A, OutMat C, Triangle t);

```

// updating rank-k symmetric matrix update

```

template<scalar Scalar, in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_k_update(Scalar alpha, InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec, Scalar alpha,
                                         InMat1 A, InMat2 E, OutMat C, Triangle t);

```

// updating rank-k Hermitian matrix update

```

template<scalar Scalar, in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_k_update(Scalar alpha, InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        scalar Scalar, in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec, Scalar alpha,
                                         InMat1 A, InMat2 E, OutMat C, Triangle t);

```

```

// 29.9.15.5, rank-2k update of a symmetric or Hermitian matrix

// overwriting rank-2k symmetric matrix update
template<in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                        InMat1 A, InMat2 B, OutMat C, Triangle t);

// overwriting rank-2k Hermitian matrix update
template<in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                        InMat1 A, InMat2 B, OutMat C, Triangle t);

// updating symmetric rank-2k matrix update
template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
        possibly-packed-out-matrix OutMat, class Triangle>
    void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                        InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);

// updating Hermitian rank-2k matrix update
template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);
template<class ExecutionPolicy,
        in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
        possibly-packed-out-matrix OutMat, class Triangle>
    void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
                                        InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);

// 29.9.15.6, solve multiple triangular linear systems

// solve multiple triangular systems on the left, not-in-place
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                             InMat2 B, OutMat X, BinaryDivideOp divide);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                             InMat1 A, Triangle t, DiagonalStorage d,
                                             InMat2 B, OutMat X, BinaryDivideOp divide);
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
    void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                             InMat2 B, OutMat X);
template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
    void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                             InMat1 A, Triangle t, DiagonalStorage d,

```

```

        InMat2 B, OutMat X);

// solve multiple triangular systems on the right, not-in-place
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_right_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                              InMat2 B, OutMat X, BinaryDivideOp divide);

template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                              InMat1 A, Triangle t, DiagonalStorage d,
                                              InMat2 B, OutMat X, BinaryDivideOp divide);

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
    void triangular_matrix_matrix_right_solve(InMat1 A, Triangle t, DiagonalStorage d,
                                              InMat2 B, OutMat X);

template<class ExecutionPolicy,
        in-matrix InMat1, class Triangle, class DiagonalStorage,
        in-matrix InMat2, out-matrix OutMat>
    void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                              InMat1 A, Triangle t, DiagonalStorage d,
                                              InMat2 B, OutMat X);

// solve multiple triangular systems on the left, in-place
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_left_solve(InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B, BinaryDivideOp divide);

template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                              InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B, BinaryDivideOp divide);

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_matrix_left_solve(InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B);

template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
                                              InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B);

// solve multiple triangular systems on the right, in-place
template<in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B, BinaryDivideOp divide);

template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage,
        inout-matrix InOutMat, class BinaryDivideOp>
    void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                              InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B, BinaryDivideOp divide);

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B);

template<class ExecutionPolicy,
        in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
    void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
                                              InMat A, Triangle t, DiagonalStorage d,
                                              InOutMat B);
}

```

29.9.3 General

[linalg.general]

- ¹ For the effects of all functions in 29.9, when the effects are described as “computes $R = E$ ” or “compute $R = E$ ” (for some R and mathematical expression E), the following apply:
 - (1.1) — E has the conventional mathematical meaning as written.
 - (1.2) — The pattern x^T should be read as “the transpose of x .”
 - (1.3) — The pattern x^H should be read as “the conjugate transpose of x .”
 - (1.4) — When R is the same name as a function parameter whose type is a template parameter with `Out` in its name, the intent is that the result of the computation is written to the elements of the function parameter `R`.
- ² Some of the functions and types in 29.9 distinguish between the “rows” and the “columns” of a matrix. For a matrix `A` and a multidimensional index `i`, `j` in `A.extents()`,
 - (2.1) — *row* `i` of `A` is the set of elements `A[i, k1]` for all `k1` such that `i, k1` is in `A.extents()`; and
 - (2.2) — *column* `j` of `A` is the set of elements `A[k0, j]` for all `k0` such that `k0, j` is in `A.extents()`.
- ³ Some of the functions in 29.9 distinguish between the “upper triangle,” “lower triangle,” and “diagonal” of a matrix.
 - (3.1) — The *diagonal* is the set of all elements of `A` accessed by `A[i,i]` for $0 \leq i < \min(\text{A.extent}(0), \text{A.extent}(1))$.
 - (3.2) — The *upper triangle* of a matrix `A` is the set of all elements of `A` accessed by `A[i,j]` with $i \leq j$. It includes the diagonal.
 - (3.3) — The *lower triangle* of `A` is the set of all elements of `A` accessed by `A[i,j]` with $i \geq j$. It includes the diagonal.
- ⁴ For any function `F` that takes a parameter named `t`, `t` applies to accesses done through the parameter preceding `t` in the parameter list of `F`. Let `m` be such an access-modified function parameter. `F` will only access the triangle of `m` specified by `t`. For accesses of diagonal elements `m[i, i]`, `F` will use the value *real-if-needed*(`m[i, i]`) if the name of `F` starts with `hermitian`. For accesses `m[i, j]` outside the triangle specified by `t`, `F` will use the value
 - (4.1) — *conj-if-needed*(`m[j, i]`) if the name of `F` starts with `hermitian`,
 - (4.2) — `m[j, i]` if the name of `F` starts with `symmetric`, or
 - (4.3) — the additive identity if the name of `F` starts with `triangular`.

[Example 1: Small vector product accessing only specified triangle. It would not be a precondition violation for the non-accessed matrix element to be non-zero.

```
template<class Triangle>
void triangular_matrix_vector_2x2_product(
    mdspan<const float, extents<int, 2, 2>> m,
    Triangle t,
    mdspan<const float, extents<int, 2>> x,
    mdspan<float, extents<int, 2>> y) {

    static_assert(is_same_v<Triangle, lower_triangle_t> ||
                  is_same_v<Triangle, upper_triangle_t>);

    if constexpr (is_same_v<Triangle, lower_triangle_t>) {
        y[0] = m[0,0] * x[0];          // + 0 * x[1]
        y[1] = m[1,0] * x[0] + m[1,1] * x[1];
    } else {                          // upper_triangle_t
        y[0] = m[0,0] * x[0] + m[0,1] * x[1];
        y[1] = /* 0 * x[0] + */ m[1,1] * x[1];
    }
}
```

— end example]

- ⁵ For any function `F` that takes a parameter named `d`, `d` applies to accesses done through the previous-of-the-previous parameter of `d` in the parameter list of `F`. Let `m` be such an access-modified function parameter. If `d` specifies that an implicit unit diagonal is to be assumed, then

- (5.1) — **F** will not access the diagonal of **m**; and
- (5.2) — the algorithm will interpret **m** as if it has a unit diagonal, that is, a diagonal each of whose elements behaves as a two-sided multiplicative identity (even if **m**'s value type does not have a two-sided multiplicative identity).

Otherwise, if **d** specifies that an explicit diagonal is to be assumed, then **F** will access the diagonal of **m**.

- 6 Within all the functions in 29.9, any calls to **abs**, **conj**, **imag**, and **real** are unqualified.
- 7 Two **mdspan** objects **x** and **y** *alias* each other, if they have the same extents **e**, and for every pack of integers **i** which is a multidimensional index in **e**, **x**[**i**...] and **y**[**i**...] refer to the same element.
[Note 1: This means that **x** and **y** view the same elements in the same order. — end note]
- 8 Two **mdspan** objects **x** and **y** *overlap* each other, if for some pack of integers **i** that is a multidimensional index in **x**.**extents**(), there exists a pack of integers **j** that is a multidimensional index in **y**.**extents**(), such that **x**[**i**...] and **y**[**j**...] refer to the same element.

[Note 2: Aliasing is a special case of overlapping. If **x** and **y** do not overlap, then they also do not alias each other. — end note]

29.9.4 Requirements

[linalg.reqs]

29.9.4.1 Linear algebra value types

[linalg.reqs.val]

- 1 Throughout 29.9, the following types are *linear algebra value types*:
 - (1.1) — the **value_type** type alias of any input or output **mdspan** parameter(s) of any function in 29.9; and
 - (1.2) — the **Scalar** template parameter (if any) of any function or class in 29.9.
- 2 Linear algebra value types shall model *scalar*.
- 3 A value-initialized object of linear algebra value type shall act as the additive identity.

29.9.4.2 Algorithm and class requirements

[linalg.reqs.alg]

- 1 29.9.4.2 lists common requirements for all algorithms and classes in 29.9.
- 2 All of the following statements presume that the algorithm's asymptotic complexity requirements, if any, are satisfied.
 - (2.1) — The function may make arbitrarily many objects of any linear algebra value type, value-initializing or direct-initializing them with any existing object of that type.
 - (2.2) — The *triangular solve algorithms* in 29.9.14.5, 29.9.15.3, 29.9.15.6, and 29.9.15.7 either have a **BinaryDivideOp** template parameter (see 29.9.12) and a binary function object parameter **divide** of that type, or they have effects equivalent to invoking such an algorithm. Triangular solve algorithms interpret **divide(a, b)** as **a** times the multiplicative inverse of **b**. Each triangular solve algorithm uses a sequence of evaluations of *****, ***=**, **divide**, unary **+**, binary **+**, **+=**, unary **-**, binary **-**, **-=**, and **=** operators that would produce the result specified by the algorithm's *Effects* and *Remarks* when operating on elements of a field with noncommutative multiplication. It is a precondition of the algorithm that any addend, any subtrahend, any partial sum of addends in any order (treating any difference as a sum with the second term negated), any factor, any partial product of factors respecting their order, any numerator (first argument of **divide**), any denominator (second argument of **divide**), and any assignment is a well-formed expression.
 - (2.3) — Each function in 29.9.13, 29.9.14, and 29.9.15 that is not a triangular solve algorithm will use a sequence of evaluations of *****, ***=**, **+**, **+=**, and **=** operators that would produce the result specified by the algorithm's *Effects* and *Remarks* when operating on elements of a semiring with noncommutative multiplication. It is a precondition of the algorithm that any addend, any partial sum of addends in any order, any factor, any partial product of factors respecting their order, and any assignment is a well-formed expression.
 - (2.4) — If the function has an output **mdspan**, then all addends, subtrahends (for the triangular solve algorithms), or results of the **divide** parameter on intermediate terms (if the function takes a **divide** parameter) are assignable and convertible to the output **mdspan**'s **value_type**.
 - (2.5) — The function may reorder addends and partial sums arbitrarily.

[Note 1: Factors in each product are not reordered; multiplication is not necessarily commutative. — end note]

[*Note 2:* The above requirements do not prohibit implementation approaches and optimization techniques which are not user-observable. In particular, if for all input and output arguments the `value_type` is a floating-point type, implementers are free to leverage approximations, use arithmetic operations not explicitly listed above, and compute floating-point sums in any way that improves their accuracy. — *end note*]

³ [*Note 3:* For all functions in 29.9, suppose that all input and output `mdspan` have as `value_type` a floating-point type, and any `Scalar` template argument has a floating-point type. Then, functions can do all of the following:

- (3.1) — compute floating-point sums in any way that improves their accuracy for arbitrary input;
- (3.2) — perform additional arithmetic operations (other than those specified by the function’s wording and 29.9.4.2) in order to improve performance or accuracy; and
- (3.3) — use approximations (that might not be exact even if computing with real numbers), instead of computations that would be exact if it were possible to compute without rounding error;

as long as

- (3.4) — the function satisfies the complexity requirements; and
- (3.5) — the function is logarithmically stable, as defined in Demmel 2007^[15]. Strassen’s algorithm for matrix-matrix multiply is an example of a logarithmically stable algorithm.

— *end note*]

29.9.5 Tag classes

[`linalg.tags`]

29.9.5.1 Storage order tags

[`linalg.tags.order`]

¹ The storage order tags describe the order of elements in an `mdspan` with `layout_blas_packed` (29.9.6) layout.

```
struct column_major_t {
    explicit column_major_t() = default;
};
inline constexpr column_major_t column_major{};
```

```
struct row_major_t {
    explicit row_major_t() = default;
};
inline constexpr row_major_t row_major{};
```

² `column_major_t` indicates a column-major order, and `row_major_t` indicates a row-major order.

29.9.5.2 Triangle tags

[`linalg.tags.triangle`]

```
struct upper_triangle_t {
    explicit upper_triangle_t() = default;
};
inline constexpr upper_triangle_t upper_triangle{};
```

```
struct lower_triangle_t {
    explicit lower_triangle_t() = default;
};
inline constexpr lower_triangle_t lower_triangle{};
```

¹ These tag classes specify whether algorithms and other users of a matrix (represented as an `mdspan`) access the upper triangle (`upper_triangle_t`) or lower triangle (`lower_triangle_t`) of the matrix (see also 29.9.3). This is also subject to the restrictions of `implicit_unit_diagonal_t` if that tag is also used as a function argument; see below.

29.9.5.3 Diagonal tags

[`linalg.tags.diagonal`]

```
struct implicit_unit_diagonal_t {
    explicit implicit_unit_diagonal_t() = default;
};
inline constexpr implicit_unit_diagonal_t implicit_unit_diagonal{};
```

```
struct explicit_diagonal_t {
    explicit explicit_diagonal_t() = default;
};
```



```
inline constexpr explicit_diagonal_t explicit_diagonal{};
```

- 1 These tag classes specify whether algorithms access the matrix's diagonal entries, and if not, then how algorithms interpret the matrix's implicitly represented diagonal values.
- 2 The `implicit_unit_diagonal_t` tag indicates that an implicit unit diagonal is to be assumed (29.9.3).
- 3 The `explicit_diagonal_t` tag indicates that an explicit diagonal is used (29.9.3).

29.9.6 Layouts for packed matrix types

[linalg.layout.packed]

29.9.6.1 Overview

[linalg.layout.packed.overview]

- 1 `layout_blas_packed` is an `mdspan` layout mapping policy that represents a square matrix that stores only the entries in one triangle, in a packed contiguous format. Its `Triangle` template parameter determines whether an `mdspan` with this layout stores the upper or lower triangle of the matrix. Its `StorageOrder` template parameter determines whether the layout packs the matrix's elements in column-major or row-major order.
- 2 A `StorageOrder` of `column_major_t` indicates column-major ordering. This packs matrix elements starting with the leftmost (least column index) column, and proceeding column by column, from the top entry (least row index).
- 3 A `StorageOrder` of `row_major_t` indicates row-major ordering. This packs matrix elements starting with the topmost (least row index) row, and proceeding row by row, from the leftmost (least column index) entry.
- 4 [Note 1: `layout_blas_packed` describes the data layout used by the BLAS' Symmetric Packed (SP), Hermitian Packed (HP), and Triangular Packed (TP) matrix types. — end note]

```
namespace std::linalg {
    template<class Triangle, class StorageOrder>
    class layout_blas_packed {
    public:
        using triangle_type = Triangle;
        using storage_order_type = StorageOrder;

        template<class Extents>
        struct mapping {
        public:
            using extents_type = Extents;
            using index_type = extents_type::index_type;
            using size_type = extents_type::size_type;
            using rank_type = extents_type::rank_type;
            using layout_type = layout_blas_packed;

            // 29.9.6.2, constructors
            constexpr mapping() noexcept = default;
            constexpr mapping(const mapping&) noexcept = default;
            constexpr mapping(const extents_type&) noexcept;
            template<class OtherExtents>
                constexpr explicit(!is_convertible_v<OtherExtents, extents_type>)
                    mapping(const mapping<OtherExtents>& other) noexcept;

            constexpr mapping& operator=(const mapping&) noexcept = default;

            // 29.9.6.3, observers
            constexpr const extents_type& extents() const noexcept { return extents_; }

            constexpr index_type required_span_size() const noexcept;

            template<class Index0, class Index1>
                constexpr index_type operator()(Index0 ind0, Index1 ind1) const noexcept;

            static constexpr bool is_always_unique() noexcept {
                return (extents_type::static_extent(0) != dynamic_extent &&
                    extents_type::static_extent(0) < 2) ||
                    (extents_type::static_extent(1) != dynamic_extent &&
                    extents_type::static_extent(1) < 2);
            }
        };
};
```

```

    }
    static constexpr bool is_always_exhaustive() noexcept { return true; }
    static constexpr bool is_always_strided() noexcept
    { return is_always_unique(); }

    constexpr bool is_unique() const noexcept {
        return extents_.extent(0) < 2;
    }
    constexpr bool is_exhaustive() const noexcept { return true; }
    constexpr bool is_strided() const noexcept {
        return extents_.extent(0) < 2;
    }

    constexpr index_type stride(rank_type) const noexcept;

    template<class OtherExtents>
        friend constexpr bool operator==(const mapping&, const mapping<OtherExtents>&) noexcept;

private:
    extents_type extents_{};    // exposition only
};
};
}

```

5 *Mandates:*

- (5.1) — Triangle is either `upper_triangle_t` or `lower_triangle_t`,
 - (5.2) — StorageOrder is either `column_major_t` or `row_major_t`,
 - (5.3) — Extents is a specialization of `std::extents`,
 - (5.4) — Extents::rank() equals 2,
 - (5.5) — one of
 - `extents_type::static_extent(0) == dynamic_extent`,
 - `extents_type::static_extent(1) == dynamic_extent`, or
 - `extents_type::static_extent(0) == extents_type::static_extent(1)`
 is true, and
 - (5.6) — if `Extents::rank_dynamic() == 0` is true, let N_s be equal to `Extents::static_extent(0)`; then, $N_s \times (N_s + 1)$ is representable as a value of type `index_type`.
- 6 `layout_blas_packed<T, SO>::mapping<E>` is a trivially copyable type that models `regular` for each T, SO, and E.

29.9.6.2 Constructors

[linalg.layout.packed.cons]

```
constexpr mapping(const extents_type& e) noexcept;
```

1 *Preconditions:*

- (1.1) — Let N be equal to `e.extent(0)`. Then, $N \times (N + 1)$ is representable as a value of type `index_type` (6.9.3).
- (1.2) — `e.extent(0)` equals `e.extent(1)`.

2 *Effects:* Direct-non-list-initializes `extents_` with e.

```

template<class OtherExtents>
    explicit(!is_convertible_v<OtherExtents, extents_type>)
        constexpr mapping(const mapping<OtherExtents>& other) noexcept;

```

3 *Constraints:* `is_constructible_v<extents_type, OtherExtents>` is true.

4 *Preconditions:* Let N be `other.extents().extent(0)`. Then, $N \times (N + 1)$ is representable as a value of type `index_type` (6.9.3).

5 *Effects:* Direct-non-list-initializes `extents_` with `other.extents()`.

29.9.6.3 Observers

[linalg.layout.packed.obs]

```
constexpr index_type required_span_size() const noexcept;
```

1 *Returns:* `extents_.extent(0) * (extents_.extent(0) + 1)/2`.

[Note 1: For example, a 5 x 5 packed matrix only stores 15 matrix elements. — end note]

```
template<class Index0, class Index1>
```

```
constexpr index_type operator() (Index0 ind0, Index1 ind1) const noexcept;
```

2 *Constraints:*

(2.1) — `is_convertible_v<Index0, index_type>` is true,

(2.2) — `is_convertible_v<Index1, index_type>` is true,

(2.3) — `is_nothrow_constructible_v<index_type, Index0>` is true, and

(2.4) — `is_nothrow_constructible_v<index_type, Index1>` is true.

3 Let `i` be `extents_type::index-cast(ind0)`, and let `j` be `extents_type::index-cast(ind1)`.

4 *Preconditions:* `i`, `j` is a multidimensional index in `extents_ (23.7.3.1)`.

5 *Returns:* Let `N` be `extents_.extent(0)`. Then

(5.1) — `(*this)(j, i)` if `i > j` is true; otherwise

(5.2) — `i + j * (j + 1)/2` if

`is_same_v<StorageOrder, column_major_t> && is_same_v<Triangle, upper_triangle_t>`
is true or

`is_same_v<StorageOrder, row_major_t> && is_same_v<Triangle, lower_triangle_t>`
is true; otherwise

(5.3) — `j + N * i - i * (i + 1)/2`.

```
constexpr index_type stride(rank_type r) const noexcept;
```

6 *Preconditions:*

(6.1) — `is_strided()` is true, and

(6.2) — `r < extents_type::rank()` is true.

7 *Returns:* 1.

```
template<class OtherExtents>
```

```
friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y) noexcept;
```

8 *Effects:* Equivalent to: `return x.extents() == y.extents();`

29.9.7 Exposition-only helpers

[linalg.helpers]

29.9.7.1 *abs-if-needed*

[linalg.helpers.abs]

1 The name *abs-if-needed* denotes an exposition-only function object. The expression *abs-if-needed*(E) for a subexpression E whose type is T is expression-equivalent to:

(1.1) — E if T is an unsigned integer;

(1.2) — `std::abs(E)` if T is an arithmetic type,

(1.3) — otherwise, `abs(E)`, if that expression is valid, with overload resolution performed in a context that includes the declaration

```
template<class U> U abs(U) = delete;
```

If the function selected by overload resolution does not return the absolute value of its input, the program is ill-formed, no diagnostic required.

29.9.7.2 *conj-if-needed*

[linalg.helpers.conj]

1 The name *conj-if-needed* denotes an exposition-only function object. The expression *conj-if-needed*(E) for a subexpression E whose type is T is expression-equivalent to:

- (1.1) — `conj(E)`, if `T` is not an arithmetic type and the expression `conj(E)` is valid, with overload resolution performed in a context that includes the declaration

```
template<class U> U conj(const U&) = delete;
```

If the function selected by overload resolution does not return the complex conjugate of its input, the program is ill-formed, no diagnostic required;

- (1.2) — otherwise, `E`.

29.9.7.3 *real-if-needed*

[`linalg.helpers.real`]

- ¹ The name *real-if-needed* denotes an exposition-only function object. The expression *real-if-needed*(`E`) for a subexpression `E` whose type is `T` is expression-equivalent to:

- (1.1) — `real(E)`, if `T` is not an arithmetic type and the expression `real(E)` is valid, with overload resolution performed in a context that includes the declaration

```
template<class U> U real(const U&) = delete;
```

If the function selected by overload resolution does not return the real part of its input, the program is ill-formed, no diagnostic required;

- (1.2) — otherwise, `E`.

29.9.7.4 *imag-if-needed*

[`linalg.helpers.imag`]

- ¹ The name *imag-if-needed* denotes an exposition-only function object. The expression *imag-if-needed*(`E`) for a subexpression `E` whose type is `T` is expression-equivalent to:

- (1.1) — `imag(E)`, if `T` is not an arithmetic type and the expression `imag(E)` is valid, with overload resolution performed in a context that includes the declaration

```
template<class U> U imag(const U&) = delete;
```

If the function selected by overload resolution does not return the imaginary part of its input, the program is ill-formed, no diagnostic required;

- (1.2) — otherwise, `((void)E, T{})`.

29.9.7.5 Argument concepts

[`linalg.helpers.concepts`]

- ¹ The exposition-only concepts defined in this section constrain the algorithms in 29.9.

```
template<class T>
constexpr bool is-mdspan = false;
```

```
template<class ElementType, class Extents, class Layout, class Accessor>
constexpr bool is-mdspan<mdspan<ElementType, Extents, Layout, Accessor>> = true;
```

```
template<class T>
concept in-vector =
    is-mdspan<T> && T::rank() == 1;
```

```
template<class T>
concept out-vector =
    is-mdspan<T> && T::rank() == 1 &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();
```

```
template<class T>
concept inout-vector =
    is-mdspan<T> && T::rank() == 1 &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();
```

```
template<class T>
concept in-matrix =
    is-mdspan<T> && T::rank() == 2;
```

```
template<class T>
concept out-matrix =
    is-mdspan<T> && T::rank() == 2 &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();
```

```

template<class T>
concept inout-matrix =
    is-mdspan<T> && T::rank() == 2 &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();

template<class T>
constexpr bool is-layout-blas-packed = false;    // exposition only

template<class Triangle, class StorageOrder>
constexpr bool is-layout-blas-packed<layout_blas_packed<Triangle, StorageOrder>> = true;

template<class T>
concept possibly-packed-out-matrix =
    is-mdspan<T> && T::rank() == 2 &&
    is_assignable_v<typename T::reference, typename T::element_type> &&
    (T::is_always_unique() || is-layout-blas-packed<typename T::layout_type>);

template<class T>
concept in-object =
    is-mdspan<T> && (T::rank() == 1 || T::rank() == 2);

template<class T>
concept out-object =
    is-mdspan<T> && (T::rank() == 1 || T::rank() == 2) &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();

template<class T>
concept inout-object =
    is-mdspan<T> && (T::rank() == 1 || T::rank() == 2) &&
    is_assignable_v<typename T::reference, typename T::element_type> && T::is_always_unique();

template<class T>
concept scalar =
    semiregular<T> && (!is-mdspan<T>) && (!is_execution_policy_v<T>);

```

- ² If a function in 29.9 accesses the elements of a parameter constrained by *in-vector*, *in-matrix*, or *in-object*, those accesses will not modify the elements.
- ³ Unless explicitly permitted, any *inout-vector*, *inout-matrix*, *inout-object*, *out-vector*, *out-matrix*, *out-object*, or *possibly-packed-out-matrix* parameter of a function in 29.9 shall not overlap any other *mdspan* parameter of the function.

29.9.7.6 Mandates

[linalg.helpers.mandates]

- ¹ [Note 1: These exposition-only helper functions use the less constraining input concepts even for the output arguments, because the additional constraint for assignability of elements is not necessary, and they are sometimes used in a context where the third argument is an input type too. — end note]

```

template<class MDS1, class MDS2>
requires(is-mdspan<MDS1> && is-mdspan<MDS2>)
constexpr
bool compatible-static-extents(size_t r1, size_t r2) {    // exposition only
    return MDS1::static_extent(r1) == dynamic_extent ||
        MDS2::static_extent(r2) == dynamic_extent ||
        MDS1::static_extent(r1) == MDS2::static_extent(r2);
}

template<in-vector In1, in-vector In2, in-vector Out>
constexpr bool possibly-addable() {    // exposition only
    return compatible-static-extents<Out, In1>(0, 0) &&
        compatible-static-extents<Out, In2>(0, 0) &&
        compatible-static-extents<In1, In2>(0, 0);
}

```

```

template<in-matrix In1, in-matrix In2, in-matrix Out>
constexpr bool possibly-addable() { // exposition only
    return compatible-static-extents<Out, In1>(0, 0) &&
           compatible-static-extents<Out, In1>(1, 1) &&
           compatible-static-extents<Out, In2>(0, 0) &&
           compatible-static-extents<Out, In2>(1, 1) &&
           compatible-static-extents<In1, In2>(0, 0) &&
           compatible-static-extents<In1, In2>(1, 1);
}

template<in-matrix InMat, in-vector InVec, in-vector OutVec>
constexpr bool possibly-multipliable() { // exposition only
    return compatible-static-extents<OutVec, InMat>(0, 0) &&
           compatible-static-extents<InMat, InVec>(1, 0);
}

template<in-vector InVec, in-matrix InMat, in-vector OutVec>
constexpr bool possibly-multipliable() { // exposition only
    return compatible-static-extents<OutVec, InMat>(0, 1) &&
           compatible-static-extents<InMat, InVec>(0, 0);
}

template<in-matrix InMat1, in-matrix InMat2, in-matrix OutMat>
constexpr bool possibly-multipliable() { // exposition only
    return compatible-static-extents<OutMat, InMat1>(0, 0) &&
           compatible-static-extents<OutMat, InMat2>(1, 1) &&
           compatible-static-extents<InMat1, InMat2>(1, 0);
}

```

29.9.7.7 Preconditions

[linalg.helpers.precond]

- ¹ [Note 1: These exposition-only helper functions use the less constraining input concepts even for the output arguments, because the additional constraint for assignability of elements is not necessary, and they are sometimes used in a context where the third argument is an input type too. — end note]

```

constexpr bool addable() // exposition only
{
    const in-vector auto& in1, const in-vector auto& in2, const in-vector auto& out {
    return out.extent(0) == in1.extent(0) && out.extent(0) == in2.extent(0);
}

constexpr bool addable() // exposition only
{
    const in-matrix auto& in1, const in-matrix auto& in2, const in-matrix auto& out {
    return out.extent(0) == in1.extent(0) && out.extent(1) == in1.extent(1) &&
           out.extent(0) == in2.extent(0) && out.extent(1) == in2.extent(1);
}

constexpr bool multipliable() // exposition only
{
    const in-matrix auto& in_mat, const in-vector auto& in_vec, const in-vector auto& out_vec {
    return out_vec.extent(0) == in_mat.extent(0) && in_mat.extent(1) == in_vec.extent(0);
}

constexpr bool multipliable() // exposition only
{
    const in-vector auto& in_vec, const in-matrix auto& in_mat, const in-vector auto& out_vec {
    return out_vec.extent(0) == in_mat.extent(1) && in_mat.extent(0) == in_vec.extent(0);
}

constexpr bool multipliable() // exposition only
{
    const in-matrix auto& in_mat1, const in-matrix auto& in_mat2, const in-matrix auto& out_mat {
    return out_mat.extent(0) == in_mat1.extent(0) && out_mat.extent(1) == in_mat2.extent(1) &&
           in_mat1.extent(1) == in_mat2.extent(0);
}

```

29.9.8 Scaled in-place transformation

[linalg.scaled]

29.9.8.1 Introduction

[linalg.scaled.intro]

- ¹ The `scaled` function takes a value `alpha` and an `mdspan` `x`, and returns a new read-only `mdspan` that represents the elementwise product of `alpha` with each element of `x`.

[Example 1:

```
using Vec = mdspan<double, dextents<size_t, 1>>;

// z = alpha * x + y
void z_equals_alpha_times_x_plus_y(double alpha, Vec x, Vec y, Vec z) {
    add(scaled(alpha, x), y, z);
}

// z = alpha * x + beta * y
void z_equals_alpha_times_x_plus_beta_times_y(double alpha, Vec x, double beta, Vec y, Vec z) {
    add(scaled(alpha, x), scaled(beta, y), z);
}
```

— end example]

29.9.8.2 Class template `scaled_accessor`

[linalg.scaled.scaledaccessor]

- ¹ The class template `scaled_accessor` is an `mdspan` accessor policy which upon access produces scaled elements. It is part of the implementation of `scaled` (29.9.8.3).

```
namespace std::linalg {
    template<class ScalingFactor, class NestedAccessor>
    class scaled_accessor {
    public:
        using element_type =
            declval<declval<ScalingFactor>() *
                declval<typename NestedAccessor::element_type>()>();
        using reference = remove_const_t<element_type>;
        using data_handle_type = NestedAccessor::data_handle_type;
        using offset_policy = scaled_accessor<ScalingFactor, typename NestedAccessor::offset_policy>;

        constexpr scaled_accessor() = default;
        template<class OtherNestedAccessor>
            explicit(!is_convertible_v<OtherNestedAccessor, NestedAccessor>)
                constexpr scaled_accessor(const scaled_accessor<ScalingFactor,
                    OtherNestedAccessor>& other);
        constexpr scaled_accessor(const ScalingFactor& s, const NestedAccessor& a);

        constexpr reference access(data_handle_type p, size_t i) const;
        constexpr offset_policy::data_handle_type offset(data_handle_type p, size_t i) const;

        constexpr const ScalingFactor& scaling_factor() const noexcept { return scaling-factor; }
        constexpr const NestedAccessor& nested_accessor() const noexcept { return nested-accessor; }

    private:
        ScalingFactor scaling-factor{}; // exposition only
        NestedAccessor nested-accessor{}; // exposition only
    };
}
```

- ² Mandates:

- (2.1) — `element_type` is valid and denotes a type,
- (2.2) — `is_copy_constructible_v<reference>` is true,
- (2.3) — `is_reference_v<element_type>` is false,
- (2.4) — `ScalingFactor` models `semiregular`, and
- (2.5) — `NestedAccessor` meets the accessor policy requirements (23.7.3.5.2).

```

template<class OtherNestedAccessor>
explicit(!is_convertible_v<OtherNestedAccessor, NestedAccessor>)
constexpr scaled_accessor(const scaled_accessor<ScalingFactor, OtherNestedAccessor>& other);
3   Constraints: is_constructible_v<NestedAccessor, const OtherNestedAccessor&> is true.
4   Effects:
(4.1)   — Direct-non-list-initializes scaling-factor with other.scaling_factor(), and
(4.2)   — direct-non-list-initializes nested-accessor with other.nested_accessor().

constexpr scaled_accessor(const ScalingFactor& s, const NestedAccessor& a);
5   Effects:
(5.1)   — Direct-non-list-initializes scaling-factor with s, and
(5.2)   — direct-non-list-initializes nested-accessor with a.

constexpr reference access(data_handle_type p, size_t i) const;
6   Returns:
        scaling_factor() * typename NestedAccessor::element_type(nested_accessor.access(p, i))

constexpr offset_policy::data_handle_type offset(data_handle_type p, size_t i) const;
7   Returns: nested-accessor.offset(p, i).

```

29.9.8.3 Function template scaled

[linalg.scaled.scaled]

- 1 The **scaled** function template takes a scaling factor **alpha** and an **mdspan** **x**, and returns a new read-only **mdspan** with the same domain as **x**, that represents the elementwise product of **alpha** with each element of **x**.

```

template<class ScalingFactor,
        class ElementType, class Extents, class Layout, class Accessor>
constexpr auto scaled(ScalingFactor alpha, mdspan<ElementType, Extents, Layout, Accessor> x);
2   Let SA be scaled_accessor<ScalingFactor, Accessor>.
3   Returns:
        mdspan<typename SA::element_type, Extents, Layout, SA>(x.data_handle(), x.mapping(),
                                                                SA(alpha, x.accessor()))
4   [Example 1:
void test_scaled(mdspan<double, extents<int, 10>> x)
{
    auto x_scaled = scaled(5.0, x);
    for (int i = 0; i < x.extent(0); ++i) {
        assert(x_scaled[i] == 5.0 * x[i]);
    }
}
— end example]

```

29.9.9 Conjugated in-place transformation

[linalg.conj]

29.9.9.1 Introduction

[linalg.conj.intro]

- 1 The **conjugated** function takes an **mdspan** **x**, and returns a new read-only **mdspan** **y** with the same domain as **x**, whose elements are the complex conjugates of the corresponding elements of **x**.

29.9.9.2 Class template conjugated_accessor

[linalg.conj.conjugatedaccessor]

- 1 The class template **conjugated_accessor** is an **mdspan** accessor policy which upon access produces conjugate elements. It is part of the implementation of **conjugated** (29.9.9.3).

```

namespace std::linalg {
    template<class NestedAccessor>
    class conjugated_accessor {
    public:
        using element_type =
            const decltype(conj-if-needed(declval<typename NestedAccessor::element_type>()));

```



```

using reference = remove_const_t<element_type>;
using data_handle_type = NestedAccessor::data_handle_type;
using offset_policy = conjugated_accessor<typename NestedAccessor::offset_policy>;

constexpr conjugated_accessor() = default;
constexpr conjugated_accessor(const NestedAccessor& acc);
template<class OtherNestedAccessor>
    explicit(!is_convertible_v<OtherNestedAccessor, NestedAccessor>)
        constexpr conjugated_accessor(const conjugated_accessor<OtherNestedAccessor>& other);

constexpr reference access(data_handle_type p, size_t i) const;

constexpr typename offset_policy::data_handle_type
    offset(data_handle_type p, size_t i) const;

constexpr const NestedAccessor& nested_accessor() const noexcept { return nested-accessor_; }

private:
    NestedAccessor nested-accessor_{}; // exposition only
};

```

2 *Mandates:*

- (2.1) — `element_type` is valid and denotes a type,
- (2.2) — `is_copy_constructible_v<reference>` is true,
- (2.3) — `is_reference_v<element_type>` is false, and
- (2.4) — `NestedAccessor` meets the accessor policy requirements (23.7.3.5.2).

```
constexpr conjugated_accessor(const NestedAccessor& acc);
```

- 3 *Effects:* Direct-non-list-initializes *nested-accessor_* with `acc`.

```

template<class OtherNestedAccessor>
    explicit(!is_convertible_v<OtherNestedAccessor, NestedAccessor>)
        constexpr conjugated_accessor(const conjugated_accessor<OtherNestedAccessor>& other);

```

- 4 *Constraints:* `is_constructible_v<NestedAccessor, const OtherNestedAccessor&>` is true.

- 5 *Effects:* Direct-non-list-initializes *nested-accessor_* with `other.nested_accessor()`.

```
constexpr reference access(data_handle_type p, size_t i) const;
```

- 6 *Returns:*

conj-if-needed(`typename NestedAccessor::element_type`(*nested-accessor_*.access(p, i)))

```
constexpr typename offset_policy::data_handle_type offset(data_handle_type p, size_t i) const;
```

- 7 *Returns:* *nested-accessor_*.offset(p, i).

29.9.9.3 Function template `conjugated`

[`linalg.conj.conjugated`]

```

template<class ElementType, class Extents, class Layout, class Accessor>
    constexpr auto conjugated(mdspace<ElementType, Extents, Layout, Accessor> a);

```

- 1 Let `A` be

- (1.1) — `remove_cvref_t<decltype(a.accessor().nested_accessor())>` if `Accessor` is a specialization of `conjugated_accessor`;
- (1.2) — otherwise, `Accessor` if `remove_cvref_t<ElementType>` is an arithmetic type;
- (1.3) — otherwise, `conjugated_accessor<Accessor>` if the expression `conj(E)` is valid for any subexpression `E` whose type is `remove_cvref_t<ElementType>` with overload resolution performed in a context that includes the declaration `template<class U> U conj(const U&) = delete;`;
- (1.4) — otherwise, `Accessor`.

- 2 *Returns:* Let MD be `mdspan<typename A::element_type, Extents, Layout, A>`.
- (2.1) — `MD(a.data_handle(), a.mapping(), a.accessor().nested_accessor())` if `Accessor` is a specialization of `conjugated_accessor`;
- (2.2) — otherwise, `a`, if `is_same_v<A, Accessor>` is true;
- (2.3) — otherwise, `MD(a.data_handle(), a.mapping(), conjugated_accessor(a.accessor()))`.

3 [Example 1:

```
void test_conjugated_complex(mdspan<complex<double>, extents<int, 10>> a) {
    auto a_conj = conjugated(a);
    for (int i = 0; i < a.extent(0); ++i) {
        assert(a_conj[i] == conj(a[i]));
    }
    auto a_conj_conj = conjugated(a_conj);
    for (int i = 0; i < a.extent(0); ++i) {
        assert(a_conj_conj[i] == a[i]);
    }
}
```

```
void test_conjugated_real(mdspan<double, extents<int, 10>> a) {
    auto a_conj = conjugated(a);
    for (int i = 0; i < a.extent(0); ++i) {
        assert(a_conj[i] == a[i]);
    }
    auto a_conj_conj = conjugated(a_conj);
    for (int i = 0; i < a.extent(0); ++i) {
        assert(a_conj_conj[i] == a[i]);
    }
}
```

— end example]

29.9.10 Transpose in-place transformation

[linalg.transp]

29.9.10.1 Introduction

[linalg.transp.intro]

- 1 `layout_transpose` is an `mdspan` layout mapping policy that swaps the two indices, extents, and strides of any unique `mdspan` layout mapping policy.
- 2 The `transposed` function takes an `mdspan` representing a matrix, and returns a new `mdspan` representing the transpose of the input matrix.

29.9.10.2 Exposition-only helpers for `layout_transpose` and `transposed` [linalg.transp.helpers]

- 1 The exposition-only `transpose_extents` function takes an `extents` object representing the extents of a matrix, and returns a new `extents` object representing the extents of the transpose of the matrix.
- 2 The exposition-only alias template `transpose_extents_t<InputExtents>` gives the type of `transpose_extents(e)` for a given `extents` object `e` of type `InputExtents`.

```
template<class IndexType, size_t InputExtent0, size_t InputExtent1>
constexpr extents<IndexType, InputExtent1, InputExtent0>
    transpose_extents(const extents<IndexType, InputExtent0, InputExtent1>& in);    // exposition only
```

- 3 *Returns:* `extents<IndexType, InputExtent1, InputExtent0>(in.extent(1), in.extent(0))`.

```
template<class InputExtents>
using transpose_extents_t =
    decltype(transpose_extents(declval<InputExtents>()));    // exposition only
```

29.9.10.3 Class template `layout_transpose`

[linalg.transp.layout.transpose]

- 1 `layout_transpose` is an `mdspan` layout mapping policy that swaps the two indices, extents, and strides of any `mdspan` layout mapping policy.

```
namespace std::linalg {
    template<class Layout>
    class layout_transpose {
    public:
        using nested_layout_type = Layout;
```

```

template<class Extents>
struct mapping {
private:
    using nested-mapping-type =
        Layout::template mapping<transpose-extents-t<Extents>>; // exposition only

public:
    using extents_type = Extents;
    using index_type = extents_type::index_type;
    using size_type = extents_type::size_type;
    using rank_type = extents_type::rank_type;
    using layout_type = layout_transpose;

    constexpr explicit mapping(const nested-mapping-type&);

    constexpr const extents_type& extents() const noexcept { return extents_; }

    constexpr index_type required_span_size() const
        { return nested-mapping_.required_span_size(); }

    template<class Index0, class Index1>
        constexpr index_type operator()(Index0 ind0, Index1 ind1) const
            { return nested-mapping_(ind1, ind0); }

    constexpr const nested-mapping-type& nested_mapping() const noexcept
        { return nested-mapping_; }

    static constexpr bool is_always_unique() noexcept
        { return nested-mapping-type::is_always_unique(); }
    static constexpr bool is_always_exhaustive() noexcept
        { return nested-mapping-type::is_always_exhaustive(); }
    static constexpr bool is_always_strided() noexcept
        { return nested-mapping-type::is_always_strided(); }

    constexpr bool is_unique() const { return nested-mapping_.is_unique(); }
    constexpr bool is_exhaustive() const { return nested-mapping_.is_exhaustive(); }
    constexpr bool is_strided() const { return nested-mapping_.is_strided(); }

    constexpr index_type stride(size_t r) const;

    template<class OtherExtents>
        friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y);

private:
    nested-mapping-type nested-mapping_; // exposition only
    extents_type extents_; // exposition only
};
}

```

² Layout shall meet the layout mapping policy requirements (23.7.3.4.3).

³ Mandates:

(3.1) — Extents is a specialization of `std::extents`, and

(3.2) — `Extents::rank()` equals 2.

```
constexpr explicit mapping(const nested-mapping-type& map);
```

⁴ Effects:

(4.1) — Initializes *nested-mapping_* with map, and

(4.2) — initializes *extents_* with *transpose-extents*(map.extents()).

```
constexpr index_type stride(size_t r) const;
```

5 *Preconditions:*

(5.1) — `is_strided()` is true, and

(5.2) — `r < 2` is true.

6 *Returns:* `nested-mapping_.stride(r == 0 ? 1 : 0)`.

```
template<class OtherExtents>
```

```
friend constexpr bool operator==(const mapping& x, const mapping<OtherExtents>& y);
```

7 *Constraints:* The expression `x.nested-mapping_ == y.nested-mapping_` is well-formed and its result is convertible to bool.

8 *Returns:* `x.nested-mapping_ == y.nested-mapping_`.

29.9.10.4 Function template `transposed`

[linalg.transp.transposed]

1 The `transposed` function takes a rank-2 `mdspan` representing a matrix, and returns a new `mdspan` representing the input matrix's transpose. The input matrix's data are not modified, and the returned `mdspan` accesses the input matrix's data in place.

```
template<class ElementType, class Extents, class Layout, class Accessor>
constexpr auto transposed(mdspan<ElementType, Extents, Layout, Accessor> a);
```

2 *Mandates:* `Extents::rank() == 2` is true.

3 Let `ReturnExtents` be *transpose-extents-t*<`Extents`>. Let `R` be `mdspan<ElementType, ReturnExtents, ReturnLayout, Accessor>`, where `ReturnLayout` is:

(3.1) — `layout_right` if `Layout` is `layout_left`;

(3.2) — otherwise, `layout_left` if `Layout` is `layout_right`;

(3.3) — otherwise, `layout_right_padded<PaddingValue>` if `Layout` is `layout_left_padded<PaddingValue>` for some `size_t` value `PaddingValue`;

(3.4) — otherwise, `layout_left_padded<PaddingValue>` if `Layout` is `layout_right_padded<PaddingValue>` for some `size_t` value `PaddingValue`;

(3.5) — otherwise, `layout_stride` if `Layout` is `layout_stride`;

(3.6) — otherwise, `layout_blas_packed<OppositeTriangle, OppositeStorageOrder>`, if `Layout` is `layout_blas_packed<Triangle, StorageOrder>` for some `Triangle` and `StorageOrder`, where

(3.6.1) — `OppositeTriangle` is
 `conditional_t<is_same_v<Triangle, upper_triangle_t>, lower_triangle_t, upper_triangle_t>`

and

(3.6.2) — `OppositeStorageOrder` is
 `conditional_t<is_same_v<StorageOrder, column_major_t>, row_major_t, column_major_t>`

(3.7) — otherwise, `NestedLayout` if `Layout` is `layout_transpose<NestedLayout>` for some `NestedLayout`;

(3.8) — otherwise, `layout_transpose<Layout>`.

4 *Returns:* With `ReturnMapping` being the type `typename ReturnLayout::template mapping<ReturnExtents>`:

(4.1) — if `Layout` is `layout_left`, `layout_right`, or a specialization of `layout_blas_packed`,
 `R(a.data_handle(), ReturnMapping(transpose-extents(a.mapping().extents()), a.accessor()))`

(4.2) — otherwise,
 `R(a.data_handle(), ReturnMapping(transpose-extents(a.mapping().extents()), a.mapping().stride(1), a.accessor()))`
 if `Layout` is `layout_left_padded<PaddingValue>` for some `size_t` value `PaddingValue`;

(4.3) — otherwise,

```

    R(a.data_handle(), ReturnMapping(transpose-extents(a.mapping().extents()),
    a.mapping().stride(0)), a.accessor())
    if Layout is layout_right_padded<PaddingValue> for some size_t value PaddingValue;
(4.4) — otherwise, if Layout is layout_stride,
    R(a.data_handle(), ReturnMapping(transpose-extents(a.mapping().extents()),
    array{a.mapping().stride(1), a.mapping().stride(0)}), a.accessor())
(4.5) — otherwise, if Layout is a specialization of layout_transpose,
    R(a.data_handle(), a.mapping().nested_mapping(), a.accessor())
(4.6) — otherwise,
    R(a.data_handle(), ReturnMapping(a.mapping()), a.accessor())

```

⁵ [Example 1:

```

void test_transposed(mdspan<double, extents<size_t, 3, 4>> a) {
    const auto num_rows = a.extent(0);
    const auto num_cols = a.extent(1);

    auto a_t = transposed(a);
    assert(num_rows == a_t.extent(1));
    assert(num_cols == a_t.extent(0));
    assert(a.stride(0) == a_t.stride(1));
    assert(a.stride(1) == a_t.stride(0));

    for (size_t row = 0; row < num_rows; ++row) {
        for (size_t col = 0; col < num_cols; ++col) {
            assert(a[row, col] == a_t[col, row]);
        }
    }

    auto a_t_t = transposed(a_t);
    assert(num_rows == a_t_t.extent(0));
    assert(num_cols == a_t_t.extent(1));
    assert(a.stride(0) == a_t_t.stride(0));
    assert(a.stride(1) == a_t_t.stride(1));

    for (size_t row = 0; row < num_rows; ++row) {
        for (size_t col = 0; col < num_cols; ++col) {
            assert(a[row, col] == a_t_t[row, col]);
        }
    }
}

```

— end example]

29.9.11 Conjugate transpose in-place transform [linalg.conjtransposed]

¹ The `conjugate_transposed` function returns a conjugate transpose view of an object. This combines the effects of `transposed` and `conjugated`.

```

template<class ElementType, class Extents, class Layout, class Accessor>
constexpr auto conjugate_transposed(mdspan<ElementType, Extents, Layout, Accessor> a);

```

² *Effects:* Equivalent to: `return conjugated(transposed(a));`

³ [Example 1:

```

void test_conjugate_transposed(mdspan<complex<double>, extents<size_t, 3, 4>> a) {
    const auto num_rows = a.extent(0);
    const auto num_cols = a.extent(1);

    auto a_ct = conjugate_transposed(a);
    assert(num_rows == a_ct.extent(1));
    assert(num_cols == a_ct.extent(0));
    assert(a.stride(0) == a_ct.stride(1));
    assert(a.stride(1) == a_ct.stride(0));
}

```

```

for (size_t row = 0; row < num_rows; ++row) {
    for (size_t col = 0; col < num_rows; ++col) {
        assert(a[row, col] == conj(a_ct[col, row]));
    }
}

auto a_ct_ct = conjugate_transposed(a_ct);
assert(num_rows == a_ct_ct.extent(0));
assert(num_cols == a_ct_ct.extent(1));
assert(a.stride(0) == a_ct_ct.stride(0));
assert(a.stride(1) == a_ct_ct.stride(1));

for (size_t row = 0; row < num_rows; ++row) {
    for (size_t col = 0; col < num_rows; ++col) {
        assert(a[row, col] == a_ct_ct[row, col]);
        assert(conj(a_ct[col, row]) == a_ct_ct[row, col]);
    }
}
}

— end example]

```

29.9.12 Algorithm requirements based on template parameter name[linalg.algs.reqs]

- ¹ Throughout 29.9.13, 29.9.14, and 29.9.15, where the template parameters are not constrained, the names of template parameters are used to express the following constraints.
- (1.1) — `is_execution_policy<ExecutionPolicy>::value` is true (26.3.6.2).
 - (1.2) — `Real` is any type such that `complex<Real>` is specified (29.4.1).
 - (1.3) — `Triangle` is either `upper_triangle_t` or `lower_triangle_t`.
 - (1.4) — `DiagonalStorage` is either `implicit_unit_diagonal_t` or `explicit_diagonal_t`.

[Note 1: Function templates that have a template parameter named `ExecutionPolicy` are parallel algorithms (26.3.1). — end note]

29.9.13 BLAS 1 algorithms

[linalg.algs.blas1]

29.9.13.1 Complexity

[linalg.algs.blas1.complexity]

- ¹ *Complexity:* All algorithms in 29.9.13 with `mdspan` parameters perform a count of `mdspan` array accesses and arithmetic operations that is linear in the maximum product of extents of any `mdspan` parameter.

29.9.13.2 Givens rotations

[linalg.algs.blas1.givens]

29.9.13.2.1 Compute Givens rotation

[linalg.algs.blas1.givens.lartg]

```

template<class Real>
    setup_givens_rotation_result<Real> setup_givens_rotation(Real a, Real b) noexcept;

```

```

template<class Real>
    setup_givens_rotation_result<complex<Real>>
        setup_givens_rotation(complex<Real> a, complex<Real> b) noexcept;

```

- ¹ These functions compute the Givens plane rotation represented by the two values c and s such that the 2 x 2 system of equations

$$\begin{bmatrix} c & s \\ -\bar{s} & c \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}$$

holds, where c is always a real scalar, and $c^2 + |s|^2 = 1$. That is, c and s represent a 2 x 2 matrix, that when multiplied by the right by the input vector whose components are a and b , produces a result vector whose first component r is the Euclidean norm of the input vector, and whose second component is zero.

[Note 1: These functions correspond to the LAPACK function `xLARTG`^[19]. — end note]

- ² *Returns:* `c`, `s`, `r`, where `c` and `s` form the Givens plane rotation corresponding to the input `a` and `b`, and `r` is the Euclidean norm of the two-component vector formed by `a` and `b`.

29.9.13.2.2 Apply a computed Givens rotation to vectors [linalg.algs.blas1.givens.rot]

```
template<inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(InOutVec1 x, InOutVec2 y, Real c, Real s);
template<class ExecutionPolicy, inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(ExecutionPolicy&& exec,
                               InOutVec1 x, InOutVec2 y, Real c, Real s);
template<inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(InOutVec1 x, InOutVec2 y, Real c, complex<Real> s);
template<class ExecutionPolicy, inout-vector InOutVec1, inout-vector InOutVec2, class Real>
    void apply_givens_rotation(ExecutionPolicy&& exec,
                               InOutVec1 x, InOutVec2 y, Real c, complex<Real> s);
```

1 [Note 1: These functions correspond to the BLAS function `xROT`^[16]. — end note]

2 *Mandates:* `compatible-static-extents<InOutVec1, InOutVec2>(0, 0)` is true.

3 *Preconditions:* `x.extent(0)` equals `y.extent(0)`.

4 *Effects:* Applies the plane rotation specified by `c` and `s` to the input vectors `x` and `y`, as if the rotation were a 2 x 2 matrix and the input vectors were successive rows of a matrix with two rows.

29.9.13.3 Swap matrix or vector elements [linalg.algs.blas1.swap]

```
template<inout-object InOutObj1, inout-object InOutObj2>
    void swap_elements(InOutObj1 x, InOutObj2 y);
template<class ExecutionPolicy, inout-object InOutObj1, inout-object InOutObj2>
    void swap_elements(ExecutionPolicy&& exec, InOutObj1 x, InOutObj2 y);
```

1 [Note 1: These functions correspond to the BLAS function `xSWAP`^[16]. — end note]

2 *Constraints:* `x.rank()` equals `y.rank()`.

3 *Mandates:* For all `r` in the range `[0, x.rank())`,
`compatible-static-extents<InOutObj1, InOutObj2>(r, r)`
is true.

4 *Preconditions:* `x.extents()` equals `y.extents()`.

5 *Effects:* Swaps all corresponding elements of `x` and `y`.

29.9.13.4 Multiply the elements of an object in place by a scalar [linalg.algs.blas1.scal]

```
template<scalar Scalar, inout-object InOutObj>
    void scale(Scalar alpha, InOutObj x);
template<class ExecutionPolicy, scalar Scalar, inout-object InOutObj>
    void scale(ExecutionPolicy&& exec, Scalar alpha, InOutObj x);
```

1 [Note 1: These functions correspond to the BLAS function `xSCAL`^[16]. — end note]

2 *Effects:* Overwrites `x` with the result of computing the elementwise multiplication αx , where the scalar α is `alpha`.

29.9.13.5 Copy elements of one matrix or vector into another [linalg.algs.blas1.copy]

```
template<in-object InObj, out-object OutObj>
    void copy(InObj x, OutObj y);
template<class ExecutionPolicy, in-object InObj, out-object OutObj>
    void copy(ExecutionPolicy&& exec, InObj x, OutObj y);
```

1 [Note 1: These functions correspond to the BLAS function `xCOPY`^[16]. — end note]

2 *Constraints:* `x.rank()` equals `y.rank()`.

3 *Mandates:* For all `r` in the range `[0, x.rank())`,
`compatible-static-extents<InObj, OutObj>(r, r)`
is true.

4 *Preconditions:* `x.extents()` equals `y.extents()`.

5 *Effects:* Assigns each element of `x` to the corresponding element of `y`.

29.9.13.6 Add vectors or matrices elementwise

[linalg.algs.blas1.add]

```
template<in-object InObj1, in-object InObj2, out-object OutObj>
    void add(InObj1 x, InObj2 y, OutObj z);
template<class ExecutionPolicy, in-object InObj1, in-object InObj2, out-object OutObj>
    void add(ExecutionPolicy&& exec,
            InObj1 x, InObj2 y, OutObj z);
```

- 1 [Note 1: These functions correspond to the BLAS function **xAXPY**^[16]. — end note]
- 2 *Constraints:* **x.rank()**, **y.rank()**, and **z.rank()** are all equal.
- 3 *Mandates:* *possibly-addable*<InObj1, InObj2, OutObj>() is true.
- 4 *Preconditions:* *addable*(x,y,z) is true.
- 5 *Effects:* Computes $z = x + y$.
- 6 *Remarks:* **z** may alias **x** or **y**.

29.9.13.7 Dot product of two vectors

[linalg.algs.blas1.dot]

- 1 [Note 1: The functions in this section correspond to the BLAS functions **xDOT**, **xDOTU**, and **xDOTC**^[16]. — end note]
- 2 The following elements apply to all functions in 29.9.13.7.
- 3 *Mandates:* *compatible-static-extents*<InVec1, InVec2>(0, 0) is true.
- 4 *Preconditions:* **v1.extent(0)** equals **v2.extent(0)**.

```
template<in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dot(InVec1 v1, InVec2 v2, Scalar init);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, scalar Scalar>
    Scalar dot(ExecutionPolicy&& exec,
            InVec1 v1, InVec2 v2, Scalar init);
```

- 5 These functions compute a non-conjugated dot product with an explicitly specified result type.
- 6 *Returns:* Let **N** be **v1.extent(0)**.
- (6.1) — **init** if **N** is zero;
- (6.2) — otherwise, *GENERALIZED_SUM*(plus<>(), **init**, **v1[0]*v2[0]**, ..., **v1[N-1]*v2[N-1]**).
- 7 *Remarks:* If **InVec1::value_type**, **InVec2::value_type**, and **Scalar** are all floating-point types or specializations of **complex**, and if **Scalar** has higher precision than **InVec1::value_type** or **InVec2::value_type**, then intermediate terms in the sum use **Scalar**'s precision or greater.
- ```
template<in-vector InVec1, in-vector InVec2>
 auto dot(InVec1 v1, InVec2 v2);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2>
 auto dot(ExecutionPolicy&& exec,
 InVec1 v1, InVec2 v2);
```
- 8 These functions compute a non-conjugated dot product with a default result type.
- 9 *Effects:* Let **T** be **decltype(declval<typename InVec1::value\_type>() \* declval<typename InVec2::value\_type>())**. Then,
- (9.1) — the two-parameter overload is equivalent to:
- ```
    return dot(v1, v2, T{});
```
- and
- (9.2) — the three-parameter overload is equivalent to:
- ```
 return dot(std::forward<ExecutionPolicy>(exec), v1, v2, T{});
```

```
template<in-vector InVec1, in-vector InVec2, scalar Scalar>
 Scalar dotc(InVec1 v1, InVec2 v2, Scalar init);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, scalar Scalar>
 Scalar dotc(ExecutionPolicy&& exec,
 InVec1 v1, InVec2 v2, Scalar init);
```

- 10 These functions compute a conjugated dot product with an explicitly specified result type.



11 *Effects:*

- (11.1) — The three-parameter overload is equivalent to:

```
return dot(conjugated(v1), v2, init);
```

and

- (11.2) — the four-parameter overload is equivalent to:

```
return dot(std::forward<ExecutionPolicy>(exec), conjugated(v1), v2, init);
```

```
template<in-vector InVec1, in-vector InVec2>
auto dotc(InVec1 v1, InVec2 v2);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2>
auto dotc(ExecutionPolicy&& exec,
 InVec1 v1, InVec2 v2);
```

12 These functions compute a conjugated dot product with a default result type.

13 *Effects:* Let T be `decltype(conj-if-needed(declval<typename InVec1::value_type>()) * declval<typename InVec2::value_type>())`. Then,

- (13.1) — the two-parameter overload is equivalent to:

```
return dotc(v1, v2, T{});
```

and

- (13.2) — the three-parameter overload is equivalent to

```
return dotc(std::forward<ExecutionPolicy>(exec), v1, v2, T{});
```

### 29.9.13.8 Euclidean norm of a vector

[linalg.algs.blas1.nrm2]

```
template<in-vector InVec, scalar Scalar>
Scalar vector_two_norm(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, scalar Scalar>
Scalar vector_two_norm(ExecutionPolicy&& exec, InVec v, Scalar init);
```

1 [Note 1: These functions correspond to the BLAS function `xNRM2`<sup>[16]</sup>. — end note]

2 *Mandates:* `InVec::value_type` and `Scalar` are either a floating-point type, or a specialization of complex. Let `a` be `abs-if-needed(declval<typename InVec::value_type>())` and let `init_abs` be `abs-if-needed(init)`. Then, `decltype(init_abs * init_abs + a * a)` is convertible to `Scalar`.

3 *Returns:* The square root of the sum whose terms are the following:

- (3.1) — the square of the absolute value of `init`, and  
 (3.2) — the squares of the absolute values of the elements of `v`.

[Note 2: For `init` equal to zero, this is the Euclidean norm (also called 2-norm) of the vector `v`. — end note]

4 *Remarks:* If `Scalar` has higher precision than `InVec::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

```
template<in-vector InVec>
auto vector_two_norm(InVec v);
template<class ExecutionPolicy, in-vector InVec>
auto vector_two_norm(ExecutionPolicy&& exec, InVec v);
```

5 *Effects:* Let `a` be `abs-if-needed(declval<typename InVec::value_type>())`. Let T be `decltype(a * a)`. Then,

- (5.1) — the one-parameter overload is equivalent to:

```
return vector_two_norm(v, T{});
```

and

- (5.2) — the two-parameter overload is equivalent to:

```
return vector_two_norm(std::forward<ExecutionPolicy>(exec), v, T{});
```

## 29.9.13.9 Sum of absolute values of vector elements

[linalg.algs.blas1.asum]

```
template<in-vector InVec, scalar Scalar>
 Scalar vector_abs_sum(InVec v, Scalar init);
template<class ExecutionPolicy, in-vector InVec, scalar Scalar>
 Scalar vector_abs_sum(ExecutionPolicy&& exec, InVec v, Scalar init);
```

1 [Note 1: These functions correspond to the BLAS functions SASUM, DASUM, SCASUM, and DZASUM<sup>[16]</sup>. — end note]

2 *Mandates:*

decltype(init + *abs-if-needed*(*real-if-needed*(declval<typename InVec::value\_type>())) +  
*abs-if-needed*(*imag-if-needed*(declval<typename InVec::value\_type>()))

is convertible to `Scalar`.

3 *Returns:* Let `N` be `v.extent(0)`.

(3.1) — `init` if `N` is zero;

(3.2) — otherwise, if `InVec::value_type` is an arithmetic type,

`GENERALIZED_SUM`(`plus<>()`, `init`, *abs-if-needed*(`v[0]`), ..., *abs-if-needed*(`v[N-1]`))

(3.3) — otherwise,

`GENERALIZED_SUM`(`plus<>()`, `init`,  
*abs-if-needed*(*real-if-needed*(`v[0]`)) + *abs-if-needed*(*imag-if-needed*(`v[0]`)),  
 ...,  
*abs-if-needed*(*real-if-needed*(`v[N-1]`)) + *abs-if-needed*(*imag-if-needed*(`v[N-1]`)))

4 *Remarks:* If `InVec::value_type` and `Scalar` are all floating-point types or specializations of `complex`, and if `Scalar` has higher precision than `InVec::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

```
template<in-vector InVec>
 auto vector_abs_sum(InVec v);
template<class ExecutionPolicy, in-vector InVec>
 auto vector_abs_sum(ExecutionPolicy&& exec, InVec v);
```

5 *Effects:* Let `T` be `typename InVec::value_type`. Then,

(5.1) — the one-parameter overload is equivalent to:

`return vector_abs_sum(v, T{});`

and

(5.2) — the two-parameter overload is equivalent to:

`return vector_abs_sum(std::forward<ExecutionPolicy>(exec), v, T{});`

## 29.9.13.10 Index of maximum absolute value of vector elements

[linalg.algs.blas1.iamax]

```
template<in-vector InVec>
 typename InVec::size_type vector_idx_abs_max(InVec v);
template<class ExecutionPolicy, in-vector InVec>
 typename InVec::size_type vector_idx_abs_max(ExecutionPolicy&& exec, InVec v);
```

1 [Note 1: These functions correspond to the BLAS function IxAMAX<sup>[16]</sup>. — end note]

2 Let `T` be

decltype(*abs-if-needed*(*real-if-needed*(declval<typename InVec::value\_type>())) +  
*abs-if-needed*(*imag-if-needed*(declval<typename InVec::value\_type>())))

3 *Mandates:* `declval<T>() < declval<T>()` is a valid expression.

4 *Returns:*

(4.1) — `numeric_limits<typename InVec::size_type>::max()` if `v` has zero elements;

(4.2) — otherwise, the index of the first element of `v` having largest absolute value, if `InVec::value_type` is an arithmetic type;

(4.3) — otherwise, the index of the first element `ve` of `v` for which

*abs-if-needed*(*real-if-needed*(`ve`)) + *abs-if-needed*(*imag-if-needed*(`ve`))

has the largest value.

### 29.9.13.11 Frobenius norm of a matrix

[linalg.algs.blas1.matfrobnorm]

<sup>1</sup> [Note 1: These functions exist in the BLAS standard<sup>[20]</sup> but are not part of the reference implementation. — end note]

```
template<in-matrix InMat, scalar Scalar>
 Scalar matrix_frob_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
 Scalar matrix_frob_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

<sup>2</sup> *Mandates:* InMat::value\_type and Scalar are either a floating-point type, or a specialization of complex. Let a be *abs-if-needed*(declval<typename InMat::value\_type>()) and let init\_abs be *abs-if-needed*(init). Then, decltype(init\_abs \* init\_abs + a \* a) is convertible to Scalar.

<sup>3</sup> *Returns:* The square root of the sum whose terms are the following:

(3.1) — the square of the absolute value of init, and

(3.2) — the squares of the absolute values of the elements of A.

[Note 2: For init equal to zero, this is the Frobenius norm of the matrix A. — end note]

<sup>4</sup> *Remarks:* If Scalar has higher precision than InMat::value\_type, then intermediate terms in the sum use Scalar's precision or greater.

```
template<in-matrix InMat>
 auto matrix_frob_norm(InMat A);
template<class ExecutionPolicy, in-matrix InMat>
 auto matrix_frob_norm(ExecutionPolicy&& exec, InMat A);
```

<sup>5</sup> *Effects:* Let a be *abs-if-needed*(declval<typename InMat::value\_type>()). Let T be decltype(a \* a). Then,

(5.1) — the one-parameter overload is equivalent to:

```
return matrix_frob_norm(A, T{});
```

and

(5.2) — the two-parameter overload is equivalent to:

```
return matrix_frob_norm(std::forward<ExecutionPolicy>(exec), A, T{});
```

### 29.9.13.12 One norm of a matrix

[linalg.algs.blas1.matonenorm]

<sup>1</sup> [Note 1: These functions exist in the BLAS standard<sup>[20]</sup> but are not part of the reference implementation. — end note]

```
template<in-matrix InMat, scalar Scalar>
 Scalar matrix_one_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
 Scalar matrix_one_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

<sup>2</sup> *Mandates:* decltype(*abs-if-needed*(declval<typename InMat::value\_type>())) is convertible to Scalar.

<sup>3</sup> *Returns:*

(3.1) — init if A.extent(1) is zero;

(3.2) — otherwise, the sum of init and the one norm of the matrix A.

[Note 2: The one norm of the matrix A is the maximum over all columns of A, of the sum of the absolute values of the elements of the column. — end note]

<sup>4</sup> *Remarks:* If InMat::value\_type and Scalar are all floating-point types or specializations of complex, and if Scalar has higher precision than InMat::value\_type, then intermediate terms in the sum use Scalar's precision or greater.

```
template<in-matrix InMat>
 auto matrix_one_norm(InMat A);
```

```
template<class ExecutionPolicy, in-matrix InMat>
 auto matrix_one_norm(ExecutionPolicy&& exec, InMat A);
```

(5.1) *Effects:* Let T be `decltype(abs-if-needed(declval<typename InMat::value_type>()))`. Then,  
— the one-parameter overload is equivalent to:

```
 return matrix_one_norm(A, T{});
```

and

(5.2) — the two-parameter overload is equivalent to:

```
 return matrix_one_norm(std::forward<ExecutionPolicy>(exec), A, T{});
```

### 29.9.13.13 Infinity norm of a matrix

[`linalg.algs.blas1.matinfnorm`]

<sup>1</sup> [Note 1: These functions exist in the BLAS standard<sup>[20]</sup> but are not part of the reference implementation. — *end note*]

```
template<in-matrix InMat, scalar Scalar>
 Scalar matrix_inf_norm(InMat A, Scalar init);
template<class ExecutionPolicy, in-matrix InMat, scalar Scalar>
 Scalar matrix_inf_norm(ExecutionPolicy&& exec, InMat A, Scalar init);
```

<sup>2</sup> *Mandates:* `decltype(abs-if-needed(declval<typename InMat::value_type>()))` is convertible to `Scalar`.

<sup>3</sup> *Returns:*

(3.1) — `init` if `A.extent(0)` is zero;

(3.2) — otherwise, the sum of `init` and the infinity norm of the matrix `A`.

[Note 2: The infinity norm of the matrix `A` is the maximum over all rows of `A`, of the sum of the absolute values of the elements of the row. — *end note*]

<sup>4</sup> *Remarks:* If `InMat::value_type` and `Scalar` are all floating-point types or specializations of `complex`, and if `Scalar` has higher precision than `InMat::value_type`, then intermediate terms in the sum use `Scalar`'s precision or greater.

```
template<in-matrix InMat>
 auto matrix_inf_norm(InMat A);
template<class ExecutionPolicy, in-matrix InMat>
 auto matrix_inf_norm(ExecutionPolicy&& exec, InMat A);
```

(5.1) *Effects:* Let T be `decltype(abs-if-needed(declval<typename InMat::value_type>()))`. Then,  
— the one-parameter overload is equivalent to:

```
 return matrix_inf_norm(A, T{});
```

and

(5.2) — the two-parameter overload is equivalent to:

```
 return matrix_inf_norm(std::forward<ExecutionPolicy>(exec), A, T{});
```

## 29.9.14 BLAS 2 algorithms

[`linalg.algs.blas2`]

### 29.9.14.1 General matrix-vector product

[`linalg.algs.blas2.gemv`]

<sup>1</sup> [Note 1: These functions correspond to the BLAS function `xGEMV`. — *end note*]

<sup>2</sup> The following elements apply to all functions in 29.9.14.1.

<sup>3</sup> *Mandates:*

(3.1) — `possibly-multipliable<decltype(A), decltype(x), decltype(y)>()` is true, and

(3.2) — `possibly-addable<decltype(y), decltype(y), decltype(z)>()` is true for those overloads that take a `z` parameter.

<sup>4</sup> *Preconditions:*

(4.1) — `multipliable(A,x,y)` is true, and

(4.2) — `addable(y,y,z)` is true for those overloads that take a `z` parameter.

5 *Complexity:*  $\mathcal{O}(A.\text{extent}(0) \times x.\text{extent}(0))$ .

```
template<in-matrix InMat, in-vector InVec, out-vector OutVec>
void matrix_vector_product(InMat A, InVec x, OutVec y);
template<class ExecutionPolicy, in-matrix InMat, in-vector InVec, out-vector OutVec>
void matrix_vector_product(ExecutionPolicy&& exec, InMat A, InVec x, OutVec y);
```

6 These functions perform an overwriting matrix-vector product.

7 *Effects:* Computes  $y = Ax$ .

[Example 1:

```
constexpr size_t num_rows = 5;
constexpr size_t num_cols = 6;

// y = 3.0 * A * x
void scaled_matvec_1(mdspan<double, extents<size_t, num_rows, num_cols>> A,
mdspan<double, extents<size_t, num_cols>> x, mdspan<double, extents<size_t, num_rows>> y) {
 matrix_vector_product(scaled(3.0, A), x, y);
}

// z = 7.0 times the transpose of A, times y
void scaled_transposed_matvec(mdspan<double, extents<size_t, num_rows, num_cols>> A,
mdspan<double, extents<size_t, num_rows>> y, mdspan<double, extents<size_t, num_cols>> z) {
 matrix_vector_product(scaled(7.0, transposed(A)), y, z);
}
```

— end example]

```
template<in-matrix InMat, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void matrix_vector_product(InMat A, InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
in-matrix InMat, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void matrix_vector_product(ExecutionPolicy&& exec,
InMat A, InVec1 x, InVec2 y, OutVec z);
```

8 These functions perform an updating matrix-vector product.

9 *Effects:* Computes  $z = y + Ax$ .

10 *Remarks:*  $z$  may alias  $y$ .

[Example 2:

```
// y = 3.0 * A * x + 2.0 * y
void scaled_matvec_2(mdspan<double, extents<size_t, num_rows, num_cols>> A,
mdspan<double, extents<size_t, num_cols>> x, mdspan<double, extents<size_t, num_rows>> y) {
 matrix_vector_product(scaled(3.0, A), x, scaled(2.0, y), y);
}
```

— end example]

#### 29.9.14.2 Symmetric matrix-vector product

[linalg.algs.blas2.symv]

1 [Note 1: These functions correspond to the BLAS functions xSYMV and xSPMV<sup>[17]</sup>. — end note]

2 The following elements apply to all functions in 29.9.14.2.

3 *Mandates:*

- (3.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (3.2) — `compatible-static-extents<decltype(A), decltype(A)>(0, 1)` is true;
- (3.3) — `possibly-multipliable<decltype(A), decltype(x), decltype(y)>()` is true; and
- (3.4) — `possibly-addable<decltype(y), decltype(y), decltype(z)>()` is true for those overloads that take a `z` parameter.

4 *Preconditions:*

- (4.1) — `A.extent(0)` equals `A.extent(1)`,
- (4.2) — `multipliable(A,x,y)` is true, and

(4.3) — *addable*(*y*,*y*,*z*) is true for those overloads that take a *z* parameter.

5 *Complexity*:  $\mathcal{O}(A.\text{extent}(0) \times x.\text{extent}(0))$ .

```
template<in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
void symmetric_matrix_vector_product(InMat A, Triangle t, InVec x, OutVec y);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
void symmetric_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, InVec x, OutVec y);
```

6 These functions perform an overwriting symmetric matrix-vector product, taking into account the *Triangle* parameter that applies to the symmetric matrix *A* (29.9.3).

7 *Effects*: Computes  $y = Ax$ .

```
template<in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void symmetric_matrix_vector_product(InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void symmetric_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
```

8 These functions perform an updating symmetric matrix-vector product, taking into account the *Triangle* parameter that applies to the symmetric matrix *A* (29.9.3).

9 *Effects*: Computes  $z = y + Ax$ .

10 *Remarks*: *z* may alias *y*.

### 29.9.14.3 Hermitian matrix-vector product

[linalg.algs.blas2.hemv]

1 [Note 1: These functions correspond to the BLAS functions *xHEMV* and *xHPMV*<sup>[17]</sup>. — end note]

2 The following elements apply to all functions in 29.9.14.3.

3 *Mandates*:

(3.1) — If *InMat* has *layout\_blas\_packed* layout, then the layout's *Triangle* template argument has the same type as the function's *Triangle* template argument;

(3.2) — *compatible-static-extents*<decltype(*A*), decltype(*A*)>(0, 1) is true;

(3.3) — *possibly-multipliable*<decltype(*A*), decltype(*x*), decltype(*y*)>() is true; and

(3.4) — *possibly-addable*<decltype(*y*), decltype(*y*), decltype(*z*)>() is true for those overloads that take a *z* parameter.

4 *Preconditions*:

(4.1) — *A.extent*(0) equals *A.extent*(1),

(4.2) — *multipliable*(*A*, *x*, *y*) is true, and

(4.3) — *addable*(*y*, *y*, *z*) is true for those overloads that take a *z* parameter.

5 *Complexity*:  $\mathcal{O}(A.\text{extent}(0) \times x.\text{extent}(0))$ .

```
template<in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
void hermitian_matrix_vector_product(InMat A, Triangle t, InVec x, OutVec y);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, in-vector InVec, out-vector OutVec>
void hermitian_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, InVec x, OutVec y);
```

6 These functions perform an overwriting Hermitian matrix-vector product, taking into account the *Triangle* parameter that applies to the Hermitian matrix *A* (29.9.3).

7 *Effects*: Computes  $y = Ax$ .

```
template<in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void hermitian_matrix_vector_product(InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
```

```
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, in-vector InVec1, in-vector InVec2, out-vector OutVec>
void hermitian_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, InVec1 x, InVec2 y, OutVec z);
```

These functions perform an updating Hermitian matrix-vector product, taking into account the `Triangle` parameter that applies to the Hermitian matrix `A` (29.9.3).

*Effects:* Computes  $z = y + Ax$ .

*Remarks:* `z` may alias `y`.

#### 29.9.14.4 Triangular matrix-vector product

[linalg.algs.blas2.trmv]

[Note 1: These functions correspond to the BLAS functions `xTRMV` and `xTPMV`<sup>[17]</sup>. — end note]

The following elements apply to all functions in 29.9.14.4.

*Mandates:*

- (3.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (3.2) — `compatible-static-extents<decltype(A), decltype(A)>(0, 1)` is true;
- (3.3) — `compatible-static-extents<decltype(A), decltype(y)>(0, 0)` is true;
- (3.4) — `compatible-static-extents<decltype(A), decltype(x)>(0, 0)` is true for those overloads that take an `x` parameter; and
- (3.5) — `compatible-static-extents<decltype(A), decltype(z)>(0, 0)` is true for those overloads that take a `z` parameter.

*Preconditions:*

- (4.1) — `A.extent(0)` equals `A.extent(1)`,
- (4.2) — `A.extent(0)` equals `y.extent(0)`,
- (4.3) — `A.extent(0)` equals `x.extent(0)` for those overloads that take an `x` parameter, and
- (4.4) — `A.extent(0)` equals `z.extent(0)` for those overloads that take a `z` parameter.

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
 out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, in-vector InVec,
 out-vector OutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InVec x, OutVec y);
```

These functions perform an overwriting triangular matrix-vector product, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

*Effects:* Computes  $y = Ax$ .

*Complexity:*  $\mathcal{O}(A.extent(0) \times x.extent(0))$ .

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d, InOutVec y);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InOutVec y);
```

These functions perform an in-place triangular matrix-vector product, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

[Note 2: Performing this operation in place hinders parallelization. However, other `ExecutionPolicy` specific optimizations, such as vectorization, are still possible. — end note]

*Effects:* Computes a vector  $y'$  such that  $y' = Ay$ , and assigns each element of  $y'$  to the corresponding element of  $y$ .

*Complexity:*  $\mathcal{O}(A.extent(0) \times y.extent(0))$ .



```

template<in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(InMat A, Triangle t, DiagonalStorage d,
 InVec1 x, InVec2 y, OutVec z);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec1, in-vector InVec2, out-vector OutVec>
void triangular_matrix_vector_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InVec1 x, InVec2 y, OutVec z);

```

11 These functions perform an updating triangular matrix-vector product, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

12 *Effects:* Computes  $z = y + Ax$ .

13 *Complexity:*  $\mathcal{O}(A.\text{extent}(0) \times x.\text{extent}(0))$ .

14 *Remarks:* `z` may alias `y`.

#### 29.9.14.5 Solve a triangular linear system

[linalg.algs.blas2.trsv]

1 [Note 1: These functions correspond to the BLAS functions `xTRSV` and `xTPSV`<sup>[17]</sup>. — end note]

2 The following elements apply to all functions in 29.9.14.5.

3 *Mandates:*

- (3.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (3.2) — `compatible-static-extents<decltype(A), decltype(A)>(0, 1)` is true;
- (3.3) — `compatible-static-extents<decltype(A), decltype(b)>(0, 0)` is true; and
- (3.4) — `compatible-static-extents<decltype(A), decltype(x)>(0, 0)` is true for those overloads that take an `x` parameter.

4 *Preconditions:*

- (4.1) — `A.extent(0)` equals `A.extent(1)`,
- (4.2) — `A.extent(0)` equals `b.extent(0)`, and
- (4.3) — `A.extent(0)` equals `x.extent(0)` for those overloads that take an `x` parameter.

```

template<in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec, out-vector OutVec, class BinaryDivideOp>
void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d,
 InVec b, OutVec x, BinaryDivideOp divide);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec, out-vector OutVec, class BinaryDivideOp>
void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InVec b, OutVec x, BinaryDivideOp divide);

```

5 These functions perform a triangular solve, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

6 *Effects:* Computes a vector  $x'$  such that  $b = Ax'$ , and assigns each element of  $x'$  to the corresponding element of  $x$ . If no such  $x'$  exists, then the elements of `x` are valid but unspecified.

7 *Complexity:*  $\mathcal{O}(A.\text{extent}(1) \times b.\text{extent}(0))$ .

```

template<in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec, out-vector OutVec>
void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d, InVec b, OutVec x);

```

8 *Effects:* Equivalent to:

```
triangular_matrix_vector_solve(A, t, d, b, x, divides<void>{});
```



```
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 in-vector InVec, out-vector OutVec>
void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InVec b, OutVec x);
```

9     *Effects:* Equivalent to:

```
 triangular_matrix_vector_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, b, x, divides<void>{});
```

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-vector InOutVec, class BinaryDivideOp>
void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d,
 InOutVec b, BinaryDivideOp divide);
```

```
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-vector InOutVec, class BinaryDivideOp>
void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InOutVec b, BinaryDivideOp divide);
```

10     These functions perform an in-place triangular solve, taking into account the Triangle and DiagonalStorage parameters that apply to the triangular matrix A (29.9.3).

[Note 2: Performing triangular solve in place hinders parallelization. However, other ExecutionPolicy specific optimizations, such as vectorization, are still possible. — end note]

11     *Effects:* Computes a vector  $x'$  such that  $b = Ax'$ , and assigns each element of  $x'$  to the corresponding element of  $b$ . If no such  $x'$  exists, then the elements of  $b$  are valid but unspecified.

12     *Complexity:*  $\mathcal{O}(A.\text{extent}(1) \times b.\text{extent}(0))$ .

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_solve(InMat A, Triangle t, DiagonalStorage d, InOutVec b);
```

13     *Effects:* Equivalent to:

```
 triangular_matrix_vector_solve(A, t, d, b, divides<void>{});
```

```
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-vector InOutVec>
void triangular_matrix_vector_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InOutVec b);
```

14     *Effects:* Equivalent to:

```
 triangular_matrix_vector_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, b, divides<void>{});
```

#### 29.9.14.6 Rank-1 (outer product) update of a matrix

[linalg.algs.blas2.rank1]

1     The following elements apply to all functions in 29.9.14.6.

2     *Mandates:*

(2.1) — *possibly-multipliable*<OutMat, InVec2, InVec1>() is true, and

(2.2) — *possibly-addable*<OutMat, InMat, OutMat>() is true for those overloads with an E parameter.

3     *Preconditions:*

(3.1) — *multipliable*(A, y, x) is true, and

(3.2) — *addable*(A, E, A) is true for those overloads with an E parameter.

4     *Complexity:*  $\mathcal{O}(x.\text{extent}(0) \times y.\text{extent}(0))$ .

```
template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
void matrix_rank_1_update(InVec1 x, InVec2 y, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, out-matrix OutMat>
void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x, InVec2 y, OutMat A);
```

5     These functions perform an overwriting nonsymmetric nonconjugated rank-1 update.

[Note 1: These functions correspond to the BLAS functions xGER (for real element types) and xGERU (for complex element types)<sup>[17]</sup>. — end note]

6 *Effects:* Computes  $A = xy^T$ .

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix OutMat>
void matrix_rank_1_update(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
out-matrix OutMat>
void matrix_rank_1_update(ExecutionPolicy&& exec, InVec1 x, InVec2 y, InMat E, OutMat A);
```

7 These functions perform an updating nonsymmetric nonconjugated rank-1 update.

[Note 2: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERU` (for complex element types)<sup>[17]</sup>. — end note]

8 *Effects:* Computes  $A = E + xy^T$ .

9 *Remarks:* A may alias E.

```
template<in-vector InVec1, in-vector InVec2, out-matrix OutMat>
void matrix_rank_1_update_c(InVec1 x, InVec2 y, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, out-matrix OutMat>
void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x, InVec2 y, OutMat A);
```

10 These functions perform an overwriting nonsymmetric conjugated rank-1 update.

[Note 3: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERC` (for complex element types)<sup>[17]</sup>. — end note]

11 *Effects:*

(11.1) — For the overloads without an `ExecutionPolicy` argument, equivalent to:

```
matrix_rank_1_update(x, conjugated(y), A);
```

(11.2) — otherwise, equivalent to:

```
matrix_rank_1_update(std::forward<ExecutionPolicy>(exec), x, conjugated(y), A);
```

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat, out-matrix OutMat>
void matrix_rank_1_update_c(InVec1 x, InVec2 y, InMat E, OutMat A);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
out-matrix OutMat>
void matrix_rank_1_update_c(ExecutionPolicy&& exec, InVec1 x, InVec2 y, InMat E, OutMat A);
```

12 These functions perform an updating nonsymmetric conjugated rank-1 update.

[Note 4: These functions correspond to the BLAS functions `xGER` (for real element types) and `xGERC` (for complex element types)<sup>[17]</sup>. — end note]

13 *Effects:*

(13.1) — For the overloads without an `ExecutionPolicy` argument, equivalent to:

```
matrix_rank_1_update(x, conjugated(y), E, A);
```

(13.2) — otherwise, equivalent to:

```
matrix_rank_1_update(std::forward<ExecutionPolicy>(exec), x, conjugated(y), E, A);
```

#### 29.9.14.7 Symmetric or Hermitian Rank-1 (outer product) update of a matrix [linalg.algs.blas2.symherrank1]

1 [Note 1: These functions correspond to the BLAS functions `xSYR`, `xSPR`, `xHER`, and `xHPR`<sup>[17]</sup>. They have overloads taking a scaling factor `alpha`, because it would be impossible to express the update  $A = A - \alpha xx^T$  in noncomplex arithmetic otherwise. — end note]

2 The following elements apply to all functions in 29.9.14.7.

3 For any function F in this subclause with a parameter named `t`, an `InMat` template parameter, and a function parameter `InMat E`, `t` applies to accesses done through the parameter `E`. F only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, F only uses the value `real-if-needed(E[i, i])` if the name of F starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, F only uses the value

(3.1) — `conj-if-needed(E[j, i])` if the name of F starts with `hermitian`, or

(3.2) — `E[j, i]` if the name of F starts with `symmetric`.

4 *Mandates:*

- (4.1) — If OutMat has layout\_blas\_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument;
- (4.2) — *compatible-static-extents*<decltype(A), decltype(A)>(0, 1) is true;
- (4.3) — *compatible-static-extents*<decltype(A), decltype(x)>(0, 0) is true; and
- (4.4) — *possibly-addable*<decltype(A), decltype(E), decltype(A)>() is true for those overloads with an E parameter.

5 *Preconditions:*

- (5.1) — A.extent(0) equals A.extent(1),
- (5.2) — A.extent(0) equals x.extent(0), and
- (5.3) — *addable*(A, E, A) is true for those overloads with an E parameter.

6 *Complexity:*  $\mathcal{O}(x.extent(0) \times x.extent(0))$ .

```
template<scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, OutMat A, Triangle t);
template<class ExecutionPolicy,
 scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
 Scalar alpha, InVec x, OutMat A, Triangle t);
```

7 These functions perform an overwriting symmetric rank-1 update of the symmetric matrix A, taking into account the Triangle parameter that applies to A (29.9.3).

8 *Effects:* Computes a matrix  $A = \alpha x x^T$ , where the scalar  $\alpha$  is alpha.

```
template<scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
 class Triangle>
void symmetric_matrix_rank_1_update(Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy, scalar Scalar, in-vector InVec, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_1_update(ExecutionPolicy&& exec,
 Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
```

9 These functions perform an updating symmetric rank-1 update of the symmetric matrix A using the symmetric matrix E, taking into account the Triangle parameter that applies to A and E (29.9.3).

10 *Effects:* Computes  $A = E + \alpha x x^T$ , where the scalar  $\alpha$  is alpha.

11 *Remarks:* A may alias E.

```
template<scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, OutMat A, Triangle t);
template<class ExecutionPolicy,
 scalar Scalar, in-vector InVec, possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
 Scalar alpha, InVec x, OutMat A, Triangle t);
```

12 These functions perform an overwriting Hermitian rank-1 update of the Hermitian matrix A, taking into account the Triangle parameter that applies to A (29.9.3).

13 *Effects:* Computes  $A = \alpha x x^H$ , where the scalar  $\alpha$  is *real-if-needed*(alpha).

```
template<scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_1_update(Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy,
 scalar Scalar, in-vector InVec, in-matrix InMat, possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_1_update(ExecutionPolicy&& exec,
 Scalar alpha, InVec x, InMat E, OutMat A, Triangle t);
```

14 These functions perform an updating Hermitian rank-1 update of the Hermitian matrix A using the Hermitian matrix E, taking into account the Triangle parameter that applies to A and E (29.9.3).

15 *Effects:* Computes  $A = E + \alpha x x^H$ , where the scalar  $\alpha$  is *real-if-needed*(alpha).

16 *Remarks:* A may alias E.

#### 29.9.14.8 Symmetric and Hermitian rank-2 matrix updates [linalg.algs.blas2.rank2]

1 [Note 1: These functions correspond to the BLAS functions xSYR2, xSPR2, xHER2 and xHPR2<sup>[17]</sup>. — end note]

2 The following elements apply to all functions in 29.9.14.8.

3 For any function F in this subclause with a parameter named t, an InMat template parameter, and a function parameter InMat E, t applies to accesses done through the parameter E. F only accesses the triangle of E specified by t. For accesses of diagonal elements E[i, i], F only uses the value *real-if-needed*(E[i, i]) if the name of F starts with *hermitian*. For accesses E[i, j] outside the triangle specified by t, F only uses the value

(3.1) — *conj-if-needed*(E[j, i]) if the name of F starts with *hermitian*, or

(3.2) — E[j, i] if the name of F starts with *symmetric*.

4 *Mandates:*

(4.1) — If OutMat has layout\_blas\_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument;

(4.2) — If the function has an InMat template parameter and InMat has layout\_blas\_packed layout, then the layout's Triangle template argument has the same type as the function's Triangle template argument;

(4.3) — *compatible-static-extents*<decltype(A), decltype(A)>(0, 1) is true;

(4.4) — *possibly-multipliable*<decltype(A), decltype(x), decltype(y)>() is true; and

(4.5) — *possibly-addable*<decltype(A), decltype(E), decltype(A)>() is true for those overloads with an E parameter.

5 *Preconditions:*

(5.1) — A.extent(0) equals A.extent(1),

(5.2) — *multipliable*(A, x, y) is true, and

(5.3) — *addable*(A, E, A) is true for those overloads with an E parameter.

6 *Complexity:*  $\mathcal{O}(x.extent(0) \times y.extent(0))$ .

```
template<in-vector InVec1, in-vector InVec2,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
 InVec1 x, InVec2 y, OutMat A, Triangle t);
```

7 These functions perform an overwriting symmetric rank-2 update of the symmetric matrix A, taking into account the Triangle parameter that applies to A (29.9.3).

8 *Effects:* Computes  $A = xy^T + yx^T$ .

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2_update(ExecutionPolicy&& exec,
 InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
```

9 These functions perform an updating symmetric rank-2 update of the symmetric matrix A using the symmetric matrix E, taking into account the Triangle parameter that applies to A and E (29.9.3).

10 *Effects:* Computes  $A = E + xy^T + yx^T$ .

11 *Remarks:* A may alias E.

```
template<in-vector InVec1, in-vector InVec2,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
 InVec1 x, InVec2 y, OutMat A, Triangle t);
```

12 These functions perform an overwriting Hermitian rank-2 update of the Hermitian matrix A, taking into account the `Triangle` parameter that applies to A (29.9.3).

13 *Effects:* Computes  $A = xy^H + yx^H$ .

```
template<in-vector InVec1, in-vector InVec2, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
template<class ExecutionPolicy, in-vector InVec1, in-vector InVec2, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2_update(ExecutionPolicy&& exec,
 InVec1 x, InVec2 y, InMat E, OutMat A, Triangle t);
```

14 These functions perform an updating Hermitian rank-2 update of the Hermitian matrix A using the Hermitian matrix E, taking into account the `Triangle` parameter that applies to A and E (29.9.3).

15 *Effects:* Computes  $A = E + xy^H + yx^H$ .

16 *Remarks:* A may alias E.

### 29.9.15 BLAS 3 algorithms

[linalg.algs.blas3]

#### 29.9.15.1 General matrix-matrix product

[linalg.algs.blas3.gemm]

1 [Note 1: These functions correspond to the BLAS function `xGEMM`<sup>[18]</sup>. — end note]

2 The following elements apply to all functions in 29.9.15.1 in addition to function-specific elements.

3 *Mandates:* `possibly-multipliable<decltype(A), decltype(B), decltype(C)>()` is true.

4 *Preconditions:* `multipliable(A, B, C)` is true.

5 *Complexity:*  $\mathcal{O}(A.\text{extent}(0) \times A.\text{extent}(1) \times B.\text{extent}(1))$ .

```
template<in-matrix InMat1, in-matrix InMat2, out-matrix OutMat>
void matrix_product(InMat1 A, InMat2 B, OutMat C);
template<class ExecutionPolicy, in-matrix InMat1, in-matrix InMat2, out-matrix OutMat>
void matrix_product(ExecutionPolicy&& exec, InMat1 A, InMat2 B, OutMat C);
```

6 *Effects:* Computes  $C = AB$ .

```
template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void matrix_product(InMat1 A, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void matrix_product(ExecutionPolicy&& exec, InMat1 A, InMat2 B, InMat3 E, OutMat C);
```

7 *Mandates:* `possibly-addable<InMat3, InMat3, OutMat>()` is true.

8 *Preconditions:* `addable(E, E, C)` is true.

9 *Effects:* Computes  $C = E + AB$ .

10 *Remarks:* C may alias E.

#### 29.9.15.2 Symmetric, Hermitian, and triangular matrix-matrix product

[linalg.algs.blas3.xxmm]

1 [Note 1: These functions correspond to the BLAS functions `xSYMM`, `xHEMM`, and `xTRMM`<sup>[18]</sup>. — end note]

2 The following elements apply to all functions in 29.9.15.2 in addition to function-specific elements.

3 *Mandates:*

(3.1) — `possibly-multipliable<decltype(A), decltype(B), decltype(C)>()` is true, and

(3.2) — `possibly-addable<decltype(E), decltype(E), decltype(C)>()` is true for those overloads that take an E parameter.

4 *Preconditions:*

- (4.1) — *multipliable*(A, B, C) is true, and  
 (4.2) — *addable*(E, E, C) is true for those overloads that take an E parameter.

5 *Complexity:*  $\mathcal{O}(A.\text{extent}(0) \times A.\text{extent}(1) \times B.\text{extent}(1))$ .

```
template<in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, Triangle t, InMat2 B, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec, InMat1 A, Triangle t, InMat2 B, OutMat C);

template<in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void hermitian_matrix_product(InMat1 A, Triangle t, InMat2 B, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, in-matrix InMat2, out-matrix OutMat>
void hermitian_matrix_product(ExecutionPolicy&& exec, InMat1 A, Triangle t, InMat2 B, OutMat C);

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_product(InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, OutMat C);
template<class ExecutionPolicy, in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, OutMat C);
```

- 6 These functions perform a matrix-matrix multiply, taking into account the **Triangle** and **DiagonalStorage** (if applicable) parameters that apply to the symmetric, Hermitian, or triangular (respectively) matrix A (29.9.3).

7 *Mandates:*

- (7.1) — If InMat1 has `layout_blas_packed` layout, then the layout's **Triangle** template argument has the same type as the function's **Triangle** template argument; and  
 (7.2) — *compatible-static-extents*<InMat1, InMat1>(0, 1) is true.

8 *Preconditions:*  $A.\text{extent}(0) == A.\text{extent}(1)$  is true.

9 *Effects:* Computes  $C = AB$ .

```
template<in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, InMat2 B, Triangle t, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void hermitian_matrix_product(InMat1 A, InMat2 B, Triangle t, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, out-matrix OutMat>
void hermitian_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
 out-matrix OutMat>
void triangular_matrix_product(InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
 out-matrix OutMat>
void triangular_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, OutMat C);
```

- 10 These functions perform a matrix-matrix multiply, taking into account the **Triangle** and **DiagonalStorage** (if applicable) parameters that apply to the symmetric, Hermitian, or triangular (respectively) matrix B (29.9.3).



11 *Mandates:*

- (11.1) — If `InMat2` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument; and
- (11.2) — *compatible-static-extents*<`InMat2`, `InMat2`>(0, 1) is true.

12 *Preconditions:* `B.extent(0) == B.extent(1)` is true.

13 *Effects:* Computes  $C = AB$ .

```
template<in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
 out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
 out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);

template<in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
 out-matrix OutMat>
void hermitian_matrix_product(InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, in-matrix InMat2, in-matrix InMat3,
 out-matrix OutMat>
void hermitian_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, InMat2 B, InMat3 E, OutMat C);

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void triangular_matrix_product(InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, InMat3 E,
 OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, in-matrix InMat3, out-matrix OutMat>
void triangular_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d, InMat2 B, InMat3 E,
 OutMat C);
```

14 These functions perform a potentially overwriting matrix-matrix multiply-add, taking into account the `Triangle` and `DiagonalStorage` (if applicable) parameters that apply to the symmetric, Hermitian, or triangular (respectively) matrix `A` (29.9.3).

15 *Mandates:*

- (15.1) — If `InMat1` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument; and
- (15.2) — *compatible-static-extents*<`InMat1`, `InMat1`>(0, 1) is true.

16 *Preconditions:* `A.extent(0) == A.extent(1)` is true.

17 *Effects:* Computes  $C = E + AB$ .

18 *Remarks:* `C` may alias `E`.

```
template<in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
 out-matrix OutMat>
void symmetric_matrix_product(InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
 out-matrix OutMat>
void symmetric_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
 out-matrix OutMat>
void hermitian_matrix_product(InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);
```

```

template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, in-matrix InMat3,
 out-matrix OutMat>
void hermitian_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, InMat3 E, OutMat C);

template<in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
 in-matrix InMat3, out-matrix OutMat>
void triangular_matrix_product(InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, InMat3 E,
 OutMat C);

template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2, class Triangle, class DiagonalStorage,
 in-matrix InMat3, out-matrix OutMat>
void triangular_matrix_product(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, Triangle t, DiagonalStorage d, InMat3 E,
 OutMat C);

```

These functions perform a potentially overwriting matrix-matrix multiply-add, taking into account the **Triangle** and **DiagonalStorage** (if applicable) parameters that apply to the symmetric, Hermitian, or triangular (respectively) matrix B (29.9.3).

*Mandates:*

- If **InMat2** has `layout_blas_packed` layout, then the layout's **Triangle** template argument has the same type as the function's **Triangle** template argument; and
- `compatible-static-extents<InMat2, InMat2>(0, 1)` is true.

*Preconditions:* `B.extent(0) == B.extent(1)` is true.

*Effects:* Computes  $C = E + AB$ .

*Remarks:* **C** may alias **E**.

### 29.9.15.3 In-place triangular matrix-matrix product

[linalg.algs.blas3.trmm]

These functions perform an in-place matrix-matrix multiply, taking into account the **Triangle** and **DiagonalStorage** parameters that apply to the triangular matrix **A** (29.9.3).

[Note 1: These functions correspond to the BLAS function `xTRMM`<sup>[18]</sup>. — end note]

```

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_left_product(InMat A, Triangle t, DiagonalStorage d, InOutMat C);
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_left_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InOutMat C);

```

*Mandates:*

- If **InMat** has `layout_blas_packed` layout, then the layout's **Triangle** template argument has the same type as the function's **Triangle** template argument;
- `possibly-multipliable<InMat, InOutMat, InOutMat>()` is true; and
- `compatible-static-extents<InMat, InMat>(0, 1)` is true.

*Preconditions:*

- `multipliable(A, C, C)` is true, and
- `A.extent(0) == A.extent(1)` is true.

*Effects:* Computes a matrix  $C'$  such that  $C' = AC$  and assigns each element of  $C'$  to the corresponding element of  $C$ .

*Complexity:*  $\mathcal{O}(A.extent(0) \times A.extent(1) \times C.extent(0))$ .

```

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_right_product(InMat A, Triangle t, DiagonalStorage d, InOutMat C);

```



```
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_right_product(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d, InOutMat C);
```

6 *Mandates:*

- (6.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (6.2) — `possibly-multipliable<InOutMat, InMat, InOutMat>()` is true; and
- (6.3) — `compatible-static-extents<InMat, InMat>(0, 1)` is true.

7 *Preconditions:*

- (7.1) — `multipliable(C, A, C)` is true, and
- (7.2) — `A.extent(0) == A.extent(1)` is true.

8 *Effects:* Computes a matrix  $C'$  such that  $C' = CA$  and assigns each element of  $C'$  to the corresponding element of  $C$ .

9 *Complexity:*  $\mathcal{O}(A.extent(0) \times A.extent(1) \times C.extent(0))$ .

#### 29.9.15.4 Rank-k update of a symmetric or Hermitian matrix [linalg.algs.blas3.rankk]

1 [Note 1: These functions correspond to the BLAS functions `xSYRK` and `xHERK`<sup>[18]</sup>. — end note]

2 The following elements apply to all functions in 29.9.15.4.

3 For any function `F` in this subclause with a parameter named `t`, an `InMat2` template parameter, and a function parameter `InMat2 E`, `t` applies to accesses done through the parameter `E`. `F` only accesses the triangle of `E` specified by `t`. For accesses of diagonal elements `E[i, i]`, `F` only uses the value `real-if-needed(E[i, i])` if the name of `F` starts with `hermitian`. For accesses `E[i, j]` outside the triangle specified by `t`, `F` only uses the value

- (3.1) — `conj-if-needed(E[j, i])` if the name of `F` starts with `hermitian`, or
- (3.2) — `E[j, i]` if the name of `F` starts with `symmetric`.

4 *Mandates:*

- (4.1) — If `OutMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (4.2) — If the function has an `InMat2` template parameter and if `InMat2` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (4.3) — `possibly-multipliable<decltype(A), decltype(transposed(A)), decltype(C)>()` is true; and
- (4.4) — `possibly-addable<decltype(C), decltype(E), decltype(C)>()` is true for those overloads with an `E` parameter.

5 *Preconditions:*

- (5.1) — `multipliable(A, transposed(A), C)` is true; and  
[Note 2: This implies that `C` is square. — end note]
- (5.2) — `addable(C, E, C)` is true for those overloads with an `E` parameter.

6 *Complexity:*  $\mathcal{O}(A.extent(0) \times A.extent(1) \times A.extent(0))$ .

7 *Remarks:* `C` may alias `E` for those overloads with an `E` parameter.

```
template<scalar Scalar, in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_k_update(Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy, scalar Scalar,
 in-matrix InMat, possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec,
 Scalar alpha, InMat A, OutMat C, Triangle t);
```

8 *Effects:* Computes  $C = \alpha AA^T$ , where the scalar  $\alpha$  is `alpha`.

```

template<scalar Scalar, in-matrix InMat, possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_k_update(Scalar alpha, InMat A, OutMat C, Triangle t);
template<class ExecutionPolicy, scalar Scalar, in-matrix InMat,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec, Scalar alpha, InMat A,
 OutMat C, Triangle t);

```

9 *Effects:* Computes  $C = \alpha AA^H$ , where the scalar  $\alpha$  is *real-if-needed*(alpha).

```

template<scalar Scalar, in-matrix InMat1, in-matrix InMat2, possibly-packed-out-matrix OutMat,
 class Triangle>
void symmetric_matrix_rank_k_update(Scalar alpha, InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, scalar Scalar, in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_k_update(ExecutionPolicy&& exec, Scalar alpha, InMat1 A, InMat2 E,
 OutMat C, Triangle t);

```

10 *Effects:* Computes  $C = E + \alpha AA^T$ , where the scalar  $\alpha$  is alpha.

```

template<scalar Scalar, in-matrix InMat1, in-matrix InMat2, possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_k_update(Scalar alpha, InMat1 A, InMat2 E, OutMat C, Triangle t);
template<class ExecutionPolicy, scalar Scalar, in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_k_update(ExecutionPolicy&& exec, Scalar alpha, InMat1 A, InMat2 E,
 OutMat C, Triangle t);

```

11 *Effects:* Computes  $C = E + \alpha AA^H$ , where the scalar  $\alpha$  is *real-if-needed*(alpha).

#### 29.9.15.5 Rank-2k update of a symmetric or Hermitian matrix [linalg.algs.blas3.rank2k]

1 [Note 1: These functions correspond to the BLAS functions xSYR2K and xHER2K<sup>[18]</sup>. — end note]

2 The following elements apply to all functions in 29.9.15.5.

3 For any function F in this subclause with a parameter named **t**, an **InMat3** template parameter, and a function parameter **InMat3 E**, **t** applies to accesses done through the parameter **E**. F only accesses the triangle of **E** specified by **t**. For accesses of diagonal elements **E[i, i]**, F only uses the value *real-if-needed*(**E[i, i]**) if the name of F starts with **hermitian**. For accesses **E[i, j]** outside the triangle specified by **t**, F only uses the value

(3.1) — *conj-if-needed*(**E[j, i]**) if the name of F starts with **hermitian**, or

(3.2) — **E[j, i]** if the name of F starts with **symmetric**.

4 *Mandates:*

(4.1) — If **OutMat** has **layout\_blas\_packed** layout, then the layout's **Triangle** template argument has the same type as the function's **Triangle** template argument;

(4.2) — If the function has an **InMat3** template parameter and if **InMat3** has **layout\_blas\_packed** layout, then the layout's **Triangle** template argument has the same type as the function's **Triangle** template argument;

(4.3) — *possibly-multipliable*<decltype(A), decltype(transposed(B)), decltype(C)>() is true;

(4.4) — *possibly-multipliable*<decltype(B), decltype(transposed(A)), decltype(C)>() is true; and

(4.5) — *possibly-addable*<decltype(C), decltype(E), decltype(C)>() is true for those overloads with an **E** parameter.

5 *Preconditions:*

(5.1) — *multipliable*(A, transposed(B), C) is true,

(5.2) — *multipliable*(B, transposed(A), C) is true, and

[Note 2: This and the previous imply that C is square. — end note]

(5.3) — *addable*(C, E, C) is true for those overloads with an **E** parameter.

6 *Complexity:*  $\mathcal{O}(A.\text{extent}(0) \times A.\text{extent}(1) \times B.\text{extent}(0))$ .

7 *Remarks:* C may alias E for those overloads with an **E** parameter.

```

template<in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, OutMat C, Triangle t);
template<class ExecutionPolicy, in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, OutMat C, Triangle t);

```

8 *Effects:* Computes  $C = AB^T + BA^T$ .

```

template<in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, OutMat C, Triangle t);
template<class ExecutionPolicy,
 in-matrix InMat1, in-matrix InMat2,
 possibly-packed-out-matrix OutMat, class Triangle>
void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, OutMat C, Triangle t);

```

9 *Effects:* Computes  $C = AB^H + BA^H$ .

```

template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);
template<class ExecutionPolicy, in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
 possibly-packed-out-matrix OutMat, class Triangle>
void symmetric_matrix_rank_2k_update(ExecutionPolicy&& exec,
 InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);

```

10 *Effects:* Computes  $C = E + AB^T + BA^T$ .

```

template<in-matrix InMat1, in-matrix InMat2, in-matrix InMat3, possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_2k_update(InMat1 A, InMat2 B, InMat3 E, OutMat C, Triangle t);
template<class ExecutionPolicy, in-matrix InMat1, in-matrix InMat2, in-matrix InMat3,
 possibly-packed-out-matrix OutMat,
 class Triangle>
void hermitian_matrix_rank_2k_update(ExecutionPolicy&& exec, InMat1 A, InMat2 B, InMat3 E,
 OutMat C, Triangle t);

```

11 *Effects:* Computes  $C = E + AB^H + BA^H$ .

### 29.9.15.6 Solve multiple triangular linear systems

[linalg.algs.blas3.trsm]

1 [Note 1: These functions correspond to the BLAS function xTRSM<sup>[18]</sup>. — end note]

```

template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X, BinaryDivideOp divide);
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X, BinaryDivideOp divide);

```

2 These functions perform multiple matrix solves, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

3 *Mandates:*

- (3.1) — If `InMat1` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (3.2) — `possibly-multipliable<InMat1, OutMat, InMat2>()` is true; and
- (3.3) — `compatible-static-extents<InMat1, InMat1>(0, 1)` is true.

4 *Preconditions:*

- (4.1) — *multipliable*(A, X, B) is true, and
- (4.2) — A.extent(0) == A.extent(1) is true.

5 *Effects:* Computes  $X'$  such that  $AX' = B$ , and assigns each element of  $X'$  to the corresponding element of  $X$ . If no such  $X'$  exists, then the elements of  $X$  are valid but unspecified.

6 *Complexity:*  $\mathcal{O}(A.extent(0) \times B.extent(1) \times B.extent(1))$ .

7 [Note 2: Since the triangular matrix is on the left, the desired `divide` implementation in the case of noncommutative multiplication is mathematically equivalent to  $y^{-1}x$ , where  $x$  is the first argument and  $y$  is the second argument, and  $y^{-1}$  denotes the multiplicative inverse of  $y$ . — end note]

```
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_matrix_left_solve(InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X);
```

8 *Effects:* Equivalent to:

```
triangular_matrix_matrix_left_solve(A, t, d, B, X, divides<void>{});
```

```
template<class ExecutionPolicy, in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X);
```

9 *Effects:* Equivalent to:

```
triangular_matrix_matrix_left_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, B, X, divides<void>{});
```

```
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X, BinaryDivideOp divide);
```

```
template<class ExecutionPolicy,
 in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat, class BinaryDivideOp>
void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X, BinaryDivideOp divide);
```

10 These functions perform multiple matrix solves, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix A (29.9.3).

11 *Mandates:*

- (11.1) — If `InMat1` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (11.2) — *possibly-multipliable*<OutMat, InMat1, InMat2>() is true; and
- (11.3) — *compatible-static-extents*<InMat1, InMat1>(0,1) is true.

12 *Preconditions:*

- (12.1) — *multipliable*(X, A, B) is true, and
- (12.2) — A.extent(0) == A.extent(1) is true.

13 *Effects:* Computes  $X'$  such that  $X'A = B$ , and assigns each element of  $X'$  to the corresponding element of  $X$ . If no such  $X'$  exists, then the elements of  $X$  are valid but unspecified.

14 *Complexity:*  $\mathcal{O}(B.extent(0) \cdot B.extent(1) \cdot A.extent(1))$

[Note 3: Since the triangular matrix is on the right, the desired `divide` implementation in the case of noncommutative multiplication is mathematically equivalent to  $xy^{-1}$ , where  $x$  is the first argument and  $y$  is the second argument, and  $y^{-1}$  denotes the multiplicative inverse of  $y$ . — end note]

```
template<in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_matrix_right_solve(InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X);
```

15 *Effects:* Equivalent to:

```
triangular_matrix_matrix_right_solve(A, t, d, B, X, divides<void>{});
```

```
template<class ExecutionPolicy, in-matrix InMat1, class Triangle, class DiagonalStorage,
 in-matrix InMat2, out-matrix OutMat>
void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
 InMat1 A, Triangle t, DiagonalStorage d,
 InMat2 B, OutMat X);
```

16 *Effects:* Equivalent to:

```
triangular_matrix_matrix_right_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, B, X, divides<void>{});
```

### 29.9.15.7 Solve multiple triangular linear systems in-place [linalg.algs.blas3.inplace\_trsm]

1 [Note 1: These functions correspond to the BLAS function xTRSM<sup>[18]</sup>. — end note]

```
template<in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(InMat A, Triangle t, DiagonalStorage d,
 InOutMat B, BinaryDivideOp divide);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-matrix InOutMat, class BinaryDivideOp>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InOutMat B, BinaryDivideOp divide);
```

2 These functions perform multiple in-place matrix solves, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

[Note 2: This algorithm makes it possible to compute factorizations like Cholesky and LU in place. Performing triangular solve in place hinders parallelization. However, other `ExecutionPolicy` specific optimizations, such as vectorization, are still possible. — end note]

3 *Mandates:*

- (3.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (3.2) — `possibly-multipliable<InMat, InOutMat, InOutMat>()` is true; and
- (3.3) — `compatible-static-extents<InMat, InMat>(0, 1)` is true.

4 *Preconditions:*

- (4.1) — `multipliable(A, B, B)` is true, and
- (4.2) — `A.extent(0) == A.extent(1)` is true.

5 *Effects:* Computes  $X'$  such that  $AX' = B$ , and assigns each element of  $X'$  to the corresponding element of  $B$ . If so such  $X'$  exists, then the elements of  $B$  are valid but unspecified.

6 *Complexity:*  $\mathcal{O}(B.extent(0) \times A.extent(0) \times A.extent(1))$ .

```
template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_matrix_left_solve(InMat A, Triangle t, DiagonalStorage d,
 InOutMat B);
```

7 *Effects:* Equivalent to:

```
triangular_matrix_matrix_left_solve(A, t, d, B, divides<void>{});
```

```
template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
void triangular_matrix_matrix_left_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InOutMat B);
```

8 *Effects:* Equivalent to:

```

 triangular_matrix_matrix_left_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, B, divides<void>{});

template<in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-matrix InOutMat, class BinaryDivideOp>
 void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d,
 InOutMat B, BinaryDivideOp divide);
template<class ExecutionPolicy, in-matrix InMat, class Triangle, class DiagonalStorage,
 inout-matrix InOutMat, class BinaryDivideOp>
 void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InOutMat B, BinaryDivideOp divide);

```

9 These functions perform multiple in-place matrix solves, taking into account the `Triangle` and `DiagonalStorage` parameters that apply to the triangular matrix `A` (29.9.3).

[*Note 3:* This algorithm makes it possible to compute factorizations like Cholesky and LU in place. Performing triangular solve in place hinders parallelization. However, other `ExecutionPolicy` specific optimizations, such as vectorization, are still possible. — *end note*]

10 *Mandates:*

- (10.1) — If `InMat` has `layout_blas_packed` layout, then the layout's `Triangle` template argument has the same type as the function's `Triangle` template argument;
- (10.2) — `possibly-multipliable<InOutMat, InMat, InOutMat>()` is true; and
- (10.3) — `compatible-static-extents<InMat, InMat>(0, 1)` is true.

11 *Preconditions:*

- (11.1) — `multipliable(B, A, B)` is true, and
- (11.2) — `A.extent(0) == A.extent(1)` is true.

12 *Effects:* Computes  $X'$  such that  $X'A = B$ , and assigns each element of  $X'$  to the corresponding element of  $B$ . If so such  $X'$  exists, then the elements of  $B$  are valid but unspecified.

13 *Complexity:*  $\mathcal{O}(A.extent(0) \times A.extent(1) \times B.extent(1))$ .

```

template<in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
 void triangular_matrix_matrix_right_solve(InMat A, Triangle t, DiagonalStorage d, InOutMat B);

```

14 *Effects:* Equivalent to:

```

 triangular_matrix_matrix_right_solve(A, t, d, B, divides<void>{});

template<class ExecutionPolicy,
 in-matrix InMat, class Triangle, class DiagonalStorage, inout-matrix InOutMat>
 void triangular_matrix_matrix_right_solve(ExecutionPolicy&& exec,
 InMat A, Triangle t, DiagonalStorage d,
 InOutMat B);

```

15 *Effects:* Equivalent to:

```

 triangular_matrix_matrix_right_solve(std::forward<ExecutionPolicy>(exec),
 A, t, d, B, divides<void>{});

```

## 29.10 Data-parallel types

[simd]

### 29.10.1 General

[simd.general]

1 Subclause 29.10 defines data-parallel types and operations on these types.

[*Note 1:* The intent is to support acceleration through data-parallel execution resources where available, such as SIMD registers and instructions or execution units driven by a common instruction decoder. SIMD stands for “Single Instruction Stream – Multiple Data Stream”; it is defined in Flynn 1966<sup>[21]</sup>. — *end note*]

2 The set of *vectorizable types* comprises

- (2.1) — all standard integer types, character types, and the types `float` and `double` (6.9.3);
- (2.2) — `std::float16_t`, `std::float32_t`, and `std::float64_t` if defined (6.9.4); and
- (2.3) — `complex<T>` where  $T$  is a vectorizable floating-point type.



- <sup>3</sup> The term *data-parallel type* refers to all enabled specializations of the `basic_vec` and `basic_mask` class templates. A *data-parallel object* is an object of data-parallel type.
- <sup>4</sup> Each specialization of `basic_vec` or `basic_mask` is either enabled or disabled, as described in 29.10.7.1 and 29.10.9.1.
- <sup>5</sup> A data-parallel type consists of one or more elements of an underlying vectorizable type, called the *element type*. The number of elements is a constant for each data-parallel type and called the *width* of that type. The elements in a data-parallel type are indexed from 0 to width – 1.
- <sup>6</sup> An *element-wise operation* applies a specified operation to the elements of one or more data-parallel objects. Each such application is unsequenced with respect to the others. A *unary element-wise operation* is an element-wise operation that applies a unary operation to each element of a data-parallel object. A *binary element-wise operation* is an element-wise operation that applies a binary operation to corresponding elements of two data-parallel objects.
- <sup>7</sup> Given a `basic_mask<Bytes, Abi>` object `mask`, the *selected indices* signify the integers *i* in the range `[0, mask.size())` for which `mask[i]` is `true`. Given a data-parallel object `data`, the *selected elements* signify the elements `data[i]` for all selected indices *i*.
- <sup>8</sup> The conversion from an arithmetic type *U* to a vectorizable type *T* is *value-preserving* if all possible values of *U* can be represented with type *T*.
- <sup>9</sup> If a program declares an explicit or partial specialization of any of the templates specified in subclause 29.10, the program is ill-formed, no diagnostic required.

### 29.10.2 Exposition-only types, variables, and concepts

[simd.expos]

```

using simd-size-type = see below; // exposition only
template<size_t Bytes> using integer-from = see below; // exposition only

template<class T, class Abi>
 constexpr simd-size-type simd-size-v = see below; // exposition only
template<size_t Bytes, class Abi>
 constexpr simd-size-type mask-size-v = see below; // exposition only
template<class T> constexpr size_t mask-element-size = see below; // exposition only

template<class From, class To>
 concept explicitly-convertible-to = // exposition only
 requires {
 static_cast<To>(declval<From>());
 };

template<class T> using deduced-vec-t = see below; // exposition only

template<class V>
 concept simd-vec-type = // exposition only
 same_as<V, basic_vec<typename V::value_type, typename V::abi_type>>> &&
 is_default_constructible_v<V>;

template<class V>
 concept simd-mask-type = // exposition only
 same_as<V, basic_mask<mask-element-size<V>, typename V::abi_type>>> &&
 is_default_constructible_v<V>;

template<class V>
 concept simd-floating-point = // exposition only
 simd-vec-type<V> && floating_point<typename V::value_type>;

template<class V>
 concept simd-integral = // exposition only
 simd-vec-type<V> && integral<typename V::value_type>;

template<class V>
 using simd-complex-value-type = V::value_type::value_type; // exposition only

```

```

template<class V>
concept simd-complex = // exposition only
 simd-vec-type<V> && same_as<typename V::value_type, complex<simd-complex-value-type<V>>>;

template<class T>
concept math-floating-point = // exposition only
 simd-floating-point<deduced-vec-t<T>>;

template<class BinaryOperation, class T>
concept reduction-binary-operation = see below; // exposition only

// 29.10.2.2, simd ABI tags
template<class T> using native-abi = see below; // exposition only
template<class T, simd-size-type N> using deduce-abi-t = see below; // exposition only

// 29.10.5, load and store flags
struct convert-flag; // exposition only
struct aligned-flag; // exposition only
template<size_t N> struct overaligned-flag; // exposition only

```

### 29.10.2.1 Exposition-only helpers

[simd.expos.defn]

using *simd-size-type* = *see below*;

1 *simd-size-type* is an alias for a signed integer type.

template<size\_t Bytes> using *integer-from* = *see below*;

2 *integer-from*<Bytes> is an alias for a signed integer type T such that sizeof(T) equals Bytes.

```

template<class T, class Abi>
constexpr simd-size-type simd-size-v = see below;

```

3 *simd-size-v*<T, Abi> denotes the width of `basic_vec<T, Abi>` if the specialization `basic_vec<T, Abi>` is enabled, or 0 otherwise.

```

template<size_t Bytes, class Abi>
constexpr simd-size-type mask-size-v = see below;

```

4 *mask-size-v*<Bytes, Abi> denotes the width of `basic_mask<Bytes, Abi>` if the specialization `basic_mask<Bytes, Abi>` is enabled, or 0 otherwise.

template<class T> constexpr size\_t *mask-element-size* = *see below*;

5 *mask-element-size*<`basic_mask<Bytes, Abi>`> has the value Bytes.

template<class T> using *deduced-vec-t* = *see below*;

6 Let x denote an lvalue of type `const T`.

7 *deduced-vec-t*<T> is an alias for `decltype(x + x)`.

```

template<class BinaryOperation, class T>
concept reduction-binary-operation =
 requires (const BinaryOperation binary_op, const vec<T, 1> v) {
 { binary_op(v, v) } -> same_as<vec<T, 1>>;
 };

```

8 Types `BinaryOperation` and `T` model *reduction-binary-operation*<`BinaryOperation`, `T`> only if:

(8.1) — `BinaryOperation` is a binary element-wise operation and the operation is commutative.

(8.2) — An object of type `BinaryOperation` can be invoked with two arguments of type `basic_vec<T, Abi>`, with unspecified ABI tag `Abi`, returning a `basic_vec<T, Abi>`.

### 29.10.2.2 simd ABI tags

[simd.expos.abi]

template<class T> using *native-abi* = *see below*;



```
template<class T, simd-size-type N> using deduce-abi-t = see below;
```

1 An *ABI tag* is a type that indicates a choice of size and binary representation for objects of data-parallel type.

[*Note 1*: The intent is for the size and binary representation to depend on the target architecture and compiler flags. The ABI tag, together with a given element type, implies the width. — *end note*]

2 [*Note 2*: The ABI tag is orthogonal to selecting the machine instruction set. The selected machine instruction set limits the usable ABI tag types, though (see 29.10.7.1). The ABI tags enable users to safely pass objects of data-parallel type between translation unit boundaries (e.g., function calls or I/O). — *end note*]

3 An implementation defines ABI tag types as necessary for the following aliases.

4 *deduce-abi-t*<T, N> names an ABI tag type if and only if

(4.1) — T is a vectorizable type,

(4.2) — N is greater than zero, and

(4.3) — N is not larger than an implementation-defined maximum.

Otherwise, *deduce-abi-t*<T, N> names an unspecified type. The implementation-defined maximum for N is not smaller than 64 and can differ depending on T.

5 If *deduce-abi-t*<T, N> names an ABI tag type, the following is true:

(5.1) — *simd-size-v*<T, *deduce-abi-t*<T, N>> equals N, and

(5.2) — *basic\_vec*<T, *deduce-abi-t*<T, N>> is enabled (29.10.7.1).

6 *native-abi*<T> is an implementation-defined alias for an ABI tag. *basic\_vec*<T, *native-abi*<T>> is an enabled specialization.

[*Note 3*: The intent is to use the ABI tag producing the most efficient data-parallel execution for the element type T on the currently targeted system. For target architectures with ISA extensions, compiler flags can change the type of the *native-abi*<T> alias. — *end note*]

[*Example 1*: Consider a target architecture supporting the ABI tags `__simd128` and `__simd256`, where hardware support for `__simd256` exists only for floating-point types. The implementation therefore defines *native-abi*<T> as an alias for

(6.1) — `__simd256` if T is a floating-point type, and

(6.2) — `__simd128` otherwise.

— *end example*]

### 29.10.3 Header <simd> synopsis

[*simd.syn*]

```
namespace std::simd {
 // 29.10.4, type traits
 template<class T, class U = typename T::value_type> struct alignment;
 template<class T, class U = typename T::value_type>
 constexpr size_t alignment_v = alignment<T, U>::value;

 template<class T, class V> struct rebind { using type = see below; };
 template<class T, class V> using rebind_t = rebind<T, V>::type;
 template<simd-size-type N, class V> struct resize { using type = see below; };
 template<simd-size-type N, class V> using resize_t = resize<N, V>::type;

 // 29.10.5, load and store flags
 template<class... Flags> struct flags;
 inline constexpr flags<> flag_default{};
 inline constexpr flags<convert-flag> flag_convert{};
 inline constexpr flags<aligned-flag> flag_aligned{};
 template<size_t N> requires (has_single_bit(N))
 constexpr flags<overaligned-flag<N>> flag_overaligned{};

 // 29.10.6, class template simd-iterator
 template<class V>
 class simd-iterator; // exposition only
}
```

```

// 29.10.7, class template basic_vec
template<class T, class Abi = native_abi<T>> class basic_vec;
template<class T, simd_size_type N = simd_size_v<T, native_abi<T>>>
 using vec = basic_vec<T, deduce_abi_t<T, N>>;

// 29.10.8.5, reductions
template<class T, class Abi, class BinaryOperation = plus<>>
 constexpr T reduce(const basic_vec<T, Abi>&, BinaryOperation = {});
template<class T, class Abi, class BinaryOperation = plus<>>
 constexpr T reduce(
 const basic_vec<T, Abi>& x, const typename basic_vec<T, Abi>::mask_type& mask,
 BinaryOperation binary_op = {}, type_identity_t<T> identity_element = see below);

template<class T, class BinaryOperation = plus<>>
 constexpr T reduce(const T&, BinaryOperation = {});
template<class T, class BinaryOperation = plus<>>
 constexpr T reduce(const T& x, same_as<bool> auto mask, BinaryOperation binary_op = {},
 type_identity_t<T> identity_element = see below);

template<class T, class Abi>
 constexpr T reduce_min(const basic_vec<T, Abi>&) noexcept;
template<class T, class Abi>
 constexpr T reduce_min(const basic_vec<T, Abi>&,
 const typename basic_vec<T, Abi>::mask_type&) noexcept;
template<class T, class Abi>
 constexpr T reduce_max(const basic_vec<T, Abi>&) noexcept;
template<class T, class Abi>
 constexpr T reduce_max(const basic_vec<T, Abi>&,
 const typename basic_vec<T, Abi>::mask_type&) noexcept;

template<class T> constexpr T reduce_min(const T&) noexcept;
template<class T> constexpr T reduce_min(const T&, same_as<bool> auto) noexcept;
template<class T> constexpr T reduce_max(const T&) noexcept;
template<class T> constexpr T reduce_max(const T&, same_as<bool> auto) noexcept;

// 29.10.8.6, load and store functions
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_load(R&& r, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_load(R&& r, const typename V::mask_type& k,
 flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V unchecked_load(I first, iter_difference_t<I> n,
 flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V unchecked_load(I first, iter_difference_t<I> n,
 const typename V::mask_type& k, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V unchecked_load(I first, S last, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V unchecked_load(I first, S last, const typename V::mask_type& k,
 flags<Flags...> f = {});

template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_load(R&& r, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_load(R&& r, const typename V::mask_type& k,
 flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V partial_load(I first, iter_difference_t<I> n, flags<Flags...> f = {});

```

```

template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V partial_load(I first, iter_difference_t<I> n,
 const typename V::mask_type& k, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V partial_load(I first, S last, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V partial_load(I first, S last, const typename V::mask_type& k,
 flags<Flags...> f = {});

template<class T, class Abi, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
 flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
 iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first,
 iter_difference_t<I> n, const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});

template<class T, class Abi, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
 flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
 constexpr void partial_store(
 const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
 constexpr void partial_store(
 const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
 const typename basic_vec<T, Abi>::mask_type& mask, flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});

// 29.10.8.7, static permute
inline constexpr simd-size-type zero_element = implementation-defined;
inline constexpr simd-size-type uninit_element = implementation-defined;

template<simd-size-type N = see below, simd-vec-type V, class IdxMap>
 constexpr resize_t<N, V> permute(const V& v, IdxMap&& idxmap);

```

```

template<simd-size-type N = see below, simd-mask-type V, class IdxMap>
 constexpr resize_t<N, V> permute(const V& v, IdxMap&& idxmap);

// 29.10.8.8, dynamic permute
template<simd-vec-type V, simd-integral I>
 constexpr resize_t<I::size(), V> permute(const V& v, const I& indices);
template<simd-mask-type V, simd-integral I>
 constexpr resize_t<I::size(), V> permute(const V& v, const I& indices);

// 29.10.8.9, mask permute
template<simd-vec-type V>
 constexpr V compress(const V& v, const typename V::mask_type& selector);
template<simd-mask-type V>
 constexpr V compress(const V& v, const type_identity_t<V>& selector);
template<simd-vec-type V>
 constexpr V compress(const V& v, const typename V::mask_type& selector,
 const typename V::value_type& fill_value);
template<simd-mask-type V>
 constexpr V compress(const V& v, const type_identity_t<V>& selector,
 const typename V::value_type& fill_value);

template<simd-vec-type V>
 constexpr V expand(const V& v, const typename V::mask_type& selector,
 const V& original = {});
template<simd-mask-type V>
 constexpr V expand(const V& v, const type_identity_t<V>& selector,
 const V& original = {});

// 29.10.8.10, memory permute
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_gather_from(R&& in, const I& indices, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_gather_from(R&& in, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});

template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_gather_from(R&& in, const I& indices, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_gather_from(R&& in, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});

template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr void unchecked_scatter_to(const V& v, R&& out,
 const I& indices, flags<Flags...> f = {});
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr void unchecked_scatter_to(const V& v, R&& out, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});

template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr void partial_scatter_to(const V& v, R&& out,
 const I& indices, flags<Flags...> f = {});
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
 requires ranges::sized_range<R>
 constexpr void partial_scatter_to(const V& v, R&& out, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});

```

// 29.10.8.11, creation

```
template<class T, class Abi>
 constexpr auto chunk(const basic_vec<typename T::value_type, Abi>& x) noexcept;
template<class T, class Abi>
 constexpr auto chunk(const basic_mask<mask-element-size<T>, Abi>& x) noexcept;

template<simd-size-type N, class T, class Abi>
 constexpr auto chunk(const basic_vec<T, Abi>& x) noexcept;
template<simd-size-type N, size_t Bytes, class Abi>
 constexpr auto chunk(const basic_mask<Bytes, Abi>& x) noexcept;

template<class T, class... Abis>
 constexpr resize_t<(basic_vec<T, Abis>::size() + ...),
 basic_vec<T, Abis...[0]>> cat(const basic_vec<T, Abis>&...) noexcept;
template<size_t Bytes, class... Abis>
 constexpr resize_t<(basic_mask<Bytes, Abis>::size() + ...),
 basic_mask<Bytes, Abis...[0]>> cat(const basic_mask<Bytes, Abis>&...) noexcept;

template<class T> constexpr T iota = see below;
```

// 29.10.8.12, algorithms

```
template<class T, class Abi>
 constexpr basic_vec<T, Abi>
 min(const basic_vec<T, Abi>& a, const basic_vec<T, Abi>& b) noexcept;
template<class T, class Abi>
 constexpr basic_vec<T, Abi>
 max(const basic_vec<T, Abi>& a, const basic_vec<T, Abi>& b) noexcept;
template<class T, class Abi>
 constexpr pair<basic_vec<T, Abi>, basic_vec<T, Abi>>
 minmax(const basic_vec<T, Abi>& a, const basic_vec<T, Abi>& b) noexcept;
template<class T, class Abi>
 constexpr basic_vec<T, Abi>
 clamp(const basic_vec<T, Abi>& v, const basic_vec<T, Abi>& lo,
 const basic_vec<T, Abi>& hi);

template<class T, class U>
 constexpr auto select(bool c, const T& a, const U& b)
 -> remove_cvref_t<decltype(c ? a : b)>;
template<size_t Bytes, class Abi, class T, class U>
 constexpr auto select(const basic_mask<Bytes, Abi>& c, const T& a, const U& b)
 noexcept -> decltype(simd-select-impl(c, a, b));
```

// 29.10.8.13, mathematical functions

```
template<math-floating-point V> constexpr deduced-vec-t<V> acos(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> asin(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> atan(const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> atan2(const V& y, const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> atan2(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> atan2(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V> constexpr deduced-vec-t<V> cos(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> sin(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> tan(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> acosh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> asinh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> atanh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> cosh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> sinh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> tanh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> exp(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> exp2(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> expm1(const V& x);
```

```

template<math-floating-point V>
 constexpr deduced-vec-t<V>
 frexp(const V& value, rebind_t<int, deduced-vec-t<V>>* exp);
template<math-floating-point V>
 constexpr rebind_t<int, deduced-vec-t<V>> ilogb(const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> ldexp(const V& x, const rebind_t<int, deduced-vec-t<V>>& exp);
template<math-floating-point V> constexpr deduced-vec-t<V> log(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log10(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log1p(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log2(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> logb(const V& x);
template<class T, class Abi>
 constexpr basic_vec<T, Abi>
 modf(const type_identity_t<basic_vec<T, Abi>>& value, basic_vec<T, Abi>* iptr);
template<math-floating-point V>
 constexpr deduced-vec-t<V> scalbn(const V& x, const rebind_t<int, deduced-vec-t<V>>& n);
template<math-floating-point V>
 constexpr deduced-vec-t<V> scalbln(
 const V& x, const rebind_t<long int, deduced-vec-t<V>>& n);
template<math-floating-point V> constexpr deduced-vec-t<V> cbrt(const V& x);
template<signed_integral T, class Abi>
 constexpr basic_vec<T, Abi> abs(const basic_vec<T, Abi>& j);
template<math-floating-point V> constexpr deduced-vec-t<V> abs(const V& j);
template<math-floating-point V> constexpr deduced-vec-t<V> fabs(const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const deduced-vec-t<V>& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const deduced-vec-t<V>& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const V& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const deduced-vec-t<V>& x, const deduced-vec-t<V>& y,
 const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const deduced-vec-t<V>& x, const V& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> hypot(const V& x, const deduced-vec-t<V>& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> pow(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> pow(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> pow(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V> constexpr deduced-vec-t<V> sqrt(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> erf(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> erfc(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> lgamma(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> tgamma(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> ceil(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> floor(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> nearbyint(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> rint(const V& x);

```

```

template<math-floating-point V>
 rebind_t<long int, deduced-vec-t<V>> lrint(const V& x);
template<math-floating-point V>
 rebind_t<long long int, V> llrint(const deduced-vec-t<V>& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> round(const V& x);
template<math-floating-point V>
 constexpr rebind_t<long int, deduced-vec-t<V>> lround(const V& x);
template<math-floating-point V>
 constexpr rebind_t<long long int, deduced-vec-t<V>> llround(const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> trunc(const V& x);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmod(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmod(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmod(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> remainder(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> remainder(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> remainder(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 remquo(const V& x, const V& y, rebind_t<int, deduced-vec-t<V>>* quo);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 remquo(const deduced-vec-t<V>& x, const V& y, rebind_t<int, deduced-vec-t<V>>* quo);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 remquo(const V& x, const deduced-vec-t<V>& y, rebind_t<int, deduced-vec-t<V>>* quo);
template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const V& x, const deduced-vec-t<V>& y);

```

```

template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const deduced-vec-t<V>& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const deduced-vec-t<V>& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const V& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const deduced-vec-t<V>& x, const deduced-vec-t<V>& y,
 const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const deduced-vec-t<V>& x, const V& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const deduced-vec-t<V>& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 lerp(const V& a, const V& b, const V& t) noexcept;
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const deduced-vec-t<V>& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const V& x, const deduced-vec-t<V>& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const V& x, const V& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const deduced-vec-t<V>& x, const deduced-vec-t<V>& y,
 const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const deduced-vec-t<V>& x, const V& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> lerp(const V& x, const deduced-vec-t<V>& y,
 const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr rebind_t<int, deduced-vec-t<V>> fpclassify(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isfinite(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isinf(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isnan(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isnormal(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type signbit(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreater(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreater(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreater(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreaterequal(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreaterequal(const deduced-vec-t<V>& x, const V& y);

```



```

template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isgreaterequal(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isless(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isless(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isless(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessequal(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessequal(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessequal(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessgreater(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessgreater(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 islessgreater(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isunordered(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isunordered(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type
 isunordered(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 deduced-vec-t<V> assoc_laguerre(const rebind_t<unsigned, deduced-vec-t<V>>& n,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> assoc_legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> beta(const V& x, const V& y);
template<math-floating-point V>
 deduced-vec-t<V> beta(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 deduced-vec-t<V> beta(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V> deduced-vec-t<V> comp_ellint_1(const V& k);
template<math-floating-point V> deduced-vec-t<V> comp_ellint_2(const V& k);
template<math-floating-point V>
 deduced-vec-t<V> comp_ellint_3(const V& k, const V& nu);
template<math-floating-point V>
 deduced-vec-t<V> comp_ellint_3(const deduced-vec-t<V>& k, const V& nu);
template<math-floating-point V>
 deduced-vec-t<V> comp_ellint_3(const V& k, const deduced-vec-t<V>& nu);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_i(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_i(const deduced-vec-t<V>& nu, const V& x);

```

```

template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_i(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const V& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const deduced-vec-t<V>& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const V& k, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const V& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const deduced-vec-t<V>& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const V& k, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const V& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const V& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const deduced-vec-t<V>& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const V& nu, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const deduced-vec-t<V>& nu,
 const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const V& nu,
 const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const deduced-vec-t<V>& nu,
 const deduced-vec-t<V>& phi);
template<math-floating-point V> deduced-vec-t<V> expint(const V& x);
template<math-floating-point V>
 deduced-vec-t<V> hermite(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> laguerre(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> riemann_zeta(const V& x);
template<math-floating-point V>
 deduced-vec-t<V> sph_bessel(
 const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> sph_legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& theta);

```

```

template<math-floating-point V>
 deduced-vec-t<V>
 sph_neumann(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);

// 29.10.8.14, bit manipulation
template<simd-vec-type V> constexpr V byteswap(const V& v) noexcept;
template<simd-integral V> constexpr V bit_reverse(const V& v) noexcept;
template<simd-vec-type V> constexpr V bit_ceil(const V& v);
template<simd-vec-type V> constexpr V bit_floor(const V& v) noexcept;

template<simd-vec-type V>
 constexpr typename V::mask_type has_single_bit(const V& v) noexcept;

template<simd-integral VX, simd-integral VS>
 constexpr VX shl(const VX& x, const VS& s) noexcept;
template<simd-integral V, class S>
 constexpr V shl(const V& x, S s) noexcept;

template<simd-integral VX, simd-integral VS>
 constexpr VX shr(const VX& x, const VS& s) noexcept;
template<simd-integral V, class S>
 constexpr V shr(const V& x, S s) noexcept;

template<simd-vec-type V0, simd-vec-type V1>
 constexpr V0 rotl(const V0& v, const V1& s) noexcept;
template<simd-vec-type V>
 constexpr V rotl(const V& v, int s) noexcept;

template<simd-vec-type V0, simd-vec-type V1>
 constexpr V0 rotr(const V0& v, const V1& s) noexcept;
template<simd-vec-type V>
 constexpr V rotr(const V& v, int s) noexcept;

template<simd-integral V0, simd-integral V1>
 constexpr V0 bit_repeat(const V0& v, const V1& l);
template<simd-integral V>
 constexpr V bit_repeat(const V& v, int l);

template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 bit_width(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 countl_zero(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 countl_one(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 countr_zero(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 countr_one(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V>
 popcount(const V& v) noexcept;

template<simd-integral V>
 constexpr V bit_compress(const V& v, const V& m) noexcept;
template<simd-integral V>
 constexpr V bit_expand(const V& v, const V& m) noexcept;

template<simd-integral V>
 constexpr V bit_compress(const V& v, typename V::value_type m) noexcept;

```

```

template<simd-integral V>
 constexpr V bit_expand(const V& v, typename V::value_type m) noexcept;

// 29.10.8.15, complex math
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> real(const V&) noexcept;

template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> imag(const V&) noexcept;

template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> abs(const V&);

template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> arg(const V&);

template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> norm(const V&);

template<simd-complex V> constexpr V conj(const V&);
template<simd-complex V> constexpr V proj(const V&);
template<simd-complex V> constexpr V exp(const V& v);
template<simd-complex V> constexpr V log(const V& v);
template<simd-complex V> constexpr V log10(const V& v);

template<simd-complex V> constexpr V sqrt(const V& v);
template<simd-complex V> constexpr V sin(const V& v);
template<simd-complex V> constexpr V asin(const V& v);
template<simd-complex V> constexpr V cos(const V& v);
template<simd-complex V> constexpr V acos(const V& v);
template<simd-complex V> constexpr V tan(const V& v);
template<simd-complex V> constexpr V atan(const V& v);
template<simd-complex V> constexpr V sinh(const V& v);
template<simd-complex V> constexpr V asinh(const V& v);
template<simd-complex V> constexpr V cosh(const V& v);
template<simd-complex V> constexpr V acosh(const V& v);
template<simd-complex V> constexpr V tanh(const V& v);
template<simd-complex V> constexpr V atanh(const V& v);

template<simd-floating-point V>
 rebind_t<complex<typename V::value_type>, V> polar(const V& x, const V& y = {});

template<simd-complex V> constexpr V pow(const V& x, const V& y);

// 29.10.9, class template basic_mask
template<size_t Bytes, class Abi> class basic_mask;
template<class T, simd-size-type N = simd-size-v<T, native-abi<T>>>
 using mask = vec<T, N::mask_type>;

// 29.10.10.5, reductions
template<size_t Bytes, class Abi>
 constexpr bool all_of(const basic_mask<Bytes, Abi>&) noexcept;
template<size_t Bytes, class Abi>
 constexpr bool any_of(const basic_mask<Bytes, Abi>&) noexcept;
template<size_t Bytes, class Abi>
 constexpr bool none_of(const basic_mask<Bytes, Abi>&) noexcept;
template<size_t Bytes, class Abi>
 constexpr simd-size-type reduce_count(const basic_mask<Bytes, Abi>&) noexcept;
template<size_t Bytes, class Abi>
 constexpr simd-size-type reduce_min_index(const basic_mask<Bytes, Abi>&);
template<size_t Bytes, class Abi>
 constexpr simd-size-type reduce_max_index(const basic_mask<Bytes, Abi>&);

```

```

constexpr bool all_of(same_as<bool> auto) noexcept;
constexpr bool any_of(same_as<bool> auto) noexcept;
constexpr bool none_of(same_as<bool> auto) noexcept;
constexpr simd-size-type reduce_count(same_as<bool> auto) noexcept;
constexpr simd-size-type reduce_min_index(same_as<bool> auto);
constexpr simd-size-type reduce_max_index(same_as<bool> auto);
}

```

```

namespace std {
 // See 29.10.8.12, algorithms
 using simd::min;
 using simd::max;
 using simd::minmax;
 using simd::clamp;

 // See 29.10.8.13, mathematical functions
 using simd::acos;
 using simd::asin;
 using simd::atan;
 using simd::atan2;
 using simd::cos;
 using simd::sin;
 using simd::tan;
 using simd::acosh;
 using simd::asinh;
 using simd::atanh;
 using simd::cosh;
 using simd::sinh;
 using simd::tanh;
 using simd::exp;
 using simd::exp2;
 using simd::expm1;
 using simd::frexp;
 using simd::ilogb;
 using simd::ldexp;
 using simd::log;
 using simd::log10;
 using simd::log1p;
 using simd::log2;
 using simd::logb;
 using simd::modf;
 using simd::scalbn;
 using simd::scalbln;
 using simd::cbrt;
 using simd::abs;
 using simd::fabs;
 using simd::hypot;
 using simd::pow;
 using simd::sqrt;
 using simd::erf;
 using simd::erfc;
 using simd::lgamma;
 using simd::tgamma;
 using simd::ceil;
 using simd::floor;
 using simd::nearbyint;
 using simd::rint;
 using simd::lrint;
 using simd::llrint;
 using simd::round;
 using simd::lround;
 using simd::llround;
 using simd::trunc;
 using simd::fmod;
}

```

```

using simd::remainder;
using simd::remquo;
using simd::copysign;
using simd::nextafter;
using simd::fdim;
using simd::fmax;
using simd::fmin;
using simd::fma;
using simd::lerp;
using simd::fpclassify;
using simd::isfinite;
using simd::isinf;
using simd::isnan;
using simd::isnormal;
using simd::signbit;
using simd::isgreater;
using simd::isgreaterequal;
using simd::isless;
using simd::islessequal;
using simd::islessgreater;
using simd::isunordered;
using simd::assoc_laguerre;
using simd::assoc_legendre;
using simd::beta;
using simd::comp_ellint_1;
using simd::comp_ellint_2;
using simd::comp_ellint_3;
using simd::cyl_bessel_i;
using simd::cyl_bessel_j;
using simd::cyl_bessel_k;
using simd::cyl_neumann;
using simd::ellint_1;
using simd::ellint_2;
using simd::ellint_3;
using simd::expint;
using simd::hermite;
using simd::laguerre;
using simd::legendre;
using simd::riemann_zeta;
using simd::sph_bessel;
using simd::sph_legendre;
using simd::sph_neumann;

```

*// See 29.10.8.14, bit manipulation*

```

using simd::byteswap;
using simd::bit_ceil;
using simd::bit_floor;
using simd::bit_reverse;
using simd::has_single_bit;
using simd::shl;
using simd::shr;
using simd::rotr;
using simd::rotr;
using simd::bit_repeat;
using simd::bit_width;
using simd::countl_zero;
using simd::countl_one;
using simd::countr_zero;
using simd::countr_one;
using simd::popcount;
using simd::bit_compress;
using simd::bit_expand;

```

```

// See 29.10.8.15, vec complex math
using simd::real;
using simd::imag;
using simd::arg;
using simd::norm;
using simd::conj;
using simd::proj;
using simd::polar;
}

```

## 29.10.4 Type traits

[simd.traits]

```
template<class T, class U = typename T::value_type> struct alignment { see below };
```

<sup>1</sup> alignment<T, U> has a member `value` if and only if T is a specialization of `basic_vec` and U is a vectorizable type.

<sup>2</sup> If `value` is present, the type `alignment<T, U>` is a `BinaryTypeTrait` with a base characteristic of `integral_constant<size_t, N>` for some unspecified N (29.10.7.2, 29.10.8.6).

[*Note 1*: `value` identifies the alignment restrictions on pointers used for (converting) loads and stores for the given type T on arrays of type U. — *end note*]

```
template<class T, class V> struct rebind { using type = see below; };
```

<sup>3</sup> The member `type` is present if and only if

(3.1) — V is a data-parallel type,

(3.2) — T is a vectorizable type, and

(3.3) — `deduce-abi-t<T, V::size()>` names an ABI tag type.

<sup>4</sup> If V is a specialization of `basic_vec`, let `Abi1` denote an ABI tag such that `basic_vec<T, Abi1>::size()` equals `V::size()`. If V is a specialization of `basic_mask`, let `Abi1` denote an ABI tag such that `basic_mask<sizeof(T), Abi1>::size()` equals `V::size()`.

<sup>5</sup> Where present, the member typedef `type` names `basic_vec<T, Abi1>` if V is a specialization of `basic_vec` or `basic_mask<sizeof(T), Abi1>` if V is a specialization of `basic_mask`.

```
template<simd-size-type N, class V> struct resize { using type = see below; };
```

<sup>6</sup> Let `Abi1` denote an ABI tag

(6.1) — such that `simd-size-v<typename V::value_type, Abi1>` equals N if V is a specialization of `basic_vec`,

(6.2) — otherwise such that `mask-size-v<mask-element-size<V>, Abi1>` equals N if V is a specialization of `basic_mask`.

<sup>7</sup> The member `type` is present if and only if

(7.1) — V is a data-parallel type, and

(7.2) — there exists at least one ABI tag that satisfies the above constraints for `Abi1`.

<sup>8</sup> Where present, the member typedef `type` names `basic_vec<typename V::value_type, Abi1>` if V is a specialization of `basic_vec`, or `basic_mask<mask-element-size<V>, Abi1>` if V is a specialization of `basic_mask`.

## 29.10.5 Load and store flags

[simd.flags]

### 29.10.5.1 Class template flags overview

[simd.flags.overview]

```

namespace std::simd {
 template<class... Flags> struct flags {
 // 29.10.5.2, flags operators
 template<class... Other>
 friend constexpr auto operator|(flags, flags<Other...>);
 };
}

```

<sup>1</sup> [*Note 1*: The class template `flags` acts like an integer bit-flag for types. — *end note*]

- <sup>2</sup> *Constraints:* Every type in the parameter pack `Flags` is one of *convert-flag*, *aligned-flag*, or *over-aligned-flag*<N>.

### 29.10.5.2 flags operators

[simd.flags.oper]

```
template<class... Other>
 friend constexpr auto operator|(flags a, flags<Other...> b);
```

- <sup>1</sup> *Returns:* A default-initialized object of type `flags<Flags2...>` for some `Flags2` where every type in `Flags2` is present either in template parameter pack `Flags` or in template parameter pack `Other`, and every type in template parameter packs `Flags` and `Other` is present in `Flags2`. If the packs `Flags` and `Other` contain two different specializations *overaligned-flag*<N1> and *overaligned-flag*<N2>, `Flags2` is not required to contain the specialization *overaligned-flag*<std::min(N1, N2)>.

### 29.10.6 Class template *simd-iterator*

[simd.iterator]

```
namespace std::simd {
 template<class V>
 class simd-iterator {
 V* data_ = nullptr;
 simd-size-type offset_ = 0;

 constexpr simd-iterator(V& d, simd-size-type off) noexcept;

 public:
 using value_type = V::value_type;
 using iterator_category = input_iterator_tag;
 using iterator_concept = random_access_iterator_tag;
 using difference_type = simd-size-type;

 constexpr simd-iterator() = default;

 constexpr simd-iterator(const simd-iterator&) = default;
 constexpr simd-iterator& operator=(const simd-iterator&) = default;

 constexpr simd-iterator(const simd-iterator<remove_const_t<V>>&) requires is_const_v<V>;

 constexpr value_type operator*() const;

 constexpr simd-iterator& operator++();
 constexpr simd-iterator operator++(int);
 constexpr simd-iterator& operator--();
 constexpr simd-iterator operator--(int);

 constexpr simd-iterator& operator+=(difference_type n);
 constexpr simd-iterator& operator-=(difference_type n);

 constexpr value_type operator[](difference_type n) const;

 friend constexpr bool operator==(simd-iterator a, simd-iterator b) = default;
 friend constexpr bool operator==(simd-iterator a, default_sentinel_t) noexcept;
 friend constexpr auto operator<=>(simd-iterator a, simd-iterator b);

 friend constexpr simd-iterator operator+(simd-iterator i, difference_type n);
 friend constexpr simd-iterator operator+(difference_type n, simd-iterator i);
 friend constexpr simd-iterator operator-(simd-iterator i, difference_type n);

 friend constexpr difference_type operator-(simd-iterator a, simd-iterator b);
 friend constexpr difference_type operator-(simd-iterator i, default_sentinel_t) noexcept;
 friend constexpr difference_type operator-(default_sentinel_t, simd-iterator i) noexcept;
 };
}
```

```
constexpr simd-iterator(V& d, simd-size-type off) noexcept;
```

- <sup>1</sup> *Effects:* Initializes `data_` with `addressof(d)` and `offset_` with `off`.



```

constexpr simd-iterator(const simd-iterator<remove_const_t<V>>& i) requires is_const_v<V>;
2 Effects: Initializes data_ with i.data_ and offset_ with i.offset_.

constexpr value_type operator*() const;
3 Effects: Equivalent to: return (*data_)[offset_];

constexpr simd-iterator& operator++();
4 Effects: Equivalent to: return *this += 1;

constexpr simd-iterator operator++(int);
5 Effects: Equivalent to:
 simd-iterator tmp = *this;
 *this += 1;
 return tmp;

constexpr simd-iterator& operator--();
6 Effects: Equivalent to: return *this -= 1;

constexpr simd-iterator operator--(int);
7 Effects: Equivalent to:
 simd-iterator tmp = *this;
 *this -= 1;
 return tmp;

constexpr simd-iterator& operator+=(difference_type n);
8 Preconditions: offset_ + n is in the range [0, V::size()].
9 Effects: Equivalent to:
 offset_ += n;
 return *this;

constexpr simd-iterator& operator-=(difference_type n);
10 Preconditions: offset_ - n is in the range [0, V::size()].
11 Effects: Equivalent to:
 offset_ -= n;
 return *this;

constexpr value_type operator[](difference_type n) const;
12 Effects: Equivalent to: return (*data_)[offset_ + n];

friend constexpr bool operator==(simd-iterator i, default_sentinel_t) noexcept;
13 Effects: Equivalent to: return i.offset_ == V::size();

friend constexpr auto operator<=>(simd-iterator a, simd-iterator b);
14 Preconditions: a.data_ == b.data_ is true.
15 Effects: Equivalent to: return a.offset_ <=> b.offset_;

friend constexpr simd-iterator operator+(simd-iterator i, difference_type n);
friend constexpr simd-iterator operator+(difference_type n, simd-iterator i);
16 Effects: Equivalent to: return i += n;

friend constexpr simd-iterator operator-(simd-iterator i, difference_type n);
17 Effects: Equivalent to: return i -= n;

friend constexpr difference_type operator-(simd-iterator a, simd-iterator b);
18 Preconditions: a.data_ == b.data_ is true.
19 Effects: Equivalent to: return a.offset_ - b.offset_;

```

```
friend constexpr difference_type operator-(simd-iterator i, default_sentinel_t) noexcept;
```

20     *Effects:* Equivalent to: return `i.offset_ - V::size()`;

```
friend constexpr difference_type operator-(default_sentinel_t, simd-iterator i) noexcept;
```

21     *Effects:* Equivalent to: return `V::size() - i.offset_`;

## 29.10.7 Class template `basic_vec`

[simd.class]

### 29.10.7.1 Overview

[simd.overview]

```
namespace std::simd {
 template<class T, class Abi> class basic_vec {
 using real-type = see below; // exposition only
 public:
 using value_type = T;
 using mask_type = basic_mask<sizeof(T), Abi>;
 using abi_type = Abi;
 using iterator = simd-iterator<basic_vec>;
 using const_iterator = simd-iterator<const basic_vec>;

 constexpr iterator begin() noexcept { return {*this, 0}; }
 constexpr const_iterator begin() const noexcept { return {*this, 0}; }
 constexpr const_iterator cbegin() const noexcept { return {*this, 0}; }
 constexpr default_sentinel_t end() const noexcept { return {}; }
 constexpr default_sentinel_t cend() const noexcept { return {}; }

 static constexpr integral_constant<simd-size-type, simd-size-v<T, Abi>> size {};

 constexpr basic_vec() noexcept = default;

 // 29.10.7.2, basic_vec constructors
 template<class U>
 constexpr basic_vec(U&& value) noexcept;
 template<class U, class UAbi>
 constexpr explicit(see below) basic_vec(const basic_vec<U, UAbi>&) noexcept;
 template<class G>
 constexpr explicit basic_vec(G&& gen);
 template<class R, class... Flags>
 constexpr basic_vec(R&& range, flags<Flags...> = {});
 template<class R, class... Flags>
 constexpr basic_vec(R&& range, const mask_type& mask, flags<Flags...> = {});
 constexpr basic_vec(const real-type& reals, const real-type& imags = {}) noexcept;

 // 29.10.7.3, basic_vec subscript operators
 constexpr value_type operator[](simd-size-type) const;
 template<simd-integral I>
 constexpr resize_t<I::size(), basic_vec> operator[](const I& indices) const;

 // 29.10.7.4, basic_vec complex accessors
 constexpr real-type real() const noexcept;
 constexpr real-type imag() const noexcept;
 constexpr void real(const real-type& v) noexcept;
 constexpr void imag(const real-type& v) noexcept;

 // 29.10.7.5, basic_vec unary operators
 constexpr basic_vec& operator++() noexcept;
 constexpr basic_vec operator++(int) noexcept;
 constexpr basic_vec& operator--() noexcept;
 constexpr basic_vec operator--(int) noexcept;
 constexpr mask_type operator!() const noexcept;
 constexpr basic_vec operator~() const noexcept;
 constexpr basic_vec operator+() const noexcept;
 constexpr basic_vec operator-() const noexcept;
```

```

// 29.10.8.1, basic_vec binary operators
friend constexpr basic_vec operator+(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator-(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator*(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator/(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator%(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator&(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator|(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator^(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator<<(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator>>(const basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec operator<<(const basic_vec&, simd-size-type) noexcept;
friend constexpr basic_vec operator>>(const basic_vec&, simd-size-type) noexcept;

// 29.10.8.2, basic_vec compound assignment
friend constexpr basic_vec& operator+=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator-=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator*=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator/=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator%=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator&=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator|=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator^=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator<=<=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator>=>=(basic_vec&, const basic_vec&) noexcept;
friend constexpr basic_vec& operator<=<=(basic_vec&, simd-size-type) noexcept;
friend constexpr basic_vec& operator>=>=(basic_vec&, simd-size-type) noexcept;

// 29.10.8.3, basic_vec compare operators
friend constexpr mask_type operator==(const basic_vec&, const basic_vec&) noexcept;
friend constexpr mask_type operator!=(const basic_vec&, const basic_vec&) noexcept;
friend constexpr mask_type operator>=(const basic_vec&, const basic_vec&) noexcept;
friend constexpr mask_type operator<=(const basic_vec&, const basic_vec&) noexcept;
friend constexpr mask_type operator>(const basic_vec&, const basic_vec&) noexcept;
friend constexpr mask_type operator<(const basic_vec&, const basic_vec&) noexcept;

// 29.10.8.4, basic_vec exposition only conditional operators
friend constexpr basic_vec simd-select-impl(// exposition only
 const mask_type&, const basic_vec&, const basic_vec&) noexcept;
};

template<class R, class... Ts>
 basic_vec(R&& r, Ts...) -> see below;

template<size_t Bytes, class Abi>
 basic_vec(basic_mask<Bytes, Abi>) -> see below;
}

```

<sup>1</sup> Every specialization of `basic_vec` is a complete type. The specialization of `basic_vec<T, Abi>` is

- (1.1) — enabled, if `T` is a vectorizable type, and there exists value `N` in the range `[1, 64]`, such that `Abi` names the ABI tag denoted by *deduce-abi-t*`<T, N>`,
- (1.2) — otherwise, disabled, if `T` is not a vectorizable type,
- (1.3) — otherwise, it is implementation-defined if such a specialization is enabled.

If `basic_vec<T, Abi>` is disabled, then the specialization has a deleted default constructor, deleted destructor, deleted copy constructor, and deleted copy assignment. In addition only the `value_type`, `abi_type`, and `mask_type` members are present.

If `basic_vec<T, Abi>` is enabled, then

- (1.4) — `basic_vec<T, Abi>` is trivially copyable,
- (1.5) — default-initialization of an object of such a type default-initializes all elements,
- (1.6) — value-initialization value-initializes all elements (9.5.1),

- (1.7) — `basic_vec<T, Abi>::mask_type` is an alias for an enabled specialization of `basic_mask`, and
- (1.8) — `basic_vec<T, Abi>::size()` is equal to `basic_vec<T, Abi>::mask_type::size()`.
- 2 *Recommended practice:* Implementations should support implicit conversions between specializations of `basic_vec` and appropriate implementation-defined types.
- [*Note 1:* Appropriate types are non-standard vector types which are available in the implementation. — *end note*]
- 3 If `T` is a specialization of `complex`, *real-type* denotes the same type as `rebind_t<typename T::value_type, basic_vec<T, Abi>>`, otherwise an unspecified non-array object type.

### 29.10.7.2 Constructors

[simd.ctor]

```
template<class U> constexpr basic_vec(U&& value) noexcept;
```

1 Let `From` denote the type `remove_cvref_t<U>`.

2 *Constraints:*

- (2.1) — `U` satisfies `convertible_to<value_type>` and `From` is not an arithmetic type and does not satisfy *constexpr-wrapper-like*,
- (2.2) — `From` is an arithmetic type and the conversion from `From` to `value_type` is value-preserving (29.10.1), or
- (2.3) — `From` satisfies *constexpr-wrapper-like*, `remove_cvref_t<decltype(From::value)>` is an arithmetic type, and `From::value` is representable by `value_type`.

3 *Effects:* Initializes each element to the value of the argument after conversion to `value_type`.

```
template<class U, class UAbi>
constexpr explicit(see below) basic_vec(const basic_vec<U, UAbi>& x) noexcept;
```

4 *Constraints:*

- (4.1) — `simd-size-v<U, UAbi> == size()` is true, and
- (4.2) — `U` satisfies *explicitly-convertible-to<T>*.
- 5 *Effects:* Initializes the  $i^{\text{th}}$  element with `static_cast<T>(x[i])` for all  $i$  in the range of `[0, size())`.
- 6 *Remarks:* The expression inside `explicit` evaluates to `true` if either
- (6.1) — the conversion from `U` to `value_type` is not value-preserving, or
- (6.2) — both `U` and `value_type` are integral types and the integer conversion rank (6.9.7) of `U` is greater than the integer conversion rank of `value_type`, or
- (6.3) — both `U` and `value_type` are floating-point types and the floating-point conversion rank (6.9.7) of `U` is greater than the floating-point conversion rank of `value_type`.

```
template<class G> constexpr explicit basic_vec(G&& gen);
```

7 Let `Fromi` denote the type `decltype(gen(integral_constant<simd-size-type, i>()))`.

8 *Constraints:* `Fromi` satisfies `convertible_to<value_type>` for all  $i$  in the range of `[0, size())`. In addition, for all  $i$  in the range of `[0, size())`, if `Fromi` is an arithmetic type, conversion from `Fromi` to `value_type` is value-preserving.

9 *Effects:* Initializes the  $i^{\text{th}}$  element with `static_cast<value_type>(gen(integral_constant<simd-size-type, i>()))` for all  $i$  in the range of `[0, size())`.

10 *Remarks:* `gen` is invoked exactly once for each  $i$ , in increasing order of  $i$ .

```
template<class R, class... Flags>
constexpr basic_vec(R&& r, flags<Flags...> = {});
template<class R, class... Flags>
constexpr basic_vec(R&& r, const mask_type& mask, flags<Flags...> = {});
```

11 Let `mask` be `mask_type(true)` for the overload with no `mask` parameter.

12 *Constraints:*

- (12.1) — `R` models `ranges::contiguous_range` and `ranges::sized_range`,
- (12.2) — `ranges::size(r)` is a constant expression,

- (12.3) — `ranges::size(r)` is equal to `size()`, and
- (12.4) — `ranges::range_value_t<R>` is a vectorizable type and satisfies *explicitly-convertible-to* `<T>`.
- 13 *Mandates:* If the template parameter pack `Flags` does not contain *convert-flag*, then the conversion from `ranges::range_value_t<R>` to `value_type` is value-preserving.
- 14 *Preconditions:*
- (14.1) — If the template parameter pack `Flags` contains *aligned-flag*, `ranges::data(r)` points to storage aligned by `alignment_v<basic_vec, ranges::range_value_t<R>>`.
- (14.2) — If the template parameter pack `Flags` contains *overaligned-flag* `<N>`, `ranges::data(r)` points to storage aligned by `N`.
- 15 *Effects:* Initializes the  $i^{\text{th}}$  element with `mask[i] ? static_cast<T>(ranges::data(r)[i]) : T()` for all  $i$  in the range of `[0, size())`.
- ```
template<class R, class... Ts>
    basic_vec(R&& r, Ts...) -> see below;
```
- 16 *Constraints:*
- (16.1) — `R` models `ranges::contiguous_range` and `ranges::sized_range`, and
- (16.2) — `ranges::size(r)` is a constant expression.
- 17 *Remarks:* The deduced type is equivalent to
- ```
vec<ranges::range_value_t<R>, static_cast<simd-size-type>(ranges::size(r))>
```
- ```
template<size_t Bytes, class Abi>
    basic_vec(basic_mask<Bytes, Abi> k) -> see below;
```
- 18 *Constraints:* `basic_mask<Bytes, Abi>` is an enabled specialization of `basic_mask` and `decltype(+k)` is a valid type.
- 19 *Remarks:* The deduced type is equivalent to `decltype(+k)`.
- ```
constexpr basic_vec(const real-type& reals, const real-type& imgs = {}) noexcept;
```
- 20 *Constraints:* `simd-complex<basic_vec>` is modeled.
- 21 *Effects:* Initializes the  $i^{\text{th}}$  element with `value_type(reals[i], imgs[i])` for all  $i$  in the range `[0, size())`.

### 29.10.7.3 Subscript operator

[simd.subscr]

```
constexpr value_type operator[](simd-size-type i) const;
```

- 1 *Preconditions:* `i >= 0 && i < size()` is true.
- 2 *Returns:* The value of the  $i^{\text{th}}$  element.
- 3 *Throws:* Nothing.

```
template<simd-integral I>
 constexpr resize_t<I::size(), basic_vec> operator[](const I& indices) const;
```

4 *Effects:* Equivalent to: `return permute(*this, indices);`

### 29.10.7.4 Complex accessors

[simd.complex.access]

```
constexpr real-type real() const noexcept;
constexpr real-type imag() const noexcept;
```

- 1 *Constraints:* `simd-complex<basic_vec>` is modeled.
- 2 *Returns:* An object of type *real-type* where the  $i^{\text{th}}$  element is initialized to the result of *cmplx-func* (`operator[] (i)`) for all  $i$  in the range `[0, size())`, where *cmplx-func* is the corresponding function from `<complex>`.

```
constexpr void real(const real-type& v) noexcept;
constexpr void imag(const real-type& v) noexcept;
```

- 3 *Constraints:* `simd-complex<basic_vec>` is modeled.

4 *Effects:* Replaces each element of the `basic_vec` object such that the  $i^{\text{th}}$  element is replaced with `value_type(v[i], operator[](i).imag())` or `value_type(operator[](i).real(), v[i])` for `real` and `imag` respectively, for all  $i$  in the range `[0, size())`.

### 29.10.7.5 Unary operators

[simd.unary]

1 Effects in 29.10.7.5 are applied as unary element-wise operations.

```
constexpr basic_vec& operator++() noexcept;
```

2 *Constraints:* requires `(value_type a) { ++a; }` is true.

3 *Effects:* Increments every element by one.

4 *Returns:* `*this`.

```
constexpr basic_vec operator++(int) noexcept;
```

5 *Constraints:* requires `(value_type a) { a++; }` is true.

6 *Effects:* Increments every element by one.

7 *Returns:* A copy of `*this` before incrementing.

```
constexpr basic_vec& operator--() noexcept;
```

8 *Constraints:* requires `(value_type a) { --a; }` is true.

9 *Effects:* Decrements every element by one.

10 *Returns:* `*this`.

```
constexpr basic_vec operator--(int) noexcept;
```

11 *Constraints:* requires `(value_type a) { a--; }` is true.

12 *Effects:* Decrements every element by one.

13 *Returns:* A copy of `*this` before decrementing.

```
constexpr mask_type operator!() const noexcept;
```

14 *Constraints:* requires `(const value_type a) { !a; }` is true.

15 *Returns:* A `basic_mask` object with the  $i^{\text{th}}$  element set to `!operator[](i)` for all  $i$  in the range of `[0, size())`.

```
constexpr basic_vec operator~() const noexcept;
```

16 *Constraints:* requires `(const value_type a) { ~a; }` is true.

17 *Returns:* A `basic_vec` object with the  $i^{\text{th}}$  element set to `~operator[](i)` for all  $i$  in the range of `[0, size())`.

```
constexpr basic_vec operator+() const noexcept;
```

18 *Constraints:* requires `(const value_type a) { +a; }` is true.

19 *Returns:* `*this`.

```
constexpr basic_vec operator-() const noexcept;
```

20 *Constraints:* requires `(const value_type a) { -a; }` is true.

21 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to `-operator[](i)` for all  $i$  in the range of `[0, size())`.

### 29.10.8 `basic_vec` non-member operations

[simd.nonmembers]

#### 29.10.8.1 Binary operators

[simd.binary]

```
friend constexpr basic_vec operator+(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator-(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator*(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator/(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator%(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator&(const basic_vec& lhs, const basic_vec& rhs) noexcept;
```

```
friend constexpr basic_vec operator|(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator^(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator<<(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec operator>>(const basic_vec& lhs, const basic_vec& rhs) noexcept;
```

1     Let *op* be the operator.

2     *Constraints:* requires (value\_type a, value\_type b) { a *op* b; } is true.

3     *Returns:* A basic\_vec object initialized with the results of applying *op* to lhs and rhs as a binary element-wise operation.

```
friend constexpr basic_vec operator<<(const basic_vec& v, simd-size-type n) noexcept;
friend constexpr basic_vec operator>>(const basic_vec& v, simd-size-type n) noexcept;
```

4     Let *op* be the operator.

5     *Constraints:* requires (value\_type a, simd-size-type b) { a *op* b; } is true.

6     *Returns:* A basic\_vec object where the  $i^{\text{th}}$  element is initialized to the result of applying *op* to v[i] and n for all *i* in the range of [0, size()).

### 29.10.8.2 Compound assignment

[simd.cassign]

```
friend constexpr basic_vec& operator+=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator-=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator*=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator/=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator%=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator&=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator|=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator^=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator<=<=(basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr basic_vec& operator>>=(basic_vec& lhs, const basic_vec& rhs) noexcept;
```

1     Let *op* be the operator.

2     *Constraints:* requires (value\_type a, value\_type b) { a *op* b; } is true.

3     *Effects:* These operators apply the indicated operator to lhs and rhs as an element-wise operation.

4     *Returns:* lhs.

```
friend constexpr basic_vec& operator<<=(basic_vec& lhs, simd-size-type n) noexcept;
friend constexpr basic_vec& operator>>=(basic_vec& lhs, simd-size-type n) noexcept;
```

5     Let *op* be the operator.

6     *Constraints:* requires (value\_type a, simd-size-type b) { a *op* b; } is true.

7     *Effects:* Equivalent to: return operator *op* (lhs, basic\_vec(n));

### 29.10.8.3 Comparison operators

[simd.comparison]

```
friend constexpr mask_type operator==(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr mask_type operator!=(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr mask_type operator>=(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr mask_type operator<=(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr mask_type operator>(const basic_vec& lhs, const basic_vec& rhs) noexcept;
friend constexpr mask_type operator<(const basic_vec& lhs, const basic_vec& rhs) noexcept;
```

1     Let *op* be the operator.

2     *Constraints:* requires (value\_type a, value\_type b) { a *op* b; } is true.

3     *Returns:* A basic\_mask object initialized with the results of applying *op* to lhs and rhs as a binary element-wise operation.

## 29.10.8.4 Exposition-only conditional operators

[simd.cond]

```
friend constexpr basic_vec
simd-select-impl(const mask_type& mask, const basic_vec& a, const basic_vec& b) noexcept;
```

- 1     *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element equals `mask[i] ? a[i] : b[i]` for all  $i$  in the range of `[0, size())`.

## 29.10.8.5 Reductions

[simd.reductions]

```
template<class T, class Abi, class BinaryOperation = plus<>>
constexpr T reduce(const basic_vec<T, Abi>& x, BinaryOperation binary_op = {});
```

- 1     *Constraints:* `BinaryOperation` models *reduction-binary-operation*<T>.
- 2     *Preconditions:* `binary_op` does not modify `x`.
- 3     *Returns:* `GENERALIZED_SUM(binary_op, vec<T, 1>(x[0]), ..., vec<T, 1>(x[x.size() - 1]))[0]` (26.10.2).
- 4     *Throws:* Any exception thrown from `binary_op`.

```
template<class T, class Abi, class BinaryOperation = plus<>>
constexpr T reduce(
 const basic_vec<T, Abi>& x, const typename basic_vec<T, Abi>::mask_type& mask,
 BinaryOperation binary_op = {}, type_identity_t<T> identity_element = see below);
```

- 5     *Constraints:*
- (5.1)     — `BinaryOperation` models *reduction-binary-operation*<T>.
- (5.2)     — An argument for `identity_element` is provided for the invocation, unless `BinaryOperation` is one of `plus<>`, `multiplies<>`, `bit_and<>`, `bit_or<>`, or `bit_xor<>`.
- 6     *Preconditions:*
- (6.1)     — `binary_op` does not modify `x`.
- (6.2)     — For all finite values  $y$  representable by  $T$ , the results of `y == binary_op(vec<T, 1>(identity_element), vec<T, 1>(y))[0]` and `y == binary_op(vec<T, 1>(y), vec<T, 1>(identity_element))[0]` are true.
- 7     *Returns:* If `none_of(mask)` is true, returns `identity_element`. Otherwise, returns `GENERALIZED_SUM(binary_op, vec<T, 1>(x[k0]), ..., vec<T, 1>(x[kn]))[0]` where  $k_0, \dots, k_n$  are the selected indices of `mask`.
- 8     *Throws:* Any exception thrown from `binary_op`.
- 9     *Remarks:* The default argument for `identity_element` is equal to
- (9.1)     — `T()` if `BinaryOperation` is `plus<>`,
- (9.2)     — `T(1)` if `BinaryOperation` is `multiplies<>`,
- (9.3)     — `T(~T())` if `BinaryOperation` is `bit_and<>`,
- (9.4)     — `T()` if `BinaryOperation` is `bit_or<>`, or
- (9.5)     — `T()` if `BinaryOperation` is `bit_xor<>`.

```
template<class T, class BinaryOperation = plus<>>
constexpr T reduce(const T& x, BinaryOperation binary_op = {});
template<class T, class BinaryOperation = plus<>>
constexpr T reduce(const T& x, same_as<bool> auto mask, BinaryOperation binary_op = {},
 type_identity_t<T> identity_element = see below);
```

- 10     Let `mask` be true for the overload with no `mask` parameter.
- 11     *Constraints:*
- (11.1)     —  $T$  is vectorizable.
- (11.2)     — `BinaryOperation` models *reduction-binary-operation*<T>.
- (11.3)     — An argument for `identity_element` is provided for the invocation, unless `BinaryOperation` is one of `plus<>`, `multiplies<>`, `bit_and<>`, `bit_or<>`, or `bit_xor<>`.



12 *Returns:* If `mask` is `false`, returns `identity_element`. Otherwise, returns `x`.

13 *Throws:* Nothing.

14 *Remarks:* The default argument for `identity_element` is equal to

- (14.1) — `T()` if `BinaryOperation` is `plus<>`,
- (14.2) — `T(1)` if `BinaryOperation` is `multiplies<>`,
- (14.3) — `T(~T())` if `BinaryOperation` is `bit_and<>`,
- (14.4) — `T()` if `BinaryOperation` is `bit_or<>`, or
- (14.5) — `T()` if `BinaryOperation` is `bit_xor<>`.

```
template<class T, class Abi> constexpr T reduce_min(const basic_vec<T, Abi>& x) noexcept;
```

15 *Constraints:* `T` models `totally_ordered`.

16 *Returns:* The value of an element `x[j]` for which `x[i] < x[j]` is `false` for all  $i$  in the range of `[0, basic_vec<T, Abi>::size())`.

```
template<class T, class Abi>
constexpr T reduce_min(
 const basic_vec<T, Abi>& x, const typename basic_vec<T, Abi>::mask_type& mask) noexcept;
```

17 *Constraints:* `T` models `totally_ordered`.

18 *Returns:* If `none_of(mask)` is `true`, returns `numeric_limits<T>::max()`. Otherwise, returns the value of a selected element `x[j]` for which `x[i] < x[j]` is `false` for all selected indices  $i$  of `mask`.

```
template<class T, class Abi> constexpr T reduce_max(const basic_vec<T, Abi>& x) noexcept;
```

19 *Constraints:* `T` models `totally_ordered`.

20 *Returns:* The value of an element `x[j]` for which `x[j] < x[i]` is `false` for all  $i$  in the range of `[0, basic_vec<T, Abi>::size())`.

```
template<class T, class Abi>
constexpr T reduce_max(
 const basic_vec<T, Abi>& x, const typename basic_vec<T, Abi>::mask_type& mask) noexcept;
```

21 *Constraints:* `T` models `totally_ordered`.

22 *Returns:* If `none_of(mask)` is `true`, returns `numeric_limits<T>::lowest()`. Otherwise, returns the value of a selected element `x[j]` for which `x[j] < x[i]` is `false` for all selected indices  $i$  of `mask`.

```
template<class T> constexpr T reduce_min(const T& x) noexcept;
template<class T> constexpr T reduce_min(const T& x, same_as<bool> auto mask) noexcept;
template<class T> constexpr T reduce_max(const T& x) noexcept;
template<class T> constexpr T reduce_max(const T& x, same_as<bool> auto mask) noexcept;
```

23 Let `mask` be `true` for the overloads with no `mask` parameter.

24 *Constraints:*

- (24.1) — `T` is vectorizable.
- (24.2) — `T` models `totally_ordered`.

25 *Returns:* If `mask` is `false`, returns `numeric_limits<T>::max()` for `reduce_min` and `numeric_limits<T>::lowest()` for `reduce_max`. Otherwise, returns `x`.

## 29.10.8.6 Load and store functions

[simd.loadstore]

```
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_load(R&& r, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V unchecked_load(R&& r, const typename V::mask_type& mask, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V unchecked_load(I first, iter_difference_t<I> n, flags<Flags...> f = {});
```

```

template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V unchecked_load(I first, iter_difference_t<I> n, const typename V::mask_type& mask,
 flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V unchecked_load(I first, S last, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V unchecked_load(I first, S last, const typename V::mask_type& mask,
 flags<Flags...> f = {});

```

1     Let

- (1.1)     — `mask` be `V::mask_type(true)` for the overloads with no `mask` parameter;
- (1.2)     — `R` be `span<const iter_value_t<I>>` for the overloads with no template parameter `R`;
- (1.3)     — `r` be `R(first, n)` for the overloads with an `n` parameter and `R(first, last)` for the overloads with a `last` parameter.

2     *Mandates:* If `ranges::size(r)` is a constant expression then `ranges::size(r) ≥ V::size()`.

3     *Preconditions:*

- (3.1)     — `[first, first + n)` is a valid range for the overloads with an `n` parameter.
- (3.2)     — `[first, last)` is a valid range for the overloads with a `last` parameter.
- (3.3)     — `ranges::size(r) ≥ V::size()`

4     *Effects:* Equivalent to: `return partial_load<V>(r, mask, f);`

5     *Remarks:* The default argument for template parameter `V` is `basic_vec<ranges::range_value_t<R>>`.

```

template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_load(R&& r, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, class... Flags>
 requires ranges::sized_range<R>
 constexpr V partial_load(R&& r, const typename V::mask_type& mask, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V partial_load(I first, iter_difference_t<I> n, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, class... Flags>
 constexpr V partial_load(I first, iter_difference_t<I> n, const typename V::mask_type& mask,
 flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V partial_load(I first, S last, flags<Flags...> f = {});
template<class V = see below, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
 constexpr V partial_load(I first, S last, const typename V::mask_type& mask,
 flags<Flags...> f = {});

```

6     Let

- (6.1)     — `mask` be `V::mask_type(true)` for the overloads with no `mask` parameter;
- (6.2)     — `R` be `span<const iter_value_t<I>>` for the overloads with no template parameter `R`;
- (6.3)     — `r` be `R(first, n)` for the overloads with an `n` parameter and `R(first, last)` for the overloads with a `last` parameter;
- (6.4)     — `T` be `typename V::value_type`.

7     *Mandates:*

- (7.1)     — `ranges::range_value_t<R>` is a vectorizable type and satisfies *explicitly-convertible-to*<`T`>,
- (7.2)     — `same_as<remove_cvref_t<V>, V>` is true,
- (7.3)     — `V` is an enabled specialization of `basic_vec`, and
- (7.4)     — if the template parameter pack `Flags` does not contain *convert-flag*, then the conversion from `ranges::range_value_t<R>` to `V::value_type` is value-preserving.

8     *Preconditions:*

- (8.1)     — `[first, first + n)` is a valid range for the overloads with an `n` parameter.
- (8.2)     — `[first, last)` is a valid range for the overloads with a `last` parameter.

- (8.3) — If the template parameter pack `Flags` contains *aligned-flag*, `ranges::data(r)` points to storage aligned by `alignment_v<V, ranges::range_value_t<R>>`.
- (8.4) — If the template parameter pack `Flags` contains *overaligned-flag*<N>, `ranges::data(r)` points to storage aligned by N.

9 *Returns:* A `basic_vec` object whose  $i^{\text{th}}$  element is initialized with `mask[i]` &&  $i < \text{ranges::size}(r)$  ? `static_cast<T>(ranges::data(r)[i])` : `T()` for all  $i$  in the range of  $[0, V::\text{size}())$ , where `T` is `V::value_type`.

10 *Remarks:* The default argument for template parameter `V` is `basic_vec<ranges::range_value_t<R>>`.

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r, flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, R&& r,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr void unchecked_store(const basic_vec<T, Abi>& v, I first, S last,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
```

11 *Let*

- (11.1) — `mask` be `basic_vec<T, Abi>::mask_type(true)` for the overloads with no `mask` parameter;
- (11.2) — `R` be `span<iter_value_t<I>>` for the overloads with no template parameter `R`;
- (11.3) — `r` be `R(first, n)` for the overloads with an `n` parameter and `R(first, last)` for the overloads with a `last` parameter.

12 *Mandates:* If `ranges::size(r)` is a constant expression then `ranges::size(r) ≥ simd-size-v<T, Abi>`.

13 *Preconditions:*

- (13.1) — `[first, first + n)` is a valid range for the overloads with an `n` parameter.
- (13.2) — `[first, last)` is a valid range for the overloads with a `last` parameter.
- (13.3) — `ranges::size(r) ≥ simd-size-v<T, Abi>`

14 *Effects:* Equivalent to: `partial_store(v, r, mask, f)`.

```
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r, flags<Flags...> f = {});
template<class T, class Abi, ranges::contiguous_range R, class... Flags>
requires ranges::sized_range<R>
constexpr void partial_store(const basic_vec<T, Abi>& v, R&& r,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, class... Flags>
constexpr void partial_store(const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
 flags<Flags...> f = {});
```

```

template<class T, class Abi, contiguous_iterator I, class... Flags>
constexpr void partial_store(const basic_vec<T, Abi>& v, I first, iter_difference_t<I> n,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
 flags<Flags...> f = {});
template<class T, class Abi, contiguous_iterator I, sized_sentinel_for<I> S, class... Flags>
constexpr void partial_store(const basic_vec<T, Abi>& v, I first, S last,
 const typename basic_vec<T, Abi>::mask_type& mask,
 flags<Flags...> f = {});

```

15 Let

- (15.1) — `mask` be `basic_vec<T, Abi>::mask_type(true)` for the overloads with no `mask` parameter;
- (15.2) — `R` be `span<iter_value_t<I>>` for the overloads with no template parameter `R`;
- (15.3) — `r` be `R(first, n)` for the overloads with an `n` parameter and `R(first, last)` for the overloads with a `last` parameter.

16 *Constraints:*

- (16.1) — `ranges::iterator_t<R>` models `indirectly_writable<ranges::range_value_t<R>>`, and
- (16.2) — `T` satisfies *explicitly-convertible-to* `<ranges::range_value_t<R>>`.

17 *Mandates:*

- (17.1) — `ranges::range_value_t<R>` is a vectorizable type, and
- (17.2) — if the template parameter pack `Flags` does not contain *convert-flag*, then the conversion from `T` to `ranges::range_value_t<R>` is value-preserving.

18 *Preconditions:*

- (18.1) — `[first, first + n)` is a valid range for the overloads with an `n` parameter.
- (18.2) — `[first, last)` is a valid range for the overloads with a `last` parameter.
- (18.3) — If the template parameter pack `Flags` contains *aligned-flag*, `ranges::data(r)` points to storage aligned by `alignment_v<basic_vec<T, Abi>, ranges::range_value_t<R>>`.
- (18.4) — If the template parameter pack `Flags` contains *overaligned-flag* `<N>`, `ranges::data(r)` points to storage aligned by `N`.

19 *Effects:* For all `i` in the range of `[0, basic_vec<T, Abi>::size())`, if `mask[i] && i < ranges::size(r)` is true, evaluates `ranges::data(r)[i] = static_cast<ranges::range_value_t<R>>(v[i])`.

### 29.10.8.7 Static permute

[simd.permute.static]

```

template<simd-size-type N = see below, simd-vec-type V, class IdxMap>
constexpr resize_t<N, V> permute(const V& v, IdxMap&& idxmap);
template<simd-size-type N = see below, simd-mask-type V, class IdxMap>
constexpr resize_t<N, V> permute(const V& v, IdxMap&& idxmap);

```

1 Let:

- (1.1) — *gen-fn*(`i`) be `idxmap(i, V::size())` if that expression is well-formed, and `idxmap(i)` otherwise.
- (1.2) — *perm-fn* be the following exposition-only function template:

```

template<simd-size-type I>
typename V::value_type perm-fn() {
 constexpr auto src_index = gen-fn(I);
 if constexpr (src_index == zero_element) {
 return typename V::value_type();
 } else if constexpr (src_index == uninit_element) {
 return unspecified-value;
 } else {
 return v[src_index];
 }
}

```

2 *Constraints:* At least one of `invoke_result_t<IdxMap&, simd-size-type>` and `invoke_result_t<IdxMap&, simd-size-type, simd-size-type>` satisfies `integral`.

3 *Mandates:* `gen-fn(i)` is a constant expression whose value is `zero_element`, `uninit_element`, or in the range `[0, V::size())`, for all  $i$  in the range `[0, N)`.

4 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the result of `perm-fn<i>()` for all  $i$  in the range `[0, N)`.

5 *Remarks:* The default argument for template parameter `N` is `V::size()`.

### 29.10.8.8 Dynamic permute

[simd.permute.dynamic]

```
template<simd-vec-type V, simd-integral I>
constexpr resize_t<I::size(), V> permute(const V& v, const I& indices);
template<simd-mask-type V, simd-integral I>
constexpr resize_t<I::size(), V> permute(const V& v, const I& indices);
```

1 *Preconditions:* All values in `indices` are in the range `[0, V::size())`.

2 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the result of `v[indices[i]]` for all  $i$  in the range `[0, I::size())`.

### 29.10.8.9 Mask permute

[simd.permute.mask]

```
template<simd-vec-type V>
constexpr V compress(const V& v, const typename V::mask_type& selector);
template<simd-mask-type V>
constexpr V compress(const V& v, const type_identity_t<V>& selector);
```

1 Let:

(1.1) — `bit-index(i)` be a function which returns the index of the  $i^{\text{th}}$  element of `selector` that is `true`.

(1.2) — `select-value(i)` be a function which returns `v[bit-index(i)]` for  $i$  in the range `[0, reduce_count(selector))` and a valid but unspecified value otherwise.

[Note 1: Different calls to `select-value` can return different unspecified values. — end note]

2 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the result of `select-value(i)` for all  $i$  in the range `[0, V::size())`.

```
template<simd-vec-type V>
constexpr V compress(const V& v, const typename V::mask_type& selector,
 const typename V::value_type& fill_value);
template<simd-mask-type V>
constexpr V compress(const V& v, const type_identity_t<V>& selector,
 const typename V::value_type& fill_value);
```

3 Let:

(3.1) — `bit-index(i)` be a function which returns the index of the  $i^{\text{th}}$  element of `selector` that is `true`.

(3.2) — `select-value(i)` be a function which returns `v[bit-index(i)]` for  $i$  in the range `[0, reduce_count(selector))` and `fill_value` otherwise.

4 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the result of `select-value(i)` for all  $i$  in the range `[0, V::size())`.

```
template<simd-vec-type V>
constexpr V expand(const V& v, const typename V::mask_type& selector, const V& original = {});
template<simd-mask-type V>
constexpr V expand(const V& v, const type_identity_t<V>& selector, const V& original = {});
```

5 Let:

(5.1) — `set-indices` be a list of the index positions of `true` elements in `selector`, in ascending order.

(5.2) — `bit-lookup(b)` be a function which returns the index where `b` appears in `set-indices`.

(5.3) — `select-value(i)` be a function which returns `v[bit-lookup(i)]` if `selector[i]` is `true`, otherwise returns `original[i]`.

6 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the result of *select-value*( $i$ ) for all  $i$  in the range  $[0, V::\text{size}())$ .

#### 29.10.8.10 Memory permute

[simd.permute.memory]

```
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr V unchecked_gather_from(R&& in, const I& indices, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr V unchecked_gather_from(R&& in, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});
```

1 Let mask be `typename I::mask_type(true)` for the overload with no mask parameter.

2 *Preconditions:* All values in `select(mask, indices, typename I::value_type())` are in the range  $[0, \text{ranges::size}(\text{in}))$ .

3 *Effects:* Equivalent to: `return partial_gather_from<V>(in, mask, indices, f);`

4 *Remarks:* The default argument for template parameter  $V$  is `vec<ranges::range_value_t<R>, I::size()>`.

```
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr V partial_gather_from(R&& in, const I& indices, flags<Flags...> f = {});
template<class V = see below, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr V partial_gather_from(R&& in, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});
```

5 Let:

(5.1) — mask be `typename I::mask_type(true)` for the overload with no mask parameter;

(5.2) —  $T$  be `typename V::value_type`.

6 *Constraints:* `ranges::range_value_t<R>` is a vectorizable type and satisfies *explicitly-convertible-to*< $T$ >.

7 *Mandates:*

(7.1) — `ranges::range_value_t<R>` is a vectorizable type,

(7.2) — `same_as<remove_cvref_t<V>, V>` is true,

(7.3) —  $V$  is an enabled specialization of `basic_vec`,

(7.4) — `V::size() == I::size()` is true, and

(7.5) — if the template parameter pack `Flags` does not contain *convert-flag*, then the conversion from `ranges::range_value_t<R>` to  $T$  is value-preserving.

8 *Preconditions:*

(8.1) — If the template parameter pack `Flags` contains *aligned-flag*, `ranges::data(in)` points to storage aligned by `alignment_v<V, ranges::range_value_t<R>>`.

(8.2) — If the template parameter pack `Flags` contains *overaligned-flag*< $N$ >, `ranges::data(in)` points to storage aligned by  $N$ .

9 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of

`mask[i] && indices[i] < ranges::size(in) ? static_cast<T>(ranges::data(in)[indices[i]]) : T()`  
for all  $i$  in the range  $[0, I::\text{size}())$ .

10 *Remarks:* The default argument for template parameter  $V$  is `vec<ranges::range_value_t<R>, I::size()>`.

```
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr void unchecked_scatter_to(const V& v, R&& out, const I& indices,
 flags<Flags...> f = {});
```

```
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr void unchecked_scatter_to(const V& v, R&& out, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});
```

11 Let mask be `typename I::mask_type(true)` for the overload with no mask parameter.

12 *Preconditions:* All values in `select(mask, indices, typename I::value_type())` are in the range `[0, ranges::size(out))`.

13 *Effects:* Equivalent to: `partial_scatter_to(v, out, mask, indices, f);`

```
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr void
partial_scatter_to(const V& v, R&& out, const I& indices, flags<Flags...> f = {});
template<simd-vec-type V, ranges::contiguous_range R, simd-integral I, class... Flags>
requires ranges::sized_range<R>
constexpr void partial_scatter_to(const V& v, R&& out, const typename I::mask_type& mask,
 const I& indices, flags<Flags...> f = {});
```

14 Let mask be `typename I::mask_type(true)` for the overload with no mask parameter.

15 *Constraints:*

(15.1) — `V::size() == I::size()` is true,

(15.2) — `ranges::iterator_t<R>` models `indirectly_writable<ranges::range_value_t<R>>`, and

(15.3) — `typename V::value_type` satisfies *explicitly-convertible-to* `<ranges::range_value_t<R>>`.

16 *Mandates:*

(16.1) — `ranges::range_value_t<R>` is a vectorizable type, and

(16.2) — if the template parameter pack `Flags` does not contain *convert-flag*, then the conversion from `typename V::value_type` to `ranges::range_value_t<R>` is value-preserving.

17 *Preconditions:*

(17.1) — For all selected indices *i* the values `indices[i]` are unique.

(17.2) — If the template parameter pack `Flags` contains *aligned-flag*, `ranges::data(out)` points to storage aligned by `alignment_v<V, ranges::range_value_t<R>>`.

(17.3) — If the template parameter pack `Flags` contains *overaligned-flag* `<N>`, `ranges::data(out)` points to storage aligned by `N`.

18 *Effects:* For all *i* in the range `[0, I::size())`, if `mask[i] && (indices[i] < ranges::size(out))` is true, evaluates `ranges::data(out)[indices[i]] = static_cast<ranges::range_value_t<R>>(v[i])`.

### 29.10.8.11 Creation

[simd.creation]

```
template<class T, class Abi>
constexpr auto chunk(const basic_vec<typename T::value_type, Abi>& x) noexcept;
template<class T, class Abi>
constexpr auto chunk(const basic_mask<mask-element-size<T>, Abi>& x) noexcept;
```

1 *Constraints:*

(1.1) — For the first overload, `T` is an enabled specialization of `basic_vec`. If `basic_vec<typename T::value_type, Abi>::size() % T::size()` is not 0, then `resize_t<basic_vec<typename T::value_type, Abi>::size() % T::size(), T>` is valid and denotes a type.

(1.2) — For the second overload, `T` is an enabled specialization of `basic_mask`. If `basic_mask<mask-element-size<T>, Abi>::size() % T::size()` is not 0, then `resize_t<basic_mask<mask-element-size<T>, Abi>::size() % T::size(), T>` is valid and denotes a type.

2 Let `N` be `x.size() / T::size()`.

3 *Returns:*

- (3.1) — If `x.size() % T::size() == 0` is true, an `array<T, N>` with the  $i^{\text{th}}$  `basic_vec` or `basic_mask` element of the  $j^{\text{th}}$  array element initialized to the value of the element in `x` with index  $i + j * T::size()$ .
- (3.2) — Otherwise, a tuple of  $N$  objects of type `T` and one object of type `resize_t<x.size() % T::size(), T>`. The  $i^{\text{th}}$  `basic_vec` or `basic_mask` element of the  $j^{\text{th}}$  tuple element of type `T` is initialized to the value of the element in `x` with index  $i + j * T::size()$ . The  $i^{\text{th}}$  `basic_vec` or `basic_mask` element of the  $N^{\text{th}}$  tuple element is initialized to the value of the element in `x` with index  $i + N * T::size()$ .

```
template<simd-size-type N, class T, class Abi>
constexpr auto chunk(const basic_vec<T, Abi>& x) noexcept;
```

4 *Effects:* Equivalent to: `return chunk<resize_t<N, basic_vec<T, Abi>>>(x);`

```
template<simd-size-type N, size_t Bytes, class Abi>
constexpr auto chunk(const basic_mask<Bytes, Abi>& x) noexcept;
```

5 *Effects:* Equivalent to: `return chunk<resize_t<N, basic_mask<Bytes, Abi>>>(x);`

```
template<class T, class... Abis>
constexpr resize_t<(basic_vec<T, Abis>::size() + ...),
 basic_vec<T, Abis...[0]>> cat(const basic_vec<T, Abis>&... xs) noexcept;
template<size_t Bytes, class... Abis>
constexpr resize_t<(basic_mask<Bytes, Abis>::size() + ...),
 basic_mask<Bytes, Abis...[0]>> cat(const basic_mask<Bytes, Abis>&... xs) noexcept;
```

6 *Returns:* A data-parallel object initialized with the concatenated values in the `xs` pack of data-parallel objects: The  $i^{\text{th}}$  `basic_vec`/`basic_mask` element of the  $j^{\text{th}}$  parameter in the `xs` pack is copied to the return value's element with index  $i +$  the sum of the width of the first  $j$  parameters in the `xs` pack.

```
template<class T> constexpr T iota = see below;
```

7 *Mandates:* Either `T` is vectorizable and `is_arithmetic_v<T>` is true, or

- (7.1) — `T` is an enabled specialization of `basic_vec`,
- (7.2) — `is_arithmetic_v<typename T::value_type>` is true, and
- (7.3) — `T::size() - 1 ≤ numeric_limits<typename T::value_type>::max()`.

8 *Remarks:* If `is_arithmetic_v<T>` is true, the value of `iota<T>` is equal to `T()`. Otherwise, the value of `iota<T>` is equal to `T([] (typename T::value_type i) return i; )`.

### 29.10.8.12 Algorithms

[simd.alg]

```
template<class T, class Abi>
constexpr basic_vec<T, Abi> min(const basic_vec<T, Abi>& a,
 const basic_vec<T, Abi>& b) noexcept;
```

1 *Constraints:* `T` models `totally_ordered`.

2 *Returns:* The result of the element-wise application of `min(a[i], b[i])` for all  $i$  in the range of `[0, basic_vec<T, Abi>::size())`.

```
template<class T, class Abi>
constexpr basic_vec<T, Abi> max(const basic_vec<T, Abi>& a,
 const basic_vec<T, Abi>& b) noexcept;
```

3 *Constraints:* `T` models `totally_ordered`.

4 *Returns:* The result of the element-wise application of `max(a[i], b[i])` for all  $i$  in the range of `[0, basic_vec<T, Abi>::size())`.

```
template<class T, class Abi>
constexpr pair<basic_vec<T, Abi>, basic_vec<T, Abi>>
 minmax(const basic_vec<T, Abi>& a, const basic_vec<T, Abi>& b) noexcept;
```

5 *Effects:* Equivalent to: `return pair{min(a, b), max(a, b)};`



```
template<class T, class Abi>
constexpr basic_vec<T, Abi> clamp(
 const basic_vec<T, Abi>& v, const basic_vec<T, Abi>& lo, const basic_vec<T, Abi>& hi);
```

6     *Constraints:* T models totally\_ordered.

7     *Preconditions:* No element in lo is greater than the corresponding element in hi.

8     *Returns:* The result of element-wise application of clamp(v[i], lo[i], hi[i]) for all i in the range of [0, basic\_vec<T, Abi>::size()).

```
template<class T, class U>
constexpr auto select(bool c, const T& a, const U& b)
-> remove_cvref_t<decltype(c ? a : b)>;
```

9     *Effects:* Equivalent to: return c ? a : b;

```
template<size_t Bytes, class Abi, class T, class U>
constexpr auto select(const basic_mask<Bytes, Abi>& c, const T& a, const U& b)
noexcept -> decltype(simd-select-impl(c, a, b));
```

10    *Effects:* Equivalent to:

```
 return simd-select-impl(c, a, b);
```

where *simd-select-impl* is found by argument-dependent lookup (6.5.4) contrary to 16.4.2.2.

### 29.10.8.13 Mathematical functions

[simd.math]

```
template<math-floating-point V>
constexpr rebind_t<int, deduced-vec-t<V>> ilogb(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> abs(const V& j);
template<math-floating-point V>
constexpr deduced-vec-t<V> fabs(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> ceil(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> floor(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> nearbyint(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> rint(const V& x);
template<math-floating-point V>
constexpr rebind_t<long int, deduced-vec-t<V>> lrint(const V& x);
template<math-floating-point V>
constexpr rebind_t<long long int, deduced-vec-t<V>> llrint(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> round(const V& x);
template<math-floating-point V>
constexpr rebind_t<long int, deduced-vec-t<V>> lround(const V& x);
template<math-floating-point V>
constexpr rebind_t<long long int, deduced-vec-t<V>> llround(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> fmod(const V& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> fmod(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> fmod(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> trunc(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> remainder(const V& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> remainder(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> remainder(const V& x, const deduced-vec-t<V>& y);
```

```

template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> copysign(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> nextafter(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fdim(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmax(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const V& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fmin(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const deduced-vec-t<V>& x, const V& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const deduced-vec-t<V>& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V> fma(const V& x, const V& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 fma(const deduced-vec-t<V>& x, const deduced-vec-t<V>& y, const V& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 fma(const deduced-vec-t<V>& x, const V& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 fma(const V& x, const deduced-vec-t<V>& y, const deduced-vec-t<V>& z);
template<math-floating-point V>
 constexpr rebind_t<int, deduced-vec-t<V>> fpclassify(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isfinite(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isinf(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isnan(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isnormal(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type signbit(const V& x);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isgreater(const V& x, const V& y);
template<math-floating-point V>
 constexpr typename deduced-vec-t<V>::mask_type isgreater(const deduced-vec-t<V>& x, const V& y);

```

```

template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type isgreater(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isgreaterequal(const V& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isgreaterequal(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isgreaterequal(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isless(const V& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isless(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isless(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessequal(const V& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessequal(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessequal(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessgreater(const V& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessgreater(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
islessgreater(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isunordered(const V& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isunordered(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr typename deduced-vec-t<V>::mask_type
isunordered(const V& x, const deduced-vec-t<V>& y);

```

- 1 Let *Ret* denote the return type of the specialization of a function template with the name *math-func*.  
Let *math-func-vec* denote:

```

template<class... Args>
Ret math-func-vec(const Args&... args) {
 return Ret([&](simd-size-type i) {
 return math-func(deduced-vec-t<V>(args)[i]...);
 });
}

```

- 2 *Returns*: A value *ret* of type *Ret*, that is element-wise equal to the result of calling *math-func-vec* with the parameters of the above functions. If in an invocation of a scalar overload of *math-func* for index *i* in *math-func-vec* a domain, pole, or range error would occur, the value of *ret[i]* is unspecified.
- 3 *Remarks*: It is unspecified whether *errno* (19.4) is accessed.

```

template<math-floating-point V>
constexpr deduced-vec-t<V> ldexp(const V& x, const rebind_t<int, deduced-vec-t<V>>& exp);
template<math-floating-point V>
constexpr deduced-vec-t<V> scalbn(const V& x, const rebind_t<int, deduced-vec-t<V>>& n);
template<math-floating-point V>
constexpr deduced-vec-t<V>
scalbln(const V& x, const rebind_t<long int, deduced-vec-t<V>>& n);

```

- 4 Let *Ret* be *deduced-vec-t<V>*. Let *math-func* denote the name of the function template. Let *math-func-vec* denote:

```

Ret math-func-vec(const deduced-vec-t<V>& a, const auto& b) {
 return Ret([&](simd-size-type i) {
 return math-func(a[i], b[i]);
 });
}

```

- 5 *Returns:* A value *ret* of type *Ret*, that is element-wise equal to the result of calling *math-func-vec* with the parameters of the above functions. If in an invocation of a scalar overload of *math-func* for index *i* in *math-func-vec* a domain, pole, or range error would occur, the value of *ret[i]* is unspecified.

- 6 *Remarks:* It is unspecified whether *errno* (19.4) is accessed.

```

template<signed_integral T, class Abi>
constexpr basic_vec<T, Abi> abs(const basic_vec<T, Abi>& j);

```

- 7 *Preconditions:* *all\_of(j >= -numeric\_limits<T>::max())* is true.

- 8 *Returns:* An object where the *i*<sup>th</sup> element is initialized to the result of *std::abs(j[i])* for all *i* in the range *[0, j.size())*.

```

template<math-floating-point V> constexpr deduced-vec-t<V> acos(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> asin(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> atan(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> atan2(const V& y, const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> atan2(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> atan2(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V> constexpr deduced-vec-t<V> cos(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> sin(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> tan(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> acosh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> asinh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> atanh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> cosh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> sinh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> tanh(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> exp(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> exp2(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> expm1(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log10(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log1p(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> log2(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> logb(const V& x);
template<math-floating-point V> constexpr deduced-vec-t<V> cbrt(const V& x);
template<math-floating-point V>
constexpr deduced-vec-t<V> hypot(const V& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> hypot(const deduced-vec-t<V>& x, const V& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> hypot(const V& x, const deduced-vec-t<V>& y);
template<math-floating-point V>
constexpr deduced-vec-t<V> hypot(const V& x, const V& y, const V& z);

```

§ 29.10.8.13

© ISO/IEC  
1929

```

template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_j(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_bessel_k(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const V& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const deduced-vec-t<V>& nu, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> cyl_neumann(const V& nu, const deduced-vec-t<V>& x);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const V& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const deduced-vec-t<V>& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_1(const V& k, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const V& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const deduced-vec-t<V>& k, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_2(const V& k, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const V& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const V& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const deduced-vec-t<V>& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const V& nu, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const deduced-vec-t<V>& nu, const V& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const deduced-vec-t<V>& k, const V& nu, const deduced-vec-t<V>& phi);
template<math-floating-point V>
 deduced-vec-t<V> ellint_3(const V& k, const deduced-vec-t<V>& nu, const deduced-vec-t<V>& phi);
template<math-floating-point V> deduced-vec-t<V> expint(const V& x);
template<math-floating-point V> deduced-vec-t<V> riemann_zeta(const V& x);

```

- 9 Let *Ret* denote the return type of the specialization of a function template with the name *math-func*.  
Let *math-func-vec* denote:

```

template<class... Args>
Ret math-func-vec(const Args&... args) {
 return Ret([&](simd-size-type i) {
 return math-func(deduced-vec-t<V>(args)[i]...);
 });
}

```

- 10 *Returns:* A value *ret* of type *Ret*, that is element-wise approximately equal to the result of calling *math-func-vec* with the parameters of the above functions. If in an invocation of a scalar overload of *math-func* for index *i* in *math-func-vec* a domain, pole, or range error would occur, the value of *ret*[*i*] is unspecified.

- 11 *Remarks:* It is unspecified whether *errno* (19.4) is accessed.

```

template<math-floating-point V>
 deduced-vec-t<V> assoc_laguerre(const rebind_t<unsigned, deduced-vec-t<V>>& n,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> assoc_legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> sph_legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l,
 const rebind_t<unsigned, deduced-vec-t<V>>& m, const V& theta);

```

12 Let *math-func* denote the name of the function template. Let *math-func-vec* denote:

```

auto math-func-vec(const auto& a, const auto& b, const deduced-vec-t<V>& c) {
 return deduced-vec-t<V>([&](simd-size-type i) {
 return std::math-func(a[i], b[i], c[i]);
 });
}

```

13 *Returns:* An object that is element-wise approximately equal to the result of calling *math-func-vec* with the parameters of the above functions.

```

template<math-floating-point V>
 deduced-vec-t<V> hermite(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> laguerre(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> legendre(const rebind_t<unsigned, deduced-vec-t<V>>& l, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> sph_bessel(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);
template<math-floating-point V>
 deduced-vec-t<V> sph_neumann(const rebind_t<unsigned, deduced-vec-t<V>>& n, const V& x);

```

14 Let *math-func* denote the name of the function template. Let *math-func-vec* denote:

```

auto math-func-vec(const auto& a, const deduced-vec-t<V>& b) {
 return deduced-vec-t<V>([&](simd-size-type i) { return std::math-func(a[i], b[i]); });
}

```

15 *Returns:* An object that is element-wise approximately equal to the result of calling *math-func-vec* with the parameters of the above functions.

```

template<math-floating-point V>
 constexpr deduced-vec-t<V> frexp(const V& value, rebind_t<int, deduced-vec-t<V>>& exp);

```

16 Let *Ret* be *deduced-vec-t<V>*. Let *frexp-vec* denote:

```

template<class V>
 pair<Ret, rebind_t<int, Ret>> frexp-vec(const deduced-vec-t<V>& x) {
 int r1[Ret::size()];
 Ret r0([&](simd-size-type i) {
 return frexp(x[i], &r1[i]);
 });
 return {r0, rebind_t<int, Ret>(r1)};
}

```

Let *ret* be a value of type *pair<Ret, rebind\_t<int, Ret>>* that is the same value as the result of calling *frexp-vec*(*x*).

17 *Effects:* Sets *\*exp* to *ret.second*.

18 *Returns:* *ret.first*.

```

template<math-floating-point V>
 constexpr deduced-vec-t<V> remquo(const V& x, const V& y,
 rebind_t<int, deduced-vec-t<V>>& quo);
template<math-floating-point V>
 constexpr deduced-vec-t<V>
 remquo(const deduced-vec-t<V>& x, const V& y, rebind_t<int, deduced-vec-t<V>>& quo);

```



```
template<math-floating-point V>
constexpr deduced-vec-t<V>
remquo(const V& x, const deduced-vec-t<V>& y, rebind_t<int, deduced-vec-t<V>>* quo);
```

19 Let *V0* be *deduced-vec-t<V>*. Let *remquo-vec* denote:

```
pair<V0, rebind_t<int, V0>> remquo-vec(const V0& x, const V0& y) {
 int r1[V0::size()];
 V0 r0([&](simd-size-type i) { return remquo(x[i], y[i], &r1[i]); });
 return {r0, rebind_t<int, V0>(r1)};
}
```

Let *ret* be a value of type *pair<V0, rebind\_t<int, V0>>* that is the same value as the result of calling *remquo-vec*(*x*, *y*). If in an invocation of a scalar overload of *remquo* for index *i* in *remquo-vec* a domain, pole, or range error would occur, the value of *ret*[*i*] is unspecified.

20 *Effects*: Sets *\*quo* to *ret.second*.

21 *Returns*: *ret.first*.

22 *Remarks*: It is unspecified whether *errno* (19.4) is accessed.

```
template<class T, class Abi>
constexpr basic_vec<T, Abi> modf(const type_identity_t<basic_vec<T, Abi>>& value,
 basic_vec<T, Abi>* iptr);
```

23 Let *V* be *basic\_vec<T, Abi>*. Let *modf-vec* denote:

```
pair<V, V> modf-vec(const V& x) {
 T r1[V::size()];
 V r0([&](simd-size-type i) {
 return modf(V(x)[i], &r1[i]);
 });
 return {r0, V(r1)};
}
```

Let *ret* be a value of type *pair<V, V>* that is the same value as the result of calling *modf-vec*(*value*).

24 *Effects*: Sets *\*iptr* to *ret.second*.

25 *Returns*: *ret.first*.

#### 29.10.8.14 Bit manipulation

[simd.bit]

```
template<simd-vec-type V> constexpr V byteswap(const V& v) noexcept;
```

1 *Constraints*: The type *V::value\_type* models integral.

2 *Returns*: A *basic\_vec* object where the *i*<sup>th</sup> element is initialized to the result of *std::byteswap*(*v*[*i*]) for all *i* in the range *[0, V::size())*.

```
template<simd-integral V> constexpr V bit_reverse(const V& v) noexcept;
```

3 *Constraints*: The type *V::value\_type* is an unsigned integer type (6.9.3).

4 *Returns*: A *basic\_vec* object where the *i*<sup>th</sup> element is initialized to the result of *std::bit\_reverse*(*v*[*i*]) for all *i* in the range *[0, V::size())*.

```
template<simd-vec-type V> constexpr V bit_ceil(const V& v);
```

5 *Constraints*: The type *V::value\_type* is an unsigned integer type (6.9.3).

6 *Preconditions*: For every *i* in the range *[0, V::size())*, the smallest power of 2 greater than or equal to *v*[*i*] is representable as a value of type *V::value\_type*.

7 *Returns*: A *basic\_vec* object where the *i*<sup>th</sup> element is initialized to the result of *std::bit\_ceil*(*v*[*i*]) for all *i* in the range *[0, V::size())*.

8 *Remarks*: A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression (7.7.2).

```
template<simd-vec-type V> constexpr V bit_floor(const V& v) noexcept;
```

9 *Constraints*: The type *V::value\_type* is an unsigned integer type (6.9.3).



10 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `std::bit_floor(v[i])` for all  $i$  in the range `[0, V::size())`.

```
template<simd-vec-type V>
constexpr typename V::mask_type has_single_bit(const V& v) noexcept;
```

11 *Constraints:* The type `V::value_type` is an unsigned integer type (6.9.3).

12 *Returns:* A `basic_mask` object where the  $i^{\text{th}}$  element is initialized to the result of `std::has_single_bit(v[i])` for all  $i$  in the range `[0, V::size())`.

```
template<simd-integral VX, simd-integral VS>
constexpr VX shl(const VX& x, const VS& s) noexcept;
template<simd-integral VX, simd-integral VS>
constexpr VX shr(const VX& x, const VS& s) noexcept;
```

13 *Constraints:*

(13.1) — Each of `VX::value_type` and `VS::value_type` is a signed or unsigned integer type (6.9.3),

(13.2) — `VX::size() == VS::size()` is true, and

(13.3) — `sizeof(typename VX::value_type) == sizeof(typename VS::value_type)` is true.

14 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit_func(x[i], s[i])` for all  $i$  in the range `[0, VX::size())`, where `bit_func` is the corresponding scalar function from `<bit>`.

```
template<simd-integral V, class S>
constexpr V shl(const V& x, S s) noexcept;
template<simd-integral V, class S>
constexpr V shr(const V& x, S s) noexcept;
```

15 *Constraints:* Each of `VX::value_type` and `S` is a signed or unsigned integer type (6.9.3).

16 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit_func(x[i], s)` for all  $i$  in the range `[0, VX::size())`, where `bit_func` is the corresponding scalar function from `<bit>`.

```
template<simd-vec-type V0, simd-vec-type V1>
constexpr V0 rotl(const V0& v0, const V1& v1) noexcept;
template<simd-vec-type V0, simd-vec-type V1>
constexpr V0 rotr(const V0& v0, const V1& v1) noexcept;
template<simd-integral V0, simd-integral V1>
constexpr V0 bit_repeat(const V0& v0, const V1& v1);
```

17 *Constraints:*

(17.1) — The type `V0::value_type` is an unsigned integer type (6.9.3),

(17.2) — the type `V1::value_type` models integral,

(17.3) — `V0::size() == V1::size()` is true, and

(17.4) — `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is true.

18 *Preconditions:* For `bit_repeat`, `v1[i] > 0` is true for all  $i$  in the range `[0, V0::size())`.

19 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit_func(v0[i], static_cast<int>(v1[i]))` for all  $i$  in the range `[0, V0::size())`, where `bit_func` is the corresponding scalar function from `<bit>`.

20 *Remarks:* A function call expression that violates the precondition in the *Preconditions:* element is not a core constant expression (7.7.2).

```
template<simd-vec-type V> constexpr V rotl(const V& v, int s) noexcept;
template<simd-vec-type V> constexpr V rotr(const V& v, int s) noexcept;
template<simd-integral V> constexpr V bit_repeat(const V& v, int l);
```

21 *Constraints:* The type `V::value_type` is an unsigned integer type (6.9.3).

22 *Preconditions:* For `bit_repeat`, `l > 0` is true.

23 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit_func(v[i], s)` for all  $i$  in the range `[0, V::size())`, where `bit_func` is the corresponding scalar function from `<bit>`.

24 *Remarks:* A function call expression that violates the precondition in the *Preconditions:* element is not a core constant expression (7.7.2).

```
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> bit_width(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> countl_zero(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> countl_one(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> countr_zero(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> countr_one(const V& v) noexcept;
template<simd-vec-type V>
 constexpr rebind_t<make_signed_t<typename V::value_type>, V> popcount(const V& v) noexcept;
```

25 *Constraints:* The type `V::value_type` is an unsigned integer type (6.9.3).

26 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit-func(v[i])` for all  $i$  in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-integral V>
 constexpr V bit_compress(const V& v, const V& m) noexcept;
template<simd-integral V>
 constexpr V bit_expand(const V& v, const V& m) noexcept;
```

27 *Constraints:* The type `V::value_type` is an unsigned integer type (6.9.3).

28 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit-func(v[i], m[i])` for all  $i$  in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-integral V>
 constexpr V bit_compress(const V& v, typename V::value_type m) noexcept;
template<simd-integral V>
 constexpr V bit_expand(const V& v, typename V::value_type m) noexcept;
```

29 *Constraints:* The type `V::value_type` is an unsigned integer type (6.9.3).

30 *Returns:* A `basic_vec` object where the  $i^{\text{th}}$  element is initialized to the result of `bit-func(v[i], m)` for all  $i$  in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

### 29.10.8.15 Complex math

[simd.complex.math]

```
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> real(const V&) noexcept;
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> imag(const V&) noexcept;
```

```
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> abs(const V&);
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> arg(const V&);
template<simd-complex V>
 constexpr rebind_t<simd-complex-value-type<V>, V> norm(const V&);
template<simd-complex V> constexpr V conj(const V&);
template<simd-complex V> constexpr V proj(const V&);
```

```
template<simd-complex V> constexpr V exp(const V& v);
template<simd-complex V> constexpr V log(const V& v);
template<simd-complex V> constexpr V log10(const V& v);
```

```
template<simd-complex V> constexpr V sqrt(const V& v);
template<simd-complex V> constexpr V sin(const V& v);
template<simd-complex V> constexpr V asin(const V& v);
template<simd-complex V> constexpr V cos(const V& v);
template<simd-complex V> constexpr V acos(const V& v);
template<simd-complex V> constexpr V tan(const V& v);
```

```
template<simd-complex V> constexpr V atan(const V& v);
template<simd-complex V> constexpr V sinh(const V& v);
template<simd-complex V> constexpr V asinh(const V& v);
template<simd-complex V> constexpr V cosh(const V& v);
template<simd-complex V> constexpr V acosh(const V& v);
template<simd-complex V> constexpr V tanh(const V& v);
template<simd-complex V> constexpr V atanh(const V& v);
```

1 *Returns:* A `basic_vec` object `ret` where the  $i^{\text{th}}$  element is initialized to the result of `cmplx-func(v[i])` for all  $i$  in the range `[0, V::size())`, where `cmplx-func` is the corresponding function from `<complex>`. If in an invocation of `cmplx-func` for index  $i$  a domain, pole, or range error would occur, the value of `ret[i]` is unspecified.

2 *Remarks:* It is unspecified whether `errno` (19.4) is accessed.

```
template<simd-floating-point V>
rebind_t<complex<typename V::value_type>, V> polar(const V& x, const V& y = {});
```

```
template<simd-complex V> constexpr V pow(const V& x, const V& y);
```

3 *Returns:* A `basic_vec` object `ret` where the  $i^{\text{th}}$  element is initialized to the result of `cmplx-func(x[i], y[i])` for all  $i$  in the range `[0, V::size())`, where `cmplx-func` is the corresponding function from `<complex>`. If in an invocation of `cmplx-func` for index  $i$  a domain, pole, or range error would occur, the value of `ret[i]` is unspecified.

4 *Remarks:* It is unspecified whether `errno` (19.4) is accessed.

## 29.10.9 Class template `basic_mask`

[`simd.mask.class`]

### 29.10.9.1 Overview

[`simd.mask.overview`]

```
namespace std::simd {
 template<size_t Bytes, class Abi> class basic_mask {
 public:
 using value_type = bool;
 using abi_type = Abi;
 using iterator = simd-iterator<basic_mask>;
 using const_iterator = simd-iterator<const basic_mask>;

 constexpr iterator begin() noexcept { return { *this, 0 }; }
 constexpr const_iterator begin() const noexcept { return { *this, 0 }; }
 constexpr const_iterator cbegin() const noexcept { return { *this, 0 }; }
 constexpr default_sentinel_t end() const noexcept { return {}; }
 constexpr default_sentinel_t cend() const noexcept { return {}; }

 static constexpr integral_constant<simd-size-type, mask-size-v<Bytes, Abi>> size {};

 constexpr basic_mask() noexcept = default;

 // 29.10.9.2, basic_mask constructors
 constexpr explicit basic_mask(same_as<value_type> auto) noexcept;
 template<size_t UBytes, class UAbi>
 constexpr explicit basic_mask(const basic_mask<UBytes, UAbi>&) noexcept;
 template<class G>
 constexpr explicit basic_mask(G&& gen);
 template<same_as<bitset<size()>> T>
 constexpr basic_mask(const T& b) noexcept;
 template<unsigned_integral T>
 requires (!same_as<T, value_type>)
 constexpr explicit basic_mask(T val) noexcept;

 // 29.10.9.3, basic_mask subscript operators
 constexpr value_type operator[] (simd-size-type) const;
 template<simd-integral I>
 constexpr resize_t<I::size(), basic_mask> operator[] (const I& indices) const;
```

```

// 29.10.9.4, basic_mask unary operators
constexpr basic_mask operator!() const noexcept;
constexpr see below operator+() const noexcept;
constexpr see below operator-() const noexcept;
constexpr see below operator~() const noexcept;

// 29.10.9.5, basic_mask conversions
template<class U, class A>
 constexpr explicit(sizeof(U) != Bytes) operator basic_vec<U, A>() const noexcept;
constexpr bitset<size()> to_bitset() const noexcept;
constexpr unsigned long long to_ullong() const;

// 29.10.10.1, basic_mask binary operators
friend constexpr basic_mask
 operator&&(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator||(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator&(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator|(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator^(const basic_mask&, const basic_mask&) noexcept;

// 29.10.10.2, basic_mask compound assignment
friend constexpr basic_mask&
 operator&=(basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask&
 operator|=(basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask&
 operator^=(basic_mask&, const basic_mask&) noexcept;

// 29.10.10.3, basic_mask comparisons
friend constexpr basic_mask
 operator==(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator!=(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator>=(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator<=(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator>(const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask
 operator<(const basic_mask&, const basic_mask&) noexcept;

// 29.10.10.4, basic_mask exposition only conditional operators
friend constexpr basic_mask simd-select-impl(// exposition only
 const basic_mask&, const basic_mask&, const basic_mask&) noexcept;
friend constexpr basic_mask simd-select-impl(// exposition only
 const basic_mask&, same_as<bool> auto, same_as<bool> auto) noexcept;
template<class T0, class T1>
 friend constexpr vec<see below, size()>
 simd-select-impl(const basic_mask&, const T0&, const T1&) noexcept; // exposition only
};
}

```

<sup>1</sup> Every specialization of `basic_mask` is a complete type. The specialization of `basic_mask<Bytes, Abi>` is:

- (1.1) — disabled, if there is no vectorizable type `T` such that `Bytes` is equal to `sizeof(T)`,
- (1.2) — otherwise, enabled, if there exists a vectorizable type `T` and a value `N` in the range `[1, 64]` such that `Bytes` is equal to `sizeof(T)` and `Abi` names the ABI tag denoted by *decode-abi-t*<`T`, `N`>,
- (1.3) — otherwise, it is implementation-defined if such a specialization is enabled.

If `basic_mask<Bytes, Abi>` is disabled, the specialization has a deleted default constructor, deleted destructor, deleted copy constructor, and deleted copy assignment. In addition only the `value_type` and `abi_type` members are present.

If `basic_mask<Bytes, Abi>` is enabled, `basic_mask<Bytes, Abi>` is trivially copyable.

- 2 *Recommended practice:* Implementations should support implicit conversions between specializations of `basic_mask` and appropriate implementation-defined types.

[Note 1: Appropriate types are non-standard vector types which are available in the implementation. — end note]

### 29.10.9.2 Constructors

[simd.mask.ctor]

```
constexpr explicit basic_mask(same_as<value_type> auto) noexcept;
```

- 1 *Effects:* Initializes each element with `x`.

```
template<size_t UBytes, class UAbi>
constexpr explicit basic_mask(const basic_mask<UBytes, UAbi>& x) noexcept;
```

- 2 *Constraints:* `basic_mask<UBytes, UAbi>::size() == size()` is true.

- 3 *Effects:* Initializes the  $i^{\text{th}}$  element with `x[i]` for all  $i$  in the range of `[0, size())`.

```
template<class G> constexpr explicit basic_mask(G&& gen);
```

- 4 *Constraints:* The expression `gen(integral_constant<simd-size-type, i>())` is well-formed and its type is `bool` for all  $i$  in the range of `[0, size())`.

- 5 *Effects:* Initializes the  $i^{\text{th}}$  element with `gen(integral_constant<simd-size-type, i>())` for all  $i$  in the range of `[0, size())`.

- 6 *Remarks:* `gen` is invoked exactly once for each  $i$ , in increasing order of  $i$ .

```
template<same_as<bitset<size()>> T>
constexpr basic_mask(const T& b) noexcept;
```

- 7 *Effects:* Initializes the  $i^{\text{th}}$  element with `b[i]` for all  $i$  in the range `[0, size())`.

```
template<unsigned_integral T>
requires (!same_as<T, value_type>)
constexpr explicit basic_mask(T val) noexcept;
```

- 8 *Effects:* Initializes the first  $M$  elements to the corresponding bit values in `val`, where  $M$  is the smaller of `size()` and the number of bits in the value representation (6.9.1) of the type of `val`. If  $M$  is less than `size()`, the remaining elements are initialized to zero.

### 29.10.9.3 Subscript operator

[simd.mask.subscr]

```
constexpr value_type operator[](simd-size-type i) const;
```

- 1 *Preconditions:* `i >= 0 && i < size()` is true.

- 2 *Returns:* The value of the  $i^{\text{th}}$  element.

- 3 *Throws:* Nothing.

```
template<simd-integral I>
constexpr resize_t<I::size(), basic_mask> operator[](const I& indices) const;
```

- 4 *Effects:* Equivalent to: `return permute(*this, indices);`

### 29.10.9.4 Unary operators

[simd.mask.unary]

```
constexpr basic_mask operator!() const noexcept;
constexpr see below operator+() const noexcept;
constexpr see below operator-() const noexcept;
constexpr see below operator~() const noexcept;
```

- 1 Let `op` be the operator.

- 2 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to the results of applying `op` to `operator[] (i)` for all  $i$  in the range of `[0, size())`.

3 *Remarks:* If there exists a vectorizable signed integer type `I` such that `sizeof(I) == Bytes` is true, `operator+`, `operator-`, and `operator` return an enabled specialization `R` of `basic_vec` such that `R::value_type` denotes *integer-from*`<Bytes>` and `R::size() == size()` is true. Otherwise, these operators are defined as deleted and their return types are unspecified.

### 29.10.9.5 Conversions

[simd.mask.conv]

```
template<class U, class A>
```

```
constexpr explicit(sizeof(U) != Bytes) operator basic_vec<U, A>() const noexcept;
```

1 *Constraints:* `simd-size-v<U, A> == simd-size-v<T, Abi>`.

2 *Returns:* A data-parallel object where the  $i^{\text{th}}$  element is initialized to `static_cast<U>(operator[] (i))`.

```
constexpr bitset<size()> to_bitset() const noexcept;
```

3 *Returns:* A `bitset<size()>` object where the  $i^{\text{th}}$  element is initialized to `operator[] (i)` for all  $i$  in the range `[0, size())`.

```
constexpr unsigned long long to_ullong() const;
```

4 Let  $N$  be the width of `unsigned long long`.

5 *Preconditions:*

(5.1) — `size() <= N` is true, or

(5.2) — for all  $i$  in the range `[N, size())`, `operator[] (i)` returns false.

6 *Returns:* The integral value corresponding to the bits in `*this`.

7 *Throws:* Nothing.

### 29.10.10 basic\_mask non-member operations

[simd.mask.nonmembers]

#### 29.10.10.1 Binary operators

[simd.mask.binary]

```
friend constexpr basic_mask
```

```
operator&&(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask
```

```
operator||(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask
```

```
operator&(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask
```

```
operator|(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask
```

```
operator^(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

1 Let *op* be the operator.

2 *Returns:* A `basic_mask` object initialized with the results of applying *op* to `lhs` and `rhs` as a binary element-wise operation.

#### 29.10.10.2 Compound assignment

[simd.mask.cassign]

```
friend constexpr basic_mask&
```

```
operator&=(basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask&
```

```
operator|=(basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```
friend constexpr basic_mask&
```

```
operator^=(basic_mask& lhs, const basic_mask& rhs) noexcept;
```

1 Let *op* be the operator.

2 *Effects:* These operators apply *op* to `lhs` and `rhs` as a binary element-wise operation.

3 *Returns:* `lhs`.

#### 29.10.10.3 Comparisons

[simd.mask.comparison]

```
friend constexpr basic_mask
```

```
operator==(const basic_mask& lhs, const basic_mask& rhs) noexcept;
```

```

friend constexpr basic_mask
operator!=(const basic_mask& lhs, const basic_mask& rhs) noexcept;
friend constexpr basic_mask
operator>=(const basic_mask& lhs, const basic_mask& rhs) noexcept;
friend constexpr basic_mask
operator<=(const basic_mask& lhs, const basic_mask& rhs) noexcept;
friend constexpr basic_mask
operator>(const basic_mask& lhs, const basic_mask& rhs) noexcept;
friend constexpr basic_mask
operator<(const basic_mask& lhs, const basic_mask& rhs) noexcept;

```

1 Let *op* be the operator.

2 *Returns:* A `basic_mask` object initialized with the results of applying *op* to *lhs* and *rhs* as a binary element-wise operation.

#### 29.10.10.4 Exposition-only conditional operators

[simd.mask.cond]

```

friend constexpr basic_mask simd-select-impl(
const basic_mask& mask, const basic_mask& a, const basic_mask& b) noexcept;

```

1 *Returns:* A `basic_mask` object where the  $i^{\text{th}}$  element equals `mask[i] ? a[i] : b[i]` for all  $i$  in the range of `[0, size())`.

```

friend constexpr basic_mask
simd-select-impl(const basic_mask& mask, same_as<bool> auto a, same_as<bool> auto b) noexcept;

```

2 *Returns:* A `basic_mask` object where the  $i^{\text{th}}$  element equals `mask[i] ? a : b` for all  $i$  in the range of `[0, size())`.

```

template<class T0, class T1>
friend constexpr vec<see below, size()>
simd-select-impl(const basic_mask& mask, const T0& a, const T1& b) noexcept;

```

3 *Constraints:*

(3.1) — `same_as<T0, T1>` is true,

(3.2) — `T0` is a vectorizable type, and

(3.3) — `sizeof(T0) == Bytes`.

4 *Returns:* A `vec<T0, size()>` object where the  $i^{\text{th}}$  element equals `mask[i] ? a : b` for all  $i$  in the range of `[0, size())`.

#### 29.10.10.5 Reductions

[simd.mask.reductions]

```

template<size_t Bytes, class Abi>
constexpr bool all_of(const basic_mask<Bytes, Abi>& k) noexcept;

```

1 *Returns:* true if all boolean elements in *k* are true, otherwise false.

```

template<size_t Bytes, class Abi>
constexpr bool any_of(const basic_mask<Bytes, Abi>& k) noexcept;

```

2 *Returns:* true if at least one boolean element in *k* is true, otherwise false.

```

template<size_t Bytes, class Abi>
constexpr bool none_of(const basic_mask<Bytes, Abi>& k) noexcept;

```

3 *Returns:* `!any_of(k)`.

```

template<size_t Bytes, class Abi>
constexpr simd-size-type reduce_count(const basic_mask<Bytes, Abi>& k) noexcept;

```

4 *Returns:* The number of boolean elements in *k* that are true.

```

template<size_t Bytes, class Abi>
constexpr simd-size-type reduce_min_index(const basic_mask<Bytes, Abi>& k);

```

5 *Preconditions:* `any_of(k)` is true.

6 *Returns:* The lowest element index  $i$  where `k[i]` is true.

```
template<size_t Bytes, class Abi>
constexpr simd-size-type reduce_max_index(const basic_mask<Bytes, Abi>& k);
```

7     *Preconditions:* any\_of(k) is true.

8     *Returns:* The greatest element index *i* where k[*i*] is true.

```
constexpr bool all_of(same_as<bool> auto x) noexcept;
constexpr bool any_of(same_as<bool> auto x) noexcept;
constexpr simd-size-type reduce_count(same_as<bool> auto x) noexcept;
```

9     *Returns:* x.

```
constexpr bool none_of(same_as<bool> auto x) noexcept;
```

10    *Returns:* !x.

```
constexpr simd-size-type reduce_min_index(same_as<bool> auto x);
constexpr simd-size-type reduce_max_index(same_as<bool> auto x);
```

11    *Preconditions:* x is true.

12    *Returns:* 0.

## 29.11 C compatibility

[numerics.c]

### 29.11.1 Header <stdckdint.h> synopsis

[stdckdint.h.syn]

```
#define __STDC_VERSION__ 202311L
```

```
template<class type1, class type2, class type3>
bool ckd_add(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_sub(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_mul(type1* result, type2 a, type3 b);
```

1    SEE ALSO: ISO/IEC 9899:2024, 7.20

### 29.11.2 Checked integer operations

[numerics.c.ckdint]

```
template<class type1, class type2, class type3>
bool ckd_add(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_sub(type1* result, type2 a, type3 b);
template<class type1, class type2, class type3>
bool ckd_mul(type1* result, type2 a, type3 b);
```

1     *Mandates:* Each of the types type1, type2, and type3 is a signed or unsigned integer type (6.9.3).

2     *Remarks:* Each function template has the same semantics as the corresponding type-generic macro with the same name specified in SEE ALSO: ISO/IEC 9899:2024, 7.20.



## 30 Time library

[time]

### 30.1 General

[time.general]

- <sup>1</sup> This Clause describes the chrono library (30.2) and various C functions (30.15) that provide generally useful time utilities, as summarized in Table 130.

Table 130 — Time library summary [tab:time.summary]

| Subclause                                   | Header   |
|---------------------------------------------|----------|
| 30.3 <i>Cpp17Clock</i> requirements         |          |
| 30.4 Time-related traits                    | <chrono> |
| 30.5 Class template <code>duration</code>   |          |
| 30.6 Class template <code>time_point</code> |          |
| 30.7 Clocks                                 |          |
| 30.8 Civil calendar                         |          |
| 30.9 Class template <code>hh_mm_ss</code>   |          |
| 30.10 12/24 hour functions                  |          |
| 30.11 Time zones                            |          |
| 30.12 Formatting                            |          |
| 30.13 Parsing                               |          |
| 30.14 Hash support                          |          |
| 30.15 C library time utilities              | <ctime>  |

### 30.2 Header <chrono> synopsis

[time.syn]

```
#include <compare> // see 17.12.1

namespace std::chrono {
 // 30.5, class template duration
 template<class Rep, class Period = ratio<1>> class duration;

 // 30.6, class template time_point
 template<class Clock, class Duration = typename Clock::duration> class time_point;
}

namespace std {
 // 30.4.3, common_type specializations
 template<class Rep1, class Period1, class Rep2, class Period2>
 struct common_type<chrono::duration<Rep1, Period1>,
 chrono::duration<Rep2, Period2>>;

 template<class Clock, class Duration1, class Duration2>
 struct common_type<chrono::time_point<Clock, Duration1>,
 chrono::time_point<Clock, Duration2>>;
}

namespace std::chrono {
 // 30.4, customization traits
 template<class Rep> struct treat_as_floating_point;
 template<class Rep>
 constexpr bool treat_as_floating_point_v = treat_as_floating_point<Rep>::value;

 template<class Rep> struct duration_values;

 template<class T> struct is_clock;
 template<class T> constexpr bool is_clock_v = is_clock<T>::value;
```

```

// 30.5.6, duration arithmetic
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
 operator+(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
 operator-(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period, class Rep2>
 constexpr duration<common_type_t<Rep1, Rep2>, Period>
 operator*(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Rep2, class Period>
 constexpr duration<common_type_t<Rep1, Rep2>, Period>
 operator*(const Rep1& s, const duration<Rep2, Period>& d);
template<class Rep1, class Period, class Rep2>
 constexpr duration<common_type_t<Rep1, Rep2>, Period>
 operator/(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr common_type_t<Rep1, Rep2>
 operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period, class Rep2>
 constexpr duration<common_type_t<Rep1, Rep2>, Period>
 operator%(const duration<Rep1, Period>& d, const Rep2& s);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
 operator%(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);

// 30.5.7, duration comparisons
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr bool operator==(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr bool operator<(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr bool operator>(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Rep2, class Period2>
 requires see below
 constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);

// 30.5.8, conversions
template<class ToDuration, class Rep, class Period>
 constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period>
 constexpr ToDuration floor(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period>
 constexpr ToDuration ceil(const duration<Rep, Period>& d);
template<class ToDuration, class Rep, class Period>
 constexpr ToDuration round(const duration<Rep, Period>& d);

// 30.5.11, duration I/O
template<class charT, class traits, class Rep, class Period>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 const duration<Rep, Period>& d);

```

```

template<class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 duration<Rep, Period>& d,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// convenience typedefs
using nanoseconds = duration<signed integer type of at least 64 bits, nano>;
using microseconds = duration<signed integer type of at least 55 bits, micro>;
using milliseconds = duration<signed integer type of at least 45 bits, milli>;
using seconds = duration<signed integer type of at least 35 bits>;
using minutes = duration<signed integer type of at least 29 bits, ratio< 60>>;
using hours = duration<signed integer type of at least 23 bits, ratio<3600>>;
using days = duration<signed integer type of at least 25 bits,
 ratio_multiply<ratio<24>, hours::period>>;
using weeks = duration<signed integer type of at least 22 bits,
 ratio_multiply<ratio<7>, days::period>>;
using years = duration<signed integer type of at least 17 bits,
 ratio_multiply<ratio<146097, 400>, days::period>>;
using months = duration<signed integer type of at least 20 bits,
 ratio_divide<years::period, ratio<12>>>;

// 30.6.6, time_point arithmetic
template<class Clock, class Duration1, class Rep2, class Period2>
 constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
 operator+(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Rep1, class Period1, class Clock, class Duration2>
 constexpr time_point<Clock, common_type_t<duration<Rep1, Period1>, Duration2>>
 operator+(const duration<Rep1, Period1>& lhs, const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Rep2, class Period2>
 constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
 operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
template<class Clock, class Duration1, class Duration2>
 constexpr common_type_t<Duration1, Duration2>
 operator-(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);

// 30.6.7, time_point comparisons
template<class Clock, class Duration1, class Duration2>
 constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
 constexpr bool operator< (const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
 constexpr bool operator> (const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
 constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, class Duration2>
 constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
template<class Clock, class Duration1, three_way_comparable_with<Duration1> Duration2>
 constexpr auto operator<=>(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);

// 30.6.8, conversions
template<class ToDuration, class Clock, class Duration>
 constexpr time_point<Clock, ToDuration>
 time_point_cast(const time_point<Clock, Duration>& t);
template<class ToDuration, class Clock, class Duration>
 constexpr time_point<Clock, ToDuration> floor(const time_point<Clock, Duration>& tp);

```

```

template<class ToDuration, class Clock, class Duration>
 constexpr time_point<Clock, ToDuration> ceil(const time_point<Clock, Duration>& tp);
template<class ToDuration, class Clock, class Duration>
 constexpr time_point<Clock, ToDuration> round(const time_point<Clock, Duration>& tp);

// 30.5.10, specialized algorithms
template<class Rep, class Period>
 constexpr duration<Rep, Period> abs(duration<Rep, Period> d);

// 30.7.2, class system_clock
class system_clock;

template<class Duration>
 using sys_time = time_point<system_clock, Duration>;
using sys_seconds = sys_time<seconds>;
using sys_days = sys_time<days>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const sys_days& dp);

template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 sys_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.7.3, class utc_clock
class utc_clock;

template<class Duration>
 using utc_time = time_point<utc_clock, Duration>;
using utc_seconds = utc_time<seconds>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 utc_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

struct leap_second_info;

template<class Duration>
 leap_second_info get_leap_second_info(const utc_time<Duration>& ut);

// 30.7.4, class tai_clock
class tai_clock;

template<class Duration>
 using tai_time = time_point<tai_clock, Duration>;
using tai_seconds = tai_time<seconds>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);

```

```

template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 tai_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.7.5, class gps_clock
class gps_clock;

template<class Duration>
 using gps_time = time_point<gps_clock, Duration>;
using gps_seconds = gps_time<seconds>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 gps_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.7.6, type file_clock
using file_clock = see below;

template<class Duration>
 using file_time = time_point<file_clock, Duration>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const file_time<Duration>& tp);
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 file_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.7.7, class steady_clock
class steady_clock;

// 30.7.8, class high_resolution_clock
class high_resolution_clock;

// 30.7.9, local time
struct local_t {};
template<class Duration>
 using local_time = time_point<local_t, Duration>;
using local_seconds = local_time<seconds>;
using local_days = local_time<days>;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& tp);
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 local_time<Duration>& tp,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

```

```

// 30.7.10, time_point conversions
template<class DestClock, class SourceClock>
 struct clock_time_conversion;

template<class DestClock, class SourceClock, class Duration>
 auto clock_cast(const time_point<SourceClock, Duration>& t);

// 30.8.2, class last_spec
struct last_spec;

// 30.8.3, class day
class day;

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;

constexpr day operator+(const day& x, const days& y) noexcept;
constexpr day operator+(const days& x, const day& y) noexcept;
constexpr day operator-(const day& x, const days& y) noexcept;
constexpr days operator-(const day& x, const day& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const day& d);
template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 day& d, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.4, class month
class month;

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr strong_ordering operator<=>(const month& x, const month& y) noexcept;

constexpr month operator+(const month& x, const months& y) noexcept;
constexpr month operator+(const months& x, const month& y) noexcept;
constexpr month operator-(const month& x, const months& y) noexcept;
constexpr months operator-(const month& x, const month& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month& m);
template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 month& m, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.5, class year
class year;

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr strong_ordering operator<=>(const year& x, const year& y) noexcept;

constexpr year operator+(const year& x, const years& y) noexcept;
constexpr year operator+(const years& x, const year& y) noexcept;
constexpr year operator-(const year& x, const years& y) noexcept;
constexpr years operator-(const year& x, const year& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year& y);

```

```

template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year& y, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.6, class weekday
class weekday;

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;

constexpr weekday operator+(const weekday& x, const days& y) noexcept;
constexpr weekday operator+(const days& x, const weekday& y) noexcept;
constexpr weekday operator-(const weekday& x, const days& y) noexcept;
constexpr days operator-(const weekday& x, const weekday& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const weekday& wd);

template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 weekday& wd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.7, class weekday_indexed
class weekday_indexed;

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);

// 30.8.8, class weekday_last
class weekday_last;

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);

// 30.8.9, class month_day
class month_day;

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr strong_ordering operator<=>(const month_day& x, const month_day& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month_day& md);

template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 month_day& md, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.10, class month_day_last
class month_day_last;

```

```

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr strong_ordering operator<=(const month_day_last& x,
 const month_day_last& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);

// 30.8.11, class month_weekday
class month_weekday;

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);

// 30.8.12, class month_weekday_last
class month_weekday_last;

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);

// 30.8.13, class year_month
class year_month;

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr strong_ordering operator<=(const year_month& x, const year_month& y) noexcept;

constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month& ym);

template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year_month& ym, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.14, class year_month_day
class year_month_day;

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr strong_ordering operator<=(const year_month_day& x,
 const year_month_day& y) noexcept;

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;

```



```

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);

template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year_month_day& ymd,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

// 30.8.15, class year_month_day_last
class year_month_day_last;

constexpr bool operator==(const year_month_day_last& x,
 const year_month_day_last& y) noexcept;
constexpr strong_ordering operator<=>(const year_month_day_last& x,
 const year_month_day_last& y) noexcept;

constexpr year_month_day_last
 operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last
 operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last
 operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
constexpr year_month_day_last
 operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last
 operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last
 operator-(const year_month_day_last& ymdl, const years& dy) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);

// 30.8.16, class year_month_weekday
class year_month_weekday;

constexpr bool operator==(const year_month_weekday& x,
 const year_month_weekday& y) noexcept;

constexpr year_month_weekday
 operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday
 operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday
 operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
constexpr year_month_weekday
 operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday
 operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday
 operator-(const year_month_weekday& ymwd, const years& dy) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwd);

// 30.8.17, class year_month_weekday_last
class year_month_weekday_last;

constexpr bool operator==(const year_month_weekday_last& x,
 const year_month_weekday_last& y) noexcept;

```

```

constexpr year_month_weekday_last
 operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last
 operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last
 operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
constexpr year_month_weekday_last
 operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last
 operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last
 operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);

// 30.8.18, civil calendar conventional syntax operators
constexpr year_month
 operator/(const year& y, const month& m) noexcept;
constexpr year_month
 operator/(const year& y, int m) noexcept;
constexpr month_day
 operator/(const month& m, const day& d) noexcept;
constexpr month_day
 operator/(const month& m, int d) noexcept;
constexpr month_day
 operator/(int m, const day& d) noexcept;
constexpr month_day
 operator/(const day& d, const month& m) noexcept;
constexpr month_day
 operator/(const day& d, int m) noexcept;
constexpr month_day_last
 operator/(const month& m, last_spec) noexcept;
constexpr month_day_last
 operator/(int m, last_spec) noexcept;
constexpr month_day_last
 operator/(last_spec, const month& m) noexcept;
constexpr month_day_last
 operator/(last_spec, int m) noexcept;
constexpr month_weekday
 operator/(const month& m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday
 operator/(int m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday
 operator/(const weekday_indexed& wdi, const month& m) noexcept;
constexpr month_weekday
 operator/(const weekday_indexed& wdi, int m) noexcept;
constexpr month_weekday_last
 operator/(const month& m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last
 operator/(int m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last
 operator/(const weekday_last& wdl, const month& m) noexcept;
constexpr month_weekday_last
 operator/(const weekday_last& wdl, int m) noexcept;
constexpr year_month_day
 operator/(const year_month& ym, const day& d) noexcept;
constexpr year_month_day
 operator/(const year_month& ym, int d) noexcept;
constexpr year_month_day
 operator/(const year& y, const month_day& md) noexcept;
constexpr year_month_day
 operator/(int y, const month_day& md) noexcept;

```

```

constexpr year_month_day
 operator/(const month_day& md, const year& y) noexcept;
constexpr year_month_day
 operator/(const month_day& md, int y) noexcept;
constexpr year_month_day_last
 operator/(const year_month& ym, last_spec) noexcept;
constexpr year_month_day_last
 operator/(const year& y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last
 operator/(int y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last
 operator/(const month_day_last& mdl, const year& y) noexcept;
constexpr year_month_day_last
 operator/(const month_day_last& mdl, int y) noexcept;
constexpr year_month_weekday
 operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;
constexpr year_month_weekday
 operator/(const year& y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday
 operator/(int y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday
 operator/(const month_weekday& mwd, const year& y) noexcept;
constexpr year_month_weekday
 operator/(const month_weekday& mwd, int y) noexcept;
constexpr year_month_weekday_last
 operator/(const year_month& ym, const weekday_last& wdl) noexcept;
constexpr year_month_weekday_last
 operator/(const year& y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last
 operator/(int y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last
 operator/(const month_weekday_last& mwdl, const year& y) noexcept;
constexpr year_month_weekday_last
 operator/(const month_weekday_last& mwdl, int y) noexcept;

// 30.9, class template hh_mm_ss
template<class Duration> class hh_mm_ss;

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const hh_mm_ss<Duration>& hms);

// 30.10, 12/24 hour functions
constexpr bool is_am(const hours& h) noexcept;
constexpr bool is_pm(const hours& h) noexcept;
constexpr hours make12(const hours& h) noexcept;
constexpr hours make24(const hours& h, bool is_pm) noexcept;

// 30.11.2, time zone database
struct tzdb;
class tzdb_list;

// 30.11.2.3, time zone database access
const tzdb& get_tzdb();
tzdb_list& get_tzdb_list();
const time_zone* locate_zone(string_view tz_name);
const time_zone* current_zone();

// 30.11.2.4, remote time zone database support
const tzdb& reload_tzdb();
string remote_version();

// 30.11.3, exception classes
class nonexistent_local_time;

```

```

class ambiguous_local_time;

// 30.11.4, information classes
struct sys_info;
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const sys_info& si);

struct local_info;
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const local_info& li);

// 30.11.5, class time_zone
enum class choose {earliest, latest};
class time_zone;

bool operator==(const time_zone& x, const time_zone& y) noexcept;
strong_ordering operator<=>(const time_zone& x, const time_zone& y) noexcept;

// 30.11.6, class template zoned_traits
template<class T> struct zoned_traits;

// 30.11.7, class template zoned_time
template<class Duration, class TimeZonePtr = const time_zone*> class zoned_time;

using zoned_seconds = zoned_time<seconds>;

template<class Duration1, class Duration2, class TimeZonePtr>
 bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
 const zoned_time<Duration2, TimeZonePtr>& y);

template<class charT, class traits, class Duration, class TimeZonePtr>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os,
 const zoned_time<Duration, TimeZonePtr>& t);

// 30.11.8, leap second support
class leap_second;

constexpr bool operator==(const leap_second& x, const leap_second& y);
constexpr strong_ordering operator<=>(const leap_second& x, const leap_second& y);

template<class Duration>
 constexpr bool operator==(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
 constexpr bool operator< (const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
 constexpr bool operator< (const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
 constexpr bool operator> (const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
 constexpr bool operator> (const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
 constexpr bool operator<=(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
 constexpr bool operator<=(const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
 constexpr bool operator>=(const leap_second& x, const sys_time<Duration>& y);
template<class Duration>
 constexpr bool operator>=(const sys_time<Duration>& x, const leap_second& y);
template<class Duration>
 requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
 constexpr auto operator<=>(const leap_second& x, const sys_time<Duration>& y);

```

```

// 30.11.9, class time_zone_link
class time_zone_link;

bool operator==(const time_zone_link& x, const time_zone_link& y);
strong_ordering operator<=>(const time_zone_link& x, const time_zone_link& y);

// 30.12, formatting
template<class Duration> struct local-time-format-t; // exposition only
template<class Duration>
 local-time-format-t<Duration>
 local_time_format(local_time<Duration> time, const string* abbrev = nullptr,
 const seconds* offset_sec = nullptr);
}

namespace std {
 template<class Rep, class Period, class charT>
 struct formatter<chrono::duration<Rep, Period>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::sys_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::utc_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::tai_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::gps_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::file_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::local_time<Duration>, charT>;
 template<class Duration, class charT>
 struct formatter<chrono::local-time-format-t<Duration>, charT>;
 template<class charT> struct formatter<chrono::day, charT>;
 template<class charT> struct formatter<chrono::month, charT>;
 template<class charT> struct formatter<chrono::year, charT>;
 template<class charT> struct formatter<chrono::weekday, charT>;
 template<class charT> struct formatter<chrono::weekday_indexed, charT>;
 template<class charT> struct formatter<chrono::weekday_last, charT>;
 template<class charT> struct formatter<chrono::month_day, charT>;
 template<class charT> struct formatter<chrono::month_day_last, charT>;
 template<class charT> struct formatter<chrono::month_weekday, charT>;
 template<class charT> struct formatter<chrono::month_weekday_last, charT>;
 template<class charT> struct formatter<chrono::year_month, charT>;
 template<class charT> struct formatter<chrono::year_month_day, charT>;
 template<class charT> struct formatter<chrono::year_month_day_last, charT>;
 template<class charT> struct formatter<chrono::year_month_weekday, charT>;
 template<class charT> struct formatter<chrono::year_month_weekday_last, charT>;
 template<class Rep, class Period, class charT>
 struct formatter<chrono::hh_mm_ss<chrono::duration<Rep, Period>>, charT>;
 template<class charT> struct formatter<chrono::sys_info, charT>;
 template<class charT> struct formatter<chrono::local_info, charT>;
 template<class Duration, class TimeZonePtr, class charT>
 struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>;
}

namespace std::chrono {
 // 30.13, parsing
 template<class charT, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp);
 template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);
}

```

```

template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev);
template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev);

template<class charT, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp, minutes& offset);
template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 minutes& offset);

template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

// calendrical constants
inline constexpr last_spec last{};

inline constexpr weekday Sunday{0};
inline constexpr weekday Monday{1};
inline constexpr weekday Tuesday{2};
inline constexpr weekday Wednesday{3};
inline constexpr weekday Thursday{4};
inline constexpr weekday Friday{5};
inline constexpr weekday Saturday{6};

inline constexpr month January{1};
inline constexpr month February{2};
inline constexpr month March{3};
inline constexpr month April{4};
inline constexpr month May{5};
inline constexpr month June{6};
inline constexpr month July{7};
inline constexpr month August{8};
inline constexpr month September{9};
inline constexpr month October{10};
inline constexpr month November{11};
inline constexpr month December{12};
}

namespace std::inline literals::inline chrono_literals {
 // 30.5.9, suffixes for duration literals
 constexpr chrono::hours operator""h(unsigned long long);
 constexpr chrono::duration<unspecified, ratio<3600, 1>> operator""h(long double);

 constexpr chrono::minutes operator""min(unsigned long long);
 constexpr chrono::duration<unspecified, ratio<60, 1>> operator""min(long double);

 constexpr chrono::seconds operator""s(unsigned long long);
 constexpr chrono::duration<unspecified> operator""s(long double);

 constexpr chrono::milliseconds operator""ms(unsigned long long);
 constexpr chrono::duration<unspecified, milli> operator""ms(long double);

```

```

constexpr chrono::microseconds operator""us(unsigned long long);
constexpr chrono::duration<unspecified, micro> operator""us(long double);

constexpr chrono::nanoseconds operator""ns(unsigned long long);
constexpr chrono::duration<unspecified, nano> operator""ns(long double);

// 30.8.3.3, non-member functions
constexpr chrono::day operator""d(unsigned long long d) noexcept;

// 30.8.5.3, non-member functions
constexpr chrono::year operator""y(unsigned long long y) noexcept;
}

namespace std::chrono {
 using namespace literals::chrono_literals;
}

namespace std {
 // 30.14, hash support
 template<class T> struct hash;
 template<class Rep, class Period> struct hash<chrono::duration<Rep, Period>>;
 template<class Clock, class Duration> struct hash<chrono::time_point<Clock, Duration>>;
 template<> struct hash<chrono::day>;
 template<> struct hash<chrono::month>;
 template<> struct hash<chrono::year>;
 template<> struct hash<chrono::weekday>;
 template<> struct hash<chrono::weekday_indexed>;
 template<> struct hash<chrono::weekday_last>;
 template<> struct hash<chrono::month_day>;
 template<> struct hash<chrono::month_day_last>;
 template<> struct hash<chrono::month_weekday>;
 template<> struct hash<chrono::month_weekday_last>;
 template<> struct hash<chrono::year_month>;
 template<> struct hash<chrono::year_month_day>;
 template<> struct hash<chrono::year_month_day_last>;
 template<> struct hash<chrono::year_month_weekday>;
 template<> struct hash<chrono::year_month_weekday_last>;
 template<class Duration, class TimeZonePtr>
 struct hash<chrono::zoned_time<Duration, TimeZonePtr>>;
 template<> struct hash<chrono::leap_second>;
}

```

### 30.3 Cpp17Clock requirements

[time.clock.req]

- <sup>1</sup> A clock is a bundle consisting of a `duration`, a `time_point`, and a function `now()` to get the current `time_point`. The origin of the clock's `time_point` is referred to as the clock's *epoch*. A clock shall meet the requirements in Table 131.
- <sup>2</sup> In Table 131 C1 and C2 denote clock types. `t1` and `t2` are values returned by `C1::now()` where the call returning `t1` happens before (6.10.2) the call returning `t2` and both of these calls occur before `C1::time_point::max()`.

[Note 1: This means C1 did not wrap around between `t1` and `t2`. — end note]

Table 131 — Cpp17Clock requirements [tab:time.clock]

| Expression                | Return type                                                | Operational semantics                                  |
|---------------------------|------------------------------------------------------------|--------------------------------------------------------|
| <code>C1::rep</code>      | An arithmetic type or a class emulating an arithmetic type | The representation type of <code>C1::duration</code> . |
| <code>C1::period</code>   | a specialization of <code>ratio</code>                     | The tick period of the clock in seconds.               |
| <code>C1::duration</code> | <code>chrono::duration&lt;C1::rep, C1::period&gt;</code>   | The duration type of the clock.                        |



Table 131 — *Cpp17Clock* requirements (continued)

| Expression                  | Return type                                                                                          | Operational semantics                                                                                                                                                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>C1::time_point</code> | <code>chrono::time_point&lt;C1&gt;</code> or <code>chrono::time_point&lt;C2, C1::duration&gt;</code> | The <code>time_point</code> type of the clock. C1 and C2 shall refer to the same epoch.                                                                                                                                             |
| <code>C1::is_steady</code>  | <code>const bool</code>                                                                              | <code>true</code> if <code>t1 &lt;= t2</code> is always <code>true</code> and the time between clock ticks is constant, otherwise <code>false</code> .<br><code>C1::is_steady</code> shall be usable in constant expressions (7.7). |
| <code>C1::now()</code>      | <code>C1::time_point</code>                                                                          | Returns a <code>time_point</code> object representing the current point in time.                                                                                                                                                    |

<sup>3</sup> [Note 2: The relative difference in durations between those reported by a given clock and the SI definition is a measure of the quality of implementation. — end note]

<sup>4</sup> A type TC meets the *Cpp17TrivialClock* requirements if

(4.1) — TC meets the *Cpp17Clock* requirements,

(4.2) — the types `TC::rep`, `TC::duration`, and `TC::time_point` meet the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) and *Cpp17Swappable* (16.4.4.3) requirements and the requirements of numeric types (29.2),

[Note 3: This means, in particular, that operations on these types will not throw exceptions. — end note]

(4.3) — the function `TC::now()` does not throw exceptions, and

(4.4) — the type `TC::time_point::clock` meets the *Cpp17TrivialClock* requirements, recursively.

## 30.4 Time-related traits

[time.traits]

### 30.4.1 `treat_as_floating_point`

[time.traits.is.fp]

```
template<class Rep> struct treat_as_floating_point : is_floating_point<Rep> { };
```

<sup>1</sup> The `duration` template uses the `treat_as_floating_point` trait to help determine if a `duration` object can be converted to another `duration` with a different tick period. If `treat_as_floating_point_v<Rep>` is `true`, then implicit conversions are allowed among `durations`. Otherwise, the implicit convertibility depends on the tick periods of the `durations`.

[Note 1: The intention of this trait is to indicate whether a given class behaves like a floating-point type, and thus allows division of one value by another with acceptable loss of precision. If `treat_as_floating_point_v<Rep>` is `false`, `Rep` will be treated as if it behaved like an integral type for the purpose of these conversions. — end note]

### 30.4.2 `duration_values`

[time.traits.duration.values]

```
template<class Rep>
struct duration_values {
public:
 static constexpr Rep zero() noexcept;
 static constexpr Rep min() noexcept;
 static constexpr Rep max() noexcept;
};
```

<sup>1</sup> The `duration` template uses the `duration_values` trait to construct special values of the `duration`'s representation (`Rep`). This is done because the representation can be a class type with behavior that requires some other implementation to return these special values. In that case, the author of that class type should specialize `duration_values` to return the indicated values.

```
static constexpr Rep zero() noexcept;
```

<sup>2</sup> Returns: `Rep(0)`.

[Note 1: `Rep(0)` is specified instead of `Rep()` because `Rep()` can have some other meaning, such as an uninitialized value. — end note]



3 *Remarks:* The value returned shall be the additive identity.

```
static constexpr Rep min() noexcept;
```

4 *Returns:* `numeric_limits<Rep>::lowest()`.

5 *Remarks:* The value returned shall compare less than or equal to `zero()`.

```
static constexpr Rep max() noexcept;
```

6 *Returns:* `numeric_limits<Rep>::max()`.

7 *Remarks:* The value returned shall compare greater than `zero()`.

### 30.4.3 Specializations of `common_type`

[time.traits.specializations]

```
template<class Rep1, class Period1, class Rep2, class Period2>
struct common_type<chrono::duration<Rep1, Period1>, chrono::duration<Rep2, Period2>> {
 using type = chrono::duration<common_type_t<Rep1, Rep2>, see below>;
};
```

1 The period of the duration indicated by this specialization of `common_type` is the greatest common divisor of `Period1` and `Period2`.

[Note 1: This can be computed by forming a ratio of the greatest common divisor of `Period1::num` and `Period2::num` and the least common multiple of `Period1::den` and `Period2::den`. — end note]

2 [Note 2: The typedef name `type` is a synonym for the duration with the largest tick period possible where both duration arguments will convert to it without requiring a division operation. The representation of this type is intended to be able to hold any value resulting from this conversion with no truncation error, although floating-point durations can have round-off errors. — end note]

```
template<class Clock, class Duration1, class Duration2>
struct common_type<chrono::time_point<Clock, Duration1>, chrono::time_point<Clock, Duration2>> {
 using type = chrono::time_point<Clock, common_type_t<Duration1, Duration2>>;
};
```

3 The common type of two `time_point` types is a `time_point` with the same clock as the two types and the common type of their two durations.

### 30.4.4 Class template `is_clock`

[time.traits.is.clock]

```
template<class T> struct is_clock;
```

1 `is_clock` is a *Cpp17UnaryTypeTrait* (21.3.2) with a base characteristic of `true_type` if `T` meets the *Cpp17Clock* requirements (30.3), otherwise `false_type`. For the purposes of the specification of this trait, the extent to which an implementation determines that a type cannot meet the *Cpp17Clock* requirements is unspecified, except that as a minimum a type `T` shall not qualify as a *Cpp17Clock* unless it meets all of the following conditions:

- (1.1) — the *qualified-ids* `T::rep`, `T::period`, `T::duration`, and `T::time_point` are valid and each denotes a type (13.10.3),
- (1.2) — the expression `T::is_steady` is well-formed when treated as an unevaluated operand (7.2.3),
- (1.3) — the expression `T::now()` is well-formed when treated as an unevaluated operand.

2 The behavior of a program that adds specializations for `is_clock` is undefined.

## 30.5 Class template `duration`

[time.duration]

### 30.5.1 General

[time.duration.general]

1 A `duration` type measures time between two points in time (`time_points`). A `duration` has a representation which holds a count of ticks and a tick period. The tick period is the amount of time which occurs from one tick to the next, in units of seconds. It is expressed as a rational constant using the template `ratio`.

```
namespace std::chrono {
 template<class Rep, class Period = ratio<1>>
 class duration {
 public:
 using rep = Rep;
 using period = Period::type;
```

```

private:
 rep rep_; // exposition only

public:
 // 30.5.2, construct/copy/destroy
 constexpr duration() = default;
 template<class Rep2>
 constexpr explicit duration(const Rep2& r);
 template<class Rep2, class Period2>
 constexpr duration(const duration<Rep2, Period2>& d);
 ~duration() = default;
 duration(const duration&) = default;
 duration& operator=(const duration&) = default;

 // 30.5.3, observer
 constexpr rep count() const;

 // 30.5.4, arithmetic
 constexpr common_type_t<duration> operator+() const;
 constexpr common_type_t<duration> operator-() const;
 constexpr duration& operator++();
 constexpr duration operator++(int);
 constexpr duration& operator--();
 constexpr duration operator--(int);

 constexpr duration& operator+=(const duration& d);
 constexpr duration& operator-=(const duration& d);

 constexpr duration& operator*=(const rep& rhs);
 constexpr duration& operator/=(const rep& rhs);
 constexpr duration& operator%=(const rep& rhs);
 constexpr duration& operator%=(const duration& rhs);

 // 30.5.5, special values
 static constexpr duration zero() noexcept;
 static constexpr duration min() noexcept;
 static constexpr duration max() noexcept;
};

```

- 2 Rep shall be an arithmetic type or a class emulating an arithmetic type. If a specialization of **duration** is instantiated with a cv-qualified type or a specialization of **duration** as the argument for the template parameter Rep, the program is ill-formed.
- 3 If Period is not a specialization of **ratio**, the program is ill-formed. If **Period::num** is not positive, the program is ill-formed.
- 4 Members of **duration** do not throw exceptions other than those thrown by the indicated operations on their representations.
- 5 The defaulted copy constructor of **duration** shall be a constexpr function if and only if the required initialization of the member **rep\_** for copy and move, respectively, would be constexpr-suitable (9.2.6).
- 6 [Example 1:

```

 duration<long, ratio<60>> d0; // holds a count of minutes using a long
 duration<long long, milli> d1; // holds a count of milliseconds using a long long
 duration<double, ratio<1, 30>> d2; // holds a count with a tick period of $\frac{1}{30}$ of a second
 // (30 Hz) using a double

```

— end example]

### 30.5.2 Constructors

[time.duration.cons]

```

template<class Rep2>
 constexpr explicit duration(const Rep2& r);

```

- 1 Constraints: **is\_convertible\_v<const Rep2&, rep>** is true and

- (1.1) — `treat_as_floating_point_v<rep>` is true or  
 (1.2) — `treat_as_floating_point_v<Rep2>` is false.

[Example 1:

```
duration<int, milli> d(3); // OK
duration<int, milli> d2(3.5); // error
```

— end example]

2 *Effects:* Initializes `rep_` with `r`.

```
template<class Rep2, class Period2>
constexpr duration(const duration<Rep2, Period2>& d);
```

3 *Constraints:* `is_convertible_v<const Rep2&, rep>` is true. `ratio_divide<typename Period2::type, period>` is a valid ratio specialization. Either:

- (3.1) — `treat_as_floating_point_v<rep>` is true; or  
 (3.2) — `ratio_divide<Period2, period>::den` is 1 and `treat_as_floating_point_v<Rep2>` is false.

[Note 1: This requirement prevents implicit truncation errors when converting between integral-based `duration` types. Such a construction could easily lead to confusion about the value of the `duration`. — end note]

[Example 2:

```
duration<int, milli> ms(3);
duration<int, micro> us = ms; // OK
duration<int, milli> ms2 = us; // error
```

— end example]

4 *Effects:* Initializes `rep_` with `duration_cast<duration>(d).count()`.

### 30.5.3 Observer

[time.duration.observer]

```
constexpr rep count() const;
```

1 *Returns:* `rep_`.

### 30.5.4 Arithmetic

[time.duration.arithmetic]

```
constexpr common_type_t<duration> operator+() const;
```

1 *Returns:* `common_type_t<duration>(*this)`.

```
constexpr common_type_t<duration> operator-() const;
```

2 *Returns:* `common_type_t<duration>(-rep_)`.

```
constexpr duration& operator++();
```

3 *Effects:* Equivalent to: `++rep_`.

4 *Returns:* `*this`.

```
constexpr duration operator++(int);
```

5 *Effects:* Equivalent to: `return duration(rep_++);`

```
constexpr duration& operator--();
```

6 *Effects:* Equivalent to: `--rep_`.

7 *Returns:* `*this`.

```
constexpr duration operator--(int);
```

8 *Effects:* Equivalent to: `return duration(rep_--);`

```
constexpr duration& operator+=(const duration& d);
```

9 *Effects:* Equivalent to: `rep_ += d.count()`.

10 *Returns:* `*this`.

```
constexpr duration& operator--=(const duration& d);
```

11     *Effects:* Equivalent to: *rep\_* -= d.count().

12     *Returns:* \*this.

```
constexpr duration& operator*=(const rep& rhs);
```

13     *Effects:* Equivalent to: *rep\_* \*= rhs.

14     *Returns:* \*this.

```
constexpr duration& operator/=(const rep& rhs);
```

15     *Effects:* Equivalent to: *rep\_* /= rhs.

16     *Returns:* \*this.

```
constexpr duration& operator%=(const rep& rhs);
```

17     *Effects:* Equivalent to: *rep\_* %= rhs.

18     *Returns:* \*this.

```
constexpr duration& operator%=(const duration& rhs);
```

19     *Effects:* Equivalent to: *rep\_* %= rhs.count().

20     *Returns:* \*this.

### 30.5.5 Special values

[time.duration.special]

```
static constexpr duration zero() noexcept;
```

1     *Returns:* duration(duration\_values<rep>::zero()).

```
static constexpr duration min() noexcept;
```

2     *Returns:* duration(duration\_values<rep>::min()).

```
static constexpr duration max() noexcept;
```

3     *Returns:* duration(duration\_values<rep>::max()).

### 30.5.6 Non-member arithmetic

[time.duration.nonmember]

1 In the function descriptions that follow, unless stated otherwise, let CD represent the return type of the function.

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
operator+(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

2     *Returns:* CD(CD(lhs).count() + CD(rhs).count()).

```
template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
operator-(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

3     *Returns:* CD(CD(lhs).count() - CD(rhs).count()).

```
template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const duration<Rep1, Period>& d, const Rep2& s);
```

4     *Constraints:* is\_convertible\_v<const Rep2&, common\_type\_t<Rep1, Rep2>> is true.

5     *Returns:* CD(CD(d).count() \* s).

```
template<class Rep1, class Rep2, class Period>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator*(const Rep1& s, const duration<Rep2, Period>& d);
```

6     *Constraints:* is\_convertible\_v<const Rep1&, common\_type\_t<Rep1, Rep2>> is true.

7     *Returns:* d \* s.

```

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator/(const duration<Rep1, Period>& d, const Rep2& s);
8 Constraints: is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>> is true and Rep2 is
 not a specialization of duration.
9 Returns: CD(CD(d).count() / s).

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<Rep1, Rep2>
operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
10 Let CD be common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>.
11 Returns: CD(lhs).count() / CD(rhs).count().

template<class Rep1, class Period, class Rep2>
constexpr duration<common_type_t<Rep1, Rep2>, Period>
operator%(const duration<Rep1, Period>& d, const Rep2& s);
12 Constraints: is_convertible_v<const Rep2&, common_type_t<Rep1, Rep2>> is true and Rep2 is
 not a specialization of duration.
13 Returns: CD(CD(d).count() % s).

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
operator%(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
14 Returns: CD(CD(lhs).count() % CD(rhs).count()).

```

### 30.5.7 Comparisons

[time.duration.comparisons]

1 In the function descriptions that follow, CT represents common\_type\_t<A, B>, where A and B are the types of the two arguments to the function.

```

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator==(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
2 Returns: CT(lhs).count() == CT(rhs).count().

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
3 Returns: CT(lhs).count() < CT(rhs).count().

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
4 Returns: rhs < lhs.

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
5 Returns: !(rhs < lhs).

template<class Rep1, class Period1, class Rep2, class Period2>
constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
6 Returns: !(lhs < rhs).

template<class Rep1, class Period1, class Rep2, class Period2>
requires three_way_comparable<typename CT::rep>
constexpr auto operator<=>(const duration<Rep1, Period1>& lhs,
 const duration<Rep2, Period2>& rhs);
7 Returns: CT(lhs).count() <=> CT(rhs).count().

```

## 30.5.8 Conversions

[time.duration.cast]

```
template<class ToDuration, class Rep, class Period>
constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

1 *Constraints:* ToDuration is a specialization of duration.

2 *Returns:* Let CF be ratio\_divide<Period, typename ToDuration::period>, and CR be common\_type<typename ToDuration::rep, Rep, intmax\_t>::type.

(2.1) — If CF::num == 1 and CF::den == 1, returns  
           ToDuration(static\_cast<typename ToDuration::rep>(d.count()))

(2.2) — otherwise, if CF::num != 1 and CF::den == 1, returns  
           ToDuration(static\_cast<typename ToDuration::rep>(
                       static\_cast<CR>(d.count()) \* static\_cast<CR>(CF::num)))

(2.3) — otherwise, if CF::num == 1 and CF::den != 1, returns  
           ToDuration(static\_cast<typename ToDuration::rep>(
                       static\_cast<CR>(d.count()) / static\_cast<CR>(CF::den)))

(2.4) — otherwise, returns  
           ToDuration(static\_cast<typename ToDuration::rep>(
                       static\_cast<CR>(d.count()) \* static\_cast<CR>(CF::num) / static\_cast<CR>(CF::den)))

3 [Note 1: This function does not use any implicit conversions; all conversions are done with `static_cast`. It avoids multiplications and divisions when it is known at compile time that one or more arguments is 1. Intermediate computations are carried out in the widest representation and only converted to the destination representation at the final step. — end note]

```
template<class ToDuration, class Rep, class Period>
constexpr ToDuration floor(const duration<Rep, Period>& d);
```

4 *Constraints:* ToDuration is a specialization of duration.

5 *Returns:* The greatest result `t` representable in ToDuration for which `t <= d`.

```
template<class ToDuration, class Rep, class Period>
constexpr ToDuration ceil(const duration<Rep, Period>& d);
```

6 *Constraints:* ToDuration is a specialization of duration.

7 *Returns:* The least result `t` representable in ToDuration for which `t >= d`.

```
template<class ToDuration, class Rep, class Period>
constexpr ToDuration round(const duration<Rep, Period>& d);
```

8 *Constraints:* ToDuration is a specialization of duration and `treat_as_floating_point_v<typename ToDuration::rep>` is false.

9 *Returns:* The value of ToDuration that is closest to `d`. If there are two closest values, then return the value `t` for which `t % 2 == 0`.

## 30.5.9 Suffixes for duration literals

[time.duration.literals]

1 This subclause describes literal suffixes for constructing duration literals. The suffixes `h`, `min`, `s`, `ms`, `us`, `ns` denote duration values of the corresponding types `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, and `nanoseconds` respectively if they are applied to *integer-literals*.

2 If any of these suffixes are applied to a *floating-point-literal* the result is a `chrono::duration` literal with an unspecified floating-point representation.

3 If any of these suffixes are applied to an *integer-literal* and the resulting `chrono::duration` value cannot be represented in the result type because of overflow, the program is ill-formed.

4 [Example 1: The following code shows some duration literals.

```
using namespace std::chrono_literals;
auto constexpr aday=24h;
auto constexpr lesson=45min;
auto constexpr halfanhour=0.5h;
```

— end example]

```
constexpr chrono::hours operator""h(unsigned long long hours);
constexpr chrono::duration<unspecified, ratio<3600, 1>> operator""h(long double hours);
```

5 *Returns:* A duration literal representing hours hours.

```
constexpr chrono::minutes operator""min(unsigned long long minutes);
constexpr chrono::duration<unspecified, ratio<60, 1>> operator""min(long double minutes);
```

6 *Returns:* A duration literal representing minutes minutes.

```
constexpr chrono::seconds operator""s(unsigned long long sec);
constexpr chrono::duration<unspecified> operator""s(long double sec);
```

7 *Returns:* A duration literal representing sec seconds.

8 [Note 1: The same suffix `s` is used for `basic_string` but there is no conflict, since duration suffixes apply to numbers and string literal suffixes apply to character array literals. — end note]

```
constexpr chrono::milliseconds operator""ms(unsigned long long msec);
constexpr chrono::duration<unspecified, milli> operator""ms(long double msec);
```

9 *Returns:* A duration literal representing msec milliseconds.

```
constexpr chrono::microseconds operator""us(unsigned long long usec);
constexpr chrono::duration<unspecified, micro> operator""us(long double usec);
```

10 *Returns:* A duration literal representing usec microseconds.

```
constexpr chrono::nanoseconds operator""ns(unsigned long long nsec);
constexpr chrono::duration<unspecified, nano> operator""ns(long double nsec);
```

11 *Returns:* A duration literal representing nsec nanoseconds.

### 30.5.10 Algorithms

[time.duration.alg]

```
template<class Rep, class Period>
constexpr duration<Rep, Period> abs(duration<Rep, Period> d);
```

1 *Constraints:* `numeric_limits<Rep>::is_signed` is true.

2 *Returns:* If `d >= d.zero()`, return `d`, otherwise return `-d`.

### 30.5.11 I/O

[time.duration.io]

```
template<class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

1 *Effects:* Inserts the duration `d` onto the stream `os` as if it were implemented as follows:

```
basic_ostringstream<charT, traits> s;
s.flags(os.flags());
s.imbue(os.getloc());
s.precision(os.precision());
s << d.count() << units-suffix;
return os << s.str();
```

where *units-suffix* depends on the type `Period::type` as follows:

- (1.1) — If `Period::type` is `atto`, *units-suffix* is `"as"`.
- (1.2) — Otherwise, if `Period::type` is `femto`, *units-suffix* is `"fs"`.
- (1.3) — Otherwise, if `Period::type` is `pico`, *units-suffix* is `"ps"`.
- (1.4) — Otherwise, if `Period::type` is `nano`, *units-suffix* is `"ns"`.
- (1.5) — Otherwise, if `Period::type` is `micro`, it is implementation-defined whether *units-suffix* is `"µs"` (`"\u00b5\u0073"`) or `"us"`.
- (1.6) — Otherwise, if `Period::type` is `milli`, *units-suffix* is `"ms"`.
- (1.7) — Otherwise, if `Period::type` is `centi`, *units-suffix* is `"cs"`.
- (1.8) — Otherwise, if `Period::type` is `deci`, *units-suffix* is `"ds"`.

- (1.9) — Otherwise, if `Period::type` is `ratio<1>`, *units-suffix* is "s".
- (1.10) — Otherwise, if `Period::type` is `deca`, *units-suffix* is "das".
- (1.11) — Otherwise, if `Period::type` is `hecto`, *units-suffix* is "hs".
- (1.12) — Otherwise, if `Period::type` is `kilo`, *units-suffix* is "ks".
- (1.13) — Otherwise, if `Period::type` is `mega`, *units-suffix* is "Ms".
- (1.14) — Otherwise, if `Period::type` is `giga`, *units-suffix* is "Gs".
- (1.15) — Otherwise, if `Period::type` is `tera`, *units-suffix* is "Ts".
- (1.16) — Otherwise, if `Period::type` is `peta`, *units-suffix* is "Ps".
- (1.17) — Otherwise, if `Period::type` is `exa`, *units-suffix* is "Es".
- (1.18) — Otherwise, if `Period::type` is `ratio<60>`, *units-suffix* is "min".
- (1.19) — Otherwise, if `Period::type` is `ratio<3600>`, *units-suffix* is "h".
- (1.20) — Otherwise, if `Period::type` is `ratio<86400>`, *units-suffix* is "d".
- (1.21) — Otherwise, if `Period::type::den == 1`, *units-suffix* is "[num]s".
- (1.22) — Otherwise, *units-suffix* is "[num/den]s".

In the list above, the use of *num* and *den* refers to the static data members of `Period::type`, which are converted to arrays of `charT` using a decimal conversion with no leading zeroes.

2 *Returns:* `os`.

```
template<class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 duration<Rep, Period>& d,
 basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

3 *Effects:* Attempts to parse the input stream `is` into the duration `d` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid duration, `is.setstate(ios_base::failbit)` is called and `d` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

4 *Returns:* `is`.

## 30.6 Class template `time_point`

[time.point]

### 30.6.1 General

[time.point.general]

```
namespace std::chrono {
 template<class Clock, class Duration = typename Clock::duration>
 class time_point {
 public:
 using clock = Clock;
 using duration = Duration;
 using rep = duration::rep;
 using period = duration::period;

 private:
 duration d_; // exposition only

 public:
 // 30.6.2, construct
 constexpr time_point(); // has value epoch
 constexpr explicit time_point(const duration& d); // same as time_point() + d
 template<class Duration2>
 constexpr time_point(const time_point<clock, Duration2>& t);

 // 30.6.3, observer
 constexpr duration time_since_epoch() const;
```



```

// 30.6.4, arithmetic
constexpr time_point& operator++();
constexpr time_point operator++(int);
constexpr time_point& operator--();
constexpr time_point operator--(int);
constexpr time_point& operator+=(const duration& d);
constexpr time_point& operator-=(const duration& d);

// 30.6.5, special values
static constexpr time_point min() noexcept;
static constexpr time_point max() noexcept;
};
}

```

1 If Duration is not a specialization of duration, the program is ill-formed.

### 30.6.2 Constructors

[time.point.cons]

```
constexpr time_point();
```

1 *Effects:* Initializes  $d_$  with `duration::zero()`. Such a `time_point` object represents the epoch.

```
constexpr explicit time_point(const duration& d);
```

2 *Effects:* Initializes  $d_$  with  $d$ . Such a `time_point` object represents the epoch +  $d$ .

```
template<class Duration2>
constexpr time_point(const time_point<clock, Duration2>& t);
```

3 *Constraints:* `is_convertible_v<Duration2, duration>` is true.

4 *Effects:* Initializes  $d_$  with `t.time_since_epoch()`.

### 30.6.3 Observer

[time.point.observer]

```
constexpr duration time_since_epoch() const;
```

1 *Returns:*  $d_$ .

### 30.6.4 Arithmetic

[time.point.arithmetic]

```
constexpr time_point& operator++();
```

1 *Effects:* Equivalent to: `++ $d_$` .

2 *Returns:* `*this`.

```
constexpr time_point operator++(int);
```

3 *Effects:* Equivalent to: `return time_point{ $d_++$ };`

```
constexpr time_point& operator--();
```

4 *Effects:* Equivalent to: `-- $d_$` .

5 *Returns:* `*this`.

```
constexpr time_point operator--(int);
```

6 *Effects:* Equivalent to: `return time_point{ $d_--$ };`

```
constexpr time_point& operator+=(const duration& d);
```

7 *Effects:* Equivalent to:  `$d_ += d$` .

8 *Returns:* `*this`.

```
constexpr time_point& operator-=(const duration& d);
```

9 *Effects:* Equivalent to:  `$d_ -= d$` .

10 *Returns:* `*this`.

**30.6.5 Special values****[time.point.special]**

```
static constexpr time_point min() noexcept;
```

1     *Returns:* time\_point(duration::min()).

```
static constexpr time_point max() noexcept;
```

2     *Returns:* time\_point(duration::max()).

**30.6.6 Non-member arithmetic****[time.point.nonmember]**

```
template<class Clock, class Duration1, class Rep2, class Period2>
constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
operator+(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

1     *Returns:* CT(lhs.time\_since\_epoch() + rhs), where CT is the type of the return value.

```
template<class Rep1, class Period1, class Clock, class Duration2>
constexpr time_point<Clock, common_type_t<duration<Rep1, Period1>, Duration2>>
operator+(const duration<Rep1, Period1>& lhs, const time_point<Clock, Duration2>& rhs);
```

2     *Returns:* rhs + lhs.

```
template<class Clock, class Duration1, class Rep2, class Period2>
constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

3     *Returns:* CT(lhs.time\_since\_epoch() - rhs), where CT is the type of the return value.

```
template<class Clock, class Duration1, class Duration2>
constexpr common_type_t<Duration1, Duration2>
operator-(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

4     *Returns:* lhs.time\_since\_epoch() - rhs.time\_since\_epoch().

**30.6.7 Comparisons****[time.point.comparisons]**

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

1     *Returns:* lhs.time\_since\_epoch() == rhs.time\_since\_epoch().

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator<(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

2     *Returns:* lhs.time\_since\_epoch() < rhs.time\_since\_epoch().

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator>(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

3     *Returns:* rhs < lhs.

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

4     *Returns:* !(rhs < lhs).

```
template<class Clock, class Duration1, class Duration2>
constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

5     *Returns:* !(lhs < rhs).

```
template<class Clock, class Duration1,
 three_way_comparable_with<Duration1> Duration2>
constexpr auto operator<=>(const time_point<Clock, Duration1>& lhs,
 const time_point<Clock, Duration2>& rhs);
```

6 *Returns:* lhs.time\_since\_epoch() <=> rhs.time\_since\_epoch().

### 30.6.8 Conversions

[time.point.cast]

```
template<class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration> time_point_cast(const time_point<Clock, Duration>& t);
```

1 *Constraints:* ToDuration is a specialization of duration.

2 *Returns:*

```
time_point<Clock, ToDuration>(duration_cast<ToDuration>(t.time_since_epoch()))
```

```
template<class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration> floor(const time_point<Clock, Duration>& tp);
```

3 *Constraints:* ToDuration is a specialization of duration.

4 *Returns:* time\_point<Clock, ToDuration>(floor<ToDuration>(tp.time\_since\_epoch())).

```
template<class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration> ceil(const time_point<Clock, Duration>& tp);
```

5 *Constraints:* ToDuration is a specialization of duration.

6 *Returns:* time\_point<Clock, ToDuration>(ceil<ToDuration>(tp.time\_since\_epoch())).

```
template<class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration> round(const time_point<Clock, Duration>& tp);
```

7 *Constraints:* ToDuration is a specialization of duration, and treat\_as\_floating\_point\_v<typename ToDuration::rep> is false.

8 *Returns:* time\_point<Clock, ToDuration>(round<ToDuration>(tp.time\_since\_epoch())).

## 30.7 Clocks

[time.clock]

### 30.7.1 General

[time.clock.general]

1 The types defined in 30.7 meet the *Cpp17TrivialClock* requirements (30.3) unless otherwise specified.

### 30.7.2 Class system\_clock

[time.clock.system]

#### 30.7.2.1 Overview

[time.clock.system.overview]

```
namespace std::chrono {
 class system_clock {
 public:
 using rep = see below;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<system_clock>;
 static constexpr bool is_steady = unspecified;

 static time_point now() noexcept;

 // mapping to/from C type time_t
 static time_t to_time_t (const time_point& t) noexcept;
 static time_point from_time_t(time_t t) noexcept;
 };
}
```

1 Objects of type `system_clock` represent wall clock time from the system-wide realtime clock. Objects of type `sys_time<Duration>` measure time since 1970-01-01 00:00:00 UTC excluding leap seconds. This measure is commonly referred to as *Unix time*. This measure facilitates an efficient mapping between `sys_time` and calendar types (30.8).

[Example 1:

`sys_seconds{sys_days{1970y/January/1}}.time_since_epoch()` is 0s.

`sys_seconds{sys_days{2000y/January/1}}.time_since_epoch()` is 946'684'800s, which is 10'957 \* 86'400s.

— end example]

### 30.7.2.2 Members

[time.clock.system.members]

using system\_clock::rep = unspecified;

1 *Constraints:* `system_clock::duration::min() < system_clock::duration::zero()` is true.

[Note 1: This implies that `rep` is a signed type. — end note]

static time\_t to\_time\_t(const time\_point& t) noexcept;

2 *Returns:* A `time_t` object that represents the same point in time as `t` when both values are restricted to the coarser of the precisions of `time_t` and `time_point`. It is implementation-defined whether values are rounded or truncated to the required precision.

static time\_point from\_time\_t(time\_t t) noexcept;

3 *Returns:* A `time_point` object that represents the same point in time as `t` when both values are restricted to the coarser of the precisions of `time_t` and `time_point`. It is implementation-defined whether values are rounded or truncated to the required precision.

### 30.7.2.3 Non-member functions

[time.clock.system.nonmembers]

template<class charT, class traits, class Duration>

basic\_ostream<charT, traits>&

operator<<(basic\_ostream<charT, traits>& os, const sys\_time<Duration>& tp);

1 *Constraints:* `treat_as_floating_point_v<typename Duration::rep>` is false, and `Duration{1} < days{1}` is true.

2 *Effects:* Equivalent to:

return os << format(os.getloc(), *STATICALLY-WIDEN*<charT>("{:L%F %T}"), tp);

3 [Example 1:

cout << sys\_seconds{0s} << '\n'; // 1970-01-01 00:00:00

cout << sys\_seconds{946'684'800s} << '\n'; // 2000-01-01 00:00:00

cout << sys\_seconds{946'688'523s} << '\n'; // 2000-01-01 01:02:03

— end example]

template<class charT, class traits>

basic\_ostream<charT, traits>&

operator<<(basic\_ostream<charT, traits>& os, const sys\_days& dp);

4 *Effects:* `os << year_month_day{dp}`.

5 *Returns:* `os`.

template<class charT, class traits, class Duration, class Alloc = allocator<charT>>

basic\_istream<charT, traits>&

from\_stream(basic\_istream<charT, traits>& is, const charT\* fmt,

sys\_time<Duration>& tp, basic\_string<charT, traits, Alloc>\* abbrev = nullptr,

minutes\* offset = nullptr);

6 *Effects:* Attempts to parse the input stream `is` into the `sys_time` `tp` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` is called and `tp` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

7 *Returns:* `is`.

**30.7.3 Class utc\_clock****[time.clock.utc]****30.7.3.1 Overview****[time.clock.utc.overview]**

```

namespace std::chrono {
 class utc_clock {
 public:
 using rep = a signed arithmetic type;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<utc_clock>;
 static constexpr bool is_steady = unspecified;

 static time_point now();

 template<class Duration>
 static sys_time<common_type_t<Duration, seconds>>
 to_sys(const utc_time<Duration>& t);
 template<class Duration>
 static utc_time<common_type_t<Duration, seconds>>
 from_sys(const sys_time<Duration>& t);
 };
}

```

- <sup>1</sup> In contrast to `sys_time`, which does not take leap seconds into account, `utc_clock` and its associated `time_point`, `utc_time`, count time, including leap seconds, since 1970-01-01 00:00:00 UTC.

[*Note 1:* The UTC time standard began on 1972-01-01 00:00:10 TAI. To measure time since this epoch instead, one can add/subtract the constant `sys_days{1972y/1/1}` - `sys_days{1970y/1/1}` (63'072'000s) from the `utc_time`. — *end note*]

[*Example 1:*

`clock_cast<utc_clock>(sys_seconds{sys_days{1970y/January/1}}).time_since_epoch()` is 0s.

`clock_cast<utc_clock>(sys_seconds{sys_days{2000y/January/1}}).time_since_epoch()` is 946'684'822s, which is  $10'957 * 86'400s + 22s$ .

— *end example*]

- <sup>2</sup> `utc_clock` is not a *Cpp17TrivialClock* unless the implementation can guarantee that `utc_clock::now()` does not propagate an exception.

[*Note 2:* `noexcept(from_sys(system_clock::now()))` is false. — *end note*]

**30.7.3.2 Member functions****[time.clock.utc.members]**

```
static time_point now();
```

- <sup>1</sup> *Returns:* `from_sys(system_clock::now())`, or a more accurate value of `utc_time`.

```

template<class Duration>
static sys_time<common_type_t<Duration, seconds>>
 to_sys(const utc_time<Duration>& u);

```

- <sup>2</sup> *Returns:* A `sys_time` `t`, such that `from_sys(t) == u` if such a mapping exists. Otherwise `u` represents a `time_point` during a positive leap second insertion, the conversion counts that leap second as not inserted, and the last representable value of `sys_time` prior to the insertion of the leap second is returned.

```

template<class Duration>
static utc_time<common_type_t<Duration, seconds>>
 from_sys(const sys_time<Duration>& t);

```

- <sup>3</sup> *Returns:* A `utc_time` `u`, such that `u.time_since_epoch() - t.time_since_epoch()` is equal to the sum of leap seconds that were inserted between `t` and 1970-01-01. If `t` is exactly the date of leap second insertion, then the conversion counts that leap second as inserted.

[*Example 1:*

```

auto t = sys_days{July/1/2015} - 2ns;
auto u = utc_clock::from_sys(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
t += 1ns;

```

```

 u = utc_clock::from_sys(t);
 assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
 t += 1ns;
 u = utc_clock::from_sys(t);
 assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
 t += 1ns;
 u = utc_clock::from_sys(t);
 assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
— end example]

```

### 30.7.3.3 Non-member functions

[time.clock.utc.nonmembers]

```

template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);

```

1 *Effects:* Equivalent to:

```
 return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%F %T}"), t);
```

2 [*Example 1:*

```

 auto t = sys_days{July/1/2015} - 500ms;
 auto u = clock_cast<utc_clock>(t);
 for (auto i = 0; i < 8; ++i, u += 250ms)
 cout << u << " UTC\n";

```

Produces this output:

```

2015-06-30 23:59:59.500 UTC
2015-06-30 23:59:59.750 UTC
2015-06-30 23:59:60.000 UTC
2015-06-30 23:59:60.250 UTC
2015-06-30 23:59:60.500 UTC
2015-06-30 23:59:60.750 UTC
2015-07-01 00:00:00.000 UTC
2015-07-01 00:00:00.250 UTC

```

— end example]

```

template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 utc_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

```

3 *Effects:* Attempts to parse the input stream *is* into the *utc\_time* *tp* using the format flags given in the NTCTS *fmt* as specified in 30.13. If the parse fails to decode a valid date, *is.setstate(ios\_base::failbit)* is called and *tp* is not modified. If *%Z* is used and successfully parsed, that value will be assigned to *\*abbrev* if *abbrev* is non-null. If *%z* (or a modified variant) is used and successfully parsed, that value will be assigned to *\*offset* if *offset* is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to *tp*.

4 *Returns:* *is*.

```

struct leap_second_info {
 bool is_leap_second;
 seconds elapsed;
};

```

5 The type *leap\_second\_info* has data members and special members specified above. It has no base classes or members other than those specified.

```

template<class Duration>
 leap_second_info get_leap_second_info(const utc_time<Duration>& ut);

```

6 *Returns:* A *leap\_second\_info* *lsi*, where *lsi.is\_leap\_second* is *true* if *ut* is during a positive leap second insertion, and otherwise *false*. *lsi.elapsed* is the sum of leap seconds between 1970-01-01 and *ut*. If *lsi.is\_leap\_second* is *true*, the leap second referred to by *ut* is included in the sum.

### 30.7.4 Class tai\_clock

[time.clock.tai]

#### 30.7.4.1 Overview

[time.clock.tai.overview]

```

namespace std::chrono {
 class tai_clock {
 public:
 using rep = a signed arithmetic type;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<tai_clock>;
 static constexpr bool is_steady = unspecified;

 static time_point now();

 template<class Duration>
 static utc_time<common_type_t<Duration, seconds>>
 to_utc(const tai_time<Duration>&) noexcept;
 template<class Duration>
 static tai_time<common_type_t<Duration, seconds>>
 from_utc(const utc_time<Duration>&) noexcept;
 };
}

```

<sup>1</sup> The clock `tai_clock` measures seconds since 1958-01-01 00:00:00 and is offset 10s ahead of UTC at this date. That is, 1958-01-01 00:00:00 TAI is equivalent to 1957-12-31 23:59:50 UTC. Leap seconds are not inserted into TAI. Therefore every time a leap second is inserted into UTC, UTC shifts another second with respect to TAI. For example by 2000-01-01 there had been 22 positive and 0 negative leap seconds inserted so 2000-01-01 00:00:00 UTC is equivalent to 2000-01-01 00:00:32 TAI (22s plus the initial 10s offset).

<sup>2</sup> `tai_clock` is not a *Cpp17TrivialClock* unless the implementation can guarantee that `tai_clock::now()` does not propagate an exception.

[Note 1: `noexcept(from_utc(utc_clock::now()))` is false. — end note]

#### 30.7.4.2 Member functions

[time.clock.tai.members]

```
static time_point now();
```

<sup>1</sup> *Returns:* `from_utc(utc_clock::now())`, or a more accurate value of `tai_time`.

```

template<class Duration>
 static utc_time<common_type_t<Duration, seconds>>
 to_utc(const tai_time<Duration>& t) noexcept;

```

<sup>2</sup> *Returns:*

```
utc_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 378691210s
```

[Note 1:

```
378691210s == sys_days{1970y/January/1} - sys_days{1958y/January/1} + 10s
```

— end note]

```

template<class Duration>
 static tai_time<common_type_t<Duration, seconds>>
 from_utc(const utc_time<Duration>& t) noexcept;

```

<sup>3</sup> *Returns:*

```
tai_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 378691210s
```

[Note 2:

```
378691210s == sys_days{1970y/January/1} - sys_days{1958y/January/1} + 10s
```

— end note]

## 30.7.4.3 Non-member functions

[time.clock.tai.nonmembers]

```
template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);
```

1 *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%F %T}"), t);
```

2 [*Example 1:*

```
auto st = sys_days{2000y/January/1};
auto tt = clock_cast<tai_clock>(st);
cout << format("{0:%F %T %Z} == {1:%F %T %Z}\n", st, tt);
```

Produces this output:

```
2000-01-01 00:00:00 UTC == 2000-01-01 00:00:32 TAI
```

— end example]

```
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 tai_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

3 *Effects:* Attempts to parse the input stream *is* into the *tai\_time* *tp* using the format flags given in the NTCTS *fmt* as specified in 30.13. If the parse fails to decode a valid date, *is.setstate(ios\_base::failbit)* is called and *tp* is not modified. If *%Z* is used and successfully parsed, that value will be assigned to *\*abbrev* if *abbrev* is non-null. If *%z* (or a modified variant) is used and successfully parsed, that value will be assigned to *\*offset* if *offset* is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to *tp*.

4 *Returns:* *is*.

30.7.5 Class *gps\_clock*

[time.clock.gps]

## 30.7.5.1 Overview

[time.clock.gps.overview]

```
namespace std::chrono {
class gps_clock {
public:
 using rep = a signed arithmetic type;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<gps_clock>;
 static constexpr bool is_steady = unspecified;

 static time_point now();

 template<class Duration>
 static utc_time<common_type_t<Duration, seconds>>
 to_utc(const gps_time<Duration>&) noexcept;
 template<class Duration>
 static gps_time<common_type_t<Duration, seconds>>
 from_utc(const utc_time<Duration>&) noexcept;
};
}
```

1 The clock *gps\_clock* measures seconds since the first Sunday of January, 1980 00:00:00 UTC. Leap seconds are not inserted into GPS. Therefore every time a leap second is inserted into UTC, UTC shifts another second with respect to GPS. Aside from the offset from 1958y/January/1 to 1980y/January/Sunday[1], GPS is behind TAI by 19s due to the 10s offset between 1958 and 1970 and the additional 9 leap seconds inserted between 1970 and 1980.

2 *gps\_clock* is not a *Cpp17TrivialClock* unless the implementation can guarantee that *gps\_clock::now()* does not propagate an exception.

[Note 1: *noexcept(from\_utc(utc\_clock::now()))* is false. — end note]



## 30.7.5.2 Member functions

[time.clock.gps.members]

```
static time_point now();
```

1 *Returns:* from\_utc(utc\_clock::now()), or a more accurate value of gps\_time.

```
template<class Duration>
static utc_time<common_type_t<Duration, seconds>>
to_utc(const gps_time<Duration>& t) noexcept;
```

2 *Returns:*

```
 utc_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 315964809s
```

[Note 1:

```
 315964809s == sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1} + 9s
```

— end note]

```
template<class Duration>
static gps_time<common_type_t<Duration, seconds>>
from_utc(const utc_time<Duration>& t) noexcept;
```

3 *Returns:*

```
 gps_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 315964809s
```

[Note 2:

```
 315964809s == sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1} + 9s
```

— end note]

## 30.7.5.3 Non-member functions

[time.clock.gps.nonmembers]

```
template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);
```

1 *Effects:* Equivalent to:

```
 return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%F %T}"), t);
```

2 [Example 1:

```
 auto st = sys_days{2000y/January/1};
 auto gt = clock_cast<gps_clock>(st);
 cout << format("{0:%F %T %Z} == {1:%F %T %Z}\n", st, gt);
```

Produces this output:

```
 2000-01-01 00:00:00 UTC == 2000-01-01 00:00:13 GPS
```

— end example]

```
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 gps_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

3 *Effects:* Attempts to parse the input stream is into the gps\_time tp using the format flags given in the NTCTS fmt as specified in 30.13. If the parse fails to decode a valid date, is.setstate(ios\_base::failbit) is called and tp is not modified. If %Z is used and successfully parsed, that value will be assigned to \*abbrev if abbrev is non-null. If %z (or a modified variant) is used and successfully parsed, that value will be assigned to \*offset if offset is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to tp.

4 *Returns:* is.

## 30.7.6 Type file\_clock

[time.clock.file]

## 30.7.6.1 Overview

[time.clock.file.overview]

```
namespace std::chrono {
 using file_clock = see below;
}
```

- <sup>1</sup> `file_clock` is an alias for a type meeting the *Cpp17TrivialClock* requirements (30.3), and using a signed arithmetic type for `file_clock::rep`. `file_clock` is used to create the `time_point` system used for `file_time_type` (31.12). Its epoch is unspecified, and `noexcept(file_clock::now())` is true.

[Note 1: The type that `file_clock` denotes can be in a different namespace than `std::chrono`, such as `std::file-system`. — end note]

### 30.7.6.2 Member functions

[time.clock.file.members]

- <sup>1</sup> The type denoted by `file_clock` provides precisely one of the following two sets of static member functions:

```
template<class Duration>
 static sys_time<see below>
 to_sys(const file_time<Duration>&);
template<class Duration>
 static file_time<see below>
 from_sys(const sys_time<Duration>&);
```

OR:

```
template<class Duration>
 static utc_time<see below>
 to_utc(const file_time<Duration>&);
template<class Duration>
 static file_time<see below>
 from_utc(const utc_time<Duration>&);
```

These member functions shall provide `time_point` conversions consistent with those specified by `utc_clock`, `tai_clock`, and `gps_clock`. The Duration of the resultant `time_point` is computed from the Duration of the input `time_point`.

### 30.7.6.3 Non-member functions

[time.clock.file.nonmembers]

```
template<class charT, class traits, class Duration>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const file_time<Duration>& t);
```

- <sup>1</sup> *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%F %T}"), t);
```

```
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 file_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

- <sup>2</sup> *Effects:* Attempts to parse the input stream `is` into the `file_time` `tp` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` is called and `tp` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

- <sup>3</sup> *Returns:* `is`.

### 30.7.7 Class `steady_clock`

[time.clock.steady]

```
namespace std::chrono {
 class steady_clock {
 public:
 using rep = unspecified;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<unspecified, duration>;
 static constexpr bool is_steady = true;

 static time_point now() noexcept;
 };
}
```

- <sup>1</sup> Objects of class `steady_clock` represent clocks for which values of `time_point` never decrease as physical time advances and for which values of `time_point` advance at a steady rate relative to real time. That is, the clock may not be adjusted.

### 30.7.8 Class `high_resolution_clock`

[time.clock.hires]

```
namespace std::chrono {
 class high_resolution_clock {
 public:
 using rep = unspecified;
 using period = ratio<unspecified, unspecified>;
 using duration = chrono::duration<rep, period>;
 using time_point = chrono::time_point<unspecified, duration>;
 static constexpr bool is_steady = unspecified;

 static time_point now() noexcept;
 };
}
```

- <sup>1</sup> Objects of class `high_resolution_clock` represent clocks with the shortest tick period. `high_resolution_clock` may be a synonym for `system_clock` or `steady_clock`.

### 30.7.9 Local time

[time.clock.local]

- <sup>1</sup> The family of time points denoted by `local_time<Duration>` are based on the pseudo clock `local_t`. `local_t` has no member `now()` and thus does not meet the clock requirements. Nevertheless `local_time<Duration>` serves the vital role of representing local time with respect to a not-yet-specified time zone. Aside from being able to get the current time, the complete `time_point` algebra is available for `local_time<Duration>` (just as for `sys_time<Duration>`).

```
template<class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& lt);
```

<sup>2</sup> *Constraints:* `os << sys_time<Duration>lt.time_since_epoch()` is a valid expression.

<sup>3</sup> *Effects:*

```
 os << sys_time<Duration>{lt.time_since_epoch()};
```

<sup>4</sup> *Returns:* `os`.

```
template<class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 local_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

- <sup>5</sup> *Effects:* Attempts to parse the input stream `is` into the `local_time` `tp` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` is called and `tp` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.
- <sup>6</sup> *Returns:* `is`.

### 30.7.10 time\_point conversions

[time.clock.cast]

#### 30.7.10.1 Class template `clock_time_conversion`

[time.clock.conv]

```
namespace std::chrono {
 template<class DestClock, class SourceClock>
 struct clock_time_conversion {};
}
```

- <sup>1</sup> `clock_time_conversion` serves as a trait which can be used to specify how to convert a source `time_point` of type `time_point<SourceClock, Duration>` to a destination `time_point` of type `time_point<DestClock, Duration>` via a specialization: `clock_time_conversion<DestClock, SourceClock>`. A specialization of `clock_time_conversion<DestClock, SourceClock>` shall provide a const-qualified `operator()` that takes a parameter of type `time_point<SourceClock, Duration>` and returns a `time_point<DestClock,`

`OtherDuration`> representing an equivalent point in time. `OtherDuration` is a `chrono::duration` whose specialization is computed from the input `Duration` in a manner which can vary for each `clock_time_conversion` specialization. A program may specialize `clock_time_conversion` if at least one of the template parameters is a user-defined clock type.

- <sup>2</sup> Several specializations are provided by the implementation, as described in 30.7.10.2, 30.7.10.3, 30.7.10.4, and 30.7.10.5.

### 30.7.10.2 Identity conversions

[time.clock.cast.id]

```
template<class Clock>
struct clock_time_conversion<Clock, Clock> {
 template<class Duration>
 time_point<Clock, Duration>
 operator()(const time_point<Clock, Duration>& t) const;
};
```

```
template<class Duration>
time_point<Clock, Duration>
operator()(const time_point<Clock, Duration>& t) const;
```

- <sup>1</sup> *Returns:* t.

```
template<>
struct clock_time_conversion<system_clock, system_clock> {
 template<class Duration>
 sys_time<Duration>
 operator()(const sys_time<Duration>& t) const;
};
```

```
template<class Duration>
sys_time<Duration>
operator()(const sys_time<Duration>& t) const;
```

- <sup>2</sup> *Returns:* t.

```
template<>
struct clock_time_conversion<utc_clock, utc_clock> {
 template<class Duration>
 utc_time<Duration>
 operator()(const utc_time<Duration>& t) const;
};
```

```
template<class Duration>
utc_time<Duration>
operator()(const utc_time<Duration>& t) const;
```

- <sup>3</sup> *Returns:* t.

### 30.7.10.3 Conversions between `system_clock` and `utc_clock`

[time.clock.cast.sys.utc]

```
template<>
struct clock_time_conversion<utc_clock, system_clock> {
 template<class Duration>
 utc_time<common_type_t<Duration, seconds>>
 operator()(const sys_time<Duration>& t) const;
};
```

```
template<class Duration>
utc_time<common_type_t<Duration, seconds>>
operator()(const sys_time<Duration>& t) const;
```

- <sup>1</sup> *Returns:* `utc_clock::from_sys(t)`.

```
template<>
struct clock_time_conversion<system_clock, utc_clock> {
 template<class Duration>
 sys_time<common_type_t<Duration, seconds>>
 operator()(const utc_time<Duration>& t) const;
};
```

```
template<class Duration>
sys_time<common_type_t<Duration, seconds>>
operator()(const utc_time<Duration>& t) const;
```

2 *Returns:* utc\_clock::to\_sys(t).

#### 30.7.10.4 Conversions between system\_clock and other clocks

[time.clock.cast.sys]

```
template<class SourceClock>
struct clock_time_conversion<system_clock, SourceClock> {
 template<class Duration>
 auto operator()(const time_point<SourceClock, Duration>& t) const
 -> decltype(SourceClock::to_sys(t));
};
```

```
template<class Duration>
auto operator()(const time_point<SourceClock, Duration>& t) const
-> decltype(SourceClock::to_sys(t));
```

1 *Constraints:* SourceClock::to\_sys(t) is well-formed.

2 *Mandates:* SourceClock::to\_sys(t) returns a sys\_time<Duration2> for some type Duration2 (30.6.1).

3 *Returns:* SourceClock::to\_sys(t).

```
template<class DestClock>
struct clock_time_conversion<DestClock, system_clock> {
 template<class Duration>
 auto operator()(const sys_time<Duration>& t) const
 -> decltype(DestClock::from_sys(t));
};
```

```
template<class Duration>
auto operator()(const sys_time<Duration>& t) const
-> decltype(DestClock::from_sys(t));
```

4 *Constraints:* DestClock::from\_sys(t) is well-formed.

5 *Mandates:* DestClock::from\_sys(t) returns a time\_point<DestClock, Duration2> for some type Duration2 (30.6.1).

6 *Returns:* DestClock::from\_sys(t).

#### 30.7.10.5 Conversions between utc\_clock and other clocks

[time.clock.cast.utc]

```
template<class SourceClock>
struct clock_time_conversion<utc_clock, SourceClock> {
 template<class Duration>
 auto operator()(const time_point<SourceClock, Duration>& t) const
 -> decltype(SourceClock::to_utc(t));
};
```

```
template<class Duration>
auto operator()(const time_point<SourceClock, Duration>& t) const
-> decltype(SourceClock::to_utc(t));
```

1 *Constraints:* SourceClock::to\_utc(t) is well-formed.

2 *Mandates:* SourceClock::to\_utc(t) returns a utc\_time<Duration2> for some type Duration2 (30.6.1).

3 *Returns:* SourceClock::to\_utc(t).

```
template<class DestClock>
struct clock_time_conversion<DestClock, utc_clock> {
 template<class Duration>
 auto operator()(const utc_time<Duration>& t) const
 -> decltype(DestClock::from_utc(t));
};
```

```
template<class Duration>
auto operator()(const utc_time<Duration>& t) const
-> decltype(DestClock::from_utc(t));
```

4     *Constraints:* DestClock::from\_utc(t) is well-formed.

5     *Mandates:* DestClock::from\_utc(t) returns a time\_point<DestClock, Duration2> for some type Duration2 (30.6.1).

6     *Returns:* DestClock::from\_utc(t).

### 30.7.10.6 Function template clock\_cast

[time.clock.cast.fn]

```
template<class DestClock, class SourceClock, class Duration>
auto clock_cast(const time_point<SourceClock, Duration>& t);
```

1     *Constraints:* At least one of the following clock time conversion expressions is well-formed:

- (1.1)     — clock\_time\_conversion<DestClock, SourceClock>{}(t)
- (1.2)     — clock\_time\_conversion<DestClock, system\_clock>{}(
            clock\_time\_conversion<system\_clock, SourceClock>{}(t))
- (1.3)     — clock\_time\_conversion<DestClock, utc\_clock>{}(
            clock\_time\_conversion<utc\_clock, SourceClock>{}(t))
- (1.4)     — clock\_time\_conversion<DestClock, utc\_clock>{}(
            clock\_time\_conversion<utc\_clock, system\_clock>{}(
            clock\_time\_conversion<system\_clock, SourceClock>{}(t)))
- (1.5)     — clock\_time\_conversion<DestClock, system\_clock>{}(
            clock\_time\_conversion<system\_clock, utc\_clock>{}(
            clock\_time\_conversion<utc\_clock, SourceClock>{}(t)))

A clock time conversion expression is considered better than another clock time conversion expression if it involves fewer operator() calls on clock\_time\_conversion specializations.

2     *Mandates:* Among the well-formed clock time conversion expressions from the above list, there is a unique best expression.

3     *Returns:* The best well-formed clock time conversion expression in the above list.

## 30.8 The civil calendar

[time.cal]

### 30.8.1 General

[time.cal.general]

1 The types in 30.8 describe the civil (Gregorian) calendar and its relationship to sys\_days and local\_days.

### 30.8.2 Class last\_spec

[time.cal.last]

```
namespace std::chrono {
struct last_spec {
explicit last_spec() = default;
};
}
```

1 The type last\_spec is used in conjunction with other calendar types to specify the last in a sequence. For example, depending on context, it can represent the last day of a month, or the last day of the week of a month.

### 30.8.3 Class day

[time.cal.day]

#### 30.8.3.1 Overview

[time.cal.day.overview]

```
namespace std::chrono {
class day {
unsigned char d_; // exposition only
public:
day() = default;
constexpr explicit day(unsigned d) noexcept;
```

```

constexpr day& operator++() noexcept;
constexpr day operator++(int) noexcept;
constexpr day& operator--() noexcept;
constexpr day operator--(int) noexcept;

constexpr day& operator+=(const days& d) noexcept;
constexpr day& operator-=(const days& d) noexcept;

constexpr explicit operator unsigned() const noexcept;
constexpr bool ok() const noexcept;
};
}

```

<sup>1</sup> **day** represents a day of a month. It normally holds values in the range 1 to 31, but may hold non-negative values outside this range. It can be constructed with any **unsigned** value, which will be subsequently truncated to fit into **day**'s unspecified internal storage. **day** meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements, and participates in basic arithmetic with **days** objects, which represent a difference between two **day** objects.

<sup>2</sup> **day** is a trivially copyable and standard-layout class type.

### 30.8.3.2 Member functions

[time.cal.day.members]

```
constexpr explicit day(unsigned d) noexcept;
```

<sup>1</sup> *Effects:* Initializes **d\_** with **d**. The value held is unspecified if **d** is not in the range [0, 255].

```
constexpr day& operator++() noexcept;
```

<sup>2</sup> *Effects:* ++**d\_**.

<sup>3</sup> *Returns:* \*this.

```
constexpr day operator++(int) noexcept;
```

<sup>4</sup> *Effects:* ++(\*this).

<sup>5</sup> *Returns:* A copy of \*this as it existed on entry to this member function.

```
constexpr day& operator--() noexcept;
```

<sup>6</sup> *Effects:* Equivalent to: --**d\_**.

<sup>7</sup> *Returns:* \*this.

```
constexpr day operator--(int) noexcept;
```

<sup>8</sup> *Effects:* --(\*this).

<sup>9</sup> *Returns:* A copy of \*this as it existed on entry to this member function.

```
constexpr day& operator+=(const days& d) noexcept;
```

<sup>10</sup> *Effects:* \*this = \*this + **d**.

<sup>11</sup> *Returns:* \*this.

```
constexpr day& operator-=(const days& d) noexcept;
```

<sup>12</sup> *Effects:* \*this = \*this - **d**.

<sup>13</sup> *Returns:* \*this.

```
constexpr explicit operator unsigned() const noexcept;
```

<sup>14</sup> *Returns:* **d\_**.

```
constexpr bool ok() const noexcept;
```

<sup>15</sup> *Returns:* 1 <= **d\_** && **d\_** <= 31.

**30.8.3.3 Non-member functions****[time.cal.day.nonmembers]**

```
constexpr bool operator==(const day& x, const day& y) noexcept;
```

1     *Returns:* unsigned{x} == unsigned{y}.

```
constexpr strong_ordering operator<=>(const day& x, const day& y) noexcept;
```

2     *Returns:* unsigned{x} <=> unsigned{y}.

```
constexpr day operator+(const day& x, const days& y) noexcept;
```

3     *Returns:* day(unsigned{x} + y.count()).

```
constexpr day operator+(const days& x, const day& y) noexcept;
```

4     *Returns:* y + x.

```
constexpr day operator-(const day& x, const days& y) noexcept;
```

5     *Returns:* x + -y.

```
constexpr days operator-(const day& x, const day& y) noexcept;
```

6     *Returns:* days{int(unsigned{x}) - int(unsigned{y})}.

```
template<class charT, class traits>
```

```
 basic_ostream<charT, traits>&
```

```
 operator<<(basic_ostream<charT, traits>& os, const day& d);
```

7     *Effects:* Equivalent to:

```
 return os << (d.ok() ?
```

```
 format(STATICALLY-WIDEN<charT>("{:%d}"), d) :
```

```
 format(STATICALLY-WIDEN<charT>("{:%d} is not a valid day"), d));
```

```
template<class charT, class traits, class Alloc = allocator<charT>>
```

```
 basic_istream<charT, traits>&
```

```
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
```

```
 day& d, basic_string<charT, traits, Alloc>* abbrev = nullptr,
```

```
 minutes* offset = nullptr);
```

8     *Effects:* Attempts to parse the input stream *is* into the day *d* using the format flags given in the NTCTS *fmt* as specified in 30.13. If the parse fails to decode a valid day, *is.setstate(ios\_base::failbit)* is called and *d* is not modified. If %Z is used and successfully parsed, that value will be assigned to \**abbrev* if *abbrev* is non-null. If %z (or a modified variant) is used and successfully parsed, that value will be assigned to \**offset* if *offset* is non-null.

9     *Returns:* *is*.

```
constexpr chrono::day operator""d(unsigned long long d) noexcept;
```

10    *Returns:* day{static\_cast<unsigned>(d)}.

**30.8.4 Class month****[time.cal.month]****30.8.4.1 Overview****[time.cal.month.overview]**

```
namespace std::chrono {
```

```
 class month {
```

```
 unsigned char m_; // exposition only
```

```
 public:
```

```
 month() = default;
```

```
 constexpr explicit month(unsigned m) noexcept;
```

```
 constexpr month& operator++() noexcept;
```

```
 constexpr month operator++(int) noexcept;
```

```
 constexpr month& operator--() noexcept;
```

```
 constexpr month operator--(int) noexcept;
```

```
 constexpr month& operator+=(const months& m) noexcept;
```

```
 constexpr month& operator-=(const months& m) noexcept;
```



```

 constexpr explicit operator unsigned() const noexcept;
 constexpr bool ok() const noexcept;
};
}

```

<sup>1</sup> `month` represents a month of a year. It normally holds values in the range 1 to 12, but may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `month`'s unspecified internal storage. `month` meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements, and participates in basic arithmetic with `months` objects, which represent a difference between two `month` objects.

<sup>2</sup> `month` is a trivially copyable and standard-layout class type.

#### 30.8.4.2 Member functions

[time.cal.month.members]

```
constexpr explicit month(unsigned m) noexcept;
```

<sup>1</sup> *Effects:* Initializes `m_` with `m`. The value held is unspecified if `m` is not in the range `[0, 255]`.

```
constexpr month& operator++() noexcept;
```

<sup>2</sup> *Effects:* `*this += months{1}`.

<sup>3</sup> *Returns:* `*this`.

```
constexpr month operator++(int) noexcept;
```

<sup>4</sup> *Effects:* `++(*this)`.

<sup>5</sup> *Returns:* A copy of `*this` as it existed on entry to this member function.

```
constexpr month& operator--() noexcept;
```

<sup>6</sup> *Effects:* `*this -= months{1}`.

<sup>7</sup> *Returns:* `*this`.

```
constexpr month operator--(int) noexcept;
```

<sup>8</sup> *Effects:* `--(*this)`.

<sup>9</sup> *Returns:* A copy of `*this` as it existed on entry to this member function.

```
constexpr month& operator+=(const months& m) noexcept;
```

<sup>10</sup> *Effects:* `*this = *this + m`.

<sup>11</sup> *Returns:* `*this`.

```
constexpr month& operator-=(const months& m) noexcept;
```

<sup>12</sup> *Effects:* `*this = *this - m`.

<sup>13</sup> *Returns:* `*this`.

```
constexpr explicit operator unsigned() const noexcept;
```

<sup>14</sup> *Returns:* `m_`.

```
constexpr bool ok() const noexcept;
```

<sup>15</sup> *Returns:* `1 <= m_ && m_ <= 12`.

#### 30.8.4.3 Non-member functions

[time.cal.month.nonmembers]

```
constexpr bool operator==(const month& x, const month& y) noexcept;
```

<sup>1</sup> *Returns:* `unsigned{x} == unsigned{y}`.

```
constexpr strong_ordering operator<=>(const month& x, const month& y) noexcept;
```

<sup>2</sup> *Returns:* `unsigned{x} <=> unsigned{y}`.

```
constexpr month operator+(const month& x, const months& y) noexcept;
```

<sup>3</sup> *Returns:*

```
month{modulo(static_cast<long long>(unsigned{x}) + (y.count() - 1), 12) + 1}
```

where `modulo(n, 12)` computes the remainder of `n` divided by 12 using Euclidean division.

[*Note 1:* Given a divisor of 12, Euclidean division truncates towards negative infinity and always produces a remainder in the range of `[0, 11]`. Assuming no overflow in the signed summation, this operation results in a `month` holding a value in the range `[1, 12]` even if `!x.ok()`. — *end note*]

[*Example 1:* `February + months{11} == January`. — *end example*]

```
constexpr month operator+(const months& x, const month& y) noexcept;
```

4     *Returns:* `y + x`.

```
constexpr month operator-(const month& x, const months& y) noexcept;
```

5     *Returns:* `x + -y`.

```
constexpr months operator-(const month& x, const month& y) noexcept;
```

6     *Returns:* If `x.ok() == true` and `y.ok() == true`, returns a value `m` in the range `[months{0}, months{11}]` satisfying `y + m == x`. Otherwise the value returned is unspecified.

[*Example 2:* `January - February == months{11}`. — *end example*]

```
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month& m);
```

7     *Effects:* Equivalent to:

```
 return os << (m.ok() ?
 format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%b}"), m) :
 format(os.getloc(), STATICALLY-WIDEN<charT>("{} is not a valid month"),
 static_cast<unsigned>(m)));
```

```
template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 month& m, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

8     *Effects:* Attempts to parse the input stream `is` into the `month` `m` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid month, `is.setstate(ios_base::failbit)` is called and `m` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

9     *Returns:* `is`.

## 30.8.5 Class `year`

[[time.cal.year](#)]

### 30.8.5.1 Overview

[[time.cal.year.overview](#)]

```
namespace std::chrono {
 class year {
 short y_; // exposition only
 public:
 year() = default;
 constexpr explicit year(int y) noexcept;

 constexpr year& operator++() noexcept;
 constexpr year operator++(int) noexcept;
 constexpr year& operator--() noexcept;
 constexpr year operator--(int) noexcept;

 constexpr year& operator+=(const years& y) noexcept;
 constexpr year& operator-=(const years& y) noexcept;

 constexpr year operator+() const noexcept;
 constexpr year operator-() const noexcept;

 constexpr bool is_leap() const noexcept;
```

```

constexpr explicit operator int() const noexcept;
constexpr bool ok() const noexcept;

static constexpr year min() noexcept;
static constexpr year max() noexcept;
};
}

```

<sup>1</sup> `year` represents a year in the civil calendar. It can represent values in the range `[min(),max()]`. It can be constructed with any `int` value, which will be subsequently truncated to fit into `year`'s unspecified internal storage. `year` meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements, and participates in basic arithmetic with `years` objects, which represent a difference between two `year` objects.

<sup>2</sup> `year` is a trivially copyable and standard-layout class type.

### 30.8.5.2 Member functions

[time.cal.year.members]

```
constexpr explicit year(int y) noexcept;
```

<sup>1</sup> *Effects:* Initializes `y_` with `y`. The value held is unspecified if `y` is not in the range `[-32767, 32767]`.

```
constexpr year& operator++() noexcept;
```

<sup>2</sup> *Effects:* `++y_`.

<sup>3</sup> *Returns:* `*this`.

```
constexpr year operator++(int) noexcept;
```

<sup>4</sup> *Effects:* `++(*this)`.

<sup>5</sup> *Returns:* A copy of `*this` as it existed on entry to this member function.

```
constexpr year& operator--() noexcept;
```

<sup>6</sup> *Effects:* `--y_`.

<sup>7</sup> *Returns:* `*this`.

```
constexpr year operator--(int) noexcept;
```

<sup>8</sup> *Effects:* `--(*this)`.

<sup>9</sup> *Returns:* A copy of `*this` as it existed on entry to this member function.

```
constexpr year& operator+=(const years& y) noexcept;
```

<sup>10</sup> *Effects:* `*this = *this + y`.

<sup>11</sup> *Returns:* `*this`.

```
constexpr year& operator-=(const years& y) noexcept;
```

<sup>12</sup> *Effects:* `*this = *this - y`.

<sup>13</sup> *Returns:* `*this`.

```
constexpr year operator+() const noexcept;
```

<sup>14</sup> *Returns:* `*this`.

```
constexpr year operator-() const noexcept;
```

<sup>15</sup> *Returns:* `year{-y_}`.

```
constexpr bool is_leap() const noexcept;
```

<sup>16</sup> *Returns:* `y_ % 4 == 0 && (y_ % 100 != 0 || y_ % 400 == 0)`.

```
constexpr explicit operator int() const noexcept;
```

<sup>17</sup> *Returns:* `y_`.

```
constexpr bool ok() const noexcept;
```

<sup>18</sup> *Returns:* `min().y_ <= y_ && y_ <= max().y_`.

```
static constexpr year min() noexcept;
19 Returns: year{-32767}.
```

```
static constexpr year max() noexcept;
20 Returns: year{32767}.
```

### 30.8.5.3 Non-member functions

[time.cal.year.nonmembers]

```
constexpr bool operator==(const year& x, const year& y) noexcept;
1 Returns: int{x} == int{y}.
```

```
constexpr strong_ordering operator<=>(const year& x, const year& y) noexcept;
2 Returns: int{x} <=> int{y}.
```

```
constexpr year operator+(const year& x, const years& y) noexcept;
3 Returns: year{int{x} + static_cast<int>(y.count())}.
```

```
constexpr year operator+(const years& x, const year& y) noexcept;
4 Returns: y + x.
```

```
constexpr year operator-(const year& x, const years& y) noexcept;
5 Returns: x + -y.
```

```
constexpr years operator-(const year& x, const year& y) noexcept;
6 Returns: years{int{x} - int{y}}.
```

```
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year& y);
7 Effects: Equivalent to:
 return os << (y.ok() ?
 format(STATICALLY-WIDEN<charT>("{:Y}"), y) :
 format(STATICALLY-WIDEN<charT>("{:Y} is not a valid year"), y));
```

```
template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year& y, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

8 *Effects:* Attempts to parse the input stream *is* into the year *y* using the format flags given in the NTCTS *fmt* as specified in 30.13. If the parse fails to decode a valid year, *is.setstate(ios\_base::failbit)* is called and *y* is not modified. If *%Z* is used and successfully parsed, that value will be assigned to *\*abbrev* if *abbrev* is non-null. If *%z* (or a modified variant) is used and successfully parsed, that value will be assigned to *\*offset* if *offset* is non-null.

9 *Returns:* *is*.

```
constexpr chrono::year operator""y(unsigned long long y) noexcept;
10 Returns: year{static_cast<int>(y)}.
```

## 30.8.6 Class weekday

[time.cal.wd]

### 30.8.6.1 Overview

[time.cal.wd.overview]

```
namespace std::chrono {
 class weekday {
 unsigned char wd_; // exposition only
 public:
 weekday() = default;
 constexpr explicit weekday(unsigned wd) noexcept;
 constexpr weekday(const sys_days& dp) noexcept;
 constexpr explicit weekday(const local_days& dp) noexcept;
```

```

constexpr weekday& operator++() noexcept;
constexpr weekday operator++(int) noexcept;
constexpr weekday& operator--() noexcept;
constexpr weekday operator--(int) noexcept;

constexpr weekday& operator+=(const days& d) noexcept;
constexpr weekday& operator-=(const days& d) noexcept;

constexpr unsigned c_encoding() const noexcept;
constexpr unsigned iso_encoding() const noexcept;
constexpr bool ok() const noexcept;

constexpr weekday_indexed operator[](unsigned index) const noexcept;
constexpr weekday_last operator[](last_spec) const noexcept;
};
}

```

- <sup>1</sup> **weekday** represents a day of the week in the civil calendar. It normally holds values in the range 0 to 6, corresponding to Sunday through Saturday, but it may hold non-negative values outside this range. It can be constructed with any **unsigned** value, which will be subsequently truncated to fit into **weekday**'s unspecified internal storage. **weekday** meets the *Cpp17EqualityComparable* (Table 28) requirements.

[Note 1: **weekday** is not *Cpp17LessThanComparable* because there is no universal consensus on which day is the first day of the week. **weekday**'s arithmetic operations treat the days of the week as a circular range, with no beginning and no end. — end note]

- <sup>2</sup> **weekday** is a trivially copyable and standard-layout class type.

### 30.8.6.2 Member functions

[time.cal.wd.members]

```
constexpr explicit weekday(unsigned wd) noexcept;
```

- <sup>1</sup> *Effects:* Initializes *wd\_* with *wd* == 7 ? 0 : *wd*. The value held is unspecified if *wd* is not in the range [0, 255].

```
constexpr weekday(const sys_days& dp) noexcept;
```

- <sup>2</sup> *Effects:* Computes what day of the week corresponds to the *sys\_days* *dp*, and initializes that day of the week in *wd\_*.

- <sup>3</sup> [Example 1: If *dp* represents 1970-01-01, the constructed **weekday** represents Thursday by storing 4 in *wd\_*. — end example]

```
constexpr explicit weekday(const local_days& dp) noexcept;
```

- <sup>4</sup> *Effects:* Computes what day of the week corresponds to the *local\_days* *dp*, and initializes that day of the week in *wd\_*.

- <sup>5</sup> *Postconditions:* The value is identical to that constructed from *sys\_days*{*dp.time\_since\_epoch()*}.

```
constexpr weekday& operator++() noexcept;
```

- <sup>6</sup> *Effects:* *\*this* += *days*{1}.

- <sup>7</sup> *Returns:* *\*this*.

```
constexpr weekday operator++(int) noexcept;
```

- <sup>8</sup> *Effects:* ++(\**this*).

- <sup>9</sup> *Returns:* A copy of *\*this* as it existed on entry to this member function.

```
constexpr weekday& operator--() noexcept;
```

- <sup>10</sup> *Effects:* *\*this* -= *days*{1}.

- <sup>11</sup> *Returns:* *\*this*.

```
constexpr weekday operator--(int) noexcept;
```

- <sup>12</sup> *Effects:* --(\**this*).

- <sup>13</sup> *Returns:* A copy of *\*this* as it existed on entry to this member function.

```

constexpr weekday& operator+=(const days& d) noexcept;
14 Effects: *this = *this + d.
15 Returns: *this.

constexpr weekday& operator-=(const days& d) noexcept;
16 Effects: *this = *this - d.
17 Returns: *this.

constexpr unsigned c_encoding() const noexcept;
18 Returns: wd_.

constexpr unsigned iso_encoding() const noexcept;
19 Returns: wd_ == 0u ? 7u : wd_.

constexpr bool ok() const noexcept;
20 Returns: wd_ <= 6.

constexpr weekday_indexed operator[](unsigned index) const noexcept;
21 Returns: {*this, index}.

constexpr weekday_last operator[](last_spec) const noexcept;
22 Returns: weekday_last{*this}.

```

### 30.8.6.3 Non-member functions

[time.cal.wd.nonmembers]

```

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
1 Returns: x.wd_ == y.wd_.

```

```

constexpr weekday operator+(const weekday& x, const days& y) noexcept;
2 Returns:

```

```

 weekday{modulo(static_cast<long long>(x.wd_) + y.count(), 7)}

```

where modulo(*n*, 7) computes the remainder of *n* divided by 7 using Euclidean division.

[*Note 1:* Given a divisor of 7, Euclidean division truncates towards negative infinity and always produces a remainder in the range of [0, 6]. Assuming no overflow in the signed summation, this operation results in a `weekday` holding a value in the range [0, 6] even if !`x.ok()`. — *end note*]

[*Example 1:* Monday + days{6} == Sunday. — *end example*]

```

constexpr weekday operator+(const days& x, const weekday& y) noexcept;
3 Returns: y + x.

```

```

constexpr weekday operator-(const weekday& x, const days& y) noexcept;
4 Returns: x + -y.

```

```

constexpr days operator-(const weekday& x, const weekday& y) noexcept;
5 Returns: If x.ok() == true and y.ok() == true, returns a value d in the range [days{0}, days{6}]
 satisfying y + d == x. Otherwise the value returned is unspecified.
 [Example 2: Sunday - Monday == days{6}. — end example]

```

```

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const weekday& wd);
6 Effects: Equivalent to:
 return os << (wd.ok() ?
 format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%a}"), wd) :
 format(os.getloc(), STATICALLY-WIDEN<charT>("{} is not a valid weekday"),
 static_cast<unsigned>(wd.wd_)));

```

```
template<class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 weekday& wd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

7 *Effects:* Attempts to parse the input stream `is` into the `weekday` `wd` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid weekday, `is.setstate(ios_base::failbit)` is called and `wd` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

8 *Returns:* `is`.

### 30.8.7 Class `weekday_indexed`

[time.cal.wdidx]

#### 30.8.7.1 Overview

[time.cal.wdidx.overview]

```
namespace std::chrono {
class weekday_indexed {
 chrono::weekday wd_; // exposition only
 unsigned char index_; // exposition only

public:
 weekday_indexed() = default;
 constexpr weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;

 constexpr chrono::weekday weekday() const noexcept;
 constexpr unsigned index() const noexcept;
 constexpr bool ok() const noexcept;
};
}
```

1 `weekday_indexed` represents a `weekday` and a small index in the range 1 to 5. This class is used to represent the first, second, third, fourth, or fifth weekday of a month.

2 [Note 1: A `weekday_indexed` object can be constructed by indexing a `weekday` with an `unsigned`. — end note]

[Example 1:

```
constexpr auto wdi = Sunday[2]; // wdi is the second Sunday of an as yet unspecified month
static_assert(wdi.weekday() == Sunday);
static_assert(wdi.index() == 2);
```

— end example]

3 `weekday_indexed` is a trivially copyable and standard-layout class type.

#### 30.8.7.2 Member functions

[time.cal.wdidx.members]

```
constexpr weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;
```

1 *Effects:* Initializes `wd_` with `wd` and `index_` with `index`. The values held are unspecified if `!wd.ok()` or `index` is not in the range `[0, 7]`.

```
constexpr chrono::weekday weekday() const noexcept;
```

2 *Returns:* `wd_`.

```
constexpr unsigned index() const noexcept;
```

3 *Returns:* `index_`.

```
constexpr bool ok() const noexcept;
```

4 *Returns:* `wd_.ok() && 1 <= index_ && index_ <= 5`.

#### 30.8.7.3 Non-member functions

[time.cal.wdidx.nonmembers]

```
constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

1 *Returns:* `x.weekday() == y.weekday() && x.index() == y.index()`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

2 *Effects:* Equivalent to:

```
auto i = wdi.index();
return os << (i >= 1 && i <= 5 ?
 format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}[{}]", wdi.weekday(), i) :
 format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}[{} is not a valid index]",
 wdi.weekday(), i));
```

### 30.8.8 Class `weekday_last`

[time.cal.wdlast]

#### 30.8.8.1 Overview

[time.cal.wdlast.overview]

```
namespace std::chrono {
 class weekday_last {
 chrono::weekday wd_; // exposition only

 public:
 constexpr explicit weekday_last(const chrono::weekday& wd) noexcept;

 constexpr chrono::weekday weekday() const noexcept;
 constexpr bool ok() const noexcept;
 };
}
```

1 `weekday_last` represents the last weekday of a month.

2 [Note 1: A `weekday_last` object can be constructed by indexing a `weekday` with `last`. — end note]

[Example 1:

```
constexpr auto wdl = Sunday[last]; // wdl is the last Sunday of an as yet unspecified month
static_assert(wdl.weekday() == Sunday);
```

— end example]

3 `weekday_last` is a trivially copyable and standard-layout class type.

#### 30.8.8.2 Member functions

[time.cal.wdlast.members]

```
constexpr explicit weekday_last(const chrono::weekday& wd) noexcept;
```

1 *Effects:* Initializes `wd_` with `wd`.

```
constexpr chrono::weekday weekday() const noexcept;
```

2 *Returns:* `wd_`.

```
constexpr bool ok() const noexcept;
```

3 *Returns:* `wd_.ok()`.

#### 30.8.8.3 Non-member functions

[time.cal.wdlast.nonmembers]

```
constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
```

1 *Returns:* `x.weekday() == y.weekday()`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

2 *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}[last]", wdl.weekday());
```

### 30.8.9 Class `month_day`

[time.cal.md]

#### 30.8.9.1 Overview

[time.cal.md.overview]

```
namespace std::chrono {
 class month_day {
 chrono::month m_; // exposition only
```



```

 chrono::day d_; // exposition only

public:
 month_day() = default;
 constexpr month_day(const chrono::month& m, const chrono::day& d) noexcept;

 constexpr chrono::month month() const noexcept;
 constexpr chrono::day day() const noexcept;
 constexpr bool ok() const noexcept;
};

```

<sup>1</sup> `month_day` represents a specific day of a specific month, but with an unspecified year. `month_day` meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements.

<sup>2</sup> `month_day` is a trivially copyable and standard-layout class type.

### 30.8.9.2 Member functions

[time.cal.md.members]

```
constexpr month_day(const chrono::month& m, const chrono::day& d) noexcept;
```

<sup>1</sup> *Effects:* Initializes `m_` with `m`, and `d_` with `d`.

```
constexpr chrono::month month() const noexcept;
```

<sup>2</sup> *Returns:* `m_`.

```
constexpr chrono::day day() const noexcept;
```

<sup>3</sup> *Returns:* `d_`.

```
constexpr bool ok() const noexcept;
```

<sup>4</sup> *Returns:* `true` if `m_.ok()` is `true`, `1d <= d_`, and `d_` is less than or equal to the number of days in month `m_`; otherwise returns `false`. When `m_ == February`, the number of days is considered to be 29.

### 30.8.9.3 Non-member functions

[time.cal.md.nonmembers]

```
constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
```

<sup>1</sup> *Returns:* `x.month() == y.month() && x.day() == y.day()`.

```
constexpr strong_ordering operator<==(const month_day& x, const month_day& y) noexcept;
```

<sup>2</sup> *Effects:* Equivalent to:

```

 if (auto c = x.month() <=> y.month(); c != 0) return c;
 return x.day() <=> y.day();

```

```

template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_day& md);

```

<sup>3</sup> *Effects:* Equivalent to:

```

 return os << format(os.getloc(), STatically-WIDEN<charT>("{:L}/{:}"),
 md.month(), md.day());

```

```

template<class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 month_day& md, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);

```

<sup>4</sup> *Effects:* Attempts to parse the input stream `is` into the `month_day` `md` using the format flags given in the NTCTS `fmt` as specified in 30.13. If the parse fails to decode a valid `month_day`, `is.setstate(ios_base::failbit)` is called and `md` is not modified. If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null. If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

<sup>5</sup> *Returns:* `is`.

**30.8.10 Class month\_day\_last****[time.cal.mdlast]**

```

namespace std::chrono {
 class month_day_last {
 chrono::month m_; // exposition only

 public:
 constexpr explicit month_day_last(const chrono::month& m) noexcept;

 constexpr chrono::month month() const noexcept;
 constexpr bool ok() const noexcept;
 };
}

```

<sup>1</sup> month\_day\_last represents the last day of a month.

<sup>2</sup> [Note 1: A month\_day\_last object can be constructed using the expression `m/last` or `last/m`, where `m` is an expression of type `month`. — end note]

[Example 1:

```

constexpr auto mdl = February/last; // mdl is the last day of February of an as yet unspecified year
static_assert(mdl.month() == February);
— end example]

```

<sup>3</sup> month\_day\_last is a trivially copyable and standard-layout class type.

```
constexpr explicit month_day_last(const chrono::month& m) noexcept;
```

<sup>4</sup> *Effects:* Initializes `m_` with `m`.

```
constexpr month month() const noexcept;
```

<sup>5</sup> *Returns:* `m_`.

```
constexpr bool ok() const noexcept;
```

<sup>6</sup> *Returns:* `m_.ok()`.

```
constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
```

<sup>7</sup> *Returns:* `x.month() == y.month()`.

```
constexpr strong_ordering operator<=>(const month_day_last& x, const month_day_last& y) noexcept;
```

<sup>8</sup> *Returns:* `x.month() <=> y.month()`.

```

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);

```

<sup>9</sup> *Effects:* Equivalent to:

```
return os << format(os.getloc(), STatically-WIDEN<charT>("{:L}/last"), mdl.month());
```

**30.8.11 Class month\_weekday****[time.cal.mwd]****30.8.11.1 Overview****[time.cal.mwd.overview]**

```

namespace std::chrono {
 class month_weekday {
 chrono::month m_; // exposition only
 chrono::weekday_indexed wdi_; // exposition only
 public:
 constexpr month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;

 constexpr chrono::month month() const noexcept;
 constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
 constexpr bool ok() const noexcept;
 };
}

```

<sup>1</sup> month\_weekday represents the  $n^{\text{th}}$  weekday of a month, of an as yet unspecified year. To do this the month\_weekday stores a month and a weekday\_indexed.

<sup>2</sup> [Example 1:

```
constexpr auto mwd
 = February/Tuesday[3]; // mwd is the third Tuesday of February of an as yet unspecified year
static_assert(mwd.month() == February);
static_assert(mwd.weekday_indexed() == Tuesday[3]);
— end example]
```

<sup>3</sup> month\_weekday is a trivially copyable and standard-layout class type.

### 30.8.11.2 Member functions

[time.cal.mwd.members]

```
constexpr month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;
```

<sup>1</sup> *Effects:* Initializes *m\_* with *m*, and *wdi\_* with *wdi*.

```
constexpr chrono::month month() const noexcept;
```

<sup>2</sup> *Returns:* *m\_*.

```
constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
```

<sup>3</sup> *Returns:* *wdi\_*.

```
constexpr bool ok() const noexcept;
```

<sup>4</sup> *Returns:* *m\_.ok()* && *wdi\_.ok()*.

### 30.8.11.3 Non-member functions

[time.cal.mwd.nonmembers]

```
constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
```

<sup>1</sup> *Returns:* *x.month()* == *y.month()* && *x.weekday\_indexed()* == *y.weekday\_indexed()*.

```
template<class charT, class traits>
```

```
basic_ostream<charT, traits>&
```

```
operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

<sup>2</sup> *Effects:* Equivalent to:

```
return os << format(os.getloc(), STatically-WIDEN<charT>("{:L}/{:L}"),
 mwd.month(), mwd.weekday_indexed());
```

## 30.8.12 Class month\_weekday\_last

[time.cal.mwdlast]

### 30.8.12.1 Overview

[time.cal.mwdlast.overview]

```
namespace std::chrono {
 class month_weekday_last {
 chrono::month m_; // exposition only
 chrono::weekday_last wdl_; // exposition only
 public:
 constexpr month_weekday_last(const chrono::month& m,
 const chrono::weekday_last& wdl) noexcept;

 constexpr chrono::month month() const noexcept;
 constexpr chrono::weekday_last weekday_last() const noexcept;
 constexpr bool ok() const noexcept;
 };
}
```

<sup>1</sup> month\_weekday\_last represents the last weekday of a month, of an as yet unspecified year. To do this the month\_weekday\_last stores a month and a weekday\_last.

<sup>2</sup> [Example 1:

```
constexpr auto mwd
 = February/Tuesday[last]; // mwd is the last Tuesday of February of an as yet unspecified year
static_assert(mwd.month() == February);
static_assert(mwd.weekday_last() == Tuesday[last]);
— end example]
```

<sup>3</sup> month\_weekday\_last is a trivially copyable and standard-layout class type.

**30.8.12.2 Member functions****[time.cal.mwdlast.members]**

```
constexpr month_weekday_last(const chrono::month& m,
 const chrono::weekday_last& wdl) noexcept;
```

<sup>1</sup> *Effects:* Initializes *m\_* with *m*, and *wdl\_* with *wdl*.

```
constexpr chrono::month month() const noexcept;
```

<sup>2</sup> *Returns:* *m\_*.

```
constexpr chrono::weekday_last weekday_last() const noexcept;
```

<sup>3</sup> *Returns:* *wdl\_*.

```
constexpr bool ok() const noexcept;
```

<sup>4</sup> *Returns:* *m\_.ok()* && *wdl\_.ok()*.

**30.8.12.3 Non-member functions****[time.cal.mwdlast.nonmembers]**

```
constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

<sup>1</sup> *Returns:* *x.month()* == *y.month()* && *x.weekday\_last()* == *y.weekday\_last()*.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

<sup>2</sup> *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L}/{:L}"),
 mwdl.month(), mwdl.weekday_last());
```

**30.8.13 Class year\_month****[time.cal.ym]****30.8.13.1 Overview****[time.cal.ym.overview]**

```
namespace std::chrono {
 class year_month {
 chrono::year y_; // exposition only
 chrono::month m_; // exposition only

 public:
 year_month() = default;
 constexpr year_month(const chrono::year& y, const chrono::month& m) noexcept;

 constexpr chrono::year year() const noexcept;
 constexpr chrono::month month() const noexcept;

 constexpr year_month& operator+=(const months& dm) noexcept;
 constexpr year_month& operator-=(const months& dm) noexcept;
 constexpr year_month& operator+=(const years& dy) noexcept;
 constexpr year_month& operator-=(const years& dy) noexcept;

 constexpr bool ok() const noexcept;
 };
}
```

<sup>1</sup> *year\_month* represents a specific month of a specific year, but with an unspecified day. *year\_month* is a field-based time point with a resolution of months. *year\_month* meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements.

<sup>2</sup> *year\_month* is a trivially copyable and standard-layout class type.

**30.8.13.2 Member functions****[time.cal.ym.members]**

```
constexpr year_month(const chrono::year& y, const chrono::month& m) noexcept;
```

<sup>1</sup> *Effects:* Initializes *y\_* with *y*, and *m\_* with *m*.

```
constexpr chrono::year year() const noexcept;
```

2     *Returns:* *y\_*.

```
constexpr chrono::month month() const noexcept;
```

3     *Returns:* *m\_*.

```
constexpr year_month& operator+=(const months& dm) noexcept;
```

4     *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

5     *Effects:* *\*this* = *\*this* + *dm*.

6     *Returns:* *\*this*.

```
constexpr year_month& operator-=(const months& dm) noexcept;
```

7     *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

8     *Effects:* *\*this* = *\*this* - *dm*.

9     *Returns:* *\*this*.

```
constexpr year_month& operator+=(const years& dy) noexcept;
```

10    *Effects:* *\*this* = *\*this* + *dy*.

11    *Returns:* *\*this*.

```
constexpr year_month& operator-=(const years& dy) noexcept;
```

12    *Effects:* *\*this* = *\*this* - *dy*.

13    *Returns:* *\*this*.

```
constexpr bool ok() const noexcept;
```

14    *Returns:* *y\_.ok()* && *m\_.ok()*.

### 30.8.13.3 Non-member functions

[time.cal.ym.nonmembers]

```
constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
```

1     *Returns:* *x.year()* == *y.year()* && *x.month()* == *y.month()*.

```
constexpr strong_ordering operator<=>(const year_month& x, const year_month& y) noexcept;
```

2     *Effects:* Equivalent to:

```
 if (auto c = x.year() <=> y.year(); c != 0) return c;
 return x.month() <=> y.month();
```

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
```

3     *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

4     *Returns:* A **year\_month** value *z* such that *z.ok()* && *z* - *ym* == *dm* is true.

5     *Complexity:*  $\mathcal{O}(1)$  with respect to the value of *dm*.

```
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
```

6     *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

7     *Returns:* *ym* + *dm*.

```
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
```

8     *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

9     *Returns:* *ym* + -*dm*.

```
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
```

10 *Returns:*

```
x.year() - y.year() + months{static_cast<int>(unsigned{x.month()}) -
 static_cast<int>(unsigned{y.month()})}
```

```
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
```

11 *Returns:* (ym.year() + dy) / ym.month().

```
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
```

12 *Returns:* ym + dy.

```
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;
```

13 *Returns:* ym + -dy.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month& ym);
```

14 *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{:L}"),
 ym.year(), ym.month());
```

```
template<class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year_month& ym, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

15 *Effects:* Attempts to parse the input stream *is* into the *year\_month* *ym* using the format flags given in the NTCTS *fmt* as specified in 30.13. If the parse fails to decode a valid *year\_month*, *is.setstate(ios\_base::failbit)* is called and *ym* is not modified. If *%Z* is used and successfully parsed, that value will be assigned to *\*abbrev* if *abbrev* is non-null. If *%z* (or a modified variant) is used and successfully parsed, that value will be assigned to *\*offset* if *offset* is non-null.

16 *Returns:* *is*.

### 30.8.14 Class *year\_month\_day*

[time.cal.ymd]

#### 30.8.14.1 Overview

[time.cal.ymd.overview]

```
namespace std::chrono {
 class year_month_day {
 chrono::year y_; // exposition only
 chrono::month m_; // exposition only
 chrono::day d_; // exposition only

 public:
 year_month_day() = default;
 constexpr year_month_day(const chrono::year& y, const chrono::month& m,
 const chrono::day& d) noexcept;
 constexpr year_month_day(const year_month_day_last& ymdl) noexcept;
 constexpr year_month_day(const sys_days& dp) noexcept;
 constexpr explicit year_month_day(const local_days& dp) noexcept;

 constexpr year_month_day& operator+=(const months& m) noexcept;
 constexpr year_month_day& operator-=(const months& m) noexcept;
 constexpr year_month_day& operator+=(const years& y) noexcept;
 constexpr year_month_day& operator-=(const years& y) noexcept;

 constexpr chrono::year year() const noexcept;
 constexpr chrono::month month() const noexcept;
 constexpr chrono::day day() const noexcept;

 constexpr operator sys_days() const noexcept;
 constexpr explicit operator local_days() const noexcept;
```

```

 constexpr bool ok() const noexcept;
};

```

- <sup>1</sup> `year_month_day` represents a specific year, month, and day. `year_month_day` is a field-based time point with a resolution of days.

[*Note 1: `year_month_day` supports `years`- and `months`-oriented arithmetic, but not `days`-oriented arithmetic. For the latter, there is a conversion to `sys_days`, which efficiently supports `days`-oriented arithmetic. — end note*]

`year_month_day` meets the *Cpp17EqualityComparable* (Table 28) and *Cpp17LessThanComparable* (Table 29) requirements.

- <sup>2</sup> `year_month_day` is a trivially copyable and standard-layout class type.

### 30.8.14.2 Member functions

[time.cal.ymd.members]

```

constexpr year_month_day(const chrono::year& y, const chrono::month& m,
 const chrono::day& d) noexcept;

```

- <sup>1</sup> *Effects:* Initializes `y_` with `y`, `m_` with `m`, and `d_` with `d`.

```

constexpr year_month_day(const year_month_day_last& ymdl) noexcept;

```

- <sup>2</sup> *Effects:* Initializes `y_` with `ymdl.year()`, `m_` with `ymdl.month()`, and `d_` with `ymdl.day()`.

[*Note 1: This conversion from `year_month_day_last` to `year_month_day` can be more efficient than converting a `year_month_day_last` to a `sys_days`, and then converting that `sys_days` to a `year_month_day`. — end note*]

```

constexpr year_month_day(const sys_days& dp) noexcept;

```

- <sup>3</sup> *Effects:* Constructs an object of type `year_month_day` that corresponds to the date represented by `dp`.

- <sup>4</sup> *Remarks:* For any value `ymd` of type `year_month_day` for which `ymd.ok()` is true, `ymd == year_month_day{sys_days{ymd}}` is true.

```

constexpr explicit year_month_day(const local_days& dp) noexcept;

```

- <sup>5</sup> *Effects:* Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```

constexpr year_month_day& operator+=(const months& m) noexcept;

```

- <sup>6</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

- <sup>7</sup> *Effects:* `*this = *this + m`.

- <sup>8</sup> *Returns:* `*this`.

```

constexpr year_month_day& operator-=(const months& m) noexcept;

```

- <sup>9</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

- <sup>10</sup> *Effects:* `*this = *this - m`.

- <sup>11</sup> *Returns:* `*this`.

```

constexpr year_month_day& year_month_day::operator+=(const years& y) noexcept;

```

- <sup>12</sup> *Effects:* `*this = *this + y`.

- <sup>13</sup> *Returns:* `*this`.

```

constexpr year_month_day& year_month_day::operator-=(const years& y) noexcept;

```

- <sup>14</sup> *Effects:* `*this = *this - y`.

- <sup>15</sup> *Returns:* `*this`.

```

constexpr chrono::year year() const noexcept;

```

- <sup>16</sup> *Returns:* `y_`.

```

constexpr chrono::month month() const noexcept;

```

- <sup>17</sup> *Returns:* `m_`.

```
constexpr chrono::day day() const noexcept;
```

18 *Returns:*  $d_$ .

```
constexpr operator sys_days() const noexcept;
```

19 *Returns:* If `ok()`, returns a `sys_days` holding a count of days from the `sys_days` epoch to `*this` (a negative value if `*this` represents a date prior to the `sys_days` epoch). Otherwise, if `y_.ok()` && `m_.ok()` is true, returns `sys_days{y_/m_/1d} + (d_ - 1d)`. Otherwise the value returned is unspecified.

20 *Remarks:* A `sys_days` in the range `[days{-12687428}, days{11248737}]` which is converted to a `year_month_day` has the same value when converted back to a `sys_days`.

21 *[Example 1:*

```
 static_assert(year_month_day{sys_days{2017y/January/0}} == 2016y/December/31);
 static_assert(year_month_day{sys_days{2017y/January/31}} == 2017y/January/31);
 static_assert(year_month_day{sys_days{2017y/January/32}} == 2017y/February/1);
 — end example]
```

```
constexpr explicit operator local_days() const noexcept;
```

22 *Returns:* `local_days{sys_days{*this}.time_since_epoch()}.`

```
constexpr bool ok() const noexcept;
```

23 *Returns:* If `y_.ok()` is true, and `m_.ok()` is true, and `d_` is in the range `[1d, (y_/m_/last).day()]`, then returns true; otherwise returns false.

### 30.8.14.3 Non-member functions

[time.cal.ymd.nonmembers]

```
constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
```

1 *Returns:* `x.year() == y.year() && x.month() == y.month() && x.day() == y.day()`.

```
constexpr strong_ordering operator<=>(const year_month_day& x, const year_month_day& y) noexcept;
```

2 *Effects:* Equivalent to:

```
 if (auto c = x.year() <=> y.year(); c != 0) return c;
 if (auto c = x.month() <=> y.month(); c != 0) return c;
 return x.day() <=> y.day();
```

```
constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
```

3 *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

4 *Returns:* `(ymd.year() / ymd.month() + dm) / ymd.day()`.

5 *[Note 1:* If `ymd.day()` is in the range `[1d, 28d]`, `ok()` will return true for the resultant `year_month_day`. — end note]

```
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
```

6 *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

7 *Returns:* `ymd + dm`.

```
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
```

8 *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

9 *Returns:* `ymd + (-dm)`.

```
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
```

10 *Returns:* `(ymd.year() + dy) / ymd.month() / ymd.day()`.

11 *[Note 2:* If `ymd.month()` is February and `ymd.day()` is not in the range `[1d, 28d]`, `ok()` can return false for the resultant `year_month_day`. — end note]



```
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
```

12 *Returns:* ymd + dy.

```
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;
```

13 *Returns:* ymd + (-dy).

```
template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);
```

14 *Effects:* Equivalent to:

```
 return os << (ymd.ok() ?
 format(STATICALLY-WIDEN<charT>("{:%F}"), ymd) :
 format(STATICALLY-WIDEN<charT>("{:%F} is not a valid date"), ymd));
```

```
template<class charT, class traits, class Alloc = allocator<charT>>
 basic_istream<charT, traits>&
 from_stream(basic_istream<charT, traits>& is, const charT* fmt,
 year_month_day& ymd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
 minutes* offset = nullptr);
```

15 *Effects:* Attempts to parse the input stream is into the year\_month\_day ymd using the format flags given in the NTCTS fmt as specified in 30.13. If the parse fails to decode a valid year\_month\_day, is.setstate(ios\_base::failbit) is called and ymd is not modified. If %Z is used and successfully parsed, that value will be assigned to \*abbrev if abbrev is non-null. If %z (or a modified variant) is used and successfully parsed, that value will be assigned to \*offset if offset is non-null.

16 *Returns:* is.

### 30.8.15 Class year\_month\_day\_last

[time.cal.ymdlast]

#### 30.8.15.1 Overview

[time.cal.ymdlast.overview]

```
namespace std::chrono {
 class year_month_day_last {
 chrono::year y_; // exposition only
 chrono::month_day_last mdl_; // exposition only

 public:
 constexpr year_month_day_last(const chrono::year& y,
 const chrono::month_day_last& mdl) noexcept;

 constexpr year_month_day_last& operator+=(const months& m) noexcept;
 constexpr year_month_day_last& operator-=(const months& m) noexcept;
 constexpr year_month_day_last& operator+=(const years& y) noexcept;
 constexpr year_month_day_last& operator-=(const years& y) noexcept;

 constexpr chrono::year year() const noexcept;
 constexpr chrono::month month() const noexcept;
 constexpr chrono::month_day_last month_day_last() const noexcept;
 constexpr chrono::day day() const noexcept;

 constexpr operator sys_days() const noexcept;
 constexpr explicit operator local_days() const noexcept;
 constexpr bool ok() const noexcept;
 };
}
```

<sup>1</sup> year\_month\_day\_last represents the last day of a specific year and month. year\_month\_day\_last is a field-based time point with a resolution of days, except that it is restricted to pointing to the last day of a year and month.

[Note 1: year\_month\_day\_last supports years- and months-oriented arithmetic, but not days-oriented arithmetic. For the latter, there is a conversion to sys\_days, which efficiently supports days-oriented arithmetic. — end note]

year\_month\_day\_last meets the Cpp17EqualityComparable (Table 28) and Cpp17LessThanComparable (Table 29) requirements.

<sup>2</sup> `year_month_day_last` is a trivially copyable and standard-layout class type.

### 30.8.15.2 Member functions

[time.cal.ymdlast.members]

```
constexpr year_month_day_last(const chrono::year& y,
 const chrono::month_day_last& mdl) noexcept;
```

<sup>1</sup> *Effects:* Initializes `y_` with `y` and `mdl_` with `mdl`.

```
constexpr year_month_day_last& operator+=(const months& m) noexcept;
```

<sup>2</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

<sup>3</sup> *Effects:* `*this = *this + m`.

<sup>4</sup> *Returns:* `*this`.

```
constexpr year_month_day_last& operator-=(const months& m) noexcept;
```

<sup>5</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

<sup>6</sup> *Effects:* `*this = *this - m`.

<sup>7</sup> *Returns:* `*this`.

```
constexpr year_month_day_last& operator+=(const years& y) noexcept;
```

<sup>8</sup> *Effects:* `*this = *this + y`.

<sup>9</sup> *Returns:* `*this`.

```
constexpr year_month_day_last& operator-=(const years& y) noexcept;
```

<sup>10</sup> *Effects:* `*this = *this - y`.

<sup>11</sup> *Returns:* `*this`.

```
constexpr chrono::year year() const noexcept;
```

<sup>12</sup> *Returns:* `y_`.

```
constexpr chrono::month month() const noexcept;
```

<sup>13</sup> *Returns:* `mdl_.month()`.

```
constexpr chrono::month_day_last month_day_last() const noexcept;
```

<sup>14</sup> *Returns:* `mdl_`.

```
constexpr chrono::day day() const noexcept;
```

<sup>15</sup> *Returns:* If `ok()` is `true`, returns a `day` representing the last day of the (`year`, `month`) pair represented by `*this`. Otherwise, the returned value is unspecified.

<sup>16</sup> [Note 1: This value might be computed on demand. — end note]

```
constexpr operator sys_days() const noexcept;
```

<sup>17</sup> *Returns:* `sys_days{year()/month()/day()}`.

```
constexpr explicit operator local_days() const noexcept;
```

<sup>18</sup> *Returns:* `local_days{sys_days{*this}.time_since_epoch()}`.

```
constexpr bool ok() const noexcept;
```

<sup>19</sup> *Returns:* `y_.ok() && mdl_.ok()`.

### 30.8.15.3 Non-member functions

[time.cal.ymdlast.nonmembers]

```
constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

<sup>1</sup> *Returns:* `x.year() == y.year() && x.month_day_last() == y.month_day_last()`.

```
constexpr strong_ordering operator<=>(const year_month_day_last& x,
 const year_month_day_last& y) noexcept;
```

2     *Effects:* Equivalent to:

```
 if (auto c = x.year() <=> y.year(); c != 0) return c;
 return x.month_day_last() <=> y.month_day_last();
```

```
constexpr year_month_day_last
operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
```

3     *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

4     *Returns:* `(ymdl.year() / ymdl.month() + dm) / last`.

```
constexpr year_month_day_last
operator-(const months& dm, const year_month_day_last& ymdl) noexcept;
```

5     *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

6     *Returns:* `ymdl + dm`.

```
constexpr year_month_day_last
operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
```

7     *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

8     *Returns:* `ymdl + (-dm)`.

```
constexpr year_month_day_last
operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
```

9     *Returns:* `{ymdl.year()+dy, ymdl.month_day_last()}`.

```
constexpr year_month_day_last
operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
```

10    *Returns:* `ymdl + dy`.

```
constexpr year_month_day_last
operator-(const year_month_day_last& ymdl, const years& dy) noexcept;
```

11    *Returns:* `ymdl + (-dy)`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

12    *Effects:* Equivalent to:

```
 return os << format(os.getloc(), STatically-WIDEN<charT>("{}/{:L}"),
 ymdl.year(), ymdl.month_day_last());
```

### 30.8.16 Class `year_month_weekday`

[time.cal.ymwd]

#### 30.8.16.1 Overview

[time.cal.ymwd.overview]

```
namespace std::chrono {
 class year_month_weekday {
 chrono::year y_; // exposition only
 chrono::month m_; // exposition only
 chrono::weekday_indexed wdi_; // exposition only

 public:
 year_month_weekday() = default;
 constexpr year_month_weekday(const chrono::year& y, const chrono::month& m,
 const chrono::weekday_indexed& wdi) noexcept;
 constexpr year_month_weekday(const sys_days& dp) noexcept;
 constexpr explicit year_month_weekday(const local_days& dp) noexcept;
```

```
constexpr year_month_weekday& operator+=(const months& m) noexcept;
constexpr year_month_weekday& operator-=(const months& m) noexcept;
constexpr year_month_weekday& operator+=(const years& y) noexcept;
constexpr year_month_weekday& operator-=(const years& y) noexcept;
```

```
constexpr chrono::year year() const noexcept;
constexpr chrono::month month() const noexcept;
constexpr chrono::weekday weekday() const noexcept;
constexpr unsigned index() const noexcept;
constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
```

```
constexpr operator sys_days() const noexcept;
constexpr explicit operator local_days() const noexcept;
constexpr bool ok() const noexcept;
```

```
};
```

```
}
```

- <sup>1</sup> `year_month_weekday` represents a specific year, month, and  $n^{\text{th}}$  weekday of the month. `year_month_weekday` is a field-based time point with a resolution of days.

[*Note 1: `year_month_weekday` supports years- and months-oriented arithmetic, but not days-oriented arithmetic. For the latter, there is a conversion to `sys_days`, which efficiently supports days-oriented arithmetic. — end note*]

`year_month_weekday` meets the *Cpp17EqualityComparable* (Table 28) requirements.

- <sup>2</sup> `year_month_weekday` is a trivially copyable and standard-layout class type.

### 30.8.16.2 Member functions

[time.cal.ymwd.members]

```
constexpr year_month_weekday(const chrono::year& y, const chrono::month& m,
 const chrono::weekday_indexed& wdi) noexcept;
```

- <sup>1</sup> *Effects:* Initializes `y_` with `y`, `m_` with `m`, and `wdi_` with `wdi`.

```
constexpr year_month_weekday(const sys_days& dp) noexcept;
```

- <sup>2</sup> *Effects:* Constructs an object of type `year_month_weekday` which corresponds to the date represented by `dp`.

- <sup>3</sup> *Remarks:* For any value `ymwd` of type `year_month_weekday` for which `ymwd.ok()` is true, `ymwd == year_month_weekday{sys_days{ymwd}}` is true.

```
constexpr explicit year_month_weekday(const local_days& dp) noexcept;
```

- <sup>4</sup> *Effects:* Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```
constexpr year_month_weekday& operator+=(const months& m) noexcept;
```

- <sup>5</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

- <sup>6</sup> *Effects:* `*this = *this + m`.

- <sup>7</sup> *Returns:* `*this`.

```
constexpr year_month_weekday& operator-=(const months& m) noexcept;
```

- <sup>8</sup> *Constraints:* If the argument supplied by the caller for the `months` parameter is convertible to `years`, its implicit conversion sequence to `years` is worse than its implicit conversion sequence to `months` (12.2.4.3).

- <sup>9</sup> *Effects:* `*this = *this - m`.

- <sup>10</sup> *Returns:* `*this`.

```
constexpr year_month_weekday& operator+=(const years& y) noexcept;
```

- <sup>11</sup> *Effects:* `*this = *this + y`.

- <sup>12</sup> *Returns:* `*this`.

```
constexpr year_month_weekday& operator-=(const years& y) noexcept;
```

- <sup>13</sup> *Effects:* `*this = *this - y`.

- <sup>14</sup> *Returns:* `*this`.

```

constexpr chrono::year year() const noexcept;
15 Returns: y_.

constexpr chrono::month month() const noexcept;
16 Returns: m_.

constexpr chrono::weekday weekday() const noexcept;
17 Returns: wdi_.weekday().

constexpr unsigned index() const noexcept;
18 Returns: wdi_.index().

constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
19 Returns: wdi_.

constexpr operator sys_days() const noexcept;
20 Returns: If y_.ok() && m_.ok() && wdi_.weekday().ok(), returns a sys_days that represents
the date (index() - 1) * 7 days after the first weekday() of year()/month(). If index() is 0
the returned sys_days represents the date 7 days prior to the first weekday() of year()/month().
Otherwise the returned value is unspecified.

constexpr explicit operator local_days() const noexcept;
21 Returns: local_days{sys_days{*this}.time_since_epoch()}.

constexpr bool ok() const noexcept;
22 Returns: If any of y_.ok(), m_.ok(), or wdi_.ok() is false, returns false. Otherwise, if *this
represents a valid date, returns true. Otherwise, returns false.

```

### 30.8.16.3 Non-member functions

[time.cal.ymwd.nonmembers]

```

constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
1 Returns:
 x.year() == y.year() && x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
2 Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its
implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.2.4.3).
3 Returns: (ymwd.year() / ymwd.month() + dm) / ymwd.weekday_indexed().

constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
4 Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its
implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.2.4.3).
5 Returns: ymwd + dm.

constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
6 Constraints: If the argument supplied by the caller for the months parameter is convertible to years, its
implicit conversion sequence to years is worse than its implicit conversion sequence to months (12.2.4.3).
7 Returns: ymwd + (-dm).

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
8 Returns: {ymwd.year()+dy, ymwd.month(), ymwd.weekday_indexed()}.

constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
9 Returns: ymwd + dy.

constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;
10 Returns: ymwd + (-dy).

```

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwd);
```

11 *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{:L}/{:L}"),
ymwd.year(), ymwd.month(), ymwd.weekday_indexed());
```

### 30.8.17 Class year\_month\_weekday\_last

[time.cal.ymwdlast]

#### 30.8.17.1 Overview

[time.cal.ymwdlast.overview]

```
namespace std::chrono {
class year_month_weekday_last {
 chrono::year y_; // exposition only
 chrono::month m_; // exposition only
 chrono::weekday_last wdl_; // exposition only

public:
 constexpr year_month_weekday_last(const chrono::year& y, const chrono::month& m,
 const chrono::weekday_last& wdl) noexcept;

 constexpr year_month_weekday_last& operator+=(const months& m) noexcept;
 constexpr year_month_weekday_last& operator-=(const months& m) noexcept;
 constexpr year_month_weekday_last& operator+=(const years& y) noexcept;
 constexpr year_month_weekday_last& operator-=(const years& y) noexcept;

 constexpr chrono::year year() const noexcept;
 constexpr chrono::month month() const noexcept;
 constexpr chrono::weekday weekday() const noexcept;
 constexpr chrono::weekday_last weekday_last() const noexcept;

 constexpr operator sys_days() const noexcept;
 constexpr explicit operator local_days() const noexcept;
 constexpr bool ok() const noexcept;
};
}
```

- 1 **year\_month\_weekday\_last** represents a specific year, month, and last weekday of the month. **year\_month\_weekday\_last** is a field-based time point with a resolution of **days**, except that it is restricted to pointing to the last weekday of a year and month.

[*Note 1:* **year\_month\_weekday\_last** supports **years**- and **months**-oriented arithmetic, but not **days**-oriented arithmetic. For the latter, there is a conversion to **sys\_days**, which efficiently supports **days**-oriented arithmetic. — *end note*]

**year\_month\_weekday\_last** meets the *Cpp17EqualityComparable* (Table 28) requirements.

- 2 **year\_month\_weekday\_last** is a trivially copyable and standard-layout class type.

#### 30.8.17.2 Member functions

[time.cal.ymwdlast.members]

```
constexpr year_month_weekday_last(const chrono::year& y, const chrono::month& m,
 const chrono::weekday_last& wdl) noexcept;
```

- 1 *Effects:* Initializes **y\_** with **y**, **m\_** with **m**, and **wdl\_** with **wdl**.

```
constexpr year_month_weekday_last& operator+=(const months& m) noexcept;
```

- 2 *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

- 3 *Effects:* **\*this = \*this + m**.

- 4 *Returns:* **\*this**.

```
constexpr year_month_weekday_last& operator-=(const months& m) noexcept;
```

- 5 *Constraints:* If the argument supplied by the caller for the **months** parameter is convertible to **years**, its implicit conversion sequence to **years** is worse than its implicit conversion sequence to **months** (12.2.4.3).

- 6 *Effects:* **\*this = \*this - m**.

7       *Returns:* *\*this*.

      constexpr year\_month\_weekday\_last& operator+=(const years& y) noexcept;

8       *Effects:* *\*this* = *\*this* + *y*.

9       *Returns:* *\*this*.

      constexpr year\_month\_weekday\_last& operator-=(const years& y) noexcept;

10      *Effects:* *\*this* = *\*this* - *y*.

11      *Returns:* *\*this*.

      constexpr chrono::year year() const noexcept;

12      *Returns:* *y\_*.

      constexpr chrono::month month() const noexcept;

13      *Returns:* *m\_*.

      constexpr chrono::weekday weekday() const noexcept;

14      *Returns:* *wdl\_*.weekday().

      constexpr chrono::weekday\_last weekday\_last() const noexcept;

15      *Returns:* *wdl\_*.

      constexpr operator sys\_days() const noexcept;

16      *Returns:* If *ok()* == *true*, returns a *sys\_days* that represents the last *weekday()* of *year()/month()*. Otherwise the returned value is unspecified.

      constexpr explicit operator local\_days() const noexcept;

17      *Returns:* *local\_days{sys\_days{\*this}.time\_since\_epoch()}*.

      constexpr bool ok() const noexcept;

18      *Returns:* *y\_.ok()* && *m\_.ok()* && *wdl\_.ok()*.

### 30.8.17.3 Non-member functions

[time.cal.ymwdlast.nonmembers]

constexpr bool operator==(const year\_month\_weekday\_last& x,  
                          const year\_month\_weekday\_last& y) noexcept;

1       *Returns:*

*x.year()* == *y.year()* && *x.month()* == *y.month()* && *x.weekday\_last()* == *y.weekday\_last()*

constexpr year\_month\_weekday\_last  
operator+(const year\_month\_weekday\_last& ymwdl, const months& dm) noexcept;

2       *Constraints:* If the argument supplied by the caller for the *months* parameter is convertible to *years*, its implicit conversion sequence to *years* is worse than its implicit conversion sequence to *months* (12.2.4.3).

3       *Returns:* (*ymwdl.year()* / *ymwdl.month()* + *dm*) / *ymwdl.weekday\_last()*.

constexpr year\_month\_weekday\_last  
operator+(const months& dm, const year\_month\_weekday\_last& ymwdl) noexcept;

4       *Constraints:* If the argument supplied by the caller for the *months* parameter is convertible to *years*, its implicit conversion sequence to *years* is worse than its implicit conversion sequence to *months* (12.2.4.3).

5       *Returns:* *ymwdl* + *dm*.

constexpr year\_month\_weekday\_last  
operator-(const year\_month\_weekday\_last& ymwdl, const months& dm) noexcept;

6       *Constraints:* If the argument supplied by the caller for the *months* parameter is convertible to *years*, its implicit conversion sequence to *years* is worse than its implicit conversion sequence to *months* (12.2.4.3).

7       *Returns:* *ymwdl* + (-*dm*).

```

constexpr year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
8 Returns: {ymwdl.year()+dy, ymwdl.month(), ymwdl.weekday_last()}.

constexpr year_month_weekday_last
operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
9 Returns: ymwdl + dy.

constexpr year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
10 Returns: ymwdl + (-dy).

template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
11 Effects: Equivalent to:
 return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{}/{:L}/{:L}"),
 ymwdl.year(), ymwdl.month(), ymwdl.weekday_last());

```

### 30.8.18 Conventional syntax operators [time.cal.operators]

- 1 A set of overloaded `operator/` functions provides a conventional syntax for the creation of civil calendar dates.

- 2 [Note 1: The year, month, and day are accepted in any of the following 3 orders:

```

year/month/day
month/day/year
day/month/year

```

Anywhere a *day* is needed, any of the following can also be specified:

```

last
weekday[i]
weekday[last]

```

— end note]

- 3 [Note 2: Partial-date types such as `year_month` and `month_day` can be created by not applying the second division operator for any of the three orders. For example:

```

year_month ym = 2015y/April;
month_day md1 = April/4;
month_day md2 = 4d/April;

```

— end note]

- 4 [Example 1:

```

auto a = 2015/4/4; // a == int(125)
auto b = 2015y/4/4; // b == year_month_day{year(2015), month(4), day(4)}
auto c = 2015y/4d/April; // error: no viable operator/ for first /
auto d = 2015/April/4; // error: no viable operator/ for first /

```

— end example]

```

constexpr year_month
operator/(const year& y, const month& m) noexcept;

```

- 5 Returns: {y, m}.

```

constexpr year_month
operator/(const year& y, int m) noexcept;

```

- 6 Returns: y / month(m).

```

constexpr month_day
operator/(const month& m, const day& d) noexcept;

```

- 7 Returns: {m, d}.



```

constexpr month_day
operator/(const month& m, int d) noexcept;
8 Returns: m / day(d).

constexpr month_day
operator/(int m, const day& d) noexcept;
9 Returns: month(m) / d.

constexpr month_day
operator/(const day& d, const month& m) noexcept;
10 Returns: m / d.

constexpr month_day
operator/(const day& d, int m) noexcept;
11 Returns: month(m) / d.

constexpr month_day_last
operator/(const month& m, last_spec) noexcept;
12 Returns: month_day_last{m}.

constexpr month_day_last
operator/(int m, last_spec) noexcept;
13 Returns: month(m) / last.

constexpr month_day_last
operator/(last_spec, const month& m) noexcept;
14 Returns: m / last.

constexpr month_day_last
operator/(last_spec, int m) noexcept;
15 Returns: month(m) / last.

constexpr month_weekday
operator/(const month& m, const weekday_indexed& wdi) noexcept;
16 Returns: {m, wdi}.

constexpr month_weekday
operator/(int m, const weekday_indexed& wdi) noexcept;
17 Returns: month(m) / wdi.

constexpr month_weekday
operator/(const weekday_indexed& wdi, const month& m) noexcept;
18 Returns: m / wdi.

constexpr month_weekday
operator/(const weekday_indexed& wdi, int m) noexcept;
19 Returns: month(m) / wdi.

constexpr month_weekday_last
operator/(const month& m, const weekday_last& wdl) noexcept;
20 Returns: {m, wdl}.

constexpr month_weekday_last
operator/(int m, const weekday_last& wdl) noexcept;
21 Returns: month(m) / wdl.

constexpr month_weekday_last
operator/(const weekday_last& wdl, const month& m) noexcept;
22 Returns: m / wdl.

```

```

constexpr month_weekday_last
 operator/(const weekday_last& wdl, int m) noexcept;
23 Returns: month(m) / wdl.

constexpr year_month_day
 operator/(const year_month& ym, const day& d) noexcept;
24 Returns: {ym.year(), ym.month(), d}.

constexpr year_month_day
 operator/(const year_month& ym, int d) noexcept;
25 Returns: ym / day(d).

constexpr year_month_day
 operator/(const year& y, const month_day& md) noexcept;
26 Returns: y / md.month() / md.day().

constexpr year_month_day
 operator/(int y, const month_day& md) noexcept;
27 Returns: year(y) / md.

constexpr year_month_day
 operator/(const month_day& md, const year& y) noexcept;
28 Returns: y / md.

constexpr year_month_day
 operator/(const month_day& md, int y) noexcept;
29 Returns: year(y) / md.

constexpr year_month_day_last
 operator/(const year_month& ym, last_spec) noexcept;
30 Returns: {ym.year(), month_day_last{ym.month()}}.

constexpr year_month_day_last
 operator/(const year& y, const month_day_last& mdl) noexcept;
31 Returns: {y, mdl}.

constexpr year_month_day_last
 operator/(int y, const month_day_last& mdl) noexcept;
32 Returns: year(y) / mdl.

constexpr year_month_day_last
 operator/(const month_day_last& mdl, const year& y) noexcept;
33 Returns: y / mdl.

constexpr year_month_day_last
 operator/(const month_day_last& mdl, int y) noexcept;
34 Returns: year(y) / mdl.

constexpr year_month_weekday
 operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;
35 Returns: {ym.year(), ym.month(), wdi}.

constexpr year_month_weekday
 operator/(const year& y, const month_weekday& mwd) noexcept;
36 Returns: {y, mwd.month(), mwd.weekday_indexed()}.

constexpr year_month_weekday
 operator/(int y, const month_weekday& mwd) noexcept;
37 Returns: year(y) / mwd.

```

```

constexpr year_month_weekday
operator/(const month_weekday& mwd, const year& y) noexcept;
38 Returns: y / mwd.

constexpr year_month_weekday
operator/(const month_weekday& mwd, int y) noexcept;
39 Returns: year(y) / mwd.

constexpr year_month_weekday_last
operator/(const year_month& ym, const weekday_last& wdl) noexcept;
40 Returns: {ym.year(), ym.month(), wdl}.

constexpr year_month_weekday_last
operator/(const year& y, const month_weekday_last& mwdl) noexcept;
41 Returns: {y, mwdl.month(), mwdl.weekday_last()}.

constexpr year_month_weekday_last
operator/(int y, const month_weekday_last& mwdl) noexcept;
42 Returns: year(y) / mwdl.

constexpr year_month_weekday_last
operator/(const month_weekday_last& mwdl, const year& y) noexcept;
43 Returns: y / mwdl.

constexpr year_month_weekday_last
operator/(const month_weekday_last& mwdl, int y) noexcept;
44 Returns: year(y) / mwdl.

```

## 30.9 Class template `hh_mm_ss`

[time.hms]

### 30.9.1 Overview

[time.hms.overview]

```

namespace std::chrono {
 template<class Duration> class hh_mm_ss {
 public:
 static constexpr unsigned fractional_width = see below;
 using precision = see below;

 constexpr hh_mm_ss() noexcept : hh_mm_ss{Duration::zero()} {}
 constexpr explicit hh_mm_ss(Duration d);

 constexpr bool is_negative() const noexcept;
 constexpr chrono::hours hours() const noexcept;
 constexpr chrono::minutes minutes() const noexcept;
 constexpr chrono::seconds seconds() const noexcept;
 constexpr precision subseconds() const noexcept;

 constexpr explicit operator precision() const noexcept;
 constexpr precision to_duration() const noexcept;

 private:
 bool is-neg; // exposition only
 chrono::hours h_; // exposition only
 chrono::minutes m_; // exposition only
 chrono::seconds s_; // exposition only
 precision ss_; // exposition only
 };
}

```

<sup>1</sup> The `hh_mm_ss` class template splits a `duration` into a multi-field time structure *hours:minutes:seconds* and possibly *subseconds*, where *subseconds* will be a duration unit based on a non-positive power of 10. The `Duration` template parameter dictates the precision to which the time is split. A `hh_mm_ss` models negative durations with a distinct `is_negative` getter that returns `true` when the input duration is negative. The

individual duration fields always return non-negative durations even when `is_negative()` indicates the structure is representing a negative duration.

- <sup>2</sup> If `Duration` is not a specialization of `duration`, the program is ill-formed.

### 30.9.2 Members

[time.hms.members]

static constexpr unsigned fractional\_width = see below;

- <sup>1</sup> `fractional_width` is the number of fractional decimal digits represented by `precision`. `fractional_width` has the value of the smallest possible integer in the range  $[0, 18]$  such that `precision` will exactly represent all values of `Duration`. If no such value of `fractional_width` exists, then `fractional_width` is 6.

[Example 1: See Table 132 for some durations, the resulting `fractional_width`, and the formatted fractional second output of `Duration{1}`.]

Table 132 — Examples for `fractional_width` [tab:time.hms.width]

| Duration                                                | fractional_width | Formatted fractional second output |
|---------------------------------------------------------|------------------|------------------------------------|
| hours, minutes, and seconds                             | 0                |                                    |
| milliseconds                                            | 3                | 0.001                              |
| microseconds                                            | 6                | 0.000001                           |
| nanoseconds                                             | 9                | 0.000000001                        |
| <code>duration&lt;int, ratio&lt;1, 2&gt;&gt;</code>     | 1                | 0.5                                |
| <code>duration&lt;int, ratio&lt;1, 3&gt;&gt;</code>     | 6                | 0.333333                           |
| <code>duration&lt;int, ratio&lt;1, 4&gt;&gt;</code>     | 2                | 0.25                               |
| <code>duration&lt;int, ratio&lt;1, 5&gt;&gt;</code>     | 1                | 0.2                                |
| <code>duration&lt;int, ratio&lt;1, 6&gt;&gt;</code>     | 6                | 0.166666                           |
| <code>duration&lt;int, ratio&lt;1, 7&gt;&gt;</code>     | 6                | 0.142857                           |
| <code>duration&lt;int, ratio&lt;1, 8&gt;&gt;</code>     | 3                | 0.125                              |
| <code>duration&lt;int, ratio&lt;1, 9&gt;&gt;</code>     | 6                | 0.111111                           |
| <code>duration&lt;int, ratio&lt;1, 10&gt;&gt;</code>    | 1                | 0.1                                |
| <code>duration&lt;int, ratio&lt;756, 625&gt;&gt;</code> | 4                | 0.2096                             |

— end example]

using precision = see below;

- <sup>2</sup> precision is

`duration<common_type_t<Duration::rep, seconds::rep>, ratio<1, 10fractional_width>>`

constexpr explicit hh\_mm\_ss(Duration d);

- <sup>3</sup> *Effects:* Constructs an object of type `hh_mm_ss` which represents the `Duration` `d` with precision `precision`.

- (3.1) — Initializes *is-neg* with `d < Duration::zero()`. Let *ABS\_D* represent `-d` if *is-neg* is true and `d` otherwise.
- (3.2) — Initializes *h\_* with `duration_cast<chrono::hours>(ABS_D)`.
- (3.3) — Initializes *m\_* with `duration_cast<chrono::minutes>(ABS_D - hours())`.
- (3.4) — Initializes *s\_* with `duration_cast<chrono::seconds>(ABS_D - hours() - minutes())`.
- (3.5) — If `treat_as_floating_point_v<precision::rep>` is true, initializes *ss\_* with `ABS_D - hours() - minutes() - seconds()`. Otherwise, initializes *ss\_* with `duration_cast<precision>(ABS_D - hours() - minutes() - seconds())`.

[Note 1: When `precision::rep` is integral and `precision::period` is `ratio<1>`, `subseconds()` always returns a value equal to 0s. — end note]

- <sup>4</sup> *Postconditions:* If `treat_as_floating_point_v<precision::rep>` is true, `to_duration()` returns `d`, otherwise `to_duration()` returns `duration_cast<precision>(d)`.

constexpr bool is\_negative() const noexcept;

- <sup>5</sup> Returns: *is-neg*.

```
constexpr chrono::hours hours() const noexcept;
```

6     *Returns:*  $h_.$

```
constexpr chrono::minutes minutes() const noexcept;
```

7     *Returns:*  $m_.$

```
constexpr chrono::seconds seconds() const noexcept;
```

8     *Returns:*  $s_.$

```
constexpr precision subseconds() const noexcept;
```

9     *Returns:*  $ss_.$

```
constexpr precision to_duration() const noexcept;
```

10    *Returns:* If *is-neg*, returns  $-(h_ + m_ + s_ + ss_)$ , otherwise returns  $h_ + m_ + s_ + ss_.$

```
constexpr explicit operator precision() const noexcept;
```

11    *Returns:* `to_duration()`.

### 30.9.3 Non-members

[time.hms.nonmembers]

```
template<class charT, class traits, class Duration>
```

```
basic_ostream<charT, traits>&
```

```
operator<<(basic_ostream<charT, traits>& os, const hh_mm_ss<Duration>& hms);
```

1     *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%T}"), hms);
```

2     [*Example 1:*

```
for (auto ms : {-4083007ms, 4083007ms, 65745123ms}) {
 hh_mm_ss hms{ms};
 cout << hms << '\n';
}
cout << hh_mm_ss{65745s} << '\n';
```

Produces the output (assuming the "C" locale):

```
-01:08:03.007
01:08:03.007
18:15:45.123
18:15:45
```

— *end example*]

### 30.10 12/24 hours functions

[time.12]

1 These functions aid in translating between a 12h format time of day and a 24h format time of day.

```
constexpr bool is_am(const hours& h) noexcept;
```

2     *Returns:*  $0h \leq h \ \&\& \ h \leq 11h.$

```
constexpr bool is_pm(const hours& h) noexcept;
```

3     *Returns:*  $12h \leq h \ \&\& \ h \leq 23h.$

```
constexpr hours make12(const hours& h) noexcept;
```

4     *Returns:* The 12-hour equivalent of  $h$  in the range  $[1h, 12h]$ . If  $h$  is not in the range  $[0h, 23h]$ , the value returned is unspecified.

```
constexpr hours make24(const hours& h, bool is_pm) noexcept;
```

5     *Returns:* If *is\_pm* is false, returns the 24-hour equivalent of  $h$  in the range  $[0h, 11h]$ , assuming  $h$  represents an ante meridiem hour. Otherwise, returns the 24-hour equivalent of  $h$  in the range  $[12h, 23h]$ , assuming  $h$  represents a post meridiem hour. If  $h$  is not in the range  $[1h, 12h]$ , the value returned is unspecified.

**30.11 Time zones****[time.zone]****30.11.1 General****[time.zone.general]**

- <sup>1</sup> 30.11 describes an interface for accessing the IANA Time Zone Database<sup>[10]</sup> that interoperates with `sys_time` and `local_time`. This interface provides time zone support to both the civil calendar types (30.8) and to user-defined calendars.

**30.11.2 Time zone database****[time.zone.db]****30.11.2.1 Class tzdb****[time.zone.db.tzdb]**

```
namespace std::chrono {
 struct tzdb {
 string version;
 vector<time_zone> zones;
 vector<time_zone_link> links;
 vector<leap_second> leap_seconds;

 const time_zone* locate_zone(string_view tz_name) const;
 const time_zone* current_zone() const;
 };
}
```

- <sup>1</sup> Each vector in a `tzdb` object is sorted to enable fast lookup.

```
const time_zone* locate_zone(string_view tz_name) const;
```

- <sup>2</sup> *Returns:*

- (2.1) — If `zones` contains an element `tz` for which `tz.name() == tz_name`, a pointer to `tz`;
- (2.2) — otherwise, if `links` contains an element `tz_l` for which `tz_l.name() == tz_name`, then a pointer to the element `tz` of `zones` for which `tz.name() == tz_l.target()`.

[Note 1: A `time_zone_link` specifies an alternative name for a `time_zone`. — end note]

- <sup>3</sup> *Throws:* If a `const time_zone*` cannot be found as described in the *Returns:* element, throws a `runtime_error`.

[Note 2: On non-exceptional return, the return value is always a pointer to a valid `time_zone`. — end note]

```
const time_zone* current_zone() const;
```

- <sup>4</sup> *Returns:* A pointer to the time zone which the computer has set as its local time zone.

**30.11.2.2 Class tzdb\_list****[time.zone.db.list]**

```
namespace std::chrono {
 class tzdb_list {
 public:
 tzdb_list(const tzdb_list&) = delete;
 tzdb_list& operator=(const tzdb_list&) = delete;

 // unspecified additional constructors

 class const_iterator;

 const tzdb& front() const noexcept;

 const_iterator erase_after(const_iterator p);

 const_iterator begin() const noexcept;
 const_iterator end() const noexcept;

 const_iterator cbegin() const noexcept;
 const_iterator cend() const noexcept;
 };
}
```

- <sup>1</sup> The `tzdb_list` database is a singleton; the unique object of type `tzdb_list` can be accessed via the `get_tzdb_list()` function.

[*Note 1*: This access is only needed for those applications that need to have long uptimes and have a need to update the time zone database while running. Other applications can implicitly access the `front()` of this list via the read-only namespace scope functions `get_tzdb()`, `locate_zone()`, and `current_zone()`. — *end note*]

The `tzdb_list` object contains a list of `tzdb` objects.

- 2 `tzdb_list::const_iterator` is a constant iterator which meets the *Cpp17ForwardIterator* requirements and has a value type of `tzdb`.

```
const tzdb& front() const noexcept;
```

- 3 *Synchronization*: This operation is thread-safe with respect to `reload_tzdb()`.

[*Note 2*: `reload_tzdb()` pushes a new `tzdb` onto the front of this container. — *end note*]

- 4 *Returns*: A reference to the first `tzdb` in the container.

```
const_iterator erase_after(const_iterator p);
```

- 5 *Preconditions*: The iterator following `p` is dereferenceable.

- 6 *Effects*: Erases the `tzdb` referred to by the iterator following `p`.

- 7 *Postconditions*: No pointers, references, or iterators are invalidated except those referring to the erased `tzdb`.

[*Note 3*: It is not possible to erase the `tzdb` referred to by `begin()`. — *end note*]

- 8 *Returns*: An iterator pointing to the element following the one that was erased, or `end()` if no such element exists.

- 9 *Throws*: Nothing.

```
const_iterator begin() const noexcept;
```

- 10 *Returns*: An iterator referring to the first `tzdb` in the container.

```
const_iterator end() const noexcept;
```

- 11 *Returns*: An iterator referring to the position one past the last `tzdb` in the container.

```
const_iterator cbegin() const noexcept;
```

- 12 *Returns*: `begin()`.

```
const_iterator cend() const noexcept;
```

- 13 *Returns*: `end()`.

### 30.11.2.3 Time zone database access

[`time.zone.db.access`]

```
tzdb_list& get_tzdb_list();
```

- 1 *Effects*: If this is the first access to the time zone database, initializes the database. If this call initializes the database, the resulting database will be a `tzdb_list` holding a single initialized `tzdb`.

- 2 *Synchronization*: It is safe to call this function from multiple threads at one time.

- 3 *Returns*: A reference to the database.

- 4 *Throws*: `runtime_error` if for any reason a reference cannot be returned to a valid `tzdb_list` containing one or more valid `tzdb`s.

```
const tzdb& get_tzdb();
```

- 5 *Returns*: `get_tzdb_list().front()`.

```
const time_zone* locate_zone(string_view tz_name);
```

- 6 *Returns*: `get_tzdb().locate_zone(tz_name)`.

- 7 [*Note 1*: The time zone database will be initialized if this is the first reference to the database. — *end note*]

```
const time_zone* current_zone();
```

- 8 *Returns*: `get_tzdb().current_zone()`.

### 30.11.2.4 Remote time zone database support

[time.zone.db.remote]

- 1 The local time zone database is that supplied by the implementation when the program first accesses the database, for example via `current_zone()`. While the program is running, the implementation may choose to update the time zone database. This update shall not impact the program in any way unless the program calls the functions in this subclause. This potentially updated time zone database is referred to as the *remote time zone database*.

```
const tzdb& reload_tzdb();
```

- 2 *Effects:* This function first checks the version of the remote time zone database. If the versions of the local and remote databases are the same, there are no effects. Otherwise the remote database is pushed to the front of the `tzdb_list` accessed by `get_tzdb_list()`.
- 3 *Synchronization:* This function is thread-safe with respect to `get_tzdb_list().front()` and `get_tzdb_list().erase_after()`.
- 4 *Postconditions:* No pointers, references, or iterators are invalidated.
- 5 *Returns:* `get_tzdb_list().front()`.
- 6 *Throws:* `runtime_error` if for any reason a reference cannot be returned to a valid `tzdb`.

```
string remote_version();
```

- 7 *Returns:* The latest remote database version.
- [Note 1: This can be compared with `get_tzdb().version` to discover if the local and remote databases are equivalent. — end note]

### 30.11.3 Exception classes

[time.zone.exception]

#### 30.11.3.1 Class `nonexistent_local_time`

[time.zone.exception.nonexist]

```
namespace std::chrono {
 class nonexistent_local_time : public runtime_error {
 public:
 template<class Duration>
 nonexistent_local_time(const local_time<Duration>& tp, const local_info& i);
 };
}
```

- 1 `nonexistent_local_time` is thrown when an attempt is made to convert a non-existent `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```
template<class Duration>
 nonexistent_local_time(const local_time<Duration>& tp, const local_info& i);
```

- 2 *Preconditions:* `i.result == local_info::nonexistent` is true.
- 3 *Effects:* Initializes the base class with a sequence of `char` equivalent to that produced by `os.str()` initialized as shown below:

```
ostringstream os;
os << tp << " is in a gap between\n"
 << local_seconds{i.first.end.time_since_epoch()} + i.first.offset << ' '
 << i.first.abbrev << " and\n"
 << local_seconds{i.second.begin.time_since_epoch()} + i.second.offset << ' '
 << i.second.abbrev
 << " which are both equivalent to\n"
 << i.first.end << " UTC";
```

- 4 [Example 1:

```
#include <chrono>
#include <iostream>

int main() {
 using namespace std::chrono;
 try {
 auto zt = zoned_time{"America/New_York",
 local_days{Sunday[2]/March/2016} + 2h + 30min};
```



```

 } catch (const nonexistent_local_time& e) {
 std::cout << e.what() << '\n';
 }
}

```

Produces the output:

```

2016-03-13 02:30:00 is in a gap between
2016-03-13 02:00:00 EST and
2016-03-13 03:00:00 EDT which are both equivalent to
2016-03-13 07:00:00 UTC

```

— end example]

### 30.11.3.2 Class `ambiguous_local_time`

[time.zone.exception.ambig]

```

namespace std::chrono {
 class ambiguous_local_time : public runtime_error {
 public:
 template<class Duration>
 ambiguous_local_time(const local_time<Duration>& tp, const local_info& i);
 };
}

```

<sup>1</sup> `ambiguous_local_time` is thrown when an attempt is made to convert an ambiguous `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```

template<class Duration>
 ambiguous_local_time(const local_time<Duration>& tp, const local_info& i);

```

<sup>2</sup> *Preconditions:* `i.result == local_info::ambiguous` is true.

<sup>3</sup> *Effects:* Initializes the base class with a sequence of `char` equivalent to that produced by `os.str()` initialized as shown below:

```

 ostream os;
 os << tp << " is ambiguous. It could be\n"
 << tp << ' ' << i.first.abbrev << " == "
 << tp - i.first.offset << " UTC or\n"
 << tp << ' ' << i.second.abbrev << " == "
 << tp - i.second.offset << " UTC";

```

<sup>4</sup> [Example 1:

```

#include <chrono>
#include <iostream>

int main() {
 using namespace std::chrono;
 try {
 auto zt = zoned_time{"America/New_York",
 local_days{Sunday[1]/November/2016} + 1h + 30min};
 } catch (const ambiguous_local_time& e) {
 std::cout << e.what() << '\n';
 }
}

```

Produces the output:

```

2016-11-06 01:30:00 is ambiguous. It could be
2016-11-06 01:30:00 EDT == 2016-11-06 05:30:00 UTC or
2016-11-06 01:30:00 EST == 2016-11-06 06:30:00 UTC

```

— end example]

## 30.11.4 Information classes

[time.zone.info]

### 30.11.4.1 Class `sys_info`

[time.zone.info.sys]

```

namespace std::chrono {
 struct sys_info {
 sys_seconds begin;
 };
}

```

```

 sys_seconds end;
 seconds offset;
 minutes save;
 string abbrev;
 };
}

```

- <sup>1</sup> A `sys_info` object can be obtained from the combination of a `time_zone` and either a `sys_time` or `local_time`. It can also be obtained from a `zoned_time`, which is effectively a pair of a `time_zone` and `sys_time`.
- <sup>2</sup> [*Note 1*: This type provides a low-level interface to time zone information. Typical conversions from `sys_time` to `local_time` will use this class implicitly, not explicitly. — *end note*]
- <sup>3</sup> The `begin` and `end` data members indicate that, for the associated `time_zone` and `time_point`, the `offset` and `abbrev` are in effect in the range `[begin, end)`. This information can be used to efficiently iterate the transitions of a `time_zone`.
- <sup>4</sup> The `offset` data member indicates the UTC offset in effect for the associated `time_zone` and `time_point`. The relationship between `local_time` and `sys_time` is:

```
offset = local_time - sys_time
```

- <sup>5</sup> The `save` data member is extra information not normally needed for conversion between `local_time` and `sys_time`. If `save != 0min`, this `sys_info` is said to be on “daylight saving” time, and `offset - save` suggests what offset this `time_zone` might use if it were off daylight saving time. However, this information should not be taken as authoritative. The only sure way to get such information is to query the `time_zone` with a `time_point` that returns a `sys_info` where `save == 0min`. There is no guarantee what `time_point` might return such a `sys_info` except that it is guaranteed not to be in the range `[begin, end)` (if `save != 0min` for this `sys_info`).
- <sup>6</sup> The `abbrev` data member indicates the current abbreviation used for the associated `time_zone` and `time_point`. Abbreviations are not unique among the `time_zones`, and so one cannot reliably map abbreviations back to a `time_zone` and UTC offset.

```

template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const sys_info& r);

```

<sup>7</sup> *Effects*: Streams out the `sys_info` object `r` in an unspecified format.

<sup>8</sup> *Returns*: `os`.

#### 30.11.4.2 Class `local_info`

[time.zone.info.local]

```

namespace std::chrono {
 struct local_info {
 static constexpr int unique = 0;
 static constexpr int nonexistent = 1;
 static constexpr int ambiguous = 2;

 int result;
 sys_info first;
 sys_info second;
 };
}

```

- <sup>1</sup> [*Note 1*: This type provides a low-level interface to time zone information. Typical conversions from `local_time` to `sys_time` will use this class implicitly, not explicitly. — *end note*]
- <sup>2</sup> Describes the result of converting a `local_time` to a `sys_time` as follows:
  - (2.1) — When a `local_time` to `sys_time` conversion is unique, `result == unique`, `first` will be filled out with the correct `sys_info`, and `second` will be zero-initialized.
  - (2.2) — If the conversion stems from a nonexistent `local_time` then `result == nonexistent`, `first` will be filled out with the `sys_info` that ends just prior to the `local_time`, and `second` will be filled out with the `sys_info` that begins just after the `local_time`.
  - (2.3) — If the conversion stems from an ambiguous `local_time`, then `result == ambiguous`, `first` will be filled out with the `sys_info` that ends just after the `local_time`, and `second` will be filled out with the `sys_info` that starts just before the `local_time`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_info& r);
```

3     *Effects:* Streams out the `local_info` object `r` in an unspecified format.

4     *Returns:* `os`.

### 30.11.5 Class `time_zone`

[time.zone.timezone]

#### 30.11.5.1 Overview

[time.zone.overview]

```
namespace std::chrono {
 class time_zone {
 public:
 time_zone(time_zone&&) = default;
 time_zone& operator=(time_zone&&) = default;

 // unspecified additional constructors

 string_view name() const noexcept;

 template<class Duration> sys_info get_info(const sys_time<Duration>& st) const;
 template<class Duration> local_info get_info(const local_time<Duration>& tp) const;

 template<class Duration>
 sys_time<common_type_t<Duration, seconds>>
 to_sys(const local_time<Duration>& tp) const;

 template<class Duration>
 sys_time<common_type_t<Duration, seconds>>
 to_sys(const local_time<Duration>& tp, choose z) const;

 template<class Duration>
 local_time<common_type_t<Duration, seconds>>
 to_local(const sys_time<Duration>& tp) const;
 };
}
```

- <sup>1</sup> A `time_zone` represents all time zone transitions for a specific geographic area. `time_zone` construction is unspecified, and performed as part of database initialization.

[*Note 1:* `const time_zone` objects can be accessed via functions such as `locate_zone`. — end note]

#### 30.11.5.2 Member functions

[time.zone.members]

```
string_view name() const noexcept;
```

- <sup>1</sup>     *Returns:* The name of the `time_zone`.

- <sup>2</sup>     [*Example 1:* "America/New\_York". — end example]

```
template<class Duration>
sys_info get_info(const sys_time<Duration>& st) const;
```

- <sup>3</sup>     *Returns:* A `sys_info` `i` for which `st` is in the range `[i.begin, i.end)`.

```
template<class Duration>
local_info get_info(const local_time<Duration>& tp) const;
```

- <sup>4</sup>     *Returns:* A `local_info` for `tp`.

```
template<class Duration>
sys_time<common_type_t<Duration, seconds>>
to_sys(const local_time<Duration>& tp) const;
```

- <sup>5</sup>     *Returns:* A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`.

- <sup>6</sup>     *Throws:* If the conversion from `tp` to a `sys_time` is ambiguous, throws `ambiguous_local_time`. If the `tp` represents a non-existent time between two UTC `time_points`, throws `nonexistent_local_time`.

```
template<class Duration>
sys_time<common_type_t<Duration, seconds>>
to_sys(const local_time<Duration>& tp, choose z) const;
```

- 7 *Returns:* A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`. If the conversion from `tp` to a `sys_time` is ambiguous, returns the earlier `sys_time` if `z == choose::earliest`, and returns the later `sys_time` if `z == choose::latest`. If the `tp` represents a non-existent time between two UTC `time_points`, then the two UTC `time_points` will be the same, and that UTC `time_point` will be returned.

```
template<class Duration>
local_time<common_type_t<Duration, seconds>>
to_local(const sys_time<Duration>& tp) const;
```

- 8 *Returns:* The `local_time` associated with `tp` and this `time_zone`.

### 30.11.5.3 Non-member functions

[time.zone.nonmembers]

```
bool operator==(const time_zone& x, const time_zone& y) noexcept;
```

- 1 *Returns:* `x.name() == y.name()`.

```
strong_ordering operator<=(const time_zone& x, const time_zone& y) noexcept;
```

- 2 *Returns:* `x.name() <= y.name()`.

### 30.11.6 Class template `zoned_traits`

[time.zone.zonedtraits]

```
namespace std::chrono {
template<class T> struct zoned_traits {};
}
```

- 1 `zoned_traits` provides a means for customizing the behavior of `zoned_time<Duration, TimeZonePtr>` for the `zoned_time` default constructor, and constructors taking `string_view`. A specialization for `const time_zone*` is provided by the implementation:

```
namespace std::chrono {
template<> struct zoned_traits<const time_zone*> {
static const time_zone* default_zone();
static const time_zone* locate_zone(string_view name);
};
}
```

```
static const time_zone* default_zone();
```

- 2 *Returns:* `std::chrono::locate_zone("UTC")`.

```
static const time_zone* locate_zone(string_view name);
```

- 3 *Returns:* `std::chrono::locate_zone(name)`.

### 30.11.7 Class template `zoned_time`

[time.zone.zonedtime]

#### 30.11.7.1 Overview

[time.zone.zonedtime.overview]

```
namespace std::chrono {
template<class Duration, class TimeZonePtr = const time_zone*>
class zoned_time {
public:
using duration = common_type_t<Duration, seconds>;

private:
TimeZonePtr zone_; // exposition only
sys_time<duration> tp_; // exposition only

using traits_ = zoned_traits<TimeZonePtr>; // exposition only

public:
zoned_time();
zoned_time(const zoned_time&) = default;
```

```

zoned_time& operator=(const zoned_time&) = default;

zoned_time(const sys_time<Duration>& st);
explicit zoned_time(TimeZonePtr z);
explicit zoned_time(string_view name);

template<class Duration2>
 zoned_time(const zoned_time<Duration2, TimeZonePtr>& y);

zoned_time(TimeZonePtr z, const sys_time<Duration>& st);
zoned_time(string_view name, const sys_time<Duration>& st);

zoned_time(TimeZonePtr z, const local_time<Duration>& tp);
zoned_time(string_view name, const local_time<Duration>& tp);
zoned_time(TimeZonePtr z, const local_time<Duration>& tp, choose c);
zoned_time(string_view name, const local_time<Duration>& tp, choose c);

template<class Duration2, class TimeZonePtr2>
 zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y);
template<class Duration2, class TimeZonePtr2>
 zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y, choose);

template<class Duration2, class TimeZonePtr2>
 zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y);
template<class Duration2, class TimeZonePtr2>
 zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y, choose c);

zoned_time& operator=(const sys_time<Duration>& st);
zoned_time& operator=(const local_time<Duration>& lt);

operator sys_time<duration>() const;
explicit operator local_time<duration>() const;

TimeZonePtr get_time_zone() const;
local_time<duration> get_local_time() const;
sys_time<duration> get_sys_time() const;
sys_info get_info() const;
};

zoned_time() -> zoned_time<seconds>;

template<class Duration>
 zoned_time(sys_time<Duration>)
 -> zoned_time<common_type_t<Duration, seconds>>;

template<class TimeZonePtrOrName>
 using time-zone-representation = // exposition only
 conditional_t<is_convertible_v<TimeZonePtrOrName, string_view>,
 const time_zone*,
 remove_cvref_t<TimeZonePtrOrName>>;

template<class TimeZonePtrOrName>
 zoned_time(TimeZonePtrOrName&&)
 -> zoned_time<seconds, time-zone-representation<TimeZonePtrOrName>>;

template<class TimeZonePtrOrName, class Duration>
 zoned_time(TimeZonePtrOrName&&, sys_time<Duration>)
 -> zoned_time<common_type_t<Duration, seconds>,
 time-zone-representation<TimeZonePtrOrName>>;

template<class TimeZonePtrOrName, class Duration>
 zoned_time(TimeZonePtrOrName&&, local_time<Duration>,
 choose = choose::earliest)
 -> zoned_time<common_type_t<Duration, seconds>,

```

```

 time-zone-representation<TimeZonePtrOrName>>;

 template<class Duration, class TimeZonePtrOrName, class TimeZonePtr2>
 zoned_time(TimeZonePtrOrName&&, zoned_time<Duration, TimeZonePtr2>,
 choose = choose::earliest)
 -> zoned_time<common_type_t<Duration, seconds>,
 time-zone-representation<TimeZonePtrOrName>>;
}

```

<sup>1</sup> `zoned_time` represents a logical pairing of a `time_zone` and a `time_point` with precision `Duration`. `zoned_time<Duration>` maintains the invariant that it always refers to a valid time zone and represents a point in time that exists and is not ambiguous in that time zone.

<sup>2</sup> If `Duration` is not a specialization of `chrono::duration`, the program is ill-formed.

<sup>3</sup> Every constructor of `zoned_time` that accepts a `string_view` as its first parameter does not participate in class template argument deduction (12.2.2.9).

### 30.11.7.2 Constructors

[time.zone.zonedtime.ctor]

```
zoned_time();
```

<sup>1</sup> *Constraints:* `traits_::default_zone()` is a well-formed expression.

<sup>2</sup> *Effects:* Initializes `zone_` with `traits_::default_zone()` and default constructs `tp_`.

```
zoned_time(const sys_time<Duration>& st);
```

<sup>3</sup> *Constraints:* `traits_::default_zone()` is a well-formed expression.

<sup>4</sup> *Effects:* Initializes `zone_` with `traits_::default_zone()` and `tp_` with `st`.

```
explicit zoned_time(TimeZonePtr z);
```

<sup>5</sup> *Preconditions:* `z` refers to a time zone.

<sup>6</sup> *Effects:* Initializes `zone_` with `std::move(z)` and default constructs `tp_`.

```
explicit zoned_time(string_view name);
```

<sup>7</sup> *Constraints:* `traits_::locate_zone(string_view{})` is a well-formed expression and `zoned_time` is constructible from the return type of `traits_::locate_zone(string_view{})`.

<sup>8</sup> *Effects:* Initializes `zone_` with `traits_::locate_zone(name)` and default constructs `tp_`.

```
template<class Duration2>
 zoned_time(const zoned_time<Duration2, TimeZonePtr>& y);
```

<sup>9</sup> *Constraints:* `is_convertible_v<sys_time<Duration2>, sys_time<Duration>>` is true.

<sup>10</sup> *Effects:* Initializes `zone_` with `y.zone_` and `tp_` with `y.tp_`.

```
zoned_time(TimeZonePtr z, const sys_time<Duration>& st);
```

<sup>11</sup> *Preconditions:* `z` refers to a time zone.

<sup>12</sup> *Effects:* Initializes `zone_` with `std::move(z)` and `tp_` with `st`.

```
zoned_time(string_view name, const sys_time<Duration>& st);
```

<sup>13</sup> *Constraints:* `zoned_time` is constructible from the return type of `traits_::locate_zone(name)` and `st`.

<sup>14</sup> *Effects:* Equivalent to construction with `{traits_::locate_zone(name), st}`.

```
zoned_time(TimeZonePtr z, const local_time<Duration>& tp);
```

<sup>15</sup> *Constraints:*

```

 is_convertible_v<
 decltype(declval<TimeZonePtr>&()->to_sys(local_time<Duration>{})),
 sys_time<duration>>
 is true.

```

<sup>16</sup> *Preconditions:* `z` refers to a time zone.

17 *Effects:* Initializes `zone_` with `std::move(z)` and `tp_` with `zone_->to_sys(tp)`.  
`zoned_time(string_view name, const local_time<Duration>& tp);`

18 *Constraints:* `zoned_time` is constructible from the return type of `traits_::locate_zone(name)` and `tp`.  
19 *Effects:* Equivalent to construction with `{traits_::locate_zone(name), tp}`.  
`zoned_time(TimeZonePtr z, const local_time<Duration>& tp, choose c);`

20 *Constraints:*  
`is_convertible_v<`  
`decltype(declval<TimeZonePtr>()->to_sys(local_time<Duration>{}, choose::earliest)),`  
`sys_time<duration>>`  
`is true.`

21 *Preconditions:* `z` refers to a time zone.  
22 *Effects:* Initializes `zone_` with `std::move(z)` and `tp_` with `zone_->to_sys(tp, c)`.  
`zoned_time(string_view name, const local_time<Duration>& tp, choose c);`

23 *Constraints:* `zoned_time` is constructible from the return type of `traits_::locate_zone(name)`, `local_time<Duration>`, and `choose`.  
24 *Effects:* Equivalent to construction with `{traits_::locate_zone(name), tp, c}`.  
`template<class Duration2, class TimeZonePtr2>`  
`zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y);`

25 *Constraints:* `is_convertible_v<sys_time<Duration2>, sys_time<Duration>>` is true.  
26 *Preconditions:* `z` refers to a valid time zone.  
27 *Effects:* Initializes `zone_` with `std::move(z)` and `tp_` with `y.tp_`.  
`template<class Duration2, class TimeZonePtr2>`  
`zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y, choose);`

28 *Constraints:* `is_convertible_v<sys_time<Duration2>, sys_time<Duration>>` is true.  
29 *Preconditions:* `z` refers to a valid time zone.  
30 *Effects:* Equivalent to construction with `{z, y}`.  
31 [Note 1: The `choose` parameter has no effect. — end note]

`template<class Duration2, class TimeZonePtr2>`  
`zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y);`

32 *Constraints:* `zoned_time` is constructible from the return type of `traits_::locate_zone(name)` and the type `zoned_time<Duration2, TimeZonePtr2>`.  
33 *Effects:* Equivalent to construction with `{traits_::locate_zone(name), y}`.  
`template<class Duration2, class TimeZonePtr2>`  
`zoned_time(string_view name, const zoned_time<Duration2, TimeZonePtr2>& y, choose c);`

34 *Constraints:* `zoned_time` is constructible from the return type of `traits_::locate_zone(name)`, the type `zoned_time<Duration2, TimeZonePtr2>`, and the type `choose`.  
35 *Effects:* Equivalent to construction with `{traits_::locate_zone(name), y, c}`.  
36 [Note 2: The `choose` parameter has no effect. — end note]

### 30.11.7.3 Member functions

[time.zone.zonedtime.members]

`zoned_time& operator=(const sys_time<Duration>& st);`

1 *Effects:* After assignment, `get_sys_time() == st`. This assignment has no effect on the return value of `get_time_zone()`.

2 *Returns:* `*this`.

```
zoned_time& operator=(const local_time<Duration>& lt);
```

3     *Effects:* After assignment, `get_local_time()` == `lt`. This assignment has no effect on the return value of `get_time_zone()`.

4     *Returns:* `*this`.

```
operator sys_time<duration>() const;
```

5     *Returns:* `get_sys_time()`.

```
explicit operator local_time<duration>() const;
```

6     *Returns:* `get_local_time()`.

```
TimeZonePtr get_time_zone() const;
```

7     *Returns:* `zone_`.

```
local_time<duration> get_local_time() const;
```

8     *Returns:* `zone_->to_local(tp_)`.

```
sys_time<duration> get_sys_time() const;
```

9     *Returns:* `tp_`.

```
sys_info get_info() const;
```

10    *Returns:* `zone_->get_info(tp_)`.

#### 30.11.7.4 Non-member functions

[time.zone.zonedtime.nonmembers]

```
template<class Duration1, class Duration2, class TimeZonePtr>
bool operator==(const zoned_time<Duration1, TimeZonePtr>& x,
 const zoned_time<Duration2, TimeZonePtr>& y);
```

1     *Returns:* `x.zone_ == y.zone_ && x.tp_ == y.tp_`.

```
template<class charT, class traits, class Duration, class TimeZonePtr>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
 const zoned_time<Duration, TimeZonePtr>& t);
```

2     *Effects:* Equivalent to:

```
return os << format(os.getloc(), STATICALLY-WIDEN<charT>("{:L%F %T %Z}"), t);
```

#### 30.11.8 Class leap\_second

[time.zone.leap]

##### 30.11.8.1 Overview

[time.zone.leap.overview]

```
namespace std::chrono {
 class leap_second {
 public:
 leap_second(const leap_second&) = default;
 leap_second& operator=(const leap_second&) = default;

 // unspecified additional constructors

 constexpr sys_seconds date() const noexcept;
 constexpr seconds value() const noexcept;
 };
}
```

1    Objects of type `leap_second` representing the date and value of the leap second insertions are constructed and stored in the time zone database when initialized.

2    [Example 1:

```
for (auto& l : get_tzdb().leap_seconds)
 if (l <= sys_days{2018y/March/17d})
 cout << l.date() << ": " << l.value() << '\n';
```



Produces the output:

```
1972-07-01 00:00:00: 1s
1973-01-01 00:00:00: 1s
1974-01-01 00:00:00: 1s
1975-01-01 00:00:00: 1s
1976-01-01 00:00:00: 1s
1977-01-01 00:00:00: 1s
1978-01-01 00:00:00: 1s
1979-01-01 00:00:00: 1s
1980-01-01 00:00:00: 1s
1981-07-01 00:00:00: 1s
1982-07-01 00:00:00: 1s
1983-07-01 00:00:00: 1s
1985-07-01 00:00:00: 1s
1988-01-01 00:00:00: 1s
1990-01-01 00:00:00: 1s
1991-01-01 00:00:00: 1s
1992-07-01 00:00:00: 1s
1993-07-01 00:00:00: 1s
1994-07-01 00:00:00: 1s
1996-01-01 00:00:00: 1s
1997-07-01 00:00:00: 1s
1999-01-01 00:00:00: 1s
2006-01-01 00:00:00: 1s
2009-01-01 00:00:00: 1s
2012-07-01 00:00:00: 1s
2015-07-01 00:00:00: 1s
2017-01-01 00:00:00: 1s
```

— *end example*]

### 30.11.8.2 Member functions

[time.zone.leap.members]

```
constexpr sys_seconds date() const noexcept;
```

1     *Returns:* The date and time at which the leap second was inserted.

```
constexpr seconds value() const noexcept;
```

2     *Returns:* +1s to indicate a positive leap second or -1s to indicate a negative leap second.

[*Note 1:* All leap seconds inserted up through 2022 were positive leap seconds. — *end note*]

### 30.11.8.3 Non-member functions

[time.zone.leap.nonmembers]

```
constexpr bool operator==(const leap_second& x, const leap_second& y) noexcept;
```

1     *Returns:* x.date() == y.date().

```
constexpr strong_ordering operator<=>(const leap_second& x, const leap_second& y) noexcept;
```

2     *Returns:* x.date() <=> y.date().

```
template<class Duration>
constexpr bool operator==(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

3     *Returns:* x.date() == y.

```
template<class Duration>
constexpr bool operator<(const leap_second& x, const sys_time<Duration>& y) noexcept;
```

4     *Returns:* x.date() < y.

```
template<class Duration>
constexpr bool operator<(const sys_time<Duration>& x, const leap_second& y) noexcept;
```

5     *Returns:* x < y.date().

```

template<class Duration>
constexpr bool operator>(const leap_second& x, const sys_time<Duration>& y) noexcept;
6 Returns: y < x.

template<class Duration>
constexpr bool operator>(const sys_time<Duration>& x, const leap_second& y) noexcept;
7 Returns: y < x.

template<class Duration>
constexpr bool operator<=(const leap_second& x, const sys_time<Duration>& y) noexcept;
8 Returns: !(y < x).

template<class Duration>
constexpr bool operator<=(const sys_time<Duration>& x, const leap_second& y) noexcept;
9 Returns: !(y < x).

template<class Duration>
constexpr bool operator>=(const leap_second& x, const sys_time<Duration>& y) noexcept;
10 Returns: !(x < y).

template<class Duration>
constexpr bool operator>=(const sys_time<Duration>& x, const leap_second& y) noexcept;
11 Returns: !(x < y).

template<class Duration>
requires three_way_comparable_with<sys_seconds, sys_time<Duration>>
constexpr auto operator<=>(const leap_second& x, const sys_time<Duration>& y) noexcept;
12 Returns: x.date() <=> y.

```

### 30.11.9 Class `time_zone_link`

[time.zone.link]

#### 30.11.9.1 Overview

[time.zone.link.overview]

```

namespace std::chrono {
 class time_zone_link {
 public:
 time_zone_link(time_zone_link&&) = default;
 time_zone_link& operator=(time_zone_link&&) = default;

 // unspecified additional constructors

 string_view name() const noexcept;
 string_view target() const noexcept;
 };
}

```

- <sup>1</sup> A `time_zone_link` specifies an alternative name for a `time_zone`. `time_zone_links` are constructed when the time zone database is initialized.

#### 30.11.9.2 Member functions

[time.zone.link.members]

```
string_view name() const noexcept;
```

- <sup>1</sup> Returns: The alternative name for the time zone.

```
string_view target() const noexcept;
```

- <sup>2</sup> Returns: The name of the `time_zone` for which this `time_zone_link` provides an alternative name.

#### 30.11.9.3 Non-member functions

[time.zone.link.nonmembers]

```
bool operator==(const time_zone_link& x, const time_zone_link& y) noexcept;
```

- <sup>1</sup> Returns: `x.name() == y.name()`.

```
strong_ordering operator<=>(const time_zone_link& x, const time_zone_link& y) noexcept;
```

<sup>2</sup> Returns: `x.name() <=> y.name()`.

## 30.12 Formatting

[time.format]

- <sup>1</sup> Each `formatter` (28.5.6) specialization in the chrono library (30.2) meets the *Formatter* requirements (28.5.6.1). The `parse` member functions of these formatters interpret the format specification as a *chrono-format-spec* according to the following syntax:

```
chrono-format-spec:
 fill-and-alignopt widthopt precisionopt Lopt chrono-specsopt

chrono-specs:
 conversion-spec
 chrono-specs conversion-spec
 chrono-specs literal-char

literal-char:
 any character other than {, }, or %

conversion-spec:
 % modifieropt type

modifier: one of
 E O

type: one of
 a A b B c C d D e F g G h H I j m M n
 p q Q r R S t T u U V w W x X y Y z Z %
```

The productions *fill-and-align*, *width*, and *precision* are described in 28.5.2. Giving a *precision* specification in the *chrono-format-spec* is valid only for types that are specializations of `std::chrono::duration` for which the nested *typedef-name* `rep` denotes a floating-point type. For all other types, an exception of type `format_error` is thrown if the *chrono-format-spec* contains a *precision* specification. All ordinary multibyte characters represented by *literal-char* are copied unchanged to the output.

- <sup>2</sup> A *formatting locale* is an instance of `locale` used by a formatting function, defined as

- (2.1) — the "C" locale if the L option is not present in *chrono-format-spec*, otherwise
- (2.2) — the locale passed to the formatting function if any, otherwise
- (2.3) — the global locale.

- <sup>3</sup> Each conversion specifier *conversion-spec* is replaced by appropriate characters as described in Table 133; the formats specified in ISO 8601-1:2019 shall be used where so described. Some of the conversion specifiers depend on the formatting locale. If the string literal encoding is a Unicode encoding form and the locale is among an implementation-defined set of locales, each replacement that depends on the locale is performed as if the replacement character sequence is converted to the string literal encoding. If the formatted object does not contain the information the conversion specifier refers to, an exception of type `format_error` is thrown.

- <sup>4</sup> The result of formatting a `std::chrono::duration` instance holding a negative value, or an `hh_mm_ss` object `h` for which `h.is_negative()` is `true`, is equivalent to the output of the corresponding positive value, with a *STATICALLY-WIDEN*`<charT>("-")` character sequence placed before the replacement of the initial conversion specifier.

[Example 1:

```
cout << format("{:%T}", -10'000s); // prints: -02:46:40
cout << format("{:%H:%M:%S}", -10'000s); // prints: -02:46:40
cout << format("minutes {:%M, hours %H, seconds %S}", -10'000s);
 // prints: minutes -46, hours 02, seconds 40
```

— end example]

- <sup>5</sup> Unless explicitly requested, the result of formatting a chrono type does not contain time zone abbreviation and time zone offset information. If the information is available, the conversion specifiers `%Z` and `%z` will format this information (respectively).

[Note 1: If the information is not available and a `%Z` or `%z` conversion specifier appears in the *chrono-format-spec*, an exception of type `format_error` is thrown, as described above. — end note]

- <sup>6</sup> If the type being formatted does not contain the information that the format flag needs, an exception of type `format_error` is thrown.

[*Example 2:* A `duration` does not contain enough information to format as a `weekday`. — *end example*]

However, if a flag refers to a “time of day” (e.g., `%H`, `%I`, `%p`, etc.), then a specialization of `duration` is interpreted as the time of day elapsed since midnight.

**Table 133 — Meaning of conversion specifiers** [tab:time.format.spec]

| Specifier       | Replacement                                                                                                                                                                                                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>%a</code> | The locale’s abbreviated weekday name. If the value does not contain a valid weekday, an exception of type <code>format_error</code> is thrown.                                                                                                                                              |
| <code>%A</code> | The locale’s full weekday name. If the value does not contain a valid weekday, an exception of type <code>format_error</code> is thrown.                                                                                                                                                     |
| <code>%b</code> | The locale’s abbreviated month name. If the value does not contain a valid month, an exception of type <code>format_error</code> is thrown.                                                                                                                                                  |
| <code>%B</code> | The locale’s full month name. If the value does not contain a valid month, an exception of type <code>format_error</code> is thrown.                                                                                                                                                         |
| <code>%c</code> | The locale’s date and time representation. The modified command <code>%Ec</code> produces the locale’s alternate date and time representation.                                                                                                                                               |
| <code>%C</code> | The year divided by 100 using floored division. If the result is a single decimal digit, it is prefixed with 0. The modified command <code>%EC</code> produces the locale’s alternative representation of the century.                                                                       |
| <code>%d</code> | The day of month as a decimal number. If the result is a single decimal digit, it is prefixed with 0. The modified command <code>%Od</code> produces the locale’s alternative representation.                                                                                                |
| <code>%D</code> | Equivalent to <code>%m/%d/%y</code> .                                                                                                                                                                                                                                                        |
| <code>%e</code> | The day of month as a decimal number. If the result is a single decimal digit, it is prefixed with a space. The modified command <code>%Oe</code> produces the locale’s alternative representation.                                                                                          |
| <code>%F</code> | Equivalent to <code>%Y-%m-%d</code> .                                                                                                                                                                                                                                                        |
| <code>%g</code> | The last two decimal digits of the calendar year as specified in ISO 8601-1:2019 for the week calendar. If the result is a single digit it is prefixed by 0.                                                                                                                                 |
| <code>%G</code> | The calendar year as a decimal number, as specified in ISO 8601-1:2019 for the week calendar. If the result is less than four digits it is left-padded with 0 to four digits.                                                                                                                |
| <code>%h</code> | Equivalent to <code>%b</code> .                                                                                                                                                                                                                                                              |
| <code>%H</code> | The hour (24-hour clock) as a decimal number. If the result is a single digit, it is prefixed with 0. The modified command <code>%OH</code> produces the locale’s alternative representation.                                                                                                |
| <code>%I</code> | The hour (12-hour clock) as a decimal number. If the result is a single digit, it is prefixed with 0. The modified command <code>%OI</code> produces the locale’s alternative representation.                                                                                                |
| <code>%j</code> | If the type being formatted is a specialization of <code>duration</code> , the decimal number of <code>days</code> without padding. Otherwise, the day of the year as a decimal number. January 1 is 001. If the result is less than three digits, it is left-padded with 0 to three digits. |
| <code>%m</code> | The month as a decimal number. Jan is 01. If the result is a single digit, it is prefixed with 0. The modified command <code>%Om</code> produces the locale’s alternative representation.                                                                                                    |
| <code>%M</code> | The minute as a decimal number. If the result is a single digit, it is prefixed with 0. The modified command <code>%OM</code> produces the locale’s alternative representation.                                                                                                              |
| <code>%n</code> | A new-line character.                                                                                                                                                                                                                                                                        |
| <code>%p</code> | The locale’s equivalent of the AM/PM designations associated with a 12-hour clock.                                                                                                                                                                                                           |
| <code>%q</code> | The duration’s unit suffix as specified in 30.5.11.                                                                                                                                                                                                                                          |
| <code>%Q</code> | The duration’s numeric value (as if extracted via <code>.count()</code> ).                                                                                                                                                                                                                   |
| <code>%r</code> | The locale’s 12-hour clock time.                                                                                                                                                                                                                                                             |
| <code>%R</code> | Equivalent to <code>%H:%M</code> .                                                                                                                                                                                                                                                           |

Table 133 — Meaning of conversion specifiers (continued)

| Specifier | Replacement                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %S        | Seconds as a decimal number. If the number of seconds is less than 10, the result is prefixed with 0. If the precision of the input cannot be exactly represented with seconds, then the format is a decimal floating-point number with a fixed format and a precision matching that of the precision of the input (or to a microseconds precision if the conversion to floating-point decimal seconds cannot be made within 18 fractional digits). The character for the decimal point is localized according to the locale. The modified command %OS produces the locale's alternative representation. |
| %t        | A horizontal-tab character.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| %T        | Equivalent to %H:%M:%S.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| %u        | The calendar day of week as a decimal number (1-7), as specified in ISO 8601-1:2019, where Monday is 1. The modified command %Ou produces the locale's alternative representation.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| %U        | The week number of the year as a decimal number. The first Sunday of the year is the first day of week 01. Days of the same year prior to that are in week 00. If the result is a single digit, it is prefixed with 0. The modified command %OU produces the locale's alternative representation.                                                                                                                                                                                                                                                                                                        |
| %V        | The calendar week of year as a decimal number, as specified in ISO 8601-1:2019 for the week calendar. If the result is a single digit, it is prefixed with 0. The modified command %OV produces the locale's alternative representation.                                                                                                                                                                                                                                                                                                                                                                 |
| %w        | The weekday as a decimal number (0-6), where Sunday is 0. The modified command %Ow produces the locale's alternative representation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| %W        | The week number of the year as a decimal number. The first Monday of the year is the first day of week 01. Days of the same year prior to that are in week 00. If the result is a single digit, it is prefixed with 0. The modified command %OW produces the locale's alternative representation.                                                                                                                                                                                                                                                                                                        |
| %x        | The locale's date representation. The modified command %Ex produces the locale's alternate date representation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| %X        | The locale's time representation. The modified command %EX produces the locale's alternate time representation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| %y        | The last two decimal digits of the year, regardless of the sign of the year. If the result is a single digit, it is prefixed by 0. The modified command %Oy produces the locale's alternative representation. The modified command %Ey produces the locale's alternative representation of offset from %EC (year only).<br>[Example 3:<br>cout << format("{:%C %y}", -1976y);      // prints -20 76<br>— end example]                                                                                                                                                                                    |
| %Y        | The year as a decimal number. If the result is less than four digits, it is left-padded with 0 to four digits. The modified command %EY produces the locale's alternative full year representation.                                                                                                                                                                                                                                                                                                                                                                                                      |
| %z        | The offset from UTC as specified in ISO 8601-1:2019, 5.3.4.1. For example -0430 refers to 4 hours 30 minutes behind UTC. If the offset is zero, +0000 is used. The modified commands %Ez and %Oz insert a : between the hours and minutes: -04:30. If the offset information is not available, an exception of type <code>format_error</code> is thrown.                                                                                                                                                                                                                                                 |
| %Z        | The time zone abbreviation. If the time zone abbreviation is not available, an exception of type <code>format_error</code> is thrown.                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| %%        | A % character.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

- <sup>7</sup> Unless otherwise specified, if the *chrono-specs* is omitted, the chrono object is formatted as if by streaming it to `basic_ostream<charT> os` with the formatting locale imbued and copying `os.str()` through the output iterator of the context with additional padding and adjustments as specified by the format specifiers.

[Example 4:

```
string s = format("{:=>8}", 42ms); // value of s is "====42ms"
— end example]
```

- 8 For `chrono::duration` the library only provides the following specialization of `enable_nonlocking_formatter_optimization`:

```
template<class Rep, class Period>
constexpr bool enable_nonlocking_formatter_optimization<
 chrono::duration<Rep, Period>> =
 enable_nonlocking_formatter_optimization<Rep>;
```

- 9 For `chrono::zoned_time` the library only provides the following specialization of `enable_nonlocking_formatter_optimization`:

```
template<class Duration>
constexpr bool enable_nonlocking_formatter_optimization<
 chrono::zoned_time<Duration, const std::chrono::time_zone*>> = true;
```

```
template<class Duration, class charT>
struct formatter<chrono::sys_time<Duration>, charT>;
```

- 10 *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("UTC")`. If `%z` (or a modified variant of `%z`) is used, an offset of 0min is formatted.

```
template<class Duration, class charT>
struct formatter<chrono::utc_time<Duration>, charT>;
```

- 11 *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("UTC")`. If `%z` (or a modified variant of `%z`) is used, an offset of 0min is formatted. If the argument represents a time during a positive leap second insertion, and if a seconds field is formatted, the integral portion of that format is `STATICALLY-WIDEN<charT>("60")`.

```
template<class Duration, class charT>
struct formatter<chrono::tai_time<Duration>, charT>;
```

- 12 *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("TAI")`. If `%z` (or a modified variant of `%z`) is used, an offset of 0min is formatted. The date and time formatted are equivalent to those formatted by a `sys_time` initialized with

```
sys_time<Duration>{tp.time_since_epoch()} -
 (sys_days{1970y/January/1} - sys_days{1958y/January/1})
```

```
template<class Duration, class charT>
struct formatter<chrono::gps_time<Duration>, charT>;
```

- 13 *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("GPS")`. If `%z` (or a modified variant of `%z`) is used, an offset of 0min is formatted. The date and time formatted are equivalent to those formatted by a `sys_time` initialized with

```
sys_time<Duration>{tp.time_since_epoch()} +
 (sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1})
```

```
template<class Duration, class charT>
struct formatter<chrono::file_time<Duration>, charT>;
```

- 14 *Remarks:* If `%Z` is used, it is replaced with `STATICALLY-WIDEN<charT>("UTC")`. If `%z` (or a modified variant of `%z`) is used, an offset of 0min is formatted. The date and time formatted are equivalent to those formatted by a `sys_time` initialized with `clock_cast<system_clock>(t)`, or by a `utc_time` initialized with `clock_cast<utc_clock>(t)`, where `t` is the first argument to format.

```
template<class Duration, class charT>
struct formatter<chrono::local_time<Duration>, charT>;
```

- 15 *Remarks:* If `%Z`, `%z`, or a modified version of `%z` is used, an exception of type `format_error` is thrown.

```
template<class Duration> struct local-time-format-t {
 local_time<Duration> time_; // exposition only
 const string* abbrev_; // exposition only
 const seconds* offset-sec; // exposition only
};
```

```

template<class Duration>
 local-time-format-t<Duration>
 local_time_format(local_time<Duration> time, const string* abbrev = nullptr,
 const seconds* offset_sec = nullptr);
16 Returns: {time, abbrev, offset_sec}.

template<class Duration, class charT>
 struct formatter<chrono::local-time-format-t<Duration>, charT>;
17 Let f be a local-time-format-t<Duration> object passed to formatter::format.
18 Remarks: If the chrono-specs is omitted, the result is equivalent to using %F %T %Z as the chrono-specs.
 If %Z is used, it is replaced with *f.abbrev_ if f.abbrev_ is not a null pointer value. If %Z is used and
 f.abbrev_ is a null pointer value, an exception of type format_error is thrown. If %z (or a modified
 variant of %z) is used, it is formatted with the value of *f.offset-sec if f.offset-sec is not a null
 pointer value. If %z (or a modified variant of %z) is used and f.offset-sec is a null pointer value,
 then an exception of type format_error is thrown.

template<class Duration, class TimeZonePtr, class charT>
 struct formatter<chrono::zoned_time<Duration, TimeZonePtr>, charT>
 {
 : formatter<chrono::local-time-format-t<common_type_t<Duration, seconds>>, charT> {
 template<class FormatContext>
 typename FormatContext::iterator
 format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
 };

template<class FormatContext>
 typename FormatContext::iterator
 format(const chrono::zoned_time<Duration, TimeZonePtr>& tp, FormatContext& ctx) const;
19 Effects: Equivalent to:

 sys_info info = tp.get_info();
 return formatter<chrono::local-time-format-t<common_type_t<Duration, seconds>>, charT>::
 format({tp.get_local_time(), &info.abbrev, &info.offset}, ctx);

```

### 30.13 Parsing

[time.parse]

- 1 Each **parse** overload specified in this subclause calls **from\_stream** unqualified, so as to enable argument-dependent lookup (6.5.4). In the following paragraphs, let *is* denote an object of type **basic\_istream**<charT, traits> and let *I* be **basic\_istream**<charT, traits>&, where *charT* and *traits* are template parameters in that context.
- 2 *Recommended practice*: Implementations should make it difficult to accidentally store or use a manipulator that may contain a dangling reference to a format string, for example by making the manipulators produced by **parse** immovable and preventing stream extraction into an lvalue of such a manipulator type.

```

template<class charT, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp);
template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);

```

- 3 Let *F* be *fmt* for the first overload and *fmt.c\_str*() for the second overload. Let *traits* be **char\_traits**<charT> for the first overload.

- 4 *Constraints*: The expression

```
from_stream(declval<basic_istream<charT, traits>&>(), F, tp)
```

is well-formed when treated as an unevaluated operand (7.2.3).

- 5 *Returns*: A manipulator such that the expression `is >> parse(fmt, tp)` has type *I*, has value *is*, and calls **from\_stream**(*is*, *F*, *tp*).

```

template<class charT, class traits, class Alloc, class Parsable>
 unspecified
 parse(const charT* fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev);

```

```
template<class charT, class traits, class Alloc, class Parsable>
 unspecified
```

```
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev);
```

6 Let  $F$  be `fmt` for the first overload and `fmt.c_str()` for the second overload.

7 *Constraints:* The expression

```
 from_stream(declval<basic_istream<charT, traits>&>(), F , tp, addressof(abbrev))
```

is well-formed when treated as an unevaluated operand (7.2.3).

8 *Returns:* A manipulator such that the expression `>> parse(fmt, tp, abbrev)` has type `I`, has value `is`, and calls `from_stream(is,  $F$ , tp, addressof(abbrev))`.

```
template<class charT, class Parsable>
```

```
 unspecified
```

```
 parse(const charT* fmt, Parsable& tp, minutes& offset);
```

```
template<class charT, class traits, class Alloc, class Parsable>
```

```
 unspecified
```

```
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 minutes& offset);
```

9 Let  $F$  be `fmt` for the first overload and `fmt.c_str()` for the second overload. Let `traits` be `char_traits<charT>` and `Alloc` be `allocator<charT>` for the first overload.

10 *Constraints:* The expression

```
 from_stream(declval<basic_istream<charT, traits>&>(),
 F , tp,
 declval<basic_string<charT, traits, Alloc>*>(),
 &offset)
```

is well-formed when treated as an unevaluated operand (7.2.3).

11 *Returns:* A manipulator such that the expression `>> parse(fmt, tp, offset)` has type `I`, has value `is`, and calls:

```
 from_stream(is,
 F , tp,
 static_cast<basic_string<charT, traits, Alloc>*>(nullptr),
 &offset)
```

```
template<class charT, class traits, class Alloc, class Parsable>
```

```
 unspecified
```

```
 parse(const charT* fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

```
template<class charT, class traits, class Alloc, class Parsable>
```

```
 unspecified
```

```
 parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
 basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

12 Let  $F$  be `fmt` for the first overload and `fmt.c_str()` for the second overload.

13 *Constraints:* The expression

```
 from_stream(declval<basic_istream<charT, traits>&>(),
 F , tp, addressof(abbrev), &offset)
```

is well-formed when treated as an unevaluated operand (7.2.3).

14 *Returns:* A manipulator such that the expression `>> parse(fmt, tp, abbrev, offset)` has type `I`, has value `is`, and calls `from_stream(is,  $F$ , tp, addressof(abbrev), &offset)`.

15 All `from_stream` overloads behave as unformatted input functions, except that they have an unspecified effect on the value returned by subsequent calls to `basic_istream<>::gcount()`. Each overload takes a format string containing ordinary characters and flags which have special meaning. Each flag begins with a %. Some flags can be modified by `E` or `O`. During parsing each flag interprets characters as parts of date and time types according to Table 134. Some flags can be modified by a width parameter given as a positive decimal integer called out as  $N$  below which governs how many characters are parsed from the stream in interpreting the flag. All characters in the format string that are not represented in Table 134, except for



whitespace, are parsed unchanged from the stream. A whitespace character matches zero or more whitespace characters in the input stream.

- <sup>16</sup> If the type being parsed cannot represent the information that the format flag refers to, `is.setstate(ios_base::failbit)` is called.

[*Example 1*: A `duration` cannot represent a `weekday`. — *end example*]

However, if a flag refers to a “time of day” (e.g., `%H`, `%I`, `%p`, etc.), then a specialization of `duration` is parsed as the time of day elapsed since midnight.

- <sup>17</sup> If the `from_stream` overload fails to parse everything specified by the format string, or if insufficient information is parsed to specify a complete duration, time point, or calendrical data structure, `setstate(ios_base::failbit)` is called on the `basic_istream`.

**Table 134 — Meaning of parse flags** [tab:time.parse.spec]

| Flag            | Parsed value                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>%a</code> | The locale’s full or abbreviated case-insensitive weekday name.                                                                                                                                                                                                                                                                                                                              |
| <code>%A</code> | Equivalent to <code>%a</code> .                                                                                                                                                                                                                                                                                                                                                              |
| <code>%b</code> | The locale’s full or abbreviated case-insensitive month name.                                                                                                                                                                                                                                                                                                                                |
| <code>%B</code> | Equivalent to <code>%b</code> .                                                                                                                                                                                                                                                                                                                                                              |
| <code>%c</code> | The locale’s date and time representation. The modified command <code>%Ec</code> interprets the locale’s alternate date and time representation.                                                                                                                                                                                                                                             |
| <code>%C</code> | The century as a decimal number. The modified command <code>%NC</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%EC</code> interprets the locale’s alternative representation of the century.                                                           |
| <code>%d</code> | The day of the month as a decimal number. The modified command <code>%Nd</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%Od</code> interprets the locale’s alternative representation of the day of the month.                                         |
| <code>%D</code> | Equivalent to <code>%m/%d/%y</code> .                                                                                                                                                                                                                                                                                                                                                        |
| <code>%e</code> | Equivalent to <code>%d</code> and can be modified like <code>%d</code> .                                                                                                                                                                                                                                                                                                                     |
| <code>%F</code> | Equivalent to <code>%Y-%m-%d</code> . If modified with a width <i>N</i> , the width is applied to only <code>%Y</code> .                                                                                                                                                                                                                                                                     |
| <code>%g</code> | The last two decimal digits of the calendar year, as specified in ISO 8601-1:2019 for the week calendar. The modified command <code>%Ng</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required.                                                                                            |
| <code>%G</code> | The calendar year as a decimal number, as specified in ISO 8601-1:2019 for the week calendar. The modified command <code>%NG</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 4. Leading zeroes are permitted but not required.                                                                                                       |
| <code>%h</code> | Equivalent to <code>%b</code> .                                                                                                                                                                                                                                                                                                                                                              |
| <code>%H</code> | The hour (24-hour clock) as a decimal number. The modified command <code>%NH</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%OH</code> interprets the locale’s alternative representation.                                                             |
| <code>%I</code> | The hour (12-hour clock) as a decimal number. The modified command <code>%NI</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%OI</code> interprets the locale’s alternative representation.                                                             |
| <code>%j</code> | If the type being parsed is a specialization of <code>duration</code> , a decimal number of <code>days</code> . Otherwise, the day of the year as a decimal number. January 1 is 1. In either case, the modified command <code>%Nj</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 3. Leading zeroes are permitted but not required. |
| <code>%m</code> | The month as a decimal number. January is 1. The modified command <code>%Nm</code> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command <code>%Om</code> interprets the locale’s alternative representation.                                                              |

Table 134 — Meaning of parse flags (continued)

| Flag | Parsed value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| %M   | The minutes as a decimal number. The modified command %NM specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OM interprets the locale's alternative representation.                                                                                                                                                                                                                                                                              |
| %n   | Matches one whitespace character.<br>[Note 1: %n, %t, and a space can be combined to match a wide range of whitespace patterns. For example, "%n " matches one or more whitespace characters, and "%n%t%t" matches one to three whitespace characters. — end note]                                                                                                                                                                                                                                                                                                     |
| %p   | The locale's equivalent of the AM/PM designations associated with a 12-hour clock.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| %r   | The locale's 12-hour clock time.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| %R   | Equivalent to %H:%M.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| %S   | The seconds as a decimal number. The modified command %NS specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2 if the input time has a precision convertible to seconds. Otherwise the default width is determined by the decimal precision of the input and the field is interpreted as a long double in a fixed format. If encountered, the locale determines the decimal point character. Leading zeroes are permitted but not required. The modified command %OS interprets the locale's alternative representation. |
| %t   | Matches zero or one whitespace characters.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| %T   | Equivalent to %H:%M:%S.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| %u   | The calendar day of week as a decimal number (1-7), as specified in ISO 8601-1:2019, where Monday is 1. The modified command %Nu specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 1. Leading zeroes are permitted but not required.                                                                                                                                                                                                                                                                                    |
| %U   | The week number of the year as a decimal number. The first Sunday of the year is the first day of week 01. Days of the same year prior to that are in week 00. The modified command %NU specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OU interprets the locale's alternative representation.                                                                                                                                                |
| %V   | The calendar week of year as a decimal number, as specified in ISO 8601-1:2019 for the week calendar. The modified command %NV specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required.                                                                                                                                                                                                                                                                                      |
| %w   | The weekday as a decimal number (0-6), where Sunday is 0. The modified command %Nw specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 1. Leading zeroes are permitted but not required. The modified command %Ow interprets the locale's alternative representation.                                                                                                                                                                                                                                                     |
| %W   | The week number of the year as a decimal number. The first Monday of the year is the first day of week 01. Days of the same year prior to that are in week 00. The modified command %NW specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command %OW interprets the locale's alternative representation.                                                                                                                                                |
| %x   | The locale's date representation. The modified command %Ex interprets the locale's alternate date representation.                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| %X   | The locale's time representation. The modified command %EX interprets the locale's alternate time representation.                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

Table 134 — Meaning of parse flags (continued)

| Flag      | Parsed value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>%y</b> | The last two decimal digits of the year, regardless of the sign of the year. If the century is not otherwise specified (e.g., with <b>%C</b> ), values in the range [69, 99] are presumed to refer to the years 1969 to 1999, and values in the range [00, 68] are presumed to refer to the years 2000 to 2068. The modified command <b>%Ny</b> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands <b>%Ey</b> and <b>%Oy</b> interpret the locale's alternative representation.<br>[Example 2:<br>year y;<br>istringstream{"-20 76"} >> parse("%3C %y", y); // y == -1976y is true<br>— end example] |
| <b>%Y</b> | The year as a decimal number. The modified command <b>%NY</b> specifies the maximum number of characters to read. If <i>N</i> is not specified, the default is 4. Leading zeroes are permitted but not required. The modified command <b>%EY</b> interprets the locale's alternative representation.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>%z</b> | The offset from UTC in the format [+ -]hh[mm]. For example -0430 refers to 4 hours 30 minutes behind UTC, and 04 refers to 4 hours ahead of UTC. The modified commands <b>%Ez</b> and <b>%Oz</b> parse a : between the hours and minutes and render leading zeroes on the hour field optional: [+ -]h[h][:mm]. For example -04:30 refers to 4 hours 30 minutes behind UTC, and 4 refers to 4 hours ahead of UTC.                                                                                                                                                                                                                                                                                                           |
| <b>%Z</b> | The time zone abbreviation or name. A single word is parsed. This word can only contain characters from the basic character set (5.3.1) that are alphanumeric, or one of '_', '/', '-', or '+'.<br>'_', '/', '-', or '+'.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>%%</b> | A % character is extracted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

## 30.14 Hash support

[time.hash]

```
template<class Rep, class Period> struct hash<chrono::duration<Rep, Period>>;
```

- 1 The specialization `hash<chrono::duration<Rep, Period>>` is enabled (22.10.19) if and only if `hash<Rep>` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<class Clock, class Duration> struct hash<chrono::time_point<Clock, Duration>>;
```

- 2 The specialization `hash<chrono::time_point<Clock, Duration>>` is enabled (22.10.19) if and only if `hash<Duration>` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::day>;
template<> struct hash<chrono::month>;
template<> struct hash<chrono::year>;
template<> struct hash<chrono::weekday>;
template<> struct hash<chrono::weekday_indexed>;
template<> struct hash<chrono::weekday_last>;
template<> struct hash<chrono::month_day>;
template<> struct hash<chrono::month_day_last>;
template<> struct hash<chrono::month_weekday>;
template<> struct hash<chrono::month_weekday_last>;
template<> struct hash<chrono::year_month>;
template<> struct hash<chrono::year_month_day>;
template<> struct hash<chrono::year_month_day_last>;
template<> struct hash<chrono::year_month_weekday>;
template<> struct hash<chrono::year_month_weekday_last>;
```

- 3 The specializations are enabled (22.10.19).

[Note 1: All the `hash<Key>` specializations listed above meet the *Cpp17Hash* requirements, even when called on objects `k` of type `Key` such that `k.ok()` is `false`. — end note]

```
template<class Duration, class TimeZonePtr>
 struct hash<chrono::zoned_time<Duration, TimeZonePtr>>;
```

- 4 The specialization `hash<chrono::zoned_time<Duration, TimeZonePtr>>` is enabled (22.10.19) if and only if `hash<Duration>` is enabled and `hash<TimeZonePtr>` is enabled. The member functions are not guaranteed to be `noexcept`.

```
template<> struct hash<chrono::leap_second>;
```

- 5 The specialization is enabled (22.10.19).

### 30.15 Header `<ctime>` synopsis

[ctime.syn]

```
#define __STDC_VERSION__ 202311L

#define NULL see 17.2.3
#define CLOCKS_PER_SEC see below
#define TIME_UTC see below
#define TIME_MONOTONIC see below // optional
#define TIME_ACTIVE see below // optional
#define TIME_THREAD_ACTIVE see below // optional

namespace std {
 using size_t = see 17.2.4;
 using clock_t = see below;
 using time_t = see below;

 struct timespec;
 struct tm;

 clock_t clock();
 double difftime(time_t time1, time_t time0);
 time_t mktime(tm* timeptr);
 time_t timegm(tm* timeptr);
 time_t time(time_t* timer);
 int timespec_get(timespec* ts, int base);
 int timespec_getres(timespec* ts, int base);
 tm* gmtime(const time_t* timer);
 tm* gmtime_r(const time_t* timer, tm* buf);
 tm* localtime(const time_t* timer);
 tm* localtime_r(const time_t* timer, tm* buf);
 size_t strftime(char* s, size_t maxsize, const char* format, const tm* timeptr);
}
```

- <sup>1</sup> The contents of the header `<ctime>` are the same as the C standard library header `<time.h>`.<sup>233</sup>
- <sup>2</sup> The functions `gmtime` and `localtime` are not required to avoid data races (16.4.6.10).

SEE ALSO: ISO/IEC 9899:2024, 7.29

<sup>233</sup> `strftime` supports the C conversion specifiers C, D, e, F, g, G, h, r, R, t, T, u, V, and z, and the modifiers E and O.

# 31 Input/output library [input.output]

## 31.1 General [input.output.general]

- <sup>1</sup> This Clause describes components that C++ programs may use to perform input/output operations.
- <sup>2</sup> The following subclauses describe requirements for stream parameters, and components for forward declarations of iostreams, predefined iostreams objects, base iostreams classes, stream buffering, stream formatting and manipulators, string streams, and file streams, as summarized in [Table 135](#).

**Table 135 — Input/output library summary** [tab:iostreams.summary]

| Subclause                                         | Header                                                                                                                  |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <a href="#">31.2</a> Requirements                 |                                                                                                                         |
| <a href="#">31.3</a> Forward declarations         | <code>&lt;iosfwd&gt;</code>                                                                                             |
| <a href="#">31.4</a> Standard iostream objects    | <code>&lt;iostream&gt;</code>                                                                                           |
| <a href="#">31.5</a> Iostreams base classes       | <code>&lt;ios&gt;</code>                                                                                                |
| <a href="#">31.6</a> Stream buffers               | <code>&lt;streambuf&gt;</code>                                                                                          |
| <a href="#">31.7</a> Formatting and manipulators  | <code>&lt;istream&gt;</code> , <code>&lt;ostream&gt;</code> , <code>&lt;iomanip&gt;</code> , <code>&lt;print&gt;</code> |
| <a href="#">31.8</a> String streams               | <code>&lt;sstream&gt;</code>                                                                                            |
| <a href="#">31.9</a> Span-based streams           | <code>&lt;spanstream&gt;</code>                                                                                         |
| <a href="#">31.10</a> File streams                | <code>&lt;fstream&gt;</code>                                                                                            |
| <a href="#">31.11</a> Synchronized output streams | <code>&lt;syncstream&gt;</code>                                                                                         |
| <a href="#">31.12</a> File systems                | <code>&lt;filesystem&gt;</code>                                                                                         |
| <a href="#">31.13</a> C library files             | <code>&lt;cstdio&gt;</code> , <code>&lt;cinttypes&gt;</code>                                                            |

## 31.2 Iostreams requirements [iostreams.requirements]

### 31.2.1 Imbue limitations [iostream.limits.imbue]

- <sup>1</sup> No function described in [Clause 31](#) except for `ios_base::imbue` and `basic_filebuf::pubimbue` causes any instance of `basic_ios::imbue` or `basic_streambuf::imbue` to be called. If any user function called from a function declared in [Clause 31](#) or as an overriding virtual function of any class declared in [Clause 31](#) calls `imbue`, the behavior is undefined.

### 31.2.2 Types [stream.types]

```
using streamoff = implementation-defined;
```

- <sup>1</sup> The type `streamoff` is a synonym for one of the signed basic integral types of sufficient size to represent the maximum possible file size for the operating system.<sup>234</sup>

```
using streamsize = implementation-defined;
```

- <sup>2</sup> The type `streamsize` is a synonym for one of the signed basic integral types. It is used to represent the number of characters transferred in an I/O operation, or the size of I/O buffers.<sup>235</sup>

### 31.2.3 Positioning type limitations [iostreams.limits.pos]

- <sup>1</sup> The classes of [Clause 31](#) with template arguments `charT` and `traits` behave as described if `traits::pos_type` and `traits::off_type` are `streampos` and `streamoff` respectively. Except as noted explicitly below, their behavior when `traits::pos_type` and `traits::off_type` are other types is implementation-defined.
- <sup>2</sup> [Note 1: For each of the specializations of `char_traits` defined in [27.2.4](#), `state_type` denotes `mbstate_t`, `pos_type` denotes `fpos<mbstate_t>`, and `off_type` denotes `streamoff`. — end note]
- <sup>3</sup> In the classes of [Clause 31](#), a template parameter with name `charT` represents a member of the set of types containing `char`, `wchar_t`, and any other implementation-defined character container types ([3.10](#)) that meet the requirements for a character on which any of the iostream components can be instantiated.

<sup>234</sup>) Typically `long long`.

<sup>235</sup>) Most places where `streamsize` is used would use `size_t` in C, or `ssize_t` in POSIX.

**31.2.4 Thread safety****[iostreams.threadsafety]**

- <sup>1</sup> Concurrent access to a stream object (31.8, 31.10), stream buffer object (31.6), or C Library stream (31.13) by multiple threads may result in a data race (6.10.2) unless otherwise specified (31.4).

[Note 1: Data races result in undefined behavior (6.10.2). — end note]

- <sup>2</sup> If one thread makes a library call *a* that writes a value to a stream and, as a result, another thread reads this value from the stream through a library call *b* such that this does not result in a data race, then *a*'s write synchronizes with *b*'s read.

**31.3 Forward declarations****[iostream.forward]****31.3.1 Header <iosfwd> synopsis****[iosfwd.syn]**

```
namespace std {
 template<class charT> struct char_traits;
 template<> struct char_traits<char>;
 template<> struct char_traits<char8_t>;
 template<> struct char_traits<char16_t>;
 template<> struct char_traits<char32_t>;
 template<> struct char_traits<wchar_t>;

 template<class T> class allocator;

 template<class charT, class traits = char_traits<charT>>
 class basic_ios;
 template<class charT, class traits = char_traits<charT>>
 class basic_streambuf;
 template<class charT, class traits = char_traits<charT>>
 class basic_istream;
 template<class charT, class traits = char_traits<charT>>
 class basic_ostream;
 template<class charT, class traits = char_traits<charT>>
 class basic_iostream;

 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringbuf;
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_istringstream;
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_ostringstream;
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringstream;

 template<class charT, class traits = char_traits<charT>>
 class basic_spanbuf;
 template<class charT, class traits = char_traits<charT>>
 class basic_ispanstream;
 template<class charT, class traits = char_traits<charT>>
 class basic_ospansstream;
 template<class charT, class traits = char_traits<charT>>
 class basic_spanstream;

 template<class charT, class traits = char_traits<charT>>
 class basic_filebuf;
 template<class charT, class traits = char_traits<charT>>
 class basic_ifstream;
 template<class charT, class traits = char_traits<charT>>
 class basic_ofstream;
 template<class charT, class traits = char_traits<charT>>
 class basic_fstream;

 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_syncbuf;
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_osyncstream;
```

```

template<class charT, class traits = char_traits<charT>>
 class istreambuf_iterator;
template<class charT, class traits = char_traits<charT>>
 class ostreambuf_iterator;

using ios = basic_ios<char>;
using wios = basic_ios<wchar_t>;

using streambuf = basic_streambuf<char>;
using istream = basic_istream<char>;
using ostream = basic_ostream<char>;
using iostream = basic_iostream<char>;

using stringbuf = basic_stringbuf<char>;
using istringstream = basic_istream<char>;
using ostreamstream = basic_ostream<char>;
using stringstream = basic_stringstream<char>;

using spanbuf = basic_spanbuf<char>;
using ispanstream = basic_ispanstream<char>;
using ospanstream = basic_ostream<char>;
using spanstream = basic_spanstream<char>;

using filebuf = basic_filebuf<char>;
using ifstream = basic_ifstream<char>;
using ofstream = basic_ofstream<char>;
using fstream = basic_fstream<char>;

using syncbuf = basic_syncbuf<char>;
using osyncstream = basic_osyncstream<char>;

using wstreambuf = basic_streambuf<wchar_t>;
using wistream = basic_istream<wchar_t>;
using wostream = basic_ostream<wchar_t>;
using wiostream = basic_iostream<wchar_t>;

using wstringbuf = basic_stringbuf<wchar_t>;
using wistringstream = basic_istream<wchar_t>;
using wostringstream = basic_ostream<wchar_t>;
using wstringstream = basic_stringstream<wchar_t>;

using wspanbuf = basic_spanbuf<wchar_t>;
using wispanstream = basic_ispanstream<wchar_t>;
using wospanstream = basic_ostream<wchar_t>;
using wspanstream = basic_spanstream<wchar_t>;

using wfilebuf = basic_filebuf<wchar_t>;
using wifstream = basic_ifstream<wchar_t>;
using wofstream = basic_ofstream<wchar_t>;
using wfstream = basic_fstream<wchar_t>;

using wsyncbuf = basic_syncbuf<wchar_t>;
using wosyncstream = basic_osyncstream<wchar_t>;

template<class state> class fpos;
using streampos = fpos<char_traits<char>::state_type>;
using wstreampos = fpos<char_traits<wchar_t>::state_type>;
using u8streampos = fpos<char_traits<char8_t>::state_type>;
using u16streampos = fpos<char_traits<char16_t>::state_type>;
using u32streampos = fpos<char_traits<char32_t>::state_type>;
}

```

- <sup>1</sup> Default template arguments are described as appearing both in `<iosfwd>` and in the synopsis of other headers but it is well-formed to include both `<iosfwd>` and one or more of the other headers.<sup>236</sup>

### 31.3.2 Overview

[[iostream.forward.overview](#)]

- <sup>1</sup> The class template specialization `basic_ios<charT, traits>` serves as a virtual base class for the class templates `basic_istream`, `basic_ostream`, and class templates derived from them. `basic_iostream` is a class template derived from both `basic_istream<charT, traits>` and `basic_ostream<charT, traits>`.
- <sup>2</sup> The class template specialization `basic_streambuf<charT, traits>` serves as a base class for class templates `basic_stringbuf`, `basic_filebuf`, `basic_syncbuf`, and `basic_spanbuf`.
- <sup>3</sup> The class template specialization `basic_istream<charT, traits>` serves as a base class for class templates `basic_istreamstream`, `basic_ifstream`, and `basic_ispanstream`.
- <sup>4</sup> The class template specialization `basic_ostream<charT, traits>` serves as a base class for class templates `basic_ostreamstream`, `basic_ofstream`, `basic_osyncstream`, and `basic_ospstream`.
- <sup>5</sup> The class template specialization `basic_iostream<charT, traits>` serves as a base class for class templates `basic_stringstream`, `basic_fstream`, and `basic_spanstream`.
- <sup>6</sup> [Note 1: For each of the class templates above, the program is ill-formed if `traits::char_type` is not the same type as `charT` (27.2). — end note]
- <sup>7</sup> Other *typedef-names* designate class template specializations for `char` or `wchar_t` types.
- <sup>8</sup> Specializations of the class template `fpos` are used for specifying file position information.  
[Example 1: The types `streampos` and `wstreampos` are used for positioning streams specialized on `char` and `wchar_t` respectively. — end example]
- <sup>9</sup> [Note 2: This synopsis suggests a circularity between `streampos` and `char_traits<char>`. An implementation can avoid this circularity by substituting equivalent types. — end note]

## 31.4 Standard iostream objects

[[iostream.objects](#)]

### 31.4.1 Header `<iostream>` synopsis

[[iostream.syn](#)]

```
#include <ios> // see 31.5.1
#include <streambuf> // see 31.6.1
#include <istream> // see 31.7.1
#include <ostream> // see 31.7.2

namespace std {
 extern istream cin;
 extern ostream cout;
 extern ostream cerr;
 extern ostream clog;

 extern wistream wcin;
 extern wostream wcout;
 extern wostream wcerr;
 extern wostream wclog;
}
```

### 31.4.2 Overview

[[iostream.objects.overview](#)]

- <sup>1</sup> In this Clause, the type name `FILE` refers to the type `FILE` declared in `<cstdio>` (31.13.1).
- <sup>2</sup> The header `<iostream>` declares objects that associate objects with the standard C streams provided for by the functions declared in `<cstdio>`, and includes all the headers necessary to use these objects. The dynamic types of the stream buffers initially associated with these objects are unspecified, but they have the behavior specified for `std::basic_filebuf<char>` or `std::basic_filebuf<wchar_t>`.
- <sup>3</sup> The objects are constructed and the associations are established at some time prior to or during the first time an object of class `ios_base::Init` is constructed, and in any case before the body of `main` (6.10.3.1) begins execution. The objects are not destroyed during program execution.<sup>237</sup>

<sup>236</sup>) It is the implementation's responsibility to implement headers so that including `<iosfwd>` and other headers does not violate the rules about multiple occurrences of default arguments.

<sup>237</sup>) Constructors and destructors for objects with static storage duration can access these objects to read input from `stdin` or write output to `stdout` or `stderr`.



<sup>4</sup> *Recommended practice:* If it is possible for them to do so, implementations should initialize the objects earlier than required.

<sup>5</sup> The results of including `<iostream>` in a translation unit shall be as if `<iostream>` defined an instance of `ios_base::Init` with static storage duration. Each C++ library module (16.4.2.4) in a hosted implementation shall behave as if it contains an interface unit that defines an unexported `ios_base::Init` variable with ordered initialization (6.10.3.3).

[*Note 1:* As a result, the definition of that variable is appearance-ordered before any declaration following the point of importation of a C++ library module. Whether such a definition exists is unobservable by a program that does not reference any of the standard iostream objects. — *end note*]

<sup>6</sup> Mixing operations on corresponding wide- and narrow-character streams follows the same semantics as mixing such operations on `FILEs`, as specified in the C standard library.

<sup>7</sup> Concurrent access to a synchronized (31.5.2.5) standard iostream object's formatted and unformatted input (31.7.5.2) and output (31.7.6.2) functions or a standard C stream by multiple threads does not result in a data race (6.10.2).

[*Note 2:* Unsynchronized concurrent use of these objects and streams by multiple threads can result in interleaved characters. — *end note*]

SEE ALSO: ISO/IEC 9899:2024, 7.23.2

### 31.4.3 Narrow stream objects

[`narrow.stream.objects`]

```
istream cin;
```

<sup>1</sup> The object `cin` controls input from a stream buffer associated with the object `stdin`, declared in `<cstdio>` (31.13.1).

<sup>2</sup> After the object `cin` is initialized, `cin.tie()` returns `&cout`. Its state is otherwise the same as required for `basic_ios<char>::init` (31.5.4.2).

```
ostream cout;
```

<sup>3</sup> The object `cout` controls output to a stream buffer associated with the object `stdout`, declared in `<cstdio>` (31.13.1).

```
ostream cerr;
```

<sup>4</sup> The object `cerr` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (31.13.1).

<sup>5</sup> After the object `cerr` is initialized, `cerr.flags() & unitbuf` is nonzero and `cerr.tie()` returns `&cout`. Its state is otherwise the same as required for `basic_ios<char>::init` (31.5.4.2).

```
ostream clog;
```

<sup>6</sup> The object `clog` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (31.13.1).

### 31.4.4 Wide stream objects

[`wide.stream.objects`]

```
wistream wcin;
```

<sup>1</sup> The object `wcin` controls input from a stream buffer associated with the object `stdin`, declared in `<cstdio>` (31.13.1).

<sup>2</sup> After the object `wcin` is initialized, `wcin.tie()` returns `&wcout`. Its state is otherwise the same as required for `basic_ios<wchar_t>::init` (31.5.4.2).

```
wostream wcout;
```

<sup>3</sup> The object `wcout` controls output to a stream buffer associated with the object `stdout`, declared in `<cstdio>` (31.13.1).

```
wostream wcerr;
```

<sup>4</sup> The object `wcerr` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (31.13.1).

- 5 After the object `wcerr` is initialized, `wcerr.flags()` & `unitbuf` is nonzero and `wcerr.tie()` returns `&wcout`. Its state is otherwise the same as required for `basic_ios<wchar_t>::init` (31.5.4.2).

`wostream wlog;`

- 6 The object `wlog` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (31.13.1).

## 31.5 Iostreams base classes

[iostreams.base]

### 31.5.1 Header `<ios>` synopsis

[ios.syn]

```
#include <iosfwd> // see 31.3.1

namespace std {
 // 31.2.2, types
 using streamoff = implementation-defined;
 using streamsize = implementation-defined;
 // 31.5.3, class template fpos
 template<class stateT> class fpos;

 // 31.5.2, class ios_base
 class ios_base;
 // 31.5.4, class template basic_ios
 template<class charT, class traits = char_traits<charT>>
 class basic_ios;

 // 31.5.5, manipulators
 ios_base& boolalpha (ios_base& str);
 ios_base& noboolalpha(ios_base& str);

 ios_base& showbase (ios_base& str);
 ios_base& noshowbase (ios_base& str);

 ios_base& showpoint (ios_base& str);
 ios_base& noshowpoint(ios_base& str);

 ios_base& showpos (ios_base& str);
 ios_base& noshowpos (ios_base& str);

 ios_base& skipws (ios_base& str);
 ios_base& noskipws (ios_base& str);

 ios_base& uppercase (ios_base& str);
 ios_base& nouppercase(ios_base& str);

 ios_base& unitbuf (ios_base& str);
 ios_base& nunitbuf (ios_base& str);

 // 31.5.5.2, adjustfield
 ios_base& internal (ios_base& str);
 ios_base& left (ios_base& str);
 ios_base& right (ios_base& str);

 // 31.5.5.3, basefield
 ios_base& dec (ios_base& str);
 ios_base& hex (ios_base& str);
 ios_base& oct (ios_base& str);

 // 31.5.5.4, floatfield
 ios_base& fixed (ios_base& str);
 ios_base& scientific (ios_base& str);
 ios_base& hexfloat (ios_base& str);
 ios_base& defaultfloat(ios_base& str);
```

```

// 31.5.6, error reporting
enum class io_errc {
 stream = 1
};

template<> struct is_error_code_enum<io_errc> : public true_type { };
error_code make_error_code(io_errc e) noexcept;
error_condition make_error_condition(io_errc e) noexcept;
const error_category& iostream_category() noexcept;
}

```

## 31.5.2 Class ios\_base

[ios.base]

### 31.5.2.1 General

[ios.base.general]

```

namespace std {
 class ios_base {
 public:
 class failure; // see below

 // 31.5.2.2.2, fmtflags
 using fmtflags = T1;
 static constexpr fmtflags boolalpha = unspecified;
 static constexpr fmtflags dec = unspecified;
 static constexpr fmtflags fixed = unspecified;
 static constexpr fmtflags hex = unspecified;
 static constexpr fmtflags internal = unspecified;
 static constexpr fmtflags left = unspecified;
 static constexpr fmtflags oct = unspecified;
 static constexpr fmtflags right = unspecified;
 static constexpr fmtflags scientific = unspecified;
 static constexpr fmtflags showbase = unspecified;
 static constexpr fmtflags showpoint = unspecified;
 static constexpr fmtflags showpos = unspecified;
 static constexpr fmtflags skipws = unspecified;
 static constexpr fmtflags unitbuf = unspecified;
 static constexpr fmtflags uppercase = unspecified;
 static constexpr fmtflags adjustfield = see below;
 static constexpr fmtflags basefield = see below;
 static constexpr fmtflags floatfield = see below;

 // 31.5.2.2.3, iostate
 using iostate = T2;
 static constexpr iostate badbit = unspecified;
 static constexpr iostate eofbit = unspecified;
 static constexpr iostate failbit = unspecified;
 static constexpr iostate goodbit = see below;

 // 31.5.2.2.4, openmode
 using openmode = T3;
 static constexpr openmode app = unspecified;
 static constexpr openmode ate = unspecified;
 static constexpr openmode binary = unspecified;
 static constexpr openmode in = unspecified;
 static constexpr openmode noreplace = unspecified;
 static constexpr openmode out = unspecified;
 static constexpr openmode trunc = unspecified;

 // 31.5.2.2.5, seekdir
 using seekdir = T4;
 static constexpr seekdir beg = unspecified;
 static constexpr seekdir cur = unspecified;
 static constexpr seekdir end = unspecified;

 class Init;
 };
}

```

```

// 31.5.2.3, fmtflags state
fmtflags flags() const;
fmtflags flags(fmtflags fmtfl);
fmtflags setf(fmtflags fmtfl);
fmtflags setf(fmtflags fmtfl, fmtflags mask);
void unsetf(fmtflags mask);

streamsize precision() const;
streamsize precision(streamsize prec);
streamsize width() const;
streamsize width(streamsize wide);

// 31.5.2.4, locales
locale imbue(const locale& loc);
locale getloc() const;

// 31.5.2.6, storage
static int xalloc();
long& iword(int idx);
void*& pword(int idx);

// destructor
virtual ~ios_base();

// 31.5.2.7, callbacks
enum event { erase_event, imbue_event, copyfmt_event };
using event_callback = void (*)(event, ios_base&, int idx);
void register_callback(event_callback fn, int idx);

ios_base(const ios_base&) = delete;
ios_base& operator=(const ios_base&) = delete;

static bool sync_with_stdio(bool sync = true);

protected:
 ios_base();

private:
 static int index; // exposition only
 long* iarray; // exposition only
 void** parray; // exposition only
};
}

```

<sup>1</sup> `ios_base` defines several member types:

- (1.1) — a type `failure`, defined as either a class derived from `system_error` or a synonym for a class derived from `system_error`;
- (1.2) — a class `Init`;
- (1.3) — three bitmask types, `fmtflags`, `iostate`, and `openmode`;
- (1.4) — an enumerated type, `seekdir`.

<sup>2</sup> It maintains several kinds of data:

- (2.1) — state information that reflects the integrity of the stream buffer;
- (2.2) — control information that influences how to interpret (format) input sequences and how to generate (format) output sequences;
- (2.3) — additional information that is stored by the program for its private use.

<sup>3</sup> [Note 1: For the sake of exposition, the maintained data is presented here as:

- (3.1) — `static int index`, specifies the next available unique index for the integer or pointer arrays maintained for the private use of the program, initialized to an unspecified value;

- (3.2) — `long* iarray`, points to the first element of an arbitrary-length `long` array maintained for the private use of the program;
- (3.3) — `void** parray`, points to the first element of an arbitrary-length pointer array maintained for the private use of the program.
- *end note*]

### 31.5.2.2 Types

[ios.types]

#### 31.5.2.2.1 Class `ios_base::failure`

[ios.failure]

```
namespace std {
 class ios_base::failure : public system_error {
 public:
 explicit failure(const string& msg, const error_code& ec = io_errc::stream);
 explicit failure(const char* msg, const error_code& ec = io_errc::stream);
 };
}
```

- <sup>1</sup> An implementation is permitted to define `ios_base::failure` as a synonym for a class with equivalent functionality to class `ios_base::failure` shown in this subclause.

[*Note 1:* When `ios_base::failure` is a synonym for another type, that type needs to provide a nested type `failure` to emulate the injected-class-name. — *end note*]

The class `failure` defines the base class for the types of all objects thrown as exceptions, by functions in the `iostreams` library, to report errors detected during stream buffer operations.

- <sup>2</sup> When throwing `ios_base::failure` exceptions, implementations should provide values of `ec` that identify the specific reason for the failure.

[*Note 2:* Errors arising from the operating system would typically be reported as `system_category()` errors with an error value of the error number reported by the operating system. Errors arising from within the stream library would typically be reported as `error_code(io_errc::stream, iostream_category())`. — *end note*]

```
explicit failure(const string& msg, const error_code& ec = io_errc::stream);
```

- <sup>3</sup> *Effects:* Constructs the base class with `msg` and `ec`.

```
explicit failure(const char* msg, const error_code& ec = io_errc::stream);
```

- <sup>4</sup> *Effects:* Constructs the base class with `msg` and `ec`.

#### 31.5.2.2.2 Type `ios_base::fmtflags`

[ios.fmtflags]

```
using fmtflags = T1;
```

- <sup>1</sup> The type `fmtflags` is a bitmask type (16.3.3.3.3). Setting its elements has the effects indicated in Table 136.
- <sup>2</sup> Type `fmtflags` also defines the constants indicated in Table 137.

#### 31.5.2.2.3 Type `ios_base::iostate`

[ios.iostate]

```
using iostate = T2;
```

- <sup>1</sup> The type `iostate` is a bitmask type (16.3.3.3.3) that contains the elements indicated in Table 138.
- <sup>2</sup> Type `iostate` also defines the constant:
- (2.1) — `goodbit`, the value zero.

#### 31.5.2.2.4 Type `ios_base::openmode`

[ios.openmode]

```
using openmode = T3;
```

- <sup>1</sup> The type `openmode` is a bitmask type (16.3.3.3.3). It contains the elements indicated in Table 139.

#### 31.5.2.2.5 Type `ios_base::seekdir`

[ios.seekdir]

```
using seekdir = T4;
```

- <sup>1</sup> The type `seekdir` is an enumerated type (16.3.3.3.2) that contains the elements indicated in Table 140.

Table 136 — `fmtflags` effects [tab:ios.fmtflags]

| Element                 | Effect(s) if set                                                                                                                                   |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>boolalpha</code>  | insert and extract <code>bool</code> type in alphabetic format                                                                                     |
| <code>dec</code>        | converts integer input or generates integer output in decimal base                                                                                 |
| <code>fixed</code>      | generate floating-point output in fixed-point notation                                                                                             |
| <code>hex</code>        | converts integer input or generates integer output in hexadecimal base                                                                             |
| <code>internal</code>   | adds fill characters at a designated internal point in certain generated output, or identical to <code>right</code> if no such point is designated |
| <code>left</code>       | adds fill characters on the right (final positions) of certain generated output                                                                    |
| <code>oct</code>        | converts integer input or generates integer output in octal base                                                                                   |
| <code>right</code>      | adds fill characters on the left (initial positions) of certain generated output                                                                   |
| <code>scientific</code> | generates floating-point output in scientific notation                                                                                             |
| <code>showbase</code>   | generates a prefix indicating the numeric base of generated integer output                                                                         |
| <code>showpoint</code>  | generates a decimal-point character unconditionally in generated floating-point output                                                             |
| <code>showpos</code>    | generates a <code>+</code> sign in non-negative generated numeric output                                                                           |
| <code>skipws</code>     | skips leading whitespace before certain input operations                                                                                           |
| <code>unitbuf</code>    | flushes output after each output operation                                                                                                         |
| <code>uppercase</code>  | replaces certain lowercase letters with their uppercase equivalents in generated output                                                            |

Table 137 — `fmtflags` constants [tab:ios.fmtflags.const]

| Constant                 | Allowable values                                               |
|--------------------------|----------------------------------------------------------------|
| <code>adjustfield</code> | <code>left</code>   <code>right</code>   <code>internal</code> |
| <code>basefield</code>   | <code>dec</code>   <code>oct</code>   <code>hex</code>         |
| <code>floatfield</code>  | <code>scientific</code>   <code>fixed</code>                   |

Table 138 — `iostate` effects [tab:ios.iostate]

| Element              | Effect(s) if set                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>badbit</code>  | indicates a loss of integrity in an input or output sequence (such as an irrecoverable read error from a file);                                  |
| <code>eofbit</code>  | indicates that an input operation reached the end of an input sequence;                                                                          |
| <code>failbit</code> | indicates that an input operation failed to read the expected characters, or that an output operation failed to generate the desired characters. |

Table 139 — `openmode` effects [tab:ios.openmode]

| Element                | Effect(s) if set                                                  |
|------------------------|-------------------------------------------------------------------|
| <code>app</code>       | seek to end before each write                                     |
| <code>ate</code>       | open and seek to end immediately after opening                    |
| <code>binary</code>    | perform input and output in binary mode (as opposed to text mode) |
| <code>in</code>        | open for input                                                    |
| <code>noreplace</code> | open in exclusive mode                                            |
| <code>out</code>       | open for output                                                   |
| <code>trunc</code>     | truncate an existing stream when opening                          |

Table 140 — seekdir effects [tab:ios.seekdir]

| Element | Meaning                                                                                 |
|---------|-----------------------------------------------------------------------------------------|
| beg     | request a seek (for subsequent input or output) relative to the beginning of the stream |
| cur     | request a seek relative to the current position within the sequence                     |
| end     | request a seek relative to the current end of the sequence                              |

**31.5.2.2.6 Class ios\_base::Init**

[ios.init]

```

namespace std {
 class ios_base::Init {
 public:
 Init();
 Init(const Init&) = default;
 ~Init();
 Init& operator=(const Init&) = default;
 };
}

```

- <sup>1</sup> The class `Init` describes an object whose construction ensures the construction of the eight objects declared in `<iostream>` (31.4) that associate file stream buffers with the standard C streams provided for by the functions declared in `<cstdio>` (31.13.1).

`Init();`

- <sup>2</sup> *Effects:* Constructs and initializes the objects `cin`, `cout`, `cerr`, `clog`, `wcin`, `wcout`, `wcerr`, and `wclog` if they have not already been constructed and initialized.

`~Init();`

- <sup>3</sup> *Effects:* If there are no other instances of the class still in existence, calls `cout.flush()`, `cerr.flush()`, `clog.flush()`, `wcout.flush()`, `wcerr.flush()`, `wclog.flush()`.

**31.5.2.3 State functions**

[fmtflags.state]

```
fmtflags flags() const;
```

- <sup>1</sup> *Returns:* The format control information for both input and output.

```
fmtflags flags(fmtflags fmtfl);
```

- <sup>2</sup> *Postconditions:* `fmtfl == flags()`.

- <sup>3</sup> *Returns:* The previous value of `flags()`.

```
fmtflags setf(fmtflags fmtfl);
```

- <sup>4</sup> *Effects:* Sets `fmtfl` in `flags()`.

- <sup>5</sup> *Returns:* The previous value of `flags()`.

```
fmtflags setf(fmtflags fmtfl, fmtflags mask);
```

- <sup>6</sup> *Effects:* Clears `mask` in `flags()`, sets `fmtfl & mask` in `flags()`.

- <sup>7</sup> *Returns:* The previous value of `flags()`.

```
void unsetf(fmtflags mask);
```

- <sup>8</sup> *Effects:* Clears `mask` in `flags()`.

```
streamsize precision() const;
```

- <sup>9</sup> *Returns:* The precision to generate on certain output conversions.

```
streamsize precision(streamsize prec);
```

- <sup>10</sup> *Postconditions:* `prec == precision()`.

- <sup>11</sup> *Returns:* The previous value of `precision()`.

```
streamsize width() const;
```

12     *Returns:* The minimum field width (number of characters) to generate on certain output conversions.

```
streamsize width(streamsize wide);
```

13     *Postconditions:* `wide == width()`.

14     *Returns:* The previous value of `width()`.

#### 31.5.2.4 Functions

[ios.base.locales]

```
locale imbue(const locale& loc);
```

1     *Effects:* Calls each registered callback pair (`fn`, `idx`) (31.5.2.7) as `(*fn)(imbue_event, *this, idx)` at such a time that a call to `ios_base::getloc()` from within `fn` returns the new locale value `loc`.

2     *Postconditions:* `loc == getloc()`.

3     *Returns:* The previous value of `getloc()`.

```
locale getloc() const;
```

4     *Returns:* If no locale has been imbued, a copy of the global C++ locale, `locale()`, in effect at the time of construction. Otherwise, returns the imbued locale, to be used to perform locale-dependent input and output operations.

#### 31.5.2.5 Static members

[ios.members.static]

```
static bool sync_with_stdio(bool sync = true);
```

1     *Effects:* If any input or output operation has occurred using the standard streams prior to the call, the effect is implementation-defined. Otherwise, called with a `false` argument, it allows the standard streams to operate independently of the standard C streams.

2     *Returns:* `true` if the previous state of the standard iostream objects (31.4) was synchronized and otherwise returns `false`. The first time it is called, the function returns `true`.

3     *Remarks:* When a standard iostream object `str` is *synchronized* with a standard stdio stream `f`, the effect of inserting a character `c` by

```
 fputc(f, c);
```

is the same as the effect of

```
 str.rdbuf()->sputc(c);
```

for any sequences of characters; the effect of extracting a character `c` by

```
 c = fgetc(f);
```

is the same as the effect of

```
 c = str.rdbuf()->sgetc();
```

for any sequences of characters; and the effect of pushing back a character `c` by

```
 ungetc(c, f);
```

is the same as the effect of

```
 str.rdbuf()->sputbackc(c);
```

for any sequence of characters.<sup>238</sup>

#### 31.5.2.6 Storage functions

[ios.base.storage]

```
static int xalloc();
```

1     *Returns:* `index ++`.

2     *Remarks:* Concurrent access to this function by multiple threads does not result in a data race (6.10.2).

---

238) This implies that operations on a standard iostream object can be mixed arbitrarily with operations on the corresponding stdio stream. In practical terms, synchronization usually means that a standard iostream object and a standard stdio object share a buffer.



```
long& iword(int idx);
```

3     *Preconditions:* `idx` is a value obtained by a call to `xalloc`.

4     *Effects:* If `iarray` is a null pointer, allocates an array of `long` of unspecified size and stores a pointer to its first element in `iarray`. The function then extends the array pointed at by `iarray` as necessary to include the element `iarray[idx]`. Each newly allocated element of the array is initialized to zero. The reference returned is invalid after any other operation on the object.<sup>239</sup> However, the value of the storage referred to is retained, so that until the next call to `copyfmt`, calling `iword` with the same index yields another reference to the same value. If the function fails<sup>240</sup> and `*this` is a base class subobject of a `basic_ios<>` object or subobject, the effect is equivalent to calling `basic_ios<>::setstate(badbit)` on the derived object (which may throw `failure`).

5     *Returns:* On success `iarray[idx]`. On failure, a valid `long&` initialized to 0.

```
void*& pword(int idx);
```

6     *Preconditions:* `idx` is a value obtained by a call to `xalloc`.

7     *Effects:* If `parray` is a null pointer, allocates an array of pointers to `void` of unspecified size and stores a pointer to its first element in `parray`. The function then extends the array pointed at by `parray` as necessary to include the element `parray[idx]`. Each newly allocated element of the array is initialized to a null pointer. The reference returned is invalid after any other operation on the object. However, the value of the storage referred to is retained, so that until the next call to `copyfmt`, calling `pword` with the same index yields another reference to the same value. If the function fails<sup>241</sup> and `*this` is a base class subobject of a `basic_ios<>` object or subobject, the effect is equivalent to calling `basic_ios<>::setstate(badbit)` on the derived object (which may throw `failure`).

8     *Returns:* On success `parray[idx]`. On failure a valid `void*&` initialized to 0.

9     *Remarks:* After a subsequent call to `pword(int)` for the same object, the earlier return value may no longer be valid.

### 31.5.2.7 Callbacks

[ios.base.callback]

```
void register_callback(event_callback fn, int idx);
```

1     *Preconditions:* The function `fn` does not throw exceptions.

2     *Effects:* Registers the pair `(fn, idx)` such that during calls to `imbue()` (31.5.2.4), `copyfmt()`, or `~ios_base()` (31.5.2.8), the function `fn` is called with argument `idx`. Functions registered are called when an event occurs, in opposite order of registration. Functions registered while a callback function is active are not called until the next event.

3     *Remarks:* Identical pairs are not merged. A function registered twice will be called twice.

### 31.5.2.8 Constructors and destructor

[ios.base.cons]

```
ios_base();
```

1     *Effects:* Each `ios_base` member has an indeterminate value after construction. The object's members shall be initialized by calling `basic_ios::init` before the object's first use or before it is destroyed, whichever comes first; otherwise the behavior is undefined.

```
~ios_base();
```

2     *Effects:* Calls each registered callback pair `(fn, idx)` (31.5.2.7) as `(*fn)(erase_event, *this, idx)` at such time that any `ios_base` member function called from within `fn` has well-defined results. Then, any memory obtained is deallocated.

## 31.5.3 Class template `fpos`

[fpos]

### 31.5.3.1 General

[fpos.general]

```
namespace std {
 template<class stateT> class fpos {
```

239) An implementation is free to implement both the integer array pointed at by `iarray` and the pointer array pointed at by `parray` as sparse data structures, possibly with a one-element cache for each.

240) For example, because it cannot allocate space.

241) For example, because it cannot allocate space.

```

public:
 // 31.5.3.2, members
 stateT state() const;
 void state(stateT);

private:
 stateT st; // exposition only
};
}

```

### 31.5.3.2 Members

[fpos.members]

```
void state(stateT s);
```

1 *Effects:* Assigns `s` to `st`.

```
stateT state() const;
```

2 *Returns:* Current value of `st`.

### 31.5.3.3 Requirements

[fpos.operations]

1 An `fpos` type specifies file position information. It holds a state object whose type is equal to the template parameter `stateT`. Type `stateT` shall meet the *Cpp17DefaultConstructible* (Table 30), *Cpp17CopyConstructible* (Table 32), *Cpp17CopyAssignable* (Table 34), and *Cpp17Destructible* (Table 35) requirements. If `is_trivially_copy_constructible_v<stateT>` is true, then `fpos<stateT>` has a trivial copy constructor. If `is_trivially_copy_assignable_v<stateT>` is true, then `fpos<stateT>` has a trivial copy assignment operator. If `is_trivially_destructible_v<stateT>` is true, then `fpos<stateT>` has a trivial destructor. All specializations of `fpos` meet the *Cpp17DefaultConstructible*, *Cpp17CopyConstructible*, *Cpp17CopyAssignable*, *Cpp17Destructible*, and *Cpp17EqualityComparable* (Table 28) requirements. In addition, the expressions shown in Table 141 are valid and have the indicated semantics. In that table,

- (1.1) — `P` refers to a specialization of `fpos`,
- (1.2) — `p` and `q` refer to values of type `P` or `const P`,
- (1.3) — `p1` and `q1` refer to modifiable lvalues of type `P`,
- (1.4) — `0` refers to type `streamoff`, and
- (1.5) — `o` and `o2` refer to values of type `streamoff` or `const streamoff`.

Table 141 — Position type requirements [tab:fpos.operations]

| Expression                                    | Return type            | Operational semantics             | Assertion/note pre-/post-condition                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------|------------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>P(o)</code>                             | <code>P</code>         | converts from <code>offset</code> | <i>Effects:</i> Value-initializes the state object.                                                                                                                                                                                                                                                                                                                                                      |
| <code>P p(o);</code><br><code>P p = o;</code> |                        |                                   | <i>Effects:</i> Value-initializes the state object.<br><i>Postconditions:</i> <code>p == P(o)</code> is true.                                                                                                                                                                                                                                                                                            |
| <code>P()</code>                              | <code>P</code>         | <code>P(0)</code>                 |                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>P p;</code>                             |                        | <code>P p(0);</code>              |                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>0(p)</code>                             | <code>streamoff</code> | converts to <code>offset</code>   | <code>P(0(p)) == p</code>                                                                                                                                                                                                                                                                                                                                                                                |
| <code>p == q</code>                           | <code>bool</code>      |                                   | <i>Remarks:</i> For any two values <code>o</code> and <code>o2</code> , if <code>p</code> is obtained from <code>o</code> converted to <code>P</code> or from a copy of such <code>P</code> value and if <code>q</code> is obtained from <code>o2</code> converted to <code>P</code> or from a copy of such <code>P</code> value, then <code>p == q</code> is true only if <code>o == o2</code> is true. |
| <code>p != q</code>                           | <code>bool</code>      | <code>!(p == q)</code>            |                                                                                                                                                                                                                                                                                                                                                                                                          |
| <code>p + o</code>                            | <code>P</code>         | <code>+ offset</code>             | <i>Remarks:</i> With <code>q1 = p + o;</code> , then: <code>q1 - o == p</code>                                                                                                                                                                                                                                                                                                                           |

Table 141 — Position type requirements (continued)

| Expression           | Return type                   | Operational semantics  | Assertion/note pre-/post-condition                                                                       |
|----------------------|-------------------------------|------------------------|----------------------------------------------------------------------------------------------------------|
| <code>p1 += o</code> | <code>P&amp;</code>           | <code>+= offset</code> | <i>Remarks:</i> With <code>q1 = p1</code> ; before the <code>+=</code> , then: <code>p1 - o == q1</code> |
| <code>p - o</code>   | <code>P</code>                | <code>- offset</code>  | <i>Remarks:</i> With <code>q1 = p - o</code> ;; then: <code>q1 + o == p</code>                           |
| <code>p1 -= o</code> | <code>P&amp;</code>           | <code>-= offset</code> | <i>Remarks:</i> With <code>q1 = p1</code> ; before the <code>-=</code> , then: <code>p1 + o == q1</code> |
| <code>o + p</code>   | convertible to <code>P</code> | <code>p + o</code>     | <code>P(o + p) == p + o</code>                                                                           |
| <code>p - q</code>   | <code>streamoff</code>        | distance               | <code>p == q + (p - q)</code>                                                                            |

- <sup>2</sup> Stream operations that return a value of type `traits::pos_type` return `P(0(-1))` as an invalid value to signal an error. If this value is used as an argument to any `istream`, `ostream`, or `streambuf` member that accepts a value of type `traits::pos_type` then the behavior of that function is undefined.

### 31.5.4 Class template `basic_ios`

[ios]

#### 31.5.4.1 Overview

[ios.overview]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_ios : public ios_base {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;

 // 31.5.4.4, flags functions
 explicit operator bool() const;
 bool operator!() const;
 iostate rdstate() const;
 void clear(iostate state = goodbit);
 void setstate(iostate state);
 bool good() const;
 bool eof() const;
 bool fail() const;
 bool bad() const;

 iostate exceptions() const;
 void exceptions(iostate except);

 // 31.5.4.2, constructor/destructor
 explicit basic_ios(basic_streambuf<charT, traits>* sb);
 virtual ~basic_ios();

 // 31.5.4.3, members
 basic_ostream<charT, traits>* tie() const;
 basic_ostream<charT, traits>* tie(basic_ostream<charT, traits>* tiestr);

 basic_streambuf<charT, traits>* rdbuf() const;
 basic_streambuf<charT, traits>* rdbuf(basic_streambuf<charT, traits>* sb);

 basic_ios& copyfmt(const basic_ios& rhs);

 char_type fill() const;
 char_type fill(char_type ch);

 locale imbue(const locale& loc);
 };

```

```

char narrow(char_type c, char dfault) const;
char_type widen(char c) const;

basic_ios(const basic_ios&) = delete;
basic_ios& operator=(const basic_ios&) = delete;

protected:
 basic_ios();
 void init(basic_streambuf<charT, traits>* sb);
 void move(basic_ios& rhs);
 void move(basic_ios&& rhs);
 void swap(basic_ios& rhs) noexcept;
 void set_rdbuf(basic_streambuf<charT, traits>* sb);
};
}

```

#### 31.5.4.2 Constructors

[basic.ios.cons]

```
explicit basic_ios(basic_streambuf<charT, traits>* sb);
```

<sup>1</sup> *Effects:* Assigns initial values to its member objects by calling `init(sb)`.

```
basic_ios();
```

<sup>2</sup> *Effects:* Leaves its member objects uninitialized. The object shall be initialized by calling `basic_ios::init` before its first use or before it is destroyed, whichever comes first; otherwise the behavior is undefined.

```
~basic_ios();
```

<sup>3</sup> *Remarks:* The destructor does not destroy `rdbuf()`.

```
void init(basic_streambuf<charT, traits>* sb);
```

<sup>4</sup> *Postconditions:* The postconditions of this function are indicated in [Table 142](#).

**Table 142** — `basic_ios::init()` effects [tab:basic.ios.cons]

| Element                   | Value                                                               |
|---------------------------|---------------------------------------------------------------------|
| <code>rdbuf()</code>      | <code>sb</code>                                                     |
| <code>tie()</code>        | <code>0</code>                                                      |
| <code>rdstate()</code>    | goodbit if <code>sb</code> is not a null pointer, otherwise badbit. |
| <code>exceptions()</code> | goodbit                                                             |
| <code>flags()</code>      | <code>skipws   dec</code>                                           |
| <code>width()</code>      | <code>0</code>                                                      |
| <code>precision()</code>  | <code>6</code>                                                      |
| <code>fill()</code>       | <code>widen(' ')</code>                                             |
| <code>getloc()</code>     | a copy of the value returned by <code>locale()</code>               |
| <i>iarray</i>             | a null pointer                                                      |
| <i>parray</i>             | a null pointer                                                      |

#### 31.5.4.3 Member functions

[basic.ios.members]

```
basic_ostream<charT, traits>* tie() const;
```

<sup>1</sup> *Returns:* An output sequence that is *tied* to (synchronized with) the sequence controlled by the stream buffer.

```
basic_ostream<charT, traits>* tie(basic_ostream<charT, traits>* tiestr);
```

<sup>2</sup> *Preconditions:* If `tiestr` is not null, `tiestr` is not reachable by traversing the linked list of tied stream objects starting from `tiestr->tie()`.

<sup>3</sup> *Postconditions:* `tiestr == tie()`.

4       *Returns:* The previous value of `tie()`.

```
basic_streambuf<charT, traits>* rdbuf() const;
```

5       *Returns:* A pointer to the `streambuf` associated with the stream.

```
basic_streambuf<charT, traits>* rdbuf(basic_streambuf<charT, traits>* sb);
```

6       *Effects:* Calls `clear()`.

7       *Postconditions:* `sb == rdbuf()`.

8       *Returns:* The previous value of `rdbuf()`.

```
locale imbue(const locale& loc);
```

9       *Effects:* Calls `ios_base::imbue(loc)` (31.5.2.4) and if `rdbuf() != 0` then `rdbuf()->pubimbue(loc)` (31.6.3.3.1).

10      *Returns:* The prior value of `ios_base::imbue()`.

```
char narrow(char_type c, char dfault) const;
```

11      *Returns:* `use_facet<ctype<char_type>>(getloc()).narrow(c, dfault)`.

```
char_type widen(char c) const;
```

12      *Returns:* `use_facet<ctype<char_type>>(getloc()).widen(c)`.

```
char_type fill() const;
```

13      *Returns:* The character used to pad (fill) an output conversion to the specified field width.

```
char_type fill(char_type fillch);
```

14      *Postconditions:* `traits::eq(fillch, fill())`.

15      *Returns:* The previous value of `fill()`.

```
basic_ios& copyfmt(const basic_ios& rhs);
```

16      *Effects:* If `(this == addressof(rhs))` is `true` does nothing. Otherwise assigns to the member objects of `*this` the corresponding member objects of `rhs` as follows:

- (16.1)      — calls each registered callback pair `(fn, idx)` as `(*fn)(erase_event, *this, idx)`;
- (16.2)      — then, assigns to the member objects of `*this` the corresponding member objects of `rhs`, except that
  - (16.2.1)      — `rdstate()`, `rdbuf()`, and `exceptions()` are left unchanged;
  - (16.2.2)      — the contents of arrays pointed at by `pword` and `word` are copied, not the pointers themselves;<sup>242</sup> and
  - (16.2.3)      — if any newly stored pointer values in `*this` point at objects stored outside the object `rhs` and those objects are destroyed when `rhs` is destroyed, the newly stored pointer values are altered to point at newly constructed copies of the objects;
- (16.3)      — then, calls each callback pair that was copied from `rhs` as `(*fn)(copyfmt_event, *this, idx)`;
- (16.4)      — then, calls `exceptions(rhs.exceptions())`.

17      [*Note 1:* The second pass through the callback pairs permits a copied `pword` value to be zeroed, or to have its referent deep copied or reference counted, or to have other special action taken. — *end note*]

18      *Postconditions:* The postconditions of this function are indicated in Table 143.

19      *Returns:* `*this`.

```
void move(basic_ios& rhs);
void move(basic_ios&& rhs);
```

20      *Postconditions:* `*this` has the state that `rhs` had before the function call, except that `rdbuf()` returns `nullptr`. `rhs` is in a valid but unspecified state, except that `rhs.rdbuf()` returns the same value as it returned before the function call, and `rhs.tie()` returns `nullptr`.

<sup>242</sup> This suggests an infinite amount of copying, but the implementation can keep track of the maximum element of the arrays that is nonzero.

Table 143 — `basic_ios::copyfmt()` effects [tab:basic.ios.copyfmt]

| Element                   | Value                         |
|---------------------------|-------------------------------|
| <code>rdbuf()</code>      | <i>unchanged</i>              |
| <code>tie()</code>        | <code>rhs.tie()</code>        |
| <code>rdstate()</code>    | <i>unchanged</i>              |
| <code>exceptions()</code> | <code>rhs.exceptions()</code> |
| <code>flags()</code>      | <code>rhs.flags()</code>      |
| <code>width()</code>      | <code>rhs.width()</code>      |
| <code>precision()</code>  | <code>rhs.precision()</code>  |
| <code>fill()</code>       | <code>rhs.fill()</code>       |
| <code>getloc()</code>     | <code>rhs.getloc()</code>     |

`void swap(basic_ios& rhs) noexcept;`

21 *Effects:* The states of `*this` and `rhs` are exchanged, except that `rdbuf()` returns the same value as it returned before the function call, and `rhs.rdbuf()` returns the same value as it returned before the function call.

`void set_rdbuf(basic_streambuf<charT, traits>* sb);`

22 *Preconditions:* `sb != nullptr` is true.

23 *Effects:* Associates the `basic_streambuf` object pointed to by `sb` with this stream without calling `clear()`.

24 *Postconditions:* `rdbuf() == sb` is true.

25 *Throws:* Nothing.

#### 31.5.4.4 Flags functions

[iostate.flags]

`explicit operator bool() const;`

1 *Returns:* `!fail()`.

`bool operator!() const;`

2 *Returns:* `fail()`.

`iostate rdstate() const;`

3 *Returns:* The error state of the stream buffer.

`void clear(iostate state = goodbit);`

4 *Effects:* If `((state | (rdbuf() ? goodbit : badbit)) & exceptions()) == 0`, returns. Otherwise, the function throws an object of class `ios_base::failure` (31.5.2.2.1), constructed with implementation-defined argument values.

5 *Postconditions:* If `rdbuf() != 0` then `state == rdstate()`; otherwise `rdstate() == (state | ios_base::badbit)`.

`void setstate(iostate state);`

6 *Effects:* Calls `clear(rdstate() | state)` (which may throw `ios_base::failure` (31.5.2.2.1)).

`bool good() const;`

7 *Returns:* `rdstate() == 0`.

`bool eof() const;`

8 *Returns:* true if eofbit is set in `rdstate()`.

`bool fail() const;`

9 *Returns:* true if failbit or badbit is set in `rdstate()`.<sup>243</sup>

<sup>243</sup> Checking `badbit` also for `fail()` is historical practice.

```
bool bad() const;
```

10     *Returns:* true if badbit is set in rdstate().

```
iosstate exceptions() const;
```

11     *Returns:* A mask that determines what elements set in rdstate() cause exceptions to be thrown.

```
void exceptions(iosstate except);
```

12     *Effects:* Calls clear(rdstate()).

13     *Postconditions:* except == exceptions().

### 31.5.5 ios\_base manipulators

[std.ios.manip]

#### 31.5.5.1 fmtflags manipulators

[fmtflags.manip]

1 Each function specified in this subclause is a designated addressable function (16.4.5.2.1).

```
ios_base& boolalpha(ios_base& str);
```

2     *Effects:* Calls str.setf(ios\_base::boolalpha).

3     *Returns:* str.

```
ios_base& noboolalpha(ios_base& str);
```

4     *Effects:* Calls str.unsetf(ios\_base::boolalpha).

5     *Returns:* str.

```
ios_base& showbase(ios_base& str);
```

6     *Effects:* Calls str.setf(ios\_base::showbase).

7     *Returns:* str.

```
ios_base& noshowbase(ios_base& str);
```

8     *Effects:* Calls str.unsetf(ios\_base::showbase).

9     *Returns:* str.

```
ios_base& showpoint(ios_base& str);
```

10     *Effects:* Calls str.setf(ios\_base::showpoint).

11     *Returns:* str.

```
ios_base& noshowpoint(ios_base& str);
```

12     *Effects:* Calls str.unsetf(ios\_base::showpoint).

13     *Returns:* str.

```
ios_base& showpos(ios_base& str);
```

14     *Effects:* Calls str.setf(ios\_base::showpos).

15     *Returns:* str.

```
ios_base& noshowpos(ios_base& str);
```

16     *Effects:* Calls str.unsetf(ios\_base::showpos).

17     *Returns:* str.

```
ios_base& skipws(ios_base& str);
```

18     *Effects:* Calls str.setf(ios\_base::skipws).

19     *Returns:* str.

```
ios_base& noskipws(ios_base& str);
```

20     *Effects:* Calls str.unsetf(ios\_base::skipws).

21     *Returns:* str.

```
ios_base& uppercase(ios_base& str);
```

22     *Effects:* Calls `str.setf(ios_base::uppercase)`.

23     *Returns:* `str`.

```
ios_base& nouppercase(ios_base& str);
```

24     *Effects:* Calls `str.unsetf(ios_base::uppercase)`.

25     *Returns:* `str`.

```
ios_base& unitbuf(ios_base& str);
```

26     *Effects:* Calls `str.setf(ios_base::unitbuf)`.

27     *Returns:* `str`.

```
ios_base& nunitbuf(ios_base& str);
```

28     *Effects:* Calls `str.unsetf(ios_base::unitbuf)`.

29     *Returns:* `str`.

### 31.5.5.2 adjustfield manipulators

[adjustfield.manip]

1 Each function specified in this subclause is a designated addressable function ([16.4.5.2.1](#)).

```
ios_base& internal(ios_base& str);
```

2     *Effects:* Calls `str.setf(ios_base::internal, ios_base::adjustfield)`.

3     *Returns:* `str`.

```
ios_base& left(ios_base& str);
```

4     *Effects:* Calls `str.setf(ios_base::left, ios_base::adjustfield)`.

5     *Returns:* `str`.

```
ios_base& right(ios_base& str);
```

6     *Effects:* Calls `str.setf(ios_base::right, ios_base::adjustfield)`.

7     *Returns:* `str`.

### 31.5.5.3 basefield manipulators

[basefield.manip]

1 Each function specified in this subclause is a designated addressable function ([16.4.5.2.1](#)).

```
ios_base& dec(ios_base& str);
```

2     *Effects:* Calls `str.setf(ios_base::dec, ios_base::basefield)`.

3     *Returns:* `str`.<sup>244</sup>

```
ios_base& hex(ios_base& str);
```

4     *Effects:* Calls `str.setf(ios_base::hex, ios_base::basefield)`.

5     *Returns:* `str`.

```
ios_base& oct(ios_base& str);
```

6     *Effects:* Calls `str.setf(ios_base::oct, ios_base::basefield)`.

7     *Returns:* `str`.

### 31.5.5.4 floatfield manipulators

[floatfield.manip]

1 Each function specified in this subclause is a designated addressable function ([16.4.5.2.1](#)).

```
ios_base& fixed(ios_base& str);
```

2     *Effects:* Calls `str.setf(ios_base::fixed, ios_base::floatfield)`.

3     *Returns:* `str`.

---

<sup>244</sup>) The function signature `dec(ios_base&)` can be called by the function signature `basic_ostream& stream::operator<<(ios_base& (*)(ios_base&))` to permit expressions of the form `cout << dec` to change the format flags stored in `cout`.



```
ios_base& scientific(ios_base& str);
```

4     *Effects:* Calls `str.setf(ios_base::scientific, ios_base::floatfield)`.

5     *Returns:* `str`.

```
ios_base& hexfloat(ios_base& str);
```

6     *Effects:* Calls `str.setf(ios_base::fixed | ios_base::scientific, ios_base::floatfield)`.

7     *Returns:* `str`.

8     [*Note 1:* `ios_base::hex` cannot be used to specify a hexadecimal floating-point format, because it is not part of `ios_base::floatfield` (Table 137). — *end note*]

```
ios_base& defaultfloat(ios_base& str);
```

9     *Effects:* Calls `str.unsetf(ios_base::floatfield)`.

10    *Returns:* `str`.

### 31.5.6 Error reporting

[error.reporting]

```
error_code make_error_code(io_errc e) noexcept;
```

1     *Returns:* `error_code(static_cast<int>(e), iostream_category())`.

```
error_condition make_error_condition(io_errc e) noexcept;
```

2     *Returns:* `error_condition(static_cast<int>(e), iostream_category())`.

```
const error_category& iostream_category() noexcept;
```

3     *Returns:* A reference to an object of a type derived from class `error_category`.

4     The object's `default_error_condition` and equivalent virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function shall return a pointer to the string "iostream".

## 31.6 Stream buffers

[stream.buffers]

### 31.6.1 Header <streambuf> synopsis

[streambuf.syn]

```
namespace std {
 // 31.6.3, class template basic_streambuf
 template<class charT, class traits = char_traits<charT>>
 class basic_streambuf;
 using streambuf = basic_streambuf<char>;
 using wstreambuf = basic_streambuf<wchar_t>;
}
```

1     The header <streambuf> defines types that control input from and output to *character* sequences.

### 31.6.2 Stream buffer requirements

[streambuf.reqts]

1     Stream buffers can impose various constraints on the sequences they control. Some constraints are:

- (1.1) — The controlled input sequence can be not readable.
  - (1.2) — The controlled output sequence can be not writable.
  - (1.3) — The controlled sequences can be associated with the contents of other representations for character sequences, such as external files.
  - (1.4) — The controlled sequences can support operations *directly* to or from associated sequences.
  - (1.5) — The controlled sequences can impose limitations on how the program can read characters from a sequence, write characters to a sequence, put characters back into an input sequence, or alter the stream position.
- 2     Each sequence is characterized by three pointers which, if non-null, all point into the same `charT` array object. The array object represents, at any moment, a (sub)sequence of characters from the sequence. Operations performed on a sequence alter the values stored in these pointers, perform reads and writes directly to or from associated sequences, and alter “the stream position” and conversion state as needed to maintain this subsequence relationship. The three pointers are:

- (2.1) — the *beginning pointer*, or lowest element address in the array (called **xbeg** here);
  - (2.2) — the *next pointer*, or next element address that is a current candidate for reading or writing (called **xnext** here);
  - (2.3) — the *end pointer*, or first element address beyond the end of the array (called **xend** here).
- <sup>3</sup> The following semantic constraints shall always apply for any set of three pointers for a sequence, using the pointer names given immediately above:
- (3.1) — If **xnext** is not a null pointer, then **xbeg** and **xend** shall also be non-null pointers into the same **charT** array, as described above; otherwise, **xbeg** and **xend** shall also be null.
  - (3.2) — If **xnext** is not a null pointer and **xnext** < **xend** for an output sequence, then a *write position* is available. In this case, **\*xnext** shall be assignable as the next element to write (to put, or to store a character value, into the sequence).
  - (3.3) — If **xnext** is not a null pointer and **xbeg** < **xnext** for an input sequence, then a *putback position* is available. In this case, **xnext[-1]** shall have a defined value and is the next (preceding) element to store a character that is put back into the input sequence.
  - (3.4) — If **xnext** is not a null pointer and **xnext** < **xend** for an input sequence, then a *read position* is available. In this case, **\*xnext** shall have a defined value and is the next element to read (to get, or to obtain a character value, from the sequence).

### 31.6.3 Class template **basic\_streambuf**

[streambuf]

#### 31.6.3.1 General

[streambuf.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_streambuf {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;

 virtual ~basic_streambuf();

 // 31.6.3.3.1, locales
 locale pubimbue(const locale& loc);
 locale getloc() const;

 // 31.6.3.3.2, buffer and positioning
 basic_streambuf* pubsetbuf(char_type* s, streamsize n);
 pos_type pubseekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which
 = ios_base::in | ios_base::out);
 pos_type pubseekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out);
 int pubsync();

 // get and put areas
 // 31.6.3.3.3, get area
 streamsize in_avail();
 int_type snextc();
 int_type sbumpc();
 int_type sgetc();
 streamsize sgetn(char_type* s, streamsize n);

 // 31.6.3.3.4, putback
 int_type sputbackc(char_type c);
 int_type sungetc();
 };

```

```

// 31.6.3.3.5, put area
int_type sputc(char_type c);
streamsize sputn(const char_type* s, streamsize n);

protected:
 basic_streambuf();
 basic_streambuf(const basic_streambuf& rhs);
 basic_streambuf& operator=(const basic_streambuf& rhs);

 void swap(basic_streambuf& rhs);

// 31.6.3.4.2, get area access
char_type* eback() const;
char_type* gptr() const;
char_type* egptr() const;
void gbump(int n);
void setg(char_type* gbeg, char_type* gnext, char_type* gend);

// 31.6.3.4.3, put area access
char_type* pbase() const;
char_type* pptr() const;
char_type* ep_ptr() const;
void pbump(int n);
void setp(char_type* pbeg, char_type* pend);

// 31.6.3.5, virtual functions
// 31.6.3.5.1, locales
virtual void imbue(const locale& loc);

// 31.6.3.5.2, buffer management and positioning
virtual basic_streambuf* setbuf(char_type* s, streamsize n);
virtual pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which
 = ios_base::in | ios_base::out);
virtual pos_type seekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out);
virtual int sync();

// 31.6.3.5.3, get area
virtual streamsize showmanyc();
virtual streamsize xsgetn(char_type* s, streamsize n);
virtual int_type underflow();
virtual int_type uflow();

// 31.6.3.5.4, putback
virtual int_type pbackfail(int_type c = traits::eof());

// 31.6.3.5.5, put area
virtual streamsize xsputn(const char_type* s, streamsize n);
virtual int_type overflow(int_type c = traits::eof());
};
}

```

<sup>1</sup> The class template `basic_streambuf` serves as a base class for deriving various *stream buffers* whose objects each control two *character sequences*:

- (1.1) — a character *input sequence*;
- (1.2) — a character *output sequence*.

### 31.6.3.2 Constructors

[streambuf.cons]

```
basic_streambuf();
```

- 1 *Effects:* Initializes:<sup>245</sup>
- (1.1) — all pointer member objects to null pointers,
  - (1.2) — the `getloc()` member to a copy of the global locale, `locale()`, at the time of construction.
- 2 *Remarks:* Once the `getloc()` member is initialized, results of calling locale member functions, and of members of facets so obtained, can safely be cached until the next time the member `imbue` is called.

```
basic_streambuf(const basic_streambuf& rhs);
```

- 3 *Postconditions:*
- (3.1) — `eback() == rhs.eback()`
  - (3.2) — `gptr() == rhs.gptr()`
  - (3.3) — `egptr() == rhs.egptr()`
  - (3.4) — `pbase() == rhs.pbase()`
  - (3.5) — `pptr() == rhs.pptr()`
  - (3.6) — `epptr() == rhs.epptr()`
  - (3.7) — `getloc() == rhs.getloc()`

```
~basic_streambuf();
```

- 4 *Effects:* None.

### 31.6.3.3 Public member functions

[streambuf.members]

#### 31.6.3.3.1 Locales

[streambuf.locales]

```
locale pubimbue(const locale& loc);
```

- 1 *Effects:* Calls `imbue(loc)`.
- 2 *Postconditions:* `loc == getloc()`.
- 3 *Returns:* Previous value of `getloc()`.

```
locale getloc() const;
```

- 4 *Returns:* If `pubimbue()` has ever been called, then the last value of `loc` supplied, otherwise the current global locale, `locale()`, in effect at the time of construction. If called after `pubimbue()` has been called but before `pubimbue` has returned (i.e., from within the call of `imbue()`) then it returns the previous value.

#### 31.6.3.3.2 Buffer management and positioning

[streambuf.buffer]

```
basic_streambuf* pubsetbuf(char_type* s, streamsize n);
```

- 1 *Returns:* `setbuf(s, n)`.
- ```
pos_type pubseekoff(off_type off, ios_base::seekdir way,
                    ios_base::openmode which
                    = ios_base::in | ios_base::out);
```
- 2 *Returns:* `seekoff(off, way, which)`.

```
pos_type pubseekpos(pos_type sp,
                    ios_base::openmode which
                    = ios_base::in | ios_base::out);
```

- 3 *Returns:* `seekpos(sp, which)`.

```
int pubsync();
```

- 4 *Returns:* `sync()`.

245) The default constructor is protected for class `basic_streambuf` to assure that only objects for classes derived from this class can be constructed.

31.6.3.3.3 Get area**[streambuf.pub.get]**

streamsize in_avail();

- 1 *Returns:* If a read position is available, returns `egptr() - gptr()`. Otherwise returns `showmanyc()` (31.6.3.5.3).

int_type snextc();

- 2 *Effects:* Calls `sbumpc()`.

- 3 *Returns:* If that function returns `traits::eof()`, returns `traits::eof()`. Otherwise, returns `sgetc()`.

int_type sbumpc();

- 4 *Effects:* If the input sequence read position is not available, returns `uflow()`. Otherwise, returns `traits::to_int_type(*gptr())` and increments the next pointer for the input sequence.

int_type sgetc();

- 5 *Returns:* If the input sequence read position is not available, returns `underflow()`. Otherwise, returns `traits::to_int_type(*gptr())`.

streamsize sgetn(char_type* s, streamsize n);

- 6 *Returns:* `xsgetn(s, n)`.

31.6.3.3.4 Putback**[streambuf.pub.pback]**

int_type sputbackc(char_type c);

- 1 *Effects:* If the input sequence putback position is not available, or if `traits::eq(c, gptr()[-1])` is false, returns `pbackfail(traits::to_int_type(c))`. Otherwise, decrements the next pointer for the input sequence and returns `traits::to_int_type(*gptr())`.

int_type sungetc();

- 2 *Effects:* If the input sequence putback position is not available, returns `pbackfail()`. Otherwise, decrements the next pointer for the input sequence and returns `traits::to_int_type(*gptr())`.

31.6.3.3.5 Put area**[streambuf.pub.put]**

int_type sputc(char_type c);

- 1 *Effects:* If the output sequence write position is not available, returns `overflow(traits::to_int_type(c))`. Otherwise, stores `c` at the next pointer for the output sequence, increments the pointer, and returns `traits::to_int_type(c)`.

streamsize sputn(const char_type* s, streamsize n);

- 2 *Returns:* `xspun(s, n)`.

31.6.3.4 Protected member functions**[streambuf.protected]****31.6.3.4.1 Assignment****[streambuf.assign]**

basic_streambuf& operator=(const basic_streambuf& rhs);

- 1 *Postconditions:*

- (1.1) — `eback() == rhs.eback()`
- (1.2) — `gptr() == rhs.gptr()`
- (1.3) — `egptr() == rhs.egptr()`
- (1.4) — `pbase() == rhs.pbase()`
- (1.5) — `pptr() == rhs.pptr()`
- (1.6) — `epptr() == rhs.epptr()`
- (1.7) — `getloc() == rhs.getloc()`

- 2 *Returns:* `*this`.

```
void swap(basic_streambuf& rhs);
```

3 *Effects:* Swaps the data members of `rhs` and `*this`.

31.6.3.4.2 Get area access

[streambuf.get.area]

```
char_type* eback() const;
```

1 *Returns:* The beginning pointer for the input sequence.

```
char_type* gptr() const;
```

2 *Returns:* The next pointer for the input sequence.

```
char_type* egptr() const;
```

3 *Returns:* The end pointer for the input sequence.

```
void gbump(int n);
```

4 *Effects:* Adds `n` to the next pointer for the input sequence.

```
void setg(char_type* gbeg, char_type* gnext, char_type* gend);
```

5 *Preconditions:* `[gbeg, gnext)`, `[gbeg, gend)`, and `[gnext, gend)` are all valid ranges.

6 *Postconditions:* `gbeg == eback()`, `gnext == gptr()`, and `gend == egptr()` are all true.

31.6.3.4.3 Put area access

[streambuf.put.area]

```
char_type* pbase() const;
```

1 *Returns:* The beginning pointer for the output sequence.

```
char_type* pptr() const;
```

2 *Returns:* The next pointer for the output sequence.

```
char_type* epptr() const;
```

3 *Returns:* The end pointer for the output sequence.

```
void pbump(int n);
```

4 *Effects:* Adds `n` to the next pointer for the output sequence.

```
void setp(char_type* pbeg, char_type* pend);
```

5 *Preconditions:* `[pbeg, pend)` is a valid range.

6 *Postconditions:* `pbeg == pbase()`, `pbeg == pptr()`, and `pend == epptr()` are all true.

31.6.3.5 Virtual functions

[streambuf.virtuals]

31.6.3.5.1 Locales

[streambuf.virt.locales]

```
void imbue(const locale&);
```

1 *Effects:* Change any translations based on locale.

2 *Remarks:* Allows the derived class to be informed of changes in locale at the time they occur. Between invocations of this function a class derived from `streambuf` can safely cache results of calls to locale functions and to members of facets so obtained.

3 *Default behavior:* Does nothing.

31.6.3.5.2 Buffer management and positioning

[streambuf.virt.buffer]

```
basic_streambuf* setbuf(char_type* s, streamsize n);
```

1 *Effects:* Influences stream buffering in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (31.8.2.5, 31.10.3.5).

2 *Default behavior:* Does nothing. Returns `this`.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
                ios_base::openmode which
                = ios_base::in | ios_base::out);
```

3 *Effects:* Alters the stream positions within one or more of the controlled sequences in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (31.8.2.5, 31.10.3.5).

4 *Default behavior:* Returns `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp,
                ios_base::openmode which
                = ios_base::in | ios_base::out);
```

5 *Effects:* Alters the stream positions within one or more of the controlled sequences in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (31.8.2, 31.10.3).

6 *Default behavior:* Returns `pos_type(off_type(-1))`.

```
int sync();
```

7 *Effects:* Synchronizes the controlled sequences with the arrays. That is, if `pbase()` is non-null the characters between `pbase()` and `pptr()` are written to the controlled sequence. The pointers may then be reset as appropriate.

8 *Returns:* `-1` on failure. What constitutes failure is determined by each derived class (31.10.3.5).

9 *Default behavior:* Returns zero.

31.6.3.5.3 Get area

[`streambuf.virt.get`]

```
streamsize showmanyc();246
```

1 *Returns:* An estimate of the number of characters available in the sequence, or `-1`. If it returns a positive value, then successive calls to `underflow()` will not return `traits::eof()` until at least that number of characters have been extracted from the stream. If `showmanyc()` returns `-1`, then calls to `underflow()` or `uflow()` will fail.²⁴⁷

2 *Default behavior:* Returns zero.

3 *Remarks:* Uses `traits::eof()`.

```
streamsize xsgetn(char_type* s, streamsize n);
```

4 *Effects:* Assigns up to `n` characters to successive elements of the array whose first element is designated by `s`. The characters assigned are read from the input sequence as if by repeated calls to `sbumpc()`. Assigning stops when either `n` characters have been assigned or a call to `sbumpc()` would return `traits::eof()`.

5 *Returns:* The number of characters assigned.²⁴⁸

6 *Remarks:* Uses `traits::eof()`.

```
int_type underflow();
```

7 The *pending sequence* of characters is defined as the concatenation of

(7.1) — the empty sequence if `gptr()` is null, otherwise the characters in `[gptr(), egptr())`, followed by

(7.2) — some (possibly empty) sequence of characters read from the input sequence.

8 The *result character* is the first character of the pending sequence if it is non-empty, otherwise the next character that would be read from the input sequence.

9 The *backup sequence* is the empty sequence if `eback()` is null, otherwise the characters in `[eback(), gptr())`.

10 *Effects:* The function sets up the `gptr()` and `egptr()` such that if the pending sequence is non-empty, then `egptr()` is non-null and the characters in `[gptr(), egptr())` are the characters in the pending sequence, otherwise either `gptr()` is null or `gptr() == egptr()`.

²⁴⁶) The morphemes of `showmanyc` are “es-how-many-see”, not “show-manic”.

²⁴⁷) `underflow` or `uflow` can fail by throwing an exception prematurely. The intention is not only that the calls will not return `eof()` but that they will return “immediately”.

²⁴⁸) Classes derived from `basic_streambuf` can provide more efficient ways to implement `xsgetn()` and `xsputn()` by overriding these definitions from the base class.

If `eback()` and `gptr()` are non-null then the function is not constrained as to their contents, but the “usual backup condition” is that either

- (11.1) — the backup sequence contains at least `gptr() - eback()` characters, in which case the characters in `[eback(), gptr())` agree with the last `gptr() - eback()` characters of the backup sequence, or
- (11.2) — the characters in `[gptr() - n, gptr())` agree with the backup sequence (where `n` is the length of the backup sequence).

Returns: `traits::to_int_type(c)`, where `c` is the first *character* of the *pending sequence*, without moving the input sequence position past it. If the pending sequence is null then the function returns `traits::eof()` to indicate failure.

Default behavior: Returns `traits::eof()`.

Remarks: The public members of `basic_streambuf` call this virtual function only if `gptr()` is null or `gptr() >= egptr()`.

`int_type uflow();`

Preconditions: The constraints are the same as for `underflow()`, except that the result character is transferred from the pending sequence to the backup sequence, and the pending sequence is not empty before the transfer.

Default behavior: Calls `underflow()`. If `underflow()` returns `traits::eof()`, returns `traits::eof()`. Otherwise, returns the value of `traits::to_int_type(*gptr())` and increments the value of the next pointer for the input sequence.

Returns: `traits::eof()` to indicate failure.

31.6.3.5.4 Putback

[`streambuf.virt.pback`]

`int_type pbackfail(int_type c = traits::eof());`

The *pending sequence* is defined as for `underflow()`, with the modifications that

- (1.1) — If `traits::eq_int_type(c, traits::eof())` returns `true`, then the input sequence is backed up one character before the pending sequence is determined.
- (1.2) — If `traits::eq_int_type(c, traits::eof())` returns `false`, then `c` is prepended. Whether the input sequence is backed up or modified in any other way is unspecified.

Postconditions: On return, the constraints of `gptr()`, `eback()`, and `pptr()` are the same as for `underflow()`.

Returns: `traits::eof()` to indicate failure. Failure may occur because the input sequence could not be backed up, or if for some other reason the pointers cannot be set consistent with the constraints. `pbackfail()` is called only when put back has really failed.

Returns some value other than `traits::eof()` to indicate success.

Default behavior: Returns `traits::eof()`.

Remarks: The public functions of `basic_streambuf` call this virtual function only when `gptr()` is null, `gptr() == eback()`, or `traits::eq(traits::to_char_type(c), gptr()[-1])` returns `false`. Other calls shall also satisfy that constraint.

31.6.3.5.5 Put area

[`streambuf.virt.put`]

`streamsize xsputn(const char_type* s, streamsize n);`

Effects: Writes up to `n` characters to the output sequence as if by repeated calls to `sputc(c)`. The characters written are obtained from successive elements of the array whose first element is designated by `s`. Writing stops when either `n` characters have been written or a call to `sputc(c)` would return `traits::eof()`. It is unspecified whether the function calls `overflow()` when `pptr() == epptr()` becomes `true` or whether it achieves the same effects by other means.

Returns: The number of characters written.

`int_type overflow(int_type c = traits::eof());`

Effects: Consumes some initial subsequence of the characters of the *pending sequence*. The pending sequence is defined as the concatenation of

- (3.1) — the empty sequence if `pbase()` is null, otherwise the `pptr()` - `pbase()` characters beginning at `pbase()`, followed by
 - (3.2) — the empty sequence if `traits::eq_int_type(c, traits::eof())` returns `true`, otherwise the sequence consisting of `c`.
- 4 *Preconditions:* Every overriding definition of this virtual function obeys the following constraints:
- (4.1) — The effect of consuming a character on the associated output sequence is specified.²⁴⁹
 - (4.2) — Let `r` be the number of characters in the pending sequence not consumed. If `r` is nonzero then `pbase()` and `pptr()` are set so that: `pptr() - pbase() == r` and the `r` characters starting at `pbase()` are the associated output stream. In case `r` is zero (all characters of the pending sequence have been consumed) then either `pbase()` is set to `nullptr`, or `pbase()` and `pptr()` are both set to the same non-null value.
 - (4.3) — The function may fail if either appending some character to the associated output stream fails or if it is unable to establish `pbase()` and `pptr()` according to the above rules.
- 5 *Returns:* `traits::eof()` or throws an exception if the function fails.
Otherwise, returns some value other than `traits::eof()` to indicate success.²⁵⁰
- 6 *Default behavior:* Returns `traits::eof()`.
- 7 *Remarks:* The member functions `sputc()` and `sputn()` call this function in case that no room can be found in the put buffer enough to accommodate the argument character sequence.

31.7 Formatting and manipulators

[iostream.format]

31.7.1 Header <istream> synopsis

[istream.syn]

```

namespace std {
    // 31.7.5.2, class template basic_istream
    template<class charT, class traits = char_traits<charT>>
        class basic_istream;

    using istream = basic_istream<char>;
    using wistream = basic_istream<wchar_t>;

    // 31.7.5.7, class template basic_iostream
    template<class charT, class traits = char_traits<charT>>
        class basic_iostream;

    using iostream = basic_iostream<char>;
    using wiostream = basic_iostream<wchar_t>;

    // 31.7.5.5, standard basic_istream manipulators
    template<class charT, class traits>
        basic_istream<charT, traits>& ws(basic_istream<charT, traits>& is);

    // 31.7.5.6, rvalue stream extraction
    template<class Istream, class T>
        Istream&& operator>>(Istream&& is, T&& x);
}

```

31.7.2 Header <ostream> synopsis

[ostream.syn]

```

namespace std {
    // 31.7.6.2, class template basic_ostream
    template<class charT, class traits = char_traits<charT>>
        class basic_ostream;

    using ostream = basic_ostream<char>;
    using wostream = basic_ostream<wchar_t>;

```

249) That is, for each class derived from a specialization of `basic_streambuf` in this Clause (31.8.2, 31.10.3), a specification of how consuming a character affects the associated output sequence is given. There is no requirement on a program-defined class.

250) Typically, `overflow` returns `c` to indicate success, except when `traits::eq_int_type(c, traits::eof())` returns `true`, in which case it returns `traits::not_eof(c)`.

```

// 31.7.6.5, standard basic_ostream manipulators
template<class charT, class traits>
    basic_ostream<charT, traits>& endl(basic_ostream<charT, traits>& os);
template<class charT, class traits>
    basic_ostream<charT, traits>& ends(basic_ostream<charT, traits>& os);
template<class charT, class traits>
    basic_ostream<charT, traits>& flush(basic_ostream<charT, traits>& os);

template<class charT, class traits>
    basic_ostream<charT, traits>& emit_on_flush(basic_ostream<charT, traits>& os);
template<class charT, class traits>
    basic_ostream<charT, traits>& noemit_on_flush(basic_ostream<charT, traits>& os);
template<class charT, class traits>
    basic_ostream<charT, traits>& flush_emit(basic_ostream<charT, traits>& os);

// 31.7.6.6, rvalue stream insertion
template<class Ostream, class T>
    Ostream&& operator<< (Ostream&& os, const T& x);

// 31.7.6.3.5, print functions
template<class... Args>
    void print(ostream& os, format_string<Args...> fmt, Args&&... args);
template<class... Args>
    void println(ostream& os, format_string<Args...> fmt, Args&&... args);
void println(ostream& os);

void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
}

```

31.7.3 Header <iomanip> synopsis

[iomanip.syn]

```

namespace std {
// 31.7.7, standard manipulators
unspecified resetiosflags(ios_base::fmtflags mask);
unspecified setiosflags (ios_base::fmtflags mask);
unspecified setbase(int base);
template<class charT> unspecified setfill(charT c);
unspecified setprecision(int n);
unspecified setw(int n);

// 31.7.8, extended manipulators
template<class moneyT> unspecified get_money(moneyT& mon, bool intl = false);
template<class moneyT> unspecified put_money(const moneyT& mon, bool intl = false);
template<class charT> unspecified get_time(tm* tmb, const charT* fmt);
template<class charT> unspecified put_time(const tm* tmb, const charT* fmt);

// 31.7.9, quoted manipulators
template<class charT>
    unspecified quoted(const charT* s, charT delim = charT(''), charT escape = charT('\\'));

template<class charT, class traits, class Allocator>
    unspecified quoted(const basic_string<charT, traits, Allocator>& s,
        charT delim = charT(''), charT escape = charT('\\'));

template<class charT, class traits, class Allocator>
    unspecified quoted(basic_string<charT, traits, Allocator>& s,
        charT delim = charT(''), charT escape = charT('\\'));

template<class charT, class traits>
    unspecified quoted(basic_string_view<charT, traits> s,
        charT delim = charT(''), charT escape = charT('\\'));
}

```

31.7.4 Header <print> synopsis

[print.syn]

```

namespace std {
    // 31.7.10, print functions
    template<class... Args>
        void print(format_string<Args...> fmt, Args&&... args);
    template<class... Args>
        void print(FILE* stream, format_string<Args...> fmt, Args&&... args);

    template<class... Args>
        void println(format_string<Args...> fmt, Args&&... args);
    void println();
    template<class... Args>
        void println(FILE* stream, format_string<Args...> fmt, Args&&... args);
    void println(FILE* stream);

    void vprint_unicode(string_view fmt, format_args args);
    void vprint_unicode(FILE* stream, string_view fmt, format_args args);
    void vprint_unicode_buffered(FILE* stream, string_view fmt, format_args args);

    void vprint_nonunicode(string_view fmt, format_args args);
    void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
    void vprint_nonunicode_buffered(FILE* stream, string_view fmt, format_args args);
}

```

31.7.5 Input streams

[input.streams]

31.7.5.1 General

[input.streams.general]

- ¹ The header <istream> defines two class templates and a function template that control input from a stream buffer, along with a function template that extracts from stream rvalues.

31.7.5.2 Class template basic_istream

[istream]

31.7.5.2.1 General

[istream.general]

- ¹ When a function is specified with a type placeholder of *extended-floating-point-type*, the implementation provides overloads for all cv-unqualified extended floating-point types (6.9.3) in lieu of *extended-floating-point-type*.

```

namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_istream : virtual public basic_ios<charT, traits> {
    public:
        // types (inherited from basic_ios (31.5.4))
        using char_type    = charT;
        using int_type      = traits::int_type;
        using pos_type      = traits::pos_type;
        using off_type      = traits::off_type;
        using traits_type   = traits;

        // 31.7.5.2.2, constructor/destructor
        explicit basic_istream(basic_streambuf<charT, traits>* sb);
        virtual ~basic_istream();

        // 31.7.5.2.4, prefix/suffix
        class sentry;

        // 31.7.5.3, formatted input
        basic_istream& operator>>(basic_istream& (*pf)(basic_istream&));
        basic_istream& operator>>(basic_ios<charT, traits>& (*pf)(basic_ios<charT, traits>&));
        basic_istream& operator>>(ios_base& (*pf)(ios_base&));

        basic_istream& operator>>(bool& n);
        basic_istream& operator>>(short& n);
        basic_istream& operator>>(unsigned short& n);
        basic_istream& operator>>(int& n);
    };
}

```

```

basic_istream& operator>>(unsigned int& n);
basic_istream& operator>>(long& n);
basic_istream& operator>>(unsigned long& n);
basic_istream& operator>>(long long& n);
basic_istream& operator>>(unsigned long long& n);
basic_istream& operator>>(float& f);
basic_istream& operator>>(double& f);
basic_istream& operator>>(long double& f);
basic_istream& operator>>(extended-floating-point-type& f);

basic_istream& operator>>(void*& p);
basic_istream& operator>>(basic_streambuf<char_type, traits>* sb);

// 31.7.5.4, unformatted input
streamsize gcount() const;
int_type get();
basic_istream& get(char_type& c);
basic_istream& get(char_type* s, streamsize n);
basic_istream& get(char_type* s, streamsize n, char_type delim);
basic_istream& get(basic_streambuf<char_type, traits>& sb);
basic_istream& get(basic_streambuf<char_type, traits>& sb, char_type delim);

basic_istream& getline(char_type* s, streamsize n);
basic_istream& getline(char_type* s, streamsize n, char_type delim);

basic_istream& ignore(streamsize n = 1, int_type delim = traits::eof());
basic_istream& ignore(streamsize n, char_type delim);
int_type peek();
basic_istream& read (char_type* s, streamsize n);
streamsize readsome(char_type* s, streamsize n);

basic_istream& putback(char_type c);
basic_istream& unget();
int sync();

pos_type tellg();
basic_istream& seekg(pos_type);
basic_istream& seekg(off_type, ios_base::seekdir);

protected:
// 31.7.5.2.2, copy/move constructor
basic_istream(const basic_istream&) = delete;
basic_istream(basic_istream&& rhs);

// 31.7.5.2.3, assignment and swap
basic_istream& operator=(const basic_istream&) = delete;
basic_istream& operator=(basic_istream&& rhs);
void swap(basic_istream& rhs);
};

// 31.7.5.3.3, character extraction templates
template<class charT, class traits>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, charT&);

template<class charT, class traits, size_t N>
    basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, charT(&)[N]);
}

```

- ² The class template `basic_istream` defines a number of member function signatures that assist in reading and interpreting input from sequences controlled by a stream buffer.
- ³ Two groups of member function signatures share common properties: the *formatted input functions* (or *extractors*) and the *unformatted input functions*. Both groups of input functions are described as if they

obtain (or *extract*) input *characters* by calling `rdbuf()->sbumpc()` or `rdbuf()->sgetc()`. They may use other public members of `istream`.

31.7.5.2.2 Constructors

[istream.cons]

```
explicit basic_istream(basic_streambuf<charT, traits>* sb);
```

1 *Effects*: Initializes the base class subobject with `basic_ios::init(sb)` (31.5.4.2).

2 *Postconditions*: `gcount() == 0`.

```
basic_istream(basic_istream&& rhs);
```

3 *Effects*: Default constructs the base class, copies the `gcount()` from `rhs`, calls `basic_ios<charT, traits>::move(rhs)` to initialize the base class, and sets the `gcount()` for `rhs` to 0.

```
virtual ~basic_istream();
```

4 *Remarks*: Does not perform any operations of `rdbuf()`.

31.7.5.2.3 Assignment and swap

[istream.assign]

```
basic_istream& operator=(basic_istream&& rhs);
```

1 *Effects*: Equivalent to `swap(rhs)`.

2 *Returns*: `*this`.

```
void swap(basic_istream& rhs);
```

3 *Effects*: Calls `basic_ios<charT, traits>::swap(rhs)`. Exchanges the values returned by `gcount()` and `rhs.gcount()`.

31.7.5.2.4 Class `basic_istream::sentry`

[istream.sentry]

```
namespace std {
    template<class charT, class traits>
    class basic_istream<charT, traits>::sentry {
        bool ok_;
        // exposition only

    public:
        explicit sentry(basic_istream& is, bool noskipws = false);
        ~sentry();
        explicit operator bool() const { return ok_; }
        sentry(const sentry&) = delete;
        sentry& operator=(const sentry&) = delete;
    };
}
```

1 The class `sentry` defines a class that is responsible for doing exception safe prefix and suffix operations.

```
explicit sentry(basic_istream& is, bool noskipws = false);
```

2 *Effects*: If `is.good()` is false, calls `is.setstate(failbit)`. Otherwise, prepares for formatted or unformatted input. First, if `is.tie()` is not a null pointer, the function calls `is.tie()->flush()` to synchronize the output sequence with any associated external C stream. Except that this call can be suppressed if the put area of `is.tie()` is empty. Further an implementation is allowed to defer the call to `flush` until a call of `is.rdbuf()->underflow()` occurs. If no such call occurs before the `sentry` object is destroyed, the call to `flush` may be eliminated entirely.²⁵¹ If `noskipws` is zero and `is.flags() & ios_base::skipws` is nonzero, the function extracts and discards each character as long as the next available input character `c` is a whitespace character. If `is.rdbuf()->sbumpc()` or `is.rdbuf()->sgetc()` returns `traits::eof()`, the function calls `setstate(failbit | eofbit)` (which may throw `ios_base::failure`).

3 *Remarks*: The constructor

```
explicit sentry(basic_istream& is, bool noskipws = false)
```

²⁵¹) This will be possible only in functions that are part of the library. The semantics of the constructor used in user code is as specified.

uses the currently imbued locale in `is`, to determine whether the next input character is whitespace or not.

- 4 To decide if the character `c` is a whitespace character, the constructor performs as if it executes the following code fragment:

```
const ctype<charT>& ctype = use_facet<ctype<charT>>(is.getloc());
if (ctype.is(ctype.space, c) != 0)
    // c is a whitespace character.
```

- 5 If, after any preparation is completed, `is.good()` is `true`, `ok_ != false` otherwise, `ok_ == false`. During preparation, the constructor may call `setstate(failbit)` (which may throw `ios_base::failure` (31.5.4.4)).²⁵²

```
~sentry();
```

- 6 *Effects:* None.

```
explicit operator bool() const;
```

- 7 *Returns:* `ok_`.

31.7.5.3 Formatted input functions

[istream.formatted]

31.7.5.3.1 Common requirements

[istream.formatted.reqmts]

- 1 Each formatted input function begins execution by constructing an object of type `ios_base::iostate`, termed the local error state, and initializing it to `ios_base::goodbit`. It then creates an object of class `sentry` with the `noskipws` (second) argument `false`. If the `sentry` object returns `true`, when converted to a value of type `bool`, the function endeavors to obtain the requested input. Otherwise, if the `sentry` constructor exits by throwing an exception or if the `sentry` object produces `false` when converted to a value of type `bool`, the function returns without attempting to obtain any input. If `rdbuf()->sbumpc()` or `rdbuf()->sgetc()` returns `traits::eof()`, then `ios_base::eofbit` is set in the local error state and the input function stops trying to obtain the requested input. If an exception is thrown during input then `ios_base::badbit` is set in the local error state, `*this`'s error state is set to the local error state, and the exception is rethrown if `(exceptions() & badbit) != 0`. After extraction is done, the input function calls `setstate`, which sets `*this`'s error state to the local error state, and may throw an exception. In any case, the formatted input function destroys the `sentry` object. If no exception has been thrown, it returns `*this`.

31.7.5.3.2 Arithmetic extractors

[istream.formatted.arithmetic]

```
basic_istream& operator>>(unsigned short& val);
basic_istream& operator>>(unsigned int& val);
basic_istream& operator>>(long& val);
basic_istream& operator>>(unsigned long& val);
basic_istream& operator>>(long long& val);
basic_istream& operator>>(unsigned long long& val);
basic_istream& operator>>(float& val);
basic_istream& operator>>(double& val);
basic_istream& operator>>(long double& val);
basic_istream& operator>>(bool& val);
basic_istream& operator>>(void*& val);
```

- 1 As in the case of the inserters, these extractors depend on the locale's `num_get<>` (28.3.4.3.2) object to perform parsing the input stream data. These extractors behave as formatted input functions (as described in 31.7.5.3.1). After a `sentry` object is constructed, the conversion occurs as if performed by the following code fragment, where `state` represents the input function's local error state:

```
using numget = num_get<charT, istreambuf_iterator<charT, traits>>;
use_facet<numget>(loc).get(*this, 0, *this, state, val);
```

In the above fragment, `loc` stands for the private member of the `basic_ios` class.

[Note 1: The first argument provides an object of the `istreambuf_iterator` class which is an iterator pointed to an input stream. It bypasses istreams and uses streambufs directly. — end note]

Class `locale` relies on this type as its interface to `istream`, so that it does not need to depend directly on `istream`.

²⁵²) The `sentry` constructor and destructor can also perform additional implementation-dependent operations.

```
basic_istream& operator>>(short& val);
```

- 2 The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
using numget = num_get<charT, istreambuf_iterator<charT, traits>>;
long lval;
use_facet<numget>(loc).get(*this, 0, *this, state, lval);
if (lval < numeric_limits<short>::min()) {
    state |= ios_base::failbit;
    val = numeric_limits<short>::min();
} else if (numeric_limits<short>::max() < lval) {
    state |= ios_base::failbit;
    val = numeric_limits<short>::max();
} else
    val = static_cast<short>(lval);
```

```
basic_istream& operator>>(int& val);
```

- 3 The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
using numget = num_get<charT, istreambuf_iterator<charT, traits>>;
long lval;
use_facet<numget>(loc).get(*this, 0, *this, state, lval);
if (lval < numeric_limits<int>::min()) {
    state |= ios_base::failbit;
    val = numeric_limits<int>::min();
} else if (numeric_limits<int>::max() < lval) {
    state |= ios_base::failbit;
    val = numeric_limits<int>::max();
} else
    val = static_cast<int>(lval);
```

```
basic_istream& operator>>(extended-floating-point-type& val);
```

- 4 If the floating-point conversion rank of *extended-floating-point-type* is not less than or equal to that of `long double`, then an invocation of the operator function is conditionally supported with implementation-defined semantics.

- 5 Otherwise, let FP be a standard floating-point type:

- (5.1) — if the floating-point conversion rank of *extended-floating-point-type* is less than or equal to that of `float`, then FP is `float`,
- (5.2) — otherwise, if the floating-point conversion rank of *extended-floating-point-type* is less than or equal to that of `double`, then FP is `double`,
- (5.3) — otherwise, FP is `long double`.

- 6 The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
using numget = num_get<charT, istreambuf_iterator<charT, traits>>;
FP fval;
use_facet<numget>(loc).get(*this, 0, *this, state, fval);
if (fval < -numeric_limits<extended-floating-point-type>::max()) {
    state |= ios_base::failbit;
    val = -numeric_limits<extended-floating-point-type>::max();
} else if (numeric_limits<extended-floating-point-type>::max() < fval) {
    state |= ios_base::failbit;
    val = numeric_limits<extended-floating-point-type>::max();
} else {
    val = static_cast<extended-floating-point-type>(fval);
}
```

[Note 2: When the extended floating-point type has a floating-point conversion rank that is not equal to the rank of any standard floating-point type, then double rounding during the conversion can result in inaccurate results. `from_chars` can be used in situations where maximum accuracy is important. — end note]

31.7.5.3.3 `basic_istream::operator>>`

[istream.extractors]

```
basic_istream& operator>>(basic_istream& (*pf)(basic_istream&));
```

1 *Effects:* None. This extractor does not behave as a formatted input function (as described in 31.7.5.3.1).

2 *Returns:* `pf(*this)`.²⁵³

```
basic_istream& operator>>(basic_ios<charT, traits>& (*pf)(basic_ios<charT, traits>&));
```

3 *Effects:* Calls `pf(*this)`. This extractor does not behave as a formatted input function (as described in 31.7.5.3.1).

4 *Returns:* `*this`.

```
basic_istream& operator>>(ios_base& (*pf)(ios_base&));
```

5 *Effects:* Calls `pf(*this)`.²⁵⁴ This extractor does not behave as a formatted input function (as described in 31.7.5.3.1).

6 *Returns:* `*this`.

```
template<class charT, class traits, size_t N>
```

```
basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>& in, charT (&s)[N]);
```

7 *Effects:* Behaves like a formatted input member (as described in 31.7.5.3.1) of `in`. After a `sentry` object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

8 Characters are extracted and stored until any of the following occurs:

- (8.1) — `n - 1` characters are stored;
- (8.2) — end of file occurs on the input sequence;
- (8.3) — letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, c)` is true.

9 `operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

10 If the function extracted no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

11 *Returns:* `in`.

```
template<class charT, class traits>
```

```
basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>& in, charT& c);
```

12 *Effects:* Behaves like a formatted input member (as described in 31.7.5.3.1) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

13 *Returns:* `in`.

```
basic_istream& operator>>(basic_streambuf<charT, traits>* sb);
```

14 *Effects:* Behaves as an unformatted input function (31.7.5.4). If `sb` is null, calls `setstate(failbit)`, which may throw `ios_base::failure` (31.5.4.4). After a `sentry` object is constructed, extracts characters from `*this` and inserts them in the output sequence controlled by `sb`. Characters are extracted and inserted until any of the following occurs:

- (14.1) — end-of-file occurs on the input sequence;
- (14.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);
- (14.3) — an exception occurs (in which case the exception is caught).

15 If the function inserts no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

16 *Returns:* `*this`.

²⁵³) See, for example, the function signature `ws(basic_istream&)` (31.7.5.5).

²⁵⁴) See, for example, the function signature `dec(ios_base&)` (31.5.5.3).

31.7.5.4 Unformatted input functions

[istream.unformatted]

- ¹ Each unformatted input function begins execution by constructing an object of type `ios_base::iostate`, termed the local error state, and initializing it to `ios_base::goodbit`. It then creates an object of class `sentry` with the default argument `noskipws` (second) argument `true`. If the `sentry` object returns `true`, when converted to a value of type `bool`, the function endeavors to obtain the requested input. Otherwise, if the `sentry` constructor exits by throwing an exception or if the `sentry` object produces `false`, when converted to a value of type `bool`, the function returns without attempting to obtain any input. In either case the number of extracted characters is set to 0; unformatted input functions taking a character array of nonzero size as an argument shall also store a null character (using `charT()`) in the first location of the array. If `rdbuf()->sbumpc()` or `rdbuf()->sgetc()` returns `traits::eof()`, then `ios_base::eofbit` is set in the local error state and the input function stops trying to obtain the requested input. If an exception is thrown during input then `ios_base::badbit` is set in the local error state, `*this`'s error state is set to the local error state, and the exception is rethrown if `(exceptions() & badbit) != 0`. If no exception has been thrown it stores the number of characters extracted in a member object. After extraction is done, the input function calls `setstate`, which sets `*this`'s error state to the local error state, and may throw an exception. In any event the `sentry` object is destroyed before leaving the unformatted input function.

```
streamsize gcount() const;
```

- ² *Effects:* None. This member function does not behave as an unformatted input function (as described above).

- ³ *Returns:* The number of characters extracted by the last unformatted input member function called for the object. If the number cannot be represented, returns `numeric_limits<streamsize>::max()`.

```
int_type get();
```

- ⁴ *Effects:* Behaves as an unformatted input function (as described above). After constructing a `sentry` object, extracts a character `c`, if one is available. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

- ⁵ *Returns:* `c` if available, otherwise `traits::eof()`.

```
basic_istream& get(char_type& c);
```

- ⁶ *Effects:* Behaves as an unformatted input function (as described above). After constructing a `sentry` object, extracts a character, if one is available, and assigns it to `c`.²⁵⁵ Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

- ⁷ *Returns:* `*this`.

```
basic_istream& get(char_type* s, streamsize n, char_type delim);
```

- ⁸ *Effects:* Behaves as an unformatted input function (as described above). After constructing a `sentry` object, extracts characters and stores them into successive locations of an array whose first element is designated by `s`.²⁵⁶ Characters are extracted and stored until any of the following occurs:

- (8.1) — `n` is less than one or `n - 1` characters are stored;
- (8.2) — end-of-file occurs on the input sequence;
- (8.3) — `traits::eq(c, delim)` for the next available input character `c` (in which case `c` is not extracted).

- ⁹ If the function stores no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called. In any case, if `n` is greater than zero it then stores a null character into the next successive location of the array.

- ¹⁰ *Returns:* `*this`.

```
basic_istream& get(char_type* s, streamsize n);
```

- ¹¹ *Effects:* Calls `get(s, n, widen(' \n '))`.

- ¹² *Returns:* Value returned by the call.

²⁵⁵) Note that this function is not overloaded on types `signed char` and `unsigned char`.

²⁵⁶) Note that this function is not overloaded on types `signed char` and `unsigned char`.

```
basic_istream& get(basic_streambuf<char_type, traits>& sb, char_type delim);
```

13 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, extracts characters and inserts them in the output sequence controlled by **sb**. Characters are extracted and inserted until any of the following occurs:

- (13.1) — end-of-file occurs on the input sequence;
- (13.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);
- (13.3) — `traits::eq(c, delim)` for the next available input character **c** (in which case **c** is not extracted);
- (13.4) — an exception occurs (in which case, the exception is caught but not rethrown).

14 If the function inserts no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

15 *Returns:* `*this`.

```
basic_istream& get(basic_streambuf<char_type, traits>& sb);
```

16 *Effects:* Calls `get(sb, widen('\n'))`.

17 *Returns:* Value returned by the call.

```
basic_istream& getline(char_type* s, streamsize n, char_type delim);
```

18 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, extracts characters and stores them into successive locations of an array whose first element is designated by **s**.²⁵⁷ Characters are extracted and stored until one of the following occurs:

1. end-of-file occurs on the input sequence;
2. `traits::eq(c, delim)` for the next available input character **c** (in which case the input character is extracted but not stored);²⁵⁸
3. **n** is less than one or **n** - 1 characters are stored (in which case the function calls `setstate(failbit)`).

19 These conditions are tested in the order shown.²⁵⁹

20 If the function extracts no characters, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.²⁶⁰

21 In any case, if **n** is greater than zero, it then stores a null character (using `charT()`) into the next successive location of the array.

22 *Returns:* `*this`.

23 [Example 1:

```
#include <iostream>

int main() {
    using namespace std;
    const int line_buffer_size = 100;

    char buffer[line_buffer_size];
    int line_number = 0;
    while (cin.getline(buffer, line_buffer_size, '\n') || cin.gcount()) {
        int count = cin.gcount();
        if (cin.eof())
            cout << "Partial final line";    // cin.fail() is false
        else if (cin.fail()) {
            cout << "Partial long line";
            cin.clear(cin.rdstate() & ~ios_base::failbit);
        } else {
            count--;                          // Don't include newline in count
        }
    }
}
```

257) Note that this function is not overloaded on types `signed char` and `unsigned char`.

258) Since the final input character is “extracted”, it is counted in the `gcount()`, even though it is not stored.

259) This allows an input line which exactly fills the buffer, without setting `failbit`. This is different behavior than the historical AT&T implementation.

260) This implies an empty input line will not cause `failbit` to be set.

```

        cout << "Line " << ++line_number;
    }
    cout << " (" << count << " chars): " << buffer << endl;
}
}
— end example]

```

```
basic_istream& getline(char_type* s, streamsize n);
```

24 *Returns:* `getline(s, n, widen('\n'))`.

```
basic_istream& ignore(streamsize n = 1, int_type delim = traits::eof());
```

25 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, extracts characters and discards them. Characters are extracted until any of the following occurs:

- (25.1) — `n != numeric_limits<streamsize>::max()` (17.3.5) and `n` characters have been extracted so far;
- (25.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`, which may throw `ios_base::failure` (31.5.4.4));
- (25.3) — `traits::eq_int_type(traits::to_int_type(c), delim)` for the next available input character `c` (in which case `c` is extracted).

[Note 1: The last condition will never occur if `traits::eq_int_type(delim, traits::eof())`. — end note]

26 *Returns:* `*this`.

```
basic_istream& ignore(streamsize n, char_type delim);
```

27 *Constraints:* `is_same_v<char_type, char>` is true.

28 *Effects:* Equivalent to: `return ignore(n, traits::to_int_type(delim));`

```
int_type peek();
```

29 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, reads but does not extract the current input character.

30 *Returns:* `traits::eof()` if `good()` is false. Otherwise, returns `rdbuf()->sgetc()`.

```
basic_istream& read(char_type* s, streamsize n);
```

31 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. Otherwise extracts characters and stores them into successive locations of an array whose first element is designated by `s`.²⁶¹ Characters are extracted and stored until either of the following occurs:

- (31.1) — `n` characters are stored;
- (31.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(failbit | eofbit)`, which may throw `ios_base::failure` (31.5.4.4)).

32 *Returns:* `*this`.

```
streamsize readsome(char_type* s, streamsize n);
```

33 *Effects:* Behaves as an unformatted input function (as described above). After constructing a **sentry** object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. Otherwise extracts characters and stores them into successive locations of an array whose first element is designated by `s`. If `rdbuf()->in_avail() == -1`, calls `setstate(eofbit)` (which may throw `ios_base::failure` (31.5.4.4)), and extracts no characters;

- (33.1) — If `rdbuf()->in_avail() == 0`, extracts no characters
- (33.2) — If `rdbuf()->in_avail() > 0`, extracts `min(rdbuf()->in_avail(), n)`.

34 *Returns:* The number of characters extracted.

261) Note that this function is not overloaded on types `signed char` and `unsigned char`.

```
basic_istream& putback(char_type c);
```

35 *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears `eofbit`. After constructing a `sentry` object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. If `rdbuf()` is not null, calls `rdbuf()->sputbackc(c)`. If `rdbuf()` is null, or if `sputbackc` returns `traits::eof()`, calls `setstate(badbit)` (which may throw `ios_base::failure` (31.5.4.4)).

[Note 2: This function extracts no characters, so the value returned by the next call to `gcount()` is 0. — end note]

36 *Returns:* `*this`.

```
basic_istream& unget();
```

37 *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears `eofbit`. After constructing a `sentry` object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. If `rdbuf()` is not null, calls `rdbuf()->sungetc()`. If `rdbuf()` is null, or if `sungetc` returns `traits::eof()`, calls `setstate(badbit)` (which may throw `ios_base::failure` (31.5.4.4)).

[Note 3: This function extracts no characters, so the value returned by the next call to `gcount()` is 0. — end note]

38 *Returns:* `*this`.

```
int sync();
```

39 *Effects:* Behaves as an unformatted input function (as described above), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `gcount()`. After constructing a `sentry` object, if `rdbuf()` is a null pointer, returns -1. Otherwise, calls `rdbuf()->pubsync()` and, if that function returns -1 calls `setstate(badbit)` (which may throw `ios_base::failure` (31.5.4.4)), and returns -1. Otherwise, returns zero.

```
pos_type tellg();
```

40 *Effects:* Behaves as an unformatted input function (as described above), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `gcount()`.

41 *Returns:* After constructing a `sentry` object, if `fail() != false`, returns `pos_type(-1)` to indicate failure. Otherwise, returns `rdbuf()->pubseekoff(0, cur, in)`.

```
basic_istream& seekg(pos_type pos);
```

42 *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears `eofbit`, it does not count the number of characters extracted, and it does not affect the value returned by subsequent calls to `gcount()`. After constructing a `sentry` object, if `fail() != true`, executes `rdbuf()->pubseekpos(pos, ios_base::in)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

43 *Returns:* `*this`.

```
basic_istream& seekg(off_type off, ios_base::seekdir dir);
```

44 *Effects:* Behaves as an unformatted input function (as described above), except that the function first clears `eofbit`, does not count the number of characters extracted, and does not affect the value returned by subsequent calls to `gcount()`. After constructing a `sentry` object, if `fail() != true`, executes `rdbuf()->pubseekoff(off, dir, ios_base::in)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

45 *Returns:* `*this`.

31.7.5.5 Standard `basic_istream` manipulators

[istream.manip]

- ¹ Each instantiation of the function template specified in this subclause is a designated addressable function (16.4.5.2.1).

```
template<class charT, class traits>
basic_istream<charT, traits>& ws(basic_istream<charT, traits>& is);
```

- 2 *Effects:* Behaves as an unformatted input function (31.7.5.4), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `is.gcount()`. After constructing a `sentry` object extracts characters as long as the next available character `c` is whitespace or until there are no more characters in the sequence. Whitespace characters are distinguished with the same criterion as used by `sentry::sentry` (31.7.5.2.4). If `ws` stops extracting characters because there are no more available it sets `eofbit`, but not `failbit`.

- 3 *Returns:* `is`.

31.7.5.6 Rvalue stream extraction

[istream.rvalue]

```
template<class Istream, class T>
Istream&& operator>>(Istream&& is, T&& x);
```

- 1 *Constraints:* The expression `is >> std::forward<T>(x)` is well-formed when treated as an unevaluated operand (7.2.3) and `Istream` is publicly and unambiguously derived from `ios_base`.

- 2 *Effects:* Equivalent to:

```
is >> std::forward<T>(x);
return std::move(is);
```

31.7.5.7 Class template basic_iostream

[iostreamclass]

31.7.5.7.1 General

[iostreamclass.general]

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_iostream
    : public basic_istream<charT, traits>,
      public basic_ostream<charT, traits> {
    public:
        using char_type    = charT;
        using int_type      = traits::int_type;
        using pos_type      = traits::pos_type;
        using off_type      = traits::off_type;
        using traits_type   = traits;

        // 31.7.5.7.2, constructor
        explicit basic_iostream(basic_streambuf<charT, traits>* sb);

        // 31.7.5.7.3, destructor
        virtual ~basic_iostream();

    protected:
        // 31.7.5.7.2, constructor
        basic_iostream(const basic_iostream&) = delete;
        basic_iostream(basic_iostream&& rhs);

        // 31.7.5.7.4, assignment and swap
        basic_iostream& operator=(const basic_iostream&) = delete;
        basic_iostream& operator=(basic_iostream&& rhs);
        void swap(basic_iostream& rhs);
    };
}
```

- 1 The class template `basic_iostream` inherits a number of functions that allow reading input and writing output to sequences controlled by a stream buffer.

31.7.5.7.2 Constructors

[iostream.cons]

```
explicit basic_iostream(basic_streambuf<charT, traits>* sb);
```

- 1 *Effects:* Initializes the base class subobjects with `basic_istream<charT, traits>(sb)` (31.7.5.2) and `basic_ostream<charT, traits>(sb)` (31.7.6.2).

- 2 *Postconditions:* `rdbuf() == sb` and `gcount() == 0`.

```
basic_istream(basic_istream&& rhs);
```

- 3 *Effects:* Move constructs from the rvalue *rhs* by constructing the `basic_istream` base class with `std::move(rhs)`.

31.7.5.7.3 Destructor

[istream.dest]

```
virtual ~basic_istream();
```

- 1 *Remarks:* Does not perform any operations on `rdbuf()`.

31.7.5.7.4 Assignment and swap

[istream.assign]

```
basic_istream& operator=(basic_istream&& rhs);
```

- 1 *Effects:* Equivalent to `swap(rhs)`.

```
void swap(basic_istream& rhs);
```

- 2 *Effects:* Calls `basic_istream<charT, traits>::swap(rhs)`.

31.7.6 Output streams

[output.streams]

31.7.6.1 General

[output.streams.general]

- 1 The header `<ostream>` defines a class template and several function templates that control output to a stream buffer, along with a function template that inserts into stream rvalues.

31.7.6.2 Class template `basic_ostream`

[ostream]

31.7.6.2.1 General

[ostream.general]

- 1 When a function has a parameter type *extended-floating-point-type*, the implementation provides overloads for all cv-unqualified extended floating-point types (6.9.3).

```
namespace std {
    template<class charT, class traits = char_traits<charT>>
    class basic_ostream : virtual public basic_ios<charT, traits> {
    public:
        // types (inherited from basic_ios (31.5.4))
        using char_type    = charT;
        using int_type      = traits::int_type;
        using pos_type      = traits::pos_type;
        using off_type      = traits::off_type;
        using traits_type    = traits;

        // 31.7.6.2.2, constructor/destructor
        explicit basic_ostream(basic_streambuf<char_type, traits>* sb);
        virtual ~basic_ostream();

        // 31.7.6.2.4, prefix/suffix
        class sentry;

        // 31.7.6.3, formatted output
        basic_ostream& operator<<(basic_ostream& (*pf)(basic_ostream&));
        basic_ostream& operator<<(basic_ios<charT, traits>& (*pf)(basic_ios<charT, traits>&));
        basic_ostream& operator<<(ios_base& (*pf)(ios_base&));

        basic_ostream& operator<<(bool n);
        basic_ostream& operator<<(short n);
        basic_ostream& operator<<(unsigned short n);
        basic_ostream& operator<<(int n);
        basic_ostream& operator<<(unsigned int n);
        basic_ostream& operator<<(long n);
        basic_ostream& operator<<(unsigned long n);
        basic_ostream& operator<<(long long n);
        basic_ostream& operator<<(unsigned long long n);
        basic_ostream& operator<<(float f);
        basic_ostream& operator<<(double f);
        basic_ostream& operator<<(long double f);
```

```

basic_ostream& operator<<(extended-floating-point-type f);

basic_ostream& operator<<(const void* p);
basic_ostream& operator<<(const volatile void* p);
basic_ostream& operator<<(nullptr_t);
basic_ostream& operator<<(basic_streambuf<char_type, traits>* sb);

// 31.7.6.4, unformatted output
basic_ostream& put(char_type c);
basic_ostream& write(const char_type* s, streamsize n);

basic_ostream& flush();

// 31.7.6.2.5, seeks
pos_type tellp();
basic_ostream& seekp(pos_type);
basic_ostream& seekp(off_type, ios_base::seekdir);

protected:
// 31.7.6.2.2, copy/move constructor
basic_ostream(const basic_ostream&) = delete;
basic_ostream(basic_ostream&& rhs);

// 31.7.6.2.3, assignment and swap
basic_ostream& operator=(const basic_ostream&) = delete;
basic_ostream& operator=(basic_ostream&& rhs);
void swap(basic_ostream& rhs);
};

// 31.7.6.3.4, character inserters
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, charT);
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, char);
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char);

template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, wchar_t) = delete;
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char8_t) = delete;
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char16_t) = delete;
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char32_t) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, char8_t) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, char16_t) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, char32_t) = delete;

template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const charT*);
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const char*);
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char*);

```



```

template<class traits>
    basic_ostream<char, traits>&
        operator<<(basic_ostream<char, traits>&, const wchar_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<<(basic_ostream<char, traits>&, const char8_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<<(basic_ostream<char, traits>&, const char16_t*) = delete;
template<class traits>
    basic_ostream<char, traits>&
        operator<<(basic_ostream<char, traits>&, const char32_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, const char8_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, const char16_t*) = delete;
template<class traits>
    basic_ostream<wchar_t, traits>&
        operator<<(basic_ostream<wchar_t, traits>&, const char32_t*) = delete;
}

```

- ² The class template `basic_ostream` defines a number of member function signatures that assist in formatting and writing output to output sequences controlled by a stream buffer.
- ³ Two groups of member function signatures share common properties: the *formatted output functions* (or *inserters*) and the *unformatted output functions*. Both groups of output functions generate (or *insert*) output *characters* by actions equivalent to calling `rddbuf()->sputc(int_type)`. They may use other public members of `basic_ostream` except that they shall not invoke any virtual members of `rddbuf()` except `overflow()`, `xspn()`, and `sync()`.
- ⁴ If one of these called functions throws an exception, then unless explicitly noted otherwise the output function sets `badbit` in the error state. If `badbit` is set in `exceptions()`, the output function rethrows the exception without completing its actions, otherwise it does not throw anything and proceeds as if the called function had returned a failure indication.
- ⁵ [Note 1: The deleted overloads of `operator<<` prevent formatting characters as integers and strings as pointers. — end note]

31.7.6.2.2 Constructors

[ostream.cons]

```
explicit basic_ostream(basic_streambuf<charT, traits>* sb);
```

- ¹ *Effects*: Initializes the base class subobject with `basic_ios<charT, traits>::init(sb)` (31.5.4.2).
- ² *Postconditions*: `rddbuf() == sb`.

```
basic_ostream(basic_ostream&& rhs);
```

- ³ *Effects*: Move constructs from the rvalue `rhs`. This is accomplished by default constructing the base class and calling `basic_ios<charT, traits>::move(rhs)` to initialize the base class.

```
virtual ~basic_ostream();
```

- ⁴ *Remarks*: Does not perform any operations on `rddbuf()`.

31.7.6.2.3 Assignment and swap

[ostream.assign]

```
basic_ostream& operator=(basic_ostream&& rhs);
```

- ¹ *Effects*: Equivalent to `swap(rhs)`.
- ² *Returns*: `*this`.

```
void swap(basic_ostream& rhs);
```

- ³ *Effects*: Calls `basic_ios<charT, traits>::swap(rhs)`.

31.7.6.2.4 Class `basic_ostream::sentry`

[ostream.sentry]

```

namespace std {
    template<class charT, class traits>
    class basic_ostream<charT, traits>::sentry {
        bool ok_;           // exposition only

    public:
        explicit sentry(basic_ostream& os);
        ~sentry();
        explicit operator bool() const { return ok_; }

        sentry(const sentry&) = delete;
        sentry& operator=(const sentry&) = delete;
    };
}

```

- ¹ The class `sentry` defines a class that is responsible for doing exception safe prefix and suffix operations.

```
explicit sentry(basic_ostream& os);
```

- ² If `os.good()` is nonzero, prepares for formatted or unformatted output. If `os.tie()` is not a null pointer, calls `os.tie()->flush()`.²⁶²
- ³ If, after any preparation is completed, `os.good()` is true, `ok_ == true` otherwise, `ok_ == false`. During preparation, the constructor may call `setstate(failbit)` (which may throw `ios_base::failure` (31.5.4.4)).²⁶³

```
~sentry();
```

- ⁴ If `(os.flags() & ios_base::unitbuf) && !uncaught_exceptions() && os.good()` is true, calls `os.rdbuf()->pubsync()`. If that function returns `-1` or exits via an exception, sets `badbit` in `os.rdstate()` without propagating an exception.

```
explicit operator bool() const;
```

- ⁵ *Effects:* Returns `ok_`.

31.7.6.2.5 Seek members

[ostream.seek]

- ¹ Each seek member function begins execution by constructing an object of class `sentry`. It returns by destroying the `sentry` object.

```
pos_type tellp();
```

- ² *Returns:* If `fail() != false`, returns `pos_type(-1)` to indicate failure. Otherwise, returns `rdbuf()->pubseekoff(0, cur, out)`.

```
basic_ostream& seekp(pos_type pos);
```

- ³ *Effects:* If `fail() != true`, executes `rdbuf()->pubseekpos(pos, ios_base::out)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

- ⁴ *Returns:* `*this`.

```
basic_ostream& seekp(off_type off, ios_base::seekdir dir);
```

- ⁵ *Effects:* If `fail() != true`, executes `rdbuf()->pubseekoff(off, dir, ios_base::out)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

- ⁶ *Returns:* `*this`.

31.7.6.3 Formatted output functions

[ostream.formatted]

31.7.6.3.1 Common requirements

[ostream.formatted.reqmts]

- ¹ Each formatted output function begins execution by constructing an object of class `sentry`. If that object returns `true` when converted to a value of type `bool`, the function endeavors to generate the requested output. If the generation fails, then the formatted output function does `setstate(ios_base::failbit)`,

²⁶²) The call `os.tie()->flush()` does not necessarily occur if the function can determine that no synchronization is necessary.

²⁶³) The `sentry` constructor and destructor can also perform additional implementation-dependent operations.

which can throw an exception. If an exception is thrown during output, then `ios_base::badbit` is set²⁶⁴ in `*this`'s error state. If `(exceptions() & badbit) != 0` then the exception is rethrown. Whether or not an exception is thrown, the `sentry` object is destroyed before leaving the formatted output function. If no exception is thrown, the result of the formatted output function is `*this`.

- ² The descriptions of the individual formatted output functions describe how they perform output and do not mention the `sentry` object.
- ³ If a formatted output function of a stream `os` determines padding, it does so as follows. Given a `charT` character sequence `seq` where `charT` is the character container type of the stream, if the length of `seq` is less than `os.width()`, then enough copies of `os.fill()` are added to this sequence as necessary to pad to a width of `os.width()` characters. If `(os.flags() & ios_base::adjustfield) == ios_base::left` is `true`, the fill characters are placed after the character sequence; otherwise, they are placed before the character sequence.

31.7.6.3.2 Arithmetic inserters

[ostream.inserters.arithmetic]

```
basic_ostream& operator<<(bool val);
basic_ostream& operator<<(short val);
basic_ostream& operator<<(unsigned short val);
basic_ostream& operator<<(int val);
basic_ostream& operator<<(unsigned int val);
basic_ostream& operator<<(long val);
basic_ostream& operator<<(unsigned long val);
basic_ostream& operator<<(long long val);
basic_ostream& operator<<(unsigned long long val);
basic_ostream& operator<<(float val);
basic_ostream& operator<<(double val);
basic_ostream& operator<<(long double val);
basic_ostream& operator<<(const void* val);
```

- ¹ *Effects:* The classes `num_get<>` and `num_put<>` handle locale-dependent numeric formatting and parsing. These inserter functions use the imbued locale value to perform numeric formatting. When `val` is of type `bool`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, `long double`, or `const void*`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(), val).failed();
```

When `val` is of type `short` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
    ? static_cast<long>(static_cast<unsigned short>(val))
    : static_cast<long>(val)).failed();
```

When `val` is of type `int` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
    ? static_cast<long>(static_cast<unsigned int>(val))
    : static_cast<long>(val)).failed();
```

When `val` is of type `unsigned short` or `unsigned int` the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(), static_cast<unsigned long>(val)).failed();
```

When `val` is of type `float` the formatting conversion occurs as if it performed the following code fragment:

²⁶⁴) This is done without causing an `ios_base::failure` to be thrown.

```
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(), static_cast<double>(val)).failed();
```

2 The first argument provides an object of the `ostreambuf_iterator<>` class which is an iterator for class `basic_ostream<>`. It bypasses `ostreams` and uses `streambufs` directly. Class `locale` relies on these types as its interface to `iostreams`, since for flexibility it has been abstracted away from direct dependence on `ostream`. The second parameter is a reference to the base class subobject of type `ios_base`. It provides formatting specifications such as field width, and a locale from which to obtain other facets. If `failed` is `true` then does `setstate(badbit)`, which may throw an exception, and returns.

3 *Returns:* `*this`.

```
basic_ostream& operator<<(const volatile void* p);
```

4 *Effects:* Equivalent to: `return operator<<(const_cast<const void*>(p));`

```
basic_ostream& operator<<(extended-floating-point-type val);
```

5 *Effects:* If the floating-point conversion rank of *extended-floating-point-type* is less than or equal to that of `double`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(), static_cast<double>(val)).failed();
```

Otherwise, if the floating-point conversion rank of *extended-floating-point-type* is less than or equal to that of `long double`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<num_put<charT, ostreambuf_iterator<charT, traits>>>(
    getloc()).put(*this, *this, fill(), static_cast<long double>(val)).failed();
```

Otherwise, an invocation of the operator function is conditionally supported with implementation-defined semantics.

If `failed` is `true` then does `setstate(badbit)`, which may throw an exception, and returns.

6 *Returns:* `*this`.

31.7.6.3.3 `basic_ostream::operator<<` [ostream.inserters]

```
basic_ostream& operator<<(basic_ostream& (*pf)(basic_ostream&));
```

1 *Effects:* None. Does not behave as a formatted output function (as described in 31.7.6.3.1).

2 *Returns:* `pf(*this)`.²⁶⁵

```
basic_ostream& operator<<(basic_ios<charT, traits>& (*pf)(basic_ios<charT, traits>&));
```

3 *Effects:* Calls `pf(*this)`. This inserter does not behave as a formatted output function (as described in 31.7.6.3.1).

4 *Returns:* `*this`.²⁶⁶

```
basic_ostream& operator<<(ios_base& (*pf)(ios_base&));
```

5 *Effects:* Calls `pf(*this)`. This inserter does not behave as a formatted output function (as described in 31.7.6.3.1).

6 *Returns:* `*this`.

```
basic_ostream& operator<<(basic_streambuf<charT, traits>* sb);
```

7 *Effects:* Behaves as an unformatted output function (31.7.6.4). After the `sentry` object is constructed, if `sb` is null calls `setstate(badbit)` (which may throw `ios_base::failure`).

8 Gets characters from `sb` and inserts them in `*this`. Characters are read from `sb` and inserted until any of the following occurs:

- (8.1) — end-of-file occurs on the input sequence;
- (8.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);

²⁶⁵ See, for example, the function signature `endl(basic_ostream&)` (31.7.6.5).

²⁶⁶ See, for example, the function signature `dec(ios_base&)` (31.5.5.3).

(8.3) — an exception occurs while getting a character from `sb`.

9 If the function inserts no characters, it calls `setstate(failbit)` (which may throw `ios_base::failure` (31.5.4.4)). If an exception was thrown while extracting a character, the function sets `failbit` in the error state, and if `failbit` is set in `exceptions()` the caught exception is rethrown.

10 *Returns:* `*this`.

```
basic_ostream& operator<<(nullptr_t);
```

11 *Effects:* Equivalent to:

```
return *this << s;
```

where `s` is an implementation-defined NTCTS (3.36).

31.7.6.3.4 Character inserter function templates

[ostream.inserters.character]

```
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, charT c);
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, char c);
// specialization
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, char c);
```

1 *Effects:* Behaves as a formatted output function (31.7.6.3.1) of `out`. Constructs a character sequence `seq`. If `c` has type `char` and the character container type of the stream is not `char`, then `seq` consists of `out.widen(c)`; otherwise `seq` consists of `c`. Determines padding for `seq` as described in 31.7.6.3.1. Inserts `seq` into `out`. Calls `os.width(0)`.

2 *Returns:* `out`.

```
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, const charT* s);
template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>& out, const char* s);
template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, const char* s);
```

3 *Preconditions:* `s` is not a null pointer.

4 *Effects:* Behaves like a formatted inserter (as described in 31.7.6.3.1) of `out`. Creates a character sequence `seq` of `n` characters starting at `s`, each widened using `out.widen()` (31.5.4.3), where `n` is the number that would be computed as if by:

(4.1) — `traits::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const charT*`, and also for the overload where the first argument is of type `basic_ostream<char, traits>&` and the second is of type `const char*`,

(4.2) — `char_traits<char>::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const char*`.

Determines padding for `seq` as described in 31.7.6.3.1. Inserts `seq` into `out`. Calls `width(0)`.

5 *Returns:* `out`.

31.7.6.3.5 Print

[ostream.formatted.print]

```
template<class... Args>
    void print(ostream& os, format_string<Args...> fmt, Args&&... args);
```

1 *Effects:* If the ordinary literal encoding (5.3.1) is UTF-8, equivalent to:

```
vprint_unicode(os, fmt.str, make_format_args(args...));
```

Otherwise, equivalent to:

```
vprint_nonunicode(os, fmt.str, make_format_args(args...));
```

```
template<class... Args>
    void println(ostream& os, format_string<Args...> fmt, Args&&... args);
```

2 *Effects:* Equivalent to:

```

    print(os, "{}\n", format(os.getloc(), fmt, std::forward<Args>(args)...));

void println(ostream& os);
3    Effects: Equivalent to:
    print(os, "\n");

```

```

void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);

```

- 4 *Effects:* Behaves as a formatted output function (31.7.6.3.1) of `os`, except that:
- (4.1) — failure to generate output is reported as specified below, and
 - (4.2) — any exception thrown by the call to `vformat` is propagated without regard to the value of `os.exceptions()` and without turning on `ios_base::badbit` in the error state of `os`.

After constructing a `sentry` object, the function initializes a variable with automatic storage duration via

- ```

 string out = vformat(os.getloc(), fmt, args);

```
- (4.3) — If the function is `vprint_unicode` and `os` is a stream that refers to a terminal that is capable of displaying Unicode only via a native Unicode API, which is determined in an implementation-defined manner, flushes `os` and then writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined. Then establishes an observable checkpoint (4.1.2).
  - (4.4) — Otherwise inserts the character sequence `[out.begin(), out.end())` into `os`.

If writing to the terminal or inserting into `os` fails, calls `os.setstate(ios_base::badbit)` (which may throw `ios_base::failure`).

- 5 *Recommended practice:* For `vprint_unicode`, if invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

#### 31.7.6.4 Unformatted output functions [ostream.unformatted]

- 1 Each unformatted output function begins execution by constructing an object of class `sentry`. If that object returns `true`, while converting to a value of type `bool`, the function endeavors to generate the requested output. If an exception is thrown during output, then `ios_base::badbit` is set<sup>267</sup> in `*this`'s error state. If `(exceptions() & badbit) != 0` then the exception is rethrown. In any case, the unformatted output function ends by destroying the `sentry` object, then, if no exception was thrown, returning the value specified for the unformatted output function.

```

basic_ostream& put(char_type c);

```

- 2 *Effects:* Behaves as an unformatted output function (as described above). After constructing a `sentry` object, inserts the character `c`, if possible.<sup>268</sup>
- 3 Otherwise, calls `setstate(badbit)` (which may throw `ios_base::failure` (31.5.4.4)).
- 4 *Returns:* `*this`.

```

basic_ostream& write(const char_type* s, streamsize n);

```

- 5 *Effects:* Behaves as an unformatted output function (as described above). After constructing a `sentry` object, obtains characters to insert from successive locations of an array whose first element is designated by `s`.<sup>269</sup> Characters are inserted until either of the following occurs:
- (5.1) — `n` characters are inserted;
  - (5.2) — inserting in the output sequence fails (in which case the function calls `setstate(badbit)`, which may throw `ios_base::failure` (31.5.4.4)).
- 6 *Returns:* `*this`.

<sup>267</sup>) This is done without causing an `ios_base::failure` to be thrown.

<sup>268</sup>) Note that this function is not overloaded on types `signed char` and `unsigned char`.

<sup>269</sup>) Note that this function is not overloaded on types `signed char` and `unsigned char`.

```
basic_ostream& flush();
```

7 *Effects:* Behaves as an unformatted output function (as described above). If `rdbuf()` is not a null pointer, constructs a `sentry` object. If that object returns `true` when converted to a value of type `bool` the function calls `rdbuf()->pubsync()`. If that function returns `-1` calls `setstate(badbit)` (which may throw `ios_base::failure` (31.5.4.4)). Otherwise, if the `sentry` object returns `false`, does nothing.

8 *Returns:* `*this`.

### 31.7.6.5 Standard `basic_ostream` manipulators

[ostream.manip]

1 Each instantiation of any of the function templates specified in this subclause is a designated addressable function (16.4.5.2.1).

```
template<class charT, class traits>
 basic_ostream<charT, traits>& endl(basic_ostream<charT, traits>& os);
```

2 *Effects:* Calls `os.put(os.widen('\n'))`, then `os.flush()`.

3 *Returns:* `os`.

```
template<class charT, class traits>
 basic_ostream<charT, traits>& ends(basic_ostream<charT, traits>& os);
```

4 *Effects:* Inserts a null character into the output sequence: calls `os.put(charT())`.

5 *Returns:* `os`.

```
template<class charT, class traits>
 basic_ostream<charT, traits>& flush(basic_ostream<charT, traits>& os);
```

6 *Effects:* Calls `os.flush()`.

7 *Returns:* `os`.

```
template<class charT, class traits>
 basic_ostream<charT, traits>& emit_on_flush(basic_ostream<charT, traits>& os);
```

8 *Effects:* If `os.rdbuf()` is a `basic_syncbuf<charT, traits, Allocator>*`, called `buf` for the purpose of exposition, calls `buf->set_emit_on_sync(true)`. Otherwise this manipulator has no effect.

[Note 1: To work around the issue that the `Allocator` template argument cannot be deduced, implementations can introduce an intermediate base class to `basic_syncbuf` that manages its *emit-on-sync* flag. — end note]

9 *Returns:* `os`.

```
template<class charT, class traits>
 basic_ostream<charT, traits>& noemit_on_flush(basic_ostream<charT, traits>& os);
```

10 *Effects:* If `os.rdbuf()` is a `basic_syncbuf<charT, traits, Allocator>*`, called `buf` for the purpose of exposition, calls `buf->set_emit_on_sync(false)`. Otherwise this manipulator has no effect.

11 *Returns:* `os`.

```
template<class charT, class traits>
 basic_ostream<charT, traits>& flush_emit(basic_ostream<charT, traits>& os);
```

12 *Effects:* Calls `os.flush()`. Then, if `os.rdbuf()` is a `basic_syncbuf<charT, traits, Allocator>*`, called `buf` for the purpose of exposition, behaves as an unformatted output function (31.7.6.4) of `os`. After constructing a `sentry` object, calls `buf->emit()`. If that call returns `false`, calls `os.setstate(ios_base::badbit)`.

13 *Returns:* `os`.

### 31.7.6.6 Rvalue stream insertion

[ostream.rvalue]

```
template<class Ostream, class T>
 Ostream&& operator<<(Ostream&& os, const T& x);
```

1 *Constraints:* The expression `os << x` is well-formed when treated as an unevaluated operand and `Ostream` is publicly and unambiguously derived from `ios_base`.

2 *Effects:* As if by: `os << x`;

3 *Returns:* `std::move(os)`.

### 31.7.7 Standard manipulators

[std.manip]

1 The header `<iomanip>` defines several functions that support extractors and inserters that alter information maintained by class `ios_base` and its derived classes.

*unspecified* `resetiosflags(ios_base::fmtflags mask);`

2 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << resetiosflags(mask)` behaves as if it called `f(out, mask)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> resetiosflags(mask)` behaves as if it called `f(in, mask)`, where the function `f` is defined as:<sup>270</sup>

```
void f(ios_base& str, ios_base::fmtflags mask) {
 // reset specified flags
 str.setf(ios_base::fmtflags(0), mask);
}
```

The expression `out << resetiosflags(mask)` has type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> resetiosflags(mask)` has type `basic_istream<charT, traits>&` and value `in`.

*unspecified* `setiosflags(ios_base::fmtflags mask);`

3 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setiosflags(mask)` behaves as if it called `f(out, mask)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setiosflags(mask)` behaves as if it called `f(in, mask)`, where the function `f` is defined as:

```
void f(ios_base& str, ios_base::fmtflags mask) {
 // set specified flags
 str.setf(mask);
}
```

The expression `out << setiosflags(mask)` has type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setiosflags(mask)` has type `basic_istream<charT, traits>&` and value `in`.

*unspecified* `setbase(int base);`

4 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setbase(base)` behaves as if it called `f(out, base)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setbase(base)` behaves as if it called `f(in, base)`, where the function `f` is defined as:

```
void f(ios_base& str, int base) {
 // set basefield
 str.setf(base == 8 ? ios_base::oct :
 base == 10 ? ios_base::dec :
 base == 16 ? ios_base::hex :
 ios_base::fmtflags(0), ios_base::basefield);
}
```

The expression `out << setbase(base)` has type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setbase(base)` has type `basic_istream<charT, traits>&` and value `in`.

*unspecified* `setfill(char_type c);`

5 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` and `c` has type `charT` then the expression `out << setfill(c)` behaves as if it called `f(out, c)`, where the function `f` is defined as:

<sup>270</sup> The expression `cin >> resetiosflags(ios_base::skipws)` clears `ios_base::skipws` in the format flags stored in the `basic_istream<charT, traits>` object `cin` (the same as `cin >> noskipws`), and the expression `cout << resetiosflags(ios_base::showbase)` clears `ios_base::showbase` in the format flags stored in the `basic_ostream<charT, traits>` object `cout` (the same as `cout << noshowbase`).



```
template<class charT, class traits>
void f(basic_ios<charT, traits>& str, charT c) {
 // set fill character
 str.fill(c);
}
```

The expression `out << setfill(c)` has type `basic_ostream<charT, traits>&` and value `out`.

*unspecified* `setprecision(int n);`

- 6 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setprecision(n)` behaves as if it called `f(out, n)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setprecision(n)` behaves as if it called `f(in, n)`, where the function `f` is defined as:

```
void f(ios_base& str, int n) {
 // set precision
 str.precision(n);
}
```

The expression `out << setprecision(n)` has type `basic_ostream<charT, traits>&` and value `out`.  
The expression `in >> setprecision(n)` has type `basic_istream<charT, traits>&` and value `in`.

*unspecified* `setw(int n);`

- 7 *Returns:* An object of unspecified type such that if `out` is an instance of `basic_ostream<charT, traits>` then the expression `out << setw(n)` behaves as if it called `f(out, n)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setw(n)` behaves as if it called `f(in, n)`, where the function `f` is defined as:

```
void f(ios_base& str, int n) {
 // set width
 str.width(n);
}
```

The expression `out << setw(n)` has type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setw(n)` has type `basic_istream<charT, traits>&` and value `in`.

### 31.7.8 Extended manipulators

[ext.manip]

- 1 The header `<iomanip>` defines several functions that support extractors and inserters that allow for the parsing and formatting of sequences and values for money and time.

```
template<class moneyT> unspecified get_money(moneyT& mon, bool intl = false);
```

- 2 *Mandates:* The type `moneyT` is either `long double` or a specialization of the `basic_string` template (Clause 27).
- 3 *Effects:* The expression `in >> get_money(mon, intl)` described below behaves as a formatted input function (31.7.5.3.1).
- 4 *Returns:* An object of unspecified type such that if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> get_money(mon, intl)` behaves as if it called `f(in, mon, intl)`, where the function `f` is defined as:

```
template<class charT, class traits, class moneyT>
void f(basic_ios<charT, traits>& str, moneyT& mon, bool intl) {
 using Iter = istreambuf_iterator<charT, traits>;
 using MoneyGet = money_get<charT, Iter>;

 ios_base::iostate err = ios_base::goodbit;
 const MoneyGet& mg = use_facet<MoneyGet>(str.getloc());

 mg.get(Iter(str.rdbuf()), Iter(), intl, str, err, mon);

 if (ios_base::goodbit != err)
 str.setstate(err);
}
```



The expression in `>> get_money(mon, intl)` has type `basic_istream<charT, traits>&` and value in.

```
template<class moneyT> unspecified put_money(const moneyT& mon, bool intl = false);
```

5 *Mandates:* The type `moneyT` is either `long double` or a specialization of the `basic_string` template (Clause 27).

6 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << put_money(mon, intl)` behaves as a formatted output function (31.7.6.3.1) that calls `f(out, mon, intl)`, where the function `f` is defined as:

```
template<class charT, class traits, class moneyT>
void f(basic_ios<charT, traits>& str, const moneyT& mon, bool intl) {
 using Iter = ostreambuf_iterator<charT, traits>;
 using MoneyPut = money_put<charT, Iter>;

 const MoneyPut& mp = use_facet<MoneyPut>(str.getloc());
 const Iter end = mp.put(Iter(str.rdbuf()), intl, str, str.fill(), mon);

 if (end.failed())
 str.setstate(ios_base::badbit);
}
```

The expression `out << put_money(mon, intl)` has type `basic_ostream<charT, traits>&` and value `out`.

```
template<class charT> unspecified get_time(tm* tmb, const charT* fmt);
```

7 *Preconditions:* The argument `tmb` is a valid pointer to an object of type `tm`, and `[fmt, fmt + char_traits<charT>::length(fmt))` is a valid range.

8 *Returns:* An object of unspecified type such that if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> get_time(tmb, fmt)` behaves as if it called `f(in, tmb, fmt)`, where the function `f` is defined as:

```
template<class charT, class traits>
void f(basic_ios<charT, traits>& str, tm* tmb, const charT* fmt) {
 using Iter = istreambuf_iterator<charT, traits>;
 using TimeGet = time_get<charT, Iter>;

 ios_base::iostate err = ios_base::goodbit;
 const TimeGet& tg = use_facet<TimeGet>(str.getloc());

 tg.get(Iter(str.rdbuf()), Iter(), str, err, tmb,
 fmt, fmt + traits::length(fmt));

 if (err != ios_base::goodbit)
 str.setstate(err);
}
```

The expression `in >> get_time(tmb, fmt)` has type `basic_istream<charT, traits>&` and value in.

```
template<class charT> unspecified put_time(const tm* tmb, const charT* fmt);
```

9 *Preconditions:* The argument `tmb` is a valid pointer to an object of type `tm`, and `[fmt, fmt + char_traits<charT>::length(fmt))` is a valid range.

10 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << put_time(tmb, fmt)` behaves as if it called `f(out, tmb, fmt)`, where the function `f` is defined as:

```
template<class charT, class traits>
void f(basic_ios<charT, traits>& str, const tm* tmb, const charT* fmt) {
 using Iter = ostreambuf_iterator<charT, traits>;
 using TimePut = time_put<charT, Iter>;

 const TimePut& tp = use_facet<TimePut>(str.getloc());
```

```

const Iter end = tp.put(Iter(str.rdbuf()), str, str.fill(), tmb,
 fmt, fmt + traits::length(fmt));

if (end.failed())
 str.setstate(ios_base::badbit);
}

```

The expression `out << put_time(tmb, fmt)` has type `basic_ostream<charT, traits>&` and value `out`.

### 31.7.9 Quoted manipulators

[quoted.manip]

- <sup>1</sup> [Note 1: Quoted manipulators provide string insertion and extraction of quoted strings (for example, XML and CSV formats). Quoted manipulators are useful in ensuring that the content of a string with embedded spaces remains unchanged if inserted and then extracted via stream I/O. — end note]

```

template<class charT>
 unspecified quoted(const charT* s, charT delim = charT('\"'), charT escape = charT('\\'));
template<class charT, class traits, class Allocator>
 unspecified quoted(const basic_string<charT, traits, Allocator>& s,
 charT delim = charT('\"'), charT escape = charT('\\'));
template<class charT, class traits>
 unspecified quoted(basic_string_view<charT, traits> s,
 charT delim = charT('\"'), charT escape = charT('\\'));

```

- <sup>2</sup> *Returns:* An object of unspecified type such that if `out` is an instance of `basic_ostream` with member type `char_type` the same as `charT` and with member type `traits_type`, which in the second and third forms is the same as `traits`, then the expression `out << quoted(s, delim, escape)` behaves as a formatted output function (31.7.6.3.1) of `out`. This forms a character sequence `seq`, initially consisting of the following elements:

- (2.1) — `delim`.
- (2.2) — Each character in `s`. If the character to be output is equal to `escape` or `delim`, as determined by `traits_type::eq`, first output `escape`.
- (2.3) — `delim`.

Let `x` be the number of elements initially in `seq`. Then padding is determined for `seq` as described in 31.7.6.3.1, `seq` is inserted as if by calling `out.rdbuf()->sputn(seq, n)`, where `n` is the larger of `out.width()` and `x`, and `out.width(0)` is called. The expression `out << quoted(s, delim, escape)` has type `basic_ostream<charT, traits>&` and value `out`.

```

template<class charT, class traits, class Allocator>
 unspecified quoted(basic_string<charT, traits, Allocator>& s,
 charT delim = charT('\"'), charT escape = charT('\\'));

```

- <sup>3</sup> *Returns:* An object of unspecified type such that:

- (3.1) — If `in` is an instance of `basic_istream` with member types `char_type` and `traits_type` the same as `charT` and `traits`, respectively, then the expression `in >> quoted(s, delim, escape)` behaves as if it extracts the following characters from `in` using `operator>>(basic_istream<charT, traits>&, charT&)` (31.7.5.3.3) which may throw `ios_base::failure` (31.5.2.2.1):
  - (3.1.1) — If the first character extracted is equal to `delim`, as determined by `traits_type::eq`, then:
    - (3.1.1.1) — Turn off the `skipws` flag.
    - (3.1.1.2) — `s.clear()`
    - (3.1.1.3) — Until an unescaped `delim` character is reached or `!in`, extract characters from `in` and append them to `s`, except that if an `escape` is reached, ignore it and append the next character to `s`.
    - (3.1.1.4) — Discard the final `delim` character.
    - (3.1.1.5) — Restore the `skipws` flag to its original value.
  - (3.1.2) — Otherwise, `in >> s`.

- (3.2) — If `out` is an instance of `basic_ostream` with member types `char_type` and `traits_type` the same as `charT` and `traits`, respectively, then the expression `out << quoted(s, delim, escape)` behaves as specified for the `const basic_string<charT, traits, Allocator>&` overload of the `quoted` function.
- (3.3) — The expression `in >> quoted(s, delim, escape)` has type `basic_istream<charT, traits>&` and value `in`.
- (3.4) — The expression `out << quoted(s, delim, escape)` has type `basic_ostream<charT, traits>&` and value `out`.

### 31.7.10 Print functions

[print.fun]

```
template<class... Args>
void print(format_string<Args...> fmt, Args&&... args);
```

1 *Effects:* Equivalent to:

```
print(stdout, fmt, std::forward<Args>(args)...);
```

```
template<class... Args>
void print(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

2 *Effects:* Let `locksafe` be `(enable_nonlocking_formatter_optimization<remove_cvref_t<Args>>&& ...)`. If the ordinary literal encoding (5.3.1) is UTF-8, equivalent to:

```
locksafe
? vprint_unicode(stream, fmt.str, make_format_args(args...))
: vprint_unicode_buffered(stream, fmt.str, make_format_args(args...));
```

Otherwise, equivalent to:

```
locksafe
? vprint_nonunicode(stream, fmt.str, make_format_args(args...))
: vprint_nonunicode_buffered(stream, fmt.str, make_format_args(args...));
```

```
template<class... Args>
void println(format_string<Args...> fmt, Args&&... args);
```

3 *Effects:* Equivalent to:

```
println(stdout, fmt, std::forward<Args>(args)...);
```

```
void println();
```

4 *Effects:* Equivalent to:

```
println(stdout);
```

```
template<class... Args>
void println(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

5 *Effects:* Equivalent to:

```
print(stream, dynamic_format(string(fmt.get()) + '\n'), std::forward<Args>(args)...);
```

```
void println(FILE* stream);
```

6 *Effects:* Equivalent to:

```
print(stream, "\n");
```

```
void vprint_unicode(string_view fmt, format_args args);
```

7 *Effects:* Equivalent to:

```
vprint_unicode(stdout, fmt, args);
```

```
void vprint_unicode_buffered(FILE* stream, string_view fmt, format_args args);
```

8 *Effects:* Equivalent to:

```
string out = vformat(fmt, args);
vprint_unicode(stream, "{}", make_format_args(out));
```

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

9     *Preconditions:* **stream** is a valid pointer to an output C stream.

10    *Effects:* Locks **stream**. Let **out** denote the character representation of formatting arguments provided by **args** formatted according to specifications given in **fmt**.

(10.1)    — If **stream** refers to a terminal that is capable of displaying Unicode only via a native Unicode API, flushes **stream** and then writes **out** to the terminal using the native Unicode API; if **out** contains invalid code units, the behavior is undefined. Then establishes an observable checkpoint (4.1.2).

(10.2)    — Otherwise writes **out** to **stream** unchanged.

Unconditionally unlocks **stream** on function exit.

SEE ALSO: ISO/IEC 9899:2024, 7.23.2.

[*Note 1:* On Windows the native Unicode API is `WriteConsoleW` and **stream** referring to a terminal means that `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` returns nonzero. — *end note*]

11    *Throws:* Any exception thrown by the call to `vformat` (28.5.3). `system_error` if writing to the terminal or **stream** fails. May throw `bad_alloc`.

12    *Recommended practice:* If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

```
void vprint_nonunicode(string_view fmt, format_args args);
```

13    *Effects:* Equivalent to:

```
 vprint_nonunicode(stdout, fmt, args);
```

```
void vprint_nonunicode_buffered(FILE* stream, string_view fmt, format_args args);
```

14    *Effects:* Equivalent to:

```
 string out = vformat(fmt, args);
```

```
 vprint_nonunicode(stream, "{}", make_format_args(out));
```

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

15    *Preconditions:* **stream** is a valid pointer to an output C stream.

16    *Effects:* While holding the lock on **stream**, writes the character representation of formatting arguments provided by **args** formatted according to specifications given in **fmt** to **stream**.

17    *Throws:* Any exception thrown by the call to `vformat` (28.5.3). `system_error` if writing to **stream** fails. May throw `bad_alloc`.

## 31.8 String-based streams

[string.streams]

### 31.8.1 Header <sstream> synopsis

[sstream.syn]

```
namespace std {
 // 31.8.2, class template basic_stringbuf
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringbuf;

 template<class charT, class traits, class Allocator>
 void swap(basic_stringbuf<charT, traits, Allocator>& x,
 basic_stringbuf<charT, traits, Allocator>& y) noexcept(noexcept(x.swap(y)));

 using stringbuf = basic_stringbuf<char>;
 using wstringbuf = basic_stringbuf<wchar_t>;

 // 31.8.3, class template basic_istreamstream
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_istreamstream;

 template<class charT, class traits, class Allocator>
 void swap(basic_istreamstream<charT, traits, Allocator>& x,
 basic_istreamstream<charT, traits, Allocator>& y);
```

```

using istringstream = basic_istringstream<char>;
using wistringstream = basic_istringstream<wchar_t>;

// 31.8.4, class template basic_ostringstream
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_ostringstream;

template<class charT, class traits, class Allocator>
 void swap(basic_ostringstream<charT, traits, Allocator>& x,
 basic_ostringstream<charT, traits, Allocator>& y);

using ostreamstream = basic_ostringstream<char>;
using wostringstream = basic_ostringstream<wchar_t>;

// 31.8.5, class template basic_stringstream
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringstream;

template<class charT, class traits, class Allocator>
 void swap(basic_stringstream<charT, traits, Allocator>& x,
 basic_stringstream<charT, traits, Allocator>& y);

using stringstream = basic_stringstream<char>;
using wstringstream = basic_stringstream<wchar_t>;
}

```

- <sup>1</sup> The header <sstream> defines four class templates and eight types that associate stream buffers with objects of class `basic_string`, as described in 27.4.

## 31.8.2 Class template `basic_stringbuf`

[stringbuf]

### 31.8.2.1 General

[stringbuf.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringbuf : public basic_streambuf<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;
 using allocator_type = Allocator;

 // 31.8.2.2, constructors
 basic_stringbuf() : basic_stringbuf(ios_base::in | ios_base::out) {}
 explicit basic_stringbuf(ios_base::openmode which);
 explicit basic_stringbuf(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
 explicit basic_stringbuf(const Allocator& a)
 : basic_stringbuf(ios_base::in | ios_base::out, a) {}
 basic_stringbuf(ios_base::openmode which, const Allocator& a);
 explicit basic_stringbuf(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
 template<class SAlloc>
 basic_stringbuf(
 const basic_string<charT, traits, SAlloc>& s, const Allocator& a)
 : basic_stringbuf(s, ios_base::in | ios_base::out, a) {}
 template<class SAlloc>
 basic_stringbuf(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
 };
 };
}

```

```

template<class SAlloc>
 explicit basic_stringbuf(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
template<class T>
 explicit basic_stringbuf(const T& t,
 ios_base::openmode which = ios_base::in | ios_base::out);
template<class T>
 basic_stringbuf(const T& t, const Allocator& a);
template<class T>
 basic_stringbuf(const T& t, ios_base::openmode which, const Allocator& a);
basic_stringbuf(const basic_stringbuf&) = delete;
basic_stringbuf(basic_stringbuf&& rhs);
basic_stringbuf(basic_stringbuf&& rhs, const Allocator& a);

// 31.8.2.3, assignment and swap
basic_stringbuf& operator=(const basic_stringbuf&) = delete;
basic_stringbuf& operator=(basic_stringbuf&& rhs);
void swap(basic_stringbuf& rhs) noexcept(see below);

// 31.8.2.4, getters and setters
allocator_type get_allocator() const noexcept;

basic_string<charT, traits, Allocator> str() const &;
template<class SAlloc>
 basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
basic_string<charT, traits, Allocator> str() &&;
basic_string_view<charT, traits> view() const noexcept;

void str(const basic_string<charT, traits, Allocator>& s);
template<class SAlloc>
 void str(const basic_string<charT, traits, SAlloc>& s);
void str(basic_string<charT, traits, Allocator>&& s);
template<class T>
 void str(const T& t);

protected:
// 31.8.2.5, overridden virtual functions
int_type underflow() override;
int_type pbackfail(int_type c = traits::eof()) override;
int_type overflow(int_type c = traits::eof()) override;
basic_streambuf<charT, traits>* setbuf(charT*, streamsize) override;

pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
pos_type seekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;

private:
 ios_base::openmode mode; // exposition only
 basic_string<charT, traits, Allocator> buf; // exposition only
 void init_buf_ptrs(); // exposition only
};
}

```

<sup>1</sup> The class `basic_stringbuf` is derived from `basic_streambuf` to associate possibly the input sequence and possibly the output sequence with a sequence of arbitrary *characters*. The sequence can be initialized from, or made available as, an object of class `basic_string`.

<sup>2</sup> For the sake of exposition, the maintained data and internal pointer initialization is presented here as:

(2.1) — `ios_base::openmode mode`, has in set if the input sequence can be read, and out set if the output sequence can be written.

- (2.2) — `basic_string<charT, traits, Allocator> buf` contains the underlying character sequence.
- (2.3) — `init-buf-ptrs()` sets the base class' get area (31.6.3.4.2) and put area (31.6.3.4.3) pointers after initializing, moving from, or assigning to `buf` accordingly.

### 31.8.2.2 Constructors

[stringbuf.cons]

```
explicit basic_stringbuf(ios_base::openmode which);
```

- 1 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), and *mode* with *which*. It is implementation-defined whether the sequence pointers (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) are initialized to null pointers.

- 2 *Postconditions:* `str().empty()` is true.

```
explicit basic_stringbuf(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
```

- 3 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), *mode* with *which*, and *buf* with *s*, then calls `init-buf-ptrs()`.

```
basic_stringbuf(ios_base::openmode which, const Allocator& a);
```

- 4 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), *mode* with *which*, and *buf* with *a*, then calls `init-buf-ptrs()`.

- 5 *Postconditions:* `str().empty()` is true.

```
explicit basic_stringbuf(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
```

- 6 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), *mode* with *which*, and *buf* with `std::move(s)`, then calls `init-buf-ptrs()`.

```
template<class SAlloc>
basic_stringbuf(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
```

- 7 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), *mode* with *which*, and *buf* with `{s,a}`, then calls `init-buf-ptrs()`.

```
template<class SAlloc>
explicit basic_stringbuf(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::in | ios_base::out);
```

- 8 *Constraints:* `is_same_v<SAlloc, Allocator>` is false.

- 9 *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), *mode* with *which*, and *buf* with *s*, then calls `init-buf-ptrs()`.

```
template<class T>
explicit basic_stringbuf(const T& t, ios_base::openmode which = ios_base::in | ios_base::out);
template<class T>
basic_stringbuf(const T& t, const Allocator& a);
template<class T>
basic_stringbuf(const T& t, ios_base::openmode which, const Allocator& a);
```

- 10 Let *which* be `ios_base::in | ios_base::out` for the overload with no parameter *which*, and *a* be `Allocator()` for the overload with no parameter *a*.

- 11 *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

- 12 *Effects:* Creates a variable *sv* as if by `basic_string_view<charT, traits> sv = t`, then value-initializes the base class, initializes *mode* with *which*, and direct-non-list-initializes *buf* with *sv*, *a*, then calls `init-buf-ptrs()`.



```
basic_stringbuf(basic_stringbuf&& rhs);
basic_stringbuf(basic_stringbuf&& rhs, const Allocator& a);
```

13 *Effects:* Copy constructs the base class from `rhs` and initializes `mode` with `rhs.mode`. In the first form `buf` is initialized from `std::move(rhs).str()`. In the second form `buf` is initialized from `{std::move(rhs).str(), a}`. It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had.

14 *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

- (14.1) — `str() == rhs_p.str()`
- (14.2) — `gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`
- (14.3) — `egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`
- (14.4) — `pptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`
- (14.5) — `epptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`
- (14.6) — `if (eback()) eback() != rhs_a.eback()`
- (14.7) — `if (gptr()) gptr() != rhs_a.gptr()`
- (14.8) — `if (egptr()) egptr() != rhs_a.egptr()`
- (14.9) — `if (pbase()) pbase() != rhs_a.pbase()`
- (14.10) — `if (pptr()) pptr() != rhs_a.pptr()`
- (14.11) — `if (epptr()) epptr() != rhs_a.epptr()`
- (14.12) — `getloc() == rhs_p.getloc()`
- (14.13) — `rhs` is empty but usable, as if `std::move(rhs).str()` was called.

### 31.8.2.3 Assignment and swap

[stringbuf.assign]

```
basic_stringbuf& operator=(basic_stringbuf&& rhs);
```

1 *Effects:* After the move assignment `*this` has the observable state it would have had if it had been move constructed from `rhs` (see 31.8.2.2).

2 *Returns:* `*this`.

```
void swap(basic_stringbuf& rhs) noexcept(see below);
```

3 *Preconditions:* `allocator_traits<Allocator>::propagate_on_container_swap::value` is true or `get_allocator() == rhs.get_allocator()` is true.

4 *Effects:* Exchanges the state of `*this` and `rhs`.

5 *Remarks:* The exception specification is equivalent to:

```
allocator_traits<Allocator>::propagate_on_container_swap::value ||
allocator_traits<Allocator>::is_always_equal::value
```

```
template<class charT, class traits, class Allocator>
void swap(basic_stringbuf<charT, traits, Allocator>& x,
 basic_stringbuf<charT, traits, Allocator>& y) noexcept(noexcept(x.swap(y)));
```

6 *Effects:* Equivalent to `x.swap(y)`.

### 31.8.2.4 Member functions

[stringbuf.members]

1 The member functions getting the underlying character sequence all refer to a `high_mark` value, where `high_mark` represents the position one past the highest initialized character in the buffer. Characters can be initialized by writing to the stream, by constructing the `basic_stringbuf` passing a `basic_string` argument, or by calling one of the `str` member functions passing a `basic_string` as an argument. In the latter case, all characters initialized prior to the call are now considered uninitialized (except for those characters re-initialized by the new `basic_string`).

```
void init_buf_ptrs();
```

2 *Effects:* Initializes the input and output sequences from `buf` according to `mode`.



3 *Postconditions:*

(3.1) — If `ios_base::out` is set in *mode*, `pbase()` points to `buf.front()` and `eptr() >= pbase() + buf.size()` is true;

(3.1.1) — in addition, if `ios_base::ate` is set in *mode*, `pptr() == pbase() + buf.size()` is true,

(3.1.2) — otherwise `pptr() == pbase()` is true.

(3.2) — If `ios_base::in` is set in *mode*, `eback()` points to `buf.front()`, and `(gptr() == eback() && egptr() == eback() + buf.size())` is true.

4 [Note 1: For efficiency reasons, stream buffer operations can violate invariants of `buf` while it is held encapsulated in the `basic_stringbuf`, e.g., by writing to characters in the range `[buf.data() + buf.size(), buf.data() + buf.capacity())`. All operations retrieving a `basic_string` from `buf` ensure that the `basic_string` invariants hold on the returned value. — end note]

`allocator_type get_allocator() const noexcept;`

5 *Returns:* `buf.get_allocator()`.

`basic_string<charT, traits, Allocator> str() const &;`

6 *Effects:* Equivalent to:

`return basic_string<charT, traits, Allocator>(view(), get_allocator());`

`template<class SAlloc>`

`basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;`

7 *Constraints:* `SAlloc` is a type that qualifies as an allocator (23.2.2.2).

8 *Effects:* Equivalent to:

`return basic_string<charT, traits, SAlloc>(view(), sa);`

`basic_string<charT, traits, Allocator> str() &&;`

9 *Postconditions:* The underlying character sequence `buf` is empty and `pbase()`, `pptr()`, `eptr()`, `eback()`, `gptr()`, and `egptr()` are initialized as if by calling *init-buf-ptrs()* with an empty `buf`.

10 *Returns:* A `basic_string<charT, traits, Allocator>` object move constructed from the `basic_stringbuf`'s underlying character sequence in `buf`. This can be achieved by first adjusting `buf` to have the same content as `view()`.

`basic_string_view<charT, traits> view() const noexcept;`

11 Let `sv` be `basic_string_view<charT, traits>`.

12 *Returns:* A `sv` object referring to the `basic_stringbuf`'s underlying character sequence in `buf`:

(12.1) — If `ios_base::out` is set in *mode*, then `sv(pbase(), high_mark - pbase())` is returned.

(12.2) — Otherwise, if `ios_base::in` is set in *mode*, then `sv(eback(), egptr() - eback())` is returned.

(12.3) — Otherwise, `sv()` is returned.

13 [Note 2: Using the returned `sv` object after destruction or invalidation of the character sequence underlying *this* is undefined behavior, unless `sv.empty()` is true. — end note]

`void str(const basic_string<charT, traits, Allocator>& s);`

14 *Effects:* Equivalent to:

`buf = s;`  
`init-buf-ptrs();`

`template<class SAlloc>`

`void str(const basic_string<charT, traits, SAlloc>& s);`

15 *Constraints:* `is_same_v<SAlloc, Allocator>` is false.

16 *Effects:* Equivalent to:

`buf = s;`  
`init-buf-ptrs();`

```
void str(basic_string<charT, traits, Allocator>&& s);
```

17 *Effects:* Equivalent to:

```
buf = std::move(s);
init_buf_ptrs();
```

```
template<class T>
void str(const T& t);
```

18 *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

19 *Effects:* Equivalent to:

```
basic_string_view<charT, traits> sv = t;
buf = sv;
init_buf_ptrs();
```

### 31.8.2.5 Overridden virtual functions

[stringbuf.virtuals]

```
int_type underflow() override;
```

1 *Returns:* If the input sequence has a read position available, returns `traits::to_int_type(*gptr())`. Otherwise, returns `traits::eof()`. Any character in the underlying buffer which has been initialized is considered to be part of the input sequence.

```
int_type pbackfail(int_type c = traits::eof()) override;
```

2 *Effects:* Puts back the character designated by `c` to the input sequence, if possible, in one of three ways:

(2.1) — If `traits::eq_int_type(c, traits::eof())` returns false and if the input sequence has a putback position available, and if `traits::eq(to_char_type(c), gptr()[-1])` returns true, assigns `gptr() - 1` to `gptr()`.

Returns: `c`.

(2.2) — If `traits::eq_int_type(c, traits::eof())` returns false and if the input sequence has a putback position available, and if `mode & ios_base::out` is nonzero, assigns `c` to `*--gptr()`.

Returns: `c`.

(2.3) — If `traits::eq_int_type(c, traits::eof())` returns true and if the input sequence has a putback position available, assigns `gptr() - 1` to `gptr()`.

Returns: `traits::not_eof(c)`.

3 *Returns:* As specified above, or `traits::eof()` to indicate failure.

4 *Remarks:* If the function can succeed in more than one of these ways, it is unspecified which way is chosen.

```
int_type overflow(int_type c = traits::eof()) override;
```

5 *Effects:* Appends the character designated by `c` to the output sequence, if possible, in one of two ways:

(5.1) — If `traits::eq_int_type(c, traits::eof())` returns false and if either the output sequence has a write position available or the function makes a write position available (as described below), the function calls `sputc(c)`.

Signals success by returning `c`.

(5.2) — If `traits::eq_int_type(c, traits::eof())` returns true, there is no character to append.

Signals success by returning a value other than `traits::eof()`.

6 *Returns:* As specified above, or `traits::eof()` to indicate failure.

7 *Remarks:* The function can alter the number of write positions available as a result of any call.

8 The function can make a write position available only if `ios_base::out` is set in *mode*. To make a write position available, the function reallocates (or initially allocates) an array object with a sufficient number of elements to hold the current array object (if any), plus at least one additional write position. If `ios_base::in` is set in *mode*, the function alters the read end pointer `egptr()` to point just past the new write position.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
```

- 9 *Effects:* Alters the stream position within one of the controlled sequences, if possible, as indicated in Table 144.

Table 144 — **seekoff** positioning [tab:stringbuf.seekoff.pos]

| Conditions                                                                                                                                                             | Result                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <code>ios_base::in</code> is set in which                                                                                                                              | positions the input sequence                      |
| <code>ios_base::out</code> is set in which                                                                                                                             | positions the output sequence                     |
| both <code>ios_base::in</code> and <code>ios_base::out</code> are set in which and either<br><code>way == ios_base::beg</code> or<br><code>way == ios_base::end</code> | positions both the input and the output sequences |
| Otherwise                                                                                                                                                              | the positioning operation fails.                  |

- 10 For a sequence to be positioned, the function determines **newoff** as indicated in Table 145. If the sequence's next pointer (either `gptr()` or `pptr()`) is a null pointer and **newoff** is nonzero, the positioning operation fails.

Table 145 — **newoff** values [tab:stringbuf.seekoff.newoff]

| Condition                         | <b>newoff</b> Value                                                                  |
|-----------------------------------|--------------------------------------------------------------------------------------|
| <code>way == ios_base::beg</code> | 0                                                                                    |
| <code>way == ios_base::cur</code> | the next pointer minus the beginning pointer ( <code>xnext - xbeg</code> ).          |
| <code>way == ios_base::end</code> | the high mark pointer minus the beginning pointer ( <code>high_mark - xbeg</code> ). |

- 11 If  $(\text{newoff} + \text{off}) < 0$ , or if  $\text{newoff} + \text{off}$  refers to an uninitialized character (31.8.2.4), the positioning operation fails. Otherwise, the function assigns `xbeg + newoff + off` to the next pointer `xnext`.

- 12 *Returns:* `pos_type(newoff)`, constructed from the resultant offset **newoff** (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
```

- 13 *Effects:* Equivalent to `seekoff(off_type(sp), ios_base::beg, which)`.

- 14 *Returns:* `sp` to indicate success, or `pos_type(off_type(-1))` to indicate failure.

```
basic_streambuf<charT, traits>* setbuf(charT* s, streamsize n) override;
```

- 15 *Effects:* implementation-defined, except that `setbuf(0, 0)` has no effect.

- 16 *Returns:* `this`.

### 31.8.3 Class template `basic_istreamream`

[istreamream]

#### 31.8.3.1 General

[istreamream.general]

```
namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_istreamream : public basic_istream<charT, traits> {
 public:
 using char_type = charT;
```

```

using int_type = traits::int_type;
using pos_type = traits::pos_type;
using off_type = traits::off_type;
using traits_type = traits;
using allocator_type = Allocator;

// 31.8.3.2, constructors
basic_istreamstream() : basic_istreamstream(ios_base::in) {}
explicit basic_istreamstream(ios_base::openmode which);
explicit basic_istreamstream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::in);
basic_istreamstream(ios_base::openmode which, const Allocator& a);
explicit basic_istreamstream(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::in);
template<class SAlloc>
 basic_istreamstream(
 const basic_string<charT, traits, SAlloc>& s, const Allocator& a)
 : basic_istreamstream(s, ios_base::in, a) {}
template<class SAlloc>
 basic_istreamstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
template<class SAlloc>
 explicit basic_istreamstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::in);
template<class T>
 explicit basic_istreamstream(const T& t, ios_base::openmode which = ios_base::in);
template<class T>
 basic_istreamstream(const T& t, const Allocator& a);
template<class T>
 basic_istreamstream(const T& t, ios_base::openmode which, const Allocator& a);
basic_istreamstream(const basic_istreamstream&) = delete;
basic_istreamstream(basic_istreamstream&& rhs);

basic_istreamstream& operator=(const basic_istreamstream&) = delete;
basic_istreamstream& operator=(basic_istreamstream&& rhs);

// 31.8.3.3, swap
void swap(basic_istreamstream& rhs);

// 31.8.3.4, members
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
basic_string<charT, traits, Allocator> str() const &;
template<class SAlloc>
 basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
basic_string<charT, traits, Allocator> str() &&;
basic_string_view<charT, traits> view() const noexcept;

void str(const basic_string<charT, traits, Allocator>& s);
template<class SAlloc>
 void str(const basic_string<charT, traits, SAlloc>& s);
void str(basic_string<charT, traits, Allocator>&& s);
template<class T>
 void str(const T& t);

private:
 basic_stringbuf<charT, traits, Allocator> sb; // exposition only
};
}

```

- <sup>1</sup> The class `basic_istream<charT, traits, Allocator>` supports reading objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf<charT, traits, Allocator>` object to control the associated storage. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `stringbuf` object.

### 31.8.3.2 Constructors

[istreamstream.cons]

```
explicit basic_istream(ios_base::openmode which);
```

- <sup>1</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::in)` (31.8.2.2).

```
explicit basic_istream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::in);
```

- <sup>2</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::in)` (31.8.2.2).

```
basic_istream(ios_base::openmode which, const Allocator& a);
```

- <sup>3</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::in, a)` (31.8.2.2).

```
explicit basic_istream(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::in);
```

- <sup>4</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(std::move(s), which | ios_base::in)` (31.8.2.2).

```
template<class SAlloc>
 basic_istream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
```

- <sup>5</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::in, a)` (31.8.2.2).

```
template<class SAlloc>
 explicit basic_istream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::in);
```

- <sup>6</sup> *Constraints:* `is_same_v<SAlloc, Allocator>` is false.

- <sup>7</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::in)` (31.8.2.2).

```
template<class T>
 explicit basic_istream(const T& t, ios_base::openmode which = ios_base::in);
template<class T>
 basic_istream(const T& t, const Allocator& a);
template<class T>
 basic_istream(const T& t, ios_base::openmode which, const Allocator& a);
```

- <sup>8</sup> Let `which` be `ios_base::in` for the overload with no parameter `which`, and `a` be `Allocator()` for the overload with no parameter `a`.

- <sup>9</sup> *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

- <sup>10</sup> *Effects:* Initializes the base class with `addressof(sb)`, and direct-non-list-initializes `sb` with `t`, which | `ios_base::in`, `a`.

```
basic_istringstream(basic_istringstream&& rhs);
```

- 11 *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Then calls `basic_istream<charT, traits>::set_rdbuf(addressof(sb))` to install the contained `basic_stringbuf`.

### 31.8.3.3 Swap

[istringstream.swap]

```
void swap(basic_istringstream& rhs);
```

- 1 *Effects:* Equivalent to:

```
 basic_istream<charT, traits>::swap(rhs);
 sb.swap(rhs.sb);
```

```
template<class charT, class traits, class Allocator>
void swap(basic_istringstream<charT, traits, Allocator>& x,
 basic_istringstream<charT, traits, Allocator>& y);
```

- 2 *Effects:* Equivalent to `x.swap(y)`.

### 31.8.3.4 Member functions

[istringstream.members]

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

- 1 *Returns:* `const_cast<basic_stringbuf<charT, traits, Allocator>*>(addressof(sb))`.

```
basic_string<charT, traits, Allocator> str() const &;
```

- 2 *Effects:* Equivalent to: `return rdbuf()->str();`

```
template<class SAlloc>
basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
```

- 3 *Effects:* Equivalent to: `return rdbuf()->str(sa);`

```
basic_string<charT, traits, Allocator> str() &&;
```

- 4 *Effects:* Equivalent to: `return std::move(*rdbuf()).str();`

```
basic_string_view<charT, traits> view() const noexcept;
```

- 5 *Effects:* Equivalent to: `return rdbuf()->view();`

```
void str(const basic_string<charT, traits, Allocator>& s);
```

- 6 *Effects:* Equivalent to: `rdbuf()->str(s);`

```
template<class SAlloc>
void str(const basic_string<charT, traits, SAlloc>& s);
```

- 7 *Effects:* Equivalent to: `rdbuf()->str(s);`

```
void str(basic_string<charT, traits, Allocator>&& s);
```

- 8 *Effects:* Equivalent to: `rdbuf()->str(std::move(s));`

```
template<class T>
void str(const T& t);
```

- 9 *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

- 10 *Effects:* Equivalent to: `rdbuf()->str(t);`

## 31.8.4 Class template `basic_ostringstream`

[ostringstream]

### 31.8.4.1 General

[ostringstream.general]

```
namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_ostringstream : public basic_ostream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
```

```

using off_type = traits::off_type;
using traits_type = traits;
using allocator_type = Allocator;

// 31.8.4.2, constructors
basic_ostringstream() : basic_ostringstream(ios_base::out) {}
explicit basic_ostringstream(ios_base::openmode which);
explicit basic_ostringstream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::out);
basic_ostringstream(ios_base::openmode which, const Allocator& a);
explicit basic_ostringstream(
 const basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::out);
template<class SAlloc>
 basic_ostringstream(
 const basic_string<charT, traits, SAlloc>& s, const Allocator& a)
 : basic_ostringstream(s, ios_base::out, a) {}
template<class SAlloc>
 basic_ostringstream(
 const basic_string<charT, traits, SAlloc>&& s,
 ios_base::openmode which, const Allocator& a);
template<class SAlloc>
 explicit basic_ostringstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::out);
template<class T>
 explicit basic_ostringstream(const T& t, ios_base::openmode which = ios_base::out);
template<class T>
 basic_ostringstream(const T& t, const Allocator& a);
template<class T>
 basic_ostringstream(const T& t, ios_base::openmode which, const Allocator& a);
basic_ostringstream(const basic_ostringstream&) = delete;
basic_ostringstream(basic_ostringstream&& rhs);

basic_ostringstream& operator=(const basic_ostringstream&) = delete;
basic_ostringstream& operator=(basic_ostringstream&& rhs);

// 31.8.4.3, swap
void swap(basic_ostringstream& rhs);

// 31.8.4.4, members
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;

basic_string<charT, traits, Allocator> str() const &;
template<class SAlloc>
 basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
basic_string<charT, traits, Allocator> str() &&;
basic_string_view<charT, traits> view() const noexcept;

void str(const basic_string<charT, traits, Allocator>& s);
template<class SAlloc>
 void str(const basic_string<charT, traits, SAlloc>& s);
void str(basic_string<charT, traits, Allocator>&& s);
template<class T>
 void str(const T& t);

private:
 basic_stringbuf<charT, traits, Allocator> sb; // exposition only
};
}

```

- <sup>1</sup> The class `basic_ostringstream<charT, traits, Allocator>` supports writing objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf` object to control the associated storage. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `stringbuf` object.

#### 31.8.4.2 Constructors

[`ostringstream.cons`]

```
explicit basic_ostringstream(ios_base::openmode which);
```

- <sup>1</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::out)` (31.8.2.2).

```
explicit basic_ostringstream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::out);
```

- <sup>2</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::out)` (31.8.2.2).

```
basic_ostringstream(ios_base::openmode which, const Allocator& a);
```

- <sup>3</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::out, a)` (31.8.2.2).

```
explicit basic_ostringstream(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::out);
```

- <sup>4</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(std::move(s), which | ios_base::out)` (31.8.2.2).

```
template<class SAlloc>
basic_ostringstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
```

- <sup>5</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::out, a)` (31.8.2.2).

```
template<class SAlloc>
explicit basic_ostringstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::out);
```

- <sup>6</sup> *Constraints:* `is_same_v<SAlloc, Allocator>` is false.

- <sup>7</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which | ios_base::out)` (31.8.2.2).

```
template<class T>
explicit basic_ostringstream(const T& t, ios_base::openmode which = ios_base::out);
template<class T>
basic_ostringstream(const T& t, const Allocator& a);
template<class T>
basic_ostringstream(const T& t, ios_base::openmode which, const Allocator& a);
```

- <sup>8</sup> Let `which` be `ios_base::out` for the overload with no parameter `which`, and `a` be `Allocator()` for the overload with no parameter `a`.

- <sup>9</sup> *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

- <sup>10</sup> *Effects:* Initializes the base class with `addressof(sb)`, and direct-non-list-initializes `sb` with `t`, `which | ios_base::out`, `a`.



```
basic_ostringstream(basic_ostringstream&& rhs);
```

- 11 *Effects:* Move constructs from the rvalue rhs. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Then calls `basic_ostream<charT, traits>::set_rdbuf(addressof(sb))` to install the contained `basic_stringbuf`.

#### 31.8.4.3 Swap

[ostream.swap]

```
void swap(basic_ostringstream& rhs);
```

- 1 *Effects:* Equivalent to:

```
 basic_ostream<charT, traits>::swap(rhs);
 sb.swap(rhs.sb);
```

```
template<class charT, class traits, class Allocator>
void swap(basic_ostringstream<charT, traits, Allocator>& x,
 basic_ostringstream<charT, traits, Allocator>& y);
```

- 2 *Effects:* Equivalent to `x.swap(y)`.

#### 31.8.4.4 Member functions

[ostream.members]

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

- 1 *Returns:* `const_cast<basic_stringbuf<charT, traits, Allocator>*>(addressof(sb))`.

```
basic_string<charT, traits, Allocator> str() const &;
```

- 2 *Effects:* Equivalent to: `return rdbuf()->str();`

```
template<class SAlloc>
basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
```

- 3 *Effects:* Equivalent to: `return rdbuf()->str(sa);`

```
basic_string<charT, traits, Allocator> str() &&;
```

- 4 *Effects:* Equivalent to: `return std::move(*rdbuf()).str();`

```
basic_string_view<charT, traits> view() const noexcept;
```

- 5 *Effects:* Equivalent to: `return rdbuf()->view();`

```
void str(const basic_string<charT, traits, Allocator>& s);
```

- 6 *Effects:* Equivalent to: `rdbuf()->str(s);`

```
template<class SAlloc>
void str(const basic_string<charT, traits, SAlloc>& s);
```

- 7 *Effects:* Equivalent to: `rdbuf()->str(s);`

```
void str(basic_string<charT, traits, Allocator>&& s);
```

- 8 *Effects:* Equivalent to: `rdbuf()->str(std::move(s));`

```
template<class T>
void str(const T& t);
```

- 9 *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

- 10 *Effects:* Equivalent to: `rdbuf()->str(t);`

### 31.8.5 Class template `basic_stringstream`

[stringstream]

#### 31.8.5.1 General

[stringstream.general]

```
namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_stringstream : public basic_istream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
```

```

using off_type = traits::off_type;
using traits_type = traits;
using allocator_type = Allocator;

// 31.8.5.2, constructors
basic_stringstream() : basic_stringstream(ios_base::out | ios_base::in) {}
explicit basic_stringstream(ios_base::openmode which);
explicit basic_stringstream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
basic_stringstream(ios_base::openmode which, const Allocator& a);
explicit basic_stringstream(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
template<class SAlloc>
 basic_stringstream(
 const basic_string<charT, traits, SAlloc>& s, const Allocator& a)
 : basic_stringstream(s, ios_base::out | ios_base::in, a) {}
template<class SAlloc>
 basic_stringstream(
 const basic_string<charT, traits, SAlloc>&& s,
 ios_base::openmode which, const Allocator& a);
template<class SAlloc>
 explicit basic_stringstream(
 const basic_string<charT, traits, SAlloc>&& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
template<class T>
 explicit basic_stringstream(const T& t,
 ios_base::openmode which = ios_base::out | ios_base::in);
template<class T>
 basic_stringstream(const T& t, const Allocator& a);
template<class T>
 basic_stringstream(const T& t, ios_base::openmode which, const Allocator& a);
basic_stringstream(const basic_stringstream&) = delete;
basic_stringstream(basic_stringstream&& rhs);

basic_stringstream& operator=(const basic_stringstream&) = delete;
basic_stringstream& operator=(basic_stringstream&& rhs);

// 31.8.5.3, swap
void swap(basic_stringstream& rhs);

// 31.8.5.4, members
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;

basic_string<charT, traits, Allocator> str() const &;
template<class SAlloc>
 basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
basic_string<charT, traits, Allocator> str() &&;
basic_string_view<charT, traits> view() const noexcept;

void str(const basic_string<charT, traits, Allocator>& s);
template<class SAlloc>
 void str(const basic_string<charT, traits, SAlloc>& s);
void str(basic_string<charT, traits, Allocator>&& s);
template<class T>
 void str(const T& t);

private:
 basic_stringbuf<charT, traits, Allocator> sb; // exposition only
};
}

```

<sup>1</sup> The class template `basic_stringstream<charT, traits>` supports reading and writing from objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf<charT, traits, Allocator>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as

(1.1) — `sb`, the `stringbuf` object.

### 31.8.5.2 Constructors

[`stringstream.cons`]

```
explicit basic_stringstream(ios_base::openmode which);
```

<sup>1</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which)`.

```
explicit basic_stringstream(
 const basic_string<charT, traits, Allocator>& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
```

<sup>2</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which)`.

```
basic_stringstream(ios_base::openmode which, const Allocator& a);
```

<sup>3</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(which, a)` (31.8.2.2).

```
explicit basic_stringstream(
 basic_string<charT, traits, Allocator>&& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
```

<sup>4</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(std::move(s), which)` (31.8.2.2).

```
template<class SAlloc>
basic_stringstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which, const Allocator& a);
```

<sup>5</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which, a)` (31.8.2.2).

```
template<class SAlloc>
explicit basic_stringstream(
 const basic_string<charT, traits, SAlloc>& s,
 ios_base::openmode which = ios_base::out | ios_base::in);
```

<sup>6</sup> *Constraints:* `is_same_v<SAlloc, Allocator>` is false.

<sup>7</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_stringbuf<charT, traits, Allocator>(s, which)` (31.8.2.2).

```
template<class T>
explicit basic_stringstream(const T& t, ios_base::openmode which = ios_base::out | ios_base::in);
template<class T>
basic_stringstream(const T& t, const Allocator& a);
template<class T>
basic_stringstream(const T& t, ios_base::openmode which, const Allocator& a);
```

<sup>8</sup> Let `which` be `ios_base::out | ios_base::in` for the overload with no parameter `which`, and `a` be `Allocator()` for the overload with no parameter `a`.

<sup>9</sup> *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

<sup>10</sup> *Effects:* Initializes the base class with `addressof(sb)`, and direct-non-list-initializes `sb` with `t`, `which`, `a`.

```
basic_stringstream(basic_stringstream&& rhs);
```

<sup>11</sup> *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Then calls `basic_iostream<charT, traits>::set_rdbuf(addressof(sb))` to install the contained `basic_stringbuf`.

**31.8.5.3 Swap****[stringstream.swap]**

```
void swap(basic_stringstream& rhs);
```

1     *Effects:* Equivalent to:

```
 basic_iostream<charT, traits>::swap(rhs);
 sb.swap(rhs.sb);
```

```
template<class charT, class traits, class Allocator>
void swap(basic_stringstream<charT, traits, Allocator>& x,
 basic_stringstream<charT, traits, Allocator>& y);
```

2     *Effects:* Equivalent to `x.swap(y)`.

**31.8.5.4 Member functions****[stringstream.members]**

```
basic_stringbuf<charT, traits, Allocator>* rdbuf() const;
```

1     *Returns:* `const_cast<basic_stringbuf<charT, traits, Allocator>*>(addressof(sb))`.

```
basic_string<charT, traits, Allocator> str() const &;
```

2     *Effects:* Equivalent to: `return rdbuf()->str()`;

```
template<class SAlloc>
basic_string<charT, traits, SAlloc> str(const SAlloc& sa) const;
```

3     *Effects:* Equivalent to: `return rdbuf()->str(sa)`;

```
basic_string<charT, traits, Allocator> str() &&;
```

4     *Effects:* Equivalent to: `return std::move(*rdbuf()).str()`;

```
basic_string_view<charT, traits> view() const noexcept;
```

5     *Effects:* Equivalent to: `return rdbuf()->view()`;

```
void str(const basic_string<charT, traits, Allocator>& s);
```

6     *Effects:* Equivalent to: `rdbuf()->str(s)`;

```
template<class SAlloc>
void str(const basic_string<charT, traits, SAlloc>& s);
```

7     *Effects:* Equivalent to: `rdbuf()->str(s)`;

```
void str(basic_string<charT, traits, Allocator>&& s);
```

8     *Effects:* Equivalent to: `rdbuf()->str(std::move(s))`;

```
template<class T>
void str(const T& t);
```

9     *Constraints:* `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

10    *Effects:* Equivalent to: `rdbuf()->str(t)`;

**31.9 Span-based streams****[span.streams]****31.9.1 Overview****[span.streams.overview]**

1 The header `<spanstream>` defines class templates and types that associate stream buffers with objects whose types are specializations of `span` as described in 23.7.2.2.

[Note 1: A user of these classes is responsible for ensuring that the character sequence represented by the given `span` outlives the use of the sequence by objects of the classes in 31.9. Using multiple `basic_spanbuf` objects referring to overlapping underlying sequences from different threads, where at least one `basic_spanbuf` object is used for writing to the sequence, results in a data race. — end note]

**31.9.2 Header `<spanstream>` synopsis****[spanstream.syn]**

```
namespace std {
 // 31.9.3, class template basic_spanbuf
 template<class charT, class traits = char_traits<charT>>
 class basic_spanbuf;
```

```

template<class charT, class traits>
 void swap(basic_spanbuf<charT, traits>& x, basic_spanbuf<charT, traits>& y);

using spanbuf = basic_spanbuf<char>;
using wspanbuf = basic_spanbuf<wchar_t>;

// 31.9.4, class template basic_ispanstream
template<class charT, class traits = char_traits<charT>>
 class basic_ispanstream;

template<class charT, class traits>
 void swap(basic_ispanstream<charT, traits>& x, basic_ispanstream<charT, traits>& y);

using ispanstream = basic_ispanstream<char>;
using wspanstream = basic_ispanstream<wchar_t>;

// 31.9.5, class template basic_ostream
template<class charT, class traits = char_traits<charT>>
 class basic_ostream;

template<class charT, class traits>
 void swap(basic_ostream<charT, traits>& x, basic_ostream<charT, traits>& y);

using ostream = basic_ostream<char>;
using wostream = basic_ostream<wchar_t>;

// 31.9.6, class template basic_spanstream
template<class charT, class traits = char_traits<charT>>
 class basic_spanstream;

template<class charT, class traits>
 void swap(basic_spanstream<charT, traits>& x, basic_spanstream<charT, traits>& y);

using spanstream = basic_spanstream<char>;
using wspanstream = basic_spanstream<wchar_t>;
}

```

### 31.9.3 Class template basic\_spanbuf

[spanbuf]

#### 31.9.3.1 General

[spanbuf.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_spanbuf
 : public basic_streambuf<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;

 // 31.9.3.2, constructors
 basic_spanbuf() : basic_spanbuf(ios_base::in | ios_base::out) {}
 explicit basic_spanbuf(ios_base::openmode which)
 : basic_spanbuf(std::span<charT>(), which) {}
 explicit basic_spanbuf(std::span<charT> s,
 ios_base::openmode which = ios_base::in | ios_base::out);
 basic_spanbuf(const basic_spanbuf&) = delete;
 basic_spanbuf(basic_spanbuf&& rhs);

 // 31.9.3.3, assignment and swap
 basic_spanbuf& operator=(const basic_spanbuf&) = delete;
 basic_spanbuf& operator=(basic_spanbuf&& rhs);
 void swap(basic_spanbuf& rhs);

```

```

// 31.9.3.4, member functions
std::span<charT> span() const noexcept;
void span(std::span<charT> s) noexcept;

protected:
// 31.9.3.5, overridden virtual functions
basic_streambuf<charT, traits>* setbuf(charT*, streamsize) override;
pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which = ios_base::in | ios_base::out) override;
pos_type seekpos(pos_type sp,
 ios_base::openmode which = ios_base::in | ios_base::out) override;

private:
 ios_base::openmode mode; // exposition only
 std::span<charT> buf; // exposition only
};
}

```

<sup>1</sup> The class template `basic_spanbuf` is derived from `basic_streambuf` to associate possibly the input sequence and possibly the output sequence with a sequence of arbitrary characters. The sequence is provided by an object of class `span<charT>`.

<sup>2</sup> For the sake of exposition, the maintained data is presented here as:

- (2.1) — `ios_base::openmode mode`, has `in` set if the input sequence can be read, and `out` set if the output sequence can be written.
- (2.2) — `std::span<charT> buf` is the view to the underlying character sequence.

### 31.9.3.2 Constructors

[spanbuf.cons]

```

explicit basic_spanbuf(std::span<charT> s,
 ios_base::openmode which = ios_base::in | ios_base::out);

```

<sup>1</sup> *Effects:* Initializes the base class with `basic_streambuf()` (31.6.3.2), and `mode` with `which`. Initializes the internal pointers as if calling `span(s)`.

```

basic_spanbuf(basic_spanbuf&& rhs);

```

<sup>2</sup> *Effects:* Initializes the base class with `std::move(rhs)` and `mode` with `std::move(rhs.mode)` and `buf` with `std::move(rhs.buf)`. The sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. It is implementation-defined whether `rhs.buf.empty()` returns true after the move.

<sup>3</sup> *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction.

- (3.1) — `span().data() == rhs_p.span().data()`
- (3.2) — `span().size() == rhs_p.span().size()`
- (3.3) — `eback() == rhs_p.eback()`
- (3.4) — `gptr() == rhs_p.gptr()`
- (3.5) — `egptr() == rhs_p.egptr()`
- (3.6) — `pbase() == rhs_p.pbase()`
- (3.7) — `pptr() == rhs_p.pptr()`
- (3.8) — `epptr() == rhs_p.epptr()`
- (3.9) — `getloc() == rhs_p.getloc()`

### 31.9.3.3 Assignment and swap

[spanbuf.assign]

```

basic_spanbuf& operator=(basic_spanbuf&& rhs);

```

<sup>1</sup> *Effects:* Equivalent to:

```

 basic_spanbuf tmp{std::move(rhs)};
 this->swap(tmp);
 return *this;

```

```
void swap(basic_spanbuf& rhs);
```

2 *Effects:* Equivalent to:

```
 basic_streambuf<charT, traits>::swap(rhs);
 std::swap(mode, rhs.mode);
 std::swap(buf, rhs.buf);
```

```
template<class charT, class traits>
```

```
 void swap(basic_spanbuf<charT, traits>& x, basic_spanbuf<charT, traits>& y);
```

3 *Effects:* Equivalent to `x.swap(y)`.

### 31.9.3.4 Member functions

[spanbuf.members]

```
std::span<charT> span() const noexcept;
```

1 *Returns:* If `ios_base::out` is set in *mode*, returns `std::span<charT>(pbase(), pptr())`, otherwise returns *buf*.

[Note 1: In contrast to `basic_stringbuf`, the underlying sequence never grows and is not owned. An owning copy can be obtained by converting the result to `basic_string<charT>`. — end note]

```
void span(std::span<charT> s) noexcept;
```

2 *Effects:* *buf* = *s*. Initializes the input and output sequences according to *mode*.

3 *Postconditions:*

(3.1) — If `ios_base::out` is set in *mode*, `pbase() == s.data() && epptr() == pbase() + s.size()` is true;

(3.1.1) — in addition, if `ios_base::ate` is set in *mode*, `pptr() == pbase() + s.size()` is true,

(3.1.2) — otherwise `pptr() == pbase()` is true.

(3.2) — If `ios_base::in` is set in *mode*, `eback() == s.data() && gptra() == eback() && egptr() == eback() + s.size()` is true.

### 31.9.3.5 Overridden virtual functions

[spanbuf.virtuals]

1 [Note 1: Because the underlying buffer is of fixed size, neither `overflow`, `underflow`, nor `pbackfail` can provide useful behavior. — end note]

```
pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which = ios_base::in | ios_base::out) override;
```

2 *Effects:* Alters the stream position within one or both of the controlled sequences, if possible, as follows:

(2.1) — If `ios_base::in` is set in *which*, positions the input sequence; *xnext* is `gptra()`, *xbeg* is `eback()`.

(2.2) — If `ios_base::out` is set in *which*, positions the output sequence; *xnext* is `pptra()`, *xbeg* is `pbase()`.

3 If both `ios_base::in` and `ios_base::out` are set in *which* and *way* is `ios_base::cur`, the positioning operation fails.

4 For a sequence to be positioned, if its next pointer *xnext* (either `gptra()` or `pptra()`) is a null pointer and the new offset *newoff* as computed below is nonzero, the positioning operation fails. Otherwise, the function determines *baseoff* as a value of type *off\_type* as follows:

(4.1) — 0 when *way* is `ios_base::beg`;

(4.2) — `(pptra() - pbase())` for the output sequence, or `(gptra() - eback())` for the input sequence when *way* is `ios_base::cur`;

(4.3) — when *way* is `ios_base::end`:

(4.3.1) — `(pptra() - pbase())` if `ios_base::out` is set in *mode* and `ios_base::in` is not set in *mode*,

(4.3.2) — `buf.size()` otherwise.

5 If *baseoff* + *off* would overflow, or if *baseoff* + *off* is less than zero, or if *baseoff* + *off* is greater than `buf.size()`, the positioning operation fails. Otherwise, the function computes

```
 off_type newoff = baseoff + off;
```

and assigns `xbeg + newoff` to the next pointer `xnext`.

6 *Returns:* `pos_type(off_type(-1))` if the positioning operation fails; `pos_type(newoff)` otherwise.

`pos_type seekpos(pos_type sp, ios_base::openmode which = ios_base::in | ios_base::out) override;`

7 *Effects:* Equivalent to:

```
return seekoff(off_type(sp), ios_base::beg, which);
```

`basic_streambuf<charT, traits>* setbuf(charT* s, streamsize n) override;`

8 *Effects:* Equivalent to:

```
this->span(std::span<charT>(s, n));
return this;
```

### 31.9.4 Class template `basic_ispanstream`

[`ispanstream`]

#### 31.9.4.1 General

[`ispanstream.general`]

```
namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_ispanstream
 : public basic_istream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;

 // 31.9.4.2, constructors
 explicit basic_ispanstream(std::span<charT> s,
 ios_base::openmode which = ios_base::in);
 basic_ispanstream(const basic_ispanstream&) = delete;
 basic_ispanstream(basic_ispanstream&& rhs);
 template<class ROS> explicit basic_ispanstream(ROS&& s);

 basic_ispanstream& operator=(const basic_ispanstream&) = delete;
 basic_ispanstream& operator=(basic_ispanstream&& rhs);

 // 31.9.4.3, swap
 void swap(basic_ispanstream& rhs);

 // 31.9.4.4, member functions
 basic_spanbuf<charT, traits>* rdbuf() const noexcept;

 std::span<const charT> span() const noexcept;
 void span(std::span<charT> s) noexcept;
 template<class ROS> void span(ROS&& s) noexcept;

 private:
 basic_spanbuf<charT, traits> sb; // exposition only
 };
}
```

1 [Note 1: Constructing an `ispanstream` from a *string-literal* includes the termination character `'\0'` in the underlying `spanbuf`. — end note]

#### 31.9.4.2 Constructors

[`ispanstream.cons`]

`explicit basic_ispanstream(std::span<charT> s, ios_base::openmode which = ios_base::in);`

1 *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(s, which | ios_base::in)` (31.9.3.2).



```
basic_ispanstream(basic_ispanstream&& rhs);
```

- 2 *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_istream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

```
template<class ROS> explicit basic_ispanstream(ROS&& s)
```

- 3 *Constraints:* ROS models `ranges::borrowed_range`. `!convertible_to<ROS, std::span<charT>>` && `convertible_to<ROS, std::span<charT const>>` is true.

- 4 *Effects:* Let `sp` be `std::span<const charT>(std::forward<ROS>(s))`. Equivalent to:

```
basic_ispanstream(std::span<charT>(const_cast<charT*>(sp.data()), sp.size()))
```

### 31.9.4.3 Swap

[ispanstream.swap]

```
void swap(basic_ispanstream& rhs);
```

- 1 *Effects:* Equivalent to:

```
basic_istream<charT, traits>::swap(rhs);
sb.swap(rhs.sb);
```

```
template<class charT, class traits>
```

```
void swap(basic_ispanstream<charT, traits>& x, basic_ispanstream<charT, traits>& y);
```

- 2 *Effects:* Equivalent to `x.swap(y)`.

### 31.9.4.4 Member functions

[ispanstream.members]

```
basic_spanbuf<charT, traits>* rdbuf() const noexcept;
```

- 1 *Effects:* Equivalent to:

```
return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));
```

```
std::span<const charT> span() const noexcept;
```

- 2 *Effects:* Equivalent to: `return rdbuf()->span();`

```
void span(std::span<charT> s) noexcept;
```

- 3 *Effects:* Equivalent to `rdbuf()->span(s)`.

```
template<class ROS> void span(ROS&& s) noexcept;
```

- 4 *Constraints:* ROS models `ranges::borrowed_range`. (`!convertible_to<ROS, std::span<charT>>`) && `convertible_to<ROS, std::span<const charT>>` is true.

- 5 *Effects:* Let `sp` be `std::span<const charT>(std::forward<ROS>(s))`. Equivalent to:

```
this->span(std::span<charT>(const_cast<charT*>(sp.data()), sp.size()));
```

## 31.9.5 Class template basic\_ostream

[ostream]

### 31.9.5.1 General

[ostream.general]

```
namespace std {
```

```
template<class charT, class traits = char_traits<charT>>
```

```
class basic_ostream
```

```
: public basic_ostream<charT, traits> {
```

```
public:
```

```
using char_type = charT;
```

```
using int_type = traits::int_type;
```

```
using pos_type = traits::pos_type;
```

```
using off_type = traits::off_type;
```

```
using traits_type = traits;
```

```
// 31.9.5.2, constructors
```

```
explicit basic_ostream(std::span<charT> s,
```

```
ios_base::openmode which = ios_base::out);
```

```
basic_ostream(const basic_ostream&) = delete;
```

```
basic_ostream(basic_ostream&& rhs);
```

```

basic_ostream& operator=(const basic_ostream&) = delete;
basic_ostream& operator=(basic_ostream&& rhs);

// 31.9.5.3, swap
void swap(basic_ostream& rhs);

// 31.9.5.4, member functions
basic_spanbuf<charT, traits>* rdbuf() const noexcept;

std::span<charT> span() const noexcept;
void span(std::span<charT> s) noexcept;

private:
 basic_spanbuf<charT, traits> sb; // exposition only
};
}

```

### 31.9.5.2 Constructors

[ostream.cons]

```

explicit basic_ostream(std::span<charT> s,
 ios_base::openmode which = ios_base::out);

```

- 1 *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(s, which | ios_base::out)` (31.9.3.2).

```

basic_ostream(basic_ostream&& rhs) noexcept;

```

- 2 *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_ostream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

### 31.9.5.3 Swap

[ostream.swap]

```

void swap(basic_ostream& rhs);

```

- 1 *Effects:* Equivalent to:
- ```

    basic_ostream<charT, traits>::swap(rhs);
    sb.swap(rhs.sb);

```

```

template<class charT, class traits>
void swap(basic_ostream<charT, traits>& x, basic_ostream<charT, traits>& y);

```

- 2 *Effects:* Equivalent to `x.swap(y)`.

31.9.5.4 Member functions

[ostream.members]

```

basic_spanbuf<charT, traits>* rdbuf() const noexcept;

```

- 1 *Effects:* Equivalent to:
- ```

 return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));

```

```

std::span<charT> span() const noexcept;

```

- 2 *Effects:* Equivalent to: `return rdbuf()->span();`

```

void span(std::span<charT> s) noexcept;

```

- 3 *Effects:* Equivalent to `rdbuf()->span(s)`.

## 31.9.6 Class template basic\_spanstream

[spanstream]

### 31.9.6.1 General

[spanstream.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_spanstream
 : public basic_istream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;

```

```

using pos_type = traits::pos_type;
using off_type = traits::off_type;
using traits_type = traits;

// 31.9.6.2, constructors
explicit basic_spanstream(std::span<charT> s,
 ios_base::openmode which = ios_base::out | ios_base::in);
basic_spanstream(const basic_spanstream&) = delete;
basic_spanstream(basic_spanstream&& rhs);

basic_spanstream& operator=(const basic_spanstream&) = delete;
basic_spanstream& operator=(basic_spanstream&& rhs);

// 31.9.6.3, swap
void swap(basic_spanstream& rhs);

// 31.9.6.4, members
basic_spanbuf<charT, traits>* rdbuf() const noexcept;

std::span<charT> span() const noexcept;
void span(std::span<charT> s) noexcept;

private:
 basic_spanbuf<charT, traits> sb; // exposition only
};
}

```

### 31.9.6.2 Constructors

[spanstream.cons]

```

explicit basic_spanstream(std::span<charT> s,
 ios_base::openmode which = ios_base::out | ios_base::in);

```

- 1 *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` and `sb` with `basic_spanbuf<charT, traits>(s, which)` (31.9.3.2).

```

basic_spanstream(basic_spanstream&& rhs);

```

- 2 *Effects:* Initializes the base class with `std::move(rhs)` and `sb` with `std::move(rhs.sb)`. Next, `basic_iostream<charT, traits>::set_rdbuf(addressof(sb))` is called to install the contained `basic_spanbuf`.

### 31.9.6.3 Swap

[spanstream.swap]

```

void swap(basic_spanstream& rhs);

```

- 1 *Effects:* Equivalent to:
- ```

    basic_iostream<charT, traits>::swap(rhs);
    sb.swap(rhs.sb);

```

```

template<class charT, class traits>
void swap(basic_spanstream<charT, traits>& x, basic_spanstream<charT, traits>& y);

```

- 2 *Effects:* Equivalent to `x.swap(y)`.

31.9.6.4 Member functions

[spanstream.members]

```

basic_spanbuf<charT, traits>* rdbuf() const noexcept;

```

- 1 *Effects:* Equivalent to:
- ```

 return const_cast<basic_spanbuf<charT, traits>*>(addressof(sb));

```

```

std::span<charT> span() const noexcept;

```

- 2 *Effects:* Equivalent to: `return rdbuf()->span();`

```

void span(std::span<charT> s) noexcept;

```

- 3 *Effects:* Equivalent to `rdbuf()->span(s)`.

## 31.10 File-based streams

[file.streams]

### 31.10.1 Header <fstream> synopsis

[fstream.syn]

```

namespace std {
 // 31.10.3, class template basic_filebuf
 template<class charT, class traits = char_traits<charT>>
 class basic_filebuf;

 template<class charT, class traits>
 void swap(basic_filebuf<charT, traits>& x, basic_filebuf<charT, traits>& y);

 using filebuf = basic_filebuf<char>;
 using wfilebuf = basic_filebuf<wchar_t>;

 // 31.10.4, class template basic_ifstream
 template<class charT, class traits = char_traits<charT>>
 class basic_ifstream;

 template<class charT, class traits>
 void swap(basic_ifstream<charT, traits>& x, basic_ifstream<charT, traits>& y);

 using ifstream = basic_ifstream<char>;
 using wifstream = basic_ifstream<wchar_t>;

 // 31.10.5, class template basic_ofstream
 template<class charT, class traits = char_traits<charT>>
 class basic_ofstream;

 template<class charT, class traits>
 void swap(basic_ofstream<charT, traits>& x, basic_ofstream<charT, traits>& y);

 using ofstream = basic_ofstream<char>;
 using wofstream = basic_ofstream<wchar_t>;

 // 31.10.6, class template basic_fstream
 template<class charT, class traits = char_traits<charT>>
 class basic_fstream;

 template<class charT, class traits>
 void swap(basic_fstream<charT, traits>& x, basic_fstream<charT, traits>& y);

 using fstream = basic_fstream<char>;
 using wfstream = basic_fstream<wchar_t>;
}

```

- <sup>1</sup> The header <fstream> defines four class templates and eight types that associate stream buffers with files and assist reading and writing files.
- <sup>2</sup> [Note 1: The class template `basic_filebuf` treats a file as a source or sink of bytes. In an environment that uses a large character set, the file typically holds multibyte character sequences and the `basic_filebuf` object converts those multibyte sequences into wide character sequences. — end note]
- <sup>3</sup> In subclause 31.10, member functions taking arguments of `const filesystem::path::value_type*` are only provided on systems where `filesystem::path::value_type` (31.12.6) is not `char`.  
[Note 2: These functions enable class `path` support for systems with a wide native path character type, such as `wchar_t`. — end note]

### 31.10.2 Native handles

[file.native]

- <sup>1</sup> Several classes described in 31.10 have a member `native_handle_type`.
- <sup>2</sup> The type `native_handle_type` represents a platform-specific *native handle* to a file. It is trivially copyable and models `semiregular`.  
[Note 1: For operating systems based on POSIX, `native_handle_type` is `int`. For Windows-based operating systems, `native_handle_type` is `HANDLE`. — end note]

31.10.3 Class template `basic_filebuf`

[filebuf]

## 31.10.3.1 General

[filebuf.general]

```

namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_filebuf : public basic_streambuf<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;
 using native_handle_type = implementation-defined; // see 31.10.2

 // 31.10.3.2, constructors/destructor
 basic_filebuf();
 basic_filebuf(const basic_filebuf&) = delete;
 basic_filebuf(basic_filebuf&& rhs);
 virtual ~basic_filebuf();

 // 31.10.3.3, assignment and swap
 basic_filebuf& operator=(const basic_filebuf&) = delete;
 basic_filebuf& operator=(basic_filebuf&& rhs);
 void swap(basic_filebuf& rhs);

 // 31.10.3.4, members
 bool is_open() const;
 basic_filebuf* open(const char* s, ios_base::openmode mode);
 basic_filebuf* open(const filesystem::path::value_type* s,
 ios_base::openmode mode); // wide systems only; see 31.10.1
 basic_filebuf* open(const string& s, ios_base::openmode mode);
 basic_filebuf* open(const filesystem::path& s, ios_base::openmode mode);
 basic_filebuf* close();
 native_handle_type native_handle() const noexcept;

 protected:
 // 31.10.3.5, overridden virtual functions
 streamsize showmanyc() override;
 int_type underflow() override;
 int_type uflow() override;
 int_type pbackfail(int_type c = traits::eof()) override;
 int_type overflow (int_type c = traits::eof()) override;

 basic_streambuf<charT, traits>* setbuf(char_type* s, streamsize n) override;

 pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which = ios_base::in | ios_base::out) override;
 pos_type seekpos(pos_type sp,
 ios_base::openmode which = ios_base::in | ios_base::out) override;

 int sync() override;
 void imbue(const locale& loc) override;
 };
}

```

- <sup>1</sup> The class `basic_filebuf<charT, traits>` associates both the input sequence and the output sequence with a file.
- <sup>2</sup> The restrictions on reading and writing a sequence controlled by an object of class `basic_filebuf<charT, traits>` are the same as for reading and writing with the C standard library `FILEs`.
- <sup>3</sup> In particular:
  - (3.1) — If the file is not open for reading the input sequence cannot be read.
  - (3.2) — If the file is not open for writing the output sequence cannot be written.

- (3.3) — A joint file position is maintained for both the input sequence and the output sequence.
- 4 An instance of `basic_filebuf` behaves as described in 31.10.3 provided `traits::pos_type` is `fpos<traits::state_type>`. Otherwise the behavior is undefined.
- 5 The file associated with a `basic_filebuf` has an associated value of type `native_handle_type`, called the native handle (31.10.2) of that file. This native handle can be obtained by calling the member function `native_handle`.
- 6 For any opened `basic_filebuf` `f`, the native handle returned by `f.native_handle()` is invalidated when `f.close()` is called, or `f` is destroyed.
- 7 In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as `a_codecvt` in following subclauses, obtained as if by

```
const codecvt<charT, char, typename traits::state_type>& a_codecvt =
 use_facet<codecvt<charT, char, typename traits::state_type>>(getloc());
```

### 31.10.3.2 Constructors

[filebuf.cons]

```
basic_filebuf();
```

- 1 *Effects:* Initializes the base class with `basic_streambuf<charT, traits>()` (31.6.3.2).

- 2 *Postconditions:* `is_open() == false`.

```
basic_filebuf(basic_filebuf&& rhs);
```

- 3 *Effects:* It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. Whether they do or not, `*this` and `rhs` reference separate buffers (if any at all) after the construction. Additionally `*this` references the file which `rhs` did before the construction, and `rhs` references no file after the construction. The openmode, locale and any other state of `rhs` is also copied.

- 4 *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

- (4.1) — `is_open() == rhs_p.is_open()`
- (4.2) — `rhs_a.is_open() == false`
- (4.3) — `gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`
- (4.4) — `egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`
- (4.5) — `pptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`
- (4.6) — `epptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`
- (4.7) — `if (eback()) eback() != rhs_a.eback()`
- (4.8) — `if (gptr()) gptr() != rhs_a.gptr()`
- (4.9) — `if (egptr()) egptr() != rhs_a.egptr()`
- (4.10) — `if (pbase()) pbase() != rhs_a.pbase()`
- (4.11) — `if (pptr()) pptr() != rhs_a.pptr()`
- (4.12) — `if (epptr()) epptr() != rhs_a.epptr()`

```
virtual ~basic_filebuf();
```

- 5 *Effects:* Calls `close()`. If an exception occurs during the destruction of the object, including the call to `close()`, the exception is caught but not rethrown (see 16.4.6.14).

### 31.10.3.3 Assignment and swap

[filebuf.assign]

```
basic_filebuf& operator=(basic_filebuf&& rhs);
```

- 1 *Effects:* Calls `close()` then move assigns from `rhs`. After the move assignment `*this` has the observable state it would have had if it had been move constructed from `rhs` (see 31.10.3.2).

- 2 *Returns:* `*this`.

```
void swap(basic_filebuf& rhs);
```

3 *Effects:* Exchanges the state of *\*this* and *rhs*.

```
template<class charT, class traits>
void swap(basic_filebuf<charT, traits>& x, basic_filebuf<charT, traits>& y);
```

4 *Effects:* Equivalent to *x.swap(y)*.

### 31.10.3.4 Member functions

[filebuf.members]

```
bool is_open() const;
```

1 *Returns:* **true** if a previous call to **open** succeeded (returned a non-null value) and there has been no intervening call to **close**.

```
basic_filebuf* open(const char* s, ios_base::openmode mode);
basic_filebuf* open(const filesystem::path::value_type* s,
 ios_base::openmode mode); // wide systems only; see 31.10.1
```

2 *Preconditions:* *s* points to an NTCTS (3.36).

3 *Effects:* If **is\_open()** **!= false**, returns a null pointer. Otherwise, initializes the **filebuf** as required. It then opens the file to which *s* resolves, if possible, as if by a call to **fopen** with the second argument determined from *mode* & *~ios\_base::ate* as indicated in Table 146. If *mode* is not some combination of flags shown in the table then the open fails.

Table 146 — File open modes [tab:filebuf.open.modes]

| binary | ios_base flag combination |     |       |     |           | stdio equivalent |
|--------|---------------------------|-----|-------|-----|-----------|------------------|
|        | in                        | out | trunc | app | noreplace |                  |
|        |                           | +   |       |     |           | "w"              |
|        |                           | +   |       |     | +         | "wx"             |
|        |                           | +   | +     |     |           | "w"              |
|        |                           | +   | +     |     | +         | "wx"             |
|        |                           | +   |       | +   |           | "a"              |
|        |                           |     |       | +   |           | "a"              |
|        | +                         |     |       |     |           | "r"              |
|        | +                         | +   |       |     |           | "r+"             |
|        | +                         | +   | +     |     |           | "w+"             |
|        | +                         | +   | +     |     | +         | "w+x"            |
|        | +                         | +   |       | +   |           | "a+"             |
|        | +                         |     |       | +   |           | "a+"             |
| +      |                           | +   |       |     |           | "wb"             |
| +      |                           | +   |       |     | +         | "wbx"            |
| +      |                           | +   | +     |     |           | "wb"             |
| +      |                           | +   | +     |     | +         | "wbx"            |
| +      |                           | +   |       | +   |           | "ab"             |
| +      |                           |     |       | +   |           | "ab"             |
| +      | +                         |     |       |     |           | "rb"             |
| +      | +                         | +   |       |     |           | "r+b"            |
| +      | +                         | +   | +     |     |           | "w+b"            |
| +      | +                         | +   | +     |     | +         | "w+bx"           |
| +      | +                         | +   |       | +   |           | "a+b"            |
| +      | +                         |     |       | +   |           | "a+b"            |

4 If the open operation succeeds and *ios\_base::ate* is set in *mode*, positions the file to the end (as if by calling **fseek(file, 0, SEEK\_END)**, where *file* is the pointer returned by calling **fopen**).<sup>271</sup>

5 If the repositioning operation fails, calls **close()** and returns a null pointer to indicate failure.

<sup>271</sup> The macro **SEEK\_END** is defined, and the function signatures **fopen(const char\*, const char\*)** and **fseek(FILE\*, long, int)** are declared, in **<cstdio>** (31.13.1).

6 *Returns:* `this` if successful, a null pointer otherwise.

```
basic_filebuf* open(const string& s, ios_base::openmode mode);
basic_filebuf* open(const filesystem::path& s, ios_base::openmode mode);
```

7 *Returns:* `open(s.c_str(), mode)`;

```
basic_filebuf* close();
```

8 *Effects:* If `is_open() == false`, returns a null pointer. If a put area exists, calls `overflow(trait::eof())` to flush characters. If the last virtual member function called on `*this` (between `underflow`, `overflow`, `seekoff`, and `seekpos`) was `overflow` then calls `a_codecvt.unshift` (possibly several times) to determine a termination sequence, inserts those characters and calls `overflow(trait::eof())` again. Finally, regardless of whether any of the preceding calls fails or throws an exception, the function closes the file (as if by calling `fclose(file)`). If any of the calls made by the function, including `fclose`, fails, `close` fails by returning a null pointer. If one of these calls throws an exception, the exception is caught and rethrown after closing the file.

9 *Postconditions:* `is_open() == false`.

10 *Returns:* `this` on success, a null pointer otherwise.

```
native_handle_type native_handle() const noexcept;
```

11 *Preconditions:* `is_open()` is true.

12 *Returns:* The native handle associated with `*this`.

### 31.10.3.5 Overridden virtual functions

[filebuf.virtuals]

```
streamsize showmanyc() override;
```

1 *Effects:* Behaves the same as `basic_streambuf::showmanyc()` (31.6.3.5).

2 *Remarks:* An implementation may provide an overriding definition for this function signature if it can determine whether more characters can be read from the input sequence.

```
int_type underflow() override;
```

3 *Effects:* Behaves according to the description of `basic_streambuf<charT, traits>::underflow()`, with the specialization that a sequence of characters is read from the input sequence as if by reading from the associated file into an internal buffer (`extern_buf`) and then as if by doing:

```
char extern_buf[XSIZE];
const char* extern_end;
charT intern_buf[ISIZE];
charT* intern_end;
codecvt_base::result r =
 a_codecvt.in(state, extern_buf, extern_buf+XSIZE, extern_end,
 intern_buf, intern_buf+ISIZE, intern_end);
```

This shall be done in such a way that the class can recover the position (`fpos_t`) corresponding to each character between `intern_buf` and `intern_end`. If the value of `r` indicates that `a_codecvt.in()` ran out of space in `intern_buf`, retry with a larger `intern_buf`.

```
int_type uflow() override;
```

4 *Effects:* Behaves according to the description of `basic_streambuf<charT, traits>::uflow()`, with the specialization that a sequence of characters is read from the input with the same method as used by `underflow`.

```
int_type pbackfail(int_type c = trait::eof()) override;
```

5 *Effects:* Puts back the character designated by `c` to the input sequence, if possible, in one of three ways:

- (5.1) — If `trait::eq_int_type(c, trait::eof())` returns false and if the function makes a putback position available and if `trait::eq(to_char_type(c), gptr()[-1])` returns true, decrements the next pointer for the input sequence, `gptr()`.

Returns: `c`.



- (5.2) — If `traits::eq_int_type(c, traits::eof())` returns `false` and if the function makes a putback position available and if the function is permitted to assign to the putback position, decrements the next pointer for the input sequence, and stores `c` there.

Returns: `c`.

- (5.3) — If `traits::eq_int_type(c, traits::eof())` returns `true`, and if either the input sequence has a putback position available or the function makes a putback position available, decrements the next pointer for the input sequence, `gptr()`.

Returns: `traits::not_eof(c)`.

6 *Returns:* As specified above, or `traits::eof()` to indicate failure.

7 *Remarks:* If `is_open() == false`, the function always fails.

8 The function does not put back a character directly to the input sequence.

9 If the function can succeed in more than one of these ways, it is unspecified which way is chosen. The function can alter the number of putback positions available as a result of any call.

`int_type overflow(int_type c = traits::eof()) override;`

- 10 *Effects:* Behaves according to the description of `basic_streambuf<charT, traits>::overflow(c)`, except that the behavior of “consuming characters” is performed by first converting as if by:

```
charT* b = pbase();
charT* p = pptr();
const charT* end;
char xbuf[XSIZE];
char* xbuf_end;
codecvt_base::result r =
 a_codecvt.out(state, b, p, end, xbuf, xbuf+XSIZE, xbuf_end);
```

and then

- (10.1) — If `r == codecvt_base::error` then fail.
- (10.2) — If `r == codecvt_base::noconv` then output characters from `b` up to (and not including) `p`.
- (10.3) — If `r == codecvt_base::partial` then output to the file characters from `xbuf` up to `xbuf_end`, and repeat using characters from `end` to `p`. If output fails, fail (without repeating).
- (10.4) — Otherwise output from `xbuf` to `xbuf_end`, and fail if output fails. At this point if `b != p` and `b == end` (`xbuf` isn't large enough) then increase `XSIZE` and repeat from the beginning.

Then establishes an observable checkpoint (4.1.2).

- 11 *Returns:* `traits::not_eof(c)` to indicate success, and `traits::eof()` to indicate failure. If `is_open() == false`, the function always fails.

`basic_streambuf* setbuf(char_type* s, streamsize n) override;`

- 12 *Effects:* If `setbuf(0, 0)` is called on a stream before any I/O has occurred on that stream, the stream becomes unbuffered. Otherwise the results are implementation-defined. “Unbuffered” means that `pbase()` and `pptr()` always return null and output to the file should appear as soon as possible.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
```

- 13 *Effects:* Let `width` denote `a_codecvt.encoding()`. If `is_open() == false`, or `off != 0 && width <= 0`, then the positioning operation fails. Otherwise, if `way != basic_ios::cur` or `off != 0`, and if the last operation was output, then update the output sequence and write any unshift sequence. Next, seek to the new position: if `width > 0`, call `fseek(file, width * off, whence)`, otherwise call `fseek(file, 0, whence)`.

- 14 *Returns:* A newly constructed `pos_type` object that stores the resultant stream position, if possible. If the positioning operation fails, or if the object cannot represent the resultant stream position, returns `pos_type(off_type(-1))`.

- 15 *Remarks:* “The last operation was output” means either the last virtual operation was overflow or the put buffer is non-empty. “Write any unshift sequence” means, if `width` is less than zero then

call `a_codecvt.unshift(state, xbuf, xbuf+XSIZE, xbuf_end)` and output the resulting unshift sequence. The function determines one of three values for the argument `whence`, of type `int`, as indicated in Table 147.

Table 147 — seekoff effects [tab:filebuf.seekoff]

| way                         | Value | stdio Equivalent      |
|-----------------------------|-------|-----------------------|
| <code>basic_ios::beg</code> |       | <code>SEEK_SET</code> |
| <code>basic_ios::cur</code> |       | <code>SEEK_CUR</code> |
| <code>basic_ios::end</code> |       | <code>SEEK_END</code> |

```
pos_type seekpos(pos_type sp,
 ios_base::openmode which
 = ios_base::in | ios_base::out) override;
```

16 Alters the file position, if possible, to correspond to the position stored in `sp` (as described below). Altering the file position performs as follows:

1. if `(om & ios_base::out) != 0`, then update the output sequence and write any unshift sequence;
2. set the file position to `sp` as if by a call to `fsetpos`;
3. if `(om & ios_base::in) != 0`, then update the input sequence;

where `om` is the open mode passed to the last call to `open()`. The operation fails if `is_open()` returns `false`.

17 If `sp` is an invalid stream position, or if the function positions neither sequence, the positioning operation fails. If `sp` has not been obtained by a previous successful call to one of the positioning functions (`seekoff` or `seekpos`) on the same file the effects are undefined.

18 *Returns:* `sp` on success. Otherwise returns `pos_type(off_type(-1))`.

```
int sync() override;
```

19 *Effects:* If a put area exists, calls `filebuf::overflow` to write the characters to the file, then flushes the file as if by calling `fflush(file)`. If a get area exists, the effect is implementation-defined.

```
void imbue(const locale& loc) override;
```

20 *Preconditions:* If the file is not positioned at its beginning and the encoding of the current locale as determined by `a_codecvt.encoding()` is state-dependent (28.3.4.2.5.3) then that facet is the same as the corresponding facet of `loc`.

21 *Effects:* Causes characters inserted or extracted after this call to be converted according to `loc` until another call of `imbue`.

22 *Remarks:* This may require reconversion of previously converted characters. This in turn may require the implementation to be able to reconstruct the original contents of the file.

### 31.10.4 Class template `basic_ifstream`

[`ifstream`]

#### 31.10.4.1 General

[`ifstream.general`]

```
namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_ifstream : public basic_istream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;
 using native_handle_type = basic_filebuf<charT, traits>::native_handle_type;

 // 31.10.4.2, constructors
 basic_ifstream();
```

```

explicit basic_ifstream(const char* s,
 ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in); // wide systems only; see 31.10.1
explicit basic_ifstream(const string& s,
 ios_base::openmode mode = ios_base::in);
template<class T>
 explicit basic_ifstream(const T& s, ios_base::openmode mode = ios_base::in);
basic_ifstream(const basic_ifstream&) = delete;
basic_ifstream(basic_ifstream&& rhs);

basic_ifstream& operator=(const basic_ifstream&) = delete;
basic_ifstream& operator=(basic_ifstream&& rhs);

// 31.10.4.3, swap
void swap(basic_ifstream& rhs);

// 31.10.4.4, members
basic_filebuf<charT, traits>* rdbuf() const;
native_handle_type native_handle() const noexcept;

bool is_open() const;
void open(const char* s, ios_base::openmode mode = ios_base::in);
void open(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in); // wide systems only; see 31.10.1
void open(const string& s, ios_base::openmode mode = ios_base::in);
void open(const filesystem::path& s, ios_base::openmode mode = ios_base::in);
void close();

private:
 basic_filebuf<charT, traits> sb; // exposition only
};
}

```

- <sup>1</sup> The class `basic_ifstream<charT, traits>` supports reading from named files. It uses a `basic_filebuf<charT, traits>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the filebuf object.

### 31.10.4.2 Constructors

[ifstream.cons]

`basic_ifstream();`

- <sup>1</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2.2) and `sb` with `basic_filebuf<charT, traits>()` (31.10.3.2).

```

explicit basic_ifstream(const char* s,
 ios_base::openmode mode = ios_base::in);
explicit basic_ifstream(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in); // wide systems only; see 31.10.1

```

- <sup>2</sup> *Effects:* Initializes the base class with `basic_istream<charT, traits>(addressof(sb))` (31.7.5.2.2) and `sb` with `basic_filebuf<charT, traits>()` (31.10.3.2), then calls `rdbuf()->open(s, mode | ios_base::in)`. If that function returns a null pointer, calls `setstate(failbit)`.

```

explicit basic_ifstream(const string& s,
 ios_base::openmode mode = ios_base::in);

```

- <sup>3</sup> *Effects:* Equivalent to `basic_ifstream(s.c_str(), mode)`.

```

template<class T>
 explicit basic_ifstream(const T& s, ios_base::openmode mode = ios_base::in);

```

- <sup>4</sup> *Constraints:* `is_same_v<T, filesystem::path>` is true.

- <sup>5</sup> *Effects:* Equivalent to `basic_ifstream(s.c_str(), mode)`.

```
basic_ifstream(basic_ifstream&& rhs);
```

- 6 *Effects:* Move constructs the base class, and the contained `basic_filebuf`. Then calls `basic_ifstream<charT, traits>::set_rdbuf(addressof(sb))` to install the contained `basic_filebuf`.

### 31.10.4.3 Swap

[ifstream.swap]

```
void swap(basic_ifstream& rhs);
```

- 1 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_ifstream<charT, traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template<class charT, class traits>
```

```
void swap(basic_ifstream<charT, traits>& x, basic_ifstream<charT, traits>& y);
```

- 2 *Effects:* Equivalent to `x.swap(y)`.

### 31.10.4.4 Member functions

[ifstream.members]

```
basic_filebuf<charT, traits>* rdbuf() const;
```

- 1 *Returns:* `const_cast<basic_filebuf<charT, traits>*>(addressof(sb))`.

```
native_handle_type native_handle() const noexcept;
```

- 2 *Effects:* Equivalent to: `return rdbuf()->native_handle();`

```
bool is_open() const;
```

- 3 *Returns:* `rdbuf()->is_open()`.

```
void open(const char* s, ios_base::openmode mode = ios_base::in);
```

```
void open(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in); // wide systems only; see 31.10.1
```

- 4 *Effects:* Calls `rdbuf()->open(s, mode | ios_base::in)`. If that function does not return a null pointer calls `clear()`, otherwise calls `setstate(failbit)` (which may throw `ios_base::failure`) (31.5.4.4).

```
void open(const string& s, ios_base::openmode mode = ios_base::in);
```

```
void open(const filesystem::path& s, ios_base::openmode mode = ios_base::in);
```

- 5 *Effects:* Calls `open(s.c_str(), mode)`.

```
void close();
```

- 6 *Effects:* Calls `rdbuf()->close()` and, if that function returns a null pointer, calls `setstate(failbit)` (which may throw `ios_base::failure`) (31.5.4.4).

## 31.10.5 Class template `basic_ofstream`

[ofstream]

### 31.10.5.1 General

[ofstream.general]

```
namespace std {
```

```
template<class charT, class traits = char_traits<charT>>
```

```
class basic_ofstream : public basic_ostream<charT, traits> {
```

```
public:
```

```
using char_type = charT;
```

```
using int_type = traits::int_type;
```

```
using pos_type = traits::pos_type;
```

```
using off_type = traits::off_type;
```

```
using traits_type = traits;
```

```
using native_handle_type = basic_filebuf<charT, traits>::native_handle_type;
```

```
// 31.10.5.2, constructors
```

```
basic_ofstream();
```

```
explicit basic_ofstream(const char* s,
 ios_base::openmode mode = ios_base::out);
```

```
explicit basic_ofstream(const filesystem::path::value_type* s, // wide systems only; see 31.10.1
 ios_base::openmode mode = ios_base::out);
```

```
explicit basic_ofstream(const string& s,
 ios_base::openmode mode = ios_base::out);
```

```

template<class T>
 explicit basic_ofstream(const T& s, ios_base::openmode mode = ios_base::out);
basic_ofstream(const basic_ofstream&) = delete;
basic_ofstream(basic_ofstream&& rhs);

basic_ofstream& operator=(const basic_ofstream&) = delete;
basic_ofstream& operator=(basic_ofstream&& rhs);

// 31.10.5.3, swap
void swap(basic_ofstream& rhs);

// 31.10.5.4, members
basic_filebuf<charT, traits>* rdbuf() const;
native_handle_type native_handle() const noexcept;

bool is_open() const;
void open(const char* s, ios_base::openmode mode = ios_base::out);
void open(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::out); // wide systems only; see 31.10.1
void open(const string& s, ios_base::openmode mode = ios_base::out);
void open(const filesystem::path& s, ios_base::openmode mode = ios_base::out);
void close();

private:
 basic_filebuf<charT, traits> sb; // exposition only
};
}

```

- <sup>1</sup> The class `basic_ofstream<charT, traits>` supports writing to named files. It uses a `basic_filebuf<charT, traits>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the filebuf object.

### 31.10.5.2 Constructors

[ofstream.cons]

```
basic_ofstream();
```

- <sup>1</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2.2) and `sb` with `basic_filebuf<charT, traits>()` (31.10.3.2).

```

explicit basic_ofstream(const char* s,
 ios_base::openmode mode = ios_base::out);
explicit basic_ofstream(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::out); // wide systems only; see 31.10.1

```

- <sup>2</sup> *Effects:* Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))` (31.7.6.2.2) and `sb` with `basic_filebuf<charT, traits>()` (31.10.3.2), then calls `rdbuf()->open(s, mode | ios_base::out)`. If that function returns a null pointer, calls `setstate(failbit)`.

```

explicit basic_ofstream(const string& s,
 ios_base::openmode mode = ios_base::out);

```

- <sup>3</sup> *Effects:* Equivalent to `basic_ofstream(s.c_str(), mode)`.

```

template<class T>
 explicit basic_ofstream(const T& s, ios_base::openmode mode = ios_base::out);

```

- <sup>4</sup> *Constraints:* `is_same_v<T, filesystem::path>` is true.

- <sup>5</sup> *Effects:* Equivalent to `basic_ofstream(s.c_str(), mode)`.

```
basic_ofstream(basic_ofstream&& rhs);
```

- <sup>6</sup> *Effects:* Move constructs the base class, and the contained `basic_filebuf`. Then calls `basic_ostream<charT, traits>::set_rdbuf(addressof(sb))` to install the contained `basic_filebuf`.

## 31.10.5.3 Swap

[ofstream.swap]

```
void swap(basic_ofstream& rhs);
```

- 1 *Effects:* Exchanges the state of *\*this* and *rhs* by calling `basic_ofstream<charT, traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template<class charT, class traits>
void swap(basic_ofstream<charT, traits>& x, basic_ofstream<charT, traits>& y);
```

- 2 *Effects:* Equivalent to `x.swap(y)`.

## 31.10.5.4 Member functions

[ofstream.members]

```
basic_filebuf<charT, traits>* rdbuf() const;
```

- 1 *Returns:* `const_cast<basic_filebuf<charT, traits>*>(addressof(sb))`.

```
native_handle_type native_handle() const noexcept;
```

- 2 *Effects:* Equivalent to: `return rdbuf()->native_handle();`

```
bool is_open() const;
```

- 3 *Returns:* `rdbuf()->is_open()`.

```
void open(const char* s, ios_base::openmode mode = ios_base::out);
```

```
void open(const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::out); // wide systems only; see 31.10.1
```

- 4 *Effects:* Calls `rdbuf()->open(s, mode | ios_base::out)`. If that function does not return a null pointer calls `clear()`, otherwise calls `setstate(failbit)` (which may throw `ios_base::failure`) (31.5.4.4).

```
void close();
```

- 5 *Effects:* Calls `rdbuf()->close()` and, if that function fails (returns a null pointer), calls `setstate(failbit)` (which may throw `ios_base::failure`) (31.5.4.4).

```
void open(const string& s, ios_base::openmode mode = ios_base::out);
```

```
void open(const filesystem::path& s, ios_base::openmode mode = ios_base::out);
```

- 6 *Effects:* Calls `open(s.c_str(), mode)`.

## 31.10.6 Class template basic\_fstream

[fstream]

## 31.10.6.1 General

[fstream.general]

```
namespace std {
 template<class charT, class traits = char_traits<charT>>
 class basic_fstream : public basic_iostream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;
 using native_handle_type = basic_filebuf<charT, traits>::native_handle_type;

 // 31.10.6.2, constructors
 basic_fstream();
 explicit basic_fstream(
 const char* s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
 explicit basic_fstream(
 const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in | ios_base::out); // wide systems only; see 31.10.1
 explicit basic_fstream(
 const string& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
 };
}
```

```

template<class T>
 explicit basic_fstream(const T& s, ios_base::openmode mode = ios_base::in | ios_base::out);
basic_fstream(const basic_fstream&) = delete;
basic_fstream(basic_fstream&& rhs);

basic_fstream& operator=(const basic_fstream&) = delete;
basic_fstream& operator=(basic_fstream&& rhs);

// 31.10.6.3, swap
void swap(basic_fstream& rhs);

// 31.10.6.4, members
basic_filebuf<charT, traits>* rdbuf() const;
native_handle_type native_handle() const noexcept;

bool is_open() const;
void open(
 const char* s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
void open(
 const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in | ios_base::out); // wide systems only; see 31.10.1
void open(
 const string& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
void open(
 const filesystem::path& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
void close();

private:
 basic_filebuf<charT, traits> sb; // exposition only
};
}

```

- <sup>1</sup> The class template `basic_fstream<charT, traits>` supports reading and writing from named files. It uses a `basic_filebuf<charT, traits>` object to control the associated sequences. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `basic_filebuf` object.

### 31.10.6.2 Constructors

[fstream.cons]

`basic_fstream();`

- <sup>1</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_filebuf<charT, traits>()`.

```

explicit basic_fstream(
 const char* s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
explicit basic_fstream(
 const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in | ios_base::out); // wide systems only; see 31.10.1

```

- <sup>2</sup> *Effects:* Initializes the base class with `basic_iostream<charT, traits>(addressof(sb))` (31.7.5.7.2) and `sb` with `basic_filebuf<charT, traits>()`. Then calls `rdbuf()->open(s, mode)`. If that function returns a null pointer, calls `setstate(failbit)`.

```

explicit basic_fstream(
 const string& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);

```

- <sup>3</sup> *Effects:* Equivalent to `basic_fstream(s.c_str(), mode)`.

```

template<class T>
 explicit basic_fstream(const T& s, ios_base::openmode mode = ios_base::in | ios_base::out);
4 Constraints: is_same_v<T, filesystem::path> is true.
5 Effects: Equivalent to basic_fstream(s.c_str(), mode).

basic_fstream(basic_fstream&& rhs);
6 Effects: Move constructs the base class, and the contained basic_filebuf. Then calls basic_
 istream<charT, traits>::set_rdbuf(addressof(sb)) to install the contained basic_filebuf.

```

### 31.10.6.3 Swap

[fstream.swap]

```

void swap(basic_fstream& rhs);
1 Effects: Exchanges the state of *this and rhs by calling basic_istream<charT, traits>::swap(rhs)
 and sb.swap(rhs.sb).

template<class charT, class traits>
 void swap(basic_fstream<charT, traits>& x,
 basic_fstream<charT, traits>& y);
2 Effects: Equivalent to x.swap(y).

```

### 31.10.6.4 Member functions

[fstream.members]

```

basic_filebuf<charT, traits>* rdbuf() const;
1 Returns: const_cast<basic_filebuf<charT, traits>*>(addressof(sb)).

native_handle_type native_handle() const noexcept;
2 Effects: Equivalent to: return rdbuf()->native_handle();

bool is_open() const;
3 Returns: rdbuf()->is_open().

void open(
 const char* s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
void open(
 const filesystem::path::value_type* s,
 ios_base::openmode mode = ios_base::in | ios_base::out); // wide systems only; see 31.10.1
4 Effects: Calls rdbuf()->open(s, mode). If that function does not return a null pointer calls clear(),
 otherwise calls setstate(failbit) (which may throw ios_base::failure) (31.5.4.4).

void open(
 const string& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
void open(
 const filesystem::path& s,
 ios_base::openmode mode = ios_base::in | ios_base::out);
5 Effects: Calls open(s.c_str(), mode).

void close();
6 Effects: Calls rdbuf()->close() and, if that function returns a null pointer, calls setstate(failbit)
 (which may throw ios_base::failure) (31.5.4.4).

```

## 31.11 Synchronized output streams

[syncstream]

### 31.11.1 Header <syncstream> synopsis

[syncstream.syn]

```

#include <ostream> // see 31.7.2

namespace std {
 // 31.11.2, class template basic_syncbuf
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_syncbuf;

```



```

// 31.11.2.6, specialized algorithms
template<class charT, class traits, class Allocator>
 void swap(basic_syncbuf<charT, traits, Allocator>&,
 basic_syncbuf<charT, traits, Allocator>&);

using syncbuf = basic_syncbuf<char>;
using wsyncbuf = basic_syncbuf<wchar_t>;

// 31.11.3, class template basic_osyncstream
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_osyncstream;

using ostream = basic_osyncstream<char>;
using wostream = basic_osyncstream<wchar_t>;
}

```

- <sup>1</sup> The header <syncstream> provides a mechanism to synchronize execution agents writing to the same stream.

### 31.11.2 Class template basic\_syncbuf

[syncstream.syncbuf]

#### 31.11.2.1 Overview

[syncstream.syncbuf.overview]

```

namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_syncbuf : public basic_streambuf<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;
 using allocator_type = Allocator;

 using streambuf_type = basic_streambuf<charT, traits>;

 // 31.11.2.2, construction and destruction
 basic_syncbuf()
 : basic_syncbuf(nullptr) {}
 explicit basic_syncbuf(streambuf_type* obuf)
 : basic_syncbuf(obuf, Allocator()) {}
 basic_syncbuf(streambuf_type*, const Allocator&);
 basic_syncbuf(basic_syncbuf&&);
 ~basic_syncbuf();

 // 31.11.2.3, assignment and swap
 basic_syncbuf& operator=(basic_syncbuf&&);
 void swap(basic_syncbuf&);

 // 31.11.2.4, member functions
 bool emit();
 streambuf_type* get_wrapped() const noexcept;
 allocator_type get_allocator() const noexcept;
 void set_emit_on_sync(bool) noexcept;

 protected:
 // 31.11.2.5, overridden virtual functions
 int sync() override;

 private:
 streambuf_type* wrapped; // exposition only
 bool emit_on_sync{}; // exposition only
 };
}

```

- <sup>1</sup> Class template basic\_syncbuf stores character data written to it, known as the associated output, into internal buffers allocated using the object's allocator. The associated output is transferred to the wrapped

stream buffer object *\*wrapped* when `emit()` is called or when the `basic_syncbuf` object is destroyed. Such transfers are atomic with respect to transfers by other `basic_syncbuf` objects with the same wrapped stream buffer object.

### 31.11.2.2 Construction and destruction

[syncstream.syncbuf.cons]

```
basic_syncbuf(streambuf_type* obuf, const Allocator& allocator);
```

1 *Effects:* Sets *wrapped* to *obuf*.

2 *Postconditions:* `get_wrapped() == obuf` and `get_allocator() == allocator` are true.

3 *Throws:* Nothing unless an exception is thrown by the construction of a mutex or by memory allocation.

4 *Remarks:* A copy of `allocator` is used to allocate memory for internal buffers holding the associated output.

```
basic_syncbuf(basic_syncbuf&& other);
```

5 *Postconditions:* The value returned by `this->get_wrapped()` is the value returned by `other.get_wrapped()` prior to calling this constructor. Output stored in `other` prior to calling this constructor will be stored in *\*this* afterwards. `other.pbase() == other.pptr()` and `other.get_wrapped() == nullptr` are true.

6 *Remarks:* This constructor disassociates `other` from its wrapped stream buffer, ensuring destruction of `other` produces no output.

```
~basic_syncbuf();
```

7 *Effects:* Calls `emit()`.

8 *Throws:* Nothing. If an exception is thrown from `emit()`, the destructor catches and ignores that exception.

### 31.11.2.3 Assignment and swap

[syncstream.syncbuf.assign]

```
basic_syncbuf& operator=(basic_syncbuf&& rhs);
```

1 *Effects:* Calls `emit()` then move assigns from `rhs`. After the move assignment *\*this* has the observable state it would have had if it had been move constructed from `rhs` (31.11.2.2).

2 *Postconditions:*

(2.1) — `rhs.get_wrapped() == nullptr` is true.

(2.2) — `this->get_allocator() == rhs.get_allocator()` is true when  
`allocator_traits<Allocator>::propagate_on_container_move_assignment::value`  
is true; otherwise, the allocator is unchanged.

3 *Returns:* *\*this*.

4 *Remarks:* This assignment operator disassociates `rhs` from its wrapped stream buffer, ensuring destruction of `rhs` produces no output.

```
void swap(basic_syncbuf& other);
```

5 *Preconditions:* Either `allocator_traits<Allocator>::propagate_on_container_swap::value` is true or `this->get_allocator() == other.get_allocator()` is true.

6 *Effects:* Exchanges the state of *\*this* and `other`.

### 31.11.2.4 Member functions

[syncstream.syncbuf.members]

```
bool emit();
```

1 *Effects:* Atomically transfers the associated output of *\*this* to the stream buffer *\*wrapped*, so that it appears in the output stream as a contiguous sequence of characters. `wrapped->pubsync()` is called if and only if a call was made to `sync()` since the most recent call to `emit()`, if any.

2 *Synchronization:* All `emit()` calls transferring characters to the same stream buffer object appear to execute in a total order consistent with the “happens before” relation (6.10.2.2), where each `emit()` call synchronizes with subsequent `emit()` calls in that total order.

3 *Postconditions:* On success, the associated output is empty.

4 *Returns:* **true** if all of the following conditions hold; otherwise **false**:

(4.1) — *wrapped* == nullptr is false.

(4.2) — All of the characters in the associated output were successfully transferred.

(4.3) — The call to *wrapped*→pubsync() (if any) succeeded.

5 *Remarks:* May call member functions of *wrapped* while holding a lock uniquely associated with *wrapped*.

```
streambuf_type* get_wrapped() const noexcept;
```

6 *Returns:* *wrapped*.

```
allocator_type get_allocator() const noexcept;
```

7 *Returns:* A copy of the allocator that was set in the constructor or assignment operator.

```
void set_emit_on_sync(bool b) noexcept;
```

8 *Effects:* *emit-on-sync* = b.

### 31.11.2.5 Overridden virtual functions

[syncstream.syncbuf.virtuals]

```
int sync() override;
```

1 *Effects:* Records that the wrapped stream buffer is to be flushed. Then, if *emit-on-sync* is **true**, calls *emit*().

[Note 1: If *emit-on-sync* is **false**, the actual flush is delayed until a call to *emit*(). — end note]

2 *Returns:* If *emit*() was called and returned **false**, returns -1; otherwise 0.

### 31.11.2.6 Specialized algorithms

[syncstream.syncbuf.special]

```
template<class charT, class traits, class Allocator>
void swap(basic_syncbuf<charT, traits, Allocator>& a,
 basic_syncbuf<charT, traits, Allocator>& b);
```

1 *Effects:* Equivalent to a.swap(b).

## 31.11.3 Class template basic\_osyncstream

[syncstream.osyncstream]

### 31.11.3.1 Overview

[syncstream.osyncstream.overview]

```
namespace std {
 template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
 class basic_osyncstream : public basic_ostream<charT, traits> {
 public:
 using char_type = charT;
 using int_type = traits::int_type;
 using pos_type = traits::pos_type;
 using off_type = traits::off_type;
 using traits_type = traits;

 using allocator_type = Allocator;
 using streambuf_type = basic_streambuf<charT, traits>;
 using syncbuf_type = basic_syncbuf<charT, traits, Allocator>;

 // 31.11.3.2, construction and destruction
 basic_osyncstream(streambuf_type*, const Allocator&);
 explicit basic_osyncstream(streambuf_type* obuf)
 : basic_osyncstream(obuf, Allocator()) {}
 basic_osyncstream(basic_ostream<charT, traits>& os, const Allocator& allocator)
 : basic_osyncstream(os.rdbuf(), allocator) {}
 explicit basic_osyncstream(basic_ostream<charT, traits>& os)
 : basic_osyncstream(os, Allocator()) {}
 basic_osyncstream(basic_osyncstream&&) noexcept;
 ~basic_osyncstream();
 };
}
```

```

// assignment
basic_osyncstream& operator=(basic_osyncstream&&);

// 31.11.3.3, member functions
void emit();
streambuf_type* get_wrapped() const noexcept;
syncbuf_type* rdbuf() const noexcept { return const_cast<syncbuf_type*>(addressof(sb)); }

private:
 syncbuf_type sb; // exposition only
};
}

```

<sup>1</sup> Allocator shall meet the *Cpp17Allocator* requirements (16.4.4.6.1).

<sup>2</sup> [Example 1: A named variable can be used within a block statement for streaming.

```

{
 ostream bout(cout);
 bout << "Hello, ";
 bout << "World!";
 bout << endl; // flush is noted
 bout << "and more!\n";
} // characters are transferred and cout is flushed

```

— end example]

<sup>3</sup> [Example 2: A temporary object can be used for streaming within a single statement.

```
ostream(cout) << "Hello, " << "World!" << '\n';
```

In this example, cout is not flushed. — end example]

### 31.11.3.2 Construction and destruction

[syncstream.osyncstream.cons]

```
basic_osyncstream(streambuf_type* buf, const Allocator& allocator);
```

<sup>1</sup> *Effects:* Initializes *sb* from *buf* and *allocator*. Initializes the base class with `basic_ostream<charT, traits>(addressof(sb))`.

<sup>2</sup> [Note 1: The member functions of the provided stream buffer can be called from `emit()` while a lock is held, which might result in a deadlock if used incautiously. — end note]

<sup>3</sup> *Postconditions:* `get_wrapped() == buf` is true.

```
basic_osyncstream(basic_osyncstream&& other) noexcept;
```

<sup>4</sup> *Effects:* Move constructs the base class and *sb* from the corresponding subobjects of *other*, and calls `basic_ostream<charT, traits>::set_rdbuf(addressof(sb))`.

<sup>5</sup> *Postconditions:* The value returned by `get_wrapped()` is the value returned by `other.get_wrapped()` prior to calling this constructor. `nullptr == other.get_wrapped()` is true.

### 31.11.3.3 Member functions

[syncstream.osyncstream.members]

```
void emit();
```

<sup>1</sup> *Effects:* Behaves as an unformatted output function (31.7.6.4). After constructing a `sentry` object, calls `sb.emit()`. If that call returns `false`, calls `setstate(ios_base::badbit)`.

<sup>2</sup> [Example 1: A flush on a `basic_osyncstream` does not flush immediately:

```

{
 ostream bout(cout);
 bout << "Hello," << '\n'; // no flush
 bout.emit(); // characters transferred; cout not flushed
 bout << "World!" << endl; // flush noted; cout not flushed
 bout.emit(); // characters transferred; cout flushed
 bout << "Greetings." << '\n'; // no flush
} // characters transferred; cout not flushed

```

— end example]

3 [Example 2: The function `emit()` can be used to handle exceptions from operations on the underlying stream.

```
{
 ostream bout(cout);
 bout << "Hello, " << "World!" << '\n';
 try {
 bout.emit();
 } catch (...) {
 // handle exception
 }
}
```

— end example]

`streambuf_type* get_wrapped() const noexcept;`

4 Returns: `sb.get_wrapped()`.

5 [Example 3: Obtaining the wrapped stream buffer with `get_wrapped()` allows wrapping it again with an `ostream`. For example,

```
{
 ostream bout1(cout);
 bout1 << "Hello, ";
 {
 ostream(bout1.get_wrapped()) << "Goodbye, " << "Planet!" << '\n';
 }
 bout1 << "World!" << '\n';
}
```

produces the *uninterleaved* output

```
Goodbye, Planet!
Hello, World!
```

— end example]

## 31.12 File systems

[filesystems]

### 31.12.1 General

[fs.general]

- 1 Subclause 31.12 describes operations on file systems and their components, such as paths, regular files, and directories.
- 2 A *file system* is a collection of files and their attributes.
- 3 A *file* is an object within a file system that holds user or system data. Files can be written to, or read from, or both. A file has certain attributes, including type. File types include regular files and directories. Other types of files, such as symbolic links, may be supported by the implementation.
- 4 A *directory* is a file within a file system that acts as a container of directory entries that contain information about other files, possibly including other directory files. The *parent directory* of a directory is the directory that both contains a directory entry for the given directory and is represented by the dot-dot filename (31.12.6.2) in the given directory. The *parent directory* of other types of files is a directory containing a directory entry for the file under discussion.
- 5 A *link* is an object that associates a filename with a file. Several links can associate names with the same file. A *hard link* is a link to an existing file. Some file systems support multiple hard links to a file. If the last hard link to a file is removed, the file itself is removed.

[Note 1: A hard link can be thought of as a shared-ownership smart pointer to a file. — end note]

A *symbolic link* is a type of file with the property that when the file is encountered during pathname resolution (31.12.6), a string stored by the file is used to modify the pathname resolution.

[Note 2: Symbolic links are often called symlinks. A symbolic link can be thought of as a raw pointer to a file. If the file pointed to does not exist, the symbolic link is said to be a “dangling” symbolic link. — end note]

### 31.12.2 Conformance

[fs.conformance]

#### 31.12.2.1 General

[fs.conformance.general]

- 1 Conformance is specified in terms of behavior. Ideal behavior is not always implementable, so the conformance subclauses take that into account.

**31.12.2.2 POSIX conformance****[fs.conform.9945]**

- <sup>1</sup> Some behavior is specified by reference to POSIX. How such behavior is actually implemented is unspecified.  
[*Note 1*: This constitutes an “as if” rule allowing implementations to call native operating system or other APIs. — *end note*]
- <sup>2</sup> Implementations should provide such behavior as it is defined by POSIX. Implementations shall document any behavior that differs from the behavior defined by POSIX. Implementations that do not support exact POSIX behavior should provide behavior as close to POSIX behavior as is reasonable given the limitations of actual operating systems and file systems. If an implementation cannot provide any reasonable behavior, the implementation shall report an error as specified in 31.12.5.  
[*Note 2*: This allows users to rely on an exception being thrown or an error code being set when an implementation cannot provide any reasonable behavior. — *end note*]
- <sup>3</sup> Implementations are not required to provide behavior that is not supported by a particular file system.  
[*Example 1*: The FAT file system used by some memory cards, camera memory, and floppy disks does not support hard links, symlinks, and many other features of more capable file systems, so implementations are not required to support those features on the FAT file system but instead are required to report an error as described above. — *end example*]

**31.12.2.3 Operating system dependent behavior conformance****[fs.conform.os]**

- <sup>1</sup> Behavior that is specified as being *operating system dependent* is dependent upon the behavior and characteristics of an operating system. The operating system an implementation is dependent upon is implementation-defined.
- <sup>2</sup> It is permissible for an implementation to be dependent upon an operating system emulator rather than the actual underlying operating system.

**31.12.2.4 File system race behavior****[fs.race.behavior]**

- <sup>1</sup> A *file system race* is the condition that occurs when multiple threads, processes, or computers interleave access and modification of the same object within a file system. Behavior is undefined if calls to functions provided by subclause 31.12 introduce a file system race.
- <sup>2</sup> If the possibility of a file system race would make it unreliable for a program to test for a precondition before calling a function described herein, *Preconditions*: is not specified for the function.  
[*Note 1*: As a design practice, preconditions are not specified when it is unreasonable for a program to detect them prior to calling the function. — *end note*]

**31.12.3 Requirements****[fs.req]**

- <sup>1</sup> Throughout subclause 31.12, `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t` are collectively called *encoded character types*.
- <sup>2</sup> Functions with template parameters named `EcharT` shall not participate in overload resolution unless `EcharT` is one of the encoded character types.
- <sup>3</sup> Template parameters named `InputIterator` shall meet the *Cpp17InputIterator* requirements (24.3.5.3) and shall have a value type that is one of the encoded character types.
- <sup>4</sup> [*Note 1*: Use of an encoded character type implies an associated character set and encoding. Since `signed char` and `unsigned char` have no implied character set and encoding, they are not included as permitted types. — *end note*]
- <sup>5</sup> Template parameters named `Allocator` shall meet the *Cpp17Allocator* requirements (16.4.4.6.1).

**31.12.4 Header `<filesystem>` synopsis****[fs.filesystem.syn]**

```
#include <compare> // see 17.12.1

namespace std::filesystem {
 // 31.12.6, paths
 class path;

 // 31.12.6.8, path non-member functions
 void swap(path& lhs, path& rhs) noexcept;
 size_t hash_value(const path& p) noexcept;
```

```

// 31.12.7, filesystem errors
class filesystem_error;

// 31.12.10, directory entries
class directory_entry;

// 31.12.11, directory iterators
class directory_iterator;

// 31.12.11.3, range access for directory iterators
directory_iterator begin(directory_iterator iter) noexcept;
directory_iterator end(directory_iterator) noexcept;

// 31.12.12, recursive directory iterators
class recursive_directory_iterator;

// 31.12.12.3, range access for recursive directory iterators
recursive_directory_iterator begin(recursive_directory_iterator iter) noexcept;
recursive_directory_iterator end(recursive_directory_iterator) noexcept;

// 31.12.9, file status
class file_status;

struct space_info {
 uintmax_t capacity;
 uintmax_t free;
 uintmax_t available;

 friend bool operator==(const space_info&, const space_info&) = default;
};

// 31.12.8, enumerations
enum class file_type;
enum class perms;
enum class perm_options;
enum class copy_options;
enum class directory_options;

using file_time_type = chrono::time_point<chrono::file_clock>;

// 31.12.13, filesystem operations
path absolute(const path& p);
path absolute(const path& p, error_code& ec);

path canonical(const path& p);
path canonical(const path& p, error_code& ec);

void copy(const path& from, const path& to);
void copy(const path& from, const path& to, error_code& ec);
void copy(const path& from, const path& to, copy_options options);
void copy(const path& from, const path& to, copy_options options,
 error_code& ec);

bool copy_file(const path& from, const path& to);
bool copy_file(const path& from, const path& to, error_code& ec);
bool copy_file(const path& from, const path& to, copy_options option);
bool copy_file(const path& from, const path& to, copy_options option,
 error_code& ec);

void copy_symlink(const path& existing_symlink, const path& new_symlink);
void copy_symlink(const path& existing_symlink, const path& new_symlink,
 error_code& ec) noexcept;

```

```

bool create_directories(const path& p);
bool create_directories(const path& p, error_code& ec);

bool create_directory(const path& p);
bool create_directory(const path& p, error_code& ec) noexcept;

bool create_directory(const path& p, const path& attributes);
bool create_directory(const path& p, const path& attributes,
 error_code& ec) noexcept;

void create_directory_symlink(const path& to, const path& new_symlink);
void create_directory_symlink(const path& to, const path& new_symlink,
 error_code& ec) noexcept;

void create_hard_link(const path& to, const path& new_hard_link);
void create_hard_link(const path& to, const path& new_hard_link,
 error_code& ec) noexcept;

void create_symlink(const path& to, const path& new_symlink);
void create_symlink(const path& to, const path& new_symlink,
 error_code& ec) noexcept;

path current_path();
path current_path(error_code& ec);
void current_path(const path& p);
void current_path(const path& p, error_code& ec) noexcept;

bool equivalent(const path& p1, const path& p2);
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;

bool exists(file_status s) noexcept;
bool exists(const path& p);
bool exists(const path& p, error_code& ec) noexcept;

uintmax_t file_size(const path& p);
uintmax_t file_size(const path& p, error_code& ec) noexcept;

uintmax_t hard_link_count(const path& p);
uintmax_t hard_link_count(const path& p, error_code& ec) noexcept;

bool is_block_file(file_status s) noexcept;
bool is_block_file(const path& p);
bool is_block_file(const path& p, error_code& ec) noexcept;

bool is_character_file(file_status s) noexcept;
bool is_character_file(const path& p);
bool is_character_file(const path& p, error_code& ec) noexcept;

bool is_directory(file_status s) noexcept;
bool is_directory(const path& p);
bool is_directory(const path& p, error_code& ec) noexcept;

bool is_empty(const path& p);
bool is_empty(const path& p, error_code& ec);

bool is_fifo(file_status s) noexcept;
bool is_fifo(const path& p);
bool is_fifo(const path& p, error_code& ec) noexcept;

bool is_other(file_status s) noexcept;
bool is_other(const path& p);
bool is_other(const path& p, error_code& ec) noexcept;

```



```

bool is_regular_file(file_status s) noexcept;
bool is_regular_file(const path& p);
bool is_regular_file(const path& p, error_code& ec) noexcept;

bool is_socket(file_status s) noexcept;
bool is_socket(const path& p);
bool is_socket(const path& p, error_code& ec) noexcept;

bool is_symlink(file_status s) noexcept;
bool is_symlink(const path& p);
bool is_symlink(const path& p, error_code& ec) noexcept;

file_time_type last_write_time(const path& p);
file_time_type last_write_time(const path& p, error_code& ec) noexcept;
void last_write_time(const path& p, file_time_type new_time);
void last_write_time(const path& p, file_time_type new_time,
 error_code& ec) noexcept;

void permissions(const path& p, perms prms, perm_options opts=perm_options::replace);
void permissions(const path& p, perms prms, error_code& ec) noexcept;
void permissions(const path& p, perms prms, perm_options opts, error_code& ec);

path proximate(const path& p, error_code& ec);
path proximate(const path& p, const path& base = current_path());
path proximate(const path& p, const path& base, error_code& ec);

path read_symlink(const path& p);
path read_symlink(const path& p, error_code& ec);

path relative(const path& p, error_code& ec);
path relative(const path& p, const path& base = current_path());
path relative(const path& p, const path& base, error_code& ec);

bool remove(const path& p);
bool remove(const path& p, error_code& ec) noexcept;

uintmax_t remove_all(const path& p);
uintmax_t remove_all(const path& p, error_code& ec);

void rename(const path& from, const path& to);
void rename(const path& from, const path& to, error_code& ec) noexcept;

void resize_file(const path& p, uintmax_t size);
void resize_file(const path& p, uintmax_t size, error_code& ec) noexcept;

space_info space(const path& p);
space_info space(const path& p, error_code& ec) noexcept;

file_status status(const path& p);
file_status status(const path& p, error_code& ec) noexcept;

bool status_known(file_status s) noexcept;

file_status symlink_status(const path& p);
file_status symlink_status(const path& p, error_code& ec) noexcept;

path temp_directory_path();
path temp_directory_path(error_code& ec);

path weakly_canonical(const path& p);
path weakly_canonical(const path& p, error_code& ec);
}

```

```

namespace std {
 // 31.12.6.9, formatting support
 template<class charT> struct formatter<filesystem::path, charT>;

 // 31.12.6.10, hash support
 template<class T> struct hash;
 template<> struct hash<filesystem::path>;
}

namespace std::ranges {
 template<>
 inline constexpr bool enable_borrowed_range<filesystem::directory_iterator> = true;
 template<>
 inline constexpr bool enable_borrowed_range<filesystem::recursive_directory_iterator> = true;

 template<>
 inline constexpr bool enable_view<filesystem::directory_iterator> = true;
 template<>
 inline constexpr bool enable_view<filesystem::recursive_directory_iterator> = true;
}

```

- <sup>1</sup> Implementations should ensure that the resolution and range of `file_time_type` reflect the operating system dependent resolution and range of file time values.

### 31.12.5 Error reporting

[fs.err.report]

- <sup>1</sup> Filesystem library functions often provide two overloads, one that throws an exception to report file system errors, and another that sets an `error_code`.

[Note 1: This supports two common use cases:

- (1.1) — Uses where file system errors are truly exceptional and indicate a serious failure. Throwing an exception is an appropriate response.
  - (1.2) — Uses where file system errors are routine and do not necessarily represent failure. Returning an error code is the most appropriate response. This allows application specific error handling, including simply ignoring the error.
- end note]

- <sup>2</sup> Functions not having an argument of type `error_code&` handle errors as follows, unless otherwise specified:

- (2.1) — When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, an exception of type `filesystem_error` shall be thrown. For functions with a single path argument, that argument shall be passed to the `filesystem_error` constructor with a single path argument. For functions with two path arguments, the first of these arguments shall be passed to the `filesystem_error` constructor as the `path1` argument, and the second shall be passed as the `path2` argument. The `filesystem_error` constructor's `error_code` argument is set as appropriate for the specific operating system dependent error.
- (2.2) — Failure to allocate storage is reported by throwing an exception as described in 16.4.6.14.
- (2.3) — Destructors throw nothing.

- <sup>3</sup> Functions having an argument of type `error_code&` handle errors as follows, unless otherwise specified:

- (3.1) — If a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, the `error_code&` argument is set as appropriate for the specific operating system dependent error. Otherwise, `clear()` is called on the `error_code&` argument.

### 31.12.6 Class path

[fs.class.path]

#### 31.12.6.1 General

[fs.class.path.general]

- <sup>1</sup> An object of class `path` represents a path and contains a pathname. Such an object is concerned only with the lexical and syntactic aspects of a path. The path does not necessarily exist in external storage, and the pathname is not necessarily valid for the current operating system or for a particular file system.
- <sup>2</sup> [Note 1: Class `path` is used to support the differences between the string types used by different operating systems to represent pathnames, and to perform conversions between encodings when necessary. — end note]

- <sup>3</sup> A *path* is a sequence of elements that identify the location of a file within a filesystem. The elements are the *root-name<sub>opt</sub>*, *root-directory<sub>opt</sub>*, and an optional sequence of *filenames* (31.12.6.2). The maximum number of elements in the sequence is operating system dependent (31.12.2.3).
- <sup>4</sup> An *absolute path* is a path that unambiguously identifies the location of a file without reference to an additional starting location. The elements of a path that determine if it is absolute are operating system dependent. A *relative path* is a path that is not absolute, and as such, only unambiguously identifies the location of a file when resolved relative to an implied starting location. The elements of a path that determine if it is relative are operating system dependent.

[Note 2: Pathnames “.” and “..” are relative paths. — end note]

- <sup>5</sup> A *pathname* is a character string that represents the name of a path. Pathnames are formatted according to the generic pathname format grammar (31.12.6.2) or according to an operating system dependent *native pathname format* accepted by the host operating system.
- <sup>6</sup> *Pathname resolution* is the operating system dependent mechanism for resolving a pathname to a particular file in a file hierarchy. There may be multiple pathnames that resolve to the same file.

[Example 1: For POSIX-based operating systems, this mechanism is specified in POSIX, section 4.12, Pathname resolution. — end example]

```
namespace std::filesystem {
 class path {
 public:
 using value_type = see below;
 using string_type = basic_string<value_type>;
 static constexpr value_type preferred_separator = see below;

 // 31.12.8.1, enumeration format
 enum format;

 // 31.12.6.5.1, constructors and destructor
 path() noexcept;
 path(const path& p);
 path(path&& p) noexcept;
 path(string_type&& source, format fmt = auto_format);
 template<class Source>
 path(const Source& source, format fmt = auto_format);
 template<class InputIterator>
 path(InputIterator first, InputIterator last, format fmt = auto_format);
 template<class Source>
 path(const Source& source, const locale& loc, format fmt = auto_format);
 template<class InputIterator>
 path(InputIterator first, InputIterator last, const locale& loc, format fmt = auto_format);
 ~path();

 // 31.12.6.5.2, assignments
 path& operator=(const path& p);
 path& operator=(path&& p) noexcept;
 path& operator=(string_type&& source);
 path& assign(string_type&& source);
 template<class Source>
 path& operator=(const Source& source);
 template<class Source>
 path& assign(const Source& source);
 template<class InputIterator>
 path& assign(InputIterator first, InputIterator last);

 // 31.12.6.5.3, appends
 path& operator/=(const path& p);
 template<class Source>
 path& operator/=(const Source& source);
 template<class Source>
 path& append(const Source& source);
```

```

template<class InputIterator>
 path& append(InputIterator first, InputIterator last);

// 31.12.6.5.4, concatenation
path& operator+=(const path& x);
path& operator+=(const string_type& x);
path& operator+=(basic_string_view<value_type> x);
path& operator+=(const value_type* x);
path& operator+=(value_type x);
template<class Source>
 path& operator+=(const Source& x);
template<class EcharT>
 path& operator+=(EcharT x);
template<class Source>
 path& concat(const Source& x);
template<class InputIterator>
 path& concat(InputIterator first, InputIterator last);

// 31.12.6.5.5, modifiers
void clear() noexcept;
path& make_preferred();
path& remove_filename();
path& replace_filename(const path& replacement);
path& replace_extension(const path& replacement = path());
void swap(path& rhs) noexcept;

// 31.12.6.8, non-member operators
friend bool operator==(const path& lhs, const path& rhs) noexcept;
friend strong_ordering operator<=>(const path& lhs, const path& rhs) noexcept;

friend path operator/(const path& lhs, const path& rhs);

// 31.12.6.5.6, native format observers
const string_type& native() const noexcept;
const value_type* c_str() const noexcept;
operator string_type() const;

template<class EcharT, class traits = char_traits<EcharT>,
 class Allocator = allocator<EcharT>>
 basic_string<EcharT, traits, Allocator>
 string(const Allocator& a = Allocator()) const;
std::string display_string() const;
std::string native_encoded_string() const;
std::wstring wstring() const;
std::u8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;

// 31.12.6.5.7, generic format observers
template<class EcharT, class traits = char_traits<EcharT>,
 class Allocator = allocator<EcharT>>
 basic_string<EcharT, traits, Allocator>
 generic_string(const Allocator& a = Allocator()) const;
std::string generic_display_string() const;
std::string generic_native_encoded_string() const;
std::wstring generic_wstring() const;
std::u8string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;

// 31.12.6.5.8, compare
int compare(const path& p) const noexcept;
int compare(const string_type& s) const;
int compare(basic_string_view<value_type> s) const;

```

```

int compare(const value_type* s) const;

// 31.12.6.5.9, decomposition
path root_name() const;
path root_directory() const;
path root_path() const;
path relative_path() const;
path parent_path() const;
path filename() const;
path stem() const;
path extension() const;

// 31.12.6.5.10, query
bool empty() const noexcept;
bool has_root_name() const;
bool has_root_directory() const;
bool has_root_path() const;
bool has_relative_path() const;
bool has_parent_path() const;
bool has_filename() const;
bool has_stem() const;
bool has_extension() const;
bool is_absolute() const;
bool is_relative() const;

// 31.12.6.5.11, generation
path lexically_normal() const;
path lexically_relative(const path& base) const;
path lexically_proximate(const path& base) const;

// 31.12.6.6, iterators
class iterator;
using const_iterator = iterator;

iterator begin() const;
iterator end() const;

// 31.12.6.7, path inserter and extractor
template<class charT, class traits>
 friend basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& os, const path& p);
template<class charT, class traits>
 friend basic_istream<charT, traits>&
 operator>>(basic_istream<charT, traits>& is, path& p);
};
}

```

- <sup>7</sup> `value_type` is a *typedef-name* for the operating system dependent encoded character type used to represent pathnames.
- <sup>8</sup> The value of the `preferred_separator` member is the operating system dependent *preferred-separator* character (31.12.6.2).
- <sup>9</sup> [Example 2: For POSIX-based operating systems, `value_type` is `char` and `preferred_separator` is the slash character ('/'). For Windows-based operating systems, `value_type` is `wchar_t` and `preferred_separator` is the backslash character (L'\\'). — end example]

### 31.12.6.2 Generic pathname format

[fs.path.generic]

*pathname:*

*root-name*<sub>opt</sub> *root-directory*<sub>opt</sub> *relative-path*

*root-name:*

operating system dependent sequences of characters  
implementation-defined sequences of characters

*root-directory*:  
*directory-separator*

*relative-path*:  
*filename*  
*filename directory-separator relative-path*  
 an empty path

*filename*:  
 non-empty sequence of characters other than *directory-separator* characters

*directory-separator*:  
*preferred-separator directory-separator<sub>opt</sub>*  
*fallback-separator directory-separator<sub>opt</sub>*

*preferred-separator*:  
 operating system dependent directory separator character

*fallback-separator*:  
 /, if *preferred-separator* is not /

<sup>1</sup> A *filename* is the name of a file. The *dot* and *dot-dot* filenames, consisting solely of one and two period characters respectively, have special meaning. The following characteristics of filenames are operating system dependent:

(1.1) — The permitted characters.

[*Example 1*: Some operating systems prohibit the ASCII control characters (0x00 – 0x1F) in filenames. — *end example*]

[*Note 1*: Wider portability can be achieved by limiting *filename* characters to the POSIX Portable Filename Character Set:

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z  
 a b c d e f g h i j k l m n o p q r s t u v w x y z  
 0 1 2 3 4 5 6 7 8 9 . \_ - — *end note*]

(1.2) — The maximum permitted length.

(1.3) — Filenames that are not permitted.

(1.4) — Filenames that have special meaning.

(1.5) — Case awareness and sensitivity during path resolution.

(1.6) — Special rules that may apply to file types other than regular files, such as directories.

<sup>2</sup> Except in a *root-name*, multiple successive *directory-separator* characters are considered to be the same as one *directory-separator* character.

<sup>3</sup> The dot filename is treated as a reference to the current directory. The dot-dot filename is treated as a reference to the parent directory. What the dot-dot filename refers to relative to *root-directory* is implementation-defined. Specific filenames may have special meanings for a particular operating system.

<sup>4</sup> A *root-name* identifies the starting location for pathname resolution (31.12.6). If there are no operating system dependent *root-names*, at least one implementation-defined *root-name* is required.

[*Note 2*: Many operating systems define a name beginning with two *directory-separator* characters as a *root-name* that identifies network or other resource locations. Some operating systems define a single letter followed by a colon as a drive specifier — a *root-name* identifying a specific device such as a disk drive. — *end note*]

<sup>5</sup> If a *root-name* is otherwise ambiguous, the possibility with the longest sequence of characters is chosen.

[*Note 3*: On a POSIX-like operating system, it is impossible to have a *root-name* and a *relative-path* without an intervening *root-directory* element. — *end note*]

<sup>6</sup> *Normalization* of a generic format pathname means:

1. If the path is empty, stop.

2. Replace each slash character in the *root-name* with a *preferred-separator*.

3. Replace each *directory-separator* with a *preferred-separator*.

[*Note 4*: The generic pathname grammar defines *directory-separator* as one or more slashes and *preferred-separators*. — *end note*]

4. Remove each dot filename and any immediately following *directory-separator*.

5. As long as any appear, remove a non-dot-dot filename immediately followed by a *directory-separator* and a dot-dot filename, along with any immediately following *directory-separator*.
6. If there is a *root-directory*, remove all dot-dot filenames and any *directory-separators* immediately following them.

[Note 5: These dot-dot filenames attempt to refer to nonexistent parent directories. — end note]

7. If the last filename is dot-dot, remove any trailing *directory-separator*.

8. If the path is empty, add a dot.

The result of normalization is a path in *normal form*, which is said to be *normalized*.

### 31.12.6.3 Conversions

[fs.path.cvt]

#### 31.12.6.3.1 Argument format conversions

[fs.path.fmt.cvt]

- <sup>1</sup> [Note 1: The format conversions described in this subclause are not applied on POSIX-based operating systems because on these systems:

- (1.1) — The generic format is acceptable as a native path.
- (1.2) — There is no need to distinguish between native format and generic format in function arguments.
- (1.3) — Paths for regular files and paths for directories share the same syntax.

— end note]

- <sup>2</sup> Several functions are defined to accept *detected-format* arguments, which are character sequences. A detected-format argument represents a path using either a pathname in the generic format (31.12.6.2) or a pathname in the native format (31.12.6). Such an argument is taken to be in the generic format if and only if it matches the generic format and is not acceptable to the operating system as a native path.

- <sup>3</sup> [Note 2: Some operating systems have no unambiguous way to distinguish between native format and generic format arguments. This is by design as it simplifies use for operating systems that do not require disambiguation. It is possible that an implementation for an operating system where disambiguation is needed distinguishes between the formats. — end note]

- <sup>4</sup> Pathnames are converted as needed between the generic and native formats in an operating-system-dependent manner. Let  $G(n)$  and  $N(g)$  in a mathematical sense be the implementation's functions that convert native-to-generic and generic-to-native formats respectively. If  $g=G(n)$  for some  $n$ , then  $G(N(g))=g$ ; if  $n=N(g)$  for some  $g$ , then  $N(G(n))=n$ .

[Note 3: Neither  $G$  nor  $N$  need be invertible. — end note]

- <sup>5</sup> If the native format requires paths for regular files to be formatted differently from paths for directories, the path shall be treated as a directory path if its last element is a *directory-separator*, otherwise it shall be treated as a path to a regular file.

- <sup>6</sup> [Note 4: A path stores a native format pathname (31.12.6.5.6) and acts as if it also stores a generic format pathname, related as given below. The implementation can generate the generic format pathname based on the native format pathname (and possibly other information) when requested. — end note]

- <sup>7</sup> When a path is constructed from or is assigned a single representation separate from any path, the other representation is selected by the appropriate conversion function ( $G$  or  $N$ ).

- <sup>8</sup> When the (new) value  $p$  of one representation of a path is derived from the representation of that or another path, a value  $q$  is chosen for the other representation. The value  $q$  converts to  $p$  (by  $G$  or  $N$  as appropriate) if any such value does so;  $q$  is otherwise unspecified.

[Note 5: If  $q$  is the result of converting any path at all, it is the result of converting  $p$ . — end note]

#### 31.12.6.3.2 Type and encoding conversions

[fs.path.type.cvt]

- <sup>1</sup> The *native encoding* of an ordinary character string is the operating system dependent current encoding for pathnames (31.12.6). The *native encoding* for wide character strings is the implementation-defined execution wide-character set encoding (16.3.3.3.4).

- <sup>2</sup> For member function arguments that take character sequences representing paths and for member functions returning strings, value type and encoding conversion is performed if the value type of the argument or return value differs from `path::value_type`. For the argument or return value, the method of conversion and the encoding to be converted to is determined by its value type:

- (2.1) — **char**: The encoding is the native ordinary encoding. The method of conversion, if any, is operating system dependent.

[*Note 1:* For POSIX-based operating systems `path::value_type` is `char` so no conversion from `char` value type arguments or to `char` value type return values is performed. For Windows-based operating systems, the native ordinary encoding is determined by the current locale encoding and the Windows `AreFileApisANSI` function. — *end note*]

[*Note 2:* This results in behavior identical to other C and C++ standard library functions that perform file operations using ordinary character strings to identify paths. Changing this behavior would be surprising and error-prone. — *end note*]

- (2.2) — `wchar_t`: The encoding is the native wide encoding. The method of conversion is unspecified.

[*Note 3:* For Windows-based operating systems `path::value_type` is `wchar_t` so no conversion from `wchar_t` value type arguments or to `wchar_t` value type return values is performed. — *end note*]

- (2.3) — `char8_t`: The encoding is UTF-8. The method of conversion is unspecified.

- (2.4) — `char16_t`: The encoding is UTF-16. The method of conversion is unspecified.

- (2.5) — `char32_t`: The encoding is UTF-32. The method of conversion is unspecified.

- <sup>3</sup> If the encoding being converted to has no representation for source characters, the resulting converted characters, if any, are unspecified. Implementations should not modify member function arguments if already of type `path::value_type`.

#### 31.12.6.4 Requirements

[fs.path.req]

- <sup>1</sup> In addition to the requirements (31.12.3), function template parameters named `Source` shall be one of:

- (1.1) — `basic_string<EcharT, traits, Allocator>`. A function argument `const Source& source` shall have an effective range `[source.begin(), source.end())`.
- (1.2) — `basic_string_view<EcharT, traits>`. A function argument `const Source& source` shall have an effective range `[source.begin(), source.end())`.
- (1.3) — A type meeting the *Cpp17InputIterator* requirements that iterates over an NTCTS. The value type shall be an encoded character type. A function argument `const Source& source` shall have an effective range `[source, end)` where `end` is the first iterator value with an element value equal to `iterator_traits<Source>::value_type()`.
- (1.4) — A character array that after array-to-pointer decay results in a pointer to the start of an NTCTS. The value type shall be an encoded character type. A function argument `const Source& source` shall have an effective range `[source, end)` where `end` is the first iterator value with an element value equal to `iterator_traits<decay_t<Source>>::value_type()`.

- <sup>2</sup> Functions taking template parameters named `Source` shall not participate in overload resolution unless `Source` denotes a type other than `path`, and either

- (2.1) — `Source` is a specialization of `basic_string` or `basic_string_view`, or
- (2.2) — the *qualified-id* `iterator_traits<decay_t<Source>>::value_type` is valid and denotes a possibly `const` encoded character type (13.10.3).

- <sup>3</sup> [*Note 1:* See path conversions (31.12.6.3) for how the value types above and their encodings convert to `path::value_type` and its encoding. — *end note*]

- <sup>4</sup> Arguments of type `Source` shall not be null pointers.

#### 31.12.6.5 Members

[fs.path.member]

##### 31.12.6.5.1 Constructors

[fs.path.construct]

`path()` `noexcept`;

- <sup>1</sup> *Postconditions:* `empty()` is `true`.

`path(const path& p);`  
`path(path&& p) noexcept;`

- <sup>2</sup> *Effects:* Constructs an object of class `path` having the same pathname in the native and generic formats, respectively, as the original value of `p`. In the second form, `p` is left in a valid but unspecified state.



```
path(string_type&& source, format fmt = auto_format);
```

- 3 *Effects:* Constructs an object of class `path` for which the pathname in the detected-format of `source` has the original value of `source` (31.12.6.3.1), converting format if required (31.12.6.3.1). `source` is left in a valid but unspecified state.

```
template<class Source>
 path(const Source& source, format fmt = auto_format);
template<class InputIterator>
 path(InputIterator first, InputIterator last, format fmt = auto_format);
```

- 4 *Effects:* Let `s` be the effective range of `source` (31.12.6.4) or the range `[first, last)`, with the encoding converted if required (31.12.6.3). Finds the detected-format of `s` (31.12.6.3.1) and constructs an object of class `path` for which the pathname in that format is `s`.

```
template<class Source>
 path(const Source& source, const locale& loc, format fmt = auto_format);
template<class InputIterator>
 path(InputIterator first, InputIterator last, const locale& loc, format fmt = auto_format);
```

- 5 *Mandates:* The value type of `Source` and `InputIterator` is `char`.

- 6 *Effects:* Let `s` be the effective range of `source` or the range `[first, last)`, after converting the encoding as follows:

- (6.1) — If `value_type` is `wchar_t`, converts to the native wide encoding (31.12.6.3.2) using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`.
- (6.2) — Otherwise a conversion is performed using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`, and then a second conversion to the current ordinary encoding.

- 7 Finds the detected-format of `s` (31.12.6.3.1) and constructs an object of class `path` for which the pathname in that format is `s`.

[*Example 1:* A string is to be read from a database that is encoded in ISO/IEC 8859-1, and used to create a directory:

```
namespace fs = std::filesystem;
std::string latin1_string = read_latin1_data();
codecvt_8859_1<wchar_t> latin1_facet;
std::locale latin1_locale(std::locale(), latin1_facet);
fs::create_directory(fs::path(latin1_string, latin1_locale));
```

For POSIX-based operating systems, the path is constructed by first using `latin1_facet` to convert ISO/IEC 8859-1 encoded `latin1_string` to a wide character string in the native wide encoding (31.12.6.3.2). The resulting wide string is then converted to an ordinary character pathname string in the current native ordinary encoding. If the native wide encoding is UTF-16 or UTF-32, and the current native ordinary encoding is UTF-8, all of the characters in the ISO/IEC 8859-1 character set will be converted to their Unicode representation, but for other native ordinary encodings some characters may have no representation.

For Windows-based operating systems, the path is constructed by using `latin1_facet` to convert ISO/IEC 8859-1 encoded `latin1_string` to a UTF-16 encoded wide character pathname string. All of the characters in the ISO/IEC 8859-1 character set will be converted to their Unicode representation. — *end example*]

### 31.12.6.5.2 Assignments

[`fs.path.assign`]

```
path& operator=(const path& p);
```

- 1 *Effects:* If `*this` and `p` are the same object, has no effect. Otherwise, sets both respective pathnames of `*this` to the respective pathnames of `p`.

- 2 *Returns:* `*this`.

```
path& operator=(path&& p) noexcept;
```

- 3 *Effects:* If `*this` and `p` are the same object, has no effect. Otherwise, sets both respective pathnames of `*this` to the respective pathnames of `p`. `p` is left in a valid but unspecified state.

[*Note 1:* A valid implementation is `swap(p)`. — *end note*]

- 4 *Returns:* `*this`.

```
path& operator=(string_type&& source);
path& assign(string_type&& source);
```

5     *Effects:* Sets the pathname in the detected-format of `source` to the original value of `source`. `source` is left in a valid but unspecified state.

6     *Returns:* `*this`.

```
template<class Source>
 path& operator=(const Source& source);
template<class Source>
 path& assign(const Source& source);
template<class InputIterator>
 path& assign(InputIterator first, InputIterator last);
```

7     *Effects:* Let `s` be the effective range of `source` (31.12.6.4) or the range `[first, last)`, with the encoding converted if required (31.12.6.3). Finds the detected-format of `s` (31.12.6.3.1) and sets the pathname in that format to `s`.

8     *Returns:* `*this`.

### 31.12.6.5.3 Appends

[fs.path.append]

1     The append operations use `operator/=` to denote their semantic effect of appending *preferred-separator* when needed.

```
path& operator/=(const path& p);
```

2     *Effects:* If `p.is_absolute() || (p.has_root_name() && p.root_name() != root_name())`, then `operator=(p)`.

3     Otherwise, modifies `*this` as if by these steps:

- (3.1) — If `p.has_root_directory()`, then removes any root directory and relative path from the generic format pathname. Otherwise, if `!has_root_directory() && is_absolute()` is true or if `has_filename()` is true, then appends `path::preferred_separator` to the generic format pathname.
- (3.2) — Then appends the native format pathname of `p`, omitting any *root-name* from its generic format pathname, to the native format pathname.

4     [*Example 1:* Even if `//host` is interpreted as a *root-name*, both of the paths `path("//host")/"foo"` and `path("//host/")/"foo"` equal `"//host/foo"` (although the former might use backslash as the preferred separator).

Expression examples:

```
// On POSIX,
path("foo") /= path(""); // yields path("foo/")
path("foo") /= path("/bar"); // yields path("/bar")

// On Windows,
path("foo") /= path(""); // yields path("foo\\")
path("foo") /= path("/bar"); // yields path("/bar")
path("foo") /= path("c:/bar"); // yields path("c:/bar")
path("foo") /= path("c:"); // yields path("c:")
path("c:") /= path(""); // yields path("c:")
path("c:foo") /= path("/bar"); // yields path("c:/bar")
path("c:foo") /= path("c:bar"); // yields path("c:foo\\bar")
```

— end example]

5     *Returns:* `*this`.

```
template<class Source>
 path& operator/=(const Source& source);
template<class Source>
 path& append(const Source& source);
```

6     *Effects:* Equivalent to: `return operator/=(path(source));`

```
template<class InputIterator>
path& append(InputIterator first, InputIterator last);
```

7     *Effects:* Equivalent to: `return operator+=(path(first, last));`

#### 31.12.6.5.4 Concatenation

[fs.path.concat]

```
path& operator+=(const path& x);
path& operator+=(const string_type& x);
path& operator+=(basic_string_view<value_type> x);
path& operator+=(const value_type* x);
template<class Source>
path& operator+=(const Source& x);
template<class Source>
path& concat(const Source& x);
```

1     *Effects:* Appends `path(x).native()` to the pathname in the native format.

[*Note 1:* This directly manipulates the value of `native()`, which is not necessarily portable between operating systems. — *end note*]

2     *Returns:* `*this`.

```
path& operator+=(value_type x);
template<class EcharT>
path& operator+=(EcharT x);
```

3     *Effects:* Equivalent to: `return *this += basic_string_view(&x, 1);`

```
template<class InputIterator>
path& concat(InputIterator first, InputIterator last);
```

4     *Effects:* Equivalent to: `return *this += path(first, last);`

#### 31.12.6.5.5 Modifiers

[fs.path.modifiers]

```
void clear() noexcept;
```

1     *Postconditions:* `empty()` is true.

```
path& make_preferred();
```

2     *Effects:* Each *directory-separator* of the pathname in the generic format is converted to *preferred-separator*.

3     *Returns:* `*this`.

4     [*Example 1:*

```
path p("foo/bar");
std::cout << p << '\n';
p.make_preferred();
std::cout << p << '\n';
```

On an operating system where *preferred-separator* is a slash, the output is:

```
"foo/bar"
"foo/bar"
```

On an operating system where *preferred-separator* is a backslash, the output is:

```
"foo/bar"
"foo\bar"
```

— *end example*]

```
path& remove_filename();
```

5     *Effects:* Remove the generic format pathname of `filename()` from the generic format pathname.

6     *Postconditions:* `!has_filename()`.

7     *Returns:* `*this`.

8     [*Example 2:*

```
path("foo/bar").remove_filename(); // yields "foo/"
path("foo/").remove_filename(); // yields "foo/"
```

```

 path("/foo").remove_filename(); // yields "/"
 path("/").remove_filename(); // yields "/"
— end example]

```

```

path& replace_filename(const path& replacement);

```

9 *Effects:* Equivalent to:

```

 remove_filename();
 operator/=(replacement);

```

10 *Returns:* *\*this*.

11 *[Example 3:*

```

 path("/foo").replace_filename("bar"); // yields "/bar" on POSIX
 path("/").replace_filename("bar"); // yields "/bar" on POSIX
— end example]

```

```

path& replace_extension(const path& replacement = path());

```

12 *Effects:*

- (12.1) — Any existing `extension()` (31.12.6.5.9) is removed from the pathname in the generic format, then
- (12.2) — If `replacement` is not empty and does not begin with a dot character, a dot character is appended to the pathname in the generic format, then
- (12.3) — `operator+=(replacement);`.

13 *Returns:* *\*this*.

```

void swap(path& rhs) noexcept;

```

14 *Effects:* Swaps the contents (in all formats) of the two paths.

15 *Complexity:* Constant time.

### 31.12.6.5.6 Native format observers

[fs.path.native.obs]

1 The string returned by all native format observers is in the native pathname format (31.12.6).

```

const string_type& native() const noexcept;

```

2 *Returns:* The pathname in the native format.

```

const value_type* c_str() const noexcept;

```

3 *Effects:* Equivalent to: `return native().c_str();`

```

operator string_type() const;

```

4 *Returns:* `native()`.

```

template<class EcharT, class traits = char_traits<EcharT>, class Allocator = allocator<EcharT>>
 basic_string<EcharT, traits, Allocator> string(const Allocator& a = Allocator()) const;

```

5 *Returns:* `native()`.

6 *Remarks:* All memory allocation, including for the return value, shall be performed by `a`. Conversion, if any, is specified by 31.12.6.3.

```

std::string native_encoded_string() const;
std::wstring wstring() const;
std::u8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;

```

7 *Returns:* `native()`.

8 *Remarks:* Conversion, if any, is performed as specified by 31.12.6.3.

```

std::string display_string() const;

```

9 *Returns:* `std::format("{ }", *this)`.

[*Note 1*: The returned string is suitable for use with formatting (28.5.5) and print functions (31.7.10). — *end note*]

### 31.12.6.5.7 Generic format observers

[fs.path.generic.obs]

<sup>1</sup> Generic format observer functions return strings formatted according to the generic pathname format (31.12.6.2). A single slash ('/') character is used as the *directory-separator*.

<sup>2</sup> [*Example 1*: On an operating system that uses backslash as its *preferred-separator*,

```
path("foo\\bar").generic_string()
```

returns "foo/bar". — *end example*]

```
template<class EcharT, class traits = char_traits<EcharT>, class Allocator = allocator<EcharT>>
basic_string<EcharT, traits, Allocator> generic_string(const Allocator& a = Allocator()) const;
```

<sup>3</sup> *Returns*: The pathname in the generic format.

<sup>4</sup> *Remarks*: All memory allocation, including for the return value, shall be performed by **a**. Conversion, if any, is specified by 31.12.6.3.

```
std::string generic_native_encoded_string() const;
std::wstring generic_wstring() const;
std::u8string generic_u8string() const;
std::u16string generic_u16string() const;
std::u32string generic_u32string() const;
```

<sup>5</sup> *Returns*: The pathname in the generic format.

<sup>6</sup> *Remarks*: Conversion, if any, is specified by 31.12.6.3.

```
std::string generic_display_string() const;
```

<sup>7</sup> *Returns*: `std::format("{:g}", *this)`.

[*Note 1*: The returned string is suitable for use with formatting (28.5.5) and print functions (31.7.10). — *end note*]

### 31.12.6.5.8 Compare

[fs.path.compare]

```
int compare(const path& p) const noexcept;
```

<sup>1</sup> *Returns*:

- (1.1) — Let `rootNameComparison` be the result of `this->root_name().native().compare(p.root_name().native())`. If `rootNameComparison` is not 0, `rootNameComparison`.
- (1.2) — Otherwise, if `!this->has_root_directory()` and `p.has_root_directory()`, a value less than 0.
- (1.3) — Otherwise, if `this->has_root_directory()` and `!p.has_root_directory()`, a value greater than 0.
- (1.4) — Otherwise, if `native()` for the elements of `this->relative_path()` are lexicographically less than `native()` for the elements of `p.relative_path()`, a value less than 0.
- (1.5) — Otherwise, if `native()` for the elements of `this->relative_path()` are lexicographically greater than `native()` for the elements of `p.relative_path()`, a value greater than 0.
- (1.6) — Otherwise, 0.

```
int compare(const string_type& s) const;
int compare(basic_string_view<value_type> s) const;
int compare(const value_type* s) const;
```

<sup>2</sup> *Effects*: Equivalent to: `return compare(path(s));`

### 31.12.6.5.9 Decomposition

[fs.path.decompose]

```
path root_name() const;
```

<sup>1</sup> *Returns*: *root-name*, if the pathname in the generic format includes *root-name*, otherwise `path()`.

```
path root_directory() const;
```

2 *Returns:* *root-directory*, if the pathname in the generic format includes *root-directory*, otherwise *path()*.

```
path root_path() const;
```

3 *Returns:* *root\_name()* / *root\_directory()*.

```
path relative_path() const;
```

4 *Returns:* A path composed from the pathname in the generic format, if *empty()* is *false*, beginning with the first *filename* after *root\_path()*. Otherwise, *path()*.

```
path parent_path() const;
```

5 *Returns:* *\*this* if *has\_relative\_path()* is *false*, otherwise a path whose generic format pathname is the longest prefix of the generic format pathname of *\*this* that produces one fewer element in its iteration.

```
path filename() const;
```

6 *Returns:* *relative\_path().empty()* ? *path()* : *\*--end()*.

7 [Example 1:

```
 path("/foo/bar.txt").filename(); // yields "bar.txt"
 path("/foo/bar").filename(); // yields "bar"
 path("/foo/bar/").filename(); // yields ""
 path("/").filename(); // yields ""
 path("//host").filename(); // yields ""
 path(".").filename(); // yields "."
 path("..").filename(); // yields ".."
 — end example]
```

```
path stem() const;
```

8 *Returns:* Let *f* be the generic format pathname of *filename()*. Returns a path whose pathname in the generic format is

(8.1) — *f*, if it contains no periods other than a leading period or consists solely of one or two periods;

(8.2) — otherwise, the prefix of *f* ending before its last period.

9 [Example 2:

```
 std::cout << path("/foo/bar.txt").stem(); // outputs "bar"
 path p = "foo.bar.baz.tar";
 for (; !p.extension().empty(); p = p.stem())
 std::cout << p.extension() << '\n';
 // outputs: .tar
 // .baz
 // .bar
 — end example]
```

```
path extension() const;
```

10 *Returns:* A path whose pathname in the generic format is the suffix of *filename()* not included in *stem()*.

11 [Example 3:

```
 path("/foo/bar.txt").extension(); // yields ".txt" and stem() is "bar"
 path("/foo/bar").extension(); // yields "" and stem() is "bar"
 path("/foo/.profile").extension(); // yields "" and stem() is ".profile"
 path(".bar").extension(); // yields "" and stem() is ".bar"
 path("..bar").extension(); // yields ".bar" and stem() is "."
 — end example]
```

12 [Note 1: The period is included in the return value so that it is possible to distinguish between no extension and an empty extension. — end note]

13 [Note 2: On non-POSIX operating systems, for a path *p*, it is possible that *p.stem()* + *p.extension()* == *p.filename()* is *false*, even though the generic format pathnames are the same. — end note]

**31.12.6.5.10 Query****[fs.path.query]**

```
bool empty() const noexcept;
```

1     *Returns:* **true** if the pathname in the generic format is empty, otherwise **false**.

```
bool has_root_path() const;
```

2     *Returns:* `!root_path().empty()`.

```
bool has_root_name() const;
```

3     *Returns:* `!root_name().empty()`.

```
bool has_root_directory() const;
```

4     *Returns:* `!root_directory().empty()`.

```
bool has_relative_path() const;
```

5     *Returns:* `!relative_path().empty()`.

```
bool has_parent_path() const;
```

6     *Returns:* `!parent_path().empty()`.

```
bool has_filename() const;
```

7     *Returns:* `!filename().empty()`.

```
bool has_stem() const;
```

8     *Returns:* `!stem().empty()`.

```
bool has_extension() const;
```

9     *Returns:* `!extension().empty()`.

```
bool is_absolute() const;
```

10    *Returns:* **true** if the pathname in the native format contains an absolute path (31.12.6), otherwise **false**.

11    [*Example 1:* `path("/")`.`is_absolute()` is **true** for POSIX-based operating systems, and **false** for Windows-based operating systems. — *end example*]

```
bool is_relative() const;
```

12    *Returns:* `!is_absolute()`.

**31.12.6.5.11 Generation****[fs.path.gen]**

```
path lexically_normal() const;
```

1     *Returns:* A path whose pathname in the generic format is the normal form (31.12.6.2) of the pathname in the generic format of `*this`.

2     [*Example 1:*

```
 assert(path("foo/./bar/..").lexically_normal() == "foo/");
 assert(path("foo/././bar/..").lexically_normal() == "foo/");
```

The above assertions will succeed. On Windows, the returned path's *directory-separator* characters will be backslashes rather than slashes, but that does not affect `path` equality. — *end example*]

```
path lexically_relative(const path& base) const;
```

3     *Effects:* If:

(3.1)    — `root_name() != base.root_name()` is **true**, or

(3.2)    — `is_absolute() != base.is_absolute()` is **true**, or

(3.3)    — `!has_root_directory() && base.has_root_directory()` is **true**, or

(3.4)    — any *filename* in `relative_path()` or `base.relative_path()` can be interpreted as a *root-name*, returns `path()`.

[Note 1: On a POSIX implementation, no *filename* in a *relative-path* is acceptable as a *root-name*. — end note]

Determines the first mismatched element of *\*this* and *base* as if by:

```
auto [a, b] = mismatch(begin(), end(), base.begin(), base.end());
```

Then,

- (3.5) — if *a == end()* and *b == base.end()*, returns *path(".")*; otherwise
- (3.6) — let *n* be the number of *filename* elements in [*b*, *base.end()*) that are not dot or dot-dot or empty, minus the number that are dot-dot. If *n* < 0, returns *path()*; otherwise
- (3.7) — if *n == 0* and (*a == end()* || *a->empty()*), returns *path(".")*; otherwise
- (3.8) — returns an object of class *path* that is default-constructed, followed by
- (3.8.1) — application of *operator/=(path(".."))* *n* times, and then
- (3.8.2) — application of *operator/=* for each element in [*a*, *end()*).

4 *Returns:* *\*this* made relative to *base*. Does not resolve (31.12.6) symlinks. Does not first normalize (31.12.6.2) *\*this* or *base*.

5 [Example 2:

```
assert(path("/a/d").lexically_relative("/a/b/c") == "../d");
assert(path("/a/b/c").lexically_relative("/a/d") == "../b/c");
assert(path("a/b/c").lexically_relative("a") == "b/c");
assert(path("a/b/c").lexically_relative("a/b/c/x/y") == "../..");
assert(path("a/b/c").lexically_relative("a/b/c") == ".");
assert(path("a/b").lexically_relative("c/d") == "../..a/b");
```

The above assertions will succeed. On Windows, the returned path's *directory-separator* characters will be backslashes rather than slashes, but that does not affect *path* equality. — end example]

6 [Note 2: If symlink following semantics are desired, use the operational function *relative()*. — end note]

7 [Note 3: If normalization (31.12.6.2) is needed to ensure consistent matching of elements, apply *lexically\_normal()* to *\*this*, *base*, or both. — end note]

```
path lexically_proximate(const path& base) const;
```

8 *Returns:* If the value of *lexically\_relative(base)* is not an empty path, return it. Otherwise return *\*this*.

9 [Note 4: If symlink following semantics are desired, use the operational function *proximate()*. — end note]

10 [Note 5: If normalization (31.12.6.2) is needed to ensure consistent matching of elements, apply *lexically\_normal()* to *\*this*, *base*, or both. — end note]

### 31.12.6.6 Iterators

[fs.path.itr]

1 Path iterators iterate over the elements of the pathname in the generic format (31.12.6.2).

2 A *path::iterator* is a constant iterator meeting all the requirements of a bidirectional iterator (24.3.5.6) except that, for dereferenceable iterators *a* and *b* of type *path::iterator* with *a == b*, there is no requirement that *\*a* and *\*b* are bound to the same object. Its *value\_type* is *path*.

3 Calling any non-const member function of a *path* object invalidates all iterators referring to elements of that object.

4 For the elements of the pathname in the generic format, the forward traversal order is as follows:

(4.1) — The *root-name* element, if present.

(4.2) — The *root-directory* element, if present.

[Note 1: It is possible that the use of the generic format is needed to ensure correct lexicographical comparison. — end note]

(4.3) — Each successive *filename* element, if present.

(4.4) — An empty element, if a trailing non-root *directory-separator* is present.

5 The backward traversal order is the reverse of forward traversal.



```
iterator begin() const;
```

6     *Returns:* An iterator for the first present element in the traversal list above. If no elements are present, the end iterator.

```
iterator end() const;
```

7     *Returns:* The end iterator.

### 31.12.6.7 Inserter and extractor

[fs.path.io]

```
template<class charT, class traits>
friend basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const path& p);
```

1     *Effects:* Equivalent to `os << quoted(p.string<charT, traits>())`.

[*Note 1:* The `quoted` function is described in 31.7.9. — end note]

2     *Returns:* `os`.

```
template<class charT, class traits>
friend basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, path& p);
```

3     *Effects:* Equivalent to:

```
 basic_string<charT, traits> tmp;
 is >> quoted(tmp);
 p = tmp;
```

4     *Returns:* `is`.

### 31.12.6.8 Non-member functions

[fs.path.nonmember]

```
void swap(path& lhs, path& rhs) noexcept;
```

1     *Effects:* Equivalent to `lhs.swap(rhs)`.

```
size_t hash_value(const path& p) noexcept;
```

2     *Returns:* A hash value for the path `p`. If for two paths, `p1 == p2` then `hash_value(p1) == hash_value(p2)`.

```
friend bool operator==(const path& lhs, const path& rhs) noexcept;
```

3     *Returns:* `lhs.compare(rhs) == 0`.

4     [*Note 1:* Path equality and path equivalence have different semantics.

(4.1)     — Equality is determined by the `path` non-member `operator==`, which considers the two paths' lexical representations only.

[*Example 1:* `path("foo") == "bar"` is never `true`. — end example]

(4.2)     — Equivalence is determined by the `equivalent()` non-member function, which determines if two paths resolve (31.12.6) to the same file system entity.

[*Example 2:* `equivalent("foo", "bar")` will be `true` when both paths resolve to the same file. — end example]

— end note]

```
friend strong_ordering operator<=>(const path& lhs, const path& rhs) noexcept;
```

5     *Returns:* `lhs.compare(rhs) <=> 0`.

```
friend path operator/(const path& lhs, const path& rhs);
```

6     *Effects:* Equivalent to: `return path(lhs) /= rhs;`

### 31.12.6.9 Formatting support

[fs.path.fmtr]

#### 31.12.6.9.1 Formatting support overview

[fs.path.fmtr.general]

```
namespace std {
 template<class charT> struct formatter<filesystem::path, charT> {
 constexpr void set_debug_format();
```

```
constexpr typename basic_format_parse_context<charT>::iterator
 parse(basic_format_parse_context<charT>& ctx);

template<class FormatContext>
 typename FormatContext::iterator
 format(const filesystem::path& path, FormatContext& ctx) const;
};
}
```

### 31.12.6.9.2 Formatting support functions

[fs.path.fmtr.funcs]

- <sup>1</sup> Formatting of paths uses formatting specifiers of the form

*path-format-spec*:  
*fill-and-align*<sub>opt</sub> *width*<sub>opt</sub> ?<sub>opt</sub> *g*<sub>opt</sub>

where the productions *fill-and-align* and *width* are described in 28.5.2. If the ? option is used then the path is formatted as an escaped string (28.5.6.5).

```
constexpr void set_debug_format();
```

- <sup>2</sup> *Effects*: Modifies the state of the formatter to be as if the *path-format-spec* parsed by the last call to *parse* contained the ? option.

```
constexpr typename basic_format_parse_context<charT>::iterator
 parse(basic_format_parse_context<charT>& ctx);
```

- <sup>3</sup> *Effects*: Parses the format specifier as a *path-format-spec* and stores the parsed specifiers in *\*this*.

- <sup>4</sup> *Returns*: An iterator past the end of the *path-format-spec*.

```
template<class FormatContext>
 typename FormatContext::iterator
 format(const filesystem::path& p, FormatContext& ctx) const;
```

- <sup>5</sup> *Effects*: Let *s* be *p.generic\_string<filesystem::path::value\_type>()* if the *g* option is used, otherwise *p.native()*. Writes *s* into *ctx.out()*, adjusted according to the *path-format-spec*. If *charT* is *char*, *path::value\_type* is *wchar\_t*, and the literal encoding is UTF-8, then the escaped path is transcoded from the native encoding for wide character strings to UTF-8 with maximal subparts of ill-formed subsequences substituted with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion. If *charT* and *path::value\_type* are the same then no transcoding is performed. Otherwise, transcoding is implementation-defined.

- <sup>6</sup> *Returns*: An iterator past the end of the output range.

### 31.12.6.10 Hash support

[fs.path.hash]

```
template<> struct hash<filesystem::path>;
```

- <sup>1</sup> For an object *p* of type *filesystem::path*, *hash<filesystem::path>()*(*p*) evaluates to the same result as *filesystem::hash\_value(p)*.

## 31.12.7 Class filesystem\_error

[fs.class.filesystem.error]

### 31.12.7.1 General

[fs.class.filesystem.error.general]

```
namespace std::filesystem {
 class filesystem_error : public system_error {
 public:
 filesystem_error(const string& what_arg, error_code ec);
 filesystem_error(const string& what_arg,
 const path& p1, error_code ec);
 filesystem_error(const string& what_arg,
 const path& p1, const path& p2, error_code ec);

 const path& path1() const noexcept;
 const path& path2() const noexcept;
 const char* what() const noexcept override;
 };
}
```

- <sup>1</sup> The class `filesystem_error` defines the type of objects thrown as exceptions to report file system errors from functions described in subclause 31.12.

### 31.12.7.2 Members

[fs.filesystem.error.members]

- <sup>1</sup> Constructors are provided that store zero, one, or two paths associated with an error.

```
filesystem_error(const string& what_arg, error_code ec);
```

<sup>2</sup> *Postconditions:*

- (2.1) — `code() == ec` is true,
- (2.2) — `path1().empty()` is true,
- (2.3) — `path2().empty()` is true, and
- (2.4) — `string_view(what()).find(what_arg.c_str()) != string_view::npos` is true.

```
filesystem_error(const string& what_arg, const path& p1, error_code ec);
```

<sup>3</sup> *Postconditions:*

- (3.1) — `code() == ec` is true,
- (3.2) — `path1()` returns a reference to the stored copy of `p1`,
- (3.3) — `path2().empty()` is true, and
- (3.4) — `string_view(what()).find(what_arg.c_str()) != string_view::npos` is true.

```
filesystem_error(const string& what_arg, const path& p1, const path& p2, error_code ec);
```

<sup>4</sup> *Postconditions:*

- (4.1) — `code() == ec`,
- (4.2) — `path1()` returns a reference to the stored copy of `p1`,
- (4.3) — `path2()` returns a reference to the stored copy of `p2`, and
- (4.4) — `string_view(what()).find(what_arg.c_str()) != string_view::npos`.

```
const path& path1() const noexcept;
```

- <sup>5</sup> *Returns:* A reference to the copy of `p1` stored by the constructor, or, if none, an empty path.

```
const path& path2() const noexcept;
```

- <sup>6</sup> *Returns:* A reference to the copy of `p2` stored by the constructor, or, if none, an empty path.

```
const char* what() const noexcept override;
```

- <sup>7</sup> *Returns:* An NTBS that incorporates the `what_arg` argument supplied to the constructor. The exact format is unspecified. Implementations should include the `system_error::what()` string and the pathnames of `path1` and `path2` in the native format in the returned string.

### 31.12.8 Enumerations

[fs.enum]

#### 31.12.8.1 Enum `path::format`

[fs.enum.path.format]

- <sup>1</sup> This enum specifies constants used to identify the format of the character sequence, with the meanings listed in Table 148.

#### 31.12.8.2 Enum class `file_type`

[fs.enum.file.type]

- <sup>1</sup> This enum class specifies constants used to identify file types, with the meanings listed in Table 149. The values of the constants are distinct.

#### 31.12.8.3 Enum class `copy_options`

[fs.enum.copy.opts]

- <sup>1</sup> The `enum class` type `copy_options` is a bitmask type (16.3.3.3.3) that specifies bitmask constants used to control the semantics of copy operations. The constants are specified in option groups with the meanings listed in Table 150. The constant `none` represents the empty bitmask, and is shown in each option group for purposes of exposition; implementations shall provide only a single definition. Every other constant in the table represents a distinct bitmask element.

Table 148 — Enum `path::format` [tab:fs.enum.path.format]

| Name                        | Meaning                                                                                                                                                                                                                                                                                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>native_format</code>  | The native pathname format.                                                                                                                                                                                                                                                                                                                                     |
| <code>generic_format</code> | The generic pathname format.                                                                                                                                                                                                                                                                                                                                    |
| <code>auto_format</code>    | The interpretation of the format of the character sequence is implementation-defined. The implementation may inspect the content of the character sequence to determine the format.<br><i>Recommended practice:</i> For POSIX-based systems, native and generic formats are equivalent and the character sequence should always be interpreted in the same way. |

Table 149 — Enum class `file_type` [tab:fs.enum.file.type]

| Constant                            | Meaning                                                                                                                                                                                                                                           |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>none</code>                   | The type of the file has not been determined or an error occurred while trying to determine the type.                                                                                                                                             |
| <code>not_found</code>              | Pseudo-type indicating the file was not found.<br>[ <i>Note 1:</i> The file not being found is not considered an error while determining the type of a file. — <i>end note</i> ]                                                                  |
| <code>regular</code>                | Regular file                                                                                                                                                                                                                                      |
| <code>directory</code>              | Directory file                                                                                                                                                                                                                                    |
| <code>symlink</code>                | Symbolic link file                                                                                                                                                                                                                                |
| <code>block</code>                  | Block special file                                                                                                                                                                                                                                |
| <code>character</code>              | Character special file                                                                                                                                                                                                                            |
| <code>fifo</code>                   | FIFO or pipe file                                                                                                                                                                                                                                 |
| <code>socket</code>                 | Socket file                                                                                                                                                                                                                                       |
| <code>implementation-defined</code> | Implementations that support file systems having file types in addition to the above <code>file_type</code> types shall supply implementation-defined <code>file_type</code> constants to separately identify each of those additional file types |
| <code>unknown</code>                | The file exists but the type cannot be determined                                                                                                                                                                                                 |

31.12.8.4 Enum class `perms`

[fs.enum.perms]

- <sup>1</sup> The enum class type `perms` is a bitmask type (16.3.3.3.3) that specifies bitmask constants used to identify file permissions, with the meanings listed in Table 151.

31.12.8.5 Enum class `perm_options`

[fs.enum.perm.opts]

- <sup>1</sup> The enum class type `perm_options` is a bitmask type (16.3.3.3.3) that specifies bitmask constants used to control the semantics of permissions operations, with the meanings listed in Table 152. The bitmask constants are bitmask elements. In Table 152 `perm` denotes a value of type `perms` passed to `permissions`.

31.12.8.6 Enum class `directory_options`

[fs.enum.dir.opts]

- <sup>1</sup> The enum class type `directory_options` is a bitmask type (16.3.3.3.3) that specifies bitmask constants used to identify directory traversal options, with the meanings listed in Table 153. The constant `none` represents the empty bitmask; every other constant in the table represents a distinct bitmask element.

31.12.9 Class `file_status`

[fs.class.file.status]

## 31.12.9.1 General

[fs.class.file.status.general]

```
namespace std::filesystem {
 class file_status {
 public:
 // 31.12.9.2, constructors and destructor
 file_status() noexcept : file_status(file_type::none) {}
 explicit file_status(file_type ft,
 perms prms = perms::unknown) noexcept;
```

Table 150 — Enum class `copy_options` [tab:fs.enum.copy.opts]

| Option group controlling <code>copy_file</code> function effects for existing target files |                                                                                                                                                    |
|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Constant                                                                                   | Meaning                                                                                                                                            |
| <code>none</code>                                                                          | (Default) Error; file already exists.                                                                                                              |
| <code>skip_existing</code>                                                                 | Do not overwrite existing file, do not report an error.                                                                                            |
| <code>overwrite_existing</code>                                                            | Overwrite the existing file.                                                                                                                       |
| <code>update_existing</code>                                                               | Overwrite the existing file if it is older than the replacement file.                                                                              |
| Option group controlling copy function effects for subdirectories                          |                                                                                                                                                    |
| Constant                                                                                   | Meaning                                                                                                                                            |
| <code>none</code>                                                                          | (Default) Do not copy subdirectories.                                                                                                              |
| <code>recursive</code>                                                                     | Recursively copy subdirectories and their contents.                                                                                                |
| Option group controlling copy function effects for symbolic links                          |                                                                                                                                                    |
| Constant                                                                                   | Meaning                                                                                                                                            |
| <code>none</code>                                                                          | (Default) Follow symbolic links.                                                                                                                   |
| <code>copy_symlinks</code>                                                                 | Copy symbolic links as symbolic links rather than copying the files that they point to.                                                            |
| <code>skip_symlinks</code>                                                                 | Ignore symbolic links.                                                                                                                             |
| Option group controlling copy function effects for choosing the form of copying            |                                                                                                                                                    |
| Constant                                                                                   | Meaning                                                                                                                                            |
| <code>none</code>                                                                          | (Default) Copy content.                                                                                                                            |
| <code>directories_only</code>                                                              | Copy directory structure only, do not copy non-directory files.                                                                                    |
| <code>create_symlinks</code>                                                               | Make symbolic links instead of copies of files. The source path shall be an absolute path unless the destination path is in the current directory. |
| <code>create_hard_links</code>                                                             | Make hard links instead of copies of files.                                                                                                        |

```

file_status(const file_status&) noexcept = default;
file_status(file_status&&) noexcept = default;
~file_status();

// assignments
file_status& operator=(const file_status&) noexcept = default;
file_status& operator=(file_status&&) noexcept = default;

// 31.12.9.4, modifiers
void type(file_type ft) noexcept;
void permissions(perms prms) noexcept;

// 31.12.9.3, observers
file_type type() const noexcept;
perms permissions() const noexcept;

friend bool operator==(const file_status& lhs, const file_status& rhs) noexcept
{ return lhs.type() == rhs.type() && lhs.permissions() == rhs.permissions(); }
};
}

```

<sup>1</sup> An object of type `file_status` stores information about the type and permissions of a file.

### 31.12.9.2 Constructors

[fs.file.status.cons]

```
explicit file_status(file_type ft, perms prms = perms::unknown) noexcept;
```

<sup>1</sup> *Postconditions:* `type() == ft` and `permissions() == prms`.

Table 151 — Enum class perms [tab:fs.enum.perms]

| Name         | Value<br>(octal) | POSIX<br>macro | Definition or notes                                                                                            |
|--------------|------------------|----------------|----------------------------------------------------------------------------------------------------------------|
| none         | 0                |                | There are no permissions set for the file.                                                                     |
| owner_read   | 0400             | S_IRUSR        | Read permission, owner                                                                                         |
| owner_write  | 0200             | S_IWUSR        | Write permission, owner                                                                                        |
| owner_exec   | 0100             | S_IXUSR        | Execute/search permission, owner                                                                               |
| owner_all    | 0700             | S_IRWXU        | Read, write, execute/search by owner;<br>owner_read   owner_write   owner_exec                                 |
| group_read   | 040              | S_IRGRP        | Read permission, group                                                                                         |
| group_write  | 020              | S_IWGRP        | Write permission, group                                                                                        |
| group_exec   | 010              | S_IXGRP        | Execute/search permission, group                                                                               |
| group_all    | 070              | S_IRWXG        | Read, write, execute/search by group;<br>group_read   group_write   group_exec                                 |
| others_read  | 04               | S_IROTH        | Read permission, others                                                                                        |
| others_write | 02               | S_IWOTH        | Write permission, others                                                                                       |
| others_exec  | 01               | S_IXOTH        | Execute/search permission, others                                                                              |
| others_all   | 07               | S_IRWXO        | Read, write, execute/search by others;<br>others_read   others_write   others_exec                             |
| all          | 0777             |                | owner_all   group_all   others_all                                                                             |
| set_uid      | 04000            | S_ISUID        | Set-user-ID on execution                                                                                       |
| set_gid      | 02000            | S_ISGID        | Set-group-ID on execution                                                                                      |
| sticky_bit   | 01000            | S_ISVTX        | Operating system dependent.                                                                                    |
| mask         | 07777            |                | all   set_uid   set_gid   sticky_bit                                                                           |
| unknown      | 0xFFFF           |                | The permissions are not known, such as when a file_status object is created without specifying the permissions |

Table 152 — Enum class perm\_options [tab:fs.enum.perm.opts]

| Name     | Meaning                                                                                                                                     |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------|
| replace  | permissions shall replace the file's permission bits with perm                                                                              |
| add      | permissions shall replace the file's permission bits with the bitwise OR of perm and the file's current permission bits.                    |
| remove   | permissions shall replace the file's permission bits with the bitwise AND of the complement of perm and the file's current permission bits. |
| nofollow | permissions shall change the permissions of a symbolic link itself rather than the permissions of the file the link resolves to.            |

Table 153 — Enum class directory\_options [tab:fs.enum.dir.opts]

| Name                     | Meaning                                                            |
|--------------------------|--------------------------------------------------------------------|
| none                     | (Default) Skip directory symlinks, permission denied is an error.  |
| follow_directory_symlink | Follow rather than skip directory symlinks.                        |
| skip_permission_denied   | Skip directories that would otherwise result in permission denied. |

**31.12.9.3 Observers**

[fs.file.status.obs]

```
file_type type() const noexcept;
```

- 1 *Returns:* The value of `type()` specified by the postconditions of the most recent call to a constructor, `operator=`, or `type(file_type)` function.

```
perms permissions() const noexcept;
```

- 2 *Returns:* The value of `permissions()` specified by the postconditions of the most recent call to a constructor, `operator=`, or `permissions(perms)` function.

**31.12.9.4 Modifiers**

[fs.file.status.mods]

```
void type(file_type ft) noexcept;
```

- 1 *Postconditions:* `type() == ft`.

```
void permissions(perms prms) noexcept;
```

- 2 *Postconditions:* `permissions() == prms`.

**31.12.10 Class directory\_entry**

[fs.class.directory.entry]

**31.12.10.1 General**

[fs.class.directory.entry.general]

```
namespace std::filesystem {
 class directory_entry {
 public:
 // 31.12.10.2, constructors and destructor
 directory_entry() noexcept = default;
 directory_entry(const directory_entry&) = default;
 directory_entry(directory_entry&&) noexcept = default;
 explicit directory_entry(const filesystem::path& p);
 directory_entry(const filesystem::path& p, error_code& ec);
 ~directory_entry();

 // assignments
 directory_entry& operator=(const directory_entry&) = default;
 directory_entry& operator=(directory_entry&&) noexcept = default;

 // 31.12.10.3, modifiers
 void assign(const filesystem::path& p);
 void assign(const filesystem::path& p, error_code& ec);
 void replace_filename(const filesystem::path& p);
 void replace_filename(const filesystem::path& p, error_code& ec);
 void refresh();
 void refresh(error_code& ec) noexcept;

 // 31.12.10.4, observers
 const filesystem::path& path() const noexcept;
 operator const filesystem::path&() const noexcept;
 bool exists() const;
 bool exists(error_code& ec) const noexcept;
 bool is_block_file() const;
 bool is_block_file(error_code& ec) const noexcept;
 bool is_character_file() const;
 bool is_character_file(error_code& ec) const noexcept;
 bool is_directory() const;
 bool is_directory(error_code& ec) const noexcept;
 bool is_fifo() const;
 bool is_fifo(error_code& ec) const noexcept;
 bool is_other() const;
 bool is_other(error_code& ec) const noexcept;
 bool is_regular_file() const;
 bool is_regular_file(error_code& ec) const noexcept;
 bool is_socket() const;
 bool is_socket(error_code& ec) const noexcept;
```

```

bool is_symlink() const;
bool is_symlink(error_code& ec) const noexcept;
uintmax_t file_size() const;
uintmax_t file_size(error_code& ec) const noexcept;
uintmax_t hard_link_count() const;
uintmax_t hard_link_count(error_code& ec) const noexcept;
file_time_type last_write_time() const;
file_time_type last_write_time(error_code& ec) const noexcept;
file_status status() const;
file_status status(error_code& ec) const noexcept;
file_status symlink_status() const;
file_status symlink_status(error_code& ec) const noexcept;

bool operator==(const directory_entry& rhs) const noexcept;
strong_ordering operator<=>(const directory_entry& rhs) const noexcept;

// 31.12.10.5, inserter
template<class charT, class traits>
friend basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);

private:
 filesystem::path path-object; // exposition only
};
}

```

- 1 A `directory_entry` object stores a `path` object and may store additional objects for file attributes such as hard link count, status, symlink status, file size, and last write time.
- 2 Implementations should store such additional file attributes during directory iteration if their values are available and storing the values would allow the implementation to eliminate file system accesses by `directory_entry` observer functions (31.12.13). Such stored file attribute values are said to be *cached*.
- 3 [Note 1: `directory_iterator` can cache already available attribute values directly into a `directory_entry` object without the cost of a call to `refresh()`. — end note]
- 4 [Example 1:

```

using namespace std::filesystem;

// use possibly cached last write time to minimize disk accesses
for (auto&& x : directory_iterator("."))
{
 std::cout << x.path() << " " << x.last_write_time() << std::endl;
}

// call refresh() to refresh a stale cache
for (auto&& x : directory_iterator("."))
{
 lengthy_function(x.path()); // cache becomes stale
 x.refresh();
 std::cout << x.path() << " " << x.last_write_time() << std::endl;
}

```

On implementations that do not cache the last write time, both loops will result in a potentially expensive call to the `std::filesystem::last_write_time` function. On implementations that do cache the last write time, the first loop will use the cached value and so will not result in a potentially expensive call to the `std::filesystem::last_write_time` function. The code is portable to any implementation, regardless of whether or not it employs caching. — end example]

### 31.12.10.2 Constructors

[fs.dir.entry.cons]

```

explicit directory_entry(const filesystem::path& p);
directory_entry(const filesystem::path& p, error_code& ec);

```

- 1 *Effects:* Calls `refresh()` or `refresh(ec)`, respectively.
- 2 *Postconditions:* `path() == p` if no error occurs, otherwise `path() == filesystem::path()`.



3 *Throws:* As specified in 31.12.5.

### 31.12.10.3 Modifiers

[fs.dir.entry.mods]

```
void assign(const filesystem::path& p);
void assign(const filesystem::path& p, error_code& ec);
```

1 *Effects:* Equivalent to *path-object* = p, then `refresh()` or `refresh(ec)`, respectively. If an error occurs, the values of any cached attributes are unspecified.

2 *Throws:* As specified in 31.12.5.

```
void replace_filename(const filesystem::path& p);
void replace_filename(const filesystem::path& p, error_code& ec);
```

3 *Effects:* Equivalent to *path-object*.`replace_filename(p)`, then `refresh()` or `refresh(ec)`, respectively. If an error occurs, the values of any cached attributes are unspecified.

4 *Throws:* As specified in 31.12.5.

```
void refresh();
void refresh(error_code& ec) noexcept;
```

5 *Effects:* Stores the current values of any cached attributes of the file p resolves to. If an error occurs, an error is reported (31.12.5) and the values of any cached attributes are unspecified.

6 *Throws:* As specified in 31.12.5.

7 [Note 1: Implementations of `directory_iterator` (31.12.11) are prohibited from directly or indirectly calling the `refresh` function as described in 31.12.11.1. — end note]

### 31.12.10.4 Observers

[fs.dir.entry.obs]

1 Unqualified function names in the *Returns:* elements of the `directory_entry` observers described below refer to members of the `std::filesystem` namespace.

```
const filesystem::path& path() const noexcept;
operator const filesystem::path&() const noexcept;
```

2 *Returns:* *path-object*.

```
bool exists() const;
bool exists(error_code& ec) const noexcept;
```

3 *Returns:* `exists(this->status())` or `exists(this->status(ec))`, respectively.

4 *Throws:* As specified in 31.12.5.

```
bool is_block_file() const;
bool is_block_file(error_code& ec) const noexcept;
```

5 *Returns:* `is_block_file(this->status())` or `is_block_file(this->status(ec))`, respectively.

6 *Throws:* As specified in 31.12.5.

```
bool is_character_file() const;
bool is_character_file(error_code& ec) const noexcept;
```

7 *Returns:* `is_character_file(this->status())` or `is_character_file(this->status(ec))`, respectively.

8 *Throws:* As specified in 31.12.5.

```
bool is_directory() const;
bool is_directory(error_code& ec) const noexcept;
```

9 *Returns:* `is_directory(this->status())` or `is_directory(this->status(ec))`, respectively.

10 *Throws:* As specified in 31.12.5.

```
bool is_fifo() const;
bool is_fifo(error_code& ec) const noexcept;
```

11 *Returns:* `is_fifo(this->status())` or `is_fifo(this->status(ec))`, respectively.

12 *Throws:* As specified in 31.12.5.

```

bool is_other() const;
bool is_other(error_code& ec) const noexcept;
13 Returns: is_other(this->status()) or is_other(this->status(ec)), respectively.
14 Throws: As specified in 31.12.5.

bool is_regular_file() const;
bool is_regular_file(error_code& ec) const noexcept;
15 Returns: is_regular_file(this->status()) or is_regular_file(this->status(ec)), respectively.
16 Throws: As specified in 31.12.5.

bool is_socket() const;
bool is_socket(error_code& ec) const noexcept;
17 Returns: is_socket(this->status()) or is_socket(this->status(ec)), respectively.
18 Throws: As specified in 31.12.5.

bool is_symlink() const;
bool is_symlink(error_code& ec) const noexcept;
19 Returns: is_symlink(this->symlink_status()) or is_symlink(this->symlink_status(ec)), respectively.
20 Throws: As specified in 31.12.5.

uintmax_t file_size() const;
uintmax_t file_size(error_code& ec) const noexcept;
21 Returns: If cached, the file size attribute value. Otherwise, file_size(path()) or file_size(path(), ec), respectively.
22 Throws: As specified in 31.12.5.

uintmax_t hard_link_count() const;
uintmax_t hard_link_count(error_code& ec) const noexcept;
23 Returns: If cached, the hard link count attribute value. Otherwise, hard_link_count(path()) or hard_link_count(path(), ec), respectively.
24 Throws: As specified in 31.12.5.

file_time_type last_write_time() const;
file_time_type last_write_time(error_code& ec) const noexcept;
25 Returns: If cached, the last write time attribute value. Otherwise, last_write_time(path()) or last_write_time(path(), ec), respectively.
26 Throws: As specified in 31.12.5.

file_status status() const;
file_status status(error_code& ec) const noexcept;
27 Returns: If cached, the status attribute value. Otherwise, status(path()) or status(path(), ec), respectively.
28 Throws: As specified in 31.12.5.

file_status symlink_status() const;
file_status symlink_status(error_code& ec) const noexcept;
29 Returns: If cached, the symlink status attribute value. Otherwise, symlink_status(path()) or symlink_status(path(), ec), respectively.
30 Throws: As specified in 31.12.5.

bool operator==(const directory_entry& rhs) const noexcept;
31 Returns: path-object == rhs.path-object.

```

```
strong_ordering operator<=>(const directory_entry& rhs) const noexcept;
```

32 *Returns: path-object <=> rhs.path-object.*

### 31.12.10.5 Inserter

[fs.dir.entry.io]

```
template<class charT, class traits>
friend basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const directory_entry& d);
```

1 *Effects:* Equivalent to: `return os << d.path();`

## 31.12.11 Class `directory_iterator`

[fs.class.directory.iterator]

### 31.12.11.1 General

[fs.class.directory.iterator.general]

- 1 An object of type `directory_iterator` provides an iterator for a sequence of `directory_entry` elements representing the path and any cached attribute values (31.12.10) for each file in a directory or in an implementation-defined directory-like file type.

[Note 1: For iteration into subdirectories, see class `recursive_directory_iterator` (31.12.12). — end note]

```
namespace std::filesystem {
class directory_iterator {
public:
 using iterator_category = input_iterator_tag;
 using value_type = directory_entry;
 using difference_type = ptrdiff_t;
 using pointer = const directory_entry*;
 using reference = const directory_entry&;

 // 31.12.11.2, member functions
 directory_iterator() noexcept;
 explicit directory_iterator(const path& p);
 directory_iterator(const path& p, directory_options options);
 directory_iterator(const path& p, error_code& ec);
 directory_iterator(const path& p, directory_options options,
 error_code& ec);
 directory_iterator(const directory_iterator& rhs);
 directory_iterator(directory_iterator&& rhs) noexcept;
 ~directory_iterator();

 directory_iterator& operator=(const directory_iterator& rhs);
 directory_iterator& operator=(directory_iterator&& rhs) noexcept;

 const directory_entry& operator*() const;
 const directory_entry* operator->() const;
 directory_iterator& operator++();
 directory_iterator& increment(error_code& ec);

 bool operator==(default_sentinel_t) const noexcept {
 return *this == directory_iterator();
 }

 // other members as required by 24.3.5.3, input iterators
};
}
```

- 2 `directory_iterator` meets the *Cpp17InputIterator* requirements (24.3.5.3).
- 3 If an iterator of type `directory_iterator` reports an error or is advanced past the last directory element, that iterator shall become equal to the end iterator value. The `directory_iterator` default constructor shall create an iterator equal to the end iterator value, and this shall be the only valid iterator for the end condition.
- 4 The end iterator is not dereferenceable.
- 5 Two end iterators are always equal. An end iterator shall not be equal to a non-end iterator.

- <sup>6</sup> The result of calling the `path()` member of the `directory_entry` object obtained by dereferencing a `directory_iterator` is a reference to a `path` object composed of the directory argument from which the iterator was constructed with the filename of the directory entry appended as if by `operator/=`.
- <sup>7</sup> Directory iteration shall not yield directory entries for the current (dot) and parent (dot-dot) directories.
- <sup>8</sup> The order of directory entries obtained by dereferencing successive increments of a `directory_iterator` is unspecified.
- <sup>9</sup> Constructors and non-const `directory_iterator` member functions store the values of any cached attributes (31.12.10) in the `directory_entry` element returned by `operator*`. `directory_iterator` member functions shall not directly or indirectly call any `directory_entry` refresh function.
- [Note 2: The exact mechanism for storing cached attribute values is not exposed to users. — end note]
- <sup>10</sup> [Note 3: A path obtained by dereferencing a directory iterator might not actually exist; it could be a symbolic link to a non-existent file. Recursively walking directory trees for purposes of removing and renaming entries might invalidate symbolic links that are being followed. — end note]
- <sup>11</sup> [Note 4: If a file is removed from or added to a directory after the construction of a `directory_iterator` for the directory, it is unspecified whether or not subsequently incrementing the iterator will ever result in an iterator referencing the removed or added directory entry. See POSIX `readdir`. — end note]

### 31.12.11.2 Members

[fs.dir.itr.members]

`directory_iterator()` noexcept;

- <sup>1</sup> *Effects:* Constructs the end iterator.

```
explicit directory_iterator(const path& p);
directory_iterator(const path& p, directory_options options);
directory_iterator(const path& p, error_code& ec);
directory_iterator(const path& p, directory_options options, error_code& ec);
```

- <sup>2</sup> *Effects:* For the directory that `p` resolves to, constructs an iterator for the first element in a sequence of `directory_entry` elements representing the files in the directory, if any; otherwise the end iterator. However, if

(`options & directory_options::skip_permission_denied`) != `directory_options::none`

and construction encounters an error indicating that permission to access `p` is denied, constructs the end iterator and does not report an error.

- <sup>3</sup> *Throws:* As specified in 31.12.5.

- <sup>4</sup> [Note 1: To iterate over the current directory, use `directory_iterator(".")` rather than `directory_iterator("")`. — end note]

```
directory_iterator(const directory_iterator& rhs);
directory_iterator(directory_iterator&& rhs) noexcept;
```

- <sup>5</sup> *Postconditions:* `*this` has the original value of `rhs`.

```
directory_iterator& operator=(const directory_iterator& rhs);
directory_iterator& operator=(directory_iterator&& rhs) noexcept;
```

- <sup>6</sup> *Effects:* If `*this` and `rhs` are the same object, the member has no effect.

- <sup>7</sup> *Postconditions:* `*this` has the original value of `rhs`.

- <sup>8</sup> *Returns:* `*this`.

```
directory_iterator& operator++();
directory_iterator& increment(error_code& ec);
```

- <sup>9</sup> *Effects:* As specified for the prefix increment operation of Input iterators (24.3.5.3).

- <sup>10</sup> *Returns:* `*this`.

- <sup>11</sup> *Throws:* As specified in 31.12.5.

### 31.12.11.3 Non-member functions

[fs.dir.itr.nonmembers]

- <sup>1</sup> These functions enable range access for `directory_iterator`.

```
directory_iterator begin(directory_iterator iter) noexcept;
```

2     *Returns:* iter.

```
directory_iterator end(directory_iterator) noexcept;
```

3     *Returns:* directory\_iterator().

### 31.12.12 Class recursive\_directory\_iterator

[fs.class.rec.dir.itr]

#### 31.12.12.1 General

[fs.class.rec.dir.itr.general]

1 An object of type recursive\_directory\_iterator provides an iterator for a sequence of directory\_entry elements representing the files in a directory or in an implementation-defined directory-like file type, and its subdirectories.

```
namespace std::filesystem {
 class recursive_directory_iterator {
 public:
 using iterator_category = input_iterator_tag;
 using value_type = directory_entry;
 using difference_type = ptrdiff_t;
 using pointer = const directory_entry*;
 using reference = const directory_entry&;

 // 31.12.12.2, constructors and destructor
 recursive_directory_iterator() noexcept;
 explicit recursive_directory_iterator(const path& p);
 recursive_directory_iterator(const path& p, directory_options options);
 recursive_directory_iterator(const path& p, directory_options options,
 error_code& ec);
 recursive_directory_iterator(const path& p, error_code& ec);
 recursive_directory_iterator(const recursive_directory_iterator& rhs);
 recursive_directory_iterator(recursive_directory_iterator&& rhs) noexcept;
 ~recursive_directory_iterator();

 // 31.12.12.2, observers
 directory_options options() const;
 int depth() const;
 bool recursion_pending() const;

 const directory_entry& operator*() const;
 const directory_entry* operator->() const;

 // 31.12.12.2, modifiers
 recursive_directory_iterator&
 operator=(const recursive_directory_iterator& rhs);
 recursive_directory_iterator&
 operator=(recursive_directory_iterator&& rhs) noexcept;

 recursive_directory_iterator& operator++();
 recursive_directory_iterator& increment(error_code& ec);

 void pop();
 void pop(error_code& ec);
 void disable_recursion_pending();

 bool operator==(default_sentinel_t) const noexcept {
 return *this == recursive_directory_iterator();
 }

 // other members as required by 24.3.5.3, input iterators
 };
}
```

2 Calling options, depth, recursion\_pending, pop or disable\_recursion\_pending on an iterator that is not dereferenceable results in undefined behavior.

3 The behavior of a `recursive_directory_iterator` is the same as a `directory_iterator` unless otherwise specified.

4 [Note 1: If the directory structure being iterated over contains cycles then it is possible that the end iterator is unreachable. — end note]

### 31.12.12.2 Members

[fs.rec.dir.itr.members]

```
recursive_directory_iterator() noexcept;
```

1 *Effects:* Constructs the end iterator.

```
explicit recursive_directory_iterator(const path& p);
recursive_directory_iterator(const path& p, directory_options options);
recursive_directory_iterator(const path& p, directory_options options, error_code& ec);
recursive_directory_iterator(const path& p, error_code& ec);
```

2 *Effects:* Constructs an iterator representing the first entry in the directory to which `p` resolves, if any; otherwise, the end iterator. However, if

(`options & directory_options::skip_permission_denied`) `!= directory_options::none`

and construction encounters an error indicating that permission to access `p` is denied, constructs the end iterator and does not report an error.

3 *Postconditions:* `options() == options` for the signatures with a `directory_options` argument, otherwise `options() == directory_options::none`.

4 *Throws:* As specified in 31.12.5.

5 [Note 1: Use `recursive_directory_iterator(".")` rather than `recursive_directory_iterator("")` to iterate over the current directory. — end note]

6 [Note 2: By default, `recursive_directory_iterator` does not follow directory symlinks. To follow directory symlinks, specify `options` as `directory_options::follow_directory_symlink`. — end note]

```
recursive_directory_iterator(const recursive_directory_iterator& rhs);
```

7 *Postconditions:*

(7.1) — `options() == rhs.options()`

(7.2) — `depth() == rhs.depth()`

(7.3) — `recursion_pending() == rhs.recursion_pending()`

```
recursive_directory_iterator(recursive_directory_iterator&& rhs) noexcept;
```

8 *Postconditions:* `options()`, `depth()`, and `recursion_pending()` have the values that `rhs.options()`, `rhs.depth()`, and `rhs.recursion_pending()`, respectively, had before the function call.

```
recursive_directory_iterator& operator=(const recursive_directory_iterator& rhs);
```

9 *Effects:* If `*this` and `rhs` are the same object, the member has no effect.

10 *Postconditions:*

(10.1) — `options() == rhs.options()`

(10.2) — `depth() == rhs.depth()`

(10.3) — `recursion_pending() == rhs.recursion_pending()`

11 *Returns:* `*this`.

```
recursive_directory_iterator& operator=(recursive_directory_iterator&& rhs) noexcept;
```

12 *Effects:* If `*this` and `rhs` are the same object, the member has no effect.

13 *Postconditions:* `options()`, `depth()`, and `recursion_pending()` have the values that `rhs.options()`, `rhs.depth()`, and `rhs.recursion_pending()`, respectively, had before the function call.

14 *Returns:* `*this`.

```
directory_options options() const;
```

15 *Returns:* The value of the argument passed to the constructor for the `options` parameter, if present, otherwise `directory_options::none`.

16 *Throws:* Nothing.

`int depth() const;`

17 *Returns:* The current depth of the directory tree being traversed.

[*Note 3:* The initial directory is depth 0, its immediate subdirectories are depth 1, and so forth. — *end note*]

18 *Throws:* Nothing.

`bool recursion_pending() const;`

19 *Returns:* `true` if `disable_recursion_pending()` has not been called subsequent to the prior construction or increment operation, otherwise `false`.

20 *Throws:* Nothing.

`recursive_directory_iterator& operator++();`

`recursive_directory_iterator& increment(error_code& ec);`

21 *Effects:* As specified for the prefix increment operation of Input iterators (24.3.5.3), except that:

(21.1) — If there are no more entries at the current depth, then if `depth() != 0` iteration over the parent directory resumes; otherwise `*this = recursive_directory_iterator()`.

(21.2) — Otherwise if

`recursion_pending() && is_directory((*this)->status()) &&`  
`(!is_symlink((*this)->symlink_status()) ||`  
`(options() & directory_options::follow_directory_symlink) != directory_options::none)`

then either directory `(*this)->path()` is recursively iterated into or, if

`(options() & directory_options::skip_permission_denied) != directory_options::none`

and an error occurs indicating that permission to access directory `(*this)->path()` is denied, then directory `(*this)->path()` is treated as an empty directory and no error is reported.

22 *Returns:* `*this`.

23 *Throws:* As specified in 31.12.5.

`void pop();`

`void pop(error_code& ec);`

24 *Effects:* If `depth() == 0`, set `*this` to `recursive_directory_iterator()`. Otherwise, cease iteration of the directory currently being iterated over, and continue iteration over the parent directory.

25 *Throws:* As specified in 31.12.5.

26 *Remarks:* Any copies of the previous value of `*this` are no longer required to be dereferenceable nor to be in the domain of `==`.

`void disable_recursion_pending();`

27 *Postconditions:* `recursion_pending() == false`.

28 [*Note 4:* `disable_recursion_pending()` is used to prevent unwanted recursion into a directory. — *end note*]

### 31.12.12.3 Non-member functions

[fs.rec.dir.itr.nonmembers]

1 These functions enable use of `recursive_directory_iterator` with range-based `for` statements.

`recursive_directory_iterator begin(recursive_directory_iterator iter) noexcept;`

2 *Returns:* `iter`.

`recursive_directory_iterator end(recursive_directory_iterator) noexcept;`

3 *Returns:* `recursive_directory_iterator()`.

### 31.12.13 Filesystem operation functions

[fs.op.funcs]

#### 31.12.13.1 General

[fs.op.funcs.general]

1 Filesystem operation functions query or modify files, including directories, in external storage.

- <sup>2</sup> [Note 1: Because hardware failures, network failures, file system races (31.12.2.4), and many other kinds of errors occur frequently in file system operations, any filesystem operation function, no matter how apparently innocuous, can encounter an error; see 31.12.5. — end note]

### 31.12.13.2 Absolute

[fs.op.absolute]

```
path filesystem::absolute(const path& p);
path filesystem::absolute(const path& p, error_code& ec);
```

- <sup>1</sup> *Effects:* Composes an absolute path referencing the same file system location as `p` according to the operating system (31.12.2.3).
- <sup>2</sup> *Returns:* The composed path. The signature with argument `ec` returns `path()` if an error occurs.
- <sup>3</sup> [Note 1: For the returned path, `rp`, `rp.is_absolute()` is `true` unless an error occurs. — end note]
- <sup>4</sup> *Throws:* As specified in 31.12.5.
- <sup>5</sup> [Note 2: To resolve symlinks or perform other sanitization that can involve queries to secondary storage, such as hard disks, consider `canonical` (31.12.13.3). — end note]
- <sup>6</sup> [Note 3: Implementations are strongly encouraged to not query secondary storage, and not consider `!exists(p)` an error. — end note]
- <sup>7</sup> [Example 1: For POSIX-based operating systems, `absolute(p)` is simply `current_path()/p`. For Windows-based operating systems, `absolute` might have the same semantics as `GetFullPathNameW`. — end example]

### 31.12.13.3 Canonical

[fs.op.canonical]

```
path filesystem::canonical(const path& p);
path filesystem::canonical(const path& p, error_code& ec);
```

- <sup>1</sup> *Effects:* Converts `p` to an absolute path that has no symbolic link, dot, or dot-dot elements in its pathname in the generic format.
- <sup>2</sup> *Returns:* A path that refers to the same file system object as `absolute(p)`. The signature with argument `ec` returns `path()` if an error occurs.
- <sup>3</sup> *Throws:* As specified in 31.12.5.
- <sup>4</sup> *Remarks:* `!exists(p)` is an error.

### 31.12.13.4 Copy

[fs.op.copy]

```
void filesystem::copy(const path& from, const path& to);
```

- <sup>1</sup> *Effects:* Equivalent to `copy(from, to, copy_options::none)`.

```
void filesystem::copy(const path& from, const path& to, error_code& ec);
```

- <sup>2</sup> *Effects:* Equivalent to `copy(from, to, copy_options::none, ec)`.

```
void filesystem::copy(const path& from, const path& to, copy_options options);
void filesystem::copy(const path& from, const path& to, copy_options options,
 error_code& ec);
```

- <sup>3</sup> *Preconditions:* At most one element from each option group (31.12.8.3) is set in `options`.

- <sup>4</sup> *Effects:* Before the first use of `f` and `t`:

- (4.1) — If
- (`options & copy_options::create_symlinks`) != `copy_options::none` ||  
 (`options & copy_options::skip_symlinks`) != `copy_options::none`
- then `auto f = symlink_status(from)` and if needed `auto t = symlink_status(to)`.
- (4.2) — Otherwise, if
- (`options & copy_options::copy_symlinks`) != `copy_options::none`
- then `auto f = symlink_status(from)` and if needed `auto t = status(to)`.
- (4.3) — Otherwise, `auto f = status(from)` and if needed `auto t = status(to)`.



Effects are then as follows:

- (4.4) — If `f.type()` or `t.type()` is an implementation-defined file type (31.12.8.2), then the effects are implementation-defined.
  - (4.5) — Otherwise, an error is reported as specified in 31.12.5 if
    - (4.5.1) — `exists(f)` is false, or
    - (4.5.2) — `equivalent(from, to)` is true, or
    - (4.5.3) — `is_other(f) || is_other(t)` is true, or
    - (4.5.4) — `is_directory(f) && is_regular_file(t)` is true.
  - (4.6) — Otherwise, if `is_symlink(f)`, then:
    - (4.6.1) — If `(options & copy_options::skip_symlinks) != copy_options::none` then return.
    - (4.6.2) — Otherwise if
      - `!exists(t) && (options & copy_options::copy_symlinks) != copy_options::none`
      - then `copy_symlink(from, to)`.
    - (4.6.3) — Otherwise report an error as specified in 31.12.5.
  - (4.7) — Otherwise, if `is_regular_file(f)`, then:
    - (4.7.1) — If `(options & copy_options::directories_only) != copy_options::none`, then return.
    - (4.7.2) — Otherwise, if `(options & copy_options::create_symlinks) != copy_options::none`, then create a symbolic link to the source file.
    - (4.7.3) — Otherwise, if `(options & copy_options::create_hard_links) != copy_options::none`, then create a hard link to the source file.
    - (4.7.4) — Otherwise, if `is_directory(t)`, then `copy_file(from, to/from.filename(), options)`.
    - (4.7.5) — Otherwise, `copy_file(from, to, options)`.
  - (4.8) — Otherwise, if
    - `is_directory(f) &&`
    - `(options & copy_options::create_symlinks) != copy_options::none`
    - then report an error with an `error_code` argument equal to `make_error_code(errc::is_a_directory)`.
  - (4.9) — Otherwise, if
    - `is_directory(f) &&`
    - `((options & copy_options::recursive) != copy_options::none ||`
    - `options == copy_options::none)`
    - then:
      - (4.9.1) — If `exists(t)` is false, then `create_directory(to, from)`.
      - (4.9.2) — Then, iterate over the files in `from`, as if by
        - `for (const directory_entry& x : directory_iterator(from))`
        - `copy(x.path(), to/x.path().filename(),`
        - `options | copy_options::in-recursive-copy);`
      - where *in-recursive-copy* is a bitmask element of `copy_options` that is not one of the elements in 31.12.8.3.
  - (4.10) — Otherwise, for the signature with argument `ec`, `ec.clear()`.
  - (4.11) — Otherwise, no effects.
- 5 *Throws:* As specified in 31.12.5.
- 6 *Remarks:* For the signature with argument `ec`, any library functions called by the implementation shall have an `error_code` argument if applicable.
- 7 *[Example 1:* Given this directory structure:
- ```
/dir1
  file1
```

```

file2
dir2
  file3

```

Calling `copy("/dir1", "/dir3")` would result in:

```

/dir1
  file1
  file2
  dir2
    file3
/dir3
  file1
  file2

```

Alternatively, calling `copy("/dir1", "/dir3", copy_options::recursive)` would result in:

```

/dir1
  file1
  file2
  dir2
    file3
/dir3
  file1
  file2
  dir2
    file3

```

— end example]

31.12.13.5 Copy file

[fs.op.copy.file]

```

bool filesystem::copy_file(const path& from, const path& to);
bool filesystem::copy_file(const path& from, const path& to, error_code& ec);

```

1 *Returns:* `copy_file(from, to, copy_options::none)` or
`copy_file(from, to, copy_options::none, ec)`, respectively.

2 *Throws:* As specified in 31.12.5.

```

bool filesystem::copy_file(const path& from, const path& to, copy_options options);
bool filesystem::copy_file(const path& from, const path& to, copy_options options,
    error_code& ec);

```

3 *Preconditions:* At most one element from each option group (31.12.8.3) is set in `options`.

4 *Effects:* As follows:

- (4.1) — Report an error as specified in 31.12.5 if
 - (4.1.1) — `is_regular_file(from)` is false, or
 - (4.1.2) — `exists(to)` is true and `is_regular_file(to)` is false, or
 - (4.1.3) — `exists(to)` is true and `equivalent(from, to)` is true, or
 - (4.1.4) — `exists(to)` is true and


```

          (options & (copy_options::skip_existing |
              copy_options::overwrite_existing |
              copy_options::update_existing)) == copy_options::none
          
```
- (4.2) — Otherwise, copy the contents and attributes of the file `from` resolves to, to the file `to` resolves to, if
 - (4.2.1) — `exists(to)` is false, or
 - (4.2.2) — `(options & copy_options::overwrite_existing) != copy_options::none`, or
 - (4.2.3) — `(options & copy_options::update_existing) != copy_options::none` and `from` is more recent than `to`, determined as if by use of the `last_write_time` function (31.12.13.26).
- (4.3) — Otherwise, no effects.

- 5 *Returns:* **true** if the **from** file was copied, otherwise **false**. The signature with argument **ec** returns **false** if an error occurs.
- 6 *Throws:* As specified in 31.12.5.
- 7 *Complexity:* At most one direct or indirect invocation of **status(to)**.

31.12.13.6 Copy symlink

[fs.op.copy.symlink]

```
void filesystem::copy_symlink(const path& existing_symlink, const path& new_symlink);
void filesystem::copy_symlink(const path& existing_symlink, const path& new_symlink,
                             error_code& ec) noexcept;
```

- 1 *Effects:* Equivalent to *function*(*read_symlink*(existing_symlink), new_symlink) or *function*(*read_symlink*(existing_symlink, ec), new_symlink, ec), respectively, where in each case *function* is *create_symlink* or *create_directory_symlink* as appropriate.
- 2 *Throws:* As specified in 31.12.5.

31.12.13.7 Create directories

[fs.op.create.directories]

```
bool filesystem::create_directories(const path& p);
bool filesystem::create_directories(const path& p, error_code& ec);
```

- 1 *Effects:* Calls *create_directory* for each element of **p** that does not exist.
- 2 *Returns:* **true** if a new directory was created for the directory **p** resolves to, otherwise **false**.
- 3 *Throws:* As specified in 31.12.5.
- 4 *Complexity:* $\mathcal{O}(n)$ where n is the number of elements of **p**.

31.12.13.8 Create directory

[fs.op.create.directory]

```
bool filesystem::create_directory(const path& p);
bool filesystem::create_directory(const path& p, error_code& ec) noexcept;
```

- 1 *Effects:* Creates the directory **p** resolves to, as if by POSIX *mkdir* with a second argument of **static_cast<int>(perms::all)**. If *mkdir* fails because **p** resolves to an existing directory, no error is reported. Otherwise on failure an error is reported.
- 2 *Returns:* **true** if a new directory was created, otherwise **false**.
- 3 *Throws:* As specified in 31.12.5.

```
bool filesystem::create_directory(const path& p, const path& existing_p);
bool filesystem::create_directory(const path& p, const path& existing_p, error_code& ec) noexcept;
```

- 4 *Effects:* Creates the directory **p** resolves to, with attributes copied from directory **existing_p**. The set of attributes copied is operating system dependent. If *mkdir* fails because **p** resolves to an existing directory, no error is reported. Otherwise on failure an error is reported.
- [Note 1: For POSIX-based operating systems, the attributes are those copied by native API *stat*(existing_p.c_str(), &attributes_stat) followed by *mkdir*(p.c_str(), attributes_stat.st_mode). For Windows-based operating systems, the attributes are those copied by native API *CreateDirectoryExW*(existing_p.c_str(), p.c_str(), 0). —end note]
- 5 *Returns:* **true** if a new directory was created with attributes copied from directory **existing_p**, otherwise **false**.
- 6 *Throws:* As specified in 31.12.5.

31.12.13.9 Create directory symlink

[fs.op.create.dir.symlink]

```
void filesystem::create_directory_symlink(const path& to, const path& new_symlink);
void filesystem::create_directory_symlink(const path& to, const path& new_symlink,
                                         error_code& ec) noexcept;
```

- 1 *Effects:* Establishes the postcondition, as if by POSIX *symlink*.
- 2 *Postconditions:* **new_symlink** resolves to a symbolic link file that contains an unspecified representation of **to**.
- 3 *Throws:* As specified in 31.12.5.

4 [Note 1: Some operating systems require symlink creation to identify that the link is to a directory. Thus, `create_symlink` (instead of `create_directory_symlink`) cannot be used reliably to create directory symlinks. — end note]

5 [Note 2: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]

31.12.13.10 Create hard link

[fs.op.create.hard.lk]

```
void filesystem::create_hard_link(const path& to, const path& new_hard_link);
void filesystem::create_hard_link(const path& to, const path& new_hard_link,
                                error_code& ec) noexcept;
```

1 *Effects:* Establishes the postcondition, as if by POSIX `link`.

2 *Postconditions:*

(2.1) — `exists(to) && exists(new_hard_link) && equivalent(to, new_hard_link)`

(2.2) — The contents of the file or directory `to` resolves to are unchanged.

3 *Throws:* As specified in 31.12.5.

4 [Note 1: Some operating systems do not support hard links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support hard links regardless of the operating system. Some file systems limit the number of links per file. — end note]

31.12.13.11 Create symlink

[fs.op.create.symlink]

```
void filesystem::create_symlink(const path& to, const path& new_symlink);
void filesystem::create_symlink(const path& to, const path& new_symlink,
                                error_code& ec) noexcept;
```

1 *Effects:* Establishes the postcondition, as if by POSIX `symlink`.

2 *Postconditions:* `new_symlink` resolves to a symbolic link file that contains an unspecified representation of `to`.

3 *Throws:* As specified in 31.12.5.

4 [Note 1: Some operating systems do not support symbolic links at all or support them only for regular files. Some file systems (such as the FAT file system) do not support symbolic links regardless of the operating system. — end note]

31.12.13.12 Current path

[fs.op.current.path]

```
path filesystem::current_path();
path filesystem::current_path(error_code& ec);
```

1 *Returns:* The absolute path of the current working directory, whose pathname in the native format is obtained as if by POSIX `getcwd`. The signature with argument `ec` returns `path()` if an error occurs.

2 *Throws:* As specified in 31.12.5.

3 *Remarks:* The current working directory is the directory, associated with the process, that is used as the starting location in pathname resolution for relative paths.

4 [Note 1: The current path as returned by many operating systems is a dangerous global variable and can be changed unexpectedly by third-party or system library functions, or by another thread. — end note]

```
void filesystem::current_path(const path& p);
void filesystem::current_path(const path& p, error_code& ec) noexcept;
```

5 *Effects:* Establishes the postcondition, as if by POSIX `chdir`.

6 *Postconditions:* `equivalent(p, current_path())`.

7 *Throws:* As specified in 31.12.5.

8 [Note 2: The current path for many operating systems is a dangerous global state and can be changed unexpectedly by third-party or system library functions, or by another thread. — end note]

31.12.13.13 Equivalent**[fs.op.equivalent]**

```
bool filesystem::equivalent(const path& p1, const path& p2);
bool filesystem::equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
```

1 Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location.

[*Note 1*: On POSIX platforms, this is determined as if by the values of the POSIX `stat` class, obtained as if by `stat` for the two paths, having equal `st_dev` values and equal `st_ino` values. — *end note*]

2 *Returns*: `true`, if `p1` and `p2` resolve to the same file system entity, otherwise `false`. The signature with argument `ec` returns `false` if an error occurs.

3 *Throws*: As specified in 31.12.5.

4 *Remarks*: `!exists(p1) || !exists(p2)` is an error.

31.12.13.14 Exists**[fs.op.exists]**

```
bool filesystem::exists(file_status s) noexcept;
```

1 *Returns*: `status_known(s) && s.type() != file_type::not_found`.

```
bool filesystem::exists(const path& p);
bool filesystem::exists(const path& p, error_code& ec) noexcept;
```

2 Let `s` be a `file_status`, determined as if by `status(p)` or `status(p, ec)`, respectively.

3 *Effects*: The signature with argument `ec` calls `ec.clear()` if `status_known(s)`.

4 *Returns*: `exists(s)`.

5 *Throws*: As specified in 31.12.5.

31.12.13.15 File size**[fs.op.file.size]**

```
uintmax_t filesystem::file_size(const path& p);
uintmax_t filesystem::file_size(const path& p, error_code& ec) noexcept;
```

1 *Effects*: If `exists(p)` is `false`, an error is reported (31.12.5).

2 *Returns*:

(2.1) — If `is_regular_file(p)`, the size in bytes of the file `p` resolves to, determined as if by the value of the POSIX `stat` class member `st_size` obtained as if by POSIX `stat`.

(2.2) — Otherwise, the result is implementation-defined.

The signature with argument `ec` returns `static_cast<uintmax_t>(-1)` if an error occurs.

3 *Throws*: As specified in 31.12.5.

31.12.13.16 Hard link count**[fs.op.hard.lk.ct]**

```
uintmax_t filesystem::hard_link_count(const path& p);
uintmax_t filesystem::hard_link_count(const path& p, error_code& ec) noexcept;
```

1 *Returns*: The number of hard links for `p`. The signature with argument `ec` returns `static_cast<uintmax_t>(-1)` if an error occurs.

2 *Throws*: As specified in 31.12.5.

31.12.13.17 Is block file**[fs.op.is.block.file]**

```
bool filesystem::is_block_file(file_status s) noexcept;
```

1 *Returns*: `s.type() == file_type::block`.

```
bool filesystem::is_block_file(const path& p);
bool filesystem::is_block_file(const path& p, error_code& ec) noexcept;
```

2 *Returns*: `is_block_file(status(p))` or `is_block_file(status(p, ec))`, respectively. The signature with argument `ec` returns `false` if an error occurs.

3 *Throws*: As specified in 31.12.5.

31.12.13.18 Is character file**[fs.op.is.char.file]**

```
bool filesystem::is_character_file(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::character`.

```
bool filesystem::is_character_file(const path& p);
```

```
bool filesystem::is_character_file(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_character_file(status(p))` or `is_character_file(status(p, ec))`, respectively. The signature with argument `ec` returns `false` if an error occurs.

3 *Throws:* As specified in [31.12.5](#).

31.12.13.19 Is directory**[fs.op.is.directory]**

```
bool filesystem::is_directory(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::directory`.

```
bool filesystem::is_directory(const path& p);
```

```
bool filesystem::is_directory(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_directory(status(p))` or `is_directory(status(p, ec))`, respectively. The signature with argument `ec` returns `false` if an error occurs.

3 *Throws:* As specified in [31.12.5](#).

31.12.13.20 Is empty**[fs.op.is.empty]**

```
bool filesystem::is_empty(const path& p);
```

```
bool filesystem::is_empty(const path& p, error_code& ec);
```

1 *Effects:*

- (1.1) — Determine `file_status s`, as if by `status(p)` or `status(p, ec)`, respectively.
- (1.2) — For the signature with argument `ec`, return `false` if an error occurred.
- (1.3) — Otherwise, if `is_directory(s)`:
 - (1.3.1) — Create a variable `itr`, as if by `directory_iterator itr(p)` or `directory_iterator itr(p, ec)`, respectively.
 - (1.3.2) — For the signature with argument `ec`, return `false` if an error occurred.
 - (1.3.3) — Otherwise, return `itr == directory_iterator()`.
- (1.4) — Otherwise:
 - (1.4.1) — Determine `uintmax_t sz`, as if by `file_size(p)` or `file_size(p, ec)`, respectively.
 - (1.4.2) — For the signature with argument `ec`, return `false` if an error occurred.
 - (1.4.3) — Otherwise, return `sz == 0`.

2 *Throws:* As specified in [31.12.5](#).

31.12.13.21 Is fifo**[fs.op.is.fifo]**

```
bool filesystem::is_fifo(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::fifo`.

```
bool filesystem::is_fifo(const path& p);
```

```
bool filesystem::is_fifo(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_fifo(status(p))` or `is_fifo(status(p, ec))`, respectively. The signature with argument `ec` returns `false` if an error occurs.

3 *Throws:* As specified in [31.12.5](#).

31.12.13.22 Is other**[fs.op.is.other]**

```
bool filesystem::is_other(file_status s) noexcept;
```

1 *Returns:* `exists(s) && !is_regular_file(s) && !is_directory(s) && !is_symlink(s)`.

```
bool filesystem::is_other(const path& p);
bool filesystem::is_other(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_other(status(p))` or `is_other(status(p, ec))`, respectively. The signature with argument `ec` returns false if an error occurs.

3 *Throws:* As specified in 31.12.5.

31.12.13.23 Is regular file

[fs.op.is.regular.file]

```
bool filesystem::is_regular_file(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::regular`.

```
bool filesystem::is_regular_file(const path& p);
```

2 *Returns:* `is_regular_file(status(p))`.

3 *Throws:* `filesystem_error` if `status(p)` would throw `filesystem_error`.

```
bool filesystem::is_regular_file(const path& p, error_code& ec) noexcept;
```

4 *Effects:* Sets `ec` as if by `status(p, ec)`.

[Note 1: `file_type::none`, `file_type::not_found` and `file_type::unknown` cases set `ec` to error values. To distinguish between cases, call the `status` function directly. — end note]

5 *Returns:* `is_regular_file(status(p, ec))`. Returns false if an error occurs.

31.12.13.24 Is socket

[fs.op.is.socket]

```
bool filesystem::is_socket(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::socket`.

```
bool filesystem::is_socket(const path& p);
```

```
bool filesystem::is_socket(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_socket(status(p))` or `is_socket(status(p, ec))`, respectively. The signature with argument `ec` returns false if an error occurs.

3 *Throws:* As specified in 31.12.5.

31.12.13.25 Is symlink

[fs.op.is.symlink]

```
bool filesystem::is_symlink(file_status s) noexcept;
```

1 *Returns:* `s.type() == file_type::symlink`.

```
bool filesystem::is_symlink(const path& p);
```

```
bool filesystem::is_symlink(const path& p, error_code& ec) noexcept;
```

2 *Returns:* `is_symlink(symlink_status(p))` or `is_symlink(symlink_status(p, ec))`, respectively. The signature with argument `ec` returns false if an error occurs.

3 *Throws:* As specified in 31.12.5.

31.12.13.26 Last write time

[fs.op.last.write.time]

```
file_time_type filesystem::last_write_time(const path& p);
```

```
file_time_type filesystem::last_write_time(const path& p, error_code& ec) noexcept;
```

1 *Returns:* The time of last data modification of `p`, determined as if by the value of the POSIX `stat` class member `st_mtime` obtained as if by POSIX `stat`. The signature with argument `ec` returns `file_time_type::min()` if an error occurs.

2 *Throws:* As specified in 31.12.5.

```
void filesystem::last_write_time(const path& p, file_time_type new_time);
```

```
void filesystem::last_write_time(const path& p, file_time_type new_time,
                                error_code& ec) noexcept;
```

3 *Effects:* Sets the time of last data modification of the file resolved to by `p` to `new_time`, as if by POSIX `futimens`.

4 *Throws:* As specified in 31.12.5.

5 [Note 1: A postcondition of `last_write_time(p) == new_time` is not specified because it does not necessarily hold for file systems with coarse time granularity. — end note]

31.12.13.27 Permissions

[fs.op.permissions]

```
void filesystem::permissions(const path& p, perms prms, perm_options opts=perm_options::replace);
void filesystem::permissions(const path& p, perms prms, error_code& ec) noexcept;
void filesystem::permissions(const path& p, perms prms, perm_options opts, error_code& ec);
```

1 *Preconditions:* Exactly one of the `perm_options` constants `replace`, `add`, or `remove` is present in `opts`.

2 *Effects:* Applies the action specified by `opts` to the file `p` resolves to, or to file `p` itself if `p` is a symbolic link and `perm_options::nofollow` is set in `opts`. The action is applied as if by POSIX `fchmodat`.

3 [Note 1: Conceptually permissions are viewed as bits, but the actual implementation can use some other mechanism. — end note]

4 *Throws:* As specified in 31.12.5.

5 *Remarks:* The second signature behaves as if it had an additional parameter `perm_options opts` with an argument of `perm_options::replace`.

31.12.13.28 Proximate

[fs.op.proximate]

```
path filesystem::proximate(const path& p, error_code& ec);
```

1 *Returns:* `proximate(p, current_path(), ec)`.

2 *Throws:* As specified in 31.12.5.

```
path filesystem::proximate(const path& p, const path& base = current_path());
path filesystem::proximate(const path& p, const path& base, error_code& ec);
```

3 *Returns:* For the first form:

```
    weakly_canonical(p).lexically_proximate(weakly_canonical(base));
```

For the second form:

```
    weakly_canonical(p, ec).lexically_proximate(weakly_canonical(base, ec));
```

or `path()` at the first error occurrence, if any.

4 *Throws:* As specified in 31.12.5.

31.12.13.29 Read symlink

[fs.op.read.symlink]

```
path filesystem::read_symlink(const path& p);
path filesystem::read_symlink(const path& p, error_code& ec);
```

1 *Returns:* If `p` resolves to a symbolic link, a `path` object containing the contents of that symbolic link. The signature with argument `ec` returns `path()` if an error occurs.

2 *Throws:* As specified in 31.12.5.

[Note 1: It is an error if `p` does not resolve to a symbolic link. — end note]

31.12.13.30 Relative

[fs.op.relative]

```
path filesystem::relative(const path& p, error_code& ec);
```

1 *Returns:* `relative(p, current_path(), ec)`.

2 *Throws:* As specified in 31.12.5.

```
path filesystem::relative(const path& p, const path& base = current_path());
path filesystem::relative(const path& p, const path& base, error_code& ec);
```

3 *Returns:* For the first form:

```
    weakly_canonical(p).lexically_relative(weakly_canonical(base));
```

For the second form:

```
    weakly_canonical(p, ec).lexically_relative(weakly_canonical(base, ec));
```

or `path()` at the first error occurrence, if any.

4 *Throws:* As specified in 31.12.5.

31.12.13.31 Remove**[fs.op.remove]**

```
bool filesystem::remove(const path& p);
bool filesystem::remove(const path& p, error_code& ec) noexcept;
```

1 *Effects:* If `exists(symlink_status(p, ec))`, the file `p` is removed as if by POSIX `remove`.

[*Note 1:* A symbolic link is itself removed, rather than the file it resolves to. — *end note*]

2 *Postconditions:* `exists(symlink_status(p))` is `false`.

3 *Returns:* `true` if a file `p` has been removed and `false` otherwise.

[*Note 2:* Absence of a file `p` is not an error. — *end note*]

4 *Throws:* As specified in 31.12.5.

31.12.13.32 Remove all**[fs.op.remove.all]**

```
uintmax_t filesystem::remove_all(const path& p);
uintmax_t filesystem::remove_all(const path& p, error_code& ec);
```

1 *Effects:* Recursively deletes the contents of `p` if it exists, then deletes file `p` itself, as if by POSIX `remove`.

[*Note 1:* A symbolic link is itself removed, rather than the file it resolves to. — *end note*]

2 *Postconditions:* `exists(symlink_status(p))` is `false`.

3 *Returns:* The number of files removed. The signature with argument `ec` returns `static_cast<uintmax_t>(-1)` if an error occurs.

4 *Throws:* As specified in 31.12.5.

31.12.13.33 Rename**[fs.op.rename]**

```
void filesystem::rename(const path& old_p, const path& new_p);
void filesystem::rename(const path& old_p, const path& new_p, error_code& ec) noexcept;
```

1 *Effects:* Renames `old_p` to `new_p`, as if by POSIX `rename`.

[*Note 1:*

(1.1) — If `old_p` and `new_p` resolve to the same existing file, no action is taken.

(1.2) — Otherwise, the rename can include the following effects:

(1.2.1) — if `new_p` resolves to an existing non-directory file, `new_p` is removed; otherwise,

(1.2.2) — if `new_p` resolves to an existing directory, `new_p` is removed if empty on POSIX compliant operating systems but might be an error on other operating systems.

A symbolic link is itself renamed, rather than the file it resolves to. — *end note*]

2 *Throws:* As specified in 31.12.5.

31.12.13.34 Resize file**[fs.op.resize.file]**

```
void filesystem::resize_file(const path& p, uintmax_t new_size);
void filesystem::resize_file(const path& p, uintmax_t new_size, error_code& ec) noexcept;
```

1 *Effects:* Causes the size that would be returned by `file_size(p)` to be equal to `new_size`, as if by POSIX `truncate`.

2 *Throws:* As specified in 31.12.5.

31.12.13.35 Space**[fs.op.space]**

```
space_info filesystem::space(const path& p);
space_info filesystem::space(const path& p, error_code& ec) noexcept;
```

1 *Returns:* An object of type `space_info`. The value of the `space_info` object is determined as if by using POSIX `statvfs` to obtain a POSIX `struct statvfs`, and then multiplying its `f_blocks`, `f_bfree`, and `f_bavail` members by its `f_frsize` member, and assigning the results to the `capacity`, `free`, and `available` members respectively. Any members for which the value cannot be determined shall be set to `static_cast<uintmax_t>(-1)`. For the signature with argument `ec`, all members are set to `static_cast<uintmax_t>(-1)` if an error occurs.

2 *Throws:* As specified in 31.12.5.

3 *Remarks:* The value of member `space_info::available` is operating system dependent.

[*Note 1:* `available` might be less than `free`. — *end note*]

31.12.13.36 Status

[fs.op.status]

```
file_status filesystem::status(const path& p);
```

1 *Effects:* As if by:

```
    error_code ec;
    file_status result = status(p, ec);
    if (result.type() == file_type::none)
        throw filesystem_error(implementation-supplied-message, p, ec);
    return result;
```

2 *Returns:* See above.

3 *Throws:* `filesystem_error`.

[*Note 1:* `result` values of `file_status(file_type::not_found)` and `file_status(file_type::unknown)` are not considered failures and do not cause an exception to be thrown. — *end note*]

```
file_status filesystem::status(const path& p, error_code& ec) noexcept;
```

4 *Effects:* If possible, determines the attributes of the file `p` resolves to, as if by using POSIX `stat` to obtain a POSIX `struct stat`. If, during attribute determination, the underlying file system API reports an error, sets `ec` to indicate the specific error reported. Otherwise, `ec.clear()`.

[*Note 2:* This allows users to inspect the specifics of underlying API errors even when the value returned by `status` is not `file_status(file_type::none)`. — *end note*]

5 Let `prms` denote the result of `(m & perms::mask)`, where `m` is determined as if by converting the `st_mode` member of the obtained `struct stat` to the type `perms`.

6 *Returns:*

(6.1) — If `ec != error_code()`:

(6.1.1) — If the specific error indicates that `p` cannot be resolved because some element of the path does not exist, returns `file_status(file_type::not_found)`.

(6.1.2) — Otherwise, if the specific error indicates that `p` can be resolved but the attributes cannot be determined, returns `file_status(file_type::unknown)`.

(6.1.3) — Otherwise, returns `file_status(file_type::none)`.

[*Note 3:* These semantics distinguish between `p` being known not to exist, `p` existing but not being able to determine its attributes, and there being an error that prevents even knowing if `p` exists. These distinctions are important to some use cases. — *end note*]

(6.2) — Otherwise,

(6.2.1) — If the attributes indicate a regular file, as if by POSIX `S_ISREG`, returns `file_status(file_type::regular, prms)`.

[*Note 4:* `file_type::regular` implies appropriate `<fstream>` operations would succeed, assuming no hardware, permission, access, or file system race errors. Lack of `file_type::regular` does not necessarily imply `<fstream>` operations would fail on a directory. — *end note*]

(6.2.2) — Otherwise, if the attributes indicate a directory, as if by POSIX `S_ISDIR`, returns `file_status(file_type::directory, prms)`.

[*Note 5:* `file_type::directory` implies that calling `directory_iterator(p)` would succeed. — *end note*]

(6.2.3) — Otherwise, if the attributes indicate a block special file, as if by POSIX `S_ISBLK`, returns `file_status(file_type::block, prms)`.

(6.2.4) — Otherwise, if the attributes indicate a character special file, as if by POSIX `S_ISCHR`, returns `file_status(file_type::character, prms)`.

(6.2.5) — Otherwise, if the attributes indicate a fifo or pipe file, as if by POSIX `S_ISFIFO`, returns `file_status(file_type::fifo, prms)`.

- (6.2.6) — Otherwise, if the attributes indicate a socket, as if by POSIX `S_ISSOCK`, returns `file_status(file_type::socket, prms)`.
- (6.2.7) — Otherwise, if the attributes indicate an implementation-defined file type (31.12.8.2), returns `file_status(file_type::A, prms)`, where *A* is the constant for the implementation-defined file type.
- (6.2.8) — Otherwise, returns `file_status(file_type::unknown, prms)`.
- 7 *Remarks:* If a symbolic link is encountered during pathname resolution, pathname resolution continues using the contents of the symbolic link.

31.12.13.37 Status known**[fs.op.status.known]**

```
bool filesystem::status_known(file_status s) noexcept;
```

- 1 *Returns:* `s.type() != file_type::none`.

31.12.13.38 Symlink status**[fs.op.symlink.status]**

```
file_status filesystem::symlink_status(const path& p);
file_status filesystem::symlink_status(const path& p, error_code& ec) noexcept;
```

- 1 *Effects:* Same as `status`, above, except that the attributes of `p` are determined as if by using POSIX `lstat` to obtain a POSIX struct `stat`.
- 2 Let `prms` denote the result of `(m & perms::mask)`, where `m` is determined as if by converting the `st_mode` member of the obtained struct `stat` to the type `perms`.
- 3 *Returns:* Same as `status`, above, except that if the attributes indicate a symbolic link, as if by POSIX `S_ISLNK`, returns `file_status(file_type::symlink, prms)`. The signature with argument `ec` returns `file_status(file_type::none)` if an error occurs.
- 4 *Throws:* As specified in 31.12.5.
- 5 *Remarks:* Pathname resolution terminates if `p` names a symbolic link.

31.12.13.39 Temporary directory path**[fs.op.temp.dir.path]**

```
path filesystem::temp_directory_path();
path filesystem::temp_directory_path(error_code& ec);
```

- 1 Let `p` be an unspecified directory path suitable for temporary files.
- 2 *Effects:* If `exists(p)` is `false` or `is_directory(p)` is `false`, an error is reported (31.12.5).
- 3 *Returns:* The path `p`. The signature with argument `ec` returns `path()` if an error occurs.
- 4 *Throws:* As specified in 31.12.5.
- 5 [Example 1: For POSIX-based operating systems, an implementation might return the path supplied by the first environment variable found in the list `TMPDIR`, `TMP`, `TEMP`, `TEMPDIR`, or if none of these are found, `"/tmp"`.
For Windows-based operating systems, an implementation might return the path reported by the Windows `GetTempPath` API function. — end example]

31.12.13.40 Weakly canonical**[fs.op.weakly.canonical]**

```
path filesystem::weakly_canonical(const path& p);
path filesystem::weakly_canonical(const path& p, error_code& ec);
```

- 1 *Effects:* Using `status(p)` or `status(p, ec)`, respectively, to determine existence, return a path composed by `operator/=` from the result of calling `canonical` with a path argument composed of the leading elements of `p` that exist, if any, followed by the elements of `p` that do not exist, if any. For the first form, `canonical` is called without an `error_code` argument. For the second form, `canonical` is called with `ec` as an `error_code` argument, and `path()` is returned at the first error occurrence, if any.
- 2 *Postconditions:* The returned path is in normal form (31.12.6.2).
- 3 *Returns:* `p` with symlinks resolved and the result normalized (31.12.6.2).
- 4 *Throws:* As specified in 31.12.5.

31.13 C library files

[c.files]

31.13.1 Header <stdio> synopsis

[stdio.syn]

```

#define __STDC_VERSION__ 202311L

namespace std {
    using size_t = see 17.2.4;
    using FILE = see below;
    using fpos_t = see below;
}

#define NULL see 17.2.3
#define _IOFBF see below
#define _IOLBF see below
#define _IONBF see below
#define BUFSIZ see below
#define EOF see below
#define FOPEN_MAX see below
#define FILENAME_MAX see below
#define _PRINTF_NAN_LEN_MAX see below
#define L_tmpnam see below
#define SEEK_CUR see below
#define SEEK_END see below
#define SEEK_SET see below
#define TMP_MAX see below
#define stderr see below
#define stdin see below
#define stdout see below

namespace std {
    int remove(const char* filename);
    int rename(const char* old_p, const char* new_p);
    FILE* tmpfile();
    char* tmpnam(char* s);
    int fclose(FILE* stream);
    int fflush(FILE* stream);
    FILE* fopen(const char* filename, const char* mode);
    FILE* freopen(const char* filename, const char* mode, FILE* stream);
    void setbuf(FILE* stream, char* buf);
    int setvbuf(FILE* stream, char* buf, int mode, size_t size);
    int fprintf(FILE* stream, const char* format, ...);
    int fscanf(FILE* stream, const char* format, ...);
    int printf(const char* format, ...);
    int scanf(const char* format, ...);
    int snprintf(char* s, size_t n, const char* format, ...);
    int sprintf(char* s, const char* format, ...);
    int sscanf(const char* s, const char* format, ...);
    int vfprintf(FILE* stream, const char* format, va_list arg);
    int vfscanf(FILE* stream, const char* format, va_list arg);
    int vprintf(const char* format, va_list arg);
    int vscanf(const char* format, va_list arg);
    int vsnprintf(char* s, size_t n, const char* format, va_list arg);
    int vsprintf(char* s, const char* format, va_list arg);
    int vsscanf(const char* s, const char* format, va_list arg);
    int fgetc(FILE* stream);
    char* fgets(char* s, int n, FILE* stream);
    int fputc(int c, FILE* stream);
    int fputs(const char* s, FILE* stream);
    int getc(FILE* stream);
    int getchar();
    int putc(int c, FILE* stream);
    int putchar(int c);
    int puts(const char* s);
    int ungetc(int c, FILE* stream);

```

```

size_t fread(void* ptr, size_t size, size_t nmemb, FILE* stream);
size_t fwrite(const void* ptr, size_t size, size_t nmemb, FILE* stream);
int fgetpos(FILE* stream, fpos_t* pos);
int fseek(FILE* stream, long int offset, int whence);
int fsetpos(FILE* stream, const fpos_t* pos);
long int ftell(FILE* stream);
void rewind(FILE* stream);
void clearerr(FILE* stream);
int feof(FILE* stream);
int ferror(FILE* stream);
void perror(const char* s);
}

```

- ¹ The contents and meaning of the header `<cstdio>` are the same as the C standard library header `<stdio.h>`.
- ² The return from each function call that delivers data to the host environment to be written to a file (SEE ALSO: ISO/IEC 9899:2024, 7.23.3) is an observable checkpoint (4.1.2).
- ³ When the input item for the `%p` conversion of the `fscanf` function (or equivalent) is one that can be produced from more than one pointer value (6.9.5), the pointer that results is an unspecified choice among all those values that would result in the program having defined behavior. If no such value exists, the behavior is undefined.
- ⁴ Calls to the function `tmpnam` with an argument that is a null pointer value may introduce a data race (16.4.6.10) with other calls to `tmpnam` with an argument that is a null pointer value.

SEE ALSO: ISO/IEC 9899:2024, 7.23

31.13.2 Header `<cinttypes>` synopsis

[`cinttypes.syn`]

```
#include <cstdio> // see 17.4.1
```

```

namespace std {
    using imaxdiv_t = see below;

    constexpr intmax_t imaxabs(intmax_t j);
    constexpr imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
    intmax_t strtoumax(const char* nptr, char** endptr, int base);
    uintmax_t strtoumax(const char* nptr, char** endptr, int base);
    intmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);
    uintmax_t wcstoumax(const wchar_t* nptr, wchar_t** endptr, int base);

    constexpr intmax_t abs(intmax_t); // optional, see below
    constexpr imaxdiv_t div(intmax_t, intmax_t); // optional, see below
}

```

```
#define __STDC_VERSION_INTTYPES_H__ 202311L
```

```

#define PRIdN see below
#define PRIiN see below
#define PRIdN see below
#define PRIuN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below
#define PRIdN see below

```

```

#define PRiXLEASTN see below
#define PRiBLEASTN see below
#define PRiBLEASTN see below
#define SCNdLEASTN see below
#define SCNiLEASTN see below
#define SCNoLEASTN see below
#define SCNuLEASTN see below
#define SCNxLEASTN see below
#define SCNbLEASTN see below
#define PRIdFASTN see below
#define PRIiFASTN see below
#define PRIoFASTN see below
#define PRIuFASTN see below
#define PRiXFASTN see below
#define PRiXFASTN see below
#define PRiBFASTN see below
#define PRIBFASTN see below
#define SCNdFASTN see below
#define SCNiFASTN see below
#define SCNoFASTN see below
#define SCNuFASTN see below
#define SCNxFASTN see below
#define SCNbFASTN see below
#define PRIdMAX see below
#define PRIiMAX see below
#define PRIoMAX see below
#define PRIuMAX see below
#define PRiXMAX see below
#define PRiXMAX see below
#define PRiBMAX see below
#define PRIBMAX see below
#define SCNdMAX see below
#define SCNiMAX see below
#define SCNoMAX see below
#define SCNuMAX see below
#define SCNxMAX see below
#define SCNbMAX see below
#define PRIdPTR see below
#define PRIiPTR see below
#define PRIoPTR see below
#define PRIuPTR see below
#define PRiXPTR see below
#define PRiXPTR see below
#define PRiBPTR see below
#define PRIBPTR see below
#define SCNdPTR see below
#define SCNiPTR see below
#define SCNoPTR see below
#define SCNuPTR see below
#define SCNxPTR see below
#define SCNbPTR see below

```

¹ The contents and meaning of the header `<inttypes>` are the same as the C standard library header `<inttypes.h>`, with the following changes:

- (1.1) — The header `<inttypes>` includes the header `<cstdint>` (17.4.1) instead of `<stdint.h>`, and
- (1.2) — if and only if the type `intmax_t` designates an extended integer type (6.9.3), the following function signatures are added:

```

constexpr intmax_t abs(intmax_t);
constexpr imaxdiv_t div(intmax_t, intmax_t);

```

which shall have the same semantics as the function signatures `constexpr intmax_t imaxabs(intmax_t)` and `constexpr imaxdiv_t imaxdiv(intmax_t, intmax_t)`, respectively.

SEE ALSO: ISO/IEC 9899:2024, 7.8

- ² Each of the **PRI** macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in 17.4.1. Each of the **SCN** macros listed in this subclause is defined if and only if the implementation defines the corresponding *typedef-name* in 17.4.1 and has a suitable **fscanf** length modifier for the type. Each of the **PRIB** macros listed in this subclause is defined if and only if **fprintf** supports the **B** conversion specifier.

32 Concurrency support library [thread]

32.1 General [thread.general]

- ¹ The following subclauses describe components to create and manage threads (6.10.2), perform mutual exclusion, and communicate conditions and values between threads, as summarized in Table 154.

Table 154 — Concurrency support library summary [tab:thread.summary]

Subclause	Header
32.2	Requirements
32.3	Stop tokens <stop_token>
32.4	Threads <thread>
32.5	Atomic operations <atomic>, <stdatomic.h>
32.6	Mutual exclusion <mutex>, <shared_mutex>
32.7	Condition variables <condition_variable>
32.8	Semaphores <semaphore>
32.9	Coordination types <latch>, <barrier>
32.10	Futures <future>
32.11	Safe reclamation <rcu>, <hazard_pointer>

32.2 Requirements [thread.req]

32.2.1 Template parameter names [thread.req.paramname]

- ¹ Throughout this Clause, the names of template parameters are used to express type requirements. `Predicate` is a function object type (22.10). Let `pred` denote an lvalue of type `Predicate`. Then the expression `pred()` shall be well-formed and the type `decltype(pred())` shall model *boolean-testable* (18.5.2). The return value of `pred()`, converted to `bool`, yields `true` if the corresponding test condition is satisfied, and `false` otherwise. If a template parameter is named `Clock`, the corresponding template argument shall be a type `C` that meets the *Cpp17Clock* requirements (30.3); the program is ill-formed if `is_clock_v<C>` is `false`.

32.2.2 Exceptions [thread.req.exception]

- ¹ Some functions described in this Clause are specified to throw exceptions of type `system_error` (19.5.8). Such exceptions are thrown if any of the function's error conditions is detected or a call to an operating system or other underlying API results in an error that prevents the library function from meeting its specifications. Failure to allocate storage is reported as described in 16.4.6.14.

[Example 1: Consider a function in this Clause that is specified to throw exceptions of type `system_error` and specifies error conditions that include `operation_not_permitted` for a thread that does not have the privilege to perform the operation. Assume that, during the execution of this function, an `errno` of `EPERM` is reported by a POSIX API call used by the implementation. Since POSIX specifies an `errno` of `EPERM` when “the caller does not have the privilege to perform the operation”, the implementation maps `EPERM` to an `error_condition` of `operation_not_permitted` (19.5) and an exception of type `system_error` is thrown. — end example]

- ² The `error_code` reported by such an exception's `code()` member function compares equal to one of the conditions specified in the function's error condition element.

32.2.3 Native handles [thread.req.native]

- ¹ Several classes described in this Clause have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined.

[Note 1: These members allow implementations to provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — end note]

32.2.4 Timing specifications

[thread.req.timing]

- ¹ Several functions described in this Clause take an argument to specify a timeout. These timeouts are specified as either a `duration` or a `time_point` type as specified in [Clause 30](#).
- ² Implementations necessarily have some delay in returning from a timeout. Any overhead in interrupt response, function return, and scheduling induces a “quality of implementation” delay, expressed as duration D_i . Ideally, this delay would be zero. Further, any contention for processor and memory resources induces a “quality of management” delay, expressed as duration D_m . The delay durations may vary from timeout to timeout, but in all cases shorter is better.
- ³ The functions whose names end in `_for` take an argument that specifies a duration. These functions produce relative timeouts. Implementations should use a steady clock to measure time for these functions.²⁷² Given a duration argument D_t , the real-time duration of the timeout is $D_t + D_i + D_m$.
- ⁴ The functions whose names end in `_until` take an argument that specifies a time point. These functions produce absolute timeouts. Implementations should use the clock specified in the time point to measure time for these functions. Given a clock time point argument C_t , the clock time point of the return from timeout should be $C_t + D_i + D_m$ when the clock is not adjusted during the timeout. If the clock is adjusted to the time C_a during the timeout, the behavior should be as follows:
 - (4.1) — If $C_a > C_t$, the waiting function should wake as soon as possible, i.e., $C_a + D_i + D_m$, since the timeout is already satisfied. This specification may result in the total duration of the wait decreasing when measured against a steady clock.
 - (4.2) — If $C_a \leq C_t$, the waiting function should not time out until `Clock::now()` returns a time $C_n \geq C_t$, i.e., waking at $C_t + D_i + D_m$.

[Note 1: When the clock is adjusted backwards, this specification can result in the total duration of the wait increasing when measured against a steady clock. When the clock is adjusted forwards, this specification can result in the total duration of the wait decreasing when measured against a steady clock. — end note]

An implementation returns from such a timeout at any point from the time specified above to the time it would return from a steady-clock relative timeout on the difference between C_t and the time point of the call to the `_until` function.

Recommended practice: Implementations should decrease the duration of the wait when the clock is adjusted forwards.

- ⁵ [Note 2: If the clock is not synchronized with a steady clock, e.g., a CPU time clock, these timeouts can fail to provide useful functionality. — end note]
- ⁶ The resolution of timing provided by an implementation depends on both operating system and hardware. The finest resolution provided by an implementation is called the *native resolution*.
- ⁷ Implementation-provided clocks that are used for these functions meet the *Cpp17TrivialClock* requirements ([30.3](#)).
- ⁸ A function that takes an argument which specifies a timeout will throw if, during its execution, a clock, time point, or time duration throws an exception. Such exceptions are referred to as *timeout-related exceptions*.
[Note 3: Instantiations of clock, time point and duration types supplied by the implementation as specified in [30.7](#) do not throw exceptions. — end note]

32.2.5 Requirements for *Cpp17Lockable* types

[thread.req.lockable]

32.2.5.1 General

[thread.req.lockable.general]

- ¹ An *execution agent* is an entity such as a thread that may perform work in parallel with other execution agents.
[Note 1: Implementations or users can introduce other kinds of agents such as processes or thread-pool tasks. — end note]
The calling agent is determined by context, e.g., the calling thread that contains the call, and so on.
- ² [Note 2: Some lockable objects are “agent oblivious” in that they work for any execution agent model because they do not determine or store the agent’s ID (e.g., an ordinary spin lock). — end note]

²⁷²) Implementations for which standard time units are meaningful will typically have a steady clock within their hardware implementation.

- ³ The standard library templates `unique_lock` (32.6.5.4), `shared_lock` (32.6.5.5), `scoped_lock` (32.6.5.3), `lock_guard` (32.6.5.2), `lock`, `try_lock` (32.6.6), and `condition_variable_any` (32.7.5) all operate on user-supplied lockable objects. The *Cpp17BasicLockable* requirements, the *Cpp17Lockable* requirements, the *Cpp17TimedLockable* requirements, the *Cpp17SharedLockable* requirements, and the *Cpp17SharedTimedLockable* requirements list the requirements imposed by these library types in order to acquire or release ownership of a lock by a given execution agent.

[Note 3: The nature of any lock ownership and any synchronization it entails are not part of these requirements. — end note]

- ⁴ A lock on an object `m` is said to be

- (4.1) — a *non-shared lock* if it is acquired by a call to `lock`, `try_lock`, `try_lock_for`, or `try_lock_until` on `m`, or
- (4.2) — a *shared lock* if it is acquired by a call to `lock_shared`, `try_lock_shared`, `try_lock_shared_for`, or `try_lock_shared_until` on `m`.

[Note 4: Only the method of lock acquisition is considered; the nature of any lock ownership is not part of these definitions. — end note]

32.2.5.2 *Cpp17BasicLockable* requirements

[thread.req.lockable.basic]

- ¹ A type `L` meets the *Cpp17BasicLockable* requirements if the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`).

`m.lock()`

- ² *Effects*: Blocks until a lock can be acquired for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

`m.unlock()`

- ³ *Preconditions*: The current execution agent holds a non-shared lock on `m`.

- ⁴ *Effects*: Releases a non-shared lock on `m` held by the current execution agent.

- ⁵ *Throws*: Nothing.

32.2.5.3 *Cpp17Lockable* requirements

[thread.req.lockable.req]

- ¹ A type `L` meets the *Cpp17Lockable* requirements if it meets the *Cpp17BasicLockable* requirements and the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`).

`m.try_lock()`

- ² *Effects*: Attempts to acquire a lock for the current execution agent without blocking. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

- ³ *Return type*: `bool`.

- ⁴ *Returns*: `true` if the lock was acquired, otherwise `false`.

32.2.5.4 *Cpp17TimedLockable* requirements

[thread.req.lockable.timed]

- ¹ A type `L` meets the *Cpp17TimedLockable* requirements if it meets the *Cpp17Lockable* requirements and the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`, `rel_time` denotes a value of a specialization of `duration` (30.5), and `abs_time` denotes a value of a specialization of `time_point` (30.6)).

`m.try_lock_for(rel_time)`

- ² *Effects*: Attempts to acquire a lock for the current execution agent within the relative timeout (32.2.4) specified by `rel_time`. The function will not return within the timeout specified by `rel_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

- ³ *Return type*: `bool`.

- ⁴ *Returns*: `true` if the lock was acquired, otherwise `false`.

`m.try_lock_until(abs_time)`

- 5 *Effects:* Attempts to acquire a lock for the current execution agent before the absolute timeout (32.2.4) specified by `abs_time`. The function will not return before the timeout specified by `abs_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

6 *Return type:* `bool`.

7 *Returns:* `true` if the lock was acquired, otherwise `false`.

32.2.5.5 *Cpp17SharedLockable* requirements [thread.req.lockable.shared]

- 1 A type `L` meets the *Cpp17SharedLockable* requirements if the following expressions are well-formed, have the specified semantics, and the expression `m.try_lock_shared()` has type `bool` (`m` denotes a value of type `L`):

`m.lock_shared()`

- 2 *Effects:* Blocks until a lock can be acquired for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

`m.try_lock_shared()`

- 3 *Effects:* Attempts to acquire a lock for the current execution agent without blocking. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

4 *Returns:* `true` if the lock was acquired, `false` otherwise.

`m.unlock_shared()`

- 5 *Preconditions:* The current execution agent holds a shared lock on `m`.

6 *Effects:* Releases a shared lock on `m` held by the current execution agent.

7 *Throws:* Nothing.

32.2.5.6 *Cpp17SharedTimedLockable* requirements [thread.req.lockable.shared.timed]

- 1 A type `L` meets the *Cpp17SharedTimedLockable* requirements if it meets the *Cpp17SharedLockable* requirements, and the following expressions are well-formed, have type `bool`, and have the specified semantics (`m` denotes a value of type `L`, `rel_time` denotes a value of a specialization of `chrono::duration`, and `abs_time` denotes a value of a specialization of `chrono::time_point`).

`m.try_lock_shared_for(rel_time)`

- 2 *Effects:* Attempts to acquire a lock for the current execution agent within the relative timeout (32.2.4) specified by `rel_time`. The function will not return within the timeout specified by `rel_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

3 *Returns:* `true` if the lock was acquired, `false` otherwise.

`m.try_lock_shared_until(abs_time)`

- 4 *Effects:* Attempts to acquire a lock for the current execution agent before the absolute timeout (32.2.4) specified by `abs_time`. The function will not return before the timeout specified by `abs_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock has not been acquired for the current execution agent.

5 *Returns:* `true` if the lock was acquired, `false` otherwise.

32.3 Stop tokens [thread.stoptoken]

32.3.1 Introduction [thread.stoptoken.intro]

- 1 Subclause 32.3 describes components that can be used to asynchronously request that an operation stops execution in a timely manner, typically because the result is no longer required. Such a request is called a *stop request*.
- 2 The concepts *stoppable-source*, *stoppable_token*, and *stoppable-callback-for* specify the required syntax and semantics of shared access to a *stop state*. Any object modeling *stoppable-source*, *stoppable_token*, or *stoppable-callback-for* that refers to the same stop state is an *associated stoppable-source*, *stoppable_token*, or *stoppable-callback-for*, respectively.

- ³ An object of a type that models `stoppable_token` can be passed to an operation that can either
- (3.1) — actively poll the token to check if there has been a stop request, or
 - (3.2) — register a callback that will be called in the event that a stop request is made.

A stop request made via an object whose type models *stoppable-source* will be visible to all associated `stoppable_token` and *stoppable-source* objects. Once a stop request has been made it cannot be withdrawn (a subsequent stop request has no effect).

- ⁴ Callbacks registered via an object whose type models *stoppable-callback-for* are called when a stop request is first made by any associated *stoppable-source* object.
- ⁵ The types `stop_source` and `stop_token` and the class template `stop_callback` implement the semantics of shared ownership of a stop state. The last remaining owner of the stop state automatically releases the resources associated with the stop state.
- ⁶ An object of type `inplace_stop_source` is the sole owner of its stop state. An object of type `inplace_stop_token` or of a specialization of the class template `inplace_stop_callback` does not participate in ownership of its associated stop state.

[Note 1: They are for use when all uses of the associated token and callback objects are known to nest within the lifetime of the `inplace_stop_source` object. — end note]

32.3.2 Header `<stop_token>` synopsis

[thread.stoptoken.syn]

```
namespace std {
    // 32.3.3, stop token concepts
    template<class CallbackFn, class Token, class Initializer = CallbackFn>
        concept stoppable-callback-for = see below;           // exposition only

    template<class Token>
        concept stoppable_token = see below;

    template<class Token>
        concept unstoppable_token = see below;

    template<class Source>
        concept stoppable-source = see below;                 // exposition only

    // 32.3.4, class stop_token
    class stop_token;

    // 32.3.5, class stop_source
    class stop_source;

    // no-shared-stop-state indicator
    struct nostopstate_t {
        explicit nostopstate_t() = default;
    };
    inline constexpr nostopstate_t nostopstate{};

    // 32.3.6, class template stop_callback
    template<class Callback>
        class stop_callback;

    // 32.3.7, class never_stop_token
    class never_stop_token;

    // 32.3.8, class inplace_stop_token
    class inplace_stop_token;

    // 32.3.9, class inplace_stop_source
    class inplace_stop_source;

    // 32.3.10, class template inplace_stop_callback
    template<class CallbackFn>
        class inplace_stop_callback;
```

```

template<class T, class CallbackFn>
    using stop_callback_for_t = T::template callback_type<CallbackFn>;
}

```

32.3.3 Stop token concepts

[stoptoken.concepts]

- ¹ The exposition-only *stoppable-callback-for* concept checks for a callback compatible with a given Token type.

```

template<class CallbackFn, class Token, class Initializer = CallbackFn>
    concept stoppable-callback-for =                // exposition only
        invocable<CallbackFn> &&
        constructible_from<CallbackFn, Initializer> &&
        requires { typename stop_callback_for_t<Token, CallbackFn>; } &&
        constructible_from<stop_callback_for_t<Token, CallbackFn>, const Token&, Initializer>;

```

- ² Let *t* and *u* be distinct, valid objects of type Token that reference the same logical stop state; let *init* be an expression such that `same_as<decltype(init), Initializer>` is true; and let SCB denote the type `stop_callback_for_t<Token, CallbackFn>`.

- ³ The concept *stoppable-callback-for*<CallbackFn, Token, Initializer> is modeled only if:

- (3.1) — The following concepts are modeled:
 - (3.1.1) — `constructible_from<SCB, Token, Initializer>`
 - (3.1.2) — `constructible_from<SCB, Token&, Initializer>`
 - (3.1.3) — `constructible_from<SCB, const Token, Initializer>`
- (3.2) — An object of type SCB has an associated callback function of type CallbackFn. Let *scb* be an object of type SCB and let *callback_fn* denote *scb*'s associated callback function. Direct-non-list-initializing *scb* from arguments *t* and *init* shall execute a *stoppable callback registration* as follows:
 - (3.2.1) — If *t.stop_possible()* is true:
 - (3.2.1.1) — *callback_fn* shall be direct-initialized with *init*.
 - (3.2.1.2) — Construction of *scb* shall only throw exceptions thrown by the initialization of *callback_fn* from *init*.
 - (3.2.1.3) — The callback invocation `std::forward<CallbackFn>(callback_fn)()` shall be registered with *t*'s associated stop state as follows:
 - (3.2.1.3) — If *t.stop_requested()* evaluates to false at the time of registration, the callback invocation is added to the stop state's list of callbacks such that `std::forward<CallbackFn>(callback_fn)()` is evaluated if a stop request is made on the stop state.
 - (3.2.1.3) — Otherwise, `std::forward<CallbackFn>(callback_fn)()` shall be immediately evaluated on the thread executing *scb*'s constructor, and the callback invocation shall not be added to the list of callback invocations.
 - If the callback invocation was added to stop state's list of callbacks, *scb* shall be associated with the stop state.

[Note 1: If *t.stop_possible()* is false, there is no requirement that the initialization of *scb* causes the initialization of *callback_fn*. — end note]
- (3.3) — Destruction of *scb* shall execute a *stoppable callback deregistration* as follows (in order):
 - (3.3.1) — If the constructor of *scb* did not register a callback invocation with *t*'s stop state, then the stoppable callback deregistration shall have no effect other than destroying *callback_fn* if it was constructed.
 - (3.3.2) — Otherwise, the invocation of *callback_fn* shall be removed from the associated stop state.
 - (3.3.3) — If *callback_fn* is concurrently executing on another thread, then the stoppable callback deregistration shall block (3.6) until the invocation of *callback_fn* returns such that the return from the invocation of *callback_fn* strongly happens before (6.10.2.2) the destruction of *callback_fn*.
 - (3.3.4) — If *callback_fn* is executing on the current thread, then the destructor shall not block waiting for the return from the invocation of *callback_fn*.

- (3.3.5) — A stoppable callback deregistration shall not block on the completion of the invocation of some other callback registered with the same logical stop state.
- (3.3.6) — The stoppable callback deregistration shall destroy `callback_fn`.

- 4 The `stoppable_token` concept checks for the basic interface of a stop token that is copyable and allows polling to see if stop has been requested and also whether a stop request is possible. The `unstoppable_token` concept checks for a `stoppable_token` type that does not allow stopping.

```
template<template<class> class>
    struct check-type-alias-exists;                                // exposition only

template<class Token>
concept stoppable_token =
    requires (const Token tok) {
        typename check-type-alias-exists<Token::template callback_type>;
        { tok.stop_requested() } noexcept -> same_as<bool>;
        { tok.stop_possible() } noexcept -> same_as<bool>;
        { Token(tok) } noexcept;                                // see implicit expression variations (18.2)
    } &&
    copyable<Token> &&
    equality_comparable<Token>;

template<class Token>
concept unstoppable_token =
    stoppable_token<Token> &&
    requires (const Token tok) {
        requires bool_constant<(!tok.stop_possible())>::value;
    };
};
```

- 5 An object whose type models `stoppable_token` has at most one associated logical stop state. A `stoppable_token` object with no associated stop state is said to be *disengaged*.
- 6 Let SP be an evaluation of `t.stop_possible()` that is false, and let SR be an evaluation of `t.stop_requested()` that is true.
- 7 The type `Token` models `stoppable_token` only if:
- (7.1) — Any evaluation of `u.stop_possible()` or `u.stop_requested()` that happens after (6.10.2.2) SP is false.
- (7.2) — Any evaluation of `u.stop_possible()` or `u.stop_requested()` that happens after SR is true.
- (7.3) — For any types `CallbackFn` and `Initializer` such that *stoppable-callback-for*<`CallbackFn`, `Token`, `Initializer`> is satisfied, *stoppable-callback-for*<`CallbackFn`, `Token`, `Initializer`> is modeled.
- (7.4) — If `t` is disengaged, evaluations of `t.stop_possible()` and `t.stop_requested()` are false.
- (7.5) — If `t` and `u` reference the same stop state, or if both `t` and `u` are disengaged, `t == u` is true; otherwise, it is false.
- 8 An object whose type models the exposition-only *stoppable-source* concept can be queried whether stop has been requested (`stop_requested`) and whether stop is possible (`stop_possible`). It is a factory for associated stop tokens (`get_token`), and a stop request can be made on it (`request_stop`). It maintains a list of registered stop callback invocations that it executes when a stop request is first made.

```
template<class Source>
concept stoppable-source =                                       // exposition only
    requires (Source& src, const Source csrc) {                 // see implicit expression variations (18.2)
        { csrc.get_token() } -> stoppable_token;
        { csrc.stop_possible() } noexcept -> same_as<bool>;
        { csrc.stop_requested() } noexcept -> same_as<bool>;
        { src.request_stop() } -> same_as<bool>;
    };
};
```

- 9 An object whose type models *stoppable-source* has at most one associated logical stop state. If it has no associated stop state, it is said to be disengaged. Let `s` be an object whose type models *stoppable-source* and that is disengaged. `s.stop_possible()` and `s.stop_requested()` shall be false.

- ¹⁰ Let `t` be an object whose type models *stoppable-source*. If `t` is disengaged, `t.get_token()` shall return a disengaged stop token; otherwise, it shall return a stop token that is associated with the stop state of `t`.
- ¹¹ Calls to the member functions `request_stop`, `stop_requested`, and `stop_possible` and similarly named member functions on associated `stoppable_token` objects do not introduce data races. A call to `request_stop` that returns `true` synchronizes with a call to `stop_requested` on an associated `stoppable_token` or *stoppable-source* object that returns `true`. Registration of a callback synchronizes with the invocation of that callback.
- ¹² If the *stoppable-source* is disengaged, `request_stop` shall have no effect and return `false`. Otherwise, it shall execute a *stop request operation* on the associated stop state. A stop request operation determines whether the stop state has received a stop request, and if not, makes a stop request. The determination and making of the stop request shall happen atomically, as-if by a read-modify-write operation (6.10.2.2). If the request was made, the stop state's registered callback invocations shall be synchronously executed. If an invocation of a callback exits via an exception then `terminate` shall be invoked (14.6.2).

[Note 2: No constraint is placed on the order in which the callback invocations are executed. — end note]

`request_stop` shall return `true` if a stop request was made, and `false` otherwise. After a call to `request_stop` either a call to `stop_possible` shall return `false` or a call to `stop_requested` shall return `true`.

[Note 3: A stop request includes notifying all condition variables of type `condition_variable_any` temporarily registered during an interruptible wait (32.7.5.3). — end note]

32.3.4 Class `stop_token`

[stoptoken]

32.3.4.1 General

[stoptoken.general]

- ¹ The class `stop_token` models the concept `stoppable_token`. It shares ownership of its stop state, if any, with its associated `stop_source` object (32.3.5) and any `stop_token` objects to which it compares equal.

```
namespace std {
    class stop_token {
    public:
        template<class CallbackFn>
            using callback_type = stop_callback<CallbackFn>;

        stop_token() noexcept = default;

        // 32.3.4.2, member functions
        void swap(stop_token&) noexcept;

        bool stop_requested() const noexcept;
        bool stop_possible() const noexcept;

        bool operator==(const stop_token& rhs) noexcept = default;

    private:
        shared_ptr<unspecified> stop-state;           // exposition only
    };
}
```

- ² *stop-state* refers to the `stop_token`'s associated stop state. A `stop_token` object is disengaged when *stop-state* is empty.

32.3.4.2 Member functions

[stoptoken.mem]

```
void swap(stop_token& rhs) noexcept;
```

- ¹ *Effects:* Equivalent to:

```
stop-state.swap(rhs.stop-state);
```

```
bool stop_requested() const noexcept;
```

- ² *Returns:* `true` if *stop-state* refers to a stop state that has received a stop request; otherwise, `false`.

```
bool stop_possible() const noexcept;
```

- ³ *Returns:* `false` if

- (3.1) — ***this** is disengaged, or
- (3.2) — a stop request was not made and there are no associated `stop_source` objects; otherwise, `true`.

32.3.5 Class `stop_source`

[stopsource]

32.3.5.1 General

[stopsource.general]

```
namespace std {
    class stop_source {
    public:
        // 32.3.5.2, constructors, copy, and assignment
        stop_source();
        explicit stop_source(nostopstate_t) noexcept {}

        // 32.3.5.3, member functions
        void swap(stop_source&) noexcept;

        stop_token get_token() const noexcept;
        bool stop_possible() const noexcept;
        bool stop_requested() const noexcept;
        bool request_stop() noexcept;

        bool operator==(const stop_source& rhs) noexcept = default;

    private:
        shared_ptr<unspecified> stop-state;           // exposition only
    };
}
```

- ¹ *stop-state* refers to the `stop_source`'s associated stop state. A `stop_source` object is disengaged when *stop-state* is empty.
- ² `stop_source` models *stoppable-source*, copyable, equality_comparable, and swappable.

32.3.5.2 Constructors, copy, and assignment

[stopsource.cons]

`stop_source();`

- ¹ *Effects*: Initializes *stop-state* with a pointer to a new stop state.
- ² *Postconditions*: `stop_possible()` is `true` and `stop_requested()` is `false`.
- ³ *Throws*: `bad_alloc` if memory cannot be allocated for the stop state.

32.3.5.3 Member functions

[stopsource.mem]

`void swap(stop_source& rhs) noexcept;`

- ¹ *Effects*: Equivalent to:
- ```
stop-state.swap(rhs.stop-state);
```

`stop_token get_token() const noexcept;`

- <sup>2</sup> *Returns*: `stop_token()` if `stop_possible()` is `false`; otherwise a new associated `stop_token` object; i.e., its *stop-state* member is equal to the *stop-state* member of **\*this**.

`bool stop_possible() const noexcept;`

- <sup>3</sup> *Returns*: *stop-state* != `nullptr`.

`bool stop_requested() const noexcept;`

- <sup>4</sup> *Returns*: `true` if *stop-state* refers to a stop state that has received a stop request; otherwise, `false`.

`bool request_stop() noexcept;`

- <sup>5</sup> *Effects*: Executes a stop request operation (32.3.3) on the associated stop state, if any.



**32.3.6 Class template stop\_callback****[stopcallback]****32.3.6.1 General****[stopcallback.general]**

```

1 namespace std {
 template<class CallbackFn>
 class stop_callback {
 public:
 using callback_type = CallbackFn;

 // 32.3.6.2, constructors and destructor
 template<class Initializer>
 explicit stop_callback(const stop_token& st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);
 template<class Initializer>
 explicit stop_callback(stop_token&& st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);
 ~stop_callback();

 stop_callback(const stop_callback&) = delete;
 stop_callback(stop_callback&&) = delete;
 stop_callback& operator=(const stop_callback&) = delete;
 stop_callback& operator=(stop_callback&&) = delete;

 private:
 CallbackFn callback_fn; // exposition only
 };

 template<class CallbackFn>
 stop_callback(stop_token, CallbackFn) -> stop_callback<CallbackFn>;
}

```

- 2 *Mandates:* `stop_callback` is instantiated with an argument for the template parameter `CallbackFn` that satisfies both invocable and destructible.
- 3 *Remarks:* For a type `Initializer`, if *stoppable-callback-for*<`CallbackFn`, `stop_token`, `Initializer`> is satisfied, then *stoppable-callback-for*<`CallbackFn`, `stop_token`, `Initializer`> is modeled. The exposition-only *callback-fn* member is the associated callback function (32.3.3) of `stop_callback<CallbackFn>` objects.

**32.3.6.2 Constructors and destructor****[stopcallback.cons]**

```

template<class Initializer>
explicit stop_callback(const stop_token& st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);

template<class Initializer>
explicit stop_callback(stop_token&& st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);

```

- 1 *Constraints:* `CallbackFn` and `Initializer` satisfy *constructible\_from*<`CallbackFn`, `Initializer`>.
- 2 *Effects:* Initializes *callback-fn* with `std::forward<Initializer>(init)` and executes a stoppable callback registration (32.3.3). If a callback is registered with `st`'s shared stop state, then `*this` acquires shared ownership of that stop state.
- `~stop_callback();`
- 3 *Effects:* Executes a stoppable callback deregistration (32.3.3) and releases ownership of the stop state, if any.

**32.3.7 Class never\_stop\_token****[stoptoken.never]**

- 1 The class `never_stop_token` models the *unstoppable\_token* concept. It provides a stop token interface, but also provides static information that a stop is never possible nor requested.

```

namespace std {
 class never_stop_token {
 struct callback-type { // exposition only
 explicit callback-type(never_stop_token, auto&&) noexcept {}
 };
 public:
 template<class>
 using callback_type = callback-type;

 static constexpr bool stop_requested() noexcept { return false; }
 static constexpr bool stop_possible() noexcept { return false; }

 bool operator==(const never_stop_token&) const = default;
 };
}

```

### 32.3.8 Class `inplace_stop_token`

[stoptoken.inplace]

#### 32.3.8.1 General

[stoptoken.inplace.general]

- <sup>1</sup> The class `inplace_stop_token` models the concept `stoppable_token`. It references the stop state of its associated `inplace_stop_source` object (32.3.9), if any.

```

namespace std {
 class inplace_stop_token {
 public:
 template<class CallbackFn>
 using callback_type = inplace_stop_callback<CallbackFn>;

 inplace_stop_token() = default;
 bool operator==(const inplace_stop_token&) const = default;

 // 32.3.8.2, member functions
 bool stop_requested() const noexcept;
 bool stop_possible() const noexcept;
 void swap(inplace_stop_token&) noexcept;

 private:
 const inplace_stop_source* stop-source = nullptr; // exposition only
 };
}

```

#### 32.3.8.2 Member functions

[stoptoken.inplace.mem]

`void swap(inplace_stop_token& rhs) noexcept;`

- <sup>1</sup> *Effects:* Exchanges the values of *stop-source* and *rhs.stop-source*.

`bool stop_requested() const noexcept;`

- <sup>2</sup> *Effects:* Equivalent to:

```
return stop-source != nullptr && stop-source->stop_requested();
```

- <sup>3</sup> [Note 1: As specified in 6.8.4, the behavior of `stop_requested` is undefined unless the call strongly happens before the start of the destructor of the associated `inplace_stop_source` object, if any. — end note]

`stop_possible() const noexcept;`

- <sup>4</sup> *Returns:* *stop-source* != nullptr.

- <sup>5</sup> [Note 2: As specified in 6.8.6.1, the behavior of `stop_possible` is implementation-defined unless the call strongly happens before the end of the storage duration of the associated `inplace_stop_source` object, if any. — end note]

### 32.3.9 Class `inplace_stop_source`

[stopsource.inplace]

#### 32.3.9.1 General

[stopsource.inplace.general]

- <sup>1</sup> The class `inplace_stop_source` models *stoppable-source*.

```

namespace std {
 class inplace_stop_source {
 public:
 // 32.3.9.2, constructors
 constexpr inplace_stop_source() noexcept;

 inplace_stop_source(inplace_stop_source&&) = delete;
 inplace_stop_source(const inplace_stop_source&) = delete;
 inplace_stop_source& operator=(inplace_stop_source&&) = delete;
 inplace_stop_source& operator=(const inplace_stop_source&) = delete;
 ~inplace_stop_source();

 // 32.3.9.3, stop handling
 constexpr inplace_stop_token get_token() const noexcept;
 static constexpr bool stop_possible() noexcept { return true; }
 bool stop_requested() const noexcept;
 bool request_stop() noexcept;
 };
}

```

### 32.3.9.2 Constructors

[stopsource.inplace.cons]

```
constexpr inplace_stop_source() noexcept;
```

1 *Effects:* Initializes a new stop state inside *\*this*.

2 *Postconditions:* `stop_requested()` is false.

### 32.3.9.3 Member functions

[stopsource.inplace.mem]

```
constexpr inplace_stop_token get_token() const noexcept;
```

1 *Returns:* A new associated `inplace_stop_token` object whose *stop-source* member is equal to *this*.

```
bool stop_requested() const noexcept;
```

2 *Returns:* true if the stop state inside *\*this* has received a stop request; otherwise, false.

```
bool request_stop() noexcept;
```

3 *Effects:* Executes a stop request operation (32.3.3).

4 *Postconditions:* `stop_requested()` is true.

## 32.3.10 Class template `inplace_stop_callback`

[stopcallback.inplace]

### 32.3.10.1 General

[stopcallback.inplace.general]

```

namespace std {
 template<class CallbackFn>
 class inplace_stop_callback {
 public:
 using callback_type = CallbackFn;

 // 32.3.10.2, constructors and destructor
 template<class Initializer>
 explicit inplace_stop_callback(inplace_stop_token st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);
 ~inplace_stop_callback();

 inplace_stop_callback(inplace_stop_callback&&) = delete;
 inplace_stop_callback(const inplace_stop_callback&) = delete;
 inplace_stop_callback& operator=(inplace_stop_callback&&) = delete;
 inplace_stop_callback& operator=(const inplace_stop_callback&) = delete;

 private:
 CallbackFn callback_fn; // exposition only
 };
}

```

```

template<class CallbackFn>
 inplace_stop_callback(inplace_stop_token, CallbackFn)
 -> inplace_stop_callback<CallbackFn>;
}

```

<sup>1</sup> *Mandates:* CallbackFn satisfies both invocable and destructible.

<sup>2</sup> *Remarks:* For a type Initializer, if

```
stoppable-callback-for<CallbackFn, inplace_stop_token, Initializer>
```

is satisfied, then

```
stoppable-callback-for<CallbackFn, inplace_stop_token, Initializer>
```

is modeled. For an `inplace_stop_callback<CallbackFn>` object, the exposition-only *callback-fn* member is its associated callback function (32.3.3).

### 32.3.10.2 Constructors and destructor

[stopcallback.inplace.cons]

```

template<class Initializer>
 explicit inplace_stop_callback(inplace_stop_token st, Initializer&& init)
 noexcept(is_nothrow_constructible_v<CallbackFn, Initializer>);

```

<sup>1</sup> *Constraints:* `constructible_from<CallbackFn, Initializer>` is satisfied.

<sup>2</sup> *Effects:* Initializes *callback-fn* with `std::forward<Initializer>(init)` and executes a stoppable callback registration (32.3.3).

```
~inplace_stop_callback();
```

<sup>3</sup> *Effects:* Executes a stoppable callback deregistration (32.3.3).

## 32.4 Threads

[thread.threads]

### 32.4.1 General

[thread.threads.general]

<sup>1</sup> 32.4 describes components that can be used to create and manage threads.

[Note 1: These threads are intended to map one-to-one with operating system threads. — end note]

### 32.4.2 Header <thread> synopsis

[thread.syn]

```

#include <compare> // see 17.12.1

namespace std {
 // 32.4.3, class thread
 class thread;

 void swap(thread& x, thread& y) noexcept;

 // 32.4.4, class jthread
 class jthread;

 // 32.4.5, namespace this_thread
 namespace this_thread {
 thread::id get_id() noexcept;

 void yield() noexcept;
 template<class Clock, class Duration>
 void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
 template<class Rep, class Period>
 void sleep_for(const chrono::duration<Rep, Period>& rel_time);
 }
}

```

### 32.4.3 Class thread

[thread.thread.class]

#### 32.4.3.1 General

[thread.thread.class.general]

<sup>1</sup> The class `thread` provides a mechanism to create a new thread of execution, to join with a thread (i.e., wait for a thread to complete), and to perform other operations that manage and query the state of a thread. A

**thread** object uniquely represents a particular thread of execution. That representation may be transferred to other **thread** objects in such a way that no two **thread** objects simultaneously represent the same thread of execution. A thread of execution is *detached* when no **thread** object represents that thread. Objects of class **thread** can be in a state that does not represent a thread of execution.

[*Note 1*: A **thread** object does not represent a thread of execution after default construction, after being moved from, or after a successful call to **detach** or **join**. — *end note*]

```
namespace std {
 class thread {
 public:
 // 32.4.3.3, class thread::id
 class id;

 // 32.4.3.2, thread attributes
 template<same_as<char> T>
 class name_hint;
 template<class T>
 name_hint(const T*) -> name_hint<T>;
 template<class T>
 name_hint(basic_string<T>) -> name_hint<T>;
 class stack_size_hint;

 using native_handle_type = implementation-defined; // see 32.2.3

 // construct/copy/destroy
 thread() noexcept;
 template<class... Args>
 explicit thread(Args&&... args);
 ~thread();
 thread(const thread&) = delete;
 thread(thread&&) noexcept;
 thread& operator=(const thread&) = delete;
 thread& operator=(thread&&) noexcept;

 // 32.4.3.7, members
 void swap(thread&) noexcept;
 bool joinable() const noexcept;
 void join();
 void detach();
 id get_id() const noexcept;
 native_handle_type native_handle(); // see 32.2.3

 // static members
 static unsigned int hardware_concurrency() noexcept;
 };
}
```

### 32.4.3.2 Thread attributes

[thread.attributes]

#### 32.4.3.2.1 General

[thread.attributes.general]

- <sup>1</sup> *Thread attributes* can be used to define additional implementation-defined behaviors on a **thread** or **jthread** object.
- <sup>2</sup> The set of *thread attribute types* contains **thread::name\_hint**, **thread::stack\_size\_hint**, and an implementation-defined set of additional thread-attributes.
- <sup>3</sup> When an object of a thread attribute type is passed to a **thread** or **jthread** constructor, it may affect the new thread's behavior.

#### 32.4.3.2.2 Class **thread::name\_hint**

[thread.attributes.hint]

```
namespace std {
 template<same_as<char> T>
 class thread::name_hint {
 public:
 constexpr explicit name_hint(basic_string_view<T> n) noexcept;
```

```

 name_hint(name_hint&&) = delete;
 name_hint(const name_hint&) = delete;

private:
 basic_string_view<T> name; // exposition only
};
}

```

- <sup>1</sup> The `name_hint` thread attribute can be used to set the name of a thread for debugging purposes or platform-specific display mechanisms.

*Recommended practice:* Implementations should not store the value of the `name_hint` attribute in the `thread` or `jthread` object.

*Recommended practice:* `name` is interpreted as a string in the associated character encoding of `T` (Table 12).

```
constexpr explicit name_hint(basic_string_view<T> n) noexcept;
```

- <sup>2</sup> *Effects:* Initializes `name` with `n`.

#### 32.4.3.2.3 Class `thread::stack_size_hint` [thread.attributes.size]

```

namespace std {
 class thread::stack_size_hint {
 public:
 constexpr explicit stack_size_hint(size_t s) noexcept;

 private:
 size_t size; // exposition only
 };
}

```

- <sup>1</sup> `stack_size_hint` can be used to configure a desired size in bytes for platform-specific storage required for a thread.

[Note 1: Such storage is typically used for variables with automatic storage duration. — end note]

- <sup>2</sup> The stack size set by the implementation may be adjusted up or down to meet platform-specific requirements.
- <sup>3</sup> If `size` is zero, the thread attribute is ignored.

```
constexpr explicit stack_size_hint(size_t s) noexcept;
```

- <sup>4</sup> *Effects:* Initializes `size` with `s`.

#### 32.4.3.3 Class `thread::id` [thread.thread.id]

```

namespace std {
 class thread::id {
 public:
 id() noexcept;

 bool operator==(thread::id x, thread::id y) noexcept;
 strong_ordering operator<=>(thread::id x, thread::id y) noexcept;

 template<class charT, class traits>
 basic_ostream<charT, traits>&
 operator<<(basic_ostream<charT, traits>& out, thread::id id);

 template<class charT> struct formatter<thread::id, charT>;

 // hash support
 template<class T> struct hash;
 template<> struct hash<thread::id>;
 };
}

```

- <sup>1</sup> An object of type `thread::id` provides a unique identifier for each thread of execution and a single distinct value for all `thread` objects that do not represent a thread of execution (32.4.3). Each thread of execution has an associated `thread::id` object that is not equal to the `thread::id` object of any other thread of execution

and that is not equal to the `thread::id` object of any `thread` object that does not represent threads of execution.

2 The *text representation* for the character type `charT` of an object of type `thread::id` is an unspecified sequence of `charT` such that, for two objects of type `thread::id` `x` and `y`, if `x == y` is true, the `thread::id` objects have the same text representation, and if `x != y` is true, the `thread::id` objects have distinct text representations.

3 `thread::id` is a trivially copyable class (11.2). The library may reuse the value of a `thread::id` of a terminated thread that can no longer be joined.

4 [Note 1: Relational operators allow `thread::id` objects to be used as keys in associative containers. — end note]

```
id() noexcept;
```

5 *Postconditions:* The constructed object does not represent a thread of execution.

```
bool operator==(thread::id x, thread::id y) noexcept;
```

6 *Returns:* true only if `x` and `y` represent the same thread of execution or neither `x` nor `y` represents a thread of execution.

```
strong_ordering operator<=(thread::id x, thread::id y) noexcept;
```

7 Let  $P(x,y)$  be an unspecified total ordering over `thread::id` as described in 26.8.

8 *Returns:* `strong_ordering::less` if  $P(x,y)$  is true. Otherwise, `strong_ordering::greater` if  $P(y,x)$  is true. Otherwise, `strong_ordering::equal`.

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& out, thread::id id);
```

9 *Effects:* Inserts the text representation for `charT` of `id` into `out`.

10 *Returns:* `out`.

```
template<class charT> struct formatter<thread::id, charT>;
```

11 `formatter<thread::id, charT>` interprets *format-spec* as a *thread-id-format-spec*. The syntax of format specifications is as follows:

```
thread-id-format-spec:
fill-and-alignopt widthopt
```

[Note 2: The productions *fill-and-align* and *width* are described in 28.5.2.2. — end note]

12 If the *align* option is omitted it defaults to `>`.

13 A `thread::id` object is formatted by writing its text representation for `charT` to the output with additional padding and adjustments as specified by the format specifiers.

```
template<> struct hash<thread::id>;
```

14 The specialization is enabled (22.10.19).

#### 32.4.3.4 Constructors

[thread.thread.constr]

```
thread() noexcept;
```

1 *Effects:* The object does not represent a thread of execution.

2 *Postconditions:* `get_id() == id()`.

```
template<class... Args> explicit thread(Args&&... args);
```

3 *Constraints:*

(3.1) — `Args` is not an empty pack, and

(3.2) — `remove_cvref_t<Args...[0]>` is not the same type as `thread`.

4 Let  $i$  be the smallest value such that `decay_t<Args...[i]>` is not a thread attribute type, if such a value exists. Then the following notation is used:

(4.1) — Let  $F$  be `Args...[i]`.

- (4.2) — Let **f** be `args...[i]`.
- (4.3) — Let **Attrs** be a pack of the types `Args...[j]` for each  $j$  such that  $0 \leq j < i$ .
- (4.4) — Let **attrs** be a pack of the expressions `args...[j]` for each  $j$  such that  $0 \leq j < i$ .
- (4.5) — Let **FArgs** be a pack of the types `Args...[j]` for each  $j$  such that  $i < j < \text{sizeof}...(args)$ .
- (4.6) — Let **fargs** be a pack of the expressions `args...[j]` for each  $j$  such that  $i < j < \text{sizeof}...(args)$ .

5 *Mandates:* The value  $i$  exists, and the following are all **true**:

- (5.1) — `is_constructible_v<decay_t<F>, F>`,
- (5.2) — `(is_constructible_v<decay_t<FArgs>, FArgs> && ...)`,
- (5.3) — `is_invocable_v<decay_t<F>, decay_t<FArgs>...>`, and
- (5.4) — no type is present more than once in the pack `remove_cvref_t<Attrs>`.

6 *Effects:* The new thread of execution executes

```
 invoke(auto(std::forward<F>(f)), // for invoke, see 22.10.5
 auto(std::forward<FArgs>(fargs))...)
```

with the values produced by `auto` being materialized (7.3.5) in the constructing thread. Any return value from this invocation is ignored.

[*Note 1:* This implies that any exceptions not thrown from the invocation of the copy of **f** will be thrown in the constructing thread, not the new thread. — *end note*]

If the invocation of `invoke` terminates with an uncaught exception, `terminate` is invoked (14.6.2).

[*Note 2:* The parameters `attrs...` can affect the behavior of the new thread (32.4.3.2). — *end note*]

7 *Synchronization:* The completion of the invocation of the constructor synchronizes with the beginning of the invocation of the copy of **f**.

8 *Postconditions:* `get_id() != id()`. `*this` represents the newly started thread.

9 *Throws:* `system_error` if unable to start the new thread.

10 *Error conditions:*

- (10.1) — `resource_unavailable_try_again` — the system lacked the necessary resources to create another thread, or the system-imposed limit on the number of threads in a process would be exceeded.

```
thread(thread&& x) noexcept;
```

11 *Postconditions:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the start of construction.

### 32.4.3.5 Destructor

[`thread.thread.destr`]

```
~thread();
```

1 *Effects:* If `joinable()`, invokes `terminate` (14.6.2). Otherwise, has no effects.

[*Note 1:* Either implicitly detaching or joining a `joinable()` thread in its destructor can result in difficult to debug correctness (for detach) or performance (for join) bugs encountered only when an exception is thrown. These bugs can be avoided by ensuring that the destructor is never executed while the thread is still joinable. — *end note*]

### 32.4.3.6 Assignment

[`thread.thread.assign`]

```
thread& operator=(thread&& x) noexcept;
```

1 *Effects:* If `joinable()`, invokes `terminate` (14.6.2). Otherwise, assigns the state of `x` to `*this` and sets `x` to a default constructed state.

2 *Postconditions:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the assignment.

3 *Returns:* `*this`.



**32.4.3.7 Members****[thread.thread.member]**

void swap(thread& x) noexcept;

1     *Effects:* Swaps the state of *\*this* and *x*.

bool joinable() const noexcept;

2     *Returns:* get\_id() != id().

void join();

3     *Effects:* Blocks until the thread represented by *\*this* has completed.

4     *Synchronization:* The completion of the thread represented by *\*this* synchronizes with (6.10.2) the corresponding successful join() return.

[Note 1: Operations on *\*this* are not synchronized. — end note]

5     *Postconditions:* The thread represented by *\*this* has completed. get\_id() == id().

6     *Throws:* system\_error when an exception is required (32.2.2).

7     *Error conditions:*

(7.1)     — resource\_deadlock\_would\_occur — if deadlock is detected or get\_id() == this\_thread::get\_id().

(7.2)     — no\_such\_process — if the thread is not valid.

(7.3)     — invalid\_argument — if the thread is not joinable.

void detach();

8     *Effects:* The thread represented by *\*this* continues execution without the calling thread blocking. When detach() returns, *\*this* no longer represents the possibly continuing thread of execution. When the thread previously represented by *\*this* ends execution, the implementation releases any owned resources.

9     *Postconditions:* get\_id() == id().

10     *Throws:* system\_error when an exception is required (32.2.2).

11     *Error conditions:*

(11.1)     — no\_such\_process — if the thread is not valid.

(11.2)     — invalid\_argument — if the thread is not joinable.

id get\_id() const noexcept;

12     *Returns:* A default constructed id object if *\*this* does not represent a thread, otherwise this\_thread::get\_id() for the thread of execution represented by *\*this*.

**32.4.3.8 Static members****[thread.thread.static]**

unsigned hardware\_concurrency() noexcept;

1     *Returns:* The number of hardware thread contexts.

[Note 1: This value should only be considered to be a hint. — end note]

If this value is not computable or well-defined, an implementation should return 0.

**32.4.3.9 Specialized algorithms****[thread.thread.algorithm]**

void swap(thread& x, thread& y) noexcept;

1     *Effects:* As if by x.swap(y).

**32.4.4 Class jthread****[thread.jthread.class]****32.4.4.1 General****[thread.jthread.class.general]**

1 The class jthread provides a mechanism to create a new thread of execution. The functionality is the same as for class thread (32.4.3) with the additional abilities to provide a stop\_token (32.3) to the new thread of execution, make stop requests, and automatically join.

```

namespace std {
 class jthread {
 public:
 // types
 using id = thread::id;
 using native_handle_type = thread::native_handle_type;

 template<class T>
 using name_hint = thread::name_hint<T>;
 using stack_size_hint = thread::stack_size_hint;

 // 32.4.4.2, constructors, move, and assignment
 jthread() noexcept;
 template<class... Args>
 explicit jthread(Args&&... args);
 ~jthread();
 jthread(const jthread&) = delete;
 jthread(jthread&&) noexcept;
 jthread& operator=(const jthread&) = delete;
 jthread& operator=(jthread&&) noexcept;

 // 32.4.4.3, members
 void swap(jthread&) noexcept;
 bool joinable() const noexcept;
 void join();
 void detach();
 id get_id() const noexcept;
 native_handle_type native_handle(); // see 32.2.3

 // 32.4.4.4, stop token handling
 stop_source get_stop_source() noexcept;
 stop_token get_stop_token() const noexcept;
 bool request_stop() noexcept;

 // 32.4.4.5, specialized algorithms
 friend void swap(jthread& lhs, jthread& rhs) noexcept;

 // 32.4.4.6, static members
 static unsigned int hardware_concurrency() noexcept;

 private:
 stop_source ssource; // exposition only
 };
}

```

#### 32.4.4.2 Constructors, move, and assignment

[thread.jthread.cons]

jthread() noexcept;

1 *Effects:* Constructs a jthread object that does not represent a thread of execution.

2 *Postconditions:* get\_id() == id() is true and ssource.stop\_possible() is false.

template<class... Args> explicit jthread(Args&&... args);

3 *Constraints:*

(3.1) — Args is not an empty pack, and

(3.2) — remove\_cvref\_t<Args...[0]> is not the same type as jthread.

4 Let  $i$  be the smallest value such that decay\_t<Args...[ $i$ ]> is not a thread attribute type, if such a value exists. Then the following notation is used:

(4.1) — Let  $F$  be Args...[ $i$ ].

(4.2) — Let  $f$  be args...[ $i$ ].

(4.3) — Let Attrs be a pack of the types Args...[ $j$ ] for each  $j$  such that  $0 \leq j < i$ .

- (4.4) — Let `attrs` be a pack of the expressions `args...[j]` for each  $j$  such that  $0 \leq j < i$ .
- (4.5) — Let `FArgs` be a pack of the types `Args...[j]` for each  $j$  such that  $i < j < \text{sizeof}...(args)$ .
- (4.6) — Let `fargs` be a pack of the expressions `args...[j]` for each  $j$  such that  $i < j < \text{sizeof}...(args)$ .

5 *Mandates:* The value  $i$  exists, and the following are all true:

- (5.1) — `is_constructible_v<decay_t<F>, F>`,
- (5.2) — `(is_constructible_v<decay_t<FArgs>, FArgs> && ...)`,
- (5.3) — `is_invocable_v<decay_t<F>, decay_t<FArgs>...> || is_invocable_v<decay_t<F>, stop_token, decay_t<FArgs>...>`, and
- (5.4) — no type is present more than once in the pack `remove_cvref_t<Attrs>`.

6 *Effects:* Initializes `ssource`. The new thread of execution executes

```
invoke(auto(std::forward<F>(f)), get_stop_token(), // for invoke, see 22.10.5
auto(std::forward<FArgs>(fargs))...)
```

if that expression is well-formed, otherwise

```
invoke(auto(std::forward<F>(f)), auto(std::forward<FArgs>(fargs))...)
```

with the values produced by `auto` being materialized (7.3.5) in the constructing thread. Any return value from this invocation is ignored.

[Note 1: This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. — end note]

If the `invoke` expression exits via an exception, `terminate` is called.

[Note 2: The parameters `attrs...` can affect the behavior of the new thread (32.4.3.2). — end note]

7 *Synchronization:* The completion of the invocation of the constructor synchronizes with the beginning of the invocation of the copy of `f`.

8 *Postconditions:* `get_id() != id()` is true and `ssource.stop_possible()` is true and `*this` represents the newly started thread.

[Note 3: The calling thread can make a stop request only once, because it cannot replace this stop token. — end note]

9 *Throws:* `system_error` if unable to start the new thread.

10 *Error conditions:*

- (10.1) — `resource_unavailable_try_again` — the system lacked the necessary resources to create another thread, or the system-imposed limit on the number of threads in a process would be exceeded.

```
jthread(jthread&& x) noexcept;
```

11 *Postconditions:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the start of construction. `ssource` has the value of `x.ssource` prior to the start of construction and `x.ssource.stop_possible()` is false.

```
~jthread();
```

12 *Effects:* If `joinable()` is true, calls `request_stop()` and then `join()`.

[Note 4: Operations on `*this` are not synchronized. — end note]

```
jthread& operator=(jthread&& x) noexcept;
```

13 *Effects:* If `&x == this` is true, there are no effects. Otherwise, if `joinable()` is true, calls `request_stop()` and then `join()`, then assigns the state of `x` to `*this` and sets `x` to a default constructed state.

14 *Postconditions:* `get_id()` returns the value of `x.get_id()` prior to the assignment. `ssource` has the value of `x.ssource` prior to the assignment.

15 *Returns:* `*this`.

**32.4.4.3 Members**

[thread.jthread.mem]

```
void swap(jthread& x) noexcept;
```

1     *Effects:* Exchanges the values of *\*this* and *x*.

```
bool joinable() const noexcept;
```

2     *Returns:* `get_id() != id()`.

```
void join();
```

3     *Effects:* Blocks until the thread represented by *\*this* has completed.

4     *Synchronization:* The completion of the thread represented by *\*this* synchronizes with (6.10.2) the corresponding successful `join()` return.

[Note 1: Operations on *\*this* are not synchronized. — end note]

5     *Postconditions:* The thread represented by *\*this* has completed. `get_id() == id()`.

6     *Throws:* `system_error` when an exception is required (32.2.2).

7     *Error conditions:*

(7.1)     — `resource_deadlock_would_occur` — if deadlock is detected or `get_id() == this_thread::get_id()`.

(7.2)     — `no_such_process` — if the thread is not valid.

(7.3)     — `invalid_argument` — if the thread is not joinable.

```
void detach();
```

8     *Effects:* The thread represented by *\*this* continues execution without the calling thread blocking. When `detach()` returns, *\*this* no longer represents the possibly continuing thread of execution. When the thread previously represented by *\*this* ends execution, the implementation releases any owned resources.

9     *Postconditions:* `get_id() == id()`.

10     *Throws:* `system_error` when an exception is required (32.2.2).

11     *Error conditions:*

(11.1)     — `no_such_process` — if the thread is not valid.

(11.2)     — `invalid_argument` — if the thread is not joinable.

```
id get_id() const noexcept;
```

12     *Returns:* A default constructed `id` object if *\*this* does not represent a thread, otherwise `this_thread::get_id()` for the thread of execution represented by *\*this*.

**32.4.4.4 Stop token handling**

[thread.jthread.stop]

```
stop_source get_stop_source() noexcept;
```

1     *Effects:* Equivalent to: `return ssource;`

```
stop_token get_stop_token() const noexcept;
```

2     *Effects:* Equivalent to: `return ssource.get_token();`

```
bool request_stop() noexcept;
```

3     *Effects:* Equivalent to: `return ssource.request_stop();`

**32.4.4.5 Specialized algorithms**

[thread.jthread.special]

```
friend void swap(jthread& x, jthread& y) noexcept;
```

1     *Effects:* Equivalent to: `x.swap(y)`.

**32.4.4.6 Static members**

[thread.jthread.static]

```
static unsigned int hardware_concurrency() noexcept;
```

1 *Returns:* `thread::hardware_concurrency()`.

**32.4.5 Namespace `this_thread`**

[thread.thread.this]

```
namespace std::this_thread {
 thread::id get_id() noexcept;

 void yield() noexcept;
 template<class Clock, class Duration>
 void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
 template<class Rep, class Period>
 void sleep_for(const chrono::duration<Rep, Period>& rel_time);
}
```

```
thread::id this_thread::get_id() noexcept;
```

1 *Returns:* An object of type `thread::id` that uniquely identifies the current thread of execution. Every invocation from this thread of execution returns the same value. The object returned does not compare equal to a default-constructed `thread::id`.

```
void this_thread::yield() noexcept;
```

2 *Effects:* Offers the implementation the opportunity to reschedule.

3 *Synchronization:* None.

```
template<class Clock, class Duration>
 void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
```

4 *Effects:* Blocks the calling thread for the absolute timeout (32.2.4) specified by `abs_time`.

5 *Synchronization:* None.

6 *Throws:* Timeout-related exceptions (32.2.4).

```
template<class Rep, class Period>
 void sleep_for(const chrono::duration<Rep, Period>& rel_time);
```

7 *Effects:* Blocks the calling thread for the relative timeout (32.2.4) specified by `rel_time`.

8 *Synchronization:* None.

9 *Throws:* Timeout-related exceptions (32.2.4).

**32.5 Atomic operations**

[atomics]

**32.5.1 General**

[atomics.general]

1 Let *COPYCV*(FROM, TO) be an alias for type TO with the addition of FROM's top-level cv-qualifiers.

2 Subclause 32.5 describes components for fine-grained atomic access. This access is provided via operations on atomic objects.

**32.5.2 Header `<atomic>` synopsis**

[atomics.syn]

```
// mostly freestanding
namespace std {
 // 32.5.4, order and consistency
 enum class memory_order : unspecified;
 inline constexpr memory_order memory_order_relaxed = memory_order::relaxed;
 inline constexpr memory_order memory_order_acquire = memory_order::acquire;
 inline constexpr memory_order memory_order_release = memory_order::release;
 inline constexpr memory_order memory_order_acq_rel = memory_order::acq_rel;
 inline constexpr memory_order memory_order_seq_cst = memory_order::seq_cst;
}
```

```

// 32.5.5, lock-free property
#define ATOMIC_BOOL_LOCK_FREE unspecified
#define ATOMIC_CHAR_LOCK_FREE unspecified
#define ATOMIC_CHAR8_T_LOCK_FREE unspecified
#define ATOMIC_CHAR16_T_LOCK_FREE unspecified
#define ATOMIC_CHAR32_T_LOCK_FREE unspecified
#define ATOMIC_WCHAR_T_LOCK_FREE unspecified
#define ATOMIC_SHORT_LOCK_FREE unspecified
#define ATOMIC_INT_LOCK_FREE unspecified
#define ATOMIC_LONG_LOCK_FREE unspecified
#define ATOMIC_LLONG_LOCK_FREE unspecified
#define ATOMIC_POINTER_LOCK_FREE unspecified

namespace std {
 // 32.5.7, class template atomic_ref
 template<class T> struct atomic_ref;

 // 32.5.8, class template atomic
 template<class T> struct atomic;
 // 32.5.8.5, partial specialization for pointers
 template<class T> struct atomic<T*>;

 // 32.5.9, non-member functions
 template<class T>
 bool atomic_is_lock_free(const volatile atomic<T*>*) noexcept;
 template<class T>
 bool atomic_is_lock_free(const atomic<T*>*) noexcept;
 template<class T>
 void atomic_store(volatile atomic<T*>*, typename atomic<T*>::value_type) noexcept;
 template<class T>
 constexpr void atomic_store(atomic<T*>*, typename atomic<T*>::value_type) noexcept;
 template<class T>
 void atomic_store_explicit(volatile atomic<T*>*, typename atomic<T*>::value_type,
 memory_order) noexcept;
 template<class T>
 constexpr void atomic_store_explicit(atomic<T*>*, typename atomic<T*>::value_type,
 memory_order) noexcept;

 template<class T>
 T atomic_load(const volatile atomic<T*>*) noexcept;
 template<class T>
 constexpr T atomic_load(const atomic<T*>*) noexcept;
 template<class T>
 T atomic_load_explicit(const volatile atomic<T*>*, memory_order) noexcept;
 template<class T>
 constexpr T atomic_load_explicit(const atomic<T*>*, memory_order) noexcept;
 template<class T>
 T atomic_exchange(volatile atomic<T*>*, typename atomic<T*>::value_type) noexcept;
 template<class T>
 constexpr T atomic_exchange(atomic<T*>*, typename atomic<T*>::value_type) noexcept;
 template<class T>
 T atomic_exchange_explicit(volatile atomic<T*>*, typename atomic<T*>::value_type,
 memory_order) noexcept;
 template<class T>
 constexpr T atomic_exchange_explicit(atomic<T*>*, typename atomic<T*>::value_type,
 memory_order) noexcept;

 template<class T>
 bool atomic_compare_exchange_weak(volatile atomic<T*>*,
 typename atomic<T*>::value_type*,
 typename atomic<T*>::value_type) noexcept;

 template<class T>
 constexpr bool atomic_compare_exchange_weak(atomic<T*>*,
 typename atomic<T*>::value_type*,
 typename atomic<T*>::value_type) noexcept;

```

```

template<class T>
 bool atomic_compare_exchange_strong(volatile atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type) noexcept;

template<class T>
 constexpr bool atomic_compare_exchange_strong(atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type) noexcept;

template<class T>
 bool atomic_compare_exchange_weak_explicit(volatile atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type,
 memory_order, memory_order) noexcept;

template<class T>
 constexpr bool atomic_compare_exchange_weak_explicit(atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type,
 memory_order, memory_order) noexcept;

template<class T>
 bool atomic_compare_exchange_strong_explicit(volatile atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type,
 memory_order, memory_order) noexcept;

template<class T>
 constexpr bool atomic_compare_exchange_strong_explicit(atomic<T>*,
 typename atomic<T>::value_type*,
 typename atomic<T>::value_type,
 memory_order, memory_order) noexcept;

template<class T>
 T atomic_fetch_add(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 constexpr T atomic_fetch_add(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 T atomic_fetch_add_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_add_explicit(atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;

template<class T>
 T atomic_fetch_sub(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 constexpr T atomic_fetch_sub(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 T atomic_fetch_sub_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_sub_explicit(atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;

template<class T>
 T atomic_fetch_and(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr T atomic_fetch_and(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 T atomic_fetch_and_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_and_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 T atomic_fetch_or(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr T atomic_fetch_or(atomic<T>*, typename atomic<T>::value_type) noexcept;

```

```

template<class T>
 T atomic_fetch_or_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_or_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 T atomic_fetch_xor(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr T atomic_fetch_xor(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 T atomic_fetch_xor_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_xor_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 T atomic_fetch_max(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr T atomic_fetch_max(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 T atomic_fetch_max_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_max_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 T atomic_fetch_min(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr T atomic_fetch_min(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 T atomic_fetch_min_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;
template<class T>
 constexpr T atomic_fetch_min_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_store_add(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 constexpr void atomic_store_add(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_store_add_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 constexpr void atomic_store_add_explicit(atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 void atomic_store_sub(volatile atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 constexpr void atomic_store_sub(atomic<T>*, typename atomic<T>::difference_type) noexcept;
template<class T>
 void atomic_store_sub_explicit(volatile atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 constexpr void atomic_store_sub_explicit(atomic<T>*, typename atomic<T>::difference_type,
 memory_order) noexcept;
template<class T>
 void atomic_store_and(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_store_and(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_store_and_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

```



```

template<class T>
 constexpr void atomic_store_and_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_store_or(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_store_or(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_store_or_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 constexpr void atomic_store_or_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_store_xor(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_store_xor(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_store_xor_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 constexpr void atomic_store_xor_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_store_max(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_store_max(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_store_max_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 constexpr void atomic_store_max_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_store_min(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_store_min(atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_store_min_explicit(volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 constexpr void atomic_store_min_explicit(atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_wait(const volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 constexpr void atomic_wait(const atomic<T>*, typename atomic<T>::value_type) noexcept;
template<class T>
 void atomic_wait_explicit(const volatile atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 constexpr void atomic_wait_explicit(const atomic<T>*, typename atomic<T>::value_type,
 memory_order) noexcept;

template<class T>
 void atomic_notify_one(volatile atomic<T>*) noexcept;
template<class T>
 constexpr void atomic_notify_one(atomic<T>*) noexcept;
template<class T>
 void atomic_notify_all(volatile atomic<T>*) noexcept;
template<class T>
 constexpr void atomic_notify_all(atomic<T>*) noexcept;

```

// 32.5.3, type aliases

```
using atomic_bool = atomic<bool>;
using atomic_char = atomic<char>;
using atomic_schar = atomic<signed char>;
using atomic_uchar = atomic<unsigned char>;
using atomic_short = atomic<short>;
using atomic_ushort = atomic<unsigned short>;
using atomic_int = atomic<int>;
using atomic_uint = atomic<unsigned int>;
using atomic_long = atomic<long>;
using atomic_ulong = atomic<unsigned long>;
using atomic_llong = atomic<long long>;
using atomic_ullong = atomic<unsigned long long>;
using atomic_char8_t = atomic<char8_t>;
using atomic_char16_t = atomic<char16_t>;
using atomic_char32_t = atomic<char32_t>;
using atomic_wchar_t = atomic<wchar_t>;
```

```
using atomic_int8_t = atomic<int8_t>;
using atomic_uint8_t = atomic<uint8_t>;
using atomic_int16_t = atomic<int16_t>;
using atomic_uint16_t = atomic<uint16_t>;
using atomic_int32_t = atomic<int32_t>;
using atomic_uint32_t = atomic<uint32_t>;
using atomic_int64_t = atomic<int64_t>;
using atomic_uint64_t = atomic<uint64_t>;
```

```
using atomic_int_least8_t = atomic<int_least8_t>;
using atomic_uint_least8_t = atomic<uint_least8_t>;
using atomic_int_least16_t = atomic<int_least16_t>;
using atomic_uint_least16_t = atomic<uint_least16_t>;
using atomic_int_least32_t = atomic<int_least32_t>;
using atomic_uint_least32_t = atomic<uint_least32_t>;
using atomic_int_least64_t = atomic<int_least64_t>;
using atomic_uint_least64_t = atomic<uint_least64_t>;
```

```
using atomic_int_fast8_t = atomic<int_fast8_t>;
using atomic_uint_fast8_t = atomic<uint_fast8_t>;
using atomic_int_fast16_t = atomic<int_fast16_t>;
using atomic_uint_fast16_t = atomic<uint_fast16_t>;
using atomic_int_fast32_t = atomic<int_fast32_t>;
using atomic_uint_fast32_t = atomic<uint_fast32_t>;
using atomic_int_fast64_t = atomic<int_fast64_t>;
using atomic_uint_fast64_t = atomic<uint_fast64_t>;
```

```
using atomic_intptr_t = atomic<intptr_t>;
using atomic_uintptr_t = atomic<uintptr_t>;
using atomic_size_t = atomic<size_t>;
using atomic_ptrdiff_t = atomic<ptrdiff_t>;
using atomic_intmax_t = atomic<intmax_t>;
using atomic_uintmax_t = atomic<uintmax_t>;
```

```
using atomic_signed_lock_free = see below; // hosted
using atomic_unsigned_lock_free = see below; // hosted
```

// 32.5.10, flag type and operations

```
struct atomic_flag;
```

```
bool atomic_flag_test(const volatile atomic_flag*) noexcept;
constexpr bool atomic_flag_test(const atomic_flag*) noexcept;
bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_order) noexcept;
constexpr bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcept;
bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
constexpr bool atomic_flag_test_and_set(atomic_flag*) noexcept;
```

```

bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
constexpr bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
void atomic_flag_clear(volatile atomic_flag*) noexcept;
constexpr void atomic_flag_clear(atomic_flag*) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
constexpr void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;

void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
constexpr void atomic_flag_wait(const atomic_flag*, bool) noexcept;
void atomic_flag_wait_explicit(const volatile atomic_flag*, bool, memory_order) noexcept;
constexpr void atomic_flag_wait_explicit(const atomic_flag*, bool, memory_order) noexcept;
void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
constexpr void atomic_flag_notify_one(atomic_flag*) noexcept;
void atomic_flag_notify_all(volatile atomic_flag*) noexcept;
constexpr void atomic_flag_notify_all(atomic_flag*) noexcept;
#define ATOMIC_FLAG_INIT see below

// 32.5.11, fences
extern "C" constexpr void atomic_thread_fence(memory_order) noexcept;
extern "C" constexpr void atomic_signal_fence(memory_order) noexcept;
}

```

### 32.5.3 Type aliases

[atomics.alias]

- <sup>1</sup> The type aliases `atomic_intN_t` and `atomic_uintN_t` are declared if and only if `intN_t` and `uintN_t` are declared, respectively.
- <sup>2</sup> The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` name specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, and whose `is_always_lock_free` property is true.

[Note 1: These aliases are optional in freestanding implementations (16.4.2.5). — end note]

Implementations should choose for these aliases the integral specializations of `atomic` for which the atomic waiting and notifying operations (32.5.6) are most efficient.

### 32.5.4 Order and consistency

[atomics.order]

```

namespace std {
 enum class memory_order : unspecified {
 relaxed = 0, acquire = 2, release = 3, acq_rel = 4, seq_cst = 5
 };
}

```

- <sup>1</sup> The enumeration `memory_order` specifies the detailed regular (non-atomic) memory synchronization order as defined in 6.10.2 and may provide for operation ordering. Its enumerated values and their meanings are as follows:
  - (1.1) — `memory_order::relaxed`: no operation orders memory.
  - (1.2) — `memory_order::release`, `memory_order::acq_rel`, and `memory_order::seq_cst`: a store operation performs a release operation on the affected memory location.
  - (1.3) — `memory_order::acquire`, `memory_order::acq_rel`, and `memory_order::seq_cst`: a load operation performs an acquire operation on the affected memory location.

[Note 1: Atomic operations specifying `memory_order::relaxed` are relaxed with respect to memory ordering. Implementations must still guarantee that any given atomic access to a particular atomic object be indivisible with respect to all other atomic accesses to that object. — end note]

- <sup>2</sup> An atomic operation *A* that performs a release operation on an atomic object *M* synchronizes with an atomic operation *B* that performs an acquire operation on *M* and takes its value from any side effect in the release sequence headed by *A*.
- <sup>3</sup> An atomic operation *A* on some atomic object *M* is *coherence-ordered before* another atomic operation *B* on *M* if
  - (3.1) — *A* is a modification, and *B* reads the value stored by *A*, or
  - (3.2) — *A* precedes *B* in the modification order of *M*, or

- (3.3) —  $A$  and  $B$  are not the same atomic read-modify-write operation, and there exists an atomic modification  $X$  of  $M$  such that  $A$  reads the value stored by  $X$  and  $X$  precedes  $B$  in the modification order of  $M$ , or
- (3.4) — there exists an atomic modification  $X$  of  $M$  such that  $A$  is coherence-ordered before  $X$  and  $X$  is coherence-ordered before  $B$ .
- 4 There is a single total order  $S$  on all `memory_order::seq_cst` operations, including fences, that satisfies the following constraints. First, if  $A$  and  $B$  are `memory_order::seq_cst` operations and  $A$  strongly happens before  $B$ , then  $A$  precedes  $B$  in  $S$ . Second, for every pair of atomic operations  $A$  and  $B$  on an object  $M$ , where  $A$  is coherence-ordered before  $B$ , the following four conditions are required to be satisfied by  $S$ :
- (4.1) — if  $A$  and  $B$  are both `memory_order::seq_cst` operations, then  $A$  precedes  $B$  in  $S$ ; and
- (4.2) — if  $A$  is a `memory_order::seq_cst` operation and  $B$  happens before a `memory_order::seq_cst` fence  $Y$ , then  $A$  precedes  $Y$  in  $S$ ; and
- (4.3) — if a `memory_order::seq_cst` fence  $X$  happens before  $A$  and  $B$  is a `memory_order::seq_cst` operation, then  $X$  precedes  $B$  in  $S$ ; and
- (4.4) — if a `memory_order::seq_cst` fence  $X$  happens before  $A$  and  $B$  happens before a `memory_order::seq_cst` fence  $Y$ , then  $X$  precedes  $Y$  in  $S$ .
- 5 [Note 2: This definition ensures that  $S$  is consistent with the modification order of any atomic object  $M$ . It also ensures that a `memory_order::seq_cst` load  $A$  of  $M$  gets its value either from the last modification of  $M$  that precedes  $A$  in  $S$  or from some non-`memory_order::seq_cst` modification of  $M$  that does not happen before any modification of  $M$  that precedes  $A$  in  $S$ . — end note]
- 6 [Note 3: We do not require that  $S$  be consistent with “happens before” (6.10.2.2). This allows more efficient implementation of `memory_order::acquire` and `memory_order::release` on some machine architectures. It can produce surprising results when these are mixed with `memory_order::seq_cst` accesses. — end note]
- 7 [Note 4: `memory_order::seq_cst` ensures sequential consistency only for a program that is free of data races and uses exclusively `memory_order::seq_cst` atomic operations. Any use of weaker ordering will invalidate this guarantee unless extreme care is used. In many cases, `memory_order::seq_cst` atomic operations are reorderable with respect to other atomic operations performed by the same thread. — end note]
- 8 Implementations should ensure that no “out-of-thin-air” values are computed that circularly depend on their own computation.

[Note 5: For example, with  $x$  and  $y$  initially zero,

```
// Thread 1:
r1 = y.load(memory_order::relaxed);
x.store(r1, memory_order::relaxed);

// Thread 2:
r2 = x.load(memory_order::relaxed);
y.store(r2, memory_order::relaxed);
```

this recommendation discourages producing `r1 == r2 == 42`, since the store of 42 to  $y$  is only possible if the store to  $x$  stores 42, which circularly depends on the store to  $y$  storing 42. Note that without this restriction, such an execution is possible. — end note]

- 9 [Note 6: The recommendation similarly disallows `r1 == r2 == 42` in the following example, with  $x$  and  $y$  again initially zero:

```
// Thread 1:
r1 = x.load(memory_order::relaxed);
if (r1 == 42) y.store(42, memory_order::relaxed);

// Thread 2:
r2 = y.load(memory_order::relaxed);
if (r2 == 42) x.store(42, memory_order::relaxed);
```

— end note]

[Note 7: An implementation that translates to instruction sequences for a physical machine and treats non-volatile atomic accesses as though they were volatile does not produce out-of-thin-air values for a program free of undefined behavior. An implementation that performs only thread-at-a-time analysis in support of optimizations that omit and merge non-volatile atomic accesses to the same object and reorder non-volatile atomic accesses to different objects (when permitted by the as-if rule) and otherwise treats non-volatile atomic accesses as though they were volatile does not produce out-of-thin-air values for a program free of undefined behavior. The possibility of out-of-thin-air values remains for implementations not meeting these restrictions. — end note]

- 10 [Note 8: Unlike abstract machines, physical machines are subject to instruction-execution and information-transfer latencies. Physical machines carefully manage any speculative execution so as to avoid generating out-of-thin-air values at the hardware level. — end note]
- 11 An atomic read-modify-write operation always reads the value from the side effect that immediately precedes the side effect of the read-modify-write operation in the modification order.
- 12 An *atomic modify-write operation* is an atomic read-modify-write operation with weaker synchronization requirements as specified in 32.5.11.
- [Note 9: The intent is for atomic modify-write operations to be implemented using mechanisms that are not ordered, in hardware, by the implementation of acquire fences. No other semantic or hardware property (e.g., that the mechanism is a far atomic operation) is implied. — end note]
- 13 *Recommended practice:* The implementation should make atomic stores visible to atomic loads, and atomic loads should observe atomic stores, within a reasonable amount of time.

### 32.5.5 Lock-free property

[atomics.lockfree]

```
#define ATOMIC_BOOL_LOCK_FREE unspecified
#define ATOMIC_CHAR_LOCK_FREE unspecified
#define ATOMIC_CHAR8_T_LOCK_FREE unspecified
#define ATOMIC_CHAR16_T_LOCK_FREE unspecified
#define ATOMIC_CHAR32_T_LOCK_FREE unspecified
#define ATOMIC_WCHAR_T_LOCK_FREE unspecified
#define ATOMIC_SHORT_LOCK_FREE unspecified
#define ATOMIC_INT_LOCK_FREE unspecified
#define ATOMIC_LONG_LOCK_FREE unspecified
#define ATOMIC_LLONG_LOCK_FREE unspecified
#define ATOMIC_POINTER_LOCK_FREE unspecified
```

- 1 The ATOMIC\_...\_LOCK\_FREE macros indicate the lock-free property of the corresponding atomic types, with the signed and unsigned variants grouped together. The properties also apply to the corresponding (partial) specializations of the `atomic` template. A value of 0 indicates that the types are never lock-free. A value of 1 indicates that the types are sometimes lock-free. A value of 2 indicates that the types are always lock-free.
- 2 On a hosted implementation (16.4.2.5), at least one signed integral specialization of the `atomic` template, along with the specialization for the corresponding unsigned type (6.9.3), is always lock-free.
- 3 The functions `atomic<T>::is_lock_free` and `atomic_is_lock_free` (32.5.8.2) indicate whether the object is lock-free. In any given program execution, the result of the lock-free query is the same for all atomic objects of the same type.
- 4 Atomic operations that are not lock-free are considered to potentially block (6.10.2.3).
- 5 *Recommended practice:* Operations that are lock-free should also be address-free.<sup>273</sup> The implementation of these operations should not depend on any per-process state.

[Note 1: This restriction enables communication by memory that is mapped into a process more than once and by memory that is shared between two processes. — end note]

### 32.5.6 Waiting and notifying

[atomics.wait]

- 1 *Atomic waiting operations* and *atomic notifying operations* provide a mechanism to wait for the value of an atomic object to change more efficiently than can be achieved with polling. An atomic waiting operation may block until it is unblocked by an atomic notifying operation, according to each function's effects.

[Note 1: Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. — end note]

- 2 [Note 2: The following functions are atomic waiting operations:

- (2.1) — `atomic<T>::wait`,
- (2.2) — `atomic_flag::wait`,
- (2.3) — `atomic_wait` and `atomic_wait_explicit`,
- (2.4) — `atomic_flag_wait` and `atomic_flag_wait_explicit`, and
- (2.5) — `atomic_ref<T>::wait`.

— end note]

<sup>273</sup>) That is, atomic operations on the same memory location via two different addresses will communicate atomically.

<sup>3</sup> [Note 3: The following functions are atomic notifying operations:

- (3.1) — `atomic<T>::notify_one` and `atomic<T>::notify_all`,
- (3.2) — `atomic_flag::notify_one` and `atomic_flag::notify_all`,
- (3.3) — `atomic_notify_one` and `atomic_notify_all`,
- (3.4) — `atomic_flag_notify_one` and `atomic_flag_notify_all`, and
- (3.5) — `atomic_ref<T>::notify_one` and `atomic_ref<T>::notify_all`.

— *end note*]

<sup>4</sup> A call to an atomic waiting operation on an atomic object M is *eligible to be unblocked* by a call to an atomic notifying operation on M if there exist side effects X and Y on M such that:

- (4.1) — the atomic waiting operation has blocked after observing the result of X,
- (4.2) — X precedes Y in the modification order of M, and
- (4.3) — Y happens before the call to the atomic notifying operation.

### 32.5.7 Class template `atomic_ref`

[atomics.ref.generic]

#### 32.5.7.1 General

[atomics.ref.generic.general]

```
namespace std {
 template<class T> struct atomic_ref {
 private:
 T* ptr; // exposition only

 public:
 using value_type = remove_cv_t<T>;
 using address-return-type = COPYCV(T, void)*; // exposition only
 static constexpr size_t required_alignment = implementation-defined;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr explicit atomic_ref(T&);
 explicit atomic_ref(T&&) = delete;
 constexpr atomic_ref(const atomic_ref&) noexcept;
 template<class U>
 constexpr atomic_ref(const atomic_ref<U>&) noexcept;
 atomic_ref& operator=(const atomic_ref&) = delete;

 constexpr void store(value_type, memory_order = memory_order::seq_cst) const noexcept;
 constexpr value_type operator=(value_type) const noexcept;
 constexpr value_type load(memory_order = memory_order::seq_cst) const noexcept;
 constexpr operator value_type() const noexcept;

 constexpr value_type exchange(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
 constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order, memory_order) const noexcept;
 constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order, memory_order) const noexcept;
 constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;
 constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;

 constexpr void wait(value_type, memory_order = memory_order::seq_cst) const noexcept;
 constexpr void notify_one() const noexcept;
 constexpr void notify_all() const noexcept;
 constexpr address-return-type address() const noexcept;
 };
}
```

- <sup>1</sup> An `atomic_ref` object applies atomic operations (32.5.1) to the object referenced by `*ptr` such that, for the lifetime (6.8.4) of the `atomic_ref` object, the object referenced by `*ptr` is an atomic object (6.10.2.2).
- <sup>2</sup> The program is ill-formed if `is_trivially_copyable_v<T>` is false.
- <sup>3</sup> The lifetime (6.8.4) of an object referenced by `*ptr` shall exceed the lifetime of all `atomic_refs` that reference the object. While any `atomic_ref` instances exist that reference the `*ptr` object, all accesses to that object shall exclusively occur through those `atomic_ref` instances. No subobject of the object referenced by `atomic_ref` shall be concurrently referenced by any other `atomic_ref` object.
- <sup>4</sup> Atomic operations applied to an object through a referencing `atomic_ref` are atomic with respect to atomic operations applied through any other `atomic_ref` referencing the same object.  
*[Note 1: Atomic operations or the `atomic_ref` constructor can acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations to be applied to the referenced object. — end note]*
- <sup>5</sup> The program is ill-formed if `is_always_lock_free` is false and `is_volatile_v<T>` is true.

### 32.5.7.2 Operations

[atomics.ref.ops]

```
static constexpr size_t required_alignment;
```

- <sup>1</sup> The alignment required for an object to be referenced by an atomic reference, which is at least `alignof(T)`. For any type `U` similar to `T`, the value of `required_alignment` is the same as `atomic_ref<remove_cv_t<U>>::required_alignment`.

- <sup>2</sup> *[Note 1: An implementation can choose to define `atomic_ref<T>::required_alignment` to a value greater than `alignof(T)` in order to ensure that operations on all objects of type `atomic_ref<T>` are lock-free. — end note]*

```
static constexpr bool is_always_lock_free;
```

- <sup>3</sup> The static data member `is_always_lock_free` is true if the `atomic_ref` type's operations are always lock-free, and false otherwise. For any type `U` similar to `T`, the value of `is_always_lock_free` is the same as `atomic_ref<remove_cv_t<U>>::is_always_lock_free`.

```
bool is_lock_free() const noexcept;
```

- <sup>4</sup> *Returns:* true if operations on all objects of the type `atomic_ref<T>` are lock-free, false otherwise.

```
constexpr atomic_ref(T& obj);
```

- <sup>5</sup> *Preconditions:* The referenced object is aligned to `required_alignment`.

- <sup>6</sup> *Postconditions:* `*this` references `obj`.

- <sup>7</sup> *Throws:* Nothing.

```
constexpr atomic_ref(const atomic_ref& ref) noexcept;
```

- <sup>8</sup> *Postconditions:* `*this` references the object referenced by `ref`.

```
template<class U>
```

```
constexpr atomic_ref(const atomic_ref<U>&) noexcept;
```

- <sup>9</sup> *Constraints:*

- (9.1) — `T` and `U` are similar types (7.3.6), and

- (9.2) — `is_convertible_v<U*, T*>` is true.

- <sup>10</sup> *Postconditions:* `*this` references the object referenced by `ref`.

```
constexpr void store(value_type desired,
 memory_order order = memory_order::seq_cst) const noexcept;
```

- <sup>11</sup> *Constraints:* `is_const_v<T>` is false.

- <sup>12</sup> *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

- <sup>13</sup> *Effects:* Atomically replaces the value referenced by `*ptr` with the value of `desired`. Memory is affected according to the value of `order`.

```
constexpr value_type operator=(value_type desired) const noexcept;
```

14     *Constraints:* `is_const_v<T>` is false.

15     *Effects:* Equivalent to:

```
 store(desired);
 return desired;
```

```
constexpr value_type load(memory_order order = memory_order::seq_cst) const noexcept;
```

16     *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

17     *Effects:* Memory is affected according to the value of `order`.

18     *Returns:* Atomically returns the value referenced by *\*ptr*.

```
constexpr operator value_type() const noexcept;
```

19     *Effects:* Equivalent to: `return load();`

```
constexpr value_type exchange(value_type desired,
 memory_order order = memory_order::seq_cst) const noexcept;
```

20     *Constraints:* `is_const_v<T>` is false.

21     *Effects:* Atomically replaces the value referenced by *\*ptr* with `desired`. Memory is affected according to the value of `order`. This operation is an atomic read-modify-write operation (6.10.2).

22     *Returns:* Atomically returns the value referenced by *\*ptr* immediately before the effects.

```
constexpr bool compare_exchange_weak(value_type& expected, value_type desired,
 memory_order success, memory_order failure) const noexcept;
```

```
constexpr bool compare_exchange_strong(value_type& expected, value_type desired,
 memory_order success, memory_order failure) const noexcept;
```

```
constexpr bool compare_exchange_weak(value_type& expected, value_type desired,
 memory_order order = memory_order::seq_cst) const noexcept;
```

```
constexpr bool compare_exchange_strong(value_type& expected, value_type desired,
 memory_order order = memory_order::seq_cst) const noexcept;
```

23     *Constraints:* `is_const_v<T>` is false.

24     *Preconditions:* `failure` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

25     *Effects:* Retrieves the value in `expected`. It then atomically compares the value representation of the value referenced by *\*ptr* for equality with that previously retrieved from `expected`, and if `true`, replaces the value referenced by *\*ptr* with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is `false` then, after the atomic operation, the value in `expected` is replaced by the value read from the value referenced by *\*ptr* during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations (6.10.2.2) on the value referenced by *\*ptr*. Otherwise, these operations are atomic load operations on that memory.

26     *Returns:* The result of the comparison.

27     *Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the value representations referred to by `expected` and *ptr* compare equal, it may return `false` and store back to `expected` the same value representation that was originally there.

[Note 2: This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield



better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — *end note*]

```
constexpr void wait(value_type old, memory_order order = memory_order::seq_cst) const noexcept;
```

28     *Preconditions:* order is memory\_order::relaxed, memory\_order::acquire, or memory\_order::seq\_cst.

29     *Effects:* Repeatedly performs the following steps, in order:

- (29.1)     — Evaluates load(order) and compares its value representation for equality against that of old.
- (29.2)     — If they compare unequal, returns.
- (29.3)     — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

30     *Remarks:* This function is an atomic waiting operation (32.5.6) on atomic object \*ptr.

```
constexpr void notify_one() const noexcept;
```

31     *Constraints:* is\_const\_v<T> is false.

32     *Effects:* Unblocks the execution of at least one atomic waiting operation on \*ptr that is eligible to be unblocked (32.5.6) by this call, if any such atomic waiting operations exist.

33     *Remarks:* This function is an atomic notifying operation (32.5.6) on atomic object \*ptr.

```
constexpr void notify_all() const noexcept;
```

34     *Constraints:* is\_const\_v<T> is false.

35     *Effects:* Unblocks the execution of all atomic waiting operations on \*ptr that are eligible to be unblocked (32.5.6) by this call.

36     *Remarks:* This function is an atomic notifying operation (32.5.6) on atomic object \*ptr.

```
constexpr address-return-type address() const noexcept;
```

37     *Returns:* ptr.

### 32.5.7.3 Specializations for integral types

[atomics.ref.int]

- 1 There are specializations of the atomic\_ref class template for all integral types except cv bool. For each such type *integral-type*, the specialization atomic\_ref<integral-type> provides additional atomic operations appropriate to integral types.

[*Note 1:* The specialization atomic\_ref<bool> uses the primary template (32.5.7). — *end note*]

- 2 The program is ill-formed if is\_always\_lock\_free is false and is\_volatile\_v<integral-type> is true.

```
namespace std {
 template<> struct atomic_ref<integral-type> {
 private:
 integral-type* ptr; // exposition only

 public:
 using value_type = remove_cv_t<integral-type>;
 using difference_type = value_type;
 using address-return-type = COPYCV(integral-type, void)*; // exposition only
 static constexpr size_t required_alignment = implementation-defined;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr explicit atomic_ref(integral-type&);
 explicit atomic_ref(integral-type&&) = delete;
 constexpr atomic_ref(const atomic_ref&) noexcept;
 template<class U>
 constexpr atomic_ref(const atomic_ref<U>&) noexcept;
 atomic_ref& operator=(const atomic_ref&) = delete;

 constexpr void store(value_type, memory_order = memory_order::seq_cst) const noexcept;
 constexpr value_type operator=(value_type) const noexcept;
```

```

constexpr value_type load(memory_order = memory_order::seq_cst) const noexcept;
constexpr operator value_type() const noexcept;

constexpr value_type exchange(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order, memory_order) const noexcept;
constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order, memory_order) const noexcept;
constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type fetch_add(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_sub(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_and(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_or(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_xor(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_max(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_min(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr void store_add(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_sub(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_and(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_or(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_xor(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_max(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_min(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type operator++(int) const noexcept;
constexpr value_type operator--(int) const noexcept;
constexpr value_type operator++() const noexcept;
constexpr value_type operator--() const noexcept;
constexpr value_type operator+=(value_type) const noexcept;
constexpr value_type operator-=(value_type) const noexcept;
constexpr value_type operator&=(value_type) const noexcept;
constexpr value_type operator|=(value_type) const noexcept;
constexpr value_type operator^=(value_type) const noexcept;

constexpr void wait(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void notify_one() const noexcept;
constexpr void notify_all() const noexcept;
constexpr address-return-type address() const noexcept;
};
}

```

<sup>3</sup> Descriptions are provided below only for members that differ from the primary template.

<sup>4</sup> The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in [Table 155](#).

```
constexpr value_type fetch_key(value_type operand,
 memory_order order = memory_order::seq_cst) const noexcept;
```

5     *Constraints:* `is_const_v<integral-type>` is false.

6     *Effects:* Atomically replaces the value referenced by *\*ptr* with the result of the computation applied to the value referenced by *\*ptr* and the given operand. Memory is affected according to the value of *order*. These operations are atomic read-modify-write operations (6.10.2.2).

7     *Returns:* Atomically, the value referenced by *\*ptr* immediately before the effects.

8     *Remarks:* Except for `fetch_max` and `fetch_min`, for signed integer types the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.

[Note 2: There are no undefined results arising from the computation. — end note]

9     For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with the object value and the first parameter as the arguments.

```
constexpr void store_key(value_type operand,
 memory_order order = memory_order::seq_cst) const noexcept;
```

10     *Constraints:* `is_const_v<integral-type>` is false.

11     *Preconditions:* *order* is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

12     *Effects:* Atomically replaces the value referenced by *\*ptr* with the result of the computation applied to the value referenced by *\*ptr* and the given operand. Memory is affected according to the value of *order*. These operations are atomic modify-write operations (32.5.4).

13     *Remarks:* Except for `store_max` and `store_min`, for signed integer types, the result is as if *\*ptr* and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.

[Note 3: There are no undefined results arising from the computation. — end note]

For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with *\*ptr* and the first parameter as the arguments.

```
constexpr value_type operator op=(value_type operand) const noexcept;
```

14     *Constraints:* `is_const_v<integral-type>` is false.

15     *Effects:* Equivalent to: `return fetch_key(operand) op operand;`

#### 32.5.7.4 Specializations for floating-point types

[atomics.ref.float]

1     There are specializations of the `atomic_ref` class template for all floating-point types. For each such type *floating-point-type*, the specialization `atomic_ref<floating-point-type>` provides additional atomic operations appropriate to floating-point types.

2     The program is ill-formed if `is_always_lock_free` is false and `is_volatile_v<floating-point-type>` is true.

```
namespace std {
 template<> struct atomic_ref<floating-point-type> {
 private:
 floating-point-type* ptr; // exposition only

 public:
 using value_type = remove_cv_t<floating-point-type>;
 using difference_type = value_type;
 using address-return-type = COPYCV(floating-point-type, void)*; // exposition only
 static constexpr size_t required_alignment = implementation-defined;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr explicit atomic_ref(floating-point-type&);
 explicit atomic_ref(floating-point-type&&) = delete;
```

```

constexpr atomic_ref(const atomic_ref&) noexcept;
template<class U>
 constexpr atomic_ref(const atomic_ref<U>&) noexcept;
atomic_ref& operator=(const atomic_ref&) = delete;

constexpr void store(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type operator=(value_type) const noexcept;
constexpr value_type load(memory_order = memory_order::seq_cst) const noexcept;
constexpr operator value_type() const noexcept;

constexpr value_type exchange(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order, memory_order) const noexcept;
constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order, memory_order) const noexcept;
constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type fetch_add(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_sub(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type fetch_max(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_min(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_fmaximum(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_fminimum(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_fmaximum_num(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_fminimum_num(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr void store_add(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_sub(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_max(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_min(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_fmaximum(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_fminimum(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_fmaximum_num(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_fminimum_num(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type operator+=(value_type) const noexcept;
constexpr value_type operator-=(value_type) const noexcept;

constexpr void wait(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void notify_one() const noexcept;
constexpr void notify_all() const noexcept;
constexpr address-return-type address() const noexcept;
};
}

```

3 Descriptions are provided below only for members that differ from the primary template.

4 The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 155, except for the keys `max`, `min`, `fmaximum`, `fminimum`, `fmaximum_num`, and `fminimum_num`, which are specified below.

```
constexpr value_type fetch_key(value_type operand,
 memory_order order = memory_order::seq_cst) const noexcept;
```

5 *Constraints:* `is_const_v<floating-point-type>` is false.

6 *Effects:* Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (6.10.2.2).

7 *Returns:* Atomically, the value referenced by `*ptr` immediately before the effects.

8 *Remarks:* If the result is not a representable value for its type (7.1), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `std::numeric_limits<value_type>` traits associated with the floating-point type (17.3.3). The floating-point environment (29.3) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment.

(8.1) — For `fetch_fmaximum` and `fetch_fminimum`, the maximum and minimum computation is performed as if by `fmaximum` and `fminimum`, respectively, with `*ptr` and the first parameter as the arguments.

(8.2) — For `fetch_fmaximum_num` and `fetch_fminimum_num`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with `*ptr` and the first parameter as the arguments.

(8.3) — For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with `*ptr` and the first parameter as the arguments, except that:

(8.3.1) — If both arguments are NaN, an unspecified NaN value is stored at `*ptr`.

(8.3.2) — If exactly one argument is a NaN, either the other argument or an unspecified NaN value is stored at `*ptr`; it is unspecified which.

(8.3.3) — If the arguments are differently signed zeros, which of these values is stored at `*ptr` is unspecified.

9 *Recommended practice:* The implementation of `fetch_max` and `fetch_min` should treat negative zero as smaller than positive zero.

```
constexpr void store_key(value_type operand,
 memory_order order = memory_order::seq_cst) const noexcept;
```

10 *Constraints:* `is_const_v<floating-point-type>` is false.

11 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

12 *Effects:* Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4).

13 *Remarks:* If the result is not a representable value for its type (7.1), the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `numeric_limits<floating-point-type>` traits associated with the floating-point type (17.3.3). The floating-point environment (29.3) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment. The arithmetic rules of floating-point atomic modify-write operations may be different from operations on floating-point types or atomic floating-point types.

[Note 1: Tree reductions are permitted for atomic modify-write operations. — end note]

(13.1) — For `store_fmaximum` and `store_fminimum`, the maximum and minimum computation is performed as if by `fmaximum` and `fminimum`, respectively, with `*ptr` and the first parameter as the arguments.

- (13.2) — For `store_fmaximum_num` and `store_fminimum_num`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with `*ptr` and the first parameter as the arguments.
- (13.3) — For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with `*ptr` and the first parameter as the arguments, except that:
- (13.3.1) — If both arguments are NaN, an unspecified NaN value is stored at `*ptr`.
- (13.3.2) — If exactly one argument is a NaN, either the other argument or an unspecified NaN value is stored at `*ptr`, it is unspecified which.
- (13.3.3) — If the arguments are differently signed zeros, which of these values is stored at `*ptr` is unspecified.
- 14 *Recommended practice:* The implementation of `store_max` and `store_min` should treat negative zero as smaller than positive zero.

`constexpr value_type operator op=(value_type operand) const noexcept;`

15 *Constraints:* `is_const_v<floating-point-type>` is false.

16 *Effects:* Equivalent to: `return fetch_key(operand) op operand;`

### 32.5.7.5 Specialization for pointers

[atomics.ref.pointer]

- 1 There are specializations of the `atomic_ref` class template for all pointer-to-object types. For each such type *pointer-type*, the specialization `atomic_ref<pointer-type>` provides additional atomic operations appropriate to pointer types.
- 2 Except during constant evaluation, when an operation on an atomic pointer would otherwise result in a pointer value `p` that is not valid in the context of the evaluation (6.9.5), that result instead is `reinterpret_cast<T*>(reinterpret_cast<uintptr_t>(p))`.
- 3 The program is ill-formed if `is_always_lock_free` is false and `is_volatile_v<pointer-type>` is true.

```
namespace std {
 template<> struct atomic_ref<pointer-type> {
 private:
 pointer-type* ptr; // exposition only

 public:
 using value_type = remove_cv_t<pointer-type>;
 using difference_type = ptrdiff_t;
 using address-return-type = COPYCV(pointer-type, void)*; // exposition only
 static constexpr size_t required_alignment = implementation-defined;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr explicit atomic_ref(pointer-type&);
 explicit atomic_ref(pointer-type&&) = delete;
 constexpr atomic_ref(const atomic_ref&) noexcept;
 template<class U>
 constexpr atomic_ref(const atomic_ref<U>&) noexcept;
 atomic_ref& operator=(const atomic_ref&) = delete;

 constexpr void store(value_type, memory_order = memory_order::seq_cst) const noexcept;
 constexpr value_type operator=(value_type) const noexcept;
 constexpr value_type load(memory_order = memory_order::seq_cst) const noexcept;
 constexpr operator value_type() const noexcept;

 constexpr value_type exchange(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
 constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order, memory_order) const noexcept;
 constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order, memory_order) const noexcept;
```

```

constexpr bool compare_exchange_weak(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr bool compare_exchange_strong(value_type&, value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type fetch_add(difference_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_sub(difference_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_max(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr value_type fetch_min(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr void store_add(difference_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_sub(difference_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_max(value_type,
 memory_order = memory_order::seq_cst) const noexcept;
constexpr void store_min(value_type,
 memory_order = memory_order::seq_cst) const noexcept;

constexpr value_type operator++(int) const noexcept;
constexpr value_type operator--(int) const noexcept;
constexpr value_type operator++() const noexcept;
constexpr value_type operator--() const noexcept;
constexpr value_type operator+=(difference_type) const noexcept;
constexpr value_type operator-=(difference_type) const noexcept;

constexpr void wait(value_type, memory_order = memory_order::seq_cst) const noexcept;
constexpr void notify_one() const noexcept;
constexpr void notify_all() const noexcept;
constexpr address-return-type address() const noexcept;
};
}

```

<sup>4</sup> Descriptions are provided below only for members that differ from the primary template.

<sup>5</sup> The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in [Table 156](#).

```

constexpr value_type fetch_key(see above operand,
 memory_order order = memory_order::seq_cst) const noexcept;

```

<sup>6</sup> *Constraints:* `is_const_v<pointer-type>` is false.

<sup>7</sup> *Mandates:* `remove_pointer_t<pointer-type>` is a complete object type.

<sup>8</sup> *Effects:* Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([6.10.2.2](#)).

<sup>9</sup> *Returns:* Atomically, the value referenced by `*ptr` immediately before the effects.

<sup>10</sup> *Remarks:* The result may be an undefined address, but the operations otherwise have no undefined behavior.

<sup>11</sup> For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms ([26.8.9](#)), respectively, with the object value and the first parameter as the arguments.

[*Note 1:* If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering ([Table 29](#), [7.6.9](#)). — *end note*]

```

constexpr void store_key(see above operand,
 memory_order order = memory_order::seq_cst) const noexcept;

```

<sup>12</sup> *Constraints:* `is_const_v<pointer-type>` is false.

<sup>13</sup> *Mandates:* `remove_pointer_t<pointer-type>` is a complete object type.



14 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

15 *Effects:* Atomically replaces the value referenced by `*ptr` with the result of the computation applied to the value referenced by `*ptr` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4).

16 *Remarks:* The result may be an undefined address, but the operations otherwise have no undefined behavior. For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with `*ptr` and the first parameter as the arguments.

[*Note 2:* If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering (Table 29, 7.6.9). — end note]

```
constexpr value_type operator op=(difference_type operand) const noexcept;
```

17 *Constraints:* `is_const_v<pointer-type>` is false.

18 *Effects:* Equivalent to: `return fetch_key(operand) op operand;`

### 32.5.7.6 Member operators common to integers and pointers to objects [atomics.ref.memop]

1 Let *referred-type* be *pointer-type* for the specializations in 32.5.7.5 and be *integral-type* for the specializations in 32.5.7.3.

```
constexpr value_type operator++(int) const noexcept;
```

2 *Constraints:* `is_const_v<referred-type>` is false.

3 *Effects:* Equivalent to: `return fetch_add(1);`

```
constexpr value_type operator--(int) const noexcept;
```

4 *Constraints:* `is_const_v<referred-type>` is false.

5 *Effects:* Equivalent to: `return fetch_sub(1);`

```
constexpr value_type operator++() const noexcept;
```

6 *Constraints:* `is_const_v<referred-type>` is false.

7 *Effects:* Equivalent to: `return fetch_add(1) + 1;`

```
constexpr value_type operator--() const noexcept;
```

8 *Constraints:* `is_const_v<referred-type>` is false.

9 *Effects:* Equivalent to: `return fetch_sub(1) - 1;`

## 32.5.8 Class template atomic

[atomics.types.generic]

### 32.5.8.1 General

[atomics.types.generic.general]

```
namespace std {
 template<class T> struct atomic {
 using value_type = T;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const volatile noexcept;
 bool is_lock_free() const noexcept;

 // 32.5.8.2, operations on atomic types
 constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>);
 constexpr atomic(T) noexcept;
 atomic(const atomic&) = delete;
 atomic& operator=(const atomic&) = delete;
 atomic& operator=(const atomic&) volatile = delete;

 T load(memory_order = memory_order::seq_cst) const volatile noexcept;
 constexpr T load(memory_order = memory_order::seq_cst) const noexcept;
 operator T() const volatile noexcept;
 constexpr operator T() const noexcept;
 void store(T, memory_order = memory_order::seq_cst) volatile noexcept;
```



```

constexpr void store(T, memory_order = memory_order::seq_cst) noexcept;
T operator=(T) volatile noexcept;
constexpr T operator=(T) noexcept;

T exchange(T, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T exchange(T, memory_order = memory_order::seq_cst) noexcept;
bool compare_exchange_weak(T&, T, memory_order, memory_order) volatile noexcept;
constexpr bool compare_exchange_weak(T&, T, memory_order, memory_order) noexcept;
bool compare_exchange_strong(T&, T, memory_order, memory_order) volatile noexcept;
constexpr bool compare_exchange_strong(T&, T, memory_order, memory_order) noexcept;
bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst) noexcept;
bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_cst) noexcept;

void wait(T, memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr void wait(T, memory_order = memory_order::seq_cst) const noexcept;
void notify_one() volatile noexcept;
constexpr void notify_one() noexcept;
void notify_all() volatile noexcept;
constexpr void notify_all() noexcept;
};
}

```

- <sup>1</sup> The template argument for T shall meet the *Cpp17CopyConstructible* and *Cpp17CopyAssignable* requirements. The program is ill-formed if any of

- (1.1) — `is_trivially_copyable_v<T>`,
- (1.2) — `is_copy_constructible_v<T>`,
- (1.3) — `is_move_constructible_v<T>`,
- (1.4) — `is_copy_assignable_v<T>`,
- (1.5) — `is_move_assignable_v<T>`, or
- (1.6) — `is_same_v<T, remove_cv_t<T>>`,

is false.

[Note 1: Type arguments that are not also statically initializable can be difficult to use. — end note]

- <sup>2</sup> The specialization `atomic<bool>` is a standard-layout struct. It has a trivial destructor.
- <sup>3</sup> [Note 2: The representation of an atomic specialization need not have the same size and alignment requirement as its corresponding argument type. — end note]

### 32.5.8.2 Operations on atomic types

[atomics.types.operations]

```
constexpr atomic() noexcept(is_nothrow_default_constructible_v<T>);
```

- <sup>1</sup> *Constraints:* `is_default_constructible_v<T>` is true.

- <sup>2</sup> *Effects:* Initializes the atomic object with the value of `T()`. Initialization is not an atomic operation (6.10.2).

```
constexpr atomic(T desired) noexcept;
```

- <sup>3</sup> *Effects:* Initializes the object with the value `desired`. Initialization is not an atomic operation (6.10.2).

[Note 1: It is possible to have an access to an atomic object A race with its construction, for example by communicating the address of the just-constructed object A to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing A in the receiving thread. This results in undefined behavior. — end note]

```
static constexpr bool is_always_lock_free = implementation-defined;
```

- <sup>4</sup> The static data member `is_always_lock_free` is true if the atomic type's operations are always lock-free, and false otherwise.

[Note 2: The value of `is_always_lock_free` is consistent with the value of the corresponding `ATOMIC_..._LOCK_FREE` macro, if defined. — end note]

```
bool is_lock_free() const volatile noexcept;
bool is_lock_free() const noexcept;
```

5     *Returns:* true if the object's operations are lock-free, false otherwise.

[Note 3: The return value of the `is_lock_free` member function is consistent with the value of `is_always_lock_free` for the same type. — end note]

```
void store(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr void store(T desired, memory_order order = memory_order::seq_cst) noexcept;
```

6     *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

7     *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

8     *Effects:* Atomically replaces the value pointed to by `this` with the value of `desired`. Memory is affected according to the value of `order`.

```
T operator=(T desired) volatile noexcept;
constexpr T operator=(T desired) noexcept;
```

9     *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

10    *Effects:* Equivalent to `store(desired)`.

11    *Returns:* `desired`.

```
T load(memory_order order = memory_order::seq_cst) const volatile noexcept;
constexpr T load(memory_order order = memory_order::seq_cst) const noexcept;
```

12    *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

13    *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

14    *Effects:* Memory is affected according to the value of `order`.

15    *Returns:* Atomically returns the value pointed to by `this`.

```
operator T() const volatile noexcept;
constexpr operator T() const noexcept;
```

16    *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

17    *Effects:* Equivalent to: `return load();`

```
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr T exchange(T desired, memory_order order = memory_order::seq_cst) noexcept;
```

18    *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

19    *Effects:* Atomically replaces the value pointed to by `this` with `desired`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (6.10.2).

20    *Returns:* Atomically returns the value pointed to by `this` immediately before the effects.

```
bool compare_exchange_weak(T& expected, T desired,
 memory_order success, memory_order failure) volatile noexcept;
constexpr bool compare_exchange_weak(T& expected, T desired,
 memory_order success, memory_order failure) noexcept;
bool compare_exchange_strong(T& expected, T desired,
 memory_order success, memory_order failure) volatile noexcept;
constexpr bool compare_exchange_strong(T& expected, T desired,
 memory_order success, memory_order failure) noexcept;
bool compare_exchange_weak(T& expected, T desired,
 memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_weak(T& expected, T desired,
 memory_order order = memory_order::seq_cst) noexcept;
bool compare_exchange_strong(T& expected, T desired,
 memory_order order = memory_order::seq_cst) volatile noexcept;
```

```
constexpr bool compare_exchange_strong(T& expected, T desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

21 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is `true`.

22 *Preconditions:* `failure` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

23 *Effects:* Retrieves the value in `expected`. It then atomically compares the value representation of the value pointed to by `this` for equality with that previously retrieved from `expected`, and if `true`, replaces the value pointed to by `this` with that in `desired`. If and only if the comparison is `true`, memory is affected according to the value of `success`, and if the comparison is `false`, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`. If and only if the comparison is `false` then, after the atomic operation, the value in `expected` is replaced by the value pointed to by `this` during the atomic comparison. If the operation returns `true`, these operations are atomic read-modify-write operations (6.10.2.2) on the memory pointed to by `this`. Otherwise, these operations are atomic load operations on that memory.

24 *Returns:* The result of the comparison.

25 [Note 4: For example, the effect of `compare_exchange_strong` on objects without padding bits (6.9.1) is

```
 if (memcmp(this, &expected, sizeof(*this)) == 0)
 memcpy(this, &desired, sizeof(*this));
 else
 memcpy(&expected, this, sizeof(*this));
```

— end note]

[Example 1: The expected use of the compare-and-exchange operations is as follows. The compare-and-exchange operations will update `expected` when another iteration of the loop is needed.

```
 expected = current.load();
 do {
 desired = function(expected);
 } while (!current.compare_exchange_weak(expected, desired));
```

— end example]

[Example 2: Because the expected value is updated only on failure, code releasing the memory containing the `expected` value on success will work. For example, list head insertion will act atomically and would not introduce a data race in the following code:

```
 do {
 p->next = head;
 } while (!head.compare_exchange_weak(p->next, p)); // make new list node point to the current head // try to insert
```

— end example]

26 *Recommended practice:* Implementations should ensure that weak compare-and-exchange operations do not consistently return `false` unless either the atomic object has value different from `expected` or there are concurrent modifications to the atomic object.

27 *Remarks:* A weak compare-and-exchange operation may fail spuriously. That is, even when the value representations referred to by `expected` and `this` compare equal, it may return `false` and store back to `expected` the same value representation that was originally there.

[Note 5: This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop. When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — end note]

28 [Note 6: Under cases where the `memcpy` and `memcmp` semantics of the compare-and-exchange operations apply, the comparisons can fail for values that compare equal with `operator==` if the value representation has trap bits or alternate representations of the same value. Notably, on implementations conforming to ISO/IEC 60559, floating-point `-0.0` and `+0.0` will not compare equal with `memcmp` but will compare equal with `operator==`, and NaNs with the same payload will compare equal with `memcmp` but will not compare equal with `operator==`. — end note]

[*Note 7:* Because compare-and-exchange acts on an object's value representation, padding bits that never participate in the object's value representation are ignored. As a consequence, the following code is guaranteed to avoid spurious failure:

```
struct padded {
 char clank = 0x42;
 // Padding here.
 unsigned biff = 0xC0DEFEFE;
};
atomic<padded> pad = {};

bool zap() {
 padded expected, desired{0, 0};
 return pad.compare_exchange_strong(expected, desired);
}
```

— *end note*]

[*Note 8:* For a union with bits that participate in the value representation of some members but not others, compare-and-exchange might always fail. This is because such padding bits have an indeterminate value when they do not participate in the value representation of the active member. As a consequence, the following code is not guaranteed to ever succeed:

```
union pony {
 double celestia = 0.;
 short luna; // padded
};
atomic<pony> princesses = {};

bool party(pony desired) {
 pony expected;
 return princesses.compare_exchange_strong(expected, desired);
}
```

— *end note*]

```
void wait(T old, memory_order order = memory_order::seq_cst) const volatile noexcept;
constexpr void wait(T old, memory_order order = memory_order::seq_cst) const noexcept;
```

29     *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

30     *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

31     *Effects:* Repeatedly performs the following steps, in order:

- (31.1)     — Evaluates `load(order)` and compares its value representation for equality against that of `old`.
- (31.2)     — If they compare unequal, returns.
- (31.3)     — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

32     *Remarks:* This function is an atomic waiting operation (32.5.6).

```
void notify_one() volatile noexcept;
constexpr void notify_one() noexcept;
```

33     *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

34     *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked (32.5.6) by this call, if any such atomic waiting operations exist.

35     *Remarks:* This function is an atomic notifying operation (32.5.6).

```
void notify_all() volatile noexcept;
constexpr void notify_all() noexcept;
```

36     *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

37     *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked (32.5.6) by this call.

38     *Remarks:* This function is an atomic notifying operation (32.5.6).

## 32.5.8.3 Specializations for integers

[atomics.types.int]

- <sup>1</sup> There are specializations of the `atomic` class template for each character type (6.9.3), each standard integer type, and any other types needed by the typedefs in the header `<stdint>` (17.4.1). For each such type *integral-type*, the specialization `atomic<integral-type>` provides additional atomic operations appropriate to integral types.

[Note 1: The specialization `atomic<bool>` uses the primary template (32.5.8). — end note]

```
namespace std {
 template<> struct atomic<integral-type> {
 using value_type = integral-type;
 using difference_type = value_type;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const volatile noexcept;
 bool is_lock_free() const noexcept;

 constexpr atomic() noexcept;
 constexpr atomic(integral-type) noexcept;
 atomic(const atomic&) = delete;
 atomic& operator=(const atomic&) = delete;
 atomic& operator=(const atomic&) volatile = delete;

 void store(integral-type, memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr void store(integral-type, memory_order = memory_order::seq_cst) noexcept;
 integral-type operator=(integral-type) volatile noexcept;
 constexpr integral-type operator=(integral-type) noexcept;
 integral-type load(memory_order = memory_order::seq_cst) const volatile noexcept;
 constexpr integral-type load(memory_order = memory_order::seq_cst) const noexcept;
 operator integral-type() const volatile noexcept;
 constexpr operator integral-type() const noexcept;

 integral-type exchange(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr integral-type exchange(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
 bool compare_exchange_weak(integral-type&, integral-type,
 memory_order, memory_order) volatile noexcept;
 constexpr bool compare_exchange_weak(integral-type&, integral-type,
 memory_order, memory_order) noexcept;
 bool compare_exchange_strong(integral-type&, integral-type,
 memory_order, memory_order) volatile noexcept;
 constexpr bool compare_exchange_strong(integral-type&, integral-type,
 memory_order, memory_order) noexcept;
 bool compare_exchange_weak(integral-type&, integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr bool compare_exchange_weak(integral-type&, integral-type,
 memory_order = memory_order::seq_cst) noexcept;
 bool compare_exchange_strong(integral-type&, integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr bool compare_exchange_strong(integral-type&, integral-type,
 memory_order = memory_order::seq_cst) noexcept;

 integral-type fetch_add(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr integral-type fetch_add(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
 integral-type fetch_sub(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr integral-type fetch_sub(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
 integral-type fetch_and(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
```

```

constexpr integral-type fetch_and(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
integral-type fetch_or(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr integral-type fetch_or(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
integral-type fetch_xor(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr integral-type fetch_xor(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
integral-type fetch_max(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr integral-type fetch_max(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
integral-type fetch_min(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr integral-type fetch_min(integral-type,
 memory_order = memory_order::seq_cst) noexcept;

void store_add(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_add(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_sub(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_sub(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_and(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_and(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_or(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_or(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_xor(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_xor(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_max(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_max(integral-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_min(integral-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_min(integral-type,
 memory_order = memory_order::seq_cst) noexcept;

integral-type operator++(int) volatile noexcept;
constexpr integral-type operator++(int) noexcept;
integral-type operator--(int) volatile noexcept;
constexpr integral-type operator--(int) noexcept;
integral-type operator++() volatile noexcept;
constexpr integral-type operator++() noexcept;
integral-type operator--() volatile noexcept;
constexpr integral-type operator--() noexcept;
integral-type operator+=(integral-type) volatile noexcept;
constexpr integral-type operator+=(integral-type) noexcept;
integral-type operator-=(integral-type) volatile noexcept;
constexpr integral-type operator-=(integral-type) noexcept;
integral-type operator&=(integral-type) volatile noexcept;
constexpr integral-type operator&=(integral-type) noexcept;
integral-type operator|=(integral-type) volatile noexcept;
constexpr integral-type operator|=(integral-type) noexcept;

```

```

 integral-type operator^=(integral-type) volatile noexcept;
 constexpr integral-type operator^=(integral-type) noexcept;

 void wait(integral-type, memory_order = memory_order::seq_cst) const volatile noexcept;
 constexpr void wait(integral-type, memory_order = memory_order::seq_cst) const noexcept;
 void notify_one() volatile noexcept;
 constexpr void notify_one() noexcept;
 void notify_all() volatile noexcept;
 constexpr void notify_all() noexcept;
};
}

```

- 2 The atomic integral specializations are standard-layout structs. They each have a trivial destructor.
- 3 Descriptions are provided below only for members that differ from the primary template.
- 4 The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in [Table 155](#).

**Table 155 — Atomic arithmetic computations** [tab:atomic.types.int.comp]

| key | Op | Computation | key | Op | Computation          |
|-----|----|-------------|-----|----|----------------------|
| add | +  | addition    | and | &  | bitwise and          |
| sub | -  | subtraction | or  |    | bitwise inclusive or |
| max |    | maximum     | xor | ^  | bitwise exclusive or |
| min |    | minimum     |     |    |                      |

```

integral-type fetch_key(integral-type operand,
 memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr integral-type fetch_key(integral-type operand,
 memory_order order = memory_order::seq_cst) noexcept;

```

- 5 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.
- 6 *Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations ([6.10.2](#)).
- 7 *Returns:* Atomically, the value pointed to by `this` immediately before the effects.
- 8 *Remarks:* Except for `fetch_max` and `fetch_min`, for signed integer types the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
- [Note 2: There are no undefined results arising from the computation. — end note]
- 9 For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms ([26.8.9](#)), respectively, with the object value and the first parameter as the arguments.

```

void store_key(integral-type operand,
 memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr void store_key(integral-type operand,
 memory_order order = memory_order::seq_cst) noexcept;

```

- 10 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.
- 11 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.
- 12 *Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations ([32.5.4](#)).
- 13 *Remarks:* Except for `store_max` and `store_min`, for signed integer types, the result is as if the value pointed to by `this` and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
- [Note 3: There are no undefined results arising from the computation. — end note]



For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with the value pointed to by `this` and the first parameter as the arguments.

```
integral-type operator op=(integral-type operand) volatile noexcept;
constexpr integral-type operator op=(integral-type operand) noexcept;
```

14 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

15 *Effects:* Equivalent to: `return fetch_key(operand) op operand;`

#### 32.5.8.4 Specializations for floating-point types [atomics.types.float]

- 1 There are specializations of the atomic class template for all cv-unqualified floating-point types. For each such type *floating-point-type*, the specialization `atomic<floating-point-type>` provides additional atomic operations appropriate to floating-point types.

```
namespace std {
 template<> struct atomic<floating-point-type> {
 using value_type = floating-point-type;
 using difference_type = value_type;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const volatile noexcept;
 bool is_lock_free() const noexcept;

 constexpr atomic() noexcept;
 constexpr atomic(floating-point-type) noexcept;
 atomic(const atomic&) = delete;
 atomic& operator=(const atomic&) = delete;
 atomic& operator=(const atomic&) volatile = delete;

 void store(floating-point-type, memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr void store(floating-point-type, memory_order = memory_order::seq_cst) noexcept;
 floating-point-type operator=(floating-point-type) volatile noexcept;
 constexpr floating-point-type operator=(floating-point-type) noexcept;
 floating-point-type load(memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr floating-point-type load(memory_order = memory_order::seq_cst) noexcept;
 operator floating-point-type() volatile noexcept;
 constexpr operator floating-point-type() noexcept;

 floating-point-type exchange(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr floating-point-type exchange(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
 bool compare_exchange_weak(floating-point-type&, floating-point-type,
 memory_order, memory_order) volatile noexcept;
 constexpr bool compare_exchange_weak(floating-point-type&, floating-point-type,
 memory_order, memory_order) noexcept;
 bool compare_exchange_strong(floating-point-type&, floating-point-type,
 memory_order, memory_order) volatile noexcept;
 constexpr bool compare_exchange_strong(floating-point-type&, floating-point-type,
 memory_order, memory_order) noexcept;
 bool compare_exchange_weak(floating-point-type&, floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr bool compare_exchange_weak(floating-point-type&, floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
 bool compare_exchange_strong(floating-point-type&, floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr bool compare_exchange_strong(floating-point-type&, floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;

 floating-point-type fetch_add(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
 constexpr floating-point-type fetch_add(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
```



```

floating-point-type fetch_sub(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_sub(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_max(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_max(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_min(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_min(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_fmaximum(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_fmaximum(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_fminimum(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_fminimum(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_fmaximum_num(
 floating-point-type, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_fmaximum_num(
 floating-point-type, memory_order = memory_order::seq_cst) noexcept;
floating-point-type fetch_fminimum_num(
 floating-point-type, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_fminimum_num(
 floating-point-type, memory_order = memory_order::seq_cst) noexcept;

void store_add(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_add(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_sub(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_sub(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_max(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_max(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_min(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_min(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_fmaximum(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_fmaximum(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_fminimum(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_fminimum(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_fmaximum_num(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_fmaximum_num(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;
void store_fminimum_num(floating-point-type,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_fminimum_num(floating-point-type,
 memory_order = memory_order::seq_cst) noexcept;

floating-point-type operator+=(floating-point-type) volatile noexcept;
constexpr floating-point-type operator+=(floating-point-type) noexcept;

```

```

floating-point-type operator==(floating-point-type) volatile noexcept;
constexpr floating-point-type operator==(floating-point-type) noexcept;

void wait(floating-point-type, memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr void wait(floating-point-type,
 memory_order = memory_order::seq_cst) const noexcept;
void notify_one() volatile noexcept;
constexpr void notify_one() noexcept;
void notify_all() volatile noexcept;
constexpr void notify_all() noexcept;
};
}

```

2 The atomic floating-point specializations are standard-layout structs. They each have a trivial destructor.

3 Descriptions are provided below only for members that differ from the primary template.

4 The following operations perform arithmetic addition and subtraction computations. The correspondence among key, operator, and computation is specified in Table 155, except for the keys max, min, fmaximum, fminimum, fmaximum\_num, and fminimum\_num, which are specified below.

```

floating-point-type fetch_key(floating-point-type operand,
 memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr floating-point-type fetch_key(floating-point-type operand,
 memory_order order = memory_order::seq_cst) noexcept;

```

5 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

6 *Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (6.10.2).

7 *Returns:* Atomically, the value pointed to by `this` immediately before the effects.

8 *Remarks:* If the result is not a representable value for its type (7.1) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type (17.3.3). The floating-point environment (29.3) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment.

- (8.1) — For `fetch_fmaximum` and `fetch_fminimum`, the maximum and minimum computation is performed as if by `fmaximum` and `fminimum`, respectively, with the value pointed to by `this` and the first parameter as the arguments.
- (8.2) — For `fetch_fmaximum_num` and `fetch_fminimum_num`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with the value pointed to by `this` and the first parameter as the arguments.
- (8.3) — For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with the value pointed to by `this` and the first parameter as the arguments, except that:
  - (8.3.1) — If both arguments are NaN, an unspecified NaN value replaces the value pointed to by `this`.
  - (8.3.2) — If exactly one argument is a NaN, either the other argument or an unspecified NaN value replaces the value pointed to by `this`; it is unspecified which.
  - (8.3.3) — If the arguments are differently signed zeros, which of these values replaces the value pointed to by `this` is unspecified.

9 *Recommended practice:* The implementation of `fetch_max` and `fetch_min` should treat negative zero as smaller than positive zero.

```

void store_key(floating-point-type operand,
 memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr void store_key(floating-point-type operand,
 memory_order order = memory_order::seq_cst) noexcept;

```

10 *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

*Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

*Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given operand. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4).

*Remarks:* If the result is not a representable value for its type (7.1) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `numeric_limits<floating-point-type>` traits associated with the floating-point type (17.3.3). The floating-point environment (29.3) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment. The arithmetic rules of floating-point atomic modify-write operations may be different from operations on floating-point types or atomic floating-point types.

[*Note 1:* Tree reductions are permitted for atomic modify-write operations. — *end note*]

— For `store_fmaximum` and `store_fminimum`, the maximum and minimum computation is performed as if by `fmaximum` and `fminimum`, respectively, with the value pointed to by `this` and the first parameter as the arguments.

— For `store_fmaximum_num` and `store_fminimum_num`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with the value pointed to by `this` and the first parameter as the arguments.

— For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `fmaximum_num` and `fminimum_num`, respectively, with the value pointed to by `this` and the first parameter as the arguments, except that:

— If both arguments are NaN, an unspecified NaN value replaces the value pointed to by `this`.

— If exactly one argument is a NaN, either the other argument or an unspecified NaN value replaces the value pointed to by `this`; it is unspecified which.

— If the arguments are differently signed zeros, which of these values replaces the value pointed to by `this` is unspecified.

*Recommended practice:* The implementation of `store_max` and `store_min` should treat negative zero as smaller than positive zero.

```
floating-point-type operator op=(floating-point-type operand) volatile noexcept;
constexpr floating-point-type operator op=(floating-point-type operand) noexcept;
```

*Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

*Effects:* Equivalent to: `return fetch_key(operand) op operand;`

*Remarks:* If the result is not a representable value for its type (7.1) the result is unspecified, but the operations otherwise have no undefined behavior. Atomic arithmetic operations on *floating-point-type* should conform to the `std::numeric_limits<floating-point-type>` traits associated with the floating-point type (17.3.3). The floating-point environment (29.3) for atomic arithmetic operations on *floating-point-type* may be different than the calling thread's floating-point environment.

### 32.5.8.5 Partial specialization for pointers

[`atomics.types.pointer`]

```
namespace std {
 template<class T> struct atomic<T*> {
 using value_type = T*;
 using difference_type = ptrdiff_t;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const volatile noexcept;
 bool is_lock_free() const noexcept;

 constexpr atomic() noexcept;
 constexpr atomic(T*) noexcept;
 atomic(const atomic&) = delete;
 atomic& operator=(const atomic&) = delete;
 atomic& operator=(const atomic&) volatile = delete;
```

```

void store(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store(T*, memory_order = memory_order::seq_cst) noexcept;
T* operator=(T*) volatile noexcept;
constexpr T* operator=(T*) noexcept;
T* load(memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr T* load(memory_order = memory_order::seq_cst) const noexcept;
operator T*() const volatile noexcept;
constexpr operator T*() const noexcept;

T* exchange(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T* exchange(T*, memory_order = memory_order::seq_cst) noexcept;
bool compare_exchange_weak(T*&, T*, memory_order, memory_order) volatile noexcept;
constexpr bool compare_exchange_weak(T*&, T*, memory_order, memory_order) noexcept;
bool compare_exchange_strong(T*&, T*, memory_order, memory_order) volatile noexcept;
constexpr bool compare_exchange_strong(T*&, T*, memory_order, memory_order) noexcept;
bool compare_exchange_weak(T*&, T*,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_weak(T*&, T*,
 memory_order = memory_order::seq_cst) noexcept;
bool compare_exchange_strong(T*&, T*,
 memory_order = memory_order::seq_cst) volatile noexcept;
constexpr bool compare_exchange_strong(T*&, T*,
 memory_order = memory_order::seq_cst) noexcept;

T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;
T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;
T* fetch_max(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T* fetch_max(T*, memory_order = memory_order::seq_cst) noexcept;
T* fetch_min(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr T* fetch_min(T*, memory_order = memory_order::seq_cst) noexcept;

void store_add(ptrdiff_t, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_add(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;
void store_sub(ptrdiff_t, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_sub(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;
void store_max(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_max(T*, memory_order = memory_order::seq_cst) noexcept;
void store_min(T*, memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void store_min(T*, memory_order = memory_order::seq_cst) noexcept;

T* operator++(int) volatile noexcept;
constexpr T* operator++(int) noexcept;
T* operator--(int) volatile noexcept;
constexpr T* operator--(int) noexcept;
T* operator++() volatile noexcept;
constexpr T* operator++() noexcept;
T* operator--() volatile noexcept;
constexpr T* operator--() noexcept;
T* operator+=(ptrdiff_t) volatile noexcept;
constexpr T* operator+=(ptrdiff_t) noexcept;
T* operator-=(ptrdiff_t) volatile noexcept;
constexpr T* operator-=(ptrdiff_t) noexcept;

void wait(T*, memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
void notify_one() volatile noexcept;
constexpr void notify_one() noexcept;
void notify_all() volatile noexcept;
constexpr void notify_all() noexcept;
};
}

```

- <sup>1</sup> There is a partial specialization of the `atomic` class template for pointers. Specializations of this partial specialization are standard-layout structs. They each have a trivial destructor.
- <sup>2</sup> Except during constant evaluation, when an operation on an atomic pointer would otherwise result in a pointer value `p` that is not valid in the context of the evaluation (6.9.5), that result instead is `reinterpret_cast<T*>(reinterpret_cast<uintptr_t>(p))`.
- <sup>3</sup> Descriptions are provided below only for members that differ from the primary template.
- <sup>4</sup> The following operations perform pointer arithmetic. The correspondence among key, operator, and computation is specified in Table 156.

**Table 156 — Atomic pointer computations** [tab:atomic.types.pointer.comp]

| key | Op | Computation | key | Op | Computation |
|-----|----|-------------|-----|----|-------------|
| add | +  | addition    | sub | -  | subtraction |
| max |    | maximum     | min |    | minimum     |

```
T* fetch_key(see above operand, memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr T* fetch_key(see above operand, memory_order order = memory_order::seq_cst) noexcept;
```

<sup>5</sup> *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

<sup>6</sup> *Mandates:* `T` is a complete object type.

[Note 1: Pointer arithmetic on `void*` or function pointers is ill-formed. — end note]

<sup>7</sup> *Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (6.10.2).

<sup>8</sup> *Returns:* Atomically, the value pointed to by `this` immediately before the effects.

<sup>9</sup> *Remarks:* The result may be an undefined address, but the operations otherwise have no undefined behavior.

<sup>10</sup> For `fetch_max` and `fetch_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with the object value and the first parameter as the arguments.

[Note 2: If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering (Table 29, 7.6.9). — end note]

```
void store_key(see above operand, memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr void store_key(see above operand, memory_order order = memory_order::seq_cst) noexcept;
```

<sup>11</sup> *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

<sup>12</sup> *Mandates:* `T` is a complete object type.

[Note 3: Pointer arithmetic on `void*` or function pointers is ill-formed. — end note]

<sup>13</sup> *Effects:* Atomically replaces the value pointed to by `this` with the result of the computation applied to the value pointed to by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic modify-write operations (32.5.4).

<sup>14</sup> *Remarks:* The result may be an undefined address, but the operations otherwise have no undefined behavior. For `store_max` and `store_min`, the maximum and minimum computation is performed as if by `max` and `min` algorithms (26.8.9), respectively, with the value pointed to by `this` and the first parameter as the arguments.

[Note 4: If the pointers point to different complete objects (or subobjects thereof), the `<` operator does not establish a strict weak ordering (Table 29, 7.6.9). — end note]

```
T* operator op=(ptrdiff_t operand) volatile noexcept;
constexpr T* operator op=(ptrdiff_t operand) noexcept;
```

<sup>15</sup> *Constraints:* For the `volatile` overload of this function, `is_always_lock_free` is `true`.

<sup>16</sup> *Effects:* Equivalent to: `return fetch_key(operand) op operand;`

**32.5.8.6 Member operators common to integers and pointers to objects**[atomics.types.memop]

```
value_type operator++(int) volatile noexcept;
constexpr value_type operator++(int) noexcept;
```

1       *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

2       *Effects:* Equivalent to: `return fetch_add(1);`

```
value_type operator--(int) volatile noexcept;
constexpr value_type operator--(int) noexcept;
```

3       *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

4       *Effects:* Equivalent to: `return fetch_sub(1);`

```
value_type operator++() volatile noexcept;
constexpr value_type operator++() noexcept;
```

5       *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

6       *Effects:* Equivalent to: `return fetch_add(1) + 1;`

```
value_type operator--() volatile noexcept;
constexpr value_type operator--() noexcept;
```

7       *Constraints:* For the volatile overload of this function, `is_always_lock_free` is true.

8       *Effects:* Equivalent to: `return fetch_sub(1) - 1;`

**32.5.8.7 Partial specializations for smart pointers**

[util.smartptr.atomic]

**32.5.8.7.1 General**

[util.smartptr.atomic.general]

1 The library provides partial specializations of the `atomic` template for shared-ownership smart pointers (20.4.2).

[*Note 1:* The partial specializations are declared in header `<memory>` (20.2.2). — *end note*]

The behavior of all operations is as specified in 32.5.8, unless specified otherwise. The template parameter `T` of these partial specializations may be an incomplete type.

2 All changes to an atomic smart pointer in 32.5.8.7, and all associated `use_count` increments, are guaranteed to be performed atomically. Associated `use_count` decrements are sequenced after the atomic operation, but are not required to be part of it. Any associated deletion and deallocation are sequenced after the atomic update step and are not part of the atomic operation.

[*Note 2:* If the atomic operation uses locks, locks acquired by the implementation will be held when any `use_count` adjustments are performed, and will not be held when any destruction or deallocation resulting from this is performed. — *end note*]

3 [*Example 1:*

```
template<typename T> class atomic_list {
 struct node {
 T t;
 shared_ptr<node> next;
 };
 atomic<shared_ptr<node>> head;

public:
 shared_ptr<node> find(T t) const {
 auto p = head.load();
 while (p && p->t != t)
 p = p->next;

 return p;
 }

 void push_front(T t) {
 auto p = make_shared<node>();
 p->t = t;
 p->next = head;
 }
};
```

```

 while (!head.compare_exchange_weak(p->next, p)) {}
}
};
— end example]

```

### 32.5.8.7.2 Partial specialization for shared\_ptr

[util.smartptr.atomic.shared]

```

namespace std {
 template<class T> struct atomic<shared_ptr<T>> {
 using value_type = shared_ptr<T>;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr atomic() noexcept;
 constexpr atomic(nullptr_t) noexcept : atomic() { }
 constexpr atomic(shared_ptr<T> desired) noexcept;
 atomic(const atomic&) = delete;
 void operator=(const atomic&) = delete;

 constexpr shared_ptr<T> load(memory_order order = memory_order::seq_cst) const noexcept;
 constexpr operator shared_ptr<T>() const noexcept;
 constexpr void store(shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr void operator=(shared_ptr<T> desired) noexcept;
 constexpr void operator=(nullptr_t) noexcept;

 constexpr shared_ptr<T> exchange(shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
 constexpr bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
 constexpr bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;

 constexpr void wait(shared_ptr<T> old,
 memory_order order = memory_order::seq_cst) const noexcept;
 constexpr void notify_one() noexcept;
 constexpr void notify_all() noexcept;

 private:
 shared_ptr<T> p; // exposition only
 };
}

```

```
constexpr atomic() noexcept;
```

<sup>1</sup> *Effects:* Value-initializes *p*.

```
constexpr atomic(shared_ptr<T> desired) noexcept;
```

<sup>2</sup> *Effects:* Initializes the object with the value *desired*. Initialization is not an atomic operation (6.10.2).

[Note 1: It is possible to have an access to an atomic object *A* race with its construction, for example, by communicating the address of the just-constructed object *A* to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing *A* in the receiving thread. This results in undefined behavior. — end note]

```
constexpr void store(shared_ptr<T> desired, memory_order order = memory_order::seq_cst) noexcept;
```

<sup>3</sup> *Preconditions:* *order* is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.



4     *Effects:* Atomically replaces the value pointed to by `this` with the value of `desired` as if by `p.swap(desired)`. Memory is affected according to the value of `order`.

```
constexpr void operator=(shared_ptr<T> desired) noexcept;
```

5     *Effects:* Equivalent to `store(desired)`.

```
constexpr void operator=(nullptr_t) noexcept;
```

6     *Effects:* Equivalent to `store(nullptr)`.

```
constexpr shared_ptr<T> load(memory_order order = memory_order::seq_cst) const noexcept;
```

7     *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

8     *Effects:* Memory is affected according to the value of `order`.

9     *Returns:* Atomically returns `p`.

```
constexpr operator shared_ptr<T>() const noexcept;
```

10    *Effects:* Equivalent to: `return load();`

```
constexpr shared_ptr<T> exchange(shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

11    *Effects:* Atomically replaces `p` with `desired` as if by `p.swap(desired)`. Memory is affected according to the value of `order`. This is an atomic read-modify-write operation (6.10.2.2).

12    *Returns:* Atomically returns the value of `p` immediately before the effects.

```
constexpr bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
```

```
constexpr bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
```

13    *Preconditions:* `failure` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

14    *Effects:* If `p` is equivalent to `expected`, assigns `desired` to `p` and has synchronization semantics corresponding to the value of `success`, otherwise assigns `p` to `expected` and has synchronization semantics corresponding to the value of `failure`.

15    *Returns:* `true` if `p` was equivalent to `expected`, `false` otherwise.

16    *Remarks:* Two `shared_ptr` objects are equivalent if they store the same pointer value and either share ownership or are both empty. The weak form may fail spuriously. See 32.5.8.2.

17    If the operation returns `true`, `expected` is not accessed after the atomic update and the operation is an atomic read-modify-write operation (6.10.2) on the memory pointed to by `this`. Otherwise, the operation is an atomic load operation on that memory, and `expected` is updated with the existing value read from the atomic object in the attempted atomic update. The `use_count` update corresponding to the write to `expected` is part of the atomic operation. The write to `expected` itself is not required to be part of the atomic operation.

```
constexpr bool compare_exchange_weak(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

18    *Effects:* Equivalent to:

```
 return compare_exchange_weak(expected, desired, order, fail_order);
```

where `fail_order` is the same as `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`.

```
constexpr bool compare_exchange_strong(shared_ptr<T>& expected, shared_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

19    *Effects:* Equivalent to:

```
 return compare_exchange_strong(expected, desired, order, fail_order);
```



where `fail_order` is the same as `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`.

```
constexpr void wait(shared_ptr<T> old, memory_order order = memory_order::seq_cst) const noexcept;
```

20 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

21 *Effects:* Repeatedly performs the following steps, in order:

(21.1) — Evaluates `load(order)` and compares it to `old`.

(21.2) — If the two are not equivalent, returns.

(21.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

22 *Remarks:* Two `shared_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation (32.5.6).

```
constexpr void notify_one() noexcept;
```

23 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked (32.5.6) by this call, if any such atomic waiting operations exist.

24 *Remarks:* This function is an atomic notifying operation (32.5.6).

```
constexpr void notify_all() noexcept;
```

25 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked (32.5.6) by this call.

26 *Remarks:* This function is an atomic notifying operation (32.5.6).

### 32.5.8.7.3 Partial specialization for `weak_ptr`

[`util.smartptr.atomic.weak`]

```
namespace std {
 template<class T> struct atomic<weak_ptr<T>> {
 using value_type = weak_ptr<T>;

 static constexpr bool is_always_lock_free = implementation-defined;
 bool is_lock_free() const noexcept;

 constexpr atomic() noexcept;
 constexpr atomic(weak_ptr<T> desired) noexcept;
 atomic(const atomic&) = delete;
 void operator=(const atomic&) = delete;

 constexpr weak_ptr<T> load(memory_order order = memory_order::seq_cst) const noexcept;
 constexpr operator weak_ptr<T>() const noexcept;
 constexpr void store(weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr void operator=(weak_ptr<T> desired) noexcept;

 constexpr weak_ptr<T> exchange(weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
 constexpr bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
 constexpr bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
 constexpr bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;

 constexpr void wait(weak_ptr<T> old,
 memory_order order = memory_order::seq_cst) const noexcept;
 constexpr void notify_one() noexcept;
 constexpr void notify_all() noexcept;
 };
}
```

```
private:
 weak_ptr<T> p; // exposition only
};
}
```

```
constexpr atomic() noexcept;
```

1     *Effects:* Value-initializes *p*.

```
constexpr atomic(weak_ptr<T> desired) noexcept;
```

2     *Effects:* Initializes the object with the value *desired*. Initialization is not an atomic operation (6.10.2).

[*Note 1:* It is possible to have an access to an atomic object *A* race with its construction, for example, by communicating the address of the just-constructed object *A* to another thread via `memory_order::relaxed` operations on a suitable atomic pointer variable, and then immediately accessing *A* in the receiving thread. This results in undefined behavior. — *end note*]

```
constexpr void store(weak_ptr<T> desired, memory_order order = memory_order::seq_cst) noexcept;
```

3     *Preconditions:* *order* is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

4     *Effects:* Atomically replaces the value pointed to by *this* with the value of *desired* as if by *p.swap(desired)*. Memory is affected according to the value of *order*.

```
constexpr void operator=(weak_ptr<T> desired) noexcept;
```

5     *Effects:* Equivalent to `store(desired)`.

```
constexpr weak_ptr<T> load(memory_order order = memory_order::seq_cst) const noexcept;
```

6     *Preconditions:* *order* is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

7     *Effects:* Memory is affected according to the value of *order*.

8     *Returns:* Atomically returns *p*.

```
constexpr operator weak_ptr<T>() const noexcept;
```

9     *Effects:* Equivalent to: `return load();`

```
constexpr weak_ptr<T> exchange(weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

10     *Effects:* Atomically replaces *p* with *desired* as if by *p.swap(desired)*. Memory is affected according to the value of *order*. This is an atomic read-modify-write operation (6.10.2.2).

11     *Returns:* Atomically returns the value of *p* immediately before the effects.

```
constexpr bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
```

```
constexpr bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order success, memory_order failure) noexcept;
```

12     *Preconditions:* *failure* is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

13     *Effects:* If *p* is equivalent to *expected*, assigns *desired* to *p* and has synchronization semantics corresponding to the value of *success*, otherwise assigns *p* to *expected* and has synchronization semantics corresponding to the value of *failure*.

14     *Returns:* `true` if *p* was equivalent to *expected*, `false` otherwise.

15     *Remarks:* Two `weak_ptr` objects are equivalent if they store the same pointer value and either share ownership or are both empty. The weak form may fail spuriously. See 32.5.8.2.

16     If the operation returns `true`, *expected* is not accessed after the atomic update and the operation is an atomic read-modify-write operation (6.10.2) on the memory pointed to by *this*. Otherwise, the operation is an atomic load operation on that memory, and *expected* is updated with the existing value read from the atomic object in the attempted atomic update. The *use\_count* update corresponding to

the write to `expected` is part of the atomic operation. The write to `expected` itself is not required to be part of the atomic operation.

```
constexpr bool compare_exchange_weak(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

17 *Effects:* Equivalent to:

```
return compare_exchange_weak(expected, desired, order, fail_order);
```

where `fail_order` is the same as `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`.

```
constexpr bool compare_exchange_strong(weak_ptr<T>& expected, weak_ptr<T> desired,
 memory_order order = memory_order::seq_cst) noexcept;
```

18 *Effects:* Equivalent to:

```
return compare_exchange_strong(expected, desired, order, fail_order);
```

where `fail_order` is the same as `order` except that a value of `memory_order::acq_rel` shall be replaced by the value `memory_order::acquire` and a value of `memory_order::release` shall be replaced by the value `memory_order::relaxed`.

```
constexpr void wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst) const noexcept;
```

19 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

20 *Effects:* Repeatedly performs the following steps, in order:

(20.1) — Evaluates `load(order)` and compares it to `old`.

(20.2) — If the two are not equivalent, returns.

(20.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

21 *Remarks:* Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation (32.5.6).

```
constexpr void notify_one() noexcept;
```

22 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked (32.5.6) by this call, if any such atomic waiting operations exist.

23 *Remarks:* This function is an atomic notifying operation (32.5.6).

```
constexpr void notify_all() noexcept;
```

24 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked (32.5.6) by this call.

25 *Remarks:* This function is an atomic notifying operation (32.5.6).

### 32.5.9 Non-member functions

[atomics.nonmembers]

<sup>1</sup> A non-member function template whose name matches the pattern `atomic_f` or the pattern `atomic_f_explicit` invokes the member function `f`, with the value of the first parameter as the object expression and the values of the remaining parameters (if any) as the arguments of the member function call, in order. An argument for a parameter of type `atomic<T>::value_type*` is dereferenced when passed to the member function call. If no such member function exists, the program is ill-formed.

<sup>2</sup> [Note 1: The non-member functions enable programmers to write code that can be compiled as either C or C++, for example in a shared header file. — end note]

### 32.5.10 Flag type and operations

[atomics.flag]

```
namespace std {
 struct atomic_flag {
 constexpr atomic_flag() noexcept;
 atomic_flag(const atomic_flag&) = delete;
 atomic_flag& operator=(const atomic_flag&) = delete;
 atomic_flag& operator=(const atomic_flag&) volatile = delete;
```

```

bool test(memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr bool test(memory_order = memory_order::seq_cst) const noexcept;
bool test_and_set(memory_order = memory_order::seq_cst) volatile noexcept;
constexpr bool test_and_set(memory_order = memory_order::seq_cst) noexcept;
void clear(memory_order = memory_order::seq_cst) volatile noexcept;
constexpr void clear(memory_order = memory_order::seq_cst) noexcept;

void wait(bool, memory_order = memory_order::seq_cst) const volatile noexcept;
constexpr void wait(bool, memory_order = memory_order::seq_cst) const noexcept;
void notify_one() volatile noexcept;
constexpr void notify_one() noexcept;
void notify_all() volatile noexcept;
constexpr void notify_all() noexcept;
};
}

```

- 1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.
- 2 Operations on an object of type `atomic_flag` shall be lock-free. The operations should also be address-free.
- 3 The `atomic_flag` type is a standard-layout struct. It has a trivial destructor.

```
constexpr atomic_flag::atomic_flag() noexcept;
```

- 4 *Effects:* Initializes `*this` to the clear state.

```

bool atomic_flag_test(const volatile atomic_flag* object) noexcept;
constexpr bool atomic_flag_test(const atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(const volatile atomic_flag* object,
 memory_order order) noexcept;
constexpr bool atomic_flag_test_explicit(const atomic_flag* object,
 memory_order order) noexcept;
bool atomic_flag::test(memory_order order = memory_order::seq_cst) const volatile noexcept;
constexpr bool atomic_flag::test(memory_order order = memory_order::seq_cst) const noexcept;

```

- 5 For `atomic_flag_test`, let `order` be `memory_order::seq_cst`.
- 6 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.
- 7 *Effects:* Memory is affected according to the value of `order`.
- 8 *Returns:* Atomically returns the value pointed to by `object` or `this`.

```

bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
constexpr bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object, memory_order order) noexcept;
constexpr bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order order) noexcept;
bool atomic_flag::test_and_set(memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr bool atomic_flag::test_and_set(memory_order order = memory_order::seq_cst) noexcept;

```

- 9 *Effects:* Atomically sets the value pointed to by `object` or by `this` to true. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (6.10.2).

- 10 *Returns:* Atomically, the value of the object immediately before the effects.

```

void atomic_flag_clear(volatile atomic_flag* object) noexcept;
constexpr void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object, memory_order order) noexcept;
constexpr void atomic_flag_clear_explicit(atomic_flag* object, memory_order order) noexcept;
void atomic_flag::clear(memory_order order = memory_order::seq_cst) volatile noexcept;
constexpr void atomic_flag::clear(memory_order order = memory_order::seq_cst) noexcept;

```

- 11 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::release`, or `memory_order::seq_cst`.

- 12 *Effects:* Atomically sets the value pointed to by `object` or by `this` to false. Memory is affected according to the value of `order`.

```

void atomic_flag_wait(const volatile atomic_flag* object, bool old) noexcept;
constexpr void atomic_flag_wait(const atomic_flag* object, bool old) noexcept;

```

```

void atomic_flag_wait_explicit(const volatile atomic_flag* object,
 bool old, memory_order order) noexcept;
constexpr void atomic_flag_wait_explicit(const atomic_flag* object,
 bool old, memory_order order) noexcept;
void atomic_flag::wait(bool old, memory_order order =
 memory_order::seq_cst) const volatile noexcept;
constexpr void atomic_flag::wait(bool old, memory_order order =
 memory_order::seq_cst) const noexcept;

```

13 For `atomic_flag_wait`, let `order` be `memory_order::seq_cst`. Let `flag` be `object` for the non-member functions and `this` for the member functions.

14 *Preconditions:* `order` is `memory_order::relaxed`, `memory_order::acquire`, or `memory_order::seq_cst`.

15 *Effects:* Repeatedly performs the following steps, in order:

- (15.1) — Evaluates `flag->test(order) != old`.
- (15.2) — If the result of that evaluation is `true`, returns.
- (15.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

16 *Remarks:* This function is an atomic waiting operation (32.5.6).

```

void atomic_flag_notify_one(volatile atomic_flag* object) noexcept;
constexpr void atomic_flag_notify_one(atomic_flag* object) noexcept;
void atomic_flag::notify_one() volatile noexcept;
constexpr void atomic_flag::notify_one() noexcept;

```

17 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked (32.5.6) by this call, if any such atomic waiting operations exist.

18 *Remarks:* This function is an atomic notifying operation (32.5.6).

```

void atomic_flag_notify_all(volatile atomic_flag* object) noexcept;
constexpr void atomic_flag_notify_all(atomic_flag* object) noexcept;
void atomic_flag::notify_all() volatile noexcept;
constexpr void atomic_flag::notify_all() noexcept;

```

19 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked (32.5.6) by this call.

20 *Remarks:* This function is an atomic notifying operation (32.5.6).

```
#define ATOMIC_FLAG_INIT see below
```

21 *Remarks:* The macro `ATOMIC_FLAG_INIT` is defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static.

### 32.5.11 Fences

[atomics.fences]

- 1 This subclause introduces synchronization primitives called *fences*. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an *acquire fence*. A fence with release semantics is called a *release fence*.
- 2 A release fence *A* synchronizes with an acquire fence *B* if there exist atomic operations *X* and *Y*, where *Y* is not an atomic modify-write operation (32.5.4), both operating on some atomic object *M*, such that *A* is sequenced before *X*, *X* modifies *M*, *Y* is sequenced before *B*, and *Y* reads the value written by *X* or a value written by any side effect in the hypothetical release sequence *X* would head if it were a release operation.
- 3 A release fence *A* synchronizes with an atomic operation *B* that performs an acquire operation on an atomic object *M* if there exists an atomic operation *X* such that *A* is sequenced before *X*, *X* modifies *M*, and *B* reads the value written by *X* or a value written by any side effect in the hypothetical release sequence *X* would head if it were a release operation.

- <sup>4</sup> An atomic operation *A* that is a release operation on an atomic object *M* synchronizes with an acquire fence *B* if there exists some atomic operation *X* on *M* such that *X* is sequenced before *B* and reads the value written by *A* or a value written by any side effect in the release sequence headed by *A*.

```
extern "C" constexpr void atomic_thread_fence(memory_order order) noexcept;
```

- <sup>5</sup> *Effects:* Depending on the value of *order*, this operation:

- (5.1) — has no effects, if *order* == `memory_order::relaxed`;
- (5.2) — is an acquire fence, if *order* == `memory_order::acquire`;
- (5.3) — is a release fence, if *order* == `memory_order::release`;
- (5.4) — is both an acquire fence and a release fence, if *order* == `memory_order::acq_rel`;
- (5.5) — is a sequentially consistent acquire and release fence, if *order* == `memory_order::seq_cst`.

```
extern "C" constexpr void atomic_signal_fence(memory_order order) noexcept;
```

- <sup>6</sup> *Effects:* Equivalent to `atomic_thread_fence(order)`, except that the resulting ordering constraints are established only between a thread and a signal handler executed in the same thread.

- <sup>7</sup> [*Note 1:* `atomic_signal_fence` can be used to specify the order in which actions performed by the thread become visible to the signal handler. Compiler optimizations and reorderings of loads and stores are inhibited in the same way as with `atomic_thread_fence`, but the hardware fence instructions that `atomic_thread_fence` would have inserted are not emitted. — *end note*]

### 32.5.12 C compatibility

[`stdatomic.h.syn`]

The header `<stdatomic.h>` provides the following definitions:

```
template<class T>
 using std-atomic = std::atomic<T>; // exposition only

#define _Atomic(T) std-atomic<T>

#define ATOMIC_BOOL_LOCK_FREE see below
#define ATOMIC_CHAR_LOCK_FREE see below
#define ATOMIC_CHAR8_T_LOCK_FREE see below
#define ATOMIC_CHAR16_T_LOCK_FREE see below
#define ATOMIC_CHAR32_T_LOCK_FREE see below
#define ATOMIC_WCHAR_T_LOCK_FREE see below
#define ATOMIC_SHORT_LOCK_FREE see below
#define ATOMIC_INT_LOCK_FREE see below
#define ATOMIC_LONG_LOCK_FREE see below
#define ATOMIC_LLONG_LOCK_FREE see below
#define ATOMIC_POINTER_LOCK_FREE see below

using std::memory_order; // see below
using std::memory_order_relaxed; // see below
using std::memory_order_consume; // see below
using std::memory_order_acquire; // see below
using std::memory_order_release; // see below
using std::memory_order_acq_rel; // see below
using std::memory_order_seq_cst; // see below

using std::atomic_flag; // see below

using std::atomic_bool; // see below
using std::atomic_char; // see below
using std::atomic_schar; // see below
using std::atomic_uchar; // see below
using std::atomic_short; // see below
using std::atomic_ushort; // see below
using std::atomic_int; // see below
using std::atomic_uint; // see below
using std::atomic_long; // see below
using std::atomic_ulong; // see below
```

```

using std::atomic_llong; // see below
using std::atomic_ullong; // see below
using std::atomic_char8_t; // see below
using std::atomic_char16_t; // see below
using std::atomic_char32_t; // see below
using std::atomic_wchar_t; // see below
using std::atomic_int8_t; // see below
using std::atomic_uint8_t; // see below
using std::atomic_int16_t; // see below
using std::atomic_uint16_t; // see below
using std::atomic_int32_t; // see below
using std::atomic_uint32_t; // see below
using std::atomic_int64_t; // see below
using std::atomic_uint64_t; // see below
using std::atomic_int_least8_t; // see below
using std::atomic_uint_least8_t; // see below
using std::atomic_int_least16_t; // see below
using std::atomic_uint_least16_t; // see below
using std::atomic_int_least32_t; // see below
using std::atomic_uint_least32_t; // see below
using std::atomic_int_least64_t; // see below
using std::atomic_uint_least64_t; // see below
using std::atomic_int_fast8_t; // see below
using std::atomic_uint_fast8_t; // see below
using std::atomic_int_fast16_t; // see below
using std::atomic_uint_fast16_t; // see below
using std::atomic_int_fast32_t; // see below
using std::atomic_uint_fast32_t; // see below
using std::atomic_int_fast64_t; // see below
using std::atomic_uint_fast64_t; // see below
using std::atomic_intptr_t; // see below
using std::atomic_uintptr_t; // see below
using std::atomic_size_t; // see below
using std::atomic_ptrdiff_t; // see below
using std::atomic_intmax_t; // see below
using std::atomic_uintmax_t; // see below

using std::atomic_is_lock_free; // see below
using std::atomic_load; // see below
using std::atomic_load_explicit; // see below
using std::atomic_store; // see below
using std::atomic_store_explicit; // see below
using std::atomic_exchange; // see below
using std::atomic_exchange_explicit; // see below
using std::atomic_compare_exchange_strong; // see below
using std::atomic_compare_exchange_strong_explicit; // see below
using std::atomic_compare_exchange_weak; // see below
using std::atomic_compare_exchange_weak_explicit; // see below
using std::atomic_fetch_add; // see below
using std::atomic_fetch_add_explicit; // see below
using std::atomic_fetch_sub; // see below
using std::atomic_fetch_sub_explicit; // see below
using std::atomic_fetch_and; // see below
using std::atomic_fetch_and_explicit; // see below
using std::atomic_fetch_or; // see below
using std::atomic_fetch_or_explicit; // see below
using std::atomic_fetch_xor; // see below
using std::atomic_fetch_xor_explicit; // see below
using std::atomic_flag_test_and_set; // see below
using std::atomic_flag_test_and_set_explicit; // see below
using std::atomic_flag_clear; // see below
using std::atomic_flag_clear_explicit; // see below
#define ATOMIC_FLAG_INIT see below

```

```
using std::atomic_thread_fence; // see below
using std::atomic_signal_fence; // see below
```

- <sup>1</sup> Each *using-declaration* for some name *A* in the synopsis above makes available the same entity as `std::A` declared in `<atomic>` (32.5.2). Each macro listed above other than `_Atomic(T)` is defined as in `<atomic>`. It is unspecified whether `<stdatomic.h>` makes available any declarations in namespace `std`.
- <sup>2</sup> Each of the *using-declarations* for `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` listed above is declared if and only if the implementation declares the corresponding *typedef-name* in 32.5.2.
- <sup>3</sup> Neither the `_Atomic` macro, nor any of the non-macro global namespace declarations, are provided by any C++ standard library header other than `<stdatomic.h>`.
- <sup>4</sup> *Recommended practice*: Implementations should ensure that C and C++ representations of atomic objects are compatible, so that the same object can be accessed as both an `_Atomic(T)` from C code and an `atomic<T>` from C++ code. The representations should be the same, and the mechanisms used to ensure atomicity and memory ordering should be compatible.

## 32.6 Mutual exclusion

[thread.mutex]

### 32.6.1 General

[thread.mutex.general]

- <sup>1</sup> Subclause 32.6 provides mechanisms for mutual exclusion: mutexes, locks, and call once. These mechanisms ease the production of race-free programs (6.10.2).

### 32.6.2 Header `<mutex>` synopsis

[mutex.syn]

```
namespace std {
 // 32.6.4.2.2, class mutex
 class mutex;
 // 32.6.4.2.3, class recursive_mutex
 class recursive_mutex;
 // 32.6.4.3.2, class timed_mutex
 class timed_mutex;
 // 32.6.4.3.3, class recursive_timed_mutex
 class recursive_timed_mutex;

 struct defer_lock_t { explicit defer_lock_t() = default; };
 struct try_to_lock_t { explicit try_to_lock_t() = default; };
 struct adopt_lock_t { explicit adopt_lock_t() = default; };

 inline constexpr defer_lock_t defer_lock { };
 inline constexpr try_to_lock_t try_to_lock { };
 inline constexpr adopt_lock_t adopt_lock { };

 // 32.6.5, locks
 template<class Mutex> class lock_guard;
 template<class... MutexTypes> class scoped_lock;
 template<class Mutex> class unique_lock;

 template<class Mutex>
 void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y) noexcept;

 // 32.6.6, generic locking algorithms
 template<class L1, class L2, class... L3> int try_lock(L1&, L2&, L3&...);
 template<class L1, class L2, class... L3> void lock(L1&, L2&, L3&...);

 struct once_flag;

 template<class Callable, class... Args>
 void call_once(once_flag& flag, Callable&& func, Args&&... args);
}
```

### 32.6.3 Header `<shared_mutex>` synopsis

[shared.mutex.syn]

```
namespace std {
 // 32.6.4.4.2, class shared_mutex
 class shared_mutex;
```



```

// 32.6.4.5.2, class shared_timed_mutex
class shared_timed_mutex;
// 32.6.5.5, class template shared_lock
template<class Mutex> class shared_lock;
template<class Mutex>
 void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y) noexcept;
}

```

### 32.6.4 Mutex requirements

[thread.mutex.requirements]

#### 32.6.4.1 General

[thread.mutex.requirements.general]

- <sup>1</sup> A mutex object facilitates protection against data races and allows safe synchronization of data between execution agents (32.2.5). An execution agent *owns* a mutex from the time it successfully calls one of the lock functions until it calls unlock. Mutexes can be either recursive or non-recursive, and can grant simultaneous ownership to one or many execution agents. Both recursive and non-recursive mutexes are supplied.

#### 32.6.4.2 Mutex types

[thread.mutex.requirements.mutex]

##### 32.6.4.2.1 General

[thread.mutex.requirements.mutex.general]

- <sup>1</sup> The *mutex types* are the standard library types `mutex`, `recursive_mutex`, `timed_mutex`, `recursive_timed_mutex`, `shared_mutex`, and `shared_timed_mutex`. They meet the requirements set out in 32.6.4.2. In this description, `m` denotes an object of a mutex type.

[Note 1: The mutex types meet the *Cpp17Lockable* requirements (32.2.5.3). — end note]

- <sup>2</sup> The mutex types meet *Cpp17DefaultConstructible* and *Cpp17Destructible*. If initialization of an object of a mutex type fails, an exception of type `system_error` is thrown. The mutex types are neither copyable nor movable.

- <sup>3</sup> The error conditions for error codes, if any, reported by member functions of the mutex types are as follows:

- (3.1) — `resource_unavailable_try_again` — if any native handle type manipulated is not available.
- (3.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.
- (3.3) — `invalid_argument` — if any native handle type manipulated as part of mutex construction is incorrect.

- <sup>4</sup> The implementation provides lock and unlock operations, as described below. For purposes of determining the existence of a data race, these behave as atomic operations (6.10.2). The lock and unlock operations on a single mutex appear to occur in a single total order.

[Note 2: This can be viewed as the modification order (6.10.2) of the mutex. — end note]

[Note 3: Construction and destruction of an object of a mutex type need not be thread-safe; other synchronization can be used to ensure that mutex objects are initialized and visible to other threads. — end note]

- <sup>5</sup> The expression `m.lock()` is well-formed and has the following semantics:

<sup>6</sup> *Preconditions:* If `m` is of type `mutex`, `timed_mutex`, `shared_mutex`, or `shared_timed_mutex`, the calling thread does not own the mutex.

<sup>7</sup> *Effects:* Blocks the calling thread until ownership of the mutex can be obtained for the calling thread.

<sup>8</sup> *Synchronization:* Prior `unlock()` operations on the same object *synchronize with* (6.10.2) this operation.

<sup>9</sup> *Postconditions:* The calling thread owns the mutex.

<sup>10</sup> *Return type:* `void`.

<sup>11</sup> *Throws:* `system_error` when an exception is required (32.2.2).

<sup>12</sup> *Error conditions:*

- (12.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.
- (12.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

- <sup>13</sup> The expression `m.try_lock()` is well-formed and has the following semantics:

<sup>14</sup> *Preconditions:* If `m` is of type `mutex`, `timed_mutex`, `shared_mutex`, or `shared_timed_mutex`, the calling thread does not own the mutex.

<sup>15</sup> *Effects:* Attempts to obtain ownership of the mutex for the calling thread without blocking. If ownership is not obtained, there is no effect and `try_lock()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread.

[*Note 4*: This spurious failure is normally uncommon, but allows interesting implementations based on a simple compare and exchange (32.5). — *end note*]

An implementation should ensure that `try_lock()` does not consistently return `false` in the absence of contending mutex acquisitions.

16 *Synchronization*: If `try_lock()` returns `true`, prior `unlock()` operations on the same object *synchronize with* (6.10.2) this operation.

[*Note 5*: Since `lock()` does not synchronize with a failed subsequent `try_lock()`, the visibility rules are weak enough that little would be known about the state after a failure, even in the absence of spurious failures. — *end note*]

17 *Return type*: `bool`.

18 *Returns*: `true` if ownership was obtained, otherwise `false`.

19 *Throws*: Nothing.

20 The expression `m.unlock()` is well-formed and has the following semantics:

21 *Preconditions*: The calling thread owns the mutex.

22 *Effects*: Releases the calling thread's ownership of the mutex.

23 *Return type*: `void`.

24 *Synchronization*: This operation synchronizes with (6.10.2) subsequent lock operations that obtain ownership on the same object.

25 *Throws*: Nothing.

#### 32.6.4.2.2 Class `mutex`

[`thread.mutex.class`]

```
namespace std {
 class mutex {
 public:
 constexpr mutex() noexcept;
 ~mutex();

 mutex(const mutex&) = delete;
 mutex& operator=(const mutex&) = delete;

 void lock();
 bool try_lock();
 void unlock();

 using native_handle_type = implementation-defined; // see 32.2.3
 native_handle_type native_handle(); // see 32.2.3
 };
}
```

1 The class `mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a mutex object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the owning thread has released ownership with a call to `unlock()`.

2 [*Note 1*: After a thread A has called `unlock()`, releasing a mutex, it is possible for another thread B to lock the same mutex, observe that it is no longer in use, unlock it, and destroy it, before thread A appears to have returned from its unlock call. Conforming implementations handle such scenarios correctly, as long as thread A does not access the mutex after the unlock call returns. These cases typically occur when a reference-counted object contains a mutex that is used to protect the reference count. — *end note*]

3 The class `mutex` meets all of the mutex requirements (32.6.4). It is a standard-layout class (11.2).

4 [*Note 2*: A program can deadlock if the thread that owns a `mutex` object calls `lock()` on that object. If the implementation can detect the deadlock, a `resource_deadlock_would_occur` error condition might be observed. — *end note*]

5 The behavior of a program is undefined if it destroys a `mutex` object owned by any thread or a thread terminates while owning a `mutex` object.

**32.6.4.2.3 Class recursive\_mutex****[thread.mutex.recursive]**

```

namespace std {
 class recursive_mutex {
 public:
 recursive_mutex();
 ~recursive_mutex();

 recursive_mutex(const recursive_mutex&) = delete;
 recursive_mutex& operator=(const recursive_mutex&) = delete;

 void lock();
 bool try_lock() noexcept;
 void unlock();

 using native_handle_type = implementation-defined; // see 32.2.3
 native_handle_type native_handle(); // see 32.2.3
 };
}

```

- <sup>1</sup> The class `recursive_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the first thread has completely released ownership.
- <sup>2</sup> The class `recursive_mutex` meets all of the mutex requirements (32.6.4). It is a standard-layout class (11.2).
- <sup>3</sup> A thread that owns a `recursive_mutex` object may acquire additional levels of ownership by calling `lock()` or `try_lock()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_mutex` object, additional calls to `try_lock()` fail, and additional calls to `lock()` throw an exception of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()` and `try_lock()`. Only when all levels of ownership have been released may ownership be acquired by another thread.
- <sup>4</sup> The behavior of a program is undefined if
  - (4.1) — it destroys a `recursive_mutex` object owned by any thread or
  - (4.2) — a thread terminates while owning a `recursive_mutex` object.

**32.6.4.3 Timed mutex types****[thread.timedmutex.requirements]****32.6.4.3.1 General****[thread.timedmutex.requirements.general]**

- <sup>1</sup> The *timed mutex types* are the standard library types `timed_mutex`, `recursive_timed_mutex`, and `shared_timed_mutex`. They meet the requirements set out below. In this description, `m` denotes an object of a mutex type, `rel_time` denotes an object of a specialization of `duration` (30.5), and `abs_time` denotes an object of a specialization of `time_point` (30.6).

[Note 1: The timed mutex types meet the *Cpp17TimedLockable* requirements (32.2.5.4). — end note]

- <sup>2</sup> The expression `m.try_lock_for(rel_time)` is well-formed and has the following semantics:
  - <sup>3</sup> *Preconditions:* If `m` is of type `timed_mutex` or `shared_timed_mutex`, the calling thread does not own the mutex.
  - <sup>4</sup> *Effects:* The function attempts to obtain ownership of the mutex within the relative timeout (32.2.4) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function returns within the timeout specified by `rel_time` only if it has obtained ownership of the mutex object.
 

[Note 2: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — end note]
  - <sup>5</sup> *Synchronization:* If `try_lock_for()` returns `true`, prior `unlock()` operations on the same object *synchronize with* (6.10.2) this operation.
  - <sup>6</sup> *Return type:* `bool`.
  - <sup>7</sup> *Returns:* `true` if ownership was obtained, otherwise `false`.
  - <sup>8</sup> *Throws:* Timeout-related exceptions (32.2.4).

9 The expression `m.try_lock_until(abs_time)` is well-formed and has the following semantics:

10 *Preconditions:* If `m` is of type `timed_mutex` or `shared_timed_mutex`, the calling thread does not own the mutex.

11 *Effects:* The function attempts to obtain ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function returns before the absolute timeout (32.2.4) specified by `abs_time` only if it has obtained ownership of the mutex object.

[Note 3: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — end note]

12 *Synchronization:* If `try_lock_until()` returns `true`, prior `unlock()` operations on the same object synchronize with (6.10.2) this operation.

13 *Return type:* `bool`.

14 *Returns:* `true` if ownership was obtained, otherwise `false`.

15 *Throws:* Timeout-related exceptions (32.2.4).

#### 32.6.4.3.2 Class `timed_mutex`

[`thread.timedmutex.class`]

```
namespace std {
 class timed_mutex {
 public:
 timed_mutex();
 ~timed_mutex();

 timed_mutex(const timed_mutex&) = delete;
 timed_mutex& operator=(const timed_mutex&) = delete;

 void lock(); // blocking
 bool try_lock();
 template<class Rep, class Period>
 bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
 template<class Clock, class Duration>
 bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
 void unlock();

 using native_handle_type = implementation-defined; // see 32.2.3
 native_handle_type native_handle(); // see 32.2.3
 };
}
```

1 The class `timed_mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a `timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has released ownership with a call to `unlock()` or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).

2 The class `timed_mutex` meets all of the timed mutex requirements (32.6.4.3). It is a standard-layout class (11.2).

3 The behavior of a program is undefined if

- (3.1) — it destroys a `timed_mutex` object owned by any thread,
- (3.2) — a thread that owns a `timed_mutex` object calls `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object, or
- (3.3) — a thread terminates while owning a `timed_mutex` object.

#### 32.6.4.3.3 Class `recursive_timed_mutex`

[`thread.timedmutex.recursive`]

```
namespace std {
 class recursive_timed_mutex {
 public:
 recursive_timed_mutex();
 ~recursive_timed_mutex();
 };
}
```

```

recursive_timed_mutex(const recursive_timed_mutex&) = delete;
recursive_timed_mutex& operator=(const recursive_timed_mutex&) = delete;

void lock(); // blocking
bool try_lock() noexcept;
template<class Rep, class Period>
 bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
 bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
void unlock();

using native_handle_type = implementation-defined; // see 32.2.3
native_handle_type native_handle(); // see 32.2.3
};
}

```

- <sup>1</sup> The class `recursive_timed_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has completely released ownership or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).
- <sup>2</sup> The class `recursive_timed_mutex` meets all of the timed mutex requirements (32.6.4.3). It is a standard-layout class (11.2).
- <sup>3</sup> A thread that owns a `recursive_timed_mutex` object may acquire additional levels of ownership by calling `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_timed_mutex` object, additional calls to `try_lock()`, `try_lock_for()`, or `try_lock_until()` fail, and additional calls to `lock()` throw an exception of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()`, `try_lock()`, `try_lock_for()`, and `try_lock_until()`. Only when all levels of ownership have been released may ownership of the object be acquired by another thread.
- <sup>4</sup> The behavior of a program is undefined if
  - (4.1) — it destroys a `recursive_timed_mutex` object owned by any thread, or
  - (4.2) — a thread terminates while owning a `recursive_timed_mutex` object.

#### 32.6.4.4 Shared mutex types

[thread.sharedmutex.requirements]

##### 32.6.4.4.1 General

[thread.sharedmutex.requirements.general]

- <sup>1</sup> The standard library types `shared_mutex` and `shared_timed_mutex` are *shared mutex types*. Shared mutex types meet the requirements of mutex types (32.6.4.2) and additionally meet the requirements set out below. In this description, `m` denotes an object of a shared mutex type.

[Note 1: The shared mutex types meet the *Cpp17SharedLockable* requirements (32.2.5.5). — end note]

- <sup>2</sup> In addition to the exclusive lock ownership mode specified in 32.6.4.2, shared mutex types provide a *shared lock* ownership mode. Multiple execution agents can simultaneously hold a shared lock ownership of a shared mutex type. But no execution agent holds a shared lock while another execution agent holds an exclusive lock on the same shared mutex type, and vice-versa. The maximum number of execution agents which can share a shared lock on a single shared mutex type is unspecified, but is at least 10000. If more than the maximum number of execution agents attempt to obtain a shared lock, the excess execution agents block until the number of shared locks are reduced below the maximum amount by other execution agents releasing their shared lock.
- <sup>3</sup> The expression `m.lock_shared()` is well-formed and has the following semantics:
  - <sup>4</sup> *Preconditions:* The calling thread has no ownership of the mutex.
  - <sup>5</sup> *Effects:* Blocks the calling thread until shared ownership of the mutex can be obtained for the calling thread. If an exception is thrown then a shared lock has not been acquired for the current thread.
  - <sup>6</sup> *Synchronization:* Prior `unlock()` operations on the same object synchronize with (6.10.2) this operation.
  - <sup>7</sup> *Postconditions:* The calling thread has a shared lock on the mutex.
  - <sup>8</sup> *Return type:* `void`.

9 *Throws:* **system\_error** when an exception is required (32.2.2).

10 *Error conditions:*

(10.1) — **operation\_not\_permitted** — if the thread does not have the privilege to perform the operation.

(10.2) — **resource\_deadlock\_would\_occur** — if the implementation detects that a deadlock would occur.

11 The expression `m.unlock_shared()` is well-formed and has the following semantics:

12 *Preconditions:* The calling thread holds a shared lock on the mutex.

13 *Effects:* Releases a shared lock on the mutex held by the calling thread.

14 *Return type:* **void**.

15 *Synchronization:* This operation synchronizes with (6.10.2) subsequent `lock()` operations that obtain ownership on the same object.

16 *Throws:* Nothing.

17 The expression `m.try_lock_shared()` is well-formed and has the following semantics:

18 *Preconditions:* The calling thread has no ownership of the mutex.

19 *Effects:* Attempts to obtain shared ownership of the mutex for the calling thread without blocking. If shared ownership is not obtained, there is no effect and `try_lock_shared()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread.

20 *Synchronization:* If `try_lock_shared()` returns **true**, prior `unlock()` operations on the same object synchronize with (6.10.2) this operation.

21 *Return type:* **bool**.

22 *Returns:* **true** if the shared lock was acquired, otherwise **false**.

23 *Throws:* Nothing.

#### 32.6.4.4.2 Class `shared_mutex`

[thread.sharedmutex.class]

```
namespace std {
 class shared_mutex {
 public:
 shared_mutex();
 ~shared_mutex();

 shared_mutex(const shared_mutex&) = delete;
 shared_mutex& operator=(const shared_mutex&) = delete;

 // exclusive ownership
 void lock(); // blocking
 bool try_lock();
 void unlock();

 // shared ownership
 void lock_shared(); // blocking
 bool try_lock_shared();
 void unlock_shared();

 using native_handle_type = implementation-defined; // see 32.2.3
 native_handle_type native_handle(); // see 32.2.3
 };
}
```

1 The class `shared_mutex` provides a non-recursive mutex with shared ownership semantics.

2 The class `shared_mutex` meets all of the shared mutex requirements (32.6.4.4). It is a standard-layout class (11.2).

3 The behavior of a program is undefined if

(3.1) — it destroys a `shared_mutex` object owned by any thread,

(3.2) — a thread attempts to recursively gain any ownership of a `shared_mutex`, or

(3.3) — a thread terminates while possessing any ownership of a `shared_mutex`.



<sup>4</sup> `shared_mutex` may be a synonym for `shared_timed_mutex`.

### 32.6.4.5 Shared timed mutex types [thread.sharedtimedmutex.requirements]

#### 32.6.4.5.1 General [thread.sharedtimedmutex.requirements.general]

<sup>1</sup> The standard library type `shared_timed_mutex` is a *shared timed mutex type*. Shared timed mutex types meet the requirements of timed mutex types (32.6.4.3), shared mutex types (32.6.4.4), and additionally meet the requirements set out below. In this description, `m` denotes an object of a shared timed mutex type, `rel_time` denotes an object of a specialization of `duration` (30.5), and `abs_time` denotes an object of a specialization of `time_point` (30.6).

[Note 1: The shared timed mutex types meet the *Cpp17SharedTimedLockable* requirements (32.2.5.6). — end note]

<sup>2</sup> The expression `m.try_lock_shared_for(rel_time)` is well-formed and has the following semantics:

<sup>3</sup>     *Preconditions:* The calling thread has no ownership of the mutex.

<sup>4</sup>     *Effects:* Attempts to obtain shared lock ownership for the calling thread within the relative timeout (32.2.4) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock_shared()`). The function returns within the timeout specified by `rel_time` only if it has obtained shared ownership of the mutex object.

[Note 2: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — end note]

If an exception is thrown then a shared lock has not been acquired for the current thread.

<sup>5</sup>     *Synchronization:* If `try_lock_shared_for()` returns `true`, prior `unlock()` operations on the same object synchronize with (6.10.2) this operation.

<sup>6</sup>     *Return type:* `bool`.

<sup>7</sup>     *Returns:* `true` if the shared lock was acquired, otherwise `false`.

<sup>8</sup>     *Throws:* Timeout-related exceptions (32.2.4).

<sup>9</sup> The expression `m.try_lock_shared_until(abs_time)` is well-formed and has the following semantics:

<sup>10</sup>     *Preconditions:* The calling thread has no ownership of the mutex.

<sup>11</sup>     *Effects:* The function attempts to obtain shared ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain shared ownership without blocking (as if by calling `try_lock_shared()`). The function returns before the absolute timeout (32.2.4) specified by `abs_time` only if it has obtained shared ownership of the mutex object.

[Note 3: As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — end note]

If an exception is thrown then a shared lock has not been acquired for the current thread.

<sup>12</sup>     *Synchronization:* If `try_lock_shared_until()` returns `true`, prior `unlock()` operations on the same object synchronize with (6.10.2) this operation.

<sup>13</sup>     *Return type:* `bool`.

<sup>14</sup>     *Returns:* `true` if the shared lock was acquired, otherwise `false`.

<sup>15</sup>     *Throws:* Timeout-related exceptions (32.2.4).

#### 32.6.4.5.2 Class `shared_timed_mutex` [thread.sharedtimedmutex.class]

```
namespace std {
 class shared_timed_mutex {
 public:
 shared_timed_mutex();
 ~shared_timed_mutex();

 shared_timed_mutex(const shared_timed_mutex&) = delete;
 shared_timed_mutex& operator=(const shared_timed_mutex&) = delete;

 // exclusive ownership
 void lock(); // blocking
 bool try_lock();
 }
```

```

template<class Rep, class Period>
 bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
 bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
void unlock();

// shared ownership
void lock_shared(); // blocking
bool try_lock_shared();
template<class Rep, class Period>
 bool try_lock_shared_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
 bool try_lock_shared_until(const chrono::time_point<Clock, Duration>& abs_time);
void unlock_shared();
};
}

```

- <sup>1</sup> The class `shared_timed_mutex` provides a non-recursive mutex with shared ownership semantics.
- <sup>2</sup> The class `shared_timed_mutex` meets all of the shared timed mutex requirements (32.6.4.5). It is a standard-layout class (11.2).
- <sup>3</sup> The behavior of a program is undefined if
  - (3.1) — it destroys a `shared_timed_mutex` object owned by any thread,
  - (3.2) — a thread attempts to recursively gain any ownership of a `shared_timed_mutex`, or
  - (3.3) — a thread terminates while possessing any ownership of a `shared_timed_mutex`.

### 32.6.5 Locks

[thread.lock]

#### 32.6.5.1 General

[thread.lock.general]

- <sup>1</sup> A *lock* is an object that holds a reference to a lockable object and may unlock the lockable object during the lock's destruction (such as when leaving block scope). An execution agent may use a lock to aid in managing ownership of a lockable object in an exception safe manner. A lock is said to *own* a lockable object if it is currently managing the ownership of that lockable object for an execution agent. A lock does not manage the lifetime of the lockable object it references.

[Note 1: Locks are intended to ease the burden of unlocking the lockable object under both normal and exceptional circumstances. — end note]

- <sup>2</sup> Some lock constructors take tag types which describe what should be done with the lockable object during the lock's construction.

```

namespace std {
 struct defer_lock_t { }; // do not acquire ownership of the mutex
 struct try_to_lock_t { }; // try to acquire ownership of the mutex
 // without blocking
 struct adopt_lock_t { }; // assume the calling thread has already
 // obtained mutex ownership and manage it

 inline constexpr defer_lock_t defer_lock { };
 inline constexpr try_to_lock_t try_to_lock { };
 inline constexpr adopt_lock_t adopt_lock { };
}

```

#### 32.6.5.2 Class template lock\_guard

[thread.lock.guard]

```

namespace std {
 template<class Mutex>
 class lock_guard {
 public:
 using mutex_type = Mutex;

 explicit lock_guard(mutex_type& m);
 lock_guard(mutex_type& m, adopt_lock_t);
 ~lock_guard();
 };
}

```



```

 lock_guard(const lock_guard&) = delete;
 lock_guard& operator=(const lock_guard&) = delete;

private:
 mutex_type& pm; // exposition only
};
}

```

- <sup>1</sup> An object of type `lock_guard` controls the ownership of a lockable object within a scope. A `lock_guard` object maintains ownership of a lockable object throughout the `lock_guard` object's lifetime (6.8.4). The behavior of a program is undefined if the lockable object referenced by `pm` does not exist for the entire lifetime of the `lock_guard` object. The supplied `Mutex` type shall meet the *Cpp17BasicLockable* requirements (32.2.5.2).

```
explicit lock_guard(mutex_type& m);
```

- <sup>2</sup> *Effects:* Initializes `pm` with `m`. Calls `m.lock()`.

```
lock_guard(mutex_type& m, adopt_lock_t);
```

- <sup>3</sup> *Preconditions:* The calling thread holds a non-shared lock on `m`.

- <sup>4</sup> *Effects:* Initializes `pm` with `m`.

- <sup>5</sup> *Throws:* Nothing.

```
~lock_guard();
```

- <sup>6</sup> *Effects:* Equivalent to: `pm.unlock()`

### 32.6.5.3 Class template `scoped_lock`

[thread.lock.scoped]

```

namespace std {
 template<class... MutexTypes>
 class scoped_lock {
 public:
 using mutex_type = see below; // Only if sizeof...(MutexTypes) == 1 is true

 explicit scoped_lock(MutexTypes&... m);
 explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
 ~scoped_lock();

 scoped_lock(const scoped_lock&) = delete;
 scoped_lock& operator=(const scoped_lock&) = delete;

 private:
 tuple<MutexTypes&...> pm; // exposition only
 };
}

```

- <sup>1</sup> An object of type `scoped_lock` controls the ownership of lockable objects within a scope. A `scoped_lock` object maintains ownership of lockable objects throughout the `scoped_lock` object's lifetime (6.8.4). The behavior of a program is undefined if the lockable objects referenced by `pm` do not exist for the entire lifetime of the `scoped_lock` object.

- (1.1) — If `sizeof...(MutexTypes)` is one, let `Mutex` denote the sole type constituting the pack `MutexTypes`. `Mutex` shall meet the *Cpp17BasicLockable* requirements (32.2.5.2). The member *typedef-name* `mutex_type` denotes the same type as `Mutex`.

- (1.2) — Otherwise, all types in the template parameter pack `MutexTypes` shall meet the *Cpp17Lockable* requirements (32.2.5.3) and there is no member `mutex_type`.

```
explicit scoped_lock(MutexTypes&... m);
```

- <sup>2</sup> *Effects:* Initializes `pm` with `tie(m...)`. Then if `sizeof...(MutexTypes)` is 0, no effects. Otherwise if `sizeof...(MutexTypes)` is 1, then `m.lock()`. Otherwise, `lock(m...)`.

```
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
```

- <sup>3</sup> *Preconditions:* The calling thread holds a non-shared lock on each element of `m`.

- <sup>4</sup> *Effects:* Initializes `pm` with `tie(m...)`.

5 *Throws:* Nothing.

```
~scoped_lock();
```

6 *Effects:* For all *i* in  $[0, \text{sizeof} \dots (\text{MutexTypes}))$ ,  $\text{get}\langle i \rangle(\text{pm})$ .unlock().

### 32.6.5.4 Class template `unique_lock`

[thread.lock.unique]

#### 32.6.5.4.1 General

[thread.lock.unique.general]

```
namespace std {
 template<class Mutex>
 class unique_lock {
 public:
 using mutex_type = Mutex;

 // 32.6.5.4.2, construct/copy/destroy
 unique_lock() noexcept;
 explicit unique_lock(mutex_type& m);
 unique_lock(mutex_type& m, defer_lock_t) noexcept;
 unique_lock(mutex_type& m, try_to_lock_t);
 unique_lock(mutex_type& m, adopt_lock_t);
 template<class Clock, class Duration>
 unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);
 template<class Rep, class Period>
 unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
 ~unique_lock();

 unique_lock(const unique_lock&) = delete;
 unique_lock& operator=(const unique_lock&) = delete;

 unique_lock(unique_lock&& u) noexcept;
 unique_lock& operator=(unique_lock&& u) noexcept;

 // 32.6.5.4.3, locking
 void lock();
 bool try_lock();

 template<class Rep, class Period>
 bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
 template<class Clock, class Duration>
 bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);

 void unlock();

 // 32.6.5.4.4, modifiers
 void swap(unique_lock& u) noexcept;
 mutex_type* release() noexcept;

 // 32.6.5.4.5, observers
 bool owns_lock() const noexcept;
 explicit operator bool() const noexcept;
 mutex_type* mutex() const noexcept;

 private:
 mutex_type* pm; // exposition only
 bool owns; // exposition only
 };
}
```

- <sup>1</sup> An object of type `unique_lock` controls the ownership of a lockable object within a scope. Ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `unique_lock` object. Objects of type `unique_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer *pm* is not null and the lockable object pointed to by *pm* does not exist for the entire remaining lifetime (6.8.4) of the `unique_lock` object. The supplied `Mutex` type shall meet the *Cpp17BasicLockable* requirements (32.2.5.2).

<sup>2</sup> [Note 1: `unique_lock<Mutex>` meets the *Cpp17BasicLockable* requirements. If `Mutex` meets the *Cpp17Lockable* requirements (32.2.5.3), `unique_lock<Mutex>` also meets the *Cpp17Lockable* requirements; if `Mutex` meets the *Cpp17TimedLockable* requirements (32.2.5.4), `unique_lock<Mutex>` also meets the *Cpp17TimedLockable* requirements. — end note]

#### 32.6.5.4.2 Constructors, destructor, and assignment

[thread.lock.unique.cons]

```
unique_lock() noexcept;

1 Postconditions: pm == nullptr and owns == false.

explicit unique_lock(mutex_type& m);

2 Effects: Calls m.lock().

3 Postconditions: pm == addressof(m) and owns == true.

unique_lock(mutex_type& m, defer_lock_t) noexcept;

4 Postconditions: pm == addressof(m) and owns == false.

unique_lock(mutex_type& m, try_to_lock_t);

5 Preconditions: The supplied Mutex type meets the Cpp17Lockable requirements (32.2.5.3).

6 Effects: Calls m.try_lock().

7 Postconditions: pm == addressof(m) and owns == res, where res is the value returned by the
 call to m.try_lock().

unique_lock(mutex_type& m, adopt_lock_t);

8 Preconditions: The calling thread holds a non-shared lock on m.

9 Postconditions: pm == addressof(m) and owns == true.

10 Throws: Nothing.

template<class Clock, class Duration>
 unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);

11 Preconditions: The supplied Mutex type meets the Cpp17TimedLockable requirements (32.2.5.4).

12 Effects: Calls m.try_lock_until(abs_time).

13 Postconditions: pm == addressof(m) and owns == res, where res is the value returned by the call
 to m.try_lock_until(abs_time).

template<class Rep, class Period>
 unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);

14 Preconditions: The supplied Mutex type meets the Cpp17TimedLockable requirements (32.2.5.4).

15 Effects: Calls m.try_lock_for(rel_time).

16 Postconditions: pm == addressof(m) and owns == res, where res is the value returned by the call
 to m.try_lock_for(rel_time).

unique_lock(unique_lock&& u) noexcept;

17 Postconditions: pm == u.p.pm and owns == u.p.owns (where u.p is the state of u just prior to this
 construction), u.pm == 0 and u.owns == false.

unique_lock& operator=(unique_lock&& u) noexcept;

18 Effects: Equivalent to: unique_lock(std::move(u)).swap(*this)

19 Returns: *this.

~unique_lock();

20 Effects: If owns calls pm->unlock().
```

## 32.6.5.4.3 Locking

[thread.lock.unique.locking]

```
void lock();
```

1     *Effects:* As if by `pm->lock()`.

2     *Postconditions:* `owns == true`.

3     *Throws:* Any exception thrown by `pm->lock()`. `system_error` when an exception is required (32.2.2).

4     *Error conditions:*

(4.1)     — `operation_not_permitted` — if `pm` is nullptr.

(4.2)     — `resource_deadlock_would_occur` — if on entry `owns` is true.

```
bool try_lock();
```

5     *Preconditions:* The supplied Mutex meets the *Cpp17Lockable* requirements (32.2.5.3).

6     *Effects:* As if by `pm->try_lock()`.

7     *Postconditions:* `owns == res`, where `res` is the value returned by `pm->try_lock()`.

8     *Returns:* The value returned by `pm->try_lock()`.

9     *Throws:* Any exception thrown by `pm->try_lock()`. `system_error` when an exception is required (32.2.2).

10     *Error conditions:*

(10.1)     — `operation_not_permitted` — if `pm` is nullptr.

(10.2)     — `resource_deadlock_would_occur` — if on entry `owns` is true.

```
template<class Clock, class Duration>
```

```
bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
```

11     *Preconditions:* The supplied Mutex type meets the *Cpp17TimedLockable* requirements (32.2.5.4).

12     *Effects:* As if by `pm->try_lock_until(abs_time)`.

13     *Postconditions:* `owns == res`, where `res` is the value returned by `pm->try_lock_until(abs_time)`.

14     *Returns:* The value returned by `pm->try_lock_until(abs_time)`.

15     *Throws:* Any exception thrown by `pm->try_lock_until(abs_time)`. `system_error` when an exception is required (32.2.2).

16     *Error conditions:*

(16.1)     — `operation_not_permitted` — if `pm` is nullptr.

(16.2)     — `resource_deadlock_would_occur` — if on entry `owns` is true.

```
template<class Rep, class Period>
```

```
bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

17     *Preconditions:* The supplied Mutex type meets the *Cpp17TimedLockable* requirements (32.2.5.4).

18     *Effects:* As if by `pm->try_lock_for(rel_time)`.

19     *Postconditions:* `owns == res`, where `res` is the value returned by `pm->try_lock_for(rel_time)`.

20     *Returns:* The value returned by `pm->try_lock_for(rel_time)`.

21     *Throws:* Any exception thrown by `pm->try_lock_for(rel_time)`. `system_error` when an exception is required (32.2.2).

22     *Error conditions:*

(22.1)     — `operation_not_permitted` — if `pm` is nullptr.

(22.2)     — `resource_deadlock_would_occur` — if on entry `owns` is true.

```
void unlock();
```

23     *Effects:* As if by `pm->unlock()`.

24     *Postconditions:* `owns == false`.

25     *Throws:* `system_error` when an exception is required (32.2.2).

26 *Error conditions:*

(26.1) — `operation_not_permitted` — if on entry *owns* is false.

#### 32.6.5.4.4 Modifiers

[thread.lock.unique.mod]

```
void swap(unique_lock& u) noexcept;
```

1 *Effects:* Swaps the data members of *\*this* and *u*.

```
mutex_type* release() noexcept;
```

2 *Postconditions:* *pm* == 0 and *owns* == false.

3 *Returns:* The previous value of *pm*.

```
template<class Mutex>
```

```
void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y) noexcept;
```

4 *Effects:* As if by *x.swap(y)*.

#### 32.6.5.4.5 Observers

[thread.lock.unique.obs]

```
bool owns_lock() const noexcept;
```

1 *Returns:* *owns*.

```
explicit operator bool() const noexcept;
```

2 *Returns:* *owns*.

```
mutex_type *mutex() const noexcept;
```

3 *Returns:* *pm*.

#### 32.6.5.5 Class template `shared_lock`

[thread.lock.shared]

##### 32.6.5.5.1 General

[thread.lock.shared.general]

```
namespace std {
```

```
template<class Mutex>
```

```
class shared_lock {
```

```
public:
```

```
using mutex_type = Mutex;
```

```
// 32.6.5.5.2, construct/copy/destroy
```

```
shared_lock() noexcept;
```

```
explicit shared_lock(mutex_type& m); // blocking
```

```
shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

```
shared_lock(mutex_type& m, try_to_lock_t);
```

```
shared_lock(mutex_type& m, adopt_lock_t);
```

```
template<class Clock, class Duration>
```

```
shared_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);
```

```
template<class Rep, class Period>
```

```
shared_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
```

```
~shared_lock();
```

```
shared_lock(const shared_lock&) = delete;
```

```
shared_lock& operator=(const shared_lock&) = delete;
```

```
shared_lock(shared_lock&& u) noexcept;
```

```
shared_lock& operator=(shared_lock&& u) noexcept;
```

```
// 32.6.5.5.3, locking
```

```
void lock();
```

```
// blocking
```

```
bool try_lock();
```

```
template<class Rep, class Period>
```

```
bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

```
template<class Clock, class Duration>
```

```
bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
```

```
void unlock();
```

```

// 32.6.5.5.4, modifiers
void swap(shared_lock& u) noexcept;
mutex_type* release() noexcept;

// 32.6.5.5.5, observers
bool owns_lock() const noexcept;
explicit operator bool() const noexcept;
mutex_type* mutex() const noexcept;

private:
 mutex_type* pm; // exposition only
 bool owns; // exposition only
};
}

```

<sup>1</sup> An object of type `shared_lock` controls the shared ownership of a lockable object within a scope. Shared ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `shared_lock` object. Objects of type `shared_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime (6.8.4) of the `shared_lock` object. The supplied `Mutex` type shall meet the *Cpp17SharedLockable* requirements (32.2.5.5).

<sup>2</sup> [Note 1: `shared_lock<Mutex>` meets the *Cpp17Lockable* requirements (32.2.5.3). If `Mutex` meets the *Cpp17SharedTimedLockable* requirements (32.2.5.6), `shared_lock<Mutex>` also meets the *Cpp17TimedLockable* requirements (32.2.5.4). — end note]

### 32.6.5.5.2 Constructors, destructor, and assignment

[thread.lock.shared.cons]

```
shared_lock() noexcept;
```

<sup>1</sup> *Postconditions:* `pm == nullptr` and `owns == false`.

```
explicit shared_lock(mutex_type& m);
```

<sup>2</sup> *Effects:* Calls `m.lock_shared()`.

<sup>3</sup> *Postconditions:* `pm == addressof(m)` and `owns == true`.

```
shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

<sup>4</sup> *Postconditions:* `pm == addressof(m)` and `owns == false`.

```
shared_lock(mutex_type& m, try_to_lock_t);
```

<sup>5</sup> *Effects:* Calls `m.try_lock_shared()`.

<sup>6</sup> *Postconditions:* `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared()`.

```
shared_lock(mutex_type& m, adopt_lock_t);
```

<sup>7</sup> *Preconditions:* The calling thread holds a shared lock on `m`.

<sup>8</sup> *Postconditions:* `pm == addressof(m)` and `owns == true`.

```
template<class Clock, class Duration>
 shared_lock(mutex_type& m,
 const chrono::time_point<Clock, Duration>& abs_time);
```

<sup>9</sup> *Preconditions:* `Mutex` meets the *Cpp17SharedTimedLockable* requirements (32.2.5.6).

<sup>10</sup> *Effects:* Calls `m.try_lock_shared_until(abs_time)`.

<sup>11</sup> *Postconditions:* `pm == addressof(m)` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_until(abs_time)`.

```
template<class Rep, class Period>
 shared_lock(mutex_type& m,
 const chrono::duration<Rep, Period>& rel_time);
```

<sup>12</sup> *Preconditions:* `Mutex` meets the *Cpp17SharedTimedLockable* requirements (32.2.5.6).

<sup>13</sup> *Effects:* Calls `m.try_lock_shared_for(rel_time)`.

14 *Postconditions:*  $pm == \text{addressof}(m)$  and  $owns == res$  where  $res$  is the value returned by the call to  $m.\text{try\_lock\_shared\_for}(\text{rel\_time})$ .

$\sim\text{shared\_lock}()$ ;

15 *Effects:* If  $owns$  calls  $pm \rightarrow \text{unlock\_shared}()$ .

$\text{shared\_lock}(\text{shared\_lock}\&\& \text{sl})$  noexcept;

16 *Postconditions:*  $pm == \text{sl\_p}.pm$  and  $owns == \text{sl\_p}.owns$  (where  $\text{sl\_p}$  is the state of  $\text{sl}$  just prior to this construction),  $\text{sl}.pm == \text{nullptr}$  and  $\text{sl}.owns == \text{false}$ .

$\text{shared\_lock}\& \text{operator}=(\text{shared\_lock}\&\& \text{sl})$  noexcept;

17 *Effects:* Equivalent to:  $\text{shared\_lock}(\text{std}::\text{move}(\text{sl})).\text{swap}(*\text{this})$

18 *Returns:*  $*\text{this}$ .

### 32.6.5.5.3 Locking

[thread.lock.shared.locking]

$\text{void lock}()$ ;

1 *Effects:* As if by  $pm \rightarrow \text{lock\_shared}()$ .

2 *Postconditions:*  $owns == \text{true}$ .

3 *Throws:* Any exception thrown by  $pm \rightarrow \text{lock\_shared}()$ . `system_error` when an exception is required (32.2.2).

4 *Error conditions:*

(4.1) — `operation_not_permitted` — if  $pm$  is `nullptr`.

(4.2) — `resource_deadlock_would_occur` — if on entry  $owns$  is `true`.

$\text{bool try\_lock}()$ ;

5 *Effects:* As if by  $pm \rightarrow \text{try\_lock\_shared}()$ .

6 *Postconditions:*  $owns == res$ , where  $res$  is the value returned by the call to  $pm \rightarrow \text{try\_lock\_shared}()$ .

7 *Returns:* The value returned by the call to  $pm \rightarrow \text{try\_lock\_shared}()$ .

8 *Throws:* Any exception thrown by  $pm \rightarrow \text{try\_lock\_shared}()$ . `system_error` when an exception is required (32.2.2).

9 *Error conditions:*

(9.1) — `operation_not_permitted` — if  $pm$  is `nullptr`.

(9.2) — `resource_deadlock_would_occur` — if on entry  $owns$  is `true`.

$\text{template}<\text{class Clock}, \text{class Duration}>$

$\text{bool try\_lock\_until}(\text{const chrono}::\text{time\_point}<\text{Clock}, \text{Duration}>\& \text{abs\_time});$

10 *Preconditions:* Mutex meets the *Cpp17SharedTimedLockable* requirements (32.2.5.6).

11 *Effects:* As if by  $pm \rightarrow \text{try\_lock\_shared\_until}(\text{abs\_time})$ .

12 *Postconditions:*  $owns == res$ , where  $res$  is the value returned by the call to  $pm \rightarrow \text{try\_lock\_shared\_until}(\text{abs\_time})$ .

13 *Returns:* The value returned by the call to  $pm \rightarrow \text{try\_lock\_shared\_until}(\text{abs\_time})$ .

14 *Throws:* Any exception thrown by  $pm \rightarrow \text{try\_lock\_shared\_until}(\text{abs\_time})$ . `system_error` when an exception is required (32.2.2).

15 *Error conditions:*

(15.1) — `operation_not_permitted` — if  $pm$  is `nullptr`.

(15.2) — `resource_deadlock_would_occur` — if on entry  $owns$  is `true`.

$\text{template}<\text{class Rep}, \text{class Period}>$

$\text{bool try\_lock\_for}(\text{const chrono}::\text{duration}<\text{Rep}, \text{Period}>\& \text{rel\_time});$

16 *Preconditions:* Mutex meets the *Cpp17SharedTimedLockable* requirements (32.2.5.6).

17 *Effects:* As if by  $pm \rightarrow \text{try\_lock\_shared\_for}(\text{rel\_time})$ .

18 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_for(rel_time)`.

19 *Returns:* The value returned by the call to `pm->try_lock_shared_for(rel_time)`.

20 *Throws:* Any exception thrown by `pm->try_lock_shared_for(rel_time)`. `system_error` when an exception is required (32.2.2).

21 *Error conditions:*

(21.1) — `operation_not_permitted` — if `pm` is `nullptr`.

(21.2) — `resource_deadlock_would_occur` — if on entry `owns` is `true`.

`void unlock();`

22 *Effects:* As if by `pm->unlock_shared()`.

23 *Postconditions:* `owns == false`.

24 *Throws:* `system_error` when an exception is required (32.2.2).

25 *Error conditions:*

(25.1) — `operation_not_permitted` — if on entry `owns` is `false`.

#### 32.6.5.5.4 Modifiers

[thread.lock.shared.mod]

`void swap(shared_lock& s1) noexcept;`

1 *Effects:* Swaps the data members of `*this` and `s1`.

`mutex_type* release() noexcept;`

2 *Postconditions:* `pm == nullptr` and `owns == false`.

3 *Returns:* The previous value of `pm`.

`template<class Mutex>`  
`void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y) noexcept;`

4 *Effects:* As if by `x.swap(y)`.

#### 32.6.5.5.5 Observers

[thread.lock.shared.obs]

`bool owns_lock() const noexcept;`

1 *Returns:* `owns`.

`explicit operator bool() const noexcept;`

2 *Returns:* `owns`.

`mutex_type* mutex() const noexcept;`

3 *Returns:* `pm`.

### 32.6.6 Generic locking algorithms

[thread.lock.algorithm]

`template<class L1, class L2, class... L3> int try_lock(L1&, L2&, L3&...);`

1 *Preconditions:* Each template parameter type meets the *Cpp17Lockable* requirements.  
 [Note 1: The `unique_lock` class template meets these requirements when suitably instantiated. — end note]

2 *Effects:* Calls `try_lock()` for each argument in order beginning with the first until all arguments have been processed or a call to `try_lock()` fails, either by returning `false` or by throwing an exception. If a call to `try_lock()` fails, `unlock()` is called for all prior arguments with no further calls to `try_lock()`.

3 *Returns:* -1 if all calls to `try_lock()` returned `true`, otherwise a zero-based index value that indicates the argument for which `try_lock()` returned `false`.

`template<class L1, class L2, class... L3> void lock(L1&, L2&, L3&...);`

4 *Preconditions:* Each template parameter type meets the *Cpp17Lockable* requirements.  
 [Note 2: The `unique_lock` class template meets these requirements when suitably instantiated. — end note]



5 *Effects:* All arguments are locked via a sequence of calls to `lock()`, `try_lock()`, or `unlock()` on each argument. The sequence of calls does not result in deadlock, but is otherwise unspecified.

[*Note 3:* A deadlock avoidance algorithm such as try-and-back-off can be used, but the specific algorithm is not specified to avoid over-constraining implementations. — *end note*]

If a call to `lock()` or `try_lock()` throws an exception, `unlock()` is called for any argument that had been locked by a call to `lock()` or `try_lock()`.

### 32.6.7 Call once

[`thread.once`]

#### 32.6.7.1 Struct `once_flag`

[`thread.once.onceflag`]

```
namespace std {
 struct once_flag {
 constexpr once_flag() noexcept;

 once_flag(const once_flag&) = delete;
 once_flag& operator=(const once_flag&) = delete;
 };
}
```

1 The class `once_flag` is an opaque data structure that `call_once` uses to initialize data without causing a data race or deadlock.

```
constexpr once_flag() noexcept;
```

2 *Synchronization:* The construction of a `once_flag` object is not synchronized.

3 *Postconditions:* The object's internal state is set to indicate to an invocation of `call_once` with the object as its initial argument that no function has been called.

#### 32.6.7.2 Function `call_once`

[`thread.once.callonce`]

```
template<class Callable, class... Args>
void call_once(once_flag& flag, Callable&& func, Args&&... args);
```

1 *Mandates:* `is_invocable_v<Callable, Args...>` is true.

2 *Effects:* An execution of `call_once` that does not call its `func` is a *passive* execution. An execution of `call_once` that calls its `func` is an *active* execution. An active execution evaluates `INVOKE(std::forward<Callable>(func), std::forward<Args>(args)...) (22.10.4)`. If such a call to `func` throws an exception the execution is *exceptional*, otherwise it is *returning*. An exceptional execution propagates the exception to the caller of `call_once`. Among all executions of `call_once` for any given `once_flag`: at most one is a returning execution; if there is a returning execution, it is the last active execution; and there are passive executions only if there is a returning execution.

[*Note 1:* Passive executions allow other threads to reliably observe the results produced by the earlier returning execution. — *end note*]

3 *Synchronization:* For any given `once_flag`: all active executions occur in a total order; completion of an active execution synchronizes with (6.10.2) the start of the next one in this total order; and the returning execution synchronizes with the return from all passive executions.

4 *Throws:* `system_error` when an exception is required (32.2.2), or any exception thrown by `func`.

5 [*Example 1:*

```
// global flag, regular function
void init();
std::once_flag flag;

void f() {
 std::call_once(flag, init);
}

// function static flag, function object
struct initializer {
 void operator()();
};
```

```

void g() {
 static std::once_flag flag2;
 std::call_once(flag2, initializer());
}

// object flag, member function
class information {
 std::once_flag verified;
 void verifier();
public:
 void verify() { std::call_once(verified, &information::verifier, *this); }
};
— end example]

```

## 32.7 Condition variables

[thread.condition]

### 32.7.1 General

[thread.condition.general]

- <sup>1</sup> Condition variables provide synchronization primitives used to block a thread until notified by some other thread that some condition is met or until a system time is reached. Class `condition_variable` provides a condition variable that can only wait on an object of type `unique_lock<mutex>`, allowing the implementation to be more efficient. Class `condition_variable_any` provides a general condition variable that can wait on objects of user-supplied lock types.
- <sup>2</sup> Condition variables permit concurrent invocation of the `wait`, `wait_for`, `wait_until`, `notify_one` and `notify_all` member functions.
- <sup>3</sup> The executions of `notify_one` and `notify_all` are atomic. The executions of `wait`, `wait_for`, and `wait_until` are performed in three atomic parts:
  1. the release of the mutex and entry into the waiting state;
  2. the unblocking of the wait; and
  3. the reacquisition of the lock.
- <sup>4</sup> The implementation behaves as if all executions of `notify_one`, `notify_all`, and each part of the `wait`, `wait_for`, and `wait_until` executions are executed in a single unspecified total order consistent with the “happens before” order.
- <sup>5</sup> Condition variable construction and destruction need not be synchronized.
- <sup>6</sup> The definitions in 32.7 make use of the following exposition-only function:

```

template<class Dur>
chrono::steady_clock::time_point rel-to-abs(const Dur& rel_time) {
 return chrono::steady_clock::now() + chrono::ceil<chrono::steady_clock::duration>(rel_time);
}

```

### 32.7.2 Header <condition\_variable> synopsis

[condition.variable.syn]

```

namespace std {
 // 32.7.4, class condition_variable
 class condition_variable;
 // 32.7.5, class condition_variable_any
 class condition_variable_any;

 // 32.7.3, non-member functions
 void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);

 enum class cv_status { no_timeout, timeout };
}

```

### 32.7.3 Non-member functions

[thread.condition.nonmember]

```
void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);
```

- <sup>1</sup> *Preconditions:* `lk` is locked by the calling thread and either
  - (1.1) — no other thread is waiting on `cond`, or

- (1.2) — `lk.mutex()` returns the same value for each of the lock arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.
- 2 *Effects:* Transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits. This notification is sequenced after all objects with thread storage duration associated with the current thread have been destroyed and is equivalent to:
- ```
cond.notify_all();
lk.unlock();
```
- 3 *Synchronization:* The implied `lk.unlock()` call is sequenced after the destruction of all objects with thread storage duration associated with the current thread.
- 4 [Note 1: The supplied lock is held until the thread exits, which might cause deadlock due to lock ordering issues. — *end note*]
- 5 [Note 2: It is the user's responsibility to ensure that waiting threads do not incorrectly assume that the thread has finished if they experience spurious wakeups. This typically requires that the condition being waited for is satisfied while holding the lock on `lk`, and that this lock is not released and reacquired prior to calling `notify_all_at_thread_exit`. — *end note*]

32.7.4 Class `condition_variable`

[`thread.condition.condvar`]

```
namespace std {
    class condition_variable {
    public:
        condition_variable();
        ~condition_variable();

        condition_variable(const condition_variable&) = delete;
        condition_variable& operator=(const condition_variable&) = delete;

        void notify_one() noexcept;
        void notify_all() noexcept;
        void wait(unique_lock<mutex>& lock);
        template<class Predicate>
            void wait(unique_lock<mutex>& lock, Predicate pred);
        template<class Clock, class Duration>
            cv_status wait_until(unique_lock<mutex>& lock,
                                chrono::time_point<Clock, Duration> abs_time);
        template<class Clock, class Duration, class Predicate>
            bool wait_until(unique_lock<mutex>& lock,
                            chrono::time_point<Clock, Duration> abs_time,
                            Predicate pred);
        template<class Rep, class Period>
            cv_status wait_for(unique_lock<mutex>& lock,
                               chrono::duration<Rep, Period> rel_time);
        template<class Rep, class Period, class Predicate>
            bool wait_for(unique_lock<mutex>& lock,
                           chrono::duration<Rep, Period> rel_time,
                           Predicate pred);

        using native_handle_type = implementation-defined;           // see 32.2.3
        native_handle_type native_handle();                          // see 32.2.3
    };
}
```

- 1 The class `condition_variable` is a standard-layout class (11.2).
- ```
condition_variable();
```
- 2 *Throws:* `system_error` when an exception is required (32.2.2).
- 3 *Error conditions:*
- (3.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

```
~condition_variable();
```

4     *Preconditions:* There is no thread blocked on *\*this*.

[*Note 1:* That is, all threads have been notified; they can subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait needs to happen before destruction. Undefined behavior ensues if a thread waits on *\*this* once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

```
void notify_one() noexcept;
```

5     *Effects:* If any threads are blocked waiting for *\*this*, unblocks one of those threads.

```
void notify_all() noexcept;
```

6     *Effects:* Unblocks all threads that are blocked waiting for *\*this*.

```
void wait(unique_lock<mutex>& lock);
```

7     *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(7.1)     — no other thread is waiting on this `condition_variable` object or

(7.2)     — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

8     *Effects:*

(8.1)     — Atomically calls `lock.unlock()` and blocks on *\*this*.

(8.2)     — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(8.3)     — The function will unblock when signaled by a call to `notify_one()` or a call to `notify_all()`, or spuriously.

9     *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

10     *Throws:* Nothing.

11     *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[*Note 2:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Predicate>
```

```
void wait(unique_lock<mutex>& lock, Predicate pred);
```

12     *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(12.1)     — no other thread is waiting on this `condition_variable` object or

(12.2)     — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

13     *Effects:* Equivalent to:

```
while (!pred())
 wait(lock);
```

14     *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

15     *Throws:* Any exception thrown by `pred`.

16     *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[*Note 3:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

```
template<class Clock, class Duration>
```

```
cv_status wait_until(unique_lock<mutex>& lock,
 chrono::time_point<Clock, Duration> abs_time);
```

17     *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(17.1)     — no other thread is waiting on this `condition_variable` object or

(17.2)     — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

18 *Effects:*

- (18.1) — Atomically calls `lock.unlock()` and blocks on `*this`.
- (18.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.
- (18.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout (32.2.4) specified by `abs_time`, or spuriously.
- (18.4) — If the function exits via an exception, `lock.lock()` is called prior to exiting the function.

19 *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

20 *Returns:* `cv_status::timeout` if the absolute timeout (32.2.4) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

21 *Throws:* Timeout-related exceptions (32.2.4).

22 *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[Note 4: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Rep, class Period>
cv_status wait_for(unique_lock<mutex>& lock,
 chrono::duration<Rep, Period> rel_time);
```

23 *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

- (23.1) — no other thread is waiting on this `condition_variable` object or
- (23.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

24 *Effects:* Equivalent to:

```
 return wait_until(lock, rel-to-abs(rel_time));
```

25 *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

26 *Returns:* `cv_status::timeout` if the relative timeout (32.2.4) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

27 *Throws:* Timeout-related exceptions (32.2.4).

28 *Remarks:* If the function fails to meet the postcondition, `terminate` is invoked (14.6.2).

[Note 5: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Clock, class Duration, class Predicate>
bool wait_until(unique_lock<mutex>& lock,
 chrono::time_point<Clock, Duration> abs_time,
 Predicate pred);
```

29 *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

- (29.1) — no other thread is waiting on this `condition_variable` object or
- (29.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

30 *Effects:* Equivalent to:

```
 while (!pred())
 if (wait_until(lock, abs_time) == cv_status::timeout)
 return pred();
 return true;
```

31 *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

32 [Note 6: The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered. — end note]

33 *Throws:* Timeout-related exceptions (32.2.4) or any exception thrown by `pred`.

34 *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[Note 7: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Rep, class Period, class Predicate>
bool wait_for(unique_lock<mutex>& lock,
 chrono::duration<Rep, Period> rel_time,
 Predicate pred);
```

- 35     *Preconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either
- (35.1)     — no other thread is waiting on this `condition_variable` object or
- (35.2)     — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.
- 36     *Effects:* Equivalent to:
- ```
return wait_until(lock, rel-to-abs(rel_time), std::move(pred));
```
- 37 [*Note 8:* There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — *end note*]
- 38 *Postconditions:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.
- 39 [*Note 9:* The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered. — *end note*]
- 40 *Throws:* Timeout-related exceptions (32.2.4) or any exception thrown by `pred`.
- 41 *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).
- [*Note 10:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

32.7.5 Class `condition_variable_any` [thread.condition.condvarany]

32.7.5.1 General [thread.condition.condvarany.general]

- ¹ In 32.7.5, template arguments for template parameters named `Lock` shall meet the *Cpp17BasicLockable* requirements (32.2.5.2).

[*Note 1:* All of the standard mutex types meet this requirement. If a type other than one of the standard mutex types or a `unique_lock` wrapper for a standard mutex type is used with `condition_variable_any`, any necessary synchronization is assumed to be in place with respect to the predicate associated with the `condition_variable_any` instance. — *end note*]

```
namespace std {
    class condition_variable_any {
    public:
        condition_variable_any();
        ~condition_variable_any();

        condition_variable_any(const condition_variable_any&) = delete;
        condition_variable_any& operator=(const condition_variable_any&) = delete;

        void notify_one() noexcept;
        void notify_all() noexcept;

        // 32.7.5.2, noninterruptible waits
        template<class Lock>
            void wait(Lock& lock);
        template<class Lock, class Predicate>
            void wait(Lock& lock, Predicate pred);

        template<class Lock, class Clock, class Duration>
            cv_status wait_until(Lock& lock, chrono::time_point<Clock, Duration> abs_time);
        template<class Lock, class Clock, class Duration, class Predicate>
            bool wait_until(Lock& lock, chrono::time_point<Clock, Duration> abs_time,
                           Predicate pred);
        template<class Lock, class Rep, class Period>
            cv_status wait_for(Lock& lock, chrono::duration<Rep, Period> rel_time);
        template<class Lock, class Rep, class Period, class Predicate>
            bool wait_for(Lock& lock, chrono::duration<Rep, Period> rel_time, Predicate pred);

        // 32.7.5.3, interruptible waits
        template<class Lock, class Predicate>
            bool wait(Lock& lock, stop_token stoken, Predicate pred);
```

```

template<class Lock, class Clock, class Duration, class Predicate>
    bool wait_until(Lock& lock, stop_token stoken,
                    chrono::time_point<Clock, Duration> abs_time, Predicate pred);
template<class Lock, class Rep, class Period, class Predicate>
    bool wait_for(Lock& lock, stop_token stoken,
                  chrono::duration<Rep, Period> rel_time, Predicate pred);
};
}

```

```
condition_variable_any();
```

2 *Throws:* `bad_alloc` or `system_error` when an exception is required (32.2.2).

3 *Error conditions:*

- (3.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.
- (3.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

```
~condition_variable_any();
```

4 *Preconditions:* There is no thread blocked on `*this`.

[*Note 2:* That is, all threads have been notified; they can subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait needs to happen before destruction. Undefined behavior ensues if a thread waits on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

```
void notify_one() noexcept;
```

5 *Effects:* If any threads are blocked waiting for `*this`, unblocks one of those threads.

```
void notify_all() noexcept;
```

6 *Effects:* Unblocks all threads that are blocked waiting for `*this`.

32.7.5.2 Noninterruptible waits

[`thread.condvarany.wait`]

```

template<class Lock>
    void wait(Lock& lock);

```

1 *Effects:*

- (1.1) — Atomically calls `lock.unlock()` and blocks on `*this`.
- (1.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.
- (1.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, or spuriously.

2 *Postconditions:* `lock` is locked by the calling thread.

3 *Throws:* Nothing.

4 *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[*Note 1:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

```

template<class Lock, class Predicate>
    void wait(Lock& lock, Predicate pred);

```

5 *Effects:* Equivalent to:

```

    while (!pred())
        wait(lock);

```

```

template<class Lock, class Clock, class Duration>
    cv_status wait_until(Lock& lock, chrono::time_point<Clock, Duration> abs_time);

```

6 *Effects:*

- (6.1) — Atomically calls `lock.unlock()` and blocks on `*this`.
- (6.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.

- (6.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout (32.2.4) specified by `abs_time`, or spuriously.
- (6.4) — If the function exits via an exception, `lock.lock()` is called prior to exiting the function.

7 *Postconditions:* `lock` is locked by the calling thread.

8 *Returns:* `cv_status::timeout` if the absolute timeout (32.2.4) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

9 *Throws:* Timeout-related exceptions (32.2.4).

10 *Remarks:* If the function fails to meet the postcondition, `terminate()` is invoked (14.6.2).

[Note 2: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Lock, class Rep, class Period>
cv_status wait_for(Lock& lock, chrono::duration<Rep, Period> rel_time);
```

11 *Effects:* Equivalent to:

```
    return wait_until(lock, rel-to-abs(rel_time));
```

12 *Postconditions:* `lock` is locked by the calling thread.

13 *Returns:* `cv_status::timeout` if the relative timeout (32.2.4) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

14 *Throws:* Timeout-related exceptions (32.2.4).

15 *Remarks:* If the function fails to meet the postcondition, `terminate` is invoked (14.6.2).

[Note 3: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Lock, class Clock, class Duration, class Predicate>
bool wait_until(Lock& lock, chrono::time_point<Clock, Duration> abs_time, Predicate pred);
```

16 *Effects:* Equivalent to:

```
    while (!pred())
        if (wait_until(lock, abs_time) == cv_status::timeout)
            return pred();
    return true;
```

17 [Note 4: There is no blocking if `pred()` is initially `true`, or if the timeout has already expired. — end note]

18 [Note 5: The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered. — end note]

```
template<class Lock, class Rep, class Period, class Predicate>
bool wait_for(Lock& lock, chrono::duration<Rep, Period> rel_time, Predicate pred);
```

19 *Effects:* Equivalent to:

```
    return wait_until(lock, rel-to-abs(rel_time), std::move(pred));
```

32.7.5.3 Interruptible waits

[`thread.condvarany.intwait`]

- 1 The following wait functions will be notified when there is a stop request on the passed `stop_token`. In that case the functions return immediately, returning `false` if the predicate evaluates to `false`.

```
template<class Lock, class Predicate>
bool wait(Lock& lock, stop_token token, Predicate pred);
```

2 *Effects:* Registers for the duration of this call `*this` to get notified on a stop request on `token` during this call and then equivalent to:

```
    while (!token.stop_requested()) {
        if (pred())
            return true;
        wait(lock);
    }
    return pred();
```

3 [Note 1: The returned value indicates whether the predicate evaluated to `true` regardless of whether there was a stop request. — end note]

4 *Postconditions:* `lock` is locked by the calling thread.

5 *Throws:* Any exception thrown by `pred`.

6 *Remarks:* If the function fails to meet the postcondition, `terminate` is called (14.6.2).

[Note 2: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Lock, class Clock, class Duration, class Predicate>
bool wait_until(Lock& lock, stop_token stoken,
               chrono::time_point<Clock, Duration> abs_time, Predicate pred);
```

7 *Effects:* Registers for the duration of this call `*this` to get notified on a stop request on `stoken` during this call and then equivalent to:

```
while (!stoken.stop_requested()) {
    if (pred())
        return true;
    if (wait_until(lock, abs_time) == cv_status::timeout)
        return pred();
}
return pred();
```

8 [Note 3: There is no blocking if `pred()` is initially true, `stoken.stop_requested()` was already true or the timeout has already expired. — end note]

9 [Note 4: The returned value indicates whether the predicate evaluated to true regardless of whether the timeout was triggered or a stop request was made. — end note]

10 *Postconditions:* `lock` is locked by the calling thread.

11 *Throws:* Timeout-related exceptions (32.2.4), or any exception thrown by `pred`.

12 *Remarks:* If the function fails to meet the postcondition, `terminate` is called (14.6.2).

[Note 5: This can happen if the re-locking of the mutex throws an exception. — end note]

```
template<class Lock, class Rep, class Period, class Predicate>
bool wait_for(Lock& lock, stop_token stoken,
              chrono::duration<Rep, Period> rel_time, Predicate pred);
```

13 *Effects:* Equivalent to:

```
return wait_until(lock, std::move(stoken), rel-to-abs(rel_time), std::move(pred));
```

32.8 Semaphore [thread.sema]

32.8.1 General [thread.sema.general]

1 Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.

2 A counting semaphore is a semaphore object that models a non-negative resource count. A binary semaphore is a semaphore object that has only two states. A binary semaphore should be more efficient than the default implementation of a counting semaphore with a unit resource count.

32.8.2 Header <semaphore> synopsis [semaphore.syn]

```
namespace std {
    // 32.8.3, class template counting_semaphore
    template<ptrdiff_t least_max_value = implementation-defined>
        class counting_semaphore;

    using binary_semaphore = counting_semaphore<1>;
}
```

32.8.3 Class template counting_semaphore [thread.sema.cnt]

```
namespace std {
    template<ptrdiff_t least_max_value = implementation-defined>
        class counting_semaphore {
        public:
            static constexpr ptrdiff_t max() noexcept;
```

```

constexpr explicit counting_semaphore(ptrdiff_t desired);
~counting_semaphore();

counting_semaphore(const counting_semaphore&) = delete;
counting_semaphore& operator=(const counting_semaphore&) = delete;

void release(ptrdiff_t update = 1);
void acquire();
bool try_acquire() noexcept;
template<class Rep, class Period>
    bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
    bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_time);

private:
    ptrdiff_t counter;          // exposition only
};

```

1 Class template `counting_semaphore` maintains an internal counter that is initialized when the semaphore is created. The counter is decremented when a thread acquires the semaphore, and is incremented when a thread releases the semaphore. If a thread tries to acquire the semaphore when the counter is zero, the thread will block until another thread increments the counter by releasing the semaphore.

2 `least_max_value` shall be non-negative; otherwise the program is ill-formed.

3 Concurrent invocations of the member functions of `counting_semaphore`, other than its destructor, do not introduce data races.

```
static constexpr ptrdiff_t max() noexcept;
```

4 *Returns:* The maximum value of `counter`. This value is greater than or equal to `least_max_value`.

```
constexpr explicit counting_semaphore(ptrdiff_t desired);
```

5 *Preconditions:* `desired >= 0` is true, and `desired <= max()` is true.

6 *Effects:* Initializes `counter` with `desired`.

7 *Throws:* Nothing.

```
void release(ptrdiff_t update = 1);
```

8 *Preconditions:* `update >= 0` is true, and `update <= max() - counter` is true.

9 *Effects:* Atomically execute `counter += update`. Then, unblocks any threads that are waiting for `counter` to be greater than zero.

10 *Synchronization:* Strongly happens before invocations of `try_acquire` that observe the result of the effects.

11 *Throws:* `system_error` when an exception is required (32.2.2).

12 *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

```
bool try_acquire() noexcept;
```

13 *Effects:* Attempts to atomically decrement `counter` if it is positive, without blocking. If `counter` is not decremented, there is no effect and `try_acquire` immediately returns. An implementation may fail to decrement `counter` even if it is positive.

[Note 1: This spurious failure is normally uncommon, but allows interesting implementations based on a simple compare and exchange (32.5). — end note]

An implementation should ensure that `try_acquire` does not consistently return `false` in the absence of contending semaphore operations.

14 *Returns:* `true` if `counter` was decremented, otherwise `false`.

```
void acquire();
```

15 *Effects:* Repeatedly performs the following steps, in order:

- (15.1) — Evaluates `try_acquire()`. If the result is `true`, returns.
- (15.2) — Blocks on `*this` until `counter` is greater than zero.
- 16 *Throws:* `system_error` when an exception is required (32.2.2).
- 17 *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).
- ```
template<class Rep, class Period>
 bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
 bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_time);
```
- 18 *Effects:* Repeatedly performs the following steps, in order:
- (18.1) — Evaluates `try_acquire()`. If the result is `true`, returns `true`.
- (18.2) — Blocks on `*this` until `counter` is greater than zero or until the timeout expires. If it is unblocked by the timeout expiring, returns `false`.
- The timeout expires (32.2.4) when the current time is after `abs_time` (for `try_acquire_until`) or when at least `rel_time` has passed from the start of the function (for `try_acquire_for`).
- 19 *Throws:* Timeout-related exceptions (32.2.4), or `system_error` when a non-timeout-related exception is required (32.2.2).
- 20 *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

## 32.9 Coordination types

[thread.coord]

### 32.9.1 General

[thread.coord.general]

- <sup>1</sup> Subclause 32.9 describes various concepts related to thread coordination, and defines the coordination types `latch` and `barrier`. These types facilitate concurrent computation performed by a number of threads.

### 32.9.2 Latches

[thread.latch]

#### 32.9.2.1 General

[thread.latch.general]

- <sup>1</sup> A latch is a thread coordination mechanism that allows any number of threads to block until an expected number of threads arrive at the latch (via the `count_down` function). The expected count is set when the latch is created. An individual latch is a single-use object; once the expected count has been reached, the latch cannot be reused.

#### 32.9.2.2 Header <latch> synopsis

[latch.syn]

```
namespace std {
 class latch;
}
```

#### 32.9.2.3 Class latch

[thread.latch.class]

```
namespace std {
 class latch {
 public:
 static constexpr ptrdiff_t max() noexcept;

 constexpr explicit latch(ptrdiff_t expected);
 ~latch();

 latch(const latch&) = delete;
 latch& operator=(const latch&) = delete;

 void count_down(ptrdiff_t update = 1);
 bool try_wait() const noexcept;
 void wait() const;
 void arrive_and_wait(ptrdiff_t update = 1);

 private:
 ptrdiff_t counter; // exposition only
 };
}
```

- <sup>1</sup> A **latch** maintains an internal counter that is initialized when the latch is created. Threads can block on the latch object, waiting for counter to be decremented to zero.
- <sup>2</sup> Concurrent invocations of the member functions of **latch**, other than its destructor, do not introduce data races.

```
static constexpr ptrdiff_t max() noexcept;
```

- <sup>3</sup> *Returns:* The maximum value of *counter* that the implementation supports.

```
constexpr explicit latch(ptrdiff_t expected);
```

- <sup>4</sup> *Preconditions:* `expected >= 0` is true and `expected <= max()` is true.

- <sup>5</sup> *Effects:* Initializes *counter* with *expected*.

- <sup>6</sup> *Throws:* Nothing.

```
void count_down(ptrdiff_t update = 1);
```

- <sup>7</sup> *Preconditions:* `update >= 0` is true, and `update <= counter` is true.

- <sup>8</sup> *Effects:* Atomically decrements *counter* by *update*. If *counter* is equal to zero, unblocks all threads blocked on *\*this*.

- <sup>9</sup> *Synchronization:* Strongly happens before the returns from all calls that are unblocked.

- <sup>10</sup> *Throws:* **system\_error** when an exception is required (32.2.2).

- <sup>11</sup> *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

```
bool try_wait() const noexcept;
```

- <sup>12</sup> *Returns:* With very low probability **false**. Otherwise *counter* == 0.

```
void wait() const;
```

- <sup>13</sup> *Effects:* If *counter* equals zero, returns immediately. Otherwise, blocks on *\*this* until a call to `count_down` that decrements *counter* to zero.

- <sup>14</sup> *Throws:* **system\_error** when an exception is required (32.2.2).

- <sup>15</sup> *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

```
void arrive_and_wait(ptrdiff_t update = 1);
```

- <sup>16</sup> *Effects:* Equivalent to:

```
count_down(update);
wait();
```

### 32.9.3 Barriers

[thread.barrier]

#### 32.9.3.1 General

[thread.barrier.general]

- <sup>1</sup> A barrier is a thread coordination mechanism whose lifetime consists of a sequence of barrier phases, where each phase allows at most an expected number of threads to block until the expected number of threads arrive at the barrier.

[Note 1: A barrier is useful for managing repeated tasks that are handled by multiple threads. — end note]

#### 32.9.3.2 Header <barrier> synopsis

[barrier.syn]

```
namespace std {
 template<class CompletionFunction = see below>
 class barrier;
}
```

#### 32.9.3.3 Class template barrier

[thread.barrier.class]

```
namespace std {
 template<class CompletionFunction = see below>
 class barrier {
 public:
 using arrival_token = see below;

 static constexpr ptrdiff_t max() noexcept;
```

```

constexpr explicit barrier(ptrdiff_t expected,
 CompletionFunction f = CompletionFunction());
~barrier();

barrier(const barrier&) = delete;
barrier& operator=(const barrier&) = delete;

arrival_token arrive(ptrdiff_t update = 1);
void wait(arrival_token&& arrival) const;

void arrive_and_wait();
void arrive_and_drop();

private:
 CompletionFunction completion; // exposition only
};

```

<sup>1</sup> Each *barrier phase* consists of the following steps:

- (1.1) — The expected count is decremented by each call to `arrive` or `arrive_and_drop`.
- (1.2) — Exactly once after the expected count reaches zero, a thread executes the completion step during its call to `arrive`, `arrive_and_drop`, or `wait`, except that it is implementation-defined whether the step executes if no thread calls `wait`.
- (1.3) — When the completion step finishes, the expected count is reset to what was specified by the `expected` argument to the constructor, possibly adjusted by calls to `arrive_and_drop`, and the next phase starts.

<sup>2</sup> Each phase defines a *phase synchronization point*. Threads that arrive at the barrier during the phase can block on the phase synchronization point by calling `wait`, and will remain blocked until the phase completion step is run.

<sup>3</sup> The *phase completion step* that is executed at the end of each phase has the following effects:

- (3.1) — Invokes the completion function, equivalent to `completion()`.
- (3.2) — Unblocks all threads that are blocked on the phase synchronization point.

The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step. For specializations that do not have the default value of the `CompletionFunction` template parameter, the behavior is undefined if any of the barrier object's member functions other than `wait` are called while the completion step is in progress.

<sup>4</sup> Concurrent invocations of the member functions of `barrier`, other than its destructor, do not introduce data races. The member functions `arrive` and `arrive_and_drop` execute atomically.

<sup>5</sup> `CompletionFunction` shall meet the *Cpp17MoveConstructible* (Table 31) and *Cpp17Destructible* (Table 35) requirements. `is_nothrow_invocable_v<CompletionFunction&>` shall be `true`.

<sup>6</sup> The default value of the `CompletionFunction` template parameter is an unspecified type, such that, in addition to satisfying the requirements of `CompletionFunction`, it meets the *Cpp17DefaultConstructible* requirements (Table 30) and `completion()` has no effects.

<sup>7</sup> `barrier::arrival_token` is an unspecified type, such that it meets the *Cpp17MoveConstructible* (Table 31), *Cpp17MoveAssignable* (Table 33), and *Cpp17Destructible* (Table 35) requirements.

```
static constexpr ptrdiff_t max() noexcept;
```

<sup>8</sup> *Returns:* The maximum expected count that the implementation supports.

```
constexpr explicit barrier(ptrdiff_t expected,
 CompletionFunction f = CompletionFunction());
```

<sup>9</sup> *Preconditions:* `expected >= 0` is true and `expected <= max()` is true.

<sup>10</sup> *Effects:* Sets both the initial expected count for each barrier phase and the current expected count for the first phase to `expected`. Initializes `completion` with `std::move(f)`. Starts the first phase.

[Note 1: If `expected` is 0 this object can only be destroyed. — end note]

<sup>11</sup> *Throws:* Any exception thrown by `CompletionFunction`'s move constructor.

```
arrival_token arrive(ptrdiff_t update = 1);
```

12     *Preconditions:* `update > 0` is `true`, and `update` is less than or equal to the expected count for the current barrier phase.

13     *Effects:* Constructs an object of type `arrival_token` that is associated with the phase synchronization point for the current phase. Then, decrements the expected count by `update`.

14     *Synchronization:* The call to `arrive` strongly happens before the start of the phase completion step for the current phase.

15     *Returns:* The constructed `arrival_token` object.

16     *Throws:* `system_error` when an exception is required (32.2.2).

17     *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

18     [Note 2: This call can cause the completion step for the current phase to start. — end note]

```
void wait(arrival_token&& arrival) const;
```

19     *Preconditions:* `arrival` is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.

20     *Effects:* Blocks at the synchronization point associated with `std::move(arrival)` until the phase completion step of the synchronization point's phase is run.

[Note 3: If `arrival` is associated with the synchronization point for a previous phase, the call returns immediately. — end note]

21     *Throws:* `system_error` when an exception is required (32.2.2).

22     *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

```
void arrive_and_wait();
```

23     *Effects:* Equivalent to: `wait(arrive())`.

```
void arrive_and_drop();
```

24     *Preconditions:* The expected count for the current barrier phase is greater than zero.

25     *Effects:* Decrements the initial expected count for all subsequent phases by one. Then decrements the expected count for the current phase by one.

26     *Synchronization:* The call to `arrive_and_drop` strongly happens before the start of the phase completion step for the current phase.

27     *Throws:* `system_error` when an exception is required (32.2.2).

28     *Error conditions:* Any of the error conditions allowed for mutex types (32.6.4.2).

29     [Note 4: This call can cause the completion step for the current phase to start. — end note]

## 32.10 Futures

[futures]

### 32.10.1 Overview

[futures.overview]

1     32.10 describes components that a C++ program can use to retrieve in one thread the result (value or exception) from a function that has run in the same thread or another thread.

[Note 1: These components are not restricted to multi-threaded programs but can be useful in single-threaded programs as well. — end note]

### 32.10.2 Header <future> synopsis

[future.syn]

```
namespace std {
 enum class future_errc {
 broken_promise = implementation-defined,
 future_already_retrieved = implementation-defined,
 promise_already_satisfied = implementation-defined,
 no_state = implementation-defined
 };
};
```

```

enum class launch : unspecified {
 async = unspecified,
 deferred = unspecified,
 implementation-defined
};

enum class future_status {
 ready,
 timeout,
 deferred
};

// 32.10.3, error handling
template<> struct is_error_code_enum<future_errc> : public true_type { };
error_code make_error_code(future_errc e) noexcept;
error_condition make_error_condition(future_errc e) noexcept;

const error_category& future_category() noexcept;

// 32.10.4, class future_error
class future_error;

// 32.10.6, class template promise
template<class R> class promise;
template<class R> class promise<R&>;
template<> class promise<void>;

template<class R>
 void swap(promise<R>& x, promise<R>& y) noexcept;

// 32.10.7, class template future
template<class R> class future;
template<class R> class future<R&>;
template<> class future<void>;

// 32.10.8, class template shared_future
template<class R> class shared_future;
template<class R> class shared_future<R&>;
template<> class shared_future<void>;

// 32.10.10, class template packaged_task
template<class> class packaged_task; // not defined
template<class R, class... ArgTypes>
 class packaged_task<R(ArgTypes...)>;

template<class R, class... ArgTypes>
 void swap(packaged_task<R(ArgTypes...)>&, packaged_task<R(ArgTypes...)>&) noexcept;

// 32.10.9, function template async
template<class F, class... Args>
 future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
 async(F&& f, Args&&... args);
template<class F, class... Args>
 future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
 async(launch policy, F&& f, Args&&... args);
}

```

<sup>1</sup> The enum type `launch` is a bitmask type (16.3.3.3.3) with elements `launch::async` and `launch::deferred`.

[Note 1: Implementations can provide bitmasks to specify restrictions on task interaction by functions launched by `async()` applicable to a corresponding subset of available launch policies. Implementations can extend the behavior of the first overload of `async()` by adding their extensions to the launch policy under the “as if” rule. — end note]

<sup>2</sup> The enumerator values of `future_errc` are distinct and not zero.

**32.10.3 Error handling****[futures.errors]**

```
const error_category& future_category() noexcept;
```

1 *Returns:* A reference to an object of a type derived from class `error_category`.

2 The object's `default_error_condition` and equivalent virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function returns a pointer to the string "future".

```
error_code make_error_code(future_errc e) noexcept;
```

3 *Returns:* `error_code(static_cast<int>(e), future_category())`.

```
error_condition make_error_condition(future_errc e) noexcept;
```

4 *Returns:* `error_condition(static_cast<int>(e), future_category())`.

**32.10.4 Class `future_error`****[futures.future.error]**

```
namespace std {
 class future_error : public logic_error {
 public:
 explicit future_error(future_errc e);

 const error_code& code() const noexcept;
 const char* what() const noexcept;

 private:
 error_code ec_; // exposition only
 };
}
```

```
explicit future_error(future_errc e);
```

1 *Effects:* Initializes `ec_` with `make_error_code(e)`.

```
const error_code& code() const noexcept;
```

2 *Returns:* `ec_`.

```
const char* what() const noexcept;
```

3 *Returns:* An NTBS incorporating `code().message()`.

**32.10.5 Shared state****[futures.state]**

1 Many of the classes introduced in subclause 32.10 use some state to communicate results. This *shared state* consists of some state information and some (possibly not yet evaluated) *result*, which can be a (possibly void) value or an exception.

[Note 1: Futures, promises, and tasks defined in this Clause reference such shared state. — end note]

2 [Note 2: The result can be any kind of object including a function to compute that result, as used by `async` when `policy` is `launch::deferred`. — end note]

3 An *asynchronous return object* is an object that reads results from a shared state. A *waiting function* of an asynchronous return object is one that potentially blocks to wait for the shared state to be made ready. If a waiting function can return before the state is made ready because of a timeout (32.2.5), then it is a *timed waiting function*, otherwise it is a *non-timed waiting function*.

4 An *asynchronous provider* is an object that provides a result to a shared state. The result of a shared state is set by respective functions on the asynchronous provider.

[Example 1: Promises and tasks are examples of asynchronous providers. — end example]

The means of setting the result of a shared state is specified in the description of those classes and functions that create such a state object.

5 When an asynchronous return object or an asynchronous provider is said to release its shared state, it means:

- (5.1) — if the return object or provider holds the last reference to its shared state, the shared state is destroyed; and
- (5.2) — the return object or provider gives up its reference to its shared state; and



- (5.3) — these actions will not block for the shared state to become ready, except that it may block if all of the following are true: the shared state was created by a call to `std::async`, the shared state is not yet ready, and this was the last reference to the shared state.

6 When an asynchronous provider is said to make its shared state ready, it means:

- (6.1) — first, the provider marks its shared state as ready; and
- (6.2) — second, the provider unblocks any execution agents waiting for its shared state to become ready.

7 When an asynchronous provider is said to abandon its shared state, it means:

- (7.1) — first, if that state is not ready, the provider
  - (7.1.1) — stores an exception object of type `future_error` with an error condition of `broken_promise` within its shared state; and then
  - (7.1.2) — makes its shared state ready;
- (7.2) — second, the provider releases its shared state.

8 A shared state is *ready* only if it holds a value or an exception ready for retrieval. Waiting for a shared state to become ready may invoke code to compute the result on the waiting thread if so specified in the description of the class or function that creates the state object.

9 Calls to functions that successfully set the stored result of a shared state synchronize with (6.10.2) calls to functions successfully detecting the ready state resulting from that setting. The storage of the result (whether normal or exceptional) into the shared state synchronizes with (6.10.2) the successful return from a call to a waiting function on the shared state.

10 Some functions (e.g., `promise::set_value_at_thread_exit`) delay making the shared state ready until the calling thread exits. The destruction of each of that thread's objects with thread storage duration (6.8.6.3) is sequenced before making that shared state ready.

11 Access to the result of the same shared state may conflict (6.10.2).

[*Note 3:* This explicitly specifies that the result of the shared state is visible in the objects that reference this state in the sense of data race avoidance (16.4.6.10). For example, concurrent accesses through references returned by `shared_future::get()` (32.10.8) must either use read-only operations or provide additional synchronization. — *end note*]

### 32.10.6 Class template promise

[`futures.promise`]

```
namespace std {
 template<class R>
 class promise {
 public:
 promise();
 template<class Allocator>
 promise(allocator_arg_t, const Allocator& a);
 promise(promise&& rhs) noexcept;
 promise(const promise&) = delete;
 ~promise();

 // assignment
 promise& operator=(promise&& rhs) noexcept;
 promise& operator=(const promise&) = delete;
 void swap(promise& other) noexcept;

 // retrieving the result
 future<R> get_future();

 // setting the result
 void set_value(see below);
 void set_exception(exception_ptr p);

 // setting the result with deferred notification
 void set_value_at_thread_exit(see below);
 void set_exception_at_thread_exit(exception_ptr p);
 };
}
```

```
};
}
```

- 1 For the primary template, `R` shall be an object type that meets the *Cpp17Destructible* requirements.
- 2 The implementation provides the template `promise` and two specializations, `promise<R&&>` and `promise<void>`. These differ only in the argument type of the member functions `set_value` and `set_value_at_thread_exit`, as set out in their descriptions, below.
- 3 The `set_value`, `set_exception`, `set_value_at_thread_exit`, and `set_exception_at_thread_exit` member functions behave as though they acquire a single mutex associated with the promise object while updating the promise object.

```
promise();
template<class Allocator>
promise(allocator_arg_t, const Allocator& a);
```

- 4 *Effects*: Creates a shared state. The second constructor uses the allocator `a` to allocate memory for the shared state.

```
promise(promise&& rhs) noexcept;
```

- 5 *Effects*: Transfers ownership of the shared state of `rhs` (if any) to the newly-constructed object.

- 6 *Postconditions*: `rhs` has no shared state.

```
~promise();
```

- 7 *Effects*: Abandons any shared state (32.10.5).

```
promise& operator=(promise&& rhs) noexcept;
```

- 8 *Effects*: Equivalent to `promise(std::move(rhs)).swap(*this)`.

- 9 *Returns*: `*this`.

```
void swap(promise& other) noexcept;
```

- 10 *Effects*: Exchanges the shared state of `*this` and `other`.

- 11 *Postconditions*: `*this` has the shared state (if any) that `other` had prior to the call to `swap`. `other` has the shared state (if any) that `*this` had prior to the call to `swap`.

```
future<R> get_future();
```

- 12 *Synchronization*: Calls to this function do not introduce data races (6.10.2) with calls to `set_value`, `set_exception`, `set_value_at_thread_exit`, or `set_exception_at_thread_exit`.

[Note 1: Such calls need not synchronize with each other. — end note]

- 13 *Returns*: A `future<R>` object with the same shared state as `*this`.

- 14 *Throws*: `future_error` if `*this` has no shared state or if `get_future` has already been called on a `promise` with the same shared state as `*this`.

- 15 *Error conditions*:

- (15.1) — `future_already_retrieved` if `get_future` has already been called on a `promise` with the same shared state as `*this`.

- (15.2) — `no_state` if `*this` has no shared state.

```
void promise::set_value(const R& r);
void promise::set_value(R&& r);
void promise<R&&>::set_value(R& r);
void promise<void>::set_value();
```

- 16 *Effects*: Atomically stores the value `r` in the shared state and makes that state ready (32.10.5).

- 17 *Throws*:

- (17.1) — `future_error` if its shared state already has a stored value or exception, or

- (17.2) — for the first version, any exception thrown by the constructor selected to copy an object of `R`, or

- (17.3) — for the second version, any exception thrown by the constructor selected to move an object of `R`.

18 *Error conditions:*

(18.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(18.2) — `no_state` if `*this` has no shared state.

`void set_exception(exception_ptr p);`

19 *Preconditions:* `p` is not null.

20 *Effects:* Atomically stores the exception pointer `p` in the shared state and makes that state ready (32.10.5).

21 *Throws:* `future_error` if its shared state already has a stored value or exception.

22 *Error conditions:*

(22.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(22.2) — `no_state` if `*this` has no shared state.

`void promise::set_value_at_thread_exit(const R& r);`  
`void promise::set_value_at_thread_exit(R&& r);`  
`void promise<R>::set_value_at_thread_exit(R& r);`  
`void promise<void>::set_value_at_thread_exit();`

23 *Effects:* Stores the value `r` in the shared state without making that state ready immediately. Schedules that state to be made ready when the current thread exits, after all objects with thread storage duration associated with the current thread have been destroyed.

24 *Throws:*

(24.1) — `future_error` if its shared state already has a stored value or exception, or

(24.2) — for the first version, any exception thrown by the constructor selected to copy an object of `R`, or

(24.3) — for the second version, any exception thrown by the constructor selected to move an object of `R`.

25 *Error conditions:*

(25.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(25.2) — `no_state` if `*this` has no shared state.

`void set_exception_at_thread_exit(exception_ptr p);`

26 *Preconditions:* `p` is not null.

27 *Effects:* Stores the exception pointer `p` in the shared state without making that state ready immediately. Schedules that state to be made ready when the current thread exits, after all objects with thread storage duration associated with the current thread have been destroyed.

28 *Throws:* `future_error` if an error condition occurs.

29 *Error conditions:*

(29.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(29.2) — `no_state` if `*this` has no shared state.

`template<class R>`  
`void swap(promise<R>& x, promise<R>& y) noexcept;`

30 *Effects:* As if by `x.swap(y)`.

### 32.10.7 Class template `future`

[`futures.unique.future`]

1 The class template `future` defines a type for asynchronous return objects which do not share their shared state with other asynchronous return objects. A default-constructed `future` object has no shared state. A `future` object with shared state can be created by functions on asynchronous providers (32.10.5) or by the move constructor and shares its shared state with the original asynchronous provider. The result (value or exception) of a `future` object can be set by calling a respective function on an object that shares the same shared state.

2 [Note 1: Member functions of `future` do not synchronize with themselves or with member functions of `shared_future`. — end note]

- <sup>3</sup> The effect of calling any member function other than the destructor, the move assignment operator, **share**, or **valid** on a **future** object for which **valid()** == **false** is undefined.

[*Note 2:* It is valid to move from a future object for which **valid()** == **false**. — *end note*]

*Recommended practice:* Implementations should detect this case and throw an object of type **future\_error** with an error condition of **future\_errc::no\_state**.

```
namespace std {
 template<class R>
 class future {
 public:
 future() noexcept;
 future(future&&) noexcept;
 future(const future&) = delete;
 ~future();
 future& operator=(const future&) = delete;
 future& operator=(future&&) noexcept;
 shared_future<R> share() noexcept;

 // retrieving the value
 see below get();

 // functions to check state
 bool valid() const noexcept;

 void wait() const;
 template<class Rep, class Period>
 future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
 template<class Clock, class Duration>
 future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
 };
}
```

- <sup>4</sup> For the primary template, **R** shall be an object type that meets the *Cpp17Destructible* requirements.
- <sup>5</sup> The implementation provides the template **future** and two specializations, **future<R&>** and **future<void>**. These differ only in the return type and return value of the member function **get**, as set out in its description, below.

```
future() noexcept;
```

- <sup>6</sup> *Effects:* The object does not refer to a shared state.

- <sup>7</sup> *Postconditions:* **valid()** == **false**.

```
future(future&& rhs) noexcept;
```

- <sup>8</sup> *Effects:* Move constructs a **future** object that refers to the shared state that was originally referred to by **rhs** (if any).

- <sup>9</sup> *Postconditions:*

(9.1) — **valid()** returns the same value as **rhs.valid()** prior to the constructor invocation.

(9.2) — **rhs.valid()** == **false**.

```
~future();
```

- <sup>10</sup> *Effects:*

(10.1) — Releases any shared state (32.10.5);

(10.2) — destroys **\*this**.

```
future& operator=(future&& rhs) noexcept;
```

- <sup>11</sup> *Effects:* If **addressof(rhs)** == **this** is true, there are no effects. Otherwise:

(11.1) — Releases any shared state (32.10.5).

(11.2) — move assigns the contents of **rhs** to **\*this**.

```

12 Postconditions:
(12.1) — valid() returns the same value as rhs.valid() prior to the assignment.
(12.2) — If addressof(rhs) == this is false, rhs.valid() == false.

 shared_future<R> share() noexcept;
13 Postconditions: valid() == false.
14 Returns: shared_future<R>(std::move(*this)).

 R future::get();
 R& future<R&>::get();
 void future<void>::get();
15 [Note 3: As described above, the template and its two required specializations differ only in the return type
 and return value of the member function get. — end note]
16 Effects:
(16.1) — wait()s until the shared state is ready, then retrieves the value stored in the shared state;
(16.2) — releases any shared state (32.10.5).
17 Postconditions: valid() == false.
18 Returns:
(18.1) — future::get() returns the value v stored in the object's shared state as std::move(v).
(18.2) — future<R&>::get() returns the reference stored as value in the object's shared state.
(18.3) — future<void>::get() returns nothing.
19 Throws: The stored exception, if an exception was stored in the shared state.

 bool valid() const noexcept;
20 Returns: true only if *this refers to a shared state.

 void wait() const;
21 Effects: Blocks until the shared state is ready.

 template<class Rep, class Period>
 future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
22 Effects: None if the shared state contains a deferred function (32.10.9), otherwise blocks until the
 shared state is ready or until the relative timeout (32.2.4) specified by rel_time has expired.
23 Returns:
(23.1) — future_status::deferred if the shared state contains a deferred function.
(23.2) — future_status::ready if the shared state is ready.
(23.3) — future_status::timeout if the function is returning because the relative timeout (32.2.4) specified
 by rel_time has expired.
24 Throws: timeout-related exceptions (32.2.4).

 template<class Clock, class Duration>
 future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
25 Effects: None if the shared state contains a deferred function (32.10.9), otherwise blocks until the
 shared state is ready or until the absolute timeout (32.2.4) specified by abs_time has expired.
26 Returns:
(26.1) — future_status::deferred if the shared state contains a deferred function.
(26.2) — future_status::ready if the shared state is ready.
(26.3) — future_status::timeout if the function is returning because the absolute timeout (32.2.4)
 specified by abs_time has expired.
27 Throws: timeout-related exceptions (32.2.4).

```

32.10.8 Class template `shared_future`[`futures.shared.future`]

<sup>1</sup> The class template `shared_future` defines a type for asynchronous return objects which may share their shared state with other asynchronous return objects. A default-constructed `shared_future` object has no shared state. A `shared_future` object with shared state can be created by conversion from a `future` object and shares its shared state with the original asynchronous provider (32.10.5) of the shared state. The result (value or exception) of a `shared_future` object can be set by calling a respective function on an object that shares the same shared state.

<sup>2</sup> [Note 1: Member functions of `shared_future` do not synchronize with themselves, but they synchronize with the shared state. — end note]

<sup>3</sup> The effect of calling any member function other than the destructor, the move assignment operator, the copy assignment operator, or `valid()` on a `shared_future` object for which `valid() == false` is undefined.

[Note 2: It is valid to copy or move from a `shared_future` object for which `valid()` is `false`. — end note]

*Recommended practice:* Implementations should detect this case and throw an object of type `future_error` with an error condition of `future_errc::no_state`.

```
namespace std {
 template<class R>
 class shared_future {
 public:
 shared_future() noexcept;
 shared_future(const shared_future& rhs) noexcept;
 shared_future(future<R>&&) noexcept;
 shared_future(shared_future&& rhs) noexcept;
 ~shared_future();
 shared_future& operator=(const shared_future& rhs) noexcept;
 shared_future& operator=(shared_future&& rhs) noexcept;

 // retrieving the value
 see below get() const;

 // functions to check state
 bool valid() const noexcept;

 void wait() const;
 template<class Rep, class Period>
 future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
 template<class Clock, class Duration>
 future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
 };
}
```

<sup>4</sup> For the primary template, `R` shall be an object type that meets the *Cpp17Destructible* requirements.

<sup>5</sup> The implementation provides the template `shared_future` and two specializations, `shared_future<R>` and `shared_future<void>`. These differ only in the return type and return value of the member function `get`, as set out in its description, below.

```
shared_future() noexcept;
```

<sup>6</sup> *Effects:* The object does not refer to a shared state.

<sup>7</sup> *Postconditions:* `valid() == false`.

```
shared_future(const shared_future& rhs) noexcept;
```

<sup>8</sup> *Effects:* The object refers to the same shared state as `rhs` (if any).

<sup>9</sup> *Postconditions:* `valid()` returns the same value as `rhs.valid()`.

```
shared_future(future<R>&& rhs) noexcept;
shared_future(shared_future&& rhs) noexcept;
```

<sup>10</sup> *Effects:* Move constructs a `shared_future` object that refers to the shared state that was originally referred to by `rhs` (if any).

<sup>11</sup> *Postconditions:*

(11.1) — `valid()` returns the same value as `rhs.valid()` returned prior to the constructor invocation.

(11.2) — `rhs.valid() == false`.

```
~shared_future();
```

12 *Effects:*

(12.1) — Releases any shared state (32.10.5);

(12.2) — destroys `*this`.

```
shared_future& operator=(shared_future&& rhs) noexcept;
```

13 *Effects:* If `addressof(rhs) == this` is true, there are no effects. Otherwise:

(13.1) — Releases any shared state (32.10.5);

(13.2) — move assigns the contents of `rhs` to `*this`.

14 *Postconditions:*

(14.1) — `valid()` returns the same value as `rhs.valid()` returned prior to the assignment.

(14.2) — If `addressof(rhs) == this` is false, `rhs.valid() == false`.

```
shared_future& operator=(const shared_future& rhs) noexcept;
```

15 *Effects:* If `addressof(rhs) == this` is true, there are no effects. Otherwise:

(15.1) — Releases any shared state (32.10.5);

(15.2) — assigns the contents of `rhs` to `*this`.  
[Note 3: As a result, `*this` refers to the same shared state as `rhs` (if any). — end note]

16 *Postconditions:* `valid() == rhs.valid()`.

```
const R& shared_future::get() const;
R& shared_future<R&>::get() const;
void shared_future<void>::get() const;
```

17 [Note 4: As described above, the template and its two required specializations differ only in the return type and return value of the member function `get`. — end note]

18 [Note 5: Access to a value object stored in the shared state is unsynchronized, so operations on `R` might introduce a data race (6.10.2). — end note]

19 *Effects:* `wait()`s until the shared state is ready, then retrieves the value stored in the shared state.

20 *Returns:*

(20.1) — `shared_future::get()` returns a const reference to the value stored in the object's shared state.  
[Note 6: Access through that reference after the shared state has been destroyed produces undefined behavior; this can be avoided by not storing the reference in any storage with a greater lifetime than the `shared_future` object that returned the reference. — end note]

(20.2) — `shared_future<R&>::get()` returns the reference stored as value in the object's shared state.

(20.3) — `shared_future<void>::get()` returns nothing.

21 *Throws:* The stored exception, if an exception was stored in the shared state.

```
bool valid() const noexcept;
```

22 *Returns:* true only if `*this` refers to a shared state.

```
void wait() const;
```

23 *Effects:* Blocks until the shared state is ready.

```
template<class Rep, class Period>
future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
```

24 *Effects:* None if the shared state contains a deferred function (32.10.9), otherwise blocks until the shared state is ready or until the relative timeout (32.2.4) specified by `rel_time` has expired.

25 *Returns:*

(25.1) — `future_status::deferred` if the shared state contains a deferred function.



- (25.2) — `future_status::ready` if the shared state is ready.
- (25.3) — `future_status::timeout` if the function is returning because the relative timeout (32.2.4) specified by `rel_time` has expired.

26 *Throws:* timeout-related exceptions (32.2.4).

```
template<class Clock, class Duration>
 future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
```

27 *Effects:* None if the shared state contains a deferred function (32.10.9), otherwise blocks until the shared state is ready or until the absolute timeout (32.2.4) specified by `abs_time` has expired.

28 *Returns:*

- (28.1) — `future_status::deferred` if the shared state contains a deferred function.
- (28.2) — `future_status::ready` if the shared state is ready.
- (28.3) — `future_status::timeout` if the function is returning because the absolute timeout (32.2.4) specified by `abs_time` has expired.

29 *Throws:* timeout-related exceptions (32.2.4).

### 32.10.9 Function template `async`

[`futures.async`]

- 1 The function template `async` provides a mechanism to launch a function potentially in a new thread and provides the result of the function in a `future` object with which it shares a shared state.

```
template<class F, class... Args>
 future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
 async(F&& f, Args&&... args);
template<class F, class... Args>
 future<invoke_result_t<decay_t<F>, decay_t<Args>...>>
 async(launch_policy, F&& f, Args&&... args);
```

2 *Mandates:* The following are all true:

- (2.1) — `is_constructible_v<decay_t<F>, F>`,
- (2.2) — `(is_constructible_v<decay_t<Args>, Args> && ...)`, and
- (2.3) — `is_invocable_v<decay_t<F>, decay_t<Args>...>`.

3 *Effects:* The first function behaves the same as a call to the second function with a policy argument of `launch::async` | `launch::deferred` and the same arguments for `F` and `Args`. The second function creates a shared state that is associated with the returned `future` object. The further behavior of the second function depends on the policy argument as follows (if more than one of these conditions applies, the implementation may choose any of the corresponding policies):

- (3.1) — If `launch::async` is set in policy, calls `invoke(auto(std::forward<F>(f)), auto(std::forward<Args>(args))...)` (22.10.5, 32.4.3.4) as if in a new thread of execution represented by a `thread` object with the values produced by `auto` being materialized (7.3.5) in the thread that called `async`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of `invoke(auto(std::forward<F>(f)), auto(std::forward<Args>(args))...)` is stored as the exceptional result in the shared state. The `thread` object is stored in the shared state and affects the behavior of any asynchronous return objects that reference that state.
- (3.2) — If `launch::deferred` is set in policy, stores `auto(std::forward<F>(f))` and `auto(std::forward<Args>(args))...` in the shared state. These copies of `f` and `args` constitute a *deferred function*. Invocation of the deferred function evaluates `invoke(std::move(g), std::move(xyz))` where `g` is the stored value of `auto(std::forward<F>(f))` and `xyz` is the stored copy of `auto(std::forward<Args>(args))...`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of the deferred function is stored as the exceptional result in the shared state. The shared state is not made ready until the function has completed. The first call to a non-timed waiting function (32.10.5) on an asynchronous return object referring to this shared state invokes the deferred function in the thread that called the waiting function. Once evaluation of `invoke(std::move(g), std::move(xyz))` begins, the function is no longer considered deferred.



*Recommended practice:* If this policy is specified together with other policies, such as when using a `policy` value of `launch::async` | `launch::deferred`, implementations should defer invocation or the selection of the policy when no more concurrency can be effectively exploited.

- (3.3) — If no value is set in the launch policy, or a value is set that is neither specified in this document nor by the implementation, the behavior is undefined.

4 *Synchronization:* The invocation of `async` synchronizes with the invocation of `f`. The completion of the function `f` is sequenced before the shared state is made ready.

[*Note 1:* These apply regardless of the provided `policy` argument, and even if the corresponding `future` object is moved to another thread. However, it is possible for `f` not to be called at all, in which case its completion never happens. — *end note*]

If the implementation chooses the `launch::async` policy,

- (4.1) — a call to a waiting function on an asynchronous return object that shares the shared state created by this `async` call shall block until the associated thread has completed, as if joined, or else time out (32.4.3.7);
- (4.2) — the associated thread completion synchronizes with (6.10.2) the return from the first function that successfully detects the ready status of the shared state or with the return from the last function that releases the shared state, whichever happens first.

5 *Returns:* An object of type `future<invoke_result_t<decay_t<F>, decay_t<Args>...>>` that refers to the shared state created by this call to `async`.

[*Note 2:* If a future obtained from `async` is moved outside the local scope, the future's destructor can block for the shared state to become ready. — *end note*]

6 *Throws:* `system_error` if `policy == launch::async` and the implementation is unable to start a new thread, or `std::bad_alloc` if memory for the internal data structures cannot be allocated.

7 *Error conditions:*

- (7.1) — `resource_unavailable_try_again` — if `policy == launch::async` and the system is unable to start a new thread.

8 [*Example 1:*

```
int work1(int value);
int work2(int value);
int work(int value) {
 auto handle = std::async(=[]{ return work2(value); });
 int tmp = work1(value);
 return tmp + handle.get(); // #1
}
```

[*Note 3:* Line #1 might not result in concurrency because the `async` call uses the default policy, which might use `launch::deferred`, in which case the lambda might not be invoked until the `get()` call; in that case, `work1` and `work2` are called on the same thread and there is no concurrency. — *end note*]

— *end example*]

### 32.10.10 Class template `packaged_task`

[`futures.task`]

#### 32.10.10.1 General

[`futures.task.general`]

- 1 The class template `packaged_task` defines a type for wrapping a function or callable object so that the return value of the function or callable object is stored in a future when it is invoked.
- 2 When the `packaged_task` object is invoked, its stored task is invoked and the result (whether normal or exceptional) stored in the shared state. Any futures that share the shared state will then be able to access the stored result.

```
namespace std {
 template<class> class packaged_task; // not defined

 template<class R, class... ArgTypes>
 class packaged_task<R(ArgTypes...)> {
 public:
 // construction and destruction
 packaged_task() noexcept;
```

```

template<class F>
 explicit packaged_task(F&& f);
template<class F, class Allocator>
 explicit packaged_task(allocator_arg_t, const Allocator& a, F&& f);
~packaged_task();

// no copy
packaged_task(const packaged_task&) = delete;
packaged_task& operator=(const packaged_task&) = delete;

// move support
packaged_task(packaged_task&& rhs) noexcept;
packaged_task& operator=(packaged_task&& rhs) noexcept;
void swap(packaged_task& other) noexcept;

bool valid() const noexcept;

// result retrieval
future<R> get_future();

// execution
void operator()(ArgTypes...);
void make_ready_at_thread_exit(ArgTypes...);

void reset();
};

template<class R, class... ArgTypes>
 packaged_task(R (*)(ArgTypes...)) -> packaged_task<R(ArgTypes...)>;

template<class F> packaged_task(F) -> packaged_task<see below>;
}

```

### 32.10.10.2 Member functions

[futures.task.members]

packaged\_task() noexcept;

1     *Effects:* The object has no shared state and no stored task.

```

template<class F>
 explicit packaged_task(F&& f);

```

2     *Effects:* Equivalent to `packaged_task(allocator_arg, allocator<int>(), std::forward<F>(f))`.

```

template<class F, class Allocator>
 explicit packaged_task(allocator_arg_t, const Allocator& a, F&& f);

```

3     *Constraints:* `remove_cvref_t<F>` is not the same type as `packaged_task<R(ArgTypes...)>`.

4     *Mandates:* `is_invocable_r_v<R, decay_t<F>&, ArgTypes...>` is true.

5     *Preconditions:* `Allocator` meets the *Cpp17Allocator* requirements (16.4.4.6.1).

6     *Effects:* Let `A2` be `allocator_traits<Allocator>::rebind_alloc<unspecified>` and let `a2` be an object of type `A2` initialized with `A2(a)`. Constructs a new `packaged_task` object with a stored task of type `decay_t<F>` and a shared state. Initializes the object's stored task with `std::forward<F>(f)`. Uses `a2` to allocate storage for the shared state and stores a copy of `a2` in the shared state.

7     *Throws:* Any exceptions thrown by the initialization of the stored task. If storage for the shared state cannot be allocated, any exception thrown by `A2::allocate`.

```

template<class F> packaged_task(F) -> packaged_task<see below>;

```

8     *Constraints:* `&F::operator()` is well-formed when treated as an unevaluated operand (7.2.3) and either

(8.1)     — `F::operator()` is a non-static member function and `decltype(&F::operator())` is either of the form `R(G::*)(A...) cv &opt noexceptopt` or of the form `R(*) (G, A...) noexceptopt` for a type `G`, or

(8.2) — `F::operator()` is a static member function and `decltype(&F::operator())` is of the form `R(*) (A...) noexceptopt`.

9 *Remarks:* The deduced type is `packaged_task<R(A...)>`.

```
packaged_task(packaged_task&& rhs) noexcept;
```

10 *Effects:* Transfers ownership of `rhs`'s shared state to `*this`, leaving `rhs` with no shared state. Moves the stored task from `rhs` to `*this`.

11 *Postconditions:* `rhs` has no shared state.

```
packaged_task& operator=(packaged_task&& rhs) noexcept;
```

12 *Effects:* Equivalent to `packaged_task(std::move(rhs)).swap(*this)`.

13 *Returns:* `*this`.

```
~packaged_task();
```

14 *Effects:* Abandons any shared state (32.10.5).

```
void swap(packaged_task& other) noexcept;
```

15 *Effects:* Exchanges the shared states and stored tasks of `*this` and `other`.

16 *Postconditions:* `*this` has the same shared state and stored task (if any) as `other` prior to the call to `swap`. `other` has the same shared state and stored task (if any) as `*this` prior to the call to `swap`.

```
bool valid() const noexcept;
```

17 *Returns:* `true` only if `*this` has a shared state.

```
future<R> get_future();
```

18 *Synchronization:* Calls to this function do not introduce data races (6.10.2) with calls to `operator()` or `make_ready_at_thread_exit`.

[Note 1: Such calls need not synchronize with each other. — end note]

19 *Returns:* A `future` object that shares the same shared state as `*this`.

20 *Throws:* A `future_error` object if an error occurs.

21 *Error conditions:*

(21.1) — `future_already_retrieved` if `get_future` has already been called on a `packaged_task` object with the same shared state as `*this`.

(21.2) — `no_state` if `*this` has no shared state.

```
void operator()(ArgTypes... args);
```

22 *Effects:* As if by `INVOKE<R>(f, t1, t2, ..., tN)` (22.10.4), where `f` is the stored task of `*this` and `t1, t2, ..., tN` are the values in `args...`. If the task returns normally, the return value is stored as the asynchronous result in the shared state of `*this`, otherwise the exception thrown by the task is stored. The shared state of `*this` is made ready, and any threads blocked in a function waiting for the shared state of `*this` to become ready are unblocked.

23 *Throws:* A `future_error` exception object if there is no shared state or the stored task has already been invoked.

24 *Error conditions:*

(24.1) — `promise_already_satisfied` if the stored task has already been invoked.

(24.2) — `no_state` if `*this` has no shared state.

```
void make_ready_at_thread_exit(ArgTypes... args);
```

25 *Effects:* As if by `INVOKE<R>(f, t1, t2, ..., tN)` (22.10.4), where `f` is the stored task and `t1, t2, ..., tN` are the values in `args...`. If the task returns normally, the return value is stored as the asynchronous result in the shared state of `*this`, otherwise the exception thrown by the task is stored. In either case, this is done without making that state ready (32.10.5) immediately. Schedules the shared state to be made ready when the current thread exits, after all objects with thread storage duration associated with the current thread have been destroyed.

26 *Throws:* `future_error` if an error condition occurs.

27 *Error conditions:*

(27.1) — `promise_already_satisfied` if the stored task has already been invoked.

(27.2) — `no_state` if `*this` has no shared state.

```
void reset();
```

28 *Effects:* Equivalent to:

```
if (!valid()) {
 throw future_error(future_errc::no_state);
}
*this = packaged_task(allocator_arg, a, std::move(f));
```

where `f` is the task stored in `*this` and `a` is the allocator stored in the shared state.

[*Note 2:* This constructs a new shared state for `*this`. The old state is abandoned (32.10.5). — *end note*]

29 *Throws:*

(29.1) — Any exception thrown by the `packaged_task` constructor.

(29.2) — `future_error` with an error condition of `no_state` if `*this` has no shared state.

### 32.10.10.3 Globals

[`futures.task.nonmembers`]

```
template<class R, class... ArgTypes>
void swap(packaged_task<R(ArgTypes...)>& x, packaged_task<R(ArgTypes...)>& y) noexcept;
```

1 *Effects:* As if by `x.swap(y)`.

## 32.11 Safe reclamation

[`saferecl`]

### 32.11.1 General

[`saferecl.general`]

1 Subclause 32.11 contains safe-reclamation techniques, which are most frequently used to straightforwardly resolve access-deletion races.

### 32.11.2 Read-copy update (RCU)

[`saferecl.rcu`]

#### 32.11.2.1 General

[`saferecl.rcu.general`]

1 RCU is a synchronization mechanism that can be used for linked data structures that are frequently read, but seldom updated. RCU does not provide mutual exclusion, but instead allows the user to schedule specified actions such as deletion at some later time.

2 A class type `T` is *rcu-protectable* if it has exactly one base class of type `rcu_obj_base<T, D>` for some `D`, and that base is public and non-virtual, and it has no base classes of type `rcu_obj_base<X, Y>` for any other combination `X, Y`. An object is *rcu-protectable* if it is of *rcu-protectable* type.

3 An invocation of `unlock U` on an `rcu_domain dom` corresponds to an invocation of `lock L` on `dom` if `L` is sequenced before `U` and either

(3.1) — no other invocation of `lock` on `dom` is sequenced after `L` and before `U`, or

(3.2) — every invocation of `unlock U2` on `dom` such that `L` is sequenced before `U2` and `U2` is sequenced before `U` corresponds to an invocation of `lock L2` on `dom` such that `L` is sequenced before `L2` and `L2` is sequenced before `U2`.

[*Note 1:* This pairs nested locks and unlocks on a given domain in each thread. — *end note*]

4 A *region of RCU protection* on a domain `dom` starts with a `lock L` on `dom` and ends with its corresponding `unlock U`.

5 Given a region of RCU protection `R` on a domain `dom` and given an evaluation `E` that scheduled another evaluation `F` in `dom`, if `E` does not strongly happen before the start of `R`, the end of `R` strongly happens before evaluating `F`.

6 The evaluation of a scheduled evaluation is potentially concurrent with any other scheduled evaluation. Each scheduled evaluation is evaluated at most once.

## 32.11.2.2 Header &lt;rcu&gt; synopsis

[rcu.syn]

```

namespace std {
 // 32.11.2.3, class template rcu_obj_base
 template<class T, class D = default_delete<T>> class rcu_obj_base;

 // 32.11.2.4, class rcu_domain
 class rcu_domain;

 // 32.11.2.4.3, non-member functions
 rcu_domain& rcu_default_domain() noexcept;
 void rcu_synchronize(rcu_domain& dom = rcu_default_domain()) noexcept;
 void rcu_barrier(rcu_domain& dom = rcu_default_domain()) noexcept;
 template<class T, class D = default_delete<T>>
 void rcu_retire(T* p, D d = D(), rcu_domain& dom = rcu_default_domain());
}

```

## 32.11.2.3 Class template rcu\_obj\_base

[saferecl.rcu.base]

- <sup>1</sup> Objects of type T to be protected by RCU inherit from a specialization rcu\_obj\_base<T, D> for some D.

```

namespace std {
 template<class T, class D = default_delete<T>>
 class rcu_obj_base {
 public:
 void retire(D d = D(), rcu_domain& dom = rcu_default_domain()) noexcept;
 protected:
 rcu_obj_base() = default;
 rcu_obj_base(const rcu_obj_base&) = default;
 rcu_obj_base(rcu_obj_base&&) = default;
 rcu_obj_base& operator=(const rcu_obj_base&) = default;
 rcu_obj_base& operator=(rcu_obj_base&&) = default;
 ~rcu_obj_base() = default;
 private:
 D deleter; // exposition only
 };
}

```

- <sup>2</sup> The behavior of a program that adds specializations for rcu\_obj\_base is undefined.
- <sup>3</sup> T may be an incomplete type. It shall be complete before any member of the resulting specialization of rcu\_obj\_base is referenced.
- <sup>4</sup> D shall be a function object type (22.10) for which, given a value d of type D and a value ptr of type T\*, the expression d(ptr) is valid.
- <sup>5</sup> D shall meet the requirements for Cpp17DefaultConstructible and Cpp17MoveAssignable.
- <sup>6</sup> If D is trivially copyable, all specializations of rcu\_obj\_base<T, D> are trivially copyable.

```
void retire(D d = D(), rcu_domain& dom = rcu_default_domain()) noexcept;
```

- <sup>7</sup> *Mandates:* T is an rcu-protectable type.
- <sup>8</sup> *Preconditions:* \*this is a base class subobject of an object x of type T. The member function rcu\_obj\_base<T, D>::retire was not invoked on x before. The assignment to deleter does not exit via an exception.
- <sup>9</sup> *Effects:* Evaluates deleter = std::move(d) and schedules the evaluation of the expression deleter(addressof(x)) in the domain dom; the behavior is undefined if that evaluation exits via an exception. May invoke scheduled evaluations in dom.
- [Note 1: If such evaluations acquire resources held across any invocation of retire on dom, deadlock can occur. — end note]

## 32.11.2.4 Class rcu\_domain

[saferecl.rcu.domain]

## 32.11.2.4.1 General

[saferecl.rcu.domain.general]

```

namespace std {
 class rcu_domain {
 public:

```

```

 rcu_domain(const rcu_domain&) = delete;
 rcu_domain& operator=(const rcu_domain&) = delete;

 void lock() noexcept;
 bool try_lock() noexcept;
 void unlock() noexcept;
};
}

```

<sup>1</sup> This class meets the requirements of *Cpp17Lockable* (32.2.5.3) and provides regions of RCU protection.

[Example 1:

```
std::scoped_lock<rcu_domain> rlock(rcu_default_domain());
```

— end example]

<sup>2</sup> The functions `lock` and `unlock` establish (possibly nested) regions of RCU protection.

#### 32.11.2.4.2 Member functions

[saferecl.rcu.domain.members]

```
void lock() noexcept;
```

<sup>1</sup> *Effects:* Opens a region of RCU protection.

<sup>2</sup> *Remarks:* Calls to `lock` do not introduce a data race (6.10.2.2) involving `*this`.

```
bool try_lock() noexcept;
```

<sup>3</sup> *Effects:* Equivalent to `lock()`.

<sup>4</sup> *Returns:* `true`.

```
void unlock() noexcept;
```

<sup>5</sup> *Preconditions:* A call to `lock` that opened an unclosed region of RCU protection is sequenced before the call to `unlock`.

<sup>6</sup> *Effects:* Closes the unclosed region of RCU protection that was most recently opened. May invoke scheduled evaluations in `*this`.

<sup>7</sup> [Note 1: If such evaluations acquire resources held across any invocation of `unlock` on `*this`, deadlock can occur. — end note]

<sup>8</sup> *Remarks:* Calls to `unlock` do not introduce a data race involving `*this`.

[Note 2: Evaluation of scheduled evaluations can still cause a data race. — end note]

#### 32.11.2.4.3 Non-member functions

[saferecl.rcu.domain.func]

```
rcu_domain& rcu_default_domain() noexcept;
```

<sup>1</sup> *Returns:* A reference to a static-duration object of type `rcu_domain`. A reference to the same object is returned every time this function is called.

```
void rcu_synchronize(rcu_domain& dom = rcu_default_domain()) noexcept;
```

<sup>2</sup> *Effects:* If the call to `rcu_synchronize` does not strongly happen before the lock opening an RCU protection region `R` on `dom`, blocks until the `unlock` closing `R` happens.

<sup>3</sup> *Synchronization:* The `unlock` closing `R` strongly happens before the return from `rcu_synchronize`.

```
void rcu_barrier(rcu_domain& dom = rcu_default_domain()) noexcept;
```

<sup>4</sup> *Effects:* May evaluate any scheduled evaluations in `dom`. For any evaluation that happens before the call to `rcu_barrier` and that schedules an evaluation `E` in `dom`, blocks until `E` has been evaluated.

<sup>5</sup> *Synchronization:* The evaluation of any such `E` strongly happens before the return from `rcu_barrier`.

[Note 1: A call to `rcu_barrier` does not imply a call to `rcu_synchronize` and vice versa. — end note]

```
template<class T, class D = default_delete<T>>
```

```
void rcu_retire(T* p, D d = D(), rcu_domain& dom = rcu_default_domain());
```

<sup>6</sup> *Mandates:* `is_move_constructible_v<D>` is `true` and the expression `d(p)` is well-formed.

<sup>7</sup> *Preconditions:* `D` meets the *Cpp17MoveConstructible* and *Cpp17Destructible* requirements.

8 *Effects:* May allocate memory. It is unspecified whether the memory allocation is performed by invoking operator `new`. Initializes an object `d1` of type `D` from `std::move(d)`. Schedules the evaluation of `d1(p)` in the domain `dom`; the behavior is undefined if that evaluation exits via an exception. May invoke scheduled evaluations in `dom`.

[Note 2: If `rcu_retire` exits via an exception, no evaluation is scheduled. — end note]

9 *Throws:* `bad_alloc` or any exception thrown by the initialization of `d1`.

10 [Note 3: If scheduled evaluations acquire resources held across any invocation of `rcu_retire` on `dom`, deadlock can occur. — end note]

### 32.11.3 Hazard pointers

[saferecl.hp]

#### 32.11.3.1 General

[saferecl.hp.general]

1 A hazard pointer is a single-writer multi-reader pointer that can be owned by at most one thread at any time. Only the owner of the hazard pointer can set its value, while any number of threads may read its value. The owner thread sets the value of a hazard pointer to point to an object in order to indicate to concurrent threads—which may delete such an object—that the object is not yet safe to delete.

2 A class type `T` is *hazard-protectable* if it has exactly one base class of type `hazard_pointer_obj_base<T, D>` for some `D`, that base is public and non-virtual, and it has no base classes of type `hazard_pointer_obj_base<T2, D2>` for any other combination `T2, D2`. An object is *hazard-protectable* if it is of hazard-protectable type.

3 The time span between creation and destruction of a hazard pointer `h` is partitioned into a series of *protection epochs*; in each protection epoch, `h` either is *associated with* a hazard-protectable object, or is *unassociated*. Upon creation, a hazard pointer is unassociated. Changing the association (possibly to the same object) initiates a new protection epoch and ends the preceding one.

4 An object `x` of hazard-protectable type `T` is *retired* with a deleter of type `D` when the member function `hazard_pointer_obj_base<T, D>::retire` is invoked on `x`. Any given object `x` shall be retired at most once.

5 A retired object `x` is *reclaimed* by invoking its deleter with a pointer to `x`; the behavior is undefined if that invocation exits via an exception.

6 A hazard-protectable object `x` is *possibly-reclaimable* with respect to an evaluation `A` if

(6.1) — `x` is not reclaimed; and

(6.2) — `x` is retired in an evaluation `R` and `A` does not happen before `R`; and

(6.3) — for all hazard pointers `h` and for every protection epoch `E` of `h` during which `h` is associated with `x`:

(6.3.1) — if the beginning of `E` happens before `R`, the end of `E` strongly happens before `A`; and

(6.3.2) — if `E` began by an evaluation of `try_protect` with argument `src`, label its atomic load operation `L`. If there exists an atomic modification `B` on `src` such that `L` observes a modification that is modification-ordered before `B`, and `B` happens before `x` is retired, the end of `E` strongly happens before `A`.

[Note 1: In typical use, a store to `src` sequenced before retiring `x` will be such an atomic operation `B`. — end note]

[Note 2: The latter two conditions convey the informal notion that a protection epoch that began before retiring `x`, as implied either by the happens-before relation or the coherence order of some source, delays the reclamation of `x`. — end note]

7 The number of possibly-reclaimable objects has an unspecified bound.

[Note 3: The bound can be a function of the number of hazard pointers, the number of threads that retire objects, and the number of threads that use hazard pointers. — end note]

[Example 1: The following example shows how hazard pointers allow updates to be carried out in the presence of concurrent readers. The object of type `hazard_pointer` in `print_name` protects the object `*ptr` from being reclaimed by `ptr->retire` until the end of the protection epoch.

```
struct Name : public hazard_pointer_obj_base<Name> { /* details */ };
atomic<Name*> name;
// called often and in parallel!
void print_name() {
 hazard_pointer h = make_hazard_pointer();
```

```

 Name* ptr = h.protect(name); // Protection epoch starts
 // ... safe to access *ptr
} // Protection epoch ends.

```

```

// called rarely, but possibly concurrently with print_name
void update_name(Name* new_name) {
 Name* ptr = name.exchange(new_name);
 ptr->retire();
}

```

— end example]

### 32.11.3.2 Header <hazard\_pointer> synopsis

[hazard.pointer.syn]

```

namespace std {
 // 32.11.3.3, class template hazard_pointer_obj_base
 template<class T, class D = default_delete<T>> class hazard_pointer_obj_base;

 // 32.11.3.4, class hazard_pointer
 class hazard_pointer;

 // 32.11.3.4.4, non-member functions
 hazard_pointer make_hazard_pointer();
 void swap(hazard_pointer&, hazard_pointer&) noexcept;
 void make_hazard_pointer_batch(span<hazard_pointer>);
 void clear_hazard_pointer_batch(span<hazard_pointer>) noexcept;
}

```

### 32.11.3.3 Class template hazard\_pointer\_obj\_base

[saferecl.hp.base]

```

namespace std {
 template<class T, class D = default_delete<T>>
 class hazard_pointer_obj_base {
 public:
 void retire(D d = D()) noexcept;
 protected:
 hazard_pointer_obj_base() = default;
 hazard_pointer_obj_base(const hazard_pointer_obj_base&) = default;
 hazard_pointer_obj_base(hazard_pointer_obj_base&&) = default;
 hazard_pointer_obj_base& operator=(const hazard_pointer_obj_base&) = default;
 hazard_pointer_obj_base& operator=(hazard_pointer_obj_base&&) = default;
 ~hazard_pointer_obj_base() = default;
 private:
 D deleter; // exposition only
 };
}

```

- <sup>1</sup> D shall be a function object type (22.10.4) for which, given a value d of type D and a value ptr of type T\*, the expression d(ptr) is valid.
- <sup>2</sup> The behavior of a program that adds specializations for hazard\_pointer\_obj\_base is undefined.
- <sup>3</sup> D shall meet the requirements for Cpp17DefaultConstructible and Cpp17MoveAssignable.
- <sup>4</sup> T may be an incomplete type. It shall be complete before any member of the resulting specialization of hazard\_pointer\_obj\_base is referenced.

```
void retire(D d = D()) noexcept;
```

- <sup>5</sup> *Mandates:* T is a hazard-protectable type.
- <sup>6</sup> *Preconditions:* \*this is a base class subobject of an object x of type T. x is not retired. Move-assigning d to deleter does not exit via an exception.
- <sup>7</sup> *Effects:* Move-assigns d to deleter, thereby setting it as the deleter of x, then retires x. May reclaim possibly-reclaimable objects.



**32.11.3.4 Class hazard\_pointer**

[saferecl.hp.holder]

**32.11.3.4.1 General**

[saferecl.hp.holder.general]

```

namespace std {
 class hazard_pointer {
 public:
 hazard_pointer() noexcept;
 hazard_pointer(hazard_pointer&&) noexcept;
 hazard_pointer& operator=(hazard_pointer&&) noexcept;
 ~hazard_pointer();

 bool empty() const noexcept;
 template<class T> T* protect(const atomic<T*>& src) noexcept;
 template<class T> bool try_protect(T*& ptr, const atomic<T*>& src) noexcept;
 template<class T> void reset_protection(const T* ptr) noexcept;
 void reset_protection(nullptr_t = nullptr) noexcept;
 void swap(hazard_pointer&) noexcept;
 };
}

```

- <sup>1</sup> An object of type `hazard_pointer` is either empty or *owns* a hazard pointer. Each hazard pointer is owned by exactly one object of type `hazard_pointer`.

[Note 1: An empty `hazard_pointer` object is different from a `hazard_pointer` object that owns an unassociated hazard pointer. An empty `hazard_pointer` object does not own any hazard pointers. — end note]

**32.11.3.4.2 Constructors, destructor, and assignment**

[saferecl.hp.holder.ctor]

```
hazard_pointer() noexcept;
```

- <sup>1</sup> *Postconditions:* `*this` is empty.

```
hazard_pointer(hazard_pointer&& other) noexcept;
```

- <sup>2</sup> *Postconditions:* If `other` is empty, `*this` is empty. Otherwise, `*this` owns the hazard pointer originally owned by `other`; `other` is empty.

```
~hazard_pointer();
```

- <sup>3</sup> *Effects:* If `*this` is not empty, destroys the hazard pointer owned by `*this`, thereby ending its current protection epoch.

```
hazard_pointer& operator=(hazard_pointer&& other) noexcept;
```

- <sup>4</sup> *Effects:* If `this == &other` is true, no effect. Otherwise, if `*this` is not empty, destroys the hazard pointer owned by `*this`, thereby ending its current protection epoch.

- <sup>5</sup> *Postconditions:* If `other` was empty, `*this` is empty. Otherwise, `*this` owns the hazard pointer originally owned by `other`. If `this != &other` is true, `other` is empty.

- <sup>6</sup> *Returns:* `*this`.

**32.11.3.4.3 Member functions**

[saferecl.hp.holder.mem]

```
bool empty() const noexcept;
```

- <sup>1</sup> *Returns:* true if and only if `*this` is empty.

```
template<class T> T* protect(const atomic<T*>& src) noexcept;
```

- <sup>2</sup> *Effects:* Equivalent to:

```

 T* ptr = src.load(memory_order::relaxed);
 while (!try_protect(ptr, src)) {}
 return ptr;

```

```
template<class T> bool try_protect(T*& ptr, const atomic<T*>& src) noexcept;
```

- <sup>3</sup> *Mandates:* T is a hazard-protectable type.

- <sup>4</sup> *Preconditions:* `*this` is not empty.

- <sup>5</sup> *Effects:* Performs the following steps in order:

(5.1) — Initializes a variable `old` of type `T*` with the value of `ptr`.

(5.2) — Evaluates `reset_protection(old)`.

(5.3) — Assigns the value of `src.load(memory_order::acquire)` to `ptr`.

(5.4) — If `old == ptr` is false, evaluates `reset_protection()`.

6 *Returns:* `old == ptr`.

```
template<class T> void reset_protection(const T* ptr) noexcept;
```

7 *Mandates:* `T` is a hazard-protectable type.

8 *Preconditions:* `*this` is not empty.

9 *Effects:* If `ptr` is a null pointer value, invokes `reset_protection()`. Otherwise, associates the hazard pointer owned by `*this` with `*ptr`, thereby ending the current protection epoch.

10 *Complexity:* Constant.

```
void reset_protection(nullptr_t = nullptr) noexcept;
```

11 *Preconditions:* `*this` is not empty.

12 *Postconditions:* The hazard pointer owned by `*this` is unassociated.

13 *Complexity:* Constant.

```
void swap(hazard_pointer& other) noexcept;
```

14 *Effects:* Swaps the hazard pointer ownership of this object with that of `other`.

[*Note 1:* The owned hazard pointers, if any, remain unchanged during the swap and continue to be associated with the respective objects that they were protecting before the swap, if any. No protection epochs are ended or initiated. — *end note*]

15 *Complexity:* Constant.

#### 32.11.3.4.4 Non-member functions

[saferecl.hp.holder.nonmem]

```
hazard_pointer make_hazard_pointer();
```

1 *Effects:* Constructs a hazard pointer.

2 *Returns:* A `hazard_pointer` object that owns the newly-constructed hazard pointer.

3 *Throws:* May throw `bad_alloc` if memory for the hazard pointer could not be allocated.

```
void swap(hazard_pointer& a, hazard_pointer& b) noexcept;
```

4 *Effects:* Equivalent to `a.swap(b)`.

```
void make_hazard_pointer_batch(span<hazard_pointer> batch);
```

5 *Effects:* If an exception is thrown, there are no effects. Otherwise, for each element `e` of `batch` such that `e.empty()` is true, constructs a hazard pointer and makes `e` the owner of that hazard pointer.

6 *Throws:* `bad_alloc` if memory for any of the newly-constructed hazard pointers could not be allocated.

```
void clear_hazard_pointer_batch(span<hazard_pointer> batch) noexcept;
```

7 *Effects:* For each element `e` of `batch`, if `e.empty()` is false, destroys the hazard pointer owned by `e` and makes `e` empty.

## 33 Execution control library [exec]

### 33.1 General [exec.general]

- <sup>1</sup> This Clause describes components supporting execution of function objects (22.10).
- <sup>2</sup> The following subclauses describe the requirements, concepts, and components for execution control primitives as summarized in Table 157.

**Table 157 — Execution control library summary** [tab:exec.summary]

| Subclause                             | Header      |
|---------------------------------------|-------------|
| <a href="#">33.6</a> Schedulers       | <execution> |
| <a href="#">33.7</a> Receivers        |             |
| <a href="#">33.8</a> Operation states |             |
| <a href="#">33.9</a> Senders          |             |

- <sup>3</sup> Table 158 shows the types of customization point objects (16.3.3.3.5) used in the execution control library.

**Table 158 — Types of customization point objects in the execution control library**  
[tab:exec.pos]

| Customization point object type | Purpose                                                                                    | Examples                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| core                            | provide core execution functionality, and connection between core components               | e.g., <code>connect</code> , <code>start</code>                                                                                                                                                                                                                                                                                                                                                                 |
| completion functions            | called by senders to announce the completion of the work (success, error, or cancellation) | <code>set_value</code> , <code>set_error</code> , <code>set_stopped</code>                                                                                                                                                                                                                                                                                                                                      |
| senders                         | allow the specialization of the provided sender algorithms                                 | <ul style="list-style-type: none"> <li>— sender factories (e.g., <code>schedule</code>, <code>just</code>, <code>read_env</code>)</li> <li>— sender adaptors (e.g., <code>continues_on</code>, <code>then</code>, <code>let_value</code>)</li> <li>— sender consumers (e.g., <code>sync_wait</code>)</li> </ul>                                                                                                 |
| queries                         | allow querying different properties of objects                                             | <ul style="list-style-type: none"> <li>— general queries (e.g., <code>get_allocator</code>, <code>get_stop_token</code>)</li> <li>— environment queries (e.g., <code>get_scheduler</code>, <code>get_delegation_scheduler</code>)</li> <li>— scheduler queries (e.g., <code>get_forward_progress_guarantee</code>)</li> <li>— sender attribute queries (e.g., <code>get_completion_scheduler</code>)</li> </ul> |

- <sup>4</sup> This clause makes use of the following exposition-only entities.
- <sup>5</sup> For a subexpression `expr`, let `MANDATE-NOTHROW(expr)` be expression-equivalent to `expr`.  
*Mandates:* `noexcept(expr)` is true.

```

6 namespace std {
 template<class T>
 concept movable-value = // exposition only
 move_constructible<decay_t<T>> &&
 constructible_from<decay_t<T>, T> &&
 (!is_array_v<remove_reference_t<T>>);
}

```

7 For function types  $F1$  and  $F2$  denoting  $R1(\text{Args1} \dots)$  and  $R2(\text{Args2} \dots)$ , respectively, *MATCHING-SIG* ( $F1$ ,  $F2$ ) is true if and only if `same_as< $R1(\text{Args1} \dots)$ ,  $R2(\text{Args2} \dots)$ >` is true.

8 For a subexpression `err`, let `Err` be `decltype((err))` and let *AS-EXCEPT-PTR* (`err`) be:

(8.1) — `err` if `decay_t<Err>` denotes the type `exception_ptr`.

*Preconditions:* `!err` is false.

(8.2) — Otherwise, `make_exception_ptr(system_error(err))` if `decay_t<Err>` denotes the type `error_code`.

(8.3) — Otherwise, `make_exception_ptr(err)`.

9 For a subexpression `expr`, let *AS-CONST* (`expr`) be expression-equivalent to

```

[] (const auto& x) noexcept -> const auto& { return x; } (expr)

```

### 33.2 Queries and queryables

[exec.queryable]

#### 33.2.1 General

[exec.queryable.general]

1 A *queryable object* is a read-only collection of key/value pair where each key is a customization point object known as a *query object*. A *query* is an invocation of a query object with a queryable object as its first argument and a (possibly empty) set of additional arguments. A query imposes syntactic and semantic requirements on its invocations.

2 Let  $q$  be a query object, let  $\text{args}$  be a (possibly empty) pack of subexpressions, let  $\text{env}$  be a subexpression that refers to a queryable object  $o$  of type  $O$ , and let  $\text{cenv}$  be a subexpression referring to  $o$  such that `decltype((cenv))` is `const O&`. The expression  $q(\text{env}, \text{args} \dots)$  is equal to (18.2) the expression  $q(\text{cenv}, \text{args} \dots)$ .

3 The type of a query expression cannot be `void`.

4 The expression  $q(\text{env}, \text{args} \dots)$  is equality-preserving (18.2) and does not modify the query object or the arguments.

5 If the expression  $\text{env.query}(q, \text{args} \dots)$  is well-formed, then it is expression-equivalent to  $q(\text{env}, \text{args} \dots)$ .

6 Unless otherwise specified, the result of a query is valid as long as the queryable object is valid.

#### 33.2.2 queryable concept

[exec.queryable.concept]

```

namespace std {
 template<class T>
 concept queryable = destructible<T>; // exposition only
}

```

1 The exposition-only *queryable* concept specifies the constraints on the types of queryable objects.

2 Let  $\text{env}$  be an object of type  $\text{Env}$ . The type  $\text{Env}$  models *queryable* if for each callable object  $q$  and a pack of subexpressions  $\text{args}$ , if `requires {  $q(\text{env}, \text{args} \dots)$  }` is true then  $q(\text{env}, \text{args} \dots)$  meets any semantic requirements imposed by  $q$ .

### 33.3 Asynchronous operations

[exec.async.ops]

1 An *execution resource* is a program entity that manages a (possibly dynamic) set of execution agents (32.2.5.1), which it uses to execute parallel work on behalf of callers.

[Example 1: The currently active thread, a system-provided thread pool, and uses of an API associated with an external hardware accelerator are all examples of execution resources. — end example]

Execution resources execute asynchronous operations. An execution resource is either valid or invalid.

2 An *asynchronous operation* is a distinct unit of program execution that

(2.1) — is explicitly created;

- (2.2) — can be explicitly started once at most;
- (2.3) — once started, eventually completes exactly once with a (possibly empty) set of result datums and in exactly one of three *dispositions*: success, failure, or cancellation;
- (2.3.1) — A successful completion, also known as a *value completion*, can have an arbitrary number of result datums.
- (2.3.2) — A failure completion, also known as an *error completion*, has a single result datum.
- (2.3.3) — A cancellation completion, also known as a *stopped completion*, has no result datum.

An asynchronous operation's *async result* is its disposition and its (possibly empty) set of result datums.

- (2.4) — can complete on a different execution resource than the execution resource on which it started; and
- (2.5) — can create and start other asynchronous operations called *child operations*. A child operation is an asynchronous operation that is created by the parent operation and, if started, completes before the parent operation completes. A *parent operation* is the asynchronous operation that created a particular child operation.

[Note 1: An asynchronous operation can execute synchronously; that is, it can complete during the execution of its start operation on the thread of execution that started it. — end note]

- 3 An asynchronous operation has associated state known as its *operation state*.
- 4 An asynchronous operation has an associated environment. An *environment* is a queryable object (33.2) representing the execution-time properties of the operation's caller. The caller of an asynchronous operation is its parent operation or the function that created it.
- 5 An asynchronous operation has an associated receiver. A *receiver* is an aggregation of three handlers for the three asynchronous completion dispositions:
  - (5.1) — a value completion handler for a value completion,
  - (5.2) — an error completion handler for an error completion, and
  - (5.3) — a stopped completion handler for a stopped completion.

A receiver has an associated environment. An asynchronous operation's operation state owns the operation's receiver. The environment of an asynchronous operation is equal to its receiver's environment.

- 6 For each completion disposition, there is a *completion function*. A completion function is a customization point object (16.3.3.3.5) that accepts an asynchronous operation's receiver as the first argument and the result datums of the asynchronous operation as additional arguments. The value completion function invokes the receiver's value completion handler with the value result datums; likewise for the error completion function and the stopped completion function. A completion function has an associated type known as its *completion tag* that is the unqualified type of the completion function. A valid invocation of a completion function is called a *completion operation*.
- 7 The *lifetime of an asynchronous operation*, also known as the operation's *async lifetime*, begins when its start operation begins executing and ends when its completion operation begins executing. If the lifetime of an asynchronous operation's associated operation state ends before the lifetime of the asynchronous operation, the behavior is undefined. After an asynchronous operation executes a completion operation, its associated operation state is invalid. Accessing any part of an invalid operation state is undefined behavior.
- 8 An asynchronous operation shall not execute a completion operation before its start operation has begun executing. After its start operation has begun executing, exactly one completion operation shall execute. The lifetime of an asynchronous operation's operation state can end during the execution of the completion operation.
- 9 A *sender* is a factory for one or more asynchronous operations. *Connecting* a sender and a receiver creates an asynchronous operation. The asynchronous operation's associated receiver is equal to the receiver used to create it, and its associated environment is equal to the environment associated with the receiver used to create it. The lifetime of an asynchronous operation's associated operation state does not depend on the lifetimes of either the sender or the receiver from which it was created. A sender is started when it is connected to a receiver and the resulting asynchronous operation is started. A sender's *async result* is the *async result* of the asynchronous operation created by connecting it to a receiver. A sender sends its results by way of the asynchronous operation(s) it produces, and a receiver receives those results. A sender is either valid or invalid; it becomes invalid when its parent sender (see below) becomes invalid.

- <sup>10</sup> A *scheduler* is an abstraction of an execution resource with a uniform, generic interface for scheduling work onto that resource. It is a factory for senders whose asynchronous operations execute value completion operations on an execution agent belonging to the scheduler's associated execution resource. A *schedule-expression* obtains such a sender from a scheduler. A *schedule sender* is the result of a schedule expression. On success, an asynchronous operation produced by a schedule sender executes a value completion operation with an empty set of result datums. Multiple schedulers can refer to the same execution resource. A scheduler can be valid or invalid. A scheduler becomes invalid when the execution resource to which it refers becomes invalid, as do any schedule senders obtained from the scheduler, and any operation states obtained from those senders.
- <sup>11</sup> An asynchronous operation has one or more associated completion schedulers for each of its possible dispositions. A *completion scheduler* is a scheduler whose associated execution resource is used to execute a completion operation for an asynchronous operation. A value completion scheduler is a scheduler on which an asynchronous operation's value completion operation can execute. Likewise for error completion schedulers and stopped completion schedulers.
- <sup>12</sup> A sender has an associated queryable object (33.2) known as its *attributes* that describes various characteristics of the sender and of the asynchronous operation(s) it produces. For each disposition, there is a query object for reading the associated completion scheduler from a sender's attributes; i.e., a value completion scheduler query object for reading a sender's value completion scheduler, etc. If a completion scheduler query is well-formed, the returned completion scheduler is unique for that disposition for any asynchronous operation the sender creates. A schedule sender is required to have a value completion scheduler attribute whose value is equal to the scheduler that produced the schedule sender.
- <sup>13</sup> A *completion signature* is a function type that describes a completion operation. An asynchronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates (6.3). For a completion function `set`, receiver `rcvr`, and pack of arguments `args`, let `c` be the completion operation `set(rcvr, args...)`, and let `F` be the function type `decltype(auto(set))(decltype((args))...)`. A completion signature `Sig` is associated with `c` if and only if `MATCHING-SIG(Sig, F)` is `true` (33.1). Together, a sender type and an environment type `Env` determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type `Env`. The type of the receiver does not affect an asynchronous operation's completion signatures, only the type of the receiver's environment. A *non-dependent sender* is a sender type whose completion signatures are knowable independent of an execution environment.
- <sup>14</sup> A sender algorithm is a function that takes and/or returns a sender. There are three categories of sender algorithms:
- (14.1) — A *sender factory* is a function that takes non-senders as arguments and that returns a sender.
  - (14.2) — A *sender adaptor* is a function that constructs and returns a parent sender from a set of one or more child senders and a (possibly empty) set of additional arguments. An asynchronous operation created by a parent sender is a parent operation to the child operations created by the child senders.
  - (14.3) — A *sender consumer* is a function that takes one or more senders and a (possibly empty) set of additional arguments, and whose return type is not the type of a sender.

### 33.4 Header <execution> synopsis

[execution.syn]

```

namespace std {
 // 26.3.6.2, execution policy type trait
 template<class T> struct is_execution_policy; // freestanding
 template<class T> constexpr bool is_execution_policy_v = // freestanding
 is_execution_policy<T>::value;
}

namespace std::execution {
 // 26.3.6.3, sequenced execution policy
 class sequenced_policy;

 // 26.3.6.4, parallel execution policy
 class parallel_policy;

 // 26.3.6.5, parallel and unsequenced execution policy
 class parallel_unsequenced_policy;

```

```

// 26.3.6.6, unsequenced execution policy
class unsequenced_policy;

// 26.3.6.7, execution policy objects
inline constexpr sequenced_policy seq{ unspecified };
inline constexpr parallel_policy par{ unspecified };
inline constexpr parallel_unsequenced_policy par_unseq{ unspecified };
inline constexpr unsequenced_policy unseq{ unspecified };
}

namespace std {
// 33.1, helper concepts
template<class T>
 concept movable-value = see below; // exposition only

template<class From, class To>
 concept decays-to = same_as<decay_t<From>, To>; // exposition only

template<class T>
 concept class-type = decays-to<T, T> && is_class_v<T>; // exposition only

// 33.2, queryable objects
template<class T>
 concept queryable = see below; // exposition only

// 33.5, queries
struct forwarding_query_t { unspecified };
struct get_allocator_t { unspecified };
struct get_stop_token_t { unspecified };

inline constexpr forwarding_query_t forwarding_query{};
inline constexpr get_allocator_t get_allocator{};
inline constexpr get_stop_token_t get_stop_token{};

template<class T>
 using stop_token_of_t = remove_cvref_t<decltype(get_stop_token(declval<T>()))>;

template<class T>
 concept forwarding-query = forwarding_query(T{}); // exposition only
}

namespace std::execution {
// 33.5, queries
struct get_domain_t { unspecified };
struct get_scheduler_t { unspecified };
struct get_start_scheduler_t { unspecified };
struct get_delegation_scheduler_t { unspecified };
struct get_forward_progress_guarantee_t { unspecified };
template<class CP0>
 struct get_completion_scheduler_t { unspecified };
template<class CP0 = void>
 struct get_completion_domain_t { unspecified };
struct get_await_completion_adaptor_t { unspecified };

inline constexpr get_domain_t get_domain{};
inline constexpr get_scheduler_t get_scheduler{};
inline constexpr get_start_scheduler_t get_start_scheduler{};
inline constexpr get_delegation_scheduler_t get_delegation_scheduler{};
enum class forward_progress_guarantee;
inline constexpr get_forward_progress_guarantee_t get_forward_progress_guarantee{};
template<class CP0>
 constexpr get_completion_scheduler_t<CP0> get_completion_scheduler{};
template<class CP0 = void>
 constexpr get_completion_domain_t<CP0> get_completion_domain{};
}

```

```

inline constexpr get_await_completion_adaptor_t get_await_completion_adaptor{};

struct get_env_t { unspecified };
inline constexpr get_env_t get_env{};

template<class T>
 using env_of_t = decltype(get_env(declval<T>()));

// 33.9.5, execution domains
template<class... Domains>
 struct indeterminate_domain;

// 33.9.6, execution domains
struct default_domain;

// 33.6, schedulers
struct scheduler_tag {};

template<class Sch>
 concept scheduler = see below;

// 33.7, receivers
struct receiver_tag {};

template<class Rcvr>
 concept receiver = see below;

template<class Rcvr, class Completions>
 concept receiver_of = see below; // exposition only

template<class Rcvr, class ChildOp>
 concept inlinable_receiver = see below;

struct set_value_t { unspecified };
struct set_error_t { unspecified };
struct set_stopped_t { unspecified };

inline constexpr set_value_t set_value{};
inline constexpr set_error_t set_error{};
inline constexpr set_stopped_t set_stopped{};

// 33.8, operation states
struct operation_state_tag {};

template<class O>
 concept operation_state = see below;

struct start_t;
inline constexpr start_t start{};

// 33.9, senders
struct sender_tag {};

template<class Sndr>
 inline constexpr bool enable_sender = see below;

template<class Sndr>
 concept sender = see below;

template<class Sndr, class... Env>
 concept sender_in = see below;

template<class Sndr>
 concept dependent_sender = see below;

```



```

template<class Sndr, class Rcvr>
 concept sender-to = see below; // exposition only

template<class... Ts>
 struct type-list; // exposition only

template<class... Ts>
 using decayed-tuple = tuple<decay_t<Ts>...>; // exposition only

template<class... Ts>
 using variant-or-empty = see below; // exposition only

template<class Sndr, class Env = env<>,
 template<class...> class Tuple = decayed-tuple,
 template<class...> class Variant = variant-or-empty>
 requires sender_in<Sndr, Env>
 using value_types_of_t = see below;

template<class Sndr, class Env = env<>,
 template<class...> class Variant = variant-or-empty>
 requires sender_in<Sndr, Env>
 using error_types_of_t = see below;

template<class Sndr, class Env = env<>>
 requires sender_in<Sndr, Env>
 constexpr bool sends_stopped = see below;

template<class Sndr, class... Env>
 using single-sender-value-type = see below; // exposition only

template<class Sndr, class... Env>
 concept single-sender = see below; // exposition only

template<sender Sndr>
 using tag_of_t = see below;

// 33.9.7, sender transformations
template<sender Sndr, queryable Env>
 constexpr decltype(auto) transform_sender(Sndr&& sndr,
 const Env& env) noexcept(see below);

// 33.9.8, sender algorithm application
template<class Domain, class Tag, sender Sndr, class... Args>
 constexpr decltype(auto) apply_sender(
 Domain dom, Tag, Sndr&& sndr, Args&&... args) noexcept(see below);

// 33.9.9, get completion signatures
template<class Sndr, class... Env>
 constexpr auto get_completion_signatures() -> valid-completion-signatures auto;

template<class Sndr, class... Env>
 requires sender_in<Sndr, Env...>
 using completion_signatures_of_t = decltype(get_completion_signatures<Sndr, Env...>());

// 33.9.10, the connect sender algorithm
struct connect_t;
inline constexpr connect_t connect{};

template<class Sndr, class Rcvr>
 using connect_result_t =
 decltype(connect(declval<Sndr>(), declval<Rcvr>()));

// 33.9.11, sender factories
struct just_t { unspecified };

```

```

struct just_error_t { unspecified };
struct just_stopped_t { unspecified };
struct schedule_t { unspecified };

inline constexpr just_t just{};
inline constexpr just_error_t just_error{};
inline constexpr just_stopped_t just_stopped{};
inline constexpr schedule_t schedule{};
inline constexpr unspecified read_env{};

template<scheduler Sch>
 using schedule_result_t = decltype(schedule(declval<Sch>()));

// 33.9.12, sender adaptors
template<class-type D>
 struct sender_adaptor_closure { };

struct starts_on_t { unspecified };
struct continues_on_t { unspecified };
struct on_t { unspecified };
struct schedule_from_t { unspecified };
struct then_t { unspecified };
struct upon_error_t { unspecified };
struct upon_stopped_t { unspecified };
struct let_value_t { unspecified };
struct let_error_t { unspecified };
struct let_stopped_t { unspecified };
struct bulk_t { unspecified };
struct bulk_chunked_t { unspecified };
struct bulk_unchunked_t { unspecified };
struct when_all_t { unspecified };
struct when_all_with_variant_t { unspecified };
struct into_variant_t { unspecified };
struct stopped_as_optional_t { unspecified };
struct stopped_as_error_t { unspecified };
struct associate_t { unspecified };
struct spawn_future_t { unspecified };

inline constexpr unspecified write_env{};
inline constexpr unspecified unstoppable{};
inline constexpr starts_on_t starts_on{};
inline constexpr continues_on_t continues_on{};
inline constexpr on_t on{};
inline constexpr schedule_from_t schedule_from{};
inline constexpr then_t then{};
inline constexpr upon_error_t upon_error{};
inline constexpr upon_stopped_t upon_stopped{};
inline constexpr let_value_t let_value{};
inline constexpr let_error_t let_error{};
inline constexpr let_stopped_t let_stopped{};
inline constexpr bulk_t bulk{};
inline constexpr bulk_chunked_t bulk_chunked{};
inline constexpr bulk_unchunked_t bulk_unchunked{};
inline constexpr when_all_t when_all{};
inline constexpr when_all_with_variant_t when_all_with_variant{};
inline constexpr into_variant_t into_variant{};
inline constexpr stopped_as_optional_t stopped_as_optional{};
inline constexpr stopped_as_error_t stopped_as_error{};
inline constexpr associate_t associate{};
inline constexpr spawn_future_t spawn_future{};
}

```

```

namespace std::this_thread {
 // 33.9.13, consumers
 struct sync_wait_t { unspecified };
 struct sync_wait_with_variant_t { unspecified };

 inline constexpr sync_wait_t sync_wait{};
 inline constexpr sync_wait_with_variant_t sync_wait_with_variant{};
}

namespace std::execution {
 // 33.9.13, consumers
 struct spawn_t { unspecified };
 inline constexpr spawn_t spawn{};

 // 33.10, completion signatures
 template<class Fn>
 concept completion_signature = see below; // exposition only

 template<completion_signature... Fns>
 struct completion_signatures;

 template<class Sigs>
 concept valid_completion_signatures = see below; // exposition only

 struct dependent_sender_error : exception {};

 // 33.11.1, class template prop
 template<class QueryTag, class ValueType>
 struct prop;

 // 33.11.2, class template env
 template<queryable... Envs>
 struct env;

 // 33.12.1, run_loop
 class run_loop;

 // 33.13.1, coroutine utility as_awaitable
 struct as_awaitable_t { unspecified };
 inline constexpr as_awaitable_t as_awaitable{};

 // 33.13.2, coroutine utility with_awaitable_senders
 template<class-type Promise>
 struct with_awaitable_senders;

 // 33.13.3, coroutine utility affine
 struct affine_t { unspecified };
 inline constexpr affine_t affine{};

 // 33.13.4, inline scheduler
 class inline_scheduler;

 // 33.13.5, task scheduler
 class task_scheduler;

 template<class E>
 struct with_error {
 using type = remove_cvref_t<E>;
 type error;
 };
 template<class E>
 with_error(E) -> with_error<E>;

```

```

// 33.13.6, class template task
template<class T, class Environment>
 class task;

// 33.14.1, scope concepts
template<class Token>
 concept scope_association = see below;
template<class Token>
 concept scope_token = see below;

// 33.14.2.2, simple counting scope
class simple_counting_scope;

// 33.14.2.3, counting scope
class counting_scope;

// 33.15, parallel scheduler
class parallel_scheduler;
parallel_scheduler get_parallel_scheduler();

// 33.16, namespace parallel_scheduler_replacement
namespace parallel_scheduler_replacement {
 struct receiver_proxy;
 struct bulk_item_receiver_proxy;
 struct parallel_scheduler_backend;

 shared_ptr<parallel_scheduler_backend> query_parallel_scheduler_backend();
}
}

```

<sup>1</sup> The exposition-only type *variant-or-empty*<Ts...> is defined as follows:

- (1.1) — If `sizeof...(Ts)` is greater than zero, *variant-or-empty*<Ts...> denotes *variant*<Us...> where Us... is the pack decay\_t<Ts>... with duplicate types removed.
- (1.2) — Otherwise, *variant-or-empty*<Ts...> denotes the exposition-only class type:

```

namespace std::execution {
 struct empty-variant { // exposition only
 empty-variant() = delete;
 };
}

```

<sup>2</sup> For type Sndr and pack of types Env, let CS be *completion\_signatures\_of\_t*<Sndr, Env...>. Then *single-sender-value-type*<Sndr, Env...> is ill-formed if CS is ill-formed; otherwise, it is an alias for:

- (2.1) — *gather-signatures*<set\_value\_t, CS, decay\_t, type\_identity\_t> if that type is well-formed,
- (2.2) — Otherwise, void if *gather-signatures*<set\_value\_t, CS, tuple, variant> is *variant*<tuple<>> or *variant*<>,
- (2.3) — Otherwise, *gather-signatures*<set\_value\_t, CS, *decayed-tuple*, type\_identity\_t> if that type is well-formed,
- (2.4) — Otherwise, *single-sender-value-type*<Sndr, Env...> is ill-formed.

<sup>3</sup> The exposition-only concept *single-sender* is defined as follows:

```

namespace std::execution {
 template<class Sndr, class... Env>
 concept single-sender = sender_in<Sndr, Env...> &&
 requires {
 typename single-sender-value-type<Sndr, Env...>;
 };
}

```

<sup>4</sup> A type satisfies and models the exposition-only concept *valid-completion-signatures* if it is a specialization of the *completion\_signatures* class template.

### 33.5 Queries

[exec.queries]

#### 33.5.1 Query utilities

[exec.queries.expos]

- <sup>1</sup> Subclause 33.5 makes use of the following exposition-only entities.
- <sup>2</sup> For subexpressions *q* and *tag* and pack *args*, let *TRY-QUERY*(*q*, *tag*, *args*...) be expression-equivalent to *AS-CONST*(*q*).*query*(*tag*, *args*...) if that expression is well-formed, and *AS-CONST*(*q*).*query*(*tag*) otherwise except that *args*... is evaluated.
- <sup>3</sup> For subexpressions *q* and *tag* and pack *args*, let *HIDE-SCHED*(*q*) be an object *o* such that *o*.*query*(*tag*, *args*...) is ill-formed when the decayed type of *tag* is *get\_scheduler\_t* or *get\_domain\_t*, and *q*.*query*(*tag*, *args*...) otherwise.

#### 33.5.2 forwarding\_query

[exec.fwd.env]

- <sup>1</sup> *forwarding\_query* asks a query object whether it should be forwarded through queryable adaptors.
- <sup>2</sup> The name *forwarding\_query* denotes a query object. For some query object *q* of type *Q*, *forwarding\_query*(*q*) is expression-equivalent to:
  - (2.1) — *MANDATE-NOT\_THROW*(*q*.*query*(*forwarding\_query*)) if that expression is well-formed.  
*Mandates*: The expression above has type *bool* and is a core constant expression if *q* is a core constant expression.
  - (2.2) — Otherwise, *true* if *derived\_from*<*Q*, *forwarding\_query\_t*> is *true*.
  - (2.3) — Otherwise, *false*.

#### 33.5.3 get\_allocator

[exec.get.allocator]

- <sup>1</sup> *get\_allocator* asks a queryable object for its associated allocator.
- <sup>2</sup> The name *get\_allocator* denotes a query object. For a subexpression *env*, *get\_allocator*(*env*) is expression-equivalent to *MANDATE-NOT\_THROW*(*AS-CONST*(*env*).*query*(*get\_allocator*)).  
*Mandates*: If the expression above is well-formed, its type satisfies *simple\_allocator* (16.4.4.6.1).
- <sup>3</sup> *forwarding\_query*(*get\_allocator*) is a core constant expression and has value *true*.

#### 33.5.4 get\_stop\_token

[exec.get.stop.token]

- <sup>1</sup> *get\_stop\_token* asks a queryable object for an associated stop token.
- <sup>2</sup> The name *get\_stop\_token* denotes a query object. For a subexpression *env*, *get\_stop\_token*(*env*) is expression-equivalent to:
  - (2.1) — *MANDATE-NOT\_THROW*(*AS-CONST*(*env*).*query*(*get\_stop\_token*)) if that expression is well-formed.  
*Mandates*: The type of the expression above satisfies *stoppable\_token*.
  - (2.2) — Otherwise, *never\_stop\_token*{}
- <sup>3</sup> *forwarding\_query*(*get\_stop\_token*) is a core constant expression and has value *true*.

#### 33.5.5 execution::get\_env

[exec.get.env]

- <sup>1</sup> *execution::get\_env* is a customization point object. For a subexpression *o*, *execution::get\_env*(*o*) is expression-equivalent to:
  - (1.1) — *MANDATE-NOT\_THROW*(*AS-CONST*(*o*).*get\_env*()) if that expression is well-formed.  
*Mandates*: The type of the expression above satisfies *queryable* (33.2).
  - (1.2) — Otherwise, *env<>{}*.
- <sup>2</sup> The value of *get\_env*(*o*) shall be valid while *o* is valid.
- <sup>3</sup> [Note 1: When passed a sender object, *get\_env* returns the sender's associated attributes. When passed a receiver, *get\_env* returns the receiver's associated execution environment. — end note]

#### 33.5.6 execution::get\_domain

[exec.get.domain]

- <sup>1</sup> *get\_domain* asks a queryable object for its associated execution domain tag.

- <sup>2</sup> The name `get_domain` denotes a query object. For a subexpression `env`, `get_domain(env)` is expression-equivalent to `MANDATE-NOTHROW(D())`, where `D` is the type of the first of the following expressions that is well-formed:

- (2.1) — `auto(AS-CONST(env).query(get_domain))`.
- (2.2) — `get_completion_domain<set_value_t>(get_scheduler(env), HIDE-SCHED(env))`.
- (2.3) — `default_domain()`, except that `env` is evaluated.

- <sup>3</sup> `forwarding_query(execution::get_domain)` is a core constant expression and has value `true`.

### 33.5.7 `execution::get_scheduler`

[`exec.get.scheduler`]

- <sup>1</sup> `get_scheduler` asks a queryable object for its associated scheduler.
- <sup>2</sup> The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to `get_completion_scheduler<set_value_t>(MANDATE-NOTHROW(AS-CONST(env).query(get_scheduler)), HIDE-SCHED(env))`.
- Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.
- <sup>3</sup> `forwarding_query(execution::get_scheduler)` is a core constant expression and has value `true`.

### 33.5.8 `execution::get_start_scheduler`

[`exec.get.start.scheduler`]

- <sup>1</sup> `get_start_scheduler` asks a queryable object for the scheduler an operation will be or was started on.
- <sup>2</sup> The name `get_start_scheduler` denotes a query object. For a subexpression `env`, `get_start_scheduler(env)` is expression-equivalent to `MANDATE-NOTHROW(AS-CONST(env).query(get_start_scheduler))`.
- Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.
- <sup>3</sup> `forwarding_query(execution::get_start_scheduler)` is a core constant expression and has value `true`.
- <sup>4</sup> Given subexpressions `sndr` and `rcvr` such that `sender-to<decltype((sndr)), decltype((rcvr))>` is `true` and the expression `get_start_scheduler(get_env(rcvr))` is well-formed, an operation state that is the result of calling `connect(sndr, rcvr)` shall, if it is started, be started on an execution agent associated with the scheduler `get_start_scheduler(get_env(rcvr))`.

### 33.5.9 `execution::get_delegation_scheduler`

[`exec.get.delegation.scheduler`]

- <sup>1</sup> `get_delegation_scheduler` asks a queryable object for a scheduler that can be used to delegate work to for the purpose of forward progress delegation (6.10.2.3).
- <sup>2</sup> The name `get_delegation_scheduler` denotes a query object. For a subexpression `env`, `get_delegation_scheduler(env)` is expression-equivalent to `MANDATE-NOTHROW(AS-CONST(env).query(get_delegation_scheduler))`.
- Mandates:* If the expression above is well-formed, its type satisfies `scheduler`.
- <sup>3</sup> `forwarding_query(execution::get_delegation_scheduler)` is a core constant expression and has value `true`.

### 33.5.10 `execution::get_forward_progress_guarantee`

[`exec.get.fwd.progress`]

```
namespace std::execution {
 enum class forward_progress_guarantee {
 concurrent,
 parallel,
 weakly_parallel
 };
}
```

- <sup>1</sup> `get_forward_progress_guarantee` asks a scheduler about the forward progress guarantee of execution agents created by that scheduler's associated execution resource (6.10.2.3).
- <sup>2</sup> The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype((sch))`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:
- `MANDATE-NOTHROW(AS-CONST(sch).query(get_forward_progress_guarantee))`

*Mandates:* The type of the expression above is `forward_progress_guarantee`.

- <sup>3</sup> If `get_forward_progress_guarantee(sch)` for some scheduler `sch` returns `forward_progress_guarantee::concurrent`, all execution agents created by that scheduler's associated execution resource shall provide the concurrent forward progress guarantee. If it returns `forward_progress_guarantee::parallel`, all such execution agents shall provide at least the parallel forward progress guarantee.

### 33.5.11 `execution::get_completion_scheduler` [exec.get.compl.sched]

- <sup>1</sup> The name `get_completion_scheduler` denotes a query object template.
- <sup>2</sup> Let *completion-fn* be a completion function (33.3); let *completion-fn-tag* be the associated completion tag of *completion-fn*; let *args* and *envs* be packs of subexpressions; and let *sndr* be a subexpression such that `sender<decltype((sndr))>` is true and `get_completion_scheduler<completion-fn-tag>(get_env(sndr), envs...)` is well-formed and denotes a scheduler *sch*.
- <sup>3</sup> `get_completion_scheduler<completion-fn-tag>` obtains the completion scheduler associated with a completion tag from a sender's attributes.
- <sup>4</sup> For subexpression *sch1* and pack *envs*, let *sch2* be `TRY-QUERY(sch1, get_completion_scheduler<set_value_t>, envs...)` and let `RECURSE-QUERY(sch1, envs...)` be expression-equivalent to *sch1* if *sch2* is ill-formed or if *sch1* and *sch2* have the same type and compare equal; otherwise, `RECURSE-QUERY(sch2, envs...)`.
- <sup>5</sup> For a subexpression *q* and pack *envs*, the expression `get_completion_scheduler<completion-fn-tag>(q, envs...)` is ill-formed if *completion-fn-tag* is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, the expression is expression-equivalent to:
- (5.1) — `MANDATE-NOTHROW(RECURSE-QUERY(TRY-QUERY(q, get_completion_scheduler<completion-fn-tag>, envs...), envs...))` if that expression is well-formed, except that *envs...* is evaluated only once.
- (5.2) — Otherwise, `auto(q)` if the type of *q* satisfies `scheduler` and *envs* is not an empty pack, except that *envs...* is evaluated.
- (5.3) — Otherwise, `get_completion_scheduler<completion-fn-tag>(q, envs...)` is ill-formed.

*Mandates:* If `get_completion_scheduler<completion-fn-tag>(q, envs...)` is well-formed, its type satisfies `scheduler`.

- <sup>6</sup> For a type *Tag*, subexpression *sndr*, and pack *envs*, let *CS* be `completion_signatures_of_t<decay_t<decltype((sndr))>, decltype((envs))...>`. If both `get_completion_scheduler<Tag>(get_env(sndr), envs...)` and *CS* are well-formed and `CS().count-of(Tag()) == 0` is true, the program is ill-formed.
- <sup>7</sup> If an asynchronous operation created by connecting *sndr* with a receiver *rcvr* causes the evaluation of `completion-fn(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent that belongs to *sch*'s associated execution resource.
- <sup>8</sup> The expression `forwarding_query(get_completion_scheduler<completion-fn-tag>)` is a core constant expression and has value `true`.

### 33.5.12 `execution::get_completion_domain` [exec.get.compl.domain]

- <sup>1</sup> `get_completion_domain<completion-tag>` obtains the completion domain associated with a completion tag from a sender's attributes.
- <sup>2</sup> The name `get_completion_domain` denotes a query object template. For a subexpression *attrs* and pack *envs*, the expression `get_completion_domain<completion-tag>(attrs, envs...)` is ill-formed if *completion-tag* is not one of `void`, `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, the expression is expression-equivalent to `MANDATE-NOTHROW(D())`, where *D* is:
- (2.1) — The type of `TRY-QUERY(attrs, get_completion_domain<completion-tag>, envs...)` if that expression is well-formed.
- (2.2) — Otherwise, the type of `get_completion_domain<set_value_t>(attrs, envs...)` if *completion-tag* is `void`.
- (2.3) — Otherwise, the type of



```
TRY-QUERY(get_completion_scheduler<completion-tag>(attrs, envs...),
 get_completion_domain<set_value_t>, envs...)
```

if that expression is well-formed.

- (2.4) — Otherwise, `default_domain` if `scheduler<decltype((attrs))>` is true and `envs` is not an empty pack.
- (2.5) — Otherwise, `get_completion_domain<completion-tag>(attrs, envs...)` is ill-formed.
- <sup>3</sup> For a type `Tag`, subexpression `sndr`, and pack `envs`, let `CS` be `completion_signatures_of_t<decay_t<decltype((sndr))>, decltype((envs))...>`. If both `get_completion_domain<Tag>(get_env(sndr), envs...)` and `CS` are well-formed and `CS().count-of(Tag()) == 0` is true, the program is ill-formed.
- <sup>4</sup> Let `completion-fn` be a completion function (33.3); let `completion-tag` be the associated completion tag of `completion-fn`; let `args` and `envs` be packs of subexpressions; and let `sndr` be a subexpression such that `sender<decltype((sndr))>` is true and `get_completion_domain<completion-tag>(get_env(sndr), envs...)` is well-formed and denotes a domain tag `D`. If an asynchronous operation created by connecting `sndr` with a receiver `rcvr` causes the evaluation of `completion-fn(rcvr, args...)`, the behavior is undefined unless the evaluation happens on an execution agent of an execution resource whose associated execution domain tag is `D`.

The expression `forwarding_query(get_completion_domain<completion-tag>)` is a core constant expression and has value true.

### 33.5.13 execution::get\_await\_completion\_adaptor [exec.get.await.adapt]

- <sup>1</sup> `get_await_completion_adaptor` asks a queryable object for its associated awaitable completion adaptor.
- <sup>2</sup> The name `get_await_completion_adaptor` denotes a query object. For a subexpression `env`,  

```
get_await_completion_adaptor(env)
```

is expression-equivalent to  

```
MANDATE-NOTROW(AS-CONST(env).query(get_await_completion_adaptor))
```
- <sup>3</sup> `forwarding_query(execution::get_await_completion_adaptor)` is a core constant expression and has value true.

## 33.6 Schedulers [exec.sched]

- <sup>1</sup> The `scheduler` concept defines the requirements of a scheduler type (33.3). `schedule` is a customization point object that accepts a scheduler. A valid invocation of `schedule` is a schedule-expression.

```
namespace std::execution {
 template<class Sch>
 concept scheduler =
 derived_from<typename remove_cvref_t<Sch>::scheduler_concept, scheduler_tag> &&
 queryable<Sch> &&
 requires(Sch&& sch) {
 { schedule(std::forward<Sch>(sch)) } -> sender;
 { get_forward_progress_guarantee(sch) } -> same_as<forward_progress_guarantee>;
 } &&
 equality_comparable<remove_cvref_t<Sch>> &&
 copyable<remove_cvref_t<Sch>>;
}
```

- <sup>2</sup> Let `Sch` be the type of a scheduler and let `Env` be the type of an execution environment for which `sender_in<schedule_result_t<Sch>, Env>` is satisfied. Then `sender-in-of<schedule_result_t<Sch>, Env>` shall be modeled.
- <sup>3</sup> No operation required by `copyable<remove_cvref_t<Sch>>` and `equality_comparable<remove_cvref_t<Sch>>` shall exit via an exception. None of these operations, nor a scheduler type's `schedule` function, shall introduce data races as a result of potentially concurrent (6.10.2.2) invocations of those operations from different threads.
- <sup>4</sup> For any two values `sch1` and `sch2` of some scheduler type `Sch`, `sch1 == sch2` shall return true only if both `sch1` and `sch2` share the same associated execution resource.
- <sup>5</sup> For a given scheduler expression `sch`, if the expression `get_completion_scheduler<set_value_t>(get_env(schedule(sch)))` is well-formed, it shall compare equal to `sch`.



- <sup>6</sup> For a given scheduler expression `sch`, type `T`, and pack of subexpressions `envs`, the following expressions are either both ill-formed, or both well-formed with the same type:

- (6.1) — `get_completion_domain<T>(sch, envs...)`  
 (6.2) — `get_completion_domain<T>(get_env(schedule(sch)), envs...)`

Likewise, the following expressions are either both ill-formed, or both well-formed with the same type and value:

- (6.3) — `get_completion_scheduler<T>(sch, envs...)`  
 (6.4) — `get_completion_scheduler<T>(get_env(schedule(sch)), envs...)`

- <sup>7</sup> A scheduler type's destructor shall not block pending completion of any receivers connected to the sender objects returned from `schedule`.  
<sup>8</sup> The exposition-only *infallible-scheduler* concept defines the requirements of a scheduler type whose `schedule` asynchronous operation can only complete with `set_value` unless stop can be requested:

```
template<class Sch, class Env>
concept infallible_scheduler =
 scheduler<Sch> &&
 (same_as<completion_signatures<set_value_t()>,
 completion_signatures_of_t<decltype(schedule(declval<Sch>())), Env>> ||
 (!unstoppable_token<stop_token_of_t<Env>> &&
 (same_as<completion_signatures<set_value_t(), set_stopped_t()>,
 completion_signatures_of_t<decltype(schedule(declval<Sch>())), Env>> ||
 same_as<completion_signatures<set_stopped_t(), set_value_t()>,
 completion_signatures_of_t<decltype(schedule(declval<Sch>())), Env>>)));
```

### 33.7 Receivers

[exec.recv]

#### 33.7.1 Receiver concepts

[exec.recv.concepts]

- <sup>1</sup> A receiver represents the continuation of an asynchronous operation. The `receiver` concept defines the requirements for a receiver type (33.3). The exposition-only concept *receiver-of* defines the requirements for a receiver type that is usable as the first argument of a set of completion operations corresponding to a set of completion signatures. The `get_env` customization point object is used to access a receiver's associated environment.

```
namespace std::execution {
 template<class Rcvr>
 concept receiver =
 derived_from<typename remove_cvref_t<Rcvr>::receiver_concept, receiver_tag> &&
 requires(const remove_cvref_t<Rcvr>& rcvr) {
 { get_env(rcvr) } -> queryable;
 } &&
 move_constructible<remove_cvref_t<Rcvr>> && // rvalues are movable, and
 constructible_from<remove_cvref_t<Rcvr>, Rcvr> && // lvalues are copyable, and
 is_nothrow_move_constructible_v<remove_cvref_t<Rcvr>>; // no-throw-movable

 template<class Signature, class Rcvr>
 concept valid_completion_for = // exposition only
 requires (Signature* sig) {
 []<class Tag, class... Args>(Tag*)(Args...)
 requires callable<Tag, remove_cvref_t<Rcvr>, Args...>
 {}(sig);
 };

 template<class Rcvr, class Completions>
 concept has_completions = // exposition only
 requires (Completions* completions) {
 []<valid_completion_for<Rcvr>... Sigs>(completion_signatures<Sigs...>*)
 {}(completions);
 };
};
```

```

template<class Rcvr, class Completions>
 concept receiver-of = // exposition only
 receiver<Rcvr> && has-completions<Rcvr, Completions>;
}

```

<sup>2</sup> Class types that are marked **final** do not model the **receiver** concept.

<sup>3</sup> Let **rcvr** be a receiver and let **op\_state** be an operation state associated with an asynchronous operation created by connecting **rcvr** with a sender. Let **token** be a stop token equal to **get\_stop\_token(get\_env(rcvr))**. **token** shall remain valid for the duration of the asynchronous operation's lifetime (33.3).

[Note 1: This means that, unless it knows about further guarantees provided by the type of **rcvr**, the implementation of **op\_state** cannot use **token** after it executes a completion operation. This also implies that any stop callbacks registered on **token** must be destroyed before the invocation of the completion operation. — end note]

```

4 namespace std::execution {
 template<class Rcvr, class ChildOp>
 concept inlinable_receiver = receiver<Rcvr> &&
 requires (ChildOp* child) {
 { remove_cvref_t<Rcvr>::make_receiver_for(child) } noexcept
 -> same_as<remove_cvref_t<Rcvr>>;
 };
}

```

The **inlinable\_receiver** concept defines the requirements for a receiver that can be reconstructed on demand from a pointer to the operation state object created when the receiver was connected to a sender. Given a receiver object **rcvr** of type **Rcvr** which was connected to a sender producing an operation state object **op** of type **Op**, **Rcvr** models **inlinable\_receiver<Op>** only if the expression **Rcvr::make\_receiver\_for(addressof(op))** evaluates to a receiver that is equal to **rcvr** (18.2).

[Note 2: Such a receiver does not need to be stored as a data member of **op** as it can be recreated on demand. — end note]

**ChildOp** may be an incomplete type.

<sup>5</sup> Given objects  $O_0, \dots, O_n$ ,  $O_n$  is *transitively constructed* from  $O_0$  if

(5.1) — **remove\_cvref\_t<decltype( $O_n$ )>** denotes the same type as **remove\_cvref\_t<decltype( $O_0$ )>** and

(5.2) — either

(5.2.1) —  $O_1$  was initialized by decay-copying a reference to  $O_0$ , or

(5.2.2) —  $n > 1$  and  $O_{n-1}$  is transitively constructed from  $O_0$  and  $O_n$  was initialized from a non-const, non-volatile rvalue reference to  $O_{n-1}$ .

<sup>6</sup> Let  $E$  be some well-formed expression **connect(sndr, rcvr)**.  $E$  *inlines the receiver* **rcvr** if the lifetimes of all objects transitively constructed from **rcvr** during the evaluation of  $E$  end within the evaluation of  $E$ .

[Note 3: This means such an expression does not store the receiver in the operation state. — end note]

<sup>7</sup> Let  $E$  be some well-formed expression **connect(sndr, rcvr)** where

(7.1) — **sndr** denotes a sender type defined by this document and

(7.2) —  $E$  inlines the receiver **rcvr**.

Then, let **op** be the result of the evaluation of  $E$ , and wherever the specification of the operation associated with **op** contains a glvalue which would denote an object transitively constructed from **rcvr**, that glvalue instead denotes the result of applying the temporary materialization conversion to the expression **remove\_cvref\_t<decltype(rcvr)>::make\_receiver\_for(addressof(op))**.

<sup>8</sup> Let **StdRcvr** be a receiver type defined by this document. Given some operation state type **Op**, it is unspecified whether **StdRcvr** models **inlinable\_receiver<Op>**.

<sup>9</sup> Let **StdOp** be some operation state type defined by this document, and let **Sndr** and **Rcvr** be types such that **is\_same\_v<connect\_result\_t<Sndr, Rcvr>, StdOp>** is true. If **Rcvr** models **inlinable\_receiver<StdOp>** then it is implementation-defined whether, given an object **rcvr** of type **Rcvr**, the **connect** operation which produces an object of type **StdOp** inlines the receiver **rcvr**.

### 33.7.2 execution::set\_value

[exec.set.value]

<sup>1</sup> **set\_value** is a value completion function (33.3). Its associated completion tag is **set\_value\_t**. The expression **set\_value(rcvr, vs...)** for a subexpression **rcvr** and pack of subexpressions **vs** is ill-formed if **rcvr** is an

lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NOTHROW(rcvr.set_value(vs...))`.

*Mandates:* If the expression above is well-formed, its type is void.

### 33.7.3 `execution::set_error`

[exec.set.error]

- <sup>1</sup> `set_error` is an error completion function (33.3). Its associated completion tag is `set_error_t`. The expression `set_error(rcvr, err)` for some subexpressions `rcvr` and `err` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NOTHROW(rcvr.set_error(err))`.

*Mandates:* If the expression above is well-formed, its type is void.

### 33.7.4 `execution::set_stopped`

[exec.set.stopped]

- <sup>1</sup> `set_stopped` is a stopped completion function (33.3). Its associated completion tag is `set_stopped_t`. The expression `set_stopped(rcvr)` for a subexpression `rcvr` is ill-formed if `rcvr` is an lvalue or an rvalue of const type. Otherwise, it is expression-equivalent to `MANDATE-NOTHROW(rcvr.set_stopped())`.

*Mandates:* If the expression above is well-formed, its type is void.

## 33.8 Operation states

[exec.opstate]

### 33.8.1 General

[exec.opstate.general]

- <sup>1</sup> The `operation_state` concept defines the requirements of an operation state type (33.3).

```
namespace std::execution {
 template<class O>
 concept operation_state =
 derived_from<typename O::operation_state_concept, operation_state_tag> &&
 requires (O& o) {
 start(o);
 };
}
```

- <sup>2</sup> If an `operation_state` object is destroyed during the lifetime of its asynchronous operation (33.3), the behavior is undefined.

[Note 1: The `operation_state` concept does not impose requirements on any operations other than destruction and `start`, including copy and move operations. Invoking any such operation on an object whose type models `operation_state` can lead to undefined behavior. — end note]

- <sup>3</sup> The program is ill-formed if it performs a copy or move construction or assignment operation on an operation state object created by connecting a library-provided sender.

### 33.8.2 `execution::start`

[exec.opstate.start]

- <sup>1</sup> The name `start` denotes a customization point object that starts (33.3) the asynchronous operation associated with the operation state object. For a subexpression `op`, the expression `start(op)` is ill-formed if `op` is an rvalue. Otherwise, it is expression-equivalent to `MANDATE-NOTHROW(op.start())`.

*Mandates:* If the expression above is well-formed, its type is void.

- <sup>2</sup> If `op.start()` does not start (33.3) the asynchronous operation associated with the operation state `op`, the behavior of calling `start(op)` is undefined.

## 33.9 Senders

[exec.snd]

### 33.9.1 General

[exec.snd.general]

- <sup>1</sup> Subclauses 33.9.11 and 33.9.12 define customizable algorithms that return senders. Each algorithm has a default implementation. Let `sndr` be the result of an invocation of such an algorithm or an object equal to the result (18.2), and let `Sndr` be `decltype((sndr))`. Let `rcvr` be a receiver of type `Rcvr` with associated environment `env` of type `Env` such that `sender-to<Sndr, Rcvr>` is true. For the default implementation of the algorithm that produced `sndr`, connecting `sndr` to `rcvr` and starting the resulting operation state (33.3) necessarily results in the potential evaluation (6.3) of a set of completion operations whose first argument is a subexpression equal to `rcvr`.
- <sup>2</sup> Let `Sigs` be a pack of completion signatures corresponding to this set of completion operations, and let `CS` be the type of the expression `get_completion_signatures<Sndr, Env>()`. Then `CS` is a specialization of the

class template `completion_signatures` (33.10), the set of whose template arguments is `Sigs`. If none of the types in `Sigs` are dependent on the type `Env`, then the expression `get_completion_signatures<Sndr>()` is well-formed and its type is `CS`.

- <sup>3</sup> Each completion operation can potentially be evaluated on one of several different execution agents as determined by the semantics of the algorithm, the environment of the receiver, and the completions of any child senders. For a completion tag `T`, let `DsT` be a pack comprised of the set of domain tags associated with the execution agents that could potentially evaluate any of the operation's completions with tag `T`. If there are no potentially evaluated completion operations with tag type `T`, then `get_completion_domain<T>(get_env(sndr), env)` is ill-formed; otherwise, it has type `COMMON-DOMAIN<DsT...>` (33.9.2).

[Example 1: Let `sndr2` be the sender `then(sndr, fn)`. `sndr2` has the same `set_value` completion domain tag as `sndr`, but if `fn`'s evaluation is potentially throwing, `sndr2`'s `set_error` completion domain tag would be the `COMMON-DOMAIN` of `sndr`'s value and error completion domain tags, in accordance with the semantics of the `then` algorithm (33.9.12.9). — end example]

- <sup>4</sup> If `sndr` can determine that all of its completion operations with tag `T` happen on execution agents associated with a particular scheduler `sch` (as determined by the semantics of the algorithm, the environment of the receiver, and the completion schedulers of any child senders), then `get_completion_scheduler<T>(get_env(sndr), env)` is well-formed and has the type and value of `sch`; otherwise, it is ill-formed.

[Example 2: Let `sndr2` be the sender from the example above. The `set_value` completion scheduler of `sndr2` is the `set_value` completion scheduler of `sndr`, if any. But `sndr2` can only report a `set_error` completion scheduler when invocations of `fn` are not potentially throwing or when `sndr` has no `set_error` completions. When `fn` can throw, `sndr2` could complete with `set_error` either by forwarding an error completion from `sndr` or by completing with the exception thrown by `fn`, which would happen on an agent associated with `sndr`'s `set_value` completion scheduler. — end example]

- <sup>5</sup> If a user-provided implementation of the algorithm that produced `sndr` is selected instead of the default:

- (5.1) — Any completion signature that is in the set of types denoted by `completion_signatures_of_t<Sndr, Env>` and that is not part of `Sigs` shall correspond to error or stopped completion operations, unless otherwise specified.
- (5.2) — If none of the types in `Sigs` are dependent on the type `Env`, then `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` shall denote the same type.

- <sup>6</sup> Various function templates in subclause 33.9 can throw an exception of type `unspecified-exception`. Each such exception object is of an unspecified type such that a `handler` of type `exception` matches (14.4) the exception object but a `handler` of type `dependent_sender_error` does not.

[Note 1: There is no requirement that two such exception objects have the same type. — end note]

### 33.9.2 Exposition-only entities

[exec.snd.expos]

- <sup>1</sup> Given an expression `sndr`, whose type is any sender type defined in the standard library, it is unspecified whether the expression `sndr.affine()` is well-formed. If that expression is well-formed, then the evaluation thereof meets the semantic requirements of the `affine` (33.13.3) algorithm.
- <sup>2</sup> Given an expression `sndr`, whose type is any sender type defined in the standard library, and an expression `p`, whose type is a promise type, it is unspecified whether the expression `sndr.as_awaitable(p)` is well-formed. If that expression is well-formed, then the evaluation thereof meets the semantic requirements of the `as_awaitable` (33.13.1) algorithm.
- <sup>3</sup> Subclause 33.9 makes use of the following exposition-only entities.
- <sup>4</sup> For a queryable object `env`, `FWD-ENV(env)` is an expression whose type satisfies `queryable` such that for a query object `q` and a pack of subexpressions `as`, the expression `FWD-ENV(env).query(q, as...)` is ill-formed if `forwarding_query(q)` is `false`; otherwise, it is expression-equivalent to `env.query(q, as...)`. The type `FWD-ENV-T(Env)` is `decltype(FWD-ENV(declval<Env>()))`.
- <sup>5</sup> For a query object `q`, a subexpression `v`, and a pack of subexpressions `args`, `MAKE-ENV(q, v)` is an expression `env` whose type satisfies `queryable` such that the result of `env.query(q, args...)` has a value equal to `v` (18.2). Unless otherwise stated, the object to which `env.query(q, args...)` refers remains valid while `env` remains valid.
- <sup>6</sup> For two queryable objects `env1` and `env2`, a query object `q`, and a pack of subexpressions `as`, `JOIN-ENV(env1, env2)` is an expression `env3` whose type satisfies `queryable` such that `env3.query(q, as...)` is expression-equivalent to:

- (6.1) — `env1.query(q, as...)` if that expression is well-formed,
- (6.2) — otherwise, `env2.query(q, as...)` if that expression is well-formed,
- (6.3) — otherwise, `env3.query(q, as...)` is ill-formed.

- 7 The results of *FWD-ENV*, *MAKE-ENV*, and *JOIN-ENV* can be context-dependent; i.e., they can evaluate to expressions with different types and value categories in different contexts for the same arguments.
- 8 For a pack of subexpressions `domains`, *COMMON-DOMAIN*(`domains...`) is expression-equivalent to `common_type_t<decltype(auto(domains))...>()` if that expression is well-formed, and `indeterminate_domain<Ds...>()` otherwise, where `Ds` is the pack of types consisting of `decltype(auto(domains))...` with duplicate types removed.
- 9 For a type `Tag`, subexpression `sndr`, and pack `envs`, *COMPL-DOMAIN*(`Tag, sndr, envs`) is expression-equivalent to `D()`, where `D` is the type of `get_completion_domain<Tag>(get_env(sndr), envs...)` if that expression is well-formed or `envs` is an empty pack, and `indeterminate_domain()` otherwise.
- 10 For a scheduler `sch`, *SCHED-ENV*(`sch`) is an expression `o` whose type satisfies *queryable* such that `o.query(get_start_scheduler)` is a prvalue with the same type and value as `sch`, and such that `o.query(get_domain)` is expression-equivalent to `sch.query(get_domain)`.
- 11 For two subexpressions `rcvr` and `expr`, *SET-VALUE*(`rcvr, expr`) is expression-equivalent to `(expr, set_value(std::move(rcvr)))` if the type of `expr` is `void`; otherwise, `set_value(std::move(rcvr), expr)`. *TRY-EVAL*(`rcvr, expr`) is equivalent to:

```
try {
 expr;
} catch(...) {
 set_error(std::move(rcvr), current_exception());
}
```

if `expr` is potentially-throwing; otherwise, `expr`. *TRY-SET-VALUE*(`rcvr, expr`) is

```
TRY-EVAL(rcvr, SET-VALUE(rcvr, expr))
```

except that `rcvr` is evaluated only once.

```
template<class Tag, class Env, class Default>
constexpr decltype(auto) query-with-default(
 Tag, const Env& env, Default&& value) noexcept(see below);
```

- 12 Let `e` be the expression `Tag()(env)` if that expression is well-formed; otherwise, it is `static_cast<Default>(std::forward<Default>(value))`.

- 13 *Returns:* `e`.

- 14 *Remarks:* The expression in the `noexcept` clause is `noexcept(e)`.

- 15 

```
template<callable Fun>
 requires is_nothrow_move_constructible_v<Fun>
struct emplace-from {
 Fun fun; // exposition only
 using type = call-result-t<Fun>;

 constexpr operator type() && noexcept(nothrow-callable<Fun>) {
 return std::move(fun)();
 }

 constexpr type operator()() && noexcept(nothrow-callable<Fun>) {
 return std::move(fun)();
 }
};
```

[Note 1: *emplace-from* is used to emplace non-movable types into tuple, optional, variant, and similar types.  
— end note]

- 16 

```
struct on-stop-request {
 inplace_stop_source& stop_src; // exposition only
 void operator()() noexcept { stop_src.request_stop(); }
};
```

```

17 template<class T0, class T1, ..., class Tn>
 struct product-type { // exposition only
 T0 t0; // exposition only
 T1 t1; // exposition only
 ⋮
 Tn tn; // exposition only

 template<size_t I, class Self>
 constexpr decltype(auto) get(this Self&& self) noexcept; // exposition only

 template<class Self, class Fn>
 constexpr decltype(auto) apply(this Self&& self, Fn&& fn) // exposition only
 noexcept(see below);
 };

```

18 [Note 2: *product-type* is presented here in pseudo-code form for the sake of exposition. It can be approximated in standard C++ with a tuple-like implementation that takes care to keep the type an aggregate that can be used as the initializer of a structured binding declaration. — end note]

[Note 3: An expression of type *product-type* is usable as the initializer of a structured binding declaration (9.7). — end note]

```

template<size_t I, class Self>
constexpr decltype(auto) get(this Self&& self) noexcept;

```

19 *Effects*: Equivalent to:

```

 auto& [...ts] = self;
 return std::forward_like<Self>(ts...[I]);

```

```

template<class Self, class Fn>
constexpr decltype(auto) apply(this Self&& self, Fn&& fn) noexcept(see below);

```

20 *Constraints*: The expression in the *return* statement below is well-formed.

21 *Effects*: Equivalent to:

```

 auto& [...ts] = self;
 return std::forward<Fn>(fn)(std::forward_like<Self>(ts)...);

```

22 *Remarks*: The expression in the *noexcept* clause is *true* if the *return* statement above is not potentially throwing; otherwise, *false*.

23 Let *valid-specialization* be the following concept:

```

namespace std::execution {
 template<template<class...> class T, class... Args>
 concept valid-specialization = // exposition only
 requires { typename T<Args...>; };
}

```

```

template<class Tag, class Data = see below, class... Child>
constexpr auto make-sender(Tag tag, Data&& data, Child&&... child);

```

24 *Mandates*: The following expressions are true:

(24.1) — *semiregular*<Tag>

(24.2) — *movable-value*<Data>

(24.3) — (*sender*<Child> && ...)

(24.4) — *dependent\_sender*<Sndr> || *sender\_in*<Sndr>, where *Sndr* is *basic-sender*<Tag, decay\_t<Data>, decay\_t<Child>...> as defined below.

*Recommended practice*: If evaluation of *sender\_in*<Sndr> results in an uncaught exception from the evaluation of *get\_completion\_signatures*<Sndr>(), the implementation should include information about that exception in the resulting diagnostic.

25 *Returns*: A prvalue of type *basic-sender*<Tag, decay\_t<Data>, decay\_t<Child>...> that has been direct-list-initialized with the forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below.

26 *Remarks:* The default template argument for the `Data` template parameter denotes an unspecified empty trivially copyable class type that models `semiregular`.

```

27 namespace std::execution {
 template<class Snдр, class Rcvr> // exposition only
 using state-type = decay_t<call-result-t<
 decltype(impls-for<tag_of_t<Snдр>>::get-state), Snдр, Rcvr&>>;

 template<class Index, class Snдр, class Rcvr> // exposition only
 using env-type = call-result-t<
 decltype(impls-for<tag_of_t<Snдр>>::get-env), Index,
 state-type<Snдр, Rcvr&>, const Rcvr&>;

 template<class Snдр>
 using data-type = decltype(declval<Snдр>().template get<1>()); // exposition only

 template<class Snдр, size_t I = 0>
 using child-type = decltype(declval<Snдр>().template get<I+2>()); // exposition only

 template<class Snдр>
 using indices-for = remove_reference_t<Snдр>::indices-for; // exposition only

 template<class Snдр, class Rcvr>
 struct basic-state { // exposition only
 basic-state(Snдр&& snдр, Rcvr&& rcvr) noexcept(see below)
 : rcvr(std::move(rcvr))
 , state(impls-for<tag_of_t<Snдр>>::get-state(std::forward<Snдр>(snдр), rcvr)) { }

 Rcvr rcvr; // exposition only
 state-type<Snдр, Rcvr> state; // exposition only
 };
}

```

28 The expression in the `noexcept` clause of the constructor of `basic-state` is

```

is_nothrow_move_constructible_v<Rcvr> &&
nothrow_callable<decltype(impls-for<tag_of_t<Snдр>>::get-state), Snдр, Rcvr&> &&
(same_as<state-type<Snдр, Rcvr>, get-state-result> ||
is_nothrow_constructible_v<state-type<Snдр, Rcvr>, get-state-result>)

```

where `get-state-result` is

```

call-result-t<decltype(impls-for<tag_of_t<Snдр>>::get-state), Snдр, Rcvr&>.

```

```

29 namespace std::execution {
 template<class Snдр, class Rcvr, class Index>
 requires valid-specialization<env-type, Index, Snдр, Rcvr>
 struct basic-receiver { // exposition only
 using receiver_concept = receiver_tag;

 using tag-t = tag_of_t<Snдр>;
 using state-t = state-type<Snдр, Rcvr>; // exposition only
 static constexpr const auto& complete = impls-for<tag-t>::complete; // exposition only

 template<class... Args>
 void set_value(Args&&... args) && noexcept {
 complete(Index(), op->state, op->rcvr, set_value_t(), std::forward<Args>(args)...);
 }

 template<class Error>
 void set_error(Error&& err) && noexcept {
 complete(Index(), op->state, op->rcvr, set_error_t(), std::forward<Error>(err));
 }

 void set_stopped() && noexcept {
 complete(Index(), op->state, op->rcvr, set_stopped_t());
 }
 }
}

```

```

 auto get_env() const noexcept -> env-type<Index, Sndr, Rcvr> {
 return impls-for<tag-t>::get-env(Index(), op->state, op->rcvr);
 }

 basic-state<Sndr, Rcvr>* op; // exposition only
};
}

namespace std::execution {
 constexpr auto connect-all = see below; // exposition only

 template<class Sndr, class Rcvr>
 using connect-all-result = call-result-t< // exposition only
 decltype(connect-all), basic-state<Sndr, Rcvr>*, Sndr, indices-for<Sndr>>;
}

```

30 The object *connect-all* is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr, size_t... Is>(
 basic-state<Sndr, Rcvr>* op, Sndr&& sndr, index_sequence<Is...>) noexcept(see below)
-> decltype(auto) {
 auto& [_ , data, ...child] = sndr;
 return product-type{connect(
 std::forward_like<Sndr>(child),
 basic-receiver<Sndr, Rcvr, integral_constant<size_t, Is>>{op})...};
}

```

31 *Constraints:* The expression in the `return` statement is well-formed.

32 *Remarks:* The expression in the `noexcept` clause is true if the `return` statement is not potentially throwing; otherwise, false.

```

33 namespace std::execution {
 template<class Sndr, class Rcvr>
 requires valid-specialization<state-type, Sndr, Rcvr> &&
 valid-specialization<connect-all-result, Sndr, Rcvr>
 struct basic-operation : basic-state<Sndr, Rcvr> { // exposition only
 using operation_state_concept = operation_state_tag;
 using tag-t = tag_of_t<Sndr>; // exposition only

 connect-all-result<Sndr, Rcvr> inner-ops; // exposition only

 basic-operation(Sndr&& sndr, Rcvr&& rcvr) noexcept(see below) // exposition only
 : basic-state<Sndr, Rcvr>(std::forward<Sndr>(sndr), std::move(rcvr)),
 inner-ops(connect-all(this, std::forward<Sndr>(sndr), indices-for<Sndr>()))
 {}

 void start() & noexcept {
 auto& [...ops] = inner-ops;
 impls-for<tag-t>::start(this->state, this->rcvr, ops...);
 }
 };
}

```

34 The expression in the `noexcept` clause of the constructor of *basic-operation* is:

```

is_nothrow_constructible_v<basic-state<Self, Rcvr>, Self, Rcvr> &&
noexcept(connect-all(this, std::forward<Sndr>(sndr), indices-for<Sndr>()))

```

```

35 namespace std::execution {
 struct default-impls { // exposition only
 static constexpr auto get-env = see below; // exposition only
 static constexpr auto get-state = see below; // exposition only
 static constexpr auto start = see below; // exposition only
 static constexpr auto complete = see below; // exposition only

 template<class Sndr, class... Env>
 static constexpr void check-types(); // exposition only
 };
}

```



```
};
```

```
template<class Tag>
struct impls-for : default-impls {}; // exposition only
}
```

36 The member `default-impls::get-env` is initialized with a callable object equivalent to the following lambda:

```
[(auto, auto&, const auto& rcvr) noexcept -> decltype(auto) {
 return FWD-ENV(get_env(rcvr));
}]
```

37 The member `default-impls::get-state` is initialized with a callable object equivalent to the following lambda:

```
[(<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept -> decltype(auto) {
 auto& [_ , data, ...child] = sndr;
 return allocator-aware-forward(std::forward_like<Sndr>(data), rcvr);
}]
```

38 The member `default-impls::start` is initialized with a callable object equivalent to the following lambda:

```
[(auto&, auto&, auto&... ops) noexcept -> void {
 (execution::start(ops), ...);
}]
```

39 The member `default-impls::complete` is initialized with a callable object equivalent to the following lambda:

```
[(<class Index, class Rcvr, class Tag, class... Args>(
 Index, auto& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
-> void requires callable<Tag, Rcvr, Args...> {
 static_assert(Index::value == 0);
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
}]
```

```
template<class Sndr, class... Env>
static consteval void default-impls::check-types();
```

40 Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by `indices-for<Sndr>`.

41 *Effects:* Equivalent to:

```
((void)get_completion_signatures<child-type<Sndr, Is>, FWD-ENV-T(Env)...>(), ...);
```

42 [Note 4: For any types `T` and `S`, and pack `E`, let `e` be the expression `impls-for<T>::check-types<S, E...>()`. Then exactly one of the following is true:

- (42.1) — `e` is ill-formed, or
- (42.2) — the evaluation of `e` exits with an exception, or
- (42.3) — `e` is a core constant expression.

When `e` is a core constant expression, the pack `S, E...` uniquely determines a set of completion signatures.  
— end note]

```
43 namespace std::execution {
 template<class Tag, class Data, class... Child>
 struct basic-sender : product-type<Tag, Data, Child...> { // exposition only
 using sender_concept = sender_tag;
 using indices-for = index_sequence_for<Child...>; // exposition only

 decltype(auto) get_env() const noexcept {
 auto& [_ , data, ...child] = *this;
 return impls-for<Tag>::get-attrs(data, child...);
 }

 template<decays-to<basic-sender> Self, receiver Rcvr>
 auto connect(this Self&& self, Rcvr rcvr) noexcept(see below)
 -> basic-operation<Self, Rcvr> {
```

```

 return {std::forward<Self>(self), std::move(rcvr)};
 }

 template<decays-to<basic-sender> Self, class... Env>
 static constexpr auto get_completion_signatures();
};

```

44 It is unspecified whether a specialization of *basic-sender* is an aggregate.

45 An expression of type *basic-sender* is usable as the initializer of a structured binding declaration (9.7).

46 The expression in the `noexcept` clause of the `connect` member function of *basic-sender* is:

```
is_nothrow_constructible_v<basic-operation<Self, Rcvr>, Self, Rcvr>
```

```

template<class Tag, class Data, class... Child>
template<decays-to<basic-sender> Sndr, class... Env>
constexpr auto basic-sender<Tag, Data, Child...>::get_completion_signatures();

```

47 Let *Rcvr* be the type of a receiver whose environment has type *E*, where *E* is the first type in the list *Env...*, *env*<>. Let *CHECK-TYPES*() be the expression *impls-for*<Tag>::template *check-types*<*Sndr*, *E*>(), and let *CS* be a type determined as follows:

(47.1) — If *CHECK-TYPES*() is a core constant expression, let *op* be an lvalue subexpression whose type is *connect\_result\_t*<*Sndr*, *Rcvr*>. Then *CS* is the specialization of *completion\_signatures* the set of whose template arguments correspond to the set of completion operations that are potentially evaluated (6.3) as a result of evaluating *op.start*().

(47.2) — Otherwise, *CS* is *completion\_signatures*<>.

48 *Constraints*: *CHECK-TYPES*() is a well-formed expression.

49 *Effects*: Equivalent to:

```

CHECK-TYPES();
return CS();

```

```

50 template<class... Fns>
struct overload-set : Fns... {
 using Fns::operator()...;
};

51 struct not-a-sender {
 using sender_concept = sender_tag;

 template<class Sndr>
 static constexpr auto get_completion_signatures() -> completion_signatures<> {
 throw unspecified-exception();
 }
};

```

```

52 struct not-a-scheduler {
 using scheduler_concept = scheduler_tag;

 constexpr auto schedule() const noexcept {
 return not-a-sender();
 }
};

```

```

53 constexpr void decay-copyable-result-datums(auto cs) {
 cs.for-each([]<class Tag, class... Ts>(Tag*)(Ts...)) {
 if constexpr (!is_constructible_v<decay_t<Ts>, Ts> &&...)
 throw unspecified-exception();
 });
}

```

```

template<class T, class Context>
decltype(auto) allocator-aware-forward(T&& obj, Context&& context); // exposition only

```

54 *allocator-aware-forward* is an exposition-only function template used to either create a new object of type *remove\_cvref\_t*<*T*> from *obj* or forward *obj* depending on whether an allocator is available. If the

environment associated with `context` provides an allocator (i.e., the expression `get_allocator(get_env(context))` is valid), let `alloc` be the result of this expression and let `P` be `remove_cvref_t<T>`.

55 *Returns:*

- (55.1) — If `alloc` is not defined, returns `std::forward<T>(obj)`,
- (55.2) — otherwise if `P` is a specialization of *product-type*, returns an object of type `P` whose elements are initialized using
 

```
make_obj_using_allocator<decltype(e)>(alloc, std::forward_like<T>(e))
```

 where `e` is the corresponding element of `obj`,
- (55.3) — otherwise, returns `make_obj_using_allocator<P>(alloc, std::forward<T>(obj))`.

```
template<class Fn, class Default, class... Args>
constexpr decltype(auto) call-with-default(
 Fn&& fn, Default&& value, Args&&... args) noexcept(see below);
```

56 Let `expr` be the expression `std::forward<Fn>(fn)(std::forward<Args>(args)...) if that expression is well-formed; otherwise, it is static_cast<Default>(std::forward<Default>(value)).`

57 *Returns:* `expr`.

58 *Remarks:* The expression in the `noexcept` clause is `noexcept(expr)`.

```
59 template<class Tag>
struct inline-attrs {
 see below
};
```

60 For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_scheduler<Tag>, env)` is expression-equivalent to `get_scheduler(env)`.

61 For a subexpression `env`, `inline-attrs<Tag>{}.query(get_completion_domain<Tag>, env)` is expression-equivalent to `get_domain(env)`.

### 33.9.3 Sender concepts

[exec.snd.concepts]

- 1 The **sender** concept defines the requirements for a sender type (33.3). The **sender\_in** concept defines the requirements for a sender type that can create asynchronous operations given an associated environment type. The exposition-only concept *sender-to* defines the requirements for a sender type that can connect with a specific receiver type. The `get_env` customization point object is used to access a sender's associated attributes. The `connect` customization point object is used to connect (33.3) a sender and a receiver to produce an operation state.

```
namespace std::execution {
 template<auto>
 concept is-constant = true; // exposition only

 template<class Sndr>
 concept is-sender = // exposition only
 derived_from<typename Sndr::sender_concept, sender_tag>;

 template<class Sndr>
 concept enable-sender = // exposition only
 is-sender<Sndr> ||
 is-awaitable<Sndr, env-promise<env<>>>>; // 33.9.4

 template<class Sndr>
 inline constexpr bool enable_sender = enable-sender<Sndr>;

 template<class Sndr>
 consteval bool is-dependent-sender-helper() try { // exposition only
 get_completion_signatures<Sndr>();
 return false;
 } catch (dependent_sender_error&) {
 return true;
 }
}
```

```

template<class Sndr>
concept sender =
 enable_sender<remove_cvref_t<Sndr>> &&
 requires (const remove_cvref_t<Sndr>& sndr) {
 { get_env(sndr) } -> queryable;
 } &&
 move_constructible<remove_cvref_t<Sndr>> &&
 constructible_from<remove_cvref_t<Sndr>, Sndr>;

template<class Sndr, class... Env>
concept sender_in =
 sender<Sndr> &&
 (sizeof...(Env) <= 1) &&
 (queryable<Env> &&...) &&
 is-constant<get_completion_signatures<Sndr, Env...>()>;

template<class Sndr>
concept dependent_sender =
 sender<Sndr> && bool_constant<is-dependent-sender-helper<Sndr>()>::value;

template<class Sndr, class Rcvr>
concept sender-to = // exposition only
 sender_in<Sndr, env_of_t<Rcvr>> &&
 receiver-of<Rcvr, completion_signatures_of_t<Sndr, env_of_t<Rcvr>>> &&
 requires (Sndr&& sndr, Rcvr&& rcvr) {
 connect(std::forward<Sndr>(sndr), std::forward<Rcvr>(rcvr));
 };
}

```

- <sup>2</sup> For a type `Sndr`, if `sender<Sndr>` is true and `dependent_sender<Sndr>` is false, then `Sndr` is a non-dependent sender (33.3).
- <sup>3</sup> Given a subexpression `sndr`, let `Sndr` be `decltype((sndr))` and let `rcvr` be a receiver with an associated environment whose type is `Env`. A completion operation is a *permissible completion* for `Sndr` and `Env` if its completion signature appears in the argument list of the specialization of `completion_signatures` denoted by `completion_signatures_of_t<Sndr, Env>`. `Sndr` and `Env` model `sender_in<Sndr, Env>` if all the completion operations that are potentially evaluated by connecting `sndr` to `rcvr` and starting the resulting operation state are permissible completions for `Sndr` and `Env`.
- <sup>4</sup> *Remarks:* Pursuant to 16.4.5.2.1, users may specialize `enable_sender` to `true` for cv-unqualified program-defined types that model `sender`, and `false` for types that do not. Such specializations shall be usable in constant expressions (7.7.4) and have type `const bool`.
- <sup>5</sup> The exposition-only concepts *sender-of* and *sender-in-of* define the requirements for a sender type that completes with a given unique set of value result types.

```

namespace std::execution {
 template<class... As>
 using value-signature = set_value_t(As...); // exposition only

 template<class Sndr, class SetValue, class... Env>
 concept sender-in-of-impl = // exposition only
 sender_in<Sndr, Env...> &&
 MATCHING-SIG(SetValue, // see 33.1
 gather-signatures<set_value_t, // see 33.10
 completion_signatures_of_t<Sndr, Env...>,
 value-signature,
 type_identity_t>);

 template<class Sndr, class Env, class... Values>
 concept sender-in-of = // exposition only
 sender-in-of-impl<Sndr, set_value_t(Values...), Env>;
}

```

```

template<class Sndr, class... Values>
 concept sender-of = // exposition only
 sender-in-of-impl<Sndr, set_value_t(Values...)>;
}

```

6 Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`. The type `tag_of_t<Sndr>` is as follows:

(6.1) — If the declaration

```
auto&& [tag, data, ...children] = sndr;
```

would be well-formed, `tag_of_t<Sndr>` is an alias for `decltype(auto(tag))`.

(6.2) — Otherwise, `tag_of_t<Sndr>` is ill-formed.

7 Let `sender-for` be an exposition-only concept defined as follows:

```

namespace std::execution {
 template<class Sndr, class Tag>
 concept sender-for =
 sender<Sndr> &&
 same_as<tag_of_t<Sndr>, Tag>;
}

```

8 For a type `T`, `SET-VALUE-SIG(T)` denotes the type `set_value_t()` if `T` is *cv* `void`; otherwise, it denotes the type `set_value_t(T)`.

9 Library-provided sender types

(9.1) — always expose an overload of a member `connect` that accepts an rvalue sender and

(9.2) — only expose an overload of a member `connect` that accepts an lvalue sender if they model `copy_constructible`.

### 33.9.4 Awaitable helpers

[exec.awaitable]

1 The sender concepts recognize awaitables as senders. For [Clause 33](#), an *awaitable* is an expression that would be well-formed as the operand of a `co_await` expression within a given context.

2 For a subexpression `c`, let `GET-AWAITER(c, p)` be expression-equivalent to the series of transformations and conversions applied to `c` as the operand of an *await-expression* in a coroutine, resulting in lvalue `e` as described by [7.6.2.4](#), where `p` is an lvalue referring to the coroutine's promise, which has type `Promise`.

[Note 1: This includes the invocation of the promise type's `await_transform` member if any, the invocation of the operator `co_await` picked by overload resolution if any, and any necessary implicit conversions and materializations. — end note]

Let `GET-AWAITER(c)` be expression-equivalent to `GET-AWAITER(c, q)` where `q` is an lvalue of an unspecified empty class type *none-such* that lacks an `await_transform` member, and where `coroutine_handle<none-such>` behaves as `coroutine_handle<void>`.

3 Let *is-awaitable* be the following exposition-only concept:

```

namespace std {
 template<class T>
 concept await-suspend-result = see below; // exposition only

 template<class A, class... Promise>
 concept is-awaiter = // exposition only
 requires (A& a, coroutine_handle<Promise...> h) {
 a.await_ready() ? 1 : 0;
 { a.await_suspend(h) } -> await-suspend-result;
 a.await_resume();
 };

 template<class C, class... Promise>
 concept is-awaitable = // exposition only
 requires (C (*fc)() noexcept, Promise&... p) {
 { GET-AWAITER(fc(), p...) } -> is-awaiter<Promise...>;
 };
}

```

*await-suspend-result*<T> is true if and only if one of the following is true:

- (3.1) — T is void, or
  - (3.2) — T is bool, or
  - (3.3) — T is a specialization of `coroutine_handle`.
- <sup>4</sup> For a subexpression *c* such that `decltype((c))` is type *C*, and an lvalue *p* of type `Promise`, *await-result-type*<*C*, `Promise`> denotes the type `decltype(GET-AWAITER(c, p).await_resume())` and *await-result-type*<*C*> denotes the type `decltype(GET-AWAITER(c).await_resume())`.
- <sup>5</sup> Let *with-await-transform* be the exposition-only class template:

```
namespace std::execution {
 template<class T, class Promise>
 concept has-as-awaitable = // exposition only
 requires (T&& t, Promise& p) {
 { std::forward<T>(t).asAwaitable(p) } -> isAwaitable<Promise&>;
 };

 template<class Derived>
 struct with-await-transform { // exposition only
 template<class T>
 T&& await_transform(T&& value) noexcept {
 return std::forward<T>(value);
 }

 template<has-as-awaitable<Derived> T>
 auto await_transform(T&& value)
 noexcept(noexcept(std::forward<T>(value).asAwaitable(declval<Derived&>())))
 -> decltype(std::forward<T>(value).asAwaitable(declval<Derived&>())) {
 return std::forward<T>(value).asAwaitable(static_cast<Derived&>(*this));
 }
 };
}
```

- <sup>6</sup> Let *env-promise* be the exposition-only class template:

```
namespace std::execution {
 template<class Env>
 struct env_promise : with-await-transform<env_promise<Env>> { // exposition only
 unspecified get_return_object() noexcept;
 unspecified initial_suspend() noexcept;
 unspecified final_suspend() noexcept;
 void unhandled_exception() noexcept;
 void return_void() noexcept;
 coroutine_handle<> unhandled_stopped() noexcept;

 const Env& get_env() const noexcept;
 };
}
```

[Note 2: Specializations of *env-promise* are used only for the purpose of type computation; its members need not be defined. — end note]

### 33.9.5 execution::indeterminate\_domain

[exec.domain.indeterminate]

- <sup>1</sup> namespace std::execution {
- ```
    template<class... Domains>
        struct indeterminate_domain {
            indeterminate_domain() = default;
            constexpr indeterminate_domain(auto&&) noexcept {}

            template<class Tag, sender Sndr, queryable Env>
                static constexpr decltype(auto) transform_sender(Tag, Sndr&& sndr, const Env& env)
                    noexcept(see below);
        };
}
```

```
template<class Tag, sender Sndr, queryable Env>
static constexpr decltype(auto) transform_sender(Tag, Sndr&& sndr, const Env& env)
noexcept(see below);
```

2 *Mandates:* For each type *D* in *Domains...*, the expression

```
D().transform_sender(Tag(), std::forward<Sndr>(sndr), env)
```

is either ill-formed or has the same decayed type as `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`.

3 *Returns:* `default_domain().transform_sender(Tag(), std::forward<Sndr>(sndr), env)`.

4 *Remarks:* For a pack of types *Ds*,

```
common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<Ds...>>
```

is `indeterminate_domain<Us...>` where *Us* is a pack that contains each type in *Domains...*, *Ds...* except with duplicate types removed. For a type *D* that is not a specialization of `indeterminate_domain`, `common_type_t<indeterminate_domain<Domains...>, D>` is *D* if *Domains* is an empty pack, and `common_type_t<indeterminate_domain<Domains...>, indeterminate_domain<D>>` otherwise.

33.9.6 `execution::default_domain`

[`exec.domain.default`]

```
1 namespace std::execution {
  struct default_domain {
    template<class Tag, sender Sndr, queryable Env>
      static constexpr decltype(auto) transform_sender(Tag, Sndr&& sndr, const Env& env)
        noexcept(see below);

    template<class Tag, sender Sndr, class... Args>
      static constexpr decltype(auto) apply_sender(Tag, Sndr&& sndr, Args&&... args)
        noexcept(see below);
  };
}
```

```
template<class Tag, sender Sndr, queryable Env>
constexpr decltype(auto) transform_sender(Tag, Sndr&& sndr, const Env& env)
noexcept(see below);
```

2 Let *e* be the expression

```
tag_of_t<Sndr>().transform_sender(Tag(), std::forward<Sndr>(sndr), env)
```

if that expression is well-formed; otherwise, `static_cast<Sndr>(std::forward<Sndr>(sndr))`.

3 *Returns:* *e*.

4 *Remarks:* The exception specification is equivalent to `noexcept(e)`.

```
template<class Tag, sender Sndr, class... Args>
constexpr decltype(auto) apply_sender(Tag, Sndr&& sndr, Args&&... args)
noexcept(see below);
```

5 Let *e* be the expression

```
Tag().apply_sender(std::forward<Sndr>(sndr), std::forward<Args>(args)...)

```

6 *Constraints:* *e* is a well-formed expression.

7 *Returns:* *e*.

8 *Remarks:* The exception specification is equivalent to `noexcept(e)`.

33.9.7 `execution::transform_sender`

[`exec.snd.transform`]

```
namespace std::execution {
  template<sender Sndr, queryable Env>
    constexpr decltype(auto) transform_sender(Sndr&& sndr, const Env& env)
      noexcept(see below);
}
```

1 For a subexpression *s*, let *start-domain* be *D()* where *D* is the decayed type of `get_domain(env)` if that expression is well-formed, and `default_domain` otherwise.

- 2 Let *completion-domain*(*s*) be *D*() where *D* is the decayed type of `get_completion_domain<>(get_env(s), env)` if that is well-formed; otherwise, *D* is `default_domain`.
- 3 Let *transformed-sndr*(*dom*, *tag*, *s*) be the expression
`dom.transform_sender(tag, s, env)`
 if that expression is well-formed; otherwise,
`default_domain().transform_sender(tag, s, env)`
- Let *transform-recurse*(*dom*, *tag*, *s*) be the expression *transformed-sndr*(*dom*, *tag*, *s*) if *transformed-sndr*(*dom*, *tag*, *s*) and *s* have the same type ignoring cv-qualifiers; otherwise, it is the expression *transform-recurse*(*dom2*, *tag*, *s2*), where *s2* is *transformed-sender*(*dom*, *tag*, *s*) and *dom2* is *start-domain* if *tag* is *start*, and *completion-domain*(*s2*) otherwise.
- Let *tmp-sndr* be the expression
`transform-recurse(completion-domain(sndr), set_value, sndr)`
- and let *final-sndr* be the expression
`transform-recurse(start-domain, start, tmp-sndr)`
- 4 Returns: *final-sndr*.
- 5 Remarks: The exception specification is equivalent to `noexcept(final-sndr)`.

33.9.8 execution::apply_sender

[exec.snd.apply]

```
namespace std::execution {
  template<class Domain, class Tag, sender Sndr, class... Args>
  constexpr decltype(auto) apply_sender(Domain dom, Tag, Sndr&& sndr, Args&&... args)
    noexcept(see below);
}
```

- 1 Let *e* be the expression
`dom.apply_sender(Tag(), std::forward<Sndr>(sndr), std::forward<Args>(args)...)`
 if that expression is well-formed; otherwise,
`default_domain().apply_sender(Tag(), std::forward<Sndr>(sndr), std::forward<Args>(args)...)`
- 2 Constraints: The expression *e* is well-formed.
- 3 Returns: *e*.
- 4 Remarks: The exception specification is equivalent to `noexcept(e)`.

33.9.9 execution::get_completion_signatures

[exec.getcomplsigs]

```
template<class Sndr, class... Env>
constexpr auto get_completion_signatures() -> valid-completion-signatures auto;
```

- 1 Let *except* be an rvalue subexpression of an unspecified class type *Except* such that `move_constructible<Except> && derived_from<Except, exception> is true`. Let *CHECKED-COMPLSIGS*(*e*) be *e* if *e* is a core constant expression whose type satisfies *valid-completion-signatures*; otherwise, it is the following expression:
`(e, throw except, completion_signatures())`
- Let *get-complsigs*<*Sndr*, *Env*...>() be expression-equivalent to `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`. Let *NewSndr* be *Sndr* if `sizeof...(Env) == 0` is true; otherwise, `decltype(s)` where *s* is the following expression:
`transform_sender(
 declval<Sndr>(),
 declval<Env>()...)`
- 2 Constraints: `sizeof...(Env) <= 1` is true.
- 3 Effects: Equivalent to: `return e`; where *e* is expression-equivalent to the following:
- (3.1) — *CHECKED-COMPLSIGS*(*get-complsigs*<*NewSndr*, *Env*...>()) if *get-complsigs*<*NewSndr*, *Env*...>() is a well-formed expression.

- (3.2) — Otherwise, *CHECKED-COMPLSIGS*(*get-complsig*s<NewSndr>()) if *get-complsig*s<NewSndr>() is a well-formed expression.
- (3.3) — Otherwise,
- ```
completion_signatures<
 SET-VALUE-SIG(await-result-type<NewSndr, env-promise<Env>...>), // 33.9.3
 set_error_t(exception_ptr),
 set_stopped_t()>
```
- if *is-awaitable*<NewSndr, *env-promise*<Env>...> is true.
- (3.4) — Otherwise, (throw *dependent-sender-error*(), *completion\_signatures*()) if *sizeof*...(Env) == 0 is true, where *dependent-sender-error* is *dependent\_sender\_error* or an unspecified type derived publicly and unambiguously from *dependent\_sender\_error*.
- (3.5) — Otherwise, (throw *except*, *completion\_signatures*()).
- 4 Given a type Env, if *completion\_signatures\_of\_t*<Sndr> and *completion\_signatures\_of\_t*<Sndr, Env> are both well-formed, the former shall be a superset of the latter, with the difference corresponding to error or stopped completion operations.
- 5 Let *rcvr* be an rvalue whose type *Rcvr* models receiver, and let *Sndr* be the type of a sender such that *sender\_in*<Sndr, *env\_of\_t*<*Rcvr*>> is true. Let *Sigs*... be the template arguments of the *completion\_signatures* specialization named by *completion\_signatures\_of\_t*<Sndr, *env\_of\_t*<*Rcvr*>>. Let *CS0* be a completion function. If sender *Sndr* or its operation state cause the expression *CS0*(*rcvr*, *args*...) to be potentially evaluated (6.3) then there shall be a signature *Sig* in *Sigs*... such that
- ```
MATCHING-SIG(decayed-typeof<CS0>(decltype(args)...), Sig)
```
- is true (33.1).

33.9.10 execution::connect

[exec.connect]

- 1 *connect* connects (33.3) a sender with a receiver.
- 2 The name *connect* denotes a customization point object. For subexpressions *sndr* and *rcvr*, let *Sndr* be *decltype*((*sndr*)) and *Rcvr* be *decltype*((*rcvr*)), let *new_sndr* be the expression *transform_sender*(*sndr*, *get_env*(*rcvr*)) and let *DS* and *DR* be *decay_t*<*decltype*((*new_sndr*))> and *decay_t*<*Rcvr*>, respectively.
- 3 Let *connect-awaitable-promise* be the following exposition-only class:

```
namespace std::execution {
    struct connect-awaitable-promise : with-await-transform<connect-awaitable-promise> {

        connect-awaitable-promise(DS&, DR& rcvr) noexcept : rcvr(rcvr) {}

        suspend_always initial_suspend() noexcept { return {}; }
        [[noreturn]] suspend_always final_suspend() noexcept { terminate(); }
        [[noreturn]] void unhandled_exception() noexcept { terminate(); }
        [[noreturn]] void return_void() noexcept { terminate(); }

        coroutine_handle<> unhandled_stopped() noexcept {
            set_stopped(std::move(rcvr));
            return noop_coroutine();
        }

        operation-state-task get_return_object() noexcept {
            return operation-state-task{
                coroutine_handle<connect-awaitable-promise>::from_promise(*this)};
        }

        env_of_t<DR> get_env() const noexcept {
            return execution::get_env(rcvr);
        }

    private:
        DR& rcvr; // exposition only
    };
}
```

```
};
}
```

- 4 Let *operation-state-task* be the following exposition-only class:

```
namespace std::execution {
    struct operation-state-task {                                // exposition only
        using operation_state_concept = operation_state_tag;
        using promise_type = connect-awaitable-promise;

        explicit operation-state-task(coroutine_handle<> h) noexcept : coro(h) {}
        operation-state-task(operation-state-task&&) = delete;
        ~operation-state-task() { coro.destroy(); }

        void start() & noexcept {
            coro.resume();
        }

    private:
        coroutine_handle<> coro;                                // exposition only
    };
}
```

- 5 Let *V* name the type *await-result-type*<*DS*, *connect-awaitable-promise*>, let *Sigs* name the type

```
completion_signatures<
    SET-VALUE-SIG(V), // see 33.9.3
    set_error_t(exception_ptr),
    set_stopped_t(>>
```

and let *connect-awaitable* be an exposition-only coroutine defined as follows:

```
namespace std::execution {
    template<class Fun, class... Ts>
    auto suspend-complete(Fun fun, Ts&&... as) noexcept { // exposition only
        auto fn = [&, fun]() noexcept { fun(std::forward<Ts>(as)...); };

        struct awaiter {
            decltype(fn) fn; // exposition only

            static constexpr bool await_ready() noexcept { return false; }
            void await_suspend(coroutine_handle<>) noexcept { fn(); }
            [[noreturn]] void await_resume() noexcept { unreachable(); }
        };
        return awaiter{fn};
    }

    operation-state-task connect-awaitable(DS sndr, DR rcvr) requires receiver-of<DR, Sigs> {
        exception_ptr ep;
        try {
            if constexpr (same_as<V, void>) {
                co_await std::move(sndr);
                co_await suspend-complete(set_value, std::move(rcvr));
            } else {
                co_await suspend-complete(set_value, std::move(rcvr), co_await std::move(sndr));
            }
        } catch(...) {
            ep = current_exception();
        }
        co_await suspend-complete(set_error, std::move(rcvr), std::move(ep));
    }
}
```

- 6 The expression *connect*(*sndr*, *rcvr*) is expression-equivalent to:

(6.1) — *new_sndr*.*connect*(*rcvr*) if that expression is well-formed.

Mandates: The type of the expression above satisfies *operation_state*.

- (6.2) — Otherwise, *connect-awaitable*(*new_sndr*, *rcvr*).

Except that *rcvr* is evaluated only once. The program is ill-formed, no diagnostic required, if there exists an rvalue expression *rcvr2* such that:

- (6.3) — *decltype*(*rcvr2*) models receiver,
 (6.4) — *noexcept*(*rcvr2*) is true,
 (6.5) — *is_same_v*<*decltype*(*get_env*(*rcvr2*)), *decltype*(*get_env*(*rcvr*))> is true,
 (6.6) — *noexcept*(*execution::connect*(*sndr*, *rcvr*)) is true, and
 (6.7) — *noexcept*(*execution::connect*(*sndr*, *rcvr2*)) is well-formed and evaluates to false.

[*Note 1*: This allows determination of whether *connect* throws with only the context of the environment, such as within *get_completion_signatures*. — *end note*]

Mandates: The following are true:

- (6.8) — *sender_in*<*Sndr*, *env_of_t*<*Rcvr*>>
 (6.9) — *receiver-of*<*Rcvr*, *completion_signatures_of_t*<*Sndr*, *env_of_t*<*Rcvr*>>>

33.9.11 Sender factories

[*exec.factories*]

33.9.11.1 *execution::schedule*

[*exec.schedule*]

- ¹ *schedule* obtains a schedule sender (33.3) from a scheduler.
- ² The name *schedule* denotes a customization point object. For a subexpression *sch*, the expression *schedule*(*sch*) is expression-equivalent to *sch.schedule*() .
- ³ *Mandates*: The type of *sch.schedule*() satisfies *sender*.

33.9.11.2 *execution::just*, *execution::just_error*, *execution::just_stopped*

[*exec.just*]

- ¹ *just*, *just_error*, and *just_stopped* are sender factories whose asynchronous operations complete synchronously in their start operation with a value completion operation, an error completion operation, or a stopped completion operation, respectively.
- ² The names *just*, *just_error*, and *just_stopped* denote customization point objects. Let *just-cpo* be one of *just*, *just_error*, or *just_stopped*. For a pack of subexpressions *ts*, let *Ts* be the pack of types *decltype*((*ts*)). The expression *just-cpo*(*ts...*) is ill-formed if
 - (2.1) — (*movable-value*<*Ts*> &&...) is false, or
 - (2.2) — *just-cpo* is *just_error* and *sizeof...(ts) == 1* is false, or
 - (2.3) — *just-cpo* is *just_stopped* and *sizeof...(ts) == 0* is false.

Otherwise, it is expression-equivalent to *make-sender*(*just-cpo*, *product-type*{*ts...*}).

For *just*, *just_error*, and *just_stopped*, let *set-cpo* be *set_value*, *set_error*, and *set_stopped*, respectively. The exposition-only class template *impls-for* (33.9.2) is specialized for *just-cpo* as follows:

```
namespace std::execution {
  template<>
  struct impls-for<decayed-typeof<just-cpo>> : default-impls {
    static constexpr auto start =
      [](auto& state, auto& rcvr) noexcept -> void {
        auto& [...ts] = state;
        set-cpo(std::move(rcvr), std::move(ts)...);
      };
  };
}
```

33.9.11.3 *execution::read_env*

[*exec.read.env*]

- ¹ *read_env* is a sender factory for a sender whose asynchronous operation completes synchronously in its start operation with a value completion result equal to a value read from the receiver's associated environment.
- ² *read_env* is a customization point object. For some query object *q*, the expression *read_env*(*q*) is expression-equivalent to *make-sender*(*read_env*, *q*).
- ³ The exposition-only class template *impls-for* (33.9.2) is specialized for *read_env* as follows:

```

namespace std::execution {
    template<>
    struct impls-for<decayed-typeof<read_env>> : default-impls {
        static constexpr auto start =
            [] (auto query, auto& rcvr) noexcept -> void {
                TRY-SET-VALUE(rcvr, query(get_env(rcvr)));
            };
    };

    template<class Sndr, class Env>
    static consteval void check-types();
}

```

```

template<class Sndr, class Env>
static consteval void check-types();

```

4 Let Q be `decay_t<data-type<Sndr>>`.

5 *Throws:* An exception of type *unspecified-exception* (33.9.1) if the expression `Q()(env)` is ill-formed or has type `void`, where `env` is an lvalue subexpression whose type is `Env`.

33.9.12 Sender adaptors

[exec.adapt]

33.9.12.1 General

[exec.adapt.general]

- 1 Subclause 33.9.12 specifies a set of sender adaptors.
- 2 The bitwise inclusive OR operator is overloaded for the purpose of creating sender chains. The adaptors also support function call syntax with equivalent semantics.
- 3 Unless otherwise specified:
 - (3.1) — A sender adaptor is prohibited from causing observable effects, apart from moving and copying its arguments, before the returned sender is connected with a receiver using `connect`, and `start` is called on the resulting operation state.
 - (3.2) — A parent sender (33.3) with a single child sender `sndr` has an associated attribute object equal to `FWD-ENV(get_env(sndr))` (33.5.2) except that the `get_completion_scheduler<completion-tag>` and `get_completion_domain<completion-tag>` queries are handled as described in 33.9.1.
 - (3.3) — A parent sender with more than one child sender has an associated attributes object equal to `env<>{}` except that the `get_completion_scheduler<completion-tag>` and `get_completion_domain<completion-tag>` queries are handled as described in 33.9.1.
 - (3.4) — When a parent sender is connected to a receiver `rcvr`, any receiver used to connect a child sender has an associated environment equal to `FWD-ENV(get_env(rcvr))`.
 - (3.5) — An adaptor whose child senders are all non-dependent (33.3) is itself non-dependent.
 - (3.6) — These requirements apply to any function that is selected by the implementation of the sender adaptor.
 - (3.7) — *Recommended practice:* Implementations should use the completion signatures of the adaptors to communicate type errors to users and to propagate any such type errors from child senders.
- 4 If a sender returned from a sender adaptor specified in 33.9.12 is specified to include `set_error_t(Err)` among its set of completion signatures where `decay_t<Err>` denotes the type `exception_ptr`, but the implementation does not potentially evaluate an error completion operation with an `exception_ptr` argument, the implementation is allowed to omit the `exception_ptr` error completion signature from the set.

33.9.12.2 Closure objects

[exec.adapt.obj]

- 1 A *pipeable sender adaptor closure object* is a function object that accepts one or more `sender` arguments and returns a `sender`. For a pipeable sender adaptor closure object `c` and an expression `sndr` such that `decltype((sndr))` models `sender`, the following expressions are equivalent and yield a `sender`:

```

c(sndr)
sndr | c

```

Given an additional pipeable sender adaptor closure object `d`, the expression `c | d` produces another pipeable sender adaptor closure object `e`:

`e` is a perfect forwarding call wrapper (22.10.4) with the following properties:

- (1.1) — Its target object is an object `d2` of type `decltype(auto(d))` direct-non-list-initialized with `d`.
- (1.2) — It has one bound argument entity, an object `c2` of type `decltype(auto(c))` direct-non-list-initialized with `c`.
- (1.3) — Its call pattern is `d2(c2(arg))`, where `arg` is the argument used in a function call expression of `e`.

The expression `c | d` is well-formed if and only if the initializations of the state entities (22.10.3) of `e` are all well-formed.

- 2 An object `t` of type `T` is a pipeable sender adaptor closure object if `T` models `derived_from<sender_adaptor_closure<T>>`, `T` has no other base classes of type `sender_adaptor_closure<U>` for any other type `U`, and `T` does not satisfy `sender`.
- 3 The template parameter `D` for `sender_adaptor_closure` can be an incomplete type. Before any expression of type `cv D` appears as an operand to the `|` operator, `D` shall be complete and model `derived_from<sender_adaptor_closure<D>>`. The behavior of an expression involving an object of type `cv D` as an operand to the `|` operator is undefined if overload resolution selects a program-defined `operator|` function.
- 4 A *pipeable sender adaptor object* is a customization point object that accepts a `sender` as its first argument and returns a `sender`. If a pipeable sender adaptor object accepts only one argument, then it is a pipeable sender adaptor closure object.
- 5 If a pipeable sender adaptor object adaptor accepts more than one argument, then let `sndr` be an expression such that `decltype((sndr))` models `sender`, let `args...` be arguments such that `adaptor(sndr, args...)` is a well-formed expression as specified below, and let `BoundArgs` be a pack that denotes `decltype(auto(args))...`. The expression `adaptor(args...)` produces a pipeable sender adaptor closure object `f` that is a perfect forwarding call wrapper with the following properties:
 - (5.1) — Its target object is a copy of adaptor.
 - (5.2) — Its bound argument entities `bound_args` consist of objects of types `BoundArgs...` direct-non-list-initialized with `std::forward, respectively.`
 - (5.3) — Its call pattern is `adaptor(rcvr, bound_args...)`, where `rcvr` is the argument used in a function call expression of `f`.

The expression `adaptor(args...)` is well-formed if and only if the initializations of the bound argument entities of the result, as specified above, are all well-formed.

33.9.12.3 execution::write_env

[exec.write.env]

- 1 `write_env` is a sender adaptor that accepts a sender and a queryable object, and that returns a sender that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the *queryable* object to the result of `get_env(rcvr)`.
- 2 `write_env` is a customization point object. For some subexpressions `sndr` and `env`, if `decltype((sndr))` does not satisfy `sender` or if `decltype((env))` does not satisfy *queryable*, the expression `write_env(sndr, env)` is ill-formed. Otherwise, it is expression-equivalent to `make_sender(write_env, env, sndr)`.
- 3 Let *write-env-t* denote the type `decltype(auto(write_env))`. The exposition-only class template *impls-for* (33.9.2) is specialized for *write-env-t* as follows:

```
template<>
struct impls_for<write-env-t> : default_impls {
    static constexpr auto join_env(const auto& state, const auto& env) noexcept {
        return see below;
    }

    static constexpr auto get_env =
        [](auto, const auto& state, const auto& rcvr) noexcept {
            return join_env(state, FWD-ENV(get_env(rcvr)));
        };

    template<class Sndr, class... Env>
        static consteval void check_types();
};
```

- 4 Invocation of `impls_for<write-env-t>::join_env` returns an object `e` such that
 - (4.1) — `decltype(e)` models *queryable* and

(4.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to `state.query(q)` if that expression is valid, otherwise, `e.query(q)` is expression-equivalent to `env.query(q)`.

- ⁵ For a type `Sndr` and a pack of types `Env`, let `State` be `data-type<Sndr>` and let `JoinEnv` be the pack `decltype(join-env(declval<State>(), FWD-ENV(declval<Env>())))`. Then `impls-for<write-env-t>::check-types<Sndr, Env...>()` is expression-equivalent to `get_completion_signatures<child-type<Sndr>, JoinEnv...>()`.

33.9.12.4 execution::unstoppable

[exec.unstoppable]

- ¹ `unstoppable` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver but with an object of type `never_stop_token` as the result of the `get_stop_token` query.
- ² For a subexpression `sndr`, `unstoppable(sndr)` is expression-equivalent to `write_env(sndr, prop(get_stop_token, never_stop_token{}))`.

33.9.12.5 execution::starts_on

[exec.starts.on]

- ¹ `starts_on` adapts an input sender into a sender that will start on an execution agent belonging to a particular scheduler's associated execution resource.
- ² The name `starts_on` denotes a customization point object. For subexpressions `sch` and `sndr`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, `starts_on(sch, sndr)` is ill-formed.
- ³ Otherwise, the expression `starts_on(sch, sndr)` is expression-equivalent to: `make-sender(starts_on, sch, sndr)`.
- ⁴ Let `out_sndr` and `env` be subexpressions such that `OutSndr` is `decltype((out_sndr))`. If `sender-for<OutSndr, starts_on_t>` is false, then the expression `starts_on.transform_sender(set_value, out_sndr, env)` is ill-formed; otherwise `starts_on.transform_sender(set_value, out_sndr, env)` is equivalent to:

```
auto&& [_, sch, sndr] = out_sndr;
return let_value(
    continues_on(just(), sch),
    [sndr = std::forward_like<OutSndr>(sndr)]() mutable
    noexcept(is_nothrow_move_constructible_v<decay_t<OutSndr>>) {
        return std::move(sndr);
    });
```

- ⁵ Let `out_sndr` be a subexpression denoting a sender returned from `starts_on(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is true. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall start `sndr` on an execution agent of the associated execution resource of `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

33.9.12.6 execution::continues_on

[exec.continues.on]

- ¹ `continues_on` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource.
- ² The name `continues_on` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy `scheduler`, or `Sndr` does not satisfy `sender`, `continues_on(sndr, sch)` is ill-formed.
- ³ Otherwise, the expression `continues_on(sndr, sch)` is expression-equivalent to `make-sender(continues_on, sch, schedule_from(sndr))`.
- ⁴ The exposition-only class template `impls-for` (33.9.2) is specialized for `continues_on_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<continues_on_t> : default-impls {
        static constexpr auto get_state = see below;
        static constexpr auto complete = see below;
    };
}
```

```

    template<class Sndr, class... Env>
        static consteval void check-types();
};

```

- 5 The member *impls-for*<continues_on_t>::*get-state* is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below)
    requires sender_in<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)> {
    auto& [_ , sch, child] = sndr;

    using sched_t = decltype(auto(sch));
    using variant_t = see below;
    using receiver_t = see below;
    using operation_t = connect_result_t<schedule_result_t<sched_t>, receiver_t>;
    constexpr bool nothrow = noexcept(connect(schedule(sch), receiver_t{nullptr}));

    struct state-type {
        Rcvr& rcvr;                // exposition only
        variant_t async-result;  // exposition only
        operation_t op-state;    // exposition only

        explicit state-type(sched_t sch, Rcvr& rcvr) noexcept(nothrow)
            : rcvr(rcvr), op-state(connect(schedule(sch), receiver_t{this})) {}
    };

    return state-type{sch, rcvr};
}

```

```

template<class Sndr, class... Env>
    static consteval void check-types();

```

- 6 *Effects*: Equivalent to:

```

    get_completion_signatures<schedule_result_t<data-type<Sndr>>, FWD-ENV-T(Env)...>();
    auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
    decay-copyable-result-datums(cs);    // see 33.9.2

```

- 7 Objects of the local class *state-type* can be used to initialize a structured binding.

- 8 Let Sigs be a pack of the arguments to the *completion_signatures* specialization named by *completion_signatures_of_t*<*child-type*<Sndr>, *FWD-ENV-T*(env_of_t<Rcvr>)>. Let *as-tuple* be an alias template such that *as-tuple*<Tag(Args...)> denotes the type *decayed-tuple*<Tag, Args...>, and let *is-nothrow-decay-copy-sig* be a variable template such that *auto*(*is-nothrow-decay-copy-sig*<Tag(Args...)>) is a constant expression of type *bool* and equal to (*is_nothrow_constructible_v*<*decay_t*<Args>, Args> && ...). Let *error-completion* be a pack consisting of the type *set_error_t*(*exception_ptr*) if (*is-nothrow-decay-copy-sig*<Sigs> &&...) is false, and an empty pack otherwise. Then *variant_t* denotes the type *variant*<*monostate*, *as-tuple*<Sigs>..., *error-completion*...>, except with duplicate types removed.

- 9 *receiver_t* is an alias for the following exposition-only class:

```

namespace std::execution {
    struct receiver-type {
        using receiver_concept = receiver_tag;
        state-type* state;    // exposition only

        void set_value() && noexcept {
            visit(
                [this]<class Tuple>(Tuple& result) noexcept -> void {
                    if constexpr (!same_as<monostate, Tuple>) {
                        auto& [tag, ...args] = result;
                        tag(std::move(state->rcvr), std::move(args)...);
                    }
                },
                state->async-result);
        }
    };
}

```



```

template<class Error>
void set_error(Error&& err) && noexcept {
    execution::set_error(std::move(state->rcvr), std::forward<Error>(err));
}

void set_stopped() && noexcept {
    execution::set_stopped(std::move(state->rcvr));
}

decltype(auto) get_env() const noexcept {
    return FWD-ENV(execution::get_env(state->rcvr));
}
};
}

```

- ¹⁰ The expression in the `noexcept` clause of the lambda is `true` if the construction of the returned *state-type* object is not potentially throwing; otherwise, `false`.
- ¹¹ The member *impls-for*<continues_on_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```

[]<class Tag, class... Args>(auto, auto& state, auto& rcvr, Tag, Args&&... args) noexcept
-> void {
    using result_t = decayed-tuple<Tag, Args...>;
    constexpr bool nothrow = (is_nothrow_constructible_v<decay_t<Args>, Args> && ...);

    try {
        state.async-result.template emplace<result_t>(Tag(), std::forward<Args>(args)...);
    } catch (...) {
        if constexpr (!nothrow)
            state.async-result.template emplace<tuple<set_error_t,
                                                    exception_ptr>>(set_error, current_exception());
    }
    start(state.op-state);
};

```

- ¹² Let `out_sndr` be a subexpression denoting a sender returned from `continues_on_t(sndr, sch)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is `true`. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall start `sndr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

33.9.12.7 execution::schedule_from

[exec.schedule.from]

- ¹ The name `schedule_from` denotes a customization point object. For some subexpression `sndr`, if `decltype((sndr))` does not satisfy `sender`, `schedule_from(sndr)` is ill-formed. Otherwise, the expression `schedule_from(sndr)` is expression-equivalent to *make-sender*(`schedule_from`, {}, `sndr`).

[Note 1: `schedule_from` is used by schedulers to control how to transition off of their schedulers' associated execution contexts. — end note]

33.9.12.8 execution::on

[exec.on]

- ¹ The `on` sender adaptor has two forms:
- (1.1) — `on(sch, sndr)`, which starts a sender `sndr` on an execution agent belonging to a scheduler `sch`'s associated execution resource and that, upon `sndr`'s completion, transfers execution back to the execution resource on which the `on` sender was started.
 - (1.2) — `on(sndr, sch, closure)`, which upon completion of a sender `sndr`, transfers execution to an execution agent belonging to a scheduler `sch`'s associated execution resource, then executes a sender adaptor closure `closure` with the async results of the sender, and that then transfers execution back to the execution resource on which `sndr` completed.
- ² The name `on` denotes a pipeable sender adaptor object. For subexpressions `sch` and `sndr`, `on(sch, sndr)` is ill-formed if any of the following is `true`:

- (2.1) — `decltype((sch))` does not satisfy `scheduler`, or
 - (2.2) — `decltype((sndr))` does not satisfy `sender` and `sndr` is not a pipeable sender adaptor closure object (33.9.12.2), or
 - (2.3) — `decltype((sndr))` satisfies `sender` and `sndr` is also a pipeable sender adaptor closure object.
- 3 Otherwise, if `decltype((sndr))` satisfies `sender`, the expression `on(sch, sndr)` is expression-equivalent to `make-sender(on, sch, sndr)`.
- 4 For subexpressions `sndr`, `sch`, and `closure`, if
- (4.1) — `decltype((sch))` does not satisfy `scheduler`, or
 - (4.2) — `decltype((sndr))` does not satisfy `sender`, or
 - (4.3) — `closure` is not a pipeable sender adaptor closure object (33.9.12.2),
- the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is expression-equivalent to `make-sender(on, product-type{sch, closure}, sndr)`.
- 5 Let `out_sndr` and `env` be subexpressions, let `OutSndr` be `decltype((out_sndr))`, and let `Env` be `decltype((env))`. If `sender-for<OutSndr, on_t>` is false, then the expression `on.transform_sender(set_value, out_sndr, env)` is ill-formed.
- 6 Otherwise, the expression `on.transform_sender(set_value, out_sndr, env)` has effects equivalent to:
- ```

auto&& [_, data, child] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
 auto orig_sch = call-with-default(get_start_scheduler, not-a-scheduler(), env);
 return continues_on(
 starts_on(std::forward_like<OutSndr>(data), std::forward_like<OutSndr>(child)),
 std::move(orig_sch));
} else {
 auto& [sch, closure] = data;
 auto orig_sch = call-with-default(
 get_completion_scheduler<set_value_t>, not-a-scheduler(), get_env(child), env);
 return continues_on(
 std::forward_like<OutSndr>(closure)(continues_on(std::forward_like<OutSndr>(child), sch)),
 orig_sch);
}

```
- 7 Let `out_sndr` be a subexpression denoting a sender returned from `on(sch, sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is true. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall
- (7.1) — remember the current scheduler, which is obtained by `get_start_scheduler(get_env(rcvr))`;
  - (7.2) — start `sndr` on an execution agent belonging to `sch`'s associated execution resource;
  - (7.3) — upon `sndr`'s completion, transfer execution back to the execution resource associated with the scheduler remembered in step 1; and
  - (7.4) — forward `sndr`'s async result to `out_rcvr`.
- If any scheduling operation fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.
- 8 Let `out_sndr` be a subexpression denoting a sender returned from `on(sndr, sch, closure)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSndr, Env>` is true. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` with `out_rcvr`. Calling `start(op)` shall
- (8.1) — remember the current scheduler, which is `get_completion_scheduler<set_value_t>(get_env(sndr), get_env(out_rcvr))`;
  - (8.2) — start `sndr` on the current execution agent;
  - (8.3) — upon `sndr`'s completion, transfer execution to an agent owned by `sch`'s associated execution resource;
  - (8.4) — forward `sndr`'s async result as if by connecting and starting a sender `closure(S)`, where `S` is a sender that completes synchronously with `sndr`'s async result; and

- (8.5) — upon completion of the operation started in the previous step, transfer execution back to the execution resource associated with the scheduler remembered in step 1 and forward the operation's async result to `out_rcvr`.

If any scheduling operation fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

### 33.9.12.9 `execution::then`, `execution::upon_error`, `execution::upon_stopped` [exec.then]

- 1 `then` attaches an invocable as a continuation for an input sender's value completion operation. `upon_error` and `upon_stopped` do the same for the error and stopped completion operations, respectively, sending the result of the invocable as a value completion.
- 2 The names `then`, `upon_error`, and `upon_stopped` denote pipeable sender adaptor objects. Let the expression `then-cpo` be one of `then`, `upon_error`, or `upon_stopped`. For subexpressions `sndr` and `f`, if `decltype((sndr))` does not satisfy `sender`, or `decltype((f))` does not satisfy `movable-value`, `then-cpo(sndr, f)` is ill-formed.
- 3 Otherwise, the expression `then-cpo(sndr, f)` is expression-equivalent to `make-sender(then-cpo, f, sndr)`.
- 4 For `then`, `upon_error`, and `upon_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped`, respectively. The exposition-only class template `impls-for` (33.9.2) is specialized for `then-cpo` as follows:

```
namespace std::execution {
 template<>
 struct impls-for<decayed-typeof<then-cpo>> : default-impls {
 static constexpr auto complete =
 []<class Tag, class... Args>
 (auto, auto& fn, auto& rcvr, Tag, Args&&... args) noexcept -> void {
 if constexpr (same_as<Tag, decayed-typeof<set-cpo>>) {
 TRY-SET-VALUE(rcvr,
 invoke(std::move(fn), std::forward<Args>(args)...));
 } else {
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
 }
 };

 template<class Sndr, class... Env>
 static constexpr void check-types();
 };
}
```

```
template<class Sndr, class... Env>
static constexpr void check-types();
```

- 5 *Effects:* Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
auto fn = []<class... Ts>(decayed-typeof<set-cpo>(*) (Ts...)) {
 if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>, Ts...>)
 throw unspecified-exception();
};
cs.for-each(overload-set{fn, [] (auto){}});
```

- 6 The expression `then-cpo(sndr, f)` has undefined behavior unless it returns a sender `out_sndr` that
  - (6.1) — invokes `f` or a copy of such with the value, error, or stopped result datums of `sndr` for `then`, `upon_error`, and `upon_stopped`, respectively, using the result value of `f` as `out_sndr`'s value completion, and
  - (6.2) — forwards all other completion operations unchanged.

### 33.9.12.10 `execution::let_value`, `execution::let_error`, `execution::let_stopped` [exec.let]

- 1 `let_value`, `let_error`, and `let_stopped` transform a sender's value, error, and stopped completions, respectively, into a new child asynchronous operation by passing the sender's result datums to a user-specified callable, which returns a new sender that is connected and started.
- 2 For `let_value`, `let_error`, and `let_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped`, respectively. Let the expression `let-cpo` be one of `let_value`, `let_error`, or `let_stopped`. Let `let-tag`

denote a unique, empty class type for each of `let_value`, `let_error`, and `let_stopped`. For subexpressions `sndr` and `env`, let `let-env(sndr, env)` be expression-equivalent to the first well-formed expression below:

- (2.1) — `SCHED-ENV(get_completion_scheduler<decayed-typeof<set-cpo>>(get_env(sndr)), FWD-ENV(env))`
- (2.2) — `MAKE-ENV(get_domain, get_completion_domain<decayed-typeof<set-cpo>>(get_env(sndr)), FWD-ENV(env))`
- (2.3) — `(void(sndr), env<>{})`

<sup>3</sup> The names `let_value`, `let_error`, and `let_stopped` denote pipeable sender adaptor objects. For subexpressions `sndr` and `f`, let `F` be the decayed type of `f`. If `decltype((sndr))` does not satisfy `sender` or if `decltype((f))` does not satisfy `movable-value`, the expression `let-cpo(sndr, f)` is ill-formed. If `F` does not satisfy `invocable`, the expression `let_stopped(sndr, f)` is ill-formed.

<sup>4</sup> Otherwise, the expression `let-cpo(sndr, f)` is expression-equivalent to `make-sender(let-cpo, f, sndr)`.

<sup>5</sup> Let `let-data` denote the following exposition-only class template:

```
template<class Sndr, class Fn>
struct let_data {
 Sndr sndr; // exposition only
 Fn fn; // exposition only
};
```

<sup>6</sup> Then let the expression `let-cpo.transform_sender(s, es...)` be expression-equivalent to:

```
make-sender(let-tag{}, let-data{s.template get<2>(), s.template get<1>()})
```

except that `s` is evaluated only once.

<sup>7</sup> The exposition-only class template `impls-for` (33.9.2) is specialized for `let-tag` as follows:

```
namespace std::execution {
 template<>
 struct impls_for<let-tag> : default_impls {
 static constexpr auto get_state = see below;
 static constexpr auto start = see below;

 template<class Sndr, class... Env>
 static constexpr void check_types();
 };
}
```

<sup>8</sup> Let `receiver2` denote the following exposition-only class template:

```
namespace std::execution {
 template<class Rcvr, class Env>
 struct receiver2 {
 using receiver_concept = receiver_tag;

 template<class... Args>
 void set_value(Args&&... args) && noexcept {
 execution::set_value(std::move(rcvr), std::forward<Args>(args)...);
 }

 template<class Error>
 void set_error(Error&& err) && noexcept {
 execution::set_error(std::move(rcvr), std::forward<Error>(err));
 }

 void set_stopped() && noexcept {
 execution::set_stopped(std::move(rcvr));
 }

 decltype(auto) get_env() const noexcept {
 return see below;
 }
 };
}
```

```

 Rcvr& rcvr; // exposition only
 Env env; // exposition only
};
}

```

Invocation of the function `receiver2::get_env` returns an object `e` such that

- (8.1) — `decltype(e)` models *queryable* and
- (8.2) — given a query object `q` and a pack of subexpressions `args`, the expression `e.query(q, args...)` is expression-equivalent to `env.query(q, args...)` if that expression is valid; otherwise, if the type of `q` satisfies *forwarding-query*, `e.query(q, args...)` is expression-equivalent to `get_env(rcvr).query(q, args...)`; otherwise, `e.query(q, args...)` is ill-formed.

```

template<class Sndr, class... Env>
static consteval void check-types();

```

9 *Effects*: Equivalent to:

```

using LetFn = remove_cvref_t<data-type<Sndr>>;
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
auto fn = []<class... Ts>(decayed-typeof<set-cpo>(*) (Ts...)) {
 if constexpr (!is-valid-let-sender) // see below
 throw unspecified-exception();
};
cs.for-each(overload-set(fn, [] (auto){}));

```

where *is-valid-let-sender* is true if and only if all of the following are true:

- (9.1) — `(constructible_from<decay_t<Ts>, Ts> &&...)`
- (9.2) — `invocable<LetFn, decay_t<Ts>&...>`
- (9.3) — `sender<invoke_result_t<LetFn, decay_t<Ts>&...>>`
- (9.4) — `sizeof...(Env) == 0 || sender_in<invoke_result_t<LetFn, decay_t<Ts>&...>, env-t...>`

where *env-t* is the pack `decltype(JOIN-ENV(let-env(declval<child-type<Sndr>>()), declval<Env>()), FWD-ENV(declval<Env>()))`.

10 Let *let-state* denote the following exposition-only class template:

```

template<class Cpo, class Sndr, class Fn, class Rcvr, class ArgsVariant, class OpsVariant>
struct let-state {
 using env_t = decltype(let-env(declval<Sndr>(), get_env(declval<Rcvr&>()))); // exposition only
 Fn fn; // exposition only
 env_t env; // exposition only
 ArgsVariant args; // exposition only
 OpsVariant ops; // exposition only

 template<class Tag, class... Ts>
 constexpr void impl(Rcvr& rcvr, Tag tag, Ts&&... ts) noexcept { // exposition only
 using args_t = decayed-tuple<Ts...>;
 using receiver_type = receiver2<Rcvr, env_t>;
 using sender_type = apply_result_t<Fn, args_t&>;
 if constexpr (is_same_v<Tag, Cpo>) {
 try {
 auto& tuple = args.template emplace<args_t>(std::forward<Ts>(ts)...);
 ops.template emplace<monostate>();
 auto&& sndr = apply(std::move(fn), tuple);
 using op_t = connect_result_t<sender_type, receiver_type>;
 auto mkop2 = [&] {
 return connect(std::forward<sender_type>(sndr),
 receiver_type{rcvr, env});
 };
 auto& op = ops.template emplace<op_t>(emplace-from{mkop2});
 start(op);
 }
 }
 }
};

```

```

 } catch (...) {
 constexpr bool nothrow =
 is_nothrow_constructible_v<args_t, Ts...> &&
 is_nothrow_applicable_v<Fn, args_t&> &&
 noexcept(connect(declval<sender_type>(), receiver_type{rcvr, env}));
 if constexpr (!nothrow) {
 set_error(std::move(rcvr), current_exception());
 }
 }
} else {
 tag(std::move(rcvr), std::forward<Ts>(ts)...);
}
}

struct receiver { // exposition only
 let-state& state; // exposition only
 Rcvr& rcvr; // exposition only

 using receiver_concept = receiver_tag;

 template<class... Args>
 constexpr void set_value(Args&&... args) noexcept {
 state.impl(rcvr, execution::set_value, std::forward<Args>(args)...);
 }
 template<class... Args>
 constexpr void set_error(Args&&... args) noexcept {
 state.impl(rcvr, execution::set_error, std::forward<Args>(args)...);
 }
 template<class... Args>
 constexpr void set_stopped(Args&&... args) noexcept {
 state.impl(rcvr, execution::set_stopped, std::forward<Args>(args)...);
 }

 constexpr env_of_t<const Rcvr&> get_env() const noexcept {
 return execution::get_env(rcvr);
 }
};

using op_t = connect_result_t<Sndr, receiver>; // exposition only

constexpr let-state(Sndr&& sndr, Fn fn, Rcvr& rcvr) // exposition only
 : fn(std::move(fn)), env(let-env(sndr, get_env(rcvr))),
 ops(in_place_type<op_t>, std::forward<Sndr>(sndr), receiver{*this, rcvr}) {}
};

```

- 11 *impls-for<let-tag>::get-state* is initialized with a callable object equivalent to the following:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) requires see below {
 auto& [_ , data] = sndr;
 auto& [child, fn] = data;
 using child_t = decltype(std::forward_like<Sndr>(child));
 using fn_t = decay_t<decltype(fn)>;
 using args_variant_t = see below;
 using ops_variant_t = see below;
 using state_t = let-state<decayed-typeof<set-cpo>, child_t, fn_t, Rcvr,
 args_variant_t, ops_variant_t>;
 return state_t(std::forward_like<Sndr>(child), std::forward_like<Sndr>(fn), rcvr);
}

```

- 12 Let *Sigs* be a pack of the arguments to the *completion\_signatures* specialization named by *completion\_signatures\_of\_t<child-type<Sndr>, FWD-ENV-T(env\_of\_t<Rcvr>)>*. Let *LetSigs* be a pack of those types in *Sigs* with a return type of *decayed-typeof<set-cpo>*. Let *as-tuple* be an alias template such that *as-tuple<Tag(Args...)>* denotes the type *decayed-tuple<Args...>*. Then *args\_variant\_t* denotes the type *variant<monostate, as-tuple<LetSigs>...>* except with duplicate types removed.

- 13 Given a type `Tag` and a pack `Args`, let `as-sndr2` be an alias template such that `as-sndr2<Tag(Args...)>` denotes the type `call-result-t<F, decay_t<Args>&...>`. Then `ops_variant_t` denotes the type

```
variant<monostate,
 connect_result_t<child_t, let-state::receiver>,
 connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, env_t>>...>
```

except with duplicate types removed.

- 14 The *requires-clause* constraining the above lambda is satisfied if and only if the types `args_variant_t` and `ops2_variant_t` are well-formed.
- 15 `impls-for<let-tag>::start` is initialized with a callable object equivalent to the following:

```
[]<class State, class Rcvr>(State& state, Rcvr&) noexcept {
 start(get<typename State::op_t>(state.ops));
}
```

- 16 Let the subexpression `out_sndr` denote the result of the invocation `let-cpo(sndr, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation (33.3) that, when started:

- (16.1) — invokes `f` when `set-cpo` is called with `sndr`'s result datums,
- (16.2) — makes its completion dependent on the completion of a sender returned by `f`, and
- (16.3) — propagates the other completion operations sent by `sndr`.

#### 33.9.12.11 `execution::bulk`, `execution::bulk_chunked`, and `execution::bulk_unchunked` [`exec.bulk`]

- 1 `bulk`, `bulk_chunked`, and `bulk_unchunked` run a task repeatedly for every index in an index space.

- 2 The names `bulk`, `bulk_chunked`, and `bulk_unchunked` denote pipeable sender adaptor objects. Let `bulk-algo` be either `bulk`, `bulk_chunked`, or `bulk_unchunked`. For subexpressions `sndr`, `policy`, `shape`, and `f`, let `Policy` be `remove_cvref_t<decltype(policy)>`, `Shape` be `decltype(auto(shape))`, and `Func` be `decay_t<decltype((f))>`. If

- (2.1) — `decltype((sndr))` does not satisfy `sender`, or
- (2.2) — `is_execution_policy_v<Policy>` is false, or
- (2.3) — `Shape` does not satisfy `integral`, or
- (2.4) — `Func` does not model `copy_constructible`,

`bulk-algo(sndr, policy, shape, f)` is ill-formed.

- 3 Otherwise, the expression `bulk-algo(sndr, policy, shape, f)` is expression-equivalent to:

```
make-sender(bulk-algo, product-type<see below, Shape, Func>{policy, shape, f}, sndr)
```

The first template argument of *product-type* is `Policy` if `Policy` models `copy_constructible`, and `const Policy&` otherwise.

- 4 Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If `sender-for<Sndr, bulk_t>` is false, then the expression `bulk.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto [_, data, child] = sndr;
auto& [policy, shape, f] = data;
auto new_f = [func = std::move(f)](Shape begin, Shape end, auto&&... vs)
 noexcept(noexcept(f(begin, vs...))) {
 while (begin != end) func(begin++, vs...);
}
return bulk_chunked(std::move(child), policy, shape, std::move(new_f));
```

[Note 1: This causes the `bulk(sndr, policy, shape, f)` sender to be expressed in terms of `bulk_chunked(sndr, policy, shape, f)` when it is connected to a receiver whose execution domain does not customize `bulk`. — end note]

- 5 The exposition-only class template `impls-for` (33.9.2) is specialized for `bulk_chunked_t` as follows:

```

namespace std::execution {
 template<>
 struct impls-for<bulk_chunked_t> : default-impls {
 static constexpr auto complete = see below;

 template<class Sndr, class... Env>
 static consteval void check-types();
 };
}

```

The member *impls-for*<bulk\_chunked\_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```

[]<class Index, class State, class Rcvr, class Tag, class... Args>
(Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
-> void requires see below {
 if constexpr (same_as<Tag, set_value_t>) {
 auto& [policy, shape, f] = state;
 constexpr bool nothrow = noexcept(f(auto(shape), auto(shape), args...));
 TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
 f(static_cast<decltype(auto(shape))>(0), auto(shape), args...);
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
 }());
 } else {
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
 }
}

```

The expression in the *requires-clause* of the lambda above is *true* if and only if *Tag* denotes a type other than *set\_value\_t* or if the expression *f*(*auto*(*shape*), *auto*(*shape*), *args*...) is well-formed.

```

template<class Sndr, class... Env>
 static consteval void check-types();

```

6     *Effects*: Equivalent to:

```

 auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...\>();
 auto fn = []<class... Ts>(set_value_t(*) (Ts...)) {
 using data_type = data-type<Sndr>;
 if constexpr (!invocable<remove_cvref_t<child-type<data_type>&,
 remove_cvref_t<data-type<data_type>>, Ts&...\>)
 throw unspecified-exception();
 };
 cs.for-each(overload-set(fn, [] (auto){}));

```

7     The exposition-only class template *impls-for* (33.9.2) is specialized for *bulk\_unchunked\_t* as follows:

```

namespace std::execution {
 template<>
 struct impls-for<bulk_unchunked_t> : default-impls {
 static constexpr auto complete = see below;

 template<class Sndr, class... Env>
 static consteval void check-types();
 };
}

```

The member *impls-for*<bulk\_unchunked\_t>::*complete* is initialized with a callable object equivalent to the following lambda:

```

[]<class Index, class State, class Rcvr, class Tag, class... Args>
(Index, State& state, Rcvr& rcvr, Tag, Args&&... args) noexcept
-> void requires see below {
 if constexpr (same_as<Tag, set_value_t>) {
 auto& [policy, shape, f] = state;
 constexpr bool nothrow = noexcept(f(auto(shape), args...));
 TRY-EVAL(rcvr, [&]() noexcept(nothrow) {
 for (decltype(auto(shape)) i = 0; i < shape; ++i) {
 f(auto(i), args...);
 }
 });
 }
}

```

```

 }
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
}());
} else {
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
}
}

```

The expression in the *requires-clause* of the lambda above is `true` if and only if `Tag` denotes a type other than `set_value_t` or if the expression `f(auto(shape), args...)` is well-formed.

```

template<class Sndr, class... Env>
static consteval void check-types();

```

8 *Effects*: Equivalent to:

```

 auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
 auto fn = []<class... Ts>(set_value_t*)(Ts...) {
 using data_type = data-type<Sndr>;
 if constexpr (!invocable<remove_cvref_t<child-type<data_type>&,
 remove_cvref_t<data-type<data_type>>, Ts&...>)
 throw unspecified-exception();
 };
 cs.for-each(overload-set(fn, [] (auto){}));

```

9 Let the subexpression `out_sndr` denote the result of the invocation `bulk_algo(sndr, policy, shape, f)` or an object equal to such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation (33.3) that, when started:

(9.1) — If `sndr` has a successful completion, where `args` is a pack of lvalue subexpressions referring to the value completion result datums of `sndr`, or decayed copies of those values if they model `copy_constructible`, then:

(9.1.1) — If `out_sndr` also completes successfully, then:

(9.1.1.1) — for `bulk`, invokes `f(i, args...)` for every `i` of type `Shape` from 0 to `shape`;

(9.1.1.2) — for `bulk_unchunked`, invokes `f(i, args...)` for every `i` of type `Shape` from 0 to `shape`;

*Recommended practice*: The underlying scheduler should execute each iteration on a distinct execution agent.

(9.1.1.3) — for `bulk_chunked`, invokes `f(b, e, args...)` zero or more times with pairs of `b` and `e` of type `Shape` in range `[0, shape]`, such that `b < e` and for every `i` of type `Shape` from 0 to `shape`, there is exactly one invocation with a pair `b` and `e`, such that `i` is in the range `[b, e)`.

(9.1.2) — If `out_sndr` completes with `set_error(rcvr, eptr)`, then the asynchronous operation may invoke a subset of the invocations of `f` before the error completion handler is called, and `eptr` is an `exception_ptr` containing either:

(9.1.2.1) — an exception thrown by an invocation of `f`, or

(9.1.2.2) — a `bad_alloc` exception if the implementation fails to allocate required resources, or

(9.1.2.3) — an exception derived from `runtime_error`.

(9.1.3) — If `out_sndr` completes with `set_stopped(rcvr)`, then the asynchronous operation may invoke a subset of the invocations of `f` before the stopped completion handler.

(9.2) — If `sndr` does not complete with `set_value`, then the completion is forwarded to `recv`.

(9.3) — For `bulk_algo`, the parameter `policy` describes the manner in which the execution of the asynchronous operations corresponding to these algorithms may be parallelized and the manner in which they apply `f`. Permissions and requirements on parallel algorithm element access functions (26.3.3) apply to `f`.

10 [Note 2: The asynchronous operation corresponding to `bulk_algo(sndr, policy, shape, f)` can complete with `set_stopped` if cancellation is requested or ignore cancellation requests. — end note]

### 33.9.12.12 `execution::when_all`

[`exec.when.all`]

1 `when_all` and `when_all_with_variant` both adapt multiple input senders into a sender that completes when all input senders have completed. `when_all` only accepts senders with a single value completion signature and



on success concatenates all the input senders' value result datums into its own value completion operation. `when_all_with_variant(sndrs...)` is semantically equivalent to `when_all(into_variant(sndrs)...)`, where `sndrs` is a pack of subexpressions whose types model `sender`.

- <sup>2</sup> The names `when_all` and `when_all_with_variant` denote customization point objects. Let `sndrs` be a pack of subexpressions and let `Sndrs` be a pack of the types `decltype((sndrs))...`. The expressions `when_all(sndrs...)` and `when_all_with_variant(sndrs...)` are ill-formed if any of the following is true:

(2.1) — `sizeof...(sndrs)` is 0, or

(2.2) — `(sender<Sndrs> && ...)` is false.

- <sup>3</sup> The expression `when_all(sndrs...)` is expression-equivalent to `make_sender(when_all, {}, sndrs...)`.

- <sup>4</sup> The exposition-only class template *impls-for* (33.9.2) is specialized for `when_all_t` as follows:

```
namespace std::execution {
 template<>
 struct impls-for<when_all_t> : default-impls {
 static constexpr auto get-env = see below;
 static constexpr auto get-state = see below;
 static constexpr auto start = see below;
 static constexpr auto complete = see below;

 template<class Sndr, class... Env>
 static consteval void check-types();
 };
}
```

- <sup>5</sup> Let *make-when-all-env* be the following exposition-only function template:

```
template<class Env>
constexpr auto make-when-all-env(inplace_stop_source& stop_src, // exposition only
 Env&& env) noexcept;
```

- <sup>6</sup> *Returns*: An object `e` such that

(6.1) — `decltype(e)` models *queryable*, and

(6.2) — `e.query(get_stop_token)` is expression-equivalent to `stop_src.get_token()`, and

(6.3) — given a query object `q` with type other than `cv get_stop_token_t`, `e.query(q)` is expression-equivalent to `env.query(q)` if the type of `q` satisfies *forwarding-query*, and ill-formed otherwise.

- <sup>7</sup> Let *when-all-env* be an alias template such that `when-all-env<Env>` denotes the type `decltype(make-when-all-env(declval<inplace_stop_source&>(), declval<Env>()))`.

```
template<class Sndr, class... Env>
static consteval void check-types();
```

- <sup>8</sup> Let `Is` be the pack of integral template arguments of the `integer_sequence` specialization denoted by *indices-for*<Sndr>.

- <sup>9</sup> *Effects*: Equivalent to:

```
auto fn = []<class Child>() {
 auto cs = get_completion_signatures<Child, when-all-env<Env>...>();
 if constexpr (cs.count-of(set_value) >= 2)
 throw unspecified-exception();
 decay-copyable-result-datums(cs); // see 33.9.2
};
(fn.template operator()<child-type<Sndr, Is>>(), ...);
```

- <sup>10</sup> The member `impls-for<when_all_t>::get-env` is initialized with a callable object equivalent to the following lambda expression:

```
[]<class State, class Rcvr>(auto&&, State& state, const Rcvr& rcvr) noexcept {
 return make-when-all-env(state.stop_src, get_env(rcvr));
}
```

- <sup>11</sup> The member `impls-for<when_all_t>::get-state` is initialized with a callable object equivalent to the following lambda expression:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(noexcept(e)) -> decltype(e) {
 return e;
}

```

where  $e$  is the expression

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

and where *make-state* is the following exposition-only class template:

```

enum class disposition { started, error, stopped }; // exposition only

template<class Rcvr>
struct make-state {
 template<class... Sndrs>
 auto operator()(auto, auto, Sndrs&&... sndrs) const {
 using values_tuple = see below;
 using errors_variant = see below;
 using stop_callback = stop_callback_for_t<stop_token_of_t<env_of_t<Rcvr>>, on-stop-request>;

 struct state-type {
 void arrive(Rcvr& rcvr) noexcept { // exposition only
 if (0 == --count) {
 complete(rcvr);
 }
 }

 void complete(Rcvr& rcvr) noexcept; // exposition only

 atomic<size_t> count{sizeof...(sndrs)}; // exposition only
 inplace_stop_source stop_src{}; // exposition only
 atomic<disposition> disp{disposition::started}; // exposition only
 errors_variant errors{}; // exposition only
 values_tuple values{}; // exposition only
 optional<stop_callback> on_stop{nullptr}; // exposition only
 };

 return state-type{};
 }
};

```

<sup>12</sup> Let *copy-fail* be `exception_ptr` if decay-copying any of the child senders' result datums can potentially throw; otherwise, *none-such*, where *none-such* is an unspecified empty class type.

<sup>13</sup> The alias `values_tuple` denotes the type

```
tuple<value_types_of_t<Sndrs, FWD-ENV-T(env_of_t<Rcvr>), decayed-tuple, optional>...>
```

if that type is well-formed; otherwise, `tuple<>`.

<sup>14</sup> The alias `errors_variant` denotes the type `variant<none-such, copy-fail, Es...>` with duplicate types removed, where *Es* is the pack of the decayed types of all the child senders' possible error result datums.

<sup>15</sup> The member `void state-type::complete(Rcvr& rcvr) noexcept` behaves as follows:

(15.1) — If `disp` is equal to `disposition::started`, evaluates:

```

auto tie = []<class... T>(tuple<T...>& t) noexcept { return tuple<T...>(t); };
auto set = [&](auto&... t) noexcept { set_value(std::move(rcvr), std::move(t)...); };

on_stop.reset();
apply(
 [&](auto&... opts) noexcept {
 apply(set, tuple_cat(tie(*opts)...));
 },
 values);

```

(15.2) — Otherwise, if `disp` is equal to `disposition::error`, evaluates:

```
on_stop.reset();
```

```

visit(
 [&]<class Error>(Error& error) noexcept {
 if constexpr (!same_as<Error, none-such>) {
 set_error(std::move(rcvr), std::move(error));
 }
 },
 errors);

```

(15.3) — Otherwise, evaluates:

```

if constexpr (sends-stopped) {
 on_stop.reset();
 set_stopped(std::move(rcvr));
}

```

where *sends-stopped* equals true if and only if there exists an element *S* of *Sndrs* such that *completion\_signatures\_of\_t*<*S*, *when-all-env*<*env\_of\_t*<*Rcvr*>>> contains *set\_stopped\_t*().

<sup>16</sup> The member *impls-for*<*when\_all\_t*>::*start* is initialized with a callable object equivalent to the following lambda expression:

```

[]<class State, class Rcvr, class... Ops>(
 State& state, Rcvr& rcvr, Ops&... ops) noexcept -> void {
 state.on_stop.emplace(
 get_stop_token(get_env(rcvr)),
 on-stop-request{state.stop_src});
 (start(ops), ...);
}

```

<sup>17</sup> The member *impls-for*<*when\_all\_t*>::*complete* is initialized with a callable object equivalent to the following lambda expression:

```

[]<class Index, class State, class Rcvr, class Set, class... Args>(
 this auto& complete, Index, State& state, Rcvr& rcvr, Set, Args&&... args) noexcept -> void {
 if constexpr (same_as<Set, set_error_t>) {
 if (disposition::error != state.disp.exchange(disposition::error)) {
 state.stop_src.request_stop();
 TRY-EMPLACE-ERROR(state.errors, std::forward<Args>(args)...);
 }
 } else if constexpr (same_as<Set, set_stopped_t>) {
 auto expected = disposition::started;
 if (state.disp.compare_exchange_strong(expected, disposition::stopped)) {
 state.stop_src.request_stop();
 }
 } else if constexpr (!same_as<decltype(State::values), tuple<>>) {
 if (state.disp == disposition::started) {
 auto& opt = get<Index::value>(state.values);
 TRY-EMPLACE-VALUE(complete, opt, std::forward<Args>(args)...);
 }
 }
 state.arrive(rcvr);
}

```

where *TRY-EMPLACE-ERROR* (*v*, *e*), for subexpressions *v* and *e*, is equivalent to:

```

try {
 v.template emplace<decltype(auto(e))>(e);
} catch (...) {
 v.template emplace<exception_ptr>(current_exception());
}

```

if the expression *decltype(auto(e))(e)* is potentially throwing; otherwise, *v.template emplace<decltype(auto(e))>(e)*; and where *TRY-EMPLACE-VALUE* (*c*, *o*, *as...*), for subexpressions *c*, *o*, and pack of subexpressions *as*, is equivalent to:

```

try {
 o.emplace(as...);
} catch (...) {
 c(Index(), state, rcvr, set_error, current_exception());
}

```

```
 return;
}
```

if the expression *decayed-tuple*<decltype(as)...>{as...} is potentially throwing; otherwise, `o.emplace(as...)`.

- 18 The expression `when_all_with_variant(sndrs...)` is expression-equivalent to *make-sender*(`when_all_with_variant, {}, sndrs...`).
- 19 Given subexpressions `sndr` and `env`, if *sender-for*<decltype((sndr)), `when_all_with_variant_t`> is `false`, then the expression `when_all_with_variant.transform_sender(set_value, sndr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_ , _ , ...child] = sndr;
return when_all(into_variant(std::forward_like<decltype((sndr))>(child))...);
```

[Note 1: This causes the `when_all_with_variant(sndrs...)` sender to become `when_all(into_variant(sndrs)...` when it is connected with a receiver whose execution domain does not customize `when_all_with_variant`. — end note]

### 33.9.12.13 execution::into\_variant

[exec.into.variant]

- 1 `into_variant` adapts a sender with multiple value completion signatures into a sender with just one value completion signature consisting of a `variant` of tuples.
- 2 The name `into_variant` denotes a pipeable sender adaptor object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `Sndr` does not satisfy *sender*, `into_variant(sndr)` is ill-formed.
- 3 Otherwise, the expression `into_variant(sndr)` is expression-equivalent to *make-sender*(`into_variant, {}, sndr`).
- 4 The exposition-only class template *impls-for* (33.9.2) is specialized for `into_variant` as follows:

```
namespace std::execution {
 template<>
 struct impls_for<into_variant_t> : default_impls {
 static constexpr auto get_state = see below;
 static constexpr auto complete = see below;

 template<class Sndr, class... Env>
 static consteval void check_types() {
 auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-T(Env)...>();
 decay-copyable-result-datums(cs); // see 33.9.2
 }
 };
}
```

- 5 The member *impls-for*<`into_variant_t`>::*get-state* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept
-> type_identity<value_types_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>>> {
 return {};
}
```

- 6 The member *impls-for*<`into_variant_t`>::*complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class State, class Rcvr, class Tag, class... Args>(
 auto, State, Rcvr& rcvr, Tag, Args&&... args) noexcept -> void {
 if constexpr (same_as<Tag, set_value_t>) {
 using variant_type = State::type;
 TRY-SET-VALUE(rcvr, variant_type(decayed-tuple<Args...>{std::forward<Args>(args)...}));
 } else {
 Tag()(std::move(rcvr), std::forward<Args>(args)...);
 }
}
```

### 33.9.12.14 execution::stopped\_as\_optional

[exec.stopped.opt]

- 1 `stopped_as_optional` maps a sender's stopped completion operation into a value completion operation as a disengaged optional. The sender's value completion operation is also converted into an `optional`. The

result is a sender that never completes with stopped, reporting cancellation by completing with a disengaged optional.

- <sup>2</sup> The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `snr`, let `Snr` be `decltype((snr))`. The expression `stopped_as_optional(snr)` is expression-equivalent to `make_sender(stopped_as_optional, {}, snr)`.
- <sup>3</sup> The exposition-only class template `impls-for` (33.9.2) is specialized for `stopped_as_optional_t` as follows:

```
namespace std::execution {
 template<>
 struct impls-for<stopped_as_optional_t> : default-impls {
 template<class Snr, class... Env>
 static constexpr void check-types() {
 default-impls::check-types<Snr, Env...>();
 if constexpr (!requires {
 requires (!same_as<void, single-sender-value-type<child-type<Snr>,
 FWD-ENV-T(Env)...>>); })
 throw unspecified-exception();
 }
 };
}
```

- <sup>4</sup> Let `snr` and `env` be subexpressions such that `Snr` is `decltype((snr))` and `Env` is `decltype((env))`. If `sender-for<Snr, stopped_as_optional_t>` is false then the expression `stopped_as_optional.transform_sender(set_value, snr, env)` is ill-formed; otherwise, if `sender_in<child-type<Snr>, FWD-ENV-T(Env)>` is false, the expression `stopped_as_optional.transform_sender(set_value, snr, env)` is equivalent to `not-a-sender()`; otherwise, it is equivalent to:

```
auto&& [_, _, child] = snr;
using V = single-sender-value-type<child-type<Snr>, FWD-ENV-T(Env)>;
return let_stopped(
 then(std::forward_like<Snr>(child),
 [<class... Ts>(Ts&&... ts) noexcept(is_nothrow_constructible_v<V, Ts...>) {
 return optional<V>(in_place, std::forward<Ts>(ts)...);
 })),
 [<>() noexcept { return just(optional<V>()); }]);
```

### 33.9.12.15 `execution::stopped_as_error`

[exec.stopped.err]

- <sup>1</sup> `stopped_as_error` maps an input sender's stopped completion operation into an error completion operation as a custom error type. The result is a sender that never completes with stopped, reporting cancellation by completing with an error.
- <sup>2</sup> The name `stopped_as_error` denotes a pipeable sender adaptor object. For some subexpressions `snr` and `err`, let `Snr` be `decltype((snr))` and let `Err` be `decltype((err))`. If the type `Snr` does not satisfy `sender` or if the type `Err` does not satisfy `movable-value`, `stopped_as_error(snr, err)` is ill-formed. Otherwise, the expression `stopped_as_error(snr, err)` is expression-equivalent to `make_sender(stopped_as_error, err, snr)`.
- <sup>3</sup> Let `snr` and `env` be subexpressions such that `Snr` is `decltype((snr))` and `Env` is `decltype((env))`. If `sender-for<Snr, stopped_as_error_t>` is false, then the expression `stopped_as_error.transform_sender(set_value, snr, env)` is ill-formed; otherwise, it is equivalent to:

```
auto&& [_, err, child] = snr;
using E = decltype(auto(err));
return let_stopped(
 std::forward_like<Snr>(child),
 [err = std::forward_like<Snr>(err)]() mutable noexcept(is_nothrow_move_constructible_v<E>) {
 return just_error(std::move(err));
 });
```

### 33.9.12.16 `execution::associate`

[exec.associate]

- <sup>1</sup> `associate` tries to associate a sender with an async scope such that the scope can track the lifetime of any asynchronous operations created with the sender.
- <sup>2</sup> Let `associate-data` be the following exposition-only class template:

```

namespace std::execution {
 template<scope_token Token, sender Sender>
 struct associate_data { // exposition only
 using wrap_sender = // exposition only
 remove_cvref_t<decltype(declval<Token>().wrap(declval<Sender>()))>;
 using assoc_t = decltype(declval<Token>().try_associate()); // exposition only
 using sender_ref = // exposition only
 unique_ptr<wrap_sender, decltype([](auto* p) noexcept { destroy_at(p); })>;

 explicit associate_data(Token t, Sender&& s)
 : sndr(t.wrap(std::forward<Sender>(s))),
 assoc([&] {
 sender_ref guard{addressof(sndr)};
 auto assoc = t.try_associate();
 if (assoc) {
 guard.release();
 }
 return assoc;
 }) {}

 associate_data(const associate_data& other)
 noexcept(is_nothrow_copy_constructible_v<wrap_sender> &&
 noexcept(other.assoc.try_associate()));

 associate_data(associate_data&& other)
 noexcept(is_nothrow_move_constructible_v<wrap_sender>)
 : associate_data(std::move(other).release()) {}

 ~associate_data();

 pair<assoc_t, sender_ref> release() && noexcept;

 private:
 associate_data(pair<assoc_t, sender_ref> parts); // exposition only
 union {
 wrap_sender sndr; // exposition only
 };
 assoc_t assoc; // exposition only
 };

 template<scope_token Token, sender Sender>
 associate_data(Token, Sender&&) -> associate_data<Token, Sender>;
}

```

- 3 For an *associate\_data* object *a*, `bool(a.assoc)` is true if and only if an association was successfully made and is owned by *a*.

```

associate_data(const associate_data& other)
 noexcept(is_nothrow_copy_constructible_v<wrap_sender> &&
 noexcept(other.assoc.try_associate()));

```

- 4 *Constraints:* *wrap\_sender* models `copy_constructible`.

- 5 *Effects:* Initializes *assoc* with `other.assoc.try_associate()`. If `bool(assoc)` is true, initializes *sndr* with `other.sndr`.

```

associate_data(pair<assoc_t, sender_ref> parts);

```

- 6 *Effects:* Initializes *assoc* with `std::move(parts.first)`. If `bool(assoc)` is true, initializes *sndr* with `std::move(*parts.second)`.

```

~associate_data();

```

- 7 *Effects:* If `bool(assoc)` is true, destroys *sndr*.

```
pair<assoc_t, sender_ref> release() && noexcept;
```

- 8 *Effects:* Constructs an object *u* of type *sender\_ref* that is initialized with `addressof(sndr)` if `bool(assoc)` is true and with `nullptr` otherwise, then returns `pair{std::move(assoc), std::move(u)}`.

9 The name `associate` denotes a pipeable sender adaptor object. For subexpressions `sndr` and `token`:

- (9.1) — If `decltype((sndr))` does not satisfy `sender`, or `remove_cvref_t<decltype((token))>` does not satisfy `scope_token`, then `associate(sndr, token)` is ill-formed.
- (9.2) — Otherwise, the expression `associate(sndr, token)` is expression-equivalent to `make_sender(associate, associate_data(token, sndr))`.

10 The exposition-only class template *impls-for* (33.9.2) is specialized for `associate_t` as follows:

```
namespace std::execution {
 template<>
 struct impls_for<associate_t> : default_impls {
 static constexpr auto get_state = see below; // exposition only
 static constexpr auto start = see below; // exposition only

 template<class Sndr, class... Env>
 static constexpr void check_types() { // exposition only
 using associate_data_t = remove_cvref_t<data_type<Sndr>>;
 using child_type_t = associate_data_t::wrap_sender;
 (void)get_completion_signatures<child_type_t, FWD-ENV-T(Env)...>();
 }
 };
}
```

11 The member `impls_for<associate_t>::get_state` is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below) {
 auto&& [_ , data] = std::forward<Sndr>(sndr);

 using associate_data_t = remove_cvref_t<decltype(data)>;
 using assoc_t = associate_data_t::assoc_t;
 using sender_ref_t = associate_data_t::sender_ref;

 using op_t = connect_result_t<typename sender_ref_t::element_type, Rcvr>;

 struct op_state {
 assoc_t assoc; // exposition only
 union {
 Rcvr* rcvr; // exposition only
 op_t op; // exposition only
 };

 explicit op_state(pair<assoc_t, sender_ref_t> parts, Rcvr& r)
 : assoc(std::move(parts.first)) {
 if (assoc) {
 ::new (voidify(op)) op_t(connect(std::move(*parts.second), std::move(r)));
 } else {
 rcvr = addressof(r);
 }
 }

 explicit op_state(associate_data_t&& ad, Rcvr& r)
 : op_state(std::move(ad).release(), r) {}

 explicit op_state(const associate_data_t& ad, Rcvr& r)
 requires copy_constructible<associate_data_t>
 : op_state(associate_data_t(ad).release(), r) {}

 op_state(op_state&&) = delete;
 };
}
```

```

~op_state() {
 if (assoc) {
 op.~op_t();
 }
}

void run() noexcept { // exposition only
 if (assoc) {
 start(op);
 } else {
 set_stopped(std::move(*rcvr));
 }
}
};

return op_state{std::forward_like<Sndr>(data), rcvr};
}

```

- <sup>12</sup> The expression in the noexcept clause of *impls-for*<associate\_t>::get-state is

```

(is_same_v<Sndr, remove_cvref_t<Sndr>> ||
 is_nothrow_constructible_v<remove_cvref_t<Sndr>, Sndr>) &&
nothrow_callable<connect_t, wrap_sender, Rcvr>

```

where *wrap\_sender* is the type *remove\_cvref\_t*<*data-type*<Sndr>>::*wrap\_sender*.

- <sup>13</sup> The member *impls-for*<associate\_t>::start is initialized with a callable object equivalent to the following lambda:

```

[] (auto& state, auto&) noexcept -> void {
 state.run();
}

```

- <sup>14</sup> The evaluation of *associate*(sndr, token) may cause side effects observable via token's associated async scope object.

### 33.9.12.17 Exposition-only execution::stop-when [exec.stop.when]

- <sup>1</sup> *stop-when* fuses an additional stop token *t* into a sender so that, upon connecting to a receiver *r*, the resulting operation state receives stop requests from both *t* and the token returned from *get\_stop\_token*(*get\_env*(*r*)).
- <sup>2</sup> The name *stop-when* denotes an exposition-only sender adaptor. For subexpressions *sndr* and *token*:
- (2.1) — If *decltype*((*sndr*)) does not satisfy *sender*, or *remove\_cvref\_t*<*decltype*((*token*))> does not satisfy *stoppable\_token*, then *stop-when*(*sndr*, *token*) is ill-formed.
  - (2.2) — Otherwise, if *remove\_cvref\_t*<*decltype*((*token*))> models *unstoppable\_token* then *stop-when*(*sndr*, *token*) is expression-equivalent to (void)*token*, *sndr*, except that *token* and *sndr* are indeterminately sequenced.
  - (2.3) — Otherwise, *stop-when*(*sndr*, *token*) returns a sender *osndr*. If *osndr* is connected to a receiver *r*, let *rtoken* be the result of *get\_stop\_token*(*get\_env*(*r*)).
    - (2.3.1) — If the type of *rtoken* models *unstoppable\_token* then the effects of connecting *osndr* to *r* are equivalent to *connect*(*write\_env*(*sndr*, *prop*(*get\_stop\_token*, *token*)), *r*).
    - (2.3.2) — Otherwise, the effects of connecting *osndr* to *r* are equivalent to *connect*(*write\_env*(*sndr*, *prop*(*get\_stop\_token*, *token*)), *r*) where *token* is an object of an exposition-only type *token-t* such that:
      - (2.3.2.1) — *token-t* models *stoppable\_token*;
      - (2.3.2.2) — *token*.stop\_requested() returns *token*.stop\_requested() || *rtoken*.stop\_requested();
      - (2.3.2.3) — *token*.stop\_possible() returns *token*.stop\_possible() || *rtoken*.stop\_possible(); and
      - (2.3.2.4) — for types *Fn* and *Init* such that both *invocable*<*Fn*> and *constructible\_from*<*Fn*, *Init*> are modeled, *token-t*::*callback\_type*<*Fn*> models *stoppable\_callback\_for*<*Fn*, *token-t*, *Init*>.



[*Note 1:* For an object `fn` of type `Fn` constructed from a value, `init`, of type `Init`, registering `fn` using `token-t::callback_type<Fn>(token, init)` results in an invocation of `fn` when a callback registered with `token` or `rtoken` would be invoked. `fn` is invoked at most once. — *end note*]

### 33.9.12.18 `execution::spawn_future`

[`exec.spawn.future`]

- <sup>1</sup> `spawn_future` attempts to associate the given input sender with the given token's async scope and, on success, eagerly starts the input sender; the return value is a sender that, when connected and started, completes with either the result of the eagerly-started input sender or with `set_stopped` if the input sender was not started.

- <sup>2</sup> The name `spawn_future` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`,

- (2.1) — let `Sndr` be `decltype((sndr))`,
- (2.2) — let `Token` be `remove_cvref_t<decltype((token))>`, and
- (2.3) — let `Env` be `remove_cvref_t<decltype((env))>`.

If any of `sender<Sndr>`, `scope_token<Token>`, or `queryable<Env>` are not satisfied, the expression `spawn_future(sndr, token, env)` is ill-formed.

- <sup>3</sup> Let `try-cancelable` be the exposition-only class:

```
namespace std::execution {
 struct try-cancelable { // exposition only
 virtual void try-cancel() noexcept = 0; // exposition only
 };
}
```

- <sup>4</sup> Let `spawn-future-state-base` be the exposition-only class template:

```
namespace std::execution {
 template<class Completions>
 struct spawn-future-state-base; // exposition only

 template<class... Sigs>
 struct spawn-future-state-base<completion_signatures<Sigs...>> // exposition only
 : try-cancelable {
 using variant-t = see below; // exposition only
 variant-t result; // exposition only
 virtual void complete() noexcept = 0; // exposition only
 };
}
```

- <sup>5</sup> Let `Sigs` be the pack of arguments to the `completion_signatures` specialization provided as a parameter to the `spawn-future-state-base` class template. Let `as-tuple` be an alias template that transforms a completion signature `Tag(Args...)` into the tuple specialization `decayed-tuple<Tag, Args...>`.

- (5.1) — If `is_nothrow_constructible_v<decay_t<Arg>, Arg>` is true for every type `Arg` in every parameter pack `Args` in every completion signature `Tag(Args...)` in `Sigs` then `variant-t` denotes the type `variant<monostate, tuple<set_stopped_t>, as-tuple<Sigs>...>`, except with duplicate types removed.
- (5.2) — Otherwise `variant-t` denotes the type `variant<monostate, tuple<set_stopped_t>, tuple<set_error_t, exception_ptr>, as-tuple<Sigs>...>`, except with duplicate types removed.

- <sup>6</sup> Let `spawn-future-receiver` be the exposition-only class template:

```
namespace std::execution {
 template<class Completions>
 struct spawn-future-receiver { // exposition only
 using receiver_concept = receiver_tag;

 spawn-future-state-base<Completions>* state; // exposition only

 template<class... T>
 void set_value(T&&... t) && noexcept {
 set-complete<set_value_t>(std::forward<T>(t)...);
 }
 };
}
```

```

template<class E>
void set_error(E&& e) && noexcept {
 set-complete<set_error_t>(std::forward<E>(e));
}

void set_stopped() && noexcept {
 set-complete<set_stopped_t>();
}

private:
template<class CPO, class... T>
void set-complete(T&&... t) noexcept { // exposition only
 constexpr bool nothrow = (is_nothrow_constructible_v<decay_t<T>, T> && ...);
 try {
 state->result.template emplace<decayed-tuple<CPO, T...>>(CPO{},
 std::forward<T>(t)...);
 }
 catch (...) {
 if constexpr (!nothrow) {
 using tuple_t = decayed-tuple<set_error_t, exception_ptr>;
 state->result.template emplace<tuple_t>(set_error_t{}, current_exception());
 }
 }
 state->complete();
}
};
}

```

- 7 Let *ssource-t* be an unspecified type that models *stoppable-source* and *default\_initializable*, such that a default-initialized object of type *ssource-t* has an associated stop state. Let *ssource* be an lvalue of type *ssource-t*. Let *stoken-t* be `decltype(ssource.get_token())`. Let *future-spawned-sender* be the alias template:

```

template<sender Sender, class Env>
using future-spawned-sender = // exposition only
 decltype(write_env(stop-when(declval<Sender>(), declval<stoken-t>()), declval<Env>())));

```

- 8 Let *spawn-future-state* be the exposition-only class template:

```

namespace std::execution {
template<class Alloc, scope_token Token, sender Sender, class Env>
struct spawn-future-state // exposition only
 : spawn-future-state-base<completion_signatures_of_t<future-spawned-sender<Sender, Env>,
 env<>>> {

 using sigs-t = // exposition only
 completion_signatures_of_t<future-spawned-sender<Sender, Env>, env<>>;
 using receiver-t = // exposition only
 spawn-future-receiver<sigs-t>;
 using op-t = // exposition only
 connect_result_t<future-spawned-sender<Sender, Env>, receiver-t>;

 spawn-future-state(Alloc alloc, Sender&& sndr, Token token, Env env) // exposition only
 : alloc(std::move(alloc)),
 op(connect(
 write_env(stop-when(std::forward<Sender>(sndr), ssource.get_token()), std::move(env)),
 receiver-t(this))),
 assoc(token.try_associate()) {
 if (assoc)
 start(op);
 else
 set_stopped(receiver-t(this));
 }

 void complete() noexcept override; // exposition only
 void consume(receiver auto& rcvr) noexcept; // exposition only
 void abandon() noexcept; // exposition only
}

```

```

void try-cancel() noexcept override { // exposition only
 ssource.request_stop();
 try-set-stopped();
}
void try-set-stopped() noexcept; // exposition only

private:
 using assoc-t = // exposition only
 remove_cvref_t<decltype(declval<Token&>().try_associate())>;

 Alloc alloc; // exposition only
 ssource-t ssource; // exposition only
 op-t op; // exposition only
 assoc-t assoc; // exposition only

 void destroy() noexcept; // exposition only
};

template<class Alloc, scope_token Token, sender Sender, class Env> // exposition only
 spawn-future-state(Alloc alloc, Sender&& sndr, Token token, Env env)
 -> spawn-future-state<Alloc, Token, Sender, Env>;
}

```

- <sup>9</sup> For purposes of determining the existence of a data race, *complete*, *consume*, *try-set-stopped*, and *abandon* behave as atomic operations (6.10.2). These operations on a single object of a type that is a specialization of *spawn-future-state* appear to occur in a single total order.

```
void complete() noexcept;
```

10 *Effects:*

- (10.1) — No effects if this invocation of *complete* happens before an invocation of *consume*, *try-set-stopped*, or *abandon* on *\*this*;
- (10.2) — otherwise, if an invocation of *consume* on *\*this* happens before this invocation of *complete* and no invocation of *try-set-stopped* on *\*this* happened before this invocation of *complete* then there is a receiver, *rcvr*, registered and that receiver is deregistered and completed as if by *consume(rcvr)*;
- (10.3) — otherwise, *destroy* is invoked.

```
void consume(receiver auto& rcvr) noexcept;
```

11 *Effects:*

- (11.1) — If this invocation of *consume* happens before an invocation of *complete* on *\*this* and no invocation of *try-set-stopped* on *\*this* happened before this invocation of *consume* then *rcvr* is registered to be completed when *complete* is subsequently invoked on *\*this*;
- (11.2) — otherwise, if this invocation of *consume* happens after an invocation of *try-set-stopped* on *\*this* and no invocation of *complete* on *\*this* happened before this invocation of *consume* then *rcvr* is completed as if by *set\_stopped(std::move(rcvr))*;
- (11.3) — otherwise, *rcvr* is completed as if by:

```

 std::move(this->result).visit(
 [&rcvr](auto&& tuple) noexcept {
 if constexpr (!same_as<remove_reference_t<decltype(tuple)>, monostate>) {
 apply([&rcvr](auto cpo, auto&&... vals) {
 cpo(std::move(rcvr), std::move(vals)...);
 }, std::move(tuple));
 }
 });
 destroy();

```

```
void try-set-stopped() noexcept;
```

12 *Effects:*

- (12.1) — If an invocation of *consume* on *\*this* happens before this invocation of *try-set-stopped* and no invocation of *complete* on *\*this* happened before this invocation of *try-set-stopped* then there is a receiver, *rcvr*, registered and that receiver is deregistered and completed as if by *set\_stopped(std::move(rcvr)), destroy()*;

- (12.2) — otherwise, no effects.

```
void abandon() noexcept;
```

13 *Effects:*

- (13.1) — If this invocation of *abandon* happens before an invocation of *complete* on *\*this* then equivalent to:

```
 ssource.request_stop();
```

- (13.2) — otherwise, *destroy* is invoked.

```
void destroy() noexcept;
```

14 *Effects:* Equivalent to:

```
 auto associated = std::move(this->associated);
 {
 using traits = allocator_traits<Alloc>::template rebind_traits<spawn-future-state>;
 typename traits::allocator_type alloc(std::move(this->alloc));
 traits::destroy(alloc, this);
 traits::deallocate(alloc, this, 1);
 }
```

- 15 Let *future-operation* be the exposition-only class template:

```
namespace std::execution {
 template<class StatePtr, class Rcvr>
 struct future-operation {
 struct callback {
 try-cancelable* state;
 // exposition only

 void operator()() noexcept {
 state->try-cancel();
 };
 };

 using stop-token-t =
 stop_token_of_t<env_of_t<Rcvr>>;
 // exposition only

 using stop-callback-t =
 stop_callback_for_t<stop-token-t, callback>;
 // exposition only

 struct rcvr-t {
 using receiver_concept = receiver_tag;
 future-operation* op;
 // exposition only

 template<class... T>
 void set_value(T&&... ts) && noexcept {
 op->set-complete<set_value_t>(std::forward<T>(ts)...);
 }

 template<class E>
 void set_error(E&& e) && noexcept {
 op->set-complete<set_error_t>(std::forward<E>(e));
 }

 void set_stopped() && noexcept {
 op->set-complete<set_stopped_t>();
 }

 env_of_t<Rcvr> get_env() const noexcept {
 return execution::get_env(op->rcvr);
 }
 };
 };
};
```

```

 }
};

Rcvr rcvr; // exposition only
StatePtr state; // exposition only
rcvr-t inner; // exposition only
optional<stop-callback-t> stopCallback; // exposition only

future-operation(StatePtr state, Rcvr rcvr) noexcept // exposition only
: rcvr(std::move(rcvr)), state(std::move(state)), inner(this)
{}

future-operation(future-operation&&) = delete;

void run() & noexcept { // exposition only
 constexpr bool nothrow =
 is_nothrow_constructible_v<stop-callback-t, stop-token-t, callback>;
 try {
 stopCallback.emplace(get_stop_token(rcvr), callback(state.get()));
 }
 catch (...) {
 if constexpr (!nothrow) {
 set_error(std::move(rcvr), current_exception());
 return;
 }
 }

 state.release()->consume(inner);
}

template<class CP0, class... T>
void set-complete(T&&... ts) noexcept { // exposition only
 stopCallback.reset();
 CP0{ }(std::move(rcvr), std::forward<T>(ts)...);
}
};
}

```

- 16 The exposition-only class template *impls-for* (33.9.2) is specialized for `spawn_future_t` as follows:

```

namespace std::execution {
 template<>
 struct impls-for<spawn_future_t> : default-impls {
 static constexpr auto start = see below; // exposition only
 static constexpr auto get-state = see below; // exposition only
 };
}

```

- 17 The member `impls-for<spawn_future_t>::start` is initialized with a callable object equivalent to the following lambda:

```

[](auto& state, auto&) noexcept -> void {
 state.run();
}

```

- 18 The member `impls-for<spawn_future_t>::get-state` is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr sndr, Rcvr& rcvr) noexcept {
 auto& [_ , data] = sndr;
 using state_ptr = remove_cvref_t<decltype(data)>;
 return future-operation<state_ptr, Rcvr>(std::move(data), std::move(rcvr));
}

```

- 19 For the expression `spawn_future(sndr, token, env)` let `new_sender` be the expression `token.wrap(sndr)` and let `alloc` and `senv` be defined as follows:

- (19.1) — if the expression `get_allocator(env)` is well-formed, then `alloc` is the result of `get_allocator(env)` and `senv` is the expression `env`;
  - (19.2) — otherwise, if the expression `get_allocator(get_env(new_sender))` is well-formed, then `alloc` is the result of `get_allocator(get_env(new_sender))` and `senv` is the expression `JOIN-ENV(prop(get_allocator, alloc), env)`;
  - (19.3) — otherwise, `alloc` is `allocator<void>()` and `senv` is the expression `env`.
- 20 The expression `spawn_future(sndr, token, env)` has the following effects:
- (20.1) — Uses `alloc` to allocate and construct an object `s` of type `decltype(spawn_future_state(alloc, token.wrap(sndr), token, senv))` from `alloc`, `token.wrap(sndr)`, `token`, and `senv`. If an exception is thrown then any constructed objects are destroyed and any allocated memory is deallocated.
  - (20.2) — Constructs an object `u` of a type that is a specialization of `unique_ptr` such that:
    - (20.2.1) — `u.get()` is equal to the address of `s`, and
    - (20.2.2) — `u.get_deleter()(u.release())` is equivalent to `u.release()->abandon()`.
  - (20.3) — Returns `make_sender(spawn_future, std::move(u))`.
- 21 The expression `spawn_future(sndr, token)` is expression-equivalent to `spawn_future(sndr, token, execution::env<>())`.

### 33.9.13 Sender consumers

[exec.consumers]

#### 33.9.13.1 `this_thread::sync_wait`

[exec.sync.wait]

- 1 `this_thread::sync_wait` and `this_thread::sync_wait_with_variant` are used to block the current thread of execution until the specified sender completes and to return its async result. `sync_wait` mandates that the input sender has exactly one value completion signature.

- 2 Let *sync-wait-env* be the following exposition-only class type:

```
namespace std::this_thread {
 struct sync-wait-env {
 execution::run_loop* loop; // exposition only

 auto query(execution::get_scheduler_t) const noexcept {
 return loop->get_scheduler();
 }

 auto query(execution::get_start_scheduler_t) const noexcept {
 return loop->get_scheduler();
 }

 auto query(execution::get_delegation_scheduler_t) const noexcept {
 return loop->get_scheduler();
 }
 };
}
```

- 3 Let *sync-wait-result-type* and *sync-wait-with-variant-result-type* be exposition-only alias templates defined as follows:

```
namespace std::this_thread {
 template<execution::sender_in<sync-wait-env> Sndr>
 using sync-wait-result-type =
 optional<execution::value_types_of_t<Sndr, sync-wait-env, decayed-tuple,
 type_identity_t>>;

 template<execution::sender_in<sync-wait-env> Sndr>
 using sync-wait-with-variant-result-type =
 optional<execution::value_types_of_t<Sndr, sync-wait-env>>;
}
```

- 4 The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. The expression `this_thread::sync_wait(sndr)` is equivalent to `apply_`

`sender(Domain(), sync_wait, sndr)`, where `Domain` is the type of `get_completion_domain<set_value_t>(get_env(sndr), sync_wait_env{})`.

5 *Mandates:*

- (5.1) — `sender_in<Sndr, sync_wait_env>` is true.
  - (5.2) — The type `sync_wait_result_type<Sndr>` is well-formed.
  - (5.3) — `same_as<decltype(e), sync_wait_result_type<Sndr>>` is true, where *e* is the `apply_sender` expression above.
- 6 Let `sync_wait_state` and `sync_wait_receiver` be the following exposition-only class templates:

```
namespace std::this_thread {
 template<class Sndr>
 struct sync_wait_state { // exposition only
 execution::run_loop loop; // exposition only
 exception_ptr error; // exposition only
 sync_wait_result_type<Sndr> result; // exposition only
 };

 template<class Sndr>
 struct sync_wait_receiver { // exposition only
 using receiver_concept = execution::receiver_tag;
 sync_wait_state<Sndr>* state; // exposition only

 template<class... Args>
 void set_value(Args&&... args) && noexcept;

 template<class Error>
 void set_error(Error&& err) && noexcept;

 void set_stopped() && noexcept;

 sync_wait_env get_env() const noexcept { return {&state->loop}; }
 };
}
```

```
template<class... Args>
void set_value(Args&&... args) && noexcept;
```

7 *Effects:* Equivalent to:

```
try {
 state->result.emplace(std::forward<Args>(args)...);
} catch (...) {
 state->error = current_exception();
}
state->loop.finish();
```

```
template<class Error>
void set_error(Error&& err) && noexcept;
```

8 *Effects:* Equivalent to:

```
state->error = AS-EXCEPT-PTR(std::forward<Error>(err)); // see 33.1
state->loop.finish();
```

```
void set_stopped() && noexcept;
```

9 *Effects:* Equivalent to `state->loop.finish()`.

- 10 For a subexpression `sndr`, let `Sndr` be `decltype((sndr))`. If `sender-to<Sndr, sync_wait_receiver<Sndr>>` is false, the expression `sync_wait.apply_sender(sndr)` is ill-formed; otherwise, it is equivalent to:

```
sync_wait_state<Sndr> state;
auto op = connect(sndr, sync_wait_receiver<Sndr>{&state});
start(op);
```

```

state.loop.run();
if (state.error) {
 rethrow_exception(std::move(state.error));
}
return std::move(state.result);

```

11 The behavior of `this_thread::sync_wait(sndr)` is undefined unless:

- (11.1) — It blocks the current thread of execution (3.6) with forward progress guarantee delegation (6.10.2.3) until the specified sender completes.  
*[Note 1: The default implementation of `sync_wait` achieves forward progress guarantee delegation by providing a `run_loop` scheduler via the `get_delegation_scheduler` query on the `sync-wait-receiver`'s environment. The `run_loop` is driven by the current thread of execution. — end note]*
- (11.2) — It returns the specified sender's async results as follows:
  - (11.2.1) — For a value completion, the result datums are returned in a `tuple` in an engaged `optional` object.
  - (11.2.2) — For an error completion, an exception is thrown.
  - (11.2.3) — For a stopped completion, a disengaged `optional` object is returned.

### 33.9.13.2 `this_thread::sync_wait_with_variant` [exec.sync.wait.var]

1 The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. The expression `this_thread::sync_wait_with_variant(sndr)` is equivalent to `apply_sender(Domain(), sync_wait_with_variant, sndr)`, where `Domain` is the type of `get_completion_domain<set_value_t>(get_env(sndr), sync-wait-env{})`.

2 *Mandates:*

- (2.1) — `sender_in<Sndr, sync-wait-env>` is true.
- (2.2) — The type `sync-wait-with-variant-result-type<Sndr>` is well-formed.
- (2.3) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is true, where `e` is the `apply_sender` expression above.

3 The expression `sync_wait_with_variant.apply_sender(sndr)` is equivalent to:

```

using result_type = sync-wait-with-variant-result-type<Sndr>;
if (auto opt_value = sync_wait(into_variant(sndr))) {
 return result_type(std::move(get<0>(*opt_value)));
}
return result_type(nullopt);

```

4 The behavior of `this_thread::sync_wait_with_variant(sndr)` is undefined unless:

- (4.1) — It blocks the current thread of execution (3.6) with forward progress guarantee delegation (6.10.2.3) until the specified sender completes.  
*[Note 1: The default implementation of `sync_wait_with_variant` achieves forward progress guarantee delegation by relying on the forward progress guarantee delegation provided by `sync_wait`. — end note]*
- (4.2) — It returns the specified sender's async results as follows:
  - (4.2.1) — For a value completion, the result datums are returned in an engaged `optional` object that contains a `variant` of `tuples`.
  - (4.2.2) — For an error completion, an exception is thrown.
  - (4.2.3) — For a stopped completion, a disengaged `optional` object is returned.

### 33.9.13.3 `execution::spawn` [exec.spawn]

1 `spawn` attempts to associate the given input sender with the given token's async scope and, on success, eagerly starts the input sender.

2 The name `spawn` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`,

- (2.1) — let `Sndr` be `decltype((sndr))`,
- (2.2) — let `Token` be `remove_cvref_t<decltype((token))>`, and
- (2.3) — let `Env` be `remove_cvref_t<decltype((env))>`.



If any of `sender<Sndr>`, `scope_token<Token>`, or `queryable<Env>` are not satisfied, the expression `spawn(sndr, token, env)` is ill-formed.

- <sup>3</sup> Let *spawn-state-base* be the exposition-only class:

```
namespace std::execution {
 struct spawn-state-base {
 virtual void complete() noexcept = 0;
 };
}
```

- <sup>4</sup> Let *spawn-receiver* be the exposition-only class:

```
namespace std::execution {
 struct spawn-receiver {
 using receiver_concept = receiver_tag;

 spawn-state-base* state;
 void set_value() && noexcept { state->complete(); }
 void set_stopped() && noexcept { state->complete(); }
 };
}
```

- <sup>5</sup> Let *spawn-state* be the exposition-only class template:

```
namespace std::execution {
 template<class Alloc, scope_token Token, sender Sender>
 struct spawn-state : spawn-state-base {
 using op-t = connect_result_t<Sender, spawn-receiver>;

 spawn-state(Alloc alloc, Sender&& sndr, Token token);
 void complete() noexcept override;
 void run() noexcept;

 private:
 using assoc-t =
 remove_cvref_t<decltype(declval<Token&>().try_associate())>;

 Alloc alloc;
 op-t op;
 assoc-t assoc;
 };
}
```

*spawn-state*(Alloc alloc, Sender&& sndr, Token token);

- <sup>6</sup> *Effects*: Initializes *alloc* with `std::move(alloc)`, *op* with `connect(std::move(sndr), spawn-receiver(this))`, and *assoc* with `token.try_associate()`.

`void run() noexcept;`

- <sup>7</sup> *Effects*: Equivalent to:

```
if (assoc)
 start(op);
else
 complete();
```

`void complete() noexcept override;`

- <sup>8</sup> *Effects*: Equivalent to:

```
auto assoc = std::move(this->assoc);
{
 using traits = allocator_traits<Alloc>::template rebind_traits<spawn-state>;
 typename traits::allocator_type alloc(this->alloc);
 traits::destroy(alloc, this);
 traits::deallocate(alloc, this, 1);
}
```

<sup>9</sup> For the expression `spawn(sndr, token, env)` let `new_sender` be the expression `token.wrap(sndr)` and let `alloc` and `senv` be defined as follows:

- (9.1) — if the expression `get_allocator(env)` is well-formed, then `alloc` is the result of `get_allocator(env)` and `senv` is the expression `env`,
- (9.2) — otherwise if the expression `get_allocator(get_env(new_sender))` is well-formed, then `alloc` is the result of `get_allocator(get_env(new_sender))` and `senv` is the expression `JOIN-ENV(prop(get_allocator, alloc), env)`,
- (9.3) — otherwise `alloc` is `allocator<void>()` and `senv` is the expression `env`.

<sup>10</sup> The expression `spawn(sndr, token, env)` is of type `void` and has the following effects:

- (10.1) — Uses `alloc` to allocate and construct an object `o` of type `decltype(spawn-state(alloc, write_env(token.wrap(sndr), senv), token))` from `alloc`, `write_env(token.wrap(sndr), senv)`, and `token` and then invokes `o.run()`. If an exception is thrown then any constructed objects are destroyed and any allocated memory is deallocated.
- <sup>11</sup> The expression `spawn(sndr, token)` is expression-equivalent to `spawn(sndr, token, execution::env<>())`.

### 33.10 Completion signatures

[exec.cmplsig]

<sup>1</sup> `completion_signatures` is a type that encodes a set of completion signatures (33.3).

<sup>2</sup> [Example 1:

```
struct my_sender {
 using sender_concept = sender_tag;
 using completion_signatures =
 execution::completion_signatures<
 set_value_t(),
 set_value_t(int, float),
 set_error_t(exception_ptr),
 set_error_t(error_code),
 set_stopped_t()>;
};
```

Declares `my_sender` to be a sender that can complete by calling one of the following for a receiver expression `rcvr`:

- (2.1) — `set_value(rcvr)`
  - (2.2) — `set_value(rcvr, int{...}, float{...})`
  - (2.3) — `set_error(rcvr, exception_ptr{...})`
  - (2.4) — `set_error(rcvr, error_code{...})`
  - (2.5) — `set_stopped(rcvr)`
- end example]

<sup>3</sup> This subclause makes use of the following exposition-only entities:

```
template<class Fn>
 concept completion_signature = see below;
```

<sup>4</sup> A type `Fn` satisfies *completion\_signature* if and only if it is a function type with one of the following forms:

- (4.1) — `set_value_t(Vs...)`, where `Vs` is a pack of object or reference types.
- (4.2) — `set_error_t(Err)`, where `Err` is an object or reference type.
- (4.3) — `set_stopped_t()`

```
5 template<bool>
 struct indirect-meta-apply {
 template<template<class...> class T, class... As>
 using meta-apply = T<As...>;
 // exposition only
 };
```

```
template<class...>
 concept always-true = true;
 // exposition only
```

```
template<class Tag,
 valid-completion-signatures Completions,
 template<class...> class Tuple,
 template<class...> class Variant>
using gather-signatures = see below;
```

- <sup>6</sup> Let *Fns* be a pack of the arguments of the *completion\_signatures* specialization named by *Completions*, let *TagFns* be a pack of the function types in *Fns* whose return types are *Tag*, and let *Ts<sub>n</sub>* be a pack of the function argument types in the *n*-th type in *TagFns*. Then, given two variadic templates *Tuple* and *Variant*, the type *gather-signatures*<*Tag*, *Completions*, *Tuple*, *Variant*> names the type

```
META-APPLY(Variant, META-APPLY(Tuple, Ts0...),
 META-APPLY(Tuple, Ts1...),
 ...,
 META-APPLY(Tuple, Tsm-1...))
```

where *m* is the size of the pack *TagFns* and *META-APPLY*(*T*, *As*...) is equivalent to:

```
typename indirect-meta-apply<always-true<As...>>::template meta-apply<T, As...>
```

- <sup>7</sup> [Note 1: The purpose of *META-APPLY* is to make it valid to use non-variadic templates as *Variant* and *Tuple* arguments to *gather-signatures*. — end note]

```
8 namespace std::execution {
 template<completion-signature... Fns>
 struct completion_signatures {
 template<class Tag>
 static constexpr size_t count-of(Tag) { return see below; }

 template<class Fn>
 static constexpr void for-each(Fn&& fn) { // exposition only
 (fn(static_cast<Fns*>(nullptr)), ...);
 }
 };

 template<class Sndr, class Env = env<>,
 template<class...> class Tuple = decayed-tuple,
 template<class...> class Variant = variant-or-empty>
 requires sender_in<Sndr, Env>
 using value_types_of_t =
 gather-signatures<set_value_t, completion_signatures_of_t<Sndr, Env>, Tuple, Variant>;

 template<class Sndr, class Env = env<>,
 template<class...> class Variant = variant-or-empty>
 requires sender_in<Sndr, Env>
 using error_types_of_t =
 gather-signatures<set_error_t, completion_signatures_of_t<Sndr, Env>,
 type_identity_t, Variant>;

 template<class Sndr, class Env = env<>>
 requires sender_in<Sndr, Env>
 constexpr bool sends_stopped =
 !same_as<type-list<>,
 gather-signatures<set_stopped_t, completion_signatures_of_t<Sndr, Env>,
 type-list, type-list>>;
}
```

- <sup>9</sup> For a subexpression *tag*, let *Tag* be the decayed type of *tag*. *completion\_signatures*<*Fns*...>::*count-of*(*tag*) returns the count of function types in *Fns*... that are of the form *Tag*(*Ts*...) where *Ts* is a pack of types.

### 33.11 Queryable utilities

[exec.envs]

#### 33.11.1 Class template *prop*

[exec.prop]

```
namespace std::execution {
 template<class QueryTag, class ValueType>
 struct prop {
 QueryTag query_; // exposition only
 };
}
```

```

ValueType value_; // exposition only

constexpr const ValueType& query(QueryTag, auto&&...) const noexcept {
 return value_;
}
};

template<class QueryTag, class ValueType>
prop(QueryTag, ValueType) -> prop<QueryTag, unwrap_reference_t<ValueType>>;
}

```

- <sup>1</sup> Class template `prop` is for building a queryable object from a query object and a value.
- <sup>2</sup> *Mandates:* `callable<QueryTag, prop-like<ValueType>>` is modeled, where *prop-like* is the following exposition-only class template:

```

template<class ValueType>
struct prop-like { // exposition only
 const ValueType& query(auto) const noexcept;
};

```

- <sup>3</sup> [Example 1:

```

template<sender Sndr>
sender auto parameterize_work(Sndr sndr) {
 // Make an environment such that get_allocator(env) returns a reference to a copy of my_alloc{}.
 auto e = prop(get_allocator, my_alloc{});

 // Parameterize the input sender so that it will use our custom execution environment.
 return write_env(sndr, e);
}

```

— end example]

- <sup>4</sup> Specializations of `prop` are not assignable.

### 33.11.2 Class template `env`

[exec.env]

```

namespace std::execution {
 template<queryable... Envs>
 struct env {
 Envs0 envs0; // exposition only
 Envs1 envs1; // exposition only
 :
 Envsn-1 envsn-1; // exposition only

 template<class QueryTag, class... Args>
 constexpr decltype(auto) query(QueryTag q, Args&&... args) const noexcept(see below);
 };

 template<class... Envs>
 env(Envs...) -> env<unwrap_reference_t<Envs>...>;
}

```

- <sup>1</sup> The class template `env` is used to construct a queryable object from several queryable objects. Query invocations on the resulting object are resolved by attempting to query each subobject in lexical order.
- <sup>2</sup> Specializations of `env` are not assignable.
- <sup>3</sup> It is unspecified whether `env` supports initialization using a parenthesized *expression-list* (9.5), unless the *expression-list* consist of a single element of type (possibly `const`) `env`.
- <sup>4</sup> [Example 1:

```

template<sender Sndr>
sender auto parameterize_work(Sndr sndr) {
 // Make an environment such that:
 // get_allocator(env) returns a reference to a copy of my_alloc{}
 // get_scheduler(env) returns a reference to a copy of my_sched{}
 auto e = env{prop(get_allocator, my_alloc{}),

```

```

 prop(get_scheduler, my_sched{}));

 // Parameterize the input sender so that it will use our custom execution environment.
 return write_env(sndr, e);
}
—end example]

```

```

template<class QueryTag, class... Args>
constexpr decltype(auto) query(QueryTag q, Args&&... args) const noexcept(see below);

```

5 Let *has-query* be the following exposition-only concept:

```

 template<class Env, class QueryTag, class... Args>
 concept has-query = // exposition only
 requires (const Env& env, Args&&... args) {
 env.query(QueryTag(), std::forward<Args>(args)...);
 };

```

6 Let *fe* be the first element of *envs*<sub>0</sub>, *envs*<sub>1</sub>, ..., *envs*<sub>*n*−1</sub> such that the expression *fe*.query(*q*, std::forward<Args>(args)...) is well-formed.

7 *Constraints*: (*has-query*<Env, QueryTag, Args...> || ...) is true.

8 *Effects*: Equivalent to: return *fe*.query(*q*, std::forward<Args>(args)...) ;

9 *Remarks*: The expression in the noexcept clause is equivalent to noexcept(*fe*.query(*q*, std::forward<Args>(args)...) ).

### 33.12 Execution contexts

[exec.ctx]

#### 33.12.1 execution::run\_loop

[exec.run.loop]

##### 33.12.1.1 General

[exec.run.loop.general]

- 1 A *run\_loop* is an execution resource on which work can be scheduled. It maintains a thread-safe first-in-first-out queue of work. Its *run* member function removes elements from the queue and executes them in a loop on the thread of execution that calls *run*.
- 2 A *run\_loop* instance has an associated *count* that corresponds to the number of work items that are in its queue. Additionally, a *run\_loop* instance has an associated state that can be one of *starting*, *running*, *finishing*, or *finished*.
- 3 Concurrent invocations of the member functions of *run\_loop* other than *run* and its destructor do not introduce data races. The member functions *pop-front*, *push-back*, and *finish* execute atomically.
- 4 *Recommended practice*: Implementations should use an intrusive queue of operation states to hold the work units to make scheduling allocation-free.

```

namespace std::execution {
 class run_loop {
 // 33.12.1.2, associated types
 class run-loop-scheduler; // exposition only
 class run-loop-sender; // exposition only
 struct run-loop-opstate-base { // exposition only
 virtual void execute() noexcept = 0; // exposition only
 run_loop* loop; // exposition only
 run-loop-opstate-base* next; // exposition only
 };
 template<class Rcvr>
 using run-loop-opstate = unspecified; // exposition only

 // 33.12.1.4, member functions
 run-loop-opstate-base* pop-front() noexcept; // exposition only
 void push-back(run-loop-opstate-base*) noexcept; // exposition only

 public:
 // 33.12.1.3, constructor and destructor
 run_loop() noexcept;
 run_loop(run_loop&&) = delete;
 ~run_loop();
 };
}

```

```

// 33.12.1.4, member functions
run-loop-scheduler get_scheduler() noexcept;
void run() noexcept;
void finish() noexcept;
};
}

```

### 33.12.1.2 Associated types

[exec.run.loop.types]

```
class run-loop-scheduler;
```

<sup>1</sup> *run-loop-scheduler* is an unspecified type that models *scheduler*.

<sup>2</sup> Instances of *run-loop-scheduler* remain valid until the end of the lifetime of the *run\_loop* instance from which they were obtained.

<sup>3</sup> Two instances of *run-loop-scheduler* compare equal if and only if they were obtained from the same *run\_loop* instance.

<sup>4</sup> Let *sch* be an expression of type *run-loop-scheduler*. The expression *schedule(sch)* has type *run-loop-sender* and is not potentially-throwing if *sch* is not potentially-throwing.

<sup>5</sup> For type *set-tag* other than *set\_error\_t*, the expression *get\_completion\_scheduler<set-tag>(get\_env(schedule(sch))) == sch* evaluates to *true*.

```
class run-loop-sender;
```

<sup>6</sup> *run-loop-sender* is an exposition-only type that satisfies *sender*. Let *E* be the type of an environment. If *unstoppable\_token<stop\_token\_of\_t<E>>* is *true*, then *completion\_signatures\_of\_t<run-loop-sender, E>* is *completion\_signatures<set\_value\_t()>*. Otherwise, it is *completion\_signatures<set\_value\_t(), set\_stopped\_t()>*.

<sup>7</sup> An instance of *run-loop-sender* remains valid until the end of the lifetime of its associated *run\_loop* instance.

<sup>8</sup> Let *sndr* be an expression of type *run-loop-sender*, let *rcvr* be an expression such that *receiver-of<decltype((rcvr)), CS>* is *true* where *CS* is the *completion\_signatures* specialization above. Let *C* be either *set\_value\_t* or *set\_stopped\_t*. Then:

- (8.1) — The expression *connect(sndr, rcvr)* has type *run-loop-opstate<decay\_t<decltype((rcvr))>>* and is potentially-throwing if and only if *(void(sndr), auto(rcvr))* is potentially-throwing.
- (8.2) — The expression *get\_completion\_scheduler<C>(get\_env(sndr))* has type *run-loop-scheduler* and compares equal to the *run-loop-scheduler* instance from which *sndr* was obtained.

```
template<class Rcvr>
struct run-loop-opstate;
```

<sup>9</sup> *run-loop-opstate<Rcvr>* inherits privately and unambiguously from *run-loop-opstate-base*.

<sup>10</sup> Let *o* be a non-const lvalue of type *run-loop-opstate<Rcvr>*, and let *REC(o)* be a non-const lvalue reference to an instance of type *Rcvr* that was initialized with the expression *rcvr* passed to the invocation of *connect* that returned *o*. Then:

- (10.1) — The object to which *REC(o)* refers remains valid for the lifetime of the object to which *o* refers.
- (10.2) — The type *run-loop-opstate<Rcvr>* overrides *run-loop-opstate-base::execute()* such that *o.execute()* is equivalent to:

```

if (get_stop_token(REC(o)).stop_requested()) {
 set_stopped(std::move(REC(o)));
} else {
 set_value(std::move(REC(o)));
}

```

- (10.3) — The expression *start(o)* is equivalent to:

```
o.loop->push-back(addressof(o));
```

**33.12.1.3 Constructor and destructor****[exec.run.loop.ctor]**`run_loop() noexcept;`

1 *Postconditions:* The `run_loop` instance's count is 0 and its state is starting.

`~run_loop();`

2 *Effects:* If the `run_loop` instance's count is not 0 or if its state is running, invokes `terminate` (14.6.2). Otherwise, has no effects.

**33.12.1.4 Member functions****[exec.run.loop.members]**`run-loop-opstate-base* pop-front() noexcept;`

1 *Effects:* Blocks (3.6) until one of the following conditions is `true`:

- (1.1) — The `run_loop` instance's count is 0 and its state is finishing, in which case `pop-front` sets the state to finished and returns `nullptr`; or
- (1.2) — the `run_loop` instance's count is greater than 0, in which case an item is removed from the front of the queue, the count is decremented by 1, and the removed item is returned.

`void push-back(run-loop-opstate-base* item) noexcept;`

2 *Effects:* Adds `item` to the back of the queue and increments the `run_loop` instance's count by 1.

3 *Synchronization:* This operation synchronizes with the `pop-front` operation that obtains `item`.

`run-loop-scheduler get_scheduler() noexcept;`

4 *Returns:* An instance of `run-loop-scheduler` that can be used to schedule work onto this `run_loop` instance.

`void run() noexcept;`

5 *Preconditions:* The `run_loop` instance's state is either starting or finishing.

6 *Effects:* If the `run_loop` instance's state is starting, sets the state to running, otherwise leaves the state unchanged. Then, equivalent to:

```
while (auto* op = pop-front()) {
 op->execute();
}
```

7 *Remarks:* When the `run_loop` instance's state changes, it does so without introducing data races.

`void finish() noexcept;`

8 *Preconditions:* The `run_loop` instance's state is either starting or running.

9 *Effects:* Changes the `run_loop` instance's state to finishing.

10 *Synchronization:* `finish` synchronizes with the `pop-front` operation that returns `nullptr`.

**33.13 Coroutine utilities****[exec.coro.util]****33.13.1 execution::as\_awaitable****[exec.as.awaitable]**

1 `as_awaitable` transforms an object into one that is awaitable within a particular coroutine. Subclause 33.13 makes use of the following exposition-only entities:

```
namespace std::execution {
 template<class Sndr, class Promise>
 concept awaitable-sender =
 single-sender<Sndr, env_of_t<Promise>>> &&
 sender-to<Sndr,
 typename sender-awaitable<Sndr, Promise>::awaitable-receiver> && // see below
 requires (Promise& p) {
 { p.unhandled_stopped() } -> convertible_to<coroutine_handle<>>;
 };
}
```

```
template<class Sndr>
concept has-queryable-await-completion-adaptor = // exposition only
 sender<Sndr> &&
 requires(Sndr&& sender) {
 get_await_completion_adaptor(get_env(sender));
 };

```

```
template<class Sndr, class Promise>
 class sender-awaitable; // exposition only
}

```

- <sup>2</sup> The type *sender-awaitable*<Sndr, Promise> is equivalent to:

```
namespace std::execution {
 template<class Sndr, class Promise>
 class sender-awaitable {
 struct unit {}; // exposition only
 using value-type = // exposition only
 single-sender-value-type<Sndr, env_of_t<Promise>>;
 using result-type = // exposition only
 conditional_t<is_void_v<value-type>, unit, value-type>;
 struct awaitable-receiver; // exposition only

 variant<monostate, result-type, exception_ptr> result{}; // exposition only
 connect_result_t<Sndr, awaitable-receiver> state; // exposition only

 public:
 sender-awaitable(Sndr&& sndr, Promise& p);
 static constexpr bool await_ready() noexcept { return false; }
 void await_suspend(coroutine_handle<Promise>) noexcept { start(state); }
 value-type await_resume();
 };
}

```

- <sup>3</sup> *awaitable-receiver* is equivalent to:

```
struct awaitable-receiver {
 using receiver_concept = receiver_tag;
 variant<monostate, result-type, exception_ptr>* result_ptr; // exposition only
 coroutine_handle<Promise> continuation; // exposition only
 // see below
};

```

- <sup>4</sup> Let *rcvr* be an rvalue expression of type *awaitable-receiver*, let *crcvr* be a const lvalue that refers to *rcvr*, let *vs* be a pack of subexpressions, and let *err* be an expression of type *Err*. Let *MAKE-NOEXCEPT*(*expr*) for some subexpression *expr* be expression-equivalent to *[&] noexcept -> decltype(auto) { return (expr); }()*. Then:

- (4.1) — The expression *set\_value(rcvr, vs...)* is equivalent to:

```
try {
 rcvr.result_ptr->template emplace<1>(vs...);
} catch(...) {
 rcvr.result_ptr->template emplace<2>(current_exception());
}
MAKE-NOEXCEPT(rcvr.continuation.resume());

```

*Mandates:* *constructible\_from<result-type, decltype((vs))...>* is satisfied.

- (4.2) — The expression *set\_error(rcvr, err)* is equivalent to:

```
try {
 rcvr.result_ptr->template emplace<2>(AS-EXCEPT-PTR(err)); // see 33.1
} catch(...) {
 rcvr.result_ptr->template emplace<2>(current_exception());
}
MAKE-NOEXCEPT(rcvr.continuation.resume());

```

- (4.3) — The expression *set\_stopped(rcvr)* is equivalent to:



- `MAKE-NOEXCEPT(`  
`static_cast<coroutine_handle<>>(<`  
`rcvr.continuation.promise().unhandled_stopped()).resume());`  
(4.4) — For any expression *tag* whose type satisfies *forwarding-query* and for any pack of subexpressions *as*, `get_env(rcvr).query(tag, as...)` is expression-equivalent to:  
`tag(get_env(as_const(MAKE-NOEXCEPT(rcvr.continuation.promise()))),`  
`as...)`  
`sender-awaitable(Sndr&& sndr, Promise& p);`  
5 *Effects:* Initializes *state* with  
`connect(std::forward<Sndr>(sndr),`  
`awaitable-receiver{addressof(result), coroutine_handle<Promise>::from_promise(p)});`  
`value-type await_resume();`  
6 *Effects:* Equivalent to:  
`if (result.index() == 2)`  
`rethrow_exception(get<2>(result));`  
`if constexpr (!is_void_v<value-type>)`  
`return std::forward<value-type>(get<1>(result));`  
7 *as\_awaitable* is a customization point object. For subexpressions *expr* and *p*, where *Expr* names the type `decltype((expr))` and *Promise* names the type `decay_t<decltype((p))>`, `as_awaitable(expr, p)` is ill-formed if *p* is not an lvalue, otherwise `as_awaitable(expr, p)` is expression-equivalent to the expression specified below, except that the evaluations of *expr* and *p* are indeterminately sequenced:  
(7.1) — `expr.as_awaitable(p)` if that expression is well-formed.  
*Mandates:* *is-awaitable*<*A*, *Promise*> is true, where *A* is the type of the expression above.  
(7.2) — Otherwise,  
`adapt-for-await-completion(transform_sender(expr, get_env(p))).as_awaitable(p)`  
if this expression is well-formed, `sender_in<Expr, env_of_t<Promise>>` is true, and *single-sender-value-type*<*Expr*, *env\_of\_t*<*Promise*>> is well-formed, except that *p* is only evaluated once.  
(7.3) — Otherwise, `(void(p), expr)` if `decltype(GET-AWAITER(expr))` satisfies *is-awaiter*<*Promise*>.  
(7.4) — Otherwise,  
`sender-awaitable{adapt-for-await-completion(transform_sender(expr, get_env(p))), p}`  
if `sender_in<Expr, env_of_t<Promise>>` is true and *single-sender-value-type*<*Expr*, *env\_of\_t*<*Promise*>> is well-formed, except that *p* is only evaluated once.  
(7.5) — Otherwise, `(void(p), expr)`.  
8 *adapt-for-await-completion*(*s*) is expression-equivalent to  
(8.1) — `get_await_completion_adaptor(get_env(s))(s)` if that is well-formed, except that *s* is evaluated only once,  
(8.2) — otherwise, *s*.

### 33.13.2 execution::with\_awaitable\_senders

[exec.with.awaitable.senders]

- 1 *with\_awaitable\_senders*, when used as the base class of a coroutine promise type, makes senders awaitable in that coroutine type.

In addition, it provides a default implementation of `unhandled_stopped` such that if a sender completes by calling `set_stopped`, it is treated as if an uncatchable "stopped" exception were thrown from the *await-expression*.

[Note 1: The coroutine is never resumed, and the `unhandled_stopped` of the coroutine caller's promise type is called.  
— end note]

```

namespace std::execution {
 template<class-type Promise>
 struct with_awaitable_senders {
 template<class OtherPromise>

```

```

 requires (!same_as<OtherPromise, void>)
void set_continuation(coroutine_handle<OtherPromise> h) noexcept;

coroutine_handle<> continuation() const noexcept { return continuation; }

coroutine_handle<> unhandled_stopped() noexcept {
 return stopped-handler(continuation.address());
}

template<class Value>
see below await_transform(Value&& value);

private:
[[noreturn]] static coroutine_handle<>
 default-unhandled-stopped(void*) noexcept { // exposition only
 terminate();
 }
coroutine_handle<> continuation{}; // exposition only
coroutine_handle<> (*stopped-handler)(void*) noexcept = // exposition only
 &default-unhandled-stopped;
};
}

```

```

template<class OtherPromise>
 requires (!same_as<OtherPromise, void>)
void set_continuation(coroutine_handle<OtherPromise> h) noexcept;

```

2 *Effects*: Equivalent to:

```

 continuation = h;
if constexpr (requires(OtherPromise& other) { other.unhandled_stopped(); }) {
 stopped-handler = [](void* p) noexcept -> coroutine_handle<> {
 return coroutine_handle<OtherPromise>::from_address(p)
 .promise().unhandled_stopped();
 };
} else {
 stopped-handler = &default-unhandled-stopped;
}

```

```

template<class Value>
call-result-t<as_awaitable_t, Value, Promise&> await_transform(Value&& value);

```

3 *Effects*: Equivalent to:

```

 return as_awaitable(std::forward<Value>(value), static_cast<Promise&>(*this));

```

### 33.13.3 execution::affine

[exec.affine]

- 1 **affine** adapts a sender into one that completes on the receiver's scheduler. If the algorithm determines that the adapted sender already completes on the correct scheduler it can avoid any scheduling operation.
- 2 The name **affine** denotes a pipeable sender adaptor object. For a subexpression **sndr**, if `decltype((sndr))` does not satisfy **sender**, **affine(sndr)** is ill-formed.
- 3 Otherwise, the expression **affine(sndr)** is expression-equivalent to *make-sender*(**affine**, *env*<>(), **sndr**).
- 4 For a subexpression **sch** whose type models **scheduler**, let *UNSTOPPABLE-SCHEDULER*(**sch**) be an expression **e** whose type models **scheduler** such that:
  - (4.1) — *schedule*(**e**) is expression-equivalent to *unstoppable*(*schedule*(**sch**)).
  - (4.2) — For any query object **q** and pack of subexpressions **args...**, **e.query(q, args...)** is expression-equivalent to **sch.query(q, args...)**.
  - (4.3) — The expression **e == UNSTOPPABLE-SCHEDULER(other)** is expression-equivalent to **sch == other**.
- 5 Let **sndr** and **ev** be subexpressions such that **Sndr** is `decltype((sndr))`. If *sender-for*<**Sndr**, **affine\_t**> is **false**, then the expression **affine.transform\_sender(sndr, ev)** is ill-formed; otherwise, it is equal to:

```

auto& [_, _, child] = sndr;

```

```

if constexpr (requires { std::forward_like<Sndr>(child).affine(); }) {
 return std::forward_like<Sndr>(child).affine();
} else {
 return continues_on(std::forward_like<Sndr>(child),
 UNSTOPPABLE-SCHEDULER(get_start_scheduler(ev)));
}

```

- <sup>6</sup> *Recommended practice:* Implementations should provide `affine` member functions for senders that are known to resume on the scheduler where they were started. Example senders for which that is the case are `just`, `just_error`, `just_stopped`, `read_env`, and `write_env`.
- <sup>7</sup> Let `out_sndr` be a subexpression denoting a sender returned from `affine(sndr)` or one equal to such, and let `OutSndr` be the type `decltype((out_sndr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env`. If `get_start_scheduler(get_env(out_rcvr))` is ill-formed or does not satisfy *infallible-scheduler*<`Env`>, then evaluation of the expression `get_completion_signatures<OutSndr, Env>()` exits with an exception. Let `op` be an lvalue referring to the operation state that results from connecting `out_sndr` to `out_rcvr`. Calling `start(op)` will start `sndr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If the current execution resource is the same as the execution resource associated with `sch`, the completion operation on `out_rcvr` may be called before `start(op)` completes.

### 33.13.4 `execution::inline_scheduler`

[exec.inline.scheduler]

```

namespace std::execution {
 class inline_scheduler {
 class inline-sender; // exposition only

 template<receiver R>
 class inline-state; // exposition only

 public:
 using scheduler_concept = scheduler_tag;

 constexpr inline-sender schedule() noexcept { return {}; }
 constexpr bool operator==(const inline_scheduler&) const noexcept = default;
 };
}

```

- <sup>1</sup> `inline_scheduler` is a class that models `scheduler` (33.6). All objects of type `inline_scheduler` are equal. For a subexpression `sch` of type `inline_scheduler`, a query object `q`, and a pack of subexpressions `args`, the expression `sch.query(q, args...)` is expression-equivalent to `inline-attrs<set_value_t>().query(q, args...)`.
- <sup>2</sup> `inline-sender` is an exposition-only type that satisfies `sender`. The type `completion_signatures_of_t<inline-sender>` is `completion_signatures<set_value_t>`.
- <sup>3</sup> Let `sndr` be an expression of type `inline-sender`, let `rcvr` be an expression such that *receiver-of*<`decltype((rcvr))`, `CS`> is true where `CS` is `completion_signatures<set_value_t>`, then the expression `connect(sndr, rcvr)` has type `inline-state<remove_cvref_t<decltype((rcvr))>>` and is potentially-throwing if and only if `((void)sndr, auto(rcvr))` is potentially-throwing.
- <sup>4</sup> Let `o` be a non-const lvalue of type `inline-state<Rcvr>`, and let `REC(o)` be a non-const lvalue reference to an object of type `Rcvr` that was initialized with the expression `rcvr` passed to an invocation of `connect` that returned `o`, then:
- (4.1) — the object to which `REC(o)` refers remains valid for the lifetime of the object to which `o` refers, and
- (4.2) — the expression `start(o)` is equivalent to `set_value(std::move(REC(o)))`.

### 33.13.5 `execution::task_scheduler`

[exec.task.scheduler]

```

namespace std::execution {
 class task_scheduler {
 class ts-domain; // exposition only

 template<scheduler Sch>
 class backend-for; // exposition only
 };
}

```

```

public:
 using scheduler_concept = scheduler_tag;

 template<class Sch, class Allocator = allocator<void>>
 requires (!same_as<task_scheduler, remove_cvref_t<Sch>>) && scheduler<Sch>
 explicit task_scheduler(Sch&& sch, Allocator alloc = {});

 task_scheduler(const task_scheduler&) = default;
 task_scheduler& operator=(const task_scheduler&) = default;

 see below schedule();

 friend bool operator==(const task_scheduler& lhs, const task_scheduler& rhs) noexcept;
 template<class Sch>
 requires (!same_as<task_scheduler, Sch>) && scheduler<Sch>
 friend bool operator==(const task_scheduler& lhs, const Sch& rhs) noexcept;

private:
 // see 33.16.4
 shared_ptr<parallel_scheduler_replacement::parallel_scheduler_backend> sch_; // exposition only
};

```

- <sup>1</sup> `task_scheduler` is a class that models `scheduler` (33.6). Given an object `s` of type `task_scheduler`, let  $SCHED(s)$  be the object pointed to by the pointer owned by `s.sch_`. The expression `get_forward_progress_guarantee(s)` is equivalent to `get_forward_progress_guarantee(SCHED(s))`. The expression `get_completion_domain<set_value_t>(s)` is equivalent to `task_scheduler::ts-domain()`.

```

template<class Sch, class Allocator = allocator<void>>
 requires(!same_as<task_scheduler, remove_cvref_t<Sch>>) && scheduler<Sch>
explicit task_scheduler(Sch&& sch, Allocator alloc = {});

```

- <sup>2</sup> *Mandates:* `Sch` satisfies *infallible-scheduler*<env><>>.

- <sup>3</sup> *Effects:* Initializes `sch_` with:

```
allocate_shared<backend-for<remove_cvref_t<Sch>>>(alloc, std::forward<Sch>(sch))
```

- <sup>4</sup> *Recommended practice:* Implementations should avoid the use of dynamically allocated memory for small scheduler objects.

- <sup>5</sup> *Remarks:* Any allocations performed by calls on `*this` are performed using a copy of `alloc`.

```
bool operator==(const task_scheduler& lhs, const task_scheduler& rhs) noexcept;
```

- <sup>6</sup> *Effects:* Equivalent to: `return lhs == SCHED(rhs);`

```

template<class Sch>
 requires (!same_as<task_scheduler, Sch>)
 && scheduler<Sch>
bool operator==(const task_scheduler& lhs, const Sch& rhs) noexcept;

```

- <sup>7</sup> *Returns:* `false` if the type of  $SCHED(lhs)$  is not `Sch`, otherwise  $SCHED(lhs) == rhs$ .

- <sup>8</sup> For an lvalue `r` of a type derived from `receiver_proxy`, let *WRAP-RCVR*(`r`) be an object of a type that models receiver and whose completion handlers result in invoking the corresponding completion handlers of `r`.

```

namespace std::execution {
 template<scheduler Sch>
 class task_scheduler::backend-for
 : public parallel_scheduler_replacement::parallel_scheduler_backend { // exposition only
 public:
 explicit backend-for(Sch sch) : sched_(std::move(sch)) {}

 void schedule(receiver_proxy& r, span<byte> s) noexcept override;
 void schedule_bulk_chunked(size_t shape, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept override;
 void schedule_bulk_unchunked(size_t shape, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept override;

```

```

private:
 Sch sched_;
};
}

```

- <sup>9</sup> Let *env* be a pack of subexpressions, and let *just-sndr-like* be a sender whose only value completion signature is `set_value_t()` and for which the expression `get_completion_scheduler<set_value_t>(get_env(just-sndr-like), env...)` is expression-equivalent to `get_completion_scheduler<set_value_t>(sched_, env...)`.

```
void schedule(receiver_proxy& r, span<byte> s) noexcept override;
```

- <sup>10</sup> *Effects:* Constructs an operation state *os* with `connect(schedule(sched_), WRAP-RCVR(r))` and calls `start(os)`.

```
void schedule_bulk_chunked(size_t shape, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept override;
```

- <sup>11</sup> *Effects:* Let *chunk\_size* be an integer less than or equal to *shape*, let *num\_chunks* be  $(\text{shape} + \text{chunk\_size} - 1) / \text{chunk\_size}$ , and let *fn* be a function object such that for an integer *i*, *fn*(*i*) calls `r.execute(i * chunk_size, m)`, where *m* is the lesser of  $(i + 1) * \text{chunk\_size}$  and *shape*. Constructs an operation state *os* as if with

```
connect(bulk(just-sndr-like, par, num_chunks, fn), WRAP-RCVR(r))
```

and calls `start(os)`.

```
void schedule_bulk_unchunked(size_t shape, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept override;
```

- <sup>12</sup> *Effects:* Let *fn* be a function object such that for an integer *i*, *fn*(*i*) is equivalent to `r.execute(i, i + 1)`. Constructs an operation state *os* as if with

```
connect(bulk(just-sndr-like, par, shape, fn), WRAP-RCVR(r))
```

and calls `start(os)`.

see below `schedule()`;

- <sup>13</sup> *Returns:* A prvalue *ts-sndr* whose type models sender such that:

- (13.1) — `get_completion_scheduler<set_value_t>(get_env(ts-sndr))` is equal to `*this`.
- (13.2) — `get_completion_domain<set_value_t>(get_env(ts-sndr))` is expression-equivalent to `ts-domain()`.
- (13.3) — If a receiver *rcvr* is connected to *ts-sndr* and the resulting operation state is started, calls `sch->schedule(r, s)`, where
  - (13.3.1) — *r* is a proxy for *rcvr* with base `parallel_scheduler_replacement::receiver_proxy` (33.15) and
  - (13.3.2) — *s* is a preallocated backend storage for *r*.
- (13.4) — For any type *E*, `completion_signatures_of_t<decltype(ts-sndr), E>` denotes `completion_signatures<set_value_t>` if `unstoppable_token<stop_token_of_t<E>>` is true, and otherwise `completion_signatures<set_value_t(), set_stopped_t>`.

```

namespace std::execution {
 class task_scheduler::ts-domain : public default_domain { // exposition only
 public:
 template<class BulkSndr, class Env>
 static constexpr auto transform_sender(set_value_t, BulkSndr&& bulk_sndr, const Env& env)
 noexcept(see below);
 };
}

```

```

template<class BulkSndr, class Env>
 static constexpr auto transform_sender(BulkSndr&& bulk_sndr, const Env& env)
 noexcept(is_nothrow_constructible_v<decay_t<BulkSndr>, BulkSndr>);

```

- <sup>14</sup> *Constraints:*

- (14.1) — `sender_in<BulkSndr, Env>` is true,
- (14.2) — `auto(std::forward<BulkSndr>(bulk_sndr))` is well-formed, and
- (14.3) — either
  - (14.3.1) — `sender_for<BulkSndr, bulk_chunked_t>` or
  - (14.3.2) — `sender_for<BulkSndr, bulk_unchunked_t>`
 is true.

15 *Effects:* Equivalent to:

```
auto& [_, data, child] = bulk_sndr;
auto& [_, shape, fn] = data;
auto sch = call-with-default(get_completion_scheduler<set_value_t>,
 not-a-scheduler(), get_env(child), FWD-ENV(env));
return e;
```

where *e* is *not-a-sender*() if the type of *sch* is not `task_scheduler`; otherwise, it is a prvalue whose type models `sender` such that, if it is connected to a receiver *rcvr* and the resulting operation state is started, *child* is connected to an unspecified receiver *R* and started. The expression `get_env(R)` is expression-equivalent to `FWD-ENV(get_env(rcvr-copy))`, where *rcvr-copy* is an lvalue subexpression designating an object decay-copied from *rcvr*.

If *child* completes with an error or a stopped completion, the completion operation is forwarded unchanged to *rcvr*. Otherwise, let *args* be a pack of lvalue subexpressions designating objects decay-copied from the value result datums. Then:

- (15.1) — If *bulk\_sndr* was the result of the evaluation of an expression equivalent to `bulk_chunked(child, policy, shape, fn)` or a copy of such, then `sch->schedule_bulk_chunked(shape, r, s)` is called where *r* is a bulk chunked proxy (33.15) for *rcvr* with callable *fn* and arguments *args*, and *s* is a preallocated backend storage for *r*.
- (15.2) — Otherwise, calls `sch->schedule_bulk_unchunked(shape, r, s)` where *r* is a bulk unchunked proxy for *rcvr* with callable *fn* and arguments *args*, and *s* is a preallocated backend storage for *r*.

### 33.13.6 execution::task

[exec.task]

#### 33.13.6.1 task overview

[task.overview]

- <sup>1</sup> The `task` class template represents a sender that can be used as the return type of coroutines. The first template parameter *T* defines the type of the value completion datum (33.3) if *T* is not `void`. Otherwise, there are no value completion datums. Inside coroutines returning `task<T, E>` the operand of `co_return` (if any) becomes the argument of `set_value`. The second template parameter *Environment* is used to customize the behavior of `task`.

#### 33.13.6.2 Class template task

[task.class]

```
namespace std::execution {
 template<class T = void, class Environment = env<>>
 class task {
 // 33.13.6.4
 template<receiver Rcvr>
 class state; // exposition only

 public:
 using sender_concept = sender_tag;
 using allocator_type = see below;
 using start_scheduler_type = see below;
 using stop_source_type = see below;
 using stop_token_type = decltype(declval<stop_source_type>().get_token());
 using error_types = see below;

 // 33.13.6.5
 class promise_type;
```

```

 task(task&&) noexcept;
 ~task();

 template<class Self, class... Env>
 static consteval auto get_completion_signatures();

 template<receiver Rcvr>
 state<Rcvr> connect(Rcvr&& rcvr) &&;

private:
 coroutine_handle<promise_type> handle; // exposition only
};

```

- 1 **task**<T, E> models sender (33.9) if T is void, a reference type, or a *cv*-unqualified non-array object type and E is a class type. Otherwise a program that instantiates the definition of **task**<T, E> is ill-formed.
- 2 The nested types of **task** template specializations are determined based on the **Environment** parameter:
  - (2.1) — **allocator\_type** is **Environment::allocator\_type** if that *qualified-id* is valid and denotes a type, **allocator<byte>** otherwise.
  - (2.2) — **start\_scheduler\_type** is **Environment::start\_scheduler\_type** if that *qualified-id* is valid and denotes a type, **task\_scheduler** otherwise.
  - (2.3) — **stop\_source\_type** is **Environment::stop\_source\_type** if that *qualified-id* is valid and denotes a type, **inplace\_stop\_source** otherwise.
  - (2.4) — **error\_types** is **Environment::error\_types** if that *qualified-id* is valid and denotes a type, **completion\_signatures<set\_error\_t(exception\_ptr)>** otherwise.
- 3 A program is ill-formed if **error\_types** is not a specialization of **execution::completion\_signatures** or if the template arguments of that specialization contain an element which is not of the form **set\_error\_t(E)** for some type E.
- 4 **allocator\_type** shall meet the *Cpp17Allocator* requirements, **scheduler\_type** shall model **scheduler**, and **stop\_source\_type** shall model **stoppable-source**.

### 33.13.6.3 task members

[task.members]

```
task(task&& other) noexcept;
```

- 1 *Effects:* Initializes *handle* with **exchange**(other.*handle*, {}).  

```
~task();
```

- 2 *Effects:* Equivalent to:

```
 if (handle)
 handle.destroy();
```

```

template<class Self, class... Env>
 static consteval auto get_completion_signatures();

```

- 3 Let the type C be a specialization of **execution::completion\_signatures** with the template arguments (in unspecified order):
  - (3.1) — **set\_value\_t()** if T is void, and **set\_value\_t(T)** otherwise;
  - (3.2) — template arguments of the specialization of **execution::completion\_signatures** denoted by **error\_types**; and
  - (3.3) — **set\_stopped\_t()**.
- 4 *Returns:* C().

```

template<receiver Rcvr>
 state<Rcvr> connect(Rcvr&& rcvr) &&;

```

- 5 *Mandates:* At least one of the expressions **allocator\_type**(**get\_allocator**(**get\_env**(rcvr))) and **allocator\_type**() is well-formed.
- 6 *Preconditions:* **bool**(*handle*) is true.

7 *Effects:* Equivalent to:

```
return state<Rcvr>(exchange(handle, {}), std::forward<Rcvr>(recv));
```

### 33.13.6.4 Class template `task::state`

[task.state]

```
namespace std::execution {
 template<class T, class Environment>
 template<receiver Rcvr>
 class task<T, Environment>::state { // exposition only
 public:
 using operation_state_concept = operation_state_tag;

 template<class R>
 state(coroutine_handle<promise_type> h, R&& rr);

 ~state();

 void start() & noexcept;

 stop_token_type get-stop-token(); // exposition only

 private:
 using own-env-t = see below; // exposition only
 coroutine_handle<promise_type> handle; // exposition only
 remove_cvref_t<Rcvr> rcvr; // exposition only
 optional<stop_source_type> source; // exposition only
 own-env-t own-env; // exposition only
 Environment environment; // exposition only
 optional<T> result; // exposition only; present only if is_void_v<T> is false
 exception_ptr error; // exposition only
 };
}
```

1 The type *own-env-t* is `Environment::template env_type<decltype(get_env(declval<Rcvr>()))>` if that *qualified-id* is valid and denotes a type, `env<>` otherwise.

```
template<class R>
 state(coroutine_handle<promise_type> h, R&& rr);
```

2 *Effects:* Initializes

- (2.1) — *handle* with `std::move(h)`;
- (2.2) — *rcvr* with `std::forward<R>(rr)`;
- (2.3) — *own-env* with *own-env-t* (`get_env(rcvr)`) if that expression is valid and *own-env-t* () otherwise. If neither of these expressions is valid, the program is ill-formed.
- (2.4) — *environment* with `Environment(own-env)` if that expression is valid, otherwise `Environment(get_env(rcvr))` if this expression is valid, otherwise `Environment()`. If neither of these expressions is valid, the program is ill-formed.

```
~state();
```

3 *Effects:* Equivalent to:

```
if (handle)
 handle.destroy();
```

```
void start() & noexcept;
```

4 *Effects:* Let *prom* be the object `handle.promise()`. Associates `STATE(prom)`, `RCVR(prom)`, and `SCHED(prom)` with `*this` as follows:

- (4.1) — `STATE(prom)` is `*this`.
- (4.2) — `RCVR(prom)` is *rcvr*.
- (4.3) — `SCHED(prom)` is the object initialized with `start_scheduler_type(get_start_scheduler(get_env(rcvr)))` if that expression is valid and `start_scheduler_type()` otherwise. If neither of these expressions is valid, the program is ill-formed.



Finally, invokes *handle.resume()*.

```
stop_token_type get-stop-token(); // exposition only
```

5 *Effects:* If `same_as<decltype(declval<stop_source_type>().get_token()), decltype(get_stop_token(get_env(rcvr)))>` is true, returns `get_stop_token(get_env(rcvr))`. Otherwise, if `source.has_value()` is false, initializes the contained value of *source* such that

(5.1) — *source*→`stop_requested()` returns `get_stop_token(get_env(rcvr))→stop_requested()`; and

(5.2) — *source*→`stop_possible()` returns `get_stop_token(get_env(rcvr))→stop_possible()`.

Finally, returns *source*→`get_token()`.

### 33.13.6.5 Class `task::promise_type`

[task.promise]

```
namespace std::execution {
 template<class T, class Environment>
 class task<T, Environment>::promise_type {
 public:
 task get_return_object() noexcept;

 static constexpr suspend_always initial_suspend() noexcept { return {}; }
 auto final_suspend() noexcept;

 void unhandled_exception();
 coroutine_handle<> unhandled_stopped() noexcept;

 void return_void(); // present only if is_void_v<T> is true
 template<class V = T>
 void return_value(V&& value); // present only if is_void_v<T> is false

 template<class E>
 unspecified yield_value(with_error<E> error);

 template<sender Sender>
 auto await_transform(Sender&& sndr);

 unspecified get_env() const noexcept;

 void* operator new(size_t size);
 template<class Alloc, class... Args>
 void* operator new(size_t size, allocator_arg_t, Alloc alloc, Args&&...);
 template<class This, class Alloc, class... Args>
 void* operator new(size_t size, const This&, allocator_arg_t, Alloc alloc, Args&&...);

 void operator delete(void* pointer, size_t size) noexcept;
 };
}
```

1 Let *prom* be an object of `promise_type` and let *tsk* be the task object created by *prom*.`get_return_object()`. The description below refers to objects *STATE*(*prom*), *RCVR*(*prom*), and *SCHED*(*prom*) associated with *tsk* during evaluation of `task::state<Rcvr>::start` for some receiver *Rcvr*.

```
task get_return_object() noexcept;
```

2 *Returns:* A task object whose member *handle* is `coroutine_handle<promise_type>::from_promise(*this)`.

```
auto final_suspend() noexcept;
```

3 *Returns:* An awaitable object of unspecified type (7.6.2.4) whose member functions arrange for the completion of the asynchronous operation associated with *STATE*(*\*this*). Let *st* be a reference to *STATE*(*\*this*). The asynchronous completion first destroys the coroutine frame using *st.handle.destroy()* and then invokes:

(3.1) — `set_error(std::move(st.rcvr), std::move(st.error))` if `bool(st.error)` is true, otherwise

- (3.2) — `set_value(std::move(st.rcvr))` if `is_void_v<T>` is true, and otherwise
- (3.3) — `set_value(std::move(st.rcvr), *std::move(st.result))`.

```
template<class Err>
 auto yield_value(with_error<Err> err);
```

4 *Mandates:* `std::move(err.error)` is convertible to exactly one of the `set_error_t` argument types of `error_types`. Let *Cerr* be that type.

5 *Returns:* An awaitable object of unspecified type (7.6.2.4) whose member functions arrange for the calling coroutine to be suspended and then completes the asynchronous operation associated with `STATE(*this)`. Let *st* be a reference to `STATE(*this)`. Then the asynchronous operation completes by first destroying the coroutine frame using `st.handle.destroy()` and then invoking `set_error(std::move(st.rcvr), Cerr(std::move(err.error)))`.

```
template<sender Sender>
 auto await_transform(Sender&& sndr);
```

6 *Returns:* If `same_as<inline_scheduler, start_scheduler_type>` is true, returns `as_awaitable(std::forward<Sender>(sndr), *this)`; otherwise returns `as_awaitable(affine(std::forward<Sender>(sndr)), *this)`.

```
void unhandled_exception();
```

7 *Effects:* If the signature `set_error_t(exception_ptr)` is not an element of `error_types`, calls `terminate()` (14.6.2). Otherwise, stores `current_exception()` into `STATE(*this).error`.

```
coroutine_handle<> unhandled_stopped() noexcept;
```

8 *Effects:* Completes the asynchronous operation associated with `STATE(*this)`. Let *st* be a reference to `STATE(*this)`. The asynchronous operation is completed by first destroying the coroutine frame using `st.handle.destroy()` and then invoking `set_stopped(std::move(st.rcvr))`.

9 *Returns:* `noop_coroutine()`.

```
void return_void();
```

10 *Effects:* Does nothing.

```
template<class V>
 void return_value(V&& v);
```

11 *Effects:* Equivalent to `result.emplace(std::forward<V>(v))`.

```
unspecified get_env() const noexcept;
```

12 *Returns:* An object *env* such that queries are forwarded as follows:

- (12.1) — `env.query(get_start_scheduler)` returns `start_scheduler_type(SCHED(*this))`.
- (12.2) — `env.query(get_allocator)` returns `allocator_type(get_allocator(get_env(RCVR(*this)))` if this expression is well-formed and `allocator_type()` otherwise.
- (12.3) — `env.query(get_stop_token)` returns `STATE(*this).get-stop-token()`.
- (12.4) — For any other query *q* and arguments *a...* a call to `env.query(q, a...)` returns `STATE(*this).environment.query(q, a...)` if this expression is well-formed and `forwarding_query(q)` is well-formed and is true. Otherwise `env.query(q, a...)` is ill-formed.

```
void* operator new(size_t size);
```

13 *Effects:* Equivalent to: `return operator new(size, allocator_arg, allocator_type());`

```
template<class Alloc, class... Args>
 void* operator new(size_t size, allocator_arg_t, Alloc alloc, Args&&...);
template<class This, class Alloc, class... Args>
 void* operator new(size_t size, const This&, allocator_arg_t, Alloc alloc, Args&&...);
```

14 Let *PAlloc* be `allocator_traits<Alloc>::template rebind_alloc<U>`, where *U* is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

15 *Mandates:* `allocator_traits<PAlloc>::pointer` is a pointer type.

16 *Effects:* Initializes an allocator `palloc` of type `PAlloc` with `alloc`. Uses `palloc` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `palloc`.

17 *Returns:* A pointer to the allocated storage.

```
void operator delete(void* pointer, size_t size) noexcept;
```

18 *Preconditions:* `pointer` was returned from an invocation of the above overload of `operator new` with a size argument equal to `size`.

19 *Effects:* Deallocates the storage pointed to by `pointer` using an allocator equal to that used to allocate it.

### 33.14 Execution scope utilities

[exec.scope]

#### 33.14.1 Execution scope concepts

[exec.scope.concepts]

- 1 The `scope_association` concept defines the requirements on a type `Assoc`. An object of type `Assoc` is *engaged* if and only if it owns an association with an async scope, referred to as its *associated scope*.

```
namespace std::execution {
 template<class Assoc>
 concept scope_association =
 movable<Assoc> &&
 is_nothrow_move_constructible_v<Assoc> &&
 is_nothrow_move_assignable_v<Assoc> &&
 default_initializable<Assoc> &&
 requires(const Assoc assoc) {
 { static_cast<bool>(assoc) } noexcept;
 { assoc.try_associate() } -> same_as<Assoc>;
 };
}
```

- 2 A type `Assoc` models `scope_association` only if:

- (2.1) — a default constructed object of type `Assoc` is not engaged;
- (2.2) — for an object `assoc` of type `Assoc`, `static_cast<bool>(assoc)` is `true` if and only if `assoc` is engaged;
- (2.3) — no two distinct objects of type `Assoc` own the same association;
- (2.4) — for an object `assoc` of type `Assoc`, destroying `assoc` releases the association owned by `assoc`, if any;
- (2.5) — for an object `assoc` of type `Assoc`, after it is used as the source operand of a move constructor, the `assoc` is not engaged;
- (2.6) — for distinct objects `assoc1` and `assoc2` of type `Assoc`, after evaluating `assoc1 = std::move(assoc2)`, the association owned by `assoc1`, if any, is released and `assoc2` is not engaged;
- (2.7) — for an object `assoc` of type `Assoc` that is engaged, `assoc.try_associate()` either returns an object that is not engaged or acquires a new association with `assoc`'s associated scope and returns an engaged object that owns that association;
- (2.8) — for an object `assoc` of type `Assoc` that is not engaged, `assoc.try_associate()` returns an object that is not engaged.

- 3 The `scope_token` concept defines the requirements on a type `Token` that can be used to create associations between senders and an async scope. Every object of type `Token` is associated with an async scope that is referred to as its *associated scope*.

- 4 Let `test-sender` and `test-env` be unspecified types such that `sender_in<test-sender, test-env>` is modeled.

```
namespace std::execution {
 template<class Token>
 concept scope_token =
 copyable<Token> &&
 requires(const Token token) {
 { token.try_associate() } -> scope_association;
 { token.wrap(declval<test-sender>()) } -> sender_in<test-env>;
 };
}
```

```
 };
}
```

<sup>5</sup> A type `Token` models `scope_token` only if:

- (5.1) — no exceptions are thrown from copy construction, move construction, copy assignment, or move assignment of objects of type `Token`;
- (5.2) — for an object `token` of type `Token`, `token.try_associate()` either returns an object that is not engaged or acquires a new association with `token`'s associated scope and returns an engaged object that owns that association; and
- (5.3) — given an lvalue `token` of type (possibly `const`) `Token`, for all expressions `sndr` such that `decltype((sndr))` models `sender`:
  - (5.3.1) — `token.wrap(sndr)` is a valid expression,
  - (5.3.2) — `decltype(token.wrap(sndr))` models `sender`, and
  - (5.3.3) — `completion_signatures_of_t<decltype(token.wrap(sndr)), E>` contains the same completion signatures as `completion_signatures_of_t<decltype((sndr)), E>` for all types `E` such that `sender_in<decltype((sndr)), E>` is modeled.

### 33.14.2 Counting Scopes

[exec.counting.scopes]

#### 33.14.2.1 General

[exec.counting.scopes.general]

<sup>1</sup> Scopes of type `simple_counting_scope` and `counting_scope` maintain counts of associations. Let:

- (1.1) — `Scope` be either `simple_counting_scope` or `counting_scope`,
- (1.2) — `scope` be an object of type `Scope`,
- (1.3) — `tkn` be an object of type `Scope::token` obtained from `scope.get_token()`,
- (1.4) — `jsndr` be a sender obtained from `scope.join()`, and
- (1.5) — `op` be an operation state obtained from connecting `jsndr` to a receiver.

During its lifetime `scope` goes through different states which govern what operations are allowed and the result of these operations:

- (1.6) — *unused*: a newly constructed object starts in the *unused* state.
- (1.7) — *open*: when `tkn.try_associate()` is called while `scope` is in the *unused* state, `scope` moves to the *open* state.
- (1.8) — *open-and-joining*: when the operation state `op` is started while `scope` is in the *unused* or *open* state, `scope` moves to the *open-and-joining* state.
- (1.9) — *closed*: when `scope.close()` is called while `scope` is in the *open* state, `scope` moves to the *closed* state.
- (1.10) — *unused-and-closed*: when `scope.close()` is called while `scope` is in the *unused* state, `scope` moves to the *unused-and-closed* state.
- (1.11) — *closed-and-joining*: when `scope.close()` is called while `scope` is in the *open-and-joining* state or the operation state `op` is started while `scope` is in the *closed* or *unused-and-closed* state, `scope` moves to the *closed-and-joining* state.
- (1.12) — *joined*: when the count of associations drops to zero while `scope` is in the *open-and-joining* or *closed-and-joining* state, `scope` moves to the *joined* state.

<sup>2</sup> *Recommended practice*: For `simple_counting_scope` and `counting_scope`, implementations should store the state and the count of associations in a single member of type `size_t`.

<sup>3</sup> Subclause 33.14.2 makes use of the following exposition-only entities:

```
struct scope-join-t {}; // exposition only

enum scope-state-type { // exposition only
 unused, // exposition only
 open, // exposition only
 closed, // exposition only
 open-and-joining, // exposition only
 closed-and-joining, // exposition only
};
```

```

 unused-and-closed, // exposition only
 joined, // exposition only
};

```

```

template<class Scope>
 struct association-t; // exposition only

```

- 4 The exposition-only class template *impls-for* (33.9.2) is specialized for *scope-join-t* as follows:

```

namespace std::execution {
 template<>
 struct impls-for<scope-join-t> : default-impls {
 template<class Scope, class Rcvr>
 struct state { // exposition only
 struct rcvr-t { // exposition only
 using receiver_concept = receiver_tag;

 Rcvr& rcvr; // exposition only

 void set_value() && noexcept {
 execution::set_value(std::move(rcvr));
 }

 template<class E>
 void set_error(E&& e) && noexcept {
 execution::set_error(std::move(rcvr), std::forward<E>(e));
 }

 void set_stopped() && noexcept {
 execution::set_stopped(std::move(rcvr));
 }

 decltype(auto) get_env() const noexcept {
 return execution::get_env(rcvr);
 }
 };

 using sched-sender = // exposition only
 decltype(schedule(get_start_scheduler(get_env(declval<Rcvr&>()))));
 using op-t = // exposition only
 connect_result_t<sched-sender, rcvr-t>;

 Scope* scope; // exposition only
 Rcvr& receiver; // exposition only
 op-t op; // exposition only

 state(Scope* scope, Rcvr& rcvr) // exposition only
 noexcept(nothrow-callable<connect_t, sched-sender, rcvr-t>)
 : scope(scope),
 receiver(rcvr),
 op(connect(schedule(get_start_scheduler(get_env(rcvr))), rcvr-t(rcvr))) {}

 void complete() noexcept { // exposition only
 start(op);
 }

 void complete-inline() noexcept { // exposition only
 set_value(std::move(receiver));
 }
 };
 };

 static constexpr auto get-state = // exposition only
 [<class Rcvr>(auto&& sender, Rcvr& receiver)
 noexcept(is_nothrow_constructible_v<state<Rcvr>, data-type<decltype(sender)>, Rcvr&>) {
 auto[, self] = sender;

```

```

 return state(self, receiver);
};

static constexpr auto start = // exposition only
[] (auto& s, auto&) noexcept {
 if (s.scope->start-join-sender(s))
 s.complete-inline();
};
};
}

```

<sup>5</sup> *association-t* is a class template, specializations of which model *scope\_association* and contain an exposition-only member *scope* of type *Scope\**. For a class type *Scope* and an object *assoc* of type *association-t*<*Scope*>:

- (5.1) — *assoc.scope* points to its associated scope,
- (5.2) — *assoc* is engaged when *assoc.scope* != *nullptr* is true,
- (5.3) — if *assoc* is engaged, then *assoc.try\_associate()* is equivalent to *assoc.scope->try\_associate()*, and
- (5.4) — the association owned by *assoc* is released by invoking *assoc.scope->disassociate()*.

### 33.14.2.2 Simple Counting Scope

[exec.scope.simple.counting]

#### 33.14.2.2.1 General

[exec.scope.simple.counting.general]

```

namespace std::execution {
 class simple_counting_scope {
 public:
 // 33.14.2.2.4, token
 struct token;

 using assoc-t = association-t<simple_counting_scope>; // exposition only

 static constexpr size_t max_associations = implementation-defined;

 // 33.14.2.2.2, constructor and destructor
 simple_counting_scope() noexcept;
 simple_counting_scope(simple_counting_scope&&) = delete;
 ~simple_counting_scope();

 // 33.14.2.2.3, members
 token get_token() noexcept;
 void close() noexcept;
 sender auto join() noexcept;

 private:
 size_t count; // exposition only
 scope-state-type state; // exposition only

 assoc-t try_associate() noexcept; // exposition only
 void disassociate() noexcept; // exposition only
 template<class State>
 bool start-join-sender(State& state) noexcept; // exposition only
 };
}

```

<sup>1</sup> For purposes of determining the existence of a data race, *get\_token*, *close*, *join*, *try\_associate*, *disassociate*, and *start-join-sender* behave as atomic operations (6.10.2). These operations on a single object of type *simple\_counting\_scope* appear to occur in a single total order.

#### 33.14.2.2.2 Constructor and Destructor

[exec.simple.counting.ctor]

```
simple_counting_scope() noexcept;
```

<sup>1</sup> *Postconditions:* *count* is 0 and *state* is *unused*.

```
~simple_counting_scope();
```

- 2 *Effects:* If *state* is not one of *joined*, *unused*, or *unused-and-closed*, invokes *terminate* (14.6.2). Otherwise, has no effects.

### 33.14.2.2.3 Members

[exec.simple.counting.mem]

```
token get_token() noexcept;
```

- 1 *Returns:* An object *t* of type `simple_counting_scope::token` such that `t.scope == this` is true.

```
void close() noexcept;
```

- 2 *Effects:* If *state* is

- (2.1) — *unused*, then changes *state* to *unused-and-closed*;
- (2.2) — *open*, then changes *state* to *closed*;
- (2.3) — *open-and-joining*, then changes *state* to *closed-and-joining*;
- (2.4) — otherwise, no effects.

- 3 *Postconditions:* Any subsequent call to *try-associate*() on *\*this* returns false.

```
sender auto join() noexcept;
```

- 4 *Returns:* *make-sender(scope-join-t(), this)*.

```
assoc-t try-associate() noexcept;
```

- 5 *Effects:* If *count* is equal to `max_associations`, then no effects. Otherwise, if *state* is

- (5.1) — *unused*, then increments *count* and changes *state* to *open*;
- (5.2) — *open* or *open-and-joining*, then increments *count*;
- (5.3) — otherwise, no effects.

- 6 *Returns:* If *count* was incremented, an object of type *assoc-t* that is engaged and associated with *\*this*, and *assoc-t*() otherwise.

```
void disassociate() noexcept;
```

- 7 *Preconditions:* *count* is greater than zero.

- 8 *Effects:* Decrements *count*. If *count* is zero after decrementing and *state* is *open-and-joining* or *closed-and-joining*, changes *state* to *joined* and calls *complete*() on all objects registered with *\*this*.

[Note 1: Calling *complete*() on any registered object can cause *\*this* to be destroyed. — end note]

```
template<class State>
```

```
bool start-join-sender(State& st) noexcept;
```

- 9 *Effects:* If *count* is zero, then changes *state* to *joined* and returns true. Otherwise, if *state* is

- (9.1) — *open* or *open-and-joining*, then changes *state* to *open-and-joining*, registers *st* with *\*this* and returns false;
- (9.2) — *closed* or *closed-and-joining*, then changes *state* to *closed-and-joining*, registers *st* with *\*this* and returns false.

### 33.14.2.2.4 Token

[exec.simple.counting.token]

```
namespace std::execution {
 struct simple_counting_scope::token {
 template<sender Sender>
 Sender&& wrap(Sender&& snd) const noexcept;
 assoc-t try_associate() const noexcept;

 private:
 simple_counting_scope* scope; // exposition only
 };
}
```

```
template<sender Sender>
 Sender&& wrap(Sender&& snd) const noexcept;
```

1     *Returns:* `std::forward<Sender>(snd)`.

```
assoc-t try_associate() const noexcept;
```

2     *Effects:* Equivalent to: `return scope->try_associate()`;

### 33.14.2.3 Counting Scope

[exec.scope.counting]

```
namespace std::execution {
 class counting_scope {
 public:
 using assoc-t = association-t<counting_scope>; // exposition only

 struct token {
 template<sender Sender>
 sender auto wrap(Sender&& snd) const noexcept(see below);
 assoc-t try_associate() const noexcept;

 private:
 counting_scope* scope; // exposition only
 };

 static constexpr size_t max_associations = implementation-defined;

 counting_scope() noexcept;
 counting_scope(counting_scope&&) = delete;
 ~counting_scope();

 token get_token() noexcept;
 void close() noexcept;
 sender auto join() noexcept;
 void request_stop() noexcept;

 private:
 size_t count; // exposition only
 scope-state-type state; // exposition only
 inplace_stop_source s_source; // exposition only

 assoc-t try_associate() noexcept; // exposition only
 void disassociate() noexcept; // exposition only

 template<class State>
 bool start-join-sender(State& state) noexcept; // exposition only
 };
}
```

1     `counting_scope` differs from `simple_counting_scope` by adding support for cancellation. Unless specified below, the semantics of members of `counting_scope` are the same as the corresponding members of `simple_counting_scope`.

```
token get_token() noexcept;
```

2     *Returns:* An object `t` of type `counting_scope::token` such that `t.scope == this` is true.

```
void request_stop() noexcept;
```

3     *Effects:* Equivalent to `s_source.request_stop()`.

4     *Remarks:* Calls to `request_stop` do not introduce data races.

```
assoc-t try_associate() noexcept;
```

5     *Effects:* If `count` is equal to `max_associations`, then no effects. Otherwise, if `state` is

(5.1)     — *unused*, then increments `count` and changes `state` to *open*;

(5.2)     — *open* or *open-and-joining*, then increments `count`;



(5.3) — otherwise, no effects.

6 *Returns:* If *count* was incremented, an object of type *assoc-t* that is engaged and associated with *\*this*, and *assoc-t* () otherwise.

```
template<sender Sender>
 sender auto counting_scope::token::wrap(Sender&& snd) const
 noexcept(is_nothrow_constructible_v<remove_cvref_t<Sender>, Sender>);

7 Effects: Equivalent to:
 return stop-when(std::forward<Sender>(snd), scope->s_source.get_token());

 assoc-t counting_scope::token::try_associate() const noexcept;
8 Effects: Equivalent to: return scope->try-associate();
```

### 33.15 Parallel scheduler

[exec.par.scheduler]

```
namespace std::execution {
 class parallel_scheduler {
 unspecified
 };
}
```

1 *parallel\_scheduler* models *scheduler*.

2 Let *sch* be an object of type *parallel\_scheduler*, and let *BACKEND-OF*(*sch*) be *\*ptr*, where *sch* is associated with *ptr*.

3 The expression *get\_forward\_progress\_guarantee*(*sch*) has the value *forward\_progress\_guarantee::parallel*.

4 Let *sch2* be an object of type *parallel\_scheduler*. Two objects *sch* and *sch2* compare equal if and only if *BACKEND-OF*(*sch*) and *BACKEND-OF*(*sch2*) refer to the same object.

5 Let *rcvr* be a receiver. A *proxy for rcvr with base B* is an lvalue *r* of type *B* such that:

(5.1) — *r.set\_value*() has effects equivalent to *set\_value*(std::move(*rcvr*)).

(5.2) — *r.set\_error*(*e*), where *e* is an *exception\_ptr* object, has effects equivalent to *set\_error*(std::move(*rcvr*), std::move(*e*)).

(5.3) — *r.set\_stopped*() has effects equivalent to *set\_stopped*(std::move(*rcvr*)).

6 A *preallocated backend storage for a proxy r* is an object *s* of type *span<byte>* such that the range *s* remains valid and may be overwritten until one of *set\_value*, *set\_error*, or *set\_stopped* is called on *r*.

[Note 1: The storage referenced by *s* can be used as temporary storage for operations launched via calls to *parallel\_scheduler\_backend*. — end note]

7 Let *b* be *BACKEND-OF*(*sch*), let *sndr* be the object returned by *schedule*(*sch*), and let *rcvr* be a receiver. If *rcvr* is connected to *sndr* and the resulting operation state is started, then:

(7.1) — If *sndr* completes successfully, then *b.schedule*(*r*, *s*) is called, where

(7.1.1) — *r* is a proxy for *rcvr* with base *parallel\_scheduler\_replacement::receiver\_proxy* and

(7.1.2) — *s* is a preallocated backend storage for *r*.

(7.2) — All other completion operations are forwarded unchanged.

8 The expression *get\_domain*(*sch*) returns an expression of exposition-only type *parallel-scheduler-domain*, that is equivalent with:

```
struct parallel-scheduler-domain {
 template<sender-for<bulk_chunked_t> Sndr, queryable Env>
 static constexpr decltype(auto)
 transform_sender(set_value_t, Sndr&& sndr, const Env& env) const noexcept {
 return see below;
 }
 template<sender-for<bulk_unchunked_t> Sndr, queryable Env>
 static constexpr decltype(auto)
 transform_sender(set_value_t, Sndr&& sndr, const Env& env) const noexcept {
 return see below;
 }
}
```

```
 }
};
```

- 9 For argument `sndr` of the above `transform_sender`, let `child`, `pol`, `shape` and `f` be defined by the following declarations:

```
 auto& [_ , data, child] = sndr;
 auto& [pol, shape, f] = data;
```

10 Let `p` be

- (10.1) — `true`, if the type of the expression `pol` is `cv parallel_policy` or `cv parallel_unsequenced_policy`;
- (10.2) — implementation-defined, if `pol` is an implementation-defined execution policy;
- (10.3) — `false`, otherwise.

11 The `transform_sender` overload that accepts senders with tag `bulk_chunked_t` returns a sender such that if it is connected to a receiver `rcvr` and the resulting operation state is started, then:

- (11.1) — If `child` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating `vals`, then `b.schedule_bulk_chunked(p ? shape : 1, r, s)` is called, where
  - (11.1.1) — `r` is a proxy for `rcvr` with base `parallel_scheduler_replacement::bulk_item_receiver_proxy` such that `r.execute(i, j)` for indices `i` and `j` has effects equivalent to `f(i, j, args...)` if `p` is true and `f(0, shape, args...)` otherwise; and
  - (11.1.2) — `s` is a preallocated backend storage for `r`.
- (11.2) — All other completion operations are forwarded unchanged.

12 The `transform_sender` overload that accepts senders with tag `bulk_unchunked_t` returns a sender such that if it is connected to a receiver `rcvr` and the resulting operation state is started, then:

- (12.1) — If `child` completes with values `vals`, let `args` be a pack of lvalue subexpressions designating `vals`, then `b.schedule_bulk_unchunked(p ? shape : 1, r, s)` is called, where
  - (12.1.1) — `r` is a proxy for `rcvr` with base `parallel_scheduler_replacement::bulk_item_receiver_proxy` such that `r.execute(i, i + 1)` for index `i` has effects equivalent to `f(i, args...)` if `p` is true and
 

```
for (decltype(shape) i = 0; i < shape; i++) { f(i, args...); }
```

 otherwise; and
  - (12.1.2) — `s` is a preallocated backend storage for `r`.
- (12.2) — All other completion operations are forwarded unchanged.

```
parallel_scheduler get_parallel_scheduler();
```

- 13 *Effects:* Let `eb` be the result of `parallel_scheduler_replacement::query_parallel_scheduler_backend()`. If `eb == nullptr` is true, calls `terminate` (14.6.2). Otherwise, returns a `parallel_scheduler` object associated with `eb`.

### 33.16 Namespace `parallel_scheduler_replacement`

[exec.parschedrepl]

#### 33.16.1 General

[exec.parschedrepl.general]

- 1 Facilities in the `parallel_scheduler_replacement` namespace allow users to replace the default implementation of `parallel_scheduler`.

#### 33.16.2 Receiver proxies

[exec.parschedrepl.recvproxy]

```
namespace std::execution::parallel_scheduler_replacement {
 struct receiver_proxy {
 protected:
 virtual bool query_env(unspecified) noexcept = 0; // exposition only

 public:
 virtual void set_value() noexcept = 0;
 virtual void set_error(exception_ptr) noexcept = 0;
 virtual void set_stopped() noexcept = 0;
```

```

 template<class P, class-type Query>
 optional<P> try_query(Query q) const noexcept;
};

struct bulk_item_receiver_proxy : receiver_proxy {
 virtual void execute(size_t, size_t) noexcept = 0;
};
}

```

- <sup>1</sup> **receiver\_proxy** represents a receiver that will be notified by the implementations of **parallel\_scheduler\_backend** to trigger the completion operations. **bulk\_item\_receiver\_proxy** is derived from **receiver\_proxy** and is used for **bulk\_chunked** and **bulk\_unchunked** customizations that will also receive notifications from implementations of **parallel\_scheduler\_backend** corresponding to different iterations.

```

template<class P, class-type Query>
 optional<P> try_query(Query q) const noexcept;

```

- <sup>2</sup> *Mandates:* P is a cv-unqualified non-array object type.

- <sup>3</sup> *Returns:* Let **env** be the environment of the receiver represented by **\*this**. If

- (3.1) — **Query** is not a member of an implementation-defined set of supported queries; or
- (3.2) — P is not a member of an implementation-defined set of supported result types for **Query**; or
- (3.3) — the expression **q(env)** is not well-formed,

then returns **nullopt**. Otherwise, if **q(env)** has type *cv* P, then returns **q(env)**. Otherwise, returns an implementation-defined value of type **optional<P>**.

- <sup>4</sup> *Remarks:* **get\_stop\_token\_t** is in the implementation-defined set of supported queries, and **inplace\_stop\_token** is a member of the implementation-defined set of supported result types for **get\_stop\_token\_t**.

- <sup>5</sup> *Recommended practice:* If P is **inplace\_stop\_token** and T is a type other than **inplace\_stop\_token** that models **stoppable\_token**, **try\_query** should return an object of type **inplace\_stop\_token** such that until one of **set\_value**, **set\_error** or **set\_stopped** is called on **\*this**, all calls to **try\_query** return equivalent **inplace\_stop\_token** objects.

### 33.16.3 query\_parallel\_scheduler\_backend

[exec.parschedrepl.query]

```
shared_ptr<parallel_scheduler_backend> query_parallel_scheduler_backend();
```

- <sup>1</sup> **query\_parallel\_scheduler\_backend()** returns the implementation object for a parallel scheduler.

- <sup>2</sup> *Returns:* An object **p**, such that **p.get()** points to a **parallel\_scheduler\_backend** object that is a base-class subobject of some most derived object **o** within its lifetime. The lifetime of **o** does not end as long as there exists a **shared\_ptr** object **q** within its lifetime such that **q.owner\_equal(p)** is true.

- <sup>3</sup> *Remarks:* This function is replaceable (9.6.5).

### 33.16.4 Class parallel\_scheduler\_backend

[exec.parschedrepl.psb]

```

namespace std::execution::parallel_scheduler_replacement {
 struct parallel_scheduler_backend {
 virtual ~parallel_scheduler_backend() = default;

 virtual void schedule(receiver_proxy&, span<byte>) noexcept = 0;
 virtual void schedule_bulk_chunked(size_t, bulk_item_receiver_proxy&,
 span<byte>) noexcept = 0;
 virtual void schedule_bulk_unchunked(size_t, bulk_item_receiver_proxy&,
 span<byte>) noexcept = 0;
 };
}

```

```
virtual void schedule(receiver_proxy& r, span<byte> s) noexcept = 0;
```

- <sup>1</sup> *Preconditions:* The ends of the lifetime of **\*this**, of the lifetime of the object referred to by **r**, and of the duration of any storage referenced by **s** all happen after the beginning of the evaluation of the call to **set\_value**, **set\_error**, or **set\_stopped** on **r** (see below).

2 *Effects:* A derived class shall implement this function such that:

(2.1) — One of the following expressions is evaluated:

(2.1.1) — `r.set_value()`, if no error occurs, and the work is successful;

(2.1.2) — `r.set_error(eptr)`, if an error occurs, where `eptr` is an object of type `exception_ptr`;

(2.1.3) — `r.set_stopped()`, if the work is canceled.

(2.2) — Any call to `r.set_value()` happens on an execution agent of the execution context represented by `*this`.

3 *Remarks:* The storage referenced by `s` may be used by `*this` as temporary storage for the duration of the operation launched by this call.

```
virtual void schedule_bulk_chunked(size_t n, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept = 0;
```

4 *Preconditions:* The ends of the lifetime of `*this`, of the lifetime of the object referred to by `r`, and of the duration of any storage referenced by `s` all happen after the beginning of the evaluation of the call to `set_value`, `set_error`, or `set_stopped` on `r` (see below).

5 *Effects:* A derived class shall implement this function such that:

(5.1) — Eventually, one of the following expressions is evaluated:

(5.1.1) — `r.set_value()`, if no error occurs, and the work is successful;

(5.1.2) — `r.set_error(eptr)`, if an error occurs, where `eptr` is an object of type `exception_ptr`;

(5.1.3) — `r.set_stopped()`, if the work is canceled.

(5.2) — If `r.execute(b, e)` is called, then `b` and `e` are in the range  $[0, n]$  and  $b < e$ .

(5.3) — For each  $i$  in  $[0, n]$ , there is at most one call to `r.execute(b, e)` such that  $i$  is in the range  $[b, e)$ .

(5.4) — If `r.set_value()` is called, then for each  $i$  in  $[0, n]$ , there is exactly one call to `r.execute(b, e)` such that  $i$  is in the range  $[b, e)$ .

(5.5) — All calls to `execute` on `r` happen before the call to either `set_value`, `set_error`, or `set_stopped` on `r`.

(5.6) — All calls to `execute` and `set_value` on `r` are made on execution agents of the execution context represented by `*this`.

6 *Remarks:* The storage referenced by `s` may be used by `*this` as temporary storage for the duration of the operation launched by this call.

```
virtual void schedule_bulk_unchunked(size_t n, bulk_item_receiver_proxy& r,
 span<byte> s) noexcept = 0;
```

7 *Preconditions:* The ends of the lifetime of `*this`, of the lifetime of the object referred to by `r`, and of the duration of any storage referenced by `s` all happen after the beginning of the evaluation of the call to `set_value`, `set_error`, or `set_stopped` on `r` (see below).

8 *Effects:* A derived class shall implement this function such that:

(8.1) — Eventually, one of the following expressions is evaluated:

(8.1.1) — `r.set_value()`, if no error occurs, and the work is successful;

(8.1.2) — `r.set_error(eptr)`, if an error occurs, where `eptr` is an object of type `exception_ptr`;

(8.1.3) — `r.set_stopped()`, if the work is canceled.

(8.2) — If `r.execute(b, e)` is called, then `b` is in the range  $[0, n)$  and `e` is equal to `b + 1`. For each  $i$  in  $[0, n)$ , there is at most one call to `r.execute(i, i + 1)`.

(8.3) — If `r.set_value()` is called, then for each  $i$  in  $[0, n)$ , there is exactly one call to `r.execute(i, i + 1)`.

(8.4) — All calls to `execute` on `r` happen before the call to either `set_value`, `set_error`, or `set_stopped` on `r`.

(8.5) — All calls to `execute` and `set_value` on `r` are made on execution agents of the execution context represented by `*this`.

- <sup>9</sup> *Remarks:* The storage referenced by **s** may be used by **\*this** as temporary storage for the duration of the operation launched by this call.

# Annex A

## (informative)

### Grammar summary

[gram]

#### A.1 General

[gram.general]

- <sup>1</sup> This summary of C++ grammar is intended to be an aid to comprehension. It is not an exact statement of the language. In particular, the grammar described here accepts a superset of valid C++ constructs. Disambiguation rules (8.11, 9.2, 6.5.2) are applied to distinguish expressions from declarations. Further, access control, ambiguity, and type rules are used to weed out syntactically valid but meaningless constructs.

#### A.2 Keywords

[gram.key]

- <sup>1</sup> New context-dependent keywords are introduced into a program by `typedef` (9.2.4), `namespace` (9.9.2), `class` (Clause 11), `enumeration` (9.8.1), and `template` (Clause 13) declarations.

```
typedef-name:
 identifier
 simple-template-id

namespace-name:
 identifier
 namespace-alias

namespace-alias:
 identifier

class-name:
 identifier
 simple-template-id

enum-name:
 identifier

simple-template-name:
 identifier
```

#### A.3 Lexical conventions

[gram.lex]

```
n-char:
 any member of the translation character set except the U+007D RIGHT CURLY BRACKET or new-line
 character

n-char-sequence:
 n-char n-char-sequenceopt

named-universal-character:
 \N{ n-char-sequence }

hex-quad:
 hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

simple-hexadecimal-digit-sequence:
 hexadecimal-digit simple-hexadecimal-digit-sequenceopt

universal-character-name:
 \u hex-quad
 \U hex-quad hex-quad
 \u{ simple-hexadecimal-digit-sequence }
 named-universal-character
```

*preprocessing-token:*

*header-name*  
*import-keyword*  
*module-keyword*  
*export-keyword*  
*identifier*  
*pp-number*  
*character-literal*  
*user-defined-character-literal*  
*string-literal*  
*user-defined-string-literal*  
*preprocessing-op-or-punc*  
 each non-whitespace character that cannot be one of the above

*header-name:*

< *h-char-sequence* >  
 " *q-char-sequence* "

*h-char-sequence:*

*h-char h-char-sequence<sub>opt</sub>*

*h-char:*

any member of the translation character set except new-line and U+003E GREATER-THAN SIGN

*q-char-sequence:*

*q-char q-char-sequence<sub>opt</sub>*

*q-char:*

any member of the translation character set except new-line and U+0022 QUOTATION MARK

*pp-number:*

*digit*  
 . *digit*  
*pp-number identifier-continue*  
*pp-number* ' *digit*  
*pp-number* ' *nondigit*  
*pp-number* *e sign*  
*pp-number* *E sign*  
*pp-number* *p sign*  
*pp-number* *P sign*  
*pp-number* .

*preprocessing-op-or-punc:*

*preprocessing-operator*  
*operator-or-punctuator*

*preprocessing-operator:* one of

#      ##      %:      %: %:

*operator-or-punctuator:* one of

|        |       |        |        |        |       |       |    |   |  |
|--------|-------|--------|--------|--------|-------|-------|----|---|--|
| {      | }     | [      | ]      | (      | )     | [:    | :] |   |  |
| <%     | %>    | <:     | :>     | ;      | :     | ...   |    |   |  |
| ?      | ::    | .      | .*     | ->     | ->*   | ^^    | ~  |   |  |
| !      | +     | -      | *      | /      | %     | ^     | &  |   |  |
| =      | +=    | -=     | *=     | /=     | %=    | ^=    | &= | = |  |
| ==     | !=    | <      | >      | <=     | >=    | <=>   | && |   |  |
| <<     | >>    | <<=    | >>=    | ++     | --    | ,     |    |   |  |
| and    | or    | xor    | not    | bitand | bitor | compl |    |   |  |
| and_eq | or_eq | xor_eq | not_eq |        |       |       |    |   |  |

*token:*

*identifier*  
*keyword*  
*literal*  
*header-name*  
*operator-or-punctuator*

*identifier:*

*identifier-start*  
*identifier identifier-continue*

*identifier-start:**nondigit*

an element of the translation character set with the Unicode property `XID_Start` or the Unicode property `ID_Compat_Math_Start`

*identifier-continue:**digit**nondigit*

an element of the translation character set with the Unicode property `XID_Continue` or the Unicode property `ID_Compat_Math_Continue`

*nondigit: one of*

a b c d e f g h i j k l m  
 n o p q r s t u v w x y z  
 A B C D E F G H I J K L M  
 N O P Q R S T U V W X Y Z \_

*digit: one of*

0 1 2 3 4 5 6 7 8 9

*keyword:*

any identifier listed in [Table 5](#)

*import-keyword**module-keyword**export-keyword**literal:**integer-literal**character-literal**floating-point-literal**string-literal**boolean-literal**pointer-literal**user-defined-literal**integer-literal:*

*binary-literal* *integer-suffix*<sub>opt</sub>

*octal-literal* *integer-suffix*<sub>opt</sub>

*decimal-literal* *integer-suffix*<sub>opt</sub>

*hexadecimal-literal* *integer-suffix*<sub>opt</sub>

*binary-literal:*

0b *binary-digit*

0B *binary-digit*

*binary-literal* ' <sub>opt</sub> *binary-digit*

*octal-literal:*

0

*octal-literal* ' <sub>opt</sub> *octal-digit*

*decimal-literal:*

*nonzero-digit*

*decimal-literal* ' <sub>opt</sub> *digit*

*hexadecimal-literal:*

*hexadecimal-prefix* *hexadecimal-digit-sequence*

*binary-digit: one of*

0 1

*octal-digit: one of*

0 1 2 3 4 5 6 7

*nonzero-digit: one of*

1 2 3 4 5 6 7 8 9

*hexadecimal-prefix: one of*

0x 0X

*hexadecimal-digit-sequence:*

*hexadecimal-digit*

*hexadecimal-digit-sequence* ' <sub>opt</sub> *hexadecimal-digit*



*hexadecimal-digit*: one of

0 1 2 3 4 5 6 7 8 9  
a b c d e f  
A B C D E F

*integer-suffix*:

*unsigned-suffix* *long-suffix*<sub>opt</sub>  
*unsigned-suffix* *long-long-suffix*<sub>opt</sub>  
*unsigned-suffix* *size-suffix*<sub>opt</sub>  
*long-suffix* *unsigned-suffix*<sub>opt</sub>  
*long-long-suffix* *unsigned-suffix*<sub>opt</sub>  
*size-suffix* *unsigned-suffix*<sub>opt</sub>

*unsigned-suffix*: one of

u U

*long-suffix*: one of

l L

*long-long-suffix*: one of

ll LL

*size-suffix*: one of

z Z

*character-literal*:

*encoding-prefix*<sub>opt</sub> ' *c-char-sequence* '

*encoding-prefix*: one of

u8 u U L

*c-char-sequence*:

*c-char* *c-char-sequence*<sub>opt</sub>

*c-char*:

*basic-c-char*  
*escape-sequence*  
*universal-character-name*

*basic-c-char*:

any member of the translation character set except the U+0027 APOSTROPHE,  
U+005C REVERSE SOLIDUS, or new-line character

*escape-sequence*:

*simple-escape-sequence*  
*numeric-escape-sequence*  
*conditional-escape-sequence*

*simple-escape-sequence*:

\ *simple-escape-sequence-char*

*simple-escape-sequence-char*: one of

' " ? \ a b f n r t v

*numeric-escape-sequence*:

*octal-escape-sequence*  
*hexadecimal-escape-sequence*

*simple-octal-digit-sequence*:

*octal-digit* *simple-octal-digit-sequence*<sub>opt</sub>

*octal-escape-sequence*:

\ *octal-digit*  
\ *octal-digit* *octal-digit*  
\ *octal-digit* *octal-digit* *octal-digit*  
\ o{ *simple-octal-digit-sequence* }

*hexadecimal-escape-sequence*:

\ x *simple-hexadecimal-digit-sequence*  
\ x{ *simple-hexadecimal-digit-sequence* }

*conditional-escape-sequence*:

\ *conditional-escape-sequence-char*

*conditional-escape-sequence-char*:

any member of the basic character set that is not an *octal-digit*, a *simple-escape-sequence-char*, or the characters N, o, u, U, or x

*floating-point-literal*:

*decimal-floating-point-literal*  
*hexadecimal-floating-point-literal*

*decimal-floating-point-literal*:

*fractional-constant* *exponent-part*<sub>opt</sub> *floating-point-suffix*<sub>opt</sub>  
*digit-sequence* *exponent-part* *floating-point-suffix*<sub>opt</sub>

*hexadecimal-floating-point-literal*:

*hexadecimal-prefix* *hexadecimal-fractional-constant* *binary-exponent-part* *floating-point-suffix*<sub>opt</sub>  
*hexadecimal-prefix* *hexadecimal-digit-sequence* *binary-exponent-part* *floating-point-suffix*<sub>opt</sub>

*fractional-constant*:

*digit-sequence*<sub>opt</sub> . *digit-sequence*  
*digit-sequence* .

*hexadecimal-fractional-constant*:

*hexadecimal-digit-sequence*<sub>opt</sub> . *hexadecimal-digit-sequence*  
*hexadecimal-digit-sequence* .

*exponent-part*:

e *sign*<sub>opt</sub> *digit-sequence*  
E *sign*<sub>opt</sub> *digit-sequence*

*binary-exponent-part*:

p *sign*<sub>opt</sub> *digit-sequence*  
P *sign*<sub>opt</sub> *digit-sequence*

*sign*: one of

+ -

*digit-sequence*:

*digit*  
*digit-sequence* ' <sub>opt</sub> *digit*

*floating-point-suffix*: one of

f l f16 f32 f64 f128 bf16 F L F16 F32 F64 F128 BF16

*string-literal*:

*encoding-prefix*<sub>opt</sub> *plain-string-literal*  
*encoding-prefix*<sub>opt</sub> R *raw-string*

*plain-string-literal*:

" *s-char-sequence*<sub>opt</sub> "

*s-char-sequence*:

*s-char* *s-char-sequence*<sub>opt</sub>

*s-char*:

*basic-s-char*  
*escape-sequence*  
*universal-character-name*

*basic-s-char*:

any member of the translation character set except the U+0022 QUOTATION MARK,  
U+005C REVERSE SOLIDUS, or new-line character

*raw-string*:

" *d-char-sequence*<sub>opt</sub> ( *r-char-sequence*<sub>opt</sub> ) *d-char-sequence*<sub>opt</sub> "

*r-char-sequence*:

*r-char* *r-char-sequence*<sub>opt</sub>

*r-char*:

any member of the translation character set, except a U+0029 RIGHT PARENTHESIS followed by  
the initial *d-char-sequence* (which may be empty) followed by a U+0022 QUOTATION MARK

*d-char-sequence*:

*d-char* *d-char-sequence*<sub>opt</sub>

*d-char*:

any member of the basic character set except:

U+0020 SPACE, U+0028 LEFT PARENTHESIS, U+0029 RIGHT PARENTHESIS, U+005C REVERSE SOLIDUS,  
U+0009 CHARACTER TABULATION, U+000B LINE TABULATION, U+000C FORM FEED, and new-line

*unevaluated-string*:

*string-literal*

*boolean-literal*:

false  
true

*pointer-literal*:

*nullptr*

*user-defined-literal*:

*user-defined-integer-literal*  
*user-defined-floating-point-literal*  
*user-defined-string-literal*  
*user-defined-character-literal*

*user-defined-integer-literal*:

*decimal-literal ud-suffix*  
*octal-literal ud-suffix*  
*hexadecimal-literal ud-suffix*  
*binary-literal ud-suffix*

*user-defined-floating-point-literal*:

*fractional-constant exponent-part<sub>opt</sub> ud-suffix*  
*digit-sequence exponent-part ud-suffix*  
*hexadecimal-prefix hexadecimal-fractional-constant binary-exponent-part ud-suffix*  
*hexadecimal-prefix hexadecimal-digit-sequence binary-exponent-part ud-suffix*

*user-defined-string-literal*:

*string-literal ud-suffix*

*user-defined-character-literal*:

*character-literal ud-suffix*

*ud-suffix*:

*identifier*

## A.4 Basics

[gram.basic]

*splice-specifier*:

[*: constant-expression* :]

*splice-specialization-specifier*:

*splice-specifier* < *template-argument-list<sub>opt</sub>* >

*translation-unit*:

*declaration-seq<sub>opt</sub>*  
*global-module-fragment<sub>opt</sub> module-declaration declaration-seq<sub>opt</sub> private-module-fragment<sub>opt</sub>*

## A.5 Expressions

[gram.expr]

*primary-expression*:

*literal*  
*this*  
( *expression* )  
*id-expression*  
*lambda-expression*  
*fold-expression*  
*requires-expression*  
*splice-expression*

*id-expression*:

*unqualified-id*  
*qualified-id*  
*pack-index-expression*

*unqualified-id*:

- identifier*
- operator-function-id*
- conversion-function-id*
- literal-operator-id*
- ~ type-name*
- ~ computed-type-specifier*
- template-id*

*qualified-id*:

- nested-name-specifier template<sub>opt</sub> unqualified-id*

*nested-name-specifier*:

- ::*
- type-name ::*
- namespace-name ::*
- computed-type-specifier ::*
- splice-scope-specifier ::*
- nested-name-specifier identifier ::*
- nested-name-specifier template<sub>opt</sub> simple-template-id ::*

*splice-scope-specifier*:

- splice-specifier*
- template<sub>opt</sub> splice-specialization-specifier*

*pack-index-expression*:

- id-expression ... [ constant-expression ]*

*lambda-expression*:

- lambda-introducer attribute-specifier-seq<sub>opt</sub> lambda-declarator compound-statement*
- lambda-introducer < template-parameter-list > requires-clause<sub>opt</sub> attribute-specifier-seq<sub>opt</sub>*  
*lambda-declarator compound-statement*

*lambda-introducer*:

- [ lambda-capture<sub>opt</sub> ]*

*lambda-declarator*:

- lambda-specifier-seq noexcept-specifier<sub>opt</sub> attribute-specifier-seq<sub>opt</sub> trailing-return-type<sub>opt</sub>*  
*function-contract-specifier-seq<sub>opt</sub>*
- noexcept-specifier attribute-specifier-seq<sub>opt</sub> trailing-return-type<sub>opt</sub> function-contract-specifier-seq<sub>opt</sub>*  
*trailing-return-type<sub>opt</sub> function-contract-specifier-seq<sub>opt</sub>*
- ( parameter-declaration-clause ) lambda-specifier-seq<sub>opt</sub> noexcept-specifier<sub>opt</sub> attribute-specifier-seq<sub>opt</sub>*  
*trailing-return-type<sub>opt</sub> requires-clause<sub>opt</sub> function-contract-specifier-seq<sub>opt</sub>*

*lambda-specifier*:

- constexpr*
- constexpr*
- mutable*
- static*

*lambda-specifier-seq*:

- lambda-specifier lambda-specifier-seq<sub>opt</sub>*

*lambda-capture*:

- capture-default*
- capture-list*
- capture-default , capture-list*

*capture-default*:

- &*
- =*

*capture-list*:

- capture*
- capture-list , capture*

*capture*:

- simple-capture*
- init-capture*

*simple-capture*:  
*identifier* ...<sub>opt</sub>  
 & *identifier* ...<sub>opt</sub>  
 this  
 \* this

*init-capture*:  
 ...<sub>opt</sub> *identifier initializer*  
 & ...<sub>opt</sub> *identifier initializer*

*fold-expression*:  
 ( *cast-expression* *fold-operator* ... )  
 ( ... *fold-operator* *cast-expression* )  
 ( *cast-expression* *fold-operator* ... *fold-operator* *cast-expression* )

*fold-operator*: one of  
 +   -   \*   /   %   ^   &   |   <<   >>  
 +=   -=   \*=   /=   %=   ^=   &=   |=   <<=   >>=   =  
 ==   !=   <   >   <=   >=   &&   ||   ,   .\*   ->\*

*requires-expression*:  
 requires *requirement-parameter-list*<sub>opt</sub> *requirement-body*

*requirement-parameter-list*:  
 ( *parameter-declaration-clause* )

*requirement-body*:  
 { *requirement-seq* }

*requirement-seq*:  
*requirement* *requirement-seq*<sub>opt</sub>

*requirement*:  
*simple-requirement*  
*type-requirement*  
*compound-requirement*  
*nested-requirement*

*simple-requirement*:  
*expression* ;

*type-requirement*:  
 typename *nested-name-specifier*<sub>opt</sub> *type-name* ;  
 typename *splice-specifier* ;  
 typename *splice-specialization-specifier* ;

*compound-requirement*:  
 { *expression* } *noexcept-specifier*<sub>opt</sub> *return-type-requirement*<sub>opt</sub> ;

*return-type-requirement*:  
 -> *type-constraint*

*nested-requirement*:  
 requires *constraint-expression* ;

*splice-expression*:  
*splice-specifier*  
 template *splice-specifier*  
 template *splice-specialization-specifier*

*postfix-expression*:

*primary-expression*  
*postfix-expression* [ *expression-list*<sub>opt</sub> ]  
*postfix-expression* ( *expression-list*<sub>opt</sub> )  
*simple-type-specifier* ( *expression-list*<sub>opt</sub> )  
*typename-specifier* ( *expression-list*<sub>opt</sub> )  
*simple-type-specifier* *braced-init-list*  
*typename-specifier* *braced-init-list*  
*postfix-expression* . *template*<sub>opt</sub> *id-expression*  
*postfix-expression* . *splice-expression*  
*postfix-expression* -> *template*<sub>opt</sub> *id-expression*  
*postfix-expression* -> *splice-expression*  
*postfix-expression* ++  
*postfix-expression* --  
*dynamic\_cast* < *type-id* > ( *expression* )  
*static\_cast* < *type-id* > ( *expression* )  
*reinterpret\_cast* < *type-id* > ( *expression* )  
*const\_cast* < *type-id* > ( *expression* )  
*typeid* ( *expression* )  
*typeid* ( *type-id* )

*expression-list*:

*initializer-list*

*unary-expression*:

*postfix-expression*  
*unary-operator* *cast-expression*  
++ *cast-expression*  
-- *cast-expression*  
*await-expression*  
*sizeof* *unary-expression*  
*sizeof* ( *nofun-type-id* )  
*sizeof* ... ( *identifier* )  
*alignof* ( *nofun-type-id* )  
*noexcept-expression*  
*new-expression*  
*delete-expression*  
*reflect-expression*

*unary-operator*: one of

\* & + - ! ~

*await-expression*:

co\_await *cast-expression*

*noexcept-expression*:

noexcept ( *expression* )

*new-expression*:

::<sub>opt</sub> *new* *new-placement*<sub>opt</sub> *new-type-id* *new-initializer*<sub>opt</sub>  
::<sub>opt</sub> *new* *new-placement*<sub>opt</sub> ( *nofun-type-id* ) *new-initializer*<sub>opt</sub>

*new-placement*:

( *expression-list* )

*new-type-id*:

*type-specifier-seq* *new-declarator*<sub>opt</sub>

*new-declarator*:

*ptr-operator* *new-declarator*<sub>opt</sub>  
*noptr-new-declarator*

*noptr-new-declarator*:

[ *expression*<sub>opt</sub> ] *attribute-specifier-seq*<sub>opt</sub>  
*noptr-new-declarator* [ *constant-expression* ] *attribute-specifier-seq*<sub>opt</sub>

*new-initializer*:

( *expression-list*<sub>opt</sub> )  
*braced-init-list*

```

delete-expression:
 ::opt delete cast-expression
 ::opt delete [] cast-expression

reflect-expression:
 ^^ ::
 ^^ reflection-name
 ^^ type-id
 ^^ id-expression

reflection-name:
 nested-name-specifieropt identifier
 nested-name-specifier template identifier

cast-expression:
 unary-expression
 (nofun-type-id) cast-expression

pm-expression:
 cast-expression
 pm-expression .* cast-expression
 pm-expression ->* cast-expression

multiplicative-expression:
 pm-expression
 multiplicative-expression * pm-expression
 multiplicative-expression / pm-expression
 multiplicative-expression % pm-expression

additive-expression:
 multiplicative-expression
 additive-expression + multiplicative-expression
 additive-expression - multiplicative-expression

shift-expression:
 additive-expression
 shift-expression << additive-expression
 shift-expression >> additive-expression

compare-expression:
 shift-expression
 compare-expression <=> shift-expression

relational-expression:
 compare-expression
 relational-expression < compare-expression
 relational-expression > compare-expression
 relational-expression <= compare-expression
 relational-expression >= compare-expression

equality-expression:
 relational-expression
 equality-expression == relational-expression
 equality-expression != relational-expression

and-expression:
 equality-expression
 and-expression & equality-expression

exclusive-or-expression:
 and-expression
 exclusive-or-expression ^ and-expression

inclusive-or-expression:
 exclusive-or-expression
 inclusive-or-expression | exclusive-or-expression

logical-and-expression:
 inclusive-or-expression
 logical-and-expression && inclusive-or-expression

```

*logical-or-expression*:  
*logical-and-expression*  
*logical-or-expression* || *logical-and-expression*

*conditional-expression*:  
*logical-or-expression*  
*logical-or-expression* ? *expression* : *assignment-expression*

*yield-expression*:  
 co\_yield *assignment-expression*  
 co\_yield *braced-init-list*

*throw-expression*:  
 throw *assignment-expression*<sub>opt</sub>

*assignment-expression*:  
*conditional-expression*  
*yield-expression*  
*throw-expression*  
*logical-or-expression* *assignment-operator* *initializer-clause*

*assignment-operator*: one of  
 = \*= /= %= += -= >>= <<= &= ^= |=

*expression*:  
*assignment-expression*  
*expression* , *assignment-expression*

*constant-expression*:  
*conditional-expression*

## A.6 Statements

[gram.stmt]

*statement*:  
*labeled-statement*  
*attribute-specifier-seq*<sub>opt</sub> *expression-statement*  
*attribute-specifier-seq*<sub>opt</sub> *compound-statement*  
*attribute-specifier-seq*<sub>opt</sub> *selection-statement*  
*attribute-specifier-seq*<sub>opt</sub> *iteration-statement*  
*attribute-specifier-seq*<sub>opt</sub> *expansion-statement*  
*attribute-specifier-seq*<sub>opt</sub> *jump-statement*  
*attribute-specifier-seq*<sub>opt</sub> *assertion-statement*  
*declaration-statement*  
*attribute-specifier-seq*<sub>opt</sub> *try-block*

*init-statement*:  
*expression-statement*  
*simple-declaration*  
*alias-declaration*

*condition*:  
*expression*  
*condition-declaration*

*condition-declaration*:  
*attribute-specifier-seq*<sub>opt</sub> *decl-specifier-seq* *declarator* *brace-or-equal-initializer*  
*structured-binding-declaration* *initializer*

*for-range-declaration*:  
*attribute-specifier-seq*<sub>opt</sub> *decl-specifier-seq* *declarator*  
*structured-binding-declaration*

*for-range-initializer*:  
*expr-or-braced-init-list*

*label*:  
*attribute-specifier-seq*<sub>opt</sub> *identifier* :  
*attribute-specifier-seq*<sub>opt</sub> *case* *constant-expression* :  
*attribute-specifier-seq*<sub>opt</sub> *default* :

*labeled-statement*:  
*label* *statement*



```

expression-statement:
 expressionopt ;

compound-statement:
 { statement-seqopt label-seqopt }

statement-seq:
 statement statement-seqopt

label-seq:
 label label-seqopt

selection-statement:
 if constexpropt (init-statementopt condition) statement
 if constexpropt (init-statementopt condition) statement else statement
 if !opt constexpr compound-statement
 if !opt constexpr compound-statement else statement
 switch (init-statementopt condition) statement

iteration-statement:
 while (condition) statement
 do statement while (expression) ;
 for (init-statement conditionopt ; expressionopt) statement
 for (init-statementopt for-range-declaration : for-range-initializer) statement

expansion-statement:
 template for (init-statementopt for-range-declaration : expansion-initializer) compound-statement

expansion-initializer:
 expression
 expansion-init-list

expansion-init-list:
 { expression-list , opt }
 { }

jump-statement:
 break ;
 continue ;
 return expr-or-braced-init-listopt ;
 coroutine-return-statement
 goto identifier ;

coroutine-return-statement:
 co_return expr-or-braced-init-listopt ;

assertion-statement:
 contract_assert attribute-specifier-seqopt (conditional-expression) ;

declaration-statement:
 block-declaration

```

## A.7 Declarations

[gram.dcl]

```

declaration-seq:
 declaration declaration-seqopt

declaration:
 name-declaration
 special-declaration

name-declaration:
 block-declaration
 nodeclspec-function-declaration
 function-definition
 friend-type-declaration
 template-declaration
 deduction-guide
 linkage-specification
 namespace-definition
 empty-declaration
 attribute-declaration
 module-import-declaration

```

*special-declaration*:

- explicit-instantiation*
- explicit-specialization*
- export-declaration*

*block-declaration*:

- simple-declaration*
- asm-declaration*
- namespace-alias-definition*
- using-declaration*
- using-enum-declaration*
- using-directive*
- static\_assert-declaration*
- constexpr-block-declaration*
- alias-declaration*
- opaque-enum-declaration*

*nodeclspec-function-declaration*:

- attribute-specifier-seq*<sub>opt</sub> *declarator* ;

*alias-declaration*:

- using identifier attribute-specifier-seq*<sub>opt</sub> = *defining-type-id* ;

*sb-identifier*:

- ...opt identifier attribute-specifier-seq*<sub>opt</sub>

*sb-identifier-list*:

- sb-identifier*
- sb-identifier-list* , *sb-identifier*

*structured-binding-declaration*:

- attribute-specifier-seq*<sub>opt</sub> *decl-specifier-seq* *ref-qualifier*<sub>opt</sub> [ *sb-identifier-list* ]

*simple-declaration*:

- decl-specifier-seq* *init-declarator-list*<sub>opt</sub> ;
- attribute-specifier-seq* *decl-specifier-seq* *init-declarator-list* ;
- structured-binding-declaration* *initializer* ;

*static\_assert-message*:

- unevaluated-string*
- constant-expression*

*static\_assert-declaration*:

- static\_assert* ( *constant-expression* ) ;
- static\_assert* ( *constant-expression* , *static\_assert-message* ) ;

*constexpr-block-declaration*:

- constexpr* *compound-statement*

*empty-declaration*:

- ;

*attribute-declaration*:

- attribute-specifier-seq* ;

*decl-specifier*:

- storage-class-specifier*
- defining-type-specifier*
- function-specifier*
- friend*
- typedef*
- constexpr*
- constexpr*
- constexpr*
- inline*

*decl-specifier-seq*:

- decl-specifier* *attribute-specifier-seq*<sub>opt</sub>
- decl-specifier* *decl-specifier-seq*

*storage-class-specifier*:

- static*
- thread\_local*
- extern*
- mutable*

*function-specifier*:

- virtual*
- explicit-specifier*

*explicit-specifier*:

- explicit* ( *constant-expression* )
- explicit*

*typedef-name*:

- identifier*
- simple-template-id*

*type-specifier*:

- simple-type-specifier*
- elaborated-type-specifier*
- typename-specifier*
- cv-qualifier*

*type-specifier-seq*:

- type-specifier* *attribute-specifier-seq*<sub>opt</sub>
- type-specifier* *type-specifier-seq*

*defining-type-specifier*:

- type-specifier*
- class-specifier*
- enum-specifier*

*defining-type-specifier-seq*:

- defining-type-specifier* *attribute-specifier-seq*<sub>opt</sub>
- defining-type-specifier* *defining-type-specifier-seq*

*simple-type-specifier*:

- nested-name-specifier*<sub>opt</sub> *type-name*
- nested-name-specifier* *template* *simple-template-id*
- computed-type-specifier*
- placeholder-type-specifier*
- nested-name-specifier*<sub>opt</sub> *simple-template-name*
- pack-index-template-name*
- char*
- char8\_t*
- char16\_t*
- char32\_t*
- wchar\_t*
- bool*
- short*
- int*
- long*
- signed*
- unsigned*
- float*
- double*
- void*

*type-name*:

- class-name*
- enum-name*
- typedef-name*

*computed-type-specifier*:

- decltype-specifier*
- pack-index-specifier*
- splice-type-specifier*

*pack-index-specifier*:  
 typedef-name ... [ constant-expression ]

*elaborated-type-specifier*:  
 class-key attribute-specifier-seq<sub>opt</sub> nested-name-specifier<sub>opt</sub> identifier  
 class-key simple-template-id  
 class-key nested-name-specifier template<sub>opt</sub> simple-template-id  
 enum nested-name-specifier<sub>opt</sub> identifier

*decltype-specifier*:  
 decltype ( expression )

*placeholder-type-specifier*:  
 type-constraint<sub>opt</sub> auto  
 type-constraint<sub>opt</sub> decltype ( auto )

*splice-type-specifier*:  
 typename<sub>opt</sub> splice-specifier  
 typename<sub>opt</sub> splice-specialization-specifier

*init-declarator-list*:  
 init-declarator  
 init-declarator-list , init-declarator

*init-declarator*:  
 declarator initializer  
 declarator requires-clause<sub>opt</sub> function-contract-specifier-seq<sub>opt</sub>

*declarator*:  
 ptr-declarator  
 noptr-declarator parameters-and-qualifiers trailing-return-type

*ptr-declarator*:  
 noptr-declarator  
 ptr-operator ptr-declarator

*noptr-declarator*:  
 declarator-id attribute-specifier-seq<sub>opt</sub>  
 noptr-declarator parameters-and-qualifiers  
 noptr-declarator [ constant-expression<sub>opt</sub> ] attribute-specifier-seq<sub>opt</sub>  
 ( ptr-declarator )

*parameters-and-qualifiers*:  
 ( parameter-declaration-clause ) cv-qualifier-seq<sub>opt</sub>  
 ref-qualifier<sub>opt</sub> noexcept-specifier<sub>opt</sub> attribute-specifier-seq<sub>opt</sub>

*trailing-return-type*:  
 -> nofun-type-id

*ptr-operator*:  
 \* attribute-specifier-seq<sub>opt</sub> cv-qualifier-seq<sub>opt</sub>  
 & attribute-specifier-seq<sub>opt</sub>  
 && attribute-specifier-seq<sub>opt</sub>  
 nested-name-specifier \* attribute-specifier-seq<sub>opt</sub> cv-qualifier-seq<sub>opt</sub>

*cv-qualifier-seq*:  
 cv-qualifier cv-qualifier-seq<sub>opt</sub>

*cv-qualifier*:  
 const  
 volatile

*ref-qualifier*:  
 &  
 &&

*declarator-id*:  
 ...<sub>opt</sub> id-expression

*nofun-type-id*:  
 type-specifier-seq nofun-declarator<sub>opt</sub>

*nofun-declarator*:  
 ptr-nofun-declarator  
 noptr-nofun-declarator parameters-and-qualifiers trailing-return-type

*ptr-nofun-declarator*:

- noptr-nofun-declarator*
- ptr-operator ptr-nofun-declarator<sub>opt</sub>*

*noptr-nofun-declarator*:

- noptr-nofun-declarator parameters-and-qualifiers*
- noptr-nofun-declarator<sub>opt</sub> [ constant-expression<sub>opt</sub> ] attribute-specifier-seq<sub>opt</sub>*
- ( ptr-nofun-declarator )*

*type-id*:

- type-specifier-seq abstract-declarator<sub>opt</sub>*

*defining-type-id*:

- defining-type-specifier-seq abstract-declarator<sub>opt</sub>*

*abstract-declarator*:

- ptr-abstract-declarator*
- noptr-abstract-declarator<sub>opt</sub> parameters-and-qualifiers trailing-return-type*
- abstract-pack-declarator*

*ptr-abstract-declarator*:

- noptr-abstract-declarator*
- ptr-operator ptr-abstract-declarator<sub>opt</sub>*

*noptr-abstract-declarator*:

- noptr-abstract-declarator<sub>opt</sub> parameters-and-qualifiers*
- noptr-abstract-declarator<sub>opt</sub> [ constant-expression<sub>opt</sub> ] attribute-specifier-seq<sub>opt</sub>*
- ( ptr-abstract-declarator )*

*abstract-pack-declarator*:

- noptr-abstract-pack-declarator*
- ptr-operator abstract-pack-declarator*

*noptr-abstract-pack-declarator*:

- noptr-abstract-pack-declarator parameters-and-qualifiers*
- ...

*parameter-declaration-clause*:

- ...
- parameter-declaration-list<sub>opt</sub>*
- parameter-declaration-list , ...*
- parameter-declaration-list ...*

*parameter-declaration-list*:

- parameter-declaration*
- parameter-declaration-list , parameter-declaration*

*parameter-declaration*:

- attribute-specifier-seq<sub>opt</sub> this<sub>opt</sub> decl-specifier-seq declarator*
- attribute-specifier-seq<sub>opt</sub> decl-specifier-seq declarator = initializer-clause*
- attribute-specifier-seq<sub>opt</sub> this<sub>opt</sub> decl-specifier-seq abstract-declarator<sub>opt</sub>*
- attribute-specifier-seq<sub>opt</sub> decl-specifier-seq abstract-declarator<sub>opt</sub> = initializer-clause*

*function-contract-specifier-seq*:

- function-contract-specifier function-contract-specifier-seq<sub>opt</sub>*

*function-contract-specifier*:

- precondition-specifier*
- postcondition-specifier*

*precondition-specifier*:

- pre attribute-specifier-seq<sub>opt</sub> ( conditional-expression )*

*postcondition-specifier*:

- post attribute-specifier-seq<sub>opt</sub> ( result-name-introducer<sub>opt</sub> conditional-expression )*

*attributed-identifier*:

- identifier attribute-specifier-seq<sub>opt</sub>*

*result-name-introducer*:

- attributed-identifier :*

*initializer*:

- brace-or-equal-initializer*
- ( expression-list )*

*brace-or-equal-initializer*:  
     = *initializer-clause*  
     *braced-init-list*

*initializer-clause*:  
     *assignment-expression*  
     *braced-init-list*

*braced-init-list*:  
     { *initializer-list* ,<sub>opt</sub> }  
     { *designated-initializer-list* ,<sub>opt</sub> }  
     { }

*initializer-list*:  
     *initializer-clause* . . .<sub>opt</sub>  
     *initializer-list* , *initializer-clause* . . .<sub>opt</sub>

*designated-only-initializer-list*:  
     *designated-initializer-clause*  
     *designated-only-initializer-list* , *designated-initializer-clause*

*designated-initializer-list*:  
     *designated-only-initializer-list*  
     *initializer-list* , *designated-only-initializer-list*

*designated-initializer-clause*:  
     *designator* *brace-or-equal-initializer*

*designator*:  
     . *identifier*

*expr-or-braced-init-list*:  
     *expression*  
     *braced-init-list*

*function-definition*:  
     *attribute-specifier-seq*<sub>opt</sub> *decl-specifier-seq*<sub>opt</sub> *declarator* *virt-specifier-seq*<sub>opt</sub>  
         *function-contract-specifier-seq*<sub>opt</sub> *function-body*  
     *attribute-specifier-seq*<sub>opt</sub> *decl-specifier-seq*<sub>opt</sub> *declarator* *requires-clause*  
         *function-contract-specifier-seq*<sub>opt</sub> *function-body*

*function-body*:  
     *ctor-initializer*<sub>opt</sub> *compound-statement*  
     *function-try-block*  
     = *default* ;  
     *deleted-function-body*

*deleted-function-body*:  
     = *delete* ;  
     = *delete* ( *unevaluated-string* ) ;

*enum-name*:  
     *identifier*

*enum-specifier*:  
     *enum-head* { *enumerator-list*<sub>opt</sub> }  
     *enum-head* { *enumerator-list* , }

*enum-head*:  
     *enum-key* *attribute-specifier-seq*<sub>opt</sub> *enum-head-name*<sub>opt</sub> *enum-base*<sub>opt</sub>

*enum-head-name*:  
     *nested-name-specifier*<sub>opt</sub> *identifier*

*opaque-enum-declaration*:  
     *enum-key* *attribute-specifier-seq*<sub>opt</sub> *enum-head-name* *enum-base*<sub>opt</sub> ;

*enum-key*:  
     *enum*  
     *enum* *class*  
     *enum* *struct*

*enum-base*:  
     : *type-specifier-seq*

*enumerator-list*:  
     *enumerator-definition*  
     *enumerator-list* , *enumerator-definition*  
*enumerator-definition*:  
     *enumerator*  
     *enumerator* = *constant-expression*  
*enumerator*:  
     *identifier* *attribute-specifier-seq*<sub>opt</sub>  
*using-enum-declaration*:  
     using enum *using-enum-declarator* ;  
*using-enum-declarator*:  
     *nested-name-specifier*<sub>opt</sub> *identifier*  
     *nested-name-specifier*<sub>opt</sub> *simple-template-id*  
     *splice-type-specifier*  
*namespace-name*:  
     *identifier*  
     *namespace-alias*  
*namespace-definition*:  
     *named-namespace-definition*  
     *unnamed-namespace-definition*  
     *nested-namespace-definition*  
*named-namespace-definition*:  
     inline<sub>opt</sub> namespace *attribute-specifier-seq*<sub>opt</sub> *identifier* { *namespace-body* }  
*unnamed-namespace-definition*:  
     inline<sub>opt</sub> namespace *attribute-specifier-seq*<sub>opt</sub> { *namespace-body* }  
*nested-namespace-definition*:  
     namespace *enclosing-namespace-specifier* :: inline<sub>opt</sub> *identifier* { *namespace-body* }  
*enclosing-namespace-specifier*:  
     *identifier*  
     *enclosing-namespace-specifier* :: inline<sub>opt</sub> *identifier*  
*namespace-body*:  
     *declaration-seq*<sub>opt</sub>  
*namespace-alias*:  
     *identifier*  
*namespace-alias-definition*:  
     namespace *identifier* = *qualified-namespace-specifier* ;  
     namespace *identifier* = *splice-specifier* ;  
*qualified-namespace-specifier*:  
     *nested-name-specifier*<sub>opt</sub> *namespace-name*  
*using-directive*:  
     *attribute-specifier-seq*<sub>opt</sub> using namespace *nested-name-specifier*<sub>opt</sub> *namespace-name* ;  
     *attribute-specifier-seq*<sub>opt</sub> using namespace *splice-specifier* ;  
*using-declaration*:  
     using *using-declarator-list* ;  
*using-declarator-list*:  
     *using-declarator* ...<sub>opt</sub>  
     *using-declarator-list* , *using-declarator* ...<sub>opt</sub>  
*using-declarator*:  
     typename<sub>opt</sub> *nested-name-specifier* *unqualified-id*  
*asm-declaration*:  
     *attribute-specifier-seq*<sub>opt</sub> asm ( *balanced-token-seq* ) ;  
*linkage-specification*:  
     extern *unevaluated-string* { *declaration-seq*<sub>opt</sub> }  
     extern *unevaluated-string* *name-declaration*  
*attribute-specifier-seq*:  
     *attribute-specifier* *attribute-specifier-seq*<sub>opt</sub>

*attribute-specifier*:

- [ [ *attribute-using-prefix*<sub>opt</sub> *attribute-list* ] ]
- [ [ *annotation-list* ] ]
- alignment-specifier*

*alignment-specifier*:

- alignas* ( *nofun-type-id* ...<sub>opt</sub> )
- alignas* ( *constant-expression* ...<sub>opt</sub> )

*attribute-using-prefix*:

- using *attribute-namespace* :

*attribute-list*:

- attribute*<sub>opt</sub>
- attribute-list* , *attribute*<sub>opt</sub>
- attribute* ...
- attribute-list* , *attribute* ...

*annotation-list*:

- annotation* ...<sub>opt</sub>
- annotation-list* , *annotation* ...<sub>opt</sub>

*attribute*:

- attribute-token* *attribute-argument-clause*<sub>opt</sub>

*annotation*:

- = *constant-expression*

*attribute-token*:

- identifier*
- attribute-scoped-token*

*attribute-scoped-token*:

- attribute-namespace* :: *identifier*

*attribute-namespace*:

- identifier*

*attribute-argument-clause*:

- ( *balanced-token-seq*<sub>opt</sub> )

*balanced-token-seq*:

- balanced-token* *balanced-token-seq*<sub>opt</sub>

*balanced-token*:

- ( *balanced-token-seq*<sub>opt</sub> )
- [ *balanced-token-seq*<sub>opt</sub> ]
- { *balanced-token-seq*<sub>opt</sub> }
- [ : *balanced-token-seq*<sub>opt</sub> : ]
- any *token* other than ( , ) , [ , ] , { , } , [ : , or : ]

## A.8 Modules

[gram.module]

*module-declaration*:

- export-keyword<sub>opt</sub> *module-keyword* *module-name* *module-partition*<sub>opt</sub> *attribute-specifier-seq*<sub>opt</sub> ;

*module-name*:

- module-name-qualifier*<sub>opt</sub> *identifier*

*module-partition*:

- : *module-name-qualifier*<sub>opt</sub> *identifier*

*module-name-qualifier*:

- identifier* .
- module-name-qualifier* *identifier* .

*export-declaration*:

- export *name-declaration*
- export { *declaration-seq*<sub>opt</sub> }
- export-keyword *module-import-declaration*

*module-import-declaration*:

- import-keyword *module-name* *attribute-specifier-seq*<sub>opt</sub> ;
- import-keyword *module-partition* *attribute-specifier-seq*<sub>opt</sub> ;
- import-keyword *header-name* *attribute-specifier-seq*<sub>opt</sub> ;



*global-module-fragment*:  
*module-keyword* ; *declaration-seq<sub>opt</sub>*

*private-module-fragment*:  
*module-keyword* : *private* ; *declaration-seq<sub>opt</sub>*

## A.9 Classes

[gram.class]

*class-name*:  
*identifier*  
*simple-template-id*

*class-specifier*:  
*class-head* { *member-specification<sub>opt</sub>* }

*class-head*:  
*class-key* *attribute-specifier-seq<sub>opt</sub>* *class-head-name* *class-property-specifier-seq<sub>opt</sub>* *base-clause<sub>opt</sub>*  
*class-key* *attribute-specifier-seq<sub>opt</sub>* *base-clause<sub>opt</sub>*

*class-head-name*:  
*nested-name-specifier<sub>opt</sub>* *class-name*

*class-property-specifier-seq*:  
*class-property-specifier* *class-property-specifier-seq<sub>opt</sub>*

*class-property-specifier*:  
*final*

*class-key*:  
*class*  
*struct*  
*union*

*member-specification*:  
*member-declaration* *member-specification<sub>opt</sub>*  
*access-specifier* : *member-specification<sub>opt</sub>*

*member-declaration*:  
*attribute-specifier-seq<sub>opt</sub>* *decl-specifier-seq<sub>opt</sub>* *member-declarator-list<sub>opt</sub>* ;  
*function-definition*  
*friend-type-declaration*  
*using-declaration*  
*using-enum-declaration*  
*static\_assert-declaration*  
*constexpr-block-declaration*  
*template-declaration*  
*explicit-specialization*  
*deduction-guide*  
*alias-declaration*  
*opaque-enum-declaration*  
*empty-declaration*

*member-declarator-list*:  
*member-declarator*  
*member-declarator-list* , *member-declarator*

*member-declarator*:  
*declarator* *virt-specifier-seq<sub>opt</sub>* *function-contract-specifier-seq<sub>opt</sub>* *pure-specifier<sub>opt</sub>*  
*declarator* *requires-clause* *function-contract-specifier-seq<sub>opt</sub>*  
*declarator* *brace-or-equal-initializer*  
*identifier<sub>opt</sub>* *attribute-specifier-seq<sub>opt</sub>* : *constant-expression* *brace-or-equal-initializer<sub>opt</sub>*

*virt-specifier-seq*:  
*virt-specifier* *virt-specifier-seq<sub>opt</sub>*

*virt-specifier*:  
*override*  
*final*

*pure-specifier*:  
 = 0

*friend-type-declaration*:  
*friend* *friend-type-specifier-list* ;

*friend-type-specifier-list*:  
*friend-type-specifier* ... *opt*  
*friend-type-specifier-list* , *friend-type-specifier* ... *opt*

*friend-type-specifier*:  
*simple-type-specifier*  
*elaborated-type-specifier*  
*typename-specifier*

*conversion-function-id*:  
operator *conversion-type-id*

*conversion-type-id*:  
*type-specifier-seq* *conversion-declarator*<sub>opt</sub>

*conversion-declarator*:  
*ptr-operator* *conversion-declarator*<sub>opt</sub>

*base-clause*:  
: *base-specifier-list*

*base-specifier-list*:  
*base-specifier* ... *opt*  
*base-specifier-list* , *base-specifier* ... *opt*

*base-specifier*:  
*attribute-specifier-seq*<sub>opt</sub> *class-or-decltype*  
*attribute-specifier-seq*<sub>opt</sub> virtual *access-specifier*<sub>opt</sub> *class-or-decltype*  
*attribute-specifier-seq*<sub>opt</sub> *access-specifier* virtual<sub>opt</sub> *class-or-decltype*

*class-or-decltype*:  
*nested-name-specifier*<sub>opt</sub> *type-name*  
*nested-name-specifier* template *simple-template-id*  
*computed-type-specifier*

*access-specifier*:  
private  
protected  
public

*ctor-initializer*:  
: *mem-initializer-list*

*mem-initializer-list*:  
*mem-initializer* ... *opt*  
*mem-initializer-list* , *mem-initializer* ... *opt*

*mem-initializer*:  
*mem-initializer-id* ( *expression-list*<sub>opt</sub> )  
*mem-initializer-id* *braced-init-list*

*mem-initializer-id*:  
*class-or-decltype*  
*identifier*

## A.10 Overloading

[gram.over]

*operator-function-id*:  
operator *operator*

*operator*: one of

|     |        |       |          |              |    |    |     |
|-----|--------|-------|----------|--------------|----|----|-----|
| new | delete | new[] | delete[] | co_await ( ) | [] | -> | ->* |
| ~   | !      | +     | -        | *            | /  | %  | &   |
|     | =      | +=    | -=       | *=           | /= | %= | &=  |
| =   | ==     | !=    | <        | >            | <= | >= | <=> |
|     | <<     | >>    | <<=      | >>=          | ++ | -- | ,   |

*literal-operator-id*:  
operator *unevaluated-string identifier*  
operator *user-defined-string-literal*

## A.11 Templates

[gram.temp]

*template-declaration*:  
     *template-head declaration*  
     *template-head concept-definition*

*template-head*:  
     *template* < *template-parameter-list* > *requires-clause*<sub>opt</sub>

*template-parameter-list*:  
     *template-parameter*  
     *template-parameter-list* , *template-parameter*

*requires-clause*:  
     *requires constraint-logical-or-expression*

*constraint-logical-or-expression*:  
     *constraint-logical-and-expression*  
     *constraint-logical-or-expression* || *constraint-logical-and-expression*

*constraint-logical-and-expression*:  
     *primary-expression*  
     *constraint-logical-and-expression* && *primary-expression*

*template-parameter*:  
     *type-parameter*  
     *parameter-declaration*  
     *type-tt-parameter*  
     *variable-tt-parameter*  
     *concept-tt-parameter*

*type-parameter*:  
     *type-parameter-key* . . . <sub>opt</sub> *identifier*<sub>opt</sub>  
     *type-parameter-key identifier*<sub>opt</sub> = *type-id*  
     *type-constraint* . . . <sub>opt</sub> *identifier*<sub>opt</sub>  
     *type-constraint identifier*<sub>opt</sub> = *type-id*

*type-parameter-key*:  
     class  
     typename

*type-constraint*:  
     *nested-name-specifier*<sub>opt</sub> *concept-name*  
     *nested-name-specifier*<sub>opt</sub> *concept-name* < *template-argument-list*<sub>opt</sub> >

*type-tt-parameter*:  
     *template-head type-parameter-key* . . . <sub>opt</sub> *identifier*<sub>opt</sub>  
     *template-head type-parameter-key identifier*<sub>opt</sub> *type-tt-parameter-default*

*type-tt-parameter-default*:  
     = *nested-name-specifier*<sub>opt</sub> *template-name*  
     = *nested-name-specifier* *template* *template-name*

*variable-tt-parameter*:  
     *template-head auto* . . . <sub>opt</sub> *identifier*<sub>opt</sub>  
     *template-head auto identifier*<sub>opt</sub> = *nested-name-specifier*<sub>opt</sub> *template-name*

*concept-tt-parameter*:  
     *template* < *template-parameter-list* > *concept* . . . <sub>opt</sub> *identifier*<sub>opt</sub>  
     *template* < *template-parameter-list* > *concept identifier*<sub>opt</sub> = *nested-name-specifier*<sub>opt</sub> *template-name*

*simple-template-id*:  
     *template-name* < *template-argument-list*<sub>opt</sub> >

*template-id*:  
     *simple-template-id*  
     *operator-function-id* < *template-argument-list*<sub>opt</sub> >  
     *literal-operator-id* < *template-argument-list*<sub>opt</sub> >

*template-name*:  
     *simple-template-name*  
     *pack-index-template-name*

*pack-index-template-name*:  
     *simple-template-name* . . . [ *constant-expression* ]

*simple-template-name:*  
*identifier*

*template-argument-list:*  
*template-argument* ...<sub>opt</sub>  
*template-argument-list* , *template-argument* ...<sub>opt</sub>

*template-argument:*  
*template-argument-name*  
*constant-expression*  
*type-id*  
*braced-init-list*

*template-argument-name:*  
*nested-name-specifier*<sub>opt</sub> *identifier*  
*nested-name-specifier* *template* *identifier*

*constraint-expression:*  
*logical-or-expression*

*deduction-guide:*  
*explicit-specifier*<sub>opt</sub> *simple-template-name* ( *parameter-declaration-clause* ) -> *simple-template-id* *requires-clause*<sub>opt</sub> ;

*concept-definition:*  
*concept* *concept-name* *attribute-specifier-seq*<sub>opt</sub> = *constraint-expression* ;

*concept-name:*  
*identifier*

*typename-specifier:*  
*typename* *nested-name-specifier* *identifier*  
*typename* *nested-name-specifier* *template*<sub>opt</sub> *simple-template-id*

*explicit-instantiation:*  
*extern*<sub>opt</sub> *template* *declaration*

*explicit-specialization:*  
*template* < > *declaration*

## A.12 Exception handling

[gram.except]

*try-block:*  
*try* *compound-statement* *handler-seq*

*function-try-block:*  
*try* *ctor-initializer*<sub>opt</sub> *compound-statement* *handler-seq*

*handler-seq:*  
*handler* *handler-seq*<sub>opt</sub>

*handler:*  
*catch* ( *exception-declaration* ) *compound-statement*

*exception-declaration:*  
*attribute-specifier-seq*<sub>opt</sub> *type-specifier-seq* *declarator*  
*attribute-specifier-seq*<sub>opt</sub> *type-specifier-seq* *abstract-declarator*<sub>opt</sub>  
 ...

*noexcept-specifier:*  
*noexcept* ( *constant-expression* )  
*noexcept*

## A.13 Preprocessing directives

[gram.cpp]

*preprocessing-file:*  
*group*<sub>opt</sub>  
*module-file*

*module-file:*  
*line-directives*<sub>opt</sub> *pp-global-module-fragment*<sub>opt</sub> *pp-module* *group*<sub>opt</sub> *pp-private-module-fragment*<sub>opt</sub>

*pp-global-module-fragment:*  
*module* ; *new-line* *group*<sub>opt</sub>

```

pp-private-module-fragment:
 module : private ; new-line groupopt

group:
 group-part
 group group-part

group-part:
 control-line
 if-section
 text-line
 # conditionally-supported-directive

control-line:
 # include pp-tokens new-line
 pp-import
 # embed pp-tokens new-line
 # define identifier replacement-list new-line
 # define identifier lparen identifier-listopt) replacement-list new-line
 # define identifier lparen ...) replacement-list new-line
 # define identifier lparen identifier-list , ...) replacement-list new-line
 # undef identifier new-line
 line-directive
 # error pp-tokensopt new-line
 # warning pp-tokensopt new-line
 # pragma pp-tokensopt new-line
 # new-line

line-directives:
 line-directive line-directivesopt

if-section:
 if-group elif-groupsopt else-groupopt endif-line

if-group:
 # if constant-expression new-line groupopt
 # ifdef identifier new-line groupopt
 # ifndef identifier new-line groupopt

elif-groups:
 elif-group elif-groupsopt

elif-group:
 # elif constant-expression new-line groupopt
 # elifdef identifier new-line groupopt
 # elifndef identifier new-line groupopt

else-group:
 # else new-line groupopt

endif-line:
 # endif new-line

text-line:
 pp-tokensopt new-line

conditionally-supported-directive:
 pp-tokens new-line

lparen:
 a (character not immediately preceded by whitespace

identifier-list:
 identifier
 identifier-list , identifier

replacement-list:
 pp-tokensopt

pp-tokens:
 preprocessing-token pp-tokensopt

embed-parameter-seq:
 embed-parameter embed-parameter-seqopt

```

*embed-parameter*:

- embed-standard-parameter*
- embed-prefixed-parameter*

*embed-standard-parameter*:

- limit ( *pp-balanced-token-seq* )
- offset ( *pp-balanced-token-seq* )
- prefix ( *pp-balanced-token-seq<sub>opt</sub>* )
- suffix ( *pp-balanced-token-seq<sub>opt</sub>* )
- if\_empty ( *pp-balanced-token-seq<sub>opt</sub>* )

*embed-prefixed-parameter*:

- identifier* :: *identifier*
- identifier* :: *identifier* ( *pp-balanced-token-seq<sub>opt</sub>* )

*pp-balanced-token-seq*:

- pp-balanced-token* *pp-balanced-token-seq<sub>opt</sub>*

*pp-balanced-token*:

- ( *pp-balanced-token-seq<sub>opt</sub>* )
- [ *pp-balanced-token-seq<sub>opt</sub>* ]
- { *pp-balanced-token-seq<sub>opt</sub>* }
- any *pp-token* except:
  - parenthesis (U+0028 LEFT PARENTHESIS and U+0029 RIGHT PARENTHESIS),
  - bracket (U+005B LEFT SQUARE BRACKET and U+005D RIGHT SQUARE BRACKET), or
  - brace (U+007B LEFT CURLY BRACKET and U+007D RIGHT CURLY BRACKET).

*new-line*:

- the new-line character

*defined-macro-expression*:

- defined *identifier*
- defined ( *identifier* )

*h-preprocessing-token*:

- any *preprocessing-token* other than >

*h-pp-tokens*:

- h-preprocessing-token* *h-pp-tokens<sub>opt</sub>*

*header-name-tokens*:

- plain-string-literal*
- < *h-pp-tokens* >

*has-include-expression*:

- \_\_has\_include ( *header-name* )
- \_\_has\_include ( *header-name-tokens* )

*has-embed-expression*:

- \_\_has\_embed ( *header-name* *pp-balanced-token-seq<sub>opt</sub>* )
- \_\_has\_embed ( *header-name-tokens* *pp-balanced-token-seq<sub>opt</sub>* )

*has-attribute-expression*:

- \_\_has\_cpp\_attribute ( *pp-tokens* )

*pp-module*:

- export<sub>opt</sub> module *pp-tokens<sub>opt</sub>* *new-line*

*pp-import*:

- export<sub>opt</sub> import *header-name* *pp-tokens<sub>opt</sub>* ; *new-line*
- export<sub>opt</sub> import *header-name-tokens* *pp-tokens<sub>opt</sub>* ; *new-line*
- export<sub>opt</sub> import *pp-tokens* ; *new-line*

*va-opt-replacement*:

- \_\_VA\_OPT\_\_ ( *pp-tokens<sub>opt</sub>* )

*line-directive*:

- # line *pp-tokens* *new-line*

# Annex B

## (informative)

### Implementation quantities [implimits]

<sup>1</sup> Implementations can exhibit limitations for various quantities; some possibilities are presented in the following list. The bracketed number following each quantity is a potential minimum value for that quantity.

- (1.1) — Nesting levels of compound statements (8.4), iteration control structures (8.6), and selection control structures (8.5) [256].
- (1.2) — Nesting levels of conditional inclusion (15.2) [256].
- (1.3) — Pointer (9.3.4.2), pointer-to-member (9.3.4.4), array (9.3.4.5), and function (9.3.4.6) declarators (in any combination) modifying a type in a declaration [256].
- (1.4) — Nesting levels of parenthesized expressions (7.5.4) within a full-expression [256].
- (1.5) — Number of characters in an internal identifier (5.11) or macro name (15.7) [1 024].
- (1.6) — Number of characters in an external identifier (5.11, 6.7) [1 024].
- (1.7) — External identifiers (6.7) in one translation unit [65 536].
- (1.8) — Identifiers with block scope declared in one block (6.4.3) [1 024].
- (1.9) — Structured bindings (9.7) introduced in one declaration [256].
- (1.10) — Macro identifiers (15.7) simultaneously defined in one translation unit [65 536].
- (1.11) — Parameters in one function definition (9.6.1) [256].
- (1.12) — Arguments in one function call (7.6.1.3) [256].
- (1.13) — Parameters in one macro definition (15.7) [256].
- (1.14) — Arguments in one macro invocation (15.7) [256].
- (1.15) — Characters in one logical source line (5.2) [65 536].
- (1.16) — Characters in a *string-literal* (5.13.5) (after concatenation (5.2)) [65 536].
- (1.17) — Size of an object (6.8.2) [262 144].
- (1.18) — Nesting levels for `#include` files (15.3) [256].
- (1.19) — Case labels for a `switch` statement (8.5.3) (excluding those for any nested `switch` statements) [16 384].
- (1.20) — Non-static data members (including inherited ones) in a single class (11.4) [16 384].
- (1.21) — Lambda-captures in one *lambda-expression* (7.5.6.3) [256].
- (1.22) — Enumeration constants in a single enumeration (9.8.1) [4 096].
- (1.23) — Levels of nested class definitions (11.4.12) in a single *member-specification* [256].
- (1.24) — Functions registered by `atexit()` (17.5) [32].
- (1.25) — Functions registered by `at_quick_exit()` (17.5) [32].
- (1.26) — Direct and indirect base classes (11.7) [16 384].
- (1.27) — Direct base classes for a single class (11.7) [1 024].
- (1.28) — Class members declared in a single *member-specification* (including member functions) (11.4) [4 096].
- (1.29) — Final overriding virtual functions in a class, accessible or not (11.7.3) [16 384].
- (1.30) — Direct and indirect virtual bases of a class (11.7.2) [1 024].
- (1.31) — Static data members of a class (11.4.9.3) [1 024].

- (1.32) — Friend declarations in a class (11.8.4) [4 096].
- (1.33) — Access control declarations in a class (11.8.2) [4 096].
- (1.34) — Member initializers in a constructor definition (11.9.3) [6 144].
- (1.35) — *initializer-clauses* in one *braced-init-list* (9.5) [16 384].
- (1.36) — Scope qualifications of one identifier (7.5.5.3) [256].
- (1.37) — Nested *linkage-specifications* (9.12) [1 024].
- (1.38) — Recursive constexpr function invocations (9.2.6) [512].
- (1.39) — Full-expressions evaluated within a core constant expression (7.7.2) [1 048 576].
- (1.40) — Template parameters in a template declaration (13.2) [1 024].
- (1.41) — Recursively nested template instantiations (13.9.2), including substitution during template argument deduction (13.10.3) [1 024].
- (1.42) — Handlers per try block (14.4) [256].
- (1.43) — Number of placeholders (22.10.15.5) [10].
- (1.44) — Number of hazard-protectable possibly-reclaimable objects (32.11.3.1) [256].



# Annex C

## (informative)

### Compatibility

[diff]

#### C.1 C++ and ISO C++ 2026

[diff.cpp26]

##### C.1.1 General

[diff.cpp26.general]

- <sup>1</sup> Subclause C.1 lists the differences between C++ and ISO C++ 2026, by the chapters of this document.

##### C.1.2 Clause 6: basics

[diff.cpp26.basic]

- <sup>1</sup> **Affected subclause:** 6.8.6.5.3

**Change:** Potentially throwing exception specifications on deallocation functions are ill-formed.

**Rationale:** Removes potential undefined behavior.

**Effect on original feature:** Valid C++ 2026 code that declares a deallocation function with a potentially throwing exception specification becomes ill-formed.

[Example 1:

```
struct T {
 void operator delete(void *) noexcept(false); // ill-formed; previously well-formed
};
```

— end example]

##### C.1.3 Clause 9: declarations

[diff.cpp26.dcl]

- <sup>1</sup> **Affected subclause:** 9.5

**Change:** Support for designated initialization of base classes of aggregates.

**Rationale:** New functionality.

**Effect on original feature:** Some valid C++ 2026 code may fail to compile.

[Example 1:

```
struct A { int a; };
struct B : A { int b; };

void f(A); // #1
void f(B); // #2

void g() {
 f({.a=1}); // ambiguous between #1 and #2; previously called #1
}
```

— end example]

- <sup>2</sup> **Affected subclause:** 9.6.2

**Change:** It is no longer valid for explicitly defaulted assignment operators to have a *cv-qualifier-seq* or *&& ref-qualifier*, or for an explicitly defaulted move assignment operator to have a parameter of type “reference to T” where T is cv-qualified.

**Rationale:** Removal of rarely-used and confusing feature.

**Effect on original feature:** A valid C++ 2026 program which uses these features on an explicitly defaulted assignment operator is ill-formed.

[Example 2:

```
struct S {
 S& operator=(const S&) && = default; // ill-formed; previously well-formed
};
```

```

struct T {
 T& operator=(const T&) const = default; // ill-formed; previously well-formed but deleted
};
struct U {
 U& operator=(const U&&) = default; // ill-formed; previously well-formed but deleted
};
— end example]

```

## C.2 C++ and ISO C++ 2023

[diff.cpp23]

### C.2.1 General

[diff.cpp23.general]

- <sup>1</sup> Subclause C.2 lists the differences between C++ and ISO C++ 2023, by the chapters of this document.

### C.2.2 Clause 5: lexical conventions

[diff.cpp23.lex]

- <sup>1</sup> Affected subclause: 5.8

**Change:** New operator `^^`.

**Rationale:** Required for new features.

**Effect on original feature:** Valid C++ 2023 code that contains two consecutive `^` tokens can be ill-formed in this revision of C++.

[Example 1:

```

struct C { int operator^(int); };
int operator^(int (C::*p)(int), C);
int i = &C::operator^^C{}; // ill-formed; previously well-formed
— end example]

```

- <sup>2</sup> Affected subclause: 5.12

**Change:** New keywords.

**Rationale:** Required for new features.

- (2.1) — The `contract_assert` keyword is added to introduce a contract assertion through an *assertion-statement* (8.9).

**Effect on original feature:** Valid C++ 2023 code using `contract_assert` as an identifier is not valid in this revision of C++.

### C.2.3 Clause 7: expressions

[diff.cpp23.expr]

- <sup>1</sup> Affected subclause: 7.4

**Change:** Operations mixing a value of an enumeration type and a value of a different enumeration type or of a floating-point type are no longer valid.

**Rationale:** Reinforcing type safety.

**Effect on original feature:** A valid C++ 2023 program that performs operations mixing a value of an enumeration type and a value of a different enumeration type or of a floating-point type is ill-formed.

[Example 1:

```

enum E1 { e };
enum E2 { f };
bool b = e <= 3.7; // ill-formed; previously well-formed
int k = f - e; // ill-formed; previously well-formed
auto x = true ? e : f; // ill-formed; previously well-formed
— end example]

```

- <sup>2</sup> Affected subclauses: 7.6.9 and 7.6.10

**Change:** Comparing two objects of array type is no longer valid.

**Rationale:** The old behavior was confusing since it compared not the contents of the two arrays, but their addresses.

**Effect on original feature:** A valid C++ 2023 program directly comparing two array objects is rejected as ill-formed in this document.

[Example 2:

```

int arr1[5];
int arr2[5];
bool same = arr1 == arr2; // ill-formed; previously well-formed

```

```

 bool idem = arr1 == +arr2; // compare addresses
 bool less = arr1 < +arr2; // compare addresses, unspecified result
— end example]

```

### 3 Affected subclause: 7.6.2.9

**Change:** Calling `delete` on a pointer to an incomplete class is ill-formed.

**Rationale:** Reduce undefined behavior.

**Effect on original feature:** A valid C++ 2023 program that calls `delete` on an incomplete class type is ill-formed.

[Example 3:

```

 struct S;

 void f(S *p) {
 delete p; // ill-formed; previously well-formed
 }

 struct S {};
— end example]

```

## C.2.4 Clause 9: declarations

[diff.cpp23.dcl.dcl]

### 1 Affected subclause: 9.5.5

**Change:** Pointer comparisons between `initializer_list` objects' backing arrays are unspecified.

**Rationale:** Permit the implementation to store backing arrays in static read-only memory.

**Effect on original feature:** Valid C++ 2023 code that relies on the result of pointer comparison between backing arrays may change behavior.

[Example 1:

```

 bool ne(std::initializer_list<int> a, std::initializer_list<int> b) {
 return a.begin() != b.begin() + 1;
 }
 bool b = ne({2,3}, {1,2,3}); // unspecified result; previously false
— end example]

```

### 2 Affected subclause: 9.3.4.5

**Change:** Previously, `T... [n]` would declare a pack of function parameters. `T... [n]` is now a *pack-index-specifier*.

**Rationale:** Improve the handling of packs.

**Effect on original feature:** Valid C++ 2023 code that declares a pack of parameters without specifying a *declarator-id* becomes ill-formed.

[Example 2:

```

 template <typename... T>
 void f(T... [1]);
 template <typename... T>
 void g(T... ptr[1]);
 int main() {
 f<int, double>(nullptr, nullptr); // ill-formed, previously void f<int, double>(int [1], double [1])
 g<int, double>(nullptr, nullptr); // ok
 }
— end example]

```

### 3 Affected subclause: 9.13.1

**Change:** New token `:]`.

**Rationale:** Required for new features.

**Effect on original feature:** Valid C++ 2023 code that contained an *attribute-specifier* with an *attribute-using-prefix* but no attributes and no whitespace is ill-formed in this revision of C++.

[Example 3:

```

 struct [[using CC:]] C; // ill-formed; previously well-formed
 struct [[using DD:]] D; // OK

```

— end example]

### C.2.5 Clause 13: templates

[diff.cpp23.temp]

#### <sup>1</sup> Affected subclause: 13.5

**Change:** Some atomic constraints become fold expanded constraints.

**Rationale:** Permit the subsumption of fold expressions.

**Effect on original feature:** Valid C++ 2023 code may become ill-formed.

[Example 1:

```
template <typename ...V> struct A;
struct S {
 static constexpr int compare(const S&) { return 1; }
};

template <typename ...T, typename ...U>
void f(A<T ...> *, A<U ...> *)
requires (T::compare(U{}) && ...); // was well-formed (atomic constraint of type bool),
 // now ill-formed (results in an atomic constraint of type int)

void g(A<S, S> *ap) {
 f(ap, ap);
}
```

— end example]

#### <sup>2</sup> Affected subclause: 13.10.3.2

**Change:** Template argument deduction from overload sets succeeds in more cases.

**Rationale:** Allow consideration of constraints to disambiguate overload sets used as parameters in function calls.

**Effect on original feature:** Valid C++ 2023 code may become ill-formed.

[Example 2:

```
template <typename T>
void f(T &&, void (*)(T &&));

void g(int &); // #1
inline namespace A {
 void g(short &&); // #2
}
inline namespace B {
 void g(short &&); // #3
}

void q() {
 int x;
 f(x, g); // ill-formed; previously well-formed, deducing T = int&
}
```

There is no change to the applicable deduction rules for the individual `g` candidates: Type deduction from #1 does not succeed; type deductions from #2 and #3 both succeed. — end example]

### C.2.6 Clause 15: preprocessing directives

[diff.cpp23.cpp]

#### <sup>1</sup> Affected subclause: 15.7.1

**Change:** Additional restrictions on macro names.

**Rationale:** Avoid hard to diagnose or non-portable constructs.

**Effect on original feature:** Keywords, names of identifiers with special meaning (5.11), and (unless otherwise specified) *attribute-tokens* specified in 9.13 may not be used as macro names. For example, valid C++ 2023 code that defines `post` or `pre` as macros is invalid in this revision of C++.

### C.2.7 Clause 16: library introduction

[diff.cpp23.library]

#### <sup>1</sup> Affected subclause: 16.4.2.3

**Change:** New headers.

**Rationale:** New functionality.

**Effect on original feature:** The following C++ headers are new: `<contracts>` (17.10), `<debugging>`

(19.7.2), `<hazard_pointer>` (32.11.3.2), `<hive>` (23.3.8), `<inplace_vector>` (23.3.15), `<linalg>` (29.9.2), `<meta>` (21.4.1), `<rcu>` (32.11.2.2), `<simd>` (29.10.3), `<stdbit.h>` (22.12), `<stdckdint.h>` (29.11.1), and `<text_encoding>` (28.4.1). Valid C++ 2023 code that `#includes` headers with these names may be invalid in this revision of C++.

## C.2.8 Clause 20: memory management library [diff.cpp23.mem]

### <sup>1</sup> Affected subclause: 20.2.12

**Change:** Calling `realloc` with a non-null pointer and zero size has erroneous behavior.

**Rationale:** The C standard library does not define this behavior.

**Effect on original feature:** Valid C++ 2023 code that calls `realloc` with a non-null pointer and a size of zero is erroneous and may change behavior.

## C.2.9 Clause 27: strings library [diff.cpp23.strings]

### <sup>1</sup> Affected subclause: 27.4.5

**Change:** Output of floating-point overloads of `to_string` and `to_wstring`.

**Rationale:** Prevent loss of information and improve consistency with other formatting facilities.

**Effect on original feature:** `to_string` and `to_wstring` function calls that take floating-point arguments may produce a different output.

[Example 1:

```
auto s = std::to_string(1e-7); // "1e-07"
 // previously "0.000000" with '.' possibly
 // changed according to the global C locale
```

— end example]

## C.2.10 Clause 31: input/output library [diff.cpp23.io]

### <sup>1</sup> Affected subclause: 31.7.5.4

**Change:** Overloaded `std::basic_istream<char, traits>::ignore`.

**Rationale:** Allow `char` values to be used as delimiters.

**Effect on original feature:** Calls to `istream::ignore` with a second argument of `char` type can change behavior. Calls to `istream::ignore` with a second argument that is neither `int` nor `char` type can become ill-formed.

[Example 1:

```
std::istringstream in("\xF0\x9F\xA4\xA1 Clown Face");
in.ignore(100, '\xA1'); // ignore up to '\xA1' delimiter,
 // previously might have ignored to EOF
in.ignore(100, -1L); // ambiguous overload,
 // previously equivalent to (int)-1L
```

— end example]

## C.2.11 Annex D: compatibility features [diff.cpp23.depr]

### <sup>1</sup> Change: Remove the type alias `allocator<T>::is_always_equal`.

**Rationale:** Non-empty allocator classes derived from `allocator` needed to explicitly define an `is_always_equal` member type so that `allocator_traits` would not use the one from the allocator base class.

**Effect on original feature:** It is simpler to correctly define an allocator class with an allocator base class.

[Example 1:

```
template <class T>
struct MyAlloc : allocator<T> {
 int tag;
};

static_assert(!allocator_traits<MyAlloc<int>>::is_always_equal); // Error in C++ 2023,
 // OK in C++ 2026
```

— end example]

### <sup>2</sup> Change: Removal of atomic access API for `shared_ptr` objects.

**Rationale:** The old behavior was brittle. `shared_ptr` objects using the old API were not protected by the type system, and certain interactions with code not using this API would, in some cases, silently produce

undefined behavior. A complete type-safe replacement is provided in the form of `atomic<shared_ptr<T>>`.

**Effect on original feature:** A valid C++ 2023 program that relies on the presence of the removed functions may fail to compile.

- 3 **Change:** Remove the `basic_string::reserve()` overload with no parameters.

**Rationale:** The overload of `reserve` with no parameters is redundant. The `shrink_to_fit` member function can be used instead.

**Effect on original feature:** A valid C++ 2023 program that calls `reserve()` on a `basic_string` object may fail to compile. The old functionality can be achieved by calling `shrink_to_fit()` instead, or the function call can be safely eliminated with no side effects.

- 4 **Change:** Remove header `<codecvt>` and all its contents.

**Rationale:** The header has been deprecated for the previous three editions of this document and no longer implements the current Unicode standard, supporting only the obsolete UCS-2 encoding. Ongoing support is at implementer's discretion, exercising freedoms granted by 16.4.5.3.2.

**Effect on original feature:** A valid C++ 2023 program `#include`-ing the header or importing the header unit may fail to compile. Code that uses any of the following names by importing the standard library modules may fail to compile:

- (4.1) — `codecvt_mode`,
- (4.2) — `codecvt_utf16`,
- (4.3) — `codecvt_utf8`,
- (4.4) — `codecvt_utf8_utf16`,
- (4.5) — `consume_header`,
- (4.6) — `generate_header`, and
- (4.7) — `little_endian`.

- 5 **Change:** Remove header `<strstream>` and all its contents.

**Rationale:** The header has been deprecated since the original C++ standard; the `<spanstream>` header provides an updated, safer facility. Ongoing support is at implementer's discretion, exercising freedoms granted by 16.4.5.3.2.

**Effect on original feature:** A valid C++ 2023 program `#include`-ing the header or importing the header unit may become ill-formed. Code that uses any of the following classes by importing one of the standard library modules may become ill-formed:

- (5.1) — `istrstream`
- (5.2) — `ostrstream`
- (5.3) — `strstream`
- (5.4) — `strstreambuf`

- 6 **Change:** Remove convenience interfaces `wstring_convert` and `wbuffer_convert`.

**Rationale:** These features were underspecified with no clear error reporting mechanism and were deprecated for the last three editions of this document. Ongoing support is at implementer's discretion, exercising freedoms granted by 16.4.5.3.2.

**Effect on original feature:** A valid C++ 2023 program using these interfaces may become ill-formed.

### C.3 C++ and ISO C++ 2020

[diff.cpp20]

#### C.3.1 General

[diff.cpp20.general]

- 1 Subclause C.3 lists the differences between C++ and ISO C++ 2020, in addition to those listed above, by the chapters of this document.

#### C.3.2 Clause 5: lexical conventions

[diff.cpp20.lex]

- 1 **Affected subclause:** 5.11

**Change:** Previously valid identifiers containing characters not present in UAX #44 properties `XID_Start` or `XID_Continue`, or not in Normalization Form C, are now rejected.

**Rationale:** Prevent confusing characters in identifiers. Requiring normalization of names ensures consistent linker behavior.

**Effect on original feature:** Some identifiers are no longer well-formed.

<sup>2</sup> **Affected subclause:** 5.13.5

**Change:** Concatenated *string-literals* can no longer have conflicting *encoding-prefixes*.

**Rationale:** Removal of unimplemented conditionally-supported feature.

**Effect on original feature:** Concatenation of *string-literals* with different *encoding-prefixes* is now ill-formed.

[Example 1:

```
 auto c = L"a" U"b"; // was conditionally-supported; now ill-formed
— end example]
```

### C.3.3 Clause 7: expressions

[diff.cpp20.expr]

<sup>1</sup> **Affected subclause:** 7.5.5.2

**Change:** Change move-eligible *id-expressions* from lvalues to xvalues.

**Rationale:** Simplify the rules for implicit move.

**Effect on original feature:** Valid C++ 2020 code that relies on a returned *id-expression*'s being an lvalue may change behavior or fail to compile.

[Example 1:

```
 decltype(auto) f(int&& x) { return (x); } // returns int&&; previously returned int&
 int& g(int&& x) { return x; } // ill-formed; previously well-formed
— end example]
```

<sup>2</sup> **Affected subclause:** 7.6.1.2

**Change:** Change the meaning of comma in subscript expressions.

**Rationale:** Enable repurposing a deprecated syntax to support multidimensional indexing.

**Effect on original feature:** Valid C++ 2020 code that uses a comma expression within a subscript expression may fail to compile.

[Example 2:

```
 arr[1, 2] // was equivalent to arr[(1, 2)],
 // now equivalent to arr.operator[](1, 2) or ill-formed
— end example]
```

### C.3.4 Clause 8: statements

[diff.cpp20.stmt]

<sup>1</sup> **Affected subclause:** 8.6.5

**Change:** The lifetime of temporary objects in the *for-range-initializer* is extended until the end of the loop (6.8.7).

**Rationale:** Improve usability of the range-based *for* statement.

**Effect on original feature:** Destructors of some temporary objects are invoked later.

[Example 1:

```
 void f() {
 std::vector<int> v = { 42, 17, 13 };
 std::mutex m;

 for (int x :
 static_cast<void>(std::lock_guard<std::mutex>(m)), v) { // lock released in C++ 2020
 std::lock_guard<std::mutex> guard(m); // OK in C++ 2020, now deadlocks
 }
 }
— end example]
```

### C.3.5 Clause 9: declarations

[diff.cpp20.dcl]

<sup>1</sup> **Affected subclause:** 9.5.3

**Change:** UTF-8 string literals may initialize arrays of *char* or *unsigned char*.

**Rationale:** Compatibility with previously written code that conformed to previous versions of this document.

**Effect on original feature:** Arrays of *char* or *unsigned char* may now be initialized with a UTF-8 string literal. This can affect initialization that includes arrays that are directly initialized within class types, typically aggregates.

[Example 1:

```
struct A {
 char8_t s[10];
};
struct B {
 char s[10];
};

void f(A);
void f(B);

int main() {
 f({u8""}); // ambiguous
}
```

— end example]

### C.3.6 Clause 12: overloading

[diff.cpp20.over]

#### <sup>1</sup> Affected subclause: 12.2.2.9

**Change:** Deducing class template arguments from inherited constructors.

**Rationale:** Making class template argument deduction consistent with construction.

**Effect on original feature:** Valid ISO C++ 2020 code may become ill-formed or change meaning in this revision of C++.

[Example 1:

```
template<typename T>
struct B {
 B(T);
};

template<typename T>
struct D : B<T *> {
 D(T);
 using B<T *>::B;
 T m = "a"; // #1
};

D d(""); // ill-formed at #1 (T deduced as const char);
 // previously well-formed (T deduced as const char*)
```

— end example]

### C.3.7 Clause 13: templates

[diff.cpp20.temp]

#### <sup>1</sup> Affected subclause: 13.10.3.6

**Change:** Deducing template arguments from exception specifications.

**Rationale:** Facilitate generic handling of throwing and non-throwing functions.

**Effect on original feature:** Valid ISO C++ 2020 code may be ill-formed in this revision of C++.

[Example 1:

```
template<bool> struct A { };
template<bool B> void f(void (*) (A) noexcept(B));
void g(A<false>) noexcept;
void h() {
 f(g); // ill-formed; previously well-formed
}
```

— end example]

### C.3.8 Clause 16: library introduction

[diff.cpp20.library]

#### <sup>1</sup> Affected subclause: 16.4.2.3

**Change:** New headers.

**Rationale:** New functionality.

**Effect on original feature:** The following C++ headers are new: `<expected>` (22.8.2), `<flat_map>` (23.6.7),



<flat\_set> (23.6.10), <generator> (25.8.2), <mdspan> (23.7.3.2), <print> (31.7.4), <spanstream> (31.9.2), <stacktrace> (19.6.2), <stdatomic.h> (32.5.12), and <stdfloat> (17.4.2). Valid C++ 2020 code that #includes headers with these names may be invalid in this revision of C++.

### C.3.9 Clause 18: concepts library

[diff.cpp20.concepts]

#### <sup>1</sup> Affected subclauses: 17.12.4, 18.5.4, and 18.5.5

**Change:** Replace `common_reference_with` in `three_way_comparable_with`, `equality_comparable_with`, and `totally_ordered_with` with an exposition-only concept.

**Rationale:** Allow uncopyable, but movable, types to model these concepts.

**Effect on original feature:** Valid C++ 2020 code relying on subsumption with `common_reference_with` may fail to compile in this revision of C++.

[Example 1:

```
template<class T, class U>
 requires equality_comparable_with<T, U>
bool attempted_equals(const T& t, const U& u); // previously selected overload

template<class T, class U>
 requires common_reference_with<const remove_reference_t<T>&, const remove_reference_t<U>&>
bool attempted_equals(const T& t, const U& u); // ambiguous overload; previously
 // rejected by partial ordering

bool test(shared_ptr<int> p) {
 return attempted_equals(p, nullptr); // ill-formed; previously well-formed
}
```

— end example]

### C.3.10 Clause 20: memory management library

[diff.cpp20.memory]

#### <sup>1</sup> Affected subclause: 20.2.9.1

**Change:** Forbid partial and explicit program-defined specializations of `allocator_traits`.

**Rationale:** Allow addition of `allocate_at_least` to `allocator_traits`, and potentially other members in the future.

**Effect on original feature:** Valid C++ 2020 code that partially or explicitly specializes `allocator_traits` is ill-formed with no diagnostic required in this revision of C++.

### C.3.11 Clause 22: general utilities library

[diff.cpp20.utilities]

#### <sup>1</sup> Affected subclause: 28.5

**Change:** Signature changes: `format`, `format_to`, `vformat_to`, `format_to_n`, `formatted_size`. Removal of `format_args_t`.

**Rationale:** Improve safety via compile-time format string checks, avoid unnecessary template instantiations.

**Effect on original feature:** Valid C++ 2020 code that contained errors in format strings or relied on previous format string signatures or `format_args_t` may become ill-formed.

[Example 1:

```
auto s = std::format("{:d}", "I am not a number"); // ill-formed,
 // previously threw format_error
```

— end example]

#### <sup>2</sup> Affected subclause: 28.5

**Change:** Signature changes: `format`, `format_to`, `format_to_n`, `formatted_size`.

**Rationale:** Enable formatting of views that do not support iteration when const-qualified and that are not copyable.

**Effect on original feature:** Valid C++ 2020 code that passes bit-fields to formatting functions may become ill-formed.

[Example 2:

```
struct tiny {
 int bit: 1;
};
```

```

 auto t = tiny();
 std::format("{:}", t.bit); // ill-formed, previously returned "0"
— end example]

```

### 3 Affected subclause: 28.5.2.2

**Change:** Restrict types of formatting arguments used as *width* or *precision* in a *std-format-spec*.

**Rationale:** Disallow types that do not have useful or portable semantics as a formatting width or precision.

**Effect on original feature:** Valid C++ 2020 code that passes a boolean or character type as *arg-id* becomes invalid.

[Example 3:

```

 std::format("{:.*^{}}", "", true); // ill-formed, previously returned "*"
 std::format("{:.*^{}}", "", '1'); // ill-formed, previously returned an
 // implementation-defined number of '*' characters
— end example]

```

### 4 Affected subclause: 28.5.6.4

**Change:** Removed the `formatter` specialization:

```

template<size_t N> struct formatter<const charT[N], charT>;

```

**Rationale:** The specialization is inconsistent with the design of `formatter`, which is intended to be instantiated only with cv-unqualified object types.

**Effect on original feature:** Valid C++ 2020 code that instantiated the removed specialization can become ill-formed.

## C.3.12 Clause 27: strings library

[diff.cpp20.strings]

### 1 Affected subclause: 27.4

**Change:** Additional rvalue overload for the `substr` member function and the corresponding constructor.

**Rationale:** Improve efficiency of operations on rvalues.

**Effect on original feature:** Valid C++ 2020 code that created a substring by calling `substr` (or the corresponding constructor) on an xvalue expression with type `S` that is a specialization of `basic_string` may change meaning in this revision of C++.

[Example 1:

```

 std::string s1 = "some long string that forces allocation", s2 = s1;
 std::move(s1).substr(10, 5);
 assert(s1 == s2); // unspecified, previously guaranteed to be true
 std::string s3(std::move(s2), 10, 5);
 assert(s1 == s2); // unspecified, previously guaranteed to be true
— end example]

```

## C.3.13 Clause 23: containers library

[diff.cpp20.containers]

### 1 Affected subclauses: 23.2.7 and 23.2.8

**Change:** Heterogeneous `extract` and `erase` overloads for associative containers.

**Rationale:** Improve efficiency of erasing elements from associative containers.

**Effect on original feature:** Valid C++ 2020 code may fail to compile in this revision of C++.

[Example 1:

```

 struct B {
 auto operator<=>(const B&) const = default;
 };

 struct D : private B {
 void f(std::set<B, std::less<>>& s) {
 s.erase(*this); // ill-formed; previously well-formed
 }
 };
— end example]

```

**C.3.14 Clause 32: concurrency support library****[diff.cpp20.thread]**<sup>1</sup> **Affected subclause:** 32.9.3

**Change:** In this revision of C++, it is implementation-defined whether a barrier's phase completion step runs if no thread calls `wait`. Previously the phase completion step was guaranteed to run on the last thread that calls `arrive` or `arrive_and_drop` during the phase. In this revision of C++, it can run on any of the threads that arrived or waited at the barrier during the phase.

**Rationale:** Correct contradictory wording and improve implementation flexibility for performance.

**Effect on original feature:** Valid C++ 2020 code using a barrier might have different semantics in this revision of C++ if it depends on a completion function's side effects occurring exactly once, on a specific thread running the phase completion step, or on a completion function's side effects occurring without `wait` having been called.

[Example 1:

```
auto b0 = std::barrier(1);
b0.arrive();
b0.arrive(); // implementation-defined; previously well-defined

int data = 0;
auto b1 = std::barrier(1, [&] { data++; });
b1.arrive();
assert(data == 1); // implementation-defined; previously well-defined
b1.arrive(); // implementation-defined; previously well-defined
```

— end example]

**C.4 C++ and ISO C++ 2017****[diff.cpp17]****C.4.1 General****[diff.cpp17.general]**

- <sup>1</sup> Subclause C.4 lists the differences between C++ and ISO C++ 2017, in addition to those listed above, by the chapters of this document.

**C.4.2 Clause 5: lexical conventions****[diff.cpp17.lex]**<sup>1</sup> **Affected subclauses:** 5.5, 10.1, 10.3, 15.1, 15.5, and 15.6

**Change:** New identifiers with special meaning.

**Rationale:** Required for new features.

**Effect on original feature:** Logical lines beginning with `module` or `import` may be interpreted differently in this revision of C++.

[Example 1:

```
class module {};
module m1; // was variable declaration; now module-declaration
module *m2; // variable declaration

class import {};
import j1; // was variable declaration; now module-import-declaration
::import j2; // variable declaration
```

— end example]

<sup>2</sup> **Affected subclause:** 5.6

**Change:** `header-name` tokens are formed in more contexts.

**Rationale:** Required for new features.

**Effect on original feature:** When the identifier `import` is followed by a `<` character, a `header-name` token may be formed.

[Example 2:

```
template<typename> class import {};
import<int> f(); // ill-formed; previously well-formed
::import<int> g(); // OK
```

— end example]

<sup>3</sup> **Affected subclause:** 5.12

**Change:** New keywords.

**Rationale:** Required for new features.

- (3.1) — The `char8_t` keyword is added to differentiate the types of ordinary and UTF-8 literals (5.13.5).
- (3.2) — The `concept` keyword is added to enable the definition of concepts (13.7.9).
- (3.3) — The `constexpr` keyword is added to declare immediate functions (9.2.6).
- (3.4) — The `constinit` keyword is added to prevent unintended dynamic initialization (9.2.7).
- (3.5) — The `co_await`, `co_yield`, and `co_return` keywords are added to enable the definition of coroutines (9.6.4).
- (3.6) — The `requires` keyword is added to introduce constraints through a *requires-clause* (13.1) or a *requires-expression* (7.5.8).

**Effect on original feature:** Valid C++ 2017 code using `char8_t`, `concept`, `constexpr`, `constinit`, `co_await`, `co_yield`, `co_return`, or `requires` as an identifier is not valid in this revision of C++.

#### 4 Affected subclause: 5.8

**Change:** New operator `<=>`.

**Rationale:** Necessary for new functionality.

**Effect on original feature:** Valid C++ 2017 code that contains a `<=` token immediately followed by a `>` token may be ill-formed or have different semantics in this revision of C++.

[Example 3:

```
namespace N {
 struct X {};
 bool operator<=(X, X);
 template<bool(X, X)> struct Y {};
 Y<operator<=> y; // ill-formed; previously well-formed
}
```

— end example]

#### 5 Affected subclause: 5.13

**Change:** Type of UTF-8 string and character literals.

**Rationale:** Required for new features. The changed types enable function overloading, template specialization, and type deduction to distinguish ordinary and UTF-8 string and character literals.

**Effect on original feature:** Valid C++ 2017 code that depends on UTF-8 string literals having type “array of `const char`” and UTF-8 character literals having type “`char`” is not valid in this revision of C++.

[Example 4:

```
const auto *u8s = u8"text"; // u8s previously deduced as const char*; now deduced as const char8_t*
const char *ps = u8s; // ill-formed; previously well-formed

auto u8c = u8'c'; // u8c previously deduced as char; now deduced as char8_t
char *pc = &u8c; // ill-formed; previously well-formed

std::string s = u8"text"; // ill-formed; previously well-formed

void f(const char *s);
f(u8"text"); // ill-formed; previously well-formed

template<typename> struct ct;
template<> struct ct<char> {
 using type = char;
};
ct<decltype(u8'c')>::type x; // ill-formed; previously well-formed.
```

— end example]

### C.4.3 Clause 6: basics

[diff.cpp17.basic]

#### 1 Affected subclause: 6.8.4

**Change:** A pseudo-destructor call ends the lifetime of the object to which it is applied.

**Rationale:** Increase consistency of the language model.

**Effect on original feature:** Valid ISO C++ 2017 code may be ill-formed or have undefined behavior in this revision of C++.

[Example 1:

```
int f() {
 int a = 123;
 using T = int;
 a.~T();
 return a; // undefined behavior; previously returned 123
}
```

— end example]

## <sup>2</sup> Affected subclause: 6.10.2.2

**Change:** Except for the initial release operation, a release sequence consists solely of atomic read-modify-write operations.

**Rationale:** Removal of rarely used and confusing feature.

**Effect on original feature:** If a `memory_order_release` atomic store is followed by a `memory_order_relaxed` store to the same variable by the same thread, then reading the latter value with a `memory_order_acquire` load no longer provides any “happens before” guarantees, even in the absence of intervening stores by another thread.

### C.4.4 Clause 7: expressions

[diff.cpp17.expr]

## <sup>1</sup> Affected subclause: 7.5.6.3

**Change:** Implicit lambda capture may capture additional entities.

**Rationale:** Rule simplification, necessary to resolve interactions with `constexpr` if.

**Effect on original feature:** Lambdas with a *capture-default* may capture local entities that were not captured in C++ 2017 if those entities are only referenced in contexts that do not result in an odr-use.

### C.4.5 Clause 9: declarations

[diff.cpp17.dcl.dcl]

## <sup>1</sup> Affected subclause: 9.2.4

**Change:** Unnamed classes with a typedef name for linkage purposes can contain only C-compatible constructs.

**Rationale:** Necessary for implementability.

**Effect on original feature:** Valid C++ 2017 code may be ill-formed in this revision of C++.

[Example 1:

```
typedef struct {
 void f() {} // ill-formed; previously well-formed
} S;
```

— end example]

## <sup>2</sup> Affected subclause: 9.3.4.7

**Change:** A function cannot have different default arguments in different translation units.

**Rationale:** Required for modules support.

**Effect on original feature:** Valid C++ 2017 code may be ill-formed in this revision of C++, with no diagnostic required.

[Example 2:

```
// Translation unit 1
int f(int a = 42);
int g() { return f(); }

// Translation unit 2
int f(int a = 76) { return a; } // ill-formed, no diagnostic required; previously well-formed
int g();
int main() { return g(); } // used to return 42
```

— end example]

## <sup>3</sup> Affected subclause: 9.5.2

**Change:** A class that has user-declared constructors is never an aggregate.

**Rationale:** Remove potentially error-prone aggregate initialization which may apply notwithstanding the declared constructors of a class.

**Effect on original feature:** Valid C++ 2017 code that aggregate-initializes a type with a user-declared constructor may be ill-formed or have different semantics in this revision of C++.

[Example 3:

```

struct A { // not an aggregate; previously an aggregate
 A() = delete;
};

struct B { // not an aggregate; previously an aggregate
 B() = default;
 int i = 0;
};

struct C { // not an aggregate; previously an aggregate
 C(C&&) = default;
 int a, b;
};

A a{}; // ill-formed; previously well-formed
B b = {1}; // ill-formed; previously well-formed
auto* c = new C{2, 3}; // ill-formed; previously well-formed

struct Y;

struct X {
 operator Y();
};

struct Y { // not an aggregate; previously an aggregate
 Y(const Y&) = default;
 X x;
};

Y y{X{}}; // copy constructor call; previously aggregate-initialization
— end example]
```

#### 4 Affected subclause: 9.5.5

**Change:** Boolean conversion from a pointer or pointer-to-member type is now a narrowing conversion.

**Rationale:** Catches bugs.

**Effect on original feature:** Valid C++ 2017 code may fail to compile in this revision of C++.

[Example 4:

```

bool y[] = { "bc" }; // ill-formed; previously well-formed
— end example]
```

### C.4.6 Clause 11: classes

[diff.cpp17.class]

#### 1 Affected subclauses: 11.4.5 and 11.4.8.3

**Change:** The class name can no longer be used parenthesized immediately after an `explicit decl-specifier` in a constructor declaration. The *conversion-function-id* can no longer be used parenthesized immediately after an `explicit decl-specifier` in a conversion function declaration.

**Rationale:** Necessary for new functionality.

**Effect on original feature:** Valid C++ 2017 code may fail to compile in this revision of C++.

[Example 1:

```

struct S {
 explicit (S)(const S&); // ill-formed; previously well-formed
 explicit (operator int)(); // ill-formed; previously well-formed
 explicit(true) (S)(int); // OK
};
— end example]
```

#### 2 Affected subclauses: 11.4.5 and 11.4.7

**Change:** A *simple-template-id* is no longer valid as the *declarator-id* of a constructor or destructor.

**Rationale:** Remove potentially error-prone option for redundancy.

**Effect on original feature:** Valid C++ 2017 code may fail to compile in this revision of C++.

[Example 2:

```
template<class T>
struct A {
 A<T>(); // error: simple-template-id not allowed for constructor
 A(int); // OK, injected-class-name used
 ~A<T>(); // error: simple-template-id not allowed for destructor
};
```

— end example]

### <sup>3</sup> Affected subclause: 11.9.6

**Change:** A function returning an implicitly movable entity may invoke a constructor taking an rvalue reference to a type different from that of the returned expression. Function and catch-clause parameters can be thrown using move constructors.

**Rationale:** Side effect of making it easier to write more efficient code that takes advantage of moves.

**Effect on original feature:** Valid C++ 2017 code may fail to compile or have different semantics in this revision of C++.

[Example 3:

```
struct base {
 base();
 base(base const &);
private:
 base(base &&);
};

struct derived : base {};

base f(base b) {
 throw b; // error: base(base &&) is private
 derived d;
 return d; // error: base(base &&) is private
}

struct S {
 S(const char *s) : m(s) { }
 S(const S&) = default;
 S(S&& other) : m(other.m) { other.m = nullptr; }
 const char * m;
};

S consume(S&& s) { return s; }

void g() {
 S s("text");
 consume(static_cast<S&&>(s));
 char c = *s.m; // undefined behavior; previously ok
}
```

— end example]

## C.4.7 Clause 12: overloading

[diff.cpp17.over]

### <sup>1</sup> Affected subclause: 12.2.2.3

**Change:** Equality and inequality expressions can now find reversed and rewritten candidates.

**Rationale:** Improve consistency of equality with three-way comparison and make it easier to write the full complement of equality operations.

**Effect on original feature:** For certain pairs of types where one is convertible to the other, equality or inequality expressions between an object of one type and an object of the other type invoke a different operator. Also, for certain types, equality or inequality expressions between two objects of that type become ambiguous.

[Example 1:

```

struct A {
 operator int() const;
};

bool operator==(A, int); // #1
// #2 is built-in candidate: bool operator==(int, int);
// #3 is built-in candidate: bool operator!=(int, int);

int check(A x, A y) {
 return (x == y) + // ill-formed; previously well-formed
 (10 == x) + // calls #1, previously selected #2
 (10 != x); // calls #1, previously selected #3
}

```

— end example]

## <sup>2</sup> Affected subclause: 12.2.2.3

**Change:** Overload resolution may change for equality operators (7.6.10).

**Rationale:** Support calling `operator==` with reversed order of arguments.

**Effect on original feature:** Valid C++ 2017 code that uses equality operators with conversion functions may be ill-formed or have different semantics in this revision of C++.

[Example 2:

```

struct A {
 operator int() const { return 10; }
};

bool operator==(A, int); // #1
// #2 is built-in candidate: bool operator==(int, int);
bool b = 10 == A(); // calls #1 with reversed order of arguments; previously selected #2

struct B {
 bool operator==(const B&); // member function with no cv-qualifier
};
B b1;
bool eq = (b1 == b1); // ambiguous; previously well-formed

```

— end example]

## C.4.8 Clause 13: templates

[diff.cpp17.temp]

### <sup>1</sup> Affected subclause: 13.3

**Change:** An *unqualified-id* that is followed by a `<` and for which name lookup finds nothing or finds a function will be treated as a *template-name* in order to potentially cause argument-dependent lookup to be performed.

**Rationale:** It was problematic to call a function template with an explicit template argument list via argument-dependent lookup because of the need to have a template with the same name visible via normal lookup.

**Effect on original feature:** Previously valid code that uses a function name as the left operand of a `<` operator would become ill-formed.

[Example 1:

```

struct A {};
bool operator<(void (*fp)(), A);
void f() {}
int main() {
 A a;
 f < a; // ill-formed; previously well-formed
 (f) < a; // still well-formed
}

```

— end example]

## C.4.9 Clause 14: exception handling

[diff.cpp17.except]

### <sup>1</sup> Affected subclause: 14.5

**Change:** Remove `throw()` exception specification.



**Rationale:** Removal of obsolete feature that has been replaced by `noexcept`.

**Effect on original feature:** A valid C++ 2017 function declaration, member function declaration, function pointer declaration, or function reference declaration that uses `throw()` for its exception specification will be rejected as ill-formed in this revision of C++. It should simply be replaced with `noexcept` for no change of meaning since C++ 2017.

[*Note 1:* There is no way to write a function declaration that is non-throwing in this revision of C++ and is also non-throwing in C++ 2003 except by using the preprocessor to generate a different token sequence in each case. — *end note*]

#### C.4.10 Clause 16: library introduction

[diff.cpp17.library]

##### <sup>1</sup> Affected subclause: 16.4.2.3

**Change:** New headers.

**Rationale:** New functionality.

**Effect on original feature:** The following C++ headers are new: `<barrier>` (32.9.3.2), `<bit>` (22.11.2), `<charconv>` (28.2.1), `<compare>` (17.12.1), `<concepts>` (18.3), `<coroutine>` (17.13.2), `<format>` (28.5.1), `<latch>` (32.9.2.2), `<numbers>` (29.8.1), `<ranges>` (25.2), `<semaphore>` (32.8.2), `<source_location>` (17.8.1), `<span>` (23.7.2.1), `<stop_token>` (32.3.2), `<syncstream>` (31.11.1), and `<version>` (17.3.1). Valid C++ 2017 code that `#includes` headers with these names may be invalid in this revision of C++.

##### <sup>2</sup> Affected subclause: 16.4.2.3

**Change:** Remove vacuous C++ header files.

**Rationale:** The empty headers implied a false requirement to achieve C compatibility with the C++ headers.

**Effect on original feature:** A valid C++ 2017 program that `#includes` any of the following headers may fail to compile: `<ccomplex>`, `<ciso646>`, `<cstdalign>`, `<cstdbool>`, and `<ctgmath>`. To retain the same behavior:

- (2.1) — a `#include` of `<ccomplex>` can be replaced by a `#include` of `<complex>` (29.4.2),
- (2.2) — a `#include` of `<ctgmath>` can be replaced by a `#include` of `<cmath>` (29.7.1) and a `#include` of `<complex>`, and
- (2.3) — a `#include` of `<ciso646>`, `<cstdalign>`, or `<cstdbool>` can simply be removed.

#### C.4.11 Clause 23: containers library

[diff.cpp17.containers]

##### <sup>1</sup> Affected subclauses: 23.3.7 and 23.3.11

**Change:** Return types of `remove`, `remove_if`, and `unique` changed from `void` to `container::size_type`.

**Rationale:** Improve efficiency and convenience of finding number of removed elements.

**Effect on original feature:** Code that depends on the return types might have different semantics in this revision of C++. Translation units compiled against this version of C++ may be incompatible with translation units compiled against C++ 2017, either failing to link or having undefined behavior.

#### C.4.12 Clause 24: iterators library

[diff.cpp17.iterators]

##### <sup>1</sup> Affected subclause: 24.3.2.3

**Change:** The specialization of `iterator_traits` for `void*` and for function pointer types no longer contains any nested typedefs.

**Rationale:** Corrects an issue misidentifying pointer types that are not incrementable as iterator types.

**Effect on original feature:** A valid C++ 2017 program that relies on the presence of the typedefs may fail to compile, or have different behavior.

#### C.4.13 Clause 26: algorithms library

[diff.cpp17.alg.reqs]

##### <sup>1</sup> Affected subclause: 26.2

**Change:** The number and order of deducible template parameters for algorithm declarations is now unspecified, instead of being as-declared.

**Rationale:** Increase implementor freedom and allow some function templates to be implemented as function objects with templated call operators.

**Effect on original feature:** A valid C++ 2017 program that passes explicit template arguments to algorithms not explicitly specified to allow such in this version of C++ may fail to compile or have undefined behavior.

**C.4.14 Clause 31: input/output library****[diff.cpp17.input.output]****1 Affected subclause: 31.7.5.3.3****Change:** Character array extraction only takes array types.**Rationale:** Increase safety via preventing buffer overflow at compile time.**Effect on original feature:** Valid C++ 2017 code may fail to compile in this revision of C++.*[Example 1:*

```

auto p = new char[100];
char q[100];
std::cin >> std::setw(20) >> p; // ill-formed; previously well-formed
std::cin >> std::setw(20) >> q; // OK

```

*— end example]***2 Affected subclause: 31.7.6.3.4****Change:** Overload resolution for ostream inserters used with UTF-8 literals.**Rationale:** Required for new features.**Effect on original feature:** Valid C++ 2017 code that passes UTF-8 literals to `basic_ostream<char, ...>::operator<<` or `basic_ostream<wchar_t, ...>::operator<<` is now ill-formed.*[Example 2:*

```

std::cout << u8"text"; // previously called operator<<(const char*) and printed a string;
 // now ill-formed
std::cout << u8'X'; // previously called operator<<(char) and printed a character;
 // now ill-formed

```

*— end example]***3 Affected subclause: 31.7.6.3.4****Change:** Overload resolution for ostream inserters used with `wchar_t`, `char16_t`, or `char32_t` types.**Rationale:** Removal of surprising behavior.**Effect on original feature:** Valid C++ 2017 code that passes `wchar_t`, `char16_t`, or `char32_t` characters or strings to `basic_ostream<char, ...>::operator<<` or that passes `char16_t` or `char32_t` characters or strings to `basic_ostream<wchar_t, ...>::operator<<` is now ill-formed.*[Example 3:*

```

std::cout << u"text"; // previously formatted the string as a pointer value;
 // now ill-formed
std::cout << u'X'; // previously formatted the character as an integer value;
 // now ill-formed

```

*— end example]***4 Affected subclause: 31.12.6****Change:** Return type of filesystem path format observer member functions.**Rationale:** Required for new features.**Effect on original feature:** Valid C++ 2017 code that depends on the `u8string()` and `generic_u8string()` member functions of `std::filesystem::path` returning `std::string` is not valid in this revision of C++.*[Example 4:*

```

std::filesystem::path p;
std::string s1 = p.u8string(); // ill-formed; previously well-formed
std::string s2 = p.generic_u8string(); // ill-formed; previously well-formed

```

*— end example]***C.4.15 Annex D: compatibility features****[diff.cpp17.depr]****1 Change:** Remove `uncaught_exception`.**Rationale:** The function did not have a clear specification when multiple exceptions were active, and has been superseded by `uncaught_exceptions`.**Effect on original feature:** A valid C++ 2017 program that calls `std::uncaught_exception` may fail to compile. It can be revised to use `std::uncaught_exceptions` instead, for clear and portable semantics.**2 Change:** Remove support for adaptable function API.**Rationale:** The deprecated support relied on a limited convention that could not be extended to support the general case or new language features. It has been superseded by direct language support with `decltype`,

and by the `std::bind` and `std::not_fn` function templates.

**Effect on original feature:** A valid C++ 2017 program that relies on the presence of `result_type`, `argument_type`, `first_argument_type`, or `second_argument_type` in a standard library class may fail to compile. A valid C++ 2017 program that calls `not1` or `not2`, or uses the class templates `unary_negate` or `binary_negate`, may fail to compile.

- <sup>3</sup> **Change:** Remove redundant members from `std::allocator`.

**Rationale:** `std::allocator` was overspecified, encouraging direct usage in user containers rather than relying on `std::allocator_traits`, leading to poor containers.

**Effect on original feature:** A valid C++ 2017 program that directly makes use of the `pointer`, `const_pointer`, `reference`, `const_reference`, `rebind`, `address`, `construct`, `destroy`, or `max_size` members of `std::allocator`, or that directly calls `allocate` with an additional hint argument, may fail to compile.

- <sup>4</sup> **Change:** Remove `raw_storage_iterator`.

**Rationale:** The iterator encouraged use of potentially-throwing algorithms, but did not return the number of elements successfully constructed, as would be necessary to destroy them.

**Effect on original feature:** A valid C++ 2017 program that uses this iterator class may fail to compile.

- <sup>5</sup> **Change:** Remove temporary buffers API.

**Rationale:** The temporary buffer facility was intended to provide an efficient optimization for small memory requests, but there is little evidence this was achieved in practice, while requiring the user to provide their own exception-safe wrappers to guard use of the facility in many cases.

**Effect on original feature:** A valid C++ 2017 program that calls `get_temporary_buffer` or `return_temporary_buffer` may fail to compile.

- <sup>6</sup> **Change:** Remove `shared_ptr::unique`.

**Rationale:** The result of a call to this member function is not reliable in the presence of multiple threads and weak pointers. The member function `use_count` is similarly unreliable, but has a clearer contract in such cases, and remains available for well-defined use in single-threaded cases.

**Effect on original feature:** A valid C++ 2017 program that calls `unique` on a `shared_ptr` object may fail to compile.

- <sup>7</sup> **Affected subclause:** [D.13](#)

**Change:** Remove deprecated type traits.

**Rationale:** The traits had unreliable or awkward interfaces. The `is_literal_type` trait provided no way to detect which subset of member functions of a type were declared `constexpr`. The `result_of` trait had a surprising syntax that did not directly support function types. It has been superseded by the `invoke_result` trait.

**Effect on original feature:** A valid C++ 2017 program that relies on the `is_literal_type` or `result_of` type traits, on the `is_literal_type_v` variable template, or on the `result_of_t` alias template may fail to compile.

## C.5 C++ and ISO C++ 2014

[diff.cpp14]

### C.5.1 General

[diff.cpp14.general]

- <sup>1</sup> Subclause [C.5](#) lists the differences between C++ and ISO C++ 2014, in addition to those listed above, by the chapters of this document.

### C.5.2 Clause 5: lexical conventions

[diff.cpp14.lex]

- <sup>1</sup> **Affected subclause:** [5.2](#)

**Change:** Removal of trigraph support as a required feature.

**Rationale:** Prevents accidental uses of trigraphs in non-raw string literals and comments.

**Effect on original feature:** Valid C++ 2014 code that uses trigraphs may not be valid or may have different semantics in this revision of C++. Implementations may choose to translate trigraphs as specified in C++ 2014 if they appear outside of a raw string literal, as part of the implementation-defined mapping from input source file characters to the translation character set.

- <sup>2</sup> **Affected subclause:** [5.7](#)

**Change:** *pp-number* can contain *p sign* and *P sign*.

**Rationale:** Necessary to enable *hexadecimal-floating-point-literals*.

**Effect on original feature:** Valid C++ 2014 code may fail to compile or produce different results in this revision of C++. Specifically, character sequences like `0p+0` and `0e1_p+0` are three separate tokens each in C++ 2014, but one single token in this revision of C++.

[Example 1:

```
#define F(a) b ## a
int b0p = F(0p+0); // ill-formed; equivalent to "int b0p = b0p + 0;" in C++ 2014
— end example]
```

### C.5.3 Clause 7: expressions

[diff.cpp14.expr]

#### <sup>1</sup> Affected subclauses: 7.6.1.6 and 7.6.2.3

**Change:** Remove increment operator with `bool` operand.

**Rationale:** Obsolete feature with occasionally surprising semantics.

**Effect on original feature:** A valid C++ 2014 expression utilizing the increment operator on a `bool` lvalue is ill-formed in this revision of C++.

#### <sup>2</sup> Affected subclauses: 7.6.2.8 and 7.6.2.9

**Change:** Dynamic allocation mechanism for over-aligned types.

**Rationale:** Simplify use of over-aligned types.

**Effect on original feature:** In C++ 2014 code that uses a *new-expression* to allocate an object with an over-aligned class type, where that class has no allocation functions of its own, `::operator new(std::size_t)` is used to allocate the memory. In this revision of C++, `::operator new(std::size_t, std::align_val_t)` is used instead.

### C.5.4 Clause 9: declarations

[diff.cpp14.dcl.dcl]

#### <sup>1</sup> Affected subclause: 9.2.2

**Change:** Removal of *register storage-class-specifier*.

**Rationale:** Enable repurposing of deprecated keyword in future revisions of C++.

**Effect on original feature:** A valid C++ 2014 declaration utilizing the *register storage-class-specifier* is ill-formed in this revision of C++. The specifier can simply be removed to retain the original meaning.

#### <sup>2</sup> Affected subclause: 9.2.9.7

**Change:** auto deduction from *braced-init-list*.

**Rationale:** More intuitive deduction behavior.

**Effect on original feature:** Valid C++ 2014 code may fail to compile or may change meaning in this revision of C++.

[Example 1:

```
auto x1{1}; // was std::initializer_list<int>, now int
auto x2{1, 2}; // was std::initializer_list<int>, now ill-formed
— end example]
```

#### <sup>3</sup> Affected subclause: 9.3.4.6

**Change:** Make exception specifications be part of the type system.

**Rationale:** Improve type-safety.

**Effect on original feature:** Valid C++ 2014 code may fail to compile or change meaning in this revision of C++.

[Example 2:

```
void g1() noexcept;
void g2();
template<class T> int f(T *, T *);
int x = f(g1, g2); // ill-formed; previously well-formed
— end example]
```

#### <sup>4</sup> Affected subclause: 9.5.2

**Change:** Definition of an aggregate is extended to apply to user-defined types with base classes.

**Rationale:** To increase convenience of aggregate initialization.

**Effect on original feature:** Valid C++ 2014 code may fail to compile or produce different results in this revision of C++; initialization from an empty initializer list will perform aggregate initialization instead of invoking a default constructor for the affected types.

[Example 3:

```
struct derived;
struct base {
 friend struct derived;
```

```
private:
 base();
};
struct derived : base {};

derived d1{}; // error; the code was well-formed in C++ 2014
derived d2; // still OK
— end example]
```

### C.5.5 Clause 11: classes

[diff.cpp14.class]

#### <sup>1</sup> Affected subclause: 11.9.4

**Change:** Inheriting a constructor no longer injects a constructor into the derived class.

**Rationale:** Better interaction with other language features.

**Effect on original feature:** Valid C++ 2014 code that uses inheriting constructors may not be valid or may have different semantics. A *using-declaration* that names a constructor now makes the corresponding base class constructors visible to initializations of the derived class rather than declaring additional derived class constructors.

[Example 1:

```
struct A {
 template<typename T> A(T, typename T::type = 0);
 A(int);
};
struct B : A {
 using A::A;
 B(int);
};
B b(42L); // now calls B(int), used to call B<long>(long),
 // which called A(int) due to substitution failure
 // in A<long>(long).
```

— end example]

### C.5.6 Clause 13: templates

[diff.cpp14.temp]

#### <sup>1</sup> Affected subclause: 13.10.3.6

**Change:** Allowance to deduce from the type of a constant template argument.

**Rationale:** In combination with the ability to declare constant template arguments with placeholder types, allows partial specializations to decompose from the type deduced for the constant template argument.

**Effect on original feature:** Valid C++ 2014 code may fail to compile or produce different results in this revision of C++.

[Example 1:

```
template <int N> struct A;
template <typename T, T N> int foo(A<N> *) = delete;
void foo(void *);
void bar(A<0> *p) {
 foo(p); // ill-formed; previously well-formed
}
```

— end example]

### C.5.7 Clause 14: exception handling

[diff.cpp14.except]

#### <sup>1</sup> Affected subclause: 14.5

**Change:** Remove dynamic exception specifications.

**Rationale:** Dynamic exception specifications were a deprecated feature that was complex and brittle in use. They interacted badly with the type system, which became a more significant issue in this revision of C++ where (non-dynamic) exception specifications are part of the function type.

**Effect on original feature:** A valid C++ 2014 function declaration, member function declaration, function pointer declaration, or function reference declaration, if it has a potentially throwing dynamic exception specification, is rejected as ill-formed in this revision of C++. Violating a non-throwing dynamic exception specification calls `terminate` rather than `unexpected`, and it is unspecified whether stack unwinding is performed prior to such a call.

**C.5.8 Clause 16: library introduction****[diff.cpp14.library]**<sup>1</sup> **Affected subclause:** 16.4.2.3**Change:** New headers.**Rationale:** New functionality.**Effect on original feature:** The following C++ headers are new: `<any>` (22.7.2), `<charconv>` (28.2.1), `<execution>` (33.4), `<filesystem>` (31.12.4), `<memory_resource>` (20.6.1), `<optional>` (22.5.2), `<string_view>` (27.3.2), and `<variant>` (22.6.2). Valid C++ 2014 code that `#includes` headers with these names may be invalid in this revision of C++.<sup>2</sup> **Affected subclause:** 16.4.5.2.3**Change:** New reserved namespaces.**Rationale:** Reserve namespaces for future revisions of the standard library that might otherwise be incompatible with existing programs.**Effect on original feature:** The global namespaces `std` followed by an arbitrary sequence of *digits* (5.11) are reserved for future standardization. Valid C++ 2014 code that uses such a top-level namespace, e.g., `std2`, may be invalid in this revision of C++.**C.5.9 Clause 22: general utilities library****[diff.cpp14.utilities]**<sup>1</sup> **Affected subclause:** 22.10.17**Change:** Constructors taking allocators removed.**Rationale:** No implementation consensus.**Effect on original feature:** Valid C++ 2014 code may fail to compile or may change meaning in this revision of C++. Specifically, constructing a `std::function` with an allocator is ill-formed and uses-allocator construction will not pass an allocator to `std::function` constructors in this revision of C++.<sup>2</sup> **Affected subclause:** 20.4.2.2**Change:** Different constraint on conversions from `unique_ptr`.**Rationale:** Adding array support to `shared_ptr`, via the syntax `shared_ptr<T[]>` and `shared_ptr<T[N]>`.**Effect on original feature:** Valid C++ 2014 code may fail to compile or may change meaning in this revision of C++.*[Example 1:*

```
#include <memory>
std::unique_ptr<int[]> arr(new int[1]);
std::shared_ptr<int> ptr(std::move(arr)); // error: int(*)[] is not compatible with int*
```

*— end example]***C.5.10 Clause 27: strings library****[diff.cpp14.string]**<sup>1</sup> **Affected subclause:** 27.4.3**Change:** Non-const `.data()` member added.**Rationale:** The lack of a non-const `.data()` differed from the similar member of `std::vector`. This change regularizes behavior.**Effect on original feature:** Overloaded functions which have differing code paths for `char*` and `const char*` arguments will execute differently when called with a non-const string's `.data()` member in this revision of C++.*[Example 1:*

```
int f(char *) = delete;
int f(const char *);
string s;
int x = f(s.data()); // ill-formed; previously well-formed
```

*— end example]***C.5.11 Clause 23: containers library****[diff.cpp14.containers]**<sup>1</sup> **Affected subclause:** 23.2.7**Change:** Requirements change:**Rationale:** Increase portability, clarification of associative container requirements.**Effect on original feature:** Valid C++ 2014 code that attempts to use associative containers having a comparison object with non-const function call operator may fail to compile in this revision of C++.



[Example 1:

```
#include <set>

struct compare
{
 bool operator()(int a, int b)
 {
 return a < b;
 }
};

int main() {
 const std::set<int, compare> s;
 s.find(0);
}
```

— end example]

### C.5.12 Annex D: compatibility features [diff.cpp14.depr]

- <sup>1</sup> **Change:** The class templates `auto_ptr`, `unary_function`, and `binary_function`, the function templates `random_shuffle`, and the function templates (and their return types) `ptr_fun`, `mem_fun`, `mem_fun_ref`, `bind1st`, and `bind2nd` are not defined.

**Rationale:** Superseded by new features.

**Effect on original feature:** Valid C++ 2014 code that uses these class templates and function templates may fail to compile in this revision of C++.

- <sup>2</sup> **Change:** Remove old iostreams members [depr.ios.members].

**Rationale:** Redundant feature for compatibility with pre-standard code has served its time.

**Effect on original feature:** A valid C++ 2014 program using these identifiers may be ill-formed in this revision of C++.

## C.6 C++ and ISO C++ 2011 [diff.cpp11]

### C.6.1 General [diff.cpp11.general]

- <sup>1</sup> Subclause C.6 lists the differences between C++ and ISO C++ 2011, in addition to those listed above, by the chapters of this document.

### C.6.2 Clause 5: lexical conventions [diff.cpp11.lex]

- <sup>1</sup> **Affected subclause:** 5.7

**Change:** *pp-number* can contain one or more single quotes.

**Rationale:** Necessary to enable single quotes as digit separators.

**Effect on original feature:** Valid C++ 2011 code may fail to compile or may change meaning in this revision of C++. For example, the following code is valid both in C++ 2011 and in this revision of C++, but the macro invocation produces different outcomes because the single quotes delimit a *character-literal* in C++ 2011, whereas they are digit separators in this revision of C++.

[Example 1:

```
#define M(x, ...) __VA_ARGS__
int x[2] = { M(1'2,3'4, 5) };
// int x[2] = { 5 }; — C++ 2011
// int x[2] = { 3'4, 5 }; — this revision of C++
```

— end example]

### C.6.3 Clause 6: basics [diff.cpp11.basic]

- <sup>1</sup> **Affected subclause:** 6.8.6.5.3

**Change:** New usual (non-placement) deallocator.

**Rationale:** Required for sized deallocation.

**Effect on original feature:** Valid C++ 2011 code can declare a global placement allocation function and deallocation function as follows:

```
void* operator new(std::size_t, std::size_t);
void operator delete(void*, std::size_t) noexcept;
```

In this revision of C++, however, the declaration of `operator delete` might match a predefined usual (non-placement) `operator delete` (6.8.6.5). If so, the program is ill-formed, as it was for class member allocation functions and deallocation functions (7.6.2.8).

#### C.6.4 Clause 7: expressions

[diff.cpp11.expr]

##### <sup>1</sup> Affected subclause: 7.6.16

**Change:** A conditional expression with a throw expression as its second or third operand keeps the type and value category of the other operand.

**Rationale:** Formerly mandated conversions (lvalue-to-rvalue (7.3.2), array-to-pointer (7.3.3), and function-to-pointer (7.3.4) standard conversions), especially the creation of the temporary due to lvalue-to-rvalue conversion, were considered gratuitous and surprising.

**Effect on original feature:** Valid C++ 2011 code that relies on the conversions may behave differently in this revision of C++.

[Example 1:

```
struct S {
 int x = 1;
 void mf() { x = 2; }
};
int f(bool cond) {
 S s;
 (cond ? s : throw 0).mf();
 return s.x;
}
```

In C++ 2011, `f(true)` returns 1. In this revision of C++, it returns 2.

```
sizeof(true ? "" : throw 0)
```

In C++ 2011, the expression yields `sizeof(const char*)`. In this revision of C++, it yields `sizeof(const char[1])`.  
— end example]

#### C.6.5 Clause 9: declarations

[diff.cpp11.dcl.dcl]

##### <sup>1</sup> Affected subclause: 9.2.6

**Change:** `constexpr` non-static member functions are not implicitly `const` member functions.

**Rationale:** Necessary to allow `constexpr` member functions to mutate the object.

**Effect on original feature:** Valid C++ 2011 code may fail to compile in this revision of C++.

[Example 1:

```
struct S {
 constexpr const int &f();
 int &f();
};
```

This code is valid in C++ 2011 but invalid in this revision of C++ because it declares the same member function twice with different return types. — end example]

##### <sup>2</sup> Affected subclause: 9.5.2

**Change:** Classes with default member initializers can be aggregates.

**Rationale:** Necessary to allow default member initializers to be used by aggregate initialization.

**Effect on original feature:** Valid C++ 2011 code may fail to compile or may change meaning in this revision of C++.

[Example 2:

```
struct S { // Aggregate in C++ 2014 onwards.
 int m = 1;
};
struct X {
 operator int();
 operator S();
};
X a{};
S b{a}; // uses copy constructor in C++ 2011,
 // performs aggregate initialization in this revision of C++
```

— end example]



**C.6.6 Clause 16: library introduction****[diff.cpp11.library]**<sup>1</sup> **Affected subclause:** [16.4.2.3](#)**Change:** New header.**Rationale:** New functionality.**Effect on original feature:** The C++ header `<shared_mutex>` ([32.6.3](#)) is new. Valid C++ 2011 code that `#includes` a header with that name may be invalid in this revision of C++.**C.6.7 Clause 31: input/output library****[diff.cpp11.input.output]**<sup>1</sup> **Affected subclause:** [31.13](#)**Change:** `gets` is not defined.**Rationale:** Use of `gets` is considered dangerous.**Effect on original feature:** Valid C++ 2011 code that uses the `gets` function may fail to compile in this revision of C++.**C.7 C++ and ISO C++ 2003****[diff.cpp03]****C.7.1 General****[diff.cpp03.general]**

- <sup>1</sup>
- Subclause
- [C.7](#)
- lists the differences between C++ and ISO C++ 2003, in addition to those listed above, by the chapters of this document.

**C.7.2 Clause 5: lexical conventions****[diff.cpp03.lex]**<sup>1</sup> **Affected subclause:** [5.5](#)**Change:** New kinds of *string-literals*.**Rationale:** Required for new features.**Effect on original feature:** Valid C++ 2003 code may fail to compile or produce different results in this revision of C++. Specifically, macros named `R`, `u8`, `u8R`, `u`, `uR`, `U`, `UR`, or `LR` will not be expanded when adjacent to a *string-literal* but will be interpreted as part of the *string-literal*.*[Example 1:*

```
#define u8 "abc"
const char* s = u8"def"; // Previously "abcdef", now "def"
— end example]
```

<sup>2</sup> **Affected subclause:** [5.5](#)**Change:** User-defined literal string support.**Rationale:** Required for new features.**Effect on original feature:** Valid C++ 2003 code may fail to compile or produce different results in this revision of C++.*[Example 2:*

```
#define _x "there"
"hello"_x // #1
```

Previously, `#1` would have consisted of two separate preprocessing tokens and the macro `_x` would have been expanded. In this revision of C++, `#1` consists of a single preprocessing token, so the macro is not expanded. — end example]<sup>3</sup> **Affected subclause:** [5.12](#)**Change:** New keywords.**Rationale:** Required for new features.**Effect on original feature:** Added to [Table 5](#), the following identifiers are new keywords: `alignas`, `alignof`, `char16_t`, `char32_t`, `constexpr`, `decltype`, `noexcept`, `nullptr`, `static_assert`, and `thread_local`. Valid C++ 2003 code using these identifiers is invalid in this revision of C++.<sup>4</sup> **Affected subclause:** [5.13.2](#)**Change:** Type of integer literals.**Rationale:** C99 compatibility.**Effect on original feature:** Certain integer literals larger than can be represented by `long` could change from an unsigned integer type to `signed long long`.**C.7.3 Clause 7: expressions****[diff.cpp03.expr]**<sup>1</sup> **Affected subclause:** [7.3.12](#)**Change:** Only literals are integer null pointer constants.

**Rationale:** Removing surprising interactions with templates and constant expressions.

**Effect on original feature:** Valid C++ 2003 code may fail to compile or produce different results in this revision of C++.

[Example 1:

```
void f(void *); // #1
void f(...); // #2
template<int N> void g() {
 f(0*N); // calls #2; used to call #1
}
```

— end example]

## 2 Affected subclause: 7.6.1.8

**Change:** Evaluation of operands in `typeid`.

**Rationale:** Introduce additional expression value categories.

**Effect on original feature:** Valid C++ 2003 code that uses xvalues as operands for `typeid` may change behavior in this revision of C++.

[Example 2:

```
void f() {
 struct B {
 B() {}
 virtual ~B() { }
 };

 struct C { B b; };
 typeid(C().b); // unevaluated in C++ 2003, evaluated in C++ 2011
}
```

— end example]

## 3 Affected subclause: 7.6.5

**Change:** Specify rounding for results of integer `/` and `%`.

**Rationale:** Increase portability, C99 compatibility.

**Effect on original feature:** Valid C++ 2003 code that uses integer division rounds the result toward 0 or toward negative infinity, whereas this revision of C++ always rounds the result toward 0.

## 4 Affected subclause: 7.6.14

**Change:** `&&` is valid in a *type-name*.

**Rationale:** Required for new features.

**Effect on original feature:** Valid C++ 2003 code may fail to compile or produce different results in this revision of C++.

[Example 3:

```
bool b1 = new int && false; // previously false, now ill-formed
struct S { operator int(); };
bool b2 = &S::operator int && false; // previously false, now ill-formed
```

— end example]

## 5 Affected subclause: 7.6.16

**Change:** Fewer copies in the conditional operator.

**Rationale:** Introduce additional expression value categories.

**Effect on original feature:** Valid C++ 2003 code that uses xvalues as operands for the conditional operator may change behavior in this revision of C++.

[Example 4:

```
void f() {
 struct B {
 B() {}
 B(const B&) { }
 };
 struct D : B {};

 struct BB { B b; };
 struct DD { D d; };
```

```

 true ? BB().b : DD().d; // additional copy in C++ 2003, no copy or move in C++ 2011
}
— end example]

```

#### C.7.4 Clause 9: declarations

[diff.cpp03.dcl.dcl]

##### <sup>1</sup> Affected subclause: 9.2

**Change:** Remove `auto` as a storage class specifier.

**Rationale:** New feature.

**Effect on original feature:** Valid C++ 2003 code that uses the keyword `auto` as a storage class specifier may be invalid in this revision of C++. In this revision of C++, `auto` indicates that the type of a variable is to be deduced from its initializer expression.

##### <sup>2</sup> Affected subclause: 9.5.5

**Change:** Narrowing restrictions in aggregate initializers.

**Rationale:** Catches bugs.

**Effect on original feature:** Valid C++ 2003 code may fail to compile in this revision of C++.

[Example 1:

```
int x[] = { 2.0 };
```

This code is valid in C++ 2003 but invalid in this revision of C++ because `double` to `int` is a narrowing conversion. — end example]

##### <sup>3</sup> Affected subclause: 9.12

**Change:** Names declared in an unnamed namespace changed from external linkage to internal linkage; language linkage applies to names with external linkage only.

**Rationale:** Alignment with user expectations.

**Effect on original feature:** Valid C++ 2003 code may violate the one-definition rule (6.3) in this revision of C++.

[Example 2:

```

namespace { extern "C" { extern int x; } } // #1, previously external linkage and C language linkage,
 // now internal linkage and C++ language linkage
namespace A { extern "C" int x = 42; } // #2, external linkage and C language linkage
int main(void) { return x; }

```

This code is valid in C++ 2003, but #2 is not a definition for #1 in this revision of C++, violating the one-definition rule. — end example]

#### C.7.5 Clause 11: classes

[diff.cpp03.class]

##### <sup>1</sup> Affected subclauses: 11.4.5.2, 11.4.7, 11.4.5.3, and 11.4.6

**Change:** Implicitly-declared special member functions are defined as deleted when the implicit definition would have been ill-formed.

**Rationale:** Improves template argument deduction failure.

**Effect on original feature:** A valid C++ 2003 program that uses one of these special member functions in a context where the definition is not required (e.g., in an expression that is not potentially evaluated) becomes ill-formed.

##### <sup>2</sup> Affected subclause: 11.4.7

**Change:** User-declared destructors have an implicit exception specification.

**Rationale:** Clarification of destructor requirements.

**Effect on original feature:** Valid C++ 2003 code may execute differently in this revision of C++. In particular, destructors that throw exceptions will call `std::terminate` (without calling `std::unexpected`) if their exception specification is non-throwing.

#### C.7.6 Clause 13: templates

[diff.cpp03.temp]

##### <sup>1</sup> Affected subclause: 13.2

**Change:** Repurpose `export` for modules (Clause 10, 15.5, 15.6).

**Rationale:** No implementation consensus for the C++ 2003 meaning of `export`.

**Effect on original feature:** A valid C++ 2003 program containing `export` is ill-formed in this revision of C++.

<sup>2</sup> **Affected subclause:** [13.4](#)

**Change:** Remove whitespace requirement for nested closing template right angle brackets.

**Rationale:** Considered a persistent but minor annoyance. Template aliases representing non-class types would exacerbate whitespace issues.

**Effect on original feature:** Change to semantics of well-defined expression. A valid C++ 2003 expression containing a right angle bracket (“>”) followed immediately by another right angle bracket may now be treated as closing two templates.

[*Example 1:*

```
template <class T> struct X { };
template <int N> struct Y { };
X< Y< 1 >> 2 > > x;
```

This code is valid in C++ 2003 because “>>” is a right-shift operator, but invalid in this revision of C++ because “>>” closes two templates. — *end example*]

<sup>3</sup> **Affected subclause:** [13.8.4.2](#)

**Change:** Allow dependent calls of functions with internal linkage.

**Rationale:** Overly constrained, simplify overload resolution rules.

**Effect on original feature:** A valid C++ 2003 program can get a different result in this revision of C++.

## C.7.7 Clause 16: library introduction

[diff.cpp03.library]

<sup>1</sup> **Affected:** [Clause 16](#) – [Clause 33](#)

**Change:** New reserved identifiers.

**Rationale:** Required by new features.

**Effect on original feature:** Valid C++ 2003 code that uses any identifiers added to the C++ standard library by later revisions of C++ may fail to compile or produce different results in this revision of C++. A comprehensive list of identifiers used by the C++ standard library can be found in the Index of Library Names in this document.

<sup>2</sup> **Affected subclause:** [16.4.2.3](#)

**Change:** New headers.

**Rationale:** New functionality.

**Effect on original feature:** The following C++ headers are new: `<array>` ([23.3.2](#)), `<atomic>` ([32.5.2](#)), `<chrono>` ([30.2](#)), `<condition_variable>` ([32.7.2](#)), `<forward_list>` ([23.3.6](#)), `<future>` ([32.10.2](#)), `<initializer_list>` ([17.11.2](#)), `<mutex>` ([32.6.2](#)), `<random>` ([29.5.2](#)), `<ratio>` ([21.5.2](#)), `<regex>` ([28.6.3](#)), `<scoped_allocator>` ([20.7.1](#)), `<system_error>` ([19.5.2](#)), `<thread>` ([32.4.2](#)), `<tuple>` ([22.4.2](#)), `<typeindex>` ([17.7.6](#)), `<type_traits>` ([21.3.3](#)), `<unordered_map>` ([23.5.2](#)), and `<unordered_set>` ([23.5.5](#)). In addition the following C compatibility headers are new: `<cfenv>` ([29.3.1](#)), `<cinttypes>` ([31.13.2](#)), `<cstdint>` ([17.4.1](#)), and `<cuchar>` ([28.7.4](#)). Valid C++ 2003 code that `#includes` headers with these names may be invalid in this revision of C++.

<sup>3</sup> **Affected subclause:** [16.4.4.3](#)

**Change:** Function `swap` moved to a different header.

**Rationale:** Remove dependency on `<algorithm>` ([26.4](#)) for `swap`.

**Effect on original feature:** Valid C++ 2003 code that has been compiled expecting `swap` to be in `<algorithm>` ([26.4](#)) may have to instead include `<utility>` ([22.2.1](#)).

<sup>4</sup> **Affected subclause:** [16.4.5.2.2](#)

**Change:** New reserved namespace.

**Rationale:** New functionality.

**Effect on original feature:** The global namespace `posix` is now reserved for standardization. Valid C++ 2003 code that uses a top-level namespace `posix` may be invalid in this revision of C++.

<sup>5</sup> **Affected subclause:** [16.4.5.3.3](#)

**Change:** Additional restrictions on macro names.

**Rationale:** Avoid hard to diagnose or non-portable constructs.

**Effect on original feature:** Names of attribute identifiers may not be used as macro names. Valid C++ 2003 code that defines `override`, `final`, or `noreturn` as macros is invalid in this revision of C++.

## C.7.8 Clause 17: language support library

[diff.cpp03.language.support]

<sup>1</sup> **Affected subclause:** [17.6.3.2](#)

**Change:** operator `new` may throw exceptions other than `std::bad_alloc`.

**Rationale:** Consistent application of `noexcept`.

**Effect on original feature:** Valid C++ 2003 code that assumes that global operator `new` only throws `std::bad_alloc` may execute differently in this revision of C++. Valid C++ 2003 code that replaces the global replaceable operator `new` is ill-formed in this revision of C++, because the exception specification of `throw(std::bad_alloc)` was removed.

### C.7.9 Clause 19: diagnostics library

[diff.cpp03.diagnostics]

<sup>1</sup> **Affected subclause:** 19.4

**Change:** Thread-local error numbers.

**Rationale:** Support for new thread facilities.

**Effect on original feature:** Valid but implementation-specific C++ 2003 code that relies on `errno` being the same across threads may change behavior in this revision of C++.

### C.7.10 Clause 22: general utilities library

[diff.cpp03.utilities]

<sup>1</sup> **Affected subclauses:** 22.10.6, 22.10.7, 22.10.8, 22.10.10, and 22.10.11

**Change:** Standard function object types no longer derived from `std::unary_function` or `std::binary_function`.

**Rationale:** Superseded by new feature; `unary_function` and `binary_function` are no longer defined.

**Effect on original feature:** Valid C++ 2003 code that depends on function object types being derived from `unary_function` or `binary_function` may fail to compile in this revision of C++.

### C.7.11 Clause 27: strings library

[diff.cpp03.strings]

<sup>1</sup> **Affected subclause:** 27.4

**Change:** `basic_string` requirements no longer allow reference-counted strings.

**Rationale:** Invalidation is subtly different with reference-counted strings. This change regularizes behavior.

**Effect on original feature:** Valid C++ 2003 code may execute differently in this revision of C++.

<sup>2</sup> **Affected subclause:** 27.4.3.2

**Change:** Loosen `basic_string` invalidation rules.

**Rationale:** Allow small-string optimization.

**Effect on original feature:** Valid C++ 2003 code may execute differently in this revision of C++. Some `const` member functions, such as `data` and `c_str`, no longer invalidate iterators.

### C.7.12 Clause 23: containers library

[diff.cpp03.containers]

<sup>1</sup> **Affected subclause:** 23.2

**Change:** Complexity of `size()` member functions now constant.

**Rationale:** Lack of specification of complexity of `size()` resulted in divergent implementations with inconsistent performance characteristics.

**Effect on original feature:** Some container implementations that conform to C++ 2003 may not conform to the specified `size()` requirements in this revision of C++. Adjusting containers such as `std::list` to the stricter requirements may require incompatible changes.

<sup>2</sup> **Affected subclause:** 23.2

**Change:** Requirements change: relaxation.

**Rationale:** Clarification.

**Effect on original feature:** Valid C++ 2003 code that attempts to meet the specified container requirements may now be over-specified. Code that attempted to be portable across containers may need to be adjusted as follows:

- (2.1) — not all containers provide `size()`; use `empty()` instead of `size() == 0`;
- (2.2) — not all containers are empty after construction (`array`);
- (2.3) — not all containers have constant complexity for `swap()` (`array`).

<sup>3</sup> **Affected subclause:** 23.2

**Change:** Requirements change: default constructible.

**Rationale:** Clarification of container requirements.

**Effect on original feature:** Valid C++ 2003 code that attempts to explicitly instantiate a container using a user-defined type with no default constructor may fail to compile.

<sup>4</sup> **Affected subclauses:** 23.2.4 and 23.2.7

**Change:** Signature changes: from `void` return types.

**Rationale:** Old signature threw away useful information that may be expensive to recalculate.

**Effect on original feature:** The following member functions have changed:

- (4.1) — `erase(iterator)` for `set`, `multiset`, `map`, `multimap`
- (4.2) — `erase(begin, end)` for `set`, `multiset`, `map`, `multimap`
- (4.3) — `insert(pos, num, val)` for `vector`, `deque`, `list`, `forward_list`
- (4.4) — `insert(pos, beg, end)` for `vector`, `deque`, `list`, `forward_list`

Valid C++ 2003 code that relies on these functions returning `void` (e.g., code that creates a pointer to member function that points to one of these functions) will fail to compile with this revision of C++.

5 **Affected subclauses:** [23.2.4](#) and [23.2.7](#)

**Change:** Signature changes: from `iterator` to `const_iterator` parameters.

**Rationale:** Overspecification.

**Effect on original feature:** The signatures of the following member functions changed from taking an `iterator` to taking a `const_iterator`:

- (5.1) — `insert(iterator, val)` for `vector`, `deque`, `list`, `set`, `multiset`, `map`, `multimap`
- (5.2) — `insert(pos, beg, end)` for `vector`, `deque`, `list`, `forward_list`
- (5.3) — `erase(begin, end)` for `set`, `multiset`, `map`, `multimap`
- (5.4) — all forms of `list::splice`
- (5.5) — all forms of `list::merge`

Valid C++ 2003 code that uses these functions may fail to compile with this revision of C++.

6 **Affected subclauses:** [23.2.4](#) and [23.2.7](#)

**Change:** Signature changes: `resize`.

**Rationale:** Performance, compatibility with move semantics.

**Effect on original feature:** For `vector`, `deque`, and `list` the fill value passed to `resize` is now passed by reference instead of by value, and an additional overload of `resize` has been added. Valid C++ 2003 code that uses this function may fail to compile with this revision of C++.

### C.7.13 Clause 26: algorithms library

[diff.cpp03.algorithms]

1 **Affected subclause:** [26.1](#)

**Change:** Result state of inputs after application of some algorithms.

**Rationale:** Required by new feature.

**Effect on original feature:** A valid C++ 2003 program may detect that an object with a valid but unspecified state has a different valid but unspecified state with this revision of C++. For example, `std::remove` and `std::remove_if` may leave the tail of the input sequence with a different set of values than previously.

### C.7.14 Clause 29: numerics library

[diff.cpp03.numerics]

1 **Affected subclause:** [29.4](#)

**Change:** Specified representation of complex numbers.

**Rationale:** Compatibility with C99.

**Effect on original feature:** Valid C++ 2003 code that uses implementation-specific knowledge about the binary representation of the required template specializations of `std::complex` may not be compatible with this revision of C++.

### C.7.15 28.3: localization library

[diff.cpp03.locale]

1 **Affected subclause:** [28.3.4.3.2.3](#)

**Change:** The `num_get` facet recognizes hexadecimal floating-point values.

**Rationale:** Required by new feature.

**Effect on original feature:** Valid C++ 2003 code may have different behavior in this revision of C++.

### C.7.16 Clause 31: input/output library

[diff.cpp03.input.output]

1 **Affected subclauses:** [31.7.5.2.4](#), [31.7.6.2.4](#), and [31.5.4.4](#)

**Change:** Specify use of `explicit` in existing boolean conversion functions.

**Rationale:** Clarify intentions, avoid workarounds.



**Effect on original feature:** Valid C++ 2003 code that relies on implicit boolean conversions will fail to compile with this revision of C++. Such conversions occur in the following conditions:

- (1.1) — passing a value to a function that takes an argument of type `bool`;
- (1.2) — using `operator==` to compare to `false` or `true`;
- (1.3) — returning a value from a function with a return type of `bool`;
- (1.4) — initializing members of type `bool` via aggregate initialization;
- (1.5) — initializing a `const bool&` which would bind to a temporary object.

<sup>2</sup> **Affected subclause:** 31.5.2.2.1

**Change:** Change base class of `std::ios_base::failure`.

**Rationale:** More detailed error messages.

**Effect on original feature:** `std::ios_base::failure` is no longer derived directly from `std::exception`, but is now derived from `std::system_error`, which in turn is derived from `std::runtime_error`. Valid C++ 2003 code that assumes that `std::ios_base::failure` is derived directly from `std::exception` may execute differently in this revision of C++.

<sup>3</sup> **Affected subclause:** 31.5.2

**Change:** Flag types in `std::ios_base` are now bitmasks with values defined as constexpr static members.

**Rationale:** Required for new features.

**Effect on original feature:** Valid C++ 2003 code that relies on `std::ios_base` flag types being represented as `std::bitset` or as an integer type may fail to compile with this revision of C++.

[Example 1:

```
#include <iostream>

int main() {
 int flag = std::ios_base::hex;
 std::cout.setf(flag); // error: setf does not take argument of type int
}
```

— end example]

## C.8 C++ and C

[diff.iso]

### C.8.1 General

[diff.iso.general]

- <sup>1</sup> Subclause C.8 lists the differences between C++ and C, in addition to those listed above, by the chapters of this document.

### C.8.2 Clause 5: lexical conventions

[diff.lex]

<sup>1</sup> **Affected subclause:** 5.12

**Change:** New Keywords.

New keywords are added to C++; see 5.12.

**Rationale:** These keywords were added in order to implement the new semantics of C++.

**Effect on original feature:** Change to semantics of well-defined feature. Any C programs that used any of these keywords as identifiers are not valid C++ programs.

**Difficulty of converting:** Syntactic transformation. Converting one specific program is easy. Converting a large collection of related programs takes more work.

**How widely used:** Common.

<sup>2</sup> **Affected subclause:** 5.13.3

**Change:** Type of *character-literal* is changed from `int` to `char`.

**Rationale:** This is needed for improved overloaded function argument type matching.

[Example 1:

```
int function(int i);
int function(char c);

function('x');
```

It is preferable that this call match the second version of function rather than the first. — end example]

**Effect on original feature:** Change to semantics of well-defined feature. C programs which depend on

```
sizeof('x') == sizeof(int)
```

will not work the same as C++ programs.

**Difficulty of converting:** Simple.

**How widely used:** Programs which depend upon `sizeof('x')` are probably rare.

<sup>3</sup> **Affected subclause:** 5.13.5

**Change:** Concatenated *string-literals* can no longer have conflicting *encoding-prefixes*.

**Rationale:** Removal of non-portable feature.

**Effect on original feature:** Concatenation of *string-literals* with different *encoding-prefixes* is now ill-formed.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

<sup>4</sup> **Affected subclause:** 5.13.5

**Change:** String literals made `const`.

The type of a *string-literal* is changed from “array of `char`” to “array of `const char`”. The type of a UTF-8 string literal is changed from “array of `char`” to “array of `const char8_t`”. The type of a UTF-16 string literal is changed from “array of *some-integer-type*” to “array of `const char16_t`”. The type of a UTF-32 string literal is changed from “array of *some-integer-type*” to “array of `const char32_t`”. The type of a wide string literal is changed from “array of `wchar_t`” to “array of `const wchar_t`”.

**Rationale:** This avoids calling an inappropriate overloaded function, which might expect to be able to modify its argument.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Syntactic transformation. The fix is to add a cast:

```
char* p = "abc"; // valid in C, invalid in C++
void f(char*) {
 char* p = (char*)"abc"; // OK, cast added
 f(p);
 f((char*)"def"); // OK, cast added
}
```

**How widely used:** Programs that have a legitimate reason to treat string literal objects as potentially modifiable memory are probably rare.

### C.8.3 Clause 6: basics

[diff.basic]

<sup>1</sup> **Affected subclause:** 6.2

**Change:** C++ does not have “tentative definitions” as in C.

[*Example 1:* At file scope,

```
int i;
int i;
```

is valid in C, invalid in C++. — *end example*]

This makes it impossible to define mutually referential file-local objects with static storage duration, if initializers are restricted to the syntactic forms of C.

[*Example 2:*

```
struct X { int i; struct X* next; };

static struct X a;
static struct X b = { 0, &a };
static struct X a = { 1, &b };
```

— *end example*]

**Rationale:** This avoids having different initialization rules for fundamental types and user-defined types.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. In C++, the initializer for one of a set of mutually-referential file-local objects with static storage duration must invoke a function call to achieve the initialization.

**How widely used:** Seldom.

<sup>2</sup> **Affected subclause:** 6.4

**Change:** A `struct` is a scope in C++, not in C.



[*Example 3:*

```
struct X {
 struct Y { int a; } b;
};
struct Y c;
```

is valid in C but not in C++, which would require `X::Y c;`. — *end example*]

**Rationale:** Class scope is crucial to C++, and a struct is a class.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** C programs use `struct` extremely frequently, but the change is only noticeable when `struct`, enumeration, or enumerator names are referred to outside the `struct`. The latter is probably rare.

<sup>3</sup> **Affected subclause:** 6.7 [also 9.2.9]

**Change:** A name of file scope that is explicitly declared `const`, and not explicitly declared `extern`, has internal linkage, while in C it would have external linkage.

**Rationale:** Because `const` objects may be used as values during translation in C++, this feature urges programmers to provide an explicit initializer for each `const` object. This feature allows the user to put `const` objects in source files that are included in more than one translation unit.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** Seldom.

<sup>4</sup> **Affected subclause:** 6.10.3.1

**Change:** The `main` function cannot be called recursively and cannot have its address taken.

**Rationale:** The `main` function may require special actions.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Trivial: create an intermediary function such as `mymain(argc, argv)`.

**How widely used:** Seldom.

<sup>5</sup> **Affected subclause:** 6.9

**Change:** C allows mixing between “compatible types” in several places where C++ does not.

For example, enumerated types are “compatible” with their underlying types in C but, in C++, enumerations are types distinct from their underlying types.

**Rationale:** Stricter type checking is essential for C++.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. The “typesafe linkage” mechanism will find many, but not all, such problems. Some cases are allowed by C++ according to the “layout compatibility rules” of this document.

**How widely used:** Common.

## C.8.4 Clause 7: expressions

[diff.expr]

<sup>1</sup> **Affected subclause:** 7.3.12

**Change:** Converting a prvalue of type “pointer to *cv* void” to a pointer-to-object type requires casting.

[*Example 1:*

```
char a[10];
void* b=a;
void foo() {
 char* c=b;
}
```

C accepts this usage of “pointer to `void`” being assigned to a pointer to object type. C++ does not. — *end example*]

**Rationale:** C++ tries harder than C to enforce compile-time type safety.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Can be automated. Violations will be diagnosed by the C++ translator. The fix is to add a cast.

[*Example 2:*

```
char* c = (char*) b;
```

— *end example*]

**How widely used:** Common.

<sup>2</sup> **Affected subclause:** 7.4

**Change:** Operations mixing a value of an enumeration type and a value of a different enumeration type or of a floating-point type are not valid.

[Example 3:

```
enum E1 { e };
enum E2 { f };
int b = e <= 3.7; // valid in C; ill-formed in C++
int k = f - e; // valid in C; ill-formed in C++
int x = 1 ? e : f; // valid in C; ill-formed in C++
```

— end example]

**Rationale:** Reinforcing type safety in C++.

**Effect on original feature:** Well-formed C code will not compile with this International Standard.

**Difficulty of converting:** Violations will be diagnosed by the C++ translator. The original behavior can be restored with a cast or integral promotion.

[Example 4:

```
enum E1 { e };
enum E2 { f };
int b = (int)e <= 3.7;
int k = +f - e;
```

— end example]

**How widely used:** Uncommon.

<sup>3</sup> **Affected subclause:** 7.4

**Change:** The usual arithmetic conversions differ, between C and C++, in the treatment of enumeration types with no fixed underlying type. In C++, values of such types are not specified as converting to the underlying type of the enumeration as part of the usual arithmetic conversions. Instead, integral promotions (7.3.7) based on the values of the enumeration (9.8.1) are applied.

**Rationale:** Avoids the difference that can arise in C when replacing an enumeration constant in an expression with a cast of the same enumeration constant to the enumeration type. Allows C++ to treat an enumerator with behavior like that of `int` even while giving the enumerator the type of its enumeration.

**Effect on original feature:** Changes to semantics of well-defined feature. Some C expressions that have a dependence upon the implementation-defined underlying type of affected enumeration types will yield different results (as they would if a different underlying type is chosen by the C implementation).

[Example 5:

```
typedef enum { E0 } E; // the underlying type of E can be unsigned int
static_assert(((E)E0 - 1 < 0) == (E0 - 1 < 0), "Must pass for C++");
```

The `static_assert` may fail in C. — end example]

**Difficulty of converting:** Programs must add explicit casts to the underlying type.

**How widely used:** Rare.

<sup>4</sup> **Affected subclauses:** 7.6.1.6 and 7.6.2.3

**Change:** Decrement operator is not allowed with `bool` operand.

**Rationale:** Feature with surprising semantics.

**Effect on original feature:** A valid C expression utilizing the decrement operator on a `bool` lvalue (for instance, via the C typedef in `<stdbool.h>` (17.15.5)) is ill-formed in C++.

<sup>5</sup> **Affected subclause:** 7.6.2.2

**Change:** In certain contexts, taking the address of a dereferenced null or past-the-end pointer value is well-defined in C (and yields the original pointer value), but results in undefined behavior in C++.

[Example 6:

```
void f() {
 char *p = nullptr;
 char *p2 = &*p; // well-defined in C, undefined behavior in C++
 char *p3 = &p[0]; // well-defined in C, undefined behavior in C++
 int a[5];
```

```
int *q = &a[5]; // well-defined in C, undefined behavior in C++
}
— end example]
```

**Rationale:** Consistent treatment of lvalues in C++.

**Effect on original feature:** Well-formed and well-defined C code exhibits undefined behavior in C++.

**Difficulty of converting:** Syntactic transformation to pointer arithmetic and possible addition of a check for null pointer values.

**How widely used:** Occasionally.

<sup>6</sup> **Affected subclauses:** 7.6.2.5 and 7.6.3

**Change:** In C++, types can only be defined in declarations, not in expressions.

In C, a `sizeof` expression or cast expression may define a new type.

[Example 7:

```
p = (void*)(struct x {int i;} *)0;
```

defines a new type, struct x. — end example]

**Rationale:** This prohibition helps to clarify the location of definitions in the source code.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

<sup>7</sup> **Affected subclauses:** 7.6.9 and 7.6.10

**Change:** C allows directly comparing two objects of array type; C++ does not.

**Rationale:** The behavior is confusing because it compares not the contents of the two arrays, but their addresses.

**Effect on original feature:** Deletion of semantically well-defined feature that had unspecified behavior in common use cases.

**Difficulty of converting:** Violations will be diagnosed by the C++ translator. The original behavior can be replicated by explicitly casting either array to a pointer, such as by using a unary `+`.

[Example 8:

```
int arr1[5];
int arr2[5];
int same = arr1 == arr2; // valid C, ill-formed C++
int idem = arr1 == +arr2; // valid C++, constraint violation in C
```

— end example]

**How widely used:** Rare.

<sup>8</sup> **Affected subclauses:** 7.6.16, 7.6.19, and 7.6.20

**Change:** The result of a conditional expression, an assignment expression, or a comma expression may be an lvalue.

**Rationale:** C++ is an object-oriented language, placing relatively more emphasis on lvalues. For example, function calls may yield lvalues.

**Effect on original feature:** Change to semantics of well-defined feature. Some C expressions that implicitly rely on lvalue-to-rvalue conversions will yield different results.

[Example 9:

```
char arr[100];
sizeof(0, arr)
```

yields 100 in C++ and `sizeof(char*)` in C. — end example]

**Difficulty of converting:** Programs must add explicit casts to the appropriate rvalue.

**How widely used:** Rare.

## C.8.5 Clause 8: statements

[diff.stat]

<sup>1</sup> **Affected subclauses:** 8.5.3 and 8.8.6

**Change:** It is now invalid to jump past a declaration with explicit or implicit initializer (except across entire block not entered).

**Rationale:** Constructors used in initializers may allocate resources which need to be de-allocated upon leaving the block. Allowing jump past initializers would require complicated runtime determination of allocation. Furthermore, many operations on such an uninitialized object have undefined behavior. With this

simple compile-time rule, C++ assures that if an initialized variable is in scope, then it has assuredly been initialized.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** Seldom.

<sup>2</sup> **Affected subclause:** 8.8.4

**Change:** It is now invalid to return (explicitly or implicitly) from a function which is declared to return a value without actually returning a value.

**Rationale:** The caller and callee may assume fairly elaborate return-value mechanisms for the return of class objects. If some flow paths execute a return without specifying any value, the implementation must embody many more complications. Besides, promising to return a value of a given type, and then not returning such a value, has always been recognized to be a questionable practice, tolerated only because very-old C had no distinction between functions with `void` and `int` return types.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. Add an appropriate return value to the source code, such as zero.

**How widely used:** Seldom. For several years, many existing C implementations have produced warnings in this case.

## C.8.6 Clause 9: declarations

[diff.dcl]

<sup>1</sup> **Affected subclause:** 9.1

**Change:** In C++, no declaration of a variable can have *cv* `void` type. In C, this is allowed, unless the declaration is a definition.

[Example 1:

```
extern void x; // valid C, invalid in C++
— end example]
```

**Rationale:** Stricter type checking in C++.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

<sup>2</sup> **Affected subclause:** 9.2.2

**Change:** In C++, the `static` or `extern` specifiers can only be applied to names of objects or functions. Using these specifiers with type declarations is illegal in C++. In C, these specifiers are ignored when used on type declarations.

[Example 2:

```
static struct S { // valid C, invalid in C++
 int i;
};
— end example]
```

**Rationale:** Storage class specifiers don't have any meaning when associated with a type. In C++, class members can be declared with the `static` storage class specifier. Storage class specifiers on type declarations can be confusing for users.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

<sup>3</sup> **Affected subclause:** 9.2.2

**Change:** In C++, `register` is not a storage class specifier.

**Rationale:** The storage class specifier had no effect in C++.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Common.

<sup>4</sup> **Affected subclause:** 9.2.4

**Change:** A C++ *typedef-name* must be different from any class type name declared in the same scope (except if the typedef is a synonym of the class name with the same name). In C, a *typedef-name* and a struct tag name declared in the same scope can have the same name (because they have different name spaces).

[Example 3:

```
typedef struct name1 { /* ... */ } name1; // valid C and C++
struct name { /* ... */ };
typedef int name; // valid C, invalid C++
```

— end example]

**Rationale:** For ease of use, C++ doesn't require that a type name be prefixed with the keywords `class`, `struct` or `union` when used in object declarations or type casts.

[Example 4:

```
class name { /* ... */ };
name i; // i has type class name
```

— end example]

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. One of the 2 types has to be renamed.

**How widely used:** Seldom.

<sup>5</sup> **Affected subclause:** 9.2.9 [see also 6.7]

**Change:** Const objects must be initialized in C++ but can be left uninitialized in C.

**Rationale:** A const object cannot be assigned to so it must be initialized to hold a useful value.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** Seldom.

<sup>6</sup> **Affected subclause:** 9.2.9.7

**Change:** The keyword `auto` cannot be used as a storage class specifier.

[Example 5:

```
void f() {
 auto int x; // valid C, invalid C++
}
```

— end example]

**Rationale:** Allowing the use of `auto` to deduce the type of a variable from its initializer results in undesired interpretations of `auto` as a storage class specifier in certain contexts.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Rare.

<sup>7</sup> **Affected subclause:** 9.3.4.6

**Change:** In C++, a function declared with an empty parameter list takes no arguments. In C, an empty parameter list means that the number and type of the function arguments are unknown.

[Example 6:

```
int f(); // means int f(void) in C++
 // int f(unknown) in C
```

— end example]

**Rationale:** This is to avoid function calls with the wrong number or type of arguments.

**Effect on original feature:** Change to semantics of well-defined feature. This feature was marked as “obsolescent” in C.

**Difficulty of converting:** Syntactic transformation. The function declarations using C incomplete declaration style must be completed to become full prototype declarations. A program may need to be updated further if different calls to the same (non-prototype) function have different numbers of arguments or if the type of corresponding arguments differed.

**How widely used:** Common.

<sup>8</sup> **Affected subclause:** 9.3.4.6 [see 7.6.2.5]

**Change:** In C++, types may not be defined in return or parameter types. In C, these type definitions are allowed.

[Example 7:

```
void f(struct S { int a; } arg) {} // valid C, invalid C++
enum E { A, B, C } f() {} // valid C, invalid C++
```

— end example]

**Rationale:** When comparing types in different translation units, C++ relies on name equivalence when C relies on structural equivalence. Regarding parameter types: since the type defined in a parameter list would be in the scope of the function, the only legal calls in C++ would be from within the function itself.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. The type definitions must be moved to file scope, or in header files.

**How widely used:** Seldom. This style of type definition is seen as poor coding style.

9 **Affected subclause:** 9.6

**Change:** In C++, the syntax for function definition excludes the “old-style” C function. In C, “old-style” syntax is allowed, but deprecated as “obsolescent”.

**Rationale:** Prototypes are essential to type safety.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Common in old programs, but already known to be obsolescent.

10 **Affected subclause:** 9.5.2

**Change:** In C++, designated initialization support is restricted compared to the corresponding functionality in C. In C++, designators for non-static data members must be specified in declaration order, designators for array elements and nested designators are not supported, and designated and non-designated initializers cannot be mixed in the same initializer list.

[Example 8:

```
struct A { int x, y; };
struct B { struct A a; };
struct A a = { .y = 1, .x = 2 }; // valid C, invalid C++
int arr[3] = { [1] = 5 }; // valid C, invalid C++
struct B b = { .a.x = 0 }; // valid C, invalid C++
struct A c = { .x = 1, 2 }; // valid C, invalid C++
```

— end example]

**Rationale:** In C++, members are destroyed in reverse construction order and the elements of an initializer list are evaluated in lexical order, so member initializers must be specified in order. Array designators conflict with *lambda-expression* syntax. Nested designators are seldom used.

**Effect on original feature:** Deletion of feature that is incompatible with C++.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Out-of-order initializers are common. The other features are seldom used.

11 **Affected subclause:** 9.5.3

**Change:** In C++, when initializing an array of character with a string, the number of characters in the string (including the terminating ‘\0’) must not exceed the number of elements in the array. In C, an array can be initialized with a string even if the array is not large enough to contain the string-terminating ‘\0’.

[Example 9:

```
char array[4] = "abcd"; // valid C, invalid C++
```

— end example]

**Rationale:** When these non-terminated arrays are manipulated by standard string functions, there is potential for major catastrophe.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. The arrays must be declared one element bigger to contain the string terminating ‘\0’.

**How widely used:** Seldom. This style of array initialization is seen as poor coding style.

12 **Affected subclause:** 9.8.1

**Change:** C++ objects of enumeration type can only be assigned values of the same enumeration type. In C, objects of enumeration type can be assigned values of any integral type.

[Example 10:

```
enum color { red, blue, green };
enum color c = 1; // valid C, invalid C++
```

— end example]

**Rationale:** The type-safe nature of C++.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation. (The type error produced by the assignment can be automatically corrected by applying an explicit cast.)

**How widely used:** Common.

<sup>13</sup> **Affected subclause:** 9.8.1

**Change:** In C++, the type of an enumerator is its enumeration. In C, the type of an enumerator is an integer type.

[Example 11:

```
enum e { A };
void f() {
 auto x = A;
 int *p = &x; // valid C, invalid C++
}
```

— end example]

**Rationale:** In C++, an enumeration is a distinct type.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** Seldom.

<sup>14</sup> **Affected subclause:** 9.13.2

**Change:** In C++, an *alignment-specifier* is an *attribute-specifier*. In C, an *alignment-specifier* is a *declaration-specifier*.

[Example 12:

```
#include <stdalign.h>
unsigned alignas(8) int x; // valid C, invalid C++
unsigned int y alignas(8); // valid C++, invalid C
```

— end example]

**Rationale:** C++ requires unambiguous placement of the *alignment-specifier*.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

## C.8.7 Clause 11: classes

[diff.class]

<sup>1</sup> **Affected subclause:** 11.3 [see also 9.2.4]

**Change:** In C++, a class declaration introduces the class name into the scope where it is declared and hides any object, function or other declaration of that name in an enclosing scope. In C, an inner scope declaration of a struct tag name never hides the name of an object or function in an outer scope.

[Example 1:

```
int x[99];
void f() {
 struct x { int a; };
 sizeof(x); /* size of the array in C */
 /* size of the struct in C++ */
}
```

— end example]

**Rationale:** This is one of the few incompatibilities between C and C++ that can be attributed to the new C++ name space definition where a name can be declared as a type and as a non-type in a single scope causing the non-type name to hide the type name and requiring that the keywords **class**, **struct**, **union** or **enum** be used to refer to the type name. This new name space definition provides important notational conveniences to C++ programmers and helps making the use of the user-defined types as similar as possible to the use of fundamental types. The advantages of the new name space definition were judged to outweigh by far the incompatibility with C described above.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation. If the hidden name that needs to be accessed is at global scope, the `::` C++ operator can be used. If the hidden name is at block scope, either the type or the



struct tag has to be renamed.

**How widely used:** Seldom.

<sup>2</sup> **Affected subclause:** 11.4.5.3

**Change:** Copying volatile objects.

The implicitly-declared copy constructor and implicitly-declared copy assignment operator cannot make a copy of a volatile lvalue.

[*Example 2:* The following is valid in C:

```
struct X { int i; };
volatile struct X x1 = {0};
struct X x2 = x1; // invalid C++
struct X x3;
x3 = x1; // also invalid C++
```

— end example]

**Rationale:** Several alternatives were debated at length. Changing the parameter to `volatile const X&` would greatly complicate the generation of efficient code for class objects. Discussion of providing two alternative signatures for these implicitly-defined operations raised unanswered concerns about creating ambiguities and complicating the rules that specify the formation of these operators according to the bases and members.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. If volatile semantics are required for the copy, a user-declared constructor or assignment must be provided. If non-volatile semantics are required, an explicit `const_cast` can be used.

**How widely used:** Seldom.

<sup>3</sup> **Affected subclause:** 11.4.10

**Change:** Bit-fields of type plain `int` are signed.

**Rationale:** The signedness needs to be consistent among template specializations. For consistency, the implementation freedom was eliminated for non-dependent types, too.

**Effect on original feature:** The choice is implementation-defined in C, but not so in C++.

**Difficulty of converting:** Syntactic transformation.

**How widely used:** Seldom.

<sup>4</sup> **Affected subclause:** 11.4.12

**Change:** In C++, the name of a nested class is local to its enclosing class. In C the name of the nested class belongs to the same scope as the name of the outermost enclosing class.

[*Example 3:*

```
struct X {
 struct Y { /* ... */ } y;
};
struct Y yy; // valid C, invalid C++
```

— end example]

**Rationale:** C++ classes have member functions which require that classes establish scopes. The C rule would leave classes as an incomplete scope mechanism which would prevent C++ programmers from maintaining locality within a class. A coherent set of scope rules for C++ based on the C rule would be very complicated and C++ programmers would be unable to predict reliably the meanings of nontrivial examples involving nested or local functions.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation. To make the struct type name visible in the scope of the enclosing struct, the struct tag can be declared in the scope of the enclosing struct, before the enclosing struct is defined.

[*Example 4:*

```
struct Y; // struct Y and struct X are at the same scope
struct X {
 struct Y { /* ... */ } y;
};
```

— end example]



All the definitions of C struct types enclosed in other struct definitions and accessed outside the scope of the enclosing struct can be exported to the scope of the enclosing struct. Note: this is a consequence of the difference in scope rules, which is documented in 6.4.

**How widely used:** Seldom.

<sup>5</sup> **Affected subclause:** 6.5.2

**Change:** In C++, a *typedef-name* may not be redeclared in a class definition after being used in that definition.

[Example 5:

```
typedef int I;
struct S {
 I i;
 int I; // valid C, invalid C++
};
```

— end example]

**Rationale:** When classes become complicated, allowing such a redefinition after the type has been used can create confusion for C++ programmers as to what the meaning of I really is.

**Effect on original feature:** Deletion of semantically well-defined feature.

**Difficulty of converting:** Semantic transformation. Either the type or the struct member has to be renamed.

**How widely used:** Seldom.

## C.8.8 Clause 15: preprocessing directives

[diff.cpp]

<sup>1</sup> **Affected subclause:** 15.12

**Change:** Whether `__STDC__` is defined and if so, what its value is, are implementation-defined.

**Rationale:** C++ is not identical to C. Mandating that `__STDC__` be defined would require that translators make an incorrect claim.

**Effect on original feature:** Change to semantics of well-defined feature.

**Difficulty of converting:** Semantic transformation.

**How widely used:** Programs and headers that reference `__STDC__` are quite common.

## C.9 C standard library

[diff.library]

### C.9.1 General

[diff.library.general]

- <sup>1</sup> Subclause C.9 summarizes the explicit changes in headers, definitions, declarations, or behavior between the C standard library in the C standard and the parts of the C++ standard library that were included from the C standard library.

### C.9.2 Modifications to headers

[diff.mods.to.headers]

- <sup>1</sup> For compatibility with the C standard library, the C++ standard library provides the C headers enumerated in 17.15.
- <sup>2</sup> There are no C++ headers for the C standard library's headers `<stdnoreturn.h>` and `<threads.h>`, nor are these headers from the C standard library headers themselves part of C++.
- <sup>3</sup> The C headers `<complex.h>` and `<tgmath.h>` do not contain any of the content from the C standard library and instead merely include other headers from the C++ standard library.

### C.9.3 Modifications to definitions

[diff.mods.to.definitions]

#### C.9.3.1 Types `char8_t`, `char16_t`, and `char32_t`

[diff.char16]

- <sup>1</sup> The types `char8_t`, `char16_t`, and `char32_t` are distinct types rather than typedefs to existing integral types. The tokens `char8_t`, `char16_t`, and `char32_t` are keywords in C++ (5.12). They do not appear as macro or type names defined in `<cuchar>` (28.7.4).

#### C.9.3.2 Type `wchar_t`

[diff.wchar.t]

- <sup>1</sup> The type `wchar_t` is a distinct type rather than a typedef to an existing integral type. The token `wchar_t` is a keyword in C++ (5.12). It does not appear as a macro or type name defined in any of `<cstddef>` (17.2.1), `<cstdlib>` (17.2.2), or `<wchar>` (28.7.3).

**C.9.3.3 Header <iso646.h>****[diff.header.iso646.h]**

- <sup>1</sup> The tokens `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not`, `not_eq`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in C++ (5.12), and are not introduced as macros by <iso646.h> (17.15.3).

**C.9.3.4 Macro NULL****[diff.null]**

- <sup>1</sup> The macro `NULL`, defined in any of <clocale> (28.3.5.1), <cstddef> (17.2.1), <cstdio> (31.13.1), <cstdlib> (17.2.2), <cstring> (27.5.1), <ctime> (30.15), or <wchar> (28.7.3), is an implementation-defined null pointer constant in C++ (17.2).

**C.9.4 Modifications to declarations****[diff.mods.to.declarations]**

- <sup>1</sup> Header <cstring> (27.5.1): The following functions have different declarations:

- (1.1) — `strchr`
- (1.2) — `strpbrk`
- (1.3) — `strrchr`
- (1.4) — `strstr`
- (1.5) — `memchr`

Subclause 27.5.1 describes the changes.

- <sup>2</sup> Header <wchar> (28.7.3): The following functions have different declarations:

- (2.1) — `wcschr`
- (2.2) — `wcspbrk`
- (2.3) — `wcsrchr`
- (2.4) — `wcsstr`
- (2.5) — `wmemchr`

Subclause 28.7.3 describes the changes.

- <sup>3</sup> Header <cstddef> (17.2.1) declares the names `nullptr_t`, `byte`, and `to_integer`, and the operators and operator templates in 17.2.5, in addition to the names declared in <stddef.h> (17.15) in the C standard library.

**C.9.5 Modifications to behavior****[diff.mods.to.behavior]****C.9.5.1 General****[diff.mods.to.behavior.general]**

- <sup>1</sup> Header <cstdlib> (17.2.2): The following functions have different behavior:

- (1.1) — `atexit`
- (1.2) — `exit`
- (1.3) — `abort`

Subclause 17.5 describes the changes.

- <sup>2</sup> Header <csetjmp> (17.14.3): The following functions have different behavior:

- (2.1) — `longjmp`

Subclause 17.14.3 describes the changes.

**C.9.5.2 Macro `offsetof`(*type*, *member-designator*)****[diff.offsetof]**

- <sup>1</sup> The macro `offsetof`, defined in <cstddef> (17.2.1), accepts a restricted set of *type* arguments in C++. Subclause 17.2.4 describes the change.

**C.9.5.3 Memory allocation functions****[diff.malloc]**

- <sup>1</sup> The functions `aligned_alloc`, `calloc`, `malloc`, and `realloc` are restricted in C++. Subclause 20.2.12 describes the changes.

# Annex D

## (normative)

### Compatibility features

[depr]

#### D.1 General

[depr.general]

- <sup>1</sup> This Annex describes features of this document that are specified for compatibility with existing implementations.
- <sup>2</sup> These are deprecated features, where *deprecated* is defined as: Normative for the current revision of C++, but having been identified as a candidate for removal from future revisions. An implementation may declare library names and entities described in this Clause with the `deprecated` attribute (9.13.4).

#### D.2 Non-local use of TU-local entities

[depr.local]

- <sup>1</sup> A declaration of a non-TU-local entity that is an exposure (6.7) is deprecated.

[Note 1: Such a declaration in an importable module unit is ill-formed. — end note]

[Example 1:

```
namespace {
 struct A {
 void f() {}
 };
}
A h();
inline void g() {A().f();}
```

// deprecated: not internal linkage

// deprecated: inline and not internal linkage

— end example]

#### D.3 Implicit capture of `*this` by reference

[depr.capture.this]

- <sup>1</sup> For compatibility with prior revisions of C++, a *lambda-expression* with *capture-default* = (7.5.6.3) may implicitly capture `*this` by reference.

[Example 1:

```
struct X {
 int x;
 void foo(int n) {
 auto f = [=]() { x = n; };
 auto g = [=, this]() { x = n; };
 }
};
```

// deprecated: x means this->x, not a copy thereof

// recommended replacement

— end example]

#### D.4 Deprecated volatile types

[depr.volatile.type]

- <sup>1</sup> Postfix `++` and `--` expressions (7.6.1.6) and prefix `++` and `--` expressions (7.6.2.3) of volatile-qualified arithmetic and pointer types are deprecated.

[Example 1:

```
volatile int velociraptor;
++velociraptor;
```

// deprecated

— end example]

- <sup>2</sup> Certain assignments where the left operand is a volatile-qualified non-class type are deprecated; see 7.6.19.

[Example 2:

```
int neck, tail;
volatile int brachiosaur;
brachiosaur = neck; // OK
tail = brachiosaur; // OK
tail = brachiosaur = neck; // deprecated
brachiosaur += neck; // OK
```

— end example]

- <sup>3</sup> A function type (9.3.4.6) with a parameter with volatile-qualified type or with a volatile-qualified return type is deprecated.

[Example 3:

```
volatile struct amber jurassic(); // deprecated
void trex(volatile short left_arm, volatile short right_arm); // deprecated
void fly(volatile struct pterosaur* pteranodon); // OK
```

— end example]

- <sup>4</sup> A structured binding (9.7) of a volatile-qualified type is deprecated.

[Example 4:

```
struct linhenykus { short forelimb; };
void park(linhenykus alvarezsauroid) {
 volatile auto [what_is_this] = alvarezsauroid; // deprecated
 // ...
}
```

— end example]

## D.5 Non-comma-separated ellipsis parameters

[depr.ellipsis.comma]

A *parameter-declaration-clause* of the form *parameter-declaration-list* ... is deprecated (9.3.4.6).

[Example 1:

```
void f(int...); // deprecated
void g(auto...); // OK, declares a function parameter pack
void h(auto.....); // deprecated
```

— end example]

## D.6 Implicit declaration of copy functions

[depr.impldec]

- <sup>1</sup> The implicit definition of a copy constructor (11.4.5.3) as defaulted is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor (11.4.7). The implicit definition of a copy assignment operator (11.4.6) as defaulted is deprecated if the class has a user-declared copy constructor or a user-declared destructor. It is possible that future versions of C++ will specify that these implicit definitions are deleted (9.6.3).

## D.7 Redclaration of static constexpr data members

[depr.static constexpr]

- <sup>1</sup> For compatibility with prior revisions of C++, a **constexpr** static data member may be redundantly redeclared outside the class with no initializer (6.2, 11.4.9.3). This usage is deprecated.

[Example 1:

```
struct A {
 static constexpr int n = 5; // definition (declaration in C++ 2014)
};

constexpr int A::n; // redundant declaration (definition in C++ 2014)
```

— end example]

## D.8 Literal operator function declarations using an identifier

[depr.lit]

- <sup>1</sup> A *literal-operator-id* (12.6) of the form

```
operator unevaluated-string identifier
```

is deprecated.

**D.9 template keyword before qualified names**

[depr.template.template]

- <sup>1</sup> The use of the keyword `template` before the qualified name of a class or alias template without a template argument list is deprecated (13.3).

**D.10 has\_denorm members in numeric\_limits**

[depr.numeric.limits.has.denorm]

- <sup>1</sup> The following type is defined in addition to those specified in <limits> (17.3.3):

```
namespace std {
 enum float_denorm_style {
 denorm_indeterminate = -1,
 denorm_absent = 0,
 denorm_present = 1
 };
}
```

- <sup>2</sup> The following members are defined in addition to those specified in 17.3.5.1:

```
static constexpr float_denorm_style has_denorm = denorm_absent;
static constexpr bool has_denorm_loss = false;
```

- <sup>3</sup> The values of `has_denorm` and `has_denorm_loss` of specializations of `numeric_limits` are unspecified.

- <sup>4</sup> The following members of the specialization `numeric_limits<bool>` are defined in addition to those specified in 17.3.5.3:

```
static constexpr float_denorm_style has_denorm = denorm_absent;
static constexpr bool has_denorm_loss = false;
```

**D.11 Deprecated C macros**

[depr.c.macros]

- <sup>1</sup> The header <cfloat> (17.3.7) has the following macros:

```
#define FLT_HAS_SUBNORM see below
#define DBL_HAS_SUBNORM see below
#define LDBL_HAS_SUBNORM see below
#define DECIMAL_DIG see below
```

The header defines these macros the same as the C standard library header <float.h>.

SEE ALSO: ISO/IEC 9899:2024, 5.3.5.3.3, 7.33.6

- <sup>2</sup> In addition to being available via inclusion of the <cfloat> header, the macros `INFINITY` and `NAN` are available when <cmath> (29.7.1) is included.

SEE ALSO: ISO/IEC 9899:2024, 7.12

- <sup>3</sup> The header <stdbool.h> (17.15.5) has the following macro:

```
#define __bool_true_false_are_defined 1
```

SEE ALSO: ISO/IEC 9899:2024, 7.19

**D.12 Deprecated error numbers**

[depr.cerrno]

- <sup>1</sup> The header <cerrno> (19.4.2) has the following additional macros:

```
#define ENODATA see below
#define ENOSR see below
#define ENOSTR see below
#define ETIME see below
```

- <sup>2</sup> The meaning of these macros is defined by the POSIX standard.

- <sup>3</sup> The following `enum errc` enumerators are defined in addition to those specified in 19.5.2:

```
no_message_available, // ENODATA
no_stream_resources, // ENOSR
not_a_stream, // ENOSTR
stream_timeout, // ETIME
```

- <sup>4</sup> The value of each `enum errc` enumerator above is the same as the value of the <cerrno> macro shown in the above synopsis.

## D.13 Deprecated type traits

[depr.meta.types]

- <sup>1</sup> The header `<type_traits>` (21.3.3) has the following addition:

```
namespace std {
 template<class T> struct is_trivial;
 template<class T> constexpr bool is_trivial_v = is_trivial<T>::value;
 template<class T> struct is_pod;
 template<class T> constexpr bool is_pod_v = is_pod<T>::value;
 template<size_t Len, size_t Align = default-alignment> // see below
 struct aligned_storage;
 template<size_t Len, size_t Align = default-alignment> // see below
 using aligned_storage_t = aligned_storage<Len, Align>::type;
 template<size_t Len, class... Types>
 struct aligned_union;
 template<size_t Len, class... Types>
 using aligned_union_t = aligned_union<Len, Types...>::type;
}
```

- <sup>2</sup> The behavior of a program that adds specializations for any of the templates defined in this subclause is undefined, unless explicitly permitted by the specification of the corresponding template.

- <sup>3</sup> A *trivial class* is a class that is trivially copyable and has one or more eligible default constructors, all of which are trivial.

[Note 1: In particular, a trivial class does not have virtual functions or virtual base classes. — end note]

A *trivial type* is a scalar type, a trivial class, an array of such a type, or a cv-qualified version of one of these types.

- <sup>4</sup> A *POD class* is a class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD class (or array thereof). A *POD type* is a scalar type, a POD class, an array of such a type, or a cv-qualified version of one of these types.

```
template<class T> struct is_trivial;
```

- <sup>5</sup> *Preconditions:* `remove_all_extents_t<T>` shall be a complete type or *cv void*.

- <sup>6</sup> *Remarks:* `is_trivial<T>` is a *Cpp17UnaryTypeTrait* (21.3.2) with a base characteristic of `true_type` if `T` is a trivial type, and `false_type` otherwise.

- <sup>7</sup> [Note 2: It is unspecified whether a closure type (7.5.6.2) is a trivial type. — end note]

```
template<class T> struct is_pod;
```

- <sup>8</sup> *Preconditions:* `remove_all_extents_t<T>` shall be a complete type or *cv void*.

- <sup>9</sup> *Remarks:* `is_pod<T>` is a *Cpp17UnaryTypeTrait* (21.3.2) with a base characteristic of `true_type` if `T` is a POD type, and `false_type` otherwise.

- <sup>10</sup> [Note 3: It is unspecified whether a closure type (7.5.6.2) is a POD type. — end note]

```
template<size_t Len, size_t Align = default-alignment>
 struct aligned_storage;
```

- <sup>11</sup> The value of *default-alignment* is the most stringent alignment requirement for any object type whose size is no greater than `Len` (6.9).

- <sup>12</sup> *Mandates:* `Len` is not zero. `Align` is equal to `alignof(T)` for some type `T` or to *default-alignment*.

- <sup>13</sup> The member typedef `type` denotes a trivial standard-layout type suitable for use as uninitialized storage for any object whose size is at most `Len` and whose alignment is a divisor of `Align`.

- <sup>14</sup> [Note 4: Uses of `aligned_storage<Len, Align>::type` can be replaced by an array `std::byte[Len]` declared with `alignas(Align)`. — end note]

- <sup>15</sup> [Note 5: A typical implementation would define `aligned_storage` as:

```
template<size_t Len, size_t Alignment>
struct aligned_storage {
 typedef struct {
 alignas(Alignment) unsigned char __data[Len];
 } type;
};
```

— end note]

```
template<size_t Len, class... Types>
struct aligned_union;
```

16 *Mandates:* At least one type is provided. Each type in the template parameter pack **Types** is a complete object type.

17 The member typedef **type** denotes a trivial standard-layout type suitable for use as uninitialized storage for any object whose type is listed in **Types**; its size shall be at least **Len**. The static member **alignment\_value** is an integral constant of type **size\_t** whose value is the strictest alignment of all types listed in **Types**.

## D.14 Relational operators

[depr.relops]

1 The header <utility> (22.2.1) has the following additions:

```
namespace std::rel_ops {
 template<class T> bool operator!=(const T&, const T&);
 template<class T> bool operator> (const T&, const T&);
 template<class T> bool operator<=(const T&, const T&);
 template<class T> bool operator>=(const T&, const T&);
}
```

2 To avoid redundant definitions of **operator!=** out of **operator==** and operators **>**, **<=**, and **>=** out of **operator<**, the library provides the following:

```
template<class T> bool operator!=(const T& x, const T& y);
```

3 *Preconditions:* T meets the *Cpp17EqualityComparable* requirements (Table 28).

4 *Returns:* !(x == y).

```
template<class T> bool operator>(const T& x, const T& y);
```

5 *Preconditions:* T meets the *Cpp17LessThanComparable* requirements (Table 29).

6 *Returns:* y < x.

```
template<class T> bool operator<=(const T& x, const T& y);
```

7 *Preconditions:* T meets the *Cpp17LessThanComparable* requirements (Table 29).

8 *Returns:* !(y < x).

```
template<class T> bool operator>=(const T& x, const T& y);
```

9 *Preconditions:* T meets the *Cpp17LessThanComparable* requirements (Table 29).

10 *Returns:* !(x < y).

## D.15 Tuple

[depr.tuple]

1 The header <tuple> (22.4.2) has the following additions:

```
namespace std {
 template<class T> struct tuple_size<volatile T>;
 template<class T> struct tuple_size<const volatile T>;

 template<size_t I, class T> struct tuple_element<I, volatile T>;
 template<size_t I, class T> struct tuple_element<I, const volatile T>;
}
```

```
template<class T> struct tuple_size<volatile T>;
```

```
template<class T> struct tuple_size<const volatile T>;
```

2 Let TS denote **tuple\_size<T>** of the cv-unqualified type T. If the expression **TS::value** is well-formed when treated as an unevaluated operand (7.2.3), then specializations of each of the two templates meet the *Cpp17TransformationTrait* requirements with a base characteristic of **integral\_constant<size\_t, TS::value>**. Otherwise, they have no member **value**.

3 Access checking is performed as if in a context unrelated to TS and T. Only the validity of the immediate context of the expression is considered.

- 4 In addition to being available via inclusion of the `<tuple>` (22.4.2) header, the two templates are available when any of the headers `<array>` (23.3.2), `<ranges>` (25.2), or `<utility>` (22.2.1) are included.

```
template<size_t I, class T> struct tuple_element<I, volatile T>;
template<size_t I, class T> struct tuple_element<I, const volatile T>;
```

- 5 Let TE denote `tuple_element_t<I, T>` of the cv-unqualified type T. Then specializations of each of the two templates meet the *Cpp17TransformationTrait* requirements with a member typedef `type` that names the following type:

- (5.1) — for the first specialization, `volatile TE`, and  
 (5.2) — for the second specialization, `const volatile TE`.

- 6 In addition to being available via inclusion of the `<tuple>` (22.4.2) header, the two templates are available when any of the headers `<array>` (23.3.2), `<ranges>` (25.2), or `<utility>` (22.2.1) are included.

## D.16 Variant

[depr.variant]

- 1 The header `<variant>` (22.6.2) has the following additions:

```
namespace std {
 template<class T> struct variant_size<volatile T>;
 template<class T> struct variant_size<const volatile T>;

 template<size_t I, class T> struct variant_alternative<I, volatile T>;
 template<size_t I, class T> struct variant_alternative<I, const volatile T>;
}
```

```
template<class T> struct variant_size<volatile T>;
template<class T> struct variant_size<const volatile T>;
```

- 2 Let VS denote `variant_size<T>` of the cv-unqualified type T. Then specializations of each of the two templates meet the *Cpp17UnaryTypeTrait* requirements with a base characteristic of `integral_constant<size_t, VS::value>`.

```
template<size_t I, class T> struct variant_alternative<I, volatile T>;
template<size_t I, class T> struct variant_alternative<I, const volatile T>;
```

- 3 Let VA denote `variant_alternative<I, T>` of the cv-unqualified type T. Then specializations of each of the two templates meet the *Cpp17TransformationTrait* requirements with a member typedef `type` that names the following type:

- (3.1) — for the first specialization, `volatile VA::type`, and  
 (3.2) — for the second specialization, `const volatile VA::type`.

## D.17 Deprecated `vector<bool, Allocator>` swap

[depr.vector.bool.swap]

- 1 The following member is declared in addition to those members specified in 23.3.14:

```
namespace std {
 template<class Allocator> class vector<bool, Allocator> {
 public:
 static constexpr void swap(reference x, reference y) noexcept;
 };
}
```

```
static constexpr void swap(reference x, reference y) noexcept;
```

- 2 *Effects:* Exchanges the values denoted by x and y as if by:

```
bool b = x;
x = y;
y = b;
```

## D.18 Deprecated iterator class template

[depr.iterator]

- 1 The header `<iterator>` (24.2) has the following addition:



```

namespace std {
 template<class Category, class T, class Distance = ptrdiff_t,
 class Pointer = T*, class Reference = T&>
 struct iterator {
 using iterator_category = Category;
 using value_type = T;
 using difference_type = Distance;
 using pointer = Pointer;
 using reference = Reference;
 };
}

```

- <sup>2</sup> The `iterator` template may be used as a base class to ease the definition of required types for new iterators.
- <sup>3</sup> [*Note 1*: If the new iterator type is a class template, then these aliases will not be visible from within the iterator class's template definition, but only to callers of that class. — *end note*]
- <sup>4</sup> [*Example 1*: If a C++ program wants to define a bidirectional iterator for some data structure containing `double` and such that it works on a large memory model of the implementation, it can do so with:

```

class MyIterator :
 public iterator<bidirectional_iterator_tag, double, long, T*, T&> {
 // code implementing ++, etc.
};
— end example]

```

## D.19 Deprecated `move_iterator` access

[depr.move.iter.elem]

- <sup>1</sup> The following member is declared in addition to those members specified in 24.5.4.6:

```

namespace std {
 template<class Iterator>
 class move_iterator {
 public:
 constexpr pointer operator->() const;
 };
}

constexpr pointer operator->() const;

```

- <sup>2</sup> *Returns*: current.

## D.20 Deprecated locale category facets

[depr.locale.category]

- <sup>1</sup> The `ctype` locale category includes the following facets as if they were specified in Table 91 of 28.3.3.1.2.1.

```

codecvt<char16_t, char, mbstate_t>
codecvt<char32_t, char, mbstate_t>
codecvt<char16_t, char8_t, mbstate_t>
codecvt<char32_t, char8_t, mbstate_t>

```

- <sup>2</sup> The `ctype` locale category includes the following facets as if they were specified in Table 92 of 28.3.3.1.2.1.

```

codecvt_byname<char16_t, char, mbstate_t>
codecvt_byname<char32_t, char, mbstate_t>
codecvt_byname<char16_t, char8_t, mbstate_t>
codecvt_byname<char32_t, char8_t, mbstate_t>

```

- <sup>3</sup> The following class template specializations are required in addition to those specified in 28.3.4.2.5. The specializations `codecvt<char16_t, char, mbstate_t>` and `codecvt<char16_t, char8_t, mbstate_t>` convert between the UTF-16 and UTF-8 encoding forms, and the specializations `codecvt<char32_t, char, mbstate_t>` and `codecvt<char32_t, char8_t, mbstate_t>` convert between the UTF-32 and UTF-8 encoding forms.

## D.21 Deprecated formatting

[depr.format]

### D.21.1 Header `<format>` synopsis

[depr.format.syn]

- <sup>1</sup> The header `<format>` (28.5.1) has the following additions:

```
namespace std {
 template<class Visitor, class Context>
 decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context> arg);
}
```

### D.21.2 Formatting arguments

[depr.format.arg]

```
template<class Visitor, class Context>
 decltype(auto) visit_format_arg(Visitor&& vis, basic_format_arg<Context> arg);
```

- <sup>1</sup> *Effects:* Equivalent to: `return visit(std::forward<Visitor>(vis), arg.value_);`

### D.22 Deprecated time formatting

[depr.ctime]

- <sup>1</sup> The header `<ctime>` (30.15) has the following additions:

```
char* asctime(const tm* timeptr);
char* ctime(const time_t* timer);
```

- <sup>2</sup> The functions `asctime` and `ctime` are not required to avoid data races (16.4.6.10).

SEE ALSO: ISO/IEC 9899:2024, 7.29

### D.23 Deprecated file systems

[depr.filesystems]

#### D.23.1 Deprecated filesystem path factory functions

[depr.fs.path.factory]

- <sup>1</sup> The header `<filesystem>` (31.12.4) has the following additions:

```
template<class Source>
 path u8path(const Source& source);
template<class InputIterator>
 path u8path(InputIterator first, InputIterator last);
```

- <sup>2</sup> *Mandates:* The value type of `Source` and `InputIterator` is `char` or `char8_t`.

- <sup>3</sup> *Preconditions:* The `source` and `[first,last)` sequences are UTF-8 encoded. `Source` meets the requirements specified in 31.12.6.4.

- <sup>4</sup> *Returns:*

- (4.1) — If `path::value_type` is `char` and the current native narrow encoding (31.12.6.3.2) is UTF-8, return `path(source)` or `path(first, last)`; otherwise,
- (4.2) — if `path::value_type` is `wchar_t` and the native wide encoding is UTF-16, or if `path::value_type` is `char16_t` or `char32_t`, convert `source` or `[first,last)` to a temporary, `tmp`, of type `path::string_type` and return `path(tmp)`; otherwise,
- (4.3) — convert `source` or `[first,last)` to a temporary, `tmp`, of type `u32string` and return `path(tmp)`.

- <sup>5</sup> *Remarks:* Argument format conversion (31.12.6.3.1) applies to the arguments for these functions. How Unicode encoding conversions are performed is unspecified.

- <sup>6</sup> [*Example 1:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.

For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.

For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*

[*Note 1:* The example above is representative of a historical use of `filesystem::u8path`. To indicate a UTF-8 encoding, passing a `std::u8string` to `path`'s constructor is preferred as it is consistent with `path`'s handling of other encodings. — *end note*]

**D.23.2 Deprecated filesystem path format observers****[depr.fs.path.obs]**

- <sup>1</sup> The following members are declared in addition to those members specified in 31.12.6.5:

```
namespace std::filesystem {
 class path {
 public:
 std::string string() const;
 std::string generic_string() const;
 };
}
```

```
std::string string() const;
```

- <sup>2</sup> *Returns:* native\_encoded\_string().

```
std::string generic_string() const;
```

- <sup>3</sup> *Returns:* generic\_native\_encoded\_string().

**D.24 Deprecated atomic operations****[depr.atomics]****D.24.1 General****[depr.atomics.general]**

- <sup>1</sup> The header <atomic> (32.5.2) has the following additions.

```
namespace std {
 template<class T>
 void atomic_init(volatile atomic<T>*, typename atomic<T>::value_type) noexcept;
 template<class T>
 void atomic_init(atomic<T>*, typename atomic<T>::value_type) noexcept;
 template<class T>
 constexpr T kill_dependency(T y) noexcept; // freestanding
 inline constexpr memory_order memory_order_consume = memory_order::consume; // freestanding

 #define ATOMIC_VAR_INIT(value) see below
}
```

**D.24.2 Volatile access****[depr.atomics.volatile]**

- <sup>1</sup> If an atomic (32.5.8) specialization has one of the following overloads, then that overload participates in overload resolution even if atomic<T>::is\_always\_lock\_free is false:

```
void store(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
T operator=(T desired) volatile noexcept;
T load(memory_order order = memory_order::seq_cst) const volatile noexcept;
operator T() const volatile noexcept;
T exchange(T desired, memory_order order = memory_order::seq_cst) volatile noexcept;
bool compare_exchange_weak(T& expected, T desired,
 memory_order success, memory_order failure) volatile noexcept;
bool compare_exchange_strong(T& expected, T desired,
 memory_order success, memory_order failure) volatile noexcept;
bool compare_exchange_weak(T& expected, T desired,
 memory_order order = memory_order::seq_cst) volatile noexcept;
bool compare_exchange_strong(T& expected, T desired,
 memory_order order = memory_order::seq_cst) volatile noexcept;
T fetch_key(T operand, memory_order order = memory_order::seq_cst) volatile noexcept;
T operator op=(T operand) volatile noexcept;
T* fetch_key(ptrdiff_t operand, memory_order order = memory_order::seq_cst) volatile noexcept;
value_type operator++(int) volatile noexcept;
value_type operator--(int) volatile noexcept;
value_type operator++() volatile noexcept;
value_type operator--() volatile noexcept;
void wait(T old, memory_order order = memory_order::seq_cst) const volatile noexcept;
void notify_one() volatile noexcept;
void notify_all() volatile noexcept;
```

**D.24.3 Non-member functions****[depr.atomics.nonmembers]**

```
template<class T>
 void atomic_init(volatile atomic<T>* object, typename atomic<T>::value_type desired) noexcept;
template<class T>
 void atomic_init(atomic<T>* object, typename atomic<T>::value_type desired) noexcept;
```

<sup>1</sup> *Effects:* Equivalent to: `atomic_store_explicit(object, desired, memory_order::relaxed);`

**D.24.4 Operations on atomic types****[depr.atomics.types.operations]**

```
#define ATOMIC_VAR_INIT(value) see below
```

<sup>1</sup> The macro expands to a token sequence suitable for constant initialization of an atomic variable with static storage duration of a type that is initialization-compatible with `value`.

[*Note 1:* This operation possibly needs to initialize locks. — *end note*]

Concurrent access to the variable being initialized, even via an atomic operation, constitutes a data race.

[*Example 1:*

```
 atomic<int> v = ATOMIC_VAR_INIT(5);
```

— *end example*]

**D.24.5 `memory_order::consume`****[depr.atomics.order]**

<sup>1</sup> The `memory_order` enumeration contains an additional enumerator:

```
 consume = 1
```

The `memory_order::consume` enumerator is allowed wherever `memory_order::acquire` is allowed, and it has the same meaning.

```
template<class T> constexpr T kill_dependency(T y) noexcept;
```

<sup>2</sup> *Returns:* `y`.

**D.25 Deprecated signed char and unsigned char extraction [depr.istream.extractors]**

<sup>1</sup> The header `<istream>` (31.7.1) has the following additions:

```
template<class traits>
 basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in, unsigned char& c);
template<class traits>
 basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in, signed char& c);
```

<sup>2</sup> *Effects:* Behaves like a formatted input member (as described in (31.7.5.3.1)) of `in`. A character is extracted from `in`, if one is available, and stored in `c`. Otherwise, `ios_base::failbit` is set in the input function's local error state before `setstate` is called.

<sup>3</sup> *Returns:* `in`.

```
4 template<class traits, size_t N>
 basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in, unsigned char (&s)[N]);
template<class traits, size_t N>
 basic_istream<char, traits>& operator>>(basic_istream<char, traits>& in, signed char (&s)[N]);
```

<sup>5</sup> *Effects:* Behaves like a formatted input member (as described in (31.7.5.3.1)) of `in`. After a sentry object is constructed, `operator>>` extracts characters and stores them into `s`. If `width()` is greater than zero, `n` is `min(size_t(width()), N)`. Otherwise `n` is `N`. `n` is the maximum number of characters stored.

<sup>6</sup> Characters are extracted and stored until any of the following occurs:

- (6.1) — `n - 1` characters are stored;
- (6.2) — end of file occurs on the input sequence;
- (6.3) — letting `ct` be `use_facet<ctype<charT>>(in.getloc())`, `ct.is(ct.space, s)` is true.

- 7 `operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.
- 8 If the function extracted no characters, `ios_bit::failbit` is set in the input function's local error state before `setstate` is called.
- 9 *Returns:* `in`.

## D.26 Deprecated signed char and unsigned char insertion [depr.ostream.inserters]

- 1 The header `<ostream>` (31.7.2) has the following additions:

```
2 template<class traits>
 basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, signed char c);
template<class traits>
 basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>& out, unsigned char c);
```

- 3 *Effects:* Equivalent to: `return out << static_cast<char>(c);`

```
4 template<class traits>
 basic_ostream<char, traits>&
 operator<<(basic_ostream<char, traits>& out, const signed char* s);
template<class traits>
 basic_ostream<char, traits>&
 operator<<(basic_ostream<char, traits>& out, const unsigned char* s);
```

- 5 *Effects:* Equivalent to: `return out << reinterpret_cast<const char*>(s);`

# Annex E

## (informative)

### Conformance with UAX #31

[uaxid]

#### E.1 General

[uaxid.general]

- <sup>1</sup> This Annex describes the choices made in application of UAX #31 (“Unicode Identifier and Pattern Syntax”) to C++ in terms of the requirements from UAX #31 and how they do or do not apply to this document. In terms of UAX #31, this document conforms by meeting the requirements R1 “Default Identifiers” and R4 “Equivalent Normalized Identifiers” from UAX #31. The other requirements from UAX #31, also listed below, are either alternatives not taken or do not apply to this document.

#### E.2 R1 Default identifiers

[uaxid.def]

##### E.2.1 General

[uaxid.def.general]

- <sup>1</sup> UAX #31 specifies a default syntax for identifiers based on properties from the Unicode Character Database, UAX #44. The general syntax is

```
<Identifier> := <Start> <Continue>* (<Medial> <Continue>+)*
```

where <Start> has the `XID_Start` property, <Continue> has the `XID_Continue` property, and <Medial> is a list of characters permitted between continue characters. UAX #31 also specifies some standard profiles, including the Mathematical Compatibility Notation profile. For C++ we apply the Mathematical Compatibility Notation profile, and we add the character U+005F LOW LINE, or `_`, to the set of permitted <Start> characters. The <Medial> set is empty. In the grammar used in UAX #31, this is

```
<Identifier> := <Start> <Continue>*
<Start> := XID_Start + ID_Compat_Math_Start + U+005F
<Continue> := <Start> + XID_Continue + ID_Compat_Math_Continue
```

- <sup>2</sup> This is described in the C++ grammar in 5.11, where *identifier* is formed from *identifier-start* or *identifier* followed by *identifier-continue*.

##### E.2.2 R1b Stable identifiers

[uaxid.def.stable]

- <sup>1</sup> An implementation of UAX #31 may choose to guarantee that identifiers are stable across versions of the Unicode Standard. Once a string qualifies as an identifier it does so in all future versions.
- <sup>2</sup> C++ does not make this guarantee, except to the extent that UAX #31 guarantees the stability of the `XID_Start` and `XID_Continue` properties.

#### E.3 R2 Immutable identifiers

[uaxid.immutable]

- <sup>1</sup> An implementation may choose to guarantee that the set of identifiers will never change by fixing the set of code points allowed in identifiers forever.
- <sup>2</sup> C++ does not choose to make this guarantee. As scripts are added to Unicode, additional characters in those scripts may become available for use in identifiers.

#### E.4 R3 Pattern\_White\_Space and Pattern\_Syntax characters

[uaxid.pattern]

- <sup>1</sup> UAX #31 describes how formal languages such as computer languages should describe and implement their use of whitespace and syntactically significant characters during the processes of lexing and parsing.
- <sup>2</sup> This document does not claim conformance with this requirement from UAX #31.

**E.5 R4 Equivalent normalized identifiers** [uaxid.eqn]

- <sup>1</sup> UAX #31 requires that implementations describe how identifiers are compared and considered equivalent.
- <sup>2</sup> This document requires that identifiers be in Normalization Form C and therefore identifiers that compare the same under NFC are equivalent. This is described in 5.11.

**E.6 R5 Equivalent case-insensitive identifiers** [uaxid.eqci]

- <sup>1</sup> This document considers case to be significant in identifier comparison, and does not do any case folding. This requirement from UAX #31 does not apply to this document.

**E.7 R6 Filtered normalized identifiers** [uaxid.filter]

- <sup>1</sup> If any characters are excluded from normalization, UAX #31 requires a precise specification of those exclusions.
- <sup>2</sup> This document does not make any such exclusions.

**E.8 R7 Filtered case-insensitive identifiers** [uaxid.filterci]

- <sup>1</sup> C++ identifiers are case sensitive, and therefore this requirement from UAX #31 does not apply.

**E.9 R8 Hashtag identifiers** [uaxid.hashtag]

- <sup>1</sup> There are no hashtags in C++, so this requirement from UAX #31 does not apply.

# Annex F

## (informative)

### Core undefined behavior

[ub]

#### F.1 General

[ub.general]

This Annex documents undefined behavior explicitly called out in [Clause 4](#) through [Clause 15](#) using the following phrases:

- is undefined
- the behavior is undefined
- the behavior of the program is undefined
- has undefined behavior
- have undefined behavior
- result has undefined behavior
- results in undefined behavior

Undefined behavior that is implicit is not covered by this annex. Each entry contains a title, a cross-reference, a summary of the circumstances, and code examples. The code examples are not intended to exhaustively cover all possible ways of invoking that case.

#### F.2 Clause 6: Basics

[ub.basic]

##### F.2.1

[ub:intro.object.implicit.create]

Specified in: [6.8.2](#)

- <sup>1</sup> For each operation that is specified as implicitly creating objects, that operation implicitly creates and starts the lifetime of zero or more objects of implicit-lifetime types ([6.9](#)) in its specified region of storage if doing so would result in the program having defined behavior. If no such set of objects would give the program defined behavior, the behavior of the program is undefined.

- <sup>2</sup> [Example 1:

```
void f()
{
 void *p = malloc(sizeof(int) + sizeof(float));
 reinterpret_cast<int>(p) = 0;
 reinterpret_cast<float>(p) = 0.0f; // undefined behavior, cannot create
 // both int and float in same place
}
```

— end example]

##### F.2.2

[ub:intro.object.implicit.pointer]

Specified in: [6.8.2](#)

- <sup>1</sup> After implicitly creating objects within a specified region of storage, some operations are described as producing a pointer to a suitable created object ([6.9](#)). These operations select one of the implicitly-created objects whose address is the address of the start of the region of storage, and produce a pointer value that points to that object, if that value would result in the program having defined behavior. If no such pointer value would give the program defined behavior, the behavior of the program is undefined.

- <sup>2</sup> [Example 1:



```

#include <cstdlib>
struct X {
 int a, b;
 ~X() = delete; // deleted destructor makes X a non-implicit-lifetime class
};

X* make_x() {
 X* p = (X*)std::malloc(sizeof(struct X)); // Cannot implicitly create an object because X
 // is not an implicit-lifetime class.
 p->a = 1; // undefined behavior, no set of objects give us defined behavior
 return p;
}
— end example]

```

[Example 2:

```

#include <cstdlib>

struct X {
 int a;
};

struct Y {
 int x;
};

Y* make_y() {
 X* p1 = (X*)std::malloc(sizeof(struct X)); // object #1
 p1->a = 1;
 Y* p2 = (Y*)p1;
 p2->x = 2; // undefined behavior, p2 points to an in-lifetime object of type X,
 // which is not similar to Y (7.6.1.5)
 return p2;
}
— end example]

```

### F.2.3

[ub:basic.align.object.alignment]

Specified in: 6.8.3

- <sup>1</sup> All instances of a type must be created in storage that meets the alignment requirement of that type.
- <sup>2</sup> [Example 1:

```

struct alignas(4) S {};

void make_misaligned()
{
 alignas(S) char s[sizeof(S) + 1];
 new (&s+1) S(); // undefined behavior, &s+1 will yield a pointer to
 // a char which is 1 byte away from an address with
 // an alignment of 4 and so cannot have an alignment of 4
}
— end example]

```

### F.2.4

[ub:lifetime.outside.pointer.delete]

Specified in: 6.8.4

- <sup>1</sup> For a pointer pointing to an object outside of its lifetime, behavior is undefined if the object will be or was of a class type with a non-trivial destructor and the pointer is used as the operand of a *delete-expression*.
  - <sup>2</sup> [Example 1:
- ```

struct S {
    float f = 0;
    ~S() {}
};

```

```

void f() {
    S* p = new S;
    p->~S();
    delete p;      // undefined behavior, operand of delete, lifetime has ended and S
                   // has a non-trivial destructor
}

```

— end example]

F.2.5

[ub:lifetime.outside.pointer.member]

Specified in: 6.8.4

- ¹ For a pointer pointing to an object outside of its lifetime, behavior is undefined if the pointer is used to access a non-static data member or call a non-static member function of the object.

² [Example 1:

```

struct S {
    float f = 0;
};

float f() {
    S s;
    S* p = &s;
    s.~S();
    return p->f; // undefined behavior, accessing non-static data member after end of lifetime
}

```

— end example]

F.2.6

[ub:lifetime.outside.pointer.virtual]

Specified in: 6.8.4

- ¹ For a pointer pointing to an object outside of its lifetime, behavior is undefined if the pointer is implicitly converted (7.3.12) to a pointer to a virtual base class (or base class of a virtual base class).

² [Example 1:

```

struct B {};
struct D : virtual B {};
void f() {
    D d;
    D* p = &d;
    d.~D();
    B* b = p; // undefined behavior
}

```

— end example]

F.2.7

[ub:lifetime.outside.pointer.dynamic.cast]

Specified in: 6.8.4

- ¹ For a pointer pointing to an object outside of its lifetime, behavior is undefined if the pointer is used as the operand of a `dynamic_cast` (7.6.1.7).

² [Example 1:

```

struct B { virtual ~B() = default; };
struct D : B {};
void f()
{
    D d;
    B* bp = &d;
    d.~D();
    D* dp = dynamic_cast<D*>(bp); // undefined behavior
}

```

— end example]

F.2.8**[ub:lifetime.outside.glvalue.access]****Specified in:** 6.8.4

- ¹ Behavior is undefined if a glvalue referring to an object outside of its lifetime is used to access the object.

² [Example 1:

```
void f() {
    int x = int{10};
    using T = int;
    x.~T();
    int y = x;    // undefined behavior, glvalue used to access the
                  // object after the lifetime has ended
}
```

— end example]

F.2.9**[ub:lifetime.outside.glvalue.member]****Specified in:** 6.8.4

- ¹ Behavior is undefined if a glvalue referring to an object outside of its lifetime is used to call a non-static member function of the object.

² [Example 1:

```
struct A {
    void f() {}
};

void f() {
    A a;
    a.~A();
    a.f();    // undefined behavior, glvalue used to access a non-static member
              // function after the lifetime has ended
}
```

— end example]

F.2.10**[ub:lifetime.outside.glvalue.virtual]****Specified in:** 6.8.4

- ¹ Behavior is undefined if a glvalue referring to an object outside of its lifetime is bound to a reference to a virtual base class.

² [Example 1:

```
struct B {};
struct D : virtual B {
};

void f() {
    D d;
    d.~D();
    B& b = d;    // undefined behavior
}
```

— end example]

F.2.11**[ub:lifetime.outside.glvalue.dynamic.cast]****Specified in:** 6.8.4

- ¹ Behavior is undefined if a glvalue referring to an object outside of its lifetime is used as the operand of a `dynamic_cast` or as the operand of `typeid`.

² [Example 1:

```
struct B { virtual ~B(); };
struct D : virtual B {};
```

```

void f() {
    D d;
    B& br = d;
    d.~D();
    D& dr = dynamic_cast<D&>(br);    // undefined behavior
}

```

— end example]

F.2.12

[ub:original.type.implicit.destructor]

Specified in: 6.8.4

- ¹ The behavior is undefined if a non-trivial implicit destructor call occurs when the type of the object inhabiting the associated storage is not the original type associated with that storage.

- ² [Example 1:

```

class T {};

struct B {
    ~B();
};

void h() {
    B b;
    new (&b) T;
}    // undefined behavior at block exit

```

— end example]

F.2.13

[ub:creating.within.const.complete.obj]

Specified in: 6.8.4

- ¹ Creating a new object within the storage that a const complete object with static, thread, or automatic storage duration occupies, or within the storage that such a const object used to occupy before its lifetime ended, results in undefined behavior.

- ² [Example 1:

```

struct B {
    B();
    ~B();
};

const B b;

void h() {
    b.~B();
    new (const_cast<B*>(&b)) const B; // undefined behavior
}

```

— end example]

F.2.14

[ub:basic.indet.value]

Specified in: 6.8.5

- ¹ When the result of an evaluation is an indeterminate value (but not just an erroneous value) the behavior is undefined.

- ² [Example 1:

```

void g() {
    int x [[ indeterminate ]];
    int y = x;    // undefined behavior
}

```

— end example]

F.2.15**[ub:basic.stc.alloc.dealloc.constraint]****Specified in:** 6.8.6.5.1

- ¹ If the behavior of an allocation or deallocation function does not satisfy the semantic constraints specified in 6.8.6.5.2 and 6.8.6.5.3, the behavior is undefined.

² [Example 1:

```
#include <new>
void* operator new(std::size_t sz) {
    if (sz == 0)
        return nullptr;           // undefined behavior, should return non-null pointer

    return std::malloc(1);         // undefined behavior, if successful should return allocation with
                                   // length in bytes is at least as large as the requested size
}
void operator delete(void* ptr) noexcept {
    throw 0;                       // undefined behavior, terminates by throwing an exception
}
```

— end example]

F.2.16**[ub:basic.stc.alloc.zero.dereference]****Specified in:** 6.8.6.5.2

- ¹ The pointer returned when invoking an allocation function with a size of zero cannot be dereferenced.

² [Example 1:

```
void test()
{
    char* c = static_cast<char*>(operator new(0z));
    c[0] = 'X';           // undefined behavior
}
```

— end example]

F.2.17**[ub:basic.compound.invalid.pointer]****Specified in:** 6.9.5

- ¹ Indirection or the invocation of a deallocation function with a pointer value referencing storage that has been freed has undefined behavior. (Most other uses of such a pointer have implementation-defined behavior.)

² [Example 1:

```
void f()
{
    int *x = new int{5};
    delete x;
    int y = *x; // undefined behavior
    delete x;  // undefined behavior
}
```

— end example]

F.2.18**[ub:intro.execution.unsequenced.modification]****Specified in:** 6.10.1

- ¹ If a side effect on a memory location (6.8.1) is unsequenced relative to either another side effect on the same memory location or a value computation using the value of any object in the same memory location, and they are not potentially concurrent (6.10.2), the behavior is undefined.

² [Example 1:

```
void g(int i) {
    i = 7, i++, i++; // i becomes 9

    i = i++ + 1;    // the value of i is incremented
    i = i++ + i;    // undefined behavior
    i = i + 1;      // the value of i is incremented
}
```

— end example]

F.2.19

[ub:intro.races.data]

Specified in: 6.10.2.2

- ¹ The execution of a program contains a data race if it contains two potentially concurrent conflicting actions, at least one of which is not atomic, and neither happens before the other, except for the special case for signal handlers described in 6.10.2.2. Any such data race results in undefined behavior.

² [Example 1:

```
int count = 0;
auto f = [&] { count++; };
std::thread t1{f}, t2{f}, t3{f};
// undefined behavior t1, t2 and t3 have a data race on access of variable count
```

— end example]

F.2.20

[ub:intro.progress.stops]

Specified in: 6.10.2.3

- ¹ The behavior is undefined if a thread of execution that has not terminated stops making execution steps.

² [Example 1:

```
bool stop() { return false; }

void busy_wait_thread() {
    while (!stop()); // undefined behavior, thread makes no progress but the loop
} // is not trivial because stop() is not a constant expression
```

```
int main() {
    std::thread t(busy_wait_thread);
    t.join();
}
```

— end example]

F.2.21

[ub:basic.start.main.exit.during.destruction]

Specified in: 6.10.3.1

- ¹ If `std::exit` is called to end a program during the destruction of an object with static or thread storage duration, the program has undefined behavior.

² [Example 1:

```
#include <cstdlib>

struct Exiter {
    ~Exiter() { std::exit(0); }
};

Exiter ex;

int main() {}
// undefined behavior when destructor of static variable ex is called it will call std::exit
```

— end example]

F.2.22

[ub:basic.start.term.use.after.destruction]

Specified in: 6.10.3.4

- ¹ If a function contains a block-scope object of static or thread storage duration that has been destroyed and the function is called during the destruction of an object with static or thread storage duration, the program has undefined behavior if the flow of control passes through the definition of the previously destroyed block-scope object. Likewise, the behavior is undefined if the block-scope object is used indirectly (i.e., through a pointer) after its destruction.

² [Example 1:

```

struct A {};
void f() {
    static A a;
}

struct B {
    B() { f(); }
};
struct C {
    ~C() { f(); }
};

C c;
B b;    // call to f() in constructor begins lifetime of a

int main() {}
// undefined behavior, static objects are destructed in reverse order, in this case a then b and
// finally c. When the destructor of c is called, it calls f() which passes through the definition of
// previously destroyed block-scope object
— end example]
```

F.3 Clause 7: Expressions

[ub.expr]

F.3.1

[ub.expr.expr.eval]

Specified in: 7.1

- ¹ If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.

² [Example 1:

```

#include <limits>
int main() {
    // Assuming 32-bit int, the range of values is: -2,147,483,648 to 2,147,483,647.
    int x1 = std::numeric_limits<int>::max() + 1;
    // undefined behavior, 2,147,483,647 + 1 is not representable as an int
    int x2 = std::numeric_limits<int>::min() / -1;
    // undefined behavior, -2,147,483,648 / -1 is not representable as an int
}
— end example]
```

F.3.2

[ub.expr.basic.lvalue.strict.aliasing.violation]

Specified in: 7.2.1

- ¹ If a program attempts to access (3.1) the stored value of an object whose dynamic type is T through a glvalue whose type is not similar (7.3.6) to T (or its corresponding signed or unsigned types) the behavior is undefined.

² [Example 1:

```

int foo(float* f, int* i) {
    *i = 1;
    *f = 0.f;    // undefined behavior, glvalue is not similar to int

    return *i;
}

int main() {
    int x = 0;

    x = foo(reinterpret_cast<float*>(&x), &x);
}
— end example]
```

F.3.3**[ub:expr.basic.lvalue.union.initialization]****Specified in:** 7.2.1

- ¹ If a program invokes a defaulted copy/move constructor or defaulted copy/move assignment operator of a union with an argument that is not an object of a similar type within its lifetime, the behavior is undefined.

² [Example 1:

```
union U { int x; };
void f()
{
    char u[sizeof(U)];
    U o = reinterpret_cast<U&>(u);    // undefined behavior
}
```

— end example]

F.3.4**[ub:expr.type.reference.lifetime]****Specified in:** 7.2.2

- ¹ Evaluating a reference when an equivalent use of a pointer denoting the same object would be invalid has undefined behavior.

² [Example 1:

```
void g()
{
    int* ip = new int(5);
    int& i = *ip;
    delete ip;
    i;    // undefined behavior
}
```

— end example]

F.3.5**[ub:conv.lval.valid.representation]****Specified in:** 7.3.2

- ¹ Performing an lvalue-to-rvalue conversion on an object whose value representation is not valid for its type has undefined behavior.

² [Example 1:

```
bool f() {
    bool b = true;
    char c = 42;
    memcpy(&b, &c, 1);
    return b;    // undefined behavior if 42 is not a valid value representation for bool
}
```

— end example]

F.3.6**[ub:conv.double.out.of.range]****Specified in:** 7.3.10

- ¹ Converting a floating-point value to a type that cannot represent the value has undefined behavior.

² [Example 1:

```
#include <limits>

int main() {
    // Assuming 32-bit int, 32-bit float and 64-bit double.
    double d2 = std::numeric_limits<double>::max();
    float f = d2;    // undefined behavior on systems where the range of representable values
                    // of float is [-max,+max]; on systems where the range of representable
                    // values is [-inf,+inf] this would not be undefined behavior
    int i = d2;    // undefined behavior, the max value of double is not representable as int
}
```

— end example]

F.3.7**[ub:conv.fpint.float.not.represented]****Specified in:** 7.3.11

- ¹ When converting a floating-point value to an integer type, if the value is not representable in the destination type, the behavior is undefined.

² [Example 1:

```
#include <limits>

int main() {
    // Assuming 32-bit int, the range of values is: -2,147,483,648 to
    // 2,147,483,647. Assuming 32-bit float and 64-bit double.
    double d = (double)std::numeric_limits<int>::max() + 1;
    int x1 = d;    // undefined behavior, 2,147,483,647 + 1 is not representable as int
}
```

— end example]

F.3.8**[ub:conv.fpint.int.not.represented]****Specified in:** 7.3.11

- ¹ When converting a value of integer or unscoped enumeration type to a floating-point type, if the value is not representable in the destination type the behavior is undefined.

² [Example 1:

```
int main() {
    unsigned long long x2 = -1;
    float f = x2; // undefined behavior on systems where f does not include
                 // a representation for infinity and the maximum value for float
                 // is smaller than the maximum value for unsigned long long
}
```

— end example]

F.3.9**[ub:conv.ptr.virtual.base]****Specified in:** 7.3.12

- ¹ Converting a pointer to a derived class D to a pointer to a virtual base class B that does not point to a valid object within its lifetime has undefined behavior.

² [Example 1:

```
struct B {};
struct D : virtual B {};
void f()
{
    D ds[1];
    B* b = &ds[1];    // undefined behavior
}
```

— end example]

F.3.10**[ub:conv.member.missing.member]****Specified in:** 7.3.13

- ¹ The conversion of a pointer to a member of a base class to a pointer to member of a derived class that could not contain that member has undefined behavior.

² [Example 1:

```
struct B {};
struct D1 : B { int d1; };
struct D2 : B {};
void f()
{
    int (D1::*pd1) = &D1::d1;
    int (B::*pb) = static_cast<int (B::*)>(pd1);
    int (D2::*pd2) = pb;    // undefined behavior
}
```

— end example]

F.3.11

[ub:expr.call.different.type]

Specified in: 7.6.1.3

- ¹ Calling a function through an expression whose function type is not call-compatible with the function type of the called function's definition results in undefined behavior.

² [Example 1:

```
using f_float = int (*)(float);
using f_int = int (*)(int);

int f1(float) { return 10; }

int f2(int) { return 20; }

int main() {
    return reinterpret_cast<f_int>(f1)(20);    // undefined behavior, the function type of the expression
                                              // is different from the called function's definition
}
```

— end example]

F.3.12

[ub:expr.ref.member.not.similar]

Specified in: 7.6.1.5

- ¹ In a class member access E1.E2, where E2 is a non-static member, the behavior is undefined if E1 refers to an object whose type is not similar to the type of the expression E1.

² [Example 1:

```
struct A { int i; };
struct B { int j; };
A a;
int x = reinterpret_cast<B&>(a).j;    // undefined behavior
```

— end example]

F.3.13

[ub:expr.dynamic.cast.pointer.lifetime]

Specified in: 7.6.1.7

- ¹ Evaluating a `dynamic_cast` on a non-null pointer that points to an object (of polymorphic type) of the wrong type or to an object not within its lifetime has undefined behavior.

² [Example 1:

```
struct B { virtual ~B(); };
void f() {
    B bs[1];
    B* dp = dynamic_cast<B*>(bs+1);    // undefined behavior
}
```

— end example]

F.3.14

[ub:expr.dynamic.cast.glvalue.lifetime]

Specified in: 7.6.1.7

- ¹ Evaluating a `dynamic_cast` on a reference that denotes an object (of polymorphic type) of a type that is not similar to the referenced type or an object not within its lifetime has undefined behavior.

² [Example 1:

```
struct B { virtual ~B(); };
void f() {
    B bs[1];
    B& dr = dynamic_cast<B&>(bs[1]);    // undefined behavior
}
```

— end example]

F.3.15**[ub:expr.static.cast.base.class]****Specified in:** 7.6.1.9

- ¹ A glvalue of type B can be cast to the type “reference to D” if B is a base class of D, otherwise the behavior is undefined.

² [Example 1:

```
struct B {};
struct D1 : B {};
struct D2 : B {};

void f() {
    D1 d;
    B &b = d;
    static_cast<D2 &>(b); // undefined behavior, base class object of type D1 not D2
}
```

— end example]

F.3.16**[ub:expr.static.cast.enum.outside.range]****Specified in:** 7.6.1.9

- ¹ If the enumeration type does not have a fixed underlying type, the value is unchanged if the original value is within the range of the enumeration values (9.8.1), and otherwise, the behavior is undefined.

² [Example 1:

```
enum A { e1 = 1, e2 };

void f() {
    enum A a = static_cast<A>(4); // undefined behavior, 4 is not within the range of enumeration values
}
```

— end example]

F.3.17**[ub:expr.static.cast.fp.outside.range]****Specified in:** 7.6.1.9

- ¹ An explicit conversion of a floating-point value that is outside the range of the target type has undefined behavior.

² [Example 1: If float does not adhere to ISO/IEC 60559 and cannot represent positive infinity, a sufficiently large double value will be outside the (finite) range of float.

```
void f() {
    double d = FLT_MAX;
    d *= 16;
    float f = static_cast<float>(d); // undefined behavior
}
```

— end example]

F.3.18**[ub:expr.static.cast.downcast.wrong.derived.type]****Specified in:** 7.6.1.9

- ¹ Casting from a pointer to a base class to a pointer to a derived class when there is no enclosing object of that derived class at the specified location has undefined behavior.

² [Example 1:

```
struct B {};
struct D1 : B {};
struct D2 : B {};

void f() {
    B *bp = new D1;
    static_cast<D2 *>(bp); // undefined behavior, base class object of type D1 not D2
}
```

— end example]

F.3.19**[ub:expr.static.cast.does.not.contain.original.member]****Specified in:** 7.6.1.9

- ¹ A pointer to member of derived class D can be cast to a pointer to member of base class B (with certain restrictions on *cv* qualifiers) as long as B contains the original member, or is a base or derived class of the class containing the original member; otherwise the behavior is undefined.

² [Example 1:

```

    struct A {
        char c;
    };

    struct B {
        int i;
    };

    struct D : A, B {};

    int main() {
        char D::*p1 = &D::c;
        char B::*p2 = static_cast<char B::*>(p1); // undefined behavior, B does not contain the original member
        c
    }

```

— end example]

F.3.20**[ub:expr.unary.dereference]****Specified in:** 7.6.2.2

- ¹ Dereferencing a pointer that does not point to an object or function has undefined behavior.

² [Example 1:

```

    int f()
    {
        int *p = nullptr;
        return *p;    // undefined behavior
    }

```

— end example]

F.3.21**[ub:expr.new.non.allocating.null]****Specified in:** 7.6.2.8

- ¹ If the allocation function is a non-allocating form (17.6.3.4) that returns null, the behavior is undefined.

² [Example 1:

```

#include <new>
[[nodiscard]] void* operator new(std::size_t size, void* ptr) noexcept {
    return nullptr;    // undefined behavior, should return non-null pointer
}

```

— end example]

[Example 2:

```

#include <new>

struct A {
    int x;
};

int main() {
    char *p = nullptr;
    A *a = new (p) A;    // undefined behavior, non-allocating new returning nullptr
}

```

— end example]

F.3.22**[ub:expr.delete.mismatch]****Specified in:** 7.6.2.9

¹ Using array delete on the result of a single object new expression has undefined behavior.

² [Example 1:

```
int* x = new int;
delete[] x;           // undefined behavior, allocated using single object new expression
— end example]
```

F.3.23**[ub:expr.delete.array.mismatch]****Specified in:** 7.6.2.9

¹ Using single object delete on the result of an array new expression has undefined behavior.

² [Example 1:

```
int* x = new int[10];
delete x;           // undefined behavior, allocated using array new expression
— end example]
```

F.3.24**[ub:expr.delete.dynamic.type.differ]****Specified in:** 7.6.2.9

¹ If the static type of the object to be deleted is different from its dynamic type and the selected deallocation function is not a destroying operator delete, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined.

² [Example 1:

```
struct B {
    int a;
};

struct D : public B {
    int b;
};

void f() {
    B* b = new D;
    delete b;           // undefined behavior, no virtual destructor
}
— end example]
```

F.3.25**[ub:expr.delete.dynamic.array.dynamic.type.differ]****Specified in:** 7.6.2.9

¹ In an array delete expression, if the dynamic type of the object to be deleted differs from its static type, the behavior is undefined.

² [Example 1:

```
struct B {
    virtual ~B();
    void operator delete[](void*, std::size_t);
};

struct D : B {
    void operator delete[](void*, std::size_t);
};

void f(int i) {
    D* dp = new D[i];
    delete[] dp;         // uses D::operator delete[](void*, std::size_t)
    B* bp = new D[i];
    delete[] bp;         // undefined behavior
}
```

— end example]

F.3.26

[ub:expr.mptr.oper.not.contain.member]

Specified in: 7.6.4

- ¹ Abbreviating *pm-expression*.**cast-expression* as *E1.*E2*, *E1* is called the object expression. If the dynamic type of *E1* does not contain the member to which *E2* refers, the behavior is undefined.

- ² [Example 1:

```
struct B {};
struct D : B {
    int x;
};

void f() {
    B *b = new B;
    D *d = static_cast<D *>(b);
    int D::*p = &D::x;
    (*d).*p = 1;           // undefined behavior, dynamic type B does not contain x
}
```

— end example]

F.3.27

[ub:expr.mptr.oper.member.func.null]

Specified in: 7.6.4

- ¹ If the second operand in a *.** expression is the null member pointer value (7.3.13), the behavior is undefined.

- ² [Example 1:

```
struct S {
    int f();
};

void f() {
    S cs;
    int (S::*pm)() = nullptr;
    (cs.*pm)();           // undefined behavior, the second operand is null
}
```

— end example]

F.3.28

[ub:expr.mul.div.by.zero]

Specified in: 7.6.5

- ¹ Division by zero has undefined behavior.

- ² [Example 1:

```
int main() {
    int x = 1 / 0;           // undefined behavior, division by zero
    double d = 1.0 / 0.0;    // undefined behavior on systems where the range of
                             // representable values of double is [-max,+max], on systems where
                             // representable values is [-inf,+inf] this would not be undefined behavior
}
```

— end example]

F.3.29

[ub:expr.mul.representable.type.result]

Specified in: 7.6.5

- ¹ If the quotient *a/b* is representable in the type of the result, (*a/b*)**b* + *a*%*b* is equal to *a*; otherwise, the behavior of both *a/b* and *a*%*b* is undefined.

- ² [Example 1:

```
#include <limits>
```

```
int main() {
    int x = std::numeric_limits<int>::min() / -1;
    // Assuming LP64 -2,147,483,648 which when divided by -1
    // gives us 2,147,483,648 which is not representable by int.
}
—end example]
```

F.3.30**[ub:expr.add.out.of.bounds]****Specified in:** 7.6.6¹ Creating an out of bounds pointer has undefined behavior.² [Example 1:

```
static const int arrs[10]{};

int main() {
    const int *y = arrs + 11;           // undefined behavior, creating an out of bounds pointer
}
—end example]
```

[Example 2:

```
static const int arrs[10][10]{};

int main() {
    const int(*y)[10] = arrs + 11;     // undefined behavior, creating an out of bounds pointer
}
—end example]
```

F.3.31**[ub:expr.add.sub.diff.pointers]****Specified in:** 7.6.6¹ Subtracting pointers that are not part of the same array has undefined behavior.² [Example 1:

```
#include <cstddef>
void f() {
    int x[2];
    int y[2];
    int* p1 = x;
    int* p2 = y;
    std::ptrdiff_t off = p1 - p2; // undefined behavior, p1 and p2 point to different arrays
}
—end example]
```

F.3.32**[ub:expr.add.not.similar]****Specified in:** 7.6.6¹ For addition or subtraction of two expressions *P* and *Q*, if *P* or *Q* have type “pointer to *cv* *T*”, where *T* and the array element type are not similar (7.3.5), the behavior is undefined.² [Example 1:

```
struct S { int i; };
struct T : S { double d; };

void f(const S* s, std::size_t count) {
    for (const S* end = s + count; s != end; ++s) {
        // undefined behavior, T is not similar to S but s points to a T[]
        /* ... */
    }
}
```

```
int main() {
    T test[5];
    f(test, 5);
}
```

— end example]

F.3.33

[ub:expr.shift.neg.and.width]

Specified in: 7.6.7

- ¹ Shifting by a negative amount or equal or greater than the bit-width of a type has undefined behavior.

- ² [Example 1:

```
int y = 1 << -1;           // undefined behavior, shift is negative

static_assert(sizeof(int) == 4 && CHAR_BIT == 8);
int y1 = 1 << 32;          // undefined behavior, shift is equal to the bit width of int
int y2 = 1 >> 32;          // undefined behavior, shift is equal to the bit width of int
```

— end example]

F.3.34

[ub:expr.assign.overlap]

Specified in: 7.6.19

- ¹ Overlap in the storage between the source and destination can result in undefined behavior.

- ² [Example 1:

```
int x = 1;
char* c = reinterpret_cast<char*>(&x);
x = *c;           // undefined behavior, source overlaps storage of destination
```

— end example]

F.4 Clause 8: Statements

[ub.stmt]

F.4.1

[ub:stmt.return.flow.off]

Specified in: 8.8.4

- ¹ Flowing off the end of a function other than `main` or a coroutine results in undefined behavior if the return type is not *cv void*.

- ² [Example 1:

```
int f(int x) {
    if (x)
        return 1;
    // undefined behavior if x is 0
}

void b() {
    int x = f(0); // undefined behavior, using 0 as an argument will cause f(...) to flow
                 // off the end without a return statement
}
```

— end example]

F.4.2

[ub:stmt.return.coroutine.flow.off]

Specified in: 8.8.5

- ¹ Flowing off the end of a coroutine function body that does not return `void` has undefined behavior.

- ² [Example 1:

```
#include <cassert>
#include <coroutine>
#include <iostream>
#include <vector>
```



```

class resumable {
public:
    struct promise_type;
    using coro_handle = std::coroutine_handle<promise_type>;
    resumable(coro_handle handle) : handle_(handle) { assert(handle); }
    resumable(resumable&) = delete;
    resumable(resumable&&) = delete;
    bool resume() {
        if (not handle_.done())
            handle_.resume();
        return not handle_.done();
    }
    ~resumable() { handle_.destroy(); }
    const char* return_val();

private:
    coro_handle handle_;
};

struct resumable::promise_type {
    using coro_handle = std::coroutine_handle<promise_type>;
    const char* string_;
    auto get_return_object() { return coro_handle::from_promise(*this); }
    auto initial_suspend() { return std::suspend_always(); }
    auto final_suspend() noexcept { return std::suspend_always(); }
    void unhandled_exception() { std::terminate(); }
    void return_value(const char* string) { string_ = string; }
};

const char* resumable::return_val() {
    return handle_.promise().string_;
}

resumable foo() {
    std::cout << "Hello" << std::endl;
    co_await std::suspend_always();
    // undefined behavior, falling off the end of coroutine that does not return void
}

int main() {
    resumable res = foo();
    while (res.resume())
        ;
    std::cout << res.return_val() << std::endl;
}

```

— end example]

F.4.3

[ub:stmt.dcl.local.static.init.recursive]

Specified in: 8.10

¹ If control re-enters the declaration recursively while the variable is being initialized, the behavior is undefined.

² [Example 1:

```

int foo(int i) {
    static int s = foo(2 * i);    // undefined behavior, recursive call
    return i + 1;
}

```

— end example]

F.5 Clause 9: Declarations

[ub:dcl]

F.5.1

[ub:dcl.type.cv.modify.const.obj]

Specified in: 9.2.9.2

¹ Any attempt to modify a const object during its lifetime results in undefined behavior.

² [Example 1:

```
const int* ciq = new const int(3); // initialized as required
int* iq = const_cast<int*>(ciq); // cast required
*iq = 4; // undefined behavior, modifies a const object
```

— end example]

F.5.2

[ub:dcl.type.cv.access.volatile]

Specified in: 9.2.9.2

¹ If an attempt is made to access an object defined with a volatile-qualified type through the use of a non-volatile glvalue, the behavior is undefined

² [Example 1:

```
volatile int x = 0;
int& y = const_cast<int&>(x);
std::cout << y; // undefined behavior, accessing volatile through non-volatile glvalue
```

— end example]

F.5.3

[ub:dcl.ref.incompatible.function]

Specified in: 9.3.4.3

¹ Initializing a reference to a function with a value that is a function that is not call-compatible (7.6.1.3) with the type of the reference has undefined behavior.

² [Example 1:

```
void f(float x);
void (&g)(int) = reinterpret_cast<void (&)(int)>(f); // undefined behavior
```

— end example]

F.5.4

[ub:dcl.ref.incompatible.type]

Specified in: 9.3.4.3

¹ Initializing a reference to an object with a value that is not type-accessible (7.2.1) through the type of the reference has undefined behavior.

² [Example 1:

```
float g;
int& i = reinterpret_cast<int&>(g); // undefined behavior
```

— end example]

F.5.5

[ub:dcl.ref.uninitialized.reference]

Specified in: 9.3.4.3

¹ Evaluating a reference prior to initializing that reference has undefined behavior.

² [Example 1:

```
extern int &ir1;
int i2 = ir1; // undefined behavior, ir1 not yet initialized
int i3 = 17;
int &ir1 = i3;
```

— end example]

F.5.6

[ub:dcl.fct.def.coroutine.resume.not.suspended]

Specified in: 9.6.4

¹ Invoking a resumption member function for a coroutine that is not suspended results in undefined behavior.

² [Example 1:

```
#include <coroutine>
```

```

struct minig {
    struct promise_type {
        int val;
        minig get_return_object() { return { *this }; }
        constexpr suspend_always initial_suspend() noexcept { return {}; }
        constexpr suspend_always final_suspend() noexcept { return {}; }
        constexpr void return_void() noexcept {}
        [[noreturn]] void unhandled_exception() noexcept { throw; }
        suspend_always yield_value(int v) noexcept {
            val = v;
            return {};
        }
    };
    using HDL = coroutine_handle<promise_type>;
    HDL coro;
    minig(promise_type& p) : coro(HDL::from_promise(p)) {}
    ~minig() { coro.destroy(); }
    bool move_next() {
        coro.resume();
        return !coro.done();
    }
    int current_value() { return coro.promise().val; }
};

static minig f(int n) {
    for (int i = 0; i < n; ++i)
        co_yield i;
}

int main() {
    auto g = f(10);
    int sum = 0;
    while (g.move_next())
        sum += g.current_value();

    g.move_next();    // undefined behavior, will call coro.resume() but final_suspend
                     // has already returned, even though it returned suspend_always

    return sum;
}
— end example]

```

F.5.7**[ub:dcl.fct.def.coroutine.destroy.not.suspended]**

Specified in: 9.6.4

¹ Invoking `destroy()` on a coroutine that is not suspended has undefined behavior.² [Example 1:

```

#include <coroutine>

struct minig {
    struct promise_type {
        int val;
        minig get_return_object() { return { *this }; }
        constexpr suspend_always initial_suspend() noexcept { return {}; }
        constexpr suspend_always final_suspend() noexcept { return {}; }
        constexpr void return_void() noexcept {}
        [[noreturn]] void unhandled_exception() { throw; }
        suspend_always yield_value(int v) noexcept {
            val = v;
            return {};
        }
    };
};
using HDL = coroutine_handle<promise_type>;

```

```

HDL coro;
minig(promise_type& p) : coro(HDL::from_promise(p)) {}
~minig() { coro.destroy(); }
bool move_next() {
    coro.resume();
    return !coro.done();
}
int current_value() {
    // this coroutine is not suspended therefore a call to destroy has undefined behavior
    coro.destroy();
    return coro.promise().val;
}
};

static minig f(int n) {
    for (int i = 0; i < n; ++i)
        co_yield i;
}

int main() {
    auto g = f(10);
    int sum = 0;
    while (g.move_next())
        sum += g.current_value();

    return sum;
}
— end example]

```

F.5.8

[ub:dcl.attr.assume.false]

Specified in: 9.13.3

- ¹ If an assumption expression would not evaluate to true at the point where it appears the behavior is undefined.
- ² [Example 1:

```

int g(int x) {
    [[assume(x >= 0)]];
    return x/32;
}

int f() {
    return g(-10);    // undefined behavior, assumption in g is false
}
— end example]

```

F.5.9

[ub:dcl.attr.noreturn.eventually.returns]

Specified in: 9.13.10

- ¹ If a function **f** is called where **f** was previously declared with the **noreturn** attribute and **f** eventually returns, the behavior is undefined.
- ² [Example 1:

```

[[noreturn]] void f(int i) {
    if (i > 0)
        throw "positive";
}

int main() {
    f(0);    // undefined behavior, returns from a [[noreturn]] function
}
— end example]

```

F.6 Clause 11: Classes**[ub.class]****F.6.1****[ub:class.dtor.no.longer.exists]****Specified in:** 11.4.7

- ¹ Once a destructor is invoked for an object, the object's lifetime has ended; the behavior is undefined if the destructor is invoked for an object whose lifetime has ended.

² [Example 1:

```

    struct A {
        ~A() {}
    };

    int main() {
        A a;
        a.~A();
    } // undefined behavior, lifetime of a already ended before implicit destructor
— end example]
```

F.6.2**[ub:class.abstract.pure.virtual]****Specified in:** 11.7.4

- ¹ Calling a pure virtual function from a constructor or destructor in an abstract class has undefined behavior.

² [Example 1:

```

    struct B {
        virtual void f() = 0;
        B() {
            f(); // undefined behavior, f is pure virtual and is being called from the constructor
        }
    };

    struct D : B {
        void f() override;
    };
— end example]
```

F.6.3**[ub:class.base.init.mem.fun]****Specified in:** 11.9.3

- ¹ Calling a member function before all the *mem-initializers* for base classes have completed has undefined behavior.

² [Example 1:

```

    class A {
    public:
        A(int);
    };

    class B : public A {
        int j;

    public:
        int f();
        B()
            : A(f()), // undefined behavior, calls member function but base A not yet initialized
              j(f()) {} // defined, bases are all initialized
    };

    class C {
    public:
        C(int);
    };

```

```

class D : public B, C {
    int i;

public:
    D()
        : C(f()),          // undefined behavior, calls member function but base C not yet initialized
          i(f()) {}        // defined, bases are all initialized
};
— end example]

```

F.6.4**[ub:class.ctor.before.ctor]****Specified in:** 11.9.5

- ¹ For an object with a non-trivial constructor, referring to any non-static member or base class of the object before the constructor begins execution results in undefined behavior.

² [Example 1:

```

struct W {
    int j;
};
struct X : public virtual W {};
struct Y {
    int *p;
    X x;
    Y() : p(&x.j) {          // undefined behavior, x is not yet constructed
    }
};
— end example]

```

F.6.5**[ub:class.ctor.after.dtor]****Specified in:** 11.9.5

- ¹ For an object with a non-trivial destructor, referring to any non-static member or base class of the object after the destructor finishes execution has undefined behavior.

² [Example 1:

```

struct X {
    int i;
    ~X();          // non-trivial
};
X& g()
{
    static X x;
    return x;
}
void f()
{
    X* px = &g();
    px->~X();
    int j = px->i;  // undefined behavior
}
— end example]

```

F.6.6**[ub:class.ctor.convert.pointer]****Specified in:** 11.9.5

- ¹ Converting a pointer to a base class of an object, when construction of that object has not started or destruction of that object has finished, has undefined behavior.

² [Example 1:

```

struct A { };
struct B : virtual A { };
struct C : B { };

```

```

struct D : virtual A { D(A*); };
struct X { X(A*); };

struct E : C, D, X {
    E() : D(this),      // undefined behavior, upcast from E* to A* might use path E* → D* → A*
                        // but D is not constructed

                        // “D((C*)this)” would be defined, E* → C* is defined because E() has started,
                        // and C* → A* is defined because C is fully constructed

    X(this) {}          // defined, upon construction of X, C/B/D/A sublattice is fully constructed
};
— end example]

```

F.6.7**[ub:class.cdtor.form.pointer]****Specified in:** 11.9.5

- ¹ When forming a pointer to a direct non-static member of a class, construction must have started and destruction must not have finished otherwise the behavior is undefined.

² [Example 1:

```

struct A {
    int i = 0;
};
struct B {
    int *p;
    A a;
    B() : p(&a.i) {} // undefined behavior
};
— end example]

```

F.6.8**[ub:class.cdtor.virtual.not.x]****Specified in:** 11.9.5

- ¹ When a virtual function is called directly or indirectly from a constructor or from a destructor, including during the construction or destruction of the class's non-static data members, and the object to which the call applies is the object (call it **x**) under construction or destruction, the function called is the final overrider in the constructor's or destructor's class and not one overriding it in a more-derived class. If the virtual function call uses an explicit class member access (7.6.1.5) and the object expression refers to the complete object of **x** or one of that object's base class subobjects but not **x** or one of its base class subobjects, the behavior is undefined.

² [Example 1:

```

struct V {
    virtual void f();
    virtual void g();
};

struct A : virtual V {
    virtual void f();
};

struct B : virtual V {
    virtual void g();
    B(V *, A *);
};

struct D : A, B {
    virtual void f();
    virtual void g();
    D() : B((A *)this, this) {}
};

```

```

B::B(V *v, A *a) {
    f();           // calls V::f, not A::f
    g();           // calls B::g, not D::g
    v->g();         // v is base of B, the call is defined, calls B::g
    a->f();         // undefined behavior, a's type not a base of B
}

```

— end example]

F.6.9

[ub:class.ctor.typeid]

Specified in: 11.9.5

- ¹ If the operand of `typeid` refers to the object under construction or destruction and the static type of the operand is neither the constructor or destructor's class nor one of its bases, the behavior is undefined.

- ² [Example 1:

```

struct V {
    virtual void f();
};

struct A : virtual V {};
struct B : virtual V {
    B(V *, A *);
};

struct D : A, B {
    D() : B((A *)this, this) {}
};

B::B(V *v, A *a) {
    typeid(*this);           // std::type_info for B
    typeid(*v);              // defined, *v has type V, a base of B yields std::type_info for B
    typeid(*a);              // undefined behavior, type A not a base of B
}

```

— end example]

F.6.10

[ub:class.ctor.dynamic.cast]

Specified in: 11.9.5

- ¹ If the operand of the `dynamic_cast` refers to the object under construction or destruction and the static type of the operand is not a pointer to or object of the constructor or destructor's own class or one of its bases, the `dynamic_cast` results in undefined behavior.

- ² [Example 1:

```

struct V {
    virtual void f();
};

struct A : virtual V {};
struct B : virtual V {
    B(V *, A *);
};

struct D : A, B {
    D() : B((A *)this, this) {}
};

B::B(V *v, A *a) {
    dynamic_cast<B *>(v);           // defined, v of type V*, V base of B results in B*
    dynamic_cast<B *>(a);           // undefined behavior, a has type A*, A not a base of B
}

```

— end example]

F.7 Clause 13: Templates**[ub.temp]****F.7.1****[ub:temp.inst.inf.recursion]****Specified in:** 13.9.2¹ The result of an infinite recursion in template instantiation is undefined.² [Example 1:

```

template <class T>
class X {
    X<T> *p;    // OK
    X<T *> a;    // implicit instantiation of X<T> requires
                // the implicit instantiation of X<T*> which requires
                // the implicit instantiation of X<T**> which ...
};

int main() {
    X<int> x;    // undefined behavior, instantiation will lead to infinite recursion
}

```

— end example]

F.8 Clause 14: Exception handling**[ub.except]****F.8.1****[ub:except.handle.handler.ctor.dtor]****Specified in:** 14.4¹ Referring to any non-static member or base class of an object in the handler for a *function-try-block* of a constructor or destructor for that object results in undefined behavior.² [Example 1:

```

#include <iostream>

struct A {
    A() try : x(0 ? 1 : throw 1) {
    } catch (int) {
        std::cout << "y: " << y << std::endl;    // undefined behavior, referring to non-static member y in
                                                // the handler of function-try-block
    }
    int x;
    int y = 42;
};

```

— end example]

Annex G

(informative)

Ill-formed, no diagnostic required [ifndr]

G.1 General [ifndr.general]

This Annex documents rules for which no diagnostic is required, called out in [Clause 4](#) through [Clause 15](#) using the following phrases:

- no diagnostic is required
- no diagnostic required
- a diagnostic is required only if

Each entry contains a title, a cross-reference, a summary of the circumstances, and code examples. The code examples are not intended to exhaustively cover all possible ways of invoking that case.

G.2 Clause 5: Lexical conventions [ifndr.lex]

G.2.1 [ifndr:lex.name.reserved]

Specified in: [5.11](#)

¹ Using an identifier reserved for use by C++ is ill-formed, no diagnostic required.

² [Example 1:

```
int _z; // IFNDR, _z is reserved because it starts with _ at global scope

int main() {
    int __x;    // IFNDR, __x is reserved because it starts with __
    int _Y;     // IFNDR, _Y is reserved because it starts with _ followed by a capital letter
    int x__y;   // IFNDR, x__y is reserved because it contains __
}
```

— end example]

G.3 Clause 6: Basics [ifndr.basic]

G.3.1 [ifndr:basic.link.consistent.types]

Specified in: [6.7](#)

¹ Having multiple declarations of the same entity with different kinds when those declarations are not reachable from one another is ill-formed, no diagnostic required.

² [Example 1:

Module interface of M:

```
void g();           // #1
void h();           // #2
template <typename T> int j; // #3
```

Module interface of N:

```
int g();           // same entity as #1, different type
namespace h {}     // same entity as #2, not both namespaces
template <int X> int j; // same entity as #3, non-equivalent template heads
```

Other translation unit:

```
import M;
import N;
// IFNDR due to the mismatched pairs above
```

— end example]

G.3.2

[ifndr:basic.def.odr.minimum.one.def]

Specified in: 6.3

- ¹ Not having a definition for a function or variable that is odr-used from a non-discarded statement (8.5.2) is ill-formed, no diagnostic required.

- ² [Example 1:

```
auto f() {
    struct A {};
    return A{};
}
decltype(f()) g();
auto x = g();    // IFNDR, function g is used but not defined in this translation unit, and cannot
                  // be defined in any other translation unit because its type does not have linkage
```

— end example]

G.3.3

[ifndr:basic.def.odr.injected.match]

Specified in: 6.3

- ¹ Defining a definable item D with an injected declaration (7.7.6) in one translation unit and a definition in a different translation unit when D is not attached to a named module or neither definition is reachable from the other is ill-formed, no diagnostic required.

- ² [Example 1:

Translation unit #1:

```
struct S;
constexpr { std::meta::define_aggregate(S, {}); }    // #1
```

Translation unit #2:

```
struct S {};    // IFNDR, definition here, injected declaration at #1
```

— end example]

G.3.4

[ifndr:basic.def.odr.maximum.one.def]

Specified in: 6.3

- ¹ If there are definitions in different translation units of a non-inline non-templated function or variable that are not attached to a named module or are not reachable from one another, the program is ill-formed, no diagnostic required.

- ² [Example 1:

Translation unit #1:

```
export module M:A;    // module partition
void f() {}          // #1
void g() {}          // #2
```

Translation unit #2:

```
export module M:B;    // module partition
void f() {}          // IFNDR, #1 not reachable
```

Translation unit #3:

```
export module M;    // primary module interface unit
export import :A;
void g();           // error: #2 is reachable
```

— end example]

G.3.5

[ifndr:basic.def.odr.definition.matches]

Specified in: 6.3

- ¹ If there are definitions in different translation units of a definable item D where

- (1.1) — D is not defined by an injected declaration (7.7.6),
 - (1.2) — D is not an inline or templated function or variable, and
 - (1.3) — D is not attached to a named module or the declarations are not reachable from one another,
- that do not satisfy the matching rules described in 6.3, the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
inline void f() {}           // #1
inline void g() {}           // #2
inline void h() {[]{}();}    // #3
namespace { int i = 0; }
inline void j() {++i;}        // #4
```

Translation unit #2:

```
inline void f() {}           // OK, same as #1
inline void g() {};          // IFNDR, different sequence of tokens than #2
inline void h() {[]{}();}    // IFNDR, closure has different type than #3
namespace { int i = 0; }
inline void j() {++i;}        // IFNDR, i refers to different entity than in #4
```

— end example]

G.3.6

[ifndr:basic.def.odr.unnamed.enum.same.type]

Specified in: 6.3

- ¹ Having multiple unnamed enumeration definitions in the same scope that have the same first enumerator name and do not have typedef names for linkage purposes (9.8.1) that are not the same enumeration is ill-formed, no diagnostic required.

² [Example 1:

Source file "a.h":

```
enum { a };
```

Source file "b.h":

```
enum { a, b };
```

Source file "main.cpp":

```
import "a.h";
import "b.h";
auto n = decltype(a)::b;           // IFNDR, more than one unnamed enum definition reachable at
                                   // this point but their types are not the same
```

— end example]

G.3.7

[ifndr:basic.contract.vastart.contract.predicate]

Specified in: 6.11.1

- ¹ The use of `va_start` (17.14.2) within the predicate of a contract assertion is ill-formed, no diagnostic required.

² [Example 1:

```
void f(...)
{
    va_list args;
    contract_assert((va_start(const_cast<decltype(args)>(args)), true)) // IFNDR
}
```

— end example]

G.3.8

[ifndr:basic.contract.handler.replacing.nonreplaceable]

Specified in: 6.11.3

- ¹ On platforms where the contract-violation handler is not replaceable (9.6.5) a function declaration which could be such a replacement function is ill-formed, no diagnostic required.

² [Example 1:

```
#include <contracts>
void handle_contract_violation(const std::contracts::contract_violation& violation);
    // IFNDR, if contract-violation handler is not replaceable
— end example]
```

G.3.9

[ifndr:class.member.lookup.name.refers.diff.decl]

Specified in: 6.5.2

¹ A name *N* used in a class *S* referring to a different declaration when resolved in its context than when re-evaluated in the completed scope of *S* is ill-formed, no diagnostic required.

² [Example 1:

```
struct foo {};

struct bar {
    foo *m_foo;

    foo *foo() {
        return m_foo;
    } // IFNDR, foo now refers to member function foo() while previously referred to struct foo
};
— end example]
```

[Example 2:

```
struct B {
    static int f();
};

struct D : public B {
    using B::f;
    int g(decltype(f()) x) {
        return 0;
    } // IFNDR, decltype(f()) will refer to B::f() here but if
    // moved to the end of D it would refer to D::f()
    static float f();
};

int main() {
    D d;

    return d.g(0);
}
— end example]
```

G.4 Clause 7: Expressions

[ifndr.expr]

G.4.1

[ifndr:expr.prim.req.always.sub.fail]

Specified in: 7.5.8.1

¹ If the substitution of template arguments into a *requirement* would always result in a substitution failure, the program is ill-formed; no diagnostic required.

² [Example 1:

```
template <typename T> concept C = requires {
    new int[-(int)sizeof(T)]; // IFNDR, the size of the allocation is required to be greater
    // than zero but can never be
};
— end example]
```

G.5 Clause 8: Statements**[ifndr.stmt]****G.5.1****[ifndr:stmt.ambig.bound.diff.parse]****Specified in:** 8.11

- ¹ If, during parsing, a name in a template parameter is bound differently than it would be bound during a trial parse, the program is ill-formed. No diagnostic is required.

² [Example 1:

```
template <int N> struct A { const static int a = 20; };

template <> struct A<100> { using a = char; };

const int x = 10;

int main() {
    using T = const int;
    T(x)
    (100), (y)(A<x>::a); // IFNDR, during trial parse the template parameter x is bound to
                        // the global x later during parsing the template parameter x
                        // is bound to the local x declared on the same line
}
```

— end example]

G.6 Clause 9: Declarations**[ifndr.dcl]****G.6.1****[ifndr:dcl.constinit.specifier.not.reachable]****Specified in:** 9.2.7

- ¹ If the initializing declaration of a variable without the `constinit` specifier has the `constinit` specifier applied to declarations that are not reachable from that initializing declaration, the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
int x = 5; // initializing declaration of x
```

Translation unit #2:

```
extern constinit int x; // IFNDR, not reachable from initializing declaration of x
```

— end example]

G.6.2**[ifndr:dcl.inline.missing.on.definition]****Specified in:** 9.2.8

- ¹ If a function or variable with external or module linkage is declared inline but there is no inline declaration reachable from the end of some definition domain the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
inline int f();
```

Translation unit #2:

```
int f() { return 17; }
// IFNDR, end of definition domain but no inline declaration of f is reachable
```

— end example]

G.6.3**[ifndr:dcl.fct.default.inline.same.defaults]****Specified in:** 9.3.4.7

- ¹ If the accumulated set of default arguments for a given inline function with definitions in multiple translation units is different at the end of different translation units, the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
inline int f(int x,    int y = 2);
inline int f(int x = 1, int y);
// IFNDR, default arguments of f are 1 and 2
```

Translation unit #2:

```
inline int f(int x = 3, int y = 4);
// IFNDR, default arguments of f are 3 and 4
— end example]
```

G.6.4 [ifndr:dcl.contract.func.mismatched.contract.specifiers]

Specified in: 9.4.1

- ¹ If two different first declarations of a function (which must therefore not be reachable from one another) do not have equivalent function contract specifiers the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
int f(int x) pre(x >= 0); // IFNDR, pre present, not present in the other first declaration of f
int g(int x) pre(x == 0); // IFNDR, pre differs from the other first declaration of g
int h(int x) pre(x <= 0); // OK, pre equivalent to pre on the other first declaration of h
```

Translation unit #2:

```
int f(int x); // IFNDR, pre not present, present in the other first declaration of f
int g(int x) pre(x != 0); // IFNDR, pre differs from the other first declaration of g
int h(int y) pre(y <= 0); // OK, pre equivalent to pre on the other first declaration of h
— end example]
```

G.6.5 [ifndr:dcl.fct.def.replace.bad.replacement]

Specified in: 9.6.5

- ¹ A declaration of a replaceable function that is inline, not attached to the global module, does not have C++ language linkage, does not have the required return type, or is not a valid redeclaration of the corresponding declaration in a standard library header (if there is one) then the program is ill-formed, no diagnostic required.

² [Example 1:

```
extern "C" // IFNDR, wrong language linkage
inline    // IFNDR, inline
int       // IFNDR, wrong return type
handle_contract_violation(const std::contracts::contract_violation&) {}

void* operator new(decltype(sizeof(0))) noexcept; // IFNDR, mismatched exception specification to
// declaration in <new>
```

— end example]

G.6.6 [ifndr:dcl.link.mismatched.language.linkage]

Specified in: 9.12

- ¹ If two declarations of an entity do not have the same language linkage and neither is reachable from the other the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
extern "C" { void f(); }
```

Translation unit #2:

```
extern "C++" { void f(); } // IFNDR, different language linkage
— end example]
```

G.6.7**[ifndr:dcl.align.diff.translation.units]****Specified in:** 9.13.2

- ¹ No diagnostic is required if declarations of an entity have different *alignment-specifiers* in different translation units.

² [Example 1:

Translation unit #1:

```
struct S { int x; } s, *p = &s;
```

Translation unit #2:

```
struct alignas(16) S;    // IFNDR, definition of S lacks alignment
extern S* p;
```

— end example]

G.6.8**[ifndr:dcl.attr.indet.mismatched.declarations]****Specified in:** 9.13.6

- ¹ If two first declarations of a function declare a function parameter with mismatched uses of the *indeterminate* attribute, the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
int h(int x [[indeterminate]]); // IFNDR, mismatched [[indeterminate]] to other first declaration of h
```

Translation unit #2:

```
int h(int x);                // IFNDR, mismatched [[indeterminate]] to other first declaration of h
```

— end example]

G.6.9**[ifndr:dcl.attr.noreturn.trans.unit.mismatch]****Specified in:** 9.13.10

- ¹ No diagnostic is required if a function is declared in one translation unit with the *noreturn* attribute but has declarations in other translation units without the attribute.

² [Example 1:

Translation unit #1:

```
[[noreturn]] void f() {}
```

Translation unit #2:

```
void f();                // IFNDR, declared without noreturn
```

— end example]

G.7 Clause 10: Modules**[ifndr.module]****G.7.1****[ifndr:module.unit.reserved.identifiers]****Specified in:** 10.1

- ¹ Specifying a *module-name* beginning with an identifier consisting of *std* followed by zero or more digits, or containing a reserved identifier (5.10) in a *module-declaration* is ill-formed, no diagnostic required.

² [Example 1:

```
module std;                // IFNDR, std is not allowed at the beginning
module module;            // IFNDR, module is a reserved identifier
module std0;              // IFNDR, std followed by digits is not allowed at the beginning
export module _Test;      // IFNDR, _Test is a reserved identifier
export module te__st;     // IFNDR, te__st is a reserved identifier
```

— end example]

G.7.2**[ifndr:module.unit.named.module.no.partition]****Specified in:** 10.1

¹ Having multiple primary module interface units for a named module is ill-formed, no diagnostic required.

² [Example 1:

```
module A;
export import :Internals;      // IFNDR, module partition not allowed
— end example]
```

G.7.3**[ifndr:module.unit.unexported.module.partition]****Specified in:** 10.1

¹ If a module partition of a module that is a module interface unit but is not directly or indirectly exported by the primary module interface unit (10.3), the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
export module M;      // primary module interface unit
export import :A;
```

Translation unit #2:

```
export module M:A;    // OK, directly exported by M
export import :B;
```

Translation unit #3:

```
export module M:B;    // OK, indirectly exported by M
```

Translation unit #4:

```
export module M:C;    // IFNDR, not directly or indirectly exported by M
— end example]
```

G.7.4**[ifndr:module.private.frag.other.module.units]****Specified in:** 10.5

¹ If a module has a private module fragment and there is another module unit of that module, the program is ill-formed, no diagnostic required.

² [Example 1:

Translation unit #1:

```
export module M;
module :private;      // private module fragment
```

Translation unit #2:

```
module M:A;           // IFNDR, partition of M with private module fragment
— end example]
```

G.8 Clause 11: Classes**[ifndr.class]****G.8.1****[ifndr:class.base.init.delegate.itself]****Specified in:** 11.9.3

¹ If a constructor delegates to itself directly or indirectly, the program is ill-formed, no diagnostic required

² [Example 1:

```
struct C {
  C( int ) { }          // #1: non-delegating constructor
  C(): C(42) { }        // #2: delegates to #1
  C( char c ) : C(42.0) { } // #3: IFNDR due to recursion with #4
  C( double d ) : C('a') { } // #4: IFNDR due to recursion with #3
};
— end example]
```

G.8.2**[ifndr:class.virtual.pure.or.defined]****Specified in:** 11.7.3

- ¹ If a virtual function that is not pure has no definition, the program is ill-formed, no diagnostic required.

² [Example 1:

```
class A {
    virtual void f();
};

int main() {
    A a;      // IFNDR, virtual function that is not pure but has no definition
}
```

— end example]

G.9 Clause 12: Overloading**[ifndr.over]****G.9.1****[ifndr:over.literal.reserved]****Specified in:** 12.6

- ¹ Some literal suffix identifiers are reserved for future standardization. A declaration whose literal-operator-id uses such a literal suffix identifier is ill-formed, no diagnostic required.

² [Example 1:

```
float operator "E(const char*);      // IFNDR, reserved literal suffix
double operator " _Bq(long double); // IFNDR, uses the reserved identifier _Bq
```

— end example]

G.10 Clause 13: Templates**[ifndr.temp]****G.10.1****[ifndr:temp.pre.reach.def]****Specified in:** 13.1

- ¹ A definition of a function template, member function of a class template, variable template, or static data member of a class template that is not reachable from the end of every definition domain (6.3) in which it is implicitly instantiated (13.9.2) and whose corresponding specialization is not explicitly instantiated (13.9.3) in some translation unit is ill-formed, no diagnostic required.

² [Example 1:

Source file "a.h":

```
template <typename T>
void f();
```

Source file "a.cpp":

```
#include "a.h"
int main() {
    f<int>(); // IFNDR, function template implicitly instantiated but not reachable definition
}
```

— end example]

G.10.2**[ifndr:temp.arg.template.sat.constraints]****Specified in:** 13.4.4

- ¹ Any partial specializations (13.7.6) associated with the primary template are considered when a specialization based on the template template-parameter is instantiated. If a specialization is not reachable from the point of instantiation, and it would have been selected had it been reachable, the program is ill-formed, no diagnostic required.

² [Example 1:

```
template<class T> struct A {
    int x;
};
```

```

template<template<class U> class V> struct C {
    V<int> y;
    V<int*> z;
};

C<A> c;

// IFNDR, specialization is not reachable from point of instantiation above and it would have
// been selected if it had
template<class T> struct A<T*> {
    long x;
};
— end example]

```

G.10.3**[ifndr:temp.constr.atomic.equiv.but.not.equiv]****Specified in:** 13.5.2.3

- ¹ If the validity or meaning of the program depends on whether two atomic constraints are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required.
- ² [Example 1:

```

template <int N> concept Add1 = true;
template <unsigned N> void f2()
requires Add1<2 * N>;
template <unsigned N> int f2()
requires Add1<N * 2> && true;
void h2() {
    f2<0>();           // IFNDR, requires determination of subsumption between atomic constraints
                      // that are functionally equivalent but not equivalent
}
— end example]

```

G.10.4**[ifndr:temp.constr.atomic.sat.result.diff]****Specified in:** 13.5.2.3

- ¹ If, at different points in the program, the satisfaction result is different for identical atomic constraints and template arguments, the program is ill-formed, no diagnostic required.
- ² [Example 1:

```

template<class T>
concept Complete = sizeof(T) == sizeof(T);

struct A;
static_assert(!Complete<A>);    // #1
struct A {};
static_assert(Complete<A>);     // IFNDR, satisfaction result differs from point #1
— end example]

```

G.10.5**[ifndr:temp.constr.normal.invalid]****Specified in:** 13.5.4

- ¹ If during constraint normalization any such substitution results in an invalid type or expression, the program is ill-formed, no diagnostic is required.
- ² [Example 1:

```

template<typename T> concept A = T::value || true;
template<typename U> concept B = A<U*>;
template<typename V> concept C = B<V&>;           // IFNDR, it would form the invalid type V&*
                                                  // in the parameter mapping
— end example]

```

G.10.6**[ifndr:temp.spec.partial.general.partial.reachable]****Specified in:** 13.7.6.1

- ¹ If a partial specialization is not reachable from a use of a template specialization that would make use of that partial specialization as the result of an implicit or explicit instantiation, the program is ill-formed, no diagnostic required.

² [Example 1:

```
template<typename T> class X{
public:
    void foo(){};
};
```

```
template class X<void*>;           // IFNDR, explicit instantiation and partial specialization is not reachable
```

```
template<typename T> class X<T*>{
public:
    void baz();
};
```

— end example]

G.10.7**[ifndr:temp.over.link.equiv.not.equiv]****Specified in:** 13.7.7.2

- ¹ If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required.

² [Example 1:

```
template <int I>
struct A{};
```

// IFNDR, the following declarations are functionally equivalent but not equivalent

```
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+1+2+3+4>);
```

— end example]

G.10.8**[ifndr:temp.res.general.default.but.not.found]****Specified in:** 13.8.1

- ¹ If the validity or meaning of the program would be changed by considering a default argument or default template argument introduced in a declaration that is reachable from the point of instantiation of a specialization (13.8.4.1) but is not found by lookup for the specialization, the program is ill-formed, no diagnostic required.

² [Example 1:

```
void f(long);           // #1
void f(int, int);       // #2
template<typename T> void g(T t) { f(t); }
void f(int, int = 0);    // #3
void h() { g(0); }      // IFNDR, selects #3 here but selects #1 using lookup for g<int>
```

— end example]

G.10.9**[ifndr:temp.point.diff.pt.diff.meaning]****Specified in:** 13.8.4.1

- ¹ A specialization for a class template has at most one point of instantiation within a translation unit. A specialization for any template may have points of instantiation in multiple translation units. If two different points of instantiation give a template specialization different meanings according to the one-definition rule (6.3), the program is ill-formed, no diagnostic required.

² [Example 1:

Source file "a.h":

```
#include <type_traits>
```

```
template <typename T, typename Enabler = void>
struct is_complete : std::false_type {};

template <typename T>
struct is_complete<T, std::void_t<decltype(sizeof(T) != 0)>> : std::true_type {};
```

Source file "a.cpp":

```
#include "a.h"
struct X;
static_assert(!is_complete<X>::value); // IFNDR, different specialization selected in "b.cpp"
```

Source file "b.cpp":

```
#include "a.h"
struct X { };
static_assert(is_complete<X>::value); // IFNDR, different specialization selected in "a.cpp"
```

— end example]

G.10.10

[ifndr:temp.dep.candidate.different.lookup.different]

Specified in: 13.8.4.2

- ¹ If considering all function declarations with external linkage in the associated namespaces in all translations would make a dependent call (13.8.3) ill-formed or find a better match, the program is ill-formed, no diagnostic required.

- ² [Example 1:

Translation unit #1:

```
namespace A {
    struct S {};
    void f(S&, long x, int y); // #3
    void g(S&, int x);        // #4
}
```

Translation unit #2:

```
namespace A {
    struct S {};
    void f(S&, int x, long y); // #5
    void g(S&, long x);        // #6
}
template <typename T>
void h(T& t)
{
    f(t, 1, 1); // Selects #5 in h<A::S>, would be ambiguous call if all declarations were considered.
    g(t, 1);    // Selects #6 in h<A::S>, would select #4 if all declarations were considered.
}
```

— end example]

G.10.11

[ifndr:temp.explicit.decl.implicit.inst]

Specified in: 13.9.3

- ¹ If an entity that is the subject of an explicit instantiation declaration and that is also used in a way that would otherwise cause an implicit instantiation (13.9.2) in the translation unit is not the subject of an explicit instantiation definition somewhere in the program the program is ill-formed, no diagnostic required.

- ² [Example 1:

```
extern template class std::vector<int>; // explicit instantiation declaration

int main() {
    std::cout << std::vector<int>().size(); // IFNDR, implicit instantiation but no explicit
                                           // instantiation definition
}
```

— end example]

G.10.12**[ifndr:temp.expl.spec.unreachable.declaration]****Specified in:** 13.9.4

- ¹ If an implicit instantiation of a template would occur and there is an unreachable explicit specialization that would have matched, the program is ill-formed, no diagnostic required.

² [Example 1:

Source file "a.h":

```
template <typename T> struct S {};
```

Translation unit #2:

```
#include "a.h"
template <> struct S<int> { int oops; };    // #1
```

Translation unit #3:

```
#include "a.h"
S<int> s;    // IFNDR, #1 is not reachable but would have matched
```

— end example]

G.10.13**[ifndr:temp.expl.spec.missing.definition]****Specified in:** 13.9.4

- ¹ If an explicit specialization of a template is declared but there is no definition provided for that specialization, the program is ill-formed, no diagnostic required.

² [Example 1:

```
template <typename T> int f(T&&) { return 0; }
template <> int f<int>(int&&);
int j = f(1);    // IFNDR, odr-use of f<int> with no definition
```

— end example]

G.10.14**[ifndr:temp.deduct.general.diff.order]****Specified in:** 13.10.3.1

- ¹ If substitution into different declarations of the same function template would cause template instantiations to occur in a different order or not at all, the program is ill-formed, no diagnostic required.

² [Example 1:

```
template <class T> struct A { using X = typename T::X; };
template <class T> typename T::X h(typename A<T>::X);    // #1
template <class T> auto h(typename A<T>::X) -> typename T::X;    // redeclaration #2
template <class T> void h(...) { }
```

```
void x() {
    h<int>(0);    // Substituting into #1 forms an invalid type from T::X and does
                // not attempt to instantiate A.
                // Substituting into #2 instantiates A<T>, which is ill-formed.
}
```

— end example]

G.11 Clause 15: Preprocessing directives**[ifndr.cpp]****G.11.1****[ifndr:cpp.cond.defined.after.macro]****Specified in:** 15.2

- ¹ If the expansion of a macro produces the preprocessing token **defined** the program is ill-formed, no diagnostic required.

² [Example 1:

```
#define A defined
#if A    // IFNDR, defined is generated by macro replacement in controlling expression
#endif
```

— end example]

G.11.2**[ifndr:cpp.cond.defined.malformed]****Specified in:** [15.2](#)

- ¹ If the **defined** unary operator is used when it does not match one of the specified grammatical forms, the program is ill-formed, no diagnostic required.

- ² [*Example 1:*

```
#define A
#define B A)
#if defined ( B // IFNDR, unary operator defined did not match valid form before replacement
#endif
```

— *end example*]

G.11.3**[ifndr:cpp.include.malformed.headername]****Specified in:** [15.3](#)

- ¹ If the *header-name-tokens* after an **include** directive cannot be formed into a *header-name* (with implementation-defined treatment of whitespace), the program is ill-formed, no diagnostic required.

- ² [*Example 1:*

```
#include `` // IFNDR, does not match one of the two allowable forms
```

— *end example*]

Bibliography

- [1] ISO 4217:2015, *Codes for the representation of currencies*
- [2] ISO/IEC 14882:2023, *Programming Languages — C++*
- [3] ISO/IEC 14882:2020, *Programming Languages — C++*
- [4] ISO/IEC 14882:2017, *Programming Languages — C++*
- [5] ISO/IEC 14882:2014, *Information technology — Programming Languages — C++*
- [6] ISO/IEC 14882:2011, *Information technology — Programming Languages — C++*
- [7] ISO/IEC 14882:2003, *Programming Languages — C++*
- [8] ISO/IEC TS 18661-3:2015, *Information Technology — Programming languages, their environments, and system software interfaces — Floating-point extensions for C — Part 3: Interchange and extended types*
- [9] IANA Character Sets Database. Available from: <https://www.iana.org/assignments/character-sets/>, 2021-04-01
- [10] IANA Time Zone Database. Available from: <https://www.iana.org/time-zones>
- [11] Unicode Character Mapping Markup Language [online]. Edited by Mark Davis and Markus Scherer. Revision 5.0.1; 2017-05-31 Available from: <https://www.unicode.org/reports/tr22/tr22-8.html>
- [12] Bjarne Stroustrup, *The C++ Programming Language, second edition*, Chapter R. Addison-Wesley Publishing Company, ISBN 0-201-53992-6, copyright ©1991 AT&T
- [13] Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, Appendix A. Prentice-Hall, 1978, ISBN 0-13-110163-3, copyright ©1978 AT&T
- [14] P. J. Plauger, *The Draft Standard C++ Library*. Prentice-Hall, ISBN 0-13-117003-1, copyright ©1995 P. J. Plauger
- [15] J. Demmel, I. Dumitriu, and O. Holtz, *Fast linear algebra is stable*, Numerische Mathematik 108 (59–91), 2007.
- [16] C. L. Lawson, R. J. Hanson, D. Kincaid, and F. T. Krogh, *Basic linear algebra subprograms for Fortran usage*. ACM Trans. Math. Soft., Vol. 5, pp. 308–323, 1979.
- [17] Jack J. Dongarra, Jeremy Du Croz, Sven Hammarling, and Richard J. Hanson, *An Extended Set of FORTRAN Basic Linear Algebra Subprograms*. ACM Trans. Math. Soft., Vol. 14, No. 1, pp. 1–17, Mar. 1988.
- [18] Jack J. Dongarra, Jeremy Du Croz, Sven Hammarling, and Iain Duff, *A Set of Level 3 Basic Linear Algebra Subprograms*. ACM Trans. Math. Soft., Vol. 16, No. 1, pp. 1–17, Mar. 1990.
- [19] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen, *LAPACK Users' Guide, Third Edition*. SIAM, Philadelphia, PA, USA, 1999.
- [20] L. Susan Blackford, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, Michael Heroux, Linda Kaufman, Andrew Lumsdaine, Antoine Petit, Roldan Pozo, Karin Remington, and R. Clint Whaley, *An Updated Set of Basic Linear Algebra Subprograms (BLAS)*. ACM Trans. Math. Soft., Vol. 28, Issue 2, pp. 135–151, 2002.
- [21] Michael J. Flynn, *Very High-Speed Computing Systems*. Proceedings of the IEEE, Vol. 54, Issue 12, pp. 1901–1909, 1966.

Cross-references

Each clause and subclause label is listed below along with the corresponding clause or subclause number and page number, in alphabetical order by label.

accumulate (26.10.3)	1569	alg.req.ind.swap (24.3.7.4)	1220
adjacent.difference (26.10.12)	1577	alg.req.mergeable (24.3.7.7)	1221
adjustfield.manip (31.5.5.2)	2052	alg.req.permutable (24.3.7.6)	1221
alg.adjacent.find (26.6.10)	1498	alg.req.sortable (24.3.7.8)	1221
alg.all.of (26.6.1)	1489	alg.reverse (26.7.10)	1526
alg.any.of (26.6.2)	1489	alg.rotate (26.7.11)	1528
alg.binary.search (26.8.4)	1539	alg.search (26.6.15)	1503
alg.binary.search.general (26.8.4.1)	1539	alg.set.operations (26.8.7)	1548
alg.c.library (26.13)	1587	alg.set.operations.general (26.8.7.1)	1548
alg.clamp (26.8.10)	1562	alg.shift (26.7.14)	1530
alg.contains (26.6.4)	1490	alg.sort (26.8.2)	1532
alg.copy (26.7.1)	1509	alg.sorting (26.8)	1532
alg.count (26.6.11)	1499	alg.sorting.general (26.8.1)	1532
alg.ends.with (26.6.17)	1506	alg.starts.with (26.6.16)	1506
alg.equal (26.6.13)	1501	alg.swap (26.7.3)	1513
alg.fill (26.7.6)	1519	alg.three.way (26.8.12)	1564
alg.find (26.6.6)	1493	alg.transform (26.7.4)	1514
alg.find.end (26.6.8)	1496	alg.unique (26.7.9)	1524
alg.find.first.of (26.6.9)	1497	algorithm.stable (16.4.6.8)	576
alg.find.last (26.6.7)	1495	algorithm.syn (26.4)	1432
alg.fold (26.6.18)	1507	algorithms (Clause 26)	1426
alg.foreach (26.6.5)	1491	algorithms.general (26.1)	1426
alg.func.obj (16.3.3.4)	554	algorithms.parallel (26.3)	1428
alg.generate (26.7.7)	1520	algorithms.parallel.defns (26.3.1)	1428
alg.heap.operations (26.8.8)	1555	algorithms.parallel.exceptions (26.3.4)	1431
alg.heap.operations.general (26.8.8.1)	1555	algorithms.parallel.exec (26.3.3)	1429
alg.is.permutation (26.6.14)	1503	algorithms.parallel.overloads (26.3.5)	1431
alg.lex.comparison (26.8.11)	1563	algorithms.parallel.user (26.3.2)	1429
alg.merge (26.8.6)	1545	algorithms.requirements (26.2)	1426
alg.min.max (26.8.9)	1558	algorithms.results (26.5)	1486
alg.mismatch (26.6.12)	1500	alloc.errors (17.6.4)	605
alg.modifying.operations (26.7)	1509	allocator.adaptor (20.7)	745
alg.move (26.7.2)	1512	allocator.adaptor.cnstr (20.7.3)	747
alg.none.of (26.6.3)	1490	allocator.adaptor.members (20.7.4)	748
alg.nonmodifying (26.6)	1489	allocator.adaptor.syn (20.7.1)	745
alg.nth.element (26.8.3)	1538	allocator.adaptor.types (20.7.2)	747
alg.partitions (26.8.5)	1541	allocator.globals (20.2.10.3)	693
alg.permutation.generators (26.8.13)	1564	allocator.members (20.2.10.2)	692
alg.rand (26.12)	1586	allocator.requirements (16.4.4.6)	565
alg.rand.general (26.12.1)	1586	allocator.requirements.completeness (16.4.4.6.2)	571
alg.rand.generate (26.12.2)	1586	allocator.requirements.general (16.4.4.6.1)	565
alg.random.sample (26.7.12)	1529	allocator.tag (20.2.7)	686
alg.random.shuffle (26.7.13)	1530	allocator.traits (20.2.9)	690
alg.remove (26.7.8)	1521	allocator.traits.general (20.2.9.1)	690
alg.replace (26.7.5)	1516	allocator.traits.members (20.2.9.3)	691
alg.req (24.3.7)	1219	allocator.traits.other (20.2.9.4)	692
alg.req.general (24.3.7.1)	1219	allocator.traits.types (20.2.9.2)	690
alg.req.ind.cmp (24.3.7.5)	1221	allocator.uses (20.2.8)	687
alg.req.ind.copy (24.3.7.3)	1220	allocator.uses.construction (20.2.8.2)	687
alg.req.ind.move (24.3.7.2)	1219		

- allocator.uses.trait (20.2.8.1) 687
- alt.headers (16.4.5.4) 574
- any (22.7) 883
- any.assign (22.7.4.3) 886
- any.bad.any.cast (22.7.3) 884
- any.class (22.7.4) 884
- any.class.general (22.7.4.1) 884
- any.cons (22.7.4.2) 885
- any.general (22.7.1) 883
- any.modifiers (22.7.4.4) 886
- any.nonmembers (22.7.5) 887
- any.observers (22.7.4.5) 887
- any.synop (22.7.2) 883
- arithmetic.operations (22.10.7) 924
- arithmetic.operations.divides (22.10.7.5) 925
- arithmetic.operations.general (22.10.7.1) 924
- arithmetic.operations.minus (22.10.7.3) 924
- arithmetic.operations.modulus (22.10.7.6) 925
- arithmetic.operations.multiplies (22.10.7.4) 925
- arithmetic.operations.negate (22.10.7.7) 926
- arithmetic.operations.plus (22.10.7.2) 924
- array (23.3.3) 996
- array.cons (23.3.3.2) 997
- array.creation (23.3.3.6) 998
- array.members (23.3.3.3) 998
- array.overview (23.3.3.1) 996
- array.special (23.3.3.4) 998
- array.syn (23.3.2) 996
- array.tuple (23.3.3.7) 998
- array.zero (23.3.3.5) 998
- assertions (19.3) 653
- assertions.assert (19.3.3) 653
- assertions.general (19.3.1) 653
- associative (23.4) 1042
- associative.general (23.4.1) 1042
- associative.map.syn (23.4.2) 1042
- associative.reqmts (23.2.7) 976
- associative.reqmts.except (23.2.7.2) 984
- associative.reqmts.general (23.2.7.1) 976
- associative.set.syn (23.4.5) 1054
- atomics (32.5) 2201
- atomics.alias (32.5.3) 2207
- atomics.fences (32.5.11) 2241
- atomics.flag (32.5.10) 2239
- atomics.general (32.5.1) 2201
- atomics.lockfree (32.5.5) 2209
- atomics.nonmembers (32.5.9) 2239
- atomics.order (32.5.4) 2207
- atomics.ref.float (32.5.7.4) 2215
- atomics.ref.generic (32.5.7) 2210
- atomics.ref.generic.general (32.5.7.1) 2210
- atomics.ref.int (32.5.7.3) 2213
- atomics.ref.memop (32.5.7.6) 2220
- atomics.ref.ops (32.5.7.2) 2211
- atomics.ref.pointer (32.5.7.5) 2218
- atomics.syn (32.5.2) 2201
- atomics.types.float (32.5.8.4) 2228
- atomics.types.generic (32.5.8) 2220
- atomics.types.generic.general (32.5.8.1) 2220
- atomics.types.int (32.5.8.3) 2224
- atomics.types.memop (32.5.8.6) 2233
- atomics.types.operations (32.5.8.2) 2221
- atomics.types.pointer (32.5.8.5) 2231
- atomics.wait (32.5.6) 2209
- back.insert.iter.general (24.5.2.2.1) 1229
- back.insert.iter.ops (24.5.2.2.2) 1229
- back.insert.iterator (24.5.2.2) 1229
- back.inserter (24.5.2.2.3) 1230
- bad.alloc (17.6.4.1) 605
- bad.cast (17.7.4) 608
- bad.exception (17.9.4) 613
- bad.typeid (17.7.5) 608
- barrier.syn (32.9.3.2) 2272
- basefield.manip (31.5.5.3) 2052
- basic (Clause 6) 32
- basic.align (6.8.3) 68
- basic.compound (6.9.5) 86
- basic.contract (6.11) 102
- basic.contract.eval (6.11.2) 102
- basic.contract.general (6.11.1) 102
- basic.contract.handler (6.11.3) 104
- basic.def (6.2) 33
- basic.def.odr (6.3) 35
- basic.exec (6.10) 90
- basic.extended.fp (6.9.4) 86
- basic.fundamental (6.9.3) 82
- basic.indet (6.8.5) 72
- basic.ios.cons (31.5.4.2) 2048
- basic.ios.members (31.5.4.3) 2048
- basic.life (6.8.4) 69
- basic.link (6.7) 60
- basic.lookup (6.5) 46
- basic.lookup.argdep (6.5.4) 51
- basic.lookup.elab (6.5.6) 59
- basic.lookup.general (6.5.1) 46
- basic.lookup.qual (6.5.5) 54
- basic.lookup.qual.general (6.5.5.1) 54
- basic.lookup.udir (6.5.7) 59
- basic.lookup.unqual (6.5.3) 50
- basic.lval (7.2.1) 106
- basic.memobj (6.8) 64
- basic.namespace (9.9) 279
- basic.namespace.general (9.9.1) 279
- basic.pre (6.1) 32
- basic.scope (6.4) 40
- basic.scope.block (6.4.3) 44
- basic.scope.class (6.4.7) 45
- basic.scope.contract (6.4.10) 46
- basic.scope.enum (6.4.8) 46
- basic.scope.lambda (6.4.5) 45
- basic.scope.namespace (6.4.6) 45
- basic.scope.param (6.4.4) 45
- basic.scope.pdecl (6.4.2) 43
- basic.scope.scope (6.4.1) 40
- basic.scope.temp (6.4.9) 46
- basic.splice (6.6) 59
- basic.start (6.10.3) 98

- basic.start.dynamic (6.10.3.3) 99
- basic.start.main (6.10.3.1) 98
- basic.start.static (6.10.3.2) 98
- basic.start.term (6.10.3.4) 100
- basic.stc (6.8.6) 74
- basic.stc.auto (6.8.6.4) 74
- basic.stc.dynamic (6.8.6.5) 74
- basic.stc.dynamic.allocation (6.8.6.5.2) 75
- basic.stc.dynamic.deallocation (6.8.6.5.3) 76
- basic.stc.dynamic.general (6.8.6.5.1) 75
- basic.stc.general (6.8.6.1) 74
- basic.stc.static (6.8.6.2) 74
- basic.stc.thread (6.8.6.3) 74
- basic.string (27.4.3) 1607
- basic.string.general (27.4.3.1) 1607
- basic.string.hash (27.4.6) 1632
- basic.string.literals (27.4.7) 1632
- basic.type.qualifier (6.9.6) 88
- basic.types (6.9) 80
- basic.types.general (6.9.1) 80
- basic.types.trivial (6.9.2) 82
- bidirectional.iterators (24.3.5.6) 1216
- binary.search (26.8.4.5) 1541
- bit (22.11) 952
- bit.byteswap (22.11.4) 954
- bit.cast (22.11.3) 953
- bit.count (22.11.8) 955
- bit.endian (22.11.10) 957
- bit.general (22.11.1) 952
- bit.permute (22.11.9) 956
- bit.pow.two (22.11.5) 954
- bit.rotate (22.11.7) 955
- bit.shift (22.11.6) 955
- bit.syn (22.11.2) 952
- bitmask.types (16.3.3.3.3) 552
- bitset (22.9) 910
- bitset.cons (22.9.2.2) 912
- bitset.hash (22.9.3) 916
- bitset.members (22.9.2.3) 913
- bitset.operators (22.9.4) 916
- bitset.syn (22.9.1) 910
- bitwise.operations (22.10.11) 931
- bitwise.operations.and (22.10.11.2) 931
- bitwise.operations.general (22.10.11.1) 931
- bitwise.operations.not (22.10.11.5) 932
- bitwise.operations.or (22.10.11.3) 932
- bitwise.operations.xor (22.10.11.4) 932
- byte.strings (16.3.3.3.4.2) 554
- c.files (31.13) 2175
- c.locales (28.3.5) 1676
- c.malloc (20.2.12) 693
- c.math (29.7) 1824
- c.math.abs (29.7.2) 1830
- c.math.fpclass (29.7.5) 1831
- c.math.hypot3 (29.7.3) 1831
- c.math.lerp (29.7.4) 1831
- c.math.rand (29.5.10) 1805
- c.mb.wcs (28.7.5) 1749
- c.strings (27.5) 1632
- cassert.syn (19.3.2) 653
- category.collate (28.3.4.5) 1663
- category.ctype (28.3.4.2) 1645
- category.ctype.general (28.3.4.2.1) 1645
- category.messages (28.3.4.8) 1675
- category.messages.general (28.3.4.8.1) 1675
- category.monetary (28.3.4.7) 1669
- category.monetary.general (28.3.4.7.1) 1669
- category.numeric (28.3.4.3) 1653
- category.numeric.general (28.3.4.3.1) 1654
- category.time (28.3.4.6) 1664
- category.time.general (28.3.4.6.1) 1664
- cctype.syn (28.7.1) 1746
- cerrno.syn (19.4.2) 654
- cfenv (29.3) 1751
- cfenv.syn (29.3.1) 1751
- cfenv.thread (29.3.2) 1752
- cfloat.syn (17.3.7) 596
- char.traits (27.2) 1589
- char.traits.general (27.2.1) 1589
- char.traits.require (27.2.2) 1589
- char.traits.specializations (27.2.4) 1591
- char.traits.specializations.char (27.2.4.2) 1591
- char.traits.specializations.char16.t (27.2.4.4) 1592
- char.traits.specializations.char32.t (27.2.4.5) 1593
- char.traits.specializations.char8.t (27.2.4.3) 1592
- char.traits.specializations.general (27.2.4.1) 1591
- char.traits.specializations.wchar.t (27.2.4.6) 1593
- char.traits.typedefs (27.2.3) 1591
- character.seq (16.3.3.3.4) 553
- character.seq.general (16.3.3.3.4.1) 553
- charconv (28.2) 1634
- charconv.from.chars (28.2.3) 1636
- charconv.syn (28.2.1) 1634
- charconv.to.chars (28.2.2) 1635
- cinttypes.syn (31.13.2) 2177
- class (Clause 11) 314
- class.abstract (11.7.4) 350
- class.access (11.8) 351
- class.access.base (11.8.3) 354
- class.access.general (11.8.1) 351
- class.access.nest (11.8.8) 361
- class.access.spec (11.8.2) 353
- class.access.virt (11.8.6) 360
- class.base.init (11.9.3) 362
- class.bit (11.4.10) 337
- class.cdtor (11.9.5) 369
- class.compare (11.10) 374
- class.compare.default (11.10.1) 374
- class.compare.secondary (11.10.4) 376
- class.conv (11.4.8) 333
- class.conv.ctor (11.4.8.2) 334
- class.conv.fct (11.4.8.3) 334
- class.conv.general (11.4.8.1) 333
- class.copy.assign (11.4.6) 328
- class.copy.ctor (11.4.5.3) 326
- class.copy.elision (11.9.6) 371
- class.ctor (11.4.5) 324

- class.ctor.general (11.4.5.1) 324
- class.default.ctor (11.4.5.2) 325
- class.derived (11.7) 344
- class.derived.general (11.7.1) 344
- class.dtor (11.4.7) 330
- class.eq (11.10.2) 375
- class.expl.init (11.9.2) 362
- class.free (11.4.11) 338
- class.friend (11.8.4) 356
- class.gslice (29.6.6) 1819
- class.gslice.overview (29.6.6.1) 1819
- class.inhctor.init (11.9.4) 367
- class.init (11.9) 361
- class.init.general (11.9.1) 362
- class.local (11.6) 343
- class.mem (11.4) 318
- class.mem.general (11.4.1) 318
- class.member.lookup (6.5.2) 47
- class.mfct (11.4.2) 322
- class.mfct.non.static (11.4.3) 323
- class.mi (11.7.2) 345
- class.name (11.3) 316
- class.nest (11.4.12) 340
- class.paths (11.8.7) 361
- class.pre (11.1) 314
- class.prop (11.2) 315
- class.protected (11.8.5) 360
- class.qual (6.5.5.2) 56
- class.slice (29.6.4) 1817
- class.slice.overview (29.6.4.1) 1817
- class.spaceship (11.10.3) 375
- class.static (11.4.9) 336
- class.static.data (11.4.9.3) 337
- class.static.general (11.4.9.1) 336
- class.static.mfct (11.4.9.2) 337
- class.temporary (6.8.7) 76
- class.union (11.5) 341
- class.union.anon (11.5.2) 342
- class.union.general (11.5.1) 341
- class.virtual (11.7.3) 347
- classification (28.3.3.3.1) 1644
- climits.syn (17.3.6) 595
- clocale.data.races (28.3.5.2) 1676
- clocale.syn (28.3.5.1) 1676
- cmath.syn (29.7.1) 1824
- cmp (17.12) 619
- cmp.alg (17.12.6) 625
- cmp.categories (17.12.2) 620
- cmp.categories.pre (17.12.2.1) 620
- cmp.common (17.12.3) 623
- cmp.concept (17.12.4) 624
- cmp.partialord (17.12.2.2) 620
- cmp.result (17.12.5) 625
- cmp.strongord (17.12.2.4) 622
- cmp.weakord (17.12.2.3) 621
- cmplx.over (29.4.10) 1759
- common.iter.access (24.5.5.4) 1244
- common.iter.cmp (24.5.5.6) 1245
- common.iter.const (24.5.5.3) 1243
- common.iter.cust (24.5.5.7) 1245
- common.iter.nav (24.5.5.5) 1244
- common.iter.types (24.5.5.2) 1243
- common.iterator (24.5.5.1) 1242
- compare.syn (17.12.1) 619
- compare.type (17.12.7) 627
- comparisons (22.10.8) 926
- comparisons.equal.to (22.10.8.2) 926
- comparisons.general (22.10.8.1) 926
- comparisons.greater (22.10.8.4) 927
- comparisons.greater.equal (22.10.8.6) 927
- comparisons.less (22.10.8.5) 927
- comparisons.less.equal (22.10.8.7) 928
- comparisons.not.equal.to (22.10.8.3) 927
- comparisons.three.way (22.10.8.8) 928
- complex (29.4.3) 1754
- complex.h.syn (17.15.2) 635
- complex.literals (29.4.11) 1759
- complex.member.ops (29.4.5) 1755
- complex.members (29.4.4) 1755
- complex.numbers (29.4) 1752
- complex.numbers.general (29.4.1) 1752
- complex.ops (29.4.6) 1756
- complex.syn (29.4.2) 1752
- complex.transcendentals (29.4.8) 1757
- complex.tuple (29.4.9) 1758
- complex.value.ops (29.4.7) 1757
- compliance (16.4.2.5) 559
- concept.assignable (18.4.8) 642
- concept.booleantestable (18.5.2) 645
- concept.common (18.4.6) 641
- concept.commonref (18.4.5) 641
- concept.comparisoncommontype (18.5.3) 646
- concept.constructible (18.4.11) 644
- concept.convertible (18.4.4) 640
- concept.copyconstructible (18.4.14) 645
- concept.default.init (18.4.12) 644
- concept.derived (18.4.3) 640
- concept.destructible (18.4.10) 644
- concept.equalitycomparable (18.5.4) 647
- concept.equiv (18.7.6) 649
- concept.invocable (18.7.2) 648
- concept.moveconstructible (18.4.13) 645
- concept.predicate (18.7.4) 649
- concept.regularinvocable (18.7.3) 649
- concept.relation (18.7.5) 649
- concept.same (18.4.2) 640
- concept.strictweakorder (18.7.7) 649
- concept.swappable (18.4.9) 642
- concept.totallyordered (18.5.5) 647
- concepts (Clause 18) 637
- concepts.arithmetic (18.4.7) 642
- concepts.callable (18.7) 648
- concepts.callable.general (18.7.1) 648
- concepts.compare (18.5) 645
- concepts.compare.general (18.5.1) 645
- concepts.equality (18.2) 637
- concepts.general (18.1) 637
- concepts.lang (18.4) 640

- concepts.lang.general (18.4.1) 640
- concepts.object (18.6) 648
- concepts.syn (18.3) 638
- condition.variable.syn (32.7.2) 2262
- conforming (16.4.6) 575
- conforming.overview (16.4.6.1) 575
- cons.slice (29.6.4.2) 1817
- const.iterators (24.5.3) 1232
- const.iterators.alias (24.5.3.2) 1232
- const.iterators.general (24.5.3.1) 1232
- const.iterators.iterator (24.5.3.3) 1232
- const.iterators.ops (24.5.3.5) 1234
- const.iterators.types (24.5.3.4) 1234
- const.wrap.class (22.2.10) 828
- constexpr.functions (16.4.6.7) 576
- constraints (16.4.5) 571
- constraints.overview (16.4.5.1) 571
- container.adaptors (23.6) 1089
- container.adaptors.format (23.6.13) 1137
- container.adaptors.general (23.6.1) 1089
- container.alloc.reqmts (23.2.2.5) 965
- container.insert.return (23.2.6) 975
- container.intro.reqmts (23.2.2.1) 960
- container.node (23.2.5) 973
- container.node.cons (23.2.5.2) 974
- container.node.dtor (23.2.5.3) 974
- container.node.modifiers (23.2.5.5) 975
- container.node.observers (23.2.5.4) 975
- container.node.overview (23.2.5.1) 973
- container.opt.reqmts (23.2.2.4) 965
- container.reqmts (23.2.2.2) 961
- container.requirements (23.2) 960
- container.requirements.dataraces (23.2.3) 967
- container.requirements.general (23.2.2) 960
- container.requirements.pre (23.2.1) 960
- container.rev.reqmts (23.2.2.3) 964
- containers (Clause 23) 960
- containers.general (23.1) 960
- contents (16.4.2.2) 557
- contracts.syn (17.10.1) 616
- conv (7.3) 109
- conv.array (7.3.3) 111
- conv.bool (7.3.15) 114
- conv.double (7.3.10) 113
- conv.fctptr (7.3.14) 114
- conv.fpint (7.3.11) 113
- conv.fpprom (7.3.8) 113
- conv.func (7.3.4) 111
- conv.general (7.3.1) 109
- conv.integral (7.3.9) 113
- conv.lval (7.3.2) 110
- conv.mem (7.3.13) 114
- conv.prom (7.3.7) 112
- conv.ptr (7.3.12) 113
- conv.qual (7.3.6) 111
- conv.rank (6.9.7) 89
- conv.rval (7.3.5) 111
- conventions (16.3.3) 551
- conventions.general (16.3.3.1) 551
- conversions.character (28.3.3.3.2) 1644
- coro.generator (25.8) 1420
- coro.generator.class (25.8.3) 1420
- coro.generator.iterator (25.8.6) 1424
- coro.generator.members (25.8.4) 1421
- coro.generator.promise (25.8.5) 1422
- coroutine.generator.overview (25.8.1) 1420
- coroutine.handle (17.13.4) 628
- coroutine.handle.compare (17.13.4.8) 631
- coroutine.handle.con (17.13.4.2) 629
- coroutine.handle.conv (17.13.4.3) 630
- coroutine.handle.export.import (17.13.4.4) 630
- coroutine.handle.general (17.13.4.1) 628
- coroutine.handle.hash (17.13.4.9) 631
- coroutine.handle.noop (17.13.5.2) 631
- coroutine.handle.noop.address (17.13.5.2.6) 632
- coroutine.handle.noop.conv (17.13.5.2.2) 631
- coroutine.handle.noop.general (17.13.5.2.1) 631
- coroutine.handle.noop.observers (17.13.5.2.3) 631
- coroutine.handle.noop.promise (17.13.5.2.5) 632
- coroutine.handle.noop.resumption (17.13.5.2.4) 632
- coroutine.handle.observers (17.13.4.5) 630
- coroutine.handle.promise (17.13.4.7) 630
- coroutine.handle.resumption (17.13.4.6) 630
- coroutine.noop (17.13.5) 631
- coroutine.noop.coroutine (17.13.5.3) 632
- coroutine.promise.noop (17.13.5.1) 631
- coroutine.syn (17.13.2) 628
- coroutine.traits (17.13.3) 628
- coroutine.traits.general (17.13.3.1) 628
- coroutine.traits.primary (17.13.3.2) 628
- coroutine.trivial.awaitables (17.13.6) 632
- counted.iter.access (24.5.7.3) 1248
- counted.iter.cmp (24.5.7.6) 1249
- counted.iter.const (24.5.7.2) 1247
- counted.iter.cust (24.5.7.7) 1250
- counted.iter.elem (24.5.7.4) 1248
- counted.iter.nav (24.5.7.5) 1248
- counted.iterator (24.5.7.1) 1246
- cpp (Clause 15) 523
- cpp.concat (15.7.4) 540
- cpp.cond (15.2) 525
- cpp.embed (15.4) 529
- cpp.embed.gen (15.4.1) 529
- cpp.embed.param (15.4.2) 532
- cpp.embed.param.if.empty (15.4.2.5) 533
- cpp.embed.param.limit (15.4.2.1) 532
- cpp.embed.param.offset (15.4.2.2) 532
- cpp.embed.param.prefix (15.4.2.3) 533
- cpp.embed.param.suffix (15.4.2.4) 533
- cpp.error (15.9) 542
- cpp.import (15.6) 535
- cpp.include (15.3) 528
- cpp.line (15.8) 542
- cpp.module (15.5) 534
- cpp.null (15.11) 543
- cpp.pragma (15.10) 543
- cpp.pragma.op (15.13) 546

- cpp.pre (15.1) 523
- cpp.predefined (15.12) 543
- cpp.replace (15.7) 537
- cpp.replace.general (15.7.1) 537
- cpp.rescan (15.7.5) 541
- cpp.scope (15.7.6) 542
- cpp.stringize (15.7.3) 540
- cpp.subst (15.7.2) 538
- csetjmp.syn (17.14.3) 633
- csignal.syn (17.14.4) 633
- cstdarg.syn (17.14.2) 633
- cstddef.syn (17.2.1) 579
- cstdint.syn (17.4.1) 597
- cstdio.syn (31.13.1) 2176
- cstdlib.syn (17.2.2) 580
- cstring.syn (27.5.1) 1632
- ctime.syn (30.15) 2032
- cuchar.syn (28.7.4) 1749
- customization.point.object (16.3.3.3.5) 554
- cwchar.syn (28.7.3) 1747
- cwctype.syn (28.7.2) 1747

- dcl (Clause 9) 206
- dcl.align (9.13.2) 294
- dcl.ambig.res (9.3.3) 230
- dcl.array (9.3.4.5) 236
- dcl.asm (9.11) 289
- dcl.attr (9.13) 292
- dcl.attr.annotation (9.13.12) 300
- dcl.attr.assume (9.13.3) 295
- dcl.attr.deprecated (9.13.4) 295
- dcl.attr.fallthrough (9.13.5) 296
- dcl.attr.grammar (9.13.1) 292
- dcl.attr.indet (9.13.6) 296
- dcl.attr.likelihood (9.13.7) 297
- dcl.attr.nodiscard (9.13.9) 298
- dcl.attr.noreturn (9.13.10) 299
- dcl.attr.nouniqueaddr (9.13.11) 299
- dcl.attr.unused (9.13.8) 297
- dcl.constexpr (9.2.6) 213
- dcl.constinit (9.2.7) 215
- dcl.contract (9.4) 246
- dcl.contract.func (9.4.1) 246
- dcl.contract.res (9.4.2) 247
- dcl.decl (9.3) 227
- dcl.decl.general (9.3.1) 227
- dcl.enum (9.8.1) 276
- dcl.fct (9.3.4.6) 238
- dcl.fct.def (9.6) 268
- dcl.fct.def.coroutine (9.6.4) 271
- dcl.fct.def.default (9.6.2) 269
- dcl.fct.def.delete (9.6.3) 270
- dcl.fct.def.general (9.6.1) 268
- dcl.fct.def.replace (9.6.5) 274
- dcl.fct.default (9.3.4.7) 243
- dcl.fct.spec (9.2.3) 212
- dcl.friend (9.2.5) 213
- dcl.init (9.5) 248
- dcl.init.aggr (9.5.2) 253
- dcl.init.general (9.5.1) 249
- dcl.init.list (9.5.5) 262
- dcl.init.ref (9.5.4) 259
- dcl.init.string (9.5.3) 259
- dcl.inline (9.2.8) 216
- dcl.link (9.12) 290
- dcl.meaning (9.3.4) 231
- dcl.meaning.general (9.3.4.1) 231
- dcl.mptr (9.3.4.4) 236
- dcl.name (9.3.2) 229
- dcl.pre (9.1) 206
- dcl.ptr (9.3.4.2) 233
- dcl.ref (9.3.4.3) 234
- dcl.spec (9.2) 209
- dcl.spec.auto (9.2.9.7) 222
- dcl.spec.auto.general (9.2.9.7.1) 222
- dcl.spec.general (9.2.1) 209
- dcl.stc (9.2.2) 210
- dcl.struct.bind (9.7) 274
- dcl.type (9.2.9) 216
- dcl.type.auto.deduct (9.2.9.7.2) 225
- dcl.type.class.deduct (9.2.9.8) 226
- dcl.type.cv (9.2.9.2) 217
- dcl.type.decltype (9.2.9.6) 221
- dcl.type.elab (9.2.9.5) 220
- dcl.type.general (9.2.9.1) 216
- dcl.type.pack.index (9.2.9.4) 220
- dcl.type.simple (9.2.9.3) 218
- dcl.type.splice (9.2.9.9) 227
- dcl.typedef (9.2.4) 212
- debugging (19.7) 670
- debugging.general (19.7.1) 670
- debugging.syn (19.7.2) 670
- debugging.utility (19.7.3) 670
- declval (22.2.6) 826
- default allocator (20.2.10) 692
- default allocator.general (20.2.10.1) 692
- default.sentinel (24.5.6) 1245
- defns.access (3.1) 3
- defns.argument (3.2) 3
- defns.argument.macro (3.3) 3
- defns.argument.templ (3.5) 3
- defns.argument.throw (3.4) 3
- defns.block (3.6) 3
- defns.block.stmt (3.7) 3
- defns.c.lib (3.8) 3
- defns.character (3.9) 3
- defns.character.container (3.10) 3
- defns.component (3.12) 4
- defns.cond.suppl (3.13) 4
- defns.const.eval (3.14) 4
- defns.const.subexpr (3.15) 4
- defns.deadlock (3.16) 4
- defns.default.behavior.impl (3.17) 4
- defns.diagnostic (3.18) 4
- defns.dynamic.type (3.19) 4
- defns.dynamic.type.prvalue (3.20) 4
- defns.erroneous (3.21) 4
- defns.expression.equivalent (3.22) 5

- defns.handler (3.25) 5
- defns.ill.formed (3.26) 5
- defns.impl.defined (3.27) 5
- defns.impl.limits (3.29) 5
- defns.impl.prop (3.45) 7
- defns.locale.specific (3.30) 5
- defns.modifier (3.32) 5
- defns.move.assign (3.33) 6
- defns.move.constr (3.34) 6
- defns.nonconst.libcall (3.35) 6
- defns.ntcts (3.36) 6
- defns.observer (3.37) 6
- defns.order.ptr (3.28) 5
- defns.parameter (3.38) 6
- defns.parameter.macro (3.39) 6
- defns.parameter.templ (3.40) 6
- defns.prog.def.spec (3.42) 6
- defns.prog.def.type (3.43) 6
- defns.projection (3.44) 6
- defns.referenceable (3.46) 7
- defns.regex.collating.element (3.11) 4
- defns.regex.finite.state.machine (3.23) 5
- defns.regex.format.specifier (3.24) 5
- defns.regex.matched (3.31) 5
- defns.regex.primary.equivalence.class (3.41) 6
- defns.regex.regular.expression (3.47) 7
- defns.regex.subexpression (3.63) 8
- defns.replacement (3.48) 7
- defns.required.behavior (3.49) 7
- defns.reserved.function (3.50) 7
- defns.signature (3.52) 7
- defns.signature.friend (3.53) 7
- defns.signature.member (3.57) 8
- defns.signature.member.spec (3.59) 8
- defns.signature.member.templ (3.58) 8
- defns.signature.spec (3.56) 8
- defns.signature.templ (3.54) 8
- defns.signature.templ.friend (3.55) 8
- defns.signature.template.head (3.60) 8
- defns.stable (3.61) 8
- defns.static.type (3.62) 8
- defns.traits (3.64) 8
- defns.unblock (3.65) 8
- defns.undefined (3.66) 9
- defns.undefined.runtime (3.51) 7
- defns.unspecified (3.67) 9
- defns.valid (3.68) 9
- defns.well.formed (3.69) 9
- depr (Annex D) 2455
- depr.atomics (D.24) 2463
- depr.atomics.general (D.24.1) 2463
- depr.atomics.nonmembers (D.24.3) 2463
- depr.atomics.order (D.24.5) 2464
- depr.atomics.types.operations (D.24.4) 2464
- depr.atomics.volatile (D.24.2) 2463
- depr.c.macros (D.11) 2457
- depr.capture.this (D.3) 2455
- depr.cerrno (D.12) 2457
- depr.ctime (D.22) 2462
- depr.ellipsis.comma (D.5) 2456
- depr.filesystems (D.23) 2462
- depr.format (D.21) 2461
- depr.format.arg (D.21.2) 2462
- depr.format.syn (D.21.1) 2461
- depr.fs.path.factory (D.23.1) 2462
- depr.fs.path.obs (D.23.2) 2462
- depr.general (D.1) 2455
- depr.impldec (D.6) 2456
- depr.istream.extractors (D.25) 2464
- depr.iterator (D.18) 2460
- depr.lit (D.8) 2456
- depr.local (D.2) 2455
- depr.locale.category (D.20) 2461
- depr.meta.types (D.13) 2457
- depr.move.iter.elem (D.19) 2461
- depr.numeric.limits.has.denorm (D.10) 2457
- depr.ostream.inserters (D.26) 2465
- depr.relops (D.14) 2459
- depr.static.constexpr (D.7) 2456
- depr.template.template (D.9) 2456
- depr.tuple (D.15) 2459
- depr.variant (D.16) 2460
- depr.vector.bool.swap (D.17) 2460
- depr.volatile.type (D.4) 2455
- deque (23.3.5) 999
- deque.capacity (23.3.5.3) 1002
- deque.cons (23.3.5.2) 1001
- deque.erasure (23.3.5.5) 1003
- deque.modifiers (23.3.5.4) 1002
- deque.overview (23.3.5.1) 999
- deque.syn (23.3.4) 999
- derivation (16.4.6.13) 577
- derived.classes (16.4.5.5) 574
- description (16.3) 548
- description.general (16.3.1) 548
- diagnostics (Clause 19) 650
- diagnostics.general (19.1) 650
- diff (Annex C) 2413
- diff.basic (C.8.3) 2444
- diff.char16 (C.9.3.1) 2453
- diff.class (C.8.7) 2451
- diff.cpp (C.8.8) 2453
- diff.cpp03 (C.7) 2437
- diff.cpp03.algorithms (C.7.13) 2442
- diff.cpp03.class (C.7.5) 2439
- diff.cpp03.containers (C.7.12) 2441
- diff.cpp03.dcl.dcl (C.7.4) 2439
- diff.cpp03.diagnostics (C.7.9) 2441
- diff.cpp03.expr (C.7.3) 2437
- diff.cpp03.general (C.7.1) 2437
- diff.cpp03.input.output (C.7.16) 2442
- diff.cpp03.language.support (C.7.8) 2440
- diff.cpp03.lex (C.7.2) 2437
- diff.cpp03.library (C.7.7) 2440
- diff.cpp03.locale (C.7.15) 2442
- diff.cpp03.numerics (C.7.14) 2442
- diff.cpp03.strings (C.7.11) 2441
- diff.cpp03.temp (C.7.6) 2439

- diff.cpp03.utilities (C.7.10) 2441
- diff.cpp11 (C.6) 2435
- diff.cpp11.basic (C.6.3) 2435
- diff.cpp11.dcl.dcl (C.6.5) 2436
- diff.cpp11.expr (C.6.4) 2436
- diff.cpp11.general (C.6.1) 2435
- diff.cpp11.input.output (C.6.7) 2437
- diff.cpp11.lex (C.6.2) 2435
- diff.cpp11.library (C.6.6) 2436
- diff.cpp14 (C.5) 2431
- diff.cpp14.class (C.5.5) 2433
- diff.cpp14.containers (C.5.11) 2434
- diff.cpp14.dcl.dcl (C.5.4) 2432
- diff.cpp14.depr (C.5.12) 2435
- diff.cpp14.except (C.5.7) 2433
- diff.cpp14.expr (C.5.3) 2432
- diff.cpp14.general (C.5.1) 2431
- diff.cpp14.lex (C.5.2) 2431
- diff.cpp14.library (C.5.8) 2433
- diff.cpp14.string (C.5.10) 2434
- diff.cpp14.temp (C.5.6) 2433
- diff.cpp14.utilities (C.5.9) 2434
- diff.cpp17 (C.4) 2423
- diff.cpp17.alg.reqs (C.4.13) 2429
- diff.cpp17.basic (C.4.3) 2424
- diff.cpp17.class (C.4.6) 2426
- diff.cpp17.containers (C.4.11) 2429
- diff.cpp17.dcl.dcl (C.4.5) 2425
- diff.cpp17.depr (C.4.15) 2430
- diff.cpp17.except (C.4.9) 2428
- diff.cpp17.expr (C.4.4) 2425
- diff.cpp17.general (C.4.1) 2423
- diff.cpp17.input.output (C.4.14) 2429
- diff.cpp17.iterators (C.4.12) 2429
- diff.cpp17.lex (C.4.2) 2423
- diff.cpp17.library (C.4.10) 2429
- diff.cpp17.over (C.4.7) 2427
- diff.cpp17.temp (C.4.8) 2428
- diff.cpp20 (C.3) 2418
- diff.cpp20.concepts (C.3.9) 2421
- diff.cpp20.containers (C.3.13) 2422
- diff.cpp20.dcl (C.3.5) 2419
- diff.cpp20.expr (C.3.3) 2419
- diff.cpp20.general (C.3.1) 2418
- diff.cpp20.lex (C.3.2) 2418
- diff.cpp20.library (C.3.8) 2420
- diff.cpp20.memory (C.3.10) 2421
- diff.cpp20.over (C.3.6) 2420
- diff.cpp20.stmt (C.3.4) 2419
- diff.cpp20.strings (C.3.12) 2422
- diff.cpp20.temp (C.3.7) 2420
- diff.cpp20.thread (C.3.14) 2422
- diff.cpp20.utilities (C.3.11) 2421
- diff.cpp23 (C.2) 2414
- diff.cpp23.cpp (C.2.6) 2416
- diff.cpp23.dcl.dcl (C.2.4) 2415
- diff.cpp23.depr (C.2.11) 2417
- diff.cpp23.expr (C.2.3) 2414
- diff.cpp23.general (C.2.1) 2414
- diff.cpp23.io (C.2.10) 2417
- diff.cpp23.lex (C.2.2) 2414
- diff.cpp23.library (C.2.7) 2416
- diff.cpp23.mem (C.2.8) 2417
- diff.cpp23.strings (C.2.9) 2417
- diff.cpp23.temp (C.2.5) 2416
- diff.cpp26 (C.1) 2413
- diff.cpp26.basic (C.1.2) 2413
- diff.cpp26.dcl (C.1.3) 2413
- diff.cpp26.general (C.1.1) 2413
- diff.dcl (C.8.6) 2448
- diff.expr (C.8.4) 2445
- diff.header.iso646.h (C.9.3.3) 2453
- diff.iso (C.8) 2443
- diff.iso.general (C.8.1) 2443
- diff.lex (C.8.2) 2443
- diff.library (C.9) 2453
- diff.library.general (C.9.1) 2453
- diff.malloc (C.9.5.3) 2454
- diff.mods.to.behavior (C.9.5) 2454
- diff.mods.to.behavior.general (C.9.5.1) 2454
- diff.mods.to.declarations (C.9.4) 2454
- diff.mods.to.definitions (C.9.3) 2453
- diff.mods.to.headers (C.9.2) 2453
- diff.null (C.9.3.4) 2454
- diff.offsetof (C.9.5.2) 2454
- diff.stat (C.8.5) 2447
- diff.wchar.t (C.9.3.2) 2453
- domain.error (19.2.4) 651
- enum (9.8) 276
- enum.udecl (9.8.2) 279
- enumerated.types (16.3.3.3.2) 552
- equal.range (26.8.4.4) 1540
- errno (19.4) 653
- errno.general (19.4.1) 654
- error.reporting (31.5.6) 2053
- except (Clause 14) 514
- except.ctor (14.3) 516
- except.handle (14.4) 517
- except.nested (17.9.8) 615
- except.pre (14.1) 514
- except.spec (14.5) 519
- except.special (14.6) 521
- except.special.general (14.6.1) 521
- except.terminate (14.6.2) 521
- except.throw (14.2) 515
- exception (17.9.3) 612
- exception.syn (17.9.2) 612
- exception.terminate (17.9.5) 613
- exclusive.scan (26.10.8) 1573
- exec (Clause 33) 2295
- exec.adapt (33.9.12) 2328
- exec.adapt.general (33.9.12.1) 2328
- exec.adapt.obj (33.9.12.2) 2328
- exec.affine (33.13.3) 2366
- exec.as.awaitable (33.13.1) 2363
- exec.associate (33.9.12.16) 2345
- exec.async.ops (33.3) 2296

- exec.awaitable (33.9.4) 2321
- exec.bulk (33.9.12.11) 2338
- exec.cmpsig (33.10) 2358
- exec.connect (33.9.10) 2325
- exec.consumers (33.9.13) 2354
- exec.continues.on (33.9.12.6) 2330
- exec.coro.util (33.13) 2363
- exec.counting.scopes (33.14.2) 2376
- exec.counting.scopes.general (33.14.2.1) 2376
- exec.ctx (33.12) 2361
- exec.domain.default (33.9.6) 2323
- exec.domain.indeterminate (33.9.5) 2322
- exec.env (33.11.2) 2360
- exec.envs (33.11) 2359
- exec.factories (33.9.11) 2327
- exec.fwd.env (33.5.2) 2305
- exec.general (33.1) 2295
- exec.get allocator (33.5.3) 2305
- exec.get.await.adapt (33.5.13) 2308
- exec.get.compl.domain (33.5.12) 2307
- exec.get.compl.sched (33.5.11) 2307
- exec.get.delegation.scheduler (33.5.9) 2306
- exec.get.domain (33.5.6) 2305
- exec.get.env (33.5.5) 2305
- exec.get.fwd.progress (33.5.10) 2306
- exec.get.scheduler (33.5.7) 2306
- exec.get.start.scheduler (33.5.8) 2306
- exec.get.stop.token (33.5.4) 2305
- exec.getcompsigs (33.9.9) 2324
- exec.inline.scheduler (33.13.4) 2367
- exec.info.variant (33.9.12.13) 2344
- exec.just (33.9.11.2) 2327
- exec.let (33.9.12.10) 2334
- exec.on (33.9.12.8) 2332
- exec.opstate (33.8) 2311
- exec.opstate.general (33.8.1) 2311
- exec.opstate.start (33.8.2) 2311
- exec.par.scheduler (33.15) 2381
- exec.parschedrepl (33.16) 2382
- exec.parschedrepl.general (33.16.1) 2382
- exec.parschedrepl.psb (33.16.4) 2383
- exec.parschedrepl.query (33.16.3) 2383
- exec.parschedrepl.recvproxy (33.16.2) 2382
- exec.prop (33.11.1) 2359
- exec.queries (33.5) 2304
- exec.queries.expos (33.5.1) 2305
- exec.queryable (33.2) 2296
- exec.queryable.concept (33.2.2) 2296
- exec.queryable.general (33.2.1) 2296
- exec.read.env (33.9.11.3) 2327
- exec.recv (33.7) 2309
- exec.recv.concepts (33.7.1) 2309
- exec.run.loop (33.12.1) 2361
- exec.run.loop.ctor (33.12.1.3) 2362
- exec.run.loop.general (33.12.1.1) 2361
- exec.run.loop.members (33.12.1.4) 2363
- exec.run.loop.types (33.12.1.2) 2362
- exec.sched (33.6) 2308
- exec.schedule (33.9.11.1) 2327
- exec.schedule.from (33.9.12.7) 2332
- exec.scope (33.14) 2375
- exec.scope.concepts (33.14.1) 2375
- exec.scope.counting (33.14.2.3) 2380
- exec.scope.simple.counting (33.14.2.2) 2378
- exec.scope.simple.counting.general (33.14.2.2.1) 2378
- exec.set.error (33.7.3) 2311
- exec.set.stopped (33.7.4) 2311
- exec.set.value (33.7.2) 2310
- exec.simple.counting.ctor (33.14.2.2.2) 2378
- exec.simple.counting.mem (33.14.2.2.3) 2379
- exec.simple.counting.token (33.14.2.2.4) 2379
- exec.snd (33.9) 2311
- exec.snd.apply (33.9.8) 2324
- exec.snd.concepts (33.9.3) 2319
- exec.snd.expos (33.9.2) 2312
- exec.snd.general (33.9.1) 2311
- exec.snd.transform (33.9.7) 2323
- exec.spawn (33.9.13.3) 2356
- exec.spawn.future (33.9.12.18) 2349
- exec.starts.on (33.9.12.5) 2330
- exec.stop.when (33.9.12.17) 2348
- exec.stopped.err (33.9.12.15) 2345
- exec.stopped.opt (33.9.12.14) 2344
- exec.sync.wait (33.9.13.1) 2354
- exec.sync.wait.var (33.9.13.2) 2356
- exec.task (33.13.6) 2370
- exec.task.scheduler (33.13.5) 2367
- exec.then (33.9.12.9) 2334
- exec.unstoppable (33.9.12.4) 2330
- exec.when.all (33.9.12.12) 2340
- exec.with.awaitable.senders (33.13.2) 2365
- exec.write.env (33.9.12.3) 2329
- execpol (26.3.6) 1431
- execpol.general (26.3.6.1) 1431
- execpol.objects (26.3.6.7) 1432
- execpol.par (26.3.6.4) 1432
- execpol.parunseq (26.3.6.5) 1432
- execpol.seq (26.3.6.3) 1432
- execpol.type (26.3.6.2) 1431
- execpol.unseq (26.3.6.6) 1432
- execution.syn (33.4) 2298
- expected (22.8) 888
- expected.bad (22.8.4) 890
- expected.bad.void (22.8.5) 891
- expected.expected (22.8.6) 891
- expected.general (22.8.1) 888
- expected.object.assign (22.8.6.4) 896
- expected.object.cons (22.8.6.2) 893
- expected.object.dtor (22.8.6.3) 896
- expected.object.eq (22.8.6.8) 902
- expected.object.general (22.8.6.1) 891
- expected.object.monadic (22.8.6.7) 901
- expected.object.obs (22.8.6.6) 899
- expected.object.swap (22.8.6.5) 898
- expected.syn (22.8.2) 888
- expected.un.cons (22.8.3.2) 890
- expected.un.eq (22.8.3.5) 890

- expected.un.general (22.8.3.1) 889
- expected.un.obs (22.8.3.3) 890
- expected.un.swap (22.8.3.4) 890
- expected.unexpected (22.8.3) 889
- expected.void (22.8.7) 903
- expected.void.assign (22.8.7.4) 906
- expected.void.cons (22.8.7.2) 904
- expected.void.dtor (22.8.7.3) 906
- expected.void.eq (22.8.7.8) 909
- expected.void.general (22.8.7.1) 903
- expected.void.monadic (22.8.7.7) 908
- expected.void.obs (22.8.7.6) 907
- expected.void.swap (22.8.7.5) 907
- expos.only.entity (16.3.3.2) 551
- expr (Clause 7) 105
- expr.add (7.6.6) 168
- expr.alignof (7.6.2.6) 155
- expr.arith.conv (7.4) 114
- expr.assign (7.6.19) 175
- expr.await (7.6.2.4) 153
- expr.bit.and (7.6.11) 172
- expr.call (7.6.1.3) 140
- expr.cast (7.6.3) 165
- expr.comma (7.6.20) 176
- expr.compound (7.6) 139
- expr.cond (7.6.16) 173
- expr.const (7.7) 177
- expr.const.cast (7.6.1.11) 150
- expr.const.const (7.7.3) 182
- expr.const.core (7.7.2) 177
- expr.const.defns (7.7.7) 190
- expr.const.general (7.7.1) 177
- expr.const.imm (7.7.5) 186
- expr.const.init (7.7.4) 184
- expr.const.reflect (7.7.6) 188
- expr.context (7.2.3) 109
- expr.delete (7.6.2.9) 161
- expr.dynamic.cast (7.6.1.7) 145
- expr.eq (7.6.10) 170
- expr.log.and (7.6.14) 172
- expr.log.or (7.6.15) 173
- expr.mptr.oper (7.6.4) 166
- expr.mul (7.6.5) 167
- expr.new (7.6.2.8) 155
- expr.or (7.6.13) 172
- expr.post (7.6.1) 139
- expr.post.general (7.6.1.1) 139
- expr.post.incr (7.6.1.6) 145
- expr.pre (7.1) 105
- expr.pre.incr (7.6.2.3) 153
- expr.prim (7.5) 115
- expr.prim.fold (7.5.7) 134
- expr.prim.grammar (7.5.1) 115
- expr.prim.id (7.5.5) 116
- expr.prim.id.dtor (7.5.5.5) 122
- expr.prim.id.general (7.5.5.1) 117
- expr.prim.id.qual (7.5.5.3) 120
- expr.prim.id.unqual (7.5.5.2) 118
- expr.prim.lambda (7.5.6) 123
- expr.prim.lambda.capture (7.5.6.3) 129
- expr.prim.lambda.closure (7.5.6.2) 124
- expr.prim.lambda.general (7.5.6.1) 123
- expr.prim.literal (7.5.2) 115
- expr.prim.pack.index (7.5.5.4) 122
- expr.prim.paren (7.5.4) 116
- expr.prim.req (7.5.8) 134
- expr.prim.req.compound (7.5.8.4) 136
- expr.prim.req.general (7.5.8.1) 134
- expr.prim.req.nested (7.5.8.5) 137
- expr.prim.req.simple (7.5.8.2) 136
- expr.prim.req.type (7.5.8.3) 136
- expr.prim.splice (7.5.9) 137
- expr.prim.this (7.5.3) 115
- expr.prop (7.2) 106
- expr.ref (7.6.1.5) 143
- expr.reflect (7.6.2.10) 163
- expr.reinterpret.cast (7.6.1.10) 149
- expr.rel (7.6.9) 170
- expr.shift (7.6.7) 168
- expr.sizeof (7.6.2.5) 154
- expr.spaceship (7.6.8) 169
- expr.static.cast (7.6.1.9) 147
- expr.sub (7.6.1.2) 139
- expr.throw (7.6.18) 175
- expr.type (7.2.2) 108
- expr.type.conv (7.6.1.4) 142
- expr.typeid (7.6.1.8) 146
- expr.unary (7.6.2) 151
- expr.unary.general (7.6.2.1) 151
- expr.unary.noexcept (7.6.2.7) 155
- expr.unary.op (7.6.2.2) 151
- expr.xor (7.6.12) 172
- expr.yield (7.6.17) 174
- ext.manip (31.7.8) 2084
- extern.names (16.4.5.3.4) 572
- extern.types (16.4.5.3.5) 573
- facet ctype.char.dtor (28.3.4.2.4.2) 1648
- facet ctype.char.members (28.3.4.2.4.3) 1649
- facet ctype.char.statics (28.3.4.2.4.4) 1649
- facet ctype.char.virtuals (28.3.4.2.4.5) 1649
- facet ctype.special (28.3.4.2.4) 1648
- facet ctype.special.general (28.3.4.2.4.1) 1648
- facet.num.get.members (28.3.4.3.2.2) 1655
- facet.num.get.virtuals (28.3.4.3.2.3) 1655
- facet.num.put.members (28.3.4.3.3.2) 1658
- facet.num.put.virtuals (28.3.4.3.3.3) 1658
- facet.numpunct (28.3.4.4) 1661
- facet.numpunct.members (28.3.4.4.1.2) 1662
- facet.numpunct.virtuals (28.3.4.4.1.3) 1662
- file.native (31.10.2) 2112
- file.streams (31.10) 2111
- filebuf (31.10.3) 2112
- filebuf.assign (31.10.3.3) 2114
- filebuf.cons (31.10.3.2) 2114
- filebuf.general (31.10.3.1) 2113
- filebuf.members (31.10.3.4) 2115
- filebuf.virtuals (31.10.3.5) 2116

- filesystems (31.12) 2129
- flat.map (23.6.8) 1102
- flat.map.access (23.6.8.6) 1110
- flat.map.capacity (23.6.8.5) 1110
- flat.map.cons (23.6.8.3) 1108
- flat.map.cons.alloc (23.6.8.4) 1109
- flat.map.defn (23.6.8.2) 1103
- flat.map.erasure (23.6.8.8) 1114
- flat.map.modifiers (23.6.8.7) 1111
- flat.map.overview (23.6.8.1) 1102
- flat.map.syn (23.6.7) 1101
- flat.multimap (23.6.9) 1114
- flat.multimap.cons (23.6.9.3) 1120
- flat.multimap.cons.alloc (23.6.9.4) 1120
- flat.multimap.defn (23.6.9.2) 1115
- flat.multimap.erasure (23.6.9.5) 1121
- flat.multimap.overview (23.6.9.1) 1114
- flat.multiset (23.6.12) 1130
- flat.multiset.cons (23.6.12.3) 1135
- flat.multiset.cons.alloc (23.6.12.4) 1135
- flat.multiset.defn (23.6.12.2) 1130
- flat.multiset.erasure (23.6.12.6) 1137
- flat.multiset.modifiers (23.6.12.5) 1136
- flat.multiset.overview (23.6.12.1) 1130
- flat.set (23.6.11) 1122
- flat.set.cons (23.6.11.3) 1127
- flat.set.cons.alloc (23.6.11.4) 1127
- flat.set.defn (23.6.11.2) 1123
- flat.set.erasure (23.6.11.6) 1129
- flat.set.modifiers (23.6.11.5) 1128
- flat.set.overview (23.6.11.1) 1122
- flat.set.syn (23.6.10) 1122
- floatfield.manip (31.5.5.4) 2052
- fmtflags.manip (31.5.5.1) 2051
- fmtflags.state (31.5.2.3) 2043
- format (28.5) 1684
- format.arg (28.5.8.1) 1708
- format.arg.store (28.5.8.2) 1710
- format.args (28.5.8.3) 1710
- format.arguments (28.5.8) 1708
- format.context (28.5.6.7) 1702
- format.err.report (28.5.3) 1693
- format.error (28.5.10) 1712
- format.fmt.string (28.5.4) 1693
- format.formattable (28.5.6.3) 1697
- format.formatter (28.5.6) 1696
- format.formatter.locking (28.5.6.2) 1696
- format.formatter.spec (28.5.6.4) 1697
- format.functions (28.5.5) 1694
- format.parse.ctx (28.5.6.6) 1700
- format.range (28.5.7) 1703
- format.range.fmtdef (28.5.7.3) 1706
- format.range.fmtkind (28.5.7.1) 1703
- format.range.fmtmap (28.5.7.4) 1706
- format.range.fmtset (28.5.7.5) 1707
- format.range.fmtstr (28.5.7.6) 1708
- format.range.formatter (28.5.7.2) 1704
- format.string (28.5.2) 1687
- format.string.escaped (28.5.6.5) 1699
- format.string.general (28.5.2.1) 1687
- format.string.std (28.5.2.2) 1688
- format.syn (28.5.1) 1684
- format.tuple (28.5.9) 1711
- formatter.requirements (28.5.6.1) 1696
- forward (22.2.4) 825
- forward.iterators (24.3.5.5) 1215
- forward.list (23.3.7) 1004
- forward.list.access (23.3.7.4) 1007
- forward.list.cons (23.3.7.2) 1007
- forward.list.erasure (23.3.7.7) 1011
- forward.list.iter (23.3.7.3) 1007
- forward.list.modifiers (23.3.7.5) 1007
- forward.list.ops (23.3.7.6) 1009
- forward.list.overview (23.3.7.1) 1004
- forward.list.syn (23.3.6) 1004
- fpos (31.5.3) 2045
- fpos.general (31.5.3.1) 2045
- fpos.members (31.5.3.2) 2046
- fpos.operations (31.5.3.3) 2046
- freestanding.item (16.3.3.7) 555
- front.insert.iter.general (24.5.2.3.1) 1230
- front.insert.iter.ops (24.5.2.3.2) 1230
- front.insert.iterator (24.5.2.3) 1230
- front.inserter (24.5.2.3.3) 1231
- fs.class.directory.entry (31.12.10) 2155
- fs.class.directory.entry.general (31.12.10.1) 2155
- fs.class.directory.iterator (31.12.11) 2159
- fs.class.directory.iterator.general (31.12.11.1) 2159
- fs.class.file.status (31.12.9) 2152
- fs.class.file.status.general (31.12.9.1) 2152
- fs.class.filesystem.error (31.12.7) 2150
- fs.class.filesystem.error.general (31.12.7.1) 2150
- fs.class.path (31.12.6) 2134
- fs.class.path.general (31.12.6.1) 2134
- fs.class.rec.dir.itr (31.12.12) 2161
- fs.class.rec.dir.itr.general (31.12.12.1) 2161
- fs.conform.9945 (31.12.2.2) 2129
- fs.conform.os (31.12.2.3) 2130
- fs.conformance (31.12.2) 2129
- fs.conformance.general (31.12.2.1) 2129
- fs.dir.entry.cons (31.12.10.2) 2156
- fs.dir.entry.io (31.12.10.5) 2159
- fs.dir.entry.mods (31.12.10.3) 2157
- fs.dir.entry.obs (31.12.10.4) 2157
- fs.dir.itr.members (31.12.11.2) 2160
- fs.dir.itr.nonmembers (31.12.11.3) 2160
- fs.enum (31.12.8) 2151
- fs.enum.copy.opts (31.12.8.3) 2151
- fs.enum.dir.opts (31.12.8.6) 2152
- fs.enum.file.type (31.12.8.2) 2151
- fs.enum.path.format (31.12.8.1) 2151
- fs.enum.perm.opts (31.12.8.5) 2152
- fs.enum.perms (31.12.8.4) 2151
- fs.err.report (31.12.5) 2134
- fs.file.status.cons (31.12.9.2) 2153
- fs.file.status.mods (31.12.9.4) 2155
- fs.file.status.obs (31.12.9.3) 2153

- fs.filesystem.error.members (31.12.7.2) 2151
- fs.filesystem.syn (31.12.4) 2130
- fs.general (31.12.1) 2129
- fs.op.absolute (31.12.13.2) 2164
- fs.op.canonical (31.12.13.3) 2164
- fs.op.copy (31.12.13.4) 2164
- fs.op.copy.file (31.12.13.5) 2166
- fs.op.copy.symlink (31.12.13.6) 2167
- fs.op.create.dir.symlink (31.12.13.9) 2167
- fs.op.create.directories (31.12.13.7) 2167
- fs.op.create.directory (31.12.13.8) 2167
- fs.op.create.hard.lk (31.12.13.10) 2168
- fs.op.create.symlink (31.12.13.11) 2168
- fs.op.current.path (31.12.13.12) 2168
- fs.op.equivalent (31.12.13.13) 2168
- fs.op.exists (31.12.13.14) 2169
- fs.op.file.size (31.12.13.15) 2169
- fs.op.funcs (31.12.13) 2163
- fs.op.funcs.general (31.12.13.1) 2163
- fs.op.hard.lk.ct (31.12.13.16) 2169
- fs.op.is.block.file (31.12.13.17) 2169
- fs.op.is.char.file (31.12.13.18) 2169
- fs.op.is.directory (31.12.13.19) 2170
- fs.op.is.empty (31.12.13.20) 2170
- fs.op.is.fifo (31.12.13.21) 2170
- fs.op.is.other (31.12.13.22) 2170
- fs.op.is.regular.file (31.12.13.23) 2171
- fs.op.is.socket (31.12.13.24) 2171
- fs.op.is.symlink (31.12.13.25) 2171
- fs.op.last.write.time (31.12.13.26) 2171
- fs.op.permissions (31.12.13.27) 2172
- fs.op.proximate (31.12.13.28) 2172
- fs.op.read.symlink (31.12.13.29) 2172
- fs.op.relative (31.12.13.30) 2172
- fs.op.remove (31.12.13.31) 2173
- fs.op.remove.all (31.12.13.32) 2173
- fs.op.rename (31.12.13.33) 2173
- fs.op.resize.file (31.12.13.34) 2173
- fs.op.space (31.12.13.35) 2173
- fs.op.status (31.12.13.36) 2174
- fs.op.status.known (31.12.13.37) 2175
- fs.op.symlink.status (31.12.13.38) 2175
- fs.op.temp.dir.path (31.12.13.39) 2175
- fs.op.weakly.canonical (31.12.13.40) 2175
- fs.path.append (31.12.6.5.3) 2142
- fs.path.assign (31.12.6.5.2) 2141
- fs.path.compare (31.12.6.5.8) 2145
- fs.path.concat (31.12.6.5.4) 2143
- fs.path.construct (31.12.6.5.1) 2140
- fs.path.cvt (31.12.6.3) 2139
- fs.path.decompose (31.12.6.5.9) 2145
- fs.path.fmt.cvt (31.12.6.3.1) 2139
- fs.path.fmtr (31.12.6.9) 2149
- fs.path.fmtr.funcs (31.12.6.9.2) 2150
- fs.path.fmtr.general (31.12.6.9.1) 2149
- fs.path.gen (31.12.6.5.11) 2147
- fs.path.generic (31.12.6.2) 2137
- fs.path.generic.obs (31.12.6.5.7) 2145
- fs.path.hash (31.12.6.10) 2150
- fs.path.io (31.12.6.7) 2149
- fs.path.itr (31.12.6.6) 2148
- fs.path.member (31.12.6.5) 2140
- fs.path.modifiers (31.12.6.5.5) 2143
- fs.path.native.obs (31.12.6.5.6) 2144
- fs.path.nonmember (31.12.6.8) 2149
- fs.path.query (31.12.6.5.10) 2146
- fs.path.req (31.12.6.4) 2140
- fs.path.type.cvt (31.12.6.3.2) 2139
- fs.race.behavior (31.12.2.4) 2130
- fs.rec.dir.itr.members (31.12.12.2) 2162
- fs.rec.dir.itr.nonmembers (31.12.12.3) 2163
- fs.req (31.12.3) 2130
- fstream (31.10.6) 2122
- fstream.cons (31.10.6.2) 2123
- fstream.general (31.10.6.1) 2122
- fstream.members (31.10.6.4) 2124
- fstream.swap (31.10.6.3) 2124
- fstream.syn (31.10.1) 2112
- func.bind (22.10.15) 934
- func.bind.bind (22.10.15.4) 935
- func.bind.general (22.10.15.1) 934
- func.bind.isbind (22.10.15.2) 934
- func.bind.isplace (22.10.15.3) 934
- func.bind.partial (22.10.14) 933
- func.bind.place (22.10.15.5) 936
- func.def (22.10.3) 920
- func.identity (22.10.12) 933
- func.invoke (22.10.5) 921
- func.memfn (22.10.16) 936
- func.not.fn (22.10.13) 933
- func.require (22.10.4) 920
- func.search (22.10.18) 949
- func.search.bm (22.10.18.3) 950
- func.search.bmh (22.10.18.4) 951
- func.search.default (22.10.18.2) 949
- func.search.general (22.10.18.1) 949
- func.wrap (22.10.17) 936
- func.wrap.badcall (22.10.17.2) 936
- func.wrap.copy (22.10.17.5) 943
- func.wrap.copy.class (22.10.17.5.2) 943
- func.wrap.copy.ctor (22.10.17.5.3) 944
- func.wrap.copy.general (22.10.17.5.1) 943
- func.wrap.copy.inv (22.10.17.5.4) 946
- func.wrap.copy.util (22.10.17.5.5) 946
- func.wrap.func (22.10.17.3) 937
- func.wrap.func.alg (22.10.17.3.8) 940
- func.wrap.func.cap (22.10.17.3.4) 939
- func.wrap.func.con (22.10.17.3.2) 937
- func.wrap.func.general (22.10.17.3.1) 937
- func.wrap.func.inv (22.10.17.3.5) 939
- func.wrap.func.mod (22.10.17.3.3) 939
- func.wrap.func.nullptr (22.10.17.3.7) 940
- func.wrap.func.targ (22.10.17.3.6) 939
- func.wrap.general (22.10.17.1) 936
- func.wrap.move (22.10.17.4) 940
- func.wrap.move.class (22.10.17.4.2) 940
- func.wrap.move.ctor (22.10.17.4.3) 941
- func.wrap.move.general (22.10.17.4.1) 940

- func.wrap.move.inv (22.10.17.4.4) 942
- func.wrap.move.util (22.10.17.4.5) 942
- func.wrap.ref (22.10.17.6) 946
- func.wrap.ref.class (22.10.17.6.2) 946
- func.wrap.ref.ctor (22.10.17.6.3) 947
- func.wrap.ref.deduct (22.10.17.6.5) 949
- func.wrap.ref.general (22.10.17.6.1) 946
- func.wrap.ref.inv (22.10.17.6.4) 949
- function.objects (22.10) 916
- function.objects.general (22.10.1) 916
- functional.syn (22.10.2) 917
- functions.within.classes (16.3.3.5) 555
- future.syn (32.10.2) 2274
- futures (32.10) 2274
- futures.async (32.10.9) 2284
- futures.errors (32.10.3) 2275
- futures.future.error (32.10.4) 2276
- futures.overview (32.10.1) 2274
- futures.promise (32.10.6) 2277
- futures.shared.future (32.10.8) 2281
- futures.state (32.10.5) 2276
- futures.task (32.10.10) 2285
- futures.task.general (32.10.10.1) 2285
- futures.task.members (32.10.10.2) 2286
- futures.task.nonmembers (32.10.10.3) 2288
- futures.unique.future (32.10.7) 2279

- generator.syn (25.8.2) 1420
- get.new.handler (17.6.4.5) 606
- get.terminate (17.9.5.3) 613
- global.functions (16.4.6.4) 576
- gram (Annex A) 2386
- gram.basic (A.4) 2391
- gram.class (A.9) 2405
- gram.cpp (A.13) 2408
- gram.dcl (A.7) 2397
- gram.except (A.12) 2408
- gram.expr (A.5) 2391
- gram.general (A.1) 2386
- gram.key (A.2) 2386
- gram.lex (A.3) 2386
- gram.module (A.8) 2404
- gram.over (A.10) 2406
- gram.stmt (A.6) 2396
- gram.temp (A.11) 2406
- gslice.access (29.6.6.3) 1820
- gslice.array.assign (29.6.7.2) 1821
- gslice.array.comp.assign (29.6.7.3) 1821
- gslice.array.fill (29.6.7.4) 1821
- gslice.cons (29.6.6.2) 1820

- handler.functions (16.4.5.7) 574
- hardware.interference (17.6.6) 606
- hash.requirements (16.4.4.5) 564
- hazard.pointer.syn (32.11.3.2) 2292
- headers (16.4.2.3) 557
- hidden.friends (16.4.6.6) 576
- hive (23.3.9) 1012
- hive.capacity (23.3.9.3) 1017
- hive.cons (23.3.9.2) 1015
- hive.erasure (23.3.9.6) 1020
- hive.modifiers (23.3.9.4) 1018
- hive.operations (23.3.9.5) 1019
- hive.overview (23.3.9.1) 1012
- hive.syn (23.3.8) 1011

- ifndr (Annex G) 2494
- ifndr.basic (G.3) 2494
- ifndr.class (G.8) 2501
- ifndr.cpp (G.11) 2506
- ifndr.dcl (G.6) 2498
- ifndr.expr (G.4) 2497
- ifndr.general (G.1) 2494
- ifndr.lex (G.2) 2494
- ifndr.module (G.7) 2500
- ifndr.over (G.9) 2502
- ifndr.stmt (G.5) 2497
- ifndr.temp (G.10) 2502
- ifstream (31.10.4) 2118
- ifstream.cons (31.10.4.2) 2119
- ifstream.general (31.10.4.1) 2118
- ifstream.members (31.10.4.4) 2120
- ifstream.swap (31.10.4.3) 2120
- implimits (Annex B) 2411
- includes (26.8.7.2) 1548
- inclusive.scan (26.10.9) 1574
- incrementable.traits (24.3.2.1) 1202
- indirect (20.5.1) 726
- indirect.array.assign (29.6.9.2) 1823
- indirect.array.comp.assign (29.6.9.3) 1823
- indirect.array.fill (29.6.9.4) 1823
- indirect.assign (20.5.1.5) 729
- indirect.comp.with.t (20.5.1.9) 731
- indirect.ctor (20.5.1.3) 727
- indirect.dtor (20.5.1.4) 729
- indirect.general (20.5.1.1) 726
- indirect.hash (20.5.1.10) 731
- indirect.obs (20.5.1.6) 730
- indirect.relops (20.5.1.8) 731
- indirect.swap (20.5.1.7) 731
- indirect.syn (20.5.1.2) 726
- indirectcallable (24.3.6) 1218
- indirectcallable.general (24.3.6.1) 1218
- indirectcallable.indirectinvocable (24.3.6.3) 1218
- indirectcallable.traits (24.3.6.2) 1218
- initializer.list.syn (17.11.2) 618
- inner.product (26.10.5) 1571
- inout.ptr (20.4.4.4) 725
- inout.ptr.t (20.4.4.3) 724
- inplace.vector (23.3.16) 1036
- inplace.vector.capacity (23.3.16.3) 1039
- inplace.vector.cons (23.3.16.2) 1039
- inplace.vector.data (23.3.16.4) 1040
- inplace.vector.erasure (23.3.16.6) 1041
- inplace.vector.modifiers (23.3.16.5) 1040
- inplace.vector.overview (23.3.16.1) 1036
- inplace.vector.syn (23.3.15) 1036
- input.iterators (24.3.5.3) 1214

- input.output (Clause 31) 2033
- input.output.general (31.1) 2033
- input.streams (31.7.5) 2063
- input.streams.general (31.7.5.1) 2063
- insert.iter.general (24.5.2.4.1) 1231
- insert.iter.ops (24.5.2.4.2) 1231
- insert.iterator (24.5.2.4) 1231
- insert.iterators (24.5.2) 1229
- insert.iterators.general (24.5.2.1) 1229
- inserter (24.5.2.4.3) 1232
- intro (Clause 4) 9
- intro.abstract (4.1.2) 11
- intro.compliance (4.1) 10
- intro.compliance.general (4.1.1) 10
- intro.defs (Clause 3) 2
- intro.execution (6.10.1) 90
- intro.memory (6.8.1) 64
- intro.multithread (6.10.2) 93
- intro.multithread.general (6.10.2.1) 93
- intro.object (6.8.2) 65
- intro.progress (6.10.2.3) 96
- intro.races (6.10.2.2) 94
- intro.refs (Clause 2) 1
- intro.scope (Clause 1) 1
- intro.structure (4.2) 12
- intseq (21.2) 750
- intseq.binding (21.2.4) 750
- intseq.general (21.2.1) 750
- intseq.intseq (21.2.2) 750
- intseq.make (21.2.3) 750
- invalid.argument (19.2.5) 651
- iomanip.syn (31.7.3) 2062
- ios (31.5.4) 2047
- ios.base (31.5.2) 2039
- ios.base.callback (31.5.2.7) 2045
- ios.base.cons (31.5.2.8) 2045
- ios.base.general (31.5.2.1) 2039
- ios.base.locales (31.5.2.4) 2044
- ios.base.storage (31.5.2.6) 2044
- ios.failure (31.5.2.2.1) 2041
- ios.fmtflags (31.5.2.2.2) 2041
- ios.init (31.5.2.2.6) 2041
- ios.iostate (31.5.2.2.3) 2041
- ios.members.static (31.5.2.5) 2044
- ios.openmode (31.5.2.2.4) 2041
- ios.overview (31.5.4.1) 2047
- ios.seekdir (31.5.2.2.5) 2041
- ios.syn (31.5.1) 2038
- ios.types (31.5.2.2) 2041
- iosfwd.syn (31.3.1) 2034
- iostate.flags (31.5.4.4) 2050
- iostream.assign (31.7.5.7.4) 2074
- iostream.cons (31.7.5.7.2) 2073
- iostream.dest (31.7.5.7.3) 2074
- iostream.format (31.7) 2061
- iostream.forward (31.3) 2034
- iostream.forward.overview (31.3.2) 2036
- iostream.limits.imbue (31.2.1) 2033
- iostream.objects (31.4) 2036
- iostream.objects.overview (31.4.2) 2036
- iostream.syn (31.4.1) 2036
- iostreamclass (31.7.5.7) 2073
- iostreamclass.general (31.7.5.7.1) 2073
- iostreams.base (31.5) 2038
- iostreams.limits.pos (31.2.3) 2033
- iostreams.requirements (31.2) 2033
- iostreams.threadsafety (31.2.4) 2033
- is.heap (26.8.8.6) 1557
- is.sorted (26.8.2.5) 1537
- iso646.h.syn (17.15.3) 635
- ispanstream (31.9.4) 2108
- ispanstream.cons (31.9.4.2) 2108
- ispanstream.general (31.9.4.1) 2108
- ispanstream.members (31.9.4.4) 2109
- ispanstream.swap (31.9.4.3) 2109
- istream (31.7.5.2) 2063
- istream.assign (31.7.5.2.3) 2065
- istream.cons (31.7.5.2.2) 2065
- istream.extractors (31.7.5.3.3) 2067
- istream.formatted (31.7.5.3) 2066
- istream.formatted.arithmetic (31.7.5.3.2) 2066
- istream.formatted.reqmts (31.7.5.3.1) 2066
- istream.general (31.7.5.2.1) 2063
- istream.iterator (24.6.2) 1250
- istream.iterator.cons (24.6.2.2) 1251
- istream.iterator.general (24.6.2.1) 1250
- istream.iterator.ops (24.6.2.3) 1251
- istream.manip (31.7.5.5) 2072
- istream.rvalue (31.7.5.6) 2073
- istream.sentry (31.7.5.2.4) 2065
- istream.syn (31.7.1) 2061
- istream.unformatted (31.7.5.4) 2068
- istreambuf.iterator (24.6.4) 1253
- istreambuf.iterator.cons (24.6.4.3) 1254
- istreambuf.iterator.general (24.6.4.1) 1253
- istreambuf.iterator.ops (24.6.4.4) 1254
- istreambuf.iterator.proxy (24.6.4.2) 1254
- istringstream (31.8.3) 2095
- istringstream.cons (31.8.3.2) 2097
- istringstream.general (31.8.3.1) 2095
- istringstream.members (31.8.3.4) 2098
- istringstream.swap (31.8.3.3) 2098
- iterator.assoc.types (24.3.2) 1201
- iterator.concept.bidir (24.3.4.12) 1212
- iterator.concept.contiguous (24.3.4.14) 1213
- iterator.concept.forward (24.3.4.11) 1212
- iterator.concept.inc (24.3.4.5) 1210
- iterator.concept.input (24.3.4.9) 1211
- iterator.concept.iterator (24.3.4.6) 1210
- iterator.concept.output (24.3.4.10) 1211
- iterator.concept.random.access (24.3.4.13) 1212
- iterator.concept.readable (24.3.4.2) 1207
- iterator.concept.sentinel (24.3.4.7) 1210
- iterator.concept.sizedsentinel (24.3.4.8) 1211
- iterator.concept.winc (24.3.4.4) 1208
- iterator.concept.writable (24.3.4.3) 1208
- iterator.concepts (24.3.4) 1207
- iterator.concepts.general (24.3.4.1) 1207

- iterator.cpp17 (24.3.5) 1213
- iterator.cpp17.general (24.3.5.1) 1214
- iterator.cust (24.3.3) 1206
- iterator.cust.move (24.3.3.1) 1206
- iterator.cust.swap (24.3.3.2) 1206
- iterator.iterators (24.3.5.2) 1214
- iterator.operations (24.4.3) 1222
- iterator.primitives (24.4) 1221
- iterator.primitives.general (24.4.1) 1221
- iterator.range (24.7) 1256
- iterator.requirements (24.3) 1200
- iterator.requirements.general (24.3.1) 1200
- iterator.synopsis (24.2) 1192
- iterator.traits (24.3.2.3) 1203
- iterators (Clause 24) 1192
- iterators.common (24.5.5) 1241
- iterators.counted (24.5.7) 1246
- iterators.general (24.1) 1192

- latch.syn (32.9.2.2) 2271
- length.error (19.2.6) 651
- lex (Clause 5) 13
- lex.bool (5.13.7) 29
- lex.ccon (5.13.3) 23
- lex.char (5.3) 15
- lex.charset (5.3.1) 15
- lex.comment (5.4) 17
- lex.digraph (5.9) 19
- lex.ext (5.13.9) 30
- lex.fcon (5.13.4) 25
- lex.header (5.6) 18
- lex.icon (5.13.2) 22
- lex.key (5.12) 21
- lex.literal (5.13) 22
- lex.literal.kinds (5.13.1) 22
- lex.name (5.11) 20
- lex.nullptr (5.13.8) 29
- lex.operators (5.8) 19
- lex.phases (5.2) 13
- lex.ppnumber (5.7) 19
- lex.pptoken (5.5) 17
- lex.separate (5.1) 13
- lex.string (5.13.5) 27
- lex.string.uneval (5.13.6) 29
- lex.token (5.10) 20
- lex.universal.char (5.3.2) 15
- lib.types.movedfrom (16.4.6.17) 578
- library (Clause 16) 547
- library.c (16.2) 548
- library.class.props (16.4.6.11) 577
- library.general (16.1) 547
- limits.syn (17.3.3) 589
- linalg (29.9) 1837
- linalg.algs.blas1 (29.9.13) 1866
- linalg.algs.blas1.add (29.9.13.6) 1867
- linalg.algs.blas1.asum (29.9.13.9) 1869
- linalg.algs.blas1.complexity (29.9.13.1) 1866
- linalg.algs.blas1.copy (29.9.13.5) 1867
- linalg.algs.blas1.dot (29.9.13.7) 1868
- linalg.algs.blas1.givens (29.9.13.2) 1866
- linalg.algs.blas1.givens.lartg (29.9.13.2.1) 1866
- linalg.algs.blas1.givens.rot (29.9.13.2.2) 1866
- linalg.algs.blas1.iamax (29.9.13.10) 1870
- linalg.algs.blas1.matfrobnorm (29.9.13.11) 1871
- linalg.algs.blas1.matinfnorm (29.9.13.13) 1872
- linalg.algs.blas1.matonenorm (29.9.13.12) 1871
- linalg.algs.blas1.nrm2 (29.9.13.8) 1869
- linalg.algs.blas1.scal (29.9.13.4) 1867
- linalg.algs.blas1.swap (29.9.13.3) 1867
- linalg.algs.blas2 (29.9.14) 1872
- linalg.algs.blas2.gemv (29.9.14.1) 1872
- linalg.algs.blas2.hemv (29.9.14.3) 1874
- linalg.algs.blas2.rank1 (29.9.14.6) 1877
- linalg.algs.blas2.rank2 (29.9.14.8) 1880
- linalg.algs.blas2.sympherrank1 (29.9.14.7) 1878
- linalg.algs.blas2.symv (29.9.14.2) 1873
- linalg.algs.blas2.trmv (29.9.14.4) 1875
- linalg.algs.blas2.trsv (29.9.14.5) 1876
- linalg.algs.blas3 (29.9.15) 1881
- linalg.algs.blas3.gemm (29.9.15.1) 1881
- linalg.algs.blas3.inplacetrsm (29.9.15.7) 1889
- linalg.algs.blas3.rank2k (29.9.15.5) 1886
- linalg.algs.blas3.rankk (29.9.15.4) 1885
- linalg.algs.blas3.trmm (29.9.15.3) 1884
- linalg.algs.blas3.trsm (29.9.15.6) 1887
- linalg.algs.blas3.xxmm (29.9.15.2) 1881
- linalg.algs.reqs (29.9.12) 1866
- linalg.conj (29.9.9) 1860
- linalg.conj.conjugated (29.9.9.3) 1861
- linalg.conj.conjugatedaccessor (29.9.9.2) 1860
- linalg.conj.intro (29.9.9.1) 1860
- linalg.conjtransposed (29.9.11) 1865
- linalg.general (29.9.3) 1850
- linalg.helpers (29.9.7) 1855
- linalg.helpers.abs (29.9.7.1) 1855
- linalg.helpers.concepts (29.9.7.5) 1856
- linalg.helpers.conj (29.9.7.2) 1855
- linalg.helpers.imag (29.9.7.4) 1856
- linalg.helpers.mandates (29.9.7.6) 1857
- linalg.helpers.precond (29.9.7.7) 1858
- linalg.helpers.real (29.9.7.3) 1856
- linalg.layout.packed (29.9.6) 1853
- linalg.layout.packed.cons (29.9.6.2) 1854
- linalg.layout.packed.obs (29.9.6.3) 1854
- linalg.layout.packed.overview (29.9.6.1) 1853
- linalg.overview (29.9.1) 1837
- linalg.reqs (29.9.4) 1851
- linalg.reqs.alg (29.9.4.2) 1851
- linalg.reqs.val (29.9.4.1) 1851
- linalg.scaled (29.9.8) 1858
- linalg.scaled.intro (29.9.8.1) 1859
- linalg.scaled.scaled (29.9.8.3) 1860
- linalg.scaled.scaledaccessor (29.9.8.2) 1859
- linalg.syn (29.9.2) 1837
- linalg.tags (29.9.5) 1852
- linalg.tags.diagonal (29.9.5.3) 1852
- linalg.tags.order (29.9.5.1) 1852
- linalg.tags.triangle (29.9.5.2) 1852

- linalg.transp (29.9.10) 1862
- linalg.transp.helpers (29.9.10.2) 1862
- linalg.transp.intro (29.9.10.1) 1862
- linalg.transp.layout.transpose (29.9.10.3) 1862
- linalg.transp.transposed (29.9.10.4) 1864
- list (23.3.11) 1021
- list.capacity (23.3.11.3) 1024
- list.cons (23.3.11.2) 1023
- list.erasure (23.3.11.6) 1027
- list.modifiers (23.3.11.4) 1024
- list.ops (23.3.11.5) 1025
- list.overview (23.3.11.1) 1021
- list.syn (23.3.10) 1020
- locale (28.3.3.1) 1638
- locale.categories (28.3.4) 1644
- locale.categories.general (28.3.4.1) 1644
- locale.category (28.3.3.1.2.1) 1640
- locale.codecvrt (28.3.4.2.5) 1650
- locale.codecvrt.byname (28.3.4.2.6) 1653
- locale.codecvrt.general (28.3.4.2.5.1) 1650
- locale.codecvrt.members (28.3.4.2.5.2) 1651
- locale.codecvrt.virtuals (28.3.4.2.5.3) 1651
- locale.collate (28.3.4.5.1) 1663
- locale.collate.byname (28.3.4.5.2) 1664
- locale.collate.general (28.3.4.5.1.1) 1663
- locale.collate.members (28.3.4.5.1.2) 1663
- locale.collate.virtuals (28.3.4.5.1.3) 1663
- locale.cons (28.3.3.1.3) 1642
- locale.convenience (28.3.3.3) 1644
- locale.ctype (28.3.4.2.2) 1645
- locale.ctype.byname (28.3.4.2.3) 1647
- locale.ctype.general (28.3.4.2.2.1) 1645
- locale.ctype.members (28.3.4.2.2.2) 1646
- locale.ctype.virtuals (28.3.4.2.2.3) 1646
- locale.facet (28.3.3.1.2.2) 1641
- locale.general (28.3.3.1.1) 1638
- locale.global.templates (28.3.3.2) 1644
- locale.id (28.3.3.1.2.3) 1642
- locale.members (28.3.3.1.4) 1643
- locale.messages (28.3.4.8.2) 1675
- locale.messages.byname (28.3.4.8.3) 1676
- locale.messages.general (28.3.4.8.2.1) 1675
- locale.messages.members (28.3.4.8.2.2) 1675
- locale.messages.virtuals (28.3.4.8.2.3) 1675
- locale.money.get (28.3.4.7.2) 1669
- locale.money.get.general (28.3.4.7.2.1) 1669
- locale.money.get.members (28.3.4.7.2.2) 1670
- locale.money.get.virtuals (28.3.4.7.2.3) 1670
- locale.money.put (28.3.4.7.3) 1671
- locale.money.put.general (28.3.4.7.3.1) 1671
- locale.money.put.members (28.3.4.7.3.2) 1671
- locale.money.put.virtuals (28.3.4.7.3.3) 1672
- locale.moneyput (28.3.4.7.4) 1672
- locale.moneyput.byname (28.3.4.7.5) 1674
- locale.moneyput.general (28.3.4.7.4.1) 1672
- locale.moneyput.members (28.3.4.7.4.2) 1673
- locale.moneyput.virtuals (28.3.4.7.4.3) 1674
- locale.nm.put (28.3.4.3.3) 1658
- locale.nm.put.general (28.3.4.3.3.1) 1658
- locale.num.get (28.3.4.3.2) 1654
- locale.num.get.general (28.3.4.3.2.1) 1654
- locale.numpunct (28.3.4.4.1) 1661
- locale.numpunct.byname (28.3.4.4.2) 1662
- locale.numpunct.general (28.3.4.4.1.1) 1661
- locale.operators (28.3.3.1.5) 1643
- locale.statics (28.3.3.1.6) 1643
- locale.syn (28.3.2) 1637
- locale.time.get (28.3.4.6.2) 1664
- locale.time.get.byname (28.3.4.6.3) 1668
- locale.time.get.general (28.3.4.6.2.1) 1664
- locale.time.get.members (28.3.4.6.2.2) 1665
- locale.time.get.virtuals (28.3.4.6.2.3) 1666
- locale.time.put (28.3.4.6.4) 1668
- locale.time.put.byname (28.3.4.6.5) 1669
- locale.time.put.general (28.3.4.6.4.1) 1668
- locale.time.put.members (28.3.4.6.4.2) 1668
- locale.time.put.virtuals (28.3.4.6.4.3) 1669
- locale.types (28.3.3.1.2) 1640
- locales (28.3.3) 1638
- localization (28.3) 1636
- localization.general (28.3.1) 1636
- logic.error (19.2.3) 650
- logical.operations (22.10.10) 930
- logical.operations.and (22.10.10.2) 930
- logical.operations.general (22.10.10.1) 930
- logical.operations.not (22.10.10.4) 931
- logical.operations.or (22.10.10.3) 931
- lower.bound (26.8.4.2) 1539
- macro.names (16.4.5.3.3) 572
- make.heap (26.8.8.4) 1556
- map (23.4.3) 1043
- map.access (23.4.3.3) 1047
- map.cons (23.4.3.2) 1047
- map.erasure (23.4.3.5) 1050
- map.modifiers (23.4.3.4) 1048
- map.overview (23.4.3.1) 1043
- mask.array.assign (29.6.8.2) 1822
- mask.array.comp.assign (29.6.8.3) 1822
- mask.array.fill (29.6.8.4) 1822
- math.constants (29.8.2) 1837
- mdspan.accessor (23.7.3.5) 1172
- mdspan.accessor.aligned (23.7.3.5.4) 1174
- mdspan.accessor.aligned.members (23.7.3.5.4.2) 1175
- mdspan.accessor.aligned.overview (23.7.3.5.4.1) 1174
- mdspan.accessor.default (23.7.3.5.3) 1173
- mdspan.accessor.default.members (23.7.3.5.3.2) 1173
- mdspan.accessor.default.overview (23.7.3.5.3.1) 1173
- mdspan.accessor.general (23.7.3.5.1) 1172
- mdspan.accessor.reqmts (23.7.3.5.2) 1172
- mdspan.copy (23.7.3.8) 1190
- mdspan.extents (23.7.3.3) 1146
- mdspan.extents.cmp (23.7.3.3.5) 1149
- mdspan.extents.cons (23.7.3.3.3) 1147

- mdspan.extents.dextents (23.7.3.3.6) 1149
- mdspan.extents.dims (23.7.3.3.7) 1149
- mdspan.extents.expo (23.7.3.3.2) 1147
- mdspan.extents.obs (23.7.3.3.4) 1149
- mdspan.extents.overview (23.7.3.3.1) 1146
- mdspan.layout (23.7.3.4) 1149
- mdspan.layout.general (23.7.3.4.1) 1149
- mdspan.layout.left (23.7.3.4.5) 1152
- mdspan.layout.left.cons (23.7.3.4.5.2) 1153
- mdspan.layout.left.obs (23.7.3.4.5.3) 1154
- mdspan.layout.left.overview (23.7.3.4.5.1) 1152
- mdspan.layout.leftpad (23.7.3.4.8) 1161
- mdspan.layout.leftpad.cons (23.7.3.4.8.3) 1163
- mdspan.layout.leftpad.expo (23.7.3.4.8.2) 1163
- mdspan.layout.leftpad.obs (23.7.3.4.8.4) 1165
- mdspan.layout.leftpad.overview (23.7.3.4.8.1) 1161
- mdspan.layout.policy.overview (23.7.3.4.4) 1151
- mdspan.layout.policy.reqmts (23.7.3.4.3) 1151
- mdspan.layout.reqmts (23.7.3.4.2) 1150
- mdspan.layout.right (23.7.3.4.6) 1155
- mdspan.layout.right.cons (23.7.3.4.6.2) 1156
- mdspan.layout.right.obs (23.7.3.4.6.3) 1157
- mdspan.layout.right.overview (23.7.3.4.6.1) 1155
- mdspan.layout.rightpad (23.7.3.4.9) 1166
- mdspan.layout.rightpad.cons (23.7.3.4.9.3) 1169
- mdspan.layout.rightpad.expo (23.7.3.4.9.2) 1168
- mdspan.layout.rightpad.obs (23.7.3.4.9.4) 1171
- mdspan.layout.rightpad.overview (23.7.3.4.9.1) 1166
- mdspan.layout.stride (23.7.3.4.7) 1157
- mdspan.layout.stride.cons (23.7.3.4.7.3) 1159
- mdspan.layout.stride.expo (23.7.3.4.7.2) 1159
- mdspan.layout.stride.obs (23.7.3.4.7.4) 1160
- mdspan.layout.stride.overview (23.7.3.4.7.1) 1158
- mdspan.mdspan (23.7.3.6) 1175
- mdspan.mdspan.cons (23.7.3.6.2) 1178
- mdspan.mdspan.members (23.7.3.6.3) 1180
- mdspan.mdspan.overview (23.7.3.6.1) 1175
- mdspan.overview (23.7.3.1) 1144
- mdspan.sub (23.7.3.7) 1181
- mdspan.sub.canonical (23.7.3.7.5) 1184
- mdspan.sub.extents (23.7.3.7.6) 1185
- mdspan.sub.helpers (23.7.3.7.4) 1183
- mdspan.sub.map (23.7.3.7.7) 1185
- mdspan.sub.map.common (23.7.3.7.7.2) 1186
- mdspan.sub.map.left (23.7.3.7.7.3) 1187
- mdspan.sub.map.leftpad (23.7.3.7.7.6) 1188
- mdspan.sub.map.result (23.7.3.7.7.3) 1183
- mdspan.sub.map.right (23.7.3.7.7.4) 1187
- mdspan.sub.map.rightpad (23.7.3.7.7.7) 1189
- mdspan.sub.map.sliceable (23.7.3.7.7.1) 1185
- mdspan.sub.map.stride (23.7.3.7.7.5) 1188
- mdspan.sub.overview (23.7.3.7.1) 1181
- mdspan.sub.range.slices (23.7.3.7.2) 1182
- mdspan.sub.sub (23.7.3.7.8) 1189
- mdspan.syn (23.7.3.2) 1144
- mem (Clause 20) 672
- mem.composite.types (20.5) 725
- mem.general (20.1) 672
- mem.poly allocator.class (20.6.3) 738
- mem.poly allocator.class.general (20.6.3.1) 738
- mem.poly allocator.ctor (20.6.3.2) 739
- mem.poly allocator.eq (20.6.3.4) 741
- mem.poly allocator.mem (20.6.3.3) 739
- mem.res (20.6) 737
- mem.res.class (20.6.2) 737
- mem.res.class.general (20.6.2.1) 737
- mem.res.eq (20.6.2.4) 738
- mem.res.global (20.6.4) 741
- mem.res.monotonic.buffer (20.6.6) 744
- mem.res.monotonic.buffer.ctor (20.6.6.2) 745
- mem.res.monotonic.buffer.general (20.6.6.1) 744
- mem.res.monotonic.buffer.mem (20.6.6.3) 745
- mem.res.pool (20.6.5) 741
- mem.res.pool.ctor (20.6.5.3) 743
- mem.res.pool.mem (20.6.5.4) 743
- mem.res.pool.options (20.6.5.2) 743
- mem.res.pool.overview (20.6.5.1) 741
- mem.res.private (20.6.2.3) 738
- mem.res.public (20.6.2.2) 737
- mem.res.syn (20.6.1) 737
- member.functions (16.4.6.5) 576
- memory (20.2) 672
- memory.general (20.2.1) 672
- memory.syn (20.2.2) 672
- meta (Clause 21) 750
- meta.const.eval (21.3.11) 778
- meta.define.static (21.4.3) 786
- meta.general (21.1) 750
- meta.help (21.3.4) 758
- meta.logical (21.3.9) 777
- meta.member (21.3.10) 778
- meta.reflection (21.4) 779
- meta.reflection.access.context (21.4.9) 801
- meta.reflection.access.queries (21.4.10) 802
- meta.reflection.annotation (21.4.13) 807
- meta.reflection.define.aggregate (21.4.17) 811
- meta.reflection.exception (21.4.4) 788
- meta.reflection.extract (21.4.14) 808
- meta.reflection.layout (21.4.12) 805
- meta.reflection.member.queries (21.4.11) 803
- meta.reflection.names (21.4.6) 789
- meta.reflection.operators (21.4.5) 789
- meta.reflection.queries (21.4.7) 792
- meta.reflection.result (21.4.16) 810
- meta.reflection.scope (21.4.8) 799
- meta.reflection.substitute (21.4.15) 809
- meta.reflection.traits (21.4.18) 813
- meta.rel (21.3.7) 769
- meta.rqmts (21.3.2) 751
- meta.string.literal (21.4.2) 786
- meta.syn (21.4.1) 779
- meta.trans (21.3.8) 771
- meta.trans.arr (21.3.8.5) 772
- meta.trans.cv (21.3.8.2) 771
- meta.trans.general (21.3.8.1) 771
- meta.trans.other (21.3.8.7) 773

- meta.trans.ptr (21.3.8.6) 773
- meta.trans.ref (21.3.8.3) 772
- meta.trans.sign (21.3.8.4) 772
- meta.type.synop (21.3.3) 751
- meta.unary (21.3.5) 758
- meta.unary.cat (21.3.5.2) 759
- meta.unary.comp (21.3.5.3) 760
- meta.unary.general (21.3.5.1) 759
- meta.unary.prop (21.3.5.4) 760
- meta.unary.prop.query (21.3.6) 768
- module (Clause 10) 302
- module.context (10.6) 310
- module.global.frag (10.4) 307
- module.import (10.3) 306
- module.interface (10.2) 303
- module.private.frag (10.5) 309
- module.reach (10.7) 311
- module.unit (10.1) 302
- move.iter.cons (24.5.4.4) 1238
- move.iter.elem (24.5.4.6) 1239
- move.iter.nav (24.5.4.7) 1239
- move.iter.nonmember (24.5.4.9) 1240
- move.iter.op.comp (24.5.4.8) 1239
- move.iter.op.conv (24.5.4.5) 1239
- move.iter.requirements (24.5.4.3) 1238
- move.iterator (24.5.4.2) 1237
- move.iterators (24.5.4) 1237
- move.iterators.general (24.5.4.1) 1237
- move.sent.ops (24.5.4.11) 1241
- move.sentinel (24.5.4.10) 1241
- multibyte.strings (16.3.3.3.4.3) 554
- multimap (23.4.4) 1050
- multimap.cons (23.4.4.2) 1053
- multimap.erasure (23.4.4.4) 1054
- multimap.modifiers (23.4.4.3) 1054
- multimap.overview (23.4.4.1) 1050
- multiset (23.4.7) 1059
- multiset.cons (23.4.7.2) 1062
- multiset.erasure (23.4.7.3) 1062
- multiset.overview (23.4.7.1) 1059
- mutex.syn (32.6.2) 2244

- namespace.alias (9.9.3) 282
- namespace.constraints (16.4.5.2) 571
- namespace.def (9.9.2) 280
- namespace.def.general (9.9.2.1) 280
- namespace.future (16.4.5.2.3) 572
- namespace.posix (16.4.5.2.2) 572
- namespace.qual (6.5.5.3) 56
- namespace.std (16.4.5.2.1) 571
- namespace.udecl (9.10) 284
- namespace.udir (9.9.4) 282
- namespace.unnamed (9.9.2.2) 281
- narrow.stream.objects (31.4.3) 2037
- new.badlength (17.6.4.2) 605
- new.delete (17.6.3) 601
- new.delete.array (17.6.3.3) 603
- new.delete.dataraces (17.6.3.5) 605
- new.delete.general (17.6.3.1) 601
- new.delete.placement (17.6.3.4) 604
- new.delete.single (17.6.3.2) 601
- new.handler (17.6.4.3) 605
- new.syn (17.6.2) 600
- nullablepointer.requirements (16.4.4.4) 564
- numarray (29.6) 1805
- numbers (29.8) 1836
- numbers.syn (29.8.1) 1837
- numeric.iota (26.10.13) 1578
- numeric.limits (17.3.5) 590
- numeric.limits.general (17.3.5.1) 590
- numeric.limits.members (17.3.5.2) 591
- numeric.ops (26.10) 1569
- numeric.ops.gcd (26.10.14) 1578
- numeric.ops.general (26.10.1) 1569
- numeric.ops.lcm (26.10.15) 1578
- numeric.ops.midpoint (26.10.16) 1578
- numeric.ops.overview (26.9) 1565
- numeric.requirements (29.2) 1751
- numeric.sat (26.10.17) 1579
- numeric.sat.cast (26.10.17.2) 1579
- numeric.sat.func (26.10.17.1) 1579
- numeric.special (17.3.5.3) 594
- numerics (Clause 29) 1751
- numerics.c (29.11) 1940
- numerics.c.ckdint (29.11.2) 1940
- numerics.defns (26.10.2) 1569
- numerics.general (29.1) 1751

- obj.lifetime (20.2.6) 686
- objects.within.classes (16.3.3.6) 555
- ofstream (31.10.5) 2120
- ofstream.cons (31.10.5.2) 2121
- ofstream.general (31.10.5.1) 2120
- ofstream.members (31.10.5.4) 2122
- ofstream.swap (31.10.5.3) 2121
- optional (22.5) 852
- optional.assign (22.5.3.4) 857
- optional.bad.access (22.5.6) 867
- optional.comp.with.t (22.5.9) 869
- optional.ctor (22.5.3.2) 855
- optional.dtor (22.5.3.3) 856
- optional.general (22.5.1) 852
- optional.hash (22.5.11) 871
- optional.iterators (22.5.3.6) 860
- optional.mod (22.5.3.9) 862
- optional.monadic (22.5.3.8) 861
- optional.nullopts (22.5.8) 869
- optional.nullopt (22.5.5) 867
- optional.observe (22.5.3.7) 860
- optional.optional (22.5.3) 853
- optional.optional.general (22.5.3.1) 853
- optional.optional.ref (22.5.4) 862
- optional.optional.ref.general (22.5.4.1) 863
- optional.ref.assign (22.5.4.3) 865
- optional.ref.ctor (22.5.4.2) 864
- optional.ref.expos (22.5.4.9) 867
- optional.ref.iterators (22.5.4.5) 866
- optional.ref.mod (22.5.4.8) 867

- optional.ref.monadic (22.5.4.7) 866
- optional.ref.observe (22.5.4.6) 866
- optional.ref.swap (22.5.4.4) 865
- optional.relops (22.5.7) 868
- optional.specalg (22.5.10) 871
- optional.swap (22.5.3.5) 859
- optional.syn (22.5.2) 852
- organization (16.4.2) 557
- organization.general (16.4.2.1) 557
- ospanstream (31.9.5) 2109
- ospanstream.cons (31.9.5.2) 2110
- ospanstream.general (31.9.5.1) 2109
- ospanstream.members (31.9.5.4) 2110
- ospanstream.swap (31.9.5.3) 2110
- ostream (31.7.6.2) 2074
- ostream.assign (31.7.6.2.3) 2076
- ostream.cons (31.7.6.2.2) 2076
- ostream.formatted (31.7.6.3) 2077
- ostream.formatted.print (31.7.6.3.5) 2080
- ostream.formatted.reqmts (31.7.6.3.1) 2077
- ostream.general (31.7.6.2.1) 2074
- ostream.inserters (31.7.6.3.3) 2079
- ostream.inserters.arithmetic (31.7.6.3.2) 2078
- ostream.inserters.character (31.7.6.3.4) 2080
- ostream.iterator (24.6.3) 1252
- ostream.iterator.cons.des (24.6.3.2) 1252
- ostream.iterator.general (24.6.3.1) 1252
- ostream.iterator.ops (24.6.3.3) 1253
- ostream.manip (31.7.6.5) 2082
- ostream.rvalue (31.7.6.6) 2082
- ostream.seek (31.7.6.2.5) 2077
- ostream.sentry (31.7.6.2.4) 2076
- ostream.syn (31.7.2) 2061
- ostream.unformatted (31.7.6.4) 2081
- ostreambuf.iter.cons (24.6.5.2) 1255
- ostreambuf.iter.ops (24.6.5.3) 1255
- ostreambuf.iterator (24.6.5) 1255
- ostreambuf.iterator.general (24.6.5.1) 1255
- ostreamstream (31.8.4) 2098
- ostreamstream.cons (31.8.4.2) 2100
- ostreamstream.general (31.8.4.1) 2098
- ostreamstream.members (31.8.4.4) 2101
- ostreamstream.swap (31.8.4.3) 2101
- out.of.range (19.2.7) 651
- out.ptr (20.4.4.2) 723
- out.ptr.t (20.4.4.1) 722
- output.iterators (24.3.5.4) 1215
- output.streams (31.7.6) 2074
- output.streams.general (31.7.6.1) 2074
- over (Clause 12) 377
- over.assign (12.4.3.2) 409
- over.best.ics (12.2.4.2) 396
- over.best.ics.general (12.2.4.2.1) 396
- over.binary (12.4.3) 409
- over.binary.general (12.4.3.1) 409
- over.built (12.5) 412
- over.call (12.4.4) 409
- over.call.func (12.2.2.2.2) 380
- over.call.object (12.2.2.2.3) 381
- over.ics.ellipsis (12.2.4.2.4) 399
- over.ics.list (12.2.4.2.6) 399
- over.ics.rank (12.2.4.3) 402
- over.ics.ref (12.2.4.2.5) 399
- over.ics.scs (12.2.4.2.2) 398
- over.ics.user (12.2.4.2.3) 398
- over.inc (12.4.7) 410
- over.inc.default (12.4.7.2) 411
- over.inc.general (12.4.7.1) 410
- over.literal (12.6) 414
- over.match (12.2) 377
- over.match.best (12.2.4) 393
- over.match.best.general (12.2.4.1) 393
- over.match.call (12.2.2.2) 379
- over.match.call.general (12.2.2.2.1) 380
- over.match.class.deduct (12.2.2.9) 387
- over.match.conv (12.2.2.6) 386
- over.match.copy (12.2.2.5) 385
- over.match.ctor (12.2.2.4) 385
- over.match.funcs (12.2.2) 378
- over.match.funcs.general (12.2.2.1) 378
- over.match.general (12.2.1) 377
- over.match.list (12.2.2.8) 386
- over.match.oper (12.2.2.3) 382
- over.match.ref (12.2.2.7) 386
- over.match.viable (12.2.3) 392
- over.oper (12.4) 407
- over.oper.general (12.4.1) 407
- over.over (12.3) 406
- over.pre (12.1) 377
- over.ref (12.4.6) 410
- over.sub (12.4.5) 410
- over.unary (12.4.2) 408
- overflow.error (19.2.10) 652
- pair.astuple (22.3.4) 837
- pair.pieces (22.3.5) 837
- pairs (22.3) 832
- pairs.general (22.3.1) 832
- pairs.pair (22.3.2) 832
- pairs.spec (22.3.3) 836
- partial.sort (26.8.2.3) 1534
- partial.sort.copy (26.8.2.4) 1535
- partial.sum (26.10.7) 1572
- pointer.conversion (20.2.4) 685
- pointer.traits (20.2.3) 684
- pointer.traits.functions (20.2.3.3) 684
- pointer.traits.general (20.2.3.1) 684
- pointer.traits.optmem (20.2.3.4) 685
- pointer.traits.types (20.2.3.2) 684
- polymorphic (20.5.2) 732
- polymorphic.assign (20.5.2.5) 735
- polymorphic.ctor (20.5.2.3) 733
- polymorphic.dtor (20.5.2.4) 735
- polymorphic.general (20.5.2.1) 732
- polymorphic.obs (20.5.2.6) 736
- polymorphic.swap (20.5.2.7) 736
- polymorphic.syn (20.5.2.2) 732
- pop.heap (26.8.8.3) 1555

- predef.iterators (24.5) 1224
- print.fun (31.7.10) 2087
- print.syn (31.7.4) 2062
- priority.queue (23.6.4) 1094
- priqueue.cons (23.6.4.2) 1096
- priqueue.cons.alloc (23.6.4.3) 1096
- priqueue.members (23.6.4.4) 1097
- priqueue.overview (23.6.4.1) 1094
- priqueue.special (23.6.4.5) 1098
- projected (24.3.6.4) 1219
- propagation (17.9.7) 614
- protection.within.classes (16.4.6.12) 577
- ptr.align (20.2.5) 685
- ptr.laundry (17.6.5) 606
- ptrtag (20.3) 694
- ptrtag.bits (20.3.1) 694
- ptrtag.pair (20.3.2) 694
- ptrtag.pair.accessors (20.3.2.5) 696
- ptrtag.pair.comp (20.3.2.7) 697
- ptrtag.pair.cons (20.3.2.2) 695
- ptrtag.pair.general (20.3.2.1) 694
- ptrtag.pair.get (20.3.2.8) 697
- ptrtag.pair.overalign (20.3.2.3) 696
- ptrtag.pair.swap (20.3.2.6) 697
- ptrtag.pair.tagops (20.3.2.4) 696
- push.heap (26.8.8.2) 1555

- queue (23.6.3) 1091
- queue.cons (23.6.3.2) 1092
- queue.cons.alloc (23.6.3.3) 1092
- queue.defn (23.6.3.1) 1091
- queue.mod (23.6.3.4) 1093
- queue.ops (23.6.3.5) 1093
- queue.special (23.6.3.6) 1093
- queue.syn (23.6.2) 1090
- quoted.manip (31.7.9) 2086

- rand (29.5) 1759
- rand.adapt (29.5.5) 1776
- rand.adapt.disc (29.5.5.2) 1776
- rand.adapt.general (29.5.5.1) 1776
- rand.adapt.ibits (29.5.5.3) 1777
- rand.adapt.shuf (29.5.5.4) 1778
- rand.device (29.5.7) 1780
- rand.dist (29.5.9) 1784
- rand.dist.bern (29.5.9.3) 1786
- rand.dist.bern.bernoulli (29.5.9.3.1) 1786
- rand.dist.bern.bin (29.5.9.3.2) 1787
- rand.dist.bern.geo (29.5.9.3.3) 1788
- rand.dist.bern.negbin (29.5.9.3.4) 1789
- rand.dist.general (29.5.9.1) 1784
- rand.dist.norm (29.5.9.5) 1794
- rand.dist.norm.cauchy (29.5.9.5.4) 1797
- rand.dist.norm.chisq (29.5.9.5.3) 1796
- rand.dist.norm.f (29.5.9.5.5) 1798
- rand.dist.norm.lognormal (29.5.9.5.2) 1795
- rand.dist.norm.normal (29.5.9.5.1) 1794
- rand.dist.norm.t (29.5.9.5.6) 1799
- rand.dist.pois (29.5.9.4) 1789
- rand.dist.pois.exp (29.5.9.4.2) 1790
- rand.dist.pois.extreme (29.5.9.4.5) 1793
- rand.dist.pois.gamma (29.5.9.4.3) 1791
- rand.dist.pois.poisson (29.5.9.4.1) 1790
- rand.dist.pois.weibull (29.5.9.4.4) 1792
- rand.dist.samp (29.5.9.6) 1800
- rand.dist.samp.discrete (29.5.9.6.1) 1800
- rand.dist.samp.pconst (29.5.9.6.2) 1801
- rand.dist.samp.plinear (29.5.9.6.3) 1803
- rand.dist.uni (29.5.9.2) 1784
- rand.dist.uni.int (29.5.9.2.1) 1784
- rand.dist.uni.real (29.5.9.2.2) 1785
- rand.eng (29.5.4) 1769
- rand.eng.general (29.5.4.1) 1769
- rand.eng.lcong (29.5.4.2) 1770
- rand.eng.mers (29.5.4.3) 1771
- rand.eng.philox (29.5.4.5) 1774
- rand.eng.sub (29.5.4.4) 1772
- rand.general (29.5.1) 1759
- rand.predef (29.5.6) 1779
- rand.req (29.5.3) 1762
- rand.req.adapt (29.5.3.5) 1766
- rand.req.dist (29.5.3.6) 1767
- rand.req.eng (29.5.3.4) 1764
- rand.req.genl (29.5.3.1) 1763
- rand.req.seedseq (29.5.3.2) 1763
- rand.req.urng (29.5.3.3) 1764
- rand.synopsis (29.5.2) 1760
- rand.util (29.5.8) 1781
- rand.util.canonical (29.5.8.2) 1783
- rand.util.seedseq (29.5.8.1) 1781
- random.access.iterators (24.3.5.7) 1216
- range.access (25.3) 1268
- range.access.begin (25.3.2) 1268
- range.access.cbegin (25.3.4) 1269
- range.access.cend (25.3.5) 1269
- range.access.crbegin (25.3.8) 1270
- range.access.crend (25.3.9) 1271
- range.access.end (25.3.3) 1269
- range.access.general (25.3.1) 1268
- range.access.rbegin (25.3.6) 1269
- range.access.rend (25.3.7) 1270
- range.adaptor.helpers (25.7.5) 1300
- range.adaptor.object (25.7.2) 1298
- range.adaptors (25.7) 1298
- range.adaptors.general (25.7.1) 1298
- range.adjacent (25.7.27) 1375
- range.adjacent.iterator (25.7.27.3) 1376
- range.adjacent.overview (25.7.27.1) 1375
- range.adjacent.sentinel (25.7.27.4) 1379
- range.adjacent.transform (25.7.28) 1380
- range.adjacent.transform.iterator (25.7.28.3) 1382
- range.adjacent.transform.overview (25.7.28.1) 1380
- range.adjacent.transform.sentinel (25.7.28.4) 1384
- range.adjacent.transform.view (25.7.28.2) 1381
- range.adjacent.view (25.7.27.2) 1375

- range.all (25.7.6) 1301
- range.all.general (25.7.6.1) 1301
- range.approximately.sized (25.4.3) 1274
- range.as.const (25.7.22) 1353
- range.as.const.overview (25.7.22.1) 1353
- range.as.const.view (25.7.22.2) 1353
- range.as.input (25.7.35) 1417
- range.as.input.iterator (25.7.35.3) 1418
- range.as.input.overview (25.7.35.1) 1417
- range.as.input.view (25.7.35.2) 1417
- range.as.rvalue (25.7.7) 1303
- range.as.rvalue.overview (25.7.7.1) 1303
- range.as.rvalue.view (25.7.7.2) 1303
- range.cache.latest (25.7.34) 1414
- range.cache.latest.iterator (25.7.34.3) 1415
- range.cache.latest.overview (25.7.34.1) 1414
- range.cache.latest.sentinel (25.7.34.4) 1416
- range.cache.latest.view (25.7.34.2) 1414
- range.cartesian (25.7.33) 1407
- range.cartesian.iterator (25.7.33.3) 1410
- range.cartesian.overview (25.7.33.1) 1407
- range.cartesian.view (25.7.33.2) 1408
- range.chunk (25.7.29) 1385
- range.chunk.by (25.7.31) 1400
- range.chunk.by.iter (25.7.31.3) 1401
- range.chunk.by.overview (25.7.31.1) 1400
- range.chunk.by.view (25.7.31.2) 1400
- range.chunk.fwd.iter (25.7.29.7) 1391
- range.chunk.inner.iter (25.7.29.5) 1388
- range.chunk.outer.iter (25.7.29.3) 1387
- range.chunk.outer.value (25.7.29.4) 1388
- range.chunk.overview (25.7.29.1) 1385
- range.chunk.view.fwd (25.7.29.6) 1390
- range.chunk.view.input (25.7.29.2) 1386
- range.cmp (22.10.9) 929
- range.common (25.7.20) 1350
- range.common.overview (25.7.20.1) 1350
- range.common.view (25.7.20.2) 1350
- range.concat (25.7.18) 1341
- range.concat.iterator (25.7.18.3) 1344
- range.concat.overview (25.7.18.1) 1341
- range.concat.view (25.7.18.2) 1341
- range.counted (25.7.19) 1350
- range.dangling (25.5.5) 1282
- range.drop (25.7.12) 1318
- range.drop.overview (25.7.12.1) 1318
- range.drop.view (25.7.12.2) 1319
- range.drop.while (25.7.13) 1320
- range.drop.while.overview (25.7.13.1) 1320
- range.drop.while.view (25.7.13.2) 1320
- range.elements (25.7.23) 1354
- range.elements.iterator (25.7.23.3) 1356
- range.elements.overview (25.7.23.1) 1354
- range.elements.sentinel (25.7.23.4) 1359
- range.elements.view (25.7.23.2) 1355
- range.elementsof (25.5.6) 1282
- range.empty (25.6.2) 1285
- range.empty.overview (25.6.2.1) 1285
- range.empty.view (25.6.2.2) 1285
- range.enumerate (25.7.24) 1360
- range.enumerate.iterator (25.7.24.3) 1361
- range.enumerate.overview (25.7.24.1) 1360
- range.enumerate.sentinel (25.7.24.4) 1363
- range.enumerate.view (25.7.24.2) 1360
- range.error (19.2.9) 652
- range.factories (25.6) 1285
- range.factories.general (25.6.1) 1285
- range.filter (25.7.8) 1304
- range.filter.iterator (25.7.8.3) 1305
- range.filter.overview (25.7.8.1) 1304
- range.filter.sentinel (25.7.8.4) 1307
- range.filter.view (25.7.8.2) 1304
- range.iota (25.6.4) 1286
- range.iota.iterator (25.6.4.3) 1289
- range.iota.overview (25.6.4.1) 1287
- range.iota.sentinel (25.6.4.4) 1292
- range.iota.view (25.6.4.2) 1287
- range.istream (25.6.6) 1296
- range.istream.iterator (25.6.6.3) 1297
- range.istream.overview (25.6.6.1) 1296
- range.istream.view (25.6.6.2) 1296
- range.iter.op.advance (24.4.4.2) 1223
- range.iter.op.distance (24.4.4.3) 1223
- range.iter.op.next (24.4.4.4) 1224
- range.iter.op.prev (24.4.4.5) 1224
- range.iter.ops (24.4.4) 1223
- range.iter.ops.general (24.4.4.1) 1223
- range.join (25.7.14) 1321
- range.join.iterator (25.7.14.3) 1322
- range.join.overview (25.7.14.1) 1321
- range.join.sentinel (25.7.14.4) 1325
- range.join.view (25.7.14.2) 1321
- range.join.with (25.7.15) 1326
- range.join.with.iterator (25.7.15.3) 1328
- range.join.with.overview (25.7.15.1) 1326
- range.join.with.sentinel (25.7.15.4) 1332
- range.join.with.view (25.7.15.2) 1326
- range.lazy.split (25.7.16) 1332
- range.lazy.split.inner (25.7.16.5) 1336
- range.lazy.split.outer (25.7.16.3) 1334
- range.lazy.split.outer.value (25.7.16.4) 1336
- range.lazy.split.overview (25.7.16.1) 1332
- range.lazy.split.view (25.7.16.2) 1332
- range.move.wrap (25.7.3) 1299
- range.nonprop.cache (25.7.4) 1299
- range.owning.view (25.7.6.3) 1302
- range.prim.cdata (25.3.15) 1272
- range.prim.data (25.3.14) 1272
- range.prim.empty (25.3.13) 1272
- range.prim.size (25.3.10) 1271
- range.prim.size.hint (25.3.12) 1271
- range.prim.ssize (25.3.11) 1271
- range.range (25.4.2) 1273
- range.ref.view (25.7.6.2) 1301
- range.refinements (25.4.6) 1275
- range.repeat (25.6.5) 1293
- range.repeat.iterator (25.6.5.3) 1294
- range.repeat.overview (25.6.5.1) 1293

- range.repeat.view (25.6.5.2) 1293
- range.req (25.4) 1272
- range.req.general (25.4.1) 1273
- range.reverse (25.7.21) 1351
- range.reverse.overview (25.7.21.1) 1351
- range.reverse.view (25.7.21.2) 1352
- range.single (25.6.3) 1285
- range.single.overview (25.6.3.1) 1285
- range.single.view (25.6.3.2) 1285
- range.sized (25.4.4) 1274
- range.slide (25.7.30) 1394
- range.slide.iterator (25.7.30.3) 1396
- range.slide.overview (25.7.30.1) 1394
- range.slide.sentinel (25.7.30.4) 1399
- range.slide.view (25.7.30.2) 1394
- range.split (25.7.17) 1338
- range.split.iterator (25.7.17.3) 1339
- range.split.overview (25.7.17.1) 1338
- range.split.sentinel (25.7.17.4) 1340
- range.split.view (25.7.17.2) 1338
- range.stride (25.7.32) 1402
- range.stride.iterator (25.7.32.3) 1404
- range.stride.overview (25.7.32.1) 1402
- range.stride.view (25.7.32.2) 1402
- range.subrange (25.5.4) 1279
- range.subrange.access (25.5.4.3) 1281
- range.subrange.ctor (25.5.4.2) 1280
- range.subrange.general (25.5.4.1) 1279
- range.take (25.7.10) 1313
- range.take.overview (25.7.10.1) 1313
- range.take.sentinel (25.7.10.3) 1316
- range.take.view (25.7.10.2) 1314
- range.take.while (25.7.11) 1316
- range.take.while.overview (25.7.11.1) 1316
- range.take.while.sentinel (25.7.11.3) 1317
- range.take.while.view (25.7.11.2) 1317
- range.transform (25.7.9) 1308
- range.transform.iterator (25.7.9.3) 1309
- range.transform.overview (25.7.9.1) 1308
- range.transform.sentinel (25.7.9.4) 1312
- range.transform.view (25.7.9.2) 1308
- range.utility (25.5) 1276
- range.utility.conv (25.5.7) 1283
- range.utility.conv.adapters (25.5.7.3) 1284
- range.utility.conv.general (25.5.7.1) 1283
- range.utility.conv.to (25.5.7.2) 1283
- range.utility.general (25.5.1) 1276
- range.utility.helpers (25.5.2) 1276
- range.view (25.4.5) 1274
- range.zip (25.7.25) 1364
- range.zip.iterator (25.7.25.3) 1366
- range.zip.overview (25.7.25.1) 1364
- range.zip.sentinel (25.7.25.4) 1369
- range.zip.transform (25.7.26) 1370
- range.zip.transform.iterator (25.7.26.3) 1371
- range.zip.transform.overview (25.7.26.1) 1370
- range.zip.transform.sentinel (25.7.26.4) 1374
- range.zip.transform.view (25.7.26.2) 1370
- range.zip.view (25.7.25.2) 1365
- ranges (Clause 25) 1259
- ranges.general (25.1) 1259
- ranges.syn (25.2) 1259
- ratio (21.5) 817
- ratio.arithmetic (21.5.4) 819
- ratio.comparison (21.5.5) 819
- ratio.general (21.5.1) 817
- ratio.ratio (21.5.3) 818
- ratio.si (21.5.6) 820
- ratio.syn (21.5.2) 817
- rcu.syn (32.11.2.2) 2288
- re (28.6) 1713
- re.alg (28.6.10) 1735
- re.alg.match (28.6.10.2) 1735
- re.alg.replace (28.6.10.4) 1738
- re.alg.search (28.6.10.3) 1737
- re.badexp (28.6.5) 1722
- re.const (28.6.4) 1719
- re.const.general (28.6.4.1) 1719
- re.err (28.6.4.4) 1721
- re.except (28.6.10.1) 1735
- re.general (28.6.1) 1713
- re.grammar (28.6.12) 1745
- re.iter (28.6.11) 1739
- re.matchflag (28.6.4.3) 1719
- re.regex (28.6.7) 1724
- re.regex.assign (28.6.7.3) 1727
- re.regex.construct (28.6.7.2) 1726
- re.regex.general (28.6.7.1) 1724
- re.regex.locale (28.6.7.5) 1728
- re.regex.nonmemb (28.6.7.7) 1728
- re.regex.operations (28.6.7.4) 1728
- re.regex.swap (28.6.7.6) 1728
- re.regiter (28.6.11.1) 1740
- re.regiter.cnstr (28.6.11.1.2) 1740
- re.regiter.comp (28.6.11.1.3) 1741
- re.regiter.deref (28.6.11.1.4) 1741
- re.regiter.general (28.6.11.1.1) 1740
- re.regiter.incr (28.6.11.1.5) 1741
- re.req (28.6.2) 1713
- re.results (28.6.9) 1731
- re.results.acc (28.6.9.5) 1733
- re.results.all (28.6.9.7) 1735
- re.results.const (28.6.9.2) 1732
- re.results.form (28.6.9.6) 1734
- re.results.general (28.6.9.1) 1731
- re.results.nonmember (28.6.9.9) 1735
- re.results.size (28.6.9.4) 1733
- re.results.state (28.6.9.3) 1733
- re.results.swap (28.6.9.8) 1735
- re.submatch (28.6.8) 1729
- re.submatch.general (28.6.8.1) 1729
- re.submatch.members (28.6.8.2) 1729
- re.submatch.op (28.6.8.3) 1730
- re.syn (28.6.3) 1715
- re.synopt (28.6.4.2) 1719
- re.tokiter (28.6.11.2) 1742
- re.tokiter.cnstr (28.6.11.2.2) 1743
- re.tokiter.comp (28.6.11.2.3) 1744

- re.tokiter.deref (28.6.11.2.4) 1744
- re.tokiter.general (28.6.11.2.1) 1742
- re.tokiter.incr (28.6.11.2.5) 1744
- re.traits (28.6.6) 1722
- readable.traits (24.3.2.2) 1202
- reduce (26.10.4) 1570
- reentrancy (16.4.6.9) 577
- refwrap (22.10.6) 921
- refwrap.access (22.10.6.4) 922
- refwrap.assign (22.10.6.3) 922
- refwrap.common.ref (22.10.6.8) 924
- refwrap.comparisons (22.10.6.6) 923
- refwrap.const (22.10.6.2) 922
- refwrap.general (22.10.6.1) 921
- refwrap.helpers (22.10.6.7) 923
- refwrap.invoke (22.10.6.5) 923
- replacement.functions (16.4.5.6) 574
- requirements (16.4) 556
- requirements.general (16.4.1) 556
- res.contract.assertions (16.4.6.15) 578
- res.on.arguments (16.4.5.9) 575
- res.on.data.races (16.4.6.10) 577
- res.on.exception.handling (16.4.6.14) 578
- res.on.functions (16.4.5.8) 574
- res.on.headers (16.4.6.2) 576
- res.on.macro.definitions (16.4.6.3) 576
- res.on.objects (16.4.5.10) 575
- res.on.requirements (16.4.5.11) 575
- reserved.names (16.4.5.3) 572
- reserved.names.general (16.4.5.3.1) 572
- reverse.iter.cmp (24.5.1.8) 1227
- reverse.iter.cons (24.5.1.4) 1226
- reverse.iter.conv (24.5.1.5) 1226
- reverse.iter.elem (24.5.1.6) 1226
- reverse.iter.nav (24.5.1.7) 1226
- reverse.iter.nonmember (24.5.1.9) 1228
- reverse.iter.requirements (24.5.1.3) 1225
- reverse.iterator (24.5.1.2) 1225
- reverse.iterators (24.5.1) 1225
- reverse.iterators.general (24.5.1.1) 1225
- round.style (17.3.4) 589
- runtime.error (19.2.8) 652
- safercl (32.11) 2288
- safercl.general (32.11.1) 2288
- safercl.hp (32.11.3) 2291
- safercl.hp.base (32.11.3.3) 2292
- safercl.hp.general (32.11.3.1) 2291
- safercl.hp.holder (32.11.3.4) 2292
- safercl.hp.holder.ctor (32.11.3.4.2) 2293
- safercl.hp.holder.general (32.11.3.4.1) 2293
- safercl.hp.holder.mem (32.11.3.4.3) 2293
- safercl.hp.holder.nonmem (32.11.3.4.4) 2294
- safercl.rcu (32.11.2) 2288
- safercl.rcu.base (32.11.2.3) 2289
- safercl.rcu.domain (32.11.2.4) 2289
- safercl.rcu.domain.func (32.11.2.4.3) 2290
- safercl.rcu.domain.general (32.11.2.4.1) 2289
- safercl.rcu.domain.members (32.11.2.4.2) 2290
- safercl.rcu.general (32.11.2.1) 2288
- scoped.adaptor.operators (20.7.5) 749
- semaphore.syn (32.8.2) 2269
- sequence.reqmts (23.2.4) 967
- sequences (23.3) 995
- sequences.general (23.3.1) 995
- set (23.4.6) 1055
- set.cons (23.4.6.2) 1058
- set.difference (26.8.7.5) 1552
- set.erasure (23.4.6.3) 1058
- set.intersection (26.8.7.4) 1550
- set.modifiers (23.4.6.4) 1059
- set.new.handler (17.6.4.4) 606
- set.overview (23.4.6.1) 1055
- set.symmetric.difference (26.8.7.6) 1553
- set.terminate (17.9.5.2) 613
- set.union (26.8.7.3) 1549
- sf.cmath (29.7.6) 1831
- sf.cmath.assoc.laguerre (29.7.6.2) 1831
- sf.cmath.assoc.legendre (29.7.6.3) 1832
- sf.cmath.beta (29.7.6.4) 1832
- sf.cmath.comp.ellint.1 (29.7.6.5) 1832
- sf.cmath.comp.ellint.2 (29.7.6.6) 1832
- sf.cmath.comp.ellint.3 (29.7.6.7) 1832
- sf.cmath.cyl.bessel.i (29.7.6.8) 1833
- sf.cmath.cyl.bessel.j (29.7.6.9) 1833
- sf.cmath.cyl.bessel.k (29.7.6.10) 1833
- sf.cmath.cyl.neumann (29.7.6.11) 1834
- sf.cmath.ellint.1 (29.7.6.12) 1834
- sf.cmath.ellint.2 (29.7.6.13) 1834
- sf.cmath.ellint.3 (29.7.6.14) 1834
- sf.cmath.expint (29.7.6.15) 1834
- sf.cmath.general (29.7.6.1) 1831
- sf.cmath.hermite (29.7.6.16) 1835
- sf.cmath.laguerre (29.7.6.17) 1835
- sf.cmath.legendre (29.7.6.18) 1835
- sf.cmath.riemann.zeta (29.7.6.19) 1835
- sf.cmath.sph.bessel (29.7.6.20) 1836
- sf.cmath.sph.legendre (29.7.6.21) 1836
- sf.cmath.sph.neumann (29.7.6.22) 1836
- shared.mutex.syn (32.6.3) 2244
- simd (29.10) 1890
- simd.alg (29.10.8.12) 1924
- simd.binary (29.10.8.1) 1914
- simd.bit (29.10.8.14) 1932
- simd.cassign (29.10.8.2) 1915
- simd.class (29.10.7) 1910
- simd.comparison (29.10.8.3) 1915
- simd.complex.access (29.10.7.4) 1913
- simd.complex.math (29.10.8.15) 1934
- simd.cond (29.10.8.4) 1915
- simd.creation (29.10.8.11) 1923
- simd.ctor (29.10.7.2) 1912
- simd.expos (29.10.2) 1891
- simd.expos.abi (29.10.2.2) 1892
- simd.expos.defn (29.10.2.1) 1892
- simd.flags (29.10.5) 1907
- simd.flags.oper (29.10.5.2) 1908
- simd.flags.overview (29.10.5.1) 1907

- simd.general (29.10.1) 1890
- simd.iterator (29.10.6) 1908
- simd.loadstore (29.10.8.6) 1917
- simd.mask.binary (29.10.10.1) 1938
- simd.mask.cassign (29.10.10.2) 1938
- simd.mask.class (29.10.9) 1935
- simd.mask.comparison (29.10.10.3) 1938
- simd.mask.cond (29.10.10.4) 1939
- simd.mask.conv (29.10.9.5) 1938
- simd.mask.ctor (29.10.9.2) 1937
- simd.mask.nonmembers (29.10.10) 1938
- simd.mask.overview (29.10.9.1) 1935
- simd.mask.reductions (29.10.10.5) 1939
- simd.mask.subscr (29.10.9.3) 1937
- simd.mask.unary (29.10.9.4) 1937
- simd.math (29.10.8.13) 1925
- simd.nonmembers (29.10.8) 1914
- simd.overview (29.10.7.1) 1910
- simd.permute.dynamic (29.10.8.8) 1921
- simd.permute.mask (29.10.8.9) 1921
- simd.permute.memory (29.10.8.10) 1922
- simd.permute.static (29.10.8.7) 1920
- simd.reductions (29.10.8.5) 1916
- simd.subscr (29.10.7.3) 1913
- simd.syn (29.10.3) 1893
- simd.traits (29.10.4) 1907
- simd.unary (29.10.7.5) 1914
- slice.access (29.6.4.3) 1818
- slice.arr.assign (29.6.5.2) 1818
- slice.arr.comp.assign (29.6.5.3) 1819
- slice.arr.fill (29.6.5.4) 1819
- slice.ops (29.6.4.4) 1818
- smartptr (20.4) 697
- smartptr.adapt (20.4.4) 722
- sort (26.8.2.1) 1533
- sort.heap (26.8.8.5) 1556
- source.location.syn (17.8.1) 610
- span.cons (23.7.2.2.2) 1140
- span.deduct (23.7.2.2.3) 1142
- span.elem (23.7.2.2.6) 1143
- span.iterators (23.7.2.2.7) 1143
- span.objectrep (23.7.2.3) 1144
- span.obs (23.7.2.2.5) 1143
- span.overview (23.7.2.2.1) 1138
- span.streams (31.9) 2104
- span.streams.overview (31.9.1) 2104
- span.sub (23.7.2.2.4) 1142
- span.syn (23.7.2.1) 1138
- spanbuf (31.9.3) 2105
- spanbuf.assign (31.9.3.3) 2106
- spanbuf.cons (31.9.3.2) 2106
- spanbuf.general (31.9.3.1) 2105
- spanbuf.members (31.9.3.4) 2107
- spanbuf.virtuals (31.9.3.5) 2107
- spanstream (31.9.6) 2110
- spanstream.cons (31.9.6.2) 2111
- spanstream.general (31.9.6.1) 2110
- spanstream.members (31.9.6.4) 2111
- spanstream.swap (31.9.6.3) 2111
- spanstream.syn (31.9.2) 2104
- special (11.4.4) 323
- special.mem.concepts (26.11.2) 1580
- specialized.addressof (20.2.11) 693
- specialized.algorithms (26.11) 1579
- specialized.algorithms.general (26.11.1) 1579
- specialized.construct (26.11.8) 1585
- specialized.destroy (26.11.9) 1585
- sstream.syn (31.8.1) 2088
- stable.sort (26.8.2.2) 1533
- stack (23.6.6) 1098
- stack.cons (23.6.6.3) 1100
- stack.cons.alloc (23.6.6.4) 1100
- stack.defn (23.6.6.2) 1099
- stack.general (23.6.6.1) 1099
- stack.mod (23.6.6.5) 1100
- stack.ops (23.6.6.6) 1101
- stack.special (23.6.6.7) 1101
- stack.syn (23.6.5) 1098
- stacktrace (19.6) 664
- stacktrace.basic (19.6.4) 666
- stacktrace.basic.cmp (19.6.4.4) 669
- stacktrace.basic.cons (19.6.4.2) 667
- stacktrace.basic.hash (19.6.6) 670
- stacktrace.basic.mod (19.6.4.5) 669
- stacktrace.basic.nonmem (19.6.4.6) 669
- stacktrace.basic.obs (19.6.4.3) 668
- stacktrace.basic.overview (19.6.4.1) 666
- stacktrace.entry (19.6.3) 665
- stacktrace.entry.cmp (19.6.3.5) 666
- stacktrace.entry.cons (19.6.3.2) 665
- stacktrace.entry.obs (19.6.3.3) 665
- stacktrace.entry.overview (19.6.3.1) 665
- stacktrace.entry.query (19.6.3.4) 666
- stacktrace.format (19.6.5) 670
- stacktrace.general (19.6.1) 664
- stacktrace.syn (19.6.2) 664
- std.exceptions (19.2) 650
- std.exceptions.general (19.2.1) 650
- std.ios.manip (31.5.5) 2051
- std.iterator.tags (24.4.2) 1221
- std.manip (31.7.7) 2083
- std.modules (16.4.2.4) 558
- stdalign.h.syn (17.15.4) 635
- stdatomic.h.syn (32.5.12) 2242
- stdbit.h.syn (22.12) 957
- stdbool.h.syn (17.15.5) 635
- stdckdint.h.syn (29.11.1) 1940
- stdexcept.syn (19.2.2) 650
- stdfloat.syn (17.4.2) 598
- stmt (Clause 8) 192
- stmt.ambig (8.11) 204
- stmt.block (8.4) 193
- stmt.break (8.8.2) 201
- stmt.cont (8.8.3) 201
- stmt.contract.assert (8.9) 202
- stmt.dcl (8.10) 203
- stmt.do (8.6.3) 197
- stmt.expand (8.7) 198

- stmt.expr (8.3) 193
- stmt.for (8.6.4) 197
- stmt.goto (8.8.6) 202
- stmt.if (8.5.2) 194
- stmt.iter (8.6) 196
- stmt.iter.general (8.6.1) 196
- stmt.jump (8.8) 201
- stmt.jump.general (8.8.1) 201
- stmt.label (8.2) 193
- stmt.pre (8.1) 192
- stmt.ranged (8.6.5) 198
- stmt.return (8.8.4) 201
- stmt.return.coroutine (8.8.5) 202
- stmt.select (8.5) 194
- stmt.select.general (8.5.1) 194
- stmt.switch (8.5.3) 195
- stmt.while (8.6.2) 197
- stopcallback (32.3.6) 2188
- stopcallback.cons (32.3.6.2) 2189
- stopcallback.general (32.3.6.1) 2189
- stopcallback.inplace (32.3.10) 2191
- stopcallback.inplace.cons (32.3.10.2) 2192
- stopcallback.inplace.general (32.3.10.1) 2191
- stopsource (32.3.5) 2188
- stopsource.cons (32.3.5.2) 2188
- stopsource.general (32.3.5.1) 2188
- stopsource.inplace (32.3.9) 2190
- stopsource.inplace.cons (32.3.9.2) 2191
- stopsource.inplace.general (32.3.9.1) 2190
- stopsource.inplace.mem (32.3.9.3) 2191
- stopsource.mem (32.3.5.3) 2188
- stoptoken (32.3.4) 2187
- stoptoken.concepts (32.3.3) 2185
- stoptoken.general (32.3.4.1) 2187
- stoptoken.inplace (32.3.8) 2190
- stoptoken.inplace.general (32.3.8.1) 2190
- stoptoken.inplace.mem (32.3.8.2) 2190
- stoptoken.mem (32.3.4.2) 2187
- stoptoken.never (32.3.7) 2189
- stream.buffer (31.6) 2053
- stream.iterators (24.6) 1250
- stream.iterators.general (24.6.1) 1250
- stream.types (31.2.2) 2033
- streambuf (31.6.3) 2054
- streambuf.assign (31.6.3.4.1) 2057
- streambuf.buffer (31.6.3.3.2) 2056
- streambuf.cons (31.6.3.2) 2055
- streambuf.general (31.6.3.1) 2054
- streambuf.get.area (31.6.3.4.2) 2058
- streambuf.locales (31.6.3.3.1) 2056
- streambuf.members (31.6.3.3) 2056
- streambuf.protected (31.6.3.4) 2057
- streambuf.pub.get (31.6.3.3.3) 2056
- streambuf.pub.pback (31.6.3.3.4) 2057
- streambuf.pub.put (31.6.3.3.5) 2057
- streambuf.put.area (31.6.3.4.3) 2058
- streambuf.reqts (31.6.2) 2053
- streambuf.syn (31.6.1) 2053
- streambuf.virt.buffer (31.6.3.5.2) 2058
- streambuf.virt.get (31.6.3.5.3) 2059
- streambuf.virt.locales (31.6.3.5.1) 2058
- streambuf.virt.pback (31.6.3.5.4) 2060
- streambuf.virt.put (31.6.3.5.5) 2060
- streambuf.virtuals (31.6.3.5) 2058
- string.access (27.4.3.6) 1617
- string.accessors (27.4.3.8.1) 1624
- string.append (27.4.3.7.2) 1618
- string.assign (27.4.3.7.3) 1619
- string.capacity (27.4.3.5) 1615
- string.classes (27.4) 1603
- string.classes.general (27.4.1) 1604
- string.cmp (27.4.4.2) 1628
- string.compare (27.4.3.8.4) 1625
- string.cons (27.4.3.3) 1613
- string.contains (27.4.3.8.7) 1626
- string.conversions (27.4.5) 1630
- string.copy (27.4.3.7.7) 1623
- string.ends.with (27.4.3.8.6) 1626
- string.erase (27.4.3.7.5) 1621
- string.erasure (27.4.4.5) 1630
- string.find (27.4.3.8.2) 1624
- string.insert (27.4.3.7.4) 1620
- string.io (27.4.4.4) 1629
- string.iterators (27.4.3.4) 1615
- string.modifiers (27.4.3.7) 1617
- string.nonmembers (27.4.4) 1626
- string.op.append (27.4.3.7.1) 1617
- string.op.plus (27.4.4.1) 1626
- string.ops (27.4.3.8) 1624
- string.replace (27.4.3.7.6) 1622
- string.require (27.4.3.2) 1612
- string.special (27.4.4.3) 1629
- string.starts.with (27.4.3.8.5) 1626
- string.streams (31.8) 2088
- string.substr (27.4.3.8.3) 1625
- string.swap (27.4.3.7.8) 1624
- string.syn (27.4.2) 1604
- string.view (27.3) 1594
- string.view.access (27.3.3.6) 1599
- string.view.capacity (27.3.3.5) 1599
- string.view.comparison (27.3.4) 1602
- string.view.cons (27.3.3.2) 1597
- string.view.deduct (27.3.3.3) 1598
- string.view.find (27.3.3.9) 1601
- string.view.general (27.3.1) 1594
- string.view.hash (27.3.6) 1603
- string.view.io (27.3.5) 1603
- string.view.iterators (27.3.3.4) 1598
- string.view.literals (27.3.7) 1603
- string.view.modifiers (27.3.3.7) 1600
- string.view.ops (27.3.3.8) 1600
- string.view.synop (27.3.2) 1594
- string.view.template (27.3.3) 1595
- string.view.template.general (27.3.3.1) 1595
- stringbuf (31.8.2) 2089
- stringbuf.assign (31.8.2.3) 2092
- stringbuf.cons (31.8.2.2) 2091
- stringbuf.general (31.8.2.1) 2089

- stringbuf.members (31.8.2.4) 2092
- stringbuf.virtuals (31.8.2.5) 2094
- strings (Clause 27) 1589
- strings.general (27.1) 1589
- stringstream (31.8.5) 2101
- stringstream.cons (31.8.5.2) 2103
- stringstream.general (31.8.5.1) 2101
- stringstream.members (31.8.5.4) 2104
- stringstream.swap (31.8.5.3) 2103
- structure (16.3.2) 548
- structure.elements (16.3.2.1) 548
- structure.requirements (16.3.2.3) 549
- structure.see.also (16.3.2.5) 551
- structure.specifications (16.3.2.4) 549
- structure.summary (16.3.2.2) 548
- support (Clause 17) 579
- support.arith.types (17.4) 597
- support.c.headers (17.15) 634
- support.c.headers.general (17.15.1) 634
- support.c.headers.other (17.15.7) 635
- support.contract (17.10) 616
- support.contract.enum (17.10.2) 616
- support.contract.invoke (17.10.4) 618
- support.contract.violation (17.10.3) 617
- support.coroutine (17.13) 627
- support.coroutine.general (17.13.1) 628
- support.dynamic (17.6) 600
- support.dynamic.general (17.6.1) 600
- support.exception (17.9) 612
- support.exception.general (17.9.1) 612
- support.general (17.1) 579
- support.initlist (17.11) 618
- support.initlist.access (17.11.4) 618
- support.initlist.cons (17.11.3) 618
- support.initlist.general (17.11.1) 618
- support.limits (17.3) 583
- support.limits.general (17.3.1) 583
- support.rtti (17.7) 607
- support.rtti.general (17.7.1) 607
- support.runtime (17.14) 632
- support.runtime.general (17.14.1) 632
- support.signal (17.14.5) 634
- support.srcloc (17.8) 609
- support.srcloc.class (17.8.2) 610
- support.srcloc.class.general (17.8.2.1) 610
- support.srcloc.cons (17.8.2.2) 610
- support.srcloc.obs (17.8.2.3) 611
- support.start.term (17.5) 599
- support.types (17.2) 579
- support.types.byteops (17.2.5) 582
- support.types.layout (17.2.4) 582
- support.types.nullptr (17.2.3) 581
- swappable.requirements (16.4.4.3) 562
- syncstream (31.11) 2124
- syncstream.osyncstream (31.11.3) 2127
- syncstream.osyncstream.cons (31.11.3.2) 2128
- syncstream.osyncstream.members (31.11.3.3) 2128
- syncstream.osyncstream.overview (31.11.3.1) 2127
- syncstream.syn (31.11.1) 2124
- syncstream.syncbuf (31.11.2) 2125
- syncstream.syncbuf.assign (31.11.2.3) 2126
- syncstream.syncbuf.cons (31.11.2.2) 2126
- syncstream.syncbuf.members (31.11.2.4) 2126
- syncstream.syncbuf.overview (31.11.2.1) 2125
- syncstream.syncbuf.special (31.11.2.6) 2127
- syncstream.syncbuf.virtuals (31.11.2.5) 2127
- syntax (4.3) 12
- syserr (19.5) 655
- syserr.compare (19.5.6) 662
- syserr.errcat (19.5.3) 657
- syserr.errcat.derived (19.5.3.4) 658
- syserr.errcat.nonvirtuals (19.5.3.3) 658
- syserr.errcat.objects (19.5.3.5) 658
- syserr.errcat.overview (19.5.3.1) 657
- syserr.errcat.virtuals (19.5.3.2) 658
- syserr.errcode (19.5.4) 659
- syserr.errcode.constructors (19.5.4.2) 659
- syserr.errcode.modifiers (19.5.4.3) 660
- syserr.errcode.nonmembers (19.5.4.5) 660
- syserr.errcode.observers (19.5.4.4) 660
- syserr.errcode.overview (19.5.4.1) 659
- syserr.errcondition (19.5.5) 661
- syserr.errcondition.constructors (19.5.5.2) 662
- syserr.errcondition.modifiers (19.5.5.3) 662
- syserr.errcondition.nonmembers (19.5.5.5) 662
- syserr.errcondition.observers (19.5.5.4) 662
- syserr.errcondition.overview (19.5.5.1) 661
- syserr.fmt (19.5.4.6) 660
- syserr.general (19.5.1) 655
- syserr.hash (19.5.7) 663
- syserr.syserr (19.5.8) 663
- syserr.syserr.members (19.5.8.2) 663
- syserr.syserr.overview (19.5.8.1) 663
- system.error.syn (19.5.2) 655
- task.class (33.13.6.2) 2370
- task.members (33.13.6.3) 2371
- task.overview (33.13.6.1) 2370
- task.promise (33.13.6.5) 2373
- task.state (33.13.6.4) 2372
- temp (Clause 13) 416
- temp.alias (13.7.8) 461
- temp.arg (13.4) 426
- temp.arg.explicit (13.10.2) 492
- temp.arg.general (13.4.1) 426
- temp.arg.nontype (13.4.3) 428
- temp.arg.template (13.4.4) 430
- temp.arg.type (13.4.2) 428
- temp.class (13.7.2) 441
- temp.class.general (13.7.2.1) 442
- temp.concept (13.7.9) 462
- temp.constr (13.5) 432
- temp.constr.atomic (13.5.2.3) 433
- temp.constr.concept (13.5.2.4) 435
- temp.constr.constr (13.5.2) 432

- temp.constr.constr.general (13.5.2.1) 432
- temp.constr.decl (13.5.3) 436
- temp.constr.fold (13.5.2.5) 435
- temp.constr.general (13.5.1) 432
- temp.constr.normal (13.5.4) 437
- temp.constr.op (13.5.2.2) 433
- temp.constr.order (13.5.5) 439
- temp.decls (13.7) 441
- temp.decls.general (13.7.1) 441
- temp.deduct (13.10.3) 494
- temp.deduct.call (13.10.3.2) 499
- temp.deduct.conv (13.10.3.4) 502
- temp.deduct.decl (13.10.3.7) 512
- temp.deduct.funcaddr (13.10.3.3) 502
- temp.deduct.general (13.10.3.1) 494
- temp.deduct.guide (13.7.2.3) 443
- temp.deduct.partial (13.10.3.5) 503
- temp.deduct.type (13.10.3.6) 504
- temp.dep (13.8.3) 469
- temp.dep.candidate (13.8.4.2) 477
- temp.dep.constexpr (13.8.3.4) 474
- temp.dep.expr (13.8.3.3) 472
- temp.dep.general (13.8.3.1) 469
- temp.dep.namespace (13.8.3.6) 476
- temp.dep.res (13.8.4) 476
- temp.dep.splice (13.8.3.5) 475
- temp.dep.temp (13.8.3.7) 476
- temp.dep.type (13.8.3.2) 469
- temp.expl.spec (13.9.4) 487
- temp.explicit (13.9.3) 485
- temp.fct (13.7.7) 456
- temp.fct.general (13.7.7.1) 456
- temp.fct.spec (13.10) 492
- temp.fct.spec.general (13.10.1) 492
- temp.friend (13.7.5) 450
- temp.func.order (13.7.7.3) 458
- temp.inst (13.9.2) 481
- temp.local (13.8.2) 467
- temp.mem (13.7.3) 445
- temp.mem.class (13.7.2.4) 444
- temp.mem.enum (13.7.2.6) 444
- temp.mem.func (13.7.2.2) 442
- temp.names (13.3) 422
- temp.over (13.10.4) 512
- temp.over.link (13.7.7.2) 456
- temp.param (13.2) 418
- temp.point (13.8.4.1) 476
- temp.pre (13.1) 416
- temp.res (13.8) 463
- temp.res.general (13.8.1) 463
- temp.spec (13.9) 480
- temp.spec.general (13.9.1) 480
- temp.spec.partial (13.7.6) 451
- temp.spec.partial.general (13.7.6.1) 451
- temp.spec.partial.match (13.7.6.2) 453
- temp.spec.partial.member (13.7.6.4) 455
- temp.spec.partial.order (13.7.6.3) 454
- temp.static (13.7.2.5) 444
- temp.type (13.6) 440
- temp.variadic (13.7.4) 446
- template.bitset (22.9.2) 910
- template.bitset.general (22.9.2.1) 910
- template.gslice.array (29.6.7) 1820
- template.gslice.array.overview (29.6.7.1) 1820
- template.indirect.array (29.6.9) 1822
- template.indirect.array.overview (29.6.9.1) 1822
- template.mask.array (29.6.8) 1821
- template.mask.array.overview (29.6.8.1) 1821
- template.slice.array (29.6.5) 1818
- template.slice.array.overview (29.6.5.1) 1818
- template.valarray (29.6.2) 1808
- template.valarray.overview (29.6.2.1) 1808
- terminate (17.9.5.4) 614
- terminate.handler (17.9.5.1) 613
- text (Clause 28) 1634
- text.c.strings (28.7) 1746
- text.encoding (28.4) 1676
- text.encoding.aliases (28.4.2.5) 1680
- text.encoding.class (28.4.2) 1677
- text.encoding.cmp (28.4.2.4) 1680
- text.encoding.general (28.4.2.2) 1678
- text.encoding.hash (28.4.2.7) 1684
- text.encoding.id (28.4.2.6) 1680
- text.encoding.members (28.4.2.3) 1678
- text.encoding.overview (28.4.2.1) 1677
- text.encoding.syn (28.4.1) 1677
- text.general (28.1) 1634
- tgmash.h.syn (17.15.6) 635
- thread (Clause 32) 2180
- thread.attributes (32.4.3.2) 2193
- thread.attributes.general (32.4.3.2.1) 2193
- thread.attributes.hint (32.4.3.2.2) 2193
- thread.attributes.size (32.4.3.2.3) 2194
- thread.barrier (32.9.3) 2272
- thread.barrier.class (32.9.3.3) 2272
- thread.barrier.general (32.9.3.1) 2272
- thread.condition (32.7) 2262
- thread.condition.condvar (32.7.4) 2263
- thread.condition.condvarany (32.7.5) 2266
- thread.condition.condvarany.general (32.7.5.1) 2266
- thread.condition.general (32.7.1) 2262
- thread.condition.nonmember (32.7.3) 2262
- thread.condvarany.intwait (32.7.5.3) 2268
- thread.condvarany.wait (32.7.5.2) 2267
- thread.coord (32.9) 2271
- thread.coord.general (32.9.1) 2271
- thread.general (32.1) 2180
- thread.jthread.class (32.4.4) 2197
- thread.jthread.class.general (32.4.4.1) 2197
- thread.jthread.cons (32.4.4.2) 2198
- thread.jthread.mem (32.4.4.3) 2199
- thread.jthread.special (32.4.4.5) 2200
- thread.jthread.static (32.4.4.6) 2200
- thread.jthread.stop (32.4.4.4) 2200
- thread.latch (32.9.2) 2271
- thread.latch.class (32.9.2.3) 2271
- thread.latch.general (32.9.2.1) 2271

- thread.lock (32.6.5) 2252
- thread.lock.algorithm (32.6.6) 2260
- thread.lock.general (32.6.5.1) 2252
- thread.lock.guard (32.6.5.2) 2252
- thread.lock.scoped (32.6.5.3) 2253
- thread.lock.shared (32.6.5.5) 2257
- thread.lock.shared.cons (32.6.5.5.2) 2258
- thread.lock.shared.general (32.6.5.5.1) 2257
- thread.lock.shared.locking (32.6.5.5.3) 2259
- thread.lock.shared.mod (32.6.5.5.4) 2260
- thread.lock.shared.obs (32.6.5.5.5) 2260
- thread.lock.unique (32.6.5.4) 2254
- thread.lock.unique.cons (32.6.5.4.2) 2255
- thread.lock.unique.general (32.6.5.4.1) 2254
- thread.lock.unique.locking (32.6.5.4.3) 2255
- thread.lock.unique.mod (32.6.5.4.4) 2257
- thread.lock.unique.obs (32.6.5.4.5) 2257
- thread.mutex (32.6) 2244
- thread.mutex.class (32.6.4.2.2) 2246
- thread.mutex.general (32.6.1) 2244
- thread.mutex.recursive (32.6.4.2.3) 2246
- thread.mutex.requirements (32.6.4) 2245
- thread.mutex.requirements.general (32.6.4.1) 2245
- thread.mutex.requirements.mutex (32.6.4.2) 2245
- thread.mutex.requirements.mutex.general (32.6.4.2.1) 2245
- thread.once (32.6.7) 2261
- thread.once.callonce (32.6.7.2) 2261
- thread.once.onceflag (32.6.7.1) 2261
- thread.req (32.2) 2180
- thread.req.exception (32.2.2) 2180
- thread.req.lockable (32.2.5) 2181
- thread.req.lockable.basic (32.2.5.2) 2182
- thread.req.lockable.general (32.2.5.1) 2181
- thread.req.lockable.req (32.2.5.3) 2182
- thread.req.lockable.shared (32.2.5.5) 2183
- thread.req.lockable.shared.timed (32.2.5.6) 2183
- thread.req.lockable.timed (32.2.5.4) 2182
- thread.req.native (32.2.3) 2180
- thread.req.paramname (32.2.1) 2180
- thread.req.timing (32.2.4) 2180
- thread.sema (32.8) 2269
- thread.sema.cnt (32.8.3) 2269
- thread.sema.general (32.8.1) 2269
- thread.sharedmutex.class (32.6.4.4.2) 2250
- thread.sharedmutex.requirements (32.6.4.4) 2249
- thread.sharedmutex.requirements.general (32.6.4.4.1) 2249
- thread.sharedtimedmutex.class (32.6.4.5.2) 2251
- thread.sharedtimedmutex.requirements (32.6.4.5) 2251
- thread.sharedtimedmutex.requirements.general (32.6.4.5.1) 2251
- thread.stoptoken (32.3) 2183
- thread.stoptoken.intro (32.3.1) 2183
- thread.stoptoken.syn (32.3.2) 2184
- thread.syn (32.4.2) 2192
- thread.thread.algorithm (32.4.3.9) 2197
- thread.thread.assign (32.4.3.6) 2196
- thread.thread.class (32.4.3) 2192
- thread.thread.class.general (32.4.3.1) 2192
- thread.thread.constr (32.4.3.4) 2195
- thread.thread.destr (32.4.3.5) 2196
- thread.thread.id (32.4.3.3) 2194
- thread.thread.member (32.4.3.7) 2196
- thread.thread.static (32.4.3.8) 2197
- thread.thread.this (32.4.5) 2201
- thread.threads (32.4) 2192
- thread.threads.general (32.4.1) 2192
- thread.timedmutex.class (32.6.4.3.2) 2248
- thread.timedmutex.recursive (32.6.4.3.3) 2248
- thread.timedmutex.requirements (32.6.4.3) 2247
- thread.timedmutex.requirements.general (32.6.4.3.1) 2247
- time (Clause 30) 1941
- time.12 (30.10) 2009
- time.cal (30.8) 1978
- time.cal.day (30.8.3) 1978
- time.cal.day.members (30.8.3.2) 1979
- time.cal.day.nonmembers (30.8.3.3) 1979
- time.cal.day.overview (30.8.3.1) 1978
- time.cal.general (30.8.1) 1978
- time.cal.last (30.8.2) 1978
- time.cal.md (30.8.9) 1988
- time.cal.md.members (30.8.9.2) 1989
- time.cal.md.nonmembers (30.8.9.3) 1989
- time.cal.md.overview (30.8.9.1) 1988
- time.cal.mdlast (30.8.10) 1989
- time.cal.month (30.8.4) 1980
- time.cal.month.members (30.8.4.2) 1981
- time.cal.month.nonmembers (30.8.4.3) 1981
- time.cal.month.overview (30.8.4.1) 1980
- time.cal.mwd (30.8.11) 1990
- time.cal.mwd.members (30.8.11.2) 1991
- time.cal.mwd.nonmembers (30.8.11.3) 1991
- time.cal.mwd.overview (30.8.11.1) 1990
- time.cal.mwdlast (30.8.12) 1991
- time.cal.mwdlast.members (30.8.12.2) 1991
- time.cal.mwdlast.nonmembers (30.8.12.3) 1992
- time.cal.mwdlast.overview (30.8.12.1) 1991
- time.cal.operators (30.8.18) 2004
- time.cal.wd (30.8.6) 1984
- time.cal.wd.members (30.8.6.2) 1985
- time.cal.wd.nonmembers (30.8.6.3) 1986
- time.cal.wd.overview (30.8.6.1) 1984
- time.cal.wdidx (30.8.7) 1987
- time.cal.wdidx.members (30.8.7.2) 1987
- time.cal.wdidx.nonmembers (30.8.7.3) 1987
- time.cal.wdidx.overview (30.8.7.1) 1987
- time.cal.wdlast (30.8.8) 1988
- time.cal.wdlast.members (30.8.8.2) 1988
- time.cal.wdlast.nonmembers (30.8.8.3) 1988
- time.cal.wdlast.overview (30.8.8.1) 1988
- time.cal.year (30.8.5) 1982
- time.cal.year.members (30.8.5.2) 1983
- time.cal.year.nonmembers (30.8.5.3) 1984
- time.cal.year.overview (30.8.5.1) 1982

time.cal.ym (30.8.13) 1992
 time.cal.ym.members (30.8.13.2) 1992
 time.cal.ym.nonmembers (30.8.13.3) 1993
 time.cal.ym.overview (30.8.13.1) 1992
 time.cal.ymd (30.8.14) 1994
 time.cal.ymd.members (30.8.14.2) 1995
 time.cal.ymd.nonmembers (30.8.14.3) 1996
 time.cal.ymd.overview (30.8.14.1) 1994
 time.cal.ymdlast (30.8.15) 1997
 time.cal.ymdlast.members (30.8.15.2) 1998
 time.cal.ymdlast.nonmembers (30.8.15.3) 1998
 time.cal.ymdlast.overview (30.8.15.1) 1997
 time.cal.ymwd (30.8.16) 1999
 time.cal.ymwd.members (30.8.16.2) 2000
 time.cal.ymwd.nonmembers (30.8.16.3) 2001
 time.cal.ymwd.overview (30.8.16.1) 1999
 time.cal.ymwdlast (30.8.17) 2002
 time.cal.ymwdlast.members (30.8.17.2) 2002
 time.cal.ymwdlast.nonmembers (30.8.17.3) 2003
 time.cal.ymwdlast.overview (30.8.17.1) 2002
 time.clock (30.7) 1967
 time.clock.cast (30.7.10) 1975
 time.clock.cast.fn (30.7.10.6) 1978
 time.clock.cast.id (30.7.10.2) 1976
 time.clock.cast.sys (30.7.10.4) 1977
 time.clock.cast.sys.utc (30.7.10.3) 1976
 time.clock.cast.utc (30.7.10.5) 1977
 time.clock.conv (30.7.10.1) 1975
 time.clock.file (30.7.6) 1973
 time.clock.file.members (30.7.6.2) 1974
 time.clock.file.nonmembers (30.7.6.3) 1974
 time.clock.file.overview (30.7.6.1) 1973
 time.clock.general (30.7.1) 1967
 time.clock.gps (30.7.5) 1972
 time.clock.gps.members (30.7.5.2) 1972
 time.clock.gps.nonmembers (30.7.5.3) 1973
 time.clock.gps.overview (30.7.5.1) 1972
 time.clock.hires (30.7.8) 1975
 time.clock.local (30.7.9) 1975
 time.clock.req (30.3) 1955
 time.clock.steady (30.7.7) 1974
 time.clock.system (30.7.2) 1967
 time.clock.system.members (30.7.2.2) 1968
 time.clock.system.nonmembers (30.7.2.3) 1968
 time.clock.system.overview (30.7.2.1) 1967
 time.clock.tai (30.7.4) 1970
 time.clock.tai.members (30.7.4.2) 1971
 time.clock.tai.nonmembers (30.7.4.3) 1971
 time.clock.tai.overview (30.7.4.1) 1971
 time.clock.utc (30.7.3) 1968
 time.clock.utc.members (30.7.3.2) 1969
 time.clock.utc.nonmembers (30.7.3.3) 1970
 time.clock.utc.overview (30.7.3.1) 1969
 time.duration (30.5) 1957
 time.duration.alg (30.5.10) 1963
 time.duration.arithmetic (30.5.4) 1959
 time.duration.cast (30.5.8) 1961
 time.duration.comparisons (30.5.7) 1961
 time.duration.cons (30.5.2) 1958
 time.duration.general (30.5.1) 1957
 time.duration.io (30.5.11) 1963
 time.duration.literals (30.5.9) 1962
 time.duration.nonmember (30.5.6) 1960
 time.duration.observer (30.5.3) 1959
 time.duration.special (30.5.5) 1960
 time.format (30.12) 2023
 time.general (30.1) 1941
 time.hash (30.14) 2031
 time.hms (30.9) 2007
 time.hms.members (30.9.2) 2008
 time.hms.nonmembers (30.9.3) 2009
 time.hms.overview (30.9.1) 2007
 time.parse (30.13) 2027
 time.point (30.6) 1964
 time.point.arithmetic (30.6.4) 1965
 time.point.cast (30.6.8) 1967
 time.point.comparisons (30.6.7) 1966
 time.point.cons (30.6.2) 1965
 time.point.general (30.6.1) 1964
 time.point.nonmember (30.6.6) 1966
 time.point.observer (30.6.3) 1965
 time.point.special (30.6.5) 1965
 time.syn (30.2) 1941
 time.traits (30.4) 1956
 time.traits.duration.values (30.4.2) 1956
 time.traits.is.clock (30.4.4) 1957
 time.traits.is.fp (30.4.1) 1956
 time.traits.specializations (30.4.3) 1957
 time.zone (30.11) 2009
 time.zone.db (30.11.2) 2010
 time.zone.db.access (30.11.2.3) 2011
 time.zone.db.list (30.11.2.2) 2010
 time.zone.db.remote (30.11.2.4) 2011
 time.zone.db.tzdb (30.11.2.1) 2010
 time.zone.exception (30.11.3) 2012
 time.zone.exception.ambig (30.11.3.2) 2013
 time.zone.exception.nonexist (30.11.3.1) 2012
 time.zone.general (30.11.1) 2010
 time.zone.info (30.11.4) 2013
 time.zone.info.local (30.11.4.2) 2014
 time.zone.info.sys (30.11.4.1) 2013
 time.zone.leap (30.11.8) 2020
 time.zone.leap.members (30.11.8.2) 2021
 time.zone.leap.nonmembers (30.11.8.3) 2021
 time.zone.leap.overview (30.11.8.1) 2020
 time.zone.link (30.11.9) 2022
 time.zone.link.members (30.11.9.2) 2022
 time.zone.link.nonmembers (30.11.9.3) 2022
 time.zone.link.overview (30.11.9.1) 2022
 time.zone.members (30.11.5.2) 2015
 time.zone.nonmembers (30.11.5.3) 2016
 time.zone.overview (30.11.5.1) 2015
 time.zone.timezone (30.11.5) 2015
 time.zone.zonedtime (30.11.7) 2016
 time.zone.zonedtime.ctor (30.11.7.2) 2018
 time.zone.zonedtime.members (30.11.7.3) 2019
 time.zone.zonedtime.nonmembers (30.11.7.4) 2020

- time.zone.zonedtime.overview (30.11.7.1) 2016
- time.zone.zonedtraits (30.11.6) 2016
- transform.exclusive.scan (26.10.10) 1575
- transform.inclusive.scan (26.10.11) 1576
- transform.reduce (26.10.6) 1571
- tuple (22.4) 837
- tuple.apply (22.4.6) 848
- tuple.assign (22.4.4.3) 844
- tuple.cnstr (22.4.4.2) 842
- tuple.common.ref (22.4.10) 851
- tuple.creation (22.4.5) 847
- tuple.elem (22.4.8) 849
- tuple.general (22.4.1) 838
- tuple.helper (22.4.7) 849
- tuple.like (22.4.3) 839
- tuple.rel (22.4.9) 850
- tuple.special (22.4.12) 851
- tuple.swap (22.4.4.4) 847
- tuple.syn (22.4.2) 838
- tuple.traits (22.4.11) 851
- tuple.tuple (22.4.4) 839
- tuple.tuple.general (22.4.4.1) 840
- type.descriptions (16.3.3.3) 552
- type.descriptions.general (16.3.3.3.1) 552
- type.index (17.7.7) 608
- type.index.synopsis (17.7.6) 608
- type.info (17.7.3) 607
- type.traits (21.3) 751
- type.traits.general (21.3.1) 751
- typeinfo.syn (17.7.2) 607

- uaxid (Annex E) 2466
- uaxid.def (E.2) 2466
- uaxid.def.general (E.2.1) 2466
- uaxid.def.stable (E.2.2) 2466
- uaxid.eqci (E.6) 2467
- uaxid.eqn (E.5) 2466
- uaxid.filter (E.7) 2467
- uaxid.filterci (E.8) 2467
- uaxid.general (E.1) 2466
- uaxid.hashtag (E.9) 2467
- uaxid.immutable (E.3) 2466
- uaxid.pattern (E.4) 2466
- ub (Annex F) 2468
- ub.basic (F.2) 2468
- ub.class (F.6) 2488
- ub.dcl (F.5) 2485
- ub.except (F.8) 2493
- ub.expr (F.3) 2475
- ub.general (F.1) 2468
- ub.stmt (F.4) 2484
- ub.temp (F.7) 2492
- uncaught.exceptions (17.9.6) 614
- underflow.error (19.2.11) 653
- uninitialized.construct.default (26.11.3) 1581
- uninitialized.construct.value (26.11.4) 1582
- uninitialized.copy (26.11.5) 1583
- uninitialized.fill (26.11.7) 1584
- uninitialized.move (26.11.6) 1583

- unique.ptr (20.4.1) 697
- unique.ptr.create (20.4.1.5) 704
- unique.ptr.dltr (20.4.1.2) 698
- unique.ptr.dltr.dflt (20.4.1.2.2) 698
- unique.ptr.dltr.dflt1 (20.4.1.2.3) 698
- unique.ptr.dltr.general (20.4.1.2.1) 698
- unique.ptr.general (20.4.1.1) 697
- unique.ptr.io (20.4.1.7) 706
- unique.ptr.runtime (20.4.1.4) 702
- unique.ptr.runtime.asgn (20.4.1.4.3) 704
- unique.ptr.runtime.ctor (20.4.1.4.2) 703
- unique.ptr.runtime.general (20.4.1.4.1) 702
- unique.ptr.runtime.modifiers (20.4.1.4.5) 704
- unique.ptr.runtime.observers (20.4.1.4.4) 704
- unique.ptr.single (20.4.1.3) 698
- unique.ptr.single.asgn (20.4.1.3.4) 701
- unique.ptr.single.ctor (20.4.1.3.2) 699
- unique.ptr.single.dtor (20.4.1.3.3) 701
- unique.ptr.single.general (20.4.1.3.1) 698
- unique.ptr.single.modifiers (20.4.1.3.6) 702
- unique.ptr.single.observers (20.4.1.3.5) 701
- unique.ptr.special (20.4.1.6) 705
- unord (23.5) 1063
- unord.general (23.5.1) 1063
- unord.hash (22.10.19) 952
- unord.map (23.5.3) 1064
- unord.map.cnstr (23.5.3.2) 1069
- unord.map.elem (23.5.3.3) 1070
- unord.map.erasure (23.5.3.5) 1072
- unord.map.modifiers (23.5.3.4) 1070
- unord.map.overview (23.5.3.1) 1064
- unord.map.syn (23.5.2) 1063
- unord.multimap (23.5.4) 1072
- unord.multimap.cnstr (23.5.4.2) 1077
- unord.multimap.erasure (23.5.4.4) 1077
- unord.multimap.modifiers (23.5.4.3) 1077
- unord.multimap.overview (23.5.4.1) 1072
- unord.multiset (23.5.7) 1084
- unord.multiset.cnstr (23.5.7.2) 1088
- unord.multiset.erasure (23.5.7.3) 1089
- unord.multiset.overview (23.5.7.1) 1084
- unord.req (23.2.8) 984
- unord.req.except (23.2.8.2) 995
- unord.req.general (23.2.8.1) 984
- unord.set (23.5.6) 1079
- unord.set.cnstr (23.5.6.2) 1083
- unord.set.erasure (23.5.6.3) 1083
- unord.set.modifiers (23.5.6.4) 1084
- unord.set.overview (23.5.6.1) 1079
- unord.set.syn (23.5.5) 1078
- unreachable.sentinel (24.5.8) 1250
- upper.bound (26.8.4.3) 1540
- using (16.4.3) 561
- using.headers (16.4.3.2) 561
- using.linkage (16.4.3.3) 561
- using.overview (16.4.3.1) 561
- usrlit.suffix (16.4.5.3.6) 574
- util.sharedptr (20.4.2) 706
- util.smartptr.atomic (32.5.8.7) 2234

- util.smartptr.atomic.general (32.5.8.7.1) 2234
- util.smartptr.atomic.shared (32.5.8.7.2) 2235
- util.smartptr.atomic.weak (32.5.8.7.3) 2237
- util.smartptr.enab (20.4.2.7) 721
- util.smartptr.getdeleter (20.4.2.2.11) 716
- util.smartptr.hash (20.4.3) 721
- util.smartptr.owner.equal (20.4.2.6) 720
- util.smartptr.owner.hash (20.4.2.5) 720
- util.smartptr.ownerless (20.4.2.4) 719
- util.smartptr.shared (20.4.2.2) 707
- util.smartptr.shared.assign (20.4.2.2.4) 710
- util.smartptr.shared.cast (20.4.2.2.10) 715
- util.smartptr.shared.cmp (20.4.2.2.8) 715
- util.smartptr.shared.const (20.4.2.2.2) 708
- util.smartptr.shared.create (20.4.2.2.7) 712
- util.smartptr.shared.dest (20.4.2.2.3) 710
- util.smartptr.shared.general (20.4.2.2.1) 707
- util.smartptr.shared.io (20.4.2.2.12) 717
- util.smartptr.shared.mod (20.4.2.2.5) 711
- util.smartptr.shared.obs (20.4.2.2.6) 711
- util.smartptr.shared.spec (20.4.2.2.9) 715
- util.smartptr.weak (20.4.2.3) 717
- util.smartptr.weak.assign (20.4.2.3.4) 718
- util.smartptr.weak.bad (20.4.2.1) 706
- util.smartptr.weak.const (20.4.2.3.2) 718
- util.smartptr.weak.dest (20.4.2.3.3) 718
- util.smartptr.weak.general (20.4.2.3.1) 717
- util.smartptr.weak.mod (20.4.2.3.5) 718
- util.smartptr.weak.obs (20.4.2.3.6) 718
- util.smartptr.weak.spec (20.4.2.3.7) 719
- utilities (Clause 22) 821
- utilities.general (22.1) 821
- utility (22.2) 821
- utility.arg.requirements (16.4.4.2) 561
- utility.as.const (22.2.5) 826
- utility.exchange (22.2.3) 825
- utility.intcmp (22.2.7) 827
- utility.requirements (16.4.4) 561
- utility.requirements.general (16.4.4.1) 561
- utility.swap (22.2.2) 824
- utility.syn (22.2.1) 821
- utility.undefined (22.2.9) 828
- utility.underlying (22.2.8) 827
- valarray.access (29.6.2.4) 1811
- valarray.assign (29.6.2.3) 1810
- valarray.binary (29.6.3.1) 1815
- valarray.cassign (29.6.2.7) 1813
- valarray.comparison (29.6.3.2) 1816
- valarray.cons (29.6.2.2) 1809
- valarray.members (29.6.2.9) 1814
- valarray.nonmembers (29.6.3) 1815
- valarray.range (29.6.2.8) 1813
- valarray.special (29.6.3.4) 1817
- valarray.sub (29.6.2.5) 1811
- valarray.syn (29.6.1) 1805
- valarray.transcend (29.6.3.3) 1817
- valarray.unary (29.6.2.6) 1812
- value.error.codes (16.4.6.16) 578
- variant (22.6) 871
- variant.assign (22.6.3.4) 876
- variant.bad.access (22.6.11) 883
- variant.ctor (22.6.3.2) 874
- variant.dtor (22.6.3.3) 876
- variant.general (22.6.1) 871
- variant.get (22.6.5) 879
- variant.hash (22.6.12) 883
- variant.helper (22.6.4) 879
- variant.mod (22.6.3.5) 878
- variant.monostate (22.6.8) 882
- variant.monostate.relops (22.6.9) 883
- variant.relops (22.6.6) 881
- variant.specalg (22.6.10) 883
- variant.status (22.6.3.6) 878
- variant.swap (22.6.3.7) 879
- variant.syn (22.6.2) 871
- variant.variant (22.6.3) 873
- variant.variant.general (22.6.3.1) 873
- variant.visit (22.6.7) 882
- vector (23.3.13) 1028
- vector.bool (23.3.14) 1033
- vector.bool.fmt (23.3.14.2) 1035
- vector.bool.pspc (23.3.14.1) 1033
- vector.capacity (23.3.13.3) 1030
- vector.cons (23.3.13.2) 1030
- vector.data (23.3.13.4) 1031
- vector.erasure (23.3.13.6) 1032
- vector.modifiers (23.3.13.5) 1032
- vector.overview (23.3.13.1) 1028
- vector.syn (23.3.12) 1027
- version.syn (17.3.2) 583
- view.interface (25.5.3) 1277
- view.interface.general (25.5.3.1) 1277
- view.interface.members (25.5.3.2) 1278
- views (23.7) 1138
- views.contiguous (23.7.2) 1138
- views.general (23.7.1) 1138
- views.multidim (23.7.3) 1144
- views.span (23.7.2.2) 1138
- wide.stream.objects (31.4.4) 2037
- zombie.names (16.4.5.3.2) 572

Cross-references from ISO C++ 2017

All clause and subclause labels from ISO C++ 2017 (ISO/IEC 14882:2017, *Programming Language — C++*) are present in this document, with the exceptions described below.

[alg.all_of](#) *see* [alg.all.of](#)
[alg.any_of](#) *see* [alg.any.of](#)
[alg.is_permutation](#) *see* [alg.is.permutation](#)
[alg.none_of](#) *see* [alg.none.of](#)
[any.bad_any_cast](#) *see* [any.bad.any.cast](#)
[array.data](#) *see* [array.members](#)
[array.fill](#) *see* [array.members](#)
[array.size](#) *see* [array.members](#)
[array.swap](#) *see* [array.members](#)

[back.insert.iter.cons](#) *see* [back.insert.iter.ops](#)
[back.insert.iter.op*](#) *see* [back.insert.iter.ops](#)
[back.insert.iter.op++](#) *see* [back.insert.iter.ops](#)
[back.insert.iter.op=](#) *see* [back.insert.iter.ops](#)
[basic.funscope](#) *see* [stmt.label](#)
[basic.lookup.classref](#) *see* [basic.lookup.qual](#)
[basic.scope.declarative](#) *see* [basic.scope.scope](#)
[basic.scope.hiding](#) *see* [basic.lookup](#)
[basic.scope.proto](#) *see* [basic.scope.param](#)
[basic.stc.dynamic.safety](#) *removed*
[basic.stc.inherit](#) *see* [basic.stc.general](#)

[char.traits.specializations.char16_t](#) *see* [char.traits.specializations.char16.t](#)
[char.traits.specializations.char32_t](#) *see* [char.traits.specializations.char32.t](#)
[class.copy](#) *see* [class.mem](#)
[class.mfct.non-static](#) *see* [class.mfct.non.static](#)
[class.mfct.non-static.general](#) *see* [class.mfct.non.static](#)
[class.nested.type](#) *see* [diff.basic](#)
[class.this](#) *see* [expr.prim.this](#)
[comparisons.equal_to](#) *see* [comparisons.equal.to](#)
[comparisons.greater_equal](#) *see* [comparisons.greater.equal](#)
[comparisons.less_equal](#) *see* [comparisons.less.equal](#)

[comparisons.not_equal_to](#) *see* [comparisons.not.equal.to](#)
[complex.special](#) *see* [complex.members](#)
[condition_variable.syn](#) *see* [condition.variable.syn](#)
[container.gen.reqmts](#) *see* [container.requirements.general](#)
[conversions](#) *see* [locale.convenience](#)
[cst dint](#) *see* [support.arith.types](#)
[cst dint.general](#) *removed*

[dcl.attr.depend](#) *removed*
[dcl.dcl](#) *see* [dcl](#)
[default.sentinel](#) *see* [default.sentinel](#)
[definitions](#) *see* [intro.defs](#)

[defs.arbitrary.stream](#) *removed*
[defs.comparison](#) *removed*
[defs.default.behavior.func](#) *removed*
[defs.direct-non-list-init](#) *see* [dcl.init.list](#)
[defs.expression-equivalent](#) *see* [defs.expression.equivalent](#)
[defs.iostream.templates](#) *removed*
[defs.multibyte](#) *see* [multibyte.strings](#)
[defs.repositional.stream](#) *removed*
[denorm.style](#) *see* [depr.numeric.limits.has.denorm](#)
[depr.arith.conv.enum](#) *removed*
[depr.array.comp](#) *removed*
[depr.atomics.flag](#) *removed*
[depr.c.headers](#) *see* [support.c.headers](#)
[depr.c.headers.general](#) *see* [support.c.headers.general](#)
[depr.c.headers.other](#) *see* [support.c.headers.other](#)
[depr.ccomplex.syn](#) *removed*
[depr.codecvt.syn](#) *removed*
[depr.comma.subscript](#) *removed*
[depr.complex.h.syn](#) *see* [complex.h.syn](#)
[depr.conversions](#) *removed*
[depr.conversions.buffer](#) *removed*
[depr.conversions.general](#) *removed*
[depr.conversions.string](#) *removed*
[depr.cpp.headers](#) *removed*
[depr.cstdalign.syn](#) *removed*
[depr.cstdbool.syn](#) *removed*
[depr.ctgmath.syn](#) *removed*
[depr.default allocator](#) *removed*
[depr.except.spec](#) *removed*
[depr.func.adaptor.binding](#) *removed*
[depr.func.adaptor.typedefs](#) *removed*
[depr.iso646.h.syn](#) *see* [iso646.h.syn](#)
[depr.istrstream](#) *removed*
[depr.istrstream.cons](#) *removed*
[depr.istrstream.general](#) *removed*
[depr.istrstream.members](#) *removed*
[depr.iterator.basic](#) *see* [depr.iterator](#)
[depr.iterator.primitives](#) *see* [depr.iterator](#)
[depr.locale.stdcv](#) *removed*
[depr.locale.stdcv.general](#) *removed*
[depr.locale.stdcv.req](#) *removed*
[depr.mem.poly.allocator.mem](#) *see* [mem.poly.allocator.mem](#)
[depr.negators](#) *removed*
[depr.ostrstream](#) *removed*
[depr.ostrstream.cons](#) *removed*
[depr.ostrstream.general](#) *removed*
[depr.ostrstream.members](#) *removed*

depr.res.on.required *removed*
 depr.static_constexpr *see* [depr.static.constexpr](#)
 depr.stdalign.h.syn *see* [stdalign.h.syn](#)
 depr.stdbool.h.syn *see* [stdbool.h.syn](#)
 depr.storage.iterator *removed*
 depr.str.strstreams *removed*
 depr.string.capacity *removed*
 depr.strstream *removed*
 depr.strstream.cons *removed*
 depr.strstream.dest *removed*
 depr.strstream.general *removed*
 depr.strstream.oper *removed*
 depr.strstream.syn *removed*
 depr.strstreambuf *removed*
 depr.strstreambuf.cons *removed*
 depr.strstreambuf.general *removed*
 depr.strstreambuf.members *removed*
 depr.strstreambuf.virtuals *removed*
 depr.temporary.buffer *removed*
 depr.tgmath.h.syn *see* [tgmath.h.syn](#)
 depr.uncaught *removed*
 depr.util.smartptr.shared.atomic *removed*
 depr.util.smartptr.shared.obs *removed*
 depr.weak.result_type *removed*
 deque.special *removed*
 diff.conv *see* [diff.expr](#)
 diff.cpp03.conv *see* [diff.cpp03.expr](#)
 diff.cpp03.dcl.decl *see* [diff.cpp03.dcl.dcl](#)
 diff.cpp03.special *see* [diff.cpp03.class](#)
 diff.cpp11.dcl.decl *see* [diff.cpp11.dcl.dcl](#)
 diff.cpp14.decl *see* [diff.cpp14.dcl.dcl](#)
 diff.cpp14.special *see* [diff.cpp14.class](#)
 diff.decl *see* [diff.dcl](#)
 diff.header.assert.h *removed*
 diff.header.stdalign.h *removed*
 diff.header.stdbool.h *removed*
 diff.special *see* [diff.class](#)

 except.uncaught *see* [except.throw](#)
 expos.only.func *see* [expos.only.entity](#)
 expos.only.types *removed*
 expr.ass *see* [expr.assign](#)
 expr.pseudo *see* [expr.prim.id.dtor](#)

 facets.examples *removed*
 forward_list.syn *see* [forward.list.syn](#)
 forwardlist *see* [forward.list](#)
 forwardlist.access *see* [forward.list.access](#)
 forwardlist.cons *see* [forward.list.cons](#)
 forwardlist.iter *see* [forward.list.iter](#)
 forwardlist.modifiers *see* [forward.list.modifiers](#)
 forwardlist.ops *see* [forward.list.ops](#)
 forwardlist.overview *see* [forward.list.overview](#)
 forwardlist.spec *removed*
 fp.style *removed*
 front.insert.iter.cons *see* [front.insert.iter.ops](#)
 front.insert.iter.op* *see* [front.insert.iter.ops](#)
 front.insert.iter.op++ *see* [front.insert.iter.ops](#)
 front.insert.iter.op= *see* [front.insert.iter.ops](#)

fs.class.directory_entry *see* [fs.class.directory.entry](#)

 fs.class.directory_iterator *see*
 [fs.class.directory.iterator](#)
 fs.class.file_status *see* [fs.class.file.status](#)
 fs.class.filesystem_error *see*
 [fs.class.filesystem.error](#)
 fs.definitions *see* [fs.class.path](#), [fs.conform.os](#),
 [fs.general](#), [fs.path.fmt.cvt](#),
 [fs.path.generic](#), [fs.race.behavior](#)
 fs.enum.file_type *see* [fs.enum.file.type](#)
 fs.file_status.cons *see* [fs.file.status.cons](#)
 fs.file_status.mods *see* [fs.file.status.mods](#)
 fs.file_status.obs *see* [fs.file.status.obs](#)
 fs.filesystem_error.members *see*
 [fs.filesystem.error.members](#)
 fs.norm.ref *removed*
 fs.op.copy_file *see* [fs.op.copy.file](#)
 fs.op.copy_symlink *see* [fs.op.copy.symlink](#)
 fs.op.create_dir_symlk *see* [fs.op.create.dir.symlk](#)
 fs.op.create_directories *see* [fs.op.create.directories](#)

 fs.op.create_directory *see* [fs.op.create.directory](#)
 fs.op.create_hard_lk *see* [fs.op.create.hard.lk](#)
 fs.op.create_symlink *see* [fs.op.create.symlink](#)
 fs.op.current_path *see* [fs.op.current.path](#)
 fs.op.file_size *see* [fs.op.file.size](#)
 fs.op.hard_lk_ct *see* [fs.op.hard.lk.ct](#)
 fs.op.is_block_file *see* [fs.op.is.block.file](#)
 fs.op.is_char_file *see* [fs.op.is.char.file](#)
 fs.op.is_directory *see* [fs.op.is.directory](#)
 fs.op.is_empty *see* [fs.op.is.empty](#)
 fs.op.is_fifo *see* [fs.op.is.fifo](#)
 fs.op.is_other *see* [fs.op.is.other](#)
 fs.op.is_regular_file *see* [fs.op.is.regular.file](#)
 fs.op.is_socket *see* [fs.op.is.socket](#)
 fs.op.is_symlink *see* [fs.op.is.symlink](#)
 fs.op.last_write_time *see* [fs.op.last.write.time](#)
 fs.op.read_symlink *see* [fs.op.read.symlink](#)
 fs.op.remove_all *see* [fs.op.remove.all](#)
 fs.op.resize_file *see* [fs.op.resize.file](#)
 fs.op.status_known *see* [fs.op.status.known](#)
 fs.op.symlink_status *see* [fs.op.symlink.status](#)
 fs.op.temp_dir_path *see* [fs.op.temp.dir.path](#)
 fs.op.weakly_canonical *see* [fs.op.weakly.canonical](#)
 fs.path.factory *see* [depr.fs.path.factory](#)
 fs.req.general *see* [fs.req](#)
 fs.req.namespace *removed*
 fstream.assign *see* [fstream.swap](#)
 func.bind.front *see* [func.bind.partial](#)
 func.not_fn *see* [func.not.fn](#)
 func.wrap.badcall.const *see* [func.wrap.badcall](#)
 futures.future_error *see* [futures.future.error](#)
 futures.shared_future *see* [futures.shared.future](#)
 futures.unique_future *see* [futures.unique.future](#)

 gram.decl *see* [gram.dcl](#)
 gram.derived *see* [gram.class](#)
 gram.special *see* [gram.class](#)

ifstream.assign *see* [ifstream.swap](#)
 initializer_list.syn *see* [initializer.list.syn](#)
 insert.iter.cons *see* [insert.iter.ops](#)
 insert.iter.op* *see* [insert.iter.ops](#)
 insert.iter.op++ *see* [insert.iter.ops](#)
 insert.iter.op= *see* [insert.iter.ops](#)
 intro.ack *removed*
 ios::failure *see* [ios.failure](#)
 ios::fmtflags *see* [ios.fmtflags](#)
 ios::Init *see* [ios.init](#)
 ios::iostate *see* [ios.iostate](#)
 ios::openmode *see* [ios.openmode](#)
 ios::seekdir *see* [ios.seekdir](#)
 istream::sentry *see* [istream.sentry](#)
 istringstream.assign *see* [istringstream.swap](#)
 iterator.container *see* [iterator.range](#)

 language.support *see* [support](#)
 list.special *removed*

 map.special *removed*
 mismatch *see* [alg.mismatch](#)
 move.iter.op.+ *see* [move.iter.nav](#)
 move.iter.op.+= *see* [move.iter.nav](#)
 move.iter.op.- *see* [move.iter.nav](#)
 move.iter.op.-= *see* [move.iter.nav](#)
 move.iter.op.const *see* [move.iter.cons](#)
 move.iter.op.decr *see* [move.iter.nav](#)
 move.iter.op.incr *see* [move.iter.nav](#)
 move.iter.op.index *see* [move.iter.elem](#)
 move.iter.op.ref *see* [move.iter.elem](#)
 move.iter.op.star *see* [move.iter.elem](#)
 move.iter.op= *see* [move.iter.cons](#)
 move.iter.ops *removed*
 multimap.special *removed*
 multiset.special *removed*

 namespace.memdef *see* [namespace.def](#)

 ofstream.assign *see* [ofstream.swap](#)
 operators *see* [depr.relops](#)
 optional.comp_with_t *see* [optional.comp.with.t](#)
 ostream::sentry *see* [ostream.sentry](#)
 ostringstream.assign *see* [ostringstream.swap](#)
 over.ass *see* [over.assign](#)
 over.dcl *see* [basic.link](#)
 over.load *see* [basic.scope.scope](#)

 range.semi.wrap *see* [range.move.wrap](#)
 range.split.inner *see* [range.lazy.split.inner](#)
 range.split.outer *see* [range.lazy.split.outer](#)
 range.split.outer.value *see* [range.lazy.split.outer.value](#)
 re.def *see* [intro.defs](#)
 re.regex.const *see* [re.regex](#)
 re.regex.nmswap *see* [re.regex.nonmemb](#)
 res.on.expects *see* [structure.specifications](#)
 res.on.pointer.storage *removed*
 res.on.required *removed*

reverse.iter.make *see* [reverse.iter.nonmember](#)
 reverse.iter.op!= *see* [reverse.iter.cmp](#)
 reverse.iter.op+ *see* [reverse.iter.nav](#)
 reverse.iter.op++ *see* [reverse.iter.nav](#)
 reverse.iter.op+= *see* [reverse.iter.nav](#)
 reverse.iter.op- *see* [reverse.iter.nav](#)
 reverse.iter.op- *see* [reverse.iter.nav](#)
 reverse.iter.op= *see* [reverse.iter.nav](#)
 reverse.iter.op.star *see* [reverse.iter.elem](#)
 reverse.iter.op< *see* [reverse.iter.cmp](#)
 reverse.iter.op<= *see* [reverse.iter.cmp](#)
 reverse.iter.op= *see* [reverse.iter.cons](#)
 reverse.iter.op== *see* [reverse.iter.cmp](#)
 reverse.iter.op> *see* [reverse.iter.cmp](#)
 reverse.iter.op>= *see* [reverse.iter.cmp](#)
 reverse.iter.opdiff *see* [reverse.iter.nonmember](#)
 reverse.iter.opindex *see* [reverse.iter.elem](#)
 reverse.iter.opref *see* [reverse.iter.elem](#)
 reverse.iter.ops *removed*
 reverse.iter.opsum *see* [reverse.iter.nonmember](#)

 set.special *removed*
 sf.cmath.assoc_laguerre *see* [sf.cmath.assoc.laguerre](#)
 sf.cmath.assoc_legendre *see* [sf.cmath.assoc.legendre](#)
 sf.cmath.comp_ellint_1 *see* [sf.cmath.comp.ellint.1](#)

 sf.cmath.comp_ellint_2 *see* [sf.cmath.comp.ellint.2](#)

 sf.cmath.comp_ellint_3 *see* [sf.cmath.comp.ellint.3](#)

 sf.cmath.cyl_bessel_i *see* [sf.cmath.cyl.bessel.i](#)
 sf.cmath.cyl_bessel_j *see* [sf.cmath.cyl.bessel.j](#)
 sf.cmath.cyl_bessel_k *see* [sf.cmath.cyl.bessel.k](#)
 sf.cmath.cyl_neumann *see* [sf.cmath.cyl.neumann](#)
 sf.cmath.ellint_1 *see* [sf.cmath.ellint.1](#)
 sf.cmath.ellint_2 *see* [sf.cmath.ellint.2](#)
 sf.cmath.ellint_3 *see* [sf.cmath.ellint.3](#)
 sf.cmath.riemann_zeta *see* [sf.cmath.riemann.zeta](#)

 sf.cmath.sph_bessel *see* [sf.cmath.sph.bessel](#)
 sf.cmath.sph_legendre *see* [sf.cmath.sph.legendre](#)
 sf.cmath.sph_neumann *see* [sf.cmath.sph.neumann](#)

shared_mutex.syn *see* [shared.mutex.syn](#)
 stmt.stmt *see* [stmt](#)
 stopsource.nonmembers *see* [stopsource](#)
 stoptoken.cons *see* [stopsource](#)
 stoptoken.nonmembers *see* [stopsource](#)
 string.find.first.not.of *see* [string.find](#)
 string.find.first.of *see* [string.find](#)
 string.find.last.not.of *see* [string.find](#)
 string.find.last.of *see* [string.find](#)
 string.op!= *see* [string.cmp](#)
 string.op+ *see* [string.op.plus](#)
 string.op+= *see* [string.op.append](#)
 string.op< *see* [string.cmp](#)
 string.op<= *see* [string.cmp](#)

[string.op>](#) *see* [string.cmp](#)
[string.op>=](#) *see* [string.cmp](#)
[string.operator==](#) *see* [string.cmp](#)
[string.rfind](#) *see* [string.find](#)
[stringstream.assign](#) *see* [stringstream.swap](#)
[support.initlist.range](#) *removed*
[system__error.syn](#) *see* [system.error.syn](#)

[temp.class.order](#) *see* [temp.spec.partial.order](#)
[temp.class.spec](#) *see* [temp.spec.partial](#)
[temp.class.spec.general](#) *see*
 [temp.spec.partial.general](#)
[temp.class.spec.match](#) *see* [temp.spec.partial.match](#)

[temp.class.spec.mfunc](#) *see*
 [temp.spec.partial.member](#)
[temp.inject](#) *see* [temp.friend](#)
[temp.nondep](#) *see* [temp.res](#)
[thread.decaycopy](#) *removed*
[time.traits.duration__values](#) *see*
 [time.traits.duration.values](#)
[time.traits.is_fp](#) *see* [time.traits.is.fp](#)
[type.index.hash](#) *removed*
[type.index.members](#) *removed*
[type.index.overview](#) *removed*

[uaxid.def.rfmt](#) *removed*
[unord.map.swap](#) *removed*
[unord.multimap.swap](#) *removed*
[unord.multiset.swap](#) *removed*
[unord.set.swap](#) *removed*
[unreachable.sentinel](#) *see* [unreachable.sentinel](#)
[util.dynamic.safety](#) *removed*
[util.smartptr](#) *removed*
[util.smartptr.shared.atomic](#) *removed*
[utility.as__const](#) *see* [utility.as.const](#)
[utility.from.chars](#) *see* [charconv.from.chars](#)
[utility.to.chars](#) *see* [charconv.to.chars](#)
[utility.unreachable](#) *see* [utility.undefined](#)

[variant.traits](#) *removed*
[vector.special](#) *removed*

Index

Constructions whose name appears in *monospaced italics* are for exposition only.

Symbols

!, *see* operator, logical negation
 !=, *see* operator, inequality
 (), *see* operator, function call, *see* declarator, function
 *, *see* operator, indirection, *see* operator, multiplication, *see* declarator, pointer
 +, *see* operator, unary plus, *see* operator, addition
 ++, *see* operator, increment
 ,, *see* operator, comma
 −, *see* operator, unary minus, *see* operator, subtraction
 −−, *see* operator, decrement
 −>, *see* operator, class member access
 −>*, *see* operator, pointer to member
 ., *see* operator, class member access
 .*, *see* operator, pointer to member
 . . . , *see* ellipsis
 /, *see* operator, division
 :
 bit-field declaration, 337
 label specifier, 193
 ::, *see* operator, scope resolution
 ::*, *see* declarator, pointer-to-member
 <, *see* operator, less than
 template and, 422, 424
 <<, *see* operator, left shift
 <=, *see* operator, less than or equal to
 <=>, *see* operator, three-way comparison
 =, *see* assignment operator
 ==, *see* operator, equality
 >, *see* operator, greater than
 >=, *see* operator, greater than or equal to
 >>, *see* operator, right shift
 ?:, *see* operator, conditional expression
 [], *see* operator, subscripting, *see* declarator, array
 # operator, 537, 540
 ## operator, 540
 #define, 537
 #elif, 527
 #elifdef, 528
 #elifndef, 528
 #else, 528
 #embed, 529
 #endif, 528
 #error, *see* preprocessing directive, error
 #if, 527, 576
 #ifdef, 528
 #ifndef, 528
 #include, 528, 561

#line, *see* preprocessing directive, line control
 #pragma, *see* preprocessing directive, pragma
 #undef, 542, 572
 %, *see* operator, remainder
 &, *see* operator, address-of, *see* operator, bitwise AND, *see* declarator, reference
 &&, *see* operator, logical AND
 ^, *see* operator, bitwise exclusive OR
 \, *see* backslash character
 submdspan
 canonical index type for `IndexType`, 1181
 canonical slice type for `IndexType`, 1181
 slice range of `s` for the k^{th} extent of `e`, 1182
 slice type for `IndexType`, 1181
 valid slice for the k^{th} extent of `e`, 1182
 valid slice type for the k^{th} extent of `E`, 1182
 {}
 block statement, 193
 class declaration, 314
 class definition, 314
 enum declaration, 276
 initializer list, 253
 _ , *see* character, underscore
 __cplusplus, 543
 __cpp_aggregate_bases, 544
 __cpp_aggregate_nsdmi, 544
 __cpp_aggregate_paren_init, 544
 __cpp_alias_templates, 544
 __cpp_aligned_new, 544
 __cpp_attributes, 544
 __cpp_auto_cast, 544
 __cpp_binary_literals, 544
 __cpp_capture_star_this, 544
 __cpp_char8_t, 544
 __cpp_concepts, 544
 __cpp_conditional_explicit, 544
 __cpp_consteval, 544
 __cpp_constexpr, 544
 __cpp_constexpr_dynamic_alloc, 544
 __cpp_constexpr_exceptions, 544
 __cpp_constexpr_in_decltype, 544
 __cpp_constexpr_virtual_inheritance, 544
 __cpp_constinit, 544
 __cpp_contracts, 544
 __cpp_decltype, 544
 __cpp_decltype_auto, 544
 __cpp_deduction_guides, 544
 __cpp_delegating_constructors, 544
 __cpp_deleted_function, 544
 __cpp_designated_initializers, 544
 __cpp_enumerator_attributes, 544

__cpp_expansion_statements, 544
 __cpp_explicit_this_parameter, 544
 __cpp_fold_expressions, 544
 __cpp_generic_lambdas, 544
 __cpp_guaranteed_copy_elision, 544
 __cpp_hex_float, 544
 __cpp_if_consteval, 544
 __cpp_if_constexpr, 544
 __cpp_impl_coroutine, 544
 __cpp_impl_destroying_delete, 544
 __cpp_impl_reflection, 544
 __cpp_impl_three_way_comparison, 544
 __cpp_implicit_move, 545
 __cpp_inheriting_constructors, 545
 __cpp_init_captures, 545
 __cpp_initializer_lists, 545
 __cpp_inline_variables, 545
 __cpp_lambdas, 545
 __cpp_modules, 545
 __cpp_multidimensional_subscript, 545
 __cpp_named_character_escapes, 545
 __cpp_namespace_attributes, 545
 __cpp_noexcept_function_type, 545
 __cpp_nontype_template_args, 545
 __cpp_nontype_template_parameter_auto, 545
 __cpp_nsdmi, 545
 __cpp_pack_indexing, 545
 __cpp_placeholder_variables, 545
 __cpp_pp_embed, 545
 __cpp_range_based_for, 545
 __cpp_raw_strings, 545
 __cpp_ref_qualifiers, 545
 __cpp_return_type_deduction, 545
 __cpp_rvalue_references, 545
 __cpp_size_t_suffix, 545
 __cpp_sized_deallocation, 545
 __cpp_static_assert, 545
 __cpp_static_call_operator, 545
 __cpp_structured_bindings, 545
 __cpp_template_parameters, 545
 __cpp_template_template_args, 545
 __cpp_threadsafe_static_init, 545
 __cpp_trivial_union, 545
 __cpp_unicode_characters, 545
 __cpp_unicode_literals, 545
 __cpp_user_defined_literals, 545
 __cpp_using_enum, 545
 __cpp_variable_templates, 545
 __cpp_variadic_friend, 545
 __cpp_variadic_templates, 545
 __cpp_variadic_using, 545
 __DATE__, 543
 __FILE__, 543
 __func__, 268
 __has_cpp_attribute, 526
 __has_embed, 526
 __has_include, 526
 __LINE__, 543
 __STDC__, 546

__STDC_HOSTED__, 544
 __STDC_ISO_10646__, 546
 __STDC_MB_MIGHT_NEQ_WC__, 546
 __STDC_VERSION__, 546
 __STDCPP_BFLOAT16_T__, 546
 __STDCPP_DEFAULT_NEW_ALIGNMENT__, 544
 __STDCPP_FLOAT128_T__, 546
 __STDCPP_FLOAT16_T__, 545
 __STDCPP_FLOAT32_T__, 545
 __STDCPP_FLOAT64_T__, 545
 __STDCPP_THREADS__, 546
 __TIME__, 544
 __VA_ARGS__, 537, 538
 __VA_OPT__, 537–539
 |, *see* operator, bitwise inclusive OR
 ||, *see* operator, logical OR
 ~, *see* operator, ones' complement, *see* destructor

Numbers

0

null character, *see* character, null
 string terminator, 29

A

abbreviated

template function, *see* template, function,
 abbreviated

ABI tag, 1893

abort, 102, 201

absolute path, *see* path, absolute

abstract class, *see* class, abstract

abstract-declarator, 229, 2401

abstract-pack-declarator, 230, 2401

access, 3

access control, 351–361

anonymous union, 342

base class, 354

base class member, 344

class member, 143

default, 352

default argument, 353

friend function, 356

member function and, 324

member name, 352

multiple access, 361

nested class, 361

private, 351

protected, 352, 360

public, 352

using-declaration and, 289

virtual function, 360

access specifier, 353, 354

access-specifier, 344, 2406

accessible, 355

accessible range, *see* range, accessible

accessor policy, 1172

acquire, 82

active, *see* variable, active

- union member, [341](#)
- active block, *see* block, active
- active macro definition, *see* macro, active
- addition operator, *see* operator, addition
- additive-expression*, [168](#), [2395](#)
- address, [87](#), [171](#)
- addressable function, *see* function, addressable
- aggregate, [253](#)
 - elements, [253](#)
- aggregate base, *see* base, aggregate
- aggregate deduction candidate, *see* candidate, aggregate deduction
- aggregate initialization, [253](#)
- algorithm
 - stable, [8](#), [577](#)
- algorithm function object, [555](#)
- alias
 - namespace, [282](#)
 - type, [212](#)
- alias template, *see* template, alias
- alias-declaration*, [206](#), [2398](#)
- alignas**, [294](#)
 - keyword, [21](#), [292](#), [2404](#)
- alignment, [68](#)
 - extended, [69](#)
 - fundamental, [68](#)
 - new-extended, [69](#)
 - stricter, [69](#)
 - stronger, [69](#)
 - weaker, [69](#)
- alignment requirement
 - implementation-defined, [68](#)
- alignment-specifier*, [292](#), [2404](#)
- alignof**, [155](#)
 - keyword, [21](#), [38](#), [68](#), [69](#), [151](#), [155](#), [474](#), [475](#), [2394](#)
- allocated type, *see* type, allocated
- allocation
 - alignment storage, [158](#)
 - implementation-defined bit-field, [338](#)
- allocation function, [75](#)
 - class-specific, [338](#)
- allocator-aware container, *see* container, allocator-aware
- alternate form
 - format string, [1690](#)
- alternative token, *see* token, alternative
- ambiguity
 - base class member, [47](#)
 - class conversion, [49](#)
 - declaration type, [209](#)
 - declaration versus cast, [230](#)
 - declaration versus expression, [204](#)
 - function declaration, [251](#)
 - member access, [47](#)
 - overloaded function, [377](#)
 - parentheses and, [156](#)
- ambiguous, [46](#)
- ambiguous conversion sequence, *see* conversion sequence, ambiguous
- Amendment 1, [573](#)
- and**
 - keyword, [19](#), [20](#), [22](#), [2387](#), [2454](#)
- and-expression*, [172](#), [2395](#)
- and_eq**
 - keyword, [19](#), [20](#), [22](#), [2387](#), [2454](#)
- annotation*, [292](#), [2404](#)
- annotation-list*, [292](#), [2404](#)
- anonymous union, [342](#)
 - member, *see* member, anonymous union
 - variable, *see* variable, anonymous union
- appearance-ordered, [99](#)
- appertain, [257](#), [293](#)
- argc**, [98](#)
- argument, [3](#), [575](#), [576](#), [652](#)
 - access checking and default, [353](#)
 - binding of default, [244](#)
 - evaluation of default, [244](#), [245](#)
 - example of default, [243](#), [244](#)
 - function call expression, [3](#)
 - function-like macro, [3](#)
 - overloaded operator and default, [408](#)
 - reference, [142](#)
 - scope of default, [245](#)
 - template, [426](#)
 - template instantiation, [3](#)
 - throw expression, [3](#)
 - type checking of default, [244](#)
- argument and name hiding
 - default, [245](#)
- argument and virtual function
 - default, [245](#)
- argument forwarding call wrapper, [921](#)
- argument list
 - empty, [239](#)
 - variable, [239](#)
- argument passing, [142](#)
 - reference and, [260](#)
- argument substitution, *see* macro, argument substitution
- argument type
 - unknown, [239](#)
- argument-dependent lookup, *see* lookup, argument-dependent
- argv**, [98](#)
- arithmetic
 - pointer, [168](#)
 - unsigned, [82](#)
- arithmetic expression, *see* expression, arithmetic
- array
 - backing, [266](#)
 - bound, [236](#)
 - const**, [89](#)
 - delete**, [161](#)
 - element, [237](#)
 - first element, [237](#)
 - handler of type, [518](#)

- `new`, 157
 - parameter of type, 239
 - `sizeof`, 155
 - template parameter of type, 421
 - array**
 - as aggregate, 996
 - contiguous storage, 996
 - creation, 998
 - initialization, 996, 998
 - tuple interface to, 999
 - zero sized, 998
 - array size
 - default, 237
 - array type, 237
 - arrow operator, *see* operator, class member access
 - as-if rule, 11
 - asm**
 - implementation-defined, 289
 - keyword, 21, 290, 2403
 - asm-declaration*, 290, 2403
 - assembler, 289
 - assertion
 - postcondition, 246
 - precondition, 246
 - statement, 203
 - assertion-statement*, 203, 2397
 - assignment
 - and lvalue, 175
 - conversion by, 176
 - copy, *see* assignment operator, copy
 - move, 6, *see* assignment operator, move
 - reference, 259
 - assignment operator
 - copy, 323, 328–330
 - hidden, 330
 - implicitly declared, 329
 - non-trivial, 330
 - trivial, 330
 - virtual bases and, 330
 - move, 323, 328–330
 - hidden, 330
 - implicitly declared, 329
 - non-trivial, 330
 - trivial, 330
 - overloaded, 409
- assignment-expression*, 175, 2396
- assignment-operator*, 176, 2396
- associated, 2183
- associated constraints, 436
- associated element, *see* element, associated
- associated scope, *see* scope, associated
- associated with, 88
- associative container, *see* container, associative
- associative containers
 - exception safety, 984
 - requirements, 984
 - unordered, *see* unordered associative containers
- assumption, 295
- async lifetime, 2297
- async result, *see* result, async
- asynchronous operation, *see* operation, asynchronous
- asynchronous provider, 2276
- asynchronous return object, 2276
- at least as constrained, 439
- at least as specialized as, *see* more specialized
- `atexit`, 101
- atomic
 - notifying operation, 2209
 - operation, 93–98
 - smart pointers, 2234–2239
 - waiting operation, 2209
 - eligible to be unblocked, 2210
- atomic constraint, *see* constraint, atomic identical, 434
- atomic modify-write operation, *see* modify-write operation, atomic
- attached
 - declaration, 303
 - entity, 62
- attempt, 1783
- attribute, 292–301, 2298
 - alignment, 294
 - annotations, 300
 - deprecated, 295
 - fallthrough, 296
 - indeterminate, 296
 - likely, 297
 - maybe unused, 298
 - no unique address, 299
 - nodiscard, 298
 - noreturn, 299
 - syntax and semantics, 292
 - unlikely, 297
- attribute*, 292, 2404
- attribute-argument-clause*, 293, 2404
- attribute-declaration*, 207, 2398
- attribute-list*, 292, 2404
- attribute-namespace*, 293, 2404
- attribute-scoped-token*, 293, 2404
- attribute-specifier*, 292, 2404
- attribute-specifier-seq*, 292, 2403
- attribute-token*, 293, 2404
- attribute-using-prefix*, 292, 2404
- attributed-identifier*, 248, 2401
- auto**
 - keyword, 21, 124, 130, 170, 193, 198, 205, 222–225, 228, 230, 238, 242, 243, 376, 2400, 2432, 2439, 2449
- automatic storage duration, *see* storage duration, automatic
- await expression
 - final, 272
 - initial, 272
- await-expression*, 153, 2394
- awaitable, 2321

B

backing array, *see* array, backing

backslash character, 25

`bad_alloc`, 159

`bad_cast`, 146

`bad_typeid`, 147

balanced-token, 293, 2404

balanced-token-seq, 293, 2404

barrier

phase synchronization point, 2273

barrier phase, 2273

base

aggregate, 254

base characteristic, 751

base class, 344, 345

dependent, 470

direct, 344

indirect, 344

non-virtual, 345

private, 354

protected, 354

public, 354

virtual, 345

base class subobject, 65

base prefix, 1692

base-2 representation, 83

base-clause, 344, 2406

base-specifier, 344, 2406

base-specifier-list, 344, 2406

basic character set, *see* character set, basic

basic literal character set, *see* character set, basic
literal

basic-c-char, 23, 2389

basic-s-char, 27, 2390

BasicFormatter, 1696

behavior

conditionally-supported, 4, 11

default, 4, 551

erroneous, 4

implementation-defined, 5, 11

locale-specific, 5

observable, 11, 12

on receipt of signal, 93

required, 7, 551

runtime-undefined, 7

undefined, 9–11, 1253

unspecified, 9, 11

Bernoulli distributions, 1786–1789

Bessel functions

I_ν , 1833

J_ν , 1833

K_ν , 1833

N_ν , 1834

j_n , 1836

n_n , 1836

beta functions B, 1832

better conversion, *see* conversion, better

better conversion sequence, *see* conversion
sequence, better

binary element-wise operation, *see* operation,
binary element-wise

binary fold, 134

binary left fold, 134

binary operator

interpretation of, 409

overloaded, 409

binary operator function, *see* operator function,
binary

binary right fold, 134

binary-digit, 22, 2388

binary-exponent-part, 26, 2390

binary-literal, 22, 2388

bind directly, 262

binding

reference, 260

bit-field, 337

address of, 338

alignment of, 338

converted, 112

implementation-defined alignment of, 338

implementation-defined sign of, 2452

type of, 337

unnamed, 338

zero width of, 338

bitand

keyword, 19, 20, 22, 2387, 2454

bitmask

element, 553

empty, 553

value

clear, 553

is set, 553

set, 553

bitor

keyword, 19, 20, 22, 2387, 2454

block

active, 1012

reserved, 1012

block (execution), 3, 2182, 2197, 2200, 2201,
2245–2249, 2264, 2267, 2270–2274, 2281,
2283, 2284

with forward progress guarantee delegation,
97

block (statement), 3, *see* statement, compound
initialization in, 203

structure, 203

block scope, *see* scope, block

block variable, 44

block-declaration, 206, 2398

body

function, 268

replacement, 271

Bond

James Bond, 131

bool

keyword, 21, 83, 84, 110–115, 123, 145,
152–155, 169–171, 173, 184, 194, 208,
212, 218, 519, 1701, 2399, 2446, 2476

Boolean literal, 29

boolean literal, *see* literal, boolean

Boolean type, 83

boolean-literal, 29, 2391

bound argument entity, 920

bound arguments, 935

bound, of array, 236

brace-or-equal-initializer, 249, 2402

braced-init-list, 249, 2402

bracket resource search, *see* search, bracket
resource

brains
names that want to eat your, 572

break
keyword, 21, 196, 201, 514, 2397

buckets, 985

built-in candidate, 383

built-in operators, *see* operators, built-in

byte, 64, 154

C

C
linkage to, 290
standard library, 3

c-char, 23, 2389

c-char-sequence, 23, 2389

C++ library headers
importable, 557

C++ library modules, 559

call
dependent, 469
nodiscard, 298
operator function, 408

call pattern, 921

call signature, 920

call wrapper, 920, 921
forwarding, 921
perfect forwarding, 921
simple, 921
type, 920

call-compatible, 141

callable object, *see* object, callable

callable type, *see* type, callable

callback deregistration
stoppable, 2185

candidate, 377
aggregate deduction, 387
usable, 378

candidate initializer, 429

capture
implicit, 130

capture, 129, 2392

capture-default, 129, 2392

capture-list, 129, 2392

captured, 131

by copy, 132
by reference, 133

case
keyword, 21, 193, 195, 196, 203, 2396

cast
base class, 148
const, 150, 165
derived class, 148
dynamic, 145, 608
construction and, 371
destruction and, 371
integer to pointer, 149
lvalue, 147, 149
pointer to integer, 149
pointer-to-function, 149
pointer-to-member, 148, 150
reference, 147, 150
reinterpret, 149, 165
integer to pointer, 149
lvalue, 149
pointer to integer, 149
pointer-to-function, 149
pointer-to-member, 150
reference, 150
static, 147, 165
lvalue, 147
reference, 147
undefined pointer-to-function, 149

cast-expression, 165, 2395

casting, 143

casting away constness, 151

catch, 514
keyword, 21, 514, 633, 2408

category tag, 1221

cats
interfering with canines, 606

<ccomplex>
absence thereof, 573, 2429

char
implementation-defined sign of, 83
keyword, 21, 25, 28, 65, 69, 80, 82–84, 88, 89,
98, 108, 158, 208, 218, 259, 515, 1640,
2150, 2399, 2419

char-like object, 1589

char-like type, 1589

char16_t, *see* type, **char16_t**
keyword, 3, 21, 25, 28, 83, 84, 112, 113, 115,
155, 218, 219, 251, 259, 548, 827, 1749,
2130, 2140, 2399, 2430, 2437, 2453, 2462

char32_t, *see* type, **char32_t**
keyword, 3, 21, 25, 28, 83, 84, 112, 113, 115,
155, 218, 219, 251, 259, 548, 827, 1749,
2130, 2140, 2399, 2430, 2437, 2453, 2462

char8_t, *see* type, **char8_t**
keyword, 3, 21, 25, 28, 83, 84, 112, 113, 115,
218, 219, 251, 259, 548, 827, 1749, 2130,
2140, 2399, 2424, 2453, 2462

character, 3
decimal-point, 554

- fill, 1689
- formatted as escaped, 1699
- multibyte, 554
- null, 15
- signed, 83
- source file, 13
- terminating null, 554
- underscore, 21, 574
 - in identifier, 21
- character literal, *see* literal, character
- character sequence, 554
- character set, 15
 - basic, 15
 - basic literal, 15, 64
 - execution, 553
 - translation, 15
- character string literal, 540
- character type, *see* type, character
- character-literal*, 23, 2389
- characteristic sequence, *see* sequence,
 - characteristic
- checking
 - point of error, 466
 - syntax, 466
- checkpoints
 - observable, 11
- child operations, *see* operations, child
- chunks, 741
- <ciso646>
 - absence thereof, 573, 2429
- class, 87, 314–376
 - abstract, 351
 - base, 574, 577, 578
 - cast to incomplete, 166
 - constructor and abstract, 351
 - current, 116
 - definition, 35
 - derived, 577
 - designating, 356
 - implicit-lifetime, 316
 - linkage of, 60
 - linkage specification, 291
 - local, 343, *see* local class
 - member function, *see* member function, class
 - nested, 340
 - polymorphic, 347
 - scope of enumerator, 278
 - standard-layout, 81, 315
 - templated, 418
 - trivial, 81, 2458
 - trivially copyable, 81, 315
 - union-like, 343
 - unnamed, 213, 314
 - variant member of, 343
- class
 - keyword, 21, 80, 221, 276, 314, 316, 318, 352, 354, 418, 419, 483, 1769, 2402, 2405, 2407, 2449, 2451
- class member access operator function, *see*
 - operator function, class member access
- class name
 - elaborated, 220, 317
 - point of declaration, 318
 - typedef, 213, 318
- class object
 - member, 321
 - sizeof, 155
- class object copy, *see* constructor, copy
- class object initialization, *see* constructor
- class scope, *see* scope, class
- class-head*, 314, 2405
- class-head-name*, 314, 2405
- class-key*, 314, 2405
- class-name*, 314, 2405
- class-or-decltype*, 344, 2406
- class-property-specifier*, 314, 2405
- class-property-specifier-seq*, 314, 2405
- class-specifier*, 314, 2405
- closure object, 123
- closure type, 124
- co_await, 153
 - keyword, 21, 153, 174, 274, 408, 2394, 2406, 2424
- co_return, 202
 - keyword, 21, 120, 202, 2397, 2424
- co_yield, 174
 - keyword, 21, 174, 2396, 2424
- code unit, 15
- <codecvt>
 - absence thereof, 573, 2418
- coherence
 - read-read, 95
 - read-write, 95
 - write-read, 95
 - write-write, 95
- coherence-ordered before, 2207
- collapsing slice type, *see* slice type, collapsing
- collating element, 4
- comma operator, *see* operator, comma
- comment, 17
 - /* */, 17
 - //, 17
- common comparison type, 376
- common initial sequence, 321
- compare-expression*, 169, 2395
- comparison
 - pointer, 171
 - pointer to function, 171
 - undefined pointer, 168
- comparison category types, 620
- comparison operator function, *see* operator
 - function, comparison
- compatible
 - template template parameter and template argument, 431
- compatible for subsumption, *see* subsumption,
 - compatible for

- compatible with
 - `shared_ptr`, 708
- compilation
 - separate, 13
- compiler control line, *see* preprocessing directive
- `compl`
 - keyword, 19, 20, 22, 2387, 2454
- complete object, 65
- complete object of, 66
- complete-class context, 320
- completely defined, 320
- completion
 - error, 2297
 - permissible, 2320
 - stopped, 2297
 - value, 2297
- completion function, *see* function, completion
- completion operation, *see* operation, completion
- completion scheduler, 2298
- completion signature, 2298
- completion tag, *see* tag, completion
- component, 4
- component name, 118, 121, 218, 220, 236, 285, 314, 344, 418, 423, 464
- composite pointer type, 108
- compound statement, *see* statement, compound
- compound-requirement*, 136, 2393
- compound-statement*, 193, 2397
- computed-type-specifier*, 218, 2399
- concatenation
 - macro argument, *see* `##` operator
 - string, 14, 28
- concept, 462
 - model, 575
 - type, 463
- concept**
 - keyword, 21, 462, 2408
- concept template parameter, *see* template
 - parameter, concept
- concept-definition*, 462, 2408
- concept-dependent constraint, *see* constraint,
 - concept-dependent
- concept-id, 426
- concept-name*, 462, 2408
- concept-tt-parameter*, 418, 2407
- concurrent forward progress guarantees, 97
- condition*, 192, 2396
- conditions*
 - rules for, 192
- condition-declaration*, 192, 2396
- conditional-escape-sequence*, 24, 2389
- conditional-escape-sequence-char*, 24, 2390
- conditional-expression
 - throw-expression in, 173
- conditional-expression*, 173, 2396
- conditionally-supported behavior, *see* behavior,
 - conditionally-supported
- conditionally-supported-directive*, 524, 2409
- conflict, 94
- conformance requirements, 10–12
 - class templates, 10
 - classes, 10
 - general, 10–11
 - library, 10
 - method of description, 10
- conjunction, 433
- connect, 2297
- consistency
 - linkage, 210
 - linkage specification, 291
 - type declaration, 62
- const**, 88
 - cast away, 151
 - constructor and, 325
 - destructor and, 331
 - keyword, 6, 21, 28, 89, 102, 104, 107, 109, 112, 116, 118, 119, 122, 125, 142, 144, 146, 161, 167, 208, 210, 212, 216, 217, 229, 234, 239, 247, 248, 260, 325–327, 329, 331, 337, 338, 378, 393, 399, 495, 515, 554, 564, 606, 613, 642, 645, 771, 965–967, 1208, 1702, 1768, 1820, 2400, 2436, 2441, 2445, 2452
 - linkage of, 60
- const object, *see* object, const
 - undefined change to, 217
- const volatile object, *see* object, const volatile
- const-default-constructible, 250
- const-qualified, 88
- const-volatile-qualified, 88
- const_cast**, *see* cast, const
 - keyword, 21, 78, 139, 142, 150, 151, 165, 166, 424, 465, 474, 475, 965, 1208, 2394, 2452
- constant, 22, 115
 - enumeration, 277
 - immediate, 183
 - null pointer, 113, 114
 - underlying, 300
- constant destruction, *see* destruction, constant
- constant evaluation, 4
- constant expression, *see* expression, constant
- constant initialization, 99
- constant iterator, 1200
- constant subexpression, 4
- constant template parameter, *see* template
 - parameter, constant
- constant-expression*, 177, 2396
- constant-initializable, 185
- constant-initialized, 185
- constexpr**
 - keyword, 21, 98, 123, 125, 186, 194, 195, 207, 209, 213–215, 233, 323, 324, 374, 451, 486, 2392, 2397, 2398, 2424
- constexpr if statement, *see* statement, constexpr if
- constexpr-block-declaration*, 207, 2398
- constexpr-only value, *see* value, constexpr-only
- constexpr**

- keyword, 21, 33, 98, 123, 125, 126, 180, 183, 186, 193–195, 200, 209, 213–215, 233, 249, 323, 324, 331, 337, 374, 415, 451, 466, 486, 576, 590, 874, 921, 1241, 1251, 1253, 2392, 2397, 2398, 2431, 2436, 2437, 2456
- constexpr function, 214
- constexpr if, 194
- constexpr iterator, 1201
- constexpr-enabled, 1698
- constexpr-referenceable, 184
- constexpr-representable, 185
- constexpr-suitable, 214
- constexpr-unknown, 181
- constexpr-unknown representation, *see* representation, constexpr-unknown
- constinit**
 - keyword, 21, 209, 215, 233, 2398, 2424
- constituent expression, 90
- constituent reference, 67
- constituent value, 67
- constraint, 432
 - associated, *see* associated constraints
 - atomic, 434
 - concept-dependent, 435
 - fold expanded, 435
 - immediately-declared, 420
 - normalization, 437–439
 - satisfaction
 - atomic, 434
 - concept-dependent, 435
 - conjunction, 433
 - disjunction, 433
 - subsumption, 439
- constraint-expression*, 436, 2408
- constraint-logical-and-expression*, 416, 2407
- constraint-logical-or-expression*, 416, 2407
- construct
 - separately instantiated, 441
- constructed
 - transitively, 2310
- construction, 369–371
 - deemed, 252
 - dynamic cast and, 371
 - member access, 369
 - move, 6
 - pointer to member or base, 370
 - typeid** operator, 371
 - virtual function call, 370
- constructor, 324
 - address of, 325
 - array of class objects and, 362
 - copy, 77, 323, 326–328, 555
 - elision, 371, 372
 - implicitly declared, 327
 - nontrivial, 328
 - trivial, 327
 - default, 323, 325
 - non-trivial, 325
 - trivial, 325
 - delegating, 363
 - exception handling, *see* exception handling, constructors and destructors
 - explicit call, 324
 - implicitly invoked, 325
 - inheritance of, 325
 - inherited, 285
 - move, 323, 326–328
 - elision, 371, 372
 - implicitly declared, 327
 - non-trivial, 328
 - trivial, 327
 - non-trivial, 325
 - target, 363
 - union**, 341
- constructor, conversion by, *see* conversion, user-defined
- contain a value
 - optional**, 855
- contained value
 - any**, 885
 - optional**, 855
 - optional.ref**, 864
 - variant**, 874
- container, 961
 - allocator-aware, 965
 - associative, 977
 - contiguous, 964
 - reversible, 964
 - sequence, 968
 - unordered associative, 986
- contains a value
 - optional.ref**, 864
- context
 - evaluation, 189
 - immediate, 496
 - non-deduced, 505
 - type-only, 464
- contextually converted constant expression of type **bool**, *see* conversion, contextual
- contextually converted to **bool**, *see* conversion, contextual to **bool**
- contextually implicitly converted, 110
- contiguous container, *see* container, contiguous
- continue**
 - and handler, 514
 - and try block, 514
 - keyword, 21, 197, 201, 514, 2397
- contract assertion, 102–104
 - function, 246–248
 - postcondition, *see* assertion, postcondition
 - precondition, *see* assertion, precondition
 - statement, *see* assertion, statement
- contract evaluation semantics
 - checking, 10, 102
 - enforce, 102, 103
 - ignore, 102, 185
 - observe, 102, 103

- quick-enforce, 102, 103
- terminating, 10, 102, 103, 617
- contract violation, 102
- contract-assertion predicate, *see* predicate,
 - contract-assertion
- contract-assertion scope, *see* scope,
 - contract-assertion
- contract-terminated, 103
- contract-violation handler, 104
 - default, 104
- `contract_assert`
 - keyword, 21, 2414
- `contract_violation`, 103, 104
- control line, *see* preprocessing directive
- control-flow-limited statement, *see* statement,
 - control-flow-limited
- control-line*, 523, 2409
- controlling expression, *see* expression, controlling
- conventions, 551
 - lexical, 13–31
- conversion
 - argument, 238
 - array-to-pointer, 111
 - better, 402
 - `bool`, 113
 - boolean, 114
 - class, 333
 - contextual, 110
 - contextual to `bool`, 110
 - contextual to constant expression of type
 - `bool`, 184
 - deduced return type of user-defined, 336
 - derived-to-base, 397
 - floating to integral, 113
 - floating-point, 113
 - function pointer, 114
 - function-to-pointer, 111
 - immediate-escalating, 186
 - implementation-defined pointer integer, 149
 - implicit, 109, 110, 333
 - implicit user-defined, 333
 - inheritance of user-defined, 336
 - integer rank, 89
 - integral, 113
 - integral to floating, 113
 - lvalue-to-rvalue, 110, 2447
 - narrowing, 267
 - null member pointer, 114
 - null pointer, 114
 - overload resolution and, 393
 - overload resolution and pointer, 407
 - pointer, 113
 - pointer-to-member, 114
 - `void*`, 114
 - qualification, 111–112
 - return type, 201
 - standard, 109–114
 - temporary materialization, 111
 - to signed, 113
 - to unsigned, 113
 - user-defined, 333, 334
 - usual arithmetic, 115
 - virtual user-defined, 336
- conversion explicit type, *see* casting
- conversion function, *see* conversion, user-defined,
 - see* function, conversion
- conversion rank, 398
 - floating-point, 90
- conversion sequence
 - ambiguous, 397
 - better, 402
 - implicit, 396
 - indistinguishable, 402
 - standard, 109
 - user-defined, 398
 - worse, 402
- conversion-declarator*, 335, 2406
- conversion-function-id*, 334, 2406
- conversion-type-id*, 334, 2406
- converted bit-field, *see* bit-field, converted
- converted constant expression, *see* expression,
 - converted constant
- copy
 - class object, *see* constructor, copy, *see*
 - assignment operator, copy
- copy deduction candidate, 387
- copy elision, *see* constructor, copy, elision
- copy-initialization, 250
- copy-list-initialization, 262
- `copyable_function::is-callable-from`, 944
- core constant expression, *see* expression, core
 - constant
- coroutine, 271
 - await expression, 272
 - promise type, 271
 - resumer, 272
 - suspend point, 154
- coroutine return, *see* `co_return`
- coroutine state, 272
- coroutine-return-statement*, 202, 2397
- correspond, 41
- corresponding direct subobject, *see* subobject,
 - corresponding direct
- corresponding object parameter, *see* object
 - parameter, corresponding
- corresponding overloads, 41
- corresponding signature, *see* signature,
 - corresponding
- count, 2361
- counted range, *see* range, counted
- Cpp17Allocator*, 565
- Cpp17BasicLockable*, 2182
- Cpp17BidirectionalIterator*, 1217
- Cpp17BinaryTypeTrait*, 751
- Cpp17Clock*, 1955
- Cpp17CopyAssignable*, 562
- Cpp17CopyConstructible*, 562
- Cpp17CopyInsertable* into X, 966

Cpp17DefaultConstructible, 562
Cpp17DefaultInsertable into **X**, 965
Cpp17Destructible, 562
Cpp17EmplaceConstructible into **X** from **args**, 966
Cpp17EqualityComparable, 562
Cpp17Erasable from **X**, 966
Cpp17ForwardIterator, 1216
Cpp17Hash, 565
Cpp17InputIterator, 1214, 1215
Cpp17Iterator, 1214
Cpp17LessThanComparable, 562
Cpp17Lockable, 2182
Cpp17MoveAssignable, 562
Cpp17MoveConstructible, 562
Cpp17MoveInsertable into **X**, 965
Cpp17NullablePointer, 564
Cpp17OutputIterator, 1216
Cpp17RandomAccessIterator, 1217, 1218
Cpp17SharedLockable, 2183
Cpp17SharedTimedLockable, 2183
Cpp17Swappable, 563
Cpp17TimedLockable, 2182
Cpp17TransformationTrait, 751
Cpp17TrivialClock, 1956
Cpp17UnaryTypeTrait, 751
Cpp17ValueSwappable, 563
<cstdlibalign>
 absence thereof, 573, 2429
<cstdlibbool>
 absence thereof, 573, 2429
<ctgmath>
 absence thereof, 573, 2429
ctor-initializer, 363, 2406
current class, *see* class, current
current instantiation, 469
 dependent member of the, 471
 member of the, 470
currently handled exception, *see* exception
 handling, currently handled exception
customization point object, *see* object,
 customization point
cv-qualifier, 88
 top-level, 89
cv-qualifier, 229, 2400
cv-qualifier-seq, 229, 2400
cv-unqualified, 88

D

d-char, 27, 2391
d-char-sequence, 27, 2390
DAG
 multiple inheritance, 346
 non-virtual base class, 346
 virtual base class, 346
data member, *see* member, 319
 non-static, 319
 static, 319

data member description, *see* description, data
 member
data race, 96
data-parallel object, *see* object, data-parallel
data-parallel type, *see* type, data-parallel
deadlock, 4
deallocation function, 75
 class-specific, 338
 usual, 76
debug-enabled, 1698
decay
 array, *see* conversion, array-to-pointer
 function, *see* conversion, function-to-pointer
decay-copied from, 552
decimal-floating-point-literal, 25, 2390
decimal-literal, 22, 2388
decision variable, *see* variable, decision
decl-reachable, 307
decl-specifier, 209, 2398
decl-specifier-seq, 209, 2398
declaration, 32, 33, 206–301
 array, 237
 asm, 289
 bit-field, 337
 class name, 33
 constant pointer, 233
 default argument, 243–246
 definition versus, 33
 disqualifying, 646
 ellipsis in function, 142, 239
 exported, 304
 extern, 33
 extern reference, 260
 first, 33
 forward, 211
 forward class, 317
 freestanding item, 555
 function, 33, 208, 238
 injected, 188
 local class, 343
 locus, *see* locus
 member, 318
 multiple, 62
 name, 33
 name-independent, 42
 nominable, 43
 object, 208
 opaque enum, 33
 overloaded name and friend, 358
 parameter, 33, 238
 parentheses in, 230, 233
 point of, *see* locus
 pointer, 233
 potentially conflict, 42
 precede, 47
 reference, 234
 static member, 33
 storage class, 210

structured binding, *see* structured binding
 declaration
 type, 231
 typedef, 33, 208, 212, *see* alias, type
declaration, 206, 2397
 declaration hiding, *see* name hiding
declaration-seq, 206, 2397
declaration-statement, 203, 2397
 declarative, 121
 declarator, 33, 207, 227–268
 array, 236
 function, 238–243
 meaning of, 231–246
 multidimensional array, 237
 pointer, 233
 pointer-to-member, 236
 reference, 234
declarator, 228, 2400
declarator-id, 229, 2400
 declared specialization, *see* specialization, declared
decltype
 keyword, 21, 84, 137, 221, 222, 226, 496, 2400, 2430, 2437
decltype-specifier, 221, 2400
 decrement operator
 overloaded, *see* overloading, decrement
 operator
 decrement operator function, *see* operator function, decrement
 deducible template, *see* template, deducible
 deduction
 class template argument, 443
 class template arguments, 143, 218, 226, 387
 deduction substitution loci, 496
 placeholder type, 225
 deduction guide
 freestanding item, 556
 deduction substitution loci, 496
deduction-guide, 443, 2408
 deemed construction, *see* construction, deemed
default
 keyword, 21, 193, 196, 268, 2396, 2402
 default access control, *see* access control, default
 default argument
 overload resolution and, 392
 default argument instantiation, 484
 default constructor, *see* constructor, default
 default contract-violation handler, *see*
 contract-violation handler, default
 default member initializer, 320
 default memory resource pointer, 741
 default template argument, *see* template argument, default
 default-initialization, 250
 default-inserted, 965
 defaulted, 269
 deferred function, *see* function, deferred
 definable item, *see* item, definable
 define, 33

defined, 526
 defined prefix, *see* prefix, defined
defined-macro-expression, 526, 2410
defining-type-id, 229, 2401
defining-type-specifier, 216, 2399
defining-type-specifier-seq, 216, 2399
 definition, 33, 34
 class, 314, 319
 class name as type, 316
 constructor, 268
 coroutine, 271
 declaration as, 208
 deleted, 270
 function, 268–271
 deleted, 270
 explicitly-defaulted, 269
 local class, 343
 member function, 323
 namespace, 280
 nested class, 340
 program semantics affected by, 483
 pure virtual function, 351
 scope of class, 317
 static member, 337
 virtual function, 349
 definition domain, 38
 definitions, 3–9
 delegating constructor, *see* constructor, delegating
 delete
 array, 161
 single-object, 161
delete, 75, 161
 destructor and, 161, 332
 keyword, 21, 75, 76, 157, 161, 162, 268, 332, 408, 474, 603, 634, 698, 2395, 2402, 2406
 overloading and, 76
 single-object, 161
 type of, 338
 undefined, 161
delete-expression, 161, 2395
 deleted definition, *see* definition, deleted
 deleted function, *see* function, deleted
deleted-function-body, 268, 2402
 deleter, 697
 denormalized value, *see* number, subnormal
 denote, 33
 dependent base class, *see* base class, dependent
 dependent call, *see* call, dependent
 dependent member of the current instantiation,
 see current instantiation, dependent
 member of the
 dependent name, *see* name, dependent
 dereferenceable iterator, *see* iterator,
 dereferenceable
 dereferencing, *see* indirection
 derivation, *see* inheritance
 derived class, 344–351
 most, *see* most derived class
 derived object

- most, *see* most derived object
- description
 - data member, [322](#)
- designate, [59](#)
 - aggregate element, [254](#)
- designated-initializer-clause*, [249](#), [2402](#)
- designated-initializer-list*, [249](#), [2402](#)
- designated-only-initializer-list*, [249](#), [2402](#)
- designating class, *see* class, designating
- designator*, [249](#), [2402](#)
- destringization, [546](#)
- destroying operator delete, *see* operator delete,
 - destroying
- destruction, [369–371](#)
 - constant, [190](#)
 - dynamic cast and, [371](#)
 - member access, [369](#)
 - pointer to member or base, [370](#)
 - typeid** operator, [371](#)
 - virtual function call, [370](#)
- destructor, [331](#), [555](#)
 - address of, [331](#)
 - default, [331](#)
 - exception handling, *see* exception handling,
 - constructors and destructors
 - explicit call, [332](#)
 - implicit call, [332](#)
 - non-trivial, [331](#)
 - program termination and, [332](#)
 - prospective, [331](#)
 - pure virtual, [332](#)
 - selected, [331](#)
 - union**, [341](#)
 - virtual, [332](#)
- destructuring expansion statement, *see* expansion
 - statement, destructuring
- diagnosable rules, [10](#)
- diagnostic message, *see* message, diagnostic
- diagonal, [1850](#)
- difference type, [1200](#)
- digit*, [20](#), [2388](#)
- digit-sequence*, [26](#), [2390](#)
- digraph, *see* token, alternative, [20](#)
- direct base class, *see* base class, direct
- direct base class relationship, *see* relationship,
 - direct base class
- direct member, *see* member, direct
- direct-initialization, [251](#)
- direct-list-initialization, [262](#)
- direct-non-list-initialization, [262](#)
- directed acyclic graph, *see* DAG
- directive, preprocessing, *see* preprocessing
 - directive
- directive-introducing token, *see* token,
 - directive-introducing
- directory, [2129](#)
- directory-separator*, [2138](#)
- discarded
 - declaration, [308](#)

- discarded statement, [194](#)
- discarded-value expression, [109](#)
- disengaged, [2186](#)
- disjunction, [433](#)
- disposition, [2297](#)
- disqualifying declaration, *see* declaration,
 - disqualifying
- disqualifying parameter, *see* parameter,
 - disqualifying
- distribution, *see* random number distribution
- do**
 - keyword, [21](#), [196](#), [197](#), [2397](#)
- dogs
 - obliviousness to interference, [607](#)
- domain error, [1831](#)
- dominance
 - virtual base class, [49](#)
- dot
 - filename, [2138](#)
- dot operator, *see* operator, class member access
- dot-dot
 - filename, [2138](#)
- double**
 - keyword, [21](#), [26](#), [69](#), [84](#), [90](#), [113](#), [218](#), [1701](#), [1702](#), [2399](#)
- dynamic binding, *see* function, virtual
- dynamic extent, *see* extent, dynamic
- dynamic initialization, *see* initialization, dynamic
- dynamic type, *see* type, dynamic
- dynamic_cast**, *see* cast, dynamic
 - keyword, [21](#), [38](#), [70](#), [71](#), [78](#), [139](#), [145](#), [146](#), [165](#), [180](#), [366](#), [367](#), [371](#), [424](#), [465](#), [474](#), [475](#), [515](#), [520](#), [608](#), [615](#), [634](#), [2394](#), [2471](#), [2478](#)

E

- E (complete elliptic integrals), [1832](#)
- E (incomplete elliptic integrals), [1834](#)
- ECMA-262, [1713](#)
- ECMAScript, [1720](#), [1745](#)
- Ei (exponential integrals), [1835](#)
- elaborated type specifier, *see* class name,
 - elaborated
- elaborated-type-specifier*, [220](#), [2400](#)
- element
 - associated, [253](#)
- element access functions, [1428](#)
- element block, [1012](#)
 - bounds, [1012](#)
- element type, [237](#), *see* type, element
- element-wise operation, *see* operation,
 - element-wise
- elements
 - selected, [1891](#)
- elif-group*, [523](#), [2409](#)
- elif-groups*, [523](#), [2409](#)
- eligible
 - for subsumption, [439](#)

- eligible special member function, *see* special member function, eligible
- eligible to be unblocked, 2210
- elision
 - copy, *see* constructor, copy, elision
 - copy constructor, *see* constructor, copy, elision
 - move constructor, *see* constructor, move, elision
- ellipsis
 - conversion sequence, 142, 399
 - overload resolution and, 392
- elliptic integrals
 - complete Π , 1833
 - complete E, 1832
 - complete K, 1832
 - incomplete Π , 1834
 - incomplete E, 1834
 - incomplete F, 1834
- else
 - keyword, 21, 194, 195, 2397
- else-group*, 523, 2409
- embed-parameter*, 524, 2410
- embed-parameter-seq*, 524, 2409
- embed-prefixed-parameter*, 524, 2410
- embed-standard-parameter*, 524, 2410
- empty-declaration*, 207, 2398
- enclosed by statement, 192
- enclosing scope, *see* scope, enclosing
- enclosing statement, 192
- enclosing-namespace-specifier*, 280, 2403
- encoded character type, 2130
- encoding
 - literal, 15
 - ordinary literal, 15
 - registered character, 1678
 - alias, 1678
 - wide literal, 15
- encoding-prefix*, 23, 2389
- end-of-file, 916
- endif-line*, 523, 2409
- engaged, 2375
- engine, *see* random number engine
- engine adaptor, *see* random number engine adaptor
- entity, 33
 - associated, 52
 - belong, 41
 - freestanding item, 556
 - implicitly movable, 120
 - local, 33
 - templated, 417
 - underlying, 33
 - user-declared, 319
- enum, 87
 - keyword, 21, 59, 220, 221, 276–279, 2275, 2400, 2402, 2403, 2451
 - type of, 276, 277
 - underlying type, *see* type, underlying
- enum name
 - typedef**, 213
 - enum-base*, 276, 2402
 - enum-head*, 276, 2402
 - enum-head-name*, 276, 2402
 - enum-key*, 276, 2402
 - enum-name*, 276, 2402
 - enum-specifier*, 276, 2402
- enumerated element, 552
- enumerated type, *see* type, enumerated
- enumerating expansion statement, *see* expansion statement, enumerating
- enumeration, 276
 - linkage of, 60
 - scoped, 277
 - unscoped, 277
 - using declaration, 279
- enumeration scope, *see* scope, enumeration
- enumeration type
 - conversion to, 148
 - static_cast**
 - conversion to, 148
- enumerator
 - definition, 35
 - scoped, 277
 - unscoped, 277
 - value of, 277
- enumerator*, 276, 2403
- enumerator-definition*, 276, 2403
- enumerator-list*, 276, 2403
- environment, 2297
 - program, 98
- epoch, 1955
 - protection, 2291
- equality operator function, *see* operator function, equality
- equality-expression*, 170, 2395
- equivalence
 - template type, 440
 - type, 212, 316
- equivalent
 - expressions, 457
 - template-heads*, 458
 - template-parameters*, 458
- equivalent-key group, 985
- equivalently-valued, 570
- erroneous value, *see* value, erroneous
- error completion, *see* completion, error
- escape character, *see* backslash character
- escape sequence
 - format string, 1687
- escape-sequence*, 24, 2389
- escaped, 1699
- estimated field width, *see* field width, estimated
- Eulerian integral of the first kind, *see* beta functions B
- evaluate in sequence, 104
- evaluation, 92
 - order of argument, 141

- point of evaluation, during, 189
- signal-safe, 634
- unspecified order of, 92, 100
- unspecified order of argument, 141
- unspecified order of function call, 141
- evaluation context, *see* context, evaluation
- evaluation semantics, 102, *see* contract evaluation semantics
- exception
 - arithmetic, 105
 - undefined arithmetic, 105
- exception handling, 514–522
 - constructors and destructors, 517
 - currently handled exception, 519
 - exception object, 516
 - constructor, 516
 - destructor, 516
 - function try block, 515
 - `goto`, 514
 - handler, 514, 516, 518–519, 578
 - active, 519
 - array in, 518
 - incomplete type in, 518
 - match, 518–519
 - pointer to function in, 518
 - rvalue reference in, 518
 - memory, 516
 - nearest handler, 516
 - rethrow, 175, 516
 - rethrowing, 516
 - `switch`, 514
 - `terminate` called, 175, 516, 518
 - throwing, 175, 515, 516
 - try block, 514
 - uncaught, 516
- exception object, *see* exception handling, exception object
- exception specification, 519–521
 - `noexcept`
 - constant expression and, 519
 - non-throwing, 519
 - potentially-throwing, 519
 - virtual function and, 520
- exception-declaration*, 514, 2408
- exclusive-or-expression*, 172, 2395
- execution agent, 2181
- execution character set, *see* character set, execution
- execution policy, 1431
- execution policy template parameter, *see* template parameter, execution policy
- execution resource, *see* resource, execution
- execution step, 97
- execution wide-character set, *see* wide-character set, execution
- `exit`, 98, 100, 201
- expansion statement
 - destructuring, 199
 - enumerating, 199
 - iterating, 199
 - expansion-init-list*, 199, 2397
 - expansion-initializer*, 199, 2397
 - expansion-iterable, 199
 - expansion-statement*, 198, 2397
- explicit**
 - keyword, 21, 212, 833, 834, 842, 843, 856, 1142, 1763, 2399, 2426, 2442
- explicit object member function, *see* member function, explicit object
- explicit object parameter, *see* parameter, explicit object
- explicit type conversion, *see* casting
- explicit-instantiation*, 486, 2408
- explicit-object-parameter-declaration, 239
- explicit-specialization*, 487, 2408
- explicit-specifier*, 212, 2399
- explicitly captured, 129
- explicitly initialized elements
 - aggregate, 253
- exponent-part*, 26, 2390
- exponential integrals Ei, 1835
- export**
 - keyword, 21, 303, 524, 534, 535, 2404, 2410, 2439
- export-declaration*, 303, 2404
- export-keyword*, 18
- exposure, 63
- expr-or-braced-init-list*, 249, 2402
- expression, 105–191
 - additive operators, 168
 - `alignof`, 155
 - arithmetic, 105
 - assignment and compound assignment, 175
 - `await`, 153
 - bitwise AND, 172
 - bitwise exclusive OR, 172
 - bitwise inclusive OR, 172
 - cast, 143, 165–166
 - class member access, 143
 - comma, 176
 - conditional operator, 173
 - const cast, 150
 - constant, 177, 183
 - controlling, 196
 - converted constant, 183
 - core constant, 177
 - decrement, 145, 153
 - `delete`, 161
 - destructor call, 122
 - domain, 637
 - dynamic cast, 145
 - equality operators, 170
 - equality-preserving, 637
 - equivalent, *see* equivalent, expressions
 - fold, 134
 - function call, 140
 - functionally equivalent, *see* functionally equivalent, expressions

- immediate-escalating, 186
- increment, 145, 153
- integral constant, 183
- lambda, 123–134
- left-shift-operator, 169
- logical AND, 172
- logical OR, 173
- multiplicative operators, 167
- new**, 155
- noexcept**, 155
- order of evaluation of, 106
- parenthesized, 116
- pointer-to-member, 166
- pointer-to-member constant, 152
- postfix, 139–151
- potentially constant evaluated, 190
- potentially evaluated, 35
- primary, 115–137
- pseudo-destructor call, 122
- reference, 108
- reinterpret cast, 149
- relational operators, 170
- requires, 134–137
- right-shift-operator, 169
- rvalue reference, 107
- sizeof**, 154
- spaceship, 169
- static cast, 147
- subscript, 139
- three-way comparison, 169
- throw**, 175
- type identification, 146
- type-dependent, 469
- unary, 151–155
- unary operator, 151
- value-dependent, 469
- yield, 174
- expression*, 176, 2396
- expression-equivalent, 5
- expression-list*, 139, 2394
- expression-statement*, 193, 2397
- extend, *see* namespace, extend
- extended alignment, *see* alignment, extended
- extended integer type, *see* type, extended integer
- extended signed integer type, *see* type, extended signed integer
- extended unsigned integer type, *see* type, extended unsigned integer
- extent
 - dynamic, 1147
 - static, 1147
- extern**, 210
 - keyword, 21, 33, 44, 60, 74, 210, 211, 232, 233, 235, 260, 290, 292, 320, 486, 2399, 2403, 2408, 2445, 2448
 - linkage of, 210
- extern "C"**, 561, 573
- extern "C++"**, 561, 573
- extern template**, *see* instantiation, explicit
- external linkage, *see* linkage, external
- F**
 - F (incomplete elliptic integrals), 1834
 - facet, 1641
 - fallback-separator*, 2138
 - fallthrough statement, *see* statement, fallthrough
 - false**
 - keyword, 21, 29, 83, 102, 113, 114, 135, 136, 152–154, 170–173, 617, 2391
 - feature-test macro, *see* macro, feature-test
 - field width
 - estimated, 1689
 - minimum, 1689
 - field width units, *see* units, field width
 - file, 2129
 - file attributes, 2156
 - cached, 2156
 - file system, 2129
 - file system race, 2130
 - file, source, *see* source file
 - filename, 2138
 - filename*, 2138
 - fill character, *see* character, fill
 - final**, 21
 - keyword, 21, 314, 318, 348, 2405
 - final await expression, *see* await expression, final
 - final overrider, 347
 - final suspend point, *see* suspend point, final
 - finished, 2361
 - finishing, 2361
 - finite state machine, 5
 - first declaration, *see* declaration, first
 - float**
 - keyword, 21, 26, 84, 90, 113, 218, 1701, 2399
 - floating-point conversion rank, *see* conversion rank, floating-point
 - floating-point literal, *see* literal, floating-point
 - floating-point promotion, 113
 - floating-point type, *see* type, floating-point
 - floating-point-literal*, 25, 2390
 - floating-point-suffix*, 26, 2390
 - fold
 - binary, 134
 - unary, 134
 - fold expanded constraint, *see* constraint, fold expanded
 - fold-expression*, 134, 2393
 - fold-operator*, 134, 2393
 - follow
 - program point, 47
 - for**
 - keyword, 21, 44, 79, 192, 196–198, 2163, 2397, 2419
 - for-range-declaration*, 192, 2396
 - for-range-initializer*, 192, 2396
 - format specification
 - format string, 1688

- format specifier, 5
- format string, 1687
- Formatter*, 1696
- formatter**
 - debug-enabled specialization of, 1698
- formatting locale, *see* locale, formatting
- forward**, 825
- forward progress guarantees
 - concurrent, 97
 - delegation of, 97
 - parallel, 97
 - weakly parallel, 97
- forwarding reference, 500
- fractional-constant*, 26, 2390
- free store, *see also delete*, *see also new*, 338
- freestanding deleted function, *see* function,
 - freestanding deleted
- freestanding implementation, *see* implementation,
 - freestanding
- freestanding item, 555
- friend
 - virtual** and, 349
 - access specifier and, 358
 - class access and, 357
 - inheritance and, 358
 - local class and, 359
 - template and, 450
- friend**
 - keyword, 21, 209, 213, 233, 318, 324, 331, 374, 417, 2398, 2405
- friend function
 - access and, 356
 - inline, 358
 - linkage of, 61
 - member function and, 356
 - nested class, 341
- friend-type-declaration*, 318, 2405
- friend-type-specifier*, 319, 2406
- friend-type-specifier-list*, 318, 2406
- full-expression, 91
- function, *see also* friend function, *see also* inline
 - function, *see also* member function, *see also* virtual function
 - addressable, 572
 - allocation, 75, 157
 - completion, 2297
 - conversion, 334, 335
 - deallocation, 76
 - deferred, 2284
 - definition, 35
 - deleted, 270
 - freestanding deleted, 555
 - global, 573, 576
 - handler, 5
 - handler of type, 518
 - immediate, 186
 - immediate-escalating, 186
 - inline, 216
 - linkage specification overloaded, 291
 - modifier, 6
 - named by expression or conversion, 36
 - needed for constant evaluation, 190
 - non-template, 242
 - observer, 6
 - operator, 407
 - template, 407
 - overload resolution and, 378
 - overloaded, *see* overloading
 - parameter of type, 239
 - pointer to member, 167
 - program semantics affected by the existence
 - of a function definition, 483
 - replaceable, 274
 - replacement, 7, 274
 - reserved, 7
 - template parameter of type, 421
 - templated, 418
 - vararg, 239
 - viable, 377
 - virtual, 347–350
 - override, 347
 - pure, 351
 - virtual function call, 140
 - virtual member, 574
 - waiting, 2276
- function argument, *see* argument
- function call, 142
 - recursive, 142
 - undefined, 149
- function call operator
 - overloaded, 409
- function call operator function, *see* operator
 - function, function call
- function contract assertion, *see* contract assertion,
 - function
- function object, 916
 - binders, 934–936
 - mem_fn**, 936
 - reference_wrapper**, 921
 - type, 916
 - wrapper, 936–949
- function parameter, *see* parameter
- function parameter pack, 446
- function parameter scope, *see* scope, function
 - parameter
- function pointer type, 87
- function return, *see return*
- function return type, *see* return type
- function try block, *see* exception handling,
 - function try block
- function type, *see* type, function
 - function-body*, 268, 2402
 - function-contract-specifier*, 246, 2401
 - function-contract-specifier-seq*, 246, 2401
 - function-definition*, 268, 2402
- function-like macro, *see* macro, function-like
- function-local predefined variable, *see* variable,
 - function-local predefined

function-specifier, 212, 2399
function-try-block, 514, 2408
function_ref::is-convertible-from-specialization, 947
function_ref::is-invokable-using, 947
 functionally equivalent
 expressions, 457
 template-heads, 458
 functions
 candidate, 477
 fundamental alignment, *see* alignment, fundamental
 fundamental type, 86
 destructor and, 333
 fundamental type conversion, *see* conversion, user-defined
 future
 shared state, 2276

G

generated destructor, *see* destructor, default
 generic lambda, 124
 generic parameter type placeholder, 222
 global module, *see* module, global
 global module fragment, 307
 global namespace, *see* namespace, global
 global scope, *see* scope, global
global-module-fragment, 307, 2405
 glvalue, 106
 goto
 and handler, 514
 and try block, 514
 initialization and, 203
 keyword, 21, 32, 180, 197, 201, 202, 514, 2397
 grammar, 2386
 regular expression, 1745
group, 523, 2409
group-part, 523, 2409

H

H_n (Hermite polynomials), 1835
h-char, 19, 2387
h-char-sequence, 19, 2387
h-pp-tokens, 526, 2410
h-preprocessing-token, 526, 2410
 handle
 native, 2112
 handler, *see* exception handling, handler
handler, 514, 2408
handler-seq, 514, 2408
 happens after, 94
 happens before, 94
 hard link, 2129
 hardened implementation, *see* implementation, hardened
has-attribute-expression, 526, 2410
has-embed-expression, 526, 2410
has-include-expression, 526, 2410

hash
 instantiation restrictions, 952
 hash code, 985
 hash function, 984
 hash tables, *see* unordered associative containers
 hazard pointer, 2291
 associated, 2291
 unassociated, 2291
 hazard-protectable, 2291
 header, 557
 C, 573, 576, 635
 C library, 561
 C++ library, 557
 importable, 306
 name, 18–19
 header search, *see* search, header
 header unit, 306
 preprocessing, 535
header-name, 18, 2387
header-name-tokens, 526, 2410
 headers
 C library, 634
 heap with respect to `comp` and `proj`, 1555
 Hermite polynomials H_n , 1835
hex-quad, 17, 2386
hexadecimal-digit, 22, 2389
hexadecimal-digit-sequence, 22, 2388
hexadecimal-escape-sequence, 24, 2389
hexadecimal-floating-point-literal, 25, 2390
hexadecimal-fractional-constant, 26, 2390
hexadecimal-literal, 22, 2388
hexadecimal-prefix, 22, 2388
 host scope, *see* scope, host
 hosted implementation, *see* implementation, hosted
 hosted library facilities, *see* library facilities, hosted

I

I_ν (Bessel functions), 1833
id-expression, 117, 2391
 identical
 atomic constraints, *see* atomic constraint, identical
 identifier, 20–21, 118, 207
identifier, 20, 2387
identifier-continue, 20, 2388
identifier-list, 524, 2409
identifier-start, 20, 2388
 if
 keyword, 21, 110, 194, 195, 197, 1759, 2397
if-group, 523, 2409
if-section, 523, 2409
 ill-formed program, *see* program, ill-formed
 immediate constant, *see* constant, immediate
 immediate context, *see* context, immediate
 immediate function, *see* function, immediate
 immediate function context, 186

- immediate invocation, [186](#)
- immediate object, *see* object, immediate
- immediate scope, *see* scope, immediate
- immediate subexpression, [91](#)
- immediate-escalating, [186](#)
 - conversion, *see* conversion,
 - immediate-escalating
 - expression, *see* expression,
 - immediate-escalating
 - function, *see* function, immediate-escalating
- implementation
 - freestanding, [10](#), [94](#), [98](#), [544](#), [559–561](#), [2207](#)
 - hardened, [11](#)
 - hosted, [10](#), [544](#), [559](#)
- implementation limits, *see* limits, implementation
- implementation-defined behavior, *see* behavior, implementation-defined
- implementation-dependent, [487](#), [583](#), [2066](#), [2077](#)
- implementation-generated, [34](#)
- implementation-resource-count, [530](#)
- implementation-resource-width, [530](#)
- implicit conversion, *see* conversion, implicit
- implicit conversion sequence, *see* conversion sequence, implicit
- implicit object member function, *see* member function, implicit object
- implicit object parameter, *see* object parameter, implicit
- implicit-lifetime class, *see* class, implicit-lifetime
- implicit-lifetime type, *see* type, implicit-lifetime
- implicitly movable entity, *see* entity, implicitly movable
- implicitly-declared default constructor, *see* constructor, default, [325](#)
- implied object argument, [378](#)
 - implicit conversion sequences, [378](#)
 - non-static member function and, [378](#)
- import, [306](#)
- import, [21](#), [561](#)
 - keyword, [18](#), [21](#), [524](#), [529](#), [535](#), [2410](#)
- import-keyword, [18](#)
- importable C++ library headers, *see* C++ library headers, importable
- importable header, *see* header, importable
- inactive union elemental subobject, *see* subobject, union elemental, inactive
- inclusion
 - conditional, *see* preprocessing directive, conditional inclusion
 - source file, *see* preprocessing directive, source-file inclusion
 - source-file inclusion
- inclusive-or-expression*, [172](#), [2395](#)
- incomplete, [168](#)
- incomplete type, *see* type, incomplete
- incompletely-defined object type, *see* object type, incompletely-defined
- increment operator
 - overloaded, *see* overloading, increment operator
- increment operator function, *see* operator function, increment
- incrementable, [1209](#)
- indeterminate value, *see* value, indeterminate
- indeterminately sequenced, [92](#)
- index
 - multidimensional, [1144](#)
- index space
 - multidimensional, [1144](#)
- indices
 - selected, [1891](#)
- indirect base class, *see* base class, indirect
- indirection, [151](#)
- inheritance, [344](#)
 - using-declaration* and, [285](#)
- init-capture*, [129](#), [2393](#)
- init-capture* pack, [134](#), [447](#)
- init-declarator*, [227](#), [2400](#)
- init-declarator-list*, [227](#), [2400](#)
- init-statement*, [192](#), [2396](#)
- initial await expression, *see* await expression, initial
- initial suspend point, *see* suspend point, initial
- initialization, [99](#), [249–268](#)
 - aggregate, [253](#)
 - array, [253](#)
 - array of class objects, [258](#), [362](#)
 - automatic, [203](#)
 - base class, [363](#), [364](#)
 - by inherited constructor, [367](#)
 - character array, [259](#)
 - class object, *see also* constructor, [253](#), [362–369](#)
 - const**, [217](#), [252](#)
 - const member, [364](#)
 - constant, [99](#)
 - constructor and, [362](#)
 - copy, [250](#)
 - default, [249](#)
 - default constructor and, [362](#)
 - definition and, [208](#)
 - direct, [250](#)
 - dynamic, [99](#)
 - dynamic block-scope, [203](#)
 - dynamic non-block, [99](#)
 - explicit, [362](#)
 - jump past, [203](#)
 - list-initialization, [262–268](#)
 - local **static**, [203](#)
 - local **thread_local**, [203](#)
 - member, [363](#)
 - member function call during, [366](#)
 - member object, [364](#)
 - order of, [99](#), [345](#)
 - order of base class, [365](#)
 - order of member, [365](#)
 - order of virtual base class, [365](#)
 - overloaded assignment and, [362](#)
 - parameter, [141](#)

- reference, 235, 259
- reference member, 364
- static, 99
- static and thread, 99
- static member, 337
- union**, 259
- vacuous, 69
- virtual base class, 328
- zero-initialization, 99, 249
- initializer**
 - base class, 268
 - member, 268
 - pack expansion, 367
 - scope of member, 366
 - temporary and declarator, 78
- initializer*, 249, 2401
- initializer-clause*, 249, 2402
- initializer-list*, 249, 2402
- initializer-list constructor, 262
- initializing declaration, 253
- injected declaration, *see* declaration, injected
- injected-class-name, 314
- inline**, 576
- inline**
 - keyword, 21, 98, 209, 216, 233, 280, 281, 323, 324, 331, 415, 451, 486, 2398, 2403
 - linkage of, 60
- inline function, 216, *see* function, inline
- inline namespace, *see* namespace, inline
- inline namespace set, 281
- inline variable, 216, *see* variable, inline
- inlines the receiver, 2310
- instantiation
 - explicit, 485
 - point of, 476
 - template implicit, 481
- instantiation context, 310
- int**
 - keyword, 21, 25, 82, 83, 87, 89, 98, 107, 109, 112, 113, 157, 218, 486, 1701, 2112, 2399, 2446, 2448, 2452
- integer literal, *see* literal, integer
- integer type, 84
- integer-class type, *see* type, integer-class
- integer-like, 1208
- integer-literal*, 22, 2388
- integer-suffix*, 23, 2389
- integral constant expression, *see* expression,
 - integral constant
- integral promotion, 113
- integral type, 84
 - implementation-defined **sizeof**, 82
- interface dependency, 307
- internal linkage, *see* linkage, internal
- invalid iterator, *see* iterator, invalid
- invalid pointer value, *see* value, invalid pointer
- invocation
 - macro, 538
- invocation sequence, 664
- INVOKE*, 920
- item**
 - definable, 35
 - freestanding, *see* freestanding item
- iterating expansion statement, *see* expansion
 - statement, iterating
- iteration statement
 - trivially empty, 196
- iteration-statement*, 196, 201, 2397
- iterator, 1200
 - constexpr, 1201
 - dereferenceable, 1201
 - invalid, 1201
 - past-the-end, 1201
- J**
 - j_n (spherical Bessel functions), 1836
 - J_ν (Bessel functions), 1833
 - Jessie, 334
 - jump-statement*, 201, 2397
- K**
 - K (complete elliptic integrals), 1832
 - K_ν (Bessel functions), 1833
 - key parameter, *see* parameter, key
 - keyword, 21–22, 2386
 - keyword*, 21, 2388
 - known to be initialized, 517
- L**
 - L_n (Laguerre polynomials), 1835
 - L_n^m (associated Laguerre polynomials), 1831
 - label, 202
 - case, 193, 195, 196
 - default, 193, 195, 196
 - scope of, 193
 - label*, 193, 2396
 - label-seq*, 194, 2397
 - labeled-statement*, 193, 2396
 - Laguerre polynomials
 - L_n , 1835
 - L_n^m , 1831
 - lambda scope, *see* scope, lambda
 - lambda-capture*, 129, 2392
 - lambda-declarator*, 123, 2392
 - lambda-expression*, 123, 2392
 - lambda-introducer*, 123, 218, 2392
 - lambda-specifier*, 123, 2392
 - lambda-specifier-seq*, 123, 2392
 - language linkage, 290
 - lattice, *see* DAG, *see* subobject
 - layout
 - bit-field, 338
 - class object, 321, 345
 - layout mapping, 1150
 - layout mapping policy, 1151
 - layout-compatible
 - class, 322

- enumeration, 278
- layout-compatible type, *see* type, layout-compatible
- left shift
 - undefined, 169
- left shift operator, *see* operator, left shift
- left-pad, 1703
- Legendre functions Y_ℓ^m , 1836
- Legendre polynomials
 - P_ℓ , 1835
 - P_ℓ^m , 1832
- letter, 553
- lexical conventions, *see* conventions, lexical
- library
 - C standard, 548, 553, 557, 561, 634, 2453
 - C++ standard, 547, 574, 575, 578
- library call
 - non-constant, 6
- library clauses, 12
- library facilities
 - hosted, 561
- lifetime, 69
- lifetime of an asynchronous operation, 2297
- limit-value, 530
- limits
 - implementation, 5
- line number, 542
- line splicing, 13
- line-directive*, 542, 2410
- line-directives*, 523, 2409
- linear algebra value type, *see* value type, linear algebra
- link, 2129
- linkage, 32, 33, 60–62
 - const** and, 60
 - external, 60, 561, 573
 - implementation-defined object, 292
 - inline** and, 60
 - internal, 60
 - module, 60
 - no, 60, 61
 - static** and, 60
- linkage specification, *see* specification, linkage
- linkage-specification*, 290, 2403
- linking, 15, 60
- list-initialization, 262
- literal, 22–31, 115
 - base of integer, 23
 - boolean, 29
 - char16_t**, 25
 - char32_t**, 25
 - char8_t**, 25
 - character, 23, 25
 - ordinary, 25
 - UTF-16, 25
 - UTF-32, 25
 - UTF-8, 25
 - wide, 25
 - complex, 1759
 - constant, 22
 - float**, 26
 - floating-point, 25, 26
 - integer, 22, 23
 - long**, 23
 - long double**, 26
 - multicharacter, 25
 - narrow-character, 28
 - operator, 414
 - raw, 415
 - template, 414
 - template numeric, 415
 - template string, 415
 - pointer, 29
 - string, 27
 - char16_t**, 27
 - char32_t**, 27
 - narrow, 28
 - ordinary, 28
 - raw, 18, 27
 - undefined change to, 29
 - UTF-16, 28
 - UTF-32, 28
 - UTF-8, 28
 - wide, 28
 - suffix identifier, 414
 - type of character, 25
 - type of floating-point, 26
 - type of integer, 23
 - unsigned**, 23
 - user-defined, 30
- literal*, 22, 2388
- literal encoding, *see* encoding, literal
- literal type, *see* type, literal
- literal-operator-id*, 414, 2406
- living dead
 - name of, 572
- local class, *see* class, local
- friend, 359
- member function in, 323
- local entity, *see* entity, local
- local scope, *see* scope, block
- local variable
 - destruction of, 201
- locale, 1713, 1715, 1720
 - formatting, 2023
- locale-specific, 553
- locale-specific behavior, *see* behavior, locale-specific
- locale-specific form
 - format string, 1691
- lock
 - non-shared, 2182
 - shared, 2182
- lock-free execution, 96
- locus, 43–44
- logical-and-expression*, 172, 2395
- logical-or-expression*, 173, 2396
- long**

keyword, 21, 26, 69, 82–84, 89, 90, 112, 113, 218, 1701, 2399
 typedef and, 210
long-long-suffix, 23, 2389
long-suffix, 23, 2389
 lookup
 ambiguous, 46
 argument-dependent, 51
 class member, 56
 elaborated type specifier, 59
 member name, 47
 name, 46–59
 namespace aliases and, 59
 qualified name, 54–59
 type-only, 47
 unqualified name, 50
 using-directives and, 59
 lookup context, 55
 lookup set, 47
 loop
 trivial infinite, 196
 lower triangle, *see* triangle, lower
 lowercase, 553
lparen, 524, 2409
 lvalue, 106, 2447
 lvalue reference, 234

M

macro
 active, 536
 argument substitution, 538
 definition, 536
 feature-test, 544, 583
 freestanding item, 556
 function-like, 537, 538
 arguments, 538
 import, 535–537
 masking, 576
 name, 537
 object-like, 537
 point of definition, 536
 point of import, 535
 point of undefinition, 536
 pragma operator, 546
 predefined, 543
 replacement, 537–542
 replacement list, 537
 rescanning and replacement, 541
 scope of definition, 542
 main function, 98
 implementation-defined linkage of, 98
 implementation-defined parameters to, 98
 parameters to, 98
 return from, 98, 100
 make progress
 thread, 97
make-unsigned-like-t, 1268
 manifestly constant-evaluated, 190

matched, 5
 mathematical special functions, 1831–1836
mem-initializer, 363, 2406
mem-initializer-id, 363, 2406
mem-initializer-list, 363, 2406
 member
 anonymous union, 342
 class **static**, 74
 default initializer, 320
 direct, 319
 namespace, 279
 non-static, 319
 static, 319, 336
 template and **static**, 444
 member access operator
 overloaded, 410
 member candidate, 383
 member data
 static, 337
 member function, 319
 class, 323
 const, 323
 const volatile, 323
 constructor and, 325
 destructor and, 332
 explicit object, 240
 friend, 358
 implicit object, 240
 inline, 323
 local class, 343
 nested class, 361
 non-static, 319, 323
 overload resolution and, 378
 static, 319, 337
 union, 341
 volatile, 323
 member of the current instantiation, *see* current instantiation, member of the
 member pointer to, *see* pointer to member
 member subobject, 65
member-declaration, 318, 2405
member-declarator, 318, 2405
member-declarator-list, 318, 2405
 member-qualified name, *see* name, member-qualified
member-specification, 318, 2405
 memory location, 64
 memory management, *see* delete, *see* new
 memory model, 64–65
 message
 diagnostic, 4, 10
 minimum field width, *see* field width, minimum
 model
 concept, 575
 modifiable, 108
 modification order, 94
 modify-write operation
 atomic, 2209
 module, 303

- exported, 307
- global, 303
- named, 302
- reserved name of, 302
- module, 21
 - keyword, 18, 21, 523, 524, 534, 2408–2410
- module implementation unit, 302
- module interface unit, 302
- module linkage, *see* linkage, module
- module partition, 302
- module unit, 302
- module unit purview, *see* purview, module unit
- module-declaration*, 302, 2404
- module-file*, 523, 2408
- module-import-declaration*, 306, 2404
- module-keyword*, 18
- module-name*, 302, 2404
- module-name-qualifier*, 302, 2404
- module-partition*, 302, 2404
- more constrained, 439
- more cv-qualified, 89
- more partial-ordering-constrained, 440
- more specialized, 454, 504
 - class template, 454
 - function template, 504
- most derived class, 66
- most derived object, 66
 - bit-field, 66
 - zero size subobject, 66
- move
 - class object, *see* constructor, move, *see* assignment operator, move
- move, 826
- move-eligible, 120
- move_only_function::is-callable-from*, 941
- multi-pass guarantee, 1212, 1216
- multibyte character, *see* character, multibyte
- multicharacter literal, *see* literal, multicharacter
- multidimensional index, *see* index,
 - multidimensional
- multidimensional index space, *see* index space,
 - multidimensional
- multiple inheritance, 344, 345
 - virtual and, 349
- multiple threads, *see* threads, multiple
- multiplicative-expression*, 167, 2395
- mutable, 210
 - keyword, 21, 118, 123, 125, 133, 144, 152, 167, 210–212, 217, 218, 233, 337, 2392, 2399
- mutable iterator, 1200
- mutex types, 2245

N

- n_n (spherical Neumann functions), 1836
- N_ν (Neumann functions), 1834
- n-char*, 15, 2386
- n-char-sequence*, 15, 2386
- name, 20, 32, 62, 117

- address of cv-qualified, 152
- bound, 41
- dependent, 469
- elaborated
 - enum, 220
- length of, 20
- macro, *see* macro, name
- member-qualified, 54
- predefined macro, *see* macro, predefined
- qualified, 54
- reserved, 572
- same, 32
- terminal, 118
- unqualified, 50
- zombie, 572
- name class, *see* class name
- name hiding, 43, 121, 203
 - using-declaration and, 288
- name-declaration*, 206, 2397
- named module, *see* module, named
- named-namespace-definition*, 280, 2403
- named-universal-character*, 15, 2386
- namespace, 557
 - alias, 282
 - definition, 280
 - extend, 280
 - global, 21, 280
 - inline, 280
 - unnamed, 281
- namespace
 - keyword, 21, 280–282, 2403
- namespace alias, *see* alias, namespace
- namespace scope, *see* scope, namespace
- namespace-alias*, 282, 2403
- namespace-alias-definition*, 282, 2403
- namespace-body*, 280, 2403
- namespace-definition*, 280, 2403
- namespace-name*, 280, 2403
- namespaces, 279–284
- NaN, 1831
- narrow character type, *see* type, narrow character
- narrowing conversion, *see* conversion, narrowing
- native encoding, 2139
- native handle, *see* handle, native
- native pathname format, 2135
- NDEBUG, 561
- necessarily reachable, *see* reachable, necessarily
- needed
 - exception specification, 521
 - function contract assertion, 247
- needed for constant evaluation, 190
- nested class, *see* class, nested
 - local class, 343
- nested within, 66
- nested-name-specifier*, 121, 2392
- nested-namespace-definition*, 280, 2403
- nested-requirement*, 137, 2393
- Neumann functions
 - N_ν , 1834

n_n, 1836

new, 75, 155, 157

- array of class objects and, 159
- constructor and, 159
- default constructor and, 159
- exception and, 159
- initialization and, 159
- keyword, 21, 68, 75, 76, 156, 157, 159, 161, 184, 408, 474, 521, 603, 634, 707, 2291, 2394, 2406
- scoping and, 157
- storage allocation, 156
- type of, 338
- unspecified constructor and, 159
- unspecified order of evaluation, 159

new-declarator, 156, 2394

new-expression, 156, 2394

- placement, 158

new-extended alignment, *see* alignment,

- new-extended

new-initializer, 156, 2394

new-line, 524, 2410

new-placement, 156, 2394

new-type-id, 156, 2394

new_handler, 76

no linkage, *see* linkage, no

node handle, 973

nodeclspec-function-declaration, 206, 2398

nodiscard call, *see* call, nodiscard

nodiscard type, *see* type, nodiscard

noexcept, 155

- keyword, 21, 84, 91, 108, 114, 127, 155, 238, 474, 507, 509, 510, 519, 521, 578, 721, 871, 883, 952, 1291, 2031, 2032, 2394, 2408, 2429, 2437, 2441

noexcept-expression, 155, 2394

noexcept-specifier, 519, 2408

nofun-declarator, 229, 2400

nofun-type-id, 229, 2400

nominable, 43

non-dependent sender, *see* sender, non-dependent

non-initialization odr-use, *see* odr-use,

- non-initialization

non-member candidate, 383

non-object parameter, *see* parameter, non-object

non-object-parameter-type-list, 240

non-shared lock, *see* lock, non-shared

non-static data member, *see* data member,

- non-static

non-static member, *see* member, non-static

non-static member function, *see* member function,

- non-static

non-template function, *see* function, non-template

non-throwing exception specification, 519

non-virtual base class, *see* base class, non-virtual

nondigit, 20, 2388

nonzero-digit, 22, 2388

noptr-abstract-declarator, 229, 2401

noptr-abstract-pack-declarator, 230, 2401

noptr-declarator, 229, 2400

noptr-new-declarator, 156, 2394

noptr-nofun-declarator, 229, 2401

normal distributions, 1794–1800

normal form

- constraint, 437
- path, 2139

normalization

- constraint, *see* constraint, normalization
- path, *see* path, normalization

normative references, *see* references, normative

not

- keyword, 19, 20, 22, 2387, 2454

not_eq

- keyword, 19, 20, 22, 2387, 2454

notation

- syntax, 12

NTBS, 554

- empty, 554
- length, 554
- static, 554
- value, 554

NTCTS, 6

NTMBS, 554

- static, 554

null character, *see* character, null

null member pointer conversion, *see* conversion,

- null member pointer

null pointer conversion, *see* conversion, null

- pointer

null pointer value, *see* value, null pointer

null reflection, *see* reflection, null

null statement, *see* statement, null

null wide character, *see* wide-character, null

nullopt

- keyword, 1041

nullptr

- keyword, 21, 29, 272, 273, 581, 602, 716, 745, 880, 888, 974, 1252, 1254, 2049, 2061, 2256, 2259, 2260, 2391, 2437

number

- preprocessing, 19
- subnormal, 591, 593, 2457

numeric type, *see* type, numeric

numeric-escape-sequence, 24, 2389

numeric_limits, 589

O

object, *see also* object model, 32, 65

- acquiring a value representation, 82
- byte copying and, 82
- callable, 920
- complete, 65
- const, 89
- const volatile, 89
- customization point, 554
- data-parallel, 1891
- definition, 35

- destructor and placement of, 333
- destructor static, 100
- exception, *see* exception handling, exception
 - object
- immediate, 182
- implicit creation, 68
- linkage specification, 292
- local **static**, 74
- nested within, 66
- nonzero size, 66
- potentially non-unique, 66
- providing storage for, 65
- queryable, 2296
- range adaptor, 1299
- range adaptor closure, 1298
- reified, 1268
- suitable created, 68
- target, 920
- temporary, 76
- volatile, 89
- zero size, 66
- object class, *see* class object
- object expression, 143, 167
- object lifetime, 69–72
- object model, 65–68
- object parameter, *see* parameter, object
 - corresponding, 41
 - implicit, 378
- object pointer type, 87
- object type, 81
 - incompletely-defined, 80
- object-like macro, *see* macro, object-like
- observable behavior, *see* behavior, observable, *see*
 - behavior, observable
- observable checkpoints, *see* checkpoints,
 - observable
- octal-digit, 22, 2388
- octal-escape-sequence, 24, 2389
- octal-literal, 22, 2388
- odr-usable, 37
- odr-use, 36
 - non-initialization, 100
- one-definition rule, 35–40
- opaque-enum-declaration, 276, 2402
- operating system dependent, 2130
- operation
 - asynchronous, 2296
 - binary element-wise, 1891
 - completion, 2297
 - element-wise, 1891
 - parent, 2297
 - stop request, 2187
 - unary element-wise, 1891
- operation state, *see* state, operation
- operations
 - child, 2297
- operator, 19, 408
 - *=**, 175
 - +=**, 175
 - =**, 175
 - /=**, 175
 - <<=**, 175
 - >>=**, 175
 - %=**, 175
 - &=**, 175
 - ^=**, 175
 - |=**, 175
 - addition, 168
 - additive, 168
 - address-of, 151
 - assignment, 175, 555
 - bitwise, 172
 - bitwise AND, 172
 - bitwise exclusive OR, 172
 - bitwise inclusive OR, 172
 - cast, 151, 229
 - class member access, 143
 - comma, 176
 - comparison
 - secondary, 376
 - conditional expression, 173
 - copy assignment, *see* assignment operator,
 - copy
 - decrement, 145, 151, 153
 - defaulted comparison operator function, 374
 - division, 167
 - equality, 170
 - defaulted, 375
 - deleted, 374
 - function call, 140, 408
 - greater than, 170
 - greater than or equal to, 170
 - implementation, 407
 - increment, 145, 151, 153
 - indirection, 151
 - inequality, 170
 - defaulted, 376
 - left shift, 169
 - less than, 170
 - less than or equal to, 170
 - logical AND, 172
 - logical negation, 151, 152
 - logical OR, 173
 - move assignment, *see* assignment operator,
 - move
 - multiplication, 167
 - multiplicative, 167
 - ones' complement, 151, 152
 - overloaded, 105, 407
 - pointer to member, 166
 - pragma, *see* macro, pragma operator
 - precedence of, 106
 - reflection, 163
 - relational, 170
 - defaulted, 376
 - remainder, 167
 - right shift, 169
 - scope resolution, 54, 121, 157, 323, 344, 350

- side effects and comma, 176
 - side effects and logical AND, 173
 - side effects and logical OR, 173
 - sizeof**, 151, 154
 - spaceship, 169
 - subscripting, 139, 408
 - subtraction, 168
 - three-way comparison, 169
 - defaulted, 375
 - deleted, 374
 - unary, 151
 - unary minus, 151, 152
 - unary plus, 151, 152
 - operator*, 408, 2406
 - operator**
 - keyword, 21, 68, 75, 76, 80, 152, 153, 157, 162, 308, 334, 379, 381–385, 391, 394, 397, 403, 407–411, 414, 469, 471, 2291, 2406
 - operator delete**
 - destroying, 76
 - operator delete**, *see also delete*, 157, 162
 - operator function**, *see function*, *operator*
 - binary, 409
 - class member access, 410
 - comparison, 409
 - decrement, 411
 - equality, 409
 - function call, 409
 - increment, 410
 - prefix unary, 408
 - relational, 409
 - simple assignment, 409
 - subscripting, 410
 - three-way comparison, 409
 - operator new**, *see also new*, 157
 - operator overloading**, *see overloading*, *operator*
 - operator use**
 - scope resolution, 337
 - operator-function-id*, 407, 2406
 - operator-or-punctuator*, 19, 2387
 - operators**
 - built-in, 105
 - optimization of temporary, *see temporary*,
 - elimination of
 - optional object, 852
 - or**
 - keyword, 19, 20, 22, 2387, 2454
 - or_eq**
 - keyword, 19, 20, 22, 2387, 2454
 - order of evaluation in expression, *see expression*,
 - order of evaluation of
 - order of execution
 - base class constructor, 326
 - base class destructor, 332
 - constructor and array, 362
 - constructor and static data members, 362
 - destructor, 332
 - destructor and array, 332
 - member constructor, 326
 - member destructor, 332
 - ordering
 - function template partial, *see template*,
 - function, partial ordering
 - ordinary character type, *see type*, *ordinary character*
 - ordinary literal encoding, *see encoding*, *ordinary literal*
 - over-aligned type, *see type*, *over-aligned*
 - overflow, 106
 - undefined, 105
 - overload resolution, 377
 - overload set, 46
 - overloaded function, *see overloading*
 - address of, 152, 406
 - overloaded operator, *see overloading*, *operator inheritance of*, 408
 - overloading, 241, 317, 377–414, 456
 - address of overloaded function, 406
 - argument lists, 378–392
 - assignment operator, 409
 - binary operator, 409
 - built-in operators and, 412
 - candidate functions, 378–392
 - decrement operator, 411
 - example of, 377
 - function call operator, 409
 - increment operator, 410
 - member access operator, 410
 - operator, 407–411
 - resolution, 377–406
 - best viable function, 393–408
 - better viable function, 393
 - contexts, 377
 - function call syntax, 380–382
 - function template, 512
 - implicit conversions and, 396–406
 - initialization, 385, 386
 - operators, 382
 - template, 458
 - viable functions, 392–408
 - subscripting operator, 410
 - unary operator, 408
 - user-defined literal, 414
 - using directive and, 284
 - using-declaration and, 289
- overloads
 - floating-point, 1759
- override, *see function*, *virtual*, *override*
- override**, 21
 - keyword, 21, 318, 348, 2405
- overrider
 - final, 347
- own, 697
- owning
 - hazard pointer, 2293

P

P_ℓ (Legendre polynomials), 1835

P_ℓ^m (associated Legendre polynomials), 1832

pack, 447

unexpanded, 448

pack expansion, 447

pattern, 447

pack-index-expression, 122, 2392

pack-index-specifier, 220, 2400

pack-index-template-name, 422, 2407

padding bits, 80

padding width, *see* width, padding

pair

tuple interface to, 832

parallel algorithm, 1428

parallel forward progress guarantees, 97

parameter, 6

catch clause, 6

disqualifying, 646

explicit object, 240

function, 6

function-like macro, 6

key, 646

macro, 538

non-object, 240

object, 240

reference, 234

scope of, 45

template, 6, 33

void, 239

parameter declaration, 33

parameter list

variable, 142, 239

parameter mapping, 434

parameter-declaration, 238, 2401

parameter-declaration-clause, 238, 2401

parameter-declaration-list, 238, 2401

parameter-type-list, 239

parameterized type, *see* template

parameters-and-qualifiers, 229, 2400

parent directory, 2129

parent operation, *see* operation, parent

parent scope, *see* scope, parent

past-the-end iterator, *see* iterator, past-the-end

path, 2135

absolute, 2135

normalization, 2138–2139

relative, 2135

path equality, 2149

path-format-spec, 2150

pathname, 2135

pathname, 2137

pathname resolution, 2135

pattern, *see* pack expansion, pattern

perfect forwarding call wrapper, *see* call wrapper,
perfect forwarding

period, 554

permissible completion, *see* completion,
permissible

permissible types, *see* types, permissible

phase completion step, 2273

phase synchronization point, *see* barrier, phase
synchronization point

phases of translation, *see* translation, phases

Π (complete elliptic integrals), 1833

Π (incomplete elliptic integrals), 1834

piecewise construction, 834

pipeable sender adaptor closure object, *see* sender
adaptor closure object, pipeable

pipeable sender adaptor object, *see* sender
adaptor object, pipeable

placeholder type deduction, 225

placeholder-type-specifier, 222, 2400

placeholders

freestanding item, 936

placement *new-expression*, *see* *new-expression*,
placement

plain lock-free atomic operation, 634

plain-string-literal, 27, 2390

pm-expression, 166, 2395

POD, 2458

point, 87

of evaluation, 189

program, 47

synthesized, 188

point of

evaluation, 189

macro definition, *see* macro, point of
definition

macro import, *see* macro, point of import

macro undefinition, *see* macro, point of
undefinition

point of evaluation during evaluation, 189

pointer, *see also* **void***

composite pointer type, 108

strict total order, 5

zero, *see* value, null pointer

pointer literal, *see* literal, pointer

pointer past the end of, 87

pointer to, 87

pointer to member, 87, 166, 236

pointer-interconvertible, 88

pointer-literal, 29, 2391

Poisson distributions, 1790–1794

polymorphic class, *see* class, polymorphic

pool resource classes, 741

pools, 741

population, 1530

POSIX, 547

extended regular expressions, 1720

regular expressions, 1720

possibly-reclaimable, 2291

post

keyword, 21

postcondition assertion, *see* assertion,
postcondition

- postcondition-specifier*, 246, 2401
- postfix ++ and --
 - overloading, 410
- postfix ++, 145
- postfix --, 145
- postfix-expression*, 139, 2394
- potential results, 35
- potentially concurrent, 95
- potentially conflict, 42
- potentially constant evaluated, 190
- potentially evaluated, 35
- potentially non-unique object, *see* object,
 - potentially non-unique
- potentially usable in constant expressions, 185
- potentially-constant, 185
- potentially-evaluated subexpression, *see*
 - subexpression, potentially-evaluated
- potentially-overlapping subobject, 66
- potentially-throwing
 - exception specification, 519
 - expression, 520
- pp-balanced-token*, 524, 2410
- pp-balanced-token-seq*, 524, 2410
- pp-global-module-fragment*, 523, 2408
- pp-import*, 535, 2410
- pp-module*, 534, 2410
- pp-module-name*, 534
- pp-module-name-qualifier*, 534
- pp-module-partition*, 534
- pp-number*, 19, 2387
- pp-private-module-fragment*, 523, 2409
- pp-tokens*, 524, 2409
- pre**
 - keyword, 21
- preallocated backend storage for a proxy, 2381
- precede, *see* declaration, precede
- precedence of operator, *see* operator, precedence
 - of
- precondition assertion, *see* assertion, precondition
- precondition-specifier*, 246, 2401
- predicate
 - contract-assertion, 102
- preferred-separator*, 2138
- prefix
 - defined, 11
 - L, 27
 - R, 27
- prefix ++ and --
 - overloading, 410
- prefix ++, 153
- prefix --, 153
- prefix unary operator function, *see* operator
 - function, prefix unary
- preprocessing, 524
- preprocessing directive, 523–546
 - conditional inclusion, 526
 - diagnostic, 543
 - embed a resource, 529
 - error, 543
 - header inclusion, 528
 - import, 535
 - line control, 542
 - macro replacement, *see* macro, replacement
 - module, 534
 - null, 543
 - pragma, 543
 - source-file inclusion, 528
 - warning, 543
- preprocessing translation unit, *see* translation
 - unit, preprocessing
- preprocessing-file*, 523, 2408
- preprocessing-op-or-punc*, 19, 2387
- preprocessing-operator*, 19, 2387
- preprocessing-token*, 17, 2387
- primary equivalence class, 6
- primary module interface unit, 302
- primary template, *see* template, primary
- primary-expression*, 115, 2391
- private**, *see* access control, **private**
 - keyword, 21, 309, 344, 354, 523, 2405, 2406, 2409
- private-module-fragment*, 309, 2405
- produce, 188
- program, 60
 - ill-formed, 5
 - startup, 98–100
 - termination, 100–102
 - well-formed, 9, 11
- program execution, 11–12, 90–102
 - abstract machine, 11
 - as-if rule, *see* as-if rule
- program point, *see* point, program
- program semantics
 - affected by the existence of a variable or
 - function definition, 483
- projection, 7
- promise object, 272
- promise type, *see* coroutine, promise type
- promoted integral type, 412
- promotion
 - bool to int, 113
 - default argument promotion, 142
 - floating-point, 113
 - integral, 112
- property
 - of the implementation, 7
- prospective destructor, *see* destructor, prospective
- protected**, *see* access control, **protected**
 - keyword, 21, 344, 354, 2406
- protection, *see* access control, 577
- protection epoch, *see* epoch, protection
- prototype parameter
 - concept, 463
- provides storage, 65
- proxy, 2381
- prvalue, 106
- pseudo-destructor, 122
- ptr-abstract-declarator*, 229, 2401

ptr-declarator, 228, 2400
ptr-nofun-declarator, 229, 2401
ptr-operator, 229, 2400
ptrdiff_t, 168
public, *see* access control, **public**
 keyword, 21, 344, 354, 2406
 punctuator, 19
pure-specifier, 318, 2405
 purview
 global module, 303
 module unit, 303
 named module, 303

Q

q-char, 19, 2387
q-char-sequence, 19, 2387
 qualification
 explicit, 54
 qualification-combined type, *see* type,
 qualification-combined
 qualification-decomposition, 111
 qualified name, *see* name, qualified
qualified-id, 121, 2392
qualified-namespace-specifier, 282, 2403
 query, 2296
 query object, 2296
 queryable object, *see* object, queryable
 quote resource search, *see* search, quote resource

R

r-char, 27, 2390
r-char-sequence, 27, 2390
 random number distribution
 requirements, 1767–1769
 random number distributions
 Bernoulli, 1786–1789
 normal, 1794–1800
 Poisson, 1790–1794
 sampling, 1800–1805
 uniform, 1784–1786
 random number engine
 requirements, 1765–1766
 with predefined parameters, 1779–1780
 random number engine adaptor
 with predefined parameters, 1779–1780
 random number generation, 1759–1805
 distributions, 1784–1805
 engines, 1769–1779
 predefined engines and adaptors, 1779–1780
 requirements, 1763–1769
 synopsis, 1760–1762
 utilities, 1781–1784
 random number generator, *see* uniform random
 bit generator
 range, 1201
 accessible, 1172
 counted, 1201, 1350
 valid, 1201

 valid, 1201
 range adaptor closure object, *see* object, range
 adaptor closure
 range adaptor object, *see* object, range adaptor
 rank, 1144
 rank index, 1144
 raw string literal, 27
raw-string, 27, 2390
 rcu-protectable, 2288
 reachable
 declaration, 312
 necessarily
 translation unit, 311
 translation unit, 311
 reachable from, 1201
 declaration, 312
 ready, 1731, 2277
 receiver, 2297
 reclaimed, 2291
 redeclaration, 33
ref-qualifier, 229, 2400
 reference, 87
 assignment to, 176
 call by, 142
 forwarding, 500
 lvalue, 87
 null, 235
 rvalue, 87
 sizeof, 155
 reference collapsing, 235
 reference lifetime, 70
 reference-compatible, 260
 reference-related, 260
 references
 normative, 2
reflect-expression, 163, 2395
 reflection, 84
 null, 84
 reflection operator, *see* operator, reflection
reflection-name, 163, 2395
 region of RCU protection, 2288
register
 keyword, 21, 2432, 2448
register storage class, 2432
 registered character encoding, *see* encoding,
 registered character
 registration
 stoppable callback, 2185
 regular expression, 1713–1746
 grammar, 1745
 matched, 5
 requirements, 1713
 regular expression traits, 1745
 requirements, 1713, 1723
 transform_primary, 1714, 1746
 reified object, *see* object, reified
reinterpret_cast, *see* cast, reinterpret
 keyword, 21, 78, 88, 139, 149, 150, 165, 166,
 179, 424, 465, 474, 475, 2394

- relational operator function, *see* operator function, relational
 - relational-expression*, 170, 2395
 - relationship
 - direct base class, 344
 - relative path, *see* path, relative
 - relative-path*, 2138
 - release sequence, 94
 - remainder operator, *see* operator, remainder
 - remote time zone database, 2012
 - replaceable
 - function, *see* function, replaceable
 - replacement
 - macro, *see* macro, replacement
 - replacement body, *see* body, replacement
 - replacement field
 - format string, 1687
 - replacement function, *see* function, replacement
 - replacement-list*, 524, 2409
 - represent, 85
 - representation
 - constexpr-unknown, 177
 - object, 80
 - value, 80
 - represents the address, 87
 - requirement
 - compound, 136
 - nested, 137
 - simple, 136
 - type, 136
 - requirement*, 135, 2393
 - requirement-body*, 135, 2393
 - requirement-parameter-list*, 135, 2393
 - requirement-seq*, 135, 2393
 - requirements, 549
 - container, 960, 985, 996, 998, 1731
 - not required for unordered associated containers, 984
 - iterator, 1200
 - numeric type, 1751
 - random number distribution, 1767–1769
 - random number engine, 1765–1766
 - regular expression traits, 1713, 1723
 - seed sequence, 1763–1764
 - sequence, 1731
 - uniform random bit generator, 1764
 - unordered associative container, 985
 - requires**
 - keyword, 21, 135–137, 416, 2393, 2407
 - requires-clause*, 416, 2407
 - trailing, 228
 - requires-expression*, 135, 2393
 - rescanning and replacement, *see* macro, rescanning and replacement
 - reserved block, *see* block, reserved
 - reserved identifier, 21
 - reset, 697
 - resolution, *see* overloading, resolution
 - resource
 - execution, 2296
 - resource-count, 530
 - resource-offset, 530
 - restriction, 575, 576, 578
 - address of bit-field, 338
 - anonymous **union**, 342
 - bit-field, 338
 - constructor, 325
 - destructor, 331
 - extern**, 210
 - local class, 343
 - operator overloading, 408
 - overloading, 408
 - pointer to bit-field, 338
 - reference, 235
 - static**, 210
 - static member local class, 344
 - union**, 341
 - result
 - async, 2297
 - glvalue, 107
 - prvalue, 107
 - result binding, 248
 - result object, 107
 - result-name-introducer*, 248, 2401
 - rethrow, *see* exception handling, rethrow
 - retired, 2291
 - return**, 201
 - and handler, 514
 - and try block, 514
 - constructor and, 201
 - keyword, 21, 84, 98, 110, 124, 142, 201, 514, 517, 519, 2397
 - reference and, 260
 - return statement**, *see* **return**
 - return type, 241
 - covariant, 348
 - return-type-requirement*, 136, 2393
 - reversible container, *see* container, reversible
 - rewritten candidate, 383
 - right shift operator, *see* operator, right shift
 - root-directory*, 2138
 - root-name*, 2137
 - rounding, 113
 - running, 2361
 - rvalue, 106
 - lvalue conversion to, *see* conversion, lvalue-to-rvalue, 2447
 - rvalue reference, 234
- ## S
- s-char*, 27, 2390
 - s-char-sequence*, 27, 2390
 - sample, 1530
 - sampling distributions, 1800–1805
 - satisfy, *see* constraint, satisfaction
 - sb-identifier*, 206, 2398
 - sb-identifier-list*, 206, 2398

- scalar type, *see* type, scalar
- schedule sender, 2298
- schedule-expression, 2298
- scheduler, 2298
- scope, 1, 32, 40–46, 207
 - anonymous **union** at namespace, 342
 - associated, 2375
 - block, 44
 - class, 46
 - contract-assertion, 46
 - declarations and, *see* locus
 - destructor and exit from, 201
 - enclosing, 40
 - enumeration, 46
 - function parameter, 45
 - function prototype, *see* scope, function parameter
 - global, 40
 - handler*, 44
 - host, 41
 - immediate, 40
 - inhabit, 40
 - intervene, 40
 - iteration-statement*, 196
 - lambda, 45
 - macro definition, *see* macro, scope of definition
 - name lookup and, *see* lookup, name namespace, 45
 - parent, 40
 - search, 47
 - single, 47
 - selection-statement*, 194
 - target, 41
 - template parameter, 46
- scope resolution operator, *see* operator, scope resolution
- scoped enumeration, *see* enumeration, scoped
- search, 47
 - bracket resource, 529
 - header, 528
 - quote resource, 530
 - single, 47
 - source file, 528
- secondary comparison operator, 376
- seed sequence, 1763
 - requirements, 1763–1764
- selected destructor, *see* destructor, selected
- selected elements, *see* elements, selected
- selected indices, *see* indices, selected
- selection-statement*, 194, 2397
- semantics
 - class member, 143
- sender, 2297
 - non-dependent, 2298
- sender adaptor, 2298
- sender adaptor closure object
 - pipeable, 2328
- sender adaptor object
 - pipeable, 2329
- sender consumer, 2298
- sender factory, 2298
- sentinel, 1201
- separate compilation, *see* compilation, separate
- separate translation, *see* compilation, separate
- separately instantiated construct, *see* construct, separately instantiated
- sequence
 - characteristic, 188
 - sorted
 - with respect to a comparator, 1532
 - with respect to a comparator and projection, 1532
- sequence container, *see* container, sequence
- sequenced after, 92
- sequenced before, 92
- sequencing operator, *see* operator, comma
- shared lock, *see* lock, shared, 2249
- shared mutex types, 2249
- shared state, *see* future, shared state
- shared timed mutex type, 2251
- shift operator
 - left, *see* operator, left shift
 - right, *see* operator, right shift
- shift-expression*, 169, 2395
- short
 - keyword, 21, 82–84, 89, 218, 2399
 - typedef** and, 210
- side effects, 11, 78, 92, 94–96, 193, 365, 371, 538, 577
 - visible, 95
- sign*, 26, 2390
- signal, 93
- signal-safe
 - _Exit**, 599
 - abort**, 599
 - evaluation, *see* evaluation, signal-safe
 - forward**, 825
 - initializer_list** functions, 618
 - memcpy**, 1633
 - memmove**, 1633
 - move**, 825
 - move_if_noexcept**, 825
 - numeric_limits** members, 591
 - quick_exit**, 600
 - signal**, 634
 - type traits, 751
- signature, 7, 8
 - corresponding, 41
- signed
 - keyword, 21, 82–84, 89, 218, 2399
 - typedef** and, 210
- signed integer representation
 - two's complement, 83, 278, 818, 2215, 2227
- signed integer type, *see* type, signed integer
- signed-integer-class type, *see* type, signed-integer-class
- signed-integer-like, 1208

- significand, 26
- similar types, 112
- simple assignment operator function, *see* operator function, simple assignment
- simple call wrapper, 921
- simple-capture*, 129, 2393
- simple-declaration*, 206, 2398
- simple-escape-sequence*, 24, 2389
- simple-escape-sequence-char*, 24, 2389
- simple-hexadecimal-digit-sequence*, 17, 2386
- simple-octal-digit-sequence*, 24, 2389
- simple-requirement*, 136, 2393
- simple-template-id*, 422, 2407
- simple-template-name*, 422, 2408
- simple-type-specifier*, 218, 2399
- single search, *see* search, single
- size of a multidimensional index space, 1144
- size-suffix*, 23, 2389
- size_t**, 155
- sizeof**
 - keyword, 21, 38, 80, 151, 154, 155, 231, 474, 475, 2394, 2447
- skipped, 1551, 1553
- slice type
 - collapsing, 1181
 - unit-stride, 1182
- sliceable layout mapping requirements, 1186
- smart pointers, 706–722
- source file, 13, 561, 574
- source file character, *see* character, source file
- source file search, *see* search, source file
- source line
 - logical, 13
- special member function, *see* constructor, *see* assignment operator, *see* destructor
 - eligible, 324
- special-declaration*, 206, 2398
- specialization, 480
 - class template partial, 451
 - declared, 481
 - program-defined, 6
 - template, 480
 - template explicit, 487
- specification
 - linkage, 290–292
 - extern**, 290
 - implementation-defined, 290
 - nesting, 290
 - template argument, 492
- specifications
 - C standard library exception, 578
 - C++, 578
- specifier, 209–227
 - constexpr**, 213
 - constexpr**, 213
 - function, 214
 - constinit**, 215
 - cv-qualifier, 217
 - declaration, 209
 - explicit**, 212
 - friend**, 213, 577
 - function, 212
 - inline**, 216
 - storage class, 210
 - type, *see* type specifier
 - typedef**, 212
 - virtual**, 212
 - specifier access, *see* access specifier
 - spherical harmonics Y_ℓ^m , 1836
 - splice, 59–60
 - splice-expression*, 137, 2393
 - splice-scope-specifier*, 121, 2392
 - splice-specialization-specifier*, 59, 2391
 - splice-specifier*, 59, 2391
 - splice-type-specifier*, 227, 2400
 - stable algorithm, 8, 577
 - stack unwinding, 517
 - stacktrace, 664
 - stacktrace entry, 664
 - standard
 - structure of, 12
 - standard integer type, *see* type, standard integer
 - standard signed integer type, *see* type, standard signed integer
 - standard unsigned integer type, *see* type, standard unsigned integer
 - standard-layout class, *see* class, standard-layout
 - standard-layout struct, *see* struct, standard-layout
 - standard-layout type, *see* type, standard-layout
 - standard-layout union, *see* union, standard-layout
 - start
 - program, 99
 - starting, 2361
 - startup
 - program, 561
 - state, 885
 - operation, 2297
 - state entity, 920
 - statement, 192–205
 - continue** in **for**, 197
 - break**, 201
 - compound, 193
 - constexpr if, 195
 - continue**, 201
 - control-flow-limited, 193
 - declaration, 203
 - declaration in **switch**, 196
 - declaration in **while**, 197
 - do**, 196, 197
 - empty, 193
 - enclosed by, 192
 - enclosing, 192
 - expansion, 198–201
 - expression, 193
 - fallthrough, 296
 - for**, 196, 197
 - goto**, 193, 201, 202
 - if**, 194

- iteration, 196–198
- jump, 201
- labeled, 193
- null, 193
- range based for, 198
- selection, 194–196
- switch, 194, 195, 201
- while, 196, 197
- statement, 192, 2396
- statement-seq, 193, 2397
- static, 210
 - destruction of local, 204
 - keyword, 21, 60, 74, 98, 123, 125, 210, 216, 233, 240, 319, 320, 337–339, 342, 590, 2221, 2392, 2399, 2448
 - linkage of, 60, 210
- static data member, *see* data member, static
- static extent, *see* extent, static
- static initialization, *see* initialization, static
- static member, *see* member, static
- static member function, *see* member function, static
- static storage duration, *see* storage duration, static
- static type, *see* type, static
- static_assert, 208
 - failed, 208
 - keyword, 21, 207, 2398, 2437
- static_assert-declaration, 207, 2398
- static_assert-message, 206, 2398
- static_cast, *see* cast, static
 - keyword, 21, 38, 78, 107, 139, 147, 148, 150, 165, 166, 251, 375, 424, 465, 474, 475, 570, 1962, 2394
- STATICALLY-WIDEN, 554
- __STDC_EMBED_EMPTY__, 543
- __STDC_EMBED_FOUND__, 543
- __STDC_EMBED_NOT_FOUND__, 543
- <stdnoreturn.h>
 - absence thereof, 557, 2453
- stop request, 2183
- stop request operation, *see* operation, stop request
- stop state, 2183
- stoppable callback deregistration, *see* callback deregistration, stoppable
- stoppable callback registration, *see* registration, stoppable callback
- stopped completion, *see* completion, stopped
- storage
 - reachable through a pointer value, 88
- storage class, 32
- storage duration, 74
 - automatic, 74
 - class member, 74
 - dynamic, 74–75, 157
 - local object, 74
 - static, 74
 - thread, 74
- storage management, *see* delete, *see* new
- storage-class-specifier, 210, 2399
- string
 - formatted as escaped, 1699
 - null terminator, 1607
 - null-terminated byte, *see* NTBS
 - null-terminated character type, 6
 - null-terminated multibyte, *see* NTMBS
 - type of, 27
- string literal, *see* literal, string
- string-literal, 27, 2390
- stringize, *see* # operator
- stringizing argument, 540
- strongly happens before, 95
- <strstream>
 - absence thereof, 573, 2418
- struct
 - standard-layout, 316
- struct
 - keyword, 21, 221, 276, 314, 316, 317, 352, 354, 763, 774, 837, 2402, 2405, 2444, 2445, 2449, 2451
- structural type, *see* type, structural
- structure tag, *see* class name
- structured binding, 274
- structured binding declaration, 207, 274
- structured binding pack, 447
- structured binding size, 274
- structured-binding-declaration, 206, 2398
- sub-expression
 - regular expression, 8
- subexpression, 91
 - potentially-evaluated, 91
- subnormal number, *see* number, subnormal
- subobject, *see also* object model, 65
 - corresponding direct, 65
 - initialized, known to be, 517
 - union elemental, 67
 - inactive, 67
- subscript expression, *see* expression, subscript
- subscripting operator
 - overloaded, 410
- subscripting operator function, *see* operator function, subscripting
- subsequence rule
 - overloading, 403
- substatement, 192
- substitutability, 620
- subsume, *see* constraint, subsumption
- subsumption
 - compatible for, 436
 - eligible for, 439
- subtraction operator, *see* operator, subtraction
- suffix
 - F, 26
 - f, 26
 - L, 23, 26
 - l, 23, 26
 - U, 23
 - u, 23

suitable created object, *see* object, suitable created

summary

- compatibility with C, 2443
- compatibility with ISO C++ 2003, 2437
- compatibility with ISO C++ 2011, 2435
- compatibility with ISO C++ 2014, 2431
- compatibility with ISO C++ 2017, 2423
- compatibility with ISO C++ 2020, 2418
- compatibility with ISO C++ 2023, 2414
- compatibility with ISO C++ 2026, 2413
- syntax, 2386

surrogate call function, 382

suspend point, 154

- final, 272

- initial, 272

swappable, 563

swappable with, 563

switch

- and handler, 514

- and try block, 514

- keyword, 21, 110, 193–196, 201, 203, 296, 2397, 2411

symbolic link, 2129

synchronize with, 94

synonym, 282

syntax

- class member, 143

synthesized point, *see* point, synthesized

synthesized three-way comparison, *see* three-way comparison, synthesized

T

tag

- completion, 2297

target constructor, *see* constructor, target

target object, *see* object, target

target scope, *see* scope, target

template, 416–513

- alias, 461

- class, 442

- deducible, 218

- deducible arguments of, 389

- function, 492

- abbreviated, 242

- corresponding object parameter, 41

- corresponding signature, 41

- key parameter of, 646

- partial ordering, 458

- member function, 443

- primary, 441

- static data member, 416

- variable, 416

template, 416

- keyword, 21, 35, 50, 59, 121, 122, 137, 139, 143, 163, 164, 198, 218–220, 344, 389, 410, 416, 423, 424, 441, 442, 464, 480,

- 486–489, 491, 850, 2386, 2392–2395, 2397, 2399, 2400, 2406–2408, 2457

template argument

- default, 421

template instantiation, 480

template name

- linkage of, 417

template parameter, 33

- concept, 419

- constant, 419

- execution policy, 1428

- type, 419

- type template, 419

- variable template, 419

template parameter pack, 446

template parameter scope, *see* scope, template parameter

template parameters

- template, 419

template template parameters, *see* template parameters, template

template-argument, 423, 2408

template-argument-equivalent, 440

template-argument-list, 423, 2408

template-argument-name, 423, 2408

template-declaration, 416, 2407

template-head, 416, 2407

template-id, 422, 2407

- valid, 425

template-name, 422, 2407

template-parameter, 418, 2407

template-parameter-list, 416, 2407

templated, 417

templated class, *see* class, templated

templated function, *see* function, templated

templated variable, *see* variable, templated

temporary, 76

- constructor for, 78

- destruction of, 78

- destructor for, 78

- elimination of, 76, 371

- implementation-defined generation of, 76

- order of destruction of, 78

temporary object, *see* object, temporary

terminal name, *see* name, terminal

terminate, 522

- called, 175, 516, 518, 522

termination

- program, 98, 102

terminology

- pointer, 87

text representation, 2195

text-line, 523, 2409

this, 116

- keyword, 21, 33, 36, 37, 50, 71, 115, 116, 119, 128–132, 177, 178, 181, 238, 239, 244, 268, 323, 337, 360, 366, 369, 380, 381, 472, 577, 939, 2058, 2095, 2116, 2222,

2223, 2227, 2230, 2233, 2236, 2238, 2240,
2241, 2391, 2393, 2401

thread, 93

thread attribute, 2193

thread attribute types, *see* types, thread attribute

thread of execution, 93

thread storage duration, *see* storage duration,
thread

thread_local, 210
keyword, 21, 74, 210, 233, 320, 337, 486, 488,
2399, 2437

threads
multiple, 93–98

<threads.h>
absence thereof, 557, 2453

three-way comparison
synthesized, 375

three-way comparison operator function, *see*
operator function, three-way comparison

throw, 175
keyword, 3, 21, 175, 474, 2396

throw-expression, 175, 2396

throwing, *see* exception handling, throwing

timed mutex types, 2247

to-unsigned-like, 1268

token, 20
alternative, 20
directive-introducing, 524
immediately after, 18
preprocessing, 17–18

token, 20, 2387

trailing *requires-clause*, *see* *requires-clause*, trailing
trailing-return-type, 229, 2400

traits, 8

transform_primary
regular expression traits, 1714, 1746

transitively constructed, *see* constructed,
transitively

translation
phases, 13–15
separate, *see* compilation, separate

translation character set, *see* character set,
translation

translation unit, 14, 60
name and, 33
preprocessing, 13

translation-unit, 60, 2391

transparently replaceable, 71

triangle
lower, 1850
upper, 1850

trigraph sequence, 2431

trivial class, *see* class, trivial

trivial infinite loop, *see* loop, trivial infinite

trivial type, *see* type, trivial

trivially copyable class, *see* class, trivially copyable

trivially copyable type, *see* type, trivially copyable

trivially empty iteration statement, *see* iteration
statement, trivially empty

true
keyword, 21, 29, 83, 102, 103, 113, 114, 135,
152, 154, 170–173, 185, 519, 521, 617,
998, 1032, 2140, 2391

truncation, 113

try, 514
keyword, 21, 514, 516, 2408

try block, *see* exception handling, try block
try-block, 514, 2408

TU-local
entity, 63
value or object, 63

tuple
and pair, 832

type, 32, 80–89
allocated, 155
arithmetic, 84
promoted, 412
array, 87
bitmask, 552, 553
Boolean, 83
callable, 920
char, 83
char16_t, 25, 27, 83
char32_t, 25, 27, 83
char8_t, 25, 83
character, 83, 84
character container, 4
class and, 314
compound, 87
const, 216
constexpr-unknown representation, 177
data-parallel, 1891
destination, 251
double, 84
dynamic, 4
element, 1891
enumerated, 552
example of incomplete, 81
extended integer, 83
extended signed integer, 82
extended unsigned integer, 82
float, 84
floating-point, 84
extended, 84
standard, 84
function, 87, 238, 239
fundamental, 86
implementation-defined **sizeof**, 82
implicit-lifetime, 81
incomplete, 35, 38, 43, 80, 110, 140, 143, 145,
152, 154, 155, 161, 344
incompletely-defined object, 80
int, 82
integer-class, 1208
integral, 84
promoted, 412
layout-compatible, 82
literal, 81

- long, 82
- long double, 84
- long long, 82
- narrow character, 83
- nodiscard, 298
- numeric, 1751
- ordinary character, 83
- over-aligned, 69
- pointer, 87
- polymorphic, 347
- program-defined, 6
- qualification-combined, 112
- referenceable, 7
- scalar, 81
- short, 82
- signed char, 82, 83
- signed integer, 82
- signed-integer-class, 1208
- similar, *see* similar types
- standard integer, 83
- standard signed integer, 82
- standard unsigned integer, 82
- standard-layout, 81
- static, 8
- structural, 420
- trivial, 2458
- trivially copyable, 81, 82
- underlying, 83
 - char16_t, 83, 113
 - char32_t, 83, 113
 - char8_t, 83
 - enumeration, 112, 277
 - fixed, 277
 - wchar_t, 83, 113
- unsigned, 82
- unsigned char, 82, 83
- unsigned int, 82
- unsigned integer, 82
- unsigned long, 82
- unsigned long long, 82
- unsigned short, 82
- unsigned-integer-class, 1208
- vectorizable, 1890
- void, 84
- volatile, 216
- wchar_t, 25, 27, 83
- type alias, *see* alias, type
- type checking
 - argument, 142
- type concept, *see* concept, type
- type conversion, explicit, *see* casting
- type generator, *see* template
- type name, 229
- type pun, 150
- type specifier
 - auto, 222
 - bool, 218
 - char, 218
 - char16_t, 218
 - char32_t, 218
 - char8_t, 218
 - const, 217
 - decltype, 221
 - decltype(auto), 222
 - double, 218
 - elaborated, 59, 220
 - enum, 220
 - float, 218
 - int, 218
 - long, 218
 - short, 218
 - signed, 218
 - simple, 218
 - unsigned, 218
 - void, 218
 - volatile, 217, 218
 - wchar_t, 218
- type template parameter, *see* template parameter, type
- type template template parameter, *see* template parameter, type template
- type-accessible, 108
- type-constraint, 418, 2407
- type-id, 229, 2401
- type-name, 218, 2399
- type-only
 - context, *see* context, type-only
 - lookup, *see* lookup, type-only
- type-parameter, 418, 2407
- type-parameter-key, 418, 2407
- type-requirement, 136, 2393
- type-specifier, 216, 2399
- type-specifier-seq, 216, 2399
- type-tt-parameter, 418, 2407
- type-tt-parameter-default, 418, 2407
- type_info, 146
- typedef
 - function, 241
- typedef
 - keyword, 21, 33, 80, 208, 209, 212, 1769, 1780, 1957, 2398
- typedef name for linkage purposes, 213
- typedef-name, 212, 2399
- typeid, 146
 - construction and, 371
 - destruction and, 371
- keyword, 21, 38, 71, 84, 130, 138, 139, 146, 147, 180, 474, 475, 515, 520, 2394, 2438, 2471
- typename, 220
 - keyword, 21, 59, 121, 136, 138, 218, 219, 227, 284, 285, 389, 418, 419, 423, 424, 464, 465, 471, 501, 2393, 2400, 2403, 2407, 2408
- typename-specifier, 464, 2408
- types
 - implementation-defined, 552
 - permissible, 379

- thread attribute, 2193
- U**
- ud-suffix*, 30, 2391
- unary element-wise operation, *see* operation,
 - unary element-wise
- unary fold, 134
- unary left fold, 134
- unary operator
 - interpretation of, 408
 - overloaded, 408
- unary right fold, 134
- unary-expression*, 151, 2394
- unary-operator*, 151, 2394
- unblock, 8
- uncaught exception, 516
- undefined, 7, 572, 574, 575, 1816, 1820, 1823, 2047, 2081, 2088
- undefined behavior, *see* behavior, undefined
- underlying constant, *see* constant, underlying
- underlying entity, *see* entity, underlying
- underlying type, *see* type, underlying
- unevaluated operand, 109
- unevaluated-string*, 29, 2391
- uniform distributions, 1784–1786
- uniform random bit generator
 - requirements, 1764
- union, 341
 - elemental subobject, *see* subobject, union elemental
 - inactive elemental subobject, *see* subobject, union elemental, inactive
 - standard-layout, 316
- union**, 87, 341
 - anonymous, 342
 - global anonymous, 342
 - keyword, 21, 221, 314, 316, 341, 342, 352, 2405, 2449, 2451
- union elemental subobject, *see* subobject, union elemental
- union-like class, *see* class, union-like
- unique pointer, 697
- unit
 - translation, 537, 561, 572
- unit-stride slice type, *see* slice type, unit-stride
- units
 - field width, 1689
- universal-character-name*, 17, 2386
- Unix time, 1967
- unnamed bit-field, *see* bit-field, unnamed
- unnamed class, *see* class, unnamed
- unnamed-namespace-definition*, 280, 2403
- unordered associative container, *see* container, unordered associative
- unordered associative containers, 985
 - complexity, 984
 - equality function, 984
 - equivalent keys, 985, 1072, 1084
 - exception safety, 995
 - hash function, 984
 - iterator invalidation, 995
 - iterators, 995
 - lack of comparison functions, 984
 - requirements, 984, 985, 995
 - unique keys, 985, 1064, 1079
- unordered_map**
 - element access, 1070
 - unique keys, 1064
- unordered_multimap**
 - equivalent keys, 1072
- unordered_multiset**
 - equivalent keys, 1084
- unordered_set**
 - unique keys, 1079
- unqualified name, *see* name, unqualified
- unqualified-id*, 118, 2392
- unscoped enumeration, *see* enumeration, unscoped
- unsequenced, 92
- unsigned**
 - keyword, 21, 65, 69, 80, 82–84, 108, 112, 113, 158, 218, 259, 1701, 2399, 2419
 - typedef** and, 210
- unsigned integer type, *see* type, unsigned integer
- unsigned-integer-class type, *see* type, unsigned-integer-class
- unsigned-integer-like, 1209
- unsigned-suffix*, 23, 2389
- unspecified, 602, 603, 607, 1535, 2094
- unspecified behavior, *see* behavior, unspecified
- unwinding
 - stack, 517
- upper triangle, *see* triangle, upper
- uppercase, 21, 553
- upstream allocator, 742
- usable
 - binary operator expression, 374
- usable candidate, *see* candidate, usable
- usable in constant expressions
 - object or reference, 186
 - variable, 185
- user-declared entity, *see* entity, user-declared
- user-defined conversion sequence, *see* conversion sequence, user-defined
- user-defined literal, *see* literal, user-defined
 - overloaded, 414
- user-defined-character-literal*, 30, 2391
- user-defined-floating-point-literal*, 30, 2391
- user-defined-integer-literal*, 30, 2391
- user-defined-literal*, 30, 2391
- user-defined-string-literal*, 30, 2391
- user-provided, 269
- uses-allocator construction, 687
- using**
 - keyword, 21, 206, 279, 281, 282, 284, 292, 2398, 2403, 2404
- using-declaration, 284–289
- using-declaration*, 284, 2403

using-declarator, 284, 2403
using-declarator-list, 284, 2403
using-directive, 282–284
using-directive, 282, 2403
using-enum-declaration, 279, 2403
using-enum-declarator, 279, 2403
 usual arithmetic conversions, *see* conversion, usual arithmetic
 usual deallocation function, 76
 UTF-16, 15, 259, 2140, 2444, 2461
 UTF-32, 15, 259, 2140, 2444, 2461
 UTF-8, 15, 64, 259, 1592, 2140, 2423, 2424, 2430, 2444, 2461

V

va-opt-replacement, 538, 2410
 vacuous initialization, *see* initialization, vacuous
 valid but unspecified state, 9
 valid contained type
 optional, 855
 valid in the context of, 88
 valid range, *see* range, valid
 value, 82
 associated with an evaluation, 88
 call by, 142
 consteval-only, 182
 denormalized, *see* number, subnormal
 erroneous, 72
 indeterminate, 72
 invalid pointer, 87
 null member pointer, 114
 null pointer, 87, 113
 representation, 82
 acquire, 82
 undefined unrepresentable integral, 113
 valid in the context of an evaluation, 88
 value category, 106
 value completion, *see* completion, value
 value computation, 78, 92–93, 95, 96, 145, 159, 175
 value type, 1200
 linear algebra, 1851
 value-initialization, 250
 value-preserving, 1891
 valueless
 indirect object, 726
 polymorphic object, 732
 vararg function, *see* function, vararg
 variable, 33
 active, 203
 anonymous union, 342
 decision, 193
 function-local predefined, 268
 indeterminate uninitialized, 249
 inline, 216
 needed for constant evaluation, 190
 program semantics affected by the existence
 of a variable definition, 483
 templated, 418

variable arguments, 538
 variable template
 definition of, 416
 variable template template parameter, *see*
 template parameter, variable template
variable-tt-parameter, 418, 2407
 variant member, 343
 vectorizable type, *see* type, vectorizable
 vectorization-unsafe, 1428
virt-specifier, 318, 2405
virt-specifier-seq, 318, 2405
virtual
 keyword, 21, 140, 212, 225, 233, 240, 331, 332, 337, 344–347, 349, 374, 446, 578, 2399, 2406
 virtual base class, *see* base class, virtual
 virtual function, *see* function, virtual, *see* function, virtual
 virtual function call, 350
 constructor and, 370
 destructor and, 370
 undefined pure, 351
 visible side effects, *see* side effects, visible
void
 keyword, 21, 38, 70, 75–77, 80, 81, 84, 87–89, 106–109, 114, 140, 142, 143, 145, 146, 148–150, 152, 153, 161, 170, 173, 175, 179, 201, 202, 218, 219, 223, 225, 234, 236, 237, 239, 241, 248, 268, 298, 299, 335, 337, 405, 421, 429, 465, 496, 498, 499, 506, 516, 518, 551, 624, 643, 653, 685, 711, 759, 761–767, 769–771, 773–775, 921, 962, 967, 970, 972, 973, 980, 981, 990–992, 994, 1203–1205, 1214, 1243, 1702, 1764, 1765, 1768, 2045, 2245, 2246, 2249, 2250, 2399, 2429, 2441, 2442, 2445, 2448, 2458, 2484
void*
 type, 88
void&, 234
volatile, 88
 constructor and, 325
 destructor and, 331
 implementation-defined, 218
 keyword, 21, 89, 92, 96, 144, 229, 772, 2400, 2452
 volatile object, *see* object, volatile
 volatile-qualified, 88

W

waiting function, *see* function, waiting
wchar_t, *see* type, **wchar_t**
 keyword, 3, 21, 25, 28, 83, 84, 112, 113, 115, 155, 218, 219, 251, 259, 546, 548, 553, 554, 580, 581, 827, 1640, 1662, 1698, 1709, 1713, 1715, 1749, 2033, 2036, 2112, 2130, 2137, 2140, 2141, 2150, 2399, 2430, 2444, 2453, 2462

weakly parallel forward progress guarantees, [97](#)
 well-formed program, *see* program, well-formed
while
 keyword, [21](#), [180](#), [192](#), [193](#), [196–198](#), [2397](#)
 whitespace, [17](#)
 wide literal encoding, *see* encoding, wide literal
 wide-character, [25](#)
 null, [15](#)
 wide-character set
 execution, [553](#)
 width, [82](#), [337](#), [1891](#)
 of integer-class type, [1208](#)
 padding, [1689](#)
 worse conversion sequence, *see* conversion
 sequence, worse
 writable, [1200](#)

X

XID_Continue, [20](#), [2418](#), [2466](#)
 XID_Start, [20](#), [2418](#), [2466](#)
xor
 keyword, [19](#), [20](#), [22](#), [2387](#), [2454](#)
xor_eq
 keyword, [19](#), [20](#), [22](#), [2387](#), [2454](#)
 xvalue, [106](#)

Y

Y_ℓ^m (spherical associated Legendre functions),
 [1836](#)
yield-expression, [174](#), [2396](#)

Z

zero
 division by undefined, [105](#)
 remainder undefined, [105](#)
 undefined division by, [167](#)
 zero-initialization, [249](#)
 zeta functions ζ , [1835](#)

Index of grammar productions

The first bold page number for each entry is the page in the general text where the grammar production is defined. The second bold page number is the corresponding page in the Grammar summary ([Annex A](#)). Other page numbers refer to pages where the grammar production is mentioned in the general text.

- abstract-declarator*, 227, **229**, 230, 239, 242, 243, **2401**
- abstract-pack-declarator*, **230**, **2401**
- access-specifier*, 344, **344**, 353, 354, **2406**
- additive-expression*, **168**, **2395**
- alias-declaration*, 33, 43, **206**, 212, 213, 230, 240, 308, 319, 416, 441, 461, **2398**
- alignment-specifier*, 68, 69, **292**, 293, 294, 322, 447, 449, 813, **2404**, 2451, 2500
- and-expression*, **172**, **2395**
- annotation*, **292**, 293, 300, 447, **2404**
- annotation-list*, **292**, 447, **2404**
- asm-declaration*, 180, 207, 290, **290**, **2403**
- assertion-statement*, 102, 104, 126, 203, **203**, 617, **2397**, 2414
- assignment-expression*, 140, **175**, 208, 225, 254, 274, 337, 362, 385, 475, 653, **2396**
- assignment-operator*, **176**, **2396**
- attribute*, **292**, 293, 294, 447, **2404**
- attribute-argument-clause*, 293, **293**, 294–299, **2404**
- attribute-declaration*, 34, **207**, 209, **2398**
- attribute-list*, **292**, 293, 447, **2404**
- attribute-namespace*, 293, **293**, 294, **2404**
- attribute-scoped-token*, 293, **293**, 294, **2404**
- attribute-specifier*, **292**, 293, 294, **2404**, 2415, 2451
- attribute-specifier-seq*, 45, 123, 125, 156, 192–194, 203, 207, 209, 212, 216, 220, 231, 233, 234, 236–238, 246, 248, 268, 274, 276, 277, 281, 282, 290, **292**, 293, 297, 302, 306, 314, 321, 324, 331, 335, 337, 344, 462, 486, 514, **2403**
- attribute-token*, 21, 32, 293, **293**, 294–299, 526, 537, **2404**, 2416
- attribute-using-prefix*, **292**, 293, **2404**, 2415
- attributed-identifier*, 248, **248**, **2401**
- await-expression*, 93, 153, **153**, 154, 179, 271, 272, 632, 2321, 2365, **2394**
- balanced-token*, **293**, **2404**
- balanced-token-seq*, 290, 293, **293**, 294, **2404**
- base-clause*, 315, **344**, 481, **2406**
- base-specifier*, 48, 56, 300, 344, **344**, 345, 353, 357, 447, 807, **2406**
- base-specifier-list*, 330, 344, **344**, 353, 365, 367, 375, 388, 447, 449, 805, **2406**
- basic-c-char*, **23**, 25, **2389**
- basic-s-char*, **27**, 29, **2390**
- binary-digit*, **22**, 23, **2388**
- binary-exponent-part*, **26**, 27, **2390**
- binary-literal*, **22**, 23, **2388**
- block-declaration*, 32, **206**, **2398**
- boolean-literal*, **29**, **2391**
- brace-or-equal-initializer*, 90, 208, 225, 249, **249**, 250, 254, 319, 320, 325, 337, 473, **2402**
- braced-init-list*, 3, 77, 78, 90, 143, 157, 176, 190, 202, 225, **249**, 251, 252, 254, 256, 257, 262, 263, 265, 362, 364, 366, 387, 428, 469, 474, 475, **2402**, 2412, 2432
- c-char*, **23**, 25, **2389**
- c-char-sequence*, 13, 17, **23**, 25, **2389**
- capture*, **129**, 130, 447, **2392**
- capture-default*, 37, 129, **129**, 130, 132, **2392**, 2425, 2455
- capture-list*, **129**, 447, **2392**
- cast-expression*, 134, 153, 165, **165**, 167, 190, 339, 408, 448, 449, 475, **2395**, 2482
- character-literal*, 15, 17, **23**, 25, 527, 537, 540, 1647, **2389**, 2435, 2443
- class-head*, 43, 294, 314, **314**, **2405**
- class-head-name*, 45, 46, 121, 314, **314**, 315, **2405**
- class-key*, 59, 207, 221, **314**, 315, 316, 341, 354, 442, **2405**
- class-name*, 12, 46, 207, 213, 220, 221, 314, **314**, 317, 318, 331, **2405**
- class-or-decltype*, 55, 315, 344, **344**, 363, 388, 464, **2406**
- class-property-specifier*, **314**, 315, 761, **2405**
- class-property-specifier-seq*, **314**, 315, **2405**
- class-specifier*, 14, 32, 43, 46, 47, 63, 138, 139, 207, 217, 314, **314**, 320, 323, 803, 807, **2405**
- compare-expression*, **169**, **2395**
- compound-requirement*, 136, **136**, 137, **2393**
- compound-statement*, 44, 45, 91, 98, 118, 124, 128, 132, 133, 153, 186, 192, 193, **193**, 194–196, 199, 201, 202, 325, 364, 365, 372, 417, 441, 466, 477, 484, 514–517, 519, **2397**
- computed-type-specifier*, 54, 118, 121, 122, 153, **218**, 332, **2399**
- concept-definition*, 34, 44, 462, **462**, 463, **2408**
- concept-name*, 44, 62, 307, 418, 420, 426, 435, 462, **462**, 474, **2408**
- concept-tt-parameter*, 32, 46, 418, **418**, 419, 420, 430, 447, **2407**
- condition*, 32, 192, **192**, 193, 195–198, 207, 208, 250, 251, 275, 515, **2396**
- condition-declaration*, **192**, 193, **2396**
- conditional-escape-sequence*, **24**, 25, 29, **2389**

- conditional-escape-sequence-char*, **24**, **2390**
- conditional-expression*, **4**, **173**, **173**, **203**, **246**, **2396**
- conditionally-supported-directive*, **524**, **525**, **538**, **2409**
- constant-expression*, **3**, **59**, **91**, **122**, **136**, **137**, **156**, **177**, **185**, **190**, **195**, **197**, **208**, **212**, **220**, **236**, **277**, **294**, **300**, **320**, **337**, **379**, **423**, **424**, **429**, **441**, **475**, **505**, **519**, **520**, **532**, **550**, **2396**
- consteval-block-declaration*, **14**, **34**, **128**, **180**, **188**, **190**, **207**, **209**, **319**, **797**, **800**, **2398**
- constraint-expression*, **44**, **118**, **137**, **228**, **387**, **418**, **420**, **426**, **433**, **436**, **436**, **437**, **438**, **458**, **463**, **466**, **550**, **2408**
- constraint-logical-and-expression*, **416**, **2407**
- constraint-logical-or-expression*, **228**, **416**, **418**, **2407**
- control-line*, **523**, **535**, **2409**
- conversion-declarator*, **335**, **2406**
- conversion-function-id*, **32**, **47**, **48**, **50**, **118**, **238**, **334**, **335**, **379**, **423**, **469**, **471**, **473**, **502**, **2406**, **2426**
- conversion-type-id*, **50**, **334**, **335**, **382**, **464**, **469**, **471**, **502**, **2406**
- coroutine-return-statement*, **202**, **271**, **2397**
- ctor-initializer*, **267**, **268**, **325**, **363**, **363**, **364**, **366**, **515**, **516**, **2406**
- cv-qualifier*, **89**, **165**, **175**, **207**, **210**, **217**, **229**, **233**, **234**, **236**, **239**, **274**, **320**, **323**, **324**, **412**, **420**, **518**, **554**, **2400**
- cv-qualifier-seq*, **116**, **147**, **167**, **217**, **229**, **233**, **236–238**, **240**, **241**, **268**, **382**, **2400**, **2413**
- d-char*, **18**, **27**, **2391**
- d-char-sequence*, **27**, **27**, **2390**, **2390**
- decimal-floating-point-literal*, **25**, **26**, **27**, **2390**
- decimal-literal*, **22**, **23**, **24**, **2388**
- decl-specifier*, **193**, **200**, **207**, **209**, **209**, **210**, **212**, **220**, **222**, **223**, **226**, **228**, **274**, **324**, **331**, **335**, **465**, **2398**, **2426**
- decl-specifier-seq*, **89**, **193**, **200**, **207–209**, **209**, **210**, **212**, **216**, **217**, **222**, **223**, **226**, **228**, **233**, **268**, **274**, **276**, **320**, **324**, **331**, **335**, **358**, **465**, **2398**
- declaration*, **12**, **32**, **34**, **41**, **204**, **205**, **206**, **213**, **216**, **230**, **231**, **280**, **306**, **308**, **319**, **357**, **416**, **417**, **461**, **486**, **488**, **2397**
- declaration-seq*, **12**, **33**, **206**, **304**, **306**, **308**, **310**, **477**, **2397**
- declaration-statement*, **153**, **203**, **203**, **2397**
- declarator*, **32**, **33**, **45**, **51**, **89**, **116**, **193**, **204**, **205**, **207**, **212**, **223**, **226–228**, **228**, **230–233**, **243**, **268**, **319**, **320**, **324**, **331**, **335**, **465**, **486**, **2400**
- declarator-id*, **45**, **46**, **51**, **56**, **118**, **121**, **130**, **207**, **208**, **229**, **231–234**, **236–238**, **241–243**, **274**, **303**, **319**, **320**, **331**, **335**, **374**, **407**, **414**, **417**, **423**, **465**, **500**, **503**, **508**, **512**, **2400**, **2415**, **2426**
- decltype-specifier*, **77**, **107**, **127**, **219**, **221**, **221**, **222**, **234**, **235**, **308**, **441**, **505**, **2400**
- deduction-guide*, **34**, **45**, **207**, **319**, **387**, **389**, **395**, **416**, **443**, **444**, **2408**
- defined-macro-expression*, **526**, **526**, **527**, **2410**
- defining-type-id*, **62**, **213**, **219**, **229**, **388**, **389**, **461**, **462**, **464**, **472**, **2401**
- defining-type-specifier*, **63**, **207**, **210**, **212**, **216**, **216**, **217**, **335**, **2399**
- defining-type-specifier-seq*, **213**, **216**, **216**, **2399**
- delete-expression*, **36**, **70**, **75**, **158**, **161**, **161**, **162**, **163**, **179**, **332**, **339**, **602**, **604**, **2395**, **2469**
- deleted-function-body*, **268**, **270**, **2402**
- designated-initializer-clause*, **249**, **250**, **254**, **262**, **517**, **2402**
- designated-initializer-list*, **249**, **252–254**, **262–264**, **387**, **400**, **2402**
- designated-only-initializer-list*, **249**, **253**, **254**, **2402**
- designator*, **249**, **252**, **253**, **263**, **2402**
- digit*, **20**, **23**, **26**, **27**, **302**, **529**, **572**, **2388**, **2434**
- digit-sequence*, **26**, **26**, **2390**
- directory-separator*, **2138**, **2138**, **2139**, **2143**, **2145**, **2147**, **2148**
- elaborated-type-specifier*, **32**, **33**, **35**, **41**, **44**, **55**, **56**, **59**, **121**, **207**, **213**, **220**, **220**, **221**, **294**, **303**, **314**, **317**, **318**, **450**, **464**, **467**, **486**, **2400**
- elif-group*, **523**, **2409**
- elif-groups*, **523**, **2409**
- else-group*, **523**, **2409**
- embed-parameter*, **524**, **526**, **530**, **532–534**, **2410**
- embed-parameter-seq*, **524**, **526**, **530**, **532**, **533**, **2409**
- embed-prefixed-parameter*, **524**, **525**, **2410**
- embed-standard-parameter*, **524**, **2410**
- empty-declaration*, **33**, **34**, **207**, **209**, **319**, **342**, **2398**
- enclosing-namespace-specifier*, **280**, **281**, **2403**
- encoding-prefix*, **14**, **15**, **23**, **25**, **28**, **29**, **414**, **546**, **2389**, **2419**, **2444**
- endif-line*, **523**, **2409**
- enum-base*, **276**, **276**, **277**, **2402**
- enum-head*, **43**, **276**, **276**, **2402**
- enum-head-name*, **45**, **46**, **121**, **276**, **277**, **2402**
- enum-key*, **276**, **277**, **2402**
- enum-name*, **207**, **220**, **276**, **276**, **2402**
- enum-specifier*, **32**, **43**, **46**, **207**, **217**, **276**, **277–279**, **319**, **320**, **792**, **795**, **2402**
- enumerator*, **207**, **276**, **277**, **2403**
- enumerator-definition*, **32**, **43**, **276**, **277**, **2403**
- enumerator-list*, **46**, **276**, **277**, **278**, **481**, **2403**
- equality-expression*, **170**, **2395**
- escape-sequence*, **24**, **2389**
- exception-declaration*, **32**, **38**, **294**, **372**, **514**, **514**, **516**, **518**, **519**, **2408**
- exclusive-or-expression*, **172**, **2395**
- expansion-init-list*, **79**, **199**, **199**, **2397**
- expansion-initializer*, **44**, **79**, **199**, **199**, **200**, **473**, **2397**

- expansion-statement*, 44, 192, 193, **198**, 199, 201, 417, 441, 466, 473, 474, 477, **2397**
- explicit-instantiation*, 208, 211, 231, 486, **486**, **2408**
- explicit-specialization*, 34, 208, 220, 231, **487**, 488, **2408**
- explicit-specifier*, 212, **212**, 233, 324, 379, 389, 496, **2399**
- exponent-part*, **26**, 27, **2390**
- export-declaration*, 61, 280, **303**, 304, 306, 417, 488, **2404**
- export-keyword*, 17, 18, **18**, 21, 302, 303, 534, 535, **2388**
- expr-or-braced-init-list*, 201, 202, **249**, **2402**
- expression*, 22, 78, 84, 109, 136, 147, 156, 157, **176**, 193, 195–198, 202, 225, 372, 434, 437, 441, 472, 475, 505, 550, **2396**
- expression-list*, 79, 90, 139, **139**, 140, 141, 143, 157, 159, 199, 225, 250–252, 362, 364, 366, 380, 385, 387, 449, 469, 475, 2360, **2394**
- expression-statement*, **193**, 204, **2397**
- fallback-separator*, **2138**
- filename*, 2135, 2138, **2138**, 2146–2148
- floating-point-literal*, 18, 19, **25**, 26, 27, 1962, **2390**
- floating-point-suffix*, 26, **26**, **2390**
- fold-expression*, **134**, 448, 449, 474, **2393**
- fold-operator*, 134, **134**, 435–439, 449, **2393**
- for-range-declaration*, 33, 44, 79, 192, **192**, 193, 199, 200, 417, 473, 474, **2396**
- for-range-initializer*, 44, 79, **192**, 198, 262, **2396**, **2419**
- fractional-constant*, 26, **26**, **2390**
- friend-type-declaration*, 207, **318**, 320, 357, 416, 417, 447, **2405**
- friend-type-specifier*, 220, **319**, 357, 447, 464, **2406**
- friend-type-specifier-list*, **318**, **2406**
- function-body*, 33, 44, 45, 63, 128, 153, 202, 208, 268, **268**, 269–271, 274, 374, 411, 517, **2402**
- function-contract-specifier*, 125, 246, **246**, 247, 320, 441, 490, 496, **2401**
- function-contract-specifier-seq*, 228, 246, **246**, 247, 319, **2401**
- function-definition*, 33, 45, 116, 207, 208, 212, 268, **268**, 271, 374, 465, **2402**
- function-specifier*, 212, **212**, **2399**
- function-try-block*, 45, 104, 120, 141, 514, **514**, 515, 517, 519, 634, **2408**, 2493
- global-module-fragment*, 303, 307, **307**, 561, **2405**
- group*, **523**, 525, 535, **2409**
- group-part*, **523**, **2409**
- h-char*, **19**, **2387**
- h-char-sequence*, 13, 19, **19**, 529, 530, **2387**
- h-pp-tokens*, **526**, **2410**
- h-preprocessing-token*, **526**, **2410**
- handler*, 40, 44, 45, 153, 192, 372, **514**, 515, 518, 519, 578, 2312, **2408**
- handler-seq*, **514**, 515, **2408**
- has-attribute-expression*, 295–299, 321, 526, **526**, 527, **2410**
- has-embed-expression*, 526, **526**, 527, 543, **2410**
- has-include-expression*, 526, **526**, 527, **2410**
- header-name*, 13, 18, **18**, 19, 306, 524, 526, 528–532, 535, 540, **2387**, 2423, 2507
- header-name-tokens*, **526**, 529, 535, **2410**, 2507
- hex-quad*, 17, **17**, **2386**
- hexadecimal-digit*, 17, **22**, 23, 25, 26, 29, **2389**
- hexadecimal-digit-sequence*, **22**, 26, **2388**
- hexadecimal-escape-sequence*, **24**, 25, 29, **2389**
- hexadecimal-floating-point-literal*, **25**, 26, 27, **2390**, **2431**
- hexadecimal-fractional-constant*, 26, **26**, **2390**
- hexadecimal-literal*, **22**, 23, **2388**
- hexadecimal-prefix*, **22**, **2388**
- id-expression*, 3, 35, 36, 42, 54, 62, 102, 107, 109, 117, **117**, 118, 120–122, 130, 132, 133, 138, 140, 143, 164, 165, 178, 181, 221, 231, 232, 245, 248, 274, 276, 307, 324, 331, 335, 377, 380, 420, 429, 432, 438, 448, 449, 473–475, 510, 610, 637, **2391**, **2419**
- identifier*, 12, **20**, 21, 32, 43, 44, 54, 59, 118, 121, 122, 129, 130, 132, 155, 163, 164, 193, 207, 208, 212, 220, 231, 248, 252–254, 263, 274, 276, 277, 280–282, 293, 298, 302, 314, 317, 318, 322, 338, 414, 419, 423, 425, 426, 444, 447, 461, 462, 464, 470, 473, 524, 525, 527, 528, 534, 536, 812, **2387**, 2466
- identifier-continue*, **20**, **2388**, 2466
- identifier-list*, 12, 44, **524**, 537, **2409**
- identifier-start*, **20**, **2388**, 2466
- if-group*, **523**, **2409**
- if-section*, **523**, **2409**
- import-keyword*, 17, 18, **18**, 21, 535, 2388
- inclusive-or-expression*, **172**, **2395**
- init-capture*, 32, 33, 42, 91, 118, 129, **129**, 130, 132–134, 165, 447, 449, **2393**
- init-declarator*, 32, 45, 91, 198, 203, 207, 210, 223, **227**, 228, 429, 486, **2400**
- init-declarator-list*, 207, 210, 217, 223, 226, 227, **227**, 417, 486, **2400**
- init-statement*, 153, 192, **192**, 194–197, **2396**
- initializer*, 33, 63, 79, 91, 129, 132, 133, 153, 208, 223, 228, 235, 249, **249**, 250–252, 274, 275, 337, 362, 385, 447, 486, **2401**
- initializer-clause*, 90, 139–141, 159, 225, 243, 244, **249**, 252, 254, 255, 257–259, 262, 265, 337, 428, 447, **2402**, 2412
- initializer-list*, **249**, 254–256, 258, 262, 264–266, 362, 387, 447, **2402**
- integer-literal*, 18, 19, **22**, 23, 24, 30, 526, 1962, **2388**
- integer-suffix*, 23, **23**, 24, 543, **2389**
- iteration-statement*, 192, 196, **196**, 201, **2397**

jump-statement, 32, **201**, **2397**

keyword, **21**, **2388**

label, 32, 193, **193**, **2396**

label-seq, **194**, **2397**

labeled-statement, 192, 193, **193**, **2396**

lambda-capture, 123–125, 127–129, **129**, **2392**

lambda-declarator, 45, 46, 123, **123**, 124, 125, 130, 240, 243, 246, 465, **2392**

lambda-expression, 37, 39, 40, 45, 62, 63, 91, 116, 118, 123, **123**, 124–134, 153, 161, 178, 192, 213, 222, 247, 308, 417, 457, 462, 473, 484, 496, 517, 800, **2392**, 2411, 2450, 2455

lambda-introducer, 45, **123**, 129, 161, 800, **2392**

lambda-specifier, 123, **123**, **2392**

lambda-specifier-seq, 123, **123**, **2392**

line-directive, **542**, **2410**

line-directives, **523**, **2409**

linkage-specification, 33, 62, 98, 207, 211, 290, **290**, 292, 303, 306, 416, **2403**, 2412

literal, **22**, 30, 115, **2388**

literal-operator-id, 30, 32, 118, 414, **414**, 440, **2406**, 2456

logical-and-expression, **172**, **2395**

logical-or-expression, **173**, **2396**

long-long-suffix, **23**, **2389**

long-suffix, **23**, **2389**

lparen, **524**, **2409**

mem-initializer, 91, 256, 262, 363, **363**, 364–367, 371, 447, 517, **2406**, 2489

mem-initializer-id, 268, 363, **363**, 364, 447, **2406**

mem-initializer-list, 363, **363**, 364, 447, **2406**

member-declaration, 32, 210, 217, 228, 233, 268, 276, 285, 286, 289, **318**, 319–321, 324, 331, 335, 342, 465, **2405**

member-declarator, 32, 45, 116, 228, 240, **318**, 319–321, 337, 474, **2405**

member-declarator-list, 210, 217, 226, **318**, 320, 321, **2405**

member-specification, 46, **318**, 319, 320, 324, 331, 335, 337, 342, 353, 358, 374, 375, 481, 797, **2405**, 2411

module-declaration, 47, 302, **302**, 303, 306, 307, **2404**, 2500

module-file, **523**, 535, **2408**

module-import-declaration, 47, 290, 303, 304, 306, **306**, 307, **2404**

module-keyword, 17, 18, **18**, 21, 534, **2388**

module-name, 32, 302, **302**, 306, **2404**, 2500

module-name-qualifier, **302**, **2404**

module-partition, 302, **302**, 303, 306, **2404**

multiplicative-expression, **167**, **2395**

n-char, **15**, **2386**

n-char-sequence, **15**, 17, **2386**

name-declaration, 32, **206**, 207, 304, **2397**

named-namespace-definition, 280, **280**, 281, **2403**

named-universal-character, **15**, 17, **2386**

namespace-alias, 282, **282**, **2403**

namespace-alias-definition, 33, 59, 282, **282**, 308, 476, **2403**

namespace-body, 45, 280, **280**, 281, **2403**

namespace-definition, 44, 45, 207, 280, **280**, 281, 304, 306, 308, **2403**

namespace-name, 46, 54, 59, 121, 122, 280, **280**, 281, 282, 307, 475, 476, 572, **2403**

nested-name-specifier, 50, 51, 54–56, 59, 62, 121, **121**, 122, 136, 163, 164, 218–220, 236, 276, 277, 282, 285, 286, 303, 314, 331, 344, 356, 360, 418, 419, 423–425, 451, 464, 498, 505, **2392**

nested-namespace-definition, **280**, 281, **2403**

nested-requirement, 136, 137, **137**, **2393**

new-declarator, 156, **156**, **2394**

new-expression, 36, 38, 65, 70, 74, 75, 93, 155, 156, **156**, 157–162, 179, 223, 226, 262, 270, 332, 333, 342, 515, 517, 520, 602–605, 1580, **2394**, 2432

new-initializer, 70, 79, 156, **156**, 157, 159, 251, **2394**

new-line, **524**, 535, **2410**

new-placement, **156**, 158, 159, 161, **2394**

new-type-id, 89, 155, 156, **156**, 157, 223, 226, 229, 464, 473, **2394**

nodeclspec-function-declaration, 45, **206**, 207, 208, **2398**

noexcept-expression, 155, **155**, 475, **2394**

noexcept-specifier, 125, 136, 273, 320, 331, 340, 374, 441, 467, 477, 496, 507, 509, 519, **519**, 520, 521, **2408**

nofun-declarator, **229**, 230, **2400**

nofun-type-id, 154–157, 223, 226, 229, **229**, 230, 294, 473, 475, **2400**

nondigit, **20**, 529, **2388**

nonzero-digit, **22**, **2388**

noptr-abstract-declarator, **229**, **2401**

noptr-abstract-pack-declarator, **230**, **2401**

noptr-declarator, 45, **229**, 335, **2400**

noptr-new-declarator, 156, **156**, 157, **2394**

noptr-nofun-declarator, **229**, **2401**

numeric-escape-sequence, **24**, 25, 29, **2389**

octal-digit, **22**, 23, 25, 29, **2388**

octal-escape-sequence, **24**, 25, 29, **2389**

octal-literal, **22**, 23, **2388**

opaque-enum-declaration, 33, 43, 276, **276**, 277, 319, **2402**

operator, **408**, **2406**

operator-function-id, 32, 118, 407, **407**, 408, 440, 789, 790, **2406**

operator-or-punctuator, 19, **19**, **2387**

pack-index-expression, 122, **122**, 165, 447, 449, 474, 505, **2392**

- pack-index-specifier*, 219, 220, **220**, 441, 448, 449, 472, 505, **2400**, 2415
- pack-index-template-name*, 218, **422**, 423, 441, 448, 449, **2407**
- parameter-declaration*, 32, 33, 45, 212, 222, 227, 238, **238**, 239, 242–244, 271, 296, 300, 415, 419, 447, 465, 508, **2401**
- parameter-declaration-clause*, 40, 45, 118, 124, 125, 130, 135, 231, 238, **238**, 239, 242, 243, 271, 374, 387, 414, 415, 444, **2401**, 2456
- parameter-declaration-list*, **238**, 239, 500, 505, **2401**, 2456
- parameters-and-qualifiers*, **229**, **2400**
- path-format-spec*, **2150**
- pathname*, **2137**
- placeholder-type-specifier*, 218, 219, 222, **222**, 225, 226, 242, **2400**
- plain-string-literal*, **27**, **2390**
- pm-expression*, **166**, 167, **2395**, 2482
- pointer-literal*, **29**, **2391**
- postcondition-specifier*, 102, 246, **246**, 247, 248, **2401**
- postfix-expression*, 51, 52, 122, 139, **139**, 140, 141, 143, 177, 179, 380, 381, 410, 469, 520, 555, 610, 916, **2394**
- pp-balanced-token*, **524**, **2410**
- pp-balanced-token-seq*, **524**, 532, 533, **2410**
- pp-global-module-fragment*, **523**, 525, **2408**
- pp-import*, 525, 535, **535**, 536, **2410**
- pp-module*, 534, **534**, **2410**
- pp-module-name*, 534, **534**
- pp-module-name-qualifier*, **534**
- pp-module-partition*, 534, **534**
- pp-number*, **19**, 526, 527, **2387**, 2431, 2435
- pp-private-module-fragment*, **523**, **2409**
- pp-tokens*, **524**, 525, 526, 530, 534, 535, 539, 546, **2409**
- precondition-specifier*, 102, 246, **246**, **2401**
- preferred-separator*, 2137, 2138, **2138**, 2142, 2143, 2145
- preprocessing-file*, 14, **523**, 525, **2408**
- preprocessing-op-or-punc*, **19**, **2387**
- preprocessing-operator*, **19**, **2387**
- preprocessing-token*, **17**, 20, 21, 526, **2387**, **2410**
- primary-expression*, **115**, 117, **2391**
- private-module-fragment*, 38, 47, 60, 63, 223, 304, 306, 309, **309**, 310, 312, 477, **2405**
- ptr-abstract-declarator*, **229**, **2401**
- ptr-declarator*, **228**, 324, 331, **2400**
- ptr-nofun-declarator*, **229**, **2401**
- ptr-operator*, 50, **229**, 236, **2400**
- pure-specifier*, **318**, 319–321, 332, 351, **2405**
- q-char*, **19**, **2387**
- q-char-sequence*, 13, 19, **19**, 529, 530, **2387**
- qualified-id*, 12, 51, 54, 117, 121, **121**, 122, 140, 143, 152, 153, 207, 231, 232, 275, 303, 324, 333, 335, 336, 351, 356, 419, 465, 475, 498, 505, 557, 628, 672, 684, 687, 690, 691, 699, 776, 777, 964, 967, 976, 984, 985, 995, 1048–1050, 1059, 1070–1072, 1084, 1110–1113, 1128, 1151, 1205, 1207, 1238, 1243, 1284, 1603, 1703, 1704, 1957, 2140, 2371, 2372, **2392**
- qualified-namespace-specifier*, 55, 282, **282**, 476, **2403**
- r-char*, 18, 27, **27**, 29, **2390**
- r-char-sequence*, 13, 17, **27**, **2390**
- raw-string*, 27, **27**, **2390**
- ref-qualifier*, 7, 8, 41, 147, 167, **229**, 234, 236, 238, 240, 241, 269, 274, 323, 324, 348, 378, 403, 412, 459, **2400**, 2413
- reflect-expression*, 39, 62, 117, 163, **163**, 164, 165, 240, 245, 270, 461, 475, **2395**
- reflection-name*, 55, 163, **163**, 164, 475, **2395**
- relational-expression*, **170**, **2395**
- relative-path*, 2138, **2138**, 2148
- replacement-list*, **524**, 537, **2409**
- requirement*, 135, **135**, 136, **2393**, 2497
- requirement-body*, 45, **135**, **2393**
- requirement-parameter-list*, 45, 135, **135**, 184, 465, **2393**
- requirement-seq*, **135**, **2393**
- requires-clause*, 8, 41, 118, 123, 124, 126, 135, 228, 241, 268, 291, 347, 348, 374, 387, **416**, 418, 432, 436, 441, 451, 458, 485, 800, 1244, 1289, 1302, 1349, 1350, 2338–2340, **2407**, 2424
- requires-expression*, 37, 45, 84, 134, 135, **135**, 136, 165, 637, 638, **2393**, 2424
- result-name-introducer*, 32, 44, 46, 102, 247, 248, **248**, **2401**
- return-type-requirement*, **136**, 137, **2393**
- root-directory*, 2135, 2138, **2138**, 2139, 2146, 2148
- root-name*, 2135, **2137**, 2138, 2142, 2145, 2147, 2148
- s-char*, 27, **27**, 29, **2390**
- s-char-sequence*, 13, 17, **27**, **2390**
- sb-identifier*, **206**, 207, 274, 447, **2398**
- sb-identifier-list*, **206**, 274, **2398**
- selection-statement*, 192, 194, **194**, **2397**
- shift-expression*, **169**, **2395**
- sign*, **26**, 27, **2390**, 2431
- simple-capture*, 37, 129, **129**, 130, 134, **2393**
- simple-declaration*, 153, **206**, 207, 208, 210, 217, 227, 228, 233, 465, 486, **2398**
- simple-escape-sequence*, **24**, 25, 26, 29, **2389**
- simple-escape-sequence-char*, **24**, **2389**
- simple-hexadecimal-digit-sequence*, 17, **17**, **2386**
- simple-octal-digit-sequence*, **24**, **2389**
- simple-requirement*, 136, **136**, 137, **2393**
- simple-template-id*, 39, 43, 121, 122, 143, 164, 212, 213, 218–220, 303, 308, 314, 318, 344, 387, 389, 417, **422**, 423, 425, 426, 428, 444, 451, 453, 454, 464, 466, 472, 480, 486, 501, 510, **2407**, 2426

- simple-template-name*, 218, **422**, 423, 441, 444, 448, 449, 461, **2408**
- simple-type-specifier*, 143, 204, 218, **218**, 219, 220, 223, 226, 231, 464, 473, 475, **2399**
- size-suffix*, 23, **23**, **2389**
- special-declaration*, **206**, **2398**
- splice-expression*, 35, 36, 107, 109, 117, 118, 120, **137**, 138, 139, 143, 144, 152, 181, 221, 245, 308, 356, 360, 377, 380, 392, 424, 446, 474, 475, 793, **2393**
- splice-scope-specifier*, 54, 121, **121**, 308, 475, **2392**
- splice-specialization-specifier*, 39, 59, **59**, 121, 138, 139, 227, 308, 424–426, 428, 461, 465, 472, 474, 475, 494, 501, 510, **2391**
- splice-specifier*, 59, **59**, 62, 121, 138, 139, 184, 218, 221, 227, 282, 308, 423, 425, 465, 472, 474–476, 505, **2391**
- splice-type-specifier*, 219, 227, **227**, 279, 308, 387, 389, 443, 472, **2400**
- statement*, 40, 192, **192**, 195, 197, 201, 204, 497, 514, **2396**
- statement-seq*, 192, **193**, **2397**
- static_assert-declaration*, 10, 34, **207**, 208, 319, 342, 466, 550, **2398**
- static_assert-message*, 190, **206**, 208, **2398**
- storage-class-specifier*, 42, 210, **210**, 211, 274, 292, 320, 342, 358, 419, 486, 488, **2399**, 2432
- string-literal*, 14, 15, 17, 18, 27, **27**, 28, 29, 31, 88, 115, 157, 251, 259, 263, 388, 400, 430, 529, 531, 537, 540, 542, 546, 554, 1600, 2108, **2390**, 2411, 2419, 2437, 2444
- structured-binding-declaration*, 192, 193, 200, **206**, 207, 208, **2398**
- template-argument*, 31, 240, 353, **423**, 424–428, 430–432, 434, 440, 443, 447, 450, 461, 466, 467, 476, 480, 492–494, 499, 501, 510–512, 871, 882, **2408**
- template-argument-list*, 59, 176, 389, 423, **423**, 424, 427, 441, 447, 449, 452, 454, 467, 472, 871, 882, **2408**
- template-argument-name*, **423**, 425, 426, **2408**
- template-declaration*, 34, 41, 46, 207, 208, 213, 220, 231, 319, 357, 416, **416**, 417, 418, 461, 468, **2407**
- template-head*, 8, 34, 41, 46, 62, 122, 242, 387, **416**, 426, 431, 432, 441, 442, 445, 452, 458, 459, 469, **2407**
- template-id*, 39, 53, 54, 118, 207, 231, 275, 286, 303, 422, **422**, 423–426, 428, 434, 440, 441, 446, 454, 457, 461, 462, 469, 473, 480, 494, 809, 810, **2407**
- template-name*, 52, 62, 164, 213, 219, 307, 308, 318, 387, 389, 419, **422**, 423, 425, 426, 440, 443, 444, 448, 451, 467, 468, **2407**, 2428
- template-parameter*, 33, 41, 44, 46, 222, 227, 242, 243, 415, 418, **418**, 419–422, 426, 428, 431, 432, 434, 440, 458, 460, 461, 465, 469, 491, 493, **2407**
- template-parameter-list*, 6, 41, 46, 124, 130, 242, 415, 416, **416**, 418, 419, 421, 422, 426, 434, 436, 454, 458, 460, 461, 468, 491, **2407**
- text-line*, **523**, 525, 534, **2409**
- throw-expression*, 93, 120, 173, 175, **175**, 179, 372, 515, 518, 520, 522, **2396**
- token*, **20**, 21, 293, 303, 812, **2387**, 2404
- trailing-return-type*, 116, 124, 127, 205, 222, 223, **229**, 230, 238, 241, 464, 800, **2400**
- translation-unit*, 14, **60**, 306, 308, 477, **2391**
- try-block*, 120, 192, 372, 514, **514**, 515, 634, **2408**
- type-constraint*, 126, 137, 218, 222, 225, 226, 242, 418, **418**, 419–421, 436, 441, 458, 485, 505, **2407**
- type-id*, 3, 89, 112, 139, 147, 164, 229, **229**, 230, 240, 335, 424, 426, 428, 465, 473, 475, **2401**
- type-name*, 54, 55, 59, 118, 121, 122, 136, 153, 164, 204, 210, 218, **218**, 219, 220, 231, 307, 332, 344, 426, 442, 467, **2399**, 2438
- type-parameter*, 32, 220, 240, 242, **418**, 419, 420, 447, 465, 511, **2407**
- type-parameter-key*, **418**, 419, **2407**
- type-requirement*, 136, **136**, 464, **2393**
- type-specifier*, 50, 62, 193, 210, 216, **216**, 217, 218, 222, 223, 227, 238, 317, 320, 464, **2399**
- type-specifier-seq*, 156, 216, **216**, 217, 223, 226, 277, **2399**
- type-tt-parameter*, 32, 46, 418, **418**, 419, 430, 447, **2407**
- type-tt-parameter-default*, **418**, **2407**
- typedef-name*, 12, 46, 52, 155, 208, 210, 212, **212**, 213, 220, 222, 230, 234, 235, 240, 419, 441, 448, 449, 461, 465, 546, 598, 624, 625, 758, 774–776, 820, 851, 1201, 1225, 1234, 1238, 1243, 1295, 1306, 1311, 1324, 1329, 1337, 1345, 1357, 1362, 1372, 1383, 1405, 1589, 1604, 1713, 1764, 2023, 2036, 2137, 2179, 2244, 2253, **2399**, 2448, 2453
- typename-specifier*, 55, 143, 217, 464, **464**, 473, 475, 550, **2408**
- ud-suffix*, 30, **30**, 31, 414, 791, **2391**
- unary-expression*, 147, **151**, 333, 408, 475, **2394**
- unary-operator*, 147, **151**, 408, 1209, **2394**
- unevaluated-string*, 29, **29**, 208, 270, 290, 295, 298, 414, **2391**
- universal-character-name*, 13, 17, **17**, 20, 25, 28, 29, 290, 540, 812, **2386**
- unnamed-namespace-definition*, 44, **280**, 281, **2403**
- unqualified-id*, 51, 54, 118, **118**, 119, 121, 152, 153, 165, 207, 231, 238, 285, 324, 331, 335, 417, 419, 469, 486, **2392**, 2428
- unsigned-suffix*, **23**, **2389**
- user-defined-character-literal*, **30**, 31, **2391**
- user-defined-floating-point-literal*, **30**, **30**, **2391**
- user-defined-integer-literal*, **30**, **30**, **2391**
- user-defined-literal*, 17, **30**, **30**, 115, **2391**

user-defined-string-literal, [30](#), [31](#), [414](#), [2391](#)
using-declaration, [33](#), [47](#), [52](#), [56](#), [207](#), [279](#), [284](#),
[285–287](#), [289](#), [304](#), [308](#), [319](#), [330](#), [352](#),
[378](#), [447](#), [452](#), [466](#), [557](#), [636](#), [803](#), [2244](#),
[2403](#), [2433](#)
using-declarator, [32](#), [41](#), [43](#), [47](#), [54](#), [56](#), [86](#), [163](#),
[245](#), [284](#), [285](#), [286](#), [288](#), [289](#), [304](#), [352](#),
[417](#), [423](#), [447](#), [463](#), [465](#), [471](#), [2403](#)
using-declarator-list, [284](#), [2403](#)
using-directive, [34](#), [42](#), [50](#), [56](#), [59](#), [207](#), [220](#), [281](#),
[282](#), [282](#), [283](#), [284](#), [2403](#)
using-enum-declaration, [34](#), [279](#), [279](#), [2403](#)
using-enum-declarator, [279](#), [279](#), [464](#), [2403](#)

va-opt-replacement, [538](#), [538](#), [539](#), [2410](#)
variable-tt-parameter, [32](#), [46](#), [418](#), [418](#), [419](#), [430](#),
[447](#), [2407](#)
virt-specifier, [318](#), [321](#), [348](#), [2405](#)
virt-specifier-seq, [268](#), [318](#), [319](#), [321](#), [2405](#)

yield-expression, [153](#), [174](#), [174](#), [179](#), [271](#), [1423](#),
[1424](#), [2396](#)

Index of library headers

The bold page number for each entry refers to the page where the synopsis of the header is shown.

<algorithm>, 583–585, 587, **1432**, 2440
 <any>, 583, **883**, 2434
 <array>, 583, 585, 586, 588, 849, 995, 996, **996**,
 1256, 2440, 2460
 <assert.h>, 561, **635**
 <atomic>, 584, **2201**, 2244, 2440, 2463

 <barrier>, 584, **2272**, 2429
 <bit>, 584–586, **952**, 1933, 1934, 2429
 <bitset>, 584, 588, **910**

 <cassert>, 561, **653**
 <cctype>, 1646, **1747**
 <cerrno>, 573, 585, **654**, 657, 2457
 <cfenv>, **1751**, 1752, 2440
 <cfloat>, 84, 583, **596**, 597, 2457
 <charconv>, 584, 585, 588, **1634**, 2429, 2434
 <chrono>, 584, **1941**, 2440
 <cinttypes>, **2177**, 2178, 2440
 <climits>, 64, 84, 583, **595**, 596
 <locale>, **1676**, 2454
 <cmath>, 584–586, 635, **1824**, 1830, 2429, 2457
 <compare>, 584, 588, **619**, 2429
 <complex>, 584, 635, 849, 1752, **1753**, 1913, 1935,
 2429
 <complex.h>, 635, **635**, 2453
 <concepts>, 584, **638**, 2429
 <condition_variable>, **2262**, 2440
 <contracts>, 585, 587, 616, **616**, 2416
 <coroutine>, 585, 628, **628**, 2429
 <csetjmp>, 573, 632, **633**, 2454
 <csignal>, 632, **634**
 <csdarg>, 239, 573, 632, **633**
 <csddef>, 155, **580**, 584, 2453, 2454
 <csdint>, 83, 597, **597**, 2178, 2225, 2440
 <csdio>, 600, 2036–2038, 2043, 2115, **2176**,
 2177, 2454
 <csdlib>, 101, 102, **580**, 581, 584, 585, 599, 600,
 603, 632, 636, 672, 693, 1587, 1657, 1749,
 1805, 1830, 2453, 2454
 <cstring>, 554, 585, **1633**, 2454
 <ctime>, 1638, 2032, **2032**, 2454, 2462
 <ctype.h>, **635**, 1747
 <cuchar>, 573, 1749, **1749**, 2440, 2453
 <wchar>, 573, 585, 1592, **1747**, 1749, 2453, 2454
 <cwctype>, 573, **1747**

 <debugging>, 585, **670**, 2416
 <deque>, 583–586, 588, 995, **999**, 1256

 <errno.h>, **635**, 654
 <exception>, 584, 585, 588, **612**
 <execution>, 585, 587, 588, 1431, 1432, **2298**,
 2434
 <expected>, 584, 585, 588, **889**, 2420

 <fenv.h>, **635**, 1752
 <filesystem>, 584, 585, **2130**, 2434, 2462
 <flat_map>, 584–586, 1089, **1101**, 1256, 2420
 <flat_set>, 584, 585, 1089, **1122**, 1256, 2421
 <float.h>, 597, **635**, 2457
 <format>, 584, 585, **1684**, **1698**, 2429, 2461
 <forward_list>, 583–586, 588, 995, **1004**, 1256,
 2440
 <fstream>, 585, 2112, **2112**, 2174
 <functional>, 584–588, **917**
 <future>, **2274**, 2440

 <generator>, 586, 1259, **1420**, 2421

 <hazard_pointer>, 586, **2292**, 2417
 <hive>, 586, 995, **1011**, 1256, 2417

 <initializer_list>, 586, **618**, 2440
 <inplace_vector>, 584, 586, 589, 995, **1036**,
 1256, 2417
 <inttypes.h>, **635**, 2178
 <iomanip>, 587, **2062**, 2083, 2084
 <ios>, 586, **2038**
 <iosfwd>, 584, 588, **2034**, 2036
 <iostream>, 2036, **2036**, 2037, 2043
 <iso646.h>, 558, 635, **635**, 2454
 <istream>, 584, **2061**, 2063, 2464
 <iterator>, 583–588, **1192**, 1256, 1268, 2460

 <latch>, 586, **2271**, 2429
 <limits>, 84, 583, 584, **589**, 2457
 <limits.h>, 596, **635**
 <linalg>, 586, **1838**, 2417
 <list>, 583–586, 589, 995, **1020**, 1256
 <locale>, 584, **1637**, 1638
 <locale.h>, **635**, 1676

 <map>, 583–586, 588, 1042, **1042**, 1256
 <math.h>, **635**, 1830
 <mdspan>, 583, 585, 586, 588, 589, 1138, **1144**,
 2421
 <memory>, 583–589, 672, **672**, 1579, 2234
 <memory_resource>, 586, 587, **737**, 2434
 <meta>, 583, 585–587, **779**, 2417
 <mutex>, 587, **2244**, 2440

 <new>, 75, 559, 584–586, **600**
 <numbers>, 586, **1837**, 2429
 <numeric>, 583–587, **1565**

<optional>, 584, 585, 587, 589, **852**, 1256, 2434
 <ostream>, 584, 587, **2061**, 2074, 2465

 <print>, 587, **2063**, 2421

 <queue>, 583–585, 1089, **1090**

 <random>, 585, 587, 1586, **1760**, 2440
 <ranges>, 583, 585, 587–589, 849, **1259**, 1268, 2429, 2460
 <ratio>, 585, 587, **817**, 2440
 <rcu>, 587, **2289**, 2417
 <regex>, 586, 1256, 1715, **1715**, 2440

 <scoped_allocator>, 583, **745**, 2440
 <semaphore>, 587, **2269**, 2429
 <set>, 583–586, 1042, **1054**, 1256
 <setjmp.h>, 633, **635**
 <shared_mutex>, 587, **2244**, 2437
 <signal.h>, 634, **635**
 <simd>, 588, 1256, **1893**, 2417
 <source_location>, 588, **610**, 2429
 , 588, 589, 1138, **1138**, 1256, 2429
 <spanstream>, 588, **2104**, 2418, 2421
 <sstream>, 588, **2088**, 2089
 <stack>, 583–585, 1089, **1098**
 <stacktrace>, 585, 588, **664**, 1256, 2421
 <stdalign.h>, 634, 635, **635**
 <stdarg.h>, 633, **635**
 <stdatomic.h>, 588, 635, **2242**, 2244, 2421
 <stdbit.h>, 559, 588, 635, **957**, 959, 2417
 <stdbool.h>, 634, 635, **635**, 2446, 2457
 <stdckdint.h>, 559, 588, 635, **1940**, 2417
 <stddef.h>, 580, 582, **635**, 2454
 <stdexcept>, 584, **650**
 <stdfloat>, 86, 599, **599**, 2421
 <stdint.h>, 598, **635**, 2178
 <stdio.h>, **635**, 2177
 <stdlib.h>, 581, **635**, 636
 <stop_token>, 586, **2184**, 2429
 <streambuf>, 2053, **2053**
 <string>, 583–586, 588, 1256, 1591, **1604**
 <string.h>, **635**, 1633
 <string_view>, 584, 585, 588, 1256, **1594**, 2434
 <syncstream>, 588, **2124**, 2125, 2429
 <system_error>, 585, **655**, 657, 2440

 <text_encoding>, 588, **1677**, 2417
 <tgmath.h>, 635, **635**, 2453
 <thread>, 585, 586, 588, **2192**, 2440
 <time.h>, **635**, 2032
 <tuple>, 583–588, 824, 838, **838**, 849, 2440, 2459, 2460
 <type_traits>, 583, 584, 586–588, **751**, 2440, 2458
 <typeindex>, 607, **608**, 2440
 <typeinfo>, 585, 607, **607**

 <uchar.h>, **635**, 1749

 <unordered_map>, 583–586, 588, 1063, **1063**, 1256, 2440
 <unordered_set>, 583–586, 1063, **1078**, 1256, 2440
 <utility>, 563, 583–588, **821**, 838, 849, 2440, 2459, 2460

 <valarray>, 588, 589, 1256, **1805**, 1807
 <variant>, 584, 585, 588, **871**, 2434, 2460
 <vector>, 583, 585, 586, 589, 995, **1027**, 1256
 <version>, 583, **583**, 589, 2429

 <wchar.h>, **635**, 1749
 <wctype.h>, **635**, 1747

Index of library names

Constructions whose name appears in *italics* are for exposition only.

Symbols

FWD-ENV-T, 2312

FWD-ENV, 2312

MAP_RANK, 1183

abandon

execution::*spawn-future-state*, 2352

advance-bwd

concat_view::*iterator*, 1346

advance-fwd

concat_view::*iterator*, 1346

basic-sender

get_completion_signatures, 2318

call-with-default, 2319

canonical-index, 1183

canonical-slice, 1184

check-types

default-impls, 2317

impls-for<decayed-typeof<let-cpo>>, 2336

impls-for<decayed-typeof<then-cpo>>, 2334

impls-for<decayed-typeof<read_env>>, 2328

impls-for<bulk_t>, 2340

impls-for<into_variant_t>, 2344

impls-for<schedule_from_t>, 2331

impls-for<stopped_as_optional_t>, 2345

impls-for<when_all_t>, 2341

comp-name

text_encoding, 1679

complete

execution::*spawn-future-state*, 2351

execution::*spawn-state*, 2357

consume

execution::*spawn-future-state*, 2351

count-of

execution::*completion_signatures*, 2359

decay-copyable-result-datums, 2318

default-impls

check-types, 2317

destroy

execution::*spawn-future-state*, 2352

disassociate

execution::*simple_counting_scope*, 2379

distance-from

cartesian_product_view, 1412

find-next

chunk_by_view, 1401

split_view, 1339

find-prev

chunk_by_view, 1401

for-each

execution::*completion_signatures*, 2359

get-element

elements_view::*iterator*, 1357

get-inner

join_with_view::*iterator*, 1330

get-stop-token

task::*state*, 2373

impls-for<decayed-typeof<let-cpo>>, 2335

check-types, 2336

impls-for<decayed-typeof<then-cpo>>, 2334

check-types, 2334

impls-for<decayed-typeof<read_env>>, 2327

check-types, 2328

impls-for<write-env-t>, 2329

impls-for<bulk_chunked_t>, 2338

impls-for<bulk_t>

check-types, 2340

impls-for<continues_on_t>, 2330

impls-for<into_variant_t>, 2344

check-types, 2344

impls-for<schedule_from_t>

check-types, 2331

impls-for<stopped_as_optional_t>, 2345

check-types, 2345

impls-for<when_all_t>, 2341

check-types, 2341

inline-attrs, 2319

is-dependent-sender-helper, 2319

make-when-all-env, 2341

next

cartesian_product_view, 1411

not-a-scheduler, 2318

not-a-sender, 2318

get_completion_signatures, 2318

schedule, 2318

outer

join_view::*iterator*, 1324

join_with_view::*iterator*, 1330

overload-set, 2318

prev

cartesian_product_view, 1412

concat_view::*iterator*, 1346

run

execution::*spawn-state*, 2357

satisfy

concat_view::*iterator*, 1345

join_view::*iterator*, 1324

join_with_view::*iterator*, 1330

start-join-sender

execution::*simple_counting_scope*, 2379

synth-three-way-result
 indirect, 731
tag-bit-width, 695
try-associate
 execution::counting_scope, 2380
 execution::simple_counting_scope, 2379
try-set-stopped
 execution::spawn-future-state, 2351
unspecified-exception, 2312
update-inner
 join_with_view::iterator, 1330
 _Atomic, 2242, 2244
 _Exit, 580, 599
 _IOFBF, 2176
 _IOLBF, 2176
 _IONBF, 2176
 _PRINTF_NAN_LEN_MAX, 2176
 __STDC_ENDIAN_BIG__, 957
 __STDC_ENDIAN_LITTLE__, 957
 __STDC_ENDIAN_NATIVE__, 957
 __STDC_VERSION_STDBIT_H__, 957
 __STDC_VERSION_STDCKDINT_H__, 1940
 __cpp_lib_adaptor_iterator_pair_
 constructor, 583
 __cpp_lib_addressof_constexpr, 583
 __cpp_lib_algorithm_default_value_type,
 583
 __cpp_lib_algorithm_iterator_
 requirements, 583
 __cpp_lib_aligned_accessor, 583
 __cpp_lib_allocate_at_least, 583
 __cpp_lib_allocator_traits_is_always_
 equal, 583
 __cpp_lib_any, 583
 __cpp_lib_apply, 583
 __cpp_lib_array_constexpr, 583
 __cpp_lib_as_const, 583
 __cpp_lib_associative_heterogeneous_
 erasure, 583
 __cpp_lib_associative_heterogeneous_
 insertion, 584
 __cpp_lib_assume_aligned, 584
 __cpp_lib_atomic_flag_test, 584
 __cpp_lib_atomic_float, 584
 __cpp_lib_atomic_is_always_lock_free, 584
 __cpp_lib_atomic_lock_free_type_aliases,
 584
 __cpp_lib_atomic_min_max, 584
 __cpp_lib_atomic_reductions, 584
 __cpp_lib_atomic_ref, 584
 __cpp_lib_atomic_shared_ptr, 584
 __cpp_lib_atomic_value_initialization, 584
 __cpp_lib_atomic_wait, 584
 __cpp_lib_barrier, 584
 __cpp_lib_bind_back, 584
 __cpp_lib_bind_front, 584
 __cpp_lib_bit_cast, 584
 __cpp_lib_bitops, 584
 __cpp_lib_bitset, 584
 __cpp_lib_bool_constant, 584
 __cpp_lib_bounded_array_traits, 584
 __cpp_lib_boyer_moore_searcher, 584
 __cpp_lib_byte, 584
 __cpp_lib_byteswap, 584
 __cpp_lib_char8_t, 584
 __cpp_lib_chrono, 584
 __cpp_lib_chrono_udls, 584
 __cpp_lib_clamp, 584
 __cpp_lib_common_reference, 584
 __cpp_lib_common_reference_wrapper, 584
 __cpp_lib_complex_udls, 584
 __cpp_lib_concepts, 584
 __cpp_lib_constant_wrapper, 584
 __cpp_lib_constexpr_algorithms, 584
 __cpp_lib_constexpr_atomic, 584
 __cpp_lib_constexpr_bitset, 584
 __cpp_lib_constexpr_charconv, 584
 __cpp_lib_constexpr_cmath, 584
 __cpp_lib_constexpr_complex, 584
 __cpp_lib_constexpr_deque, 584
 __cpp_lib_constexpr_dynamic_alloc, 584
 __cpp_lib_constexpr_exceptions, 584
 __cpp_lib_constexpr_flat_map, 584
 __cpp_lib_constexpr_flat_set, 584
 __cpp_lib_constexpr_format, 584
 __cpp_lib_constexpr_forward_list, 584
 __cpp_lib_constexpr_functional, 584
 __cpp_lib_constexpr_inplace_vector, 584
 __cpp_lib_constexpr_iterator, 584
 __cpp_lib_constexpr_list, 584
 __cpp_lib_constexpr_map, 584
 __cpp_lib_constexpr_memory, 584
 __cpp_lib_constexpr_new, 584
 __cpp_lib_constexpr_numeric, 584
 __cpp_lib_constexpr_queue, 584
 __cpp_lib_constexpr_set, 584
 __cpp_lib_constexpr_stack, 584
 __cpp_lib_constexpr_string, 584
 __cpp_lib_constexpr_string_view, 584
 __cpp_lib_constexpr_tuple, 584
 __cpp_lib_constexpr_typeinfo, 585
 __cpp_lib_constexpr_unordered_map, 585
 __cpp_lib_constexpr_unordered_set, 585
 __cpp_lib_constexpr_utility, 585
 __cpp_lib_constexpr_vector, 585
 __cpp_lib_constrained_equality, 585
 __cpp_lib_containers_ranges, 585
 __cpp_lib_contracts, 585
 __cpp_lib_copyable_function, 585
 __cpp_lib_coroutine, 585
 __cpp_lib_counting_scope, 585
 __cpp_lib_debugging, 585
 __cpp_lib_define_static, 585
 __cpp_lib_destroying_delete, 585
 __cpp_lib_enable_shared_from_this, 585
 __cpp_lib_endian, 585
 __cpp_lib_erase_if, 585
 __cpp_lib_exception_ptr_cast, 585

```

__cpp_lib_exchange_function, 585
__cpp_lib_execution, 585
__cpp_lib_expected, 585
__cpp_lib_filesystem, 585
__cpp_lib_flat_map, 585
__cpp_lib_flat_set, 585
__cpp_lib_format, 585
__cpp_lib_format_path, 585
__cpp_lib_format_ranges, 585
__cpp_lib_format_uchar, 585
__cpp_lib_formatters, 585
__cpp_lib_forward_like, 585
__cpp_lib_freestanding_algorithm, 585
__cpp_lib_freestanding_array, 585
__cpp_lib_freestanding_char_traits, 585
__cpp_lib_freestanding_charconv, 585
__cpp_lib_freestanding_cstdlib, 585
__cpp_lib_freestanding_cstring, 585
__cpp_lib_freestanding_cwchar, 585
__cpp_lib_freestanding_errc, 585
__cpp_lib_freestanding_execution, 585
__cpp_lib_freestanding_expected, 585
__cpp_lib_freestanding_feature_test_ -
    macros, 585
__cpp_lib_freestanding_functional, 585
__cpp_lib_freestanding_iterator, 585
__cpp_lib_freestanding_mdspan, 585
__cpp_lib_freestanding_memory, 585
__cpp_lib_freestanding_numeric, 585
__cpp_lib_freestanding_operator_new, 585
__cpp_lib_freestanding_optional, 585
__cpp_lib_freestanding_random, 585
__cpp_lib_freestanding_ranges, 585
__cpp_lib_freestanding_ratio, 585
__cpp_lib_freestanding_string_view, 585
__cpp_lib_freestanding_tuple, 585
__cpp_lib_freestanding_utility, 585
__cpp_lib_freestanding_variant, 585
__cpp_lib_fstream_native_handle, 585
__cpp_lib_function_ref, 585
__cpp_lib_gcd_lcm, 585
__cpp_lib_generator, 586
__cpp_lib_generic_associative_lookup, 586
__cpp_lib_generic_unordered_lookup, 586
__cpp_lib_harden_array, 588
__cpp_lib_harden_basic_stacktrace, 588
__cpp_lib_harden_basic_string, 588
__cpp_lib_harden_basic_string_view, 588
__cpp_lib_harden_bitset, 588
__cpp_lib_harden_common_iterator, 588
__cpp_lib_harden_counted_iterator, 588
__cpp_lib_harden_deque, 588
__cpp_lib_harden_expected, 588
__cpp_lib_harden_forward_list, 588
__cpp_lib_harden_inplace_vector, 589
__cpp_lib_harden_list, 589
__cpp_lib_harden_mdspan, 589
__cpp_lib_harden_optional, 589
__cpp_lib_harden_shared_ptr_array, 589
__cpp_lib_harden_span, 589
__cpp_lib_harden_valarray, 589
__cpp_lib_harden_vector, 589
__cpp_lib_harden_view_interface, 589
__cpp_lib_hardware_interference_size, 586
__cpp_lib_has_unique_object_ -
    representations, 586
__cpp_lib_hazard_pointer, 586
__cpp_lib_hive, 586
__cpp_lib_hypot, 586
__cpp_lib_incomplete_container_elements,
    586
__cpp_lib_indirect, 586
__cpp_lib_initializer_list, 586
__cpp_lib_inplace_vector, 586
__cpp_lib_int_pow2, 586
__cpp_lib_integer_comparison_functions,
    586
__cpp_lib_integer_sequence, 586
__cpp_lib_integral_constant_callable, 586
__cpp_lib_interpolate, 586
__cpp_lib_invoke, 586
__cpp_lib_invoke_r, 586
__cpp_lib_ios_noreplace, 586
__cpp_lib_is_aggregate, 586
__cpp_lib_is_constant_evaluated, 586
__cpp_lib_is_final, 586
__cpp_lib_is_implicit_lifetime, 586
__cpp_lib_is_invocable, 586
__cpp_lib_is_layout_compatible, 586
__cpp_lib_is_nothrow_convertible, 586
__cpp_lib_is_null_pointer, 586
__cpp_lib_is_pointer_interconvertible, 586
__cpp_lib_is_scoped_enum, 586
__cpp_lib_is_structural, 586
__cpp_lib_is_sufficiently_aligned, 586
__cpp_lib_is_swappable, 586
__cpp_lib_is_virtual_base_of, 586
__cpp_lib_is_within_lifetime, 586
__cpp_lib_jthread, 586
__cpp_lib_latch, 586
__cpp_lib_laundry, 586
__cpp_lib_linalg, 586
__cpp_lib_list_remove_return_type, 586
__cpp_lib_logical_traits, 586
__cpp_lib_make_from_tuple, 586
__cpp_lib_make_reverse_iterator, 586
__cpp_lib_make_unique, 586
__cpp_lib_map_lookup, 586
__cpp_lib_map_try_emplace, 586
__cpp_lib_math_constants, 586
__cpp_lib_math_special_functions, 586
__cpp_lib_mdspan, 586
__cpp_lib_mdspan_copy, 586
__cpp_lib_memory_resource, 586
__cpp_lib_modules, 586
__cpp_lib_move_iterator_concept, 586
__cpp_lib_move_only_function, 586
__cpp_lib_node_extract, 586

```


__cpp_lib_nonmember_container_access, 586
 __cpp_lib_not_fn, 587
 __cpp_lib_null_iterators, 587
 __cpp_lib_observable_checkpoint, 587
 __cpp_lib_optional, 587
 __cpp_lib_optional_range_support, 587
 __cpp_lib_out_ptr, 587
 __cpp_lib_parallel_algorithm, 587
 __cpp_lib_parallel_scheduler, 587
 __cpp_lib_philox_engine, 587
 __cpp_lib_pointer_tag_pair, 587
 __cpp_lib_polymorphic, 587
 __cpp_lib_polymorphic_allocator, 587
 __cpp_lib_print, 587
 __cpp_lib_quoted_string_io, 587
 __cpp_lib_ranges, 587
 __cpp_lib_ranges_as_const, 587
 __cpp_lib_ranges_as_input, 587
 __cpp_lib_ranges_as_rvalue, 587
 __cpp_lib_ranges_cache_latest, 587
 __cpp_lib_ranges_cartesian_product, 587
 __cpp_lib_ranges_chunk, 587
 __cpp_lib_ranges_chunk_by, 587
 __cpp_lib_ranges_concat, 587
 __cpp_lib_ranges_contains, 587
 __cpp_lib_ranges_enumerate, 587
 __cpp_lib_ranges_filter, 587
 __cpp_lib_ranges_find_last, 587
 __cpp_lib_ranges_fold, 587
 __cpp_lib_ranges_generate_random, 587
 __cpp_lib_ranges_indices, 587
 __cpp_lib_ranges_iota, 587
 __cpp_lib_ranges_join_with, 587
 __cpp_lib_ranges_repeat, 587
 __cpp_lib_ranges_reserve_hint, 587
 __cpp_lib_ranges_slide, 587
 __cpp_lib_ranges_starts_ends_with, 587
 __cpp_lib_ranges_stride, 587
 __cpp_lib_ranges_to_container, 587
 __cpp_lib_ranges_zip, 587
 __cpp_lib_ratio, 587
 __cpp_lib_raw_memory_algorithms, 587
 __cpp_lib_rcu, 587
 __cpp_lib_reference_from_temporary, 587
 __cpp_lib_reference_wrapper, 587
 __cpp_lib_reflection, 587
 __cpp_lib_remove_cvref, 587
 __cpp_lib_replaceable_contract_violation_handler, 587
 __cpp_lib_result_of_sfinae, 587
 __cpp_lib_robust_nonmodifying_seq_ops, 587
 __cpp_lib_sample, 587
 __cpp_lib_saturation_arithmetic, 587
 __cpp_lib_scoped_lock, 587
 __cpp_lib_semaphore, 587
 __cpp_lib_senders, 587
 __cpp_lib_shared_mutex, 587
 __cpp_lib_shared_ptr_arrays, 587
 __cpp_lib_shared_ptr_weak_type, 587
 __cpp_lib_shared_timed_mutex, 587
 __cpp_lib_shift, 587
 __cpp_lib_simd, 588
 __cpp_lib_simd_bitops, 588
 __cpp_lib_simd_complex, 588
 __cpp_lib_simd_permutations, 588
 __cpp_lib_smart_ptr_for_overwrite, 588
 __cpp_lib_smart_ptr_owner_equality, 588
 __cpp_lib_source_location, 588
 __cpp_lib_span, 588
 __cpp_lib_spanstream, 588
 __cpp_lib_ssize, 588
 __cpp_lib_sstream_from_string_view, 588
 __cpp_lib_stacktrace, 588
 __cpp_lib_start_lifetime, 588
 __cpp_lib_start_lifetime_as, 588
 __cpp_lib_starts_ends_with, 588
 __cpp_lib_stdatomic_h, 588
 __cpp_lib_stdbit_h, 588
 __cpp_lib_stdckdint_h, 588
 __cpp_lib_string_contains, 588
 __cpp_lib_string_resize_and_overwrite, 588
 __cpp_lib_string_subview, 588
 __cpp_lib_string_udls, 588
 __cpp_lib_string_view, 588
 __cpp_lib_submdspan, 588
 __cpp_lib_syncbuf, 588
 __cpp_lib_task, 588
 __cpp_lib_text_encoding, 588
 __cpp_lib_thread_attributes, 588
 __cpp_lib_three_way_comparison, 588
 __cpp_lib_to_address, 588
 __cpp_lib_to_array, 588
 __cpp_lib_to_chars, 588
 __cpp_lib_to_string, 588
 __cpp_lib_to_underlying, 588
 __cpp_lib_transformation_trait_aliases, 588
 __cpp_lib_transparent_operators, 588
 __cpp_lib_tuple_element_t, 588
 __cpp_lib_tuple_like, 588
 __cpp_lib_tuples_by_type, 588
 __cpp_lib_type_identity, 588
 __cpp_lib_type_order, 588
 __cpp_lib_type_trait_variable_templates, 588
 __cpp_lib_uncaught_exceptions, 588
 __cpp_lib_unordered_map_try_emplace, 588
 __cpp_lib_unreachable, 588
 __cpp_lib_unwrap_ref, 588
 __cpp_lib_valarray, 588
 __cpp_lib_variant, 588
 __cpp_lib_view_interface, 588
 __cpp_lib_void_t, 588

Numbers
 _1, 935, 936
 _2, 935, 936

_3, 935, 936
 _4, 935, 936
 _5, 935, 936
 _6, 935, 936
 _7, 935, 936
 _8, 935, 936
 _9, 935, 936
 _10, 935, 936

A

a

- cauchy_distribution, 1798
- extreme_value_distribution, 1794
- uniform_int_distribution, 1785
- uniform_real_distribution, 1786
- weibull_distribution, 1793

abbrev

- sys_info, 2014

abi_type

- basic_mask, 1935
- basic_vec, 1910

abort, 102, 201, 580, 599, 606, 613

abs, 580, 1830, 2177

- complex, 1757
- duration, 1963
- simd, 1925, 1928, 1934
- valarray, 1817

absolute, 2164

access

- aligned_accessor, 1175
- conjugated_accessor, 1861
- default_accessor, 1173
- scaled_accessor, 1860

access_context, 801

- current, 801
- designating_class, 801
- scope, 801
- unchecked, 802
- unprivileged, 802
- via, 802

accessor

- mdspan, 1176

accessor_type

- mdspan, 1175

accumulate, 1569

acos, 1824

- complex, 1757
- simd, 1928, 1934
- valarray, 1817

acosf, 1824

acosh, 1824

- complex, 1757
- simd, 1928, 1934

acoshf, 1824

acoshl, 1824

acosl, 1824

acq_rel

- memory_order, 2207

acquire

- counting_semaphore, 2270
- memory_order, 2207

add, 1868

- perm_options, 2154

add_const, 771, 815

add_const_t, 754

add_cv, 772, 815

add_cv_t, 754

add_lvalue_reference, 772, 815

add_lvalue_reference_t, 754

add_pointer, 773, 815

add_pointer_t, 754

add_rvalue_reference, 772, 815

add_rvalue_reference_t, 754

add_volatile, 772, 815

add_volatile_t, 754

address

- atomic_ref<T>, 2213
- coroutine_handle, 630
- coroutine_handle<noop_coroutine_-\n promise>, 632

address_family_not_supported

- errc, 655

address_in_use

- errc, 655

address_not_available

- errc, 655

addressof, 693

adjacent

- views, 1375

adjacent_difference, 1577

adjacent_find, 1498

adjacent_transform

- views, 1380

adjacent_transform_view, 1381

- base, 1381
- begin, 1381
- constructor, 1382
- end, 1381
- reserve_hint, 1381
- size, 1381

adjacent_transform_view::iterator, 1382

- constructor, 1383
- operator*, 1383
- operator+, 1384
- operator++, 1383
- operator+=, 1384
- operator-, 1384
- operator--, 1383
- operator-=, 1384
- operator==, 1384
- operator[], 1384

adjacent_transform_view::sentinel, 1384

- constructor, 1385
- operator-, 1385
- operator==, 1385

adjacent_view, 1375

- begin, 1375

- constructor, 1376
 - end, 1375
 - reserve_hint, 1376
 - size, 1376
- adjacent_view::iterator, 1376
 - constructor, 1377, 1378
 - iter_move, 1379
 - iter_swap, 1379
 - operator*, 1378
 - operator+, 1379
 - operator++, 1378
 - operator+=, 1378
 - operator-, 1379
 - operator--, 1378
 - operator==, 1378
 - operator<, 1378
 - operator<=, 1379
 - operator<=>, 1379
 - operator==, 1378
 - operator>, 1379
 - operator>=, 1379
 - operator[], 1378
- adjacent_view::sentinel, 1379
 - constructor, 1380
 - operator-, 1380
 - operator==, 1380
- AdobeStandardEncoding
 - text_encoding, 1682
- adopt_lock, 2252
- adopt_lock_t, 2252
- advance, 1222, 1223
 - subrange, 1281
- advance_to
 - basic_format_context, 1703
 - basic_format_parse_context, 1701
- affine, 2303
- affine_t, 2303
- aliases
 - text_encoding, 1679
- aliases_view
 - text_encoding, 1680
- align, 685
- align_val_t, 600
- aligned_accessor, 1174
 - access, 1175
 - byte_alignment, 1174
 - constructor, 1175
 - data_handle_type, 1174
 - element_type, 1174
 - offset, 1175
 - offset_policy, 1174
 - operator default_accessor, 1175
 - reference, 1174
- aligned_alloc, 580, 693, 2454
- aligned_storage, 2458
- aligned_storage_t, 2458
- aligned_union, 2459
- aligned_union_t, 2458
- alignment
 - simd, 1907
- alignment_of, 768, 806
- alignment_of_v, 757
- alignment_v
 - simd, 1893
- all
 - bitset, 915
 - perms, 2154
 - views, 1301
- all_of, 1489
 - simd, 1939, 1940
- all_t, 1259
- allocate
 - allocator, 692
 - allocator_traits, 691
 - memory_resource, 738
 - polymorphic_allocator, 739
 - scoped_allocator_adaptor, 748
- allocate_at_least
 - allocator, 693
 - allocator_traits, 691
- allocate_bytes
 - polymorphic_allocator, 740
- allocate_object
 - polymorphic_allocator, 740
- allocate_shared, 712–715
- allocation_result, 674
 - count, 674
 - ptr, 674
- allocator, 692
 - allocate, 692
 - allocate_at_least, 693
 - deallocate, 693
 - difference_type, 692
 - operator=, 692
 - operator==, 693
 - propagate_on_container_move_assignment, 692
 - size_type, 692
 - value_type, 692
- allocator_arg, 687
- allocator_arg_t, 687
- allocator_traits, 690
 - allocate, 691
 - allocate_at_least, 691
 - const_pointer, 691
 - const_void_pointer, 691
 - construct, 692
 - deallocate, 691
 - destroy, 692
 - difference_type, 691
 - is_always_equal, 691
 - max_size, 692
 - pointer, 690
 - propagate_on_container_copy_assignment, 691
 - propagate_on_container_move_assignment, 691
 - propagate_on_container_swap, 691

- rebind_alloc, 691
- select_on_container_copy_construction, 692
- size_type, 691
- void_pointer, 691
- allocator_type
 - allocator-aware containers, 966
 - basic_string, 1607
- alnum
 - ctype_base, 1645
- alpha
 - ctype_base, 1645
 - gamma_distribution, 1792
- already_connected
 - errc, 655
- always_noconv
 - codecvt, 1651
- ambiguous
 - local_info, 2014
- ambiguous_local_time, 2013
 - constructor, 2013
- Amiga1251
 - text_encoding, 1684
- and_then
 - expected, 901
 - expected<void>, 908
 - optional, 861
- annotations_of, 807
- annotations_of_with_type, 808
- any
 - constructor, 885, 886
 - destructor, 886
 - emplace, 886, 887
 - has_value, 887
 - operator=, 886
 - reset, 887
 - swap, 887
 - type, 887
- any (member)
 - bitset, 916
- any_cast, 888
- any_of, 1489
 - simd, 1939, 1940
- append
 - basic_string, 1618, 1619
 - path, 2142, 2143
- append_range
 - basic_string, 1619
 - inplace_vector, 1040
- apply, 848
 - valarray, 1814
- apply_givens_rotation, 1867
- apply_result, 775
- apply_result_t, 755
- apply_sender, 2324
 - default_domain, 2323
- approximately_sized_range, 1274
- arg, 1759
 - basic_format_context, 1702
 - complex, 1757
 - simd, 1934
- argument_list_too_long
 - errc, 655
- argument_out_of_domain
 - errc, 655
- argument_type
 - zombie, 573
- array, 996, 997, 999
 - begin, 962, 997
 - cbegin, 962
 - cend, 962
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - crbegin, 965
 - crend, 965
 - data, 998
 - difference_type, 961
 - empty, 963
 - end, 962, 997
 - fill, 998
 - get, 999
 - iterator, 961
 - max_size, 963, 997
 - operator!=, 962
 - operator<=, 965
 - operator=, 961
 - operator==, 962
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reverse_iterator, 964
 - size, 963, 997, 998
 - size_type, 961
 - swap, 962, 998
 - value_type, 961
- arrive
 - barrier, 2274
- arrive_and_drop
 - barrier, 2274
- arrive_and_wait
 - barrier, 2274
 - latch, 2272
- as_awaitable, 2303
- as_awaitable_t, 2303
- as_bytes, 1144
- as_const, 826
- as_const_view::iterator, 1353
 - constructor, 1354
- as_input_view, 1417
 - begin, 1418
 - constructor, 1418
 - end, 1418
 - reserve_hint, 1418
 - size, 1418
- as_input_view::iterator, 1418
 - base, 1419
 - constructor, 1419

- iter_move, 1420
 - iter_swap, 1420
 - operator++, 1419
 - operator-, 1419
 - operator==, 1419
- as_rvalue
 - views, 1303
- as_rvalue_view, 1303
 - constructor, 1304
- as_writable_bytes, 1144
- ASCII
 - text_encoding, 1680
- asin, 1824
 - complex, 1757
 - simd, 1928, 1934
 - valarray, 1817
- asinf, 1824
- asinh, 1825
 - complex, 1758
 - simd, 1928, 1934
- asinhf, 1825
- asinhf, 1825
- asinl, 1824
- assert, 653
- assign
 - basic_regex, 1727, 1728
 - basic_string, 1619, 1620
 - directory_entry, 2157
 - error_code, 660
 - error_condition, 662
 - path, 2142
 - sequence containers, 970
- assign_range
 - basic_string, 1620
 - sequence containers, 970
- assignable_from, 642
- assoc_laguerre, 1831
 - simd, 1931
- assoc_laguerref, 1831
- assoc_laguerrel, 1831
- assoc_legendre, 1832
 - simd, 1931
- assoc_legendref, 1832
- assoc_legendrel, 1832
- associate, 2302
- associate_t, 2302
- assume_aligned, 685
- async, 2284
 - launch, 2275
- at
 - basic_stacktrace, 669
 - basic_string, 1617
 - basic_string_view, 1599
 - flat_map, 1110
 - map, 1048
 - mdspan, 1180
 - span, 1143
 - unordered_map, 1070
 - view_interface, 1278
- at_quick_exit, 580, 600
- atan, 1824
 - complex, 1757
 - simd, 1928, 1934
 - valarray, 1817
- atan2, 1824
 - simd, 1928
 - valarray, 1817
- atan2f, 1824
- atan2l, 1824
- atanf, 1824
- atanh, 1825
 - complex, 1758
 - simd, 1928, 1934
- atanhf, 1825
- atanhl, 1825
- atanl, 1824
- atexit, 101, 580, 599
- atof, 580
- atoi, 580
- atol, 580
- atoll, 580
- atomic, 2220, 2221
 - compare_exchange_strong, 2222
 - compare_exchange_weak, 2222
 - constructor, 2221
 - exchange, 2222
 - is_always_lock_free, 2221
 - is_lock_free, 2222
 - load, 2222
 - notify_all, 2224
 - notify_one, 2224
 - operator *type*, 2222
 - operator=, 2222
 - store, 2222
 - value_type, 2220
 - wait, 2224
- atomic<floating-point-type>, 2228
 - compare_exchange_strong, 2222
 - compare_exchange_weak, 2222
 - constructor, 2221
 - exchange, 2222
 - fetch_add, 2230
 - fetch_fmaximum, 2230
 - fetch_fmaximum_num, 2230
 - fetch_fminimum, 2230
 - fetch_fminimum_num, 2230
 - fetch_max, 2230
 - fetch_min, 2230
 - fetch_sub, 2230
 - is_always_lock_free, 2221
 - is_lock_free, 2222
 - load, 2222
 - notify_all, 2224
 - notify_one, 2224
 - operator *floating-point-type*, 2222
 - operator+=, 2231
 - operator-=, 2231
 - operator=, 2222

```

store, 2222
store_fmaximum, 2230
store_fmaximum_num, 2230
store_fminimum, 2230
store_fminimum_num, 2230
store_max, 2230
store_min, 2230
wait, 2224
atomic<integral-type>, 2225
    compare_exchange_strong, 2222
    compare_exchange_weak, 2222
    constructor, 2221
    exchange, 2222
    fetch_add, 2227
    fetch_and, 2227
    fetch_max, 2227
    fetch_min, 2227
    fetch_or, 2227
    fetch_sub, 2227
    fetch_xor, 2227
    is_always_lock_free, 2221
    is_lock_free, 2222
    load, 2222
    notify_all, 2224
    notify_one, 2224
    operator integral-type, 2222
    operator++, 2234
    operator+=, 2228
    operator--, 2234
    operator-=, 2228
    operator=, 2222
    operator&=, 2228
    operator^=, 2228
    operator|=, 2228
    store, 2222
    store_add, 2227
    store_and, 2227
    store_max, 2227
    store_min, 2227
    store_or, 2227
    store_sub, 2227
    wait, 2224
atomic<shared_ptr<T>>, 2235
    compare_exchange_strong, 2236
    compare_exchange_weak, 2236
    constructor, 2235
    exchange, 2236
    is_always_lock_free, 2221
    is_lock_free, 2222
    load, 2236
    notify_all, 2237
    notify_one, 2237
    operator shared_ptr<T>, 2236
    operator=, 2236
    store, 2235
    wait, 2237
atomic<T*>, 2231, 2233
    compare_exchange_strong, 2222
    compare_exchange_weak, 2222
    constructor, 2221
    exchange, 2222
    fetch_add, 2233
    fetch_max, 2233
    fetch_min, 2233
    fetch_sub, 2233
    is_always_lock_free, 2221
    is_lock_free, 2222
    load, 2222
    notify_all, 2224
    notify_one, 2224
    operator T*, 2222
    operator++, 2234
    operator+=, 2228, 2231, 2233
    operator--, 2234
    operator-=, 2228, 2231, 2233
    operator=, 2222
    store, 2222
    store_add, 2233
    store_max, 2233
    store_min, 2233
    store_sub, 2233
    wait, 2224
atomic<weak_ptr<T>>, 2237
    compare_exchange_strong, 2239
    compare_exchange_weak, 2238, 2239
    constructor, 2238
    exchange, 2238
    is_always_lock_free, 2221
    is_lock_free, 2222
    load, 2238
    notify_all, 2239
    notify_one, 2239
    operator weak_ptr<T>, 2238
    operator=, 2238
    store, 2238
    wait, 2239
atomic_bool, 2207, 2242
ATOMIC_BOOL_LOCK_FREE, 2202, 2209, 2242
atomic_char, 2207, 2242
atomic_char16_t, 2207, 2243
ATOMIC_CHAR16_T_LOCK_FREE, 2202, 2209, 2242
atomic_char32_t, 2207, 2243
ATOMIC_CHAR32_T_LOCK_FREE, 2202, 2209, 2242
atomic_char8_t, 2207, 2243
ATOMIC_CHAR8_T_LOCK_FREE, 2202, 2209, 2242
ATOMIC_CHAR_LOCK_FREE, 2202, 2209, 2242
atomic_compare_exchange_strong, 2222, 2243
atomic_compare_exchange_strong_explicit,
    2222, 2243
atomic_compare_exchange_weak, 2222, 2243
atomic_compare_exchange_weak_explicit,
    2222, 2243
atomic_exchange, 2222, 2243
atomic_exchange_explicit, 2222, 2243
atomic_fetch_add, 2227, 2230, 2233, 2243
atomic_fetch_add_explicit, 2227, 2230, 2233,
    2243
atomic_fetch_and, 2227, 2243

```

atomic_fetch_and_explicit, 2227, 2243
 atomic_fetch_max, 2227, 2233
 atomic_fetch_max_explicit, 2227, 2233
 atomic_fetch_min, 2227, 2233
 atomic_fetch_min_explicit, 2227, 2233
 atomic_fetch_or, 2227, 2243
 atomic_fetch_or_explicit, 2227, 2243
 atomic_fetch_sub, 2227, 2230, 2233, 2243
 atomic_fetch_sub_explicit, 2227, 2230, 2233, 2243
 atomic_fetch_xor, 2227, 2243
 atomic_fetch_xor_explicit, 2227, 2243
 atomic_flag, 2242
 clear, 2240
 constructor, 2240
 test, 2240
 test_and_set, 2240
 wait, 2240
 atomic_flag_clear, 2240, 2243
 atomic_flag_clear_explicit, 2240, 2243
 ATOMIC_FLAG_INIT, 2207, 2241, 2243
 atomic_flag_test, 2240
 atomic_flag_test_and_set, 2240, 2243
 atomic_flag_test_and_set_explicit, 2240, 2243
 atomic_flag_test_explicit, 2240
 atomic_flag_wait, 2240
 atomic_flag_wait_explicit, 2240
 atomic_init, 2464
 atomic_int, 2207, 2242
 atomic_int16_t, 2207, 2243
 atomic_int32_t, 2207, 2243
 atomic_int64_t, 2207, 2243
 atomic_int8_t, 2207, 2243
 atomic_int_fast16_t, 2207, 2243
 atomic_int_fast32_t, 2207, 2243
 atomic_int_fast64_t, 2207, 2243
 atomic_int_fast8_t, 2207, 2243
 atomic_int_least16_t, 2207, 2243
 atomic_int_least32_t, 2207, 2243
 atomic_int_least64_t, 2207, 2243
 atomic_int_least8_t, 2207, 2243
 ATOMIC_INT_LOCK_FREE, 2202, 2209, 2242
 atomic_intmax_t, 2207, 2243
 atomic_intptr_t, 2207, 2243
 atomic_is_lock_free, 2222, 2243
 atomic_llong, 2207, 2243
 ATOMIC_LLONG_LOCK_FREE, 2202, 2209, 2242
 atomic_load, 2222, 2243
 atomic_load_explicit, 2222, 2243
 atomic_long, 2207, 2242
 ATOMIC_LONG_LOCK_FREE, 2202, 2209, 2242
 ATOMIC_POINTER_LOCK_FREE, 2202, 2209, 2242
 atomic_ptrdiff_t, 2207, 2243
 atomic_ref, 2210
 compare_exchange_strong, 2212
 compare_exchange_weak, 2212
 constructor, 2211
 exchange, 2212
 is_always_lock_free, 2211
 is_lock_free, 2211
 load, 2212
 operator *type*, 2212
 operator=, 2212
 required_alignment, 2211
 store, 2211
 value_type, 2210
 atomic_ref<floating-point-type>, 2215
 compare_exchange_strong, 2212
 compare_exchange_weak, 2212
 constructor, 2211
 exchange, 2212
 fetch_add, 2217
 fetch_fmaximum, 2217
 fetch_fmaximum_num, 2217
 fetch_fminimum, 2217
 fetch_fminimum_num, 2217
 fetch_max, 2217
 fetch_min, 2217
 fetch_sub, 2217
 is_always_lock_free, 2211
 is_lock_free, 2211
 load, 2212
 operator *floating-point-type*, 2212
 operator+=, 2218
 operator-=, 2218
 operator=, 2212
 required_alignment, 2211
 store, 2211
 store_add, 2217
 store_fmaximum, 2217
 store_fmaximum_num, 2217
 store_fminimum, 2217
 store_fminimum_num, 2217
 store_max, 2217
 store_min, 2217
 store_sub, 2217
 atomic_ref<integral-type>, 2213
 compare_exchange_strong, 2212
 compare_exchange_weak, 2212
 constructor, 2211
 exchange, 2212
 fetch_add, 2214
 fetch_and, 2214
 fetch_max, 2214
 fetch_min, 2214
 fetch_or, 2214
 fetch_sub, 2214
 fetch_xor, 2214
 is_always_lock_free, 2211
 is_lock_free, 2211
 load, 2212
 operator *integral-type*, 2212
 operator++, 2220
 operator+=, 2215
 operator--, 2220
 operator-=, 2215
 operator=, 2212

operator&=, 2215
 operator^=, 2215
 operator|=, 2215
 required_alignment, 2211
 store, 2211
 store_add, 2215
 store_and, 2215
 store_max, 2215
 store_min, 2215
 store_or, 2215
 store_sub, 2215
 atomic_ref<pointer-type>, 2218
 compare_exchange_strong, 2212
 compare_exchange_weak, 2212
 constructor, 2211
 exchange, 2212
 fetch_add, 2219
 fetch_max, 2219
 fetch_min, 2219
 fetch_sub, 2219
 is_always_lock_free, 2211
 is_lock_free, 2211
 load, 2212
 operator pointer-type, 2212
 operator++, 2220
 operator+=, 2220
 operator--, 2220
 operator-=, 2220
 operator=, 2212
 required_alignment, 2211
 store, 2211
 store_add, 2219
 store_max, 2219
 store_min, 2219
 store_sub, 2219
 atomic_ref<T>
 address, 2213
 notify_all, 2213
 notify_one, 2213
 wait, 2213
 atomic_schar, 2207, 2242
 atomic_short, 2207, 2242
 ATOMIC_SHORT_LOCK_FREE, 2202, 2209, 2242
 atomic_signal_fence, 2242, 2244
 atomic_signed_lock_free, 2207
 atomic_size_t, 2207, 2243
 atomic_store, 2222, 2243
 atomic_store_add, 2227, 2233
 atomic_store_add_explicit, 2227, 2233
 atomic_store_and, 2227
 atomic_store_and_explicit, 2227
 atomic_store_explicit, 2222, 2243
 atomic_store_max, 2227, 2233
 atomic_store_max_explicit, 2227, 2233
 atomic_store_min, 2227, 2233
 atomic_store_min_explicit, 2227, 2233
 atomic_store_or, 2227
 atomic_store_or_explicit, 2227
 atomic_store_sub, 2227, 2233
 atomic_store_sub_explicit, 2227, 2233
 atomic_thread_fence, 2242, 2244
 atomic_uchar, 2207, 2242
 atomic_uint, 2207, 2242
 atomic_uint16_t, 2207, 2243
 atomic_uint32_t, 2207, 2243
 atomic_uint64_t, 2207, 2243
 atomic_uint8_t, 2207, 2243
 atomic_uint_fast16_t, 2207, 2243
 atomic_uint_fast32_t, 2207, 2243
 atomic_uint_fast64_t, 2207, 2243
 atomic_uint_fast8_t, 2207, 2243
 atomic_uint_least16_t, 2207, 2243
 atomic_uint_least32_t, 2207, 2243
 atomic_uint_least64_t, 2207, 2243
 atomic_uint_least8_t, 2207, 2243
 atomic_uintmax_t, 2207, 2243
 atomic_uintptr_t, 2207, 2243
 atomic_ullong, 2207, 2243
 atomic_ulong, 2207, 2242
 atomic_unsigned_lock_free, 2207
 atomic_ushort, 2207, 2242
 ATOMIC_VAR_INIT, 2463, 2464
 atomic_wchar_t, 2207, 2243
 ATOMIC_WCHAR_T_LOCK_FREE, 2202, 2209, 2242
 atto, 818
 auto_ptr
 zombie, 573
 auto_ptr_ref
 zombie, 573
 await_ready
 suspend_always, 632
 suspend_never, 632
 await_resume
 suspend_always, 632
 suspend_never, 632
 await_suspend
 suspend_always, 632
 suspend_never, 632
 await_transform
 task::promise_type, 2374
 with_awaitable_senders, 2366
 awk
 syntax_option_type, 1719, 1720
B
 b
 cauchy_distribution, 1798
 extreme_value_distribution, 1794
 uniform_int_distribution, 1785
 uniform_real_distribution, 1786
 weibull_distribution, 1793
 back
 basic_string, 1617
 basic_string_view, 1599
 span, 1143
 view_interface, 1278
 back_insert_iterator, 1229

- constructor, 1230
 - operator*, 1230
 - operator++, 1230
 - operator=, 1230
- back_inserter, 1230
- bad
 - basic_ios, 2051
- bad_address
 - errc, 655
- bad_alloc, 159, 602, 605, 606
 - constructor, 605
 - what, 605
- bad_any_cast, 884
 - what, 884
- bad_array_new_length, 605
 - constructor, 605
 - what, 605
- bad_cast, 146, 607, 608
 - constructor, 608
 - what, 608
- bad_exception, 613
 - constructor, 613
 - what, 613
- bad_expected_access, 891
 - constructor, 891
 - error, 891
 - what, 891
- bad_file_descriptor
 - errc, 655
- bad_function_call, 936
 - what, 937
- bad_message
 - errc, 655
- bad_optional_access
 - what, 868
- bad_typeid, 147, 607, 608
 - constructor, 608
 - what, 608
- bad_variant_access, 883
 - what, 883
- bad_weak_ptr, 706
 - what, 706
- barrier
 - arrive, 2274
 - arrive_and_drop, 2274
 - arrive_and_wait, 2274
 - constructor, 2273
 - max, 2273
 - wait, 2274
- base
 - adjacent_transform_view, 1381
 - as_input_view::iterator, 1419
 - basic_const_iterator, 1235
 - cache_latest_view::iterator, 1416
 - cache_latest_view::sentinel, 1417
 - chunk_view::inner-iterator, 1389
 - chunk_view::iterator, 1393
 - common_view, 1350
 - counted_iterator, 1248
 - drop_view, 1319
 - drop_while_view, 1320
 - elements_view, 1355
 - elements_view::iterator, 1357
 - elements_view::sentinel, 1359
 - enumerate_view::iterator, 1362
 - enumerate_view::sentinel, 1364
 - filter_view, 1304
 - filter_view::iterator, 1306
 - filter_view::sentinel, 1308
 - join_view, 1321
 - lazy_split_view, 1333
 - lazy_split_view::inner-iterator, 1337
 - move_iterator, 1239
 - move_sentinel, 1241
 - reverse_iterator, 1226
 - reverse_view, 1352
 - split_view::iterator, 1340
 - stride_view::iterator, 1406
 - take_view, 1314
 - take_view::sentinel, 1316
 - take_while_view, 1317
 - transform_view, 1308
 - transform_view::iterator, 1311
 - transform_view::sentinel, 1313
- bases_of, 805
- basic
 - syntax_option_type, 1719, 1720
- basic_common_reference, 774, 924
 - tuple, 851
- basic_const_iterator, 1232
 - base, 1235
 - constructor, 1235
 - operator*, 1235
 - operator+, 1236
 - operator++, 1235
 - operator+=, 1235
 - operator-, 1237
 - operator--, 1235
 - operator=, 1235
 - operator->, 1235
 - operator<, 1236
 - operator<=, 1236
 - operator<=>, 1236
 - operator==, 1235
 - operator>, 1236
 - operator>=, 1236
 - operator[], 1235
- basic_filebuf, 2034, 2113
 - close, 2116
 - constructor, 2114
 - destructor, 2114
 - imbue, 2118
 - is_open, 2115
 - native_handle, 2116
 - open, 2115, 2116
 - operator=, 2114
 - overflow, 2117
 - pbackfail, 2116

- seekoff, 2117
- seekpos, 2118
- setbuf, 2117
- showmanyc, 2116
- swap, 2115
- sync, 2118
- uflow, 2116
- underflow, 2116
- basic_filebuf<char>, 2112
- basic_filebuf<wchar_t>, 2112
- basic_format_arg, 1708
 - constructor, 1709
 - handle, 1710
 - operator bool, 1709
 - visit, 1709
- basic_format_arg::handle, 1710
 - constructor, 1710
 - format, 1710
- basic_format_args
 - constructor, 1711
 - get, 1711
- basic_format_context, 1702
 - advance_to, 1703
 - arg, 1702
 - char_type, 1702
 - formatter_type, 1702
 - iterator, 1702
 - locale, 1702
 - out, 1703
- basic_format_parse_context, 1700
 - advance_to, 1701
 - begin, 1701
 - char_type, 1700
 - check_arg_id, 1701
 - check_dynamic_spec, 1701
 - check_dynamic_spec_integral, 1702
 - check_dynamic_spec_string, 1702
 - const_iterator, 1700
 - constructor, 1701
 - end, 1701
 - formatter, 2150
 - iterator, 1700
 - next_arg_id, 1701
- basic_format_string, 1693
- basic_fstream, 2034, 2122
 - close, 2124
 - constructor, 2123, 2124
 - is_open, 2124
 - native_handle, 2124
 - open, 2124
 - rdbuf, 2124
 - swap, 2124
- basic_fstream<char>, 2112
- basic_fstream<wchar_t>, 2112
- basic_ifstream, 2034, 2118
 - close, 2120
 - constructor, 2119, 2120
 - is_open, 2120
 - native_handle, 2120
- open, 2120
- rdbuf, 2120
- swap, 2120
- basic_ifstream<char>, 2112
- basic_ifstream<wchar_t>, 2112
- basic_ios, 2034, 2047
 - bad, 2051
 - clear, 2050
 - constructor, 2048
 - copyfmt, 2049
 - destructor, 2048
 - eof, 2050
 - exceptions, 2051
 - fail, 2050
 - fill, 2049
 - good, 2050
 - imbue, 2049
 - init, 2048, 2065
 - move, 2049
 - narrow, 2049
 - operator bool, 2050
 - operator!, 2050
 - rdbuf, 2049
 - rdstate, 2050
 - set_rdbuf, 2050
 - setstate, 2050
 - swap, 2050
 - tie, 2048
 - widen, 2049
- basic_ios<char>, 2039
- basic_ios<wchar_t>, 2039
- basic_iostream, 2073
 - constructor, 2073, 2074
 - destructor, 2074
 - operator=, 2074
 - swap, 2074
- basic_ispanstream, 2108
 - constructor, 2108, 2109
 - rdbuf, 2109
 - span, 2109
 - swap, 2109
- basic_istream, 2034, 2063
 - constructor, 2065
 - destructor, 2065
 - gcount, 2069
 - get, 2069, 2070
 - getline, 2070, 2071
 - ignore, 2071
 - operator=, 2065
 - operator>>, 2066–2068, 2073
 - peek, 2071
 - putback, 2072
 - read, 2071
 - readsome, 2071
 - seekg, 2072
 - sentry, 2065
 - swap, 2065
 - sync, 2072
 - tellg, 2072

- unget, 2072
- basic_istream::sentry, 2065
 - constructor, 2065
 - destructor, 2066
 - operator bool, 2066
- basic_istream<char>, 2061
- basic_istream<wchar_t>, 2061
- basic_istream_view, 1296
 - constructor, 1297
 - end, 1297
- basic_istream_view::iterator, 1297
 - constructor, 1297
 - operator*, 1298
 - operator++, 1297
 - operator==, 1298
- basic_istringstream, 2034, 2095
 - constructor, 2097, 2098
 - rdbuf, 2098
 - str, 2098
 - swap, 2098
 - view, 2098
- basic_istringstream<char>, 2088
- basic_istringstream<wchar_t>, 2088
- basic_mask
 - abi_type, 1935
 - begin, 1935
 - cbegin, 1935
 - cend, 1935
 - const_iterator, 1908, 1935
 - constructor, 1937
 - end, 1935
 - iterator, 1908, 1935
 - operator basic_vec, 1938
 - operator!, 1937
 - operator!=, 1938
 - operator+, 1937
 - operator-, 1937
 - operator<, 1938
 - operator<=, 1938
 - operator==, 1938
 - operator>, 1938
 - operator>=, 1938
 - operator[], 1937
 - operator&, 1938
 - operator&=, 1938
 - operator&&, 1938
 - operator~, 1938
 - operator^=, 1938
 - operator~, 1937
 - operator|, 1938
 - operator|=, 1938
 - operator||, 1938
 - size, 1935
 - to_bitset, 1938
 - to_ullong, 1938
 - value_type, 1935
- basic_ofstream, 2034, 2120
 - close, 2122
 - constructor, 2121
 - is_open, 2122
 - native_handle, 2122
 - open, 2122
 - rdbuf, 2122
 - swap, 2122
- basic_ofstream<char>, 2112
- basic_ofstream<wchar_t>, 2112
- basic_ostream, 1731, 2034, 2074
 - constructor, 2076
 - destructor, 2076
 - flush, 2082
 - init, 2076
 - operator<<, 2078–2080, 2082
 - operator=, 2076
 - put, 2081
 - seekp, 2077
 - sentry, 2077
 - swap, 2076
 - tellp, 2077
 - write, 2081
- basic_ostream::sentry, 2077
 - constructor, 2077
 - destructor, 2077
 - operator bool, 2077
- basic_ostream<char>, 2062
- basic_ostream<wchar_t>, 2062
- basic_ostringstream, 2034, 2098
 - constructor, 2100, 2101
 - rdbuf, 2101
 - str, 2101
 - swap, 2101
 - view, 2101
- basic_ostringstream<char>, 2088
- basic_ostringstream<wchar_t>, 2088
- basic_osyncstream, 2034, 2127
 - constructor, 2128
 - set_emit_on_sync, 2128, 2129
- basic_regex, 1715, 1724, 1725, 1745
 - assign, 1727, 1728
 - constructor, 1726, 1727
 - flag_type, 1728
 - getloc, 1728
 - imbue, 1728
 - mark_count, 1728
 - operator=, 1727
 - swap, 1728, 1729
- basic_spanbuf, 2105
 - constructor, 2106
 - operator=, 2106
 - seekoff, 2107
 - seekpos, 2108
 - setbuf, 2108
 - span, 2107
 - swap, 2107

- basic_spanstream, 2110
 - constructor, 2111
 - rdbuf, 2111
 - span, 2111
 - swap, 2111
- basic_stacktrace, 666
 - at, 669
 - begin, 668
 - cbegin, 668
 - cend, 668
 - const_iterator, 668
 - constructor, 668
 - crbegin, 668
 - crend, 668
 - current, 667, 668
 - empty, 669
 - end, 668
 - get_allocator, 668
 - max_size, 669
 - operator<<, 670
 - operator<=>, 669
 - operator=, 668
 - operator==, 669
 - operator[], 669
 - rbegin, 668
 - rend, 668
 - size, 669
 - swap, 669
 - to_string, 669
- basic_streambuf, 2034, 2054
 - constructor, 2055, 2056
 - destructor, 2056
 - eback, 2058
 - egptr, 2058
 - epptr, 2058
 - gbump, 2058
 - getloc, 2056
 - gptr, 2058
 - imbue, 2058
 - in_avail, 2057
 - operator=, 2057
 - overflow, 2060
 - pbackfail, 2060
 - pbase, 2058
 - pbump, 2058
 - pptr, 2058
 - pubimbue, 2056
 - pubseekoff, 2056
 - pubseekpos, 2056
 - pubsetbuf, 2056
 - pubsync, 2056
 - sbumpc, 2057
 - seekoff, 2059
 - seekpos, 2059
 - setbuf, 2058, 2095
 - setg, 2058
 - setp, 2058
 - sgetc, 2057
 - sgetn, 2057
 - showmanyc, 2059, 2116
 - snextc, 2057
 - sputbackc, 2057
 - sputc, 2057
 - sputn, 2057
 - sungetc, 2057
 - swap, 2058
 - sync, 2059
 - uflow, 2060
 - underflow, 2059
 - xsgetn, 2059
 - xspn, 2060
- basic_streambuf<char>, 2053
- basic_streambuf<wchar_t>, 2053
- basic_string, 1607, 1626, 2089
 - allocator_type, 1607
 - append, 1618, 1619
 - append_range, 1619
 - assign, 1619, 1620
 - assign_range, 1620
 - at, 1617
 - back, 1617
 - begin, 1615
 - c_str, 1624
 - capacity, 1616
 - cbegin, 1615
 - cend, 1615
 - clear, 1617
 - compare, 1625, 1626
 - const_iterator, 1607
 - const_pointer, 1607
 - const_reference, 1607
 - const_reverse_iterator, 1607
 - constructor, 1613, 1614
 - contains, 1626
 - copy, 1624
 - crbegin, 1615
 - crend, 1615
 - data, 1624
 - difference_type, 1607
 - empty, 1617
 - end, 1615
 - ends_with, 1626
 - erase, 1621, 1622, 1630
 - erase_if, 1630
 - find, 1624
 - find_first_not_of, 1624
 - find_first_of, 1624
 - find_last_not_of, 1624
 - find_last_of, 1624
 - front, 1617
 - get_allocator, 1624
 - getline, 1629, 1630
 - hash, 1632
 - insert, 1620, 1621
 - insert_range, 1621
 - iterator, 1607
 - length, 1615
 - max_size, 1616

operator basic_string_view, 1624
 operator+, 1626–1628
 operator+=, 1617, 1618
 operator<<, 1629
 operator<=>, 965
 operator=, 1615
 operator>>, 1629
 operator[], 1617
 pointer, 1607
 pop_back, 1622
 push_back, 1619
 rbegin, 1615
 reference, 1607
 rend, 1615
 replace, 1622, 1623
 replace_with_range, 1623
 reserve, 1616
 resize, 1616
 resize_and_overwrite, 1616
 reverse_iterator, 1607
 rfind, 1624
 shrink_to_fit, 1617
 size, 1615
 size_type, 1607
 starts_with, 1626
 substr, 1625
 subview, 1625
 swap, 1624, 1629
 traits_type, 1607
 value_type, 1607
 basic_string_view, 1595
 at, 1599
 back, 1599
 begin, 1599
 cbegin, 1599
 cend, 1599
 compare, 1600, 1601
 const_iterator, 1595, 1598
 const_pointer, 1595
 const_reference, 1595
 const_reverse_iterator, 1595
 constructor, 1597, 1598
 contains, 1601
 copy, 1600
 crbegin, 1599
 crend, 1599
 data, 1599
 difference_type, 1595
 empty, 1599
 end, 1599
 ends_with, 1601
 find, 1602
 find_first_not_of, 1602
 find_first_of, 1602
 find_last_not_of, 1602
 find_last_of, 1602
 front, 1599
 iterator, 1595
 length, 1599
 max_size, 1599
 operator<<, 1603
 operator<=>, 1603
 operator==, 1603
 operator[], 1599
 pointer, 1595
 rbegin, 1599
 reference, 1595
 remove_prefix, 1600
 remove_suffix, 1600
 rend, 1599
 reverse_iterator, 1595
 rfind, 1602
 size, 1599
 size_type, 1595
 starts_with, 1601
 substr, 1600
 subview, 1600
 swap, 1600
 traits_type, 1595
 value_type, 1595
 basic_stringbuf, 2034, 2089
 constructor, 2091, 2092
 get_allocator, 2093
 operator=, 2092
 overflow, 2094
 pbackfail, 2094
 seekoff, 2095
 seekpos, 2095
 str, 2093, 2094
 swap, 2092
 underflow, 2094
 view, 2093
 basic_stringbuf<char>, 2088
 basic_stringbuf<wchar_t>, 2088
 basic_stringstream, 2034, 2101
 constructor, 2103
 rdbuf, 2104
 str, 2104
 swap, 2104
 view, 2104
 basic_stringstream<char>, 2088
 basic_stringstream<wchar_t>, 2088
 basic_syncbuf, 2034, 2125
 constructor, 2126
 destructor, 2126
 emit, 2126
 get_allocator, 2127
 get_wrapped, 2127
 operator=, 2126
 set_emit_on_sync, 2127
 swap, 2126, 2127
 sync, 2127
 basic_vec, 1910
 abi_type, 1910
 begin, 1910
 cbegin, 1910
 cend, 1910
 const_iterator, 1908, 1910

constructor, 1912, 1913
 end, 1910
 imag, 1913
 iterator, 1908, 1910
 mask_type, 1910
 operator!, 1914
 operator!=, 1915
 operator*, 1914
 operator*=, 1915
 operator+, 1914
 operator++, 1914
 operator+=, 1915
 operator-, 1914
 operator--, 1914
 operator-=, 1915
 operator/, 1914
 operator/=, 1915
 operator<, 1915
 operator<<, 1914, 1915
 operator<=, 1915
 operator<=, 1915
 operator==, 1915
 operator>, 1915
 operator>=, 1915
 operator>>, 1914, 1915
 operator>>=, 1915
 operator[], 1913
 operator%, 1914
 operator%=, 1915
 operator&, 1914
 operator&=, 1915
 operator~, 1914
 operator~=, 1915
 operator~, 1914
 operator|, 1914
 operator|=, 1915
 real, 1913
 size, 1910
 value_type, 1910
 before
 type_info, 608
 before_begin
 forward_list, 1007
 begin, 618, 1268
 adjacent_transform_view, 1381
 adjacent_view, 1375
 array, 997
 as_input_view, 1418
 basic_format_parse_context, 1701
 basic_mask, 1935
 basic_stacktrace, 668
 basic_string, 1615
 basic_string_view, 1599
 basic_vec, 1910
 cache_latest_view, 1415
 cartesian_product_view, 1409
 chunk_by_view, 1401
 chunk_view, 1386, 1390
 chunk_view::outer_iterator::value_-
 type, 1388
 common_view, 1350
 concat_view, 1343
 containers, 962
 directory_iterator, 2160
 drop_view, 1320
 drop_while_view, 1321
 elements_view, 1355
 enumerate_view, 1360
 filter_view, 1305
 generator, 1422
 initializer_list, 619
 iota_view, 1289
 join_view, 1321
 lazy_split_view, 1333
 lazy_split_-
 view::outer_iterator::value_type,
 1336
 match_results, 1734
 optional, 860
 path, 2148
 recursive_directory_iterator, 2163
 repeat_view, 1294
 reverse_view, 1353
 single_view, 1286
 slide_view, 1395, 1396
 span, 1144
 split_view, 1339
 subrange, 1281
 sys_info, 2014
 take_view, 1314
 take_while_view, 1317
 text_encoding::aliases_view, 1680
 transform_view, 1309
 tzdb_list, 2011
 unordered associative containers, 993
 valarray, 1814
 zip_transform_view, 1370
 zip_view, 1365
 begin(C&), 1256
 begin(T (&)[N]), 1256
 bernoulli_distribution, 1786
 constructor, 1787
 p, 1787
 result_type, 1786
 beta, 1832
 gamma_distribution, 1792
 simd, 1928
 betaf, 1832
 betal, 1832
 bfloat16_t, 599
 bidirectional_iterator, 1212
 bidirectional_iterator_tag, 1221
 bidirectional_range, 1275
 big
 endian, 957
 Big5
 text_encoding, 1683

Big5HKSCS
 text_encoding, 1684
 binary_function
 zombie, 573
 binary_negate
 zombie, 573
 binary_search, 1541
 binary_semaphore, 2269
 binary_transform_result, 1452
 bind, 935–936
 bind1st
 zombie, 573
 bind2nd
 zombie, 573
 bind_back, 933, 934
 bind_front, 933, 934
 binder1st
 zombie, 573
 binder2nd
 zombie, 573
 binomial_distribution, 1787
 constructor, 1788
 p, 1788
 result_type, 1787
 t, 1788
 bit_and, 931
 operator(), 931
 bit_and<>, 931
 operator(), 932
 bit_cast, 954
 bit_ceil, 954
 simd, 1932
 bit_compress, 956
 simd, 1934
 bit_expand, 957
 simd, 1934
 bit_floor, 955
 simd, 1932
 bit_not
 operator(), 932
 bit_not<>, 932
 operator(), 932
 bit_or, 932
 operator(), 932
 bit_or<>, 932
 operator(), 932
 bit_repeat, 956
 simd, 1933
 bit_reverse, 956
 simd, 1932
 bit_size_of, 806
 bit_width, 955
 simd, 1934
 bit_xor, 932
 operator(), 932
 bit_xor<>, 932
 operator(), 932
 bitset, 910
 all, 915
 any, 916
 constructor, 912, 913
 count, 915
 flip, 914
 none, 916
 operator<<, 914, 916
 operator<=, 914
 operator==, 915
 operator>>, 914, 916
 operator>=, 914
 operator[], 915
 operator&, 916
 operator&=, 913
 operator~, 916
 operator^, 914
 operator^=, 914
 operator~, 914
 operator|, 916
 operator|=, 913
 reset, 914
 set, 914
 size, 915
 test, 915
 to_string, 915
 to_ullong, 915
 to_ulong, 915
 bitset::reference
 constructor, 912
 destructor, 912
 flip, 912
 operator bool, 912
 operator=, 912
 operator~, 912
 swap, 912
 blank
 ctype_base, 1645
 block
 file_type, 2152
 block_capacity_default_limits
 hive, 1018
 block_capacity_hard_limits
 hive, 1018
 block_capacity_limits
 hive, 1017
 BOCU1
 text_encoding, 1682
 bool_constant, 751, 758
 BOOL_WIDTH, 595
 boolalpha, 2051
 borrowed_iterator_t, 1282
 borrowed_range, 1273
 borrowed_subrange_t, 1282
 boyer_moore_horspool_searcher, 951
 constructor, 951
 operator(), 952
 boyer_moore_searcher, 950
 constructor, 950
 operator(), 951
 breakpoint, 670
 breakpoint_if_debugging, 670

BRF
 text_encoding, 1684
broken_pipe
 errc, 655
broken_promise
 future_errc, 2274
bsearch, 580, 1587
btowc, 1747
bucket
 unordered associative containers, 993
bucket_count
 unordered associative containers, 993
bucket_size
 unordered associative containers, 993
BUFSIZ, 2176
bulk, 2302
bulk_chunked, 2302
bulk_chunked_t, 2302
bulk_item_receiver_proxy, 2383
bulk_t, 2302
bulk_unchunked, 2302
bulk_unchunked_t, 2302
byte, 579
 operator<<, 582
 operator<=, 582
 operator>>, 582
 operator>=, 582
 operator&, 583
 operator&=, 583
 operator^, 583
 operator^=, 583
 operator~, 583
 operator|, 583
 operator|=, 582
 to_integer, 583
byte_alignment
 aligned_accessor, 1174
byteswap, 954
 simd, 1932

C
c16rtomb, 1749
c32rtomb, 1749
c8rtomb, 1749
c_encoding
 weekday, 1986
c_str
 basic_string, 1624
 path, 2144
cache_latest_view, 1414
 begin, 1415
 constructor, 1415
 end, 1415
 reserve_hint, 1415
 size, 1415
cache_latest_view::iterator, 1415
cache_latest_view::iterator
 base, 1416
 constructor, 1416
 iter_move, 1416
 iter_swap, 1416
 operator*, 1416
 operator++, 1416
 operator-, 1417
 operator==, 1417
cache_latest_view::sentinel, 1416
 base, 1417
 constructor, 1417
 operator-, 1417
 operator==, 1417
cacos
 complex, 1757
cacosh
 complex, 1757
call_once, 2261
callback_type
 inplace_stop_callback, 2191
 inplace_stop_token, 2190
 never_stop_token, 2190
 stop_callback, 2189
 stop_token, 2187
calloc, 580, 693, 2454
can_substitute, 809
canonical, 2164
canonical_slices, 1184
capacity
 basic_string, 1616
 hive, 1017
 inplace_vector, 1039
 vector, 1031
cartesian_product_view, 1407, 1408
 distance-from, 1412
 next, 1411
 prev, 1412
 begin, 1409
 constructor, 1409
 end, 1409, 1410
 size, 1410
cartesian_product_view::iterator, 1410
 constructor, 1412
 iter_move, 1414
 iter_swap, 1414
 operator*, 1412
 operator+, 1413
 operator++, 1412
 operator+=, 1413
 operator-, 1413, 1414
 operator--, 1412
 operator==, 1413
 operator<=, 1413
 operator==, 1413
 operator[], 1413
casin
 complex, 1757
casinh
 complex, 1758
cat

- simd, 1924
- catalog
 - messages_base, 1675
 - messages_byname, 1676
- catan
 - complex, 1757
- catanh
 - complex, 1758
- category
 - error_code, 660
 - error_condition, 662
 - locale, 1640
- cauchy_distribution, 1797
 - a, 1798
 - b, 1798
 - constructor, 1798
 - result_type, 1797
- cbefore_begin
 - forward_list, 1007
- cbegin, 1269
 - basic_mask, 1935
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - basic_vec, 1910
 - containers, 962
 - tzdb_list, 2011
 - unordered associative containers, 994
- cbegin(const C&), 1257
- cbrt, 1826
 - simd, 1928
- cbrtf, 1826
- cbrtl, 1826
- cdata, 1272
- ceil, 1826
 - duration, 1962
 - simd, 1925
 - time_point, 1967
- ceilf, 1826
- ceil, 1826
- cend, 1269
 - basic_mask, 1935
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - basic_vec, 1910
 - containers, 962
 - tzdb_list, 2011
 - unordered associative containers, 994
- cend(const C&), 1257
- centi, 818
- cerr, 2037
- CESU8
 - text_encoding, 1682
- CHAR_BIT, 64, 530, 595
- char_class_type
 - regex_traits, 1723
- CHAR_MAX, 595
- CHAR_MIN, 595
- char_traits, 1591–1593
 - char_type, 1591
 - int_type, 1591
 - state_type, 1591
- char_type
 - basic_format_context, 1702
 - basic_format_parse_context, 1700
 - char_traits, 1591
 - collate, 1663
 - ctype, 1645
 - ctype<char>, 1648
 - messages, 1675
 - money_get, 1670
 - money_put, 1671
 - moneypunct, 1672
 - num_get, 1654
 - num_put, 1658
 - numpunct, 1661
 - numpunct_byname, 1663
 - time_get, 1664
 - time_put, 1668
 - time_put_byname, 1669
- CHAR_WIDTH, 595
- character
 - file_type, 2152
- chars_format, 1634
 - fixed, 1634
 - general, 1634
 - hex, 1634
 - scientific, 1634
- check_arg_id
 - basic_format_parse_context, 1701
- check_dynamic_spec
 - basic_format_parse_context, 1701
- check_dynamic_spec_integral
 - basic_format_parse_context, 1702
- check_dynamic_spec_string
 - basic_format_parse_context, 1702
- chi_squared_distribution, 1796
 - constructor, 1797
 - n, 1797
 - result_type, 1796
- choose, 1941
 - earliest, 1941
 - latest, 1941
- chrono, 1941
- chunk
 - simd, 1923, 1924
 - views, 1385
- chunk_by
 - views, 1400
- chunk_by_view, 1400
 - find-next, 1401
 - find-prev, 1401
 - begin, 1401
 - constructor, 1401
 - end, 1401
 - pred, 1401
- chunk_by_view::iterator, 1401

- constructor, 1402
 - operator*, 1402
 - operator++, 1402
 - operator--, 1402
 - operator==, 1402
- chunk_view, 1386, 1390
 - begin, 1386, 1390
 - constructor, 1386, 1391
 - end, 1387, 1390
 - reserve_hint, 1387, 1391
 - size, 1387, 1391
- chunk_view::inner-iterator, 1389
 - base, 1389
 - constructor, 1389
 - iter_move, 1390
 - iter_swap, 1390
 - operator*, 1389
 - operator++, 1389
 - operator-, 1390
 - operator==, 1390
- chunk_view::iterator, 1391
 - base, 1393
 - constructor, 1392
 - operator*, 1393
 - operator+, 1394
 - operator++, 1393
 - operator+=, 1393
 - operator-, 1394
 - operator--, 1393
 - operator==, 1393
 - operator<, 1393
 - operator<=, 1394
 - operator<=>, 1394
 - operator==, 1393
 - operator>, 1393
 - operator>=, 1394
 - operator[], 1393
- chunk_view::outer-iterator, 1387
 - constructor, 1387
 - operator*, 1387
 - operator++, 1387, 1388
 - operator-, 1388
 - operator==, 1388
- chunk_view::outer-iterator::value_type, 1388
 - begin, 1388
 - constructor, 1388
 - end, 1388
 - reserve_hint, 1388
 - size, 1388
- cin, 2037
- ckd_add, 1940
- ckd_mul, 1940
- ckd_sub, 1940
- clamp, 1562
 - simd, 1925
- classic
 - locale, 1644
- classic_table
 - ctype<char>, 1649
- clear
 - atomic_flag, 2240
 - basic_ios, 2050
 - basic_string, 1617
 - error_code, 660
 - error_condition, 662
 - forward_list, 1009
 - ordered associative containers, 982
 - path, 2143
 - sequence containers, 970
 - unordered associative containers, 992
- clear_hazard_pointer_batch, 2294
- clearerr, 2176
- clock, 2032
- clock_cast, 1978
- clock_t, 2032
- clock_time_conversion, 1975
 - operator(), 1976–1978
- CLOCKS_PER_SEC, 2032
- clog, 2037
- close
 - basic_filebuf, 2116
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ofstream, 2122
 - execution::simple_counting_scope, 2379
 - messages, 1675
- cmp_equal, 827
- cmp_greater, 827
- cmp_greater_equal, 827
- cmp_less, 827
- cmp_less_equal, 827
- cmp_not_equal, 827
- cntrl
 - ctype_base, 1645
- code
 - future_error, 2276
 - system_error, 664
- codecvt, 1650
 - always_noconv, 1651
 - do_always_noconv, 1653
 - do_encoding, 1653
 - do_in, 1651
 - do_length, 1653
 - do_max_length, 1653
 - do_out, 1651
 - do_unshift, 1652
 - encoding, 1651
 - extern_type, 1650
 - id, 1650
 - in, 1651
 - intern_type, 1650
 - length, 1651
 - max_length, 1651
 - out, 1651
 - state_type, 1650
 - unshift, 1651
- codecvt_base, 1650

- error, 1650
- noconv, 1650
- ok, 1650
- partial, 1650
- result, 1650
- codecvt_byname, 1653
- codecvt_mode
 - zombie, 573
- codecvt_utf16
 - zombie, 573
- codecvt_utf8
 - zombie, 573
- codecvt_utf8_utf16
 - zombie, 573
- collate, 1663
 - char_type, 1663
 - compare, 1663
 - do_compare, 1664
 - do_hash, 1664
 - do_transform, 1664
 - hash, 1663
 - id, 1663
 - string_type, 1663
 - syntax_option_type, 1719, 1720, 1746
 - transform, 1663
- collate_byname, 1664
 - string_type, 1664
- column_major, 1852
- column_major_t, 1852
- combine
 - locale, 1643
- common
 - views, 1350
- common_comparison_category, 624
- common_comparison_category_t, 619
- common_iterator, 1242
 - constructor, 1243
 - iter_move, 1245
 - iter_swap, 1245
 - operator*, 1244
 - operator++, 1244
 - operator-, 1245
 - operator->, 1244
 - operator=, 1243
 - operator==, 1245
- common_range, 1276
- common_reference, 774, 816
- common_reference_t, 755
- common_reference_with, 641
- common_type, 775, 816, 1957
 - duration, 1957
 - tuple, 851
- common_type_t, 755
- common_view, 1350
 - base, 1350
 - begin, 1350
 - constructor, 1351
 - end, 1350
 - size, 1350
- common_with, 641
- comp
 - map::value_compare, 1043
 - multimap::value_compare, 1050
- comp_ellint_1, 1832
 - simd, 1928
- comp_ellint_1f, 1832
- comp_ellint_1l, 1832
- comp_ellint_2, 1832
 - simd, 1928
- comp_ellint_2f, 1832
- comp_ellint_2l, 1832
- comp_ellint_3, 1833
 - simd, 1928
- comp_ellint_3f, 1833
- comp_ellint_3l, 1833
- compare
 - basic_string, 1625, 1626
 - basic_string_view, 1600, 1601
 - collate, 1663
 - path, 2145
 - sub_match, 1729
- compare_exchange_strong
 - atomic, 2222
 - atomic<floating-point-type>, 2222
 - atomic<integral-type>, 2222
 - atomic<shared_ptr<T>>, 2236
 - atomic<T*>, 2222
 - atomic<weak_ptr<T>>, 2239
 - atomic_ref, 2212
 - atomic_ref<floating-point-type>, 2212
 - atomic_ref<integral-type>, 2212
 - atomic_ref<pointer-type>, 2212
- compare_exchange_weak
 - atomic, 2222
 - atomic<floating-point-type>, 2222
 - atomic<integral-type>, 2222
 - atomic<shared_ptr<T>>, 2236
 - atomic<T*>, 2222
 - atomic<weak_ptr<T>>, 2238, 2239
 - atomic_ref, 2212
 - atomic_ref<floating-point-type>, 2212
 - atomic_ref<integral-type>, 2212
 - atomic_ref<pointer-type>, 2212
- compare_partial_order_fallback, 627
- compare_strong_order_fallback, 626
- compare_three_way, 928
- compare_weak_order_fallback, 627
- completion_signatures, 2303
- complex, 1754
 - abs, 1757
 - acos, 1757
 - acosh, 1757
 - arg, 1757
 - asin, 1757
 - asinh, 1758
 - atan, 1757
 - atanh, 1758
 - cacos, 1757

cacosh, 1757
 casin, 1757
 casinh, 1758
 catan, 1757
 catanh, 1758
 conj, 1757
 constructor, 1755
 cos, 1758
 cosh, 1758
 exp, 1758
 get, 1759
 imag, 1755, 1757
 log, 1758
 log10, 1758
 norm, 1757
 operator"i, 1759
 operator"if, 1759
 operator"il, 1759
 operator*, 1756
 operator*=, 1755, 1756
 operator+, 1756
 operator+=, 1755
 operator-, 1756
 operator-=, 1755
 operator/, 1756
 operator/=: 1755, 1756
 operator<, 1756
 operator=, 1755
 operator==, 1756
 operator>>, 1756
 polar, 1757
 pow, 1758
 proj, 1757
 real, 1755, 1757
 sin, 1758
 sinh, 1758
 sqrt, 1758
 tan, 1758
 tanh, 1758
 value_type, 1754
 compress
 simd, 1921
 concat
 path, 2143
 concat_view, 1341
 begin, 1343
 constructor, 1343
 end, 1343
 iterator, 1344
 reserve_hint, 1343
 size, 1343
 concat_view::iterator, 1344
 advance-bwd, 1346
 advance-fwd, 1346
 prev, 1346
 satisfy, 1345
 constructor, 1346
 iter_move, 1349
 iter_swap, 1349
 operator*, 1347
 operator+, 1348
 operator++, 1347
 operator+=, 1347
 operator-, 1348, 1349
 operator--, 1347
 operator-=, 1347
 operator<, 1348
 operator==, 1347, 1348
 operator[], 1347
 condition_variable, 2263
 constructor, 2263
 destructor, 2264
 notify_all, 2264
 notify_one, 2264
 wait, 2264
 wait_for, 2265, 2266
 wait_until, 2264, 2265
 condition_variable_any, 2266
 constructor, 2267
 destructor, 2267
 notify_all, 2267
 notify_one, 2267
 wait, 2267
 wait_for, 2268
 wait_until, 2267, 2268
 conditional, 774
 conditional_t, 755
 conj, 1759
 complex, 1757
 simd, 1934
 conjugate_transposed, 1865
 conjugated, 1861
 conjugated_accessor, 1860
 access, 1861
 constructor, 1861
 data_handle_type, 1861
 element_type, 1860
 nested_accessor, 1861
 offset, 1861
 offset_policy, 1861
 reference, 1861
 conjunction, 777
 conjunction_v, 758
 connect, 2301
 task, 2371
 connect_result_t, 2301
 connect_t, 2301
 connection_aborted
 errc, 656
 connection_already_in_progress
 errc, 656
 connection_refused
 errc, 656
 connection_reset
 errc, 656
 const_iterator, 1232
 basic_format_parse_context, 1700
 basic_mask, 1908, 1935

- basic_stacktrace, 668
- basic_string, 1607
- basic_string_view, 1595, 1598
- basic_vec, 1908, 1910
- containers, 961
- optional, 860
- const_iterator_t, 1260
- const_local_iterator
 - unordered associative containers, 986
- const_mem_fun1_ref_t
 - zombie, 573
- const_mem_fun1_t
 - zombie, 573
- const_mem_fun_ref_t
 - zombie, 573
- const_mem_fun_t
 - zombie, 573
- const_pointer
 - allocator_traits, 691
 - basic_string, 1607
 - basic_string_view, 1595
 - scoped_allocator_adaptor, 746
- const_pointer_cast
 - shared_ptr, 716
- const_reference
 - basic_string, 1607
 - basic_string_view, 1595
 - containers, 961
- const_reverse_iterator
 - basic_string, 1607
 - basic_string_view, 1595
 - reversible containers, 964
- const_sentinel, 1232
- const_sentinel_t, 1260
- const_void_pointer
 - allocator_traits, 691
 - scoped_allocator_adaptor, 746
- constant_of, 793
- constant_range, 1276
- constant_wrapper, 830
 - operator(), 831
 - operator[], 831
- construct
 - allocator_traits, 692
 - polymorphic_allocator, 740
 - scoped_allocator_adaptor, 749
- construct_at, 1585
- constructible_from, 644
- consume
 - memory_order, 2464
- consume_header
 - zombie, 573
- contains, 1490
 - basic_string, 1626
 - basic_string_view, 1601
 - ordered associative containers, 983
 - unordered associative containers, 992
- contains_subrange, 1490
- contiguous_iterator, 1213
- contiguous_iterator_tag, 1221
- contiguous_range, 1276
- continuation
 - with_awaitable_senders, 2366
- continues_on, 2302
- continues_on_t, 2302
- contract_violation, 103, 104, 616, 617
- converted
 - zombie, 573
- convertible_to, 641
- copy, 1190, 1509, 1867
 - basic_string, 1624
 - basic_string_view, 1600
 - path, 2164
- copy_backward, 1511
- copy_backward_result, 1449
- copy_constructible, 645
- copy_file, 2166
- copy_if, 1510
- copy_if_result, 1449
- copy_n, 1510
- copy_n_result, 1448
- copy_options, 2151
 - copy_symlinks, 2153
 - create_hard_links, 2153
 - create_symlinks, 2153
 - directories_only, 2153
 - none, 2153
 - overwrite_existing, 2153
 - recursive, 2153
 - skip_existing, 2153
 - skip_symlinks, 2153
 - update_existing, 2153
- copy_result, 1448
- copy_symlink, 2167
- copy_symlinks
 - copy_options, 2153
- copyable, 648
- copyable_function, 943
 - constructor, 944, 945
 - destructor, 946
 - operator bool, 946
 - operator(), 946
 - operator=, 945
 - operator==, 946
 - swap, 946
- copyfmt
 - basic_ios, 2049
- copysign, 1827
 - simd, 1925
- copysignf, 1827
- copysignl, 1827
- coroutine_handle, 628
 - address, 630
 - constructor, 629
 - destroy, 630
 - done, 630
 - from_address, 630
 - from_promise, 630

- hash, 631
- operator bool, 630
- operator coroutine_handle<>, 630
- operator!=, 631
- operator(), 630
- operator<=>, 631
- operator=, 630
- operator==, 631
- promise, 631
- resume, 630
- coroutine_handle<noop_coroutine_promise>, 631
 - address, 632
 - destroy, 632
 - done, 632
 - operator bool, 632
 - operator coroutine_handle<>, 631
 - operator(), 632
 - promise, 632
 - resume, 632
- coroutine_traits, 628
- cos, 1824
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- cosf, 1824
- cosh, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- coshf, 1825
- coshl, 1825
- cosl, 1824
- count, 1499
 - allocation_result, 674
 - bitset, 915
 - counted_iterator, 1248
 - duration, 1959
 - ordered associative containers, 982
 - unordered associative containers, 992
- count_down
 - latch, 2272
- count_if, 1499
- counted
 - views, 1350
- counted_iterator, 1246
 - base, 1248
 - constructor, 1247, 1248
 - count, 1248
 - iter_move, 1250
 - iter_swap, 1250
 - operator*, 1248
 - operator+, 1249
 - operator++, 1248
 - operator+=, 1249
 - operator-, 1249
 - operator--, 1248
 - operator-=, 1249
 - operator->, 1248
 - operator<=>, 1250
 - operator=, 1248
 - operator==, 1249
 - operator[], 1248
- counting_semaphore
 - acquire, 2270
 - constructor, 2270
 - max, 2270
 - release, 2270
 - try_acquire, 2270
 - try_acquire_for, 2271
 - try_acquire_until, 2271
- countl_one, 955
 - simd, 1934
- countl_zero, 955
 - simd, 1934
- countr_one, 956
 - simd, 1934
- countr_zero, 956
 - simd, 1934
- cout, 2037
- CP50220
 - text_encoding, 1684
- CP51932
 - text_encoding, 1684
- crbegin, 1270
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - reversible containers, 965
- crbegin(const C& c), 1257
- create_directories, 2167
- create_directory, 2167
- create_directory_symlink, 2167
- create_hard_link, 2168
- create_hard_links
 - copy_options, 2153
- create_symlink, 2168
- create_symlinks
 - copy_options, 2153
- cref
 - reference_wrapper, 923, 924
- crend, 1271
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - reversible containers, 965
- crend(const C& c), 1257
- cross_device_link
 - errc, 656
- cshift
 - valarray, 1814
- ctype, 1645
 - char_type, 1645
 - do_is, 1646
 - do_narrow, 1647
 - do_scan_not, 1647
 - do_tolower, 1647
 - do_toupper, 1647

- do_widen, 1647
- id, 1645
- is, 1646
- narrow, 1646
- scan_is, 1646
- scan_not, 1646
- tolower, 1646
- toupper, 1646
- widen, 1646
- ctype<char>, 1648
 - char_type, 1648
 - classic_table, 1649
 - constructor, 1649
 - ctype<char>, 1649
 - destructor, 1649
 - do_narrow, 1650
 - do_tolower, 1650
 - do_toupper, 1650
 - do_widen, 1650
 - id, 1648
 - is, 1649
 - narrow, 1649
 - scan_is, 1649
 - scan_not, 1649
 - table, 1649
 - table_size, 1648
 - tolower, 1649
 - toupper, 1649
 - widen, 1649
- ctype_base, 1645
 - alnum, 1645
 - alpha, 1645
 - blank, 1645
 - cntrl, 1645
 - digit, 1645
 - do_scan_is, 1646
 - graph, 1645
 - lower, 1645
 - mask, 1645
 - print, 1645
 - punct, 1645
 - space, 1645
 - upper, 1645
 - xdigit, 1645
- ctype_byname, 1647
 - mask, 1647
- curr_symbol
 - money_punct, 1673
- current
 - access_context, 801
 - basic_stacktrace, 667, 668
- current_class, 800
- current_exception, 614
- current_function, 800
- current_namespace, 801
- current_path, 2168
- current_zone, 2011
 - tzdb, 2010
- cv_status, 2262

- no_timeout, 2262
- timeout, 2262
- cw, 822
- cyl_bessel_i, 1833
 - simd, 1928
- cyl_bessel_if, 1833
- cyl_bessel_il, 1833
- cyl_bessel_j, 1833
 - simd, 1928
- cyl_bessel_jf, 1833
- cyl_bessel_jl, 1833
- cyl_bessel_k, 1833
 - simd, 1928
- cyl_bessel_kf, 1833
- cyl_bessel_kl, 1833
- cyl_neumann, 1834
 - simd, 1928
- cyl_neumannf, 1834
- cyl_neumannl, 1834

D

- dangling, 1282
- data, 1272
 - array, 998
 - basic_string, 1624
 - basic_string_view, 1599
 - initializer_list, 619
 - inplace_vector, 1040
 - single_view, 1286
 - span, 1143
 - vector, 1032
- data(C& c), 1258
- data(T (&array)[N]), 1258
- data_handle
 - mdspan, 1176
- data_handle_type
 - aligned_accessor, 1174
 - conjugated_accessor, 1861
 - default_accessor, 1173
 - mdspan, 1175
 - scaled_accessor, 1859
- data_member_options, 811
- data_member_spec, 811
- date
 - leap_second, 2021
- date_order
 - time_get, 1665
- dateorder
 - time_base, 1664
 - time_get_byname, 1668
- day, 1978
 - constructor, 1979
 - from_stream, 1980
 - hash, 2031
 - month_day, 1989
 - ok, 1979
 - operator unsigned, 1979
 - operator""d, 1980

- operator+, 1980
- operator++, 1979
- operator+=", 1979
- operator-, 1980
- operator--, 1979
- operator-=, 1979
- operator<<, 1980
- operator<=>, 1980
- operator==, 1980
- year_month_day, 1996
- year_month_day_last, 1998
- days, 1941
- DBL_DECIMAL_DIG, 596
- DBL_DIG, 596
- DBL_EPSILON, 596
- DBL_HAS_SUBNORM, 2457
- DBL_MANT_DIG, 596
- DBL_MAX, 596
- DBL_MAX_10_EXP, 596
- DBL_MAX_EXP, 596
- DBL_MIN, 596
- DBL_MIN_10_EXP, 596
- DBL_MIN_EXP, 596
- DBL_SNAN, 596
- DBL_TRUE_MIN, 597
- dealias, 798
- deallocate
 - allocator, 693
 - allocator_traits, 691
 - memory_resource, 738
 - polymorphic_allocator, 740
 - scoped_allocator_adaptor, 748
- deallocate_bytes
 - polymorphic_allocator, 740
- deallocate_object
 - polymorphic_allocator, 740
- debug_string
 - range_format, 1686
- dec, 2052, 2079
- deca, 818
- decay, 774, 815
- decay-copy*, 551
- decay_t, 755
- deci, 818
- DECIMAL_DIG, 2457
- decimal_point
 - moneypunct, 1673
 - numpunct, 1662
- declare_no_pointers
 - zombie, 573
- declare_reachable
 - zombie, 573
- declval, 826
- DECMCS
 - text_encoding, 1682
- default_accessor, 1173
 - access, 1173
 - constructor, 1173
 - data_handle_type, 1173
 - element_type, 1173
 - offset, 1174
 - offset_policy, 1173
 - reference, 1173
- default_delete
 - constructor, 698
 - operator(), 698
- default_domain, 2323
 - apply_sender, 2323
 - transform_sender, 2323
- default_error_condition
 - error_category, 658
 - error_code, 660
- default_initializable, 645
- default_random_engine, 1780
- default_searcher, 950
 - constructor, 950
 - operator(), 950
- default_sentinel, 1198
- default_sentinel_t, 1246
- default_zone
 - zoned_traits<const time_zone*>, 2016
- defaultfloat, 2053
- defer_lock, 2252
- defer_lock_t, 2252
- deferred
 - future_status, 2275
 - launch, 2275
- define_aggregate, 812
- define_static_array, 787
- define_static_object, 788
- define_static_string, 787
- delete
 - operator, 602–605, 694
- denorm_absent
 - float_denorm_style, 2457
- denorm_indeterminate
 - float_denorm_style, 2457
- denorm_min
 - numeric_limits, 593
- denorm_present
 - float_denorm_style, 2457
- densities
 - piecewise_constant_distribution, 1803
 - piecewise_linear_distribution, 1805
- dependent_sender, 2320
- depth
 - recursive_directory_iterator, 2163
- deque, 999
 - allocator_type, 966
 - assign, 970
 - assign_range, 970
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 970
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964

- constructor, 1002
- crbegin, 965
- crend, 965
- difference_type, 961
- emplace, 969, 1002
- empty, 963
- end, 962
- erase, 970, 1003
- erase_if, 1003
- get_allocator, 966
- insert, 969, 1002
- insert_range, 969
- iterator, 961
- max_size, 963
- operator!=, 962
- operator<=>, 965
- operator=, 961, 967
- operator==, 962
- push_back, 1002
- push_front, 1002
- rbegin, 964
- reference, 961
- rend, 965
- resize, 1002
- reverse_iterator, 964
- shrink_to_fit, 1002
- size, 963
- size_type, 961
- swap, 962, 967
- value_type, 961
- derived_from, 640
- description
 - stacktrace_entry, 666
- designating_class
 - access_context, 801
- destination_address_required
 - errc, 656
- destroy, 1586
 - allocator_traits, 692
 - coroutine_handle, 630
 - coroutine_handle<noop_coroutine_ -
promise>, 632
 - polymorphic_allocator, 741
 - scoped_allocator_adaptor, 749
- destroy_at, 1586
- destroy_n, 1586
- destroying_delete, 600
- destroying_delete_t, 600
- destructible, 644
- detach
 - jthread, 2200
 - thread, 2197
- device_or_resource_busy
 - errc, 656
- dextents, 1149
- difference_type
 - allocator, 692
 - allocator_traits, 691
 - basic_string, 1607
 - basic_string_view, 1595
 - containers, 961
 - iterator_traits, 1204, 1205
 - pointer_traits, 684
 - scoped_allocator_adaptor, 746
- difftime, 2032
- digit
 - ctype_base, 1645
- digits
 - numeric_limits, 591
- digits10
 - numeric_limits, 591
- dims, 1149
- directories_only
 - copy_options, 2153
- directory
 - file_type, 2152
- directory_entry, 2155
 - assign, 2157
 - constructor, 2156
 - exists, 2157
 - file_size, 2158
 - hard_link_count, 2158
 - is_block_file, 2157
 - is_character_file, 2157
 - is_directory, 2157
 - is_fifo, 2157
 - is_other, 2158
 - is_regular_file, 2158
 - is_socket, 2158
 - is_symlink, 2158
 - last_write_time, 2158
 - operator const filesystem::path&, 2157
 - operator<<, 2159
 - operator<=>, 2159
 - operator==, 2158
 - path, 2157
 - refresh, 2157
 - replace_filename, 2157
 - status, 2158
 - symlink_status, 2158
- directory_iterator, 2159
 - begin, 2160
 - constructor, 2160
 - end, 2161
 - increment, 2160
 - operator++, 2160
 - operator=, 2160
- directory_not_empty
 - errc, 656
- directory_options, 2152
 - follow_directory_symlink, 2154
 - none, 2154
 - skip_permission_denied, 2154
- disable_recursion_pending
 - recursive_directory_iterator, 2163
- disable_sized_range, 1274
- disable_sized_sentinel_for, 1211
 - move_iterator, 1198

- reverse_iterator, 1197
- disabled
 - range_format, 1686
- discard_block_engine, 1776
 - constructor, 1777
 - result_type, 1776
- discrete_distribution, 1800
 - constructor, 1801
 - probabilities, 1801
 - result_type, 1800
- disjunction, 777
- disjunction_v, 758
- display_string
 - path, 2144
- display_string_of, 791
- distance, 1222, 1224
- div, 580, 2177
- div_t, 580
- divides, 925
 - operator(), 925
- divides<>, 925
 - operator(), 925
- DKUS
 - text_encoding, 1682
- dmy
 - time_base, 1664
- do_allocate
 - memory_resource, 738
 - monotonic_buffer_resource, 745
 - synchronized_pool_resource, 744
 - unsynchronized_pool_resource, 744
- do_always_noconv
 - codecvt, 1653
- do_close
 - message, 1676
- do_compare
 - collate, 1664
- do_curr_symbol
 - money_punct, 1674
- do_date_order
 - time_get, 1666
- do_deallocate
 - memory_resource, 738
 - monotonic_buffer_resource, 745
 - synchronized_pool_resource, 744
 - unsynchronized_pool_resource, 744
- do_decimal_point
 - money_punct, 1674
 - num_punct, 1662
- do_encoding
 - codecvt, 1653
- do_falsename
 - num_punct, 1662
- do_frac_digits
 - money_punct, 1674
- do_get
 - messages, 1676
 - money_get, 1670
 - num_get, 1655, 1657
 - time_get, 1667
- do_get_date
 - time_get, 1667
- do_get_monthname
 - time_get, 1667
- do_get_time
 - time_get, 1667
- do_get_weekday
 - time_get, 1667
- do_get_year
 - time_get, 1667
- do_grouping
 - money_punct, 1674
 - num_punct, 1662
- do_hash
 - collate, 1664
- do_in
 - codecvt, 1651
- do_is
 - ctype, 1646
- do_is_equal
 - memory_resource, 738
 - monotonic_buffer_resource, 745
 - synchronized_pool_resource, 744
 - unsynchronized_pool_resource, 744
- do_length
 - codecvt, 1653
- do_max_length
 - codecvt, 1653
- do_narrow, 1649
 - ctype, 1647
 - ctype<char>, 1650
- do_neg_format
 - money_punct, 1674
- do_negative_sign
 - money_punct, 1674
- do_open
 - messages, 1676
- do_out
 - codecvt, 1651
- do_pos_format
 - money_punct, 1674
- do_positive_sign
 - money_punct, 1674
- do_put
 - money_put, 1672
 - num_put, 1658, 1661
 - time_put, 1669
- do_scan_is
 - ctype_base, 1646
- do_scan_not
 - ctype, 1647
- do_thousands_sep
 - money_punct, 1674
 - num_punct, 1662
- do_tolower
 - ctype, 1647
 - ctype<char>, 1650
- do_toupper

- ctype, 1647
- ctype<char>, 1650
- do_transform
 - collate, 1664
- do_truename
 - numpunct, 1662
- do_unshift
 - codecvt, 1652
- do_widen, 1649
 - ctype, 1647
 - ctype<char>, 1650
- domain_error, 650, 651
 - constructor, 651
- done
 - coroutine_handle, 630
 - coroutine_handle<noop_coroutine_-
promise>, 632
- dot, 1868
- dotc, 1868, 1869
- double_t, 1824
- drop
 - views, 1318
- drop_view, 1319
 - base, 1319
 - begin, 1320
 - constructor, 1320
 - end, 1319
 - size, 1319
- drop_while
 - views, 1320
- drop_while_view, 1320
 - base, 1320
 - begin, 1321
 - constructor, 1321
 - end, 1320
 - pred, 1321
- duration, 1957
 - abs, 1963
 - ceil, 1962
 - common_type, 1957
 - constructor, 1958, 1959
 - count, 1959
 - duration_cast, 1962
 - floor, 1962
 - from_stream, 1964
 - hash, 2031
 - max, 1960
 - min, 1960
 - operator"h, 1963
 - operator"min, 1963
 - operator"ms, 1963
 - operator"ns, 1963
 - operator"s, 1963
 - operator"us, 1963
 - operator*, 1960
 - operator*=, 1960
 - operator+, 1959, 1960, 1966
 - operator++, 1959
 - operator+=, 1959

- operator-, 1959, 1960, 1966
- operator--, 1959
- operator=, 1960
- operator/, 1961
- operator/=: 1960
- operator<, 1961
- operator<<, 1963
- operator<=, 1961
- operator<=>, 1961
- operator==, 1961
- operator>, 1961
- operator>=, 1961
- operator%, 1961
- operator%=: 1960
- round, 1962
- zero, 1960
- duration_cast, 1962
- duration, 1962
- duration_values, 1956
 - max, 1957
 - min, 1957
 - zero, 1956
- dynamic_extent, 1138
- dynamic_pointer_cast
 - shared_ptr, 716

E

- e, 1837
- E2BIG, 654
- e_v, 1837
- EACCES, 654
- EADDRINUSE, 654
- EADDRNOTAVAIL, 654
- EAFNOSUPPORT, 654
- EAGAIN, 654
- EALREADY, 654
- earliest
 - choose, 1941
- eback
 - basic_streambuf, 2058
- EBADF, 654
- EBADMSG, 654
- EBCDICATDEA
 - text_encoding, 1683
- EBCDICCAFR
 - text_encoding, 1683
- EBCDICDKNO
 - text_encoding, 1683
- EBCDICDKNOA
 - text_encoding, 1683
- EBCDICES
 - text_encoding, 1683
- EBCDICESA
 - text_encoding, 1683
- EBCDICESSE
 - text_encoding, 1683
- EBCDICFISE
 - text_encoding, 1683

EBCDICFISEA
 text_encoding, 1683
 EBCDICFR
 text_encoding, 1683
 EBCDICIT
 text_encoding, 1683
 EBCDICPT
 text_encoding, 1683
 EBCDICUK
 text_encoding, 1683
 EBCDICUS
 text_encoding, 1684
 EBUSY, 654
 ec
 from_chars_result, 1634
 to_chars_result, 1634
 ECANCELED, 654
 ECHILD, 654
 ECMAScript
 syntax_option_type, 1719, 1720
 ECONNABORTED, 654
 ECONNREFUSED, 654
 ECONNRESET, 654
 EDEADLK, 654
 EDESTADDRREQ, 654
 EDOM, 654
 EEXIST, 654
 EFAULT, 654
 EFBIG, 654
 egamma, 1837
 egamma_v, 1837
 egptr
 basic_streambuf, 2058
 egrep
 syntax_option_type, 1719, 1720
 EHOSTUNREACH, 654
 EIDRM, 654
 EILSEQ, 654
 EINPROGRESS, 654
 EINTR, 654
 EINVAL, 654
 EIO, 654
 EISCONN, 654
 EISDIR, 654
 element_type
 aligned_accessor, 1174
 conjugated_accessor, 1860
 default_accessor, 1173
 mdspan, 1175
 pointer_traits, 684
 scaled_accessor, 1859
 elements
 views, 1354
 elements_view, 1355
 base, 1355
 begin, 1355
 constructor, 1356
 end, 1355
 size, 1355
 elements_view::iterator, 1356
 get_element, 1357
 base, 1357
 constructor, 1357
 operator+, 1358
 operator++, 1358
 operator+=, 1358
 operator-, 1359
 operator--, 1358
 operator-=, 1358
 operator<, 1358
 operator<=, 1358
 operator<=>, 1358
 operator==, 1358
 operator>, 1358
 operator>=, 1358
 elements_view::sentinel, 1359
 base, 1359
 constructor, 1359
 operator-, 1360
 operator==, 1359
 ellint_1, 1834
 simd, 1928
 ellint_1f, 1834
 ellint_1l, 1834
 ellint_2, 1834
 simd, 1928
 ellint_2f, 1834
 ellint_2l, 1834
 ellint_3, 1834
 simd, 1928
 ellint_3f, 1834
 ellint_3l, 1834
 ELOOP, 654
 EMFILE, 654
 emit
 basic_syncbuf, 2126
 emit_on_flush, 2082
 EMLINK, 654
 emplace
 any, 886, 887
 deque, 1002
 expected, 898
 expected<void>, 907
 flat_map, 1111
 flat_multiset, 1136
 hive, 1018
 inplace_vector, 1040
 optional, 859
 ordered associative containers, 979
 priority_queue, 1098
 sequence containers, 969
 unordered associative containers, 989
 variant, 878
 emplace_after
 forward_list, 1009
 emplace_back
 inplace_vector, 1040
 emplace_front

- forward_list, 1007
- emplace_hint
 - ordered associative containers, 979
 - unordered associative containers, 989
- empty, 1272
 - basic_stacktrace, 669
 - basic_string, 1617
 - basic_string_view, 1599
 - containers, 963
 - hazard_pointer, 2293
 - initializer_list, 619
 - iota_view, 1289
 - match_results, 1733
 - mdspan, 1181
 - path, 2147
 - single_view, 1286
 - span, 1143
 - subrange, 1281
 - views, 1262
- empty(C& c), 1257
- empty(T (&array)[N]), 1258
- empty_view, 1285
- EMSGSIZE, 654
- enable_borrowed_range, 1273
 - adjacent_view, 1266
 - as_const_view, 1265
 - as_rvalue_view, 1263
 - basic_string_view, 1594
 - chunk_view, 1267
 - common_view, 1265
 - directory_iterator, 2134
 - drop_view, 1264
 - drop_while_view, 1264
 - elements_view, 1265
 - empty_view, 1262
 - enumerate_view, 1266
 - iota_view, 1262
 - owning_view, 1263
 - recursive_directory_iterator, 2134
 - ref_view, 1263
 - reverse_view, 1265
 - slide_view, 1267
 - span, 1138
 - stride_view, 1267
 - subrange, 1261
 - take_view, 1263
 - zip_view, 1266
- enable_if, 774
- enable_if_t, 755
- enable_nonlocking_formatter_optimization, 1696
- enable_shared_from_this, 721
 - constructor, 721
 - operator=, 721
 - shared_from_this, 721
 - weak_from_this, 721
- enable_view, 1275
 - basic_string_view, 1594
 - directory_iterator, 2134
 - recursive_directory_iterator, 2134
 - span, 1138
- ENAMETOOLONG, 654
- encoding
 - codecvt, 1651
 - locale, 1643
- end, 618, 1269
 - adjacent_transform_view, 1381
 - adjacent_view, 1375
 - array, 997
 - as_input_view, 1418
 - basic_format_parse_context, 1701
 - basic_istream_view, 1297
 - basic_mask, 1935
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - basic_vec, 1910
 - cache_latest_view, 1415
 - cartesian_product_view, 1409, 1410
 - chunk_by_view, 1401
 - chunk_view, 1387, 1390
 - chunk_view::outer-iterator::value_type, 1388
 - common_view, 1350
 - concat_view, 1343
 - containers, 962
 - directory_iterator, 2161
 - drop_view, 1319
 - drop_while_view, 1320
 - elements_view, 1355
 - enumerate_view, 1360
 - filter_view, 1304
 - generator, 1422
 - initializer_list, 619
 - iota_view, 1289
 - join_view, 1321
 - lazy_split_view, 1333
 - lazy_split_view::outer-iterator::value_type, 1336
 - match_results, 1734
 - optional, 860
 - path, 2149
 - recursive_directory_iterator, 2163
 - repeat_view, 1294
 - reverse_view, 1353
 - single_view, 1286
 - slide_view, 1396
 - span, 1144
 - subrange, 1281
 - sys_info, 2014
 - take_view, 1314
 - take_while_view, 1317
 - text_encoding::aliases_view, 1680
 - transform_view, 1309
 - tzdb_list, 2011
 - unordered associative containers, 994
 - valarray, 1814

- zip_transform_view, 1370
- zip_view, 1365
- end(C&), 1256
- end(T (&)[N]), 1256
- endian, 957
 - big, 957
 - little, 957
 - native, 957
- endl, 2079, 2082
- ends, 2082
- ends_with, 1506
 - basic_string, 1626
 - basic_string_view, 1601
- ENETDOWN, 654
- ENETRESET, 654
- ENETUNREACH, 654
- ENFILE, 654
- ENOBUFFS, 654
- ENODATA, 2457
- ENODEV, 654
- ENOENT, 654
- ENOEXEC, 654
- ENOLCK, 654
- ENOLINK, 654
- ENOMEM, 654
- ENOMSG, 654
- ENOPROTOOPT, 654
- ENOSPC, 654
- ENOSR, 2457
- ENOSTR, 2457
- ENOSYS, 654
- ENOTCONN, 654
- ENOTDIR, 654
- ENOTEMPTY, 655
- ENOTRECOVERABLE, 655
- ENOTSOCK, 655
- ENOTSUP, 655
- ENOTTY, 655
- entropy
 - random_device, 1781
- enumerate
 - views, 1360
- enumerate_view, 1360
 - begin, 1360
 - constructor, 1361
 - end, 1360
 - size, 1360
- enumerate_view::iterator, 1361
 - base, 1362
 - constructor, 1362
 - index, 1362
 - operator+, 1363
 - operator++, 1362, 1363
 - operator+=, 1363
 - operator-, 1363
 - operator--, 1363
 - operator-=, 1363
 - operator<=>, 1363
 - operator==, 1363
- enumerate_view::sentinel, 1363
 - base, 1364
 - constructor, 1364
 - operator-, 1364
 - operator==, 1364
- enumerators_of, 805
- env, 2360
 - query, 2361
- env_of_t, 2300
- environment
 - text_encoding, 1679
- environment_is
 - text_encoding, 1679
- ENXIO, 655
- EOF, 2176
- eof
 - basic_ios, 2050
- EOPNOTSUPP, 655
- EOverflow, 655
- EOWNERDEAD, 655
- EPERM, 655
- EPIPE, 655
- ep_ptr
 - basic_streambuf, 2058
- EPROTO, 655
- EPROTONOSUPPORT, 655
- EPROTOTYPE, 655
- epsilon
 - numeric_limits, 592
- equal, 1501
 - istreambuf_iterator, 1254
 - strong_ordering, 622
- equal_range, 1540
 - ordered associative containers, 983
 - unordered associative containers, 993
- equal_to, 926, 929
 - operator(), 926
- equal_to<>, 926
 - operator(), 926
- equality_comparable, 647
- equality_comparable_with, 647
- equivalence_relation, 649
- equivalent, 2169
 - error_category, 658
 - partial_ordering, 620
 - strong_ordering, 622
 - weak_ordering, 621
- ERANGE, 655
- erase
 - basic_string, 1621, 1622, 1630
 - deque, 1003
 - forward_list, 1011
 - hive, 1019, 1020
 - inplace_vector, 1041
 - list, 1025, 1027
 - ordered associative containers, 981, 982
 - sequence containers, 970
 - unordered associative containers, 991, 992
 - vector, 1032

- erase_after
 - forward_list, 1009
 - tzdb_list, 2011
- erase_if
 - basic_string, 1630
 - deque, 1003
 - flat_map, 1114
 - flat_multimap, 1122
 - flat_multiset, 1137
 - flat_set, 1130
 - forward_list, 1011
 - hive, 1020
 - inplace_vector, 1042
 - list, 1027
 - map, 1050
 - multimap, 1054
 - multiset, 1063
 - set, 1059
 - unordered_map, 1072
 - unordered_multimap, 1078
 - unordered_multiset, 1089
 - unordered_set, 1084
 - vector, 1033
- erf, 1826
 - simd, 1928
- erfc, 1826
 - simd, 1928
- erfcf, 1826
- erfcl, 1826
- erff, 1826
- erfl, 1826
- EROFS, 655
- errc, 655
 - address_family_not_supported, 655
 - address_in_use, 655
 - address_not_available, 655
 - already_connected, 655
 - argument_list_too_long, 655
 - argument_out_of_domain, 655
 - bad_address, 655
 - bad_file_descriptor, 655
 - bad_message, 655
 - broken_pipe, 655
 - connection_aborted, 656
 - connection_already_in_progress, 656
 - connection_refused, 656
 - connection_reset, 656
 - cross_device_link, 656
 - destination_address_required, 656
 - device_or_resource_busy, 656
 - directory_not_empty, 656
 - executable_format_error, 656
 - file_exists, 656
 - file_too_large, 656
 - filename_too_long, 656
 - function_not_supported, 656
 - host_unreachable, 656
 - identifier_removed, 656
 - illegal_byte_sequence, 656
 - inappropriate_io_control_operation, 656
 - interrupted, 656
 - invalid_argument, 656
 - invalid_seek, 656
 - io_error, 656
 - is_a_directory, 656
 - make_error_code, 660
 - make_error_condition, 662
 - message_size, 656
 - network_down, 656
 - network_reset, 656
 - network_unreachable, 656
 - no_buffer_space, 656
 - no_child_process, 656
 - no_link, 656
 - no_lock_available, 656
 - no_message, 656
 - no_message_available, 2457
 - no_protocol_option, 656
 - no_space_on_device, 656
 - no_stream_resources, 2457
 - no_such_device, 656
 - no_such_device_or_address, 656
 - no_such_file_or_directory, 656
 - no_such_process, 656
 - not_a_directory, 656
 - not_a_socket, 656
 - not_a_stream, 2457
 - not_connected, 656
 - not_enough_memory, 656
 - not_supported, 656
 - operation_canceled, 656
 - operation_in_progress, 656
 - operation_not_permitted, 656
 - operation_not_supported, 656
 - operation_would_block, 656
 - owner_dead, 656
 - permission_denied, 656
 - protocol_error, 656
 - protocol_not_supported, 656
 - read_only_file_system, 656
 - resource_deadlock_would_occur, 656
 - resource_unavailable_try_again, 656
 - result_out_of_range, 656
 - state_not_recoverable, 656
 - stream_timeout, 2457
 - text_file_busy, 656
 - timed_out, 656
 - too_many_files_open, 656
 - too_many_files_open_in_system, 656
 - too_many_links, 656
 - too_many_symbolic_link_levels, 656
 - value_too_large, 656
 - wrong_protocol_type, 656
- errno, 654
- error
 - bad_expected_access, 891
 - codecvt_base, 1650

- expected, 900
- expected<void>, 907
- unexpected, 890
- error_category, 655, 657
 - constructor, 657
 - default_error_condition, 658
 - destructor, 657
 - equivalent, 658
 - message, 658
 - name, 658
 - operator<=>, 658
 - operator==, 658
- error_code, 655, 659
 - assign, 660
 - category, 660
 - clear, 660
 - constructor, 659, 660
 - default_error_condition, 660
 - hash, 663
 - message, 660
 - operator bool, 660
 - operator<, 660
 - operator<=>, 663
 - operator=, 660
 - operator==, 663
 - value, 660
- error_condition, 655, 661
 - assign, 662
 - category, 662
 - clear, 662
 - constructor, 662
 - message, 662
 - operator bool, 662
 - operator<=>, 663
 - operator=, 662
 - operator==, 663
 - value, 662
- error_or
 - expected, 900
 - expected<void>, 908
- error_type, 1721, 1722
 - expected, 892
 - expected<void>, 903
 - regex_constants, 1721, 1722
- ESPIPE, 655
- ESRCH, 655
- ETIME, 2457
- ETIMEDOUT, 655
- ETXTBSY, 655
- EUCFixWidJapanese
 - text_encoding, 1680
- EUCKR
 - text_encoding, 1681
- EUCPkdFmtJapanese
 - text_encoding, 1680
- event
 - ios_base, 2040
- EWOLDBLOCK, 655
- exa, 818
- exception, 612, 788
 - constructor, 612, 613, 788
 - destructor, 613
 - from, 789
 - operator=, 613
 - u8what, 789
 - what, 613, 788
 - where, 789
- exception_ptr, 614
- exception_ptr_cast, 615
- exceptions
 - basic_ios, 2051
- exchange, 825
 - atomic, 2222
 - atomic<floating-point-type>, 2222
 - atomic<integral-type>, 2222
 - atomic<shared_ptr<T>>, 2236
 - atomic<T*>, 2222
 - atomic<weak_ptr<T>>, 2238
 - atomic_ref, 2212
 - atomic_ref<floating-point-type>, 2212
 - atomic_ref<integral-type>, 2212
 - atomic_ref<pointer-type>, 2212
- exclusive_scan, 1573
- EXDEV, 655
- executable_format_error
 - errc, 656
- execution
 - par, 1432
 - par_unseq, 1432
 - seq, 1432
 - unseq, 1432
- execution::associate_data, 2345
 - constructor, 2346
 - destructor, 2346
 - release, 2347
- execution::impls-for<scope-join-t>, 2377
- execution::impls-for<associate_t>, 2347
- execution::impls-for<spawn_future_t>, 2353
- execution::spawn_future_receiver, 2349
- execution::spawn_future_state_base, 2349
- execution::spawn_future_state, 2350
 - abandon, 2352
 - complete, 2351
 - consume, 2351
 - destroy, 2352
 - try-set-stopped, 2351
- execution::spawn_receiver, 2357
- execution::spawn_state_base, 2357
- execution::spawn_state, 2357
 - complete, 2357
 - run, 2357
 - constructor, 2357
- execution::try-cancelable, 2349
- execution::completion_signatures, 2359
 - count-of, 2359
 - for-each, 2359
- execution::counting_scope, 2380
 - try-associate, 2380

- get_token, 2380
- request_stop, 2380
- token, 2380
- execution::counting_scope::token
 - wrap, 2381
- execution::parallel_policy, 1432
- execution::parallel_unsequenced_policy, 1432
- execution::sequenced_policy, 1432
- execution::simple_counting_scope, 2378
 - disassociate*, 2379
 - start-join-sender*, 2379
 - try-associate*, 2379
 - close, 2379
 - constructor, 2378
 - destructor, 2379
 - get_token, 2379
 - join, 2379
- execution::simple_counting_scope::token, 2379
 - try_associate, 2380
 - wrap, 2380
- execution::unsequenced_policy, 1432
- exists, 2169
 - directory_entry, 2157
- exit, 98, 100, 201, 580, 599, 606
- EXIT_FAILURE, 580
- EXIT_SUCCESS, 580
- exp, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- exp2, 1825
 - simd, 1928
- exp2f, 1825
- exp2l, 1825
- expand
 - simd, 1921
- expected, 888
 - and_then, 901
 - constructor, 894–896
 - destructor, 896
 - emplace, 898
 - error, 900
 - error_or, 900
 - error_type, 892
 - has_error, 900
 - has_value, 900
 - operator bool, 900
 - operator*, 900
 - operator->, 899
 - operator=, 896–898
 - operator==, 902, 903
 - or_else, 901
 - rebind, 892
 - swap, 898, 899
 - transform, 901, 902
 - transform_error, 902
 - unexpected_type, 892
 - value, 900
 - value_or, 900
 - value_type, 892
- expected<void>
 - and_then, 908
 - constructor, 904, 905
 - destructor, 906
 - emplace, 907
 - error, 907
 - error_or, 908
 - error_type, 903
 - has_error, 907
 - has_value, 907
 - operator bool, 907
 - operator*, 907
 - operator=, 906
 - operator==, 909, 910
 - or_else, 908
 - rebind, 903
 - swap, 907
 - transform, 909
 - transform_error, 909
 - unexpected_type, 903
 - value, 907
 - value_type, 903
- expf, 1825
- expint, 1835
 - simd, 1928
- expintf, 1835
- expintl, 1835
- expired
 - weak_ptr, 719
- expl, 1825
- explicit_diagonal, 1853
- explicit_diagonal_t, 1852
- expm1, 1825
 - simd, 1928
- expm1f, 1825
- expm1l, 1825
- exponential_distribution, 1790
 - constructor, 1791
 - lambda, 1791
 - result_type, 1790
- extended
 - syntax_option_type, 1719, 1720
- extension
 - path, 2146
- extent, 768, 817
 - extent_slice, 1182
 - extents, 1149
 - mdspan, 1176
- extent_slice, 1182
 - extent, 1182
 - extent_type, 1182
 - offset, 1182
 - offset_type, 1182
 - stride, 1183
 - stride_type, 1182
- extent_type

- extent_slice, 1182
- extent_v, 757
- extents, 1146
 - constructor, 1147–1149
 - extent, 1149
 - index_type, 1146
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1153
 - layout_left_padded::mapping, 1162
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1176
 - operator==, 1149
 - rank, 1146
 - rank_dynamic, 1146
 - rank_type, 1146
 - size_type, 1146
 - static_extent, 1149
- extents_type
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1152
 - layout_left_padded::mapping, 1161
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1175
- extern_type
 - codecv_t, 1650
- extract, 809
 - flat_map, 1114
 - flat_multiset, 1137
 - flat_set, 1129
 - ordered associative containers, 981
 - unordered associative containers, 991
- extreme_value_distribution, 1793
 - a, 1794
 - b, 1794
 - constructor, 1794
 - result_type, 1793
- F**
- fabs, 1826
 - simd, 1925
- fabsf, 1826
- fabsl, 1826
- facet
 - locale, 1641
- fail
 - basic_ios, 2050
- failed
 - ostreambuf_iterator, 1256
- failure
 - ios_base, 2041
- false_type, 751, 758
- falsename
 - numpunct, 1662
- fclose, 2116, 2176
- fdim, 1828
 - simd, 1925
- fdimf, 1828
- fdiml, 1828
- FE_ALL_EXCEPT, 1751
- FE_DFL_ENV, 1751
- FE_DIVBYZERO, 1751
- FE_DOWNWARD, 1751
- FE_INEXACT, 1751
- FE_INVALID, 1751
- FE_OVERFLOW, 1751
- FE_TONEAREST, 1751
- FE_TOWARDZERO, 1751
- FE_UNDERFLOW, 1751
- FE_UPWARD, 1751
- feclearexcept, 1751
- fegetenv, 1751
- fegetexceptflag, 1751
- fegetround, 1751
- fehldexcept, 1751
- femto, 818
- fenv_t, 1751
- feof, 2176
- feraiseexcept, 1751
- ferror, 2176
- fesetenv, 1751
- fesetexceptflag, 1751
- fesetround, 1751
- fetch_add
 - atomic<floating-point-type>, 2230
 - atomic<integral-type>, 2227
 - atomic<T*>, 2233
 - atomic_ref<floating-point-type>, 2217
 - atomic_ref<integral-type>, 2214
 - atomic_ref<pointer-type>, 2219
- fetch_and
 - atomic<integral-type>, 2227
 - atomic_ref<integral-type>, 2214
- fetch_fmaximum
 - atomic<floating-point-type>, 2230
 - atomic_ref<floating-point-type>, 2217
- fetch_fmaximum_num
 - atomic<floating-point-type>, 2230
 - atomic_ref<floating-point-type>, 2217
- fetch_fminimum
 - atomic<floating-point-type>, 2230
 - atomic_ref<floating-point-type>, 2217
- fetch_fminimum_num
 - atomic<floating-point-type>, 2230
 - atomic_ref<floating-point-type>, 2217
- fetch_max
 - atomic<floating-point-type>, 2230
 - atomic<integral-type>, 2227
 - atomic<T*>, 2233
 - atomic_ref<floating-point-type>, 2217
 - atomic_ref<integral-type>, 2214
 - atomic_ref<pointer-type>, 2219

- fetch_min
 - atomic<floating-point-type>, 2230
 - atomic<integral-type>, 2227
 - atomic<T*>, 2233
 - atomic_ref<floating-point-type>, 2217
 - atomic_ref<integral-type>, 2214
 - atomic_ref<pointer-type>, 2219
- fetch_or
 - atomic<integral-type>, 2227
 - atomic_ref<integral-type>, 2214
- fetch_sub
 - atomic<floating-point-type>, 2230
 - atomic<integral-type>, 2227
 - atomic<T*>, 2233
 - atomic_ref<floating-point-type>, 2217
 - atomic_ref<integral-type>, 2214
 - atomic_ref<pointer-type>, 2219
- fetch_xor
 - atomic<integral-type>, 2227
 - atomic_ref<integral-type>, 2214
- fetestexcept, 1751
- feupdateenv, 1751
- fexcept_t, 1751
- fflush, 2176
- fgetc, 2176
- fgetpos, 2176
- fgets, 2176
- fgetwc, 1747
- fgetws, 1747
- field
 - money_base::pattern, 1672
- fifo
 - file_type, 2152
- FILE, 2176
- file_clock, 1973
 - now, 1974
- file_exists
 - errc, 656
- file_size, 2169
 - directory_entry, 2158
- file_status, 2152
 - constructor, 2153
 - permissions, 2155
 - type, 2155
- file_time, 1941
 - from_stream, 1974
 - operator<<, 1974
- file_too_large
 - errc, 656
- file_type, 2151
 - block, 2152
 - character, 2152
 - directory, 2152
 - fifo, 2152
 - none, 2152
 - not_found, 2152
 - regular, 2152
 - socket, 2152
 - symlink, 2152
 - unknown, 2152
- filebuf, 2034, 2112
- filename
 - path, 2146
- FILENAME_MAX, 2176
- filename_too_long
 - errc, 656
- filesystem_error, 2150
 - constructor, 2151
 - path1, 2151
 - path2, 2151
 - what, 2151
- fill, 1191, 1519
 - array, 998
 - basic_ios, 2049
- fill_n, 1519
- filter
 - views, 1304
- filter_view, 1304
 - base, 1304
 - begin, 1305
 - constructor, 1305
 - end, 1304
 - iterator, 1305
 - pred, 1305
 - sentinel, 1307
- filter_view::iterator, 1305
 - base, 1306
 - constructor, 1306
 - iter_move, 1307
 - iter_swap, 1307
 - operator*, 1306
 - operator++, 1307
 - operator--, 1307
 - operator->, 1306
 - operator==, 1307
- filter_view::sentinel, 1307
 - base, 1308
 - constructor, 1307
 - operator==, 1308
- final_suspend
 - generator::promise_type, 1423
 - task::promise_type, 2373
- find, 1493
 - basic_string, 1624
 - basic_string_view, 1602
 - ordered associative containers, 982
 - unordered associative containers, 992
- find_end, 1496
- find_first_not_of
 - basic_string, 1624
 - basic_string_view, 1602
- find_first_of, 1497
 - basic_string, 1624
 - basic_string_view, 1602
- find_if, 1493
- find_if_not, 1493
- find_last, 1495
- find_last_not_of

- basic_string, 1624
- basic_string_view, 1602
- find_last_of
 - basic_string, 1624
 - basic_string_view, 1602
- finish
 - run_loop, 2363
- first
 - local_info, 2014
 - range_slice, 1183
 - span, 1142, 1143
- first_argument_type
 - zombie, 573
- fisher_f_distribution, 1798
 - constructor, 1799
 - m, 1799
 - n, 1799
 - result_type, 1798
- fixed, 2052
 - chars_format, 1634
- flag_aligned
 - simd, 1893
- flag_convert
 - simd, 1893
- flag_default
 - simd, 1893
- flag_overaligned
 - simd, 1893
- flag_type
 - basic_regex, 1728
- flags
 - ios_base, 1644, 2043
 - simd, 1907
- flat_map, 1102
 - at, 1110
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 1109
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979, 1111
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1114
 - extract, 981, 1114
 - find, 982
 - insert, 979–981, 1111
 - insert_or_assign, 1113
 - insert_range, 1112
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lookup, 1111
 - lower_bound, 983
 - mapped_type, 977
 - max_size, 963, 1110
 - merge, 981
 - node_type, 977
 - operator!=, 962
 - operator<=>, 965
 - operator=, 961
 - operator==, 962
 - operator[], 1110
 - rbegin, 964
 - reference, 961
 - rend, 965
 - replace, 1114
 - reverse_iterator, 964
 - size, 963, 1110
 - size_type, 961
 - swap, 962, 1114
 - try_emplace, 1112
 - upper_bound, 983
 - value_comp, 979
 - value_compare, 977
 - value_type, 961, 977
- flat_multimap, 1114
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 1120, 1121
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1122
 - extract, 981
 - find, 982
 - insert, 979–981
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lower_bound, 983
 - mapped_type, 977

max_size, 963
 merge, 981
 node_type, 977
 operator!=, 962
 operator<=>, 965
 operator=, 961
 operator==, 962
 rbegin, 964
 reference, 961
 rend, 965
 reverse_iterator, 964
 size, 963
 size_type, 961
 swap, 962
 upper_bound, 983
 value_comp, 979
 value_compare, 977
 value_type, 961, 977
 flat_multiset, 1130
 begin, 962
 cbegin, 962
 cend, 962
 clear, 982
 const_iterator, 961
 const_reference, 961
 const_reverse_iterator, 964
 constructor, 1135
 contains, 983
 count, 982
 crbegin, 965
 crend, 965
 difference_type, 961
 emplace, 979, 1136
 emplace_hint, 979
 empty, 963
 end, 962
 equal_range, 983
 erase, 981, 982
 erase_if, 1137
 extract, 981, 1137
 find, 982
 insert, 979–981, 1136
 insert_range, 1136, 1137
 iterator, 961
 key_comp, 979
 key_compare, 977
 key_type, 977
 lower_bound, 983
 mapped_type, 977
 max_size, 963
 merge, 981
 node_type, 977
 operator!=, 962
 operator<=>, 965
 operator=, 961
 operator==, 962
 rbegin, 964
 reference, 961
 rend, 965
 replace, 1129
 reverse_iterator, 964
 size, 963
 size_type, 961
 swap, 962, 1129
 upper_bound, 983
 value_comp, 979
 value_compare, 977
 value_type, 961, 977
 flat_set, 1122, 1123
 begin, 962
 cbegin, 962
 cend, 962
 clear, 982
 const_iterator, 961
 const_reference, 961
 const_reverse_iterator, 964
 constructor, 1127, 1128
 contains, 983
 count, 982
 crbegin, 965
 crend, 965
 difference_type, 961
 emplace, 979
 emplace_hint, 979
 empty, 963
 end, 962
 equal_range, 983
 erase, 981, 982
 erase_if, 1130
 extract, 981, 1129
 find, 982
 insert, 979–981, 1128, 1129
 insert_range, 1129
 iterator, 961
 key_comp, 979
 key_compare, 977
 key_type, 977
 lower_bound, 983
 mapped_type, 977
 max_size, 963
 merge, 981
 node_type, 977
 operator!=, 962
 operator<=>, 965
 operator=, 961
 operator==, 962
 rbegin, 964
 reference, 961
 rend, 965
 replace, 1129
 reverse_iterator, 964
 size, 963
 size_type, 961
 swap, 962, 1129
 upper_bound, 983
 value_comp, 979
 value_compare, 977
 value_type, 961, 977

- flip
 - bitset, 914
 - bitset::reference, 912
 - vector<bool>, 1035
 - vector<bool>::reference, 1035
- float128_t, 599
- float16_t, 599
- float32_t, 599
- float64_t, 599
- float_denorm_style, 2457
 - denorm_absent, 2457
 - denorm_indeterminate, 2457
 - denorm_present, 2457
 - numeric_limits, 2457
- float_round_style, 589
- float_t, 1824
- floating_point, 642
- floor, 1826
 - duration, 1962
 - simd, 1925
 - time_point, 1967
- floorf, 1826
- floorl, 1827
- FLT_DECIMAL_DIG, 596
- FLT_DIG, 596
- FLT_EPSILON, 596
- FLT_EVAL_METHOD, 596
- FLT_HAS_SUBNORM, 2457
- FLT_MANT_DIG, 596
- FLT_MAX, 596
- FLT_MAX_10_EXP, 596
- FLT_MAX_EXP, 596
- FLT_MIN, 596
- FLT_MIN_10_EXP, 596
- FLT_MIN_EXP, 596
- FLT_RADIX, 596
- FLT_ROUNDS, 596
- FLT_SNAN, 596
- FLT_TRUE_MIN, 596
- flush, 2043, 2065, 2077, 2082
 - basic_ostream, 2082
- flush_emit, 2082
- fma, 1828
 - simd, 1925
- fmaf, 1828
- fmal, 1828
- fmax, 1828
 - simd, 1925
- fmaxf, 1828
- fmaximum, 1828
- fmaximum_num, 1828
- fmaxl, 1828
- fmin, 1828
 - simd, 1925
- fminf, 1828
- fminimum, 1828
- fminimum_num, 1828
- fminl, 1828
- fmod, 1827
 - simd, 1925
- fmodf, 1827
- fmodl, 1827
- fmtflags
 - ios_base, 2041, 2083
- fold_left, 1507
- fold_left_first, 1507
- fold_left_first_with_iter, 1508
- fold_left_first_with_iter_result, 1447
- fold_left_with_iter, 1508
- fold_left_with_iter_result, 1447
- fold_right, 1508
- fold_right_last, 1508
- follow_directory_symlink
 - directory_options, 2154
- fopen, 2115, 2176
- FOPEN_MAX, 2176
- for_each, 1491
- for_each_n, 1492, 1493
- for_each_n_result, 1435
- for_each_result, 1435
- format, 1694, 2023–2027
 - basic_format_arg::handle, 1710
 - formatter, 661, 1036, 1138, 1712, 2150
 - formatter<chrono::zoned_time>, 2027
 - match_results, 1734, 1735
 - range-default-formatter*, 1706–1708
 - range_formatter, 1705
- format_args, 1684
- format_context, 1684, 1702
- format_default, 1719, 1721
- format_error, 1713
 - constructor, 1713
- format_first_only, 1719, 1721, 1738
- format_kind, 1703
- format_no_copy, 1719, 1721, 1738
- format_parse_context, 1684
- format_sed, 1719, 1721
- format_string, 1685
- format_to, 1694, 1695
- format_to_n, 1695
- format_to_n_result, 1684
 - out, 1684
 - size, 1684
- formattable, 1697
- formatted_size, 1696
- formatter, 660, 1036, 1137, 1698, 1711, 2149
 - basic_format_parse_context, 2150
 - format, 661, 1036, 1138, 1712, 2150
 - parse, 661, 1036, 1138, 1712
 - set_brackets, 1712
 - set_debug_format, 661, 2150
 - set_separator, 1712
 - specializations
 - character types, 1698
 - chrono::file_time, 2026
 - chrono::gps_time, 2026
 - chrono::local-time-format-t, 2027
 - chrono::local_time, 2026

- chrono::sys_time, 2026
- chrono::tai_time, 2026
- chrono::utc_time, 2026
- chrono::zoned_time, 2027
- floating-point types, 1698
- integer types, 1698
- nullptr_t, 1698
- pointer types, 1698
- string types, 1698
- thread::id, 2195
- formatter<chrono::zoned_time>
 - format, 2027
- formatter_type
 - basic_format_context, 1702
- forward, 825
- forward_as_tuple, 847
 - tuple, 847
- forward_iterator, 1212
- forward_iterator_tag, 1221
- forward_like, 825
- forward_list
 - allocator_type, 966
 - assign, 970
 - assign_range, 970
 - before_begin, 1007
 - begin, 962
 - cbefore_begin, 1007
 - cbegin, 962
 - cend, 962
 - clear, 970, 1009
 - const_iterator, 961
 - const_reference, 961
 - constructor, 1007
 - difference_type, 961
 - emplace, 969
 - emplace_after, 1009
 - emplace_front, 1007
 - empty, 963
 - end, 962
 - erase, 970, 1011
 - erase_after, 1009
 - erase_if, 1011
 - front, 1007
 - get_allocator, 966
 - insert, 969
 - insert_after, 1008
 - insert_range, 969
 - insert_range_after, 1008
 - iterator, 961
 - max_size, 963
 - merge, 1010
 - operator!=, 962
 - operator<=>, 965
 - operator=, 961, 967
 - operator==, 962
 - pop, 1008
 - prepend_range, 1008
 - push_front, 1008
 - reference, 961
 - remove, 1010
 - remove_if, 1010
 - resize, 1009
 - reverse, 1011
 - size, 963
 - size_type, 961
 - sort, 1011
 - splice_after, 1009, 1010
 - swap, 962, 967
 - unique, 1010
 - value_type, 961
- forward_progress_guarantee, 2306
- forward_range, 1275
- forwarding_query, 2299
- forwarding_query_t, 2299
- found
 - in_found_result, 1488
- FP_FAST_FMA, 1824
- FP_FAST_FMAF, 1824
- FP_FAST_FMAL, 1824
- FP_ILOGB0, 1824
- FP_ILOGBNAN, 1824
- FP_INFINITE, 1824
- FP_NAN, 1824
- FP_NORMAL, 1824
- FP_SUBNORMAL, 1824
- FP_ZERO, 1824
- fpclassify, 1828
 - simd, 1925
- fpos, 2034, 2039, 2045, 2046
 - state, 2046
- fpos_t, 2176
- fprintf, 2176
- fputc, 2176
- fputs, 2176
- fputwc, 1747
- fputws, 1747
- frac_digits
 - money_punct, 1673
- fread, 2176
- free, 580, 694
- freeze
 - zombie, 573
- freopen, 2176
- frexp, 1825
 - simd, 1931
- frexpf, 1825
- frexpl, 1825
- from
 - exception, 789
- from_address
 - coroutine_handle, 630
- from_bytes
 - zombie, 573
- from_chars, 1636
- from_chars_result, 1634
 - ec, 1634
 - ptr, 1634
- from_overaligned, 696

from_promise
 coroutine_handle, 630
 from_stream
 day, 1980
 duration, 1964
 file_time, 1974
 gps_time, 1973
 local_time, 1975
 month, 1982
 month_day, 1989
 sys_time, 1968
 tai_time, 1972
 utc_time, 1970
 weekday, 1987
 year, 1984
 year_month, 1994
 year_month_day, 1997
 from_sys
 utc_clock, 1969
 from_tagged, 696
 from_time_t
 system_clock, 1968
 from_utc
 gps_clock, 1973
 tai_clock, 1971
 front
 basic_string, 1617
 basic_string_view, 1599
 forward_list, 1007
 span, 1143
 tzdb_list, 2011
 view_interface, 1278
 front_insert_iterator, 1230
 constructor, 1230
 operator*, 1231
 operator++, 1231
 operator=, 1230, 1231
 front_inserter, 1231
 fscanf, 2176
 fseek, 2115, 2176
 fsetpos, 2176
 fstream, 2034, 2112
 ftell, 2176
 full_extent, 1145
 full_extent_t, 1145
 fun
 in_fun_result, 1486
 function, 937
 constructor, 938
 destructor, 939
 invocation, 939
 operator bool, 939
 operator(), 939
 operator=, 939
 operator==, 940
 result_type, 937
 swap, 939, 940
 target, 940
 target_type, 939
 function_not_supported
 errc, 656
 function_ref, 946
 constructor, 947, 948
 operator(), 949
 operator=, 948
 future, 2280
 constructor, 2280
 get, 2281
 operator=, 2280
 share, 2281
 valid, 2281
 wait, 2281
 wait_for, 2281
 wait_until, 2281
 future_already_retrieved
 future_errc, 2274
 future_category, 2276
 future_errc, 2274
 broken_promise, 2274
 future_already_retrieved, 2274
 make_error_code, 2276
 make_error_condition, 2276
 no_state, 2274
 promise_already_satisfied, 2274
 future_error, 2276
 code, 2276
 constructor, 2276
 what, 2276
 future_status, 2275
 deferred, 2275
 ready, 2275
 timeout, 2275
 fwide, 1747
 fwprintf, 1747
 fwrite, 2176
 fwscanf, 1747

G
 gamma_distribution, 1791
 alpha, 1792
 beta, 1792
 constructor, 1792
 result_type, 1791
 GB18030
 text_encoding, 1682
 GB2312
 text_encoding, 1683
 GBK
 text_encoding, 1682
 gbump
 basic_streambuf, 2058
 gcd, 1578
 gcount
 basic_istream, 2069
 general
 chars_format, 1634
GENERALIZED_NONCOMMUTATIVE_SUM, 1569

GENERALIZED_SUM, 1569
 generate, 1520
 seed_seq, 1782
 generate_canonical, 1783
 generate_header
 zombie, 573
 generate_n, 1520
 generate_random, 1586, 1587
 generator, 1420
 begin, 1422
 constructor, 1421
 destructor, 1421
 end, 1422
 operator=, 1421
 generator::iterator
 constructor, 1425
 operator*, 1425
 operator++, 1425
 operator=, 1425
 operator==, 1425
 generator::promise_type
 final_suspend, 1423
 get_return_object, 1423
 operator delete, 1424
 operator new, 1424
 unhandled_exception, 1424
 yield_value, 1423
 generic_category, 657, 658
 generic_display_string
 path, 2145
 generic_string
 path, 2145, 2463
 generic_system_encoded_string
 path, 2145
 generic_u16string
 path, 2145
 generic_u32string
 path, 2145
 generic_u8string
 path, 2145
 generic_wstring
 path, 2145
 geometric_distribution, 1788
 constructor, 1788
 p, 1788
 result_type, 1788
 get
 array, 999
 basic_format_args, 1711
 basic_istream, 2069, 2070
 complex, 1759
 future, 2281
 integer_sequence, 751
 messages, 1675
 money_get, 1670
 num_get, 1655
 pair, 837
 pointer_tag_pair, 697
 reference_wrapper, 923
 shared_future, 2283
 shared_ptr, 711
 subrange, 1282
 time_get, 1666
 tuple, 849, 850
 unique_ptr, 702
 variant, 880
 get_allocator, 2299
 allocator-aware containers, 966
 basic_stacktrace, 668
 basic_string, 1624
 basic_stringbuf, 2093
 basic_syncbuf, 2127
 indirect, 731
 match_results, 1735
 polymorphic, 736
 get_allocator_t, 2299
 get_completion_domain, 2299
 get_completion_domain_t, 2299
 get_completion_scheduler, 2299
 get_completion_scheduler_t, 2299
 get_completion_signatures, 2324
 basic-sender, 2318
 not-a-sender, 2318
 task, 2371
 get_date
 time_get, 1665
 get_default_resource, 741
 get_delegation_scheduler, 2299
 get_delegation_scheduler_t, 2299
 get_deleter
 shared_ptr, 716
 unique_ptr, 702
 get_domain, 2299
 get_domain_t, 2299
 get_env, 2300
 task::promise_type, 2374
 get_env_t, 2300
 get_forward_progress_guarantee, 2299
 get_forward_progress_guarantee_t, 2299
 get_future
 packaged_task, 2287
 promise, 2278
 get_id
 jthread, 2200
 this_thread, 2201
 thread, 2197
 get_if, 880
 variant, 880
 get_info
 time_zone, 2015
 zoned_time, 2020
 get_iterator
 hive, 1020
 get_leap_second_info, 1970
 get_local_time
 zoned_time, 2020
 get_money, 2084
 get_monthname

- time_get, 1666
- get_new_handler, 574, 606
- get_parallel_scheduler, 2382
- get_pointer_safety
 - zombie, 573
- get_return_object
 - generator::promise_type, 1423
 - task::promise_type, 2373
- get_scheduler, 2299
 - run_loop, 2363
- get_scheduler_t, 2299
- get_start_scheduler, 2299
- get_start_scheduler_t, 2299
- get_stop_source
 - jthread, 2200
- get_stop_token, 2299
 - jthread, 2200
- get_stop_token_t, 2299
- get_sys_time
 - zoned_time, 2020
- get_temporary_buffer
 - zombie, 573
- get_terminate, 574, 613
- get_time, 2085
 - time_get, 1665
- get_time_zone
 - zoned_time, 2020
- get_token
 - execution::counting_scope, 2380
 - execution::simple_counting_scope, 2379
 - inplace_stop_source, 2191
 - stop_source, 2188
- get_tzdb, 2011
- get_tzdb_list, 2011
- get_unexpected
 - zombie, 573
- get_weekday
 - time_get, 1666
- get_wrapped
 - basic_syncbuf, 2127
- get_year
 - time_get, 1666
- getc, 2176
- getchar, 2176
- getenv, 580, 632
- getline
 - basic_istream, 2070, 2071
 - basic_string, 1629, 1630
- getloc, 1724
 - basic_regex, 1728
 - basic_streambuf, 2056
 - ios_base, 2044
- gets
 - zombie, 573
- getwc, 1747
- getwchar, 1747
- giga, 818
- global
 - locale, 1643
- gmtime, 2032
- gmtime_r, 2032
- good
 - basic_ios, 2050
- gps_clock, 1972
 - from_utc, 1973
 - now, 1973
 - to_utc, 1973
- gps_seconds, 1941
- gps_time, 1941
 - from_stream, 1973
 - operator<<, 1973
- gptr
 - basic_streambuf, 2058
- graph
 - ctype_base, 1645
- greater, 927, 929
 - operator(), 927
 - partial_ordering, 620
 - strong_ordering, 622
 - weak_ordering, 621
- greater<>, 927
 - operator(), 927
- greater_equal, 928, 930
 - operator(), 928
- greater_equal<>, 928
 - operator(), 928
- grep
 - syntax_option_type, 1719, 1720
- group_all
 - perms, 2154
- group_exec
 - perms, 2154
- group_read
 - perms, 2154
- group_write
 - perms, 2154
- grouping
 - moneypunct, 1673
 - numpunct, 1662
- gslice, 1819
 - constructor, 1820
 - size, 1820
 - start, 1820
 - stride, 1820
- gslice_array, 1820, 1821
 - operator==, 1821
 - operator+=, 1821
 - operator-=, 1821
 - operator/=: 1821
 - operator<=, 1821
 - operator=, 1821
 - operator>=, 1821
 - operator%=: 1821
 - operator&=: 1821
 - operator^=: 1821
 - operator|=, 1821
 - value_type, 1820

H

- HalfWidthKatakana
 - text_encoding, 1680
- handle
 - basic_format_arg, 1710
- hard_link_count, 2169
 - directory_entry, 2158
- hardware_concurrency
 - jthread, 2201
 - thread, 2197
- hardware_constructive_interference_size, 607
- hardware_destructive_interference_size, 606
- has_automatic_storage_duration, 795
- has_c_language_linkage, 795
- has_default_argument, 796
- has_default_member_initializer, 797
- has_denorm_loss
 - numeric_limits, 2457
- has_error
 - expected, 900
 - expected<void>, 907
- has_extension
 - path, 2147
- has_external_linkage, 795
- has_facet
 - locale, 1644
- has_filename
 - path, 2147
- has_identifier, 789
- has_inaccessible_bases, 803
- has_inaccessible_nonstatic_data_members, 803
- has_inaccessible_subobjects, 803
- has_infinity
 - numeric_limits, 592
- has_internal_linkage, 795
- has_linkage, 795
- has_module_linkage, 795
- has_parent, 797
- has_parent_path
 - path, 2147
- has_quiet_NaN
 - numeric_limits, 592
- has_relative_path
 - path, 2147
- has_root_directory
 - path, 2147
- has_root_name
 - path, 2147
- has_root_path
 - path, 2147
- has_signaling_NaN
 - numeric_limits, 593
- has_single_bit, 954
 - simd, 1933
- has_static_storage_duration, 795
- has_stem
 - path, 2147
- has_template_arguments, 798
- has_thread_storage_duration, 795
- has_unique_object_representations, 765, 768, 815
- has_unique_object_representations_v, 757
- has_value
 - any, 887
 - expected, 900
 - expected<void>, 907
 - optional, 861
- has_virtual_destructor, 765, 815
- has_virtual_destructor_v, 757
- hash, 952
 - basic_string, 1632
 - collate, 1663
 - coroutine_handle, 631
 - day, 2031
 - duration, 2031
 - error_code, 663
 - indirect, 731
 - leap_second, 2032
 - monostate, 824, 883
 - month, 2031
 - month_day, 2031
 - month_day_last, 2031
 - month_weekday, 2031
 - month_weekday_last, 2031
 - optional, 871
 - shared_ptr, 722
 - string, 1632
 - string_view, 1603
 - text_encoding, 1684
 - thread::id, 2195
 - time_point, 2031
 - type_index, 609
 - u16string, 1632
 - u16string_view, 1603
 - u32string, 1632
 - u32string_view, 1603
 - u8string, 1632
 - u8string_view, 1603
 - unique_ptr, 721
 - variant, 883
 - weekday, 2031
 - weekday_indexed, 2031
 - weekday_last, 2031
 - wstring, 1632
 - wstring_view, 1603
 - year, 2031
 - year_month, 2031
 - year_month_day, 2031
 - year_month_day_last, 2031
 - year_month_weekday, 2031
 - year_month_weekday_last, 2031
 - zoned_time, 2032
- hash_code, 916
 - type_index, 609

- type_info, 608
- hash_function
 - unordered associative containers, 989
- hash_value
 - path, 2149
- hasher
 - unordered associative containers, 986
- hazard_pointer
 - constructor, 2293
 - destructor, 2293
 - empty, 2293
 - operator=, 2293
 - protect, 2293
 - reset_protection, 2294
 - swap, 2294
 - try_protect, 2293
- hazard_pointer_obj_base
 - retire, 2292
- hecto, 818
- hermite, 1835
 - simd, 1931
- hermitef, 1835
- hermitel, 1835
- hermitian_matrix_rank_1_update, 1879
- hermitian_matrix_rank_2_update, 1881
- hermitian_matrix_rank_2k_update, 1887
- hermitian_matrix_rank_k_update, 1886
- hermitian_matrix_vector_product, 1874
- hex, 2052
 - chars_format, 1634
- hexfloat, 2053
- hh_mm_ss
 - hours, 2009
 - is_negative, 2008
 - minutes, 2009
 - operator precision, 2009
 - seconds, 2009
 - subseconds, 2009
 - to_duration, 2009
- high_resolution_clock, 1975
- hive, 1013
 - allocator_type, 966
 - begin, 962
 - block_capacity_default_limits, 1018
 - block_capacity_hard_limits, 1018
 - block_capacity_limits, 1017
 - capacity, 1017
 - cbegin, 962
 - cend, 962
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 1015, 1016
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 1018
 - empty, 963
 - end, 962
 - erase, 1019, 1020
 - erase_if, 1020
 - get_allocator, 966
 - get_iterator, 1020
 - insert, 1018, 1019
 - is_within_hard_limits, 1018
 - iterator, 961
 - max_size, 963
 - operator=, 961, 967, 1016
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reserve, 1017
 - reshape, 1018
 - reverse_iterator, 964
 - shrink_to_fit, 1017
 - size, 963
 - size_type, 961
 - sort, 1020
 - splice, 1019
 - swap, 962, 967, 1019
 - trim_capacity, 1017
 - unique, 1019
 - value_type, 961
- hive_limits, 1011
 - max, 1011
 - min, 1011
- hms, 2007
- holds_alternative, 880
 - variant, 880
- host_unreachable
 - errc, 656
- hours, 1941
 - hh_mm_ss, 2009
- HPDesktop
 - text_encoding, 1683
- HPLegal
 - text_encoding, 1683
- HPMath8
 - text_encoding, 1683
- HPPiFont
 - text_encoding, 1683
- HPPSMath
 - text_encoding, 1683
- HPRoman8
 - text_encoding, 1682
- HUGE_VAL, 1824
- HUGE_VALF, 1824
- HUGE_VALL, 1824
- hypot, 1826
 - 3-argument form, 1831
 - simd, 1928
- hypotf, 1826
- hypotl, 1826
- HZGB2312
 - text_encoding, 1684

I

IBM00858	text_encoding, 1684
IBM00924	text_encoding, 1684
IBM01140	text_encoding, 1684
IBM01141	text_encoding, 1684
IBM01142	text_encoding, 1684
IBM01143	text_encoding, 1684
IBM01144	text_encoding, 1684
IBM01145	text_encoding, 1684
IBM01146	text_encoding, 1684
IBM01147	text_encoding, 1684
IBM01148	text_encoding, 1684
IBM01149	text_encoding, 1684
IBM037	text_encoding, 1683
IBM038	text_encoding, 1683
IBM1026	text_encoding, 1683
IBM1047	text_encoding, 1684
IBM273	text_encoding, 1683
IBM274	text_encoding, 1683
IBM275	text_encoding, 1683
IBM277	text_encoding, 1683
IBM278	text_encoding, 1683
IBM280	text_encoding, 1683
IBM281	text_encoding, 1683
IBM284	text_encoding, 1683
IBM285	text_encoding, 1683
IBM290	text_encoding, 1683
IBM297	text_encoding, 1683
IBM420	text_encoding, 1683
IBM423	text_encoding, 1683
IBM424	text_encoding, 1683
IBM500	text_encoding, 1683
IBM851	text_encoding, 1683
IBM855	text_encoding, 1683
IBM857	text_encoding, 1683
IBM860	text_encoding, 1683
IBM861	text_encoding, 1683
IBM863	text_encoding, 1683
IBM864	text_encoding, 1683
IBM865	text_encoding, 1683
IBM866	text_encoding, 1684
IBM868	text_encoding, 1683
IBM869	text_encoding, 1683
IBM870	text_encoding, 1683
IBM871	text_encoding, 1683
IBM880	text_encoding, 1683
IBM891	text_encoding, 1683
IBM903	text_encoding, 1683
IBM904	text_encoding, 1683
IBM905	text_encoding, 1683
IBM918	text_encoding, 1683
IBMEBCDICATDE	text_encoding, 1683
IBMSymbols	text_encoding, 1683
IBMThai	text_encoding, 1683
icase	syntax_option_type, 1719, 1720
id	codecvt, 1650 collate, 1663 ctype, 1645 ctype<char>, 1648 locale, 1642 messages, 1675 money_get, 1670

- money_put, 1671
- money_punct, 1673
- num_get, 1654
- num_put, 1658
- num_punct, 1661
- text_encoding, 1680
- thread, 2194
- time_get, 1665
- time_put, 1668
- identifier_of, 791
- identifier_removed
 - errc, 656
- identity, 933
- ifstream, 2034, 2112
- ignore, 847
 - basic_istream, 2071
- illegal_byte_sequence
 - errc, 656
- ilogb, 1825
 - simd, 1925
- ilogbf, 1825
- ilogbl, 1825
- imag, 1759
 - basic_vec, 1913
 - complex, 1755, 1757
 - simd, 1934
- imaxabs, 2177
- imaxdiv, 2177
- imaxdiv_t, 2177
- imbue, 1724
 - basic_filebuf, 2118
 - basic_ios, 2049
 - basic_regex, 1728
 - basic_streambuf, 2058
 - ios_base, 2044
- implicit_unit_diagonal, 1852
- implicit_unit_diagonal_t, 1852
- in
 - codecvt, 1651
 - in_found_result, 1488
 - in_fun_result, 1486
 - in_out_out_result, 1487
 - in_out_result, 1487
 - in_value_result, 1488
- in1
 - in_in_out_result, 1487
 - in_in_result, 1486
- in2
 - in_in_out_result, 1487
 - in_in_result, 1486
- in_avail
 - basic_streambuf, 2057
- in_found_result, 1488
 - found, 1488
 - in, 1488
- in_fun_result, 1486
 - fun, 1486
 - in, 1486
- in_in_out_result, 1487
 - in1, 1487
 - in2, 1487
 - out, 1487
- in_in_result, 1486
 - in1, 1486
 - in2, 1486
- in_out_out_result, 1487
 - in, 1487
 - out1, 1487
 - out2, 1487
- in_out_result, 1487
 - in, 1487
 - out, 1487
- in_place, 824
- in_place_index, 824
- in_place_index_t, 824
- in_place_t, 824
- in_place_type, 824
- in_place_type_t, 824
- in_range, 827
- in_value_result, 1488
 - in, 1488
 - value, 1488
- inappropriate_io_control_operation
 - errc, 656
- includes, 1548
- inclusive_scan, 1574
- increment
 - directory_iterator, 2160
 - recursive_directory_iterator, 2163
- incrementable, 1210
- incrementable_traits, 1202
- independent_bits_engine, 1777, 1778
 - result_type, 1778
- indeterminate_domain, 2322
 - constructor, 2322
 - transform_sender, 2323
- index
 - enumerate_view::iterator, 1362
 - variant, 879
 - weekday_indexed, 1987
 - year_month_weekday, 2001
- index_sequence, 822
- index_sequence_for, 822
- index_type
 - extents, 1146
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1152
 - layout_left_padded::mapping, 1161
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1175
- indices
 - views, 1287
- indirect, 726
 - synth-three-way-result, 731
 - constructor, 727–729

- destructor, 729
- get_allocator, 731
- hash, 731
- operator*, 730
- operator->, 730
- operator=, 729, 730
- operator==, 731
- swap, 731
- valueless_after_move, 731
- indirect_array, 1822
 - operator*=, 1823
 - operator+=, 1823
 - operator-=, 1823
 - operator/=: 1823
 - operator<=, 1823
 - operator=, 1823, 1824
 - operator>=, 1823
 - operator[], 1823
 - operator%=, 1823
 - operator&=, 1823
 - operator^=, 1823
 - operator|=, 1823
 - value_type, 1822
- indirect_binary_predicate, 1219
- indirect_equivalence_relation, 1219
- indirect_result_t, 1194
- indirect_strict_weak_order, 1219
- indirect_unary_predicate, 1218
- indirectly_comparable, 1221
- indirectly_copyable, 1220
- indirectly_copyable_storable, 1220
- indirectly_movable, 1220
- indirectly_movable_storable, 1220
- indirectly_readable, 1207
- indirectly_readable_traits, 1202
- indirectly_regular_unary_invocable, 1218
- indirectly_swappable, 1220
- indirectly_unary_invocable, 1218
- indirectly_writable, 1208
- INFINITY, 596, 1824
- infinity
 - numeric_limits, 593
- Init
 - ios_base, 2043
- init
 - basic_ios, 2048, 2065
 - basic_ostream, 2076
- initial_suspend
 - task::promise_type, 2373
- initializer_list, 618
 - begin, 619
 - constructor, 618
 - data, 619
 - empty, 619
 - end, 619
 - size, 619
- inlinable_receiver, 2310
- inline_scheduler, 2303, 2367
- inner_allocator
 - scoped_allocator_adaptor, 748
- inner_allocator_type
 - scoped_allocator_adaptor, 747
- inner_product, 1571
- inout_ptr, 725
- inout_ptr_t, 724
 - constructor, 724
 - destructor, 725
- inplace_merge, 1547
- inplace_stop_callback, 2191
 - callback_type, 2191
 - constructor, 2192
 - destructor, 2192
- inplace_stop_source, 2190
 - constructor, 2191
 - get_token, 2191
 - request_stop, 2191
 - stop_possible, 2191
 - stop_requested, 2191
- inplace_stop_token, 2190
 - callback_type, 2190
 - stop_possible, 2190
 - stop_requested, 2190
 - swap, 2190
- inplace_vector, 1036
 - append_range, 1040
 - assign, 970
 - assign_range, 970
 - begin, 962
 - capacity, 1039
 - cbegin, 962
 - cend, 962
 - clear, 970
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 1039
 - crbegin, 965
 - crend, 965
 - data, 1040
 - difference_type, 961
 - emplace, 969, 1040
 - emplace_back, 1040
 - empty, 963
 - end, 962
 - erase, 970, 1041
 - erase_if, 1042
 - insert, 969, 1040
 - insert_range, 969, 1040
 - iterator, 961
 - max_size, 963, 1039
 - operator!=, 962
 - operator=, 961
 - operator==, 962
 - pop_back, 1041
 - push_back, 1040
 - rbegin, 964
 - reference, 961
 - rend, 965

- reserve, 1040
- resize, 1040
- reverse_iterator, 964
- shrink_to_fit, 1040
- size, 963
- size_type, 961
- swap, 962, 1041
- try_emplace_back, 1041
- try_push_back, 1041
- unchecked_emplace_back, 1041
- unchecked_push_back, 1041
- value_type, 961
- input_iterator, 1211
- input_iterator_tag, 1221
- input_or_output_iterator, 1210
- input_range, 1275
- insert
 - basic_string, 1620, 1621
 - deque, 1002
 - flat_map, 1111
 - flat_multiset, 1136
 - flat_set, 1128, 1129
 - hive, 1018, 1019
 - inplace_vector, 1040
 - list, 1024
 - map, 1048
 - multimap, 1054
 - ordered associative containers, 979–981
 - sequence containers, 969
 - set, 1059
 - unordered associative containers, 989, 990
 - unordered_map, 1070
 - unordered_multimap, 1077
 - unordered_set, 1084
 - vector, 1032
- insert_after
 - forward_list, 1008
- insert_iterator, 1231
 - constructor, 1231
 - operator*, 1232
 - operator++, 1232
 - operator=, 1231
- insert_or_assign
 - flat_map, 1113
 - map, 1049, 1050
 - unordered_map, 1071, 1072
- insert_range
 - basic_string, 1621
 - flat_map, 1112
 - flat_multiset, 1136, 1137
 - flat_set, 1129
 - inplace_vector, 1040
 - sequence containers, 969
 - unordered associative containers, 990
- insert_range_after
 - forward_list, 1008
- inserter, 1232
- int16_t, 597
- int32_t, 597
- int64_t, 597
- int8_t, 597
- int_fast16_t, 597
- int_fast32_t, 597
- int_fast64_t, 597
- int_fast8_t, 597
- INT_FASTN_MAX, 597
- INT_FASTN_MIN, 597
- int_least16_t, 597
- int_least32_t, 597
- int_least64_t, 597
- int_least8_t, 597
- INT_LEASTN_MAX, 597
- INT_LEASTN_MIN, 597
- INT_MAX, 596
- INT_MIN, 596
- int_type
 - char_traits, 1591
- INT_WIDTH, 595
- integer_sequence, 750
 - get, 751
 - value_type, 750
- integral, 642
- integral_constant, 758
 - value_type, 758
- intern_type
 - codecvt, 1650
- internal, 2052
- interrupted
 - errc, 656
- intervals
 - piecewise_constant_distribution, 1803
 - piecewise_linear_distribution, 1804
- intl
 - moneypunct, 1673
- INTMAX_C, 597
- INTMAX_MAX, 597
- INTMAX_MIN, 597
- intmax_t, 597
- INTN_C, 597
- INTN_MAX, 597
- INTN_MIN, 597
- into_variant, 2302
- into_variant_t, 2302
- INTPTR_MAX, 597
- INTPTR_MIN, 597
- intptr_t, 597
- inv_pi, 1837
- inv_pi_v, 1837
- inv_sqrt3, 1837
- inv_sqrt3_v, 1837
- inv_sqrtpi, 1837
- inv_sqrtpi_v, 1837
- invalid_argument, 650, 651, 912, 913
 - constructor, 651
 - errc, 656
- invalid_seek
 - errc, 656
- INVARIANT

text_encoding, 1681
 invocable, 649
 INVOKE, 920, 921
 invoke, 921
 invoke_r, 921
 invoke_result, 774, 816
 invoke_result_t, 755
 io_errc, 2038, 2039
 make_error_code, 2053
 make_error_condition, 2053
 stream, 2039
 io_error
 errc, 656
 io_state
 zombie, 573
 ios, 2034, 2039
 ios_base, 2039
 constructor, 2045
 destructor, 2045
 event, 2040
 failure, 2041
 flags, 1644, 2043
 fmtflags, 2041, 2083
 getloc, 2044
 imbue, 2044
 Init, 2043
 iostate, 2041
 iword, 2045
 openmode, 2041
 precision, 1644, 2043
 pword, 2045
 register_callback, 2045
 seekdir, 2041
 setf, 2043
 sync_with_stdio, 2044
 unsetf, 2043
 width, 1644, 2044
 xalloc, 2044
 ios_base::failure, 2041
 constructor, 2041
 ios_base::Init, 2043
 constructor, 2043
 destructor, 2043
 iostate
 ios_base, 2041
 iostream_category, 2053
 iota, 1578
 simd, 1924
 views, 1287
 iota_result, 1569
 iota_view, 1287
 begin, 1289
 constructor, 1289
 empty, 1289
 end, 1289
 size, 1289
 iota_view::iterator, 1289
 constructor, 1291
 operator*, 1291
 operator+, 1292
 operator++, 1291
 operator+=, 1291
 operator-, 1292
 operator--, 1291
 operator=, 1291
 operator<, 1291
 operator<=, 1292
 operator<=>, 1292
 operator==, 1291
 operator>, 1292
 operator>=, 1292
 operator[], 1291
 iota_view::sentinel, 1292
 constructor, 1293
 operator-, 1293
 operator==, 1293
 is
 ctype, 1646
 ctype<char>, 1649
 is-vector-bool-reference, 1035
 is_a_directory
 errc, 656
 is_absolute
 path, 2147
 is_abstract, 761
 is_abstract_type, 814
 is_abstract_v, 756
 is_accessible, 802
 is_aggregate, 761
 is_aggregate_type, 814
 is_aggregate_v, 756
 is_alias_template, 797
 is_always_equal
 allocator_traits, 691
 scoped_allocator_adaptor, 747
 is_always_exhaustive
 layout_blas_packed::mapping, 1854
 layout_left::mapping, 1153
 layout_left_padded::mapping, 1166
 layout_right::mapping, 1155
 layout_right_padded::mapping, 1171
 layout_stride::mapping, 1161
 layout_transpose::mapping, 1863
 mdspan, 1176
 is_always_lock_free
 atomic, 2221
 atomic<floating-point-type>, 2221
 atomic<integral-type>, 2221
 atomic<shared_ptr<T>>, 2221
 atomic<T*>, 2221
 atomic<weak_ptr<T>>, 2221
 atomic_ref, 2211
 atomic_ref<floating-point-type>, 2211
 atomic_ref<integral-type>, 2211
 atomic_ref<pointer-type>, 2211
 is_always_strided
 layout_blas_packed::mapping, 1854
 layout_left::mapping, 1153

- layout_left_padded::mapping, 1162
- layout_right::mapping, 1155
- layout_right_padded::mapping, 1167
- layout_stride::mapping, 1158
- layout_transpose::mapping, 1863
- mdspan, 1177
- is_always_unique
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1153
 - layout_left_padded::mapping, 1162
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1176
- is_am, 2009
- is_annotation, 794
- is_applicable, 770
- is_applicable_v, 758
- is_arithmetic, 760
- is_arithmetic_type, 814
- is_arithmetic_v, 756
- is_array, 759
- is_array_type, 813
- is_array_v, 755
- is_assignable, 762
- is_assignable_type, 814
- is_assignable_v, 756
- is_assignment, 796
- is_base, 797
- is_base_of, 769
- is_base_of_type, 815
- is_base_of_v, 758
- is_bind_expression, 934
- is_bind_expression_v, 918
- is_bit_field, 794
- is_block_file, 2169
 - directory_entry, 2157
- is_bounded
 - numeric_limits, 593
- is_bounded_array, 761
- is_bounded_array_type, 814
- is_bounded_array_v, 756
- is_character_file, 2170
 - directory_entry, 2157
- is_class, 759
- is_class_member, 797
- is_class_template, 797
- is_class_type, 814
- is_class_v, 755
- is_clock, 1957
- is_clock_v, 1941
- is_complete_type, 795
- is_compound, 760
- is_compound_type, 814
- is_compound_v, 756
- is_concept, 797
- is_const, 761, 794
- is_const_type, 814
- is_const_v, 756
- is_constant_evaluated, 778
- is_constructible, 762, 768
- is_constructible_type, 814
- is_constructible_v, 756
- is_constructor, 796
- is_constructor_template, 797
- is_conversion_function, 796
- is_conversion_function_template, 797
- is_convertible, 769, 771
- is_convertible_type, 815
- is_convertible_v, 758
- is_copy_assignable, 762
- is_copy_assignable_type, 814
- is_copy_assignable_v, 756
- is_copy_assignment, 796
- is_copy_constructible, 762
- is_copy_constructible_type, 814
- is_copy_constructible_v, 756
- is_copy_constructor, 796
- is_corresponding_member, 778
- is_data_member_spec, 812
- is_debugger_present, 671
- is_default_constructible, 762
- is_default_constructible_type, 814
- is_default_constructible_v, 756
- is_default_constructor, 796
- is_defaulted, 794
- is_deleted, 794
- is_destructible, 763
- is_destructible_type, 814
- is_destructible_v, 757
- is_destructor, 796
- is_directory, 2170
 - directory_entry, 2157
- is_empty
 - class, 761
 - function, 2170
- is_empty_type, 814
- is_empty_v, 756
- is_enum, 759
- is_enum_type, 813
- is_enum_v, 755
- is_enumerable_type, 795
- is_enumerator, 794
- is_eq, 619
- is_equal
 - memory_resource, 738
- is_error_code_enum, 655
- is_error_code_enum_v, 657
- is_error_condition_enum, 655
- is_error_condition_enum_v, 657
- is_exact
 - numeric_limits, 592
- is_execution_policy, 1431
- is_execution_policy_v, 2298
- is_exhaustive
 - layout_blas_packed::mapping, 1854
 - layout_left::mapping, 1153

layout_left_padded::mapping, 1166
 layout_right::mapping, 1155
 layout_right_padded::mapping, 1171
 layout_stride::mapping, 1161
 layout_transpose::mapping, 1863
 mdspan, 1177
 is_explicit, 794
 is_explicit_object_parameter, 796
 is_fifo, 2170
 directory_entry, 2157
 is_final, 761, 794
 is_final_type, 814
 is_final_v, 756
 is_floating_point, 759
 is_floating_point_type, 813
 is_floating_point_v, 755
 is_function, 759, 796
 is_function_parameter, 796
 is_function_template, 797
 is_function_type, 814
 is_function_v, 755
 is_fundamental, 760
 is_fundamental_type, 814
 is_fundamental_v, 756
 is_geq, 619
 is_gt, 619
 is_gteq, 619
 is_heap, 1557
 is_heap_until, 1558
 is_iec559
 numeric_limits, 593
 is_implicit_lifetime, 765
 is_implicit_lifetime_type, 815
 is_implicit_lifetime_v, 757
 is_integer
 numeric_limits, 591
 is_integral, 759
 is_integral_type, 813
 is_integral_v, 755
 is_invocable, 770
 is_invocable_r, 770
 is_invocable_r_type, 815
 is_invocable_r_v, 758
 is_invocable_type, 815
 is_invocable_v, 758
 is_layout_compatible, 769
 is_layout_compatible_type, 815
 is_layout_compatible_v, 758
 is_leap
 year, 1983
 is_literal_operator, 796
 is_literal_operator_template, 797
 is_literal_type
 zombie, 573
 is_literal_type_v
 zombie, 573
 is_lock_free
 atomic, 2222
 atomic<floating-point-type>, 2222
 atomic<integral-type>, 2222
 atomic<shared_ptr<T>>, 2222
 atomic<T*>, 2222
 atomic<weak_ptr<T>>, 2222
 atomic_ref, 2211
 atomic_ref<floating-point-type>, 2211
 atomic_ref<integral-type>, 2211
 atomic_ref<pointer-type>, 2211
 is_lt, 619
 is_lteq, 619
 is_lvalue_reference, 759
 is_lvalue_reference_qualified, 794
 is_lvalue_reference_type, 813
 is_lvalue_reference_v, 755
 is_member_function_pointer, 759
 is_member_function_pointer_type, 813
 is_member_function_pointer_v, 755
 is_member_object_pointer, 759
 is_member_object_pointer_type, 813
 is_member_object_pointer_v, 755
 is_member_pointer, 760
 is_member_pointer_type, 814
 is_member_pointer_v, 756
 is_modulo
 numeric_limits, 593
 is_move_assignable, 762
 is_move_assignable_type, 814
 is_move_assignable_v, 756
 is_move_assignment, 796
 is_move_constructible, 762
 is_move_constructible_type, 814
 is_move_constructible_v, 756
 is_move_constructor, 796
 is_mutable_member, 794
 is_namespace, 795
 is_namespace_alias, 796
 is_namespace_member, 797
 is_negative
 hh_mm_ss, 2008
 is_neq, 619
 is_noexcept, 794
 is_nonstatic_data_member, 797
 is_nothrow_applicable, 770
 is_nothrow_applicable_v, 758
 is_nothrow_assignable, 765
 is_nothrow_assignable_type, 814
 is_nothrow_assignable_v, 757
 is_nothrow_constructible, 764
 is_nothrow_constructible_type, 814
 is_nothrow_constructible_v, 757
 is_nothrow_convertible, 769
 is_nothrow_convertible_type, 815
 is_nothrow_convertible_v, 758
 is_nothrow_copy_assignable, 765
 is_nothrow_copy_assignable_type, 814
 is_nothrow_copy_assignable_v, 757
 is_nothrow_copy_constructible, 764
 is_nothrow_copy_constructible_type, 814
 is_nothrow_default_constructible, 764

- is_nothrow_default_constructible_type, 814
- is_nothrow_default_constructible_v, 757
- is_nothrow_destructible, 765
- is_nothrow_destructible_type, 815
- is_nothrow_destructible_v, 757
- is_nothrow_invocable, 770
- is_nothrow_invocable_r, 770
- is_nothrow_invocable_r_type, 815
- is_nothrow_invocable_r_v, 758
- is_nothrow_invocable_type, 815
- is_nothrow_invocable_v, 758
- is_nothrow_move_assignable, 765
- is_nothrow_move_assignable_type, 815
- is_nothrow_move_assignable_v, 757
- is_nothrow_move_constructible, 764
- is_nothrow_move_constructible_type, 814
- is_nothrow_move_constructible_v, 757
- is_nothrow_swappable, 765
- is_nothrow_swappable_type, 815
- is_nothrow_swappable_v, 757
- is_nothrow_swappable_with, 765
- is_nothrow_swappable_with_type, 815
- is_nothrow_swappable_with_v, 757
- is_null_pointer, 759
- is_null_pointer_type, 813
- is_null_pointer_v, 755
- is_object, 760, 797
- is_object_type, 814
- is_object_v, 756
- is_open
 - basic_filebuf, 2115
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ofstream, 2122
- is_operator_function, 796
- is_operator_function_template, 797
- is_other, 2170, 2171
 - directory_entry, 2158
- is_override, 794
- is_partitioned, 1542
- is_permutation, 1503
- is_placeholder, 935
- is_placeholder_v, 918
- is_pm, 2009
- is_pod, 2458
- is_pointer, 759
- is_pointer_interconvertible_base_of, 770
- is_pointer_interconvertible_base_of_type, 815
- is_pointer_interconvertible_base_of_v, 758
- is_pointer_interconvertible_with_class, 778
- is_pointer_type, 813
- is_pointer_v, 755
- is_polymorphic, 761
- is_polymorphic_type, 814
- is_polymorphic_v, 756
- is_private, 793
- is_protected, 793
- is_public, 793
- is_pure_virtual, 794
- is_reference, 760
- is_reference_type, 814
- is_reference_v, 756
- is_reflection, 760
- is_reflection_type, 814
- is_reflection_v, 755
- is_regular_file, 2171
 - directory_entry, 2158
- is_relative
 - path, 2147
- is_rvalue_reference, 759
- is_rvalue_reference_qualified, 794
- is_rvalue_reference_type, 813
- is_rvalue_reference_v, 755
- is_same_type, 815
- is_same_v, 758
- is_scalar, 760
- is_scalar_type, 814
- is_scalar_v, 756
- is_scoped_enum, 761
- is_scoped_enum_type, 814
- is_scoped_enum_v, 756
- is_signed
 - class, 761
 - numeric_limits, 591
- is_signed_type, 814
- is_signed_v, 756
- is_socket, 2171
 - directory_entry, 2158
- is_sorted, 1537
- is_sorted_until, 1537
- is_special_member_function, 796
- is_standard_layout, 761
- is_standard_layout_type, 814
- is_standard_layout_v, 756
- is_static_member, 797
- is_strided
 - layout_blas_packed::mapping, 1854
 - layout_left::mapping, 1153
 - layout_left_padded::mapping, 1162
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1177
- is_string_literal, 786
- is_structural, 761
- is_structural_type, 814
- is_structural_v, 756
- is_structured_binding, 797
- is_sufficiently_aligned, 686
- is_swappable, 763
- is_swappable_type, 814
- is_swappable_v, 757
- is_swappable_with, 763
- is_swappable_with_type, 814
- is_swappable_with_v, 756

- is_symlink, 2171
 - directory_entry, 2158
- is_template, 796
- is_trivial, 2458
- is_trivially_assignable, 764
- is_trivially_assignable_type, 814
- is_trivially_assignable_v, 757
- is_trivially_constructible, 763
- is_trivially_constructible_type, 814
- is_trivially_constructible_v, 757
- is_trivially_copy_assignable, 764
- is_trivially_copy_assignable_type, 814
- is_trivially_copy_assignable_v, 757
- is_trivially_copy_constructible, 764
- is_trivially_copy_constructible_type, 814
- is_trivially_copy_constructible_v, 757
- is_trivially_copyable, 761
- is_trivially_copyable_type, 814
- is_trivially_copyable_v, 756
- is_trivially_default_constructible, 763
- is_trivially_default_constructible_type, 814
- is_trivially_default_constructible_v, 757
- is_trivially_destructible, 764
- is_trivially_destructible_type, 814
- is_trivially_destructible_v, 757
- is_trivially_move_assignable, 764
- is_trivially_move_assignable_type, 814
- is_trivially_move_assignable_v, 757
- is_trivially_move_constructible, 764
- is_trivially_move_constructible_type, 814
- is_trivially_move_constructible_v, 757
- is_type, 795
- is_type_alias, 796
- is_unbounded_array, 761
- is_unbounded_array_type, 814
- is_unbounded_array_v, 756
- is_union, 759
- is_union_type, 813
- is_union_v, 755
- is_unique
 - layout_blas_packed::mapping, 1854
 - layout_left::mapping, 1153
 - layout_left_padded::mapping, 1162
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1177
- is_unsigned, 761
- is_unsigned_type, 814
- is_unsigned_v, 756
- is_user_declared, 794
- is_user_provided, 794
- is_value, 797
- is_vararg_function, 796
- is_variable, 795
- is_variable_template, 797
- is_virtual, 794
- is_virtual_base_of, 769
- is_virtual_base_of_type, 815
- is_virtual_base_of_v, 758
- is_void, 759
- is_void_type, 813
- is_void_v, 755
- is_volatile, 761, 794
- is_volatile_type, 814
- is_volatile_v, 756
- is_within_hard_limits
 - hive, 1018
- is_within_lifetime, 779
- isalnum, 1644, 1746
- isalpha, 1644, 1746
- isblank, 1644, 1746
- iscntrl, 1644, 1746
- isctype
 - regex_traits, 1723
 - regular expression traits, 1745
- isdigit, 1644, 1746
- isfinite, 1828
 - simd, 1925
- isgraph, 1644, 1746
- isgreater, 1828
 - simd, 1925
- isgreaterequal, 1828
 - simd, 1925
- isinf, 1828
 - simd, 1925
- isless, 1828
 - simd, 1925
- islessequal, 1828
 - simd, 1925
- islessgreater, 1828
 - simd, 1925
- islower, 1644, 1746
- isnan, 1828
 - simd, 1925
- isnormal, 1828
 - simd, 1925
- ISO102T617bit
 - text_encoding, 1681
- ISO10367Box
 - text_encoding, 1682
- ISO103T618bit
 - text_encoding, 1681
- ISO10646UTF1
 - text_encoding, 1681
- ISO10Swedish
 - text_encoding, 1681
- ISO111ECMACyrillic
 - text_encoding, 1681
- ISO115481
 - text_encoding, 1682
- ISO11SwedishForNames
 - text_encoding, 1680
- ISO121Canadian1
 - text_encoding, 1681
- ISO122Canadian2

text_encoding, 1681	ISO42JISC62261978
ISO123CSAZ24341985gr	text_encoding, 1681
text_encoding, 1681	ISO47BSViewdata
ISO128T101G2	text_encoding, 1681
text_encoding, 1681	ISO49INIS
ISO139CSN369103	text_encoding, 1681
text_encoding, 1681	ISO4UnitedKingdom
ISO13JISC6220jp	text_encoding, 1680
text_encoding, 1681	ISO50INIS8
ISO141JUSIB1002	text_encoding, 1681
text_encoding, 1681	ISO51INISCyrillic
ISO143IECP271	text_encoding, 1681
text_encoding, 1681	ISO54271981
ISO146Serbian	text_encoding, 1681
text_encoding, 1681	ISO5427Cyrillic
ISO147Macedonian	text_encoding, 1681
text_encoding, 1682	ISO5428Greek
ISO14JISC6220ro	text_encoding, 1681
text_encoding, 1681	ISO57GB1988
ISO150	text_encoding, 1681
text_encoding, 1682	ISO58GB231280
ISO151Cuba	text_encoding, 1681
text_encoding, 1682	ISO60DanishNorwegian
ISO153GOST1976874	text_encoding, 1681
text_encoding, 1682	ISO61Norwegian2
ISO158Lap	text_encoding, 1681
text_encoding, 1682	ISO646basic1983
ISO159JISX02121990	text_encoding, 1681
text_encoding, 1682	ISO646Danish
ISO15Italian	text_encoding, 1682
text_encoding, 1680	ISO6937Add
ISO16Portuguese	text_encoding, 1682
text_encoding, 1681	ISO69French
ISO17Spanish	text_encoding, 1681
text_encoding, 1680	ISO70VideotexSuppl
ISO18Greek70ld	text_encoding, 1681
text_encoding, 1681	ISO84Portuguese2
ISO19LatinGreek	text_encoding, 1681
text_encoding, 1681	ISO85Spanish2
ISO2022CN	text_encoding, 1681
text_encoding, 1682	ISO86Hungarian
ISO2022CNEXT	text_encoding, 1681
text_encoding, 1682	ISO87JISX0208
ISO2022JP	text_encoding, 1681
text_encoding, 1681	ISO885913
ISO2022JP2	text_encoding, 1682
text_encoding, 1681	ISO885914
ISO2022KR	text_encoding, 1682
text_encoding, 1681	ISO885915
ISO2033	text_encoding, 1682
text_encoding, 1681	ISO885916
ISO21German	text_encoding, 1682
text_encoding, 1681	ISO88596E
ISO25French	text_encoding, 1681
text_encoding, 1681	ISO88596I
ISO27LatinGreek1	text_encoding, 1681
text_encoding, 1681	ISO88598E
ISO2IntlRefVersion	text_encoding, 1681
text_encoding, 1681	ISO88598I

text_encoding, 1681
 ISO8859Supp
 text_encoding, 1682
 ISO88Greek7
 text_encoding, 1681
 ISO89ASMO449
 text_encoding, 1681
 ISO90
 text_encoding, 1681
 ISO91JISC62291984a
 text_encoding, 1681
 ISO92JISC62991984b
 text_encoding, 1681
 ISO93JIS62291984badd
 text_encoding, 1681
 ISO94JIS62291984hand
 text_encoding, 1681
 ISO95JIS62291984handadd
 text_encoding, 1681
 ISO96JISC62291984kana
 text_encoding, 1681
 ISO99NAPLPS
 text_encoding, 1681
 iso_encoding
 weekday, 1986
 ISOLatin1
 text_encoding, 1680
 ISOLatin2
 text_encoding, 1680
 ISOLatin3
 text_encoding, 1680
 ISOLatin4
 text_encoding, 1680
 ISOLatin5
 text_encoding, 1680
 ISOLatin6
 text_encoding, 1680
 ISOLatinArabic
 text_encoding, 1680
 ISOLatinCyrillic
 text_encoding, 1680
 ISOLatinGreek
 text_encoding, 1680
 ISOLatinHebrew
 text_encoding, 1680
 ISOTextComm
 text_encoding, 1680
 ispanstream, 2104
 isprint, 1644, 1746
 ispunct, 1644, 1746
 isspace, 1644, 1746
 istream, 2034, 2061
 istream_iterator, 1250
 constructor, 1251
 destructor, 1251
 operator*, 1251
 operator++, 1252
 operator->, 1252
 operator==, 1252
 istream_view, 1262
 istreambuf_iterator, 1253, 2034
 constructor, 1254
 equal, 1254
 operator*, 1254
 operator++, 1254
 operator==, 1255
 proxy, 1254
 istreamstring, 2034, 2088
 istrstream
 zombie, 573
 isunordered, 1828
 simd, 1925
 isupper, 1644, 1746
 iswalnum, 1747
 iswalpha, 1747
 iswblank, 1747
 iswcntrl, 1747
 iswctype, 1747
 iswdigit, 1747
 iswgraph, 1747
 iswlower, 1747
 iswprint, 1747
 iswpunct, 1747
 iswspace, 1747
 iswupper, 1747
 iswxdigit, 1747
 isxdigit, 1644, 1746
 iter_common_reference_t, 1193
 iter_const_reference_t, 1232
 iter_difference_t, 1202
 iter_move, 1206
 adjacent_view::iterator, 1379
 as_input_view::iterator, 1420
 cache_latest_view::iterator, 1416
 cartesian_product_view::iterator, 1414
 chunk_view::inner-iterator, 1390
 common_iterator, 1245
 concat_view::iterator, 1349
 counted_iterator, 1250
 filter_view::iterator, 1307
 izip_view::iterator, 1369
 move_iterator, 1240
 reverse_iterator, 1228
 stride_view::iterator, 1407
 iter_reference_t, 1192
 iter_rvalue_reference_t, 1193
 iter_swap, 1206, 1514
 adjacent_view::iterator, 1379
 as_input_view::iterator, 1420
 cache_latest_view::iterator, 1416
 cartesian_product_view::iterator, 1414
 chunk_view::inner-iterator, 1390
 common_iterator, 1245
 concat_view::iterator, 1349
 counted_iterator, 1250
 filter_view::iterator, 1307
 izip_view::iterator, 1369
 join_view::iterator, 1325

- lazy_split_view::inner-iterator, 1338
- move_iterator, 1240
- reverse_iterator, 1228
- stride_view::iterator, 1407
- iter_type
 - money_get, 1670
 - money_put, 1671
 - num_get, 1654
 - num_put, 1658
 - time_get, 1664
 - time_get_byname, 1668
 - time_put, 1668
 - time_put_byname, 1669
- iter_value_t, 1203
- iterator, 2460
 - basic_format_context, 1702
 - basic_format_parse_context, 1700
 - basic_mask, 1908, 1935
 - basic_string, 1607
 - basic_string_view, 1595
 - basic_vec, 1908, 1910
 - concat_view, 1344
 - containers, 961
 - filter_view, 1305
 - optional, 860
 - path, 2148
 - span, 1144
 - transform_view::iterator, 1311
- iterator_category
 - iterator_traits, 1203, 1205
- iterator_concept
 - iterator_traits, 1205
- iterator_t, 1259
- iterator_traits, 1203
 - difference_type, 1204, 1205
 - iterator_category, 1203, 1205
 - iterator_concept, 1205
 - pointer, 1203, 1205
 - reference, 1203–1205
 - value_type, 1204, 1205
- isword
 - ios_base, 2045
- izip_view::iterator
 - iter_move, 1369
 - iter_swap, 1369
 - operator*, 1367
 - operator+, 1368
 - operator++, 1367
 - operator+=, 1368
 - operator-, 1368
 - operator--, 1367
 - operator-=, 1368
 - operator<=>, 1368
 - operator==, 1368
 - operator[], 1368

J

- JISEncoding
 - text_encoding, 1680
- jmp_buf, 633
- join
 - execution::simple_counting_scope, 2379
 - jthread, 2200
 - thread, 2197
 - views, 1321
- join_view, 1321
 - base, 1321
 - begin, 1321
 - constructor, 1322
 - end, 1321
- join_view::iterator, 1322
 - outer, 1324
 - satisfy, 1324
 - constructor, 1325
 - iter_swap, 1325
 - operator++, 1325
 - operator--, 1325
 - operator->, 1325
 - operator==, 1325
- join_view::sentinel, 1326
 - constructor, 1326
 - operator==, 1326
- join_with
 - views, 1326
- join_with_view, 1326
 - constructor, 1328
- join_with_view::iterator, 1328
 - get-inner, 1330
 - outer, 1330
 - satisfy, 1330
 - update-inner, 1330
 - constructor, 1330, 1331
 - operator++, 1331
 - operator--, 1331
 - operator==, 1332
- join_with_view::sentinel, 1332
 - constructor, 1332
 - operator==, 1332
- joinable
 - jthread, 2200
 - thread, 2197
- jthread, 2197
 - constructor, 2198, 2199
 - destructor, 2199
 - detach, 2200
 - get_id, 2200
 - get_stop_source, 2200
 - get_stop_token, 2200
 - hardware_concurrency, 2201
 - join, 2200
 - joinable, 2200
 - operator=, 2199
 - request_stop, 2200
 - swap, 2200

- just, 2302
- just_error, 2302
- just_error_t, 2302
- just_stopped, 2302
- just_stopped_t, 2302
- just_t, 2301
- K**
- k
 - negative_binomial_distribution, 1789
- key_comp
 - ordered associative containers, 979
- key_compare
 - ordered associative containers, 977
- key_eq
 - unordered associative containers, 989
- key_equal
 - unordered associative containers, 986
- key_type
 - ordered associative containers, 977
 - unordered associative containers, 986
- keys
 - views, 1265
- keys_view, 1265
- kilo, 818
- knuth_b, 1780
- KOI7switched
 - text_encoding, 1684
- KOI8R
 - text_encoding, 1684
- KOI8U
 - text_encoding, 1684
- KSC56011987
 - text_encoding, 1681
- KSC5636
 - text_encoding, 1682
- KZ1048
 - text_encoding, 1682
- L**
- L_tmpnam, 2176
- labs, 580
- laguerre, 1835
 - simd, 1931
- laguerref, 1835
- laguerrel, 1835
- lambda
 - exponential_distribution, 1791
- largest_required_pool_block
 - pool_options, 743
- last
 - range_slice, 1183
 - span, 1142, 1143
- last_spec, 1978
- last_write_time, 2171
 - directory_entry, 2158
- latch
 - arrive_and_wait, 2272
 - constructor, 2272
 - count_down, 2272
 - max, 2272
 - try_wait, 2272
 - wait, 2272
- latest
 - choose, 1941
- launch, 2275
 - async, 2275
 - deferred, 2275
- launder, 606
- layout_blas_packed, 1853
 - mapping, 1853
 - storage_order_type, 1853
 - triangle_type, 1853
- layout_blas_packed::mapping
 - constructor, 1854
 - extents, 1853
 - extents_type, 1853
 - index_type, 1853
 - is_always_exhaustive, 1854
 - is_always_strided, 1854
 - is_always_unique, 1853
 - is_exhaustive, 1854
 - is_strided, 1854
 - is_unique, 1854
 - layout_type, 1853
 - operator(), 1855
 - operator==, 1855
 - rank_type, 1853
 - required_span_size, 1855
 - size_type, 1853
 - stride, 1855
- layout_left, 1152
 - mapping, 1152
- layout_left::mapping
 - constructor, 1153, 1154
 - extents, 1153
 - extents_type, 1152
 - index_type, 1152
 - is_always_exhaustive, 1153
 - is_always_strided, 1153
 - is_always_unique, 1153
 - is_exhaustive, 1153
 - is_strided, 1153
 - is_unique, 1153
 - layout_type, 1152
 - operator(), 1154
 - operator==, 1155
 - rank_type, 1152
 - required_span_size, 1154
 - size_type, 1152
 - stride, 1155
 - submdspan-mapping-impl*, 1187
 - submdspan_mapping, 1153
- layout_left_padded, 1152
 - mapping, 1161
- layout_left_padded::mapping
 - constructor, 1163–1165

- extents, 1162
- extents_type, 1161
- index_type, 1161
- is_always_exhaustive, 1166
- is_always_strided, 1162
- is_always_unique, 1162
- is_exhaustive, 1166
- is_strided, 1162
- is_unique, 1162
- layout_type, 1161
- operator(), 1166
- operator==, 1166
- padding_value, 1161
- rank_type, 1161
- required_span_size, 1166
- size_type, 1161
- stride, 1166
- strides, 1166
- submdspan-mapping-impl*, 1188
- submdspan_mapping, 1162
- layout_right, 1152
 - mapping, 1155
- layout_right::mapping
 - constructor, 1156, 1157
 - extents, 1155
 - extents_type, 1155
 - index_type, 1155
 - is_always_exhaustive, 1155
 - is_always_strided, 1155
 - is_always_unique, 1155
 - is_exhaustive, 1155
 - is_strided, 1155
 - is_unique, 1155
 - layout_type, 1155
 - operator(), 1157
 - operator==, 1157
 - rank_type, 1155
 - required_span_size, 1157
 - size_type, 1155
 - stride, 1157
 - submdspan-mapping-impl*, 1187
 - submdspan_mapping, 1156
- layout_right_padded, 1152
 - mapping, 1166
- layout_right_padded::mapping
 - constructor, 1169–1171
 - extents, 1167
 - extents_type, 1167
 - index_type, 1167
 - is_always_exhaustive, 1171
 - is_always_strided, 1167
 - is_always_unique, 1167
 - is_exhaustive, 1171
 - is_strided, 1167
 - is_unique, 1167
 - layout_type, 1167
 - operator(), 1171
 - operator==, 1172
 - padding_value, 1167
 - rank_type, 1167
 - required_span_size, 1171
 - size_type, 1167
 - stride, 1172
 - strides, 1171
 - submdspan-mapping-impl*, 1189
 - submdspan_mapping, 1168
- layout_stride, 1152
 - mapping, 1158
- layout_stride::mapping
 - constructor, 1159, 1160
 - extents, 1158
 - extents_type, 1158
 - index_type, 1158
 - is_always_exhaustive, 1161
 - is_always_strided, 1158
 - is_always_unique, 1158
 - is_exhaustive, 1161
 - is_strided, 1158
 - is_unique, 1158
 - layout_type, 1158
 - operator(), 1160
 - operator==, 1161
 - rank_type, 1158
 - required_span_size, 1160
 - size_type, 1158
 - stride, 1158
 - strides, 1158
 - submdspan-mapping-impl*, 1188
 - submdspan_mapping, 1159
- layout_transpose, 1862
 - mapping, 1863
 - nested_layout_type, 1862
- layout_transpose::mapping
 - constructor, 1863
 - extents, 1863
 - extents_type, 1863
 - index_type, 1863
 - is_always_exhaustive, 1863
 - is_always_strided, 1863
 - is_always_unique, 1863
 - is_exhaustive, 1863
 - is_strided, 1863
 - is_unique, 1863
 - layout_type, 1863
 - nested_mapping, 1863
 - operator(), 1863
 - operator==, 1864
 - rank_type, 1863
 - required_span_size, 1863
 - size_type, 1863
 - stride, 1864
- layout_type
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1152
 - layout_left_padded::mapping, 1161
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158

layout_transpose::mapping, 1863
 mdspan, 1175
 lazy_split
 views, 1332
 lazy_split_view, 1333
 base, 1333
 begin, 1333
 constructor, 1334
 end, 1333
 lazy_split_view::inner-iterator, 1336
 base, 1337
 constructor, 1337
 iter_swap, 1338
 operator++, 1337
 operator==, 1338
 lazy_split_view::outer-iterator, 1334
 constructor, 1335
 operator*, 1335
 operator++, 1335
 operator==, 1336
 lazy_split_view::outer-iterator::value_-
 type, 1336
 begin, 1336
 constructor, 1336
 end, 1336
 LC_ALL, 1676
 LC_COLLATE, 1676
 LC_CTYPE, 1676
 LC_MONETARY, 1676
 LC_NUMERIC, 1676
 LC_TIME, 1676
 lcm, 1578
 lconv, 1676
 LDBL_DECIMAL_DIG, 596
 LDBL_DIG, 596
 LDBL_EPSILON, 596
 LDBL_HAS_SUBNORM, 2457
 LDBL_MANT_DIG, 596
 LDBL_MAX, 596
 LDBL_MAX_10_EXP, 596
 LDBL_MAX_EXP, 596
 LDBL_MIN, 596
 LDBL_MIN_10_EXP, 596
 LDBL_MIN_EXP, 596
 LDBL_SNAN, 596
 LDBL_TRUE_MIN, 597
 ldexp, 1825
 simd, 1928
 ldexpf, 1825
 ldexpl, 1825
 ldiv, 580
 ldiv_t, 580
 leap_second, 2020
 date, 2021
 hash, 2032
 operator<, 2021
 operator<=, 2022
 operator<=>, 2021, 2022
 operator==, 2021
 operator>, 2022
 operator>=, 2022
 value, 2021
 leap_second_info, 1970
 left, 2052
 legendre, 1835
 simd, 1931
 legendref, 1835
 legendrel, 1835
 length
 basic_string, 1615
 basic_string_view, 1599
 codecvt, 1651
 match_results, 1733
 regex_traits, 1723
 sub_match, 1729
 length_error, 650, 651
 constructor, 651
 lerp, 1831
 simd, 1928
 less, 927, 929
 operator(), 927
 partial_ordering, 620
 strong_ordering, 622
 weak_ordering, 621
 less<>, 927
 operator(), 927
 less_equal, 928, 930
 operator(), 928
 less_equal<>, 928
 operator(), 928
 let_error, 2302
 let_error_t, 2302
 let_stopped, 2302
 let_stopped_t, 2302
 let_value, 2302
 let_value_t, 2302
 lexically_normal
 path, 2147
 lexically_proximate
 path, 2148
 lexically_relative
 path, 2147
 lexicographical_compare, 1563
 lexicographical_compare_three_way, 1564
 lgamma, 1826
 simd, 1928
 lgammaf, 1826
 lgammal, 1826
 linear_congruential_engine, 1770
 constructor, 1771
 result_type, 1770
 list, 1021
 allocator_type, 966
 assign, 970
 assign_range, 970
 begin, 962
 cbegin, 962
 cend, 962

- clear, 970
- const_iterator, 961
- const_reference, 961
- const_reverse_iterator, 964
- constructor, 1023, 1024
- crbegin, 965
- crend, 965
- difference_type, 961
- emplace, 969
- empty, 963
- end, 962
- erase, 970, 1025, 1027
- erase_if, 1027
- get_allocator, 966
- insert, 969, 1024
- insert_range, 969
- iterator, 961
- max_size, 963
- merge, 1026
- operator!=, 962
- operator<=>, 965
- operator=, 961, 967
- operator==, 962
- rbegin, 964
- reference, 961
- remove, 1026
- rend, 965
- resize, 1024
- reverse, 1026
- reverse_iterator, 964
- size, 963
- size_type, 961
- sort, 1027
- splice, 1025
- swap, 962, 967
- unique, 1026
- value_type, 961
- literal
 - text_encoding, 1679
- little
 - endian, 957
- little_endian
 - zombie, 573
- llabs, 580
- lldiv, 580
- lldiv_t, 580
- LLONG_MAX, 596
- LLONG_MIN, 596
- LLONG_WIDTH, 595
- llrint, 1827
 - simd, 1925
- llrintf, 1827
- llrintl, 1827
- llround, 1827
 - simd, 1925
- llroundf, 1827
- llroundl, 1827
- ln10, 1837
- ln10_v, 1837
- ln2, 1837
- ln2_v, 1837
- load
 - atomic, 2222
 - atomic<floating-point-type>, 2222
 - atomic<integral-type>, 2222
 - atomic<shared_ptr<T>>, 2236
 - atomic<T*>, 2222
 - atomic<weak_ptr<T>>, 2238
 - atomic_ref, 2212
 - atomic_ref<floating-point-type>, 2212
 - atomic_ref<integral-type>, 2212
 - atomic_ref<pointer-type>, 2212
- load_factor
 - unordered associative containers, 994
- local-time-format-t, 2026
- local_days, 1941
- local_info, 2014
 - ambiguous, 2014
 - first, 2014
 - nonexistent, 2014
 - operator<<, 2015
 - result, 2014
 - second, 2014
 - unique, 2014
- local_iterator
 - unordered associative containers, 986
- local_seconds, 1941
- local_t, 1941
- local_time, 1941, 1975
 - from_stream, 1975
 - operator<<, 1975
- local_time_format, 2026
- locale, 1724, 1728, 1745
 - basic_format_context, 1702
 - category, 1640
 - classic, 1644
 - combine, 1643
 - constructor, 1642
 - encoding, 1643
 - facet, 1641
 - global, 1643
 - has_facet, 1644
 - id, 1642
 - name, 1643
 - operator(), 1643
 - operator=, 1643
 - operator==, 1643
 - use_facet, 1644
- localeconv, 1676
- localtime, 2032
- localtime_r, 2032
- locate_zone, 2011
 - tzdb, 2010
 - zoned_traits<const time_zone*>, 2016
- lock, 2260
 - rcu_domain, 2290
 - shared_lock, 2259
 - unique_lock, 2256

- weak_ptr, 719
- lock_guard, 2252
 - constructor, 2253
 - destructor, 2253
- log, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- log10, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- log10e, 1837
- log10e_v, 1837
- log10f, 1825
- log10l, 1825
- log1p, 1825
 - simd, 1928
- log1pf, 1825
- log1pl, 1825
- log2, 1825
 - simd, 1928
- log2e, 1837
- log2e_v, 1837
- log2f, 1825
- log2l, 1825
- logb, 1825
 - simd, 1928
- logbf, 1825
- logbl, 1825
- logf, 1825
- logic_error, 650
 - constructor, 650, 651
- logical_and, 930
 - operator(), 930
- logical_and<>, 930
 - operator(), 930
- logical_not, 931
 - operator(), 931
- logical_not<>, 931
 - operator(), 931
- logical_or, 931
 - operator(), 931
- logical_or<>, 931
 - operator(), 931
- logl, 1825
- lognormal_distribution, 1795
 - constructor, 1796
 - m, 1796
 - result_type, 1795
 - s, 1796
- LONG_MAX, 596
- LONG_MIN, 596
- LONG_WIDTH, 595
- longjmp, 633
- lookup
 - flat_map, 1111
 - map, 1048
 - unordered_map, 1070
- lookup_classname
 - regex_traits, 1723
 - regular expression traits, 1745
- lookup_collatename
 - regex_traits, 1723
 - regular expression traits, 1745
- lower
 - ctype_base, 1645
- lower_bound, 1539
 - ordered associative containers, 983
- lower_triangle, 1852
- lower_triangle_t, 1852
- lowest
 - numeric_limits, 591
- lrint, 1827
 - simd, 1925
- lrintf, 1827
- lrintl, 1827
- lround, 1827
 - simd, 1925
- lroundf, 1827
- lroundl, 1827

M

- m
 - fisher_f_distribution, 1799
 - lognormal_distribution, 1796
- Macintosh
 - text_encoding, 1683
- make12, 2009
- make24, 2009
- make_any, 887
- make_const_iterator, 1197
- make_const_sentinel, 1197
- make_error_code
 - errc, 660
 - future_errc, 2276
 - io_errc, 2053
- make_error_condition
 - errc, 662
 - future_errc, 2276
 - io_errc, 2053
- make_exception_ptr, 615
- make_format_args, 1710
- make_from_tuple, 848
- make_hazard_pointer, 2294
- make_hazard_pointer_batch, 2294
- make_heap, 1556
- make_index_sequence, 822
- make_integer_sequence, 750
- make_move_iterator, 1241
- make_obj_using_allocator, 689
- make_optional, 871
- make_pair, 836
- make_preferred
 - path, 2143
- make_ready_at_thread_exit
 - packaged_task, 2287

- make_reverse_iterator, 1229
- make_shared, 712–715
- make_signed, 772, 815
- make_signed_t, 754
- make_tuple, 847
 - tuple, 847
- make_unique, 704, 705
- make_unsigned, 772, 815
- make_unsigned_t, 754
- make_wformat_args, 1710
- malloc, 580, 693, 2454
- map, 1043
 - allocator_type, 966
 - at, 1048
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 978, 1047
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1050
 - extract, 981
 - find, 982
 - get_allocator, 966
 - insert, 979–981, 1048
 - insert_or_assign, 1049, 1050
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lookup, 1048
 - lower_bound, 983
 - mapped_type, 977
 - max_size, 963
 - merge, 981
 - node_type, 977
 - operator!=, 962
 - operator<, 1047
 - operator<=, 965
 - operator=, 961, 967
 - operator==, 962, 1047
 - range_format, 1686
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reverse_iterator, 964
 - size, 963
 - size_type, 961
 - swap, 962, 967
 - try_emplace, 1048, 1049
 - upper_bound, 983
 - value_comp, 979
 - value_compare, 977
 - value_type, 961, 977
- map::value_compare
 - comp, 1043
 - operator(), 1043
- mapped_type
 - ordered associative containers, 977
 - unordered associative containers, 986
- mapping
 - layout_blas_packed, 1853
 - layout_left, 1152
 - layout_left_padded, 1161
 - layout_right, 1155
 - layout_right_padded, 1166
 - layout_stride, 1158
 - layout_transpose, 1863
 - mdspan, 1176
 - submdspan_mapping_result, 1183
- mapping_type
 - mdspan, 1175
- mark_count
 - basic_regex, 1728
- mask
 - ctype_base, 1645
 - ctype_byname, 1647
 - perms, 2154
 - simd, 1904
- mask_array, 1821
 - operator==, 1822
 - operator+=, 1822
 - operator-=, 1822
 - operator/=: 1822
 - operator<<=, 1822
 - operator=, 1822
 - operator>>=, 1822
 - operator[], 1822
 - operator%=: 1822
 - operator&=: 1822
 - operator^=: 1822
 - operator|=, 1822
 - value_type, 1821
- mask_type
 - basic_vec, 1910
- match_any, 1719, 1721
- match_continuous, 1719, 1721, 1741
- match_default, 1719
- match_flag_type, 1719, 1720, 1746
 - regex_constants, 1719
- match_not_bol, 1719, 1720
- match_not_bow, 1719, 1721
- match_not_eol, 1719, 1720
- match_not_eow, 1719, 1721
- match_not_null, 1719, 1721, 1741

- match_prev_avail, 1719, 1721, 1741
- match_results, 1731, 1740, 1742
 - begin, 1734
 - constructor, 1732
 - empty, 1733
 - end, 1734
 - format, 1734, 1735
 - get_allocator, 1735
 - length, 1733
 - matched, 1731
 - max_size, 1733
 - operator=, 1733
 - operator==, 1735
 - operator[], 1734
 - position, 1733
 - prefix, 1734
 - ready, 1733
 - size, 1733
 - str, 1733
 - suffix, 1734
 - swap, 1735
- matched
 - match_results, 1731
- MATH_ERREXCEPT, 1824
- math_errhandling, 1824
- MATH_ERRNO, 1824
- matrix_frob_norm, 1871
- matrix_inf_norm, 1872
- matrix_one_norm, 1871
- matrix_product, 1881
- matrix_rank_1_update, 1877, 1878
- matrix_rank_1_update_c, 1878
- matrix_vector_product, 1873
- max, 1559
 - barrier, 2273
 - counting_semaphore, 2270
 - duration, 1960
 - duration_values, 1957
 - hive_limits, 1011
 - latch, 2272
 - min_max_result, 1488
 - numeric_limits, 591
 - simd, 1924
 - time_point, 1966
 - valarray, 1814
 - year, 1984
- max_align_t, 579, 582
- max_blocks_per_chunk
 - pool_options, 743
- max_bucket_count
 - unordered associative containers, 993
- max_digits10
 - numeric_limits, 591
- max_element, 1561
- max_exponent
 - numeric_limits, 592
- max_exponent10
 - numeric_limits, 592
- max_length
 - codecvt, 1651
- max_load_factor
 - unordered associative containers, 994
- max_name_length
 - text_encoding, 1677
- max_pointer_bits_available, 694
- max_size
 - allocator_traits, 692
 - array, 997
 - basic_stacktrace, 669
 - basic_string, 1616
 - basic_string_view, 1599
 - containers, 963
 - flat_map, 1110
 - inplace_vector, 1039
 - match_results, 1733
 - scoped_allocator_adaptor, 748
- MB_CUR_MAX, 580
- MB_LEN_MAX, 596
- mblen, 580, 1749
- mbrlen, 1747, 1749
- mbrtoc16, 1749
- mbrtoc32, 1749
- mbrtoc8, 1749
- mbrtowc, 1747, 1749
- mbsinit, 1747, 1749
- mbsrtowcs, 1747, 1749
- mbstate_t, 1747, 1749
- mbstowcs, 580, 1749
- mbtowc, 580, 1749
- mdspan, 1175
 - accessor, 1176
 - accessor_type, 1175
 - at, 1180
 - constructor, 1178, 1179
 - data_handle, 1176
 - data_handle_type, 1175
 - element_type, 1175
 - empty, 1181
 - extent, 1176
 - extents, 1176
 - extents_type, 1175
 - index_type, 1175
 - is_always_exhaustive, 1176
 - is_always_strided, 1177
 - is_always_unique, 1176
 - is_exhaustive, 1177
 - is_strided, 1177
 - is_unique, 1177
 - layout_type, 1175
 - mapping, 1176
 - mapping_type, 1175
 - operator[], 1180
 - rank, 1176
 - rank_dynamic, 1176
 - rank_type, 1175
 - reference, 1175
 - size, 1181
 - size_type, 1175

- static_extent, 1176
- stride, 1177
- swap, 1181
- value_type, 1175
- mdy
 - time_base, 1664
- mean
 - normal_distribution, 1795
 - poisson_distribution, 1790
 - student_t_distribution, 1800
- mega, 818
- mem_fn, 936
- mem_fun
 - zombie, 573
- mem_fun1_ref_t
 - zombie, 573
- mem_fun1_t
 - zombie, 573
- mem_fun_ref
 - zombie, 573
- mem_fun_ref_t
 - zombie, 573
- mem_fun_t
 - zombie, 573
- member_offset, 805
 - total_bits, 805
- members_of, 803
- memchr, 1632
- memcmp, 1632
- memcpy, 1632
- memmove, 1632
- memory_order, 2207, 2242, 2464
 - acq_rel, 2207
 - acquire, 2207
 - consume, 2464
 - relaxed, 2207
 - release, 2207
 - seq_cst, 2207
- memory_order_acq_rel, 2207, 2242
- memory_order_acquire, 2207, 2242
- memory_order_consume, 2242
- memory_order_relaxed, 2207, 2242
- memory_order_release, 2207, 2242
- memory_order_seq_cst, 2207, 2242
- memory_resource, 737
 - allocate, 738
 - deallocate, 738
 - destructor, 738
 - do_allocate, 738
 - do_deallocate, 738
 - do_is_equal, 738
 - is_equal, 738
 - operator=, 737
 - operator==, 738
- memset, 1632
- merge, 1545
 - forward_list, 1010
 - list, 1026
 - ordered associative containers, 981
 - unordered associative containers, 991
- merge_result, 1473
- mergeable, 1221
- mersenne_twister_engine, 1771
 - constructor, 1772
 - result_type, 1771
- message
 - do_close, 1676
 - error_category, 658
 - error_code, 660
 - error_condition, 662
- message_size
 - errc, 656
- messages, 1675
 - char_type, 1675
 - close, 1675
 - do_get, 1676
 - do_open, 1676
 - get, 1675
 - id, 1675
 - open, 1675
 - string_type, 1675
- messages_base, 1675
 - catalog, 1675
- messages_byname, 1676
 - catalog, 1676
 - string_type, 1676
- mib
 - text_encoding, 1679
- micro, 818
- microseconds, 1941
- MicrosoftPublishing
 - text_encoding, 1683
- midpoint, 1578, 1579
- milli, 818
- milliseconds, 1941
- min, 1558
 - duration, 1960
 - duration_values, 1957
 - hive_limits, 1011
 - min_max_result, 1488
 - numeric_limits, 591
 - simd, 1924
 - time_point, 1966
 - valarray, 1814
 - year, 1984
- min_element, 1560
- min_exponent
 - numeric_limits, 592
- min_exponent10
 - numeric_limits, 592
- min_max_result, 1488
 - max, 1488
 - min, 1488
- minmax, 1560
 - simd, 1924
- minmax_element, 1562
- minmax_element_result, 1483
- minmax_result, 1482

minstd_rand, 1780
 minstd_rand0, 1779
 minus, 924
 operator(), 924
 minus<>, 925
 operator(), 925
 minutes, 1941
 hh_mm_ss, 2009
 mismatch, 1500
 mismatch_result, 1442
 mktime, 2032
 Mnem
 text_encoding, 1684
 Mnemonic
 text_encoding, 1684
 modf, 1826
 simd, 1932
 modff, 1826
 modfl, 1826
 modulus, 925
 operator(), 925
 modulus<>, 926
 operator(), 926
 money_base, 1672
 none, 1672
 part, 1672
 pattern, 1672
 sign, 1672
 space, 1672
 symbol, 1672
 value, 1672
 money_base::pattern
 field, 1672
 money_get, 1669
 char_type, 1670
 do_get, 1670
 get, 1670
 id, 1670
 iter_type, 1670
 string_type, 1670
 money_put, 1671
 char_type, 1671
 do_put, 1672
 id, 1671
 iter_type, 1671
 put, 1671
 string_type, 1671
 moneypunct, 1672
 char_type, 1672
 curr_symbol, 1673
 decimal_point, 1673
 do_curr_symbol, 1674
 do_decimal_point, 1674
 do_frac_digits, 1674
 do_grouping, 1674
 do_neg_format, 1674
 do_negative_sign, 1674
 do_pos_format, 1674
 do_positive_sign, 1674
 do_thousands_sep, 1674
 frac_digits, 1673
 grouping, 1673
 id, 1673
 intl, 1673
 negative_sign, 1673
 positive_sign, 1673
 string_type, 1672
 thousands_sep, 1673
 moneypunct_byname, 1674
 pattern, 1674
 string_type, 1674
 monostate, 824, 883
 hash, 824, 883
 operator<=>, 824, 883
 operator==, 824, 883
 monotonic_buffer_resource, 744
 constructor, 745
 destructor, 745
 do_allocate, 745
 do_deallocate, 745
 do_is_equal, 745
 release, 745
 upstream_resource, 745
 month, 1980
 constructor, 1981
 from_stream, 1982
 hash, 2031
 month_day, 1989
 month_day_last, 1990
 month_weekday, 1991
 month_weekday_last, 1992
 ok, 1981
 operator unsigned, 1981
 operator+, 1981, 1982
 operator++, 1981
 operator+&, 1981
 operator-, 1982
 operator--, 1981
 operator-=, 1981
 operator<<, 1982
 operator<=>, 1981
 operator==, 1981
 year_month, 1993
 year_month_day, 1995
 year_month_day_last, 1998
 year_month_weekday, 2001
 year_month_weekday_last, 2003
 month_day, 1988
 constructor, 1989
 day, 1989
 from_stream, 1989
 hash, 2031
 month, 1989
 ok, 1989
 operator<<, 1989
 operator<=>, 1989
 operator==, 1989
 month_day_last, 1990

- constructor, 1990
- hash, 2031
- month, 1990
- ok, 1990
- operator<<, 1990
- operator<=>, 1990
- operator==, 1990
- year_month_day_last, 1998
- month_weekday, 1990
 - constructor, 1991
 - hash, 2031
 - month, 1991
 - ok, 1991
 - operator<<, 1991
 - operator==, 1991
 - weekday_indexed, 1991
- month_weekday_last, 1991
 - constructor, 1992
 - hash, 2031
 - month, 1992
 - ok, 1992
 - operator<<, 1992
 - operator==, 1992
 - weekday_last, 1992
- months, 1941
- movable, 648
- move
 - algorithm, 1512
 - basic_ios, 2049
 - function, 826
- move_backward, 1513
- move_backward_result, 1450
- move_constructible, 645
- move_if_noexcept, 826
- move_iterator, 1237
 - base, 1239
 - constructor, 1238
 - iter_move, 1240
 - iter_swap, 1240
 - operator*, 1239
 - operator+, 1239, 1240
 - operator++, 1239
 - operator+=, 1239
 - operator-, 1239, 1240
 - operator--, 1239
 - operator-=, 1239
 - operator->, 2461
 - operator<, 1240
 - operator<=, 1240
 - operator<=>, 1240
 - operator=, 1239
 - operator==, 1239
 - operator>, 1240
 - operator>=, 1240
 - operator[], 1239
- move_only_function, 940
 - constructor, 941, 942
 - destructor, 942
 - operator bool, 942
 - operator(), 942
 - operator=, 942
 - operator==, 943
 - swap, 943
- move_result, 1450
- move_sentinel, 1241
 - base, 1241
 - constructor, 1241
 - operator=, 1241
- mt19937, 1780
- mt19937_64, 1780
- multiline
 - syntax_option_type, 1720
- multimap, 1050
 - allocator_type, 966
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 978, 1053, 1054
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1054
 - extract, 981
 - find, 982
 - get_allocator, 966
 - insert, 979–981, 1054
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lower_bound, 983
 - mapped_type, 977
 - max_size, 963
 - merge, 981
 - node_type, 977
 - operator!=, 962
 - operator<, 1053
 - operator<=>, 965
 - operator=, 961, 967
 - operator==, 962, 1053
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reverse_iterator, 964
 - size, 963
 - size_type, 961

- swap, 962, 967
- upper_bound, 983
- value_comp, 979
- value_compare, 977
- value_type, 961, 977
- multimap::value_compare
 - comp, 1050
 - operator(), 1050
- multiplies, 925
 - operator(), 925
- multiplies<>, 925
 - operator(), 925
- multiset, 1059
 - allocator_type, 966
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 978, 1062
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1063
 - extract, 981
 - find, 982
 - get_allocator, 966
 - insert, 979–981
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lower_bound, 983
 - mapped_type, 977
 - max_size, 963
 - merge, 981
 - node_type, 977
 - operator!=, 962
 - operator<, 1062
 - operator<=, 965
 - operator=, 961, 967
 - operator==, 962, 1062
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reverse_iterator, 964
 - size, 963
 - size_type, 961
 - swap, 962, 967

- upper_bound, 983
- value_comp, 979
- value_compare, 977
- value_type, 961, 977
- mutex, 2246
 - shared_lock, 2260
 - unique_lock, 2257

N

- n
 - chi_squared_distribution, 1797
 - fisher_f_distribution, 1799
- name
 - error_category, 658
 - locale, 1643
 - text_encoding, 1679
 - time_zone, 2015
 - time_zone_link, 2022
 - type_index, 609
 - type_info, 608
- NAN, 596, 1824
- nan, 1827
- nanf, 1827
- nanl, 1827
- nano, 818
- nanoseconds, 1941
- narrow
 - basic_ios, 2049
 - ctype, 1646
 - ctype<char>, 1649
- native
 - endian, 957
 - path, 2144
- native_handle
 - basic_filebuf, 2116
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ofstream, 2122
 - stacktrace_entry, 665
- native_handle_type, 2112
- NATSSEFI
 - text_encoding, 1681
- NATSSEFIADD
 - text_encoding, 1681
- NDEBUG, 561
- nearbyint, 1827
 - simd, 1925
- nearbyintf, 1827
- nearbyintl, 1827
- negate, 926
 - operator(), 926
- negate<>, 926
 - operator(), 926
- negation, 778
- negation_v, 758
- negative_binomial_distribution, 1789
 - constructor, 1789
- k, 1789

- p, 1789
- result_type, 1789
- negative_sign
 - money_punct, 1673
- nested_accessor
 - conjugated_accessor, 1861
 - scaled_accessor, 1859
- nested_exception, 615
 - constructor, 615
 - nested_ptr, 615
 - rethrow_if_nested, 616
 - rethrow_nested, 615
 - throw_with_nested, 616
- nested_layout_type
 - layout_transpose, 1862
- nested_mapping
 - layout_transpose::mapping, 1863
- nested_ptr
 - nested_exception, 615
- network_down
 - errc, 656
- network_reset
 - errc, 656
- network_unreachable
 - errc, 656
- never_stop_token, 2189
 - callback_type, 2190
 - stop_possible, 2190
 - stop_requested, 2190
- new
 - operator, 601–605, 693
- new_delete_resource, 741
- new_handler, 606
- new_object
 - polymorphic_allocator, 740
- next, 1222, 1224
 - subrange, 1281
- next_arg_id
 - basic_format_parse_context, 1701
- next_permutation, 1564
- next_permutation_result, 1485
- nextafter, 1827
 - simd, 1925
- nextafterf, 1827
- nextafterl, 1827
- nextdown, 1828
- nextdownf, 1828
- nextdownl, 1828
- nexttoward, 1827
- nexttowardf, 1827
- nexttowardl, 1827
- nextup, 1827
- nextupf, 1827
- nextupl, 1828
- no_buffer_space
 - errc, 656
- no_child_process
 - errc, 656
- no_link
 - errc, 656
- no_lock_available
 - errc, 656
- no_message
 - errc, 656
- no_message_available
 - errc, 2457
- no_order
 - time_base, 1664
- no_protocol_option
 - errc, 656
- no_space_on_device
 - errc, 656
- no_state
 - future_errc, 2274
- no_stream_resources
 - errc, 2457
- no_such_device
 - errc, 656
- no_such_device_or_address
 - errc, 656
- no_such_file_or_directory
 - errc, 656
- no_such_process
 - errc, 656
- no_timeout
 - cv_status, 2262
- noboolalpha, 2051
- noconv
 - codecvt_base, 1650
- node_type
 - ordered associative containers, 977
 - unordered associative containers, 987
- noemit_on_flush, 2082
- nofollow
 - perm_options, 2154
- none
 - bitset, 916
 - copy_options, 2153
 - directory_options, 2154
 - file_type, 2152
 - money_base, 1672
 - perms, 2154
- none_of, 1490
 - simd, 1939, 1940
- nonexistent
 - local_info, 2014
- nonexistent_local_time, 2012
 - constructor, 2012
- nonstatic_data_members_of, 805
- noop_coroutine, 632
- noop_coroutine_handle, 628
- noop_coroutine_promise, 631
- norm, 1759
 - complex, 1757
 - simd, 1934
- normal_distribution, 1794
 - constructor, 1795
 - mean, 1795

result_type, 1794
 stddev, 1795
 noshowbase, 2051
 noshowpoint, 2051
 noshowpos, 2051
 noskipws, 2051
 nostopstate, 2184
 nostopstate_t, 2184
 nosubs
 syntax_option_type, 1719, 1720
 not1
 zombie, 573
 not2
 zombie, 573
 not_a_directory
 errc, 656
 not_a_socket
 errc, 656
 not_a_stream
 errc, 2457
 not_connected
 errc, 656
 not_enough_memory
 errc, 656
 not_equal_to, 927, 929
 operator(), 927
 not_equal_to<>, 927
 operator(), 927
 not_fn, 933
 not_found
 file_type, 2152
 not_supported
 errc, 656
 nothrow, 600
 nothrow_t, 600
 notify_all
 atomic, 2224
 atomic<floating-point-type>, 2224
 atomic<integral-type>, 2224
 atomic<shared_ptr<T*>, 2237
 atomic<T*>, 2224
 atomic<weak_ptr<T*>, 2239
 atomic_ref<T>, 2213
 condition_variable, 2264
 condition_variable_any, 2267
 notify_all_at_thread_exit, 2262
 notify_one
 atomic, 2224
 atomic<floating-point-type>, 2224
 atomic<integral-type>, 2224
 atomic<shared_ptr<T*>, 2237
 atomic<T*>, 2224
 atomic<weak_ptr<T*>, 2239
 atomic_ref<T>, 2213
 condition_variable, 2264
 condition_variable_any, 2267
 nunitbuf, 2052
 nuppercase, 2052
 now
 file_clock, 1974
 gps_clock, 1973
 tai_clock, 1971
 utc_clock, 1969
 nth_element, 1538
 NULL, 579, 580, 582, 1676, 1749, 2032, 2176
 null_memory_resource, 741
 nullopt, 867
 nullopt_t, 867
 nullptr_t, 579, 581
 num_get, 1654
 char_type, 1654
 do_get, 1655, 1657
 get, 1655
 id, 1654
 iter_type, 1654
 num_put, 1658
 char_type, 1658
 do_put, 1658, 1661
 id, 1658
 iter_type, 1658
 put, 1658
 numeric_limits, 589, 590
 denorm_min, 593
 digits, 591
 digits10, 591
 epsilon, 592
 float_denorm_style, 2457
 has_denorm_loss, 2457
 has_infinity, 592
 has_quiet_NaN, 592
 has_signaling_NaN, 593
 infinity, 593
 is_bounded, 593
 is_exact, 592
 is_iec559, 593
 is_integer, 591
 is_modulo, 593
 is_signed, 591
 lowest, 591
 max, 591
 max_digits10, 591
 max_exponent, 592
 max_exponent10, 592
 min, 591
 min_exponent, 592
 min_exponent10, 592
 quiet_NaN, 593
 radix, 592
 round_error, 592
 round_style, 594
 signaling_NaN, 593
 tinyness_before, 593
 traps, 593
 numeric_limits<bool>, 595
 numpunct, 1661
 char_type, 1661
 decimal_point, 1662
 do_decimal_point, 1662

- do_falsename, 1662
- do_grouping, 1662
- do_thousands_sep, 1662
- do_truename, 1662
- falsename, 1662
- grouping, 1662
- id, 1661
- string_type, 1661
- thousands_sep, 1662
- truename, 1662
- numpunct_byname, 1662
 - char_type, 1663
 - string_type, 1663

O

- object_of, 792
- observable_checkpoint, 828
- oct, 2052
- offset
 - aligned_accessor, 1175
 - conjugated_accessor, 1861
 - default_accessor, 1174
 - extent_slice, 1182
 - scaled_accessor, 1860
 - submdspan_mapping_result, 1183
 - sys_info, 2014
- offset_of, 806
- offset_policy
 - aligned_accessor, 1174
 - conjugated_accessor, 1861
 - default_accessor, 1173
 - scaled_accessor, 1859
- offset_type
 - extent_slice, 1182
- offsetof, 579, 582, 2454
- ofstream, 2034, 2112
- ok
 - codecvt_base, 1650
 - day, 1979
 - month, 1981
 - month_day, 1989
 - month_day_last, 1990
 - month_weekday, 1991
 - month_weekday_last, 1992
 - weekday, 1986
 - weekday_indexed, 1987
 - weekday_last, 1988
 - year, 1983
 - year_month, 1993
 - year_month_day, 1996
 - year_month_day_last, 1998
 - year_month_weekday, 2001
 - year_month_weekday_last, 2003
- on, 2302
- on_t, 2302
- once_flag, 2261
- op
 - zombie, 573
- open
 - basic_filebuf, 2115, 2116
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ofstream, 2122
 - messages, 1675
- open_mode
 - zombie, 573
- openmode
 - ios_base, 2041
- operation_canceled
 - errc, 656
- operation_in_progress
 - errc, 656
- operation_not_permitted
 - errc, 656
- operation_not_supported
 - errc, 656
- operation_state, 2311
- operation_state_tag, 2300
- operation_would_block
 - errc, 656
- operator
 - delete, 602–605, 694
 - new, 601–605, 693
- operator *floating-point-type*
 - atomic<*floating-point-type*>, 2222
 - atomic_ref<*floating-point-type*>, 2212
- operator *integral-type*
 - atomic<*integral-type*>, 2222
 - atomic_ref<*integral-type*>, 2212
- operator *PairLike*
 - subrange, 1281
- operator *pointer-type*
 - atomic_ref<*pointer-type*>, 2212
- operator *type*
 - atomic, 2222
 - atomic_ref, 2212
- operator basic_string
 - sub_match, 1729
- operator basic_string_view
 - basic_string, 1624
- operator basic_vec
 - basic_mask, 1938
- operator bool
 - basic_format_arg, 1709
 - basic_ios, 2050
 - basic_istream::sentry, 2066
 - basic_ostream::sentry, 2077
 - bitset::reference, 912
 - copyable_function, 946
 - coroutine_handle, 630
 - coroutine_handle<noop_coroutine_ -
promise>, 632
 - error_code, 660
 - error_condition, 662
 - expected, 900
 - expected<void>, 907
 - function, 939

- move_only_function, 942
- optional, 861
- shared_lock, 2260
- shared_ptr, 712
- stacktrace_entry, 665
- unique_lock, 2257
- unique_ptr, 702
- vector<bool>::reference, 1035
- operator const filesystem::path&
 - directory_entry, 2157
- operator coroutine_handle<>
 - coroutine_handle, 630
 - coroutine_handle<noop_coroutine_-
promise>, 631
- operator default_accessor
 - aligned_accessor, 1175
- operator delete
 - generator::promise_type, 1424
 - task::promise_type, 2375
- operator int
 - year, 1983
- operator local_days
 - year_month_day, 1996
 - year_month_day_last, 1998
 - year_month_weekday, 2001
 - year_month_weekday_last, 2003
- operator local_time
 - zoned_time, 2020
- operator new
 - generator::promise_type, 1424
 - task::promise_type, 2374
- operator partial_ordering
 - strong_ordering, 623
 - weak_ordering, 622
- operator precision
 - hh_mm_ss, 2009
- operator shared_ptr<T>
 - atomic<shared_ptr<T>>, 2236
- operator string_type
 - path, 2144
- operator sys_days
 - year_month_day, 1996
 - year_month_day_last, 1998
 - year_month_weekday, 2001
 - year_month_weekday_last, 2003
- operator sys_time
 - zoned_time, 2020
- operator T*
 - atomic<T*>, 2222
- operator T&
 - reference_wrapper, 923
- operator unsigned
 - day, 1979
 - month, 1981
- operator value_type
 - type_order, 627
- operator weak_ordering
 - strong_ordering, 623
- operator weak_ptr<T>
 - atomic<weak_ptr<T>>, 2238
- operator!
 - basic_ios, 2050
 - basic_mask, 1937
 - basic_vec, 1914
 - valarray, 1813
- operator!=, 2459
 - basic_mask, 1938
 - basic_vec, 1915
 - containers, 962
 - coroutine_handle, 631
 - optional, 868, 869
 - queue, 1093
 - reverse_iterator, 1227
 - stack, 1101
 - valarray, 1816
 - variant, 881
- operator"d
 - day, 1980
- operator"h
 - duration, 1963
- operator"i
 - complex, 1759
- operator"if
 - complex, 1759
- operator"il
 - complex, 1759
- operator"min
 - duration, 1963
- operator"ms
 - duration, 1963
- operator"ns
 - duration, 1963
- operator"s
 - duration, 1963
 - string, 1632
 - u16string, 1632
 - u32string, 1632
 - u8string, 1632
 - wstring, 1632
- operator"sv
 - string_view, 1603
 - u16string_view, 1603
 - u32string_view, 1603
 - u8string_view, 1603
 - wstring_view, 1603
- operator"us
 - duration, 1963
- operator"y
 - year, 1984
- operator()
 - bit_and, 931
 - bit_and<>, 932
 - bit_not, 932
 - bit_not<>, 932
 - bit_or, 932
 - bit_or<>, 932
 - bit_xor, 932
 - bit_xor<>, 932

boyer_moore_horspool_searcher, 952
 boyer_moore_searcher, 951
 clock_time_conversion, 1976–1978
 constant_wrapper, 831
 copyable_function, 946
 coroutine_handle, 630
 coroutine_handle<noop_coroutine_
 promise>, 632
 default_delete, 698
 default_searcher, 950
 divides, 925
 divides<>, 925
 equal_to, 926
 equal_to<>, 926
 function, 939
 function_ref, 949
 greater, 927
 greater<>, 927
 greater_equal, 928
 greater_equal<>, 928
 layout_blas_packed::mapping, 1855
 layout_left::mapping, 1154
 layout_left_padded::mapping, 1166
 layout_right::mapping, 1157
 layout_right_padded::mapping, 1171
 layout_stride::mapping, 1160
 layout_transpose::mapping, 1863
 less, 927
 less<>, 927
 less_equal, 928
 less_equal<>, 928
 locale, 1643
 logical_and, 930
 logical_and<>, 930
 logical_not, 931
 logical_not<>, 931
 logical_or, 931
 logical_or<>, 931
 map::value_compare, 1043
 minus, 924
 minus<>, 925
 modulus, 925
 modulus<>, 926
 move_only_function, 942
 multimap::value_compare, 1050
 multiplies, 925
 multiplies<>, 925
 negate, 926
 negate<>, 926
 not_equal_to, 927
 not_equal_to<>, 927
 owner_equal, 720
 owner_hash, 720
 owner_less, 720
 packaged_task, 2287
 plus, 924
 plus<>, 924
 random_device, 1781
 reference_wrapper, 923
 type_order, 627
 operator*
 adjacent_transform_view::iterator,
 1383
 adjacent_view::iterator, 1378
 back_insert_iterator, 1230
 basic_const_iterator, 1235
 basic_istream_view::iterator, 1298
 basic_vec, 1914
 cache_latest_view::iterator, 1416
 cartesian_product_view::iterator, 1412
 chunk_by_view::iterator, 1402
 chunk_view::inner-iterator, 1389
 chunk_view::iterator, 1393
 chunk_view::outer-iterator, 1387
 common_iterator, 1244
 complex, 1756
 concat_view::iterator, 1347
 counted_iterator, 1248
 duration, 1960
 expected, 900
 expected<void>, 907
 filter_view::iterator, 1306
 front_insert_iterator, 1231
 generator::iterator, 1425
 indirect, 730
 insert_iterator, 1232
 iota_view::iterator, 1291
 istream_iterator, 1251
 istreambuf_iterator, 1254
 izip_view::iterator, 1367
 lazy_split_view::outer-iterator, 1335
 move_iterator, 1239
 optional, 861
 ostream_iterator, 1253
 ostreambuf_iterator, 1255
 polymorphic, 736
 regex_iterator, 1741
 regex_token_iterator, 1744
 repeat_view::iterator, 1295
 reverse_iterator, 1226
 shared_ptr, 711
 slide_view::iterator, 1398
 split_view::iterator, 1340
 unique_ptr, 701
 valarray, 1815
 zip_transform_view::iterator, 1373
 operator*=
 basic_vec, 1915
 complex, 1755, 1756
 duration, 1960
 gslice_array, 1821
 indirect_array, 1823
 mask_array, 1822
 slice_array, 1819
 valarray, 1813
 operator+
 adjacent_transform_view::iterator,
 1384

adjacent_view::iterator, 1379
 basic_const_iterator, 1236
 basic_mask, 1937
 basic_string, 1626–1628
 basic_vec, 1914
 cartesian_product_view::iterator, 1413
 chunk_view::iterator, 1394
 complex, 1756
 concat_view::iterator, 1348
 counted_iterator, 1249
 day, 1980
 duration, 1959, 1960, 1966
 elements_view::iterator, 1358
 enumerate_view::iterator, 1363
 iota_view::iterator, 1292
 izip_view::iterator, 1368
 month, 1981, 1982
 move_iterator, 1239, 1240
 repeat_view::iterator, 1296
 reverse_iterator, 1227, 1228
 slide_view::iterator, 1399
 stride_view::iterator, 1407
 time_point, 1966
 transform_view::iterator, 1312
 valarray, 1813, 1815
 weekday, 1986
 year, 1983, 1984
 year_month, 1993, 1994
 year_month_day, 1996, 1997
 year_month_day_last, 1999
 year_month_weekday, 2001
 year_month_weekday_last, 2003, 2004
 zip_transform_view::iterator, 1374
 operator++
 adjacent_transform_view::iterator, 1383
 adjacent_view::iterator, 1378
 as_input_view::iterator, 1419
 atomic<integral-type>, 2234
 atomic<T*>, 2234
 atomic_ref<integral-type>, 2220
 atomic_ref<pointer-type>, 2220
 back_insert_iterator, 1230
 basic_const_iterator, 1235
 basic_istream_view::iterator, 1297
 basic_vec, 1914
 cache_latest_view::iterator, 1416
 cartesian_product_view::iterator, 1412
 chunk_by_view::iterator, 1402
 chunk_view::inner-iterator, 1389
 chunk_view::iterator, 1393
 chunk_view::outer-iterator, 1387, 1388
 common_iterator, 1244
 concat_view::iterator, 1347
 counted_iterator, 1248
 day, 1979
 directory_iterator, 2160
 duration, 1959
 elements_view::iterator, 1358
 enumerate_view::iterator, 1362, 1363
 filter_view::iterator, 1307
 front_insert_iterator, 1231
 generator::iterator, 1425
 insert_iterator, 1232
 iota_view::iterator, 1291
 istream_iterator, 1252
 istreambuf_iterator, 1254
 izip_view::iterator, 1367
 join_view::iterator, 1325
 join_with_view::iterator, 1331
 lazy_split_view::inner-iterator, 1337
 lazy_split_view::outer-iterator, 1335
 month, 1981
 move_iterator, 1239
 ostream_iterator, 1253
 ostreambuf_iterator, 1256
 recursive_directory_iterator, 2163
 regex_iterator, 1741
 regex_token_iterator, 1744
 repeat_view::iterator, 1295
 reverse_iterator, 1227
 slide_view::iterator, 1398
 split_view::iterator, 1340
 stride_view::iterator, 1406
 time_point, 1965
 transform_view::iterator, 1311
 weekday, 1985
 year, 1983
 zip_transform_view::iterator, 1373
 operator+=
 adjacent_transform_view::iterator, 1384
 adjacent_view::iterator, 1378
 atomic<floating-point-type>, 2231
 atomic<integral-type>, 2228
 atomic<T*>, 2228, 2231, 2233
 atomic_ref<floating-point-type>, 2218
 atomic_ref<integral-type>, 2215
 atomic_ref<pointer-type>, 2220
 basic_const_iterator, 1235
 basic_string, 1617, 1618
 basic_vec, 1915
 cartesian_product_view::iterator, 1413
 chunk_view::iterator, 1393
 complex, 1755
 concat_view::iterator, 1347
 counted_iterator, 1249
 day, 1979
 duration, 1959
 elements_view::iterator, 1358
 enumerate_view::iterator, 1363
 gslice_array, 1821
 indirect_array, 1823
 iota_view::iterator, 1291
 izip_view::iterator, 1368
 mask_array, 1822
 month, 1981
 move_iterator, 1239

path, 2143
 repeat_view::iterator, 1295
 reverse_iterator, 1227
 slice_array, 1819
 slide_view::iterator, 1398
 stride_view::iterator, 1406
 time_point, 1965
 transform_view::iterator, 1311
 valarray, 1813
 weekday, 1986
 year, 1983
 year_month, 1993
 year_month_day, 1995
 year_month_day_last, 1998
 year_month_weekday, 2000
 year_month_weekday_last, 2002, 2003
 zip_transform_view::iterator, 1373

operator-

- adjacent_transform_view::iterator, 1384
- adjacent_transform_view::sentinel, 1385
- adjacent_view::iterator, 1379
- adjacent_view::sentinel, 1380
- as_input_view::iterator, 1419
- basic_const_iterator, 1237
- basic_mask, 1937
- basic_vec, 1914
- cache_latest_view::iterator, 1417
- cache_latest_view::sentinel, 1417
- cartesian_product_view::iterator, 1413, 1414
- chunk_view::inner-iterator, 1390
- chunk_view::iterator, 1394
- chunk_view::outer-iterator, 1388
- common_iterator, 1245
- complex, 1756
- concat_view::iterator, 1348, 1349
- counted_iterator, 1249
- day, 1980
- duration, 1959, 1960, 1966
- elements_view::iterator, 1359
- elements_view::sentinel, 1360
- enumerate_view::iterator, 1363
- enumerate_view::sentinel, 1364
- iota_view::iterator, 1292
- iota_view::sentinel, 1293
- izip_view::iterator, 1368
- month, 1982
- move_iterator, 1239, 1240
- repeat_view::iterator, 1296
- reverse_iterator, 1227, 1228
- slide_view::iterator, 1399
- slide_view::sentinel, 1400
- stride_view::iterator, 1407
- time_point, 1966
- transform_view::iterator, 1312
- transform_view::sentinel, 1313
- valarray, 1813, 1815
- weekday, 1986
- year, 1983, 1984
- year_month, 1993, 1994
- year_month_day, 1996, 1997
- year_month_day_last, 1999
- year_month_weekday, 2001
- year_month_weekday_last, 2003, 2004
- zip_transform_view::iterator, 1374
- zip_transform_view::sentinel, 1375
- zip_view::sentinel, 1370

operator--

- adjacent_transform_view::iterator, 1383
- adjacent_view::iterator, 1378
- atomic<integral-type>, 2234
- atomic<T*>, 2234
- atomic_ref<integral-type>, 2220
- atomic_ref<pointer-type>, 2220
- basic_const_iterator, 1235
- basic_vec, 1914
- cartesian_product_view::iterator, 1412
- chunk_by_view::iterator, 1402
- chunk_view::iterator, 1393
- concat_view::iterator, 1347
- counted_iterator, 1248
- day, 1979
- duration, 1959
- elements_view::iterator, 1358
- enumerate_view::iterator, 1363
- filter_view::iterator, 1307
- iota_view::iterator, 1291
- izip_view::iterator, 1367
- join_view::iterator, 1325
- join_with_view::iterator, 1331
- month, 1981
- move_iterator, 1239
- repeat_view::iterator, 1295
- reverse_iterator, 1227
- slide_view::iterator, 1398
- stride_view::iterator, 1406
- time_point, 1965
- transform_view::iterator, 1311
- weekday, 1985
- year, 1983
- zip_transform_view::iterator, 1373

operator==

- adjacent_transform_view::iterator, 1384
- adjacent_view::iterator, 1378
- atomic<floating-point-type>, 2231
- atomic<integral-type>, 2228
- atomic<T*>, 2228, 2231, 2233
- atomic_ref<floating-point-type>, 2218
- atomic_ref<integral-type>, 2215
- atomic_ref<pointer-type>, 2220
- basic_const_iterator, 1235
- basic_vec, 1915
- cartesian_product_view::iterator, 1413
- chunk_view::iterator, 1393

- complex, 1755
- concat_view::iterator, 1347
- counted_iterator, 1249
- day, 1979
- duration, 1960
- elements_view::iterator, 1358
- enumerate_view::iterator, 1363
- gslice_array, 1821
- indirect_array, 1823
- iota_view::iterator, 1291
- izip_view::iterator, 1368
- mask_array, 1822
- month, 1981
- move_iterator, 1239
- repeat_view::iterator, 1295
- reverse_iterator, 1227
- slice_array, 1819
- slide_view::iterator, 1398
- stride_view::iterator, 1406
- time_point, 1965
- transform_view::iterator, 1311
- valarray, 1813
- weekday, 1986
- year, 1983
- year_month, 1993
- year_month_day, 1995
- year_month_day_last, 1998
- year_month_weekday, 2000
- year_month_weekday_last, 2002, 2003
- operator->
 - basic_const_iterator, 1235
 - common_iterator, 1244
 - counted_iterator, 1248
 - expected, 899
 - filter_view::iterator, 1306
 - indirect, 730
 - istream_iterator, 1252
 - join_view::iterator, 1325
 - move_iterator, 2461
 - optional, 860
 - polymorphic, 736
 - regex_iterator, 1741
 - regex_token_iterator, 1744
 - reverse_iterator, 1226
 - shared_ptr, 711
 - unique_ptr, 702
- operator/
 - basic_vec, 1914
 - calendar types, 2004–2007
 - complex, 1756
 - duration, 1961
 - path, 2149
 - valarray, 1815
- operator/=
 - basic_vec, 1915
 - complex, 1755, 1756
 - duration, 1960
 - gslice_array, 1821
 - indirect_array, 1823
 - mask_array, 1822
 - path, 2142, 2143
 - slice_array, 1819
 - valarray, 1813
- operator<
 - adjacent_view::iterator, 1378
 - basic_const_iterator, 1236
 - basic_mask, 1938
 - basic_vec, 1915
 - chunk_view::iterator, 1393
 - concat_view::iterator, 1348
 - duration, 1961
 - elements_view::iterator, 1358
 - iota_view::iterator, 1291
 - leap_second, 2021
 - map, 1047
 - move_iterator, 1240
 - multimap, 1053
 - multiset, 1062
 - optional, 868, 869
 - partial_ordering, 621
 - queue, 1093
 - reverse_iterator, 1227
 - set, 1058
 - slide_view::iterator, 1398
 - stack, 1101
 - stride_view::iterator, 1406
 - strong_ordering, 623
 - sys_time, 2021
 - time_point, 1966
 - transform_view::iterator, 1312
 - type_index, 609
 - unique_ptr, 705, 706
 - valarray, 1816
 - variant, 881
 - vector, 1030
 - weak_ordering, 622
- operator<<
 - basic_ostream, 2078–2080, 2082
 - basic_stacktrace, 670
 - basic_string, 1629
 - basic_string_view, 1603
 - basic_vec, 1914, 1915
 - bitset, 914, 916
 - byte, 582
 - complex, 1756
 - day, 1980
 - directory_entry, 2159
 - duration, 1963
 - error_code, 660
 - file_time, 1974
 - gps_time, 1973
 - local_info, 2015
 - local_time, 1975
 - month, 1982
 - month_day, 1989
 - month_day_last, 1990
 - month_weekday, 1991
 - month_weekday_last, 1992

path, 2149
 shared_ptr, 717
 stacktrace_entry, 669
 sub_match, 1731
 sys_days, 1968
 sys_info, 2014
 sys_time, 1968
 tai_time, 1972
 thread::id, 2195
 unique_ptr, 706
 utc_time, 1970
 valarray, 1815
 weekday, 1986
 weekday_indexed, 1988
 weekday_last, 1988
 year, 1984
 year_month, 1994
 year_month_day, 1997
 year_month_day_last, 1999
 year_month_weekday, 2002
 year_month_weekday_last, 2004
 zoned_time, 2020
 operator<<=
 basic_vec, 1915
 bitset, 914
 byte, 582
 gslice_array, 1821
 indirect_array, 1823
 mask_array, 1822
 slice_array, 1819
 valarray, 1813
 operator<=, 2459
 adjacent_view::iterator, 1379
 basic_const_iterator, 1236
 basic_mask, 1938
 basic_vec, 1915
 chunk_view::iterator, 1394
 duration, 1961
 elements_view::iterator, 1358
 iota_view::iterator, 1292
 leap_second, 2022
 move_iterator, 1240
 optional, 868, 870
 partial_ordering, 621
 queue, 1093
 reverse_iterator, 1228
 slide_view::iterator, 1399
 stack, 1101
 stride_view::iterator, 1407
 strong_ordering, 623
 sys_time, 2022
 time_point, 1966
 transform_view::iterator, 1312
 type_index, 609
 unique_ptr, 705, 706
 valarray, 1816
 variant, 881
 weak_ordering, 622
 operator<=>
 adjacent_view::iterator, 1379
 basic_const_iterator, 1236
 basic_stacktrace, 669
 basic_string_view, 1603
 cartesian_product_view::iterator, 1413
 chunk_view::iterator, 1394
 coroutine_handle, 631
 counted_iterator, 1250
 day, 1980
 directory_entry, 2159
 duration, 1961
 elements_view::iterator, 1358
 enumerate_view::iterator, 1363
 error_category, 658
 error_code, 663
 error_condition, 663
 iota_view::iterator, 1292
 izip_view::iterator, 1368
 leap_second, 2021, 2022
 monostate, 824, 883
 month, 1981
 month_day, 1989
 month_day_last, 1990
 move_iterator, 1240
 optional, 868, 869, 871
 optional container requirements, 965
 pair, 836
 partial_ordering, 621
 path, 2149
 pointer_tag_pair, 697
 queue, 1093
 repeat_view::iterator, 1296
 reverse_iterator, 1228
 shared_ptr, 715
 slide_view::iterator, 1399
 stack, 1101
 stride_view::iterator, 1407
 strong_ordering, 623
 sub_match, 1730
 sys_time, 2022
 thread::id, 2195
 time_zone, 2016
 time_zone_link, 2023
 transform_view::iterator, 1312
 tuple, 850
 type_index, 609
 unique_ptr, 705, 706
 variant, 881
 weak_ordering, 622
 year, 1984
 year_month, 1993
 year_month_day, 1996
 year_month_day_last, 1999
 operator=
 allocator, 692
 allocator-aware containers, 967
 any, 886
 atomic, 2222
 atomic<floating-point-type>, 2222

atomic<integral-type>, 2222
 atomic<shared_ptr<T>>, 2236
 atomic<T*>, 2222
 atomic<weak_ptr<T>>, 2238
 atomic_ref, 2212
 atomic_ref<floating-point-type>, 2212
 atomic_ref<integral-type>, 2212
 atomic_ref<pointer-type>, 2212
 back_insert_iterator, 1230
 basic_filebuf, 2114
 basic_iostream, 2074
 basic_istream, 2065
 basic_ostream, 2076
 basic_regex, 1727
 basic_spanbuf, 2106
 basic_stacktrace, 668
 basic_streambuf, 2057
 basic_string, 1615
 basic_stringbuf, 2092
 basic_syncbuf, 2126
 bitset::reference, 912
 common_iterator, 1243
 complex, 1755
 containers, 961
 copyable_function, 945
 coroutine_handle, 630
 counted_iterator, 1248
 directory_iterator, 2160
 enable_shared_from_this, 721
 error_code, 660
 error_condition, 662
 exception, 613
 expected, 896–898
 expected<void>, 906
 front_insert_iterator, 1230, 1231
 function, 939
 function_ref, 948
 future, 2280
 generator, 1421
 generator::iterator, 1425
 gslice_array, 1821
 hazard_pointer, 2293
 hive, 1016
 indirect, 729, 730
 indirect_array, 1823, 1824
 insert_iterator, 1231
 jthread, 2199
 locale, 1643
 mask_array, 1822
 match_results, 1733
 memory_resource, 737
 move_iterator, 1239
 move_only_function, 942
 move_sentinel, 1241
 optional, 857, 858
 ostream_iterator, 1253
 ostreambuf_iterator, 1255
 packaged_task, 2287
 pair, 834, 835
 path, 2141, 2142
 polymorphic, 735, 736
 promise, 2278
 recursive_directory_iterator, 2162
 reference_wrapper, 922
 reverse_iterator, 1226
 shared_future, 2283
 shared_lock, 2259
 shared_ptr, 710
 slice_array, 1818, 1819
 span, 1142
 thread, 2196
 tuple, 845–847
 unique_lock, 2255
 unique_ptr, 701, 704
 valarray, 1810
 variant, 876, 877
 vector<bool>::reference, 1035
 weak_ptr, 718
 zoned_time, 2019, 2020
 operator==
 adjacent_transform_view::iterator, 1384
 adjacent_transform_view::sentinel, 1385
 adjacent_view::iterator, 1378
 adjacent_view::sentinel, 1380
 allocator, 693
 as_input_view::iterator, 1419
 basic_const_iterator, 1235
 basic_istream_view::iterator, 1298
 basic_mask, 1938
 basic_stacktrace, 669
 basic_string_view, 1603
 basic_vec, 1915
 bitset, 915
 cache_latest_view::iterator, 1417
 cache_latest_view::sentinel, 1417
 cartesian_product_view::iterator, 1413
 chunk_by_view::iterator, 1402
 chunk_view::inner-iterator, 1390
 chunk_view::iterator, 1393
 chunk_view::outer-iterator, 1388
 common_iterator, 1245
 complex, 1756
 concat_view::iterator, 1347, 1348
 containers, 962
 copyable_function, 946
 coroutine_handle, 631
 counted_iterator, 1249
 day, 1980
 directory_entry, 2158
 duration, 1961
 elements_view::iterator, 1358
 elements_view::sentinel, 1359
 enumerate_view::iterator, 1363
 enumerate_view::sentinel, 1364
 error_category, 658
 error_code, 663

error_condition, 663
 expected, 902, 903
 expected<void>, 909, 910
 extents, 1149
 filter_view::iterator, 1307
 filter_view::sentinel, 1308
 function, 940
 generator::iterator, 1425
 indirect, 731
 iota_view::iterator, 1291
 iota_view::sentinel, 1293
 istream_iterator, 1252
 istreambuf_iterator, 1255
 izip_view::iterator, 1368
 join_view::iterator, 1325
 join_view::sentinel, 1326
 join_with_view::iterator, 1332
 join_with_view::sentinel, 1332
 layout_blas_packed::mapping, 1855
 layout_left::mapping, 1155
 layout_left_padded::mapping, 1166
 layout_right::mapping, 1157
 layout_right_padded::mapping, 1172
 layout_stride::mapping, 1161
 layout_transpose::mapping, 1864
 lazy_split_view::inner-iterator, 1338
 lazy_split_view::outer-iterator, 1336
 leap_second, 2021
 locale, 1643
 map, 1047
 match_results, 1735
 memory_resource, 738
 monostate, 824, 883
 month, 1981
 month_day, 1989
 month_day_last, 1990
 month_weekday, 1991
 month_weekday_last, 1992
 move_iterator, 1239
 move_only_function, 943
 multimap, 1053
 multiset, 1062
 optional, 868, 869
 pair, 836
 partial_ordering, 621
 path, 2149
 pointer_tag_pair, 697
 polymorphic_allocator, 741
 queue, 1093
 regex_iterator, 1741
 regex_token_iterator, 1742, 1744
 repeat_view::iterator, 1296
 reverse_iterator, 1227
 scoped_allocator_adaptor, 749
 set, 1058
 shared_ptr, 715
 slide_view::iterator, 1398
 slide_view::sentinel, 1400
 split_view::iterator, 1340
 split_view::sentinel, 1341
 stack, 1101
 stacktrace_entry, 666
 stride_view::iterator, 1406
 strong_ordering, 623
 sub_match, 1730
 sys_time, 2021
 take_view::sentinel, 1316
 take_while_view::sentinel, 1318
 task_scheduler, 2368
 text_encoding, 1680
 thread::id, 2195
 time_point, 1966
 time_zone, 2016
 time_zone_link, 2022
 transform_view::iterator, 1312
 transform_view::sentinel, 1313
 tuple, 850
 type_index, 609
 type_info, 607
 unexpected, 890
 unique_ptr, 705
 unreachable_sentinel_t, 1250
 valarray, 1816
 variant, 881
 vector, 1030
 weak_ordering, 622
 weekday, 1986
 weekday_indexed, 1987
 weekday_last, 1988
 year, 1984
 year_month, 1993
 year_month_day, 1996
 year_month_day_last, 1998
 year_month_weekday, 2001
 year_month_weekday_last, 2003
 zip_transform_view::iterator, 1373
 zip_transform_view::sentinel, 1375
 zip_view::sentinel, 1370
 zoned_time, 2020
 operator>, 2459
 adjacent_view::iterator, 1379
 basic_const_iterator, 1236
 basic_mask, 1938
 basic_vec, 1915
 chunk_view::iterator, 1393
 duration, 1961
 elements_view::iterator, 1358
 iota_view::iterator, 1292
 leap_second, 2022
 move_iterator, 1240
 optional, 868, 870
 partial_ordering, 621
 queue, 1093
 reverse_iterator, 1228
 slide_view::iterator, 1398
 stack, 1101
 stride_view::iterator, 1406
 strong_ordering, 623

sys_time, 2022
 time_point, 1966
 transform_view::iterator, 1312
 type_index, 609
 unique_ptr, 705, 706
 valarray, 1816
 variant, 881
 weak_ordering, 622
 operator>=, 2459
 adjacent_view::iterator, 1379
 basic_const_iterator, 1236
 basic_mask, 1938
 basic_vec, 1915
 chunk_view::iterator, 1394
 duration, 1961
 elements_view::iterator, 1358
 iota_view::iterator, 1292
 leap_second, 2022
 move_iterator, 1240
 optional, 868, 870
 partial_ordering, 621
 queue, 1093
 reverse_iterator, 1228
 slide_view::iterator, 1399
 stack, 1101
 stride_view::iterator, 1407
 strong_ordering, 623
 sys_time, 2022
 time_point, 1966, 1967
 transform_view::iterator, 1312
 type_index, 609
 unique_ptr, 705, 706
 valarray, 1816
 variant, 881
 weak_ordering, 622
 operator>>
 basic_istream, 2066–2068, 2073
 basic_string, 1629
 basic_vec, 1914, 1915
 bitset, 914, 916
 byte, 582
 complex, 1756
 path, 2149
 valarray, 1815
 operator>>=
 basic_vec, 1915
 bitset, 914
 byte, 582
 gslice_array, 1821
 indirect_array, 1823
 mask_array, 1822
 slice_array, 1819
 valarray, 1813
 operator[]
 adjacent_transform_view::iterator, 1384
 adjacent_view::iterator, 1378
 basic_const_iterator, 1235
 basic_mask, 1937
 basic_stacktrace, 669
 basic_string, 1617
 basic_string_view, 1599
 basic_vec, 1913
 bitset, 915
 cartesian_product_view::iterator, 1413
 chunk_view::iterator, 1393
 concat_view::iterator, 1347
 constant_wrapper, 831
 counted_iterator, 1248
 flat_map, 1110
 indirect_array, 1823
 iota_view::iterator, 1291
 izip_view::iterator, 1368
 map, 1047, 1048
 mask_array, 1822
 match_results, 1734
 mdspan, 1180
 move_iterator, 1239
 repeat_view::iterator, 1296
 reverse_iterator, 1226
 shared_ptr, 711
 slide_view::iterator, 1398
 span, 1143
 unique_ptr, 704
 unordered_map, 1070
 valarray, 1811, 1812
 weekday, 1986
 zip_transform_view::iterator, 1373
 operator%
 basic_vec, 1914
 duration, 1961
 valarray, 1815
 operator%=
 basic_vec, 1915
 duration, 1960
 gslice_array, 1821
 indirect_array, 1823
 mask_array, 1822
 slice_array, 1819
 valarray, 1813
 operator&
 basic_mask, 1938
 basic_vec, 1914
 bitset, 916
 byte, 583
 valarray, 1815
 operator&=
 atomic<integral-type>, 2228
 atomic_ref<integral-type>, 2215
 basic_mask, 1938
 basic_vec, 1915
 bitset, 913
 byte, 583
 gslice_array, 1821
 indirect_array, 1823
 mask_array, 1822
 slice_array, 1819
 valarray, 1813

- operator&&
 - basic_mask, 1938
 - valarray, 1816
- operator^
 - basic_mask, 1938
 - basic_vec, 1914
 - bitset, 916
 - byte, 583
 - valarray, 1815
- operator^=
 - atomic<*integral-type*>, 2228
 - atomic_ref<*integral-type*>, 2215
 - basic_mask, 1938
 - basic_vec, 1915
 - bitset, 914
 - byte, 583
 - gslice_array, 1821
 - indirect_array, 1823
 - mask_array, 1822
 - slice_array, 1819
 - valarray, 1813
- operator~
 - basic_mask, 1937
 - basic_vec, 1914
 - bitset, 914
 - bitset::reference, 912
 - byte, 583
 - valarray, 1813
- operator_of, 789
- operators, 789
- operator|
 - basic_mask, 1938
 - basic_vec, 1914
 - bitset, 916
 - byte, 583
 - pipeable sender adaptor closure objects, 2328
 - range adaptor closure objects, 1298
 - simd::flags, 1908
 - valarray, 1815
- operator|=
 - atomic<*integral-type*>, 2228
 - atomic_ref<*integral-type*>, 2215
 - basic_mask, 1938
 - basic_vec, 1915
 - bitset, 913
 - byte, 582
 - gslice_array, 1821
 - indirect_array, 1823
 - mask_array, 1822
 - slice_array, 1819
 - valarray, 1813
- operator||
 - basic_mask, 1938
 - valarray, 1816
- optimize
 - syntax_option_type, 1719, 1720
- optional, 853
 - and_then, 861
 - begin, 860
 - const_iterator, 860
 - constructor, 855, 856
 - destructor, 856
 - emplace, 859
 - end, 860
 - has_value, 861
 - hash, 871
 - iterator, 860
 - operator bool, 861
 - operator!=, 868, 869
 - operator*, 861
 - operator->, 860
 - operator<, 868, 869
 - operator<=, 868, 870
 - operator<=>, 868, 869, 871
 - operator=, 857, 858
 - operator==, 868, 869
 - operator>, 868, 870
 - operator>=, 868, 870
 - or_else, 862
 - reset, 862
 - swap, 859, 871
 - transform, 862
 - value, 861
 - value_or, 861
 - value_type, 853
- options
 - recursive_directory_iterator, 2162
 - synchronized_pool_resource, 744
 - unsynchronized_pool_resource, 744
- or_else
 - expected, 901
 - expected<void>, 908
 - optional, 862
- OSDEBCDICDF03IRV
 - text_encoding, 1682
- OSDEBCDICDF041
 - text_encoding, 1682
- OSDEBCDICDF0415
 - text_encoding, 1682
- ospanstream, 2104
- ostream, 2034, 2062
- ostream_iterator, 1252
 - constructor, 1252, 1253
 - operator*, 1253
 - operator++, 1253
 - operator=, 1253
- ostreambuf_iterator, 1255, 2034
 - constructor, 1255
 - failed, 1256
 - operator*, 1255
 - operator++, 1256
 - operator=, 1255
- ostreamstring, 2034, 2088
- ostrstream
 - zombie, 573
- osyncstream, 2034, 2124
- other
 - text_encoding, 1680

- others_all
 - perms, 2154
- others_exec
 - perms, 2154
- others_read
 - perms, 2154
- others_write
 - perms, 2154
- out
 - basic_format_context, 1703
 - codecvt, 1651
 - format_to_n_result, 1684
 - in_in_out_result, 1487
 - in_out_result, 1487
 - out_value_result, 1488
- out1
 - in_out_out_result, 1487
- out2
 - in_out_out_result, 1487
- out_of_range, 650, 652, 912–915
 - constructor, 652
- out_ptr, 723
- out_ptr_t, 722
 - constructor, 722
 - destructor, 723
- out_value_result, 1488
 - out, 1488
 - value, 1488
- outer_allocator
 - scoped_allocator_adaptor, 748
- outer_allocator_type
 - scoped_allocator_adaptor, 746
- output_iterator, 1211
- output_iterator_tag, 1221
- output_range, 1275
- overflow
 - basic_filebuf, 2117
 - basic_streambuf, 2060
 - basic_stringbuf, 2094
- overflow_error, 650, 652, 912, 915
 - constructor, 653
- overwrite_existing
 - copy_options, 2153
- owner_all
 - perms, 2154
- owner_before
 - shared_ptr, 712
 - weak_ptr, 719
- owner_dead
 - errc, 656
- owner_equal, 720
 - operator(), 720
 - shared_ptr, 712
 - weak_ptr, 719
- owner_exec
 - perms, 2154
- owner_hash, 720
 - operator(), 720
 - shared_ptr, 712

- weak_ptr, 719
- owner_less, 719
 - operator(), 720
- owner_read
 - perms, 2154
- owner_write
 - perms, 2154
- owning_view, 1302
 - constructor, 1303
- owns_lock
 - shared_lock, 2260
 - unique_lock, 2257

P

p

- bernoulli_distribution, 1787
- binomial_distribution, 1788
- geometric_distribution, 1788
- negative_binomial_distribution, 1789
- packaged_task, 2285
 - constructor, 2286, 2287
 - destructor, 2287
 - get_future, 2287
 - make_ready_at_thread_exit, 2287
 - operator(), 2287
 - operator=, 2287
 - reset, 2288
 - swap, 2287, 2288
 - valid, 2287
- padding_value
 - layout_left_padded::mapping, 1161
 - layout_right_padded::mapping, 1167
- pair, 832, 846
 - constructor, 833, 834
 - get, 837
 - operator<=>, 836
 - operator=, 834, 835
 - operator==, 836
 - swap, 836
- pairwise
 - views, 1266
- pairwise_transform
 - views, 1266
- par, 1432
 - execution, 1432
- par_unseq, 1432
 - execution, 1432
- parallel_scheduler, 2304, 2381
- parallel_scheduler_backend, 2383
 - schedule, 2383
 - schedule_bulk_chunked, 2384
 - schedule_bulk_unchunked, 2384
- parallel_scheduler_replacement, 2304
- param
 - seed_seq, 1783
- parameters_of, 799
- parent_of, 797
- parent_path

- path, 2146
- parse, 2027–2031
 - formatter, 661, 1036, 1138, 1712
 - range-default-formatter*, 1706–1708
 - range_formatter, 1705
- part
 - money_base, 1672
- partial
 - codecvt_base, 1650
- partial_gather_from
 - simd, 1922
- partial_load
 - simd, 1918
- partial_order, 626
- partial_ordering, 620
 - equivalent, 620
 - greater, 620
 - less, 620
 - operator<, 621
 - operator<=, 621
 - operator<=>, 621
 - operator==, 621
 - operator>, 621
 - operator>=, 621
 - unordered, 620
- partial_scatter_to
 - simd, 1923
- partial_sort, 1534
- partial_sort_copy, 1535
- partial_sort_copy_result, 1466
- partial_store
 - simd, 1919
- partial_sum, 1573
- partition, 1542
- partition_copy, 1543
- partition_copy_result, 1471
- partition_point, 1545
- path, 2134, 2463
 - append, 2142, 2143
 - assign, 2142
 - begin, 2148
 - c_str, 2144
 - clear, 2143
 - compare, 2145
 - concat, 2143
 - constructor, 2140, 2141
 - copy, 2164
 - directory_entry, 2157
 - display_string, 2144
 - empty, 2147
 - end, 2149
 - extension, 2146
 - filename, 2146
 - generic_display_string, 2145
 - generic_string, 2145, 2463
 - generic_system_encoded_string, 2145
 - generic_u16string, 2145
 - generic_u32string, 2145
 - generic_u8string, 2145
 - generic_wstring, 2145
 - has_extension, 2147
 - has_filename, 2147
 - has_parent_path, 2147
 - has_relative_path, 2147
 - has_root_directory, 2147
 - has_root_name, 2147
 - has_root_path, 2147
 - has_stem, 2147
 - hash_value, 2149
 - is_absolute, 2147
 - is_relative, 2147
 - iterator, 2148
 - lexically_normal, 2147
 - lexically_proximate, 2148
 - lexically_relative, 2147
 - make_preferred, 2143
 - native, 2144
 - operator string_type, 2144
 - operator+=, 2143
 - operator/, 2149
 - operator/=: 2142, 2143
 - operator<<, 2149
 - operator<=>, 2149
 - operator=, 2141, 2142
 - operator==, 2149
 - operator>>, 2149
 - parent_path, 2146
 - preferred_separator, 2137
 - relative_path, 2146
 - remove, 2173
 - remove_filename, 2143
 - replace_extension, 2144
 - replace_filename, 2144
 - root_directory, 2146
 - root_name, 2145
 - root_path, 2146
 - stem, 2146
 - string, 2144, 2463
 - swap, 2144, 2149
 - system_encoded_string, 2144
 - u16string, 2144
 - u32string, 2144
 - u8string, 2144
 - value_type, 2137
 - wstring, 2144
- path1
 - filesystem_error, 2151
- path2
 - filesystem_error, 2151
- pattern
 - money_base, 1672
 - money_punct_byname, 1674
- pbackfail
 - basic_filebuf, 2116
 - basic_streambuf, 2060
 - basic_stringbuf, 2094
- pbase
 - basic_streambuf, 2058

pbump
 basic_streambuf, 2058
 PC775Baltic
 text_encoding, 1684
 PC850Multilingual
 text_encoding, 1682
 PC862LatinHebrew
 text_encoding, 1682
 PC8CodePage437
 text_encoding, 1682
 PC8DanishNorwegian
 text_encoding, 1682
 PC8Turkish
 text_encoding, 1683
 pcount
 zombie, 573
 PCp852
 text_encoding, 1682
 peek
 basic_istream, 2071
 perm_options, 2152
 add, 2154
 nofollow, 2154
 remove, 2154
 replace, 2154
 permission_denied
 errc, 656
 permissions, 2172
 file_status, 2155
 perms, 2152
 all, 2154
 group_all, 2154
 group_exec, 2154
 group_read, 2154
 group_write, 2154
 mask, 2154
 none, 2154
 others_all, 2154
 others_exec, 2154
 others_read, 2154
 others_write, 2154
 owner_all, 2154
 owner_exec, 2154
 owner_read, 2154
 owner_write, 2154
 set_gid, 2154
 set_uid, 2154
 sticky_bit, 2154
 unknown, 2154
 permutable, 1221
 permute
 simd, 1920, 1921
 perror, 2176
 peta, 818
 phi, 1837
 phi_v, 1837
 philox4x32, 1780
 philox4x64, 1780
 philox_engine, 1774, 1775
 constructor, 1776
 result_type, 1775
 set_counter, 1776
 pi, 1837
 pi_v, 1837
 pico, 818
 piecewise_constant_distribution, 1801
 constructor, 1802
 densities, 1803
 intervals, 1803
 result_type, 1801
 piecewise_construct, 837
 piecewise_construct_t, 837
 piecewise_linear_distribution, 1803
 constructor, 1804
 densities, 1805
 intervals, 1804
 result_type, 1803
 placeholders, 935, 936
 plus, 924
 operator(), 924
 plus<>, 924
 operator(), 924
 pointer, 697
 allocator_traits, 690
 basic_string, 1607
 basic_string_view, 1595
 iterator_traits, 1203, 1205
 pointer_traits, 684
 scoped_allocator_adaptor, 746
 pointer_bits_available, 694
 pointer_safety
 zombie, 573
 pointer_tag_pair, 694, 695
 get, 697
 operator<=>, 697
 operator==, 697
 pointer_to
 pointer_traits, 684
 pointer_to_binary_function
 zombie, 573
 pointer_to_unary_function
 zombie, 573
 pointer_traits, 684
 difference_type, 684
 element_type, 684
 pointer, 684
 pointer_to, 684
 rebind, 684
 to_address, 685
 poisson_distribution, 1790
 constructor, 1790
 mean, 1790
 result_type, 1790
 polar
 complex, 1757
 simd, 1935
 polymorphic, 732
 constructor, 733–735

- destructor, 735
- get_allocator, 736
- operator*, 736
- operator->, 736
- operator=, 735, 736
- swap, 736, 737
- valueless_after_move, 736
- polymorphic_allocator, 738
 - allocate, 739
 - allocate_bytes, 740
 - allocate_object, 740
 - construct, 740
 - constructor, 739
 - deallocate, 740
 - deallocate_bytes, 740
 - deallocate_object, 740
 - destroy, 741
 - new_object, 740
 - operator==, 741
 - resource, 741
 - select_on_container_copy_construction, 741
 - value_type, 738
- pool_options, 742
 - largest_required_pool_block, 743
 - max_blocks_per_chunk, 743
- pop
 - forward_list, 1008
 - priority_queue, 1098
 - recursive_directory_iterator, 2163
- pop_back
 - basic_string, 1622
 - inplace_vector, 1041
- pop_heap, 1555
- popcount, 956
 - simd, 1934
- position
 - match_results, 1733
- positive_sign
 - moneypunct, 1673
- pow, 1759, 1826
 - complex, 1758
 - simd, 1928, 1935
 - valarray, 1817
- powf, 1826
- powl, 1826
- pptr
 - basic_streambuf, 2058
- precision
 - ios_base, 1644, 2043
- pred
 - chunk_by_view, 1401
 - drop_while_view, 1321
 - filter_view, 1305
 - take_while_view, 1317
- predicate, 649
- preferred
 - zombie, 573
- preferred_separator
 - path, 2137
- prefix
 - match_results, 1734
- prepend_range
 - forward_list, 1008
- prev, 1223, 1224
 - subrange, 1281
- prev_permutation, 1565
- prev_permutation_result, 1486
- PRIBFASTN, 2177
- PRIBFASTN, 2177
- PRIBLEASTN, 2177
- PRIBLEASTN, 2177
- PRIBMAX, 2178
- PRIBMAX, 2178
- PRIBN, 2177
- PRIBN, 2177
- PRIBPTR, 2178
- PRIBPTR, 2178
- PRIdFASTN, 2177
- PRIdLEASTN, 2177
- PRIdMAX, 2178
- PRIdN, 2177
- PRIdPTR, 2178
- PRIfFASTN, 2177
- PRIfLEASTN, 2177
- PRIfMAX, 2178
- PRIfN, 2177
- PRIfPTR, 2178
- print, 2080, 2087
 - ctype_base, 1645
- printf, 2176
- println, 2080, 2081, 2087
- PRIoFASTN, 2177
- PRIoLEASTN, 2177
- PRIoMAX, 2178
- PRIoN, 2177
- PRIoPTR, 2178
- priority_queue, 1094
 - constructor, 1096, 1097
 - emplace, 1098
 - pop, 1098
 - push, 1097
 - push_range, 1097
 - swap, 1098
- PRIfFASTN, 2177
- PRIfLEASTN, 2177
- PRIfMAX, 2178
- PRIfN, 2177
- PRIfPTR, 2178
- PRIfFASTN, 2177
- PRIfFASTN, 2177
- PRIfLEASTN, 2177
- PRIfLEASTN, 2177
- PRIfMAX, 2178
- PRIfMAX, 2178
- PRIfN, 2177
- PRIfN, 2177
- PRIfPTR, 2178

PRIxPTR, 2178
 probabilities
 discrete_distribution, 1801
 proj
 complex, 1757
 simd, 1934
 projected, 1219
 promise, 2277
 constructor, 2278
 coroutine_handle, 631
 coroutine_handle<noop_coroutine_-
 promise>, 632
 destructor, 2278
 get_future, 2278
 operator=, 2278
 set_exception, 2279
 set_exception_at_thread_exit, 2279
 set_value, 2278
 set_value_at_thread_exit, 2279
 swap, 2278, 2279
 promise_already_satisfied
 future_errc, 2274
 prop, 2359
 propagate_on_container_copy_assignment
 allocator_traits, 691
 scoped_allocator_adaptor, 747
 propagate_on_container_move_assignment
 allocator, 692
 allocator_traits, 691
 scoped_allocator_adaptor, 747
 propagate_on_container_swap
 allocator_traits, 691
 scoped_allocator_adaptor, 747
 protect
 hazard_pointer, 2293
 protocol_error
 errc, 656
 protocol_not_supported
 errc, 656
 proximate, 2172
 proxy
 istreambuf_iterator, 1254
 PTCP154
 text_encoding, 1684
 ptr
 allocation_result, 674
 from_chars_result, 1634
 to_chars_result, 1634
 ptr_fun
 zombie, 573
 PTRDIFF_MAX, 597
 PTRDIFF_MIN, 597
 ptrdiff_t, 579
 pubimbuf
 basic_streambuf, 2056
 pubseekoff
 basic_streambuf, 2056
 pubseekpos
 basic_streambuf, 2056

pubsetbuf
 basic_streambuf, 2056
 pubsync
 basic_streambuf, 2056
 punct
 ctype_base, 1645
 push
 priority_queue, 1097
 push_back
 basic_string, 1619
 deque, 1002
 inplace_vector, 1040
 push_front
 deque, 1002
 forward_list, 1008
 push_heap, 1555
 push_range
 priority_queue, 1097
 queue, 1093
 stack, 1101
 put
 basic_ostream, 2081
 money_put, 1671
 num_put, 1658
 time_put, 1668
 put_money, 2085
 put_time, 2085
 putback
 basic_istream, 2072
 putc, 2176
 putchar, 2176
 putenv, 632
 puts, 2176
 putwc, 1747
 putwchar, 1747
 pword
 ios_base, 2045

Q

qsort, 580, 1587
 quecto, 818, 820
 query
 env, 2361
 query_parallel_scheduler_backend, 2383
 quetta, 818, 820
 queue, 1091
 constructor, 1092, 1093
 operator<, 1093
 operator<=, 1093
 operator<=>, 1093
 operator==, 1093
 operator>, 1093
 operator>=, 1093
 push_range, 1093
 swap, 1094
 quick_exit, 580, 600
 quiet_NaN
 numeric_limits, 593
 quoted, 2086

R

- radix
 - numeric_limits, 592
- raise, 633
- rand, 580, 1805
 - discouraged, 1805
- RAND_MAX, 580
- random_access_iterator, 1213
- random_access_iterator_tag, 1221
- random_access_range, 1275
- random_device, 1781
 - constructor, 1781
 - entropy, 1781
 - operator(), 1781
 - result_type, 1781
- random_shuffle
 - zombie, 573
- range, 1273
- range-default-formatter*, 1706–1708
 - constructor, 1707
 - format, 1706–1708
 - parse, 1706–1708
 - set_brackets, 1706
 - set_separator, 1706
- range_common_reference_t, 1260
- range_const_reference_t, 1260
- range_difference_t, 1260
- range_error, 650, 652
 - constructor, 652
- range_format, 1686
 - debug_string, 1686
 - disabled, 1686
 - map, 1686
 - sequence, 1686
 - set, 1686
 - string, 1686
- range_formatter, 1704
 - format, 1705
 - parse, 1705
 - set_brackets, 1705
 - set_separator, 1705
- range_reference_t, 1260
- range_rvalue_reference_t, 1260
- range_size_t, 1260
- range_slice, 1182
 - first, 1183
 - last, 1183
 - stride, 1183
- range_value_t, 1260
- ranges
 - to, 1283–1285
- rank, 768, 817
 - extents, 1146
 - mdspan, 1176
- rank_dynamic
 - extents, 1146
 - mdspan, 1176
- rank_type
 - extents, 1146
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1152
 - layout_left_padded::mapping, 1161
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1175
- rank_v, 757
- ranlux24, 1780
- ranlux24_base, 1780
- ranlux48, 1780
- ranlux48_base, 1780
- ratio, 817, 818
- ratio_add, 819
- ratio_divide, 819
- ratio_equal, 819
- ratio_equal_v, 818
- ratio_greater, 820
- ratio_greater_equal, 820
- ratio_greater_equal_v, 818
- ratio_greater_v, 818
- ratio_less, 819
- ratio_less_equal, 820
- ratio_less_equal_v, 818
- ratio_less_v, 818
- ratio_multiply, 819
- ratio_not_equal, 819
- ratio_not_equal_v, 818
- ratio_subtract, 819
- raw_storage_iterator
 - zombie, 573
- rbegin, 1270
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - reversible containers, 964
 - span, 1144
- rbegin(C&), 1257
- rbegin(initializer_list<E>), 1257
- rbegin(T (&array)[N]), 1257
- rcu_barrier, 2290
- rcu_default_domain, 2290
- rcu_domain
 - lock, 2290
 - try_lock, 2290
 - unlock, 2290
- rcu_retire, 2290
- rcu_synchronize, 2290
- rdbuf
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ios, 2049
 - basic_istream, 2109
 - basic_istreamstream, 2098
 - basic_ofstream, 2122
 - basic_ostream, 2110
 - basic_ostreamstream, 2101

- basic_spanstream, 2111
- basic_stringstream, 2104
- rdstate
 - basic_ios, 2050
- read
 - basic_istream, 2071
- read_env, 2302
- read_only_file_system
 - errc, 656
- read_symlink, 2172
- readsome
 - basic_istream, 2071
- ready
 - future_status, 2275
 - match_results, 1733
- real, 1759
 - basic_vec, 1913
 - complex, 1755, 1757
 - simd, 1934
- realloc, 580, 693, 2454
- rebind
 - expected, 892
 - expected<void>, 903
 - pointer_traits, 684
 - simd, 1907
- rebind_alloc
 - allocator_traits, 691
- rebind_t
 - simd, 1893
- receiver, 2309
- receiver_proxy, 2382
- receiver_tag, 2300
- recursion_pending
 - recursive_directory_iterator, 2163
- recursive
 - copy_options, 2153
- recursive_directory_iterator, 2161
 - begin, 2163
 - constructor, 2162
 - depth, 2163
 - disable_recursion_pending, 2163
 - end, 2163
 - increment, 2163
 - operator++, 2163
 - operator=, 2162
 - options, 2162
 - pop, 2163
 - recursion_pending, 2163
- recursive_mutex, 2247
- recursive_timed_mutex, 2248
- reduce, 1570
 - simd, 1916
- reduce_count
 - simd, 1939, 1940
- reduce_max
 - simd, 1917
- reduce_max_index
 - simd, 1940
- reduce_min
 - simd, 1917
- reduce_min_index
 - simd, 1939, 1940
- ref
 - reference_wrapper, 923
- ref_view, 1301
 - constructor, 1301
- reference
 - aligned_accessor, 1174
 - basic_string, 1607
 - basic_string_view, 1595
 - conjugated_accessor, 1861
 - containers, 961
 - default_accessor, 1173
 - iterator_traits, 1203–1205
 - mdspan, 1175
 - scaled_accessor, 1859
 - vector<bool>, 1033
- reference_constructs_from_temporary, 766, 815
- reference_constructs_from_temporary_v, 757
- reference_converts_from_temporary, 767, 815
- reference_converts_from_temporary_v, 757
- reference_wrapper, 921
 - constructor, 922
 - cref, 923, 924
 - get, 923
 - operator T&, 923
 - operator(), 923
 - operator=, 922
 - ref, 923
- reflect_constant, 810
- reflect_constant_array, 787
- reflect_constant_string, 786
- reflect_function, 811
- reflect_object, 810
- reflection_range, 809
- refresh
 - directory_entry, 2157
- regex, 1715
- regex_constants, 1719
 - error_type, 1721, 1722
 - match_flag_type, 1719
 - syntax_option_type, 1719
- regex_error, 1722, 1725, 1746
 - constructor, 1722
- regex_iterator, 1740
 - constructor, 1741
 - increment, 1741
 - operator*, 1741
 - operator++, 1741
 - operator->, 1741
 - operator==, 1741
- regex_match, 1735–1737
- regex_replace, 1738, 1739
- regex_search, 1737, 1738
- regex_token_iterator, 1742
 - constructor, 1743, 1744
 - end-of-sequence, 1742

- operator*, 1744
- operator++, 1744
- operator->, 1744
- operator==, 1742, 1744
- regex_traits, 1722
 - char_class_type, 1723
 - isctype, 1723
 - length, 1723
 - lookup_classname, 1723
 - lookup_collatename, 1723
 - transform, 1723
 - transform_primary, 1723
 - translate, 1723
 - translate_nocase, 1723
 - value, 1724
- register_callback
 - ios_base, 2045
- regular, 648
 - file_type, 2152
- regular expression traits
 - isctype, 1745
 - lookup_classname, 1745
 - lookup_collatename, 1745
- regular_invocable, 649
- rehash
 - unordered associative containers, 994
- reinterpret_pointer_cast
 - shared_ptr, 716
- rel_ops, 2459
- relation, 649
- relative, 2172
- relative_path
 - path, 2146
- relaxed
 - memory_order, 2207
- release
 - counting_semaphore, 2270
 - execution::associate_data, 2347
 - memory_order, 2207
 - monotonic_buffer_resource, 745
 - shared_lock, 2260
 - synchronized_pool_resource, 743
 - unique_lock, 2257
 - unique_ptr, 702
 - unsynchronized_pool_resource, 743
- reload_tzdb, 2012
- remainder, 1827
 - simd, 1925
- remainderf, 1827
- remainderl, 1827
- remote_version, 2012
- remove, 1521, 2176
 - forward_list, 1010
 - list, 1026
 - path, 2173
 - perm_options, 2154
- remove_all, 2173
- remove_all_extents, 773, 815
- remove_all_extents_t, 754
- remove_const, 771, 815
- remove_const_t, 753
- remove_copy, 1522
- remove_copy_if, 1522
- remove_copy_if_result, 1458
- remove_copy_result, 1458
- remove_cv, 771, 815
- remove_cv_t, 754
- remove_cvref, 773, 815
- remove_cvref_t, 755
- remove_extent, 773, 815
- remove_extent_t, 754
- remove_filename
 - path, 2143
- remove_if, 1521
 - forward_list, 1010
- remove_pointer, 773, 815
- remove_pointer_t, 754
- remove_prefix
 - basic_string_view, 1600
- remove_reference, 772, 815
- remove_reference_t, 754
- remove_suffix
 - basic_string_view, 1600
- remove_volatile, 771, 815
- remove_volatile_t, 753
- remquo, 1827
- remquoof, 1827
- remquoq, 1827
- rename, 2173, 2176
- rend, 1270
 - basic_stacktrace, 668
 - basic_string, 1615
 - basic_string_view, 1599
 - reversible containers, 965
 - span, 1144
- rend(C&), 1257
- rend(initializer_list<E>), 1257
- rend(T (&array)[N]), 1257
- rep
 - system_clock, 1968
- repeat
 - views, 1293
- repeat_view, 1293
 - begin, 1294
 - constructor, 1294
 - end, 1294
 - size, 1294
- repeat_view::iterator, 1294
 - constructor, 1295
 - operator*, 1295
 - operator+, 1296
 - operator++, 1295
 - operator+=, 1295
 - operator-, 1296
 - operator--, 1295
 - operator-=, 1295
 - operator<=, 1296
 - operator==, 1296

- operator[], 1296
- replace, 1516
 - basic_string, 1622, 1623
 - flat_map, 1114
 - flat_multiset, 1137
 - flat_set, 1129
 - perm_options, 2154
- replace_copy, 1518
- replace_copy_if, 1518
- replace_copy_if_result, 1455
- replace_copy_result, 1454
- replace_extension
 - path, 2144
- replace_filename
 - directory_entry, 2157
 - path, 2144
- replace_if, 1516
- replace_with_range
 - basic_string, 1623
- request_stop
 - execution::counting_scope, 2380
 - inplace_stop_source, 2191
 - jthread, 2200
 - stop_source, 2188
- required_alignment
 - atomic_ref, 2211
 - atomic_ref<floating-point-type>, 2211
 - atomic_ref<integral-type>, 2211
 - atomic_ref<pointer-type>, 2211
- required_span_size
 - layout_blas_packed::mapping, 1855
 - layout_left::mapping, 1154
 - layout_left_padded::mapping, 1166
 - layout_right::mapping, 1157
 - layout_right_padded::mapping, 1171
 - layout_stride::mapping, 1160
 - layout_transpose::mapping, 1863
- reserve
 - basic_string, 1616
 - hive, 1017
 - inplace_vector, 1040
 - unordered associative containers, 994
 - vector, 1031
- reserve_hint, 1271
 - adjacent_transform_view, 1381
 - adjacent_view, 1376
 - as_input_view, 1418
 - cache_latest_view, 1415
 - chunk_view, 1387, 1391
 - chunk_view::outer-iterator::value-type, 1388
 - concat_view, 1343
 - slide_view, 1396
 - stride_view, 1404
- reset
 - any, 887
 - bitset, 914
 - optional, 862
 - packaged_task, 2288
 - shared_ptr, 711
 - unique_ptr, 702, 704
 - weak_ptr, 718
- reset_protection
 - hazard_pointer, 2294
- resetiosflags, 2083
- reshape
 - hive, 1018
- resize
 - basic_string, 1616
 - deque, 1002
 - forward_list, 1009
 - inplace_vector, 1040
 - list, 1024
 - simd, 1907
 - valarray, 1815
 - vector, 1031
- resize_and_overwrite
 - basic_string, 1616
- resize_file, 2173
- resize_t
 - simd, 1893
- resource
 - polymorphic_allocator, 741
- resource_deadlock_would_occur
 - errc, 656
- resource_unavailable_try_again
 - errc, 656
- result
 - codecvt_base, 1650
 - local_info, 2014
- result_of
 - zombie, 573
- result_of_t
 - zombie, 573
- result_out_of_range
 - errc, 656
- result_type
 - bernoulli_distribution, 1786
 - binomial_distribution, 1787
 - cauchy_distribution, 1797
 - chi_squared_distribution, 1796
 - discard_block_engine, 1776
 - discrete_distribution, 1800
 - exponential_distribution, 1790
 - extreme_value_distribution, 1793
 - fisher_f_distribution, 1798
 - function, 937
 - gamma_distribution, 1791
 - geometric_distribution, 1788
 - independent_bits_engine, 1778
 - linear_congruential_engine, 1770
 - lognormal_distribution, 1795
 - mersenne_twister_engine, 1771
 - negative_binomial_distribution, 1789
 - normal_distribution, 1794
 - philox_engine, 1775
 - piecewise_constant_distribution, 1801
 - piecewise_linear_distribution, 1803

- poisson_distribution, 1790
- random_device, 1781
- seed_seq, 1781
- shuffle_order_engine, 1778
- student_t_distribution, 1799
- subtract_with_carry_engine, 1773
- uniform_int_distribution, 1784
- uniform_real_distribution, 1785
- weibull_distribution, 1792
- resume
 - coroutine_handle, 630
 - coroutine_handle<noop_coroutine_-
promise>, 632
- rethrow_exception, 615
- rethrow_if_nested
 - nested_exception, 616
- rethrow_nested
 - nested_exception, 615
- retire
 - hazard_pointer_obj_base, 2292
- return_temporary_buffer
 - zombie, 573
- return_type_of, 799
- return_value
 - task::promise_type, 2374
- return_void
 - task::promise_type, 2374
- reverse, 1526
 - forward_list, 1011
 - list, 1026
 - views, 1351
- reverse_copy, 1527
- reverse_copy_result, 1461
- reverse_copy_truncated_result, 1461
- reverse_iterator, 1225
 - base, 1226
 - basic_string, 1607
 - basic_string_view, 1595
 - constructor, 1226
 - iter_move, 1228
 - iter_swap, 1228
 - make_reverse_iterator non-member
function, 1229
 - operator!=, 1227
 - operator*, 1226
 - operator+, 1227, 1228
 - operator++, 1227
 - operator+=, 1227
 - operator-, 1227, 1228
 - operator--, 1227
 - operator-=, 1227
 - operator->, 1226
 - operator<, 1227
 - operator<=, 1228
 - operator<=>, 1228
 - operator=, 1226
 - operator==, 1227
 - operator>, 1228
 - operator>=, 1228
 - operator[], 1226
 - reversible containers, 964
- reverse_view, 1352
 - base, 1352
 - begin, 1353
 - constructor, 1352
 - end, 1353
 - size, 1352
- rewind, 2176
- rfind
 - basic_string, 1624
 - basic_string_view, 1602
- riemann_zeta, 1835
 - simd, 1931
- riemann_zetaf, 1835
- riemann_zetal, 1835
- right, 2052
- rint, 1827
 - simd, 1925
- rintf, 1827
- rintl, 1827
- ronna, 818, 820
- ronto, 818, 820
- root_directory
 - path, 2146
- root_name
 - path, 2145
- root_path
 - path, 2146
- rotate, 1528
- rotate_copy, 1528
- rotate_copy_result, 1462
- rotate_copy_truncated_result, 1462
- rotl, 955
 - simd, 1933
- rotr, 955
 - simd, 1933
- round, 1827
 - duration, 1962
 - simd, 1925
 - time_point, 1967
- round_error
 - numeric_limits, 592
- round_indeterminate, 590
- round_style
 - numeric_limits, 594
- round_to_nearest, 590
- round_toward_infinity, 590
- round_toward_neg_infinity, 590
- round_toward_zero, 590
- roundf, 1827
- roundl, 1827
- row_major, 1852
- row_major_t, 1852
- run
 - run_loop, 2363
- run_loop, 2361
 - constructor, 2363
 - destructor, 2363

- finish, 2363
- get_scheduler, 2363
- run, 2363
- runtime_error, 650, 652
 - constructor, 652
- S
- s
 - lognormal_distribution, 1796
- same_as, 640
- sample, 1529
- saturating_add, 1579
- saturating_cast, 1579
- saturating_div, 1579
- saturating_mul, 1579
- saturating_sub, 1579
- save
 - sys_info, 2014
- sbumpc
 - basic_streambuf, 2057
- scalbln, 1826
 - simd, 1928
- scalblnf, 1826
- scalblnl, 1826
- scalbn, 1826
 - simd, 1928
- scalbnf, 1826
- scalbnl, 1826
- scale, 1867
- scaled, 1860
- scaled_accessor, 1859
 - access, 1860
 - constructor, 1860
 - data_handle_type, 1859
 - element_type, 1859
 - nested_accessor, 1859
 - offset, 1860
 - offset_policy, 1859
 - reference, 1859
 - scaling_factor, 1859
- scaling_factor
 - scaled_accessor, 1859
- scan_is
 - ctype, 1646
 - ctype<char>, 1649
- scan_not
 - ctype, 1646
 - ctype<char>, 1649
- scanf, 2176
- SCHAR_MAX, 595
- SCHAR_MIN, 595
- SCHAR_WIDTH, 595
- schedule, 2302
 - not-a-sender*, 2318
 - parallel_scheduler_backend, 2383
- schedule_bulk_chunked
 - parallel_scheduler_backend, 2384
- schedule_bulk_unchunked
 - parallel_scheduler_backend, 2384
- schedule_from, 2302
- schedule_from_t, 2302
- schedule_result_t, 2302
- schedule_t, 2302
- scheduler, 2308
- scheduler_tag, 2300
- scientific, 2053
 - chars_format, 1634
- SCNbFASTN, 2177
- SCNbLEASTN, 2177
- SCNbMAX, 2178
- SCNbN, 2177
- SCNbPTR, 2178
- SCNdFASTN, 2177
- SCNdLEASTN, 2177
- SCNdMAX, 2178
- SCNdN, 2177
- SCNdPTR, 2178
- SCNiFASTN, 2177
- SCNiLEASTN, 2177
- SCNiMAX, 2178
- SCNiN, 2177
- SCNiPTR, 2178
- SCNoFASTN, 2177
- SCNoLEASTN, 2177
- SCNoMAX, 2178
- SCNoN, 2177
- SCNoPTR, 2178
- SCNuFASTN, 2177
- SCNuLEASTN, 2177
- SCNuMAX, 2178
- SCNuN, 2177
- SCNuPTR, 2178
- SCNxFASTN, 2177
- SCNxLEASTN, 2177
- SCNxMAX, 2178
- SCNxN, 2177
- SCNxPTR, 2178
- scope
 - access_context, 801
- scope_association, 2375
- scope_token, 2375
- scoped_allocator_adaptor, 746
 - allocate, 748
 - const_pointer, 746
 - const_void_pointer, 746
 - construct, 749
 - constructor, 747, 748
 - deallocate, 748
 - destroy, 749
 - difference_type, 746
 - inner_allocator, 748
 - inner_allocator_type, 747
 - is_always_equal, 747
 - max_size, 748
 - operator==, 749
 - outer_allocator, 748
 - outer_allocator_type, 746

- pointer, 746
- propagate_on_container_copy_assignment, 747
- propagate_on_container_move_assignment, 747
- propagate_on_container_swap, 747
- select_on_container_copy_construction, 749
- size_type, 746
- value_type, 746
- void_pointer, 746
- scoped_lock, 2253
 - constructor, 2253
 - destructor, 2254
- SCSU
 - text_encoding, 1682
- search, 1504, 1506
- search_n, 1505
- second
 - local_info, 2014
- second_argument_type
 - zombie, 573
- seconds, 1941
 - hh_mm_ss, 2009
- seed_seq, 1781
 - constructor, 1782
 - generate, 1782
 - param, 1783
 - result_type, 1781
 - size, 1783
- SEEK_CUR, 2176
- seek_dir
 - zombie, 573
- SEEK_END, 2176
- SEEK_SET, 2176
- seekdir
 - ios_base, 2041
- seekg
 - basic_istream, 2072
- seekoff
 - basic_filebuf, 2117
 - basic_spanbuf, 2107
 - basic_streambuf, 2059
 - basic_stringbuf, 2095
- seekp
 - basic_ostream, 2077
- seekpos
 - basic_filebuf, 2118
 - basic_spanbuf, 2108
 - basic_streambuf, 2059
 - basic_stringbuf, 2095
- select
 - simd, 1925
- select_on_container_copy_construction
 - allocator_traits, 692
 - polymorphic_allocator, 741
 - scoped_allocator_adaptor, 749
- semiregular, 648
- sender, 2320
- sender_adaptor_closure, 2302
- sender_in, 2320
- sender_tag, 2300
- sentinel
 - filter_view, 1307
- sentinel_for, 1210
- sentinel_t, 1260
- sentry
 - basic_istream, 2065
 - basic_ostream, 2077
 - constructor, 2065
 - destructor, 2066
- seq, 1432
 - execution, 1432
- seq_cst
 - memory_order, 2207
- sequence
 - range_format, 1686
- set, 1055
 - allocator_type, 966
 - begin, 962
 - cbegin, 962
 - cend, 962
 - clear, 982
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 978, 1058
 - contains, 983
 - count, 982
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 979
 - emplace_hint, 979
 - empty, 963
 - end, 962
 - equal_range, 983
 - erase, 981, 982
 - erase_if, 1059
 - extract, 981
 - find, 982
 - get_allocator, 966
 - insert, 979–981, 1059
 - iterator, 961
 - key_comp, 979
 - key_compare, 977
 - key_type, 977
 - lower_bound, 983
 - mapped_type, 977
 - max_size, 963
 - merge, 981
 - node_type, 977
 - operator!=, 962
 - operator<, 1058
 - operator<=, 965
 - operator=, 961, 967
 - operator==, 962, 1058
 - range_format, 1686

- rbegin, 964
 - reference, 961
 - rend, 965
 - reverse_iterator, 964
 - size, 963
 - size_type, 961
 - swap, 962, 967
 - upper_bound, 983
 - value_comp, 979
 - value_compare, 977
 - value_type, 961, 977
- set (member)
 - bitset, 914
- set_brackets
 - formatter, 1712
 - range-default-formatter*, 1706
 - range_formatter, 1705
- set_continuation
 - with_awaitable_senders, 2366
- set_counter
 - philox_engine, 1776
- set_debug_format
 - formatter, 661, 2150
- set_default_resource, 741
- set_difference, 1552
- set_difference_result, 1477
- set_difference_truncated_result, 1477
- set_emit_on_sync
 - basic_osyncstream, 2128, 2129
 - basic_syncbuf, 2127
- set_error, 2300
- set_error_t, 2300
- set_exception
 - promise, 2279
- set_exception_at_thread_exit
 - promise, 2279
- set_gid
 - perms, 2154
- set_intersection, 1550
- set_intersection_result, 1476
- set_new_handler, 574, 606
- set_rdbuf
 - basic_ios, 2050
- set_separator
 - formatter, 1712
 - range-default-formatter*, 1706
 - range_formatter, 1705
- set_stopped, 2300
- set_stopped_t, 2300
- set_symmetric_difference, 1553
- set_symmetric_difference_result, 1478
- set_terminate, 574, 613
- set_uid
 - perms, 2154
- set_unexpected
 - zombie, 573
- set_union, 1549
- set_union_result, 1475
- set_value, 2300
 - promise, 2278
- set_value_at_thread_exit
 - promise, 2279
- set_value_t, 2300
- setbase, 2083
- setbuf, 2176
 - basic_filebuf, 2117
 - basic_spanbuf, 2108
 - basic_streambuf, 2058, 2095
- setenv, 632
- setf
 - ios_base, 2043
- setfill, 2083
- setg
 - basic_streambuf, 2058
- setiosflags, 2083
- setjmp, 573, 633
- setlocale, 1676
- setp
 - basic_streambuf, 2058
- setprecision, 2084
- setstate
 - basic_ios, 2050
- setup_givens_rotation, 1866
- setvbuf, 2176
- setw, 2084
- sgetc
 - basic_streambuf, 2057
- sgetn
 - basic_streambuf, 2057
- share
 - future, 2281
- shared_from_this
 - enable_shared_from_this, 721
- shared_future, 2282
 - constructor, 2282
 - destructor, 2283
 - get, 2283
 - operator=, 2283
 - valid, 2283
 - wait, 2283
 - wait_for, 2283
 - wait_until, 2284
- shared_lock, 2257
 - constructor, 2258, 2259
 - destructor, 2259
 - lock, 2259
 - mutex, 2260
 - operator bool, 2260
 - operator=, 2259
 - owns_lock, 2260
 - release, 2260
 - swap, 2260
 - try_lock, 2259
 - try_lock_for, 2259
 - try_lock_until, 2259
 - unlock, 2260
- shared_mutex, 2250
- shared_ptr, 707, 721

- const_pointer_cast, 716
- constructor, 708–710
- destructor, 710
- dynamic_pointer_cast, 716
- get, 711
- get_deleter, 716
- hash, 722
- operator bool, 712
- operator*, 711
- operator->, 711
- operator<<, 717
- operator<=>, 715
- operator=, 710
- operator==, 715
- operator[], 711
- owner_before, 712
- owner_equal, 712
- owner_hash, 712
- reinterpret_pointer_cast, 716
- reset, 711
- static_pointer_cast, 715
- swap, 711, 715
- use_count, 711
- shared_timed_mutex, 2251
- shift
 - valarray, 1814
- shift_left, 1531
- shift_right, 1531
- ShiftJIS
 - text_encoding, 1680
- shl, 955
 - simd, 1933
- showbase, 2051
- showmanyc
 - basic_filebuf, 2116
 - basic_streambuf, 2059, 2116
- showpoint, 2051
- showpos, 2051
- shr, 955
 - simd, 1933
- shrink_to_fit
 - basic_string, 1617
 - deque, 1002
 - hive, 1017
 - inplace_vector, 1040
 - vector, 1031
- SHRT_MAX, 596
- SHRT_MIN, 596
- SHRT_WIDTH, 595
- shuffle, 1530
- shuffle_order_engine, 1778
 - constructor, 1779
 - result_type, 1778
- SIG_ATOMIC_MAX, 597
- SIG_ATOMIC_MIN, 597
- sig_atomic_t, 633
- SIG_DFL, 633
- SIG_ERR, 633
- SIG_IGN, 633
- SIGABRT, 633
- SIGFPE, 633
- SIGILL, 633
- SIGINT, 633
- sign
 - money_base, 1672
- signal, 633
- signaling_NaN
 - numeric_limits, 593
- signbit, 1828
 - simd, 1925
- signed_integral, 642
- SIGSEGV, 633
- SIGTERM, 633
- simd
 - abs, 1925, 1928, 1934
 - acos, 1928, 1934
 - acosh, 1928, 1934
 - alignment, 1907
 - alignment_v, 1893
 - all_of, 1939, 1940
 - any_of, 1939, 1940
 - arg, 1934
 - asin, 1928, 1934
 - asinh, 1928, 1934
 - assoc_laguerre, 1931
 - assoc_legendre, 1931
 - atan, 1928, 1934
 - atan2, 1928
 - atanh, 1928, 1934
 - beta, 1928
 - bit_ceil, 1932
 - bit_compress, 1934
 - bit_expand, 1934
 - bit_floor, 1932
 - bit_repeat, 1933
 - bit_reverse, 1932
 - bit_width, 1934
 - byteswap, 1932
 - cat, 1924
 - cbirt, 1928
 - ceil, 1925
 - chunk, 1923, 1924
 - clamp, 1925
 - comp_ellint_1, 1928
 - comp_ellint_2, 1928
 - comp_ellint_3, 1928
 - compress, 1921
 - conj, 1934
 - copysign, 1925
 - cos, 1928, 1934
 - cosh, 1928, 1934
 - countl_one, 1934
 - countl_zero, 1934
 - countr_one, 1934
 - countr_zero, 1934
 - cyl_bessel_i, 1928
 - cyl_bessel_j, 1928
 - cyl_bessel_k, 1928

cyl_neumann, 1928
 ellint_1, 1928
 ellint_2, 1928
 ellint_3, 1928
 erf, 1928
 erfc, 1928
 exp, 1928, 1934
 exp2, 1928
 expand, 1921
 expint, 1928
 expm1, 1928
 fabs, 1925
 fdim, 1925
 flag_aligned, 1893
 flag_convert, 1893
 flag_default, 1893
 flag_overaligned, 1893
 flags, 1907
 floor, 1925
 fma, 1925
 fmax, 1925
 fmin, 1925
 fmod, 1925
 fpclassify, 1925
 frexp, 1931
 has_single_bit, 1933
 hermite, 1931
 hypot, 1928
 ilogb, 1925
 imag, 1934
 iota, 1924
 isfinite, 1925
 isgreater, 1925
 isgreaterequal, 1925
 isinf, 1925
 isless, 1925
 islessequal, 1925
 islessgreater, 1925
 isnan, 1925
 isnormal, 1925
 isunordered, 1925
 laguerre, 1931
 ldexp, 1928
 legendre, 1931
 lerp, 1928
 lgamma, 1928
 llrint, 1925
 llround, 1925
 log, 1928, 1934
 log10, 1928, 1934
 log1p, 1928
 log2, 1928
 logb, 1928
 lrint, 1925
 lround, 1925
 mask, 1904
 max, 1924
 min, 1924
 minmax, 1924
 modf, 1932
 nearbyint, 1925
 nextafter, 1925
 none_of, 1939, 1940
 norm, 1934
 partial_gather_from, 1922
 partial_load, 1918
 partial_scatter_to, 1923
 partial_store, 1919
 permute, 1920, 1921
 polar, 1935
 popcount, 1934
 pow, 1928, 1935
 proj, 1934
 real, 1934
 rebind, 1907
 rebind_t, 1893
 reduce, 1916
 reduce_count, 1939, 1940
 reduce_max, 1917
 reduce_max_index, 1940
 reduce_min, 1917
 reduce_min_index, 1939, 1940
 remainder, 1925
 resize, 1907
 resize_t, 1893
 riemann_zeta, 1931
 rint, 1925
 rotl, 1933
 rotr, 1933
 round, 1925
 scalbln, 1928
 scalbn, 1928
 select, 1925
 shl, 1933
 shr, 1933
 signbit, 1925
 sin, 1928, 1934
 sinh, 1928, 1934
 sph_bessel, 1931
 sph_legendre, 1931
 sph_neumann, 1931
 sqrt, 1928, 1934
 tan, 1928, 1934
 tanh, 1928, 1934
 tgamma, 1928
 trunc, 1925
 unchecked_gather_from, 1922
 unchecked_load, 1917
 unchecked_scatter_to, 1922
 unchecked_store, 1919
 uninit_element, 1895
 vec, 1894
 zero_element, 1895
 simd::flags
 operator|, 1908
 sin, 1824
 complex, 1758
 simd, 1928, 1934

- valarray, 1817
- sinf, 1824
- single
 - views, 1285
- single_view, 1286
 - begin, 1286
 - constructor, 1286
 - data, 1286
 - empty, 1286
 - end, 1286
 - size, 1286
- sinh, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- sinhf, 1825
- sinhl, 1825
- sinl, 1824
- size, 1271
 - adjacent_transform_view, 1381
 - adjacent_view, 1376
 - array, 997, 998
 - as_input_view, 1418
 - basic_mask, 1935
 - basic_stacktrace, 669
 - basic_string, 1615
 - basic_string_view, 1599
 - basic_vec, 1910
 - bitset, 915
 - cache_latest_view, 1415
 - cartesian_product_view, 1410
 - chunk_view, 1387, 1391
 - chunk_view::outer_iterator::value_type, 1388
 - common_view, 1350
 - concat_view, 1343
 - containers, 963
 - drop_view, 1319
 - elements_view, 1355
 - enumerate_view, 1360
 - flat_map, 1110
 - format_to_n_result, 1684
 - gslice, 1820
 - initializer_list, 619
 - iota_view, 1289
 - match_results, 1733
 - mdspan, 1181
 - repeat_view, 1294
 - reverse_view, 1352
 - seed_seq, 1783
 - single_view, 1286
 - slice, 1818
 - slide_view, 1396
 - span, 1143
 - stride_view, 1404
 - subrange, 1281
 - take_view, 1314
 - transform_view, 1308
 - valarray, 1814
 - zip_transform_view, 1370
 - zip_view, 1366
- size(C& c), 1257
- size(T (&array)[N]), 1257
- size_bytes
 - span, 1143
- SIZE_MAX, 597
- size_of, 806
- size_t, 155, 579, 580, 1632, 1747, 1749, 2032, 2176
- size_type
 - allocator, 692
 - allocator_traits, 691
 - basic_string, 1607
 - basic_string_view, 1595
 - containers, 961
 - extents, 1146
 - layout_blas_packed::mapping, 1853
 - layout_left::mapping, 1152
 - layout_left_padded::mapping, 1161
 - layout_right::mapping, 1155
 - layout_right_padded::mapping, 1167
 - layout_stride::mapping, 1158
 - layout_transpose::mapping, 1863
 - mdspan, 1175
 - scoped_allocator_adaptor, 746
- sized
 - subrange_kind, 1261
- sized_range, 1274
- sized_sentinel_for, 1211
- skip_existing
 - copy_options, 2153
- skip_permission_denied
 - directory_options, 2154
- skip_symlinks
 - copy_options, 2153
- skipws, 2051
- sleep_for
 - this_thread, 2201
- sleep_until
 - this_thread, 2201
- slice, 1817
 - constructor, 1817
 - size, 1818
 - start, 1818
 - stride, 1818
- slice_array, 1818
 - operator*=, 1819
 - operator+=", 1819
 - operator-=", 1819
 - operator/=", 1819
 - operator<<=", 1819
 - operator=, 1818, 1819
 - operator>>=", 1819
 - operator%=", 1819
 - operator&=", 1819
 - operator^=", 1819
 - operator|=, 1819
 - value_type, 1818

slide
 views, 1394
 slide_view, 1395
 begin, 1395, 1396
 constructor, 1395
 end, 1396
 reserve_hint, 1396
 size, 1396
 slide_view::iterator, 1396
 constructor, 1397, 1398
 operator*, 1398
 operator+, 1399
 operator++, 1398
 operator+==, 1398
 operator-, 1399
 operator--, 1398
 operator==, 1398
 operator<, 1398
 operator<=, 1399
 operator<=>, 1399
 operator==, 1398
 operator>, 1398
 operator>=, 1399
 operator[], 1398
 slide_view::sentinel, 1399
 constructor, 1399
 operator-, 1400
 operator==, 1400
 snprintf, 2176
 socket
 file_type, 2152
 sort, 1533
 forward_list, 1011
 hive, 1020
 list, 1027
 sort_heap, 1556
 sortable, 1221
 source_file
 stacktrace_entry, 666
 source_line
 stacktrace_entry, 666
 source_location, 610
 source_location_of, 792
 space, 2173
 ctype_base, 1645
 money_base, 1672
 span, 1138
 at, 1143
 back, 1143
 basic_ispanstream, 2109
 basic_ostream, 2110
 basic_spanbuf, 2107
 basic_spanstream, 2111
 begin, 1144
 constructor, 1140–1142
 data, 1143
 deduction guide, 1142
 empty, 1143
 end, 1144
 first, 1142, 1143
 front, 1143
 iterator, 1144
 last, 1142, 1143
 operator=, 1142
 rbegin, 1144
 rend, 1144
 size, 1143
 size_bytes, 1143
 subspan, 1142, 1143
 spanbuf, 2104
 spanstream, 2104
 spawn_future, 2302
 spawn_future_t, 2302
 spawn_t, 2303
 sph_bessel, 1836
 simd, 1931
 sph_besself, 1836
 sph_bessell, 1836
 sph_legendre, 1836
 simd, 1931
 sph_legendref, 1836
 sph_legendrel, 1836
 sph_neumann, 1836
 simd, 1931
 sph_neumannf, 1836
 sph_neumannl, 1836
 splice
 hive, 1019
 list, 1025
 splice_after
 forward_list, 1009, 1010
 split_view, 1338
 find-next, 1339
 begin, 1339
 constructor, 1339
 split_view::iterator, 1339
 base, 1340
 constructor, 1340
 operator*, 1340
 operator++, 1340
 operator==, 1340
 split_view::sentinel, 1340
 constructor, 1341
 operator==, 1341
 sprintf, 2176
 sputbackc
 basic_streambuf, 2057
 sputc
 basic_streambuf, 2057
 sputn
 basic_streambuf, 2057
 sqrt, 1826
 complex, 1758
 simd, 1928, 1934
 valarray, 1817
 sqrt2, 1837

sqrt2_v, 1837
 sqrt3, 1837
 sqrt3_v, 1837
 sqrtf, 1826
 sqrtl, 1826
 srand, 580, 1805
 sscanf, 2176
 ssize, 1271
 ssize(C& c), 1257
 ssize(T (&array)[N]), 1257
 stable_partition, 1543
 stable_sort, 1533
 stack, 1099
 constructor, 1100
 operator<, 1101
 operator<=, 1101
 operator<=>, 1101
 operator==, 1101
 operator>, 1101
 operator>=, 1101
 push_range, 1101
 swap, 1101
 stacktrace_entry, 665
 constructor, 665
 description, 666
 native_handle, 665
 operator bool, 665
 operator<<, 669
 operator==, 666
 source_file, 666
 source_line, 666
 start, 2300
 gslice, 1820
 slice, 1818
 task::state, 2372
 start_lifetime, 686
 start_lifetime_as, 686
 start_lifetime_as_array, 686
 start_t, 2300
 starts_on, 2302
 starts_on_t, 2302
 starts_with, 1506
 basic_string, 1626
 basic_string_view, 1601
 state
 fpos, 2046
 state_not_recoverable
 errc, 656
 state_type
 char_traits, 1591
 codecvt, 1650
 static_data_members_of, 805
 static_extent
 extents, 1149
 mdspan, 1176
 static_pointer_cast
 shared_ptr, 715
 status, 2174
 directory_entry, 2158
 status_known, 2175
 stdc_bit_ceil, 959
 stdc_bit_ceil_uc, 959
 stdc_bit_ceil_ui, 959
 stdc_bit_ceil_ul, 959
 stdc_bit_ceil_ull, 959
 stdc_bit_ceil_us, 959
 stdc_bit_floor, 959
 stdc_bit_floor_uc, 959
 stdc_bit_floor_ui, 959
 stdc_bit_floor_ul, 959
 stdc_bit_floor_ull, 959
 stdc_bit_floor_us, 959
 stdc_bit_width, 959
 stdc_bit_width_uc, 959
 stdc_bit_width_ui, 959
 stdc_bit_width_ul, 959
 stdc_bit_width_ull, 959
 stdc_bit_width_us, 959
 stdc_count_ones_uc, 958
 stdc_count_ones_ui, 958
 stdc_count_ones_ul, 958
 stdc_count_ones_ull, 958
 stdc_count_ones_us, 958
 stdc_count_zeros, 958
 stdc_count_zeros_uc, 958
 stdc_count_zeros_ui, 958
 stdc_count_zeros_ul, 958
 stdc_count_zeros_ull, 958
 stdc_count_zeros_us, 958
 stdc_first_leading_one_uc, 958
 stdc_first_leading_one_ui, 958
 stdc_first_leading_one_ul, 958
 stdc_first_leading_one_ull, 958
 stdc_first_leading_one_us, 958
 stdc_first_leading_zero, 958
 stdc_first_leading_zero_uc, 958
 stdc_first_leading_zero_ui, 958
 stdc_first_leading_zero_ul, 958
 stdc_first_leading_zero_ull, 958
 stdc_first_leading_zero_us, 958
 stdc_first_trailing_one_uc, 958
 stdc_first_trailing_one_ui, 958
 stdc_first_trailing_one_ul, 958
 stdc_first_trailing_one_ull, 958
 stdc_first_trailing_one_us, 958
 stdc_first_trailing_zero_uc, 958
 stdc_first_trailing_zero_ui, 958
 stdc_first_trailing_zero_ul, 958
 stdc_first_trailing_zero_ull, 958
 stdc_first_trailing_zero_us, 958
 stdc_has_single_bit, 958
 stdc_has_single_bit_uc, 958
 stdc_has_single_bit_ui, 958
 stdc_has_single_bit_ul, 958
 stdc_has_single_bit_ull, 958
 stdc_has_single_bit_us, 958
 stdc_leading_ones, 957
 stdc_leading_ones_uc, 957

stdc_leading_ones_ui, 957
 stdc_leading_ones_ul, 957
 stdc_leading_ones_ull, 957
 stdc_leading_ones_us, 957
 stdc_leading_zeros, 957
 stdc_leading_zeros_uc, 957
 stdc_leading_zeros_ui, 957
 stdc_leading_zeros_ul, 957
 stdc_leading_zeros_ull, 957
 stdc_leading_zeros_us, 957
 stdc_trailing_ones, 958
 stdc_trailing_ones_uc, 958
 stdc_trailing_ones_ui, 958
 stdc_trailing_ones_ul, 958
 stdc_trailing_ones_ull, 958
 stdc_trailing_ones_us, 958
 stdc_trailing_zeros, 958
 stdc_trailing_zeros_uc, 958
 stdc_trailing_zeros_ui, 958
 stdc_trailing_zeros_ul, 958
 stdc_trailing_zeros_ull, 958
 stdc_trailing_zeros_us, 958
 stddev
 normal_distribution, 1795
 stderr, 2176
 stdin, 2176
 stdout, 2176
 steady_clock, 1974
 stem
 path, 2146
 sticky_bit
 perms, 2154
 stod, 1631
 stof, 1631
 stoi, 1630, 1631
 stol, 1630, 1631
 stold, 1631
 stoll, 1630, 1631
 stop_callback, 2189
 callback_type, 2189
 constructor, 2189
 destructor, 2189
 stop_callback_for_t, 2185
 stop_possible
 inplace_stop_source, 2191
 inplace_stop_token, 2190
 never_stop_token, 2190
 stop_source, 2188
 stop_token, 2187
 stop_requested
 inplace_stop_source, 2191
 inplace_stop_token, 2190
 never_stop_token, 2190
 stop_source, 2188
 stop_token, 2187
 stop_source, 2188
 constructor, 2188
 get_token, 2188
 request_stop, 2188
 stop_possible, 2188
 stop_requested, 2188
 swap, 2188
 stop_token, 2187
 callback_type, 2187
 stop_possible, 2187
 stop_requested, 2187
 swap, 2187
 stoppable_token, 2186
 stopped_as_error, 2302
 stopped_as_error_t, 2302
 stopped_as_optional, 2302
 stopped_as_optional_t, 2302
 storage_order_type
 layout_blas_packed, 1853
 store
 atomic, 2222
 atomic<floating-point-type>, 2222
 atomic<integral-type>, 2222
 atomic<shared_ptr<T>>, 2235
 atomic<T*>, 2222
 atomic<weak_ptr<T>>, 2238
 atomic_ref, 2211
 atomic_ref<floating-point-type>, 2211
 atomic_ref<integral-type>, 2211
 atomic_ref<pointer-type>, 2211
 store_add
 atomic<integral-type>, 2227
 atomic<T*>, 2233
 atomic_ref<floating-point-type>, 2217
 atomic_ref<integral-type>, 2215
 atomic_ref<pointer-type>, 2219
 store_and
 atomic<integral-type>, 2227
 atomic_ref<integral-type>, 2215
 store_fmaximum
 atomic<floating-point-type>, 2230
 atomic_ref<floating-point-type>, 2217
 store_fmaximum_num
 atomic<floating-point-type>, 2230
 atomic_ref<floating-point-type>, 2217
 store_fminimum
 atomic<floating-point-type>, 2230
 atomic_ref<floating-point-type>, 2217
 store_fminimum_num
 atomic<floating-point-type>, 2230
 atomic_ref<floating-point-type>, 2217
 store_max
 atomic<floating-point-type>, 2230
 atomic<integral-type>, 2227
 atomic<T*>, 2233
 atomic_ref<floating-point-type>, 2217
 atomic_ref<integral-type>, 2215
 atomic_ref<pointer-type>, 2219
 store_min
 atomic<floating-point-type>, 2230
 atomic<integral-type>, 2227
 atomic<T*>, 2233
 atomic_ref<floating-point-type>, 2217

- `atomic_ref<integral-type>`, 2215
- `atomic_ref<pointer-type>`, 2219
- `store_or`
 - `atomic<integral-type>`, 2227
 - `atomic_ref<integral-type>`, 2215
- `store_sub`
 - `atomic<integral-type>`, 2227
 - `atomic<T*>`, 2233
 - `atomic_ref<floating-point-type>`, 2217
 - `atomic_ref<integral-type>`, 2215
 - `atomic_ref<pointer-type>`, 2219
- `stossc`
 - `zombie`, 573
- `stoul`, 1630, 1631
- `stoull`, 1630, 1631
- `str`
 - `basic_istream`, 2098
 - `basic_ostringstream`, 2101
 - `basic_stringbuf`, 2093, 2094
 - `basic_stringstream`, 2104
 - `match_results`, 1733
 - `sub_match`, 1729
- `strcat`, 1632
- `strchr`, 1632
- `strcmp`, 1632
- `strcoll`, 1632
- `strcpy`, 1632
- `strcspn`, 1632
- `stream`
 - `io_errc`, 2039
- `stream_timeout`
 - `errc`, 2457
- `streambuf`, 2034, 2053
- `streamoff`, 2033, 2046
- `streampos`, 2034
- `streamsize`, 2033
- `strerror`, 1632
- `strftime`, 1669, 2032
- `strict`
 - `zombie`, 573
- `strict_weak_order`, 649
- `stride`
 - `extent_slice`, 1183
 - `gslice`, 1820
 - `layout_blas_packed::mapping`, 1855
 - `layout_left::mapping`, 1155
 - `layout_left_padded::mapping`, 1166
 - `layout_right::mapping`, 1157
 - `layout_right_padded::mapping`, 1172
 - `layout_stride::mapping`, 1158
 - `layout_transpose::mapping`, 1864
 - `mdspan`, 1177
 - `range_slice`, 1183
 - `slice`, 1818
 - `stride_view`, 1404
- `stride_type`
 - `extent_slice`, 1182
- `stride_view`, 1403
 - constructor, 1403
 - `reserve_hint`, 1404
 - `size`, 1404
 - `stride`, 1404
- `stride_view::iterator`, 1404
 - `base`, 1406
 - constructor, 1405, 1406
 - `iter_move`, 1407
 - `iter_swap`, 1407
 - `operator+`, 1407
 - `operator++`, 1406
 - `operator+=`, 1406
 - `operator-`, 1407
 - `operator--`, 1406
 - `operator-=`, 1406
 - `operator<`, 1406
 - `operator<=`, 1407
 - `operator<=>`, 1407
 - `operator==`, 1406
 - `operator>`, 1406
 - `operator>=`, 1407
- `strides`
 - `layout_left_padded::mapping`, 1166
 - `layout_right_padded::mapping`, 1171
 - `layout_stride::mapping`, 1158
- `string`, 1606
 - `hash`, 1632
 - `operator"s`, 1632
 - `path`, 2144, 2463
 - `range_format`, 1686
- `string_type`
 - `collate`, 1663
 - `collate_byname`, 1664
 - `messages`, 1675
 - `messages_byname`, 1676
 - `money_get`, 1670
 - `money_put`, 1671
 - `moneypunct`, 1672
 - `moneypunct_byname`, 1674
 - `numpunct`, 1661
 - `numpunct_byname`, 1663
- `string_view`, 1595
 - `hash`, 1603
 - `operator"sv`, 1603
- `stringbuf`, 2034, 2088
- `stringstream`, 2034, 2088
- `strlen`, 1632
- `strncat`, 1632
- `strncmp`, 1632
- `strncpy`, 1632
- `strong_order`, 625
- `strong_ordering`, 622
 - `equal`, 622
 - `equivalent`, 622
 - `greater`, 622
 - `less`, 622
 - `operator partial_ordering`, 623
 - `operator weak_ordering`, 623
 - `operator<`, 623
 - `operator<=`, 623

- operator<=, 623
- operator==, 623
- operator>, 623
- operator>=, 623
- strpbrk, 1632
- strrchr, 1632
- strspn, 1632
- strstr, 1632
- strstream
 - zombie, 573
- strstreambuf
 - zombie, 573
- strtod, 580
- strtod, 580
- strtoimax, 2177
- strtok, 1632
- strtoll, 580
- strtold, 580
- strtoll, 580
- strtoul, 580
- strtoull, 580
- strtoumax, 2177
- strxfrm, 1632
- student_t_distribution, 1799
 - constructor, 1800
 - mean, 1800
 - result_type, 1799
- sub_match, 1729
 - compare, 1729
 - constructor, 1729
 - length, 1729
 - operator basic_string, 1729
 - operator<<, 1731
 - operator<=, 1730
 - operator==, 1730
 - str, 1729
 - swap, 1729
- subextents, 1185
- submdspan, 1190
- submdspan-mapping-impl*
 - layout_left::mapping, 1187
 - layout_left_padded::mapping, 1188
 - layout_right::mapping, 1187
 - layout_right_padded::mapping, 1189
 - layout_stride::mapping, 1188
- submdspan_mapping, 1186
 - layout_left::mapping, 1153
 - layout_left_padded::mapping, 1162
 - layout_right::mapping, 1156
 - layout_right_padded::mapping, 1168
 - layout_stride::mapping, 1159
- submdspan_mapping_result, 1183
 - mapping, 1183
 - offset, 1183
- subobjects_of, 805
- subrange, 1279
 - advance, 1281
 - begin, 1281
 - constructor, 1280
 - empty, 1281
 - end, 1281
 - get, 1282
 - next, 1281
 - operator *PairLike*, 1281
 - prev, 1281
 - size, 1281
- subrange_kind, 1261
 - sized, 1261
 - unsized, 1261
- subseconds
 - hh_mm_ss, 2009
- subspan
 - span, 1142, 1143
- substitute, 809
- substr
 - basic_string, 1625
 - basic_string_view, 1600
- subtract_with_carry_engine, 1772, 1773
 - constructor, 1773, 1774
 - result_type, 1773
- subview
 - basic_string, 1625
 - basic_string_view, 1600
- suffix
 - match_results, 1734
- sum
 - valarray, 1814
- sungetc
 - basic_streambuf, 2057
- suspend_always, 632
 - await_ready, 632
 - await_resume, 632
 - await_suspend, 632
- suspend_never, 632
 - await_ready, 632
 - await_resume, 632
 - await_suspend, 632
- swap, 643, 697, 824, 825
 - allocator-aware containers, 967
 - any, 887
 - array, 998
 - basic_filebuf, 2115
 - basic_fstream, 2124
 - basic_ifstream, 2120
 - basic_ios, 2050
 - basic_iostream, 2074
 - basic_ispanstream, 2109
 - basic_istream, 2065
 - basic_istreamstream, 2098
 - basic_ofstream, 2122
 - basic_ostream, 2110
 - basic_ostream, 2076
 - basic_ostringstream, 2101
 - basic_regex, 1728, 1729
 - basic_spanbuf, 2107
 - basic_spanstream, 2111
 - basic_stacktrace, 669
 - basic_streambuf, 2058

- basic_string, 1624, 1629
- basic_string_view, 1600
- basic_stringbuf, 2092
- basic_stringstream, 2104
- basic_syncbuf, 2126, 2127
- bitset::reference, 912
- containers, 962
- copyable_function, 946
- expected, 898, 899
- expected<void>, 907
- flat_map, 1114
- flat_multiset, 1137
- flat_set, 1129
- function, 939, 940
- hazard_pointer, 2294
- hive, 1019
- indirect, 731
- inplace_stop_token, 2190
- inplace_vector, 1041
- jthread, 2200
- match_results, 1735
- mdspan, 1181
- move_only_function, 943
- optional, 859, 871
- packaged_task, 2287, 2288
- pair, 836
- path, 2144, 2149
- polymorphic, 736, 737
- priority_queue, 1098
- promise, 2278, 2279
- queue, 1094
- shared_lock, 2260
- shared_ptr, 711, 715
- stack, 1101
- stop_source, 2188
- stop_token, 2187
- sub_match, 1729
- thread, 2197
- tuple, 847, 852
- unexpected, 890
- unique_lock, 2257
- unique_ptr, 702
- valarray, 1814, 1817
- variant, 879, 883
- vector, 1031
- vector<bool>, 2460
- vector<bool>::reference, 1035
- weak_ptr, 718, 719
- swap(unique_ptr&, unique_ptr&), 705
- swap_elements, 1867
- swap_ranges, 1513
- swap_ranges_result, 1451
- swappable, 643
- swappable_with, 643
- swprintf, 1747
- swscanf, 1747
- symbol
 - money_base, 1672
- symbol_of, 789
- symlink
 - file_type, 2152
- symlink_status, 2175
 - directory_entry, 2158
- symmetric_matrix_product, 1882, 1883
- symmetric_matrix_rank_1_update, 1879
- symmetric_matrix_rank_2_update, 1880
- symmetric_matrix_rank_2k_update, 1886, 1887
- symmetric_matrix_rank_k_update, 1885, 1886
- symmetric_matrix_vector_product, 1874
- sync
 - basic_filebuf, 2118
 - basic_istream, 2072
 - basic_streambuf, 2059
 - basic_syncbuf, 2127
- sync_wait, 2303
- sync_wait_t, 2303
- sync_wait_with_variant, 2303
- sync_wait_with_variant_t, 2303
- sync_with_stdio
 - ios_base, 2044
- syncbuf, 2034, 2124
- synchronized_pool_resource, 742
 - constructor, 743
 - destructor, 743
 - do_allocate, 744
 - do_deallocate, 744
 - do_is_equal, 744
 - options, 744
 - release, 743
 - upstream_resource, 743
- syntax_option_type, 1719
 - awk, 1719, 1720
 - basic, 1719, 1720
 - collate, 1719, 1720, 1746
 - ECMAScript, 1719, 1720
 - egrep, 1719, 1720
 - extended, 1719, 1720
 - grep, 1719, 1720
 - icase, 1719, 1720
 - multiline, 1720
 - nosubs, 1719, 1720
 - optimize, 1719, 1720
 - regex_constants, 1719
- sys_days, 1941
 - operator<<, 1968
- sys_info, 2013
 - abbrev, 2014
 - begin, 2014
 - end, 2014
 - offset, 2014
 - operator<<, 2014
 - save, 2014
- sys_seconds, 1941
- sys_time, 1941
 - from_stream, 1968
 - operator<, 2021
 - operator<<, 1968
 - operator<=, 2022

- operator<=, 2022
- operator==, 2021
- operator>, 2022
- operator>=, 2022
- system, 580, 632
- system_category, 657, 659
- system_clock, 1967
 - from_time_t, 1968
 - rep, 1968
 - to_time_t, 1968
- system_encoded_string
 - path, 2144
- system_error, 655, 663
 - code, 664
 - constructor, 663, 664
 - what, 664

T

- t
 - binomial_distribution, 1788
- table
 - ctype<char>, 1649
- table_size
 - ctype<char>, 1648
- tag, 697
- tagged_pointer, 696
- tagged_pointer_type, 696
- tai_clock, 1971
 - from_utc, 1971
 - now, 1971
 - to_utc, 1971
- tai_seconds, 1941
- tai_time, 1941
 - from_stream, 1972
 - operator<<, 1972
- take
 - views, 1313
- take_view, 1314
 - base, 1314
 - begin, 1314
 - constructor, 1316
 - end, 1314
 - size, 1314
- take_view::sentinel, 1316
 - base, 1316
 - constructor, 1316
 - operator==, 1316
- take_while, 1316
 - views, 1316
- take_while_view, 1317
 - base, 1317
 - begin, 1317
 - constructor, 1317
 - end, 1317
 - pred, 1317
- take_while_view::sentinel, 1318
 - constructor, 1318
 - operator==, 1318

- tan, 1824
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- tanf, 1824
- tanh, 1825
 - complex, 1758
 - simd, 1928, 1934
 - valarray, 1817
- tanhf, 1825
- tanh1, 1825
- tan1, 1824
- target
 - function, 940
 - time_zone_link, 2022
- target_type
 - function, 939
- task, 2304, 2370
 - connect, 2371
 - constructor, 2371
 - destructor, 2371
 - get_completion_signatures, 2371
- task::state
 - get-stop-token, 2373
 - constructor, 2372
 - destructor, 2372
 - start, 2372
- task::promise_type
 - await_transform, 2374
 - final_suspend, 2373
 - get_env, 2374
 - get_return_object, 2373
 - initial_suspend, 2373
 - operator delete, 2375
 - operator new, 2374
 - return_value, 2374
 - return_void, 2374
 - unhandled_exception, 2374
 - unhandled_stopped, 2374
 - yield_value, 2374
- task_scheduler, 2303, 2367
 - constructor, 2368
 - operator==, 2368
- tellg
 - basic_istream, 2072
- tellp
 - basic_ostream, 2077
- temp_directory_path, 2175
- template_arguments_of, 798
- template_of, 798
- tera, 818
- terminate, 599, 600, 613, 614
- terminate_handler, 574, 613
- test
 - atomic_flag, 2240
 - bitset, 915
- test_and_set
 - atomic_flag, 2240
- text_encoding, 1677

comp-name, 1679
 AdobeStandardEncoding, 1682
 aliases, 1679
 aliases_view, 1680
 Amiga1251, 1684
 ASCII, 1680
 Big5, 1683
 Big5HKSCS, 1684
 BOCU1, 1682
 BRF, 1684
 CESU8, 1682
 constructor, 1678
 CP50220, 1684
 CP51932, 1684
 DECMCS, 1682
 DKUS, 1682
 EBCDICATDEA, 1683
 EBCDICCFAFR, 1683
 EBCDICDKNO, 1683
 EBCDICDKNOA, 1683
 EBCDICES, 1683
 EBCDICESA, 1683
 EBCDICESSE, 1683
 EBCDICFISE, 1683
 EBCDICFISEA, 1683
 EBCDICFR, 1683
 EBCDICIT, 1683
 EBCDICPT, 1683
 EBCDICUK, 1683
 EBCDICUS, 1684
 environment, 1679
 environment_is, 1679
 EUCLFixWidJapanese, 1680
 EUCKR, 1681
 EUCPkdfmtJapanese, 1680
 GB18030, 1682
 GB2312, 1683
 GBK, 1682
 HalfWidthKatakana, 1680
 hash, 1684
 HPDesktop, 1683
 HPLegal, 1683
 HPMath8, 1683
 HPPiFont, 1683
 HPPSMath, 1683
 HPRoman8, 1682
 HZGB2312, 1684
 IBM00858, 1684
 IBM00924, 1684
 IBM01140, 1684
 IBM01141, 1684
 IBM01142, 1684
 IBM01143, 1684
 IBM01144, 1684
 IBM01145, 1684
 IBM01146, 1684
 IBM01147, 1684
 IBM01148, 1684
 IBM01149, 1684
 IBM037, 1683
 IBM038, 1683
 IBM1026, 1683
 IBM1047, 1684
 IBM273, 1683
 IBM274, 1683
 IBM275, 1683
 IBM277, 1683
 IBM278, 1683
 IBM280, 1683
 IBM281, 1683
 IBM284, 1683
 IBM285, 1683
 IBM290, 1683
 IBM297, 1683
 IBM420, 1683
 IBM423, 1683
 IBM424, 1683
 IBM500, 1683
 IBM851, 1683
 IBM855, 1683
 IBM857, 1683
 IBM860, 1683
 IBM861, 1683
 IBM863, 1683
 IBM864, 1683
 IBM865, 1683
 IBM866, 1684
 IBM868, 1683
 IBM869, 1683
 IBM870, 1683
 IBM871, 1683
 IBM880, 1683
 IBM891, 1683
 IBM903, 1683
 IBM904, 1683
 IBM905, 1683
 IBM918, 1683
 IBMEBCDICATDE, 1683
 IBMSymbols, 1683
 IBMThai, 1683
 id, 1680
 INVARIANT, 1681
 ISO102T617bit, 1681
 ISO10367Box, 1682
 ISO103T618bit, 1681
 ISO10646UTF1, 1681
 ISO10Swedish, 1681
 ISO111ECMACyrillic, 1681
 ISO115481, 1682
 ISO11SwedishForNames, 1680
 ISO121Canadian1, 1681
 ISO122Canadian2, 1681
 ISO123CSAZ24341985gr, 1681
 ISO128T101G2, 1681
 ISO139CSN369103, 1681
 ISO13JISC6220jp, 1681
 ISO141JUSIB1002, 1681
 ISO143IECP271, 1681

ISO146Serbian, 1681
 ISO147Macedonian, 1682
 ISO14JISC6220ro, 1681
 ISO150, 1682
 ISO151Cuba, 1682
 ISO153GOST1976874, 1682
 ISO158Lap, 1682
 ISO159JISX02121990, 1682
 ISO15Italian, 1680
 ISO16Portuguese, 1681
 ISO17Spanish, 1680
 ISO18Greek70ld, 1681
 ISO19LatinGreek, 1681
 ISO2022CN, 1682
 ISO2022CNEXT, 1682
 ISO2022JP, 1681
 ISO2022JP2, 1681
 ISO2022KR, 1681
 ISO2033, 1681
 ISO21German, 1681
 ISO25French, 1681
 ISO27LatinGreek1, 1681
 ISO2IntlRefVersion, 1681
 ISO42JISC62261978, 1681
 ISO47BSViewdata, 1681
 ISO49INIS, 1681
 ISO4UnitedKingdom, 1680
 ISO50INIS8, 1681
 ISO51INISCyrillic, 1681
 ISO54271981, 1681
 ISO5427Cyrillic, 1681
 ISO5428Greek, 1681
 ISO57GB1988, 1681
 ISO58GB231280, 1681
 ISO60DanishNorwegian, 1681
 ISO61Norwegian2, 1681
 ISO646basic1983, 1681
 ISO646Danish, 1682
 ISO6937Add, 1682
 ISO69French, 1681
 ISO70VideotexSupp1, 1681
 ISO84Portuguese2, 1681
 ISO85Spanish2, 1681
 ISO86Hungarian, 1681
 ISO87JISX0208, 1681
 ISO885913, 1682
 ISO885914, 1682
 ISO885915, 1682
 ISO885916, 1682
 ISO88596E, 1681
 ISO88596I, 1681
 ISO88598E, 1681
 ISO88598I, 1681
 ISO8859Supp, 1682
 ISO88Greek7, 1681
 ISO89ASM0449, 1681
 ISO90, 1681
 ISO91JISC62291984a, 1681
 ISO92JISC62991984b, 1681
 ISO93JIS62291984badd, 1681
 ISO94JIS62291984hand, 1681
 ISO95JIS62291984handadd, 1681
 ISO96JISC62291984kana, 1681
 ISO99NAPLPS, 1681
 ISOLatin1, 1680
 ISOLatin2, 1680
 ISOLatin3, 1680
 ISOLatin4, 1680
 ISOLatin5, 1680
 ISOLatin6, 1680
 ISOLatinArabic, 1680
 ISOLatinCyrillic, 1680
 ISOLatinGreek, 1680
 ISOLatinHebrew, 1680
 ISOTextComm, 1680
 JISEncoding, 1680
 KOI7switched, 1684
 KOI8R, 1684
 KOI8U, 1684
 KSC56011987, 1681
 KSC5636, 1682
 KZ1048, 1682
 literal, 1679
 Macintosh, 1683
 max_name_length, 1677
 mib, 1679
 MicrosoftPublishing, 1683
 Mnem, 1684
 Mnemonic, 1684
 name, 1679
 NATSSEFI, 1681
 NATSSEFIADD, 1681
 operator==, 1680
 OSDEBCDICDF03IRV, 1682
 OSDEBCDICDF041, 1682
 OSDEBCDICDF0415, 1682
 other, 1680
 PC775Baltic, 1684
 PC850Multilingual, 1682
 PC862LatinHebrew, 1682
 PC8CodePage437, 1682
 PC8DanishNorwegian, 1682
 PC8Turkish, 1683
 PCp852, 1682
 PTCP154, 1684
 SCSU, 1682
 ShiftJIS, 1680
 TIS620, 1684
 TSCII, 1684
 UCS2, 1682
 UCS4, 1682
 Unicode11, 1682
 Unicode11UTF7, 1682
 UnicodeASCII, 1682
 UnicodeIBM1261, 1682
 UnicodeIBM1264, 1682
 UnicodeIBM1265, 1682
 UnicodeIBM1268, 1682

- UnicodeIBM1276, 1682
- UnicodeJapanese, 1682
- UnicodeLatin1, 1682
- unknown, 1680
- Unknown8BiT, 1684
- USDK, 1682
- UTF16, 1682
- UTF16BE, 1682
- UTF16LE, 1682
- UTF32, 1682
- UTF32BE, 1682
- UTF32LE, 1682
- UTF7, 1682
- UTF7IMAP, 1682
- UTF8, 1682
- VenturaInternational, 1682
- VenturaMath, 1683
- VenturaUS, 1682
- VIQR, 1684
- VISCII, 1684
- windows1250, 1684
- windows1251, 1684
- windows1252, 1684
- windows1253, 1684
- windows1254, 1684
- windows1255, 1684
- windows1256, 1684
- windows1257, 1684
- windows1258, 1684
- Windows30Latin1, 1682
- Windows31J, 1683
- Windows31Latin1, 1682
- Windows31Latin2, 1682
- Windows31Latin5, 1682
- windows874, 1684
- text_encoding::aliases_view
 - begin, 1680
 - end, 1680
- text_file_busy
 - errc, 656
- tgamma, 1826
 - simd, 1928
- tgammaf, 1826
- tgamma, 1826
- then, 2302
- then_t, 2302
- this_thread
 - get_id, 2201
 - sleep_for, 2201
 - sleep_until, 2201
 - yield, 2201
- thousands_sep
 - moneypunct, 1673
 - numpunct, 1662
- thread, 2193
 - constructor, 2195, 2196
 - destructor, 2196
 - detach, 2197
 - get_id, 2197
 - hardware_concurrency, 2197
 - id, 2194
 - join, 2197
 - joinable, 2197
 - operator=, 2196
 - swap, 2197
- thread::id, 2194
 - constructor, 2195
 - hash, 2195
 - operator<<, 2195
 - operator<=, 2195
 - operator==, 2195
- thread::name_hint, 2193
 - constructor, 2194
- thread::stack_size_hint, 2194
 - constructor, 2194
- three_way_comparable, 624
- three_way_comparable_with, 625
- throw_with_nested
 - nested_exception, 616
- tie, 847
 - basic_ios, 2048
 - tuple, 847
- time, 2032
- TIME_ACTIVE, 2032
- time_base, 1664
 - dateorder, 1664
 - dmy, 1664
 - mdy, 1664
 - no_order, 1664
 - ydm, 1664
 - ymd, 1664
- time_get, 1664
 - char_type, 1664
 - date_order, 1665
 - do_date_order, 1666
 - do_get, 1667
 - do_get_date, 1667
 - do_get_monthname, 1667
 - do_get_time, 1667
 - do_get_weekday, 1667
 - do_get_year, 1667
 - get, 1666
 - get_date, 1665
 - get_monthname, 1666
 - get_time, 1665
 - get_weekday, 1666
 - get_year, 1666
 - id, 1665
 - iter_type, 1664
- time_get_byname, 1668
 - dateorder, 1668
 - iter_type, 1668
- TIME_MONOTONIC, 2032
- time_point, 1964
 - ceil, 1967
 - constructor, 1965
 - floor, 1967
 - hash, 2031

- max, 1966
- min, 1966
- operator+, 1966
- operator++, 1965
- operator+=", 1965
- operator-, 1966
- operator--, 1965
- operator-=, 1965
- operator<, 1966
- operator<=, 1966
- operator==, 1966
- operator>, 1966
- operator>=, 1966, 1967
- round, 1967
- time_point_cast, 1967
- time_since_epoch, 1965
- time_point_cast, 1967
- time_point, 1967
- time_put, 1668
 - char_type, 1668
 - do_put, 1669
 - id, 1668
 - iter_type, 1668
 - put, 1668
- time_put_byname, 1669
 - char_type, 1669
 - iter_type, 1669
- time_since_epoch
 - time_point, 1965
- time_t, 2032
- TIME_THREAD_ACTIVE, 2032
- TIME_UTC, 2032
- time_zone, 2015
 - get_info, 2015
 - name, 2015
 - operator<=>, 2016
 - operator==, 2016
 - to_local, 2016
 - to_sys, 2015, 2016
- time_zone_link, 2022
 - name, 2022
 - operator<=>, 2023
 - operator==, 2022
 - target, 2022
- timed_mutex, 2248
- timed_out
 - errc, 656
- timegm, 2032
- timeout
 - cv_status, 2262
 - future_status, 2275
- timespec, 2032
- timespec_get, 2032
- timespec_getres, 2032
- tinyness_before
 - numeric_limits, 593
- TIS620
 - text_encoding, 1684
- tm, 1747, 2032
- TMP_MAX, 2176
- tmpfile, 2176
- tmpnam, 2176
- to
 - ranges, 1283–1285
- to_address, 685
 - pointer_traits, 685
- to_array, 998
- to_bitset
 - basic_mask, 1938
- to_bytes
 - zombie, 573
- to_chars, 1635, 1636
- to_chars_result, 1634
 - ec, 1634
 - ptr, 1634
- to_duration
 - hh_mm_ss, 2009
- to_integer
 - byte, 583
- to_local
 - time_zone, 2016
- to_string, 1631
 - basic_stacktrace, 669
 - bitset, 915
- to_sys
 - time_zone, 2015, 2016
 - utc_clock, 1969
- to_time_t
 - system_clock, 1968
- to_ullong
 - basic_mask, 1938
 - bitset, 915
- to_ulong
 - bitset, 915
- to_underlying, 828
- to_utc
 - gps_clock, 1973
 - tai_clock, 1971
- to_wstring, 1632
- token
 - execution::counting_scope, 2380
- tolower, 1644, 1746
 - ctype, 1646
 - ctype<char>, 1649
- too_many_files_open
 - errc, 656
- too_many_files_open_in_system
 - errc, 656
- too_many_links
 - errc, 656
- too_many_symbolic_link_levels
 - errc, 656
- total_bits
 - member_offset, 805
- totally_ordered, 647
- totally_ordered_with, 648
- toupper, 1644, 1746
 - ctype, 1646

- ctype<char>, 1649
- towctrans, 1747
- tolower, 1747
- towupper, 1747
- traits_type
 - basic_string, 1607
 - basic_string_view, 1595
- transform, 1514
 - collate, 1663
 - expected, 901, 902
 - expected<void>, 909
 - optional, 862
 - regex_traits, 1723
 - views, 1308
- transform_error
 - expected, 902
 - expected<void>, 909
- transform_exclusive_scan, 1575
- transform_inclusive_scan, 1576
- transform_primary
 - regex_traits, 1723
- transform_reduce, 1571, 1572
- transform_sender, 2323
 - default_domain, 2323
 - indeterminate_domain, 2323
- transform_view, 1308
 - base, 1308
 - begin, 1309
 - constructor, 1309
 - end, 1309
 - size, 1308
- transform_view::iterator, 1309
 - base, 1311
 - constructor, 1311
 - iterator, 1311
 - operator+, 1312
 - operator++, 1311
 - operator+=", 1311
 - operator-, 1312
 - operator--, 1311
 - operator-=, 1311
 - operator<, 1312
 - operator<=, 1312
 - operator<=>, 1312
 - operator==, 1312
 - operator>, 1312
 - operator>=, 1312
- transform_view::sentinel, 1312
 - base, 1313
 - constructor, 1313
 - operator-, 1313
 - operator==, 1313
- translate
 - regex_traits, 1723
- translate_nocase
 - regex_traits, 1723
- transposed, 1864
- traps
 - numeric_limits, 593
- treat_as_floating_point, 1956
- treat_as_floating_point_v, 1941
- triangle_type
 - layout_blas_packed, 1853
- triangular_matrix_matrix_left_solve, 1887–1889
- triangular_matrix_matrix_right_solve, 1888–1890
- triangular_matrix_product, 1884
- triangular_matrix_right_product, 1884
- triangular_matrix_vector_product, 1875, 1876
- triangular_matrix_vector_solve, 1876, 1877
- trim_capacity
 - hive, 1017
- true_type, 751, 758
- truename
 - numpunct, 1662
- trunc, 1827
 - simd, 1925
- truncf, 1827
- truncl, 1827
- try_acquire
 - counting_semaphore, 2270
- try_acquire_for
 - counting_semaphore, 2271
- try_acquire_until
 - counting_semaphore, 2271
- try_associate
 - execution::simple_counting_scope::token, 2380
- try_emplace
 - flat_map, 1112
 - map, 1048, 1049
 - unordered_map, 1070, 1071
- try_emplace_back
 - inplace_vector, 1041
- try_lock, 2260
 - rcu_domain, 2290
 - shared_lock, 2259
 - unique_lock, 2256
- try_lock_for
 - shared_lock, 2259
 - unique_lock, 2256
- try_lock_until
 - shared_lock, 2259
 - unique_lock, 2256
- try_protect
 - hazard_pointer, 2293
- try_push_back
 - inplace_vector, 1041
- try_query, 2383
- try_to_lock, 2252
- try_to_lock_t, 2252
- try_wait
 - latch, 2272
- TSCII
 - text_encoding, 1684
- tuple, 838, 840, 999

- basic_common_reference, 851
- common_type, 851
- constructor, 842–844
- forward_as_tuple, 847
- get, 849, 850
- make_tuple, 847
- operator<=>, 850
- operator=, 845–847
- operator==, 850
- swap, 847, 852
- tie, 847
- tuple_cat, 848
- tuple_element, 750, 817, 837, 849, 999, 1758
- tuple_element_t, 839
- tuple_size, 750, 817, 837, 849, 999, 1758
 - in general, 849
- tuple_size_v, 839
- type
 - any, 887
 - file_status, 2155
- type_identity, 773
- type_identity_t, 755
- type_index, 609
 - constructor, 609
 - hash, 609
 - hash_code, 609
 - name, 609
 - operator<, 609
 - operator<=, 609
 - operator<=>, 609
 - operator==, 609
 - operator>, 609
 - operator>=, 609
- type_info, 146, 607
 - before, 608
 - hash_code, 608
 - name, 608
 - operator==, 607
- type_of, 792
- type_order, 627, 817
 - operator value_type, 627
 - operator(), 627
 - value, 627
 - value_type, 627
- type_order_v, 619
- tzdb, 2010
 - current_zone, 2010
 - locate_zone, 2010
- tzdb_list, 2010
 - begin, 2011
 - cbegin, 2011
 - cend, 2011
 - end, 2011
 - erase_after, 2011
 - front, 2011

U

- u16streampos, 2034
- u16string, 1606
 - hash, 1632
 - operator"s, 1632
 - path, 2144
- u16string_view, 1595
 - hash, 1603
 - operator"sv, 1603
- u32streampos, 2034
- u32string, 1606
 - hash, 1632
 - operator"s, 1632
 - path, 2144
- u32string_view, 1595
 - hash, 1603
 - operator"sv, 1603
- u8display_string_of, 791
- u8identifier_of, 791
- u8path, 2462
- u8string, 1606
 - hash, 1632
 - operator"s, 1632
 - path, 2144
- u8string_view, 1595
 - hash, 1603
 - operator"sv, 1603
- u8symbol_of, 789
- u8what
 - exception, 789
- UCHAR_MAX, 595
- UCHAR_WIDTH, 595
- UCS2
 - text_encoding, 1682
- UCS4
 - text_encoding, 1682
- uflow
 - basic_filebuf, 2116
 - basic_streambuf, 2060
- uint16_t, 597
- uint32_t, 597
- uint64_t, 597
- uint8_t, 597
- uint_fast16_t, 597
- uint_fast32_t, 597
- uint_fast64_t, 597
- uint_fast8_t, 597
- UINT_FASTN_MAX, 597
- uint_least16_t, 597
- uint_least32_t, 597
- uint_least64_t, 597
- uint_least8_t, 597
- UINT_LEASTN_MAX, 597
- UINT_MAX, 596
- UINT_WIDTH, 595
- UINTMAX_C, 597
- UINTMAX_MAX, 597
- uintmax_t, 597

UINTN_C, 597
 UINTN_MAX, 597
 UINTPTR_MAX, 597
 uintptr_t, 597
 ULLONG_MAX, 596
 ULLONG_WIDTH, 595
 ULONG_MAX, 596
 ULONG_WIDTH, 595
 unary_function
 zombie, 573
 unary_negate
 zombie, 573
 unary_transform_result, 1451
 uncaught_exception
 zombie, 573
 uncaught_exceptions, 516, 614
 unchecked
 access_context, 802
 unchecked_emplace_back
 inplace_vector, 1041
 unchecked_gather_from
 simd, 1922
 unchecked_load
 simd, 1917
 unchecked_push_back
 inplace_vector, 1041
 unchecked_scatter_to
 simd, 1922
 unchecked_store
 simd, 1919
 undeclare_no_pointers
 zombie, 573
 undeclare_reachable
 zombie, 573
 underflow
 basic_filebuf, 2116
 basic_streambuf, 2059
 basic_stringbuf, 2094
 underflow_error, 650, 653
 constructor, 653
 underlying_type, 774, 816
 underlying_type_t, 755
 unexpected, 889
 unexpected_t, 889
 unexpected, 889
 constructor, 890
 error, 890
 operator==, 890
 swap, 890
 unexpected_handler
 zombie, 573
 unexpected_type
 expected, 892
 expected<void>, 903
 unget
 basic_istream, 2072
 ungetc, 2176
 ungetwc, 1747
 unhandled_exception
 generator::promise_type, 1424
 task::promise_type, 2374
 unhandled_stopped
 task::promise_type, 2374
 with_awaitable_senders, 2366
 Unicode11
 text_encoding, 1682
 Unicode11UTF7
 text_encoding, 1682
 UnicodeASCII
 text_encoding, 1682
 UnicodeIBM1261
 text_encoding, 1682
 UnicodeIBM1264
 text_encoding, 1682
 UnicodeIBM1265
 text_encoding, 1682
 UnicodeIBM1268
 text_encoding, 1682
 UnicodeIBM1276
 text_encoding, 1682
 UnicodeJapanese
 text_encoding, 1682
 UnicodeLatin1
 text_encoding, 1682
 uniform_int_distribution, 1784
 a, 1785
 b, 1785
 constructor, 1785
 result_type, 1784
 uniform_random_bit_generator, 1764
 uniform_real_distribution, 1785
 a, 1786
 b, 1786
 constructor, 1786
 result_type, 1785
 uninit_element
 simd, 1895
 uninitialized_construct_using_allocator,
 690
 uninitialized_copy, 1583
 uninitialized_copy_n, 1583
 uninitialized_copy_n_result, 676
 uninitialized_copy_result, 676
 uninitialized_default_construct, 1581
 uninitialized_default_construct_n, 1582
 uninitialized_fill, 1585
 uninitialized_fill_n, 1585
 uninitialized_move, 1584
 uninitialized_move_n, 1584
 uninitialized_move_n_result, 677
 uninitialized_move_result, 677
 uninitialized_value_construct, 1582
 uninitialized_value_construct_n, 1582, 1583
 unique, 1524
 forward_list, 1010
 hive, 1019
 list, 1026
 local_info, 2014

- unique_copy, 1525
- unique_copy_result, 1460
- unique_lock, 2254
 - constructor, 2255
 - destructor, 2255
 - lock, 2256
 - mutex, 2257
 - operator bool, 2257
 - operator=, 2255
 - owns_lock, 2257
 - release, 2257
 - swap, 2257
 - try_lock, 2256
 - try_lock_for, 2256
 - try_lock_until, 2256
 - unlock, 2256
- unique_ptr, 698, 702, 710
 - constructor, 699, 700, 703, 704
 - destructor, 701
 - get, 702
 - get_deleter, 702
 - hash, 721
 - operator bool, 702
 - operator*, 701
 - operator->, 702
 - operator<, 705, 706
 - operator<<, 706
 - operator<=, 705, 706
 - operator<=>, 705, 706
 - operator=, 701, 704
 - operator==, 705
 - operator>, 705, 706
 - operator>=, 705, 706
 - operator[], 704
 - release, 702
 - reset, 702, 704
 - swap, 702
- unitbuf, 2052
- unknown
 - file_type, 2152
 - perms, 2154
 - text_encoding, 1680
- Unknown8BiT
 - text_encoding, 1684
- unlock
 - rcu_domain, 2290
 - shared_lock, 2260
 - unique_lock, 2256
- unordered
 - partial_ordering, 620
- unordered_map, 1063, 1064
 - allocator_type, 966
 - at, 1070
 - begin, 962, 993
 - bucket, 993
 - bucket_count, 993
 - bucket_size, 993
 - cbegin, 962, 994
 - cend, 962, 994
 - clear, 992
 - const_iterator, 961
 - const_local_iterator, 986
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 987, 1069
 - contains, 992
 - count, 992
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 989
 - emplace_hint, 989
 - empty, 963
 - end, 962, 994
 - equal_range, 993
 - erase, 991, 992
 - erase_if, 1072
 - extract, 991
 - find, 992
 - get_allocator, 966
 - hash_function, 989
 - hasher, 986
 - insert, 989, 990, 1070
 - insert_or_assign, 1071, 1072
 - insert_range, 990
 - iterator, 961
 - key_eq, 989
 - key_equal, 986
 - key_type, 986
 - load_factor, 994
 - local_iterator, 986
 - lookup, 1070
 - mapped_type, 986
 - max_bucket_count, 993
 - max_load_factor, 994
 - max_size, 963
 - merge, 991
 - node_type, 987
 - operator!=, 962
 - operator=, 961, 967
 - operator==, 962
 - operator[], 1070
 - rbegin, 964
 - reference, 961
 - rehash, 994
 - rend, 965
 - reserve, 994
 - reverse_iterator, 964
 - size, 963
 - size_type, 961
 - swap, 962, 967
 - try_emplace, 1070, 1071
 - value_type, 961, 986
- unordered_multimap, 1063, 1072, 1073
 - allocator_type, 966
 - begin, 962, 993
 - bucket, 993
 - bucket_count, 993

- bucket_size, 993
- cbegin, 962, 994
- cend, 962, 994
- clear, 992
- const_iterator, 961
- const_local_iterator, 986
- const_reference, 961
- const_reverse_iterator, 964
- constructor, 987, 1077
- contains, 992
- count, 992
- crbegin, 965
- crend, 965
- difference_type, 961
- emplace, 989
- emplace_hint, 989
- empty, 963
- end, 962, 994
- equal_range, 993
- erase, 991, 992
- erase_if, 1078
- extract, 991
- find, 992
- get_allocator, 966
- hash_function, 989
- hasher, 986
- insert, 989, 990, 1077
- insert_range, 990
- iterator, 961
- key_eq, 989
- key_equal, 986
- key_type, 986
- load_factor, 994
- local_iterator, 986
- mapped_type, 986
- max_bucket_count, 993
- max_load_factor, 994
- max_size, 963
- merge, 991
- node_type, 987
- operator!=, 962
- operator=, 961, 967
- operator==, 962
- rbegin, 964
- reference, 961
- rehash, 994
- rend, 965
- reserve, 994
- reverse_iterator, 964
- size, 963
- size_type, 961
- swap, 962, 967
- value_type, 961, 986
- unordered_multiset, 1078, 1084
 - allocator_type, 966
 - begin, 962, 993
 - bucket, 993
 - bucket_count, 993
 - bucket_size, 993
 - cbegin, 962, 994
 - cend, 962, 994
 - clear, 992
 - const_iterator, 961
 - const_local_iterator, 986
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 987, 1088
 - contains, 992
 - count, 992
 - crbegin, 965
 - crend, 965
 - difference_type, 961
 - emplace, 989
 - emplace_hint, 989
 - empty, 963
 - end, 962, 994
 - equal_range, 993
 - erase, 991, 992
 - erase_if, 1089
 - extract, 991
 - find, 992
 - get_allocator, 966
 - hash_function, 989
 - hasher, 986
 - insert, 989, 990
 - insert_range, 990
 - iterator, 961
 - key_eq, 989
 - key_equal, 986
 - key_type, 986
 - load_factor, 994
 - local_iterator, 986
 - mapped_type, 986
 - max_bucket_count, 993
 - max_load_factor, 994
 - max_size, 963
 - merge, 991
 - node_type, 987
 - operator!=, 962
 - operator=, 961, 967
 - operator==, 962
 - rbegin, 964
 - reference, 961
 - rehash, 994
 - rend, 965
 - reserve, 994
 - reverse_iterator, 964
 - size, 963
 - size_type, 961
 - swap, 962, 967
 - value_type, 961, 986
- unordered_set, 1078, 1079
 - allocator_type, 966
 - begin, 962, 993
 - bucket, 993
 - bucket_count, 993
 - bucket_size, 993
 - cbegin, 962, 994

- cend, 962, 994
- clear, 992
- const_iterator, 961
- const_local_iterator, 986
- const_reference, 961
- const_reverse_iterator, 964
- constructor, 987, 1083
- contains, 992
- count, 992
- crbegin, 965
- crend, 965
- difference_type, 961
- emplace, 989
- emplace_hint, 989
- empty, 963
- end, 962, 994
- equal_range, 993
- erase, 991, 992
- erase_if, 1084
- extract, 991
- find, 992
- get_allocator, 966
- hash_function, 989
- hasher, 986
- insert, 989, 990, 1084
- insert_range, 990
- iterator, 961
- key_eq, 989
- key_equal, 986
- key_type, 986
- load_factor, 994
- local_iterator, 986
- mapped_type, 986
- max_bucket_count, 993
- max_load_factor, 994
- max_size, 963
- merge, 991
- node_type, 987
- operator!=, 962
- operator=, 961, 967
- operator==, 962
- rbegin, 964
- reference, 961
- rehash, 994
- rend, 965
- reserve, 994
- reverse_iterator, 964
- size, 963
- size_type, 961
- swap, 962, 967
- value_type, 961, 986
- unprivileged
 - access_context, 802
- unreachable, 828
- unreachable_sentinel, 1198
- unreachable_sentinel_t, 1250
 - operator==, 1250
- unseq, 1432
 - execution, 1432
- unsetf
 - ios_base, 2043
- unshift
 - codecvt, 1651
- unsigned_integral, 642
- unsized
 - subrange_kind, 1261
- unstoppable, 2302
- unstoppable_token, 2186
- unsynchronized_pool_resource, 742
 - constructor, 743
 - destructor, 743
 - do_allocate, 744
 - do_deallocate, 744
 - do_is_equal, 744
 - options, 744
 - release, 743
 - upstream_resource, 743
- unwrap_ref_decay, 775, 816, 917
- unwrap_ref_decay_t, 755, 917
- unwrap_reference, 775, 816
- unwrap_reference_t, 755
- update_existing
 - copy_options, 2153
- upon_error, 2302
- upon_error_t, 2302
- upon_stopped, 2302
- upon_stopped_t, 2302
- upper
 - ctype_base, 1645
- upper_bound, 1540
 - ordered associative containers, 983
- upper_triangle, 1852
- upper_triangle_t, 1852
- uppercase, 2052
- upstream_resource
 - monotonic_buffer_resource, 745
 - synchronized_pool_resource, 743
 - unsynchronized_pool_resource, 743
- USDK
 - text_encoding, 1682
- use_count
 - shared_ptr, 711
 - weak_ptr, 719
- use_facet
 - locale, 1644
- uses_allocator, 687
- uses_allocator<tuple>, 851
- uses_allocator_construction_args, 687–689
- uses_allocator_v, 673
- USHRT_MAX, 596
- USHRT_WIDTH, 595
- utc_clock, 1969
 - from_sys, 1969
 - now, 1969
 - to_sys, 1969
- utc_seconds, 1941
- utc_time, 1941
 - from_stream, 1970

operator<<, 1970
 UTF16
 text_encoding, 1682
 UTF16BE
 text_encoding, 1682
 UTF16LE
 text_encoding, 1682
 UTF32
 text_encoding, 1682
 UTF32BE
 text_encoding, 1682
 UTF32LE
 text_encoding, 1682
 UTF7
 text_encoding, 1682
 UTF7IMAP
 text_encoding, 1682
 UTF8
 text_encoding, 1682

V

va_arg, 142, 180, 633
 va_copy, 633
 va_end, 573, 633
 va_list, 573, 633
 va_start, 180, 633
 valarray, 1808, 1821
 abs, 1817
 acos, 1817
 apply, 1814
 asin, 1817
 atan, 1817
 atan2, 1817
 begin, 1814
 constructor, 1809, 1810
 cos, 1817
 cosh, 1817
 cshift, 1814
 destructor, 1810
 end, 1814
 exp, 1817
 log, 1817
 log10, 1817
 max, 1814
 min, 1814
 operator!, 1813
 operator!=, 1816
 operator*, 1815
 operator*=, 1813
 operator+, 1813, 1815
 operator+=, 1813
 operator-, 1813, 1815
 operator-=, 1813
 operator/, 1815
 operator/=, 1813
 operator<, 1816
 operator<<, 1815
 operator<=, 1813

operator<=, 1816
 operator=, 1810
 operator==, 1816
 operator>, 1816
 operator>=, 1816
 operator>>, 1815
 operator>>=, 1813
 operator[], 1811, 1812
 operator%, 1815
 operator%=, 1813
 operator&, 1815
 operator&=, 1813
 operator&&, 1816
 operator~, 1815
 operator^=, 1813
 operator~, 1813
 operator|, 1815
 operator|=, 1813
 operator||, 1816
 pow, 1817
 resize, 1815
 shift, 1814
 sin, 1817
 sinh, 1817
 size, 1814
 sqrt, 1817
 sum, 1814
 swap, 1814, 1817
 tan, 1817
 tanh, 1817
 valid
 future, 2281
 packaged_task, 2287
 shared_future, 2283
 value
 error_code, 660
 error_condition, 662
 expected, 900
 expected<void>, 907
 in_value_result, 1488
 leap_second, 2021
 money_base, 1672
 optional, 861
 out_value_result, 1488
 regex_traits, 1724
 type_order, 627
 value_comp
 ordered associative containers, 979
 value_compare
 ordered associative containers, 977
 value_or
 expected, 900
 optional, 861
 value_too_large
 errc, 656
 value_type
 allocator, 692
 atomic, 2220
 atomic_ref, 2210

- basic_mask, 1935
- basic_string, 1607
- basic_string_view, 1595
- basic_vec, 1910
- complex, 1754
- containers, 961
- expected, 892
- expected<void>, 903
- gslice_array, 1820
- indirect_array, 1822
- integer_sequence, 750
- integral_constant, 758
- iterator_traits, 1204, 1205
- mask_array, 1821
- mdspan, 1175
- optional, 853
- ordered associative containers, 977
- path, 2137
- polymorphic_allocator, 738
- scoped_allocator_adaptor, 746
- slice_array, 1818
- type_order, 627
- unordered associative containers, 986
- value_types_of_t, 2359
- valueless_after_move
 - indirect, 731
 - polymorphic, 736
- valueless_by_exception
 - variant, 878
- values
 - views, 1265
- values_view, 1265
- variable_of, 799
- variant, 873
 - constructor, 874–876
 - destructor, 876
 - emplace, 878
 - get, 880
 - get_if, 880
 - hash, 883
 - holds_alternative, 880
 - index, 879
 - operator!=, 881
 - operator<, 881
 - operator<=, 881
 - operator<=>, 881
 - operator=, 876, 877
 - operator==, 881
 - operator>, 881
 - operator>=, 881
 - swap, 879, 883
 - valueless_by_exception, 878
 - visit, 882
- variant_alternative, 817, 879
- variant_alternative_t, 872
- variant_size, 817, 879
- variant_size_v, 871
- vec
 - simd, 1894
- vector, 1028
 - allocator_type, 966
 - assign, 970
 - assign_range, 970
 - begin, 962
 - capacity, 1031
 - cbegin, 962
 - cend, 962
 - clear, 970
 - const_iterator, 961
 - const_reference, 961
 - const_reverse_iterator, 964
 - constructor, 1030
 - crbegin, 965
 - crend, 965
 - data, 1032
 - difference_type, 961
 - emplace, 969
 - empty, 963
 - end, 962
 - erase, 970, 1032
 - erase_if, 1033
 - get_allocator, 966
 - insert, 969, 1032
 - insert_range, 969
 - iterator, 961
 - max_size, 963
 - operator!=, 962
 - operator<, 1030
 - operator<=>, 965
 - operator=, 961, 967
 - operator==, 962, 1030
 - rbegin, 964
 - reference, 961
 - rend, 965
 - reserve, 1031
 - resize, 1031
 - reverse_iterator, 964
 - shrink_to_fit, 1031
 - size, 963
 - size_type, 961
 - swap, 962, 967, 1031
 - value_type, 961
- vector<bool>, 1033
 - flip, 1035
 - reference, 1033
 - swap, 2460
- vector<bool>::reference
 - constructor, 1035
 - destructor, 1035
 - flip, 1035
 - operator bool, 1035
 - operator=, 1035
 - swap, 1035
- vector_abs_sum, 1870
- vector_idx_abs_max, 1870
- vector_two_norm, 1869
- VenturaInternational
 - text_encoding, 1682

- VenturaMath
 - text_encoding, 1683
- VenturaUS
 - text_encoding, 1682
- vformat, 1694
- vformat_to, 1695
- vfprintf, 2176
- vfscanf, 2176
- vfwprintf, 1747
- vfwscanf, 1747
- via
 - access_context, 802
- view, 1274
 - basic_istream, 2098
 - basic_ostringstream, 2101
 - basic_stringbuf, 2093
 - basic_stringstream, 2104
- view_interface, 1277
 - at, 1278
 - back, 1278
 - front, 1278
- viewable_range, 1276
- views
 - adjacent, 1375
 - adjacent_transform, 1380
 - all, 1301
 - as_rvalue, 1303
 - chunk, 1385
 - chunk_by, 1400
 - common, 1350
 - counted, 1350
 - drop, 1318
 - drop_while, 1320
 - elements, 1354
 - empty, 1262
 - enumerate, 1360
 - filter, 1304
 - indices, 1287
 - iota, 1287
 - join, 1321
 - join_with, 1326
 - keys, 1265
 - lazy_split, 1332
 - pairwise, 1266
 - pairwise_transform, 1266
 - repeat, 1293
 - reverse, 1351
 - single, 1285
 - slide, 1394
 - take, 1313
 - take_while, 1316
 - transform, 1308
 - values, 1265
 - zip, 1364
 - zip_transform, 1370
- VIQR
 - text_encoding, 1684
- VISCII
 - text_encoding, 1684
- visit, 882
 - basic_format_arg, 1709
 - variant, 882
- visit_format_arg, 2462
- void_pointer
 - allocator_traits, 691
 - scoped_allocator_adaptor, 746
- void_t, 755
- vprint_nonunicode, 2081, 2088
- vprint_nonunicode_buffered, 2088
- vprint_unicode, 2081, 2087, 2088
- vprint_unicode_buffered, 2087
- vprintf, 2176
- vscanf, 2176
- vsnprintf, 2176
- vsprintf, 2176
- vsscanf, 2176
- vswprintf, 1747
- vswscanf, 1747
- vwprintf, 1747
- vwsscanf, 1747

W

- wait
 - atomic, 2224
 - atomic<floating-point-type>, 2224
 - atomic<integral-type>, 2224
 - atomic<shared_ptr<T>>, 2237
 - atomic<T*>, 2224
 - atomic<weak_ptr<T>>, 2239
 - atomic_flag, 2240
 - atomic_ref<T>, 2213
 - barrier, 2274
 - condition_variable, 2264
 - condition_variable_any, 2267
 - future, 2281
 - latch, 2272
 - shared_future, 2283
- wait_for
 - condition_variable, 2265, 2266
 - condition_variable_any, 2268
 - future, 2281
 - shared_future, 2283
- wait_until
 - condition_variable, 2264, 2265
 - condition_variable_any, 2267, 2268
 - future, 2281
 - shared_future, 2284
- wbuffer_convert
 - zombie, 573
- wcerr, 2037
- WCHAR_MAX, 597, 1749
- WCHAR_MIN, 597, 1749
- WCHAR_WIDTH, 1749
- wcin, 2037
- wclog, 2038
- wcout, 2037
- wcrtomb, 1747, 1749

- wcscat, 1747
- wcschr, 1747
- wcscmp, 1747
- wcscoll, 1747
- wcscpy, 1747
- wcscspn, 1747
- wcsftime, 1747
- wcslen, 1747
- wcsncat, 1747
- wcsncmp, 1747
- wcsncpy, 1747
- wcspbrk, 1747
- wcsrchr, 1747
- wcsrtombs, 1747, 1750
- wcsspn, 1747
- wcsstr, 1747
- wcstod, 1747
- wcstof, 1747
- wcstoimax, 2177
- wcstok, 1747
- wcstol, 1747
- wcstold, 1747
- wcstoll, 1747
- wcstombs, 580, 1749
- wcstoul, 1747
- wcstoull, 1747
- wcstoumax, 2177
- wcsxfrm, 1747
- wctob, 1747
- wctomb, 580, 1749
- wctrans, 1747
- wctrans_t, 1747
- wctype, 1747
- wctype_t, 1747
- weak_from_this
 - enable_shared_from_this, 721
- weak_order, 626
- weak_ordering, 621
 - equivalent, 621
 - greater, 621
 - less, 621
 - operator partial_ordering, 622
 - operator<, 622
 - operator<=, 622
 - operator<=>, 622
 - operator==, 622
 - operator>, 622
 - operator>=, 622
- weak_ptr, 710, 717, 721
 - constructor, 718
 - destructor, 718
 - expired, 719
 - lock, 719
 - operator=, 718
 - owner_before, 719
 - owner_equal, 719
 - owner_hash, 719
 - reset, 718
 - swap, 718, 719
 - use_count, 719
- weakly_canonical, 2175
- weakly_incrementable, 1208
- weekday, 1984
 - c_encoding, 1986
 - constructor, 1985
 - from_stream, 1987
 - hash, 2031
 - iso_encoding, 1986
 - ok, 1986
 - operator+, 1986
 - operator++, 1985
 - operator+=, 1986
 - operator-, 1986
 - operator--, 1985
 - operator-=, 1986
 - operator<<, 1986
 - operator==, 1986
 - operator[], 1986
 - weekday_indexed, 1987
 - weekday_last, 1988
 - year_month_weekday, 2001
 - year_month_weekday_last, 2003
- weekday_indexed, 1987
 - constructor, 1987
 - hash, 2031
 - index, 1987
 - month_weekday, 1991
 - ok, 1987
 - operator<<, 1988
 - operator==, 1987
 - weekday, 1987
 - year_month_weekday, 2001
- weekday_last, 1988
 - constructor, 1988
 - hash, 2031
 - month_weekday_last, 1992
 - ok, 1988
 - operator<<, 1988
 - operator==, 1988
 - weekday, 1988
 - year_month_weekday_last, 2003
- weeks, 1941
- weibull_distribution, 1792
 - a, 1793
 - b, 1793
 - constructor, 1793
 - result_type, 1792
- WEOF, 1747, 1749
- wfilebuf, 2034, 2112
- wformat_args, 1684
- wformat_context, 1684, 1702
- wformat_parse_context, 1684
- wformat_string, 1685
- wfstream, 2034, 2112
- what
 - bad_alloc, 605
 - bad_any_cast, 884
 - bad_array_new_length, 605

- bad_cast, 608
- bad_exception, 613
- bad_expected_access, 891
- bad_function_call, 937
- bad_optional_access, 868
- bad_typeid, 608
- bad_variant_access, 883
- bad_weak_ptr, 706
- exception, 613, 788
- filesystem_error, 2151
- future_error, 2276
- system_error, 664
- when_all, 2302
- when_all_t, 2302
- when_all_with_variant, 2302
- when_all_with_variant_t, 2302
- where
 - exception, 789
- widen
 - basic_ios, 2049
 - ctype, 1646
 - ctype<char>, 1649
- width
 - ios_base, 1644, 2044
- wifstream, 2034, 2112
- windows1250
 - text_encoding, 1684
- windows1251
 - text_encoding, 1684
- windows1252
 - text_encoding, 1684
- windows1253
 - text_encoding, 1684
- windows1254
 - text_encoding, 1684
- windows1255
 - text_encoding, 1684
- windows1256
 - text_encoding, 1684
- windows1257
 - text_encoding, 1684
- windows1258
 - text_encoding, 1684
- Windows30Latin1
 - text_encoding, 1682
- Windows31J
 - text_encoding, 1683
- Windows31Latin1
 - text_encoding, 1682
- Windows31Latin2
 - text_encoding, 1682
- Windows31Latin5
 - text_encoding, 1682
- windows874
 - text_encoding, 1684
- WINT_MAX, 597
- WINT_MIN, 597
- wint_t, 1747
- wios, 2039
- wispanstream, 2104
- wistream, 2034, 2061
- wistream_view, 1262
- wistringstream, 2034, 2088
- with_awaitable_senders, 2365
 - await_transform, 2366
 - continuation, 2366
 - set_continuation, 2366
 - unhandled_stopped, 2366
- with_error, 2303
- wmemchr, 1747
- wmemcmp, 1747
- wmemcpy, 1747
- wmemmove, 1747
- wmemset, 1747
- wofstream, 2034, 2112
- wospanstream, 2104
- wostream, 2034, 2062
- wostringstream, 2034, 2088
- wosyncstream, 2034, 2124
- wprintf, 1747
- wrap
 - execution::counting_scope::token, 2381
 - execution::simple_counting_scope::token, 2380
- wregex, 1715
- write
 - basic_ostream, 2081
- write_env, 2302
- wrong_protocol_type
 - errc, 656
- ws, 2068, 2072
- wscanf, 1747
- wspanbuf, 2104
- wspanstream, 2104
- wstreambuf, 2034, 2053
- wstreampos, 2034
- wstring, 1606
 - hash, 1632
 - operator"s, 1632
 - path, 2144
- wstring_convert
 - zombie, 573
- wstring_view, 1595
 - hash, 1603
 - operator"sv, 1603
- wstringbuf, 2034, 2088
- wstringstream, 2034, 2088
- wsyncbuf, 2034, 2124

X

- xalloc
 - ios_base, 2044
- xdigit
 - ctype_base, 1645
- xsgetn
 - basic_streambuf, 2059
- xspn
 - basic_streambuf, 2060

Y

ydm

time_base, 1664

year, 1982

constructor, 1983

from_stream, 1984

hash, 2031

is_leap, 1983

max, 1984

min, 1984

ok, 1983

operator int, 1983

operator "y, 1984

operator+, 1983, 1984

operator++, 1983

operator+==, 1983

operator-, 1983, 1984

operator--, 1983

operator==, 1983

operator<<, 1984

operator<=>, 1984

operator==, 1984

year_month, 1993

year_month_day, 1995

year_month_day_last, 1998

year_month_weekday, 2001

year_month_weekday_last, 2003

year_month, 1992

constructor, 1992

from_stream, 1994

hash, 2031

month, 1993

ok, 1993

operator+, 1993, 1994

operator+==, 1993

operator-, 1993, 1994

operator==, 1993

operator<<, 1994

operator<=>, 1993

operator==, 1993

year, 1993

year_month_day, 1994

constructor, 1995

day, 1996

from_stream, 1997

hash, 2031

month, 1995

ok, 1996

operator local_days, 1996

operator sys_days, 1996

operator+, 1996, 1997

operator+==, 1995

operator-, 1996, 1997

operator==, 1995

operator<<, 1997

operator<=>, 1996

operator==, 1996

year, 1995

year_month_day_last, 1997

constructor, 1998

day, 1998

hash, 2031

month, 1998

month_day_last, 1998

ok, 1998

operator local_days, 1998

operator sys_days, 1998

operator+, 1999

operator+==, 1998

operator-, 1999

operator==, 1998

operator<<, 1999

operator<=>, 1999

operator==, 1998

year, 1998

year_month_weekday, 1999

constructor, 2000

hash, 2031

index, 2001

month, 2001

ok, 2001

operator local_days, 2001

operator sys_days, 2001

operator+, 2001

operator+==, 2000

operator-, 2001

operator==, 2000

operator<<, 2002

operator==, 2001

weekday, 2001

weekday_indexed, 2001

year, 2001

year_month_weekday_last, 2002

constructor, 2002

hash, 2031

month, 2003

ok, 2003

operator local_days, 2003

operator sys_days, 2003

operator+, 2003, 2004

operator+==, 2002, 2003

operator-, 2003, 2004

operator==, 2002, 2003

operator<<, 2004

operator==, 2003

weekday, 2003

weekday_last, 2003

year, 2003

years, 1941

yield

this_thread, 2201

yield_value

generator::promise_type, 1423

task::promise_type, 2374

ymd

time_base, 1664

yocto, 818, 820

yotta, 818, 820

default_zone, 2016

locate_zone, 2016

Z

zepto, 818, 820

zero

duration, 1960

duration_values, 1956

zero_element

simd, 1895

zetta, 818, 820

zip

views, 1364

zip_transform

views, 1370

zip_transform_view, 1370

begin, 1370

constructor, 1371

end, 1370

size, 1370

zip_transform_view::iterator, 1371

constructor, 1373

operator*, 1373

operator+, 1374

operator++, 1373

operator+=, 1373

operator-, 1374

operator--, 1373

operator==, 1373

operator[], 1373

zip_transform_view::sentinel, 1374

constructor, 1374

operator-, 1375

operator==, 1375

zip_view, 1364, 1365

begin, 1365

constructor, 1366

end, 1365

size, 1366

zip_view::iterator, 1366

constructor, 1367

zip_view::sentinel, 1369

constructor, 1369

operator-, 1370

operator==, 1370

zoned_time, 2016

constructor, 2018–2019

get_info, 2020

get_local_time, 2020

get_sys_time, 2020

get_time_zone, 2020

hash, 2032

operator local_time, 2020

operator sys_time, 2020

operator<<, 2020

operator=, 2019, 2020

operator==, 2020

zoned_traits, 2016

zoned_traits<const time_zone*>

Index of library concepts

The bold page number for each entry is the page where the concept is defined. Other page numbers refer to pages where the concept is mentioned in the general text. Concepts whose name appears in *italics* are for exposition only.

- advanceable*, 1287, 1288, **1288**, 1289–1292
- all-bidirectional*, **1301**, 1343, 1366–1368
- all-forward*, **1301**, 1343–1345, 1366, 1367
- all-random-access*, **1301**, 1343, 1345, 1348, 1366–1368, 1411, 1413
- always-true*, **2358**
- approximately_sized_range*, 968, 970, 1030, 1032, 1274, **1274**, 1284, 1301–1304, 1309, 1315, 1320, 1342, 1343, 1351, 1352, 1354, 1355, 1361, 1376, 1382, 1386, 1387, 1391, 1395, 1396, 1403, 1404, 1415, 1418
- assignable_from*, 639, 642, **642**, 643, 648, 970, 1210, 1220, 1223, 1226, 1239, 1241–1243, 1246, 1248, 1446
- await-suspend-result*, 2321, **2322**
- awaitable-sender*, **2363**
- bidirectional-common*, **1326**, 1329, 1331
- bidirectional_iterator*, 1194, 1196, 1200, 1212, **1212**, 1213, 1223, 1224, 1226, 1233–1235, 1238, 1247, 1248, 1270, 1275, 1280, 1281, 1426, 1447, 1449, 1450, 1460, 1461, 1471, 1473, 1485, 1486, 1508, 1511, 1513, 1526, 1527, 1532, 1543, 1547, 1564, 1565, 1581, 1735, 1737
- bidirectional_range*, 1265, 1275, **1275**, 1278, 1301, 1306, 1307, 1310, 1311, 1323–1326, 1329, 1331, 1352, 1356–1358, 1362, 1363, 1365, 1372, 1373, 1377, 1378, 1382, 1383, 1390–1393, 1395, 1397, 1398, 1400–1406, 1408, 1447, 1449, 1450, 1460, 1461, 1471, 1474, 1485, 1486, 1508, 1511, 1513, 1526, 1527, 1543, 1547, 1565
- boolean-testable*, 551, 562, 565, 624, 626, 627, 645, 646, **646**, 647, 649, 836, 850, 1214, 1217, 1218, 1427, 2180
- boolean-testable-impl*, 645, **645**, 646
- borrowed_range*, 1141, 1273, **1273**, 1279, 1280, 1282, 1680, 2109
- callable*, 920, **920**, 2309, 2313, 2317, 2360
- can-reference*, 1192, **1192**, 1193, 1204, 1210, 1244, 1263, 1266, 1308, 1309, 1312, 1370, 1372, 1374, 1381, 1382, 1384
- cartesian-is-sized-sentinel*, **1408**, 1411, 1413, 1414
- cartesian-product-common-arg*, 1408, **1408**
- cartesian-product-is-bidirectional*, **1408**, 1410–1412
- cartesian-product-is-common*, **1408**, 1409
- cartesian-product-is-random-access*, **1408**, 1410, 1411, 1413
- cartesian-product-is-sized*, **1408**, 1409, 1410
- class-type*, **2299**, 2302, 2303, 2365, 2383
- common_range*, 1265, 1276, **1276**, 1278, 1284, 1303, 1304, 1308, 1309, 1322–1329, 1333, 1334, 1339, 1343, 1350, 1352, 1353, 1355, 1360, 1365, 1368, 1371, 1376, 1381, 1390, 1391, 1395, 1396, 1401, 1403, 1408, 1417, 1680
- common_reference_with*, 639, 641, **641**, 642, 643, 1172, 1207, 1218, 1342, 1421, 2421
- common_with*, 639, **641**, 642, 1197, 1247, 1249, 1250
- compares-as*, 624, **624**, 625
- comparison-common-type-with*, 625, 647, **647**
- comparison-common-type-with-impl*, **646**, 647
- completion-signature*, 2303, 2358, **2358**, 2359
- concat-indirectly-readable*, 1341, 1342, **1342**
- concat-indirectly-readable-impl*, 1342, **1342**
- concat-is-bidirectional*, 1341, 1343, **1343**, 1344, 1345, 1347
- concat-is-random-access*, 1341, 1343, **1343**, 1344, 1345, 1347, 1348
- concatable*, 1326–1328, 1332, 1341, 1342, **1342**, 1343, 1344
- const-formattable-range*, 1686, **1686**
- constant-iterator*, 1197, 1232, **1232**, 1233, 1236, 1269, 1271, 1276
- constant_range*, 1276, **1276**, 1353
- constexpr-param*, **822**, 828–831, 948
- constexpr-wrapper-like*, **552**, 1138, 1912
- constructible_from*, 639, 644, **644**, 645, 676–679, 811, 1173, 1204, 1220, 1244, 1276, 1279, 1284, 1286, 1294, 1327, 1328, 1333, 1334, 1339, 1344, 1346, 1422, 1423, 1447, 1448, 1507, 1508, 1583–1585, 2185, 2189, 2192, 2296, 2309, 2320, 2336, 2348, 2364
- container-compatible-range*, **961**, 968, 977, 985, 1000–1003, 1005–1008, 1013–1015, 1018, 1021–1025, 1028–1030, 1032–1035, 1037–1040, 1044, 1045, 1047, 1051, 1052, 1054, 1056–1058, 1060–1062, 1065, 1066, 1069, 1073, 1074, 1077, 1080, 1081, 1083, 1085, 1086, 1089, 1091–1097, 1099–1101,

- 1104–1106, 1110, 1112, 1116–1118, 1121, 1124, 1125, 1128, 1129, 1131–1133, 1136, 1137, 1608–1610, 1614, 1619–1621, 1623
- contiguous_iterator*, 860, 866, 964, 1140–1142, 1144, 1194, 1200, 1201, 1213, **1213**, 1234, 1235, 1246–1248, 1272, 1276, 1278, 1350, 1598, 1813, 1894, 1895, 1917–1920
- contiguous_range*, 1141, 1142, 1275, 1276, **1276**, 1301, 1302, 1598, 1894–1896, 1912, 1913, 1917–1919, 1922, 1923
- convertible-to-non-slicing*, 1279, **1279**, 1280
- convertible_to*, 135, 462, 551, 552, 638, 641, **641**, 645, 646, 694, 924, 961, 1145, 1204, 1226, 1233, 1235, 1236, 1238, 1239, 1241–1243, 1246, 1248, 1279, 1280, 1284, 1288, 1302, 1305–1307, 1310–1313, 1316, 1318, 1323, 1325, 1326, 1329, 1331, 1332, 1334, 1335, 1342, 1344, 1346, 1350, 1355–1357, 1359, 1361, 1362, 1364, 1366, 1367, 1369, 1372–1374, 1377, 1378, 1380, 1382, 1383, 1385, 1391, 1392, 1397, 1398, 1404, 1406, 1410, 1412, 1419, 1422–1424, 1446, 1447, 1486–1489, 1694, 1912, 2109, 2363
- copy_constructible*, 571, 639, 645, **645**, 648, 787, 862, 1218, 1219, 1275, 1286, 1293, 1294, 1299, 1303, 1304, 1308, 1314, 1317, 1319, 1321, 1322, 1327, 1333, 1339, 1351–1353, 1355, 1361, 1375, 1381, 1386, 1390, 1395, 1400, 1403, 1415, 1418, 1421, 1446, 1451, 1452, 1456, 1492, 1493, 1514, 1515, 1520, 1545, 2321, 2338, 2340, 2346, 2347
- copyable*, 648, **648**, 695, 867, 947, 1150, 1172, 1178, 1198, 1204, 1220, 1242, 1261, 1262, 1265, 1275, 1280–1282, 1287, 1289, 1292, 1299, 1305, 1306, 1323, 1325, 1350, 1481, 1482, 1526, 1559, 1560, 1680, 2186, 2188, 2308, 2375
- cpp17-bidirectional-iterator*, 1204, **1204**, 1205
- cpp17-forward-iterator*, 1204, **1204**, 1205
- cpp17-input-iterator*, 1204, **1204**, 1205
- cpp17-iterator*, 1204, **1204**, 1205
- cpp17-random-access-iterator*, **1204**, 1205
- decays-to*, 2299, **2299**, 2317, 2318
- decrementable*, 1287, 1288, **1288**, 1290, 1291
- default_initializable*, 639, **645**, 648, 675, 676, 1233, 1237, 1242, 1246, 1279, 1286, 1287, 1290, 1293, 1296, 1297, 1299, 1302–1305, 1308, 1310, 1314, 1317, 1319, 1320, 1322, 1327, 1333, 1339, 1350, 1352, 1353, 1355, 1356, 1360, 1361, 1375, 1400, 1404, 1415, 1417, 1418, 1422, 1424, 1581–1583, 1680, 2350, 2375
- dependent_sender*, 2300, 2314, 2320, **2320**
- dereferenceable*, 1192, **1192**, 1193, 1242, 1244, 1246, 1248
- derived_from*, 638, 640, **640**, 1211–1213, 1225, 1238, 1243, 1275, 1278, 1284, 1296, 1298, 1306, 1311, 1324, 1329, 1330, 1337, 1345, 1357, 1373, 1383, 1405, 2305, 2308, 2309, 2311, 2319, 2324, 2329
- destructible*, 549, 639, 644, **644**, 679, 1586, 2189, 2192, 2296
- different-from*, 835, 843, 846, 847, 1233–1236, **1277**, 1279–1281, 1301, 1302
- enable-sender*, 2319, **2319**
- equality_comparable*, 571, 639, 647, **647**, 648, 695, 697, 1150, 1204, 1208, 1210, 1211, 1290, 1291, 1306, 1307, 1310, 1312, 1323, 1325, 1329, 1332, 1344, 1347, 1356, 1358, 1366, 1368, 1372, 1373, 1405, 1406, 1410, 1413, 2186, 2188, 2308
- equality_comparable_with*, 552, 647, **647**, 648, 929, 1242, 1245, 2421
- equivalence_relation*, 640, 649, **649**, 1219
- execution-policy*, 675–679, **1428**, 1431, 1433–1446, 1448–1468, 1470–1478, 1480–1485, 1489–1499, 1501, 1502, 1504–1507, 1509–1515, 1517–1529, 1531, 1533–1539, 1542–1544, 1546–1554, 1557–1563
- explicitly-convertible-to*, **1891**, 1912, 1913, 1918, 1920, 1922, 1923
- floating_point*, **642**, 1837, 1891
- formattable*, 1090, 1091, 1098, 1137, 1686, 1687, 1697, **1697**, 1704, 1705, 1711, 1712
- formattable-with*, 1697, **1697**, 1709, 1710
- forward_iterator*, 1193, 1200, 1201, 1212, **1212**, 1221, 1233, 1234, 1237, 1238, 1242, 1243, 1246, 1271–1275, 1280, 1281, 1329, 1426, 1434, 1437–1440, 1443–1446, 1460, 1462, 1467–1469, 1472, 1482–1484, 1490, 1495–1498, 1503–1506, 1525, 1526, 1528–1530, 1537–1541, 1545, 1561, 1562, 1581
- forward_range*, 968, 970, 1030, 1032, 1264, 1266, 1267, 1275, **1275**, 1277, 1278, 1301, 1304–1306, 1310, 1311, 1320–1340, 1350, 1356, 1357, 1360–1362, 1372, 1375, 1376, 1379–1382, 1384, 1390, 1391, 1395, 1396, 1399–1401, 1403–1405, 1407, 1409–1411, 1417, 1434, 1437–1440, 1443–1446, 1457, 1459, 1461–1463, 1467–1470, 1472, 1482–1484, 1491, 1495–1498, 1503–1505, 1507, 1521, 1524, 1528–1531, 1537, 1538, 1540–1542, 1545, 1561, 1562
- forwarding-query*, **2299**, 2336, 2341, 2365
- has-arrow*, **1276**, 1305, 1306, 1323, 1325
- has-as-awaitable*, 2322, **2322**
- has-completions*, **2309**, 2310

- has-elem-type*, 684, **684**
- has-member-element-type*, 1203, **1203**
- has-member-value-type*, **1202**, 1203
- has-query*, 2361, **2361**
- has-queryable-await-completion-adaptor*, **2364**
- has-tuple-element*, 1355, **1355**, 1356, 1359
- in-matrix*, 1838, 1841–1849, **1856**, 1857, 1858, 1871–1890
- in-object*, 1838, 1840, 1857, **1857**, 1867, 1868
- in-vector*, 1838, 1840–1845, **1856**, 1857, 1858, 1868–1870, 1873–1881
- incrementable, 1193, 1210, **1210**, 1212, 1288, 1290
- index-pair-like*, **1145**
- indirect_binary_predicate, **1219**, 1221, 1267, 1400, 1401, 1434, 1436, 1437, 1440, 1441, 1453–1455, 1457, 1458, 1490, 1494, 1495, 1498, 1499, 1516–1518, 1521–1523
- indirect_equivalence_relation, **1219**, 1443, 1459, 1460, 1503, 1524, 1525
- indirect_strict_weak_order, **1219**, 1221, 1466–1470, 1474, 1480–1485, 1536–1541, 1548, 1557–1563
- indirect_unary_predicate, **1218**, 1241, 1263, 1264, 1304, 1305, 1307, 1317, 1318, 1320, 1433, 1434, 1436–1438, 1441, 1449, 1453–1455, 1457–1459, 1470–1472, 1489, 1490, 1494, 1495, 1499, 1510, 1511, 1517, 1519, 1521–1523, 1542–1545
- indirectly-binary-left-foldable*, **1446**, 1447, 1448, 1507, 1508
- indirectly-binary-left-foldable-impl*, **1446**, 1447
- indirectly-binary-right-foldable*, 1447, **1447**, 1508
- indirectly-readable-impl*, 1207, **1207**
- indirectly_comparable, 1195, 1219, 1221, **1221**, 1264, 1333, 1334, 1336, 1338, 1340, 1341, 1434, 1435, 1438, 1439, 1442–1446, 1490, 1491, 1496, 1497, 1500–1502, 1504–1507
- indirectly_copyable, 1194, 1219, 1220, **1220**, 1221, 1448, 1449, 1454, 1455, 1458–1462, 1466, 1471, 1472, 1509–1511, 1518, 1519, 1522, 1523, 1525, 1527–1530, 1536, 1544
- indirectly_copyable_storable, 1220, **1220**, 1460, 1481, 1482, 1525, 1559, 1560
- indirectly_movable, 1194, 1219, 1220, **1220**, 1241, 1450, 1512, 1513
- indirectly_movable_storable, 1207, 1220, **1220**, 1221
- indirectly_readable, 1193, 1194, 1197, 1202, 1203, 1207, **1207**, 1208, 1211, 1218–1220, 1232, 1244, 1246, 1421, 1446
- indirectly_regular_unary_invocable, 1194, **1218**, 1219
- indirectly_swappable, 1194, 1219, 1220, **1220**, 1221, 1225, 1228, 1238, 1240, 1243, 1245, 1247, 1250, 1306, 1307, 1324, 1325, 1329, 1337, 1338, 1350, 1367, 1369, 1377, 1379, 1389, 1390, 1405, 1407, 1411, 1414, 1416, 1419, 1420, 1451, 1513, 1514
- indirectly_unary_invocable, **1218**, 1435, 1491–1493
- indirectly_writable, 1193, 1208, **1208**, 1211, 1219, 1220, 1451–1456, 1514–1520, 1569, 1578, 1920, 1923
- infallible-scheduler*, 2309, **2309**, 2367, 2368
- inlinable_receiver, 2300, 2310, **2310**
- inout-matrix*, 1838, 1847, 1849, 1857, **1857**, 1884, 1885, 1889, 1890
- inout-object*, 1838, 1840, 1857, **1857**, 1867
- inout-vector*, 1838–1840, 1842, 1843, **1856**, 1857, 1867, 1875, 1877
- input_iterator, 676, 677, 1193, 1197, 1198, 1200, 1205, 1211, **1211**, 1212, 1221, 1232, 1238, 1241, 1243, 1245, 1247, 1250, 1269, 1271, 1275, 1276, 1421, 1433–1437, 1439–1443, 1445–1455, 1458, 1460, 1462, 1466, 1470, 1471, 1473–1478, 1484, 1489–1491, 1493, 1494, 1497, 1499, 1500, 1502, 1506–1510, 1512–1519, 1522, 1523, 1525, 1529, 1536, 1542, 1544, 1546, 1548, 1549, 1551, 1552, 1554, 1563, 1580, 1583, 1584
- input_or_output_iterator, 1193, 1195, 1196, 1198, 1200, 1210, **1210**, 1211, 1223, 1224, 1242, 1246, 1261, 1268–1271, 1273, 1279, 1280, 1456, 1520, 1569, 1578
- input_range, 676, 677, 780, 786, 787, 809, 961, 968, 970, 1001, 1006, 1015, 1023, 1030, 1042, 1047, 1053, 1058, 1062, 1068, 1069, 1076, 1082, 1083, 1087, 1088, 1092, 1095, 1099, 1100, 1108, 1120, 1127, 1134, 1261, 1263–1267, 1275, **1275**, 1276, 1277, 1282–1284, 1296, 1303–1305, 1307–1309, 1312, 1317, 1318, 1320–1323, 1326–1328, 1332–1334, 1336, 1341, 1344, 1353, 1355, 1356, 1359, 1365, 1366, 1369–1371, 1374, 1386–1389, 1403, 1404, 1409, 1410, 1414–1418, 1421–1423, 1433–1437, 1439–1443, 1445–1455, 1458, 1460, 1462, 1466, 1470, 1471, 1473–1478, 1481, 1482, 1485, 1489, 1490, 1492, 1494, 1497, 1499, 1501, 1502, 1506–1510, 1512, 1513, 1515, 1517–1519, 1523, 1525, 1530, 1536, 1542, 1544, 1546, 1548, 1549, 1551, 1552, 1554, 1559, 1560, 1563, 1583, 1584, 1612, 1686, 1687, 1703–1708
- integer-like-with-usable-difference-type*, 1293, **1293**, 1294
- integral, 391, 392, 642, **642**, 954, 1202, 1208, 1891, 1921, 1932, 1933, 2338
- integral-constant-like*, 1138, **1138**, 1183
- invocable, 554, 640, 649, **649**, 862, 867, 1194, 1218, 1446, 1456, 1520, 1587, 1761, 1764,

- 2185, 2189, 2192, 2334–2336, 2339, 2340, 2348
- is-awaitable*, 2319, 2321, **2321**, 2322, 2325, 2365
- is-awaiter*, 2321, **2321**, 2365
- is-constant*, **2319**, 2320
- is-derived-from-optional*, **852**, 853, 871
- is-extent-slice*, 1183, **1183**, 1184
- is-range-slice*, 1183, **1183**, 1184
- is-sender*, 2319, **2319**
- layout-mapping-alike*, 1159, **1159**, 1160, 1161
- math-floating-point*, **1892**, 1897–1903, 1925–1932
- mergeable, 1195, 1219, 1221, **1221**, 1473, 1475–1478, 1546, 1549–1554
- movable, 648, **648**, 1208, 1220, 1262, 1274, 1276, 1296, 1297, 1299, 1302, 1446, 2375
- movable-value*, **2296**, 2299, 2314, 2327, 2334, 2335, 2345
- move_constructible, 639, 643, 645, **645**, 648, 862, 1244, 1262, 1263, 1266, 1277, 1286, 1293, 1294, 1299, 1308, 1309, 1312, 1355, 1370, 1371, 1374, 1381, 1382, 1384, 1511, 1524, 2296, 2309, 2320, 2324
- not-a-const-iterator*, 1232–1234, **1234**, 1236, 1237
- nothrow-bidirectional-iterator*, 674, 1581, **1581**
- nothrow-bidirectional-range*, 674, 1581, **1581**
- nothrow-callable*, **920**, 2313, 2315, 2348, 2377
- nothrow-forward-iterator*, 674–678, 1581, **1581**, 1582–1585
- nothrow-forward-range*, 674–678, 1581, **1581**, 1582–1585
- nothrow-input-iterator*, 674, 679, 1580, **1580**, 1581, 1586
- nothrow-input-range*, 674, 679, 1580, **1580**, 1581, 1586
- nothrow-random-access-iterator*, 674–679, **1581**, **1581**
- nothrow-random-access-range*, 674, 1581, **1581**
- nothrow-sentinel-for*, 674–679, 1580, **1580**, 1581–1586
- nothrow-sized-random-access-range*, 674–679, **1581**, **1581**
- nothrow-sized-sentinel-for*, 674–679, 1580, **1580**, 1581
- operation_state, 2300, 2311, **2311**, 2326
- out-matrix*, 1838, 1843, 1845–1849, **1856**, 1857, 1877, 1878, 1881–1884, 1887–1889
- out-object*, 1838, 1840, 1857, **1857**, 1867, 1868
- out-vector*, 1838, 1841–1843, **1856**, 1857, 1873–1877
- output_iterator, 1193, 1200, 1211, **1211**, 1212, 1275, 1454–1456, 1518–1520, 1587, 1695, 1697, 1702, 1761
- output_range, 1275, **1275**, 1456, 1520, 1569, 1578, 1586, 1587, 1761
- pair-like*, 674, 689, 832, 833, 835, **838**, 1145, 1279
- pair-like-convertible-from*, 1279, **1279**, 1281
- partially-ordered-with*, 624, **624**, 625, 647, 648
- permutable, 1195, 1219, 1221, **1221**, 1457, 1459–1461, 1463, 1470, 1471, 1521, 1522, 1524, 1526–1528, 1530, 1531, 1542, 1543
- possibly-packed-out-matrix*, 1838, 1844, 1845, 1847, 1848, 1857, **1857**, 1879–1881, 1885–1887
- predicate, 640, 649, **649**, 1218, 1219
- queryable*, 2296, **2296**, 2299, 2301, 2303, 2305, 2308, 2309, 2312, 2313, 2320, 2322, 2323, 2329, 2336, 2341, 2349, 2357, 2360, 2381
- random_access_iterator, 668, 677, 678, 1102, 1114, 1122, 1130, 1194, 1200, 1211–1213, **1213**, 1223, 1225, 1226, 1233–1238, 1247–1249, 1273, 1275, 1350, 1427, 1433–1446, 1448–1468, 1470–1481, 1483–1485, 1489–1499, 1501, 1502, 1504–1507, 1509, 1510, 1512, 1513, 1515, 1517–1531, 1533–1538, 1542–1544, 1546–1549, 1551, 1552, 1554–1558, 1561–1563, 1581
- random_access_range, 1275, **1275**, 1276, 1278, 1301, 1310–1315, 1318–1320, 1351, 1356–1359, 1362, 1363, 1365, 1372–1374, 1377–1379, 1382–1384, 1392–1395, 1397–1399, 1404–1408, 1463–1466, 1468, 1479–1481, 1530, 1533–1536, 1539, 1555–1558, 1680
- range, 1195, 1224, 1260, 1261, 1263, 1273, **1273**, 1274–1276, 1278, 1279, 1282, 1298, 1301–1303, 1305, 1308, 1309, 1314, 1315, 1317, 1319–1321, 1339, 1342, 1343, 1351, 1353–1355, 1365, 1371, 1376, 1381, 1396, 1401, 1403, 1409, 1418, 1428, 1580
- range-with-movable-references*, **1277**, 1360, 1361, 1363
- receiver, 2300, 2309, **2309**, 2310, 2317, 2325, 2327, 2350, 2351, 2367, 2368, 2370–2372
- receiver-of*, 2300, 2309, **2310**, 2320, 2326, 2327, 2362, 2367
- reduction-binary-operation*, 1892, **1892**, 1916
- ref-wrap-common-reference-exists-with*, **924**, **924**
- reflection_range, 783–785, 809, **809**, 812–816

- regular, 648, **648**, 665, 1147, 1153, 1156, 1159, 1162, 1168, 1209, 1210, 1854
- regular_invocable, 640, 649, **649**, 1218, 1263, 1266, 1308, 1309, 1312, 1370–1372, 1374, 1381, 1382, 1384, 1429
- relation, 640, 649, **649**
- returnable_element, 1355, **1355**, 1356, 1359
- same-as-impl, 640, **640**
- same_as, 137, 571, 624, 638, 640, **640**, 641, 642, 646, 809, 1057, 1061, 1081, 1086, 1126, 1133, 1159, 1198, 1203, 1204, 1207, 1208, 1210–1213, 1223, 1232–1236, 1242, 1247, 1261, 1262, 1276, 1277, 1282, 1283, 1287–1289, 1293, 1294, 1303, 1422–1424, 1460, 1525, 1580, 1687, 1697, 1703–1705, 1708, 1764, 1891, 1892, 1894, 1905, 1916–1918, 1922, 1935–1937, 1939, 1940, 2185, 2186, 2193, 2296, 2299, 2308–2310, 2315, 2321, 2326, 2331, 2334, 2339, 2343–2345, 2351, 2355, 2356, 2359, 2366, 2368, 2373–2375
- scalar, 1838, 1840, 1841, 1844, 1847, 1851, **1857**, 1867–1872, 1879, 1885, 1886
- scheduler, 2300, 2302, 2306–2308, **2308**, 2309, 2330, 2333, 2362, 2366–2368, 2371, 2381
- scope_association, 2304, 2375, **2375**, 2378
- scope_token, 2304, 2346, 2347, 2349–2351, 2357, 2375, **2375**, 2376
- semiregular, 554, 610, 648, **648**, 1173, 1174, 1197, 1198, 1210, 1232, 1241, 1262, 1287, 1289, 1292–1294, 1697, 1857, 1859, 2112, 2314, 2315
- sender, 2300, 2301, 2307, 2308, 2314, 2319, 2320, **2320**, 2321–2324, 2327–2330, 2332–2336, 2338, 2341, 2344–2351, 2357, 2360, 2362, 2364, 2366, 2367, 2369–2371, 2373, 2374, 2376, 2378–2381
- sender_for, 2321, **2321**, 2330, 2333, 2338, 2344, 2345, 2366, 2370, 2381
- sender-in-of, 2308, 2320, **2320**
- sender-in-of-impl, 2320, **2320**, 2321
- sender-of, 2320, **2321**
- sender-to, 2301, 2311, 2319, **2320**, 2355, 2363
- sender_in, 2300, 2301, 2304, 2308, 2314, 2319, 2320, **2320**, 2325, 2327, 2330–2333, 2336, 2345, 2354–2356, 2359, 2365, 2370, 2375, 2376
- sentinel_for, 676, 677, 1193, 1195, 1196, 1198, 1210, **1210**, 1211, 1212, 1223, 1224, 1233, 1235, 1238, 1239, 1241, 1242, 1245, 1261, 1269–1271, 1273, 1279, 1280, 1307, 1308, 1313, 1316, 1318, 1326, 1332, 1359, 1364, 1369, 1370, 1374, 1375, 1380, 1385, 1433–1480, 1482–1486, 1489–1491, 1494–1500, 1502–1531, 1533, 1534, 1536–1549, 1551, 1552, 1554–1558, 1561–1565, 1569, 1578, 1580, 1583, 1584, 1587, 1761
- signed_integral, 642, **642**, 1204, 1208, 1898, 1928
- simd_complex, **1892**, 1904, 1913, 1934, 1935
- simd-floating-point, **1891**, 1892, 1904, 1935
- simd_integral, **1891**, 1896, 1903, 1904, 1910, 1913, 1921–1923, 1932–1935, 1937
- simd-mask-type, **1891**, 1896, 1920, 1921
- simd-vec-type, 1891, **1891**, 1892, 1895, 1896, 1903, 1920–1923, 1932–1934
- simple_allocator, 571, **571**, 2305
- simple-view, **1276**, 1303, 1314, 1315, 1317, 1319, 1320, 1322, 1327, 1333, 1342, 1343, 1351, 1354, 1355, 1360, 1365, 1375, 1376, 1390, 1395, 1396, 1403, 1409, 1418
- single_sender, 2301, 2304, **2304**, 2363
- sized-random-access-range, 677, 678, **1261**, 1276, **1276**, 1433–1446, 1448–1468, 1470–1478, 1480–1485, 1489–1492, 1494–1499, 1501, 1502, 1504, 1506, 1507, 1509, 1511, 1512, 1514, 1515, 1517–1525, 1527–1529, 1531, 1533–1539, 1542–1544, 1546–1548, 1550, 1551, 1553, 1554, 1557–1563
- sized_range, 968, 970, 1141, 1224, 1260, 1273, 1274, **1274**, 1276–1280, 1283, 1301–1303, 1309, 1313–1315, 1318–1320, 1333, 1342, 1343, 1349, 1351, 1352, 1354, 1355, 1360, 1361, 1365, 1366, 1371, 1376, 1382, 1386, 1387, 1390, 1391, 1395, 1396, 1403, 1404, 1408, 1415, 1418, 1446, 1502, 1503, 1507, 1581, 1587, 1598, 1894–1896, 1912, 1913, 1917–1919, 1922, 1923
- sized_sentinel_for, 677, 678, 1141, 1193, 1195, 1197, 1198, 1211, **1211**, 1213, 1223, 1224, 1234, 1237, 1238, 1240, 1242, 1245, 1261, 1271, 1278–1280, 1289, 1293, 1310, 1312–1315, 1349, 1357, 1359, 1360, 1364, 1367–1370, 1372, 1374, 1375, 1377, 1379, 1380, 1383–1385, 1387–1390, 1392, 1394, 1397, 1399, 1400, 1405, 1407, 1408, 1417, 1419, 1433–1446, 1448–1468, 1470–1478, 1480, 1481, 1483–1485, 1489–1492, 1494–1499, 1501–1507, 1509, 1510, 1512, 1513, 1515, 1517–1529, 1531, 1533–1538, 1542–1544, 1546–1549, 1551, 1552, 1554, 1557, 1558, 1561–1563, 1580, 1598, 1894, 1895, 1918–1920
- sliceable_mapping, 1186, **1186**, 1190
- slide-caches-first, 1395, **1395**, 1396–1399
- slide-caches-last, 1395, **1395**, 1396
- slide-caches-nothing, 1395, **1395**, 1396, 1398
- sortable, 1195, 1219, 1221, **1221**, 1464–1466, 1468, 1473, 1474, 1479, 1480, 1485, 1486, 1533–1536, 1538, 1539, 1547, 1555–1557, 1564, 1565
- stoppable_callback_for, 2183–2185, **2185**, 2186, 2189, 2192, 2348
- stoppable_source, 2183, 2184, 2186, **2186**, 2187, 2188, 2190, 2350, 2371

stoppable_token, 2183, 2184, 2186, **2186**, 2187,
 2190, 2305, 2348, 2383
stream-extractable, 1296, **1296**, 1297
 strict_weak_order, 640, 649, **649**, 1219
 swappable, 639, **643**, 644, 648, 2188
 swappable_with, **643**, 644, 1207, 1350

tagging-compatible-pointee, **694**, 695, 696
 three_way_comparable, 619, 624, **624**, 625, 665,
 681, 695, 697, 706, 867, 873, 881, 965,
 1013, 1090, 1093, 1098, 1101, 1209, 1233,
 1236, 1290, 1292, 1310, 1312, 1345, 1348,
 1357, 1358, 1377, 1379, 1383, 1384, 1392,
 1394, 1397, 1399, 1405, 1407, 1961
 three_way_comparable_with, 552, 625, **625**, 681,
 705, 853, 868, 871, 928, 1196, 1198, 1228,
 1234, 1236, 1240, 1943, 1952, 1967, 2022
tiny-range, 1264, 1333, **1333**, 1334–1336, 1338
 totally_ordered, 549, 625, 639, **647**, 648, 1204,
 1213, 1288, 1290–1292, 1917, 1924, 1925
 totally_ordered_with, 625, 648, **648**, 929, 930,
 1233, 1234, 1236, 1289
tuple-like, 770, 775, 838, 839, **839**, 840, 841,
 843, 844, 846–848, 850, 851, 1355

 uniform_random_bit_generator, 1462, 1463,
 1529, 1530, 1586, 1587, 1761, 1764, **1764**
 unsigned_integral, 642, **642**, 1209, 1764, 1935,
 1937
 unstoppable_token, 2184, 2186, **2186**, 2189,
 2309, 2348, 2369
uses-nonqualification-pointer-conversion,
 1279, **1279**

valid-completion-for, 2309, **2309**
valid-completion-signatures, 2301, 2303,
2304, 2324, 2359
valid-specialization, 2314, **2314**, 2315, 2316
 view, 1261, 1263–1267, 1273, 1274, **1274**, 1275,
 1276, 1278, 1279, 1283, 1285, 1301,
 1303–1305, 1307–1309, 1312–1314,
 1316–1321, 1323, 1326, 1328, 1332–1334,
 1336, 1338, 1339, 1341, 1344, 1350, 1352,
 1353, 1355, 1356, 1359–1361, 1363, 1365,
 1366, 1369–1371, 1374–1376, 1379, 1381,
 1382, 1384, 1386–1391, 1395, 1396,
 1399–1401, 1403, 1404, 1409, 1410,
 1414–1418, 1421, 1680
 viewable_range, 1262, 1276, **1276**, 1299

weakly-equality-comparable-with, 624, 625,
 647, **647**, 1210, 1262, 1287, 1289, 1292
 weakly_incrementable, 1193, 1202, 1208, **1208**,
 1209, 1210, 1219, 1221, 1241, 1250, 1262,
 1287–1289, 1292, 1293, 1448–1452, 1458,
 1460–1462, 1471, 1473, 1475–1478, 1509,
 1510, 1512, 1514, 1515, 1522, 1523, 1525,
 1527–1530, 1544, 1546, 1549, 1551, 1552,
 1554, 1569, 1578

Index of implementation-defined behavior

The entries in this index are rough descriptions; exact specifications are at the indicated page in the general text.

`#pragma`, 543

additional execution policies supported by parallel algorithms, 1431, 1432

additional `file_type` enumerators for file systems supporting additional types of file, 2152

additional formats for `time_get::do_get_date`, 1667

additional supported forms of preprocessing directive, 525

algorithms for producing the standard random number distributions, 1784

alignment, 68

alignment additional values, 69

alignment of bit-fields within a class object, 338

allocation of bit-fields within a class object, 338

argument values to construct

`ios_base::failure`, 2050

assignability of placeholder objects, 936

barrier phrase completion without `wait`, 2273

behavior of any function in `std::meta` for implementation-specific constructs, 786

behavior of `iostream` classes when `traits::pos_type` is not `streampos` or when `traits::off_type` is not `streamoff`, 2033

behavior of non-standard attributes, 293

bits in a byte, 64

character encoding scheme of the environment, 1679

choice of larger or smaller value of *floating-point-literal*, 27

code unit sequence for *conditional-escape-sequence*, 29

column value of `source_location::current`, 611

conversions between pointers and integers, 149

conversions of `basic_mask` from/to implementation-specific vector types, 1937

conversions of `basic_vec` from/to implementation-specific vector types, 1912

converting function pointer to object pointer and vice versa, 150

default ABI tag for `basic_vec` and `basic_mask`, 1893

default configuration of a pool, 744

default `next_buffer_size` for a `monotonic_buffer_resource`, 745

default number of buckets in `unordered_map`, 1069

default number of buckets in `unordered_multimap`, 1077

default number of buckets in `unordered_multiset`, 1088, 1089

default number of buckets in `unordered_set`, 1083

default semantics of `is_debugger_present`, 671

default value for `least_max_value` template parameter of `counting_semaphore`, 2269

defining `main` in freestanding environment, 98

definition and meaning of `__STDC__`, 546, 2453

definition and meaning of `__STDC_VERSION__`, 546

definition of `NULL`, 582, 2454

derived type for `typeid`, 146

determination of kind of input file, 13

determination of places and identification of headers during header search, 528

determination of places and identification of resources during bracket resource search, 530

diagnostic message, 4

dynamic initialization of static inline variables before `main`, 100

dynamic initialization of static variables before `main`, 100

dynamic initialization of thread-local variables before entry, 100

effect of calling associated Laguerre polynomials with `n` $\geq$ 128 or `m` $\geq$ 128, 1832

effect of calling associated Legendre polynomials with `l` $\geq$ 128, 1832

effect of calling `basic_filebuf::setbuf` with nonzero arguments, 2117

effect of calling `basic_filebuf::sync` when a get area exists, 2118

effect of calling `basic_streambuf::setbuf` with nonzero arguments, 2095

effect of calling cylindrical Bessel functions of the first kind with `nu` $\geq$ 128, 1833

effect of calling cylindrical Neumann functions with `nu` $\geq$ 128, 1834

effect of calling Hermite polynomials with `n` $\geq$ 128, 1835

- effect of calling `ios_base::sync_with_stdio` after any input or output operation on standard streams, 2044
- effect of calling irregular modified cylindrical Bessel functions with `nu >= 128`, 1833
- effect of calling Laguerre polynomials with `n >= 128`, 1835
- effect of calling Legendre polynomials with `l >= 128`, 1835
- effect of calling regular modified cylindrical Bessel functions with `nu >= 128`, 1833
- effect of calling spherical associated Legendre functions with `l >= 128`, 1836
- effect of calling spherical Bessel functions with `n >= 128`, 1836
- effect of calling spherical Neumann functions with `n >= 128`, 1836
- effect of *conditional-escape-sequence* on encoding state, 29
- effect of `filesystem::copy`, 2165
- effect on C locale of calling `locale::global`, 1643
- effects of invalid `hive` limits, 1012
- encoding assumption for `format` width computation, 1691
- encoding of a error message during formatting, 661
- `error_category` for errors originating outside the operating system, 578
- evaluation semantic used for the evaluation of a contract assertion, 102
- exception type when `random_device` constructor fails, 1781
- exception type when `random_device::operator()` fails, 1781
- exception type when `shared_ptr` constructor fails, 709
- exceptions thrown by standard library functions that have a potentially-throwing exception specification, 578
- exit status, 600
- extended signed integer types, 82
- file type of the file argument of `filesystem::status`, 2175
- floating-point conversion subrank, 90
- format of diagnostic created by `assert` macro's expression, 653
- formatted character sequence generated by `time_put::do_put` in C locale, 1669
- forward progress guarantees for implicit threads of parallel algorithms (if not defined for thread), 1430
- growth factor for `monotonic_buffer_resource`, 745
- growth factor of `hive` active block capacities, 1012
- headers for freestanding implementation, 559
- how `random_device::operator()` generates values, 1781
- how `text_encoding` objects are determined to be representative of a character encoding scheme, 1678
- how the set of importable headers is determined, 306
- interactive device, 12
- interpretation of the path character sequence with format `path::auto_format`, 2152
- invalid pointer value in the context of an evaluation, 88
- largest supported value to configure the largest allocation satisfied directly by a pool, 743
- largest supported value to configure the maximum number of blocks to replenish a pool, 743
- last enumerator of `launch`, 2275
- linkage of entities between C++ and other languages, 292
- linkage of `main`, 98
- linkage of names from C standard library, 561
- locale names, 1642
- locales with Unicode support for chrono types, 2023
- mapping from name to catalog when calling `messages::do_open`, 1676
- mapping header name to header or external source file, 19
- mapping input file characters to translation character set, 13, 2431
- mapping of pointer to integer, 149
- mapping to message when calling `messages::do_get`, 1676
- maximum depth of recursive template instantiations, 485
- maximum number of repeated evaluations of a contract assertion, 104
- maximum size of an allocated object, 157, 605
- maximum width for `vec` and `mask`, 1893
- meaning of `'`, `\`, `/*`, or `//` in a *q-char-sequence* or an *h-char-sequence*, 19
- meaning of `asm` declaration, 290
- meaning of attribute declaration, 209
- meaning of dot-dot in *root-directory*, 2138
- method by which contract termination occurs, 103
- nesting limit for `#include` directives, 529
- NTCTS in `basic_ostream<charT, traits>& operator<<(nullptr_t)`, 2080
- number of placeholders for bind expressions, 918, 936
- number of threads in a program under a freestanding implementation, 94
- operating system on which implementation depends, 2130

- `operator<<` for large extended floating-point types, 2079
- `operator>>` for large extended floating-point types, 2067
- ordinary and wide literal encodings, 15
- parallelism flag for an implementation-defined execution policy, 2382
- parameters to `main`, 98
- passing argument of class type through ellipsis, 142
- presence and meaning of `native_handle_type` and `native_handle`, 2180
- presence and value of `__STDC_ISO_10646__`, 546
- rank of extended signed integer type, 90
- replaceability of the contract-violation handler, 104
- representation of floating-point types, 84
- required alignment for `atomic_ref` type's operations, 2210, 2213, 2215, 2218
- resource limits on a message catalog, 1676
- result of evaluating `hash<indirect<T, Allocator>>()(i)` if `i` is valueless, 732
- result of `filesystem::file_size`, 2169
- result of inexact floating-point conversion, 113, 148
- resuming a coroutine on a different execution agent, 630
- return type of `cartesian_product_view::size`, 1410
- return types for `<stdbit.h>` functions, 959
- return value of `bad_alloc::what`, 605
- return value of `bad_any_cast::what`, 884
- return value of `bad_array_new_length::what`, 605
- return value of `bad_cast::what`, 608
- return value of `bad_exception::what`, 613
- return value of `bad_expected_access<E>::what`, 891
- return value of
 - `bad_expected_access<void>::what`, 891
- return value of `bad_function_call::what`, 937
- return value of `bad_optional_access::what`, 868
- return value of `bad_typeid::what`, 608
- return value of `bad_variant_access::what`, 883
- return value of `bad_weak_ptr::what`, 707
- return value of `char_traits<char16_t>::eof`, 1593
- return value of `char_traits<char32_t>::eof`, 1593
- return value of `char_traits<char8_t>::eof`, 1592
- return value of `exception::what`, 613
- return value of `receiver_proxy::try_query`, 2383
- return value of `source_location::current`, 610
- return value of `type_info::name()`, 608
- search for resources during quote resource search, 530
- search for source files during source file search, 528
- semantics of an access through a volatile glvalue, 218
- semantics of `breakpoint`, 670
- semantics of linkage specifiers, 290
- semantics of parallel algorithms invoked with implementation-defined execution policies, 1431
- semantics of
 - `stacktrace_entry::native_handle`, 665
- semantics of `token` parameter and default token value used by `random_device` constructors, 1781
- sequence of places searched for a header, 530
- set of additional thread-attributes, 2193
- set of aliases of a known registered character encoding, 1678
- set of character container types that iostreams templates can be instantiated for, 1640, 2033
- set of enabled `basic_mask<Bytes, Abi>` specializations, 1936
- set of enabled `basic_vec<T, Abi>` specializations, 1911
- set of integer-class types, 1208
- signedness of `char`, 220
- size in bits of a resource, 530
- `sizeof` applied to fundamental types other than `char`, `signed char`, and `unsigned char`, 154
- stack unwinding before invocation of
 - `std::terminate`, 519, 522
- `stacktrace_entry::native_handle_type`, 665
- startup and termination in freestanding environment, 98
- strict total order over pointer values, 5
- string resulting from `__func__`, 268
- subset of extended floating-point types that can be used as `RealType` template arguments, 1763
- subset of integer types that can be used as `IntType` template arguments, 1763
- subset of unsigned integer types that can be used as `UIntType` template arguments, 1763
- subset of values of a trivially copyable type to which a value representation corresponds, 82
- support for extended alignments, 69
- support for *module-import-declarations* with non-C++ language linkage, 290
- supported forms of `#embed` prefix parameters, 525
- supported input files, 13

supported multibyte character encoding rules, 1592

supported *root-names* in addition to any operating system dependent *root-names*, 2137, 2138

text of `__DATE__` when date of translation is not available, 543

text of `__TIME__` when time of translation is not available, 544

the contents provided in the `comment` field of `contract_violation`, 617

the contents provided in the `location` field of `contract_violation`, 618

the result of `display_string_of` and `u8display_string_of`, 791

the result of `std::meta::has_parent` for entities with neither C nor C++ language linkage, 797

the value returned by `std::meta::source_location_of`, 792

threads and program points at which deferred dynamic initialization is performed, 100

total ordering of all types, 627

transcoding of a formatted path when `charT` and `path::value_type` differ, 2150

treatment of whitespace when processing a `#embed` directive, 531

treatment of whitespace when processing a `#include` directive, 529

type of a directory-like file, 2159, 2161

type of `array::const_iterator`, 997

type of `array::iterator`, 997

type of `basic_stacktrace::const_iterator`, 666

type of `basic_stacktrace::difference_type`, 666

type of `basic_stacktrace::size_type`, 666

type of `basic_string::const_iterator`, 1608

type of `basic_string::iterator`, 1608

type of `basic_string_view::const_iterator`, 1595, 1598

type of `default_random_engine`, 1780

type of `deque::const_iterator`, 1000

type of `deque::difference_type`, 1000

type of `deque::iterator`, 1000

type of `deque::size_type`, 1000

type of `flat_map::const_iterator`, 1103

type of `flat_map::iterator`, 1103

type of `flat_multimap::const_iterator`, 1115

type of `flat_multimap::iterator`, 1115

type of `flat_multiset::const_iterator`, 1131

type of `flat_multiset::iterator`, 1131

type of `flat_set::const_iterator`, 1123

type of `flat_set::iterator`, 1123

type of `forward_list::const_iterator`, 1005

type of `forward_list::difference_type`, 1005

type of `forward_list::iterator`, 1005

type of `forward_list::size_type`, 1005

type of `inplace_vector::const_iterator`, 1037

type of `inplace_vector::iterator`, 1037

type of `list::const_iterator`, 1021

type of `list::difference_type`, 1021

type of `list::iterator`, 1021

type of `list::size_type`, 1021

type of `map::const_iterator`, 1044

type of `map::difference_type`, 1044

type of `map::iterator`, 1044

type of `map::size_type`, 1044

type of `match_results::const_iterator`, 1731

type of `multimap::const_iterator`, 1051

type of `multimap::difference_type`, 1051

type of `multimap::iterator`, 1051

type of `multimap::size_type`, 1051

type of `multiset::const_iterator`, 1060

type of `multiset::difference_type`, 1060

type of `multiset::iterator`, 1060

type of `multiset::size_type`, 1060

type of `native_handle_type`, 2113

type of `ptrdiff_t`, 582

type of `ranges::cartesian_product_view::iterator::difference_type`, 1411

type of `regex_constants::error_type`, 1721

type of `regex_constants::match_flag_type`, 1720

type of `set::const_iterator`, 1055

type of `set::difference_type`, 1055

type of `set::iterator`, 1055

type of `set::size_type`, 1055

type of `size_t`, 582

type of `span::iterator`, 1139, 1144

type of `std::bfloat16_t`, 599

type of `std::float128_t`, 599

type of `std::float16_t`, 599

type of `std::float32_t`, 599

type of `std::float64_t`, 599

type of `streamoff`, 2033

type of `syntax_option_type`, 1719

type of `text_encoding::aliases_view::begin()`, 1680

type of `text_encoding::aliases_view::end()`, 1680

type of `unordered_map::const_iterator`, 1065

type of `unordered_map::const_local_iterator`, 1065

type of `unordered_map::difference_type`, 1065

type of `unordered_map::iterator`, 1065

type of `unordered_map::local_iterator`, 1065

type of `unordered_map::size_type`, 1065

type of `unordered_multimap::const_iterator`, 1073

type of `unordered_multimap::const_local_iterator`, 1073

type of `unordered_multimap::difference_type`, 1073

type of `unordered_multimap::iterator`, 1073

type of `unordered_multimap::local_iterator`, 1073

- type of `unordered_multimap::size_type`, 1073
- type of `unordered_multiset::const_iterator`, 1085
- type of `unordered_multiset::const_local_iterator`, 1085
- type of `unordered_multiset::difference_type`, 1085
- type of `unordered_multiset::iterator`, 1085
- type of `unordered_multiset::local_iterator`, 1085
- type of `unordered_multiset::size_type`, 1085
- type of `unordered_set::const_iterator`, 1079
- type of `unordered_set::const_local_iterator`, 1079
- type of `unordered_set::difference_type`, 1079
- type of `unordered_set::iterator`, 1079
- type of `unordered_set::local_iterator`, 1079
- type of `unordered_set::size_type`, 1079
- type of `vector::const_iterator`, 1028
- type of `vector::difference_type`, 1028
- type of `vector::iterator`, 1028
- type of `vector::size_type`, 1028
- type of `vector<bool>::const_iterator`, 1033
- type of `vector<bool>::const_pointer`, 1033
- type of `vector<bool>::difference_type`, 1033
- type of `vector<bool>::iterator`, 1033
- type of `vector<bool>::pointer`, 1033
- type of `vector<bool>::size_type`, 1033
- underlying type for enumeration, 277
- underlying type of `bool`, 83
- underlying type of `char`, 83
- underlying type of `wchar_t`, 83
- unit suffix when `Period::type` is `micro`, 1963
- value of bit-field that cannot represent assigned value, 176
- initializer, 252
- value of *conditional-escape-sequence*, 25
- value of `ctype<char>::table_size`, 1648
- value of `future_errc::broken_promise`, 2274
- value of `future_errc::future_already_retrieved`, 2274
- value of `future_errc::no_state`, 2274
- value of `future_errc::promise_already_satisfied`, 2274
- value of *has-attribute-expression* for non-standard attributes, 526
- value of *has-attribute-expression* for standard attributes, 526
- value of `max_pointer_bits_available`, 694
- value of non-encodable character literal or multicharacter literal, 25
- value of `pow(0,0)`, 1758
- value of result of inexact integer to floating-point conversion, 113
- value of `simd::uninit_element`, 1895
- value of `simd::zero_element`, 1895
- value of `std::execution::counting_scope::max_associations`, 2380
- value of `std::execution::simple_counting_scope::max_associations`, 2378
- value representation of pointer types, 88
- value returned by `pointer_bits_available`, 694
- values of a trivially copyable type, 82
- values of various `ATOMIC_..._LOCK_FREE` macros, 2209
- whether `<cfenv>` functions can be used to manage floating-point status, 1752
- whether a freestanding deleted function is a deleted function, 555
- whether a given `atomic` type's operations are always lock free, 2220, 2221, 2225, 2228, 2231, 2235, 2237
- whether a given `atomic_ref` type's operations are always lock free, 2210, 2213, 2215, 2218
- whether a parameter is destroyed when the function exits or at the end of the enclosing full-expression, 141
- whether an expression is a core constant expression, 180
- whether `basic_spanbuf`'s move source is empty after a move, 2106
- whether `contract_violation` has a virtual destructor, 617
- whether declarations of some members of a closure type *Q* are *Q*-members-of-eligible, 804
- whether freestanding default replaceable global allocation functions satisfy required behavior, 601
- whether freestanding implementations replace the *statement* of a trivial infinite loop with a call to the function `std::this_thread::yield`, 197
- whether functions from Annex K of the C standard library are declared when C++ headers are included, 558
- whether locale object is global or per-thread, 1640
- whether `pragma FENV_ACCESS` is supported, 1752
- whether `rand` may introduce a data race, 1805
- whether runtime-undefined behavior results in the expression being deemed non-constant, 7
- whether sequence pointers are copied by `basic_filebuf` move constructor, 2114
- whether sequence pointers are copied by `basic_stringbuf` move constructor, 2092
- whether sequence pointers are initialized to null pointers, 2091
- whether source file inclusion of importable header is replaced with `import` directive, 529
- whether stack is unwound before invoking the function `std::terminate` when a `noexcept` specification is violated, 522
- whether the implementation is a hardened implementation, 11

- whether the implementation is hosted or freestanding, [559](#)
- whether the sources for module units and header units on which the current translation unit has an interface dependency are required to be available during translation, [14](#)
- whether the thread that executes `main` and the threads created by `std::thread` or `std::jthread` provide concurrent forward progress guarantees, [97](#)
- whether `time_get::do_get_year` accepts two-digit year numbers, [1667](#)
- whether values are rounded or truncated to the required precision when converting between `time_t` values and `time_point` objects, [1968](#)
- which functions in the C++ standard library may be recursively reentered, [577](#)
- which non-standard-layout objects containing no data are considered empty, [66](#)
- which scalar types have unique object representations, [768](#)
- width of integral type, [83](#)